Domain Theory in Lean 4

Lars Warren Ericson[†], **Dana S. Scott**, **Vijay D'Silva**[‡], **Brian Milnes**[§]

October 2026
CMU-CS-26-140

School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213

[†] Catskills Research Company, [‡] Google Research, [§] Unaffiliated

Keywords: Lean 4; formal verification; domain theory; continuous lattices; neighbourhood systems; information systems; Scott topology

Abstract

This document is a machine-checked Lean 4 formalization of Dana Scott's papers on domain theory: continuous lattices (1972), datatypes as lattices (1976), neighbourhood systems (1980), and information systems (1982), together with bridge theorems relating the lattice, neighbourhood, and information systems presentations. Beyond the lattice and retract development, the 1976 material covers Scott's computability apparatus for the universal domain $\mathbf{P}\omega$ (the language LAMBDA, enumeration and recursion, definability, and enumeration degrees as combinatory subalgebras) and a descriptive-set style classification of subsets of $\mathbf{P}\omega$ (open, closed, $\mathfrak{G}_\delta$, Boolean, and related classes characterized as fibres of continuous maps). The library is registered with Palomar as PALOMAR-2026-11-12-000010 (version 1).

Contents

1 Introduction **25**

2 Continuous Lattices (1972) ✓ **26**
2.1 Introduction ✓ . . . 27
2.2 Injective spaces ✓ . . . 28
2.2.1 Definition 1.1: Injective Space ✓ . . . 29
2.2.2 Proposition 1.2 ✓ . . . 30
2.2.3 Proposition 1.3 ✓ . . . 30
2.2.4 Powers of $\mathbb{O}$ are injective ✓ . . . 31
2.2.5 Proposition 1.4 ✓ . . . 32
2.2.6 Proposition 1.5 ✓ . . . 32
2.2.7 RetractSubspace ✓ . . . 33
2.2.8 Corollary 1.6 ✓ . . . 34
2.2.9 Corollary 1.7 ✓ . . . 34
2.2.10 Absolute retracts ✓ . . . 35
2.3 Continuous lattices ✓ . . . 36
2.3.1 Specialization order ✓ . . . 37
2.3.2 Way-below relation ✓ . . . 47
2.3.3 Continuous lattice ✓ . . . 52
2.3.4 Scott maps ✓ . . . 65
2.3.5 Proposition 2.8 ✓ . . . 73
2.3.6 Proposition 2.9 ✓ . . . 74
2.3.7 Proposition 2.10 ✓ . . . 76
2.3.8 Proposition 2.11 ✓ . . . 77
2.3.9 Proof of 2.9 ✓ . . . 78
2.3.10 Continuous lattice retraction ✓ . . . 79
2.3.11 Converse retraction ✓ . . . 81
2.3.12 Continuous lattice projection ✓ . . . 83
2.3.13 Converse projection ✓ . . . 88
2.3.14 Proof of 2.10 ✓ . . . 88
2.3.15 Proof of 2.11 ✓ . . . 90
2.4 Function spaces ✓ . . . 100
2.4.1 Induced pi topology on $C(X, Y)$ ✓ . . . 100
2.4.2 Definition 3.1: Function space ✓ . . . 101
2.4.3 Elements of $[D \to D']$ ✓ . . . 104
2.4.4 Step function (pointwise) ✓ . . . 114
2.4.5 Step map $\overrightarrow{e}[e, e']$ ✓ . . . 115
2.4.6 Proposition 3.2 ✓ . . . 117
2.4.7 Product topology on $[D \to D']$ ✓ . . . 118
2.4.8 Theorem 3.3 ✓ . . . 122
2.4.9 Corollary 3.4 ✓ . . . 123
2.4.10 Scott functional abstraction ✓ . . . 125
2.4.11 Proposition 3.5 ✓ . . . 128

2.4.12 Retractions and extensions ✓ . . . 129
2.4.13 Definition 3.6 ✓ . . . 131
2.4.14 Function-space retraction and projection ✓ . . . 132
2.4.15 Proposition 3.7 ✓ . . . 133
2.4.16 Scott subspace extension ✓ . . . 134
2.4.17 Proposition 3.8 ✓ . . . 137
2.4.18 Lemma 3.9 ✓ . . . 139
2.4.19 Proposition 3.10 ✓ . . . 141
2.4.20 Definition 3.11: Projection space ✓ . . . 145
2.4.21 Proposition 3.12 ✓ . . . 148
2.4.22 Proposition 3.13 ✓ . . . 149
2.4.23 Proposition 3.14 ✓ . . . 151
2.5 Inverse limits ✓ . . . 152
2.5.1 Compatibility ✓ . . . 152
2.5.2 InverseLimit ✓ . . . 153
2.5.3 Projections from the limit ✓ . . . 154
2.5.4 The mediating map ✓ . . . 154
2.5.5 Special inverse systems ✓ . . . 156
2.5.6 Projections along the tower ✓ . . . 156
2.5.7 Tower step adjunction ✓ . . . 159
2.5.8 Embeddings along the tower ✓ . . . 160
2.5.9 Coordinates of an embedding ✓ . . . 162
2.5.10 Embeddings into the limit ✓ . . . 165
2.5.11 Order on the inverse limit ✓ . . . 166
2.5.12 Strict order on the inverse limit ✓ . . . 167
2.5.13 The limit lattice ✓ . . . 169
2.5.14 Topology of the inverse limit ✓ . . . 180
2.5.15 Proposition 4.1 ✓ . . . 184
2.5.16 Proposition 4.2 ✓ . . . 185
2.5.17 Corollary 4.3 ✓ . . . 187
2.5.18 The function space tower . . . 189
2.5.19 The functorial form ✓ . . . 216
2.5.20 Application ✓ . . . 216
2.5.21 Two projections ✓ . . . 217
2.5.22 Embedding every space ✓ . . . 217
2.5.23 Other functors ✓ . . . 218
2.5.24 Subspaces of a limit ✓ . . . 219
2.6 Correction ✓ . . . 219

3 Datatypes as Lattices (1976) ✓ 221
3.1 Introduction . . . 221
3.1.1 Domain $\mathbf{P}\omega$ ✓ . . . 222
3.1.2 Multiple-valued integers ✓ . . . 223
3.2 Continuous functions . . . 224
3.2.1 Introduction ✓ . . . 224
3.2.2 Finite subsets and the enumeration e_n ✓ . . . 225

3.2.3 Cantor pairing of integer pairs (n, m) ✓ . . . 236
3.2.4 Basic open sets and continuity on $\mathbf{P}\omega$ ✓ . . . 240
3.2.5 Graphs of continuous functions ✓ . . . 249
3.2.6 Fixed points and least fixed points . . . 253
3.2.7 Extension and embedding of continuous maps . . . 259
3.3 Computability and definability . . . 268
3.3.1 Minimal and maximal extension ✓ . . . 269
3.3.2 Meaning ✓ . . . 270
3.3.3 The language LAMBDA ✓ . . . 271
3.3.4 Term avoidance (α) ✓ . . . 277
3.3.5 Theorem 2.1 — the continuity theorem ✓ . . . 278
3.3.6 Theorem 2.2 — the conversion theorem ✓ . . . 279
3.3.7 Theorem 2.3 — the reduction theorem ✓ . . . 281
3.3.8 Theorem 2.4 — the combinator theorem ✓ . . . 282
3.3.9 The combinator Y ✓ . . . 283
3.3.10 Theorem 2.5 — the first recursion theorem ✓ . . . 285
3.3.11 The graph model ✓ . . . 286
3.3.12 Sequences ✓ . . . 288
3.3.13 Distributive functions ✓ . . . 295
3.3.14 The definability theorem ✓ . . . 303
3.3.15 A law of Y ✓ . . . 321
3.4 Enumeration and degrees . . . 323
3.4.1 Introduction ✓ . . . 323
3.4.2 G-combinator tools ✓ . . . 325
3.4.3 Theorem 3.1 ✓ . . . 335
3.4.4 Application ✓ . . . 337
3.4.5 Unpairing ✓ . . . 339
3.4.6 Numerals ✓ . . . 341
3.4.7 Val rank ✓ . . . 342
3.4.8 Theorem 3.2 ✓ . . . 343
3.4.9 Recursion codes ✓ . . . 344
3.4.10 Plotkin helpers ✓ . . . 345
3.4.11 Theorem 3.3 ✓ . . . 347
3.4.12 Theorem 3.4 ✓ . . . 349
3.4.13 Theorem 3.5 ✓ . . . 351
3.4.14 Theorem 3.6 ✓ . . . 356
3.4.15 Theorem 3.7 ✓ . . . 359
3.5 Retracts and data types . . . 362
3.5.1 Introduction ✓ . . . 362
3.5.2 Graph lemmas ✓ . . . 364
3.5.3 Predecessor set ✓ . . . 364
3.5.4 Fixpoints ✓ . . . 365
3.5.5 Directed fixpoints ✓ . . . 366
3.5.6 Definition (retract) ✓ . . . 369
3.5.7 Theorem 4.1 ✓ . . . 377

3.5.8 u is in the range of retract a ✓ . . . 379
3.5.9 Definition (ordering) ✓ . . . 379
3.5.10 Open-set retract ✓ . . . 380
3.5.11 Theorem 4.2 ✓ . . . 380
3.5.12 Theorem 4.3 ✓ . . . 382
3.5.13 C combinators ✓ . . . 384
3.5.14 Pair mediation ✓ . . . 387
3.5.15 Theorem 4.4 ✓ . . . 387
3.5.16 Theorem 4.5 ✓ . . . 392
3.5.17 Expression interpreter ✓ . . . 395
3.5.18 Theorem 4.6 ✓ . . . 416
3.6 Closure operations and algebraic lattices . . . 421
3.6.1 Introduction ✓ . . . 421
3.6.2 Isolated ✓ . . . 422
3.6.3 Bottom ✓ . . . 422
3.6.4 Square pair ✓ . . . 423
3.6.5 Predecessor identities ✓ . . . 424
3.6.6 Directed finite cover ✓ . . . 425
3.6.7 Is compact ✓ . . . 425
3.6.8 Continuous basis ✓ . . . 426
3.6.9 Countably algebraic ✓ . . . 426
3.6.10 Representation ✓ . . . 429
3.6.11 Closure operation ✓ . . . 430
3.6.12 Theorem 5.1 — the algebraic lattice theorem ✓ . . . 432
3.6.13 Theorem 5.2 — the representation theorem ✓ . . . 432
3.6.14 Theorem 5.3 — the function space theorem ✓ . . . 434
3.6.15 Theorem 5.4 — the product and sum theorem ✓ . . . 435
3.6.16 Theorem 5.5 — the universe theorem ✓ . . . 436
3.6.17 Theorem 5.6 — the limit theorem ✓ . . . 437
3.7 Subsets and their classification . . . 439
3.7.1 Introduction ✓ . . . 439
3.7.2 Scott classes ✓ . . . 440
3.7.3 Characteristic maps ✓ . . . 442
3.7.4 Odd-even packing ✓ . . . 446
3.7.5 Pair indexing ✓ . . . 449
3.7.6 F∩Gδ packing ✓ . . . 451
3.7.7 Three-valued ✓ . . . 453
3.7.8 Boolean membership ✓ . . . 461
3.7.9 Theorem 6.1 — the G theorem ✓ . . . 464
3.7.10 Theorem 6.2 — the F theorem ✓ . . . 465
3.7.11 Theorem 6.3 — the Gδ theorem ✓ . . . 467
3.7.12 Theorem 6.4 — the F∩G theorem ✓ . . . 468
3.7.13 Theorem 6.5 — the F∩Gδ theorem ✓ . . . 471
3.7.14 Theorem 6.6 — the B theorem ✓ . . . 473
3.7.15 Theorem 6.7 — the Bδ theorem ✓ . . . 475

3.8 Total functions and functionality . . . 484
3.8.1 Introduction ✓ . . . 484
3.8.2 Pair elements ✓ . . . 485
3.8.3 Restricted equivalence relations ✓ . . . 486
3.8.4 Theorem 7.1 ✓ . . . 491
3.8.5 Theorem 7.2 ✓ . . . 492
3.8.6 Theorem 7.3 ✓ . . . 497
3.8.7 Theorem 7.4 ✓ . . . 504
3.9 Acknowledgements and references . . . 516

4 Neighborhood Systems (1980) ✓ 519
4.1 Introduction . . . 519
4.2 Domains given by neighbourhoods . . . 521
4.2.1 Introduction . . . 522
4.2.2 Definition 1.1 — neighbourhood system . . . 522
4.2.3 Finite intersections . . . 523
4.2.4 Empty intersection is delta . . . 526
4.2.5 Recursive finite intersection . . . 526
4.2.6 Element order and sequence limits . . . 527
4.2.7 Total elements . . . 528
4.2.8 Two-token neighbourhoods . . . 528
4.2.9 Example 1.2 — two-token domain . . . 531
4.2.10 Three-token neighbourhoods . . . 533
4.2.11 Example 1.3 — three-step chain . . . 535
4.2.12 Binary-tree tokens . . . 536
4.2.13 More neighbourhood-system facts . . . 540
4.2.14 Three-set consistency counterexample . . . 554
4.2.15 Ideal neighbourhoods . . . 556
4.2.16 Final-segment orders . . . 559
4.2.17 Greedy enumeration . . . 562
4.2.18 Positive difference systems . . . 566
4.2.19 Rational intervals . . . 568
4.2.20 Cofinite neighbourhoods . . . 570
4.2.21 Flat and stem systems . . . 572
4.2.22 Finite nonempty neighbourhoods . . . 576
4.2.23 Branch sequences . . . 577
4.2.24 Final-segment naturals . . . 579
4.2.25 Example 1.4 — finite binary tree . . . 581
4.2.26 Are two neighbourhoods nested or disjoint? . . . 583
4.2.27 Four-token powerset . . . 584
4.2.28 Example 1.5 — all nonempty subsets . . . 585
4.2.29 Consistent iff nonempty . . . 585
4.2.30 Equivalent sequences with same limit . . . 587
4.2.31 Definition 1.6 — ideal elements . . . 588
4.2.32 Definition 1.7 — principal filter . . . 589
4.2.33 Inclusion reversing principals . . . 590

4.2.34 Principals are one-one . . . 591
4.2.35 Union of principal filters . . . 592
4.2.36 Definition 1.8 — approximation and totality . . . 593
4.2.37 Bottom element . . . 593
4.2.38 Bottom is least . . . 594
4.2.39 Finite filters are principal . . . 594
4.2.40 The cone of finite bit strings . . . 596
4.2.41 Examples 1.2–1.5 (Revisited) — infinite binary neighbourhoods . . . 597
4.2.42 Prefix order . . . 600
4.2.43 Union of finite approximations . . . 602
4.2.44 Definition 1.9 — isomorphic domains . . . 603
4.2.45 Brackets . . . 604
4.2.46 Token system . . . 606
4.2.47 Theorem 1.10 — basic open neighbourhoods . . . 608
4.2.48 Countable intersection of ideal elements . . . 610
4.2.49 Directed union of ideal elements . . . 611
4.2.50 Theorem 1.11 — intersections and directed unions . . . 613
4.2.51 Lecture I exercises . . . 617
4.3 Mappings between domains . . . 632
4.3.1 Introduction . . . 633
4.3.2 Definition 2.1 — approximable mapping . . . 633
4.3.3 Proposition 2.2 — elementwise action . . . 636
4.3.4 Two-token ideal elements . . . 638
4.3.5 Parity scan into truth . . . 642
4.3.6 Example 2.3 — parity of leading zeros . . . 645
4.3.7 Erase a run of ones . . . 647
4.3.8 Example 2.4 — erase a run of ones . . . 650
4.3.9 Identity approximable map . . . 651
4.3.10 Neighborhood systems as category objects . . . 654
4.3.11 Theorem 2.5 — category of approximable maps . . . 655
4.3.12 Proposition 2.6 — identity and composition . . . 657
4.3.13 Order isomorphism as approximable map . . . 657
4.3.14 Theorem 2.7 — isomorphisms are approximable . . . 660
4.3.15 Monotone on principals as approximable map . . . 662
4.3.16 Exercise 2.8 — determined by finite elements . . . 663
4.3.17 Exercise 2.9 — directed union formula . . . 664
4.3.18 Pointwise intersection of approximable maps . . . 665
4.3.19 Exercise 2.10 — intersection of two maps . . . 666
4.3.20 Indexed directed supremum of ideal elements . . . 667
4.3.21 Exercise 2.11 — directed unions of elements . . . 668
4.3.22 Directed supremum of approximable maps . . . 669
4.3.23 Exercise 2.12 — directed unions of maps . . . 671
4.3.24 Continuous maps as approximable . . . 671
4.3.25 Exercise 2.13 — approximable means continuous . . . 675
4.3.26 Neighbourhood correspondence of an order isomorphism . . . 676

4.3.27 Exercise 2.14 — isomorphisms of neighbourhoods . . . 678
4.3.28 One-token Sierpiński neighbourhood system . . . 679
4.3.29 Exercise 2.15 — opens classified by maps . . . 684
4.3.30 Prefixing map and parity equations . . . 684
4.3.31 Exercise 2.16 — prefixing is approximable . . . 687
4.3.32 Uniqueness of the erase-ones map . . . 688
4.3.33 Exercise 2.17 — erase-ones map . . . 690
4.3.34 Insert-zero map and left inverse . . . 691
4.3.35 Exercise 2.18 — doubling binary digits . . . 695
4.3.36 Approximable maps of two arguments . . . 696
4.3.37 Exercise 2.19 — maps of two arguments . . . 700
4.3.38 Powerset neighbourhood system . . . 701
4.3.39 Exercise 2.20 — powerset of the integers . . . 708
4.3.40 Finite and infinite sequence neighbourhood system . . . 708
4.3.41 Exercise 2.21 — finite and infinite sequences . . . 714
4.3.42 Token neighbourhood system of a closed family . . . 715
4.3.43 Exercise 2.22 — domains from closure properties . . . 722
4.4 Domain constructs . . . 723
4.4.1 Introduction . . . 724
4.4.2 Definition 3.1 — product of systems . . . 724
4.4.3 Product element order isomorphism . . . 730
4.4.4 Proposition 3.2 — elements of a product . . . 732
4.4.5 Definition 3.3 — projection mappings . . . 733
4.4.6 Proposition 3.4 — pairing and projections . . . 738
4.4.7 Theorem 3.5 — separate approximability . . . 739
4.4.8 Constant approximable map . . . 740
4.4.9 Lemma 3.6 — constant functions . . . 741
4.4.10 Elementwise substitution . . . 743
4.4.11 Proposition 3.7 — closure under substitution . . . 743
4.4.12 Step neighbourhoods of maps . . . 745
4.4.13 Intersection of range neighbourhoods . . . 747
4.4.14 Least map of a step list . . . 748
4.4.15 Definition 3.8 — function space . . . 751
4.4.16 Proposition 3.9 — consistent step neighbourhoods . . . 753
4.4.17 Function-space element isomorphism . . . 755
4.4.18 Theorem 3.10 — elements are mappings . . . 758
4.4.19 Evaluation combinator . . . 759
4.4.20 Theorem 3.11 — evaluation combinator . . . 761
4.4.21 Currying . . . 762
4.4.22 Theorem 3.12 — currying . . . 767
4.4.23 Pointwise supremum of maps . . . 769
4.4.24 Theorem 3.13 — pointwise order of maps . . . 773
4.4.25 Diagonal map . . . 774
4.4.26 Exercise 3.14 — tagged product of tokens . . . 775
4.4.27 Product isomorphisms . . . 776

4.4.28 Exercise 3.15 — product isomorphisms . . . 779
4.4.29 Infinite sequence system . . . 780
4.4.30 Exercise 3.16 — infinite sequences of elements . . . 788
4.4.31 Embedding binary sequences . . . 789
4.4.32 Exercise 3.17 — embedding binary sequences . . . 794
4.4.33 Sum system . . . 795
4.4.34 Exercise 3.18 — separated sum . . . 807
4.4.35 Induced maps . . . 807
4.4.36 Exercise 3.19 — induced maps on constructs . . . 811
4.4.37 Product functor . . . 813
4.4.38 Exercise 3.20 — sum and product functors . . . 814
4.4.39 Identity of step neighbourhoods . . . 815
4.4.40 Exercise 3.21 — identity of step neighbourhoods . . . 818
4.4.41 Composition combinator . . . 819
4.4.42 Exercise 3.22 — composition combinator . . . 820
4.4.43 Cartesian closed category . . . 822
4.4.44 Exercise 3.23 — cartesian closed category . . . 823
4.4.45 Function-space isomorphisms . . . 823
4.4.46 Exercise 3.24 — function-space isomorphisms . . . 830
4.4.47 Opens form a domain . . . 832
4.4.48 Exercise 3.25 — opens form a domain . . . 835
4.4.49 Conditional combinator . . . 835
4.4.50 Exercise 3.26 — conditional combinator . . . 848
4.4.51 Function space via closure . . . 850
4.4.52 Exercise 3.27 — function space via closure . . . 853
4.4.53 Minimal step function . . . 854
4.4.54 Exercise 3.28 — minimal step function . . . 856
4.5 Fixed points and recursion . . . 857
4.5.1 Introduction . . . 858
4.5.2 Least fixed-point element . . . 859
4.5.3 Theorem 4.1 — least fixed point . . . 862
4.5.4 Fix combinator . . . 863
4.5.5 Theorem 4.2 — fix combinator . . . 868
4.5.6 Number domain . . . 870
4.5.7 Example 4.3 — partial functions on integers . . . 875
4.5.8 Sequence domain . . . 879
4.5.9 Example 4.4 — sequence domain . . . 883
4.5.10 Definition 4.5 — Peano axioms . . . 886
4.5.11 Theorem 4.6 — Peano models isomorphic . . . 888
4.5.12 Fixed point above a seed . . . 890
4.5.13 Exercise 4.7 — fixed point above a seed . . . 893
4.5.14 Iterate-chain supremum . . . 893
4.5.15 Fixed-point induction . . . 895
4.5.16 Exercise 4.8 — fixed-point induction . . . 896
4.5.17 Psi operator . . . 897

4.5.18 Exercise 4.9 — fix as a fixed point . . . 899
4.5.19 Domain above an element . . . 900
4.5.20 Exercise 4.10 — domain above an element . . . 906
4.5.21 Uniform fix uniqueness . . . 906
4.5.22 Exercise 4.11 — characterizing the fix combinator . . . 910
4.5.23 No greatest fixed point . . . 911
4.5.24 Exercise 4.12 — no greatest fixed point . . . 913
4.5.25 Monotone fixed-point operator . . . 914
4.5.26 Exercise 4.13 — avoiding the natural numbers . . . 916
4.5.27 Greatest powerset fixed point . . . 917
4.5.28 Exercise 4.14 — greatest powerset fixed point . . . 919
4.5.29 Maximal fixed points . . . 919
4.5.30 Exercise 4.15 — maximal fixed points . . . 921
4.5.31 Optimal fixed point . . . 921
4.5.32 Exercise 4.16 — optimal fixed point . . . 923
4.5.33 Generated submonoid . . . 924
4.5.34 Exercise 4.17 — generated submonoid . . . 926
4.5.35 Number-domain checks . . . 927
4.5.36 Exercise 4.18 — checking the number domain . . . 929
4.5.37 Peano structure on sequences . . . 929
4.5.38 Exercise 4.19 — Peano axioms for sequences . . . 935
4.5.39 Fix of a composite . . . 936
4.5.40 Exercise 4.20 — fix of a composite . . . 937
4.5.41 Order as a fixed point . . . 938
4.5.42 Exercise 4.21 — order as a fixed point . . . 941
4.5.43 Inductive Peano submodel . . . 942
4.5.44 Exercise 4.22 — inductive subset of a model . . . 944
4.5.45 Unique fixed point . . . 945
4.5.46 Exercise 4.23 — commuting with the iterates . . . 946
4.5.47 Schroeder-Bernstein . . . 947
4.5.48 Exercise 4.24 — Schroeder-Bernstein theorem . . . 949
4.5.49 Unary sequence domain . . . 950
4.5.50 Exercise 4.25 — unary sequence domain . . . 954
4.6 Typed λ-calculus . . . 955
4.6.1 Introduction . . . 955
4.6.2 Theorem 5.1 — terms denote approximable maps . . . 959
4.6.3 Beta conversion . . . 961
4.6.4 Theorem 5.2 — beta conversion . . . 963
4.6.5 Paired fixed point . . . 965
4.6.6 Proposition 5.3 — simultaneous fixed points . . . 968
4.6.7 Parametric fixed point . . . 969
4.6.8 Proposition 5.4 — fixed point under a parameter . . . 972
4.6.9 Table 5.5 — table of combinators . . . 973
4.6.10 Number-domain recursion schemes . . . 979
4.6.11 Infinite-product stack combinators . . . 985

4.6.12 Number-stack argument encoding . . . 988
4.6.13 Partial-recursive scheme realizers . . . 995
4.6.14 Partial recursive λ-definability . . . 1001
4.6.15 Theorem 5.6 — partial recursive functions . . . 1006
4.6.16 Multi-argument λ-abstraction . . . 1008
4.6.17 Exercise 5.7 — currying the lambda notation . . . 1010
4.6.18 Combinatory completeness . . . 1011
4.6.19 Exercise 5.8 — combinatory completeness . . . 1014
4.6.20 Exercise 5.9 — common fixed points . . . 1015
4.6.21 Smash product . . . 1016
4.6.22 Exercise 5.10 — smash product . . . 1033
4.6.23 Diagonal and map stacks . . . 1034
4.6.24 Exercise 5.11 — stack combinators . . . 1037
4.6.25 While combinator . . . 1038
4.6.26 Exercise 5.12 — while combinator . . . 1040
4.6.27 Natural-number pairing . . . 1041
4.6.28 Exercise 5.13 — pairing function on numbers . . . 1044
4.6.29 Powerset isomorphisms . . . 1045
4.6.30 Exercise 5.13 — powerset isomorphisms . . . 1046
4.6.31 Set graphs . . . 1047
4.6.32 Exercise 5.14 — functions as graphs . . . 1050
4.6.33 Language star . . . 1051
4.6.34 Exercise 5.15 — star of a word . . . 1057
4.6.35 Sequence negation and merge . . . 1058
4.6.36 Exercise 5.16 — negation of sequences . . . 1078
4.7 Domain equations . . . 1079
4.7.1 Introduction . . . 1080
4.7.2 Indefinite product . . . 1081
4.7.3 Example 6.1 — indefinite product . . . 1090
4.7.4 seqEqB — B equation . . . 1095
4.7.5 seqEqA — A^n equation . . . 1104
4.7.6 seqEqC — C equation . . . 1115
4.7.7 seqEqRegular — regular trees . . . 1131
4.7.8 Example 6.2 — sequence domain equations . . . 1133
4.7.9 Endofunctor . . . 1135
4.7.10 Definition 6.3 — functor . . . 1136
4.7.11 T-algebra . . . 1137
4.7.12 Definition 6.4 — algebra for a functor . . . 1138
4.7.13 Initial algebra . . . 1139
4.7.14 Definition 6.5 — initial algebra . . . 1141
4.7.15 Initial iso . . . 1141
4.7.16 Proposition 6.6 — uniqueness of initial algebras . . . 1142
4.7.17 Lambek . . . 1143
4.7.18 Proposition 6.7 — functor preserves initiality . . . 1144
4.7.19 Continuous on maps . . . 1145

4.7.20 Definition 6.8 — continuity on maps . . . 1146
4.7.21 Initial homomorphism . . . 1147
4.7.22 Theorem 6.9 — homomorphism from initial algebra . . . 1151
4.7.23 Subdomain . . . 1152
4.7.24 Definition 6.10 — subdomain . . . 1153
4.7.25 Subsystem lattice . . . 1154
4.7.26 Proposition 6.11 — lattice of subsystems . . . 1159
4.7.27 Projection pair library . . . 1160
4.7.28 Proposition 6.12 — projection pair . . . 1162
4.7.29 Continuous on domains . . . 1163
4.7.30 Definition 6.13 — continuity on domains . . . 1164
4.7.31 Exists initial . . . 1165
4.7.32 Theorem 6.14 — existence of initial algebras . . . 1168
4.7.33 Proj pair subdomain . . . 1170
4.7.34 Lemma 6.15 — subdomains from projection pairs . . . 1174
4.7.35 Minimal initial . . . 1175
4.7.36 Theorem 6.16 — initial algebra is minimal . . . 1180
4.7.37 Lecture VI exercises . . . 1192
4.7.38 cAlgebra — C initial algebra . . . 1192
4.7.39 cAlgebraGen — Cn over A . . . 1203
4.7.40 Exercise 6.17 — algebras for sequence domains . . . 1233
4.7.41 iterProdInitial — D^∞ initial . . . 1234
4.7.42 Exercise 6.18 — infinite product as initial . . . 1244
4.7.43 sumProdTok — strict sum and product . . . 1244
4.7.44 Exercise 6.19 — strict sum and product . . . 1255
4.7.45 functorTokens — FExpr functor algebra . . . 1256
4.7.46 Exercise 6.20 — tokens of a functor . . . 1269
4.7.47 coalesceSmash — coalesced sum and smash . . . 1269
4.7.48 Exercise 6.21 — coalesced sum and smash . . . 1289
4.7.49 intExprDomains — integer equations . . . 1289
4.7.50 Exercise 6.22 — equations for the integers . . . 1308
4.7.51 exprDomain — syntactic Exp . . . 1308
4.7.52 Exercise 6.23 — syntactic expression domain . . . 1310
4.7.53 recDomainEq — simultaneous solutions . . . 1311
4.7.54 Exercise 6.24 — recursive domain equations . . . 1314
4.7.55 projPairLaws — projection pair laws . . . 1314
4.7.56 Exercise 6.25 — projection pair laws . . . 1316
4.7.57 liftDomain — lifting a domain . . . 1317
4.7.58 Exercise 6.26 — lifting a domain . . . 1332
4.7.59 embeddingCompare — which embeddings hold . . . 1332
4.7.60 Exercise 6.27 — which embeddings hold . . . 1349
4.7.61 finiteEmbedCancel — finite mutual embeddings . . . 1349
4.7.62 Exercise 6.28 — finite embeddings cancel . . . 1352
4.7.63 infSumProd — infinitary sum and product . . . 1352
4.7.64 Exercise 6.29 — infinitary sum and product . . . 1366

4.8 Computability in effectively given domains . . . 1367
4.8.1 Introduction . . . 1368
4.8.2 Recursive predicates . . . 1368
4.8.3 Computable presentation . . . 1439
4.8.4 Definition 7.1 — computable presentation . . . 1443
4.8.5 Computable map . . . 1444
4.8.6 Definition 7.2 — computable mapping . . . 1445
4.8.7 Computable maps form a category . . . 1447
4.8.8 Proposition 7.3 — computable maps form a category . . . 1448
4.8.9 Effective sum . . . 1449
4.8.10 Effective product . . . 1467
4.8.11 Theorem 7.4 — sum and product effective . . . 1471
4.8.12 Function-space presentation . . . 1474
4.8.13 Theorem 7.5 — function space effective . . . 1512
4.8.14 Computable fixpoint . . . 1515
4.8.15 Theorem 7.6 — fix is computable . . . 1517
4.8.16 Effectively given lists . . . 1518
4.8.17 List combinators . . . 1545
4.8.18 Proposition 7.7 — lists effectively given . . . 1551
4.8.19 Powerset neighbourhoods . . . 1552
4.8.20 Example 7.8 — powerset effectively given . . . 1555
4.8.21 Power domain . . . 1556
4.8.22 Definition 7.9 — power domain . . . 1558
4.8.23 Effective powerdomain . . . 1559
4.8.24 Proposition 7.10 — power domain effective . . . 1569
4.8.25 Finite powerdomain elements . . . 1570
4.8.26 Definition 7.11 — finite sets of elements . . . 1573
4.8.27 Finite-join approximation . . . 1574
4.8.28 V-shape counterexample . . . 1582
4.8.29 Proposition 7.12 — finite sets are approximable . . . 1585
4.8.30 Lecture VII exercises . . . 1588
4.8.31 Inclusion structures . . . 1588
4.8.32 Exercise 7.13 — axiomatic neighbourhood structure . . . 1596
4.8.33 Computable witnesses . . . 1597
4.8.34 Exercise 7.14 — primitive recursive witnesses . . . 1602
4.8.35 Scott presentation . . . 1603
4.8.36 Effective smash . . . 1606
4.8.37 Coalesced sum . . . 1620
4.8.38 Effective iterate . . . 1639
4.8.39 Exercise 7.15 — further constructs effective . . . 1646
4.8.40 Curry enumeration . . . 1646
4.8.41 Exercise 7.16 — curry as an enumeration . . . 1651
4.8.42 List recursion . . . 1652
4.8.43 Exercise 7.17 — recursion over lists . . . 1666
4.8.44 Effective isomorphism . . . 1667

4.8.45 Exercise 7.18 — effective isomorphism . . . 1678
4.8.46 Powerdomain functor . . . 1679
4.8.47 Exercise 7.19 — power domain functor . . . 1688
4.8.48 Powerdomain union . . . 1689
4.8.49 Exercise 7.20 — union on power domains . . . 1692
4.8.50 Pointwise application . . . 1693
4.8.51 Exercise 7.21 — distributing the power domain . . . 1697
4.8.52 Regular languages . . . 1698
4.8.53 Regular-language codes . . . 1736
4.8.54 Language presentation . . . 1775
4.8.55 Exercise 7.22 — domain of formal languages . . . 1799
4.8.56 Graph map . . . 1800
4.8.57 Exercise 7.23 — graph and fun computable . . . 1833
4.8.58 LUCID histories . . . 1834
4.8.59 Exercise 7.24 — LUCID history domain . . . 1878
4.9 Retracts of the universal domain . . . 1879
4.9.1 Introduction . . . 1880
4.9.2 Retraction . . . 1881
4.9.3 Definition 8.1 — retraction . . . 1883
4.9.4 Proposition 8.2 — retractions from subdomains . . . 1884
4.9.5 Projection . . . 1885
4.9.6 Definition 8.3 — projections . . . 1887
4.9.7 Fixed neighbourhood . . . 1887
4.9.8 Fix of O . . . 1895
4.9.9 Strict retraction . . . 1905
4.9.10 Smash retraction . . . 1909
4.9.11 Examples 8.4 — check, fade, and smash . . . 1915
4.9.12 Theorem 8.5 — finitary projection test . . . 1917
4.9.13 Sub . . . 1918
4.9.14 SubApprox . . . 1922
4.9.15 DprimeU . . . 1929
4.9.16 Theorem 8.6 — sub combinator . . . 1936
4.9.17 Presented intervals . . . 1938
4.9.18 Universal domain U . . . 1943
4.9.19 Split specification . . . 1947
4.9.20 Atoms of U . . . 1948
4.9.21 Y-sequence . . . 1956
4.9.22 Definition 8.7 — interval presentation of U . . . 1961
4.9.23 Fixed neighbourhood effective . . . 1963
4.9.24 U constructs . . . 1964
4.9.25 Theorem 8.8 — universality of U . . . 1965
4.9.26 Lam combinator . . . 1967
4.9.27 Sum combinator . . . 1968
4.9.28 Product combinator . . . 1974
4.9.29 Arrow combinator . . . 1976

4.9.30 Exponential map . . . 1978
4.9.31 Definition 8.9 — constructs on projections . . . 1979
4.9.32 Element iso of projection pair . . . 1982
4.9.33 Proposition 8.10 — constructs preserve projections . . . 1984
4.9.34 Lecture VIII exercises . . . 1985
4.9.35 Dedekind cut projection . . . 1985
4.9.36 Exercise 8.11 — retraction on intervals . . . 1997
4.9.37 Effective isomorphism U and V . . . 1997
4.9.38 Exercise 8.12 — integer presentation of U . . . 1998
4.9.39 Boolean-algebra filters . . . 1999
4.9.40 Generated-by filters . . . 2007
4.9.41 Exercise 8.13 — Boolean-algebra filters . . . 2008
4.9.42 Closure operator domain . . . 2008
4.9.43 Exercise 8.14 — closure operators . . . 2017
4.9.44 Finitely generated subsystems . . . 2017
4.9.45 Subsystem domain . . . 2026
4.9.46 Exercise 8.15 — domain of subsystems . . . 2033
4.9.47 Fixed-neighbourhood finitary order . . . 2033
4.9.48 Exercise 8.16 — order of projections . . . 2035
4.9.49 Embeddings into U . . . 2035
4.9.50 Embeddings into U (more) . . . 2038
4.9.51 Projection pairs . . . 2048
4.9.52 Exercise 8.17 — explicit embeddings into U . . . 2050
4.9.53 Projection combinators . . . 2051
4.9.54 Exercise 8.18 — missing proofs of 8.10 . . . 2051
4.9.55 Sum and product retracts . . . 2052
4.9.56 Exercise 8.19 — closure under sum and product . . . 2057
4.9.57 Diagonal retracts . . . 2058
4.9.58 Exercise 8.20 — self-embeddings of a domain . . . 2059
4.9.59 A-sharp on lists . . . 2060
4.9.60 A-sharp (more) . . . 2071
4.9.61 Exercise 8.21 — lists of a projection . . . 2076
4.9.62 Example C below B . . . 2076
4.9.63 Exercise 8.22 — comparing B and C . . . 2088
4.9.64 Fix operator . . . 2089
4.9.65 Exercise 8.23 — initial algebra inside U . . . 2094
4.9.66 Coupled fixed points . . . 2095
4.9.67 Exercise 8.24 — coupled domain equations . . . 2104
4.9.68 Reflexive domain . . . 2105
4.9.69 Exercise 8.25 — reflexive function space . . . 2113
4.9.70 U apply and U lam . . . 2114
4.9.71 Exercise 8.26 — untyped calculus in U . . . 2116
4.9.72 Polymorphic type . . . 2116
4.9.73 Infinite product on D . . . 2126
4.9.74 Sub on U . . . 2127

4.9.75 Infinite product on U . . . 2130
4.9.76 Exercise 8.27 — infinite cartesian products . . . 2133

5 Information Systems (1982) ✓ 2134
5.1 Introduction . . . 2135
5.2 Information systems . . . 2137
5.2.1 Introduction . . . 2137
5.2.2 Definition 2.1 . . . 2138
5.2.3 Lower-bound entailment . . . 2140
5.2.4 A first example . . . 2140
5.2.5 Interval tokens . . . 2141
5.2.6 Satisfaction . . . 2142
5.2.7 A second example . . . 2144
5.2.8 Partial-function tokens . . . 2144
5.2.9 Partial-function system . . . 2145
5.2.10 A third example . . . 2147
5.2.11 Definition 2.2 . . . 2148
5.2.12 Finite union . . . 2150
5.2.13 Proposition 2.3 . . . 2155
5.3 The elements of a system . . . 2156
5.3.1 Introduction . . . 2157
5.3.2 Definition 3.1 . . . 2158
5.3.3 Closure . . . 2160
5.3.4 Total elements . . . 2161
5.3.5 Top element . . . 2161
5.3.6 Remarks . . . 2163
5.4 Domains as lattices and as topological spaces . . . 2164
5.4.1 Introduction . . . 2165
5.4.2 Binary meet . . . 2166
5.4.3 Factoid 4.1: lattice / binary meets . . . 2168
5.4.4 Family infima . . . 2169
5.4.5 Factoid 4.2: family infs . . . 2171
5.4.6 Finite consistency . . . 2171
5.4.7 Deductive closure . . . 2173
5.4.8 Family union carrier . . . 2174
5.4.9 Family suprema . . . 2175
5.4.10 Join . . . 2176
5.4.11 Factoid 4.3: joins / conditional sups . . . 2177
5.4.12 Chains . . . 2178
5.4.13 Directed families . . . 2179
5.4.14 Directed suprema . . . 2180
5.4.15 Chain suprema . . . 2182
5.4.16 Factoid 4.4: chains & directed sups . . . 2182
5.4.17 Bottom as empty closure . . . 2183
5.4.18 Finite approximants . . . 2184
5.4.19 Factoid 4.5: finite approximants . . . 2186

5.4.20 Scott continuous maps . . . 2186
5.4.21 The basic Scott-opens . . . 2188
5.4.22 Approximable maps as Scott continuous . . . 2189
5.4.23 Factoid 4.6: topology / Scott continuity . . . 2192
5.5 Approximable mappings between domains . . . 2194
5.5.1 Introduction . . . 2195
5.5.2 Definition 5.1: approximable mappings . . . 2196
5.5.3 Element map . . . 2198
5.5.4 Definition 5.2: image / function value . . . 2201
5.5.5 Proposition 5.3: element maps / pointwise comparison . . . 2202
5.5.6 Identity map . . . 2203
5.5.7 Proposition 5.4: identity mapping . . . 2204
5.5.8 Composition . . . 2205
5.5.9 Proposition 5.5: composition . . . 2206
5.5.10 Constant map . . . 2207
5.5.11 Proposition 5.6: constant maps . . . 2209
5.6 Products of domains . . . 2210
5.6.1 Introduction . . . 2211
5.6.2 Product token predicate . . . 2212
5.6.3 Decidability of product tokens . . . 2212
5.6.4 Product tokens . . . 2213
5.6.5 Equality on product tokens . . . 2214
5.6.6 Product bottom . . . 2214
5.6.7 First-component insert commutes . . . 2215
5.6.8 Second-component insert commutes . . . 2216
5.6.9 Fold membership for first components . . . 2216
5.6.10 Fold membership for second components . . . 2217
5.6.11 First projection of product sets . . . 2218
5.6.12 Second projection of product sets . . . 2221
5.6.13 Product entailment . . . 2224
5.6.14 Definition 6.1: product system . . . 2225
5.6.15 Product library . . . 2231
5.6.16 Left injection . . . 2232
5.6.17 Right injection . . . 2233
5.6.18 Left lift . . . 2234
5.6.19 Right lift . . . 2236
5.6.20 Pairing map . . . 2238
5.6.21 Proposition 6.2: projections and pairing . . . 2240
5.7 Sums of domains . . . 2242
5.7.1 Sum tokens . . . 2243
5.7.2 Equality on sum tokens . . . 2244
5.7.3 Sum bottom . . . 2245
5.7.4 Left-component insert . . . 2245
5.7.5 Right-component insert . . . 2246
5.7.6 Left insert commutes . . . 2247

5.7.7 Right insert commutes . . . 2247
5.7.8 Fold membership for left components . . . 2248
5.7.9 Fold membership for right components . . . 2250
5.7.10 Left projection of sum sets . . . 2251
5.7.11 Right projection of sum sets . . . 2254
5.7.12 Sum consistency . . . 2258
5.7.13 Sum entailment . . . 2259
5.7.14 Definition 6.3: separated sum . . . 2260
5.7.15 Empty sum consistency . . . 2263
5.7.16 Sum library . . . 2264
5.7.17 Left sum lift . . . 2265
5.7.18 Right sum lift . . . 2267
5.7.19 Sum entailment helpers . . . 2269
5.7.20 Sum elements . . . 2272
5.7.21 Left injection map . . . 2275
5.7.22 Right injection map . . . 2277
5.7.23 Copairing map . . . 2279
5.7.24 Proposition 6.4: injections and copairing . . . 2286
5.8 The function space as a domain . . . 2287
5.8.1 Introduction . . . 2288
5.8.2 Definition 7.1: function space as an information system . . . 2289
5.8.3 Lift a pair of consistent sets . . . 2303
5.8.4 Pair of elements . . . 2308
5.8.5 Approximable map as element . . . 2314
5.8.6 Curry map . . . 2324
5.8.7 Theorem 7.2: elements, apply, and curry . . . 2337
5.8.8 Theorem 7.3: least fixed-point operator . . . 2340
5.8.9 Proposition 7.4: equational uniqueness of fix . . . 2341
5.9 Some domain equations . . . 2347
5.9.1 Introduction . . . 2348
5.9.2 Tree information system . . . 2348
5.9.3 Example 8.1: A domain of trees or S-expressions . . . 2366
5.9.4 Lambda information system . . . 2370
5.9.5 Example 8.2: A domain for λ-calculus . . . 2421
5.9.6 Universal information system . . . 2422
5.9.7 Example 8.3: A universal domain . . . 2436
5.9.8 Approximations of domains . . . 2438
5.9.9 Example 8.4: A domain of domains . . . 2445

6 Bridge Theorems 2447
6.1 Introduction . . . 2448
6.2 Basic opens . . . 2451
6.2.1 The basic open of the empty token set . . . 2452
6.2.2 Intersection of a singleton basic open . . . 2453
6.2.3 Membership in a singleton basic open . . . 2454
6.2.4 Entailment from an inclusion of basic opens . . . 2454

6.2.5 Consistency from an inclusion of basic opens . . . 2455
6.2.6 The neighbourhood system of basic opens . . . 2456
6.2.7 Basic opens generated by singleton tokens . . . 2457
6.2.8 An element as a filter of basic opens . . . 2458
6.2.9 A filter of basic opens as an element . . . 2459
6.2.10 Membership in a basic open . . . 2460
6.2.11 Return from filters of basic opens to elements . . . 2461
6.2.12 Return from elements to filters of basic opens . . . 2462
6.2.13 Information-system elements and filters of basic opens . . . 2463
6.2.14 The neighbourhood system induced by an information system . . . 2464
6.3 Neighbourhood bases . . . 2464
6.3.1 A neighbourhood basis . . . 2465
6.3.2 Intersection of basic neighbourhoods . . . 2466
6.3.3 The empty intersection is the master set . . . 2466
6.3.4 Neighbourhoods sit inside the master set . . . 2467
6.3.5 Inserting into an intersection of neighbourhoods . . . 2467
6.3.6 A singleton intersection is one neighbourhood . . . 2468
6.3.7 A lower bound of the intersection . . . 2468
6.3.8 Membership in an intersection of neighbourhoods . . . 2469
6.3.9 Intersections reverse inclusion of index sets . . . 2470
6.3.10 Intersections sit inside the master set . . . 2471
6.3.11 An intersection is contained in each of its neighbourhoods . . . 2471
6.3.12 A filter meets the intersections of its neighbourhoods . . . 2472
6.3.13 The information system of a neighbourhood basis . . . 2472
6.3.14 A neighbourhood filter as an information-system element . . . 2474
6.3.15 An information-system element as a neighbourhood filter . . . 2474
6.3.16 The carrier of a filter read as an element . . . 2476
6.3.17 Membership in the filter of an element . . . 2476
6.3.18 Return from information-system elements to filters . . . 2477
6.3.19 Return from filters to information-system elements . . . 2478
6.3.20 Neighbourhood filters and information-system elements . . . 2478
6.3.21 The information system induced by a neighbourhood basis . . . 2479
6.4 Ideals of finite elements . . . 2480
6.4.1 Compact elements . . . 2481
6.4.2 Algebraic lattices . . . 2482
6.4.3 Finite elements of an information system . . . 2482
6.4.4 The bottom element is finite . . . 2483
6.4.5 The bottom finite element . . . 2484
6.4.6 Finite approximants of an element . . . 2484
6.4.7 Finite approximants are directed . . . 2485
6.4.8 Finite approximants of a smaller element . . . 2485
6.4.9 Finite approximants are nonempty . . . 2486
6.4.10 An element bounds its finite approximants . . . 2487
6.4.11 Finite approximants form a lower set . . . 2487
6.4.12 The carrier of an ideal of finite elements . . . 2488

6.4.13 Carriers of ideals are nonempty . . . 2489
6.4.14 Carriers of ideals are directed . . . 2489
6.4.15 The ideal of finite approximants . . . 2490
6.4.16 The element determined by an ideal . . . 2491
6.4.17 Ideals of approximants are monotone . . . 2491
6.4.18 Members of an ideal lie below its element . . . 2492
6.4.19 Return from ideals to their elements . . . 2492
6.4.20 Return from elements to their ideals . . . 2493
6.4.21 Information-system elements and ideals of finite elements . . . 2494
6.4.22 The ideal completion of an information system . . . 2495
6.4.23 Membership in the finite approximants . . . 2496
6.4.24 Membership in the ideal of approximants . . . 2496
6.4.25 An algebraic lattice is a continuous lattice . . . 2497
6.5 Round way-below filters . . . 2498
6.5.1 The way-below open of a point . . . 2499
6.5.2 The way-below open of the bottom . . . 2500
6.5.3 Way-below opens reverse the order . . . 2501
6.5.4 Intersection of way-below opens . . . 2501
6.5.5 Way-below opens as a neighbourhood system . . . 2502
6.5.6 Filters of way-below opens . . . 2503
6.5.7 Round filters . . . 2503
6.5.8 The type of round filters . . . 2504
6.5.9 The principal round filter of a point . . . 2504
6.5.10 Principal filters are monotone . . . 2505
6.5.11 Points of a principal filter . . . 2506
6.5.12 Principal filters are round . . . 2506
6.5.13 Codes of a round filter . . . 2507
6.5.14 The point recovered from a round filter . . . 2508
6.5.15 Recovery after taking the principal filter . . . 2508
6.5.16 The bottom belongs to the codes . . . 2509
6.5.17 Codes of a round filter are nonempty . . . 2509
6.5.18 Codes of a round filter are directed . . . 2510
6.5.19 A round filter consists of way-below opens . . . 2510
6.5.20 A round filter is the principal filter of its point . . . 2511
6.5.21 The embedding into round filters . . . 2512
6.5.22 Round filters of a continuous lattice . . . 2513
6.5.23 The neighbourhood system of a continuous lattice . . . 2514
6.5.24 Membership in the codes of a round filter . . . 2515
6.5.25 Codes of a round filter are a lower set . . . 2515
6.5.26 Packaging the round-filter isomorphism . . . 2516
6.6 The round information system . . . 2516
6.6.1 The way-below neighbourhood basis . . . 2517
6.6.2 Round elements of the induced information system . . . 2518
6.6.3 The type of round information-system elements . . . 2519
6.6.4 Round filters and round information-system elements . . . 2519

6.6.5 Passing from round filters to round information-system elements . . . 2520
6.6.6 Round information-system elements and their ideals . . . 2520
6.6.7 A continuous lattice and its round information system . . . 2521
6.6.8 A continuous lattice and the ideals of its round elements . . . 2522
6.6.9 Neighbourhood-basis elements and ideals . . . 2522
6.6.10 Neighbourhood filters and ideals . . . 2523
6.6.11 Existence of the round identification . . . 2524
6.6.12 The way-below basis determines a neighbourhood system . . . 2525
6.7 Products, sums, and the function space . . . 2525
6.7.1 Unpairing a product element . . . 2526
6.7.2 Pairing followed by unpairing . . . 2527
6.7.3 Unpairing followed by pairing . . . 2528
6.7.4 Pairing is monotone . . . 2528
6.7.5 The order on approximable maps . . . 2529
6.7.6 The order of approximable maps, read on elements . . . 2530
6.7.7 Assembling a separated-sum element . . . 2530
6.7.8 Assembly is monotone . . . 2531
6.7.9 Trichotomy of separated-sum elements . . . 2532
6.7.10 Every separated-sum element arises by assembly . . . 2533
6.7.11 Classifying a separated-sum element . . . 2534
6.7.12 Classification after assembly . . . 2534
6.7.13 The bottom sum element has no left token . . . 2535
6.7.14 The bottom sum element has no right token . . . 2536
6.7.15 Left tokens of a right singleton . . . 2536
6.7.16 Right tokens of a left singleton . . . 2537
6.7.17 The left bottom token in a left injection . . . 2538
6.7.18 The right bottom token in a right injection . . . 2539
6.7.19 Left injection of an element . . . 2540
6.7.20 Right injection of an element . . . 2541
6.7.21 Left injection of elements is injective . . . 2542
6.7.22 Right injection of elements is injective . . . 2543
6.7.23 Assembly is injective . . . 2544
6.7.24 Assembly after classification . . . 2545
6.7.25 The order of left injections . . . 2546
6.7.26 The order of right injections . . . 2547
6.7.27 The product of two information systems . . . 2548
6.7.28 The separated sum of two information systems . . . 2549
6.7.29 The function space of two information systems . . . 2550
6.7.30 Names for the product, sum, and function-space isomorphisms . . . 2551
6.7.31 Unpairing is monotone . . . 2552
6.7.32 Left tokens of a sum element agree . . . 2552
6.7.33 Right tokens of a sum element agree . . . 2553
6.8 Scott maps . . . 2554
6.8.1 Conjugate Scott maps . . . 2554
6.8.2 Scott-continuous maps that agree on elements . . . 2555

6.8.3 The order on Scott-continuous maps . . . 2556
6.8.4 The underlying function of a conjugate Scott map . . . 2557
6.8.5 Evaluation of a conjugate Scott map . . . 2558
6.8.6 The order of conjugate Scott maps on functions . . . 2559
6.8.7 Conjugate Scott maps with the same Scott map . . . 2559
6.8.8 The order on conjugate Scott maps . . . 2560
6.8.9 Conjugation of Scott maps along an order isomorphism . . . 2561
6.8.10 From an approximable map back to a Scott-continuous map . . . 2561
6.8.11 From a Scott-continuous map back to an approximable map . . . 2562
6.8.12 Approximable maps and Scott-continuous maps . . . 2563
6.8.13 Scott maps along the round presentation . . . 2564
6.9 The tree equation . . . 2565
6.9.1 The product of a tree system with itself . . . 2566
6.9.2 Folding a sum token into a tree token . . . 2567
6.9.3 Inserting the unfolding of a tree token . . . 2567
6.9.4 Unfolding commutes with insertion . . . 2568
6.9.5 Inserting the folding of a sum token . . . 2568
6.9.6 Folding commutes with insertion . . . 2569
6.9.7 Folding yields the token or a pair of bottoms . . . 2569
6.9.8 Unfolding after folding a token . . . 2570
6.9.9 Unfolding a set of tree tokens . . . 2571
6.9.10 Folding a set of sum tokens . . . 2572
6.9.11 Fold membership after unfolding . . . 2572
6.9.12 Membership in an unfolded token set . . . 2573
6.9.13 Fold membership after folding . . . 2574
6.9.14 Membership in a folded token set . . . 2574
6.9.15 Unfolding a folded token set . . . 2575
6.9.16 Left tokens of an unfolding . . . 2576
6.9.17 Right tokens of an unfolding . . . 2577
6.9.18 Left pair-bottom entailment . . . 2578
6.9.19 Right pair-bottom entailment . . . 2578
6.9.20 Consistency of an unfolded token set . . . 2579
6.9.21 Tree entailment and sum entailment . . . 2580
6.9.22 Consistency of a folded token set . . . 2582
6.9.23 Sum entailment after folding . . . 2584
6.9.24 A closed element contains both pair bottoms . . . 2586
6.9.25 Closed tree elements are saturated for unfolding . . . 2587
6.9.26 Unfolding a tree element . . . 2588
6.9.27 Folding a sum-of-product element . . . 2589
6.9.28 Unfolding after folding an element . . . 2590
6.9.29 Folding after unfolding an element . . . 2591
6.9.30 A tree domain and the separated sum of its square . . . 2591
6.10 Illustrative example . . . 2592
6.10.1 The tree domain in three presentations . . . 2594
6.10.2 The lower-bound system of the natural numbers . . . 2596

6.10.3 The tree information system . . . 2597
6.10.4 The sum-of-product information system . . . 2597
6.10.5 Singleton atom elements . . . 2598
6.10.6 The tree equation as an order isomorphism . . . 2598
6.10.7 The semantic factor of the separated sum . . . 2599
6.10.8 Neighbourhood filters, ideals, and the identity on the tree domain . . . 2599
6.10.9 The right-hand side is a separated sum of a product . . . 2600
6.10.10 An atom belongs to its singleton closure . . . 2601
6.10.11 Unfolding an atom into the left summand . . . 2601
6.10.12 Unfolding along the tree equation . . . 2602
6.10.13 Folding along the tree equation . . . 2602
6.10.14 Existence of the tree-equation isomorphism . . . 2603
6.10.15 Existence of the neighbourhood-filter isomorphism . . . 2603
6.10.16 Existence of the ideal isomorphism . . . 2603
6.10.17 The identity map on tree elements . . . 2604

7 Acknowledgments **2605**
7.1 AI-assisted development . . . 2605
7.2 Artifact availability and reproducibility . . . 2605

8 References **2606**

List of Figures

1 Introduction. . . . 25
2 Continuous Lattices (1972) ✓. . . . 27
3 Injective spaces ✓. . . . 28
4 Injective extension: X subspace of Y, continuous maps f from X to D and f bar from Y to D. 29
5 Extension triangle: X to Y by e, f from X to O, dashed f bar from Y to O. . . . 36
6 Continuous lattices ✓. . . . 37
7 Specialization order ✓. . . . 37
8 Way-below relation ✓. . . . 47
9 Continuous lattice ✓. . . . 52
10 Scott maps ✓. . . . 65
11 Function spaces ✓. . . . 100
12 D arrow i to D prime, arrow j back; solid f from D to D double prime; dashed arrow from D prime to D double prime labeled f tilde equals f compose j 130
13 Lemma 3.9 diagram. Top: X to Y by e. Left: f down to D. Right: g bar down to D prime. Diagonals cross: g from X to D prime, f bar from Y to D. Bottom: i from D to D prime and j back. . . . 139
14 Inverse limits ✓. . . . 152
15 The limit lattice ✓. . . . 169
16 The function space tower. . . . 189
17 The projection on a function space. . . . 193
18 Proof that the limit and its function space are inverse. . . . 197
19 Two j0 projections; D0 on rows 2-3; arrows touch circles. . . . 217
20 Push-out square: D and D prime map up into D plus D prime, and 0 maps left to D and up to D prime. . . . 218
21 Datatypes as Lattices (1976) ✓. . . . 221
22 Continuous functions. . . . 224
23 Finite subsets and the enumeration e_n ✓. . . . 227
24 Basic open sets and continuity on $\mathbf{P}\omega$ ✓. . . . 240
25 Graphs of continuous functions ✓. . . . 250
26 Fixed points and least fixed points. . . . 253
27 Extension and embedding of continuous maps. . . . 264
28 Computability and definability. . . . 269
29 The definability theorem ✓. . . . 303
30 Enumeration and degrees. . . . 323
31 G-combinator tools ✓. . . . 325
32 Retracts and data types. . . . 362
33 Directed fixpoints ✓. . . . 366
34 Definition (retract) ✓. . . . 369
35 Closure operations and algebraic lattices. . . . 421
36 Subsets and their classification. . . . 439
37 Total functions and functionality. . . . 484
38 Neighborhood Systems (1980) ✓. . . . 519
39 Diagram 7 521

40 Example 1.2 — two-token domain. . . . 532
41 Example 1.3 — three-step chain. . . . 535
42 Binary tree of tokens with neighbourhood U-curves 582
43 A tree 614
44 A rough picture 615
45 A lattice 616
46 Lean module dependencies for Lecture I exercises. 617
47 Diagram 12 632
48 Diagram 13 724
49 Diagram 14 858
50 Diagram 15 955
51 Diagram 16 1080
52 Diagram 17 1367
53 Diagram 18 1880
54 Diagram 19 2134
55 Diagram 20 2137
56 Diagram 21 2157
57 Diagram 22 2165
58 Diagram 23 2195
59 Diagram 24 2211
60 Diagram 25 2243
61 Diagram 26 2288
62 Diagram 27 2347
63 Diagram 28 2447
64 Diagram 29 2449
65 Diagram 30 2452
66 Diagram 31 2465
67 Diagram 32 2481
68 Diagram 33 2499
69 Diagram 34 2517
70 Diagram 35 2526
71 Diagram 36 2554
72 Diagram 37 2566
73 Diagram 38 2593

1 Introduction

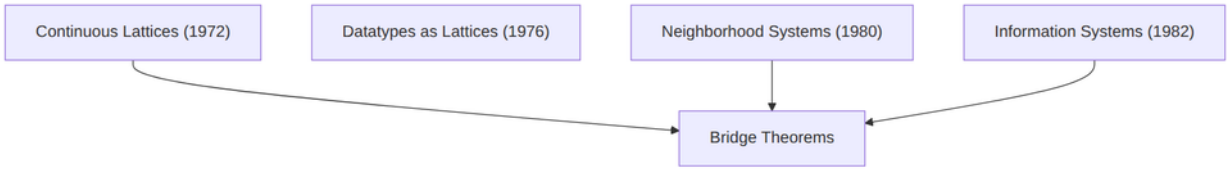


Figure 1: Introduction.

This combined library is registered with Palomar as **PALOMAR-2026-11-12-000010**, version 1 [**?**].

DomainTheory is a single Lean 4 / Mathlib library (v4.36.0-rc2, `lake build` green, no `sorry`) assembled from five prior Palomar-registered formalizations, each with a public GitHub home under `catskillsresearch`, and vendored here under `vendor/` for provenance (see `PROVENANCE.md` and `vendor/FROZEN.txt`):

- *Continuous Lattices* (1972) — Palomar [**?**], GitHub [**?**], and arXiv preprint [**?**];
- *Data Types as Lattices* (1976) — Palomar [**?**] and GitHub [**?**];
- neighbourhood systems / PRG-19 (1980) — Palomar [**?**] and GitHub [**?**];
- information systems (1982) — Palomar [**?**] and GitHub [**?**];
- cross-presentation bridges (`scott_models`) — Palomar [**?**] and GitHub [**?**].

The working Lean is the golfed `DomainTheory/` tree: `ContinuousLattice` ← 1972, `DataTypesAsLattices` ← 1976, `NeighbourhoodSystems` ← 1980, `InformationSystems` ← 1982, `BridgeTheorems` ← `scott_models`. Comparator (`Challenge.lean` / `Solution.lean` / `comparator.json`) validates (i) the cross-presentation bridges of [**?**] (1972 continuous-lattice / 1980 neighbourhood / 1982 information-system isomorphisms, including $D \simeq_o$ RoundFilter $\simeq_o$ RoundInfoSysElement and the S-expression equation $|T| \simeq_o |A + (T \times T)|$), and (ii) the selected *Data Types as Lattices* (1976) declarations of [**?**]. That 1976 selection is not only the retract and algebraic-lattice development on $\mathbf{P}\omega$: it also covers Scott's computability apparatus for the universal domain (the language LAMBDA, enumeration and recursion, definability, and enumeration degrees as combinatory subalgebras) and the descriptive-set style classification of subsets of $\mathbf{P}\omega$ (open, closed, $\mathfrak{G}_\delta$, Boolean, and related classes characterized as fibres of continuous maps). The golfed library preserves those validated results under Mathlib-style names. Distribution today is the public GitHub repository only; a CMU-CS technical report (CMU-CS-TR) is planned.

Author contributions. Lars Warren Ericson — primary formalization engineer (all Lean 4, AI-assisted proof architecture, repository assembly); Dana S. Scott — author of the original source mathematics (1972–1982), project sponsorship, historical context, and high-level advisory oversight of formalization scope; Vijay D'Silva and Brian Milnes — project initiation, pre-submission review, and advisory feedback on repository formatting and technical report generation. Source-paper authorship is distinct from combined-library co-authorship; Scott, D'Silva, and Milnes did not write the Lean.

2 Continuous Lattices (1972) ✓

Starting from the topological point of view a certain wide class of T_0-spaces is introduced having a very strong extension property for continuous functions with values in these spaces. It is then shown that all such spaces are complete lattices whose lattice structure determines the topology - these are the *continuous lattices* - and every such lattice has the extension property. With this foundation the lattices are studied in detail with respect to projections, subspaces, embeddings, and constructions such as products, sums, function spaces, and inverse limits. The main result of the paper is a proof that every topological space can be embedded in a continuous lattice which is homeomorphic (and isomorphic) to its own function space. The function algebra of such spaces provides mathematical models for the Church-Curry λ-calculus.

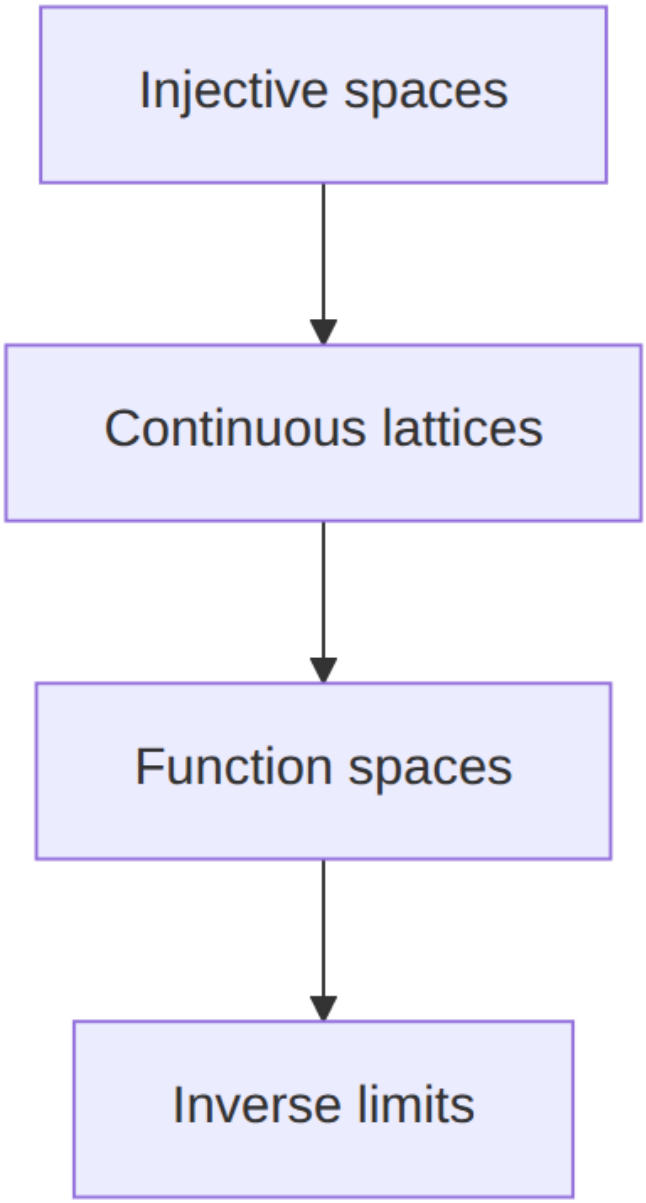


Figure 2: Continuous Lattices (1972) ✓.

2.1 Introduction ✓

Through a roundabout chain of mathematical events I have become interested in T_0-spaces, those topological spaces satisfying the weakest separation axiom to the effect that two distinct points cannot share the same system of open neighborhoods. These spaces seem to have been originally suggested by Kolmogoroff and were introduced first in [13]. Subsequent topology textbooks have dutifully recorded the definition but without much enthusiasm: mainly the idea is introduced to provide exercises. In the book [46] for example,

T_0-spaces are called *feebly semi-separated spaces*, which surely is a term expressing mild contempt. Some interest has been shown in finite T_0-spaces (finite T_1-spaces are necessarily discrete), but generally topology seems to go better under at least the Hausdorff separation axiom. The reason for this is no doubt the strong motivation we get from geometry, where points *are* points and where distinct points *can* be separated.

What I hope to show in this paper is that from a less geometric point of view T_0-spaces can be not only interesting but also natural. The interest for me lies in the construction of *function spaces*, and the main result is the production of a large number of T_0-spaces D such that D and $[D \to D]$ are *homeomorphic*. Here $[D \to D]$ is the space of all continuous functions from D into D with the topology of pointwise convergence (the product topology). It will be shown that every space can be embedded in such a space D, and that D can be chosen to have quite strong extension properties for D-valued continuous functions. These properties make D most convenient for applications to logic and recursive function theory, which was the author's original motivation. Some of the facts about these spaces seem to be most easily proved with the aid of some lattice theory, a circumstance that throws new light on the connections between topology and lattices. In fact, the required spaces are at the same time complete lattices whose topology is determined by the lattice structure in a special way, whence my title.

2.2 Injective spaces ✓

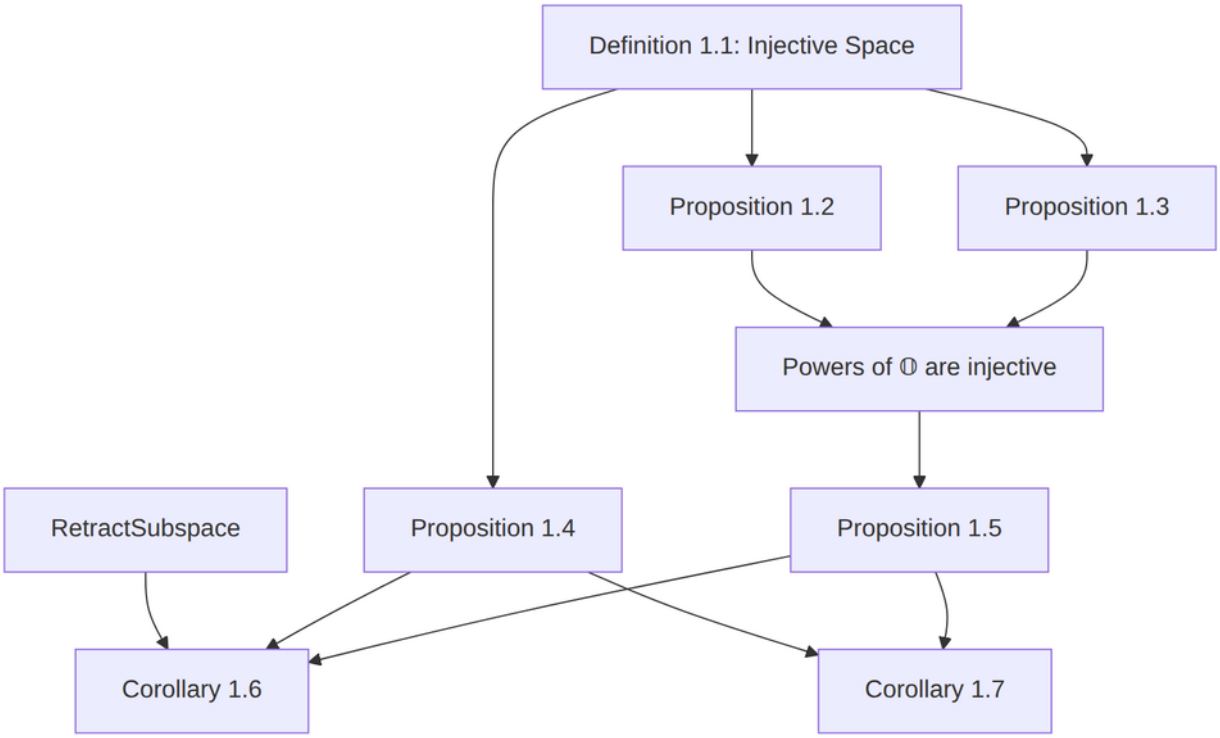


Figure 3: Injective spaces ✓.

All spaces are T_0-spaces, and we begin by defining a class of spaces to be called *injective*.

2.2.1 Definition 1.1: Injective Space ✓

1.1 Definition. A T_0-space D is *injective* iff for arbitrary spaces X and Y if $X \subseteq Y$ as a subspace, then every continuous function $f : X \to D$ can be extended to a continuous function $\bar{f} : Y \to D$. As a diagram we have:

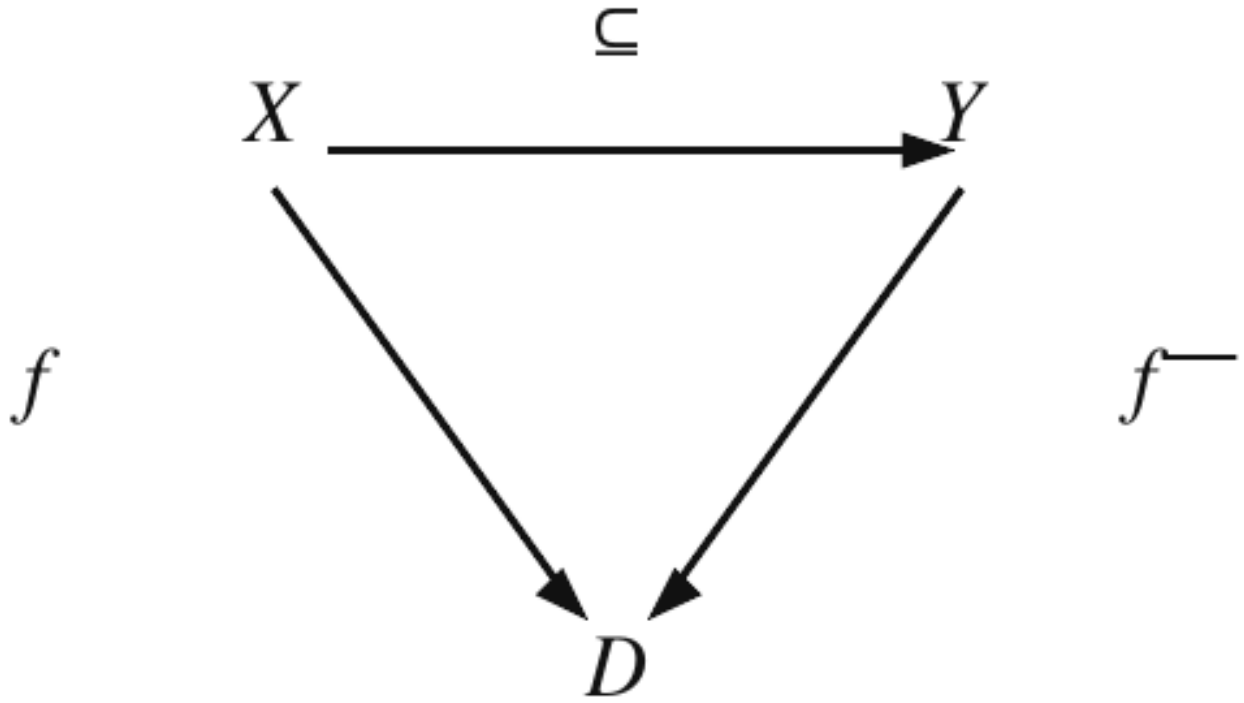


Figure 4: Injective extension: X subspace of Y, continuous maps f from X to D and f bar from Y to D.

Some people will object to this terminology because I use the subspace relationship rather than a monomorphism in the category of T_0-spaces and continuous maps. However, only the trivial 1-point space is injective in the sense of monomorphisms in that category, and so the notion is uninteresting. If the reader prefers another terminology, I do not mind. As we shall see these spaces have very strong retraction properties.

InjectiveSpace.lean

```
module

public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Maps.Basic

@[expose] public section

open Topology TopologicalSpace
```

```
universe u v

class InjectiveSpace (D : Type v) [TopologicalSpace D] : Prop where
  extension :
    ∀ {X Y : Type u} [TopologicalSpace X] [TopologicalSpace Y] (e : X → Y),
      IsEmbedding e → ∀ f : C(X, D), ∃ g : C(Y, D), ∀ x, g (e x) = f x
```

2.2.2 Proposition 1.2 ✓

A slightly less trivial example of an injective space is the 2-point space $\mathbb{O}$ with "points" $\bot$ and $\top$ where $\top$ is open but $\bot$ is not. (This space is sometimes called the *Sierpinski Space*.)

1.2 Proposition. The space $\mathbb{O}$ is injective.

Proof: As is obvious, the continuous maps $f : X \to \mathbb{O}$ are in a one-one correspondence with the open subsets of X (consider $f^{-1}(\top)$). If $X \subseteq Y$ as a subspace, then an open subset of X is the restriction of some open subset of Y. Thus any $f : X \to \mathbb{O}$ can be extended to $\bar{f} : Y \to \mathbb{O}$. □

proposition_1_2.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

open Topology TopologicalSpace

universe u

abbrev Sierpinski := Prop

theorem proposition_1_2 : InjectiveSpace.{u} Sierpinski := by
  constructor
  intro X Y _ _ e he f
  have hfopen : IsOpen {x | f x} := continuous_Prop.1 f.continuous
  rw [he.isInducing.isOpen_iff] at hfopen
  obtain ⟨V, hVopen, hVeq⟩ := hfopen
  refine ⟨⟨fun y => y ∈ V, continuous_Prop.2 hVopen⟩, fun x => ?_⟩
  have : (e x ∈ V) = (x ∈ {x | f x}) := by rw [← hVeq]; rfl
  simpa using this
```

2.2.3 Proposition 1.3 ✓

1.3 Proposition. The Cartesian product of any number of injective spaces is injective under the product topology.

Proof: The argument is standard. A map into the product can be projected onto each of the factors. Each of these projections can be extended. Then the separate maps can be put together again to make the required extended map into the product. □

We now have a large number of injective spaces, and further examples could be found using the next fact.

proposition_1_3.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Constructions
public import Mathlib.Topology.ContinuousMap.Basic

@[expose] public section

open Topology TopologicalSpace

universe u v w

theorem proposition_1_3 {ι : Type w} (D : ι → Type v) [∀ i, TopologicalSpace (D i)]
    (h : ∀ i, InjectiveSpace.{u} (D i)) : InjectiveSpace.{u} (∀ i, D i) := by
  constructor
  intro X Y _ _ e he f
  choose g hg using fun i =>
    (h i).extension e he ⟨fun x => f x i, (continuous_apply i).comp f.continuous⟩
  refine ⟨⟨fun y i => g i y, continuous_pi fun i => (g i).continuous⟩, fun x => ?_⟩
  funext i
  exact hg i x
```

2.2.4 Powers of $\mathbb{O}$ are injective ✓

Lemma. Powers of $\mathbb{O}$ are injective.

Proof. Let I be any index set and let

$$D = \mathbb{O}^I$$

be the Cartesian power of the two-point space $\mathbb{O}$ with the product topology. Each factor $\mathbb{O}$ is injective by **1.2**. By **1.3**, the product of injective spaces is injective under the product topology, so D is injective. □

Proposition 1.5 applies this lemma with $I = \mathfrak{O}$, the open subsets of the given T_0-space.

isInjectiveSpace_sierpinski_power.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Constructions
import DomainTheory.ContinuousLattice.Injective.proposition_1_2
import DomainTheory.ContinuousLattice.Injective.proposition_1_3

@[expose] public section

open Topology TopologicalSpace

universe w

theorem isInjectiveSpace_sierpinski_power (ι : Type w) :
    InjectiveSpace.{u} (ι → Prop) :=
  proposition_1_3 (fun (_ : ι) => Prop) (fun _ => proposition_1_2)
```

2.2.5 Proposition 1.4 ✓

1.4 Proposition. A retract of an injective space is injective.

Proof: Let D be injective. By a retract of D we understand a subspace $D' \subseteq D$ for which there exists a retraction map $j : D \to D'$ such that

$$D' = \{x \in D : j(x) = x\}.$$

Then if $f : X \to D'$ and $X \subseteq Y$, we have $f : X \to D$ as a continuous map also. Taking $\bar{f} : Y \to D$, we have only to form

$$j \circ \bar{f} : Y \to D'$$

to show that D' is injective. □

The relationship between arbitrary T_0-spaces and the injective spaces is given by the embedding theorem.

`proposition_1_4.lean`

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

open Topology TopologicalSpace

universe u v

theorem proposition_1_4 {D : Type v} {D' : Type v} [TopologicalSpace D] [
    TopologicalSpace D']
    (hD : InjectiveSpace.{u} D) (s : C(D', D)) (r : C(D, D')) (hrs : ∀ d, r (s d) = d)
    :
    InjectiveSpace.{u} D' := by
  constructor
  intro X Y _ _ e he f
  obtain ⟨g, hg⟩ := hD.extension e he (s.comp f)
  refine ⟨r.comp g, fun x => ?_⟩
  show r (g (e x)) = f x
  rw [hg x]
  exact hrs (f x)
```

2.2.6 Proposition 1.5 ✓

1.5 Proposition. Every T_0-space can be embedded in an injective space; in fact, in a Cartesian power of the 2-element space $\mathbb{O}$.

Proof: The proof is well known (cf. [46], Theorem 26B.9, p. 484.) But we give the argument for completeness sake. Let X be the given space, and let $\mathfrak{O}$ be the class of open subsets of X. Let

$$D = \mathbb{O}^{\mathfrak{O}}$$

be the Cartesian power of $\mathbb{O}$. Then D is injective by 1.3. Define the map $e : X \to D$ by:

$$e(x)(U) = \begin{cases} \top & \text{if } x \in U, \\ \bot & \text{if } x \notin U, \end{cases}$$

for $x \in X$ and $U \in \mathfrak{O}$. This map e is continuous in view of the topology given to $\mathbb{O}$ and to D. The map e is one-one, because X is T_0. Finally, if $U \subseteq X$ is open, then

$$e(U) = e(x) : x \in U \quad = e(x) : e(x)(U) = \top = e(X) \cap t \in D : t(U) \in \top,$$

which shows that the image $e(U)$ is open in the subspace $e(X) \subseteq D$. Therefore $e : X \to D$ is an embedding of X as a subspace in D. □

proposition_1_5.lean

```
module

public import Mathlib.Topology.ContinuousMap.T0Sierpinski
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
import DomainTheory.ContinuousLattice.Injective.isInjectiveSpace_sierpinski_power

@[expose] public section

open Topology TopologicalSpace

universe u

theorem proposition_1_5 (X : Type u) [TopologicalSpace X] [T0Space X] :
    InjectiveSpace.{u,u} (Opens X → Prop) ∧
      IsEmbedding (productOfMemOpens X) := by
  refine ⟨?_, productOfMemOpens_isEmbedding X⟩
  exact isInjectiveSpace_sierpinski_power (Opens X)
```

2.2.7 RetractSubspace ✓

Corollary 1.6 characterizes injective spaces as retracts of powers of $\mathbb{O}$. The Lean development packages "`D` is a subspace of a larger space and the inclusion splits by a continuous retraction" as `RetractSubspace D' D`: a continuous section `D' → D` that is an embedding, together with a retraction `D → D'` fixing the subspace pointwise.

Corollary 1.6 then states that `D` is injective if and only if it is such a retract of some power `ι → Prop`.

RetractSubspace.lean

```
module

public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Order.CompleteLattice.Basic

@[expose] public section

universe u
```

```
open Topology TopologicalSpace

structure RetractSubspace (D' D : Type u) [TopologicalSpace D'] [TopologicalSpace D]
    where
  section' : C(D', D)
  isEmbedding_section : IsEmbedding section'
  retraction : C(D, D')
  retraction_section : ∀ d, retraction (section' d) = d
```

2.2.8 Corollary 1.6 ✓

1.6 Corollary. The injective spaces are exactly the retracts of the Cartesian powers of $\mathbb{O}$.

Proof: Such a retract is injective by 1.4. If D is injective, then it is (homeomorphic to) a subspace of a power of $\mathbb{O}$. But since D is injective the identity function on the subspace to itself can be extended to the whole of the power of $\mathbb{O}$ providing the required retraction. □

corollary_1_6.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.RetractSubspace
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Separation.Basic
import DomainTheory.ContinuousLattice.Injective.isInjectiveSpace_sierpinski_power
import DomainTheory.ContinuousLattice.Injective.proposition_1_4
import DomainTheory.ContinuousLattice.Injective.proposition_1_5

@[expose] public section

open Topology TopologicalSpace

universe u

theorem corollary_1_6 (D : Type u) [TopologicalSpace D] [T0Space D] :
    InjectiveSpace.{u,u} D ↔ ∃ (ι : Type u), Nonempty (RetractSubspace D (ι → Prop))
    := by
  constructor
  · intro hD
    obtain ⟨r, hr⟩ := hD.extension (productOfMemOpens D) (proposition_1_5 D).2 (
    ContinuousMap.id D)
    refine ⟨Opens D, ⟨⟨productOfMemOpens D, (proposition_1_5 D).2, r, hr⟩⟩⟩
  · rintro ⟨ι, ⟨R⟩⟩
    exact proposition_1_4 (isInjectiveSpace_sierpinski_power ι) R.section' R.retraction
      R.retraction_section
```

2.2.9 Corollary 1.7 ✓

1.7 Corollary. A space is injective iff it is a retract of every space of which it is a subspace.

Proof: As in the proof of 1.6, this property is obvious for injective spaces. But in view of 1.5 every such space is a retract of a power of $\mathbb{O}$ and hence is injective. □

corollary_1_7.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Separation.Basic
import DomainTheory.ContinuousLattice.Injective.isInjectiveSpace_sierpinski_power
import DomainTheory.ContinuousLattice.Injective.proposition_1_4
import DomainTheory.ContinuousLattice.Injective.proposition_1_5

@[expose] public section

open Topology TopologicalSpace

universe u

theorem corollary_1_7 (D : Type u) [TopologicalSpace D] [T0Space D] :
    InjectiveSpace.{u,u} D ↔
      ∀ {Y : Type u} [TopologicalSpace Y] (e : D → Y), IsEmbedding e →
        ∃ r : C(Y, D), ∀ d, r (e d) = d := by
  constructor
  · intro hD Y _ e he
    obtain ⟨r, hr⟩ := hD.extension e he (ContinuousMap.id D)
    exact ⟨r, hr⟩
  · intro hD
    obtain ⟨r, hr⟩ := hD (productOfMemOpens D) (proposition_1_5 D).2
    exact proposition_1_4 (isInjectiveSpace_sierpinski_power (Opens D))
      (productOfMemOpens D) r hr
```

2.2.10 Absolute retracts ✓

As a result of these very elementary considerations, the injective spaces could be called *absolute retracts*, if one remembers to modify the standard definitions by using *arbitrary* subspaces rather than just *closed* subspaces. Note too that it is easy to show that the only continuous maps $e : X \to Y$ for which the extension property

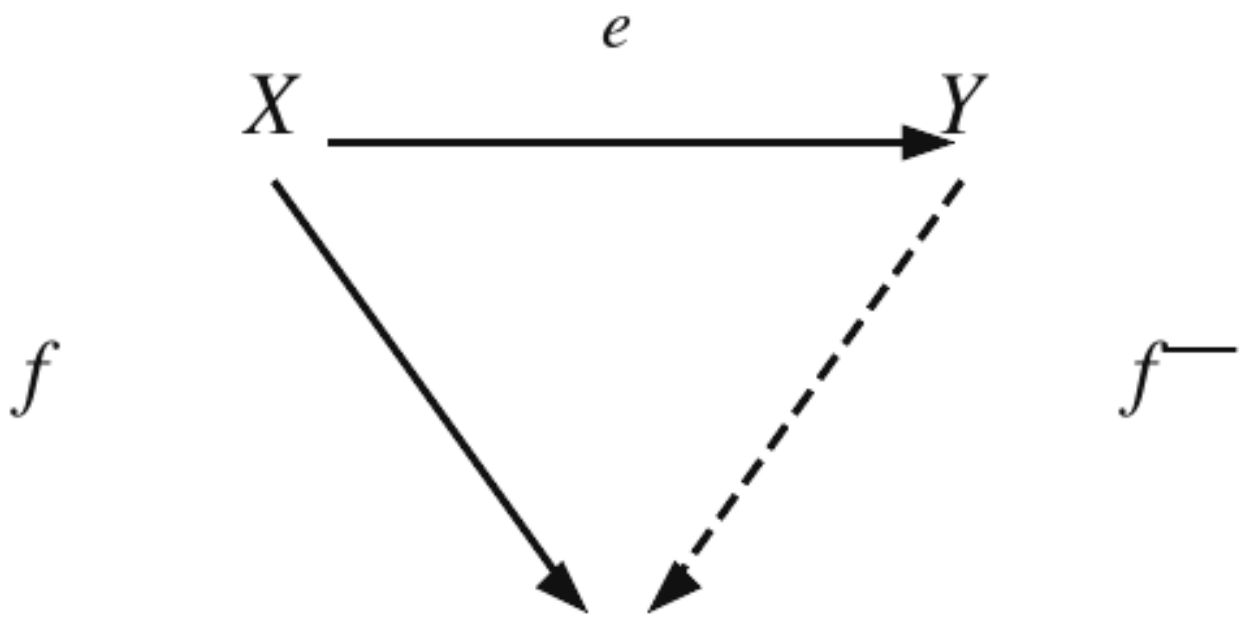


Figure 5: Extension triangle: X to Y by e, f from X to O, dashed f bar from Y to O.

could hold for *all* continuous $f : X \to \mathbb{O}$ are embeddings as subspaces.

Thus it would seem that we have a reasonably good initial grasp of the notion of injective spaces, but further constructions are considerably facilitated by the introduction of the lattice structure.

2.3 Continuous lattices ✓

Scott develops specialization, Scott-open sets, Scott topology, monotone nets, and Proposition 2.1 in Specialization order; the way-below relation, Proposition 2.2, and Definition 2.3 in Way-below relation. Proposition 2.4 through Theorem 2.12, idempotent fixed points, the order-iso and specialization lemmas used in inverse limits, and the post-theorem remarks are in Continuous lattice. Scott maps and Propositions 2.8–2.11, with their proof notes, treat Scott continuity, products, and retractions on continuous lattices on the way to Proposition 2.11 and Theorem 2.12.

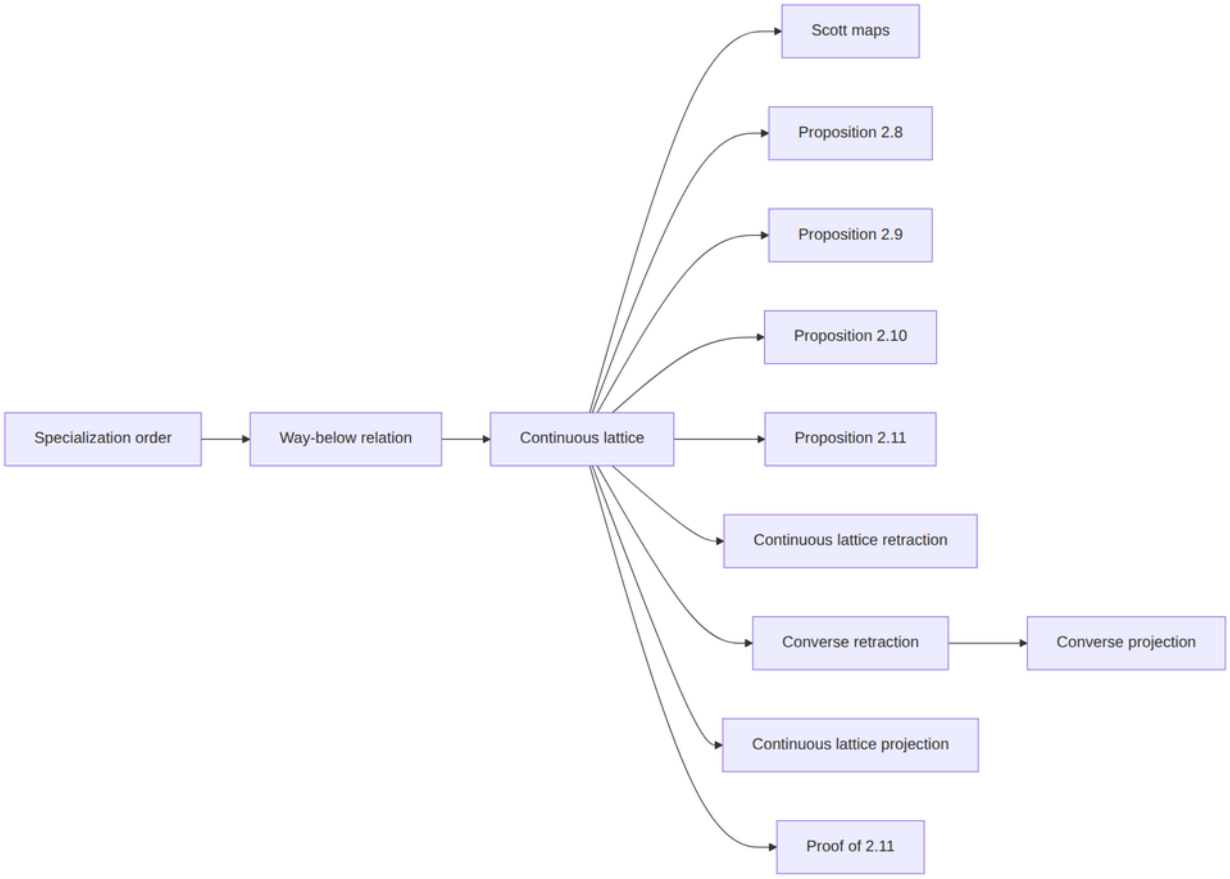


Figure 6: Continuous lattices ✓.

2.3.1 Specialization order ✓

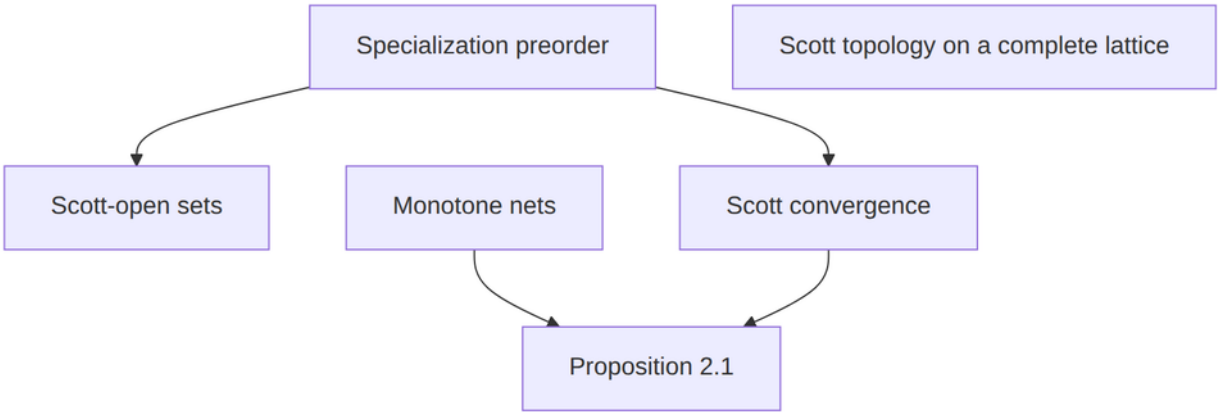


Figure 7: Specialization order ✓.

Specialization preorder Every T_0-space becomes a partially ordered set under the definition:

$$x \sqsubseteq y \text{ iff whenever } x \in U \text{ and } U \text{ is open, then } y \in U.$$

We call relation $\sqsubseteq$ **the specialization preorder** on X. It is reflexive and transitive on every topological space; on a T_0-space it is antisymmetric, and that antisymmetry is exactly the T_0-axiom.

In the converse direction, every partially ordered set $\langle X, \sqsubseteq \rangle$ can be so obtained, for we have only to define $U \subseteq X$ as being open if it satisfies the condition:

(i) whenever $x \in U$ and $x \sqsubseteq y$, then $y \in U$.

The axioms for partial order make X a T_0-space, because for any $y \in X$ the set

$$\{x \in X : x \not\sqsubseteq y\}$$

is open. This connection is not very interesting, however.

The formalization names the same predicate `SpecializationPreorder`. The lemmas below are the library tools used in the inverse-limit specialization characterization and in Theorem 2.12.

Notation. For the specialization preorder above, Mathlib also writes $y \rightsquigarrow x$ when y lies in every open neighborhood of x (the relation `Specializes`, read "y specializes x"). The arrow points from the larger point toward the smaller one; Scott's $x \sqsubseteq y$ is the same comparison with the opposite order of arguments.

Lemma (`SpecializationPreorder.iff_specializes`). For x, y in a topological space, $x \sqsubseteq y$ iff $y \rightsquigarrow x$.

Proof. Both sides mean: whenever U is open and $x \in U$, then $y \in U$. This is `specializes_iff_forall_open`. □

Lemma (`SpecializationPreorder.scott_iff`). On a complete lattice D with the Scott topology `scottTopologicalSpace`, for all $x, y \in D$,

$$x \sqsubseteq y \text{ (specialization preorder)} \iff x \le y \text{ (lattice order)}.$$

Proof. ($\Rightarrow$) Suppose $x \sqsubseteq y$ but $x \not\le y$. The set $U = \{z \in D : z \not\le y\}$ is Scott-open (`ScottOpen.not_le`). We have $x \in U$ since $x \not\le y$, so $y \in U$ by the specialization preorder; that is $y \not\le y$, a contradiction. Hence $x \le y$.

($\Leftarrow$) Suppose $x \le y$ and U is Scott-open with $x \in U$. Since U is an upset, $y \in U$. Thus $x \sqsubseteq y$. □

Lemma (`SpecializationPreorder.map`). If $x \sqsubseteq y$ in X and $f : X \to Y$ is continuous, then $f(x) \sqsubseteq f(y)$ in Y.

Proof. Let $V \subseteq Y$ be open with $f(x) \in V$. Then $x \in f^{-1}(V)$ and $f^{-1}(V)$ is open in X. From $x \sqsubseteq y$ we get $y \in f^{-1}(V)$, hence $f(y) \in V$. □

`SpecializationPreorder.lean`

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
import DomainTheory.ContinuousLattice.WayBelow.ScottOpen

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Topology Set

universe u

variable {X : Type*} {D : Type u} [TopologicalSpace X] [CompleteLattice D]

def SpecializationPreorder (x y : X) : Prop :=
  ∀ U, IsOpen U → x ∈ U → y ∈ U

namespace SpecializationPreorder

theorem iff_specializes {x y : X} :
    SpecializationPreorder x y ↔ y  ~>  x :=
  specializes_iff_forall_open.symm

variable {ι : Type u} [Preorder ι] [IsDirected ι (· ≤ ·)]

variable {x : ι → D} {L y : D}

theorem scott_iff {x y : D} :
    @SpecializationPreorder D scottTopologicalSpace x y ↔ x ≤ y := by
  constructor
  · intro hxy
    by_contra h
    exact hxy {z | ¬ z ≤ y} (ScottOpen.isOpen_iff.mpr (ScottOpen.not_le y)) h (by simp
    )
  · intro hxy U hU hxU
    exact (ScottOpen.isOpen_iff.mp hU).1 hxy hxU

theorem map {Y : Type*} [TopologicalSpace Y] {x y : X}
    (hxy : SpecializationPreorder x y) {f : X → Y} (hf : Continuous f) :
    SpecializationPreorder (f x) (f y) := by
  intro U hU hxU
  exact hxy (f ^-1' U) (hU.preimage hf) hxU

end SpecializationPreorder

end DomainTheory.ContinuousLattice
```

Scott-open sets We shall find that most of the facts about the topology of the spaces we are concerned with here can be expressed in terms of least upper bounds (lubs). It is not always the case, however, that lubs are limits. Thus, for a partially ordered set X, we impose a further restriction on its topology beyond condition (i) for saying when a subset U is open:

(ii) whenever $S \subseteq X$ is *directed*, $\bigsqcup S$ exists, and $\bigsqcup S \in U$, then $S \cap U \neq \emptyset$.

By a directed subset of X we of course mean that it is directed in the sense of the partial ordering $\sqsubseteq$. Note that in this paper directed sets are always *non-empty*.

Scott-open sets are exactly the open sets of `scottTopologicalSpace`.

On a complete lattice D we use the lattice order $\leq$ (for the specialization order on D with the Scott topology, this agrees with $\sqsubseteq$; see `SpecializationPreorder.scott_iff`). Condition (i) is that U

be an *upper set* (an upset): if $a \in U$ and $a \le b$, then $b \in U$. The formalization packages (i) and (ii) for subsets of D as `ScottOpen`. The lemmas below are the library tools that justify that equivalence.

Definition (`ScottOpen`). A subset $U \subseteq D$ is **Scott-open** when U is an upper set and, for every nonempty directed $S \subseteq D$, if $\sup S \in U$ then $S \cap U \neq \emptyset$.

Lemma (`ScottOpen.isUpperSet`). If U is Scott-open, then U is an upper set.

Proof. This is the first component of the definition. □

Lemma (`ScottOpen.inter`). If U and V are Scott-open, then $U \cap V$ is Scott-open.

Proof. $U \cap V$ is an upper set. If $\sup S \in U \cap V$ for directed nonempty S, then $\sup S \in U$ and $\sup S \in V$. By Scott-openness of U and V there are $s_1, s_2 \in S$ with $s_1 \in U$ and $s_2 \in V$. Directedness gives $s_3 \in S$ with $s_3 \ge s_1, s_2$, hence $s_3 \in U \cap V$. □

Lemma (`ScottOpen.univ`). The whole lattice D is Scott-open.

Proof. D is an upper set. If $\sup S \in D$ for nonempty S, any $s \in S$ lies in D. □

Lemma (`ScottOpen.sUnion`). If every member of a family $\mathcal{C}$ of subsets of D is Scott-open, then $\bigcup \mathcal{C}$ is Scott-open.

Proof. A union of upper sets is an upper set. If $\sup S$ lies in some $U \in \mathcal{C}$ with U Scott-open, pick $s \in S \cap U$; then $s \in \bigcup \mathcal{C}$. □

Lemma (`ScottOpen.iff_dirSupInacc`). For $U \subseteq D$,

$$U \text{ is Scott-open} \iff U \text{ is an upper set and } U \text{ is inaccessible under directed sups}$$

(in Mathlib: `IsUpperSet U ∧ DirSupInacc U`).

Proof. Both sides unfold to the same upset condition and the same "if $\sup S \in U$ for directed nonempty S, then $S \cap U \neq \emptyset$" clause, phrased either with `sSup` or with `IsLUB`. □

Lemma (`ScottOpen.preimage_orderIso`). If $e : A \to B$ is an order isomorphism and $U \subseteq B$ is Scott-open, then $e^{-1}(U) \subseteq A$ is Scott-open.

Proof. Upper-set condition pulls back along monotone e. If $\sup S \in e^{-1}(U)$ for directed nonempty $S \subseteq A$, then $\sup(e'' S) = e(\sup S) \in U$; Scott-openness of U gives $s \in S$ with $e(s) \in U$, i.e. $s \in e^{-1}(U)$. □

Lemma (`ScottOpen.isOpen_iff`). With the Scott topology `scottTopologicalSpace` on D, a set $U \subseteq D$ is topologically open iff U is Scott-open.

Proof. Rewrite via `ScottOpen.iff_dirSupInacc` and the definition of `scottTopologicalSpace` as `Topology.scott D Set.univ`; conclude with Mathlib's `IsScott.isOpen_iff_isUpperSet_and_dirSupInaccOn`. □

Lemma (`ScottOpen.not_le`). For any $L \in D$, the set $U = \{z \in D : z \not\le L\}$ is Scott-open.

Proof. If $a \in U$ and $a \le b$, then $b \not\le L$ (otherwise $a \le b \le L$). If $\sup S \in U$ for directed nonempty S but $S \cap U = \emptyset$, then every $s \in S$ satisfies $s \le L$, so $\sup S \le L$, contradicting $\sup S \in U$. □

Lemma (`ScottOpen.wayBelow`). For $x \in D$, the set

$$\{z \in D : \exists U \subseteq D,\ U \text{ Scott-open} \wedge z \in U \wedge U \subseteq \{w : x \le w\}\}$$

is Scott-open. Once the way-below relation $x \ll y$ is defined (`WayBelow`), this set is $\{z : x \ll z\}$; see `WayBelow.lean`.

Proof. If a belongs to a Scott-open U with $U \subseteq \{w : x \le w\}$ and $a \le b$, then $b \in U$ by the upset property, so b is in the same set. If $\sup S$ belongs to the set, pick a witness Scott-open U with $\sup S \in U \subseteq \{w : x \le w\}$. By Scott-openness of U, some $s \in S$ lies in U, and the same U witnesses s in $\{z : x \ll z\}$ in the existential form. □

ScottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

variable {X : Type*} {D : Type u} [TopologicalSpace X] [CompleteLattice D]

def ScottOpen (U : Set D) : Prop :=
  open CompleteLattice in
  letI : LE D := (toCompleteSemilatticeInf (α := D)).toPartialOrder.toPreorder.toLE
  letI : SupSet D := (toCompleteSemilatticeSup (α := D)).toSupSet
  IsUpperSet U ∧
    ∀ {{S : Set D}}, S.Nonempty → DirectedOn (· ≤ ·) S → sSup S ∈ U → (S ∩ U).
    Nonempty

namespace ScottOpen

theorem isUpperSet {U : Set D} (h : ScottOpen U) : IsUpperSet U := h.1

theorem inter {U V : Set D} (hU : ScottOpen U) (hV : ScottOpen V) : ScottOpen (U ∩ V)
    := by
  refine ⟨hU.1.inter hV.1, fun S hS hSdir hmem => ?_⟩
  obtain ⟨s₁, hs₁S, hs₁U⟩ := hU.2 hS hSdir hmem.1
  obtain ⟨s₂, hs₂S, hs₂V⟩ := hV.2 hS hSdir hmem.2
  obtain ⟨s₃, hs₃S, h₁, h₂⟩ := hSdir s₁ hs₁S s₂ hs₂S
  exact ⟨s₃, hs₃S, hU.1 h₁ hs₁U, hV.1 h₂ hs₂V⟩

theorem univ : ScottOpen (Set.univ : Set D) := by
  refine ⟨isUpperSet_univ, fun S hS _ _ => ?_⟩
  obtain ⟨s, hs⟩ := hS
  exact ⟨s, hs, Set.mem_univ s⟩

theorem sUnion {C : Set (Set D)} (hC : ∀ U ∈ C, ScottOpen U) : ScottOpen (⋃₀ C) := by
  refine ⟨isUpperSet_sUnion fun U hU => (hC U hU).1, fun S hS hSdir hmem => ?_⟩
  obtain ⟨U, hUC, hUmem⟩ := hmem
  obtain ⟨s, hsS, hsU⟩ := (hC U hUC).2 hS hSdir hUmem
  exact ⟨s, hsS, U, hUC, hsU⟩

theorem iff_dirSupInacc {U : Set D} : ScottOpen U ↔ IsUpperSet U ∧ DirSupInacc U := by
  constructor
  · intro ⟨hU, hU'⟩
    refine ⟨hU, fun d hd₁ hd₂ a ha hmem => ?_⟩
    rw [← IsLUB.sSup_eq ha] at hmem
    obtain ⟨s, hs, hsU⟩ := hU' hd₁ hd₂ hmem
    exact ⟨s, hs, hsU⟩
  · intro ⟨hU, hU'⟩
    refine ⟨hU, fun d hd₁ hd₂ hmem => ?_⟩
```

```
    obtain ⟨s, hs, hsU⟩ := hU' hd₁ hd₂ (isLUB_sSup d) hmem
    exact ⟨s, hs, hsU⟩

theorem isOpen_iff {U : Set D} : @IsOpen D scottTopologicalSpace U ↔ ScottOpen U := by
  rw [iff_dirSupInacc, scottTopologicalSpace]
  have hinst : @IsScott D (@Set.univ (Set D)) _ (Topology.scott D (@Set.univ (Set D)))
    :=
    @IsScott.mk D (@Set.univ (Set D)) _ (Topology.scott D (@Set.univ (Set D))) rfl
  simpa [dirSupInaccOn_univ] using
    @IsScott.isOpen_iff_isUpperSet_and_dirSupInaccOn D (@Set.univ (Set D)) _
      (Topology.scott D (@Set.univ (Set D))) U hinst

variable {ι : Type u} [Preorder ι] [IsDirected ι (· ≤ ·)]

variable {x : ι → D} {L y : D}

theorem not_le (L : D) : ScottOpen {z : D | ¬ z ≤ L} := by
  refine ⟨fun a b hab ha hb => ha (le_trans hab hb), fun S hSne hSdir hmem => ?_⟩
  by_contra hcon
  refine hmem (sSup_le fun s hs => ?_)
  by_contra hsL
  exact hcon ⟨s, hs, hsL⟩

theorem wayBelow (x : D) :
    ScottOpen {z : D | ∃ U : Set D, ScottOpen U ∧ z ∈ U ∧ U ⊆ Set.Ici x} := by
  refine ⟨fun a b hab ha => ?_, fun S hS hSdir hmem => ?_⟩
  obtain ⟨U, hU, haU, hsub⟩ := ha
  exact ⟨U, hU, hU.1 hab haU, hsub⟩
  obtain ⟨U, hU, hsupU, hsub⟩ := hmem
  obtain ⟨s, hsS, hsU⟩ := hU.2 hS hSdir hsupU
  exact ⟨s, hsS, ⟨U, hU, hsU, hsub⟩⟩

variable {A B : Type u} [CompleteLattice A] [CompleteLattice B]

theorem preimage_orderIso (e : A ≃o B) {U : Set B} (hU : ScottOpen U) :
    ScottOpen (e ⁻¹' U) := by
  refine ⟨fun x y hxy hx => hU.1 (e.monotone hxy) hx, ?_⟩
  intro S hSne hSdir hsup
  have hImgNe : (e '' S).Nonempty := hSne.image e
  have hImgDir : DirectedOn (· ≤ ·) (e '' S) := by
    rintro _ ⟨x, hx, rfl⟩ _ ⟨y, hy, rfl⟩
    obtain ⟨z, hz, hxz, hyz⟩ := hSdir x hx y hy
    exact ⟨e z, Set.mem_image_of_mem e hz, e.monotone hxz, e.monotone hyz⟩
  have hmap : e (sSup S) = sSup (e '' S) := by
    rw [e.map_sSup, sSup_image]
  obtain ⟨_, ⟨s, hs, rfl⟩, hes⟩ := hU.2 hImgNe hImgDir (hmap ▸ hsup)
  exact ⟨s, hs, hes⟩

end ScottOpen

end DomainTheory.ContinuousLattice
```

Scott topology on a complete lattice The sets satisfying (i) and (ii) form the *induced topology* on a partially ordered set X, which is still a T_0-space because the sets $\{x \in X : x \not\sqsubseteq y\}$ remain open even in the

sense of (ii). Obviously a directed set $S \subseteq X$ can be regarded as a net, and now in view of (ii) it follows that S converges to $\bigsqcup S$ — if this lub exists.

On a complete lattice D, the formalization names this induced topology **`scottTopologicalSpace`**: a concrete `TopologicalSpace D` built from Mathlib's Scott construction, **not** a global typeclass default (so we can still name Scott explicitly in later theorems and compare other topologies on the same D).

Lemma (`scottTopologicalSpace.t0Space`). D is a T_0-space for the topology `scottTopologicalSpace`.

Proof. After `let : TopologicalSpace D := scottTopologicalSpace`, implicit `[TopologicalSpace D]` in the Scott library matches the topology in the goal `@T0Space D scottTopologicalSpace`; then `IsScott` holds by definition and Mathlib supplies `T0Space`. (That `let` line does not pass arguments into the def—it registers the def as the local topology instance for the proof block.) □

`scottTopologicalSpace.lean`

```
module

public import Mathlib.Topology.Order.ScottTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

variable {X : Type*} {D : Type u} [TopologicalSpace X] [CompleteLattice D]

@[reducible] noncomputable def scottTopologicalSpace : TopologicalSpace D :=
  Topology.scott D (@Set.univ (Set D))

namespace scottTopologicalSpace

theorem t0Space : @T0Space D scottTopologicalSpace := by
  let : TopologicalSpace D := scottTopologicalSpace
  have : IsScott D (@Set.univ (Set D)) := ⟨rfl⟩
  exact inferInstanceAs (T0Space D)

end scottTopologicalSpace

end DomainTheory.ContinuousLattice
```

Monotone nets What *is* interesting in topological spaces is *convergence* and the properties of *limit points*. We shall discuss limits in terms of *nets*, in particular in terms of *monotone nets*. A monotone net in a T_0-space X is a function

$$x : I \to X$$

where $\langle I, \leq \rangle$ is a directed set and where $i \leq j$ implies $x_i \sqsubseteq x_j$ for all $i, j \in I$. In a T_1-space a monotone net is constant (hence, uninteresting) because the $\sqsubseteq$-relation is the identity.

On a complete lattice D, the library records the same idea as **`MonotoneNet`**: for a directed index type ι and a net $x : \iota \to D$, require **`Monotone x`** (i.e. $i \le j \Rightarrow x_i \le x_j$ in the lattice order). With the Scott topology, $\sqsubseteq$ agrees with $\le$ (`SpecializationPreorder.scott_iff`); **Proposition 2.1** is stated using `MonotoneNet` and `ScottConvergesTo`.

`MonotoneNet.lean`

```
module

public import Mathlib.Order.CompleteLattice.Basic
import Mathlib.Order.Directed
import Mathlib.Order.Monotone.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

variable {ι : Type u} [Preorder ι] [IsDirected ι (· ≤ ·)]

class MonotoneNet (x : ι → D) : Prop where
  out : Monotone x

end DomainTheory.ContinuousLattice
```

Scott convergence As usual (cf. [126, p. 66]) we say that a net x *converges* to an element y iff whenever U is open and $y \in U$, then for some $i \in I$ we have $x_j \in U$ for all $j \ge i$. Note that a monotone net x converges to each of its terms x_i. Suppose that a monotone net x converges to an element y which is an upper bound to all the terms of x. Then y must be the *least upper bound*, which we write as:

$$y = \bigsqcup \{x_i : i \in I\}.$$

To see this, assume that z is any other upper bound with $x_i \sqsubseteq z$ for all $i \in I$. If U is open and $y \in U$, then $x_i \in U$ for some $i \in I$. But then $z \in U$, and so $y \sqsubseteq z$ follows.

On a complete lattice D with directed index type ι, the library names Scott's convergence condition **`ScottConvergesTo`**: for a net $x : \iota \to D$ and a point $y \in D$,

$$\text{`ScottConvergesTo x y`} \iff \forall U,\ U \text{ Scott-open} \wedge y \in U \Rightarrow \text{eventually } x_j \in U.$$

"Open" in the induced Scott sense is **`ScottOpen U`**; the same sets are open for the topology **`scottTopologicalSpace`** (`ScottOpen.isOpen_iff`). The specialization preorder in that topology agrees with lattice $\le$ (`SpecializationPreorder.scott_iff`). Scott's paragraph above is the motivation for the biconditional in **Proposition 2.1**, which is proved in `proposition_2_1.lean` and stated on the separate Proposition 2.1 note (together with **`MonotoneNet`**).

`ScottConvergesTo.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

variable {ι : Type u} [Preorder ι] [IsDirected ι (· ≤ ·)]

def ScottConvergesTo (x : ι → D) (y : D) : Prop :=
  ∀ U, ScottOpen U → y ∈ U → ∃ i, ∀ j ≥ i, x j ∈ U

end DomainTheory.ContinuousLattice
```

Proposition 2.1 We can summarize this discussion as follows.

2.1 Proposition. *In a partially ordered set X with the induced topology, a monotone net $x : I \to X$ with a least upper bound converges to an element $y \in X$ iff*

$$y \sqsubseteq \bigsqcup \{x_i : i \in I\}.$$

□

On a complete lattice D with directed index type ι, Scott's "induced topology" is the Scott topology **`scottTopologicalSpace`**: convergence is **`ScottConvergesTo`**, monotonicity is **`MonotoneNet x`**, and a least upper bound L of the net is **`IsLUB (range x) L`**. Under that topology, $\sqsubseteq$ is lattice $\leq$ (`SpecializationPreorder.scott_iff`), so Scott's condition is formalized as `ScottConvergesTo x y ↔ y ≤ L`. The module **`proposition_2_1`** is Scott's biconditional; the nested lemmas in namespace **`proposition_2_1`** are the two directions.

Lemma (`proposition_2_1.le_of_converges`). If `IsLUB (range x) L` and `ScottConvergesTo x y`, then $y \leq L$.

Proof. If $y \not\leq L$, then $y \in \{z : z \not\leq L\}$, which is Scott-open (`ScottOpen.not_le L`). Convergence gives a tail of the net in that set, contradicting that L bounds every term. □

Lemma (`proposition_2_1.of_le`). If ι is nonempty, `MonotoneNet x`, `IsLUB (range x) L`, and $y \leq L$, then `ScottConvergesTo x y`.

Proof. For Scott-open U with $y \in U$, the values $\{x_i\}$ form a directed set in the lattice order; $L = \bigsqcup x_i \in U$ since U is an upset, so Scott-openness yields some term in U, and monotonicity keeps all later terms in U. □

Theorem (`proposition_2_1`). If ι is nonempty, `MonotoneNet x`, and `IsLUB (range x) L`, then `ScottConvergesTo x y ↔ y ≤ L`.

Proof. `proposition_2_1.le_of_converges` and `proposition_2_1.of_le`. □

`proposition_2_1.lean`

```
module

public import DomainTheory.ContinuousLattice.Specialization.MonotoneNet
```

```
public import DomainTheory.ContinuousLattice.Specialization.ScottConvergesTo

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

variable {D : Type u} [CompleteLattice D]

variable {ι : Type u} [Preorder ι] [IsDirected ι (· ≤ ·)]

variable {x : ι → D} {L y : D}

namespace proposition_2_1

omit [IsDirected ι (· ≤ ·)] in
theorem le_of_converges (hL : IsLUB (range x) L) (hconv : ScottConvergesTo x y) : y ≤
    L := by
  by_contra hyL
  obtain ⟨i, hi⟩ := hconv {z : D | ¬ z ≤ L} (ScottOpen.not_le L) hyL
  exact hi i le_rfl (hL.1 ⟨i, rfl⟩)

theorem of_le [Nonempty ι] (hx : MonotoneNet x) (hL : IsLUB (range x) L) (hyL : y ≤ L)
    :
    ScottConvergesTo x y := by
  intro U hU hyU
  have hdir : DirectedOn (· ≤ ·) (range x) := by
    rintro _ ⟨i, rfl⟩ _ ⟨j, rfl⟩
    obtain ⟨k, hik, hjk⟩ := IsDirected.directed (r := (· ≤ ·)) i j
    exact ⟨x k, ⟨k, rfl⟩, hx.out hik, hx.out hjk⟩
  have hLU : sSup (range x) ∈ U := by
    rw [IsLUB.sSup_eq hL]
    exact hU.1 hyL hyU
  obtain ⟨s, hsS, hsU⟩ := hU.2 (Set.range_nonempty x) hdir hLU
  obtain ⟨i₀, rfl⟩ := hsS
  refine ⟨i₀, fun j hj => hU.1 (hx.out hj) hsU⟩

end proposition_2_1

theorem proposition_2_1 [Nonempty ι] (hx : MonotoneNet x) (hL : IsLUB (range x) L) :
    ScottConvergesTo x y ↔ y ≤ L :=
  ⟨fun hconv => proposition_2_1.le_of_converges hL hconv, proposition_2_1.of_le hx hL⟩

end DomainTheory.ContinuousLattice
```

2.3.2 Way-below relation ✓

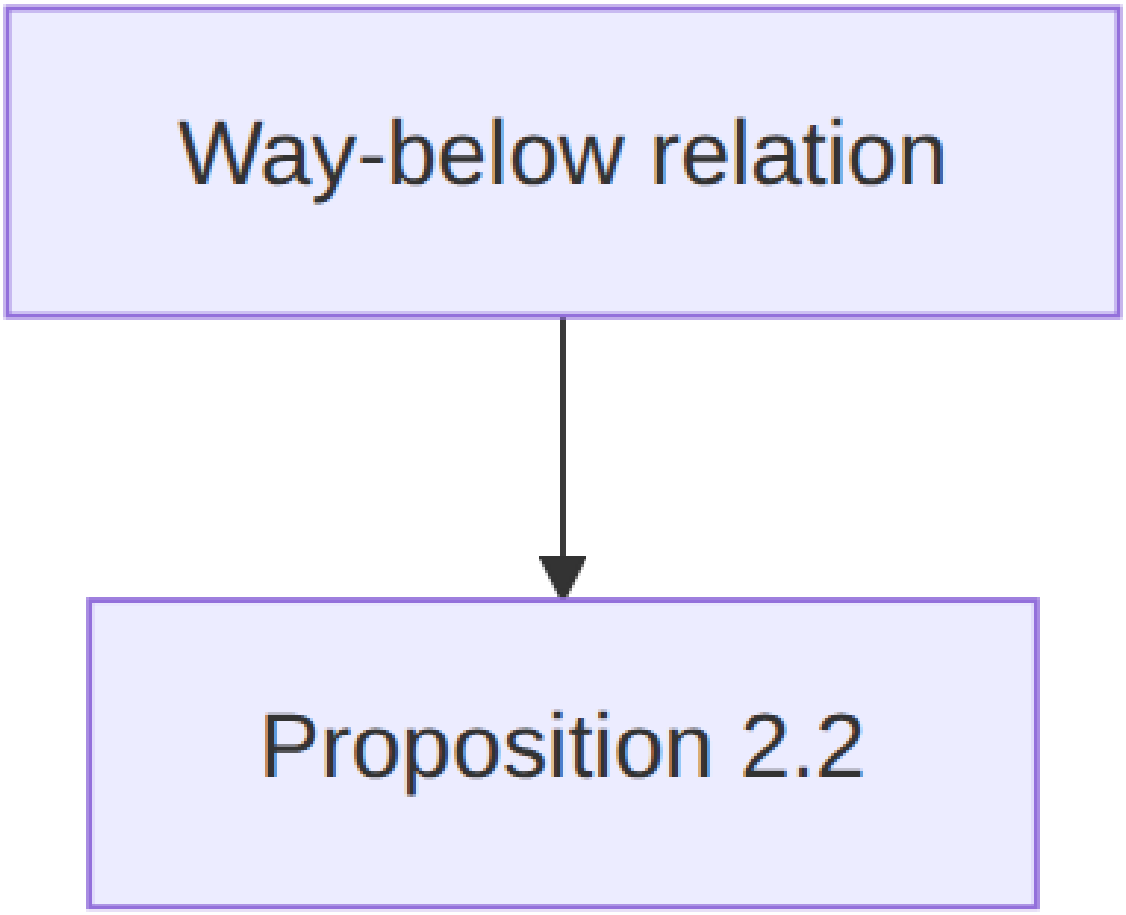


Figure 8: Way-below relation ✓.

Way-below relation Our main interest will lie with those partially ordered sets in which every subset has a lub: namely, *complete lattices*. If D is such a space we write $\bot = \bigsqcup \emptyset$ and $\top = \bigsqcup D$ for the smallest and largest elements (read: bottom and top). As is well known, greatest lower bounds must exist, for:

$$\sqcap S = \bigsqcup \{x \in D : x \sqsubseteq y\} \text{ for all } y \in S$$

gives the definition.

Given a complete lattice D we define

$$x \ll y \text{ iff } y \in \text{Int}\{z \in D : x \sqsubseteq z\},$$

where the interior is taken in the sense of the induced topology. The relation $x \ll y$ behaves somewhat like a strict ordering relation; at least its meaning is clearly that y should be definitely larger than x in the

partial ordering. Such a relation has many pleasant properties.

On a complete lattice D, the induced topology is the Scott topology **`scottTopologicalSpace`**; $\sqsubseteq$ is lattice $\leq$ (`SpecializationPreorder.scott_iff`). Scott's interior condition is equivalent to: there is a Scott-open set U with $y \in U$ and $U \subseteq \{z : x \leq z\}$ (i.e. $U \subseteq\!\uparrow x$). The library names that relation **`WayBelow`**; we write **`x`** $\ll$ **`y`** for `WayBelow x y`. The set $\{z : x \ll z\}$ is Scott-open (`ScottOpen.wayBelow`).

The elementary lemmas live in namespace **`WayBelow`** (same pattern as **`ScottOpen`**). **Proposition 2.2** is Scott's list (i)–(vii), proved in `proposition_2_2.lean` by calling these tools; see the Proposition 2.2 note.

Definition (`WayBelow`). $x \ll y$ iff $\exists U$, U Scott-open $\wedge y \in U \wedge U \subseteq \{z : x \leq z\}$.

Lemma (`WayBelow.bot`). $\bot \ll x$ for all $x \in D$.

Proof. Take $U = D$ (`ScottOpen.univ`). □

Lemma (`WayBelow.sup`). If $x \ll z$ and $y \ll z$, then $x \sqcup y \ll z$.

Proof. Intersect the two witnessing Scott-open sets (`ScottOpen.inter`). □

Lemma (`WayBelow.trans_le`). If $x \ll y$ and $y \leq z$, then $x \ll z$.

Proof. The same Scott-open witness works; $z \in U$ because U is an upset. □

Lemma (`WayBelow.le_trans`). If $x \leq y$ and $y \ll z$, then $x \ll z$.

Proof. Every $w \in U$ with $y \ll z$ satisfies $x \leq w$. □

Lemma (`WayBelow.le`). If $x \ll y$, then $x \leq y$.

Proof. $y \in U \subseteq\!\uparrow x$. □

Lemma (`WayBelow.self_iff_scottOpen_Ici`). $x \ll x$ iff $\uparrow x = \{z : x \leq z\}$ is Scott-open.

Proof. ($\Rightarrow$) A witness U for $x \ll x$ must equal $\uparrow x$. ($\Leftarrow$) Take $U = \uparrow x$. □

Lemma (`WayBelow.sSup_iff`). For nonempty directed $S \subseteq D$,

$$x \ll \sup S \iff \exists y \in S,\ x \ll y.$$

Proof. ($\Rightarrow$) Use the Scott-open witness and directed-sup accessibility in the definition of Scott-open. ($\Leftarrow$) If $x \ll y$ with $y \in S$, the witness for $x \ll y$ also works for $x \ll \sup S$. □

Lemma (`WayBelow.directedOn`). For each $y \in D$, the set $\{x : x \ll y\}$ is directed in the lattice order.

Proof. If $a, b \ll y$, then $a \sqcup b \ll y$ by `WayBelow.sup`. □

Lemma (`WayBelow.sInf`). If U is Scott-open and $y \in U$, then $\inf U \ll y$.

Proof. U itself witnesses $\inf U \ll y$: every $w \in U$ satisfies $\inf U \leq w$. □

Lemma (`WayBelow.map_orderIso`). If $e : A \to B$ is an order isomorphism and $x \ll y$ in A, then $e(x) \ll e(y)$ in B.

Proof. Pull back the Scott-open witness for $x \ll y$ along e^{-1} (`ScottOpen.preimage_orderIso`); membership and the $\uparrow$-subgraph condition follow from e and e^{-1} being order isomorphisms. □

`WayBelow.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
universe u

variable {D : Type u} [CompleteLattice D]

def WayBelow (x y : D) : Prop :=
  letI : LE D :=
    (CompleteLattice.toCompleteSemilatticeInf (α := D)).toPartialOrder.toPreorder.toLE
  letI : Preorder D :=
    (CompleteLattice.toCompleteSemilatticeInf (α := D)).toPartialOrder.toPreorder
  ∃ U : Set D, ScottOpen U ∧ y ∈ U ∧ U ⊆ Set.Ici x

@[inherit_doc] scoped infix:50 " ≪ " => WayBelow

namespace WayBelow

theorem bot (x : D) : (⊥ : D) ≪ x :=
  ⟨Set.univ, ScottOpen.univ, Set.mem_univ x, fun _ _ => bot_le⟩

theorem sup {x y z : D} (hx : x ≪ z) (hy : y ≪ z) : x ⊔ y ≪ z := by
  obtain ⟨U, hU, hzU, hUsub⟩ := hx
  obtain ⟨V, hV, hzV, hVsub⟩ := hy
  refine ⟨U ∩ V, hU.inter hV, ⟨hzU, hzV⟩, fun w hw => ?_⟩
  exact Set.mem_Ici.2 (sup_le (hUsub hw.1) (hVsub hw.2))

theorem trans_le {x y z : D} (h : x ≪ y) (hyz : y ≤ z) : x ≪ z := by
  obtain ⟨U, hU, hyU, hsub⟩ := h
  exact ⟨U, hU, hU.1 hyz hyU, hsub⟩

theorem le_trans {x y z : D} (hxy : x ≤ y) (h : y ≪ z) : x ≪ z := by
  obtain ⟨U, hU, hzU, hsub⟩ := h
  refine ⟨U, hU, hzU, fun w hw => ?_⟩
  have hyw : y ≤ w := Set.mem_Ici.1 (hsub hw)
  exact Set.mem_Ici.2 (hxy.trans hyw)

theorem le {x y : D} (h : x ≪ y) : x ≤ y := by
  obtain ⟨U, _, hyU, hsub⟩ := h
  exact hsub hyU

theorem self_iff_scottOpen_Ici {x : D} : x ≪ x ↔ ScottOpen (Set.Ici x) := by
  constructor
  · rintro ⟨U, hU, hxU, hsub⟩

    have hIci : Set.Ici x = U :=
      le_antisymm (fun w hw => hU.1 (Set.mem_Ici.1 hw) hxU) hsub
    rw [hIci]; exact hU
  · intro hopen
    exact ⟨Set.Ici x, hopen, Set.self_mem_Ici, le_refl _⟩

theorem sSup_iff {x : D} {S : Set D} (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) : x ≪ sSup S ↔ ∃ y ∈ S, x ≪ y := by
  constructor
  · rintro ⟨U, hU, hsupU, hsub⟩
    obtain ⟨s, hsS, hsU⟩ := hU.2 hS hSdir hsupU
    exact ⟨s, hsS, U, hU, hsU, hsub⟩
  · rintro ⟨y, hyS, hxy⟩
```

```
    obtain ⟨U, hU, hyU, hsub⟩ := hxy
    exact ⟨U, hU, hU.1 (le_sSup hyS) hyU, hsub⟩

theorem directedOn (y : D) : DirectedOn (· ≤ ·) {x | x ≪ y} :=
  fun a ha b hb => ⟨a ⊔ b, ha.sup hb, le_sup_left, le_sup_right⟩

theorem sInf {U : Set D} (hU : ScottOpen U) {y : D} (hy : y ∈ U) :
    sInf U ≪ y :=
  ⟨U, hU, hy, fun _ hz => Set.mem_Ici.2 (sInf_le hz)⟩

variable {A B : Type u} [CompleteLattice A] [CompleteLattice B]

theorem map_orderIso (e : A ≃o B) {x y : A} (h : x ≪ y) : e x ≪ e y := by
  obtain ⟨U, hU, hyU, hUsub⟩ := h
  refine ⟨e.symm ^-1' U, ScottOpen.preimage_orderIso e.symm hU, ?_, ?_⟩
  · simpa using hyU
  · intro z hz
    have hxz : x ≤ e.symm z := Set.mem_Ici.1 (hUsub hz)
    exact Set.mem_Ici.2 (by simpa using (e.le_iff_le.mpr hxz))

end WayBelow

end DomainTheory.ContinuousLattice
```

Proposition 2.2 **2.2 Proposition.** In a complete lattice D we have:

(i) $\bot \ll x$;

(ii) $x \ll z$ and $y \ll z$ imply $x \sqcup y \ll z$;

(iii) $x \ll y \sqsubseteq z$ implies $x \ll z$;

(iv) $x \sqsubseteq y \ll z$ implies $x \ll z$;

(v) $x \ll y$ implies $x \sqsubseteq y$;

(vi) $x \ll x$ iff $\{z \in D : x \sqsubseteq z\}$ is open;

(vii) if $S \subseteq D$ is directed, then $x \ll \bigsqcup S$ iff $x \ll y$ for some $y \in S$. □

The proofs of these statements can be safely left to the reader.

proposition_2_2.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u
```

```
variable {D : Type u} [CompleteLattice D]

theorem proposition_2_2 :
    (∀ x : D, (⊥ : D) ≪ x) ∧
    (∀ x y z : D, x ≪ z → y ≪ z → x ⊔ y ≪ z) ∧
    (∀ x y z : D, x ≪ y → y ≤ z → x ≪ z) ∧
    (∀ x y z : D, x ≤ y → y ≪ z → x ≪ z) ∧
    (∀ x y : D, x ≪ y → x ≤ y) ∧
    (∀ x : D, x ≪ x ↔ ScottOpen (Set.Ici x)) ∧
    (∀ (x : D) (S : Set D), S.Nonempty → DirectedOn (· ≤ ·) S →
      (x ≪ sSup S ↔ ∃ y ∈ S, x ≪ y)) :=
  ⟨WayBelow.bot,
    fun _ _ _ hx hy => WayBelow.sup hx hy,
    fun _ _ _ h hyz => WayBelow.trans_le h hyz,
    fun _ _ _ hxy h => WayBelow.le_trans hxy h,
    fun _ _ h => WayBelow.le h,
    fun _ => WayBelow.self_iff_scottOpen_Ici,
    fun _ _ hS hdir => WayBelow.sSup_iff hS hdir⟩

end DomainTheory.ContinuousLattice
```

2.3.3 Continuous lattice ✓

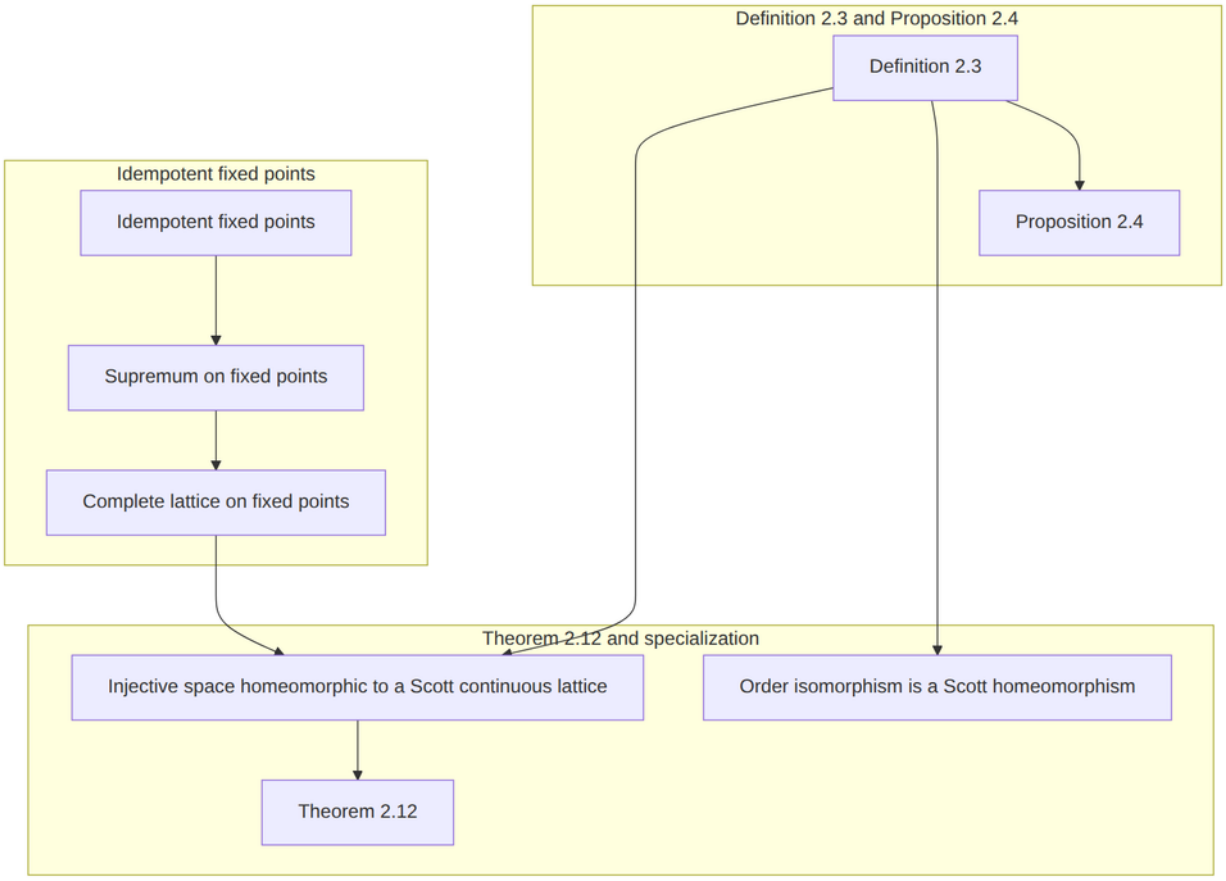


Figure 9: Continuous lattice ✓.

Definition 2.3: Continuous lattice **2.3 Definition.** A *continuous lattice* is a complete lattice D in which for every $y \in D$ we have:

$$y = \bigsqcup \{x \in D : x \ll y\}.$$

On a complete lattice D, the library records this as the typeclass **`ContinuousLattice`**: for every y, the set $\{x : x \ll y\}$ has least upper bound y. Equivalently **`sSup_wayBelow`**: $\sup\{x : x \ll y\} = y$. The way-below relation is **`WayBelow`** / $\ll$; see the Way-below relation note.

Supporting lemmas live in namespace **`ContinuousLattice`** in `ContinuousLattice.lean`. (One interpolation lemma is also exported as **`WayBelow.interpolate`** because it is stated purely in terms of $\ll$, but its proof uses **`ContinuousLattice`**.)

Lemma (`ContinuousLattice.exists_wayBelow_subset`). If D is a continuous lattice, U is Scott-open, and $z \in U$, then there exists $a \ll z$ such that $\{w : a \ll w\} \subseteq U$.

Proof. $\{x : x \ll z\}$ is directed (`WayBelow.directedOn`), has supremum z (`sSup_wayBelow`), and meets U by Scott-openness; any witness a in the intersection works, using that U is an upset. □

Lemma (`ContinuousLattice.exists_wayBelow_Ici_subset`). Under the same hypotheses, there is $a \ll z$ with $\uparrow a \subseteq U$.

Proof. As above, with $\uparrow a$ in place of $\{w : a \ll w\}$. □

Lemma (`ContinuousLattice.orderIso`). If $e : A \to B$ is an order isomorphism and B is a continuous lattice, then so is A (way-below pulls back along e^{-1} via **`WayBelow.map_orderIso`**).

Lemma (`ContinuousLattice.isContinuousLattice_iff_isLUB_sInf_nhds`). D is a continuous lattice iff for every $y \in D$, the set of infima $\inf U$ over Scott-open $U \ni y$ has least upper bound y.

Proof. ($\Rightarrow$) Each $\inf U \ll y$ (`WayBelow.sInf`); the lub of those infs is at most y; **`out`** gives equality. ($\Leftarrow$) Every $x \ll y$ lies in some Scott-open U with $y \in U$, hence $x \le \inf U$; the lub of the infs bounds all way-below elements below y, so **`out`** holds. □

Lemma (`WayBelow.interpolate`). If D is a continuous lattice and $a \ll c$, then there exists b with $a \ll b \ll c$.

Proof. The set $M = \{m : \exists x,\ m \ll x \ll c\}$ is directed; c is its supremum by **`sSup_wayBelow`**; apply **`WayBelow.sSup_iff`**. □

Proposition 2.4 (equivalent formulations via $\inf U \ll y$ and limits) is Scott's discussion after 2.3; the **`isContinuousLattice_iff_isLUB_sInf_nhds`** lemma is the Scott-open inf characterization used in the library.

`ContinuousLattice.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow

@[expose] public section

namespace DomainTheory

open scoped DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

class ContinuousLattice (D : Type u) [CompleteLattice D] : Prop where
  out : letI : LE D :=
    (CompleteLattice.toCompleteSemilatticeInf (α := D)).toPartialOrder.toPreorder.toLE
    ∀ y : D, IsLUB {x | x ≪ y} y
  sSup_wayBelow : ∀ y : D, sSup {x | x ≪ y} = y := fun y => (out y).sSup_eq

open ContinuousLattice

namespace ContinuousLattice

theorem exists_wayBelow_subset (hD : ContinuousLattice D) {U : Set D} (hU : ScottOpen
    U)
    {z : D} (hz : z ∈ U) : ∃ a, a ≪ z ∧ {w | a ≪ w} ⊆ U := by
  have hne : {a | a ≪ z}.Nonempty := ⟨⊥, WayBelow.bot z⟩
  have hsup : sSup {a | a ≪ z} ∈ U := by rw [hD.sSup_wayBelow z]; exact hz
  obtain ⟨a, haz, haU⟩ := hU.2 hne (WayBelow.directedOn z) hsup
  exact ⟨a, haz, fun w hw => hU.1 hw.le haU⟩
```

```
theorem exists_wayBelow_Ici_subset (hD : ContinuousLattice D) {U : Set D} (hU :
    ScottOpen U)
    {z : D} (hz : z ∈ U) : ∃ a, a ≪ z ∧ Set.Ici a ⊆ U := by
  have hne : {a | a ≪ z}.Nonempty := ⟨⊥, WayBelow.bot z⟩
  have hsup : sSup {a | a ≪ z} ∈ U := by rw [hD.sSup_wayBelow z]; exact hz
  obtain ⟨a, haz, haU⟩ := hU.2 hne (WayBelow.directedOn z) hsup
  exact ⟨a, haz, fun w hw => hU.1 (Set.mem_Ici.1 hw) haU⟩

theorem isContinuousLattice_iff_isLUB_sInf_nhds :
    ContinuousLattice D ↔
      ∀ y : D, IsLUB {a : D | ∃ U, ScottOpen U ∧ y ∈ U ∧ a = sInf U} y := by
  constructor
  · intro h y
    refine ⟨?_, ?_⟩
    · rintro a ⟨U, hU, hyU, rfl⟩
      exact (WayBelow.sInf hU hyU).le
    · intro b hb
      refine (h.out y).2 ?_
      intro x hx
      obtain ⟨U, hU, hyU, hsub⟩ := hx
      have hxle : x ≤ sInf U := le_sInf fun _ hz => Set.mem_Ici.1 (hsub hz)
      exact hxle.trans (hb ⟨U, hU, hyU, rfl⟩)
  · intro h
    refine { out := ?_ }
    intro y
    refine ⟨fun x hx => hx.le, ?_⟩
    intro b hb
    refine (h y).2 ?_
    rintro a ⟨U, hU, hyU, rfl⟩
    exact hb (WayBelow.sInf hU hyU)

theorem orderIso {A B : Type u} [latticeA : CompleteLattice A] [latticeB :
    CompleteLattice B]
    (e : A ≃o B) (hB : ContinuousLattice B) : ContinuousLattice A := by
  refine { out := ?_ }
  intro y
  refine ⟨fun x hx => hx.le, fun b hb => ?_⟩
  apply e.le_iff_le.mp
  exact (hB.out (e y)).2 fun z hz => by
    have hz' : e.symm z ≪ y := by
      simpa using WayBelow.map_orderIso e.symm hz
    have := hb hz'
    simpa using e.monotone this

namespace WayBelow

theorem interpolate (hD : ContinuousLattice D) {a c : D} (hac : a ≪ c) :
    ∃ b, a ≪ b ∧ b ≪ c := by
  set M : Set D := {m | ∃ x, m ≪ x ∧ x ≪ c} with hM
  have hMdir : DirectedOn (· ≤ ·) M := by
    rintro m₁ ⟨x₁, hm₁x₁, hx₁c⟩ m₂ ⟨x₂, hm₂x₂, hx₂c⟩
    obtain ⟨x₃, hx₃c, hx₁₃, hx₂₃⟩ := WayBelow.directedOn c x₁ hx₁c x₂ hx₂c
    obtain ⟨m₃, hm₃x₃, hm₁m₃, hm₂m₃⟩ :=
      WayBelow.directedOn x₃ m₁ (hm₁x₁.trans_le hx₁₃) m₂ (hm₂x₂.trans_le hx₂₃)
    exact ⟨m₃, ⟨x₃, hm₃x₃, hx₃c⟩, hm₁m₃, hm₂m₃⟩
```

```
  have hMne : M.Nonempty := ⟨⊥, a, WayBelow.bot a, hac⟩
  have hsupM : sSup M = c := by
    refine le_antisymm (sSup_le ?_) ?_
    · rintro m ⟨x, hmx, hxc⟩
      exact hmx.le.trans hxc.le
    · rw [← hD.sSup_wayBelow c]
      refine sSup_le fun x hxc => ?_
      rw [← hD.sSup_wayBelow x]
      exact sSup_le fun m hmx => le_sSup ⟨x, hmx, hxc⟩
  rw [← hsupM] at hac
  obtain ⟨m, ⟨x, hmx, hxc⟩, ham⟩ := (WayBelow.sSup_iff hMne hMdir).1 hac
  exact ⟨x, ham.trans_le hmx.le, hxc⟩

end WayBelow

end ContinuousLattice

end DomainTheory
```

Proposition 2.4 **2.4 Proposition.** A complete lattice D is continuous iff for every $y \in D$ we have:

$$y = \bigsqcup \{ \sqcap U : y \in U \},$$

where U ranges over the open subsets of D.

Proof: Suppose D is continuous. If $y \in D$ and $x \ll y$, then let $U = \{\mathrm{Int} z : x \sqsubseteq z\}$, an open set. Now $y \in U$ by definition, and $\{U \subseteq z : x \sqsubseteq z\}$. Thus,

$$x \sqsubseteq \sqcap U \sqsubseteq y.$$

It easily follows by lattice theory that the equation of 2.3 implies that of 2.4.

In the converse direction we have only to note that if U is open and $y \in U$, then $\sqcap U \ll y$. The implication from 2.4 to 2.3 results at once. □

proposition_2_4.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

theorem proposition_2_4 :
    ContinuousLattice D ↔
      ∀ y : D, IsLUB {a : D | ∃ U, ScottOpen U ∧ y ∈ U ∧ a = sInf U} y :=
  isContinuousLattice_iff_isLUB_sInf_nhds

end DomainTheory.ContinuousLattice
```

Limits and convergence What is the idea of this definition? A continuous lattice is more special than a complete lattice: not only are lubs to be limits but every element must be a limit from below. This rather rough remark can be made more precise. In any complete lattice D define the *principal limit* of a net $x : I \to D$ by the formula:

$$\lim\langle x_i : i \in I\rangle = \bigsqcup\{\sqcap x_j : j \geq i : i \in I\}.$$

Then specify that x converges to $y \in D$ iff

$$y \sqsubseteq \lim\langle\{x_i : i \in I\rangle\}.$$

Having a notion of convergence, we can then say that $U \subseteq D$ is open iff every net converging to an element of U is eventually in U. This gives nothing more than what we have called the induced topology above, as is easily checked. But now being in possession of a topology, we can redefine convergence in the usual way. Question: when do the two notions of convergence agree? Answer: if and only if D is a continuous lattice.

For obviously by construction the limit definition of convergence implies the topological. Now if D is a continuous lattice and x converges to y topologically, consider an open $U \subseteq D$ with $y \in U$. For some $i \in I$ we shall have $x_j \in U$ for all $j \geq i$. Therefore

$$\sqcap U \sqsubseteq \{\sqcap x_j : j \geq i \sqsubseteq \lim\langle x_i : i \in I\rangle\}.$$

From the formula of 2.4 it at once follows that $y \sqsubseteq \lim\langle\{x_i : i \in I\rangle\}$. Thus, in continuous lattices, we have shown that the two notions of convergence are the same.

Finally, suppose that the two notions coincide for a complete lattice D. Define a directed set $I = (U, z) : y, z \in U$, where z ranges over D and U over open subsets of D. This set is directed by the relation: $(U, z) \leq (V, w)$ iff $U \supseteq V$. Let $x : I \to D$ be given by: $x(U, z) = z$. Then x is a net converging to y topologically. But

$$\lim\langle x_i : i \in I\rangle = \bigsqcup\{\sqcap U : y \in U\}.$$

In this way we see that the assumption about the two styles of convergence implies that D is a continuous lattice in view of 2.4.

In T_0-spaces continuous functions are always monotonic (i.e. $\sqsubseteq$-preserving). For continuous lattices, by virtue of the remarks we have just made about limits, we can define the continuity of $f : D \to D'$ to mean that

$$f(\lim\langle\{x_i : i \in I\rangle) \sqsubseteq \lim'\langle f(x_i) : i \in I\rangle\}$$

for all nets $x : I \to D$. This is all very fine, but general limits are messy to work with; we shall find it easier to state results in terms of lubs as in 2.5–2.7 below.

Before going any deeper, however, we should clear up another point about topologies. Suppose that D is any T_0-space which becomes a complete lattice under its induced partial ordering. Then it is evident from our definitions that every set open in the given topology is also open in the topology induced from the lattice structure. Question: when do the two topologies agree? Answer: a sufficient condition is that the equation:

$$y = \bigsqcup\{\sqcap U : y \in U\}$$

hold for all $y \in D$, where U ranges over the given open sets. Because in that case if V is open in the lattice sense and $y \in V$, then $\sqcap U \in V$ for some set U, open in the given sense, where $y \in U$. But $U \subseteq V$ follows, and so V is a union of given open sets and is itself open in the given topology. Of course this equation implies that D is a continuous lattice by virtue of 2.4. Notice that by the same token the sets of the form $\{y \in D : x \ll y\}$ will form a basis for the open sets of a continuous lattice.

hold for all $y \in D$, where U ranges over the given open sets. Because in that case if V is open in the lattice sense and $y \in V$, then $\sqcap U \in V$ for some set U, open in the given sense, where $y \in U$. But $U \subseteq V$ follows, and so V is a union of given open sets and is itself open in the given topology. Of course this equation implies that D is a continuous lattice by virtue of 2.4. Notice that by the same token the sets of the form $\{y \in D : x \ll y\}$ will form a basis for the open sets of a continuous lattice.

Idempotent fixed points On a complete lattice L, write $e : L \to L$ for an endomap. We say e is **idempotent** when $e \circ e = e$ (equivalently $e(ex) = ex$ for all x). An element $x \in L$ is a **fixed point** of e when $e(x) = x$.

Scott's discussion after Theorem 2.12 treats a retraction $j : \mathcal{P}I \to \mathcal{P}I$ as idempotent and identifies its range with $\{X : j(X) = X\}$. The proof of Theorem 2.12 uses the same fixed-point subtype for the idempotent $e = s \circ r$ built from an injective-space retraction into a Sierpiński power.

Definition (`IdempotentFixedPoint`). For $e : L \to L$ on a complete lattice L, **`IdempotentFixedPoint e`** is the subtype $\{x \in L \mid ex = x\}$.

`IdempotentFixedPoint.lean`

```
module

public import Mathlib.Order.CompleteLattice.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {L : Type*} [CompleteLattice L]

/-- Fixed points `{x | e x = x}` of an endomap on a complete lattice. -/
abbrev IdempotentFixedPoint (e : L → L) : Type _ := {x : L // e x = x}

end DomainTheory.ContinuousLattice
```

Supremum on fixed points When e is idempotent on a complete lattice L, the fixed-point subtype **`IdempotentFixedPoint e`** needs a join operation: ambient suprema of fixed points need not be fixed, but applying e after taking the supremum lands back in the fixed-point set. The lemmas below live in namespace **`IdempotentFixedPoint.supremumSet`** in `IdempotentFixedPoint/supremumSet.lean`. See Idempotent fixed points for the subtype definition. The **`CompleteLattice`** instance built from this **`SupSet`** is Complete lattice on fixed points.

Definition (`IdempotentFixedPoint.supremumSet`). Given $e(ex) = ex$, the **`SupSet`** instance sends a set S of fixed points to $e(\sup\{\mathrm{val}\, k : k \in S\})$ in L, packaged as a fixed point using idempotence.

Proof. Definition only (`supremumSet` in Lean). □

Theorem (`IdempotentFixedPoint.supremumSet.isLUB`). With **`IdempotentFixedPoint.supremumS`** `sSup S` is a least upper bound of S in the fixed-point order (requires **`Monotone e`**).

Proof. Write $u = e(\sup \operatorname{val} S)$ for the proposed supremum.

Upper bound. If $k \in S$, then $\operatorname{val} k \le \sup \operatorname{val} S$ in L, so $e(\operatorname{val} k) \le u$ by monotonicity; fixedness gives $\operatorname{val} k = e(\operatorname{val} k) \le u$.

Least. If w is an upper bound of S among fixed points, then $\operatorname{val} k \le \operatorname{val} w$ for all $k \in S$, hence $\sup \operatorname{val} S \le \operatorname{val} w$; monotonicity yields $u \le e(\operatorname{val} w) = \operatorname{val} w$. □

supremumSet.lean

```
module

public import DomainTheory.ContinuousLattice.ContinuousLattices.IdempotentFixedPoint

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

variable {L : Type*} [CompleteLattice L] {e : L → L}

namespace IdempotentFixedPoint

/-- Supremum in the fixed-point set: `e` applied to the ambient supremum. -/
@[reducible] noncomputable def supremumSet (hidem : ∀ x, e (e x) = e x) :
    SupSet (IdempotentFixedPoint e) :=
  ⟨fun S => ⟨e (sSup (Subtype.val '' S)), hidem _⟩⟩

namespace supremumSet

theorem isLUB (hidem : ∀ x, e (e x) = e x) (hmono : Monotone e) :
    letI := IdempotentFixedPoint.supremumSet hidem
    ∀ S : Set (IdempotentFixedPoint e), IsLUB S (sSup S) := by
  let := IdempotentFixedPoint.supremumSet hidem
  intro S
  constructor
  · intro k hk
    rw [Subtype.coe_le_coe.symm, Subtype.coe_mk]
    show (k : L) ≤ e (sSup (Subtype.val '' S))
    calc (k : L) = e k := k.2.symm
      _ ≤ e (sSup (Subtype.val '' S)) :=
          hmono (le_sSup (Set.mem_image_of_mem _ hk))
  · intro w hw
    rw [Subtype.coe_le_coe.symm, Subtype.coe_mk]
    show e (sSup (Subtype.val '' S)) ≤ (w : L)
    calc e (sSup (Subtype.val '' S)) ≤ e (w : L) := by
          refine hmono (sSup_le ?_)
          rintro _ ⟨k, hk, rfl⟩
          exact hw hk
      _ = (w : L) := w.2

end supremumSet

end IdempotentFixedPoint

end DomainTheory.ContinuousLattice
```

Complete lattice on fixed points Once **`IdempotentFixedPoint.supremumSet`** provides suprema on fixed points, Mathlib's **`completeLatticeOfSup`** turns that **`SupSet`** into a full **`CompleteLattice`** on **`IdempotentFixedPoint e`**, using **`supremumSet.isLUB`** for arbitrary (not merely directed) joins. The lemmas below live in namespace **`IdempotentFixedPoint.completeLattice`** in `IdempotentFixedPoint/completeLattice.lean`. Prerequisites: Supremum on fixed points. Scott continuity and subspace topology are in `IdempotentFixedPoint/scottContinuous.lean`.

Definition (`IdempotentFixedPoint.completeLattice`). Given idempotent e and **`Monotone e`**, install **`supremumSet`**, then apply **`completeLatticeOfSup`** with witness **`supremumSet.isLUB`**.

Proof. `completeLatticeOfSup` from the **`SupSet`** and **`isLUB`** lemmas already proved in `supremumSet.lean`. □

Theorem (`IdempotentFixedPoint.completeLattice.coe`). With **`IdempotentFixedPoint.complet`** coercion to L sends `sSup S` to $e(\sup\{\mathrm{val}\, k : k \in S\})$.

Proof. Definitional: `sSup` for this instance is **`supremumSet`**, and the subtype coercion is the underlying value of that fixed point (`rfl`). □

`completeLattice.lean`

```
module

public import DomainTheory.ContinuousLattice.ContinuousLattices.IdempotentFixedPoint.
    supremumSet

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

variable {L : Type*} [CompleteLattice L] {e : L → L}

namespace IdempotentFixedPoint

@[reducible] noncomputable def completeLattice (hidem : ∀ x, e (e x) = e x) (hmono :
    Monotone e) :
    CompleteLattice (IdempotentFixedPoint e) :=
  letI := supremumSet hidem
  completeLatticeOfSup (IdempotentFixedPoint e) (supremumSet.isLUB hidem hmono)

namespace completeLattice

theorem coe (hidem : ∀ x, e (e x) = e x) (hmono : Monotone e)
    (S : Set (IdempotentFixedPoint e)) :
    letI := IdempotentFixedPoint.completeLattice hidem hmono
    ((sSup S : IdempotentFixedPoint e) : L) = e (sSup (Subtype.val '' S)) := rfl

end completeLattice

end IdempotentFixedPoint

end DomainTheory.ContinuousLattice
```

Injective space homeomorphic to a Scott continuous lattice This is the **constructive half** of Theorem 2.12 before the biconditional is stated: from injectivity alone (Corollary 1.6, Sierpiński power, idempotent fixed points, Propositions 2.9–2.10) one builds a **continuous lattice** and a **homeomorphism** to it with **Scott** topology. See Idempotent fixed points for the fixed-point subtype used in the proof. The full iff is Theorem 2.12; when a lattice order and topology are already given and specialization equals $\leq$, see Specialization equals the given complete-lattice order on injective T0 spaces.

Theorem (`injective_homeomorph_to_continuousLatticeScott`). If D is an injective T_0 space, then there exist a complete lattice E, a **`ContinuousLattice`** structure on E, and a homeomorphism $D \cong E$ where E carries its **Scott** topology.

Proof. Retract D into a Sierpiński power; form the idempotent $e = s \circ r$; take **`IdempotentFixedPoint e`** with **`completeLattice`**, **`isContinuousLattice`**, and **`scottTopology`**; homeomorph D to the fixed-point set as a subspace. □

`injective_homeomorph_to_continuousLatticeScott.lean`

```
module

public import DomainTheory.ContinuousLattice.Constructions.scottTopology
public import DomainTheory.ContinuousLattice.ContinuousLattices.IdempotentFixedPoint.
    scottContinuous
public import DomainTheory.ContinuousLattice.Injective.corollary_1_6
public import DomainTheory.ContinuousLattice.ScottMaps.continuous_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

/-- An injective `T₀` space is homeomorphic to a continuous lattice under its Scott
    topology. -/
theorem injective_homeomorph_to_continuousLatticeScott {D : Type u} [TopologicalSpace
    D]
    [T0Space D] (hD : InjectiveSpace.{u, u} D) :
    ∃ (E : Type u) (inst : CompleteLattice E),
      @ContinuousLattice E inst ∧
        Nonempty (@Homeomorph D E _ (@scottTopologicalSpace E inst)) := by
  obtain ⟨ι, ⟨R⟩⟩ := (corollary_1_6 D).1 hD

  have hidem : ∀ x, R.section' (R.retraction (R.section' (R.retraction x)))
      = R.section' (R.retraction x) :=
    fun x => congrArg R.section' (R.retraction_section (R.retraction x))
  have hL : ContinuousLattice (ι → Prop) := proposition_2_9.sierpinskiPower ι

  have hcont_e : @Continuous (ι → Prop) (ι → Prop) scottTopologicalSpace
    scottTopologicalSpace
      (fun x => R.section' (R.retraction x)) := by
    rw [scottTopology.sierpinskiPower ι]
    exact R.section'.continuous.comp R.retraction.continuous
  have hsc : PreservesDirectedSup (fun x => R.section' (R.retraction x)) :=
    continuous_preservesDirectedSup hcont_e
```

```
  refine ⟨IdempotentFixedPoint (fun x => R.section' (R.retraction x)),
    IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone hsc),
    IdempotentFixedPoint.isContinuousLattice hidem hsc hL, ?_⟩

  have heq : (@scottTopologicalSpace (IdempotentFixedPoint (fun x => R.section' (R.
    retraction x)))
      (IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone hsc))
    )
      = instTopologicalSpaceSubtype := by
    rw [IdempotentFixedPoint.scottTopology hidem hsc hL, scottTopology.sierpinskiPower
     ι]
    rfl
  rw [heq]

  have hfix : ∀ d, (fun x => R.section' (R.retraction x)) (R.section' d) = R.section' d
     :=
    fun d => congrArg R.section' (R.retraction_section d)
  exact ⟨{
    toFun := fun d => ⟨R.section' d, hfix d⟩
    invFun := fun k => R.retraction k.1
    left_inv := fun d => R.retraction_section d
    right_inv := fun k => Subtype.ext k.2
    continuous_toFun := by exact R.section'.continuous.subtype_mk hfix
    continuous_invFun := by exact R.retraction.continuous.comp continuous_subtype_val
    }⟩

end DomainTheory.ContinuousLattice
```

Theorem 2.12 **2.12 Theorem.** The injective spaces are exactly the continuous lattices.

This theorem is an immediate consequence of the preceding results: an injective space is a retract of a power of $\mathbb{O}$. But $\mathbb{O}$ is a finite lattice $(\bot \sqsubseteq \top)$, and so the given space is a continuous lattice under its induced topology. On the other hand a continuous lattice is injective. It remains then to prove 2.9–2.11.

The library splits the two directions: Injective space homeomorphic to a Scott continuous lattice (**`injective_homeomorph_to_continuousLatticeScott`**) is the injective $\Rightarrow$ homeomorph-to-Scott-CL step; **`proposition_2_11`** is continuous lattice $\Rightarrow$ injective. When a complete lattice and topology are already on the type and specialization equals $\leq$, Specialization equals the given complete-lattice order on injective T0 spaces refines the constructive direction (**`continuousLattice_scottTopology_of_injective`**).

Theorem (`theorem_2_12`). For a T_0 space D, **`InjectiveSpace D`** iff there exist a complete lattice E, a **`ContinuousLattice`** instance on E, and a homeomorphism $D \cong E$ with E carrying its Scott topology.

Proof. In `theorem_2_12.lean`: the forward implication is **`injective_homeomorph_to_continuousLatticeScott`**; the converse applies **`proposition_2_11`** and **`proposition_1_4`** to a witnessing homeomorphism. □

`theorem_2_12.lean`

```
module

public import DomainTheory.ContinuousLattice.Constructions.proposition_2_11
```

```
public import DomainTheory.ContinuousLattice.ContinuousLattices.
    injective_homeomorph_to_continuousLatticeScott
public import DomainTheory.ContinuousLattice.Injective.proposition_1_4

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

theorem theorem_2_12 {D : Type u} [TopologicalSpace D] [T0Space D] :
    InjectiveSpace.{u, u} D ↔
      ∃ (E : Type u) (inst : CompleteLattice E),
        @ContinuousLattice E inst ∧
          Nonempty (@Homeomorph D E _ (@scottTopologicalSpace E inst)) := by
  refine ⟨injective_homeomorph_to_continuousLatticeScott, ?_⟩
  rintro ⟨E, inst, hE, ⟨h⟩⟩
  let τE : TopologicalSpace E := @scottTopologicalSpace E inst
  exact @proposition_1_4 E D τE _ (proposition_2_11 hE)
    ⟨⇑h, h.continuous⟩ ⟨⇑h.symm, h.symm.continuous⟩ (fun d => h.left_inv d)

end DomainTheory.ContinuousLattice
```

Order isomorphism is a Scott homeomorphism Given complete lattices A and B and an order isomorphism $e : A \cong B$, Scott's topology on each side is expressed through **Scott-open** sets (`ScottOpen`), not merely through the order. **`ScottOpen.preimage_orderIso`** shows Scott-open sets pull back along e and e^{-1}; with **`ScottOpen.isOpen_iff`**, e is a **homeomorphism** for **`scottTopologicalSpace`** on A and B. This lemma is used in Specialization equals the given complete-lattice order on injective T0 spaces to align Scott on the given type with topology induced along an order iso built from injectivity.

Definition (`orderIsoScottHomeomorph`). For complete lattices A and B and $e : A \cong B$, **`orderIsoScottHomeomorph e`** is the homeomorphism $A \cong B$ with both sides carrying **`scottTopologicalSpace`**.

Proof. Open sets are exactly Scott-open (`ScottOpen.isOpen_iff`); continuity in each direction is **`ScottOpen.preimage_orderIso`** for e and e^{-1}. □

`orderIsoScottHomeomorph.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

variable {A B : Type u} [latticeA : CompleteLattice A] [latticeB : CompleteLattice B]
```

```
noncomputable def orderIsoScottHomeomorph (e : A ≃o B) :
    @Homeomorph A B (@scottTopologicalSpace A latticeA)
      (@scottTopologicalSpace B latticeB) := by
  letI : TopologicalSpace A := @scottTopologicalSpace A latticeA
  letI : TopologicalSpace B := @scottTopologicalSpace B latticeB
  letI : Preorder A := latticeA.toPreorder
  letI : Preorder B := latticeB.toPreorder
  exact {
  toEquiv := e.toEquiv
  continuous_toFun := by
    rw [continuous_def]
    intro U hU
    exact ScottOpen.isOpen_iff.mpr
      (ScottOpen.preimage_orderIso e (ScottOpen.isOpen_iff.mp hU))
  continuous_invFun := by
    rw [continuous_def]
    intro U hU
    exact ScottOpen.isOpen_iff.mpr
      (ScottOpen.preimage_orderIso e.symm (ScottOpen.isOpen_iff.mp hU))
  }

end DomainTheory.ContinuousLattice
```

Specialization equals the given complete-lattice order on injective T0 spaces Theorem 2.12 and Injective space homeomorphic to a Scott continuous lattice show that an injective T_0 space is homeomorphic to **some** Scott continuous lattice. In **inverse limits** (and similar setups) one already has a **complete lattice** and **topology** on the type, and **`SpecializationPreorder`** is known to agree with that lattice's $\leq$. This note is the transfer lemma: promote the existential homeomorphism to **`ContinuousLattice`** and **Scott** for **that** data.

Primary use: Proposition 4.1 and **`scottTopology_eq_induced_pi`**, after injectivity and **`specializationPreorder_iff`** are available on the inverse limit. Scott topology along an order iso is Order isomorphism is a Scott homeomorphism.

Theorem (`continuousLattice_scottTopology_of_injective`). Let A be a complete lattice with topology τ_A. If A is T_0 and injective, and for all $x, y \in A$ we have specialization $x \rightsquigarrow y$ iff $x \leq_A y$, then A is a **`ContinuousLattice`** for $\leq_A$ and τ_A equals the **Scott** topology for $\leq_A$.

Proof. **`injective_homeomorph_to_continuousLatticeScott`** gives E, a Scott **`ContinuousLattice`** on E, and a homeomorphism $h : A \cong E$.

Order iso. **`SpecializationPreorder.scott_iff`**, **`SpecializationPreorder.map`**, and `hspec` show h is an order isomorphism $e : A \cong E$.

Continuous lattice. **`ContinuousLattice.orderIso e`**.

Scott topology. **`orderIsoScottHomeomorph`** (see linked note) identifies Scott on A with the topology induced from Scott on E along e; **`Homeomorph.isInducing`** matches τ_A with the same induced topology along h. □

After Theorem 2.12 Following Theorem 2.12, Scott steps back from the proof to compare two pictures of the same equivalence.

The **lattice approach** to injective spaces gives a completely "internal" characterization: the types are complete lattices; **way-below** ($\ll$, Definition 2.3) is defined lattice-theoretically by

$$x \ll y \quad \text{iff} \quad \text{whenever } y \sqsubseteq \bigsqcup Z \text{ and } Z \subseteq D \text{ is directed,}$$

then $x \sqsubseteq z$ for some $z \in Z$; and every $y \in D$ satisfies

$$y = \bigsqcup \{x \in D : x \ll y\}.$$

That makes D a continuous lattice with the sets $\{y \in D : x \ll y\}$ as a basis for the topology. Such T_0-spaces are injective, every injective space arises this way, and the lattice structure is uniquely determined by the topology. Furthermore, as in the proof of 2.11, injectivity can be exhibited by an explicit extension formula.

The **retract approach** should also be considered. Cartesian powers $\mathbb{O}^I$ are simple: as lattices they are Boolean algebras of all subsets of I. The topology has as a basis the classes of sets containing given finite sets (the *weak topology*, cf. [184]). A continuous function

$$j : \mathcal{P}I \to \mathcal{P}I$$

is one of "finite character", so

$$j(X) = \bigcup \{j(F) : F \subseteq X\}$$

where $X \subseteq I$ and F ranges over *finite* sets. Such j is a *retraction* iff it is *idempotent* ($j \circ j = j$), so the range is the set of fixed points. As we have seen in Idempotent fixed points,

$$D = \{X \in \mathcal{P}I : j(X) = X\}$$

is a continuous lattice (under $\subseteq$ here), and *every* continuous lattice is isomorphic to one obtained this way. This is a representation theorem, but Scott warns it is not very helpful for proofs.

Finally, any space (any finite family of spaces $X, Y, \ldots$) embeds as a *subspace* of some continuous lattice D. Since D is injective, every continuous $f : X \to Y$ extends to some $\bar{f} : D \to D$; maps $D \to D$ therefore encode continuous maps on all subspaces—motivation for Function spaces.

2.3.4 Scott maps ✓

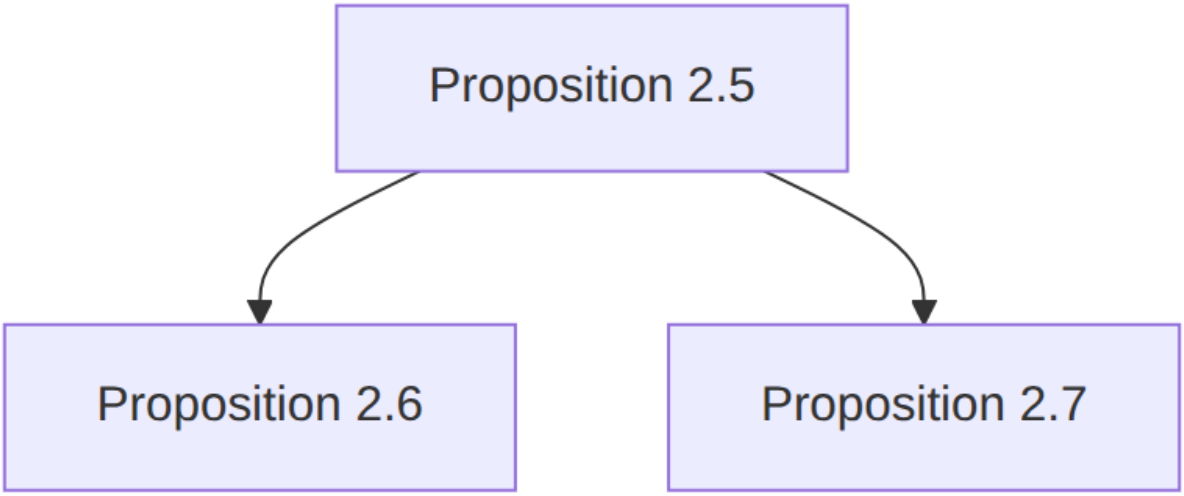


Figure 10: Scott maps ✓.

Proposition 2.5 **2.5 Proposition.** *If D and D' are complete lattices with their induced topologies, then a function $f : D \to D'$ is continuous iff for all directed subsets $S \subseteq D$:*

$$f(\bigsqcup S) = \bigsqcup \{f(x) : x \in S\}.$$

Proof: If $f : D \to D'$ is continuous, the equation follows from the definition of continuous function and the fact that lubs are limits. Assume then that the equation holds for all directed sets S. Let $U' \subseteq D'$ be open in D' and let

$$U = \{x \in D : f(x) \in U'\}.$$

We must show that U is open in D. Note first that if $x \sqsubseteq y$, then $S = x, y$ is directed; hence,

$$f(x \sqcup y) = f(y) = f(x) \sqcup f(y),$$

so $f(x) \sqsubseteq f(y)$. Thus f is monotonic and so U satisfies condition (i). That U satisfies condition (ii) follows at once from the above equation. □

`PreservesDirectedSup.lean`

```
module
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology
```

```
variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

def PreservesDirectedSup (f : D → D') : Prop :=
  ∀ {{S : Set D}}, S.Nonempty → DirectedOn (· ≤ ·) S → f (sSup S) = sSup (f '' S)

namespace PreservesDirectedSup

theorem monotone {f : D → D'} (hf : PreservesDirectedSup f) : Monotone f := by
  intro x y hxy
  have hdir : DirectedOn (· ≤ ·) ({x, y} : Set D) := directedOn_pair hxy
  have hS : ({x, y} : Set D).Nonempty := ⟨x, Set.mem_insert _ _⟩
  have hsup : sSup ({x, y} : Set D) = y := by
    calc sSup ({x, y} : Set D) = x ⊔ y := sSup_pair
      _ = y := by apply le_antisymm; exact sup_le hxy le_rfl; exact le_sup_right
  have heq := hf hS hdir
  rw [hsup, Set.image_pair] at heq
  exact le_trans (le_sSup (Set.mem_insert _ _)) heq.symm.le

theorem scottOpen_preimage {f : D → D'} (hf : PreservesDirectedSup f) {U : Set D'}
    (hU : ScottOpen U) : ScottOpen (f ^-1' U) := by
  have hmono := monotone hf
  refine ⟨fun a b hab ha => hU.1 (hmono hab) ha, fun S hS hSdir hmem => ?_⟩
  rw [Set.mem_preimage] at hmem
  have hmem' : sSup (f '' S) ∈ U := by rw [← hf hS hSdir]; exact hmem
  obtain ⟨s, hsS, hsU⟩ := hU.2 (Set.image_nonempty.2 hS)
    (fun s hs t ht => by
      obtain ⟨a, haS, rfl⟩ := hs
      obtain ⟨b, hbS, rfl⟩ := ht
      obtain ⟨c, hcS, hac, hbc⟩ := hSdir a haS b hbS
      exact ⟨f c, Set.mem_image_of_mem f hcS, hmono hac, hmono hbc⟩) hmem'
  obtain ⟨a, haS, rfl⟩ := hsS
  exact ⟨a, haS, Set.mem_preimage.2 hsU⟩

theorem continuous_of {f : D → D'} (hf : PreservesDirectedSup f) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace f := by
  rw [continuous_def]
  intro U hU
  rw [ScottOpen.isOpen_iff] at hU ⊢
  exact scottOpen_preimage hf hU

end PreservesDirectedSup

end DomainTheory.ContinuousLattice
```

preservesDirectedSup_monotone.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

theorem preservesDirectedSup_monotone {f : D → D'} (hf : PreservesDirectedSup f) :
    Monotone f :=
  PreservesDirectedSup.monotone hf

end DomainTheory.ContinuousLattice
```

scottOpen_preimage.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

theorem scottOpen_preimage {f : D → D'} (hf : PreservesDirectedSup f) {U : Set D'}
    (hU : ScottOpen U) : ScottOpen (f ^-1' U) :=
  PreservesDirectedSup.scottOpen_preimage hf hU

end DomainTheory.ContinuousLattice
```

continuous_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

theorem continuous_preservesDirectedSup {f : D → D'}
    (hf : @Continuous D D' scottTopologicalSpace scottTopologicalSpace f) :
    PreservesDirectedSup f := by
  have hD : @IsScott D univ _ (Topology.scott D univ) :=
    @IsScott.mk D univ _ (Topology.scott D univ) rfl
  have hD' : @IsScott D' univ _ (Topology.scott D' univ) :=
    @IsScott.mk D' univ _ (Topology.scott D' univ) rfl
  have hsc : ScottContinuous f :=
```

```
    scottContinuousOn_univ.1 <|
      (@Topology.IsScott.scottContinuousOn_iff_continuous D D' _
        (Topology.scott D univ) _ (Topology.scott D' univ) hD' f univ hD
        (fun _ _ _ => trivial)).2 hf
  intro S hS hSdir
  exact (hsc hS hSdir (isLUB_sSup S)).sSup_eq.symm

end DomainTheory.ContinuousLattice
```

proposition_2_5.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
import DomainTheory.ContinuousLattice.ScottMaps.continuous_preservesDirectedSup

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

theorem proposition_2_5 (f : D → D') :
    (@Continuous D D' scottTopologicalSpace scottTopologicalSpace f) ↔
      PreservesDirectedSup f :=
  ⟨continuous_preservesDirectedSup, fun hf => PreservesDirectedSup.continuous_of hf⟩

end DomainTheory.ContinuousLattice
```

Proposition 2.6 **2.6 Proposition.** *With functions from complete lattices to complete lattices, a function of several variables is continuous in the variables jointly iff it is continuous in the variables separately.*

Proof: It will be sufficient to discuss functions of two variables. The product $D \times D'$ of two complete lattices is a complete lattice, and it is easy to check that the induced topology is the product topology. Since projection is continuous, joint continuity implies separate continuity. To check the converse suppose that

$$f : D \times D' \to D''$$

is a map where the separate continuity holds as follows:

$$f(\bigsqcup S, y) = \bigsqcup f(x, y) : x \in S$$

and

$$f(x, \bigsqcup S') = \bigsqcup \{f(x, y) : y \in S'\}$$

where $S \subseteq D$ and $S' \subseteq D'$ are directed and $x \in D$ and $y \in D'$. Let now

$$S^* \subseteq D \times D'$$

be directed in the product. The projection of S^* to D and of S^* to D' produces directed subsets $S \subseteq D$ and $S' \subseteq D'$. Note that

$$\bigsqcup S^* = (\bigsqcup S, \bigsqcup S').$$

Thus by assumption

$$f(\bigsqcup S^*) = \bigsqcup \{f(x,y) : x \in S, y \in S'\}.$$

But since S^* is directed, $x \in S$ and $y \in S'$ implies $x \sqsubseteq u$ and $y \sqsubseteq v$ for $(u,v) \in S^*$. Thus by monotonicity of f we can show

$$f(\bigsqcup S^*) = \bigsqcup \{f(u,v) : (u,v) \in S^*\}$$

and that gives the joint continuity. □

One of the justifications (by euphony at least) of the term *continuous lattice* is the fact that such spaces allow for so many continuous functions. One indication of this is the result:

proposition_2_6.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
import Mathlib.Tactic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

theorem proposition_2_6 (f : D × D' → D'') :
    PreservesDirectedSup f ↔
      (∀ y, PreservesDirectedSup fun x => f (x, y)) ∧
        (∀ x, PreservesDirectedSup fun y => f (x, y)) := by
  constructor
  ·
    intro hf
    refine ⟨fun y S hS hSdir => ?_, fun x S hS hSdir => ?_⟩
    · set T : Set (D × D') := (fun x => (x, y)) '' S with hT
      have hTne : T.Nonempty := hS.image _
      have hTdir : DirectedOn (· ≤ ·) T := by
        rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
        obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
        exact ⟨(c, y), Set.mem_image_of_mem _ hc, ⟨hac, le_rfl⟩, ⟨hbc, le_rfl⟩⟩
      have hfst : Prod.fst '' T = S := by rw [hT, Set.image_image]; simp
      have hsnd : Prod.snd '' T = {y} := by rw [hT, Set.image_image]; exact hS.
    image_const y
```

```
    have hsupT : sSup T = (sSup S, y) := by
      have e1 : (sSup T).1 = sSup S := by rw [Prod.fst_sSup, hfst]
      have e2 : (sSup T).2 = y := by rw [Prod.snd_sSup, hsnd, sSup_singleton]
      exact Prod.ext_iff.mpr ⟨e1, e2⟩
    have h := hf hTne hTdir
    rw [hsupT, hT, Set.image_image] at h
    simpa using h
  · set T : Set (D × D') := (fun y => (x, y)) '' S with hT
    have hTne : T.Nonempty := hS.image _
    have hTdir : DirectedOn (· ≤ ·) T := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
      exact ⟨(x, c), Set.mem_image_of_mem _ hc, ⟨le_rfl, hac⟩, ⟨le_rfl, hbc⟩⟩
    have hfst : Prod.fst '' T = {x} := by rw [hT, Set.image_image]; exact hS.
  image_const x
    have hsnd : Prod.snd '' T = S := by rw [hT, Set.image_image]; simp
    have hsupT : sSup T = (x, sSup S) := by
      have e1 : (sSup T).1 = x := by rw [Prod.fst_sSup, hfst, sSup_singleton]
      have e2 : (sSup T).2 = sSup S := by rw [Prod.snd_sSup, hsnd]
      exact Prod.ext_iff.mpr ⟨e1, e2⟩
    have h := hf hTne hTdir
    rw [hsupT, hT, Set.image_image] at h
    simpa using h
.

  rintro ⟨h1, h2⟩ Sstar hne hdir
  have hmono1 : ∀ y, Monotone fun x => f (x, y) := fun y =>
  preservesDirectedSup_monotone (h1 y)
  have hmono2 : ∀ x, Monotone fun y => f (x, y) := fun x =>
  preservesDirectedSup_monotone (h2 x)
  have hmono : Monotone f := by
    rintro ⟨a, b⟩ ⟨c, d⟩ hpq
    exact le_trans (hmono1 b hpq.1) (hmono2 c hpq.2)
  have hSne : (Prod.fst '' Sstar).Nonempty := hne.image _
  have hS'ne : (Prod.snd '' Sstar).Nonempty := hne.image _
  have hSdir : DirectedOn (· ≤ ·) (Prod.fst '' Sstar) := hdir.fst
  have hS'dir : DirectedOn (· ≤ ·) (Prod.snd '' Sstar) := hdir.snd
  have hsupStar : sSup Sstar = (sSup (Prod.fst '' Sstar), sSup (Prod.snd '' Sstar))
  :=
    Prod.ext_iff.mpr ⟨Prod.fst_sSup _, Prod.snd_sSup _⟩
  apply le_antisymm
  · rw [hsupStar]
    have e1 : f (sSup (Prod.fst '' Sstar), sSup (Prod.snd '' Sstar))
        = sSup ((fun x => f (x, sSup (Prod.snd '' Sstar))) '' (Prod.fst '' Sstar))
  :=
      h1 (sSup (Prod.snd '' Sstar)) hSne hSdir
    rw [e1]
    apply sSup_le
    rintro _ ⟨x, hx, rfl⟩
    show f (x, sSup (Prod.snd '' Sstar)) ≤ sSup (f '' Sstar)
    have e2 : f (x, sSup (Prod.snd '' Sstar))
        = sSup ((fun y => f (x, y)) '' (Prod.snd '' Sstar)) :=
      h2 x hS'ne hS'dir
    rw [e2]
    apply sSup_le
    rintro _ ⟨y, hy, rfl⟩
    show f (x, y) ≤ sSup (f '' Sstar)
```

```
      obtain ⟨p, hpS, hpfst⟩ := hx
      obtain ⟨q, hqS, hqsnd⟩ := hy
      obtain ⟨r, hrS, hpr, hqr⟩ := hdir p hpS q hqS
      have hxr : x ≤ r.1 := hpfst ▸ hpr.1
      have hyr : y ≤ r.2 := hqsnd ▸ hqr.2
      calc f (x, y) ≤ f (r.1, r.2) := hmono ⟨hxr, hyr⟩
        _ = f r := by rw [Prod.mk.eta]
        _ ≤ sSup (f '' Sstar) := le_sSup (Set.mem_image_of_mem f hrS)
    · apply sSup_le
      rintro _ ⟨p, hp, rfl⟩
      exact hmono (le_sSup hp)

end DomainTheory.ContinuousLattice
```

Proposition 2.7 **2.7 Proposition.** *In a continuous lattice D the finitary lattice operations $\sqcup$ and $\sqcap$ are continuous.*

Proof: It is trivial to show that $\sqcup$ is continuous in every complete lattice; this is not so for $\sqcap$. In view of 2.6 we need only show

$$x \sqcap \bigsqcup S = \bigsqcup \{x \sqcap y : y \in S\}$$

for every directed $S \subseteq D$. In fact it is enough to show

$$x \sqcap \bigsqcup S \sqsubseteq \bigsqcup \{x \sqcap y : y \in S\}$$

because the opposite inequality is valid in all complete lattices. In view of the fact that D is continuous, it is enough to show that

$$t \ll x \sqcap \bigsqcup S \text{ implies } t \sqsubseteq \bigsqcup \{x \sqcap y : y \in S\}.$$

So assume $t \ll x \sqcap \bigsqcup S$. Then $t \sqsubseteq x \sqcap \bigsqcup S \sqsubseteq x$. Also $t \ll \bigsqcup S$ because $x \sqcap \bigsqcup S \sqsubseteq \bigsqcup S$. Thus $t \ll y$ for some $y \in S$ since the set

$$\{z \in D : t \ll z\}$$

is open. But then $t \sqsubseteq y$, and so $t \sqsubseteq x \sqcap y$, and the result follows. □

It is now time to provide some examples of continuous lattices.

meet_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

theorem meet_preservesDirectedSup (x : D) (hD : ContinuousLattice D) :
    PreservesDirectedSup (fun z => x ⊓ z) := by
  intro S hS hSdir
  apply le_antisymm
  · have hle : sSup {t | t ≪ x ⊓ sSup S} ≤ sSup (Set.image (fun z => x ⊓ z) S) := by
      apply sSup_le
      intro t ht
      obtain ⟨w, hwS, ht_w⟩ := (WayBelow.sSup_iff hS hSdir).1 (ht.trans_le inf_le_right
    )
      have hmem : x ⊓ w ∈ Set.image (fun z : D => x ⊓ z) S :=
        Set.mem_image_of_mem (fun z => x ⊓ z) hwS
      have hbound : x ⊓ w ≤ sSup (Set.image (fun z => x ⊓ z) S) := le_sSup hmem
      exact (le_inf (ht.le.trans inf_le_left) ht_w.le).trans hbound
    dsimp only
    rw [← hD.sSup_wayBelow (x ⊓ sSup S)]
    exact hle
  · apply sSup_le
    intro z hz
    obtain ⟨w, hwS, rfl⟩ := hz
    exact inf_le_inf le_rfl (le_sSup hwS)

end DomainTheory.ContinuousLattice
```

proposition_2_7.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.ScottMaps.meet_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

namespace proposition_2_7

theorem sup :
    @Continuous (D × D) D scottTopologicalSpace scottTopologicalSpace (fun p : D × D =
    > p.1 ⊔ p.2) :=
  PreservesDirectedSup.continuous_of <| by
    intro S hS hSdir
    simpa using (ScottContinuous.sup₂ (β := D) (d := S) hS hSdir (isLUB_sSup S)).
```

```
    sSup_eq.symm

theorem inf_left (hD : ContinuousLattice D) (y : D) :
    @Continuous D D scottTopologicalSpace scottTopologicalSpace (fun x => x ⊓ y) :=
  PreservesDirectedSup.continuous_of <| by
    intro S' hS' hSdir'
    rw [show (fun x => x ⊓ y) = fun z => y ⊓ z from funext fun x => inf_comm x y]
    have h := meet_preservesDirectedSup y hD
    exact h (S := S') hS' hSdir'

theorem inf_right (hD : ContinuousLattice D) (x : D) :
    @Continuous D D scottTopologicalSpace scottTopologicalSpace (fun y => x ⊓ y) :=
  PreservesDirectedSup.continuous_of (meet_preservesDirectedSup x hD)

end proposition_2_7

end DomainTheory.ContinuousLattice
```

2.3.5 Proposition 2.8 ✓

2.8 Proposition. A finite lattice is a continuous lattice. □

`directedOn_finite_sSup_mem.lean`

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.Preorder.Finite
public import Mathlib.Basic.Finite.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.T0Sierpinski
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Sets.Opens

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

theorem directedOn_finite_sSup_mem {S : Set D} (hSfin : S.Finite) (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) : sSup S ∈ S := by
  obtain ⟨m, hm⟩ := hSfin.exists_maximal hSne
  have hub : m ∈ upperBounds S := by
    intro s hs
    obtain ⟨c, hcS, hmc, hsc⟩ := hSdir m hm.1 s hs
    exact hsc.trans (hm.2 hcS hmc)
```

```
  have hlub : IsLUB S m := ⟨hub, fun b hb => hb hm.1⟩
  rw [hlub.sSup_eq]
  exact hm.1

end DomainTheory.ContinuousLattice
```

proposition_2_8.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.Constructions.directedOn_finite_sSup_mem
public import Mathlib.Data.Fintype.EquivFin
public import Mathlib.Basic.Finite.Defs
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

theorem proposition_2_8 [Finite D] : ContinuousLattice D := by
  refine { out := ?_ }
  intro y
  have hopen : ScottOpen (Set.Ici y) := by
    refine ⟨IsUpperSet.Ici y, fun S hSne hSdir hmem => ?_⟩
    exact ⟨sSup S, directedOn_finite_sSup_mem (Set.toFinite S) hSne hSdir, hmem⟩
  have hyy : y ≪ y := WayBelow.self_iff_scottOpen_Ici.mpr hopen
  exact ⟨fun x hx => hx.le, fun b hb => hb hyy⟩

end DomainTheory.ContinuousLattice
```

2.3.6 Proposition 2.9 ✓

2.9 Proposition. The Cartesian product of any number of continuous lattices is a continuous lattice with the induced topology agreeing with the product topology.

proposition_2_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.piWayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.Constructions.proposition_2_8
public import Mathlib.Topology.Defs.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

theorem proposition_2_9 {ι : Type*} (E : ι → Type*) [∀ i, CompleteLattice (E i)]
    (hE : ∀ i, ContinuousLattice (E i)) :
    ContinuousLattice (∀ i, E i) ∧
      (scottTopologicalSpace : TopologicalSpace (∀ i, E i)) =
        @Pi.topologicalSpace ι E (fun _ => scottTopologicalSpace) := by
  classical
  have hprod : ContinuousLattice (∀ i, E i) := by
    refine { out := ?_ }
    intro y
    refine ⟨fun x hx => hx.le, fun b hb => ?_⟩
    rw [Pi.le_def]
    intro i
    rw [← (hE i).sSup_wayBelow (y i)]
    apply sSup_le
    intro a ha
    set e : (∀ j, E j) := Function.update (⊥ : ∀ j, E j) i a with he
    have hei : e i = a := by rw [he]; exact Function.update_self i a _
    have hcyl : e ≪ y := by
      obtain ⟨U, hU, hyiU, hUsub⟩ := ha
      refine ⟨{z : ∀ j, E j | z i ∈ U}, ?_, hyiU, ?_⟩
      · refine ⟨fun z w hzw hz => hU.1 (hzw i) hz, fun S hSne hSdir hmem => ?_⟩
        rw [Set.mem_ofPred_eq, sSup_apply_eq_sSup_image] at hmem
        have hdir' : DirectedOn (· ≤ ·) (Function.eval i '' S) := by
          rintro _ ⟨f, hf, rfl⟩ _ ⟨g, hg, rfl⟩
          obtain ⟨h, hhS, hfh, hgh⟩ := hSdir f hf g hg
          exact ⟨Function.eval i h, ⟨h, hhS, rfl⟩, hfh i, hgh i⟩
        obtain ⟨t, htimg, htU⟩ := hU.2 (hSne.image _) hdir' hmem
        obtain ⟨f, hfS, rfl⟩ := htimg
        exact ⟨f, hfS, htU⟩
      · intro z hz
        rw [Set.mem_Ici, Pi.le_def]
        intro j
        rcases eq_or_ne j i with rfl | hji
        · rw [hei]; exact Set.mem_Ici.1 (hUsub hz)
        · rw [he, Function.update_of_ne hji]; exact bot_le
    have hle : e i ≤ b i := (hb hcyl) i
    rw [hei] at hle
    exact hle
  refine ⟨hprod, ?_⟩
  apply le_antisymm
  · refine le_iInf fun i => ?_
    rw [← continuous_iff_le_induced]
    exact PreservesDirectedSup.continuous_of (fun _ _ _ => sSup_apply_eq_sSup_image)
  · intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open _ (@Pi.topologicalSpace ι E (fun _ =>
    scottTopologicalSpace))]
    intro z hz
```

```
    obtain ⟨a, haz, haIci⟩ := exists_wayBelow_Ici_subset hprod hU hz
    obtain ⟨F, hF⟩ := piWayBelow.wayBelow_finite_support haz
    have hproj : ∀ i, a i ≪ z i := fun i => piWayBelow.wayBelow_proj haz i
    choose V hVopen hzV hVsub using hproj
    refine ⟨⋂ i ∈ F, (fun w : ∀ j, E j => w i) ^-1' V i, ?_, ?_, ?_⟩
    · intro w hw
      rw [Set.mem_iInter₂] at hw
      refine haIci (Set.mem_Ici.2 fun j => ?_)
      by_cases hjF : j ∈ F
      · exact Set.mem_Ici.1 (hVsub j (hw j hjF))
      · rw [hF j hjF]; exact bot_le
    · refine @isOpen_biInter_finset (∀ j, E j) ι
        (@Pi.topologicalSpace ι E (fun _ => scottTopologicalSpace)) F
        (fun i => (fun w : ∀ j, E j => w i) ^-1' V i) (fun i _ => ?_)
      have hVi : @IsOpen (E i) scottTopologicalSpace (V i) := ScottOpen.isOpen_iff.mpr
     (hVopen i)
      have hind : @IsOpen _
          (TopologicalSpace.induced (fun w : ∀ j, E j => w i) scottTopologicalSpace)
          ((fun w : ∀ j, E j => w i) ^-1' V i) :=
        (isOpen_induced_iff (t := scottTopologicalSpace)).mpr ⟨V i, hVi, rfl⟩
      exact iInf_le
        (fun i => TopologicalSpace.induced (fun w : ∀ j, E j => w i)
    scottTopologicalSpace) i _ hind
    · rw [Set.mem_iInter₂]
      exact fun i _ => hzV i

namespace proposition_2_9

theorem sierpinskiPower (ι : Type*) : ContinuousLattice (ι → Prop) :=
  (proposition_2_9 (fun _ => Prop) (fun _ => proposition_2_8)).1

end proposition_2_9

end DomainTheory.ContinuousLattice
```

2.3.7 Proposition 2.10 ✓

2.10 Proposition. A retract of a continuous lattice is a continuous lattice with the subspace topology agreeing with the induced topology.

It would seem that the continuous lattices are starting to sound suspiciously like the injective spaces. Indeed, if we can prove the following, the circle will be complete.

`proposition_2_10.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection

import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

universe u v

variable {D : Type u} {D' : Type v}
variable [CompleteLattice D] [CompleteLattice D']

theorem proposition_2_10 (R : ContinuousLatticeRetraction D D') (hD' :
    ContinuousLattice D') :
    ContinuousLattice D ∧
      (scottTopologicalSpace : TopologicalSpace D) =
        TopologicalSpace.induced (R.incl : D → D') scottTopologicalSpace := by
  constructor
  · refine { out := ?_ }
    intro d
    refine ⟨fun x hx => hx.le, fun b hb => ?_⟩
    rw [← R.sSup_image_retr_wayBelow hD' d]
    exact sSup_le fun _ ⟨x', hx', hxeq⟩ => hxeq ▸ hb (R.retr_wayBelow_of_wayBelow_incl
    hx')
  · apply le_antisymm
    · intro U hU
      rw [isOpen_induced_iff (t := scottTopologicalSpace)] at hU
      obtain ⟨V', hV'open, rfl⟩ := hU
      rw [ScottOpen.isOpen_iff] at hV'open ⊢
      exact scottOpen_preimage R.incl_preservesDirectedSup hV'open
    · intro U hU
      rw [ScottOpen.isOpen_iff] at hU
      rw [@isOpen_iff_forall_mem_open _
        (TopologicalSpace.induced (R.incl : D → D') scottTopologicalSpace)]
      intro d hd
      have hmem : sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) ∈ U := by
        rw [R.sSup_image_retr_wayBelow hD' d]; exact hd
      obtain ⟨p, hp, hpU⟩ :=
        hU.2 (R.image_retr_wayBelow_nonempty d) (R.image_retr_wayBelow_directed d)
    hmem
      obtain ⟨x', hx', rfl⟩ := hp
      refine ⟨(R.incl : D → D') ^-1' {z' | x' ≪ z'}, fun z hz => ?_, ?_, hx'⟩
      · have hjle := R.retr.monotone (show x' ≤ R.incl z from (hz : x' ≪ R.incl z).le)
        rw [R.retr_incl z] at hjle
        exact hU.1 hjle hpU
      · rw [isOpen_induced_iff (t := scottTopologicalSpace)]
        exact ⟨{z' | x' ≪ z'}, ScottOpen.isOpen_iff.mpr (ScottOpen.wayBelow x'), rfl⟩

end DomainTheory.ContinuousLattice
```

2.3.8 Proposition 2.11 ✓

2.11 Proposition. Every continuous lattice is an injective space under its induced topology.

proposition_2_11.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
```

```
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

theorem proposition_2_11 {E : Type*} [CompleteLattice E] (hE : ContinuousLattice E) :
    @InjectiveSpace E scottTopologicalSpace := by
  let : TopologicalSpace E := scottTopologicalSpace
  constructor
  intro X Y _ _ e he f
  exact ⟨⟨scottExtend e f, scottExtend_continuous hE e f⟩,
    fun x => scottExtend_eq_of_continuous hE e he f f.continuous x⟩

end DomainTheory.ContinuousLattice
```

2.3.9 Proof of 2.9 ✓

Proof of 2.9

Let D_i for $i \in I$ be a system of continuous lattices. The product

$$D^* = \prod_{i \in I} D_i$$

is a complete lattice in the usual way and has its induced topology. Suppose $y \in D^*$ and let $i \in I$. Then $y_i \in D_i$. Since D_i is a continuous lattice

$$y_i = \bigsqcup \{x \in D_i : x \ll y_i\}.$$

For $x \in D_i$, let $[x]^i \in D^*$ be defined by:

$$[x]^i_j = \begin{cases} x & \text{if } i = j, \\ \bot & \text{if } i \neq j. \end{cases}$$

Note that since D_i is continuous we have:

$$[y_i]^i = \bigsqcup \{[x]^i : x \ll y_i\},$$

and

$$y = \bigsqcup \{[y_i]^i : i \in I\}.$$

It follows that

$$y = \bigsqcup \bigcap \{ z : z_i \in U \text{ for all } i \in I,\ y_i \in U \},$$

where i ranges over I and U over the open subsets of D_i, because

$$[x]^i \sqsubseteq \bigcap \{ z : z_i \in U \}, \quad \text{where } U = \{ u \in D_i : x \ll u \}.$$

But the sets $\{ z : z_i \in U \}$ are open in the product sense, and so

$$y = \bigsqcup \bigcap \{ U : y \in U \},$$

where U ranges now over the open subsets of the product topology on D^*. By the remark following 2.4 we conclude that D^* is continuous with the lattice-induced topology being the product topology. □

2.3.10 Continuous lattice retraction ✓

Definition. For complete lattices D and D', a **continuous lattice retraction** of D into D' is a pair of Scott-continuous maps $i : D \to D'$ and $j : D' \to D$ such that $j \circ i = \mathrm{id}_D$. The library packages this as **`ContinuousLatticeRetraction D D'`** with **`incl`** (i), **`retr`** (j), and **`retr_incl`**.

In the namespace **`ContinuousLatticeRetraction`** we have

Theorem (`incl_preservesDirectedSup`). *The inclusion i preserves directed suprema: for every directed set $S \subseteq D$,*

$$i(\bigsqcup S) = \bigsqcup i(S).$$

Proof. i is Scott-continuous as a **`ScottMap`**, and that is exactly preservation of directed suprema. □

Theorem (`image_retr_wayBelow_directed`). *Fix $d \in D$. The set*

$$J_d := \{\, j(x') \mid x' \ll i(d) \,\}$$

is directed in D.

Proof. Given $j(a)$ and $j(b)$ with $a, b \ll i(d)$, the way-below relation is closed under finite suprema, so $a \sqcup b \ll i(d)$; monotonicity of j then yields an upper bound $j(a \sqcup b)$ in J_d below both elements. □

Theorem (`image_retr_wayBelow_nonempty`). *The set J_d is nonempty.*

Proof. $\bot \ll i(d)$ and $j(\bot) \in J_d$. □

Theorem (`sSup_image_retr_wayBelow`). *If D is a continuous lattice, then for every $d \in D$,*

$$\bigsqcup J_d = d.$$

Proof. The set $\{\, x' \mid x' \ll i(d) \,\}$ is directed and nonempty, so j preserves its supremum. On that set the continuous-lattice identity $i(d) = \bigsqcup\{x' \mid x' \ll i(d)\}$ and $j \circ i = \mathrm{id}_D$ give $\bigsqcup J_d = j(i(d)) = d$. □

Theorem (`retr_wayBelow_of_wayBelow_incl`). *If $x' \ll i(d)$ in D', then $j(x') \ll d$ in D.*

Proof. By the definition of $x' \ll i(d)$ there is a Scott-open $U' \subseteq D'$ with $i(d) \in U'$ and $U' \subseteq\!\uparrow x'$. The preimage $i^{-1}(U')$ is Scott-open because i preserves directed suprema; it contains d and every $z \in i^{-1}(U')$ satisfies $x' \le i(z)$, hence $j(x') \le j(i(z)) = z$. So $j(x') \ll d$. □

Definition (`ambientSSup`). *For $S \subseteq D$, the ambient supremum is $i(\bigsqcup S)$ in D' (the join taken in D, then mapped forward).*

Theorem (`retr_ambientSSup_eq_sSup`). *If $S \subseteq D$ is nonempty and directed, then*

$$j\big(i(\bigsqcup S)\big) = \bigsqcup S.$$

Proof. The image $i(S)$ is directed, so $i(\bigsqcup S) = \bigsqcup i(S)$ and j preserves that directed supremum. Pulling back along $j \circ i = \mathrm{id}_D$ on S yields the claim. □

`ContinuousLatticeRetraction.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Finset

variable {D : Type u} {D' : Type v}
variable [CompleteLattice D] [CompleteLattice D']

structure ContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

namespace ContinuousLatticeRetraction

theorem incl_preservesDirectedSup (R : ContinuousLatticeRetraction D D') :
    PreservesDirectedSup (R.incl : D → D') :=
  fun _ hS hSdir => R.incl.preservesDirectedSup_coe _ hS hSdir

theorem image_retr_wayBelow_directed (R : ContinuousLatticeRetraction D D') (d : D) :
    DirectedOn (· ≤ ·) (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) := by
  rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
  exact ⟨R.retr (a ⊔ b), ⟨a ⊔ b, WayBelow.sup ha hb, rfl⟩,
    R.retr.monotone le_sup_left, R.retr.monotone le_sup_right⟩

theorem image_retr_wayBelow_nonempty (R : ContinuousLatticeRetraction D D') (d : D) :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, WayBelow.bot _, rfl⟩

theorem sSup_image_retr_wayBelow (R : ContinuousLatticeRetraction D D') (hD' :
    ContinuousLattice D')
    (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, WayBelow.bot _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := WayBelow.directedOn _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]
```

```
theorem retr_wayBelow_of_wayBelow_incl (R : ContinuousLatticeRetraction D D') {d : D}
    {x' : D'}
    (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

noncomputable def ambientSSup (R : ContinuousLatticeRetraction D D') (S : Set D) : D'
    :=
  sSup (Set.image (R.incl : D → D') S)

theorem retr_ambientSSup_eq_sSup (R : ContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp

end ContinuousLatticeRetraction

end DomainTheory.ContinuousLattice
```

2.3.11 Converse retraction ✓

Definition. Let D and D' be complete lattices and let $i : D \to D'$ be Scott-continuous. The **converse retraction** along i is the map $j : D' \to D$ given by

$$j(x') = \bigsqcup \{x \in D \mid i(x) \le x'\}.$$

The library name is **`ConverseRetraction.ConverseRetraction i`** (`ConverseRetraction.lean`).

In the namespace **`ConverseRetraction`** we have

Theorem (`mono`). *The map $x' \mapsto j(x')$ is monotone.*

Proof. If $a' \le b'$, every x with $i(x) \le a'$ also satisfies $i(x) \le b'$, so the defining set for $j(a')$ is contained in that for $j(b')$ and $j(a') \le j(b')$. $\square$

Theorem (`preservesDirectedSup`). *Suppose $i(\bigsqcup S) = \bigsqcup i(S)$ for every $S \subseteq D$, that $x \ll y$ implies $i(x) \ll i(y)$, and that D is a continuous lattice. Then j preserves directed suprema.*

Proof. For directed $S' \subseteq D'$, one inequality follows from monotonicity: $j(\bigsqcup S')$ is an upper bound of $j(S')$. For the reverse, any x with $i(x) \le \bigsqcup S'$ is way-below that supremum in D', hence way-below some

$x' \in S'$ by directedness; then $x \le j(x') \le \bigsqcup j(S')$. Continuity of D rewrites $j(\bigsqcup S')$ as a supremum of such x. □

Theorem (`incl_converseRetr_le`). *If $i(\bigsqcup S) = \bigsqcup i(S)$ for all S, then for every $x' \in D'$, $i(j(x')) \le x'$.*

Proof. Each element of $\{x \mid i(x) \le x'\}$ maps under i into $\uparrow x'$; take the supremum. □

Theorem (`converseRetr_incl`). *Under the same $\bigsqcup$-preservation, if i is injective, then $j(i(y)) = y$.*

Proof. The set $\{x \mid i(x) \le i(y)\}$ is exactly $\downarrow y$; its supremum is y. □

`ConverseRetraction.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection

-- Namespace and map share the name `ConverseRetraction`.
set_option linter.dupNamespace false

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set

variable {D : Type u} {D' : Type v}
variable [CompleteLattice D] [CompleteLattice D']

namespace ConverseRetraction

noncomputable def ConverseRetraction (i : D → D') (x' : D') : D :=
  sSup {x | i x ≤ x'}

theorem mono (i : D → D') : Monotone (ConverseRetraction i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

theorem preservesDirectedSup (i : D → D')
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    PreservesDirectedSup (ConverseRetraction i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (ConverseRetraction i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
```

```
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact mono i (le_sSup hx'S')

theorem incl_converseRetr_le (i : D → D') (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S
    ))
    (x' : D') :
    i (ConverseRetraction i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

theorem converseRetr_incl (i : D → D') (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) :
    ConverseRetraction i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨ContinuousLatticeProjection.le_of_incl_le i hi hinj,
      fun hx => ContinuousLatticeProjection.incl_mono_of_preservesSSup i hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end ConverseRetraction

end DomainTheory.ContinuousLattice
```

2.3.12 Continuous lattice projection ✓

Definition. A **continuous lattice projection** is a continuous lattice retraction $i : D \to D'$, $j : D' \to D$ with $j \circ i = \mathrm{id}_D$ that also satisfies $i(j(x')) \le x'$ for all $x' \in D'$. The library extends **`ContinuousLatticeRetraction`** to **`ContinuousLatticeProjection D D'`** with the field **`incl_retr_le`**. Scott's Definition 3.6 uses this data for function-space extensions; Proposition 2.10 and later §3 results build on it.

In the namespace **`ContinuousLatticeProjection`** we have

Theorem (`retr_bot`). *For a projection P, $j(\bot_{D'}) = \bot_D$.*

Proof. $\bot_{D'} \le i(\bot_D)$, so $j(\bot_{D'}) \le j(i(\bot_D)) = \bot_D$; the reverse is $\bot_D \le j(\bot_{D'})$. □

Theorem (`incl_bot`). $i(\bot_D) = \bot_{D'}$.

Proof. By **`incl_retr_le`**, $i(j(\bot_{D'})) \le \bot_{D'}$; with **`retr_bot`**, $i(\bot_D) \le \bot_{D'}$. □

Theorem (`retr_eq_sSup`). *For every $x' \in D'$,*

$$j(x') = \bigsqcup \{\, x \in D \mid i(x) \le x' \,\}.$$

Proof. **`incl_retr_le`** gives $i(j(x')) \le x'$, so $j(x')$ lies in the set and is below its supremum. Any x with $i(x) \le x'$ satisfies $x = j(i(x)) \le j(x')$ by monotonicity of j. □

Theorem (`incl_wayBelow`). *If D is a continuous lattice and $x \ll y$ in D, then $i(x) \ll i(y)$ in D'.*

Proof. Rewrite y as a directed supremum of way-below elements in D' below $i(y)$, pull back along j, and use $x \ll y$ with **`incl_retr_le`** on the approximants. □

Theorem (`incl_injective`). *i is injective.*

Proof. From $i(x) = i(y)$ apply j and **`retr_incl`**. □

Theorem (`incl_sup`). $i(x \sqcup y) = i(x) \sqcup i(y)$.

Proof. Monotonicity gives one inequality; the other uses $x, y \le j(i(x) \sqcup i(y))$ from **`retr_incl`**, then **`incl_retr_le`** at $i(x) \sqcup i(y)$. □

Theorem (`incl_finset_sup`). *For finite $T \subseteq D$, $i(\bigsqcup T) = \bigsqcup i(T)$.*

Proof. Induction on T using **`incl_sup`** and **`incl_bot`**. □

Definition (`finsetSupOf`). *For $S \subseteq D$, let **`finsetSupOf S`** be the set of finite suprema $\bigsqcup T$ with $T \subseteq_{\text{fin}} S$.*

Theorem (`directedOn_finsetSupOf`). *`finsetSupOf S` is directed.*

Proof. Given two finite suprema from $T_1, T_2 \subseteq S$, their common upper bound is the supremum of $T_1 \cup T_2$. □

Theorem (`mem_finsetSupOf_of_mem`). *If $x \in S$, then $x \in \mathrm{finsetSupOf}\ S$.*

Proof. Take the singleton finset $\{x\}$. □

Theorem (`sSup_le_finsetSupOf`). $\bigsqcup \mathrm{finsetSupOf}\ S \le \bigsqcup S$.

Proof. Each finite supremum in **`finsetSupOf S`** is below $\bigsqcup S$. □

Theorem (`le_sSup_finsetSupOf`). $\bigsqcup S \le \bigsqcup \mathrm{finsetSupOf}\ S$.

Proof. Every $x \in S$ appears in **`finsetSupOf S`**. □

Theorem (`sSup_finsetSupOf`). $\bigsqcup S = \bigsqcup \mathrm{finsetSupOf}\ S$.

Proof. Antisymmetry of the two preceding inequalities. □

Theorem (`incl_sSup`). $i(\bigsqcup S) = \bigsqcup i(S)$.

Proof. Empty S uses **`incl_bot`**. Otherwise replace $\bigsqcup S$ by finite suprema via **`sSup_finsetSupOf`**, use directedness and **`incl_finset_sup`**, and compare with the image under i. □

Theorem (`incl_sup_of_preservesSSup`). *If $i(\bigsqcup S) = \bigsqcup i(S)$ for all S, then $i(x \sqcup y) = i(x) \sqcup i(y)$.*

Proof. Write $x \sqcup y$ as $\bigsqcup\{x, y\}$. □

Theorem (`incl_mono_of_preservesSSup`). *Under the same hypothesis, i is monotone.*

Proof. If $x \le y$, apply **`incl_sup_of_preservesSSup`** and read off $i(x) \le i(y)$. □

Theorem (`le_of_incl_le`). *If i is injective and preserves suprema of sets, then $i(x) \le i(y)$ implies $x \le y$.*

Proof. $i(x \sqcup y) = i(y)$ forces $x \sqcup y = y$ by injectivity. □

Building a projection from an inclusion i and the converse retraction is in Converse projection.

`ContinuousLatticeProjection.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeRetraction

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice
```

```
open Set Finset

variable {D : Type u} {D' : Type v}
variable [CompleteLattice D] [CompleteLattice D']

structure ContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
    extends ContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d

namespace ContinuousLatticeProjection

theorem retr_bot (P : ContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥ :=
    by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

theorem incl_bot (P : ContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

theorem retr_eq_sSup (P : ContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

theorem incl_wayBelow (P : ContinuousLatticeProjection D D') (hD' : ContinuousLattice
    D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @WayBelow.bot D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
```

```
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

theorem incl_injective (P : ContinuousLatticeProjection D D') {x y : D} (h : P.incl x
    = P.incl y) :
    x = y := by
  rw [← P.retr_incl x, ← P.retr_incl y, h]

theorem incl_sup (P : ContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
     y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

theorem incl_finset_sup (P : ContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

def finsetSupOf (S : Set D) : Set D :=
  { z | ∃ T : Finset D, (∀ x ∈ T, x ∈ S) ∧ z = T.sup id }

theorem directedOn_finsetSupOf (S : Set D) :
    DirectedOn (· ≤ ·) (finsetSupOf S) := by
  classical
  intro a ha b hb
  obtain ⟨T₁, hT₁, rfl⟩ := ha
  obtain ⟨T₂, hT₂, rfl⟩ := hb
  have hunion : (T₁ ∪ T₂).sup id = T₁.sup id ⊔ T₂.sup id := by
    simpa using Finset.sup_union (s₁ := T₁) (s₂ := T₂) (f := id)
  refine ⟨T₁.sup id ⊔ T₂.sup id, ⟨T₁ ∪ T₂, ?_, hunion.symm⟩, le_sup_left, le_sup_right⟩
  intro x hx
  simp only [Finset.mem_union] at hx
  exact hx.elim (hT₁ x) (hT₂ x)

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)
```

```
theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

theorem incl_sSup (P : ContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))

theorem incl_sup_of_preservesSSup (i : D → D') (hi : ∀ S : Set D, i (sSup S) = sSup (i
     '' S))
    (x y : D) :
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair

theorem incl_mono_of_preservesSSup (i : D → D') (hi : ∀ S : Set D, i (sSup S) = sSup (
    i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup i hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

theorem le_of_incl_le (i : D → D') (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup i hi, sup_eq_right.mpr
     h]
```

```
  exact sup_eq_right.mp (hinj h1)

end ContinuousLatticeProjection

end DomainTheory.ContinuousLattice
```

2.3.13 Converse projection ✓

When $i : D \to D'$ is injective, preserves suprema of sets, preserves way-below, and D is a continuous lattice, Scott builds a **continuous lattice projection** from i and the converse retraction (see Converse retraction). The library name is **`ConverseProjection i`** (`ConverseProjection.lean`).

Definition (`ConverseProjection`). *Under the hypotheses above, **`ConverseProjection i`** is the **`ContinuousLatticeProjection`** with inclusion i and retraction **`ConverseRetraction.ConverseRetrac i`**.

Proof. **`ConverseRetraction.converseRetr_incl`** and **`ConverseRetraction.incl_converseRetr_le`** supply **`retr_incl`** and **`incl_retr_le`**; Scott continuity comes from **`ConverseRetraction.`** □

`ConverseProjection.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.ConverseRetraction

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set

variable {D : Type u} {D' : Type v}
variable [CompleteLattice D] [CompleteLattice D']

noncomputable def ConverseProjection (i : D → D')
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    ContinuousLatticeProjection D D' where
  incl := ⟨i, PreservesDirectedSup.continuous_of (fun _ _ _ => hi _)⟩
  retr := ⟨ConverseRetraction.ConverseRetraction i, PreservesDirectedSup.continuous_of
    (ConverseRetraction.preservesDirectedSup i hi hwb hD)⟩
  retr_incl := fun d => ConverseRetraction.converseRetr_incl i hi hinj d
  incl_retr_le := fun x' => ConverseRetraction.incl_converseRetr_le i hi x'

end DomainTheory.ContinuousLattice
```

2.3.14 Proof of 2.10 ✓

Proof of 2.10

Let D' be a continuous lattice and let $D \subseteq D'$ be a subspace which is a retract. We have for a suitable $j : D' \to D$,

$$D = \{x \in D' : j(x) = x\},$$

where of course j is continuous.

First a note of warning: though D is a subspace it is not a sublattice; that is, the partial ordering on D is the restriction of that of D', but the lubs of D are not those of D'. We shall have to distinguish operations by adding a prime ($'$) for those of D'.

Suppose $x, y \in D$. Let $z' = x \sqcup y \in D'$ (join taken in D') and define $z = j(z') \in D$. Now $x \sqsubseteq z'$ and $y \sqsubseteq z'$, and j is monotonic, so $x \sqsubseteq z$ and $y \sqsubseteq z$. Suppose $x \sqsubseteq w$ and $y \sqsubseteq w$ with $w \in D$. Then in D' we have $x \sqcup y \sqsubseteq w$; so $z \sqsubseteq w$ also. Hence we have shown that $z = x \sqcup y$ in D.

To show that D has a least element $\bot$ (which may be larger than the $\bot' \in D'$), we need a well-known lemma about monotonic functions: Every monotonic function on a complete lattice into itself has a least fixed point. (Cf. [30, p. 115].) In our case j is monotonic and

$$\bot = \sqcap \{x \in D' : j(x) \sqsubseteq x\}$$

is the desired element in D.

Thus D is at least a semilattice with $\bot$ and $\sqcup$. To show that D is a lattice we need to show that every directed $S \subseteq D$ has a lub in D. Now we know ($S' = S$, join taken in the ambient lattice D'):

$$\bigsqcup S' \in D',$$

and this is a limit of a monotone net. So by 2.1, and the continuity of j:

$$j(\bigsqcup S') = \bigsqcup\{j(x) : x \in S\} \quad = \bigsqcup S$$

in D. In this way we now know that D is a complete lattice. We must still show that D is continuous.

Suppose $y \in D$. In D' we can write:

$$y = \bigsqcup\{x \in D' : x \ll y'\}$$

and this is the limit of a monotone net. Thus

$$j(y) = y = \bigsqcup j(x) : x \ll y, x \in D',$$

where the lub is taken in D. Note that the sets

$$U = \{z \in D : x \ll z\}$$

are open in D for each $x \in D'$. Note too that if $z \in U$, then $x \sqsubseteq z$ and so $j(x) \sqsubseteq j(z) = z$. This means that

$$j(x) \sqsubseteq \sqcap U$$

in D. We can then write in D:

$$y = \bigsqcup\{\sqcap U : y \in U\}$$

where U ranges over the open subsets of D, and so the lattice is continuous by 2.4. Inasmuch as the open sets U just used were open in the subspace topology, it follows by the remark after 2.4 that the subspace and the lattice-induced topologies coincide. □

2.3.15 Proof of 2.11 ✓

Proof of 2.11: Let D be a continuous lattice with its induced topology, and let $X \subseteq Y$ be two T_0-spaces in the subspace relation. Suppose

$$f : X \to D$$

is continuous. Define

$$\bar{f} : Y \to D$$

by the formula:

$$\bar{f}(y) = \bigsqcup \{ \sqcap \{ f(x) : x \in X \cap U \} : y \in U \},$$

where U ranges over the open subsets of Y. We need to show that $\bar{f}$ extends f and that it is continuous.

First, the continuity: Suppose that $d \in D$ and $d \ll \bar{f}(y)$; that is, $\bar{f}(y)$ belongs to a typical basic open subset of D. Since D is continuous, we can also find

$$d \ll d' \ll \bar{f}(y)$$

with $d' \in D$. From the definition of $\bar{f}$ it follows that

$$d' \sqsubseteq \sqcap \{ f(x) : x \in X \cap U \}$$

for some open $U \subseteq Y$ with $y \in U$. Thus

$$d' \sqsubseteq \bar{f}(y')$$

for all $y' \in U$ by virtue of the definition of $\bar{f}$. Since $d \ll d'$, we have also

$$d \ll \bar{f}(y')$$

for all $y' \in U$; in other words, the inverse image of the open subset of D determined by d under $\bar{f}$ is indeed open in Y, and $\bar{f}$ is thus continuous.

Next, the extension property: Note that the relationship

$$\bar{f}(x') \sqsubseteq f(x')$$

for all $x' \in X$ comes directly out of the definition of $\bar{f}$. For the converse, suppose $d \ll f(x')$ where $d \in D$. By assumption $f : X \to D$ is continuous, so

$$d \ll f(x'')$$

for all $x'' \in X \cap U$ where U is a suitable open subset of Y with $x' \in U$. In particular we have:

$$d \sqsubseteq \sqcap \{ f(x'') : x'' \in X \cap U \},$$

and so $d \sqsubseteq \bar{f}(x')$. Since $d \ll f(x')$ always implies $d \sqsubseteq \bar{f}(x')$, we see that $f(x') \sqsubseteq \bar{f}(x')$ follows by the continuity of D, and the proof is complete. □

scottExtend_aux_nonempty.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

omit [TopologicalSpace X] in
theorem scottExtend_aux_nonempty (e : X → Y) (f : X → E) (y : Y) :
    {d | ∃ V : Set Y, IsOpen V ∧ y ∈ V ∧ d = sInf (f '' (e ^-1' V))}.Nonempty :=
  ⟨_, Set.univ, isOpen_univ, Set.mem_univ y, rfl⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottTopology_prop.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

theorem scottTopology_prop :
    (scottTopologicalSpace : TopologicalSpace Prop) = sierpinskiSpace := by
  apply le_antisymm
  ·
    show (scottTopologicalSpace : TopologicalSpace Prop) ≤ TopologicalSpace.
    generateFrom {{True}}
    apply le_generateFrom
    intro s hs
    rw [Set.mem_singleton_iff] at hs
```

```
    subst hs
    rw [ScottOpen.isOpen_iff]
    refine ⟨?_, ?_⟩
    · intro a b hab ha
      rw [Set.mem_singleton_iff] at ha ⊢
      exact le_antisymm le_top (ha ▸ hab)
    · intro S _ _ hmem
      rw [Set.mem_singleton_iff] at hmem
      have hex : ∃ p ∈ S, p := by rw [← sSup_Prop_eq, hmem]; trivial
      obtain ⟨p, hpS, hp⟩ := hex
      exact ⟨p, hpS, by rw [Set.mem_singleton_iff]; exact eq_true hp⟩
  .
    rw [TopologicalSpace.le_def]
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    by_cases hT : True ∈ U
    · by_cases hF : False ∈ U
      · have hUuniv : U = Set.univ := by
          ext p
          simp only [Set.mem_univ, iff_true]
          by_cases hp : p
          · rwa [eq_true hp]
          · rwa [eq_false hp]
        rw [hUuniv]; exact isOpen_univ
      · have hUtrue : U = {True} := by
          ext p
          rw [Set.mem_singleton_iff]
          constructor
          · intro hpU
            by_cases hp : p
            · exact eq_true hp
            · exact absurd (eq_false hp ▸ hpU) hF
          · intro hp; rw [hp]; exact hT
        rw [hUtrue]; exact isOpen_singleton_true
    · have hUempty : U = ∅ := by
        ext p
        simp only [Set.mem_empty_iff_false, iff_false]
        intro hpU
        exact hT (hU.1 le_top hpU)
      rw [hUempty]; exact isOpen_empty

end DomainTheory.ContinuousLattice
```

scottExtend_aux_directed.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set
```

```
variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

omit [TopologicalSpace X] in

theorem scottExtend_aux_directed (e : X → Y) (f : X → E) (y : Y) :
    DirectedOn (· ≤ ·) {d | ∃ V : Set Y, IsOpen V ∧ y ∈ V ∧ d = sInf (f '' (e ^-1' V))}
     := by
  rintro _ ⟨V₁, hV₁o, hyV₁, rfl⟩ _ ⟨V₂, hV₂o, hyV₂, rfl⟩
  refine ⟨sInf (f '' (e ^-1' (V₁ ∩ V₂))), ⟨V₁ ∩ V₂, hV₁o.inter hV₂o, ⟨hyV₁, hyV₂⟩, rfl⟩, ?
    _, ?_⟩
  · exact sInf_le_sInf (Set.image_mono (Set.preimage_mono Set.inter_subset_left))
  · exact sInf_le_sInf (Set.image_mono (Set.preimage_mono Set.inter_subset_right))

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

piWayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import Mathlib.Data.Finset.Lattice.Basic
public import Mathlib.Data.Finset.Insert
public import Mathlib.Data.Finset.Defs
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

namespace piWayBelow

variable {D : Type*} [CompleteLattice D]

theorem update_preservesDirectedSup {ι : Type*} [DecidableEq ι] {E : ι → Type*}
    [∀ i, CompleteLattice (E i)] (z : ∀ i, E i) (i : ι) :
    PreservesDirectedSup (fun v : E i => Function.update z i v) := by
  intro T hTne _
  show Function.update z i (sSup T) = sSup ((fun v : E i => Function.update z i v) ''
    T)
  funext j
  rw [sSup_apply_eq_sSup_image, Set.image_image]
  rcases eq_or_ne j i with hji | hji
  · rw [hji, Function.update_self]
    have h : (fun v : E i => Function.eval i (Function.update z i v)) = id := by
      funext v; simp [Function.eval, Function.update_self]
```

```
    rw [h, Set.image_id]
  · rw [Function.update_of_ne hji]
    have h : (fun v : E i => Function.eval j (Function.update z i v)) = fun _ => z j
    := by
      funext v; simp [Function.eval, Function.update_of_ne hji]
    rw [h, hTne.image_const, sSup_singleton]

theorem wayBelow_proj {ι : Type*} {E : ι → Type*} [∀ i, CompleteLattice (E i)]
    {a z : ∀ i, E i} (h : a ≪ z) (i : ι) : a i ≪ z i := by
  classical
  obtain ⟨W, hW, hzW, hWsub⟩ := h
  refine ⟨(fun v : E i => Function.update z i v) ^-1' W, ?_, ?_, ?_⟩
  · exact scottOpen_preimage (update_preservesDirectedSup z i) hW
  · show Function.update z i (z i) ∈ W
    rw [Function.update_eq_self]; exact hzW
  · intro v hv
    have hav : a ≤ Function.update z i v := Set.mem_Ici.1 (hWsub hv)
    have := hav i
    rwa [Function.update_self] at this

theorem wayBelow_finite_support {ι : Type*} {E : ι → Type*} [∀ i, CompleteLattice (E i)
    ]
    {a z : ∀ i, E i} (h : a ≪ z) : ∃ F : Finset ι, ∀ j ∈ F, a j = ⊥ := by
  classical
  set Z : Finset ι → (∀ j, E j) := fun F j => if j ∈ F then z j else ⊥ with hZ
  set 𝒵 : Set (∀ j, E j) := Set.range Z with h𝒵
  have hmono : Monotone Z := by
    intro F G hFG
    rw [Pi.le_def]; intro j
    simp only [hZ]
    by_cases hjF : j ∈ F
    · rw [ite_eq_left hjF, ite_eq_left (hFG hjF)]
    · rw [ite_eq_right hjF]; exact bot_le
  have h𝒵ne : 𝒵.Nonempty := ⟨Z ∅, ∅, rfl⟩
  have h𝒵dir : DirectedOn (· ≤ ·) 𝒵 := by
    rintro _ ⟨F, rfl⟩ _ ⟨G, rfl⟩
    exact ⟨Z (F ∪ G), ⟨F ∪ G, rfl⟩, hmono Finset.subset_union_left,
      hmono Finset.subset_union_right⟩
  have hsup : sSup 𝒵 = z := by
    apply le_antisymm
    · apply sSup_le
      rintro _ ⟨F, rfl⟩
      rw [Pi.le_def]; intro j
      simp only [hZ]
      by_cases hjF : j ∈ F
      · rw [ite_eq_left hjF]
      · rw [ite_eq_right hjF]; exact bot_le
    · rw [Pi.le_def]; intro j
      rw [sSup_apply_eq_sSup_image]
      refine le_sSup ⟨Z {j}, ⟨{j}, rfl⟩, ?_⟩
      simp [hZ, Function.eval]
  rw [← hsup] at h
  obtain ⟨d, hd𝒵, had⟩ := (WayBelow.sSup_iff h𝒵ne h𝒵dir).1 h
  obtain ⟨F, rfl⟩ := hd𝒵
  refine ⟨F, fun j hjF => ?_⟩
  have hj := had.le j
```

```
  simp only [hZ, ite_eq_right hjF] at hj
  exact le_bot_iff.1 hj

end piWayBelow

end DomainTheory.ContinuousLattice
```

continuous_eq_sSup_openInfs.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

theorem continuous_eq_sSup_openInfs (hE : ContinuousLattice E) {f' : Y → E}
    (hf' : @Continuous Y E _ scottTopologicalSpace f') (y : Y) :
    f' y = sSup {d | ∃ U : Set Y, IsOpen U ∧ y ∈ U ∧ d = sInf (f' '' U)} := by
  apply le_antisymm
  · rw [← hE.sSup_wayBelow (f' y)]
    refine sSup_le fun a ha => ?_
    have hWopen : @IsOpen E scottTopologicalSpace {z : E | a ≪ z} :=
      ScottOpen.isOpen_iff.mpr (ScottOpen.wayBelow a)
    have hpre : IsOpen (f' ^-1' {z : E | a ≪ z}) := continuous_def.mp hf' _ hWopen
    have hyU : y ∈ f' ^-1' {z : E | a ≪ z} := ha
    refine le_trans ?_ (le_sSup ⟨f' ^-1' {z : E | a ≪ z}, hpre, hyU, rfl⟩)
    refine le_sInf ?_
    rintro w ⟨z, hzU, rfl⟩
    exact (hzU : a ≪ f' z).le
  · refine sSup_le ?_
    rintro d ⟨U, _hUo, hyU, rfl⟩
    exact sInf_le ⟨y, hyU, rfl⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottExtend.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

def scottExtend (e : X → Y) (f : X → E) (y : Y) : E :=
  sSup {d | ∃ V : Set Y, IsOpen V ∧ y ∈ V ∧ d = sInf (f '' (e ^-1' V))}

end InjectiveExtension

end DomainTheory.ContinuousLattice
```

scottExtend_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.Constructions.scottExtend
import DomainTheory.ContinuousLattice.Constructions.scottExtend_aux_directed
import DomainTheory.ContinuousLattice.Constructions.scottExtend_aux_nonempty

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

omit [TopologicalSpace X] in

theorem scottExtend_continuous (hE : ContinuousLattice E) (e : X → Y) (f : X → E) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f) := by
  let : TopologicalSpace E := scottTopologicalSpace
  rw [continuous_def]
  intro U hU
  rw [ScottOpen.isOpen_iff] at hU
  rw [isOpen_iff_forall_mem_open]
  intro y₀ hy₀
  have hgy₀U : scottExtend e f y₀ ∈ U := hy₀
  obtain ⟨a, haz, hasub⟩ := exists_wayBelow_subset hE hU hgy₀U
  obtain ⟨d, hd, had⟩ := (WayBelow.sSup_iff (scottExtend_aux_nonempty e f y₀)
    (scottExtend_aux_directed e f y₀)).1 haz
  obtain ⟨V, hVo, hy₀V, rfl⟩ := hd
```

```
  refine ⟨V, ?_, hVo, hy₀V⟩
  intro y' hy'V
  show scottExtend e f y' ∈ U
  refine hasub ?_
  show a ≪ scottExtend e f y'
  exact had.trans_le (le_sSup ⟨V, hVo, hy'V, rfl⟩)

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottExtend_eq_of_continuous.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Constructions.scottExtend

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

theorem scottExtend_eq_of_continuous (hE : ContinuousLattice E) (e : X → Y)
    (he : IsEmbedding e) (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f)
    (x₀ : X) :
    scottExtend e f (e x₀) = f x₀ := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro d ⟨V, hVo, hex₀V, rfl⟩
    exact sInf_le ⟨x₀, hex₀V, rfl⟩
  · rw [← hE.sSup_wayBelow (f x₀)]
    refine sSup_le fun a ha => ?_
    have hWopen : @IsOpen E scottTopologicalSpace {z : E | a ≪ z} :=
      ScottOpen.isOpen_iff.mpr (ScottOpen.wayBelow a)
    have hpre : IsOpen (f ^-1' {z : E | a ≪ z}) := continuous_def.mp hf _ hWopen
    rw [he.isInducing.isOpen_iff] at hpre
    obtain ⟨V, hVo, hVeq⟩ := hpre
    have hx₀V : x₀ ∈ e ^-1' V := by rw [hVeq]; exact ha
    refine le_trans ?_ (le_sSup ⟨V, hVo, hx₀V, rfl⟩)
    refine le_sInf ?_
    rintro w ⟨x, hxV, rfl⟩
    have hxW : x ∈ f ^-1' {z : E | a ≪ z} := by rw [← hVeq]; exact hxV
    exact (hxW : a ≪ f x).le

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottTopology.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottTopology_prop
public import DomainTheory.ContinuousLattice.Constructions.proposition_2_9
public import DomainTheory.ContinuousLattice.Constructions.proposition_2_8
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

namespace scottTopology

theorem sierpinskiPower (ι : Type*) :
    (scottTopologicalSpace : TopologicalSpace (ι → Prop)) =
      (inferInstance : TopologicalSpace (ι → Prop)) := by
  rw [(proposition_2_9 (fun _ => Prop) (fun _ => proposition_2_8)).2]
  show (@Pi.topologicalSpace ι (fun _ => Prop) (fun _ => scottTopologicalSpace)) =
    @Pi.topologicalSpace ι (fun _ => Prop) (fun _ => sierpinskiSpace)
  congr 1
  funext _
  exact scottTopology_prop

end scottTopology

end DomainTheory.ContinuousLattice
```

scottExtend_maximal.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
import DomainTheory.ContinuousLattice.Constructions.continuous_eq_sSup_openInfs

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

omit [TopologicalSpace X] in
```

```
theorem scottExtend_maximal (hE : ContinuousLattice E) (e : X → Y) {f : X → E} {f' :
    Y → E}
    (hf' : @Continuous Y E _ scottTopologicalSpace f') (h_ext : ∀ x, f' (e x) = f x) (y
     : Y) :
    f' y ≤ scottExtend e f y := by
  rw [continuous_eq_sSup_openInfs hE hf' y]
  refine sSup_le ?_
  rintro d ⟨U, hUo, hyU, rfl⟩
  refine le_trans ?_ (le_sSup ⟨U, hUo, hyU, rfl⟩)
  refine le_sInf ?_
  rintro w ⟨x, hxU, rfl⟩
  rw [← h_ext x]
  exact sInf_le ⟨e x, hxU, rfl⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottExtend_maximal_le.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
import DomainTheory.ContinuousLattice.Constructions.continuous_eq_sSup_openInfs

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

omit [TopologicalSpace X] in

theorem scottExtend_maximal_le (hE : ContinuousLattice E) (e : X → Y) {g : X → E} {f'
     : Y → E}
    (hf' : @Continuous Y E _ scottTopologicalSpace f') (h_le : ∀ x, f' (e x) ≤ g x) (y
     : Y) :
    f' y ≤ scottExtend e g y := by
  rw [continuous_eq_sSup_openInfs hE hf' y]
  refine sSup_le ?_
  rintro d ⟨U, hUo, hyU, rfl⟩
  refine le_trans ?_ (le_sSup ⟨U, hUo, hyU, rfl⟩)
  refine le_sInf ?_
  rintro w ⟨x, hxU, rfl⟩
  exact le_trans (sInf_le ⟨e x, hxU, rfl⟩) (h_le x)

end InjectiveExtension
```

```
end DomainTheory.ContinuousLattice
```

2.4 Function spaces ✓

We recall the standard definition and introduce our notation for function spaces.



Figure 11: Function spaces ✓.

2.4.1 Induced pi topology on $C(X, Y)$ ✓

Library packaging for the product (pointwise) topology on continuous maps between T_0-spaces, before Definition 3.1 specializes the eval subbase on **`C(X, Y)`**.

For T_0-spaces X and Y, Definition 3.1 identifies $[X \to Y]$ with **`C(X, Y)`** and equips it with the *product topology* (topology of pointwise convergence). Mathlib already carries the product topology on Y^X; restricting to continuous maps gives the subspace topology induced along the inclusion **`C(X, Y)`** $\hookrightarrow$ **`X → Y`**.

Definition (`continuousMapInducedPiTopology`). **`continuousMapInducedPiTopology X Y`** is that induced topology on **`C(X, Y)`**: the pi (product) topology on Y^X, pulled back along `(f : C(X, Y)) ↦ (f : X → Y)`.

This definition is the generic T_0 packaging used before lattices enter the story. On **`ScottFunctionTopology.lean`**, **`scottFunctionTopology X Y`** is built from the eval subbase of Definition 3.1; **`scottFunctionTopology.eq_induced_pi`** proves **`scottFunctionTopology X Y = continuousMapInducedPiTopology X Y`**, so Scott's subbase description and the product topology coincide on **`C(X, Y)`**.

`continuousMapInducedPiTopology.lean`

```
module
public import Mathlib.Topology.Constructions

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[reducible] noncomputable def continuousMapInducedPiTopology (X Y : Type*)
    [TopologicalSpace X] [TopologicalSpace Y] : TopologicalSpace C(X, Y) :=
  TopologicalSpace.induced (fun f : C(X, Y) => (f : X → Y))
    (@Pi.topologicalSpace X (fun _ => Y) (fun _ => inferInstance))

end DomainTheory.ContinuousLattice
```

2.4.2 Definition 3.1: Function space ✓

3.1 Definition. For T_0-spaces X and Y we let $[X \to Y]$ be the *space of all continuous functions* $f : X \to Y$ endowed with the product topology, sometimes called the topology of pointwise convergence. This topology has as a subbase sets of the form:

$$\{f : f(x) \in U\}$$

where $x \in X$ and $U \subseteq Y$ is open.

The pointwise aspect of the topology is immediately apparent in the partial ordering.

For T_0-spaces X and Y, the library identifies $[X \to Y]$ with **`C(X, Y).scottFunctionTopology.evalSubbasi`**
`X Y` is the family of subbasic sets $\{ f \in C(X,Y) \mid f(x) \in U \}$ with $x \in X$ and $U \subseteq Y$ open. **`scottFunctionTopology X Y`** is the topology generated from that subbase.

The lemmas in namespace **`scottFunctionTopology`** justify this packaging:

Lemma (`subbasis_isOpen`). For open $U \subseteq Y$ and $x \in X$, the subbasic set $\{ f \mid f(x) \in U \}$ is open in `scottFunctionTopology X Y`.

Lemma (`subbasis_isOpen_scott`). When D and D' are complete lattices and $U \subseteq D'$ is open in **`scottTopologicalSpace`**, the same subbasic set $\{ f \in C(D, D') \mid f(x) \in U \}$ is open in **`scottFunctionTopology D D'`** with both domain and codomain carrying the Scott topology (used in Corollary 3.4).

Theorem (`eq_induced_pi`). `scottFunctionTopology X Y` is the product topology on Y^X (pointwise convergence), induced along the inclusion of continuous maps into all maps $X \to Y$.

Lemma (`specializationPreorder_generateOpen`). If $f(x) \sqsubseteq g(x)$ for every $x \in X$, and $V \subseteq C(X,Y)$ is open in the topology generated from the eval subbase with $f \in V$, then $g \in V$.

Theorem (`specializationPreorder`). With the specialization preorder from `scottFunctionTopology X Y`, $f \sqsubseteq g$ iff $f(x) \sqsubseteq g(x)$ for all $x \in X$ — the pointwise specialization order on the function space (the content of Proposition 3.2, proved there as `proposition_3_2` via this lemma).

ScottFunctionTopology.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationPreorder
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    continuousMapInducedPiTopology
public import Mathlib.Topology.ContinuousMap.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

abbrev ScottFunctionSpace (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y] :=
  C(X, Y)

abbrev ScottC (D D' : Type*) [CompleteLattice D] [CompleteLattice D'] :=
  @ContinuousMap D D' scottTopologicalSpace scottTopologicalSpace

def scottFunctionTopology.evalSubbasis (X Y : Type*) [TopologicalSpace X] [
    TopologicalSpace Y] :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

@[reducible] noncomputable def scottFunctionTopology (X Y : Type*) [TopologicalSpace X
    ]
    [TopologicalSpace Y] : TopologicalSpace (C(X, Y)) :=
  generateFrom (scottFunctionTopology.evalSubbasis X Y)

namespace scottFunctionTopology

section

variable [TopologicalSpace X] [TopologicalSpace Y]

theorem subbasis_isOpen {x : X} {U : Set Y} (hU : IsOpen U) :
    @IsOpen (C(X, Y)) (scottFunctionTopology X Y) {f : C(X, Y) | f x ∈ U} :=
  isOpen_generateFrom_of_mem (s := {f : C(X, Y) | f x ∈ U}) ⟨x, U, hU, rfl⟩

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

theorem subbasis_isOpen_scott {x : D} {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    letI : TopologicalSpace D := scottTopologicalSpace
    letI : TopologicalSpace D' := scottTopologicalSpace
    @IsOpen (C(D, D')) (scottFunctionTopology D D') {f : C(D, D') | f x ∈ U} := by
  let : TopologicalSpace D := scottTopologicalSpace
  let : TopologicalSpace D' := scottTopologicalSpace
  exact subbasis_isOpen (x := x) hU

theorem eq_induced_pi (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y] :
```

```
    scottFunctionTopology X Y = continuousMapInducedPiTopology X Y := by
  apply le_antisymm
  · let : TopologicalSpace C(X, Y) := scottFunctionTopology X Y
    change scottFunctionTopology X Y ≤ continuousMapInducedPiTopology X Y
    rw [← continuous_iff_le_induced]
    exact continuous_pi fun x =>
      continuous_def.2 fun U hU =>
        subbasis_isOpen (X := X) (Y := Y) (x := x) hU
  · apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hU, rfl⟩
    change @IsOpen C(X, Y)
      (TopologicalSpace.induced (fun f : C(X, Y) => (f : X → Y))
        (@Pi.topologicalSpace X (fun _ => Y) (fun _ => inferInstance)))
      {f : C(X, Y) | f x ∈ U}
    rw [isOpen_induced_iff]
    exact ⟨{g : X → Y | g x ∈ U}, (@continuous_apply X (fun _ => Y)
      (fun _ => inferInstance) x).isOpen_preimage U hU, rfl⟩

theorem specializationPreorder_generateOpen (f g : C(X, Y)) (hfg : ∀ x,
    SpecializationPreorder (f x) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (evalSubbasis X Y) V) (hf : f ∈ V) :
    g ∈ V :=
  match hV with
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationPreorder_generateOpen f g hfg hs hfs)
      (specializationPreorder_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationPreorder_generateOpen f g hfg (hS W
     hWS) hfW⟩

theorem specializationPreorder {f g : C(X, Y)} :
    letI := scottFunctionTopology X Y
    SpecializationPreorder f g ↔ ∀ x, SpecializationPreorder (f x) (g x) := by
  let : TopologicalSpace (C(X, Y)) := scottFunctionTopology X Y
  constructor
  · intro hfg x U hU hfU
    exact hfg {h | h x ∈ U} (subbasis_isOpen (X := X) (Y := Y) hU) hfU
  · intro hfg U hU hfU
    exact specializationPreorder_generateOpen f g hfg hU hfU

end

end scottFunctionTopology

end DomainTheory.ContinuousLattice
```

2.4.3 Elements of $[D \to D']$ ✓

For continuous lattices D and D', Scott's §3 notation $[D \to D']$ is the set of maps $f : D \to D'$ that are continuous for the Scott topologies (equivalently, Scott-continuous in the sense of §2). The library names that carrier **`ScottMap D D'`**: a subtype of $D \to D'$ carrying a proof of continuity for **`scottTopologicalSpace`** on domain and codomain.

`ScottMap.lean` is **self-contained**: it does not import or mention **`stepFun`** or **`stepMap`**. Step functions for Theorem 3.3 are in Step function (pointwise) and Step map $\overrightarrow{e}[e, e']$. Scott's projection space J_D (**`ProjectionSpace`**, Definition 3.11) and sup-closure (Proposition 3.12) are in Definition 3.11: Projection space. Retractions between map spaces (**`ScottMap.retraction`**) are in Retractions and extensions. **`FunctionSpace.retraction`** and **`FunctionSpace.projection`** are in Function-space retraction and projection; Proposition 3.7 is the paper statement. Projections on map spaces and **`ScottMap.conMinContinuousLatticeProjection`** are in Definition 3.6; **`con`** / **`min`** appear in Proposition 3.13.

Namespace **`ScottMap`** collects the library tools on that carrier:

Instance (`CoeFun`). A Scott map coerces to its underlying function $D \to D'$.

Lemma (`ext`). Two Scott maps agree if they agree at every argument.

Definition (`const`). The constant Scott map at $c \in D'$; **`const_apply`** reads off c at each x.

Definition (`con`). Scott's $\mathrm{con} : D \to [D \to D]$: at x the constant endo-map $\mathrm{con}(x) = \mathrm{const}\, x$.

Definition (`min`). Scott's $\mathrm{min} : [D \to D] \to D$: **`min f = f(⊥)`** on the coercion.

Definition (`toOrderHom`). **`OrderHom`** view of $f \in [D \to D]$ for least-fixed-point arguments.

Definition (`fix`). Scott's fixed-point operator on endo-maps $[D \to D]$ (least fixpoint via **`toOrderHom`**).

Lemma (`fix_le`). **`fix f ≤ x`** from any prefix point **`f x ≤ x`**.

Lemma (`fix_eq`). **`f (fix f) = fix f`**.

Lemma (`fix_mono`). **`fix`** is monotone in the Scott map argument.

Lemma (`fix_unique`). Any least prefix point agrees with **`fix f`**.

Lemma (`fix_preservesDirectedSup`). **`fix : [D \to D] \to D`** preserves directed suprema.

Definition (`fixMap`). The Scott-continuous **`[D \to D] \to D`** map packaging **`fix`**; **`fixMap_apply`**.

Definition (`le`), `lt`. Pointwise order: $f \le g$ when $f(x) \le g(x)$ for all x.

Lemma (`le_refl'`). Reflexivity of the pointwise order.

Lemma (`le_trans'`). Transitivity.

Lemma (`le_antisymm'`). Antisymmetry.

Lemma (`lt_iff_le_not_ge'`). Strict order as "$\le$ and not $\ge$"; feeds **`instPartialOrder`**.

Instance (`instPartialOrder`). Pointwise order on **`ScottMap D D'`**.

Lemma (`le_def`). **`f ≤ g`** iff pointwise inequality on the coercion.

Lemma (`preservesDirectedSup_coe`). For nonempty directed $S \subseteq D$, $f(\bigsqcup S) = \bigsqcup f[S]$ (Proposition 2.5 on the coercion).

Lemma (`monotone`). The underlying function is monotone.

Lemma (`pointwise_sSup_preservesDirectedSup`). For $\mathcal{F} \subseteq [D \to D']$, the map $x \mapsto \bigsqcup\{f(x) \mid f \in \mathcal{F}\}$ preserves directed suprema.

Lemma (`sSupMaps_continuous`). That pointwise sup is Scott-continuous.

Definition (`sSupMaps`). The Scott map whose value at x is that pointwise sup; **`sSupMaps_apply`** is definitional.

Instance (`instSupSet`). **`sSup F`** is **`sSupMaps F`**.

Lemma (`sSup_apply`). `(sSup F) x = \bigsqcup \{ f(x) \mid f \in F \}`.

Theorem (`isLUB_sSup`). `sSup F` is a least upper bound of F in the pointwise order.

Definition (`instCompleteLattice`). `ScottMap D D'` is a complete lattice via `completeLatticeOfSup` (sup-only here; meets are inherited from the complete lattice structure on D' at each point).

Lemma (`preservesDirectedSup_comp`). If $f : D' \to D''$ and $g : D \to D'$ preserve directed suprema, so does $f \circ g$.

Definition (`comp`). Composition of Scott maps; `comp_apply` is `(f \circ g)(x) = f(g(x))`.

Lemma (`continuous`). The subtype proof: Scott continuity of the coercion.

Definition (`idMap`). The identity Scott endo-map; `idMap_apply` is reflexivity at x.

Lemma (`bot_apply`). `(⊥) x` $= \bot$ in D'.

Lemma (`sup_apply`). `(f \sqcup g)(x) = f(x) \sqcup g(x)`.

Lemma (`sInf_image_le_of_mem`). For Scott-open $U \ni y$, $\sqcap f'[U] \sqsubseteq f'(y)$ on the coercion $f' : D \to D'$.

Theorem (`eq_sSup_openInfs`). When D' is a continuous lattice, $f'(y)$ is the directed supremum over Scott-open $U \ni y$ of the values $\sqcap f'[U]$ (open-set `sInf` representation of a Scott map at a point).

Scott's *continuous* lattice structure on $[D \to D']$ (`ContinuousLattice (ScottMap D D')`) is proved later via `stepMap.isContinuousLattice`, not in this file.

`scottContinuous.lean`

```
module

public import DomainTheory.ContinuousLattice.ContinuousLattices.IdempotentFixedPoint.
    completeLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_2_10
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {L : Type*} [CompleteLattice L] {e : L → L}

namespace IdempotentFixedPoint

theorem incl_preservesDirectedSup (hidem : ∀ x, e (e x) = e x) (hsc :
    PreservesDirectedSup e) :
    letI := IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone
    hsc)
    PreservesDirectedSup (Subtype.val : IdempotentFixedPoint e → L) := by
  let := IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone hsc
    )
  intro S hS hSdir
  show e (sSup (Subtype.val '' S)) = sSup (Subtype.val '' S)
  have hdir' : DirectedOn (· ≤ ·) (Subtype.val '' S) := by
```

```
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.val, Set.mem_image_of_mem _ hc, hac, hbc⟩
  have hne' : (Subtype.val '' S).Nonempty := hS.image _
  rw [hsc hne' hdir']
  congr 1
  rw [← Set.image_comp]
  exact Set.image_congr (fun s _ => s.2)

theorem retr_preservesDirectedSup (hidem : ∀ x, e (e x) = e x) (hsc :
    PreservesDirectedSup e) :
    letI := IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone
    hsc)
    PreservesDirectedSup (fun x : L => (⟨e x, hidem x⟩ : IdempotentFixedPoint e)) := by
  let := IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone hsc
    )
  have hmono := preservesDirectedSup_monotone hsc
  intro T hT hTdir
  apply Subtype.ext
  rw [completeLattice.coe hidem hmono]
  show e (sSup T) = e (sSup (Subtype.val '' ((fun x : L =>
      (⟨e x, hidem x⟩ : IdempotentFixedPoint e)) '' T)))
  have himg : Subtype.val '' ((fun x : L => (⟨e x, hidem x⟩ : IdempotentFixedPoint e))
    '' T) =
      e '' T := by
    rw [← Set.image_comp]; rfl
  rw [himg]
  have hTdir' : DirectedOn (· ≤ ·) (e '' T) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hTdir a ha b hb
    exact ⟨e c, Set.mem_image_of_mem _ hc, hmono hac, hmono hbc⟩
  have hTne' : (e '' T).Nonempty := hT.image _
  rw [hsc hT hTdir, hsc hTne' hTdir']
  congr 1
  rw [← Set.image_comp]
  exact Set.image_congr (fun x _ => (hidem x).symm)

noncomputable def retraction (hidem : ∀ x, e (e x) = e x) (hsc : PreservesDirectedSup e
    ) :
    @ContinuousLatticeRetraction (IdempotentFixedPoint e) L
      (IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone hsc))
     _ :=
  letI := IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone
    hsc)
  { incl := ⟨Subtype.val,
      PreservesDirectedSup.continuous_of (incl_preservesDirectedSup hidem hsc)⟩
    retr := ⟨fun x => ⟨e x, hidem x⟩,
      PreservesDirectedSup.continuous_of (retr_preservesDirectedSup hidem hsc)⟩
    retr_incl := fun k => Subtype.ext k.2 }

theorem isContinuousLattice (hidem : ∀ x, e (e x) = e x) (hsc : PreservesDirectedSup e)
    (hL : ContinuousLattice L) :
    @ContinuousLattice (IdempotentFixedPoint e)
      (IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone hsc))
     := by
  let := IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone hsc
```

```
    )
  exact (proposition_2_10 (retraction hidem hsc) hL).1

theorem scottTopology (hidem : ∀ x, e (e x) = e x) (hsc : PreservesDirectedSup e)
    (hL : ContinuousLattice L) :
    @scottTopologicalSpace (IdempotentFixedPoint e)
        (IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone hsc
    )) =
      TopologicalSpace.induced (Subtype.val : IdempotentFixedPoint e → L)
    scottTopologicalSpace := by
  let := IdempotentFixedPoint.completeLattice hidem (preservesDirectedSup_monotone hsc
    )
  exact (proposition_2_10 (retraction hidem hsc) hL).2

end IdempotentFixedPoint

end DomainTheory.ContinuousLattice
```

continuousLattice_scottTopology_of_injective.lean

```
module

public import DomainTheory.ContinuousLattice.ContinuousLattices.
    injective_homeomorph_to_continuousLatticeScott
public import DomainTheory.ContinuousLattice.ContinuousLattices.
    orderIsoScottHomeomorph
public import DomainTheory.ContinuousLattice.Specialization.SpecializationPreorder

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

/-- If specialization equals a given complete-lattice order on an injective `T₀` space,
     that
lattice is continuous and the given topology is its Scott topology. -/
theorem continuousLattice_scottTopology_of_injective {A : Type u}
    [latticeA : CompleteLattice A] [τA : TopologicalSpace A]
    [T0Space A] (hA : InjectiveSpace.{u, u} A)
    (hspec : ∀ x y : A, SpecializationPreorder x y ↔ @LE.le A latticeA.toLE x y) :
    ContinuousLattice A ∧
      (@scottTopologicalSpace A latticeA) = τA := by
  obtain ⟨E, instE, hE, ⟨h⟩⟩ := injective_homeomorph_to_continuousLatticeScott hA
  let : CompleteLattice E := instE
  let : TopologicalSpace E := @scottTopologicalSpace E instE
  let : Preorder A := latticeA.toPreorder
  let : Preorder E := instE.toPreorder
  let e : A ≃o E := h.toEquiv.toOrderIso
    (fun x y hxy => by
      exact SpecializationPreorder.scott_iff.mp
        (SpecializationPreorder.map ((hspec x y).mpr hxy) h.continuous))
    (fun x y hxy => by
```

```
      exact (hspec (h.symm x) (h.symm y)).mp
        (SpecializationPreorder.map (SpecializationPreorder.scott_iff.mpr hxy) h.symm.
    continuous))
  have hcont : ContinuousLattice A := ContinuousLattice.orderIso e hE
  refine ⟨hcont, ?_⟩
  have heqOrder :
      (@scottTopologicalSpace A latticeA) =
        TopologicalSpace.induced (e : A → E) (@scottTopologicalSpace E instE) := by
    let : TopologicalSpace A := @scottTopologicalSpace A latticeA
    exact (@orderIsoScottHomeomorph A E latticeA instE e).isInducing.eq_induced
  have heqHome :
      τA = TopologicalSpace.induced (h : A → E) (@scottTopologicalSpace E instE) :=
    (@Homeomorph.isInducing A E τA (@scottTopologicalSpace E instE) h).eq_induced
  exact heqOrder.trans heqHome.symm

end DomainTheory.ContinuousLattice
```

ScottMap.lean

```
module

public import Mathlib.Order.FixedPoints
import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5 -- shake: keep

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

def ScottMap (D : Type u) (D' : Type v) [CompleteLattice D] [CompleteLattice D'] :
    Type _ :=
  { f : D → D' // @Continuous D D' scottTopologicalSpace scottTopologicalSpace f }

namespace ScottMap

instance : CoeFun (ScottMap D D') (fun _ => D → D') where
  coe f := f.1

@[ext]
theorem ext {f g : ScottMap D D'} (h : ∀ x, f x = g x) : f = g :=
  Subtype.ext (funext h)

def const (c : D') : ScottMap D D' :=
  ⟨fun _ => c, PreservesDirectedSup.continuous_of fun S hS _ => by
    obtain ⟨x, hx⟩ := hS
    apply le_antisymm
```

```
    · exact le_sSup ⟨x, hx, rfl⟩
    · apply sSup_le
      intro b hb
      obtain ⟨y, hy, rfl⟩ := hb
      exact le_rfl⟩

@[simp] theorem const_apply (c : D') (x : D) : (ScottMap.const c : D → D') x = c :=
    rfl

def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x

def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_rfl

theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g) (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)

theorem le_antisymm' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g) (hgf : ScottMap.le g f) : f = g :=
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)

theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
    ScottMap.lt f g ↔ ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl

instance instPartialOrder {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] : PartialOrder (ScottMap D D') where
  le := ScottMap.le
  lt := ScottMap.lt
  le_refl := le_refl'
  le_trans := le_trans'
  lt_iff_le_not_ge := lt_iff_le_not_ge'
  le_antisymm := le_antisymm'

theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir
```

```
theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
    obtain ⟨f, hfF, rfl⟩ := hb
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))

theorem sSupMaps_continuous {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
  PreservesDirectedSup.continuous_of (pointwise_sSup_preservesDirectedSup F)

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

noncomputable instance instSupSet {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] : SupSet (ScottMap D D') := ⟨sSupMaps⟩

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl
```

```
theorem isLUB_sSup {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
  · intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

noncomputable instance instCompleteLattice {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] : CompleteLattice (ScottMap D D') :=
  completeLatticeOfSup (ScottMap D D') isLUB_sSup

theorem preservesDirectedSup_comp {f : D' → D''} {g : D → D'} (hf :
    PreservesDirectedSup f)
    (hg : PreservesDirectedSup g) : PreservesDirectedSup (f ∘ g) := by
  intro S hS hSdir
  have hmono : Monotone g := preservesDirectedSup_monotone hg
  have hdir' : DirectedOn (· ≤ ·) (g '' S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨g c, Set.mem_image_of_mem g hc, hmono hac, hmono hbc⟩
  have hne : (g '' S).Nonempty := Set.image_nonempty.2 hS
  rw [Function.comp_apply, hg hS hSdir, hf hne hdir']
  congr 1
  exact (Set.image_comp f g S).symm

def comp (f : ScottMap D' D'') (g : ScottMap D D') : ScottMap D D'' :=
  ⟨f ∘ g, PreservesDirectedSup.continuous_of (preservesDirectedSup_comp
    ((proposition_2_5 (Subtype.val f)).mp f.property)
    ((proposition_2_5 (Subtype.val g)).mp g.property))⟩

@[simp] theorem comp_apply (f : ScottMap D' D'') (g : ScottMap D D') (x : D) :
    (f.comp g : D → D'') x = f (g x) := rfl

theorem continuous (f : ScottMap D D') :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace f :=
  f.2

def idMap : ScottMap D D :=
  ⟨fun x => x, PreservesDirectedSup.continuous_of fun S _ _ => by
    show sSup S = sSup (Set.image (fun x => x) S)
    rw [Set.image_id']⟩

@[simp] theorem idMap_apply (x : D) : (idMap : ScottMap D D) x = x := rfl
```

```
theorem bot_apply (x : D) : ((⊥ : ScottMap D D') : D → D') x = ⊥ := by
  have h : (⊥ : ScottMap D D') = sSup (∅ : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_empty, sSup_empty]

theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

theorem sInf_image_le_of_mem {f' : ScottMap D D'} {U : Set D} {y : D} (hyU : y ∈ U) :
    sInf (Set.image (f' : D → D') U) ≤ f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

theorem eq_sSup_openInfs (hD' : ContinuousLattice D') (f' : ScottMap D D') (y : D) :
    (f' : D → D') y =
      sSup { d | ∃ U, @IsOpen D scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : D → D') U) } := by
  apply le_antisymm
  · rw [← hD'.sSup_wayBelow (f' y)]
    apply sSup_le
    intro a ha
    obtain ⟨V, hV, hf'V, hVsub⟩ := ha
    have hyU : y ∈ (f' : D → D') ^-1' V := hf'V
    have hUScott : ScottOpen ((f' : D → D') ^-1' V) :=
      scottOpen_preimage ((proposition_2_5 (Subtype.val f')).mp f'.property) hV
    have hUopen : @IsOpen D scottTopologicalSpace ((f' : D → D') ^-1' V) :=
      ScottOpen.isOpen_iff.mpr hUScott
    have ha' : a ≤ sInf (Set.image (f' : D → D') ((f' : D → D') ^-1' V)) := by
      apply le_sInf
      intro d hd
      obtain ⟨z, hzV, rfl⟩ := hd
      exact Set.mem_Ici.1 (hVsub hzV)
    have hmem : sInf (Set.image (f' : D → D') ((f' : D → D') ^-1' V)) ∈
        { d | ∃ U, @IsOpen D scottTopologicalSpace U ∧ y ∈ U ∧
          d = sInf (Set.image (f' : D → D') U) } :=
      ⟨(f' : D → D') ^-1' V, hUopen, hyU, rfl⟩
    exact le_trans ha' (le_sSup hmem)
  · apply sSup_le
    intro d hd
    obtain ⟨U, _hUopen, hyU, rfl⟩ := hd
    exact sInf_image_le_of_mem (f' := f') (y := y) (U := U) hyU

noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, PreservesDirectedSup.continuous_of (by
    intro S _ _
    apply ScottMap.ext
    intro y
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩

noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, PreservesDirectedSup.continuous_of (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D → D) ⊥
```

```
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, PreservesDirectedSup.continuous_of fix_preservesDirectedSup⟩

@[simp] theorem fixMap_apply (f : ScottMap D D) : (fixMap f : D) = fix f := rfl

end ScottMap
```

```
end DomainTheory.ContinuousLattice
```

2.4.4 Step function (pointwise) ✓

Pointwise values for Scott's $\overrightarrow{e}[e, e']$ before packaging as a **`ScottMap`**.

Scott's Theorem 3.3 proof uses the pointwise step values that later become the step maps in **`ScottMap D D'`**. For $e \in D$, $e' \in D'$, and $x \in D$, **`stepFun e e' x`** is that value before continuity is packaged (the case split $e'/\bot$ from $e \ll x$).

Namespace **`stepFun`** collects:

Lemma (`of_wayBelow`). If $e \ll x$, then **`stepFun e e' x = e'`**.

Lemma (`of_not_wayBelow`). If $\neg(e \ll x)$, then **`stepFun e e' x = \bot`**.

Theorem (`preservesDirectedSup`). **`stepFun e e'`** preserves directed suprema (hence is Scott-continuous).

`stepFun.lean`

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
import Mathlib.Order.BourbakiWitt

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'

namespace stepFun

theorem of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e' :=
  iSup_pos h

theorem of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

theorem preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [of_wayBelow h]
```

```
    obtain ⟨s, hsS, hes⟩ := (ScottOpen.wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end stepFun

end DomainTheory.ContinuousLattice
```

2.4.5 Step map $\overrightarrow{e}[e, e']$ ✓

Scott's step functions as elements of **`ScottMap D D'`** (Theorem 3.3).

For $e \in D$ and $e' \in D'$, **`stepMap e e'`** is Scott's step function $\overrightarrow{e}[e, e'] \in$ **`ScottMap D D'`** (built from **`stepFun`** and **`stepFun.preservesDirectedSup`**).

Namespace **`stepMap`** collects the step-function tools used in Theorem 3.3(a):

Lemma (`apply_of_wayBelow`). If $e \ll x$, then `(stepMap e e') x = e'`.

Lemma (`le_of_wayBelow`). If $e' \ll f(e)$, then `stepMap e e' ≤ f` in **`ScottMap D D'`**.

Lemma (`wayBelow`). If $e' \ll f(e)$, then `stepMap e e' ≪ f`.

Theorem (`pointwise_sSup`). For **`ContinuousLattice D`**, **`ContinuousLattice D'`**, $f \in$ **`ScottMap D D'`**, and $x \in D$, $(fx) = \bigsqcup\{\, e' \mid \exists e,\ e \ll x \wedge e' \ll f(e) \,\}$.

Theorem (`isContinuousLattice`). If **`ContinuousLattice D`** and **`ContinuousLattice D'`**, then **`ScottMap D D'`** is a continuous lattice (Theorem 3.3(a); proof combines **`pointwise_sSup`**, **`wayBelow`**, and **`apply_of_wayBelow`**).

`stepMap.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', PreservesDirectedSup.continuous_of (stepFun.preservesDirectedSup e e')
    ⟩
```

```
namespace stepMap

theorem apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun.of_wayBelow h

theorem le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e ≪ x
  · rw [apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun.of_not_wayBelow hex]
    exact bot_le

theorem wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨(c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
    exact Set.mem_Ici.2 (le_of_wayBelow hg)

theorem pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D') (f :
    ScottMap D D')
    (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, WayBelow.bot x⟩ (WayBelow.
    directedOn x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, WayBelow.bot
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
```

```
      obtain ⟨c, hc, hac, hbc⟩ := WayBelow.directedOn x a ha b hb
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
    exact le_sSup ⟨e, hex, hwe⟩

theorem isContinuousLattice (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [apply_of_wayBelow hex] at hx

end stepMap

end DomainTheory.ContinuousLattice
```

2.4.6 Proposition 3.2 ✓

3.2 Proposition. *The induced partial ordering on* $[X \to Y]$ *is such that:*

$$f \sqsubseteq g \quad \text{iff} \quad f(x) \sqsubseteq g(x) \text{ for all } x \in X,$$

where $f, g \in [X \to Y]$. □
where $f, g \in [X \to Y]$. □
The first question, of course, is what kind of a partial ordering this is.

proposition_3_2.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottFunctionTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]

theorem proposition_3_2 {f g : C(X, Y)} :
    letI := scottFunctionTopology X Y
    (SpecializationPreorder f g ↔ ∀ x, SpecializationPreorder (f x) (g x)) :=
  scottFunctionTopology.specializationPreorder

end DomainTheory.ContinuousLattice
```

2.4.7 Product topology on $[D \to D']$ ✓

For complete lattices D and D', the library topologizes **`ScottMap D D'`** (Scott-continuous maps $D \to D'$; see Elements of $[D \to D']$) with the *product* or pointwise-convergence topology: subbasic sets have the same shape as in Definition 3.1,

$$\{f : f(x) \in U\}$$

with $x \in D$ and $U \subseteq D'$ open in **`scottTopologicalSpace`**.

Definition (`scottMapInducedPiTopology D D'`). `TopologicalSpace (ScottMap D D')` induced from the pi topology on D'^D (each fiber carries **`scottTopologicalSpace`**) along the inclusion **`ScottMap D D' ↪ (D → D')`**.

Namespace **`scottMapInducedPiTopology`** packages the equivalent eval subbase (parallel to **`scottFunctionTopology`** in Definition 3.1):

Definition (`pointwiseSubbasis`). Subbasic sets $\{\, f \mid f(x) \in U \,\}$ for $f \in$ **`ScottMap D D'`**, $x \in D$, and $U \subseteq D'$ Scott-open.

Definition (`pointwiseTopology`). `generateFrom (pointwiseSubbasis D D').`

Lemma (`subbasic_isOpen`). For Scott-open $U \subseteq D'$ and $x \in D$, the subbasic set $\{\, f \mid f(x) \in U \,\}$ is open in **`pointwiseTopology D D'`**.

Theorem (`eq_induced_pi`). `pointwiseTopology D D' = scottMapInducedPiTopology D D'` — the eval subbase and the induced pi topology agree.

When D and D' are continuous lattices, the Scott topology on **`ScottMap D D'`** (from **`isContinuousLattice`** in namespace **`stepMap`**) agrees with this product topology:

Lemma (`pointwiseSubbasic_scottOpen`). Eval subbasic sets are Scott-open in **`ScottMap D D'`**.

Lemma (`wayBelow_le_finset_sup_step`). From $\varphi \ll$ **`g`**, a finite finset of step maps majorizes φ.

Lemma (`wayBelow_finset_sup`). A finite sup of way-below elements is way-below **`g`**.

Theorem (`scottTopology_eq_pointwise`). On **`ScottMap D D'`**, **`scottTopologicalSpace = pointwiseTopology D D'`** (uses the three lemmas above).

Theorem (`scottTopology_eq_inducedPi`). On **`ScottMap D D'`**, **`scottTopologicalSpace = scottMapInducedPiTopology D D'`** (from **`scottTopology_eq_pointwise`** and **`eq_induced_pi`**). This is Theorem 3.3(b).

`scottMapInducedPiTopology.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import Mathlib.Data.Finset.Lattice.Fold

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

namespace scottMapInducedPiTopology

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (WayBelow.bot g) (fun _ ha _ hb => ha.sup
    hb) h

theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
      exact Finset.sup_le fun p hp => stepMap.le_of_wayBelow (hF p hp)
```

```
    · rw [ScottMap.le_def]
      intro x
      rw [← stepMap.pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
    e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
    by
        rw [ScottMap.sSup_apply]
        exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
      rwa [stepMap.apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

section

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

def pointwiseSubbasis (D D' : Type*) [CompleteLattice D] [CompleteLattice D'] :
    Set (Set (ScottMap D D')) :=
  { W | ∃ (x : D) (U : Set D'),
      @IsOpen D' scottTopologicalSpace U ∧ W = {f : ScottMap D D' | (f : D → D') x ∈
    U} }

@[reducible] noncomputable def pointwiseTopology (D D' : Type*)
    [CompleteLattice D] [CompleteLattice D'] : TopologicalSpace (ScottMap D D') :=
  generateFrom (pointwiseSubbasis D D')

theorem subbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D' scottTopologicalSpace U)
    :
    @IsOpen (ScottMap D D') (pointwiseTopology D D')
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩

end

end scottMapInducedPiTopology

@[reducible] noncomputable def scottMapInducedPiTopology (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] : TopologicalSpace (ScottMap D D') :=
  TopologicalSpace.induced (fun f : ScottMap D D' => (f : D → D'))
    (@Pi.topologicalSpace D (fun _ => D') (fun _ => scottTopologicalSpace))

namespace scottMapInducedPiTopology

section

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

theorem eq_induced_pi :
```

```
    pointwiseTopology D D' = scottMapInducedPiTopology D D' := by
  apply le_antisymm
  · let : TopologicalSpace D' := scottTopologicalSpace
    let : TopologicalSpace (ScottMap D D') := pointwiseTopology D D'
    change pointwiseTopology D D' ≤ scottMapInducedPiTopology D D'
    rw [← continuous_iff_le_induced]
    exact continuous_pi fun x =>
      continuous_def.2 fun U hU => subbasic_isOpen (D := D) (D' := D') x hU
  · apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hU, rfl⟩
    let : TopologicalSpace D' := scottTopologicalSpace
    change @IsOpen (ScottMap D D')
      (TopologicalSpace.induced (fun f : ScottMap D D' => (f : D → D'))
        (@Pi.topologicalSpace D (fun _ => D') (fun _ => scottTopologicalSpace)))
      {f : ScottMap D D' | (f : D → D') x ∈ U}
    rw [isOpen_induced_iff]
    exact ⟨{g : D → D' | g x ∈ U}, (@continuous_apply D (fun _ => D')
      (fun _ => scottTopologicalSpace) x).isOpen_preimage U hU, rfl⟩

theorem scottTopology_eq_pointwise (hD : ContinuousLattice D) (hD' : ContinuousLattice
    D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      pointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := stepMap.isContinuousLattice hD hD'
  apply le_antisymm
  · apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
  · intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (pointwiseTopology D D')]
    intro g hgU
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap.wayBelow (Set.mem_iInter₂.1 hh p hp)
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (pointwiseTopology D D') F
    _
        (fun p _ => subbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (ScottOpen.wayBelow
    p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

theorem scottTopology_eq_inducedPi (hD : ContinuousLattice D) (hD' : ContinuousLattice
    D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [scottTopology_eq_pointwise hD hD', eq_induced_pi]

end

end scottMapInducedPiTopology

end DomainTheory.ContinuousLattice
```

2.4.8 Theorem 3.3 ✓

3.3 Theorem. If D and D' are continuous lattices, then so is $[D \to D']$ under the induced partial ordering with the lattice topology agreeing with the product topology.

Proof: The argument is "pointwise." Thus, the constant function with value $\bot \in D'$ is obviously continuous and is the $\bot$ of $[D \to D']$ by 3.2. Since by 2.7 the lattice operation $\sqcup$ on D' is continuous, then if $f, g \in [D \to D']$ the composition $f \sqcup g$, defined by

$$(f \sqcup g)(x) = f(x) \sqcup g(x)$$

for all $x \in D$, is also continuous and represents the lub of f, g in $[D \to D']$. (The same arguments apply to $\top$ and $\sqcap$, so $[D \to D']$ is at least a lattice.) To show that $[D \to D']$ is complete it is sufficient now to show that lubs of directed subsets exist. So let $\mathcal{F} \subseteq [D \to D']$ be directed. Define a function from D into D' by the equation:

$$(\bigsqcup \mathcal{F})(x) = \bigsqcup\{f(x) : f \in \mathcal{F}\},$$

for all $x \in D$. If we can show that $\bigsqcup \mathcal{F}$ is continuous, then being in $[D \to D']$ it has to be the lub. Consider $U \subseteq D'$, an open subset. Taking the inverse image and remembering that $\mathcal{F}$ is directed, we find:

$$\{x : (\bigsqcup \mathcal{F})(x) \in U = \bigcup x : f(x) \in U : f \in \mathcal{F}\}.$$

This is an open set, and so $\bigsqcup \mathcal{F}$ is indeed continuous. (Warning: the infinite $\sqcap \mathcal{F}$ are not in general computed pointwise; however, it is easy to extend the above argument to show that arbitrary $\bigsqcup \mathcal{F}$ are.)

To show that $[D \to D']$ is continuous, we establish first that for $f \in [D \to D']$

$$f = \bigsqcup \overrightarrow{e}[e, \{e'] : e' \ll f(e)\},$$

where e ranges over D and e' over D', and where the function $\overrightarrow{e}[e, e']$ is defined by:

$$\overrightarrow{e}[e, e'](x) = \begin{cases} e' & \text{if } e \ll x, \\ \bot & \text{if not,} \end{cases}$$

for all $x \in D$. Call the function on the right f'. Calculate:

$$f'(x) = \bigsqcup \overrightarrow{e}[e, e'](x) : e' \ll f(e)$$

$$= \bigsqcup\{e' : \exists e \ll x\ [e' \ll f(e)]\}$$

$$= \bigsqcup\{e' : e' \ll f(x) = f(x)\}.$$

With the equation for f proved, note next that for all $g \in [D \to D']$, $e' \ll g(e)$ implies $\overrightarrow{e}[e, e'] \sqsubseteq g$ by an easy pointwise argument. If we let $V = \{g : e' \ll g(e)\}$, we see then that V is open in the product topology and that $\overrightarrow{e}[e, e'] \sqsubseteq \sqcap V$. We may then conclude that $f = \bigsqcup\{\sqcap V : f \in V\}$, which proves both that $[D \to D']$ is a continuous lattice and that the two topologies agree by the remark following 2.4. □

Pointwise $\sqcup$, $\top$, and $\bot$ yield a complete lattice on **`ScottMap D D'`** when D and D' are complete (**`ScottMap.instCompleteLattice`**); directed suprema are computed pointwise (**`isLUB_sSup`**, **`sSupMaps`**). Scott's step functions are **`stepMap e e'`**; **`stepMap.pointwise_sSup`**,

stepMap.wayBelow, and **stepMap.apply_of_wayBelow** formalize $f = \bigsqcup \overrightarrow{e}[e, e']$ (see **stepMap.isContinuousLattice** in Step map $\overrightarrow{e}[e, e']$). The product topology and its agreement with the Scott topology are in Product topology on $[D \to D']$ (**scottMapInducedPiTopology.scottTopology_eq_pointwise**, **scottTopology_eq_inducedPi**).

Theorem (theorem_3_3). If **ContinuousLattice D** and **ContinuousLattice D'**, then **ScottMap D D'** is a continuous lattice and its Scott topology equals **scottMapInducedPiTopology.pointwiseT D D'**.

Proof. theorem_3_3.lean combines **stepMap.isContinuousLattice** and **scottMapInducedPiTopol eq_pointwise**. □

The above theorem might possibly be generalized to $[X \to D]$ where X is merely a T_0-space, but I was unable to see the argument. In any case we are mostly interested in the continuous lattices. Note as a consequence of our proof:

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology
import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
        scottMapInducedPiTopology.pointwiseTopology D D' :=
  ⟨stepMap.isContinuousLattice hD hD', scottMapInducedPiTopology.
    scottTopology_eq_pointwise hD hD'⟩

end DomainTheory.ContinuousLattice
```

2.4.9 Corollary 3.4 ✓

3.4 Corollary. For continuous lattices D and D', the evaluation map:

$$\mathrm{eval} : [D \to D'] \times D \to D'$$

is continuous.

Proof: Here $\mathrm{eval}(f, x) = f(x)$. With f fixed, this is obviously continuous. With x fixed, we proved the continuity above with our calculation of $\bigsqcup \mathcal{F}$ in view of 2.5. Hence applying 5.3 and 2.6, we conclude that eval is jointly continuous. □

This result gives only one example of the masses of continuous functions that are available on continuous lattices. As another fundamental example we have:

Namespace **corollary_3_4** collects the evaluation tools; Scott's corollary is assembled from them:

Lemma (`eval_on_C`). For $x \in D$, application at x is continuous as a map **`ScottC D D' → D'`** when the function space carries **`scottFunctionTopology`** with Scott topologies on D and D'.

Theorem (`preservesDirectedSup`). Joint evaluation **`(f, x) ↦ f(x)`** on **`ScottMap D D' × D`** preserves directed suprema (via Propositions 2.5 and 2.6).

Theorem (`jointly_continuous`). The evaluation map **`ScottMap D D' × D → D'`** is Scott-continuous.

Theorem (`corollary_3_4`). Specialization of **`eval_on_C`** to **`ScottC D D'`** (the x-fixed step in Scott's proof).

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottFunctionTopology
import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace corollary_3_4

theorem eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU =>
    scottFunctionTopology.subbasis_isOpen_scott (D := D) (D' := D') (x := x) hU

theorem preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
  · intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

theorem jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  PreservesDirectedSup.continuous_of preservesDirectedSup
```

```
end corollary_3_4

theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  corollary_3_4.eval_on_C x

end DomainTheory.ContinuousLattice
```

2.4.10 Scott functional abstraction ✓

For continuous lattices D, D', and D'', Scott's functional abstraction

$$\mathrm{lambda}(f)(x)(y) = f(x, y)$$

maps $f \in [[D \times D'] \to D'']$ to an element of $[D \to [D' \to D'']]$ (using Theorem 3.3 on nested function spaces). The library names that map **`scottLambda`**.

Definition (`scottLambda.at`). For $f \in$ **`ScottMap (D × D') D''`** and $x \in D$, **`scottLambda.at f x`** is the Scott map $y \mapsto f(x, y)$ in **`ScottMap D' D''`**.

Definition (`scottLambda`). **`scottLambda f`** is the Scott map $x \mapsto$ **`scottLambda.at f x`** in **`ScottMap D (ScottMap D' D'')`**.

On complete lattices, currying uses two pointwise **`sSup`** facts (same file, no separate module):

Lemma (`sSup_image_prod_mk_left`). $\bigsqcup\{\,(x, y) \mid y \in S\,\} = (x, \bigsqcup S)$ for nonempty $S \subseteq D'$.

Lemma (`sSup_image_prod_mk_right`). $\bigsqcup\{\,(x, y) \mid x \in S\,\} = (\bigsqcup S, y)$ for nonempty $S \subseteq D$.

Namespace **`scottLambda`** collects the currying and section tools used to build those maps:

Lemma (`curry_right`). For fixed $x \in D$, the section $y \mapsto f(x, y)$ preserves directed suprema.

Lemma (`curry_left`). For fixed $y \in D'$, the section $x \mapsto f(x, y)$ preserves directed suprema.

Theorem (`outer`). For $f \in$ **`ScottMap (D × D') D''`**, the map $x \mapsto$ **`scottLambda.at f x`** preserves directed suprema.

Lemma (`apply`). **`((scottLambda f x) : ScottMap D' D'') y = f(x,y)`** (definitional).

The map **`scottLambda`** itself (on carriers **`ScottMap (D × D') D''`** and **`ScottMap D (ScottMap D' D'')`**) preserves directed suprema in the function variable; lubs on Scott maps are computed pointwise (Theorem 3.3).

Theorem (`preservesDirectedSup`). **`PreservesDirectedSup scottLambda`** (namespace **`scottLambda`**, `scottLambda.lean`).

This is the directed-supremum input to Scott continuity of functional abstraction (Proposition 3.5).

`scottLambda.lean`

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

namespace scottLambda

theorem curry_right (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
```

```
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

theorem curry_left (f : ScottMap (D × D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end scottLambda

noncomputable def scottLambda.at (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), PreservesDirectedSup.continuous_of (scottLambda.curry_right f x)⟩

namespace scottLambda

theorem outer (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambda.at f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambda.at f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left f y hS hSdir

end scottLambda

noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
    '') :=
```

```
  ⟨fun x => scottLambda.at f x,
    PreservesDirectedSup.continuous_of (scottLambda.outer f)⟩

namespace scottLambda

@[simp] theorem apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

theorem preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro ℱ hℱ hℱdir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply,
    Set.image_image, Set.image_image]
  congr 1

end scottLambda

end DomainTheory.ContinuousLattice
```

2.4.11 Proposition 3.5 ✓

3.5 Proposition. For continuous lattices D, D', and D'', the map of functional abstraction:

$$\text{lambda} : [[D \times D'] \to D''] \to [D \to [D' \to D'']]$$

is continuous.

Proof: Here lambda is defined by:

$$\text{lambda}(f)(x)(y) = f(x, y)$$

where $f \in [[D \times D'] \to D'']$ and $x \in D$ and $y \in D'$. What is particularly interesting here is that by virtue of 3.3 we are making use of $[D \to [D' \to D'']]$ as a continuous lattice. The principle being stated here can be formulated more broadly in this way:

> If an expression $\mathcal{E}(x, y, z, \dots)$ is continuous in all its variables $x, y, z, \dots$ with values in D' as x ranges in D, then the expression
>
> $$\lambda x : D.\mathcal{E}(x, y, z, \dots)$$
>
> with values in $[D \to D']$ is continuous in the remaining variables $y, z, \dots$.

The λ-notation is a notation for functions, where in the above the variable after the λ is the *argument* and the expression after the . is the *value* (as a function of the argument). Thus we could write:

$$\text{lambda} = \lambda f : [[D \times D'] \to D''].\lambda x : D.\lambda y : D'.f(x, y),$$

and, because $f(x, y)$ is continuous in f, x, and y, our conclusion follows. But often it is more readable not to write equations between functions but rather equations between values for definitional purposes.

The proof of the principle is easy. For let the variable y, say range over D'' and let $S \subseteq D''$ be a directed subset. Then

$$\lambda x : D.\mathcal{E}(x, \bigsqcup S, z, \dots) = \lambda x : D.\bigsqcup \mathcal{E}(x, y, z, \dots) : y \in S \quad = \bigsqcup \lambda x : D.\mathcal{E}(x, y, z, \dots) : y \in S,$$

because the lubs of functions are computed pointwise. □

We need not enumerate the many corollaries that follow easily now from this result. We mention, however, that composition $f \circ g$ of functions (on continuous lattices) is continuous in the two function variables, where we write

$$(f \circ g)(x) = f(g(x)).$$

proposition_3_5.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  PreservesDirectedSup.continuous_of scottLambda.preservesDirectedSup

end DomainTheory.ContinuousLattice
```

2.4.12 Retractions and extensions ✓

What will be useful will be to return at this point to a discussion of the injective properties of continuous lattices. If one continuous lattice is a subspace of another it is of course a retract. This relationship between spaces can be given by a pair of continuous maps

$$i : D \to D' \quad \text{and} \quad j : D' \to D,$$

where

$$j \circ i = \mathrm{id}_D = \lambda x : D.x.$$

The composition $i \circ j : D' \to i(D)$ is the retraction onto the subspace corresponding to D under i. Now if we have a diagram:

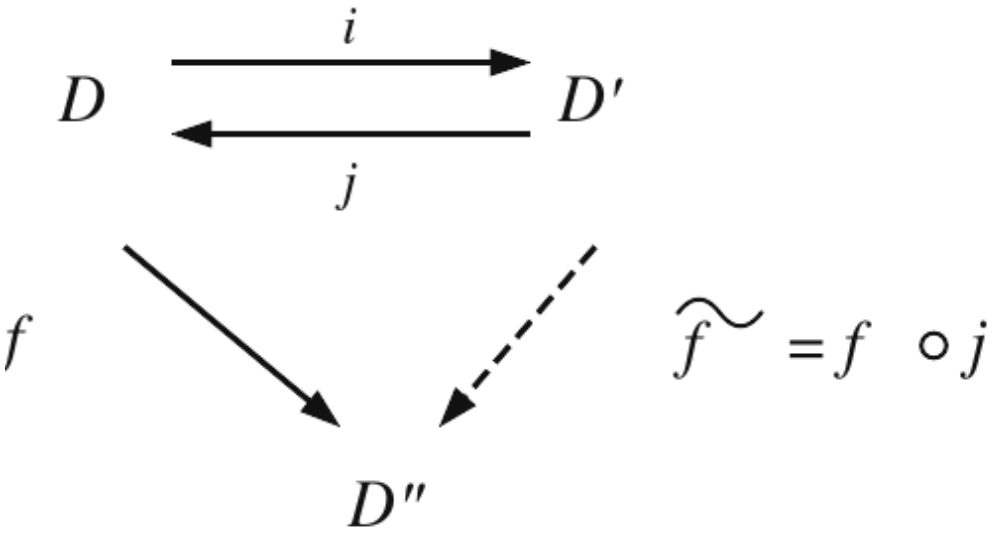


Figure 12: D arrow i to D prime, arrow j back; solid f from D to D double prime; dashed arrow from D prime to D double prime labeled f tilde equals f compose j

the given continuous f is at once extendable from D to D' by the obvious definition of $\tilde{f}$. This $\tilde{f}$ is not the $\bar{f}$ used in the proof of 2.11, and it will be well to sort out the connections. On one side note that if f' is any function which extends f, then we have $f = f' \circ i$. But this implies

$$\tilde{f} = f \circ j = f' \circ i \circ j,$$

which shows that $\tilde{f}$ is a "degraded" version of f'. There is one situation where this type of degrading is especially nice.

Retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D₀ D₀' D₁ D₁' : Type*}

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace ScottMap

structure retraction (D₀ D₀' D₁ D₁' : Type*)
    [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [CompleteLattice D₁
    '] where
  incl : ScottMap D₀ D₁ → ScottMap D₀' D₁'
  retr : ScottMap D₀' D₁' → ScottMap D₀ D₁
  retr_incl : ∀ f, retr (incl f) = f

end ScottMap

end DomainTheory.ContinuousLattice
```

2.4.13 Definition 3.6 ✓

3.6 Definition. A continuous lattice D is said to be a *projection* of a continuous lattice D' iff there are a pair of continuous maps

$$i : D \to D' \quad \text{and} \quad j : D' \to D$$

such that not only

$$j \circ i = \mathrm{id}_D,$$

but also

$$i \circ j \sqsubseteq \mathrm{id}_{D'}.$$

Thus, in case our retraction is a projection, we have $\tilde{f} \sqsubseteq f'$, which means that $\tilde{f}$ is the *minimal* extension of $f \in [D \to D'']$ to a function in $[D' \to D'']$. We will discuss the nature of $\tilde{f}$ in a moment. But before we do we pause to remark that the correspondence $f \rightsquigarrow \tilde{f}$ is *continuous*, and this fact is easily extended.

`Projection.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Retraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {D D₀ D₀' D₁ D₁' : Type*}

variable [CompleteLattice D] [CompleteLattice D₀] [CompleteLattice D₀'] [
    CompleteLattice D₁] [CompleteLattice D₁']

namespace ScottMap
```

```
structure projection (D₀ D₀' D₁ D₁' : Type*)
    [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [CompleteLattice D₁
    '] extends
    ScottMap.retraction D₀ D₀' D₁ D₁' where
  incl_retr_le : ∀ f' x, (incl (retr f')) x ≤ f' x

noncomputable def conMinContinuousLatticeProjection : ContinuousLatticeProjection D (
    ScottMap D D) where
  incl := ScottMap.con
  retr := ScottMap.min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end ScottMap

end DomainTheory.ContinuousLattice
```

2.4.14 Function-space retraction and projection ✓

When D_n is a retraction (projection) of D'_n via $i_n : D_n \to D'_n$ and $j_n : D'_n \to D_n$, Proposition 3.7 lifts that data to the function spaces $[D_0 \to D_1]$ and $[D'_0 \to D'_1]$. Scott writes the lifted pair as $\vec{i}$ and $\vec{j}$:

$$\vec{i}(f) = i_1 \circ f \circ j_0, \qquad \vec{j}(f') = j_1 \circ f' \circ i_0$$

for $f \in [D_0 \to D_1]$ and $f' \in [D'_0 \to D'_1]$. The formalization packages lattice retractions **`ContinuousLatticeRetraction`** (and projections **`ContinuousLatticeProjection`**) from Continuous lattice retraction and Continuous lattice projection; map-space carriers **`ScottMap.retraction`** and **`ScottMap.projection`** from Retractions and extensions and Definition 3.6.

Definition (`FunctionSpace.retraction`). From **`R0 : ContinuousLatticeRetraction D0 D0'`** and **`R1 : ContinuousLatticeRetraction D1 D1'`**, build **`ScottMap.retraction D0 D0' D1 D1'`** with **`incl f = R1.incl ∘ f ∘ R0.retr`** and **`retr f' = R1.retr ∘ f' ∘ R0.incl`**.

Definition (`FunctionSpace.projection`). From **`P0`** and **`P1`** as continuous lattice projections, the same **`incl`** / **`retr`** formulas on **`ScottMap`**, plus **`incl_retr_le`** from the lattice projection inequalities.

`FunctionSpace.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Projection

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {D₀ D₀' D₁ D₁' : Type*}

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace FunctionSpace

def retraction (R₀ : ContinuousLatticeRetraction D₀ D₀') (R₁ :
    ContinuousLatticeRetraction D₁ D₁') :
    ScottMap.retraction D₀ D₀' D₁ D₁' where
  incl f := ScottMap.comp R₁.incl (ScottMap.comp f R₀.retr)
  retr f' := ScottMap.comp R₁.retr (ScottMap.comp f' R₀.incl)
  retr_incl f := ScottMap.ext fun x => by simp [ScottMap.comp, R₀.retr_incl, R₁.
    retr_incl]

def projection (P₀ : ContinuousLatticeProjection D₀ D₀') (P₁ :
    ContinuousLatticeProjection D₁ D₁') :
    ScottMap.projection D₀ D₀' D₁ D₁' :=
  ScottMap.projection.mk
    (FunctionSpace.retraction P₀.toContinuousLatticeRetraction P₁.
    toContinuousLatticeRetraction)
    (by
      intro f' y
      dsimp only [FunctionSpace.retraction, ScottMap.comp, Function.comp_def]
      exact le_trans (P₁.incl_retr_le (f' (P₀.incl (P₀.retr y))))
        (ScottMap.monotone f' (P₀.incl_retr_le y)))

end FunctionSpace

end DomainTheory.ContinuousLattice
```

2.4.15 Proposition 3.7 ✓

3.7 Proposition. Suppose the two pairs of maps

$$i_n : D_n \to D'_n \quad \text{and} \quad j_n : D'_n \to D_n$$

for $n = 0, 1$ make D_n a retraction (projection) of D'_n. Then $[D_0 \to D_1]$ is also a retraction (projection) of $[D'_0 \to D'_1]$ by means of the pair of maps:

$$\vec{i}(f) = i_1 \circ f \circ j_0 \text{ , and}$$

$$\vec{j}(f') = j_1 \circ f' \circ i_0 \text{ ,}$$

where $f \in [D_0 \to D_1]$ and $f' \in [D'_0 \to D'_1]$. □

proposition_3_7.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.FunctionSpace
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D₀ D₀' D₁ D₁' : Type*}

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem proposition_3_7 (P₀ : ContinuousLatticeProjection D₀ D₀') (P₁ :
    ContinuousLatticeProjection D₁ D₁') :
    Nonempty (ScottMap.retraction D₀ D₀' D₁ D₁') ∧
    Nonempty (ScottMap.projection D₀ D₀' D₁ D₁') :=
  ⟨⟨FunctionSpace.retraction P₀.toContinuousLatticeRetraction P₁.
    toContinuousLatticeRetraction⟩,
   ⟨FunctionSpace.projection P₀ P₁⟩⟩

end DomainTheory.ContinuousLattice
```

2.4.16 Scott subspace extension ✓

When Y carries the Scott topology and $e : X \to Y$ is a subspace embedding, Scott's **subspace extension** $\bar{f} : Y \to D$ of $f : X \to D$ is built from open neighborhoods of $y \in Y$ and the infima $\sqcap f(e^{-1}(U))$ over $U \ni y$. The formalization lives in `scottSubspaceExtend.lean`.

Definition (`scottSubspaceExtend.inf`). For Scott-open $U \subseteq Y$, `inf e f U` is $\sqcap\{f(x) \mid e(x) \in U\}$.

Definition (`scottSubspaceExtend`). `scottSubspaceExtend e f y` is $\bigsqcup\{d \mid \exists U$ Scott-open, $y \in U,\ d = \inf e\, f\, U\}$.

Namespace **`scottSubspaceExtend`**:

Lemma (`inf_mono`). Monotonicity in the domain map $f \le g$ on X.

Lemma (`inf_eq_of_ext`). Agreement on $e(X)$ implies equal **`inf`**.

Lemma (`inf_eq_of_comp`). Relates **`inf e f U`** to $\sqcap f'(e(X) \cap U)$ when $f' : [Y \to D]$ extends f on $e(X)$.

Lemma (`sInf_image_le_of_subset`). Larger index set lowers **`sInf`** on images.

Lemma (`inf_mono_incl_apply`). For a continuous-lattice projection **`P : D ⇄ D'`**, **`P.incl (inf e f U) ≤ inf e (λ x, P.incl (f x)) U`**.

Lemma (`inf_mono_retr`). For the same **`P`**, **`P.retr (inf e g U) ≤ inf e (λ x, P.retr (g x)) U`**.

Lemma (`inf_mono_incl`). If **`∀ x, f x = P.retr (g x)`** on X, then **`inf e (λ x, P.incl (f x)) U ≤ inf e g U`**.

Lemma (`incl_inf`). Under the same agreement **`f = P.retr ∘ g`** on X, **`P.incl (inf e f U) ≤ inf e g U`** (**`inf_mono_incl_apply`** then **`inf_mono_incl`**).

Lemma (`mono`). / `eq_of_ext`. — for the full extension **`scottSubspaceExtend`**.

Theorem (`maximal`). Any Scott map $f' : Y \to D$ extending f on $e(X)$ satisfies $f'y \le \bar{f}(y)$ (uses **`ScottMap.eq_sSup_openInfs`** from Elements of $[D \to D']$).

`scottSubspaceExtend.lean`

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

namespace scottSubspaceExtend

variable {X Y D D' : Type*} [CompleteLattice D] [CompleteLattice D']

noncomputable def inf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

theorem inf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (U : Set Y) :
    inf e f U ≤ inf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

theorem inf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x) (U : Set Y) :
    inf e f U = inf e g' U := by
  unfold inf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]

theorem sInf_image_le_of_subset {α : Type*} {f : α → D} {S T : Set α} (hST : S ⊆ T) :
    sInf (f '' T) ≤ sInf (f '' S) :=
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩

theorem inf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : ContinuousLatticeProjection D D') :
    P.incl (inf e f U) ≤ inf e (fun x => P.incl (f x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

theorem inf_mono_retr {e : X → Y} {g : X → D'} (P : ContinuousLatticeProjection D D')
    (U : Set Y) :
    P.retr (inf e g U) ≤ inf e (fun x => P.retr (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

theorem inf_mono_incl {e : X → Y} {f : X → D} {g : X → D'} (P :
    ContinuousLatticeProjection D D')
```

```
    (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    inf e (fun x => P.incl (f x)) U ≤ inf e g U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

theorem incl_inf {e : X → Y} (P : ContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (inf e f U) ≤ inf e g U :=
  le_trans (inf_mono_incl_apply (e := e) (f := f) U P)
    (inf_mono_incl (e := e) (f := f) (g := g) P hfg U)

section inf_eq_of_comp

variable [CompleteLattice Y]

theorem inf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y} (f' : ScottMap Y D)
    (h : ∀ x, f' (e x) = f x) :
    inf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
      · exact ⟨x, rfl⟩
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [inf, hset]

end inf_eq_of_comp

end scottSubspaceExtend

variable {X Y : Type*} [CompleteLattice Y]
variable {D : Type*} [CompleteLattice D]

noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d = scottSubspaceExtend.
    inf e f U }

namespace scottSubspaceExtend

theorem mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (y : Y) :
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  unfold scottSubspaceExtend
  apply sSup_le
```

```
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : inf e g₀ U ∈
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d = inf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (inf_mono hfg U) (le_sSup hmem)

theorem eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, inf_eq_of_ext h]

variable [TopologicalSpace X] [TopologicalSpace Y]

theorem maximal (hD : ContinuousLattice D) (e : X → Y) (_he : IsEmbedding e) (f : X →
    D)
    (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := ScottMap.eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : inf e f U ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d = inf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (inf_eq_of_comp (f' := f') h_ext).symm)
  exact le_trans (le_of_eq hEq) hmain

end scottSubspaceExtend

end DomainTheory.ContinuousLattice
```

2.4.17 Proposition 3.8 ✓

Returning now to $\bar{f}$ we can prove:

3.8 Proposition. If D is a continuous lattice and $e : X \to Y$ a subspace embedding, then for each $f : X \to D$, the function $\bar{f} : Y \to D$ given by the formula

$$\bar{f}(y) = \bigsqcup \{ \sqcap f(x) : e(x) \in U : y \in U \},$$

where U ranges over open subsets of Y and x over X, is the maximal extension of f to a function in the partially ordered set $[Y \to D]$.

Proof: We are saying that $\bar{f}$ is the maximal solution to the equation

$$f = \bar{f} \circ e.$$

We already know it is a solution, so let f' be any other. We have

$$f'(y) = \bigsqcup \sqcap f'(z) : z \in U : y \in U \quad \sqsubseteq \bigsqcup \sqcap f'(z) : z \in e(X) \cap U : y \in U = \bigsqcup \sqcap f'(e(x)) : e(x) \in U : y \in U \quad = \bigsqcup \{\sqcap$$

which establishes that $f' \sqsubseteq \bar{f}$. $\square$

By the same argument we could show that $\bar{f}$ is the maximal solution of $\bar{f} \circ e \sqsubseteq f$. An interesting question is whether the correspondence $f \rightsquigarrow \bar{f}$ is continuous. I very much doubt it, but at this moment a counterexample escapes me. It is clear that the correspondence is monotonic, for if $f \sqsubseteq g$, then the formula of 3.8 shows that $\bar{f} \sqsubseteq \bar{g}$. This gives us a neat argument for the previous remark: if $g \circ e \sqsubseteq f$, then

$$\overline{g \circ e} \sqsubseteq \bar{f}.$$

But $g \circ e = \overline{g \circ e} \circ e$, so by 3.8, $g \sqsubseteq \overline{g \circ e}$, and $\bar{f}$ is thus maximal.

In the case that the range spaces are being extended, the following lemma relating the extensions will be very useful when we consider inverse limits.

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
import DomainTheory.ContinuousLattice.Constructions.scottExtend_eq_of_continuous
import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩
```

```
end InjectiveExtension
end DomainTheory.ContinuousLattice
```

2.4.18 Lemma 3.9 ✓

3.9 Lemma. Consider the diagram:

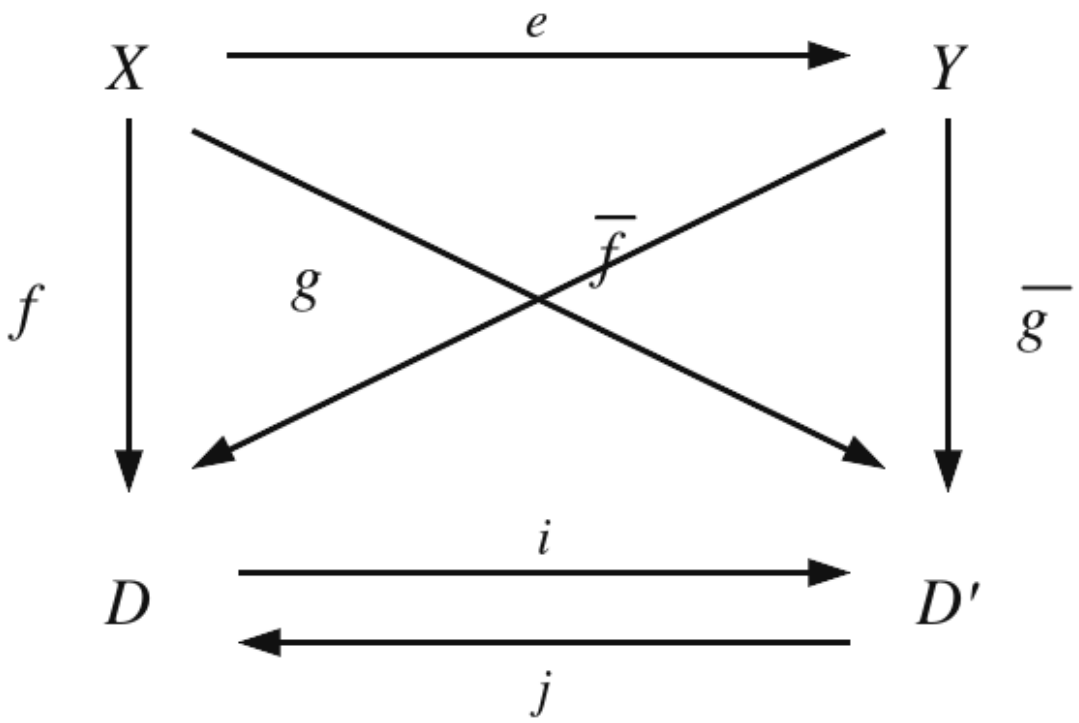


Figure 13: Lemma 3.9 diagram. Top: X to Y by e. Left: f down to D. Right: g bar down to D prime. Diagonals cross: g from X to D prime, f bar from Y to D. Bottom: i from D to D prime and j back.

where the upper row is a subspace embedding and the lower is a projection. If the given functions f and g are extended to $\bar{f}$ and $\bar{g}$ as in 3.8, and if $f = j \circ g$, then $\bar{f} = j \circ \bar{g}$ also.

Proof: $\bar{f}$ and $\bar{g}$ are maximal solutions of $f = \bar{f} \circ e$ and $g = \bar{g} \circ e$. Therefore since

$$f = j \circ g = j \circ \bar{g} \circ e,$$

we see that

$$j \circ \bar{g} \sqsubseteq \bar{f}.$$

Note also that

$$i \circ \bar{f} \circ e = i \circ f = i \circ j \circ g \sqsubseteq g,$$

and so by the remark following 3.8, we have

$$i \circ \bar{f} \sqsubseteq \bar{g}.$$

Therefore

$$\bar{f} = j \circ i \circ \bar{f} \sqsubseteq j \circ \bar{g},$$

which proves the equality. □

Whether this lemma is true for retractions in any form, I do not know. My proof seems to require the stronger projection relationship. I suspect there may be difficulties. In general projections are better behaved than retractions. By the way the word projection seems to be properly used in 3.6, for the projection $j : D \times D' \to D$ of the Cartesian product of two continuous lattices onto the first factor *is* a projection with partial inverse $i : D \to D \times D'$ defined by

$$i(x) = (x, \bot)$$

for $x \in D$.

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
import DomainTheory.ContinuousLattice.Constructions.scottExtend_eq_of_continuous
import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']

theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : ContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g

  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
```

```
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar

  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]

  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

2.4.19 Proposition 3.10 ✓

3.10 Proposition. If the continuous lattice D is a projection of the continuous lattice D' via the pair of maps i, j; then for all $S \subseteq D$ and all $x, y \in D$ we have:

(i) $i(\bigsqcup S) = \bigsqcup\{i(x) : x \in S\}$,

(ii) $i(x) = i(y)$ implies $x = y$,

(iii) $x \ll y$ implies $i(x) \ll i(y)$.

Conversely, if a map $i : D \to D'$ satisfies (i)–(iii), then there exists a continuous $j : D' \to D$ making D a projection of D', and in fact j is uniquely determined by:

(iv) $j(x') = \bigsqcup\{x \in D : i(x) \sqsubseteq x'\}$

for all $x' \in D'$.

Proof: Equation (i) holds for directed $S \subseteq D$ because i is continuous. To have it hold for arbitrary S it is only necessary to check it for finite sets, because every lub is the directed lub of finite sublubs. (The last word of that sentence is an unfortunate accident.) Further, to check the equation for finite sets it is enough

to check it for the empty set and for two element sets. Thus, $i(\bot) = \bot$, because $j(i(\bot)) = \bot$ and since $\bot \sqsubseteq i(\bot)$,

$$j(\bot) \sqsubseteq j(i(\bot)) = \bot,$$

so $j(\bot) = \bot$. Whence $i(\bot) = i(j(\bot)) \sqsubseteq \bot$. Next $i(x \sqcup y) = i(x) \sqcup i(y)$, because first

$$i(x) \sqcup i(y) \sqsubseteq i(x \sqcup y)$$

by monotonicity. Then note that

$$i(x) \sqsubseteq i(x) \sqcup i(y)$$

and so

$$x = j(i(x)) \sqsubseteq j(i(x) \sqcup i(y)).$$

Similarly

$$y \sqsubseteq j(i(x) \sqcup i(y)),$$

whence

$$x \sqcup y \sqsubseteq j(i(x) \sqcup i(y)).$$

But then

$$i(x \sqcup y) \sqsubseteq i(j(i(x) \sqcup i(y))) \sqsubseteq i(x) \sqcup i(y),$$

which completes the argument for equation (i).

Condition (ii) is obvious. For (iii) we argue as follows. Assume $x \ll y$. Since D' is continuous we can write:

$$i(y) = \bigsqcup \{z' \in D' : z' \ll i(y)\},$$

and conclude by the continuity of j that:

$$y = j(i(y)) = \bigsqcup j(z') : z' \ll i(y).$$

But $x \ll y$, so $x \ll j(z')$ for some $z' \ll i(y)$. Now $x \sqsubseteq j(z')$ follows; therefore $i(x) \sqsubseteq i(j(z')) \sqsubseteq z'$. Thus $i(x) \ll i(y)$.

Turning now to the converse, assume of the map i that it satisfies (i)–(iii). Compute:

$$i(j(x')) = \bigsqcup i(x) : i(x) \sqsubseteq x' \sqsubseteq x'.$$

This is correct because i is continuous and the set $\{x : i(x) \sqsubseteq x'\}$ is directed in view of condition (i). Thus $i \circ j \sqsubseteq \mathrm{id}_{D'}$. Note that by virtue of (i) and (ii) it is the case that

$$i(x) \sqsubseteq i(y) \text{ implies } x \sqsubseteq y.$$

(The reason is that $x \sqsubseteq y$ is equivalent to $x \sqcup y = y$.) This remark allows us to compute:

$$j(i(y)) = \bigsqcup\{x : i(x) \sqsubseteq i(y)\}$$
$$= \bigsqcup\{x : x \sqsubseteq y = y\}.$$

Hence, $j \circ i = \mathrm{id}_D$. It remains to show that j is continuous. Suppose $S' \subseteq D'$ is directed. Since j is by definition monotonic, it is sufficient to prove that

$$j(\bigsqcup S') \sqsubseteq \bigsqcup j(x') : x' \in S'.$$

Now

$$j(\bigsqcup S') = \bigsqcup\{x : i(x) \sqsubseteq \bigsqcup S'\},$$

so suppose $i(x) \sqsubseteq \bigsqcup S'$. Let $z \ll x$; whence $i(z) \ll i(x)$. Thus $i(z) \ll x'$ for some $x' \in S'$, and therefore $i(z) \sqsubseteq x'$. We obtain then $z \sqsubseteq j(x')$, which means that

$$z \sqsubseteq \bigsqcup j(x') : x' \in S'$$

holds for all $z \ll x$. By the continuity of D we conclude

$$x \sqsubseteq \bigsqcup j(x') : x' \in S'$$

holds for all x with $i(x) \sqsubseteq \bigsqcup S'$. The desired result follows. □

As a corollary of 3.10 we can easily see which *subspaces* of a continuous lattice D' are projections of it. Such a subspace $D \subseteq D'$ must first be closed under $\bigsqcup$. That is, if $S \subseteq D$, then $\bigsqcup S \in D$ for

all S, where the lub is taken in the sense of D'. The identity map on D will then satisfy (i) and (ii). But this is not enough, since we would not know that D is a continuous lattice, nor whether (iii) holds. The following additional condition would be sufficient, if assumed for all $y \in D$:

$$y = \bigsqcup\{x \in D : x \ll y\},$$

where $\ll$ is taken in the sense of D'. This implies that

$$y = \bigsqcup\{\sqcap(D \cap U) : y \in U\}$$

where U ranges over the open subsets of D' and where the $\sqcap$ is taken in the sense of D. This condition makes the subspace topology the same as the lattice topology on D and besides makes D continuous, which is just what we need. (Another way to put it is that whenever $z \ll y$, where $y \in D$ but $z \in D'$, then $z \sqsubseteq x \ll y$, for some $x \in D$.)

It seems a bit troublesome to characterize in a simple way just which maps $j : D' \to D$ are projections. (Other than saying outright that the map $i : D \to D'$ such that for all $x \in D$:

$$i(x) = \sqcap\{x' \in D' : x \sqsubseteq j(x')\}$$

is the continuous partial inverse to j.) But we can say very easily which continuous maps $j : D' \to D'$ are projections onto *subspaces*; namely, we must have

$$j = j \circ j \sqsubseteq \mathrm{id}.$$

The subspace in question then is:

$$D = \{x \in D' : j(x) = x\}.$$

This non-empty subspace is the exact range of j and is closed under $\bigsqcup$. Let $y \in D$. Then if $x' \ll y$ in D', we find $j(x') \sqsubseteq x' \ll y$. Thus since

$$y = \bigsqcup\{x' \in D' : x' \ll y\},$$

we see that

$$y = j(y) = \bigsqcup\{j(x') : x' \ll y\}.$$

But each $j(x') \in D$, so $y = \bigsqcup\{x \in D : x \ll y\}$, as desired.

The foregoing discussion suggests looking more closely at the space of all projections of a continuous lattice since they are so easily characterized.

proposition_3_10.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.ConverseProjection

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

theorem proposition_3_10 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') (i :
    D → D') :
    ((∀ S : Set D, i (sSup S) = sSup (Set.image i S))
      ∧ Function.Injective i
      ∧ (∀ x y : D, x ≪ y → i x ≪ i y))
      ↔
    (∃ P : ContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup { x | i x ≤ x' })) := by
  constructor
  · rintro ⟨hi, hinj, hwb⟩
    refine ⟨ConverseProjection i hi hinj
      (fun {x y} h => hwb x y h) hD, fun _ => rfl, fun _ => rfl⟩
  · rintro ⟨P, hincl, _⟩
    refine ⟨?_, ?_, ?_⟩
    · intro S
      rw [← hincl (sSup S), P.incl_sSup]
      congr 1
      ext y
      simp [Set.mem_image, hincl]
    · intro x y h
```

```
    have h' : (P.incl : D → D') x = (P.incl : D → D') y := by rw [hincl x, hincl y,
   h]
    exact P.incl_injective h'
  · intro x y h
    rw [← hincl x, ← hincl y]
    exact P.incl_wayBelow hD' h

end DomainTheory.ContinuousLattice
```

2.4.20 Definition 3.11: Projection space ✓

3.11 Definition. Given a continuous lattice D, we let the space of projections be denoted by:

$$J_D = j \in [D \to \{D] : j = j \circ j \sqsubseteq \mathrm{id}\}.$$

The formalization is `ProjectionSpace.lean`. **`ProjectionSpace D`** is Scott's J_D; **`ProjectionSpace.mem j`** is membership.

Lemma (`iff`). **`mem j`** iff idempotence and **`j x ≤ x`** pointwise on the coercion.

Lemma (`bot_mem`). $\bot \in$ **`J_D`**.

Lemma (`sup_mem`). / **`finsetSup_mem`**. / **`directedSup_mem`**. / **`sSup_mem`**. — **`mem`** is closed under finite and arbitrary sups in **`[D → D]`**.

Instance (`instSupSet`). / **`isLUB_sSup`**. / **`instCompleteLattice`**. — **`ProjectionSpace D`** is a complete lattice (Proposition 3.12).

`ProjectionSpace.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

variable {D : Type*} [CompleteLattice D]

namespace ProjectionSpace

/-- Scott's Def. 3.11: $j \in [D \to D]$ with $j \circ j = j$ and $j \sqsubseteq \
    mathrm{id}$. -/
def mem (j : ScottMap D D) : Prop :=
  j.comp j = j ∧ j ≤ ScottMap.idMap

theorem iff {j : ScottMap D D} :
    mem j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
```

```
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩

theorem bot_mem : mem (⊥ : ScottMap D D) := by
  rw [iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

theorem sup_mem {j k : ScottMap D D} (hj : mem j) (hk : mem k) : mem (j ⊔ k) := by
  rw [iff] at hj hk ⊢
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

theorem directedSup_mem {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, mem j) : mem (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x := fun j hj => (iff.mp (hS j hj))
    .1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (iff.mp (hS j hj)).2
  rw [iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
```

```
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

theorem finsetSup_mem {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, mem j) :
    mem (T.sup id) :=
  Finset.sup_induction bot_mem (fun _ ha _ hb => sup_mem ha hb) hT

theorem sSup_mem {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, mem j) : mem (sSup T) := by
  rw [ContinuousLatticeProjection.sSup_finsetSupOf T]
  refine directedSup_mem ⟨⊥, ∅, by simp, by simp⟩
    (ContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact finsetSup_mem fun j hj => hT j (hF j hj)

end ProjectionSpace

/-- Scott's projection space $J_D$ (Definition 3.11). -/
abbrev ProjectionSpace (D : Type*) [CompleteLattice D] : Type _ :=
  { j : ScottMap D D // ProjectionSpace.mem j }

namespace ProjectionSpace

noncomputable instance instSupSet : SupSet (ProjectionSpace D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), sSup_mem (by rintro j ⟨p, _, rfl⟩; exact p
    .2)⟩⟩

theorem isLUB_sSup (T : Set (ProjectionSpace D)) : IsLUB T (sSup T) := by
  constructor
  · intro p hp
    apply Subtype.coe_le_coe.mp
    show (p : ScottMap D D) ≤ ((sSup T : ProjectionSpace D) : ScottMap D D)
    exact le_sSup (Set.mem_image_of_mem _ hp)
  · intro q hq
    apply Subtype.coe_le_coe.mp
    show ((sSup T : ProjectionSpace D) : ScottMap D D) ≤ (q : ScottMap D D)
    refine sSup_le ?_
    rintro _ ⟨p, hp, rfl⟩
    exact Subtype.coe_le_coe.mpr (hq hp)

noncomputable instance instCompleteLattice : CompleteLattice (ProjectionSpace D) :=
  completeLatticeOfSup (ProjectionSpace D) isLUB_sSup

end ProjectionSpace

end DomainTheory.ContinuousLattice
```

2.4.21 Proposition 3.12 ✓

3.12 Proposition. For a continuous lattice D the space J_D of projections forms a complete lattice as a $\bigsqcup$-closed subspace of $[D \to D]$.

Proof: The constant function $\bot \in J_D$ obviously, so J_D contains $\bigsqcup \emptyset$. Suppose $j, k \in J_D$. We wish to show that $j \sqcup k \in J_D$. Compute:

$$(j \sqcup k)((j \sqcup k)(x)) = j(j(x) \sqcup k(x)) \sqcup k(j(x) \sqcup k(x)).$$

But note:

$$j(x) \sqsubseteq j(j(x)) \sqsubseteq j(j(x) \sqcup k(x)) \sqsubseteq j(x),$$

because $j(x) \sqcup k(x) \sqsubseteq x$. Similarly for $k(x)$. Therefore, we find that $(j \sqcup k) \circ (j \sqcup k) = j \sqcup k \sqsubseteq \mathrm{id}$.

Suppose finally that $S \subseteq J_D$ is directed. We wish to show that $\bigsqcup S \in J_D$. Clearly $\bigsqcup S \sqsubseteq \mathrm{id}$, so compute by continuity of $\circ$:

$$\bigsqcup S \circ \bigsqcup S = \bigsqcup \{j \circ j : j \in S\} = \bigsqcup \{j : j \in S\} = \bigsqcup S.$$

It follows that J_D is $\bigsqcup$-closed and hence is a complete lattice. $\square$

The significance of the above result becomes clearer if we consider the connection between projections and subspaces. Let us write:

$$D(j) = \{x \in D : j(x) = x\}.$$

For $j \in J_D$, each $D(j)$ is a projection of D onto a subspace. We show first that

$$j \sqsubseteq k \quad \text{iff} \quad D(j) \subseteq D(k).$$

Because if $j \sqsubseteq k$, then $j \sqsubseteq j \circ j \sqsubseteq k \circ j \sqsubseteq \mathrm{id} \circ j = j$. Therefore if $j(x) = x$, then $k(x) = k(j(x)) = j(x) = x$, which means that $D(j) \subseteq D(k)$. On the other hand, if $D(j) \subseteq D(k)$, then since $j(D) \subseteq D(j)$, we see that $k \circ j = j$, and so $j \sqsubseteq k \circ \mathrm{id} = k$. Hence J_D is isomorphic to the partially ordered set of subspaces of D that are projections. We thus conclude that these subspaces form a lattice. In fact, it is easy to show that

$$D(j \sqcup k) = x \sqcup \{y : x \in D(j)\}, y \in D(k).$$

Similarly, if S is a directed set of J_D, then $D(\bigsqcup S)$ is the $\bigsqcup$-closure in D of the subset:

$$\{\bigcup D(j) : j \in S.\}$$

These are not very deep facts, but their proofs were very much facilitated by the introduction of J_D and the utilization of the lattice structure of $[D \to D]$. Along the same line we can define for $j, k \in J_D$ a function $(j \to k) \in [D \to D] \to [D \to D]$ by the equation

$$(j \to k)(f) = k \circ f \circ j.$$

It is very easy to show that $(j \to k) \in J_{[D \to D]}$, that $(j \to k)$ is continuous in j and k, and that $[D \to D](j \to k)$ is isomorphic to $[D(j) \to D(k)]$. There are many other interesting operations on projections corresponding to other constructs besides these. And, just as with $(j \to k)$, the operations are

continuous. This makes it possible to prove existence theorems about subspaces by using results like the fixed-point theorem for continuous functions. It would be even nicer if J_D turns out to be a continuous lattice itself, but as far as I can tell this is not likely to be the case.

Before we turn to the iterated function-space construction by inverse limits, there are a couple of other connections between spaces and function spaces that are useful to know.

proposition_3_12.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ProjectionSpace

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

variable {D : Type*} [CompleteLattice D]

theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, ProjectionSpace.mem j) → ProjectionSpace.mem
    (sSup T))
      ∧ Nonempty (CompleteLattice (ProjectionSpace D)) :=
  ⟨fun _ h => ProjectionSpace.sSup_mem h, ⟨ProjectionSpace.instCompleteLattice⟩⟩

end DomainTheory.ContinuousLattice
```

2.4.22 Proposition 3.13 ✓

3.13 Proposition. Every continuous lattice D is a projection of its function space $[D \to D]$.

Proof: Consider the following pair of mappings $\mathrm{con} : D \to [D \to D]$ and $\mathrm{min} : [D \to D] \to D$ where

$$\mathrm{con}(x)(y) = x$$

and

$$\mathrm{min}(f) = f(\bot)$$

for all $x, y \in D$ and $f \in [D \to D]$. They are obviously continuous. The map con matches every element of D with the corresponding *constant* function in $[D \to D]$. The map min associates to every function in $[D \to D]$ its *minimum* value in the partial ordering. The proof that this pair forms a projection is trivial. □

The pair con, min are not the only pair for making D a projection of $[D \to D]$. The following pair of maps were suggested by David Park:

$$\lambda x.\vec{e}[t, x] \quad \text{and} \quad \lambda f.f(t),$$

where x ranges over D, and f over $[D \to D]$, and where t is a fixed *isolated* element of D (that is, $t \ll t$ holds). The pair con and min will result if we set $t = \bot$. (Note that the expression $\vec{e}[t, x]$, though

continuous in x, is not continuous—or even monotonic—in the variable t.) A lattice may very well possess a large number of isolated

elements, whence a large number of projections. And furthermore this is the only way the function $j = \lambda f.f(t)$ can be a projection. For assume the existence of an inverse $i : D \to [D \to D]$ satisfying the proper conditions. Then it would be the case that

$$i(x)(t) = x$$

and

$$i(f(t))(y) \sqsubseteq f(y)$$

for all $x, y \in D$ and all $f \in [D \to D]$. We can prove for all $u \in D$, if $t \not\sqsubseteq u$, then

$$i(x)(u) = \bot$$

by substituting $\vec{e}[v, x]$ for f in the second equation above, where v is chosen so that $v \ll t$ but not $v \ll u$. But then note that

$$i(x)(t) = \bigsqcup\{i(x)(u) : u \ll t\}.$$

If not $t \ll t$, then $u \ll t$ implies $t \not\sqsubseteq u$, which leads to absurdity. Hence t must be isolated, and, as we noted earlier, the function i is uniquely determined as being the one we already knew. Aside from these pairs of projections one could obtain others by combinations with automorphisms. I was unable to determine whether there are further pairs of an essentially different nature.

The topic of projections in these spaces is rather interesting since one has in a way more freedom in T_0-spaces (particularly in injective spaces) than in ordinary spaces for defining functions. As another example, consider the Cartesian square $D \times D$. Aside from the two obvious projections onto D, there is also the "diagonal" system given by the pair:

$$\lambda x.(x, x) \qquad \text{and} \qquad \lambda(x, y).x \sqcap y$$

We shall note in the next section how the choice of an initial projection effects the construction of an inverse limit.

The projections are not the only useful functions in $[D \to D] \to D$. As a final example of what can be done in function spaces we mention the fixed-point operator.

proposition_3_13.lean

```
module

import DomainTheory.ContinuousLattice.FunctionSpaces.Projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D : Type*} [CompleteLattice D]
```

```
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (ContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨ScottMap.conMinContinuousLatticeProjection⟩

end DomainTheory.ContinuousLattice
```

2.4.23 Proposition 3.14 ✓

3.14 Proposition. *For a continuous lattice D there is a uniquely determined continuous mapping*

$$\mathrm{fix} : [D \to D] \to D$$

where for all $f \in [D \to D]$ and $x \in D$

$$f(\mathrm{fix}(f)) = \mathrm{fix}(f)$$

and whenever $f(x) \sqsubseteq x$, then $\mathrm{fix}(f) \sqsubseteq x$.
and whenever $f(x) \sqsubseteq x$, then

$$\mathrm{fix}(f) \sqsubseteq x.$$

Proof: The proof of the existence of minimal fixed points in complete lattices is well known, as was mentioned in the proof of 2.10.

To establish the continuity, it is sufficient to remark that since all functions $f \in [D \to D]$ are continuous, we have

$$\mathrm{fix}(f) = \bigsqcup_{n=0}^{\infty} f^n(\bot),$$

where $f^n(x) = f(f(\ldots f(x)\ldots))$ (n times). Thus fix is the point-wise lub of continuous functions on $[D \to D]$ and is thus itself continuous. □

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D : Type*} [CompleteLattice D]

theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
```

```
  ⟨ScottMap.fixMap, fun f => ScottMap.fix_eq f,
    fun _ _ h => ScottMap.fix_le h,
    fun _ hfix hleast f => ScottMap.fix_unique (hfix f) (fun x hx => hleast f x hx)⟩

end DomainTheory.ContinuousLattice
```

2.5 Inverse limits ✓

Scott's §4 opens with a review of inverse limits for T_0-spaces: an inverse system and compatible sequences, the formal inverse limit type for $\langle D_n, P_n \rangle$, projections from the limit ($j_{\infty n}$), the mediating map into X_∞, and the turn to special inverse systems. The notes that follow build the projection tower ($\mathrm{projectionAlongTower}$, adjunction at one step), embeddings into the limit, lattice and topology on D_∞, and Propositions 4.1–4.4.

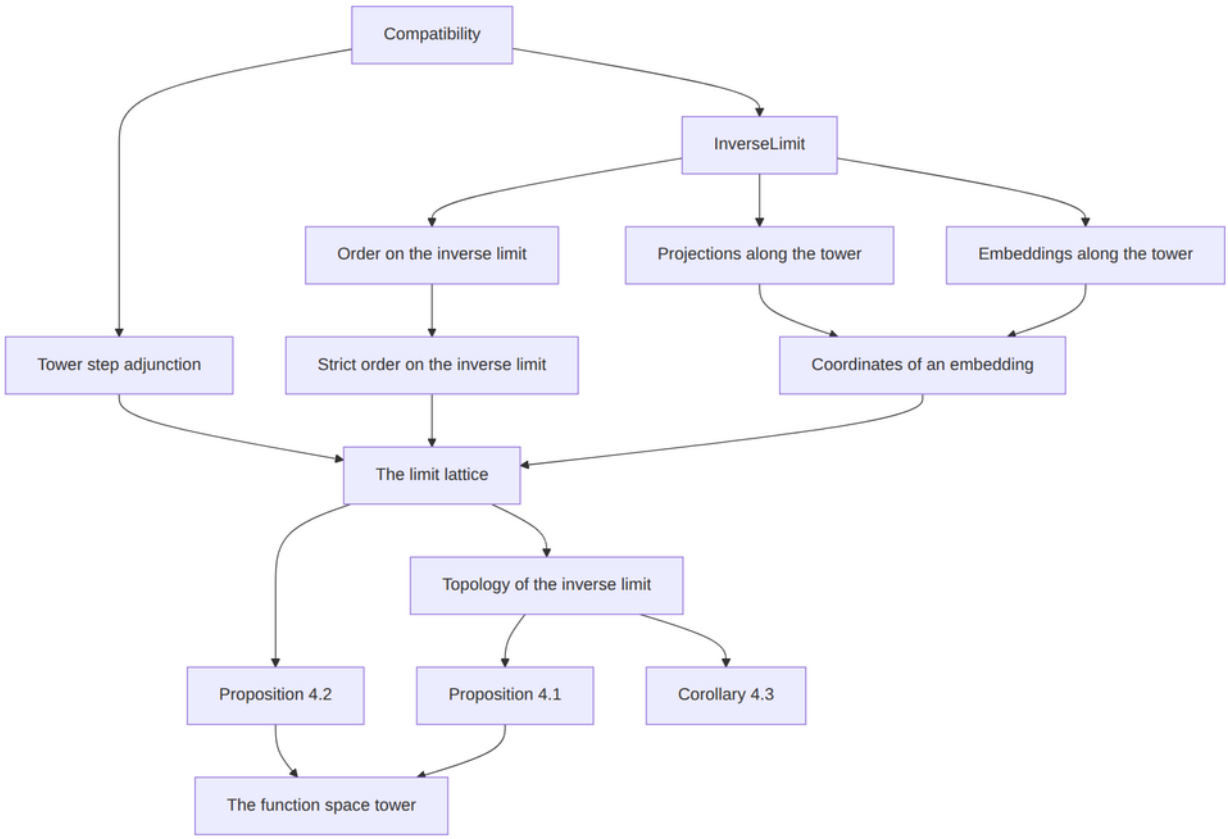


Figure 14: Inverse limits ✓.

2.5.1 Compatibility ✓

By an *inverse system of spaces* we understand as usual a sequence

$$\langle X_n, j_n \rangle_{n=0}^{\infty}$$

of T_0-spaces and continuous maps $j_n : X_{n+1} \to X_n$. The space X_∞, called the *inverse limit* of the sequence, is constructed in the familiar way as that subspace of the product space consisting of exactly those infinite sequences

$$x = \langle x_n \rangle_{n=0}^{\infty},$$

where for each n we have $x_n \in X_n$, and

$$j_n(x_{n+1}) = x_n.$$

For the continuous-lattice tower $\langle D_n, P_n \rangle$, j_n is **`(P n).retr`**.

Definition (`Compatible`). **`Compatible D P x`** holds when $\forall n,\ (P\,n).\mathrm{retr}\,(x\,(n+1)) = x\,n$ for **`x : ∀ n, D n`**.

`Compatible.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end DomainTheory.ContinuousLattice
```

2.5.2 InverseLimit ✓

The preceding paragraphs describe X_∞ as the subspace of compatible sequences. For $\langle D_n, P_n \rangle$ the library name is **`InverseLimit D P`**.

Definition (`InverseLimit`). **`InverseLimit D P`** is the subtype of tuples **`x : ∀ n, D n`** that satisfy **`Compatible`** (Scott's D_∞ before topology and lattice structure are imposed).

`InverseLimit.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u
```

```
variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}

end DomainTheory.ContinuousLattice
```

2.5.3 Projections from the limit ✓

The space X_∞ is given the product topology, and the maps $j_{\infty\{n} : X_\infty \to X_n\}$ such that

$$j_{\infty n}(x) = x_n$$

are of course continous and satisfy the recursion equation:

$$j_{\infty n} = j_n \circ j_{\infty(n+1)}.$$

On **`InverseLimit D P`**, **`limitProjectionToLevel n`** is Scott's $j_{\infty n}$: the n-th coordinate **`y.1 n`**, packaged as a **`ScottMap`** once directed suprema on the limit are available.

2.5.4 The mediating map ✓

Besides this we have the expected extension property for any system of continuous maps

$$f_n : Y \to X_n$$

where for each n

$$f_n = j_n \circ f_{n+1}.$$

Because, we can define

$$\{f_\infty : Y \to X_\infty\}$$

by the equation

$$f_\infty(y) = \langle f_n(y) \rangle_{n=0}^{\infty}$$

for all $y \in Y$; whence

$$f_n = j_{\infty n} \circ f_\infty$$

holds.

Category theory (terms) **Diagram / tower.** For $\langle D_n, P_n\rangle$, write $j_n : D_{n+1} \to D_n$ for $(P\,n)$.retr and $i_n : D_n \to D_{n+1}$ for $(P\,n)$.incl. This is a diagram indexed by $\mathbb{N}$ (vertices D_n, bonds the j_n).

Inverse limit. A **limit** of that diagram is an object D_∞ together with **projections** $j_{\infty\{n} : D_\infty \to D_n\}$ such that $j_{\infty n} = j_n \circ j_{\infty(n+1)}$. In Lean, D_∞ is **`InverseLimit D P`** and $j_{\infty n}$ is **`limitProjectionToLevel n`**.

Cone. A **cone** over the diagram consists of an **apex** object Y and **legs** $u_n : Y \to D_n$ that agree with the tower: $u_n = j_n \circ u_{n+1}$. A **mediating map into the limit** is a morphism $\{u_\infty : Y \to D_\infty\}$ with $u_n = j_{\infty n} \circ u_\infty$ for all n.

Cocone. A **cocone** over the **same** diagram consists of an apex D' and legs $f_n : D_n \to D'$ in the opposite direction, compatible with **inclusions**:

$$f_n = f_{n+1} \circ i_n$$

(in Lean: **`hf : ∀ n x, (f n) x = (f (n + 1)) ((P n).incl x)`** for **`f n : ScottMap (D n) D'`**). A **mediating map out of the limit** is $\{g : D_\infty \to D'\}$ with $f_n = g \circ i_{n\infty}$; the library name for $i_{n\infty}$ is **`embInf`**.

Name `coconeInf`. cocone + Inf means a cocone whose mediating map has **domain the inverse limit** (`Inf` = inverse limit, **not** a meet / infimum $\sqcap$ in the order). The cocone **legs** are the family **`f n`**; the **Inf** part is where g is defined.

Supremum in the formula. On continuous lattices, a point $x \in D_\infty$ is determined by its coordinates and by directed suprema of approximants. The mediating value $g(x)$ is expressed as a **supremum** $\bigsqcup_n$ over levels of the leg values $(f\,n)(x_n)$—lattice **sup**, not an infimum over n.

Reading Scott's symbols in category-theoretic terms Scott's $f_n : Y \to X_n$ are the **legs of a cone** with apex Y: compatibility is exactly $f_n = j_n \circ f_{n+1}$. His $\{f_\infty : Y \to X_\infty\}$ is the **mediating map into the inverse limit**, assembling a compatible sequence $f_\infty(y) = \langle f_n(y)\rangle$, with factorization $f_n = j_{\infty n} \circ f_\infty$. This paragraph is **not** a cocone (all arrows point **into** the stages, not **out of** them).

For $\langle D_n, P_n\rangle$, X_∞ is **`InverseLimit D P`**. The dual **cocone** story—maps $D_n \to D'$, mediating map **out of** the limit, and **`coconeInf`**—is what Corollary 4.3 formalizes; Scott's displayed f_n are not those legs.

Definition (`coconeInf`) Given **`f : ∀ n, ScottMap (D n) D'`** and **`x : InverseLimit D P`**,

$$\texttt{coconeInf}\,D\,P\,f\,x = \bigsqcup_{n=0}^{\infty} (f\,n)\,(x.1\,n).$$

When **`hf`** holds, **`coconeInf D P f`** is the unique mediating **`ScottMap (InverseLimit D P) D'`** (as **`ScottMap`**, via **`preservesDirectedSup`**).

Theorems in the namespace `coconeInf` The definition **`coconeInf`** sits outside the namespace; the lemmas below are **`coconeInf.apply`**, **`coconeInf.climb`**, and so on.

Theorem (`coconeInf.apply`). *For **`f`** and **`x`**,*

$$\texttt{coconeInf}\,D\,P\,f\,x = \bigsqcup_{n} (f\,n)\,(x.1\,n).$$

Proof. Definition of **coconeInf**. □

Theorem (coconeInf.climb). *If* **hf**, *then for* **n**, **x : D n**, *and* **h : n ≤ m**,

$$(f\,m)\,(\text{embeddingAlongTower}\,D\,P\,h\,x) = (f\,n)\,x.$$

Proof. **Nat.le_induction**: base **embeddingAlongTower.self**, successor **embeddingAlongTower.succ** and **hf**. □

Theorem (coconeInf.descend). *If* **hf**, *for* **h : m ≤ n** *and* **x : D n**,

$$(f\,m)\,(\text{projectionAlongTower}\,D\,P\,h\,x) \le (f\,n)\,x.$$

Proof. **Nat.le_induction**: base **projectionAlongTower.self**, successor **projectionAlongTower.su** **hf**, **incl_retr_le**. □

Theorem (coconeInf.comp_embInf). *If* **hf**, *for* **n**, **x : D n**,

$$\texttt{coconeInf}\;D\,P\,f\,(\text{embInf}\,D\,P\,n\,x) = (f\,n)\,x.$$

Proof. **iComp.of_le** / **iComp.of_ge**, **climb**, **descend**, and **iComp.self**. □

Theorem (coconeInf.preservesDirectedSup). *For every* **f**, ***coconeInf D P f*** *preserves directed suprema.*

Proof. Coordinatewise **limitProjectionToLevel.preservesDirectedSup**, directed sup for each **f n**, **apply**, **iSup_comm**. □

Corollary 4.3: **coconeInf D P f** is the unique mediating cocone **ScottMap**—existence from **comp_embInf**, uniqueness from **inverseLimit.eq_iSup**.

2.5.5 Special inverse systems ✓

So much for a review of inverse limits. In this paper our interest will center on rather special inverse systems and their limits.

2.5.6 Projections along the tower ✓

For an inverse system of continuous lattices $\langle D_n, P_n\rangle$ with $j_n = (P\,n).\text{retr} : D_{n+1} \to D_n$, Scott iterates the j_n but does not introduce a separate symbol for the composite. When $m \le n$, descending from level n to m means applying $j_{n-1}, j_{n-2}, \ldots, j_m$ (dually, Embeddings along the tower climbs with $i_k = (P\,k).\text{incl}$).

Definition (projectionAlongTower). Given **h : m ≤ n**, **projectionAlongTower D P h** is the map $D_n \to D_m$ obtained by recursion on **h**, at each successor step applying **(P k).retr**.

In the namespace **projectionAlongTower** we have

Theorem (self). *For* **h : m ≤ m**, ***projectionAlongTower D P h*** *is the identity on* D_m.

Proof. The recursion for **Nat.leRecOn** at a reflexive proof is **id**. □

Theorem (succ). *If* **h1 : m ≤ n** *and* **h2 : m ≤ n + 1**, *then for all* **z : D (n + 1)**,

$$\text{projectionAlongTower}\,D\,P\,h_2\,z = \text{projectionAlongTower}\,D\,P\,h_1\,(j_n(z)).$$

Proof. Unfold **projectionAlongTower** and **Nat.leRecOn_succ**. □

Theorem (mono). *If `h : m ≤ n` and `x ≤ y` in D_n, then **`projectionAlongTower D P h x ≤ projectionAlongTower D P h y`.*

Proof. Induct on **n** and **h** with **Nat.le_induction**. The base case is **self**. For the successor step, rewrite with **succ** and use monotonicity of **(P k).retr**. □

Theorem (compatible). *If* **y : InverseLimit D P**, *then for every* **n** *and* **h : m ≤ n**,

$$\mathrm{projectionAlongTower}\ D\ P\ h\ (y\,n) = y\,m.$$

Proof. Induct on **n** and **h**. The base is **self**; the successor step uses **succ** and compatibility **(P k).retr (y (k + 1)) = y k**. □

Theorem (retr). *If* **h : m + 1 ≤ n**, *then for all* **x : D n**,

$$j_m\big(\mathrm{projectionAlongTower}\ D\ P\ h\ x\big) = \mathrm{projectionAlongTower}\ D\ P\ h'\ x \quad \text{for the unique } h' : m \le n \text{ with } m{+}1 \le n.$$

Proof. Induct on **n** and **h**, unfolding **succ** and using **self** at the base. □

Theorem (preservesDirectedSup). *If* **h : m ≤ n**, *then* **projectionAlongTower D P h** *preserves directed suprema.*

Proof. Induct on **h**. At **m = n**, **self** gives the identity. For the successor step, **succ** expresses **projectionAlongTower** as a composite of the lower **projectionAlongTower** with **(P k).retr**; apply **ScottMap.preservesDirectedSup_comp** and directed-sup preservation for **(P k).retr**. □

projectionAlongTower.lean

```
module
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

def projectionAlongTower {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projectionAlongTower

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projectionAlongTower D P h x = x := by
  simp only [projectionAlongTower, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projectionAlongTower D P h2 z = projectionAlongTower D P h1 ((P n).retr z) := by
  simp only [projectionAlongTower]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]
```

```
theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projectionAlongTower D P h x ≤ projectionAlongTower D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projectionAlongTower D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projectionAlongTower D P h x) = projectionAlongTower D P (Nat.
    le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projectionAlongTower D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projectionAlongTower D P (le_refl m) = id :=
        funext (fun x => self D P (le_refl m) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projectionAlongTower D P (Nat.le_succ_of_le hk)
          = projectionAlongTower D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projectionAlongTower

end InverseLimit

end DomainTheory.ContinuousLattice
```

2.5.7 Tower step adjunction ✓

For an inverse system $\langle D_n, P_n \rangle$, fix one index n. Write $i_n = (P\,n).\mathrm{incl} : D_n \to D_{n+1}$ and $j_n = (P\,n).\mathrm{retr} : D_{n+1} \to D_n$ for the continuous lattice projection on that **step** (one rung of the tower, not the composite descent in Projections along the tower).

What an adjunction is (order theory) Two posets and two monotone maps $f : P \to Q$, $g : Q \to P$ form an **adjunction** $f \dashv g$ when comparisons after applying f are equivalent to comparisons after applying g:

$$f(x) \le y \iff x \le g(y) \qquad (x \in P,\ y \in Q).$$

Same question on the left always matches the same question on the right. In Lean this predicate is named **`GaloisConnection`**; "Galois connection" and "adjunction between posets" mean the same thing here. The map on the left (f) is the **left adjoint**, the map on the right (g) the **right adjoint**.

Reading the equivalence for one tower step: embedding $x \in D_n$ into D_{n+1} and comparing to y is the same as comparing x to the retraction $j_n(y) \in D_n$.

Why a projection step is an adjunction A continuous lattice projection is not merely two monotone maps. It satisfies $j_n \circ i_n = \mathrm{id}_{D_n}$ (**`retr_incl`**) and $i_n(j_n(y)) \le y$ for all $y \in D_{n+1}$ (**`incl_retr_le`**). Those two facts are exactly the two directions of the equivalence above:

- If $i_n(x) \le y$, apply monotone j_n and use $j_n(i_n(x)) = x$ to get $x \le j_n(y)$.
- If $x \le j_n(y)$, apply monotone i_n and use $i_n(j_n(y)) \le y$ to get $i_n(x) \le y$.

Without **`incl_retr_le`**, a retraction alone would not prove the second implication in general. The library lemma **`tower_step_galoisConnection`** records the packaged result.

Theorem (`tower_step_galoisConnection`). *For every* ***`n : ℕ`****,* ***`GaloisConnection ((P n).incl) ((P n).retr)`****: for all* ***`x : D n`****,* ***`y : D (n + 1)`****,*

$$i_n(x) \le y \iff x \le j_n(y).$$

Proof. The forward implication applies **`(P n).retr`** monotonically and uses **`retr_incl`**. The reverse uses monotonicity of **`incl`** and **`incl_retr_le`**. □

This adjunction is used when building infima on the inverse limit (see The limit lattice): the right adjoint **`retr`** preserves arbitrary meets.

`tower_step_galoisConnection.lean`

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u
```

```
section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

/-- At each index `n` of the tower, `(P n).incl` and `(P n).retr` form a Galois
    connection. -/
theorem tower_step_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
     by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```

2.5.8 Embeddings along the tower ✓

For an inverse system of continuous lattices $\langle D_n, P_n\rangle$ with $i_n = (P\,n).\mathrm{incl} : D_n \to D_{n+1}$, Scott iterates the i_n but does not introduce a separate symbol for the composite. When $n \le m$, climbing from level n to m means applying $i_n, i_{n+1}, \dots, i_{m-1}$ (dually, Projections along the tower descends with $j_k = (P\,k).\mathrm{retr}$). Each i_n is the inclusion half of the continuous lattice projection on one **step** of the tower (Tower step adjunction), not the composite climb defined here.

Definition (`embeddingAlongTower`). Given **`h : n ≤ m`**, **`embeddingAlongTower D P h`** is the map $D_n \to D_m$ obtained by recursion on **`h`**, at each successor step applying **`(P k).incl`**.

In the namespace **`embeddingAlongTower`** we have

Theorem (`self`). *For **`h : n ≤ n`**, **`embeddingAlongTower D P h`** is the identity on D_n.*

Proof. The recursion for **`Nat.leRecOn`** at a reflexive proof is **`id`**. □

Theorem (`succ`). *If **`h1 : n ≤ m`** and **`h2 : n ≤ m + 1`**, then for all **`x : D n`**,*

$$\mathrm{embeddingAlongTower}\ D\ P\ h_2\ x = i_m\big(\mathrm{embeddingAlongTower}\ D\ P\ h_1\ x\big).$$

Proof. Unfold **`embeddingAlongTower`** and **`Nat.leRecOn_succ`**. □

Theorem (`succ_left`). *If **`h1 : n ≤ m`** and **`h2 : n + 1 ≤ m`**, then for all **`x : D n`**,*

$$\mathrm{embeddingAlongTower}\ D\ P\ h_2\ (i_n(x)) = \mathrm{embeddingAlongTower}\ D\ P\ h_1\ x.$$

Proof. Unfold **`embeddingAlongTower`** and apply **`Nat.leRecOn_succ_left`**. □

Theorem (`mono`). *If **`h : n ≤ m`** and **`x ≤ y`** in D_n, then **`embeddingAlongTower D P h x ≤ embeddingAlongTower D P h y`.*

Proof. Induct on **`m`** and **`h`** with **`Nat.le_induction`**. The base case is **`self`**. For the successor step, rewrite with **`succ`** and use monotonicity of **`(P k).incl`**. □

Theorem (`le`). *If **`y : InverseLimit D P`** and **`h : n ≤ m`**, then*

$$\mathrm{embeddingAlongTower}\ D\ P\ h\ (y\,n) \le y\,m.$$

Proof. Induct on **m** and **h**. The base is **self**. For the successor step, use **succ**, monotonicity of **(P k).incl**, compatibility **(P k).retr (y (k + 1)) = y k**, and **incl_retr_le**. □

Theorem (preservesDirectedSup). *If* ***h : n ≤ m****, then* ***embeddingAlongTower D P h*** *preserves directed suprema.*

Proof. Induct on **h**. At **n = m**, **self** gives the identity. For the successor step, **succ** expresses **embeddingAlongTower** as a composite of **(P k).incl** with the lower **embeddingAlongTower**; apply **ScottMap.preservesDirectedSup_comp** and directed-sup preservation for **(P k).incl**. □

embeddingAlongTower.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

def embeddingAlongTower {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

namespace embeddingAlongTower

theorem self {n : ℕ} (h : n ≤ n) (x : D n) : embeddingAlongTower D P h x = x := by
  simp only [embeddingAlongTower, Nat.leRecOn_self]

theorem succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embeddingAlongTower D P h2 x = (P m).incl (embeddingAlongTower D P h1 x) := by
  simp only [embeddingAlongTower]
  rw [Nat.leRecOn_succ h1]

theorem succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embeddingAlongTower D P h2 ((P n).incl x) = embeddingAlongTower D P h1 x := by
  simp only [embeddingAlongTower]
  exact Nat.leRecOn_succ_left x h1 h2

theorem le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embeddingAlongTower D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (self D P _ _)
  | succ k hk ih =>
```

```
      rw [succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embeddingAlongTower D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _

theorem mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embeddingAlongTower D P h x ≤ embeddingAlongTower D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact (P k).incl.monotone ih

theorem preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embeddingAlongTower D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embeddingAlongTower D P (le_refl n) = id :=
        funext (fun x => self D P (le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embeddingAlongTower D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) ∘ embeddingAlongTower D P hk :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end embeddingAlongTower

end InverseLimit

end DomainTheory.ContinuousLattice
```

2.5.9 Coordinates of an embedding ✓

A point of the inverse limit $D_\infty = \text{InverseLimit}\ D\ P$ is a compatible family $(y_m)_{m\in\mathbb{N}}$ with $y_m \in D_m$ and $j_m(y_{m+1}) = y_m$. Scott's embedding $\{i_{n\infty} : D_n \to D_\infty\}$ (Prop. 4.2) is built **coordinate by coordinate**: for fixed n and $x \in D_n$, the m-th component $i_{n\infty}(x)_m$ must be an element of D_m that fits into such a family.

- When $m \geq n$, the only sensible choice is to **climb** from x along the tower of inclusions $i_k = (P\,k).\text{incl}$ — the map Embeddings along the tower **`embeddingAlongTower`**.
- When $m < n$, compatibility with higher coordinates forces the value at m to be obtained by **descending** from x along retractions $j_k = (P\,k).\text{retr}$ — Projections along the tower **`projectionAlongTower`**.

`iComp` is short for Scott's $i_{n\infty}$ **component**: **`iComp D P n x m`** is the m-th coordinate $(i_{n\infty}(x))_m$ (the **`i`** matches Scott's embedding $i_{n\infty}$, **`Comp`** = component at index **`m`**). There is no single tower map $D_n \to D_m$ for all **`m`**: the recipe switches at **`n`** $\leq$ **`m`** vs **`n`** $\not\leq$ **`m`**, and **`iComp`** packages that case split once so **`embInf`** can build **`i_{n\infty}(x)`** as the compatible sequence **`m`** $\mapsto$ **`iComp D P n x m`**.

Definition (`iComp`). For **`n`**, **`x : D n`**, and **`m`**,

$$\texttt{iComp}\,D\,P\,n\,x\,m = \begin{cases} \texttt{embeddingAlongTower}\,D\,P\,h\,x & \text{if } n \le m, \\ \texttt{projectionAlongTower}\,D\,P\,h'\,x & \text{if } n \not\le m, \end{cases}$$

where **`h`** and **`h'`** are the canonical proofs produced from **`n ≤ m`** or **`¬ n ≤ m`**.

In the namespace **`iComp`** we have

Theorem (`of_le`). *If **`h : n ≤ m`**, then **`iComp D P n x m = embeddingAlongTower D P h x`**.*

Theorem (`of_ge`). *If **`¬ n ≤ m`**, then **`iComp D P n x m = projectionAlongTower D P (le_of_lt (not_le.mp h)) x`**.*

Theorem (`self`). *For all **`n`**, **`x : D n`**, **`iComp D P n x n = x`**.*

Proof. **`of_le`** and **`embeddingAlongTower.self`**. □ (This is the identity **`j_{\infty n} ∘ i_{n\infty} = \mathrm{id}`** at the coordinate level.)

Theorem (`compatible`). *For every **`n`**, **`x : D n`**, the sequence **`fun m => iComp D P n x m`** satisfies the inverse-limit compatibility laws.*

Proof. Case split on **`n ≤ m`**, **`n ≤ m + 1`**, and **`m + 1 ≤ n`**; the delicate middle case **`n = m + 1`** uses **`projectionAlongTower.succ`** and **`self`**, the general descent case uses **`projectionAlongTower.retr`**. □

Theorem (`incl_le`). *If **`y : InverseLimit D P`**, then for all **`n`**, **`m`**, **`iComp D P n (y.1 n) m ≤ y.1 m`**.*

Proof. If **`n ≤ m`**, **`embeddingAlongTower.le`**; if **`n ≰ m`**, the coordinate equals **`y.1 m`** by **`projectionAlongTower.compatible`**. □

Theorem (`preservesDirectedSup`). *For fixed **`n`**, **`m`**, **`fun x => iComp D P n x m`** preserves directed suprema.*

Proof. **`by_cases`** on **`n ≤ m`**, then rewrite to **`embeddingAlongTower`** or **`projectionAlongTower`**. □

Theorem (`monotone`). *For fixed **`n`**, **`m`**, **`fun x => iComp D P n x m`** is monotone.*

Proof. From **`preservesDirectedSup`** via **`preservesDirectedSup_monotone`**. □

Use. Embeddings into the limit packages ⟨**`iComp D P n x, iComp.compatible D P n x`**⟩ as **`embInf.fun D P n x : InverseLimit D P`** — Scott's $i_{n\infty}(x)$.

`iComp.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embeddingAlongTower
public import DomainTheory.ContinuousLattice.InverseLimits.projectionAlongTower
import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit
```

```
variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embeddingAlongTower D P h x
  else projectionAlongTower D P (le_of_lt (not_le.mp h)) x

namespace iComp

theorem of_le {n m : ℕ} (h : n ≤ m) (x : D n) :
    iComp D P n x m = embeddingAlongTower D P h x :=
  dite_eq_left h

theorem of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projectionAlongTower D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [of_le D P (le_refl n), embeddingAlongTower.self]

theorem incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [of_le D P h]; exact embeddingAlongTower.le D P y h
  · rw [of_ge D P h]
    exact le_of_eq (projectionAlongTower.compatible D P y (le_of_lt (not_le.mp h)))

theorem preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embeddingAlongTower D P h :=
      funext (fun x => of_le D P h x)
    rw [hfun]; exact embeddingAlongTower.preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) =
        projectionAlongTower D P (le_of_lt (not_le.mp h)) :=
      funext (fun x => of_ge D P h x)
    rw [hfun]; exact projectionAlongTower.preservesDirectedSup D P _

theorem monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (preservesDirectedSup D P n m)

theorem compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [of_le D P h1, of_le D P (Nat.le_succ_of_le h1),
      embeddingAlongTower.succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
      subst hn
      rw [of_le D P (le_refl (m + 1)), embeddingAlongTower.self, of_ge D P h1,
        projectionAlongTower.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x,
        projectionAlongTower.self]
    · rw [of_ge D P h2, of_ge D P h1,
        projectionAlongTower.retr D P (le_of_lt (not_le.mp h2)) x]
```

```
end iComp

end InverseLimit

end DomainTheory.ContinuousLattice
```

2.5.10 Embeddings into the limit ✓

For an inverse system of continuous lattices $\langle D_n, P_n \rangle$, write $D_\infty = \mathrm{InverseLimit}\, D\, P$ for the limit lattice of compatible sequences $(y_m)_{m \in \mathbb{N}}$. Scott's **inverse embedding** at level n is a map $\{i_{n\infty} : D_n \to D_\infty\}$ that sends $x \in D_n$ to the point of D_∞ whose coordinates are determined by climbing or descending the tower from x at index n.

Definition (`embInf`). For each $n \in \mathbb{N}$, **`embInf D P n`** is the **Scott map** $D_n \to D_\infty$ whose underlying function is **`embInf.fun D P n`** and whose Scott continuity is proved from directed-sup preservation on that function.

Definition (`embInf.fun`). For $n \in \mathbb{N}$ and $x \in D_n$, **`embInf.fun D P n x`** is the element of **`InverseLimit D P`** whose m-th coordinate is **`iComp D P n x m`**—equivalently, the compatible sequence $(i_{n\infty}(x)_m)_{m \in \mathbb{N}}$ in Scott's notation.

In the namespace **`embInf`** we have

Theorem (`coe`). *For all* ***`n`, `x : D n`****, the coordinate function of* ***`embInf.fun D P n x`*** *is* ***`iComp D P n x`****:*

$$(\mathrm{embInf.fun}\, D\, P\, n\, x)_m = \mathrm{iComp}\, D\, P\, n\, x\, m.$$

Proof. Definition of **`embInf.fun`** as a subtype of **`iComp`** with compatibility. □

Theorem (`monotone`). *For each* ***`n`****, the map* ***`embInf.fun D P n : D_n → D_\infty`*** *is monotone (with respect to the lattice orders on D_n and D_∞).*

Proof. Reduce to comparisons coordinatewise via **`Subtype.coe_le_coe`**; at each **`m`** apply **`iComp.monotone`**. □

Theorem (`preservesDirectedSup`). *For each* ***`n`****,* ***`embInf.fun D P n`*** *preserves directed suprema in D_n.*

Proof. For a directed set $S \subseteq D_n$, compare suprema in D_∞ using **`coe_sSup_of_directed`** on the image **`embInf.fun D P n " S`**, then rewrite each coordinate with **`iComp.preservesDirectedSup`**. □

Theorem (`succ`). *For all* ***`n`, `x : D_n`****,*

$$i_{(n+1)\infty}\,(i_n(x)) = i_{n\infty}(x),$$

formally **`embInf D P (n + 1) ((P n).incl x) = embInf D P n x`**.

Proof. Compare coordinates **`m`** by case analysis on **`n ≤ m`**, **`n + 1 ≤ m`**, and **`m < n`**; use the computation laws for **`iComp`**, **`embeddingAlongTower`**, and **`projectionAlongTower`**. □

Theorem (`le_succ`). *If* ***`y : InverseLimit D P`****, then for every* ***`n`****,*

$$i_{n\infty}(y_n) \le i_{(n+1)\infty}(y_{n+1}).$$

Proof. Rewrite with **`succ`** at **`x = y n`**, then apply **`monotone`** for index **`n + 1`** and **`(P n).incl_retr_le`** with compatibility **`(P n).retr (y (n + 1)) = y n`**. □

Theorem (`family_directed`). *If* **`y : InverseLimit D P`**, *the set* $\{\, i_{n\infty}(y_n) \mid n \in \mathbb{N} \,\}$ *is directed in* D_∞.

Proof. **`le_succ`** gives a monotone sequence in **`n`**; apply **`directedOn_range`** and **`monotone_nat_of_le_succ`**. □

2.5.11 Order on the inverse limit ✓

A point of the inverse limit $D_\infty = \mathrm{InverseLimit}\, D\, P$ is a compatible sequence $(x_m)_{m\in\mathbb{N}}$ with $x_m \in D_m$. The **product** order on all sequences (y_m) would declare $x \le y$ when $x_m \le y_m$ at every index; the limit inherits that comparison because D_∞ is a subtype of the product.

Definition (`inverseLimitLE`). For **`x`**, **`y : InverseLimit D P`**,

$$\texttt{inverseLimitLE}\; D\, P\, x\, y \iff \forall n,\; x_n \le y_n$$

(coordinatewise $\le$ in each stage lattice D_n; in Lean, **`x.1 n ≤ y.1 n`**).

In the namespace **`inverseLimitLE`** we have

Theorem (`refl`). *For all **`x`**, **`inverseLimitLE D P x x`.*

Proof. **`le_refl`** at each coordinate. □

Theorem (`trans`). *If **`inverseLimitLE D P x y`** and **`inverseLimitLE D P y z`**, then **`inverseLimitLE D P x z`.*

Proof. **`le_trans`** coordinatewise. □

Theorem (`antisymm`). *If **`inverseLimitLE D P x y`** and **`inverseLimitLE D P y x`**, then **`x = y`.*

Proof. **`Subtype.ext`** and **`funext`**: equal coordinates give **`le_antisymm`** at each **`n`**. □

Use. Proofs that a map into D_∞ is monotone (for example **`embInf.monotone`**) rewrite with **`Subtype.coe_le_coe`** and compare coordinates in D_m at each **`m`**.

`inverseLimitLE.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise $\le$ on compatible sequences: `x ≤ y` iff `x.1 n ≤ y.1 n` for
    every `n`. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
```

```
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)

theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)

theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

2.5.12 Strict order on the inverse limit ✓

On the inverse limit $D_\infty = \mathrm{InverseLimit}\, D\, P$, the coordinatewise order Order on the inverse limit **`inverseLimitLE`** is the natural $\leq$. The **strict** comparison is the usual one for a partial order: below in $\leq$ but not above.

Definition (`inverseLimitLT`). For **`x`**, **`y : InverseLimit D P`**,

$$\texttt{inverseLimitLT}\, D\, P\, x\, y \iff \texttt{inverseLimitLE}\, D\, P\, x\, y \wedge \neg\, \texttt{inverseLimitLE}\, D\, P\, y\, x.$$

Equivalently: $x \leq y$ coordinatewise and it is **not** the case that $y \leq x$ coordinatewise (so some coordinate is strictly below, or the sequences differ).

In the namespace **`inverseLimitLT`** we have

Theorem (`iff_LE_not_GE`). *For all* ***`x`****,* ***`y`****,*

$$\texttt{inverseLimitLT}\, D\, P\, x\, y \iff \texttt{inverseLimitLE}\, D\, P\, x\, y \wedge \neg\, \texttt{inverseLimitLE}\, D\, P\, y\, x.$$

Proof. Definition of **`inverseLimitLT`**. □

`inverseLimitLT.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order: `inverseLimitLE` and not the reverse. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x

namespace inverseLimitLT

theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

2.5.13 The limit lattice ✓

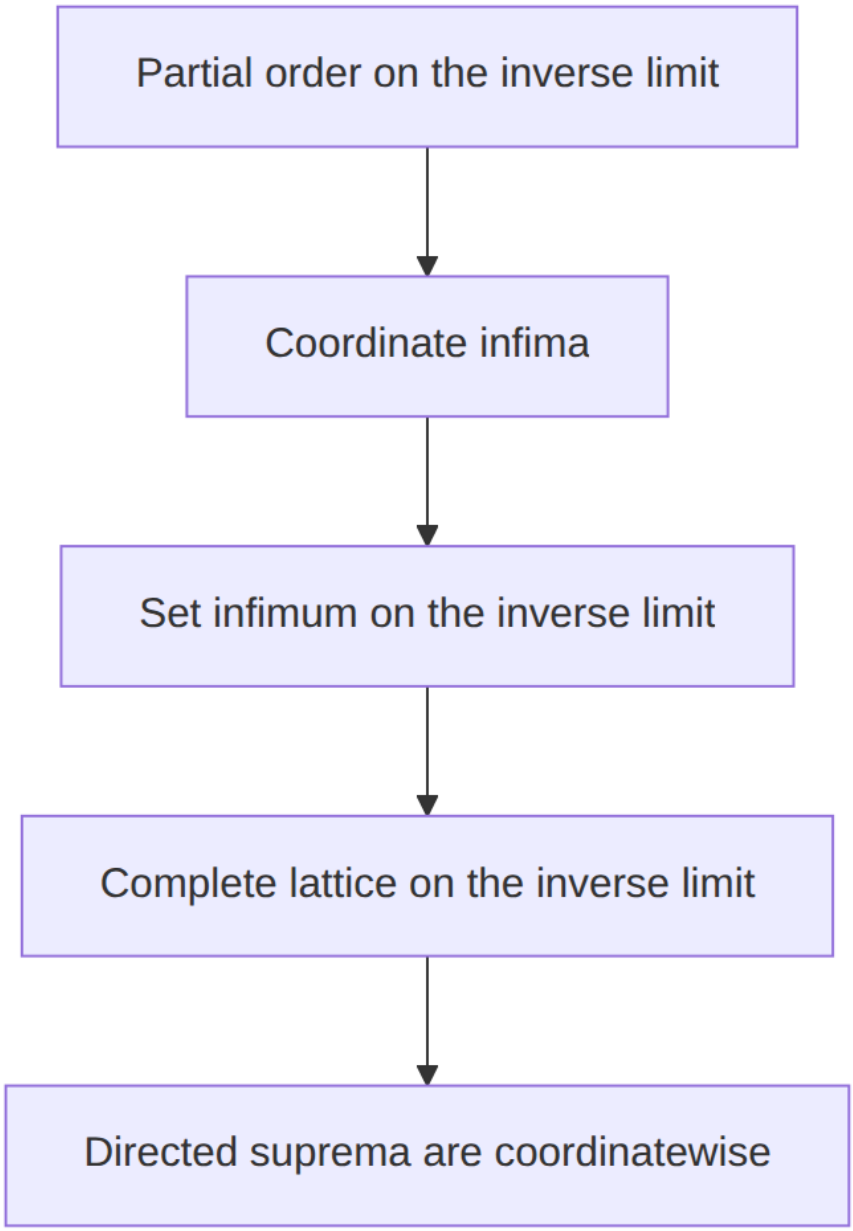


Figure 15: The limit lattice ✓.

Partial order on the inverse limit ✓ Write $D_\infty = \mathrm{InverseLimit}\, D\, P$. The coordinatewise relations Order on the inverse limit **`inverseLimitLE`** and Strict order on the inverse limit **`inverseLimitLT`** are the order on D_∞.

Definition (`inverseLimitPartialOrder`). **`inverseLimitPartialOrder D P`** is the

`PartialOrder` instance on **`InverseLimit D P`** with

- **`le := inverseLimitLE D P`**
- **`lt := inverseLimitLT D P`**
- **`le_refl`**, **`le_trans`**, **`le_antisymm`** from **`inverseLimitLE.refl`**, **`trans`**, **`antisymm`**
- **`lt_iff_le_not_ge`** from **`inverseLimitLT.iff_LE_not_GE`**

inverseLimitPartialOrder.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on `InverseLimit D P` from `inverseLimitLE` / `inverseLimitLT
    `. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

Coordinate infima ✓ For $S \subseteq D_\infty = \mathrm{InverseLimit}\, D\, P$, a natural candidate for an infimum in D_∞ is built **coordinate by coordinate**: at index n, take the infimum in D_n of the n-th coordinates of points in S.

Definition (`inverseLimitSInfCoe`). For **`S : Set (InverseLimit D P)`**,

$$(\mathtt{inverseLimitSInfCoe}\, D\, P\, S)_n = \inf\{\, x_n \mid x \in S \,\}$$

in D_n (Lean: **`sInf ((fun x => x.1 n) '' S)`**).

In the namespace **`inverseLimitSInfCoe`** we have

Theorem (apply). *For all **S**, **n**, the **n-th coordinate of** `inverseLimitSInfCoe D P S`** is that stage infimum:*

$$\texttt{inverseLimitSInfCoe}\, D\, P\, S\, n = \inf\{\, x_n \mid x \in S \,\}.$$

Proof. Definition of **`inverseLimitSInfCoe`**. □

`inverseLimitSInfCoe.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

/-- At index `n`, the infimum in `D n` of the `n`-th coordinates of `S`. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

namespace inverseLimitSInfCoe

@[simp] theorem apply (S : Set (InverseLimit D P)) (n : ℕ) :
    inverseLimitSInfCoe D P S n = sInf ((fun x : InverseLimit D P => x.1 n) '' S) :=
  rfl

end inverseLimitSInfCoe

end InverseLimit

end DomainTheory.ContinuousLattice
```

Set infimum on the inverse limit ✓ For $S \subseteq D_\infty = \text{InverseLimit}\, D\, P$, Coordinate infima **`inverseLimitSInfCoe D P S`** is the coordinatewise infimum in each D_n. The **set infimum `sInf S`** in D_∞ is that tuple, once it is a compatible sequence.

Definition (`instInfSet`). **`instInfSet D P`** is the **`InfSet`** instance

$$\texttt{sInf}\, S = \langle \texttt{inverseLimitSInfCoe}\, D\, P\, S,\ \text{compatibility proof} \rangle.$$

In the namespace **`instInfSet`** we have

Theorem (`retr_sInf`). *For **`n`**, **`A : Set (D (n + 1))`**,*

$$(P_n).\mathrm{retr}\,(\inf A) = \inf\big((P_n).\mathrm{retr}\,'' A\big).$$

Proof. **tower_step_galoisConnection** and **sInf_image**. □

Theorem (compatible_sInf). *For every* ***S : Set (InverseLimit D P)***, ***inverseLimitSInfCoe D P S*** *satisfies* ***Compatible***.

Proof. At index **n**, rewrite with **retr_sInf** and compatibility **x.2 n** for **x ∈ S**. □

Theorem (coe_sInf). *For all* ***S***, *the underlying coordinates of* ***sInf S*** *are* ***inverseLimitSInfCoe D P S:***

$$((\mathtt{sInf}\,S) : \forall n,\, D_n) = \mathtt{inverseLimitSInfCoe}\,D\,P\,S.$$

Proof. Definition of **instInfSet**. □

Theorem (isGLB_sInf'). *For all* ***S***, ***sInf S*** *is the greatest lower bound of* ***S*** *in the partial order on* D_∞.

Proof. Coordinatewise **sInf_le** and **le_sInf**. □

instInfSet.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitPartialOrder
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitSInfCoe
import DomainTheory.ContinuousLattice.InverseLimits.tower_step_galoisConnection

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

namespace instInfSet

theorem retr_sInf (n : ℕ) (A : Set (D (n + 1))) :
    (P n).retr (sInf A) = sInf ((P n).retr '' A) := by
  rw [(tower_step_galoisConnection D P n).u_sInf, sInf_image]

theorem compatible_sInf (S : Set (InverseLimit D P)) :
    Compatible D P (inverseLimitSInfCoe D P S) := by
  intro n
  rw [inverseLimitSInfCoe, inverseLimitSInfCoe, retr_sInf]
  congr 1
  rw [Set.image_image]
  exact Set.image_congr (by rintro x hx; exact x.2 n)

end instInfSet
```

```
noncomputable instance instInfSet : InfSet (InverseLimit D P) :=
  ⟨fun S => ⟨inverseLimitSInfCoe D P S, instInfSet.compatible_sInf D P S⟩⟩

namespace instInfSet

theorem coe_sInf (S : Set (InverseLimit D P)) :
    ((sInf S : InverseLimit D P) : ∀ n, D n) = inverseLimitSInfCoe D P S := rfl

theorem isGLB_sInf' (S : Set (InverseLimit D P)) :
    @IsGLB (InverseLimit D P)
      (inverseLimitPartialOrder D P).toPreorder.toLE S (sInf S) := by
  constructor
  · intro x hx n
    exact sInf_le (Set.mem_image_of_mem _ hx)
  · intro b hb n
    refine le_sInf ?_
    rintro _ ⟨x, hx, rfl⟩
    exact hb hx n

end instInfSet

end InverseLimit

end DomainTheory.ContinuousLattice
```

Complete lattice on the inverse limit ✓ Write $D_\infty = \mathrm{InverseLimit}\, D\, P$ with the coordinatewise order from **`inverseLimitLE`**. A **complete lattice** is a partially ordered set in which **every** subset S has a **greatest lower bound** (infimum, written $\inf S$ or $\bigwedge S$) and a **least upper bound** (supremum, $\sup S$ or $\bigvee S$).

What `sInf`** is.** For $S \subseteq D_\infty$, **`sInf S`** is Lean's notation for that **set infimum** in D_∞: the **largest** element $y \in D_\infty$ such that $y \le x$ for every $x \in S$. Concretely it is the compatible sequence whose n-th coordinate is $\inf\{\, x_n : x \in S \,\}$ in the stage lattice D_n (see **`instInfSet`** and **`instInfSet.isGLB_sInf'`**).

Theorem (`instCompleteLattice`). *D_∞ is a complete lattice.*

Proof. We already have a partial order on D_∞ and, for each $S \subseteq D_\infty$, an element **`sInf S`** that is the **greatest lower bound** of S (**`instInfSet.isGLB_sInf'`**). A standard order-theoretic fact is then enough: **if every subset has an infimum, the poset is a complete lattice**. Define suprema from infima by the usual dual recipe— for $S \subseteq D_\infty$, take the infimum of the set of **upper bounds** of S:

$$\sup S \;=\; \inf\{\, u \in D_\infty \mid \forall x \in S,\ x \le u \,\}.$$

That infimum exists by the construction above, and it is the least upper bound of S. Finite meets and joins agree with this inf / sup. Lean packages this construction as **`instCompleteLattice`**, applying Mathlib's **`completeLatticeOfInf`** to the partial order and the **`isGLB_sInf'`** proof. □

Definition (`instCompleteLattice`). **`instCompleteLattice D P`** is the **`CompleteLattice`** instance on **`InverseLimit D P`** obtained in this way (one declaration in **`instCompleteLattice.lean`**).

`instCompleteLattice.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instInfSet

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** Complete lattice structure on compatible sequences. -/
noncomputable instance instCompleteLattice : CompleteLattice (InverseLimit D P) :=
  @completeLatticeOfInf (InverseLimit D P) (inverseLimitPartialOrder D P)
    inferInstance (instInfSet.isGLB_sInf' D P)

end InverseLimit

end DomainTheory.ContinuousLattice
```

Directed suprema are coordinatewise ✓ Write $D_\infty = \mathrm{InverseLimit}\ D\ P$ with the coordinatewise order. D_∞ is a **complete lattice**, so every subset $S \subseteq D_\infty$ has a **least upper bound** (supremum). Lean writes that set supremum as **`sSup S`**; in ordinary notation it is $\sup S$ or $\bigvee S$— the **smallest** $u \in D_\infty$ with $x \le u$ for all $x \in S$.

A set $S \subseteq D_\infty$ is **directed** (upward) when for every $x, y \in S$ there exists $z \in S$ with $x \le z$ and $y \le z$. Each stage lattice D_n is a complete lattice, so for nonempty S the coordinate set $\{\, x_n \mid x \in S \,\} \subseteq D_n$ has a supremum there; when S is directed, that coordinatewise family is directed in D_n as well.

Theorem (`coe_sSup_of_directed`). *Suppose $S \subseteq D_\infty$ is **nonempty** and **directed**. For every $n \in \mathbb{N}$, the n-th coordinate of $\sup S$ equals the supremum taken in D_n:*

$$\big(\sup S\big)_n \;=\; \sup\{\, x_n \mid x \in S \,\}.$$

Proof. Define $f(n) \in D_n$ by $f(n) = \sup\{\, x_n \mid x \in S \,\}$ for each $n \in \mathbb{N}$.

Compatibility. A point of D_∞ must satisfy $j_n(f(n+1)) = f(n)$ for each projection $j_n = (P_n).\mathrm{retr}$. Fix n. The set $\{\, x_{n+1} \mid x \in S \,\}$ is a directed nonempty subset of D_{n+1}, so $(P_n).\mathrm{retr}$ preserves its supremum. Rewriting that preservation with the compatibility laws $j_n(x_{n+1}) = x_n$ for $x \in S$ shows $j_n(f(n+1)) = f(n)$.

Least upper bound. Let $p \in D_\infty$ be the compatible sequence f. Every $x \in S$ has $x_n \le f(n)$ for each n, so $x \le p$ in the coordinatewise order. If $b \in D_\infty$ is an upper bound of S, then for each n and each $x \in S$, $x_n \le b_n$, hence $f(n) \le b_n$ and $p \le b$. Thus $p = \sup S$.

Uniqueness of least upper bounds identifies p with **`sSup S`**. □

Use. When a proof compares $\sup$ in D_∞ with data in some D_n, it rewrites $\sup S$ with this theorem so the n-th coordinate is a supremum inside D_n (for example in **`embInf.preservesDirectedSup`**).

coe_sSup_of_directed.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

/-- Directed suprema in `InverseLimit` agree with coordinatewise suprema in the product
    . -/
theorem coe_sSup_of_directed (S : Set (InverseLimit D P)) (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    ((sSup S : InverseLimit D P) : ∀ n, D n) = sSup (Subtype.val '' S) := by
  have hcompat : Compatible D P (sSup (Subtype.val '' S)) := by
    intro n
    rw [sSup_apply_eq_sSup_image, sSup_apply_eq_sSup_image]
    set A : Set (D (n + 1)) := (fun f : ∀ m, D m => f (n + 1)) '' (Subtype.val '' S)
    with hA
    have hAne : A.Nonempty := (hSne.image _).image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨_, ⟨x, hxS, rfl⟩, rfl⟩ _ ⟨_, ⟨y, hyS, rfl⟩, rfl⟩
      obtain ⟨z, hzS, hxz, hyz⟩ := hSdir x hxS y hyS
      exact ⟨z.1 (n + 1), ⟨z.1, ⟨z, hzS, rfl⟩, rfl⟩, hxz (n + 1), hyz (n + 1)⟩
    rw [(P n).retr.preservesDirectedSup_coe A hAne hAdir]
    congr 1
    rw [hA, Set.image_image]
    exact Set.image_congr (by rintro _ ⟨x, _, rfl⟩; exact x.2 n)
  set p : InverseLimit D P := ⟨sSup (Subtype.val '' S), hcompat⟩ with hp
  have hlub : IsLUB S p := by
    constructor
    · intro x hx
      refine Subtype.coe_le_coe.mp ?_
      exact le_sSup (Set.mem_image_of_mem _ hx)
    · intro b hb
      refine Subtype.coe_le_coe.mp ?_
      refine sSup_le ?_
      rintro _ ⟨x, hx, rfl⟩
      exact Subtype.coe_le_coe.mpr (hb hx)
  rw [(isLUB_sSup S).unique hlub]

end InverseLimit

end DomainTheory.ContinuousLattice
```

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

def embInf.fun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp.compatible
    D P n x⟩

namespace embInf

@[simp] theorem coe (n : ℕ) (x : D n) : (embInf.fun D P n x).1 = iComp D P n x := rfl

theorem monotone (n : ℕ) : Monotone (embInf.fun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
  exact iComp.monotone D P n m hxx

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInf.fun D P n) := by
  intro S hS hSdir
  have hTne : (embInf.fun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (· ≤ ·) (embInf.fun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInf.fun D P n c, Set.mem_image_of_mem _ hc,
      embInf.monotone D P n hac, embInf.monotone D P n hbc⟩
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInf.fun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInf.fun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInf.coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp.preservesDirectedSup D P n m hS hSdir

end embInf

noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInf.fun D P n, PreservesDirectedSup.continuous_of (embInf.preservesDirectedSup D
```

```
    P n)⟩

namespace embInf

theorem succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp.of_le D P h2, iComp.of_le D P h1, embeddingAlongTower.succ_left D P h1
    h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp.self, iComp.of_ge D P h2,
        projectionAlongTower.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).
    incl x),
        projectionAlongTower.self, (P m).retr_incl]
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp.of_ge D P h2, iComp.of_ge D P h1,
      projectionAlongTower.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)
    ) ((P n).incl x),
      (P n).retr_incl]

theorem le_succ (x : InverseLimit D P) (n : ℕ) :
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← succ D P n (x.1 n)]
  exact embInf.monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)

theorem family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (le_succ D P x)).directed_le

end embInf

end InverseLimit

end DomainTheory.ContinuousLattice
```

limitProjectionToLevel.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set
```

```
universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

def limitProjectionToLevel.fun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace limitProjectionToLevel

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (limitProjectionToLevel.
    fun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end limitProjectionToLevel

noncomputable def limitProjectionToLevel (n : ℕ) : ScottMap (InverseLimit D P) (D n)
    :=
  ⟨limitProjectionToLevel.fun D P n,
    PreservesDirectedSup.continuous_of (limitProjectionToLevel.preservesDirectedSup D
    P n)⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf

import DomainTheory.ContinuousLattice.InverseLimits.limitProjectionToLevel
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))
```

```
variable {D' : Type*} [CompleteLattice D']

def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

namespace coconeInf

theorem apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl

theorem climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
    ∀ {m : ℕ} (h : n ≤ m), f m (embeddingAlongTower D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embeddingAlongTower.self]
  | succ k hk ih =>
      rw [embeddingAlongTower.succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

theorem descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projectionAlongTower D P h x) ≤ f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      exact le_of_eq (congrArg (f m) (projectionAlongTower.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projectionAlongTower.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projectionAlongTower D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih
     ((P k).retr x)
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

theorem comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp.of_le D P h]; exact le_of_eq (climb D P f hf x h)
    · rw [iComp.of_ge D P h]; exact descend D P f hf (le_of_lt (not_le.mp h)) x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp.self D P n x]

theorem preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
```

```
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      (limitProjectionToLevel.preservesDirectedSup D P n) hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', apply]
  rw [iSup_comm]

end coconeInf

end InverseLimit

end DomainTheory.ContinuousLattice
```

2.5.14 Topology of the inverse limit ✓

Write $D_\infty = \mathrm{InverseLimit}\, D\, P$. Scott topologizes each stage lattice D_n; on **all** sequences $(x_n)_{n\in\mathbb{N}}$ the product topology is the Tychonoff product of those Scott topologies. The **inverse limit topology** on D_∞ is the subspace topology inherited from that product: open sets in D_∞ are exactly the intersections of product opens with the set of **compatible** sequences.

Definition (`inverseLimitTopology`). `inverseLimitTopology D P` is

$$\mathcal{T}_\infty \;=\; \Big\{ U \cap D_\infty \;\Big|\; U \text{ open in } \prod_{n\in\mathbb{N}} (D_n,\ \text{Scott}) \Big\},$$

implemented as `TopologicalSpace.induced Subtype.val` from the Pi topology with `scottTopologicalSpace` on each factor.

In the namespace `inverseLimit` we have

Theorem (`coordinate_continuous`). *For each* `n`*, the coordinate map* `x ↦ x_n` *is continuous from* `inverseLimitTopology` *to Scott on* D_n.

Proof. Open sets in the product are detected coordinatewise; preimages factor through `continuous_apply`. □

Theorem (`t0Space`). $(D_\infty, \mathcal{T}_\infty)$ *is* T_0.

Proof. `Subtype.val` is an embedding into a product of T_0 spaces. □

Theorem (`specializationPreorder_iff`). *For* `x, y : InverseLimit D P`*, specialization preorder in* `inverseLimitTopology` *agrees with coordinatewise* $\le$*:*

$$x \preceq y \iff x \le y.$$

Proof. Compare `specializes` along the embedding into the product; at each index use Scott specialization on D_n. □

Theorem (`isInjectiveSpace`). *If each* `D n` *is a continuous lattice, then* $(D_\infty, \mathcal{T}_\infty)$ *is an* ***injective space*** *(Scott's §1 notion).*

Proof. Given a continuous `f : X → D_\infty` from a compact Hausdorff `X`, read coordinates `f_n`, extend each by `scottExtend`, check compatibility with `lemma_3_9`, assemble the extension map into D_∞, and compare on coordinates with `scottExtend_eq_of_continuous`. □

Theorem (`scottTopology_eq_induced_pi`). *When each stage is a continuous lattice, the* ***Scott topology*** ***`scottTopologicalSpace`*** *on* D_∞ ***equals*** ***`inverseLimitTopology`***.

Proof. **`isInjectiveSpace`** and **`specializationPreorder_iff`** pin down the Scott topology on an injective space; apply **`continuousLattice_scottTopology_of_injective`**. □

Theorem (`eq_iSup`). *Every* ***`x : InverseLimit D P`*** *is the supremum of its approximants along embeddings:*

$$x = \bigvee_{n \in \mathbb{N}} i_{n\infty}(x_n).$$

Proof. The family $\{\, i_{n\infty}(x_n) \mid n \in \mathbb{N} \,\}$ is directed; compare coordinates using **`coe_sSup_of_directed`**, **`embInf.family_directed`**, and **`iComp`**. □

In the namespace **`inverseLimitTopology`** we have

Theorem (`limitProjectionToLevel`). *Each* ***`limitProjectionToLevel n`*** *($j_{\infty n}$) is continuous.*

Proof. Same as **`coordinate_continuous`**. □

Theorem (`embInf`). *If stages are continuous lattices, each* ***`embInf n`*** *($i_{n\infty}$) is continuous into* ***`inverseLimitTopology`***.

Proof. Rewrite with **`scottTopology_eq_induced_pi`**; use the **`ScottMap`** instance on **`embInf`**. □

Theorem (`coconeInf`). *Under the same hypothesis,* ***`coconeInf f`*** *is continuous from* ***`inverseLimitTopology`***

Proof. Again identify Scott with **`inverseLimitTopology`**, then **`PreservesDirectedSup.continuous_of`** and **`coconeInf.preservesDirectedSup`**. □

`inverseLimitTopology.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.limitProjectionToLevel
import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.InverseLimits.coconeInf
public import DomainTheory.ContinuousLattice.Specialization.SpecializationPreorder
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
import DomainTheory.ContinuousLattice.Constructions.scottExtend_eq_of_continuous
import DomainTheory.ContinuousLattice.ContinuousLattices.
    continuousLattice_scottTopology_of_injective
import DomainTheory.ContinuousLattice.ContinuousLattices.lemma_3_9

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set
universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))
```

```
@[reducible] noncomputable def inverseLimitTopology : TopologicalSpace (InverseLimit D
    P) :=
  TopologicalSpace.induced (Subtype.val : InverseLimit D P → ∀ n, D n)
    (@Pi.topologicalSpace ℕ D (fun _ => scottTopologicalSpace))

namespace inverseLimit

theorem coordinate_continuous (n : ℕ) :
    @Continuous (InverseLimit D P) (D n) (inverseLimitTopology D P)
      scottTopologicalSpace (fun x => x.1 n) := by
  let : ∀ n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  rw [continuous_def]
  intro U hU
  rw [isOpen_induced_iff]
  exact ⟨{x : ∀ n, D n | x n ∈ U},
    (@continuous_apply ℕ D (fun _ => scottTopologicalSpace) n).isOpen_preimage U hU,
    rfl⟩

theorem t0Space :
    @T0Space (InverseLimit D P) (inverseLimitTopology D P) := by
  let : ∀ n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  let : ∀ n, T0Space (D n) := fun _ => scottTopologicalSpace.t0Space
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  have hval : Topology.IsEmbedding (Subtype.val : InverseLimit D P → ∀ n, D n) :=
    ⟨⟨rfl⟩, Subtype.val_injective⟩
  exact hval.t0Space

theorem specializationPreorder_iff (x y : InverseLimit D P) :
    @SpecializationPreorder (InverseLimit D P) (inverseLimitTopology D P) x y ↔ x ≤ y
     := by
  let stageOrder : ∀ n, Preorder (D n) := fun _ => inferInstance
  let limitOrder : Preorder (InverseLimit D P) := inferInstance
  let : ∀ n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : ∀ n, Preorder (D n) := stageOrder
  let : Preorder (InverseLimit D P) := limitOrder
  have hval : Topology.IsEmbedding (Subtype.val : InverseLimit D P → ∀ n, D n) :=
    ⟨⟨rfl⟩, Subtype.val_injective⟩
  rw [SpecializationPreorder.iff_specializes, ← hval.isInducing.specializes_iff,
    specializes_pi]
  constructor
  · intro h n
    rw [← SpecializationPreorder.scott_iff, SpecializationPreorder.iff_specializes]
    exact h n
  · intro h n
    rw [← SpecializationPreorder.iff_specializes, SpecializationPreorder.scott_iff]
    exact h n

theorem isInjectiveSpace (hD : ∀ n, ContinuousLattice (D n)) :
    @InjectiveSpace.{u, u} (InverseLimit D P) (inverseLimitTopology D P) := by
  let stageOrder : ∀ n, Preorder (D n) := fun _ => inferInstance
  let limitOrder : Preorder (InverseLimit D P) := inferInstance
  let : ∀ n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : ∀ n, Preorder (D n) := stageOrder
  let : Preorder (InverseLimit D P) := limitOrder
```

```
  constructor
  intro X Y _ _ e he f
  let fcoord (n : ℕ) (x : X) : D n := (f x).1 n
  have hfcoord (n : ℕ) : @Continuous X (D n) _ scottTopologicalSpace (fcoord n) :=
    (coordinate_continuous D P n).comp f.continuous
  let fbar (n : ℕ) : Y → D n := scottExtend e (fcoord n)
  have hfbar_cont (n : ℕ) :
      @Continuous Y (D n) _ scottTopologicalSpace (fbar n) :=
    scottExtend_continuous (hD n) e (fcoord n)
  have hfbar_compat (y : Y) (n : ℕ) :
      (P n).retr (fbar (n + 1) y) = fbar n y := by
    symm
    exact lemma_3_9 (hD n) (hD (n + 1)) (P n) e he
      (hfcoord n) (hfcoord (n + 1)) (fun x => (f x).2 n |>.symm) y
  let fbarInf (y : Y) : InverseLimit D P :=
    ⟨fun n => fbar n y, hfbar_compat y⟩
  have hfbarInf_cont :
      @Continuous Y (InverseLimit D P) _ (inverseLimitTopology D P) fbarInf := by
    change @Continuous Y (InverseLimit D P) _
      (TopologicalSpace.induced (Subtype.val : InverseLimit D P → ∀ n, D n)
        (@Pi.topologicalSpace ℕ D (fun _ => scottTopologicalSpace))) fbarInf
    rw [continuous_induced_rng]
    exact continuous_pi hfbar_cont
  refine ⟨⟨fbarInf, hfbarInf_cont⟩, fun x => ?_⟩
  apply Subtype.ext
  funext n
  exact scottExtend_eq_of_continuous (hD n) e he (fcoord n) (hfcoord n) x

theorem scottTopology_eq_induced_pi (hD : ∀ n, ContinuousLattice (D n)) :
    (scottTopologicalSpace : TopologicalSpace (InverseLimit D P)) =
      inverseLimitTopology D P := by
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : T0Space (InverseLimit D P) := t0Space D P
  exact (continuousLattice_scottTopology_of_injective (isInjectiveSpace D P hD)
    (specializationPreorder_iff D P)).2

theorem eq_iSup (x : InverseLimit D P) :
    x = ⋃ n, embInf D P n (x.1 n) := by
  have hdir := embInf.family_directed D P x
  have hne : (Set.range (fun n => embInf D P n (x.1 n))).Nonempty := Set.
    range_nonempty _
  refine le_antisymm ?_ ?_
  · rw [← Subtype.coe_le_coe]
    have hcoe : ((⋃ n, embInf D P n (x.1 n) : InverseLimit D P)).1
        = sSup (Subtype.val '' Set.range (fun n => embInf D P n (x.1 n))) := by
      rw [← sSup_range]
      exact coe_sSup_of_directed D P _ hne hdir
    rw [hcoe]
    intro m
    have hmem : (embInf D P m (x.1 m)).1
        ∈ Subtype.val '' Set.range (fun n => embInf D P n (x.1 n)) :=
      ⟨embInf D P m (x.1 m), ⟨m, rfl⟩, rfl⟩
    have hle := (le_sSup hmem) m
    rwa [show (embInf D P m (x.1 m)).1 m = x.1 m from iComp.self D P m (x.1 m)] at hle
  · exact iSup_le fun n => by
      rw [← Subtype.coe_le_coe]; intro m; exact iComp.incl_le D P n x m
```

```
end inverseLimit
namespace inverseLimitTopology

theorem limitProjectionToLevel (n : ℕ) :
    @Continuous (InverseLimit D P) (D n) (inverseLimitTopology D P)
      scottTopologicalSpace (DomainTheory.ContinuousLattice.limitProjectionToLevel D P
     n) :=
  inverseLimit.coordinate_continuous D P n

theorem embInf (hD : ∀ n, ContinuousLattice (D n)) (n : ℕ) :
    @Continuous (D n) (InverseLimit D P) scottTopologicalSpace
      (inverseLimitTopology D P) (DomainTheory.ContinuousLattice.embInf D P n) := by
  rw [← inverseLimit.scottTopology_eq_induced_pi D P hD]
  exact (DomainTheory.ContinuousLattice.embInf D P n).continuous

theorem coconeInf (hD : ∀ n, ContinuousLattice (D n))
    {D' : Type*} [CompleteLattice D']
    (f : ∀ n, ScottMap (D n) D') :
    @Continuous (InverseLimit D P) D' (inverseLimitTopology D P)
      scottTopologicalSpace (DomainTheory.ContinuousLattice.coconeInf D P f) := by
  rw [← inverseLimit.scottTopology_eq_induced_pi D P hD]
  exact PreservesDirectedSup.continuous_of
    (DomainTheory.ContinuousLattice.coconeInf.preservesDirectedSup D P f)

end inverseLimitTopology

end InverseLimit

end DomainTheory.ContinuousLattice
```

2.5.15 Proposition 4.1 ✓

4.1 Proposition. Let $\langle D_n, j_n\rangle_{n=0}^{\infty}$ be an inverse system of continuous lattices where each $j_n : D_{n+1} \to D_n$ is a projection. Then the inverse limit space D_∞ is also a continuous lattice.

Proof: We need only show that D_∞ as a T_0-space is injective. So suppose $\{f_\infty : X \to D_\infty\}$ is given and $X \subseteq Y$. Define $f_n : X \to D_n$ by $f_n = j_{\infty n} \circ f_\infty$. Let $\bar{f} * n : Y \to D_n$ be the maximal extension of f_n according to 3.8. Now we can see the point of Lemma 3.9: by this construction we guarantee that $\bar{f}n = j_n \circ \bar{f}n + 1$. Hence the required $\bar{f} * \infty : Y \to D_\infty$ exists. □

I do not know at the time of writing whether this theorem on inverse limits of continuous lattices extends to sequences where, say, the j_n are only retractions. Fortunately, sufficiently many projections exist to make this construction useful. Note that by reference to the product space construction of D_∞, its lattice ordering is given simply by the relation:

$$x \sqsubseteq y \quad \text{iff} \quad x_n \sqsubseteq y_n \text{ for all } n.$$

`proposition_4_1.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
```

```
import DomainTheory.ContinuousLattice.ContinuousLattices.
    continuousLattice_scottTopology_of_injective
import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

theorem proposition_4_1 (hD : ∀ n, ContinuousLattice (D n)) :
    ContinuousLattice (InverseLimit D P) := by
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : T0Space (InverseLimit D P) := inverseLimit.t0Space D P
  exact (continuousLattice_scottTopology_of_injective (inverseLimit.isInjectiveSpace D
     P hD)
    (inverseLimit.specializationPreorder_iff D P)).1

end InverseLimit
end DomainTheory.ContinuousLattice
```

2.5.16 Proposition 4.2 ✓

4.2 Proposition. Let $\langle D_n, j_n\rangle_{n=0}^{\infty}$ and D_∞ be as in 4.1. Then the maps $j_{\infty\{n} : D_\infty \to D_n\}$ are projections.

Proof: The projections $j_n : D_{n+1} \to D_n$, as we know, have their uniquely determined inverses $i_n : D_n \to D_{n+1}$. We can define $\{i_{n\infty} : D_n \to D_\infty\}$ by the equation:

$$i_{n\infty}(x) = \langle y_m\rangle_{m=0}^{\infty}$$

where

$$y_m = \begin{cases} j_m(y_{m+1}) & \text{if } m < n, \\ x & \text{if } m = n, \\ i_m(y_{m-1}) & \text{if } m > n. \end{cases}$$

The proof that $i_{n\infty}$ and $j_{\infty n}$ form a projection is now an easy computation. □

Definition (`limitLevelProjection`). For **`n : ℕ`**, **`limitLevelProjection D P n`** is the **continuous lattice projection** from D_n to $D_\infty = \mathrm{InverseLimit}\ D\ P$ with

- **`incl`** $= i_{n\infty} =$ **`embInf D P n`**
- **`retr`** $= j_{\infty n} =$ **`limitProjectionToLevel D P n`**

The retraction law **`retr_incl`** is **`iComp.self`** at index **`n`**; **`incl_retr_le`** is coordinatewise **`iComp.incl_le`**.

Theorem (`proposition_4_2`). *For every* ***`n`****, there is a continuous lattice projection from D_n to D_∞ whose retraction is $j_{\infty n}$ and whose inclusion is $i_{n\infty}$—equivalently,* ***`Nonempty (ContinuousLatticeProjection (D n) (InverseLimit D P))`****, witnessed by* ***`limitLevelProjection D P n`****.*

Proof. **`limitLevelProjection D P n`**. □

One should note also the recursion equation:

$$i_{n\infty} = i_{(n+1)\infty} \circ i_n.$$

These maps also make it possible to state this useful equation:

$$x = \bigsqcup_{n=0}^{\infty} i_{n\infty}(x_n),$$

where $x \in D_\infty$. It is easy to check that this is a monotone lub, and so we can say each $x \in D_\infty$ is the limit of its projections x_n. In fact, from what we know about projections, x_n is the best possible approximation to x in the space D_n.

`proposition_4_2.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.limitProjectionToLevel

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott?s $\{ i_{n\infty} : D_n \to D_\infty \}$ and $j_{\infty \{ n} : D_\infty \to
    D_n \}$ as one
`ContinuousLatticeProjection`. -/
noncomputable def limitLevelProjection (n : ℕ) :
    ContinuousLatticeProjection (D n) (InverseLimit D P) where
  incl := embInf D P n
  retr := limitProjectionToLevel D P n
  retr_incl x := iComp.self D P n x
  incl_retr_le y := by
    rw [← Subtype.coe_le_coe]
    intro m
    exact iComp.incl_le D P n y m

/-- **Scott 1972, Proposition 4.2.** Each $j_{\infty n}$ is a projection (with inverse
    $i_{n\infty}$). -/
theorem proposition_4_2 (n : ℕ) :
```

```
    Nonempty (ContinuousLatticeProjection (D n) (InverseLimit D P)) :=
  ⟨limitLevelProjection D P n⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

2.5.17 Corollary 4.3 ✓

4.3 Corollary. Let the spaces be as in 4.1 and 4.2. Let D' be any complete lattice and suppose continuous functions $f_n : D_n \to D'$ are given so that $f_n = f_{n+1} \circ i_n$. Then we can define $\{f_\infty : D_\infty \to D'\}$ by the equation:

$$f_\infty(x) = \bigsqcup_{n=0}^{\infty} f_n(x_n)$$

for $x \in D_\infty$, and we have $f_n = f_\infty \circ i_{n\infty}$. □

The import of this last result is that within the category of complete lattices, the space D_∞ is not only the inverse limit of the D_n, but it is also the *direct* limit. (One system of spaces here uses the j_n as connecting maps, the other the i_n.) This is the algebraic result that lies at the heart of our main result about inverse limits of function spaces.

Turning to function spaces, let $D = D_0$ be a given continuous lattice. As we have seen in 3.13, there are many ways of making D_0 a projection of $D_1 = [D_0 \to D_0]$. Choose one such given by a pair i_0, j_0. Define recursively:

$$D_{n+1} = [D_n \to D_n]$$

and introduce the pairs i_{n+1}, j_{n+1} making D_{n+1} a projection of D_{n+2} by the method of 3.7. Specifically we shall have for $x \in D_{n+1}$ and $x' \in D_{n+2}$:

$$i_{n+1}(x) = i_n \circ x \circ j_n,$$

$$j_{n+1}(x') = j_n \circ x' \circ i_n.$$

Since these spaces are more than continuous lattices being function spaces, it is interesting to note that the maps i_n preserve function value as an algebraic operation as follows:

$$i_n(f(x)) = i_{n+1}(f)(i_n(x)),$$

where $x \in D_n$ and $f \in D_{n+1}$. Thus in passing to the limit space D_∞ something of functional application must also be preserved. The precise result shows that indeed D_∞ becomes its own function space.

corollary_4_3.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf

import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))
variable {D' : Type*} [CompleteLattice D']

theorem corollary_4_3 (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) :
    ∃! g : ScottMap (InverseLimit D P) D', ∀ n x, f n x = g (embInf D P n x) := by
  refine ⟨⟨coconeInf D P f,
    PreservesDirectedSup.continuous_of (coconeInf.preservesDirectedSup D P f)⟩, ?_, ?_⟩
  · exact fun n x => (coconeInf.comp_embInf D P f hf n x).symm
  · intro g hg
    ext x

    calc g x = g (⋃ n, embInf D P n (x.1 n)) := by rw [← inverseLimit.eq_iSup D P x]
      _ = g (sSup (Set.range fun n => embInf D P n (x.1 n))) := by rw [sSup_range]
      _ = sSup (g '' Set.range fun n => embInf D P n (x.1 n)) :=
          g.preservesDirectedSup_coe _ (Set.range_nonempty _) (embInf.family_directed
    D P x)
      _ = ⋃ n, g (embInf D P n (x.1 n)) := by rw [← Set.range_comp, sSup_range]; rfl
      _ = ⋃ n, f n (x.1 n) := by simp_rw [← hg]

end InverseLimit
end DomainTheory.ContinuousLattice
```

2.5.18 The function space tower

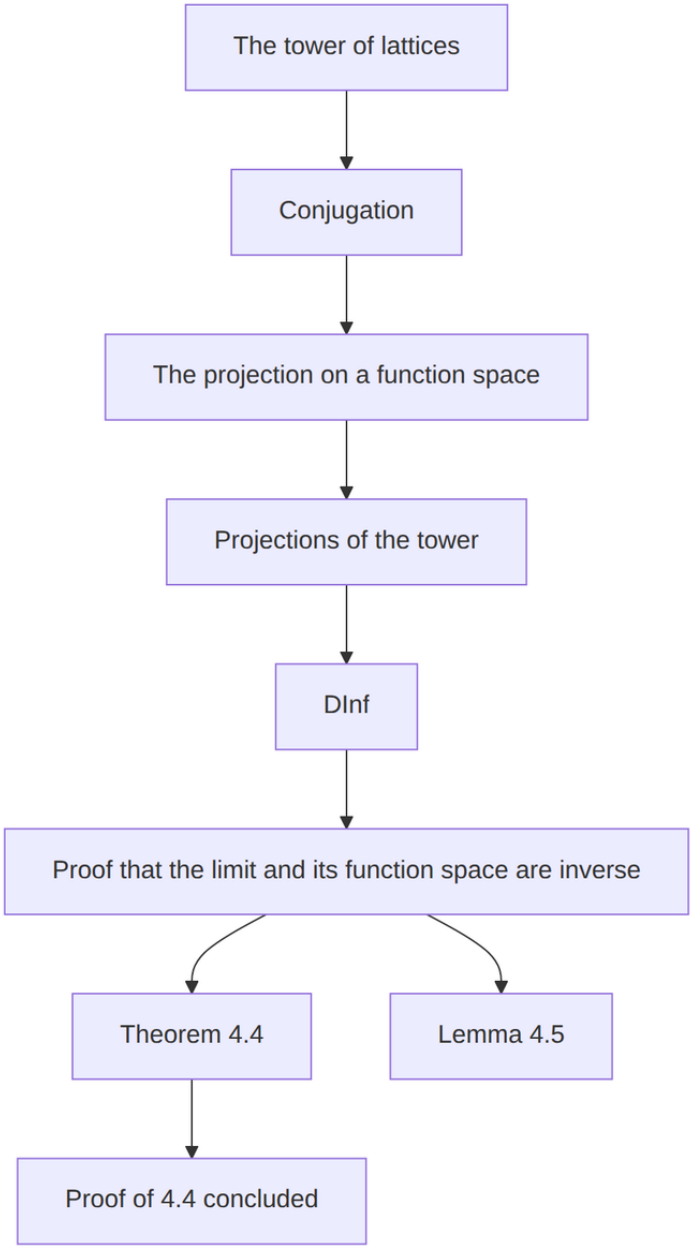


Figure 16: The function space tower.

The tower of lattices `CLat.lean`

```
module

public import Mathlib.Order.CompleteLattice.Defs
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section

universe u

namespace DomainTheory.ContinuousLattice

structure CLat : Type (u + 1) where
  carrier : Type u
  [str : CompleteLattice carrier]

attribute [instance] CLat.str

namespace CLat

noncomputable def tower (D₀ : CLat.{u}) : ℕ → CLat.{u}
  | 0 => D₀
  | (n + 1) => ⟨ScottMap (CLat.tower D₀ n).carrier (CLat.tower D₀ n).carrier⟩

end CLat

def towerType (D₀ : CLat.{u}) (n : ℕ) : Type u := (CLat.tower D₀ n).carrier

noncomputable instance towerCompleteLattice (D₀ : CLat.{u}) (n : ℕ) :
    CompleteLattice (towerType D₀ n) := (CLat.tower D₀ n).str

theorem towerType_succ (D₀ : CLat.{u}) (n : ℕ) :
    towerType D₀ (n + 1) = ScottMap (towerType D₀ n) (towerType D₀ n) := rfl

@[simp] theorem towerType_zero (D₀ : CLat.{u}) : towerType D₀ 0 = D₀.carrier := rfl

def towerToMap {D₀ : CLat.{u}} {n : ℕ} (f : towerType D₀ (n + 1)) :
    ScottMap (towerType D₀ n) (towerType D₀ n) := f

instance towerCoeFun {D₀ : CLat.{u}} {n : ℕ} :
    CoeFun (towerType D₀ (n + 1)) (fun _ => towerType D₀ n → towerType D₀ n) where
  coe f := towerToMap f

@[simp] theorem towerToMap_coe {D₀ : CLat.{u}} {n : ℕ} (f : towerType D₀ (n + 1))
    (x : towerType D₀ n) : towerToMap f x = f x := rfl

end DomainTheory.ContinuousLattice
```

Conjugation `conjMap.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.CLat
public import Mathlib.Data.Set.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

universe u
attribute [instance] CLat.str

section Conj

open Set

variable {Y Z W : Type u} [CompleteLattice Y] [CompleteLattice Z] [CompleteLattice W]

def conjMap.fun (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) :
    ScottMap W Z :=
  post.comp (f.comp pre)

namespace conjMap

theorem fun_apply (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) (x : W
    ) :
    conjMap.fun post pre f x = post (f (pre x)) := rfl

theorem preservesDirectedSup (post : ScottMap Y Z) (pre : ScottMap W Y) :
    PreservesDirectedSup (conjMap.fun post pre) := by
  intro F hF hFdir
  apply ScottMap.ext
  intro x
  have hdir : DirectedOn (· ≤ ·) ((fun f : ScottMap Y Y => f (pre x)) '' F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨c (pre x), ⟨c, hc, rfl⟩, hac (pre x), hbc (pre x)⟩
  show post ((sSup F : ScottMap Y Y) (pre x)) = (sSup (conjMap.fun post pre '' F) :
    ScottMap W Z) x
  rw [ScottMap.sSup_apply F (pre x), post.preservesDirectedSup_coe _ (hF.image _) hdir
    ,
    ScottMap.sSup_apply, Set.image_image, Set.image_image]
  rfl

end conjMap

noncomputable def conjMap (post : ScottMap Y Z) (pre : ScottMap W Y) :
    ScottMap (ScottMap Y Y) (ScottMap W Z) :=
  ⟨conjMap.fun post pre, PreservesDirectedSup.continuous_of (conjMap.
    preservesDirectedSup post pre)⟩

namespace conjMap

@[simp] theorem apply (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) (x
     : W) :
    conjMap post pre f x = post (f (pre x)) := rfl

end conjMap

end Conj

end DomainTheory.ContinuousLattice
```

The projection on a function space `functionSpaceProjection.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.functionSpaceInclFun
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.functionSpaceRetrFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.FunctionSpace

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

attribute [instance] CLat.str

namespace ContinuousLatticeProjection

theorem functionSpace_inclRetrLe {D D' : Type u} [CompleteLattice D] [CompleteLattice
    D']
    (P : ContinuousLatticeProjection D D') :
    ∀ g, functionSpaceInclFun P (functionSpaceRetrFun P g) ≤ g := by
  intro g
  exact ScottMap.le_def.mpr ((FunctionSpace.projection P P).incl_retr_le g)

theorem functionSpace_retrIncl {D D' : Type u} [CompleteLattice D] [CompleteLattice D
    ']
    (P : ContinuousLatticeProjection D D') :
    ∀ f, functionSpaceRetrFun P (functionSpaceInclFun P f) = f :=
  (FunctionSpace.projection P P).retr_incl

noncomputable def functionSpace {D D' : Type u} [CompleteLattice D] [CompleteLattice D
    ']
    (P : ContinuousLatticeProjection D D') :
    ContinuousLatticeProjection (ScottMap D D) (ScottMap D' D') where
  incl := ⟨functionSpaceInclFun P, functionSpaceInclFun.outer_continuous P⟩
  retr := ⟨functionSpaceRetrFun P, functionSpaceRetrFun.outer_continuous P⟩
  retr_incl := functionSpace_retrIncl P
  incl_retr_le := functionSpace_inclRetrLe P

end ContinuousLatticeProjection

end DomainTheory.ContinuousLattice
```

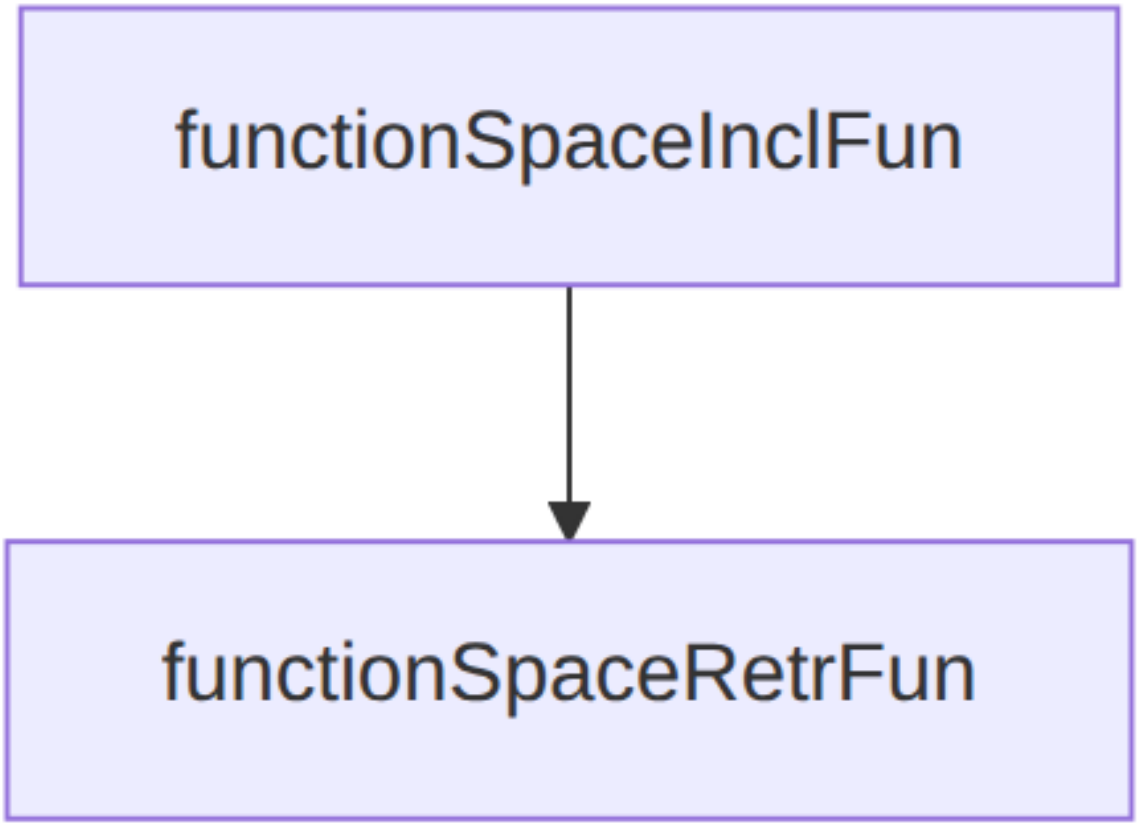


Figure 17: The projection on a function space.

functionSpaceInclFun `functionSpaceInclFun.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u
attribute [instance] CLat.str

namespace functionSpaceInclFun

theorem continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : ContinuousLatticeProjection D D') (f : ScottMap D D) :
```

```
    @Continuous D' D' scottTopologicalSpace scottTopologicalSpace
      (fun x => P.incl (f (P.retr x))) :=
  @Continuous.congr D' D' scottTopologicalSpace scottTopologicalSpace
    (P.incl.comp (f.comp P.retr)).1 (fun x => P.incl (f (P.retr x)))
    (P.incl.comp (f.comp P.retr)).continuous (fun _ => rfl)

end functionSpaceInclFun

def functionSpaceInclFun {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : ContinuousLatticeProjection D D') (f : ScottMap D D) : ScottMap D' D' :=
  ⟨fun x => P.incl (f (P.retr x)), functionSpaceInclFun.continuous P f⟩

namespace functionSpaceInclFun

theorem outer_continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : ContinuousLatticeProjection D D') :
    @Continuous (ScottMap D D) (ScottMap D' D')
      scottTopologicalSpace scottTopologicalSpace (functionSpaceInclFun P) := by
  exact @Continuous.congr (ScottMap D D) (ScottMap D' D')
    scottTopologicalSpace scottTopologicalSpace (conjMap P.incl P.retr).1
    (functionSpaceInclFun P) (conjMap P.incl P.retr).continuous
    (fun f => ScottMap.ext fun _ => rfl)

end functionSpaceInclFun

end DomainTheory.ContinuousLattice
```

functionSpaceRetrFun `functionSpaceRetrFun.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u
attribute [instance] CLat.str

namespace functionSpaceRetrFun

theorem continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : ContinuousLatticeProjection D D') (g : ScottMap D' D') :
    @Continuous D D scottTopologicalSpace scottTopologicalSpace
      (fun x => P.retr (g (P.incl x))) :=
  @Continuous.congr D D scottTopologicalSpace scottTopologicalSpace
    (P.retr.comp (g.comp P.incl)).1 (fun x => P.retr (g (P.incl x)))
    (P.retr.comp (g.comp P.incl)).continuous (fun _ => rfl)
```

```
end functionSpaceRetrFun

def functionSpaceRetrFun {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : ContinuousLatticeProjection D D') (g : ScottMap D' D') : ScottMap D D :=
  ⟨fun x => P.retr (g (P.incl x)), functionSpaceRetrFun.continuous P g⟩

namespace functionSpaceRetrFun

theorem outer_continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : ContinuousLatticeProjection D D') :
    @Continuous (ScottMap D' D') (ScottMap D D)
      scottTopologicalSpace scottTopologicalSpace (functionSpaceRetrFun P) := by
  exact @Continuous.congr (ScottMap D' D') (ScottMap D D)
    scottTopologicalSpace scottTopologicalSpace (conjMap P.retr P.incl).1
    (functionSpaceRetrFun P) (conjMap P.retr P.incl).continuous
    (fun g => ScottMap.ext fun _ => rfl)

end functionSpaceRetrFun

end DomainTheory.ContinuousLattice
```

ContinuousLatticeProjection.functionSpace

Projections of the tower towerProj.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    functionSpaceProjection
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

noncomputable def towerProj (D₀ : CLat.{u})
    (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier)) :
    ∀ n, ContinuousLatticeProjection (towerType D₀ n) (towerType D₀ (n + 1))
  | 0 => j₀
  | (n + 1) => ContinuousLatticeProjection.functionSpace (towerProj D₀ j₀ n)

namespace towerProj

theorem succ (D₀ : CLat.{u})
    (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier)) (n :
    ℕ) :
    let P := towerProj D₀ j₀ n
    (towerProj D₀ j₀ (n + 1)).incl = conjMap P.incl P.retr ∧
      (towerProj D₀ j₀ (n + 1)).retr = conjMap P.retr P.incl := by
  exact ⟨rfl, rfl⟩
```

```
section Tower

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

theorem succ_incl_apply (n : ℕ) (x : towerType D₀ (n + 1)) (y : towerType D₀ (n + 1)) :
    ((towerProj D₀ j₀ (n + 1)).incl x) y
      = (towerProj D₀ j₀ n).incl (x ((towerProj D₀ j₀ n).retr y)) := rfl

theorem succ_retr_apply (n : ℕ) (x' : towerType D₀ (n + 2)) (y : towerType D₀ n) :
    ((towerProj D₀ j₀ (n + 1)).retr x') y
      = (towerProj D₀ j₀ n).retr (x' ((towerProj D₀ j₀ n).incl y)) := rfl

theorem incl_apply (n : ℕ) (f : towerType D₀ (n + 1)) (x : towerType D₀ n) :
    (towerProj D₀ j₀ n).incl (f x)
      = ((towerProj D₀ j₀ (n + 1)).incl f) ((towerProj D₀ j₀ n).incl x) := by
  rw [succ_incl_apply, (towerProj D₀ j₀ n).retr_incl]

end Tower

end towerProj

end DomainTheory.ContinuousLattice
```

DInf DInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.towerProj
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

abbrev DInf : Type u := InverseLimit (towerType D₀) (towerProj D₀ j₀)

namespace DInf

abbrev fn : Type u := ScottMap (DInf D₀ j₀) (DInf D₀ j₀)

end DInf
```

```
end LimitMaps

end DomainTheory.ContinuousLattice
```

DInfFn

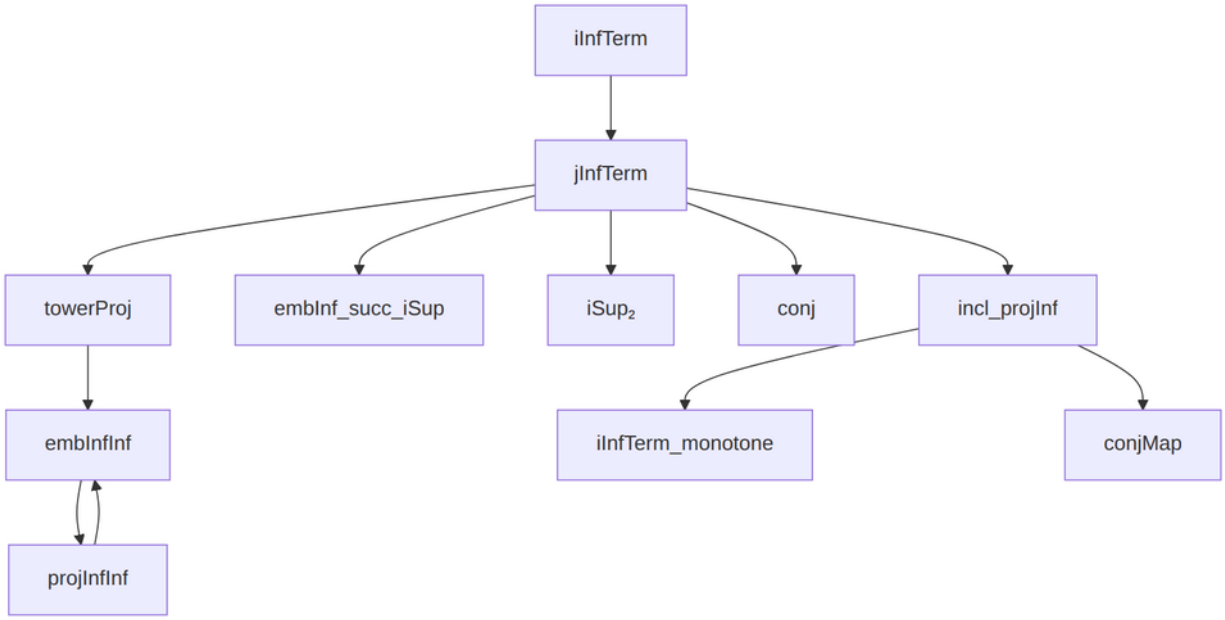


Figure 18: Proof that the limit and its function space are inverse.

Proof that the limit and its function space are inverse

iInfTerm `iInfTerm.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.limitProjectionToLevel
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.DInf

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))
```

```
noncomputable def iInfTerm (n : ℕ) : ScottMap (DInf D₀ j₀) (DInf.fn D₀ j₀) :=
  (conjMap (embInf (towerType D₀) (towerProj D₀ j₀) n)
           (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)).comp
    (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) (n + 1))

namespace iInfTerm

@[simp] theorem apply (n : ℕ) (x z : DInf D₀ j₀) :
    (iInfTerm D₀ j₀ n x) z
      = embInf (towerType D₀) (towerProj D₀ j₀) n
          ((x.1 (n + 1)) ((limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n) z
    )) := rfl

end iInfTerm

end LimitMaps

end DomainTheory.ContinuousLattice
```

embInfInf `embInfInf.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iInfTerm
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Data.Set.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

noncomputable def embInfInf : ScottMap (DInf D₀ j₀) (DInf.fn D₀ j₀) :=
  ⋃ n, iInfTerm D₀ j₀ n

namespace embInfInf

theorem apply (x : DInf D₀ j₀) :
    embInfInf D₀ j₀ x = ⋃ n, iInfTerm D₀ j₀ n x := by
  show (sSup (Set.range (iInfTerm D₀ j₀)) : ScottMap _ _) x = _
  rw [ScottMap.sSup_apply, ← Set.range_comp, sSup_range]
  rfl

theorem preservesDirectedSup :
    PreservesDirectedSup (embInfInf D₀ j₀ : DInf D₀ j₀ → DInf.fn D₀ j₀) :=
  (proposition_2_5 _).mp (embInfInf D₀ j₀).continuous
```

```
end embInfInf

end LimitMaps

end DomainTheory.ContinuousLattice
```

jInfTerm `jInfTerm.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iInfTerm

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

noncomputable def jInfTerm (n : ℕ) : ScottMap (DInf.fn D₀ j₀) (DInf D₀ j₀) :=
  (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)).comp
    (conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
             (embInf (towerType D₀) (towerProj D₀ j₀) n))

namespace jInfTerm

@[simp] theorem apply (n : ℕ) (f : DInf.fn D₀ j₀) :
    jInfTerm D₀ j₀ n f
      = embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
          (conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
                   (embInf (towerType D₀) (towerProj D₀ j₀) n) f) := rfl

end jInfTerm

end LimitMaps

end DomainTheory.ContinuousLattice
```

incl_projInf `incl_projInf_le_projInf_succ.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

theorem incl_projInf_le_projInf_succ (n : ℕ) (w : DInf D₀ j₀) :
    (towerProj D₀ j₀ n).incl (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n
    w)
      ≤ limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) (n + 1) w := by
  have h := (towerProj D₀ j₀ n).incl_retr_le
    (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) (n + 1) w)
  rwa [show (towerProj D₀ j₀ n).retr (limitProjectionToLevel (towerType D₀) (towerProj
    D₀ j₀) (n + 1) w)
      = limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n w from w.2 n] at h

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

iInfTerm_monotone `iInfTerm_monotone.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.towerProj
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    incl_projInf_le_projInf_succ
public import DomainTheory.ContinuousLattice.InverseLimits.embInf

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))
```

```
theorem iInfTerm_le_succ (x : DInf D₀ j₀) (m : ℕ) :
    iInfTerm D₀ j₀ m x ≤ iInfTerm D₀ j₀ (m + 1) x := by
  rw [ScottMap.le_def]
  intro z
  rw [iInfTerm.apply, iInfTerm.apply,
    ← embInf.succ (towerType D₀) (towerProj D₀ j₀) m
      (x.1 (m + 1) (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) m z))]
  refine embInf.monotone (towerType D₀) (towerProj D₀ j₀) (m + 1) ?_
  have hab : (towerProj D₀ j₀ m).incl (limitProjectionToLevel (towerType D₀) (towerProj
     D₀ j₀) m z)
      ≤ limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) (m + 1) z :=
    incl_projInf_le_projInf_succ D₀ j₀ m z
  calc (towerProj D₀ j₀ m).incl
          (x.1 (m + 1) (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) m z))
      = (towerProj D₀ j₀ m).incl
          (((towerProj D₀ j₀ (m + 1)).retr (x.1 (m + 2)))
            (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) m z)) := by
        rw [show ((towerProj D₀ j₀ (m + 1)).retr (x.1 (m + 2))) = x.1 (m + 1) from x.2
    (m + 1)]
    _ = (towerProj D₀ j₀ m).incl
          ((towerProj D₀ j₀ m).retr
            (x.1 (m + 2) ((towerProj D₀ j₀ m).incl
              (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) m z)))) := by
        rw [towerProj.succ_retr_apply]
    _ ≤ x.1 (m + 2) ((towerProj D₀ j₀ m).incl
          (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) m z)) :=
        (towerProj D₀ j₀ m).incl_retr_le _
    _ ≤ x.1 (m + 2) (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) (m + 1) z)
     :=
        ScottMap.monotone (x.1 (m + 2)) hab

theorem iInfTerm_monotone (x : DInf D₀ j₀) : Monotone (fun m => iInfTerm D₀ j₀ m x) :=
  monotone_nat_of_le_succ (iInfTerm_le_succ D₀ j₀ x)

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

embInf_succ_iSup embInf_succ_iSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import Mathlib.Data.Set.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str
```

```
section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

theorem embInf_succ_iSup (n : ℕ) (f : ℕ → towerType D₀ (n + 1)) (hf : Monotone f) :
    embInf (towerType D₀) (towerProj D₀ j₀) (n + 1) (⋃ m, f m) =
      ⋃ m, embInf (towerType D₀) (towerProj D₀ j₀) (n + 1) (f m) := by
  have hdir : DirectedOn (· ≤ ·) (Set.range f) :=
    directedOn_range.2 fun a b => ⟨max a b, hf (le_max_left a b), hf (le_max_right a b)
    ⟩
  have hne := Set.range_nonempty f
  rw [show (⋃ m, f m) = sSup (Set.range f) from sSup_range.symm,
    (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)).preservesDirectedSup_coe
      (Set.range f) hne hdir, ← Set.range_comp]
  rfl

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

iSup2 iSup2_monotone_eq_diagonal.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

theorem iSup₂_monotone_eq_diagonal {α : Type*} [CompleteLattice α] (f : ℕ → ℕ → α)
    (hfm : ∀ n, Monotone (f n)) (hfn : ∀ m, Monotone (fun n => f n m)) :
    ⋃ n, ⋃ m, f n m = ⋃ n, f n n := by
```

```
  apply le_antisymm
  · refine iSup_le fun n => iSup_le fun m => ?_
    have hk : n ≤ n ⊔ m := le_sup_left
    have hk' : m ≤ n ⊔ m := le_sup_right
    calc f n m ≤ f (n ⊔ m) m := hfn m hk
      _ ≤ f (n ⊔ m) (n ⊔ m) := hfm (n ⊔ m) hk'
      _ ≤ ⨆ n', f n' n' := le_iSup (fun n' => f n' n') (n ⊔ m)
  · refine iSup_le fun n => le_trans (le_iSup (f n) n) (le_iSup (fun n' => ⨆ m, f n' m)
     n)

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

conjMap conjMap_iSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    incl_projInf_le_projInf_succ
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import Mathlib.Data.Set.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

theorem conjMap_iSup (n : ℕ) (post : ScottMap (DInf D₀ j₀) (towerType D₀ n))
    (pre : ScottMap (towerType D₀ n) (DInf D₀ j₀))
    (f : ℕ → ScottMap (DInf D₀ j₀) (DInf D₀ j₀)) (hf : Monotone f) :
    conjMap post pre (⨆ m, f m) = ⨆ m, conjMap post pre (f m) := by
  have hdir : DirectedOn (· ≤ ·) (Set.range f) :=
    directedOn_range.2 fun a b => ⟨max a b, hf (le_max_left a b), hf (le_max_right a b)
    ⟩
  have hne := Set.range_nonempty f
  rw [show (⨆ m, f m) = sSup (Set.range f) from sSup_range.symm,
    (conjMap post pre).preservesDirectedSup_coe (Set.range f) hne hdir, ← Set.
    range_comp]
  rfl

theorem conjMap_incl_le_conjMap_succ (n : ℕ) (f : DInf.fn D₀ j₀) :
    (towerProj D₀ j₀ (n + 1)).incl
        (conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
```

```
                (embInf (towerType D₀) (towerProj D₀ j₀) n) f)
      ≤ conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) (n + 1))
                (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)) f := by
  refine ScottMap.le_def.mpr fun y => ?_
  calc (towerProj D₀ j₀ n).incl
         (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n
           (f (embInf (towerType D₀) (towerProj D₀ j₀) n ((towerProj D₀ j₀ n).retr y)))
    )
      ≤ (towerProj D₀ j₀ n).incl
          (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n
            (f (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1) y))) := by
        refine (towerProj D₀ j₀ n).incl.monotone
          ((limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n).monotone (f.
    monotone ?_))
        rw [← embInf.succ (towerType D₀) (towerProj D₀ j₀) n ((towerProj D₀ j₀ n).retr
    y)]
        exact embInf.monotone (towerType D₀) (towerProj D₀ j₀) (n + 1)
          ((towerProj D₀ j₀ n).incl_retr_le y)
    _ ≤ limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) (n + 1)
          (f (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1) y)) :=
        incl_projInf_le_projInf_succ D₀ j₀ n
          (f (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1) y))

end LimitMaps

end DomainTheory.ContinuousLattice
```

conj conj_iInfTerm_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.InverseLimits.proposition_4_2
public import DomainTheory.ContinuousLattice.FunctionSpaces.FunctionSpace

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

theorem conj_iInfTerm_eq (n : ℕ) (x : DInf D₀ j₀) :
    conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
            (embInf (towerType D₀) (towerProj D₀ j₀) n)
```

```
            (iInfTerm D₀ j₀ n x) = x.1 (n + 1) :=
  (FunctionSpace.projection
      (limitLevelProjection (towerType D₀) (towerProj D₀ j₀) n)
      (limitLevelProjection (towerType D₀) (towerProj D₀ j₀) n)).retr_incl
    (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) (n + 1) x)

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

towerProj `towerProj_retr_conjMap_succ.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.InverseLimits.embInf

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

theorem towerProj_retr_conjMap_succ (n : ℕ) (f : DInf.fn D₀ j₀) :
    (towerProj D₀ j₀ (n + 1)).retr
        (conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) (n + 1))
                 (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)) f)
      = conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
                (embInf (towerType D₀) (towerProj D₀ j₀) n) f := by
  apply ScottMap.ext
  intro y
  show (towerProj D₀ j₀ n).retr
      (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) (n + 1)
        (f (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
          ((towerProj D₀ j₀ n).incl y))))
    = limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n
        (f (embInf (towerType D₀) (towerProj D₀ j₀) n y))
  rw [embInf.succ (towerType D₀) (towerProj D₀ j₀) n y]
  exact (f (embInf (towerType D₀) (towerProj D₀ j₀) n y)).2 n

end Thm44b
```

```
end LimitMaps

end DomainTheory.ContinuousLattice
```

projInfInf projInfInf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap_iSup
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInf_succ_iSup
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conj_iInfTerm_eq
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iInfTerm_monotone
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iSup₂
    _monotone_eq_diagonal
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Data.Set.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

noncomputable def projInfInf : ScottMap (DInf.fn D₀ j₀) (DInf D₀ j₀) :=
  ⋃ n, jInfTerm D₀ j₀ n

namespace projInfInf

theorem apply (f : DInf.fn D₀ j₀) :
    projInfInf D₀ j₀ f = ⋃ n, jInfTerm D₀ j₀ n f := by
  show (sSup (Set.range (jInfTerm D₀ j₀)) : ScottMap _ _) f = _
  rw [ScottMap.sSup_apply, ← Set.range_comp, sSup_range]
  rfl

theorem preservesDirectedSup :
    PreservesDirectedSup (projInfInf D₀ j₀ : DInf.fn D₀ j₀ → DInf D₀ j₀) :=
  (proposition_2_5 _).mp (projInfInf D₀ j₀).continuous

theorem embInfInf_eq (x : DInf D₀ j₀) :
    projInfInf D₀ j₀ (embInfInf D₀ j₀ x) = ⋃ n, embInf (towerType D₀) (towerProj D₀ j₀)
    (n + 1) (x.1 (n + 1)) := by
  rw [apply]
  set g := fun n m =>
    embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
```

```
    (conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
             (embInf (towerType D₀) (towerProj D₀ j₀) n)
             (iInfTerm D₀ j₀ m x)) with hg
have hmono (n : ℕ) : Monotone (fun m =>
    conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
            (embInf (towerType D₀) (towerProj D₀ j₀) n)
            (iInfTerm D₀ j₀ m x)) := fun a b hab =>
  (conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
           (embInf (towerType D₀) (towerProj D₀ j₀) n)).monotone
    (ScottMap.le_def.mpr (iInfTerm_monotone D₀ j₀ x hab))
have hinner (n : ℕ) : jInfTerm D₀ j₀ n (embInfInf D₀ j₀ x) = ⋃ m, g n m := by
  calc jInfTerm D₀ j₀ n (embInfInf D₀ j₀ x)
      = embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
          (⋃ m, conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
                        (embInf (towerType D₀) (towerProj D₀ j₀) n)
                        (iInfTerm D₀ j₀ m x)) := by
        rw [jInfTerm.apply, embInfInf.apply,
          conjMap_iSup D₀ j₀ n (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀
  ) n)
            (embInf (towerType D₀) (towerProj D₀ j₀) n)
            (fun m => iInfTerm D₀ j₀ m x) (iInfTerm_monotone D₀ j₀ x)]
        rfl
    _ = ⋃ m, embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
            (conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
                     (embInf (towerType D₀) (towerProj D₀ j₀) n)
                     (iInfTerm D₀ j₀ m x)) :=
        embInf_succ_iSup D₀ j₀ n _ (hmono n)
    _ = ⋃ m, g n m := by simp only [hg]
have g_mono_m (n : ℕ) : Monotone (g n) := by
  intro a b hab
  rw [hg]
  exact (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)).monotone
    ((conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
              (embInf (towerType D₀) (towerProj D₀ j₀) n)).monotone
      (ScottMap.le_def.mpr (iInfTerm_monotone D₀ j₀ x hab)))
have g_mono_n_succ (m n : ℕ) : g n m ≤ g (n + 1) m := by
  rw [hg]
  dsimp only
  rw [← embInf.succ (towerType D₀) (towerProj D₀ j₀) (n + 1)
      (conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
               (embInf (towerType D₀) (towerProj D₀ j₀) n)
               (iInfTerm D₀ j₀ m x))]
  exact embInf.monotone (towerType D₀) (towerProj D₀ j₀) (n + 2)
    (conjMap_incl_le_conjMap_succ D₀ j₀ n (iInfTerm D₀ j₀ m x))
have g_mono_n (m : ℕ) : Monotone (fun n => g n m) :=
  monotone_nat_of_le_succ (g_mono_n_succ m)
have hin : (⋃ n, jInfTerm D₀ j₀ n (embInfInf D₀ j₀ x)) = ⋃ n, ⋃ m, g n m := by
  congr 1
  funext n
  exact hinner n
rw [hin, iSup₂_monotone_eq_diagonal g g_mono_m g_mono_n]
congr 1
funext n
rw [hg]
dsimp only
rw [conj_iInfTerm_eq D₀ j₀ n x]
```

```
theorem comp_embInfInf :
    (projInfInf D₀ j₀).comp (embInfInf D₀ j₀) = ScottMap.idMap := by
  apply ScottMap.ext
  intro x
  have hmono : Monotone (fun k => embInf (towerType D₀) (towerProj D₀ j₀) k (x.1 k)) :=
    monotone_nat_of_le_succ (embInf.le_succ (towerType D₀) (towerProj D₀ j₀) x)
  rw [ScottMap.comp_apply, ScottMap.idMap_apply, embInfInf_eq D₀ j₀ x,
    Monotone.iSup_nat_add hmono 1]
  exact (inverseLimit.eq_iSup (towerType D₀) (towerProj D₀ j₀) x).symm

theorem embInfInf (x : DInf D₀ j₀) :
    projInfInf D₀ j₀ (embInfInf D₀ j₀ x) = x := by
  have h := congrArg (fun g => g x) (comp_embInfInf D₀ j₀)
  simpa [ScottMap.comp_apply, ScottMap.idMap_apply] using h

end projInfInf

end LimitMaps

end DomainTheory.ContinuousLattice
```

embInfInf embInfInf_projInfInf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.projInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iSup₂
    _monotone_eq_diagonal
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    towerProj_retr_conjMap_succ
public import DomainTheory.ContinuousLattice.InverseLimits.lemma_4_5
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import Mathlib.Data.Set.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

theorem embInfInf_comp_projInfInf :
    (embInfInf D₀ j₀).comp (projInfInf D₀ j₀) = ScottMap.idMap := by
  apply ScottMap.ext
  intro f
  rw [ScottMap.comp_apply, ScottMap.idMap_apply]
```

```
apply ScottMap.ext
intro z

set r : ℕ → ScottMap (DInf D₀ j₀) (DInf D₀ j₀) :=
  fun n => (embInf (towerType D₀) (towerProj D₀ j₀) n).comp
            (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n) with hr
have hrw : ∀ n (w : DInf D₀ j₀), r n w
    = embInf (towerType D₀) (towerProj D₀ j₀) n
        (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n w) := by
  intro n w
  simp only [hr, ScottMap.comp_apply]
have hr_mono : ∀ (w : DInf D₀ j₀), Monotone (fun m => r m w) := by
  intro w
  refine monotone_nat_of_le_succ (fun m => ?_)
  show r m w ≤ r (m + 1) w
  rw [hrw, hrw]
  exact embInf.le_succ (towerType D₀) (towerProj D₀ j₀) w m

have hA : ∀ (w : DInf D₀ j₀), w = ⋃ m, r m w := by
  intro w
  have h1 : (⋃ m, r m w)
      = ⋃ m, embInf (towerType D₀) (towerProj D₀ j₀) m
              (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) m w) :=
    iSup_congr (fun m => hrw m w)
  rw [h1]
  exact inverseLimit.eq_iSup (towerType D₀) (towerProj D₀ j₀) w

have hsup_apply : ∀ (g : ℕ → ScottMap (DInf D₀ j₀) (DInf D₀ j₀)) (w : DInf D₀ j₀),
    (⋃ n, g n) w = ⋃ n, g n w := by
  intro g w
  rw [show (⋃ n, g n) = sSup (Set.range g) from sSup_range.symm,
    ScottMap.sSup_apply, ← Set.range_comp, sSup_range]
  rfl

have hcont : ∀ (g : ScottMap (DInf D₀ j₀) (DInf D₀ j₀)) (a : ℕ → DInf D₀ j₀),
    Monotone a → g (⋃ m, a m) = ⋃ m, g (a m) := by
  intro g a ha
  have hdir : DirectedOn (· ≤ ·) (Set.range a) :=
    directedOn_range.2 fun i j => ⟨max i j, ha (le_max_left i j), ha (le_max_right i
  j)⟩
  rw [show (⋃ m, a m) = sSup (Set.range a) from sSup_range.symm,
    g.preservesDirectedSup_coe (Set.range a) (Set.range_nonempty a) hdir,
    ← Set.range_comp, sSup_range]
  rfl

have hpi : projInfInf D₀ j₀ f
    = ⋃ k, embInf (towerType D₀) (towerProj D₀ j₀) (k + 1)
        (conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) k)
                 (embInf (towerType D₀) (towerProj D₀ j₀) k) f) := by
  rw [projInfInf.apply]
  exact iSup_congr (fun n => jInfTerm.apply D₀ j₀ n f)

have hcoord : ∀ n, (projInfInf D₀ j₀ f).1 (n + 1)
    = conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) n)
              (embInf (towerType D₀) (towerProj D₀ j₀) n) f := by
  intro n
```

```
    rw [hpi]
    exact lemma_4_5 (towerType D₀) (towerProj D₀ j₀)
      (fun k => conjMap (limitProjectionToLevel (towerType D₀) (towerProj D₀ j₀) k)
                        (embInf (towerType D₀) (towerProj D₀ j₀) k) f)
      (fun m => towerProj_retr_conjMap_succ D₀ j₀ m f) n

  have hev : embInfInf D₀ j₀ (projInfInf D₀ j₀ f) z
      = ⋃ n, (iInfTerm D₀ j₀ n (projInfInf D₀ j₀ f)) z := by
    rw [embInfInf.apply]
    exact hsup_apply (fun n => iInfTerm D₀ j₀ n (projInfInf D₀ j₀ f)) z

  have hterm : ∀ n, (iInfTerm D₀ j₀ n (projInfInf D₀ j₀ f)) z = r n (f (r n z)) := by
    intro n
    rw [hrw, hrw, iInfTerm.apply, hcoord n]
    rfl
  have hmono_frz : Monotone (fun m => f (r m z)) :=
    fun a b hab => f.monotone (hr_mono z hab)
  have hfm : ∀ n, Monotone (fun m => r n (f (r m z))) :=
    fun n _ _ hab => (r n).monotone (f.monotone (hr_mono z hab))
  have hfn : ∀ m, Monotone (fun n => r n (f (r m z))) :=
    fun m => hr_mono (f (r m z))

  have hfz : f z = ⋃ n, r n (f (r n z)) :=
    calc f z = ⋃ k, r k (f z) := hA (f z)
      _ = ⋃ k, r k (f (⋃ m, r m z)) := by
            refine iSup_congr (fun k => ?_)
            rw [← hA z]
      _ = ⋃ k, r k (⋃ m, f (r m z)) := by
            refine iSup_congr (fun k => ?_)
            rw [hcont f (fun m => r m z) (hr_mono z)]
      _ = ⋃ k, ⋃ m, r k (f (r m z)) :=
            iSup_congr (fun k => hcont (r k) (fun m => f (r m z)) hmono_frz)
      _ = ⋃ n, r n (f (r n z)) :=
            iSup₂_monotone_eq_diagonal (fun n m => r n (f (r m z))) hfm hfn
  calc embInfInf D₀ j₀ (projInfInf D₀ j₀ f) z
      = ⋃ n, (iInfTerm D₀ j₀ n (projInfInf D₀ j₀ f)) z := hev
    _ = ⋃ n, r n (f (r n z)) := iSup_congr hterm
    _ = f z := hfz.symm

theorem embInfInf_projInfInf (f : DInf.fn D₀ j₀) :
    embInfInf D₀ j₀ (projInfInf D₀ j₀ f) = f := by
  have h := congrArg (fun g => g f) (embInfInf_comp_projInfInf D₀ j₀)
  simpa [ScottMap.comp_apply, ScottMap.idMap_apply] using h

theorem embInfInf_le_iff (x y : DInf D₀ j₀) :
    embInfInf D₀ j₀ x ≤ embInfInf D₀ j₀ y ↔ x ≤ y := by
  constructor
  · intro h
    have := (projInfInf D₀ j₀).monotone h
    rwa [projInfInf.embInfInf, projInfInf.embInfInf] at this
  · intro h; exact (embInfInf D₀ j₀).monotone h

end LimitMaps

end DomainTheory.ContinuousLattice
```

Theorem 4.4 4.4 Theorem. *The inverse limit D_∞ of the recursively defined sequence $\langle D_n, j_n\rangle_{n=0}^{\infty}$ of function spaces is not only a continuous lattice, but it is also homeomorphic to its own function space $[D_\infty \to D_\infty]$.*

Proof: We can write down directly the pair of maps i_∞, j_∞ that provide the homeomorphism:

$$i_\infty(x) = \bigsqcup_{n=0}^{\infty}(i_{n\infty} \circ x_{n+1} \circ j_{\infty n}),$$

$$j_\infty(f) = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(j_{\infty n} \circ f \circ i_{n\infty}).$$

Note that these formulae are simply generalizations at the limit for the formulae we used to define i_n, j_n in the first place. Thus it is not surprising that they would provide a projection of $[D_\infty \to D_\infty]$ upon D_∞. Indeed we can compute out $j_\infty(i_\infty(x))$, noting that all the lubs are monotone and that a double monotone limit can always be replaced by a single one in view of the continuity of the operations involved, obtaining

$$j_\infty(i_\infty(x)) = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(j_{\infty n} \circ i_{n\infty} \circ x_{n+1} \circ j_{\infty n} \circ i_{n\infty}) \quad = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(x_{n+1}) = x.$$

In the converse order the calculation is only a bit more complicated. The idea is that since all the functions f are continuous and since the elements x are the limits of their approximations, then each f is actually completely determined by its

In the converse order the calculation is only a bit more complicated. The idea is that since all the functions f are continuous and since the elements x are the limits of their approximations, then each f is actually completely determined by its sequence of restrictions $j_{\infty n} \circ f \circ i_{n\infty} \in D_{n+1}$. This simple idea can be made more precise with the aid of a lemma about D_∞, which allows us in certain cases to recognize projections from limits.

theorem_4_4.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.projInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInfInf_projInfInf
public import DomainTheory.ContinuousLattice.InverseLimits.proposition_4_1
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.theorem_4_4_more

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
```

```
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44d

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

theorem theorem_4_4_inverses :
    (projInfInf D₀ j₀).comp (embInfInf D₀ j₀) = ScottMap.idMap ∧
    (embInfInf D₀ j₀).comp (projInfInf D₀ j₀) = ScottMap.idMap :=
  ⟨projInfInf.comp_embInfInf D₀ j₀, embInfInf_comp_projInfInf D₀ j₀⟩

noncomputable def theorem_4_4_orderIso : OrderIso (DInf D₀ j₀) (DInf.fn D₀ j₀) :=
  (Equiv.mk (embInfInf D₀ j₀) (projInfInf D₀ j₀)
     (projInfInf.embInfInf D₀ j₀) (embInfInf_projInfInf D₀ j₀)).toOrderIso
    (embInfInf D₀ j₀).monotone (projInfInf D₀ j₀).monotone

theorem theorem_4_4 (h₀ : ContinuousLattice D₀.carrier) :
    @ContinuousLattice (DInf D₀ j₀) inferInstance ∧
      Nonempty
        (@Homeomorph (DInf D₀ j₀) (DInf.fn D₀ j₀)
          (inverseLimitTopology (towerType D₀) (towerProj D₀ j₀))
          (scottMapInducedPiTopology (DInf D₀ j₀) (DInf D₀ j₀))) := by
  have hTower : ∀ n, ContinuousLattice (towerType D₀ n) := by
    intro n
    induction n with
    | zero => exact h₀
    | succ n ih => exact stepMap.isContinuousLattice ih ih
  refine ⟨proposition_4_1 (towerType D₀) (towerProj D₀ j₀) hTower, ?_⟩
  exact ⟨theorem_4_4.sourceTopology_homeomorph D₀ j₀ hTower⟩

end Thm44d
end LimitMaps

end DomainTheory.ContinuousLattice
```

Lemma 4.5 **4.5 Lemma.** Suppose for each n we have $u_{(n+1)} \in D_{n+1}$ and we let:

$$u = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(u_{(n+1)}).$$

Then if

$$j_{n+1}(u_{(n+2)}) = u_{(n+1)}$$

for each n, we can conclude that:

$$j_{\infty(n+1)}(u) = u_{(n+1)}.$$

Proof: If the sequence $u_{(n+1)}$ satisfies the recursion, then the limit defining u is monotonic. Therefore by continuity of projection it suffices to prove that

$$j_{\infty(n+1)}(i_{(m+1)\infty}(u_{(m+1)})) = u_{(n+1)}$$

for all $m \geqslant n$. This is obvious for $m = n$, and it can be readily proved by induction for larger m using the various recursion equations. (Properly speaking the induction is done on the quantity $(m - n)$ using both n and m as variables.) $\square$

lemma_4_5.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.limitProjectionToLevel
public import DomainTheory.ContinuousLattice.InverseLimits.projectionAlongTower
public import Mathlib.Data.Set.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
variable (P : ∀ n, ContinuousLatticeProjection (D n) (D (n + 1)))

theorem lemma_4_5 (u : ∀ n, D (n + 1))
    (hu : ∀ n, (P (n + 1)).retr (u (n + 1)) = u n) (n : ℕ) :
    (⋃ k, embInf D P (k + 1) (u k) : InverseLimit D P).1 (n + 1) = u n := by
  let a : ℕ → InverseLimit D P := fun k => embInf D P (k + 1) (u k)
  have ha_succ (k : ℕ) : a k ≤ a (k + 1) := by
    rw [show a k = embInf D P (k + 1) (u k) from rfl,
      show a (k + 1) = embInf D P (k + 2) (u (k + 1)) from rfl,
      ← embInf.succ D P (k + 1) (u k)]
    exact embInf.monotone D P (k + 2)
      (by calc (P (k + 1)).incl (u k)
            = (P (k + 1)).incl ((P (k + 1)).retr (u (k + 1))) := by rw [hu k]
          _ ≤ u (k + 1) := (P (k + 1)).incl_retr_le _)
  have ha : Monotone a := monotone_nat_of_le_succ ha_succ
  have hprojTower : ∀ d n : ℕ,
      projectionAlongTower D P (show n + 1 ≤ n + d + 1 by omega) (u (n + d)) = u n :=
    by
    intro d
    induction d with
    | zero =>
        intro n
        simpa only [Nat.add_zero] using
          (projectionAlongTower.self D P (le_refl (n + 1)) (u n))
    | succ d ih =>
        intro n
        simp only [Nat.add_succ]
        rw [projectionAlongTower.succ D P
          (show n + 1 ≤ n + d + 1 by omega)
          (show n + 1 ≤ (n + d + 1) + 1 by omega)]
```

```
      rw [hu (n + d)]
      exact ih n
  have hprojection : ∀ d n : ℕ,
      (embInf D P (n + d + 1) (u (n + d))).1 (n + 1) = u n := by
    intro d n
    cases d with
    | zero =>
      change iComp D P (n + 1) (u n) (n + 1) = u n
      exact iComp.self D P (n + 1) (u n)
    | succ d =>
      change iComp D P (n + (d + 1) + 1) (u (n + (d + 1))) (n + 1) = u n
      rw [iComp.of_ge D P (by omega)]
      exact hprojTower (d + 1) n
  have hdir : DirectedOn (· ≤ ·) (Set.range a) :=
    directedOn_range.2 ha.directed_le
  have hcont := (limitProjectionToLevel D P (n + 1)).preservesDirectedSup_coe
    (Set.range a) (Set.range_nonempty a) hdir
  rw [show (⋃ k, embInf D P (k + 1) (u k) : InverseLimit D P)
      = sSup (Set.range a) from sSup_range.symm]
  change (limitProjectionToLevel D P (n + 1)) (sSup (Set.range a)) = u n
  rw [hcont, ← Set.range_comp, sSup_range]
  apply le_antisymm
  · exact iSup_le fun m => by
      by_cases hmn : n ≤ m
      · obtain ⟨d, rfl⟩ := Nat.exists_eq_add_of_le hmn
        exact le_of_eq (hprojection d n)
      · exact le_trans ((ha (Nat.le_of_not_ge hmn)) (n + 1))
          (le_of_eq (hprojection 0 n))
  · exact le_trans (le_of_eq (hprojection 0 n).symm) (le_iSup (fun m => (a m).1 (n + 1)
    ) n)

end InverseLimit
end DomainTheory.ContinuousLattice
```

Proof of 4.4 concluded **Proof of 4.4 concluded:** The lemma applies at once to our calculation, for we find:

$$i_\infty(j_\infty(f)) = \bigsqcup_{n=0}^{\infty}(i_{n\infty}\circ j_{\infty n}\circ f\circ i_{n\infty}\circ j_{\infty n}) \quad = \bigsqcup_{n=0}^{\infty}(i_{n\infty}\circ j_{\infty n})\circ f\circ \bigsqcup_{n=0}^{\infty}(i_{n\infty}\circ j_{\infty n}).$$

Here we have just applied the continuity of f to be able to confine the lub on the right. But now by the remark following 4.2, we note the functional equation:

$$\mathrm{id} = \bigsqcup_{n=0}^{\infty}(i_{n\infty}\circ j_{\infty n}),$$

and the proof that i_∞ and j_∞ are inverse to one another is complete. □

theorem_4_4_more.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.projInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInfInf_projInfInf
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import DomainTheory.ContinuousLattice.InverseLimits.proposition_4_1
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44d

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

noncomputable def theorem_4_4.scottTopology_homeomorph :
    @Homeomorph (DInf D₀ j₀) (DInf.fn D₀ j₀) scottTopologicalSpace
    scottTopologicalSpace :=
  @Homeomorph.mk (DInf D₀ j₀) (DInf.fn D₀ j₀) scottTopologicalSpace
    scottTopologicalSpace
    (Equiv.mk (embInfInf D₀ j₀) (projInfInf D₀ j₀)
      (projInfInf.embInfInf D₀ j₀) (embInfInf_projInfInf D₀ j₀))
    (embInfInf D₀ j₀).continuous (projInfInf D₀ j₀).continuous

end Thm44d

end LimitMaps

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44d

variable (D₀ : CLat.{u})
  (j₀ : ContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

noncomputable def theorem_4_4.sourceTopology_homeomorph
    (hTower : ∀ n, ContinuousLattice (towerType D₀ n)) :
    @Homeomorph (DInf D₀ j₀) (DInf.fn D₀ j₀)
      (inverseLimitTopology (towerType D₀) (towerProj D₀ j₀))
      (scottMapInducedPiTopology (DInf D₀ j₀) (DInf D₀ j₀)) := by
  have hDInf : ContinuousLattice (DInf D₀ j₀) :=
    proposition_4_1 (towerType D₀) (towerProj D₀ j₀) hTower
  rw [← inverseLimit.scottTopology_eq_induced_pi
```

```
        (towerType D₀) (towerProj D₀ j₀) hTower,
      ← scottMapInducedPiTopology.scottTopology_eq_inducedPi hDInf hDInf]
  exact theorem_4_4.scottTopology_homeomorph D₀ j₀

end Thm44d

end LimitMaps

end DomainTheory.ContinuousLattice
```

2.5.19 The functorial form ✓

We can explain the idea of this proof in other terms using a suggestion made to me by F. W. Lawvere. Consider the category of

We can explain the idea of this proof in other terms using a suggestion made to me by F. W. Lawvere. Consider the category of continuous lattices and projections. In that category our D_∞ is, as we have remarked, both a *direct* and an *inverse* limit. Note too that with regard to projections $[D \to D']$ is a *functor*, for we can also define $[j \to j']$ when the maps are projections. In this language our particular inverse system is defined by the recursion:

$$D_{n+1} = [D_n \to D_n] \quad \text{and} \quad j_{n+1} = [j_n \to j_n],$$

where D_0 and j_0 are given in advance. Now the function space construction is continuous in its two arguments, turning an inverse limit on the right into an inverse limit and a direct limit on the *left* also into an *inverse* limit. A repeated limit can be made into a simple limit, so we can write:

$$D_\infty = \varprojlim \langle D_n, j_n \rangle_{n=0}^{\infty} \quad = \varinjlim \langle D_n, i_n \rangle_{n=0}^{\infty}$$

and

$$[D_\infty \to D_\infty] = \varprojlim \langle [D_n \to D_n], [j_n \to j_n] \rangle_{n=0}^{\infty} \quad = \varprojlim \langle D_{n+1}, j_{n+1} \rangle_{n=0}^{\infty} = D_\infty \quad \textit{(up to isomorphism).}$$

A full checking of the details involved would not make the argument appreciably simpler over the more "element-by-element" argument I have presented. In fact, the proofs are actually the same. But thinking of the result in terms of properties of functors does seem to isolate very well the essential idea and to show how simple it is.

One must only add here a note of caution: the proper choice of category must be done with care. Thus it seems to me that the use of projections rather than arbitrary continuous maps is necessary.

Inasmuch as I have not checked all details in this form, I hope what I say is correct.

2.5.20 Application ✓

Since we have shown $[D_\infty \to D_\infty]$ to be homeomorphic to D_∞, we can begin to regard them as the same. In particular there ought to be some function space structure to transfer to D_∞. This can be done by defining functional application for any elements $x, y \in D_\infty$ by the equation:

$$x(y) = \bigsqcup_{n=0}^{\infty} i_{n\infty}(x_{n+1}(y_n)).$$

Similarly we can define λ-abstraction on continuous expressions:

$$\lambda x.[\ldots x \ldots] = j_\infty(\lambda\{x : D_\infty\}.[\ldots x \ldots]),$$

and in this way D_∞ becomes a model for the λ-calculus of Church and Curry. The model-theoretic and proof-theoretic aspects of this result will be explained in another paper ([1]).

2.5.21 Two projections ✓

Suppose we were to start with the least, non-trivial lattice $0 = \top, \bot$ for D_0. Now $D_1 = [0 \to 0]$ has exactly three elements and there are just two ways of defining a projection $j_0 : D_1 \to D_0$. They are illustrated in the figure:

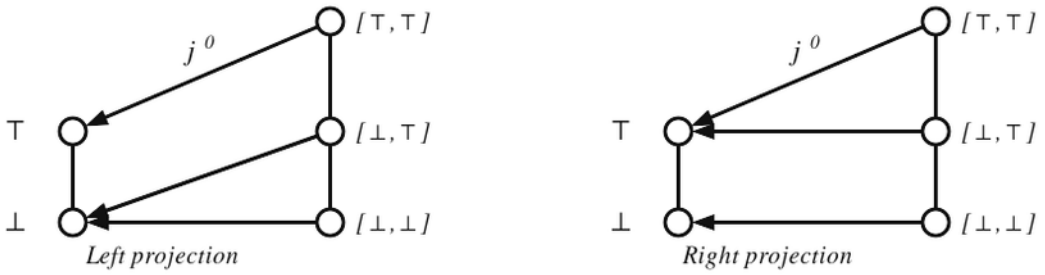


Figure 19: Two j0 projections; D0 on rows 2-3; arrows touch circles.

Hence our construction gives two limit spaces D_∞ and $D' * \infty$. Are they the same? No, they are not. It can be shown, for example, that the $\top$ of $D * \infty$ is *isolated* (that is, $\top \ll \top$), while the same is not true of $D' * \infty$. More interestingly, David Park has proved that the fixed-point operator fix mentioned in 3.14 has algebraic properties in $D * \infty$ quite different from those in $D' * \infty$. By algebraic here, we of course have reference to the functional algebra embodied in the application operation $x(y)$ defined on these limit spaces. Note, by the way, that in view of our isomorphism result we can regard fix (or any other similar continuous function for that matter) as an element of $D * \infty$. This makes the "algebra" of D_∞ quite a rich field for study.

2.5.22 Embedding every space ✓

The reader will have surely remarked that by virtue of 1.5, every T_0-space X whatsoever can be embedded as a subspace in a D_∞. Besides this all the continuous functions on X (oh, into D_∞, say) can be extended to D_∞; whence they can be regarded as *elements* of D_∞. Thus we have been able to embed not only the topology of X into D_∞ but also all of the continuous function theory over X. So far this is only a "logical" construction. For more interesting "mathematical" results we shall have to investigate whether any useful theorems about the usual function spaces $[X \to X]$ can be obtained with the aid of D_∞. This method can easily be employed for real- or complex-valued continuous functions, though it seems more oriented toward pointwise convergence than anything else. Still, there seems to be a chance it might be useful—especially if one wished to consider continuous *operators* on function spaces.

2.5.23 Other functors ✓

The idea of forming the limit space can also be applied to *other* functors besides $[D \to D]$. Thus instead of solving the "equation"

$$D = [D \to D]$$

as we have done with the D_∞ construction, we could also solve:

$$V = T + [V \times V] + [V + V] + [V \to V],$$

for example. Here $T = \bot, 0, 1, \top$ is the four-element lattice with 0 and 1 as incomparable elements. By $[V \times V]$ and $[V \to V]$ we understand the usual Cartesian product and function space construction. The $+$ operator, on the other hand works only in the category of lattices with $\top$ as an isolated element. It is defined so as to make:

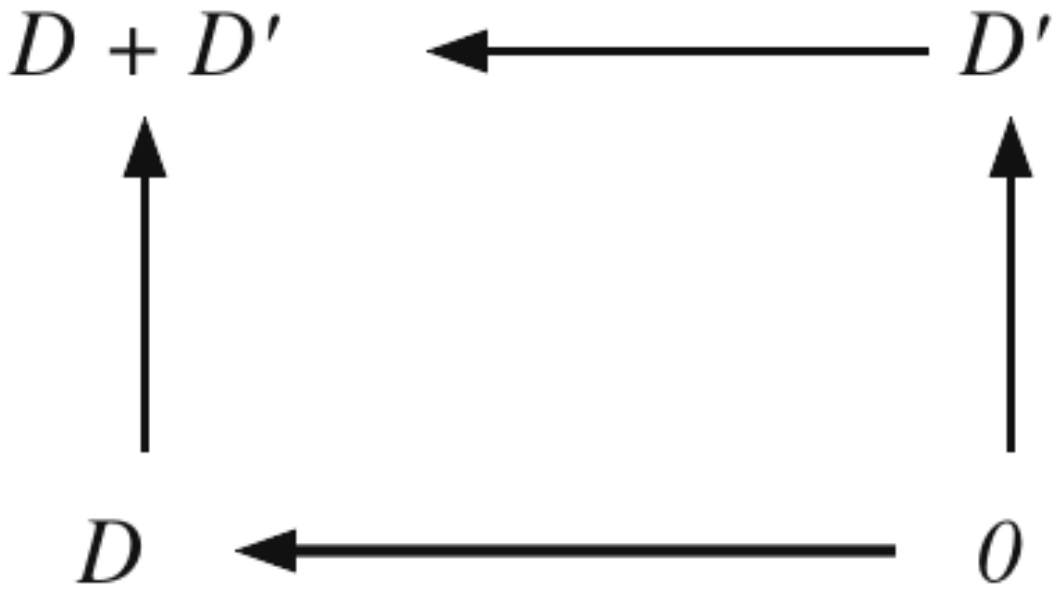


Figure 20: Push-out square: D and D prime map up into D plus D prime, and 0 maps left to D and up to D prime.

a push-out diagram, where the maps from 0 are meant to match $\bot$ with $\bot$ and $\top$ with $\top$. The point of requiring $\top$ to be isolated is that both D and D' become projections of $D + D'$. This construction, though not quite a disjoint union, has many properties in common with that operation on spaces. In particular, if we consider the category with projections as maps, the construction

$$\mathbb{F}(D) = T + [D \times D] + [D + D] + [D \to D]$$

is a *functor*. Furthermore, we can project $\mathbb{F}(T)$ onto T in an obvious way, thereby setting things up for an inverse limit construction:

$$V = \varprojlim \langle \mathbb{F}^n(T), j_n \rangle_{n=0}^{\infty}.$$

The resulting continuous lattice satisfies the desired equation up to isomorphism.

2.5.24 Subspaces of a limit ✓

The space V constructed in the way just indicated is very rich in subspaces. To see this, consider the space $\mathcal{J}_V$ of proper projections j where $j(\top) = \top$. As in 3.12 this is a complete lattice. Now that $[V \times V]$ and $[V + V]$ and $[V \to V]$ are regarded as *subspaces* of this "universe" V itself, we can easily define *continuous operations*

$$(j \times k)\ ,\ (j + k)\ ,\ \text{and}\ (j \to k)$$

on the projections obtaining again elements of $\mathcal{J}_V$. The projections so obtained correspond to the indicated constructions of subspaces, of course. (Indeed, if we had the time and space, we could show that $\mathcal{J}_V$ becomes a very interesting category). There will be a particular

projection t corresponding to T, and reason for doing all this is to show that the existence of subspaces of V can now be established by solving equations in $\mathcal{J}_V$. For example, by the fixed-point construction we could find a $j \in \mathcal{J}_V$ such that

$$j = t + (t \times j) + ((j \times j) \to j).$$

The range of j would then be a subspace $W \subseteq V$ such that W solves the equation:

$$W = T + [T \times W] + [[W \times W] \to W].$$

And these are only a few examples: simultaneous equations are possible, and many other operators are waiting for discovery and application.

2.6 Correction ✓

Correction (Added March, 1972). Robin Milner has pointed out to me that there is an error in the remark in the paragraph immediately preceding Proposition 2.5. I was mistaken in saying that if D is a T_0-space which becomes a complete lattice under its induced partial ordering, then every set open in the given topology is also open in the induced topology. There are many counterexamples to this statement. Let D be any complete lattice. There are two extreme T_0-topologies which will induce the given partial ordering. The smallest such topology has as a sub-base for its open sets those sets of the form:

$$\{x \in D : x \not\sqsubseteq y\}.$$

These sets are easily proved to be open in any T_0-topology which induces the partial ordering. At the other extreme consider sets of the form:

$$\{x \in D : y \sqsubseteq x.\}$$

Such sets will give a base for a T_0-topology that is the maximal topology inducing the given partial ordering. Clearly they need not be open in the induced lattice topology; in particular, they may well fail

to satisfy conditions (ii) on open sets. To make the remark in question correct, we must thus suppose that the given T_0-topology is *contained within* the induced lattice topology. The equation given in the paragraph indicated will then be a sufficient condition for the two topologies to be identical.

The remark was employed in the proof of three different propositions: 2.9, 2.10, and 3.3. In the case of 2.9 one must verify that the product topology is contained within the lattice topology. This need only be done for the basis for the product topology, and for such basic open sets the result needed is obvious. In the case of proposition 2.10 the question concerns a relationship between the topologies of a space and a subspace; the spaces in question are also lattices. Note in passing that a lub in the subspace is generally *larger* in the partial ordering than the corresponding lub relative to the whole space. This puts the inequalities in the wrong direction, and so it is not immediate that a relativized open set for the subspace is open in the lattice topology of the subspace. However, in this case we can appeal to the continuous retraction. Recall that the relativized open sets of the kind that we used in the proof of 2.10 are of the form:

$$U = \{z \in D : x < z\} .$$

Suppose then that S is a directed set, and that using the lub in the sense of D we have

$$\bigsqcup S \in U .$$

Referring back to the proof of 2.10 we know that ($S' = S \subseteq D$, but $\bigsqcup S'$ is the join computed in the ambient lattice D')

$$j(\bigsqcup S') = \bigsqcup S ,$$

which means that

$$x < j(\bigsqcup S') .$$

From this it follows that

$$x < j(z), \text{ some } z \in S .$$

Now $j(z) = z$, and we have what we need. This argument suffices only for a special type of open sets; but these open sets form a base for the topology, and so the argument is quite general.

Turning now to the proof of theorem 2.3 we note that the topology on the function space is simply the *relativized* product topology. There is no difficulty with lubs in this case, because, as we showed in the proof of that theorem, all lubs are calculated pointwise. Thus, it is easy to verify now that the sets open in the product topology are also open in the lattice topology.

3 Datatypes as Lattices (1976) ✓

Dependencies. Each box is one section. Lean ownership is by topic folder. The arrowhead is on the section that imports; the number is how many direct imports. Only paper-forward edges are shown (a later section importing an earlier one). An arrow already implied by a longer path is left out.

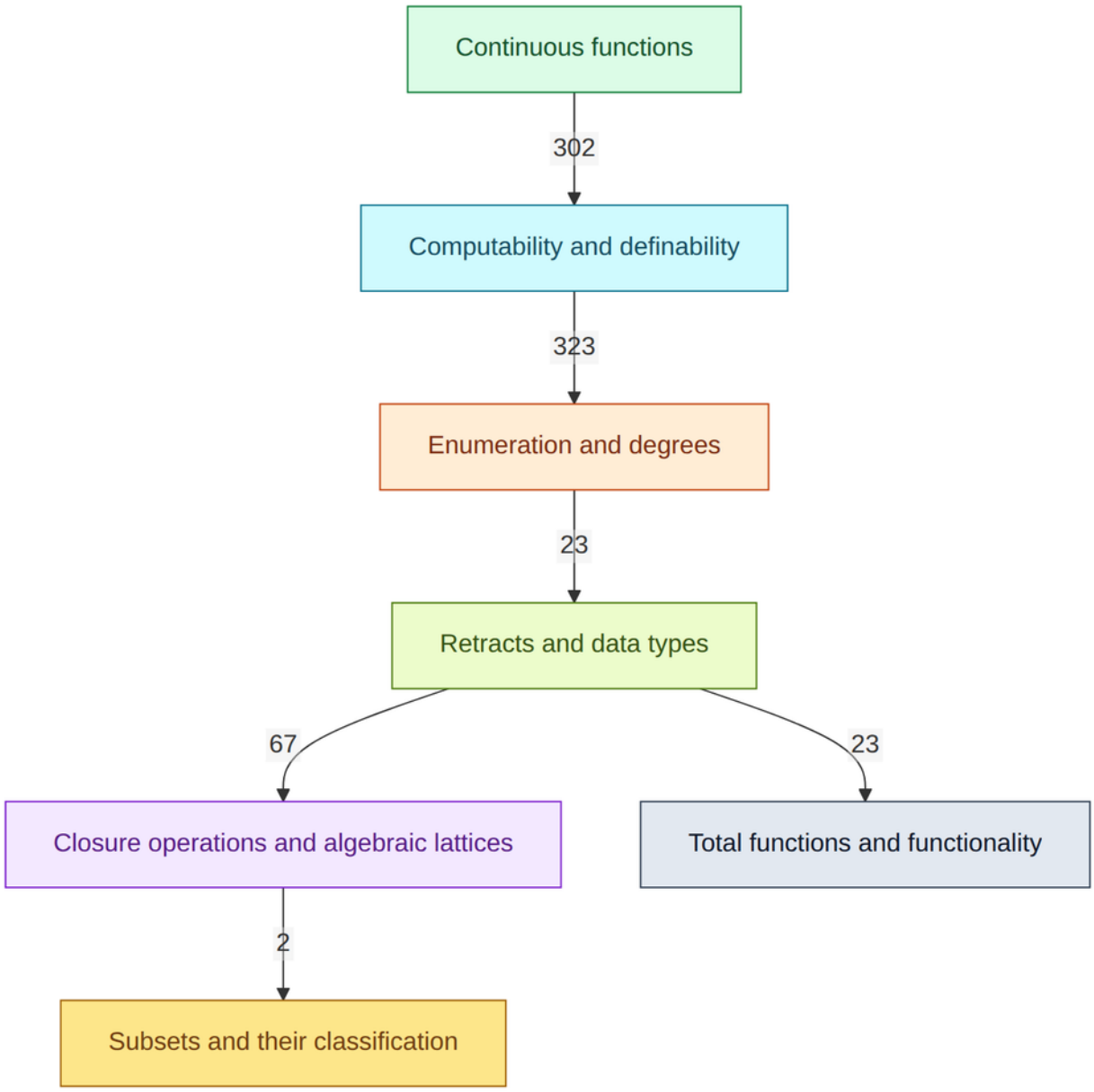


Figure 21: Datatypes as Lattices (1976) ✓.

3.1 Introduction

To the Memory of Christopher Strachey, 1916–1975

Abstract. The meaning of many kinds of expressions in programming languages can be taken as elements of certain spaces of "partial" objects. In this report these spaces are modeled in one universal domain $\mathbf{P}\omega$, the set of all subsets of the integers. This domain renders the connection of this semantic theory with the ordinary theory of number theoretic (especially general recursive) functions clear and straightforward.

Key words. programming language semantics, lattice, continuous lattice, algebraic lattice, computability, retract, combinatory logic, lambda calculus, recursion theorem, enumeration degrees, continuous function, fixed-point theorem

Introduction. Investigations begun in 1969 with Christopher Strachey led to the idea that the denotations of many kinds of expressions in programming languages could be taken as elements of certain kinds of spaces of "partial" objects. As these spaces could be treated as function spaces, their structure at first seemed excessively complicated—even impossible. But then the author discovered that there were many more spaces than we had first imagined—even wanted. They could be presented as lattices (or as some prefer, semilattices), and the main technique was to employ topological ideas, in particular the notion of a continuous function. This approach and its applications have been presented in a number of publications, but that part of the foundation concerned with *computability* (in the sense of recursion theory) was never before adequately exposed. The purpose of the present paper is to provide such a foundation and to simplify the whole presentation by a de-emphasis of abstract ideas. An Appendix and the references provide a partial guide to the literature and an indication of connections with other work.

3.1.1 Domain $\mathbf{P}\omega$ ✓

The main innovation in this report is to model everything within one "universal" domain $\mathbf{P}\omega = \{x \mid x \subseteq \omega\}$, the domain of all subsets of the set ω of nonnegative integers. The advantages are many: by the most elementary considerations $\mathbf{P}\omega$ is recognized to be a lattice and a topological space. In fact, $\mathbf{P}\omega$ is a *continuous lattice*, even an *algebraic lattice*, but in the beginning we do not even need to define such an "advanced" concept; we can save these ideas for an *analysis* of what has been done here in a more direct way. Next by taking the set of integers as the basis of the construction, the connection with the ordinary theory of number-theoretic (especially, general recursive) functions can be made clear and straightforward.

Theorem (`topElem.pos`). Some positive numeral belongs to $\top$.

Proof. $1 \in \top$. ■

`Pomega.lean`

```
module -- shake: keep-all

public import Mathlib.Data.Nat.Notation
public import Mathlib.Data.Set.Defs
public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Insert

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

abbrev Pomega := Set ℕ

def botElem : Pomega := ∅

def topElem : Pomega := Set.univ
```

```
@[simp] theorem mem_botElem (n : ℕ) : n ∈ botElem ↔ False := by simp [botElem]

@[simp] theorem mem_topElem (n : ℕ) : n ∈ topElem := Set.mem_univ n

namespace topElem

theorem pos : ∃ n, n + 1 ∈ topElem :=
  ⟨0, Set.mem_univ 1⟩

end topElem

end DomainTheory.DataTypesAsLattices
```

3.1.2 Multiple-valued integers ✓

The model $\mathbf{P}\omega$ can be intuitively viewed as the domain of *multiple-valued* integers; what is new in the presentation is that functions are not only multiple-valued but also "multiple-argumented". This remark is given a precise sense in § 2 below, but the upshot of the idea is that multiple-valued integers are regarded as objects in themselves—possibly infinite—and as something more than just the collection of single integers contained in them. This combination of the finite with the infinite into a single domain, together with the idea that a continuous function can be reduced to its *graph* (in the end, a set of integers), makes it possible to view an $x \in \mathbf{P}\omega$ at one time as a value, at another as an argument, then as an integer, then as a function, and still later as a functional (or combinator). The "paradox" of self-application (as in $x(x)$) is solved by allowing the *same* x to be used in two *different* ways. This is done in ordinary recursion theory via Gödel numbers (as in $\{e\}(e)$), but the advantage of the present theory is that not only is the function concept the *extensional* one, but it includes *arbitrary* continuous functions and not just the computable ones.

Section 1 introduces the elementary ideas on the topology of $\mathbf{P}\omega$ and the continuous functions including the fixed-point theorem. Section 2 has to do with computability and definability. The language LAMBDA is introduced as an extension of the pure λ-calculus by four arithmetical combinators; in fact, it is indicated in § 3 how the whole system could be based on *one* combinator. What is shown is that computability in $\mathbf{P}\omega$ according to the natural definition (which assumes that we already know what a recursively enumerable set of integers is) is equivalent to LAMBDA-definability. The main tool is, not surprisingly, the first recursion theorem formulated with the aid of the so-called *paradoxical combinator* $\mathbf{Y}$. The plan is hardly original, but the point is to work out what it all means in the model.

Along the way we have to show how to give every λ-term a denotation in $\mathbf{P}\omega$; the resulting principles of λ-calculus that are thereby verified are summarized in Table 1. Of these the first three, (α), (β), and (ξ), are indeed valid in the model; however, rule (η), which is a stronger version of extensionality, *fails* in the $\mathbf{P}\omega$ model. This should not be regarded as a disadvantage since the import of (η) is to suppose every object is a function. A quick construction of these special models is indicated at the end of § 5. Since $\mathbf{P}\omega$ is partially ordered by $\subseteq$, there are also laws involving this relation. Law (ξ^*) is an improvement of (ξ); while (μ) is a form of monotonicity for application.

TABLE 1

Some laws of λ-calculus

(α)	$\lambda x.\tau = \lambda y.\tau[y/x]$
(β)	$(\lambda x.\tau)(y) = \tau[y/x]$
(ξ)	$\lambda x.\tau = \lambda x.\sigma \quad \text{if} \quad \forall x.\, \tau = \sigma$

(η)	$y = \lambda x.y(x)$
(ξ^*)	$\lambda x.\tau \subseteq \lambda x.\sigma \quad$ if $\quad \forall x.\,\tau \subseteq \sigma$
(μ)	$x \subseteq y$ and $u \subseteq v$ imply $u(x) \subseteq v(y)$

Section 3 has to do with enumeration and degrees. Gödel numbers for LAMBDA are defined in a very easy way which takes advantage of the notation of combinators. This leads to the second recursion theorem, and results on incompleteness and undecidability follow along standard lines. *Relative recursiveness* is also very easy to define in the system, and we make the tie-in with *enumeration degrees* which correspond to finitely generated combinatory subalgebras of $\mathbf{P}\omega$. Finally a theorem of Myhill and Shepherdson is interpreted as a most satisfactory completeness property for definability in the system.

Sections 4 and 5 show how a calculus of *retracts* leads to quite simple definitions of a host of useful domains (as lattices). Section 6 investigates the classification of other subsets (*nonlattices*) of $\mathbf{P}\omega$; while § 7 contrasts partial (multiple-valued) functions with total functions, and interprets various theories of functionality. Connections with category theory are mentioned.

What is demonstrated in this work is how the language LAMBDA, together with its interpretation in $\mathbf{P}\omega$, is an extremely convenient vehicle for *definitions* of computable functions on complex structures (all taken as subdomains of $\mathbf{P}\omega$). It is a "high-level" programming language for recursion theory. It is *applied* combinatory logic, which in usefulness goes far beyond anything envisioned in the standard literature. What has been shown is how many interesting predicates can be *expressed* as equations between continuous functions. What is needed next is a development of the *proof theory* of the system along the lines of the work of Milner, which incorporates the author's extension of McCarthy's rule of recursion induction to this high-level language. Then we will have a flexible and practical "mathematical" theory of computation.

3.2 Continuous functions

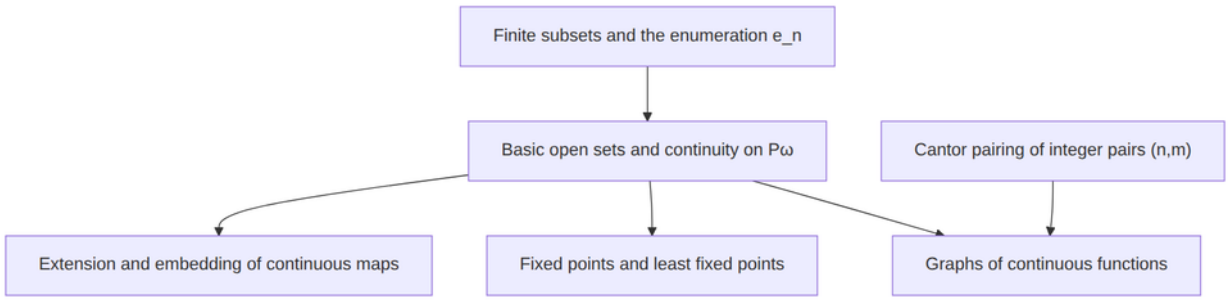


Figure 22: Continuous functions.

3.2.1 Introduction ✓

1. Continuous functions. The domain $\mathbf{P}\omega$ of all subsets of the set ω of nonnegative integers is a complete lattice under the partial ordering $\subseteq$ of set inclusion, as is well known. We use the usual symbols $\cup$, $\cap$, $\bigcup$, $\bigcap$ for the finite and infinite lattice operations of union and intersection. $\mathbf{P}\omega$ is of course also a Boolean algebra; and for complements we write

$$\sim x = \{n \mid n \notin x\}$$

where it is understood that such variables as i, j, k, l, m, n range over *integers in* ω, while u, v, w, x, y, z range over *subsets of* ω.

The domain $\mathbf{P}\omega$ can also be made into a topological space—in many ways. A common method is to match each $x \subseteq \omega$ with the corresponding characteristic function in $\{0,1\}^\omega$, and to take the induced product topology. In this way $\mathbf{P}\omega$ is a totally disconnected compact Hausdorff space homeomorphic to the Cantor "middle-third" set. This is *not* the topology we want; it is a *positive-and-negative* topology which makes the function $\sim x$ continuous. We want a weaker topology: the topology of positive "information", which has the advantage that all continuous functions possess fixed points. (The equation $x = \sim x$ is impossible.) The topology that we *do* want is exactly that appropriate to considering $\mathbf{P}\omega$ to be a continuous lattice. But all this terminology of abstract mathematics is quite unnecessary, since the required definitions can be given in very elementary terms.

For Section 1. If we give the two-point space $\{\bot, \top\}$ the weak T_0-topology with just three open sets: $\varnothing$, $\{\top\}$, $\{\bot, \top\}$, we have what is called the Sierpinski space; and its infinite product $\{\bot, \top\}^\omega$ with the product topology is the same as $\mathbf{P}\omega$. The finite sets $e_n \in \mathbf{P}\omega$ correspond exactly to the usual basic open sets for the product. For those familiar with such notions, this well-known observation makes many of the facts mentioned in this section fairly obvious. From any point of view, Theorem 1.1 and the remarks in the following paragraph are simple exercises.

3.2.2 Finite subsets and the enumeration e_n ✓

e.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.Pomega
public import Mathlib.Algebra.Group.Nat.Defs
import Mathlib.Algebra.GroupWithZero.Nat
public import Mathlib.Basic.Finite.Defs
import Mathlib.Data.Set.Finite.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def e (n : ℕ) : Pomega := {k | n.testBit k}

namespace e

private theorem pow2_eq (k : ℕ) : e (2 ^ k) = ({k} : Pomega) := by
  ext i
  simp [_root_.DomainTheory.DataTypesAsLattices.e, Nat.testBit_two_pow]

@[simp] theorem mem {n k : ℕ} : k ∈ e n ↔ n.testBit k := Iff.rfl

theorem zero : e 0 = ∅ := by
  ext k
  simp [_root_.DomainTheory.DataTypesAsLattices.e]

theorem pow2 (k : ℕ) : e (2 ^ k) = ({k} : Pomega) := pow2_eq k

theorem or (n m : ℕ) : e (n ||| m) = e n ∪ e m := by
```

```
  ext k
  simp [_root_.DomainTheory.DataTypesAsLattices.e, Nat.testBit_or]

theorem or_of_subset {n m : ℕ} {x : Pomega} (hn : e n ⊆ x) (hm : e m ⊆ x) : e (n ||| m
    ) ⊆ x := by
  rw [or]
  exact Set.union_subset hn hm

theorem le_two_pow (k : ℕ) : k ≤ 2 ^ k := by
  induction k with
  | zero => decide
  | succ k ih =>
    calc
      k + 1 ≤ 2 ^ k + 1 := Nat.succ_le_succ ih
      _ ≤ 2 ^ k + 2 ^ k := Nat.add_le_add_left Nat.one_le_two_pow _
      _ = 2 ^ k * 2 := (Nat.mul_two (2 ^ k)).symm
      _ = 2 ^ (k + 1) := (Nat.pow_succ 2 k).symm

theorem two_pow_le_of_testBit {n k : ℕ} (h : n.testBit k) : 2 ^ k ≤ n := by
  have hmod : (n >>> k) % 2 = 1 := by simpa [Nat.testBit] using h
  have hpos : 0 < n >>> k := Nat.pos_of_ne_zero fun hz => by simp [hz] at hmod
  have hdiv : 1 ≤ n / 2 ^ k := by
    rwa [Nat.shiftRight_eq_div_pow] at hpos
  simpa using (Nat.le_div_iff_mul_le (Nat.two_pow_pos k)).mp hdiv

theorem lt_two_pow' (k : ℕ) : k < 2 ^ k := by
  induction k with
  | zero => decide
  | succ k ih =>
    calc
      k + 1 ≤ 2 ^ k := Nat.succ_le_of_lt ih
      _ < 2 ^ k + 2 ^ k := Nat.lt_add_of_pos_right (Nat.two_pow_pos k)
      _ = 2 ^ (k + 1) := by rw [← Nat.two_mul, Nat.mul_comm, Nat.pow_succ]

theorem mem_le {n k : ℕ} (h : k ∈ e n) : k ≤ n :=
  (le_two_pow k).trans (two_pow_le_of_testBit h)

theorem mem_lt {n k : ℕ} (h : k ∈ e n) : k < n := by
  have hle := mem_le h
  rcases eq_or_lt_of_le hle with rfl | hlt
  · exact (lt_irrefl k ((lt_two_pow' k).trans_le (two_pow_le_of_testBit h))).elim
  · exact hlt

theorem finite (n : ℕ) : (_root_.DomainTheory.DataTypesAsLattices.e n : Pomega).Finite
    :=
  (Finset.range (n + 1)).finite_toSet.subset fun _k hk =>
    Finset.mem_range.mpr (Nat.lt_succ_of_le (mem_le hk))

end e

end DomainTheory.DataTypesAsLattices
```

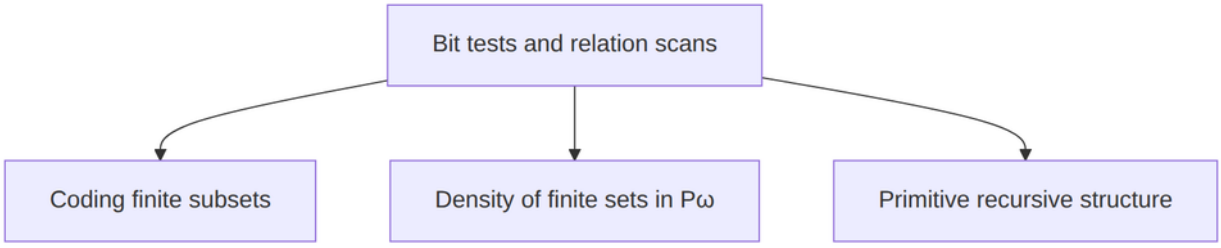


Figure 23: Finite subsets and the enumeration e_n ✓.

Definition of e_n ✓ To make the topology "visible", we introduce a standard enumeration $\{e_n \mid n \in \omega\}$ of all *finite* subsets of ω. Specifically we set

$$e_n = \{k_0, k_1, \cdots, k_{m-1}\},$$

provided that $k_0 < k_1 < \cdots < k_{m-1}$ and $n = \sum_{i<m} 2^{k_i}$. Thus n is the code number for e_n, and the elements of e_n are the exponents in the binary expansion of the integer n.

Definition. For $n \in \omega$, the finite set $e_n \in \mathbf{P}\omega$ is

$$e_n = \{k \in \omega \mid n.\mathrm{testBit}(k)\}.$$

Equivalently, $k \in e_n$ iff the binary expansion of n has a 1 in bit k.

Lemma (`e.mem`). For all $n, k \in \omega$, $k \in e_n \leftrightarrow n.\mathrm{testBit}(k)$.

Proof. This is the definition of membership in $\{k \mid n.\mathrm{testBit}(k)\}$. ■

Theorem (`e.zero`). $e_0 = \emptyset$.

Proof. A natural number has no 1-bits iff it is zero, so membership in e_0 is impossible. Extensionality gives $e_0 = \emptyset$. ■

Theorem (`e.pow2`). For each $k \in \omega$, $e_{2^k} = \{k\}$.

Proof. By `Nat.testBit_two_pow`, 2^k has a 1 in bit i exactly when $i = k$; that is the membership characterization of the singleton $\{k\}$. ■

Theorem (`e.or`). For all $n, m \in \omega$, $e_{n|m} = e_n \cup e_m$, where $|$ is bitwise OR on ω.

Proof. Bit k is set in $n \,|\, m$ iff it is set in n or in m, which is exactly membership in the union $e_n \cup e_m$. ■

Lemma (`e.or_of_subset`). If $e_n \subseteq x$ and $e_m \subseteq x$ for some $x \in \mathbf{P}\omega$, then $e_{n|m} \subseteq x$.

Proof. By `e.or`, $e_{n|m} = e_n \cup e_m$; two set inclusions into x imply the union is contained in x. ■

This is a one-to-one enumeration of finite subsets, where $k \in e_n$ always implies $k < n$,

Lemma (`e.le_two_pow`). For every $k \in \omega$, $k \leq 2^k$.

Proof. Induction on k. For $k = 0$, both sides are 1. For the step $k \to k + 1$, the induction hypothesis gives $k \leq 2^k$, hence $k + 1 \leq 2^k + 1 \leq 2^k + 2^k = 2^{k+1}$. ■

Lemma (`e.two_pow_le_of_testBit`). If $n.\mathrm{testBit}(k)$ is true, then $2^k \leq n$.

Proof. The hypothesis means $(n \gg k) \bmod 2 = 1$, so $n \gg k > 0$. Writing division by 2^k as a right shift, $1 \leq n/2^k$; equivalently $2^k \leq n$. ■

Lemma (`e.lt_two_pow'`). For every $k \in \omega$, $k < 2^k$.

Proof. Induction on k. For $k = 0$, $0 < 1$. For the step, $k + 1 \leq 2^k$ by the induction hypothesis, and $2^k < 2^k + 2^k = 2^{k+1}$ because $2^k > 0$. ■

Lemma (`e.mem_le`). If $k \in e_n$, then $k \leq n$.

Proof. Membership $k \in e_n$ is $n.\mathrm{testBit}(k)$. By `e.two_pow_le_of_testBit`, $2^k \le n$; by `e.le_two_pow`, $k \le 2^k$. Transitivity gives $k \le n$. ■

Theorem (`e.mem_lt`). If $k \in e_n$, then $k < n$.

Proof. By `e.mem_le`, $k \le n$. If $k = n$, then `e.two_pow_le_of_testBit` would give $2^k \le n = k$, while `e.lt_two_pow'` gives $k < 2^k$, a contradiction. So $k \neq n$ and therefore $k < n$. ■

This is the formal content of Scott's clause that $k \in e_n$ always implies $k < n$.

Theorem (`e.finite`). Every set e_n is finite.

Proof. If $k \in e_n$, then `e.mem_le` gives $k \le n$, so $e_n \subseteq \{0, \dots, n\}$. That interval is finite, hence so is e_n. ■

Bit tests and relation scans ✓ The relations Scott treats as primitive recursive are evaluated by scanning binary expansions. This note supplies primitive-recursive bit access and the Boolean procedures that decide membership, inclusion, and union-coding for e_n.

Theorem (`primrec_twoPow`). The function $k \mapsto 2^k$ is primitive recursive.

Proof. Primitive recursion on k with constant 1 and step doubling the accumulator. ■

Theorem (`primrec_testBit`). The function $(n, k) \mapsto n.\mathrm{testBit}(k)$ is primitive recursive.

Proof. Write $\mathrm{testBit}(n, k)$ as parity of $\lfloor n/2^k \rfloor$, using `primrec_twoPow` and division/modulo. ■

Lemma (`not_testBit_of_ge`). If $n \le k$, then $n.\mathrm{testBit}(k) = \mathrm{false}$.

Proof. A true bit would place $k \in e_n$, contradicting `e.mem_lt` with $k \not< n$. ■

Definition (`e.subsetBool`). `e.subsetBool n m` iteratively verifies every 1-bit of n is a 1-bit of m.

Lemma (`e.subsetBool_iff`). `e.subsetBool n m = true` iff $n.\mathrm{testBit}(k) \Rightarrow m.\mathrm{testBit}(k)$ for every k.

Proof. Induction on the scan fuel; bits above n are false by `not_testBit_of_ge`. ■

Theorem (`primrec_e_subsetBool`). `e.subsetBool` is primitive recursive in (n, m).

Proof. A primitive recursive loop on the bit index using `primrec_testBit`. ■

Definition (`e.eqUnionBool`). `e.eqUnionBool n m k` checks $n.\mathrm{testBit}(i) = m.\mathrm{testBit}(i) \vee k.\mathrm{testBit}(i)$ for $i \le n + m + k$.

Theorem (`e.eqUnionBool_iff`). `e.eqUnionBool n m k = true` iff $e_n = e_m \cup e_k$.

Proof. Extensionality on membership, with trivial bits beyond the scan bound. ■

Theorem (`primrec_e_eqUnionBool`). `e.eqUnionBool` is primitive recursive in (n, m, k).

Proof. Same iterator pattern as `e.subsetBool`. ■

`eScan.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Computability.Primrec.Basic

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem primrec_twoPow : Primrec fun k : ℕ => 2 ^ k :=
  (Primrec.nat_rec' (f := id) (g := fun _ => (1 : ℕ))
    (h := fun (_ : ℕ) (p : ℕ × ℕ) => p.2 * 2)
```

```
    Primrec.id (Primrec.const 1)
    (show Primrec fun q : ℕ × ℕ × ℕ => q.2.2 * 2 from
      Primrec.nat_mul.comp (Primrec.snd.comp Primrec.snd) (Primrec.const 2))).of_eq
    fun k => by
      induction k with
      | zero => simp
      | succ k ih =>
        change (Nat.rec 1 (fun n IH => (n, IH).2 * 2) k) * 2 = 2 ^ (k + 1)
        have ih' : Nat.rec 1 (fun n IH => (n, IH).2 * 2) k = 2 ^ k := by
          simpa [id] using ih
        rw [ih', Nat.pow_succ, Nat.mul_comm]

theorem primrec_testBit : Primrec₂ fun n k : ℕ => n.testBit k :=
  (Primrec.eq.decide.comp
    (Primrec.nat_mod.comp
      (Primrec.nat_div.comp Primrec.fst (primrec_twoPow.comp Primrec.snd))
      (Primrec.const 2))
    (Primrec.const 1)).of_eq fun _p =>
      Nat.testBit_eq_decide_div_mod_eq.symm

theorem not_testBit_of_ge {n k : ℕ} (h : n ≤ k) : n.testBit k = false := by
  cases ht : n.testBit k with
  | false => rfl
  | true => exact (Nat.not_le_of_gt (e.mem_lt (e.mem.mpr ht)) h).elim

theorem primrecRel_testBit_true : PrimrecRel fun (n k : ℕ) => n.testBit k = true :=
  Primrec.eq.comp primrec_testBit (Primrec.const true)

namespace e

private abbrev Enc := _root_.DomainTheory.DataTypesAsLattices.e

private theorem mem_iff {n k : ℕ} : k ∈ Enc n ↔ n.testBit k := by
  rw [Enc, mem]

private def subsetBoolAux (n m fuel : ℕ) : Bool :=
  fuel.rec true fun k ih => ih && (!(n.testBit k) || m.testBit k)

def subsetBool (n m : ℕ) : Bool := subsetBoolAux n m n

private theorem subsetBoolAux_succ (n m fuel : ℕ) :
    subsetBoolAux n m (fuel + 1) =
      (subsetBoolAux n m fuel && (!(n.testBit fuel) || m.testBit fuel)) :=
  rfl

private theorem subsetBoolAux_iff (n m fuel : ℕ) :
    subsetBoolAux n m fuel = true ↔
      ∀ k < fuel, n.testBit k = true → m.testBit k = true := by
  induction fuel with
  | zero => simp [subsetBoolAux]
  | succ fuel ih =>
    rw [subsetBoolAux_succ]
    simp only [Bool.and_eq_true, ih]
    constructor
    · rintro ⟨h, hb⟩ k hk
      rcases Nat.lt_or_eq_of_le (Nat.le_of_lt_succ hk) with hlt | rfl
```

```
      · exact h k hlt
      · intro hk'
        simp [hk'] at hb
        exact hb
    · intro h
      refine ⟨fun k hk => h k (Nat.lt_succ_of_lt hk), ?_⟩
      by_cases hk : n.testBit fuel = true
      · simp [h fuel (Nat.lt_succ_self fuel) hk]
      · simp [hk]

theorem subsetBool_iff (n m : ℕ) :
    subsetBool n m = true ↔ ∀ k, n.testBit k = true → m.testBit k = true := by
  rw [subsetBool, subsetBoolAux_iff]
  constructor
  · intro h k hk
    by_cases hkn : k < n
    · exact h k hkn hk
    · exact absurd hk (by simp [not_testBit_of_ge (Nat.le_of_not_gt hkn)])
  · intro h k _ hk
    exact h k hk

private def eqUnionBoolAux (n m k fuel : ℕ) : Bool :=
  fuel.rec true fun i ih => ih && (n.testBit i == (m.testBit i || k.testBit i))

def eqUnionBool (n m k : ℕ) : Bool := eqUnionBoolAux n m k (n + m + k + 1)

private theorem eqUnionBoolAux_succ (n m k fuel : ℕ) :
    eqUnionBoolAux n m k (fuel + 1) =
      (eqUnionBoolAux n m k fuel && (n.testBit fuel == (m.testBit fuel || k.testBit
    fuel))) :=
  rfl

private theorem eqUnionBoolAux_iff (n m k fuel : ℕ) :
    eqUnionBoolAux n m k fuel = true ↔
      ∀ i < fuel, n.testBit i = (m.testBit i || k.testBit i) := by
  induction fuel with
  | zero => simp [eqUnionBoolAux]
  | succ fuel ih =>
    rw [eqUnionBoolAux_succ]
    simp only [Bool.and_eq_true, ih]
    constructor
    · rintro ⟨h, hb⟩ i hi
      rcases Nat.lt_or_eq_of_le (Nat.le_of_lt_succ hi) with hlt | rfl
      · exact h i hlt
      · simpa [beq_iff_eq] using hb
    · intro h
      refine ⟨fun i hi => h i (Nat.lt_succ_of_lt hi), ?_⟩
      simpa [beq_iff_eq] using h fuel (Nat.lt_succ_self fuel)

theorem eqUnionBool_iff (n m k : ℕ) :
    eqUnionBool n m k = true ↔ Enc n = Enc m ∪ Enc k := by
  rw [eqUnionBool, eqUnionBoolAux_iff]
  constructor
  · intro h
    ext i
    rw [mem_iff, Set.mem_union, mem_iff (n := m), mem_iff (n := k)]
```

```
    by_cases hi : i < n + m + k + 1
    · rw [h i hi]
      simp
    · have hle : n + m + k + 1 ≤ i := Nat.le_of_not_gt (by simpa [gt_iff_lt] using hi)
      have hn_le : n ≤ n + m + k + 1 := by simp [Nat.add_assoc]
      have hm_le : m ≤ n + m + k + 1 :=
        Nat.le_trans (Nat.le_add_left m n) (Nat.le_add_right (n + m) (k + 1))
      have hk_le : k ≤ n + m + k + 1 := by
        calc
          k ≤ n + m + 1 + k := Nat.le_add_left k (n + m + 1)
          _ = n + m + k + 1 := by
            rw [Nat.add_assoc (n + m) 1 k, Nat.add_comm 1 k, ← Nat.add_assoc (n + m) k
     1]
      have hn := not_testBit_of_ge (Nat.le_trans hn_le hle)
      have hm := not_testBit_of_ge (Nat.le_trans hm_le hle)
      have hk := not_testBit_of_ge (Nat.le_trans hk_le hle)
      simp [hn, hm, hk]
  · intro h i _
    have hmem : n.testBit i = true ↔ m.testBit i = true ∨ k.testBit i = true := by
      rw [← mem_iff, h, Set.mem_union, ← mem_iff (n := m), ← mem_iff (n := k)]
    cases hn : n.testBit i <;> cases hm : m.testBit i <;> cases hk : k.testBit i <;>
      simp [hn, hm, hk] at hmem ⊢

end e

theorem eSubsetBool_iff (n m : ℕ) :
    e.subsetBool n m = true ↔ e n ⊆ e m := by
  simp [Set.subset_def, e.mem, e.subsetBool_iff]

theorem eEqUnionBool_iff (n m k : ℕ) :
    e.eqUnionBool n m k = true ↔ e n = e m ∪ e k :=
  e.eqUnionBool_iff n m k

theorem primrec_e_subsetBool : Primrec₂ e.subsetBool := by
  have hstep : Primrec₂ fun (p : ℕ × ℕ) (nih : ℕ × Bool) =>
      nih.2 && (!(p.1.testBit nih.1) || p.2.testBit nih.1) :=
    Primrec.and.comp (Primrec.snd.comp Primrec.snd)
      (Primrec.or.comp
        (Primrec.not.comp
          (primrec_testBit.comp (Primrec.fst.comp Primrec.fst) (Primrec.fst.comp
    Primrec.snd)))
        (primrec_testBit.comp (Primrec.snd.comp Primrec.fst) (Primrec.fst.comp Primrec
    .snd)))
  exact (Primrec.nat_rec' Primrec.fst (Primrec.const true) hstep).of_eq fun _ => rfl

theorem primrec_e_eqUnionBool : Primrec fun t : ℕ × ℕ × ℕ =>
    e.eqUnionBool t.1 t.2.1 t.2.2 := by
  have hbound : Primrec fun t : ℕ × ℕ × ℕ => t.1 + t.2.1 + t.2.2 + 1 :=
    Primrec.succ.comp
      (Primrec.nat_add.comp
        (Primrec.nat_add.comp Primrec.fst (Primrec.fst.comp Primrec.snd))
        (Primrec.snd.comp Primrec.snd))
  have hstep : Primrec₂ fun (p : ℕ × ℕ × ℕ) (nih : ℕ × Bool) =>
      nih.2 && (p.1.testBit nih.1 == (p.2.1.testBit nih.1 || p.2.2.testBit nih.1)) :=
    Primrec.and.comp (Primrec.snd.comp Primrec.snd)
      (Primrec.beq.comp
```

```
        (primrec_testBit.comp (Primrec.fst.comp Primrec.fst) (Primrec.fst.comp Primrec
    .snd))
        (Primrec.or.comp
          (primrec_testBit.comp
            (Primrec.fst.comp (Primrec.snd.comp Primrec.fst))
            (Primrec.fst.comp Primrec.snd))
          (primrec_testBit.comp
            (Primrec.snd.comp (Primrec.snd.comp Primrec.fst))
            (Primrec.fst.comp Primrec.snd))))
  exact (Primrec.nat_rec' hbound (Primrec.const true) hstep).of_eq fun _ => rfl

end DomainTheory.DataTypesAsLattices
```

Primitive recursive structure ✓ The function $\max(e_n)$ is (primitive) recursive in n, and the relations $k \in e_n$, $e_n \subseteq e_m$, $e_n = e_m \cup e_k$ are all (primitive) recursive in k, m, n.

Theorem (`primrecRel_mem_e`). The relation $(n, k) \mapsto k \in e_n$ is primitive recursive.

Proof. By `e.mem`, membership is $n.\mathrm{testBit}(k)$; apply `primrec_testBit`. ■

Definition (`eMax`). `eMax n` is the greatest $k \le n$ with $n.\mathrm{testBit}(k) = \mathrm{true}$, or 0 if none.

Theorem (`primrec_eMax`). The function `eMax` is primitive recursive.

Proof. `Nat.findGreatest` on the primitive-recursive predicate $n.\mathrm{testBit}(k) = \mathrm{true}$. ■

Theorem (`primrecRel_e_subset`). The relation $(n, m) \mapsto e_n \subseteq e_m$ is primitive recursive.

Proof. By `e.subsetBool_iff`, inclusion equals `e.subsetBool n m = true`; apply `primrec_e_subsetBool`. ■

Theorem (`primrecPred_e_eq_union`). The predicate $(n, m, k) \mapsto e_n = e_m \cup e_k$ is primitive recursive.

Proof. By `e.eqUnionBool_iff`, apply `primrec_e_eqUnionBool`. ■

Theorem (`scott_e_primrec_structure`). Scott's bundled claim: membership in e_n is a primitive recursive relation in (n, k); `eMax` is primitive recursive; inclusion $e_n \subseteq e_m$ is a primitive recursive relation in (n, m); and equality $e_n = e_m \cup e_k$ is a primitive recursive predicate in (n, m, k).

Proof. Conjunction of the four preceding theorems. ■

`ePrimrec.lean`

```
module

public import Mathlib.Computability.Primrec.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
import DomainTheory.DataTypesAsLattices.Pomega.eScan

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem primrecRel_mem_e : PrimrecRel fun n k => k ∈ e n :=
  primrecRel_testBit_true.of_eq fun n k => by simp [e.mem]

def eMax (n : ℕ) : ℕ := Nat.findGreatest (fun k => n.testBit k = true) n

theorem primrec_eMax : Primrec eMax := by
  exact (Primrec.nat_findGreatest Primrec.id primrecRel_testBit_true).of_eq fun _ =>
    rfl
```

```
theorem primrecRel_e_subset : PrimrecRel fun n m => e n ⊆ e m :=
  (Primrec.eq.comp primrec_e_subsetBool (Primrec.const true)).of_eq fun p =>
    eSubsetBool_iff p.1 p.2

theorem primrecPred_e_eq_union :
    PrimrecPred fun t : ℕ × ℕ × ℕ => e t.1 = e t.2.1 ∪ e t.2.2 :=
  (Primrec.eq.comp primrec_e_eqUnionBool (Primrec.const true)).of_eq fun t =>
    eEqUnionBool_iff t.1 t.2.1 t.2.2

/-- Scott (1976), §1: membership, inclusion, union-equality for `e`, and `eMax`. -/
theorem scott_e_primrec_structure :
    PrimrecRel (fun n k => k ∈ e n) ∧
      Primrec eMax ∧
        PrimrecRel (fun n m => e n ⊆ e m) ∧
          PrimrecPred (fun t : ℕ × ℕ × ℕ => e t.1 = e t.2.1 ∪ e t.2.2) :=
  ⟨primrecRel_mem_e, primrec_eMax, primrecRel_e_subset, primrecPred_e_eq_union⟩

end DomainTheory.DataTypesAsLattices
```

Coding finite subsets ✓ Scott's definition identifies e_n with the set of bit positions where n has a 1. Every finite subset of ω arises that way; this note gives an explicit constructor and the existence lemmas used later (density, Scott continuity, enumeration).

Definition (`e.codeList`). For a list $k_0 :: \cdots :: k_{m-1} :: []$, `e.codeList` returns $2^{k_0} \mid \cdots \mid 2^{k_{m-1}}$ (and 0 for the empty list).

Theorem (`e.codeList_eq`). For every list `ks` of naturals,

$$e(\texttt{e.codeList}\ ks) = \{k \mid k \in ks\}.$$

Proof. Induction on `ks`. The empty list codes $e_0 = \emptyset$. For `k :: ks`, `e.codeList` ORs in 2^k; by `e.or` and `e.pow2`, membership matches the list membership characterization. ■

Definition (`e.codeFinset`). For a finset `s`, `e.codeFinset s` is `e.codeList` applied to the list of elements of `s` (order as stored in the finset).

Theorem (`e.codeFinset_eq`). For every finset `s` of naturals,

$$e(\texttt{e.codeFinset}\ s) = s$$

as subsets of ω.

Proof. Extensionality using `e.codeList_eq` on the element list. ■

Theorem (`e.exists_code`). Every finset `s` has a code $n \in \omega$ with $e_n = s$ as sets.

Proof. Witness `e.codeFinset s` and `e.codeFinset_eq`. ■

Theorem (`e.exists_code_of_finite`). If $s \in \mathbf{P}\omega$ is finite, then $\exists n,\ e_n = s$.

Proof. Write s as a finset and apply `e.exists_code`. ■

When the exponents k_i are strictly increasing, this agrees with Scott's sum $\sum_{i<m} 2^{k_i}$; duplicate entries in a list would OR the same bit twice, so the finset formulation is the canonical coding for an arbitrary finite set.

`codeFinset.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Finset.Defs
public import Mathlib.Data.Multiset.Basic
import Mathlib.Data.Set.Finite.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace e

/-- Bit-mask code of a list of naturals: bitwise OR of $2^k$ over entries. -/
def codeList : List ℕ → ℕ
  | [] => 0
  | k :: ks => 2 ^ k ||| codeList ks

theorem codeList_eq (ks : List ℕ) : e (codeList ks) = {k | k ∈ ks} := by
  induction ks with
  | nil => simp [codeList, zero]
  | cons k ks ih =>
    ext i
    simp [codeList, or, pow2, ih]

noncomputable def codeFinset (s : Finset ℕ) : ℕ := codeList s.val.toList

theorem codeFinset_eq (s : Finset ℕ) : e (codeFinset s) = (s : Set ℕ) := by
  ext k
  simp [codeFinset, codeList_eq]

theorem exists_code (s : Finset ℕ) : ∃ n, e n = (s : Set ℕ) :=
  ⟨codeFinset s, codeFinset_eq s⟩

theorem exists_code_of_finite {s : Pomega} (hs : Set.Finite s) :
    ∃ n, e n = s := by
  obtain ⟨t, ht⟩ := hs.exists_finset_coe
  exact ⟨codeFinset t, ht ▸ codeFinset_eq t⟩

end e

end DomainTheory.DataTypesAsLattices
```

Density of finite sets in $\mathbf{P}\omega$ ✓ Topologically speaking the finite sets e_n are *dense* in the space $\mathbf{P}\omega$, for each $x \in \mathbf{P}\omega$ is the "limit" of its finite subsets in the sense that

$$x = \bigcup \{e_n \mid e_n \subseteq x\}.$$

The formalization packages each included finite piece $e_n \subseteq x$ as a subset of $\mathbf{P}\omega$ and unions over all codes n.

Definition (`e.below`). `e.below x n` is $\{k \mid e_n \subseteq x \wedge k \in e_n\}$ (equivalently e_n when $e_n \subseteq x$, and $\emptyset$ otherwise).

Lemma (`e.singleton_eq`). For each $k \in \omega$, the singleton $\{k\}$ equals e_{2^k}.

Proof. This is `e.pow2`, read backwards. ■

Lemma (`e.exists_mem`). If $k \in x$ for $x \in \mathbf{P}\omega$, then there exists n with $e_n \subseteq x$ and $k \in e_n$.

Proof. Witness $n = 2^k$; use `e.singleton_eq` and singleton inclusion. ■

Theorem (`e.union_below_eq`). For every $x \in \mathbf{P}\omega$,

$$\bigcup_n \texttt{e.below}\ x\ n = x.$$

Proof. ($\subseteq$) Each k in the union lies in some $e_n \subseteq x$. ($\supseteq$) For $k \in x$, `e.exists_mem` gives n with $k \in$ `e.below` $x\ n$. ■

This is Scott's density clause before the definition of open sets in $\mathbf{P}\omega$.

`eDense.lean`

```
module

public import Mathlib.Order.SetNotation
public import DomainTheory.DataTypesAsLattices.Pomega.e
import Mathlib.Data.Finset.Attr
import Mathlib.Tactic.Attr.Core
import Mathlib.Tactic.Bound.Init
import Mathlib.Tactic.SetLike

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace e

/-- Elements of $e_n$ when $e_n \subseteq x$ (empty if $e_n \not\subseteq x$). -/
def below (x : Pomega) (n : ℕ) : Pomega :=
  {k | e n ⊆ x ∧ k ∈ e n}

theorem singleton_eq (k : ℕ) : ({k} : Pomega) = e (2 ^ k) := (pow2 k).symm

theorem exists_mem {x : Pomega} {k : ℕ} (hk : k ∈ x) :
    ∃ n, e n ⊆ x ∧ k ∈ e n :=
  ⟨2 ^ k, by simp [pow2, Set.singleton_subset_iff, hk], by simp [pow2]⟩

theorem union_below_eq (x : Pomega) : (⋃ n, below x n) = x := by
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn, hke⟩ := Set.mem_iUnion.mp hk
    exact hn hke
  · intro hk
    refine Set.mem_iUnion.mpr ⟨2 ^ k, ?_⟩
    simp [below, pow2, Set.singleton_subset_iff, hk]

end e

end DomainTheory.DataTypesAsLattices
```

3.2.3 Cantor pairing of integer pairs (n, m) ✓

To formulate later definitions we introduce a standard enumeration of all pairs of naturals. The code follows the "little diagonal" ordering Scott describes; the inverse `unpair` is included in Lean for convenience even though the paper does not fix notation for it yet.

Definition (`pair`). The Cantor pairing is

$$\texttt{pair}\ n\ m = \tfrac{1}{2}(n+m)(n+m+1) + m.$$

Definition (`triangle`). Write $T(w) = w(w+1)/2$ for the w-th triangular number: `triangle w`.

Lemma (`pair_eq_triangle`). $\texttt{pair}\ n\ m = T(n+m) + m$.

Proof. Definition of `pair` and `triangle`. ■

Lemma (`triangle_succ`). $T(w+1) = T(w) + (w+1)$.

Proof. Expand $(w+1)(w+2)$ as $w(w+1) + 2(w+1)$ and divide by 2. ■

Lemma (`exists_triangle_bucket`). For every $k \in \omega$ there exists w with $T(w) \le k < T(w{+}1)$.

Proof. The predicate $k < T(w+1)$ holds for $w = k$ because $T(k+1) = T(k) + (k+1)$. Take the least such w by `Nat.find`; minimality gives $T(w) \le k$. ■

Theorem (`exists_pair`). For every $k \in \omega$ there exist n, m with $\texttt{pair}\ n\ m = k$.

Proof. Choose w from `exists_triangle_bucket`. Set $m = k - T(w)$ and $n = w - m$. Then $m \le w$ and $n + m = w$, so `pair_eq_triangle` and $k = T(w) + m$ yield $\texttt{pair}\ n\ m = k$. ■

Definition (`unpair`). `unpair k` is some pair (n, m) with $\texttt{pair}\ n\ m = k$, chosen from `exists_pair`.

Lemma (`pair_le_left`). $n \le \texttt{pair}\ n\ m$.

Proof. Since $n \le n + m$, we have $2n \le (n+m)(n+m+1)$ for $n > 0$, hence $n \le T(n+m)$; the case $n = 0$ is trivial. Then $n \le T(n+m) + m = \texttt{pair}\ n\ m$. ■

Lemma (`pair_lt_left`). $n < \texttt{pair}\ n\ m + 1$.

Proof. From `pair_le_left` and `Nat.lt_succ_self`. ■

Lemma (`pair_le_right`). $m \le \texttt{pair}\ n\ m$.

Proof. $\texttt{pair}\ n\ m = T(n+m) + m \ge m$. ■

Lemma (`pair_lt_right`). $m < \texttt{pair}\ n\ m + 1$.

Proof. From `pair_le_right`. ■

These are Scott's bounds $n \le (n, m)$ and $m \le (n, m)$; strict inequality to $\texttt{pair}\ n\ m + 1$ is the formal counterpart of his remark about equality only at $(0, 0)$ and $(1, 0)$.

Lemma (`pair_unpair`). $\texttt{pair}\ (\texttt{unpair}\ k).1\ (\texttt{unpair}\ k).2 = k$.

Proof. By construction from `exists_pair`. ■

Lemma (`triangle_le_of_le`). If $a \le b$ then $T(a) \le T(b)$.

Proof. Induction on $a \le b$; the step uses `triangle_succ` and monotonicity of addition. ■

Lemma (`triangle_le_pair`). For all n, m,

$$T(n+m) \le \texttt{pair}\ n\ m < T(n+m+1).$$

Proof. The lower bound is $T(n+m) + m \ge T(n+m)$. For the upper bound, `pair_eq_triangle` and `triangle_succ` give $\texttt{pair}\ n\ m = T(n+m) + m < T(n+m) + (n+m+1) = T(n+m+1)$. ■

Lemma (`triangle_lt_of_lt`). If $a < b$ then $T(a) < T(b)$.

Proof. From $a + 1 \le b$, strict inequality $T(a) < T(a+1) = T(a) + (a+1)$ and `triangle_le_of_le`. ■

Theorem (`pair_inj`). If $\texttt{pair}\ n_1\ m_1 = \texttt{pair}\ n_2\ m_2$, then $n_1 = n_2$ and $m_1 = m_2$.

Proof. Compare $w_i = n_i + m_i$ using the window $T(w_i) \le \texttt{pair}\ n_i\ m_i < T(w_i{+}1)$ from `triangle_le_pair`. If $w_1 \neq w_2$, one paired value would lie strictly below the other, contradicting equality. So $w_1 = w_2$; then $T(w) + m_1 = T(w) + m_2$ gives $m_1 = m_2$, and $n_1 + m_1 = n_2 + m_2$ gives $n_1 = n_2$. ■

Theorem (`unpair_pair`). $\texttt{unpair}\ (\texttt{pair}\ n\ m) = (n, m)$.

Proof. `pair_unpair` and injectivity `pair_inj`. ■

Together, `pair` and `unpair` form a bijection $\omega \times \omega \cong \omega$, so this is the one-to-one enumeration Scott asserts (with primitive recursive inverse left informal in the text).

`pair.lean`

```
module

public import Aesop.BuiltinRules
public import Mathlib.Data.Nat.Notation
public import Mathlib.Tactic.ToAdditive
public import Mathlib.Tactic.ToDual
import Mathlib.Data.Finset.Attr
import Mathlib.Data.Nat.Find
import Mathlib.Order.Basic
import Mathlib.Tactic.Bound.Init
import Mathlib.Tactic.Finiteness.Attr
import Mathlib.Tactic.SetLike
import Mathlib.Tactic.Widget.Calc

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def pair (n m : ℕ) : ℕ := (n + m) * (n + m + 1) / 2 + m

def triangle (w : ℕ) : ℕ := w * (w + 1) / 2

theorem pair_eq_triangle (n m : ℕ) : pair n m = triangle (n + m) + m := rfl

theorem triangle_succ (w : ℕ) : triangle (w + 1) = triangle w + (w + 1) := by
  have hex : (w + 1) * (w + 2) = w * (w + 1) + 2 * (w + 1) := by
    calc
      (w + 1) * (w + 2) = (w + 1) * w + (w + 1) * 2 := Nat.mul_add _ _ _
      _ = w * (w + 1) + 2 * (w + 1) := by
        rw [Nat.mul_comm (w + 1) w, Nat.mul_comm (w + 1) 2]
  simp only [triangle]
  rw [show (w + 1) * ((w + 1) + 1) = (w + 1) * (w + 2) from rfl, hex]
  exact Nat.add_mul_div_left _ _ (by decide : 0 < 2)

theorem exists_triangle_bucket (k : ℕ) :
    ∃ w, triangle w ≤ k ∧ k < triangle (w + 1) := by
  let P := fun w : ℕ => k < triangle (w + 1)
  have hP : ∃ w, P w := ⟨k, by
    change k < triangle (k + 1)
    rw [triangle_succ]
    exact lt_of_lt_of_le (Nat.lt_succ_self k) (Nat.le_add_left (k + 1) _)⟩
  let w := Nat.find hP
  refine ⟨w, ?_, Nat.find_spec hP⟩
  cases hw : w with
```

```
  | zero => exact Nat.zero_le k
  | succ w' =>
    have hmin : ¬ P w' :=
      Nat.find_min hP (Nat.lt_of_succ_le (by
        have : w = Nat.find hP := rfl
        rw [← this, hw]))
    exact Nat.le_of_not_gt hmin

theorem exists_pair (k : ℕ) : ∃ n m, pair n m = k := by
  obtain ⟨w, hle, hlt⟩ := exists_triangle_bucket k
  refine ⟨w - (k - triangle w), k - triangle w, ?_⟩
  have hlt' : k < triangle w + w + 1 := by
    have : k < triangle w + (w + 1) := by rwa [triangle_succ] at hlt
    exact Nat.add_assoc (triangle w) w 1 ▶ this
  have hk : k ≤ triangle w + w := Nat.lt_succ_iff.mp hlt'
  have hm : k - triangle w ≤ w :=
    Nat.le_of_add_le_add_left ((Nat.add_sub_of_le hle).symm ▶ hk)
  have : w - (k - triangle w) + (k - triangle w) = w := Nat.sub_add_cancel hm
  rw [pair_eq_triangle, this, Nat.add_sub_of_le hle]

noncomputable def unpair (k : ℕ) : ℕ × ℕ :=
  ((exists_pair k).choose, (exists_pair k).choose_spec.choose)

theorem pair_le_left (n m : ℕ) : n ≤ pair n m := by
  have htri : n ≤ triangle (n + m) := by
    cases n with
    | zero => exact Nat.zero_le _
    | succ n =>
      have hmul : n.succ * 2 ≤ (n.succ + m) * (n.succ + m + 1) :=
        Nat.mul_le_mul
          (Nat.le_add_right n.succ m)
          (Nat.succ_le_succ
            (Nat.le_trans (Nat.succ_le_succ (Nat.zero_le n)) (Nat.le_add_right n.succ
    m)))
      exact (Nat.le_div_iff_mul_le (by decide : 0 < 2)).mpr hmul
  exact htri.trans (Nat.le_add_right _ _)

theorem pair_lt_left (n m : ℕ) : n < pair n m + 1 :=
  Nat.lt_succ_of_le (pair_le_left n m)

theorem pair_le_right (n m : ℕ) : m ≤ pair n m :=
  Nat.le_add_left _ _

theorem pair_lt_right (n m : ℕ) : m < pair n m + 1 :=
  Nat.lt_succ_of_le (pair_le_right n m)

theorem pair_unpair (k : ℕ) : pair (unpair k).1 (unpair k).2 = k :=
  (exists_pair k).choose_spec.choose_spec

theorem triangle_le_of_le {a b : ℕ} (h : a ≤ b) : triangle a ≤ triangle b := by
  induction h with
  | refl => exact le_rfl
  | step h ih =>
    rw [triangle_succ]
    exact le_trans ih (Nat.le_add_right _ _)
```

```
theorem triangle_le_pair (n m : ℕ) :
    triangle (n + m) ≤ pair n m ∧ pair n m < triangle (n + m + 1) := by
  constructor
  · exact Nat.le_add_right _ _
  · rw [triangle_succ]
    exact Nat.add_lt_add_left (Nat.lt_succ_of_le (Nat.le_add_left m n)) _

theorem triangle_lt_of_lt {a b : ℕ} (h : a < b) : triangle a < triangle b := by
  have : a + 1 ≤ b := Nat.succ_le_of_lt h
  calc
    triangle a < triangle a + (a + 1) := Nat.lt_add_of_pos_right (Nat.succ_pos _)
    _ = triangle (a + 1) := (triangle_succ a).symm
    _ ≤ triangle b := triangle_le_of_le this

theorem pair_inj {n₁ m₁ n₂ m₂ : ℕ} (h : pair n₁ m₁ = pair n₂ m₂) :
    n₁ = n₂ ∧ m₁ = m₂ := by
  have h1 := triangle_le_pair n₁ m₁
  have h2 := triangle_le_pair n₂ m₂
  have hw : n₁ + m₁ = n₂ + m₂ := by
    rcases lt_trichotomy (n₁ + m₁) (n₂ + m₂) with hlt | heq | hgt
    · have hle : triangle (n₁ + m₁ + 1) ≤ triangle (n₂ + m₂) :=
        triangle_le_of_le (Nat.succ_le_of_lt hlt)
      have : pair n₁ m₁ < pair n₁ m₁ :=
        calc
          pair n₁ m₁ < triangle (n₁ + m₁ + 1) := h1.2
          _ ≤ triangle (n₂ + m₂) := hle
          _ ≤ pair n₂ m₂ := h2.1
          _ = pair n₁ m₁ := h.symm
      exact (lt_irrefl _ this).elim
    · exact heq
    · have hle : triangle (n₂ + m₂ + 1) ≤ triangle (n₁ + m₁) :=
        triangle_le_of_le (Nat.succ_le_of_lt hgt)
      have : pair n₂ m₂ < pair n₂ m₂ :=
        calc
          pair n₂ m₂ < triangle (n₂ + m₂ + 1) := h2.2
          _ ≤ triangle (n₁ + m₁) := hle
          _ ≤ pair n₁ m₁ := h1.1
          _ = pair n₂ m₂ := h
      exact (lt_irrefl _ this).elim
  have hm : m₁ = m₂ := by
    have : triangle (n₁ + m₁) + m₁ = triangle (n₂ + m₂) + m₂ := h
    rw [hw] at this
    exact Nat.add_left_cancel this
  have hn : n₁ = n₂ := Nat.add_right_cancel (hw.trans (by rw [hm]))
  exact ⟨hn, hm⟩

theorem unpair_pair (n m : ℕ) : unpair (pair n m) = (n, m) := by
  have h := pair_unpair (pair n m)
  exact Prod.ext (pair_inj h).1 (pair_inj h).2

end DomainTheory.DataTypesAsLattices
```

3.2.4 Basic open sets and continuity on $\mathbf{P}\omega$ ✓

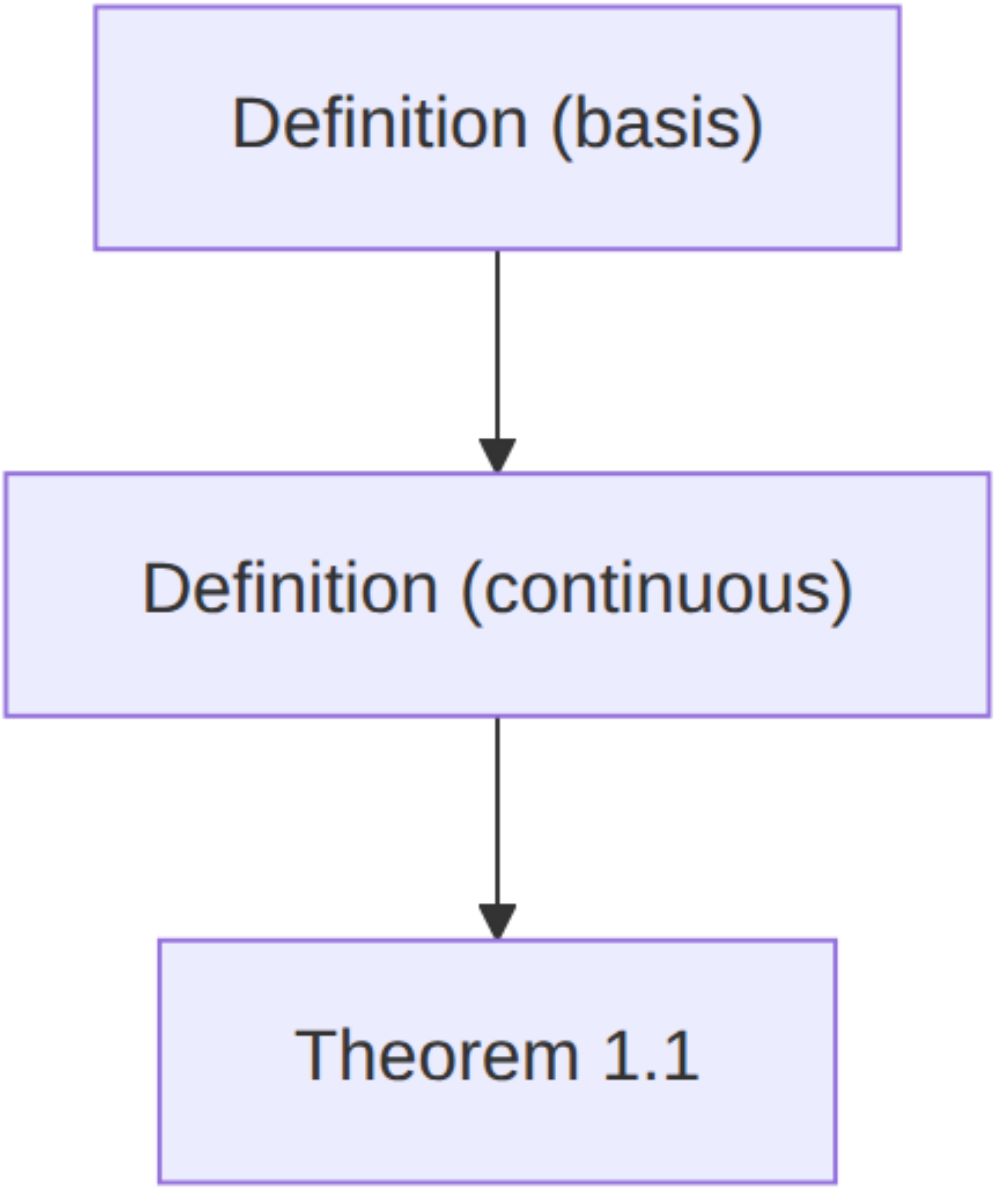


Figure 24: Basic open sets and continuity on $\mathbf{P}\omega$ ✓.

Definition (basis) ✓ **DEFINITION.** A *basis* for the neighborhoods of $\mathbf{P}\omega$ consists of those sets of the form:

$$\{x \in \mathbf{P}\omega \mid e_n \subseteq x\},$$

for a given e_n. An arbitrary *open subset* is then a union of basic neighborhoods.

It is easy to prove that an open subset $U \subseteq \mathbf{P}\omega$ is just a set of "finite character"; that is, a set such that for all $x \in \mathbf{P}\omega$ we have $x \in U$ if and only if some finite subset of x also belongs to U. An alternate approach would define directly what we mean by a continuous function using the idea that such functions must preserve limits.

Definition (`basicNhhd`). For $n \in \omega$, the basic neighborhood is $\mathrm{basicNhhd}\, n = \{x \in \mathbf{P}\omega \mid e_n \subseteq x\}$.

In the `basicNhhd` namespace we define:

Theorem (`inter`). $\mathrm{basicNhhd}\, n \cap \mathrm{basicNhhd}\, m = \mathrm{basicNhhd}\, (n \mid m)$.

Proof. Extensionality: x lies in the intersection iff $e_n \subseteq x$ and $e_m \subseteq x$ iff $e_n \cup e_m \subseteq x$ iff $e_{n|m} \subseteq x$ by `e.or`. ■

Theorem (`zero`). $\mathrm{basicNhhd}\, 0 = \mathbf{P}\omega$.

Proof. Every x satisfies $e_0 \subseteq x$ because $e_0 = \emptyset$ (`e.zero`). ■

Theorem (`mono`). If $e_n \subseteq e_m$, then $\mathrm{basicNhhd}\, m \subseteq \mathrm{basicNhhd}\, n$.

Proof. If $e_m \subseteq x$ and $e_n \subseteq e_m$, then $e_n \subseteq x$. ■

`basicNhhd.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
import Mathlib.Data.Finset.Attr
import Mathlib.Data.Set.BooleanAlgebra
import Mathlib.Tactic.Bound.Init
import Mathlib.Tactic.SetLike

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def basicNhhd (n : ℕ) : Set Pomega := {x | e n ⊆ x}

namespace basicNhhd

theorem inter (n m : ℕ) :
    basicNhhd n ∩ basicNhhd m = basicNhhd (n ||| m) := by
  ext x
  simp [basicNhhd, e.or, Set.union_subset_iff]

theorem zero : basicNhhd 0 = Set.univ := by
  ext x
  simp [basicNhhd, e.zero]

theorem mono {n m : ℕ} (h : e n ⊆ e m) : basicNhhd m ⊆ basicNhhd n := by
  intro x hx
  exact h.trans hx

end basicNhhd

end DomainTheory.DataTypesAsLattices
```

Definition (continuous) ✓ **DEFINITION.** A function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is *continuous* iff for all $x \in \mathbf{P}\omega$ we have:

$$f(x) = \bigcup \{ f(e_n) \mid e_n \subseteq x \}.$$

Again it is an easy exercise to prove that a function is continuous in the sense of this definition iff it is continuous in the usual topological sense (namely: inverse images of open sets are open). For giving proofs it is even more convenient to have the usual ε-δ formulation of continuity.

Definition (`scottPiece, scottUnion`). For $f : \mathbf{P}\omega \to \mathbf{P}\omega$, $x \in \mathbf{P}\omega$, and $n \in \omega$, let $\mathrm{scottPiece}\, f\, x\, n = \{k \mid e_n \subseteq x \wedge k \in f(e_n)\}$ and $\mathrm{scottUnion}\, f\, x = \bigcup_n \mathrm{scottPiece}\, f\, x\, n$. This is exactly Scott's union $\bigcup\{f(e_n) \mid e_n \subseteq x\}$.

Definition (`IsScottContinuous`). The typeclass `IsScottContinuous` on $f : \mathbf{P}\omega \to \mathbf{P}\omega$ packages the field `out`: for every x, $f(x) = \mathrm{scottUnion}\, f\, x$.

In the `IsScottContinuous` namespace:

Theorem (`monotone`). If f is Scott-continuous and $x \subseteq y$ in $\mathbf{P}\omega$, then $f(x) \subseteq f(y)$.

Proof. Take $m \in f(x)$. By `out` and `mem_scottUnion`, choose n with $e_n \subseteq x$ and $m \in f(e_n)$. Then $e_n \subseteq y$, so the same witness shows $m \in f(y)$ via `mem_scottUnion`. ■

Theorem (`determined`). If f and g are Scott-continuous and $f(e_n) = g(e_n)$ for every $n \in \omega$, then $f = g$.

Proof. For each x, `out` rewrites $f(x)$ and $g(x)$ as $\mathrm{scottUnion}\, f\, x$ and $\mathrm{scottUnion}\, g\, x$; extensionality on pieces uses `scottPiece` and the hypothesis on finite values $f(e_n)$, $g(e_n)$. ■

Theorem (`iff_continuous`). For $f : \mathbf{P}\omega \to \mathbf{P}\omega$, f is Scott-continuous iff f is continuous for the Scott topology on $\mathbf{P}\omega$ (generated by the basic opens $\mathrm{memOpen}\, k = \{x \mid k \in x\}$).

Proof. ($\Rightarrow$) By `continuous_generateFrom_iff`, it suffices to show each preimage $f^{-1}(\mathrm{memOpen}\, k)$ is Scott-open. Using `isScottOpen_iff_isOpen`, rewrite membership in that preimage as $k \in \mathrm{scottUnion}\, f\, x$ when $f(x) = \mathrm{scottUnion}\, f\, x$ (`out`), then as an union over indices n with $k \in f(e_n)$ and $e_n \subseteq x$ via `mem_scottUnion`; the converse direction uses `monotone` on the chosen finite witness.

($\Leftarrow$) Fix x and k. If $k \in f(x)$, Scott-openness of $f^{-1}(\mathrm{memOpen}\, k)$ (`isScottOpen_iff_isOpen`, preimage of the open set $\mathrm{memOpen}\, k$ by `isOpen_memOpen`) gives a union of basic neighborhoods $\mathrm{basicNhhd}\, n$ covering x; some $e_n \subseteq x$ still carries $k \in f(e_n)$, hence $k \in \mathrm{scottUnion}\, f\, x$ by `mem_scottUnion`. If $k \in \mathrm{scottUnion}\, f\, x$, pick n with $e_n \subseteq x$ and $k \in f(e_n)$; then $e_n \in f^{-1}(\mathrm{memOpen}\, k)$, and upward closure (`isScottOpen_isUpper`) with $e_n \subseteq x$ puts x in that preimage, so $k \in f(x)$. Thus `out` holds. ■

Theorem (`IsScottOpen.univ`). The whole space $\mathbf{P}\omega$ is Scott-open.

Proof. It is open in the Scott topology. ■

Theorem (`IsScottOpen.empty`). The empty set is Scott-open.

Proof. It is open in the Scott topology. ■

Theorem (`IsScottOpen.union`). The union of two Scott-open sets is Scott-open.

Proof. Openness is preserved by binary unions. ■

Theorem (`IsScottOpen.inter`). The intersection of two Scott-open sets is Scott-open.

Proof. Openness is preserved by binary intersections. ■

Theorem (`IsScottOpen.iUnion`). An arbitrary union of Scott-open sets is Scott-open.

Proof. Openness is preserved by arbitrary unions. ■

Theorem (`IsScottOpen.mem_iff`). If U is Scott-open, then $x \in U$ if and only if some finite $e_n \subseteq x$ already lies in U.

Proof. Unpack the basic-neighbourhood presentation of U, or use upward closure on a finite witness. ■

Theorem (`IsScottContinuous.mem_preimage`). If f is Scott-continuous, then $\{x \mid k \in f(x)\}$ is Scott-open.

Proof. It is the preimage of the subbasic open $\mathrm{memOpen}\, k$. ■

Theorem (`IsScottContinuous.not_subset`). If f is Scott-continuous, then $\{x \mid f(x) \not\subseteq e\}$ is Scott-open.

Proof. Write the set as a union over $k \notin e$ of the preimages $\{x \mid k \in f(x)\}$. ■

Theorem (`IsScottContinuous.subset_preimage`). If f is Scott-continuous and e is finite, then $\{x \mid e \subseteq f(x)\}$ is Scott-open.

Proof. Intersect the finitely many preimages $\{x \mid k \in f(x)\}$ for $k \in e$. ■

Theorem (`IsScottContinuous.ne_eq_open_union_closed`). If f is Scott-continuous and e is finite, then $\{x \mid f(x) \neq e\}$ is the union of $\{x \mid f(x) \not\subseteq e\}$ with the complement of $\{x \mid e \subseteq f(x)\}$, and those two pieces are respectively open and co-open.

Proof. Inequality of sets is failure of either inclusion; apply the two previous facts. ■

`scottPiece.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
import Mathlib.Data.Finset.Attr
import Mathlib.Tactic.Bound.Init
import Mathlib.Tactic.Continuity.Init
import Mathlib.Tactic.SetLike

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def scottPiece (f : Pomega → Pomega) (x : Pomega) (n : ℕ) : Pomega :=
  {k | e n ⊆ x ∧ k ∈ f (e n)}

end DomainTheory.DataTypesAsLattices
```

`scottUnion.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.scottPiece
public import Mathlib.Order.SetNotation
import Mathlib.Data.Finset.Attr
import Mathlib.Tactic.Bound.Init
import Mathlib.Tactic.Continuity.Init
import Mathlib.Tactic.SetLike

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def scottUnion (f : Pomega → Pomega) (x : Pomega) : Pomega :=
  ⋃ n, scottPiece f x n

end DomainTheory.DataTypesAsLattices
```

`mem_scottUnion.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
import Mathlib.Data.Finset.Attr
import Mathlib.Tactic.Bound.Init
import Mathlib.Tactic.Continuity.Init
import Mathlib.Tactic.SetLike

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem mem_scottUnion {f : Pomega → Pomega} {x : Pomega} {k : ℕ} :
    k ∈ scottUnion f x ↔ ∃ n, e n ⊆ x ∧ k ∈ f (e n) := by
  constructor
  · intro hk
    obtain ⟨n, hn, hfk⟩ := Set.mem_iUnion.mp hk
    exact ⟨n, hn, hfk⟩
  · intro ⟨n, hn, hk⟩
    exact Set.mem_iUnion.mpr ⟨n, hn, hk⟩

end DomainTheory.DataTypesAsLattices
```

IsScottOpen.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.basicNhhd
public import Mathlib.Order.SetNotation
import Mathlib.Data.Finset.Attr
import Mathlib.Tactic.Bound.Init
import Mathlib.Tactic.Continuity.Init
import Mathlib.Tactic.SetLike

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def IsScottOpen (U : Set Pomega) : Prop :=
  ∃ B : Set ℕ, U = ⋃ n ∈ B, basicNhhd n

end DomainTheory.DataTypesAsLattices
```

IsScottOpen_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_iff_isOpen
public import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_isUpper
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
import Mathlib.Tactic.Bound.Init

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

namespace IsScottOpen

theorem univ : IsScottOpen (Set.univ : Set Pomega) :=
  (isScottOpen_iff_isOpen _).mpr isOpen_univ

theorem empty : IsScottOpen (∅ : Set Pomega) :=
  (isScottOpen_iff_isOpen _).mpr isOpen_empty

theorem union {U V : Set Pomega} (hU : IsScottOpen U) (hV : IsScottOpen V) :
    IsScottOpen (U ∪ V) :=
  (isScottOpen_iff_isOpen _).mpr
    (((isScottOpen_iff_isOpen _).mp hU).union ((isScottOpen_iff_isOpen _).mp hV))

theorem inter {U V : Set Pomega} (hU : IsScottOpen U) (hV : IsScottOpen V) :
    IsScottOpen (U ∩ V) :=
  (isScottOpen_iff_isOpen _).mpr
    (((isScottOpen_iff_isOpen _).mp hU).inter ((isScottOpen_iff_isOpen _).mp hV))

theorem iUnion {ι : Type*} (U : ι → Set Pomega) (hU : ∀ i, IsScottOpen (U i)) :
    IsScottOpen (⋃ i, U i) :=
  (isScottOpen_iff_isOpen _).mpr (isOpen_iUnion fun i => (isScottOpen_iff_isOpen _).mp
    (hU i))

theorem mem_iff {U : Set Pomega} (hU : IsScottOpen U) {x : Pomega} :
    x ∈ U ↔ ∃ n, e n ⊆ x ∧ e n ∈ U := by
  constructor
  · intro hx
    obtain ⟨B, rfl⟩ := hU
    obtain ⟨n, hn, hxn⟩ := Set.mem_iUnion₂.mp hx
    exact ⟨n, hxn, Set.mem_iUnion₂.mpr ⟨n, hn, (subset_rfl : e n ⊆ e n)⟩⟩
  · intro ⟨n, hn, hne⟩
    exact isScottOpen_isUpper hU hne hn

end IsScottOpen

end DomainTheory.DataTypesAsLattices
```

IsScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen_more
import DomainTheory.DataTypesAsLattices.Continuous.isOpen_memOpen
import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_iff_isOpen
import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_isUpper
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
import Mathlib.Tactic.Bound.Init
import Mathlib.Data.Set.Finite.Basic
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

class IsScottContinuous (f : Pomega → Pomega) : Prop where
  out : ∀ x, f x = scottUnion f x

namespace IsScottContinuous

theorem monotone {f : Pomega → Pomega}
    (hf : IsScottContinuous f) {x y : Pomega} (h : x ⊆ y) : f x ⊆ f y := by
  intro m hm
  rw [hf.out x, mem_scottUnion] at hm
  rw [hf.out y, mem_scottUnion]
  obtain ⟨n, hn, hmn⟩ := hm
  exact ⟨n, subset_trans hn h, hmn⟩

theorem determined {f g : Pomega → Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g)
    (hfin : ∀ n, f (e n) = g (e n)) : f = g := by
  funext x
  rw [hf.out x, hg.out x]
  simp [scottUnion, scottPiece, hfin]

theorem iff_continuous (f : Pomega → Pomega) :
    IsScottContinuous f ↔ Continuous f := by
  constructor
  · intro hf
    refine continuous_generateFrom_iff.2 ?_
    rintro _ ⟨k, rfl⟩
    rw [← isScottOpen_iff_isOpen]
    refine ⟨{n | k ∈ f (e n)}, ?_⟩
    ext x
    constructor
    · intro hx
      have : k ∈ scottUnion f x := by rwa [← hf.out x]
      obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp this
      exact Set.mem_iUnion₂.mpr ⟨n, hkn, hn⟩
    · intro hx
      obtain ⟨n, hkn, hn⟩ := Set.mem_iUnion₂.mp hx
      exact monotone hf hn hkn
  · intro hfcont
    constructor
    intro x
    ext k
    have hopen : IsScottOpen (f ^-1' memOpen k) :=
      (isScottOpen_iff_isOpen _).mpr (IsOpen.preimage hfcont (isOpen_memOpen k))
    constructor
    · intro hk
      obtain ⟨B, hB⟩ := hopen
      have hx : x ∈ ⋃ n ∈ B, basicNhhd n := by
        have : x ∈ f ^-1' memOpen k := hk
        rwa [hB] at this
```

```
      obtain ⟨n, hn, hxn⟩ := Set.mem_iUnion₂.mp hx
      have hen : k ∈ f (e n) := by
        have : e n ∈ f ^-1' memOpen k := by
          rw [hB]
          exact Set.mem_iUnion₂.mpr ⟨n, hn, (subset_rfl : e n ⊆ e n)⟩
        exact this
      exact mem_scottUnion.mpr ⟨n, hxn, hen⟩
    · intro hk
      obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
      have : e n ∈ f ^-1' memOpen k := hkn
      exact isScottOpen_isUpper hopen this hn

theorem mem_preimage {f : Pomega → Pomega} (hf : IsScottContinuous f) (k : ℕ) :
    IsScottOpen {x | k ∈ f x} := by
  have : {x | k ∈ f x} = f ^-1' memOpen k := rfl
  rw [this, isScottOpen_iff_isOpen]
  exact IsOpen.preimage ((IsScottContinuous.iff_continuous f).mp hf) (isOpen_memOpen k
    )

theorem not_subset {f : Pomega → Pomega} (hf : IsScottContinuous f)
    (e : Pomega) : IsScottOpen {x | ¬ f x ⊆ e} := by
  have : {x | ¬ f x ⊆ e} = ⋃ k, {x | k ∈ e ∧ k ∈ f x} := by
    ext x
    constructor
    · intro hx
      obtain ⟨k, hk, hke⟩ := Set.not_subset.mp hx
      exact Set.mem_iUnion.mpr ⟨k, ⟨hke, hk⟩⟩
    · intro hx
      obtain ⟨k, hke, hk⟩ := Set.mem_iUnion.mp hx
      intro hsub
      exact hke (hsub hk)
  rw [this]
  exact IsScottOpen.iUnion _ fun k => by
    by_cases hke : k ∈ e
    · have : {x | k ∈ e ∧ k ∈ f x} = (∅ : Set Pomega) := by
        ext x; simp [hke]
      rw [this]; exact IsScottOpen.empty
    · have : {x | k ∈ e ∧ k ∈ f x} = {x | k ∈ f x} := by
        ext x; simp [hke]
      rw [this]; exact mem_preimage hf k

theorem subset_preimage {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {e : Pomega} (he : e.Finite) : IsScottOpen {x | e ⊆ f x} := by
  have : {x | e ⊆ f x} = ⋂ k ∈ he.toFinset, {x | k ∈ f x} := by
    ext x
    constructor
    · intro hx
      exact Set.mem_iInter₂.mpr fun k hk => hx (he.mem_toFinset.mp hk)
    · intro hx k hk
      exact Set.mem_iInter₂.mp hx k (he.mem_toFinset.mpr hk)
  rw [this, isScottOpen_iff_isOpen]
  exact isOpen_biInter_finset fun k _ =>
    (isScottOpen_iff_isOpen _).mp (mem_preimage hf k)

theorem ne_eq_open_union_closed {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {e : Pomega} (he : e.Finite) :
```

```
    {x | f x = e} = {x | ¬ f x ⊆ e} ∪ {x | ¬ e ⊆ f x} ∧
      IsScottOpen {x | ¬ f x ⊆ e} ∧
      IsScottOpen {x | ¬ e ⊆ f x}? := by
  refine ⟨?_, not_subset hf e, ?_⟩
  · ext x
    constructor
    · intro hne
      by_cases hsub : f x ⊆ e
      · refine Or.inr fun hsup => hne (subset_antisymm hsub hsup)
      · exact Or.inl hsub
    · intro h heq
      rcases h with h | h
      · exact h heq.subset
      · exact h heq.symm.subset
  · have : {x | ¬ e ⊆ f x}? = {x | e ⊆ f x} := by
      ext x; simp
    rw [this]
    exact subset_preimage hf he

end IsScottContinuous

end DomainTheory.DataTypesAsLattices
```

Theorem 1.1 ✓ **THEOREM 1.1** (The characterization theorem). A function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is *continuous* iff for all $x \in \mathbf{P}\omega$ and all e_m we have:

$$e_m \subseteq f(x) \quad \text{iff} \quad \exists e_n \subseteq x.\ e_m \subseteq f(e_n).$$

Note that open sets and continuous functions have a *monotonicity property*:
whenever $x \subseteq y$ *and* $x \in U$, *then* $y \in U$; *and*
whenever $x \subseteq y$, *then* $f(x) \subseteq f(y)$.

This gives a precise expression to the "positive" character of our topology. However, note too that openness and continuity mean rather more than just monotonicity. In particular, a continuous function is completely determined by the pairs of integers n, m such that $m \in f(e_n)$, as can be seen from the definition. (Hence, there are only a continuum number of continuous functions, but more than a continuum number of monotonic functions.) This brings us to the definition of the *graph* of a continuous function.

As translated into Lean, we have:

Theorem (`theorem_1_1`). For $f : \mathbf{P}\omega \to \mathbf{P}\omega$, f is Scott-continuous iff for all x and m, $e_m \subseteq f(x)$ iff there exists n with $e_n \subseteq x$ and $e_m \subseteq f(e_n)$.

Proof. The forward direction uses `IsScottContinuous.out` and membership in `scottUnion`/`scottPiece`; the converse rebuilds $f(x)$ as a union of pieces from the ε-δ hypothesis. ■

`theorem_1_1.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
import DomainTheory.DataTypesAsLattices.Continuous.exists_e_cover_finite

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem theorem_1_1 (f : Pomega → Pomega) :
    IsScottContinuous f ↔
      ∀ x m, e m ⊆ f x ↔ ∃ n, e n ⊆ x ∧ e m ⊆ f (e n) := by
  constructor
  · intro hf x m
    constructor
    · intro hmf
      have : ∀ k ∈ e m, ∃ n, e n ⊆ x ∧ k ∈ f (e n) := by
        intro k hk
        have : k ∈ scottUnion f x := by
          rw [← hf.out x]; exact hmf hk
        exact mem_scottUnion.mp this
      obtain ⟨s, hs⟩ := (e.finite m).exists_finset_coe
      obtain ⟨N, hN, hsf⟩ := exists_e_cover_finite hf s (fun k hk =>
        this k (by
          have : k ∈ (s : Set ℕ) := hk
          rwa [hs] at this))
      exact ⟨N, hN, by simpa [hs] using hsf⟩
    · intro ⟨n, hn, hmf⟩
      exact subset_trans hmf (IsScottContinuous.monotone hf hn)
  · intro h
    constructor
    intro x
    ext k
    constructor
    · intro hk
      have : e (2 ^ k) ⊆ f x := by
        simp [e.pow2, Set.singleton_subset_iff, hk]
      obtain ⟨n, hn, hkn⟩ := (h x (2 ^ k)).mp this
      refine mem_scottUnion.mpr ⟨n, hn, ?_⟩
      simpa [e.pow2, Set.singleton_subset_iff] using hkn
    · intro hk
      obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
      have : e (2 ^ k) ⊆ f (e n) := by
        simp [e.pow2, Set.singleton_subset_iff, hkn]
      have : e (2 ^ k) ⊆ f x := (h x (2 ^ k)).mpr ⟨n, hn, this⟩
      simpa [e.pow2, Set.singleton_subset_iff] using this

end DomainTheory.DataTypesAsLattices
```

3.2.5 Graphs of continuous functions ✓

graph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

def graph (f : Pomega → Pomega) : Pomega :=
  {p | ∃ n m, p = pair n m ∧ m ∈ f (e n)}

def lam (τ : Pomega → Pomega) : Pomega := graph τ

namespace lam

theorem eq_graph (τ : Pomega → Pomega) : lam τ = graph τ := rfl

end lam

end DomainTheory.DataTypesAsLattices
```

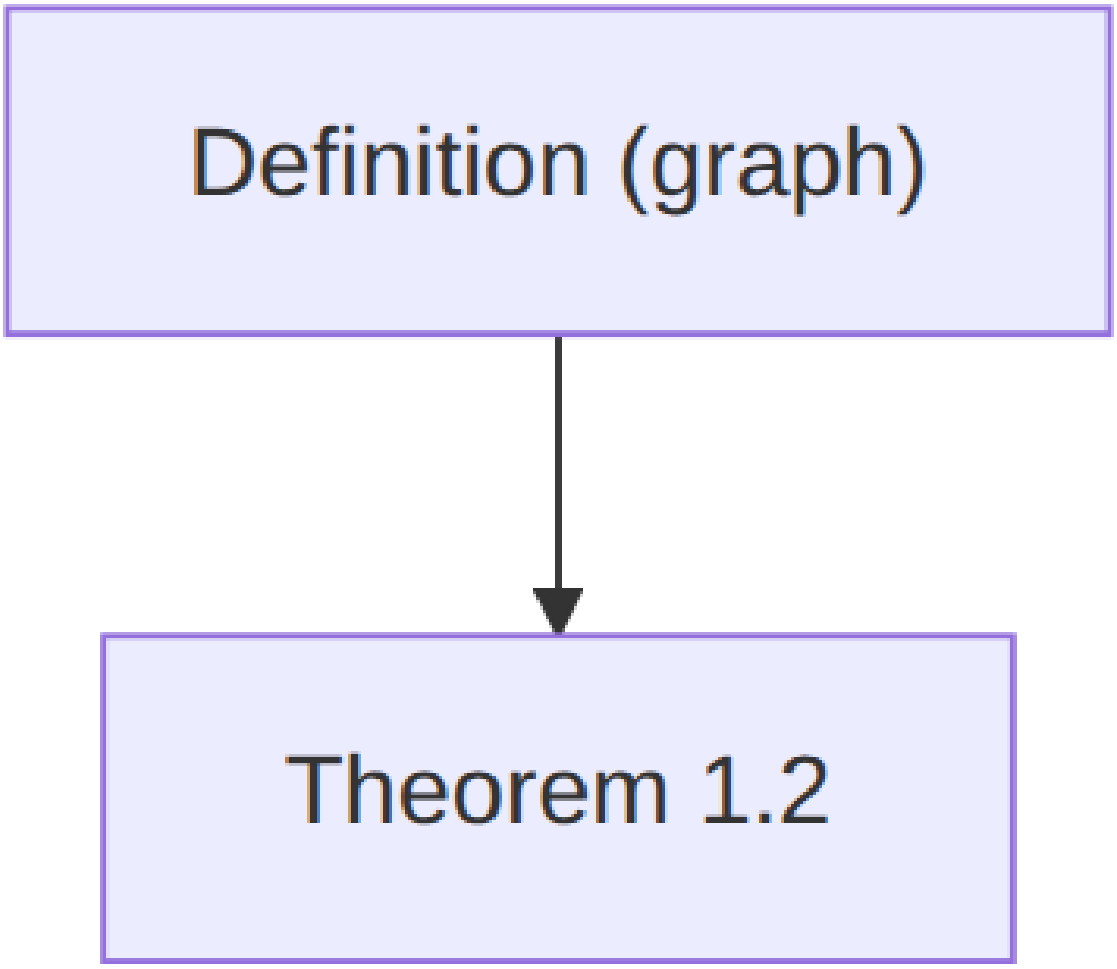


Figure 25: Graphs of continuous functions ✓.

Definition (graph) ✓ **DEFINITION.** The graph of a continuous function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is defined by the equation:

$$\mathbf{graph}(f) = \{(n, m) \mid m \in f(e_n)\};$$

while the function determined by any set $u \subseteq \omega$ is defined by the equation:

$$\mathbf{fun}(u)(x) = \{m \mid \exists e_n \subseteq x.\ (n, m) \in u\}.$$

IsGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
import Mathlib.Data.Finset.Attr
import Mathlib.Tactic.Continuity.Init
import Mathlib.Tactic.SetLike

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

class IsGraph (u : Pomega) : Prop where
  out : ∀ {{k m n}}, pair k m ∈ u → e k ⊆ e n → pair n m ∈ u

end DomainTheory.DataTypesAsLattices
```

Theorem 1.2 ✓ **THEOREM 1.2** (The graph theorem). *Every continuous function is uniquely determined by its graph in the sense that:*

(i) $\mathbf{fun}(\mathbf{graph}(f)) = f$.

Conversely, every set of integers determines a continuous function and we have:

(ii) $u \subseteq \mathbf{graph}(\mathbf{fun}(u))$,

where equality holds just in case u satisfies:

(iii) *whenever $(k, m) \in u$ and $e_k \subseteq e_n$, then $(n, m) \in u$.*

Besides functions of one variable we need to consider also functions of several variables. The official definition for one variable given above can be extended simply by saying $f(x, y, \dots)$ is continuous iff it is continuous in each of $x, y, \dots$. Those familiar with the product topology can prove that for our special positive topology on $\mathbf{P}\omega$ this is equivalent to being continuous on the product space (continuous in the variables jointly). Those interested only in elementary proofs can calculate out directly from the definition (with the aid of 1.1) that continuity behaves under combinations by substitution [as in: $f(g(x, y), h(y, x, y))$].

Proof of Theorem 1.2. Equation (i) as a functional equation comes down to

$$\{m \mid \exists e_n \subseteq x.\ m \in f(e_n)\} = f(x),$$

which is just another way of writing the definition of continuity. Thus it is indeed true for all x. Next, inclusion (ii) means that if $(n, m) \in u$, then $\exists e_k \subseteq e_n.\ (k, m) \in u$. Clearly all we need to do is take $k = n$. If we also want the converse inclusion to hold, then what we need is condition (iii).

theorem_1_2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.IsGraph
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
import DomainTheory.DataTypesAsLattices.Graph.mem_graph

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_1_2.i {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    funOf (graph f) = f := by
  funext x
  ext m
  constructor
  · intro ⟨n, hn, hp⟩
    obtain ⟨n', m', heq, hm'⟩ := hp
    obtain ⟨rfl, rfl⟩ := pair_inj heq
    exact IsScottContinuous.monotone hf hn hm'
  · intro hm
    have : m ∈ scottUnion f x := by
      rw [← hf.out x]; exact hm
    obtain ⟨n, hn, hmn⟩ := mem_scottUnion.mp this
    exact ⟨n, hn, ⟨n, m, rfl, hmn⟩⟩

theorem theorem_1_2.ii (u : Pomega) : u ⊆ graph (funOf u) := by
  intro p hp
  obtain ⟨n, m, rfl⟩ := exists_pair p
  exact ⟨n, m, rfl, ⟨n, subset_rfl, hp⟩⟩

theorem theorem_1_2.iii (u : Pomega) :
    graph (funOf u) = u ↔ IsGraph u := by
  constructor
  · intro heqGraph
    constructor
    intro k m n hkm hkn
    have hmem : pair k m ∈ graph (funOf u) := heqGraph.symm ▸ hkm
    rcases mem_graph.mp hmem with ⟨k', m', hp, hmfun⟩
    have hk : k = k' := (pair_inj hp).1
    have hm : m = m' := (pair_inj hp).2
    have hmfun' : m ∈ funOf u (e k) := by
      rw [hk, hm]; exact hmfun
    rcases hmfun' with ⟨j, hj, hpj⟩
    have hgraph : pair n m ∈ graph (funOf u) :=
      mem_graph.mpr ⟨n, m, rfl, ⟨j, subset_trans hj hkn, hpj⟩⟩
```

```
    exact heqGraph ▸ hgraph
  · intro hu
    apply subset_antisymm
    · intro p hp
      rcases hp with ⟨n, m, rfl, ⟨k, hk, hkm⟩⟩
      exact hu.out hkm hk
    · exact theorem_1_2.ii u

theorem theorem_1_2 {f : Pomega → Pomega} (hf : IsScottContinuous f) (u : Pomega) :
    funOf (graph f) = f ∧ u ⊆ graph (funOf u) ∧
      (graph (funOf u) = u ↔ IsGraph u) :=
  ⟨theorem_1_2.i hf, theorem_1_2.ii u, theorem_1_2.iii u⟩

end DomainTheory.DataTypesAsLattices
```

3.2.6 Fixed points and least fixed points

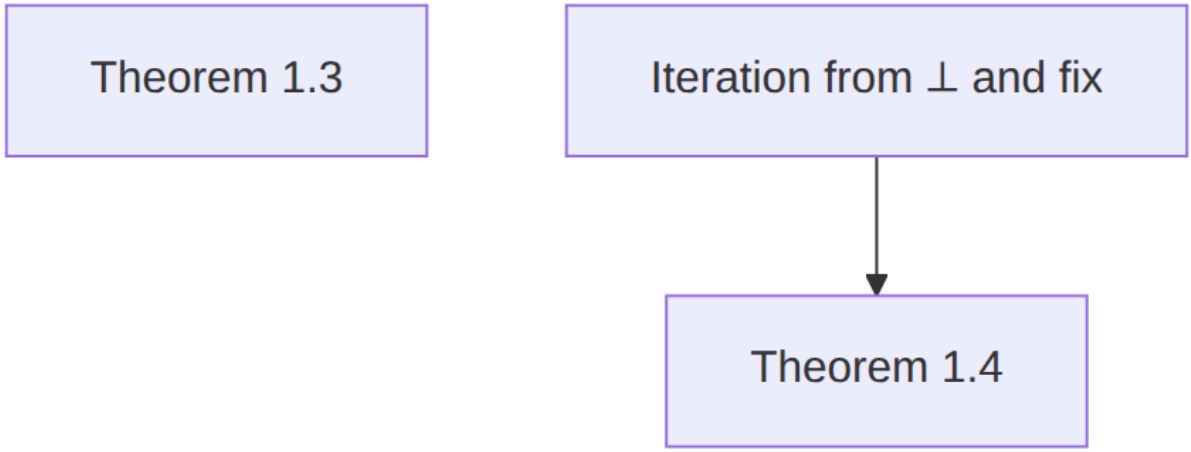


Figure 26: Fixed points and least fixed points.

Theorem 1.3 ✓ **THEOREM 1.3** (The substitution theorem). *Continuous functions of several variables on* $\mathbf{P}\omega$ *are closed under substitution.*

Proof of Theorem 1.3. Substitution is generalized composition of functions of many variables with all possible identifications and permutations of the variables; however, as we are able to define continuity by separating the variables, the argument reduces to a few special cases. The first trick is to take advantage of monotonicity. Thus, suppose $f(x, y)$ is continuous in each of its variables. What can we say of $f(x, x)$, a very special case of substitution? We calculate

$$f(x,x) = \bigcup\{f(e_n, x) \mid e_n \subseteq x\}$$

$$= \bigcup\{f(e_n, e_m) \mid e_n \subseteq x,\ e_m \subseteq x\}.$$

Then if we think of $e_k = e_n \cup e_m$ and realize that $f(e_n, e_m) \subseteq f(e_k, e_k)$, we see that

$$f(x,x) = \bigcup\{f(e_k, e_k) \mid e_k \subseteq x\}.$$

This means that $f(x,x)$ is continuous in x. This same argument works if other variables are present, as in the passage from $f(x,y,z,w)$ to $f(x,x,z,w)$. When an identification of more than two variables is required, as from $f(x,y,z,w)$ to $f(x,x,x,x)$, the principle is just applied several times.

Finally, to show that $f(g(x,y), h(y,x,y))$ is continuous, it is sufficient to show that $f(g(x,y), h(z,u,v))$ is continuous in each of its variables separately. By simply overlooking the remaining variables, this comes down to showing that $f(g(x))$ is continuous if f and g are. But the proof for ordinary composition is very easy with the aid of the characterization theorem (Theorem 1.1).

Formalization. In `theorem_1_3.lean`, namespace `theorem_1_3` holds the special cases; callers use `theorem_1_3.comp`, `theorem_1_3.diag`, etc. The paper's unary composition clause is the top-level theorem `theorem_1_3`.

Lemma (`theorem_1_3.comp`). If f and g are Scott-continuous, then $f \circ g$ is Scott-continuous.

Proof. Membership in $f(g(x))$ is routed through Theorem 1.1 on $g(x)$ and $f(g(x))$, using `mem_scottUnion` and monotonicity. ■

Lemma (`theorem_1_3.diag`). If $F(x,y)$ is continuous in x for each fixed y and in y for each fixed x, then $x \mapsto F(x,x)$ is Scott-continuous; equivalently $F(x,x) = \bigcup\{k \mid \exists n.\, e_n \subseteq x \wedge k \in F(e_n, e_n)\}$.

Proof. Scott's calculation on $f(x,x)$ via $e_n \subseteq x$ and $e_k = e_n \cup e_m$. ■

Lemma (`theorem_1_3.tuple`). Under the same separate-continuity hypothesis on $F(x,y)$, if g and h are Scott-continuous then $x \mapsto F(g(x), h(x))$ is Scott-continuous.

Proof. Apply `theorem_1_3.diag` to $(u,v) \mapsto F(g(u), h(v))$ with `theorem_1_3.comp` on each coordinate. ■

Lemma (`theorem_1_3.nary`). If $F(x,y,z)$ is continuous in each argument separately and g, h, i are Scott-continuous, then $x \mapsto F(g(x), h(x), i(x))$ is Scott-continuous.

Proof. Same pattern: `theorem_1_3.diag` on $(u,v) \mapsto F(g(u), h(u), i(v))$, with `theorem_1_3.tuple` and `theorem_1_3.comp`. ■

Theorem (`theorem_1_3`). If f and g are Scott-continuous, then $x \mapsto f(g(x))$ is Scott-continuous.

Proof. `theorem_1_3.comp`. ■

Corollary (Functionality wrappers). `continuous_on_diag`, `continuous_tuple`, and `continuous_nary` restate the diagonal, binary, and ternary substitution cases for call sites that prefer those names; their proofs invoke `theorem_1_3.diag`, `theorem_1_3.tuple`, and `theorem_1_3.comp` respectively.

`theorem_1_3.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
import DomainTheory.DataTypesAsLattices.Continuous.theorem_1_1

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace theorem_1_3

theorem comp {f g : Pomega → Pomega}
```

```
    (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
    IsScottContinuous (f ∘ g) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    have hk' : k ∈ f (g x) := hk
    have := (theorem_1_1 f).mp hf (g x) (2 ^ k)
    have hfin : e (2 ^ k) ⊆ f (g x) := by
      simpa [e.pow2, Set.singleton_subset_iff] using hk'
    obtain ⟨m, hm, hkm⟩ := this.mp hfin
    have := (theorem_1_1 g).mp hg x m
    obtain ⟨n, hn, hmn⟩ := this.mp hm
    refine mem_scottUnion.mpr ⟨n, hn, ?_⟩
    have : e (2 ^ k) ⊆ f (g (e n)) :=
      subset_trans hkm (IsScottContinuous.monotone hf hmn)
    simpa [e.pow2, Set.singleton_subset_iff] using this
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    exact IsScottContinuous.monotone hf (IsScottContinuous.monotone hg hn) hkn

theorem diag {f : Pomega → Pomega → Pomega}
    (hfx : ∀ y, IsScottContinuous (fun x => f x y))
    (hfy : ∀ x, IsScottContinuous (fun y => f x y))
    {x : Pomega} :
    f x x = ⋃ n, {k | e n ⊆ x ∧ k ∈ f (e n) (e n)} := by
  apply subset_antisymm
  · intro k hk
    have : k ∈ scottUnion (fun z => f z x) x := by
      rw [← (hfx x).out x]; exact hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp this
    have : k ∈ scottUnion (fun y => f (e n) y) x := by
      rw [← (hfy (e n)).out x]; exact hkn
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp this
    refine Set.mem_iUnion.mpr ⟨n ||| m, ?_⟩
    refine ⟨e.or_of_subset hn hm, ?_⟩
    have hle : e n ⊆ e (n ||| m) := by rw [e.or]; exact Set.subset_union_left
    have hre : e m ⊆ e (n ||| m) := by rw [e.or]; exact Set.subset_union_right
    exact IsScottContinuous.monotone (hfy (e (n ||| m))) hre
      (IsScottContinuous.monotone (hfx (e m)) hle hkm)
  · intro k hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp hk
    exact IsScottContinuous.monotone (hfy x) hn
      (IsScottContinuous.monotone (hfx (e n)) hn hkn)

theorem tuple {f : Pomega → Pomega → Pomega}
    (hfx : ∀ y, IsScottContinuous (fun x => f x y))
    (hfy : ∀ x, IsScottContinuous (fun y => f x y))
    {g h : Pomega → Pomega} (hg : IsScottContinuous g) (hh : IsScottContinuous h) :
    IsScottContinuous (fun x => f (g x) (h x)) := by
  constructor
  intro x
  have H := diag (x := x)
    (f := fun u v => f (g u) (h v))
    (fun v => comp (hfx (h v)) hg)
```

```
    (fun u => comp (hfy (g u)) hh)
  ext k
  constructor
  · intro hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ f (g (e n)) (h (e n))} := by rwa [← H]
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr ⟨n, hn, hkn⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ f (g (e n)) (h (e n))} :=
      Set.mem_iUnion.mpr ⟨n, hn, hkn⟩
    rwa [← H] at this

theorem nary {f : Pomega → Pomega → Pomega → Pomega}
    (hf0 : ∀ y z, IsScottContinuous (fun x => f x y z))
    (hf1 : ∀ x z, IsScottContinuous (fun y => f x y z))
    (hf2 : ∀ x y, IsScottContinuous (fun z => f x y z))
    {g h i : Pomega → Pomega}
    (hg : IsScottContinuous g) (hh : IsScottContinuous h) (hi : IsScottContinuous i) :
    IsScottContinuous (fun x => f (g x) (h x) (i x)) := by
  constructor
  intro x
  have H := diag (x := x)
    (f := fun u v => f (g u) (h u) (i v))
    (fun v => tuple (fun y => hf0 y (i v)) (fun x => hf1 x (i v)) hg hh)
    (fun u => comp (hf2 (g u) (h u)) hi)
  ext k
  constructor
  · intro hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ f (g (e n)) (h (e n)) (i (e n))} := by
      rwa [← H]
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr ⟨n, hn, hkn⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ f (g (e n)) (h (e n)) (i (e n))} :=
      Set.mem_iUnion.mpr ⟨n, hn, hkn⟩
    rwa [← H] at this

end theorem_1_3

/-- Scott Theorem 1.3 (unary composition case). -/
theorem theorem_1_3 {f g : Pomega → Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
    IsScottContinuous (fun x => f (g x)) :=
  theorem_1_3.comp hf hg

end DomainTheory.DataTypesAsLattices
```

Iteration from $\bot$ and fix The other general fact about continuous functions that we shall use constantly concerns fixed points, whose existence can be proved using a well-known method.

Scott's least fixed point is built from iterating f starting at the empty set $\varnothing$ (our $\bot$):

$$\mathbf{fix}(f) = \bigcup\{f^n(\varnothing) \mid n \in \omega\},$$

where $f^0(\varnothing) = \varnothing$ and $f^{n+1}(\varnothing) = f(f^n(\varnothing))$.

Definition (`iterateBot`, `fix`). For $f : \mathbf{P}\omega \to \mathbf{P}\omega$, set $\mathrm{iterateBot}\ f\ 0 = \bot$ and $\mathrm{iterateBot}\ f\ (n+1) = f(\mathrm{iterateBot}\ f\ n)$; then $\mathrm{fix}\ f = \bigcup_n \mathrm{iterateBot}\ f\ n$.

Lemma (`iterateBot_mono`). If f is Scott-continuous, then $\mathrm{iterateBot}\ f\ n \subseteq \mathrm{iterateBot}\ f\ (n+1)$ for every n.

Proof. By induction: the base case is $\bot \subseteq f(\bot)$; the step applies monotonicity of f to the induction hypothesis. ■

`iterateBot.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.eDense
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def iterateBot (f : Pomega → Pomega) : ℕ → Pomega
  | 0 => botElem
  | n + 1 => f (iterateBot f n)

end DomainTheory.DataTypesAsLattices
```

`fix.lean`

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def fix (f : Pomega → Pomega) : Pomega :=
  ⋃ n, iterateBot f n

end DomainTheory.DataTypesAsLattices
```

Theorem 1.4 **THEOREM 1.4** (The fixed-point theorem). *Every continuous function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ has a least fixed point given by the formula $\mathbf{fix}(f) = \bigcup\{f^n(\varnothing) \mid n \in \omega\}$ (see iteration from $\bot$).*

Actually **fix** is a *functional* with continuity properties of its own. We shall not give the required definitions here because they can be more easily derived from the construction of the model given in the next section.

For those familiar with the abstract theory of topological spaces we give in conclusion two general facts about continuous functions with values in $\mathbf{P}\omega$ which indicate the scope and generality of our method.

Proof of Theorem 1.4. This well-known fact holds for continuous functions on many kinds of chain-complete partial orderings; but $\mathbf{P}\omega$ illustrates the idea well enough. Suppose f had a fixed point $x = f(x)$.

Then since $\varnothing \subseteq x$ and f is monotonic, we see that $f(\varnothing) \subseteq f(x) = x$. But then again, $f(f(\varnothing)) \subseteq f(x) = x$; and so by induction, $f^n(\varnothing) \subseteq x$. This proves that $\mathbf{fix}(f) \subseteq x$; and thus if $\mathbf{fix}(f)$ is a fixed point, it must be the least one. To prove that it is a fixed point, we need a fact that will often be useful:

LEMMA (`chain_lemma`). *If $x_n \subseteq x_{n+1}$ for all n, and if f is continuous, then*

$$f\Big(\bigcup\{x_n \mid n \in \omega\}\Big) = \bigcup\{f(x_n) \mid n \in \omega\}.$$

Proof. By monotonicity, the inclusion holds in one direction. Suppose $e_m \subseteq f(\bigcup\{x_n \mid n \in \omega\})$. Then by Theorem 1.1 we have $e_m \subseteq f(e_k)$ for some $e_k \subseteq \bigcup\{x_n \mid n \in \omega\}$. Because e_k is finite and the sequence is increasing, we can argue that $e_k \subseteq x_n$ for some n. But then $f(e_k) \subseteq f(x_n)$. This shows that $e_m \subseteq \bigcup\{f(x_n) \mid n \in \omega\}$ and proves the inclusion in the other direction. (*Exercise:* Does this property characterize continuous functions?)

Proof of Theorem 1.4 concluded. Noting that $f^n(\varnothing) \subseteq f^{n+1}(\varnothing)$ holds for all n, we can calculate:

$$f(\mathbf{fix}(f)) = \bigcup\{f(f^n(\varnothing)) \mid n \in \omega\} = \bigcup\{f^{n+1}(\varnothing) \mid n \in \omega\}.$$

But this is just $\mathbf{fix}(f)$, since the only term left out is $f^0(\varnothing) = \varnothing$.

Formalization. `theorem_1_4` is Scott's statement; the proof uses `chain_lemma` (via `exists_index_of_finset` and `chain_mono`).

`theorem_1_4.lean`

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.chain_lemma
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot_mono

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_1_4 {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    f (fix f) = fix f ∧ ∀ x, f x = x → fix f ⊆ x := by
  constructor
  ·
    have hchain := chain_lemma hf (iterateBot f) (iterateBot_mono hf)
    unfold fix
    rw [hchain]
    ext k
    constructor
    · intro hk
      obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
      exact Set.mem_iUnion.mpr ⟨n + 1, hkn⟩
    · intro hk
      obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
      cases n with
      | zero => simp [iterateBot, botElem] at hkn
      | succ n => exact Set.mem_iUnion.mpr ⟨n, hkn⟩
  · intro x hx k hk
    obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
    have hsub : ∀ n, iterateBot f n ⊆ x := by
      intro n
      induction n with
```

```
      | zero => simp [iterateBot, botElem]
      | succ n ih =>
        simpa [iterateBot, hx] using IsScottContinuous.monotone hf ih
    exact hsub n hkn

end DomainTheory.DataTypesAsLattices
```

3.2.7 Extension and embedding of continuous maps

memOpen.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.basicNhhd
import Mathlib.Data.Finset.Attr
import Mathlib.Tactic.Bound.Init
import Mathlib.Tactic.Continuity.Init
import Mathlib.Tactic.SetLike

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def memOpen (k : ℕ) : Set Pomega := {x | k ∈ x}

namespace memOpen

theorem eq_basicNhhd (k : ℕ) : memOpen k = basicNhhd (2 ^ k) := by
  ext x
  simp [memOpen, basicNhhd, e.pow2, Set.singleton_subset_iff]

end memOpen

end DomainTheory.DataTypesAsLattices
```

Basic.lean

```
module

public import Mathlib.Topology.Order
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
import Mathlib.Tactic.Bound.Init

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

instance : TopologicalSpace Pomega :=
  .generateFrom (Set.range memOpen)
end DomainTheory.DataTypesAsLattices
```

extend.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
public import Mathlib.Data.Set.Lattice.Indexed
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function

def extend {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X → Pomega) (y : Y) : Pomega :=
  ⋃₀ { t | ∃ U : Set Y, IsOpen U ∧ y ∈ U ∧ t = ⋂₀ (f '' {x : X | ↑x ∈ U}) }

namespace extend

theorem mem {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X → Pomega) {y : Y} {k : ℕ} :
    k ∈ extend (Y := Y) (X := X) f y ↔
      ∃ U : Set Y, IsOpen U ∧ y ∈ U ∧ ∀ x : X, ↑x ∈ U → k ∈ f x := by
  constructor
  · intro hk
    rcases Set.mem_sUnion.mp hk with ⟨t, ⟨U, hU, hy, rfl⟩, hkt⟩
    refine ⟨U, hU, hy, ?_⟩
    intro x hx
    exact Set.mem_sInter.mp hkt (f x) ⟨x, hx, rfl⟩
  · intro ⟨U, hU, hy, hall⟩
    refine Set.mem_sUnion.mpr ⟨⋂₀ (f '' {x : X | ↑x ∈ U}), ⟨U, hU, hy, rfl⟩, ?_⟩
    intro t ht
    rcases ht with ⟨x, hx, rfl⟩
    exact hall x hx

theorem isOpen_mem {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X → Pomega) (k : ℕ) :
    IsOpen {y : Y | k ∈ extend (Y := Y) (X := X) f y} := by
  have : {y : Y | k ∈ extend (Y := Y) (X := X) f y} =
      ⋃ U ∈ {U : Set Y | IsOpen U ∧ ∀ x : X, ↑x ∈ U → k ∈ f x}, U := by
    ext y
    constructor
    · intro hy
      obtain ⟨U, hU, hyU, hall⟩ := (mem f).mp hy
      exact Set.mem_iUnion₂.mpr ⟨U, ⟨hU, hall⟩, hyU⟩
    · intro hy
      obtain ⟨U, hU, hyU⟩ := Set.mem_iUnion₂.mp hy
      exact (mem f).mpr ⟨U, hU.1, hyU, hU.2⟩
  rw [this]
  exact isOpen_biUnion fun _ hU => hU.1

end extend
```

```
end DomainTheory.DataTypesAsLattices
```

basicNhhd_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import Mathlib.Order.SetNotation
import Mathlib.Data.Finset.Attr
import Mathlib.Data.Set.Lattice.Indexed
import Mathlib.Tactic.Bound.Init
import Mathlib.Tactic.Continuity.Init
import Mathlib.Tactic.SetLike

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace basicNhhd

theorem eq_iInter (n : ℕ) :
    basicNhhd n = ⋂ k ∈ e n, memOpen k := by
  ext x
  simp [basicNhhd, memOpen, Set.subset_def]

end basicNhhd

end DomainTheory.DataTypesAsLattices
```

embed.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
public import DomainTheory.DataTypesAsLattices.Continuous.Basic
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function

def embed {X : Type*} (U : ℕ → Set X) (x : X) : Pomega :=
  {n | x ∈ U n}

namespace embed

theorem continuous {X : Type*} [TopologicalSpace X]
    (U : ℕ → Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    Continuous (embed U) :=
  continuous_generateFrom_iff.2 fun t ⟨n, hn⟩ => by
```

```
    subst hn
    change IsOpen (U n)
    exact hbasis.isOpen ⟨n, rfl⟩

theorem inducing {X : Type*} [TopologicalSpace X]
    (U : ℕ → Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    IsInducing (embed U) := by
  refine ⟨?_⟩
  rw [hbasis.eq_generateFrom]
  change generateFrom (Set.range U) =
    (generateFrom (Set.range memOpen)).induced (embed U)
  rw [induced_generateFrom_eq]
  congr 1
  ext V
  constructor
  · rintro ⟨n, rfl⟩
    exact ⟨memOpen n, ⟨n, rfl⟩, rfl⟩
  · rintro ⟨_, ⟨n, rfl⟩, rfl⟩
    exact ⟨n, rfl⟩

theorem injective {X : Type*} [TopologicalSpace X] [T0Space X]
    (U : ℕ → Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    Injective (embed U) := by
  intro x y hxy
  have hmem : ∀ n, x ∈ U n ↔ y ∈ U n := fun n =>
    show n ∈ embed U x ↔ n ∈ embed U y by rw [hxy]
  refine Inseparable.eq ?_
  refine inseparable_iff_forall_isOpen.mpr ?_
  intro V hV
  constructor
  · intro hx
    obtain ⟨t, ⟨n, rfl⟩, hxt, htV⟩ :=
      hbasis.mem_nhds_iff.mp (hV.mem_nhds hx)
    exact htV ((hmem n).mp hxt)
  · intro hy
    obtain ⟨t, ⟨n, rfl⟩, hyt, htV⟩ :=
      hbasis.mem_nhds_iff.mp (hV.mem_nhds hy)
    exact htV ((hmem n).mpr hyt)

end embed

end DomainTheory.DataTypesAsLattices
```

exists_basicNhhd_subset_of_generateOpen.lean

```
module

public import Mathlib.Topology.Order
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
import Mathlib.Tactic.Bound.Init
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace
```

```
theorem exists_basicNhhd_subset_of_generateOpen {U : Set Pomega}
    (hU : GenerateOpen (Set.range memOpen) U) {x : Pomega} (hx : x ∈ U) :
    ∃ n, e n ⊆ x ∧ basicNhhd n ⊆ U := by
  induction hU generalizing x with
  | basic s hs =>
    obtain ⟨k, rfl⟩ := hs
    refine ⟨2 ^ k, ?_, ?_⟩
    · simpa [memOpen, e.pow2, Set.singleton_subset_iff] using hx
    · rw [← memOpen.eq_basicNhhd]
  | univ =>
    exact ⟨0, by simp [e.zero], by simp [basicNhhd.zero]⟩
  | inter _s _t _hs _ht ihs iht =>
    obtain ⟨n, hn, hnU⟩ := ihs hx.1
    obtain ⟨m, hm, hmU⟩ := iht hx.2
    refine ⟨n ||| m, e.or_of_subset hn hm, ?_⟩
    rw [← basicNhhd.inter]
    exact Set.inter_subset_inter hnU hmU
  | sUnion S _hS ih =>
    obtain ⟨V, hV, hxV⟩ := hx
    obtain ⟨n, hn, hnV⟩ := ih V hV hxV
    exact ⟨n, hn, hnV.trans (Set.subset_sUnion_of_mem hV)⟩

end DomainTheory.DataTypesAsLattices
```

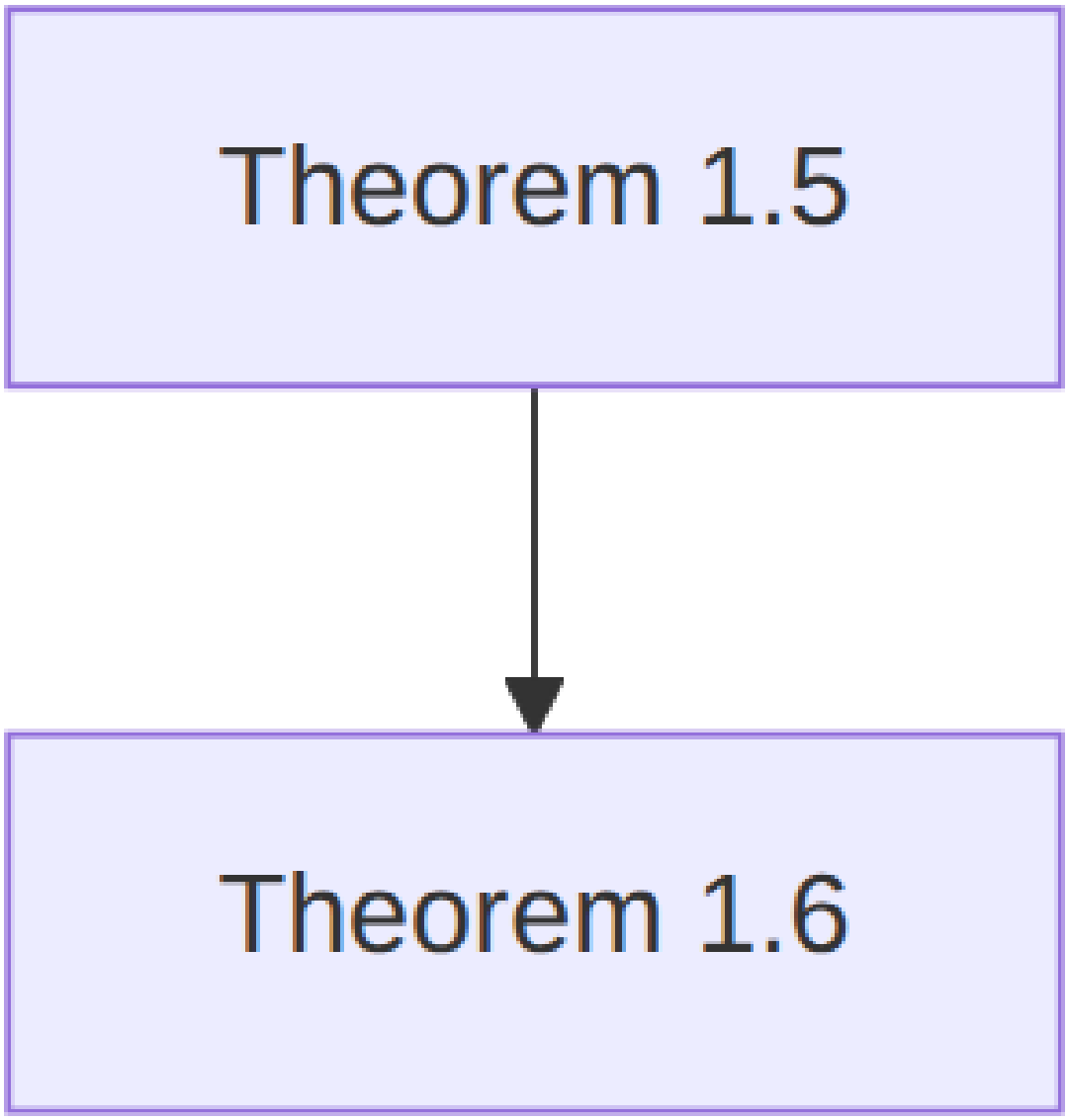


Figure 27: Extension and embedding of continuous maps.

Theorem 1.5 **THEOREM 1.5** (The extension theorem). *Let X and Y be arbitrary topological spaces where $X \subseteq Y$ as a subspace. Then every continuous function $f : X \to \mathbf{P}\omega$ can be extended to a continuous function $\bar{f} : Y \to \mathbf{P}\omega$ defined by the equation:*

$$\bar{f}(y) = \bigcup\{\bigcap\{f(x) \mid x \in X \cap U\} \mid y \in U\},$$

where $y \in Y$, and U ranges over the open subsets of Y.

Proof of Theorem 1.5. The function $\bar{f}$ is clearly well-defined even when $y \in Y$ has a neighborhood U where $X \cap U = \varnothing$: in that case $\bar{f}(y) = \omega$ by convention on the meaning of $\bigcap$ in $\mathbf{P}\omega$. In case $x \in X$, it is obvious that $\bar{f}(x) \subseteq f(x)$. For the opposite inclusion, suppose that $m \in f(x)$. Because f is continuous and $\{z \mid m \in z\}$ is open in $\mathbf{P}\omega$, there is an open subset V of X such that $x' \in V$ always implies that $m \in f(x')$. But X is a subspace of Y, so $V = X \cap U$ for some open subset U of Y. Thus we can see why $m \in \bar{f}(x)$. It remains to show that $\bar{f}$ is itself continuous.

We must show that the inverse image under $\bar{f}$ of every open subset of $\mathbf{P}\omega$ is open in Y. But the open subsets of $\mathbf{P}\omega$ are unions of finite intersections of sets of the form $\{z \mid m \in z\}$. Thus it is enough to show that $\{y \mid m \in \bar{f}(y)\}$ is always open in Y. But this set equals $\bigcup\{U \mid m \in \bigcap\{f(x) \mid x \in X \cap U\}\}$, which being a union of open sets is open. Note that what we have proved is that $\bar{f}$ is continuous no matter what function f is given; however, if f is not continuous, then $\bar{f}$ cannot be an extension of f.

For readers not as familiar with general topology we note that the idea of Theorem 1.5 can be turned into a *definition*. Suppose $X \subseteq \mathbf{P}\omega$ is a subset of $\mathbf{P}\omega$. It becomes a subspace with the relative topology. What are the continuous functions $f : X \to \mathbf{P}\omega$? From Theorem 1.5 we see that a necessary and sufficient condition is that $\bar{f} : \mathbf{P}\omega \to \mathbf{P}\omega$ be an extension of f. Thus for $x \in X$ we can write the equation $f(x) = \bar{f}(x)$ as a biconditional:

$$m \in f(x) \quad \text{iff} \quad \exists e_n \subseteq x \; \forall x' \in X \; [e_n \subseteq x' \text{ implies } m \in f(x')],$$

which is to hold for all $m \in \omega$. This form of the definition of continuity on a subspace is more complicated than the original definition, because in general $e_n \notin X$ and we cannot write $f(e_n)$.

isOpen_memOpen.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.Basic
import Mathlib.Tactic.Bound.Init

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem isOpen_memOpen (k : ℕ) : IsOpen (memOpen k) :=
  TopologicalSpace.isOpen_generateFrom_of_mem ⟨k, rfl⟩

end DomainTheory.DataTypesAsLattices
```

isOpen_basicNhhd.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.Basic
import DomainTheory.DataTypesAsLattices.Continuous.isOpen_memOpen
import Mathlib.Tactic.Bound.Init

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
open TopologicalSpace

theorem isOpen_basicNhhd (n : ℕ) : IsOpen (basicNhhd n) := by
  obtain ⟨s, hs⟩ := (e.finite n).exists_finset_coe
  have h : basicNhhd n = ⋂ k ∈ s, memOpen k := by
    ext x
    constructor
    · intro hx
      refine Set.mem_iInter₂.mpr ?_
      intro k hk
      have : k ∈ e n := by
        have : k ∈ (s : Set ℕ) := hk
        rwa [hs] at this
      exact hx this
    · intro hx k hk
      have : k ∈ s := by
        have : k ∈ e n := hk
        rwa [← hs] at this
      exact Set.mem_iInter₂.mp hx k this
  rw [h]
  exact isOpen_biInter_finset fun _ _ => isOpen_memOpen _

end DomainTheory.DataTypesAsLattices
```

theorem_1_5.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.isOpen_memOpen
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Embedding.embed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function

theorem theorem_1_5 {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X → Pomega) : Continuous (extend (Y := Y) (X := X) f) := by
  refine continuous_generateFrom_iff.2 ?_
  rintro _ ⟨k, rfl⟩
  exact extend.isOpen_mem f k

theorem theorem_1_5.extension {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X → Pomega) (hf : Continuous f) (x : X) :
    extend (Y := Y) (X := X) f ↑x = f x := by
  apply subset_antisymm
  · intro k hk
```

```
    obtain ⟨U, _, hxU, hall⟩ := (extend.mem f).mp hk
    exact hall x hxU
  · intro k hk
    have hpre : IsOpen (f ^-1' memOpen k) :=
      IsOpen.preimage hf (isOpen_memOpen k)
    obtain ⟨U, hU, hrel⟩ := isOpen_induced_iff.mp hpre
    refine (extend.mem f).mpr ⟨U, hU, ?_, ?_⟩
    · have : x ∈ Subtype.val ^-1' U := by
        rwa [hrel]
      exact this
    · intro z hz
      have : z ∈ f ^-1' memOpen k := by
        rwa [← hrel]
      exact this

end DomainTheory.DataTypesAsLattices
```

Theorem 1.6 **THEOREM 1.6** (The embedding theorem). *Every T_0-space X with a countable basis $\{U_n \mid n \in \omega\}$ for its topology can be embedded in $\mathbf{P}\omega$ by the continuous function $\varepsilon : X \to \mathbf{P}\omega$ defined by the equation:*

$$\varepsilon(x) = \{n \mid x \in U_n\}.$$

Technically the T_0-hypothesis is what is needed to show that ε is one-to-one. The upshot of these two theorems is that in looking for (reasonable) topological structures we can confine attention to the subspaces of $\mathbf{P}\omega$ and to continuous functions defined on *all* of $\mathbf{P}\omega$. Thus the emphasis on a single space is justified structurally. What we shall see in the remainder of this work is that the use of a single space is also justified practically because the required subspaces and functions can be *defined* in very simple ways by a natural method of equations.

Proof of Theorem 1.6. What T_0 means is that every point of X is uniquely determined by its neighborhoods. Now $\varepsilon(x)$ just tells you the set of indices of the (basic) neighborhoods of x. Thus it is clear that ε is one-to-one. To prove that it is continuous, we need only note:

$$\{x \mid n \in \varepsilon(x)\} = U_n,$$

which is always open. To show that ε is an embedding, we must finally check that the *images* of the open sets U_n are open in $\varepsilon(X)$. This comes down to showing:

$$\varepsilon(U_n) = \varepsilon(X) \cap \{z \mid n \in z\},$$

which is clear.

theorem_1_6.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Embedding.extend
```

```
public import DomainTheory.DataTypesAsLattices.Embedding.embed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function

theorem theorem_1_6 {X : Type*} [TopologicalSpace X] [T0Space X]
    (U : ℕ → Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    IsEmbedding (embed U) where
  toIsInducing := embed.inducing U hbasis
  injective := embed.injective U hbasis

theorem theorem_1_6.injective_continuous {X : Type*} [TopologicalSpace X]
    [T0Space X] (U : ℕ → Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    Injective (embed U) ∧ Continuous (embed U) :=
  ⟨embed.injective U hbasis, embed.continuous U hbasis⟩

end DomainTheory.DataTypesAsLattices
```

3.3 Computability and definability

Historical note. The language LAMBDA is Scott's. Plotkin and Rogers already used graphs, application, and conversion; the abstraction operation is what this section supplies.

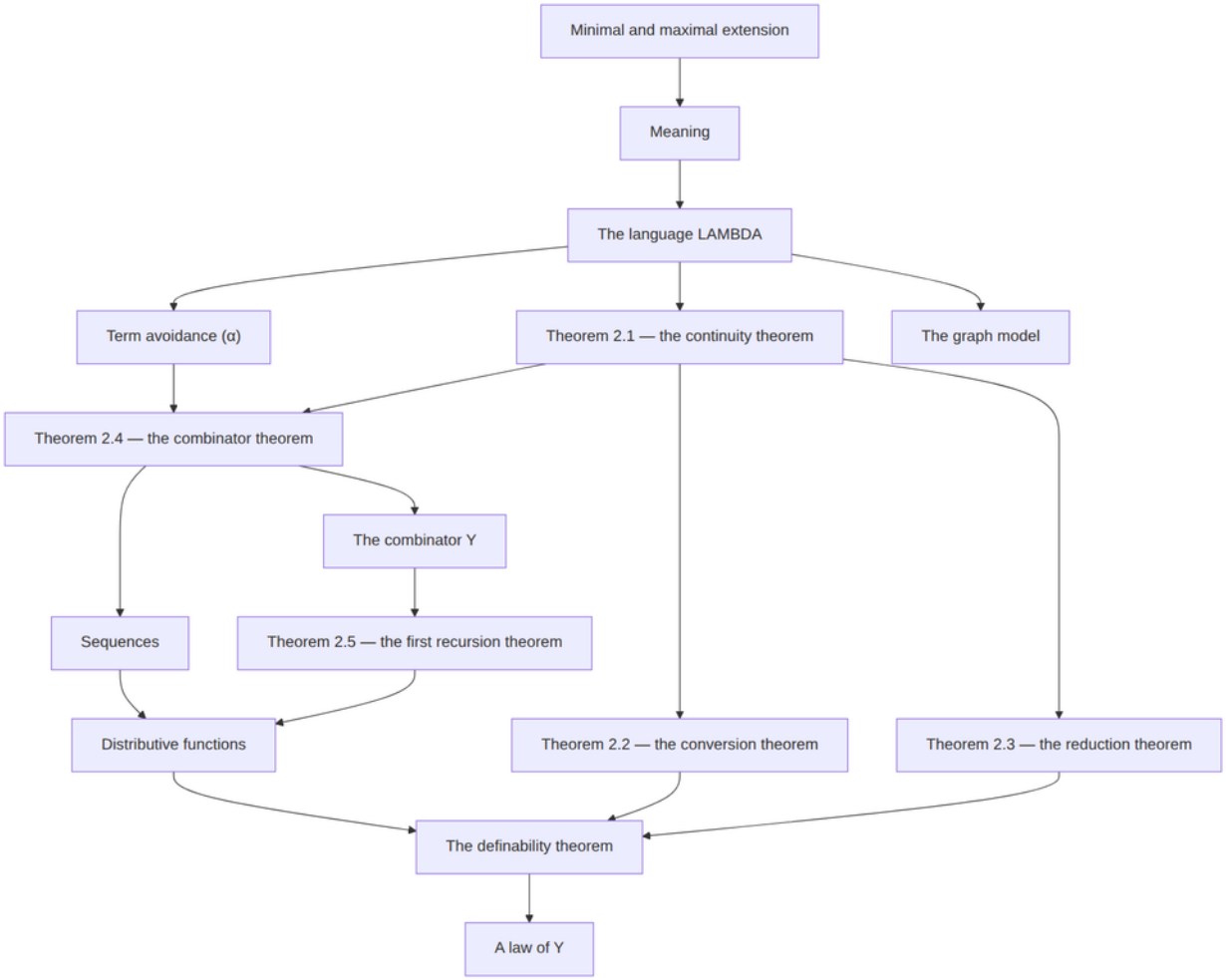


Figure 28: Computability and definability.

3.3.1 Minimal and maximal extension ✓

As a subspace $\omega \subseteq \mathbf{P}\omega$ is discrete. This implies that arbitrary functions $p : \omega \to \omega$ are continuous. Note that $p : \omega \to \mathbf{P}\omega$ as well, because $\omega \subseteq \mathbf{P}\omega$. By Theorem 1.5 we can extend p continuously to $\bar{p} : \mathbf{P}\omega \to \mathbf{P}\omega$. The formula given produces this function:

$$\bar{p}(x) = \begin{cases} \bigcap\{p(n) \mid n \in \omega\} & \text{if } x = \varnothing; \\ p(n) & \text{if } x = n \in \omega; \\ \omega & \text{otherwise.} \end{cases} \tag{2.1}$$

This is a rather abrupt extension of p (the *maximal extension*); a more gradual, continuous extension (the *minimal extension*) is determined by this equation:

$$\hat{p}(x) = \bigcup\{p(n) \mid n \in x\}. \tag{2.2}$$

The same formulae work for *all* multiple-valued functions $p : \omega \to \mathbf{P}\omega$. Functions like $f = \hat{p}$ are exactly characterized as being those continuous functions $f : \mathbf{P}\omega \to \mathbf{P}\omega$ which in addition are *distributive* in the sense of these equations:

$$f(x \cup y) = f(x) \cup f(y) \quad \text{and} \quad f(\varnothing) = \varnothing.$$

Definition (`maxExtend`, `minExtend`). For $p : \omega \to \mathbf{P}\omega$ and $x \in \mathbf{P}\omega$, the maximal extension is $\mathrm{maxExtend}\, p\, x$ (Scott's piecewise $\bar{p}(x)$ in (2.1)), and the minimal extension is $\mathrm{minExtend}\, p\, x = \bigcup\{p(n) \mid n \in x\}$.

Theorem (`eq_2_1_empty`). $\mathrm{maxExtend}\, p\, \bot = \bigcap_n p(n)$.

Proof. On $\bot$, membership in $\mathrm{maxExtend}\, p\, \bot$ is equivalent to $k \in p(n)$ for every n. ■

Theorem (`eq_2_1_singleton`). For $n \in \omega$, $\mathrm{maxExtend}\, p\, (\mathrm{ofNat}\, n) = p(n)$.

Proof. Case-split on the three disjuncts in the definition of $\mathrm{maxExtend}$; the singleton case is the only viable branch. ■

Theorem (`eq_2_1_else`). If $x \neq \bot$ and x is not a singleton $\mathrm{ofNat}\, n$, then $\mathrm{maxExtend}\, p\, x = \top$.

Proof. Every k satisfies the third disjunct in the definition. ■

Theorem (`minExtend.bot`). $\mathrm{minExtend}\, p\, \bot = \bot$.

Theorem (`minExtend.union`). $\mathrm{minExtend}\, p\, (x \cup y) = \mathrm{minExtend}\, p\, x \cup \mathrm{minExtend}\, p\, y$.

Theorem (`minExtend.isScottContinuous`). $\mathrm{minExtend}\, p$ is Scott-continuous for every $p : \omega \to \mathbf{P}\omega$.

Proof. Compare membership in $\mathrm{minExtend}\, p\, x$ with membership in $\mathrm{scottUnion}\, (\mathrm{minExtend}\, p)\, x$ via finite singleton witnesses e_{2^n}. ■

3.3.2 Meaning ✓

The purpose of the first section was to introduce in a simple-minded way the basic notions about the topology of $\mathbf{P}\omega$ and its continuous functions. In this section we wish to present the details of a powerful language for defining *particular* functions—especially computable functions—and initiate the study of the *use* of these functions. This study is then extended in different ways in the following sections.

Before looking at the language, a short discussion of the "meaning" of the elements $x \in \mathbf{P}\omega$ will be helpful from the point of view of motivation. Now in itself $x \in \mathbf{P}\omega$ is a set, but this does not reveal its *meaning*. Actually x has no "fixed" meaning, because it can be *used* in strikingly different ways; we look for meaning here solely in terms of use. Nevertheless it is possible to give some coherent guidelines.

In the first place it is convenient to let the singleton subsets $\{n\} \in \mathbf{P}\omega$ stand for the corresponding integers. In fact, we shall enforce by convention the equation $n = \{n\}$ as a way of simplifying notation. In this way, $\omega \subseteq \mathbf{P}\omega$ as a subset. (Note that our convention conflicts with such set-theoretical equations as $5 = \{0, 1, 2, 3, 4\}$. What we have done is to abandon the usual set-theoretical conventions in favor of a slight redefinition of *set of integers* which produces a more helpful convention for present purposes.) So, if we choose, a singleton "means" a single integer. The next

question is what a "large" set $x \in \mathbf{P}\omega$ could mean. Here is an answer: if we write:

$$x = \{0, 5, 17\} = 0 \cup 5 \cup 17,$$

we are thinking of x as a *multiple integer*. This is especially useful in the case of multiple-valued function where we can write:

$$f(a) = 0 \cup 5 \cup 17.$$

Then "$m \in f(a)$" can be interpreted as "m is one value of f at a." Now $a \in \mathbf{P}\omega$, too, and so it is a multiple integer also. This brings us to an important point.

A multiple integer is (often) *more* than just the (random) collection of its elements. From the definition of continuity, $m \in f(a)$ is equivalent to $m \in f(e_n)$ with $e_n \subseteq a$. We may not be able to reduce this to $m \in f(\{n\})$ with $n \in a$ without additional assumptions on f. Indeed we shall take advantage of the feature of continuous functions whereby the elements of an argument a can join in *cooperation* in determining $f(a)$. Needless to say, continuity implies that the cooperation cannot extend beyond *finite* configurations, and so we can say that a is the union (or limit) of its finite subsets. However, finitary cooperation will be found to be quite a powerful notion.

Where does this interpretation leave the empty set $\varnothing$? When we write "$f(a) = \varnothing$" we can read this as "f has no value at a", or "f is undefined at a". In this case $f(a)$ exists (as a set), but it is not "defined" as an integer. Single- (or singleton-) valued functions are "well-defined", but multiple-valued functions are rather "over-defined".

How does this interpretation fit in with monotonicity? In case $a \subseteq b$ and $m \in f(a)$, then we must have $m \in f(b)$. We can read "$a \subseteq b$" as "b is an *improvement of* a" or "b is *better-defined than* a." The point of monotonicity is that the better we define an argument, the better we define a value. "Better" does not imply "well" (that is, singleton-valuedness), and overdefinedness may well creep in. This is not the fault of the function; it is our fault for not choosing a different function.

The sets $\varnothing$ and ω play special roles. When we consider them as elements of $\mathbf{P}\omega$ we shall employ the notation:

$$\bot = \varnothing \quad \text{and} \quad \top = \omega$$

The element $\bot$ is the most "undefined" integer, and $\top$ is the most "overdefined". All others are in between.

One last general point on meaning: suppose $x \in \mathbf{P}\omega$ and $k \in x$. Then $k = (n, m)$ for suitable (uniquely determined) integers n and m. That is to say, every element of x can be regarded as an ordered pair; thus, x can be used as a relation. Such an operation as

$$(2.3) \quad x; y = \{(n, l) \mid \exists m.\, (n, m) \in x,\, (m, l) \in y\}$$

is then a continuous function of two variables that treats both x and y as relations. On the other hand we could define a quite different continuous function such as

$$(2.4) \quad x + y = \{n + m \mid n \in x,\, m \in y\}$$

which treats both x and y arithmetically. The only reason we shall probably never write $(x + y); x$ again is that the values of this perfectly well-defined continuous function are, for the most part, quite uninteresting. There is, however, no theoretical reason why we cannot use the same set with several different "meanings" in the same formula. Of course if we do so, it is to be expected that we will show the point of doing this in the special case. We turn now to the definition of the general language for defining all such functions.

3.3.3 The language LAMBDA ✓

The syntax and semantics of the language LAMBDA are set out in Table 2. The syntax is indicated on the left, and the meanings of the combinations are shown on the right as subsets of $\mathbf{P}\omega$. This is the basic language and could have been given (less understandably) in terms of combinators (see Theorem 2.4). It

is, however, a very primitive language, and we shall require many definitions before we can see why such functions as in (2.3) and (2.4) are themselves definable.

TABLE 2

The language LAMBDA

$$0 = \{0\}$$

$$x + 1 = \{n + 1 \mid n \in x\}$$

$$x - 1 = \{n \mid n + 1 \in x\}$$

$$z \supset x, y = \{n \in x \mid 0 \in z\} \cup \{m \in y \mid \exists k.\, k + 1 \in z\}$$

$$u(x) = \{m \mid \exists e_n \subseteq x.\, (n, m) \in u\}$$

$$\lambda x.\, \tau = \{(n, m) \mid m \in \tau[e_n/x]\}$$

The definition has been left somewhat informal in hopes that it will be more understandable. In the above, τ is any term of the language. LAMBDA is type-free and allows any combination to be made by substitution into the given functions. There is one primitive constant (0); there are two unary functions $(x + 1,\ x - 1)$; there is one binary function $(u(x))$ and one ternary function $(z \supset x, y)$; finally there is one variable binding operator $(\lambda x.\, \tau)$. The first three equations have obvious sense. In the fourth, $z \supset x, y$ is McCarthy's conditional expression (a test for zero). Next $u(x)$ defines *application* (u is treated as a graph and x as a set), and $\lambda x.\, \tau$ is *functional abstraction* (compare the definition of **fun**). In defining $\lambda x.\, \tau$, we use $\tau[e_n/x]$ as a shorthand for evaluating the term τ when the variable x is given the value e_n.

Note that the functions are all multiple-valued. Thus we have such a result as:

$$(2.5) \quad (6 \cup 10) + 1 = 7 \cup 11.$$

The partial character of subtraction has expression as:

$$(2.6) \quad 0 - 1 = \bot.$$

We shall see how to define $+$ and $-$ in LAMBDA later. The conditional could also have been defined by cases:

$$(2.7) \quad z \supset x, y = \begin{cases} \bot & \text{if } z = \bot; \\ x & \text{if } z = 0; \\ y & \text{if } 0 \notin z \neq \bot; \\ x \cup y & \text{if } 0 \in z \neq 0. \end{cases}$$

We say that a LAMBDA-term τ defines a function of its free variables (at least). Other results depend on this fundamental proposition:

Formalization (`Language.lean`). Below, each Lean name is the library entry; proofs in the file are short unless noted.

Definition (`Comb`). The inductive type of combinator terms: variables x_n, constants 0, suc, pred, cond, K, S, and binary application.

Definition (`Term`). The inductive type of LAMBDA terms: variables x_n; constant 0; unary successor and predecessor; ternary conditional $\mathrm{cond}(z, x, y)$ for $z \supset x, y$; binary application $\mathrm{app}(u, x)$ for $u(x)$; and abstraction $\mathrm{lam}(i, \tau)$ for $\lambda x_i.\, \tau$.

Definition (`ofNat`). For $n \in \omega$, $\mathrm{ofNat}\, n = \{n\} \subseteq \mathbf{P}\omega$.

Theorem (`mem_ofNat`). For all $n, k \in \omega$, $k \in \mathrm{ofNat}\, n \leftrightarrow k = n$.

Proof. By definition of singleton membership. ■

Definition (`predSet`). For $x \in \mathbf{P}\omega$, $\mathrm{predSet}(x) = \{k \in \omega \mid k + 1 \in x\}$ (Table 2's $x - 1$ on sets).

Definition (`succSet`). For $x \in \mathbf{P}\omega$, $\mathrm{succSet}(x) = \{k \in \omega \mid \exists n \in x.\, k = n + 1\}$ (Table 2's $x + 1$ on sets).

Theorem (`e_one`). The standard dense singleton e_1 satisfies $e_1 = \{0\}$.

Proof. $e_1 = e(2^0) = \{0\}$ by the definition of e. ■

Definition (`envSet`). For an environment $\rho : \omega \to \mathbf{P}\omega$, a variable index i, and a value $v \in \mathbf{P}\omega$, $\mathrm{envSet}\, \rho\, i\, v$ is the environment that agrees with ρ except at i, where it is v (formally, a single-point update $\rho[i \mapsto v]$).

Theorem (`envSet.comm`). If $i \neq j$, then for all ρ, v, w,

$$\mathrm{envSet}(\mathrm{envSet}\, \rho\, i\, v)\, j\, w = \mathrm{envSet}(\mathrm{envSet}\, \rho\, j\, w)\, i\, v.$$

Proof. Extensionality on indices; the two updates commute when $i \neq j$. ■

Definition (`Kcomb`). $\mathrm{Kcomb} = \mathbf{graph}(\lambda x.\, \mathbf{graph}(\lambda y.\, x))$, the graph of the K combinator.

Definition (`zeroC`). $\mathrm{zeroC} = \mathrm{ofNat}\, 0$, the denotation of the constant 0.

Definition (`predC`). $\mathrm{predC} = \mathbf{graph}(\mathrm{predSet})$, the graph of predecessor on sets.

Definition (`sucC`). $\mathrm{sucC} = \mathbf{graph}(\mathrm{succSet})$, the graph of successor on sets.

Definition (`rename`). For distinct variable indices treated as a rename $i \rightsquigarrow j$, $\mathrm{rename}\, i\, j : \mathrm{Term} \to \mathrm{Term}$ replaces free occurrences of variable i by j and leaves other constructors unchanged; on $\mathrm{lam}(k, \tau)$ the subterm τ is renamed only when $k \neq i$ (so bound variables are not captured).

Theorem (`triangle_eq_zero`). If $\mathrm{triangle}(w) = 0$, then $w = 0$.

Proof. For $w = n + 1$, the triangular number $T(n + 1) = (n + 1)(n + 2)/2$ is positive, contradicting $T(w) = 0$. ■

Theorem (`pair_zero_zero`). $\mathrm{pair}(0, 0) = 0$.

Theorem (`pair_one_zero`). $\mathrm{pair}(1, 0) = 1$.

Theorem (`pair_eq_right`). If $\mathrm{pair}(n, m) = m$, then $n = 0$ and $m = 0$.

Proof. From $\mathrm{pair}(n, m) = m$ and the identity $\mathrm{pair}(n, m) = \mathrm{triangle}(n+m)+m$, deduce $\mathrm{triangle}(n+m) = 0$, hence $n + m = 0$ by `triangle_eq_zero`, and therefore $n = m = 0$. ■

Theorem (`eq_2_5`). Scott's equation (2.5):

$$\mathrm{succSet}(\mathrm{ofNat}\, 6 \cup \mathrm{ofNat}\, 10) = \mathrm{ofNat}\, 7 \cup \mathrm{ofNat}\, 11.$$

Proof. Extensionality on membership; successor on a singleton $\{n\}$ is $\{n + 1\}$, and union distributes over the operation. ■

Theorem (`eq_2_6`). Scott's equation (2.6): $\mathrm{predSet}(\mathrm{ofNat}\, 0) = \bot$.

Proof. No k satisfies $k + 1 \in \{0\}$. ■

Lemma (`exists_least_mem`). If $a \subseteq \omega$ is nonempty, there exists $k \in a$ such that k is minimal in a (every $i < k$ satisfies $i \notin a$).

Proof. Strong induction on the witness $n \in a$: either n is already minimal below n, or some smaller element of a yields a minimal element by the induction hypothesis. ■

Lemma (`exists_least_not_mem`). If $a \subseteq \omega$ is not all of ω (equivalently, some $k \notin a$), there exists $k \notin a$ minimal with that property (every $i < k$ satisfies $i \in a$).

Proof. Dual to `exists_least_mem`, using strong induction on a witness $n \notin a$. ■

Theorem (`ofNat.combinatory`). Every numeral $\mathrm{ofNat}\, n$ is combinatory.

Proof. By induction: 0 is the zero combinator; successors apply the successor combinator. ■

Theorem (`ofNat.zero_nonempty`). The numeral 0 is nonempty.

Proof. $0 \in \{0\}$. ■

Theorem (`ofNat.zero_not_pos`). No positive numeral belongs to 0.

Proof. Membership in $\{0\}$ forces equality with 0. ■

Theorem (`topElem.ne_ofNat_zero`). $\top \neq 0$.

Proof. Otherwise $1 \in 0$. ■

`Language.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

inductive Comb where
  | var : ℕ → Comb
  | zero | suc | pred | cond | K | S
  | app : Comb → Comb → Comb

inductive Term where
  | var : ℕ → Term
  | zero : Term
  | succ : Term → Term
  | pred : Term → Term
  | cond : Term → Term → Term → Term
  | app : Term → Term → Term
  | lam : ℕ → Term → Term

def ofNat (n : ℕ) : Pomega := {n}

@[simp] theorem mem_ofNat {n k : ℕ} : k ∈ ofNat n ↔ k = n := by
  simp [ofNat, Set.mem_singleton_iff]

def predSet (x : Pomega) : Pomega :=
  {k | k + 1 ∈ x}

def succSet (x : Pomega) : Pomega :=
  {k | ∃ n ∈ x, k = n + 1}

theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e.pow2 0

def envSet (? : ℕ → Pomega) (i : ℕ) (v : Pomega) : ℕ → Pomega :=
```

```
  Function.update ? i v
namespace envSet
theorem comm {? : ℕ → Pomega} {i j : ℕ} (hij : i = j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
  by_cases hj : k = j
  · subst hj
    simp [Function.update, hij.symm]
  · by_cases hi : k = i
    · subst hi
      simp [Function.update, hij]
    · simp [Function.update, hi, hj]
end envSet

def Kcomb : Pomega := graph (fun x => graph (fun _ => x))

def zeroC : Pomega := ofNat 0

def predC : Pomega := graph predSet

def succC : Pomega := graph succSet

def rename (i j : ℕ) : Term → Term
  | .var k => if k = i then .var j else .var k
  | .zero => .zero
  | .succ t => .succ (rename i j t)
  | .pred t => .pred (rename i j t)
  | .cond z x y => .cond (rename i j z) (rename i j x) (rename i j y)
  | .app u x => .app (rename i j u) (rename i j x)
  | .lam k body => if k = i then .lam k body else .lam k (rename i j body)

theorem triangle_eq_zero {w : ℕ} (h : triangle w = 0) : w = 0 := by
  match w with
  | 0 => rfl
  | n + 1 =>
    have h1 : 1 ≤ n + 1 := Nat.succ_le_succ (Nat.zero_le _)
    have h2 : 2 ≤ (n + 1) * (n + 2) :=
      calc 2 ≤ 1 * 2 := by decide
           _ ≤ (n + 1) * (n + 2) := Nat.mul_le_mul h1 (Nat.succ_le_succ h1)
    have : 0 < (n + 1) * (n + 2) / 2 := Nat.div_pos h2 (by decide)
    exact (Nat.ne_of_gt this h).elim

theorem pair_zero_zero : pair 0 0 = 0 := by simp [pair]

theorem pair_one_zero : pair 1 0 = 1 := by simp [pair]

theorem pair_eq_right {n m : ℕ} (h : pair n m = m) : n = 0 ∧ m = 0 := by
  have : triangle (n + m) = 0 := by
    have h' : triangle (n + m) + m = m := by simpa [pair_eq_triangle] using h
    omega
  have : n + m = 0 := triangle_eq_zero this
  omega

theorem eq_2_5 : succSet (ofNat 6 ∪ ofNat 10) = ofNat 7 ∪ ofNat 11 := by
  ext k
```

```
  constructor
  · intro ⟨n, hn, hk⟩
    rcases hn with h6 | h10
    · simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    · simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  · intro hk
    rcases hk with h7 | h11
    · simp [ofNat] at h7; exact ⟨6, Or.inl (by simp [ofNat]), by simp [h7]⟩
    · simp [ofNat] at h11; exact ⟨10, Or.inr (by simp [ofNat]), by simp [h11]⟩

theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

lemma exists_least_mem {a : Pomega} (hex : ∃ k, k ∈ a) :
    ∃ k, k ∈ a ∧ ∀ i < k, i ∈ a := by
  have : ∀ n, n ∈ a → ∃ k ≤ n, k ∈ a ∧ ∀ i < k, i ∈ a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (∀ i < n, i ∈ a) with hmin | hmin
      · exact ⟨n, le_rfl, hn, hmin⟩
      · obtain ⟨i, hi⟩ := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i ∈ a := not_not.mp (Classical.not_imp.mp hi).2
        obtain ⟨k, hki, hk, hall⟩ := ih i hi' hia
        exact ⟨k, hki.trans hi'.le, hk, hall⟩
  obtain ⟨n, hn⟩ := hex
  obtain ⟨k, _, hk, hall⟩ := this n hn
  exact ⟨k, hk, hall⟩

lemma exists_least_not_mem {a : Pomega} (hex : ∃ k, k ∈ a) :
    ∃ k, k ∈ a ∧ ∀ i < k, i ∈ a := by
  have : ∀ n, n ∈ a → ∃ k ≤ n, k ∈ a ∧ ∀ i < k, i ∈ a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (∀ i < n, i ∈ a) with hmin | hmin
      · exact ⟨n, le_rfl, hn, hmin⟩
      · obtain ⟨i, hi⟩ := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i ∈ a := (Classical.not_imp.mp hi).2
        obtain ⟨k, hki, hk, hall⟩ := ih i hi' hia
        exact ⟨k, hki.trans hi'.le, hk, hall⟩
  obtain ⟨n, hn⟩ := hex
  obtain ⟨k, _, hk, hall⟩ := this n hn
  exact ⟨k, hk, hall⟩

end DomainTheory.DataTypesAsLattices
```

ofNat_more.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
public import DomainTheory.DataTypesAsLattices.Lambda.Language
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.sucC_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace ofNat

theorem combinatory : ∀ n, IsCombinatory (ofNat n)
  | 0 => by
    simpa [zeroC] using IsCombinatory.zero
  | n + 1 => by
    have h := IsCombinatory.app IsCombinatory.suc (combinatory n)
    simpa [sucC.app, succSet.ofNat] using h

theorem zero_nonempty : (ofNat 0).Nonempty :=
  ⟨0, rfl⟩

theorem zero_not_pos : ¬ ∃ n, n + 1 ∈ ofNat 0 := by
  intro ⟨n, hn⟩
  simp [ofNat] at hn

end ofNat

namespace topElem

theorem ne_ofNat_zero : topElem = ofNat 0 := by
  intro h
  have : (1 : ℕ) ∈ ofNat 0 := by rw [← h]; exact Set.mem_univ 1
  simp [ofNat] at this

end topElem

end DomainTheory.DataTypesAsLattices
```

3.3.4 Term avoidance (α) ✓

Table 1 records the basic laws of λ-conversion verified in the $\mathbf{P}\omega$ model. The first is renaming of bound variables:

$$(\alpha) \quad \lambda x.\, \tau = \lambda y.\, \tau[y/x]$$

(in the usual proviso that y is not free in τ).

Proof of Theorem 2.2 (fragment). The verification of (α) is obvious as the 'x' in '$\lambda x.\, \tau$' is a bound variable. (Care should be taken in making the proviso that 'y' is not otherwise free in τ.) The same would of course hold for any other pair of variables. We do not bother very much about alphabetic questions.

In LAMBDA (Table 2), variables are treated informally and substitution $\tau[e_n/x]$ is used in the semantics of $\lambda x.\, \tau$. In the Lean formalization, terms use de Bruijn-style indices x_i (`Term.var i`), and many

closed combinator terms in §3 are built by recursion on `Term`. To match Scott's proviso mechanically—no accidental capture of a reserved index—we use a small library predicate **`Term.Avoids`**.

Formalization (`Avoids.lean`). Each Lean name is the library entry.

Definition (`Term.Avoids`). For a variable index $i \in \omega$ and a term τ, $\tau.\texttt{Avoids}\, i$ means i does not occur free in τ: - on x_j, require $j \neq i$; - on 0, successor, predecessor, conditional, and application, require the property in each subterm; - on $\lambda x_j.\, \sigma$, require $j = i$ (then i is bound and not free) or $\sigma.\texttt{Avoids}\, i$.

This is the inductive "term avoids index i" check used throughout Enumeration and Computability when proving that a synthesized closed term leaves slots such as x_7 or x_8 free for later composition (Gödel codes, combinator algebra, and similar).

`Avoids.lean`

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Language

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def Term.Avoids (i : ℕ) : Term → Prop
  | .var j => j = i
  | .zero => True
  | .succ t => t.Avoids i
  | .pred t => t.Avoids i
  | .cond z x y => z.Avoids i ∧ x.Avoids i ∧ y.Avoids i
  | .app u x => u.Avoids i ∧ x.Avoids i
  | .lam j body => j = i ∨ body.Avoids i

end DomainTheory.DataTypesAsLattices
```

3.3.5 Theorem 2.1 — the continuity theorem ✓

THEOREM 2.1 (The continuity theorem). *All LAMBDA definable functions are continuous.*

Proof of Theorem 2.1. If we did not use the λ-notation, then all LAMBDA-definable functions would be obtained by substitution from the first five (cf. Table 2). Since they are all seen to be continuous, the result would then follow by the substitution theorem (Theorem 1.3). Bringing in λ-abstraction means that we have to combine Theorem 1.3 with this fact:

LEMMA. *If $f(x, y, z, \cdots)$ is a continuous function of all its variables, then $\lambda x.\, f(x, y, z, \cdots)$ is a continuous function of the remaining variables.*

Proof. It is enough to consider one extra variable. We compute from the definition of λ in Table 2 as follows:

$$\lambda x.\, f(x,y) = \{(n,m) \mid m \in f(e_n, y)\}$$

$$= \{(n,m) \mid \exists e_k \subseteq y.\, m \in f(e_n, e_k)\}$$

$$= \bigcup \{\{(n,m) \mid m \in f(e_n, e_k)\} \mid e_k \subseteq y\}$$

$$= \bigcup \{\lambda x.\, f(x, e_k) \mid e_k \subseteq y\}.$$

Thus $\lambda x.\, f(x,y)$ is continuous in y.

`theorem_2_1.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.interp
import Mathlib.Tactic.Bound.Init
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_2_1 (t : Term) (? : ℕ → Pomega) (i : ℕ) :
    IsScottContinuous (fun v => interp t (envSet ? i v)) :=
  interp.var_isScottContinuous t ? i

end DomainTheory.DataTypesAsLattices
```

3.3.6 Theorem 2.2 — the conversion theorem ✓

Once that is proved, we can use Theorem 1.2 to establish:

THEOREM 2.2 (The conversion theorem). *The three basic principles* (α), (β), (ξ) *of* λ*-conversion are all valid in the model.*

Proof of Theorem 2.2. The reason behind this result is the restriction to continuous functions. Theorem 2.1 shows that we cannot violate the restriction by giving definitions in LAMBDA, and the graph theorem (Theorem 1.2) shows that continuous functions correspond perfectly with their graphs.

The verification of (α) of Table 1 is obvious as the 'x' in '$\lambda x.\, \tau$' is a bound variable. (Care should be taken in making the proviso that 'y' is not otherwise free in τ.) The same would of course hold for any other pair of variables. We do not bother very much about alphabetic questions.

The verification of (β) is just a restatement of Theorem 1.2(i). Let τ define a function f (of x). Then by definition $f(x) = \tau$ and $\lambda x.\, \tau = \mathbf{graph}(f)$. Also $\mathbf{fun}(u)(x)$ in the notation of Theorem 1.2 is the same as the binary operation $u(x)$ in the notation of LAMBDA. Thus in Theorem 1.2(i) if we apply both sides to y we get nothing else than (β).

Half of property (ξ) is already implied by (β): the implication from left to right. (Just apply both sides to x.) In the other direction, $\forall x.\, \tau = \sigma$ means that τ and σ define the *same* function of x; thus, the two graphs must be equal.

Remarks on other laws. The failure of (η) simply means that not every set in $\mathbf{P}\omega$ is the graph of a function. Condition (iii) of Theorem 1.2 is equivalent to saying that $u = \lambda x.\, u(x)$, in other words, u is the graph of some function if and only if it is the graph of the function determined by u.

Law (μ) is the monotone property of application (in both variables); therefore, (μ) and (ξ) together imply (ξ^*) from left to right. Suppose that $\forall x.\, \tau \subseteq \sigma$; then clearly:

$$\{(n, m) \mid m \in \tau[e_n/x]\} \subseteq \{(n, m) \mid m \in \sigma[e_n/x]\},$$

which gives (ξ^*) from right to left.

There are, by the way, other laws valid in the model, as explained in the later results.

theorem_2_2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.beta

public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_1

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace theorem_2_2

theorem alpha (i j : ℕ) (body : Term) (? : ℕ → Pomega)
    (h : ∀ v, interp body (envSet ? i v) =
            interp (rename i j body) (envSet ? j v)) :
    interp (.lam i body) ? = interp (.lam j (rename i j body)) ? := by
  unfold interp
  exact congrArg graph (funext h)

theorem xi {i : ℕ} {τ σ : Term} {? : ℕ → Pomega}
    (h : ∀ v, interp τ (envSet ? i v) = interp σ (envSet ? i v)) :
    interp (.lam i τ) ? = interp (.lam i σ) ? := by
  simp [interp, h]

theorem beta (i : ℕ) (body : Term) (? : ℕ → Pomega) (y : Pomega) :
    funOf (interp (.lam i body) ?) y = interp body (envSet ? i y) :=
  DomainTheory.DataTypesAsLattices.beta (theorem_2_1 body ? i) y

end theorem_2_2

theorem theorem_2_2 (i : ℕ) (body : Term) (? : ℕ → Pomega) (y : Pomega) :
    (∀ j, (∀ v, interp body (envSet ? i v) =
            interp (rename i j body) (envSet ? j v)) →
        interp (.lam i body) ? = interp (.lam j (rename i j body)) ?) ∧
      funOf (interp (.lam i body) ?) y = interp body (envSet ? i y) ∧
        ∀ σ, (∀ v, interp body (envSet ? i v) = interp σ (envSet ? i v)) →
          interp (.lam i body) ? = interp (.lam i σ) ? :=
  ⟨fun j h => theorem_2_2.alpha i j body ? h,
    theorem_2_2.beta i body ? y,
    fun _σ h => theorem_2_2.xi h⟩

end DomainTheory.DataTypesAsLattices
```

3.3.7 Theorem 2.3 — the reduction theorem ✓

By "model" here we of course understand the interpretation of the language where the semantics gives terms denotations in $\mathbf{P}\omega$ according to the stated definition. Through this interpretation, more properly speaking, $\mathbf{P}\omega$ becomes a model for the axioms (α), (β), (ξ). Two well-known results of the calculus of λ-conversion allow the reduction of functions of several variables to those of *one*, and the reduction of all the primitives to *combinators* (constants)—all this with the aid of the binary operation of application.

THEOREM 2.3 (The reduction theorem). *Any continuous function of k-variables can be written as*

$$f(x_0, x_1, \cdots, x_{k-1}) = u(x_0)(x_1)\cdots(x_{k-1}),$$

where u is a suitably chosen element of $\mathbf{P}\omega$.

Proof of Theorem 2.3. This is a standard result in combinatory logic. We have only to put:

$$u = \lambda x_0 \lambda x_1 \cdots \lambda x_{n-1}.\, f(x_0, x_1, \cdots, x_{n-1}).$$

That is, we use the iteration of the process of forming the graph of a continuous function. As each step (from the inside out) keeps everything continuous, we are sure that the equation of Theorem 2.3 will hold for iterated application.

theorem_2_3.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.curryFin
public import DomainTheory.DataTypesAsLattices.Lambda.curry2
public import DomainTheory.DataTypesAsLattices.Lambda.curry3
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace theorem_2_3

theorem unary {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    ∃ u, ∀ x, funOf u x = f x :=
  ⟨graph f, fun x => by rw [theorem_1_2.i hf]⟩

theorem binary {f : Pomega → Pomega → Pomega}
    (hfx : ∀ y, IsScottContinuous (fun x => f x y))
    (hfy : ∀ x, IsScottContinuous (fun y => f x y)) :
    ∃ u, ∀ x y, funOf (funOf u x) y = f x y :=
  ⟨curry2 f hfx hfy, curry2.app f hfx hfy⟩

theorem ternary {f : Pomega → Pomega → Pomega → Pomega}
    (hf0 : ∀ y z, IsScottContinuous (fun x => f x y z))
    (hf1 : ∀ x z, IsScottContinuous (fun y => f x y z))
    (hf2 : ∀ x y, IsScottContinuous (fun z => f x y z)) :
    ∃ u, ∀ x y z, funOf (funOf (funOf u x) y) z = f x y z :=
  ⟨curry3 f hf0 hf1 hf2, curry3.app f hf0 hf1 hf2⟩

end theorem_2_3

theorem theorem_2_3 {n : ℕ} {f : (Fin n → Pomega) → Pomega}
    (hf : ScottContinuousFin f) :
```

```
    ∃ u, ∀ xs, nestApplyFin u xs = f xs :=
  ⟨curryFin f, curryFin.app hf⟩

end DomainTheory.DataTypesAsLattices
```

3.3.8 Theorem 2.4 — the combinator theorem ✓

THEOREM 2.4 (The combinator theorem). *The LAMBDA-definable func-*
...tions can be generated (from variables) by iterated application with the aid of these six constants:

$$\mathbf{0} = 0$$

$$\mathbf{suc} = \lambda x.\, x + 1$$

$$\mathbf{pred} = \lambda x.\, x - 1$$

$$\mathbf{cond} = \lambda x \lambda y \lambda z.\, z \supset x, y$$

$$\mathbf{K} = \lambda x \lambda y.\, x$$

$$\mathbf{S} = \lambda u \lambda v \lambda x.\, u(x)(v(x))$$

Proof of Theorem 2.4. This can be found in almost any reference on combinatory logic or λ-conversion. The main idea is to eliminate the λ in favor of the combinators. The fact that we have a few other kinds of terms causes no problem if we introduce the corresponding combinators. The method of proof is to show, for any LAMBDA-term τ with free variables among $x_0, x_1, \ldots, x_{n-1}$, that there is a combination γ of combinators such that:

$$\tau = \gamma(x_0)(x_1)\cdots(x_{k-1}).$$

This can be done by induction on the complexity of τ.

theorem_2_4.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.erase

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace theorem_2_4

theorem complete (t : Term) (? : ℕ → Pomega) :
    interp t ? = ofComb (erase t) ? :=
  (erase.correct t ?).symm
```

```
theorem closed (t : Term) :
    IsCombinatory (interp t (fun _ => botElem)) := by
  rw [theorem_2_4.complete]
  refine ofComb.combinatory _ _ fun _ => ?_
  have hbot : funOf predC zeroC = botElem := by
    rw [predC.app]; exact eq_2_6
  exact hbot ▶ IsCombinatory.app IsCombinatory.pred IsCombinatory.zero

end theorem_2_4

theorem theorem_2_4 : IsCombinatory zeroC ∧ IsCombinatory sucC ∧ IsCombinatory predC ∧
    IsCombinatory condC ∧ IsCombinatory Kcomb ∧ IsCombinatory Scomb :=
  ⟨.zero, .suc, .pred, .cond, .K, .S⟩

end DomainTheory.DataTypesAsLattices
```

3.3.9 The combinator Y ✓

But the result that makes all this model building and combinatory logic mathematically interesting concerns the so-called *paradoxical combinator* defined by the equation:

$$(2.8) \quad \mathbf{Y} = \lambda u.\,(\lambda x.\,u(x(x)))(\lambda x.\,u(x(x))).$$

Language, model, and bridge. Equation (2.8) is stated in **LAMBDA** (syntax: `Term`, interpretation `interp`). The **model** is $\mathbf{P}\omega$ with application `funOf` and graphs `graph`. We separate three roles in `Ycomb.lean`:

- **Language:** `Yterm` is the closed LAMBDA term for (2.8), using variables x_0, x_1 for the binders in Scott's λu and λx.
- **Model:** `Ycomb` is the element of $\mathbf{P}\omega$ obtained by turning (2.8) into a graph: for each u, apply the ω-subcombinator $\omega(u) = \lambda x.\,u(x(x))$ to itself (formalized as `omegaComb`).
- **Bridge:** `Yterm.interp` shows that under the constant environment $\rho \equiv \lambda i.\,\bot$, we have $\mathrm{interp}(\texttt{Yterm})\,\rho = \texttt{Ycomb}$—the syntax of (2.8) denotes the model element.

The lemmas in `namespace Ycomb` are **model** facts about `funOf Ycomb`; `Ycomb.combinatory` links back to **language** via Theorem 2.4 and the bridge.

Formalization (`Ycomb.lean`). Each Lean name is the library entry; proofs in the file are short unless noted.

Definition (`omegaComb`, in `omegaComb.lean`). For $u \in \mathbf{P}\omega$, $\omega(u) = \mathbf{graph}(\lambda x.\,u(x(x))$, so $\mathrm{funOf}(\omega(u))\,x = \mathrm{funOf}\,u\,(\mathrm{funOf}\,x\,x)$ (`omegaComb.app`).

Definition (`Ycomb`, model). $\texttt{Ycomb} = \mathbf{graph}(\lambda u.\,\mathrm{funOf}(\omega(u))(\omega(u)))$—the graph of $u \mapsto (\lambda x.\,u(x(x)))(\lambda x.\,u(x(x)))$ in the model.

Definition (`Yterm`, language). `Yterm` is the `Term` for (2.8): $\lambda x_0.\,(\lambda x_1.\,x_0(x_1(x_1)))(\lambda x_1.\,x_0(x_1(x_1)))$.

Theorem (`Yterm.interp`, bridge). For $\rho(i) = \bot$ for all i,

$$\mathrm{interp}(\texttt{Yterm})\,\rho = \texttt{Ycomb}.$$

Proof. Unfold `interp`, `Yterm`, `Ycomb`, and `omegaComb`; simplify updates in `envSet`. ■

Theorem (`Ycomb.app`, model). For all $u \in \mathbf{P}\omega$,

$$\mathrm{funOf}(\texttt{Ycomb})\, u = \mathrm{funOf}(\omega(u))(\omega(u)).$$

Proof. Unfold `Ycomb` as a graph and apply (β) (`beta`), using Theorem 1.3 to justify continuity of the underlying map (`theorem_1_3`, `diagApp_isScottContinuous`, `omegaComb.isScottContinuous`). ■

Theorem (`Ycomb.combinatory`, model / language). `IsCombinatory Ycomb`.

Proof. By Theorem 2.4, every closed term's interpretation is combinatory (`theorem_2_4.closed` applied to `Yterm`). Rewrite with `Yterm.interp`. ■

Theorem (`Ycomb.unfold`, model). For all $u \in \mathbf{P}\omega$,

$$\mathrm{funOf}(\texttt{Ycomb})\, u = \mathrm{funOf}\, u\, (\mathrm{funOf}(\texttt{Ycomb})\, u).$$

Proof. Chain `Ycomb.app`, the defining equation for $\omega(u)$ at $x = \omega(u)$ (`omegaComb.app`), and `Ycomb.app` again. This is the graph-model form of the usual Y fixed-point law; Theorem 2.5 treats the case when u is itself a graph. ■

`Ycomb.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.omegaComb
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Ycomb : Pomega :=
  graph (fun u => funOf (omegaComb u) (omegaComb u))

def Yterm : Term :=
  .lam 0 (.app
    (.lam 1 (.app (.var 0) (.app (.var 1) (.var 1))))
    (.lam 1 (.app (.var 0) (.app (.var 1) (.var 1)))))

namespace Yterm

theorem interp :
    _root_.DomainTheory.DataTypesAsLattices.interp Yterm (fun _ => botElem) =
      Ycomb := by
  simp [Yterm, _root_.DomainTheory.DataTypesAsLattices.interp, Ycomb, omegaComb,
    envSet, Function.update]

end Yterm

namespace Ycomb

theorem app (u : Pomega) :
    funOf Ycomb u = funOf (omegaComb u) (omegaComb u) :=
  beta (theorem_1_3 diagApp_isScottContinuous omegaComb.isScottContinuous) u

theorem combinatory : IsCombinatory Ycomb := by
```

```
  simpa [Yterm.interp] using theorem_2_4.closed Yterm

theorem unfold (u : Pomega) :
    funOf Ycomb u = funOf u (funOf Ycomb u) := by
  calc
    funOf Ycomb u = funOf (omegaComb u) (omegaComb u) := Ycomb.app u
    _ = funOf u (funOf (omegaComb u) (omegaComb u)) :=
      omegaComb.app u (omegaComb u)
    _ = funOf u (funOf Ycomb u) := by rw [Ycomb.app]

end Ycomb

end DomainTheory.DataTypesAsLattices
```

3.3.10 Theorem 2.5 — the first recursion theorem ✓

THEOREM 2.5 (The first recursion theorem). *If u is the graph of a continuous function f, then $\mathbf{Y}(u) = \mathbf{fix}(f)$, the least fixed point of f.*

There are two points to note here: the fixed point is LAMBDA-definable if f is; and $\mathbf{Y}$ defines a continuous operator. The word "recursion" is attached to the theorem because fixed points are employed to solve recursion equations. It would not be correct to call the fixed-point theorem (Theorem 1.4) the recursion theorem since it only shows that fixed points exist and not how they are definable in a language. The *second* recursion theorem (in Kleene's terminology) is related, but it involves Gödel numbers as introduced in § 3.

Proof of Theorem 2.5. The well-known calculation shows that we have from (2.8):

$$\mathbf{Y}(u) = (\lambda x.\, u(x(x)))(\lambda x.\, u(x(x))) = u(\mathbf{Y}(u)).$$

Thus $\mathbf{Y}(u)$ is a fixed point of the function $u(x)$. What is needed is the proof to show that it is the least one.

Let $d = \lambda x.\, u(x(x))$ and let a be any other fixed point of $u(x)$. To show, as we must, that $d(d) \subseteq a$, it is enough to show that $e_l \subseteq d$ always implies $e_l(e_l) \subseteq a$; because by continuity we have:

$$d(d) = \bigcup\{e_l(e_l) \mid e_l \subseteq d\}.$$

By way of induction, suppose that this implication holds for all $n < l$. Assume that $e_l \subseteq d$ and that $m \in e_l(e_l)$. We will want to use the induction hypothesis to show that $m \in a$. By the definition of application, there exists an integer n such that $(n, m) \in e_l$ and $e_n \subseteq e_l$. But $n \leq (n, m) < l$, and $e_n \subseteq d$. By the hypothesis, we have $e_n(e_n) \subseteq a$. Note that $(n, m) \in d$ also, and that d is defined by λ-abstraction; thus,

$m \in d(e_n)$ by definition. By monotonicity $u(e_n(e_n)) \subseteq u(a) = a$; therefore $m \in a$. This shows that $e_l(e_l) \subseteq a$, and the inductive proof is complete.

Remark. Note that we did not actually use the fixed-point theorem in the proof, but we did use rather special properties of the pairing (n, m) and the finite sets e_l.

`theorem_2_5.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.omega_first_recursion
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_2_5 {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    funOf Ycomb (graph f) = fix f := by
  rw [Ycomb.app]
  exact omega_first_recursion hf

end DomainTheory.DataTypesAsLattices
```

3.3.11 The graph model ✓

From this point on we see no need to distinguish continuous functions from elements of $\mathbf{P}\omega$; a continuous function will be *identified* with its graph. Note that u is a graph iff $u = \lambda x.\, u(x)$, which is equivalent to Theorem 1.2 (iii). For this reason (functions are graphs) we propose the name *Graph Model* for this model of the λ-calculus. (There is more to LAMBDA than just λ, however.)

The identification of functions with graphs entails that the function space of all continuous functions from $\mathbf{P}\omega$ into $\mathbf{P}\omega$ is to be identified (one-to-one) with the subspace

$$\mathbf{FUN} = \{u \mid u = \lambda x.\, u(x)\} \subseteq \mathbf{P}\omega.$$

The identification is *topological* in that the subspace topology agree with the product topology on the function space. This is the topology of pointwise convergence and is closely connected with the lattice structure on the function space which is also defined pointwise (that is, argumentwise). In the notation of λ-abstraction we can express this as the extension of the axiom of extensionality called (ξ^*) in Table 1 of the Introduction. The laws in Table 1 are not the only ones valid in the model, however. We may also note such argumentwise distribution

laws as:

$$(2.9) \quad (f \cup g)(x) = f(x) \cup g(x);$$

$$(2.10) \quad (\lambda x.\, \tau) \cup (\lambda x.\, \sigma) = \lambda x.\, (\tau \cup \sigma);$$

$$(2.11) \quad (f \cap g)(x) = f(x) \cap g(x);$$

$$(2.12) \quad (\lambda x.\, \tau) \cap (\lambda x.\, \sigma) = \lambda x.\, (\tau \cap \sigma).$$

In the above f and g must be graphs. It is also true that if $\mathcal{F} \subseteq \mathbf{P}\omega$, then

$$(2.13) \quad \bigcup\{f \mid f \in \mathcal{F}\}(x) = \bigcup\{f(x) \mid f \in \mathcal{F}\},$$

but the same does not hold for $\cap$.

Formalization (`graph_model.lean`). Scott's (2.9)–(2.11) are stated for graphs f, g and application; in Lean we use `graph` for functional abstraction and prove the union/intersection laws (2.10) and (2.12) in

that form, together with (ξ^*) as `xi_star`. Each name below is the library entry; proofs in the file are short unless noted.

Definition (`graph`). For $F : \mathbf{P}\omega \to \mathbf{P}\omega$, $\mathbf{graph}(F) = \{(n, m) \mid m \in F(e_n)\}$ (pairs coded as `pair n m`).

Theorem (`xi_star`, Table 1 (ξ^*)). If $\tau, \sigma : \mathbf{P}\omega \to \mathbf{P}\omega$ and $\forall x.\, \tau(x) \subseteq \sigma(x)$, then $\mathbf{graph}(\tau) \subseteq \mathbf{graph}(\sigma)$.

Proof. If $(n, m) \in \mathbf{graph}(\tau)$, then $m \in \tau(e_n)$; by hypothesis $m \in \sigma(e_n)$, so $(n, m) \in \mathbf{graph}(\sigma)$. ■

Theorem (`eq_2_10`, equation (2.10)). For all $\tau, \sigma : \mathbf{P}\omega \to \mathbf{P}\omega$,

$$\mathbf{graph}(\tau) \cup \mathbf{graph}(\sigma) = \mathbf{graph}(\lambda x.\, \tau(x) \cup \sigma(x)).$$

Proof. Extensionality on pairs: membership in the left union splits according to membership in $\mathbf{graph}(\tau)$ or $\mathbf{graph}(\sigma)$; on the right, $m \in \tau(x) \cup \sigma(x)$ iff $m \in \tau(x)$ or $m \in \sigma(x)$. ■

Theorem (`eq_2_12`, equation (2.12)). For all $\tau, \sigma : \mathbf{P}\omega \to \mathbf{P}\omega$,

$$\mathbf{graph}(\tau) \cap \mathbf{graph}(\sigma) = \mathbf{graph}(\lambda x.\, \tau(x) \cap \sigma(x)).$$

Proof. If (n, m) lies in both graphs, the argument codes n, m must agree (`pair_inj`); then $m \in \tau(e_n) \cap \sigma(e_n)$. Conversely, if m lies in both value sets at the same e_n, the pair belongs to both graphs. ■

(Equations (2.9) and (2.11) are the same distribution for application and values; (2.13) is not in this file.)

`graph_model.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.eDense

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem xi_star {τ σ : Pomega → Pomega} (h : ∀ x, τ x ⊆ σ x) :
    graph τ ⊆ graph σ := by
  intro p hp
  rcases hp with ⟨n, m, rfl, hm⟩
  exact ⟨n, m, rfl, h _ hm⟩

theorem eq_2_10 (τ σ : Pomega → Pomega) :
    graph τ ∪ graph σ = graph (fun x => τ x ∪ σ x) := by
  ext p
  constructor
  · intro hp
    rcases hp with h | h
    · rcases h with ⟨n, m, hp, hm⟩
      exact ⟨n, m, hp, Or.inl hm⟩
    · rcases h with ⟨n, m, hp, hm⟩
      exact ⟨n, m, hp, Or.inr hm⟩
  · intro h
    rcases h with ⟨n, m, hp, hm⟩
    rcases hm with hm | hm
```

```
    · exact Or.inl ⟨n, m, hp, hm⟩
    · exact Or.inr ⟨n, m, hp, hm⟩

theorem eq_2_12 (τ σ : Pomega → Pomega) :
    graph τ ∩ graph σ = graph (fun x => τ x ∩ σ x) := by
  ext p
  constructor
  · intro ⟨hτ, hσ⟩
    rcases hτ with ⟨n, m, hpτ, hmτ⟩
    rcases hσ with ⟨n', m', hpσ, hmσ⟩
    have hp' : pair n m = pair n' m' := hpτ.symm.trans hpσ
    obtain ⟨rfl, rfl⟩ := pair_inj hp'
    exact ⟨n, m, hpτ, hmτ, hmσ⟩
  · intro h
    rcases h with ⟨n, m, hp, hmτ, hmσ⟩
    exact ⟨⟨n, m, hp, hmτ⟩, ⟨n, m, hp, hmσ⟩⟩

end DomainTheory.DataTypesAsLattices
```

3.3.12 Sequences ✓

Next we use the notation $\langle x_0, x_1, \dots, x_{n-1}\rangle$ for the function $\hat{p}$ where $p : \omega \to \mathbf{P}\omega$ is defined by:

$$p(i) = \begin{cases} x_i & \text{if } i < n; \\ \bot & \text{if } i \geq n. \end{cases}$$

$$\langle\rangle = \bot \tag{2.19}$$

$$\langle x\rangle = \lambda z.\, z \supset x, \bot \tag{2.20}$$

$$\langle x, y\rangle = \lambda z.\, z \supset x, (z - 1 \supset y, \bot) \tag{2.21}$$

$$\langle x_0, x_1, \dots, x_n\rangle = \lambda z.\, z \supset x_0, \langle x_1, \dots, x_n\rangle(z - 1). \tag{2.22}$$

Obviously we should formalize the subscript notation so that $u_x = u(x)$; then we find:

$$\langle x_0, x_1, \dots, x_n\rangle_i = \begin{cases} x_i & \text{if } i < n; \\ \bot & \text{if } i \geq n. \end{cases} \tag{2.23}$$

Formalization (`sequences.lean`). *Finite* tuples $\langle x_0, \dots, x_{n-1}\rangle$ are graphs built from McCarthy conditional (`condSet`) and predecessor on arguments; *infinite* sequences are the spread $\lambda x.\ \bigcup\{u(i) \mid i \in x\}$, approximated by bounded unions `seqFunLt` and defined recursively via `seqShift`. Lean names below are the library entries.

Definition (`seqCons`). `seqCons`$(x, xs) = \mathbf{graph}(\lambda z.\, z \supset x,\ xs(\mathit{pred}\, z))$ — the graph of (2.22): head x, tail read from xs off $z - 1$.

Theorem (`seqCons.body`). The body $z \mapsto \mathit{condSet}(z, x, xs(\mathit{pred}\, z))$ is Scott-continuous.

Proof. `theorem_1_3.tuple` from continuity of conditional, identity, `funOf xs`, and `predSet`. ■

Definition (seq2). $\texttt{seq2}(x, y) = \mathbf{graph}(\lambda z.\, z \supset x,\ (\mathit{pred}\, z \supset y,\ \bot))$ — model form of (2.21) for a pair.

Theorem (seq2.app). $\mathrm{funOf}(\texttt{seq2}\, x\, y)\, z = z \supset x,\ (\mathit{pred}\, z \supset y,\ \bot)$.

Proof. (β) via `seq2.body_isScottContinuous`. ■

Theorem (eq_2_22). Same as `seq2.app` for general `seqCons`: $\mathrm{funOf}(\texttt{seqCons}\, x\, xs)\, z = z \supset x,\ \mathrm{funOf}\, xs\, (\mathit{predSet}\, z)$.

Proof. `seqCons.body` and (β). ■

Theorem (eq_2_23_zero, seq2.zero). $\mathrm{funOf}(\texttt{seq2}\, x\, y)\, 0 = x$ (argument $0 = \mathrm{ofNat}\, 0$).

Proof. Unfold `seq2.app`; case split on conditional and predecessor. ■

Theorem (eq_2_23_one, seq2.one). $\mathrm{funOf}(\texttt{seq2}\, x\, y)\, 1 = y$.

Proof. Same. ■

Theorem (eq_2_23_ge). For all n, $\mathrm{funOf}(\texttt{seq2}\, x\, y)\, (n+2) = \bot$ (empty beyond the pair).

Proof. `seq2.app` and arithmetic on `predSet` / `ofNat`. ■

Theorem (seq2.tag_ne). $\texttt{seq2}(\mathrm{ofNat}\, 0, x) \neq \texttt{seq2}(\mathrm{ofNat}\, 1, y)$ (tags 0 and 1 are distinguishable).

Proof. Apply both sides to argument 0; membership in 0 vs 1 contradicts. ■

Definition (seqFun). $\texttt{seqFun}(u, x) = \bigcup\{\mathrm{funOf}\, u\, (\mathrm{ofNat}\, i) \mid i \in x\}$ — the spread in (2.23) without λ-notation.

Definition (seqFunLt). $\texttt{seqFunLt}(u, x, n) = \bigcup\{\mathrm{funOf}\, u\, (\mathrm{ofNat}\, i) \mid i \in x,\ i < n\}$ — the same union restricted to indices $< n$.

Theorem (mem_seqFunLt). $k \in \texttt{seqFunLt}\, u\, x\, n \leftrightarrow \exists i \in x,\ i < n \wedge k \in \mathrm{funOf}\, u\, (\mathrm{ofNat}\, i)$.

Proof. Unfold `seqFunLt` as a double indexed union. ■

Theorem (seqFun.eq_iUnion_lt). $\texttt{seqFun}(u, z) = \bigcup_n \texttt{seqFunLt}(u, z, n)$.

Proof. Extensionality; each index i contributes at stage $i + 1$. ■

Theorem (seqFun.left_isScottContinuous, seqFun.right_isScottContinuous). $u \mapsto \texttt{seqFun}(u, x)$ and $x \mapsto \texttt{seqFun}(u, x)$ are Scott-continuous.

Proof. Directed unions of Scott-continuous applications (`scottUnion`, `mem_scottUnion`, `funOf.isScottContinuous`). ■

Definition (seqShift). $\texttt{seqShift}(u) = \mathbf{graph}(\lambda z.\, \texttt{seqFun}(u, \mathit{succSet}\, z))$ — reindex tail by successor on arguments (shift u_i to u_{i+1}).

Theorem (seqShift.app, seqShift.ofNat). $\mathrm{funOf}(\texttt{seqShift}\, u)\, z = \texttt{seqFun}(u, \mathit{succSet}\, z)$; at numeric i, $\mathrm{funOf}(\texttt{seqShift}\, u)\, (\mathrm{ofNat}\, i) = \mathrm{funOf}\, u\, (\mathrm{ofNat}(i+1))$.

Proof. Graph (β) and unfold `seqFun` / `succSet`. ■

Definition (termShift). $\texttt{termShift}(u) = \mathbf{graph}(\lambda t.\, \mathrm{funOf}\, u\, (\mathit{succSet}\, t))$ — the same index shift on arbitrary arguments (for LAMBDA terms later).

Theorem (termShift.app, termShift.ofNat). As for `seqShift`, with `funOf` in place of `seqFun`.

Proof. `theorem_1_3` and (β). ■

Theorem (seqFunLt.zero). $\texttt{seqFunLt}\, u\, z\, 0 = \bot$.

Proof. No index $i < 0$. ■

Theorem (seqFunLt.succ).

$$z \supset \mathrm{funOf}\, u\, 0,\ \texttt{seqFunLt}(\texttt{seqShift}\, u, \mathit{predSet}\, z, n) = \texttt{seqFunLt}\, u\, z\, (n+1).$$

This is the recursive step Scott uses because $\cup$ is not primitive in LAMBDA.

Proof. Case split on McCarthy conditional; use `seqShift.ofNat` to align indices. ■

Theorem (`seqFunLt.term`). Same identity with `termShift` in place of `seqShift` (term-level shift for definability proofs).

Proof. Parallel to `seqFunLt.succ`. ■

(Equations (2.19)–(2.20) for empty and singleton tuples are $\bot$ and `seqCons`-special cases; the dollar combinator and full LAMBDA term for `seqFun` continue in later notes.)

`sequences.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more
public import Mathlib.Topology.Separation.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def seqFun (u x : Pomega) : Pomega :=
  ⋃ i ∈ x, funOf u (ofNat i)

def seqFunLt (u x : Pomega) (n : ℕ) : Pomega :=
  ⋃ i ∈ x, ⋃ _ : i < n, funOf u (ofNat i)

theorem mem_seqFunLt {u x : Pomega} {n k : ℕ} :
    k ∈ seqFunLt u x n ↔ ∃ i ∈ x, i < n ∧ k ∈ funOf u (ofNat i) := by
  constructor
  · intro hk
    obtain ⟨i, hi, hrest⟩ := Set.mem_iUnion₂.mp hk
    obtain ⟨hlt, hki⟩ := Set.mem_iUnion.mp hrest
    exact ⟨i, hi, hlt, hki⟩
  · intro ⟨i, hi, hlt, hki⟩
    exact Set.mem_iUnion₂.mpr ⟨i, hi, Set.mem_iUnion.mpr ⟨hlt, hki⟩⟩

def seqCons (x xs : Pomega) : Pomega :=
  graph (fun z => condSet z x (funOf xs (predSet z)))

namespace seqCons

theorem body (x xs : Pomega) :
    IsScottContinuous (fun z => condSet z x (funOf xs (predSet z))) :=
  theorem_1_3.tuple
    (fun _ => condSet.isScottContinuous_left x _)
    (fun z => condSet.isScottContinuous_right z x)
    id_isScottContinuous
    (theorem_1_3 (funOf.isScottContinuous xs) predSet.isScottContinuous)

end seqCons
```

```
def seq2 (x y : Pomega) : Pomega :=
  graph (fun z => condSet z x (condSet (predSet z) y botElem))

namespace seq2

theorem body_isScottContinuous (x y : Pomega) :
    IsScottContinuous (fun z => condSet z x (condSet (predSet z) y botElem)) := by
  constructor
  intro z
  have H := theorem_1_3.diag (x := z)
    (f := fun z w => condSet z x (condSet (predSet w) y botElem))
    (fun _w => condSet.isScottContinuous_left x _)
    (fun _z =>
      theorem_1_3 (condSet.isScottContinuous_right _z x)
        (theorem_1_3 (condSet.isScottContinuous_left y botElem)
          predSet.isScottContinuous))
  ext k
  constructor
  · intro hk
    have : k ∈ ⋃ n, {k | e n ⊆ z ∧
        k ∈ condSet (e n) x (condSet (predSet (e n)) y botElem)} := by
      rwa [← H]
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr ⟨n, hn, hkn⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    have : k ∈ ⋃ n, {k | e n ⊆ z ∧
        k ∈ condSet (e n) x (condSet (predSet (e n)) y botElem)} :=
      Set.mem_iUnion.mpr ⟨n, hn, hkn⟩
    rwa [← H] at this

theorem app (x y z : Pomega) :
    funOf (seq2 x y) z = condSet z x (condSet (predSet z) y botElem) :=
  beta (seq2.body_isScottContinuous x y) z

theorem one (x y : Pomega) : funOf (seq2 x y) (ofNat 1) = y := by
  rw [app]
  ext n
  simp [condSet, predSet, ofNat, botElem]

theorem zero (x y : Pomega) : funOf (seq2 x y) (ofNat 0) = x := by
  rw [app]
  ext n
  simp [condSet, predSet, ofNat, botElem]

theorem tag_ne (x y : Pomega) :
    seq2 (ofNat 0) x = seq2 (ofNat 1) y := by
  intro h
  have hfun := congrArg (fun w => funOf w (ofNat 0)) h
  have hx : (0 : ℕ) ∈ funOf (seq2 (ofNat 0) x) (ofNat 0) :=
    ⟨1, fun k hk =>
      match k, hk with
      | 0, _ => rfl
      | k + 1, hk =>
        False.elim (Bool.false_ne_true
```

```
          (((Nat.testBit_succ 1 k).trans (Nat.zero_testBit k)).symm.trans hk)),
      ⟨1, 0, rfl, Or.inl ⟨rfl, rfl⟩⟩⟩
  have hy : (0 : ℕ) ∈ funOf (seq2 (ofNat 1) y) (ofNat 0) := by
    intro ⟨k, hk, hp⟩
    rcases hp with ⟨k', m, heq, hm⟩
    obtain ⟨rfl, rfl⟩ := pair_inj heq
    rcases hm with ⟨h1, _⟩ | ⟨_, ⟨t, ht⟩⟩
    · exact Nat.zero_ne_one h1
    · exact Nat.succ_ne_zero t (hk ht)
  exact hy (hfun ▸ hx)

end seq2

theorem eq_2_22 (x xs z : Pomega) :
    funOf (seqCons x xs) z = condSet z x (funOf xs (predSet z)) :=
  beta (seqCons.body x xs) z

theorem eq_2_23_zero (x y : Pomega) : funOf (seq2 x y) (ofNat 0) = x :=
  seq2.zero x y

theorem eq_2_23_one (x y : Pomega) : funOf (seq2 x y) (ofNat 1) = y :=
  seq2.one x y

theorem eq_2_23_ge (x y : Pomega) (n : ℕ) :
    funOf (seq2 x y) (ofNat (n + 2)) = botElem := by
  rw [seq2.app]
  ext k
  simp [condSet, predSet, ofNat, botElem]

namespace seqFun

theorem eq_iUnion_lt (u z : Pomega) :
    seqFun u z = ⋃ n, seqFunLt u z n := by
  ext k
  constructor
  · intro hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hk
    exact Set.mem_iUnion.mpr ⟨i + 1, (mem_seqFunLt).mpr ⟨i, hi, Nat.lt_succ_self i, hki
    ⟩⟩
  · intro hk
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hk
    obtain ⟨i, hi, _, hki⟩ := (mem_seqFunLt).mp hn
    exact Set.mem_iUnion₂.mpr ⟨i, hi, hki⟩

theorem left_isScottContinuous (x : Pomega) :
    IsScottContinuous (fun u => seqFun u x) := by
  constructor
  intro u
  ext k
  constructor
  · intro hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hk
    have : k ∈ scottUnion (fun v => funOf v (ofNat i)) u := by
      rw [← (funOf.isScottContinuous_left (ofNat i)).out u]; exact hki
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp this
    exact mem_scottUnion.mpr ⟨n, hn, Set.mem_iUnion₂.mpr ⟨i, hi, hkn⟩⟩
```

```
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hkn
    exact Set.mem_iUnion₂.mpr ⟨i, hi,
      IsScottContinuous.monotone (funOf.isScottContinuous_left (ofNat i)) hn hki⟩

theorem right_isScottContinuous (u : Pomega) :
    IsScottContinuous (fun x => seqFun u x) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hk
    exact mem_scottUnion.mpr ⟨2 ^ i, by simp [e.pow2, Set.singleton_subset_iff, hi],
      Set.mem_iUnion₂.mpr ⟨i, by simp [e.pow2], hki⟩⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hkn
    exact Set.mem_iUnion₂.mpr ⟨i, hn hi, hki⟩

end seqFun

def seqShift (u : Pomega) : Pomega :=
  graph (fun z => seqFun u (succSet z))

namespace seqShift

theorem isScottContinuous (u : Pomega) :
    IsScottContinuous (fun z => seqFun u (succSet z)) :=
  theorem_1_3 (seqFun.right_isScottContinuous u) succSet.isScottContinuous

theorem app (u z : Pomega) :
    funOf (seqShift u) z = seqFun u (succSet z) :=
  beta (seqShift.isScottContinuous u) z

theorem left_isScottContinuous : IsScottContinuous seqShift :=
  graph_const_isScottContinuous (fun u z => seqFun u (succSet z))
    (fun z => seqFun.left_isScottContinuous (succSet z))

theorem ofNat (u : Pomega) (i : ℕ) :
    funOf (seqShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
  rw [seqShift.app]
  ext m
  constructor
  · intro hm
    obtain ⟨j, hj, hmj⟩ := Set.mem_iUnion₂.mp hm
    simp [succSet] at hj; subst hj; exact hmj
  · intro hm
    exact Set.mem_iUnion₂.mpr ⟨i + 1, ⟨i, rfl, rfl⟩, hm⟩

end seqShift

def termShift (u : Pomega) : Pomega :=
  graph (fun t => funOf u (succSet t))
```

```
namespace termShift

theorem isScottContinuous (u : Pomega) :
    IsScottContinuous (fun t => funOf u (succSet t)) :=
  theorem_1_3 (funOf.isScottContinuous u) succSet.isScottContinuous

theorem app (u t : Pomega) :
    funOf (termShift u) t = funOf u (succSet t) :=
  beta (termShift.isScottContinuous u) t

theorem left_isScottContinuous : IsScottContinuous termShift :=
  graph_const_isScottContinuous (fun u t => funOf u (succSet t))
    (fun t => funOf.isScottContinuous_left (succSet t))

theorem ofNat (u : Pomega) (i : ℕ) :
    funOf (termShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
  rw [termShift.app, succSet.ofNat]

end termShift

namespace seqFunLt

theorem succ (u z : Pomega) (n : ℕ) :
    condSet z (funOf u (ofNat 0)) (seqFunLt (seqShift u) (predSet z) n) =
      seqFunLt u z (n + 1) := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, h0z⟩ | ⟨hpos, t, ht⟩
    · exact (mem_seqFunLt).mpr ⟨0, h0z, Nat.succ_pos n, hk0⟩
    · obtain ⟨i, hi, hlt, hki⟩ := (mem_seqFunLt).mp hpos
      refine (mem_seqFunLt).mpr ⟨i + 1, hi, Nat.succ_lt_succ hlt, ?_⟩
      simpa [seqShift.ofNat] using hki
  · intro hk
    obtain ⟨i, hi, hlt, hki⟩ := (mem_seqFunLt).mp hk
    cases i with
    | zero => exact Or.inl ⟨hki, hi⟩
    | succ i =>
      refine Or.inr ⟨?_, i, hi⟩
      refine (mem_seqFunLt).mpr ⟨i, hi, Nat.lt_of_succ_lt_succ hlt, ?_⟩
      simpa [seqShift.ofNat] using hki

theorem term (u z : Pomega) (n : ℕ) :
    condSet z (funOf u (ofNat 0)) (seqFunLt (termShift u) (predSet z) n) =
      seqFunLt u z (n + 1) := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, h0z⟩ | ⟨hpos, t, ht⟩
    · exact (mem_seqFunLt).mpr ⟨0, h0z, Nat.succ_pos n, hk0⟩
    · obtain ⟨i, hi, hlt, hki⟩ := (mem_seqFunLt).mp hpos
      refine (mem_seqFunLt).mpr ⟨i + 1, hi, Nat.succ_lt_succ hlt, ?_⟩
      simpa [termShift.ofNat] using hki
  · intro hk
    obtain ⟨i, hi, hlt, hki⟩ := (mem_seqFunLt).mp hk
    cases i with
```

```
    | zero => exact Or.inl ⟨hki, hi⟩
    | succ i =>
      refine Or.inr ⟨?_, i, hi⟩
      refine (mem_seqFunLt).mpr ⟨i, hi, Nat.lt_of_succ_lt_succ hlt, ?_⟩
      simpa [termShift.ofNat] using hki

theorem zero (u z : Pomega) : seqFunLt u z 0 = botElem := by
  ext k
  simp [mem_seqFunLt, botElem]

end seqFunLt

end DomainTheory.DataTypesAsLattices
```

3.3.13 Distributive functions ✓

Equation 2.23 gives us the method of LAMBDA-defining *finite sequences* (in a quite natural way), and the next step is to consider *infinite sequences*. But these are just the functions $\hat{p}$ where $p : \omega \to \mathbf{P}\omega$ is arbitrary. What we need then is a condition expressible in the language equivalent to saying $u = \hat{p}$ for some p. This is the same as

$$u = \lambda x.\ \bigcup\{u_i \mid i \in x\}.$$

but the $\cup$- and set-notation is not part of LAMBDA. We are forced into a recursive definition:

$$(2.24) \quad \$ = \mathbf{Y}(\lambda s \lambda u \lambda z.\ z \supset u_0,\ s(\lambda t.\ u_{t+1})(z-1)).$$

This equation generalizes (2.22) and we have:

$$(2.25) \quad \$(u) = \lambda x.\ \bigcup\{u_i \mid i \in x\}.$$

Thus the combinator \$"revalues" an element as a distributive function. This suggests introducing theλ-notation for such functions by the equation:

$$(2.26) \quad \lambda n \in \omega.\ \tau = \$(\lambda z.\ \tau[z/n]).$$

With all these conventions LAMBDA-notation becomes very much like ordinary mathematical notation without too much strain.

Recall (from minimal extension) that a continuous $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is *distributive* when $f(\varnothing) = \varnothing$ and $f(x \cup y) = f(x) \cup f(y)$. The spread $\hat{p}(x) = \bigcup\{p(n) \mid n \in x\}$ is the canonical example. `seqFun` in `sequences.lean` is $\hat{p}$ at fixed u. The combinator \$packages that revaluation, and (2.26) expresses ω-indexedλ-notation in terms of\$.

Formalization (`distributive_functions.lean`). Scott's (2.24)–(2.26) are split into a **model** step (graphs, `dollarStep`, `dollarC`) and a **language** step (`dollarStepTerm`, `termDollarStep`) linked by `interp` and Theorems 2.4–2.5. Each Lean name is the library entry; proofs in the file are short unless noted.

Definition (`dollarStepBody`). For $s, u, z \in \mathbf{P}\omega$,

$$\texttt{dollarStepBody}(s,u,z) = z \supset \text{funOf}\, u\, 0,\ \text{funOf}(\text{funOf}\, s\, (\texttt{seqShift}\, u))(\mathit{predSet}\, z)$$

— the McCarthy conditional form of the body in (2.24) (head u_0, tail from shifted index family at $z-1$).

Theorem (`dollarStepBody.isScottContinuous_s, ..._u, ..._z`). Each of $s \mapsto \texttt{dollarStepBody}(s,u,z)$, $u \mapsto \texttt{dollarStepBody}(s,u,z)$, and $z \mapsto \texttt{dollarStepBody}(s,u,z)$ is Scott-continuous.

Proof. `theorem_1_3` / `theorem_1_3.tuple` from continuity of `condSet`, `funOf`, `seqShift`, and `predSet`. ■

Definition (`dollarStep`). $\texttt{dollarStep}(s) = \mathbf{graph}(\lambda u.\, \mathbf{graph}(\lambda z.\, \texttt{dollarStepBody}(s,u,z)))$ — the graph of Scott's inner $\lambda u\, \lambda z$ in (2.24) at step parameter s.

Theorem (`dollarStep.isScottContinuous`). `IsScottContinuous dollarStep`.

Proof. `graph_const_isScottContinuous` and the body continuity lemmas. ■

Theorem (`dollarStep.app`, `dollarStep.app2`). $\mathrm{funOf}(\texttt{dollarStep}\, s)\, u = \mathbf{graph}(\lambda z.\, \texttt{dollarStepBod}$ and $\mathrm{funOf}(\mathrm{funOf}(\texttt{dollarStep}\, s)\, u)\, z = \texttt{dollarStepBody}(s,u,z)$.

Proof. (β) on nested graphs. ■

Theorem (`dollarStep.iterate`). For all n, u, z, $\mathrm{funOf}(\mathrm{funOf}(\mathrm{iterateBot}\, \texttt{dollarStep}\, n)\, u)\, z = \texttt{seqFunLt}\, u\, z\, n$.

Proof. Induction on n; base is $\bot$; step uses `dollarStep.app2`, unfold `dollarStepBody`, and `seqFunLt.succ` from `sequences.lean`. ■

Theorem (`dollarStep.fix_app`). $\mathrm{funOf}(\mathrm{funOf}(\mathrm{fix}\, \texttt{dollarStep})\, u)\, z = \texttt{seqFun}\, u\, z$.

Proof. Unfold `fix` as a directed union of iterates; identify with `seqFun.eq_iUnion_lt`. ■

Definition (`termDollarStepBody`, `termDollarStep`). Same as `dollarStepBody` / `dollarStep`, but `termShift` replaces `seqShift` (for interpreting LAMBDA terms that mention u_{t+1}).

Theorem (`termDollarStep.isScottContinuous`, `termDollarStep.app`, `termDollarStep.app2`, `termDollarStep.iterate`, `termDollarStep.fix_app`). Parallel to the `dollarStep` namespace; `iterate` uses `seqFunLt.term`.

Definition (`dollarStepTerm`). Closed `Term` for (2.24):

$$\mathbf{Y}(\lambda s\, \lambda u\, \lambda z.\, z \supset u_0,\ s(\lambda t.\, u_{t+1})(z-1))$$

in de Bruijn indices (`Language` / McCarthy conditional).

Theorem (`dollarStepTerm.interp`). With $\rho(i) = \bot$ for all i,

$$\mathrm{interp}(\texttt{dollarStepTerm})\, \rho = \mathbf{graph}(\texttt{termDollarStep}).$$

Proof. Unfold `interp`, `termDollarStep`, `termShift`, and `condSet`; reflexivity. ■

Definition (`dollarC`, combinator \$).

$$\texttt{dollarC} = \mathbf{graph}(\lambda u.\, \mathbf{graph}(\lambda x.\, \texttt{seqFun}\, u\, x)).$$

Scott's (2.25) is $\$(u) = \lambda x.\, \texttt{seqFun}(u,x)$ in graph form.

Theorem (`dollarC.app`, equation (2.25)). For all $u, x \in \mathbf{P}\omega$,

$$\mathrm{funOf}(\mathrm{funOf}\, \texttt{dollarC}\, u)\, x = \texttt{seqFun}\, u\, x.$$

Proof. `dollarC.app` and (β) using `seqFun.right_isScottContinuous`. ■

Theorem (`dollarC.eq_fix_termDollarStep`). $\texttt{dollarC} = \mathrm{fix}\, \texttt{termDollarStep}$.

Proof. Fixed-point law (Theorem 1.4); show each body value equals `seqFun` on the fixpoint, then re-graph. ■

Theorem (`dollarC.combinatory`). `IsCombinatory dollarC`.

Proof. `funOf Ycomb (interp dollarStepTerm ...) = dollarC` via `dollarStepTerm.interp`, Theorem 2.5, and `eq_fix_termDollarStep`; combine `Ycomb.combinatory` with Theorem 2.4 on the closed term. ■

Theorem (`eq_2_24`, spread recursion). For all $u, x \in \mathbf{P}\omega$,

$$\texttt{seqFun}\, u\, x = x \supset \text{funOf}\, u\, 0,\ \texttt{seqFun}(\texttt{seqShift}\, u)(\mathit{predSet}\, x).$$

This is the set-theoretic recursion underlying (2.24), before introducing \$.

Proof. Extensionality on members; split indices $i \in x$ into $i = 0$ vs $i = j + 1$; rewrite with `seqShift.app` and `ofNat`. ■

Theorem (`eq_2_24.Y`). $\texttt{dollarC} = \text{fix}\,\texttt{dollarStep}$.

Proof. Same pattern as `dollarC.eq_fix_termDollarStep`, using `dollarStep.fix_app`. ■

Theorem (`eq_2_24.Ycomb`, equation (2.24) as Y).

$$\text{funOf}\ \texttt{Ycomb}\ (\mathbf{graph}\ \texttt{dollarStep}) = \texttt{dollarC}.$$

Proof. Theorem 2.5 on `dollarStep.isScottContinuous` and `eq_2_24.Y`. ■

Theorem (`eq_2_25`). Identical to `dollarC.app` (second form of (2.25)).

Definition (`lamOmega`, equation (2.26)). For $\tau : \omega \to \mathbf{P}\omega$ (written $\tau[n]$ for $\tau(n)$),

$$\texttt{lamOmega}\,\tau = \text{funOf}\,\texttt{dollarC}\left(\mathbf{graph}\left(\lambda z.\ \bigcup\{\tau(n) \mid n \in z\}\right)\right).$$

This is Scott's $\lambda n \in \omega.\, \tau$ as \$applied to the graph of the family$z \mapsto \bigcup\{\tau(n) \mid n \in z\}$(formally:\$$(\lambda z.\, \tau[z/n])$ once $[z/n]$ is the term substitution convention).

(Primitive recursion as a distributive function — (2.27) — is in `eq_2_27.lean` / the primitive recursion note.)

`distributive_functions.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.sequences
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def dollarStepBody (s u z : Pomega) : Pomega :=
  condSet z (funOf u (ofNat 0))
    (funOf (funOf s (seqShift u)) (predSet z))

namespace dollarStepBody

theorem isScottContinuous_s (u z : Pomega) :
    IsScottContinuous (fun s => dollarStepBody s u z) := by
  have hinner : IsScottContinuous
      (fun s => funOf (funOf s (seqShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
```

```
      (g := fun s => funOf s (seqShift u))
      (funOf.isScottContinuous_left (predSet z))
      (funOf.isScottContinuous_left (seqShift u))
  exact theorem_1_3 (condSet.isScottContinuous_right z (funOf u (ofNat 0))) hinner

theorem isScottContinuous_u (s z : Pomega) :
    IsScottContinuous (fun u => dollarStepBody s u z) := by
  have helse : IsScottContinuous
      (fun u => funOf (funOf s (seqShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
      (g := fun u => funOf s (seqShift u))
      (funOf.isScottContinuous_left (predSet z))
      (theorem_1_3 (funOf.isScottContinuous s) seqShift.left_isScottContinuous)
  exact theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_mid z y)
    (fun x => condSet.isScottContinuous_right z x)
    (funOf.isScottContinuous_left (ofNat 0)) helse

theorem isScottContinuous_z (s u : Pomega) :
    IsScottContinuous (fun z => dollarStepBody s u z) :=
  theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_left (funOf u (ofNat 0)) y)
    (fun z => condSet.isScottContinuous_right z (funOf u (ofNat 0)))
    id_isScottContinuous
    (theorem_1_3 (funOf.isScottContinuous (funOf s (seqShift u)))
      predSet.isScottContinuous)

end dollarStepBody

def dollarStep (s : Pomega) : Pomega :=
  graph (fun u => graph (fun z => dollarStepBody s u z))

namespace dollarStep

theorem isScottContinuous : IsScottContinuous dollarStep :=
  graph_const_isScottContinuous
    (fun s u => graph (fun z => dollarStepBody s u z))
    (fun u => graph_const_isScottContinuous
      (fun s z => dollarStepBody s u z)
      (fun z => dollarStepBody.isScottContinuous_s u z))

theorem app (s u : Pomega) :
    funOf (dollarStep s) u = graph (fun z => dollarStepBody s u z) :=
  beta (graph_const_isScottContinuous (fun u z => dollarStepBody s u z)
    (fun z => dollarStepBody.isScottContinuous_u s z)) u

theorem app2 (s u z : Pomega) :
    funOf (funOf (dollarStep s) u) z = dollarStepBody s u z := by
  rw [dollarStep.app]
  exact beta (dollarStepBody.isScottContinuous_z s u) z

theorem iterate (n : ℕ) (u z : Pomega) :
    funOf (funOf (iterateBot dollarStep n) u) z = seqFunLt u z n := by
  induction n generalizing u z with
  | zero =>
    simp [iterateBot, funOf.bot, seqFunLt.zero]
```

```
  | succ n ih =>
    rw [iterateBot, dollarStep.app2, dollarStepBody, ih, seqFunLt.succ]

theorem fix_app (u z : Pomega) :
    funOf (funOf (fix dollarStep) u) z = seqFun u z := by
  have : funOf (funOf (fix dollarStep) u) z =
      ⋃ n, funOf (funOf (iterateBot dollarStep n) u) z := by
    simp [fix, funOf.iUnion]
  rw [this, seqFun.eq_iUnion_lt]
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨n, by simpa [dollarStep.iterate] using hn⟩
  · intro hk
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨n, by simpa [dollarStep.iterate] using hn⟩

end dollarStep

def termDollarStepBody (s u z : Pomega) : Pomega :=
  condSet z (funOf u (ofNat 0))
    (funOf (funOf s (termShift u)) (predSet z))

namespace termDollarStepBody

theorem isScottContinuous_u (s z : Pomega) :
    IsScottContinuous (fun u => termDollarStepBody s u z) := by
  have helse : IsScottContinuous
      (fun u => funOf (funOf s (termShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
      (g := fun u => funOf s (termShift u))
      (funOf.isScottContinuous_left (predSet z))
      (theorem_1_3 (funOf.isScottContinuous s) termShift.left_isScottContinuous)
  exact theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_mid z y)
    (fun x => condSet.isScottContinuous_right z x)
    (funOf.isScottContinuous_left (ofNat 0)) helse

theorem isScottContinuous_z (s u : Pomega) :
    IsScottContinuous (fun z => termDollarStepBody s u z) :=
  theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_left (funOf u (ofNat 0)) y)
    (fun z => condSet.isScottContinuous_right z (funOf u (ofNat 0)))
    id_isScottContinuous
    (theorem_1_3 (funOf.isScottContinuous (funOf s (termShift u)))
      predSet.isScottContinuous)

theorem isScottContinuous_s (u z : Pomega) :
    IsScottContinuous (fun s => termDollarStepBody s u z) := by
  have hinner : IsScottContinuous
      (fun s => funOf (funOf s (termShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
      (g := fun s => funOf s (termShift u))
      (funOf.isScottContinuous_left (predSet z))
      (funOf.isScottContinuous_left (termShift u))
```

```
  exact theorem_1_3 (condSet.isScottContinuous_right z (funOf u (ofNat 0))) hinner

end termDollarStepBody

def termDollarStep (s : Pomega) : Pomega :=
  graph (fun u => graph (fun z => termDollarStepBody s u z))

namespace termDollarStep

theorem isScottContinuous : IsScottContinuous termDollarStep :=
  graph_const_isScottContinuous
    (fun s u => graph (fun z => termDollarStepBody s u z))
    (fun u => graph_const_isScottContinuous
      (fun s z => termDollarStepBody s u z)
      (fun z => termDollarStepBody.isScottContinuous_s u z))

theorem app (s u : Pomega) :
    funOf (termDollarStep s) u = graph (fun z => termDollarStepBody s u z) :=
  beta (graph_const_isScottContinuous (fun u z => termDollarStepBody s u z)
    (fun z => termDollarStepBody.isScottContinuous_u s z)) u

theorem app2 (s u z : Pomega) :
    funOf (funOf (termDollarStep s) u) z = termDollarStepBody s u z := by
  rw [app]
  exact beta (termDollarStepBody.isScottContinuous_z s u) z

theorem iterate (n : ℕ) (u z : Pomega) :
    funOf (funOf (iterateBot termDollarStep n) u) z = seqFunLt u z n := by
  induction n generalizing u z with
  | zero =>
    simp [iterateBot, funOf.bot, seqFunLt.zero]
  | succ n ih =>
    rw [iterateBot, app2, termDollarStepBody, ih, seqFunLt.term]

theorem fix_app (u z : Pomega) :
    funOf (funOf (fix termDollarStep) u) z = seqFun u z := by
  have : funOf (funOf (fix termDollarStep) u) z =
      ⋃ n, funOf (funOf (iterateBot termDollarStep n) u) z := by
    simp [fix, funOf.iUnion]
  rw [this, seqFun.eq_iUnion_lt]
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨n, by simpa [iterate] using hn⟩
  · intro hk
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨n, by simpa [iterate] using hn⟩

end termDollarStep

def dollarStepTerm : Term :=
  .lam 0 (.lam 1 (.lam 2
    (.cond (.var 2)
      (.app (.var 1) .zero)
      (.app (.app (.var 0) (.lam 3 (.app (.var 1) (.succ (.var 3))))))
```

```
        (.pred (.var 2))))))

namespace dollarStepTerm

theorem interp :
    interp dollarStepTerm (fun _ => botElem) = graph termDollarStep := by
  unfold dollarStepTerm termDollarStep termDollarStepBody termShift DomainTheory.
    DataTypesAsLattices.interp
  apply congrArg graph
  funext s
  simp only[envSet]
  apply congrArg graph
  funext u
  simp only[envSet]
  apply congrArg graph
  funext z
  simp[envSet, condSet, succSet]
  rfl

end dollarStepTerm

def dollarC : Pomega :=
  graph (fun u => graph (fun x => seqFun u x))

namespace dollarC
theorem app (u : Pomega) :
    funOf dollarC u = graph (fun x => seqFun u x) :=
  beta (graph_const_isScottContinuous (fun u x => seqFun u x)
    (fun x => seqFun.left_isScottContinuous x)) u

theorem eq_fix_termDollarStep : dollarC = fix termDollarStep := by
  have hfix : termDollarStep (fix termDollarStep) = fix termDollarStep :=
    (theorem_1_4 termDollarStep.isScottContinuous).1
  have hbody : ∀ u z, termDollarStepBody (fix termDollarStep) u z = seqFun u z := by
    intro u z
    calc termDollarStepBody (fix termDollarStep) u z
        = funOf (funOf (termDollarStep (fix termDollarStep)) u) z :=
          (termDollarStep.app2 _ _ _).symm
      _ = funOf (funOf (fix termDollarStep) u) z := by rw [hfix]
      _ = seqFun u z := termDollarStep.fix_app u z
  have : termDollarStep (fix termDollarStep) =
      graph (fun u => graph (fun z => seqFun u z)) := by
    apply congrArg graph
    funext u
    apply congrArg graph
    funext z
    exact hbody u z
  exact this.symm.trans hfix

theorem combinatory : IsCombinatory dollarC := by
  have hY : funOf Ycomb (interp dollarStepTerm (fun _ => botElem)) = dollarC := by
    rw [dollarStepTerm.interp, theorem_2_5 termDollarStep.isScottContinuous,
      dollarC.eq_fix_termDollarStep]
  exact hY ▸ IsCombinatory.app Ycomb.combinatory (theorem_2_4.closed dollarStepTerm)

end dollarC
```

```
theorem eq_2_24 (u x : Pomega) :
    seqFun u x =
      condSet x (funOf u (ofNat 0))
        (seqFun (seqShift u) (predSet x)) := by
  ext k
  constructor
  · intro hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hk
    cases i with
    | zero => exact Or.inl ⟨hki, hi⟩
    | succ i =>
      refine Or.inr ⟨?_, i, hi⟩
      refine Set.mem_iUnion₂.mpr ⟨i, hi, ?_⟩
      have : funOf (seqShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
        rw [seqShift.app]
        ext m
        constructor
        · intro hm
          obtain ⟨j, hj, hmj⟩ := Set.mem_iUnion₂.mp hm
          simp [succSet, ofNat] at hj; subst hj; exact hmj
        · intro hm
          exact Set.mem_iUnion₂.mpr ⟨i + 1, ⟨i, rfl, rfl⟩, hm⟩
      simpa [this] using hki
  · intro hk
    rcases hk with ⟨hk0, h0x⟩ | ⟨hpos, t, ht⟩
    · exact Set.mem_iUnion₂.mpr ⟨0, h0x, hk0⟩
    · obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hpos
      have : funOf (seqShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
        rw [seqShift.app]
        ext m
        constructor
        · intro hm
          obtain ⟨j, hj, hmj⟩ := Set.mem_iUnion₂.mp hm
          simp [succSet, ofNat] at hj; subst hj; exact hmj
        · intro hm
          exact Set.mem_iUnion₂.mpr ⟨i + 1, ⟨i, rfl, rfl⟩, hm⟩
      refine Set.mem_iUnion₂.mpr ⟨i + 1, hi, ?_⟩
      simpa [this] using hki

namespace eq_2_24

theorem Y : dollarC = fix dollarStep := by
  have hfix : dollarStep (fix dollarStep) = fix dollarStep :=
    (theorem_1_4 dollarStep.isScottContinuous).1
  have hbody : ∀ u z, dollarStepBody (fix dollarStep) u z = seqFun u z := by
    intro u z
    calc dollarStepBody (fix dollarStep) u z
        = funOf (funOf (dollarStep (fix dollarStep)) u) z :=
          (dollarStep.app2 _ _ _).symm
      _ = funOf (funOf (fix dollarStep) u) z := by rw [hfix]
      _ = seqFun u z := dollarStep.fix_app u z
  have : dollarStep (fix dollarStep) =
      graph (fun u => graph (fun z => seqFun u z)) := by
    apply congrArg graph
    funext u
```

```
    apply congrArg graph
    funext z
    exact hbody u z
  exact this.symm.trans hfix

theorem Ycomb : funOf Ycomb (graph dollarStep) = dollarC := by
  rw [theorem_2_5 dollarStep.isScottContinuous, eq_2_24.Y]

end eq_2_24

theorem eq_2_25 (u x : Pomega) :
    funOf (funOf dollarC u) x = seqFun u x := by
  rw [dollarC.app]
  exact beta (seqFun.right_isScottContinuous u) x

def lamOmega (τ : ℕ → Pomega) : Pomega :=
  funOf dollarC (graph (fun z => ⋃ n ∈ z, τ n))

end DomainTheory.DataTypesAsLattices
```

3.3.14 The definability theorem ✓

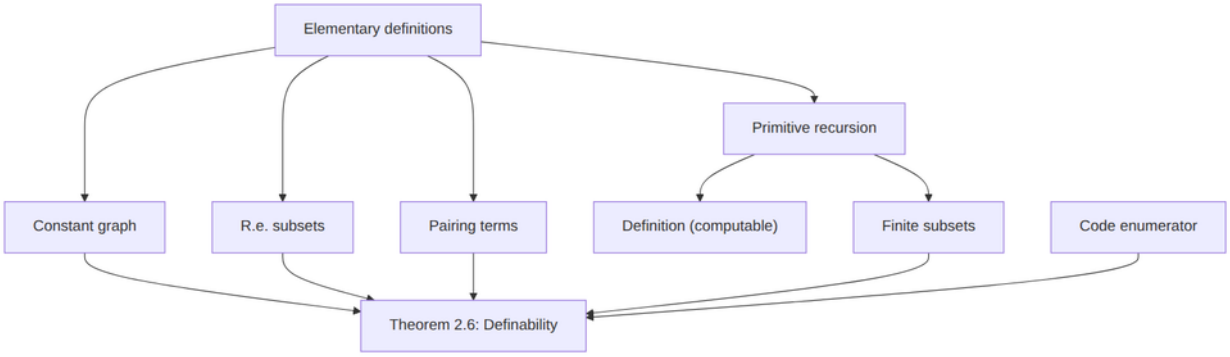


Figure 29: The definability theorem ✓.

Elementary definitions ✓ We state now a sequence of minor results which show why some simple functions and constants are LAMBDA-definable.

$$(2.14) \quad \bot = (\lambda x.\, x(x))(\lambda x.\, x(x));$$

$$(2.15) \quad x \cup y = (\lambda z.\, 0) \supset x, y; \quad \text{(Hint: } 0, 1 \in \lambda z.\, 0)$$

$$(2.16) \quad \top = \mathbf{Y}(\lambda x.\, 0 \cup (x+1));$$

$$(2.17) \quad x \cap y = \mathbf{Y}(\lambda f \lambda x \lambda y.\, x \supset (y \supset 0, \bot), f(x-1)(y-1)+1)(x)(y).$$

The elements $\bot$ and $\top$ are graphs, by the way, and we can characterize them as the only fixed points of the combinator $\mathbf{K}$:

$$(2.18)\quad a = \lambda x.\, a \quad \text{iff} \quad a = \bot \quad \text{or} \quad a = \top.$$

Formalization. Scott's (2.14)–(2.18) are separate library entries; this note's `@import` is `eq_2_18.lean`, which proves (2.18) and also holds the nested `pack4` pairing used later in §2.6. Each name below is the Lean theorem or definition; proofs live in the cited module unless noted.

Theorem (`eq_2_14`, equation (2.14)).

$$\mathrm{funOf}(\omega(\mathrm{id}))(\omega(\mathrm{id})) = \bot,$$

where $\omega(u) = \lambda x.\, u(x(x))$ is `omegaComb` and the left side is Scott's $(\lambda x.\, x(x))(\lambda x.\, x(x))$ as graphs.

Proof. Identify the ω-subcombinator with `graph (fun x => funOf x x)`; apply `omega_first_recursion` so every finite iterate is $\bot$. ■

(Module `eq_2_14.lean`.)

Theorem (`eq_2_15`, equation (2.15)). For all $x, y \in \mathbf{P}\omega$,

$$\mathrm{condSet}(\lambda z.\, 0,\, x,\, y) = x \cup y,$$

with $\lambda z.\, 0$ formalized as `graph (fun _ => ofNat 0)` and $0, 1 \in \lambda z.\, 0$ from membership in that constant graph.

Proof. Case-split on the McCarthy conditional using $0, 1 \in \mathrm{graph}(\lambda z.\, 0)$. ■

(Module `eq_2_15.lean`.)

Theorem (`eq_2_16`, equation (2.16)). $\mathrm{fix}(\texttt{topRec}) = \top$, where `topRec` is the step $x \mapsto 0 \cup (x+1)$ underlying $\mathbf{Y}(\lambda x.\, 0 \cup (x+1))$.

Proof. $\top$ is universal; for membership in $\mathrm{fix}(\texttt{topRec})$, use that iterate $n+1$ already contains n. ■

(Module `eq_2_16.lean`.)

Theorem (`eq_2_17`, equation (2.17)). For all $x, y \in \mathbf{P}\omega$,

$$\mathrm{funOf}(\mathrm{funOf}(\texttt{interC})\, x)\, y = x \cap y,$$

with `interC` the graph of Scott's Y-based intersection step.

Proof. Strong induction on the argument n; unfold `interC.app` and the conditional branches matching (2.17). ■

Theorem (`eq_2_17.step`). The same intersection as a McCarthy recursion:

$$x \cap y = x \supset (y \supset 0,\, \bot),\ f(x-1)(y-1) + 1$$

at the level of `condSet` / `predSet` / `succSet` (the body of (2.17) before abstracting f).

Proof. Extensionality; parallel case analysis to `eq_2_17`. ■

(Module `eq_2_17.lean`.)

Theorem (`eq_2_18.bot`, `eq_2_18.top`). $\bot = \lambda x.\, \bot$ and $\top = \lambda x.\, \top$ as graphs: `botElem = graph (fun _ => botElem)` and `topElem = graph (fun _ => topElem)`.

Proof. For $\bot$, no pair lies in the empty set; for $\top$, every coded pair's value lies in $\mathbf{P}\omega$. ■

Theorem (`eq_2_18`, equation (2.18)). For $a \in \mathbf{P}\omega$,

$$a = \mathbf{graph}(\lambda x.\, a) \leftrightarrow a = \bot \vee a = \top.$$

So the only **K**-fixed graphs are $\bot$ and $\top$.

Proof. If a is a constant graph and $a \neq \top$, least excluded index forces $0 \notin a$; if $a \neq \bot$, least included index yields $0 \in a$ and contradiction. Conversely use `eq_2_18.bot` / `eq_2_18.top`. ■

(Module `eq_2_18.lean` — owner of this note.)

Definition (`pack4`, `pack4.proj1`–`proj4`). Nested `Nat.pair` coding of four naturals and its unpairing projections (not numbered in Scott's (2.14)–(2.18); bundled in `eq_2_18.lean` for §2.6 computability proofs).

`eq_2_18.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
public import DomainTheory.DataTypesAsLattices.Lambda.Language
public import Mathlib.Computability.RE

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

namespace eq_2_18

theorem bot : botElem = graph (fun _ => botElem) := by
  ext p
  constructor
  · intro hp; exact hp.elim
  · intro hp
    rcases hp with ⟨_, _, _, hm⟩
    simp [botElem] at hm

theorem top : topElem = graph (fun _ => topElem) := by
  ext p
  constructor
  · intro _
    obtain ⟨n, m, rfl⟩ := exists_pair p
    exact ⟨n, m, rfl, Set.mem_univ m⟩
  · intro; trivial

end eq_2_18

theorem eq_2_18 (a : Pomega) :
    a = graph (fun _ => a) ↔ a = botElem ∨ a = topElem := by
  constructor
  · intro ha
    by_cases htop : a = topElem
    · exact Or.inr htop
    · have hex : ∃ k, k ∈ a := by
        by_contra h
        apply htop
        ext k
        constructor
```

```
        · intro; trivial
        · intro
          exact not_not.mp (not_exists.mp h k)
      obtain ⟨k, hk, hmin⟩ := exists_least_not_mem hex
      obtain ⟨n, m, hnm⟩ := exists_pair k
      have hm : m ∈ a := by
        intro hma
        have : pair n m ∈ a :=
          ha.symm ▸ (⟨n, m, rfl, hma⟩ : pair n m ∈ graph (fun _ => a))
        exact hk (by rwa [hnm] at this)
      have : m = k :=
        le_antisymm (by rw [← hnm]; exact pair_le_right n m)
          (le_of_not_gt fun hlt => hm (hmin m hlt))
      have hnmk : pair n k = k := by simpa [this] using hnm
      have ⟨_, hk0⟩ := pair_eq_right hnmk
      have h0 : 0 ∈ a := by simpa [hk0] using hk
      by_cases hbot : a = botElem
      · exact Or.inl hbot
      · obtain ⟨l, hl, hlmin⟩ := exists_least_mem (Set.nonempty_iff_ne_empty.mpr hbot)
        obtain ⟨i, j, hij⟩ := exists_pair l
        have hj : j ∈ a := by
          have : pair i j ∈ graph (fun _ => a) :=
            ha ▸ (by rwa [hij] : pair i j ∈ a)
          rcases this with ⟨i', j', hp, hj'⟩
          obtain ⟨rfl, rfl⟩ := pair_inj hp
          exact hj'
        have : j = l :=
          le_antisymm (by rw [← hij]; exact pair_le_right i j)
            (le_of_not_gt fun hlt => hlmin j hlt hj)
        have hil : pair i l = l := by simpa [this] using hij
        have ⟨_, hl0⟩ := pair_eq_right hil
        exact (h0 (by simpa [hl0] using hl)).elim
  · intro h
    rcases h with rfl | rfl
    · exact eq_2_18.bot
    · exact eq_2_18.top

def pack4 (n0 n1 n2 m : ℕ) : ℕ :=
  Nat.pair n0 (Nat.pair n1 (Nat.pair n2 m))

namespace pack4

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

def proj1 (t : ℕ) : ℕ := (Nat.unpair t).1

def proj2 (t : ℕ) : ℕ := (Nat.unpair (Nat.unpair t).2).1

def proj3 (t : ℕ) : ℕ := (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).1

def proj4 (t : ℕ) : ℕ := (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).2

end pack4
```

```
end DomainTheory.DataTypesAsLattices
```

Primitive recursion ✓ Suppose that f is any continuous function and $a \in \mathbf{P}\omega$. We can define $p : \omega \to \mathbf{P}\omega$ in the ordinary way by primitive recursion where:

$$p(0) = a;$$

$$p(n+1) = f(n)(p(n)).$$

The question is: can we give a LAMBDA-definition for $\hat{p}$ (in terms of f and a as constants, say)? The answer is clear, for we can prove:

$$(2.27) \quad \hat{p} = \mathbf{Y}(\lambda u \lambda n \in \omega.\, n \supset a,\ f(n-1)(u(n-1))).$$

This already shows that a large part of the definitions of recursion theory can be given in this special language. Of course, simultaneous (primitive) recursions can be transcribed into LAMBDA with the aid of the ordered tuples of (2.22), (2.23) above. But we can go further and connect with partial (and general) recursive functions. We state first a definition.

Formalization (`primitive_recursion.lean`). Scott's scheme $p(0) = a, p(n+1) = f(n)(p(n))$ is `primRecVal`; the spread $\hat{p}(x) = \bigcup\{p(n) \mid n \in x\}$ is `primRecHat`; the recursive step in (2.27) is `primRecStep`; **(2.27)** is `eq_2_27` via \$ (`dollarC`). Each name below is in this module.

Definition (`primRecStepTerm`). LAMBDA term for $\lambda u\, \lambda n.\, n \supset a,\ f(n-1)(u(n-1))$ (de Bruijn form with parameters a, f abstracted).

Theorem (`primRecStepTerm.interp`). Interpretation of `primRecStepTerm` matches the body of `primRecStep` at each u, n.

Proof. Four (β) steps and unfold `interp` / `condSet`. ■

Definition (`primRecStep`). $\texttt{primRecStep}(a, f)(u) = \mathbf{graph}(\lambda n.\, n \supset a,\ f(\mathit{predSet}\, n)(u(\mathit{predSet}\, n)))$ — the step function inside Scott's $\mathbf{Y}$ in (2.27).

Theorem (`primRecStep.body_isScottContinuous_n`). For fixed a, f, and u, the map $n \mapsto (n \supset a,\ f(n-1)(u(n-1)))$ is Scott continuous.

Theorem (`primRecStep.graph_of`). The graph of $u \mapsto \texttt{primRecStep}(a, f)(u)$ is the interpretation of `primRecStepTerm` applied to the parameters a and f.

Theorem (`primRecStep.isScottContinuous`). The map $u \mapsto \texttt{primRecStep}(a, f)(u)$ is Scott continuous.

Theorem (`primRecStep.app`). Applying that graph at n returns $n \supset a,\ f(n-1)(u(n-1))$.

Definition (`primRecVal`). $\texttt{primRecVal}(a, f)(0) = a, \texttt{primRecVal}(a, f)(n{+}1) = \mathrm{funOf}(\mathrm{funOf}\, f\, (\mathrm{ofNat}\, n))(\mathrm{p}$ — the numeric schema above.

Theorem (`primRecVal.fix`). $\mathrm{funOf}(\mathrm{fix}(\texttt{primRecStep}\, a\, f))(\mathrm{ofNat}\, n) = \texttt{primRecVal}(a, f)\, n$ for all n.

Proof. Fixed-point law (Theorem 1.4); case $n = 0$ and $n+1$ via `primRecStep.app` and predecessor lemmas. ■

Definition (`primRecHat`). $\texttt{primRecHat}(a, f)(x) = \bigcup\{\texttt{primRecVal}(a, f)(n) \mid n \in x\}$ — formal $\hat{p}$.

Theorem (`primRecHat.zero`, `primRecHat.ofNat`). $\texttt{primRecHat}(a, f)(0) = a$ and $\texttt{primRecHat}(a, f)(n) = \texttt{primRecVal}(a, f)(n)$ on singletons $\mathrm{ofNat}\, n$.

Theorem (`eq_2_27`, equation (2.27)). For all $a, f, x \in \mathbf{P}\omega$,

$$\texttt{primRecHat}(a, f)(x) = \text{funOf}(\text{funOf}\ \texttt{dollarC}\ (\text{fix}(\texttt{primRecStep}\ a\ f)))(x).$$

Proof. Rewrite with `eq_2_25` (spread as \$); index-wise use `primRecVal.fix`. ■

Definition (`precTerm`). A second LAMBDA term for the same scheme, written with an explicit zero value, step function, and predecessor rather than with $\mathbf{Y}$. The combinator `precC` is its interpretation.

`primitive_recursion.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_2
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_zero
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_succ
public import DomainTheory.DataTypesAsLattices.Lambda.distributive_functions
public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Language
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def primRecStepTerm : Term :=
  .lam 0 (.lam 1 (.lam 2 (.lam 3
    (.cond (.var 3) (.var 0)
      (.app (.app (.var 1) (.pred (.var 3)))
        (.app (.var 2) (.pred (.var 3))))))))

namespace primRecStepTerm

theorem interp (a f u n : Pomega) :
    funOf (funOf (funOf (funOf (interp primRecStepTerm (fun _ => botElem)) a) f) u) n
    =
      condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))) := by
  change funOf (funOf (funOf (funOf (DomainTheory.DataTypesAsLattices.interp (Term.lam
    0 _) (fun _ => botElem)) a) f) u) n = _
  rw [theorem_2_2.beta]
  rw [theorem_2_2.beta]
  rw [theorem_2_2.beta]
  rw [theorem_2_2.beta]
  simp [_root_.DomainTheory.DataTypesAsLattices.interp, envSet, Function.update]

end primRecStepTerm

def primRecStep (a f u : Pomega) : Pomega :=
  graph (fun n => condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))))

namespace primRecStep

theorem body_isScottContinuous_n (a f u : Pomega) :
    IsScottContinuous (fun n =>
      condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n)))) :=
```

```
  theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_left a y)
    (fun n => condSet.isScottContinuous_right n a)
    id_isScottContinuous
    (theorem_1_3.tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun x => funOf.isScottContinuous x)
      (theorem_1_3 (funOf.isScottContinuous f) predSet.isScottContinuous)
      (theorem_1_3 (funOf.isScottContinuous u) predSet.isScottContinuous))

theorem graph_of (a f : Pomega) :
    graph (primRecStep a f) =
      funOf (funOf (interp primRecStepTerm (fun _ => botElem)) a) f := by
  change graph (primRecStep a f) =
    funOf (funOf (DomainTheory.DataTypesAsLattices.interp (Term.lam 0 (.lam 1 (.lam 2
    (.lam 3
      (.cond (.var 3) (.var 0)
        (.app (.app (.var 1) (.pred (.var 3)))
          (.app (.var 2) (.pred (.var 3))))))))) (fun _ => botElem)) a) f
  rw [theorem_2_2.beta, theorem_2_2.beta]
  apply congrArg graph
  funext u
  apply congrArg graph
  funext n
  simp [DomainTheory.DataTypesAsLattices.interp, envSet, Function.update]

theorem isScottContinuous (a f : Pomega) :
    IsScottContinuous (primRecStep a f) :=
  graph_const_isScottContinuous
    (fun u n => condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))))
    (fun n => theorem_1_3 (condSet.isScottContinuous_right n a)
      (theorem_1_3 (funOf.isScottContinuous (funOf f (predSet n)))
        (funOf.isScottContinuous_left (predSet n))))

theorem app (a f u n : Pomega) :
    funOf (primRecStep a f u) n =
      condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))) :=
  beta (primRecStep.body_isScottContinuous_n a f u) n

end primRecStep

def primRecVal (a f : Pomega) : ℕ → Pomega
  | 0 => a
  | n + 1 => funOf (funOf f (ofNat n)) (primRecVal a f n)

namespace primRecVal

theorem fix (a f : Pomega) : ∀ n,
    funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat n) =
      primRecVal a f n
  | 0 => by
    have hfix := (theorem_1_4 (primRecStep.isScottContinuous a f)).1
    calc funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat 0)
        = funOf (primRecStep a f
            (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))) (ofNat 0) := by
          rw [hfix]
```

```
      _ = condSet (ofNat 0) a
            (funOf (funOf f (predSet (ofNat 0)))
              (funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))
                (predSet (ofNat 0)))) :=
          primRecStep.app a f _ _
      _ = a := eq_2_7_zero _ _
  | n + 1 => by
    have hfix := (theorem_1_4 (primRecStep.isScottContinuous a f)).1
    calc funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat (n + 1))
    )
        = funOf (primRecStep a f
            (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))) (ofNat (n + 1))
    := by
          rw [hfix]
      _ = condSet (ofNat (n + 1)) a
            (funOf (funOf f (predSet (ofNat (n + 1))))
              (funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))
                (predSet (ofNat (n + 1))))) :=
          primRecStep.app a f _ _
      _ = funOf (funOf f (ofNat n))
            (funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat n))
     := by
          rw [eq_2_7_succ, predSet.ofNat_succ]
      _ = funOf (funOf f (ofNat n)) (primRecVal a f n) := by
          rw [?fix? a f n]

end primRecVal

def primRecHat (a f x : Pomega) : Pomega :=
  ⋃ n ∈ x, primRecVal a f n

namespace primRecHat

theorem zero (a f : Pomega) : primRecHat a f (ofNat 0) = a := by
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion₂.mp hk
    subst hn; simpa [primRecVal] using hkn
  · intro hk
    exact Set.mem_iUnion₂.mpr ⟨0, rfl, hk⟩

theorem ofNat (a f : Pomega) : ∀ n,
    primRecHat a f (ofNat n) = primRecVal a f n
  | 0 => primRecHat.zero a f
  | n + 1 => by
    ext k
    constructor
    · intro hk
      obtain ⟨m, hm, hkm⟩ := Set.mem_iUnion₂.mp hk
      subst hm; simpa [primRecVal] using hkm
    · intro hk
      exact Set.mem_iUnion₂.mpr ⟨n + 1, rfl, hk⟩

end primRecHat
```

```
theorem eq_2_27 (a f x : Pomega) :
    primRecHat a f x = funOf (funOf dollarC (fix (primRecStep a f))) x := by
  rw [eq_2_25]
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion₂.mp hk
    refine Set.mem_iUnion₂.mpr ⟨n, hn, ?_⟩
    simpa [primRecVal.fix] using hkn
  · intro hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion₂.mp hk
    refine Set.mem_iUnion₂.mpr ⟨n, hn, ?_⟩
    simpa [primRecVal.fix] using hkn

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def precTerm : Term :=
  .lam 0
    (.app
      (.app
        (.app (.var 3) (.app (.var 4) (.app (.var 6) (.var 0))))
        (.lam 1 (.lam 2
          (.app (.var 5)
            (.app (.app (.var 8) (.app (.var 6) (.var 0)))
              (.app (.app (.var 8) (.var 1)) (.var 2))))))))
      (.app (.var 7) (.var 0)))

end DomainTheory.DataTypesAsLattices
```

Definition (computable) ✓ **DEFINITION.** A continuous function f of k-variables is *computable* iff the relationship

$$m \in f(e_{n_0})(e_{n_1})\cdots(e_{n_{k-1}})$$

is recursively enumerable (r.e.) in the integer variables $m, n_0, n_1, \ldots, n_{k-1}$.

If q is a partial recursive function in the usual sense, then we can regard it as a mapping $q : \omega \to \omega \cup \{\perp\}$, where $q(n) = \perp$ means that q is undefined at n. Saying that q is partial recursive is just to say that $m \in q(n)$ is r.e. as a relation in n and m. It is easy to see that this is in turn equivalent to the recursive enumerability of the relationship $m \in \hat{q}(e_n)$; and so our definition is formally a generalization of the usual one. But it is also intuitively reasonable. To "compute" $y = f(x)$, in the one-variable case, we proceed by enumeration. First we begin the enumeration of all finite subsets $e_n \subseteq x$. For each of these f starts up an enumeration of the set $f(e_n)$; so we sit back and observe which $m \in f(e_n)$ by enumeration. The totality of all such m for all $e_n \subseteq x$ forms in the end the set y.

R.e. subsets ✓ Scott treats the *recursively enumerable* objects as a subcollection $\mathbf{RE} \subseteq \mathbf{P}\omega$: those elements u whose membership is susceptible to enumeration. In ordinary recursion theory a set $A \subseteq \omega$ is r.e. when the relation "$n \in A$" is recursively enumerable. An element $u \in \mathbf{P}\omega$ is itself a set of natural numbers

(integers coding finite or infinite subsets), so the same idea applies: u is r.e. when one can enumerate the instances of membership $n \in u$.

This is exactly the hypothesis in Theorem 2.6 (ii), where the λ-term associated with a continuous function f is required to be an r.e. *set* of codes. The proof that LAMBDA-definable graphs are r.e. and the construction of r.e. sets from primitive recursive functions both live in the $\mathbf{RE}$ fragment of $\mathbf{P}\omega$ closed under application and definability.

Formalization (`IsRE.lean`). Mathlib's `REPred` captures "recursively enumerable as a predicate on $\mathbb{N}$". For $u \in \mathbf{P}\omega$, **`IsRE u`** holds when $\lambda n.\, n \in u$ is of that kind—equivalently, the membership set $\{\, n \in \omega \mid n \in u \,\}$ is r.e. The proof field is packaged as `IsRE.out`; downstream lemmas use `IsRE` for combinators, application, and Theorem 2.6.

Definition (`IsRE`). $u \in \mathbf{P}\omega$ is **r.e.** iff the membership predicate $n \mapsto (n \in u)$ is recursively enumerable (`IsRE u`).

Theorem (`REPred.empty`). The constantly false predicate on ω is recursively enumerable.

Theorem (`REPred.true`). The constantly true predicate, on any primcodable type, is recursively enumerable.

Theorem (`REPred.or`). If p and q are recursively enumerable predicates on the same primcodable type, then so is $a \mapsto p(a) \vee q(a)$.

`IsRE.lean`

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

class IsRE (u : Pomega) : Prop where
  out : REPred fun n : ℕ => n ∈ u

theorem REPred.empty : REPred fun _ : ℕ => False := by
  unfold REPred
  refine (Partrec.none (α := ℕ) (σ := Unit)).of_eq fun _ => ?_
  ext
  simp [Part.assert]

theorem REPred.true {α} [Primcodable α] : REPred fun _ : α => True := by
  unfold REPred
  refine (Decidable.Partrec.const' (Part.some ())).of_eq fun _ => ?_
  ext
  simp [Part.assert]

theorem REPred.or {α} [Primcodable α] {p q : α → Prop}
    (hp : REPred p) (hq : REPred q) :
    REPred fun a => p a ∨ q a := by
  obtain ⟨k, hk, H⟩ := Partrec.merge' hp hq
  have hg : Computable₂ fun (_ : α) (_ : Unit) => () :=
    (Computable.const (α := α × Unit) ()).to₂
  refine Partrec.of_eq (hk.map hg) fun a => ?_
  ext
```

```
  constructor
  · intro hu
    rw [Part.mem_map_iff] at hu
    obtain ⟨x, hx, hxu⟩ := hu
    have hx' := (H a).1 x hx
    have hpq : p a ∨ q a := by
      cases hx' with
      | inl h =>
        exact Or.inl (Part.mem_assert_iff.1 h).1
      | inr h =>
        exact Or.inr (Part.mem_assert_iff.1 h).1
    exact Part.mem_assert_iff.2 ⟨hpq, Part.mem_some_iff.2 hxu.symm⟩
  · intro hu
    have hpq : p a ∨ q a := (Part.mem_assert_iff.1 hu).1
    have hd : (k a).Dom := (H a).2.mpr <|
      hpq.imp (fun h => ⟨h, trivial⟩) (fun h => ⟨h, trivial⟩)
    rw [Part.mem_map_iff]
    exact ⟨(), ⟨hd, rfl⟩, (Part.mem_some_iff.1 (Part.mem_assert_iff.1 hu).2).symm⟩

end DomainTheory.DataTypesAsLattices
```

Finite subsets The basic neighbourhoods $e_n \subseteq \omega$ are the finite sets coded by the bits of n. The lemmas below are the bit-arithmetic used to treat membership and application as r.e. relations.

Theorem (`lt_two_pow_of_high_bits`). If every bit of n in position k or higher is off, then $n < 2^k$.

Theorem (`bits_subset_lt_two_pow`). If $e_n \subseteq e_b$, then $n < 2^{b+1}$.

Theorem (`eSubset_e_iff`). $e_n \subseteq e_b$ if and only if for every position $k < n$, the bit of n at k is off or the bit of b at k is on.

Theorem (`eSubset_funOf_e_iff`). $e_k \subseteq e_a(e_b)$ if and only if for every position $i < k$, the bit of k at i is off or i lies in the application $e_a(e_b)$.

Theorem (`eSubset_iff_stage`). Suppose the code c returns 0 on input k exactly when $k \in x$. Then $e_n \subseteq x$ if and only if there is a stage s such that, for every position $k < n$, the bit of n at k is off or c has halted on k by stage s.

Theorem (`e_subset_ofNat_iff`). $e_{\mathrm{code}} \subseteq \{n\}$ if and only if $\mathrm{code} = 0$ or $\mathrm{code} = 2^n$.

`eSubset.lean`

```
module

public import Mathlib.Computability.RE
public import Mathlib.Data.Nat.Bitwise
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Lambda.Language

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
```

```
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem lt_two_pow_of_high_bits {n k : ℕ}
    (h : ∀ j, k ≤ j → n.testBit j = false) : n < 2 ^ k := by
  induction n using Nat.binaryRec generalizing k with
  | zero => simp
  | bit b n ih =>
    cases k with
    | zero =>
      have hb : b = false := by
        simpa [Nat.testBit_bit_zero] using h 0 le_rfl
      have hn0 : n = 0 := Nat.zero_of_testBit_eq_false fun j => by
        simpa [Nat.testBit_bit_succ] using h (j + 1) (Nat.zero_le (j + 1))
      simp [hb, hn0]
    | succ k =>
      have : n < 2 ^ k := ih fun j hj => by
        simpa [Nat.testBit_bit_succ] using h (j + 1) (Nat.succ_le_succ hj)
      exact Nat.bit_lt_two_pow_succ_iff.mpr this

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem bits_subset_lt_two_pow {n b : ℕ} (h : e n ⊆ e b) : n < 2 ^ (b + 1) :=
  lt_two_pow_of_high_bits fun _j hj =>
    Bool.eq_false_iff.2 fun hbit =>
      Nat.not_le.mpr hj (e.mem_le (h (e.mem.mpr hbit)))

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem eSubset_e_iff (n b : ℕ) :
    e n ⊆ e b ↔ ∀ k < n, n.testBit k = false ∨ b.testBit k = true := by
  constructor
  · intro h k hk
    by_cases hb : n.testBit k = true
    · exact Or.inr (e.mem.mp (h (e.mem.mpr hb)))
    · exact Or.inl (Bool.eq_false_iff.mpr hb)
  · intro h k hk
    have hlt : k < n := e.mem_lt hk
    exact e.mem.mpr ((h k hlt).resolve_left fun hf =>
      Bool.eq_false_iff.mp hf (e.mem.mp hk))

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem eSubset_funOf_e_iff (k a b : ℕ) :
    e k ⊆ e a ? e b ↔
      ∀ i < k, k.testBit i = false ∨ i ∈ e a ? e b := by
  constructor
  · intro h i hi
    by_cases hb : k.testBit i = true
    · exact Or.inr (h (e.mem.mpr hb))
```

```
    · exact Or.inl (Bool.eq_false_iff.mpr hb)
  · intro h i hi
    have hlt : i < k := e.mem_lt hi
    exact (h i hlt).resolve_left fun hf =>
      Bool.eq_false_iff.mp hf (e.mem.mp hi)

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem eSubset_iff_stage {x : Pomega} {c : Code}
    (hc : Code.eval c = fun k => Part.assert (k ∈ x) fun _ => Part.some (0 : ℕ))
    (n : ℕ) :
    e n ⊆ x ↔ ∃ s, ∀ k < n, n.testBit k = false ∨ (evaln s c k).isSome = true := by
  constructor
  · intro hsub
    let stage : ℕ → ℕ := fun k =>
      if h : n.testBit k = true then
        ((evaln_complete (x := 0)).1 (by
          rw [hc]
          exact Part.mem_assert_iff.mpr ⟨hsub (e.mem.mpr h), Part.mem_some_iff.mpr rfl⟩
    )).choose
      else 0
    refine ⟨(Finset.range n).sup stage, fun k hlt => ?_⟩
    by_cases hb : n.testBit k = true
    · refine Or.inr ?_
      have hle : stage k ≤ (Finset.range n).sup stage :=
        Finset.le_sup (Finset.mem_range.mpr hlt)
      have h0 : 0 ∈ evaln (stage k) c k := by
        simp only [stage, hb, ↓reduceDIte]
        exact ((evaln_complete (x := 0)).1 (by
          rw [hc]
          exact Part.mem_assert_iff.mpr ⟨hsub (e.mem.mpr hb), Part.mem_some_iff.mpr rfl
    ⟩)).choose_spec
      exact (Option.isSome_of_mem (evaln_mono hle h0))
    · exact Or.inl (Bool.eq_false_iff.mpr hb)
  · intro ⟨s, hs⟩ k hk
    have hlt : k < n := e.mem_lt hk
    have hsome : (evaln s c k).isSome = true :=
      (hs k hlt).resolve_left fun hfalse => Bool.eq_false_iff.mp hfalse (e.mem.mp hk)
    obtain ⟨val, hval⟩ := Option.isSome_iff_exists.mp hsome
    have : val ∈ Code.eval c k :=
      evaln_sound (by simpa [Option.mem_def] using hval)
    have : val ∈ (Part.assert (k ∈ x) fun _ => Part.some (0 : ℕ)) := by
      simpa [hc] using this
    exact (Part.mem_assert_iff.mp this).1

theorem e_subset_ofNat_iff (code n : ℕ) :
    e code ⊆ ofNat n ↔ code = 0 ∨ code = 2 ^ n := by
  constructor
  · intro h
    by_cases hz : code = 0
    · exact Or.inl hz
    · have hbit : ∀ k, code.testBit k = true → k = n := by
        intro k hk
        simpa [ofNat] using h (e.mem.mpr hk)
```

```
      have hn : code.testBit n = true := by
        have hk : code.testBit code.log2 = true := Nat.testBit_log2 hz
        simpa [hbit _ hk] using hk
      refine Or.inr (Nat.eq_of_testBit_eq fun k => ?_)
      by_cases hk : k = n
      · subst hk
        simp [hn]
      · have hfalse : code.testBit k = false :=
          Bool.eq_false_iff.mpr fun hb => hk (hbit k hb)
        simp [hfalse, Ne.symm hk]
  · intro h
    rcases h with rfl | rfl
    · intro k hk
      simp [e.zero] at hk
    · intro k hk
      simpa [e.pow2, ofNat] using hk

end DomainTheory.DataTypesAsLattices
```

Pairing terms ✓ Integer pairing and its inverse are written as LAMBDA terms in addition, multiplication, subtraction, square root, and the order test. Write $s = \lfloor \sqrt{n} \rfloor$ and $k = n - s^2$. Mathlib's $\mathrm{Nat.unpair}\, n$ is the pair of components recovered from s and k.

Definition (`pairTerm`). With addition, multiplication, and subtraction supplied, this term applied to the numerals n and m denotes the numeral $\mathrm{Nat.pair}\, n\, m$.

Definition (`unpairSqrtTerm`). Square root applied to the input: the term for s.

Definition (`unpairSquareTerm`). The product of that value with itself: the term for s^2.

Definition (`unpairDeltaTerm`). The input minus that square: the term for k.

Definition (`unpairTestTerm`). The comparison of $k + 1$ with s: the term for the test $k < s$.

Definition (`unpairFstTerm`). The conditional on that test which selects s or k. Interpreted with square root, subtraction, multiplication, and $\leq$, it denotes the first component of $\mathrm{Nat.unpair}\, n$.

Definition (`unpairSndTerm`). The conditional on the same test which selects either $k - s$ or s. In the same interpretation it denotes the second component of $\mathrm{Nat.unpair}\, n$.

`pairTerm.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Language
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def pairTerm : Term :=
  .lam 3 (.lam 4
    (.cond (.app (.app (.var 2) (.var 4)) (.var 3))
      (.app (.app (.var 0)
          (.app (.app (.var 0)
              (.app (.app (.var 1) (.var 3)) (.var 3)))
            (.var 3)))
        (.var 4))
      (.app (.app (.var 0)
          (.app (.app (.var 1) (.var 4)) (.var 4)))
```

```
        (.var 3))))

def unpairSqrtTerm : Term := .app (.var 1) (.var 0)

def unpairSquareTerm : Term :=
  .app (.app (.var 3) unpairSqrtTerm) unpairSqrtTerm

def unpairDeltaTerm : Term :=
  .app (.app (.var 2) (.var 0)) unpairSquareTerm

def unpairTestTerm : Term :=
  .app (.app (.var 4) (.succ unpairDeltaTerm)) unpairSqrtTerm

def unpairFstTerm : Term :=
  .lam 0 (.cond unpairTestTerm unpairSqrtTerm unpairDeltaTerm)

def unpairSndTerm : Term :=
  .lam 0 (.cond unpairTestTerm
    (.app (.app (.var 2) unpairDeltaTerm) unpairSqrtTerm)
    unpairSqrtTerm)

end DomainTheory.DataTypesAsLattices
```

Constant graph ✓ A continuous function is computable when membership in its graph is r.e. The constant finite sets are the first case.

Theorem (`mem_graph_const_iff`). m lies in the graph of the constant function with value e_n if and only if m is a code for a pair (a, b) and the bit of n at position b is on. Equivalently, $b \in e_n$.

`mem_graph_const_iff.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import Mathlib.Data.Nat.Pairing
import Mathlib.Data.Finset.Attr
import Mathlib.Tactic.Attr.Core
import Mathlib.Tactic.Bound.Init
import Mathlib.Tactic.SetLike

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem mem_graph_const_iff (n m : ℕ) :
    m ∈ graph (fun _ => e n) ↔
      ∃ t, m = pair (Nat.unpair t).1 (Nat.unpair t).2 ∧
        n.testBit (Nat.unpair t).2 = true := by
  constructor
  · intro ⟨a, b, hm, hb⟩
    exact ⟨Nat.pair a b, by simp [Nat.unpair_pair, hm],
      by simp [Nat.unpair_pair, e.mem.mp hb]⟩
  · intro ⟨t, ht, hb⟩
```

```
    exact ⟨(Nat.unpair t).1, (Nat.unpair t).2, ht, e.mem.mpr hb⟩

end DomainTheory.DataTypesAsLattices
```

Code enumerator ✓ A nonempty r.e. set is the range of a primitive recursive function. The enumerator below reads a code and a stage.

Definition (`enumFun`). Let t code a pair (i, s). Then $\mathrm{enumFun}\, c\, t$ is $i + 1$ when the code c has halted on input i by stage s, and is 0 otherwise.

enumFun.lean

```
module

public import Mathlib.Computability.PartrecCode
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Nat.Partrec Code

def enumFun (c : Code) (t : ℕ) : ℕ :=
  bif (evaln t.unpair.2 c t.unpair.1).isSome
    then t.unpair.1 + 1
    else 0

end DomainTheory.DataTypesAsLattices
```

Theorem 2.6: Definability ✓ **THEOREM 2.6 (The definability theorem).** *For a k-ary continuous function f, the following are equivalent:*

(i) f is computable;

(ii) $\lambda x_0 \lambda x_1 \cdots \lambda x_{k-1}.\, f(x_0)(x_1)\cdots(x_{k-1})$ as a set is r.e.;

(iii) $\lambda x_0 \lambda x_1 \cdots \lambda x_{k-1}.\, f(x_0)(x_1)\cdots(x_{k-1})$ is LAMBDA-definable.

Proof of Theorem 2.6. In case of a function of several variables, we remark:

$$\lambda x_0 \lambda x_1 \cdots \lambda x_{k-1}.\, f(x_0)(x_1)\cdots(x_{k-1})$$

$$= \{(n_0, (n_1, (\cdots, (n_{k-1}, m)\cdots))) \mid m \in f(e_{n_0})(e_{n_1})\cdots(e_{n_{k-1}})\}.$$

This makes the implication from (i) to (ii) obvious. Conversely, if a graph u is r.e., then from the definition of application we have:

$$m \in u(e_{n_0})(e_{n_1})\cdots(e_{n_{k-1}}) \quad \text{iff} \quad (n_0, (n_1, (\cdots, (n_{k-1}, m)\cdots))) \in u,$$

which is r.e. in $m, n_0, n_1, \ldots, n_{k-1}$. Therefore (i) and (ii) are equivalent.

We have already proved that (ii) implies (iii). For the converse we have only to show that all LAMBDA-definable sets are r.e. For this argument we could take advantage of the combinator theorem, (Theorem

2.4). Each of the six combinators are r.e., and there is no problem of showing that if u and x are r.e., then so is $u(x)$; because it is defined in such an elementary way with existential and bounded universal number quantifiers and with membership in u and in x occurring positively. Explicitly we have:

$$m \in u(x) \quad \text{iff} \quad \exists e_n \subseteq x.\, (n,m) \in u$$

$$\text{iff} \quad \exists n \, \forall m < n \, [[m \in e_n \text{ implies } m \in x] \text{ and } (n,m) \in u].$$

As a hint for the proof we may note that the method of (2.27) shows that all primitive recursive functions p have the corresponding $\hat{p}$ LAMBDA-definable. Next we remark that a nonempty r.e. set is the range of a primitive recursive function; but the range of p is $\hat{p}(\top)$, which is clearly LAMBDA-definable. That any LAMBDA-definable set (graph) is r.e. is obvious from the definition of the language itself. More details are given in the Appendix.

We may draw some interesting conclusions from the definability theorem. In the first place, we see that the countable collection $\mathbf{RE} \subseteq \mathbf{P}\omega$ of r.e. sets is closed under application and LAMBDA-definability. Indeed it forms a model for the λ-calculus (axioms (α), (β), and (ξ^*) at least) and it also contains the arithmetical combinators. (Clearly there will be many intermediate submodels.) In the second place, we can see now how very easy it is to interpret λ-calculus in ordinary arithmetical recursion theory by means of quite elementary operations on r.e. sets. Thus the equivalence of λ-definability with partial recursiveness seems not to be all that good a piece of evidence for Church's Thesis. In his 1936 paper (a footnote on p. 346) Church says about λ-definability:

> The fact, however, that two such widely different and (in the opinion of the author) equally natural definitions of effective calculability turn out to be equivalent adds to the strength of the reasons adduced below for believing that they constitute as general a characterization of this notion as is consistent with the usual intuitive understanding of it.

The point never struck the present author as an especially telling one, and the reduction of λ-calculus to r.e. theory shows that the divergence between the theories is not at all wide. Of course it is a pleasant surprise to see how many complicated things can be defined in pure λ-calculus (without arithmetical combinators), but this fact cuts the wrong way as evidence for the thesis (we want stronger theories, not weaker ones). Post systems (or even first-order theories) are much better to mention in this connection, since they are obviously more inclusive in giving enumerations than Turing machines or Herbrand–Gödel recursion equations. But the equivalence proofs are all so easy! What one would like to see is a "natural" definition where the equivalence with r.e. is not just a mechanical exercise involving a few tricks of coding.

Formalization (`theorem_2_6.lean`).

Theorem (`theorem_2_6`). For a Scott continuous $f : \mathbf{P}\omega \to \mathbf{P}\omega$, the following are equivalent: f is computable; the graph of f is r.e.; the graph of f is combinatory; the graph of f is LAMBDA-definable. Combinatory and LAMBDA-definable agree for every set, so the last two are equivalent without using continuity of f.

Theorem (`theorem_2_6.fin`). The same four-way equivalence for a Scott continuous function of n variables, with the graph replaced by its curried form `curryFin`.

Theorem (`theorem_2_6.combinatory_zero`). The zero combinator is LAMBDA-definable.

Theorem (`theorem_2_6.lambda_continuous`). The interpretation of any LAMBDA term, with every variable except one held at $\perp$, is Scott continuous in that variable.

Theorem (`theorem_2_6.ofNat_isRE`). If the numeral for n is combinatory, then it is r.e.

theorem_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Computability.IsComputable
public import DomainTheory.DataTypesAsLattices.Computability.LambdaDefinable
public import DomainTheory.DataTypesAsLattices.Computability.ComputableFin

import DomainTheory.DataTypesAsLattices.Computability.IsREu2eiff_combinatory
import DomainTheory.DataTypesAsLattices.Computability.IsREu2enumeral
import DomainTheory.DataTypesAsLattices.Computability.combinatory_isLambdaDefinable
import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_1
import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem theorem_2_6 {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    (IsComputable f ↔ IsRE (graph f)) ∧
      (IsRE (graph f) ↔ IsCombinatory (graph f)) ∧
      (LambdaDefinable (graph f) ↔ IsCombinatory (graph f)) := by
  refine ⟨?_, IsRE.iff_combinatory, ?_⟩
  · exact ⟨fun h => h.2, fun h => ⟨hf, h⟩⟩
  · exact ⟨fun h => by
      obtain ⟨t, ht⟩ := h.out
      simpa [ht] using theorem_2_4.closed t,
      combinatory_isLambdaDefinable⟩

namespace theorem_2_6

theorem combinatory_zero : LambdaDefinable zeroC :=
  ⟨⟨Term.zero, rfl⟩⟩

theorem fin {n : ℕ} {f : (Fin n → Pomega) → Pomega}
    (hf : ScottContinuousFin f) :
    (ComputableFin f ↔ IsRE (curryFin f)) ∧
      (IsRE (curryFin f) ↔ IsCombinatory (curryFin f)) ∧
      (LambdaDefinable (curryFin f) ↔ IsCombinatory (curryFin f)) := by
  refine ⟨?_, IsRE.iff_combinatory, ?_⟩
  · exact ⟨fun h => h.out.2, fun h => ⟨⟨hf, h⟩⟩⟩
  · exact ⟨fun h => by
      obtain ⟨t, ht⟩ := h.out
      simpa [ht] using theorem_2_4.closed t,
      combinatory_isLambdaDefinable⟩

theorem lambda_continuous (t : Term) :
    IsScottContinuous (fun v => interp t (envSet (fun _ => botElem) 0 v)) :=
  theorem_2_1 t (fun _ => botElem) 0

theorem ofNat_isRE (n : ℕ) : IsCombinatory (ofNat n) → IsRE (ofNat n) :=
  fun _ => IsRE.numeral n

end theorem_2_6
```

```
end DomainTheory.DataTypesAsLattices
```

3.3.15 A law of Y ✓

In the course of the development in this section we have stated many equations which are not found in Table 1, and which involve new combinators. In conclusion we would like to mention an equation about $\mathbf{Y}$ which holds in the model, which can be stated in pure λ-calculus, and which cannot be proved by ordinary reduction (though we shall not try to justify this last statement here). In order to shorten the calculations, we note from definition (2.8) that $\mathbf{Y}(u) = \mathbf{Y}(\lambda y.\, u(y))$; so by Theorem 2.5 this also equals $u(\mathbf{Y}(u))$.

$$(2.28) \quad \mathbf{Y}(\lambda f \lambda x.\, g(x)(f(x))) = \lambda x.\, \mathbf{Y}(g(x)).$$

Call the left side f' and the right f''. Now $f'' = \lambda x.\, g(x)(\mathbf{Y}(g(x))) = \lambda x.\, g(x)(f''(x))$, thus $f' \subseteq f''$, because f' is a least fixed point. On the other hand $f' = \lambda x.\, g(x)(f'(x))$, so $f'(x) = g(x)(f'(x))$. Thus $f''(x) \subseteq f'(x)$, because $f''(x)$ is a least fixed point. As this holds for all x, we see that $f'' \subseteq f'$; and so they are equal. There must be many other such equations.

eq_2_28.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Ymap
public import DomainTheory.DataTypesAsLattices.Lambda.Fofg

import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
import DomainTheory.DataTypesAsLattices.Graph.beta
import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_28 {g : Pomega → Pomega → Pomega}
    (hgx : ∀ y, IsScottContinuous (fun x => g x y))
    (hgy : ∀ x, IsScottContinuous (fun y => g x y)) :
    fix (Fofg g) = Ymap g := by
  have hF : IsScottContinuous (Fofg g) :=
    graph_const_isScottContinuous (fun f x => g x (funOf f x))
      (fun x => theorem_1_3 (hgy x) (funOf.isScottContinuous_left x))
  have hfix : Fofg g (Ymap g) = Ymap g := by
    apply congrArg graph
    funext x
    rw [Ymap.app hgx hgy]
    exact (theorem_1_4 (hgy x)).1
  have hle : fix (Fofg g) ⊆ Ymap g := (theorem_1_4 hF).2 (Ymap g) hfix
  refine subset_antisymm hle ?_
  have hiter : ∀ n x, funOf (iterateBot (Fofg g) n) x = iterateBot (g x) n := by
    intro n
    induction n with
    | zero =>
```

```
      intro x
      ext m
      simp [iterateBot, funOf, botElem]
    | succ n ih =>
      intro x
      have hcont : IsScottContinuous
          (fun z => g z (funOf (iterateBot (Fofg g) n) z)) :=
        theorem_1_3.tuple hgx hgy id_isScottContinuous
          (funOf.isScottContinuous (iterateBot (Fofg g) n))
      change funOf (lam (fun z => g z (funOf (iterateBot (Fofg g) n) z))) x =
        g x (iterateBot (g x) n)
      rw [beta hcont, ih]
  intro p hp
  rcases hp with ⟨n, m, rfl, hm⟩
  obtain ⟨k, hkm⟩ := Set.mem_iUnion.mp hm
  cases k with
  | zero =>
    simp [iterateBot, botElem] at hkm
  | succ k =>
    have : pair n m ∈ iterateBot (Fofg g) (k + 1) :=
      ⟨n, m, rfl, by
        change m ∈ g (e n) (funOf (iterateBot (Fofg g) k) (e n))
        rw [hiter]
        exact hkm⟩
    exact Set.mem_iUnion.mpr ⟨k + 1, this⟩

end DomainTheory.DataTypesAsLattices
```

3.4 Enumeration and degrees

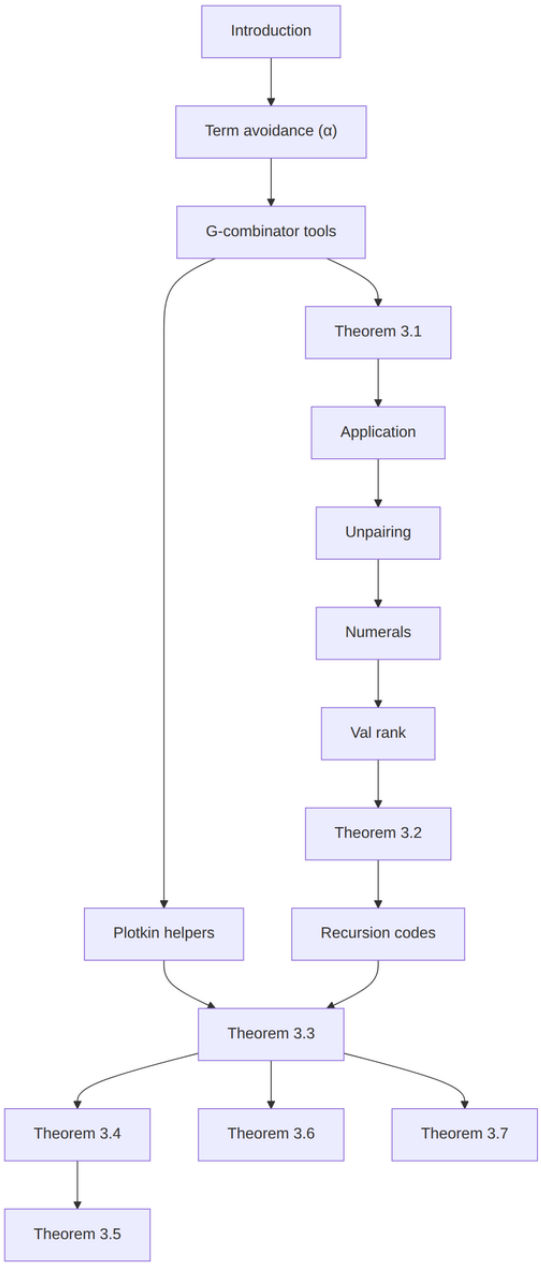


Figure 30: Enumeration and degrees.

3.4.1 Introduction ✓

Reduction to a few combinators is old in combinatory logic. The device special to this model is (3.1), which folds the arithmetical combinators into the one combinator $\mathbf{G}$. The Gödel numbering and the second recursion theorem are the standard constructions, written so that arithmetic and the type-free combinators

share a notation, and the incompleteness theorem is the standard one in that notation. Theorem 3.5 is the argument of [182]; Martin Hyland pointed out the connection. Theorem 3.6 restates enumeration degrees, as in [212], [222], and [103]. Scott notes that this area was underdeveloped beside the theory of Turing degrees. Semigroups of combinators had been discussed by Church and by Böhm, and Theorem 3.7 reads that discussion inside $\mathbf{RE} \cap \mathbf{FUN}$.

A great advantage of the combinators from the formal point of view is that (bound) variables are eliminated in favor of "algebraic" combinations. The disadvantage is that the algebra is not all that pretty, as the combinations tend to get rather long and general laws are rather few. Nevertheless as a technical device it is mildly remarkable that we can have a notation for all r.e. sets requiring so few primitives. In the model defined here the reduction to one combinator rests on a lemma about conditionals:

$$(3.1) \quad \mathbf{cond}(x)(y)(\mathbf{cond}(x)(y)) = y.$$

Recall that $\mathbf{cond}$ (or $\supset$) is a test for zero, so that:

$$(3.2) \quad \mathbf{cond}(x)(y)(0) = x.$$

This suggests that we lump all combinators of Theorem 2.4 into this one:

$$(3.3) \quad \mathbf{G} = \mathbf{cond}(\langle \mathbf{suc}, \mathbf{pred}, \mathbf{cond}, \mathbf{K}, \mathbf{S} \rangle)(0).$$

We can then readily prove Theorem 3.1 below.
(Already included: `1976_Avoids.md`.)

3.4.2 G-combinator tools ✓

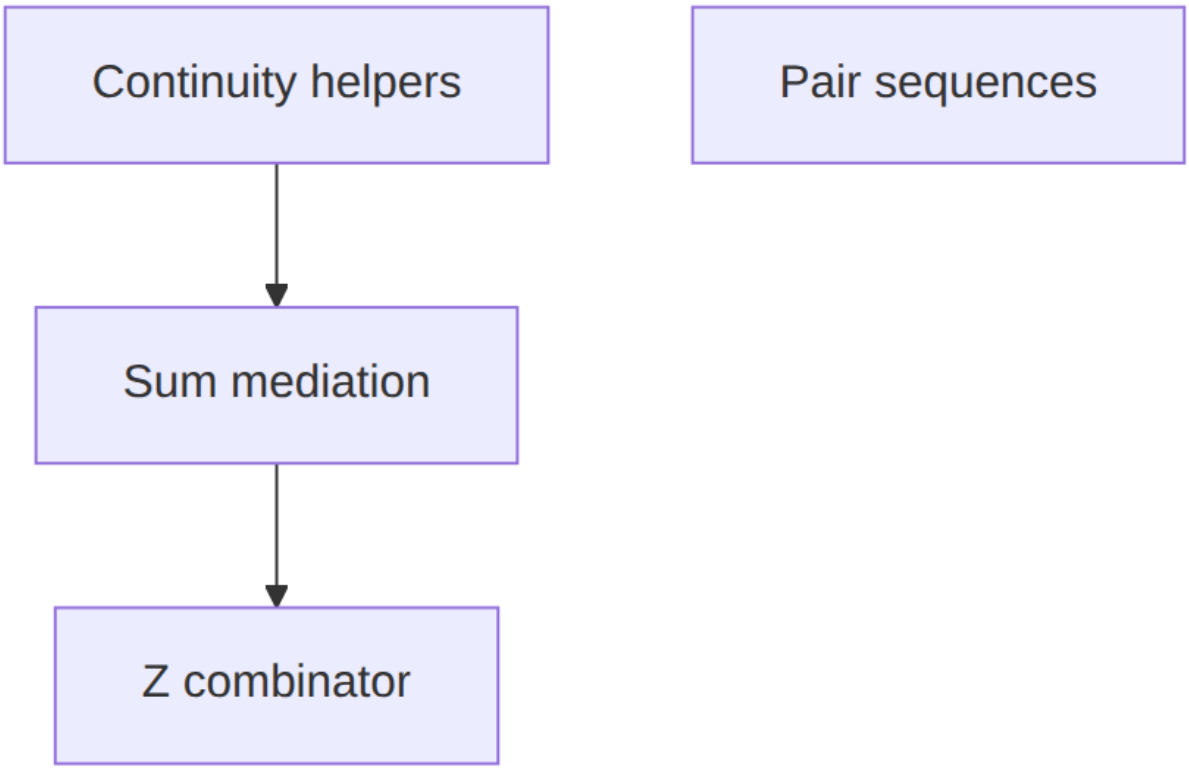


Figure 31: G-combinator tools ✓.

Continuity helpers ✓ Scott-continuity lemmas for diagonal and multi-argument composition.

Theorem (`ContinuityHelpers.on_diag`). If a binary map F is Scott-continuous in each argument separately, then $x \mapsto F(x, x)$ is Scott-continuous.

Proof. The diagonal substitution theorem identifies membership in $F(x, x)$ with a Scott-union over basic approximations $e_n \subseteq x$; translating that union into the Scott-union characterisation of continuity gives the claim. ■

Theorem (`ContinuityHelpers.tuple`). If F is Scott-continuous in each argument and g, h are Scott-continuous, then $x \mapsto F(g(x), h(x))$ is Scott-continuous.

Proof. Reduce to the diagonal case by fixing one composed argument and using substitution continuity of F along g or h. ■

Theorem (`ContinuityHelpers.nary`). If a ternary map F is Scott-continuous in each argument and g, h, i are Scott-continuous, then $x \mapsto F(g(x), h(x), i(x))$ is Scott-continuous.

Proof. Apply the tuple lemma in the first two slots with the third argument held by i, then close under diagonal continuity with substitution along i. ■

`ContinuityHelpers.lean`

```
module
```

```
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.eDense

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace ContinuityHelpers

theorem on_diag {F : Pomega → Pomega → Pomega}
    (h1 : ∀ y, IsScottContinuous (fun x => F x y))
    (h2 : ∀ x, IsScottContinuous (fun y => F x y)) :
    IsScottContinuous (fun x => F x x) := by
  constructor
  intro x
  have H := theorem_1_3.diag (x := x) (f := F) h1 h2
  ext k
  constructor
  · intro hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ F (e n) (e n)} := by rwa [← H]
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr ⟨n, hn, hkn⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ F (e n) (e n)} :=
      Set.mem_iUnion.mpr ⟨n, hn, hkn⟩
    rwa [← H] at this

theorem tuple {F : Pomega → Pomega → Pomega}
    (h1 : ∀ y, IsScottContinuous (fun x => F x y))
    (h2 : ∀ x, IsScottContinuous (fun y => F x y))
    {g h : Pomega → Pomega} (hg : IsScottContinuous g) (hh : IsScottContinuous h) :
    IsScottContinuous (fun x => F (g x) (h x)) :=
  on_diag
    (fun v => theorem_1_3 (h1 (h v)) hg)
    (fun u => theorem_1_3 (h2 (g u)) hh)

theorem nary {F : Pomega → Pomega → Pomega → Pomega}
    (hf0 : ∀ y z, IsScottContinuous (fun x => F x y z))
    (hf1 : ∀ x z, IsScottContinuous (fun y => F x y z))
    (hf2 : ∀ x y, IsScottContinuous (fun z => F x y z))
    {g h i : Pomega → Pomega}
    (hg : IsScottContinuous g) (hh : IsScottContinuous h) (hi : IsScottContinuous i) :
    IsScottContinuous (fun x => F (g x) (h x) (i x)) :=
  on_diag
    (fun v => tuple (fun y => hf0 y (i v)) (fun x => hf1 x (i v)) hg hh)
    (fun u => theorem_1_3 (hf2 (g u) (h u)) hi)

end ContinuityHelpers

end DomainTheory.DataTypesAsLattices
```

Pair sequences ✓ Projection, injectivity, and Scott continuity for the binary pairing sequence.

Theorem (`PairSeq.app_zero`). $\mathrm{pairSeq}(x, y)\, 0 = x$.

Proof. The zero clause of the underlying length-two sequence. ■

Theorem (`PairSeq.app_one`). $\mathrm{pairSeq}(x, y)\, 1 = y$.

Proof. The one clause of the underlying length-two sequence. ■

Theorem (`PairSeq.inj`). If $\mathrm{pairSeq}(x, y) = \mathrm{pairSeq}(x', y')$, then $x = x'$ and $y = y'$.

Proof. Apply the zero and one projections to both sides. ■

Theorem (`PairSeq.isScottContinuous_left`). For fixed y, the map $x \mapsto \mathrm{pairSeq}(x, y)$ is Scott-continuous.

Proof. Graph-constant continuity along the mid-argument continuity of the conditional packing that defines the sequence. ■

Theorem (`PairSeq.isScottContinuous_right`). For fixed x, the map $y \mapsto \mathrm{pairSeq}(x, y)$ is Scott-continuous.

Proof. Graph-constant continuity using continuity of the conditional in the right and mid slots. ■

`PairSeq.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace PairSeq

theorem app_zero (x y : Pomega) : funOf (pairSeq x y) (ofNat 0) = x :=
  seq2.zero x y

theorem app_one (x y : Pomega) : funOf (pairSeq x y) (ofNat 1) = y :=
  seq2.one x y

theorem inj {x y x' y' : Pomega} (h : pairSeq x y = pairSeq x' y') :
    x = x' ∧ y = y' := by
  constructor
  · have := congrArg (fun w => funOf w (ofNat 0)) h
    rw [app_zero, app_zero] at this
    exact this
  · have := congrArg (fun w => funOf w (ofNat 1)) h
    rw [app_one, app_one] at this
    exact this

theorem isScottContinuous_left (y : Pomega) :
    IsScottContinuous (fun x => pairSeq x y) :=
  graph_const_isScottContinuous
    (fun x z => condSet z x (condSet (predSet z) y botElem))
    (fun z => condSet.isScottContinuous_mid z _)

theorem isScottContinuous_right (x : Pomega) :
    IsScottContinuous (fun y => pairSeq x y) :=
```

```
  graph_const_isScottContinuous
    (fun y z => condSet z x (condSet (predSet z) y botElem))
    (fun z =>
      theorem_1_3 (condSet.isScottContinuous_right z x)
        (condSet.isScottContinuous_mid (predSet z) botElem))

end PairSeq

end DomainTheory.DataTypesAsLattices
```

Sum mediation ✓ Tag successor σ_j and the mediating map out of a binary sum.

Definition (`sigmaJ`). $\sigma_j(x) = \{(j, k+1) \mid (j,k) \in x\}$.

Theorem (`sigmaJ.isScottContinuous`). Each σ_j is Scott-continuous.

Proof. A coded successor tag pulls a basic approximation containing the predecessor tag; conversely a Scott-union witness unpacks to such a predecessor. ■

Theorem (`sigmaJ.app`). $\mathrm{graph}(\sigma_j)\, x = \sigma_j(x)$.

Proof. The continuous-graph β-law. ■

Theorem (`sigmaJ.bot`). $\sigma_j(\bot) = \bot$.

Proof. The empty set contributes no tagged pairs. ■

Theorem (`sigmaJ.of_no_tag`). If u contains no pair with left tag j, then $\sigma_j(u) = \bot$.

Proof. Every member of $\sigma_j(u)$ would require such a predecessor tag. ■

Theorem (`sigmaJ.pair_union`). If $j \neq j'$, then $\sigma_j\big(\{(j,m)\} \cup \{(j',m)\}\big) = \{(j, m+1)\}$.

Proof. Only the matching tag survives; the other tag is excluded by distinctness. ■

Theorem (`sigmaJ.succ_ne`). If $j \neq j'$, then $\sigma_j(\{(j',m)\}) = \bot$.

Proof. A membership would force the tags equal. ■

Theorem (`sigmaJ.succ_singleton`). $\sigma_j(\{(j,m)\}) = \{(j, m+1)\}$.

Proof. Direct unpacking of the single tagged pair. ■

Theorem (`sigmaJ.union_fresh`). If u has no j-tag, then $\sigma_j\big(u \cup \{(j,m)\}\big) = \{(j, m+1)\}$.

Proof. Reorder the union and apply the singleton and no-tag cases. ■

Definition (`sumMed`). $\mathrm{sumMed}(f, g)$ is the graph of

$$u \mapsto \begin{cases} f(u_1) & \text{if } u_0 = 0, \\ g(u_1) & \text{otherwise,} \end{cases}$$

via the strict conditional on the zero and one projections of u.

Theorem (`sumMed.body_isScottContinuous`). That body map in u is Scott-continuous for fixed f and g.

Proof. Ternary continuity of the conditional along continuous projections and applications of f and g. ■

Theorem (`sumMed.app`). Applying $\mathrm{sumMed}(f, g)$ recovers the conditional body.

Proof. The β-law for the continuous body. ■

Theorem (`sumMed.app_inl`). $\mathrm{sumMed}(f, g)\, \langle 0, x\rangle = f(x)$.

Proof. Unfold application and the left injection projections. ■

Theorem (`sumMed.app_inr`). $\mathrm{sumMed}(f, g)\, \langle 1, x\rangle = g(x)$.

Proof. Symmetric with the right injection. ■

Theorem (`sumMed.isGraph`). $\mathrm{sumMed}(f, g)$ equals the graph of its own application.

Proof. Pointwise graph extension from $\mathrm{sumMed.app}$. ■

Theorem (`sumMed.rel`). If $f : A \to C$ and $g : B \to C$ as morphisms of restricted equivalences, then $\mathrm{sumMed}(f, g) : A + B \to C$.

Proof. Related sum elements are left or right injections; reduce via $\mathrm{sumMed.app_inl}$ or $\mathrm{sumMed.app_inr}$ to the assumed relatedness of f or g. ■

Theorem (`sumMed.rel_unique`). Any $h : A + B \to C$ that agrees with f on left injections and with g on right injections is related (as an arrow) to $\mathrm{sumMed}(f, g)$.

Proof. On related sum elements, rewrite h along the injection form and compare to the mediating applications of f or g. ■

SumMed.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
import DomainTheory.DataTypesAsLattices.Enumeration.ContinuityHelpers
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
import DomainTheory.DataTypesAsLattices.Graph.beta
import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
import DomainTheory.DataTypesAsLattices.Retracts.graphLemmas

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def sigmaJ (j : ℕ) (x : Pomega) : Pomega :=
  {p | ∃ k, pair j (k + 1) = p ∧ pair j k ∈ x}

namespace sigmaJ

theorem isScottContinuous (j : ℕ) : IsScottContinuous (sigmaJ j) := by
  constructor
  intro x
  ext p
  constructor
  · intro ⟨k, heq, hp⟩
    refine mem_scottUnion.mpr ⟨2 ^ pair j k, ?_, k, heq, ?_⟩
    · simpa [e.pow2, Set.singleton_subset_iff] using hp
    · simp [e.pow2]
  · intro hp
    obtain ⟨n, hn, k, heq, hpk⟩ := mem_scottUnion.mp hp
    exact ⟨k, heq, hn hpk⟩

theorem app (j : ℕ) (x : Pomega) :
    funOf (graph (sigmaJ j)) x = sigmaJ j x := by
  simpa using congrArg (fun g => g x) (theorem_1_2.i (isScottContinuous j))

theorem bot (j : ℕ) : sigmaJ j botElem = botElem := by
  ext p
  simp [sigmaJ, botElem]

theorem of_no_tag {j : ℕ} {u : Pomega}
    (hu : ∀ k, pair j k ∈ u) :
```

```
    sigmaJ j u = botElem := by
  ext p
  constructor
  · intro ⟨k, _hp, hk⟩
    exact (hu k hk).elim
  · simp [botElem]

theorem pair_union (j j' m : ℕ) (hne : j = j') :
    sigmaJ j (({pair j m} : Pomega) ∪ {pair j' m}) =
      ({pair j (m + 1)} : Pomega) := by
  ext p
  constructor
  · intro ⟨k, hp, hk⟩
    rcases hk with hk | hk
    · have hkm : k = m := (pair_inj hk).2
      subst k
      simpa using hp.symm
    · have hj : j = j' := (pair_inj hk).1
      exact (hne hj).elim
  · intro hp
    exact ⟨m, hp.symm, Or.inl (by simp)⟩

theorem succ_ne (j j' m : ℕ) (hne : j = j') :
    sigmaJ j {pair j' m} = botElem := by
  ext p
  constructor
  · intro ⟨k, _hk, hp⟩
    have : pair j k = pair j' m := by
      simpa [Set.mem_singleton_iff] using hp
    exact (hne (pair_inj this).1).elim
  · intro hp
    simp [botElem] at hp

theorem succ_singleton (j m : ℕ) :
    sigmaJ j {pair j m} = ({pair j (m + 1)} : Pomega) := by
  ext p
  constructor
  · intro ⟨k, heq, hp⟩
    have : k = m := (pair_inj hp).2
    subst this
    simp [heq]
  · intro hp
    exact ⟨m, hp.symm, by simp⟩

theorem union_fresh {j m : ℕ} {u : Pomega}
    (hu : ∀ k, pair j k ∈ u) :
    sigmaJ j (u ∪ {pair j m}) = ({pair j (m + 1)} : Pomega) := by
  rw [show u ∪ {pair j m} = ({pair j m} : Pomega) ∪ u by
    exact Set.union_comm _ _]
  ext p
  constructor
  · intro ⟨k, hp, hk⟩
    rcases hk with hk | hk
    · have hkm : k = m := (pair_inj hk).2
      subst k
      simpa using hp.symm
```

```
    · exact (hu k hk).elim
  · intro hp
    exact ⟨m, hp.symm, Or.inl (by simp)⟩

end sigmaJ

def sumMed (f g : Pomega) : Pomega :=
  graph (fun u =>
    condSet (funOf u (ofNat 0))
      (funOf f (funOf u (ofNat 1)))
      (funOf g (funOf u (ofNat 1))))

namespace sumMed

theorem body_isScottContinuous (f g : Pomega) :
    IsScottContinuous (fun u =>
      condSet (funOf u (ofNat 0))
        (funOf f (funOf u (ofNat 1)))
        (funOf g (funOf u (ofNat 1)))) :=
  ContinuityHelpers.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (funOf.isScottContinuous_left (ofNat 0))
    (theorem_1_3 (funOf.isScottContinuous f)
      (funOf.isScottContinuous_left (ofNat 1)))
    (theorem_1_3 (funOf.isScottContinuous g)
      (funOf.isScottContinuous_left (ofNat 1)))

theorem app (f g u : Pomega) :
    funOf (sumMed f g) u =
      condSet (funOf u (ofNat 0))
        (funOf f (funOf u (ofNat 1)))
        (funOf g (funOf u (ofNat 1))) :=
  beta (sumMed.body_isScottContinuous f g) u

theorem app_inl (f g x : Pomega) :
    funOf (sumMed f g) (pairElem (ofNat 0) x) = funOf f x := by
  rw [sumMed.app, pairElem.fst, pairElem.snd, condSet.ofNat_zero]

theorem app_inr (f g x : Pomega) :
    funOf (sumMed f g) (pairElem (ofNat 1) x) = funOf g x := by
  rw [sumMed.app, pairElem.fst, pairElem.snd, condSet.ofNat_one]

theorem isGraph (f g : Pomega) :
    sumMed f g = graph (fun u => funOf (sumMed f g) u) := by
  apply graph_ext
  intro u
  exact (sumMed.app f g u).symm

theorem rel {A B C : RestrictedEquiv} {f g : Pomega}
    (hf : (arrowE A C).mem f) (hg : (arrowE B C).mem g) :
    (arrowE (sumE A B) C).mem (sumMed f g) := by
  refine ⟨sumMed.isGraph f g, sumMed.isGraph f g, ?_⟩
  intro u v huv
  rcases huv with ⟨hu, hv, hA⟩ | ⟨hu, hv, hB⟩
```

```
  · have hmu : funOf (sumMed f g) u = funOf f (funOf u (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hu).trans
        (sumMed.app_inl f g _)
    have hmv : funOf (sumMed f g) v = funOf f (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inl f g _)
    rw [hmu, hmv]
    exact hf.2.2 (funOf u (ofNat 1)) (funOf v (ofNat 1)) hA
  · have hmu : funOf (sumMed f g) u = funOf g (funOf u (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hu).trans
        (sumMed.app_inr f g _)
    have hmv : funOf (sumMed f g) v = funOf g (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inr f g _)
    rw [hmu, hmv]
    exact hg.2.2 (funOf u (ofNat 1)) (funOf v (ofNat 1)) hB

theorem rel_unique {A B C : RestrictedEquiv} {f g h : Pomega}
    (hh : (arrowE (sumE A B) C).mem h)
    (hinl : ∀ x y, A.rel x y →
      C.rel (funOf h (pairElem (ofNat 0) x)) (funOf f y))
    (hinr : ∀ x y, B.rel x y →
      C.rel (funOf h (pairElem (ofNat 1) x)) (funOf g y)) :
    (arrowE (sumE A B) C).rel h (sumMed f g) := by
  refine ⟨hh.1, sumMed.isGraph f g, ?_⟩
  intro u v huv
  rcases huv with ⟨hu, hv, hA⟩ | ⟨hu, hv, hB⟩
  · have hhu :
        funOf h u = funOf h (pairElem (ofNat 0) (funOf u (ofNat 1))) :=
      congrArg (fun w => funOf h w) hu
    have hmv : funOf (sumMed f g) v = funOf f (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inl f g _)
    rw [hhu, hmv]
    exact hinl (funOf u (ofNat 1)) (funOf v (ofNat 1)) hA
  · have hhu :
        funOf h u = funOf h (pairElem (ofNat 1) (funOf u (ofNat 1))) :=
      congrArg (fun w => funOf h w) hu
    have hmv : funOf (sumMed f g) v = funOf g (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inr f g _)
    rw [hhu, hmv]
    exact hinr (funOf u (ofNat 1)) (funOf v (ofNat 1)) hB

end sumMed

end DomainTheory.DataTypesAsLattices
```

Z combinator ✓ Finite iteration of application and its continuous combinator.

Definition (`Z`). For $n \in \mathbb{N}$, a map f, and an argument x,

$$Z(0, f, x) = x, \qquad Z(n+1, f, x) = f\big(Z(n, f, x)\big).$$

Theorem (`Z.rel`). If a restricted equivalence A is preserved by f versus f' on related arguments, and A relates x to y, then A relates $Z(n, f, x)$ to $Z(n, f', y)$.

Proof. Induction on n: the base is the given relatedness of x and y; the step applies the assumed preservation of A by f and f'. ■

Theorem (`Z.finite_bound`). Every member of $Z(m, e_p, e_q)$ is at most $\max(p, q)$.

Proof. Induction on m: the base uses the bound on members of e_q; the step unpacks a coded pair and uses the bound on the left factor from e_p. ■

Theorem (`Z.finite_no_tag`). The tagged pair $(\max(p, q) + 1,\ k)$ does not lie in $Z(m, e_p, e_q)$.

Proof. Any such membership would force $\max(p, q) + 1 \le \max(p, q)$ by the finite bound. ■

Theorem (`Z.graph_funOf`). Iterating the graph of $y \mapsto f(y)$ agrees with iterating f: $Z(n, \mathrm{graph}(y \mapsto f(y)), x) = Z(n, f, x)$.

Proof. Induction on n; the step uses the β-law for the continuous map f. ■

Theorem (`Z.isScottContinuous_left`). For fixed n and x, the map $f \mapsto Z(n, f, x)$ is Scott-continuous.

Proof. Induction on n: the base is a constant map; the step is continuous application of f to a continuous iterate, assembled by the tuple continuity helper. ■

Theorem (`Z.isScottContinuous_right`). For fixed n and f, the map $x \mapsto Z(n, f, x)$ is Scott-continuous.

Proof. Induction on n: the base is the identity; the step composes continuous application of f with the inductive iterate. ■

Definition (`Zcomb`). $\mathrm{Zcomb}(n) = \mathrm{graph}\big(f \mapsto \mathrm{graph}(x \mapsto Z(n, f, x))\big)$.

Theorem (`Zcomb.map_isScottContinuous`). The outer graph map $f \mapsto \mathrm{graph}(x \mapsto Z(n, f, x))$ is Scott-continuous.

Proof. Graph-constant continuity from the left continuity of Z in f. ■

Theorem (`Zcomb.app`). $\mathrm{Zcomb}(n)\ f = \mathrm{graph}(x \mapsto Z(n, f, x))$.

Proof. The β-law for the continuous outer map. ■

Theorem (`Zcomb.app2`). $\mathrm{Zcomb}(n)\ f\ x = Z(n, f, x)$.

Proof. Apply Zcomb.app, then the β-law for the continuous right iterate. ■

Theorem (`Zcomb.isGraph`). $\mathrm{Zcomb}(n)$ equals the graph of $f \mapsto \mathrm{Zcomb}(n)\ f$.

Proof. Pointwise graph extension using Zcomb.app. ■

Theorem (`Zcomb.app_isGraph`). $\mathrm{Zcomb}(n)\ f$ equals the graph of $x \mapsto \mathrm{Zcomb}(n)\ f\ x$.

Proof. From Zcomb.app and the β-law for the right iterate. ■

`Zcomb.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.ContinuityHelpers
public import DomainTheory.DataTypesAsLattices.Enumeration.SumMed
import DomainTheory.DataTypesAsLattices.Graph.beta
import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
import DomainTheory.DataTypesAsLattices.Retracts.graphLemmas
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
public import DomainTheory.DataTypesAsLattices.Pomega.pair
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Z : ℕ → Pomega → Pomega → Pomega
  | 0, _, x => x
  | n + 1, f, x => funOf f (Z n f x)

namespace Z

theorem rel (A : RestrictedEquiv) (n : ℕ) {f f' x y : Pomega}
    (hf : ∀ a b, A.rel a b → A.rel (funOf f a) (funOf f' b))
    (hxy : A.rel x y) :
    A.rel (Z n f x) (Z n f' y) := by
  induction n with
  | zero => simpa [Z] using hxy
  | succ n ih =>
    simpa [Z] using hf _ _ ih

theorem finite_bound (p q m : ℕ) :
    ∀ k ∈ Z m (e p) (e q), k ≤ max p q := by
  induction m with
  | zero =>
      intro k hk
      exact (e.mem_le hk).trans (Nat.le_max_right p q)
  | succ m ih =>
      intro k hk
      obtain ⟨r, _hr, hpair⟩ := hk
      exact (pair_le_right r k).trans
        ((e.mem_le hpair).trans (Nat.le_max_left p q))

theorem finite_no_tag (p q m k : ℕ) :
    pair (max p q + 1) k ∈ Z m (e p) (e q) := by
  intro hk
  have hle := Z.finite_bound p q m _ hk
  exact (Nat.not_succ_le_self (max p q))
    ((pair_le_left (max p q + 1) k).trans hle)

theorem graph_funOf (n : ℕ) (f x : Pomega) :
    Z n (graph (fun y => funOf f y)) x = Z n f x := by
  induction n with
  | zero => simp [Z]
  | succ n ih =>
    simp [Z, ih]
    have : funOf (graph (fun y => funOf f y)) (Z n f x) = funOf f (Z n f x) :=
      beta (funOf.isScottContinuous f) (Z n f x)
    exact this

theorem isScottContinuous_left (n : ℕ) (x : Pomega) :
    IsScottContinuous (fun f => Z n f x) := by
  induction n with
  | zero => simpa [Z] using const_isScottContinuous x
  | succ n ih =>
    simpa [Z] using
      ContinuityHelpers.tuple
```

```
        (fun y => funOf.isScottContinuous_left y)
        (fun u => funOf.isScottContinuous u)
        id_isScottContinuous ih

theorem isScottContinuous_right (n : ℕ) (f : Pomega) :
    IsScottContinuous (fun x => Z n f x) := by
  induction n with
  | zero => simpa [Z] using id_isScottContinuous
  | succ n ih =>
    simpa [Z] using theorem_1_3 (funOf.isScottContinuous f) ih

end Z

def Zcomb (n : ℕ) : Pomega :=
  graph (fun f => graph (fun x => Z n f x))

namespace Zcomb

theorem map_isScottContinuous (n : ℕ) :
    IsScottContinuous (fun f => graph (fun x => Z n f x)) :=
  graph_const_isScottContinuous (fun f x => Z n f x)
    (fun x => Z.isScottContinuous_left n x)

theorem app (n : ℕ) (f : Pomega) :
    funOf (Zcomb n) f = graph (fun x => Z n f x) :=
  beta (Zcomb.map_isScottContinuous n) f

theorem app2 (n : ℕ) (f x : Pomega) :
    funOf (funOf (Zcomb n) f) x = Z n f x := by
  rw [Zcomb.app]
  exact beta (Z.isScottContinuous_right n f) x

theorem isGraph (n : ℕ) :
    Zcomb n = graph (fun f => funOf (Zcomb n) f) := by
  apply graph_ext
  intro f
  rw [Zcomb.app]

theorem app_isGraph (n : ℕ) (f : Pomega) :
    funOf (Zcomb n) f = graph (fun x => funOf (funOf (Zcomb n) f) x) := by
  rw [Zcomb.app]
  apply graph_ext
  intro x
  exact (beta (Z.isScottContinuous_right n f) x).symm

end Zcomb

end DomainTheory.DataTypesAsLattices
```

3.4.3 Theorem 3.1 ✓

THEOREM 3.1 (The generator theorem). *All LAMBDA-definable elements can be obtained from* **G** *by iterated application.*

A distributive function f is said to be *total* iff $f(n) \in \omega$ for all $n \in \omega$. As they come from obvious

primitive recursive functions, we do not stop to write out LAMBDA-definitions of these three total functions:

$$(3.4)\quad \mathbf{apply} = \lambda n \in \omega.\, \lambda m \in \omega.\, (n, m) + 1$$

$$(3.5)\quad \mathbf{op}((n, m)) = n$$

$$(3.6)\quad \mathbf{arg}((n, m)) = m.$$

The point of these auxiliary combinators concerns our Gödel numbering of the r.e. sets. The number 0 will correspond to the generator $\mathbf{G}$; while $(n, m) + 1$ will correspond to the application of the nth set to the mth. This is formalized in the combinator $\mathbf{val}$ which is defined as the least fixed point of the equation:

$$(3.7)\quad \mathbf{val} = \lambda k \in \omega.\, k \supset \mathbf{G},\ \mathbf{val}(\mathbf{op}(k - 1))(\mathbf{val}(\mathbf{arg}(k - 1))).$$

This function accomplishes the enumeration as follows:

Theorem 3.1 is obvious from the construction of $\mathbf{G}$, because $\mathbf{G}(\mathbf{G}) = 0$ and $\mathbf{G}(0)(0) = \mathbf{suc}$, and so the $\mathbf{G}(0)(i)$ give us all the other combinators.

Formalization. Combinatory elements are those built from the base combinators $\mathbf{0}$, $\mathbf{suc}$, $\mathbf{pred}$, $\mathbf{cond}$, $\mathbf{K}$, $\mathbf{S}$ by application. Generation from $\mathbf{G}$ is the inductive closure of $\{\mathbf{G}\}$ under application. The generator theorem equates the two.

Theorem (`theorem_3_1`). An element is combinatory if and only if it is generated from $\mathbf{G}$ by iterated application.

Proof. ($\Rightarrow$) By induction on the combinatory construction: $\mathbf{0} = \mathbf{G}(\mathbf{G})$ and $\mathbf{suc} = \mathbf{G}(0)(0)$, so both lie in the $\mathbf{G}$-closure; the remaining base combinators are obtained by further applications of $\mathbf{G}$ to 0 and $1 = \mathbf{suc}(0)$ as in the generator identities; the inductive step is closure under application. ($\Leftarrow$) By induction on generation from $\mathbf{G}$: $\mathbf{G}$ itself is combinatory, and the combinatory class is closed under application. ■

`theorem_3_1.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedFromG

import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more
import DomainTheory.DataTypesAsLattices.Enumeration.sucC_from_G
import DomainTheory.DataTypesAsLattices.Enumeration.predC_from_G
import DomainTheory.DataTypesAsLattices.Enumeration.condC_from_G
import DomainTheory.DataTypesAsLattices.Enumeration.Kcomb_from_G
import DomainTheory.DataTypesAsLattices.Enumeration.Scomb_from_G
import DomainTheory.DataTypesAsLattices.Enumeration.sucC_zero

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem theorem_3_1 {u : Pomega} :
    IsCombinatory u ↔ GeneratedFromG u := by
  have hzero : GeneratedFromG zeroC := by
    simpa [Gcomb.app_self] using
```

```
      GeneratedFromG.app GeneratedFromG.G GeneratedFromG.G
  constructor
  · intro hu
    have hsuc : GeneratedFromG sucC := by
      simpa [sucC_from_G] using
        GeneratedFromG.app (GeneratedFromG.app GeneratedFromG.G hzero) hzero
    have hone : GeneratedFromG (ofNat 1) := by
      simpa [sucC_zero] using GeneratedFromG.app hsuc hzero
    induction hu with
    | zero => exact hzero
    | suc => exact hsuc
    | pred =>
      simpa [predC_from_G] using
        GeneratedFromG.app
          (GeneratedFromG.app (GeneratedFromG.app GeneratedFromG.G hzero) hone) hzero
    | cond =>
      simpa [condC_from_G] using
        GeneratedFromG.app
          (GeneratedFromG.app
            (GeneratedFromG.app (GeneratedFromG.app GeneratedFromG.G hzero) hone) hone
    )
          hzero
    | K =>
      simpa [Kcomb_from_G] using
        GeneratedFromG.app
          (GeneratedFromG.app
            (GeneratedFromG.app
              (GeneratedFromG.app (GeneratedFromG.app GeneratedFromG.G hzero) hone)
                hone)
            hone)
          hzero
    | S =>
      simpa [Scomb_from_G] using
        GeneratedFromG.app
          (GeneratedFromG.app
            (GeneratedFromG.app
              (GeneratedFromG.app (GeneratedFromG.app GeneratedFromG.G hzero) hone)
                hone)
            hone)
          hone
    | app _ _ iu iv => exact .app iu iv
  · intro hu
    induction hu with
    | G => exact Gcomb.combinatory
    | app _ _ iu iv => exact .app iu iv

end DomainTheory.DataTypesAsLattices
```

3.4.4 Application ✓

Scott codes application of combinators by operations on integers. After Theorem 3.1 he introduces three total (primitive recursive) functions used in the Gödel numbering of r.e. sets; the integer form of application is the first, and the two projections of Cantor pairing are the others. As they come from obvious primitive recursive functions, the paper does not write out LAMBDA-definitions. The point of these auxiliaries is that

0 will correspond to the generator $\mathbf{G}$, while $(n, m) + 1$ will correspond to applying the nth set to the mth.

Definition (`applyNat`, equation (3.4)). For $n, m \in \omega$,

$$\mathrm{applyNat}(n, m) = (n, m) + 1,$$

where (n, m) is Cantor pairing (`pair`). This is the integer form of $\mathbf{apply}(n)(m)$.

Definition (`opNat`, equation (3.5)). $\mathrm{opNat}(k) = (\mathrm{unpair}\, k)_1$. On a coded pair this recovers the first component, so $\mathbf{op}((n, m)) = n$.

Definition (`argNat`, equation (3.6)). $\mathrm{argNat}(k) = (\mathrm{unpair}\, k)_2$. On a coded pair this recovers the second component, so $\mathbf{arg}((n, m)) = m$.

Theorem (`eq_3_5`). For all $n, m \in \omega$, $\mathrm{opNat}((n, m)) = n$.

Proof. Unfold `opNat` and apply `unpair_pair`: $\mathrm{unpair}((n, m)) = (n, m)$. ■

Theorem (`eq_3_6`). For all $n, m \in \omega$, $\mathrm{argNat}((n, m)) = m$.

Proof. Unfold `argNat` and apply `unpair_pair`. ■

Theorem (`eq_3_8_apply`, first half of (3.8)). $\mathrm{applyNat}(0, 0) = 1$.

Proof. $(0, 0) = 0$, so $(0, 0) + 1 = 1$. ■

Theorem (`eq_3_9_apply`, first half of (3.9)). $\mathrm{applyNat}(0, 1) = 3$.

Proof. $(0, 1) = 2$, so $(0, 1) + 1 = 3$. ■

Theorem (`eq_3_10_apply`, first half of (3.10)). $\mathrm{applyNat}(3, 1) = 12$.

Proof. $(3, 1) = 11$, so $(3, 1) + 1 = 12$. ■

These three special cases are the integer codes later used for numerals and $\mathbf{val}$: by (3.8)–(3.10) one has $\mathbf{val}(1) = 0$, $\mathbf{val}(3) = \langle \mathbf{suc}, \ldots \rangle$, and $\mathbf{val}(12) = \mathbf{suc}$.

`applyNat.lean`

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

def applyNat (n m : ℕ) : ℕ := pair n m + 1

noncomputable def opNat (k : ℕ) : ℕ := (unpair k).1

noncomputable def argNat (k : ℕ) : ℕ := (unpair k).2

theorem eq_3_5 (n m : ℕ) : opNat (pair n m) = n := by simp [opNat, unpair_pair]

theorem eq_3_6 (n m : ℕ) : argNat (pair n m) = m := by simp [argNat, unpair_pair]

theorem eq_3_8_apply : applyNat 0 0 = 1 := by simp [applyNat, pair]

theorem eq_3_9_apply : applyNat 0 1 = 3 := by simp [applyNat, pair]

theorem eq_3_10_apply : applyNat 3 1 = 12 := by simp [applyNat, pair]

end DomainTheory.DataTypesAsLattices
```

3.4.5 Unpairing ✓

Cantor pairing is inverted by locating the least triangle bucket that contains a code and reading off the residual. The formalization uses a search lemma and two equivalent presentations of that search, then defines the computational unpair map.

Definition (`triangle`, from `pair.lean`). $\mathrm{triangle}(w) = w(w+1)/2$. The codes with diagonal sum w occupy the half-open interval $[\mathrm{triangle}(w),\ \mathrm{triangle}(w+1))$.

Definition (`unpairWRec`). Recursion on n: $\mathrm{unpairWRec}(k, 0) = 0$, and

$$\mathrm{unpairWRec}(k, n+1) = \begin{cases} \mathrm{unpairWRec}(k, n) & \text{if } k < \mathrm{triangle}(n+1), \\ n+1 & \text{otherwise.} \end{cases}$$

This advances the candidate bucket until k lies below $\mathrm{triangle}(w+1)$.

Definition (`unpairW`). $\mathrm{unpairW}(k) = \mathrm{unpairWRec}(k, k)$.

Definition (`unpairComp`). With $w = \mathrm{Nat.find}(\mathrm{exists_triangle_succ}\, k)$ and $m = k - \mathrm{triangle}(w)$, set $\mathrm{unpairComp}(k) = (w - m,\ m)$.

Theorem (`exists_triangle_succ`). For every $k \in \omega$ there is a w with $k < \mathrm{triangle}(w+1)$.

Proof. Take $w = k$. Then $\mathrm{triangle}(k+1) = \mathrm{triangle}(k) + (k+1)$, so $k < k+1 \le \mathrm{triangle}(k+1)$. ■

Theorem (`unpairWRec.succ`). Unfolding the successor step:

$$\mathrm{unpairWRec}(k, n+1) = \begin{cases} \mathrm{unpairWRec}(k, n) & \text{if } k < \mathrm{triangle}(n+1), \\ n+1 & \text{otherwise.} \end{cases}$$

Proof. By definition of `Nat.rec`. ■

Theorem (`unpairWRec.lt`). For all k, n, either $k < \mathrm{triangle}(\mathrm{unpairWRec}(k, n) + 1)$ or $\mathrm{unpairWRec}(k, n) = n$.

Proof. Induction on n. At 0, case-split on $k < \mathrm{triangle}(1)$. At $n+1$, if $k < \mathrm{triangle}(n+1)$ then the recursive value inherits the claim from the inductive hypothesis (and if that value equals n, the same inequality $k < \mathrm{triangle}(n+1)$ applies); otherwise the value is $n+1$. ■

Theorem (`unpairWRec.min`). If $m < \mathrm{unpairWRec}(k, n)$, then $\neg(k < \mathrm{triangle}(m+1))$.

Proof. Induction on n, generalizing m. At 0 the hypothesis $m < \mathrm{unpairWRec}(k, 0)$ is impossible. At $n+1$, if $k < \mathrm{triangle}(n+1)$ reduce to the inductive hypothesis; otherwise $\mathrm{unpairWRec}(k, n+1) = n+1$, so $m \le n$, and any $k < \mathrm{triangle}(m+1)$ would force $k < \mathrm{triangle}(n+1)$ by monotonicity of triangle, a contradiction. ■

Theorem (`unpairW.eq`). $\mathrm{unpairW}(k) = \mathrm{Nat.find}(\mathrm{exists_triangle_succ}\, k)$.

Proof. Show $\mathrm{unpairWRec}(k, k)$ satisfies the characterizing properties of Nat.find: it is a witness (`unpairWRec.lt`, using $k+1 \le \mathrm{triangle}(k+1)$ when the residual case $\mathrm{unpairWRec}(k, k) = k$ occurs), and no smaller index works (`unpairWRec.min`). ■

Theorem (`unpairComp.eq`). $\mathrm{unpairComp}(k) = \mathrm{unpair}(k)$.

Proof. Let w be that find-witness and $m = k - \mathrm{triangle}(w)$. Minimality of w gives $\mathrm{triangle}(w) \le k$, and $k < \mathrm{triangle}(w+1)$ gives $m \le w$. Then $(w - m) + m = w$, so $\mathrm{pair}(w - m,\ m) = k$ by the triangle form of pairing. Apply `unpair_pair`. ■

`unpairW.lean`

```
module

public import Mathlib.Computability.Halting
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Nat.Partrec.Code

theorem exists_triangle_succ (k : ℕ) : ∃ w, k < triangle (w + 1) :=
  ⟨k, by
    rw [triangle_succ]
    exact lt_of_lt_of_le (Nat.lt_succ_self k) (Nat.le_add_left (k + 1) _)⟩

def unpairWRec (k n : ℕ) : ℕ :=
  Nat.rec 0 (fun w ih => if k < triangle (w + 1) then ih else w + 1) n

namespace unpairWRec

theorem succ (k n : ℕ) :
    unpairWRec k (n + 1) =
      if k < triangle (n + 1) then unpairWRec k n else n + 1 :=
  rfl

theorem lt (k n : ℕ) :
    k < triangle (unpairWRec k n + 1) ∨ unpairWRec k n = n := by
  induction n with
  | zero =>
    by_cases hk : k < triangle 1
    · exact Or.inl (by simp [unpairWRec, hk])
    · exact Or.inr (by simp [unpairWRec])
  | succ n ih =>
    by_cases hk : k < triangle (n + 1)
    · simp [unpairWRec.succ, hk]
      exact ih.elim Or.inl fun heq => Or.inl (by simpa [heq] using hk)
    · simp [unpairWRec.succ, hk]

theorem min (k n m : ℕ)
    (hm : m < unpairWRec k n) : ¬k < triangle (m + 1) := by
  induction n generalizing m with
  | zero =>
    simp [unpairWRec] at hm
  | succ n ih =>
    by_cases hk : k < triangle (n + 1)
    · simp [unpairWRec.succ, hk] at hm
      exact ih m hm
    · simp [unpairWRec.succ, hk] at hm
      intro hlt
      have hm' : m ≤ n := Nat.lt_succ_iff.mp hm
      exact hk (lt_of_lt_of_le hlt (triangle_le_of_le (Nat.succ_le_succ hm')))

end unpairWRec

def unpairW (k : ℕ) : ℕ := unpairWRec k k
```

```
namespace unpairW

theorem eq (k : ℕ) : unpairW k = Nat.find (exists_triangle_succ k) := by
  refine ((Nat.find_eq_iff (exists_triangle_succ k)).mpr ⟨?_, fun m hm =>
      unpairWRec.min k k m hm⟩).symm
  rcases unpairWRec.lt k k with h | hrec
  · exact h
  · have : k + 1 ≤ triangle (k + 1) := by
      cases k with
      | zero => simp [triangle]
      | succ k =>
        rw [triangle_succ]
        exact Nat.le_add_left _ _
    simpa [unpairW, hrec] using lt_of_lt_of_le (Nat.lt_succ_self k) this

end unpairW

def unpairComp (k : ℕ) : ℕ × ℕ :=
  let w := Nat.find (exists_triangle_succ k)
  let m := k - triangle w
  (w - m, m)

namespace unpairComp

theorem eq (k : ℕ) : unpairComp k = unpair k := by
  unfold unpairComp
  set w := Nat.find (exists_triangle_succ k) with hw
  have hlt : k < triangle (w + 1) := by
    simpa [hw] using Nat.find_spec (exists_triangle_succ k)
  have hle : triangle w ≤ k := by
    cases hwc : w with
    | zero => simp [triangle]
    | succ w' =>
      have hlt' : w' < Nat.find (exists_triangle_succ k) := by
        rw [← hw, hwc]
        exact Nat.lt_succ_self w'
      exact Nat.le_of_not_gt (Nat.find_min (exists_triangle_succ k) hlt')
  have hm : k - triangle w ≤ w := by
    have : k < triangle w + (w + 1) := by
      rwa [triangle_succ] at hlt
    omega
  have hpair : pair (w - (k - triangle w)) (k - triangle w) = k := by
    have : w - (k - triangle w) + (k - triangle w) = w := Nat.sub_add_cancel hm
    simp [pair_eq_triangle, this, Nat.add_sub_of_le hle]
  rw [← unpair_pair (w - (k - triangle w)) (k - triangle w), hpair]

end unpairComp

end DomainTheory.DataTypesAsLattices
```

3.4.6 Numerals ✓

The numeral for each integer is built by iterating application, starting from the code of zero.

Definition (num, (3.11)). $\mathrm{num}(0) = 1$ and $\mathrm{num}(n+1) = \mathrm{applyNat}(12, \mathrm{num}(n))$. By (3.8) and (3.10),

1 is the code of 0 and 12 is the code of successor, so $\mathrm{num}(n)$ is the code of the numeral for n.

num.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

def num : ℕ → ℕ
  | 0 => 1
  | n + 1 => applyNat 12 (num n)

end DomainTheory.DataTypesAsLattices
```

3.4.7 Val rank ✓

The enumeration **val** is defined by recursion on application codes. Rank measures the depth of that recursion, and bit lists collect the one-bits of a code when scanning finite approximations.

Definition (`valRank`). $\mathrm{valRank}(0) = 1$, and for a positive code $p + 1$ the rank is one more than the maximum of the ranks of the two Cantor components of p.

Definition (`bitList`). $\mathrm{bitList}(j)$ is the list of indices $k \leq j$ for which the kth bit of j is set.

Definition (`Extensional`). A map p on codes is extensional relative to an enumeration val when $\mathrm{val}(n) = \mathrm{val}(m)$ implies $\mathrm{val}(p(n)) = \mathrm{val}(p(m))$. Theorem 3.5 uses this for $\mathrm{val} = \mathbf{val}$.

valRank.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.Pomega
public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

noncomputable def valRank : ℕ → ℕ
  | 0 => 1
  | p + 1 => max (valRank (unpair p).1) (valRank (unpair p).2) + 1
decreasing_by
  · exact (pair_lt_left (unpair p).1 (unpair p).2).trans_le (by
      have := pair_unpair p
      omega)
  · exact (pair_lt_right (unpair p).1 (unpair p).2).trans_le (by
      have := pair_unpair p
      omega)
```

```
def bitList (j : ℕ) : List ℕ :=
  (List.range (j + 1)).filter (fun k => j.testBit k)

class Extensional (val : ℕ → Pomega) (p : ℕ → ℕ) : Prop where
  out : ∀ n m, val n = val m → val (p n) = val (p m)

end DomainTheory.DataTypesAsLattices
```

3.4.8 Theorem 3.2 ✓

THEOREM 3.2 (The enumeration theorem). *The combinator* **val** *enumerates the LAMBDA-definable elements in that* $\mathbf{RE} = \{\mathbf{val}(n) \mid n \in \omega\}$. *Further:*

(i) $\mathbf{val}(0) = \mathbf{G}$,

(ii) $\mathbf{val}(\mathbf{apply}(n)(m)) = \mathbf{val}(n)(\mathbf{val}(m))$.

As a principal application of the Enumeration Theorem we may mention the following: suppose u is given as LAMBDA-definable. We look at its definition and rewrite it in terms of combinators—eventually in terms of $\mathbf{G}$ alone. Then using 0 and **apply** we write down the name of an integer corresponding to the combination—say, n. By Theorem 3.2 we see that we have *effectively found* from the definition an integer such that $\mathbf{val}\, n = u$. This remark can be strengthened by some numerology.

Formalization. Scott's (i)–(ii) and the set equality $\mathbf{RE} = \{\mathbf{val}(n)\}$ are assembled in `theorem_3_2.lean` from the recursive enumerator `valNat` (equation (3.7)) and Theorem 3.1. Each name below is the Lean theorem; proofs live in that module.

Theorem (`theorem_3_2`). For every n, $\mathrm{valNat}(n) \in \mathbf{RE}$; $\mathrm{valNat}(0) = \mathbf{G}$; and for all n, m,

$$\mathrm{valNat}(\mathrm{applyNat}(n, m)) = \mathrm{valNat}(n) \cdot \mathrm{valNat}(m).$$

Proof. Package three library facts on `valNat`: `valNat.combinatory` (every value is combinatory, hence in $\mathbf{RE}$), `valNat.zero`, and `valNat.app`. ■

Theorem (`theorem_3_2.surjective`). If u is combinatory, then $\exists n.\ \mathrm{valNat}(n) = u$.

Proof. By Theorem 3.1, combinatory elements are exactly those generated from $\mathbf{G}$ by application. Induct on that generation: $\mathbf{G}$ is $\mathrm{valNat}(0)$; if $u = \mathrm{valNat}(n)$ and $v = \mathrm{valNat}(m)$, then $u \cdot v = \mathrm{valNat}(\mathrm{applyNat}(n, m))$ by `valNat.app`. ■

Theorem (`theorem_3_2.i`). $\mathrm{condSet}(0, \mathbf{G}, \bot) = \mathbf{G}$.

Proof. Extensionality on membership. Left-to-right: a member of the conditional is either in $\mathbf{G}$ on the true branch, or comes from a witness that $0 \in 0$, which is impossible. Right-to-left: every member of $\mathbf{G}$ sits on the true branch. ■

Theorem (`theorem_3_2.range`). $\mathbf{RE} = \mathrm{Set.range}\ \mathrm{valNat}$.

Proof. If $u \in \mathbf{RE}$, then u is combinatory, so `surjective` supplies a preimage. Conversely, every $\mathrm{valNat}(n)$ lies in $\mathbf{RE}$ by the first conjunct of `theorem_3_2`. ■

`theorem_3_2.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.RE
public import DomainTheory.DataTypesAsLattices.Enumeration.valNat

import DomainTheory.DataTypesAsLattices.Enumeration.theorem_3_1
import Mathlib.Algebra.GroupWithZero.Nat
```

```
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem theorem_3_2 :
    (∀ n, valNat n ∈ RE) ∧
      valNat 0 = Gcomb ∧
        (∀ n m, valNat (applyNat n m) = valNat n ? valNat m) :=
  ⟨fun n => valNat.combinatory n, valNat.zero, valNat.app⟩

namespace theorem_3_2

theorem surjective {u : Pomega} (hu : IsCombinatory u) :
    ∃ n, valNat n = u := by
  revert hu
  rw [theorem_3_1]
  intro h
  induction h with
  | G => exact ⟨0, valNat.zero⟩
  | app _ _ iu iv =>
    obtain ⟨n, hn⟩ := iu
    obtain ⟨m, hm⟩ := iv
    exact ⟨applyNat n m, by rw [valNat.app, hn, hm]⟩

theorem i : condSet (ofNat 0) Gcomb botElem = Gcomb := by
  ext n
  constructor
  · intro hn
    rcases hn with ⟨hG, _⟩ | ⟨_, ⟨k, hk⟩⟩
    · exact hG
    · simp [ofNat] at hk
  · intro hG
    exact Or.inl ⟨hG, rfl⟩

theorem range : RE = Set.range valNat := by
  ext u
  constructor
  · intro hu
    exact theorem_3_2.surjective hu
  · intro ⟨n, hn⟩
    simpa [← hn] using (theorem_3_2).1 n

end theorem_3_2

end DomainTheory.DataTypesAsLattices
```

3.4.9 Recursion codes ✓

Integer codes used in the second recursion theorem package the diagonal substitution $m \mapsto \mathbf{apply}(m)(\mathbf{num}(m))$ and its relativization to a coding function v.

Definition (`secondRecSubst`). $\mathrm{secondRecSubst}(m) = \mathrm{applyNat}(m, \mathrm{num}(m))$.

Definition (`recNat`). $\mathrm{recNat}(v,n) = \mathrm{applyNat}(v(n), \mathrm{num}(v(n)))$. Theorem 3.3 uses a primitive recursive v for which $\mathbf{val}(\mathrm{recNat}(v,n))$ realizes the fixed-point equation at code n.

`recNat.lean`

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat
public import DomainTheory.DataTypesAsLattices.Enumeration.num

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

def secondRecSubst (m : ℕ) : ℕ := applyNat m (num m)

def recNat (v : ℕ → ℕ) (n : ℕ) : ℕ :=
  applyNat (v n) (num (v n))

end DomainTheory.DataTypesAsLattices
```

3.4.10 Plotkin helpers ✓

Finite Plotkin steps: normalized application of e_p, tagging via σ_j, and iterated predecessors on numerals.

Theorem (`PlotkinHelpers.normalized_finite_app`). Applying the graph of $y \mapsto e_p(y)$ recovers e_p itself: $\mathrm{graph}(y \mapsto e_p(y))\, u = e_p(u)$.

Proof. The β-law for continuous application of e_p. ■

Theorem (`PlotkinHelpers.normalized_finite_fresh_app`).

$$\mathrm{graph}(y \mapsto e_p(y))\big(Z(m, e_p, e_q) \cup \{(\max(p,q)+1,\ m)\}\big) = Z(m+1, e_p, e_q).$$

Proof. Reduce by the normalized β-law, then discard the fresh tag by the union-fresh application of e_p. ■

Theorem (`PlotkinHelpers.finite_sigma_plain_app`).

$$\big(\mathrm{graph}(y \mapsto e_p(y)) \cup \mathrm{graph}(\sigma_{\max(p,q)+1})\big)\, Z(m, e_p, e_q) = Z(m+1, e_p, e_q).$$

Proof. Split the union application; the normalized part iterates once, while σ_j on $Z(m, e_p, e_q)$ is empty by the finite no-tag bound. ■

Theorem (`PlotkinHelpers.finite_sigma_union_app`).

$$\begin{aligned}&\big(\mathrm{graph}(y \mapsto e_p(y)) \cup \mathrm{graph}(\sigma_{\max(p,q)+1})\big)\big(Z(m, e_p, e_q) \cup \{(\max(p,q)+1,\ m)\}\big)\\ &\quad = Z(m+1, e_p, e_q) \cup \{(\max(p,q)+1,\ m+1)\}.\end{aligned}$$

Proof. Split the union: the normalized fresh step yields the next iterate, and σ_j on the fresh singleton advances the tag. ■

Theorem (`PlotkinHelpers.predSet_iterate_expTag_isScottContinuous`). For each k, the map $e \mapsto \mathrm{predSet}^k(e(0))$ is Scott-continuous.

Proof. Induction on k: the base is left-continuity of application at 0; the step composes continuous predSet with the inductive map. ■

Theorem (`PlotkinHelpers.predSet_iterate_ofNat`). $\mathrm{predSet}^k(\overline{k+i}) = \overline{i}$.

Proof. Induction on k: the base is immediate; the step uses the successor law for predSet on numerals. ■

Theorem (`PlotkinHelpers.predSet_iterate_self`). $\mathrm{predSet}^k(\overline{k}) = \overline{0}$.

Proof. The previous identity at $i = 0$. ■

`PlotkinHelpers.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.SumMed
public import DomainTheory.DataTypesAsLattices.Enumeration.Zcomb

import DomainTheory.DataTypesAsLattices.Lambda.eq_2_9
import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
import DomainTheory.DataTypesAsLattices.Graph.funOf_more
import DomainTheory.DataTypesAsLattices.Graph.beta
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace PlotkinHelpers

theorem normalized_finite_app (p : ℕ) (u : Pomega) :
    funOf (graph (fun y => funOf (e p) y)) u = funOf (e p) u :=
  beta (funOf.isScottContinuous (e p)) u

theorem normalized_finite_fresh_app (p q m : ℕ) :
    funOf (graph (fun y => funOf (e p) y))
        (Z m (e p) (e q) ∪ {pair (max p q + 1) m}) =
      Z (m + 1) (e p) (e q) := by
  rw [normalized_finite_app,
    funOf.e_union_fresh p (max p q + 1) m _
      (Nat.lt_succ_of_le (Nat.le_max_left p q))]
  rfl

theorem finite_sigma_plain_app (p q m : ℕ) :
    funOf
        (graph (fun y => funOf (e p) y) ∪
          graph (sigmaJ (max p q + 1)))
        (Z m (e p) (e q)) =
      Z (m + 1) (e p) (e q) := by
  rw [eq_2_9, normalized_finite_app, sigmaJ.app,
    sigmaJ.of_no_tag (fun k => Z.finite_no_tag p q m k)]
  simp [botElem, Z]

theorem finite_sigma_union_app (p q m : ℕ) :
    funOf
        (graph (fun y => funOf (e p) y) ∪
          graph (sigmaJ (max p q + 1)))
        (Z m (e p) (e q) ∪ {pair (max p q + 1) m}) =
```

```
      Z (m + 1) (e p) (e q) ∪ {pair (max p q + 1) (m + 1)} := by
  rw [eq_2_9, normalized_finite_fresh_app, sigmaJ.app,
    sigmaJ.union_fresh (fun k => Z.finite_no_tag p q m k)]

theorem predSet_iterate_expTag_isScottContinuous (k : ℕ) :
    IsScottContinuous (fun e => predSet^[k] (funOf e (ofNat 0))) := by
  induction k with
  | zero => exact funOf.isScottContinuous_left (ofNat 0)
  | succ k ih =>
    have h := theorem_1_3 (f := predSet)
      (g := fun e => predSet^[k] (funOf e (ofNat 0)))
      predSet.isScottContinuous ih
    convert h using 1
    funext e
    exact Function.iterate_succ_apply' predSet k (funOf e (ofNat 0))

theorem predSet_iterate_ofNat :
    ∀ k i, predSet^[k] (ofNat (k + i)) = ofNat i
  | 0, i => by
    simp [Function.iterate_zero]
  | k + 1, i => by
    rw [Function.iterate_succ_apply]
    have : k + 1 + i = k + i + 1 := by omega
    rw [this, predSet.ofNat_succ]
    exact predSet_iterate_ofNat k i

theorem predSet_iterate_self (k : ℕ) :
    predSet^[k] (ofNat k) = ofNat 0 := by
  simpa using predSet_iterate_ofNat k 0

end PlotkinHelpers

end DomainTheory.DataTypesAsLattices
```

3.4.11 Theorem 3.3 ✓

THEOREM 3.3 (The second recursion theorem). Take a LAMBDA-definable element v such that:

(i) $\mathbf{val}(v(n)) = \lambda m \in \omega.\, \mathbf{val}(n)(\mathbf{apply}(m)(\mathbf{num}(m)))$,

and then define a combinator by:

(ii) $\mathbf{rec} = \lambda n \in \omega.\, \mathbf{apply}(v(n))(\mathbf{num}(v(n)))$.

Then we have a primitive recursive function with this fixed-point property:

(iii) $\mathbf{val}(\mathbf{rec}(n)) = \mathbf{val}(n)(\mathbf{rec}(n))$.

Note that if u is LAMBDA-definable, then we find first an n such that $\mathbf{val}(n) = u$. Next we calculate $k = \mathbf{rec}(n)$. This effectively gives us an integer such that $\mathbf{val}(k) = u(k)$. Gödel numbers represent expressions (combinations in $\mathbf{G}$), and $\mathbf{val}$ maps the numbers to the values denoted by the expressions in the model. The k just found thus represents an expression whose value is defined in terms of its own Gödel number. In recursion theory there are many applications of this result.

Formalization. Write $S(u)$ for the continuous function $\lambda x.\, \bigcup_{m \in x} u(\mathbf{apply}(m)(\mathbf{num}(m)))$, so Scott's right-hand side in (i) is $S(\mathbf{val}(n))$. A concrete total primitive recursive v is obtained by applying a fixed code for the operator $u \mapsto S(u)$ to n. The recursion code is $\mathbf{rec}(n) = \mathbf{apply}(v(n))(\mathbf{num}(v(n)))$ as in

(ii). A parallel self-application uses $\mathbf{Y}$: the code $\mathbf{apply}(\mathbf{Ycode}, n)$ satisfies $\mathbf{val}(\mathbf{apply}(\mathbf{Ycode}, n)) = \mathbf{val}(n)(\mathbf{val}(\mathbf{apply}(\mathbf{Ycode}, n)))$.

Theorem (`theorem_3_3.from_v`). Suppose $\mathbf{val}(v(n)) = S(\mathbf{val}(n))$. Setting $\mathbf{rec}(n) = \mathbf{apply}(v(n))(\mathbf{num}(v(n)))$, one has

$$\mathbf{val}(\mathbf{rec}(n)) = \mathbf{val}(n)(\mathbf{rec}(n)).$$

Proof. By the enumeration theorem, $\mathbf{val}(\mathbf{apply}(v(n))(\mathbf{num}(v(n)))) = \mathbf{val}(v(n))(\mathbf{val}(\mathbf{num}(v(n))))$. The hypothesis replaces the operator by $S(\mathbf{val}(n))$, and (3.12) gives $\mathbf{val}(\mathbf{num}(v(n))) = v(n)$. Unfolding S yields a union of applications of $\mathbf{val}(n)$ indexed by members of the numeral $v(n)$; that numeral is the singleton $\{v(n)\}$ in the coding, so the union reduces to the single term $\mathbf{val}(n)(\mathbf{apply}(v(n))(\mathbf{num}(v(n)))) = \mathbf{val}(n)(\mathbf{rec}(n))$. ■

Theorem (`theorem_3_3.i`). For the concrete v above, $\mathbf{val}(v(n)) = S(\mathbf{val}(n))$ for every n.

Proof. By construction $v(n)$ is the application of the code of the operator $u \mapsto S(u)$ to n. The enumeration theorem and the defining equation of that operator give $\mathbf{val}(v(n)) = S(\mathbf{val}(n))$. ■

Theorem (`theorem_3_3.ii`). For every total v and every n,

$$\mathbf{rec}(n) = \mathbf{apply}(v(n))(\mathbf{num}(v(n))).$$

Proof. This is the definition of $\mathbf{rec}$ in (ii). ■

Theorem (`theorem_3_3.val`). Writing $r(n) = \mathbf{apply}(\mathbf{Ycode}, n)$, one has

$$\mathbf{val}(r(n)) = \mathbf{val}(n)(\mathbf{val}(r(n))).$$

Proof. By the enumeration theorem, $\mathbf{val}(r(n)) = \mathbf{val}(\mathbf{Ycode})(\mathbf{val}(n))$. The code $\mathbf{Ycode}$ evaluates to the combinator $\mathbf{Y}$, and the fixed-point law for $\mathbf{Y}$ supplies $\mathbf{Y}(\mathbf{val}(n)) = \mathbf{val}(n)(\mathbf{Y}(\mathbf{val}(n)))$. ■

Theorem (`theorem_3_3`). There is a total primitive recursive v such that $\mathbf{rec}$ defined by (ii) is primitive recursive, (i) holds for this v, and (iii) holds for the corresponding $\mathbf{rec}$.

Proof. Take the concrete v of the preceding construction (application of the code of $u \mapsto S(u)$). It is primitive recursive, as is $\mathbf{rec}$ formed from it by pairing with $\mathbf{num}$. Clause (i) is the theorem just proved for this v; clause (ii) is the definition of $\mathbf{rec}$; clause (iii) follows from the general fixed-point lemma once (i) is known. ■

`theorem_3_3.lean`

```
module

public import Mathlib.Computability.Primrec.Basic
public import DomainTheory.DataTypesAsLattices.Enumeration.secondRecV
public import DomainTheory.DataTypesAsLattices.Enumeration.recCode

import DomainTheory.DataTypesAsLattices.Enumeration.valNat_codes
import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_12
import DomainTheory.DataTypesAsLattices.Enumeration.primrec_recNat
import DomainTheory.DataTypesAsLattices.Enumeration.primrec_secondRecV

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
```

```
namespace theorem_3_3

theorem from_v (v : ℕ → ℕ) (n : ℕ)
    (hv : valNat (v n) = secondRecVal (valNat n)) :
    valNat (recNat v n) = funOf (valNat n) (ofNat (recNat v n)) := by
  rw [recNat, valNat.app, hv, eq_3_12, secondRecVal.app]
  ext k
  constructor
  · intro hk
    obtain ⟨m, hm, hkm⟩ := Set.mem_iUnion₂.mp hk
    simp [ofNat] at hm; subst hm; exact hkm
  · intro hk
    exact Set.mem_iUnion₂.mpr ⟨v n, rfl, hk⟩

theorem i (n : ℕ) :
    valNat (secondRecV n) = secondRecVal (valNat n) := by
  rw [secondRecV, valNat.app, valNat.secondRecOperatorCode_eq,
    secondRecOperator.app]

theorem ii (v : ℕ → ℕ) (n : ℕ) :
    recNat v n = applyNat (v n) (num (v n)) := rfl

theorem val (n : ℕ) :
    valNat (recCode n) = funOf (valNat n) (valNat (recCode n)) := by
  rw [recCode, valNat.app, valNat.Ycode_eq]
  exact Ycomb.unfold (valNat n)

end theorem_3_3

theorem theorem_3_3 :
    ∃ v : ℕ → ℕ, Primrec v ∧
      Primrec (recNat v) ∧
      (∀ n, valNat (v n) = secondRecVal (valNat n)) ∧
      (∀ n, recNat v n = applyNat (v n) (num (v n))) ∧
      ∀ n, valNat (recNat v n) =
        funOf (valNat n) (ofNat (recNat v n)) :=
  ⟨secondRecV, primrec_secondRecV, primrec_recNat secondRecV primrec_secondRecV,
    theorem_3_3.i, theorem_3_3.ii secondRecV,
    fun n => theorem_3_3.from_v secondRecV n (theorem_3_3.i n)⟩

end DomainTheory.DataTypesAsLattices
```

3.4.12 Theorem 3.4 ✓

THEOREM 3.4 (The incompleteness theorem). The set of integers n such that $\mathbf{val}(n) = \perp$ is not r.e.; hence, there can be no effectively given formal system for enumerating all true equations between LAMBDA-terms.

(A critic may sense here an application of Church's thesis in stating the metatheoretic consequence of the nonresult.) A few details of the proof can be given to see how the notation works. First let v be a (total) primitive recursive function such that:

$$\mathbf{val}(v(n)) = n \cap \mathbf{val}(n),$$

and note that:

$$n \cap \mathbf{val}(n) = \bot \quad \text{iff} \quad n \notin \mathbf{val}(n).$$

Call the set in question in Theorem 3.4 the set b. If it were r.e., then so would be:

$$\{n \in \omega \mid v(n) \in b\} = (\lambda n \in \omega.\, v(n) \cap b \supset n, n)(\top).$$

That would mean having an integer k such that:

$$\mathbf{val}(k) = \{n \in \omega \mid v(n) \in b\}.$$

But then:

$$\begin{aligned} k \in \mathbf{val}(k) &\iff v(k) \in b \\ &\iff \mathbf{val}(v(k)) = \bot \\ &\iff k \notin \mathbf{val}(k), \end{aligned}$$

which gives us a contradiction. This is the usual diagonal argument.

theorem_3_4.lean

```
module

public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Enumeration.valNat

import DomainTheory.DataTypesAsLattices.Enumeration.valNat_codes
import DomainTheory.DataTypesAsLattices.Computability.IsREu2ecombinatory
import DomainTheory.DataTypesAsLattices.Enumeration.primrec_diagonalInterCode

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_3_4.diagonal {val : ℕ → Pomega} {v : ℕ → ℕ} {b : Set ℕ}
    (hv : ∀ k, val (v k) = ofNat k ∩ val k)
    (hb : b = {j | val j = botElem})
    (hex : ∃ k, val k = {i | v i ∈ b}) : False := by
  obtain ⟨k, hk⟩ := hex
  have hiff : k ∈ val k ↔ v k ∈ b := by
    constructor
    · intro h
      have : k ∈ {i | v i ∈ b} := by rwa [hk] at h
      exact this
    · intro h
      have : k ∈ {i | v i ∈ b} := h
      rwa [← hk] at this
  have hinter : val (v k) = botElem ↔ k ∈ val k := by
    rw [hv]
    constructor
    · intro h hk'
      have this : k ∈ ofNat k ∩ val k := ⟨rfl, hk'⟩
      rw [h] at this
      exact this
```

```
    · intro h
      ext i
      constructor
      · intro ⟨hi, hval⟩
        simp [ofNat] at hi
        subst hi
        exact (h hval).elim
      · intro hi
        exact hi.elim
  have hbmem : v k ∈ b ↔ val (v k) = botElem := by
    rw [hb]; rfl
  have : k ∈ val k ↔ k ∈ val k :=
    hiff.trans (hbmem.trans hinter)
  by_cases hkmem : k ∈ val k
  · exact (this.mp hkmem) hkmem
  · exact hkmem (this.mpr hkmem)

theorem theorem_3_4.re_diagonal {val : ℕ → Pomega} {v : ℕ → ℕ}
    (hv : ∀ k, val (v k) = ofNat k ∩ val k)
    (_hRE : IsRE {j | val j = botElem})
    (hex : ∃ k, val k = {i | v i ∈ {j | val j = botElem}}) : False :=
  theorem_3_4.diagonal hv rfl hex

theorem theorem_3_4 :
    ¬ IsRE {n | valNat n = botElem} := by
  intro hb
  have hpre : IsRE {i | diagonalInterCode i ∈ {n | valNat n = botElem}} :=
    ⟨hb.out.comp primrec_diagonalInterCode.to_comp⟩
  obtain ⟨k, hk⟩ :=
    theorem_3_2.surjective (IsRE.combinatory hpre)
  exact theorem_3_4.diagonal (val := valNat) (v := diagonalInterCode)
    valNat.diagonalInterCode_eq rfl ⟨k, hk⟩

end DomainTheory.DataTypesAsLattices
```

3.4.13 Theorem 3.5 ✓

The relationship $\mathbf{val}(n) = \mathbf{val}(m)$ means that the expressions with Gödel numbers n and m have the same value in the model. (This is not only not r.e. but is a complete Π^0_2-predicate.) A *total* mapping can be regarded as a *syntactical* transformation on expressions defined via Gödel numbers. Such a mapping p is called *extensional* if it has the property:

$$\mathbf{val}(p(n)) = \mathbf{val}(p(m)) \quad \text{whenever} \quad \mathbf{val}(n) = \mathbf{val}(m).$$

The Myhill–Shepherdson theorem shows that extensional, syntactical mappings really depend on the *values* of the expressions. Precisely we have:

THEOREM 3.5 (The completeness theorem for definability). *If a (total) extensional mapping p is LAMBDA-definable, then there is a LAMBDA-definable q such that $\mathbf{val}(p(n)) = q(\mathbf{val}(n))$ for all $n \in \omega$.*

Of course q is uniquely determined (because the values of q are given at least on the finite sets). Thus any attempt to define something *new* by means of some strange mapping on Gödel numbers is bound to fail as long as it is *effective* and *extensional*. The main part of the argument is concentrated on showing these mappings to be *continuous*; that is why q exists.

Proof of Theorem 3.5. The argument is essentially the original one of Myhill–Shepherdson. Suppose p is computable, total and extensional. Define:

$$q = \{(j, m) \mid m \in \mathbf{val}(p(\mathbf{fin}(j)))\},$$

where $\mathbf{fin}$ is primitive recursive, and for all $j \in \omega$:

$$\mathbf{val}(\mathbf{fin}(j)) = e_j.$$

Certainly $q \in \mathbf{RE}$, and we will establish the theorem if we can prove "continuity":

$$\mathbf{val}(p(n)) = \bigcup\{\mathbf{val}(p(\mathbf{fin}(j))) \mid e_j \subseteq \mathbf{val}(n)\}.$$

We proceed by contradiction. Suppose first we have a $k \in \mathbf{val}(p(n))$, where $k \notin \mathbf{val}(p(\mathbf{fin}(j)))$ whenever $e_j \subseteq \mathbf{val}(n)$. Pick r to be a primitive recursive function whose range is *not* recursive. Define s, primitive recursive, so that for all $m \in \omega$:

$$\mathbf{val}(s(m)) = \{j \in \mathbf{val}(n) \mid m \notin \{r(i) \mid i \leq j\}\}.$$

The set $\mathbf{val}(n)$ must be infinite, because p is extensional, and if $\mathbf{val}(n) = e_j = \mathbf{val}(\mathbf{fin}(j))$, then $k \notin \mathbf{val}(p(\mathbf{fin}(j))) = \mathbf{val}(p(n))$. Note that $\mathbf{val}(s(m))$, as a subset of the infinite set, is *finite* if m is in the range of r; otherwise it is equal to $\mathbf{val}(n)$. Again by the extensionality of p we see that $k \in \mathbf{val}(p(s(m)))$ if and only if m is *not* in the range of r. But this puts an r.e. condition on m equivalent to a non-r.e. condition, which shows there is no such k.

For the second case suppose we have a $k \notin \mathbf{val}(p(n))$, where for a suitable $e_j \subseteq \mathbf{val}(n)$ it is the case that $k \in \mathbf{val}(p(\mathbf{fin}(j)))$. Define:

$$t = \lambda m \in \omega.\, e_j \cup (\mathbf{val}(m) \supset \mathbf{val}(n), \mathbf{val}(n)).$$

We have:

$$t(m) = \begin{cases} e_j & \text{if } \mathbf{val}(m) = \bot; \\ \mathbf{val}(n) & \text{if not.} \end{cases}$$

We choose u primitive recursive, where:

$$\mathbf{val}(u(m)) = t(m).$$

By the choice of k, and by the extensionality of p, and by the fact that $\mathbf{val}(n) \neq e_j$, we have:

$$k \in \mathbf{val}(p(u(m))) \quad \text{iff} \quad t(m) = e_j$$
$$\text{iff} \quad \mathbf{val}(m) = \bot.$$

But this is impossible, since one side is r.e. in m and the other is not by Theorem 3.4. As both cases lead to contradiction, continuity is established and the proof is complete.

Formalization. Write $\mathbf{fin}(j)$ for a code of the finite set e_j, and write

$$Q = \{(j, m) \mid m \in \mathbf{val}(p(\mathbf{fin}(j)))\}$$

for Scott's q. Continuity is the equality $\mathbf{val}(p(n)) = Q(\mathbf{val}(n))$, proved by ruling out the two failure modes below. The enumerator's range of a fixed non-recursive primitive recursive r plays the role of Scott's $\{r(i)\}$.

Theorem (`theorem_3_5.case1`). Let p be realized by a combinatory element and be extensional for $\mathbf{val}$. Suppose $k \in \mathbf{val}(p(n))$ yet $k \notin \mathbf{val}(p(\mathbf{fin}(j)))$ whenever $e_j \subseteq \mathbf{val}(n)$, and suppose $\mathbf{val}(n)$ is infinite. Then contradiction.

Proof. Form the primitive recursive s of Scott's first case (filter $\mathbf{val}(n)$ against the initial segment of the range of r up to stage j). Extensionality yields $k \in \mathbf{val}(p(s(m)))$ if and only if m lies outside the range of r. The left-hand side is an r.e. predicate of m (membership of a fixed k after composing the realizer of p with s), while the complement of the range of r is not r.e. ■

Theorem (`theorem_3_5.case2`). Under the same hypotheses on p, suppose $e_j \subseteq \mathbf{val}(n)$, $e_j \neq \mathbf{val}(n)$, $k \in \mathbf{val}(p(\mathbf{fin}(j)))$, and $k \notin \mathbf{val}(p(n))$. Then contradiction.

Proof. Form Scott's $t(m)$ (equal to e_j when $\mathbf{val}(m) = \bot$, and to $\mathbf{val}(n)$ otherwise) and a primitive recursive code u with $\mathbf{val}(u(m)) = t(m)$. Extensionality gives $k \in \mathbf{val}(p(u(m)))$ if and only if $\mathbf{val}(m) = \bot$. The left side is r.e. in m; the right side is not, by Theorem 3.4. ■

Theorem (`theorem_3_5.q`). $(j, m) \in Q$ if and only if $m \in \mathbf{val}(p(\mathbf{fin}(j)))$.

Proof. Immediate from the definition of Q as a set of coded pairs. ■

Theorem (`theorem_3_5.q_app`). $\mathbf{val}(p(\mathbf{fin}(j))) \subseteq Q(e_j)$.

Proof. Every $m \in \mathbf{val}(p(\mathbf{fin}(j)))$ contributes the pair (j, m) to Q, and $j \in e_j$. ■

Theorem (`theorem_3_5.unique`). If q and q' are Scott-continuous and agree on every finite e_n, then $q = q'$ as maps on $\mathbf{P}\omega$.

Proof. Every x is the directed union of the finite $e_n \subseteq x$; continuity and agreement on those pieces give $q(x) = q'(x)$. ■

Theorem (`theorem_3_5.of_realizer`). If $\mathbf{val}(p(n)) = q(\mathbf{val}(n))$ holds for all n, then that same family of equations holds for all n.

Proof. Immediate. ■

Theorem (`theorem_3_5`). If a total p is LAMBDA-definable and extensional for $\mathbf{val}$, then there is a LAMBDA-definable q with $\mathbf{val}(p(n)) = q(\mathbf{val}(n))$ for all n.

Proof. Take $q = Q$ as above; it is r.e., hence combinatory. For membership of k in $\mathbf{val}(p(n))$ versus $Q(\mathbf{val}(n))$, the two directions reduce to the continuity equation. Failure of either inclusion produces one of the two contradictory configurations already ruled out. ■

`theorem_3_5.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Computability.Realizes
public import DomainTheory.DataTypesAsLattices.Enumeration.myhillQ
public import DomainTheory.DataTypesAsLattices.Enumeration.valRank

import DomainTheory.DataTypesAsLattices.Enumeration.theorem_3_4
import DomainTheory.DataTypesAsLattices.Enumeration.primrec_myhillCase2Code
import DomainTheory.DataTypesAsLattices.Enumeration.primrec_myhillCase1Code
import DomainTheory.DataTypesAsLattices.Enumeration.haltEnum_more
import DomainTheory.DataTypesAsLattices.Enumeration.myhillQ_more
import DomainTheory.DataTypesAsLattices.Enumeration.valNat_codes
import DomainTheory.DataTypesAsLattices.Pomega.codeFinset
import DomainTheory.DataTypesAsLattices.Computability.IsREu2ecombinatory
import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_bot
import Mathlib.Order.Interval.Finset.Nat
```

```
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

namespace theorem_3_5

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem case1 {u : Pomega} {p : ℕ → ℕ} {n k : ℕ}
    (hu : IsCombinatory u) (hr : Realizes u p)
    (hext : Extensional valNat p)
    (hk : k ∈ valNat (p n))
    (hnin : ¬∃ j, e j ⊆ valNat n ∧ k ∈ valNat (p (fin j)))
    (_hinf : ¬(valNat n).Finite) : False := by
  have hiff : ∀ m,
      k ∈ valNat (p (myhillCase1Code n m)) ↔ ¬∃ t, haltEnum t = m := by
    intro m
    by_cases hex : ∃ t, haltEnum t = m
    · have hfin : ({j | ∀ i ≤ j, haltEnum i = m} : Pomega).Finite :=
        (Set.finite_Iio hex.choose).subset fun j hj =>
          Nat.lt_of_not_ge fun hle => hj hex.choose hle hex.choose_spec
      have hfins : (valNat (myhillCase1Code n m)).Finite := by
        rw [valNat.myhillCase1Code_eq]
        exact hfin.inter_of_right _
      obtain ⟨j, hj⟩ := e.exists_code_of_finite hfins
      have hje : e j ⊆ valNat n := by
        rw [hj, valNat.myhillCase1Code_eq]
        exact Set.inter_subset_left
      have : k ∈ valNat (p (fin j)) := fun h => hnin ⟨j, hje, h⟩
      have heq := hext.out (myhillCase1Code n m) (fin j) (by simp [hj, valNat.fin_eq])
      simp [heq, this, hex]
    · have hall : {j | ∀ i ≤ j, haltEnum i = m} = Set.univ := by
        ext j
        constructor
        · intro _; trivial
        · intro _ i _
          exact fun h => hex ⟨i, h⟩
      have : valNat (myhillCase1Code n m) = valNat n := by
        rw [valNat.myhillCase1Code_eq, hall]
        simp
      have heq := hext.out (myhillCase1Code n m) n this
      simp [heq, hk, hex]
  have hre : REPred fun m => ¬∃ t, haltEnum t = m :=
    _root_.REPred.of_eq
      (valNat.mem_comp_primrec (k := k) hu hr (primrec_myhillCase1Code n))
      fun m => hiff m
  exact haltEnum.range_not_computable
```

```
    ((ComputablePred.computable_iff_re_compl_re').2 ⟨haltEnum.range_re, hre⟩)

theorem case2 {u : Pomega} {p : ℕ → ℕ} {n j k : ℕ}
    (hu : IsCombinatory u) (hr : Realizes u p)
    (hext : Extensional valNat p)
    (hj : e j ⊆ valNat n)
    (_hne : e j = valNat n)
    (hkfin : k ∈ valNat (p (fin j)))
    (hkn : k ∈ valNat (p n)) : False := by
  have hiff : ∀ m, k ∈ valNat (p (myhillCase2Code j n m)) ↔ valNat m = botElem := by
    intro m
    by_cases hm : valNat m = botElem
    · have : valNat (myhillCase2Code j n m) = e j := by
        rw [valNat.myhillCase2Code_eq, hm, eq_2_7_bot]
        simp [botElem]
      have := hext.out (myhillCase2Code j n m) (fin j) (by simp [this, valNat.fin_eq])
      simp [this, hkfin, hm]
    · have : valNat (myhillCase2Code j n m) = valNat n :=
        valNat.myhillCase2Code_ne_bot j n m hj hm
      have := hext.out (myhillCase2Code j n m) n this
      simp [this, hkn, hm]
  have hre : REPred fun m => valNat m = botElem :=
    _root_.REPred.of_eq
      (valNat.mem_comp_primrec (k := k) hu hr (primrec_myhillCase2Code j n))
      fun m => hiff m
  exact theorem_3_4 ⟨hre⟩

theorem of_realizer {val : ℕ → Pomega} {p : ℕ → ℕ} {q : Pomega}
    (hq : ∀ n, val (p n) = funOf q (val n)) :
    ∀ n, val (p n) = funOf q (val n) := hq

theorem q (val : ℕ → Pomega) (p : ℕ → ℕ) (j m : ℕ) :
    pair j m ∈ myhillQ val p ↔ m ∈ val (p (fin j)) := by
  constructor
  · intro ⟨j', m', hp, hm⟩
    obtain ⟨rfl, rfl⟩ := pair_inj hp
    exact hm
  · intro hm
    exact ⟨j, m, rfl, hm⟩

theorem q_app (p : ℕ → ℕ) (j : ℕ) :
    valNat (p (fin j)) ⊆ funOf (myhillQ valNat p) (e j) := by
  intro m hm
  exact ⟨j, subset_rfl, ⟨j, m, rfl, by simpa [valNat.fin_eq] using hm⟩⟩

theorem unique {q q' : Pomega → Pomega}
    (hq : IsScottContinuous q) (hq' : IsScottContinuous q')
    (h : ∀ n, q (e n) = q' (e n)) (x : Pomega) : q x = q' x := by
  rw [hq.out, hq'.out]
  ext k
  simp [scottUnion, scottPiece, h]

end theorem_3_5

theorem theorem_3_5 {p : ℕ → ℕ}
    (hp : ∃ u, IsCombinatory u ∧ Realizes u p)
```

```
    (hext : Extensional valNat p) :
    ∃ q, IsCombinatory q ∧ ∀ n, valNat (p n) = funOf q (valNat n) := by
  obtain ⟨u, hu, hr⟩ := hp
  refine ⟨myhillQ valNat p, IsRE.combinatory (myhillQ.isRE hu hr), fun n => ?_⟩
  rw [myhillQ.app]
  ext k
  constructor
  · intro hk
    by_contra hnin
    have hinf : ∀ j, e j = valNat n → False := by
      intro j hj
      have : k ∈ valNat (p (fin j)) := by
        have := hext.out n (fin j) (by simp [valNat.fin_eq, hj])
        simpa [this, valNat.fin_eq] using hk
      exact hnin (Set.mem_iUnion.mpr ⟨j,
        Set.mem_iUnion.mpr ⟨by simp [hj], this⟩⟩)

    have hfin : ¬(valNat n).Finite := by
      intro hf
      obtain ⟨j, hj⟩ := e.exists_code_of_finite hf
      exact hinf j hj
    have hnin' : ¬∃ j, e j ⊆ valNat n ∧ k ∈ valNat (p (fin j)) := by
      intro ⟨j, hj, hkj⟩
      exact hnin (Set.mem_iUnion.mpr ⟨j, Set.mem_iUnion.mpr ⟨hj, hkj⟩⟩)
    exact theorem_3_5.case1 hu hr hext hk hnin' hfin
  · intro hk
    obtain ⟨j, hk⟩ := Set.mem_iUnion.mp hk
    obtain ⟨hj, hk⟩ := Set.mem_iUnion.mp hk
    by_contra hnin
    have hne : e j = valNat n := by
      intro hj'
      have := hext.out n (fin j) (by simp [valNat.fin_eq, hj'])
      exact hnin (by simpa [this, valNat.fin_eq] using hk)
    exact theorem_3_5.case2 hu hr hext hj hne hk hnin

end DomainTheory.DataTypesAsLattices
```

3.4.14 Theorem 3.6 ✓

The preceding results indicate that the expected results on r.e. sets are forthcoming in a smooth and unified manner in this setting. Some knowledge of r.e. theory was presupposed, but analysis shows that the knowledge required is slight. The notion of primitive recursive functions should certainly be well understood together with standard examples. Partial functions need not be introduced separately since they are naturally incorporated into LAMBDA (the theory of multiple-valued functions). As a working definition of r.e. one can take either "empty or the range of a primitive recursive function" or, more uniformly, "a set of the form $\{m \mid \exists n.\, m + 1 = p(n)\}$ where p is primitive recursive". A few obvious closure properties of r.e. sets should then be proved, and then an adequate foundation for the discussion of LAMBDA will have been provided.

The point of introducing LAMBDA is that further closure properties are more easily expressed in a theory where equations can be variously interpreted as involving numbers, functionals, etc., without becoming too heavily involved in intricate Gödel numbering and encodings. Another useful feature of the present theory concerns the ease with which we can introduce *relative recursiveness*.

As we have seen, $\{\mathbf{val}(n) \mid n \in \omega\}$ is an enumeration of all r.e. sets. Suppose we add a new set a as a new constant. What are the sets enumerable in a? Answer: $\{\mathbf{val}(n)(a) \mid n \in \omega\}$, since in combinatory logic a parameter can always be factored out as an extra argument. Another way to put the point is this: for b to be enumeration reducible to a it is necessary and sufficient that $b = u(a)$ where $u \in \mathbf{RE}$. This is *word for word* the definition given by [212, pp. 146–147]. What we have done is to put the theory of enumeration operators (Friedberg–Rogers and Myhill–Shepherdson) into a general setting in which the language LAMBDA not only provides definitions but also the basis of a calculus for demonstrating properties of the operators defined. The algebraic style of this language throws a little light on the notion of enumeration degree. In the first place we can identify the degree of an element with the set of all objects reducible to it (rather than just those equivalent to it) and write

$$\mathbf{Deg}(a) = \{u(a) \mid u \in \mathbf{RE}\}.$$

The set-theoretical inclusion is then the same as the partial ordering of degrees. What kind of a partially ordered set do we have?

THEOREM 3.6 (The subalgebra theorem). *The enumeration degrees are exactly the finitely generated combinatory subalgebras of* $\mathbf{P}\omega$.

By "subalgebras" here we of course mean subsets containing $\mathbf{G}$ and closed under application (hence, they contain all of $\mathbf{RE}$, the least subalgebra). Part of the assertion is that every finitely generated subalgebra has a single generator (under application). This fact is an easy extension of Theorem 3.1. Not very much seems to be known about enumeration degrees. Joins can obviously be formed using the pairing function $\langle x, y\rangle$ on sets. Each degree is a countable set; hence, it is trivial to obtain the existence of a sequence of degrees whose *infinite join* is not a degree (not finitely generated). The intersection of subalgebras is a subalgebra—but it may not be a degree even starting with degrees. There are no minimal degrees above $\mathbf{RE}$, but there are minimal *pairs* of degrees. Also for a given degree there are only countably many degrees minimal over it; but the question of whether the partial ordering of enumeration degrees is dense seems still to be open.

Theorem 3.6 shows that the semilattice of enumeration degrees is naturally extendable to a *complete lattice* (the lattice of all subalgebras of $\mathbf{P}\omega$), but whether there is anything interesting to say about this complete lattice from the point of view of structure is not at all clear. Rogers has shown [212, pp. 151–153] that Turing degrees can be defined in terms of enumeration degrees by restricting to special elements. In our style of notation we would define the space:

$$\mathbf{TOT} = \{u \mid u = \$(u) \text{ and } \forall n \in \omega.\, u(n) \in \omega\},$$

the space of all graphs of total functions. Then the system $\{\mathbf{Deg}(u) \mid u \in \mathbf{TOT}\}$ is isomorphic to the system of Turing degrees. Now there are many other interesting subsets of $\mathbf{P}\omega$. Whether the degree structure of these various subsets is worth investigation is a question whose answer awaits some new ideas.

Proof of Theorem 3.6. Consider a degree $\mathbf{Deg}(a)$. This set is closed under application, because:

$$u(a)(v(a)) = \mathbf{S}(u)(v)(a),$$

and $\mathbf{S}(u)(v)$ is r.e. if both u and v are. Note that it also contains the element $\mathbf{G}$; hence, as a subalgebra, it is generated by a and $\mathbf{G}$.

Let A be any finitely generated subalgebra with generators $a'_0, a'_1, \ldots, a'_{n-1}$. Consider the element $a = \mathbf{cond}(\langle a'_0, a'_1, \ldots, a'_{n-1}\rangle)(\mathbf{G})$. As in the proof of Theorem 3.1, a generates A under application. It is then easy to see why $\mathbf{A} = \mathbf{Deg}(a)$.

Formalization. Write $\mathbf{Deg}(a) = \{r(a) \mid r \in \mathbf{RE}\}$ as above. Pack a finite list of generators into a single sequence element $\langle a'_0, \ldots, a'_{n-1}\rangle$ by iterating the binary sequence constructor, and form Scott's single generator

$$a = \mathbf{cond}(\langle a'_0, \ldots, a'_{n-1}\rangle)(\mathbf{G}).$$

(Thus $a(0) = \langle a'_0, \ldots\rangle$ and a still lies in any subalgebra containing the a'_i and $\mathbf{G}$.) The three lemmas below are the pieces of Scott's argument recorded in this note's Lean module.

Theorem (`theorem_3_6.degree`). For every a, the set $\mathbf{Deg}(a)$ is a combinatory subalgebra and contains a.

Proof. Closure under application is the $\mathbf{S}$-identity $r(a) \cdot s(a) = \mathbf{S}(r)(s)(a)$ already quoted; $\mathbf{G}$ appears as the constant map $\mathbf{K}(\mathbf{G})$ applied to a. The element a itself is recovered as $\mathbf{I}(a)$ with $\mathbf{I}$ combinatory. ■

Theorem (`theorem_3_6.finite`). If $a = \mathbf{cond}(\langle a'_0, \ldots\rangle)(\mathbf{G})$ is the single generator of a finite list, then the packed sequence $\langle a'_0, \ldots\rangle$ and the generator $\mathbf{G}$ both lie in $\mathbf{Deg}(a)$.

Proof. The packed sequence equals $a(0)$, and 0 is r.e., so evaluation at 0 is an r.e. operation applied to a. The generator $\mathbf{G}$ belongs to every degree by the preceding theorem. ■

Theorem (`theorem_3_6.converse`). Every x belongs to $\mathbf{Deg}(\mathbf{cond}(\langle x\rangle)(\mathbf{G}))$.

Proof. By the finite-list lemma, the singleton packing $\langle x\rangle$ lies in that degree. Applying it to 0 (again an r.e. operation, and degrees are closed under application) recovers x. ■

`theorem_3_6.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.singleGenerator

import DomainTheory.DataTypesAsLattices.Enumeration.combinatory_mem_Deg
import DomainTheory.DataTypesAsLattices.Lambda.ofNat_more
import DomainTheory.DataTypesAsLattices.Enumeration.Icomb_SK
import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more
import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

namespace theorem_3_6

theorem finite (xs : List Pomega) :
    packList xs ∈ Deg (singleGenerator xs) ∧
      Gcomb ∈ Deg (singleGenerator xs) := by
  constructor
  · refine ⟨funOf (funOf Scomb Icomb_SK) (funOf Kcomb zeroC),
      .app (.app .S (.app (.app .S .K) .K)) (.app .K .zero), ?_⟩
    rw [Scomb.beta3, Icomb_SK.app, Kcomb.beta2, singleGenerator.app_zero]
  · exact (Deg.isSubalgebra (singleGenerator xs)).out.1

theorem converse (x : Pomega) :
```

```
    x ∈ Deg (singleGenerator [x]) := by
  have hp := (theorem_3_6.finite [x]).1
  have h0 := combinatory_mem_Deg (singleGenerator [x]) (ofNat.combinatory 0)
  have : funOf (packList [x]) (ofNat 0) ∈ Deg (singleGenerator [x]) :=
    (Deg.isSubalgebra _).out.2 hp h0
  simpa [packList.cons_zero] using this

theorem degree (a : Pomega) :
    Subalgebra (Deg a) ∧ a ∈ Deg a :=
  ⟨Deg.isSubalgebra a, Deg.self a⟩

end theorem_3_6

end DomainTheory.DataTypesAsLattices
```

3.4.15 Theorem 3.7 ✓

Among the subsets of $\mathbf{P}\omega$ with natural mathematical structure, we of course have $\mathbf{FUN}$, which is a semigroup under $\circ = \lambda u \lambda v \lambda x.\, u(v(x))$. It is, however, a rather complicated semigroup. We introduce for its study three new combinators:

$$\mathbf{R} = \lambda x.\, \langle 0, x\rangle; \tag{3.13}$$

$$\mathbf{L} = \lambda x.\, x_1(x_2); \tag{3.14}$$

$$\bar{u} = \lambda x.\, x_0 \supset \langle 1, u, x_1\rangle,\ \overline{u(x_1)(x_2)}. \tag{3.15}$$

THEOREM 3.7 (The semigroup theorem). The countable semigroup $\mathbf{RE} \cap \mathbf{FUN}$ of computable enumeration operators is finitely generated by $\mathbf{R}$, $\mathbf{L}$ and $\bar{\mathbf{G}}$.

The proof rests on the verification of two equations which permit an application of Theorem 3.1:

$$\mathbf{L} \circ \bar{u} \circ \mathbf{R} = \lambda x.\, u(x) \tag{3.16}$$

$$\bar{u} \circ \bar{v} \circ \mathbf{R} = \overline{u(v)}. \tag{3.17}$$

Certainly the word problem for $\mathbf{RE} \cap \mathbf{FUN}$ is unsolvable, indeed, not even recursively enumerable. Can the semigroup be generated by *two* generators by the way?

Proof of Theorem 3.7. We first establish (3.16) and (3.17):

$$\begin{aligned}\mathbf{L} \circ \bar{u} \circ \mathbf{R} &= \lambda x.\, \mathbf{L}(\bar{u}(\langle 0, x\rangle)) \\ &= \lambda x.\, \mathbf{L}(\langle 1, u, x\rangle) \\ &= \lambda x.\, u(x).\end{aligned}$$

$$\begin{aligned}\bar{u} \circ \bar{v} \circ \mathbf{R} &= \lambda x.\, \bar{u}(\bar{v}(\langle 0, x\rangle)) \\ &= \lambda x.\, \bar{u}(\langle 1, v, x\rangle)\end{aligned}$$

$$= \lambda x.\, \overline{u(v)}(x)$$
$$= \overline{u(v)}.$$

Now starting with any $u \in \mathbf{RE}$, we write $u = \tau$, where τ is formed from $\mathbf{G}$ by application alone. By (3.17), we can write $\bar{u}$ in terms of $\bar{\mathbf{G}}$ and $\mathbf{R}$ using only $\circ$. That is, $\bar{u}$ belongs to a special subsemigroup. In view of (3.16), we find that $\lambda x.\, u(x)$ belongs to that generated by $\mathbf{R}$, $\mathbf{L}$ and $\bar{\mathbf{G}}$. But

$$\mathbf{RE} \cap \mathbf{FUN} = \{\lambda x.\, u(x) \mid u \in \mathbf{RE}\},$$

and so the theorem is proved.

Formalization. Write $\circ$ for composition of enumeration operators. Say an element is *semigroup-generated* when it lies in the least class containing $\mathbf{R}$, $\mathbf{L}$ and $\bar{\mathbf{G}}$ and closed under $\circ$. Scott's claim is that this class coincides with $\mathbf{RE} \cap \mathbf{FUN}$.

Theorem (`theorem_3_7.equations`). For all u, v, x,

$$\mathbf{L}(\bar{u}(\mathbf{R}(x))) = u(x),$$

and writing $\langle 1, v, x\rangle$ for the positive branch of $\bar{v}$ at $\mathbf{R}(x)$,

$$\langle 1, v, x\rangle_1 = v, \qquad \langle 1, v, x\rangle_2 = x.$$

Proof. The first identity is (3.16) after evaluating $\bar{u}$ at $\mathbf{R}(x) = \langle 0, x\rangle$, which takes the positive branch $\langle 1, u, x\rangle$; $\mathbf{L}$ then applies the first component to the second. The two projection equations are the components of that same coded triple, as used in the calculation of (3.17). ■

Theorem (`theorem_3_7.generation`). If $u \in \mathbf{RE} \cap \mathbf{FUN}$, then u is semigroup-generated.

Proof. By Theorem 3.1, u is obtained from $\mathbf{G}$ by application alone, so $\bar{u}$ is obtained from $\bar{\mathbf{G}}$ and $\mathbf{R}$ by composition, using (3.17) inductively. Then $\mathbf{L} \circ \bar{u} \circ \mathbf{R}$ is semigroup-generated, and (3.16) identifies that composite with the graph $\lambda x.\, u(x)$. Since $u \in \mathbf{FUN}$, this graph is u itself. ■

Theorem (`theorem_3_7`). $u \in \mathbf{RE} \cap \mathbf{FUN}$ if and only if u is semigroup-generated.

Proof. One direction is the preceding generation lemma. Conversely, each of $\mathbf{R}$, $\mathbf{L}$ and $\bar{\mathbf{G}}$ is r.e. and functional, and $\mathbf{RE} \cap \mathbf{FUN}$ is closed under composition, so every semigroup-generated element lies in the intersection. ■

`theorem_3_7.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedSemigroup
public import DomainTheory.DataTypesAsLattices.Enumeration.RE

import DomainTheory.DataTypesAsLattices.Enumeration.bar_generated
import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_17
import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_16
import DomainTheory.DataTypesAsLattices.Enumeration.generatedSemigroup_mem_RE_FUN
import DomainTheory.DataTypesAsLattices.Enumeration.theorem_3_1

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
```

```
open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

namespace theorem_3_7

theorem equations (u v x : Pomega) :
    funOf Lcomb (funOf (funOf barComb u) (funOf Rcomb x)) = funOf u x ∧
      funOf (barPos v (funOf Rcomb x)) (ofNat 1) = v ∧
        funOf (barPos v (funOf Rcomb x)) (ofNat 2) = x :=
  ⟨eq_3_16.bar u x, (eq_3_17.projections u v x).2.1,
    (eq_3_17.projections u v x).2.2⟩

theorem generation {u : Pomega}
    (huRE : u ∈ RE) (huFUN : u ∈ FUN) : GeneratedSemigroup u := by
  have hbar : GeneratedSemigroup (funOf barComb u) :=
    bar_generated ((theorem_3_1 (u := u)).mp huRE)
  have hgen : GeneratedSemigroup
      (enumComp Lcomb (enumComp (funOf barComb u) Rcomb)) :=
    .comp .L (.comp hbar .R)
  rw [eq_3_16.operator] at hgen
  simpa [FUN] using huFUN ▶ hgen

end theorem_3_7

theorem theorem_3_7 {u : Pomega} :
    u ∈ RE ∩ FUN ↔ GeneratedSemigroup u :=
  ⟨fun h => theorem_3_7.generation h.1 h.2, generatedSemigroup_mem_RE_FUN⟩

end DomainTheory.DataTypesAsLattices
```

3.5 Retracts and data types

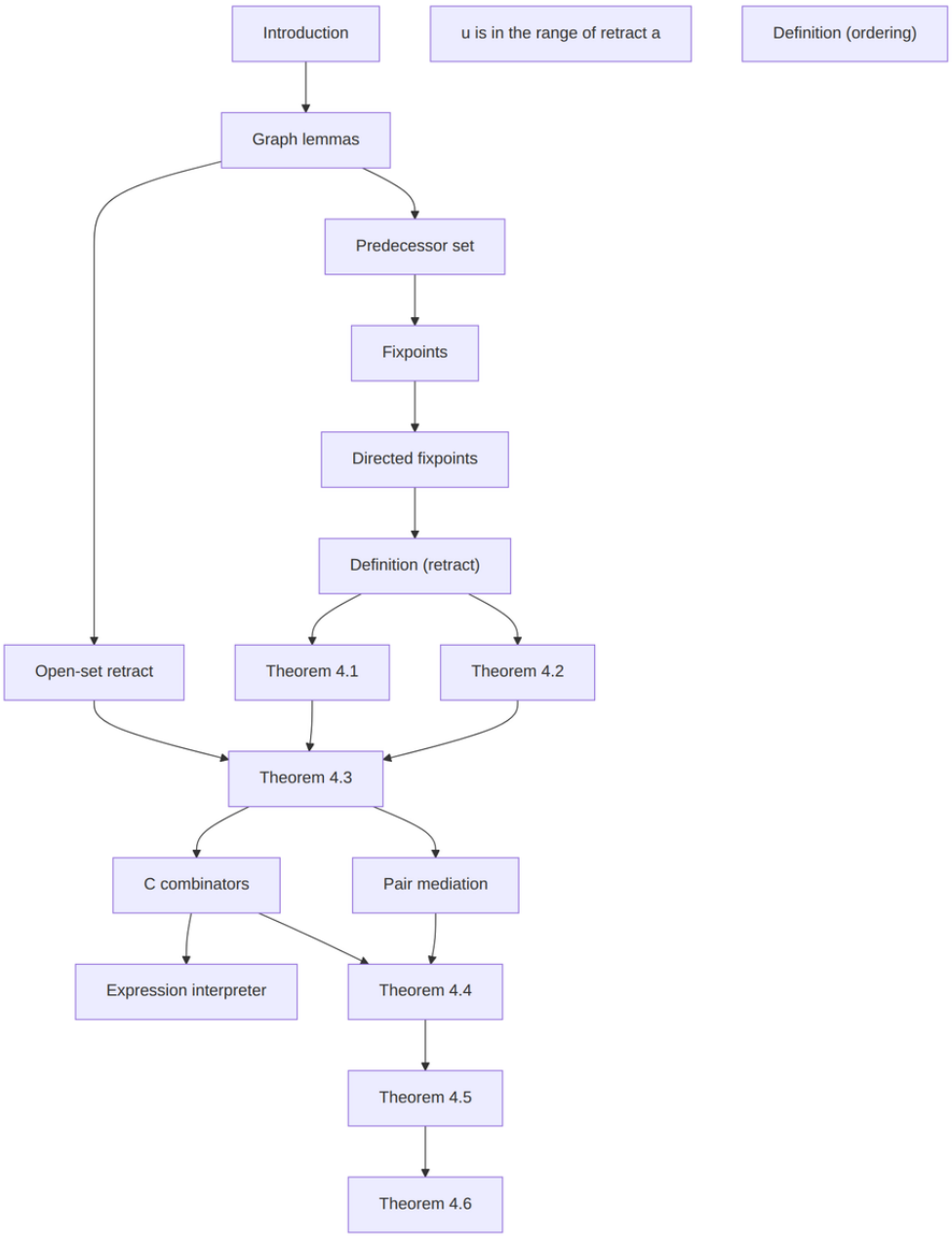


Figure 32: Retracts and data types.

3.5.1 Introduction ✓

Historical note. A retract is an ordinary notion of topology. The use of computable retracts as data types, the operations on those retracts, and the recursive definitions by fixed-point equations are Scott's. A continuous lattice, in the sense of [1], has continuous meet and join, and as a space it is a T_0-space satisfying the

extension theorem. The connection between lattices and fixed points was already known. The set of all retracts is a complete lattice by Theorem 4.1, and it is not itself a continuous lattice. Ershov solved that question after the body of the paper was written; [114] found essentially the same argument independently. The argument is given with the ordering of retracts. The correspondence was Scott's introduction to Ershov's numeration theory. Applications to the semantics of flow diagrams are in [226], and were taken up by [96] and by [207].

Data can be structured in many ways: ordered tuples, lists, arrays, trees, streams, and even operations and functions. The last point becomes clear if one thinks of *parameters*. We would normally hardly consider the pairing function $\lambda x \lambda y.\, \langle x, y \rangle$ as being in itself a piece of data. But if we treat the first variable as a *parameter*, then it can be specialized to a *fixed value*, say the element a, producing the function $\lambda y.\, \langle a, y \rangle$. This function is more likely to be the output of some process and *in itself* can be considered as a datum. It is rather like one *whole row* of a matrix. If we were to regard a two-argument function f as being a matrix, then its ath row would be exactly $\lambda y.\, f(a)(y)$. If s were a selection function, then, for example, $\lambda y.\, f(s(y))(y)$ would represent the selection of one element out of each column of the matrix. This selection could be taken as a specialization of parameters in the operator $\lambda u \lambda v \lambda y.\, u(v(y))(y)$. We have not been very definite here about the exact nature of the fixed a, f, or s, or the range of the variable y or the range of values of the function f. The point is only to recall a few elements of structure and to suggest an abstract view of data going beyond the usual iterated arrays and trees.

What then is a *data type*? Answer: a type of data. That is to say, a collection of data that have been grouped together for reasons of similarity of structure or perhaps mere convenience. Thus the collection may very well be a mixed bag, but more often than not canons of taste or demands of simplicity dictate an adherence to regularity. The grouping may be formed to eliminate irrelevant objects and focus the attention in other ways. It is frequently a matter of good organization that aids the understanding of complex definitions. In programming languages, one of the major reasons for making an explicit declaration of a data type (that is, the restriction of certain variables to certain *"modes"*) is that the computed objects of that type can enjoy a special *representation* in the machine that allows the manipulation of these objects via the chosen representation to be reasonably efficient. This is a very critical matter for good language design and good compiler writing. In this report, however, we cannot discuss the problems of representation,

important as they may be. Our objective here is conceptual organization, and we wish to show how such ideas, in the language for computable functions used here, can find the proper expression.

Which are the data types that can be defined in LAMBDA? No final answer can be given since the number is infinite and inexhaustible. From one point of view, however, there is only one: $\mathbf{P}\omega$ itself. It is the universal type and all other types are subtypes of it; so $\mathbf{P}\omega$ plays a primary role in this exposition. But in a way it is too big, or at least too complex, since each of its elements can be used in so many different ways. When we specify a subtype the intention is to restrict attention to a special use. But even the various subtypes overlap, and so the same elements still get different uses. Style in writing definitions will usually make the differentiation clear though. The main innovation to be described in this section is the use of LAMBDA expressions to define types *as well as* elements. Certain expressions define *retracts* (or better: retraction mappings), and it is the *ranges* (or as we shall see: *sets of finite points*) of such retracts that form the groupings into types. Thus LAMBDA provides a calculus of type definitions including *recursive* type definitions. Examples will be explained both here and in the following sections. Note that types as retracts turn out to be types as *lattices*, that is, types of partial and many-valued objects. The problem of cutting these lattice types down to the perfect or complete objects is discussed in § 6. Another view of types and functionality of mappings is presented in § 7.

The notion of a retract comes from (analytic) topology, but it seems almost an accident that the idea can

be applied in the present context. The word is employed not because there is some deep tie-up with topology but because it is short and rather descriptive. Three easy examples will motivate the general plan:

3.5.2 Graph lemmas ✓

Two elementary graph identities used throughout the retract constructions.

Theorem (`graph_ext`). If $f(x) = g(x)$ for all x, then $\mathrm{graph}(f) = \mathrm{graph}(g)$.

Proof. Unfold the graph coding; the pointwise equality rewrites the defining set. ■

Theorem (`graph_const_bot`). The graph of the constant map with value $\bot$ is $\bot$.

Proof. No coded pair lies in the empty set. ■

`graphLemmas.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem graph_ext {f g : Pomega → Pomega} (h : ∀ x, f x = g x) :
    graph f = graph g := by
  simp [graph, h]

theorem graph_const_bot : graph (fun _ => botElem) = botElem := by
  ext p
  simp [graph, botElem]

end DomainTheory.DataTypesAsLattices
```

3.5.3 Predecessor set ✓

Predecessor-set identities for numerals, used when building integer retracts.

Theorem (`predSet_ofNat_zero`). $\mathrm{predSet}(0) = \bot$.

Proof. The predecessor of the singleton $\{0\}$ is empty. ■

Theorem (`predSet_eq_ofNat_of_no_zero`). If $0 \notin u$ and $\mathrm{predSet}(u) = k$, then $u = k + 1$ as numerals.

Proof. Members of u are successors; the predecessor condition forces each to be exactly $k + 1$, and conversely $k + 1$ has predecessor k. ■

`predSet.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Language

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem predSet_ofNat_zero : predSet (ofNat 0) = botElem := by
```

```
  ext k
  simp [predSet, ofNat, botElem, Set.mem_singleton_iff]

theorem predSet_eq_ofNat_of_no_zero {u : Pomega} {k : ℕ}
    (h0 : 0 ∈ u) (hk : predSet u = ofNat k) :
    u = ofNat (k + 1) := by
  ext m
  constructor
  · intro hm
    have hm0 : m = 0 := fun h => h0 (by simpa [h] using hm)
    obtain ⟨t, rfl⟩ := Nat.exists_eq_succ_of_ne_zero hm0
    have : t ∈ predSet u := hm
    have : t = k := by
      have : t ∈ ofNat k := by rwa [hk] at this
      simpa [ofNat, Set.mem_singleton_iff] using this
    simp [ofNat, Set.mem_singleton_iff, this]
  · intro hm
    simp [ofNat, Set.mem_singleton_iff] at hm
    subst hm
    have : k ∈ predSet u := by
      rw [hk]; simp [ofNat, Set.mem_singleton_iff]
    exact this

end DomainTheory.DataTypesAsLattices
```

3.5.4 Fixpoints ✓

The fixed-point set of a continuous self-map of $\mathbf{P}\omega$ is the carrier of Theorem 4.1.

Definition (`Fixpoints`). $\mathrm{Fixpoints}(f) = \{x \mid f(x) = x\}$.

Definition (`Fixpoints.vals`). For a set A of fixed points (as a subtype), $\mathrm{vals}(A)$ is the underlying set of elements of $\mathbf{P}\omega$.

Theorem (`Fixpoints.vals.subset`). $\mathrm{vals}(A) \subseteq \mathrm{Fixpoints}(f)$.

Proof. Each element of $\mathrm{vals}(A)$ is the underlying point of some fixed-point witness. ■

`Fixpoints.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.eDense

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Fixpoints (f : Pomega → Pomega) : Set Pomega :=
  {x | f x = x}

def Fixpoints.vals {f : Pomega → Pomega} (A : Set (Fixpoints f)) : Set Pomega :=
  Subtype.val '' A

namespace Fixpoints.vals

theorem subset {f : Pomega → Pomega} (A : Set (Fixpoints f)) :
    Fixpoints.vals A ⊆ Fixpoints f := by
```

```
  intro x hx
  obtain ⟨y, _, rfl⟩ := hx
  exact y.property

end Fixpoints.vals

end DomainTheory.DataTypesAsLattices
```

3.5.5 Directed fixpoints ✓

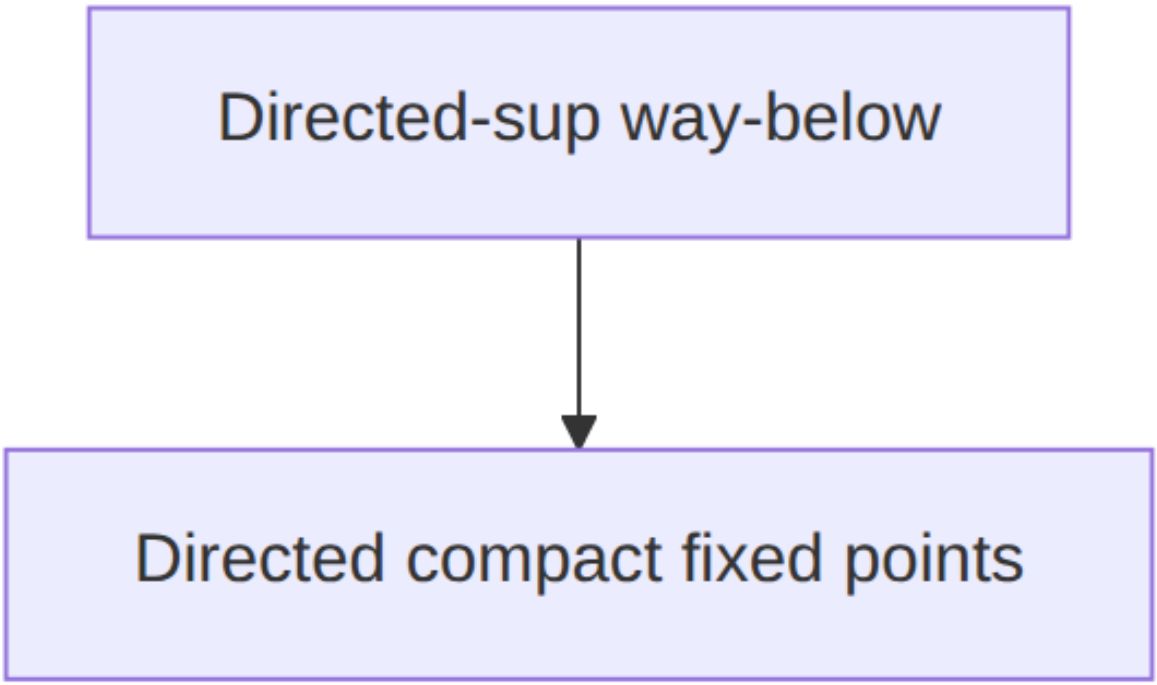


Figure 33: Directed fixpoints ✓.

Directed-sup way-below ✓ The directed-sup form of the way-below relation, used when a retract's fixed-point lattice is treated as a continuous lattice. Distinct from the Continuous Lattice chapter's Scott-open $\ll$.

Definition (`completeLatticeLE`, `completeLatticePreorder`, `completeLatticeSupSet`). The order, preorder, and supremum structures of a complete lattice, named for inference.

Definition (`DirectedSet`). A nonempty set in a preorder is directed when any two elements have a common upper bound in the set.

Definition (`WayBelow`). $x \ll y$ when every directed set whose supremum is at least y already has a member above x.

`WayBelow.lean`

```
module
```

```
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

abbrev completeLatticeLE {α : Type u} [CompleteLattice α] : LE α :=
  @Preorder.toLE α
    (@PartialOrder.toPreorder α
      (@CompleteSemilatticeInf.toPartialOrder α
        (@CompleteLattice.toCompleteSemilatticeInf α ?_?)))

abbrev completeLatticePreorder {α : Type u} [CompleteLattice α] : Preorder α :=
  @PartialOrder.toPreorder α
    (@CompleteSemilatticeInf.toPartialOrder α
      (@CompleteLattice.toCompleteSemilatticeInf α ?_?))

abbrev completeLatticeSupSet {α : Type u} [CompleteLattice α] : SupSet α :=
  @CompleteSemilatticeSup.toSupSet α
    (@CompleteLattice.toCompleteSemilatticeSup α ?_?)

class DirectedSet {α : Type u} [Preorder α] (s : Set α) : Prop where
  out : s.Nonempty ∧
    ∀ {{x}}, x ∈ s → ∀ {{y}}, y ∈ s →
      ∃ z ∈ s, x ≤ z ∧ y ≤ z

/-- Directed-sup form of ≪ (for fixed-point lattices in `Pω`).
    Distinct from the Continuous Lattice chapter?s Scott-open ≪. -/
def WayBelow {α : Type u} [CompleteLattice α] (x y : α) : Prop :=
  ∀ s : Set α, @DirectedSet α completeLatticePreorder s →
    @LE.le α completeLatticeLE y (@sSup α completeLatticeSupSet s) →
    ∃ z ∈ s, @LE.le α completeLatticeLE x z

end DomainTheory.DataTypesAsLattices
```

Directed compact fixed points ✓ Directed families of fixed points of a continuous self-map of $\mathbf{P}\omega$, and the compactness property used for continuous retract lattices.

Definition (`DirectedSubset`). A nonempty set s of fixed points of f is directed when any two members have an upper bound in s under $\subseteq$.

Definition (`IsCompactSubset`). A fixed point x is compact (relative to f) when every directed family of fixed points whose union covers x has a single member that already covers x.

Theorem (`directed_finite_cover`). If A is a directed set of fixed points and $t \subseteq \mathbf{P}\omega$ is finite and contained in $\bigcup \mathrm{vals}(A)$, then some $z \in A$ already contains t.

Proof. Induct on the finite set: the empty set is covered by any member of the nonempty directed family; for an inserted point, take upper bounds for the inductive cover and for a witness covering the new point. ■

`DirectedSubset.lean`

```
module

public import DomainTheory.DataTypesAsLattices.FixedLattices.WayBelow
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
import Mathlib.Data.Set.Finite.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

class DirectedSubset {f : Pomega → Pomega} (s : Set (Fixpoints f)) : Prop where
  out : s.Nonempty ∧
    ∀ {{x}}, x ∈ s → ∀ {{y}}, y ∈ s →
      ∃ z ∈ s, (x : Pomega) ⊆ z ∧ (y : Pomega) ⊆ z

def IsCompactSubset {f : Pomega → Pomega} (x : Fixpoints f) : Prop :=
  ∀ s : Set (Fixpoints f),
    DirectedSubset s →
      (x : Pomega) ⊆ ⋃₀ (Subtype.val '' s) →
      ∃ z ∈ s, (x : Pomega) ⊆ z

theorem directed_finite_cover {A : Set (Fixpoints f)}
    (hA : DirectedSet A) {t : Pomega} (ht : t.Finite)
    (hsub : t ⊆ ⋃₀ (Fixpoints.vals A)) :
    ∃ z ∈ A, t ⊆ z := by
  classical
  have aux : ∀ s : Finset ℕ,
      (s : Set ℕ) ⊆ ⋃₀ (Fixpoints.vals A) →
        ∃ z ∈ A, (s : Set ℕ) ⊆ z := by
    intro s
    induction s using Finset.induction with
    | empty =>
        intro _
        obtain ⟨z, hz⟩ := hA.out.1
        exact ⟨z, hz, by simp⟩
    | insert k s hks ih =>
        intro hs
        obtain ⟨u, hu, hsu⟩ :=
          ih (fun m hm => hs (by simp [hm]))
        obtain ⟨v, hv, hkv⟩ :=
          Set.mem_sUnion.mp
            (hs (show k ∈ (insert k s : Finset ℕ) by simp))
        obtain ⟨v', hv'A, rfl⟩ := hv
        obtain ⟨z, hz, huz, hvz⟩ := hA.out.2 hu hv'A
        refine ⟨z, hz, ?_⟩
        intro m hm
        simp only [Finset.coe_insert, Set.mem_insert_iff] at hm
        rcases hm with rfl | hm
        · exact hvz hkv
        · exact huz (hsu hm)
  obtain ⟨z, hzA, hz⟩ := aux ht.toFinset (by simpa using hsub)
  exact ⟨z, hzA, fun m hm => hz (by simpa using hm)⟩

end DomainTheory.DataTypesAsLattices
```

3.5.6 Definition (retract) ✓

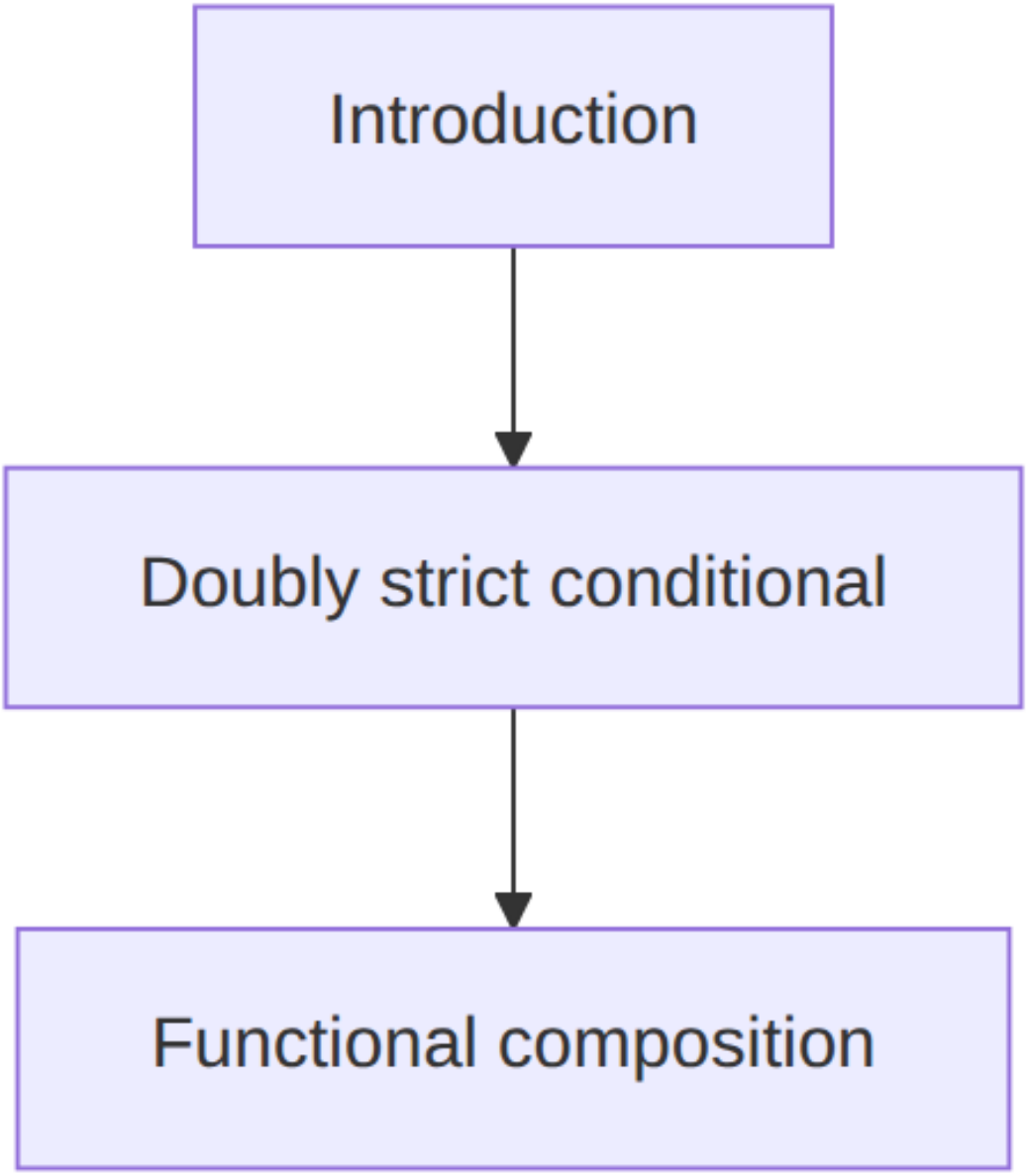


Figure 34: Definition (retract) ✓.

Introduction ✓ **DEFINITION.** An element $a \in \mathbf{P}\omega$ is called a *retract* iff it satisfies the equation $a = a \circ a$.

Of course the $\circ$-notation is used for functional composition in the standard way:

$$(4.5)\quad u \circ v = \lambda x.\, u(v(x)).$$

Consider these combinators:

$$(4.1)\quad \mathbf{fun} = \lambda u \lambda x.\, u(x);$$

$$(4.2)\quad \mathbf{pair} = \lambda u.\, \langle u_0, u_1 \rangle;$$

$$(4.3)\quad \mathbf{bool} = \lambda u.\, u \sqsupset 0, 1.$$

Here $\sqsupset$ is the *doubly strict conditional* defined by

$$(4.4)\quad z \sqsupset x, y = z \supset (z \supset x, \top), (z \supset \top, y),$$

which has the property that if z is both zero and positive, then it takes the value $\top$ instead of the value $x \cup y$.

It is quite simple to prove that each of the three combinators in (4.1)–(4.3) is a retract according to the definition. But what is the point?

Consider $\mathbf{fun}$. No matter what $u \in \mathbf{P}\omega$ we take, $\mathbf{fun}(u)$ is (the graph of) a *function*. And if u already is (the graph of) a function, then $u = \mathbf{fun}(u)$. That is to say, the range of $\mathbf{fun}$ is the same as the set of fixed points of $\mathbf{fun}$ is the same as the set of all (graphs of) functions. Any mapping a whose range and fixed-point set

coincide satisfies $a = a \circ a$, and conversely. A retract is a mapping which "retracts" the whole space onto its range and which is the identity mapping on its range. That is the import of the equation $a = a \circ a$. Strictly speaking, the range is the retract and the mapping is the retraction, but for us the mapping is more important. (Note, however, that distinct retracts can have the same range.) We let the mapping stand in for the range.

Thus the combinator **fun** represents in itself the concept of a *function* (continuous function on $\mathbf{P}\omega$ into $\mathbf{P}\omega$). Similarly **pair** represents the idea of a *pair* and **bool** the idea of being a boolean value as an element of $\{\bot, 0, 1, \top\}$, since we must think in the multiple-valued mode. What is curious (and, as we shall see, useful) is that all these retracts which are defining subspaces are at the same time elements of $\mathbf{P}\omega$.

Doubly strict conditional ✓ The doubly strict conditional of (4.4), used to define $\mathbf{bool}$.

Definition (`dcondSet`). For $z, x, y \in \mathbf{P}\omega$,

$$z \sqsupset x, y \;=\; z \supset \big(z \supset x,\, \top\big),\; \big(z \supset \top,\, y\big).$$

Equivalently, nest McCarthy's conditional: test z for zero, and on each branch test z again against $\top$ so that a mixed tag (both zero and positive) yields $\top$ instead of $x \cup y$.

Theorem (`dcondSet.isScottContinuous_left`). For fixed x, y, the map $z \mapsto z \sqsupset x, y$ is Scott-continuous.

Proof. Composition of continuous McCarthy conditionals (in each argument) with the continuous identity and the continuous maps $z \mapsto z \supset x, \top$ and $z \mapsto z \supset \top, y$. ■

Theorem (`dcondSet.isScottContinuous_mid`). For fixed z, y, the map $x \mapsto z \sqsupset x, y$ is Scott-continuous.

Proof. As a composite of continuous conditionals in the middle argument. ■

Theorem (`dcondSet.isScottContinuous_right`). For fixed z, x, the map $y \mapsto z \sqsupset x, y$ is Scott-continuous.

Proof. As a composite of continuous conditionals in the right argument. ■

Theorem (`dcondSet.bot`). $\bot \sqsupset x, y = \bot$.

Proof. Both nested conditionals see an empty tag, so the result is empty. ■

Theorem (`dcondSet.cases`). For any z, exactly one of the following holds: z is a pure zero tag ($0 \in z$ and no positive); a pure positive tag ($0 \notin z$ and some $k{+}1 \in z$); mixed (both); or empty (neither).

Proof. Case-split on membership of 0 and of some successor. ■

Theorem (`dcondSet.of_empty`). If z is empty (neither zero nor positive), then $z \sqsupset x, y = \bot$.

Proof. Neither branch of the outer conditional can fire. ■

Theorem (`dcondSet.of_zero_not_pos`). If z is a pure zero tag, then $z \sqsupset x, y = x$.

Proof. The outer test takes the "zero" branch; the inner $z \supset x, \top$ collapses to x, while the unused right branch is $\top$. ■

Theorem (`dcondSet.of_pos_not_zero`). If z is a pure positive tag, then $z \sqsupset x, y = y$.

Proof. Symmetrically: the outer test takes the "positive" branch, and $z \supset \top, y$ collapses to y. ■

Theorem (`dcondSet.of_mixed`). If z is mixed, then $z \sqsupset x, y = \top$.

Proof. Both inner conditionals evaluate to $\top$ (each has a live tag forcing the $\top$ arm), so the outer conditional yields $\top$. ■

Theorem (`dcondSet.ofNat_zero`). $0 \sqsupset x, y = x$.

Proof. The numeral 0 is a pure zero tag. ■

Theorem (`dcondSet.ofNat_one`). $1 \sqsupset x, y = y$.

Proof. The numeral 1 is a pure positive tag. ■

Theorem (`dcondSet.ofNat_succ`). For every n, $(n{+}1) \sqsupset x, y = y$.

Proof. Every positive numeral is a pure positive tag. ■

Theorem (`dcondSet.top`). $\top \sqsupset x, y = \top$.

Proof. $\top$ is mixed (contains 0 and every successor). ■

Theorem (`dcondSet.bool_idem`). $(u \sqsupset 0, 1) \sqsupset 0, 1 \;=\; u \sqsupset 0, 1$.

Proof. By the four cases on u: pure zero gives $0 \sqsupset 0, 1 = 0$; pure positive gives $1 \sqsupset 0, 1 = 1$; mixed gives $\top \sqsupset 0, 1 = \top$; empty gives $\bot \sqsupset 0, 1 = \bot$. In each case a second $\sqsupset 0, 1$ is the identity on that value. ■

`dcondSet.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condSet
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Doubly strict conditional (Scott (4.4)):
    `z ? x, y = z ? (z ? x, ⊤), (z ? ⊤, y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

namespace dcondSet
```

```
theorem isScottContinuous_left (x y : Pomega) :
    IsScottContinuous (fun z => dcondSet z x y) :=
  theorem_1_3.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    id_isScottContinuous
    (condSet.isScottContinuous_left x topElem)
    (condSet.isScottContinuous_left topElem y)

theorem bot (x y : Pomega) : dcondSet botElem x y = botElem := by
  ext n
  simp [dcondSet, condSet, botElem]

theorem cases (z : Pomega) :
    (0 ∈ z ∧ ¬ ∃ k, k + 1 ∈ z) ∨
      (0 ∈ z ∧ ∃ k, k + 1 ∈ z) ∨
        (0 ∈ z ∧ ∃ k, k + 1 ∈ z) ∨
          (0 ∈ z ∧ ¬ ∃ k, k + 1 ∈ z) := by
  by_cases h0 : 0 ∈ z
  · by_cases hp : ∃ k, k + 1 ∈ z
    · exact Or.inr (Or.inr (Or.inl ⟨h0, hp⟩))
    · exact Or.inl ⟨h0, hp⟩
  · by_cases hp : ∃ k, k + 1 ∈ z
    · exact Or.inr (Or.inl ⟨h0, hp⟩)
    · exact Or.inr (Or.inr (Or.inr ⟨h0, hp⟩))

theorem isScottContinuous_mid (z y : Pomega) :
    IsScottContinuous (fun x => dcondSet z x y) :=
  theorem_1_3 (f := fun w => condSet z w (condSet z topElem y))
    (g := fun x => condSet z x topElem)
    (condSet.isScottContinuous_mid z (condSet z topElem y))
    (condSet.isScottContinuous_mid z topElem)

theorem isScottContinuous_right (z x : Pomega) :
    IsScottContinuous (fun y => dcondSet z x y) :=
  theorem_1_3 (f := fun w => condSet z (condSet z x topElem) w)
    (g := fun y => condSet z topElem y)
    (condSet.isScottContinuous_right z (condSet z x topElem))
    (condSet.isScottContinuous_right z topElem)

theorem of_empty {z x y : Pomega}
    (h0 : 0 ∈ z) (hp : ¬ ∃ k, k + 1 ∈ z) :
    dcondSet z x y = botElem := by
  ext n
  constructor
  · intro hn
    rcases hn with ⟨_, hz⟩ | ⟨_, ht⟩
    · exact (h0 hz).elim
    · exact (hp ht).elim
  · intro hn
    simp [botElem] at hn

theorem of_mixed {z x y : Pomega}
    (h0 : 0 ∈ z) (hp : ∃ k, k + 1 ∈ z) :
```

```
    dcondSet z x y = topElem := by
  have hL : condSet z x topElem = topElem := by
    ext n
    constructor
    · intro _; exact Set.mem_univ n
    · intro _; exact Or.inr ⟨Set.mem_univ n, hp⟩
  have hR : condSet z topElem y = topElem := by
    ext n
    constructor
    · intro _; exact Set.mem_univ n
    · intro _; exact Or.inl ⟨Set.mem_univ n, h0⟩
  rw [dcondSet, hL, hR]
  ext n
  constructor
  · intro _; exact Set.mem_univ n
  · intro _; exact Or.inl ⟨Set.mem_univ n, h0⟩

theorem of_pos_not_zero {z x y : Pomega}
    (h0 : 0 ∈ z) (hp : ∃ k, k + 1 ∈ z) :
    dcondSet z x y = y := by
  have hL : condSet z x topElem = topElem := by
    ext n
    constructor
    · intro _; exact Set.mem_univ n
    · intro _; exact Or.inr ⟨Set.mem_univ n, hp⟩
  have hR : condSet z topElem y = y := by
    ext n
    constructor
    · intro hn
      rcases hn with ⟨_, hz⟩ | ⟨hy, _⟩
      · exact (h0 hz).elim
      · exact hy
    · intro hy
      exact Or.inr ⟨hy, hp⟩
  rw [dcondSet, hL, hR]
  ext n
  constructor
  · intro hn
    rcases hn with ⟨_, hz⟩ | ⟨hy, _⟩
    · exact (h0 hz).elim
    · exact hy
  · intro hy
    exact Or.inr ⟨hy, hp⟩

theorem of_zero_not_pos {z x y : Pomega}
    (h0 : 0 ∈ z) (hp : ¬ ∃ k, k + 1 ∈ z) :
    dcondSet z x y = x := by
  have hL : condSet z x topElem = x := by
    ext n
    constructor
    · intro hn
      rcases hn with ⟨hx, _⟩ | ⟨_, ht⟩
      · exact hx
      · exact (hp ht).elim
    · intro hx
      exact Or.inl ⟨hx, h0⟩
```

```
  have hR : condSet z topElem y = topElem := by
    ext n
    constructor
    · intro _; exact Set.mem_univ n
    · intro _; exact Or.inl ⟨Set.mem_univ n, h0⟩
  rw [dcondSet, hL, hR]
  ext n
  constructor
  · intro hn
    rcases hn with ⟨hx, _⟩ | ⟨_, ht⟩
    · exact hx
    · exact (hp ht).elim
  · intro hx
    exact Or.inl ⟨hx, h0⟩

theorem ofNat_one (x y : Pomega) : dcondSet (ofNat 1) x y = y := by
  ext n
  simp [dcondSet, condSet, ofNat, topElem]

theorem ofNat_succ (n : ℕ) (x y : Pomega) :
    dcondSet (ofNat (n + 1)) x y = y :=
  dcondSet.of_pos_not_zero (by simp [ofNat]) ⟨n, by simp [ofNat]⟩

theorem ofNat_zero (x y : Pomega) : dcondSet (ofNat 0) x y = x := by
  ext n
  simp [dcondSet, condSet, ofNat, topElem]

theorem top (x y : Pomega) : dcondSet topElem x y = topElem := by
  ext n
  simp [dcondSet, condSet, topElem]

/-- Idempotence of `? 0, 1` used for `bool`. -/
theorem bool_idem (u : Pomega) :
    dcondSet (dcondSet u (ofNat 0) (ofNat 1)) (ofNat 0) (ofNat 1) =
      dcondSet u (ofNat 0) (ofNat 1) := by
  rcases dcondSet.cases u with h | h | h | h
  · rw [dcondSet.of_zero_not_pos h.1 h.2, dcondSet.ofNat_zero]
  · rw [dcondSet.of_pos_not_zero h.1 h.2, dcondSet.ofNat_one]
  · rw [dcondSet.of_mixed h.1 h.2, dcondSet.top]
  · rw [dcondSet.of_empty h.1 h.2, dcondSet.bot]

end dcondSet

end DomainTheory.DataTypesAsLattices
```

Functional composition ✓ Functional composition (4.5), the retract equation, and the motivating combinators **fun** and **bool**.

Definition (`comp`). Composition of graphs is

$$u \circ v \;=\; \mathbf{graph}\big(x \mapsto u(v(x))\big),$$

Scott's equation (4.5).

Theorem (`comp.app`). $(u \circ v)(x) = u(v(x))$.

Proof. The body $x \mapsto u(v(x))$ is Scott-continuous (composite of continuous applications), so (β) applies. ■

Theorem (`comp.isScottContinuous_right`). For fixed u, the map $v \mapsto u \circ v$ is Scott-continuous.

Proof. The graphing of a family continuous in v (pointwise $x \mapsto u(v(x))$ is continuous in v) is continuous. ■

Theorem (`comp.right_const`). For fixed a, the map $u \mapsto u \circ a$ is Scott-continuous.

Proof. Likewise: $x \mapsto u(a(x))$ is continuous in u, and graphing preserves that. ■

Definition (`Retract`). An element $a \in \mathbf{P}\omega$ is a *retract* when $a = a \circ a$.

Definition (`funRetract`). Scott's **fun** of (4.1):

$$\mathbf{fun} = \mathbf{graph}\big(u \mapsto \mathbf{graph}(x \mapsto u(x))\big).$$

Theorem (`funRetract.app`). $\mathbf{fun}(u) = \mathbf{graph}(x \mapsto u(x))$.

Proof. (β): the body is continuous in u because application is continuous in the function argument. ■

Theorem (`funRetract.isRetract`). **fun** is a retract: $\mathbf{fun} = \mathbf{fun} \circ \mathbf{fun}$.

Proof. Pointwise on graphs: for each u, $\mathbf{fun}(\mathbf{fun}(u)) = \mathbf{graph}(x \mapsto \mathbf{fun}(u)(x)) = \mathbf{graph}(x \mapsto u(x)) = \mathbf{fun}(u)$, using (β) for continuous u. ■

Definition (`boolR`). Scott's **bool** of (4.3):

$$\mathbf{bool} = \mathbf{graph}\big(u \mapsto u \sqsupset 0, 1\big).$$

Theorem (`boolR.app`). $\mathbf{bool}(u) = u \sqsupset 0, 1$.

Proof. (β) from continuity of $u \mapsto u \sqsupset 0, 1$ in the left argument of $\sqsupset$. ■

Theorem (`boolR.isRetract`). **bool** is a retract: $\mathbf{bool} = \mathbf{bool} \circ \mathbf{bool}$.

Proof. Pointwise: $\mathbf{bool}(\mathbf{bool}(u)) = (u \sqsupset 0, 1) \sqsupset 0, 1 = u \sqsupset 0, 1 = \mathbf{bool}(u)$ by idempotence of $\sqsupset 0, 1$. ■

(Scott's **pair** of (4.2) is not yet a named retract module; pairing of elements is `pairSeq` elsewhere.)

`Retract.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
import DomainTheory.DataTypesAsLattices.Retracts.graphLemmas
import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
import DomainTheory.DataTypesAsLattices.Graph.beta
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Functional composition (Scott (4.5)): `u ∘ v = λx. u(v(x))`. -/
def comp (u v : Pomega) : Pomega :=
  graph (fun x => funOf u (funOf v x))

namespace comp

theorem app (u v x : Pomega) :
```

```
    funOf (comp u v) x = funOf u (funOf v x) :=
  beta (theorem_1_3 (funOf.isScottContinuous u) (funOf.isScottContinuous v)) x

theorem isGraph (u v : Pomega) :
    comp u v = graph (fun x => funOf (comp u v) x) := by
  apply graph_ext
  intro x
  exact (comp.app u v x).symm

theorem isScottContinuous_right (u : Pomega) :
    IsScottContinuous (fun v => comp u v) :=
  graph_const_isScottContinuous (fun v x => funOf u (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous u) (funOf.isScottContinuous_left x)
    )

theorem right_const (a : Pomega) :
    IsScottContinuous (fun u => comp u a) :=
  graph_const_isScottContinuous (fun u x => funOf u (funOf a x))
    (fun x => funOf.isScottContinuous_left (funOf a x))

end comp

/-- An element `a` is a retract iff `a = a ∘ a`. -/
class Retract (a : Pomega) : Prop where
  out : a = comp a a

/-- Scott?s `fun` of (4.1). -/
def funRetract : Pomega :=
  graph (fun u => graph (fun x => funOf u x))

namespace funRetract

theorem app (u : Pomega) :
    funOf funRetract u = graph (fun x => funOf u x) :=
  beta (graph_const_isScottContinuous (fun u x => funOf u x)
    (fun x => funOf.isScottContinuous_left x)) u

theorem isRetract : Retract funRetract := by
  constructor
  apply graph_ext
  intro u
  rw [funRetract.app, funRetract.app]
  apply graph_ext
  intro x
  exact (beta (funOf.isScottContinuous u) x).symm

end funRetract

/-- Scott?s `bool` of (4.3). -/
def boolR : Pomega :=
  graph (fun u => dcondSet u (ofNat 0) (ofNat 1))

namespace boolR

theorem app (u : Pomega) :
    funOf boolR u = dcondSet u (ofNat 0) (ofNat 1) :=
```

```
  beta (dcondSet.isScottContinuous_left (ofNat 0) (ofNat 1)) u

theorem isRetract : Retract boolR := by
  constructor
  apply graph_ext
  intro u
  rw [app, app, dcondSet.bool_idem]

end boolR

end DomainTheory.DataTypesAsLattices
```

3.5.7 Theorem 4.1 ✓

For Section 4. The notion of a *continuous lattice* is due to [224, 225] and we shall not review all the facts here. One special feature of these lattices is that the lattice operations of meet and join ($\sqcap$ and $\sqcup$) are continuous (that is, commute with directed sups). As topological spaces, they can be characterized as those T_0-spaces satisfying the extension theorem (which we proved for $\mathbf{P}\omega$ in Theorem 1.5).

THEOREM 4.1 (The lattice theorem). The fixed points of any continuous function form a complete lattice (under $\subseteq$); while those of a retract form a continuous lattice.

Proof of Theorem 4.1. Consider a continuous function a, and let $A = \{x \mid x = a(x)\}$. By the fixed-point theorem (Theorem 1.4) we know that A is nonempty and that it has a least element under $\subseteq$. Certainly A is partially ordered by $\subseteq$; further, A is closed under *directed* unions but not under arbitrary unions.

That is, A is not a complete sublattice of $\mathbf{P}\omega$ with regard to the lattice operations of $\mathbf{P}\omega$, but it could be a complete lattice on its own—if we can show sups exist. Thus, let $S \subseteq A$ be an arbitrary subset of A. By the fixed-point theorem, find the least solution to the equation:

$$y = \bigcup\{x \mid x \in S\} \cup a(y).$$

Clearly $x \subseteq y$ for all $x \in S$; and so $x = a(x) \subseteq a(y)$, for all $x \in S$. This means that $y = a(y)$, and thus $y \in A$. By construction, then, y is an upper bound to the elements of S. Suppose $z \in A$ is another upper bound for S. It will also satisfy the above equation; thus $y \subseteq z$, and so y is the least upper bound. A partially ordered set with sups also has infs, as is well known, and is a complete lattice.

Suppose that a is a retract. We can easily show that the fixed-point set A (with the relative topology from $\mathbf{P}\omega$) satisfies the extension theorem. For assume $f : X \to A$ is continuous, and $X \subseteq Y$ as a subspace. Now we can also regard $f : X \to \mathbf{P}\omega$ as continuous because A is a subspace of $\mathbf{P}\omega$. By Theorem 1.5 there is an extension to a continuous $\bar{f} : Y \to \mathbf{P}\omega$. But then $a \circ \bar{f} : Y \to A$ is the continuous function we want for A, and the proof is complete.

The space of retracts. Let us define:

$$\mathbf{RET} = \{a \mid a = a \circ a\},$$

the set of all retracts, which is a complete lattice in view of Theorem 4.1. It will be proved to be not a retract itself by showing it is not a continuous lattice; in fact, the meet operation on $\mathbf{RET}$ is not continuous on $\mathbf{RET}$.

The proof was kindly communicated by Y. L. Ershov and rests on distinguishing some extreme cases of retracts. Call a retract a *nonextensive* if for all nonempty finite sets x we have $x \not\subseteq a(x)$. Call a retract b *finite* if all its values are finite (i.e., $b(\top)$ is finite). If a is nonextensive and b is finite, then Ershov notes that

they are "orthogonal" in **RET** in the sense that $c = a \sqcap b = \bot$. The reason is that, since $c \subseteq b$, it is finite; but $c \subseteq a$, too, so $c(x) \subseteq a(x)$ for all x. Because c is a retract, we have $c(x) = c(c(x)) \subseteq a(c(x))$. As $c(x)$ is finite and a is nonextensive, it follows that $c(x) = \bot$ for all x.

This orthogonality is unfortunate, because consider the finite retracts $b_n = \lambda x.\, e_n$. We have here a directed set of retracts where $\bigcup\{b_n \mid n \in \omega\} = \lambda x.\, \top = \top$. If $\sqcap$ were continuous, it would follow that for nonextensive a:

$$a = a \sqcap \top = a \sqcap \bigcup\{b_n \mid n \in \omega\} = \bigcup\{a \sqcap b_n \mid n \in \omega\} = \bot,$$

showing that there are no nontrivial such a. But this is not so.

Let $\ll$ be a strict linear ordering of ω in the order type of the rational numbers. Define:

$$a(x) = \{m \mid \exists n \in x.\, m \ll n\}.$$

We see at once that a is continuous; and, because $\ll$ is transitive and dense, a is a retract. Since $\ll$ is irreflexive, it is the case for finite nonempty sets x that $\max_{\ll}(x) \notin a(x)$; hence, a is nonextensive. As $a(\top) = \top$, we find $a \neq \bot$. The proof is complete.

Note that there are many transitive, dense, irreflexive relations on ω, so there are many nonextensive retracts. These retracts, like a above, are *distributive*.

A nondistributive example is:

$$a' = \{m \mid \exists n, n' \in x.\, n \ll m \ll n'\}.$$

Many other examples are possible.

theorem_4_1.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.retractBasis

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace theorem_4_1

theorem complete (f : Pomega → Pomega) (hf : IsScottContinuous f)
    (A : Set Pomega) (hA : A ⊆ Fixpoints f) :
    gfpBelow f (⋂₀ A) ∈ Fixpoints f :=
  gfpBelow.mem f hf A hA

theorem continuous {a : Pomega} (ha : Retract a) :
    ∀ x : Fixpoints (funOf a),
      DirectedSubset (retractBasis a x) ∧
        (x : Pomega) = ⋃₀ (Subtype.val '' retractBasis a x) ∧
        ∀ y ∈ retractBasis a x,
          ∀ s : Set (Fixpoints (funOf a)),
            DirectedSubset s →
              (x : Pomega) ⊆ ⋃₀ (Subtype.val '' s) →
              ∃ z ∈ s, (y : Pomega) ⊆ z := by
  intro x
```

```
  refine ⟨retractBasis.directedSubset ha x, retractBasis.eq_sUnion ha x, ?_⟩
  intro y hy s hs hxs
  obtain ⟨n, hnx, hyn⟩ := hy
  have hen : e n ⊆ ⋃₀ (Subtype.val '' s) := hnx.trans hxs
  have hs' : DirectedSet s :=
    ⟨⟨hs.out.1, fun u hu v hv => by
      obtain ⟨w, hw, huw, hvw⟩ := hs.out.2 hu hv
      exact ⟨w, hw, huw, hvw⟩⟩⟩
  obtain ⟨z, hz, hnz⟩ :=
    directed_finite_cover hs' (e.finite n) (by simpa [Fixpoints.vals] using hen)
  refine ⟨z, hz, ?_⟩
  rw [hyn]
  have := IsScottContinuous.monotone (funOf.isScottContinuous a) hnz
  rwa [z.property] at this

end theorem_4_1

theorem theorem_4_1 {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    (∀ A : Set Pomega, A ⊆ Fixpoints f →
      gfpBelow f (⋂₀ A) ∈ Fixpoints f ∧
        lfpAbove f (⋃₀ A) ∈ Fixpoints f) ∧
      ∀ {a} (_ha : Retract a),
        Fixpoints (funOf a) = {x | typed x a} ∧
          ∀ x : Fixpoints (funOf a),
            DirectedSubset (retractBasis a x) ∧
              (x : Pomega) = ⋃₀ (Subtype.val '' retractBasis a x) := by
  constructor
  · intro A hA
    exact ⟨theorem_4_1.complete f hf A hA, lfpAbove.mem hf hA⟩
  · intro a ha
    constructor
    · ext x
      simp [Fixpoints, typed, eq_comm]
    · intro x
      exact ⟨retractBasis.directedSubset ha x, retractBasis.eq_sUnion ha x⟩

end DomainTheory.DataTypesAsLattices
```

3.5.8 u is in the range of retract a ✓

DEFINITION. If a is a retract, we write $u : a$ for $u = a(u)$ and $\lambda u : a.\, \tau$ for $\lambda u.\, \tau[a(u)/u]$.

Since retracts are sets in $\mathbf{P}\omega$, we cannot use the ordinary membership symbol to signify that u belongs to the range of a; so we write $u : a$. The other notation with the λ-operator restricts a function to the range of a. For f to be so restricted simply means $f = f \circ a$. For the range of f to be contained in that of the retract a means $f = a \circ f$. These algebraic equations will be found to be quite handy. We are going to have a calculus of retracts and mappings between them involving many operators on retracts yet to be discovered. Before we turn to this calculus, we recall the well-known connection between lattices and fixed points.

3.5.9 Definition (ordering) ✓

We note further that by the embedding theorem (Theorem 1.6), it follows that any separable (by which we mean countably-based) continuous lattice is a retract of $\mathbf{P}\omega$; hence, our universal space is indeed rich in

retracts. A very odd point is that $a = a \circ a$ is a fixed-point equation itself ($\lambda u.\, u \circ u$ is obviously continuous). Thus the retraction mappings form a complete lattice. Is this a continuous lattice? (Ershov has proved it is not; see the Appendix for a sketch.) A related question is solved positively in the next section. Actually the ordering of retracts under $\subseteq$ does not seem to be all that interesting; a more algebraic ordering is given by:

DEFINITION. For retracts a and b we write $a \sqsubseteq b$ for $a = a \circ b = b \circ a$.

The idea here should be clear: $a \sqsubseteq b$ means that a is a retract of b. It is easy to prove the:

3.5.10 Open-set retract ✓

An open-set retract used later when comparing retracts and Scott-open structure.

Definition (`openSetRetract`). openSetRetract is the graph of the map sending u to $\{m \mid \exists n.\, e_n \subseteq e_m \wedge n \in u\}$: the Scott-open sets coded by members of u.

`openSetRetract.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def openSetRetract : Pomega :=
  graph (fun u => {m | ∃ n, e n ⊆ e m ∧ n ∈ u})

end DomainTheory.DataTypesAsLattices
```

3.5.11 Theorem 4.2 ✓

THEOREM 4.2 (The partial ordering theorem). The retracts are partially ordered by $\sqsubseteq$.

There do not seem to be any lattice properties of $\sqsubseteq$ of a general nature. Note, however, that if retracts commute, $a \circ b = b \circ a$, then $a \circ b$ is the greatest lower bound under $\sqsubseteq$ of a and b. Also if we have a sequence where both $a_n \sqsubseteq a_{n+1}$ and $a_n \subseteq a_{n+1}$ for all $n \in \omega$, then $\bigcup\{a_n \mid n \in \omega\}$ is the upper bound for the a_n under $\sqsubseteq$, as can easily be argued from the definition by continuity of $\circ$.

Certainly there is no "least" retract under $\sqsubseteq$. One has $\bot = \bot \circ a$ (recall: $\bot = \lambda x.\, \bot$), but not $a \circ \bot = \bot$. This last equation means more simply that $a(\bot) = \bot$; that is, a is *strict*. For retracts strictness is thus equivalent to $\bot \sqsubseteq a$, so we can say that there is a least strict retract. The combinator $\mathbf{I} = \lambda u.\, u$ clearly represents the

largest retract (the *whole space*), and it is strict also. In a certain sense strictness can be assumed without loss of generality. For if a is not strict, let

$$b = \lambda x.\, \{n \mid a(x) \neq a(\bot)\}.$$

This function takes values in $\{\bot, \top\}$ and is continuous because $\{x \in \mathbf{P}\omega \mid a(x) \neq a(\bot)\}$ is *open*. Next define:

$$a^* = \lambda u.\, a(u) \cap b(u).$$

This is a strict retract whose range is homeomorphic (and lattice isomorphic) to that of a. Note, however, that the mapping from a to a^* is not continuous (or even monotonic).

To have a more uniform notation for retracts we shall often write **nil**, **id**, and **seq** for the combinators $\bot$, **I**, \$. Two further retracts of interest are

$$(4.6) \quad \mathbf{open} = \lambda u.\, \{m \mid \exists e_n \subseteq e_m.\, n \in u\};$$

$$(4.7) \quad \mathbf{int} = \lambda u.\, u \supset 0,\, \mathbf{int}(u-1) \supset u,\, u.$$

The range of **open** is lattice isomorphic to the lattice of open subsets of $\mathbf{P}\omega$; definition (4.6) is not a LAMBDA-definition of the retract, but such can be given.

In (4.7) we intend **int** to be the least fixed point of the equation. By induction on the least element of u (if any) one proves that:

$$\mathbf{int}(u) = \begin{cases} \bot & \text{if } u = \bot; \\ u & \text{if } u \in \omega; \\ \top & \text{otherwise.} \end{cases}$$

This retract wipes out the distinctions between multiple values, moving all above the singletons up to $\top$; its range thus has a very simple structure. The retract **int** clearly generalizes **bool**. The range of **fun** is homeomorphic to the space of all continuous functions from $\mathbf{P}\omega$ into $\mathbf{P}\omega$; the range of **pair**, to the space of all ordered pairs; the range of **seq**, to the space of all infinite sequences. A combination like $\lambda u.\, \mathbf{int} \circ \mathbf{seq}(u)$ is a retract whose range is homeomorphic to the space of infinite sequences of elements from the range of **int**.

We now wish to introduce some operators that provide systematic ways of forming new combinations of retracts. There are three principal ones:

$$(4.8) \quad a\circ \to b = \lambda u.\, b \circ u \circ a;$$

$$(4.9) \quad a \otimes b = \lambda u.\, \langle a(u_0), b(u_1)\rangle;$$

$$(4.10) \quad a \oplus b = \lambda u.\, u_0 \supset \langle 0, a(u_1)\rangle,\, \langle 1, b(u_1)\rangle.$$

These equations clearly generalize (4.1)–(4.3). Before we explain our operators, note these three equations which hold for arbitrary a, b, a', b':

$$(4.11) \quad (a\circ \to b) \circ (a'\circ \to b') = (a' \circ a)\circ \to (b \circ b');$$

$$(4.12) \quad (a \otimes b) \circ (a' \otimes b') = (a \circ a') \otimes (b \circ b');$$

$$(4.13) \quad (a \oplus b) \circ (a' \oplus b') = (a \circ a') \oplus (b \circ b').$$

The reversal of order $(a' \circ a)$ on the right-hand side of (4.11) should be remarked.

These equations will be used not only for properties of types (ranges of retracts) but also for the mappings between the types.

Proof of Theorem 4.2. The relation $\sqsubseteq$ is by the definition of retract reflexive on $\mathbf{RET}$; it is also obviously antisymmetric. To prove transitivity, suppose $a \sqsubseteq b \sqsubseteq c$, then

$$a = a \circ b = a \circ b \circ c = a \circ c.$$

Similarly, $a = c \circ a$. Note, by the way, that $a \sqsubseteq b$ implies that the range of a is included in that of b; but that the relationship $a \subseteq b$ does not imply this fact. The relationship $a \sqsubseteq b$, however, is stronger than inclusion of ranges.

theorem_4_2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.retractLe

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_4_2 {a b c : Pomega} :
    (Retract a → retractLe a a) ∧
      (retractLe a b → retractLe b a → a = b) ∧
      (retractLe a b → retractLe b c → retractLe a c) := by
  refine ⟨retractLe.refl, ?_, ?_⟩
  · intro hab hba
    exact hab.1.trans (hba.2.symm)
  · intro hab hbc
    exact retractLe.trans hab hbc

end DomainTheory.DataTypesAsLattices
```

3.5.12 Theorem 4.3 ✓

THEOREM 4.3 (The function space theorem). Suppose a, b, a', b', c are retracts. Then we have:

(i) $a\circ \to b$ is a retract, and it is strict if b is;

(ii) $u : a\circ \to b$ iff $u = \lambda x : a.\, u(x)$ and $\forall x : a.\, u(x) : b$;

(iii) if $a \sqsubseteq a'$ and $b \sqsubseteq b'$, then $a\circ \to b \sqsubseteq a'\circ \to b'$;

(iv) if $f : a\circ \to b$ and $f' : a'\circ \to b'$, then $f\circ \to f' : (b\circ \to a')\circ \to (a\circ \to b')$;

(v) if $f : a\circ \to b$ and $f' : b\circ \to c$, then $f' \circ f : a\circ \to c$.

Parts (i), (iii), (iv), and (v) can be proved using (4.8) and (4.11) in an algebraic (formal) fashion. It is (ii) that tells us what it all means: the range of $a\circ \to b$ consists exactly of those functions which are restricted to (the range of) a and which have values in b. So we can read $u : a\circ \to b$ in the normal way: u is a (continuous) mapping from a into b. In technical jargon, we can say that the (strict) retracts and continuous functions form a *category*. In fact, it is equivalent to the category of separable continuous lattices and continuous maps. In this context, (iv) shows that $\circ \to$ operates not only on spaces (retracts) but also on

maps: it is a functor contravariant in the first argument and covariant in the second. Further categorical properties will emerge.

Proofs of Theorems 4.3–4.5. We will not give full details as all the parts of these theorems are direct calculations. Consider by way of example Theorem 4.3(i). We find:

$$\begin{aligned}(a\circ\to b)\circ(a\circ\to b) &= \lambda u.\, b\circ(b\circ u\circ a)\circ a\\ &= \lambda u.\, b\circ u\circ a\\ &= a\circ\to b,\end{aligned}$$

provided that a and b are retracts. A very similar computation would verify part (iv), if one writes out the composition:

$$(a\circ\to b')\circ(f\circ\to f')\circ(b\circ\to a')$$

and uses the equations:

$$f = b\circ f\circ a \quad\text{and}\quad f' = b'\circ f'\circ a'.$$

theorem_4_3.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more
import DomainTheory.DataTypesAsLattices.Retracts.typed_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_4_3 {a b a' b' c : Pomega}
    (ha : Retract a) (hb : Retract b) (_ha' : Retract a')
    (_hb' : Retract b') (hc : Retract c) :
    Retract (arrowR a b) ∧
      (IsStrict b → IsStrict (arrowR a b)) ∧
      (∀ u, typed u (arrowR a b) ↔
        u = comp u a ∧ ∀ x, typed x a → typed (funOf u x) b) ∧
      (retractLe a a' → retractLe b b' →
        retractLe (arrowR a b) (arrowR a' b')) ∧
      (∀ f f', typed f (arrowR a b) → typed f' (arrowR a' b') →
        typed (arrowR f f') (arrowR (arrowR b a') (arrowR a b'))) ∧
      (∀ f f', typed f (arrowR a b) → typed f' (arrowR b c) →
        typed (comp f' f) (arrowR a c)) :=
  ⟨arrowR.isRetract ha hb, arrowR.isStrict, fun _ => typed.arrowR_iff ha hb,
    arrowR.mono, fun _ _ => arrowR.functor,
    fun _ _ hf hf' => typed.comp ha hb hc hf hf'⟩

end DomainTheory.DataTypesAsLattices
```

3.5.13 C combinators ✓

Sum and product combinators for retracts: tag tests, injections, diagonal, curry, and evaluation.

Definition (`whichC`). $\mathrm{whichC} = \mathrm{fstC}$.

Theorem (`whichC.app`). $\mathrm{whichC}\,u = u(0)$.

Proof. The first-projection law. ■

Definition (`outC`). $\mathrm{outC} = \mathrm{sndC}$.

Definition (`outleftC`). $\mathrm{outleftC}$ is the graph of $u \mapsto \mathrm{cond}(u(0),\, u(1),\, \bot)$.

Theorem (`outleftC.isScottContinuous`). That body map is Scott-continuous.

Proof. Continuity of the conditional in its first two arguments along continuous projections. ■

Theorem (`outleftC.app`). Applying $\mathrm{outleftC}$ recovers the conditional body.

Proof. The β-law for the continuous body. ■

Definition (`outrightC`). $\mathrm{outrightC}$ is the graph of $u \mapsto \mathrm{cond}(u(0),\, \bot,\, u(1))$.

Theorem (`outrightC.isScottContinuous`). That body map is Scott-continuous.

Proof. Continuity of the conditional in its first and third arguments along continuous projections. ■

Theorem (`outrightC.app`). Applying $\mathrm{outrightC}$ recovers the conditional body.

Proof. The β-law for the continuous body. ■

Definition (`inrightC`). $\mathrm{inrightC}$ is the graph of $x \mapsto \langle 1, x\rangle$.

Theorem (`inrightC_app`). $\mathrm{inrightC}\,x = \langle 1, x\rangle$.

Proof. The β-law for continuous right pairing at tag 1. ■

Definition (`inleftC`). $\mathrm{inleftC}$ is the graph of $x \mapsto \langle 0, x\rangle$.

Theorem (`inleftC.app`). $\mathrm{inleftC}\,x = \langle 0, x\rangle$.

Proof. The β-law for continuous right pairing at tag 0. ■

Theorem (`curryC_isScottContinuous`). The map $u \mapsto \mathrm{graph}\big(x \mapsto \mathrm{graph}(y \mapsto u(\langle x, y\rangle))\big)$ is Scott-continuous.

Proof. Nested graph-constant continuity using left-continuity of application at a paired argument. ■

Theorem (`curryC_app`). $\mathrm{curryC}\,u = \mathrm{graph}\big(x \mapsto \mathrm{graph}(y \mapsto u(\langle x, y\rangle))\big)$.

Proof. The β-law for that continuous outer map. ■

Theorem (`curry_mid_isScottContinuous`). For fixed u, the mid map $x \mapsto \mathrm{graph}(y \mapsto u(\langle x, y\rangle))$ is Scott-continuous.

Proof. Graph-constant continuity from continuous application of u along continuous left pairing. ■

Theorem (`diagC_app`). $\mathrm{diagC}\,u = \langle u, u\rangle$.

Proof. The β-law for the continuous diagonal of pairing. ■

Theorem (`evalC_isScottContinuous`). The map $u \mapsto u(0)\,(u(1))$ is Scott-continuous.

Proof. Continuity of application in both arguments along the two projections. ■

Theorem (`evalC_app`). $\mathrm{evalC}\,u = u(0)\,(u(1))$.

Proof. The β-law for that continuous body. ■

`CombC.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.curryC
public import DomainTheory.DataTypesAsLattices.Retracts.diagC
public import DomainTheory.DataTypesAsLattices.Retracts.evalC
public import DomainTheory.DataTypesAsLattices.Retracts.fstC
public import DomainTheory.DataTypesAsLattices.Retracts.sndC
```

```
import DomainTheory.DataTypesAsLattices.Enumeration.PairSeq
import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
import DomainTheory.DataTypesAsLattices.Graph.beta
import DomainTheory.DataTypesAsLattices.Enumeration.ContinuityHelpers
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def whichC : Pomega := fstC

namespace whichC

theorem app (u : Pomega) : funOf whichC u = funOf u (ofNat 0) :=
  fstC.app u

end whichC

def outC : Pomega := sndC

def outleftC : Pomega :=
  graph (fun u => condSet (funOf u (ofNat 0)) (funOf u (ofNat 1)) botElem)

namespace outleftC

theorem isScottContinuous :
    IsScottContinuous (fun u =>
      condSet (funOf u (ofNat 0)) (funOf u (ofNat 1)) botElem) :=
  ContinuityHelpers.tuple
    (fun y => condSet.isScottContinuous_left y botElem)
    (fun tes => condSet.isScottContinuous_mid tes botElem)
    (funOf.isScottContinuous_left (ofNat 0))
    (funOf.isScottContinuous_left (ofNat 1))

theorem app (u : Pomega) :
    funOf outleftC u = condSet (funOf u (ofNat 0)) (funOf u (ofNat 1)) botElem :=
  beta outleftC.isScottContinuous u

end outleftC

def outrightC : Pomega :=
  graph (fun u => condSet (funOf u (ofNat 0)) botElem (funOf u (ofNat 1)))

namespace outrightC

theorem isScottContinuous :
    IsScottContinuous (fun u =>
      condSet (funOf u (ofNat 0)) botElem (funOf u (ofNat 1))) :=
  ContinuityHelpers.tuple
    (fun y => condSet.isScottContinuous_left botElem y)
    (fun tes => condSet.isScottContinuous_right tes botElem)
    (funOf.isScottContinuous_left (ofNat 0))
    (funOf.isScottContinuous_left (ofNat 1))

theorem app (u : Pomega) :
```

```
    funOf outrightC u = condSet (funOf u (ofNat 0)) botElem (funOf u (ofNat 1)) :=
  beta outrightC.isScottContinuous u

end outrightC

def inrightC : Pomega := graph (fun x => pairSeq (ofNat 1) x)

theorem inrightC_app (x : Pomega) :
    funOf inrightC x = pairSeq (ofNat 1) x :=
  beta (PairSeq.isScottContinuous_right (ofNat 1)) x

def inleftC : Pomega := graph (fun x => pairSeq (ofNat 0) x)

namespace inleftC

theorem app (x : Pomega) :
    funOf inleftC x = pairSeq (ofNat 0) x :=
  beta (PairSeq.isScottContinuous_right (ofNat 0)) x

end inleftC

theorem curryC_isScottContinuous :
    IsScottContinuous (fun u =>
      graph (fun x => graph (fun y => funOf u (pairSeq x y)))) :=
  graph_const_isScottContinuous
    (fun u x => graph (fun y => funOf u (pairSeq x y)))
    (fun x =>
      graph_const_isScottContinuous
        (fun u y => funOf u (pairSeq x y))
        (fun y => funOf.isScottContinuous_left (pairSeq x y)))

theorem curryC_app (u : Pomega) :
    funOf curryC u = graph (fun x => graph (fun y => funOf u (pairSeq x y))) :=
  beta curryC_isScottContinuous u

theorem curry_mid_isScottContinuous (u : Pomega) :
    IsScottContinuous (fun x => graph (fun y => funOf u (pairSeq x y))) :=
  graph_const_isScottContinuous
    (fun x y => funOf u (pairSeq x y))
    (fun y => theorem_1_3 (funOf.isScottContinuous u)
      (PairSeq.isScottContinuous_left y))

theorem diagC_app (u : Pomega) : funOf diagC u = pairSeq u u :=
  beta (ContinuityHelpers.on_diag
    PairSeq.isScottContinuous_left PairSeq.isScottContinuous_right) u

theorem evalC_isScottContinuous :
    IsScottContinuous (fun u => funOf (funOf u (ofNat 0)) (funOf u (ofNat 1))) :=
  ContinuityHelpers.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous_left (ofNat 0))
    (funOf.isScottContinuous_left (ofNat 1))

theorem evalC_app (u : Pomega) :
    funOf evalC u = funOf (funOf u (ofNat 0)) (funOf u (ofNat 1)) :=
```

```
  beta evalC_isScottContinuous u

end DomainTheory.DataTypesAsLattices
```

3.5.14 Pair mediation ✓

The mediating arrow into a tensor product of retracts.

Definition (`pairMed`). For maps f, g and a retract c,

$$\mathrm{pairMed}(f, g, c) = (f \otimes g) \circ \mathrm{diagC} \circ c.$$

That is the composite of c with the diagonal into $c \otimes c$, then the tensor of f and g.

`pairMed.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.diagC

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def pairMed (f g c : Pomega) : Pomega :=
  comp (tensorR f g) (comp diagC c)

end DomainTheory.DataTypesAsLattices
```

3.5.15 Theorem 4.4 ✓

THEOREM 4.4 (The product theorem). Suppose a, b, a', b' are retracts. Then we have:

(i) $a \otimes b$ is a retract, and it is strict if a and b are;

(ii) $u : a \otimes b$ iff $u = (u_0, u_1)$ and $u_0 : a$ and $u_1 : b$;

(iii) if $a \sqsubseteq a'$ and $b \sqsubseteq b'$, then $a \otimes b \sqsubseteq a' \otimes b'$;

(iv) if $f : a\circ \to b$ and $f' : a'\circ \to b'$, then $f \otimes f' : a \otimes a'\circ \to b \otimes b'$.

Again the operator proves to be a functor, but what is stated in Theorem 4.4 is not quite enough for the standard identification of $\otimes$ as the categorical product. For this we need some additional combinators:

$$\mathbf{fst} = \lambda u.\, u_0; \tag{4.14}$$

$$\mathbf{snd} = \lambda u.\, u_1; \tag{4.15}$$

$$\mathbf{diag} = \lambda u.\, \langle u, u \rangle. \tag{4.16}$$

Then we have these properties:

$$(4.17) \quad \mathbf{fst} \circ (a \otimes b) : (a \otimes b)\circ \to a;$$

$$(4.18) \quad \mathbf{snd} \circ (a \otimes b) : (a \otimes b)\circ \to b;$$

$$(4.19) \quad \mathbf{diag} \circ a : a\circ \to a \otimes a;$$

$$(4.20) \quad \mathbf{fst} \circ (f \otimes f') = f \circ \mathbf{fst};$$

$$(4.21) \quad \mathbf{snd} \circ (f \otimes f') = f' \circ \mathbf{snd}.$$

Here a and b are retracts and f and f' are functions. Now suppose a, b, and c are retracts and $f : c\circ \to a$ and $g : c\circ \to b$. Let

$$h = (f \otimes g) \circ \mathbf{diag} \circ c.$$

We can readily prove that:

$$h : c\circ \to (a \otimes b),$$

and

$$\mathbf{fst} \circ h = f \quad \text{and} \quad \mathbf{snd} \circ h = g.$$

Furthermore, h is the *unique* such function. It is this uniqueness and existence property of functions into $a \otimes b$ that identifies the construct as a product.

There are important connections between $\circ \to$ and $\otimes$. To state these we require some additional combinators:

$$(4.22) \quad \mathbf{eval} = \lambda u.\, u_0(u_1);$$

$$(4.23) \quad \mathbf{curry} = \lambda u \lambda x \lambda y.\, u(\langle x, y\rangle).$$

If a, b, and c are retracts, the mapping properties are:

$$(4.24) \quad \mathbf{eval} \circ ((b\circ \to c) \otimes b) : ((b\circ \to c) \otimes b)\circ \to c;$$

$$(4.25) \quad \mathbf{curry} \circ ((a \otimes b)\circ \to c) : ((a \otimes b)\circ \to c)\circ \to (a\circ \to (b\circ \to c)).$$

Suppose next that $f : (a \otimes b)\circ \to c$ and $g : a\circ \to (b\circ \to c)$. We find that

$$\mathbf{eval} \circ (\mathbf{curry}(f) \otimes b) = f$$

and

$$\mathbf{curry}(\mathbf{eval} \circ (g \otimes b)) = g.$$

This shows that our category of retracts is a *Cartesian closed category*, which means roughly that product spaces and function spaces within the category interact harmoniously.

Formalization. The facts below package pairing projections, Scott's membership clause (ii) for $\otimes$, and the mediating existence/uniqueness that make $\otimes$ a categorical product. Write $h = (f \otimes g) \circ \mathbf{diag} \circ c$ for the mediating composite of the prior peer; the combinators $\mathbf{fst}$, $\mathbf{snd}$, $\mathbf{diag}$ are as in (4.14)–(4.16). Clauses (i), (iii), (iv) and the numbered equations (4.17)–(4.25) are recorded elsewhere.

Theorem (`theorem_4_4.ii`). Pairing recovers its components: $\langle x, y\rangle_0 = x$ and $\langle x, y\rangle_1 = y$.

Proof. The two coordinates of a length-2 sequence. ■

Theorem (`theorem_4_4.typed_iff`). $u : a \otimes b$ if and only if $u = \langle u_0, u_1\rangle$ with $u_0 : a$ and $u_1 : b$.

Proof. ($\Rightarrow$) Fixed-pointness under $a \otimes b$ rewrites u as $\langle a(u_0), b(u_1)\rangle$; projecting both sides yields $u_i = a(u_i)$ (resp. b), and substituting those equalities back gives the displayed pairing. ($\Leftarrow$) From $u = \langle u_0, u_1\rangle$ and the two typings, $(a \otimes b)(u) = \langle a(u_0), b(u_1)\rangle = \langle u_0, u_1\rangle = u$. ■

Theorem (`theorem_4_4.med`). If $f : c\circ \to a$ and $g : c\circ \to b$, then $h : c\circ \to (a \otimes b)$, $\mathbf{fst} \circ h = f$, and $\mathbf{snd} \circ h = g$.

Proof. Pointwise, $h(x) = \langle f(c(x)), g(c(x))\rangle$; applying $a \otimes b$ (with c a retract and the typings of f and g) returns the same pair, so h is typed into the product arrow. Composing with $\mathbf{fst}$ (resp. $\mathbf{snd}$) peels the first (resp. second) coordinate and uses that f (resp. g) already restricts along c. ■

Theorem (`theorem_4_4.med_unique`). Any $h' : c\circ \to (a \otimes b)$ with $\mathbf{fst} \circ h' = f$ and $\mathbf{snd} \circ h' = g$ equals h.

Proof. At each x, typing into $a \otimes b$ forces $h'(x) = \langle a(h'(c(x))_0), b(h'(c(x))_1)\rangle$; the projection hypotheses identify those coordinates with $f(x)$ and $g(x)$, matching $(f \otimes g)(\mathbf{diag}(c(x)))$. ■

Theorem (`theorem_4_4`). Pairing projections hold for all x, y, and $\bot$ is a retract.

Proof. The projection identity just proved, together with the known retract $\bot$. ■

`theorem_4_4.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.fstC
public import DomainTheory.DataTypesAsLattices.Retracts.sndC
public import DomainTheory.DataTypesAsLattices.Functionality.pairMed
import DomainTheory.DataTypesAsLattices.Graph.funOf_more
import DomainTheory.DataTypesAsLattices.Functionality.CombC
import DomainTheory.DataTypesAsLattices.Enumeration.PairSeq
import DomainTheory.DataTypesAsLattices.Retracts.typed_more
import DomainTheory.DataTypesAsLattices.Retracts.bot_isRetract
import DomainTheory.DataTypesAsLattices.Retracts.retract_app
import DomainTheory.DataTypesAsLattices.Retracts.graphLemmas

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace theorem_4_4

theorem ii (x y : Pomega) :
    funOf (pairSeq x y) (ofNat 0) = x ∧
      funOf (pairSeq x y) (ofNat 1) = y :=
  ⟨seq2.zero x y, seq2.one x y⟩
```

```
theorem typed_iff {a b u : Pomega} :
    typed u (tensorR a b) ↔
      u = pairSeq (funOf u (ofNat 0)) (funOf u (ofNat 1)) ∧
        typed (funOf u (ofNat 0)) a ∧ typed (funOf u (ofNat 1)) b := by
  constructor
  · intro hu
    have hpair : u =
        pairSeq (funOf a (funOf u (ofNat 0)))
          (funOf b (funOf u (ofNat 1))) := by
      simpa [typed, tensorR.app] using hu
    have h0 : funOf u (ofNat 0) = funOf a (funOf u (ofNat 0)) := by
      have := congrArg (fun w => funOf w (ofNat 0)) hpair
      simpa [PairSeq.app_zero] using this
    have h1 : funOf u (ofNat 1) = funOf b (funOf u (ofNat 1)) := by
      have := congrArg (fun w => funOf w (ofNat 1)) hpair
      simpa [PairSeq.app_one] using this
    refine ⟨hpair.trans (congr (congrArg pairSeq h0.symm) h1.symm), h0, h1⟩
  · intro ⟨hpair, ha, hb⟩
    change u = funOf (tensorR a b) u
    rw [tensorR.app, ← ha, ← hb, ← hpair]

theorem med {a b c f g : Pomega}
    (_ha : Retract a) (_hb : Retract b) (hc : Retract c)
    (hf : typed f (arrowR c a)) (hg : typed g (arrowR c b)) :
    typed (pairMed f g c) (arrowR c (tensorR a b)) ∧
      comp fstC (pairMed f g c) = f ∧
      comp sndC (pairMed f g c) = g := by
  have hf' := arrowR.typed_of hf
  have hg' := arrowR.typed_of hg
  refine ⟨?_, ?_, ?_⟩
  · change pairMed f g c =
        funOf (arrowR c (tensorR a b)) (pairMed f g c)
    rw [arrowR.app, pairMed]
    apply graph_ext
    intro x
    have hfxc : funOf f (funOf c x) =
        funOf a (funOf f (funOf c x)) := by
      have := congrArg (fun w => funOf w (funOf c x)) hf'
      simpa [comp.app, retract_app hc] using this
    have hgxc : funOf g (funOf c x) =
        funOf b (funOf g (funOf c x)) := by
      have := congrArg (fun w => funOf w (funOf c x)) hg'
      simpa [comp.app, retract_app hc] using this
    have hL : funOf (tensorR f g) (funOf diagC (funOf c x)) =
        pairSeq (funOf f (funOf c x)) (funOf g (funOf c x)) := by
      rw [diagC_app, tensorR.app, PairSeq.app_zero, PairSeq.app_one]
    have hR : funOf (tensorR a b)
        (funOf (tensorR f g) (funOf diagC (funOf c (funOf c x)))) =
        pairSeq (funOf a (funOf f (funOf c x)))
          (funOf b (funOf g (funOf c x))) := by
      rw [retract_app hc, diagC_app]
      have hinter :
          funOf (tensorR f g) (pairSeq (funOf c x) (funOf c x)) =
            pairSeq (funOf f (funOf c x)) (funOf g (funOf c x)) := by
        rw [tensorR.app, PairSeq.app_zero, PairSeq.app_one]
      rw [hinter, tensorR.app, PairSeq.app_zero, PairSeq.app_one]
```

```
    simp only [comp.app]
    rw [hL, hR]
    exact congr (congrArg pairSeq hfxc) hgxc
  · apply funOf.ext_graph (comp.isGraph _ _) (typed.isGraph hf)
    intro x
    simp only [comp.app, pairMed, fstC.app, diagC_app]
    rw [tensorR.app, PairSeq.app_zero, PairSeq.app_zero]
    have := congrArg (fun w => funOf w x) (typed.arrowR_restrict hc hf)
    simpa [comp.app] using this.symm
  · apply funOf.ext_graph (comp.isGraph _ _) (typed.isGraph hg)
    intro x
    simp only [comp.app, pairMed, sndC.app, diagC_app]
    rw [tensorR.app, PairSeq.app_one, PairSeq.app_one]
    have := congrArg (fun w => funOf w x) (typed.arrowR_restrict hc hg)
    simpa [comp.app] using this.symm

theorem med_unique {a b c f g h : Pomega}
    (ha : Retract a) (hb : Retract b) (hc : Retract c)
    (hf : typed f (arrowR c a)) (hg : typed g (arrowR c b))
    (hh : typed h (arrowR c (tensorR a b)))
    (hfst : comp fstC h = f) (hsnd : comp sndC h = g) :
    h = pairMed f g c := by
  apply funOf.ext_graph (typed.isGraph hh)
    (typed.isGraph (med ha hb hc hf hg).1)
  intro x
  have hh' := arrowR.typed_of hh
  have hx : funOf h x =
      funOf (tensorR a b) (funOf h (funOf c x)) := by
    have := congrArg (fun w => funOf w x) hh'
    simpa [comp.app] using this
  have h0 : funOf (funOf h x) (ofNat 0) = funOf f x := by
    have := congrArg (fun w => funOf w x) hfst
    simpa [comp.app, fstC.app] using this
  have h1 : funOf (funOf h x) (ofNat 1) = funOf g x := by
    have := congrArg (fun w => funOf w x) hsnd
    simpa [comp.app, sndC.app] using this
  rw [hx, tensorR.app]
  have hfx : funOf a (funOf (funOf h (funOf c x)) (ofNat 0)) = funOf f x := by
    have h0c : funOf (funOf h (funOf c x)) (ofNat 0) = funOf f (funOf c x) := by
      have := congrArg (fun w => funOf w (funOf c x)) hfst
      simpa [comp.app, fstC.app] using this
    rw [h0c]
    have := congrArg (fun w => funOf w x) (arrowR.typed_of hf)
    simpa [comp.app] using this.symm
  have hgx : funOf b (funOf (funOf h (funOf c x)) (ofNat 1)) = funOf g x := by
    have h1c : funOf (funOf h (funOf c x)) (ofNat 1) = funOf g (funOf c x) := by
      have := congrArg (fun w => funOf w (funOf c x)) hsnd
      simpa [comp.app, sndC.app] using this
    rw [h1c]
    have := congrArg (fun w => funOf w x) (arrowR.typed_of hg)
    simpa [comp.app] using this.symm
  rw [hfx, hgx]
  simp only [pairMed, comp.app, diagC_app, tensorR.app, PairSeq.app_zero,
    PairSeq.app_one]
  have hfr : funOf f (funOf c x) = funOf f x := by
    have := congrArg (fun w => funOf w x) (typed.arrowR_restrict hc hf)
```

```
    simpa [comp.app] using this.symm
  have hgr : funOf g (funOf c x) = funOf g x := by
    have := congrArg (fun w => funOf w x) (typed.arrowR_restrict hc hg)
    simpa [comp.app] using this.symm
  rw [hfr, hgr]

end theorem_4_4

theorem theorem_4_4 :
    (∀ x y, funOf (pairSeq x y) (ofNat 0) = x ∧
      funOf (pairSeq x y) (ofNat 1) = y) ∧
      Retract botElem :=
  ⟨fun x y => theorem_4_4.ii x y, bot_isRetract⟩

end DomainTheory.DataTypesAsLattices
```

3.5.16 Theorem 4.5 ✓

THEOREM 4.5 (The sum theorem). Suppose a, b, a', b' are retracts. Then we have:

(i) $a \oplus b$ is a retract, and it is always strict;

(ii) $u : a \oplus b$ iff $u = \bot$ or $u = \top$ or ($u = \langle 0, u_1 \rangle$ and $u_1 : a$) or ($u = \langle 1, u_1 \rangle$ and $u_1 : b$);

(iii) if $a \sqsubseteq a'$ and $b \sqsubseteq b'$, then $a \oplus b \sqsubseteq a' \oplus b'$;

(iv) if $f : a\circ \to b$ and $f' : a'\circ \to b'$, then $f \oplus f' : a \oplus a'\circ \to b \oplus b'$.

There are several combinators associated with $\oplus$:

$$(4.26) \quad \mathbf{inleft} = \lambda x.\, \langle 0, x \rangle;$$

$$(4.27) \quad \mathbf{inright} = \lambda x.\, \langle 1, x \rangle;$$

$$(4.28) \quad \mathbf{outleft} = \lambda u.\, u_0 \supset u_1,\, \bot;$$

$$(4.29) \quad \mathbf{outright} = \lambda u.\, u_0 \supset \bot,\, u_1;$$

$$(4.30) \quad \mathbf{which} = \lambda u.\, u_0;$$

$$(4.31) \quad \mathbf{out} = \lambda u.\, u_1.$$

(The last two are the same as **fst** and **snd**, but they will be used differently.) We find:

$$(4.32) \quad (a \oplus b) \circ \mathbf{inleft} \circ a : a\circ \to (a \oplus b);$$

$$(4.33) \quad (a \oplus b) \circ \mathbf{inright} \circ b : b\circ \to (a \oplus b);$$

$$(4.34) \quad a \circ \mathbf{outleft} \circ (a \oplus b) : (a \oplus b)\circ \to a;$$

$$(4.35) \quad b \circ \mathbf{outright} \circ (a \oplus b) : (a \oplus b)\circ \to b;$$

$$(4.36) \quad \mathbf{which} \circ (a \oplus b) : (a \oplus b)\circ \to \mathbf{bool};$$

$$(4.37) \quad a \circ \mathbf{out} \circ (a \oplus a) : (a \oplus a)\circ \to a;$$

where a and b are retracts. Most of these facts as they stand are trivial until one sets down the relations between all these maps; but there are too many to put them down here. Note, however, if a, b, and c are retracts and $f : a\circ \to c$ and $g : b\circ \to c$, then if we let

$$h = c \circ \mathbf{out} \circ (f \oplus g),$$

we have:

$$h : (a \oplus b)\circ \to c,$$

and

$$h \circ \mathbf{inleft} = f \quad \text{and} \quad h \circ \mathbf{inright} = g.$$

But, though h exists, it is not unique. So $a \oplus b$ is not the categorical sum (coproduct). The author does not know a neat categorical characterization of this operator.

There would be no difficulty in extending $\otimes$ and $\oplus$ to more factors by expanding the range of indices from $0, 1$ to $0, 1, \ldots, n-1$. The explicit formulae need not be given; but if we write $a_0 \otimes a_1 \otimes \cdots \otimes a_{n-1}$, we intend this expanded meaning rather than the iterated binary product.

To understand sums and other facts about retracts, consider the least fixed point of this equation:

$$(4.38) \quad \mathbf{tree} = \mathbf{nil} \oplus (\mathbf{tree} \otimes \mathbf{tree}).$$

To be certain that **tree** is a retract, we need a general theorem:

Proofs of Theorems 4.3–4.5. We will not give full details as all the parts of these theorems are direct calculations. Consider by way of example Theorem 4.3(i). We find:

$$\begin{aligned}(a\circ \to b) \circ (a\circ \to b) &= \lambda u.\, b \circ (b \circ u \circ a) \circ a \\ &= \lambda u.\, b \circ u \circ a \\ &= a\circ \to b,\end{aligned}$$

provided that a and b are retracts. A very similar computation would verify part (iv), if one writes out the composition:

$$(a\circ \to b') \circ (f\circ \to f') \circ (b\circ \to a')$$

and uses the equations:

$$f = b \circ f \circ a \quad \text{and} \quad f' = b' \circ f' \circ a'.$$

Formalization. The Lean owned by this note is a thin packaging of the $\bot$-case that seeds strictness of $\oplus$. The sum operator itself is

$$a \oplus b = \mathbf{graph}\big(u \mapsto u_0 \sqsupset \langle 0,\, a(u_1)\rangle,\, \langle 1,\, b(u_1)\rangle\big)$$

(using the doubly strict conditional of (4.4)). The full packaging of Scott's (i)–(ii) lives in `theorem_4_5_sum` (Functionality); clauses (iii)–(iv) and (4.32)–(4.37) are companion lemmas.

Theorem (`theorem_4_5`). For any a, b,

$$\bot \sqsupset \langle 0,\, a(\bot)\rangle,\, \langle 1,\, b(\bot)\rangle \;=\; \bot,$$

and $\bot$ is a retract.

Proof. The first conjunct is the empty-tag law for $\sqsupset$: a pure bottom discriminator yields $\bot$ regardless of the two branches. The second is the known retract $\bot = \bot \circ \bot$. ■

`theorem_4_5.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
import DomainTheory.DataTypesAsLattices.Retracts.bot_isRetract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Lambda.Language
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_4_5 (a b : Pomega) :
    dcondSet botElem
      (pairSeq (ofNat 0) (funOf a botElem))
      (pairSeq (ofNat 1) (funOf b botElem)) = botElem ∧
      Retract botElem :=
  ⟨dcondSet.bot _ _, bot_isRetract⟩

namespace theorem_4_5

theorem sum {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    Retract (plusR a b) ∧ IsStrict (plusR a b) ∧
      (∀ u, typed u (plusR a b) ↔
        u = botElem ∨ u = topElem ∨
          (∃ x, u = pairSeq (ofNat 0) x ∧ typed x a) ∨
            (∃ y, u = pairSeq (ofNat 1) y ∧ typed y b)) :=
  ⟨plusR.isRetract ha hb, plusR.isStrict a b, fun _u => plusR.typed_iff ha hb⟩

end theorem_4_5

end DomainTheory.DataTypesAsLattices
```

3.5.17 Expression interpreter ✓

Any combination of retract preserving functors can be used in this game. For example:

$$(4.39) \quad \mathbf{lamb} = \mathbf{int} \oplus (\mathbf{lamb} \circ \to \mathbf{lamb}).$$

This looks innocent, but the range of **lamb** would give a quite different and not unattractive model for the λ-calculus (plus arithmetic). What we do to investigate this model is to modify LAMBDA slightly by replacing the ternary conditional $z \supset x, y$ by a quaternary one $w \supset x, y, z$; otherwise the syntax of the language remains the same. The semantics, however, is a little more complex.

Let us use τ, σ, ρ, θ as syntactical variables for expressions in the modified language. The semantics is provided by a function $\mathcal{H}$ that maps the expressions of the language to their values in (the range of) **lamb**. To be completely rigorous we

also have to confront the question of free and bound variables. For simplicity let us index the variables of the language by integers, and let us take the variables to be $v_0, v_1, v_2, \ldots, v_n, \ldots$. We cannot simply evaluate out an expression τ to its value $\mathcal{H}[\![\tau]\!]$ until we know the values of the free variables in τ. The values of these variables will be given by an "environment" t which can be construed as a *sequence* of values in **lamb**. We can restrict these environments to the retract:

$$(4.40) \quad \mathbf{env} = \lambda t.\, \mathbf{lamb} \circ \mathbf{seq}(t).$$

When $t : \mathbf{env}$, then $t_n : \mathbf{lamb}$ is the value that the environment gives to the variable v_n. We also need to employ a transformation on environments as follows:

$$(4.41) \quad t[x/n] = \lambda m \in \omega.\, \mathbf{eq}(n)(m) \supset x,\, t_m.$$

Here **eq** is the primitive recursive function that is 0, if n, m are equal, and is 1, otherwise, for $n, m \in \omega$. The effect of $t[x/n]$ is to replace the nth term of the sequence t by the value of x, otherwise to leave the rest of the sequence unchanged. To correspond with our use of very simple variables we have selected a simple notion of environment: in the semantics of more general languages it is customary to regard an environment as a function from the set of variables into the domain of denotable values.

The correct way to evaluate a term τ given an environment t is to find $\mathcal{H}[\![\tau]\!](t)$. We use the brackets $[\![$ and $]\!]$ here simply as an aid to the eye in keeping the syntactical part separated from the rest. The environment enters as a function-argument in the usual way; thus we shall have:

$$(4.42) \quad \mathcal{H}[\![\tau]\!] : \mathbf{env} \to \mathbf{lamb}.$$

$$\begin{aligned}
\mathcal{H}[\![v_n]\!](t) &= t_n \\
\mathcal{H}[\![0]\!](t) &= \mathbf{inleft}(0) \\
\mathcal{H}[\![\tau + 1]\!](t) &= \mathbf{which}(\mathcal{H}[\![\tau]\!](t) \supset \mathbf{inleft}(\mathbf{out}(\mathcal{H}[\![\tau]\!](t)) + 1),\, \bot) \\
\mathcal{H}[\![\tau - 1]\!](t) &= \mathbf{which}(\mathcal{H}[\![\tau]\!](t) \supset \mathbf{inleft}(\mathbf{out}(\mathcal{H}[\![\tau]\!](t)) - 1),\, \bot) \\
\mathcal{H}[\![\theta \supset \tau, \sigma, \rho]\!](t) &= \mathbf{lamb}(\mathbf{which}(\mathcal{H}[\![\theta]\!](t) \supset (\mathbf{out}(\mathcal{H}[\![\theta]\!](t)) \supset \mathcal{H}[\![\tau]\!](t),\, \mathcal{H}[\![\sigma]\!](t)),\, \mathcal{H}[\![\rho]\!](t))) \\
\mathcal{H}[\![\tau(\sigma)]\!](t) &= \mathbf{which}(\mathcal{H}[\![\tau]\!](t) \supset \bot,\, \mathbf{out}(\mathcal{H}[\![\tau]\!](t))(\mathcal{H}[\![\sigma]\!](t))) \\
\mathcal{H}[\![\lambda v_n.\, \tau]\!](t) &= \mathbf{inright}(\lambda x : \mathbf{lamb}.\, \mathcal{H}[\![\tau]\!](t[x/n])).
\end{aligned} \tag{4.43}$$

A good question is: why does $\mathcal{H}$ exist? The answer is: because of the fixed-point theorem.

If we rewrite the semantic equations $\mathcal{H}[\![\tau]\!](t) = (\dots)$ in (43) by the equation $\mathcal{H}[\![\tau]\!] = \lambda t : \mathbf{env}\,(\dots)$, then $\mathcal{H}$ is seen to be a function from expressions to values in **lamb**. As the range of **lamb** is contained in $\mathbf{P}\omega$, we can say more broadly that $\mathcal{H} \in \mathbf{P}\omega^{\mathbf{Exp}}$, where **Exp** is the syntactical set of expressions and the exponential notation designates the set of *all* functions from **Exp** into $\mathbf{P}\omega$. This function set is a complete lattice because $\mathbf{P}\omega$ is. Therefore if we read (4.43) as a definition by cases on **Exp**, then we can find $\mathcal{H}$ as a suitable fixed point in the complete lattice $\mathbf{P}\omega^{\mathbf{Exp}}$. Indeed it is the fixed point of a continuous operator.

Actually we can regard **Exp** as being a subset of $\mathbf{P}\omega$ to avoid dragging in other lattices. What we need is another recursive definition of a data type:

$$(4.44)\quad \mathbf{exp} = \mathbf{int} \oplus \mathbf{nil} \oplus \mathbf{exp} \oplus \mathbf{exp} \oplus (\mathbf{exp} \otimes \mathbf{exp} \otimes \mathbf{exp} \otimes \mathbf{exp}) \\ \oplus(\mathbf{exp} \otimes \mathbf{exp}) \oplus (\mathbf{int} \otimes \mathbf{exp}).$$

Note that there are as many summands in (4.44) as there are clauses in (4.43). We can think of **exp** as giving the "abstract" syntax of the language. We use the integers to index the variables and the **nil** element to stand for the individual constant. Read (4.44) as saying that every expression is *either* a variable *or* a constant *or* the successor of an expression *or* the predecessor of an expression *or* the conditional formed from a tuple of expressions *or* the abstraction formed from a pair of a variable and an expression. We do not need in (4.44) to introduce special "symbols" for the successor, application, etc., because the separation by cases given by the $\oplus$ operation is sufficient to make the distinctions. (That is why the syntax is "abstract".) The point is that for recursive definitions it does not matter how we make the distinctions as long as they can be made. From this new point of view, we could rewrite (4.43) so as to show:

$$(4.45)\quad \mathcal{H} : \mathbf{exp} \to (\mathbf{env} \to \mathbf{lamb}),$$

which is clearly more satisfactory—especially as it is now clear that $\mathcal{H}$ is *computable*. And this is a method that can be generalized to many other languages. The method also shows why it is useful to allow *function spaces* as particular data types.

Formalization. Abstract syntax, its encoding into the **exp** retract, the tagged continuous step clauses of the interpreter, the least fixed point packaging, and the typing identity (4.45).

Definition (`LambTerm`). An expression is a variable index, zero, a successor or predecessor of an expression, a quaternary conditional of four expressions, an application of two expressions, or an abstraction of an index and a body.

Definition (`encodeExp`). Encode each constructor by the corresponding summand of (4.44): tag 0 with a numeral, tag 1 with $\perp$, tags 2–3 with a recursive encoding, tag 4 with a four-tuple of encodings, tag 5 with a pair of encodings, and tag 6 with a numeral paired to a body encoding.

Theorem (`encodeExp.typed`). Every encoding is typed into **exp**.

Proof. Induction on syntax, using the fixed-point equation for **exp** and the typing laws for the seven summands. ■

Definition (`expTag`). The tag of an encoded expression is its application to 0.

Definition (`expPayload`). The payload of an encoded expression is its application to 1.

Definition (`hApply`). Apply an interpreter candidate H to an encoded expression e at an environment t.

Theorem (`hApply.isScottContinuous_H`). Application is Scott-continuous in the interpreter candidate.

Proof. Compose left-continuity of application twice. ■

Theorem (`hApply.isScottContinuous_e`). Application is Scott-continuous in the encoded expression.

Proof. Compose left-continuity of application with continuity of the candidate. ■

Theorem (`hApply.payload_isScottContinuous_e`). Applying H at the ith component of the payload is Scott-continuous in the encoding.

Proof. Compose the previous continuity in e with continuity of payload projection. ■

Definition (`hVar`). The variable clause looks up the payload of e in the environment t.

Theorem (`hVar.isScottContinuous_t`). The variable clause is Scott-continuous in the environment.

Proof. Left-continuity of application at a fixed payload. ■

Theorem (`hVar.isScottContinuous_e`). The variable clause is Scott-continuous in the encoding.

Proof. Compose continuity of the environment with continuity of the payload. ■

Definition (`hZero`). The zero clause is the left injection of the numeral 0.

Definition (`hApp`). The application clause tests the tag of the value of the operator payload: on a right injection it applies the extracted function to the value of the operand payload; otherwise it yields $\bot$.

Theorem (`hApp.isScottContinuous_H`). The application clause is Scott-continuous in the interpreter candidate.

Proof. Continuity of the ternary conditional along continuous tag tests and continuous applications of H. ■

Theorem (`hApp.isScottContinuous_t`). The application clause is Scott-continuous in the environment.

Proof. Continuity of the conditional along continuous evaluation of the two payloads. ■

Theorem (`hApp.isScottContinuous_e`). The application clause is Scott-continuous in the encoding.

Proof. Continuity of the conditional along continuous payload projections. ■

Definition (`hSucc`). The successor clause tests the tag of the recursive value: on a left injection it re-injects the successor of the payload; otherwise it yields $\bot$.

Theorem (`hSucc.isScottContinuous_H`). The successor clause is Scott-continuous in the interpreter candidate.

Proof. Continuity of the conditional along continuous application of H, extraction, and successor. ■

Theorem (`hSucc.isScottContinuous_t`). The successor clause is Scott-continuous in the environment.

Proof. Continuity of the conditional along continuous evaluation of the payload. ■

Theorem (`hSucc.isScottContinuous_e`). The successor clause is Scott-continuous in the encoding.

Proof. Continuity of the conditional along continuous payload projection. ■

Definition (`hPred`). The predecessor clause is as for successor, with predecessor in place of successor.

Theorem (`hPred.isScottContinuous_H`). The predecessor clause is Scott-continuous in the interpreter candidate.

Proof. As for successor, with continuity of predecessor. ■

Theorem (`hPred.isScottContinuous_t`). The predecessor clause is Scott-continuous in the environment.

Proof. As for successor. ■

Theorem (`hPred.isScottContinuous_e`). The predecessor clause is Scott-continuous in the encoding.

Proof. As for successor. ■

Definition (`hCond`). The quaternary conditional clause evaluates the four payload components and selects via the **lamb**-wrapped nested conditional of (4.43).

Theorem (`hCond.isScottContinuous_H`). The conditional clause is Scott-continuous in the interpreter candidate.

Proof. Continuity of application of **lamb** along continuous nested conditionals of continuous applications of H. ■

Theorem (`hCond.isScottContinuous_t`). The conditional clause is Scott-continuous in the environment.

Proof. Continuity of **lamb**-application along continuous evaluation of the four payloads. ■

Theorem (`hCond.isScottContinuous_e`). The conditional clause is Scott-continuous in the encoding.

Proof. Continuity of **lamb**-application along continuous payload projections. ■

Definition (`hLam`). The abstraction clause right-injects the graph of $x \mapsto H$ applied to the body payload at the environment updated at the bound index by x.

Theorem (`hLam.isScottContinuous_H`). The abstraction clause is Scott-continuous in the interpreter candidate.

Proof. Continuity of right injection of a graph whose body is continuous in H. ■

Theorem (`hLam.isScottContinuous_t`). The abstraction clause is Scott-continuous in the environment.

Proof. Continuity of right injection of a graph continuous in the updated environment. ■

Theorem (`hLam.isScottContinuous_e`). The abstraction clause is Scott-continuous in the encoding.

Proof. Continuity of right injection along continuous payload projections into the update. ■

Definition (`Hstep`). The tagged step nests doubly strict conditionals on iterated predecessors of the tag, dispatching to the variable, zero, successor, predecessor, conditional, application, and abstraction clauses, with $\top$ as the residual.

Theorem (`Hstep.tag0`). On a tag-0 injection with payload x, the step returns $t(x)$.

Proof. Unfold the tag cascade at numeral 0. ■

Theorem (`Hstep.tag1`). On a tag-1 injection, the step returns the zero clause.

Proof. Unfold the tag cascade at numeral 1. ■

Theorem (`Hstep.tag2`). On a tag-2 injection, the step returns the successor clause at that encoding.

Proof. Unfold the tag cascade at numeral 2. ■

Theorem (`Hstep.tag3`). On a tag-3 injection, the step returns the predecessor clause at that encoding.

Proof. Unfold the tag cascade at numeral 3. ■

Theorem (`Hstep.tag4`). On a tag-4 injection, the step returns the conditional clause at that encoding.

Proof. Unfold the tag cascade at numeral 4. ■

Theorem (`Hstep.tag5`). On a tag-5 injection, the step returns the application clause at that encoding.

Proof. Unfold the tag cascade at numeral 5. ■

Theorem (`Hstep.tag6`). On a tag-6 injection, the step returns the abstraction clause at that encoding.

Proof. Unfold the tag cascade at numeral 6. ■

Theorem (`Hstep.isScottContinuous_H`). The step is Scott-continuous in the interpreter candidate.

Proof. Nest continuity of the doubly strict conditional along continuous clauses in H. ■

Theorem (`Hstep.isScottContinuous_t`). The step is Scott-continuous in the environment.

Proof. Nest continuity of the doubly strict conditional along continuous clauses in t. ■

Theorem (`Hstep.isScottContinuous_e`). The step is Scott-continuous in the encoding.

Proof. Nest continuity of the doubly strict conditional along continuous tag maps and continuous clauses in e. ■

Definition (`HF`). The continuous operator on interpreter candidates sends H to the graph of $e \mapsto$ the graph of $t \mapsto \mathrm{Hstep}(H, e, t)$.

Theorem (`HF.inner_isScottContinuous`). For fixed H, the map $e \mapsto$ the graph of $t \mapsto \mathrm{Hstep}(H, e, t)$ is Scott-continuous.

Proof. Graph of a family continuous in e for each t. ■

Theorem (`HF.isScottContinuous`). HF is Scott-continuous.

Proof. Graph of a family continuous in H for each e, using continuity of the step in H. ■

Theorem (`HF.app`). Applying $\mathrm{HF}(H)$ to e recovers the graph of $t \mapsto \mathrm{Hstep}(H, e, t)$.

Proof. The β-law for the inner continuous graph. ■

Theorem (`HF.app2`). Applying that graph at t recovers $\mathrm{Hstep}(H, e, t)$.

Proof. The previous application law and the β-law for the step in t. ■

Definition (`Hinterp`). The compositional semantics (4.43): variables look up the environment; zero is the left injection of 0; successor and predecessor re-inject after testing; the quaternary conditional is **lamb**-wrapped; application tests the operator tag; abstraction right-injects the graph of the body in an updated environment.

Theorem (`Hinterp.isScottContinuous`). For each expression, compositional interpretation is Scott-continuous in the environment.

Proof. Induction on syntax, assembling continuity of conditionals, injections, **lamb**, graphs, and environment update. ■

Definition (`Hfun`). The interpreter is the least fixed point of HF.

Theorem (`Hfun.fixed`). $\mathrm{HF}(\mathrm{Hfun}) = \mathrm{Hfun}$.

Proof. The fixed-point theorem for a Scott-continuous operator. ■

Theorem (`Hfun.app2`). Applying Hfun twice recovers the step at Hfun.

Proof. Congruence from the fixed-point equation and the double application law for HF. ■

Theorem (`Hfun.interp`). On an encoded expression, Hfun agrees with compositional interpretation.

Proof. Induction on syntax: tag laws select the matching clause, and inductive hypotheses rewrite recursive applications. ■

Theorem (`Hfun.decode`). Applying Hfun to an encoding yields the graph of compositional interpretation.

Proof. The fixed-point application law yields the step graph; pointwise the previous agreement rewrites it to compositional interpretation. ■

Definition (`Htyped`). The retract-arrow packaging of compositional interpretation as an element of $\mathbf{env}\circ \to \mathbf{lamb}$.

Theorem (`Htyped.app`). Applying the packaging at t yields **lamb** of compositional interpretation at the **env**-projection of t.

Proof. The retract-arrow application law and the β-law for continuous compositional interpretation. ■

Theorem (`Htyped.typed`). The packaging is typed into $\mathbf{env}\circ \to \mathbf{lamb}$.

Proof. Typing of retract-arrow application. ■

Definition (`Hcomb`). $\mathcal{H}$ as the retract-arrow packaging of Hfun into $\mathbf{exp}\circ \to (\mathbf{env}\circ \to \mathbf{lamb})$.

Theorem (`Hcomb.app`). Applying $\mathcal{H}$ at e yields the **env–lamb** packaging of Hfun at the **exp**-projection of e.

Proof. The retract-arrow application law. ■

Theorem (`Hcomb.typed`). $\mathcal{H}$ is typed into $\mathbf{exp}\circ \to (\mathbf{env}\circ \to \mathbf{lamb})$.

Proof. Typing of retract-arrow application at the three retracts. ■

Theorem (`eq_4_45`). An encoding is typed into **exp**; its compositional packaging is typed into $\mathbf{env}\circ \to \mathbf{lamb}$; and $\mathcal{H}$ on that encoding equals the compositional packaging.

Proof. Encoding typing, packaging typing, and decode after projecting the encoding through **exp**. ■

`eq_4_45.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.expR
public import DomainTheory.DataTypesAsLattices.Functionality.CombC
public import DomainTheory.DataTypesAsLattices.Retracts.updateEnv
public import DomainTheory.DataTypesAsLattices.Retracts.envR
import DomainTheory.DataTypesAsLattices.Retracts.eq_4_44
import DomainTheory.DataTypesAsLattices.Retracts.typed_more
import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
import DomainTheory.DataTypesAsLattices.Functionality.PlotkinHelpers
import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
import DomainTheory.DataTypesAsLattices.Enumeration.ContinuityHelpers
import DomainTheory.DataTypesAsLattices.Enumeration.PairSeq
import DomainTheory.DataTypesAsLattices.Lambda.succSet_more
import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
import DomainTheory.DataTypesAsLattices.Graph.beta
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

inductive LambTerm where
  | var : ℕ → LambTerm
  | zero : LambTerm
  | succ : LambTerm → LambTerm
  | pred : LambTerm → LambTerm
  | condq : LambTerm → LambTerm → LambTerm → LambTerm → LambTerm
  | app : LambTerm → LambTerm → LambTerm
  | lam : ℕ → LambTerm → LambTerm

def encodeExp : LambTerm → Pomega
  | .var n => tagInj 0 (ofNat n)
  | .zero => tagInj 1 botElem
  | .succ τ => tagInj 2 (encodeExp τ)
  | .pred τ => tagInj 3 (encodeExp τ)
  | .condq ? τ σ ? =>
      tagInj 4 (seq4 (encodeExp ?) (encodeExp τ) (encodeExp σ) (encodeExp ?))
  | .app τ σ => tagInj 5 (pairSeq (encodeExp τ) (encodeExp σ))
  | .lam n τ => tagInj 6 (pairSeq (ofNat n) (encodeExp τ))

namespace encodeExp
```

```
theorem typed : ∀ τ : LambTerm, typed (encodeExp τ) expR := by
  intro τ
  induction τ with
  | var n =>
    rw [encodeExp, DomainTheory.DataTypesAsLattices.eq_4_44]
    exact plus7.typed_inl0 (typed.ofNat_intR n)
  | zero =>
    rw [encodeExp, DomainTheory.DataTypesAsLattices.eq_4_44]
    exact plus7.typed_inl1 typed.bot_botElem
  | succ τ ih =>
    rw [encodeExp, DomainTheory.DataTypesAsLattices.eq_4_44]
    exact plus7.typed_inl2 ih
  | pred τ ih =>
    rw [encodeExp, DomainTheory.DataTypesAsLattices.eq_4_44]
    exact plus7.typed_inl3 ih
  | condq ? τ σ ? ih? ihτ ihσ ih? =>
    rw [encodeExp, DomainTheory.DataTypesAsLattices.eq_4_44]
    refine plus7.typed_inl4 ?_
    rw [DomainTheory.DataTypesAsLattices.typed, tensor4.app, seq4.zero, seq4.one, seq4
    .two,
      seq4.three, ← ih?, ← ihτ, ← ihσ, ← ih?]
  | app τ σ ihτ ihσ =>
    rw [encodeExp, DomainTheory.DataTypesAsLattices.eq_4_44]
    refine plus7.typed_inl5 ?_
    exact (typed.pairSeq_tensorR expR expR _ _).mpr ⟨ihτ, ihσ⟩
  | lam n τ ih =>
    rw [encodeExp, DomainTheory.DataTypesAsLattices.eq_4_44]
    refine plus7.typed_inl6 ?_
    exact (typed.pairSeq_tensorR intR expR _ _).mpr ⟨typed.ofNat_intR n, ih⟩

end encodeExp

def expTag (e : Pomega) : Pomega := funOf e (ofNat 0)

def expPayload (e : Pomega) : Pomega := funOf e (ofNat 1)

def hApply (H e t : Pomega) : Pomega := funOf (funOf H e) t

namespace hApply

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hApply H e t) :=
  theorem_1_3 (f := fun u => funOf u t) (g := fun H => funOf H e)
    (funOf.isScottContinuous_left t) (funOf.isScottContinuous_left e)

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => hApply H e t) :=
  theorem_1_3 (f := fun u => funOf u t) (g := funOf H)
    (funOf.isScottContinuous_left t) (funOf.isScottContinuous H)

theorem payload_isScottContinuous_e (H t : Pomega) (i : ℕ) :
    IsScottContinuous (fun e =>
      hApply H (funOf (expPayload e) (ofNat i)) t) :=
  theorem_1_3 (f := fun e => hApply H e t)
    (g := fun e => funOf (expPayload e) (ofNat i))
```

```
    (isScottContinuous_e H t)
    (theorem_1_3 (f := fun u => funOf u (ofNat i)) (g := expPayload)
      (funOf.isScottContinuous_left (ofNat i))
      (funOf.isScottContinuous_left (ofNat 1)))

end hApply

def hVar (e t : Pomega) : Pomega := funOf t (expPayload e)

namespace hVar

theorem isScottContinuous_t (e : Pomega) :
    IsScottContinuous (fun t => hVar e t) :=
  funOf.isScottContinuous_left (expPayload e)

theorem isScottContinuous_e (t : Pomega) :
    IsScottContinuous (fun e => hVar e t) :=
  theorem_1_3 (f := funOf t) (g := expPayload)
    (funOf.isScottContinuous t) (funOf.isScottContinuous_left (ofNat 1))

end hVar

def hZero : Pomega := funOf inleftC (ofNat 0)

def hApp (H e t : Pomega) : Pomega :=
  condSet (funOf whichC (hApply H (funOf (expPayload e) (ofNat 0)) t)) botElem
    (funOf (funOf outC (hApply H (funOf (expPayload e) (ofNat 0)) t))
      (hApply H (funOf (expPayload e) (ofNat 1)) t))

namespace hApp

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hApp H e t) :=
  ContinuityHelpers.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (funOf.isScottContinuous whichC)
      (hApply.isScottContinuous_H _ t))
    (const_isScottContinuous botElem)
    (ContinuityHelpers.tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun u => funOf.isScottContinuous u)
      (theorem_1_3 (funOf.isScottContinuous outC)
        (hApply.isScottContinuous_H _ t))
      (hApply.isScottContinuous_H _ t))

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => hApp H e t) :=
  ContinuityHelpers.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC)
      (g := fun t => hApply H (funOf (expPayload e) (ofNat 0)) t)
      (funOf.isScottContinuous whichC)
```

```
      (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 0)))))
    (const_isScottContinuous botElem)
    (ContinuityHelpers.tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun u => funOf.isScottContinuous u)
      (theorem_1_3 (f := funOf outC)
        (g := fun t => hApply H (funOf (expPayload e) (ofNat 0)) t)
        (funOf.isScottContinuous outC)
        (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 0)))))
      (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 1)))))

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => hApp H e t) :=
  ContinuityHelpers.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC)
      (g := fun e => hApply H (funOf (expPayload e) (ofNat 0)) t)
      (funOf.isScottContinuous whichC)
      (hApply.payload_isScottContinuous_e H t 0))
    (const_isScottContinuous botElem)
    (ContinuityHelpers.tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun u => funOf.isScottContinuous u)
      (theorem_1_3 (f := funOf outC)
        (g := fun e => hApply H (funOf (expPayload e) (ofNat 0)) t)
        (funOf.isScottContinuous outC)
        (hApply.payload_isScottContinuous_e H t 0))
      (hApply.payload_isScottContinuous_e H t 1))

end hApp

def hSucc (H e t : Pomega) : Pomega :=
  condSet (funOf whichC (hApply H (expPayload e) t))
    (funOf inleftC (succSet (funOf outC (hApply H (expPayload e) t))))
    botElem

namespace hSucc

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hSucc H e t) :=
  ContinuityHelpers.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (funOf.isScottContinuous whichC) (hApply.isScottContinuous_H _ t))
    (theorem_1_3 (funOf.isScottContinuous inleftC)
      (theorem_1_3 succSet.isScottContinuous
        (theorem_1_3 (funOf.isScottContinuous outC)
          (hApply.isScottContinuous_H _ t))))
    (const_isScottContinuous botElem)

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => hSucc H e t) :=
  ContinuityHelpers.nary
```

```
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC) (g := fun t => hApply H (expPayload e) t)
      (funOf.isScottContinuous whichC)
      (funOf.isScottContinuous (funOf H (expPayload e))))
    (theorem_1_3 (f := funOf inleftC)
      (g := fun t => succSet (funOf outC (hApply H (expPayload e) t)))
      (funOf.isScottContinuous inleftC)
      (theorem_1_3 (f := succSet)
        (g := fun t => funOf outC (hApply H (expPayload e) t))
        succSet.isScottContinuous
        (theorem_1_3 (f := funOf outC) (g := fun t => hApply H (expPayload e) t)
          (funOf.isScottContinuous outC)
          (funOf.isScottContinuous (funOf H (expPayload e))))))
    (const_isScottContinuous botElem)

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => hSucc H e t) :=
  let hrec :=
    theorem_1_3 (f := fun e => hApply H e t) (g := expPayload)
      (hApply.isScottContinuous_e H t)
      (funOf.isScottContinuous_left (ofNat 1))
  ContinuityHelpers.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC) (g := fun e => hApply H (expPayload e) t)
      (funOf.isScottContinuous whichC) hrec)
    (theorem_1_3 (f := funOf inleftC)
      (g := fun e => succSet (funOf outC (hApply H (expPayload e) t)))
      (funOf.isScottContinuous inleftC)
      (theorem_1_3 (f := succSet)
        (g := fun e => funOf outC (hApply H (expPayload e) t))
        succSet.isScottContinuous
        (theorem_1_3 (f := funOf outC) (g := fun e => hApply H (expPayload e) t)
          (funOf.isScottContinuous outC) hrec)))
    (const_isScottContinuous botElem)

end hSucc

def hPred (H e t : Pomega) : Pomega :=
  condSet (funOf whichC (hApply H (expPayload e) t))
    (funOf inleftC (predSet (funOf outC (hApply H (expPayload e) t))))
    botElem

namespace hPred

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hPred H e t) :=
  ContinuityHelpers.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (funOf.isScottContinuous whichC) (hApply.isScottContinuous_H _ t))
    (theorem_1_3 (funOf.isScottContinuous inleftC)
```

```
      (theorem_1_3 predSet.isScottContinuous
        (theorem_1_3 (funOf.isScottContinuous outC)
          (hApply.isScottContinuous_H _ t))))
    (const_isScottContinuous botElem)

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => hPred H e t) :=
  ContinuityHelpers.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC) (g := fun t => hApply H (expPayload e) t)
      (funOf.isScottContinuous whichC)
      (funOf.isScottContinuous (funOf H (expPayload e))))
    (theorem_1_3 (f := funOf inleftC)
      (g := fun t => predSet (funOf outC (hApply H (expPayload e) t)))
      (funOf.isScottContinuous inleftC)
      (theorem_1_3 (f := predSet)
        (g := fun t => funOf outC (hApply H (expPayload e) t))
        predSet.isScottContinuous
        (theorem_1_3 (f := funOf outC) (g := fun t => hApply H (expPayload e) t)
          (funOf.isScottContinuous outC)
          (funOf.isScottContinuous (funOf H (expPayload e))))))
    (const_isScottContinuous botElem)

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => hPred H e t) :=
  let hrec :=
    theorem_1_3 (f := fun e => hApply H e t) (g := expPayload)
      (hApply.isScottContinuous_e H t)
      (funOf.isScottContinuous_left (ofNat 1))
  ContinuityHelpers.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC) (g := fun e => hApply H (expPayload e) t)
      (funOf.isScottContinuous whichC) hrec)
    (theorem_1_3 (f := funOf inleftC)
      (g := fun e => predSet (funOf outC (hApply H (expPayload e) t)))
      (funOf.isScottContinuous inleftC)
      (theorem_1_3 (f := predSet)
        (g := fun e => funOf outC (hApply H (expPayload e) t))
        predSet.isScottContinuous
        (theorem_1_3 (f := funOf outC) (g := fun e => hApply H (expPayload e) t)
          (funOf.isScottContinuous outC) hrec)))
    (const_isScottContinuous botElem)

end hPred

def hCond (H e t : Pomega) : Pomega :=
  funOf lambR
    (condSet (funOf whichC (hApply H (funOf (expPayload e) (ofNat 0)) t))
      (condSet (funOf outC (hApply H (funOf (expPayload e) (ofNat 0)) t))
        (hApply H (funOf (expPayload e) (ofNat 1)) t)
        (hApply H (funOf (expPayload e) (ofNat 2)) t))
      (hApply H (funOf (expPayload e) (ofNat 3)) t))
```

```
namespace hCond

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hCond H e t) :=
  theorem_1_3 (funOf.isScottContinuous lambR)
    (ContinuityHelpers.nary
      (fun y z => condSet.isScottContinuous_left y z)
      (fun x z => condSet.isScottContinuous_mid x z)
      (fun x y => condSet.isScottContinuous_right x y)
      (theorem_1_3 (funOf.isScottContinuous whichC)
        (hApply.isScottContinuous_H _ t))
      (ContinuityHelpers.nary
        (fun y z => condSet.isScottContinuous_left y z)
        (fun x z => condSet.isScottContinuous_mid x z)
        (fun x y => condSet.isScottContinuous_right x y)
        (theorem_1_3 (funOf.isScottContinuous outC)
          (hApply.isScottContinuous_H _ t))
        (hApply.isScottContinuous_H _ t)
        (hApply.isScottContinuous_H _ t))
      (hApply.isScottContinuous_H _ t))

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => hCond H e t) :=
  theorem_1_3 (f := funOf lambR)
    (g := fun t =>
      condSet (funOf whichC (hApply H (funOf (expPayload e) (ofNat 0)) t))
        (condSet (funOf outC (hApply H (funOf (expPayload e) (ofNat 0)) t))
          (hApply H (funOf (expPayload e) (ofNat 1)) t)
          (hApply H (funOf (expPayload e) (ofNat 2)) t))
        (hApply H (funOf (expPayload e) (ofNat 3)) t))
    (funOf.isScottContinuous lambR)
    (ContinuityHelpers.nary
      (fun y z => condSet.isScottContinuous_left y z)
      (fun x z => condSet.isScottContinuous_mid x z)
      (fun x y => condSet.isScottContinuous_right x y)
      (theorem_1_3 (f := funOf whichC)
        (g := fun t => hApply H (funOf (expPayload e) (ofNat 0)) t)
        (funOf.isScottContinuous whichC)
        (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 0)))))
      (ContinuityHelpers.nary
        (fun y z => condSet.isScottContinuous_left y z)
        (fun x z => condSet.isScottContinuous_mid x z)
        (fun x y => condSet.isScottContinuous_right x y)
        (theorem_1_3 (f := funOf outC)
          (g := fun t => hApply H (funOf (expPayload e) (ofNat 0)) t)
          (funOf.isScottContinuous outC)
          (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 0)))))
        (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 1))))
        (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 2)))))
      (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 3)))))

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => hCond H e t) :=
  theorem_1_3 (f := funOf lambR)
    (g := fun e =>
```

```
      condSet (funOf whichC (hApply H (funOf (expPayload e) (ofNat 0)) t))
        (condSet (funOf outC (hApply H (funOf (expPayload e) (ofNat 0)) t))
          (hApply H (funOf (expPayload e) (ofNat 1)) t)
          (hApply H (funOf (expPayload e) (ofNat 2)) t))
        (hApply H (funOf (expPayload e) (ofNat 3)) t))
    (funOf.isScottContinuous lambR)
    (ContinuityHelpers.nary
      (fun y z => condSet.isScottContinuous_left y z)
      (fun x z => condSet.isScottContinuous_mid x z)
      (fun x y => condSet.isScottContinuous_right x y)
      (theorem_1_3 (f := funOf whichC)
        (g := fun e => hApply H (funOf (expPayload e) (ofNat 0)) t)
        (funOf.isScottContinuous whichC)
        (hApply.payload_isScottContinuous_e H t 0))
      (ContinuityHelpers.nary
        (fun y z => condSet.isScottContinuous_left y z)
        (fun x z => condSet.isScottContinuous_mid x z)
        (fun x y => condSet.isScottContinuous_right x y)
        (theorem_1_3 (f := funOf outC)
          (g := fun e => hApply H (funOf (expPayload e) (ofNat 0)) t)
          (funOf.isScottContinuous outC)


          (hApply.payload_isScottContinuous_e H t 0))
        (hApply.payload_isScottContinuous_e H t 1)
        (hApply.payload_isScottContinuous_e H t 2))
      (hApply.payload_isScottContinuous_e H t 3))

end hCond

noncomputable def hLam (H e t : Pomega) : Pomega :=
  funOf inrightC
    (graph (fun x =>
      hApply H (funOf (expPayload e) (ofNat 1))
        (updateEnv.union t x (funOf (expPayload e) (ofNat 0)))))

namespace hLam

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hLam H e t) :=
  theorem_1_3 (funOf.isScottContinuous inrightC)
    (graph_const_isScottContinuous
      (fun H x =>
        hApply H (funOf (expPayload e) (ofNat 1))
          (updateEnv.union t x (funOf (expPayload e) (ofNat 0))))
      (fun _ => hApply.isScottContinuous_H _ _))

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => hLam H e t) :=
  theorem_1_3 (f := funOf inrightC)
    (g := fun t =>
      graph (fun x =>
        hApply H (funOf (expPayload e) (ofNat 1))
          (updateEnv.union t x (funOf (expPayload e) (ofNat 0)))))
    (funOf.isScottContinuous inrightC)
    (graph_const_isScottContinuous
      (fun t x =>
```

```
        hApply H (funOf (expPayload e) (ofNat 1))
          (updateEnv.union t x (funOf (expPayload e) (ofNat 0))))
      (fun x =>
        theorem_1_3
          (f := funOf (funOf H (funOf (expPayload e) (ofNat 1))))
          (g := fun t => updateEnv.union t x (funOf (expPayload e) (ofNat 0)))
          (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 1))))
          (updateEnv.union_isScottContinuous_t x
            (funOf (expPayload e) (ofNat 0)))))

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => hLam H e t) :=
  theorem_1_3 (f := funOf inrightC)
    (g := fun e =>
      graph (fun x =>
        hApply H (funOf (expPayload e) (ofNat 1))
          (updateEnv.union t x (funOf (expPayload e) (ofNat 0)))))
    (funOf.isScottContinuous inrightC)
    (graph_const_isScottContinuous
      (fun e x =>
        hApply H (funOf (expPayload e) (ofNat 1))
          (updateEnv.union t x (funOf (expPayload e) (ofNat 0))))
      (fun x =>
        ContinuityHelpers.tuple
          (fun y =>
            theorem_1_3 (f := fun u => funOf u y) (g := funOf H)
              (funOf.isScottContinuous_left y) (funOf.isScottContinuous H))
          (fun u => funOf.isScottContinuous (funOf H u))
          (theorem_1_3 (f := fun u => funOf u (ofNat 1)) (g := expPayload)
            (funOf.isScottContinuous_left (ofNat 1))
            (funOf.isScottContinuous_left (ofNat 1)))
          (theorem_1_3 (f := updateEnv.union t x)
            (g := fun e => funOf (expPayload e) (ofNat 0))
            (updateEnv.isScottContinuous_idx t x)
            (theorem_1_3 (f := fun u => funOf u (ofNat 0)) (g := expPayload)
              (funOf.isScottContinuous_left (ofNat 0))
              (funOf.isScottContinuous_left (ofNat 1)))))))

end hLam

noncomputable def Hstep (H e t : Pomega) : Pomega :=
  dcondSet (expTag e) (hVar e t)
    (dcondSet (predSet (expTag e)) hZero
      (dcondSet (predSet (predSet (expTag e))) (hSucc H e t)
        (dcondSet (predSet^[3] (expTag e)) (hPred H e t)
          (dcondSet (predSet^[4] (expTag e)) (hCond H e t)
            (dcondSet (predSet^[5] (expTag e)) (hApp H e t)
              (dcondSet (predSet^[6] (expTag e)) (hLam H e t)
                topElem))))))

namespace Hstep

theorem tag0 (H x t : Pomega) :
    Hstep H (tagInj 0 x) t = funOf t x := by
  simp only [Hstep, expTag, expPayload, hVar, tagInj.zero, tagInj.one,
    dcondSet.ofNat_zero]
```

```
theorem tag1 (H x t : Pomega) :
    Hstep H (tagInj 1 x) t = hZero := by
  simp only [Hstep, expTag, tagInj.zero, predSet.ofNat_succ,
    dcondSet.ofNat_one, dcondSet.ofNat_zero]

theorem tag2 (H x t : Pomega) :
    Hstep H (tagInj 2 x) t = hSucc H (tagInj 2 x) t := by
  simp only [Hstep, expTag, tagInj.zero, predSet.ofNat_succ,
    dcondSet.ofNat_succ, dcondSet.ofNat_zero]

theorem tag3 (H x t : Pomega) :
    Hstep H (tagInj 3 x) t = hPred H (tagInj 3 x) t := by
  simp only [Hstep, expTag, tagInj.zero, dcondSet.ofNat_succ,
    predSet.ofNat_succ, PlotkinHelpers.predSet_iterate_self,
    dcondSet.ofNat_zero]

theorem tag4 (H x t : Pomega) :
    Hstep H (tagInj 4 x) t = hCond H (tagInj 4 x) t := by
  simp only [Hstep, expTag, tagInj.zero, dcondSet.ofNat_succ,
    predSet.ofNat_succ, PlotkinHelpers.predSet_iterate_ofNat (k := 3) (i := 1),
    PlotkinHelpers.predSet_iterate_self, dcondSet.ofNat_zero]

theorem tag5 (H x t : Pomega) :
    Hstep H (tagInj 5 x) t = hApp H (tagInj 5 x) t := by
  simp only [Hstep, expTag, tagInj.zero, dcondSet.ofNat_succ,
    predSet.ofNat_succ, PlotkinHelpers.predSet_iterate_ofNat (k := 3) (i := 2),
    PlotkinHelpers.predSet_iterate_ofNat (k := 4) (i := 1),
    PlotkinHelpers.predSet_iterate_self, dcondSet.ofNat_zero]

theorem tag6 (H x t : Pomega) :
    Hstep H (tagInj 6 x) t = hLam H (tagInj 6 x) t := by
  simp only [Hstep, expTag, tagInj.zero, dcondSet.ofNat_succ,
    predSet.ofNat_succ, PlotkinHelpers.predSet_iterate_ofNat (k := 3) (i := 3),
    PlotkinHelpers.predSet_iterate_ofNat (k := 4) (i := 2),
    PlotkinHelpers.predSet_iterate_ofNat (k := 5) (i := 1),
    PlotkinHelpers.predSet_iterate_self, dcondSet.ofNat_zero]

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => Hstep H e t) :=
  ContinuityHelpers.nary
    (fun y z => dcondSet.isScottContinuous_left y z)
    (fun x z => dcondSet.isScottContinuous_mid x z)
    (fun x y => dcondSet.isScottContinuous_right x y)
    (const_isScottContinuous (expTag e)) (const_isScottContinuous (hVar e t))
    (ContinuityHelpers.nary
      (fun y z => dcondSet.isScottContinuous_left y z)
      (fun x z => dcondSet.isScottContinuous_mid x z)
      (fun x y => dcondSet.isScottContinuous_right x y)
      (const_isScottContinuous (predSet (expTag e)))
      (const_isScottContinuous hZero)
      (ContinuityHelpers.nary
        (fun y z => dcondSet.isScottContinuous_left y z)
        (fun x z => dcondSet.isScottContinuous_mid x z)
        (fun x y => dcondSet.isScottContinuous_right x y)
        (const_isScottContinuous (predSet (predSet (expTag e))))
```

```
        (hSucc.isScottContinuous_H e t)
        (ContinuityHelpers.nary
          (fun y z => dcondSet.isScottContinuous_left y z)
          (fun x z => dcondSet.isScottContinuous_mid x z)
          (fun x y => dcondSet.isScottContinuous_right x y)
          (const_isScottContinuous (predSet^[3] (expTag e)))
          (hPred.isScottContinuous_H e t)
          (ContinuityHelpers.nary
            (fun y z => dcondSet.isScottContinuous_left y z)
            (fun x z => dcondSet.isScottContinuous_mid x z)
            (fun x y => dcondSet.isScottContinuous_right x y)
            (const_isScottContinuous (predSet^[4] (expTag e)))
            (hCond.isScottContinuous_H e t)
            (ContinuityHelpers.nary
              (fun y z => dcondSet.isScottContinuous_left y z)
              (fun x z => dcondSet.isScottContinuous_mid x z)
              (fun x y => dcondSet.isScottContinuous_right x y)
              (const_isScottContinuous (predSet^[5] (expTag e)))
              (hApp.isScottContinuous_H e t)
              (ContinuityHelpers.nary
                (fun y z => dcondSet.isScottContinuous_left y z)
                (fun x z => dcondSet.isScottContinuous_mid x z)
                (fun x y => dcondSet.isScottContinuous_right x y)
                (const_isScottContinuous (predSet^[6] (expTag e)))
                (hLam.isScottContinuous_H e t)
                (const_isScottContinuous topElem)))))))

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => Hstep H e t) :=
  ContinuityHelpers.nary
    (fun y z => dcondSet.isScottContinuous_left y z)
    (fun x z => dcondSet.isScottContinuous_mid x z)
    (fun x y => dcondSet.isScottContinuous_right x y)
    (const_isScottContinuous (expTag e))
    (hVar.isScottContinuous_t e)
    (ContinuityHelpers.nary
      (fun y z => dcondSet.isScottContinuous_left y z)
      (fun x z => dcondSet.isScottContinuous_mid x z)
      (fun x y => dcondSet.isScottContinuous_right x y)
      (const_isScottContinuous (predSet (expTag e)))
      (const_isScottContinuous hZero)
      (ContinuityHelpers.nary
        (fun y z => dcondSet.isScottContinuous_left y z)
        (fun x z => dcondSet.isScottContinuous_mid x z)
        (fun x y => dcondSet.isScottContinuous_right x y)
        (const_isScottContinuous (predSet (predSet (expTag e))))
        (hSucc.isScottContinuous_t H e)
        (ContinuityHelpers.nary
          (fun y z => dcondSet.isScottContinuous_left y z)
          (fun x z => dcondSet.isScottContinuous_mid x z)
          (fun x y => dcondSet.isScottContinuous_right x y)
          (const_isScottContinuous (predSet^[3] (expTag e)))
          (hPred.isScottContinuous_t H e)
          (ContinuityHelpers.nary
            (fun y z => dcondSet.isScottContinuous_left y z)
            (fun x z => dcondSet.isScottContinuous_mid x z)
```

```
            (fun x y => dcondSet.isScottContinuous_right x y)
            (const_isScottContinuous (predSet^[4] (expTag e)))
            (hCond.isScottContinuous_t H e)
            (ContinuityHelpers.nary
              (fun y z => dcondSet.isScottContinuous_left y z)
              (fun x z => dcondSet.isScottContinuous_mid x z)
              (fun x y => dcondSet.isScottContinuous_right x y)
              (const_isScottContinuous (predSet^[5] (expTag e)))
              (hApp.isScottContinuous_t H e)
              (ContinuityHelpers.nary
                (fun y z => dcondSet.isScottContinuous_left y z)
                (fun x z => dcondSet.isScottContinuous_mid x z)
                (fun x y => dcondSet.isScottContinuous_right x y)
                (const_isScottContinuous (predSet^[6] (expTag e)))
                (hLam.isScottContinuous_t H e)
                (const_isScottContinuous topElem)))))))

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => Hstep H e t) :=
  ContinuityHelpers.nary
    (fun y z => dcondSet.isScottContinuous_left y z)
    (fun x z => dcondSet.isScottContinuous_mid x z)
    (fun x y => dcondSet.isScottContinuous_right x y)
    (funOf.isScottContinuous_left (ofNat 0))
    (hVar.isScottContinuous_e t)
    (ContinuityHelpers.nary
      (fun y z => dcondSet.isScottContinuous_left y z)
      (fun x z => dcondSet.isScottContinuous_mid x z)
      (fun x y => dcondSet.isScottContinuous_right x y)
      (PlotkinHelpers.predSet_iterate_expTag_isScottContinuous 1)
      (const_isScottContinuous hZero)
      (ContinuityHelpers.nary
        (fun y z => dcondSet.isScottContinuous_left y z)
        (fun x z => dcondSet.isScottContinuous_mid x z)
        (fun x y => dcondSet.isScottContinuous_right x y)
        (PlotkinHelpers.predSet_iterate_expTag_isScottContinuous 2)
        (hSucc.isScottContinuous_e H t)
        (ContinuityHelpers.nary
          (fun y z => dcondSet.isScottContinuous_left y z)
          (fun x z => dcondSet.isScottContinuous_mid x z)
          (fun x y => dcondSet.isScottContinuous_right x y)
          (PlotkinHelpers.predSet_iterate_expTag_isScottContinuous 3)
          (hPred.isScottContinuous_e H t)
          (ContinuityHelpers.nary
            (fun y z => dcondSet.isScottContinuous_left y z)
            (fun x z => dcondSet.isScottContinuous_mid x z)
            (fun x y => dcondSet.isScottContinuous_right x y)
            (PlotkinHelpers.predSet_iterate_expTag_isScottContinuous 4)
            (hCond.isScottContinuous_e H t)
            (ContinuityHelpers.nary
              (fun y z => dcondSet.isScottContinuous_left y z)
              (fun x z => dcondSet.isScottContinuous_mid x z)
              (fun x y => dcondSet.isScottContinuous_right x y)
              (PlotkinHelpers.predSet_iterate_expTag_isScottContinuous 5)
              (hApp.isScottContinuous_e H t)
              (ContinuityHelpers.nary
```

```
                (fun y z => dcondSet.isScottContinuous_left y z)
                (fun x z => dcondSet.isScottContinuous_mid x z)
                (fun x y => dcondSet.isScottContinuous_right x y)
                (PlotkinHelpers.predSet_iterate_expTag_isScottContinuous 6)
                (hLam.isScottContinuous_e H t)
                (const_isScottContinuous topElem)))))))
end Hstep

noncomputable def HF (H : Pomega) : Pomega :=
  graph (fun e => graph (fun t => Hstep H e t))

namespace HF

theorem inner_isScottContinuous (H : Pomega) :
    IsScottContinuous (fun e => graph (fun t => Hstep H e t)) :=
  graph_const_isScottContinuous (fun e t => Hstep H e t)
    (fun t => Hstep.isScottContinuous_e H t)

theorem isScottContinuous : IsScottContinuous HF :=
  graph_const_isScottContinuous
    (fun H e => graph (fun t => Hstep H e t))
    (fun e => graph_const_isScottContinuous (fun H t => Hstep H e t)
      (fun t => Hstep.isScottContinuous_H e t))

theorem app (H e : Pomega) :
    funOf (HF H) e = graph (fun t => Hstep H e t) :=
  beta (HF.inner_isScottContinuous H) e

theorem app2 (H e t : Pomega) :
    funOf (funOf (HF H) e) t = Hstep H e t := by
  rw [HF.app]
  exact beta (Hstep.isScottContinuous_t H e) t

end HF

noncomputable def Hinterp : LambTerm → Pomega → Pomega
  | .var n, t => funOf t (ofNat n)
  | .zero, _ => funOf inleftC (ofNat 0)
  | .succ τ, t =>
      condSet (funOf whichC (Hinterp τ t))
        (funOf inleftC (succSet (funOf outC (Hinterp τ t))))
        botElem
  | .pred τ, t =>
      condSet (funOf whichC (Hinterp τ t))
        (funOf inleftC (predSet (funOf outC (Hinterp τ t))))
        botElem
  | .condq ? τ σ ?, t =>
      funOf lambR (condSet (funOf whichC (Hinterp ? t))
        (condSet (funOf outC (Hinterp ? t)) (Hinterp τ t) (Hinterp σ t))
        (Hinterp ? t))
  | .app τ σ, t =>
      condSet (funOf whichC (Hinterp τ t)) botElem
        (funOf (funOf outC (Hinterp τ t)) (Hinterp σ t))
  | .lam n τ, t =>
      funOf inrightC (graph (fun x => Hinterp τ (updateEnv t x n)))
```

```
namespace Hinterp

theorem isScottContinuous (τ : LambTerm) :
    IsScottContinuous (Hinterp τ) := by
  induction τ with
  | var n =>
    exact funOf.isScottContinuous_left (ofNat n)
  | zero =>
    exact const_isScottContinuous _
  | succ τ ih =>
    exact ContinuityHelpers.nary
      (fun y z => condSet.isScottContinuous_left y z)
      (fun x z => condSet.isScottContinuous_mid x z)
      (fun x y => condSet.isScottContinuous_right x y)
      (theorem_1_3 (funOf.isScottContinuous whichC) ih)
      (theorem_1_3 (funOf.isScottContinuous inleftC)
        (theorem_1_3 succSet.isScottContinuous
          (theorem_1_3 (funOf.isScottContinuous outC) ih)))
      (const_isScottContinuous botElem)
  | pred τ ih =>
    exact ContinuityHelpers.nary
      (fun y z => condSet.isScottContinuous_left y z)
      (fun x z => condSet.isScottContinuous_mid x z)
      (fun x y => condSet.isScottContinuous_right x y)
      (theorem_1_3 (funOf.isScottContinuous whichC) ih)
      (theorem_1_3 (funOf.isScottContinuous inleftC)
        (theorem_1_3 predSet.isScottContinuous
          (theorem_1_3 (funOf.isScottContinuous outC) ih)))
      (const_isScottContinuous botElem)
  | condq ? τ σ ? ih? ihτ ihσ ih? =>
    exact theorem_1_3 (f := funOf lambR)
      (g := fun t =>
       condSet (funOf whichC (Hinterp ? t))
         (condSet (funOf outC (Hinterp ? t)) (Hinterp τ t) (Hinterp σ t))
         (Hinterp ? t))
      (funOf.isScottContinuous lambR)
      (ContinuityHelpers.nary
        (fun y z => condSet.isScottContinuous_left y z)
        (fun x z => condSet.isScottContinuous_mid x z)
        (fun x y => condSet.isScottContinuous_right x y)
        (theorem_1_3 (funOf.isScottContinuous whichC) ih?)
        (ContinuityHelpers.nary
          (fun y z => condSet.isScottContinuous_left y z)
          (fun x z => condSet.isScottContinuous_mid x z)
          (fun x y => condSet.isScottContinuous_right x y)
          (theorem_1_3 (funOf.isScottContinuous outC) ih?) ihτ ihσ)
        ih?)
  | app τ σ ihτ ihσ =>
    exact ContinuityHelpers.nary
      (fun y z => condSet.isScottContinuous_left y z)
      (fun x z => condSet.isScottContinuous_mid x z)
      (fun x y => condSet.isScottContinuous_right x y)
      (theorem_1_3 (funOf.isScottContinuous whichC) ihτ)
      (const_isScottContinuous botElem)
      (ContinuityHelpers.tuple
        (fun y => funOf.isScottContinuous_left y)
```

```
        (fun u => funOf.isScottContinuous u)
        (theorem_1_3 (funOf.isScottContinuous outC) ihτ) ihσ)
  | lam n τ ih =>
    exact theorem_1_3 (f := funOf inrightC)
      (g := fun t => graph (fun x => Hinterp τ (updateEnv t x n)))
      (funOf.isScottContinuous inrightC)
      (graph_const_isScottContinuous
        (fun t x => Hinterp τ (updateEnv t x n))
        (fun x => theorem_1_3 ih
          (graph_const_isScottContinuous
            (fun t m => if m = ofNat n then x else funOf t m)
            (fun m => by
              by_cases h : m = ofNat n
              · simpa [h] using const_isScottContinuous (c := x)
              · simpa [h] using funOf.isScottContinuous_left m))))

end Hinterp

noncomputable def Hfun : Pomega := fix HF

namespace Hfun

theorem fixed : HF Hfun = Hfun :=
  (theorem_1_4 HF.isScottContinuous).1

theorem app2 (e t : Pomega) :
    funOf (funOf Hfun e) t = Hstep Hfun e t := by
  have h := congrArg (fun H => funOf (funOf H e) t) fixed
  simpa [HF.app2] using h.symm


theorem interp : ∀ τ : LambTerm, ∀ t : Pomega,
    funOf (funOf Hfun (encodeExp τ)) t = Hinterp τ t := by
  intro τ
  induction τ with
  | var n =>
    intro t
    rw [app2, encodeExp, Hstep.tag0]
    rfl
  | zero =>
    intro t
    rw [app2, encodeExp, Hstep.tag1]
    rfl
  | succ τ ih =>
    intro t
    rw [app2, encodeExp, Hstep.tag2]
    simp only [hSucc, hApply, expPayload, tagInj.one]
    rw [ih]
    rfl
  | pred τ ih =>
    intro t
    rw [app2, encodeExp, Hstep.tag3]
    simp only [hPred, hApply, expPayload, tagInj.one]
    rw [ih]
    rfl
  | condq ? τ σ ? ih? ihτ ihσ ih? =>
    intro t
```

```
    rw [app2, encodeExp, Hstep.tag4]
    simp only [hCond, hApply, expPayload, tagInj.one, seq4.zero,
      seq4.one, seq4.two, seq4.three]
    rw [ihτ, ihσ, ih?]
    rfl
  | app τ σ ihτ ihσ =>
    intro t
    rw [app2, encodeExp, Hstep.tag5]
    simp only [hApp, hApply, expPayload, tagInj.one, PairSeq.app_zero,
      PairSeq.app_one]
    rw [ihτ, ihσ]
    rfl
  | lam n τ ih =>
    intro t
    rw [app2, encodeExp, Hstep.tag6]
    simp only [hLam, hApply, expPayload, tagInj.one, PairSeq.app_zero,
      PairSeq.app_one, updateEnv.ofNat]
    congr 1
    apply congrArg graph
    funext x
    exact ih _

theorem decode (τ : LambTerm) :
    funOf Hfun (encodeExp τ) = graph (Hinterp τ) := by
  have hβ : funOf Hfun (encodeExp τ) = graph (fun t => Hstep Hfun (encodeExp τ) t) :=
    by
    have := congrArg (fun H => funOf H (encodeExp τ)) fixed
    simpa [HF.app] using this.symm
  refine hβ.trans ?_
  apply congrArg graph
  funext t
  simpa [app2] using interp τ t

end Hfun

noncomputable def Htyped (τ : LambTerm) : Pomega :=
  funOf (arrowR envR lambR) (graph (Hinterp τ))

namespace Htyped

theorem app (τ : LambTerm) (t : Pomega) :
    funOf (Htyped τ) t = funOf lambR (Hinterp τ (funOf envR t)) := by
  rw [Htyped, arrowR.app]
  simp only [comp.app]
  apply congrArg (funOf lambR)
  exact beta (Hinterp.isScottContinuous τ) (funOf envR t)

theorem typed (τ : LambTerm) :
    typed (Htyped τ) (arrowR envR lambR) :=
  typed.apply_retract
    (arrowR.isRetract envR.isRetract lambR.isRetract)

end Htyped

noncomputable def Hcomb : Pomega :=
  funOf (arrowR expR (arrowR envR lambR)) Hfun
```

```
namespace Hcomb

theorem app (e : Pomega) :
    funOf Hcomb e =
      funOf (arrowR envR lambR) (funOf Hfun (funOf expR e)) := by
  rw [Hcomb, arrowR.app]
  simp only [comp.app]

theorem typed :
    typed Hcomb (arrowR expR (arrowR envR lambR)) :=
  typed.apply_retract
    (arrowR.isRetract expR.isRetract
      (arrowR.isRetract envR.isRetract lambR.isRetract))

end Hcomb

theorem eq_4_45 (τ : LambTerm) :
    typed (encodeExp τ) expR ∧
      typed (Htyped τ) (arrowR envR lambR) ∧
        funOf Hcomb (encodeExp τ) = Htyped τ := by
  refine ⟨encodeExp.typed τ, Htyped.typed τ, ?_⟩
  rw [Hcomb.app, ← encodeExp.typed τ, Hfun.decode]
  rfl

end DomainTheory.DataTypesAsLattices
```

3.5.18 Theorem 4.6 ✓

THEOREM 4.6 (The limit theorem).

Limit. *Suppose F is a continuous function that maps retracts to retracts and let $c = \mathbf{Y}(F)$. Then c is also a retract.*

inverse_limit. *If in addition F maps strict retracts to strict retracts and is monotone in the sense that $a \sqsubseteq b$ implies $F(a) \sqsubseteq F(b)$ for all (strict) retracts a and b, then the range of c is homeomorphic to the inverse limit of the ranges of the strict retracts $F^n(\bot)$ for $n \in \omega$.*

This can be applied to the case of (4.38) where $F = \lambda z.\, \mathbf{nil} \oplus (z \otimes z)$. Thus we can analyze **tree** as an inverse limit. This approach has the great advantage over the earlier method of the author where limits were required in showing that **tree** exists. Here we use **Y** to give existence at once, and then apply Theorems 4.3–4.5 to figure out the nature of the retract.

In Theorem 4.6, the fact that c is a retract can be reasoned as follows: $\bot$ is a

retract. Thus each $F^n(\bot)$ is a retract. We compute:

$$c \circ c = \bigcup\{F^n(\bot) \mid n \in \omega\} \circ \bigcup\{F^n(\bot) \mid n \in \omega\}$$

$$= \bigcup\{F^n(\bot) \circ F^n(\bot) \mid n \in \omega\} \quad \text{(Note: same } n.\text{)}$$

$$= \bigcup\{F^n(\bot) \mid n \in \omega\} = c.$$

In case F is monotone and preserves strictness, then we can argue that each $F^n(\bot) \sqsubseteq c$. The retracts $F^n(\bot)$ are the *projections* of c onto the terms of the limit. Of course $F^n(\bot) \sqsubseteq F^m(\bot)$ if $n \leq m$. The

$u : c$ can be put into a one-to-one correspondence (homeomorphism, lattice isomorphism) with the infinite sequences $\langle v_0, v_1, \ldots, v_n, \ldots\rangle$, where $v_n : F^n(\bot)$ and $v_n = F^n(\bot)(v_{n+1})$. Indeed $v_n = F^n(\bot)(u)$ and $u = \bigcup\{v_n \mid n \in \omega\}$. This is exactly the inverse limit construction.

Retreating from generalities back to the example of **tree**, we can grant that it exists and is provably a retract. Two things in its range are $\bot$ and $\top$ by Theorem 4.5(ii), but they are not so interesting. Now $\bot : \mathbf{nil}$, so by Theorem 4.5(ii) we have $\langle 0\rangle = \langle 0, \bot\rangle : \mathbf{tree}$. Let us think of this as *the* atom. What else can we have? If $x, y : \mathbf{tree}$, then $\langle x, y\rangle : \mathbf{tree} \otimes \mathbf{tree}$ and so $\langle 1, \langle x, y\rangle\rangle : \mathbf{tree}$. Thus (the range of) **tree** contains an atom and is closed under a binary operation. Note that the atomic and nonatomic trees are distinguished by **which** and that suitable constructor and destructor functions are definable on **tree**. But the space also contains infinite trees since we can solve for the least fixed point of:

$$t = \langle 1, \langle\langle 0\rangle, t\rangle\rangle$$

and $t : \mathbf{tree}$. (Why?) And there are many other examples of infinite elements in **tree**.

A point to stress in this construction is that **tree** being LAMBDA-definable is *computable*, and there are many computable functions definable on or to (the range of) **tree**. All the "structural" functions, for example, are computable. These are functions which in other languages would be called **isatom** or **construct** or **node**, and they are all easily LAMBDA-definable. Just as with $\oplus$, $\otimes$, $\circ\!\!\rightarrow$, they are not explicit in the *notation*, but they are definable nevertheless. In the case of **node**, we could use finite sequences of Boolean values to pick out or name nodes. Thus solve for $\mathbf{name} = \mathbf{nil} \oplus \mathbf{bool} \otimes \mathbf{name}$, and then give a recursive definition of:

$$\mathbf{node} : \mathbf{name}\circ\!\!\rightarrow (\mathbf{tree}\circ\!\!\rightarrow \mathbf{tree}).$$

Another example of this method can be illustrated, if the reader will recall the Gödel numbering of § 3. It will be seen that there are similarities with the **tree** construction: instead of 0 and $\mathbf{apply}(n)(m)$, **tree** uses $\langle 0\rangle$ and $\langle 1, \langle x, y\rangle\rangle$. Note, however, that Gödel numbers are *finite* while **tree** has *infinite* objects. But the infinite objects are always *limits* of finite objects, so there *are* connections. (We discuss this again in § 6.) In particular, recursive definitions on Gödel numbers, like that of **val**, have analogues on **tree**. Here is the companion of (3.7):

$$(4.46)\quad \mathbf{vaal} = \lambda x : \mathbf{tree}.\, \mathbf{which}(x) \supset \mathbf{G},\ \mathbf{vaal}(\mathbf{fst}(\mathbf{out}(x)))(\mathbf{vaal}(\mathbf{snd}(\mathbf{out}(x)))).$$

We have $\mathbf{vaal} : \mathbf{tree} \rightarrow \mathbf{id}$, where of course (4.46) is taken as defining **vaal** as the least fixed point. This is an example of a computable function between effectively given retracts. The LAMBDA-definable elements of $\mathbf{P}\omega$ are the computable elements in the range of **vaal**.

We have discussed the category of retracts and continuous maps, but if they are all LAMBDA-definable, then they fall within the countable model **RE**. Thus there is *another* category of effectively given retracts and effectively given continuous maps. (Examples: **tree**, **id**, **vaal**, and all those retracts and maps generated by $\oplus$, $\otimes$, and $\circ\!\!\rightarrow$.) This category seems to deserve the status of a generalized recursion theory; though this is not to say that as yet very much is known about it. In fact, the proper formulation may require an enriched category rather than a restricted one. Thus instead of confining attention to the computable retracts and computable maps, it might be better to use the full category with all maps and to single out the computable ones (also maybe the finite ones) by special predicates. In effect we have avoided any methodological decisions by working in

the universal space $\mathbf{P}\omega$ and by *defining* a notion when required—if possible with the aid of LAMBDA. This makes it possible to give all the necessary definitions and to prove the theorems without at first having to worry about axiomatic problems.

5. Closure operations and algebraic lattices.

The main point of the proof of Theorem 4.6 has already been given in the text.

Formalization. Scott's two sentences are two theorems, `theorem_4_6.Limit` and `theorem_4_6.inverse_limit`; `theorem_4_6` is their conjunction. The witnessing order isomorphism is the definition `theorem_4_6.InverseLimit`. Downstream existence for **tree**, **lamb**, and **exp** uses `theorem_4_6.Limit`.

Theorem (`theorem_4_6.Limit`). If F is Scott-continuous and sends retracts to retracts, then every iterate $F^n(\bot)$ is a retract, and so is $c = \mathbf{Y}(F)$.

Proof. Induction from $\bot$ gives the iterates. For c: identify $\mathbf{Y}(F)$ with $\operatorname{fix} F = \bigcup_n F^n(\bot)$ (Theorem 2.5); the union of a chain of retracts is a graph, and application of that graph is idempotent on the union, so $c = c \circ c$. ■

Definition (`theorem_4_6.InverseLimit`). The order isomorphism

$$\operatorname{Fix}(\operatorname{fix} F) \;\simeq_o\; \varprojlim_n \operatorname{range}(F^n(\bot))$$

under the strictness and monotonicity hypotheses, with projections $u \mapsto F^n(\bot)(u)$ and reconstruction $u = \bigcup_n v_n$.

Proof. Assemble `fixedToInverseLimit` / `inverseLimitToFixed` and the union/projection identities. ■

Theorem (`theorem_4_6.inverse_limit`). Under those same extra hypotheses, such an order isomorphism exists (the ranges are homeomorphic / lattice-isomorphic in Scott's sense).

Proof. ⟨theorem_4_6.InverseLimit⟩. ■

Theorem (`theorem_4_6`). `theorem_4_6.Limit` and `theorem_4_6.inverse_limit`.

Proof. ⟨`theorem_4_6.Limit`, `theorem_4_6.inverse_limit`⟩. ■

`theorem_4_6.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Retracts.fixedToInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimitToFixed
public import DomainTheory.DataTypesAsLattices.Retracts.BasicInst55
import DomainTheory.DataTypesAsLattices.Retracts.funOf_fix_idem
import DomainTheory.DataTypesAsLattices.Retracts.iUnion_retract_isGraph
import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_project_iUnion
import DomainTheory.DataTypesAsLattices.Retracts.iterates_areRetracts
import DomainTheory.DataTypesAsLattices.Retracts.graphLemmas
import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5
import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace theorem_4_6

/-- Scott?s first sentence of Theorem 4.6: the iterates $F^n(\bot)$ and
$\mathbf{Y}(F)$ are retracts. -/
theorem Limit {F : Pomega → Pomega}
    (hf : IsScottContinuous F)
```

```
    (hret : ∀ a, Retract a → Retract (F a)) :
    (∀ n, Retract (iterateBot F n)) ∧
      Retract (funOf Ycomb (graph F)) := by
  refine ⟨iterates_areRetracts hret, ?_⟩
  have hY : funOf Ycomb (graph F) = fix F := theorem_2_5 hf
  rw [hY]
  constructor
  change fix F = graph (fun x => funOf (fix F) (funOf (fix F) x))
  have hG : IsGraph (fix F) :=
    iUnion_retract_isGraph (iterates_areRetracts hret)
  have hgr : graph (funOf (fix F)) = fix F := (theorem_1_2.iii (fix F)).mpr hG
  have hidem' : graph (fun x => funOf (fix F) (funOf (fix F) x)) =
      graph (funOf (fix F)) :=
    graph_ext (fun x => funOf_fix_idem hf hret x)
  exact (hidem'.trans hgr).symm

/-- Scott?s second sentence of Theorem 4.6: under strictness and monotonicity,
the range of $\mathbf{Y}(F)$ is order-isomorphic to the inverse limit of the
ranges of $F^n(\bot)$. -/
def InverseLimit {F : Pomega → Pomega}
    (hf : IsScottContinuous F)
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b)) :
    Fixpoints (funOf (fix F)) ≃o RetractInverseLimit F where
  toFun := fixedToInverseLimit hf hret hstrict hmono
  invFun := inverseLimitToFixed hf hret hstrict hmono
  left_inv u := by
    apply Subtype.ext
    change (⋃ n, funOf (iterateBot F n) u.1) = u.1
    rw [← funOf.iUnion, ← fix, u.property]
  right_inv v := by
    apply Subtype.ext
    funext n
    exact inverseLimit_project_iUnion hf hret hstrict hmono v n
  map_rel_iff' := by
    intro u w
    constructor
    · intro h
      change ∀ n, funOf (iterateBot F n) u.1 ⊆
        funOf (iterateBot F n) w.1 at h
      change u.1 ⊆ w.1
      have hu : u.1 = ⋃ n, funOf (iterateBot F n) u.1 := by
        calc
          u.1 = funOf (fix F) u.1 := u.property.symm
          _ = ⋃ n, funOf (iterateBot F n) u.1 := by
            change funOf (⋃ n, iterateBot F n) u.1 = _
            rw [funOf.iUnion]
      have hw : w.1 = ⋃ n, funOf (iterateBot F n) w.1 := by
        calc
          w.1 = funOf (fix F) w.1 := w.property.symm
          _ = ⋃ n, funOf (iterateBot F n) w.1 := by
            change funOf (⋃ n, iterateBot F n) w.1 = _
            rw [funOf.iUnion]
      rw [hu, hw]
```

```
      intro k hk
      obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
      exact Set.mem_iUnion.mpr ⟨n, h n hkn⟩
    · intro huw
      change u.1 ⊆ w.1 at huw
      change ∀ n, funOf (iterateBot F n) u.1 ⊆
        funOf (iterateBot F n) w.1
      intro n
      exact funOf.monotone_right (iterateBot F n) huw

/-- Scott?s second sentence as a proposition: the ranges are order-isomorphic
(witnessed by `InverseLimit`). -/
theorem inverse_limit {F : Pomega → Pomega}
    (hf : IsScottContinuous F)
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b)) :
    Nonempty (Fixpoints (funOf (fix F)) ≃o RetractInverseLimit F) :=
  ⟨InverseLimit hf hret hstrict hmono⟩

end theorem_4_6

/-- Scott?s Theorem 4.6: the two sentences, as two theorems. -/
theorem theorem_4_6 {F : Pomega → Pomega}
    (hf : IsScottContinuous F)
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b)) :
    ((∀ n, Retract (iterateBot F n)) ∧
      Retract (funOf Ycomb (graph F))) ∧
      Nonempty (Fixpoints (funOf (fix F)) ≃o RetractInverseLimit F) :=
  ⟨theorem_4_6.Limit hf hret, theorem_4_6.inverse_limit hf hret hstrict hmono⟩

end DomainTheory.DataTypesAsLattices
```

3.6 Closure operations and algebraic lattices

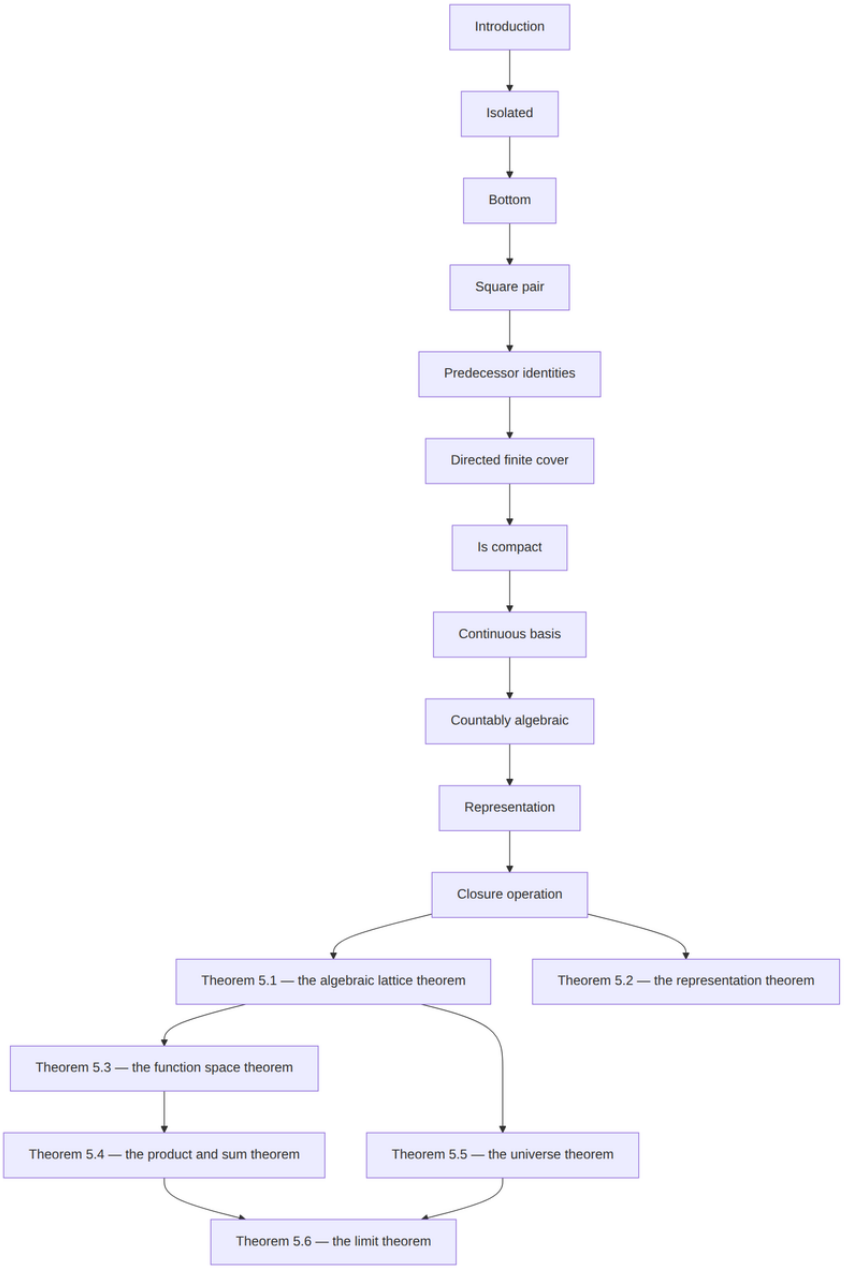


Figure 35: Closure operations and algebraic lattices.

3.6.1 Introduction ✓

Algebraic lattices and closure operations predate the paper: [101] for the lattices, [247] for the closures. Per Martin-Löf and Peter Hancock suggested that the continuous closures form a universe, and the construction of $\mathbf{V}$ is essentially theirs. The limit theorem is Scott's.

Given any family of (finitary) operations on a set (say, ω) there is a closure operation defined on the subsets of that set obtained by forming the *least subset* including the given elements and *closed* under the operations. Examples are very familiar from algebra: the subgroup generated by a set of elements, the subspace spanned by a set of vectors, the convex hull of a set of geometric points. We simplify matters here by restricting attention to closures operating on sets in $\mathbf{P}\omega$, but the idea is quite general. The main point about these "algebraic" closure operations—as distinguished from topological closure operations—is that they are *continuous*. Thus, in the case of subgroups, if an element belongs to the subgroup generated by some elements, then it also belongs to the subgroup generated by *finitely* many of them. In the context of $\mathbf{P}\omega$ we can state the characteristic condition very simply.

3.6.2 Isolated ✓

Isolated elements of $\mathbf{P}\omega$ are the finite (compact) basis sets e_n.

Formalization.

Definition (`Isolated`). An element $x \in \mathbf{P}\omega$ is isolated when $x = e_n$ for some $n \in \omega$.

Proof. The class packages that existential; `e` is the standard finite-set coding on $\mathbf{P}\omega$. ■

`Isolated.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

class Isolated (x : Pomega) : Prop where
  out : ∃ n, x = e n

end DomainTheory.DataTypesAsLattices
```

3.6.3 Bottom ✓

Elementary identities for $\bot$ in $\mathbf{P}\omega$: it is distinct from $\top$, every nonempty set meets some element, and unions with $\bot$ are trivial.

Theorem (`botElem.ne_top`). $\top \neq \bot$ in $\mathbf{P}\omega$.

Proof. Otherwise $0 \in \bot$; but $\bot$ is empty. ■

Theorem (`botElem.exists_mem`). If $x \neq \bot$, then some natural number belongs to x.

Proof. Contrapositively, a set with no members is $\bot$. ■

Theorem (`botElem.union_empty`). $x \cup \bot = x$.

Proof. Union with the empty set does not change x. ■

Theorem (`botElem.empty_union`). $\bot \cup x = x$.

Proof. The empty set is a unit for union on the left as well. ■

Theorem (`botElem.ne_ofNat_zero`). $\bot \neq 0$ in $\mathbf{P}\omega$.

Proof. Otherwise $0 \in \bot$; but $\bot$ is empty. ■

`botElem.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.Pomega
public import DomainTheory.DataTypesAsLattices.Lambda.Language
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace botElem

theorem ne_top : (topElem : Pomega) = botElem := by
  intro h
  have h0 : (0 : ℕ) ∈ (botElem : Pomega) := h ▸ Set.mem_univ 0
  simp [botElem] at h0

theorem ne_ofNat_zero : botElem = ofNat 0 := by
  intro h
  have : (0 : ℕ) ∈ botElem := by rw [h]; simp [ofNat]
  exact (mem_botElem 0).mp this

theorem exists_mem {x : Pomega} (h : x = botElem) : ∃ m, m ∈ x := by
  by_contra hc
  refine h ?_
  ext n
  simp only [mem_botElem, iff_false]
  exact fun hn => hc ⟨n, hn⟩

theorem union_empty (x : Pomega) : x ∪ botElem = x := Set.union_empty x

theorem empty_union (x : Pomega) : botElem ∪ x = x := Set.empty_union x

end botElem

end DomainTheory.DataTypesAsLattices
```

3.6.4 Square pair ✓

The square pairing $x \square y$ interlaces the elements of x and y by parity of codes.

Definition (`squarePair`). For $x, y \subseteq \omega$,

$$x \square y \;=\; \{2n \mid n \in x\} \;\cup\; \{2m+1 \mid m \in y\}.$$

Proof. Evens carry the first component and odds the second; every code has a unique parity, so the two halves do not interfere. ■

`squarePair.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.Pomega
public import Mathlib.Tactic.ToAdditive
@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
def squarePair (x y : Pomega) : Pomega :=
  {k | (∃ n ∈ x, k = 2 * n) ∨ (∃ m ∈ y, k = 2 * m + 1)}

end DomainTheory.DataTypesAsLattices
```

3.6.5 Predecessor identities ✓

Predecessor identities used when shifting codes: $\mathrm{pred}(\mathrm{succ}\, x) = x$ and pred preserves unions.

Formalization. Library facts live under `predSet` in `Lambda/predSet_more.lean`.

Theorem (`predSet.isScottContinuous`). The map $x \mapsto \mathrm{predSet}(x)$ is Scott-continuous on $\mathbf{P}\omega$.

Proof. Membership of k in $\mathrm{predSet}(x)$ is witnessed by a finite e-approximation (namely e_{2^k+1}), so the Scott-union characterization applies both ways. ■

Theorem (`predSet.ofNat_succ`). $\mathrm{predSet}(j+1) = j$ as numerals.

Proof. Unfold `ofNat` and `predSet`; the predecessor of the singleton $\{j+1\}$ is $\{j\}$. ■

Theorem (`predSet.succSet`). $\mathrm{predSet}(\mathrm{succSet}\, x) = x$.

Proof. Elements of $\mathrm{succSet}\, x$ are successors of members of x; injectivity of succ recovers x, and conversely every $k \in x$ contributes $k+1$. ■

Theorem (`predSet.union`). $\mathrm{predSet}(x \cup y) = \mathrm{predSet}(x) \cup \mathrm{predSet}(y)$.

Proof. Unfold `predSet`; predecessor distributes over union of sets of naturals. ■

`predSet_more.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.Language
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace predSet

theorem isScottContinuous : IsScottContinuous predSet := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    refine mem_scottUnion.mpr ⟨2 ^ (k + 1), ?_, ?_⟩
    · simpa [e.pow2, Set.singleton_subset_iff, predSet] using hk
    · simp [predSet, e.pow2]
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    exact hm hkm

theorem ofNat_succ (j : ℕ) : predSet (ofNat (j + 1)) = ofNat j := by
  ext k
  simp [predSet, ofNat]
```

```
theorem succSet (x : Pomega) : predSet (succSet x) = x := by
  ext k
  constructor
  · intro ⟨n, hn, heq⟩
    exact (Nat.succ_injective heq) ▸ hn
  · intro hk
    exact ⟨k, hk, rfl⟩

theorem union (x y : Pomega) :
    predSet (x ∪ y) = predSet x ∪ predSet y := by
  ext k; simp [predSet]

end predSet

end DomainTheory.DataTypesAsLattices
```

3.6.6 Directed finite cover ✓

A finite subset of the union of a directed family of fixpoints is covered by a single member of the family.

Theorem (`directed_finite_cover_subset`). Let A be a directed set of fixed points of a continuous map on $\mathbf{P}\omega$. If a finite $t \subseteq \omega$ is contained in the union of the members of A, then some single $z \in A$ already contains t.

Proof. Directedness of A as fixed points is the same as directedness of the underlying sets, so the ordinary finite-cover lemma for directed families applies. ■

`directed_finite_cover_subset.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.DirectedSubset
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem directed_finite_cover_subset {f : Pomega → Pomega}
    {A : Set (Fixpoints f)} (hA : DirectedSubset A) {t : Pomega}
    (ht : t.Finite) (hsub : t ⊆ ⋃₀ (Subtype.val '' A)) :
    ∃ z ∈ A, t ⊆ z := by
  have hA' : DirectedSet A := by
    refine ⟨⟨hA.out.1, ?_⟩⟩
    intro x hx y hy
    obtain ⟨z, hz, hxz, hyz⟩ := hA.out.2 hx hy
    exact ⟨z, hz, hxz, hyz⟩
  simpa [Fixpoints.vals] using directed_finite_cover hA' ht (by
    simpa [Fixpoints.vals] using hsub)

end DomainTheory.DataTypesAsLattices
```

3.6.7 Is compact ✓

An element of a complete lattice is compact when it is way-below itself.

Definition (`IsCompact`). In a complete lattice, x is compact when $x \ll x$: whenever a directed set has supremum at least x, some single member of the set already lies above x.

Proof. Compactness is the diagonal case of the way-below relation. ■

`IsCompact.lean`

```
module

public import DomainTheory.DataTypesAsLattices.FixedLattices.WayBelow

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def IsCompact {α : Type u} [CompleteLattice α] (x : α) : Prop :=
  WayBelow x x

end DomainTheory.DataTypesAsLattices
```

3.6.8 Continuous basis ✓

A continuous basis assigns to each element a directed family of way-below approximants whose supremum recovers the element.

Definition (`ContinuousBasis`). A map sending each x in a complete lattice to a set $\mathrm{basis}(x)$ is a continuous basis when, for every x: - $\mathrm{basis}(x)$ is directed, - every $y \in \mathrm{basis}(x)$ satisfies $y \ll x$, and - $x = \bigsqcup \mathrm{basis}(x)$.

Proof. These three clauses are the definition of a continuous basis. ■

`ContinuousBasis.lean`

```
module

public import DomainTheory.DataTypesAsLattices.FixedLattices.WayBelow

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

class ContinuousBasis {α : Type u} [CompleteLattice α]
    (basis : α → Set α) : Prop where
  out : ∀ x, @DirectedSet α completeLatticePreorder (basis x) ∧
    (∀ y ∈ basis x, WayBelow y x) ∧
    x = @sSup α completeLatticeSupSet (basis x)

end DomainTheory.DataTypesAsLattices
```

3.6.9 Countably algebraic ✓

A complete lattice is countably algebraic when its compact elements admit a countable enumeration that is dense: every element is the supremum of the compacts below it. Finite-support approximations along that enumeration are directed and recover the same supremum.

Definition (`CountablyAlgebraic`). A complete lattice is countably algebraic when there is an enumeration $n \mapsto c_n$ of compact elements such that every compact equals some c_n, and every x is the supremum of $\{c_n \mid c_n \le x\}$.

Proof. Compactness of each c_n, completeness of the enumeration, and density are the three data of the structure. ■

Definition (`finiteBasisSups`). Given a sequence d and a set x of indices, the finite-basis supremums are the elements $\bigsqcup\{d_n \mid n \in t\}$ for finite $t \subseteq x$.

Proof. Each such supremum is determined by a finite support inside x. ■

Theorem (`finiteBasisSups.sSup_eq`). The supremum of all finite-basis supremums of d along x equals $\bigsqcup\{d_n \mid n \in x\}$.

Proof. Every finite sub-supremum is below the full image supremum; conversely each singleton support is one of the finite-basis supremums. ■

Theorem (`finiteBasisSups.directed`). The set of finite-basis supremums of d along x is directed.

Proof. The empty support supplies a bottom; the union of two finite supports supplies an upper bound. ■

Theorem (`CountablyAlgebraic.compact_finite_support`). If $c_m \le \bigsqcup\{c_n \mid n \in x\}$ in a countably algebraic lattice, then finitely many indices from x already suffice.

Proof. Compactness of c_m against the directed family of finite-basis supremums yields a finite support; the preceding supremum identity rewrites that as a finite sub-image of the enumeration. ■

`CountablyAlgebraic.lean`

```
module

public import DomainTheory.DataTypesAsLattices.FixedLattices.IsCompact

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

structure CountablyAlgebraic (α : Type u) [CompleteLattice α] where
  compact : ℕ → α
  compact_isCompact : ∀ n, IsCompact (compact n)
  compact_complete : ∀ x, IsCompact x → ∃ n, x = compact n
  density : ∀ x, x = @sSup α completeLatticeSupSet
    {k : α | IsCompact k ∧ @LE.le α completeLatticeLE k x}

end DomainTheory.DataTypesAsLattices
```

`finiteBasisSups.lean`

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
def finiteBasisSups {α : Type u} [CompleteLattice α]
    (d : ℕ → α) (x : Set ℕ) : Set α :=
  {y | ∃ t : Finset ℕ, (t : Set ℕ) ⊆ x ∧
    y = sSup (d '' (t : Set ℕ))}

namespace finiteBasisSups

theorem sSup_eq {α : Type u} [CompleteLattice α]
    (d : ℕ → α) (x : Set ℕ) :
    sSup (finiteBasisSups d x) = sSup (d '' x) := by
  apply le_antisymm
  · apply sSup_le
    rintro y ⟨t, htx, rfl⟩
    apply sSup_le
    intro q hq
    exact le_sSup (Set.image_mono htx hq)
  · apply sSup_le
    rintro _ ⟨n, hn, rfl⟩
    calc
      d n ≤ sSup (d '' (({n} : Finset ℕ) : Set ℕ)) :=
        le_sSup ⟨n, by simp, rfl⟩
      _ ≤ sSup (finiteBasisSups d x) :=
        le_sSup ⟨{n}, by simpa, rfl⟩

theorem directed {α : Type u} [CompleteLattice α]
    (d : ℕ → α) (x : Set ℕ) :
    DirectedSet (finiteBasisSups d x) := by
  constructor
  constructor
  · exact ⟨sSup (d '' ((∅ : Finset ℕ) : Set ℕ)), ∅, by simp, rfl⟩
  · rintro y ⟨t, htx, rfl⟩ z ⟨u, hux, rfl⟩
    refine ⟨sSup (d '' ((t ∪ u : Finset ℕ) : Set ℕ)),
      ⟨t ∪ u, ?_, rfl⟩, ?_, ?_⟩
    · intro n hn
      simp only [Finset.coe_union, Set.mem_union] at hn
      exact hn.elim (fun h => htx h) (fun h => hux h)
    · apply sSup_le
      intro q hq
      apply le_sSup
      exact Set.image_mono (by
        intro n hn
        exact by simp [hn]) hq
    · apply sSup_le
      intro q hq
      apply le_sSup
      exact Set.image_mono (by
        intro n hn
        exact by simp [hn]) hq

end finiteBasisSups

end DomainTheory.DataTypesAsLattices
```

CountablyAlgebraic_more.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.finiteBasisSups

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace CountablyAlgebraic

theorem compact_finite_support
    {α : Type u} [CompleteLattice α] (D : CountablyAlgebraic α)
    {m : ℕ} {x : Set ℕ}
    (h : D.compact m ≤ sSup (D.compact '' x)) :
    ∃ t : Finset ℕ, (t : Set ℕ) ⊆ x ∧
      D.compact m ≤ sSup (D.compact '' (t : Set ℕ)) := by
  have h' : D.compact m ≤ sSup (finiteBasisSups D.compact x) := by
    rwa [finiteBasisSups.sSup_eq]
  obtain ⟨y, ⟨t, htx, rfl⟩, hmy⟩ :=
    D.compact_isCompact m (finiteBasisSups D.compact x)
      (finiteBasisSups.directed D.compact x) h'
  exact ⟨t, htx, hmy⟩

end CountablyAlgebraic

end DomainTheory.DataTypesAsLattices
```

3.6.10 Representation ✓

The representation of a countably algebraic lattice in $\mathbf{P}\omega$: encode by the set of compact indices below an element, and decode by the supremum of those compacts.

Definition (`representationEncode`). Given a countably algebraic lattice with compact enumeration c_n, the code of an element x is $\{n \in \omega \mid c_n \leq x\}$.

Proof. Membership of an index records that the corresponding compact lies below x. ■

Definition (`representationDecode`). The decode of a set $u \subseteq \omega$ is $\bigsqcup\{c_n \mid n \in u\}$.

Proof. The image of the codes under the enumeration, taken as a supremum. ■

Theorem (`representationDecode.encode`). Decoding the code of x recovers x.

Proof. Every compact below x appears in the code, so density says x is their supremum; conversely each selected compact is already $\leq x$. ■

`representation.lean`

```
module

public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.Pomega.Pomega

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
def representationEncode {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) (x : α) : Pomega :=
  {n | D.compact n ≤ x}

def representationDecode {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) (x : Pomega) : α :=
  sSup (D.compact '' x)

namespace representationDecode

theorem encode
    {α : Type u} [CompleteLattice α] (D : CountablyAlgebraic α) (x : α) :
    representationDecode D (representationEncode D x) = x := by
  apply le_antisymm
  · apply sSup_le
    rintro _ ⟨n, hn, rfl⟩
    exact hn
  · calc
      x = sSup {k : α | IsCompact k ∧ k ≤ x} := D.density x
      _ ≤ representationDecode D (representationEncode D x) := by
        apply sSup_le
        rintro k ⟨hkc, hkx⟩
        obtain ⟨n, rfl⟩ := D.compact_complete k hkc
        exact le_sSup ⟨n, hkx, rfl⟩

end representationDecode

end DomainTheory.DataTypesAsLattices
```

3.6.11 Closure operation ✓

DEFINITION. An element $a \in \mathbf{P}\omega$ is called a *closure operation* iff it satisfies:

$$\mathbf{I} \subseteq a = a \circ a.$$

We see by definition that a closure operation is not only continuous, but it is also a retract. This is reasonable since the closure of the closure of a subset must be equal to the closure. To say of a function that $\mathbf{I} \subseteq a$ means that $x \subseteq a(x)$ for all $x \in \mathbf{P}\omega$. In other words, every set is contained in its closure. (Note that closures are opposite to the "projections", those retracts where $a \subseteq \mathbf{I}$.) Among examples of closure operations we find $\mathbf{I}$ and $\mathbf{T}$; the first has the most closed sets (fixed points). The second has the least. (Note that $\mathbf{T} = \omega$ always is a fixed point of a closure operation; $\mathbf{T} = \lambda x.\, \mathbf{T}$ is thus the most trivial closure operation.) The examples **fun**, **open**, and **int** of § 4 are all closure operations (cf. (4.1), (4.6), (4.7)). We remarked that **fun** is a retract, but the reader should prove in addition:

$$(5.1) \quad u \subseteq \lambda x.\, u(x),$$

for all $u \in \mathbf{P}\omega$ (cf. Theorem 1.2). We note that this fact can be rewritten in the language of retracts as:

$$(5.2) \quad \mathbf{I} \subseteq \mathbf{I} \circ \to \mathbf{I},$$

the significance of which will emerge after we develop a bit of the theory of closure operations.

Unfortunately the natural definition of the retract **bool** does not yield a closure operation. In this section we adopt this modification:

$$(5.3) \quad \mathbf{boool} = \lambda u.\, u \supset 0,\, \mathbf{T} + 1.$$

The closed sets of **boool** are $\perp$, 0, $\mathbf{T} + 1$, and $\mathbf{T}$. Note that with any closure operation a, the function value $a(x)$ is the least closed set (fixed point of a) including as a subset the given set x. Thus given any family $\mathcal{C} \subseteq \mathbf{P}\omega$ of "closed" sets which is closed under the intersection of subfamilies, if we define

$$(5.4) \quad a(x) = \bigcap \{y \in \mathcal{C} \mid x \subseteq y\},$$

then this will be a closure operation *provided* it is continuous. This remark makes it easy to check that certain functions are closure operations if we can spot easily the family $\mathcal{C}$ of fixed points.

Alas, the "natural" definition of ordered pairs (cf. (2.21)) leads to projections rather than closures. Here we must choose another:

$$(5.5) \quad [x, y] = \{2n \mid n \in x\} \cup \{2m + 1 \mid m \in y\},$$

with these inverse functions:

$$(5.6) \quad [u]_0 = \{n \mid 2n \in u\},$$

$$(5.7) \quad [u]_1 = \{m \mid 2m + 1 \in u\}.$$

We shall find that the main advantage of these equations lies in the obvious equation:

$$(5.8) \quad u = [[u]_0, [u]_1],$$

which is not true for the other pairing functions. Of course we have:

$$(5.9) \quad [[x, y]]_0 = x,$$

$$(5.10) \quad [[x, y]]_1 = y.$$

We shall not extend the idea of these new functions to triples and sequences, though it is clear what to do.

Abstractly, an *algebraic lattice* is a complete lattice in which the isolated points are dense. An *isolated* (sometimes called: *compact*) point in a lattice is one that is not the limit (sup or l.u.b.) of any directed family of its proper subelements. This definition works in continuous lattices, but more generally it is better to say that if the isolated point is *contained in* a sup, then it is also contained in a finite subsup (a sup of a finite selection of elements out of the given sup). In the case of the lattice of subgroups of a group, the isolated ones are the finitely generated subgroups. The isolated points of $\mathbf{P}\omega$ are the finite sets e_n. To say that isolated points are *dense* means that every element in the lattice is the sup of the isolated points it contains. The sequel to Theorem 4.1 for closure operations relates them to algebraic lattices.

Definition (`Closure`). An element $a \in \mathbf{P}\omega$ is a closure operation when $\mathbf{I} \subseteq a$ and a is a retract ($a = a \circ a$).

Proof. Expansiveness $\mathbf{I} \subseteq a$ is $x \subseteq a(x)$ for all x; idempotence is the retract equation. Together these are Scott's display $\mathbf{I} \subseteq a = a \circ a$. ■

`Closure.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

class Closure (a : Pomega) : Prop where
  out : Icomb ⊆ a ∧ Retract a

end DomainTheory.DataTypesAsLattices
```

3.6.12 Theorem 5.1 — the algebraic lattice theorem ✓

THEOREM 5.1 (The algebraic lattice theorem). *The fixed points of any closure operation form an algebraic lattice.*

Theorem (`theorem_5_1`). The fixed points of any closure operation form an algebraic lattice: their compact elements are the $a(e_n)$, and every fixed point is the directed supremum of the compact fixed points below it.

Proof. A fixed point is compact among the fixed points precisely when it is some $a(e_n)$; the $a(e_n)$ below a given fixed point are directed and recover it as their union. ■

`theorem_5_1.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Closures.closureCompact
public import DomainTheory.DataTypesAsLattices.Closures.compactFixedBelow
import DomainTheory.DataTypesAsLattices.Closures.compact_fixed_iff
import DomainTheory.DataTypesAsLattices.Closures.compactFixedBelow_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_5_1 {a : Pomega} (ha : Closure a) :
    (∀ x : Fixpoints (funOf a),
      IsCompactSubset x ↔ ∃ n, x = closureCompact a ha n) ∧
    ∀ x : Fixpoints (funOf a),
      DirectedSubset (compactFixedBelow a x) ∧
        (x : Pomega) = ⋃₀ (Subtype.val '' compactFixedBelow a x) :=
  ⟨compact_fixed_iff.subset ha, fun x =>
    ⟨compactFixedBelow.directedSubset ha x, compactFixedBelow.sUnion ha x⟩⟩

end DomainTheory.DataTypesAsLattices
```

3.6.13 Theorem 5.2 — the representation theorem ✓

The proof is very easy if one notes that the isolated points of $\{x \mid x = a(x)\}$, where a is a closure operation, are exactly the images $a(e_n)$ of the finite sets in $\mathbf{P}\omega$. What makes Theorem 5.1 more interesting is the converse.

THEOREM 5.2 (The representation theorem for algebraic lattices). *Every algebraic lattice with a countable number of isolated points is isomorphic to the range of some closure operation.*

By Theorem 1.6 we know that the algebraic lattice is a retract, but a more direct argument makes the closure property clear. Thus, let D be the algebraic lattice with $\{d_n \mid n \in \omega\}$ as the set of all isolated points with the indicated enumeration. We shall use the square notation with symbols $\sqsubseteq$ and $\sqcup$ for the lattice ordering and sup. The desired closure operation is defined by:

$$a(x) = \{m \mid d_m \sqsubseteq \sqcup\{d_n \mid n \in x\}\}.$$

It is an easy exercise to show that from the definition of "isolated" it follows that a is continuous; and from density, it follows that D is in a one-to-one order preserving correspondence with the fixed points of a.

In the last section we introduced an algebra of retracts, much of which carries over to closure operations given the proper definitions. Without any change we can use Theorem 4.3 on function spaces, provided we check that the required retracts are closures.

Theorem (`theorem_5_2`). Every countably algebraic lattice is order-isomorphic to the fixed points of a closure operation on $\mathbf{P}\omega$: the representing map is Scott-continuous, yields a closure, and induces a bijective order correspondence with that closure's fixed points.

Proof. Continuity of the representing map, that it is a closure, and the order isomorphism onto the fixed points. ■

`theorem_5_2.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Closures.representClosureCode
import DomainTheory.DataTypesAsLattices.Closures.representationOrderIso
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_5_2 {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) :
    IsScottContinuous (representClosure D) ∧
      Closure (representClosureCode D) ∧
      ∃ φ : α → Fixpoints (funOf (representClosureCode D)),
        Function.Bijective φ ∧
          ∀ x y, x ≤ y ↔ (φ x : Pomega) ⊆ φ y :=
  ⟨representClosure.isScottContinuous D,
    representClosureCode.isClosure D,
    ⟨fun x => representationOrderIso D x,
      (representationOrderIso D).toEquiv.bijective,
      fun x y => by
        constructor
        · intro hxy
          exact (representationOrderIso D).map_rel_iff.mpr hxy
        · intro hsub
          exact (representationOrderIso D).map_rel_iff.mp hsub⟩⟩

end DomainTheory.DataTypesAsLattices
```

3.6.14 Theorem 5.3 — the function space theorem ✓

THEOREM 5.3 (The function space theorem for algebraic lattices). *Suppose that a and b are closure operations; then so is $a\circ \to b$.*

The proof comes down to showing that

$$u(x) \sqsubseteq b(u(a(x))), \tag{5.11}$$

whenever a and b are closure operations. But this is easy by monotonicity. Note that (5.1) is needed.

For those interested in topology, one can give a construction of the isolated points of the function space which is much more direct than just taking the functions $b \circ e_n \circ a$, which on the face of it do not tell us too much. But we shall not need this explicit construction here.

The reason for changing the pairing functions is to be able to form products and sums of closure operations. In the case of products, the analogue of $\otimes$ is straightforward:

$$a \boxtimes b = \lambda u.\, [a([u]_0), b([u]_1)]; \tag{5.12}$$

while for sums using $a' = \lambda x.\, 0 \cup a(x-1) + 1$ and similarly for b' we write:

$$a \boxplus b = \lambda u.\, ([u]_0 \supset 0,\, 0) \cup ([u]_1 \supset 1,\, 1) \sqsupset [a'([u]_0), \bot],\, [\bot, b'([u]_1)]. \tag{5.13}$$

We can then establish with the aid of (5.8)–(5.10):

Theorem (`theorem_5_3`). If a and b are closure operations, then $a\circ \to b$ is a retract.

Proof. Theorem 4.3(i) already gives idempotence of the function-space retract when a and b are retracts; closures are retracts. ■

Theorem (`theorem_5_3.closure`). If a and b are closure operations, then so is $a\circ \to b$.

Proof. Expansiveness $\mathbf{I} \subseteq a\circ \to b$ together with the preceding retract fact. ■

`theorem_5_3.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Closures.Closure
import DomainTheory.DataTypesAsLattices.Closures.Icomb_subset_arrowR
import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_3_more
import DomainTheory.DataTypesAsLattices.Retracts.graphLemmas
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_5_3 {a b : Pomega} (ha : Closure a) (hb : Closure b) :
    Retract (arrowR a b) := by
  constructor
  change arrowR a b = graph (fun x => funOf (arrowR a b) (funOf (arrowR a b) x))
  apply graph_ext
  intro u
  rw [theorem_4_3.i ha.out.2 hb.out.2, arrowR.app]

namespace theorem_5_3

theorem closure {a b : Pomega} (ha : Closure a) (hb : Closure b) :
    Closure (arrowR a b) :=
```

```
  ⟨⟨Icomb_subset_arrowR ha hb, theorem_5_3 ha hb⟩⟩

end theorem_5_3

end DomainTheory.DataTypesAsLattices
```

3.6.15 Theorem 5.4 — the product and sum theorem ✓

THEOREM 5.4 (The product and sum theorem for algebraic lattices). *Suppose that a and b are closure operations; then so are $a \boxtimes b$ and $a \boxplus b$. Analogues of the results in Theorems 4.4 and 4.5 carry over.*

Following the discussion in § 4, we can also show that the closure operations form a Cartesian closed category, which in some ways is better than the category of all retracts. What makes it better is the existence of a "universe".

Every continuous operation generates a closure operation by just closing up the sets under the continuous function (as a set operation). We can institutionalize this thought by means of this definition:

$$(5.14) \quad \mathbf{V} = \lambda a \lambda x.\, \mathbf{Y}(\lambda y.\, x \cup a(y)).$$

Clearly $\mathbf{V}$ is LAMBDA-definable, continuous, etc. A more understandable characterization would define $\mathbf{V}(a)(x)$ by this equation:

$$(5.15) \quad \mathbf{V}(a)(x) = \bigcap \{y \mid x \sqsubseteq y \text{ and } a(y) \sqsubseteq y\}.$$

These two definitions are easily seen to be equivalent. What is unexpected is the discovery (due in a different form to Peter Hancock and Per Martin-Löf) that $\mathbf{V}$ itself is a closure operation.

theorem_5_4.lean

```
module

public import DomainTheory.DataTypesAsLattices.Closures.boxTensor
public import DomainTheory.DataTypesAsLattices.Closures.boxPlus
import DomainTheory.DataTypesAsLattices.Closures.Icomb_subset_boxPlus
import DomainTheory.DataTypesAsLattices.Closures.Icomb_subset_boxTensor

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace theorem_5_4

theorem times {a b : Pomega} (ha : Closure a) (hb : Closure b) :
    Closure (boxTensor a b) :=
  ⟨⟨Icomb_subset_boxTensor ha hb, boxTensor.isRetract ha.out.2 hb.out.2⟩⟩

theorem plus {a b : Pomega} (ha : Closure a) (hb : Closure b) :
    Closure (boxPlus a b) :=
  ⟨⟨Icomb_subset_boxPlus ha hb, boxPlus.isRetract ha.out.2 hb.out.2⟩⟩

end theorem_5_4

theorem theorem_5_4 {a b : Pomega} (ha : Closure a) (hb : Closure b) :
```

```
    Closure (boxTensor a b) ∧ Closure (boxPlus a b) :=
  ⟨theorem_5_4.times ha hb, theorem_5_4.plus ha hb⟩

end DomainTheory.DataTypesAsLattices
```

3.6.16 Theorem 5.5 — the universe theorem ✓

THEOREM 5.5 (The universe theorem for algebraic lattices). *The function* $\mathbf{V}$ *is a closure operation and its fixed points comprise the set of all closure operations.*

Thus to say a is a closure operation, write $a : \mathbf{V}$. To have a mapping on closure operations, write $f : \mathbf{V}\circ \to \mathbf{V}$. Remark that 5.5 allows us to write $\mathbf{V} : \mathbf{V}$. It all seems rather circular, but it is quite consistent. The category of separate algebraic lattices "contains itself"—if we are careful to work through retracts of $\mathbf{P}\omega$.

The proof of Theorem 5.5 requires a few steps. We note first that for all $x, a \in \mathbf{P}\omega$:

$$(5.16) \quad x \subseteq \mathbf{V}(a)(x),$$

Let $y = \mathbf{V}(a)(x)$. This is the least y with $x \cup a(y) \subseteq y$. What is the least z with $y \cup a(z) \subseteq z$? The answer is of course y, which proves:

$$(5.17) \quad \mathbf{V}(a)(\mathbf{V}(a)(x)) = \mathbf{V}(a)(x),$$

Thus $\mathbf{V}(a)$ is always a closure operation. If a is already a closure operation, then clearly $\mathbf{V}(a)(x) = a(x)$. Therefore we have shown:

$$(5.18) \quad a = \mathbf{V}(a) \quad \text{iff} \quad a \text{ is a closure operation.}$$

But then by (5.16) and (5.17) we have by (5.18):

$$(5.19) \quad \mathbf{V}(a) = \mathbf{V}(\mathbf{V}(a)).$$

From (5.16) by monotonicity we see:

$$(5.20) \quad a(x) \subseteq a(\mathbf{V}(a)(x)) \subseteq \mathbf{V}(a)(x).$$

Hence by (5.1) we can derive:

$$(5.21) \quad a \subseteq \lambda x.\, a(x) \subseteq \lambda x.\, \mathbf{V}(a)(x) = \mathbf{V}(a).$$

From (5.19) and (5.21) it follows that $\mathbf{V}$ itself is a closure operation.

The operation $\mathbf{V}$ forms the least closure operation containing a given element, and it shows that the lattice of closure operations is not only a retract of $\mathbf{P}\omega$ but also an algebraic lattice. Since we can now use $\mathbf{V}$ as a retract, the earlier results become formulas:

$$(5.22) \quad (\lambda a : \mathbf{V}.\, \lambda b : \mathbf{V}.\, a \boxtimes b) : \mathbf{V}\circ \to (\mathbf{V}\circ \to \mathbf{V});$$

$$(5.23) \quad (\lambda a : \mathbf{V}.\, \lambda b : \mathbf{V}.\, a \boxplus b) : \mathbf{V}\circ \to (\mathbf{V}\circ \to \mathbf{V});$$

we can also state such functorial properties as:

$$(5.24)\quad (\lambda a : \mathbf{V}.\, \lambda b : \mathbf{V}.\, a\circ \to b) : \mathbf{V}\circ \to (\mathbf{V}\circ \to \mathbf{V}).$$

Using this style of notation we have:

Theorem (`theorem_5_5`). If a is a closure operation, then $\mathbf{V}(a)(x) = a(x)$ for every x, and $\mathbf{V}(a)$ is idempotent: $\mathbf{V}(a)(\mathbf{V}(a)(x)) = \mathbf{V}(a)(x)$.

Proof. The equality with $a(x)$ is (5.18) at a closure; the idempotence is (5.17). ■

`theorem_5_5.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.Vapply
import DomainTheory.DataTypesAsLattices.Closures.eq_5_18
import DomainTheory.DataTypesAsLattices.Closures.eq_5_17

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_5_5 {a : Pomega} (ha : Closure a) (x : Pomega) :
    Vapply a x = funOf a x ∧ Vapply a (Vapply a x) = Vapply a x :=
  ⟨eq_5_18.eq ha x, eq_5_17 a x⟩

end DomainTheory.DataTypesAsLattices
```

3.6.17 Theorem 5.6 — the limit theorem ✓

THEOREM 5.6 (The limit theorem for algebraic lattices).

$$(\lambda f : \mathbf{V}\circ \to \mathbf{V}.\, \mathbf{Y}(f)) : (\mathbf{V}\circ \to \mathbf{V})\circ \to \mathbf{V}.$$

In words: if f is a mapping on closure operations, then its least fixed point is also a closure operation. The proof of course holds with any retract in place of $\mathbf{V}$, but we are more interested in applications to $\mathbf{V}$. For example, note that $\mathbf{V}(\bot) = \mathbf{I}$. Now let $f = \lambda a : \mathbf{V}.\, a\circ \to a$. The least fixed point of this f is the limit of the sequence:

$$\bot,\ \mathbf{I},\ \mathbf{I}\circ \to \mathbf{I},\ (\mathbf{I}\circ \to \mathbf{I})\circ \to (\mathbf{I}\circ \to \mathbf{I}),\ ((\mathbf{I}\circ \to \mathbf{I})\circ \to (\mathbf{I}\circ \to \mathbf{I}))\circ \to ((\mathbf{I}\circ \to \mathbf{I})\circ \to (\mathbf{I}\circ \to \mathbf{I})),\ \ldots$$

and we see that all these retracts are *strict*. This means $\mathbf{Y}(f)$ is nontrivial in that it has at least *two* fixed points (viz., $\bot$ and $\top$). But $d = \mathbf{Y}(f)$ must be the least closure operation satisfying

$$(5.25)\quad d = d\circ \to d,$$

and we have thus proved that there are *nontrivial* algebraic lattices isomorphic to their own function spaces. This construction (which rests on hardly more than (5.2), since we could take $d = \mathbf{Y}(\lambda a.\, \mathbf{I} \cup (a\circ \to a))$) is much quicker than the inverse limit construction originally found by the author to give λ-calculus models satisfying (η). There are many other fixed points of (5.25) besides this least closure operation, but their connection with inverse limits is not fully investigated.

We note in conclusion that most constructions by fixed points give algebraic lattices (like **lamb** in § 4), and so we could just as well do them in $\mathbf{V}$ if we remember to use $\boxtimes$ and $\boxplus$. The one-point space is $\top$ (*not* **nil**), and so the connection with inverse limits via Theorem 4.6 is not as clear when nonstrict functions are used. For many purposes, this may not make any difference.

theorem_5_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
import DomainTheory.DataTypesAsLattices.Closures.Vcomb_more
import DomainTheory.DataTypesAsLattices.Retracts.retract_app
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_5_6 {f : Pomega} (hf : typed f (arrowR Vcomb Vcomb)) :
    typed (funOf Ycomb f) Vcomb := by
  set y := funOf Ycomb f
  have hy : y = funOf f y := Ycomb.unfold f
  have hf' : ∀ x, funOf f x = funOf Vcomb (funOf f (funOf Vcomb x)) := by
    intro x
    simpa [comp.app] using congrArg (fun w => funOf w x) (arrowR.typed_of hf)
  have hyV : y = funOf Vcomb (funOf f (funOf Vcomb y)) :=
    hy.trans (hf' y)
  have h1 : funOf Vcomb y = funOf Vcomb (funOf f y) :=
    congrArg (funOf Vcomb) hy
  have h2 : funOf Vcomb (funOf f y) =
      funOf Vcomb (funOf Vcomb (funOf f (funOf Vcomb y))) :=
    congrArg (funOf Vcomb) (hf' y)
  have h3 : funOf Vcomb (funOf Vcomb (funOf f (funOf Vcomb y))) =
      funOf Vcomb (funOf f (funOf Vcomb y)) :=
    retract_app Vcomb.isRetract _
  exact (h1.trans (h2.trans (h3.trans hyV.symm))).symm

end DomainTheory.DataTypesAsLattices
```

3.7 Subsets and their classification

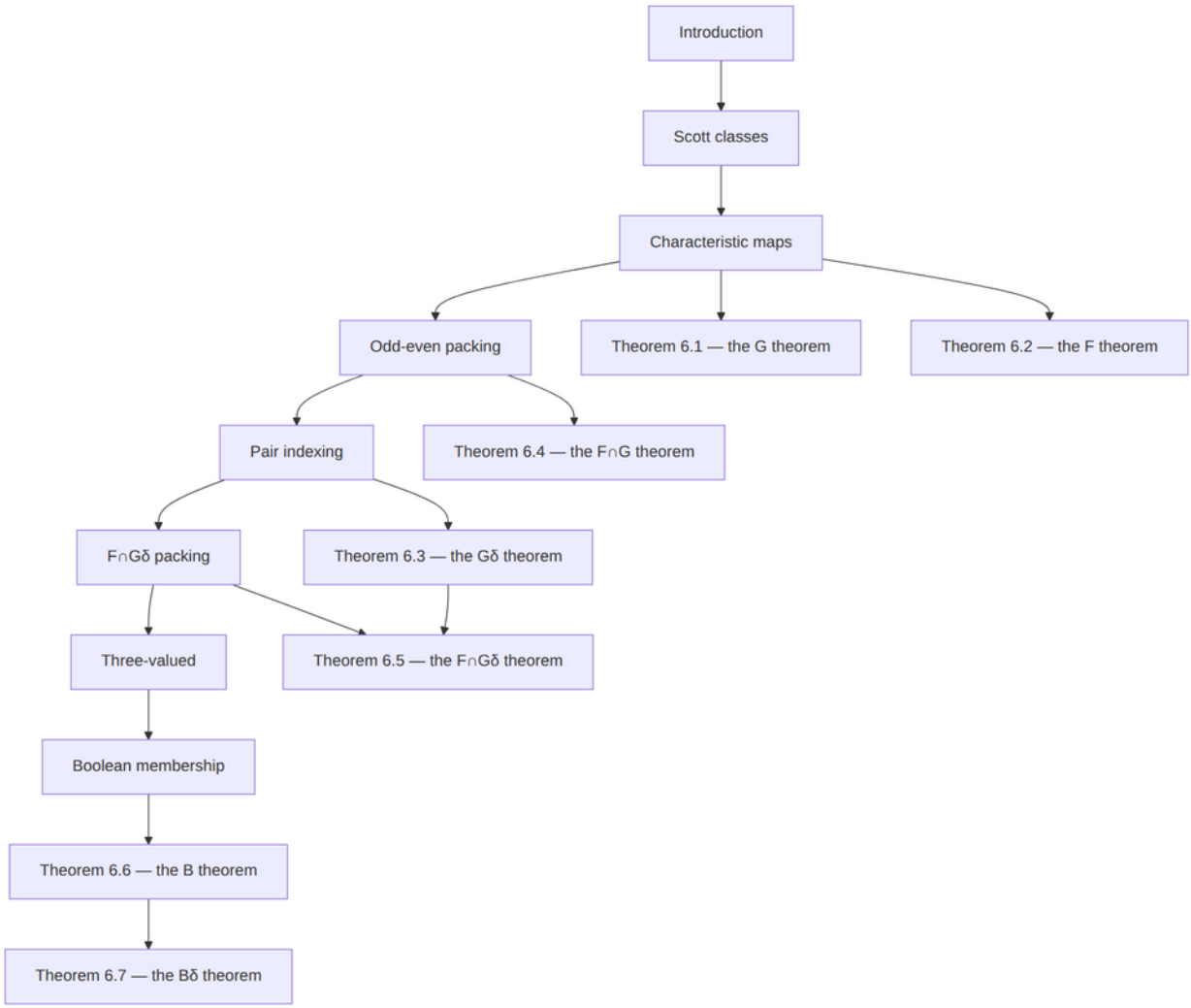


Figure 36: Subsets and their classification.

3.7.1 Introduction ✓

Retracts produce very special subsets of $\mathbf{P}\omega$: a retract always has a nonempty range which forms a lattice under $\sqsubseteq$. For example, the range of **int** is $\{\bot, \top\} \cup \omega$. We often wish to eliminate $\bot$ and $\top$; and with a retract like **tree** the situation is more complex, since combinations like $\langle 1, \langle\langle 1, \langle \bot, \langle 0\rangle\rangle\rangle, \top\rangle\rangle$ might require elimination. In these two cases the method is simple.

Historical note. The classification of these subsets by notions from descriptive set theory, and the connection with Wadge reducibility, are in the work of Adrian Tang. The normal forms for the classes, such as Table 3, are Scott's.

Consider these two functions:

$$(6.1) \quad \mathbf{mid} = \lambda x : \mathbf{int}.\, x \supset 0, 0$$

$$(6.2) \quad \mathbf{perf} = \lambda u : \mathbf{tree}.\, \mathbf{which}(u) \supset 0,\, \Delta(\mathbf{perf}(\mathbf{fst}(\mathbf{out}(u))))(\mathbf{perf}(\mathbf{snd}(\mathbf{out}(u))))$$

where Δ is a special combinator:

$$(6.3) \quad \Delta = \lambda x \lambda y.\, (x \supset (y \supset 0, \top), \top) \cup (y \supset (x \supset 0, \top), \top).$$

We find that $\omega = \{x : \mathbf{int} \mid \mathbf{mid}(x) = 0\}$. In the case of trees, note first this behavior of Δ:

Δ	$\bot$	0	$\top$
$\bot$	$\bot$	$\bot$	$\top$
0	$\bot$	0	$\top$
$\top$	$\top$	$\top$	$\top$

The question is: what subset is $\{u : \mathbf{tree} \mid \mathbf{perf}(u) = 0\}$?
Now **perf** is defined recursively. We can see that

$$\mathbf{perf}(\bot) = \bot, \quad \mathbf{perf}(\top) = \top, \quad \mathbf{perf}(\langle 0 \rangle) = 0,$$

and

$$\mathbf{perf}(\langle 1, \langle x, y \rangle \rangle) = \Delta(\mathbf{perf}(x))(\mathbf{perf}(y))$$

when $x, y : \mathbf{tree}$. Every tree, aside from $\top$ or $\bot$, is either *atomic* or a pair of trees. The atomic tree is "perfect" (that is, $\mathbf{perf}(\langle 0 \rangle) = 0$). A finite tree which does not

contain $\bot$ or $\top$ is perfect—as we can see inductively using the table above for Δ. An infinite tree is never perfect: either some branch ends in $\top$ and **perf** maps it to $\top$, or $\top$ is never reached and **perf** maps it to $\bot$. Thus the subset in question is then seen to be the set of *finite* trees generated from the atom by pairing. This is clearly a desirable subset, and it is sorted out by a function with a simple recursive definition. The general question is: what subsets can be characterized by equations? The answer can be given by reference to the topology of $\mathbf{P}\omega$.

3.7.2 Scott classes ✓

DEFINITION. Let $\mathfrak{G}$ be the class of open subsets of $\mathbf{P}\omega$, and $\mathfrak{F}$ be the class of closed subsets. Further let $\mathfrak{B}$ be the class of all (finite) Boolean combinations of open sets.

We recall from § 1 that $U \in \mathfrak{G}$ just in case for all $x \in \mathbf{P}\omega$, we have $x \in U$ if and only if some finite subset of x is in U. The class of open sets contains $\varnothing$ and $\mathbf{P}\omega$ and is closed under finite intersection and arbitrary union; in fact, it can be generated by these two closure conditions from subsets of the special form $\{x \in \mathbf{P}\omega \mid n \in x\}$ for the various $n \in \omega$. An open set is always *monotonic* (whenever $x \in U$ and $x \subseteq y$, then $y \in U$), so that every nonempty $U \in \mathfrak{G}$ has $\top \in U$.

Another characterization of openness can be given by continuous functions. Suppose $U \in \mathfrak{G}$. Define $f : \mathbf{P}\omega \to \{\bot, \top\}$ so that

$$U = \{x \mid f(x) = \top\};$$

then f is continuous. Conversely, if such an f is continuous, then U is open. But if we do not assume the range of f is included in $\{\bot, \top\}$, this is not true. For the case of general functions we know that f is continuous if and only if $\{x \mid f(x) \in V\}$ is open for all open V. This defines continuity in terms of openness, but we can turn it the other way around:

Definition (`ScottF`). $\mathfrak{F}$ is the class of closed subsets of $\mathbf{P}\omega$: those U whose complement is Scott-open.

Definition (`IsScottGdelta`). U is a $\mathfrak{G}_\delta$ when it is a countable intersection of Scott-open sets.

Definition (`IsFcapG`). U belongs to $\mathfrak{F}\dot{\cap}\mathfrak{G}$ when $U = C \cap O$ for some closed C and Scott-open O.

Definition (`IsFcapGdelta`). U belongs to $\mathfrak{F}\dot{\cap}\mathfrak{G}_\delta$ when $U = C \cap G$ for some closed C and $\mathfrak{G}_\delta$ set G.

Definition (`IsBooleanOpen`). The Boolean combinations of Scott-open sets are generated inductively by taking opens, binary unions, binary intersections, and complements.

Definition (`ScottB`). $\mathfrak{B}$ is the class of sets satisfying $\mathrm{IsBooleanOpen}$.

Definition (`IsBdelta`). U is a $\mathfrak{B}_\delta$ when it is a countable intersection of Boolean combinations of opens.

Theorem (`IsBooleanOpen.of_closed`). Every closed set is Boolean.

Proof. The complement of a closed set is open, hence Boolean, and Boolean sets are closed under complement. ■

Theorem (`IsBooleanOpen.of_fcapG`). Every set in $\mathfrak{F}\dot{\cap}\mathfrak{G}$ is Boolean.

Proof. Intersect a closed (hence Boolean) set with an open. ■

Theorem (`IsBooleanOpen.sUnion_finite`). A finite union of Boolean sets is Boolean.

Proof. By induction on the finite family: the empty union is the empty open set, and binary union closes the inductive step. ■

`Classes.lean`

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen_more
public import DomainTheory.DataTypesAsLattices.Pomega.eDense
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def ScottF : Set (Set Pomega) := {U | IsScottOpen U?}

def IsFcapG (U : Set Pomega) : Prop :=
  ∃ C O, IsScottOpen C? ∧ IsScottOpen O ∧ U = C ∩ O

def IsScottGdelta (U : Set Pomega) : Prop :=
  ∃ Us : ℕ → Set Pomega, (∀ n, IsScottOpen (Us n)) ∧ U = ⋂ n, Us n
```

```
def IsFcapGdelta (U : Set Pomega) : Prop :=
  ∃ C G, IsScottOpen C? ∧ IsScottGdelta G ∧ U = C ∩ G

inductive IsBooleanOpen : Set Pomega → Prop
  | ofOpen {U} : IsScottOpen U → IsBooleanOpen U
  | union {U V} : IsBooleanOpen U → IsBooleanOpen V → IsBooleanOpen (U ∪ V)
  | inter {U V} : IsBooleanOpen U → IsBooleanOpen V → IsBooleanOpen (U ∩ V)
  | compl {U} : IsBooleanOpen U → IsBooleanOpen U?

namespace IsBooleanOpen

theorem of_closed {U : Set Pomega} (hU : IsScottOpen U?) :
    IsBooleanOpen U := by
  have h := (IsBooleanOpen.ofOpen hU).compl
  rwa [compl_compl] at h

theorem of_fcapG {U : Set Pomega} (hU : IsFcapG U) : IsBooleanOpen U := by
  obtain ⟨C, O, hC, hO, rfl⟩ := hU
  exact (of_closed hC).inter (IsBooleanOpen.ofOpen hO)

theorem sUnion_finite {S : Set (Set Pomega)}
    (hS : S.Finite) (hU : ∀ U ∈ S, IsBooleanOpen U) :
    IsBooleanOpen (⋃₀ S) := by
  revert hU
  induction S, hS using Set.Finite.induction_on with
  | empty =>
    intro
    simp
    exact IsBooleanOpen.ofOpen IsScottOpen.empty
  | insert ha hs ih =>
    intro hUV
    rename_i a s
    simp [Set.sUnion_insert]
    exact (hUV a (Set.mem_insert a s)).union
      (ih fun u hu => hUV u (Set.mem_insert_of_mem a hu))

end IsBooleanOpen

def ScottB : Set (Set Pomega) := {U | IsBooleanOpen U}

def IsBdelta (U : Set Pomega) : Prop :=
  ∃ Bs : ℕ → Set Pomega, (∀ n, IsBooleanOpen (Bs n)) ∧ U = ⋂ n, Bs n

end DomainTheory.DataTypesAsLattices
```

3.7.3 Characteristic maps ✓

Characteristic maps detect membership in Scott-open or closed sets by continuous equations with the numerals 0, $\bot$, and $\top$.

Definition (`charOpen`). $\mathrm{charOpen}\, U\, x = \{k \mid k = 0 \wedge \exists n.\, e_n \subseteq x \wedge e_n \in U\}$.

Definition (`charTop`). $\mathrm{charTop}\, U\, x = \{k \mid \exists n.\, e_n \subseteq x \wedge e_n \in U\}$.

Definition (`closedToZero`). $\mathrm{closedToZero}\, C\, x = 0 \cup \mathrm{charTop}\, C^{\mathrm{c}}\, x$.

Theorem (`charOpen.isScottContinuous`). For every U, the map $x \mapsto \mathrm{charOpen}\, U\, x$ is Scott-continuous.

Proof. Membership of a coordinate at x is witnessed by a finite $e_n \subseteq x$ and reassembles under Scott unions. ■

Theorem (`charOpen.eq_zero`). If U is Scott-open, then $\mathrm{charOpen}\, U\, x = 0$ if and only if $x \in U$.

Proof. A witness $e_n \subseteq x$ in U and upward closure put x in U; conversely Scott-open membership supplies such a witness, forcing the value to equal the numeral 0. ■

Theorem (`charOpen.threeValued`). Every value of $\mathrm{charOpen}\, U$ is three-valued (in fact $\bot$ or 0).

Proof. Either 0 belongs to the value, which forces equality with the numeral 0, or the value is empty. ■

Theorem (`charTop.isScottContinuous`). For every U, the map $x \mapsto \mathrm{charTop}\, U\, x$ is Scott-continuous.

Proof. Finite witnesses of membership assemble under Scott unions, as for $\mathrm{charOpen}$. ■

Theorem (`charTop.eq_top`). If U is Scott-open, then $\mathrm{charTop}\, U\, x = \top$ if and only if $x \in U$.

Proof. Membership of 0 in a top value yields a finite witness in U; conversely Scott-openness supplies a witness for every coordinate of $\top$. ■

Theorem (`charTop.eq_bot`). If U is Scott-open, then $\mathrm{charTop}\, U\, x = \bot$ if and only if $x \notin U$.

Proof. Compare with the previous equivalence and the emptiness of $\bot$. ■

Theorem (`charTop.eq_bot_of_closed`). If C^{c} is Scott-open, then $\mathrm{charTop}\, C^{\mathrm{c}}\, x = \bot$ if and only if $x \in C$.

Proof. Apply the previous equivalence to the complement. ■

Theorem (`closedToZero.isScottContinuous`). For every C, the map $x \mapsto \mathrm{closedToZero}\, C\, x$ is Scott-continuous.

Proof. It is the pointwise union of a constant numeral with $\mathrm{charTop}\, C^{\mathrm{c}}$, and binary union is Scott-continuous in each argument separately. ■

Theorem (`closedToZero.threeValued`). If C^{c} is Scott-open, every value of $\mathrm{closedToZero}\, C$ is three-valued.

Proof. On C the characteristic of the complement is $\bot$, so the union equals 0; off C it is $\top$, so the union equals $\top$. ■

Theorem (`closedToZero.eq_zero`). If C^{c} is Scott-open, then $\mathrm{closedToZero}\, C\, x = 0$ if and only if $x \in C$.

Proof. Off C the value is $\top$, distinct from 0; on C it collapses to 0. ■

`CharacteristicMaps.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Classification.ThreeValued
import DomainTheory.DataTypesAsLattices.Closures.union_left_isScottContinuous
import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen_more
import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_isUpper
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
import DomainTheory.DataTypesAsLattices.Lambda.ofNat_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
open TopologicalSpace

def charOpen (U : Set Pomega) (x : Pomega) : Pomega :=
  {k | k = 0 ∧ ∃ n, e n ⊆ x ∧ e n ∈ U}

namespace charOpen

theorem isScottContinuous (U : Set Pomega) :
    IsScottContinuous (charOpen U) := by
  constructor
  intro x
  ext k
  constructor
  · intro ⟨hk0, n, hn, hne⟩
    exact mem_scottUnion.mpr ⟨n, hn, hk0, n, subset_rfl, hne⟩
  · intro hk
    obtain ⟨m, hm, hk0, n, hn, hne⟩ := mem_scottUnion.mp hk
    exact ⟨hk0, n, subset_trans hn hm, hne⟩

theorem eq_zero {U : Set Pomega} (hU : IsScottOpen U) (x : Pomega) :
    charOpen U x = ofNat 0 ↔ x ∈ U := by
  constructor
  · intro h
    have : 0 ∈ charOpen U x := by rw [h]; simp [ofNat]
    obtain ⟨_, n, hn, hne⟩ := this
    exact isScottOpen_isUpper hU hne hn
  · intro hx
    ext k
    constructor
    · intro ⟨hk0, _⟩; simp [ofNat, hk0]
    · intro hk
      simp [ofNat] at hk
      subst hk
      obtain ⟨n, hn, hne⟩ := (IsScottOpen.mem_iff hU).mp hx
      exact ⟨rfl, n, hn, hne⟩

theorem threeValued (U : Set Pomega) (x : Pomega) :
    ThreeValued (charOpen U x) := by
  constructor
  by_cases h : 0 ∈ charOpen U x
  · refine Or.inr (Or.inl ?_)
    ext k
    constructor
    · intro ⟨hk0, _⟩; simp [ofNat, hk0]
    · intro hk; simp [ofNat] at hk; subst hk; exact h
  · refine Or.inl ?_
    ext k
    constructor
    · intro hk
      obtain ⟨rfl, n, hn, hne⟩ := hk
      exact h ⟨rfl, n, hn, hne⟩
    · intro hk
      exact ((mem_botElem k).mp hk).elim

end charOpen
```

```
def charTop (U : Set Pomega) (x : Pomega) : Pomega :=
  {_k | ∃ n, e n ⊆ x ∧ e n ∈ U}

namespace charTop

theorem isScottContinuous (U : Set Pomega) :
    IsScottContinuous (charTop U) := by
  constructor
  intro x
  ext k
  constructor
  · intro ⟨n, hn, hne⟩
    exact mem_scottUnion.mpr ⟨n, hn, n, subset_rfl, hne⟩
  · intro hk
    obtain ⟨m, hm, n, hn, hne⟩ := mem_scottUnion.mp hk
    exact ⟨n, subset_trans hn hm, hne⟩

theorem eq_top {U : Set Pomega} (hU : IsScottOpen U) (x : Pomega) :
    charTop U x = topElem ↔ x ∈ U := by
  constructor
  · intro h
    have : 0 ∈ charTop U x := by rw [h]; exact Set.mem_univ 0
    obtain ⟨n, hn, hne⟩ := this
    exact isScottOpen_isUpper hU hne hn
  · intro hx
    ext k
    constructor
    · intro _; trivial
    · intro _
      exact (IsScottOpen.mem_iff hU).mp hx

theorem eq_bot {U : Set Pomega} (hU : IsScottOpen U) (x : Pomega) :
    charTop U x = botElem ↔ x ∈ U := by
  constructor
  · intro h hx
    have : charTop U x = topElem := (eq_top hU x).mpr hx
    rw [h] at this
    have : (0 : ℕ) ∈ (botElem : Pomega) := by
      rw [this]; exact Set.mem_univ 0
    exact (mem_botElem 0).mp this
  · intro hx
    ext k
    constructor
    · intro ⟨n, hn, hne⟩
      exact hx (isScottOpen_isUpper hU hne hn)
    · intro hk
      exact ((mem_botElem k).mp hk).elim

theorem eq_bot_of_closed {C : Set Pomega} (hC : IsScottOpen C?) (x : Pomega) :
    charTop C? x = botElem ↔ x ∈ C := by
  rw [eq_bot hC]
  simp

end charTop

def closedToZero (C : Set Pomega) (x : Pomega) : Pomega :=
```

```
  ofNat 0 ∪ charTop C? x

namespace closedToZero

theorem isScottContinuous (C : Set Pomega) :
    IsScottContinuous (closedToZero C) :=
  theorem_1_3.tuple (f := fun a b => (a ∪ b : Pomega))
    union_left_isScottContinuous
    (fun x => by
      convert union_left_isScottContinuous x using 1
      funext y; exact (Set.union_comm y x).symm)
    (const_isScottContinuous (ofNat 0)) (charTop.isScottContinuous C?)

theorem threeValued {C : Set Pomega} (hC : IsScottOpen C?) (x : Pomega) :
    ThreeValued (closedToZero C x) := by
  constructor
  by_cases hx : x ∈ C
  · refine Or.inr (Or.inl ?_)
    have : charTop C? x = botElem := (charTop.eq_bot_of_closed hC x).mpr hx
    ext k
    simp [closedToZero, this, ofNat, botElem]
  · refine Or.inr (Or.inr ?_)
    have : charTop C? x = topElem := (charTop.eq_top hC x).mpr hx
    ext k
    simp [closedToZero, this, ofNat, topElem]

theorem eq_zero {C : Set Pomega} (hC : IsScottOpen C?) (x : Pomega) :
    closedToZero C x = ofNat 0 ↔ x ∈ C := by
  constructor
  · intro h
    by_contra hx
    have htop : charTop C? x = topElem := (charTop.eq_top hC x).mpr hx
    have : closedToZero C x = topElem := by
      ext k
      simp [closedToZero, htop]
    exact topElem.ne_ofNat_zero (this.symm.trans h)
  · intro hx
    have : charTop C? x = botElem := (charTop.eq_bot_of_closed hC x).mpr hx
    ext k
    simp [closedToZero, this, ofNat, botElem]

end closedToZero

end DomainTheory.DataTypesAsLattices
```

3.7.4 Odd-even packing ✓

Odd–even packing interlaces two maps on the odd and even numerals and recovers each factor by parity projection.

Definition (`oddPart`). $\mathrm{oddPart}\, y = \{n \mid 2n+1 \in y\}$.

Definition (`evenPart`). $\mathrm{evenPart}\, y = \{n \mid 2n \in y\}$.

Definition (`oddEvenPack`).

$$(\mathrm{oddEvenPack}\, f\, g)\, x = \{k \mid (\exists n.\, k = 2n+1 \wedge n \in f(x)) \vee (\exists n.\, k = 2n \wedge n \in g(x))\}.$$

Theorem (`oddEvenPack.eq_zero`). $\mathrm{oddEvenPack}\ f\ g\ x = 0$ if and only if $f(x) = \bot$ and $g(x) = 0$.

Proof. An odd packed index forces $f(x)$ empty; an even packed index forces $g(x)$ to be the singleton $\{0\}$; the converse packs those values back to the numeral 0. ■

Theorem (`oddEvenPack.isScottContinuous`). If f and g are Scott-continuous, so is $\mathrm{oddEvenPack}\ f\ g$.

Proof. Each parity clause pulls a Scott-union witness from f or g and pushes it forward; monotonicity of f and g closes the converse. ■

Theorem (`oddEvenPack.evenPart`). $\mathrm{evenPart}(\mathrm{oddEvenPack}\ f\ g\ x) = g(x)$.

Proof. Even indices unpack exactly the members of $g(x)$; odd indices cannot contribute. ■

Theorem (`oddEvenPack.oddPart`). $\mathrm{oddPart}(\mathrm{oddEvenPack}\ f\ g\ x) = f(x)$.

Proof. Symmetric to the even case. ■

Theorem (`oddEvenPack.subset`). If $f(x) \subseteq \{m \mid m < N\}$ and $g(x) \subseteq \{m \mid m < M\}$, then $\mathrm{oddEvenPack}\ f\ g\ x \subseteq \{k \mid k < 2\max(N, M) + 1\}$.

Proof. An odd or even packed index is strictly less than twice the corresponding bound, plus one on the odd side. ■

`OddEvenPack.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.Language
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def oddPart (y : Pomega) : Pomega := {n | 2 * n + 1 ∈ y}

def evenPart (y : Pomega) : Pomega := {n | 2 * n ∈ y}

def oddEvenPack (f g : Pomega → Pomega) (x : Pomega) : Pomega :=
  {k | (∃ n, k = 2 * n + 1 ∧ n ∈ f x) ∨ (∃ n, k = 2 * n ∧ n ∈ g x)}

namespace oddEvenPack

theorem eq_zero {f g : Pomega → Pomega} (x : Pomega) :
    oddEvenPack f g x = ofNat 0 ↔ f x = botElem ∧ g x = ofNat 0 := by
  constructor
  · intro h
    constructor
    · ext n
      constructor
      · intro hn
        have : 2 * n + 1 ∈ oddEvenPack f g x := Or.inl ⟨n, rfl, hn⟩
        rw [h] at this
        simp [ofNat] at this
      · intro hn
        exact ((mem_botElem n).mp hn).elim
```

```
    · ext n
      constructor
      · intro hn
        have hmem : 2 * n ∈ oddEvenPack f g x := Or.inr ⟨n, rfl, hn⟩
        rw [h] at hmem
        have : 2 * n = 0 := by simpa [ofNat] using hmem
        have hn0 : n = 0 := by
          have := Nat.mul_eq_zero.mp this
          exact this.resolve_left (by decide)
        simpa [ofNat] using hn0
      · intro hn
        simp [ofNat] at hn
        subst hn
        have : (0 : ℕ) ∈ oddEvenPack f g x := by rw [h]; simp [ofNat]
        rcases this with ⟨n, hk, _⟩ | ⟨n, hk, hn⟩
        · omega
        · have : n = 0 := by omega
          simpa [this] using hn
  · intro ⟨hf, hg⟩
    ext k
    constructor
    · intro hk
      rcases hk with ⟨n, hk, hn⟩ | ⟨n, hk, hn⟩
      · rw [hf] at hn; exact ((mem_botElem n).mp hn).elim
      · rw [hg] at hn; simp [ofNat] at hn; simp [ofNat, hk, hn]
    · intro hk
      simp [ofNat] at hk
      subst hk
      refine Or.inr ⟨0, rfl, ?_⟩
      rw [hg]; simp [ofNat]

theorem isScottContinuous {f g : Pomega → Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
    IsScottContinuous (oddEvenPack f g) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    rcases hk with ⟨n, hk, hn⟩ | ⟨n, hk, hn⟩
    · have : n ∈ scottUnion f x := by rw [← hf.out x]; exact hn
      obtain ⟨p, hp, hnp⟩ := mem_scottUnion.mp this
      exact mem_scottUnion.mpr ⟨p, hp, Or.inl ⟨n, hk, hnp⟩⟩
    · have : n ∈ scottUnion g x := by rw [← hg.out x]; exact hn
      obtain ⟨p, hp, hnp⟩ := mem_scottUnion.mp this
      exact mem_scottUnion.mpr ⟨p, hp, Or.inr ⟨n, hk, hnp⟩⟩
  · intro hk
    obtain ⟨p, hp, h⟩ := mem_scottUnion.mp hk
    rcases h with ⟨n, hk, hn⟩ | ⟨n, hk, hn⟩
    · exact Or.inl ⟨n, hk, IsScottContinuous.monotone hf hp hn⟩
    · exact Or.inr ⟨n, hk, IsScottContinuous.monotone hg hp hn⟩

theorem evenPart (f g : Pomega → Pomega) (x : Pomega) :
    evenPart (oddEvenPack f g x) = g x := by
  ext n
  constructor
```

```
  · intro hn
    rcases hn with ⟨k, hk, hkn⟩ | ⟨k, hk, hkn⟩
    · have : 2 * n = 2 * k + 1 := hk
      omega
    · have : k = n := by
        have : 2 * n = 2 * k := hk
        omega
      simpa [this] using hkn
  · intro hn
    exact Or.inr ⟨n, rfl, hn⟩

theorem oddPart (f g : Pomega → Pomega) (x : Pomega) :
    oddPart (oddEvenPack f g x) = f x := by
  ext n
  constructor
  · intro hn
    rcases hn with ⟨k, hk, hkn⟩ | ⟨k, hk, hkn⟩
    · have : k = n := by
        have : 2 * n + 1 = 2 * k + 1 := hk
        omega
      simpa [this] using hkn
    · have : 2 * n + 1 = 2 * k := hk
      omega
  · intro hn
    exact Or.inl ⟨n, rfl, hn⟩

theorem subset {f g : Pomega → Pomega} {x : Pomega} {N M : ℕ}
    (hf : f x ⊆ {m | m < N}) (hg : g x ⊆ {m | m < M}) :
    oddEvenPack f g x ⊆ {k | k < 2 * Nat.max N M + 1} := by
  intro k hk
  rcases hk with ⟨n, rfl, hn⟩ | ⟨n, rfl, hn⟩
  · have hnN : n < N := hf hn
    have : n < Nat.max N M := lt_of_lt_of_le hnN (Nat.le_max_left _ _)
    exact Nat.succ_lt_succ (Nat.mul_lt_mul_of_pos_left this (by decide : 0 < 2))
  · have hnM : n < M := hg hn
    have hlt : n < Nat.max N M := lt_of_lt_of_le hnM (Nat.le_max_right _ _)
    have : 2 * n < 2 * Nat.max N M + 1 :=
      Nat.lt_succ_of_le (Nat.mul_le_mul_left 2 (Nat.le_of_lt hlt))
    exact this

end oddEvenPack

end DomainTheory.DataTypesAsLattices
```

3.7.5 Pair indexing ✓

Pair indexing packs a sequence of maps into one map via Cantor pairing of fibre indices with values.

Definition (`pairIndexed`).

$$(\mathrm{pairIndexed}\, f_s)\, x = \{k \mid \exists n\, m.\, k = \langle n, m\rangle \wedge m \in f_s(n)(x)\}.$$

Theorem (`pairIndexed.eq_top`). $\mathrm{pairIndexed}\, f_s\, x = \top$ if and only if every $f_s(n)(x)$ equals $\top$.

Proof. Every pair code belongs to the packed value precisely when each fibre is universal; surjectivity of pairing recovers the fibres. ■

Theorem (`pairIndexed.isScottContinuous`). If each $f_s(n)$ is Scott-continuous, so is pairIndexed f_s.

Proof. Membership of a pair code pulls a Scott-union witness from the corresponding fibre and reassembles under the packed Scott union; monotonicity of each fibre closes the converse. ■

`pairIndexed.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.pair
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def pairIndexed (fs : ℕ → Pomega → Pomega) (x : Pomega) : Pomega :=
  {k | ∃ n m, k = pair n m ∧ m ∈ fs n x}

namespace pairIndexed

theorem eq_top (fs : ℕ → Pomega → Pomega) (x : Pomega) :
    pairIndexed fs x = topElem ↔ ∀ n, fs n x = topElem := by
  constructor
  · intro h n
    ext m
    constructor
    · intro _; trivial
    · intro _
      have : pair n m ∈ pairIndexed fs x := by rw [h]; exact Set.mem_univ _
      obtain ⟨n', m', heq, hm⟩ := this
      obtain ⟨rfl, rfl⟩ := pair_inj heq
      exact hm
  · intro h
    ext k
    constructor
    · intro _; trivial
    · intro _
      obtain ⟨n, m, hk⟩ := exists_pair k
      exact ⟨n, m, hk.symm, by rw [h n]; exact Set.mem_univ m⟩

theorem isScottContinuous (fs : ℕ → Pomega → Pomega)
    (hfs : ∀ n, IsScottContinuous (fs n)) :
    IsScottContinuous (pairIndexed fs) := by
  constructor
  intro x
  ext k
  constructor
  · intro ⟨n, m, hk, hm⟩
    have : m ∈ scottUnion (fs n) x := by rw [← (hfs n).out x]; exact hm
    obtain ⟨p, hp, hmp⟩ := mem_scottUnion.mp this
    exact mem_scottUnion.mpr ⟨p, hp, n, m, hk, hmp⟩
  · intro hk
```

```
    obtain ⟨p, hp, n, m, hk, hm⟩ := mem_scottUnion.mp hk
    exact ⟨n, m, hk, IsScottContinuous.monotone (hfs n) hp hm⟩

end pairIndexed

end DomainTheory.DataTypesAsLattices
```

3.7.6 F∩Gδ packing ✓

The $\mathfrak{F}\dot{\cap}\mathfrak{G}_\delta$ packing combines a map that detects a closed fibre with one that detects a $\mathfrak{G}_\delta$ fibre, encoding the closed side at 0 and the open sequence on positive numerals.

Definition (`fcapGdeltaPack`).

$$(\mathrm{fcapGdeltaPack}\, f\, g)\, x = \{k \mid (k = 0 \wedge 0 \in f(x)) \vee (\exists n.\, k = n + 1 \wedge n \in g(x))\}.$$

Theorem (`fcapGdeltaPack.isScottContinuous`). If f and g are Scott-continuous, so is $\mathrm{fcapGdeltaPack}\, f\, g$.

Proof. The clause $0 \in f(x)$ and each clause $n \in g(x)$ pull Scott-union witnesses from f or g; monotonicity of f and g closes the converse. ■

Theorem (`fcapGdeltaPack.eq`). Suppose $f(x)$ is $\bot$ or $\top$. Then $\mathrm{fcapGdeltaPack}\, f\, g\, x = \mathrm{succSet}\, \top$ if and only if $f(x) = \bot$ and $g(x) = \top$.

Proof. The successor of $\top$ never contains 0, so a packed value equal to $\mathrm{succSet}\, \top$ forces $0 \notin f(x)$, hence $f(x) = \bot$ under the two-point hypothesis, while positive packed indices force $g(x) = \top$. Conversely, with $f(x) = \bot$ and $g(x) = \top$ the packed value is exactly the positive numerals. ■

`fcapGdeltaPack.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.Language
import DomainTheory.DataTypesAsLattices.Classification.ThreeValued
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def fcapGdeltaPack (f g : Pomega → Pomega) (x : Pomega) : Pomega :=
  {k | (k = 0 ∧ 0 ∈ f x) ∨ ∃ n, k = n + 1 ∧ n ∈ g x}

namespace fcapGdeltaPack

theorem isScottContinuous {f g : Pomega → Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
    IsScottContinuous (fcapGdeltaPack f g) := by
  constructor
  intro x
  ext k
```

```
  constructor
  · intro hk
    rcases hk with ⟨hk0, h0⟩ | ⟨n, hk, hn⟩
    · have : 0 ∈ scottUnion f x := by rw [← hf.out x]; exact h0
      obtain ⟨p, hp, hp0⟩ := mem_scottUnion.mp this
      exact mem_scottUnion.mpr ⟨p, hp, Or.inl ⟨hk0, hp0⟩⟩
    · have : n ∈ scottUnion g x := by rw [← hg.out x]; exact hn
      obtain ⟨p, hp, hnp⟩ := mem_scottUnion.mp this
      exact mem_scottUnion.mpr ⟨p, hp, Or.inr ⟨n, hk, hnp⟩⟩
  · intro hk
    obtain ⟨p, hp, h⟩ := mem_scottUnion.mp hk
    rcases h with ⟨hk0, h0⟩ | ⟨n, hk, hn⟩
    · exact Or.inl ⟨hk0, IsScottContinuous.monotone hf hp h0⟩
    · exact Or.inr ⟨n, hk, IsScottContinuous.monotone hg hp hn⟩

theorem eq {f g : Pomega → Pomega} {x : Pomega}
    (hfval : f x = botElem ∨ f x = topElem) :
    fcapGdeltaPack f g x = succSet topElem ↔ f x = botElem ∧ g x = topElem := by
  constructor
  · intro h
    constructor
    · have h0 : 0 ∈ fcapGdeltaPack f g x := by
        rw [h]; exact zero_not_mem_succSet_top
      have : 0 ∈ f x := fun hf0 => h0 (Or.inl ⟨rfl, hf0⟩)
      rcases hfval with hf | hf
      · exact hf
      · exact (this (by rw [hf]; exact Set.mem_univ 0)).elim
    · ext n
      constructor
      · intro _; trivial
      · intro _
        have : n + 1 ∈ fcapGdeltaPack f g x := by
          rw [h]; exact ⟨n, Set.mem_univ n, rfl⟩
        rcases this with ⟨hk0, _⟩ | ⟨m, hm, hmn⟩
        · exact (Nat.succ_ne_zero n (by simp [hk0])).elim
        · exact Nat.succ_injective hm ▸ hmn
  · intro ⟨hf, hg⟩
    ext k
    constructor
    · intro hk
      rcases hk with ⟨_, h0⟩ | ⟨n, hk, hn⟩
      · rw [hf] at h0; exact ((mem_botElem 0).mp h0).elim
      · exact ⟨n, Set.mem_univ n, hk⟩
    · intro hk
      obtain ⟨n, _, rfl⟩ := hk
      refine Or.inr ⟨n, rfl, ?_⟩
      rw [hg]; exact Set.mem_univ n

end fcapGdeltaPack

end DomainTheory.DataTypesAsLattices
```

3.7.7 Three-valued ✓

Three-valued elements of $\mathbf{P}\omega$ are $\bot$, the numeral 0, and $\top$. The auxiliary maps uvU and uvV agree on three-valued arguments precisely when at least one argument is 0.

Definition (`ThreeValued`). An element is three-valued when it equals $\bot$, 0, or $\top$.

Definition (`zeroPrime`). $\mathrm{zeroPrime} = 0 \cup 1$.

Definition (`uvU`, `uvV`). Two continuous binary combinations of arguments used to detect equalizers among three-valued values (Scott's U and V tables).

Theorem (`zeroPrime.ne_ofNat_zero`). $\mathrm{zeroPrime} \neq 0$.

Proof. $1 \in \mathrm{zeroPrime}$ but $1 \notin 0$. ■

Theorem (`zeroPrime.ne_top`). $\mathrm{zeroPrime} \neq \top$.

Proof. $2 \notin \mathrm{zeroPrime}$. ■

Theorem (`uvU.mono_left`, `uvU.mono_right`, `uvV.mono_left`, `uvV.mono_right`). Each of uvU and uvV is monotone in each argument separately.

Proof. Enlarging an argument only adds positive membership witnesses used by the clauses. ■

Theorem (`uvU.isScottContinuous_left`, `uvU.isScottContinuous_right`, `uvV.isScottContinuous_left`, `uvV.isScottContinuous_right`). Fixing either argument, both maps are Scott-continuous in the other.

Proof. Each clause that places 0 or 1 in the value is witnessed by a finite e_n from the varying argument; Scott unions reassemble those witnesses, and monotonicity closes the converse. ■

Theorem (`uvU.bot_bot`, `uvU.bot_zero`, `uvU.zero_bot`, `uvU.zero_zero`, `uvU.bot_top`, `uvU.top_bot`, `uvU.zero_top`, `uvU.top_zero`, `uvU.top_top`). The nine three-valued evaluations of uvU match Scott's U-table.

Proof. Direct case analysis on membership of 0 and of positive numerals in $\bot$, 0, and $\top$. ■

Theorem (`uvV.bot_bot`, `uvV.bot_zero`, `uvV.zero_bot`, `uvV.zero_zero`, `uvV.bot_top`, `uvV.top_bot`, `uvV.zero_top`, `uvV.top_zero`, `uvV.top_top`). The nine three-valued evaluations of uvV match Scott's V-table.

Proof. The same case analysis for the clauses defining uvV. ■

Theorem (`ThreeValued.uvU_eq_uvV`). If x and y are three-valued, then $\mathrm{uvU}\,x\,y = \mathrm{uvV}\,x\,y$ if and only if $x = 0$ or $y = 0$.

Proof. Evaluate the nine cases against the tables and the inequalities among $\bot$, 0, $\mathrm{zeroPrime}$, and $\top$. ■

`ThreeValued.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.Language
import DomainTheory.DataTypesAsLattices.Closures.botElem
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
import DomainTheory.DataTypesAsLattices.Lambda.ofNat_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def zeroPrime : Pomega := ofNat 0 ∪ ofNat 1
```

```
namespace zeroPrime

theorem ne_ofNat_zero : zeroPrime = ofNat 0 := by
  intro h
  have : (1 : ℕ) ∈ ofNat 0 := by
    have : (1 : ℕ) ∈ zeroPrime := Or.inr rfl
    rwa [h] at this
  simp [ofNat] at this

theorem ne_top : zeroPrime = topElem := by
  intro h
  have : (2 : ℕ) ∈ zeroPrime := by rw [h]; exact Set.mem_univ 2
  simp [zeroPrime, ofNat] at this

end zeroPrime

theorem zero_not_mem_succSet_top : 0 ∈ succSet topElem := by
  intro h
  obtain ⟨n, _, hn⟩ := h
  exact Nat.succ_ne_zero n hn.symm

def uvU (x y : Pomega) : Pomega :=
  {k | (k = 0 ∧ (x.Nonempty ∨ y.Nonempty)) ∨
       (k = 1 ∧ ((0 ∈ x ∧ ∃ n, n + 1 ∈ y) ∨ ((∃ n, n + 1 ∈ x) ∧ 0 ∈ y)))}

namespace uvU

theorem mono_left {x x' y : Pomega} (h : x ⊆ x') : uvU x y ⊆ uvU x' y := by
  intro k hk
  rcases hk with ⟨rfl, hne⟩ | ⟨rfl, h1⟩
  · exact Or.inl ⟨rfl, hne.imp_left fun ⟨n, hn⟩ => ⟨n, h hn⟩⟩
  · rcases h1 with h10 | ⟨⟨n, hn⟩, h0⟩
    · exact Or.inr ⟨rfl, Or.inl ⟨h h10.1, h10.2⟩⟩
    · exact Or.inr ⟨rfl, Or.inr ⟨⟨n, h hn⟩, h0⟩⟩

theorem mono_right {x y y' : Pomega} (h : y ⊆ y') : uvU x y ⊆ uvU x y' := by
  intro k hk
  rcases hk with ⟨rfl, hne⟩ | ⟨rfl, h1⟩
  · exact Or.inl ⟨rfl, hne.imp_right fun ⟨n, hn⟩ => ⟨n, h hn⟩⟩
  · rcases h1 with ⟨h0, ⟨n, hn⟩⟩ | h10
    · exact Or.inr ⟨rfl, Or.inl ⟨h0, ⟨n, h hn⟩⟩⟩
    · exact Or.inr ⟨rfl, Or.inr ⟨h10.1, h h10.2⟩⟩

theorem isScottContinuous_left (y : Pomega) :
    IsScottContinuous (fun x => uvU x y) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    rcases hk with ⟨rfl, hne⟩ | ⟨rfl, h1⟩
    · rcases hne with ⟨n, hn⟩ | hy
      · exact mem_scottUnion.mpr ⟨2 ^ n, by simp [e.pow2, Set.singleton_subset_iff, hn
    ],
          Or.inl ⟨rfl, Or.inl ⟨n, by simp [e.pow2]⟩⟩⟩
      · exact mem_scottUnion.mpr ⟨0, by simp [e.zero], Or.inl ⟨rfl, Or.inr hy⟩⟩
```

```
    · rcases h1 with ⟨h0, hp⟩ | ⟨⟨n, hn⟩, h0⟩
      · exact mem_scottUnion.mpr ⟨1, by simp [e_one, h0],
          Or.inr ⟨rfl, Or.inl ⟨by simp [e_one], hp⟩⟩⟩
      · exact mem_scottUnion.mpr ⟨2 ^ (n + 1), by simp [e.pow2, Set.
    singleton_subset_iff, hn],
          Or.inr ⟨rfl, Or.inr ⟨⟨n, by simp [e.pow2]⟩, h0⟩⟩⟩
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    exact mono_left hm hkm

theorem isScottContinuous_right (x : Pomega) :
    IsScottContinuous (fun y => uvU x y) := by
  constructor
  intro y
  ext k
  constructor
  · intro hk
    rcases hk with ⟨rfl, hne⟩ | ⟨rfl, h1⟩
    · rcases hne with hx | ⟨n, hn⟩
      · exact mem_scottUnion.mpr ⟨0, by simp [e.zero], Or.inl ⟨rfl, Or.inl hx⟩⟩
      · exact mem_scottUnion.mpr ⟨2 ^ n, by simp [e.pow2, Set.singleton_subset_iff, hn
    ],
          Or.inl ⟨rfl, Or.inr ⟨n, by simp [e.pow2]⟩⟩⟩
    · rcases h1 with ⟨h0, ⟨n, hn⟩⟩ | ⟨hp, h0⟩
      · exact mem_scottUnion.mpr ⟨2 ^ (n + 1), by simp [e.pow2, Set.
    singleton_subset_iff, hn],
          Or.inr ⟨rfl, Or.inl ⟨h0, ⟨n, by simp [e.pow2]⟩⟩⟩⟩
      · exact mem_scottUnion.mpr ⟨1, by simp [e_one, h0],
          Or.inr ⟨rfl, Or.inr ⟨hp, by simp [e_one]⟩⟩⟩
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    exact mono_right hm hkm

theorem bot_bot : uvU botElem botElem = botElem := by
  ext k
  simp [uvU, botElem]

theorem bot_zero : uvU botElem (ofNat 0) = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨hk1, h1⟩
    · simpa [ofNat] using hk0
    · rcases h1 with ⟨h0, _⟩ | ⟨⟨n, hn⟩, _⟩
      · exact ((mem_botElem 0).mp h0).elim
      · exact ((mem_botElem (n + 1)).mp hn).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl ⟨hk, Or.inr ofNat.zero_nonempty⟩

theorem zero_bot : uvU (ofNat 0) botElem = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨_, h1⟩
    · simpa [ofNat] using hk0
```

```
    · rcases h1 with ⟨_, ⟨n, hn⟩⟩ | ⟨⟨n, hn⟩, _⟩
      · exact ((mem_botElem (n + 1)).mp hn).elim
      · exact (ofNat.zero_not_pos ⟨n, hn⟩).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl ⟨hk, Or.inl ofNat.zero_nonempty⟩

theorem zero_zero : uvU (ofNat 0) (ofNat 0) = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨_, h1⟩
    · simpa [ofNat] using hk0
    · rcases h1 with ⟨_, hp⟩ | ⟨hp, _⟩
      · exact (ofNat.zero_not_pos hp).elim
      · exact (ofNat.zero_not_pos hp).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl ⟨hk, Or.inl ofNat.zero_nonempty⟩

theorem bot_top : uvU botElem topElem = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨_, h1⟩
    · simpa [ofNat] using hk0
    · rcases h1 with ⟨h0, _⟩ | ⟨⟨n, hn⟩, _⟩
      · exact ((mem_botElem 0).mp h0).elim
      · exact ((mem_botElem (n + 1)).mp hn).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl ⟨hk, Or.inr ⟨0, Set.mem_univ 0⟩⟩

theorem top_bot : uvU topElem botElem = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨_, h1⟩
    · simpa [ofNat] using hk0
    · rcases h1 with ⟨_, ⟨n, hn⟩⟩ | ⟨_, h0⟩
      · exact ((mem_botElem (n + 1)).mp hn).elim
      · exact ((mem_botElem 0).mp h0).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl ⟨hk, Or.inl ⟨0, Set.mem_univ 0⟩⟩

theorem zero_top : uvU (ofNat 0) topElem = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨hk1, _⟩
    · exact Or.inl hk0
    · exact Or.inr hk1
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl ⟨hk0, Or.inr ⟨0, Set.mem_univ 0⟩⟩
```

```
    · exact Or.inr ⟨hk1, Or.inl ⟨rfl, topElem.pos⟩⟩

theorem top_zero : uvU topElem (ofNat 0) = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨hk1, _⟩
    · exact Or.inl hk0
    · exact Or.inr hk1
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl ⟨hk0, Or.inl ⟨0, Set.mem_univ 0⟩⟩
    · exact Or.inr ⟨hk1, Or.inr ⟨topElem.pos, rfl⟩⟩

theorem top_top : uvU topElem topElem = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨hk1, _⟩
    · exact Or.inl hk0
    · exact Or.inr hk1
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl ⟨hk0, Or.inl ⟨0, Set.mem_univ 0⟩⟩
    · exact Or.inr ⟨hk1, Or.inl ⟨Set.mem_univ 0, topElem.pos⟩⟩

end uvU

def uvV (x y : Pomega) : Pomega :=
  {k | k = 0 ∨
       (k = 1 ∧ ((∃ n, n + 1 ∈ x) ∨ (∃ n, n + 1 ∈ y))) ∨
       (0 ∈ x ∧ (∃ n, n + 1 ∈ x) ∧ 0 ∈ y ∧ (∃ n, n + 1 ∈ y))}

namespace uvV

theorem mono_left {x x' y : Pomega} (h : x ⊆ x') : uvV x y ⊆ uvV x' y := by
  intro k hk
  rcases hk with hk0 | ⟨hk1, hpos⟩ | ⟨h0, ⟨n, hn⟩, h0y, hpy⟩
  · exact Or.inl hk0
  · exact Or.inr (Or.inl ⟨hk1, hpos.imp_left fun ⟨n, hn⟩ => ⟨n, h hn⟩⟩)
  · exact Or.inr (Or.inr ⟨h h0, ⟨n, h hn⟩, h0y, hpy⟩)

theorem mono_right {x y y' : Pomega} (h : y ⊆ y') : uvV x y ⊆ uvV x y' := by
  intro k hk
  rcases hk with hk0 | ⟨hk1, hpos⟩ | ⟨h0, hpx, h0y, ⟨n, hn⟩⟩
  · exact Or.inl hk0
  · exact Or.inr (Or.inl ⟨hk1, hpos.imp_right fun ⟨n, hn⟩ => ⟨n, h hn⟩⟩)
  · exact Or.inr (Or.inr ⟨h0, hpx, h h0y, ⟨n, h hn⟩⟩)

theorem isScottContinuous_left (y : Pomega) :
    IsScottContinuous (fun x => uvV x y) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
```

```
    rcases hk with hk0 | ⟨rfl, hpos⟩ | ⟨h0, ⟨n, hn⟩, h0y, hpy⟩
    · exact mem_scottUnion.mpr ⟨0, by simp [e.zero], Or.inl hk0⟩
    · rcases hpos with ⟨n, hn⟩ | hy
      · exact mem_scottUnion.mpr ⟨2 ^ (n + 1), by simp [e.pow2, Set.
    singleton_subset_iff, hn],
          Or.inr (Or.inl ⟨rfl, Or.inl ⟨n, by simp [e.pow2]⟩⟩)⟩
      · exact mem_scottUnion.mpr ⟨0, by simp [e.zero],
          Or.inr (Or.inl ⟨rfl, Or.inr hy⟩)⟩
    · refine mem_scottUnion.mpr ⟨1 ||| 2 ^ (n + 1), e.or_of_subset
        (by simp [e_one, Set.singleton_subset_iff, h0])
        (by simp [e.pow2, Set.singleton_subset_iff, hn]), ?_⟩
      refine Or.inr (Or.inr ⟨?_, ⟨n, ?_⟩, h0y, hpy⟩)
      · simp [e.or, e_one]
      · simp [e.or, e.pow2]
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    exact mono_left hm hkm

theorem isScottContinuous_right (x : Pomega) :
    IsScottContinuous (fun y => uvV x y) := by
  constructor
  intro y
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨rfl, hpos⟩ | ⟨h0, hpx, h0y, ⟨n, hn⟩⟩
    · exact mem_scottUnion.mpr ⟨0, by simp [e.zero], Or.inl hk0⟩
    · rcases hpos with hx | ⟨n, hn⟩
      · exact mem_scottUnion.mpr ⟨0, by simp [e.zero],
          Or.inr (Or.inl ⟨rfl, Or.inl hx⟩)⟩
      · exact mem_scottUnion.mpr ⟨2 ^ (n + 1), by simp [e.pow2, Set.
    singleton_subset_iff, hn],
          Or.inr (Or.inl ⟨rfl, Or.inr ⟨n, by simp [e.pow2]⟩⟩)⟩
    · refine mem_scottUnion.mpr ⟨1 ||| 2 ^ (n + 1), e.or_of_subset
        (by simp [e_one, Set.singleton_subset_iff, h0y])
        (by simp [e.pow2, Set.singleton_subset_iff, hn]), ?_⟩
      refine Or.inr (Or.inr ⟨h0, hpx, ?_, ⟨n, ?_⟩⟩)
      · simp [e.or, e_one]
      · simp [e.or, e.pow2]
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    exact mono_right hm hkm

theorem bot_bot : uvV botElem botElem = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, hpos⟩ | htop
    · simpa [ofNat] using hk0
    · rcases hpos with ⟨n, hn⟩ | ⟨n, hn⟩ <;> exact ((mem_botElem (n + 1)).mp hn).elim
    · exact ((mem_botElem 0).mp htop.1).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl hk

theorem bot_zero : uvV botElem (ofNat 0) = ofNat 0 := by
```

```
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, hpos⟩ | htop
    · simpa [ofNat] using hk0
    · rcases hpos with ⟨n, hn⟩ | hp
      · exact ((mem_botElem (n + 1)).mp hn).elim
      · exact (ofNat.zero_not_pos hp).elim
    · exact ((mem_botElem 0).mp htop.1).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl hk

theorem zero_bot : uvV (ofNat 0) botElem = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨_, hpos⟩ | htop
    · simpa [ofNat] using hk0
    · rcases hpos with hp | ⟨n, hn⟩
      · exact (ofNat.zero_not_pos hp).elim
      · exact ((mem_botElem (n + 1)).mp hn).elim
    · exact ((mem_botElem 0).mp htop.2.2.1).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl hk

theorem zero_zero : uvV (ofNat 0) (ofNat 0) = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨_, hpos⟩ | htop
    · simpa [ofNat] using hk0
    · rcases hpos with hp | hp
      · exact (ofNat.zero_not_pos hp).elim
      · exact (ofNat.zero_not_pos hp).elim
    · exact (ofNat.zero_not_pos htop.2.1).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl hk

theorem bot_top : uvV botElem topElem = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, _⟩ | htop
    · exact Or.inl hk0
    · exact Or.inr hk1
    · exact ((mem_botElem 0).mp htop.1).elim
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl hk0
    · exact Or.inr (Or.inl ⟨hk1, Or.inr topElem.pos⟩)

theorem top_bot : uvV topElem botElem = zeroPrime := by
  ext k
```

```
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, _⟩ | htop
    · exact Or.inl hk0
    · exact Or.inr hk1
    · exact ((mem_botElem 0).mp htop.2.2.1).elim
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl hk0
    · exact Or.inr (Or.inl ⟨hk1, Or.inl topElem.pos⟩)

theorem zero_top : uvV (ofNat 0) topElem = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, _⟩ | htop
    · exact Or.inl hk0
    · exact Or.inr hk1
    · exact (ofNat.zero_not_pos htop.2.1).elim
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl hk0
    · exact Or.inr (Or.inl ⟨hk1, Or.inr topElem.pos⟩)

theorem top_zero : uvV topElem (ofNat 0) = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, _⟩ | htop
    · exact Or.inl hk0
    · exact Or.inr hk1
    · exact (ofNat.zero_not_pos htop.2.2.2).elim
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl hk0
    · exact Or.inr (Or.inl ⟨hk1, Or.inl topElem.pos⟩)

theorem top_top : uvV topElem topElem = topElem := by
  ext k
  constructor
  · intro _; exact Set.mem_univ k
  · intro _
    exact Or.inr (Or.inr ⟨Set.mem_univ 0, topElem.pos, Set.mem_univ 0, topElem.pos⟩)

end uvV

class ThreeValued (x : Pomega) : Prop where
  out : x = botElem ∨ x = ofNat 0 ∨ x = topElem

namespace ThreeValued

theorem uvU_eq_uvV {x y : Pomega} (hx : ThreeValued x) (hy : ThreeValued y) :
    uvU x y = uvV x y ↔ x = ofNat 0 ∨ y = ofNat 0 := by
  rcases hx.out with rfl | rfl | rfl <;> rcases hy.out with rfl | rfl | rfl


  · simp [uvU.bot_bot, uvV.bot_bot, botElem.ne_ofNat_zero]
```

```
  · simp [uvU.bot_zero, uvV.bot_zero]
  · constructor
    · intro h
      rw [uvU.bot_top, uvV.bot_top] at h
      exact (zeroPrime.ne_ofNat_zero h.symm).elim
    · intro h
      rcases h with h | h
      · exact (botElem.ne_ofNat_zero h).elim
      · exact (topElem.ne_ofNat_zero h).elim
  · simp [uvU.zero_bot, uvV.zero_bot]
  · simp [uvU.zero_zero, uvV.zero_zero]
  · simp [uvU.zero_top, uvV.zero_top]
  · constructor
    · intro h
      rw [uvU.top_bot, uvV.top_bot] at h
      exact (zeroPrime.ne_ofNat_zero h.symm).elim
    · intro h
      rcases h with h | h
      · exact (topElem.ne_ofNat_zero h).elim
      · exact (botElem.ne_ofNat_zero h).elim
  · simp [uvU.top_zero, uvV.top_zero]
  · constructor
    · intro h
      rw [uvU.top_top, uvV.top_top] at h
      exact (zeroPrime.ne_top h).elim
    · intro h
      rcases h with h | h <;> exact (topElem.ne_ofNat_zero h).elim

end ThreeValued

end DomainTheory.DataTypesAsLattices
```

3.7.8 Boolean membership ✓

Boolean membership packages a Scott-continuous map of uniformly bounded values together with a finite set of admissible values inside that bound; the class contains every Scott-open set and is closed under the Boolean operations.

Definition (`BooleanMemE`). A set U has Boolean membership when there exist a Scott-continuous f, a bound N, and a finite set E of subsets of $\{m \mid m < N\}$ such that $f(x) \subseteq \{m \mid m < N\}$ for every x and $U = \{x \mid f(x) \in E\}$.

Theorem (`BooleanMemE.finite_Iio`). Every initial segment $\{m \mid m < N\}$ of ω is finite.

Proof. It coincides with the finite set $\{0, \ldots, N-1\}$. ■

Theorem (`BooleanMemE.finite_subsets_lt`). The set of subsets of $\{m \mid m < N\}$ is finite.

Proof. The powerset of a finite set is finite. ■

Theorem (`BooleanMemE.ofOpen`). Every Scott-open set has Boolean membership.

Proof. Take $f = \mathrm{charOpen}\, U$, bound 1, and $E = \{0\}$. Continuity of the characteristic map is already known; Scott-open membership supplies a finite witness forcing $\mathrm{charOpen}\, U\, x = 0$, and conversely a zero value yields a witness in U. ■

Theorem (`BooleanMemE.union`). Boolean membership is closed under binary unions.

Proof. Odd–even pack the two witnessing maps with bound $2\max(N, M) + 1$, and admit those packed values whose odd part lies in the first admissible set or whose even part lies in the second; the parity

projections recover membership in either factor. ■

Theorem (`BooleanMemE.inter`). Boolean membership is closed under binary intersections.

Proof. The same packing and bound, now requiring both parity projections to land in the respective admissible sets. ■

Theorem (`BooleanMemE.compl`). Boolean membership is closed under complements.

Proof. Keep the same continuous map and bound, and replace the admissible set by its relative complement inside the finite powerset of $\{m \mid m < N\}$. ■

`BooleanMemE.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
import DomainTheory.DataTypesAsLattices.Classification.CharacteristicMaps
import DomainTheory.DataTypesAsLattices.Classification.OddEvenPack
import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen_more
import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_isUpper
import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def BooleanMemE (U : Set Pomega) : Prop :=
  ∃ (f : Pomega → Pomega) (N : ℕ) (E : Set Pomega),
    IsScottContinuous f ∧
      (∀ x, f x ⊆ {m | m < N}) ∧
        E.Finite ∧ (∀ e ∈ E, e ⊆ {m | m < N}) ∧
          U = {x | f x ∈ E}

namespace BooleanMemE

theorem finite_Iio (N : ℕ) : ({m : ℕ | m < N}).Finite :=
  Set.Finite.ofFinset (Finset.range N) fun x => by simp

theorem finite_subsets_lt (N : ℕ) :
    ({y : Pomega | y ⊆ {m | m < N}}).Finite :=
  (finite_Iio N).finite_subsets

theorem ofOpen {U : Set Pomega} (hU : IsScottOpen U) :
    BooleanMemE U := by
  refine ⟨charOpen U, 1, {ofNat 0}, charOpen.isScottContinuous U, ?_, ?_, ?_, ?_⟩
  · intro x k hk
    rcases hk with ⟨rfl, _⟩
    exact Nat.zero_lt_one
  · exact Set.finite_singleton _
  · intro e he
    simp [ofNat] at he
    subst he
    intro k hk
    simp at hk
    subst hk
    exact Nat.zero_lt_one
```

```
  · ext x
    constructor
    · intro hx
      change charOpen U x = ofNat 0
      ext k
      constructor
      · intro ⟨hk0, n, hn, hne⟩
        simp [ofNat, hk0]
      · intro hk
        simp [ofNat] at hk
        subst hk
        obtain ⟨n, hn, hne⟩ := (IsScottOpen.mem_iff hU).mp hx
        exact ⟨rfl, n, hn, hne⟩
    · intro hx
      have : 0 ∈ charOpen U x := by
        have : charOpen U x = ofNat 0 := hx
        rw [this]; simp [ofNat]
      obtain ⟨_, n, hn, hne⟩ := this
      exact isScottOpen_isUpper hU hne hn

theorem union {U V : Set Pomega}
    (hU : BooleanMemE U) (hV : BooleanMemE V) : BooleanMemE (U ∪ V) := by
  obtain ⟨f, N, E, hf, hfb, hEf, hEsub, rfl⟩ := hU
  obtain ⟨g, M, F, hg, hgb, hFf, hFsub, rfl⟩ := hV
  let N' := 2 * max N M + 1
  let E' : Set Pomega :=
    {y | y ⊆ {m | m < N'} ∧ (evenPart y ∈ F ∨ oddPart y ∈ E)}
  refine ⟨oddEvenPack f g, N', E',
    oddEvenPack.isScottContinuous hf hg, ?_, ?_, ?_, ?_⟩
  · intro x
    exact oddEvenPack.subset (hfb x) (hgb x)
  · have hpow := finite_subsets_lt N'
    exact hpow.subset fun y hy => hy.1
  · intro y hy; exact hy.1
  · ext x
    constructor
    · intro hx
      refine ⟨oddEvenPack.subset (hfb x) (hgb x), ?_⟩
      rw [oddEvenPack.evenPart, oddEvenPack.oddPart]
      rcases hx with h | h
      · exact Or.inr h
      · exact Or.inl h
    · intro hx
      have hx' := hx.2
      rw [oddEvenPack.evenPart, oddEvenPack.oddPart] at hx'
      rcases hx' with h | h
      · exact Or.inr h
      · exact Or.inl h

theorem inter {U V : Set Pomega}
    (hU : BooleanMemE U) (hV : BooleanMemE V) : BooleanMemE (U ∩ V) := by
  obtain ⟨f, N, E, hf, hfb, hEf, hEsub, rfl⟩ := hU
  obtain ⟨g, M, F, hg, hgb, hFf, hFsub, rfl⟩ := hV
  let N' := 2 * max N M + 1
  let E' : Set Pomega :=
    {y | y ⊆ {m | m < N'} ∧ evenPart y ∈ F ∧ oddPart y ∈ E}
```

```
  refine ⟨oddEvenPack f g, N', E',
    oddEvenPack.isScottContinuous hf hg, ?_, ?_, ?_, ?_⟩
  · intro x
    exact oddEvenPack.subset (hfb x) (hgb x)
  · have hpow := finite_subsets_lt N'
    exact hpow.subset fun y hy => hy.1
  · intro y hy; exact hy.1
  · ext x
    constructor
    · intro hx
      refine ⟨oddEvenPack.subset (hfb x) (hgb x), ?_⟩
      rw [oddEvenPack.evenPart, oddEvenPack.oddPart]
      exact ⟨hx.2, hx.1⟩
    · intro hx
      have : evenPart (oddEvenPack f g x) ∈ F ∧
          oddPart (oddEvenPack f g x) ∈ E := ⟨hx.2.1, hx.2.2⟩
      rw [oddEvenPack.evenPart, oddEvenPack.oddPart] at this
      exact ⟨this.2, this.1⟩

theorem compl {U : Set Pomega} (hU : BooleanMemE U) :
    BooleanMemE U? := by
  obtain ⟨f, N, E, hf, hfb, hEf, hEsub, rfl⟩ := hU
  let E' : Set Pomega := {y | y ⊆ {m | m < N} ∧ y ∈ E}
  refine ⟨f, N, E', hf, hfb, ?_, fun y hy => hy.1, ?_⟩
  · exact (finite_subsets_lt N).subset fun y hy => hy.1
  · ext x
    constructor
    · intro hx
      exact ⟨hfb x, hx⟩
    · intro hx
      exact hx.2

end BooleanMemE

end DomainTheory.DataTypesAsLattices
```

3.7.9 Theorem 6.1 — the G theorem ✓

THEOREM 6.1 (The $\mathfrak{G}$ theorem). *The open subsets of* $\mathbf{P}\omega$ *are exactly the sets of the form:*

$$\{x \mid f(x) \supseteq 0\},$$

where $f : \mathbf{P}\omega \to \mathbf{P}\omega$ *is continuous.*

We could have written $0 \in f(x)$ or the equation $f(x) \cap 0 = 0$ instead of $f(x) \supseteq 0$. Note that in case $f : \mathbf{P}\omega \to \{\bot, \top\}$, then $f(x) \supseteq 0$ is equivalent to $f(x) = \top$. Also any other integer could have been used in place of 0.

We can say that $\{x \mid 0 \in x\}$ is the *typical* open set, and that every other open set can be obtained as an inverse image of the typical set by a continuous function. We shall extend this pattern to other classes, especially looking for equations. In the case of openness an inequality could also be used, giving as the typical set $\{x \mid x \neq \bot\}$. But since closed sets are just the complements of open sets, this remark gives us:

Theorem (`theorem_6_1`). A subset $U \subseteq \mathbf{P}\omega$ is Scott-open if and only if $U = \{x \mid 0 \in f(x)\}$ for some Scott-continuous $f : \mathbf{P}\omega \to \mathbf{P}\omega$.

Proof. If f is Scott-continuous, then $0 \in f(x)$ holds exactly when some finite $e_n \subseteq x$ already satisfies $0 \in f(e_n)$, so the fibre is a union of basic Scott neighbourhoods. Conversely, the characteristic map $\mathrm{charOpen}\, U$ is Scott-continuous and equals the numeral 0 precisely on U, so membership of 0 recovers U. ■

theorem_6_1.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
import DomainTheory.DataTypesAsLattices.Classification.CharacteristicMaps
import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_isUpper
import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem theorem_6_1 :
    (∀ f : Pomega → Pomega, IsScottContinuous f →
        IsScottOpen {x | 0 ∈ f x}) ∧
      (∀ U, IsScottOpen U →
        ∃ f, IsScottContinuous f ∧ U = {x | 0 ∈ f x}) := by
  constructor
  · intro f hf
    refine ⟨{n | 0 ∈ f (e n)}, ?_⟩
    ext x
    constructor
    · intro hx
      have : 0 ∈ scottUnion f x := by rwa [← hf.out x]
      obtain ⟨n, hn, h0⟩ := mem_scottUnion.mp this
      exact Set.mem_iUnion₂.mpr ⟨n, h0, hn⟩
    · intro hx
      obtain ⟨n, h0, hn⟩ := Set.mem_iUnion₂.mp hx
      exact IsScottContinuous.monotone hf hn h0
  · intro U hU
    refine ⟨charOpen U, charOpen.isScottContinuous U, ?_⟩
    ext x
    constructor
    · intro hx
      obtain ⟨B, rfl⟩ := hU
      obtain ⟨n, hn, hxn⟩ := Set.mem_iUnion₂.mp hx
      refine ⟨rfl, n, hxn, ?_⟩
      exact Set.mem_iUnion₂.mpr ⟨n, hn, (subset_rfl : e n ⊆ e n)⟩
    · intro ⟨_, n, hn, hne⟩
      exact isScottOpen_isUpper hU hne hn

end DomainTheory.DataTypesAsLattices
```

3.7.10 Theorem 6.2 — the F theorem ✓

THEOREM 6.2 (The $\mathfrak{F}$ theorem). *The closed subsets of* $\mathbf{P}\omega$ *are exactly the sets of the form:*

$$\{x \mid f(x) = \bot\},$$

where $f : \mathbf{P}\omega \to \mathbf{P}\omega$ *is continuous.*

Aside from $\{x \mid x = \bot\}$, we could have used $\{x \mid x \subseteq a\}$ as the typical closed set where $a \in \mathbf{P}\omega$ is any element whatsoever aside from $\top$. This $\top$ has, by the way, a

special character. We note:

$$\{\top\} = \bigcap\{\{x \mid n \in x\} \mid n \in \omega\}.$$

Thus $\{\top\}$ is a countable intersection of open sets, otherwise called a $\mathfrak{G}_\delta$-set. There are of course many other $\mathfrak{G}_\delta$-sets, but $\{\top\}$ is the typical one:

Theorem (`theorem_6_2`). A subset $U \subseteq \mathbf{P}\omega$ is Scott-closed if and only if $U = \{x \mid f(x) = \bot\}$ for some Scott-continuous $f : \mathbf{P}\omega \to \mathbf{P}\omega$.

Proof. If U is closed, then $\mathrm{charTop}\, U^{\mathrm{c}}$ is Scott-continuous and equals $\bot$ precisely on U. Conversely, if f is Scott-continuous then the complement of $\{x \mid f(x) = \bot\}$ is the union over k of the open fibres $\{x \mid k \in f(x)\}$, hence open, so the fibre of $\bot$ is closed. ■

`theorem_6_2.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Classification.Classes
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
import DomainTheory.DataTypesAsLattices.Classification.CharacteristicMaps
import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem theorem_6_2 :
    ∀ U, U ∈ ScottF ↔ ∃ f, IsScottContinuous f ∧ U = {x | f x = botElem} := by
  intro U
  constructor
  · intro hU
    refine ⟨charTop U?, charTop.isScottContinuous U?, ?_⟩
    ext x
    exact (charTop.eq_bot_of_closed hU x).symm
  · rintro ⟨f, hf, rfl⟩
    change IsScottOpen {x | f x = botElem}?
    have : {x | f x = botElem}? = ⋃ k, {x | k ∈ f x} := by
      ext x
      constructor
      · intro hx
        have hx' : f x = botElem := hx
        have : ∃ k, k ∈ f x := by
          by_contra hnone
          apply hx'
          ext n
          constructor
          · intro hn
```

```
            exact (hnone ⟨n, hn⟩).elim
          · intro hn
            exact ((mem_botElem n).mp hn).elim
        exact Set.mem_iUnion.mpr this
      · intro hx hbot
        obtain ⟨k, hk⟩ := Set.mem_iUnion.mp hx
        have hk' : k ∈ f x := hk
        have hbot' : f x = botElem := hbot
        rw [hbot'] at hk'
        exact ((mem_botElem k).mp hk').elim
    rw [this]
    exact IsScottOpen.iUnion _ fun k => IsScottContinuous.mem_preimage hf k

end DomainTheory.DataTypesAsLattices
```

3.7.11 Theorem 6.3 — the Gδ theorem ✓

THEOREM 6.3 (The $\mathfrak{G}_\delta$ theorem). *The countable intersections of open subsets of $\mathbf{P}\omega$ are exactly the sets of the form:*

$$\{x \mid f(x) = \top\},$$

where $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is continuous.

It may not be obvious that every $\mathfrak{G}_\delta$-set has this form. Certainly, as we have remarked, every $\mathfrak{G}$-set has this form. Thus if W is a $\mathfrak{G}_\delta$, we have:

$$W = \bigcap\{U_n \mid n \in \omega\},$$

and further,

$$U_n = \{x \mid f_n(x) = \top\},$$

where the f_n are suitably chosen continuous functions. Define the function g by the equation:

$$g(x) = \{(n, m) \mid m \in f_n(x)\}.$$

Clearly g is continuous, and we have:

$$W = \{x \mid g(x) = \top\},$$

as desired.

We let $\mathfrak{F}\dot{\cap}\mathfrak{G}$ denote the class of all sets of the form $C \cap U$, where $C \in \mathfrak{F}$ and $U \in \mathfrak{G}$. Similarly for $\mathfrak{F}\dot{\cap}\mathfrak{G}_\delta$. Now $\{x \mid x \subseteq 0\}$ is closed and $\{x \mid x \supseteq 0\}$ is open. Thus $\{0\} \in \mathfrak{F}\dot{\cap}\mathfrak{G}$. This set is typical.

Theorem (`theorem_6_3`). A subset $U \subseteq \mathbf{P}\omega$ is a $\mathfrak{G}_\delta$ if and only if $U = \{x \mid f(x) = \top\}$ for some Scott-continuous $f : \mathbf{P}\omega \to \mathbf{P}\omega$.

Proof. If $U = \bigcap_n U_n$ with each U_n Scott-open, pair-index the maps $\mathrm{charTop}\, U_n$; the packed value is $\top$ precisely when every factor is $\top$, hence precisely on the intersection. Conversely, if f is Scott-continuous then $\{x \mid f(x) = \top\}$ is the intersection over n of the open fibres $\{x \mid n \in f(x)\}$. ■

`theorem_6_3.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Classification.Classes
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
import DomainTheory.DataTypesAsLattices.Classification.CharacteristicMaps
import DomainTheory.DataTypesAsLattices.Classification.pairIndexed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem theorem_6_3 :
    ∀ U, IsScottGdelta U ↔ ∃ f, IsScottContinuous f ∧ U = {x | f x = topElem} := by
  intro U
  constructor
  · rintro ⟨Us, hUs, rfl⟩
    let fs := fun n => charTop (Us n)
    refine ⟨pairIndexed fs, pairIndexed.isScottContinuous fs fun n =>
      charTop.isScottContinuous (Us n), ?_⟩
    ext x
    constructor
    · intro hx
      change pairIndexed fs x = topElem
      exact (pairIndexed.eq_top fs x).mpr fun n =>
        (charTop.eq_top (hUs n) x).mpr (Set.mem_iInter.mp hx n)
    · intro hf
      have hf' : pairIndexed fs x = topElem := hf
      exact Set.mem_iInter.mpr fun n =>
        (charTop.eq_top (hUs n) x).mp ((pairIndexed.eq_top fs x).mp hf' n)
  · rintro ⟨f, hf, rfl⟩
    refine ⟨fun n => {x | n ∈ f x}, fun n => IsScottContinuous.mem_preimage hf n, ?_⟩
    ext x
    constructor
    · intro hx
      refine Set.mem_iInter.mpr fun n => ?_
      have : f x = topElem := hx
      simp [this]
    · intro hx
      ext k
      constructor
      · intro _; trivial
      · intro _
        exact Set.mem_iInter.mp hx k

end DomainTheory.DataTypesAsLattices
```

3.7.12 Theorem 6.4 — the F∩G theorem ✓

THEOREM 6.4 (The $\mathfrak{F}\dot{\cap}\mathfrak{G}$ theorem). *The sets that are intersections of closed sets with open sets are exactly the sets of the form:*

$$\{x \mid f(x) = 0\},$$

where $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is continuous.

Again it may not be obvious that every $\mathfrak{F}\dot{\cap}\mathfrak{G}$ set has this form. We can write:

$$C = \{x \mid f(x) = \bot\},$$

and

$$U = \{x \mid g(x) = 0\},$$

where $C \in \mathfrak{F}$ and $U \in \mathfrak{G}$ and the continuous f and g are suitably chosen. Define

$$h(x) = \{2n+1 \mid n \in f(x)\} \cup \{2n \mid n \in g(x)\},$$

and remark that h is continuous. We have:

$$C \cap U = \{x \mid h(x) = 0\},$$

as desired.

It is easy to see that $\{e\} \in \mathfrak{F}\dot{\cap}\mathfrak{G}$ if e is finite, but in general $\{a\} \notin \mathfrak{F}\dot{\cap}\mathfrak{G}_\delta$. In

In case a is infinite but not equal to $\top$ (say, $a = \{n \mid n > 0\} = \top + 1$), then $\{a\}$ is typical in its class.

Theorem (`theorem_6_4`). A subset $U \subseteq \mathbf{P}\omega$ belongs to $\mathfrak{F}\dot{\cap}\mathfrak{G}$ if and only if $U = \{x \mid f(x) = 0\}$ for some Scott-continuous $f : \mathbf{P}\omega \to \mathbf{P}\omega$.

Proof. If $U = C \cap O$ with C closed and O open, odd–even pack $\mathrm{charTop}\, C^{\mathrm{c}}$ with $\mathrm{charOpen}\, O$; the packed value equals the numeral 0 precisely when the closed factor is $\bot$ and the open factor is 0. Conversely, if f is Scott-continuous then $\{x \mid f(x) = 0\}$ is the intersection of the closed set $\{x \mid f(x) \subseteq 0\}$ with the open fibre $\{x \mid 0 \in f(x)\}$. ■

`theorem_6_4.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Classification.Classes
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.Language
import DomainTheory.DataTypesAsLattices.Classification.CharacteristicMaps
import DomainTheory.DataTypesAsLattices.Classification.OddEvenPack
import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen_more
import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_isUpper

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem theorem_6_4 :
    ∀ U, IsFcapG U ↔ ∃ f, IsScottContinuous f ∧ U = {x | f x = ofNat 0} := by
  intro U
  constructor
  · rintro ⟨C, O, hC, hO, rfl⟩
    refine ⟨oddEvenPack (charTop C?) (charOpen O),
      oddEvenPack.isScottContinuous (charTop.isScottContinuous C?)
        (charOpen.isScottContinuous O), ?_⟩
```

```
  ext x
  constructor
  · intro hx
    change oddEvenPack (charTop C?) (charOpen O) x = ofNat 0
    refine (oddEvenPack.eq_zero x).mpr ?_
    have hx' : x ∈ C ∧ x ∈ O := hx
    constructor
    · exact (charTop.eq_bot_of_closed hC x).mpr hx'.1
    · ext k
      constructor
      · intro ⟨hk0, n, hn, hne⟩
        simp [ofNat, hk0]
      · intro hk
        simp [ofNat] at hk
        subst hk
        obtain ⟨n, hn, hne⟩ := (IsScottOpen.mem_iff hO).mp hx'.2
        exact ⟨rfl, n, hn, hne⟩
  · intro hf
    have hf' := (oddEvenPack.eq_zero (f := charTop C?) (g := charOpen O) x).mp hf
    constructor
    · exact (charTop.eq_bot_of_closed hC x).mp hf'.1
    · have : 0 ∈ charOpen O x := by rw [hf'.2]; simp [ofNat]
      obtain ⟨_, n, hn, hne⟩ := this
      exact isScottOpen_isUpper hO hne hn
· rintro ⟨f, hf, rfl⟩
  refine ⟨{x | f x ⊆ ofNat 0}, {x | 0 ∈ f x}, ?_,
    IsScottContinuous.mem_preimage hf 0, ?_⟩
  · have : {x | f x ⊆ ofNat 0}? = ⋃ k : ℕ, {x | k = 0 ∧ k ∈ f x} := by
      ext x
      constructor
      · intro hx
        have : ¬ f x ⊆ ofNat 0 := hx
        obtain ⟨k, hk, hk0⟩ := Set.not_subset.mp this
        refine Set.mem_iUnion.mpr ⟨k, ?_⟩
        simp [ofNat] at hk0
        exact ⟨hk0, hk⟩
      · intro hx
        obtain ⟨k, hk0, hk⟩ := Set.mem_iUnion.mp hx
        intro hsub
        exact hk0 (hsub hk)
    rw [this]
    exact IsScottOpen.iUnion _ fun k => by
      by_cases hk0 : k = 0
      · subst hk0
        have : {x | (0 : ℕ) = 0 ∧ 0 ∈ f x} = (∅ : Set Pomega) := by
          ext x; simp
        rw [this]; exact IsScottOpen.empty
      · have : {x | k = 0 ∧ k ∈ f x} = {x | k ∈ f x} := by
          ext x; simp [hk0]
        rw [this]; exact IsScottContinuous.mem_preimage hf k
  · ext x
    constructor
    · intro hx
      change f x ⊆ ofNat 0 ∧ 0 ∈ f x
      have hx' : f x = ofNat 0 := hx
      constructor
```

```
        · simp [hx']
        · simp [hx', ofNat]
      · intro hx
        change f x = ofNat 0
        ext n
        simp [ofNat]
        constructor
        · intro hn; exact hx.1 hn
        · intro hn; simpa [hn] using hx.2

end DomainTheory.DataTypesAsLattices
```

3.7.13 Theorem 6.5 — the F∩Gδ theorem ✓

THEOREM 6.5 (The $\mathfrak{F}\dot{\cap}\mathfrak{G}_\delta$ theorem). *The sets that are intersections of closed sets with countable intersections of open sets are exactly the sets of the form:*

$$\{x \mid f(x) = a\},$$

where $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is continuous and a is a fixed infinite set not equal to $\top$.

Note that $(\mathfrak{F}\dot{\cap}\mathfrak{G})_\delta$ is the same class as $\mathfrak{F}\dot{\cap}\mathfrak{G}_\delta$, so we see by Theorem 6.4 that a good choice of a is $\lambda n \in \omega.0$.

There is no single subset of $\mathbf{P}\omega$ typical for $\mathfrak{B}$, which can be viewed as the finite unions of sets from the class $\mathfrak{F}\dot{\cap}\mathfrak{G}$.

Theorem (`theorem_6_5`). A subset $U \subseteq \mathbf{P}\omega$ belongs to $\mathfrak{F}\dot{\cap}\mathfrak{G}_\delta$ if and only if $U = \{x \mid f(x) = \mathrm{succSet}\ \top\}$ for some Scott-continuous $f : \mathbf{P}\omega \to \mathbf{P}\omega$.

Proof. If $U = C \cap G$ with C closed and G a $\mathfrak{G}_\delta$, take a continuous fibre of $\top$ for G by Theorem 6.3 and pack it with $\mathrm{charTop}\, C^{\mathrm{c}}$; the packed value equals $\mathrm{succSet}\ \top$ precisely when the closed factor is $\bot$ and the $\mathfrak{G}_\delta$ factor is $\top$. Conversely, if f is Scott-continuous then $\{x \mid f(x) = \mathrm{succSet}\ \top\}$ is the intersection of the closed set $\{x \mid f(x) \subseteq \mathrm{succSet}\ \top\}$ with the $\mathfrak{G}_\delta$ set $\{x \mid \mathrm{succSet}\ \top \subseteq f(x)\}$. ■

`theorem_6_5.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Classification.Classes
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.Language
import DomainTheory.DataTypesAsLattices.Classification.CharacteristicMaps
import DomainTheory.DataTypesAsLattices.Classification.fcapGdeltaPack
import DomainTheory.DataTypesAsLattices.Classification.theorem_6_3

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem theorem_6_5 :
    ∀ U, IsFcapGdelta U ↔
      ∃ f, IsScottContinuous f ∧ U = {x | f x = succSet topElem} := by
  intro U
```

```
  constructor
  · rintro ⟨C, G, hC, hG, rfl⟩
    obtain ⟨fG, hfG, rflG⟩ := (theorem_6_3 G).mp hG
    refine ⟨fcapGdeltaPack (charTop C?) fG,
      fcapGdeltaPack.isScottContinuous (charTop.isScottContinuous C?) hfG, ?_⟩
    ext x
    have hfval : charTop C? x = botElem ∨ charTop C? x = topElem := by
      by_cases hx : x ∈ C
      · exact Or.inl ((charTop.eq_bot_of_closed hC x).mpr hx)
      · exact Or.inr ((charTop.eq_top hC x).mpr hx)
    constructor
    · intro ⟨hxC, hxG⟩
      have hf : charTop C? x = botElem := (charTop.eq_bot_of_closed hC x).mpr hxC
      have hg : fG x = topElem := by rw [rflG] at hxG; exact hxG
      exact (fcapGdeltaPack.eq hfval).mpr ⟨hf, hg⟩
    · intro hpack
      obtain ⟨hf, hg⟩ := (fcapGdeltaPack.eq hfval).mp hpack
      exact ⟨(charTop.eq_bot_of_closed hC x).mp hf, by rw [rflG]; exact hg⟩
  · rintro ⟨f, hf, rfl⟩
    refine ⟨{x | f x ⊆ succSet topElem}, {x | succSet topElem ⊆ f x}, ?_, ?_, ?_⟩
    · have : {x | f x ⊆ succSet topElem}? = {x | 0 ∈ f x} := by
        ext x
        constructor
        · intro hx
          obtain ⟨k, hk, hknotin⟩ := Set.not_subset.mp hx
          have hk0 : k = 0 := by
            by_contra hne
            exact hknotin ⟨k - 1, Set.mem_univ _, (Nat.succ_pred_eq_of_ne_zero hne).
    symm⟩
          simpa [hk0] using hk
        · intro h0 hsub
          exact zero_not_mem_succSet_top (hsub h0)
      rw [this]
      exact IsScottContinuous.mem_preimage hf 0
    · refine ⟨fun n => {x | n + 1 ∈ f x},
        fun n => IsScottContinuous.mem_preimage hf (n + 1), ?_⟩
      ext x
      constructor
      · intro hx
        exact Set.mem_iInter.mpr fun n => hx ⟨n, Set.mem_univ n, rfl⟩
      · intro hx k hk
        obtain ⟨n, _, rfl⟩ := hk
        exact Set.mem_iInter.mp hx n
    · ext x
      constructor
      · intro hx
        change f x ⊆ succSet topElem ∧ succSet topElem ⊆ f x
        have hx' : f x = succSet topElem := hx
        constructor <;> simp [hx']
      · intro hx
        change f x = succSet topElem
        exact subset_antisymm hx.1 hx.2

end DomainTheory.DataTypesAsLattices
```

3.7.14 Theorem 6.6 — the B theorem ✓

THEOREM 6.6 (The $\mathfrak{B}$ theorem). *The sets that are Boolean combinations of open sets are exactly the sets of the form:*

$$\{x \mid f(x) \in \mathcal{E}\},$$

where $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is continuous and $\mathcal{E}$ is a finite set of finite elements of $\mathbf{P}\omega$.

To see that every $\mathfrak{B}$ set has this form, suppose that

$$V = W_0 \cup W_1 \cup \cdots \cup W_{n-1},$$

where each $W_i \in \mathfrak{F}\dot{\cap}\mathfrak{G}$. We can write:

$$W_i = \{x \mid f_i(x) = 0\},$$

where $f_i : \mathbf{P}\omega \to \{\bot, 0, \top\}$ is continuous. Then define:

$$g(x) = \{2i + j \mid j \in f_i(x) \cap \{0, 1\}, i < n\},$$

and note that g is continuous. Let

$$\mathcal{E} = \{y \subseteq \{m \mid m < 2n\} \mid \exists i < n.2i \in y, 2i + 1 \notin y\};$$

we have:

$$V = \{x \mid g(x) \in \mathcal{E}\}$$

as desired.

Theorem (`theorem_6_6`). A subset $U \subseteq \mathbf{P}\omega$ is a Boolean combination of Scott-open sets if and only if $U = \{x \mid f(x) \in E\}$ for some Scott-continuous f and some finite set E of finite elements of $\mathbf{P}\omega$.

Proof. Boolean membership yields a continuous map of uniformly bounded values with a finite admissible set of subsets of that bound, each of which is finite. Conversely, each fibre $\{x \mid f(x) = e\}$ for finite e is an $\mathfrak{F}\dot{\cap}\mathfrak{G}$ set (closed upper and open lower sandwich), hence Boolean, and a finite union of those fibres recovers $\{x \mid f(x) \in E\}$. ■

`theorem_6_6.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Classification.Classes
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
import DomainTheory.DataTypesAsLattices.Classification.BooleanMemE
import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen_more
import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_iff_isOpen
import Mathlib.Tactic.Bound.Init

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace
```

```
theorem theorem_6_6 :
    ∀ U, U ∈ ScottB ↔
      ∃ f, IsScottContinuous f ∧ ∃ E : Set Pomega,
        E.Finite ∧ (∀ e ∈ E, e.Finite) ∧ U = {x | f x ∈ E} := by
  intro U
  constructor
  · intro hU
    have : BooleanMemE U := by
      induction hU with
      | ofOpen hU => exact BooleanMemE.ofOpen hU
      | union _ _ ihU ihV => exact BooleanMemE.union ihU ihV
      | inter _ _ ihU ihV => exact BooleanMemE.inter ihU ihV
      | compl _ ih => exact BooleanMemE.compl ih
    obtain ⟨f, N, E, hf, hfb, hEf, hEsub, rfl⟩ := this
    refine ⟨f, hf, E, hEf, fun e he => (BooleanMemE.finite_Iio N).subset (hEsub e he),
    rfl⟩
  · rintro ⟨f, hf, E, hE, hEfin, rfl⟩
    have : {x | f x ∈ E} = ⋃₀ ((fun e => {x | f x = e}) '' E) := by
      ext x
      constructor
      · intro hx
        exact ⟨{y | f y = f x}, ⟨f x, hx, rfl⟩, rfl⟩
      · intro hx
        obtain ⟨_, ⟨e, he, rfl⟩, hx'⟩ := hx
        have hxeq : f x = e := hx'
        change f x ∈ E
        rwa [hxeq]
    rw [this]
    refine IsBooleanOpen.sUnion_finite (hE.image _) fun W hW => ?_
    obtain ⟨e, heE, rfl⟩ := hW
    have hefin : e.Finite := hEfin e heE
    have hopen : IsScottOpen {x | e ⊆ f x} := by
      have : {x | e ⊆ f x} = ⋂ k ∈ hefin.toFinset, {x | k ∈ f x} := by
        ext x
        constructor
        · intro hx
          exact Set.mem_iInter₂.mpr fun k hk => hx (hefin.mem_toFinset.mp hk)
        · intro hx k hk
          exact Set.mem_iInter₂.mp hx k (hefin.mem_toFinset.mpr hk)
      rw [this, isScottOpen_iff_isOpen]
      exact isOpen_biInter_finset fun k _ =>
        (isScottOpen_iff_isOpen _).mp (IsScottContinuous.mem_preimage hf k)
    have hclosed : IsScottOpen {x | f x ⊆ e}ᶜ := by
      have : {x | f x ⊆ e}ᶜ = ⋃ k, {x | k ∈ e ∧ k ∈ f x} := by
        ext x
        constructor
        · intro hx
          obtain ⟨k, hk, hke⟩ := Set.not_subset.mp hx
          exact Set.mem_iUnion.mpr ⟨k, ⟨hke, hk⟩⟩
        · intro hx
          obtain ⟨k, hke, hk⟩ := Set.mem_iUnion.mp hx
          intro hsub
          exact hke (hsub hk)
      rw [this]
      exact IsScottOpen.iUnion _ fun k => by
        by_cases hke : k ∈ e
```

```
      · have : {x | k ∈ e ∧ k ∈ f x} = (∅ : Set Pomega) := by
          ext x; simp [hke]
        rw [this]; exact IsScottOpen.empty
      · have : {x | k ∈ e ∧ k ∈ f x} = {x | k ∈ f x} := by
          ext x; simp [hke]
        rw [this]; exact IsScottContinuous.mem_preimage hf k
  exact IsBooleanOpen.of_fcapG ⟨{x | f x ⊆ e}, {x | e ⊆ f x}, hclosed, hopen, by
    ext x
    constructor
    · intro hx
      have : f x = e := by
        have : f x ∈ ({e} : Set Pomega) := hx
        simpa using this
      exact ⟨this.subset, this.symm.subset⟩
    · intro hx
      have : f x = e := subset_antisymm hx.1 hx.2
      simp [this]⟩

end DomainTheory.DataTypesAsLattices
```

3.7.15 Theorem 6.7 — the Bδ theorem ✓

THEOREM 6.7 (The $\mathfrak{B}_\delta$ theorem). *The sets that are countable intersections of Boolean combinations of open sets are exactly the sets of the form:*

$$\{x \mid f(x) = g(x)\},$$

where $f, g : \mathbf{P}\omega \to \mathbf{P}\omega$ are continuous.

This is clearly the most interesting of these characterization theorems, because equations like $f(x) = g(x)$ turn up all the time and the collection is a very rich totality of subsets of $\mathbf{P}\omega$. It includes all the retracts, since they are of the form $\{x \mid x = a(x)\}$. And much more. That every such set in Theorem 6.7 is $\mathfrak{B}_\delta$ follows from these logical transformations:

$$\{x \mid f(x) = g(x)\} = \{x \mid \forall n \in \omega[n \in f(x) \leftrightarrow n \in g(x)]\}$$

$$= \bigcap_{n=0}^{\infty} (\{x \mid n \in f(x), n \in g(x)\} \cup \{x \mid n \notin f(x), n \notin g(x)\})$$

That puts the set in the class $(\mathfrak{F} \cup \mathfrak{G})_\delta = \mathfrak{B}_\delta$.

On the other hand, we can see that $(\mathfrak{F} \cup \mathfrak{G})_\sigma$ is exactly $\mathfrak{B}_\delta$. Because, in view of Theorem 6.6, $\mathfrak{B}_\sigma$, the class of *countable* unions of $\mathfrak{B}$-sets, is exactly $(\mathfrak{F} \dot{\cap} \mathfrak{G})_\sigma$. The remark we want to make then follows by taking complements.

Now let S be an arbitrary $\mathfrak{B}_\delta$-set. We can write:

$$S = \bigcap_{n=0}^{\infty} (\{x \mid f_n(x) = 0\} \cup \{x \mid g_n(x) = 0\}),$$

where $f_n, g_n : \mathbf{P}\omega \to \{\bot, 0, \top\}$ are continuous. Now let u, v be continuous functions which on $\{\bot, 0, \top\}$ realize these two tables:

u	$\bot$	0	$\top$
$\bot$	$\bot$	0	0
0	0	0	$0'$
$\top$	0	$0'$	$0'$

v	$\bot$	0	$\top$
$\bot$	0	0	$0'$
0	0	0	$0'$
$\top$	$0'$	$0'$	$\top$

where $0' = 0 \cup \bot$. This is an exercise in many-valued logic, and we find for $x, y \in \{\bot, 0, \top\}$:

$$u(x)(y) = v(x)(y) \quad \text{iff} \quad x = 0 \text{ or } y = 0.$$

Thus define continuous functions f' and g' such

$$f' = \lambda x \lambda n \in \omega . u(f_n(x))(g_n(x)),$$
$$g' = \lambda x \lambda n \in \omega . v(f_n(x))(g_n(x)),$$

and we find:

$$S = \{x \mid f'(x) = g'(x)\}$$

as desired.

This is as far as we can go with *equations*. More complicated sets can be defined using *quantifiers*, for example the Σ^1_1 or *analytic* sets can be put in the form:

$$\{x \mid \exists y . f(x)(y) = g(x)(y)\},$$

and their complements, the Π^1_1 sets, in the form

$$\{x \mid \forall y \exists z . h(x)(y)(z) = 0\},$$

with continuous f, g, h. For the three classes we then have as "typical" sets those shown in Table 3.

It should be remarked that $\mathfrak{B}_\delta$ contains all the closed sets in the *Cantor space* topology on $\mathbf{P}\omega$ (that is, the topology obtained when it is regarded as the infinite product of *discrete* two-point spaces). Therefore the Σ^1_1 sets for the two topologies on $\mathbf{P}\omega$ are the *same*. Hence, since we know for Cantor space that $\Delta^1_1 = \Sigma^1_1 \cap \Pi^1_1$ is the class of *Borel sets*, we can conclude that the two topologies on $\mathbf{P}\omega$ have the *same* Borel sets. (That is, in both cases Δ^1_1 is the Boolean σ-algebra generated from the open sets.)

TABLE 3 *Classes and typical sets*

Classes	Typical sets
$\mathfrak{G}$	$\{x \mid 0 \in x\}$
$\mathfrak{F}$	$\{\bot\}$

Classes	Typical sets
$\mathfrak{G}_\delta$	$\{\top\}$
$\mathfrak{F}\dot{\cap}\mathfrak{G}$	$\{0\}$
$\mathfrak{F}\dot{\cap}\mathfrak{G}_\delta$	$\{\top + 1\}$
$\mathfrak{B}_\delta$	$\{u \mid u_0 = u_1\}$
Σ^1_1	$\{u \mid \exists y.\, u_0(y) = u_1(y)\}$
Π^1_1	$\{u \mid \forall y \exists z.\, u(y)(z) = 0\}$

Returning now to the example involving trees mentioned at the beginning of this section, we see that the set of perfect (finite) trees can be written in the form:

$$\{x \mid x = \mathbf{tree}(x),\, \mathbf{perf}(x) = 0\} = \{x \mid \langle x,\, \mathbf{perf}(x)\rangle = \langle \mathbf{tree}(x),\, 0\rangle\};$$

thus it is a $\mathfrak{B}_\delta$-set. (Note that $\mathfrak{B}_\delta$ are obviously closed under finite intersection by the ordered pair method just illustrated; that they are closed under finite union is a little messier to make explicit, but the essential idea is contained in the proof of Theorem 6.7.)

As another example, we might wish to allow infinite trees but not the strange tree $\top$. Consider the following function:

$$(6.4) \quad \mathbf{top} = \lambda u : \mathbf{tree}.\, \mathbf{which}(u) \sqsubseteq \bot,\, \mathbf{top}(\mathbf{fst}(\mathbf{out}(u))) \cup \mathbf{top}(\mathbf{snd}(\mathbf{out}(u))).$$

We can show that $\mathbf{top} : \mathbf{P}\omega \to \{\bot, \top\}$. For a tree u the equation $\mathbf{top}(u) = \bot$ means that it *does not* contain $\top$, or as we might say: it is *topless*. The topless trees form a closed subset of the subspace of trees. (An interesting retract is the function $\lambda u.\, \mathbf{tree}(u) \cup \mathbf{top}(u)$ whose range consists exactly of the topless trees plus *one* exceptional tree $\top$.) Such a closed subset of (the range of) a retract is a kind of *semilattice*. (We shall not introduce a precise definition here.) Every directed subset has a limit (least upper bound) and every pair with an upper bound has a least upper bound. But generally least upper bounds do not have to exist within the semilattice. The type of domains that interest us *become* continuous lattices with the *addition* of a top element $\top$ larger than all the other elements. The elimination of $\top$ is done with a function like $\mathbf{top}$ of our example. This is convincing evidence to the author that an independent theory of semilattices is quite unnecessary: they can all be *derived* from lattices. The problem is simply to define the top-cutting operation, then restriction to the "topless" elements is indicated by an equation (like $\mathbf{top}(u) = \bot$). In this way all the constructions are kept within the control of a smooth-running theory based on **LAMBDA**. This point seems to be important if one wants to keep track of which functions are computable.

An aspect of the problem of classification treated in this section which has not been given close enough attention is the explicitly constructive way of verifying the closure properties of the classes. Consider the class $\mathfrak{B}_\delta$, for example. Let B be the typical set as shown in Table 3. Then whatever $f \in \mathbf{P}\omega$ we choose, the set

$$\{x \mid f(x) \in B\}$$

is a $\mathfrak{B}_\delta$-set and every such set has this form. Thus the f's *index* the elements of the class. Suppose $f, g \in \mathbf{P}\omega$. What we should look for are two LAMBDA-definable combinators such that $\mathbf{union}(f)(g)$ and $\mathbf{inter}(f)(g)$ give the functions that index the union and intersection of the sets determined by f and g. That is, we want:

$$\{x \mid \mathbf{union}(f)(g)(x) \in B\} = \{x \mid f(x) \in B\} \cup \{x \mid g(x) \in B\}.$$

It should be possible to extract the precise definition from the outline of the proofs given above, but in general this matter needs more investigation. There may very well be certain classes where such operations are not constructive, even though the classes are simply defined.

Theorem (`theorem_6_7.of_equalizer`). Every equalizer $\{x \mid f(x) = g(x)\}$ of Scott-continuous maps is a $\mathfrak{B}_\delta$.

Proof. For each n, the set where n belongs to both values or to neither is a Boolean combination of open fibres; the equalizer is their countable intersection. ■

Theorem (`theorem_6_7.iInter_equalizer`). A countable family of continuous equalizers $\{x \mid \forall n.\, f_n(x) = g_n(x)\}$ is itself a single equalizer of continuous maps.

Proof. Pair-index the two sequences; the packed values agree precisely when every fibre agrees. ■

Theorem (`theorem_6_7.converse_of_equalizers`). If $U = \{x \mid \forall n.\, f_n(x) = g_n(x)\}$ for continuous f_n, g_n, then U is an equalizer of two continuous maps.

Proof. Apply the previous packing. ■

Theorem (`theorem_6_7.boolean_as_iInter_open_union_closed`). Every Boolean combination of opens is a countable intersection of sets of the form $O_n \cup C_n$ with O_n open and C_n closed.

Proof. By Theorem 6.6 the complement is a continuous fibre of a finite set E of finite values; induct on E, slicing each finite value as the intersection of an open upper set and a closed lower set, and reindexing the accumulating intersection. ■

Theorem (`theorem_6_7.of_bdelta`). Every $\mathfrak{B}_\delta$ is an equalizer of two Scott-continuous maps.

Proof. Expand each Boolean factor by the previous theorem, reindex the double sequence by pairing, pack open and closed characteristics with the three-valued tables U and V so that equalizers of the packed sequence recover the intersection, and collapse that sequence of equalizers to one equation. ■

Theorem (`theorem_6_7`). A subset $U \subseteq \mathbf{P}\omega$ is a $\mathfrak{B}_\delta$ if and only if $U = \{x \mid f(x) = g(x)\}$ for some Scott-continuous $f, g : \mathbf{P}\omega \to \mathbf{P}\omega$.

Proof. The two directions are the equalizer and $\mathfrak{B}_\delta$ tools above. ■

`theorem_6_7.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Classification.Classes
public import DomainTheory.DataTypesAsLattices.Classification.ThreeValued
import DomainTheory.DataTypesAsLattices.Classification.CharacteristicMaps
import DomainTheory.DataTypesAsLattices.Classification.pairIndexed
import DomainTheory.DataTypesAsLattices.Classification.theorem_6_6
import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen_more
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
import Mathlib.Tactic.Bound.Init

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

namespace theorem_6_7

theorem of_equalizer :
```

```
    ∀ U, (∃ f g, IsScottContinuous f ∧ IsScottContinuous g ∧
        U = {x | f x = g x}) → IsBdelta U := by
  rintro U ⟨f, g, hf, hg, rfl⟩
  refine ⟨fun n => {x | n ∈ f x ∧ n ∈ g x} ∪ {x | n ∈ f x ∧ n ∈ g x}, ?_, ?_⟩
  · intro n
    refine (IsBooleanOpen.ofOpen
      (IsScottOpen.inter (IsScottContinuous.mem_preimage hf n)
        (IsScottContinuous.mem_preimage hg n))).union
      (IsBooleanOpen.of_closed ?_)
    have : {x | n ∈ f x ∧ n ∈ g x}ᶜ = {x | n ∈ f x}ᶜ ∪ {x | n ∈ g x}ᶜ := by
      ext x
      constructor
      · intro hx
        have : ¬ (n ∈ f x ∧ n ∈ g x) := hx
        by_cases hfx : n ∈ f x
        · exact Or.inl hfx
        · by_cases hgx : n ∈ g x
          · exact Or.inr hgx
          · exact (this ⟨hfx, hgx⟩).elim
      · intro hx h
        rcases hx with hf' | hg'
        · exact h.1 hf'
        · exact h.2 hg'
    rw [this]
    exact IsScottOpen.union (IsScottContinuous.mem_preimage hf n)
      (IsScottContinuous.mem_preimage hg n)
  · ext x
    constructor
    · intro h
      refine Set.mem_iInter.mpr fun n => ?_
      have heq : f x = g x := h
      by_cases hn : n ∈ f x
      · exact Or.inl ⟨hn, by rwa [← heq]⟩
      · exact Or.inr ⟨hn, by simpa [heq] using hn⟩
    · intro hx
      ext n
      have hn := Set.mem_iInter.mp hx n
      constructor
      · intro hfx
        rcases hn with ⟨_, hng⟩ | ⟨hnf, _⟩
        · exact hng
        · exact (hnf hfx).elim
      · intro hgx
        rcases hn with ⟨hnf, _⟩ | ⟨_, hng⟩
        · exact hnf
        · exact (hng hgx).elim

theorem iInter_equalizer (f g : ℕ → Pomega → Pomega)
    (hf : ∀ n, IsScottContinuous (f n)) (hg : ∀ n, IsScottContinuous (g n)) :
    ∃ F G, IsScottContinuous F ∧ IsScottContinuous G ∧
      {x | ∀ n, f n x = g n x} = {x | F x = G x} :=
  ⟨pairIndexed f, pairIndexed g,
    pairIndexed.isScottContinuous f hf, pairIndexed.isScottContinuous g hg, by
    ext x
    constructor
    · intro h
```

```
      ext k
      constructor
      · intro ⟨n, m, hk, hm⟩
        exact ⟨n, m, hk, by rwa [← h n]⟩
      · intro ⟨n, m, hk, hm⟩
        exact ⟨n, m, hk, by rwa [h n]⟩
    · intro h n
      ext m
      constructor
      · intro hm
        have : pair n m ∈ pairIndexed g x := by
          have : pair n m ∈ pairIndexed f x := ⟨n, m, rfl, hm⟩
          rwa [h] at this
        obtain ⟨n', m', heq, hm'⟩ := this
        obtain ⟨rfl, rfl⟩ := pair_inj heq
        exact hm'
      · intro hm
        have : pair n m ∈ pairIndexed f x := by
          have : pair n m ∈ pairIndexed g x := ⟨n, m, rfl, hm⟩
          rwa [← h] at this
        obtain ⟨n', m', heq, hm'⟩ := this
        obtain ⟨rfl, rfl⟩ := pair_inj heq
        exact hm'⟩

theorem converse_of_equalizers (f g : ℕ → Pomega → Pomega)
    (hf : ∀ n, IsScottContinuous (f n)) (hg : ∀ n, IsScottContinuous (g n))
    {U : Set Pomega} (hU : U = {x | ∀ n, f n x = g n x}) :
    ∃ F G, IsScottContinuous F ∧ IsScottContinuous G ∧ U = {x | F x = G x} := by
  obtain ⟨F, G, hF, hG, h⟩ := iInter_equalizer f g hf hg
  exact ⟨F, G, hF, hG, hU.trans h⟩

theorem boolean_as_iInter_open_union_closed {U : Set Pomega}
    (hU : IsBooleanOpen U) :
    ∃ (O C : ℕ → Set Pomega),
      (∀ n, IsScottOpen (O n)) ∧ (∀ n, IsScottOpen (C n)?) ∧
        U = ⋂ n, O n ∪ C n := by
  have hUc : U? ∈ ScottB := hU.compl
  obtain ⟨f, hf, E, hE, hEfin, hEeq⟩ := (theorem_6_6 U?).mp hUc
  have hrec : ∀ {S : Set Pomega} (hS : S.Finite),
      (∀ e ∈ S, e.Finite) →
      ∃ O C : ℕ → Set Pomega,
        (∀ n, IsScottOpen (O n)) ∧ (∀ n, IsScottOpen (C n)?) ∧
          {x | f x ∈ S} = ⋂ n, O n ∪ C n := by
    intro S hS
    induction S, hS using Set.Finite.induction_on with
    | empty =>
      intro
      refine ⟨fun _ => Set.univ, fun _ => ∅, fun _ => IsScottOpen.univ,
        fun _ => by rw [Set.compl_empty]; exact IsScottOpen.univ, ?_⟩
      ext x
      simp
    | insert ha hs ih =>
      intro hfin
      rename_i a s
      obtain ⟨Os, Cs, hOs, hCs, hseq⟩ :=
        ih fun e he => hfin e (Set.mem_insert_of_mem a he)
```

```
      have haFin : a.Finite := hfin a (Set.mem_insert a s)
      obtain ⟨hslice, hOa, hCa⟩ := IsScottContinuous.ne_eq_open_union_closed hf haFin
      refine ⟨fun n => match n with
        | 0 => {x | ¬ f x ⊆ a}
        | _n + 1 => Os _n,
        fun n => match n with
        | 0 => {x | ¬ a ⊆ f x}
        | _n + 1 => Cs _n, ?_, ?_, ?_⟩
      · intro n
        cases n with
        | zero => exact hOa
        | succ n => exact hOs n
      · intro n
        cases n with
        | zero => exact hCa
        | succ n => exact hCs n
      · have hUeq : {x | f x ∈ insert a s} =
            ({x | f x = a} ∩ {x | f x ∈ s}) := by
          ext x
          simp [Set.mem_insert_iff]
        rw [hUeq, hslice, hseq]
        ext x
        constructor
        · intro ⟨ha', hs'⟩
          refine Set.mem_iInter.mpr fun n => ?_
          cases n with
          | zero => exact ha'
          | succ n => exact Set.mem_iInter.mp hs' n
        · intro hx
          exact ⟨Set.mem_iInter.mp hx 0,
            Set.mem_iInter.mpr fun n => Set.mem_iInter.mp hx (n + 1)⟩
  obtain ⟨O, C, hO, hC, hOC⟩ := hrec hE hEfin
  refine ⟨O, C, hO, hC, ?_⟩
  have hUeq : U = {x | f x ∈ E} := by
    ext x
    constructor
    · intro hxU hfxE
      have hxUc : x ∈ U? := by
        rw [hEeq]
        exact hfxE
      exact hxUc hxU
    · intro hx
      by_contra hxU
      apply hx
      change x ∈ {x | f x ∈ E}
      rwa [← hEeq]
  rw [hUeq, hOC]

theorem of_bdelta {U : Set Pomega} (hU : IsBdelta U) :
    ∃ f g, IsScottContinuous f ∧ IsScottContinuous g ∧ U = {x | f x = g x} := by
  obtain ⟨Bs, hB, rfl⟩ := hU
  have hslice : ∀ n, ∃ O C : ℕ → Set Pomega,
      (∀ k, IsScottOpen (O k)) ∧ (∀ k, IsScottOpen (C k)?) ∧
        Bs n = ⋂ k, O k ∪ C k :=
    fun n => boolean_as_iInter_open_union_closed (hB n)
  choose Os Cs hOs hCs hEq using hslice
```

```
  let O (m : ℕ) : Set Pomega := Os (unpair m).1 (unpair m).2
  let C (m : ℕ) : Set Pomega := Cs (unpair m).1 (unpair m).2
  have hO : ∀ m, IsScottOpen (O m) := fun m => hOs (unpair m).1 (unpair m).2
  have hC : ∀ m, IsScottOpen (C m)ᶜ := fun m => hCs (unpair m).1 (unpair m).2
  have hUC : (⋂ n, Bs n) = ⋂ m, O m ∪ C m := by
    ext x
    constructor
    · intro hx
      refine Set.mem_iInter.mpr fun m => ?_
      have hx' := Set.mem_iInter.mp hx (unpair m).1
      rw [hEq] at hx'
      exact Set.mem_iInter.mp hx' (unpair m).2
    · intro hx
      refine Set.mem_iInter.mpr fun n => ?_
      rw [hEq]
      refine Set.mem_iInter.mpr fun k => ?_
      simpa [O, C, unpair_pair] using Set.mem_iInter.mp hx (pair n k)
  let f : ℕ → Pomega → Pomega := fun m x => charOpen (O m) x
  let g : ℕ → Pomega → Pomega := fun m x => closedToZero (C m) x
  have hf : ∀ m, IsScottContinuous (f m) :=
    fun m => charOpen.isScottContinuous (O m)
  have hg : ∀ m, IsScottContinuous (g m) :=
    fun m => closedToZero.isScottContinuous (C m)
  let F : ℕ → Pomega → Pomega := fun m x => uvU (f m x) (g m x)
  let G : ℕ → Pomega → Pomega := fun m x => uvV (f m x) (g m x)
  have hF : ∀ m, IsScottContinuous (F m) :=
    fun m => theorem_1_3.tuple (uvU.isScottContinuous_left) (uvU.
    isScottContinuous_right)
      (hf m) (hg m)
  have hG : ∀ m, IsScottContinuous (G m) :=
    fun m => theorem_1_3.tuple (uvV.isScottContinuous_left) (uvV.
    isScottContinuous_right)
      (hf m) (hg m)
  have hpack : (⋂ n, Bs n) = {x | ∀ m, F m x = G m x} := by
    rw [hUC]
    ext x
    constructor
    · intro hx m
      have hx' := Set.mem_iInter.mp hx m
      have hfg : f m x = ofNat 0 ∨ g m x = ofNat 0 := by
        rcases hx' with hOm | hCm
        · exact Or.inl ((charOpen.eq_zero (hO m) x).mpr hOm)
        · exact Or.inr ((closedToZero.eq_zero (hC m) x).mpr hCm)
      exact (ThreeValued.uvU_eq_uvV (charOpen.threeValued (O m) x)
        (closedToZero.threeValued (hC m) x)).mpr hfg
    · intro hx
      refine Set.mem_iInter.mpr fun m => ?_
      have heq := (ThreeValued.uvU_eq_uvV (charOpen.threeValued (O m) x)
        (closedToZero.threeValued (hC m) x)).mp (hx m)
      rcases heq with hf0 | hg0
      · exact Or.inl ((charOpen.eq_zero (hO m) x).mp hf0)
      · exact Or.inr ((closedToZero.eq_zero (hC m) x).mp hg0)
  exact converse_of_equalizers F G hF hG hpack

end theorem_6_7
```

```
theorem theorem_6_7 :
    ∀ U, IsBdelta U ↔
      ∃ f g, IsScottContinuous f ∧ IsScottContinuous g ∧
        U = {x | f x = g x} :=
  fun U => ⟨theorem_6_7.of_bdelta, theorem_6_7.of_equalizer U⟩

end DomainTheory.DataTypesAsLattices
```

3.8 Total functions and functionality

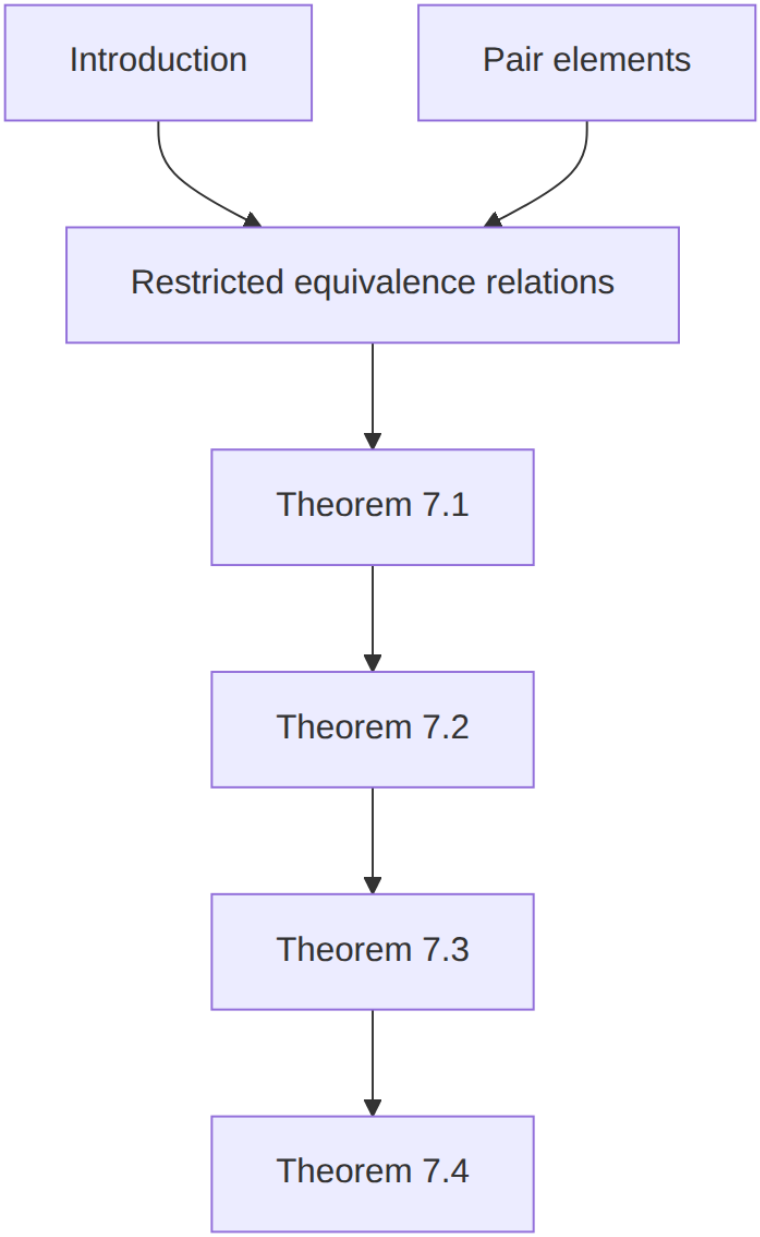


Figure 37: Total functions and functionality.

3.8.1 Introduction ✓

There is an inevitable conflict between the concepts of *total* and *partial* functions: we desire the former, but it is the latter we usually get. Total functions are better because they are "well-defined" at all their arguments, but the rub is that there is no general way of deciding when a *definition* is going to be well-defined in all its

uses. In analysis we have singularities, and in recursion theory we have endless, nonfinishing computations. In the present theory we have in effect evaded the question in two ways. First we have embraced the partial function as the norm. But secondly, and possibly confusingly, the multiple-valued functions are normal, total functions from $\mathbf{P}\omega$ into $\mathbf{P}\omega$. The point, of course, is that we are making a *model* of the partial functions in terms of ordinary mathematical functions. But note that the success of the *model* lies in *not* using arbitrary functions: it is only the continuous functions that correspond to the kind of partial functions we wanted to study. It would be a mistake to think of the variables in λ-calculus as ranging over arbitrary functions—and this mistake was made by both Church and Curry. The fixed-point operator $\mathbf{Y}$ shows that we must restrict attention to functions which do have fixed points. It is certainly the case that $\mathbf{P}\omega$ is not the only model for the λ-calculus, but it is a very satisfactory model and is rich enough to illustrate what can and what cannot be done with partial functions.

Historical note. Functionality had long been studied in combinatory logic. [104] is the introduction Scott used. This section is his reading of that theory in $\mathbf{P}\omega$. The connection with the functional interpretation of intuitionistic logic goes back to Curry and Kleene, and Theorem 7.3 is the standard result of that interpretation. Theorem 7.4, that the obvious typing characterizes the iterators, is due to Gordon Plotkin.

Whatever the pleasures of partial functions (and the multiple-valued ones, too), the desire for total functions remains. Take the integers. We are more interested in ω than $\omega \cup \{\bot, \top\}$. Since the multiple values $\bot$ and $\top$ are but two in number, it is easy to avoid them. The problem becomes tiresome in considering functions, however. The lattice represented by the retract $\mathbf{int} \circ \to \mathbf{int}$ is much too large, in that there are as many nontotal functions in this domain as total ones. The aim of the present section is to introduce an interpretation of a theory of functionality in the model $\mathbf{P}\omega$ that provides a convenient way of restricting attention to the functions (or other objects) that are total in the desired sense. The theory of functionality is rather like proposals of Curry, but not quite the same for important reasons as we will see.

In the theory of retracts of §§ 5 and 6, the plan of "restricting attention" was the very simple one of restricting to a *subset*. It was made notationally simple as the subsets in question could be parameterized by continuous functions. The retraction mappings stand in for their ranges. Even better, certain continuous functions act on these retractions as space-forming functors (such as $\oplus$ and $\circ \to$), which gives greater notational simplicity because one language is able to serve for several

tasks. When we pass to the theory of total functions, this same kind of simplicity is no longer possible owing to an increase in quantifier complexity in the necessary definitions. (This remark is made definite below.) Another point where there is some loss of simplicity concerns the representation of entities in $\mathbf{P}\omega$: subsets will no longer be enough, since we will need *quotients* of subsets. This is not a very startling point. Many constructions are affected in a natural way via equivalence classes. An equivalence relation makes you blind to certain distinctions. It may be easier also to remain a bit blind than to search for the most beautiful representative of an equivalence class: there may be nothing to choose between several candidates, and it can cost too much effort to attempt a choice. Thus our first agreement is that for many purposes a *kind* of object can be taken as a set of equivalence classes for an equivalence relation on a subset of $\mathbf{P}\omega$.

Because $\mathbf{P}\omega$ is closed under the pairing function $\lambda x \lambda y.\, \langle x, y\rangle$, we shall construe relations on subsets of $\mathbf{P}\omega$ as subsets of $\mathbf{P}\omega$ all of whose elements are ordered pairs. That is, a relation A satisfies this inclusion:

$$(7.1) \quad A \subseteq \{\langle x, y\rangle \mid x, y \in \mathbf{P}\omega\}.$$

3.8.2 Pair elements ✓

Ordered pairs of elements of $\mathbf{P}\omega$, as length-2 sequences.

Definition (`pairElem`). $\langle x, y\rangle = \mathrm{seq}_2(x, y)$.

Theorem (`pairElem.tag_ne`). $\langle 0, x\rangle \neq \langle 1, y\rangle$.

Proof. Distinct tags on length-2 sequences. ■

Theorem (`pairElem.eq_pairSeq`). $\langle x, y\rangle$ agrees with the retract pairing $\mathrm{pairSeq}(x, y)$.

Proof. Both are $\mathrm{seq}_2(x, y)$. ■

Theorem (`pairElem.fst`). $\langle x, y\rangle_0 = x$.

Proof. The zero coordinate of a length-2 sequence. ■

Theorem (`pairElem.snd`). $\langle x, y\rangle_1 = y$.

Proof. The one coordinate of a length-2 sequence. ■

Theorem (`pairElem.inj`). If $\langle x, y\rangle = \langle x', y'\rangle$, then $x = x'$ and $y = y'$.

Proof. Project both sides with the first and second coordinates. ■

`pairElem.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def pairElem (x y : Pomega) : Pomega := seq2 x y

namespace pairElem

theorem tag_ne (x y : Pomega) :
    pairElem (ofNat 0) x = pairElem (ofNat 1) y :=
  seq2.tag_ne x y

theorem eq_pairSeq (x y : Pomega) : pairElem x y = pairSeq x y := rfl

theorem fst (x y : Pomega) : funOf (pairElem x y) (ofNat 0) = x :=
  seq2.zero x y

theorem snd (x y : Pomega) : funOf (pairElem x y) (ofNat 1) = y :=
  seq2.one x y

theorem inj {x y x' y' : Pomega} (h : pairElem x y = pairElem x' y') :
    x = x' ∧ y = y' := by
  constructor
  · have := congrArg (fun w => funOf w (ofNat 0)) h
    rw [fst, fst] at this
    exact this
  · have := congrArg (fun w => funOf w (ofNat 1)) h
    rw [snd, snd] at this
    exact this

end pairElem

end DomainTheory.DataTypesAsLattices
```

3.8.3 Restricted equivalence relations ✓

DEFINITION. A (restricted) *equivalence relation* on $\mathbf{P}\omega$ is a symmetric and transitive relation on $\mathbf{P}\omega$.

Such relations are restricted because they are only reflexive on their domains—which are the same as their ranges—and these are the subsets with which the relations are concerned. We shall write $x\,A\,y$ for $\langle x, y\rangle \in A$ and $x : A$ for $x\,A\,x$. What we assume about these relations is the following:

$$(7.2) \quad x\,A\,y \text{ implies } y\,A\,x.$$

$$(7.3) \quad x\,A\,y \text{ and } y\,A\,z \text{ imply } x\,A\,z.$$

In case a is a retract, we introduce an equivalence relation to correspond:

$$(7.4) \quad E_a = \{\langle x, x\rangle \mid x : a\}.$$

This is the identity relation restricted to the range of a. Such relations (for obvious reasons) and many others satisfy an additional *intersection property*:

$$(7.5) \quad x\,A\,y \text{ and } x\,A\,z \text{ imply } x\,A\,(y \cap z).$$

We shall not generally assume (7.5) in this short discussion, but it is often convenient.

Each equivalence relation represents a *space*: the space of all its equivalence classes. Such spaces form a category more extensive than the category of retracts studied above. The familiar functors can be extended to this larger category by these definitions:

$$(7.6) \quad A \to B = \{\langle \lambda x.\, u(x), \lambda x.\, v(x)\rangle \mid u(x)\,B\,v(y) \text{ whenever } x\,A\,y\},$$

$$(7.7) \quad A \times B = \{\langle \langle x, x'\rangle, \langle y, y'\rangle\rangle \mid x\,A\,y \text{ and } x'\,B\,y'\},$$

$$(7.8) \quad A + B = \{\langle \langle 0, x\rangle, \langle 0, y\rangle\rangle \mid x\,A\,y\} \cup \{\langle \langle 1, x'\rangle, \langle 1, y'\rangle\rangle \mid x'\,B\,y'\}.$$

Formalization. Restricted equivalences are packaged as a structure with a binary relation that is symmetric and transitive. Membership $x : A$ is reflexivity $x\,A\,x$. The retract identity E_a, the functors (7.6)–(7.8), and the empty and singleton equivalences are the constructions below; symmetry and transitivity of each are part of the definition.

Definition (`RestrictedEquiv`). A restricted equivalence is a relation A on $\mathbf{P}\omega$ satisfying (7.2) and (7.3).

Definition (`RestrictedEquiv.mem`). $x : A$ means $x\,A\,x$.

Definition (`RestrictedEquiv.Ea`). E_a relates x to y just when $x : a$, $y : a$, and $x = y$.

Definition (`RestrictedEquiv.arrowE`). $A \to B$ relates f to g when both are graphs of their own application and $x\,A\,y$ implies $f(x)\,B\,g(y)$.

Definition (`RestrictedEquiv.prodE`). $A \times B$ relates u to v when both are pairs of their projections and the projections are related in A and in B respectively.

Definition (`RestrictedEquiv.sumE`). $A + B$ relates u to v when both are left injections whose payloads are related in A, or both are right injections whose payloads are related in B.

Definition (`RestrictedEquiv.emptyE`). The empty restricted equivalence relates nothing.

Definition (`RestrictedEquiv.singletonE`). The singleton on a relates x to y just when $x = y = a$.

Theorem (`singletonE.mem`). x is a member of the singleton on a if and only if $x = a$.

Proof. Unfold membership as $x\,A\,x$. ■

Definition (`RestrictedEquiv.Nk`). N_k relates x to y when $x = y = \overline{n}$ for some $n < k$.

Definition (`RestrictedEquiv.Nrel`). N relates x to y when $x = y = \overline{n}$ for some $n \in \omega$.

Definition (`RestrictedEquiv.kerE`). The kernel of a relates x to y when $a(x) = a(y)$.

Theorem (`kerE.repr`). If a is a retract, then x is kernel-related to $a(x)$.

Proof. Retract-fixedness gives $a(a(x)) = a(x)$. ■

Definition (`iterateE`). The iterate equivalence at f and x relates u to v when $u = v = Z(n, f, x)$ for some n.

RestrictedEquiv.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Functionality.pairElem
import DomainTheory.DataTypesAsLattices.Retracts.retract_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

structure RestrictedEquiv where
  rel : Pomega → Pomega → Prop
  symm : ∀ {x y}, rel x y → rel y x
  trans : ∀ {x y z}, rel x y → rel y z → rel x z

namespace RestrictedEquiv

def mem (A : RestrictedEquiv) (x : Pomega) : Prop :=
  A.rel x x

def Ea (a : Pomega) : RestrictedEquiv where
  rel := fun x y => typed x a ∧ typed y a ∧ x = y
  symm := by
    intro x y ⟨hx, hy, h⟩
    exact ⟨hy, hx, h.symm⟩
  trans := by
    intro x y z ⟨hx, hy, hxy⟩ ⟨_, hz, hyz⟩
    exact ⟨hx, hz, hxy.trans hyz⟩

def arrowE (A B : RestrictedEquiv) : RestrictedEquiv where
  rel := fun f g =>
    f = graph (fun x => funOf f x) ∧
      g = graph (fun x => funOf g x) ∧
        ∀ x y, A.rel x y → B.rel (funOf f x) (funOf g y)
  symm := by
    intro f g ⟨hf, hg, h⟩
    exact ⟨hg, hf, fun x y hxy => B.symm (h y x (A.symm hxy))⟩
  trans := by
    intro f g k ⟨hf, hg, hfg⟩ ⟨hg', hk, hgk⟩
    exact ⟨hf, hk, fun x y hxy =>
      B.trans (hfg x y hxy) (hgk y y (A.trans (A.symm hxy) hxy))⟩

def prodE (A B : RestrictedEquiv) : RestrictedEquiv where
```

```
  rel := fun u v =>
    u = pairElem (funOf u (ofNat 0)) (funOf u (ofNat 1)) ∧
      v = pairElem (funOf v (ofNat 0)) (funOf v (ofNat 1)) ∧
        A.rel (funOf u (ofNat 0)) (funOf v (ofNat 0)) ∧
          B.rel (funOf u (ofNat 1)) (funOf v (ofNat 1))
  symm := by
    intro u v ⟨hu, hv, h0, h1⟩
    exact ⟨hv, hu, A.symm h0, B.symm h1⟩
  trans := by
    intro u v w ⟨hu, hv, h0, h1⟩ ⟨hv', hw, k0, k1⟩
    exact ⟨hu, hw, A.trans h0 k0, B.trans h1 k1⟩

def sumE (A B : RestrictedEquiv) : RestrictedEquiv where
  rel := fun u v =>
    (u = pairElem (ofNat 0) (funOf u (ofNat 1)) ∧
        v = pairElem (ofNat 0) (funOf v (ofNat 1)) ∧
        A.rel (funOf u (ofNat 1)) (funOf v (ofNat 1))) ∨
      (u = pairElem (ofNat 1) (funOf u (ofNat 1)) ∧
        v = pairElem (ofNat 1) (funOf v (ofNat 1)) ∧
        B.rel (funOf u (ofNat 1)) (funOf v (ofNat 1)))
  symm := by
    intro u v h
    rcases h with ⟨hu, hv, hA⟩ | ⟨hu, hv, hB⟩
    · exact Or.inl ⟨hv, hu, A.symm hA⟩
    · exact Or.inr ⟨hv, hu, B.symm hB⟩
  trans := by
    intro u v w h1 h2
    rcases h1 with ⟨hu, hv, hA⟩ | ⟨hu, hv, hB⟩
    · rcases h2 with ⟨hv', hw, hA'⟩ | ⟨hv', hw, _⟩
      · exact Or.inl ⟨hu, hw, A.trans hA hA'⟩
      · exact (pairElem.tag_ne (funOf v (ofNat 1)) (funOf v (ofNat 1))
          (hv.symm.trans hv')).elim
    · rcases h2 with ⟨hv', hw, _⟩ | ⟨hv', hw, hB'⟩
      · exact (pairElem.tag_ne (funOf v (ofNat 1)) (funOf v (ofNat 1))
          (hv'.symm.trans hv)).elim
      · exact Or.inr ⟨hu, hw, B.trans hB hB'⟩

def emptyE : RestrictedEquiv where
  rel := fun _ _ => False
  symm := fun h => False.elim h
  trans := fun h _ => False.elim h

def singletonE (a : Pomega) : RestrictedEquiv where
  rel := fun x y => x = a ∧ y = a
  symm := by
    intro x y ⟨hx, hy⟩
    exact ⟨hy, hx⟩
  trans := by
    intro x y z ⟨hx, _hy⟩ ⟨_hy', hz⟩
    exact ⟨hx, hz⟩

def Nk (k : ℕ) : RestrictedEquiv where
  rel := fun x y => (∃ n < k, x = ofNat n) ∧ x = y
  symm := by
    intro x y ⟨hn, h⟩
    exact ⟨by rw [← h]; exact hn, h.symm⟩
```

```
  trans := by
    intro x y z ⟨hn, hxy⟩ ⟨_, hyz⟩
    exact ⟨hn, hxy.trans hyz⟩

def Nrel : RestrictedEquiv where
  rel := fun x y => (∃ n, x = ofNat n) ∧ x = y
  symm := by
    intro x y ⟨hn, h⟩
    exact ⟨by rw [← h]; exact hn, h.symm⟩
  trans := by
    intro x y z ⟨hn, hxy⟩ ⟨_, hyz⟩
    exact ⟨hn, hxy.trans hyz⟩

def kerE (a : Pomega) : RestrictedEquiv where
  rel := fun x y => funOf a x = funOf a y
  symm := by
    intro x y h
    exact h.symm
  trans := by
    intro x y z hxy hyz
    exact hxy.trans hyz

end RestrictedEquiv

export RestrictedEquiv (Ea arrowE prodE sumE emptyE singletonE Nk Nrel kerE)

namespace singletonE

theorem mem (a x : Pomega) : (singletonE a).mem x ↔ x = a :=
  ⟨fun h => h.1, fun hx => ⟨hx, hx⟩⟩

end singletonE

namespace kerE

theorem repr {a : Pomega} (ha : Retract a) (x : Pomega) :
    (kerE a).rel x (funOf a x) :=
  (retract_app ha x).symm

end kerE

end DomainTheory.DataTypesAsLattices
```

iterateE.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.Zcomb

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def iterateE (f x : Pomega) : RestrictedEquiv where
  rel := fun u v => (∃ n, u = Z n f x) ∧ u = v
```

```
  symm := by
    intro u v ⟨hn, h⟩
    exact ⟨by rw [← h]; exact hn, h.symm⟩
  trans := by
    intro u v w ⟨hn, hxy⟩ ⟨_, hyz⟩
    exact ⟨hn, hxy.trans hyz⟩

end DomainTheory.DataTypesAsLattices
```

3.8.4 Theorem 7.1 ✓

THEOREM 7.1 (The closure theorem). *If A and B are restricted equivalence relations, then so are $A \to B$, $A \times B$ and $A + B$. We find:*

(i) $f : A \to B$ iff $f = \lambda x.\, f(x)$ and whenever $x\, A\, y$, then $f(x)\, B\, f(y)$, in particular:

(ii) if $f : A \to B$ and $x : A$, then $f(x) : B$; furthermore,

(iii) $u : A \times B$ iff $u = \langle u_0, u_1 \rangle$ and $u_0 : A$ and $u_1 : B$;

(iv) $u : A + B$ iff either $u = \langle 0, u_1 \rangle$ and $u_1 : A$ or $u = \langle 1, u_1 \rangle$ and $u_1 : B$.

It follows easily from (7.1) that the restricted equivalence relations form a Cartesian closed category which—in distinction to the category of retracts—has disjoint sums (or coproducts as they are usually called in category theory). This result is probably a special case of a more general theorem. The point is that $\mathbf{P}\omega$ itself is a space in a Cartesian closed category (that of continuous lattices and continuous maps) and it contains as subspaces the Boolean space and especially its own function space and Cartesian square. In this circumstance any such rich space must be such that its restricted equivalence relations again form a good category. Our construction is not strictly categorical in nature, as we have used the elements of $\mathbf{P}\omega$ and have relied on being able to form arbitrary subsets (arbitrary relations). But a more abstract formulation must be possible. The connection with the category of retracts is indicated in the next theorem.

Formalization. The closure theorem unpacks membership in $A \to B$, $A \times B$, and $A + B$ against the definitions of those functors. Symmetry and transitivity of the functors themselves are already part of those constructions.

Theorem (`theorem_7_1`). For restricted equivalences A and B: (i) $f : A \to B$ if and only if f equals the graph of its own application and $x\, A\, y$ implies $f(x)\, B\, f(y)$; (ii) if $f : A \to B$ and $x : A$, then $f(x) : B$; (iii) $u : A \times B$ if and only if $u = \langle u_0, u_1 \rangle$ with $u_0 : A$ and $u_1 : B$; (iv) $u : A + B$ if and only if $u = \langle 0, u_1 \rangle$ with $u_1 : A$, or $u = \langle 1, u_1 \rangle$ with $u_1 : B$.

Proof. Each clause is an unpacking of membership in the corresponding functor: for the arrow, the two graph conjuncts collapse under $f = g$; preservation on a related pair yields (ii) at $x\, A\, x$; for the product and sum, the paired projections (resp. common injection tag) reduce relatedness of two copies of u to membership of the components. ■

`theorem_7_1.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem theorem_7_1 (A B : RestrictedEquiv) :
    (∀ f, (arrowE A B).mem f ↔
        f = graph (fun x => funOf f x) ∧
          ∀ x y, A.rel x y → B.rel (funOf f x) (funOf f y)) ∧
      (∀ {f x}, (arrowE A B).mem f → A.mem x → B.mem (funOf f x)) ∧
      (∀ u, (prodE A B).mem u ↔
        u = pairElem (funOf u (ofNat 0)) (funOf u (ofNat 1)) ∧
          A.mem (funOf u (ofNat 0)) ∧ B.mem (funOf u (ofNat 1))) ∧
      (∀ u, (sumE A B).mem u ↔
        (u = pairElem (ofNat 0) (funOf u (ofNat 1)) ∧
          A.mem (funOf u (ofNat 1))) ∨
        (u = pairElem (ofNat 1) (funOf u (ofNat 1)) ∧
          B.mem (funOf u (ofNat 1)))) := by
  refine ⟨?_, ?_, ?_, ?_⟩
  · intro f
    constructor
    · intro hf
      exact ⟨hf.1, fun x y hxy => hf.2.2 x y hxy⟩
    · intro ⟨hf, hpres⟩
      exact ⟨hf, hf, hpres⟩
  · intro f x hf hx
    exact hf.2.2 x x hx
  · intro u
    constructor
    · intro hu
      exact ⟨hu.1, hu.2.2.1, hu.2.2.2⟩
    · intro ⟨hu, hA, hB⟩
      exact ⟨hu, hu, hA, hB⟩
  · intro u
    constructor
    · intro hu
      rcases hu with ⟨hu, _, hA⟩ | ⟨hu, _, hB⟩
      · exact Or.inl ⟨hu, hA⟩
      · exact Or.inr ⟨hu, hB⟩
    · intro h
      rcases h with ⟨hu, hA⟩ | ⟨hu, hB⟩
      · exact Or.inl ⟨hu, hu, hA⟩
      · exact Or.inr ⟨hu, hu, hB⟩

end DomainTheory.DataTypesAsLattices
```

3.8.5 Theorem 7.2 ✓

THEOREM 7.2 (The isomorphism theorem). *If a and b are retracts, we have the following isomorphisms and identities relating the spaces:*

(i) $E_a \cong \{\langle x, y\rangle \mid a(x) = a(y)\}$;

(ii) $E_{a\circ\to b} \cong E_a \to E_b$;

(iii) $E_{a\otimes b} = E_a \times E_b$;

(iv) $E_{a\oplus b} = E_a + E_b \cup \{\langle\bot,\bot\rangle, \langle\top,\top\rangle\}$.

Part (iv) is not categorical in nature as it stands, but (ii) and (iii) indicate that E is a functor from the category of retracts into the category of equivalence relations that shows that the former is a full sub-Cartesian-closed category of the latter. We cannot pursue the categorical questions here, but note that there are many subcategories that might be of interest; for example, the equivalence relations with the intersection property are closed under $\to$, $\times$, and $+$.

Returning to the question of total functions we introduce this notation:

$$(7.9) \quad N = \{\langle n, n\rangle \mid n \in \omega\}.$$

This is the type of the integers *without* $\bot$ and $\top$, i.e., the total integers. We note that:

$$(7.10) \quad N = \{u \mid u = \langle u_0, u_0\rangle,\ u_0 = \mathbf{int}(u_0),\ \mathbf{mid}(u_0) = 0\}.$$

Thus N is a $\mathfrak{B}_\delta$-set. What is $N \to N$? We see:

$$\{(7.11) \quad f : N \to N \text{ iff } f : \mathbf{fun} \text{ and } f(n) \in \omega \text{ whenever } n \in \omega.\}$$

This $N \to N$ is indeed the type of all total functions from ω into ω. It can be shown that $N \to N$ is also a $\mathfrak{B}_\delta$-set: good. But what is $(N \to N) \to N$? This is no longer a $\mathfrak{B}_\delta$-set, the best we can say is Π^1_1. By Theorem 7.1 it corresponds to the type of all

(extensional) *continuous* total functions from $N \to N$ into N. (The condition on A and B on the right side of (7.6) makes the concept of function embodied in $A \to B$ extensional, since the functions are meant to preserve the equivalence relations.)

A more precise discussion identifies $N \to N$ as a topological space, usually called the *Baire space*. If we introduce the finite discrete spaces by:

$$(7.12) \quad N_k = \{\langle n, n\rangle \mid n < k\},$$

then $N \to N_2$ can also be identified with a topological space, usually called the *Cantor space*. In this identification we find at the next type, say either $(N \to N) \to N$ or $(N \to N_2) \to N_2$, that elements correspond to the usual notion of continuous function defined in topological terms. However, these higher type spaces are not at all conveniently taken as *topological* spaces. Certain of them can be identified as *limit spaces* according to the work of Hyland, and for these $\to$, $\times$, and $+$ have the natural interpretation. We cannot enter into these details here, but we can remark that the higher type spaces become ever more complicated. Thus $((N \to N) \to N) \to N$ is a Π^1_2-set and each $\to$ will add another quantifier to the definition. This is reasonable, because to say that a function is total is to say that *all* its values are well-behaved. But if its domain is a complex space, this statement of totality is even more complex. Despite this complexity, however, it is possible to sort out what kind of mapping properties many functions have. We shall mention a few of the combinators.

Formalization. The isomorphism theorem relates the retract identity E_a to kernels, arrows, products, and sums. The packaging theorem assembles the four clauses for retracts a and b.

Theorem (`theorem_7_2.i`). $x\ E_a\ y$ if and only if $x : a$, $y : a$, and $x = y$.

Proof. Unfolding the definition of E_a. ■

Theorem (`theorem_7_2.i_iso`). If a is a retract, then $a(x) = a(y)$ if and only if $a(x)\ E_a\ a(y)$.

Proof. Both sides reduce to equality of the images once retract-fixedness supplies $a(x) : a$ and $a(y) : a$. ■

Theorem (`theorem_7_2.ii_of_typed`). If $f : a\circ \to b$, then $f : E_a \to E_b$.

Proof. Typing into the retract arrow rewrites f as a graph; relatedness in E_a is equality of a-typed points, and the retract arrow preserves typing into b. ■

Theorem (`theorem_7_2.ii`). For retracts a and b, typing into $a\circ \to b$ implies membership in $E_a \to E_b$; relatedness in $E_a \to E_b$ passes to relatedness of the retract-arrow applications in $E_{a\circ\to b}$ and conversely.

Proof. The forward typing clause is the previous theorem; the two directions between $E_a \to E_b$ and $E_{a\circ\to b}$ compare graph preservation on a-fixed arguments with equality after applying $a\circ \to b$. ■

Theorem (`theorem_7_2.iii`). On paired arguments, $E_{a\otimes b}$ agrees with $E_a \times E_b$.

Proof. Unpack tensor typing of a pair into component typings, or reassemble them. ■

Theorem (`theorem_7_2.iii_exact`). For arbitrary u and v, $u\, E_{a\otimes b}\, v$ if and only if $u\, (E_a \times E_b)\, v$.

Proof. Reduce via the product membership characterisation of Theorem 4.4 to the previous clause on pairs. ■

Theorem (`theorem_7_2.iv`). For retracts a and b, $u\, E_{a\oplus b}\, v$ if and only if $u\, (E_a + E_b)\, v$, or both are $\bot$, or both are $\top$.

Proof. Case on the sum typing of u: bottom, top, left injection, or right injection; the converse rebuilds sum typing from the sum equivalence or the two exceptional pairs. ■

Theorem (`theorem_7_2`). For retracts a and b, clauses (i)–(iv) of the isomorphism theorem hold.

Proof. Assemble the four preceding tools. ■

`theorem_7_2.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv

import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_4
import DomainTheory.DataTypesAsLattices.Retracts.typed_more
import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more
import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_3_more
import DomainTheory.DataTypesAsLattices.Retracts.retract_app
import DomainTheory.DataTypesAsLattices.Enumeration.PairSeq
import DomainTheory.DataTypesAsLattices.Retracts.graphLemmas

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace theorem_7_2

theorem i (a x y : Pomega) :
    (Ea a).rel x y ↔ typed x a ∧ typed y a ∧ x = y :=
  Iff.rfl

theorem i_iso {a : Pomega} (ha : Retract a) (x y : Pomega) :
    funOf a x = funOf a y ↔ (Ea a).rel (funOf a x) (funOf a y) := by
  constructor
```

```
  · intro h
    refine ⟨?_, ?_, h⟩
    · exact (retract_app ha x).symm
    · exact (retract_app ha y).symm
  · intro h
    exact h.2.2

theorem ii_of_typed {a b f : Pomega} (hf : typed f (arrowR a b)) :
    (arrowE (Ea a) (Ea b)).mem f := by
  have hf' : f = comp b (comp f a) := by
    simpa [typed, arrowR.app] using hf
  have hgraph : f = graph (fun x => funOf f x) := by
    apply Eq.trans hf'
    apply graph_ext
    intro x
    have : funOf f x = funOf b (funOf f (funOf a x)) := by
      have h1 : funOf f x = funOf (comp b (comp f a)) x :=
        congrArg (fun w => funOf w x) hf'
      rw [h1, comp.app, comp.app]
    rw [comp.app, this]
  refine ⟨hgraph, hgraph, ?_⟩
  intro x y hxy
  rcases hxy with ⟨hx, hy, rfl⟩
  have hfx : typed (funOf f x) b := theorem_4_3.ii hf hx
  exact ⟨hfx, hfx, rfl⟩

theorem ii {a b : Pomega} (ha : Retract a) (hb : Retract b)
    (f g : Pomega) :
    (typed f (arrowR a b) → (arrowE (Ea a) (Ea b)).mem f) ∧
      ((arrowE (Ea a) (Ea b)).rel f g →
        (Ea (arrowR a b)).rel (funOf (arrowR a b) f) (funOf (arrowR a b) g)) ∧
      ((Ea (arrowR a b)).rel f g → (arrowE (Ea a) (Ea b)).rel f g) := by
  refine ⟨fun hf => ii_of_typed hf, ?_, ?_⟩
  · intro hfg
    rcases hfg with ⟨hfgraph, hggraph, hpres⟩
    have hftyped : typed (funOf (arrowR a b) f) (arrowR a b) :=
      (retract_app (arrowR.isRetract ha hb) f).symm
    have hgtyped : typed (funOf (arrowR a b) g) (arrowR a b) :=
      (retract_app (arrowR.isRetract ha hb) g).symm
    refine ⟨hftyped, hgtyped, ?_⟩
    rw [arrowR.app, arrowR.app]
    apply graph_ext
    intro x
    simp only [comp.app]
    have hax : typed (funOf a x) a := (retract_app ha x).symm
    have hsame := hpres (funOf a x) (funOf a x) ⟨hax, hax, rfl⟩
    have hfeq : funOf f (funOf a x) = funOf g (funOf a x) := hsame.2.2
    have hfb : typed (funOf f (funOf a x)) b := hsame.1
    have hgb : typed (funOf g (funOf a x)) b := hsame.2.1
    rw [← hfb, ← hgb, hfeq]
  · intro hfg
    rcases hfg with ⟨hf, hg, rfl⟩
    have hmem := ii_of_typed hf
    exact ⟨hmem.1, hmem.1, hmem.2.2⟩

theorem iii (a b x y x' y' : Pomega) :
```

```
    (Ea (tensorR a b)).rel (pairSeq x y) (pairSeq x' y') ↔
      (prodE (Ea a) (Ea b)).rel (pairSeq x y) (pairSeq x' y') := by
  constructor
  · intro ⟨hu, hv, heq⟩
    obtain ⟨rfl, rfl⟩ := PairSeq.inj heq
    have hxy : typed x a ∧ typed y b := (typed.pairSeq_tensorR a b x y).mp hu
    refine ⟨?_, ?_, ?_, ?_⟩
    · rw [PairSeq.app_zero, PairSeq.app_one]; rfl
    · rw [PairSeq.app_zero, PairSeq.app_one]; rfl
    · rw [PairSeq.app_zero]
      exact ⟨hxy.1, hxy.1, rfl⟩
    · rw [PairSeq.app_one]
      exact ⟨hxy.2, hxy.2, rfl⟩
  · intro ⟨_hu, _hv, h0, h1⟩
    rw [PairSeq.app_zero, PairSeq.app_zero] at h0
    rw [PairSeq.app_one, PairSeq.app_one] at h1
    rcases h0 with ⟨hx, hx', heqx⟩
    rcases h1 with ⟨hy, hy', heqy⟩
    refine ⟨(typed.pairSeq_tensorR a b x y).mpr ⟨hx, hy⟩,
      (typed.pairSeq_tensorR a b x' y').mpr ⟨hx', hy'⟩, ?_⟩
    rw [heqx, heqy]

theorem iii_exact (a b u v : Pomega) :
    (Ea (tensorR a b)).rel u v ↔ (prodE (Ea a) (Ea b)).rel u v := by
  constructor
  · intro huv
    rcases huv with ⟨hu, hv, rfl⟩
    rcases (theorem_4_4.typed_iff.mp hu) with ⟨hpair, ha, hb⟩
    refine ⟨hpair, hpair, ?_, ?_⟩
    · exact ⟨ha, ha, rfl⟩
    · exact ⟨hb, hb, rfl⟩
  · intro huv
    rcases huv with ⟨hu, hv, h0, h1⟩
    rcases h0 with ⟨hx, hx', heqx⟩
    rcases h1 with ⟨hy, hy', heqy⟩
    have huvEq : u = v := by
      rw [hu, hv, heqx, heqy]
    refine ⟨?_, ?_, huvEq⟩
    · exact theorem_4_4.typed_iff.mpr ⟨hu, hx, hy⟩
    · exact theorem_4_4.typed_iff.mpr ⟨hv, hx', hy'⟩

theorem iv {a b : Pomega} (ha : Retract a) (hb : Retract b)
    (u v : Pomega) :
    (Ea (plusR a b)).rel u v ↔
      (sumE (Ea a) (Ea b)).rel u v ∨
        (u = botElem ∧ v = botElem) ∨ (u = topElem ∧ v = topElem) := by
  constructor
  · intro huv
    rcases huv with ⟨hu, hv, huv⟩
    subst v
    rcases (plusR.typed_iff ha hb).mp hu with
      rfl | rfl | ⟨x, rfl, hx⟩ | ⟨y, rfl, hy⟩
    · exact Or.inr (Or.inl ⟨rfl, rfl⟩)
    · exact Or.inr (Or.inr ⟨rfl, rfl⟩)
    · exact Or.inl (Or.inl ⟨by rw [pairElem.eq_pairSeq, PairSeq.app_one],
          by rw [pairElem.eq_pairSeq, PairSeq.app_one],
```

```
          by simpa [PairSeq.app_one] using (show (Ea a).rel x x from ⟨hx, hx, rfl⟩)⟩)
    · exact Or.inl (Or.inr ⟨by rw [pairElem.eq_pairSeq, PairSeq.app_one],
          by rw [pairElem.eq_pairSeq, PairSeq.app_one],
          by simpa [PairSeq.app_one] using (show (Ea b).rel y y from ⟨hy, hy, rfl⟩)⟩)
  · intro h
    rcases h with h | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
    rcases h with ⟨hu, hv, hA⟩ | ⟨hu, hv, hB⟩
    · rcases hA with ⟨hxt, hyt, heq⟩
      have heq' : u = v := by
        rw [hu, hv, heq]
      refine ⟨?_, ?_, heq'⟩
      · rw [hu]; exact plusR.typed_inl hxt
      · rw [hv]; exact plusR.typed_inl hyt
    · rcases hB with ⟨hxt, hyt, heq⟩
      have heq' : u = v := by
        rw [hu, hv, heq]
      refine ⟨?_, ?_, heq'⟩
      · rw [hu]; exact plusR.typed_inr hxt
      · rw [hv]; exact plusR.typed_inr hyt
    · exact ⟨plusR.typed_bot a b, plusR.typed_bot a b, rfl⟩
    · exact ⟨plusR.typed_top a b, plusR.typed_top a b, rfl⟩

end theorem_7_2

theorem theorem_7_2 {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    (∀ x y, funOf a x = funOf a y ↔ (Ea a).rel (funOf a x) (funOf a y)) ∧
      (∀ f g, (typed f (arrowR a b) → (arrowE (Ea a) (Ea b)).mem f) ∧
        ((arrowE (Ea a) (Ea b)).rel f g →
          (Ea (arrowR a b)).rel (funOf (arrowR a b) f)
            (funOf (arrowR a b) g)) ∧
        ((Ea (arrowR a b)).rel f g → (arrowE (Ea a) (Ea b)).rel f g)) ∧
      (∀ u v, (Ea (tensorR a b)).rel u v ↔ (prodE (Ea a) (Ea b)).rel u v) ∧
      (∀ u v, (Ea (plusR a b)).rel u v ↔
        (sumE (Ea a) (Ea b)).rel u v ∨
          (u = botElem ∧ v = botElem) ∨ (u = topElem ∧ v = topElem)) :=
  ⟨fun x y => theorem_7_2.i_iso ha x y,
    fun f g => theorem_7_2.ii ha hb f g,
    theorem_7_2.iii_exact a b,
    theorem_7_2.iv ha hb⟩

end DomainTheory.DataTypesAsLattices
```

3.8.6 Theorem 7.3 ✓

THEOREM 7.3 (The functionality theorem). *The combinators* **I**, **K**, *and* **S** *enjoy the following functionality properties which hold for all equivalence relations A, B, C:*

(i) $\mathbf{I} : A \to A$;

(ii) $\mathbf{K} : A \to (B \to A)$;

(iii) $\mathbf{S} : (A \to (B \to C)) \to ((A \to B) \to (A \to C))$.

Furthermore, these combinators are uniquely determined by these properties.

Let us check that S satisfies (iii). Suppose that:

$$f\,(A \to (B \to C))\,f'.$$

We must show that:

$$\mathbf{S}(f)\,((A \to B) \to (A \to C))\,\mathbf{S}(f').$$

To this end suppose that:

$$g\,(A \to B)\,g'.$$

We must show that:

$$\mathbf{S}(f)(g)\,(A \to C)\,\mathbf{S}(f')(g').$$

To this end suppose that:

$$x\,A\,x'.$$

We must show that:

$$\mathbf{S}(f)(g)(x)\,C\,\mathbf{S}(f')(g')(x').$$

Now by definition of the combinator $\mathbf{S}$ we have:

$$\mathbf{S}(f)(g)(x) = f(x)(g(x)),$$

$$\mathbf{S}(f')(g')(x') = f'(x')(g'(x')).$$

By assumptions on g, g' and on x, x', we know:

$$g(x)\,B\,g'(x').$$

By assumptions on f, f' and on x, x', we know:

$$f(x)\,(B \to C)\,f'(x').$$

The desired conclusion now follows when we note such combinations as $\mathbf{S}(f)$ and $\mathbf{S}(f)(g)$ are indeed functions. (We are using Theorem 7.1(i) several times in this case.)

In the case of the converse, let us suppose by way of example that $k \in \mathbf{P}\omega$ is such that

$$k : A \to (B \to A)$$

holds for all equivalence relations A and B. By specializing to, say, the identity relation we see that whatever $a \in \mathbf{P}\omega$ we take, both k and $k(a)$ are functions. To establish that $k = \mathbf{K}$ we need to show that the equation:

$$k(a)(b) = a$$

holds for all $a, b \in \mathbf{P}\omega$. This is easy to prove, for we have only to set:

$$A = \{\langle a, a\rangle\} \quad \text{and} \quad B = \{\langle b, b\rangle\},$$

and the equation follows at once. Not all proofs are quite so easy, however.

In the case of the combinator $\mathbf{S}$ it is not strictly true to say that Theorem 7.3(iii) determines it outright. The exact formulation is this: if $s \in \mathbf{P}\omega$ is such that

$$s(f) = s(\lambda x \lambda y.\, f(x)(y)) \quad \text{and} \quad s(f)(g) = s(f)(\lambda x.\, g(x))$$

for all $f, g \in \mathbf{P}\omega$; and if

$$s : (A \to (B \to C)) \to ((A \to B) \to (A \to C))$$

for all A, B, C, then $s = \mathbf{S}$. In other words, we need to know that s converts its first two arguments into functions with the right number of places before we can say that its explicit functionality identifies it as being the combinator $\mathbf{S}$.

In [104] they show that the functionality property:

$$(7.13) \quad \lambda f \lambda g.\, f \circ g : (B \to C) \to ((A \to B) \to (A \to C)),$$

follows from Theorem 7.1(ii) and Theorem 7.3(ii) and (iii) in view of the identity:

$$(7.14) \quad \lambda f \lambda g.\, f \circ g = \mathbf{S}(\mathbf{K}(\mathbf{S}))(\mathbf{K})$$

(see the proof of Theorem 2.4).

A more interesting result concerns the *iterators* defined as follows:

$$(7.15) \quad \mathbf{Z}_0 = \lambda f \lambda x.\, x,$$

$$(7.16) \quad \mathbf{Z}_{n+1} = \lambda f \lambda x.\, f(\mathbf{Z}_n(f)(x)),$$

In other words, $\mathbf{Z}_n(f)(x) = f^n(x)$. These natural combinators can be typed very easily, but Gordon Plotkin has shown that the obvious typing actually characterizes them.

Formalization. Functionality of $\mathbf{I}$, $\mathbf{K}$, and $\mathbf{S}$ for all restricted equivalences, together with uniqueness under those typings (with the η-hypotheses for $\mathbf{S}$).

Theorem (`theorem_7_3.i`). $\mathbf{I} : A \to A$ for every restricted equivalence A.

Proof. $\mathbf{I}$ is a graph and preserves relatedness by acting as the identity. ■

Theorem (`theorem_7_3.ii`). $\mathbf{K} : A \to (B \to A)$ for all restricted equivalences A and B.

Proof. $\mathbf{K}$ and its applications are graphs; relatedness of the first arguments is preserved by discarding the second. ■

Theorem (`theorem_7_3.iii`). $\mathbf{S} : (A \to (B \to C)) \to ((A \to B) \to (A \to C))$ for all restricted equivalences A, B, and C.

Proof. Nested graphhood of $\mathbf{S}$ and its curried applications; at related x, x' the applications $g(x)$ and $f(x)$ are related in B and $B \to C$, so $f(x)(g(x))$ is related in C. ■

Theorem (`theorem_7_3.I_unique`). If $i : A \to A$ for every restricted equivalence A, then $i = \mathbf{I}$.

Proof. Specialize to singletons: $i(a)$ is related to itself in the singleton on a, so $i(a) = a$. ■

Theorem (`theorem_7_3.K_unique`). If $k : A \to (B \to A)$ for all restricted equivalences A and B, then $k = \mathbf{K}$.

Proof. Specialize to singletons on a and b: $k(a)(b)$ lies in the singleton on a, so equals a. ■

Theorem (`theorem_7_3.S_unique`). If s is η-stable in its first two arguments as in Scott's proviso, and $s : (A \to (B \to C)) \to ((A \to B) \to (A \to C))$ for all A, B, C, then $s = \mathbf{S}$.

Proof. Reduce to η-normalized arguments, then evaluate at a singleton argument: the typing forces $s(f)(g)(x) = f(x)(g(x))$. ■

Theorem (`theorem_7_3`). The functionality and uniqueness clauses of Theorem 7.3 hold.

Proof. Assemble the six preceding tools. ■

`theorem_7_3.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv

import DomainTheory.DataTypesAsLattices.Functionality.theorem_7_1
import DomainTheory.DataTypesAsLattices.Lambda.eta2
import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more
import DomainTheory.DataTypesAsLattices.Graph.funOf_more
import DomainTheory.DataTypesAsLattices.Retracts.graphLemmas
import DomainTheory.DataTypesAsLattices.Graph.beta

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace theorem_7_3

theorem I_unique (i : Pomega)
    (hi : ∀ A, (arrowE A A).mem i) : i = Icomb := by
  have hgraph : i = graph (fun x => funOf i x) := (hi emptyE).1
  apply funOf.ext_graph hgraph Icomb.isGraph
  intro a
  have ha : (singletonE a).mem (funOf i a) :=
    (theorem_7_1 (singletonE a) (singletonE a)).2.1 (hi (singletonE a))
      ((singletonE.mem a a).mpr rfl)
  simpa [Icomb.app, singletonE.mem] using ha

theorem K_unique (k : Pomega)
    (hk : ∀ A B, (arrowE A (arrowE B A)).mem k) : k = Kcomb := by
  have hgraph : k = graph (fun x => funOf k x) := (hk emptyE emptyE).1
  apply funOf.ext_graph hgraph Kcomb.isGraph
  intro a
  have hkA : (arrowE (singletonE a) (arrowE emptyE (singletonE a))).mem k :=
    hk (singletonE a) emptyE
  have hka : (arrowE emptyE (singletonE a)).mem (funOf k a) :=
    (theorem_7_1 (singletonE a) (arrowE emptyE (singletonE a))).2.1 hkA
      ((singletonE.mem a a).mpr rfl)
  apply funOf.ext_graph hka.1 (Kcomb.app_isGraph a)
  intro b
  have hkAB : (arrowE (singletonE a) (arrowE (singletonE b) (singletonE a))).mem k :=
    hk (singletonE a) (singletonE b)
  have hkab : (arrowE (singletonE b) (singletonE a)).mem (funOf k a) :=
```

```
    (theorem_7_1 (singletonE a) (arrowE (singletonE b) (singletonE a))).2.1 hkAB
      ((singletonE.mem a a).mpr rfl)
  have hab : (singletonE a).mem (funOf (funOf k a) b) :=
    (theorem_7_1 (singletonE b) (singletonE a)).2.1 hkab
      ((singletonE.mem b b).mpr rfl)
  simpa [Kcomb.beta2, singletonE.mem] using hab

theorem S_unique (s : Pomega)
    (h?1 : ∀ f, funOf s f =
      funOf s (graph (fun x => graph (fun y => funOf (funOf f x) y))))
    (h?2 : ∀ f g, funOf (funOf s f) g =
      funOf (funOf s f) (graph (fun x => funOf g x)))
    (hs : ∀ A B C,
      (arrowE (arrowE A (arrowE B C))
        (arrowE (arrowE A B) (arrowE A C))).mem s) :
    s = Scomb := by
  have hsgraph : s = graph (fun f => funOf s f) := (hs emptyE emptyE emptyE).1
  apply funOf.ext_graph hsgraph Scomb.isGraph
  intro f
  let f' : Pomega := graph (fun x => graph (fun y => funOf (funOf f x) y))
  have hf'app (z : Pomega) : funOf f' z = graph (fun y => funOf (funOf f z) y) :=
    eta2.app f z
  have hf'graph : f' = graph (fun z => funOf f' z) := by
    apply graph_ext
    intro z
    exact (hf'app z).symm
  have hsf : funOf s f = funOf s f' := h?1 f
  have hsf'graph : funOf s f' = graph (fun g => funOf (funOf s f') g) := by
    have hf'mem : (arrowE emptyE (arrowE emptyE emptyE)).mem f' :=
      ⟨hf'graph, hf'graph, fun _ _ h => False.elim h⟩
    have := (theorem_7_1 (arrowE emptyE (arrowE emptyE emptyE))
      (arrowE (arrowE emptyE emptyE) (arrowE emptyE emptyE))).2.1
      (hs emptyE emptyE emptyE) hf'mem
    exact this.1
  apply funOf.ext_graph (by rw [hsf]; exact hsf'graph) (Scomb.app_isGraph f)
  intro g
  let g' : Pomega := graph (funOf g)
  have hg'app (z : Pomega) : funOf g' z = funOf g z :=
    beta (funOf.isScottContinuous g) z
  have hg'graph : g' = graph (fun z => funOf g' z) := by
    apply graph_ext
    intro z
    exact (hg'app z).symm
  have hsg : funOf (funOf s f) g = funOf (funOf s f) g' := h?2 f g
  have hsg' : funOf (funOf s f) g' = funOf (funOf s f') g' := by
    rw [hsf]
  have hsg'graph :
      funOf (funOf s f') g' =
        graph (fun z => funOf (funOf (funOf s f') g') z) := by
    have hg'mem : (arrowE emptyE emptyE).mem g' :=
      ⟨hg'graph, hg'graph, fun _ _ h => False.elim h⟩
    have hf'mem : (arrowE emptyE (arrowE emptyE emptyE)).mem f' :=
      ⟨hf'graph, hf'graph, fun _ _ h => False.elim h⟩
    have hsf'mem :
        (arrowE (arrowE emptyE emptyE) (arrowE emptyE emptyE)).mem (funOf s f') :=
      (theorem_7_1 (arrowE emptyE (arrowE emptyE emptyE))
```

```
      (arrowE (arrowE emptyE emptyE) (arrowE emptyE emptyE))).2.1
      (hs emptyE emptyE emptyE) hf'mem
  exact ((theorem_7_1 (arrowE emptyE emptyE)
    (arrowE emptyE emptyE)).2.1 hsf'mem hg'mem).1
apply funOf.ext_graph
  (by rw [hsg, hsg']; exact hsg'graph) (Scomb.app2_isGraph f g)
intro arg
let A := singletonE arg
let B := singletonE (funOf g arg)
let C := singletonE (funOf (funOf f arg) (funOf g arg))
have hf'memABC : (arrowE A (arrowE B C)).mem f' := by
  refine ⟨hf'graph, hf'graph, ?_⟩
  intro x₁ x₂ hx₁₂
  have hx₁ : x₁ = arg := hx₁₂.1
  have hx₂ : x₂ = arg := hx₁₂.2
  have hfxgraph (z : Pomega) :
      funOf f' z = graph (fun y => funOf (funOf f' z) y) := by
    rw [hf'app]
    apply graph_ext
    intro y
    exact (beta (funOf.isScottContinuous (funOf f z)) y).symm
  rw [hx₁, hx₂]
  refine ⟨hfxgraph arg, hfxgraph arg, ?_⟩
  intro y₁ y₂ hy
  have hy₁ : y₁ = funOf g arg := hy.1
  have hy₂ : y₂ = funOf g arg := hy.2
  rw [hy₁, hy₂, hf'app]
  have happ :
      funOf (graph (fun y => funOf (funOf f arg) y)) (funOf g arg) =
        funOf (funOf f arg) (funOf g arg) :=
    beta (funOf.isScottContinuous (funOf f arg)) (funOf g arg)
  exact happ.symm ▶ ⟨rfl, rfl⟩
have hg'memAB : (arrowE A B).mem g' := by
  refine ⟨hg'graph, hg'graph, ?_⟩
  intro x₁ x₂ hx₁₂
  have hx₁ : x₁ = arg := hx₁₂.1
  have hx₂ : x₂ = arg := hx₁₂.2
  rw [hx₁, hx₂, hg'app]
  exact ⟨rfl, rfl⟩
have hsf'mem :
    (arrowE (arrowE A B) (arrowE A C)).mem (funOf s f') :=
  (theorem_7_1 (arrowE A (arrowE B C))
    (arrowE (arrowE A B) (arrowE A C))).2.1 (hs A B C) hf'memABC
have hsg'mem : (arrowE A C).mem (funOf (funOf s f') g') :=
  (theorem_7_1 (arrowE A B) (arrowE A C)).2.1 hsf'mem hg'memAB
have hxA : A.mem arg := (singletonE.mem arg arg).mpr rfl
have hval : C.mem (funOf (funOf (funOf s f') g') arg) :=
  (theorem_7_1 A C).2.1 hsg'mem hxA
have hval' : funOf (funOf (funOf s f') g') arg =
    funOf (funOf f arg) (funOf g arg) :=
  (singletonE.mem _ _).mp hval
have hgoal : funOf (funOf (funOf s f) g) arg =
    funOf (funOf (funOf s f') g') arg := by
  rw [hsg, hsg']
rw [hgoal, hval', Scomb.beta3]
```

```
theorem i (A : RestrictedEquiv) :
    (arrowE A A).mem Icomb :=
  ⟨Icomb.isGraph, Icomb.isGraph, fun x y hxy => by simpa [Icomb.app] using hxy⟩

theorem ii (A B : RestrictedEquiv) :
    (arrowE A (arrowE B A)).mem Kcomb := by
  refine ⟨Kcomb.isGraph, Kcomb.isGraph, ?_⟩
  intro x x' hx
  refine ⟨Kcomb.app_isGraph x, Kcomb.app_isGraph x', ?_⟩
  intro y y' _hy
  simpa [Kcomb.beta2] using hx

theorem iii (A B C : RestrictedEquiv) :
    (arrowE (arrowE A (arrowE B C))
      (arrowE (arrowE A B) (arrowE A C))).mem Scomb := by
  refine ⟨Scomb.isGraph, Scomb.isGraph, ?_⟩
  intro f f' hf
  refine ⟨Scomb.app_isGraph f, Scomb.app_isGraph f', ?_⟩
  intro g g' hg
  refine ⟨Scomb.app2_isGraph f g, Scomb.app2_isGraph f' g', ?_⟩
  intro x x' hx
  have hgx : (arrowE A B).rel g g' := hg
  have hfx : (arrowE A (arrowE B C)).rel f f' := hf
  have hB : B.rel (funOf g x) (funOf g' x') := hgx.2.2 x x' hx
  have hBC : (arrowE B C).rel (funOf f x) (funOf f' x') := hfx.2.2 x x' hx
  simpa [Scomb.beta3] using hBC.2.2 (funOf g x) (funOf g' x') hB

end theorem_7_3

theorem theorem_7_3 :
    (∀ A, (arrowE A A).mem Icomb) ∧
      (∀ A B, (arrowE A (arrowE B A)).mem Kcomb) ∧
      (∀ A B C,
        (arrowE (arrowE A (arrowE B C))
          (arrowE (arrowE A B) (arrowE A C))).mem Scomb) ∧
      (∀ i, (∀ A, (arrowE A A).mem i) → i = Icomb) ∧
      (∀ k, (∀ A B, (arrowE A (arrowE B A)).mem k) → k = Kcomb) ∧
      (∀ s,
        (∀ f, funOf s f =
          funOf s (graph (fun x => graph (fun y => funOf (funOf f x) y)))) →
        (∀ f g, funOf (funOf s f) g =
          funOf (funOf s f) (graph (fun x => funOf g x))) →
        (∀ A B C,
          (arrowE (arrowE A (arrowE B C))
            (arrowE (arrowE A B) (arrowE A C))).mem s) →
        s = Scomb) :=
  ⟨theorem_7_3.i, theorem_7_3.ii, theorem_7_3.iii,
    theorem_7_3.I_unique, theorem_7_3.K_unique, theorem_7_3.S_unique⟩

end DomainTheory.DataTypesAsLattices
```

3.8.7 Theorem 7.4 ✓

THEOREM 7.4 (The iterator theorem). *The combinators $\mathbf{Z}_n$ enjoy the following functionality property which holds for all equivalence relations A:*

(i) $\mathbf{Z}_n : (A \to A) \to (A \to A)$.

Further, if any element $z \in \mathbf{P}\omega$ satisfies (i) for all A, then it must be one of the iterators, provided that $z(f) = z(\lambda x.\, f(x))$ holds for all $f \in \mathbf{P}\omega$.

That each of the $\mathbf{Z}_n$ satisfies Theorem 7.4(i) is obvious. Suppose z were another such element. Then clearly:

$$z = \lambda f \lambda x.\, z(f)(x).$$

Suppose f and x are fixed for the moment. Let:

$$A = \{(f^n(x), f^n(x)) \mid n \in \omega\},$$

where we can suppose in addition that:

$$f = \lambda x.\, f(x).$$

Then $f : A \to A$ is clear, and so $z(f) : A \to A$ also. But $x : A$, therefore $z(f)(x) = f^n(x)$, for some $n \in \omega$, because $z(f)(x) : A$. The trouble with this easy part of the argument is that the integer n depends on f and x. What we must show is that it is independent of f and x; then $z = \mathbf{Z}_n$ will follow.

Plotkin's method for this case is to introduce some independent successor functions:

$$\sigma_j = \lambda x.\, \{(j, k+1) \mid (j, k) \in x\}. \tag{7.17}$$

Note that:

$$\sigma_j^m((j', 0)) = \begin{cases} (j, m) & \text{if } j = j'; \\ \bot & \text{if } j \neq j'. \end{cases} \tag{7.18}$$

It then follows that:

$$(\sigma_j \cup \sigma_{j'})^m((j, 0) \cup (j', 0)) = (j, m) \cup (j', m). \tag{7.19}$$

Having these identities, we return to the argument. From what we saw before, given $j \in \omega$, there is an n_j such that:

$$z(\sigma_j)((j, 0)) = \sigma_j^{n_j}((j, 0)) = (j, n_j)$$

Take any two $j, j' \in \omega$. We also know there is an $n \in \omega$ where:

$$z(\sigma_j \cup \sigma_{j'})((j, 0) \cup (j', 0)) = (j, n) \cup (j', n).$$

in view of (7.19). But since $\sigma_j \subseteq \sigma_j \cup \sigma_{j'}$ and $\sigma_{j'} \subseteq \sigma_j \cup \sigma_{j'}$, we have:

$$(j, n_j) \cup (j', n_{j'}) \subseteq (j, n) \cup (j', n).$$

It follows that

$$n_j = n = n_{j'},$$

and so they are all equal. This determines the fixed $n \in \omega$ we want.

Suppose that both f and x are finite sets in $\mathbf{P}\omega$. Choose $j > \max(f \cup x)$. Let A this time be the least equivalence relation such that:

$$f^m(x)\, A\, f^m(x) \cup (j, m)$$

holds for all $m \in \omega$. We then check that:

$$\lambda x.\, f(x)\, (A \to A)\, (\lambda x.\, f(x)) \cup \sigma_j.$$

Therefore, we have:

$$z(\lambda x.\, f(x))\, (A \to A)\, z((\lambda x.\, f(x)) \cup \sigma_j),$$

and since $x\, A\, x \cup (j, 0)$, we get:

$$z(\lambda x.\, f(x))(x)\, A\, z((\lambda x.\, f(x)) \cup \sigma_j)(x \cup (j, 0)).$$

Now there is an integer $m \in \omega$ such that:

$$\begin{aligned} z((\lambda x.\, f(x)) \cup \sigma_j)(x \cup (j, 0)) &= f^m(x) \cup \sigma_j^m((j, 0)) \\ &= f^m(x) \cup (j, m), \end{aligned}$$

where we have been able to separate f and σ because j is so large. But the right-hand side must contain $z(\sigma_j)(j, 0) = (j, n)$. Thus m and our fixed n are the same. The other element

$$z(\lambda x.\, f(x))(x) = f^q(x)$$

for some $q \in \omega$. Thus we have:

$$f^q(x)\, A\, f^n(x) \cup (j, n).$$

Again since j is so large, $(j, n) \notin f^q(x)$. Thus by our choice of A we must have $f^q(x) = f^n(x)$. This means then, since n is fixed, that for *all finite* f, x:

$$z(\lambda x.\, f(x))(x) = f^n(x).$$

But then by continuity this equation holds for *all* f, x. It follows now that $z = \mathbf{Z}_n$, by the proviso of the theorem.

These results bring up many questions which we leave unanswered here. For example, which combinators (i.e., pure λ-terms) have functionality as in the examples above, and can we decide when a term is one such? In particular, can the diagonal combinator $\lambda x.\, x(x)$ be typed? (The argument of [104, p. 81] is purely formal and does not apparently apply to the model.) What about terms in LAMBDA beyond the pure λ-calculus?

Formalization. The iterator theorem packages Plotkin's tag-erasure argument: successor identities for σ_j, a restricted equivalence that pairs finite orbits with fresh tags, and uniqueness of any η-normalized z that is functional at every $(A \to A) \to (A \to A)$.

Definition (`eraseTag`). Erase tag j from u by discarding every pair whose first component is j.

Theorem (`eraseTag.Z_finite`). On a finite orbit $Z_m(e_p, e_q)$, erasing the fresh tag $\max(p, q) + 1$ leaves the orbit unchanged.

Proof. Finite orbits never carry that fresh tag. ■

Theorem (`eraseTag.Z_finite_union`). Erasing the fresh tag from an orbit union a fresh singleton recovers the plain orbit.

Proof. Membership either lies in the orbit (kept) or is the fresh pair (discarded). ■

Definition (`plotkinGood`). An element is Plotkin-good when it is a finite orbit of e_p on e_q, or that orbit union the fresh singleton at step m.

Definition (`plotkinRel`). The generating relation identifies an orbit with the same orbit union its fresh singleton at the same step, in either order.

Definition (`plotkinE`). The Plotkin equivalence relates good elements whose equivalence closure under that generating relation identifies them.

Theorem (`plotkinE.erase`). Related elements have equal erase-tag projections at the fresh tag.

Proof. Induction on the generated equivalence, using the two erase-tag orbit identities. ■

Theorem (`plotkinE.finite_separation`). Iterating the η-normalized finite graph of e_p united with σ_j on e_q union a fresh zero yields the finite orbit union the fresh singleton.

Proof. Induction on the iterate, separating e_p from σ_j by freshness of the tag. ■

Theorem (`plotkinE.function_related`). The η-normalized graph of e_p is related in $A \to A$ to that graph united with σ_j, for A the Plotkin equivalence at the fresh tag.

Proof. Both sides are graphs; on related arguments, erase-tag agreement and the four orbit/singleton cases produce related results. ■

Theorem (`eq_7_18_eq`). $\sigma_j^m(\{(j, 0)\}) = \{(j, m)\}$.

Proof. Induction, applying the successor-on-singleton law for σ_j. ■

Theorem (`eq_7_18_ne`). If $j \neq j'$, then $\sigma_j^{m+1}(\{(j', 0)\}) = \bot$.

Proof. Induction: the first step kills a wrong tag, and σ_j preserves $\bot$. ■

Theorem (`eq_7_19`). $(\sigma_j \cup \sigma_{j'})^m(\{(j, 0)\} \cup \{(j', 0)\}) = \{(j, m)\} \cup \{(j', m)\}$.

Proof. Equal tags reduce to the previous identity; distinct tags induct using the pairwise successor law. ■

Theorem (`theorem_7_4.i`). Each iterator $\mathbf{Z}_n$ is a member of $(A \to A) \to (A \to A)$ for every restricted equivalence A.

Proof. Iterators are graphs, and relatedness is preserved by the iterate law for Z. ■

Theorem (`theorem_7_4.plotkin`). If z is η-normalized and functional at every $(A \to A) \to (A \to A)$, then for any f and x there is an n with $z(f)(x) = f^n(x)$.

Proof. Specialize to the iterate equivalence of the η-normalized f at x; functionality forces the value into that orbit. ■

Theorem (`theorem_7_4.finite`). Under those hypotheses, if $z(\sigma_j)(\{(j, 0)\}) = \{(j, n)\}$ for every j, then on finite e_p and e_q one has $z(e_p)(e_q) = e_p^n(e_q)$.

Proof. Relate the η-normalized finite graph to its union with a fresh σ_j in the Plotkin equivalence; compare orbits by erase-tag, and force the iterate index to equal the fixed n by a fresh-tag witness. ■

Theorem (`theorem_7_4.sigma_index`). Under those hypotheses there is a single n such that $z(\sigma_j)(\{(j, 0)\}) = \{(j, n)\}$ for every j.

Proof. Compare the indices at 0, at j, and at their union via (7.18)–(7.19) and monotonicity of application. ■

Theorem (`theorem_7_4.unique`). Under those hypotheses, $z = \mathbf{Z}_n$ for that n.

Proof. On finite arguments the previous two tools give agreement with $\mathbf{Z}_n$; continuity and η-normalization extend to all arguments, then graph extensionality. ■

Theorem (`theorem_7_4`). Every $\mathbf{Z}_n$ is functional at all $(A \to A) \to (A \to A)$, and every η-normalized z with that functionality is some $\mathbf{Z}_n$.

Proof. Assemble the membership and uniqueness tools. ■

`theorem_7_4.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.SumMed
public import DomainTheory.DataTypesAsLattices.Enumeration.Zcomb
import DomainTheory.DataTypesAsLattices.Functionality.theorem_7_1
import DomainTheory.DataTypesAsLattices.Functionality.iterateE
import DomainTheory.DataTypesAsLattices.Lambda.graph_model
import DomainTheory.DataTypesAsLattices.Graph.mu_law
import DomainTheory.DataTypesAsLattices.Functionality.PlotkinHelpers
import DomainTheory.DataTypesAsLattices.Lambda.eq_2_9
import DomainTheory.DataTypesAsLattices.Graph.funOf_more
import DomainTheory.DataTypesAsLattices.Retracts.graphLemmas
import DomainTheory.DataTypesAsLattices.Graph.beta
import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def eraseTag (j : ℕ) (u : Pomega) : Pomega :=
  {p | p ∈ u ∧ ∀ k, p = pair j k}

namespace eraseTag

theorem Z_finite (p q m : ℕ) :
    eraseTag (max p q + 1) (Z m (e p) (e q)) = Z m (e p) (e q) := by
  ext k
  constructor
  · exact fun hk => hk.1
  · intro hk
    exact ⟨hk, fun r hkr => Z.finite_no_tag p q m r (hkr ▸ hk)⟩

theorem Z_finite_union (p q m : ℕ) :
    eraseTag (max p q + 1)
        (Z m (e p) (e q) ∪ {pair (max p q + 1) m}) =
      Z m (e p) (e q) := by
  ext k
  constructor
  · intro ⟨hk, hnot⟩
    rcases hk with hk | hk
    · exact hk
    · exact (hnot m (by simpa using hk)).elim
  · intro hk
    exact ⟨Or.inl hk, fun r hkr => Z.finite_no_tag p q m r (hkr ▸ hk)⟩

end eraseTag
```

```
def plotkinGood (p q j : ℕ) (u : Pomega) : Prop :=
  ∃ m, u = Z m (e p) (e q) ∨
    u = Z m (e p) (e q) ∪ {pair j m}

def plotkinRel (p q j : ℕ) (u v : Pomega) : Prop :=
  ∃ m, (u = Z m (e p) (e q) ∧
      v = Z m (e p) (e q) ∪ {pair j m}) ∨
    (v = Z m (e p) (e q) ∧
      u = Z m (e p) (e q) ∪ {pair j m})

def plotkinE (p q j : ℕ) : RestrictedEquiv where
  rel u v :=
    plotkinGood p q j u ∧ plotkinGood p q j v ∧
      Relation.EqvGen (plotkinRel p q j) u v
  symm := by
    intro u v ⟨hu, hv, huv⟩
    exact ⟨hv, hu, Relation.EqvGen.symm _ _ huv⟩
  trans := by
    intro u v w ⟨hu, _hv, huv⟩ ⟨_hv', hw, hvw⟩
    exact ⟨hu, hw, Relation.EqvGen.trans _ _ _ huv hvw⟩

namespace plotkinE

theorem erase {p q : ℕ} {u v : Pomega}
    (h : (plotkinE p q (max p q + 1)).rel u v) :
    eraseTag (max p q + 1) u = eraseTag (max p q + 1) v := by
  rcases h with ⟨_hu, _hv, h⟩
  clear _hu _hv
  induction h with
  | rel x y hxy =>
      rcases hxy with ⟨m, ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩⟩
      · rw [eraseTag.Z_finite, eraseTag.Z_finite_union]
      · rw [eraseTag.Z_finite, eraseTag.Z_finite_union]
  | refl => rfl
  | symm _ _ _ ih => exact ih.symm
  | trans _ _ _ _ _ ih₁ ih₂ => exact ih₁.trans ih₂

theorem finite_separation (p q m : ℕ) :
    Z m
        (graph (fun y => funOf (e p) y) ∪
          graph (sigmaJ (max p q + 1)))
        ((e q) ∪ {pair (max p q + 1) 0}) =
      Z m (e p) (e q) ∪ {pair (max p q + 1) m} := by
  induction m with
  | zero => rfl
  | succ m ih =>
      rw [Z, DomainTheory.DataTypesAsLattices.eq_2_9, ih]
      have hbeta :
          funOf (graph (fun y => funOf (e p) y))
              (Z m (e p) (e q) ∪ {pair (max p q + 1) m}) =
            funOf (e p) (Z m (e p) (e q) ∪ {pair (max p q + 1) m}) :=
        beta (funOf.isScottContinuous (e p)) _
      rw [hbeta, funOf.e_union_fresh p (max p q + 1) m _
        (Nat.lt_succ_of_le (Nat.le_max_left p q)), sigmaJ.app,
        sigmaJ.union_fresh (fun k => Z.finite_no_tag p q m k)]
      rfl
```

```
theorem function_related (p q : ℕ) :
    (arrowE
      (plotkinE p q (max p q + 1))
      (plotkinE p q (max p q + 1))).rel
        (graph (fun y => funOf (e p) y))
        (graph (fun y => funOf (e p) y) ∪
          graph (sigmaJ (max p q + 1))) := by
  have hgf : graph (fun y => funOf (e p) y) =
      graph (fun y => funOf (graph (fun y => funOf (e p) y)) y) := by
    apply graph_ext
    intro y
    exact (beta (funOf.isScottContinuous (e p)) y).symm
  have hgu :
      graph (fun y => funOf (e p) y) ∪ graph (sigmaJ (max p q + 1)) =
        graph (fun y => funOf
          (graph (fun y => funOf (e p) y) ∪
            graph (sigmaJ (max p q + 1))) y) := by
    let F := fun y => funOf (e p) y ∪ sigmaJ (max p q + 1) y
    have hrep :
        graph (fun y => funOf (e p) y) ∪ graph (sigmaJ (max p q + 1)) =
          graph F := DomainTheory.DataTypesAsLattices.eq_2_10 _ _
    rw [hrep]
    congr 1
    funext y
    dsimp only [F]
    rw [← hrep]
    rw [DomainTheory.DataTypesAsLattices.eq_2_9, PlotkinHelpers.normalized_finite_app,
     sigmaJ.app]
  refine ⟨hgf, hgu, ?_⟩
  intro u v huv
  have herase := plotkinE.erase huv
  rcases huv.1 with ⟨r, hur | hur⟩
  <;> rcases huv.2.1 with ⟨s, hvs | hvs⟩
  all_goals
    subst u
    subst v
  · have hrs : Z r (e p) (e q) = Z s (e p) (e q) := by
      simpa [eraseTag.Z_finite] using herase
    have hrs' : Z (r + 1) (e p) (e q) = Z (s + 1) (e p) (e q) := by
      simpa [Z] using congrArg (fun w => funOf (e p) w) hrs
    have hleft :
        funOf (graph (fun y => funOf (e p) y)) (Z r (e p) (e q)) =
          Z (r + 1) (e p) (e q) := by
      rw [PlotkinHelpers.normalized_finite_app]
      rfl
    have hright :
        funOf
            (graph (fun y => funOf (e p) y) ∪
              graph (sigmaJ (max p q + 1)))
            (Z s (e p) (e q)) = Z (s + 1) (e p) (e q) :=
      PlotkinHelpers.finite_sigma_plain_app p q s
    rw [hleft, hright, ← hrs']
    exact ⟨⟨r + 1, Or.inl rfl⟩, ⟨r + 1, Or.inl rfl⟩,
      Relation.EqvGen.refl _⟩
  · have hrs : Z r (e p) (e q) = Z s (e p) (e q) := by
```

```
    exact (eraseTag.Z_finite p q r).symm.trans
      (herase.trans (eraseTag.Z_finite_union p q s))
  have hrs' : Z (r + 1) (e p) (e q) = Z (s + 1) (e p) (e q) := by
    simpa [Z] using congrArg (fun w => funOf (e p) w) hrs
  have hleft :
      funOf (graph (fun y => funOf (e p) y)) (Z r (e p) (e q)) =
        Z (r + 1) (e p) (e q) := by
    rw [PlotkinHelpers.normalized_finite_app]
    rfl
  have hright :
      funOf
          (graph (fun y => funOf (e p) y) ∪
            graph (sigmaJ (max p q + 1)))
          (Z s (e p) (e q) ∪ {pair (max p q + 1) s}) =
        Z (s + 1) (e p) (e q) ∪
          {pair (max p q + 1) (s + 1)} :=
    PlotkinHelpers.finite_sigma_union_app p q s
  rw [hleft, hright, hrs']
  exact ⟨⟨s + 1, Or.inl rfl⟩, ⟨s + 1, Or.inr rfl⟩,
    Relation.EqvGen.rel _ _
      ⟨s + 1, Or.inl ⟨rfl, rfl⟩⟩⟩
· have hrs : Z r (e p) (e q) = Z s (e p) (e q) := by
    exact (eraseTag.Z_finite_union p q r).symm.trans
      (herase.trans (eraseTag.Z_finite p q s))
  have hrs' : Z (r + 1) (e p) (e q) = Z (s + 1) (e p) (e q) := by
    simpa [Z] using congrArg (fun w => funOf (e p) w) hrs
  have hleft :
      funOf (graph (fun y => funOf (e p) y))
          (Z r (e p) (e q) ∪ {pair (max p q + 1) r}) =
        Z (r + 1) (e p) (e q) :=
    PlotkinHelpers.normalized_finite_fresh_app p q r
  have hright :
      funOf
          (graph (fun y => funOf (e p) y) ∪
            graph (sigmaJ (max p q + 1)))
          (Z s (e p) (e q)) = Z (s + 1) (e p) (e q) :=
    PlotkinHelpers.finite_sigma_plain_app p q s
  rw [hleft, hright, ← hrs']
  exact ⟨⟨r + 1, Or.inl rfl⟩, ⟨r + 1, Or.inl rfl⟩,
    Relation.EqvGen.refl _⟩
· have hrs : Z r (e p) (e q) = Z s (e p) (e q) := by
    simpa using (eraseTag.Z_finite_union p q r).symm.trans
      (herase.trans (eraseTag.Z_finite_union p q s))
  have hrs' : Z (r + 1) (e p) (e q) = Z (s + 1) (e p) (e q) := by
    simpa [Z] using congrArg (fun w => funOf (e p) w) hrs
  have hleft :
      funOf (graph (fun y => funOf (e p) y))
          (Z r (e p) (e q) ∪ {pair (max p q + 1) r}) =
        Z (r + 1) (e p) (e q) :=
    PlotkinHelpers.normalized_finite_fresh_app p q r
  have hright :
      funOf
          (graph (fun y => funOf (e p) y) ∪
            graph (sigmaJ (max p q + 1)))
          (Z s (e p) (e q) ∪ {pair (max p q + 1) s}) =
        Z (s + 1) (e p) (e q) ∪
```

```
            {pair (max p q + 1) (s + 1)} :=
      PlotkinHelpers.finite_sigma_union_app p q s
    rw [hleft, hright, hrs']
    exact ⟨⟨s + 1, Or.inl rfl⟩, ⟨s + 1, Or.inr rfl⟩,
      Relation.EqvGen.rel _ _
        ⟨s + 1, Or.inl ⟨rfl, rfl⟩⟩⟩

end plotkinE

theorem eq_7_18_eq (j m : ℕ) :
    Z m (graph (sigmaJ j)) {pair j 0} = ({pair j m} : Pomega) := by
  induction m with
  | zero => rfl
  | succ m ih =>
    calc
      Z (m + 1) (graph (sigmaJ j)) {pair j 0}
        = funOf (graph (sigmaJ j)) (Z m (graph (sigmaJ j)) {pair j 0}) := rfl
      _ = sigmaJ j (Z m (graph (sigmaJ j)) {pair j 0}) := sigmaJ.app _ _
      _ = sigmaJ j {pair j m} := by rw [ih]
      _ = {pair j (m + 1)} := sigmaJ.succ_singleton j m

theorem eq_7_18_ne (j j' m : ℕ) (hne : j = j') :
    Z (m + 1) (graph (sigmaJ j)) {pair j' 0} = botElem := by
  induction m with
  | zero =>
    calc
      Z 1 (graph (sigmaJ j)) {pair j' 0}
        = funOf (graph (sigmaJ j)) {pair j' 0} := rfl
      _ = sigmaJ j {pair j' 0} := sigmaJ.app _ _
      _ = botElem := sigmaJ.succ_ne j j' 0 hne
  | succ m ih =>
    calc
      Z (m + 1 + 1) (graph (sigmaJ j)) {pair j' 0}
        = funOf (graph (sigmaJ j))
            (Z (m + 1) (graph (sigmaJ j)) {pair j' 0}) := rfl
      _ = sigmaJ j (Z (m + 1) (graph (sigmaJ j)) {pair j' 0}) :=
          sigmaJ.app _ _
      _ = sigmaJ j botElem := by rw [ih]
      _ = botElem := sigmaJ.bot j

theorem eq_7_19 (j j' m : ℕ) :
    Z m (graph (sigmaJ j) ∪ graph (sigmaJ j'))
        (({pair j 0} : Pomega) ∪ {pair j' 0}) =
      ({pair j m} : Pomega) ∪ {pair j' m} := by
  by_cases h : j = j'
  · subst j'
    simp only [Set.union_self]
    exact eq_7_18_eq j m
  · induction m with
    | zero => rfl
    | succ m ih =>
      rw [Z, eq_2_9, ih, sigmaJ.app, sigmaJ.app,
        sigmaJ.pair_union j j' m h]
      rw [show sigmaJ j' (({pair j m} : Pomega) ∪ {pair j' m}) =
          ({pair j' (m + 1)} : Pomega) by
        simpa [Set.union_comm] using sigmaJ.pair_union j' j m (Ne.symm h)]
```

```
namespace theorem_7_4

theorem i (A : RestrictedEquiv) (n : ℕ) :
    (arrowE (arrowE A A) (arrowE A A)).mem (Zcomb n) := by
  refine ⟨Zcomb.isGraph n, Zcomb.isGraph n, ?_⟩
  intro f f' hf
  refine ⟨Zcomb.app_isGraph n f, Zcomb.app_isGraph n f', ?_⟩
  intro x y hxy
  simpa [Zcomb.app2] using Z.rel A n hf.2.2 hxy

theorem plotkin (z : Pomega)
    (h? : ∀ f, funOf z f = funOf z (graph (fun y => funOf f y)))
    (hz : ∀ A, (arrowE (arrowE A A) (arrowE A A)).mem z)
    (f x : Pomega) :
    ∃ n, funOf (funOf z f) x = Z n f x := by
  let f' : Pomega := graph (fun y => funOf f y)
  have hf'graph : f' = graph (fun y => funOf f' y) := by
    apply graph_ext
    intro y
    exact (beta (funOf.isScottContinuous f) y).symm
  let A := iterateE f' x
  have hf'mem : (arrowE A A).mem f' := by
    refine ⟨hf'graph, hf'graph, ?_⟩
    intro u v huv
    rcases huv with ⟨⟨n, rfl⟩, rfl⟩
    refine ⟨⟨n + 1, rfl⟩, rfl⟩
  have hzA : (arrowE (arrowE A A) (arrowE A A)).mem z := hz A
  have hzf : (arrowE A A).mem (funOf z f') :=
    (theorem_7_1 (arrowE A A) (arrowE A A)).2.1 hzA hf'mem
  have hx : A.mem x := ⟨⟨0, by simp [Z]⟩, rfl⟩
  have hval : A.mem (funOf (funOf z f') x) :=
    (theorem_7_1 A A).2.1 hzf hx
  obtain ⟨⟨n, hn⟩, _⟩ := hval
  refine ⟨n, ?_⟩
  have h?f : funOf z f = funOf z f' := h? f
  rw [h?f, hn, Z.graph_funOf]

theorem finite (z : Pomega)
    (h? : ∀ f, funOf z f = funOf z (graph (fun y => funOf f y)))
    (hz : ∀ A, (arrowE (arrowE A A) (arrowE A A)).mem z)
    {n : ℕ}
    (hn : ∀ j, funOf (funOf z (graph (sigmaJ j))) {pair j 0} =
      ({pair j n} : Pomega))
    (p q : ℕ) :
    funOf (funOf z (e p)) (e q) = Z n (e p) (e q) := by
  let j := max p q + 1
  let gf := graph (fun y => funOf (e p) y)
  let gu := gf ∪ graph (sigmaJ j)
  have hfg : (arrowE
      (plotkinE p q j) (plotkinE p q j)).rel gf gu := by
    simpa [j, gf, gu] using plotkinE.function_related p q
  have hzfg :
      (arrowE (plotkinE p q j) (plotkinE p q j)).rel
        (funOf z gf) (funOf z gu) :=
    (hz (plotkinE p q j)).2.2 gf gu hfg
```

```
  have hstart :
      (plotkinE p q j).rel (e q) (e q ∪ {pair j 0}) := by
    refine ⟨⟨0, Or.inl rfl⟩, ⟨0, Or.inr rfl⟩, ?_⟩
    exact Relation.EqvGen.rel _ _
      ⟨0, Or.inl ⟨rfl, rfl⟩⟩
  have hout :
      (plotkinE p q j).rel
        (funOf (funOf z gf) (e q))
        (funOf (funOf z gu) (e q ∪ {pair j 0})) :=
    hzfg.2.2 _ _ hstart
  obtain ⟨r, hr⟩ := theorem_7_4.plotkin z h? hz gf (e q)
  obtain ⟨m, hm⟩ :=
    theorem_7_4.plotkin z h? hz gu (e q ∪ {pair j 0})
  have hleft :
      funOf (funOf z gf) (e q) = Z r (e p) (e q) := by
    rw [hr]
    exact Z.graph_funOf r (e p) (e q)
  have hright :
      funOf (funOf z gu) (e q ∪ {pair j 0}) =
        Z m (e p) (e q) ∪ {pair j m} := by
    rw [hm]
    simpa [j, gf, gu] using plotkinE.finite_separation p q m
  have herase := plotkinE.erase hout
  have horbit : Z r (e p) (e q) = Z m (e p) (e q) := by
    rw [hleft, hright] at herase
    exact (eraseTag.Z_finite p q r).symm.trans
      (herase.trans (eraseTag.Z_finite_union p q m))
  have hsigmaSub :
      funOf (funOf z (graph (sigmaJ j))) {pair j 0} ⊆
        funOf (funOf z gu) (e q ∪ {pair j 0}) := by
    exact mu_law
      (funOf.monotone_right z Set.subset_union_right)
      Set.subset_union_right
  have htag : pair j n ∈ Z m (e p) (e q) ∪ {pair j m} := by
    rw [← hright]
    apply hsigmaSub
    rw [hn j]
    simp
  have hnm : n = m := by
    rcases htag with hbad | htag
    · exact (Z.finite_no_tag p q m n (by simpa [j] using hbad)).elim
    · exact (pair_inj htag).2
  rw [h? (e p), hleft, horbit, ← hnm]

theorem sigma_index (z : Pomega)
    (h? : ∀ f, funOf z f = funOf z (graph (fun y => funOf f y)))
    (hz : ∀ A, (arrowE (arrowE A A) (arrowE A A)).mem z) :
    ∃ n, ∀ j,
      funOf (funOf z (graph (sigmaJ j))) {pair j 0} =
        ({pair j n} : Pomega) := by
  obtain ⟨n, hn⟩ :=
    theorem_7_4.plotkin z h? hz (graph (sigmaJ 0)) {pair 0 0}
  refine ⟨n, ?_⟩
  intro j
  have hn' :
      funOf (funOf z (graph (sigmaJ 0))) {pair 0 0} =
```

```
      ({pair 0 n} : Pomega) := by
  rw [hn, eq_7_18_eq]
by_cases hj : j = 0
· subst j
  exact hn'
obtain ⟨nj, hnj⟩ :=
  theorem_7_4.plotkin z h? hz (graph (sigmaJ j)) {pair j 0}
have hnj' :
    funOf (funOf z (graph (sigmaJ j))) {pair j 0} =
      ({pair j nj} : Pomega) := by
  rw [hnj, eq_7_18_eq]
obtain ⟨k, hk⟩ := theorem_7_4.plotkin z h? hz
  (graph (sigmaJ 0) ∪ graph (sigmaJ j))
  (({pair 0 0} : Pomega) ∪ {pair j 0})
have hk' :
    funOf
        (funOf z (graph (sigmaJ 0) ∪ graph (sigmaJ j)))
        (({pair 0 0} : Pomega) ∪ {pair j 0}) =


      ({pair 0 k} : Pomega) ∪ {pair j k} := by
  rw [hk, eq_7_19]
have hmono0 :
    funOf (funOf z (graph (sigmaJ 0))) {pair 0 0} ⊆
      funOf
        (funOf z (graph (sigmaJ 0) ∪ graph (sigmaJ j)))
        (({pair 0 0} : Pomega) ∪ {pair j 0}) :=
  mu_law
    (funOf.monotone_right z Set.subset_union_left)
    Set.subset_union_left
have hmonoj :
    funOf (funOf z (graph (sigmaJ j))) {pair j 0} ⊆
      funOf
        (funOf z (graph (sigmaJ 0) ∪ graph (sigmaJ j)))
        (({pair 0 0} : Pomega) ∪ {pair j 0}) :=
  mu_law
    (funOf.monotone_right z Set.subset_union_right)
    Set.subset_union_right
have h0mem : pair 0 n ∈ ({pair 0 k} : Pomega) ∪ {pair j k} := by
  rw [← hk']
  apply hmono0
  rw [hn']
  simp
have hjmem : pair j nj ∈ ({pair 0 k} : Pomega) ∪ {pair j k} := by
  rw [← hk']
  apply hmonoj
  rw [hnj']
  simp
have hnk : n = k := by
  rcases h0mem with h0 | h0
  · exact (pair_inj h0).2
  · exact (hj ((pair_inj h0).1).symm).elim
have hnjk : nj = k := by
  rcases hjmem with hj0 | hjj
  · exact (hj (pair_inj hj0).1).elim
  · exact (pair_inj hjj).2
rw [hnj', hnjk, ← hnk]
```

```
theorem unique (z : Pomega)
    (h? : ∀ f, funOf z f = funOf z (graph (fun y => funOf f y)))
    (hz : ∀ A, (arrowE (arrowE A A) (arrowE A A)).mem z) :
    ∃ n, z = Zcomb n := by
  obtain ⟨n, hn⟩ := theorem_7_4.sigma_index z h? hz
  refine ⟨n, ?_⟩
  have hfinite (p : ℕ) :
      funOf z (e p) = funOf (Zcomb n) (e p) := by
    let gp := graph (fun y => funOf (e p) y)
    have hgpGraph : gp = graph (fun y => funOf gp y) := by
      apply graph_ext
      intro y
      exact (PlotkinHelpers.normalized_finite_app p y).symm
    have hgpMem : (arrowE emptyE emptyE).mem gp :=
      ⟨hgpGraph, hgpGraph, fun _ _ h => False.elim h⟩
    have hzgpGraph : funOf z gp = graph (fun y => funOf (funOf z gp) y) :=
      ((theorem_7_1 (arrowE emptyE emptyE) (arrowE emptyE emptyE)).2.1
        (hz emptyE) hgpMem).1
    have hzFiniteGraph :
        funOf z (e p) = graph (fun y => funOf (funOf z (e p)) y) := by
      rw [h? (e p)]
      exact hzgpGraph
    apply funOf.ext_graph hzFiniteGraph (Zcomb.app_isGraph n (e p))
    intro x
    have hright :
        IsScottContinuous (fun y => Z n (e p) y) :=
      Z.isScottContinuous_right n (e p)
    have hleft :
        IsScottContinuous (fun y => funOf (funOf z (e p)) y) :=
      funOf.isScottContinuous (funOf z (e p))
    have heq := congrFun
      (IsScottContinuous.determined hleft hright
        (fun q => theorem_7_4.finite z h? hz hn p q)) x
    simpa [Zcomb.app2] using heq
  have hall (f : Pomega) : funOf z f = funOf (Zcomb n) f := by
    let gf := graph (fun y => funOf f y)
    have hgfGraph : gf = graph (fun y => funOf gf y) := by
      apply graph_ext
      intro y
      exact (beta (funOf.isScottContinuous f) y).symm
    have hgfMem : (arrowE emptyE emptyE).mem gf :=
      ⟨hgfGraph, hgfGraph, fun _ _ h => False.elim h⟩
    have hzgfGraph : funOf z gf = graph (fun y => funOf (funOf z gf) y) :=
      ((theorem_7_1 (arrowE emptyE emptyE) (arrowE emptyE emptyE)).2.1
        (hz emptyE) hgfMem).1
    have hzfGraph :
        funOf z f = graph (fun y => funOf (funOf z f) y) := by
      rw [h? f]
      exact hzgfGraph
    apply funOf.ext_graph hzfGraph (Zcomb.app_isGraph n f)
    intro x
    have hleft :
        IsScottContinuous (fun g => funOf (funOf z g) x) :=
      theorem_1_3 (f := fun u => funOf u x) (g := funOf z)
        (funOf.isScottContinuous_left x)
```

```
        (funOf.isScottContinuous z)
    have hright : IsScottContinuous (fun g => Z n g x) :=
      Z.isScottContinuous_left n x
    have heq := congrFun
      (IsScottContinuous.determined hleft hright (fun p => by
        have := congrArg (fun w => funOf w x) (hfinite p)
        simpa [Zcomb.app2] using this)) f
    simpa [Zcomb.app2] using heq
  exact funOf.ext_graph (hz emptyE).1 (Zcomb.isGraph n) hall

end theorem_7_4

theorem theorem_7_4 :
    (∀ A n, (arrowE (arrowE A A) (arrowE A A)).mem (Zcomb n)) ∧
      ∀ z,
        (∀ f, funOf z f = funOf z (graph (fun y => funOf f y))) →
        (∀ A, (arrowE (arrowE A A) (arrowE A A)).mem z) →
        ∃ n, z = Zcomb n :=
  ⟨theorem_7_4.i, theorem_7_4.unique⟩

end DomainTheory.DataTypesAsLattices
```

3.9 Acknowledgements and references

Appendix B. Acknowledgments and references. My greatest overall debt is to the late Christopher Strachey, who provided not only the initial stimulus and continuing encouragement, but also what may be termed the necessary irritation. Not being a trained mathematician, he often assumed that various operations made sense without looking too closely or rigorously at the details. This was particularly the case with the λ-calculus, which he used as freely as everyday algebra. Having repeatedly and outspokenly condemned the λ-calculus as a formal system without clear mathematical foundations, it was up to me to provide some alternative. The first suggestion was a typed system put forward in [223] (unpublished, but later developed as LCF by Robin Milner and his collaborators). Experience with the type structure of function spaces, which had come to my attention from work in recursion theory by Nerode, Platek and others, soon convinced me that there were many more similar structures than might at first be imagined. In particular, a vague idea about a space with a "dense" basis led quickly to the more direct construction, by inverse limits, of function spaces of "infinite" type that were very reasonable models of the classical "type-free" λ-calculus (as well as many other calculi for other "reflexive domains"). The details can be found in [225] and [230]. Algebra was justified, but the work in doing so was tiring and the exact connections with computability were not all that easy to describe.

In the meantime [194] contained suggestions for a "set-theoretical" construction of models, but not much notice was taken of the plan at the time it was circulated—perhaps owing to a fairly sketchy presentation of the precise semantics of the λ-calculus. The present paper evolved particularly from the project of making the connections with ordinary recursion theory easier to comprehend, since a satisfactory theory of computability and programming language semantics had to face this problem. The idea of using sets of integers for a model was first put forward by the author at a meeting at Oberwolfach at Easter in 1973 and in a more definitive form at the Third Scandinavian Logic Symposium shortly thereafter (see [231] which is a preliminary and shorter version of this paper). The author gave a report on the model at the Bristol Logic Colloquium in July 1973, but did not submit a paper for the proceedings. A series of several lectures was presented at the Kiel Logic Colloquium in July 1974, covering most of the present paper which was

distributed as a preprint at the meeting. The text (but unfortunately neither acknowledgments nor references) was printed in the proceedings (Springer Lecture Notes in Mathematics, vol. 499). In 1973 after experimentation with definitions somehow forced him into the definition of the model, the author realized that it was essentially the same as Plotkin's idea and, even more surprising, it was already implicit in a very precise form in much earlier work by [182] and [87] (see also [212]) on *enumeration operators*. What had happened was that Plotkin had not made enough tie-up with recursion theory, and the recursive people had not seen the tie-up with λ-calculus, even though they knew that one could do a lot with such operators. Actually, if the author had taken his own advice in 1971 (see *Continuous lattices*, [1, end of § 2]), he would have seen that many spaces have their own continuous-function spaces as computable retracts, a fact which is just exactly the basis for the present construction; but instead he said: "it [the representation as a retract] does not seem to be of too much help in proving theorems."

Over the years in work on λ-calculus and programming language semantics, personal contact and correspondence with a large number of people has been very stimulating and helpful. I must mention particularly de Bakker, Barendregt, Bekic, Blikle, Böhm, Curry, Egli, Engeler, Ershov, Goodman, Hyland, Kreisel, Landin, Milne, Milner, Mosses, Nivat, Park, Plotkin, Reynolds, de Roever, Smyth, Stoy, Tang, Tennent, Wadsworth. (I apologize to those I have inadvertently left out of this list.) In the reference list a very imperfect attempt has been made to collect references directly relevant to the topics of this paper as well as pointers to related areas that may be of inspiration for future work. The list of papers is undoubtedly incomplete, and inaccurate as well, but the author hopes it may be of some use for those seeking orientation. It is a very vexing problem to keep such references up to date. Some remarks toward references and acknowledgments on the specific results in the various sections follow. Felipe Bracho deserves special thanks for help in the preparation of the final manuscript and with the reference list.

Section 1. The relevance of the "positive" or "weak" topology first came to the author's attention through the work of [184]. Continuous functionals were studied by Kleene and Platek and many other researchers in recursion theory. Monotonicity was particularly stressed by [193]. The graphs and the definition of application are used in the same way by [194] and [212, see p. 147]. The fixed-point theorem is very well-known. See, e.g., [248]. The extension theorem was formulated by the author, but it is very similar to many results in point-set topology; it was used in a prominent way in [1] to characterize continuous lattices. The embedding theorem is well-known; see, e.g., [46].

Section 2. The language LAMBDA is due to the author. Note in particular that Plotkin and Rogers do not define λ-abstraction, even though they know of the existence of many combinators and *could have* defined abstraction if anyone had ever asked them. In particular, they understood about conversion in many instances. The reduction and combinator theorems are well known from combinatory logic and can be found in any reference. The first recursion theorem is basic to all of elementary recursion theory; what is new here is the adaptation of David Park's proof ([189], unpublished) to the present model to show that Curry's "paradoxical" combinator actually does the recursion. The definition of computability and the definability theorem tie up the present theory with the older theory of enumeration operators.

Section 3. The idea of reduction to a few combinators is an old one in combinatory logic; the author only needed to find a small trick (formula (3.1)) which would take care of the arithmetical combinators. The ideas for the Gödel numbering and the proof of the second recursion theorem are standard, as is the proof of the incompleteness theorem. It only looks a little different since we combine arithmetic with the "type-free" combinators. The proof of the completeness theorem for definability (Theorem 3.5) is taken directly from [182]. The author is indebted to Hyland for pointing this out. The subalgebra theorem is an easy reformulation of talk about enumeration degrees: for more information on such degrees consult [212], [222], and also [103]. The area is underdeveloped as compared to Turing degrees. Semigroups of

combinators have been discussed by Church and Böhm.

Section 4. The notion of a *retract* is common in topology, but the idea of using them to define data types and of having a calculus of computable retracts is original with the author. Of course the connection between lattices and fixed points was known; more about lattices is to be found in [1]. The various operations on retracts and the idea of using fixed-point equations to define retracts recursively are due to the author. Applications to semantics were given in [226] for flow diagrams, and this has been followed up by many people, in particular [96] and [207].

Section 5. Algebraic lattices have been known for a long time (see [101]) and also closure operations (see, e.g., [247]). It was Per Martin-Löf and Peter Hancock who suggested that they might form a "universe"; in particular the construction of $\mathbf{V}$ is essentially due to them. The limit theorem is due to the author.

Section 6. More information on the classification by notions in descriptive set theory of various subsets can be found in the work of Tang who also makes

Section 6. More information on the classification by notions in descriptive set theory of various subsets can be found in the work of Tang who also makes connections with the work of Wadge. The various normal forms for the classes of sets (e.g., Table 3) are due to the author.

Section 7. Functionality has been studied for some time in combinatory logic (see, e.g., [104] for an introduction). The author had the idea to see what it all means in the models; there are, of course, connections going back to Curry and Kleene, with functional interpretations of intuitionistic logic (cf. Theorem 7.3, which is well-known). The proof of Theorem 7.4 is due to Plotkin.

Retracts (Theorem 4.1). After the main body of the paper was written, Y. L. Ershov solved the author's problem about the space of retracts. Ershov's proof is presented after the discussion of the proof of Theorem 4.1 in the Retracts section. Quite independently [114] found almost exactly the same proof. Before corresponding with Ershov, the author was totally unaware of the connections with and the importance of Ershov's extensive work in "numeration" theory (see citations in the reference list).

All defects are due to the author.

4 Neighborhood Systems (1980) ✓

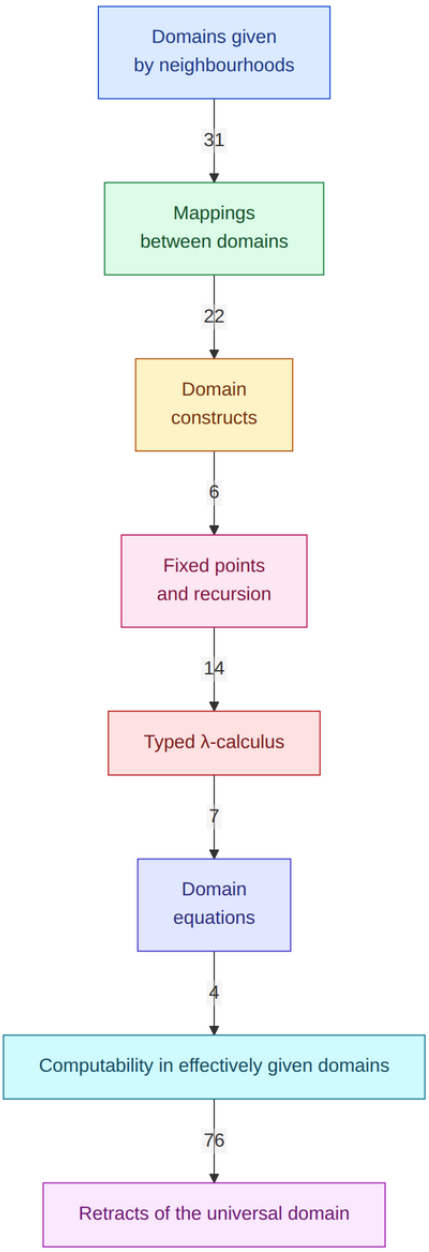


Figure 38: Neighborhood Systems (1980) ✓.

4.1 Introduction

These notes were written in conjunction with the lectures delivered by me for the Semantics of Programming Languages sequence during Michaelmas Term 1980 at Oxford. I started writing around the first week of

October and finished at the end of November. The purpose of the course was to provide the foundations needed for the method of denotational semantics; in particular, I wanted to make the connections with recursive function theory more definite and to show explicit, effectively given solutions to domain equations. Roughly, these chapters cover the first half of the book of J.E. Stoy. I plan soon to expand the notes into a book by adding additional chapters on other theoretical topics that time did not permit me to cover in one eight-week term.

When I started writing Lecture I in October, I did not know what the later lectures would contain: I could see no further ahead than part of Lecture III in the beginning. The lectures had to be typed in advance of the class meetings, however, so there was at the time of composition no opportunity for second thoughts of any major proportions: I had to write the text straight through. As a consequence there are many remarks I would like to transpose and many additional points of explanation I see are needed; further worked examples and easier exercises are also required. During the spring, after receiving many helpful comments, I was able to introduce a few changes in the text and make some necessary corrections. However, a complete retyping was impossible. Nevertheless, this preliminary version of the book seems to provide a quite detailed introduction and is sufficient to exhibit the scope of the approach and several applications.

The idea of using neighbourhood systems to give set-theoretical representations of domains had been in the back of

my mind for some time in connection with specific examples. But the thought that a systematic development along these lines might be easier to follow than the more abstract lattice-theoretic and topological approach used by myself and others in many publications only came to me during the IFIP Working Group 2.2 meeting in Copenhagen in mid-June 1980. I gave a brief public presentation at ICALP '80 in Holland in mid-July.

One large mistake I have made is to de-emphasize partial orderings too much, since at the right point the concepts and the language are in fact helpful. The basic plan is that, instead of axiomatizing the theory using partial orderings, the necessary facts come out as *theorems*. For a neighbourhood system $\mathcal{D}$, the set of elements $|\mathcal{D}|$, which consists of filters, is naturally partially ordered. And approximable mappings naturally *preserve* the ordering. And so on. The advantage I see from the point of view of exposition is that properties can be brought out one at a time instead of having to put them down all in advance of any experience with the ideas. My own feeling after writing these chapters is that the plan has worked out far better than I could have dared to hope. I was especially glad that I could generate so many *exercises*, and I hope eventually to provide many more. One interesting place at which partial orderings prove their usefulness is in visualizing domains. As it stands now the text does not contain enough in the way of pictures. This will have to be remedied in a future version. Undoubtedly to include enough explanation, several of the lectures will have to be sub-divided into separate chapters.

One major improvement is needed: to bring Exercise 2.22 into the main text. I did not know in advance how often I would need this result for giving (easy) set-theoretical characterizations of domains and structure on them. This will be an easy repair, but it will cause quite a bit of rewriting. Clearly

much more has to be said about the interplay between elements and neighbourhoods. In particular, the character of the elements of a domain, like the power set of a set, has not been sufficiently illustrated, and quite a bit of expansion on this topic is also needed.

Finally I have to explain that I had no time whatsoever to put in references and a bibliography. The ideas I have used have occurred to many, many people who have worked on domains, and I do not wish to claim originality. I *am* claiming some advantage to my style of representation, but I fully realize that a published version will have to have detailed historical references and notes at the ends of the lectures. Needless to say I should very much appreciate any advice or criticism from readers of this preliminary version.

I would like to give a warm word of thanks to the many people who have already commented on the preliminary text both at Oxford and in Boston, where I gave lectures. Very special thanks are due to Steve Comer and Steve Brookes, who spent many hours proof reading the typescripts. The biggest word of thanks, however, is reserved for Elsie Hinkes who, under very considerable pressure, did a wonderful job of typing.

Dana S. Scott
Merton College
Oxford
May 1981

4.2 Domains given by neighbourhoods

<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

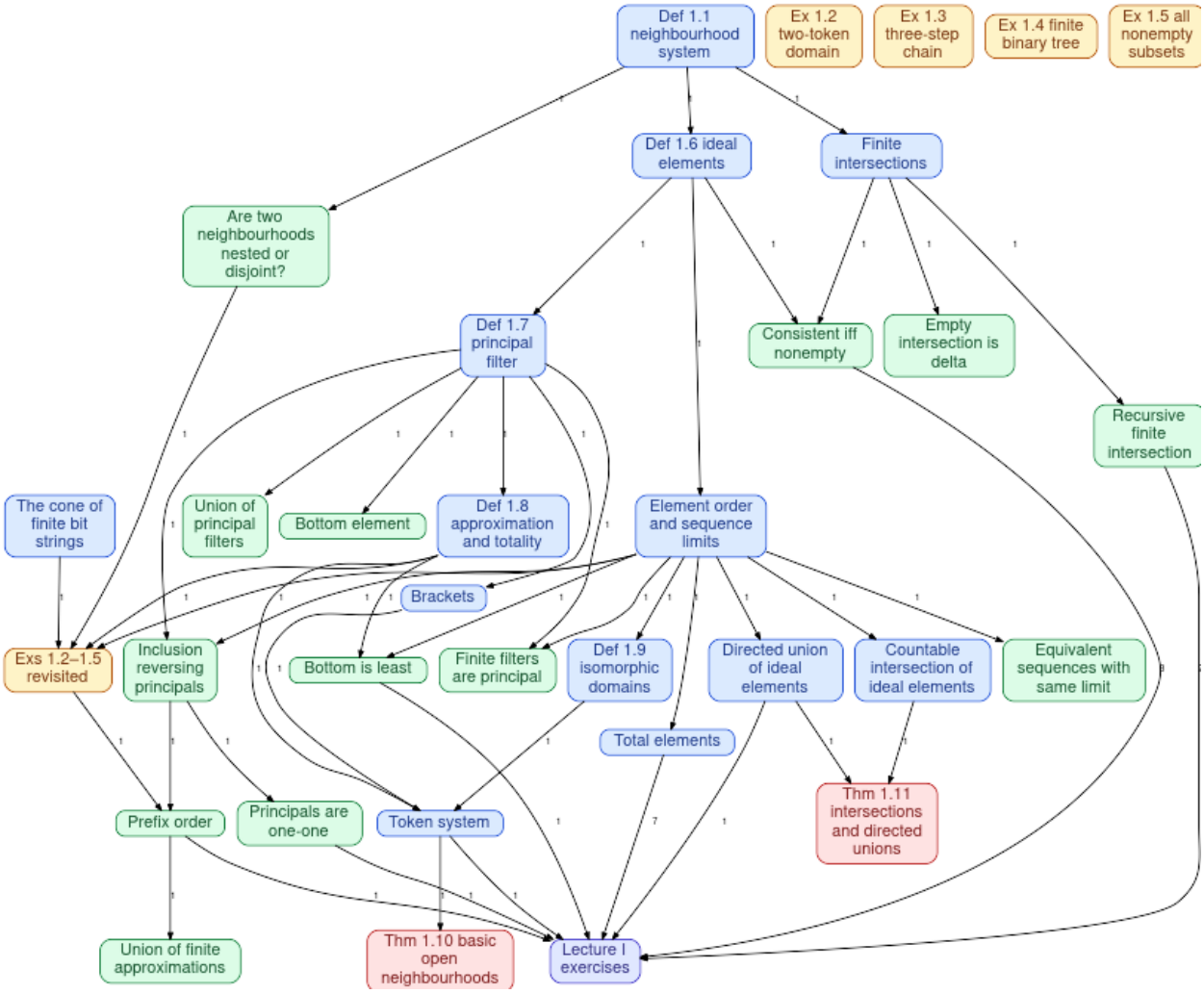


Figure 39: Diagram 7

4.2.1 Introduction

LECTURE I DOMAINS GIVEN BY NEIGHBOURHOODS

Often an object (or element) can be determined by a selection of its properties. Often it is also the case that it is easier (more convenient, more elementary) to think of these properties than it is to think of the elements themselves. Let us term the properties under consideration *neighbourhoods*, the family of those allowed a *neighbourhood system*.

Generally, the collection of these neighbourhoods is, for one reason or another, somewhat restricted; that is, a completely arbitrary property may not be allowable as a neighbourhood. Therefore, the elements determined by selections of neighbourhoods may not be as separable into the discrete objects common to the classical view of set theory. This is particularly true in working with infinite objects: it is hard to specify an infinite element completely. The theory of elements to be studied here, then, is going to permit *partial elements* as well as *total elements*, and each neighbourhood system will define a *domain* of such elements.

Since we may wish to use a neighbourhood system to introduce elements not previously investigated, the neighbourhoods do not have to be regarded as sets of the as-yet-to-be-defined objects. We can take a non-empty set Δ of *tokens* (or "traces") that function as "parts" of elements — or even as parts of "descriptions" of elements. Then a neighbourhood is a subset $X \subseteq \Delta$ containing all those tokens that provide sufficient information when taken together to "approximate" a possible element up to a certain "degree". All these words in inverted commas are vague, and in any case we shall have at the start only a *qualitative* theory of "degree of approximation". A token should be considered as a very "rough" representative of an element, and a neighbourhood should be regarded as "smoothing out" irrelevant details by grouping together *all* those representatives sharing some common feature. One neighbourhood, then, may be only a very incomplete specification of an (ideal) element; fuller specifications can be secured by taking "convergent" sequences of neighbourhoods. Even then convergence need not be to a total element.

Let us call the family of allowed neighbourhoods $\mathcal{D}$; it is a family of subsets of the set Δ. An obvious first question is: when are two neighbourhoods $X, Y \in \mathcal{D}$ neighbourhoods of the "same" element? This question of course generalizes to a (finite) sequence of neighbourhoods. This property we will call the *consistency* of the sequence of neighbourhoods. By definition this will mean that the given neighbourhoods all contain a common neighbourhood in $\mathcal{D}$. That is, for X, Y to be consistent, there must be a $Z \in \mathcal{D}$ with $Z \subseteq X$ and $Z \subseteq Y$. This is not a very informative definition, but it has something of the flavour of a notion of consistency insofar as it can be expressed within $\mathcal{D}$. When consistency holds it seems reasonable enough at first glance to say that the intersection $X \cap Y$ is also an approximation to this common element. If this is reasonable, then $X \cap Y$ should also be regarded as a neighbourhood. This assumption has many consequences, but as a preliminary theory of approximation we will find it quite workable with many natural instances. Taking intersections just means taking more and more properties of the element and putting them together "conjunctively". It is something we do all the time. We therefore accept the idea for the present for giving our first principal definition.

4.2.2 Definition 1.1 — neighbourhood system

DEFINITION 1.1. A family $\mathcal{D}$ of subsets of a given set Δ is called a *neighbourhood system* (over Δ) iff it is a non-empty family closed under the intersection of finite consistent sequences of neighbourhoods. That is to say, $\mathcal{D}$ must fulfill these two conditions:

(i) $\Delta \in \mathcal{D}$;

(ii) whenever $X, Y, Z \in \mathcal{D}$ and $Z \subseteq X \cap Y$, then $X \cap Y \in \mathcal{D}$. $\square$

Definition (`NeighborhoodSystem`). A neighbourhood system on a carrier α consists of a membership predicate on subsets of α, together with a distinguished nonempty *master* set $\Delta \subseteq \alpha$ that belongs to the system, such that every member is contained in Δ, and whenever X and Y are members and some member Z lies in $X \cap Y$, the intersection $X \cap Y$ is again a member.

Proof. This is the packaging of Scott's (i)–(ii), with the master set playing the role of Δ and the intersection clause matching the finite-consistency closure. ■

`NeighborhoodSystem.lean`

```
module

public import Mathlib.Data.Set.Defs

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

structure NeighborhoodSystem (α : Type*) where

  mem : Set α → Prop

  master : Set α

  master_nonempty : master.Nonempty

  master_mem : mem master

  inter_mem : ∀ {X Y Z : Set α}, mem X → mem Y → mem Z → Z ⊆ X ∩ Y → mem (X ∩ Y)

  sub_master : ∀ {X : Set α}, mem X → X ⊆ master

end DomainTheory.NeighbourhoodSystems
```

4.2.3 Finite intersections

We remark that by convention Δ corresponds to the intersection of an empty sequence of neighbourhoods.

The finite intersection of a sequence of neighbourhoods $X_0, \dots, X_{n-1}$ is defined by recursion on n: the empty intersection is the master set Δ, and each successor step intersects with the next neighbourhood.

Of course, from Definition 1.1(ii), we can extend the intersection property to any finite sequence. Consequently, we can say $X_0, \dots, X_{n-1}$ is consistent in $\mathcal{D}$ iff

$$\bigcap_{i<n} X_i \in \mathcal{D}.$$

Definition (`interUpTo`). Given a neighbourhood system $\mathcal{D}$ over Δ and a sequence X of subsets, write $\bigcap_{i<n} X_i$ for the set obtained by recursion: the empty intersection is Δ, and

$$\bigcap_{i<n+1} X_i \;=\; \Big(\bigcap_{i<n} X_i\Big) \cap X_n.$$

Definition (`Consistent`). A length-n initial segment of X is *consistent* when some member of $\mathcal{D}$ is contained in $\bigcap_{i<n} X_i$.

Theorem (`interUpTo.zero`). The empty intersection equals the master set.

Proof. Immediate from the definition. ■

Theorem (`interUpTo.succ`). The successor step peels the last factor as above.

Proof. Immediate from the definition. ■

Theorem (`interUpTo.subset`). If $j < n$, then $\bigcap_{i<n} X_i \subseteq X_j$.

Proof. By induction on n, using the successor equation and the two projections of an intersection. ■

Theorem (`interUpTo.mem`). If each X_i for $i < n$ is a member and the length-n segment is consistent, then $\bigcap_{i<n} X_i$ is a member.

Proof. By induction on n. The empty case is membership of Δ. At a successor, consistency of the longer segment yields consistency of the shorter one, so the inductive hypothesis and pairwise intersection closure finish the argument. ■

Theorem (`consistent_iff_interUpTo_mem`). If each X_i for $i < n$ is a member, then the length-n segment is consistent if and only if $\bigcap_{i<n} X_i$ is a member.

Proof. One direction is the membership theorem for finite intersections; the other takes the intersection itself as the witnessing neighbourhood. ■

Theorem (`interUpTo_antitone`). Lengthening the initial segment shrinks the finite intersection: if $j \leq k$, then $\bigcap_{i<k} X_i \subseteq \bigcap_{i<j} X_i$.

Proof. By induction on k. Equality of lengths is trivial; at a successor the longer intersection is contained in the shorter one by the left projection of an intersection. ■

`interUpTo.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
public import Mathlib.Data.Nat.Notation
import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

/-- Finite intersection of a sequence of neighbourhoods, by recursion on length:
the empty intersection is the master set $\Delta$; the successor step intersects
with the next neighbourhood. -/
def interUpTo (V : NeighborhoodSystem α) (X : ℕ → Set α) : ℕ → Set α
  | 0 => V.master
  | (n + 1) => interUpTo V X n ∩ X n

/-- A length-$n$ sequence is consistent when some neighbourhood sits below the
finite intersection of its first $n$ terms. -/
def Consistent (X : ℕ → Set α) (n : ℕ) : Prop :=
  ∃ Z, V.mem Z ∧ Z ⊆ V.interUpTo X n

namespace interUpTo

@[simp] theorem zero (X : ℕ → Set α) : V.interUpTo X 0 = V.master := rfl
```

```
@[simp] theorem succ (X : ℕ → Set α) (n : ℕ) :
    V.interUpTo X (n + 1) = V.interUpTo X n ∩ X n := rfl

theorem subset (X : ℕ → Set α) :
    ∀ {n j : ℕ}, j < n → V.interUpTo X n ⊆ X j := by
  intro n
  induction n with
  | zero => intro j h; exact absurd h (Nat.not_lt_zero j)
  | succ n ih =>
    intro j h
    rw [succ]
    rcases Nat.eq_or_lt_of_le (Nat.lt_succ_iff.mp h) with h' | h'
    · subst h'; exact Set.inter_subset_right
    · exact Set.inter_subset_left.trans (ih h')

/-- Closure under finite intersections of members, given consistency. -/
theorem mem (X : ℕ → Set α) :
    ∀ {n : ℕ}, (∀ i, i < n → V.mem (X i)) → V.Consistent X n →
      V.mem (V.interUpTo X n) := by
  intro n
  induction n with
  | zero => intro _ _; exact V.master_mem
  | succ n ih =>
    intro hX hcons
    obtain ⟨Z, hZmem, hZsub⟩ := hcons
    have hZsub' : Z ⊆ V.interUpTo X n ∩ X n := by rwa [succ] at hZsub
    have hconsn : V.Consistent X n :=
      ⟨Z, hZmem, hZsub'.trans Set.inter_subset_left⟩
    have hmemn : V.mem (V.interUpTo X n) :=
      ih (fun i hi => hX i (Nat.lt_succ_of_lt hi)) hconsn
    have hXn : V.mem (X n) := hX n (Nat.lt_succ_self n)
    rw [succ]
    exact V.inter_mem hmemn hXn hZmem hZsub'

end interUpTo

/-- Consistency of a finite sequence of members iff their intersection is a member. -/
theorem consistent_iff_interUpTo_mem (X : ℕ → Set α) {n : ℕ}
    (hX : ∀ i, i < n → V.mem (X i)) :
    V.Consistent X n ↔ V.mem (V.interUpTo X n) := by
  constructor
  · exact interUpTo.mem V X hX
  · intro h; exact ⟨V.interUpTo X n, h, Set.Subset.refl _⟩

/-- Lengthening the initial segment shrinks the finite intersection. -/
theorem interUpTo_antitone (X : ℕ → Set α) {j k : ℕ} (h : j ≤ k) :
    V.interUpTo X k ⊆ V.interUpTo X j := by
  induction k with
  | zero => rw [Nat.le_zero.mp h]
  | succ k ih =>
    rcases Nat.eq_or_lt_of_le h with h' | h'
    · rw [h']
    · exact Set.inter_subset_left.trans (ih (Nat.lt_succ_iff.mp h'))

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
```

4.2.4 Empty intersection is delta

$$\bigcap_{i<n} X_i = \Delta, \text{ if } n = 0.$$

Factoid_1_1a.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.interUpTo

@[expose] public section

alias DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.factoid_1_1_a :=
  DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.interUpTo.zero
```

4.2.5 Recursive finite intersection

$$\bigcap_{i<n} X_i = \left(\bigcap_{i<n-1} X_i \right) \cap X_{n-1}, \text{ if } n > 0.$$

Some examples will help us understand the notions better.

Theorem (`factoid_1_1_b`). If $n > 0$, then $\bigcap_{i<n} X_i = (\bigcap_{i<n-1} X_i) \cap X_{n-1}$.

Proof. Write $n = m + 1$; the claim is then the successor clause in the recursive definition of finite intersection. ■

Factoid_1_1b.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.interUpTo

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

/-- Scott Factoid 1.1b: if $n > 0$, then
$\bigcap_{i < n} X_i = (\bigcap_{i < n-1} X_i) \cap X_{n-1}$. -/
theorem factoid_1_1_b (X : ℕ → Set α) {n : ℕ} (hn : 0 < n) :
    V.interUpTo X n = V.interUpTo X (n - 1) ∩ X (n - 1) := by
  cases n with
  | zero => exact (Nat.not_lt_zero _ hn).elim
  | succ n =>
    rw [Nat.add_one_sub_one]
    exact interUpTo.succ V X n

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
```

4.2.6 Element order and sequence limits

Ideal elements are ordered by reverse inclusion of neighbourhoods: $x \leq y$ when every neighbourhood of x is a neighbourhood of y. Two approximating sequences are *equivalent* when each refines the other, and the *limit family* of a sequence is the collection of all neighbourhoods that eventually contain a term of the sequence.

Definition (`element_le`). Write $x \leq y$ when every neighbourhood belonging to x also belongs to y.

Theorem (`element_le_refl`). The relation $\leq$ is reflexive.

Proof. Immediate from the definition. ■

Theorem (`element_le_trans`). The relation $\leq$ is transitive.

Proof. Compose the two neighbourhood-membership implications. ■

Theorem (`Element.ext`). Ideal elements with the same neighbourhoods are equal.

Proof. Equality of the membership predicates determines the element. ■

Theorem (`element_le_antisymm`). The relation $\leq$ is antisymmetric.

Proof. Mutual $\leq$ gives the same neighbourhoods, so the elements coincide by the preceding uniqueness. ■

Together these facts make the set of ideal elements into a partially ordered set under $\leq$.

Definition (`SeqEquiv`). Sequences X and Y of neighbourhoods are *equivalent* when for every index of either sequence some term of the other is contained in it.

Definition (`limitFamily`). The *limit family* of a sequence X is the set of all neighbourhoods Z for which some term X_n is contained in Z.

`ElementOrder.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element
public import Mathlib.Order.Defs.PartialOrder

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

/-- Approximation order on ideal elements: $x \le y$ means every neighbourhood of $x$
is a neighbourhood of $y$. -/
def element_le (x y : V.Element) : Prop :=
  ∀ X, x.mem X → y.mem X

theorem element_le_refl (x : V.Element) : ∀ X, x.mem X → x.mem X :=
  fun _ h => h

theorem element_le_trans (x y z : V.Element)
    (hxy : ∀ X, x.mem X → y.mem X) (hyz : ∀ X, y.mem X → z.mem X) :
    ∀ X, x.mem X → z.mem X :=
  fun X h => hyz X (hxy X h)

theorem Element.ext {x y : V.Element} (h : ∀ X, x.mem X ↔ y.mem X) : x = y := by
  rcases x with ⟨xmem, _, _, _, _⟩
  rcases y with ⟨ymem, _, _, _, _⟩
```

```
  have hmem : xmem = ymem := funext fun X => propext (h X)
  subst hmem
  rfl

theorem element_le_antisymm (x y : V.Element)
    (hxy : ∀ X, x.mem X → y.mem X) (hyx : ∀ X, y.mem X → x.mem X) :
    x = y :=
  @Element.ext α V x y fun X => ⟨hxy X, hyx X⟩

instance : PartialOrder V.Element where
  le := element_le V
  le_refl := element_le_refl V
  le_trans := element_le_trans V
  le_antisymm := element_le_antisymm V

/-- Two sequences of neighbourhoods are equivalent when each is refined by the other.
    -/
def SeqEquiv (X Y : ℕ → Set α) : Prop :=
  (∀ m, ∃ n, X n ⊆ Y m) ∧ (∀ n, ∃ m, Y m ⊆ X n)

/-- Neighbourhoods that eventually contain a term of the sequence $X$. -/
def limitFamily (X : ℕ → Set α) : Set (Set α) := {Z | V.mem Z ∧ ∃ n, X n ⊆ Z}

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
```

4.2.7 Total elements

An ideal element is *total* when it is maximal in the approximation order: whenever $x \leq y$, also $y \leq x$.

`IsTotal.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

/-- An ideal element is *total* when it is maximal in the approximation order. -/
def IsTotal (x : V.Element) : Prop := ∀ y, x ≤ y → y ≤ x

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
```

4.2.8 Two-token neighbourhoods

The two-token neighbourhood system on $\Delta = \{0, 1\}$.

Definition (`Token`). The token set is a two-element type, standing for $\Delta = \{0, 1\}$.

Definition (`master`). The master set is the full set of tokens.

Definition (`zero`). The singleton neighbourhood $\{0\}$.

Definition (`one`). The singleton neighbourhood $\{1\}$.

Definition (`mem`). A subset of the token set belongs to the family precisely when it equals the master set or one of the two singletons.

Theorem (`mem_iff`). A set is a neighbourhood if and only if it equals Δ, $\{0\}$, or $\{1\}$.

Proof. Immediate from the three-element family. ■

Theorem (`inter_eq`). The intersection of any two neighbourhoods is again Δ, $\{0\}$, $\{1\}$, or empty.

Proof. Case analysis on the two factors, using the absorption and disjointness facts for the master and the two singletons. ■

Definition (`neighborhoodSystem`). The neighbourhood system on the two-token carrier with this membership predicate and master set.

Proof. The master is nonempty and a neighbourhood; every neighbourhood lies in the master; if two neighbourhoods meet in a set containing a third neighbourhood, their intersection cannot be empty, so by the preceding classification it is again a neighbourhood. ■

TwoToken.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder

public import Mathlib.Data.Set.Basic
public import Mathlib.Tactic.FinCases
public import Mathlib.Data.Fintype.Basic
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.TwoToken

abbrev Token := Fin 2

def master : Set Token := Set.univ

theorem inter_master (A : Set Token) : A ∩ master = A := by
  rw [master]; exact Set.inter_univ A

theorem master_inter (A : Set Token) : master ∩ A = A := by
  rw [master]; exact Set.univ_inter A

def one : Set Token := {1}

theorem master_not_subset_one : ¬master ⊆ one := by
  intro h
  have : (0 : Token) ∈ one := h (by simp [master])
  simp [one] at this

def zero : Set Token := {0}

theorem master_not_subset_zero : ¬master ⊆ zero := by
  intro h
  have : (1 : Token) ∈ zero := h (by simp [master])
```

```
  simp [zero] at this

def memSet : Set (Set Token) := {master, zero, one}

def mem (X : Set Token) : Prop := X ∈ memSet

theorem mem_master : mem master := by simp [mem, memSet, master, zero, one]

theorem mem_one : mem one := by simp [mem, memSet, master, zero, one]

theorem mem_zero : mem zero := by simp [mem, memSet, master, zero, one]

theorem mem_iff (X : Set Token) : mem X ↔ X = master ∨ X = zero ∨ X = one := by
  constructor
  · intro h
    simp [mem, memSet, master, zero, one] at h
    rcases h with rfl | rfl | rfl
    · exact Or.inl rfl
    · exact Or.inr (Or.inl rfl)
    · exact Or.inr (Or.inr rfl)
  · intro h
    rcases h with rfl | rfl | rfl
    · exact mem_master
    · exact mem_zero
    · exact mem_one

theorem eq_of_master_subset {Y : Set Token} (h : mem Y) (hsub : master ⊆ Y) : Y =
    master := by
  rcases (mem_iff Y).mp h with rfl | hzero | hone
  · rfl
  · exact absurd hzero (fun h' => master_not_subset_zero (h' ▸ hsub))
  · exact absurd hone (fun h' => master_not_subset_one (h' ▸ hsub))

theorem not_mem_empty : ¬mem (∅ : Set Token) := by
  intro h
  rcases (mem_iff (∅ : Set Token)).mp h with h | h | h
  · rw [master] at h; exact Set.empty_ne_univ h
  · simp [zero] at h
  · simp [one] at h

theorem one_inter_zero : one ∩ zero = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [zero, one]

theorem one_not_subset_zero : ¬one ⊆ zero := by
  intro h
  have : (1 : Token) ∈ zero := h (by simp [one])
  simp [zero] at this

theorem zero_inter_one : zero ∩ one = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [zero, one]

theorem inter_eq (X Y : Set Token) (h : mem X) (h' : mem Y) :
    X ∩ Y = master ∨ X ∩ Y = zero ∨ X ∩ Y = one ∨ X ∩ Y = (∅ : Set Token) := by
  rcases (mem_iff X).mp h with rfl | rfl | rfl <;>
    rcases (mem_iff Y).mp h' with rfl | rfl | rfl
  · exact Or.inl (master_inter _)
```

```
  · exact Or.inr (Or.inl (master_inter _))
  · exact Or.inr (Or.inr (Or.inl (master_inter _)))
  · exact Or.inr (Or.inl (inter_master _))
  · exact Or.inr (Or.inl (Set.inter_self _))
  · exact Or.inr (Or.inr (Or.inr zero_inter_one))
  · exact Or.inr (Or.inr (Or.inl (inter_master _)))
  · exact Or.inr (Or.inr (Or.inr one_inter_zero))
  · exact Or.inr (Or.inr (Or.inl (Set.inter_self _)))

def neighborhoodSystem : NeighborhoodSystem Token where
  mem
  master
  master_nonempty := Set.univ_nonempty
  master_mem := mem_master
  sub_master := fun _ => Set.subset_univ _
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    rcases inter_eq X Y hX hY with h | h | h | h
    · rw [h]; exact mem_master
    · rw [h]; exact mem_zero
    · rw [h]; exact mem_one
    · rw [h] at hZsub
      have hz : Z = (∅ : Set Token) := Set.subset_empty_iff.mp hZsub
      subst hz
      exact absurd hZ not_mem_empty

end DomainTheory.NeighbourhoodSystems.TwoToken
```

4.2.9 Example 1.2 — two-token domain

EXAMPLE 1.2. Let $\Delta = \{0, 1\}$ and let $\mathcal{D} = \{\{0, 1\}, \{0\}, \{1\}\}$.

In pictures we have:

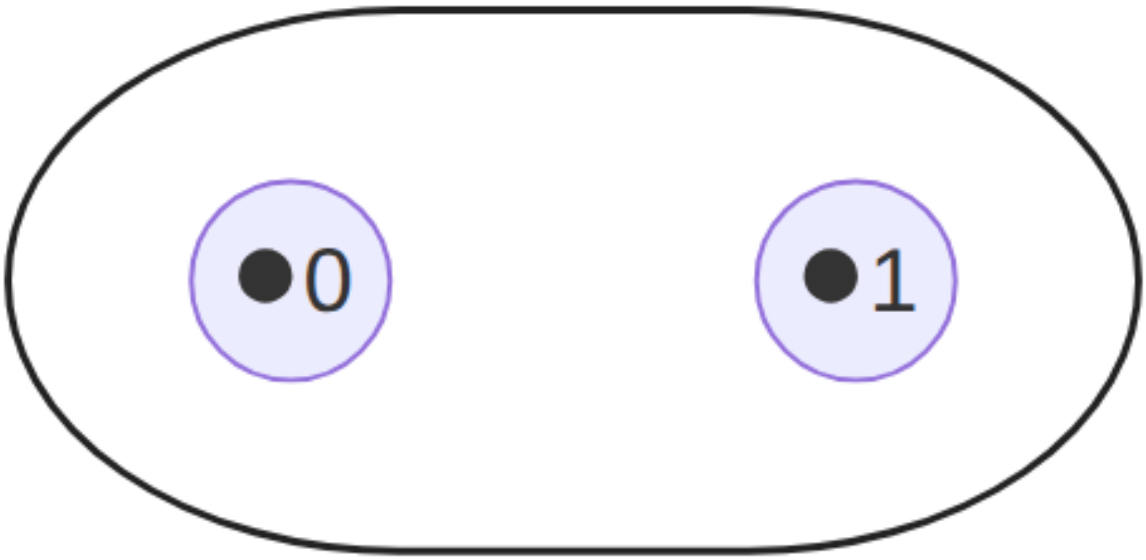


Figure 40: Example 1.2 — two-token domain.

The intention is that 0 and 1 can be completely specified and that they can be identified with the total elements. As we shall see, there is only one partial element: either we give no information (the neighbourhood $\{0,1\}$), or we decide between 0 and 1 (by giving $\{0\}$ or $\{1\}$). □

The development is with the two-token system peer.

Theorem (`example_1_2`). There is a neighbourhood system on $\{0,1\}$ whose master set is the full token set and whose neighbourhoods are exactly $\{\{0,1\},\{0\},\{1\}\}$.

Proof. By the two-token neighbourhood system. ■

`Example_1_2.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.TwoToken

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open TwoToken

/-- **Example 1.2.** Tokens `Δ = {0, 1}` and neighbourhoods `? = {{0, 1}, {0}, {1}}`
form a neighbourhood system. -/
theorem example_1_2 :
    ∃ (V : NeighborhoodSystem Token),
      V.master = (Set.univ : Set Token) ∧
        (∀ X : Set Token,
          V.mem X ↔ X = Set.univ ∨ X = ({0} : Set Token) ∨ X = ({1} : Set Token)) :=
  ⟨neighborhoodSystem, rfl, fun X => (mem_iff X).trans (by simp [master, zero, one])⟩
```

4.2.10 Three-token neighbourhoods

The three-token nested neighbourhood system.

Definition (`Token`). The token set is a three-element type, standing for $\Delta = \{0, 1, 2\}$.

Definition (`master`). The master set is the full set of tokens.

Definition (`twelve`). The intermediate neighbourhood $\{1, 2\}$.

Definition (`two`). The singleton neighbourhood $\{2\}$.

Definition (`mem`). A subset of the token set belongs to the family precisely when it equals the master set, $\{1, 2\}$, or $\{2\}$.

Theorem (`mem_iff`). A set is a neighbourhood if and only if it equals Δ, $\{1, 2\}$, or $\{2\}$.

Proof. Immediate from the three-element family. ■

Theorem (`inter_eq`). The intersection of any two neighbourhoods is again Δ, $\{1, 2\}$, or $\{2\}$.

Proof. Case analysis on the two factors, using absorption of the master and the nesting $\{2\} \subseteq \{1, 2\}$. ■

Definition (`neighborhoodSystem`). The neighbourhood system on the three-token carrier with this membership predicate and master set.

Proof. The master is nonempty and a neighbourhood; every neighbourhood lies in the master; the intersection of any two neighbourhoods is again a neighbourhood by the preceding classification. ■

ThreeToken.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder

public import Mathlib.Data.Set.Basic
public import Mathlib.Tactic.FinCases
public import Mathlib.Data.Fintype.Basic
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.ThreeToken

abbrev Token := Fin 3

def master : Set Token := Set.univ

theorem inter_master (A : Set Token) : A ∩ master = A := by
  rw [master]; exact Set.inter_univ A

theorem master_inter (A : Set Token) : master ∩ A = A := by
  rw [master]; exact Set.univ_inter A

def twelve : Set Token := (insert 2 {1} : Set Token)

def two : Set Token := {2}

def memSet : Set (Set Token) := {master, twelve, two}

def mem (X : Set Token) : Prop := X ∈ memSet

theorem mem_master : mem master := by simp [mem, memSet, master, twelve, two]

theorem mem_twelve : mem twelve := by simp [mem, memSet, master, twelve, two]
```

```
theorem mem_two : mem two := by simp [mem, memSet, master, twelve, two]

theorem mem_iff (X : Set Token) : mem X ↔ X = master ∨ X = twelve ∨ X = two := by
  constructor
  · intro h
    simp [mem, memSet, master, twelve, two] at h
    rcases h with rfl | rfl | rfl
    · exact Or.inl rfl
    · exact Or.inr (Or.inl rfl)
    · exact Or.inr (Or.inr rfl)
  · intro h
    rcases h with rfl | rfl | rfl
    · exact mem_master
    · exact mem_twelve
    · exact mem_two

theorem twelve_inter_two : twelve ∩ two = two := by
  ext t
  fin_cases t <;> simp [twelve, two]

theorem two_inter_twelve : two ∩ twelve = two := by
  ext t
  fin_cases t <;> simp [twelve, two]

theorem inter_eq (X Y : Set Token) (h : mem X) (h' : mem Y) :
    X ∩ Y = master ∨ X ∩ Y = twelve ∨ X ∩ Y = two := by
  rcases (mem_iff X).mp h with rfl | rfl | rfl <;>
    rcases (mem_iff Y).mp h' with rfl | rfl | rfl
  · exact Or.inl (master_inter _)
  · exact Or.inr (Or.inl (master_inter _))
  · exact Or.inr (Or.inr (master_inter _))
  · exact Or.inr (Or.inl (inter_master _))
  · exact Or.inr (Or.inl (Set.inter_self _))
  · exact Or.inr (Or.inr (twelve_inter_two))
  · exact Or.inr (Or.inr (inter_master _))
  · exact Or.inr (Or.inr (two_inter_twelve))
  · exact Or.inr (Or.inr (Set.inter_self _))

def neighborhoodSystem : NeighborhoodSystem Token where
  mem
  master
  master_nonempty := Set.univ_nonempty
  master_mem := mem_master
  sub_master := fun _ => Set.subset_univ _
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    rcases inter_eq X Y hX hY with h | h | h
    · rw [h]; exact mem_master
    · rw [h]; exact mem_twelve
    · rw [h]; exact mem_two

end DomainTheory.NeighbourhoodSystems.ThreeToken
```

4.2.11 Example 1.3 — three-step chain

EXAMPLE 1.3. Let $\Delta = \{0, 1, 2\}$ and let $\mathcal{D} = \{\{0, 1, 2\}, \{1, 2\}, \{2\}\}$.

In pictures we have:

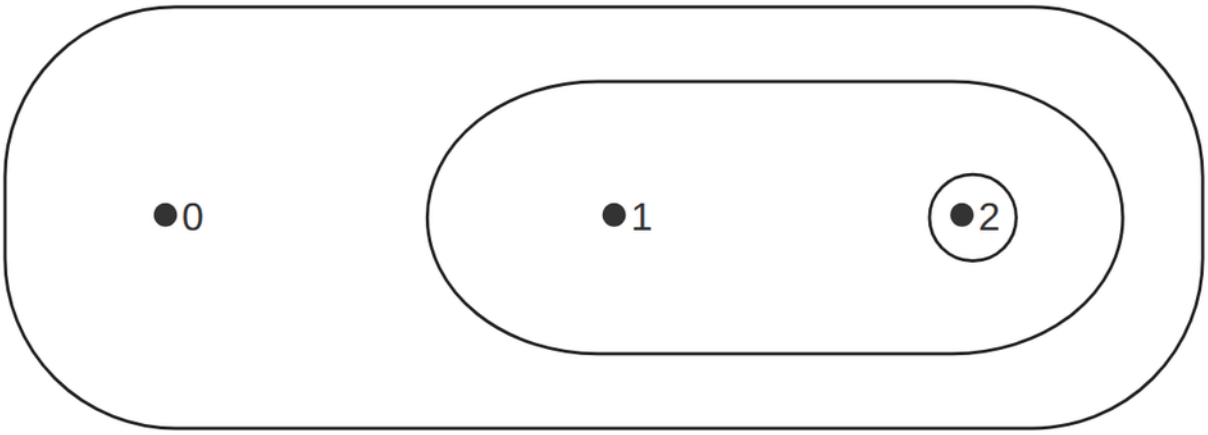


Figure 41: Example 1.3 — three-step chain.

Instead of stepping to the total element (here represented by 2) in one big step, the passage is divided into two steps. (Note 0 and 1 cannot be taken as representing total elements.) This example is not very interesting because the direction of approximation is unique. We need an example with some choice. □

The development is with the three-token system peer.

Theorem (`example_1_3`). There is a neighbourhood system on $\{0, 1, 2\}$ with master Δ and neighbourhoods Δ, $\{1, 2\}$, and $\{2\}$.

Proof. By the three-token neighbourhood system. ■

`Example_1_3.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ThreeToken

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open ThreeToken

theorem example_1_3 :
    ∃ (V : NeighborhoodSystem Token),
      V.master = (Set.univ : Set Token) ∧
        (∀ X : Set Token,
          V.mem X ↔
            X = Set.univ ∨ X = ({1, 2} : Set Token) ∨ X = ({2} : Set Token)) :=
  ⟨neighborhoodSystem, rfl, fun X => by
    have h12 : twelve = ({1, 2} : Set Token) := by
      ext t; fin_cases t <;> simp [twelve]
    refine (mem_iff X).trans ?_
```

```
    simp [master, two, h12]⟩
```

4.2.12 Binary-tree tokens

The finite binary-tree neighbourhood system on seven tokens.

Definition (`Token`). The token set is a seven-element type standing for $\Delta = \{\Lambda, 0, 1, 00, 01, 10, 11\}$.

Definition (`leaf00`), (`leaf01`), (`leaf10`), (`leaf11`). The four singleton neighbourhoods at the leaves of the tree.

Definition (`left`). The left-branch neighbourhood $\{0, 00, 01\}$.

Definition (`right`). The right-branch neighbourhood $\{1, 10, 11\}$.

Definition (`master`). The master set is the full set of tokens.

Definition (`mem`). A subset belongs to the family precisely when it equals the master set, the left or right branch, or one of the four leaves.

Theorem (`mem_iff`). A set is a neighbourhood if and only if it is one of those seven sets.

Proof. Immediate from the seven-element family. ■

Theorem (`not_mem_empty`). The empty set is not a neighbourhood.

Proof. Each of the seven neighbourhoods contains a distinguished token, so none equals the empty set. ■

Theorem (`inter_master`), (`master_inter`). Intersection with the master absorbs.

Proof. The master is the full token set. ■

Theorem (`left_inter_right`), (`right_inter_left`). The two branches are disjoint.

Proof. Direct membership check on the seven tokens. ■

Theorem (`left_inter_leaf00`), (`left_inter_leaf01`), (`leaf00_inter_left`), (`leaf01_inter_left`). Each left leaf sits inside the left branch.

Proof. Direct membership check on the seven tokens. ■

Theorem (`right_inter_leaf10`), (`right_inter_leaf11`), (`leaf10_inter_right`), (`leaf11_inter_right`). Each right leaf sits inside the right branch.

Proof. Direct membership check on the seven tokens. ■

Theorem (`left_inter_leaf10`), (`left_inter_leaf11`), (`leaf10_inter_left`), (`leaf11_inter_left`), (`right_inter_leaf00`), (`right_inter_leaf01`), (`leaf00_inter_right`), (`leaf01_inter_right`). A leaf of one branch misses the other branch.

Proof. Direct membership check on the seven tokens. ■

Theorem (`leaf00_inter_leaf01`), (`leaf00_inter_leaf10`), (`leaf00_inter_leaf11`), (`leaf01_inter_leaf00`), (`leaf01_inter_leaf10`), (`leaf01_inter_leaf11`), (`leaf10_inter_leaf00`), (`leaf10_inter_leaf01`), (`leaf10_inter_leaf11`), (`leaf11_inter_leaf00`), (`leaf11_inter_leaf01`), (`leaf11_inter_leaf10`). Distinct leaves are pairwise disjoint.

Proof. Direct membership check on the seven tokens. ■

Theorem (`inter_mem_or_empty`). The intersection of any two neighbourhoods is again a neighbourhood, or else empty.

Proof. Case analysis on the two factors, assembling absorption by the master, self-intersection, the branch and leaf lemmas above, and the empty intersections. ■

Definition (`neighborhoodSystem`). The neighbourhood system on this token carrier with the membership predicate and master set above.

Proof. The master is nonempty and a neighbourhood; every neighbourhood lies in the master; if two neighbourhoods meet in a set containing a third neighbourhood, their intersection cannot be empty (the empty set is not a neighbourhood), so by the preceding classification it is again a neighbourhood. ■

BinTree.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder

public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Fintype.Basic
public import Mathlib.Tactic.FinCases
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.BinTree

abbrev Token := Fin 7

/-- Encoding: `0 ↦ ?`, `1 ↦ 0`, `2 ↦ 1`, `3 ↦ 00`, `4 ↦ 01`, `5 ↦ 10`, `6 ↦ 11`. -/
def leaf00 : Set Token := {3}
def leaf01 : Set Token := {4}
def leaf10 : Set Token := {5}
def leaf11 : Set Token := {6}
def left : Set Token := {1, 3, 4}
def right : Set Token := {2, 5, 6}
def master : Set Token := Set.univ

def mem (X : Set Token) : Prop :=
  X = master ∨ X = left ∨ X = right ∨ X = leaf00 ∨ X = leaf01 ∨ X = leaf10 ∨ X =
    leaf11

theorem mem_iff (X : Set Token) :
    mem X ↔ X = master ∨ X = left ∨ X = right ∨ X = leaf00 ∨ X = leaf01 ∨ X = leaf10
    ∨
      X = leaf11 :=
  Iff.rfl

theorem mem_master : mem master := Or.inl rfl
theorem mem_left : mem left := Or.inr (Or.inl rfl)
theorem mem_right : mem right := Or.inr (Or.inr (Or.inl rfl))
theorem mem_leaf00 : mem leaf00 := Or.inr (Or.inr (Or.inr (Or.inl rfl)))
theorem mem_leaf01 : mem leaf01 := Or.inr (Or.inr (Or.inr (Or.inr (Or.inl rfl))))
theorem mem_leaf10 : mem leaf10 := Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl rfl)
    ))))
theorem mem_leaf11 : mem leaf11 := Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr rfl)
    ))))

theorem not_mem_empty : ¬mem (∅ : Set Token) := by
  intro h
  rcases (mem_iff _).mp h with h | h | h | h | h | h | h
  · rw [master] at h; exact Set.empty_ne_univ h
  · have h1 : (1 : Token) ∈ left := by simp [left]
    rw [← h] at h1; cases h1
  · have h1 : (2 : Token) ∈ right := by simp [right]
```

```
    rw [← h] at h1; cases h1
  · have h1 : (3 : Token) ∈ leaf00 := by simp [leaf00]
    rw [← h] at h1; cases h1
  · have h1 : (4 : Token) ∈ leaf01 := by simp [leaf01]
    rw [← h] at h1; cases h1
  · have h1 : (5 : Token) ∈ leaf10 := by simp [leaf10]
    rw [← h] at h1; cases h1
  · have h1 : (6 : Token) ∈ leaf11 := by simp [leaf11]
    rw [← h] at h1; cases h1

theorem inter_master (A : Set Token) : A ∩ master = A := by
  rw [master]; exact Set.inter_univ A

theorem master_inter (A : Set Token) : master ∩ A = A := by
  rw [master]; exact Set.univ_inter A

theorem left_inter_right : left ∩ right = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [left, right]
theorem right_inter_left : right ∩ left = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [left, right]
theorem left_inter_leaf00 : left ∩ leaf00 = leaf00 := by
  ext t; fin_cases t <;> simp [left, leaf00]
theorem left_inter_leaf01 : left ∩ leaf01 = leaf01 := by
  ext t; fin_cases t <;> simp [left, leaf01]
theorem left_inter_leaf10 : left ∩ leaf10 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [left, leaf10]
theorem left_inter_leaf11 : left ∩ leaf11 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [left, leaf11]
theorem leaf00_inter_left : leaf00 ∩ left = leaf00 := by
  ext t; fin_cases t <;> simp [left, leaf00]
theorem leaf01_inter_left : leaf01 ∩ left = leaf01 := by
  ext t; fin_cases t <;> simp [left, leaf01]
theorem leaf10_inter_left : leaf10 ∩ left = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [left, leaf10]
theorem leaf11_inter_left : leaf11 ∩ left = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [left, leaf11]
theorem right_inter_leaf00 : right ∩ leaf00 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [right, leaf00]
theorem right_inter_leaf01 : right ∩ leaf01 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [right, leaf01]
theorem right_inter_leaf10 : right ∩ leaf10 = leaf10 := by
  ext t; fin_cases t <;> simp [right, leaf10]
theorem right_inter_leaf11 : right ∩ leaf11 = leaf11 := by
  ext t; fin_cases t <;> simp [right, leaf11]
theorem leaf00_inter_right : leaf00 ∩ right = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [right, leaf00]
theorem leaf01_inter_right : leaf01 ∩ right = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [right, leaf01]
theorem leaf10_inter_right : leaf10 ∩ right = leaf10 := by
  ext t; fin_cases t <;> simp [right, leaf10]
theorem leaf11_inter_right : leaf11 ∩ right = leaf11 := by
  ext t; fin_cases t <;> simp [right, leaf11]
theorem leaf00_inter_leaf01 : leaf00 ∩ leaf01 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf00, leaf01]
theorem leaf00_inter_leaf10 : leaf00 ∩ leaf10 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf00, leaf10]
```

```
theorem leaf00_inter_leaf11 : leaf00 ∩ leaf11 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf00, leaf11]
theorem leaf01_inter_leaf00 : leaf01 ∩ leaf00 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf00, leaf01]
theorem leaf01_inter_leaf10 : leaf01 ∩ leaf10 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf01, leaf10]
theorem leaf01_inter_leaf11 : leaf01 ∩ leaf11 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf01, leaf11]
theorem leaf10_inter_leaf00 : leaf10 ∩ leaf00 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf00, leaf10]
theorem leaf10_inter_leaf01 : leaf10 ∩ leaf01 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf01, leaf10]
theorem leaf10_inter_leaf11 : leaf10 ∩ leaf11 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf10, leaf11]
theorem leaf11_inter_leaf00 : leaf11 ∩ leaf00 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf00, leaf11]
theorem leaf11_inter_leaf01 : leaf11 ∩ leaf01 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf01, leaf11]
theorem leaf11_inter_leaf10 : leaf11 ∩ leaf10 = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [leaf10, leaf11]

/-- Pairwise intersections of neighbourhoods are again neighbourhoods or empty. -/
theorem inter_mem_or_empty (X Y : Set Token) (hX : mem X) (hY : mem Y) :
    mem (X ∩ Y) ∨ X ∩ Y = (∅ : Set Token) := by
  unfold mem at hX hY ⊢
  rcases hX with rfl | rfl | rfl | rfl | rfl | rfl | rfl <;>
    rcases hY with rfl | rfl | rfl | rfl | rfl | rfl | rfl
  · exact Or.inl (Or.inl (master_inter _))
  · exact Or.inl (Or.inr (Or.inl (master_inter _)))
  · exact Or.inl (Or.inr (Or.inr (Or.inl (master_inter _))))
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inl (master_inter _)))))
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl (master_inter _))))))
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl (master_inter _)))))))
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (master_inter _)))))))
  · exact Or.inl (Or.inr (Or.inl (inter_master _)))
  · exact Or.inl (Or.inr (Or.inl (Set.inter_self _)))
  · exact Or.inr left_inter_right
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inl left_inter_leaf00))))
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl left_inter_leaf01)))))
  · exact Or.inr left_inter_leaf10
  · exact Or.inr left_inter_leaf11
  · exact Or.inl (Or.inr (Or.inr (Or.inl (inter_master _))))
  · exact Or.inr right_inter_left
  · exact Or.inl (Or.inr (Or.inr (Or.inl (Set.inter_self _))))
  · exact Or.inr right_inter_leaf00
  · exact Or.inr right_inter_leaf01
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl right_inter_leaf10))))
    ))
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr right_inter_leaf11))))
    ))
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inl (inter_master _)))))
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inl leaf00_inter_left))))
  · exact Or.inr leaf00_inter_right
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inl (Set.inter_self _)))))
  · exact Or.inr leaf00_inter_leaf01
  · exact Or.inr leaf00_inter_leaf10
```

```
  · exact Or.inr leaf00_inter_leaf11
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl (inter_master _))))))
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl leaf01_inter_left)))))
  · exact Or.inr leaf01_inter_right
  · exact Or.inr leaf01_inter_leaf00
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl (Set.inter_self _))))))
  · exact Or.inr leaf01_inter_leaf10
  · exact Or.inr leaf01_inter_leaf11
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl (inter_master _)))))))
  · exact Or.inr leaf10_inter_left
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl leaf10_inter_right))))
    ))
  · exact Or.inr leaf10_inter_leaf00
  · exact Or.inr leaf10_inter_leaf01
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl (Set.inter_self _)))))
    ))
  · exact Or.inr leaf10_inter_leaf11
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (inter_master _)))))))
  · exact Or.inr leaf11_inter_left
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr leaf11_inter_right))))
    ))
  · exact Or.inr leaf11_inter_leaf00
  · exact Or.inr leaf11_inter_leaf01
  · exact Or.inr leaf11_inter_leaf10
  · exact Or.inl (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Set.inter_self _)))))
    ))

def neighborhoodSystem : NeighborhoodSystem Token where
  mem
  master
  master_nonempty := Set.univ_nonempty
  master_mem := mem_master
  sub_master := fun _ => Set.subset_univ _
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    rcases inter_mem_or_empty X Y hX hY with h | h
    · exact h
    · rw [h] at hZsub
      have : Z = (∅ : Set Token) := Set.subset_empty_iff.mp hZsub
      subst this
      exact absurd hZ not_mem_empty

end DomainTheory.NeighbourhoodSystems.BinTree
```

4.2.13 More neighbourhood-system facts

Additional structure on neighbourhood systems (positivity, least filters, power systems, Scott topology, boundedness).

NeighborhoodSystem_more.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint
```

```
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.IsTotal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    tokenSystem
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.interUpTo
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_5a
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_1b
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Factoid_1_5a
public import Mathlib.Data.Fintype.Basic
public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Insert
public import Mathlib.Data.Set.Finite.Basic
public import Mathlib.Data.Set.Finite.Range
public import Mathlib.Tactic.FinCases
public import Mathlib.Order.Preorder.Chain
public import Mathlib.Order.Zorn
public import Mathlib.Algebra.Group.End
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Topology.Defs.Filter
public import Mathlib.Topology.Inseparable
public import Mathlib.Basic.Finite.Defs

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

def NeighborhoodSystem.IsPositive {α : Type*} (V : NeighborhoodSystem α) : Prop :=
  ∀ {{X Y : Set α}}, V.mem X → V.mem Y → (V.mem (X ∩ Y) ↔ (X ∩ Y).Nonempty)

def NeighborhoodSystem.DiffClosed {α : Type*} (D : NeighborhoodSystem α) : Prop :=
  ∀ {X Y : Set α}, D.mem X → D.mem Y → X \ Y = ∅ ∨ D.mem (X \ Y)

def NeighborhoodSystem.NoMinimal {α : Type*} (D : NeighborhoodSystem α) : Prop :=
  ∀ {X : Set α}, D.mem X →
    ∃ Y Z : Set α, D.mem Y ∧ D.mem Z ∧ Y.Nonempty ∧ Z.Nonempty ∧ Y ∩ Z = ∅ ∧ Y ∪ Z = X

def NeighborhoodSystem.ofPositive {α : Type*} (mem : Set α → Prop) (master : Set α)
    (master_nonempty : master.Nonempty) (master_mem : mem master)
    (sub_master : ∀ {X : Set α}, mem X → X ⊆ master)
    (pos : ∀ {{X Y : Set α}}, mem X → mem Y → (mem (X ∩ Y) ↔ (X ∩ Y).Nonempty)) :
    NeighborhoodSystem α where
  mem
  master
  master_nonempty
  master_mem
  sub_master
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    have hZZ : mem (Z ∩ Z) := by rwa [Set.inter_self]
```

```
    have hZne : (Z ∩ Z).Nonempty := (pos hZ hZ).mp hZZ
    rw [Set.inter_self] at hZne
    exact (pos hX hY).mpr (hZne.mono hZsub)

end DomainTheory.NeighbourhoodSystems

set_option linter.unusedSimpArgs false

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {α : Type*} (V : NeighborhoodSystem α)

def chainUnion (C : Set V.Element) (hne : C.Nonempty) (hchain : IsChain (· ≤ ·) C) : V.
    Element where
  mem X := ∃ x ∈ C, x.mem X
  sub := by rintro X ⟨x, _, hxX⟩; exact x.sub hxX
  master_mem := by obtain ⟨x, hxC⟩ := hne; exact ⟨x, hxC, x.master_mem⟩
  inter_mem := by
    rintro X Y ⟨x, hxC, hxX⟩ ⟨y, hyC, hyY⟩
    rcases hchain.total hxC hyC with hxy | hyx
    · exact ⟨y, hyC, y.inter_mem (hxy X hxX) hyY⟩
    · exact ⟨x, hxC, x.inter_mem hxX (hyx Y hyY)⟩
  up_mem := by
    rintro X Y ⟨x, hxC, hxX⟩ hY hXY
    exact ⟨x, hxC, x.up_mem hxX hY hXY⟩

theorem le_chainUnion (C : Set V.Element) (hne : C.Nonempty) (hchain : IsChain (· ≤ ·)
    C)
    {x : V.Element} (hx : x ∈ C) : x ≤ V.chainUnion C hne hchain :=
  fun _ hX => ⟨x, hx, hX⟩

theorem exists_total_ge (x : V.Element) : ∃ t, V.IsTotal t ∧ x ≤ t := by
  obtain ⟨m, hxm, hmax⟩ :=
    zorn_le_nonempty_Ici₀ x
      (fun c _ hchain y hy =>
        ⟨V.chainUnion c ⟨y, hy⟩ hchain, fun z hz => V.le_chainUnion c ⟨y, hy⟩ hchain hz⟩
    )
      x le_rfl
  exact ⟨m, fun y hy => hmax hy, hxm⟩

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end

section

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem
```

```
variable {α : Type*} (V : NeighborhoodSystem α)

def FinitelyConsistent (C : Set (Set α)) : Prop :=
  ∀ (n : ℕ) (X : ℕ → Set α), (∀ i, i < n → X i ∈ C) → V.Consistent X n

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

def appendSeq {α : Type*} (X1 : ℕ → Set α) (n1 : ℕ) (X2 : ℕ → Set α) : ℕ → Set α :=
  fun i => if i < n1 then X1 i else X2 (i - n1)

end DomainTheory.NeighbourhoodSystems

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

theorem appendSeq_mem {α : Type*} {C : Set (Set α)} {X1 : ℕ → Set α} {n1 : ℕ}
    {X2 : ℕ → Set α} {n2 : ℕ} (h1 : ∀ i, i < n1 → X1 i ∈ C) (h2 : ∀ i, i < n2 → X2 i
     ∈ C) :
    ∀ i, i < n1 + n2 → appendSeq X1 n1 X2 i ∈ C := by
  intro i hi
  simp only [appendSeq]
  by_cases h : i < n1
  · rw [ite_eq_left h]; exact h1 i h
  · rw [ite_eq_right h]; exact h2 (i - n1) (by omega)

end DomainTheory.NeighbourhoodSystems

set_option linter.unusedSimpArgs false

set_option linter.unusedSimpArgs false

set_option linter.unusedSimpArgs false

set_option linter.unusedSimpArgs false

section

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem interUpTo_appendSeq_left (X1 : ℕ → Set α) (n1 : ℕ) (X2 : ℕ → Set α) :
    ∀ {k : ℕ}, k ≤ n1 → V.interUpTo (appendSeq X1 n1 X2) k = V.interUpTo X1 k := by
```

```
  intro k
  induction k with
  | zero => intro _; rfl
  | succ k ih =>
    intro hk
    rw [interUpTo.succ, interUpTo.succ, ih (Nat.le_of_succ_le hk)]
    have happ : appendSeq X1 n1 X2 k = X1 k := by
      simp only [appendSeq, ite_eq_left (Nat.lt_of_succ_le hk)]
    rw [happ]

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

section

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem interUpTo_subset_master (X : ℕ → Set α) : ∀ n, V.interUpTo X n ⊆ V.master :=
    by
  intro n
  induction n with
  | zero => exact subset_rfl
  | succ n ih => exact Set.inter_subset_left.trans ih

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

section

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem interUpTo_appendSeq (X1 : ℕ → Set α) (n1 : ℕ) (X2 : ℕ → Set α) :
    ∀ {j : ℕ}, V.interUpTo (appendSeq X1 n1 X2) (n1 + j)
      = V.interUpTo X1 n1 ∩ V.interUpTo X2 j := by
  intro j
  induction j with
  | zero =>
    rw [Nat.add_zero, V.interUpTo_appendSeq_left X1 n1 X2 (le_refl n1), interUpTo.zero
    ]
    exact (Set.inter_eq_left.mpr (V.interUpTo_subset_master X1 n1)).symm
```

```
  | succ j ih =>
    rw [Nat.add_succ, interUpTo.succ, ih]
    have happ : appendSeq X1 n1 X2 (n1 + j) = X2 j := by
      simp only [appendSeq, ite_eq_right (by omega : ¬ n1 + j < n1)]
      congr 1
      omega
    rw [happ, interUpTo.succ, Set.inter_assoc]

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

section

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

def leastFilter (C : Set (Set α)) (hCsub : ∀ X ∈ C, V.mem X)
    (hC : V.FinitelyConsistent C) : V.Element where
  mem Y := V.mem Y ∧ ∃ (n : ℕ) (X : ℕ → Set α), (∀ i, i < n → X i ∈ C) ∧ V.interUpTo
    X n ⊆ Y
  sub h := h.1
  master_mem :=
    ⟨V.master_mem, 0, (fun _ => V.master), (fun i hi => absurd hi (Nat.not_lt_zero i)),
      V.interUpTo_subset_master _ 0⟩
  inter_mem := by
    rintro X Y ⟨hXmem, n1, X1, hX1C, hX1sub⟩ ⟨hYmem, n2, X2, hX2C, hX2sub⟩
    have hmemC : ∀ i, i < n1 + n2 → appendSeq X1 n1 X2 i ∈ C := appendSeq_mem hX1C
    hX2C
    have hintermem : V.mem (V.interUpTo (appendSeq X1 n1 X2) (n1 + n2)) :=
      (V.consistent_iff_interUpTo_mem _ (fun i hi => hCsub _ (hmemC i hi))).mp
        (hC _ _ hmemC)
    have hsub : V.interUpTo (appendSeq X1 n1 X2) (n1 + n2) ⊆ X ∩ Y := by
      rw [V.interUpTo_appendSeq X1 n1 X2]
      exact Set.inter_subset_inter hX1sub hX2sub
    exact ⟨V.inter_mem hXmem hYmem hintermem hsub, n1 + n2, appendSeq X1 n1 X2, hmemC,
    hsub⟩
  up_mem := by
    rintro X Y ⟨_, n, Xs, hXC, hsub⟩ hY hXY
    exact ⟨hY, n, Xs, hXC, hsub.trans hXY⟩

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

section

set_option linter.unusedSimpArgs false
```

```
namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

def sInf (F : Set V.Element) (hF : F.Nonempty) : V.Element where
  mem X := ∀ x ∈ F, x.mem X
  sub h := hF.elim (fun x hx => x.sub (h x hx))
  master_mem := fun x _ => x.master_mem
  inter_mem h1 h2 := fun x hx => x.inter_mem (h1 x hx) (h2 x hx)
  up_mem h hY hXY := fun x hx => x.up_mem (h x hx) hY hXY

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

set_option linter.unusedSimpArgs false

set_option linter.unusedSimpArgs false

section

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem le_sInf (F : Set V.Element) (hF : F.Nonempty) (y : V.Element) (h : ∀ x ∈ F, y
    ≤ x) :
    y ≤ V.sInf F hF :=
  fun _ hX x hx => h x hx _ hX

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end

section

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem leastFilter_le (C : Set (Set α)) (hCsub : ∀ X ∈ C, V.mem X)
    (hC : V.FinitelyConsistent C) (z : V.Element) (hz : ∀ W ∈ C, z.mem W) :
    V.leastFilter C hCsub hC ≤ z := by
  rintro Y ⟨hYmem, n, X, hXC, hsub⟩
  exact z.up_mem (z.mem_interUpTo X (fun i hi => hz (X i) (hXC i hi))) hYmem hsub
```

```
end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end

section

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem sInf_le (F : Set V.Element) (hF : F.Nonempty) {x : V.Element} (hx : x ∈ F) :
    V.sInf F hF ≤ x :=
  fun _ h => h x hx

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end

section

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem subset_leastFilter (C : Set (Set α)) (hCsub : ∀ X ∈ C, V.mem X)
    (hC : V.FinitelyConsistent C) {W : Set α} (hW : W ∈ C) :
    (V.leastFilter C hCsub hC).mem W := by
  refine ⟨hCsub W hW, 1, (fun _ => W), (fun _ _ => hW), ?_⟩
  rw [interUpTo.succ, interUpTo.zero]
  exact Set.inter_subset_right

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end

set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

end DomainTheory.NeighbourhoodSystems

set_option linter.unusedSimpArgs false

set_option linter.unusedSimpArgs false

set_option linter.unusedSimpArgs false
```

```
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

def upSet (X : Set α) : Set (Set α) := {Y | V.mem Y ∧ Y ⊆ X}

@[simp] theorem mem_upSet {X Y : Set α} : Y ∈ V.upSet X ↔ V.mem Y ∧ Y ⊆ X := Iff.rfl

theorem upSet_inter (X Y : Set α) : V.upSet X ∩ V.upSet Y = V.upSet (X ∩ Y) := by
  ext Z
  simp only [Set.mem_inter_iff, mem_upSet, Set.subset_inter_iff]
  tauto

theorem upSet_subset_iff {X Y : Set α} (hX : V.mem X) : V.upSet X ⊆ V.upSet Y ↔ X ⊆ Y
    := by
  constructor
  · intro h
    exact (h ⟨hX, subset_rfl⟩).2
  · intro hXY Z hZ
    exact ⟨hZ.1, hZ.2.trans hXY⟩


def powerSystem : NeighborhoodSystem (Set α) where
  mem S := ∃ X, V.mem X ∧ S = V.upSet X
  master := V.upSet V.master
  master_nonempty := ⟨V.master, V.master_mem, subset_rfl⟩
  master_mem := ⟨V.master, V.master_mem, rfl⟩
  inter_mem := by
    rintro S T W ⟨X, hX, rfl⟩ ⟨Y, hY, rfl⟩ ⟨Z, hZ, rfl⟩ hWsub
    have hZin : Z ∈ V.upSet X ∩ V.upSet Y := hWsub ⟨hZ, subset_rfl⟩
    obtain ⟨⟨_, hZX⟩, _, hZY⟩ := hZin
    have hXY : V.mem (X ∩ Y) := V.inter_mem hX hY hZ (Set.subset_inter hZX hZY)
    exact ⟨X ∩ Y, hXY, V.upSet_inter X Y⟩
  sub_master := by
    intro S hS
    obtain ⟨X, hX, rfl⟩ := hS
    exact (V.upSet_subset_iff hX).mpr (V.sub_master hX)

@[simp] theorem powerSystem_mem {S : Set (Set α)} :
    V.powerSystem.mem S ↔ ∃ X, V.mem X ∧ S = V.upSet X := Iff.rfl

def ofPower (y : V.powerSystem.Element) : V.Element where
  mem W := V.mem W ∧ y.mem (V.upSet W)
  sub h := h.1
  master_mem := ⟨V.master_mem, y.master_mem⟩
  inter_mem := by
    rintro X Y ⟨hX, hyX⟩ ⟨hY, hyY⟩
    have hyInter : y.mem (V.upSet X ∩ V.upSet Y) := y.inter_mem hyX hyY
    rw [V.upSet_inter] at hyInter
    obtain ⟨W, hW, hWeq⟩ := y.sub hyInter
    have hWin : W ∈ V.upSet (X ∩ Y) := by rw [hWeq]; exact ⟨hW, subset_rfl⟩
    exact ⟨V.inter_mem hX hY hW hWin.2, hyInter⟩
  up_mem := by
    rintro X Y ⟨hX, hyX⟩ hY hXY
    exact ⟨hY, y.up_mem hyX ⟨Y, hY, rfl⟩ ((V.upSet_subset_iff hX).mpr hXY)⟩
```

```
def toPower (x : V.Element) : V.powerSystem.Element where
  mem S := ∃ W, x.mem W ∧ S = V.upSet W
  sub := by rintro S ⟨W, hW, rfl⟩; exact ⟨W, x.sub hW, rfl⟩
  master_mem := ⟨V.master, x.master_mem, rfl⟩
  inter_mem := by
    rintro S T ⟨W1, hW1, rfl⟩ ⟨W2, hW2, rfl⟩
    exact ⟨W1 ∩ W2, x.inter_mem hW1 hW2, V.upSet_inter W1 W2⟩
  up_mem := by
    rintro S T ⟨W, hW, rfl⟩ hT hsub
    obtain ⟨W', hW', rfl⟩ := hT
    exact ⟨W', x.up_mem hW hW' ((V.upSet_subset_iff (x.sub hW)).mp hsub), rfl⟩

theorem upSet_injective {X Y : Set α} (hX : V.mem X) (hY : V.mem Y)
    (h : V.upSet X = V.upSet Y) : X = Y :=
  Set.Subset.antisymm ((V.upSet_subset_iff hX).mp h.subset)
    ((V.upSet_subset_iff hY).mp h.superset)

def powerIso : V.Element ≃o V.powerSystem.Element where
  toFun := V.toPower
  invFun := V.ofPower
  left_inv := by
    intro x
    apply Element.ext
    intro W
    constructor
    · rintro ⟨hW, W', hW', heq⟩
      rw [V.upSet_injective hW (x.sub hW') heq]; exact hW'
    · intro hx
      exact ⟨x.sub hx, W, hx, rfl⟩
  right_inv := by
    intro y
    apply Element.ext
    intro S
    constructor
    · rintro ⟨W, ⟨_, hyW⟩, rfl⟩
      exact hyW
    · intro hyS
      obtain ⟨W, hW, rfl⟩ := y.sub hyS
      exact ⟨W, ⟨hW, hyS⟩, rfl⟩
  map_rel_iff' := by
    intro x x'
    constructor
    · intro h W hx
      have hmem : (V.toPower x).mem (V.upSet W) := ⟨W, hx, rfl⟩
      obtain ⟨W', hW', heq⟩ := h (V.upSet W) hmem
      rw [V.upSet_injective (x.sub hx) (x'.sub hW') heq]; exact hW'
    · intro h S hS
      obtain ⟨W, hW, rfl⟩ := hS
      exact ⟨W, h W hW, rfl⟩

theorem isomorphic_powerSystem : V ≅? V.powerSystem := ⟨V.powerIso⟩

theorem powerSystem_isPositive : V.powerSystem.IsPositive := by
  rintro S T ⟨X, hX, rfl⟩ ⟨Y, hY, rfl⟩
  rw [V.upSet_inter]
```

```
  constructor
  · rintro ⟨W, hW, hWeq⟩
    rw [hWeq]
    exact ⟨W, hW, subset_rfl⟩
  · rintro ⟨Z, hZmem, hZsub⟩
    exact ⟨X ∩ Y, V.inter_mem hX hY hZmem hZsub, rfl⟩

theorem toPower_principal {X : Set α} (hX : V.mem X) :
    V.toPower (V.principal hX) = V.powerSystem.principal ⟨X, hX, rfl⟩ := by
  apply Element.ext
  intro S
  constructor
  · rintro ⟨W, ⟨hWmem, hXW⟩, rfl⟩
    exact ⟨⟨W, hWmem, rfl⟩, (V.upSet_subset_iff hX).mpr hXW⟩
  · rintro ⟨⟨W, hWmem, rfl⟩, hsub⟩
    exact ⟨W, ⟨hWmem, (V.upSet_subset_iff hX).mp hsub⟩, rfl⟩

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {α : Type*} (V : NeighborhoodSystem α)

def IsComplete {β : Type*} (V' : NeighborhoodSystem β) : Prop :=
  ∀ y : V'.Element, ∃! b : β, ∀ S, V'.mem S → (y.mem S ↔ b ∈ S)

theorem bracket_nonempty_iff {X : Set α} : (V.bracket X).Nonempty ↔ V.mem X := by
  constructor
  · rintro ⟨x, hx⟩; exact x.sub hx
  · intro hX; exact ⟨V.principal hX, V.principal_mem_bracket hX⟩

theorem consistent_iff_bracket_interUpTo_nonempty (X : ℕ → Set α) {n : ℕ}
    (hX : ∀ i, i < n → V.mem (X i)) :
    V.Consistent X n ↔ (V.bracket (V.interUpTo X n)).Nonempty := by
  rw [V.consistent_iff_interUpTo_mem X hX]
  exact V.bracket_nonempty_iff.symm

theorem consistent_iff_iInter_bracket_nonempty (X : ℕ → Set α) {n : ℕ}
    (hX : ∀ i, i < n → V.mem (X i)) :
    V.Consistent X n ↔ ∃ x : V.Element, ∀ i, i < n → x ∈ V.bracket (X i) := by
  rw [V.consistent_iff_bracket_interUpTo_nonempty X hX]
  constructor
  · rintro ⟨x, hx⟩
    exact ⟨x, fun i hi => x.up_mem hx (hX i hi) (interUpTo.subset V X hi)⟩
  · rintro ⟨x, hx⟩
    exact ⟨x, x.mem_interUpTo X (fun i hi => hx i hi)⟩

theorem tokenSystem_complete : V.tokenSystem.IsComplete := by
  intro y
  refine ⟨V.ofToken y, ?_, ?_⟩
  · rintro S ⟨W, hW, rfl⟩
    constructor
```

```
    · intro hyW; exact ⟨hW, hyW⟩
    · rintro ⟨_, hyW⟩; exact hyW
  · intro b hb
    apply Element.ext
    intro W
    by_cases hW : V.mem W
    · have hb' := hb (V.bracket W) ⟨W, hW, rfl⟩
      constructor
      · intro hbW; exact ⟨hW, hb'.mpr hbW⟩
      · intro hxW; exact hb'.mp hxW.2
    · constructor
      · intro hbW; exact absurd (b.sub hbW) hW
      · intro hxW; exact absurd ((V.ofToken y).sub hxW) hW

theorem tokenSystem_isPositive : V.tokenSystem.IsPositive := by
  rintro S T ⟨X, hX, rfl⟩ ⟨Y, hY, rfl⟩
  rw [V.bracket_inter hX hY]
  constructor
  · rintro ⟨W, hW, hWeq⟩
    rw [hWeq]
    exact ⟨V.principal hW, V.principal_mem_bracket hW⟩
  · rintro ⟨x, hx⟩
    exact ⟨X ∩ Y, x.sub hx, rfl⟩

theorem tokenSystem_toToken_bijective : Function.Bijective V.toToken :=
  V.tokenIso.bijective

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end

namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

def basicOpen (X : Set α) : Set V.Element := {x | x.mem X}

@[simp] theorem mem_basicOpen {X : Set α} {x : V.Element} :
    x ∈ V.basicOpen X ↔ x.mem X := Iff.rfl

class OpenFilter (U : Set V.Element) : Prop where
  out : ∀ x ∈ U, ∃ X, x.mem X ∧ V.basicOpen X ⊆ U

theorem basicOpen_inter_subset_left {X Y : Set α} (hX : V.mem X) :
    V.basicOpen (X ∩ Y) ⊆ V.basicOpen X :=
  fun z hz => z.up_mem hz hX Set.inter_subset_left

theorem basicOpen_inter_subset_right {X Y : Set α} (hY : V.mem Y) :
    V.basicOpen (X ∩ Y) ⊆ V.basicOpen Y :=
  fun z hz => z.up_mem hz hY Set.inter_subset_right

instance instTopologicalSpaceElement : TopologicalSpace V.Element where
  IsOpen := V.OpenFilter
```

```
  isOpen_univ := ⟨fun x _ => ⟨V.master, x.master_mem, Set.subset_univ _⟩⟩
  isOpen_inter := by
    intro U W hU hW
    constructor
    intro x hx
    obtain ⟨hxU, hxW⟩ := hx
    obtain ⟨X, hX, hXU⟩ := hU.out x hxU
    obtain ⟨Y, hY, hYW⟩ := hW.out x hxW
    refine ⟨X ∩ Y, x.inter_mem hX hY, fun z hz => ⟨hXU ?_, hYW ?_⟩⟩
    · exact V.basicOpen_inter_subset_left (x.sub hX) hz
    · exact V.basicOpen_inter_subset_right (x.sub hY) hz
  isOpen_sUnion := by
    intro S hS
    constructor
    intro x hx
    obtain ⟨t, htS, hxt⟩ := hx
    obtain ⟨X, hX, hXt⟩ := (hS t htS).out x hxt
    exact ⟨X, hX, hXt.trans fun _ ha => ⟨t, htS, ha⟩⟩

theorem isOpen_basicOpen (X : Set α) : IsOpen (V.basicOpen X) :=
  ⟨fun _ hx => ⟨X, hx, subset_rfl⟩⟩

theorem isOpen_iff_isOpenFilter (U : Set V.Element) : IsOpen U ↔ V.OpenFilter U :=
  ⟨fun h => h, fun h => h⟩

theorem isOpen_isUpperSet {U : Set V.Element} (hU : IsOpen U) :
    ∀ {{x y : V.Element}}, x ∈ U → x ≤ y → y ∈ U := by
  intro x y hxU hxy
  obtain ⟨X, hX, hXU⟩ := hU.out x hxU
  exact hXU (hxy X hX)

theorem isOpen_iff_upper_basic (U : Set V.Element) :
    IsOpen U ↔
      (∀ {{x y : V.Element}}, x ∈ U → x ≤ y → y ∈ U) ∧
        (∀ x ∈ U, ∃ X, x.mem X ∧ V.basicOpen X ⊆ U) := by
  constructor
  · intro hU
    exact ⟨V.isOpen_isUpperSet hU, hU.out⟩
  · rintro ⟨_, h2⟩
    exact ⟨h2⟩

theorem le_iff_isOpen_imp (x y : V.Element) :
    x ≤ y ↔ ∀ U : Set V.Element, IsOpen U → x ∈ U → y ∈ U := by
  constructor
  · intro hxy U hU hxU
    exact V.isOpen_isUpperSet hU hxU hxy
  · intro h
    exact fun X hX => h (V.basicOpen X) (V.isOpen_basicOpen X) hX

theorem specializes_iff_le (x y : V.Element) : y  ~>  x ↔ x ≤ y := by
  rw [specializes_iff_forall_open]
  exact (V.le_iff_isOpen_imp x y).symm

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
```

```
section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {α : Type*} (V : NeighborhoodSystem α)

def Bounded (X : Set V.Element) : Prop := ∃ y : V.Element, ∀ x ∈ X, x ≤ y

def pairSeq (U W : Set α) : ℕ → Set α := fun i => if i = 0 then U else W

def upperBounds (X : Set V.Element) : Set V.Element := {y | ∀ x ∈ X, x ≤ y}

theorem upperBounds_nonempty {X : Set V.Element} (hX : V.Bounded X) :
    (V.upperBounds X).Nonempty :=
  hX.imp fun _ hy => hy

def sSup (X : Set V.Element) (hX : V.Bounded X) : V.Element :=
  V.sInf (V.upperBounds X) (V.upperBounds_nonempty hX)

theorem bounded_iff_finite_bounded (X : Set V.Element) :
    V.Bounded X ↔ ∀ S : Set V.Element, S ⊆ X → S.Finite → V.Bounded S := by
  constructor
  · rintro ⟨y, hy⟩ S hS _
    exact ⟨y, fun x hx => hy x (hS hx)⟩
  · intro hfin
    set C : Set (Set α) := {Z | ∃ x : V.Element, x ∈ X ∧ x.mem Z} with hCdef
    have hCsub : ∀ Z ∈ C, V.mem Z := by rintro Z ⟨x, _, hxZ⟩; exact x.sub hxZ
    have hCcons : V.FinitelyConsistent C := by
      intro n seq hseqC
      choose g hgX hgmem using hseqC
      set S : Set V.Element := Set.range (fun i : Fin n => g i.1 i.2) with hSdef
      have hSfin : S.Finite := Set.finite_range _
      have hSsub : S ⊆ X := by
        rintro _ ⟨i, rfl⟩
        exact hgX i.1 i.2
      obtain ⟨y, hy⟩ := hfin S hSsub hSfin
      have hseqy : ∀ i, i < n → y.mem (seq i) := by
        intro i hi
        have hle : g i hi ≤ y := hy _ ⟨⟨i, hi⟩, rfl⟩
        exact hle (seq i) (hgmem i hi)
      have hmem : y.mem (V.interUpTo seq n) := y.mem_interUpTo seq hseqy
      exact ⟨V.interUpTo seq n, y.sub hmem, subset_rfl⟩
    refine ⟨V.leastFilter C hCsub hCcons, ?_⟩
    intro x hx Z hZ
    exact V.subset_leastFilter C hCsub hCcons ⟨x, hx, hZ⟩

theorem interUpTo_pairSeq (U W : Set α) :
    V.interUpTo (pairSeq U W) 2 = V.master ∩ U ∩ W := by
  change (V.master ∩ pairSeq U W 0) ∩ pairSeq U W 1 = V.master ∩ U ∩ W
  simp [pairSeq]

theorem consistent_pair_iff_bounded {U W : Set α} (hU : V.mem U) (hW : V.mem W) :
    V.Consistent (pairSeq U W) 2 ↔ V.Bounded {V.principal hU, V.principal hW} := by
  constructor
  · rintro ⟨Z, hZmem, hZsub⟩
```

```
    rw [V.interUpTo_pairSeq] at hZsub
    have hZU : Z ⊆ U := hZsub.trans (Set.inter_subset_left.trans Set.
    inter_subset_right)
    have hZW : Z ⊆ W := hZsub.trans Set.inter_subset_right
    refine ⟨V.principal hZmem, ?_⟩
    intro x hx
    rcases hx with rfl | rfl
    · exact (V.principal_le_iff hU hZmem).mpr hZU
    · exact (V.principal_le_iff hW hZmem).mpr hZW
  · rintro ⟨y, hy⟩
    have hyU : y.mem U :=
      hy _ (Or.inl rfl) U ⟨hU, subset_rfl⟩
    have hyW : y.mem W :=
      hy _ (Or.inr rfl) W ⟨hW, subset_rfl⟩
    have hyUW : y.mem (U ∩ W) := y.inter_mem hyU hyW
    refine ⟨U ∩ W, y.sub hyUW, ?_⟩
    rw [V.interUpTo_pairSeq]
    intro z hz
    exact ⟨⟨V.sub_master hU hz.1, hz.1⟩, hz.2⟩

theorem le_sSup (X : Set V.Element) (hX : V.Bounded X) {x : V.Element} (hx : x ∈ X) :
    x ≤ V.sSup X hX :=
  V.le_sInf (V.upperBounds X) (V.upperBounds_nonempty hX) x (fun _ hy => hy x hx)

theorem sSup_le (X : Set V.Element) (hX : V.Bounded X) {z : V.Element}
    (hz : ∀ x ∈ X, x ≤ z) : V.sSup X hX ≤ z :=
  V.sInf_le (V.upperBounds X) (V.upperBounds_nonempty hX) hz

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.14 Three-set consistency counterexample

A three-set family that is pairwise consistent but not finitely consistent.

TriSys.lean

```
module

import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Factoid_1_5a
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_1b
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    PositiveDiff
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more

public import Mathlib.Data.Set.Basic
public import Mathlib.Tactic.IntervalCases
@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems.TriSys
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

def A : Set (Fin 3) := {0, 1}

def B : Set (Fin 3) := {1, 2}

def Cc : Set (Fin 3) := {0, 2}

def family : Set (Set (Fin 3)) := {A, B, Cc}

def system : NeighborhoodSystem (Fin 3) :=
  NeighborhoodSystem.ofPositive (fun X => X.Nonempty) Set.univ
    (⟨0, Set.mem_univ 0⟩) (⟨0, Set.mem_univ 0⟩)
    (fun {_} _ => Set.subset_univ _) (fun _ _ _ _ => Iff.rfl)

theorem system_master : system.master = (Set.univ : Set (Fin 3)) := rfl

def triple : ℕ → Set (Fin 3) := fun i => if i = 0 then A else if i = 1 then B else Cc

theorem family_pairwise_nonempty :
    ∀ X ∈ family, ∀ Y ∈ family, (X ∩ Y).Nonempty := by
  intro X hX Y hY
  simp only [family, Set.mem_insert_iff, Set.mem_singleton_iff] at hX hY
  rcases hX with rfl | rfl | rfl <;> rcases hY with rfl | rfl | rfl
  · exact ⟨0, by simp [A]⟩
  · exact ⟨1, by simp [A, B]⟩
  · exact ⟨0, by simp [A, Cc]⟩
  · exact ⟨1, by simp [A, B]⟩
  · exact ⟨1, by simp [B]⟩
  · exact ⟨2, by simp [B, Cc]⟩
  · exact ⟨0, by simp [A, Cc]⟩
  · exact ⟨2, by simp [B, Cc]⟩
  · exact ⟨0, by simp [Cc]⟩

theorem triple_interUpTo_empty : system.interUpTo triple 3 = (∅ : Set (Fin 3)) := by
  simp only [NeighborhoodSystem.interUpTo.succ, NeighborhoodSystem.interUpTo.zero,
    system_master]
  ext x
  fin_cases x <;> simp [triple, A, B, Cc]

theorem triple_mem : ∀ i, i < 3 → triple i ∈ family := by
  intro i hi
  simp only [family, Set.mem_insert_iff, Set.mem_singleton_iff]
  interval_cases i
  · exact Or.inl rfl
  · exact Or.inr (Or.inl rfl)
  · exact Or.inr (Or.inr rfl)

theorem not_finitelyConsistent : ¬ system.FinitelyConsistent family := by
  intro h
  obtain ⟨Z, hZmem, hZsub⟩ := h 3 triple triple_mem
  have hZne : Z.Nonempty := hZmem
  rw [triple_interUpTo_empty, Set.subset_empty_iff] at hZsub
  rw [hZsub] at hZne
```

```
  exact Set.not_nonempty_empty hZne

end DomainTheory.NeighbourhoodSystems.TriSys
```

4.2.15 Ideal neighbourhoods

Neighbourhood families of ideals in a commutative ring.

`Ring.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import Mathlib.RingTheory.Ideal.Span

@[expose] public section

set_option linter.unusedSectionVars false
set_option linter.overlappingInstances false

namespace DomainTheory.NeighbourhoodSystems.Ring

open NeighborhoodSystem
variable {A : Type*} [CommRing A]

def IFamily (F : Finset A) : Set (Finset A) := {G | (↑F : Set A) ⊆ Ideal.span (↑G :
    Set A)}

@[simp] theorem mem_IFamily {F G : Finset A} :
    G ∈ IFamily F ↔ (↑F : Set A) ⊆ Ideal.span (↑G : Set A) := Iff.rfl

variable [DecidableEq A]

theorem IFamily_inter (F₁ F₂ : Finset A) :
    IFamily F₁ ∩ IFamily F₂ = IFamily (F₁ ∪ F₂) := by
  ext G
  simp only [IFamily, Set.mem_inter_iff, Set.mem_ofPred_eq, Finset.coe_union, Set.
    union_subset_iff]

theorem IFamily_subset_iff (F₁ F₂ : Finset A) :
    IFamily F₁ ⊆ IFamily F₂ ↔ (↑F₂ : Set A) ⊆ Ideal.span (↑F₁ : Set A) := by
  constructor
  · intro h
    have hF₁ : F₁ ∈ IFamily F₁ := mem_IFamily.mpr Ideal.subset_span
    exact (h hF₁)
  · intro h G hG
    rw [mem_IFamily] at hG ⊢
    exact h.trans (SetLike.coe_subset_coe.mpr (Ideal.span_le.mpr hG))

def ringSystem : NeighborhoodSystem (Finset A) where
  mem S := ∃ F, S = IFamily F
  master := IFamily ∅
  master_nonempty := ⟨∅, by simp [IFamily]⟩
  master_mem := ⟨∅, rfl⟩
```

```
  inter_mem := by
    rintro X Y Z ⟨F₁, rfl⟩ ⟨F₂, rfl⟩ _ _
    exact ⟨F₁ ∪ F₂, IFamily_inter F₁ F₂⟩
  sub_master := by
    rintro X ⟨F, rfl⟩
    exact (IFamily_subset_iff F ∅).mpr (by rw [Finset.coe_empty]; exact Set.
    empty_subset _)

@[simp] theorem ringSystem_mem {S : Set (Finset A)} :
    (ringSystem (A := A)).mem S ↔ ∃ F, S = IFamily F := Iff.rfl

theorem IFamily_mem (F : Finset A) : (ringSystem (A := A)).mem (IFamily F) := ⟨F, rfl⟩

def idealOf (x : (ringSystem (A := A)).Element) : Ideal A where
  carrier := {a | x.mem (IFamily {a})}
  zero_mem' := by
    show x.mem (IFamily {(0 : A)})
    refine x.up_mem x.master_mem (IFamily_mem {(0 : A)}) ?_
    intro G _
    rw [mem_IFamily, Finset.coe_singleton, Set.singleton_subset_iff]
    exact Submodule.zero_mem _
  add_mem' := by
    intro a b ha hb
    show x.mem (IFamily {a + b})
    have h := x.inter_mem ha hb
    rw [IFamily_inter] at h
    refine x.up_mem h (IFamily_mem {a + b}) ?_
    apply (IFamily_subset_iff _ _).mpr
    rw [Finset.coe_singleton, Set.singleton_subset_iff, Finset.coe_union,
      Finset.coe_singleton, Finset.coe_singleton]
    exact Ideal.add_mem _ (Ideal.subset_span (by simp)) (Ideal.subset_span (by simp))
  smul_mem' := by
    intro c a ha
    show x.mem (IFamily {c * a})
    refine x.up_mem ha (IFamily_mem {c * a}) ?_
    apply (IFamily_subset_iff _ _).mpr
    rw [Finset.coe_singleton, Set.singleton_subset_iff, Finset.coe_singleton]
    exact Submodule.smul_mem _ c (Ideal.subset_span (Set.mem_singleton a))

@[simp] theorem mem_idealOf {x : (ringSystem (A := A)).Element} {a : A} :
    a ∈ idealOf x ↔ x.mem (IFamily {a}) := Iff.rfl

theorem mem_IFamily_of_forall (x : (ringSystem (A := A)).Element) (F : Finset A)
    (h : ∀ a ∈ F, x.mem (IFamily {a})) : x.mem (IFamily F) := by
  classical
  induction F using Finset.induction with
  | empty => exact x.master_mem
  | @insert a F' _ ih =>
    have hins : (insert a F' : Finset A) = {a} ∪ F' := by
      ext y; simp [Finset.mem_insert]
    rw [hins, ← IFamily_inter]
    exact x.inter_mem (h a (by simp)) (ih (fun b hb => h b (by simp [hb])))

def ofIdeal (J : Ideal A) : (ringSystem (A := A)).Element where
  mem S := ∃ F, S = IFamily F ∧ Ideal.span (↑F : Set A) ≤ J
  sub := by rintro S ⟨F, rfl, _⟩; exact ⟨F, rfl⟩
```

```
  master_mem := by
    refine ⟨∅, rfl, ?_⟩
    rw [Finset.coe_empty, Ideal.span_empty]
    exact bot_le
  inter_mem := by
    rintro X Y ⟨F₁, rfl, h₁⟩ ⟨F₂, rfl, h₂⟩
    refine ⟨F₁ ∪ F₂, IFamily_inter F₁ F₂, ?_⟩
    rw [Finset.coe_union, Ideal.span_union]
    exact sup_le h₁ h₂
  up_mem := by
    rintro X Y ⟨F, rfl, hF⟩ ⟨F', rfl⟩ hXY
    refine ⟨F', rfl, ?_⟩
    have hsub : (↑F' : Set A) ⊆ Ideal.span (↑F : Set A) := (IFamily_subset_iff F F').
    mp hXY
    exact (Ideal.span_le.mpr hsub).trans hF

theorem ofIdeal_idealOf (x : (ringSystem (A := A)).Element) : ofIdeal (idealOf x) = x
    := by
  apply NeighborhoodSystem.Element.ext
  intro S
  constructor
  · rintro ⟨F, rfl, hF⟩
    refine mem_IFamily_of_forall x F (fun a ha => ?_)
    have haspan : a ∈ Ideal.span (↑F : Set A) := Ideal.subset_span (by simpa using ha)
    exact mem_idealOf.mp (hF haspan)
  · intro hS
    obtain ⟨F, rfl⟩ := x.sub hS
    refine ⟨F, rfl, ?_⟩
    rw [Ideal.span_le]
    intro a ha
    rw [SetLike.mem_coe, mem_idealOf]
    refine x.up_mem hS (IFamily_mem {a}) ?_
    apply (IFamily_subset_iff _ _).mpr
    rw [Finset.coe_singleton, Set.singleton_subset_iff]
    exact Ideal.subset_span ha

theorem span_eq_of_IFamily_eq {F₁ F₂ : Finset A} (h : IFamily F₁ = IFamily F₂) :
    Ideal.span (↑F₁ : Set A) = Ideal.span (↑F₂ : Set A) := by
  have h12 : (↑F₂ : Set A) ⊆ Ideal.span (↑F₁ : Set A) :=
    (IFamily_subset_iff F₁ F₂).mp h.le
  have h21 : (↑F₁ : Set A) ⊆ Ideal.span (↑F₂ : Set A) :=
    (IFamily_subset_iff F₂ F₁).mp h.ge
  exact le_antisymm (Ideal.span_le.mpr h21) (Ideal.span_le.mpr h12)

theorem mem_ofIdeal_IFamily (J : Ideal A) (F : Finset A) :
    (ofIdeal J).mem (IFamily F) ↔ Ideal.span (↑F : Set A) ≤ J := by
  constructor
  · rintro ⟨F', hF', hle⟩
    rw [span_eq_of_IFamily_eq hF']
    exact hle
  · intro h
    exact ⟨F, rfl, h⟩

theorem idealOf_ofIdeal (J : Ideal A) : idealOf (ofIdeal J) = J := by
  apply SetLike.ext
  intro a
```

```
  rw [mem_idealOf, mem_ofIdeal_IFamily, Finset.coe_singleton, Ideal.
    span_singleton_le_iff_mem]

def ringIso : (ringSystem (A := A)).Element ≃o Ideal A where
  toFun := idealOf
  invFun := ofIdeal
  left_inv := ofIdeal_idealOf
  right_inv := idealOf_ofIdeal
  map_rel_iff' := by
    intro x y
    constructor
    · intro h S hS
      obtain ⟨F, rfl⟩ := x.sub hS
      refine mem_IFamily_of_forall y F (fun a ha => ?_)
      have hax : a ∈ idealOf x := by
        rw [mem_idealOf]
        exact x.up_mem hS (IFamily_mem {a}) <| (IFamily_subset_iff _ _).mpr <| by
          rw [Finset.coe_singleton, Set.singleton_subset_iff]
          exact Ideal.subset_span (by simpa using ha)
      exact mem_idealOf.mp (h hax)
    · intro h a ha
      exact h (IFamily {a}) ha

theorem IFamily_eq_of_span_eq {F₁ F₂ : Finset A}
    (h : Ideal.span (↑F₁ : Set A) = Ideal.span (↑F₂ : Set A)) : IFamily F₁ = IFamily F₂
    := by
  apply Set.Subset.antisymm
  · exact (IFamily_subset_iff F₁ F₂).mpr (by rw [h]; exact Ideal.subset_span)
  · exact (IFamily_subset_iff F₂ F₁).mpr (by rw [← h]; exact Ideal.subset_span)

end DomainTheory.NeighbourhoodSystems.Ring
```

4.2.16 Final-segment orders

Upper-set neighbourhood systems on linear orders.

`FinalSegment.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.IsTotal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint
public import Mathlib.Algebra.Group.End
public import Mathlib.Order.UpperLower.Basic
import Mathlib.Order.WellFounded

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.FinalSegment

open NeighborhoodSystem

theorem upperSet_subset_or {Δ : Type*} [LinearOrder Δ] {X Y : Set Δ}
```

```
    (hX : IsUpperSet X) (hY : IsUpperSet Y) : X ⊆ Y ∨ Y ⊆ X := by
  by_cases h : X ⊆ Y
  · exact Or.inl h
  · right
    rw [Set.not_subset] at h
    obtain ⟨a, haX, haY⟩ := h
    intro b hbY
    rcases le_total a b with hab | hba
    · exact hX hab haX
    · exact absurd (hY hba hbY) haY

def finalSegmentSystem (Δ : Type*) [LinearOrder Δ] [Nonempty Δ] : NeighborhoodSystem
    Δ :=
  NeighborhoodSystem.ofNestedOrDisjoint
    (fun X => X.Nonempty ∧ IsUpperSet X)
    Set.univ
    Set.univ_nonempty
    ⟨Set.univ_nonempty, isUpperSet_univ⟩
    (fun _ _ hX hY => (upperSet_subset_or hX.2 hY.2).imp_right Or.inl)
    (fun _ => Set.subset_univ _)

variable {Δ : Type*} [LinearOrder Δ] [Nonempty Δ]

@[simp] theorem mem_def {X : Set Δ} :
    (finalSegmentSystem Δ).mem X ↔ X.Nonempty ∧ IsUpperSet X := Iff.rfl

@[simp] theorem master_eq : (finalSegmentSystem Δ).master = Set.univ := rfl

theorem Ici_mem (a : Δ) : (finalSegmentSystem Δ).mem (Set.Ici a) :=
  ⟨⟨a, le_refl a⟩, IsUpperSet.Ici a⟩

variable [WellFoundedLT Δ]

theorem exists_Ici_of_mem {X : Set Δ} (hX : (finalSegmentSystem Δ).mem X) :
    ∃ a, X = Set.Ici a := by
  obtain ⟨a, haX, hmin⟩ := wellFounded_lt.has_min X hX.1
  refine ⟨a, Set.ext fun b => ⟨fun hb => ?_, fun hb => ?_⟩⟩
  · rcases lt_or_ge b a with hlt | hge
    · exact absurd hlt (hmin b hb)
    · exact hge
  · exact hX.2 (Set.mem_Ici.mp hb) haX

def lowerSetOf (x : (finalSegmentSystem Δ).Element) : Set Δ :=
  {a : Δ | x.mem (Set.Ici a)}

omit [WellFoundedLT Δ] in
theorem lowerSetOf_isLowerSet (x : (finalSegmentSystem Δ).Element) :
    IsLowerSet (lowerSetOf x) := by
  intro a b hba ha
  exact x.up_mem ha (Ici_mem b) (Set.Ici_subset_Ici.mpr hba)

theorem lowerSetOf_nonempty (x : (finalSegmentSystem Δ).Element) : (lowerSetOf x).
    Nonempty := by
  obtain ⟨a, ha⟩ := exists_Ici_of_mem (X := (Set.univ : Set Δ)) ⟨Set.univ_nonempty,
    isUpperSet_univ⟩
  refine ⟨a, ?_⟩
```

```
  have hu : x.mem (Set.univ) := x.master_mem
  rw [ha] at hu
  exact hu

def ofLowerSet (S : Set Δ) (hSne : S.Nonempty) (_hS : IsLowerSet S) :
    (finalSegmentSystem Δ).Element where
  mem X := (finalSegmentSystem Δ).mem X ∧ ∃ a ∈ S, Set.Ici a ⊆ X
  sub h := h.1
  master_mem := by
    obtain ⟨a, ha⟩ := hSne
    exact ⟨⟨Set.univ_nonempty, isUpperSet_univ⟩, a, ha, Set.subset_univ _⟩
  inter_mem := by
    rintro X Y ⟨hX, a, haS, haX⟩ ⟨hY, b, hbS, hbY⟩
    have hsub : Set.Ici (max a b) ⊆ X ∩ Y :=
      Set.subset_inter
        ((Set.Ici_subset_Ici.mpr (le_max_left a b)).trans haX)
        ((Set.Ici_subset_Ici.mpr (le_max_right a b)).trans hbY)
    refine ⟨(finalSegmentSystem Δ).inter_mem hX hY (Ici_mem (max a b)) hsub, max a b,
    ?_, hsub⟩
    rcases max_choice a b with h | h
    · rw [h]; exact haS
    · rw [h]; exact hbS
  up_mem := by
    rintro X Y ⟨_, a, haS, haX⟩ hY hXY
    exact ⟨hY, a, haS, haX.trans hXY⟩

omit [WellFoundedLT Δ] in
theorem lowerSetOf_ofLowerSet (S : Set Δ) (hSne : S.Nonempty) (hS : IsLowerSet S) :
    lowerSetOf (ofLowerSet S hSne hS) = S := by
  ext a
  constructor
  · rintro ⟨_, b, hbS, hba⟩
    exact hS (Set.Ici_subset_Ici.mp hba) hbS
  · intro ha
    exact ⟨Ici_mem a, a, ha, subset_rfl⟩

theorem ofLowerSet_lowerSetOf (x : (finalSegmentSystem Δ).Element) :
    ofLowerSet (lowerSetOf x) (lowerSetOf_nonempty x) (lowerSetOf_isLowerSet x) = x :=
     by
  apply NeighborhoodSystem.Element.ext
  intro X
  constructor
  · rintro ⟨hX, a, haS, haX⟩
    exact x.up_mem haS hX haX
  · intro hX
    obtain ⟨a, rfl⟩ := exists_Ici_of_mem (x.sub hX)
    exact ⟨x.sub hX, a, hX, subset_rfl⟩

def finalSegmentClassify :
    (finalSegmentSystem Δ).Element ≃o {S : Set Δ // S.Nonempty ∧ IsLowerSet S} where
  toFun x := ⟨lowerSetOf x, lowerSetOf_nonempty x, lowerSetOf_isLowerSet x⟩
  invFun S := ofLowerSet S.1 S.2.1 S.2.2
  left_inv x := ofLowerSet_lowerSetOf x
  right_inv S := Subtype.ext (lowerSetOf_ofLowerSet S.1 S.2.1 S.2.2)
  map_rel_iff' := by
    intro x y
```

```
    constructor
    · intro h X hX
      obtain ⟨a, rfl⟩ := exists_Ici_of_mem (x.sub hX)
      exact h hX
    · intro h a ha
      exact h _ ha

omit [WellFoundedLT Δ]

def topElement : (finalSegmentSystem Δ).Element where
  mem X := (finalSegmentSystem Δ).mem X
  sub h := h
  master_mem := ⟨Set.univ_nonempty, isUpperSet_univ⟩
  inter_mem := by
    rintro X Y hX hY
    rcases upperSet_subset_or hX.2 hY.2 with h | h
    · rw [Set.inter_eq_left.mpr h]; exact hX
    · rw [Set.inter_eq_right.mpr h]; exact hY
  up_mem := fun _ hY _ => hY

omit [WellFoundedLT Δ]

theorem le_topElement (x : (finalSegmentSystem Δ).Element) : x ≤ topElement :=
  fun _ hX => x.sub hX

theorem topElement_isTotal : (finalSegmentSystem Δ).IsTotal (topElement (Δ := Δ)) :=
  fun y _ => le_topElement y

theorem eq_topElement_of_isTotal {x : (finalSegmentSystem Δ).Element}
    (hx : (finalSegmentSystem Δ).IsTotal x) : x = topElement :=
  le_antisymm (le_topElement x) (hx topElement (le_topElement x))

theorem topElement_not_principal_of_noMax (hnomax : ∀ a : Δ, ∃ b, a < b) (a : Δ) :
    topElement = (finalSegmentSystem Δ).principal (Ici_mem a) := by
  intro h
  obtain ⟨b, hab⟩ := hnomax a
  have hmem : ((finalSegmentSystem Δ).principal (Ici_mem a)).mem (Set.Ici b) := by
    rw [← h]; exact Ici_mem b
  have hsub : Set.Ici a ⊆ Set.Ici b := hmem.2
  exact absurd (Set.Ici_subset_Ici.mp hsub) (not_le.mpr hab)

end DomainTheory.NeighbourhoodSystems.FinalSegment
```

4.2.17 Greedy enumeration

Greedy selection of neighbourhoods along an enumeration.

GreedyEnum.lean

```
module

import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.BranchSeq
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.IsTotal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.interUpTo
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.GreedyEnum
section

open NeighborhoodSystem
variable {α : Type*} (V : NeighborhoodSystem α) (enum : ℕ → Set α) [DecidablePred V.
    mem]

def state : ℕ → Set α × Set α
  | 0 => (enum 0, V.master ∩ enum 0)
  | (n + 1) =>
      if V.mem ((state n).2 ∩ enum (n + 1)) then
        (enum (n + 1), (state n).2 ∩ enum (n + 1))
      else
        state n

def Y (n : ℕ) : Set α := (state V enum n).1

@[simp] theorem Y_zero : Y V enum 0 = enum 0 := rfl

def acc (n : ℕ) : Set α := (state V enum n).2

@[simp] theorem acc_zero : acc V enum 0 = V.master ∩ enum 0 := rfl

abbrev cond (n : ℕ) : Prop := V.mem (acc V enum n ∩ enum (n + 1))

theorem state_succ (n : ℕ) :
    state V enum (n + 1) =
      if cond V enum n then (enum (n + 1), acc V enum n ∩ enum (n + 1)) else state V
    enum n :=
  rfl

theorem Y_succ_neg {n : ℕ} (h : ¬ cond V enum n) : Y V enum (n + 1) = Y V enum n := by
  show (state V enum (n + 1)).1 = (state V enum n).1
  rw [state_succ, ite_eq_right h]

theorem Y_succ_pos {n : ℕ} (h : cond V enum n) : Y V enum (n + 1) = enum (n + 1) := by
  show (state V enum (n + 1)).1 = enum (n + 1)
  rw [state_succ, ite_eq_left h]

theorem acc_succ_neg {n : ℕ} (h : ¬ cond V enum n) :
    acc V enum (n + 1) = acc V enum n := by
  show (state V enum (n + 1)).2 = (state V enum n).2
  rw [state_succ, ite_eq_right h]

theorem acc_succ_pos {n : ℕ} (h : cond V enum n) :
    acc V enum (n + 1) = acc V enum n ∩ enum (n + 1) := by
  show (state V enum (n + 1)).2 = acc V enum n ∩ enum (n + 1)
  rw [state_succ, ite_eq_left h]

theorem acc_eq_interUpTo : ∀ n, acc V enum n = V.interUpTo (Y V enum) (n + 1) := by
  intro n
  induction n with
  | zero =>
    rw [acc_zero, interUpTo.succ, interUpTo.zero, Y_zero]
```

```
  | succ n ih =>
    rw [interUpTo.succ, ← ih]
    by_cases h : cond V enum n
    · rw [acc_succ_pos V enum h, Y_succ_pos V enum h]
    · rw [acc_succ_neg V enum h, Y_succ_neg V enum h]
      have hsub : acc V enum n ⊆ Y V enum n := by
        rw [ih]; exact interUpTo.subset V (Y V enum) (Nat.lt_succ_self n)
      rw [Set.inter_eq_left.mpr hsub]

section
variable (henum : ∀ n, V.mem (enum n))

include henum

theorem acc_mem : ∀ n, V.mem (acc V enum n) := by
  intro n
  induction n with
  | zero =>
    rw [acc_zero, Set.inter_eq_right.mpr (V.sub_master (henum 0))]
    exact henum 0
  | succ n ih =>
    by_cases h : cond V enum n
    · rw [acc_succ_pos V enum h]; exact h
    · rw [acc_succ_neg V enum h]; exact ih

include henum

theorem mem_interUpTo_Y : ∀ n, V.mem (V.interUpTo (Y V enum) n) := by
  intro n
  cases n with
  | zero => rw [interUpTo.zero]; exact V.master_mem
  | succ n => rw [← acc_eq_interUpTo V enum]; exact acc_mem V enum henum n
end
end

end DomainTheory.NeighbourhoodSystems.GreedyEnum

section
namespace DomainTheory.NeighbourhoodSystems.GreedyEnum
section

open NeighborhoodSystem
variable {α : Type*} (V : NeighborhoodSystem α) (enum : ℕ → Set α) [DecidablePred V.
    mem]
variable (henum : ∀ n, V.mem (enum n))

include henum

def greedyElement : V.Element where
  mem X := V.mem X ∧ ∃ n, V.interUpTo (Y V enum) n ⊆ X
  sub h := h.1
  master_mem := ⟨V.master_mem, 0, by rw [interUpTo.zero]⟩
  inter_mem := by
    rintro X X' ⟨hX, n, hn⟩ ⟨hX', m, hm⟩
    have hsub : V.interUpTo (Y V enum) (max n m) ⊆ X ∩ X' :=
      Set.subset_inter
```

```
        ((V.interUpTo_antitone (Y V enum) (le_max_left n m)).trans hn)
        ((V.interUpTo_antitone (Y V enum) (le_max_right n m)).trans hm)
    exact ⟨V.inter_mem hX hX' (mem_interUpTo_Y V enum henum (max n m)) hsub, max n m,
    hsub⟩
  up_mem := by
    rintro X X' ⟨_, n, hn⟩ hX' hsub
    exact ⟨hX', n, hn.trans hsub⟩
end

open NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α) (enum : ℕ → Set α) [DecidablePred V.
    mem]

variable (henum : ∀ n, V.mem (enum n))
include henum

theorem Y_mem : ∀ n, V.mem (Y V enum n) := by
  intro n
  induction n with
  | zero => rw [Y_zero]; exact henum 0
  | succ n ih =>
    by_cases h : cond V enum n
    · rw [Y_succ_pos V enum h]; exact henum (n + 1)
    · rw [Y_succ_neg V enum h]; exact ih

theorem Y_prefix_consistent (n : ℕ) : V.Consistent (Y V enum) n :=
  ⟨V.interUpTo (Y V enum) n, mem_interUpTo_Y V enum henum n, subset_rfl⟩

theorem greedyElement_mem_Y (n : ℕ) : (greedyElement V enum henum).mem (Y V enum n) :=
  ⟨Y_mem V enum henum n, n + 1, interUpTo.subset V (Y V enum) (Nat.lt_succ_self n)⟩

theorem greedyElement_mem_acc (n : ℕ) : (greedyElement V enum henum).mem (acc V enum n
    ) := by
  refine ⟨acc_mem V enum henum n, n + 1, ?_⟩
  rw [acc_eq_interUpTo V enum]

variable (hsurj : ∀ {X : Set α}, V.mem X → ∃ n, enum n = X)
include hsurj

theorem greedyElement_isTotal : V.IsTotal (greedyElement V enum henum) := by
  intro z hz W hW
  obtain ⟨k, rfl⟩ := hsurj (z.sub hW)
  cases k with
  | zero =>
    have : Y V enum 0 = enum 0 := Y_zero V enum
    rw [← this]
    exact greedyElement_mem_Y V enum henum 0
  | succ m =>
    by_cases h : cond V enum m
    · have hYk : Y V enum (m + 1) = enum (m + 1) := Y_succ_pos V enum h
      rw [← hYk]
      exact greedyElement_mem_Y V enum henum (m + 1)
    · exfalso
      apply h
      have hacc_z : z.mem (acc V enum m) := hz _ (greedyElement_mem_acc V enum henum m
```

```
    )
      have : z.mem (acc V enum m ∩ enum (m + 1)) := z.inter_mem hacc_z hW
      exact z.sub this

omit [DecidablePred V.mem] in

theorem filters_sequence_determined (x : V.Element) :
    ∃ s : ℕ → Set α, (∀ n, V.mem (s n)) ∧ ∀ Z, x.mem Z ↔ (V.mem Z ∧ ∃ n, s n ⊆ Z) :=
     by
  classical
  refine ⟨fun i => if x.mem (enum i) then enum i else V.master, ?_, ?_⟩
  · intro n
    by_cases h : x.mem (enum n)
    · simp only [ite_eq_left h]; exact henum n
    · simp only [ite_eq_right h]; exact V.master_mem
  · intro Z
    constructor
    · intro hZ
      obtain ⟨k, hk⟩ := hsurj (x.sub hZ)
      have hxk : x.mem (enum k) := by rw [hk]; exact hZ
      refine ⟨x.sub hZ, k, ?_⟩
      show (if x.mem (enum k) then enum k else V.master) ⊆ Z
      rw [ite_eq_left hxk]; exact hk.subset
    · rintro ⟨hZmem, n, hn⟩
      have hsx : x.mem (if x.mem (enum n) then enum n else V.master) := by
        by_cases h : x.mem (enum n)
        · simp only [ite_eq_left h]; exact h
        · simp only [ite_eq_right h]; exact x.master_mem
      exact x.up_mem hsx hZmem hn

end DomainTheory.NeighbourhoodSystems.GreedyEnum
end
```

4.2.18 Positive difference systems

Positivity for neighbourhood systems and a three-token counterexample.

`PositiveDiff.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more
public import Mathlib.Data.Fintype.Basic
public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Insert
public import Mathlib.Tactic.FinCases

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.PositiveDiff

open DomainTheory.NeighbourhoodSystems

abbrev Tok := Fin 3
```

```
def a : Set Tok := {0, 1}

def b : Set Tok := {1, 2}

def master : Set Tok := Set.univ

def memSet : Set (Set Tok) := {master, a, b}

def mem (X : Set Tok) : Prop := X ∈ memSet

theorem mem_iff (X : Set Tok) : mem X ↔ X = master ∨ X = a ∨ X = b := by
  simp only [mem, memSet, Set.mem_insert_iff, Set.mem_singleton_iff]

theorem a_inter_b : a ∩ b = ({1} : Set Tok) := by
  ext x
  fin_cases x
  · simp [a, b]
  · simp [a, b]
  · simp [a, b]

theorem inter_eq (X Y : Set Tok) (hX : mem X) (hY : mem Y) :
    X ∩ Y = master ∨ X ∩ Y = a ∨ X ∩ Y = b ∨ X ∩ Y = ({1} : Set Tok) := by
  rcases (mem_iff X).mp hX with rfl | rfl | rfl
  · rcases (mem_iff Y).mp hY with rfl | rfl | rfl
    · exact Or.inl (Set.inter_self _)
    · exact Or.inr (Or.inl (by rw [master, Set.univ_inter]))
    · exact Or.inr (Or.inr (Or.inl (by rw [master, Set.univ_inter])))
  · rcases (mem_iff Y).mp hY with rfl | rfl | rfl
    · exact Or.inr (Or.inl (by rw [master, Set.inter_univ]))
    · exact Or.inr (Or.inl (Set.inter_self _))
    · exact Or.inr (Or.inr (Or.inr a_inter_b))
  · rcases (mem_iff Y).mp hY with rfl | rfl | rfl
    · exact Or.inr (Or.inr (Or.inl (by rw [master, Set.inter_univ])))
    · exact Or.inr (Or.inr (Or.inr (by rw [Set.inter_comm, a_inter_b])))
    · exact Or.inr (Or.inr (Or.inl (Set.inter_self _)))

theorem mem_a : mem a := (mem_iff _).mpr (Or.inr (Or.inl rfl))

theorem mem_b : mem b := (mem_iff _).mpr (Or.inr (Or.inr rfl))

theorem mem_master : mem master := (mem_iff _).mpr (Or.inl rfl)

theorem not_mem_sub_one {Z : Set Tok} (hZ : mem Z) (hsub : Z ⊆ ({1} : Set Tok)) :
    False := by
  rcases (mem_iff Z).mp hZ with rfl | rfl | rfl
  · exact absurd (hsub (show (0 : Tok) ∈ master by simp [master])) (by simp)
  · exact absurd (hsub (show (0 : Tok) ∈ a by simp [a])) (by simp)
  · exact absurd (hsub (show (2 : Tok) ∈ b by simp [b])) (by simp)

def notPositiveSystem : NeighborhoodSystem Tok where
  mem
  master
  master_nonempty := Set.univ_nonempty
  master_mem := mem_master
  sub_master := fun _ => Set.subset_univ _
```

```
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    rcases inter_eq X Y hX hY with h | h | h | h
    · rw [h]; exact mem_master
    · rw [h]; exact mem_a
    · rw [h]; exact mem_b
    · rw [h] at hZsub; exact (not_mem_sub_one hZ hZsub).elim

def positiveExample : NeighborhoodSystem (Fin 2) :=
  NeighborhoodSystem.ofPositive (fun X => X.Nonempty) Set.univ
    (by exact ⟨(0 : Fin 2), Set.mem_univ 0⟩)
    (by exact ⟨(0 : Fin 2), Set.mem_univ 0⟩) (by intro X _; exact Set.subset_univ X)
    (by intro X Y _ _; exact Iff.rfl)

theorem one_not_mem : ¬ mem ({1} : Set Tok) := by
  intro h
  rcases (mem_iff _).mp h with h | h | h
  · have h0 : (0 : Tok) ∈ master := by simp [master]
    rw [← h] at h0; simp at h0
  · have h0 : (0 : Tok) ∈ a := by simp [a]
    rw [← h] at h0; simp at h0
  · have h2 : (2 : Tok) ∈ b := by simp [b]
    rw [← h] at h2; simp at h2

theorem not_isPositive : ¬ notPositiveSystem.IsPositive := by
  intro h
  have hab := h (X := a) (Y := b) mem_a mem_b
  rw [a_inter_b] at hab
  exact one_not_mem (hab.mpr ⟨1, rfl⟩)

theorem positiveExample_isPositive : positiveExample.IsPositive := by
  intro X Y _ _
  exact Iff.rfl

end DomainTheory.NeighbourhoodSystems.PositiveDiff
```

4.2.19 Rational intervals

Open intervals with rational endpoints on $\mathbb{R}$.

`RatInterval.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element
public import Mathlib.Basic.Real.Basic
import Mathlib.Algebra.Order.Archimedean.Real.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.RatInterval

open NeighborhoodSystem

def ratIntervalMem (X : Set ?) : Prop :=
```

```
  X = Set.univ ∨ ∃ a b : ?, a < b ∧ X = Set.Ioo (a : ?) (b : ?)

theorem inter_mem' {X Y : Set ?} (hX : ratIntervalMem X) (hY : ratIntervalMem Y)
    (hne : (X ∩ Y).Nonempty) : ratIntervalMem (X ∩ Y) := by
  rcases hX with rfl | ⟨a, b, hab, rfl⟩
  · rw [Set.univ_inter]; exact hY
  · rcases hY with rfl | ⟨c, d, hcd, rfl⟩
    · rw [Set.inter_univ]; exact Or.inr ⟨a, b, hab, rfl⟩
    · obtain ⟨z, ⟨hza, hzb⟩, hzc, hzd⟩ := hne
      have had : a < d := by exact_mod_cast hza.trans hzd
      have hcb : c < b := by exact_mod_cast hzc.trans hzb
      have hlt : max a c < min b d := by
        simp only [max_lt_iff, lt_min_iff]
        exact ⟨⟨hab, hcb⟩, had, hcd⟩
      have hset : Set.Ioo (a : ?) b ∩ Set.Ioo (c : ?) d
          = Set.Ioo ((max a c : ?) : ?) ((min b d : ?) : ?) := by
        rw [Set.Ioo_inter_Ioo, Rat.cast_max, Rat.cast_min]
      exact Or.inr ⟨max a c, min b d, hlt, hset⟩

theorem ratIntervalMem_nonempty {X : Set ?} (hX : ratIntervalMem X) : X.Nonempty := by
  rcases hX with rfl | ⟨a, b, hab, rfl⟩
  · exact ⟨0, Set.mem_univ 0⟩
  · rw [Set.nonempty_Ioo]; exact_mod_cast hab

def ratIntervalSystem : NeighborhoodSystem ? where
  mem := ratIntervalMem
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    exact inter_mem' hX hY ((ratIntervalMem_nonempty hZ).mono hZsub)
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem ratIntervalSystem_mem {X : Set ?} :
    ratIntervalSystem.mem X ↔ ratIntervalMem X := Iff.rfl

def filterAt (t : ?) : ratIntervalSystem.Element where
  mem X := ratIntervalMem X ∧ t ∈ X
  sub h := h.1
  master_mem := ⟨Or.inl rfl, Set.mem_univ t⟩
  inter_mem := by
    rintro X Y ⟨hX, htX⟩ ⟨hY, htY⟩
    exact ⟨inter_mem' hX hY ⟨t, htX, htY⟩, htX, htY⟩
  up_mem := by
    rintro X Y ⟨_, htX⟩ hY hXY
    exact ⟨hY, hXY htX⟩

@[simp] theorem mem_filterAt {t : ?} {X : Set ?} :
    (filterAt t).mem X ↔ ratIntervalMem X ∧ t ∈ X := Iff.rfl

theorem filterAt_injective {s t : ?} (h : filterAt s = filterAt t) : s = t := by
  by_contra hst
  rcases lt_or_gt_of_ne hst with hlt | hlt
  · obtain ⟨a, ha1, ha2⟩ := exists_rat_btwn hlt
    obtain ⟨b, hb⟩ := exists_rat_gt t
```

```
    have hint : ratIntervalMem (Set.Ioo (a : ?) (b : ?)) :=
      Or.inr ⟨a, b, by exact_mod_cast ha2.trans hb, rfl⟩
    have htmem : (filterAt t).mem (Set.Ioo (a : ?) (b : ?)) := ⟨hint, ha2, hb⟩
    rw [← h] at htmem
    exact absurd htmem.2.1 (not_lt.mpr ha1.le)
  · obtain ⟨a, ha1, ha2⟩ := exists_rat_btwn hlt
    obtain ⟨b, hb⟩ := exists_rat_gt s
    have hint : ratIntervalMem (Set.Ioo (a : ?) (b : ?)) :=
      Or.inr ⟨a, b, by exact_mod_cast ha2.trans hb, rfl⟩
    have hsmem : (filterAt s).mem (Set.Ioo (a : ?) (b : ?)) := ⟨hint, ha2, hb⟩
    rw [h] at hsmem
    exact absurd hsmem.2.1 (not_lt.mpr ha1.le)

end DomainTheory.NeighbourhoodSystems.RatInterval
```

4.2.20 Cofinite neighbourhoods

The cofinite neighbourhood system on $\mathbb{N}$.

Cofinite.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.IsTotal
public import Mathlib.Algebra.Group.End
public import Mathlib.Basic.Finite.Defs

public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Finite.Basic
public import Mathlib.Order.BooleanAlgebra.Set
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Cofinite

open NeighborhoodSystem

def cofiniteSystem : NeighborhoodSystem ℕ where
  mem X := X?.Finite
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := by rw [Set.compl_univ]; exact Set.finite_empty
  inter_mem := by
    intro X Y _ hX hY _ _
    rw [Set.compl_inter]
    exact hX.union hY
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem cofiniteSystem_mem {X : Set ℕ} : cofiniteSystem.mem X ↔ X?.Finite :=
    Iff.rfl

def ofExcluded (E : Set ℕ) : cofiniteSystem.Element where
  mem Y := Y?.Finite ∧ Y? ⊆ E
  sub h := h.1
  master_mem := by
    show (Set.univ : Set ℕ)?.Finite ∧ (Set.univ : Set ℕ)? ⊆ E
```

```
    rw [Set.compl_univ]
    exact ⟨Set.finite_empty, Set.empty_subset E⟩
  inter_mem := by
    rintro Y1 Y2 ⟨hf1, hs1⟩ ⟨hf2, hs2⟩
    rw [Set.compl_inter]
    exact ⟨hf1.union hf2, Set.union_subset hs1 hs2⟩
  up_mem := by
    rintro Y1 Y2 ⟨_, hs1⟩ hY2 hsub
    exact ⟨hY2, (Set.compl_subset_compl.mpr hsub).trans hs1⟩

@[simp] theorem mem_ofExcluded {E Y : Set ℕ} :
    (ofExcluded E).mem Y ↔ Y?.Finite ∧ Y? ⊆ E := Iff.rfl

def toExcluded (x : cofiniteSystem.Element) : Set ℕ := {n | x.mem ({n}?)}

@[simp] theorem mem_toExcluded {x : cofiniteSystem.Element} {n : ℕ} :
    n ∈ toExcluded x ↔ x.mem ({n}?) := Iff.rfl

def fullSystem : NeighborhoodSystem ℕ where
  mem _ := True
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := trivial
  inter_mem := fun _ _ _ _ => trivial
  sub_master := fun _ => Set.subset_univ _

theorem mem_compl_of_finite (x : cofiniteSystem.Element) {F : Set ℕ} (hF : F.Finite) :
    (∀ n ∈ F, x.mem ({n}?)) → x.mem (F?) := by
  induction F, hF using Set.Finite.induction_on with
  | empty => intro _; rw [Set.compl_empty]; exact x.master_mem
  | @insert a s _ _ ih =>
    intro h
    rw [Set.insert_eq, Set.compl_union]
    exact x.inter_mem (h a (Set.mem_insert a s))
      (ih (fun n hn => h n (Set.mem_insert_of_mem a hn)))

theorem mem_compl_singleton (n : ℕ) : cofiniteSystem.mem (({n} : Set ℕ)?) := by
  rw [cofiniteSystem_mem, compl_compl]
  exact Set.finite_singleton n

theorem subset_compl_singleton {Y : Set ℕ} {n : ℕ} (hn : n ∈ Y?) : Y ⊆ ({n} : Set ℕ)?
    := by
  intro m hm
  rw [Set.mem_compl_iff, Set.mem_singleton_iff]
  rintro rfl
  exact hn hm

theorem ofExcluded_toExcluded (x : cofiniteSystem.Element) :
    ofExcluded (toExcluded x) = x := by
  apply Element.ext
  intro Y
  rw [mem_ofExcluded]
  constructor
  · rintro ⟨hfin, hsub⟩
    have h := mem_compl_of_finite x hfin (fun n hn => mem_toExcluded.mp (hsub hn))
    rwa [compl_compl] at h
```

```
  · intro hxY
    refine ⟨x.sub hxY, ?_⟩
    intro n hn
    rw [mem_toExcluded]
    exact x.up_mem hxY (mem_compl_singleton n) (subset_compl_singleton hn)

theorem toExcluded_ofExcluded (E : Set ℕ) : toExcluded (ofExcluded E) = E := by
  ext n
  rw [mem_toExcluded, mem_ofExcluded]
  simp [compl_compl, Set.finite_singleton, Set.singleton_subset_iff]

def cofiniteIso : cofiniteSystem.Element ≃o Set ℕ where
  toFun := toExcluded
  invFun := ofExcluded
  left_inv := ofExcluded_toExcluded
  right_inv := toExcluded_ofExcluded
  map_rel_iff' := by
    intro x x'
    constructor
    · intro h Y hY
      have hfin : Y?.Finite := x.sub hY
      have hsub : Y? ⊆ toExcluded x' := by
        intro n hn
        have hx : x.mem (({n} : Set ℕ)?) :=
          x.up_mem hY (mem_compl_singleton n) (subset_compl_singleton hn)
        exact h (mem_toExcluded.mpr hx)
      have h2 := mem_compl_of_finite x' hfin (fun n hn => mem_toExcluded.mp (hsub hn))
      rwa [compl_compl] at h2
    · intro h n hn
      exact mem_toExcluded.mpr (h _ (mem_toExcluded.mp hn))

theorem le_ofExcluded_univ (x : cofiniteSystem.Element) :
    x ≤ ofExcluded Set.univ := by
  intro Y hY
  exact ⟨x.sub hY, Set.subset_univ _⟩

theorem ofExcluded_univ_isTotal : cofiniteSystem.IsTotal (ofExcluded Set.univ) :=
  fun y _ => le_ofExcluded_univ y

end DomainTheory.NeighbourhoodSystems.Cofinite
```

4.2.21 Flat and stem systems

The flat and stem neighbourhood systems on $\mathbb{N}$.

FlatStem.lean

```
module

import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_injective
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal_le_iff
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot
```

```
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint

import Mathlib.Tactic.NormNum
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.FlatStem

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

def flatMem (X : Set ℕ) : Prop := X = Set.univ ∨ ∃ n, X = {n}

theorem singleton_disjoint {n m : ℕ} (h : n = m) : ({n} : Set ℕ) ∩ {m} = ∅ := by
  ext k
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false]
  rintro ⟨rfl, h2⟩
  exact h h2

theorem flatNOD : NestedOrDisjoint flatMem := by
  rintro X Y (rfl | ⟨n, rfl⟩) (rfl | ⟨m, rfl⟩)
  · exact Or.inl (subset_refl _)
  · exact Or.inr (Or.inl (Set.subset_univ _))
  · exact Or.inl (Set.subset_univ _)
  · rcases eq_or_ne n m with rfl | h
    · exact Or.inl (subset_refl _)
    · exact Or.inr (Or.inr (singleton_disjoint h))

def flat : NeighborhoodSystem ℕ :=
  ofNestedOrDisjoint flatMem Set.univ Set.univ_nonempty
    (Or.inl rfl) flatNOD (fun _ => Set.subset_univ _)

@[simp] theorem flat_mem {X : Set ℕ} : flat.mem X ↔ X = Set.univ ∨ ∃ n, X = {n} :=
    Iff.rfl

@[simp] theorem flat_master : flat.master = Set.univ := rfl

def stemMem (X : Set ℕ) : Prop := X = Set.univ ∨ X = {0, 1} ∨ ∃ n, X = {n}

theorem stemNOD : NestedOrDisjoint stemMem := by
  have pair_single : ∀ m : ℕ, ({m} : Set ℕ) ⊆ {0, 1} ∨ ({0, 1} : Set ℕ) ∩ {m} = ∅ :=
    by
    intro m
    by_cases h : m = 0 ∨ m = 1
    · refine Or.inl ?_
      rcases h with rfl | rfl <;> intro k hk <;> simp_all
    · refine Or.inr ?_
      ext k
      simp only [Set.mem_inter_iff, Set.mem_insert_iff, Set.mem_singleton_iff,
        Set.mem_empty_iff_false, iff_false, not_and]
      rintro (rfl | rfl) rfl <;> simp_all
  rintro X Y (rfl | rfl | ⟨n, rfl⟩) (rfl | rfl | ⟨m, rfl⟩)
  · exact Or.inl (subset_refl _)
  · exact Or.inr (Or.inl (Set.subset_univ _))
  · exact Or.inr (Or.inl (Set.subset_univ _))
  · exact Or.inl (Set.subset_univ _)
```

```
  · exact Or.inl (subset_refl _)
  · rcases pair_single m with h | h
    · exact Or.inr (Or.inl h)
    · exact Or.inr (Or.inr h)
  · exact Or.inl (Set.subset_univ _)
  · rcases pair_single n with h | h
    · exact Or.inl h
    · exact Or.inr (Or.inr (by rw [Set.inter_comm]; exact h))
  · rcases eq_or_ne n m with rfl | h
    · exact Or.inl (subset_refl _)
    · exact Or.inr (Or.inr (singleton_disjoint h))

def stem : NeighborhoodSystem ℕ :=
  ofNestedOrDisjoint stemMem Set.univ Set.univ_nonempty
    (Or.inl rfl) stemNOD (fun _ => Set.subset_univ _)

@[simp] theorem stem_mem {X : Set ℕ} :
    stem.mem X ↔ X = Set.univ ∨ X = {0, 1} ∨ ∃ n, X = {n} := Iff.rfl

theorem flat_empty_not_mem : ¬ flat.mem (∅ : Set ℕ) := by
  rintro (h | ⟨n, h⟩)
  · exact Set.empty_ne_univ h
  · exact absurd h.symm (Set.singleton_ne_empty n)

theorem flat_mem_singleton (n : ℕ) : flat.mem {n} := Or.inr ⟨n, rfl⟩

theorem flat_classify (x : flat.Element) :
    x = flat.bot ∨ ∃ n, x = flat.principal (flat_mem_singleton n) := by
  classical
  by_cases h : ∃ n, x.mem {n}
  · obtain ⟨n, hn⟩ := h
    refine Or.inr ⟨n, ?_⟩
    have huniq : ∀ m, x.mem {m} → m = n := by
      intro m hm
      by_contra hmn
      have hi : x.mem (({m} : Set ℕ) ∩ {n}) := x.inter_mem hm hn
      rw [singleton_disjoint hmn] at hi
      exact flat_empty_not_mem (x.sub hi)
    apply Element.ext
    intro W
    constructor
    · intro hx
      rcases (x.sub hx) with rfl | ⟨k, rfl⟩
      · exact ⟨Or.inl rfl, Set.subset_univ _⟩
      · obtain rfl := huniq k hx
        exact ⟨flat_mem_singleton k, subset_rfl⟩
    · rintro ⟨hWmem, hsub⟩
      exact x.up_mem hn hWmem hsub
  · refine Or.inl ?_
    apply Element.ext
    intro W
    rw [flat.mem_bot]
    constructor
    · intro hx
      rcases (x.sub hx) with rfl | ⟨k, rfl⟩
      · rfl
```

```
      · exact absurd ⟨k, hx⟩ h
    · rintro rfl
      exact x.master_mem

theorem flat_all_finite (x : flat.Element) : ∃ (X : Set ℕ) (h : flat.mem X), x = flat.
    principal h := by
  rcases flat_classify x with rfl | ⟨n, rfl⟩
  · exact ⟨flat.master, flat.master_mem, rfl⟩
  · exact ⟨{n}, flat_mem_singleton n, rfl⟩

theorem flat_atom_maximal (n : ℕ) (y : flat.Element)
    (hle : flat.principal (flat_mem_singleton n) ≤ y) :
    y = flat.principal (flat_mem_singleton n) := by
  rcases flat_classify y with rfl | ⟨m, rfl⟩
  · exfalso
    have hmem : flat.bot.mem {n} := hle {n} ⟨flat_mem_singleton n, subset_rfl⟩
    rw [flat.mem_bot, flat_master] at hmem
    have : (n + 1) ∈ ({n} : Set ℕ) := by rw [hmem]; exact Set.mem_univ _
    simp at this
  · have hsub : ({m} : Set ℕ) ⊆ {n} :=
      (flat.principal_le_iff (flat_mem_singleton n) (flat_mem_singleton m)).mp hle
    obtain rfl : m = n := Set.mem_singleton_iff.mp (Set.singleton_subset_iff.mp hsub)
    rfl

theorem flat_infinite : Function.Injective (fun n => flat.principal (
    flat_mem_singleton n)) := by
  intro n m h
  have := flat.principal_injective (flat_mem_singleton n) (flat_mem_singleton m) h
  exact Set.singleton_injective this

theorem flat_no_three_chain : ¬ ∃ a b c : flat.Element, a < b ∧ b < c := by
  rintro ⟨a, b, c, hab, hbc⟩
  rcases flat_classify b with rfl | ⟨n, rfl⟩
  · exact absurd (le_antisymm hab.le (flat.bot_le a)) (ne_of_lt hab)
  · exact (ne_of_lt hbc) (flat_atom_maximal n c hbc.le).symm

theorem flat_no_infinite_chain : ¬ ∃ f : ℕ → flat.Element, StrictMono f := by
  rintro ⟨f, hf⟩
  exact flat_no_three_chain
    ⟨f 0, f 1, f 2, hf (by norm_num), hf (by norm_num)⟩

theorem stem_mem_pair : stem.mem {0, 1} := Or.inr (Or.inl rfl)

theorem stem_mem_univ : stem.mem Set.univ := Or.inl rfl

theorem stem_mem_zero : stem.mem {0} := Or.inr (Or.inr ⟨0, rfl⟩)

theorem stem_principal_lt {X Y : Set ℕ} (hX : stem.mem X) (hY : stem.mem Y)
    (hYX : Y ⊆ X) (hne : X ≠ Y) : stem.principal hX < stem.principal hY := by
  rw [lt_iff_le_and_ne]
  refine ⟨(stem.principal_le_iff hX hY).mpr hYX, ?_⟩
  intro he
  exact hne (stem.principal_injective hX hY he)

theorem stem_three_chain : ∃ a b c : stem.Element, a < b ∧ b < c := by
  refine ⟨stem.principal stem_mem_univ, stem.principal stem_mem_pair,
```

```
    stem.principal stem_mem_zero, ?_, ?_⟩
  · refine stem_principal_lt stem_mem_univ stem_mem_pair (Set.subset_univ _) ?_
    intro h
    have : (2 : ℕ) ∈ ({0, 1} : Set ℕ) := h ▸ Set.mem_univ 2
    simp at this
  · refine stem_principal_lt stem_mem_pair stem_mem_zero
      (Set.singleton_subset_iff.mpr (by simp)) ?_
    intro h
    have : (1 : ℕ) ∈ ({0} : Set ℕ) := h ▸ (by simp : (1 : ℕ) ∈ ({0, 1} : Set ℕ))
    simp at this

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

theorem not_isomorphic : ¬ (flat ≅? stem) := by
  rintro ⟨e⟩
  obtain ⟨a, b, c, hab, hbc⟩ := stem_three_chain
  exact flat_no_three_chain
    ⟨e.symm a, e.symm b, e.symm c, e.symm.lt_iff_lt.mpr hab, e.symm.lt_iff_lt.mpr hbc⟩

end DomainTheory.NeighbourhoodSystems.FlatStem
```

4.2.22 Finite nonempty neighbourhoods

The finite-nonempty neighbourhood system on $\mathbb{N}$.

FinNonempty.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.IsTotal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot

public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Finite.Basic
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.FinNonempty

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

def mem (X : Set ℕ) : Prop := X = Set.univ ∨ (X.Finite ∧ X.Nonempty)

theorem nonempty_of_mem {X : Set ℕ} (h : mem X) : X.Nonempty := by
  rcases h with rfl | h
  · exact Set.univ_nonempty
  · exact h.2

def neighborhoodSystem : NeighborhoodSystem ℕ where
  mem
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    have hne : (X ∩ Y).Nonempty := (nonempty_of_mem hZ).mono hZsub
```

```
    rcases hX with hX | hX
    · rw [hX, Set.univ_inter]; exact hY
    · rcases hY with hY | hY
      · rw [hY, Set.inter_univ]; exact Or.inr hX
      · exact Or.inr ⟨hX.1.inter_of_left Y, hne⟩
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem ns_mem {X : Set ℕ} : neighborhoodSystem.mem X ↔ mem X := Iff.rfl

def bot : neighborhoodSystem.Element := neighborhoodSystem.bot

def fin {X : Set ℕ} (h : neighborhoodSystem.mem X) : neighborhoodSystem.Element :=
  neighborhoodSystem.principal h

theorem empty_not_mem : ¬ mem (∅ : Set ℕ) := by
  rintro (h | ⟨_, h⟩)
  · exact Set.empty_ne_univ h
  · exact Set.not_nonempty_empty h

theorem mem_singleton (n : ℕ) : mem {n} :=
  Or.inr ⟨Set.finite_singleton n, Set.singleton_nonempty n⟩

theorem singleton_isTotal (n : ℕ) :
    neighborhoodSystem.IsTotal (neighborhoodSystem.principal (mem_singleton n)) := by
  intro y hy W hW
  have hn : y.mem {n} := hy {n} ⟨mem_singleton n, subset_rfl⟩
  refine ⟨y.sub hW, ?_⟩
  by_contra hc
  rw [Set.singleton_subset_iff] at hc
  have hempty : ({n} : Set ℕ) ∩ W = (∅ : Set ℕ) := by
    ext k
    simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false]
    rintro ⟨rfl, hk⟩
    exact hc hk
  have hi := y.inter_mem hn hW
  rw [hempty] at hi
  exact empty_not_mem (y.sub hi)

end DomainTheory.NeighbourhoodSystems.FinNonempty
```

4.2.23 Branch sequences

Infinite branches through the binary-sequence domain.

BranchSeq.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.iUnion
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.IsTotal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    sigmaBot_le_iff
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ExampleB
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.BranchSeq

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem BitDom

def prefixSeq (p : ℕ → Bool) : ℕ → Str
  | 0 => []
  | (n + 1) => prefixSeq p n ++ [p n]

@[simp] theorem prefixSeq_length (p : ℕ → Bool) (n : ℕ) : (prefixSeq p n).length = n
    := by
  induction n with
  | zero => rfl
  | succ k ih => simp [prefixSeq, ih]

def branchSeq (p : ℕ → Bool) (n : ℕ) : B.Element := sigmaBot (prefixSeq p n)

theorem prefixSeq_prefix_succ (p : ℕ → Bool) (n : ℕ) :
    prefixSeq p n <+: prefixSeq p (n + 1) := ⟨[p n], rfl⟩

theorem prefixSeq_mono (p : ℕ → Bool) {n m : ℕ} (h : n ≤ m) :
    prefixSeq p n <+: prefixSeq p m := by
  induction m, h using Nat.le_induction with
  | base => exact List.prefix_rfl
  | succ k _ ih => exact ih.trans (prefixSeq_prefix_succ p k)

theorem branchSeq_mono (p : ℕ → Bool) : Monotone (branchSeq p) := by
  intro n m h
  exact (sigmaBot_le_iff (prefixSeq p n) (prefixSeq p m)).mpr (prefixSeq_mono p h)

def branch (p : ℕ → Bool) : B.Element := B.iUnion (branchSeq p) (branchSeq_mono p)

theorem branch_mem_iff (p : ℕ → Bool) {σ : Str} :
    (branch p).mem (cone σ) ↔ ∃ n, σ <+: prefixSeq p n := by
  constructor
  · intro h
    obtain ⟨n, hn⟩ := (B.mem_iUnion (branchSeq p) (branchSeq_mono p)).mp h
    exact ⟨n, cone.subset_cone.mp hn.2⟩
  · rintro ⟨n, hn⟩
    exact (B.mem_iUnion (branchSeq p) (branchSeq_mono p)).mpr
      ⟨n, memB_cone σ, cone.subset_cone.mpr hn⟩

theorem prefix_comparable_of_mem {σ τ : Str} (y : B.Element)
    (hσ : y.mem (cone σ)) (hτ : y.mem (cone τ)) : σ <+: τ ∨ τ <+: σ := by
  have hi : y.mem (cone σ ∩ cone τ) := y.inter_mem hσ hτ
  obtain ⟨?, h?⟩ := y.sub hi
  rcases cone.trichotomy σ τ with h | h | h
  · exact Or.inr (cone.subset_cone.mp h)
  · exact Or.inl (cone.subset_cone.mp h)
  · exfalso
    have hmem : ? ∈ cone ? := cone.mem.mpr List.prefix_rfl
    rw [← h?, h] at hmem
    simp at hmem

theorem branchSeq_le_branch (p : ℕ → Bool) (n : ℕ) : branchSeq p n ≤ branch p :=
```

```
  B.le_iUnion (branchSeq p) (branchSeq_mono p) n

theorem branch_isTotal (p : ℕ → Bool) : B.IsTotal (branch p) := by
  intro y hy X hX
  obtain ⟨σ, rfl⟩ := y.sub hX
  have hk : y.mem (cone (prefixSeq p σ.length)) :=
    hy _ ((branch_mem_iff p).mpr ⟨σ.length, List.prefix_rfl⟩)
  rcases prefix_comparable_of_mem y hX hk with h | h
  · exact (branch_mem_iff p).mpr ⟨σ.length, h⟩
  · have hlen : (prefixSeq p σ.length).length = σ.length := prefixSeq_length p σ.length
    have hσ : prefixSeq p σ.length = σ := h.eq_of_length hlen
    exact (branch_mem_iff p).mpr ⟨σ.length, by rw [hσ]; exact List.prefix_rfl⟩

end DomainTheory.NeighbourhoodSystems.BranchSeq
```

4.2.24 Final-segment naturals

The final-segment neighbourhood system on $\mathbb{N}$.

TailNat.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.IsTotal
import Mathlib.Data.Nat.Find

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.TailNat

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

def tail (n : ℕ) : Set ℕ := {m | n ≤ m}

@[simp] theorem mem_tail {n m : ℕ} : m ∈ tail n ↔ n ≤ m := Iff.rfl

def mem (X : Set ℕ) : Prop := ∃ n, X = tail n

theorem tail_inter (n m : ℕ) : tail n ∩ tail m = tail (max n m) := by
  ext k
  simp only [Set.mem_inter_iff, mem_tail]
  omega

theorem tail_subset_iff {n m : ℕ} : tail n ⊆ tail m ↔ m ≤ n := by
  constructor
  · intro h
    exact h (show n ∈ tail n from le_refl n)
  · intro hmn k hk
    exact le_trans hmn hk

theorem nestedOrDisjoint : NestedOrDisjoint mem := by
```

```
  rintro X Y ⟨n, rfl⟩ ⟨m, rfl⟩
  rcases le_total n m with h | h
  · exact Or.inr (Or.inl (tail_subset_iff.mpr h))
  · exact Or.inl (tail_subset_iff.mpr h)

def neighborhoodSystem : NeighborhoodSystem ℕ :=
  ofNestedOrDisjoint mem (tail 0) ⟨0, Nat.zero_le 0⟩ ⟨0, rfl⟩ nestedOrDisjoint
    (fun {X} hX => by obtain ⟨n, rfl⟩ := hX; exact tail_subset_iff.mpr (Nat.zero_le n))

@[simp] theorem ns_mem {X : Set ℕ} : neighborhoodSystem.mem X ↔ ∃ n, X = tail n := Iff
    .rfl

theorem mem_tail_nbhd (n : ℕ) : neighborhoodSystem.mem (tail n) := ⟨n, rfl⟩

def fin (n : ℕ) : neighborhoodSystem.Element := neighborhoodSystem.principal (
    mem_tail_nbhd n)

def top : neighborhoodSystem.Element where
  mem X := neighborhoodSystem.mem X
  sub h := h
  master_mem := neighborhoodSystem.master_mem
  inter_mem := by
    rintro X Y ⟨n, rfl⟩ ⟨m, rfl⟩
    exact ⟨max n m, tail_inter n m⟩
  up_mem _ hY _ := hY

@[simp] theorem mem_top {X : Set ℕ} : top.mem X ↔ neighborhoodSystem.mem X := Iff.rfl

theorem element_eq (x : neighborhoodSystem.Element) :
    (∃ n, x = fin n) ∨ x = top := by
  classical
  by_cases h : ∀ n, x.mem (tail n)
  · refine Or.inr ?_
    apply Element.ext
    intro X
    constructor
    · intro hx; exact x.sub hx
    · rintro ⟨n, rfl⟩; exact h n
  · refine Or.inl ?_
    have h' : ∃ n, ¬ x.mem (tail n) := not_forall.mp h
    have hpos : ∀ n, ¬ x.mem (tail n) → 0 < n := by
      intro n hn
      rcases Nat.eq_zero_or_pos n with rfl | hp
      · exact absurd x.master_mem hn
      · exact hp
    have hspec : ¬ x.mem (tail (Nat.find h')) := Nat.find_spec h'
    have hmin : ∀ j, j < Nat.find h' → x.mem (tail j) := by
      intro j hj
      by_contra hc
      exact Nat.find_min h' hj hc
    have hn₀ : 0 < Nat.find h' := hpos _ hspec
    refine ⟨Nat.find h' - 1, ?_⟩
    apply Element.ext
    intro X
    have key : ∀ j, x.mem (tail j) ↔ j ≤ Nat.find h' - 1 := by
      intro j
```

```
      constructor
      · intro hj
        by_contra hc
        have hle : Nat.find h' ≤ j := by omega
        exact hspec (x.up_mem hj (mem_tail_nbhd (Nat.find h')) (tail_subset_iff.mpr
    hle))
      · intro hj
        exact hmin j (by omega)
    constructor
    · intro hx
      obtain ⟨j, rfl⟩ := x.sub hx
      exact ⟨mem_tail_nbhd j, tail_subset_iff.mpr ((key j).mp hx)⟩
    · rintro ⟨hX, hsub⟩
      obtain ⟨j, rfl⟩ := hX
      exact (key j).mpr (tail_subset_iff.mp hsub)

theorem fin_le_iff (n m : ℕ) : fin n ≤ fin m ↔ n ≤ m := by
  rw [fin, fin, neighborhoodSystem.principal_le_iff, tail_subset_iff]

theorem fin_strictMono : StrictMono fin := by
  intro n m hnm
  refine lt_of_le_of_ne ((fin_le_iff n m).mpr hnm.le) ?_
  intro heq
  have : m ≤ n := (fin_le_iff m n).mp (le_of_eq heq.symm)
  exact absurd (le_antisymm hnm.le this) (Nat.ne_of_lt hnm)

theorem le_top (x : neighborhoodSystem.Element) : x ≤ top :=
  fun _ hX => x.sub hX

theorem top_isTotal : neighborhoodSystem.IsTotal top :=
  fun y _ => le_top y

theorem isTotal_iff_top (x : neighborhoodSystem.Element) :
    neighborhoodSystem.IsTotal x ↔ x = top := by
  constructor
  · intro h
    exact le_antisymm (le_top x) (h top (le_top x))
  · rintro rfl; exact top_isTotal

end DomainTheory.NeighbourhoodSystems.TailNat
```

4.2.25 Example 1.4 — finite binary tree

EXAMPLE 1.4. Let $\Delta = \{\Lambda, 0, 1, 00, 01, 10, 11\}$ and let $\mathcal{D} = \{\Delta, \{0, 00, 01\}, \{1, 10, 11\}, \{00\}, \{01\}, \{10\}, \{11\}\}$.
Or more understandably in pictures:

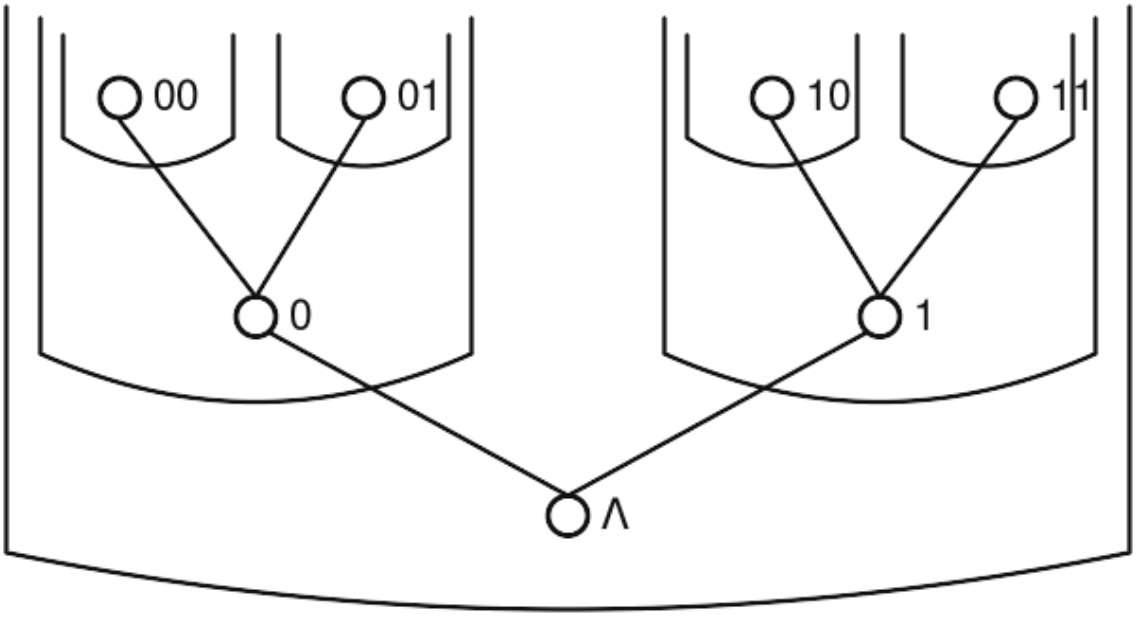


Figure 42: Binary tree of tokens with neighbourhood U-curves

The tokens are finite sequences of 0's and 1's (up to length 2) with Λ the empty sequence; they form — in the picture — the binary tree with the sequences as the nodes. The neighbourhoods are the subtrees of all nodes above a given node. Obviously this can be generalized to sequences of any length (and to trees less regular than the binary tree). The total elements of the example correspond to the top nodes 00, 01, 10, 11 and the lower nodes to the partial elements. When we are not at a top node we have only partially determined a sequence, and the branching indicates that we have some choice as to how the sequence can be extended. □

The development is with the binary-tree tokens peer.

Theorem (`example_1_4`). There is a neighbourhood system whose neighbourhoods are the listed cones and leaves of the finite binary tree.

Proof. By the binary-tree neighbourhood system. ■

`Example_1_4.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.BinTree

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open BinTree

theorem example_1_4 :
    ∃ (V : NeighborhoodSystem Token),
      V.master = (Set.univ : Set Token) ∧
        (∀ X : Set Token,
```

```
          V.mem X ↔
            X = Set.univ ∨ X = left ∨ X = right ∨
              X = leaf00 ∨ X = leaf01 ∨ X = leaf10 ∨ X = leaf11) :=
  ⟨neighborhoodSystem, rfl, fun _ => Iff.rfl⟩
```

4.2.26 Are two neighbourhoods nested or disjoint?

It should be noted that, in these three examples, the reason that we have a neighbourhood system is a simple consequence of a very special circumstance: in these systems two neighbourhoods are either disjoint or one is included in the other. This arrangement of neighbourhoods is by no means necessary.

Definition (`NestedOrDisjoint`). A family of subsets is *nested or disjoint* when any two members X and Y satisfy $X \subseteq Y$, or $Y \subseteq X$, or else $X \cap Y = \emptyset$.

Definition (`NeighborhoodSystem.ofNestedOrDisjoint`). From a nested-or-disjoint family containing a nonempty master set Δ, with every member contained in Δ, one obtains a neighbourhood system with that membership and master.

Proof. The master hypotheses are given. For the intersection axiom: if X and Y are members and some member Z lies in $X \cap Y$, then X and Y cannot be disjoint, so one includes the other and $X \cap Y$ equals the smaller of the two, hence is again a member. ■

`NestedOrDisjoint.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

/-- Neighbourhoods are nested or pairwise disjoint. -/
def NestedOrDisjoint {α : Type*} (mem : Set α → Prop) : Prop :=
  ∀ {{X Y : Set α}}, mem X → mem Y → X ⊆ Y ∨ Y ⊆ X ∨ X ∩ Y = ∅

def NeighborhoodSystem.ofNestedOrDisjoint {α : Type*} (mem : Set α → Prop) (master :
    Set α)
    (master_nonempty : master.Nonempty) (master_mem : mem master) (hnd :
    NestedOrDisjoint mem)
    (sub_master : ∀ {X : Set α}, mem X → X ⊆ master) : NeighborhoodSystem α where
  mem
  master
  master_nonempty
  master_mem
  sub_master
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    rcases hnd hX hY with h | h | h
    · rwa [Set.inter_eq_left.mpr h]
    · rwa [Set.inter_eq_right.mpr h]
    · rw [h]
      rw [h] at hZsub
```

```
      rwa [← Set.subset_empty_iff.mp hZsub]

end DomainTheory.NeighbourhoodSystems
```

4.2.27 Four-token powerset

The powerset-of-four-tokens neighbourhood system (nonempty subsets).

Definition (`Token`). The token set is a four-element type, standing for $\Delta = \{0, 1, 2, 3\}$.

Definition (`P4Mem`). A subset belongs to the family precisely when it is nonempty.

Theorem (`P4_inter_mem`). If Z is nonempty and $Z \subseteq X \cap Y$, then $X \cap Y$ is nonempty.

Proof. Nonemptiness passes along the inclusion. ■

Definition (`master`). The master set is the full set of tokens.

Theorem (`P4_master_mem`), (`P4_master_nonempty`). The master is nonempty (hence a neighbourhood).

Proof. The full token set is nonempty. ■

Theorem (`P4_sub_master`). Every neighbourhood is contained in the master.

Proof. Every set of tokens lies in the full token set. ■

Definition (`neighborhoodSystem`). The neighbourhood system on the four-token carrier whose members are exactly the nonempty subsets, with this master set.

Proof. The master is nonempty and a neighbourhood; every neighbourhood lies in the master; if two neighbourhoods meet in a set containing a third neighbourhood, their intersection is nonempty by the preceding theorem, hence again a neighbourhood. ■

`P4.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import Mathlib.Data.Fintype.Powerset

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.P4

open NeighborhoodSystem

abbrev Token := Fin 4

def P4Mem (X : Set Token) : Prop := X.Nonempty

theorem P4_inter_mem {X Y Z : Set Token} (_hX : P4Mem X) (_hY : P4Mem Y)
    (hZ : P4Mem Z) (hZsub : Z ⊆ X ∩ Y) : P4Mem (X ∩ Y) :=
  hZ.mono hZsub

def master : Set Token := Set.univ

theorem P4_master_mem : P4Mem master := by
  rw [master]; exact Set.univ_nonempty

theorem P4_master_nonempty : master.Nonempty := by
```

```
  exact ⟨0, Set.mem_univ 0⟩

theorem P4_sub_master {X : Set Token} (_h : P4Mem X) : X ⊆ master :=
  Set.subset_univ _

def neighborhoodSystem : NeighborhoodSystem Token where
  mem := P4Mem
  master
  master_nonempty := P4_master_nonempty
  master_mem := P4_master_mem
  inter_mem := P4_inter_mem
  sub_master := P4_sub_master

end DomainTheory.NeighbourhoodSystems.P4
```

4.2.28 Example 1.5 — all nonempty subsets

EXAMPLE 1.5. Let $\Delta = \{0, 1, 2, 3\}$ and let $\mathcal{D}$ be the family of all non-empty subsets of Δ.

The development is with the four-token powerset peer.

Theorem (`example_1_5`). There is a neighbourhood system on four tokens whose neighbourhoods are exactly the nonempty subsets.

Proof. By the four-token neighbourhood system. ■

`Example_1_5.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.P4

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open P4

theorem example_1_5 :
    ∃ (V : NeighborhoodSystem Token),
      V.master = (Set.univ : Set Token) ∧
        (∀ X : Set Token, V.mem X ↔ X.Nonempty) :=
  ⟨neighborhoodSystem, rfl, fun _ => Iff.rfl⟩
```

4.2.29 Consistent iff nonempty

This system is a direct generalization of Example 1.2., which was special owing to the small number of tokens. (The other examples were special by virtue of the choice of neighbourhoods.) The verification that the present $\mathcal{D}$ is a neighbourhood system rests on nothing more than the remark that sets are consistent in $\mathcal{D}$ iff they have a non-empty intersection. Clearly the arrangement of neighbourhoods in $\mathcal{D}$ can be as varied as a four-element set will allow; if Δ were made larger, the possible combinations of neighbourhoods could be made as complex as you wish. □

Theorem (`Element.mem_interUpTo`). If an ideal element belongs to each of the first n terms of a sequence of sets, then it belongs to their finite intersection $\bigcap_{i<n} X_i$.

Proof. Induction on n: the empty intersection is the master, which every ideal meets; the successor step uses that ideals are closed under pairwise intersection. ■

Theorem (`consistent_iff_inter_nonempty`). In the four-token system of all nonempty subsets, a length-n initial segment is consistent if and only if $\bigcap_{i<n} X_i$ is nonempty.

Proof. A witnessing neighbourhood below the intersection is nonempty, so the intersection is nonempty. Conversely, a nonempty intersection is itself a neighbourhood contained in itself. ■

Factoid_1_5a.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.P4
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.interUpTo
import Mathlib.Data.Set.BooleanAlgebra
import Mathlib.Data.Finset.Attr
import Mathlib.Tactic.SetLike

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

theorem Element.mem_interUpTo {α : Type*} {V : NeighborhoodSystem α} (x : V.Element)
    (X : ℕ → Set α) :
    ∀ {n : ℕ}, (∀ i, i < n → x.mem (X i)) → x.mem (V.interUpTo X n) := by
  intro n
  induction n with
  | zero => intro _; exact x.master_mem
  | succ n ih =>
    intro h
    rw [interUpTo.succ]
    exact x.inter_mem (ih (fun i hi => h i (Nat.lt_succ_of_lt hi))) (h n (Nat.
    lt_succ_self n))

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.P4

open NeighborhoodSystem

theorem consistent_iff_inter_nonempty (X : ℕ → Set Token) (n : ℕ) :
    neighborhoodSystem.Consistent X n ↔ (neighborhoodSystem.interUpTo X n).Nonempty :=
     by
  constructor
  · rintro ⟨Z, hZ, hsub⟩
    obtain ⟨z, hz⟩ := hZ
    exact ⟨z, hsub hz⟩
  · intro h
    exact ⟨_, h, subset_rfl⟩

end DomainTheory.NeighbourhoodSystems.P4
```

4.2.30 Equivalent sequences with same limit

Having some idea now of the variety of neighbourhood systems, we have to discuss what it is they do. As stressed before, the tokens do not have to correspond directly to the elements; but where, we ask, do the elements come from? One obvious suggestion for determining an element is to produce a sequence of "better and better" neighbourhoods:

$$X_0 \supseteq X_1 \supseteq \cdots \supseteq X_n \supseteq \cdots$$

Trivially, any finite initial segment of this sequence is consistent, and so each X_n is a partial approximation to the "limit". If $\mathcal{D}$ were always to be taken as *finite*, of course, there would be no point in discussing limits since any such sequence would eventually be constant. The elements in the finite case would therefore be completely represented by neighbourhoods with the *minimal* neighbourhoods corresponding to the total elements. But there are many reasons to go beyond the finite (though perhaps not too far beyond).

Suppose $\langle Y_n \rangle_{n=0}^{\infty}$ is another "convergent" sequence with

$Y_{n+1} \subseteq Y_n$ for all indices: when do the two sequences of neighbourhoods determine the *same limit*? The two sequences can surely be different; for example, $\langle Y_n \rangle_{n=0}^{\infty}$ could be a subsequence of $\langle X_n \rangle_{n=0}^{\infty}$, say, $Y_n = X_{2n}$. Still we would want to say that the same limit is obtained. Without being given any further structure on the neighbourhoods, a simple answer is just to say that each sequence goes *equally deep* as the other:

for each m there is an n with $X_n \subseteq Y_m$, and

for each n there is an m with $Y_m \subseteq X_n$.

This definition obviously puts sequences into equivalence classes, and so elements could be identified with these. But such a definition is clumsy for two reasons: it is always tiresome to work with equivalence classes, and there is no reason to think that simple infinite sequences are adequate for determining elements without some rather drastic assumptions on $\mathcal{D}$. Nevertheless, the idea is *suggestive*; we just have to find some construct to represent elements in a unique way and to phrase it in a general enough manner.

Start with $\langle X_n \rangle_{n=0}^{\infty}$ again, which "converges" as before. Think of all the other sequences equivalent to this one in the sense just defined. We can define the class of all terms of all such sequences very easily as being the family:

$$x = \{Z \in \mathcal{D} \mid X_n \subseteq Z \text{ for some } n\}.$$

It is easy to prove that if we form the analogous class for $\langle Y_n \rangle_{n=0}^{\infty}$, then the two families are *equal* if and only if the sequences are *equivalent*. Thus, we seem justified in letting x represent the limit of $\langle X_n \rangle_{n=0}^{\infty}$. All we have to do now is to remark on what sort of class x is as a subfamily of $\mathcal{D}$; what we abstract from the construction, however, will be just a bit more general than taking those x that result from sequences.

Theorem (`limitFamily_eq_iff`). If every term of the sequences X and Y is a neighbourhood, then the limit families of X and of Y coincide if and only if X and Y are equivalent.

Proof. If the families agree, each term of either sequence belongs to the common family, so some term of the other sequence is contained in it. Conversely, if the sequences refine each other, a neighbourhood containing a term of one contains a (later) term of the other, so the two families are the same. ■

`Factoid_1_5b.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
```

```
import Mathlib.Data.Set.Basic

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem limitFamily_eq_iff (X Y : ℕ → Set α)
    (hX : ∀ n, V.mem (X n)) (hY : ∀ m, V.mem (Y m)) :
    V.limitFamily X = V.limitFamily Y ↔ SeqEquiv X Y := by
  constructor
  · intro hEq
    refine ⟨fun m => ?_, fun n => ?_⟩
    · have hmem : Y m ∈ V.limitFamily Y := ⟨hY m, m, subset_rfl⟩
      rw [← hEq] at hmem
      obtain ⟨_, n, hn⟩ := hmem
      exact ⟨n, hn⟩
    · have hmem : X n ∈ V.limitFamily X := ⟨hX n, n, subset_rfl⟩
      rw [hEq] at hmem
      obtain ⟨_, m, hm⟩ := hmem
      exact ⟨m, hm⟩
  · rintro ⟨h1, h2⟩
    apply Set.ext
    intro Z
    constructor
    · rintro ⟨hZ, n, hn⟩
      obtain ⟨m, hm⟩ := h2 n
      exact ⟨hZ, m, hm.trans hn⟩
    · rintro ⟨hZ, m, hm⟩
      obtain ⟨n, hn⟩ := h1 m
      exact ⟨hZ, n, hn.trans hm⟩

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end
```

4.2.31 Definition 1.6 — ideal elements

DEFINITION 1.6. The (ideal) elements of a neighbourhood system $\mathcal{D}$ are those subfamilies $x \subseteq \mathcal{D}$ where:

(i) $\Delta \in x$;

(ii) $X, Y \in x$ always implies $X \cap Y \in x$; and

(iii) whenever $X \in x$ and $X \subseteq Y \in \mathcal{D}$, then $Y \in x$.

The *domain* of all such elements is written as $|\mathcal{D}|$. □

The idea of 1.6 is a well-known mathematical device: the families x satisfying (i)–(iii) are usually called *filters*. Most frequently the emphasis is put on the *maximal* filters, and these would be our *total* elements;

however, in general, the proof that maximal filters exist is non-constructive, so for our purposes it is better not to neglect the partial filters. When maximal filters can be found, well and good, but we do not have to insist on them. Note that the generality of 1.6 is achieved by not requiring that there is a sequence of neighbourhoods that "generates" the filter x. (See Exercise 1.22.)

We have often said that neighbourhoods determine partial elements by themselves; we now make this remark precise.

Definition (`Element`). An *ideal element* of a neighbourhood system is a predicate on sets that holds only of neighbourhoods, contains the master set Δ, is closed under pairwise intersection, and is upward closed among neighbourhoods: if it holds of X and Y is a neighbourhood with $X \subseteq Y$, then it holds of Y.

Proof. The four clauses match Scott's $x \subseteq \mathcal{D}$ together with (i)–(iii); the type of all such elements is the domain $|\mathcal{D}|$. ■

`Element.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem

@[expose] public section

section

namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

structure Element where
  mem : Set α → Prop
  sub : ∀ {X}, mem X → V.mem X
  master_mem : mem V.master
  inter_mem : ∀ {X Y}, mem X → mem Y → mem (X ∩ Y)
  up_mem : ∀ {X Y}, mem X → V.mem Y → X ⊆ Y → mem Y

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.32 Definition 1.7 — principal filter

DEFINITION 1.7. For $X \in \mathcal{D}$, the *principal filter* determined by X is defined by:

$$\uparrow X = \{Y \in \mathcal{D} \mid X \subseteq Y\}.$$

The principal filters form what we shall call the *finite elements* of the domain $|\mathcal{D}|$. □

Definition (`principal`). For a neighbourhood X, the *principal filter* $\uparrow X$ is the ideal element whose neighbourhoods are exactly the members of $\mathcal{D}$ containing X.

Proof. Membership only for neighbourhoods containing X; the master contains X because every neighbourhood sits in Δ; pairwise intersections and upward closure among neighbourhoods are immediate from the subset relation. ■

Theorem (`mem_principal`). A set Y belongs to $\uparrow X$ if and only if Y is a neighbourhood and $X \subseteq Y$.

Proof. This is the membership clause in the definition of $\uparrow X$. ■

`principal.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element
public import Mathlib.Data.Set.Basic

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {α : Type*} (V : NeighborhoodSystem α)

def principal {X : Set α} (hX : V.mem X) : V.Element where
  mem Y := V.mem Y ∧ X ⊆ Y
  sub h := h.1
  master_mem := ⟨V.master_mem, V.sub_master hX⟩
  inter_mem h1 h2 :=
    ⟨V.inter_mem h1.1 h2.1 hX (Set.subset_inter h1.2 h2.2), Set.subset_inter h1.2 h2.2⟩
  up_mem h hY hsub := ⟨hY, h.2.trans hsub⟩

@[simp] theorem mem_principal {X Y : Set α} (hX : V.mem X) :
    (V.principal hX).mem Y ↔ V.mem Y ∧ X ⊆ Y := Iff.rfl
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.33 Inclusion reversing principals

It is obvious that the correspondence between X and $\uparrow X$ is inclusion *reversing*, in the sense that

$$X \subseteq Y \quad \text{iff} \quad \uparrow Y \subseteq \uparrow X$$

for all $X, Y \in \mathcal{D}$.

Theorem (`principal_le_iff`). For neighbourhoods X and Y, one has $\uparrow X \leq \uparrow Y$ in the approximation order if and only if $Y \subseteq X$. Equivalently, $X \subseteq Y$ if and only if $\uparrow Y \subseteq \uparrow X$ as families of neighbourhoods.

Proof. If $\uparrow X \leq \uparrow Y$, then X belongs to $\uparrow Y$, so $Y \subseteq X$. Conversely, if $Y \subseteq X$ and Z belongs to $\uparrow X$, then $X \subseteq Z$, hence $Y \subseteq Z$, so Z belongs to $\uparrow Y$. ■

`principal_le_iff.lean`

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
import Mathlib.Data.Set.Basic

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem principal_le_iff {X Y : Set α} (hX : V.mem X) (hY : V.mem Y) :
    V.principal hX ≤ V.principal hY ↔ Y ⊆ X := by
  constructor
  · intro h
    exact (h X ⟨hX, subset_rfl⟩).2
  · intro hYX Z hZ
    exact ⟨hZ.1, hYX.trans hZ.2⟩

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.34 Principals are one-one

It is obvious that the correspondence between X and $\uparrow X$ is one-one.

Theorem (`principal_injective`). The map sending a neighbourhood to its principal filter is injective: if $\uparrow X = \uparrow Y$, then $X = Y$.

Proof. From the inclusion-reversing equivalence, equal principals give $Y \subseteq X$ and $X \subseteq Y$. ■

`principal_injective.lean`

```
module

import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal_le_iff
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
import Mathlib.Data.Set.Basic

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem principal_injective {X Y : Set α} (hX : V.mem X) (hY : V.mem Y)
    (h : V.principal hX = V.principal hY) : X = Y := by
  have hYX : Y ⊆ X := (V.principal_le_iff hX hY).mp (le_of_eq h)
  have hXY : X ⊆ Y := (V.principal_le_iff hY hX).mp (le_of_eq h.symm)
  exact Set.Subset.antisymm hXY hYX
```

```
end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.35 Union of principal filters

Except in very special cases, there is much more to $|\mathcal{D}|$ than just the finite elements. Much of our investigation will be concerned with finding out how much more. The finite elements are, in a certain sense, "dense" in $|\mathcal{D}|$, however, because it is also obvious from the definitions that for each $x \in |\mathcal{D}|$

$$x = \bigcup \{\uparrow X \mid X \in x\}.$$

That is, every element is a certain type of "limit" of finite elements. (This statement is made more precise in Exercise 1.21)

We note that we have now had several occasions to use inclusion relationships between elements; this is an important relationship, and we give it a special name.

Theorem (`eq_iUnion_principal`). A neighbourhood Z belongs to an ideal element x if and only if Z belongs to $\uparrow X$ for some neighbourhood X of x. Equivalently, $x = \bigcup \{\uparrow X \mid X \in x\}$.

Proof. If Z belongs to x, take $X = Z$: then Z belongs to $\uparrow Z$. Conversely, if Z belongs to $\uparrow X$ for some neighbourhood X of x, then $X \subseteq Z$, so upward closure puts Z in x. ■

`Factoid_1_7b.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
import Mathlib.Data.Set.Basic

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem eq_iUnion_principal (x : V.Element) {Z : Set α} :
    x.mem Z ↔ ∃ X, ∃ hX : x.mem X, (V.principal (x.sub hX)).mem Z := by
  constructor
  · intro hZ
    exact ⟨Z, hZ, x.sub hZ, subset_rfl⟩
  · rintro ⟨X, hX, hVZ, hXZ⟩
    exact x.up_mem hX hVZ hXZ

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.36 Definition 1.8 — approximation and totality

DEFINITION 1.8. For $x, y \in |\mathcal{D}|$, we say that x *approximates* y iff $x \subseteq y$. The element that approximates all others, $\{\Delta\}$, is called $\bot$ (read: *bottom*); it is the "least defined" element, or the "most partial" element. Elements maximal with respect to the approximation relation are called *total elements*. □

Definition (`bot`). Bottom is the principal filter of the master set: $\bot = \uparrow \Delta$.

Proof. Take the principal element determined by Δ. ■

Theorem (`mem_bot`). A set Y is a neighbourhood of $\bot$ if and only if $Y = \Delta$.

Proof. Membership in $\uparrow \Delta$ means Y is a neighbourhood containing Δ; every neighbourhood already lies in Δ, so that forces $Y = \Delta$. ■

`bot.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
import Mathlib.Data.Set.Basic

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {α : Type*} (V : NeighborhoodSystem α)

def bot : V.Element := V.principal V.master_mem

@[simp] theorem mem_bot {Y : Set α} : V.bot.mem Y ↔ Y = V.master := by
  constructor
  · rintro ⟨hY, hΔY⟩
    exact Set.Subset.antisymm (V.sub_master hY) hΔY
  · rintro rfl
    exact ⟨V.master_mem, subset_rfl⟩
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.37 Bottom element

DEFINITION 1.8. For $x, y \in |\mathcal{D}|$, we say that x *approximates* y iff $x \subseteq y$. The element that approximates all others, $\{\Delta\}$, is called $\bot$ (read: *bottom*); it is the "least defined" element, or the "most partial" element. Elements maximal with respect to the approximation relation are called *total elements*. □

Definition (`bot`). Bottom is the principal filter of the master set: $\bot = \uparrow \Delta$.

Proof. Take the principal element determined by Δ. ■

Theorem (`mem_bot`). A set Y is a neighbourhood of $\bot$ if and only if $Y = \Delta$.

Proof. Membership in $\uparrow \Delta$ means Y is a neighbourhood containing Δ; every neighbourhood already lies in Δ, so that forces $Y = \Delta$. ■

4.2.38 Bottom is least

DEFINITION 1.8. For $x, y \in |\mathcal{D}|$, we say that x *approximates* y iff $x \subseteq y$. The element that approximates all others, $\{\Delta\}$, is called $\bot$ (read: *bottom*); it is the "least defined" element, or the "most partial" element. Elements maximal with respect to the approximation relation are called *total elements*. □

Theorem (`bot_le`). Bottom approximates every ideal element: $\bot \leq x$ for all $x \in |\mathcal{D}|$.

Proof. The only neighbourhood of $\bot$ is Δ, and every ideal element contains Δ. ■

Instance (`OrderBot`). Approximation order on ideal elements has least element $\bot$.

Proof. Take bottom as $\bot = \uparrow \Delta$ and leastness from the previous theorem. ■

`bot_le.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {α : Type*} (V : NeighborhoodSystem α)

theorem bot_le (x : V.Element) : V.bot ≤ x := by
  intro Y hY
  rw [mem_bot] at hY
  subst hY
  exact x.master_mem

instance : OrderBot V.Element where
  bot := V.bot
  bot_le := V.bot_le

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.39 Finite filters are principal

EXAMPLES 1.2 – 1.5 (Revisited). The examples as given were all finite, so any explicitly given filter x is principal, the element is finite, the minimal $X \in x$ tells us all we need to know. In such simple situations there is essentially no difference between elements and neighbourhoods — except for the reversal of the order as noted. This (necessary) reversal should not, however, become a matter of confusion: the smaller the neighbourhood has become, the more it has "converged", and so the better defined the element has become. In the approximation relation the "poorer" elements are placed below the "better" with the total up at the top. This will become clearer in discussing "infinite" elements.

Theorem (`eq_principal_of_isMin`). If X belongs to an ideal element x and is $\subseteq$-minimal among the neighbourhoods of x, then $x = \uparrow X$.

Proof. Every neighbourhood of x contains X, so belongs to $\uparrow X$; conversely every member of $\uparrow X$ is an upward extension of X and hence a neighbourhood of x. ■

Theorem (`exists_minimum_of_fintype`). When the ground set is finite, every ideal element has a $\subseteq$-minimal neighbourhood.

Proof. Among neighbourhoods of x contained in Δ, take one of least cardinality; a strictly smaller neighbourhood would contradict minimality of that cardinality. ■

Theorem (`element_eq_principal_of_fintype`). When the ground set is finite, every ideal element is principal.

Proof. Combine the previous two theorems. ■

`eq_principal_of_isMin.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import Mathlib.Data.Fintype.Defs
import Mathlib.Data.Set.Card

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {α : Type*} (V : NeighborhoodSystem α)

theorem eq_principal_of_isMin (x : V.Element) {X : Set α} (hX : x.mem X)
    (hmin : ∀ Y, x.mem Y → X ⊆ Y) : x = V.principal (x.sub hX) := by
  apply Element.ext
  intro Z
  constructor
  · intro hZ
    exact ⟨x.sub hZ, hmin Z hZ⟩
  · rintro ⟨hZmem, hXZ⟩
    exact x.up_mem hX hZmem hXZ

theorem exists_minimum_of_fintype [Fintype α] (x : V.Element) :
    ∃ X, x.mem X ∧ ∀ Y, x.mem Y → X ⊆ Y := by
  classical
  suffices aux : ∀ n (A : Set α), A.ncard = n → x.mem A →
      ∃ X, x.mem X ∧ X ⊆ A ∧ ∀ Y, x.mem Y → Y ⊆ A → X ⊆ Y from by
    obtain ⟨X, hX, _, hmin⟩ := aux V.master.ncard V.master rfl x.master_mem
    exact ⟨X, hX, fun Y hY => hmin Y hY (V.sub_master (x.sub hY))⟩
  intro n
  induction n using Nat.strong_induction_on with
  | h n ih =>
    intro A hn hA
    by_cases hleast : ∀ Y, x.mem Y → Y ⊆ A → A ⊆ Y
    · exact ⟨A, hA, subset_rfl, hleast⟩
```

```
    · push Not at hleast
      obtain ⟨Y, hY, hYA, hAY⟩ := hleast
      have hss : Y ? A := ⟨hYA, hAY⟩
      have hlt : Y.ncard < n := by
        rw [← hn]
        exact Set.ncard_lt_ncard hss
      obtain ⟨X, hX, hXsub, hmin⟩ := ih Y.ncard hlt Y rfl hY
      refine ⟨X, hX, hXsub.trans hYA, ?_⟩
      intro Z hZ hZA
      have hcap : x.mem (Z ∩ Y) := x.inter_mem hZ hY
      exact (hmin (Z ∩ Y) hcap Set.inter_subset_right).trans Set.inter_subset_left

theorem element_eq_principal_of_fintype [Fintype α] (x : V.Element) :
    ∃ X, ∃ hX : V.mem X, x = V.principal hX := by
  obtain ⟨X, hX, hmin⟩ := V.exists_minimum_of_fintype x
  exact ⟨X, x.sub hX, V.eq_principal_of_isMin x hX hmin⟩

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.40 The cone of finite bit strings

We let

$$\Delta = \Sigma^*,$$

where $\Sigma = \{0, 1\}$ and Σ^* means the set of all finite sequences of 0's and 1's, with Λ being the empty sequence. A neighbourhood consists of all *extensions* of a given sequence σ:

$$\sigma\Sigma^* = \{\sigma\tau \mid \tau \in \Sigma^*\}.$$

(Refer back to the finite version of 1.4.)

Definition (`Str`). Tokens are finite bit-strings: $\Sigma^* = \mathrm{List}\,\Sigma$ with $\Sigma = \{0, 1\}$.

Definition (`cone`). The cone of a string σ is $\sigma\Sigma^* = \{w \mid \sigma \text{ prefix } w\}$.

Theorem (`cone.nil`). The cone of the empty string is all of Σ^*.

Theorem (`cone.subset_cone`). One has $\sigma\Sigma^* \subseteq \tau\Sigma^*$ if and only if τ is a prefix of σ.

Theorem (`cone.trichotomy`). Any two cones are nested or disjoint.

Proof. Compare the two prefixes: one extends the other, or else no common extension exists. ■

Theorem (`cone.injective`). Equal cones come from equal strings.

Proof. Mutual inclusion of cones yields mutual prefix, hence equal length and equal strings. ■

`cone.lean`

```
module

public import Mathlib.Data.Set.Basic
public import Mathlib.Data.List.Basic

@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems.BitDom

abbrev Str := List Bool

def cone (σ : Str) : Set Str := {w | σ <+: w}

namespace cone

@[simp] theorem mem {σ w : Str} : w ∈ BitDom.cone σ ↔ σ <+: w := Iff.rfl

theorem nil : BitDom.cone [] = Set.univ := by
  ext w
  simp [BitDom.cone]

theorem subset_cone {σ τ : Str} :
    BitDom.cone σ ⊆ BitDom.cone τ ↔ τ <+: σ := by
  constructor
  · intro h
    exact h (List.prefix_rfl : σ <+: σ)
  · intro hτσ w hw
    exact hτσ.trans hw

theorem trichotomy (σ τ : Str) :
    BitDom.cone σ ⊆ BitDom.cone τ ∨ BitDom.cone τ ⊆ BitDom.cone σ ∨
      BitDom.cone σ ∩ BitDom.cone τ = ∅ := by
  if hστ : σ <+: τ then
    exact Or.inr (Or.inl (subset_cone.mpr hστ))
  else if hτσ : τ <+: σ then
    exact Or.inl (subset_cone.mpr hτσ)
  else
    refine Or.inr (Or.inr ?_)
    ext w
    simp only [Set.mem_inter_iff, mem, Set.mem_empty_iff_false, iff_false, not_and]
    intro h1 h2
    rcases List.prefix_or_prefix_of_prefix h1 h2 with h | h
    · exact hστ h
    · exact hτσ h

theorem injective {σ τ : Str} (h : BitDom.cone σ = BitDom.cone τ) : σ = τ := by
  have h1 : τ <+: σ := subset_cone.mp (h.symm ▸ subset_rfl)
  have h2 : σ <+: τ := subset_cone.mp (h ▸ subset_rfl)
  exact h2.eq_of_length (h2.length_le.antisymm h1.length_le)

end cone

end DomainTheory.NeighbourhoodSystems.BitDom
```

4.2.41 Examples 1.2–1.5 (Revisited) — infinite binary neighbourhoods

EXAMPLES 1.2 – 1.5 (Revisited). The examples as given were all finite, so any explicitly given filter x is principal, the element is finite, the minimal $X \in x$ tells us all we need to know. In such simple situations there is essentially no difference between elements and neighbourhoods — except for the reversal of the order as noted. This (necessary) reversal should not, however, become a matter of confusion: the smaller the

neighbourhood has become, the more it has "converged", and so the better defined the element has become. In the approximation relation the "poorer" elements are placed below the "better" with the total up at the top. This will become clearer in discussing "infinite" elements.

Example 1.3 will be generalized in Exercise 1.12. Let us here generalize first 1.4. We let

$$\Delta = \Sigma^*,$$

where $\Sigma = \{0, 1\}$ and Σ^* means the set of all finite sequences of 0's and 1's, with Λ being the empty sequence. We write $\sigma\tau$ for the *concatenation* or *juxtaposition* of two sequences $\sigma, \tau \in \Sigma^*$. Define

$$B = \{\sigma\Sigma^* \mid \sigma \in \Sigma^*\}, \text{ where}$$

$$\sigma X = \{\sigma\tau \mid \tau \in X\},$$

for an arbitrary set $X \subseteq \Sigma^*$. In other words, a neighbourhood in B consists of all *extensions* of a given sequence σ. (Refer back to the finite version of 1.4.) We use the letter "B" to remind us of "binary", and this is an example we shall refer to many times. The proof (if it is not obvious) that B is a neighbourhood system should be done as an exercise.

What do we find in $|B|$? Of course $\bot = \{\Delta\} \in |B|$. For any $x \in |B|$ and $\sigma \in \Sigma^*$ define

$$\sigma x = \{Y \mid \sigma X \subseteq Y \text{ some } X \in x\}.$$

Definition (`memB`). Membership in B means being equal to some cone $\sigma\Sigma^*$.

Theorem (`nestedOrDisjoint`). Any two cones are nested or disjoint.

Proof. From the trichotomy of cones. ■

Definition (`B`). The binary neighbourhood system is the nested-or-disjoint system of all cones, with master $\Delta = \Sigma^*$.

Proof. Apply the nested-or-disjoint constructor, using the previous theorem and that every cone lies in Σ^*. ■

Definition (`prepend`). Prepending σ to a set X is $\sigma X = \{\sigma\tau \mid \tau \in X\}$.

Definition (`sigmaElt`). For $x \in |B|$ and $\sigma \in \Sigma^*$, the ideal element σx consists of those neighbourhoods Y for which $\sigma X \subseteq Y$ for some neighbourhood X of x.

Proof. Filter axioms follow from those of x together with the fact that prepending a cone yields a cone. ■

Definition (`sigmaBot`). Write $\sigma\ \bot$ for the principal filter of the cone $\sigma\Sigma^*$.

`ExampleB.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.cone
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.BitDom
```

```
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

def memB (X : Set Str) : Prop := ∃ σ, X = cone σ

theorem nestedOrDisjoint : NestedOrDisjoint memB := by
  rintro X Y ⟨σ, rfl⟩ ⟨τ, rfl⟩
  exact cone.trichotomy σ τ

def B : NeighborhoodSystem Str :=
  NeighborhoodSystem.ofNestedOrDisjoint memB Set.univ
    ⟨[], Set.mem_univ _⟩ ⟨[], cone.nil.symm⟩ nestedOrDisjoint
    (fun _ => Set.subset_univ _)

@[simp] theorem B_mem {X : Set Str} : B.mem X ↔ memB X := Iff.rfl

@[simp] theorem B_master : B.master = Set.univ := rfl

theorem memB_cone (σ : Str) : B.mem (cone σ) := ⟨σ, rfl⟩

def prepend (σ : Str) (X : Set Str) : Set Str := {w | ∃ τ, τ ∈ X ∧ w = σ ++ τ}

@[simp] theorem mem_prepend {σ : Str} {X : Set Str} {w : Str} :
    w ∈ prepend σ X ↔ ∃ τ, τ ∈ X ∧ w = σ ++ τ := Iff.rfl

theorem prepend_cone (σ ? : Str) : prepend σ (cone ?) = cone (σ ++ ?) := by
  ext w
  simp only [mem_prepend, cone.mem]
  constructor
  · rintro ⟨τ, hτ, rfl⟩
    exact (List.prefix_append_right_inj σ).mpr hτ
  · rintro ⟨t, ht⟩
    exact ⟨? ++ t, List.prefix_append ? t, by rw [← ht, List.append_assoc]⟩

theorem memB_prepend (σ : Str) {X : Set Str} (hX : B.mem X) : B.mem (prepend σ X) :=
    by
  obtain ⟨?, rfl⟩ := hX
  exact ⟨σ ++ ?, prepend_cone σ ?⟩

@[simp] theorem prepend_nil (X : Set Str) : prepend [] X = X := by
  ext w; simp [mem_prepend]

def sigmaBot (σ : Str) : B.Element := B.principal (memB_cone σ)

def sigmaElt (σ : Str) (x : B.Element) : B.Element where
  mem Y := B.mem Y ∧ ∃ X, x.mem X ∧ prepend σ X ⊆ Y
  sub h := h.1
  master_mem := ⟨B.master_mem, B.master, x.master_mem, Set.subset_univ _⟩
  inter_mem := by
    intro Y₁ Y₂ h1 h2
    obtain ⟨hY₁, X₁, hX₁, hsub₁⟩ := h1
    obtain ⟨hY₂, X₂, hX₂, hsub₂⟩ := h2
    have hXinter : x.mem (X₁ ∩ X₂) := x.inter_mem hX₁ hX₂
    have hsub : prepend σ (X₁ ∩ X₂) ⊆ Y₁ ∩ Y₂ := by
      rintro w ⟨τ, ⟨hτ₁, hτ₂⟩, rfl⟩
      exact ⟨hsub₁ ⟨τ, hτ₁, rfl⟩, hsub₂ ⟨τ, hτ₂, rfl⟩⟩
```

```
    have hZmem : B.mem (prepend σ (X₁ ∩ X₂)) := memB_prepend σ (x.sub hXinter)
    exact ⟨B.inter_mem hY₁ hY₂ hZmem hsub, X₁ ∩ X₂, hXinter, hsub⟩
  up_mem := by
    intro X Y hX hY hXY
    obtain ⟨_, X', hX', hsub'⟩ := hX
    exact ⟨hY, X', hX', hsub'.trans hXY⟩

theorem bot_mem_cone_iff {σ : Str} : B.bot.mem (cone σ) ↔ σ = [] := by
  rw [mem_bot, B_master, ← cone.nil]
  constructor
  · exact cone.injective
  · intro h; rw [h]

end DomainTheory.NeighbourhoodSystems.BitDom
```

4.2.42 Prefix order

Again there is an exercise here to show $\sigma x \in |B|$. In particular $\sigma \perp \in |B|$ for all $\sigma \in \Sigma^*$, and these are just the finite elements. The minimal element of $\sigma \perp$ is $\sigma\Delta$. Note that $\sigma_0 \perp \subseteq \sigma_1 \perp$ if and only if σ_0 is an *initial segment* of the sequence σ_1.

Theorem (`prepend_append`). Prepending concatenates: $\sigma(\tau X) = (\sigma\tau)X$.

Proof. Expand both sides as concatenations of strings. ■

Theorem (`prepend_mono`). Prepending preserves inclusion: $X' \subseteq X$ implies $\sigma X' \subseteq \sigma X$.

Proof. Every $\sigma\tau$ with $\tau \in X'$ is likewise in σX. ■

Theorem (`prepend_univ`). Prepending the master set yields the cone: $\sigma\Delta = \sigma\Sigma^*$.

Proof. Extensions of σ are exactly concatenations $\sigma\tau$ for $\tau \in \Sigma^*$. ■

Theorem (`sigmaBot_le_iff`). One has $\sigma_0 \perp \leq \sigma_1 \perp$ if and only if σ_0 is a prefix of σ_1.

Proof. Principals compare by reverse inclusion of cones, and cones reverse-include exactly when the shorter string prefixes the longer. ■

Theorem (`sigmaElt_append`). Prepending concatenates on elements: $(\sigma\tau)x = \sigma(\tau x)$.

Proof. Unwind the definition of σx and use that prepending concatenates on sets. ■

Theorem (`sigmaElt_bot`). Prepending bottom yields the principal: $\sigma \perp = \sigma \cdot \perp$.

Proof. The only neighbourhood of $\perp$ is Δ, and $\sigma\Delta$ is the cone of σ. ■

Theorem (`sigmaElt_nil`). The empty string acts as identity: $\Lambda x = x$.

Proof. Prepending the empty string leaves sets unchanged. ■

`sigmaBot_le_iff.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ExampleB
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal_le_iff
import Mathlib.Tactic.Attr.Core

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.BitDom

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem
```

```
theorem prepend_append (σ τ : Str) (X : Set Str) :
    prepend σ (prepend τ X) = prepend (σ ++ τ) X := by
  ext w
  simp only [mem_prepend]
  constructor
  · rintro ⟨?, ⟨?', h?', rfl⟩, rfl⟩
    exact ⟨?', h?', by rw [List.append_assoc]⟩
  · rintro ⟨?, h?, rfl⟩
    exact ⟨τ ++ ?, ⟨?, h?, rfl⟩, by rw [List.append_assoc]⟩

theorem prepend_mono (σ : Str) {X X' : Set Str} (h : X' ⊆ X) : prepend σ X' ⊆ prepend
    σ X := by
  rintro w ⟨τ, hτ, rfl⟩
  exact ⟨τ, h hτ, rfl⟩

theorem prepend_univ (σ : Str) : prepend σ Set.univ = cone σ := by
  ext w
  simp only [mem_prepend, Set.mem_univ, true_and, cone.mem]
  constructor
  · rintro ⟨τ, rfl⟩
    exact List.prefix_append σ τ
  · rintro ⟨t, ht⟩
    exact ⟨t, ht.symm⟩

theorem sigmaBot_le_iff (σ₀ σ₁ : Str) :
    sigmaBot σ₀ ≤ sigmaBot σ₁ ↔ σ₀ <+: σ₁ := by
  rw [sigmaBot, sigmaBot, B.principal_le_iff, cone.subset_cone]

theorem sigmaElt_append (σ τ : Str) (x : B.Element) :
    sigmaElt (σ ++ τ) x = sigmaElt σ (sigmaElt τ x) := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨hY, X, hX, hsub⟩
    refine ⟨hY, prepend τ X, ⟨memB_prepend τ (x.sub hX), X, hX, subset_rfl⟩, ?_⟩
    rw [prepend_append]; exact hsub
  · rintro ⟨hY, X, ⟨_, X', hX', hsub'⟩, hsub⟩
    refine ⟨hY, X', hX', ?_⟩
    rw [← prepend_append]
    exact (prepend_mono σ hsub').trans hsub

theorem sigmaElt_bot (σ : Str) : sigmaElt σ B.bot = sigmaBot σ := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨hY, X, hX, hsub⟩
    rw [B.mem_bot] at hX
    subst hX
    rw [B_master, prepend_univ] at hsub
    exact ⟨hY, hsub⟩
  · rintro ⟨hY, hcone⟩
    refine ⟨hY, B.master, B.mem_bot.mpr rfl, ?_⟩
    rw [B_master, prepend_univ]
    exact hcone
```

```
theorem sigmaElt_nil (x : B.Element) : sigmaElt [] x = x := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨hY, X, hX, hsub⟩
    rw [prepend_nil] at hsub
    exact x.up_mem hX hY hsub
  · intro hY
    exact ⟨x.sub hY, Y, hY, by rw [prepend_nil]⟩

end DomainTheory.NeighbourhoodSystems.BitDom
```

4.2.43 Union of finite approximations

If now $x \in |B|$ is any explicitly given element (that is, if we know for any $X \in B$ whether or not $X \in x$), we have but to work out from these definitions that

$$x = \bigcup_{n=0}^{\infty} \sigma_n \perp,$$

where the $\sigma_n \in \Sigma^*$ and each σ_n is an initial segment of the next σ_{n+1}. *In general, in any domain, an element is uniquely determined by its finite approximations, and we are just making this explicit in* $|B|$. When we have complete knowledge of x, then there are two cases: either the approximations $\sigma_n \perp$ become constant from some point on (where $n > n_0$), or not. In the first case x is finite and equal to $\sigma_{n_0} \perp$; in the second case x is infinite and the σ_n fill out an infinite (one-way) sequence.

The generalization of 1.5 to the infinite case where $\Delta = \mathbf{N} = \{0, 1, 2, 3, \ldots, n, \ldots\}$ can be made in more than one way: for instance either we use as neighbourhoods *all* non-empty subsets of Δ or just those omitting but a finite number of integers. And, as will become apparent, there are other choices giving domains of quite different characters. □

Many constructions (choices of $\mathcal{D}$) lead to the "same" domain; "sameness" is an important notion and it is to be defined in terms of "isomorphism", which in turn is to be defined in terms of approximation preserving correspondences.

Theorem (`mem_iff_exists_sigmaBot`). For $x \in |B|$ and $Z \subseteq \Sigma^*$, one has $Z \in x$ if and only if there is a string σ such that $\sigma\Sigma^* \in x$ and $Z \in \sigma \perp$.

Proof. Every neighbourhood of x is some cone $\sigma\Sigma^*$, and that cone belongs to $\sigma \perp$; conversely, if a cone of x lies under Z via $\sigma \perp$, the filter axioms put Z in x. ■

This is a pointwise membership form of "x is made of finite approximations," not the displayed chain-union $x = \bigcup_n \sigma_n \perp$ along nested prefixes, nor the finite/infinite dichotomy that follows in the prose.

`mem_iff_exists_sigmaBot.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    sigmaBot_le_iff

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.BitDom
```

```
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

theorem mem_iff_exists_sigmaBot (x : B.Element) {Z : Set Str} :
    x.mem Z ↔ ∃ σ, x.mem (cone σ) ∧ (sigmaBot σ).mem Z := by
  constructor
  · intro hZ
    obtain ⟨σ, rfl⟩ := x.sub hZ
    exact ⟨σ, hZ, x.sub hZ, subset_rfl⟩
  · rintro ⟨σ, hcone, hZmem, hsub⟩
    exact x.up_mem hcone hZmem hsub

end DomainTheory.NeighbourhoodSystems.BitDom
```

4.2.44 Definition 1.9 — isomorphic domains

DEFINITION 1.9. Two neighbourhood systems $\mathcal{D}_0$ and $\mathcal{D}_1$ determine *isomorphic domains* iff there is a one-one correspondence between $|\mathcal{D}_0|$ and $|\mathcal{D}_1|$ which preserves inclusion between the elements of the domains. In symbols we write $\mathcal{D}_0 \cong \mathcal{D}_1$. □

It is certain that the property of 1.9 is necessary, but it may not be so clear that it is sufficient. We shall in fact prove in the next lecture that an isomorphism between domains always maps finite elements to finite elements, so it always results from a one-one inclusion-preserving correspondence between neighbourhoods. This is surely as strong as could be hoped. This general result is not needed to see that particular domains are isomorphic.

In some of the examples tokens corresponded to total elements and in some to partial elements; it is not difficult to see (ex post facto) that every domain can be presented with tokens exactly corresponding to partial elements.

Definition (`DomainIso`). A domain isomorphism is an order isomorphism between the posets of ideal elements of two neighbourhood systems.

Definition (`Isomorphic`). Write $\mathcal{D}_0 \cong \mathcal{D}_1$ when such a domain isomorphism exists.

Theorem (`Isomorphic.refl`). Every neighbourhood system is isomorphic to itself.

Proof. The identity order isomorphism. ■

Theorem (`Isomorphic.symm`). Isomorphism of domains is symmetric.

Proof. Invert the order isomorphism. ■

Theorem (`Isomorphic.trans`). Isomorphism of domains is transitive.

Proof. Compose the order isomorphisms. ■

`DomainIso.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import Mathlib.Order.Hom.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

/-- Order isomorphism between the ideal-element posets of two neighbourhood systems.
    -/
```

```
abbrev DomainIso {α β : Type*} (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    : Type _ :=
  V₀.Element ≃o V₁.Element

/-- Two neighbourhood systems determine isomorphic domains. -/
def Isomorphic {α β : Type*} (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    Prop :=
  Nonempty (DomainIso V₀ V₁)

@[inherit_doc] infix:25 " ≅? " => Isomorphic

theorem Isomorphic.refl {α : Type*} (V : NeighborhoodSystem α) : V ≅? V :=
  ⟨OrderIso.refl _⟩

theorem Isomorphic.symm {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ :
    NeighborhoodSystem β}
    (h : V₀ ≅? V₁) : V₁ ≅? V₀ :=
  h.elim fun e => ⟨e.symm⟩

theorem Isomorphic.trans {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ :
    NeighborhoodSystem β}
    {V₂ : NeighborhoodSystem ?} (h₀ : V₀ ≅? V₁) (h₁ : V₁ ≅? V₂) : V₀ ≅? V₂ :=
  h₀.elim fun e₀ => h₁.elim fun e₁ => ⟨e₀.trans e₁⟩

end DomainTheory.NeighbourhoodSystems
```

4.2.45 Brackets

Given any neighbourhood system $\mathcal{D}$, define for $X \in \mathcal{D}$

$$[X] = \{x \in |\mathcal{D}| \mid X \in x\}.$$

Definition (`bracket`). The bracket of a neighbourhood X is $[X] = \{x \in |\mathcal{D}| \mid X \in x\}$.

Theorem (`mem_bracket`). An ideal element lies in $[X]$ if and only if X is one of its neighbourhoods.

Theorem (`bracket_master`). The master bracket is the whole domain: $[\Delta] = |\mathcal{D}|$.

Proof. Every ideal element contains Δ. ■

Theorem (`bracket_inter`). For neighbourhoods X and Y one has $[X] \cap [Y] = [X \cap Y]$.

Proof. Membership in both brackets means both X and Y belong to the filter, hence so does their intersection; conversely an ideal element containing $X \cap Y$ contains each factor by upward closure. ■

Theorem (`principal_mem_bracket`). The principal $\uparrow X$ lies in $[X]$.

Proof. X belongs to $\uparrow X$. ■

Theorem (`bracket_subset_iff`). For neighbourhoods X and Y, one has $[X] \subseteq [Y]$ if and only if $X \subseteq Y$.

Proof. Forward: $\uparrow X \in [X]$ forces $Y \in \uparrow X$. Reverse: upward closure of filters. ■

Theorem (`bracket_injective`). Equal brackets come from equal neighbourhoods.

Proof. Mutual inclusion of brackets yields mutual inclusion of the neighbourhoods. ■

`bracket.lean`

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {α : Type*} (V : NeighborhoodSystem α)

/-- The basic open set of ideal elements containing the neighbourhood $X$. -/
def bracket (X : Set α) : Set V.Element := {x | x.mem X}

@[simp] theorem mem_bracket {X : Set α} {x : V.Element} : x ∈ V.bracket X ↔ x.mem X :=
     Iff.rfl

@[simp] theorem bracket_master : V.bracket V.master = Set.univ := by
  ext x; simp only [mem_bracket, Set.mem_univ, iff_true]; exact x.master_mem

theorem bracket_inter {X Y : Set α} (hX : V.mem X) (hY : V.mem Y) :
    V.bracket X ∩ V.bracket Y = V.bracket (X ∩ Y) := by
  ext x
  simp only [Set.mem_inter_iff, mem_bracket]
  constructor
  · rintro ⟨hx, hy⟩
    exact x.inter_mem hx hy
  · intro hxy
    exact ⟨x.up_mem hxy hX Set.inter_subset_left, x.up_mem hxy hY Set.
    inter_subset_right⟩

theorem principal_mem_bracket {X : Set α} (hX : V.mem X) : V.principal hX ∈ V.bracket
    X :=
  ⟨hX, subset_rfl⟩

theorem bracket_subset_iff {X Y : Set α} (hX : V.mem X) (hY : V.mem Y) :
    V.bracket X ⊆ V.bracket Y ↔ X ⊆ Y := by
  constructor
  · intro h
    exact (h (V.principal_mem_bracket hX)).2
  · intro hXY x hx
    exact x.up_mem hx hY hXY

theorem bracket_injective {X Y : Set α} (hX : V.mem X) (hY : V.mem Y)
    (h : V.bracket X = V.bracket Y) : X = Y :=
  Set.Subset.antisymm ((V.bracket_subset_iff hX hY).mp h.subset)
    ((V.bracket_subset_iff hY hX).mp h.superset)

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.46 Token system

The subsets $[X] \subseteq |\mathcal{D}|$ for $X \in \mathcal{D}$ form a neighbourhood system over $|\mathcal{D}|$.

Definition (`tokenSystem`). The token system of $\mathcal{D}$ is the neighbourhood system on $|\mathcal{D}|$ whose neighbourhoods are exactly the brackets $[X]$ for $X \in \mathcal{D}$, with master $|\mathcal{D}|$.

Proof. Masterhood is $[\Delta] = |\mathcal{D}|$; intersections of brackets are brackets of intersections. ■

Theorem (`tokenSystem_mem`). A set of ideal elements is a neighbourhood of the token system if and only if it equals $[X]$ for some neighbourhood X of $\mathcal{D}$.

Definition (`toToken`). Send an ideal element x of $\mathcal{D}$ to the filter of those brackets $[X]$ with $X \in x$.

Definition (`ofToken`). Send an ideal element y of the token system to the filter of those $X \in \mathcal{D}$ with $[X] \in y$.

Definition (`tokenIso`). Those two maps are mutually inverse order isomorphisms between $|\mathcal{D}|$ and the domain of the token system.

Proof. Bracket injectivity recovers neighbourhoods from brackets; order is preserved because inclusion of filters matches inclusion of brackets. ■

Theorem (`isomorphic_tokenSystem`). Every neighbourhood system is isomorphic to its token system.

Proof. The previous order isomorphism. ■

`tokenSystem.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bracket
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {α : Type*} (V : NeighborhoodSystem α)

/-- The neighbourhood system whose tokens are ideal elements and whose
neighbourhoods are the brackets `[X] = {x | X ∈ x}`. -/
def tokenSystem : NeighborhoodSystem V.Element where
  mem S := ∃ X, V.mem X ∧ S = V.bracket X
  master := Set.univ
  master_nonempty := ⟨V.bot, Set.mem_univ _⟩
  master_mem := ⟨V.master, V.master_mem, V.bracket_master.symm⟩
  inter_mem := by
    rintro S T W ⟨X, hX, rfl⟩ ⟨Y, hY, rfl⟩ ⟨Z, hZ, rfl⟩ hWsub
    have hpz := hWsub (V.principal_mem_bracket hZ)
    obtain ⟨hzX, hzY⟩ := hpz
    have hXY : V.mem (X ∩ Y) := V.inter_mem hX hY hZ (Set.subset_inter hzX.2 hzY.2)
    exact ⟨X ∩ Y, hXY, V.bracket_inter hX hY⟩
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem tokenSystem_mem {S : Set V.Element} :
    V.tokenSystem.mem S ↔ ∃ X, V.mem X ∧ S = V.bracket X := Iff.rfl
```

```
def ofToken (y : V.tokenSystem.Element) : V.Element where
  mem X := V.mem X ∧ y.mem (V.bracket X)
  sub h := h.1
  master_mem := ⟨V.master_mem, by rw [V.bracket_master]; exact y.master_mem⟩
  inter_mem := by
    rintro X Y ⟨hX, hyX⟩ ⟨hY, hyY⟩
    have hyInter : y.mem (V.bracket X ∩ V.bracket Y) := y.inter_mem hyX hyY
    obtain ⟨W, hW, hWeq⟩ := y.sub hyInter
    have hpw := V.principal_mem_bracket hW
    rw [← hWeq] at hpw
    obtain ⟨hwX, hwY⟩ := hpw
    have hXY : V.mem (X ∩ Y) := V.inter_mem hX hY hW (Set.subset_inter hwX.2 hwY.2)
    refine ⟨hXY, ?_⟩
    rw [← V.bracket_inter hX hY]
    exact hyInter
  up_mem := by
    rintro X Y ⟨hX, hyX⟩ hY hXY
    exact ⟨hY, y.up_mem hyX ⟨Y, hY, rfl⟩ ((V.bracket_subset_iff hX hY).mpr hXY)⟩

def toToken (x : V.Element) : V.tokenSystem.Element where
  mem S := ∃ X, x.mem X ∧ S = V.bracket X
  sub := by rintro S ⟨X, hX, rfl⟩; exact ⟨X, x.sub hX, rfl⟩
  master_mem := ⟨V.master, x.master_mem, V.bracket_master.symm⟩
  inter_mem := by
    rintro S T ⟨X, hX, rfl⟩ ⟨Y, hY, rfl⟩
    exact ⟨X ∩ Y, x.inter_mem hX hY, V.bracket_inter (x.sub hX) (x.sub hY)⟩
  up_mem := by
    rintro S T ⟨X, hX, rfl⟩ hT hsub
    obtain ⟨Y, hYmem, rfl⟩ := hT
    exact ⟨Y, x.up_mem hX hYmem ((V.bracket_subset_iff (x.sub hX) hYmem).mp hsub), rfl⟩

def tokenIso : V.Element ≃o V.tokenSystem.Element where
  toFun := V.toToken
  invFun := V.ofToken
  left_inv := by
    intro x
    apply Element.ext
    intro X
    constructor
    · rintro ⟨hX, X', hX', heq⟩
      rw [V.bracket_injective hX (x.sub hX') heq]; exact hX'
    · intro hx
      exact ⟨x.sub hx, X, hx, rfl⟩
  right_inv := by
    intro y
    apply Element.ext
    intro S
    constructor
    · rintro ⟨X, ⟨_, hyX⟩, rfl⟩
      exact hyX
    · intro hyS
      obtain ⟨X, hX, rfl⟩ := y.sub hyS
      exact ⟨X, ⟨hX, hyS⟩, rfl⟩
  map_rel_iff' := by
    intro x x'
```

```
    constructor
    · intro h X hx
      have hmem : (V.toToken x).mem (V.bracket X) := ⟨X, hx, rfl⟩
      obtain ⟨X', hX', heq⟩ := h (V.bracket X) hmem
      rw [V.bracket_injective (x.sub hx) (x'.sub hX') heq]; exact hX'
    · intro h S hS
      obtain ⟨X, hX, rfl⟩ := hS
      exact ⟨X, h X hX, rfl⟩

theorem isomorphic_tokenSystem : V ≅? V.tokenSystem := ⟨V.tokenIso⟩

def filterFixedBy (x : V.Element) : V.tokenSystem.Element := V.toToken x

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.47 Theorem 1.10 — basic open neighbourhoods

THEOREM 1.10. Given any neighbourhood system $\mathcal{D}$, define for $X \in \mathcal{D}$

$$[X] = \{x \in |\mathcal{D}| \mid X \in x\}.$$

The subsets $[X] \subseteq |\mathcal{D}|$ for $X \in \mathcal{D}$ form a neighbourhood system over $|\mathcal{D}|$ which determines a domain isomorphic to $|\mathcal{D}|$.

Proof: We note first that

(1) $[\Delta] = |\mathcal{D}|$.

Next note that

(2) X, Y are consistent in $\mathcal{D}$ iff $[X] \cap [Y] \neq \emptyset$;

and that for $X, Y \in \mathcal{D}$

(3) $[X] \cap [Y] = [X \cap Y]$ if $X \cap Y \in \mathcal{D}$.

Inasmuch as

(4) $\uparrow X \in [X]$ for all $X \in \mathcal{D}$,

it easily follows that $\mathcal{D}$ and the family

$$\{[X] \mid X \in \mathcal{D}\}$$

are in a one-one, inclusion-preserving correspondence. Thus, we can induce the desired one-one correspondence between the elements of the two systems. □

The import of 1.10 is that the original tokens in Δ can be replaced by the elements of $|\mathcal{D}|$. This process replaces the neighbourhood $X \subseteq \Delta$ by the subset $[X] \subseteq |\mathcal{D}|$. As the passage is inclusion preserving, the

domain has not really changed, only its presentation. Though of some theoretical charm, the theorem is not of much use since we still have to get $\mathcal{D}$ from somewhere. It does emphasize, though, that the rôle of the tokens is simply to keep the inclusions (and intersections) of neighbourhoods sorted out. It is *not* always true that the tokens can be identified with the total elements.

The last theorem in this lecture is a result on *closure properties* of a domain with respect to set-theoretical operations which have interesting meanings with respect to approximation.

Theorem (`bracket_inter_nonempty_iff`). Neighbourhoods X and Y are consistent if and only if $[X] \cap [Y]$ is nonempty.

Proof. A common ideal element witnesses a neighbourhood under $X \cap Y$; conversely a principal of such a neighbourhood lies in both brackets. ■

Theorem (`theorem_110`). Clauses (2)–(4) of the proof, together with the token-system isomorphism, hold for every neighbourhood system.

Proof. Assemble the previous theorem with the intersection of brackets, the token-system isomorphism, and membership of principals in their brackets. (Clause (1) is the master bracket.) ■

`Theorem_1_10.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    tokenSystem

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {α : Type*} (V : NeighborhoodSystem α)

theorem bracket_inter_nonempty_iff {X Y : Set α} (hX : V.mem X) (hY : V.mem Y) :
    (V.bracket X ∩ V.bracket Y).Nonempty ↔ ∃ Z, V.mem Z ∧ Z ⊆ X ∩ Y := by
  constructor
  · rintro ⟨x, hxX, hxY⟩
    exact ⟨X ∩ Y, x.sub (x.inter_mem hxX hxY), subset_rfl⟩
  · rintro ⟨Z, hZ, hZsub⟩
    refine ⟨V.principal hZ, ?_, ?_⟩
    · exact ⟨hX, hZsub.trans Set.inter_subset_left⟩
    · exact ⟨hY, hZsub.trans Set.inter_subset_right⟩

/-- Scott's Theorem 1.10: brackets form an isomorphic presentation; clauses (2)-(4)
    plus the
token-system isomorphism. -/
theorem theorem_110 :
    (∀ {α : Type*} (V : NeighborhoodSystem α) {X Y : Set α},
      V.mem X → V.mem Y →
        ((V.bracket X ∩ V.bracket Y).Nonempty ↔ ∃ Z, V.mem Z ∧ Z ⊆ X ∩ Y)) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) {X Y : Set α},
      V.mem X → V.mem Y → V.bracket X ∩ V.bracket Y = V.bracket (X ∩ Y)) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α), V ≅? V.tokenSystem) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) {X : Set α} (hX : V.mem X),
      V.principal hX ∈ V.bracket X) :=
```

```
  ⟨bracket_inter_nonempty_iff, bracket_inter, isomorphic_tokenSystem,
    principal_mem_bracket⟩

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.48 Countable intersection of ideal elements

The countable intersection $\bigcap_n x_n$ of ideal elements is again an ideal element: a neighbourhood belongs to the meet precisely when it belongs to every x_n. It is the greatest lower bound of the sequence in the approximation order.

Definition (`iInter`). The countable intersection of a sequence of ideal elements is the filter of neighbourhoods belonging to every term.

Theorem (`mem_iInter`). A neighbourhood belongs to the countable intersection if and only if it belongs to every term of the sequence.

Theorem (`iInter_le`). The countable intersection is below each term of the sequence.

Proof. Membership in the meet means membership in that term. ■

Theorem (`le_iInter`). Any common lower bound of the sequence is below the countable intersection.

Proof. A neighbourhood of the bound belongs to every term, hence to the meet. ■

`iInter.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import Mathlib.Data.Nat.Basic
public import Mathlib.Tactic.Attr.Core

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {α : Type*} (V : NeighborhoodSystem α)

/-- Countable intersection of ideal elements: a neighbourhood belongs to the meet
iff it belongs to every term of the sequence. -/
def iInter (x : ℕ → V.Element) : V.Element where
  mem X := ∀ n, (x n).mem X
  sub h := (x 0).sub (h 0)
  master_mem n := (x n).master_mem
  inter_mem h1 h2 n := (x n).inter_mem (h1 n) (h2 n)
  up_mem h hY hXY n := (x n).up_mem (h n) hY hXY

@[simp] theorem mem_iInter (x : ℕ → V.Element) {X : Set α} :
```

```
    (V.iInter x).mem X ↔ ∀ n, (x n).mem X := Iff.rfl

theorem iInter_le (x : ℕ → V.Element) (n : ℕ) : V.iInter x ≤ x n :=
  fun _ h => h n

theorem le_iInter (x : ℕ → V.Element) (y : V.Element) (h : ∀ n, y ≤ x n) :
    y ≤ V.iInter x :=
  fun X hX n => h n X hX

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.49 Directed union of ideal elements

Given an increasing sequence $x_0 \subseteq x_1 \subseteq \cdots$ of ideal elements, the directed union $\bigcup_n x_n$ is again an ideal element: a neighbourhood belongs to the join precisely when it belongs to some x_n. It is the least upper bound of the sequence in the approximation order. (Monotonicity is needed so that neighbourhoods drawn from different stages still have a common later stage in which their intersection may be formed.)

Definition (`iUnion`). The directed union of a monotone sequence of ideal elements is the filter of neighbourhoods belonging to some term.

Theorem (`mem_iUnion`). A neighbourhood belongs to the directed union if and only if it belongs to some term of the sequence.

Theorem (`le_iUnion`). Each term of the sequence is below the directed union.

Proof. Membership in that term witnesses membership in the join. ■

Theorem (`iUnion_le`). The directed union is below every upper bound of the sequence.

Proof. A neighbourhood of the join comes from some term, hence belongs to any common upper bound. ■

Definition (`sSupDirected`). The directed supremum of a nonempty directed family of ideal elements is the filter of neighbourhoods belonging to some member of the family.

Theorem (`le_sSupDirected`). Each member of the family is below the directed supremum.

Theorem (`sSupDirected_le`). The directed supremum is below every upper bound of the family.

Proof. As for the countable monotone case: membership in the join is witnessed by a family member. ■

`iUnion.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import Mathlib.Order.Monotone.Basic
public import Mathlib.Tactic.Attr.Core

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section
```

```
variable {α : Type*} (V : NeighborhoodSystem α)

/-- Directed (monotone) countable union of ideal elements: a neighbourhood belongs to
    the
join iff it belongs to some term of the sequence. -/
def iUnion (x : ℕ → V.Element) (hmono : Monotone x) : V.Element where
  mem X := ∃ n, (x n).mem X
  sub := by rintro X ⟨n, hn⟩; exact (x n).sub hn
  master_mem := ⟨0, (x 0).master_mem⟩
  inter_mem := by
    rintro X Y ⟨n, hn⟩ ⟨m, hm⟩
    refine ⟨max n m, (x (max n m)).inter_mem ?_ ?_⟩
    · exact hmono (le_max_left n m) X hn
    · exact hmono (le_max_right n m) Y hm
  up_mem := by
    rintro X Y ⟨n, hn⟩ hY hXY
    exact ⟨n, (x n).up_mem hn hY hXY⟩

@[simp] theorem mem_iUnion (x : ℕ → V.Element) (hmono : Monotone x) {X : Set α} :
    (V.iUnion x hmono).mem X ↔ ∃ n, (x n).mem X := Iff.rfl

theorem le_iUnion (x : ℕ → V.Element) (hmono : Monotone x) (n : ℕ) :
    x n ≤ V.iUnion x hmono :=
  fun _ hX => ⟨n, hX⟩

theorem iUnion_le (x : ℕ → V.Element) (hmono : Monotone x) (y : V.Element)
    (h : ∀ n, x n ≤ y) : V.iUnion x hmono ≤ y := by
  rintro X ⟨n, hn⟩
  exact h n X hn

/-- Directed supremum of a nonempty directed family of ideal elements: a neighbourhood
belongs to the join iff it belongs to some member of the family. -/
def sSupDirected (V : NeighborhoodSystem α) (S : Set V.Element) (hne : S.Nonempty)
    (hdir : ∀ a ∈ S, ∀ b ∈ S, ∃ c ∈ S, a ≤ c ∧ b ≤ c) : V.Element where
  mem Z := ∃ s ∈ S, s.mem Z
  sub := fun ⟨s, _, hs⟩ => s.sub hs
  master_mem := by obtain ⟨s, hs⟩ := hne; exact ⟨s, hs, s.master_mem⟩
  inter_mem := by
    rintro Z Z' ⟨a, haS, haZ⟩ ⟨b, hbS, hbZ⟩
    obtain ⟨c, hcS, hac, hbc⟩ := hdir a haS b hbS
    exact ⟨c, hcS, c.inter_mem (hac Z haZ) (hbc Z' hbZ)⟩
  up_mem := by
    rintro Z Z' ⟨a, haS, haZ⟩ hZ' hZZ'
    exact ⟨a, haS, a.up_mem haZ hZ' hZZ'⟩

theorem le_sSupDirected (V : NeighborhoodSystem α) (S : Set V.Element) (hne : S.
    Nonempty)
    (hdir : ∀ a ∈ S, ∀ b ∈ S, ∃ c ∈ S, a ≤ c ∧ b ≤ c) {a : V.Element} (ha : a ∈ S) :
    a ≤ V.sSupDirected S hne hdir :=
  fun _ hZ => ⟨a, ha, hZ⟩

theorem sSupDirected_le (V : NeighborhoodSystem α) (S : Set V.Element) (hne : S.
    Nonempty)
    (hdir : ∀ a ∈ S, ∀ b ∈ S, ∃ c ∈ S, a ≤ c ∧ b ≤ c) {y : V.Element}
    (hy : ∀ s ∈ S, s ≤ y) : V.sSupDirected S hne hdir ≤ y := by
```

```
  rintro Z ⟨s, hs, hsZ⟩
  exact hy s hs Z hsZ

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

4.2.50 Theorem 1.11 — intersections and directed unions

THEOREM 1.11. If $\mathcal{D}$ is a neighbourhood system and $x_n \in |\mathcal{D}|$ for $n = 0, 1, 2, \ldots$, then

(i) $\bigcap_{n=0}^{\infty} x_n \in |\mathcal{D}|$; and

(ii) $\bigcup_{n=0}^{\infty} x_n \in |\mathcal{D}|$, provided

$$x_0 \subseteq x_1 \subseteq x_2 \subseteq \cdots \subseteq x_n \subseteq x_{n+1} \subseteq \cdots .$$

Proof: The conditions of 1.6 have to be checked. For the case of intersection, all of 1.6(i)–(iii) are quite obvious. For the case of union, only 1.6(ii) gives pause and it requires the proviso. If X and Y belong to the union, then $X \in x_n$, say, and $Y \in x_m$. But, either $n < m$ or $m < n$, and if $k = \max(n, m)$, then $X, Y \in x_k$. Since $x_k \in |\mathcal{D}|$, we have $X \cap Y \in x_k$; thus, $X \cap Y$ belongs to the union. This proves (ii). □

In words, the intersection is the best element that is at the same time an approximation to all of the elements x_n; the intersection is exactly what is common to all the given elements. The union on the other hand is just what the (increasing sequence of the) x_n approximates; the union combines contributions from all the x_n into a "better" element – but no more than that.

In thinking about domains a rough diagram of the partial-ordering relation $\subseteq$ between elements is often helpful. The picture of 1.4 is an example where the nodes represent the elements. Any finite tree growing up from a root node would also be an example. Indeed, any finite partially ordered set with least element would be an example. (Here no distinction between tokens and elements is necessary.) A lattice diagram is also illustrated.

The root node is the element $\bot$ of $|\mathcal{D}|$; there need be no top node $\top$. *Approximation* is represented by a passage from a lower node to a higher node along the rising lines. The system $\mathcal{D}$ of neighbourhoods is the collection of sets each of which is all the nodes above a given node. For infinite examples, however, care must be given to introduce limit nodes. The first few exercises should be provided with pictures to illustrate the structure.

Theorem (`theorem_111`). Countable intersections of ideal elements, and directed (monotone) countable unions, are again ideal elements, characterized by pointwise membership.

Proof. Assemble the membership characterizations of the countable intersection and of the directed union. ■

A TREE

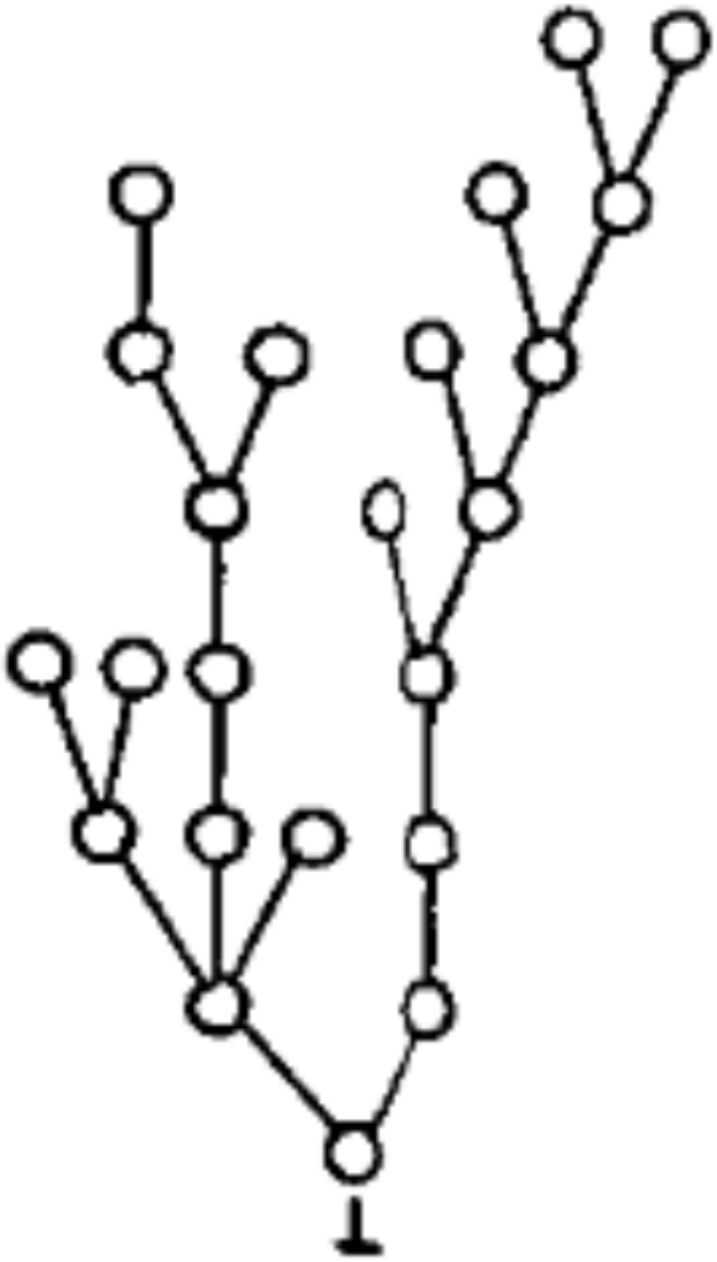


Figure 43: A tree

A ROUGH PICTURE

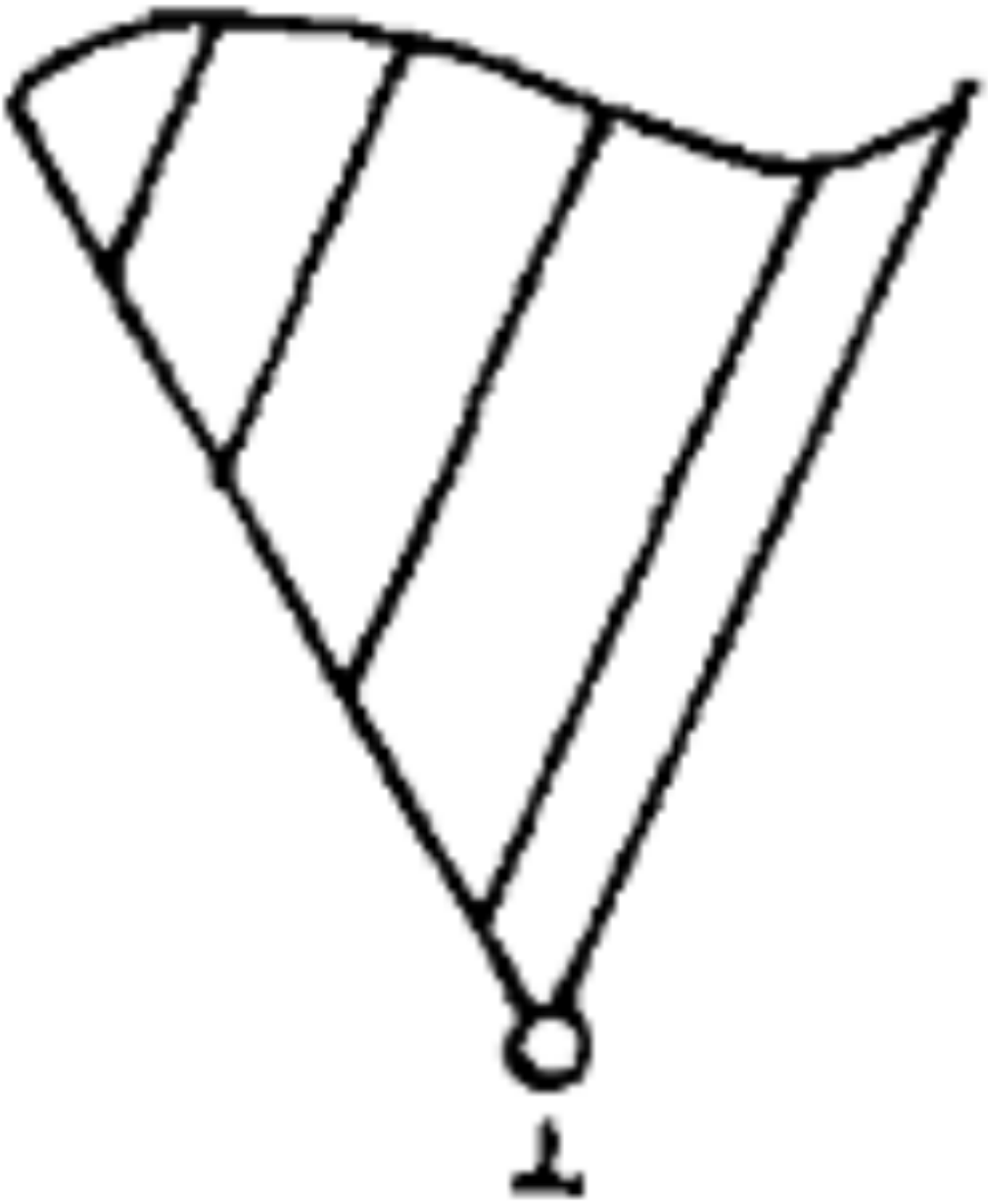


Figure 44: A rough picture

A LATTICE

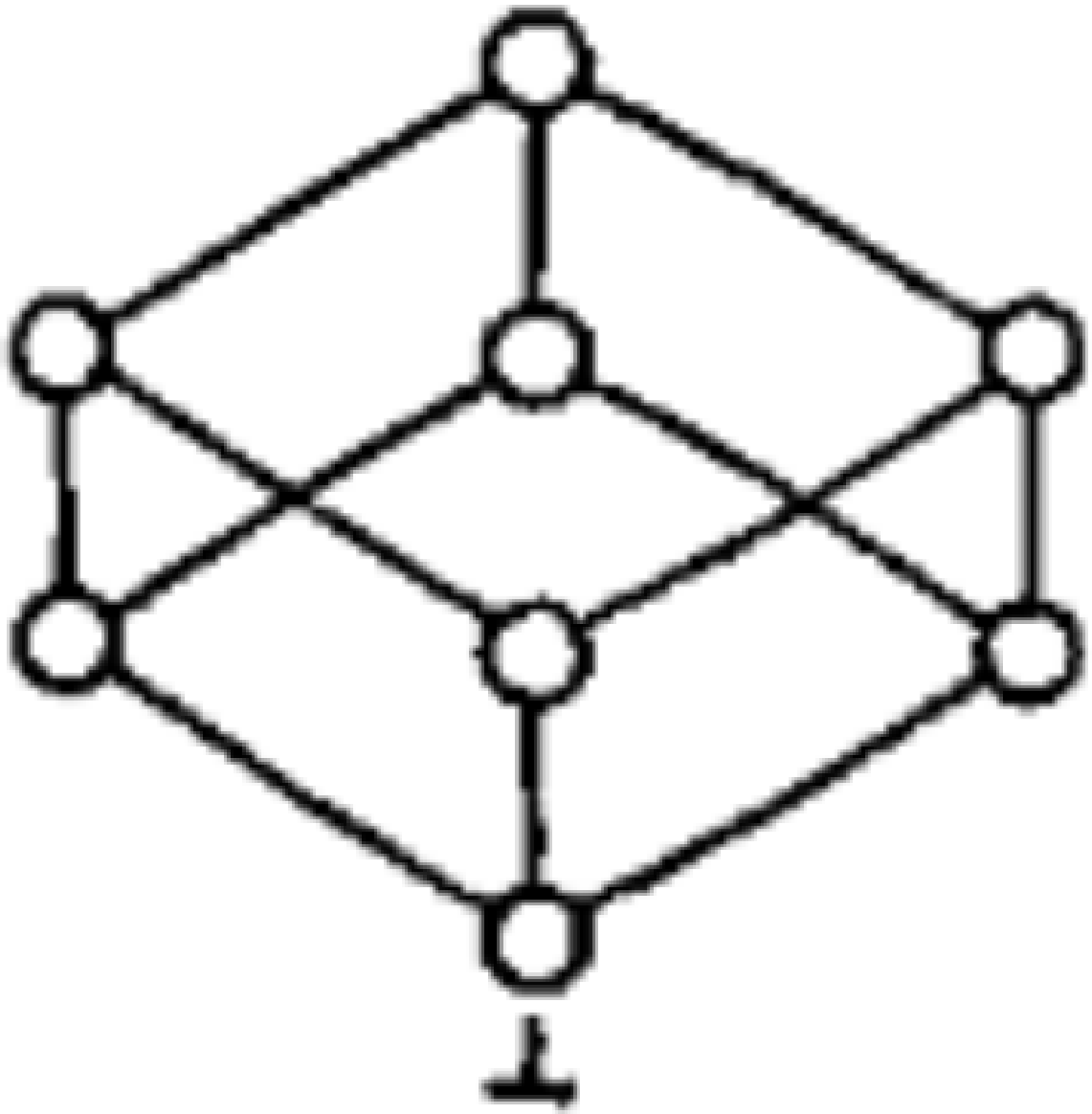


Figure 45: A lattice

Theorem_1_11.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.iInter
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.iUnion

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
```

```
namespace NeighborhoodSystem

/-- Scott's Theorem 1.11: countable intersections of ideal elements, and directed
(monotone) countable unions, are again ideal elements -- characterized by
pointwise membership. -/
theorem theorem_111 :
    (∀ {α : Type*} (V : NeighborhoodSystem α) (x : ℕ → V.Element) {X : Set α},
      (V.iInter x).mem X ↔ ∀ n, (x n).mem X) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (x : ℕ → V.Element) (hmono : Monotone x
    )
      {X : Set α}, (V.iUnion x hmono).mem X ↔ ∃ n, (x n).mem X) :=
  ⟨mem_iInter, mem_iUnion⟩

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
```

4.2.51 Lecture I exercises

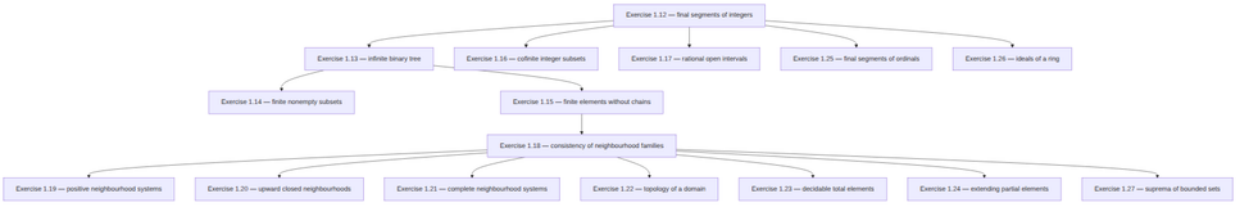


Figure 46: Lean module dependencies for Lecture I exercises.

Exercise 1.12 — final segments of integers Let $\Delta = \mathbb{N} = \{0, 1, 2, \ldots, n, \ldots\}$ be the set of non-negative integers. Use as neighbourhoods final segments:

$$\{m \in \mathbb{N} \mid m > n\}$$

for $n \in \mathbb{N}$. Verify that this is a neighbourhood system. What are the total elements? What are the finite elements? Draw a picture of the approximation relation in this domain. (Hint: there is only one limit element.)

Definition (`tail`). The final segment of threshold n is $\{m \in \mathbb{N} \mid n \leq m\}$.

Definition (`mem`). A set is a neighbourhood precisely when it equals some final segment.

Theorem (`nestedOrDisjoint`). Any two final segments are nested or disjoint (in fact nested).

Proof. Compare thresholds: the larger threshold yields the smaller segment. ■

Definition (`neighborhoodSystem`). The final-segment neighbourhood system on $\mathbb{N}$.

Definition (`fin`). The principal filter of the final segment of threshold n.

Definition (`top`). The filter of *all* final segments — the unique limit element.

Theorem (`element_eq`). Every ideal element is either some finite principal $\mathrm{fin}\, n$ or else top.

Proof. If the filter meets every final segment it is top; otherwise the least missing threshold determines a principal. ■

Theorem (`isTotal_iff_top`). An ideal element is total if and only if it equals top.

Proof. Everything approximates top, and top is maximal. ■

Theorem (`exercise_1_12`). Final segments form a nested-or-disjoint neighbourhood system, and the only total element is the limit top.

Proof. Assemble nestedness of final segments with the characterization of totality. ■

`Exercise_1_12.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.TailNat

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.TailNat
/-- Scott's Exercise 1.12: final segments of ℕ form a neighbourhood system whose
    only total element is the limit top. -/
theorem exercise_1_12 :
    (NestedOrDisjoint mem ∧
      ∀ x : neighborhoodSystem.Element, neighborhoodSystem.IsTotal x ↔ x = top) :=
  ⟨nestedOrDisjoint, isTotal_iff_top⟩
```

Exercise 1.13 — infinite binary tree Verify all the assertions made about the system B defined as the infinite generalization of Example 1.4. Draw a picture similar to that given in the text which includes nodes for all $\sigma \in \Sigma^*$. Show the neighbourhoods, how the approximation relation behaves, and where the total elements lie. (The picture is closely related to the "binary tree", but has to have limit nodes all along the top.)

Definition (`prefixSeq`). The length-n initial segment of an infinite bit path p is the finite string $p{\restriction}n$.

Definition (`branchSeq`). The finite approximations along p are the principals $(p{\restriction}n)\ \bot$.

Theorem (`branchSeq_mono`). Along any path the sequence of principals is monotone.

Proof. Longer prefixes refine shorter ones, and principals compare by reverse inclusion of cones. ■

Definition (`branch`). The limit node of a path is the directed union of its finite approximations.

Theorem (`branch_mem_iff`). The cone of σ belongs to the branch of p if and only if σ prefixes some $p{\restriction}n$.

Proof. Unwind membership in the directed union and the characterisation of cone membership for principals. ■

Theorem (`prefix_comparable_of_mem`). If an ideal element contains both $\sigma\Sigma^*$ and $\tau\Sigma^*$, then σ and τ are prefix-comparable.

Proof. Their cones intersect in a neighbourhood, so trichotomy of cones excludes the disjoint case. ■

Theorem (`branch_isTotal`). Every infinite path yields a total element of B.

Proof. Any refinement must accept every cone already present on the branch; prefix-comparability forces those cones to lie along the same path. ■

Theorem (`exercise_1_13`). For every infinite bit path p, the corresponding branch is a total element of B.

Proof. Assemble totality of branches. ■

`Exercise_1_13.lean`

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.BranchSeq

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.BranchSeq
open NeighborhoodSystem BitDom

/-- Scott's Exercise 1.13: each infinite branch through binary sequences is total. -/
theorem exercise_1_13 : ∀ p : ℕ → Bool, B.IsTotal (branch p) :=
  branch_isTotal
```

Exercise 1.14 — finite nonempty subsets Let $\Delta = \mathbb{N}$ and let $\mathcal{V}$ be the family of finite non-empty subsets of Δ plus the set Δ. Show that this is a neighbourhood system. What are the total elements? What are the finite elements? Draw a picture.

Definition (`mem`). A set is a neighbourhood when it is either all of $\mathbb{N}$ or a nonempty finite subset.

Definition (`neighborhoodSystem`). The neighbourhood system of finite nonempty subsets of $\mathbb{N}$ (together with the master set).

Proof of the axioms. The master set is $\mathbb{N}$. Intersections of neighbourhoods remain nonempty finite or reduce to a factor when one factor is the universe. ■

Definition (`fin`). The principal filter of a neighbourhood.

Theorem (`mem_singleton`). Every singleton is a neighbourhood.

Theorem (`singleton_isTotal`). The principal of a singleton is a total element.

Proof. Any refinement must contain that singleton; intersecting with a neighbourhood that omits the point would yield the empty set, which is not a neighbourhood. ■

Theorem (`exercise_1_14`). For every $n \in \mathbb{N}$, the principal of $\{n\}$ is total.

Proof. Assemble totality of singleton principals. ■

`Exercise_1_14.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    FinNonempty

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.FinNonempty
/-- Scott's Exercise 1.14: singletons are total in the finite-nonempty system. -/
theorem exercise_1_14 : ∀ n : ℕ,
    neighborhoodSystem.IsTotal (neighborhoodSystem.principal (mem_singleton n)) :=
  singleton_isTotal
```

Exercise 1.15 — finite elements without chains Construct non-isomorphic infinite domains where all elements are finite but where there are no infinite chains $\langle x_n \rangle_{n=0}^{\infty}$ of elements with $x_n \sqsubseteq x_{n+1}$ but $x_n \neq x_{n+1}$ for all n.

Definition (`flatMem`). Neighbourhoods of the flat system are the universe together with the singletons.

Definition (`flat`). The flat neighbourhood system on $\mathbb{N}$.

Theorem (`flat_all_finite`). Every ideal element of the flat system is principal (hence finite).

Proof. Classify elements as bottom or a singleton principal. ■

Theorem (`flat_no_infinite_chain`). The flat system admits no strictly monotone $\mathbb{N}$-indexed chain of elements.

Proof. There is not even a strict three-element chain: atoms are maximal above bottom. ■

Definition (`stemMem`). Neighbourhoods of the stem system are the universe, the pair $\{0, 1\}$, and the singletons.

Definition (`stem`). The stem neighbourhood system on $\mathbb{N}$.

Theorem (`stem_three_chain`). The stem system has a strict three-element chain of principals.

Proof. Compare $\uparrow \Delta$, $\uparrow \{0, 1\}$, and $\uparrow \{0\}$ by reverse inclusion. ■

Theorem (`not_isomorphic`). The flat and stem domains are not isomorphic.

Proof. An isomorphism would transport the stem's three-chain into the flat domain. ■

Theorem (`exercise_1_15`). The flat and stem domains are not isomorphic — two infinite domains of finite elements, one without and one with a short strict chain.

Proof. Assemble non-isomorphism of flat and stem. ■

`Exercise_1_15.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.FlatStem

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.FlatStem
/-- Scott's Exercise 1.15: the flat and stem systems are not isomorphic. -/
theorem exercise_1_15 : ¬ (flat ≅? stem) :=
  not_isomorphic
```

Exercise 1.16 — cofinite integer subsets Let $\Delta = \mathbf{N}$ and let $\mathcal{D}$ be the family of *cofinite* subsets of $\mathbf{N}$. Show that $|\mathcal{D}|$ is isomorphic to the partially ordered set of *all* subsets of $\mathbf{N}$ under inclusion. Construct some other neighbourhood systems where $\mathcal{D}$ is closed under finite intersection. What happens to the total elements in such systems?

Definition (`cofiniteSystem`). The neighbourhood system whose members are the cofinite subsets of $\mathbb{N}$.

Definition (`ofExcluded`). From an arbitrary set $E \subseteq \mathbb{N}$ form the filter of cofinite neighbourhoods whose complements lie in E.

Definition (`toExcluded`). From an ideal element recover the set of points whose singleton-complements belong to the filter.

Theorem (`cofiniteIso`). The map sending an ideal element to its excluded set is an order isomorphism between $|\mathcal{D}|$ and $(\mathcal{P}(\mathbb{N}), \subseteq)$.

Proof. The two constructions are mutually inverse, and approximation corresponds to inclusion of excluded sets. ■

Definition (`fullSystem`). The neighbourhood system in which every subset is a neighbourhood (closed under all intersections).

Theorem (`ofExcluded_univ_isTotal`). Excluding the whole of $\mathbb{N}$ yields a total element of the cofinite system.

Proof. Every ideal element approximates that filter. ■

Theorem (`exercise_1_16`). The cofinite system has a total element — namely the filter of all cofinite sets.

Proof. Assemble totality of the universal excluded set. ■

`Exercise_1_16.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Cofinite

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.Cofinite
/-- Scott's Exercise 1.16: the cofinite system has a total element (exclude nothing).
    -/
theorem exercise_1_16 : cofiniteSystem.IsTotal (ofExcluded Set.univ) :=
  ofExcluded_univ_isTotal
```

Exercise 1.17 — rational open intervals Let $\Delta = \mathbf{R}$ be the real line. Let $\mathcal{D}$ be the set of non-empty open intervals with rational end points plus the set Δ. Show that this is a neighbourhood system. For any real $t \in \mathbf{R}$, show that $\{X \in \mathcal{D} \mid t \in X\}$ is a filter. Is it always total? What are the total elements of $|\mathcal{D}|$? (Hint: When t is rational consider all intervals with t as a right-hand end point.)

Definition (`ratIntervalMem`). A set is a neighbourhood when it is either $\mathbb{R}$ or a nonempty open interval with rational endpoints.

Theorem (`inter_mem'`). The intersection of two neighbourhoods, when nonempty, is again a neighbourhood.

Proof. Intersecting rational open intervals yields a (possibly empty) rational open interval; nonempty intersection keeps the endpoints rational. ■

Definition (`ratIntervalSystem`). The neighbourhood system of rational open intervals on $\mathbb{R}$.

Definition (`filterAt`). For each real t, the filter of all neighbourhoods containing t.

Theorem (`filterAt_injective`). Distinct reals determine distinct point-filters.

Proof. Separate the two points by a rational open interval that contains one but not the other. ■

Theorem (`exercise_1_17`). Neighbourhoods are closed under nonempty intersection, and the point-filters separate points of $\mathbb{R}$.

Proof. Assemble the intersection lemma with injectivity of point-filters. ■

`Exercise_1_17.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    RatInterval

@[expose] public section

open DomainTheory.NeighbourhoodSystems
```

```
open DomainTheory.NeighbourhoodSystems.RatInterval
/-- Scott's Exercise 1.17: rational open intervals separate reals and are intersection
    -closed
    on nonempty meets. -/
theorem exercise_1_17 :
    (∀ {s t : ?}, filterAt s = filterAt t → s = t) ∧
      (∀ {X Y : Set ?}, ratIntervalMem X → ratIntervalMem Y → (X ∩ Y).Nonempty →
        ratIntervalMem (X ∩ Y)) :=
  ⟨filterAt_injective, inter_mem'⟩
```

Exercise 1.18 — consistency of neighbourhood families Let $\mathcal{D}$ be a neighbourhood system. Call a subset $C \subseteq \mathcal{D}$ *consistent* iff every finite subset of C is consistent in $\mathcal{D}$. Give an example where C is a subset with more than two elements, every pair of neighbourhoods in C is consistent, but C is *not* consistent. Show that if C is consistent, then there is a *least* filter $x \in |\mathcal{D}|$ with $C \subseteq x$. Show generally that the *intersection* of any non-empty collection of filters is again a filter.

Definition (`FinitelyConsistent`). A family C of neighbourhoods is finitely consistent when every finite sequence drawn from C is consistent in $\mathcal{D}$.

Definition (`leastFilter`). From a finitely consistent family C form the least ideal element containing every member of C.

Theorem (`leastFilter_le`). Any ideal element containing all of C lies above the least filter of C.

Proof. Membership in the least filter is forced by every common upper bound. ■

Definition (`sInf`). The intersection of a nonempty family of ideal elements is again an ideal element.

Theorem (`sInf_le` / `le_sInf`). Pointwise intersection is the greatest lower bound in the approximation order.

Definition (`family`). On a three-point token set, the three pairwise-overlapping doubletons $\{0,1\}$, $\{1,2\}$, $\{0,2\}$.

Theorem (`not_finitelyConsistent`). That three-set family is pairwise nonempty but not finitely consistent.

Proof. The length-three intersection is empty, so no neighbourhood witnesses consistency of the whole family. ■

Theorem (`exercise_1_18`). Least filters of finitely consistent families are least upper bounds for those families, and the three-doubleton family is a counterexample to pairwise consistency implying finite consistency.

Proof. Assemble the least-filter inequality with the triangle counterexample. ■

Exercise_1_18.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.TriSys

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
open DomainTheory.NeighbourhoodSystems.TriSys
```

```
/-- Scott's Exercise 1.18: least filters for finitely consistent families, and a
    counterexample. -/
theorem exercise_1_18 :
    (∀ {α : Type*} (V : NeighborhoodSystem α) (C : Set (Set α))
        (hCsub : ∀ X ∈ C, V.mem X) (hC : V.FinitelyConsistent C) (z : V.Element),
      (∀ W ∈ C, z.mem W) → V.leastFilter C hCsub hC ≤ z) ∧
      ¬ system.FinitelyConsistent family :=
  ⟨NeighborhoodSystem.leastFilter_le, not_finitelyConsistent⟩
```

Exercise 1.19 — positive neighbourhood systems Define a *positive neighbourhood system* to be a family $\mathcal{D}$ where (ii) of 1.1 is replaced by **(ii')** whenever $X, Y \in \mathcal{D}$, then $X \cap Y \neq \emptyset$ iff $X \cap Y \in \mathcal{D}$. Prove that a positive neighbourhood system is indeed a neighbourhood system in the sense of the earlier definition. Give an example of a neighbourhood system that is *not* positive. (Hint: (suggested by C.A.R. Hoare). Let $\Delta = \mathbf{N} \times \mathbf{N}$, in the plane. Let $\mathcal{D}$ be the family of subsets $X \subseteq \mathbf{N} \times \mathbf{N}$ where all but a finite number of places the *vertical* sections of X are the whole of $\mathbf{N}$ but at the other places the sections are finite and nonempty. Smaller examples are of course possible.)

Definition (`IsPositive`). A neighbourhood system is positive when membership of an intersection is equivalent to nonempty intersection, for any two members.

Definition (`ofPositive`). From a positive intersection law one obtains a neighbourhood system in the sense of Definition 1.1.

Proof. Nonempty intersection of members yields a member by positivity, which discharges the witness required by the ordinary intersection axiom. ■

Definition (`notPositiveSystem`). On a three-token set, the neighbourhoods are the universe together with the doubletons $\{0, 1\}$ and $\{1, 2\}$.

Theorem (`not_isPositive`). That system is not positive.

Proof. The two doubletons meet in $\{1\}$, which is nonempty but not a neighbourhood. ■

Definition (`positiveExample`). The system of all nonempty subsets of a two-point set (built via positivity).

Theorem (`positiveExample_isPositive`). That system is positive.

Proof. Nonempty subsets are closed under nonempty intersection. ■

Theorem (`exercise_1_19`). There is a neighbourhood system that fails positivity, and there is a positive example.

Proof. Assemble the two examples. ■

Exercise_1_19.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    PositiveDiff

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.PositiveDiff
/-- Scott's Exercise 1.19: a counterexample that is not positive, and a positive
    example. -/
theorem exercise_1_19 :
    ¬ notPositiveSystem.IsPositive ∧ positiveExample.IsPositive :=
```

```
⟨not_isPositive, positiveExample_isPositive⟩
```

Exercise 1.20 — upward closed neighbourhoods Let $\mathcal{D}$ be any neighbourhood system over a set Δ. Let $\Delta' = \mathcal{D}$ and define

$$\mathcal{D}' = \{\uparrow X \mid X \in \mathcal{D}\},$$

where

$$\uparrow X = \{Y \in \mathcal{D} \mid Y \subseteq X\}.$$

Show that $\mathcal{D}'$ is a positive neighbourhood system and that $|\mathcal{D}|$ and $|\mathcal{D}'|$ are isomorphic. Note that for $\mathcal{D}'$ finite elements and tokens are in a one-one correspondence.

Definition (`upSet`). For $X \in \mathcal{D}$, write $\uparrow X = \{Y \in \mathcal{D} \mid Y \subseteq X\}$.

Theorem (`upSet_inter`). Up-sets commute with intersection: $\uparrow X \cap \uparrow Y = \uparrow (X \cap Y)$.

Proof. Expand the definitions. ■

Definition (`powerSystem`). The neighbourhood system $\mathcal{D}'$ whose tokens are the members of $\mathcal{D}$ and whose neighbourhoods are the up-sets $\uparrow X$.

Definition (`toPower` / `ofPower`). The mutually inverse translations between ideal elements of $\mathcal{D}$ and of $\mathcal{D}'$.

Theorem (`isomorphic_powerSystem`). $|\mathcal{D}|$ and $|\mathcal{D}'|$ are isomorphic as domains.

Proof. The order isomorphism `powerIso` identifies approximation with inclusion of up-sets. ■

Theorem (`powerSystem_isPositive`). The system $\mathcal{D}'$ is positive.

Proof. Nonempty intersection of up-sets yields an up-set of an intersection of members. ■

Theorem (`toPower_principal`). Principals of $\mathcal{D}$ correspond to principals of $\mathcal{D}'$ under the isomorphism — finite elements match tokens.

Theorem (`exercise_1_20`). The power system is positive and domain-isomorphic to the original, with principals corresponding.

Proof. Assemble the isomorphism, positivity, and principal correspondence. ■

`Exercise_1_20.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
variable {α : Type*} (V : NeighborhoodSystem α)

/-- Scott's Exercise 1.20: a system is isomorphic to its power system; the power
    system
    is positive; principals map to principals. -/
theorem exercise_1_20 :
    (V ≅? V.powerSystem) ∧
      V.powerSystem.IsPositive ∧
        ∀ {X : Set α} (hX : V.mem X),
```

```
        V.toPower (V.principal hX) = V.powerSystem.principal ⟨X, hX, rfl⟩ :=
  ⟨V.isomorphic_powerSystem, V.powerSystem_isPositive, V.toPower_principal⟩
```

Exercise 1.21 — complete neighbourhood systems Work out in greater detail the proof of 1.10. Remark that the neighbourhood system over $|\mathcal{D}|$ so constructed is positive, thereby obtaining in a different way the same kind of conclusion as in 1.20. Show also that the system over $|\mathcal{D}|$ is complete in the sense that every filter is fixed by a unique member of the underlying set. (A filter is *fixed by a point* iff it is the filter of *all* neighbourhoods containing that point.) Remark that a complete system is one where tokens and (partial) elements can always be identified (under a suitable one-one correspondence). Show also that consistency of a set $\{X_i \mid i < n\}$ of neighbourhoods in $\mathcal{D}$ is equivalent to saying

$$\bigcap_{i<n} [X_i] \neq \emptyset.$$

Definition (`IsComplete`). A neighbourhood system is complete when every ideal element is the filter of all neighbourhoods containing a unique token.

Theorem (`bracket_nonempty_iff`). The bracket $[X]$ is nonempty if and only if X is a neighbourhood.

Proof. Any element of $[X]$ forces X to be a member; conversely the principal of X lies in $[X]$. ■

Theorem (`consistent_iff_iInter_bracket_nonempty`). A finite sequence of neighbourhoods is consistent if and only if the intersection of their brackets is nonempty.

Proof. Consistency is equivalent to membership of the finite intersection, hence to nonempty bracket of that intersection, which unwraps as a common point of the individual brackets. ■

Theorem (`tokenSystem_complete`). The token system over $|\mathcal{D}|$ is complete.

Proof. Each ideal element of the token system is fixed by the unique underlying element of $|\mathcal{D}|$ recovered by `ofToken`. ■

Theorem (`tokenSystem_isPositive`). The token system is positive.

Proof. Intersection of brackets is the bracket of the intersection. ■

Theorem (`tokenSystem_toToken_bijective`). Tokens of the token system biject with ideal elements of $\mathcal{D}$.

Proof. The domain isomorphism of Theorem 1.10 is bijective on underlying sets. ■

Theorem (`exercise_1_21`). Consistency is characterised by nonempty bracket intersections; the token system is complete and positive; and tokens identify with elements.

Proof. Assemble the consistency equivalences with completeness, positivity, and bijectivity of `toToken`. ■

`Exercise_1_21.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
variable {α : Type*} (V : NeighborhoodSystem α)
```

```
/-- Scott's Exercise 1.21: consistency via brackets; token system complete and
    positive;
    toToken is bijective. -/
theorem exercise_1_21 :
    (∀ (X : ℕ → Set α) {n : ℕ}, (∀ i, i < n → V.mem (X i)) →
        (V.Consistent X n ↔ (V.bracket (V.interUpTo X n)).Nonempty)) ∧
      (∀ (X : ℕ → Set α) {n : ℕ}, (∀ i, i < n → V.mem (X i)) →
        (V.Consistent X n ↔ ∃ x, ∀ i, i < n → x ∈ V.bracket (X i))) ∧
      V.tokenSystem.IsComplete ∧
      V.tokenSystem.IsPositive ∧
      Function.Bijective V.toToken :=
  ⟨V.consistent_iff_bracket_interUpTo_nonempty, V.
    consistent_iff_iInter_bracket_nonempty,
    V.tokenSystem_complete, V.tokenSystem_isPositive, V.tokenSystem_toToken_bijective⟩
```

Exercise 1.22 — topology of a domain (For topologists). Show that the neighbourhoods $[X]$ for $X \in \mathcal{D}$ make $|\mathcal{D}|$ into a topological space where the open subsets $\mathcal{U} \subseteq |\mathcal{D}|$ can be characterized by the following two conditions:

(i) whenever $x \in \mathcal{U}$ and $x \sqsubseteq y \in |\mathcal{D}|$, then $y \in \mathcal{U}$; and

(ii) whenever $x \in \mathcal{U}$, then $[X] \in \mathcal{U}$ for some $X \in x$.

Prove also that the inclusion relation on $|\mathcal{D}|$ can be defined topologically as:

(iii) $x \sqsubseteq y$ iff for all open $\mathcal{U} \subseteq |\mathcal{D}|$, if $x \in \mathcal{U}$ then $y \in \mathcal{U}$.

Is $|D|$ ever a Hausdorff space?

Show that if $(x_n)_{n=0}^{\infty}$ is a sequence of elements of $|D|$ with $x_n \subseteq x_{n+1}$ for all n, then

$$\bigcup_{n=0}^{\infty} x_n$$

is not only in $|D|$ but is a topological limit point of the sequence. Show that any element x is a limit point of the set $\{\uparrow X \mid X \in x\}$. Are there other limit points?

Definition (`basicOpen`). The basic open of a set X is $[X] = \{x \in |\mathcal{D}| \mid X \in x\}$.

Definition (`OpenFilter`). A set of ideal elements is open in the Scott sense when every member has a basic open neighbourhood contained in the set.

Instance (`instTopologicalSpaceElement`). Open filters form a topology on $|\mathcal{D}|$.

Proof. The universe is open via the master neighbourhood; binary intersections use pairwise intersection of neighbourhoods; arbitrary unions inherit witnesses from a component. ■

Theorem (`isOpen_basicOpen`). Every basic open is open.

Theorem (`isOpen_iff_upper_basic`). Open sets are exactly the upper sets that contain a basic open around each of their points — Scott's clauses (i) and (ii).

Proof. Unfold the open-filter definition against upward closure of filters. ■

Theorem (`le_iff_isOpen_imp`). Approximation is topological: $x \leq y$ if and only if every open containing x contains y — clause (iii).

Proof. Opens are upper; conversely test against basic opens. ■

Theorem (`specializes_iff_le`). Specialization in this topology coincides with approximation.

Theorem (`exercise_1_22`). Opens are upper basic-neighbourhood sets; basic opens are open; approximation is the open-membership order; specialization is approximation.

Proof. Assemble the preceding characterizations. ■

`Exercise_1_22.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
variable {α : Type*} (V : NeighborhoodSystem α)

/-- Scott's Exercise 1.22: Scott opens are upper sets generated by basic opens;
    specialisation
    coincides with the approximation order. -/
theorem exercise_1_22 :
    (∀ {U : Set V.Element}, IsOpen U → ∀ {{x y : V.Element}}, x ∈ U → x ≤ y → y ∈ U)
    ∧
      (∀ X : Set α, IsOpen (V.basicOpen X)) ∧
      (∀ U : Set V.Element,
        IsOpen U ↔
          (∀ {{x y : V.Element}}, x ∈ U → x ≤ y → y ∈ U) ∧
            (∀ x ∈ U, ∃ X, x.mem X ∧ V.basicOpen X ⊆ U)) ∧
      (∀ x y : V.Element, x ≤ y ↔ ∀ U : Set V.Element, IsOpen U → x ∈ U → y ∈ U) ∧
      (∀ x y : V.Element, y ≪ x ↔ x ≤ y) :=
  ⟨V.isOpen_isUpperSet, V.isOpen_basicOpen, V.isOpen_iff_upper_basic, V.
    le_iff_isOpen_imp,
    V.specializes_iff_le⟩
```

Exercise 1.23 — decidable total elements Suppose that the neighbourhood system D is countable, say,

$$D = \{X_0, X_1, X_2, \ldots, X_n, \ldots\}.$$

Suppose further that the property of consistency of finite sequences of neighbourhoods is decidable (or "effectively known"). Then the following sequence is well defined:

$$\begin{aligned} Y_0 &= X_0 \\ Y_{n+1} &= X_{n+1}, \text{ if this set is consistent with } Y_0, Y_1, \ldots, Y_n; \\ &= Y_n, \text{ if not.} \end{aligned}$$

Show that $\{Y_0, Y_1, \ldots, Y_n, \ldots\}$ is a total element of $|D|$. (Hint: Show first that $Y_0, Y_1, \ldots, Y_{n-1}$ is consistent for all n.) In such a system show that all filters can be determined by sequences.

Definition (`Y`). The greedy sequence that accepts the next enumerated neighbourhood precisely when it remains consistent with those already chosen.

Definition (`greedyElement`). The ideal element whose neighbourhoods are those containing some finite intersection of an initial segment of Y.

Theorem (`Y_prefix_consistent`). Every finite initial segment of Y is consistent.

Proof. By construction each accepted step preserves a nonempty witnessing intersection. ■

Theorem (`greedyElement_isTotal`). When the enumeration is surjective onto the neighbourhoods, the greedy element is total.

Proof. Any proper refinement would have to accept a neighbourhood rejected by greediness, contradicting consistency of the accumulated intersection. ■

Theorem (`filters_sequence_determined`). Every ideal element is determined by a sequence of neighbourhoods.

Proof. Enumerate those members of the enumeration that belong to the filter (or the master set otherwise). ■

Theorem (`exercise_1_23`). The greedy sequence is finitely consistent; under a surjective enumeration the greedy element is total; and every filter is sequence-determined.

Proof. Assemble the three preceding theorems. ■

`Exercise_1_23.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    GreedyEnum

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.GreedyEnum
open NeighborhoodSystem
variable {α : Type*} (V : NeighborhoodSystem α) (enum : ℕ → Set α) [DecidablePred V.
    mem]
variable (henum : ∀ n, V.mem (enum n))
variable (hsurj : ∀ {X : Set α}, V.mem X → ∃ n, enum n = X)
include henum hsurj

/-- Scott's Exercise 1.23: greedy neighbourhoods are finitely consistent; the greedy
    element is total when the enumeration is surjective; every filter is sequence-
    determined. -/
theorem exercise_1_23 :
    (∀ n, V.Consistent (Y V enum) n) ∧
      V.IsTotal (greedyElement V enum henum) ∧
      ∀ x : V.Element,
        ∃ s : ℕ → Set α, (∀ n, V.mem (s n)) ∧ ∀ Z, x.mem Z ↔ (V.mem Z ∧ ∃ n, s n ⊆ Z
    ) :=
  ⟨Y_prefix_consistent V enum henum, greedyElement_isTotal V enum henum hsurj,
    filters_sequence_determined V enum henum hsurj⟩
```

Exercise 1.24 — extending partial elements (For set theorists). Prove, using the Axiom of Choice, or some equivalent principle, that in every domain a partial element can always be extended to a total element. Is this assertion equivalent to the Axiom of Choice? (Hint: Remember to prove that the union of every (transfinite) chain of filters is again a filter.)

Definition (`chainUnion`). The union of a nonempty chain of ideal elements is again an ideal element: a neighbourhood belongs to the union when it belongs to some member of the chain.

Proof. Pairwise intersection of neighbourhoods drawn from comparable chain members may be formed in the larger of the two. ■

Theorem (`le_chainUnion`). Every member of the chain approximates the chain union.

Theorem (`exists_total_ge`). Every ideal element extends to a total element.

Proof. Zorn's lemma applied to the upset of the given element: chain unions supply upper bounds. ■

Theorem (`exercise_1_24`). Every partial element extends to a total element.

Proof. Assemble the Zorn extension theorem. ■

`Exercise_1_24.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
variable {α : Type*} (V : NeighborhoodSystem α)

/-- Scott's Exercise 1.24: every element is below some total element. -/
theorem exercise_1_24 (x : V.Element) : ∃ t, V.IsTotal t ∧ x ≤ t :=
  V.exists_total_ge x
```

Exercise 1.25 — final segments of ordinals (For set theorists). Let Δ be any well-ordered set (ordinal). (Even small ordinals like $\omega \cdot 3$ or ω^5 are interesting.) Let $\mathcal{D}$ be the family of non-empty *final* segments of Δ. What is $|\mathcal{D}|$? Are all elements finite? Is every approximation to a finite element finite?

Theorem (`upperSet_subset_or`). On a linear order, any two upper sets are nested.

Proof. From an element of one set missing from the other, upward closure forces the opposite inclusion. ■

Definition (`finalSegmentSystem`). The neighbourhood system of nonempty upper sets of a nonempty linear order Δ.

Theorem (`Ici_mem`). Every principal final segment $\uparrow a$ is a neighbourhood.

Theorem (`exists_Ici_of_mem`). When Δ is well-ordered, every neighbourhood equals some $\uparrow a$.

Proof. Take the least element of the neighbourhood. ■

Definition (`lowerSetOf`). The set of points a such that $\uparrow a$ belongs to the ideal element.

Definition (`ofLowerSet`). From a nonempty lower set S, the ideal element of neighbourhoods containing $\uparrow a$ for some $a \in S$.

Theorem (`finalSegmentClassify`). Ideal elements are order-isomorphic to the nonempty lower sets of Δ.

Proof. The two maps are mutually inverse once every neighbourhood is a principal final segment. ■

Definition (`topElement`). The ideal element containing every neighbourhood.

Theorem (`topElement_isTotal`). The top element is total.

Theorem (`topElement_not_principal_of_noMax`). If Δ has no greatest element, the top element is not principal.

Proof. A principal $\uparrow a$ omits every strictly later final segment. ■

Theorem (`exercise_1_25`). The top element is total, and when Δ is unbounded above it is not finite.

Proof. Assemble totality of the top with the non-principality criterion. ■

`Exercise_1_25.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    FinalSegment

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.FinalSegment
variable {Δ : Type*} [LinearOrder Δ] [Nonempty Δ]

/-- Scott's Exercise 1.25: final-segment systems have a top total element, not
    principal
    when the order has no maximum. -/
theorem exercise_1_25 :
    (finalSegmentSystem Δ).IsTotal topElement ∧
      (∀ (_hnomax : ∀ a : Δ, ∃ b, a < b) (a : Δ),
        topElement = (finalSegmentSystem Δ).principal (Ici_mem a)) :=
  ⟨topElement_isTotal, topElement_not_principal_of_noMax⟩
```

Exercise 1.26 — ideals of a ring (For algebraists). Let A be a commutative ring with unit. Let Δ be the set of finite subsets $F \subseteq A$. Define

$$I(F) = \{G \in \Delta \mid F \subseteq \text{the ideal generated by } G\}.$$

Prove that the sets of the form $I(F)$ form a neighbourhood system, and that the corresponding domain is isomorphic to the set of ring-theoretic ideals of A partially ordered by inclusion. What would happen if we excluded from Δ all F with $I(F) = I(\{1\})$, where 1 is the unit of A?

Definition (`IFamily`). The neighbourhood $I(F)$ consists of those finite sets whose generated ideal contains F.

Theorem (`IFamily_inter`). The intersection $I(F_1) \cap I(F_2)$ equals $I(F_1 \cup F_2)$.

Theorem (`IFamily_subset_iff`). Inclusion of neighbourhoods $I(F_1) \subseteq I(F_2)$ is equivalent to F_2 lying in the ideal generated by F_1.

Definition (`ringSystem`). The neighbourhood system of sets of the form $I(F)$ on the finite subsets of A.

Proof. Master neighbourhood $I(\emptyset)$; intersections via the union identity above. ■

Definition (`idealOf`). From an ideal element, the ring ideal of those a for which $I(\{a\})$ belongs to the filter.

Definition (`ofIdeal`). From a ring ideal J, the ideal element of those $I(F)$ with the span of F contained in J.

Theorem (`ringIso`). Ideal elements are order-isomorphic to the ring ideals of A ordered by inclusion.

Proof. The two maps are mutually inverse and preserve approximation. ■

Theorem (`IFamily_eq_of_span_eq`). Equal spans yield equal neighbourhoods $I(F)$.

Theorem (`exercise_1_26`). Equal spans of finite sets yield equal I-neighbourhoods.

Proof. The span-equality criterion for I-families. ■

`Exercise_1_26.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Ring

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.Ring
variable {A : Type*} [CommRing A] [DecidableEq A]

/-- Scott's Exercise 1.26: ideal neighbourhood families depend only on the spanned
    ideal. -/
theorem exercise_1_26 {F₁ F₂ : Finset A}
    (h : Ideal.span (↑F₁ : Set A) = Ideal.span (↑F₂ : Set A)) : IFamily F₁ = IFamily F₂
    :=
  IFamily_eq_of_span_eq h
```

Exercise 1.27 — suprema of bounded sets Further closure properties of domains can be proved for bounded sets. We say $X \subseteq |\mathcal{D}|$ is *bounded* iff for some $y \in |\mathcal{D}|$ we have $x \subseteq y$ for all $x \in X$. This y is called an *upper bound*. We let

$$\bigsqcup X = \bigcap \{y \in |\mathcal{D}| \mid x \subseteq y \text{ all } x \in X\}.$$

Prove that if X is bounded, then $\bigsqcup X$ is the *least upper bound* for X in $|\mathcal{D}|$. Prove also: if $U, V \in \mathcal{D}$ are neighbourhoods, then $\{U, V\}$ is consistent in $\mathcal{D}$ iff $\{\uparrow U, \uparrow V\}$ is bounded in $|\mathcal{D}|$. (That is, boundedness is for elements what consistency is for neighbourhoods.) Prove finally with the aid of 1.18 that $X \subseteq |\mathcal{D}|$ is bounded iff every finite subset of X is bounded.

Definition (`Bounded`). A set of ideal elements is bounded when it has an upper bound under approximation.

Definition (`upperBounds`). The set of all upper bounds of a given set of ideal elements.

Definition (`sSup`). The supremum of a bounded set is the least filter of its upper bounds.

Theorem (`le_sSup`). Every member of a bounded set approximates the supremum.

Theorem (`sSup_le`). The supremum is below every upper bound.

Proof. The least-filter characterisation of the infimum of the upper bounds. ■

Definition (`pairSeq`). The length-two sequence with values U and W.

Theorem (`consistent_pair_iff_bounded`). Two neighbourhoods are consistent if and only if their principal ideal elements are bounded.

Proof. A witnessing neighbourhood for the pair intersection supplies a common principal upper bound, and conversely. ■

Theorem (`bounded_iff_finite_bounded`). A set of ideal elements is bounded if and only if every finite subset is.

Proof. The forward direction is immediate. Conversely, the neighbourhoods appearing in members of the set form a finitely consistent family by the finite-boundedness hypothesis, and the least filter of that family is an upper bound (Exercise 1.18). ■

Theorem (`exercise_1_27`). Boundedness is equivalent to finite boundedness; pairwise consistency of neighbourhoods matches boundedness of their principals; and the supremum of a bounded set is its least upper bound.

Proof. Assemble the three preceding theorems with the lub inequalities. ■

`Exercise_1_27.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
variable {α : Type*} (V : NeighborhoodSystem α)

/-- Scott's Exercise 1.27: boundedness is finitary; pairwise consistency matches
    bounded
    principal pairs; sSup is the least upper bound of a bounded set. -/
theorem exercise_1_27 :
    (∀ (X : Set V.Element), V.Bounded X ↔ ∀ S ⊆ X, S.Finite → V.Bounded S) ∧
      (∀ {U W : Set α} (hU : V.mem U) (hW : V.mem W),
        V.Consistent (pairSeq U W) 2 ↔ V.Bounded {V.principal hU, V.principal hW}) ∧
      (∀ (X : Set V.Element) (hX : V.Bounded X) {x : V.Element}, x ∈ X → x ≤ V.sSup X
     hX) ∧
      (∀ (X : Set V.Element) (hX : V.Bounded X) {z : V.Element},
        (∀ x ∈ X, x ≤ z) → V.sSup X hX ≤ z) :=
  ⟨V.bounded_iff_finite_bounded, V.consistent_pair_iff_bounded,
    fun X hX _x hx => V.le_sSup X hX hx, fun X hX _z hz => V.sSup_le X hX hz⟩
```

4.3 Mappings between domains

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

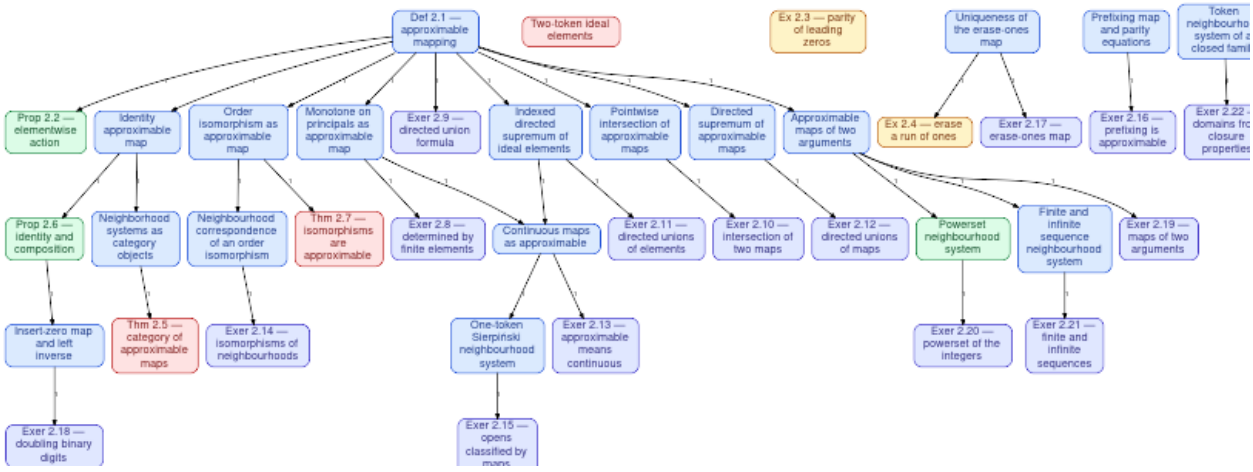


Figure 47: Diagram 12

4.3.1 Introduction

The elements of a domain are regarded as being specified by approximations: the neighbourhoods. With the idea of approximation as the dominant notion, therefore, it is natural to look for a concept of mapping (transformation of domains) that in some suitable sense preserves the spirit of the approximations. In a 'theory of computability,' where the (finite) approximations to the elements are all we can ever know at one time, the only mappings that can be computed are those that proceed by approximation, somehow passing from the neighbourhoods of one domain over to the neighbourhoods of the other.

Suppose $X \in \mathcal{D}_0$ is given - it is an approximation to certain elements of $|\mathcal{D}_0|$. (More precisely $\uparrow X$ is the approximation in the domain, but it is easier to speak of the neighbourhood X.) What can be said about the approximations of the images of these elements under the mapping we will call f? If X is not a very sharp approximation, then not very much can be said about the image in the other domain $|\mathcal{D}_1|$. Trivially, of course, we can say that Δ_1 is an approximation - because it approximates everything in its domain. Suppose, however, that we could say more. Suppose we could say that both Y and Y' approximate the image of X. If the mapping f is coherent, then it is reasonable to suppose that such a statement would imply that Y and Y' are *consistent* in $\mathcal{D}_1$. But if this is so, then since the two neighbourhoods are meant to cluster around the same images, we can feel some confidence in saying that $Y \cap Y'$ approximates these images. In other words to specify f we do not supply a unique image of X, but we say which of the $Y \in \mathcal{D}_1$ approximate the (ideal) image. To make this idea work a *monotonicity condition* is also needed since we are trying to express the idea that "if we give at least X as an approximate input to f, then we can expect at least Y as output." Thus, a mapping is taken as a kind of relation between neighbourhoods.

4.3.2 Definition 2.1 — approximable mapping

DEFINITION 2.1. An *approximable mapping* $f : D_0 \to D_1$ between domains is a binary relation $f \subseteq D_0 \times D_1$ between neighbourhoods such that

(i) $\Delta_0 f \Delta_1$;

(ii) XfY and XfY' always imply $Xf(Y \cap Y')$;

(iii) XfY, $X' \subseteq X$, and $Y \subseteq Y'$ always imply $X'fY'$. □

Condition (i) we have already discussed; in a sense it means "ask me no questions and I shall tell you no lies." In other words "zero input can expect at least zero output." The other conditions are compatible with having

$$f = \{\langle X, \Delta_1 \rangle \mid X \in D_0\};$$

that is, f might be the least informative relation and nothing more. But if it is more, then (ii) is, as we explained, a consistency condition. To explain monotonicity in (iii), suppose a mapping relationship is already known, XfY, say. If we *improve* the accuracy of X to $X' \subseteq X$ and if we *degrade* the accuracy of Y to $Y' \supseteq Y$, then we can still assert $X'fY'$ since this relationship is *less informative* than the former relationship, which was already known. Thus, we see that conditions (i)–(iii) are all reasonably argued as necessary.

One indication that the conditions of 2.1 are sufficient for a definition is that they are exactly what we need to show that f as a neighbourhood relation determines an equivalent elementwise mapping from $|D_0|$ into $|D_1|$. (Owing to the equivalence, we use the same symbol f for both.)

Definition (`ApproximableMap`). An approximable mapping between neighbourhood systems is a relation on neighbourhoods that relates the two master sets, is closed under intersection in the second argument, and is monotone under shrinking the input neighbourhood and enlarging the output neighbourhood.

Proof. The three Scott clauses are the master, intersection, and monotonicity fields; well-formedness records that related sets are neighbourhoods. ■

Theorem (`ApproximableMap.ext`). Two approximable mappings are equal when they relate the same pairs of neighbourhoods.

Definition (`ApproximableMap.toElementMap`). The elementwise action sends an ideal element x to the filter of neighbourhoods Y for which some $X \in x$ satisfies $X \; f \; Y$.

Theorem (`ApproximableMap.mem_toElementMap`). Membership in the image filter is exactly the existential formula above.

Theorem (`ApproximableMap.rel_iff_mem_principal`). The neighbourhood relation is recovered by evaluating the elementwise action at the principal filter: $X \; f \; Y$ if and only if Y belongs to $f(\uparrow X)$.

Theorem (`ApproximableMap.toElementMap_mem_iff_principal`). Membership in the elementwise image is witnessed by a principal approximation: Y belongs to $f(x)$ if and only if Y belongs to $f(\uparrow X)$ for some neighbourhood X of x.

Theorem (`ApproximableMap.ext_of_toElementMap`). Equal elementwise actions determine equal approximable mappings.

Theorem (`ApproximableMap.rel_master`). Every neighbourhood is related to the codomain master set.

Theorem (`ApproximableMap.toElementMap_mono`). The elementwise action is monotone for approximation.

`ApproximableMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {α β ? ? : Type*}

structure ApproximableMap (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) where

  rel : Set α → Set β → Prop

  rel_dom : ∀ {X Y}, rel X Y → V₀.mem X

  rel_cod : ∀ {X Y}, rel X Y → V₁.mem Y

  master_rel : rel V₀.master V₁.master

  inter_right : ∀ {X Y Y'}, rel X Y → rel X Y' → rel X (Y ∩ Y')
```

```
  mono : ∀ {X X' Y Y'}, rel X Y → X' ⊆ X → Y ⊆ Y' → V₀.mem X' → V₁.mem Y' → rel X'
    Y'

namespace ApproximableMap

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}

theorem ext {f g : ApproximableMap V₀ V₁} (h : ∀ X Y, f.rel X Y ↔ g.rel X Y) : f = g
    := by
  obtain ⟨rf, _, _, _, _, _⟩ := f
  obtain ⟨rg, _, _, _, _, _⟩ := g
  have : rf = rg := by funext X Y; exact propext (h X Y)
  subst this; rfl

def toElementMap (f : ApproximableMap V₀ V₁) (x : V₀.Element) : V₁.Element where
  mem Y := ∃ X, x.mem X ∧ f.rel X Y
  sub := fun ⟨_, _, hXY⟩ => f.rel_cod hXY
  master_mem := ⟨V₀.master, x.master_mem, f.master_rel⟩
  inter_mem := by
    rintro Y Y' ⟨X, hX, hXY⟩ ⟨X', hX', hX'Y'⟩
    have hXX'mem : x.mem (X ∩ X') := x.inter_mem hX hX'
    have hXX' : V₀.mem (X ∩ X') := x.sub hXX'mem
    refine ⟨X ∩ X', hXX'mem, ?_⟩
    have h1 : f.rel (X ∩ X') Y :=
      f.mono hXY Set.inter_subset_left subset_rfl hXX' (f.rel_cod hXY)
    have h2 : f.rel (X ∩ X') Y' :=
      f.mono hX'Y' Set.inter_subset_right subset_rfl hXX' (f.rel_cod hX'Y')
    exact f.inter_right h1 h2
  up_mem := by
    rintro Y Y' ⟨X, hX, hXY⟩ hY' hYY'
    exact ⟨X, hX, f.mono hXY subset_rfl hYY' (x.sub hX) hY'⟩

@[simp] theorem mem_toElementMap (f : ApproximableMap V₀ V₁) (x : V₀.Element) {Y : Set
    β} :
    (f.toElementMap x).mem Y ↔ ∃ X, x.mem X ∧ f.rel X Y := Iff.rfl

theorem rel_iff_mem_principal (f : ApproximableMap V₀ V₁) {X : Set α} (hX : V₀.mem X) {
    Y : Set β} :
    f.rel X Y ↔ (f.toElementMap (V₀.principal hX)).mem Y := by
  constructor
  · intro hXY
    exact ⟨X, ⟨hX, subset_rfl⟩, hXY⟩
  · rintro ⟨Z, ⟨_, hXZ⟩, hZY⟩
    exact f.mono hZY hXZ subset_rfl hX (f.rel_cod hZY)

/-- Membership in the elementwise image is witnessed by some principal approximation:
$Y \in f(x)$ iff $Y \in f(\uparrow X)$ for some $X \in x$. -/
theorem toElementMap_mem_iff_principal (f : ApproximableMap V₀ V₁) (x : V₀.Element) {Y
    : Set β} :
    (f.toElementMap x).mem Y ↔
      ∃ (X : Set α) (hx : x.mem X), (f.toElementMap (V₀.principal (x.sub hx))).mem Y :=
     by
  rw [mem_toElementMap]
  constructor
  · rintro ⟨X, hxX, hrel⟩
```

```
    exact ⟨X, hxX, (f.rel_iff_mem_principal (x.sub hxX)).mp hrel⟩
  · rintro ⟨X, hxX, hmem⟩
    exact ⟨X, hxX, (f.rel_iff_mem_principal (x.sub hxX)).mpr hmem⟩

theorem ext_of_toElementMap {f g : ApproximableMap V₀ V₁}
    (h : ∀ x, f.toElementMap x = g.toElementMap x) : f = g := by
  apply ApproximableMap.ext
  intro X Y
  by_cases hX : V₀.mem X
  · rw [f.rel_iff_mem_principal hX, g.rel_iff_mem_principal hX, h]
  · constructor
    · intro hr; exact absurd (f.rel_dom hr) hX
    · intro hr; exact absurd (g.rel_dom hr) hX

theorem rel_master (f : ApproximableMap V₀ V₁) {X : Set α} (hX : V₀.mem X) :
    f.rel X V₁.master :=
  f.mono f.master_rel (V₀.sub_master hX) subset_rfl hX V₁.master_mem

theorem toElementMap_mono (f : ApproximableMap V₀ V₁) {x y : V₀.Element} (hxy : x ≤ y)
    :
    f.toElementMap x ≤ f.toElementMap y := by
  rintro Y ⟨X, hX, hXY⟩
  exact ⟨X, hxy X hX, hXY⟩

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
```

4.3.3 Proposition 2.2 — elementwise action

PROPOSITION 2.2. Given neighbourhood systems D_0 and D_1, an approximable mapping $f : D_0 \to D_1$ always determines a function $f : |D_0| \to |D_1|$ between domains by virtue of the formula:

(i) $f(x) = \{Y \in D_1 \mid \exists X \in x.\ X f Y\}$

for all $x \in |D_0|$. Conversely, this function uniquely determines the original relation by the equivalence:

(ii) $X f Y$ iff $Y \in f(\uparrow X)$

for all $X \in D_0$ and $Y \in D_1$. Approximable functions are always monotone in the following sense:

(iii) $x \subseteq y$ always implies $f(x) \subseteq f(y)$,

for $x, y \in |D_0|$; moreover two approximable functions $f : D_0 \to D_1$ and $g : D_0 \to D_1$ are identical as relations iff

(iv) $f(x) = g(x)$, for all $x \in |D_0|$.

Proof: The argument that formula (i) always gives us $f(x) \in |D_1|$ when $x \in |D_0|$ can be safely left to the reader. Note, however, that all the conditions of 2.1 are required to show this. As for (ii), the implication from left to right follows directly from (i) because $X \in \uparrow X$. In the other direction $Y \in f(\uparrow X)$ means that $Z f Y$ holds for some $Z \in \uparrow X$. But from $X \subseteq Z$ it follows that $X f Y$, as we wished.

To prove monotonicity, assume $x \subseteq y$. Now $X \in x$ and $X f Y$ always imply $X \in y$ and $X f Y$. This means $Y \in f(x)$ always implies $Y \in f(y)$; that is, $f(x) \subseteq f(y)$.

Finally, to check that (iv) means $f = g$ as relations, all that has to be remarked that this follows from formulae (i) and (ii). □

Note that the right-hand side of (ii) can be written:

$$\uparrow Y \subseteq f(\uparrow X),$$

which can be read as saying that the partial element determined by the neighbourhood Y approximates the function value at the element determined by X. This precise relationship of course fits the informal discussion of mapping given earlier. Indeed whenever $x \in [X]$ and $X f Y$ hold, then $f(x) \in [Y]$ always follows, which is another way to construe the mapping character of f. Some examples of mappings are now called for.

Theorem (`ApproximableMap.mem_toElementMap`). Membership in the elementwise image is the existential formula of clause (i).

Theorem (`ApproximableMap.rel_iff_mem_principal`). The neighbourhood relation is recovered by evaluating at the principal filter (clause (ii)).

Theorem (`ApproximableMap.toElementMap_mono`). The elementwise action is monotone for approximation (clause (iii)).

Theorem (`ApproximableMap.ext_of_toElementMap`). Equal elementwise actions determine equal relations (clause (iv)).

Theorem (`proposition_2_2`). Approximable maps act elementwise; the action recovers the relation on principals; it is monotone; and equal actions determine equal relations.

Proof. Assemble the four preceding theorems. ■

`Proposition_2_2.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace ApproximableMap

/-- Scott's Proposition 2.2: approximable maps act elementwise; the action recovers
    the
relation on principals; it is monotone; and equal actions determine equal relations.
    -/
theorem proposition_2_2 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (f : ApproximableMap V₀ V₁) (x : V₀.Element) {Y : Set β},
      (f.toElementMap x).mem Y ↔ ∃ X, x.mem X ∧ f.rel X Y) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (f : ApproximableMap V₀ V₁) {X : Set α} (hX : V₀.mem X) {Y : Set β},
      f.rel X Y ↔ (f.toElementMap (V₀.principal hX)).mem Y) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
```

```
      (f : ApproximableMap V₀ V₁) {x y : V₀.Element},
      x ≤ y → f.toElementMap x ≤ f.toElementMap y) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {f g : ApproximableMap V₀ V₁},
      (∀ x, f.toElementMap x = g.toElementMap x) → f = g) :=
  ⟨mem_toElementMap, rel_iff_mem_principal, toElementMap_mono, ext_of_toElementMap⟩

end ApproximableMap
end DomainTheory.NeighbourhoodSystems
```

4.3.4 Two-token ideal elements

Ideal elements of the two-token neighbourhood system of Example 1.2: the unique partial element $\bot$, the two totals determined by $\{0\}$ and $\{1\}$, and the classification and order facts among them.

Definition (`bot`). The least ideal element: the filter consisting of the master neighbourhood alone.

Definition (`elemOne`). The principal filter of the neighbourhood $\{1\}$.

Theorem (`zero_not_subset_one`). The singleton $\{0\}$ is not contained in $\{1\}$.

Definition (`elemZero`). The principal filter of the neighbourhood $\{0\}$.

Theorem (`zero_ne_master`). The singleton $\{0\}$ is not the master set.

Theorem (`one_ne_master`). The singleton $\{1\}$ is not the master set.

Theorem (`mem_zero_of_mem`). Any ideal element containing $\{0\}$ equals the principal of $\{0\}$.

Theorem (`mem_one_of_mem`). Any ideal element containing $\{1\}$ equals the principal of $\{1\}$.

Theorem (`zero_ne_one`). The singletons $\{0\}$ and $\{1\}$ are distinct.

Theorem (`bot_lt_elemOne`). The bottom element strictly approximates the principal of $\{1\}$.

Theorem (`bot_lt_elemZero`). The bottom element strictly approximates the principal of $\{0\}$.

Theorem (`element_classification`). Every ideal element is bottom, the principal of $\{0\}$, or the principal of $\{1\}$.

Proof. Case on membership of the two singletons; otherwise only the master remains. ■

Theorem (`bot_is_unique_partial`). Bottom is the unique element that is neither principal of $\{0\}$ nor of $\{1\}$.

Theorem (`not_elemOne_le_elemZero`). The principal of $\{1\}$ does not approximate the principal of $\{0\}$.

Theorem (`not_elemZero_le_elemOne`). The principal of $\{0\}$ does not approximate the principal of $\{1\}$.

Theorem (`eq_bot_of_le_elemZero_of_le_elemOne`). Anything below both totals is bottom.

`TwoTokenElements.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.TwoToken

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.TwoToken

namespace neighborhoodSystem

open NeighborhoodSystem
```

```
def bot : neighborhoodSystem.Element where
  mem X := X = master
  sub h := by rw [h]; exact mem_master
  master_mem := rfl
  inter_mem := by
    intro X Y hX hY
    rw [hX, hY, master_inter]
  up_mem := by
    intro X Y hX hY hXY
    rw [hX] at hXY
    exact eq_of_master_subset hY hXY

def elemOne : neighborhoodSystem.Element where
  mem X := X = master ∨ X = one
  sub h := by
    rcases h with rfl | rfl
    · exact mem_master
    · exact mem_one
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl <;> rcases hY with rfl | rfl
    · exact Or.inl (master_inter _)
    · exact Or.inr (master_inter _)
    · exact Or.inr (inter_master _)
    · exact Or.inr (Set.inter_self _)
  up_mem := by
    intro X Y hX hY hXY
    rcases hX with rfl | rfl
    · exact Or.inl (eq_of_master_subset hY hXY)
    · rcases (mem_iff Y).mp hY with rfl | rfl | rfl
      · exact Or.inl rfl
      · exact absurd hXY one_not_subset_zero
      · exact Or.inr rfl

end neighborhoodSystem
theorem zero_not_subset_one : ¬zero ⊆ one := by
  intro h
  have : (0 : Token) ∈ one := h (by simp [zero])
  simp [one] at this

namespace neighborhoodSystem

open NeighborhoodSystem

def elemZero : neighborhoodSystem.Element where
  mem X := X = master ∨ X = zero
  sub h := by
    rcases h with rfl | rfl
    · exact mem_master
    · exact mem_zero
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl <;> rcases hY with rfl | rfl
```

```
    · exact Or.inl (master_inter _)
    · exact Or.inr (master_inter _)
    · exact Or.inr (inter_master _)
    · exact Or.inr (Set.inter_self _)
  up_mem := by
    intro X Y hX hY hXY
    rcases hX with rfl | rfl
    · exact Or.inl (eq_of_master_subset hY hXY)
    · rcases (mem_iff Y).mp hY with rfl | rfl | rfl
      · exact Or.inl rfl
      · exact Or.inr rfl
      · exact absurd hXY zero_not_subset_one

end neighborhoodSystem
theorem zero_ne_master : zero = master := by
  intro h
  have : (1 : Token) ∈ zero := h ▶ (by simp [master])
  simp [zero] at this

theorem one_ne_master : one = master := by
  intro h
  have : (0 : Token) ∈ one := h ▶ (by simp [master])
  simp [one] at this

namespace neighborhoodSystem

open NeighborhoodSystem

theorem mem_zero_of_mem (x : neighborhoodSystem.Element) (h : x.mem zero) :
    x = elemZero := by
  apply Element.ext
  intro X
  constructor
  · intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | hzero | hone
    · exact Or.inl rfl
    · exact Or.inr hzero
    · have hxone : x.mem one := hone ▶ hx
      have := x.inter_mem h hxone
      rw [zero_inter_one] at this
      exact absurd (x.sub this) not_mem_empty
  · intro hx
    rcases hx with rfl | hx
    · exact x.master_mem
    · rw [hx]; exact h

theorem mem_one_of_mem (x : neighborhoodSystem.Element) (h : x.mem one) :
    x = elemOne := by
  apply Element.ext
  intro X
  constructor
  · intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | hzero | hone
    · exact Or.inl rfl
    · have hxzero : x.mem zero := hzero ▶ hx
      have := x.inter_mem hxzero h
```

```
      rw [zero_inter_one] at this
      exact absurd (x.sub this) not_mem_empty
    · exact Or.inr hone
  · intro hx
    rcases hx with rfl | hx
    · exact x.master_mem
    · rw [hx]; exact h

theorem zero_ne_one : zero = (one : Set Token) := by
  intro h
  have h0 : (0 : Token) ∈ zero := by simp [zero]
  rw [h] at h0
  simp [one] at h0

open NeighborhoodSystem

theorem bot_lt_elemOne : bot < elemOne := by
  constructor
  · intro X hx; exact Or.inl hx
  · intro h
    have : bot.mem one := h one (Or.inr rfl)
    have hm : one = master := this
    exact one_ne_master hm

theorem bot_lt_elemZero : bot < elemZero := by
  constructor
  · intro X hx; exact Or.inl hx
  · intro h
    have : bot.mem zero := h zero (Or.inr rfl)
    have hm : zero = master := this
    exact zero_ne_master hm

theorem element_classification (x : neighborhoodSystem.Element) :
    x = bot ∨ x = elemZero ∨ x = elemOne := by
  by_cases h0 : x.mem zero
  · exact Or.inr (Or.inl (mem_zero_of_mem x h0))
  by_cases h1 : x.mem one
  · exact Or.inr (Or.inr (mem_one_of_mem x h1))
  apply Or.inl
  apply Element.ext
  intro X
  constructor
  · intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | hzero | hone
    · rfl
    · exact absurd (hzero ▸ hx) h0
    · exact absurd (hone ▸ hx) h1
  · intro hx
    rw [hx]
    exact x.master_mem

theorem bot_is_unique_partial (x : neighborhoodSystem.Element) :
    x = elemZero → x = elemOne → x = bot := by
  intro hne0 hne1
  rcases element_classification x with hx | hx | hx
  · exact hx
```

```
  · exact (hne0 hx).elim
  · exact (hne1 hx).elim

theorem not_elemOne_le_elemZero : ¬ elemOne ≤ elemZero := by
  intro h
  rcases h one (Or.inr rfl) with h' | h'
  · exact one_ne_master h'
  · exact zero_ne_one h'.symm

theorem not_elemZero_le_elemOne : ¬ elemZero ≤ elemOne := by
  intro h
  rcases h zero (Or.inr rfl) with h' | h'
  · exact zero_ne_master h'
  · exact zero_ne_one h'

theorem eq_bot_of_le_elemZero_of_le_elemOne {a : neighborhoodSystem.Element}
    (h0 : a ≤ elemZero) (h1 : a ≤ elemOne) : a = bot := by
  rcases element_classification a with rfl | rfl | rfl
  · rfl
  · exact absurd h1 not_elemZero_le_elemOne
  · exact absurd h0 not_elemOne_le_elemZero

end neighborhoodSystem

end DomainTheory.NeighbourhoodSystems.TwoToken
```

4.3.5 Parity scan into truth

Parity of leading zeros before the first one, as an approximable map from the binary-tree system into the two-token truth domain.

Definition (`T`). The two-token neighbourhood system of Example 1.2, used as the output domain.

Definition (`botElt`). The undefined element of $|T|$.

Theorem (`botElt_le`). Undefined approximates every element of $|T|$.

Definition (`falseElt`). The total element of $|T|$ standing for a negative outcome.

Definition (`scan`). Reading a finite binary string left to right, return the parity of the number of leading zeros before the first one, or undefined if no one appears.

Theorem (`scan_nil`). The empty string scans to undefined.

Theorem (`scan_true`). A string beginning with one scans to true (even parity of leading zeros).

Theorem (`scan_false`). A leading zero flips the scanned parity of the remainder.

Theorem (`scan_append`). Once a string has a definite scan value, appending any suffix leaves that value unchanged.

Definition (`trueElt`). The total element of $|T|$ standing for a positive outcome.

Definition (`valElt`). Interpret an optional boolean as undefined, true, or false in $|T|$.

Theorem (`valElt_scan_mono`). Extending a string can only refine the scanned value in the approximation order.

Definition (`parityMap`). The approximable mapping from the binary-tree system to T that relates a cone on σ to a neighbourhood belonging to the scanned value of σ.

Proof. Master, intersection, and monotonicity follow from the corresponding properties of ideal elements and the scan-extension lemma. ■

Theorem (`eq_botElt_of_le`). Anything below both true and false is undefined.

Theorem (`parityMap_rel_cone`). On cones, the relation is exactly membership of the scanned value.

Theorem (`parityMap_toElementMap_mem`). The elementwise action holds of Y precisely when some cone in the input element scans to a value containing Y.

Theorem (`parityMap_toElementMap_sigmaElt`). After a fixed prefix σ, the elementwise action on σx is scanning σ concatenated with a cone from x.

Theorem (`scan_append_falseFalse`). Two leading zeros leave the scan of the remainder unchanged.

Theorem (`scan_append_falseTrue`). A leading zero followed by a one scans to false.

Theorem (`scan_append_true`). A leading one scans to true regardless of the rest.

`Truth.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.
    TwoTokenElements
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ExampleB
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Truth
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem BitDom

abbrev T : NeighborhoodSystem TwoToken.Token := TwoToken.neighborhoodSystem

def botElt : T.Element := TwoToken.neighborhoodSystem.bot

theorem botElt_le (x : T.Element) : botElt ≤ x := by
  intro Z hZ
  have : Z = TwoToken.master := hZ
  subst this
  exact x.master_mem

def falseElt : T.Element := TwoToken.neighborhoodSystem.elemOne

def scan : Str → Option Bool
  | [] => none
  | true :: _ => some true
  | false :: t => (scan t).map (!·)

@[simp] theorem scan_nil : scan [] = none := rfl

@[simp] theorem scan_true (t : Str) : scan (true :: t) = some true := rfl

@[simp] theorem scan_false (t : Str) : scan (false :: t) = (scan t).map (!·) := rfl

theorem scan_append {σ : Str} {b : Bool} (h : scan σ = some b) (t : Str) :
    scan (σ ++ t) = some b := by
  induction σ generalizing b with
  | nil => simp at h
  | cons c σ₀ ih =>
    cases c with
```

```
    | true => simp only [scan_true] at h ⊢; exact h
    | false =>
      simp only [List.cons_append, scan_false] at h ⊢
      rw [Option.map_eq_some_iff] at h
      obtain ⟨a, ha, rfl⟩ := h
      rw [ih ha]
      rfl

def trueElt : T.Element := TwoToken.neighborhoodSystem.elemZero

def valElt : Option Bool → T.Element
  | none => botElt
  | some true => trueElt
  | some false => falseElt

theorem valElt_scan_mono {σ σ' : Str} (h : σ <+: σ') :
    valElt (scan σ) ≤ valElt (scan σ') := by
  obtain ⟨t, rfl⟩ := h
  cases hσ : scan σ with
  | none => simpa [valElt, hσ] using botElt_le _
  | some b => rw [scan_append hσ t]

def parityMap : ApproximableMap B T where
  rel X Y := ∃ σ, X = cone σ ∧ (valElt (scan σ)).mem Y
  rel_dom := fun ⟨σ, hX, _⟩ => ⟨σ, hX⟩
  rel_cod := fun ⟨_, _, hY⟩ => (valElt _).sub hY
  master_rel := by
    refine ⟨[], cone.nil.symm, ?_⟩
    show (botElt).mem TwoToken.master
    rfl
  inter_right := by
    rintro X Y Y' ⟨σ, hX, hY⟩ ⟨σ', hX', hY'⟩
    have hσ : σ = σ' := cone.injective (hX ▶ hX')
    subst hσ
    exact ⟨σ, hX, (valElt (scan σ)).inter_mem hY hY'⟩
  mono := by
    rintro X X' Y Y' ⟨σ, hX, hY⟩ hX'X hYY' hX'mem hY'mem
    obtain ⟨σ', hX'cone⟩ := hX'mem
    have hpre : σ <+: σ' := by
      apply cone.subset_cone.mp
      rw [← hX'cone, ← hX]; exact hX'X
    have hYmem' : (valElt (scan σ)).mem Y' := (valElt (scan σ)).up_mem hY hY'mem hYY'
    exact ⟨σ', hX'cone, valElt_scan_mono hpre Y' hYmem'⟩

theorem eq_botElt_of_le {a : T.Element} (h0 : a ≤ trueElt) (h1 : a ≤ falseElt) : a =
    botElt :=
  TwoToken.neighborhoodSystem.eq_bot_of_le_elemZero_of_le_elemOne h0 h1

theorem parityMap_rel_cone (σ : Str) (Y : Set TwoToken.Token) :
    parityMap.rel (cone σ) Y ↔ (valElt (scan σ)).mem Y := by
  constructor
  · rintro ⟨?, h?, hval⟩
    rw [cone.injective h?]
    exact hval
  · intro hval
    exact ⟨σ, rfl, hval⟩
```

```
theorem parityMap_toElementMap_mem (x : B.Element) (Y : Set TwoToken.Token) :
    (parityMap.toElementMap x).mem Y ↔ ∃ τ, x.mem (cone τ) ∧ (valElt (scan τ)).mem Y
    := by
  constructor
  · rintro ⟨X, hX, hrel⟩
    obtain ⟨τ, rfl⟩ := x.sub hX
    exact ⟨τ, hX, (parityMap_rel_cone τ Y).mp hrel⟩
  · rintro ⟨τ, hτ, hval⟩
    exact ⟨cone τ, hτ, (parityMap_rel_cone τ Y).mpr hval⟩

theorem parityMap_toElementMap_sigmaElt (σ : Str) (x : B.Element) (Y : Set TwoToken.
    Token) :
    (parityMap.toElementMap (sigmaElt σ x)).mem Y ↔
      ∃ τ, x.mem (cone τ) ∧ (valElt (scan (σ ++ τ))).mem Y := by
  constructor
  · rintro ⟨Z, ⟨_, X, hX, hsub⟩, ?, rfl, hval⟩
    obtain ⟨τ, rfl⟩ := x.sub hX
    rw [prepend_cone] at hsub
    exact ⟨τ, hX, valElt_scan_mono (cone.subset_cone.mp hsub) Y hval⟩
  · rintro ⟨τ, hτ, hval⟩
    exact ⟨cone (σ ++ τ), ⟨memB_cone _, cone τ, hτ, (prepend_cone σ τ).le⟩, σ ++ τ, rfl,
     hval⟩

theorem scan_append_falseFalse (τ : Str) : scan ([false, false] ++ τ) = scan τ := by
  show scan (false :: false :: τ) = scan τ
  cases h : scan τ with
  | none => simp [scan_false, h]
  | some b => simp [scan_false, h, Bool.not_not]

theorem scan_append_falseTrue (τ : Str) : scan ([false, true] ++ τ) = some false := by
  show scan (false :: true :: τ) = some false
  simp [scan_false, scan_true]

theorem scan_append_true (τ : Str) : scan ([true] ++ τ) = some true :=
  scan_true τ

end DomainTheory.NeighbourhoodSystems.Truth
```

4.3.6 Example 2.3 — parity of leading zeros

EXAMPLE 2.3. Let T be the neighbourhood system of the two-token domain of Example 1.2. To avoid confusion with some other domains, we will call the two total elements of $|T|$ respectively true and false. There is only one other finite element here, namely $\perp =$ undefined. We often use these elements as indicators of results: true indicates a positive outcome; false, a negative outcome; and $\perp$ indicates that there is not enough information to decide the outcome totally.

Let B be the system for the binary tree as in the last chapter. What we wish to define is an approximable mapping $f : B \to T$. The intuitive idea of the mapping we have in mind is that the binary sequence is being read from left to right, and we are counting the number of 0's seen before the first 1 is encountered. We then test the parity of this count; if it is even, the output is true; if not, false. Using a suggestive informal notation with three dots, some results of the function that does the counting and testing can be written as:

$$f(0000101\ldots) = \text{true}$$
$$f(1101110\ldots) = \text{true}$$
$$f(0111011\ldots) = \text{false}$$
$$f(0000000\ldots) = \perp .$$

The last equation is necessary, because 0000000 as a partial element cannot be counted as either even or odd since it can have inconsistent extensions:

$$0000000\ \perp \sqsubseteq 00000001\ \perp$$
$$0000000\ \perp \sqsubseteq 000000000001\ \perp .$$

So, as far as f is concerned, a plain string of 0's is indefinite. The same answer holds if the 0's go on infinitely.

To be more precise we want

$$f(0^n 1\ \perp) = \text{true} \quad \text{if } n \text{ is even;}$$
$$= \text{false} \quad \text{if } n \text{ is odd.}$$

As a binary relation $f \subseteq B \times T$ we will have

$X f Y$ iff $Y \in \perp$ or $X \sqsubseteq 0^n 1\Delta$ for some $n \in \mathbf{N}$ and either n is even and $Y \in$ true or n is odd and $Y \in$ false.

It should be checked that 2.1(i)–(ii) are satisfied. □

The development is with the parity scan into truth peer.

Theorem (`example_2_3`). Scanning two leading zeros leaves the scan of the remainder unchanged; a leading zero followed by a one scans to false; a leading one scans to true.

Proof. By the scan lemmas of the truth peer. ■

`Example_2_3.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Truth

@[expose] public section

open DomainTheory.NeighbourhoodSystems.Truth

/-- Scott's Example 2.3: scanning leading zeros before the first one yields the
parity map into the two-token truth domain. -/
theorem example_2_3 :
    (∀ τ, scan ([false, false] ++ τ) = scan τ) ∧
    (∀ τ, scan ([false, true] ++ τ) = some false) ∧
    (∀ τ, scan ([true] ++ τ) = some true) :=
  ⟨scan_append_falseFalse, scan_append_falseTrue, scan_append_true⟩
```

4.3.7 Erase a run of ones

Erase the first consecutive run of ones on the binary-tree system.

Definition (`SatisfiesRunEquations`). An approximable endomap of the binary-tree system satisfies the run equations when it intertwines a leading zero with the image, collapses two leading ones to one, sends a leading one-zero to a leading zero, and sends a lone leading one on bottom to bottom.

Definition (`del`). Erase a leading run of ones from a string, leaving a leading zero or the empty string.

Theorem (`del_nil`). Erasing ones from the empty string yields empty.

Theorem (`del_true`). Erasing ones after a leading one continues on the tail.

Theorem (`del_false`). A leading zero is left intact by the erase-ones string map.

Definition (`out`). Erase the first run of ones in a string, copying zeros and deleting a leading run of ones via the delete map.

Theorem (`out_nil`). The erase-ones string map sends empty to empty.

Theorem (`out_false`). A leading zero is preserved and the remainder is erased recursively.

Theorem (`out_true`). A leading one is deleted by the delete map on the tail.

Theorem (`del_append`). Deleting ones is monotone for string prefix.

Proof. Induction on the prefix. ■

Theorem (`out_append`). The erase-ones string map is monotone for string prefix.

Proof. Induction, using monotonicity of the delete map on a leading one. ■

Theorem (`out_mono`). Prefix-comparable strings have erase-ones images that remain prefix-comparable.

Proof. Unpack the prefix witness and apply the append form. ■

Definition (`runMap`). The approximable mapping on the binary-tree system that relates a cone on σ to neighbourhoods containing the cone on the string obtained by erasing the first run of ones in σ.

Proof. Master, intersection, and monotonicity follow from injectivity of cones and monotonicity of the erase-ones string map. ■

Theorem (`out_append_false`). Erasing ones after a leading zero yields a leading zero followed by the erased remainder.

Theorem (`out_append_trueFalse`). A leading one followed by a zero erases to that zero and the rest of the string.

Theorem (`out_append_trueTrue`). Two leading ones erase the same as a single leading one followed by the same tail.

Theorem (`out_true_false`). Cons form of the one-then-zero case.

Theorem (`out_true_true`). Cons form of the two-ones case.

Theorem (`key`). Any approximable map satisfying the run equations acts on finite approximations $\sigma\ \bot$ by the erase-ones string map.

Proof. Induction on σ, using the run equations at each of the empty, lone-one, one-zero, two-ones, and leading-zero cases, together with the string identities above. ■

Theorem (`runMap_rel_cone`). On cones, the relation is membership of a neighbourhood containing the erased output cone.

`RunMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ExampleB
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
```

```
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    sigmaBot_le_iff

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.RunMap
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem BitDom

structure SatisfiesRunEquations (g : ApproximableMap B B) : Prop where
  zero : ∀ x : B.Element, g.toElementMap (sigmaElt [false] x) = sigmaElt [false] (g.
    toElementMap x)
  oneOne : ∀ x : B.Element, g.toElementMap (sigmaElt [true, true] x) = g.toElementMap (
    sigmaElt [true] x)
  oneZero : ∀ x : B.Element, g.toElementMap (sigmaElt [true, false] x) = sigmaElt [
    false] x
  one : g.toElementMap (sigmaElt [true] B.bot) = B.bot

def del : Str → Str
  | [] => []
  | true :: t => del t
  | false :: t => false :: t

@[simp] theorem del_nil : del [] = [] := rfl

@[simp] theorem del_true (t : Str) : del (true :: t) = del t := rfl

@[simp] theorem del_false (t : Str) : del (false :: t) = false :: t := rfl

def out : Str → Str
  | [] => []
  | false :: t => false :: out t
  | true :: t => del t

@[simp] theorem out_nil : out [] = [] := rfl

@[simp] theorem out_false (t : Str) : out (false :: t) = false :: out t := rfl

@[simp] theorem out_true (t : Str) : out (true :: t) = del t := rfl

theorem del_append (s t : Str) : del s <+: del (s ++ t) := by
  induction s with
  | nil => simp
  | cons c s₀ ih =>
    cases c with
    | true => simpa using ih
    | false => exact ⟨t, rfl⟩

theorem out_append (σ t : Str) : out σ <+: out (σ ++ t) := by
  induction σ with
  | nil => simp
  | cons c σ₀ ih =>
    cases c with
    | true => simpa using del_append σ₀ t
    | false =>
      simp only [List.cons_append, out_false]
      obtain ⟨u, hu⟩ := ih
```

```
      exact ⟨u, by rw [List.cons_append, hu]⟩

theorem out_mono {σ σ' : Str} (h : σ <+: σ') : out σ <+: out σ' := by
  obtain ⟨t, rfl⟩ := h
  exact out_append σ t

def runMap : ApproximableMap B B where
  rel X Y := ∃ σ, X = cone σ ∧ B.mem Y ∧ cone (out σ) ⊆ Y
  rel_dom := fun ⟨σ, hX, _, _⟩ => ⟨σ, hX⟩
  rel_cod := fun ⟨_, _, hYmem, _⟩ => hYmem
  master_rel := by
    refine ⟨[], (B_master).trans cone.nil.symm, B.master_mem, ?_⟩
    rw [out_nil, B_master, cone.nil]
  inter_right := by
    rintro X Y Y' ⟨σ, hX, hYmem, hYsub⟩ ⟨σ', hX', hY'mem, hY'sub⟩
    have hσ : σ = σ' := cone.injective (hX ▶ hX')
    subst hσ
    have hsub : cone (out σ) ⊆ Y ∩ Y' := Set.subset_inter hYsub hY'sub
    exact ⟨σ, hX, B.inter_mem hYmem hY'mem (memB_cone (out σ)) hsub, hsub⟩
  mono := by
    rintro X X' Y Y' ⟨σ, hX, _, hYsub⟩ hX'X hYY' hX'mem hY'mem
    obtain ⟨σ', hX'cone⟩ := hX'mem
    have hpre : σ <+: σ' := by
      apply cone.subset_cone.mp
      rw [← hX'cone, ← hX]; exact hX'X
    have hcone : cone (out σ') ⊆ cone (out σ) := cone.subset_cone.mpr (out_mono hpre)
    exact ⟨σ', hX'cone, hY'mem, (hcone.trans hYsub).trans hYY'⟩

theorem out_append_false (τ : Str) : out ([false] ++ τ) = false :: out τ := rfl

theorem out_append_trueFalse (τ : Str) : out ([true, false] ++ τ) = false :: τ := rfl

theorem out_append_trueTrue (τ : Str) : out ([true, true] ++ τ) = out ([true] ++ τ) :=
    rfl

theorem out_true_false (t : Str) : out (true :: false :: t) = false :: t := rfl

theorem out_true_true (t : Str) : out (true :: true :: t) = out (true :: t) := rfl

theorem key (g : ApproximableMap B B) (hg : SatisfiesRunEquations g) :
    ∀ σ : Str, g.toElementMap (sigmaElt σ B.bot) = sigmaElt (out σ) B.bot
  | [] => by
      rw [show out ([] : Str) = ([] : Str) from rfl, sigmaElt_nil]
      have hm := g.toElementMap_mono (B.bot_le (sigmaElt [true] B.bot))
      rw [hg.one] at hm
      exact le_antisymm hm (B.bot_le _)
  | [true] => by
      rw [show out [true] = ([] : Str) from rfl, sigmaElt_nil]
      exact hg.one
  | true :: false :: t => by
      rw [out_true_false, show (true :: false :: t : Str) = [true, false] ++ t from
    rfl,
        sigmaElt_append, hg.oneZero, ← sigmaElt_append]
      rfl
  | true :: true :: t => by
      rw [out_true_true, show (true :: true :: t : Str) = [true, true] ++ t from rfl,
```

```
        sigmaElt_append, hg.oneOne, ← sigmaElt_append]
      exact key g hg (true :: t)
  | false :: t => by
      rw [out_false, show (false :: t : Str) = [false] ++ t from rfl, sigmaElt_append,
     hg.zero,
        key g hg t, show (false :: out t : Str) = [false] ++ out t from rfl,
    sigmaElt_append]

theorem runMap_rel_cone (σ : Str) (Y : Set Str) :
    runMap.rel (cone σ) Y ↔ B.mem Y ∧ cone (out σ) ⊆ Y := by
  constructor
  · rintro ⟨?, h?, hYmem, hsub⟩
    obtain rfl : ? = σ := (cone.injective h?).symm
    exact ⟨hYmem, hsub⟩
  · rintro ⟨hYmem, hsub⟩
    exact ⟨σ, rfl, hYmem, hsub⟩

end DomainTheory.NeighbourhoodSystems.RunMap
```

4.3.8 Example 2.4 — erase a run of ones

EXAMPLE 2.4. Let us briefly describe an approximable mapping $g : B \to B$. Informally, g can be said to "read a sequence from left to right and eliminate the first consecutive run of 1's while copying all the other digits as read." We will have

$$g(0^n 1^k 0x) = 0^{n+1}x$$

provided $k > 0$. (Here 1^k means a string of 1's of length k.) However, if 1^∞ is the infinite sequences of 1's, then

$$g(1^\infty) = \bot, \text{ and}$$
$$g(0^n 1^\infty) = 0^n.$$

This example is instructive, since it shows that a non-trivial mapping can transform a total element into a partial element. □

Aside from our being able to define particular functions outright, we can combine functions in many different ways; the idea of composition is probably the most basic scheme of combination, and there is a technical name for a family of structures with mappings that can be so combined.

The development is with the erase a run of ones peer.

Theorem (`example_2_4`). Erasing a leading zero, a leading one-zero, or two leading ones behaves as for the erase-ones string map, and the approximable endomap of the binary-tree system satisfies the run equations.

Proof. By the string identities and the run-equation witness of the erase-ones peer. ■

`Example_2_4.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.RunMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.eq_runMap
```

```
@[expose] public section

open DomainTheory.NeighbourhoodSystems.RunMap

/-- Scott's Example 2.4: erasing the first run of ones defines an approximable
endomap of the binary-tree system satisfying the run equations. -/
theorem example_2_4 :
    (∀ τ, out ([false] ++ τ) = false :: out τ) ∧
    (∀ τ, out ([true, false] ++ τ) = false :: τ) ∧
    (∀ τ, out ([true, true] ++ τ) = out ([true] ++ τ)) ∧
    SatisfiesRunEquations runMap :=
  ⟨out_append_false, out_append_trueFalse, out_append_trueTrue, runMap_satisfies⟩
```

4.3.9 Identity approximable map

The identity approximable map on a neighbourhood system, composition of approximable maps, and the category laws (units and associativity).

Definition (`idMap`). The identity approximable map on a neighbourhood system relates neighbourhoods X and Y precisely when $X \subseteq Y$.

Proof. Master, intersection, and monotonicity are immediate from the corresponding properties of neighbourhoods and inclusion. ■

Theorem (`idMap_rel`). The relation of the identity map is membership of both arguments together with inclusion.

Definition (`comp`). The composition of approximable maps relates X to Z when some intermediate neighbourhood Y is related by the first map to X and by the second to Z.

Proof. Master uses the two master relations; intersection and monotonicity follow by intersecting or shrinking the intermediate neighbourhoods via the component maps. ■

Theorem (`comp_rel`). Composition relates X to Z if and only if such an intermediate neighbourhood exists.

Theorem (`toElementMap_idMap`). The elementwise action of the identity map is the identity on ideal elements.

Proof. A neighbourhood of the image comes from a smaller neighbourhood of the input by upward closure; conversely the input neighbourhood itself witnesses the relation. ■

Theorem (`toElementMap_comp`). The elementwise action of a composite is the composite of the elementwise actions.

Proof. Reassociate the witnessing intermediate neighbourhoods. ■

Theorem (`commute_apply`). If two endo-maps commute under composition, their elementwise actions commute.

Proof. Apply the preceding theorem to both composites. ■

Theorem (`comp_assoc`). Composition of approximable maps is associative.

Proof. Reassociate the witnessing intermediate neighbourhoods. ■

Theorem (`comp_idMap`). Composing on the right with the identity leaves a map unchanged.

Proof. An intermediate neighbourhood related by identity is a supersets of the input, so monotonicity recovers the original relation, and conversely the input neighbourhood itself serves as intermediate. ■

Theorem (`idMap_comp`). Composing on the left with the identity leaves a map unchanged.

Proof. Dual to the right-unit law: the intermediate neighbourhood may be taken to be the output neighbourhood. ■

idMap.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace ApproximableMap

section
variable {α β ? ? : Type*}

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}

def idMap (V : NeighborhoodSystem α) : ApproximableMap V V where
  rel X Y := V.mem X ∧ V.mem Y ∧ X ⊆ Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨V.master_mem, V.master_mem, subset_rfl⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY, hXY⟩ ⟨_, hY', hXY'⟩
    exact ⟨hX, V.inter_mem hY hY' hX (Set.subset_inter hXY hXY'), Set.subset_inter hXY
    hXY'⟩
  mono := by
    rintro X X' Y Y' ⟨_, _, hXY⟩ hX'X hYY' hX' hY'
    exact ⟨hX', hY', (hX'X.trans hXY).trans hYY'⟩

@[simp] theorem idMap_rel {V : NeighborhoodSystem α} {X Y : Set α} :
    (idMap V).rel X Y ↔ V.mem X ∧ V.mem Y ∧ X ⊆ Y := Iff.rfl

def comp (g : ApproximableMap V₁ V₂) (f : ApproximableMap V₀ V₁) : ApproximableMap V₀ V₂
    where
  rel X Z := ∃ Y, f.rel X Y ∧ g.rel Y Z
  rel_dom := fun ⟨_, hXY, _⟩ => f.rel_dom hXY
  rel_cod := fun ⟨_, _, hYZ⟩ => g.rel_cod hYZ
  master_rel := ⟨V₁.master, f.master_rel, g.master_rel⟩
  inter_right := by
    rintro X Z Z' ⟨Y, hXY, hYZ⟩ ⟨Y', hXY', hY'Z'⟩
    refine ⟨Y ∩ Y', f.inter_right hXY hXY', ?_⟩
    have hYY'mem : V₁.mem (Y ∩ Y') := f.rel_cod (f.inter_right hXY hXY')
    have h1 : g.rel (Y ∩ Y') Z :=
      g.mono hYZ Set.inter_subset_left subset_rfl hYY'mem (g.rel_cod hYZ)
    have h2 : g.rel (Y ∩ Y') Z' :=
      g.mono hY'Z' Set.inter_subset_right subset_rfl hYY'mem (g.rel_cod hY'Z')
    exact g.inter_right h1 h2
  mono := by
    rintro X X' Z Z' ⟨Y, hXY, hYZ⟩ hX'X hZZ' hX' hZ'
    refine ⟨Y, f.mono hXY hX'X subset_rfl hX' (f.rel_cod hXY), ?_⟩
    exact g.mono hYZ subset_rfl hZZ' (g.rel_dom hYZ) hZ'

@[simp] theorem comp_rel {g : ApproximableMap V₁ V₂} {f : ApproximableMap V₀ V₁} {X :
```

```
    Set α}
    {Z : Set ?} : (g.comp f).rel X Z ↔ ∃ Y, f.rel X Y ∧ g.rel Y Z := Iff.rfl

@[simp] theorem toElementMap_idMap (x : V₀.Element) : (idMap V₀).toElementMap x = x :=
    by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨X, hXx, _, hY, hXY⟩
    exact x.up_mem hXx hY hXY
  · intro hY
    exact ⟨Y, hY, x.sub hY, x.sub hY, subset_rfl⟩

theorem toElementMap_comp (g : ApproximableMap V₁ V₂) (f : ApproximableMap V₀ V₁) (x :
    V₀.Element) :
    (g.comp f).toElementMap x = g.toElementMap (f.toElementMap x) := by
  apply Element.ext
  intro Z
  constructor
  · rintro ⟨X, hXx, Y, hXY, hYZ⟩
    exact ⟨Y, ⟨X, hXx, hXY⟩, hYZ⟩
  · rintro ⟨Y, ⟨X, hXx, hXY⟩, hYZ⟩
    exact ⟨X, hXx, Y, hXY, hYZ⟩

theorem commute_apply {V : NeighborhoodSystem α} {f g : ApproximableMap V V}
    (hcomm : f.comp g = g.comp f) (x : V.Element) :
    f.toElementMap (g.toElementMap x) = g.toElementMap (f.toElementMap x) := by
  have h1 : (f.comp g).toElementMap x = (g.comp f).toElementMap x := by rw [hcomm]
  rwa [toElementMap_comp, toElementMap_comp] at h1

theorem comp_assoc {V₃ : NeighborhoodSystem ?} (h : ApproximableMap V₂ V₃)
    (g : ApproximableMap V₁ V₂) (f : ApproximableMap V₀ V₁) :
    (h.comp g).comp f = h.comp (g.comp f) := by
  apply ApproximableMap.ext
  intro X W
  constructor
  · rintro ⟨Y, hXY, Z, hYZ, hZW⟩
    exact ⟨Z, ⟨Y, hXY, hYZ⟩, hZW⟩
  · rintro ⟨Z, ⟨Y, hXY, hYZ⟩, hZW⟩
    exact ⟨Y, hXY, Z, hYZ, hZW⟩

theorem comp_idMap (f : ApproximableMap V₀ V₁) : f.comp (idMap V₀) = f := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  · rintro ⟨Y, ⟨hX, _, hXY⟩, hYZ⟩
    exact f.mono hYZ hXY subset_rfl hX (f.rel_cod hYZ)
  · intro hXZ
    exact ⟨X, ⟨f.rel_dom hXZ, f.rel_dom hXZ, subset_rfl⟩, hXZ⟩

theorem idMap_comp (f : ApproximableMap V₀ V₁) : (idMap V₁).comp f = f := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  · rintro ⟨Y, hXY, _, hZ, hYZ⟩
    exact f.mono hXY subset_rfl hYZ (f.rel_dom hXY) hZ
```

```
  · intro hXZ
    exact ⟨Z, hXZ, f.rel_cod hXZ, f.rel_cod hXZ, subset_rfl⟩
end

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

4.3.10 Neighborhood systems as category objects

Bundle neighbourhood systems as objects of the category whose morphisms are approximable maps.

Definition (`DomainObj`). A *domain object* is a token type together with a neighbourhood system on that type. (The name records the category-theoretic rôle: such a bundle is an object of the category below. The neighbourhood system itself is the presentation; the domain of ideal elements is $|D|$.)

Definition (`instCategoryDomainObj`). Neighbourhood systems and approximable mappings form a category: morphisms are approximable maps between the underlying systems; the identity morphism is the identity approximable map; composition of morphisms is composition of approximable maps (in diagrammatic order). The unit and associativity laws are those already proved for identity and composition of approximable maps.

Proof. Immediate from the corresponding facts for the identity map and composition. ■

Theorem (`DomainObj.id_val`). The identity morphism on a domain object is the identity approximable map of its neighbourhood system.

Theorem (`DomainObj.comp_val`). Composition of morphisms of domain objects is composition of the underlying approximable maps.

DomainObj.lean

```
module

public import Mathlib.CategoryTheory.Category.Basic
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.idMap

@[expose] public section

universe w

namespace DomainTheory.NeighbourhoodSystems

/-- Bundled neighbourhood system: a carrier type together with its neighbourhood
    structure. -/
structure DomainObj : Type (w + 1) where
  carrier : Type w
  sys : NeighborhoodSystem carrier

open CategoryTheory ApproximableMap

/-- Neighbourhood systems and approximable maps form a Mathlib category. -/
instance : Category DomainObj where
  Hom D E := ApproximableMap D.sys E.sys
  id D := idMap D.sys
  comp f g := g.comp f
  id_comp f := comp_idMap f
```

```
  comp_id f := idMap_comp f
  assoc f g h := (comp_assoc h g f).symm

@[simp] theorem DomainObj.id_val (D : DomainObj) :
    (? D : ApproximableMap D.sys D.sys) = idMap D.sys := rfl

@[simp] theorem DomainObj.comp_val {D E F : DomainObj} (g : E ⟶ F) (f : D ⟶ E) :
    (f ? g : ApproximableMap D.sys F.sys) = g.comp f := rfl

end DomainTheory.NeighbourhoodSystems
```

4.3.11 Theorem 2.5 — category of approximable maps

THEOREM 2.5. The class of neighbourhood systems and approximable mappings form a *category*, where the identity mapping $I_D : D \to D$ relates $X, Y \in D$ as follows:

(i) $X\ I_D\ Y$ iff $X \subseteq Y$.

If $f : D_0 \to D_1$ and $g : D_1 \to D_2$ are given, then the *composition* $g \circ f : D_0 \to D_2$ relates $X \in D_0$ and $Z \in D_2$ as follows:

(ii) $X\ g \circ f\ Z$ iff $\exists Y \in D_1.\ X\ f\ Y$ and $Y\ g\ Z$.

Proof: (We may use [142] as the standard reference on category theory, but we require hardly more than the basic definitions at this stage.) To check that we have a category, we need to know that the identity and composition maps really are maps in the category and that certain identity and associative laws hold. Now it is obvious that I_D satisfies 2.1 (i)-(iii). Moreover if $f : D_0 \to D_1$, all we have to prove is:

$$f \circ I_{D_0} = I_{D_1} \circ f = f$$

Checking one of these equations is enough. Thus, for $X \in D_0$ and $Z \in D_1$ we find

$$\begin{aligned} X\ f \circ I_{D_0}\ Z &\text{ iff } \exists Y \in D_0.\ X \subseteq Y \text{ and } Y\ f\ Z \\ &\text{iff } X\ f\ Z. \end{aligned}$$

So, f and $f \circ I_{D_0}$ are the same mapping.

Suppose now that $f : D_0 \to D_1$ and $g : D_1 \to D_2$. We have to verify that $g \circ f$ is an approximable mapping. First off, there is no trouble in seeing that $\Delta_0\ g \circ f\ \Delta_2$ holds. Next, suppose that $X\ g \circ f\ Z$ and $X\ g \circ f\ Z'$ hold. Then we have $X\ f\ Y$ and $Y\ g\ Z$ for some choice of $Y \in D_1$. Also $X\ f\ Y'$ and $Y'\ g\ Z'$ hold for some choice of $Y' \in D_1$. By 2.1 (ii) it follows that $X\ f\ (Y \cap Y')$. Since $Y \cap Y' \subseteq Y$, we conclude $(Y \cap Y')\ g\ Z$ by 2.1 (iii); similarly $(Y \cap Y')\ g\ Z'$. Invoking 2.1 (ii) again, we obtain $(Y \cap Y')\ g\ (Z \cap Z')$, and $X\ g \circ f\ (Z \cap Z')$ is proved.

Suppose finally that $X' \subseteq X$, $X\ g \circ f\ Z$, and $Z \subseteq Z'$. Now $X\ f\ Y$ and $Y\ g\ Z$ for some $Y \in D_1$. But then $X'\ f\ Y$ holds; for a similar reason $Y\ g\ Z'$ holds also. Therefore, $X'\ g \circ f\ Z'$ is established, which means that we have checked 2.1 (iii) for $g \circ f$ and have completed the proof that $g \circ f : D_0 \to D_2$.

The verification of associativity is a purely logical deduction. Thus suppose that in addition to f and g we have $h : D_2 \to D_3$. If $X \in D_0$ and $W \in D_3$ we find

$$
\begin{aligned}
X\ h \circ (g \circ f)\ W \text{ iff } & \exists Z \in D_2.\ X\ g \circ f\ Z \text{ and } Z\ h\ W \\
\text{iff } & \exists Z \in D_2\ \exists Y \in D_1.\ X\ f\ Y \text{ and } Y\ g\ Z \text{ and } Z\ h\ W \\
\text{iff } & \exists Y \in D_1\ \exists Z \in D_2.\ X\ f\ Y \text{ and } Y\ g\ Z \text{ and } Z\ h\ W \\
\text{iff } & \exists Y \in D_1.\ X\ f\ Y \text{ and } Y\ (h \circ g)\ W \\
\text{iff } & X\ (h \circ g) \circ f\ W.
\end{aligned}
$$

So, as relations, $h \circ (g \circ f) = (h \circ g) \circ f$. □

Theorem (`theorem_2_5`). Neighbourhood systems and approximable maps form a category: the identity relates neighbourhoods by inclusion (among members of the system); composition is existential mediation by an intermediate neighbourhood; and the identity and associativity laws hold for morphisms of bundled neighbourhood systems.

Proof. The relational characterizations are those of the identity map and of composition; the unit and associativity laws are the corresponding laws of the category of neighbourhood systems. ■

`Theorem_2_5.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.DomainObj

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace ApproximableMap

open CategoryTheory

/-- Scott's Theorem 2.5: neighbourhood systems and approximable maps form a category
    --
identity and composition of relations, with the Mathlib unit and associativity laws.
    -/
theorem theorem_2_5 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} {X Y : Set α},
      (idMap V).rel X Y ↔ V.mem X ∧ V.mem Y ∧ X ⊆ Y) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} {g : ApproximableMap V₁ V₂} {f : ApproximableMap V₀ V₁
    }
      {X : Set α} {Z : Set ?},
      (g.comp f).rel X Z ↔ ∃ Y, f.rel X Y ∧ g.rel Y Z) ∧
    (∀ {D E : DomainObj} (f : D ⟶ E), ? D ? f = f) ∧
    (∀ {D E : DomainObj} (f : D ⟶ E), f ? ? E = f) ∧
    (∀ {W X Y Z : DomainObj} (f : W ⟶ X) (g : X ⟶ Y) (h : Y ⟶ Z),
      (f ? g) ? h = f ? (g ? h)) :=
  ⟨@idMap_rel, @comp_rel, Category.id_comp, Category.comp_id, Category.assoc⟩

end ApproximableMap
end DomainTheory.NeighbourhoodSystems
```

4.3.12 Proposition 2.6 — identity and composition

It may seem as though we have, in the definition of composition, written things backwards. But the reason is that when mappings are taken as elementwise functions, then the order is preserved in expressions involving the usual function value notation. We have, for example:

PROPOSITION 2.6. Given $f : D_0 \to D_1$ and $g : D_1 \to D_2$, the following equations hold:

(i) $I_{D_0}(x) = x$, and

(ii) $(g \circ f)(x) = g(f(x))$,

for all $x \in |D_0|$. □

The proof is not troublesome and is left as an exercise. In technical language the result shows that the category defined in Theorem 2.5 is equivalent to a "concrete category" of sets and functions, namely the domains and elementwise transformations of 2.2.

Toward the end of the last lecture (see 1.9) we promised to show that isomorphisms of domains always come from approximable mappings, and this we now do. It means that the category contains all the isomorphisms it should have.

Theorem (`proposition_2_6`). The identity approximable map acts as the identity on ideal elements, and the elementwise action of a composite is the composite of the elementwise actions.

Proof. Assemble the corresponding facts for the identity map and for composition. ■

`Proposition_2_6.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.idMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace ApproximableMap

/-- Scott's Proposition 2.6: the identity acts as the identity on elements, and
    composition
of approximable maps acts by composition of the elementwise maps. -/
theorem proposition_2_6 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} (x : V.Element),
      (idMap V).toElementMap x = x) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (g : ApproximableMap V₁ V₂) (f : ApproximableMap V₀ V₁
    )
      (x : V₀.Element),
      (g.comp f).toElementMap x = g.toElementMap (f.toElementMap x)) :=
  ⟨@toElementMap_idMap, @toElementMap_comp⟩

end ApproximableMap
end DomainTheory.NeighbourhoodSystems
```

4.3.13 Order isomorphism as approximable map

The approximable map induced by an order isomorphism of ideal elements, recovering the isomorphism elementwise, and sending principal filters to principal filters.

Definition (`ofIso`). An order isomorphism of ideal elements induces an approximable map: a neighbourhood X relates to Y when Y belongs to the image of the principal filter of X.

Proof. Domain and codomain membership are immediate; the master relation uses the two masters; intersection and monotonicity follow from the corresponding properties of the image filter and monotonicity of the isomorphism. ■

Theorem (`exists_principal_eq_apply_principal`). An order isomorphism of ideal elements sends each principal filter to a principal filter.

Proof. Form the directed family of preimages of principals below the image of the given principal; its directed supremum recovers that principal, and membership of the original neighbourhood yields a principal that equals the image. ■

Theorem (`toElementMap_ofIso`). The elementwise action of the induced map is the original isomorphism.

Proof. One inequality is monotonicity on principals; the reverse uses the inverse isomorphism and the defining relation. ■

Theorem (`exists_approximable_of_iso`). Every order isomorphism of ideal elements arises as the elementwise action of some approximable map.

Proof. Take the induced map and the preceding recovery theorem. ■

`ofIso.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.iUnion
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff
public import Mathlib.Order.Hom.Basic
import Mathlib.Tactic.Set

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace ApproximableMap

section
variable {α β ? ? : Type*}

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}

/-- Approximable map induced by an order isomorphism of ideal elements:
`X` relates to `Y` when `Y` belongs to the image of the principal filter of `X`. -/
def ofIso (e : V₀.Element ≃o V₁.Element) : ApproximableMap V₀ V₁ where
  rel X Y := ∃ _ : V₀.mem X, (e (V₀.principal ?V₀.mem X?)).mem Y
  rel_dom := fun h => h.1
  rel_cod := fun h => (e _).sub h.2
  master_rel := ⟨V₀.master_mem, (e _).master_mem⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY⟩ ⟨_, hY'⟩
    exact ⟨hX, (e (V₀.principal hX)).inter_mem hY hY'⟩
  mono := by
```

```
    rintro X X' Y Y' ⟨hX, hY⟩ hX'X hYY' hX' hY'
    refine ⟨hX', ?_⟩
    have hle : V₀.principal hX ≤ V₀.principal hX' := (V₀.principal_le_iff hX hX').mpr
    hX'X
    have hmem : (e (V₀.principal hX')).mem Y := (e.monotone hle) Y hY
    exact (e (V₀.principal hX')).up_mem hmem hY' hYY'

/-- An order isomorphism of ideal elements sends principal filters to principal
    filters. -/
theorem exists_principal_eq_apply_principal (e : V₀.Element ≃o V₁.Element)
    {X : Set α} (hX : V₀.mem X) :
    ∃ (Y : Set β) (hY : V₁.mem Y), e (V₀.principal hX) = V₁.principal hY := by
  set w : V₁.Element := e (V₀.principal hX) with hw
  set S : Set V₀.Element :=
    {z | ∃ (Y : Set β) (hY : V₁.mem Y), w.mem Y ∧ z = e.symm (V₁.principal hY)} with hS

  have hsymm_w : e.symm w = V₀.principal hX := by rw [hw, e.symm_apply_apply]

  have hprin_le_w : ∀ {Y : Set β} (hY : V₁.mem Y), w.mem Y → V₁.principal hY ≤ w :=
    fun hY hYw Z hZ => w.up_mem hYw hZ.1 hZ.2

  have hne : S.Nonempty :=
    ⟨e.symm (V₁.principal V₁.master_mem), V₁.master, V₁.master_mem, w.master_mem, rfl⟩

  have hdir : ∀ a ∈ S, ∀ b ∈ S, ∃ c ∈ S, a ≤ c ∧ b ≤ c := by
    rintro a ⟨Y, hY, hYw, rfl⟩ b ⟨Y', hY', hY'w, rfl⟩
    have hYY'w : w.mem (Y ∩ Y') := w.inter_mem hYw hY'w
    have hYY' : V₁.mem (Y ∩ Y') := w.sub hYY'w
    refine ⟨e.symm (V₁.principal hYY'), ⟨Y ∩ Y', hYY', hYY'w, rfl⟩, ?_, ?_⟩
    · exact e.symm.monotone ((V₁.principal_le_iff hY hYY').mpr Set.inter_subset_left)
    · exact e.symm.monotone ((V₁.principal_le_iff hY' hYY').mpr Set.inter_subset_right)

  set z : V₀.Element := V₀.sSupDirected S hne hdir with hz

  have hz_le : z ≤ V₀.principal hX := by
    apply V₀.sSupDirected_le
    rintro s ⟨Y, hY, hYw, rfl⟩
    have : e.symm (V₁.principal hY) ≤ e.symm w := e.symm.monotone (hprin_le_w hY hYw)
    rwa [hsymm_w] at this

  have hw_le_ez : w ≤ e z := by
    intro Y hYw
    have hY : V₁.mem Y := w.sub hYw
    have hmem_S : e.symm (V₁.principal hY) ∈ S := ⟨Y, hY, hYw, rfl⟩
    have h1 : e.symm (V₁.principal hY) ≤ z := V₀.le_sSupDirected S hne hdir hmem_S
    have h2 : V₁.principal hY ≤ e z := by
      have := e.monotone h1
      rwa [e.apply_symm_apply] at this
    exact h2 Y ⟨hY, subset_rfl⟩
  have hX_le_z : V₀.principal hX ≤ z := by
    have : e.symm w ≤ e.symm (e z) := e.symm.monotone hw_le_ez
    rwa [hsymm_w, e.symm_apply_apply] at this

  have hz_eq : z = V₀.principal hX := le_antisymm hz_le hX_le_z
  have hXz : z.mem X := hz_eq ▸ ⟨hX, subset_rfl⟩
  obtain ⟨s, ⟨Y, hY, hYw, rfl⟩, hXs⟩ := hXz
```

```
  refine ⟨Y, hY, ?_⟩
  have hprinX_le : V₀.principal hX ≤ e.symm (V₁.principal hY) :=
    fun Z hZ => (e.symm (V₁.principal hY)).up_mem hXs hZ.1 hZ.2
  have hw_le_prinY : w ≤ V₁.principal hY := by
    have := e.monotone hprinX_le
    rw [e.apply_symm_apply] at this
    rwa [← hw] at this
  exact le_antisymm hw_le_prinY (hprin_le_w hY hYw)

theorem toElementMap_ofIso (e : V₀.Element ≃o V₁.Element) (x : V₀.Element) :
    (ofIso e).toElementMap x = e x := by
  have hgxle : (ofIso e).toElementMap x ≤ e x := by
    rintro Y ⟨X, hXx, hX, hY⟩
    have hpx : V₀.principal hX ≤ x := fun Z hZ => x.up_mem hXx hZ.1 hZ.2
    exact (e.monotone hpx) Y hY
  have key : x = e.symm ((ofIso e).toElementMap x) := by
    apply le_antisymm
    · intro X hXx
      have hX : V₀.mem X := x.sub hXx
      have hsub : e (V₀.principal hX) ≤ (ofIso e).toElementMap x :=
        fun Y hY => ⟨X, hXx, hX, hY⟩
      have hple : V₀.principal hX ≤ e.symm ((ofIso e).toElementMap x) := by
        have h := e.symm.monotone hsub
        rwa [e.symm_apply_apply] at h
      exact hple X ⟨hX, subset_rfl⟩
    · have h := e.symm.monotone hgxle
      rwa [e.symm_apply_apply] at h
  have h1 : e x = e (e.symm ((ofIso e).toElementMap x)) := congrArg e key
  rw [e.apply_symm_apply] at h1
  exact h1.symm

theorem exists_approximable_of_iso (e : V₀.Element ≃o V₁.Element) :
    ∃ f : ApproximableMap V₀ V₁, ∀ x, f.toElementMap x = e x :=
  ⟨ofIso e, toElementMap_ofIso e⟩

end

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

4.3.14 Theorem 2.7 — isomorphisms are approximable

THEOREM 2.7. Every isomorphism between domains results from an approximable mapping between the neighbourhood systems. Moreover, finite elements are always transformed into finite elements.

Proof: Suppose that $f : |D_0| \to |D_1|$ is a one-one, inclusion-preserving function defined on elements, where the range of the function is the whole of $|D_1|$, of course. Taking the hint from 2.2, there is only one way we could define a neighbourhood mapping; namely, we consider the relation $Y \in f(\uparrow X)$ for $X \in D_0$ and $Y \in D_1$. What has to be shown is that this is an approximable mapping which determines the original function via the formula 2.2 (i).

The first part is easy; indeed, there is a general result that monotone functions on finite elements of one domain to arbitrary elements of another domain always determine approximable mappings (cf. Exercise

2.8). What remains, then, is to show that the relation re-defines the function. This comes down to showing that for $x \in |D_0|$

$$f(x) = \{Y \in D_1 \mid \exists X \in x.\, Y \in f(\uparrow X)\}.$$

Consider the right-hand side of this equation: it is a filter. (This either can be proved directly or Exercise 2.11 can be used.) Because f is an onto-function, we can call the right-hand side $f(x')$ for some $x' \in |D_0|$. But since $X \in x$ implies $\uparrow X \subseteq x$ and $f(\uparrow X) \subseteq f(x)$, the right-hand side is included in the left-hand side. In other words $f(x') \subseteq f(x)$. But, since f is an isomorphism, $x' \subseteq x$ follows.

In the other direction, if $X \in x$, then $f(\uparrow X) \subseteq f(x')$ holds by definition, so $\uparrow X \subseteq x'$. This implies $X \in x'$; and, as X is arbitrary, $x \subseteq x'$. So $x = x'$, and $f(x) = f(x')$ as desired.

Finally, consider any finite element $\uparrow X \in |D_0|$, where $X \in D_0$. What we have to show is that $f(\uparrow X)$ is finite in $|D_1|$. Because f is an isomorphism, we can associate uniquely to every $Y \in f(\uparrow X)$ an element $y_Y \subseteq\uparrow X$ in $|D_0|$ where $f(y_Y) =\uparrow Y$. (Just apply the inverse of the function f.) Define

$$z = \bigcup\{y_Y \mid Y \in f(\uparrow X)\}.$$

Because $Y' \subseteq Y$ always implies $y_{Y'} \subseteq y_Y$ and each $y_Y \in |D_0|$, it is easy to show z is a filter and hence is in $|D_0|$ also (cf. Exercise 2.11). Because each $y_Y \subseteq\uparrow X$, then $z \subseteq\uparrow X$, too. But each $y_Y \subseteq z$, so $\uparrow Y = f(y_Y) \subseteq f(z)$ and hence $Y \in f(z)$. As this holds for all $Y \in f(\uparrow X)$, the inclusion $f(\uparrow X) \subseteq f(z)$ follows, as well as $\uparrow X \subseteq z$. Therefore, $z =\uparrow X$ and so $X \in z$. But then $X \in y_Y$ for some $Y \in f(\uparrow X)$, by definition of z. Since $\uparrow X \subseteq y_Y$, we obtain $f(\uparrow X) \subseteq\uparrow Y$, but of course the opposite inclusion is also true from the choice of Y. This means that $f(\uparrow X) =\uparrow Y$ is finite in $|D_1|$ as claimed. We can apply the same argument to the inverse function; and, thus, the finite elements of $|D_0|$ and $|D_1|$ are in a one-one inclusion-preserving correspondence under the isomorphism. □

Theorem (`theorem_2_7`). Every order isomorphism of ideal elements arises as the elementwise action of an approximable map, and principal filters are sent to principal filters.

Proof. Assemble the existence of an approximable realization and the principal-to-principal lemma. ■

`Theorem_2_7.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofIso

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace ApproximableMap

/-- Scott's Theorem 2.7: every order isomorphism of domains arises from an
    approximable map,
and finite (principal) elements are sent to finite elements. -/
theorem theorem_2_7 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (e : V₀.Element ≃o V₁.Element),
      ∃ f : ApproximableMap V₀ V₁, ∀ x, f.toElementMap x = e x) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (e : V₀.Element ≃o V₁.Element) {X : Set α} (hX : V₀.mem X),
      ∃ (Y : Set β) (hY : V₁.mem Y), e (V₀.principal hX) = V₁.principal hY) :=
  ⟨@exists_approximable_of_iso, @exists_principal_eq_apply_principal⟩
```

```
end ApproximableMap
end DomainTheory.NeighbourhoodSystems
```

4.3.15 Monotone on principals as approximable map

Approximable maps determined by their action on principal filters, and the approximable map induced by a monotone assignment of ideal elements to neighbourhoods.

Definition (`ofMono`). A monotone assignment of ideal elements to neighbourhoods induces an approximable map: X relates to Y when Y belongs to the assigned element at X.

Proof. Domain and codomain membership are immediate; master, intersection, and monotonicity follow from the corresponding properties of the assigned elements and the monotonicity hypothesis. ■

Theorem (`eq_of_toElementMap_principal`). Approximable maps that agree on all principal filters are equal.

Proof. Recover the neighbourhood relation from evaluation at principals. ■

Theorem (`toElementMap_ofMono_principal`). The elementwise action of the induced map at a principal filter recovers the assigned element.

Proof. Membership in the image comes from a smaller neighbourhood by monotonicity; conversely the neighbourhood itself witnesses the relation. ■

`ofMono.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace ApproximableMap

section
variable {α β ? : Type*}

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}

/-- Approximable map induced by a monotone assignment of ideal elements to
    neighbourhoods
(equivalently: a monotone map on principal filters). -/
def ofMono (m : (X : Set α) → V₀.mem X → V₁.Element)
    (hmono : ∀ (X X' : Set α) (hX : V₀.mem X) (hX' : V₀.mem X'), X' ⊆ X → m X hX ≤ m X
    ' hX') :
    ApproximableMap V₀ V₁ where
  rel X Y := ∃ hX : V₀.mem X, (m X hX).mem Y
  rel_dom := fun ⟨hX, _⟩ => hX
  rel_cod := fun ⟨hX, hY⟩ => (m _ hX).sub hY
  master_rel := ⟨V₀.master_mem, (m _ V₀.master_mem).master_mem⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY⟩ ⟨_, hY'⟩
```

```
    exact ⟨hX, (m X hX).inter_mem hY hY'⟩
  mono := by
    rintro X X' Y Y' ⟨hX, hY⟩ hX'X hYY' hX' hY'
    have hle : m X hX ≤ m X' hX' := hmono X X' hX hX' hX'X
    exact ⟨hX', (m X' hX').up_mem (hle Y hY) hY' hYY'⟩

/-- Approximable maps agreeing on all principal filters are equal. -/
theorem eq_of_toElementMap_principal {f g : ApproximableMap V₀ V₁}
    (h : ∀ (X : Set α) (hX : V₀.mem X),
      f.toElementMap (V₀.principal hX) = g.toElementMap (V₀.principal hX)) : f = g :=
    by
  apply ApproximableMap.ext
  intro X Y
  by_cases hX : V₀.mem X
  · rw [f.rel_iff_mem_principal hX, g.rel_iff_mem_principal hX, h X hX]
  · constructor
    · intro hr; exact absurd (f.rel_dom hr) hX
    · intro hr; exact absurd (g.rel_dom hr) hX

theorem toElementMap_ofMono_principal
    (m : (X : Set α) → V₀.mem X → V₁.Element)
    (hmono : ∀ (X X' : Set α) (hX : V₀.mem X) (hX' : V₀.mem X'), X' ⊆ X → m X hX ≤ m X
    ' hX')
    (X : Set α) (hX : V₀.mem X) :
    (ofMono m hmono).toElementMap (V₀.principal hX) = m X hX := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨Z, ⟨hZmem, hXZ⟩, hZ', hmY⟩
    have hle : m Z hZ' ≤ m X hX := hmono Z X hZ' hX hXZ
    exact hle Y hmY
  · intro hmY
    exact ⟨X, ⟨hX, subset_rfl⟩, hX, hmY⟩

end

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

4.3.16 Exercise 2.8 — determined by finite elements

With reference to the proof of 2.2 show that an approximable mapping is uniquely determined by its elementwise effect on finite elements. Moreover any arbitrary monotone function on finite elements of $|D_0|$ with values in $|D_1|$ comes from an approximable $f : D_0 \to D_1$.

Theorem (`exercise_2_8`). Approximable maps that agree on all principal filters are equal, and every monotone assignment of ideal elements to neighbourhoods arises as the principal action of the induced approximable map.

Proof. Assemble uniqueness on principals with recovery of the assigned element at each principal. ■

`Exercise_2_8.lean`

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofMono

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open ApproximableMap

/-- Scott's Exercise 2.8: approximable maps are determined by their action on
    principal
filters, and every monotone assignment of elements to neighbourhoods arises that way.
    -/
theorem exercise_2_8 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {f g : ApproximableMap V₀ V₁},
      (∀ (X : Set α) (hX : V₀.mem X),
        f.toElementMap (V₀.principal hX) = g.toElementMap (V₀.principal hX)) →
      f = g) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (m : (X : Set α) → V₀.mem X → V₁.Element)
      (hmono : ∀ (X X' : Set α) (hX : V₀.mem X) (hX' : V₀.mem X'),
        X' ⊆ X → m X hX ≤ m X' hX')
      (X : Set α) (hX : V₀.mem X),
      (ofMono m hmono).toElementMap (V₀.principal hX) = m X hX) :=
  ⟨fun {_ _} {_ _} {_ _} => eq_of_toElementMap_principal,
    fun {_ _} {_ _} => toElementMap_ofMono_principal⟩
```

4.3.17 Exercise 2.9 — directed union formula

Prove that if $f : D_0 \to D_1$ is an approximable mapping, then the elementwise mapping $f : |D_0| \to |D_1|$ satisfies the equation

$$f(x) = \bigcup \{ f(\uparrow X) \mid X \in x \}$$

for all $x \in |D_0|$. Conversely, show that every elementwise function satisfying this equation comes from an approximable mapping as defined in 2.2.

Theorem (`exercise_2_9`). Membership in the elementwise image is witnessed by a principal approximation: a neighbourhood belongs to $f(x)$ if and only if it belongs to $f(\uparrow X)$ for some neighbourhood X of x.

Proof. This is the membership form of the directed-union equation; it is the corresponding library fact for the elementwise action. ■

Exercise_2_9.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open ApproximableMap
```

```
/-- Scott's Exercise 2.9 (forward direction): the elementwise image is the directed
    union of
images of principal approximations -- in membership form,
$Y \in f(x)$ iff $Y \in f(\uparrow X)$ for some $X \in x$. -/
theorem exercise_2_9 :
    ∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (f : ApproximableMap V₀ V₁) (x : V₀.Element) {Y : Set β},
      (f.toElementMap x).mem Y ↔
        ∃ (X : Set α) (hx : x.mem X), (f.toElementMap (V₀.principal (x.sub hx))).mem Y
    :=
  toElementMap_mem_iff_principal
```

4.3.18 Pointwise intersection of approximable maps

The pointwise meet of two approximable maps, whose elementwise action is intersection of filters.

Definition (`interMap`). The pointwise intersection of two approximable maps relates neighbourhoods precisely when both components do.

Proof. Master, intersection, and monotonicity hold componentwise. ■

Theorem (`mem_toElementMap_interMap`). The elementwise action of the pointwise intersection is the intersection of the elementwise actions.

Proof. One direction splits a common witness; the converse intersects the two witnessing neighbourhoods and uses monotonicity. ■

`interMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace ApproximableMap

section
variable {α β ? : Type*}

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}

/-- Pointwise meet of two approximable maps: related when both components relate. -/
def interMap (f g : ApproximableMap V₀ V₁) : ApproximableMap V₀ V₁ where
  rel X Z := f.rel X Z ∧ g.rel X Z
  rel_dom h := f.rel_dom h.1
  rel_cod h := f.rel_cod h.1
  master_rel := ⟨f.master_rel, g.master_rel⟩
  inter_right := fun ⟨hf, hg⟩ ⟨hf', hg'⟩ => ⟨f.inter_right hf hf', g.inter_right hg hg'⟩
  mono := fun ⟨hf, hg⟩ hX'X hZZ' hX' hZ' =>
    ⟨f.mono hf hX'X hZZ' hX' hZ', g.mono hg hX'X hZZ' hX' hZ'⟩
```

```
theorem mem_toElementMap_interMap (f g : ApproximableMap V₀ V₁) (x : V₀.Element) {Z :
    Set β} :
    ((interMap f g).toElementMap x).mem Z ↔
      (f.toElementMap x).mem Z ∧ (g.toElementMap x).mem Z := by
  constructor
  · rintro ⟨X, hxX, hf, hg⟩
    exact ⟨⟨X, hxX, hf⟩, ⟨X, hxX, hg⟩⟩
  · rintro ⟨⟨X, hxX, hf⟩, ⟨X', hxX', hg⟩⟩
    have hxXX' : x.mem (X ∩ X') := x.inter_mem hxX hxX'
    have hXX' : V₀.mem (X ∩ X') := x.sub hxXX'
    refine ⟨X ∩ X', hxXX', ?_, ?_⟩
    · exact f.mono hf Set.inter_subset_left subset_rfl hXX' (f.rel_cod hf)
    · exact g.mono hg Set.inter_subset_right subset_rfl hXX' (g.rel_cod hg)

end

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

4.3.19 Exercise 2.10 — intersection of two maps

Carry out the proof of Proposition 2.6; and in addition show that, if $f, g : D_0 \to D_1$ are two approximable mappings, there exists $h : D_0 \to D_1$ such that

$$h(x) = f(x) \cap g(x)$$

for all $x \in |D_0|$.

Theorem (`exercise_2_10`). The pointwise intersection of two approximable maps acts elementwise by intersection of filters.

Proof. This is the membership characterization of the pointwise meet. (The identity and composition laws of Proposition 2.6 are packaged separately.) ■

`Exercise_2_10.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.interMap

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open ApproximableMap

/-- Scott's Exercise 2.10 (meet clause): the pointwise intersection of two
    approximable maps
is again approximable and acts elementwise by intersection of filters. -/
theorem exercise_2_10 :
    ∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (f g : ApproximableMap V₀ V₁) (x : V₀.Element) {Z : Set β},
      ((interMap f g).toElementMap x).mem Z ↔
        (f.toElementMap x).mem Z ∧ (g.toElementMap x).mem Z :=
  mem_toElementMap_interMap
```

4.3.20 Indexed directed supremum of ideal elements

Directed joins of ideal elements presented by an index type, and preservation of those joins by approximable maps.

Definition (`iSupDirected`). The directed supremum of a nonempty directed family of ideal elements, presented as a function on an index type, is the directed join of the range of that function.

Theorem (`mem_iSupDirected`). A neighbourhood belongs to the indexed directed supremum if and only if it belongs to some term of the family.

Theorem (`iSupDirected_le`). The indexed directed supremum is below any common upper bound of the family.

Theorem (`le_iSupDirected`). Each term of the family is below the indexed directed supremum.

Theorem (`toElementMap_iSupDirected`). The elementwise action of an approximable map preserves indexed directed suprema.

Proof. Membership in the image of the join is membership for some term, transported by the defining formula for the elementwise action. ■

`iSupDirected.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.iUnion
public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Image

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace NeighborhoodSystem

section
variable {α : Type*}

/-- Indexed directed supremum of ideal elements (Exercise 2.11): the directed join of
    a
nonempty directed family presented as a function on an index type. -/
def iSupDirected {V : NeighborhoodSystem α} {I : Type*} [Nonempty I]
    (a : I → V.Element) (hdir : ∀ i j, ∃ k, a i ≤ a k ∧ a j ≤ a k) : V.Element :=
  V.sSupDirected (Set.range a) (Set.range_nonempty a) (by
    rintro _ ⟨i, rfl⟩ _ ⟨j, rfl⟩
    obtain ⟨k, hik, hjk⟩ := hdir i j
    exact ⟨a k, ⟨k, rfl⟩, hik, hjk⟩)

theorem mem_iSupDirected {V : NeighborhoodSystem α} {I : Type*} [Nonempty I]
    (a : I → V.Element) (hdir : ∀ i j, ∃ k, a i ≤ a k ∧ a j ≤ a k) {Z : Set α} :
    (iSupDirected a hdir).mem Z ↔ ∃ i, (a i).mem Z := by
  constructor
  · rintro ⟨s, ⟨i, rfl⟩, hsZ⟩; exact ⟨i, hsZ⟩
  · rintro ⟨i, hi⟩; exact ⟨a i, ⟨i, rfl⟩, hi⟩

theorem iSupDirected_le {V : NeighborhoodSystem α} {I : Type*} [Nonempty I]
    (a : I → V.Element) (hdir : ∀ i j, ∃ k, a i ≤ a k ∧ a j ≤ a k) {y : V.Element}
```

```
    (hy : ∀ i, a i ≤ y) : iSupDirected a hdir ≤ y := by
  intro Z hZ
  obtain ⟨i, hi⟩ := (mem_iSupDirected a hdir).mp hZ
  exact hy i Z hi

theorem le_iSupDirected {V : NeighborhoodSystem α} {I : Type*} [Nonempty I]
    (a : I → V.Element) (hdir : ∀ i j, ∃ k, a i ≤ a k ∧ a j ≤ a k) (i : I) :
    a i ≤ iSupDirected a hdir :=
  fun _ hZ => (mem_iSupDirected a hdir).mpr ⟨i, hZ⟩

end

end NeighborhoodSystem

namespace ApproximableMap

section
variable {α β : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

/-- Approximable maps preserve indexed directed suprema. -/
theorem toElementMap_iSupDirected (f : ApproximableMap V₀ V₁) {I : Type*} [Nonempty I]
    (a : I → V₀.Element) (hdir : ∀ i j, ∃ k, a i ≤ a k ∧ a j ≤ a k) :
    f.toElementMap (NeighborhoodSystem.iSupDirected a hdir) =
      NeighborhoodSystem.iSupDirected (fun i => f.toElementMap (a i))
        (fun i j => by
          obtain ⟨k, hik, hjk⟩ := hdir i j
          exact ⟨k, f.toElementMap_mono hik, f.toElementMap_mono hjk⟩) := by
  apply Element.ext
  intro Y
  rw [mem_toElementMap, NeighborhoodSystem.mem_iSupDirected]
  constructor
  · rintro ⟨X, hX, hrel⟩
    obtain ⟨i, hi⟩ := (NeighborhoodSystem.mem_iSupDirected a hdir).mp hX
    exact ⟨i, X, hi, hrel⟩
  · rintro ⟨i, X, hi, hrel⟩
    exact ⟨X, (NeighborhoodSystem.mem_iSupDirected a hdir).mpr ⟨i, hi⟩, hrel⟩

end

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

4.3.21 Exercise 2.11 — directed unions of elements

Let $(I, \leq)$ be a non-empty abstract partially ordered set; suppose it is directed in the sense that whenever $i, j \in I$, then $i \leq k$ and $j \leq k$ for some $k \in I$. Suppose that $a : I \to |D|$ is such that

$$i \leq j \text{ implies } a_i \subseteq a_j$$

for all $i, j \in I$. Prove that

$$\bigcup \{a_i \mid i \in I\}$$

is always a filter in $|D|$. (Note the ways this lemma could be used in the proof of 2.7; but be careful in defining the partially ordered set and do not confuse $\subseteq$ and $\supseteq$.) In words we could say that the domain of filters is closed under directed unions. Prove also that if $f : D \to D'$ is an approximable mapping, then for any directed union

$$f(\bigcup\{a_i \mid i \in I\}) = \bigcup\{f(a_i) \mid i \in I\};$$

that is, approximable mappings always preserve directed unions. If an elementwise function preserves directed unions, must it come from an approximable mapping? (Hint: Invoke 2.9.)

Theorem (`exercise_2_11`). Domains are closed under indexed directed unions of ideal elements (membership in the join is membership in some term), and approximable maps preserve those unions elementwise.

Proof. Assemble the membership characterization of the indexed directed supremum with preservation under the elementwise action. ■

`Exercise_2_11.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupDirected

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem

/-- Scott's Exercise 2.11: domains are closed under directed unions of ideal elements,
    and
approximable maps preserve those unions. -/
theorem exercise_2_11 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} {I : Type*} [Nonempty I]
      (a : I → V.Element) (hdir : ∀ i j, ∃ k, a i ≤ a k ∧ a j ≤ a k) {Z : Set α},
      (iSupDirected a hdir).mem Z ↔ ∃ i, (a i).mem Z) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (f : ApproximableMap V₀ V₁) {I : Type*} [Nonempty I]
      (a : I → V₀.Element) (hdir : ∀ i j, ∃ k, a i ≤ a k ∧ a j ≤ a k),
      f.toElementMap (iSupDirected a hdir) =
        iSupDirected (fun i => f.toElementMap (a i))
          (fun i j => by
            obtain ⟨k, hik, hjk⟩ := hdir i j
            exact ⟨k, f.toElementMap_mono hik, f.toElementMap_mono hjk⟩)) :=
  ⟨fun {_ _} {_ _} => mem_iSupDirected,
    fun {_ _} {_ _} => ApproximableMap.toElementMap_iSupDirected⟩
```

4.3.22 Directed supremum of approximable maps

The directed join of a nonempty directed family of approximable maps, acting elementwise by directed union of the images.

Definition (`iSupMap`). The directed supremum of a nonempty directed family of approximable maps relates neighbourhoods when some member of the family does.

Proof. Master uses any index; intersection finds a common upper bound in the directed family; monotonicity is inherited from any witnessing member. ■

Theorem (`mem_toElementMap_iSupMap`). Membership in the elementwise image of the directed supremum is membership in the image of some member of the family.

Proof. Reassociate the witnessing neighbourhood and index. ■

`iSupMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace ApproximableMap

section
variable {α β : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

/-- Directed supremum of a nonempty directed family of approximable maps (pointwise on
relations): related when some member of the family relates. -/
def iSupMap {I : Type*} [Nonempty I] (f : I → ApproximableMap V₀ V₁)
    (hdir : ∀ i j, ∃ k, (∀ X Y, (f i).rel X Y → (f k).rel X Y) ∧
      (∀ X Y, (f j).rel X Y → (f k).rel X Y)) : ApproximableMap V₀ V₁ where
  rel X Z := ∃ i, (f i).rel X Z
  rel_dom := fun ⟨i, h⟩ => (f i).rel_dom h
  rel_cod := fun ⟨i, h⟩ => (f i).rel_cod h
  master_rel := by obtain ⟨i⟩ := (inferInstance : Nonempty I); exact ⟨i, (f i).
    master_rel⟩
  inter_right := by
    rintro X Z Z' ⟨i, hi⟩ ⟨j, hj⟩
    obtain ⟨k, hik, hjk⟩ := hdir i j
    exact ⟨k, (f k).inter_right (hik X Z hi) (hjk X Z' hj)⟩
  mono := by
    rintro X X' Z Z' ⟨i, hi⟩ hX'X hZZ' hX' hZ'
    exact ⟨i, (f i).mono hi hX'X hZZ' hX' hZ'⟩

theorem mem_toElementMap_iSupMap {I : Type*} [Nonempty I] (f : I → ApproximableMap V₀
    V₁)
    (hdir : ∀ i j, ∃ k, (∀ X Y, (f i).rel X Y → (f k).rel X Y) ∧
      (∀ X Y, (f j).rel X Y → (f k).rel X Y)) (x : V₀.Element) {Y : Set β} :
    ((iSupMap f hdir).toElementMap x).mem Y ↔ ∃ i, ((f i).toElementMap x).mem Y := by
  constructor
  · rintro ⟨X, hxX, i, hrel⟩
    exact ⟨i, X, hxX, hrel⟩
  · rintro ⟨i, X, hxX, hrel⟩
    exact ⟨X, hxX, i, hrel⟩

end

end ApproximableMap
```

```
end DomainTheory.NeighbourhoodSystems
```

4.3.23 Exercise 2.12 — directed unions of maps

Suppose $(I, \leq)$ is a directed, partially ordered set and $f_i : D_0 \to D_1$ is a family of approximable mappings indexed by $i \in I$, where we assume

$$i \leq j \text{ implies } f_i(x) \subseteq f_j(x)$$

for all $i, j \in I$ and all $x \in |D_0|$. Prove that there is an approximable mapping $g : D_0 \to D_1$ where

$$g(x) = \bigcup \{f_i(x) \mid i \in I\}$$

for all $x \in |D_0|$.

Theorem (`exercise_2_12`). Membership in the elementwise image of the directed supremum of a directed family of approximable maps is membership in the image of some member of the family.

Proof. This is the membership characterization of the directed supremum of maps. ■

`Exercise_2_12.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupMap

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open ApproximableMap

/-- Scott's Exercise 2.12: a directed family of approximable maps has a directed
    supremum
whose elementwise action is the pointwise directed union of the actions. -/
theorem exercise_2_12 :
    ∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {I : Type*} [Nonempty I] (f : I → ApproximableMap V₀ V₁)
      (hdir : ∀ i j, ∃ k, (∀ X Y, (f i).rel X Y → (f k).rel X Y) ∧
        (∀ X Y, (f j).rel X Y → (f k).rel X Y))
      (x : V₀.Element) {Y : Set β},
      ((iSupMap f hdir).toElementMap x).mem Y ↔ ∃ i, ((f i).toElementMap x).mem Y :=
  mem_toElementMap_iSupMap
```

4.3.24 Continuous maps as approximable

Continuous maps of ideal elements as approximable maps, and the converse continuity of elementwise actions.

Definition (`instNonemptyMemSubtype`). For each ideal element, the type of neighbourhoods belonging to that element is nonempty (witnessed by the master neighbourhood).

Theorem (`continuous_monotone`). A continuous map of ideal elements is monotone for the approximation order.

Proof. Preimages of opens are open, and opens are upper sets for approximation. ■

Theorem (`mem_iff_principal_of_continuous`). Under a continuous map, a neighbourhood belongs to the image of an ideal element if and only if it belongs to the image of some principal filter of a neighbourhood of that element.

Proof. One direction uses continuity at the basic open determined by the neighbourhood; the converse uses monotonicity of continuous maps. ■

Theorem (`principalFamily_directed`). The family of principal filters of neighbourhoods of a fixed ideal element is directed under approximation.

Proof. Intersect two neighbourhoods and compare principals by the principal-order characterisation. ■

Theorem (`eq_iSupDirected_principal`). Every ideal element is the indexed directed supremum of the principal filters of its neighbourhoods.

Proof. Membership on either side rearranges the directed-join formula against the filter axioms. ■

Theorem (`continuous_of_monotone_iSupDirected`). A monotone map of ideal elements that preserves indexed directed joins is continuous.

Proof. Express the point as the directed join of its principals, apply preservation of joins, and extract a basic open neighbourhood from membership in an open of the image. ■

Definition (`ofContinuous`). A continuous map of ideal elements induces an approximable map by evaluating the continuous map on principal filters.

Proof. Monotonicity of the assignment follows from continuity. ■

Theorem (`continuous_toElementMap`). The elementwise action of an approximable map is continuous.

Proof. Membership of the image in an open is witnessed by a related neighbourhood, which supplies a basic open preimage. ■

Theorem (`toElementMap_ofContinuous`). The elementwise action of the approximable map induced by a continuous map recovers that continuous map.

Proof. Compare membership via the principal-union law for continuous maps and evaluation of the induced map on principals. ■

`ofContinuous.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofMono
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupDirected
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

section
open NeighborhoodSystem
universe u
variable {α : Type u} {β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem
    β}
```

```
instance instNonemptyMemSubtype (x : V₀.Element) : Nonempty {X : Set α // x.mem X} :=
  ⟨⟨V₀.master, x.master_mem⟩⟩

/-- Continuous maps of ideal elements are monotone for approximation. -/
theorem continuous_monotone {c : V₀.Element → V₁.Element} (hc : Continuous c) :
    Monotone c := by
  intro x y hxy
  rw [V₁.le_iff_isOpen_imp]
  intro U hU hxU
  exact V₀.isOpen_isUpperSet (hU.preimage hc) hxU hxy

theorem mem_iff_principal_of_continuous {c : V₀.Element → V₁.Element} (hc : Continuous
    c)
    (x : V₀.Element) {Y : Set β} :
    (c x).mem Y ↔ ∃ (X : Set α) (hx : x.mem X), (c (V₀.principal (x.sub hx))).mem Y :=
    by
  constructor
  · intro hY
    have hxpre : x ∈ c ^-1' V₁.basicOpen Y := hY
    have hopen : IsOpen (c ^-1' V₁.basicOpen Y) := (V₁.isOpen_basicOpen Y).preimage hc
    obtain ⟨X, hxX, hXU⟩ := hopen.out x hxpre
    refine ⟨X, hxX, ?_⟩
    exact hXU (show V₀.principal (x.sub hxX) ∈ V₀.basicOpen X from ⟨x.sub hxX,
    subset_rfl⟩)
  · rintro ⟨X, hxX, hY⟩
    have hple : V₀.principal (x.sub hxX) ≤ x := fun Z hZ => x.up_mem hxX hZ.1 hZ.2
    exact (continuous_monotone hc hple) Y hY

theorem principalFamily_directed (x : V₀.Element) :
    ∀ i j : {X : Set α // x.mem X}, ∃ k : {X : Set α // x.mem X},
      V₀.principal (x.sub i.2) ≤ V₀.principal (x.sub k.2) ∧
        V₀.principal (x.sub j.2) ≤ V₀.principal (x.sub k.2) :=
  fun i j => ⟨⟨i.1 ∩ j.1, x.inter_mem i.2 j.2⟩,
    (V₀.principal_le_iff (x.sub i.2) (x.sub (x.inter_mem i.2 j.2))).mpr Set.
    inter_subset_left,
    (V₀.principal_le_iff (x.sub j.2) (x.sub (x.inter_mem i.2 j.2))).mpr Set.
    inter_subset_right⟩

theorem eq_iSupDirected_principal (x : V₀.Element) :
    x = iSupDirected (fun i : {X : Set α // x.mem X} => V₀.principal (x.sub i.2))
      (principalFamily_directed x) := by
  apply Element.ext
  intro Z
  rw [mem_iSupDirected]
  constructor
  · intro hZ; exact ⟨⟨Z, hZ⟩, (V₀.mem_principal _).mpr ⟨x.sub hZ, subset_rfl⟩⟩
  · rintro ⟨⟨X, hX⟩, hZ'⟩
    obtain ⟨hZmem, hXZ⟩ := (V₀.mem_principal _).mp hZ'
    exact x.up_mem hX hZmem hXZ

/-- A monotone map that preserves indexed directed suprema is continuous. -/
theorem continuous_of_monotone_iSupDirected {c : V₀.Element → V₁.Element} (hmono :
    Monotone c)
    (hsup : ∀ {I : Type u} [Nonempty I] (d : I → V₀.Element)
      (hdir : ∀ i j, ∃ k, d i ≤ d k ∧ d j ≤ d k)
```

```
      (hdir' : ∀ i j, ∃ k, c (d i) ≤ c (d k) ∧ c (d j) ≤ c (d k)),
      c (iSupDirected d hdir) = iSupDirected (fun i => c (d i)) hdir') :
    Continuous c := by
  rw [continuous_def]
  intro U hU
  constructor
  intro x hx
  rw [Set.mem_preimage] at hx
  set fam : {X : Set α // x.mem X} → V₀.Element := fun i => V₀.principal (x.sub i.2)
    with hfam
  have hdir : ∀ i j : {X : Set α // x.mem X}, ∃ k, fam i ≤ fam k ∧ fam j ≤ fam k :=
    principalFamily_directed x
  have hdir' : ∀ i j : {X : Set α // x.mem X}, ∃ k, c (fam i) ≤ c (fam k) ∧ c (fam j)
    ≤ c (fam k) :=
    fun i j => by obtain ⟨k, hik, hjk⟩ := hdir i j; exact ⟨k, hmono hik, hmono hjk⟩
  have hxeq : c x = iSupDirected (fun i => c (fam i)) hdir' :=
    (congrArg c (eq_iSupDirected_principal x)).trans
      (hsup (I := {X : Set α // x.mem X}) fam hdir hdir')
  rw [hxeq] at hx
  obtain ⟨Y, hY, hYU⟩ := hU.out _ hx
  obtain ⟨i, hi⟩ := (mem_iSupDirected _ hdir').mp hY
  refine ⟨i.1, i.2, fun x' hx' => hYU ?_⟩
  have hple : fam i ≤ x' := fun Z hZ => x'.up_mem hx' hZ.1 hZ.2
  exact (hmono hple) Y hi

end

end NeighborhoodSystem

namespace ApproximableMap

section
open NeighborhoodSystem
universe u
variable {α : Type u} {β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem
    β}

/-- Approximable map induced by a continuous map of ideal elements. -/
def ofContinuous (c : V₀.Element → V₁.Element) (hc : Continuous c) : ApproximableMap V₀
     V₁ :=
  ofMono (fun _X hX => c (V₀.principal hX))
    (fun _X _X' hX hX' hX'X =>
      NeighborhoodSystem.continuous_monotone hc ((V₀.principal_le_iff hX hX').mpr hX'X)
    )

theorem continuous_toElementMap (f : ApproximableMap V₀ V₁) :
    Continuous (fun x => f.toElementMap x) := by
  rw [continuous_def]
  intro U hU
  show V₀.OpenFilter _
  constructor
  intro x hx
  obtain ⟨Y, hY, hYU⟩ := hU.out (f.toElementMap x) hx
  obtain ⟨X, hxX, hrel⟩ := hY
  exact ⟨X, hxX, fun x' hx' => hYU ⟨X, hx', hrel⟩⟩
```

```
theorem toElementMap_ofContinuous (c : V₀.Element → V₁.Element) (hc : Continuous c)
    (x : V₀.Element) : (ofContinuous c hc).toElementMap x = c x := by
  apply Element.ext
  intro Y
  rw [toElementMap_mem_iff_principal, NeighborhoodSystem.
    mem_iff_principal_of_continuous hc]
  constructor
  · rintro ⟨X, hxX, hmem⟩
    refine ⟨X, hxX, ?_⟩
    rwa [ofContinuous, toElementMap_ofMono_principal] at hmem
  · rintro ⟨X, hxX, hmem⟩
    refine ⟨X, hxX, ?_⟩
    rwa [ofContinuous, toElementMap_ofMono_principal]

end

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

4.3.25 Exercise 2.13 — approximable means continuous

(For topologists.) Recall Exercise 1.22 where it was shown that any domain $|D|$ is a topological space. Prove from Exercise 2.9 that the functions $f : |D_0| \to |D_1|$ determined by approximable mappings are exactly *the continuous functions between these spaces.* (Hint: To prove continuity, remark that by 2.9

$$f^{-1}[Y] = \bigcup\{[X] \mid Y \in f(\uparrow X)\};$$

hence, the inverse image of any open set is open. In the other direction, suppose that $f : |D_0| \to |D_1|$ is topologically continuous. Argue that for all $x \in |D_0|$ and all open subsets $U \subseteq |D_1|$ we have

$$f(x) \in U \text{ iff } \exists X \in x.\ f(\uparrow X) \in U.$$

This holds because an open subset of $|D_0|$ is always a union of basic open subsets of the form $[X']$ for $X \in D_0$ and because

$$x = \bigcup\{\uparrow X \mid X \in x\}$$

for all $x \in |D_0|$.)

Theorem (`exercise_2_13`). Elementwise actions of approximable maps are continuous; continuous maps of ideal elements satisfy the principal-union membership law; and every continuous map arises as the elementwise action of an approximable map.

Proof. Continuity of elementwise actions, the principal-union law for continuous maps, and recovery of a continuous map from its induced approximable map. ■

`Exercise_2_13.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofContinuous

@[expose] public section
```

```
open DomainTheory.NeighbourhoodSystems

/-- Scott's Exercise 2.13: elementwise actions of approximable maps are continuous;
    continuous
maps satisfy the principal-union membership law; and every continuous map arises as
    the
elementwise action of an approximable map. -/
theorem exercise_2_13 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (f : ApproximableMap V₀ V₁), Continuous fun x => f.toElementMap x) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {c : V₀.Element → V₁.Element}, Continuous c →
      ∀ (x : V₀.Element) {Y : Set β},
        (c x).mem Y ↔ ∃ (X : Set α) (hx : x.mem X), (c (V₀.principal (x.sub hx))).mem
    Y) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (c : V₀.Element → V₁.Element) (hc : Continuous c) (x : V₀.Element),
      (ApproximableMap.ofContinuous c hc).toElementMap x = c x) :=
  ⟨@ApproximableMap.continuous_toElementMap,
    fun {_ _} {_ _} {_} => NeighborhoodSystem.mem_iff_principal_of_continuous,
    @ApproximableMap.toElementMap_ofContinuous⟩
```

4.3.26 Neighbourhood correspondence of an order isomorphism

The neighbourhood correspondence induced by an order isomorphism of ideal elements, its preservation of intersections, and the containment characterisation of the induced approximable map.

Definition (`phi`). An order isomorphism of ideal elements determines a neighbourhood correspondence: the image of the principal filter of a neighbourhood is again a principal filter, and φ selects that neighbourhood.

Theorem (`phi_mem`). The selected neighbourhood is a neighbourhood of the codomain system.

Proof. Part of the choice data from the principal-image theorem. ■

Theorem (`phi_spec`). Applying the isomorphism to the principal filter of a neighbourhood yields the principal filter of the corresponding neighbourhood.

Proof. The remaining clause of that choice data. ■

Theorem (`phi_inter`). The correspondence preserves intersections of consistent neighbourhoods.

Proof. Compare principals via monotonicity of the isomorphism and its inverse; the least-upper-bound property of the intersection principal yields both inclusions. ■

Theorem (`rel_ofIso_iff`). The approximable map induced by the isomorphism relates X to Y if and only if Y is a neighbourhood containing the corresponding neighbourhood of X.

Proof. Rewrite membership in the image principal via the correspondence specification. ■

`phi.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofIso
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff

@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems

section
open NeighborhoodSystem ApproximableMap
variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

/-- Neighbourhood corresponding to `X` under an order isomorphism of ideal elements. -/
noncomputable def phi (e : V₀.Element ≃o V₁.Element) {X : Set α} (hX : V₀.mem X) : Set
    β :=
  Classical.choose (exists_principal_eq_apply_principal e hX)

theorem phi_mem (e : V₀.Element ≃o V₁.Element) {X : Set α} (hX : V₀.mem X) :
    V₁.mem (phi e hX) :=
  Classical.choose (Classical.choose_spec (exists_principal_eq_apply_principal e hX))

theorem phi_spec (e : V₀.Element ≃o V₁.Element) {X : Set α} (hX : V₀.mem X) :
    e (V₀.principal hX) = V₁.principal (phi_mem e hX) :=
  Classical.choose_spec (Classical.choose_spec (exists_principal_eq_apply_principal e
    hX))

theorem phi_inter (e : V₀.Element ≃o V₁.Element) {X X' : Set α}
    (hX : V₀.mem X) (hX' : V₀.mem X') (hXX' : V₀.mem (X ∩ X')) :
    phi e hXX' = phi e hX ∩ phi e hX' := by
  have hpX_le : V₀.principal hX ≤ V₀.principal hXX' :=
    (V₀.principal_le_iff hX hXX').mpr Set.inter_subset_left
  have hpX'_le : V₀.principal hX' ≤ V₀.principal hXX' :=
    (V₀.principal_le_iff hX' hXX').mpr Set.inter_subset_right
  have hleast : ∀ w : V₀.Element, V₀.principal hX ≤ w → V₀.principal hX' ≤ w →
      V₀.principal hXX' ≤ w := by
    intro w hXw hX'w Z hZ
    obtain ⟨hZmem, hsubZ⟩ := hZ
    have hwX : w.mem X := hXw X ⟨hX, subset_rfl⟩
    have hwX' : w.mem X' := hX'w X' ⟨hX', subset_rfl⟩
    exact w.up_mem (w.inter_mem hwX hwX') hZmem hsubZ
  have hqX : e (V₀.principal hX) = V₁.principal (phi_mem e hX) := phi_spec e hX
  have hqX' : e (V₀.principal hX') = V₁.principal (phi_mem e hX') := phi_spec e hX'
  have hqXX' : e (V₀.principal hXX') = V₁.principal (phi_mem e hXX') := phi_spec e hXX'
  have hsub1 : phi e hXX' ⊆ phi e hX := by
    have h : e (V₀.principal hX) ≤ e (V₀.principal hXX') := e.monotone hpX_le
    rw [hqX, hqXX'] at h
    exact (V₁.principal_le_iff (phi_mem e hX) (phi_mem e hXX')).mp h
  have hsub1' : phi e hXX' ⊆ phi e hX' := by
    have h : e (V₀.principal hX') ≤ e (V₀.principal hXX') := e.monotone hpX'_le
    rw [hqX', hqXX'] at h
    exact (V₁.principal_le_iff (phi_mem e hX') (phi_mem e hXX')).mp h
  have hsubInter : phi e hXX' ⊆ phi e hX ∩ phi e hX' := Set.subset_inter hsub1 hsub1'
  have hinter : V₁.mem (phi e hX ∩ phi e hX') :=
    V₁.inter_mem (phi_mem e hX) (phi_mem e hX') (phi_mem e hXX') hsubInter
  have hsub2 : phi e hX ∩ phi e hX' ⊆ phi e hXX' := by
    have hqXleI : e (V₀.principal hX) ≤ V₁.principal hinter := by
      rw [hqX]; exact (V₁.principal_le_iff (phi_mem e hX) hinter).mpr Set.
    inter_subset_left
    have hqX'leI : e (V₀.principal hX') ≤ V₁.principal hinter := by
      rw [hqX']; exact (V₁.principal_le_iff (phi_mem e hX') hinter).mpr Set.
    inter_subset_right
```

```
    have hpXleI : V₀.principal hX ≤ e.symm (V₁.principal hinter) := by
      have h := e.symm.monotone hqXleI; rwa [e.symm_apply_apply] at h
    have hpX'leI : V₀.principal hX' ≤ e.symm (V₁.principal hinter) := by
      have h := e.symm.monotone hqX'leI; rwa [e.symm_apply_apply] at h
    have hpXX'leI : V₀.principal hXX' ≤ e.symm (V₁.principal hinter) :=
      hleast _ hpXleI hpX'leI
    have h : e (V₀.principal hXX') ≤ V₁.principal hinter := by
      have h := e.monotone hpXX'leI; rwa [e.apply_symm_apply] at h
    rw [hqXX'] at h
    exact (V₁.principal_le_iff (phi_mem e hXX') hinter).mp h
  exact Set.Subset.antisymm hsubInter hsub2

theorem rel_ofIso_iff (e : V₀.Element ≃o V₁.Element) {X : Set α} (hX : V₀.mem X) {Y :
    Set β} :
    (ofIso e).rel X Y ↔ V₁.mem Y ∧ phi e hX ⊆ Y := by
  constructor
  · rintro ⟨_, hmem⟩
    have h : (e (V₀.principal hX)).mem Y := hmem
    rw [phi_spec e hX] at h
    exact h
  · intro h
    refine ⟨hX, ?_⟩
    rw [phi_spec e hX]
    exact h

end

end DomainTheory.NeighbourhoodSystems
```

4.3.27 Exercise 2.14 — isomorphisms of neighbourhoods

Let $f : |D_0| \to |D_1|$ be an isomorphism between domains. Let $\varphi : D_0 \to D_1$ be the one-one correspondence between neighbourhoods provided by Theorem 2.7 where

$$f(\uparrow X) = \uparrow \varphi(X)$$

for all $X \in D_0$. Show that the approximable mapping determined by f is just the relationship $\varphi(X) \subseteq Y$. In addition prove that if $X, X' \in D_0$ are consistent, then

$$\varphi(X \cap X') = \varphi(X) \cap \varphi(X').$$

Remark that the isomorphisms between domains correspond exactly to the isomorphisms between neighbourhood systems (in the sense of one-one inclusion preserving correspondences).

Theorem (`example_214`). The neighbourhood correspondence of an order isomorphism preserves intersections of consistent neighbourhoods, and the induced approximable map relates neighbourhoods exactly by containment after that correspondence.

Proof. Intersection preservation and the containment characterisation of the induced relation. ■

`Exercise_2_14.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.phi
```

```
@[expose] public section

open DomainTheory.NeighbourhoodSystems

/-- Scott's Exercise 2.14: the neighbourhood correspondence of an order isomorphism
    preserves
intersections of consistent neighbourhoods, and the induced approximable map is
    containment
after that correspondence. -/
theorem example_214 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (e : V₀.Element ≃o V₁.Element) {X X' : Set α}
      (hX : V₀.mem X) (hX' : V₀.mem X') (hXX' : V₀.mem (X ∩ X')),
      phi e hXX' = phi e hX ∩ phi e hX') ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (e : V₀.Element ≃o V₁.Element) {X : Set α} (hX : V₀.mem X) {Y : Set β},
      (ApproximableMap.ofIso e).rel X Y ↔ V₁.mem Y ∧ phi e hX ⊆ Y) :=
  ⟨@phi_inter, @rel_ofIso_iff⟩
```

4.3.28 One-token Sierpiński neighbourhood system

The one-token Sierpiński neighbourhood system, and the correspondence between its approximable maps and open subsets of a domain.

Definition (`memO`). A subset of the one-token set is a Sierpiński neighbourhood when it is the whole set or empty.

Theorem (`nestedOrDisjoint_O`). Any two Sierpiński neighbourhoods are nested or disjoint.

Proof. Case analysis on the two alternatives. ■

Theorem (`subMaster_O`). Every Sierpiński neighbourhood is contained in the universe.

Proof. Immediate. ■

Definition (`O`). The one-token Sierpiński neighbourhood system has master the universe and neighbourhoods the universe and the empty set.

Theorem (`O_mem`). Membership in the Sierpiński system is the Sierpiński neighbourhood predicate.

Proof. By definition. ■

Theorem (`O_master`). The master of the Sierpiński system is the universe.

Proof. By definition. ■

Theorem (`O_mem_empty`). The empty set is a Sierpiński neighbourhood.

Proof. One of the two alternatives in the neighbourhood predicate. ■

Theorem (`O_inter_mem`). The intersection of two Sierpiński neighbourhoods is a Sierpiński neighbourhood.

Proof. Intersecting with the universe leaves the other factor; intersecting with empty yields empty. ■

Definition (`botElt`). The bottom ideal element of the Sierpiński system is the bottom filter.

Theorem (`mem_botElt`). A neighbourhood belongs to the bottom element if and only if it is the universe.

Proof. Specialise the characterisation of the bottom filter to the Sierpiński master. ■

Definition (`topElt`). The top ideal element of the Sierpiński system is the principal filter of the empty neighbourhood.

Definition (`mapToOpen`). An approximable map into the Sierpiński system determines the set of ideal elements sent to top.

Theorem (`mem_empty_iff_eq_top`). An ideal element of the Sierpiński system contains the empty neighbourhood if and only if it is top.

Proof. The empty set is minimal among neighbourhoods, so membership characterises the corresponding principal; the converse is immediate. ■

Theorem (`element_eq_bot_or_top`). Every ideal element of the Sierpiński system is bottom or top.

Proof. If it contains empty it is top; otherwise its neighbourhoods are forced to be those of bottom. ■

Theorem (`O_ext`). Ideal elements of the Sierpiński system that agree on membership of empty are equal.

Proof. Both are bottom or both are top by the preceding dichotomy. ■

Theorem (`isOpen_mapToOpen`). The set of points sent to top by an approximable map into the Sierpiński system is open.

Proof. It is the preimage of the basic open of the empty neighbourhood under the continuous element-wise action. ■

Theorem (`univ_ne_empty`). The universe of the one-token set is nonempty.

Proof. It contains the unique token. ■

Definition (`openToMap`). An open set of ideal elements induces an approximable map into the Sierpiński system: a neighbourhood relates to a Sierpiński neighbourhood when the latter is empty only if the principal of the former lies in the open.

Proof. Domain, codomain, and master are immediate; intersection and monotonicity use the filter and upper-set properties of opens. ■

Theorem (`toElementMap_openToMap_principal_mem_empty`). Under the induced map, the image of a principal contains empty if and only if that principal lies in the open.

Proof. Unpack the defining relation at the principal. ■

Theorem (`mem_mapToOpen_openToMap`). Points belonging to the open recovered from the map induced by an open are exactly the points of that open.

Proof. One direction uses the principal-membership law and that opens are upper sets; the converse extracts a basic open neighbourhood from the open-filter property. ■

Theorem (`openToMap_mapToOpen`). The map induced by the open recovered from an approximable map into the Sierpiński system recovers that map.

Proof. Compare on principals via the empty-membership characterisation and uniqueness of Sierpiński elements by empty-membership. ■

`O.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofContinuous
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    eq_principal_of_isMin
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff
```

```
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.O
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

def memO (X : Set (Fin 1)) : Prop := X = Set.univ ∨ X = ∅

theorem nestedOrDisjoint_O : NestedOrDisjoint memO := by
  rintro X Y (rfl | rfl) (rfl | rfl)
  · exact Or.inl subset_rfl
  · exact Or.inr (Or.inl (Set.empty_subset _))
  · exact Or.inl (Set.empty_subset _)
  · exact Or.inl subset_rfl

theorem subMaster_O : ∀ {X : Set (Fin 1)}, memO X → X ⊆ Set.univ := fun _ => Set.
    subset_univ _

/-- One-token Sierpinski neighbourhood system: master and empty. -/
def system : NeighborhoodSystem (Fin 1) :=
  NeighborhoodSystem.ofNestedOrDisjoint memO Set.univ Set.univ_nonempty
    (Or.inl rfl) nestedOrDisjoint_O subMaster_O

@[simp] theorem O_mem {X : Set (Fin 1)} : system.mem X ↔ memO X := Iff.rfl

@[simp] theorem O_master : system.master = (Set.univ : Set (Fin 1)) := rfl

theorem O_mem_empty : system.mem (∅ : Set (Fin 1)) := Or.inr rfl

theorem O_inter_mem {Y Y' : Set (Fin 1)} (hY : system.mem Y) (hY' : system.mem Y') :
    system.mem (Y ∩ Y') := by
  rcases hY with rfl | rfl
  · rw [Set.univ_inter]; exact hY'
  · rw [Set.empty_inter]; exact O_mem_empty

def botElt : system.Element := system.bot

theorem mem_botElt {Y : Set (Fin 1)} : botElt.mem Y ↔ Y = (Set.univ : Set (Fin 1)) :=
    by
  rw [botElt, system.mem_bot, O_master]

def topElt : system.Element := system.principal O_mem_empty

section
variable {? : Type*} (V : NeighborhoodSystem ?)

/-- Open set classified by an approximable map into the Sierpinski system. -/
def mapToOpen (f : ApproximableMap V system) : Set V.Element := {x | f.toElementMap x
    = topElt}
end

theorem mem_empty_iff_eq_top (z : system.Element) : z.mem ∅ ↔ z = topElt := by
  constructor
```

```
  · intro hz
    exact system.eq_principal_of_isMin z hz (fun Y _ => Set.empty_subset Y)
  · rintro rfl
    exact ⟨O_mem_empty, subset_rfl⟩

theorem element_eq_bot_or_top (z : system.Element) : z = botElt ∨ z = topElt := by
  by_cases hz : z.mem ∅
  · exact Or.inr ((mem_empty_iff_eq_top z).mp hz)
  · refine Or.inl ?_
    apply Element.ext
    intro Y
    constructor
    · intro hY
      rcases (O_mem.mp (z.sub hY)) with rfl | rfl
      · exact mem_botElt.mpr rfl
      · exact absurd hY hz
    · intro hY
      rw [mem_botElt.mp hY, ← O_master]
      exact z.master_mem

theorem O_ext {z w : system.Element} (h : z.mem ∅ ↔ w.mem ∅) : z = w := by
  by_cases hz : z.mem ∅
  · rw [(mem_empty_iff_eq_top z).mp hz, (mem_empty_iff_eq_top w).mp (h.mp hz)]
  · have ez : z = botElt :=
      (element_eq_bot_or_top z).resolve_right (fun c => hz ((mem_empty_iff_eq_top z).
    mpr c))
    have ew : w = botElt :=
      (element_eq_bot_or_top w).resolve_right
        (fun c => (fun c' => hz (h.mpr c')) ((mem_empty_iff_eq_top w).mpr c))
    rw [ez, ew]

section
variable {? : Type*} (V : NeighborhoodSystem ?)

theorem isOpen_mapToOpen (f : ApproximableMap V system) : IsOpen (mapToOpen V f) := by
  have heq : mapToOpen V f = (fun x => f.toElementMap x) ^-1' system.basicOpen ∅ := by
    ext x
    simp only [mapToOpen, Set.mem_ofPred_eq, Set.mem_preimage, mem_basicOpen]
    exact (mem_empty_iff_eq_top (f.toElementMap x)).symm
  rw [heq]
  exact (system.isOpen_basicOpen ∅).preimage f.continuous_toElementMap
end

theorem univ_ne_empty : (Set.univ : Set (Fin 1)) = ∅ := by
  intro h
  have h0 : (0 : Fin 1) ∈ (∅ : Set (Fin 1)) := h ▶ Set.mem_univ 0
  simp at h0

section
variable {? : Type*} (V : NeighborhoodSystem ?)

/-- Approximable map into the Sierpinski system classifying an open set. -/
def openToMap (U : Set V.Element) (hU : IsOpen U) : ApproximableMap V system where
  rel X Y := V.mem X ∧ system.mem Y ∧ (Y = ∅ → ∃ hX : V.mem X, V.principal hX ∈ U)
  rel_dom h := h.1
  rel_cod h := h.2.1
```

```
  master_rel := ⟨V.master_mem, system.master_mem, fun heq => (univ_ne_empty (O_master ▶
     heq)).elim⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY, fY⟩ ⟨_, hY', fY'⟩
    refine ⟨hX, O_inter_mem hY hY', ?_⟩
    intro h
    rcases O_mem.mp hY with rfl | rfl
    · rw [Set.univ_inter] at h; exact fY' h
    · exact fY rfl
  mono := by
    rintro X X' Y Y' ⟨hX, _, fY⟩ hX'X hYY' hX' hY'
    refine ⟨hX', hY', ?_⟩
    intro hY'eq
    have hYeq : Y = ∅ := Set.subset_empty_iff.mp (hY'eq ▶ hYY')
    obtain ⟨hX0, hmem⟩ := fY hYeq
    have hmem' : V.principal hX ∈ U := hmem
    have hle : V.principal hX ≤ V.principal hX' := (V.principal_le_iff hX hX').mpr hX'
    X
    exact ⟨hX', V.isOpen_isUpperSet hU hmem' hle⟩

theorem toElementMap_openToMap_principal_mem_empty (U : Set V.Element) (hU : IsOpen U)
    {X : Set ?} (hX : V.mem X) :
    ((openToMap V U hU).toElementMap (V.principal hX)).mem ∅ ↔ V.principal hX ∈ U :=
   by
  rw [← (openToMap V U hU).rel_iff_mem_principal hX]
  constructor
  · rintro ⟨_, _, fimpl⟩
    obtain ⟨_, hmem⟩ := fimpl rfl
    exact hmem
  · intro h
    exact ⟨hX, O_mem_empty, fun _ => ⟨hX, h⟩⟩

theorem mem_mapToOpen_openToMap (U : Set V.Element) (hU : IsOpen U) (x : V.Element) :
    x ∈ mapToOpen V (openToMap V U hU) ↔ x ∈ U := by
  rw [mapToOpen, Set.mem_ofPred_eq, ← mem_empty_iff_eq_top,
    ApproximableMap.toElementMap_mem_iff_principal]
  constructor
  · rintro ⟨X, hxX, hmem⟩
    rw [toElementMap_openToMap_principal_mem_empty] at hmem
    have hple : V.principal (x.sub hxX) ≤ x := fun Z hZ => x.up_mem hxX hZ.1 hZ.2
    exact V.isOpen_isUpperSet hU hmem hple
  · intro hxU
    obtain ⟨X, hxX, hXU⟩ := hU.out x hxU
    refine ⟨X, hxX, ?_⟩
    rw [toElementMap_openToMap_principal_mem_empty]
    exact hXU (show V.principal (x.sub hxX) ∈ V.basicOpen X from ⟨x.sub hxX, subset_rfl
    ⟩)

theorem openToMap_mapToOpen (f : ApproximableMap V system) :
    openToMap V (mapToOpen V f) (isOpen_mapToOpen V f) = f := by
  apply ApproximableMap.eq_of_toElementMap_principal
  intro X hX
  apply O_ext
  rw [toElementMap_openToMap_principal_mem_empty, mapToOpen, Set.mem_ofPred_eq,
    ← mem_empty_iff_eq_top]
end
```

```
end DomainTheory.NeighbourhoodSystems.O
```

4.3.29 Exercise 2.15 — opens classified by maps

(For topologists.) Consider the one-token system with

$$\mathcal{O} = \{\{0\}, \emptyset\}.$$

We can regard $|\mathcal{O}|$ as having just two finite elements $\bot$ (bottom) and $\top$ (top), where $\bot \sqsubseteq \top$. For any system D, show that the open subsets U of $|D|$ are in a one-one correspondence with the approximable mappings $f : D \to \mathcal{O}$, where the correspondence is given by the equation

$$U = \{x \in |D| \mid f(x) = \top\}.$$

What are the open subsets of $|\mathcal{O}|$? of $|\mathbb{T}|$? of $|\mathbb{B}|$?

Theorem (`example_215`). Opens of a domain are recovered as the points sent to top by the approximable map they induce into the Sierpiński system, and every approximable map into that system arises from the open it classifies.

Proof. Round-trip membership for the open recovered from an induced map, and recovery of a map from the open it classifies. ■

`Exercise_2_15.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.O

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.O

/-- Scott's Exercise 2.15: opens of a domain are classified by approximable maps into
    the
one-token Sierpinski system. -/
theorem example_215 :
    (∀ {? : Type*} (V : NeighborhoodSystem ?) (U : Set V.Element) (hU : IsOpen U)
      (x : V.Element), x ∈ mapToOpen V (openToMap V U hU) ↔ x ∈ U) ∧
    (∀ {? : Type*} (V : NeighborhoodSystem ?) (f : ApproximableMap V system),
      openToMap V (mapToOpen V f) (isOpen_mapToOpen V f) = f) :=
  ⟨@mem_mapToOpen_openToMap, @openToMap_mapToOpen⟩
```

4.3.30 Prefixing map and parity equations

The approximable map that prefixes by a fixed string, and uniqueness of the parity map among solutions of its three equations.

Definition (`SatisfiesParityEquations`). An approximable map from binary trees to the two-token system satisfies the parity equations when it sends a one-prefixed element to true, a zero-one-prefixed element to false, and a zero-zero-prefixed element to the image of the unprefixed element.

Definition (`sigmaMap`). Prefixing neighbourhoods by a fixed string determines an approximable endomap of the binary-tree system: X relates to Y when the prefix of X is contained in Y.

Proof. Domain and codomain membership are immediate; master uses the two masters; intersection and monotonicity follow from the corresponding properties of prefixed neighbourhoods. ■

Theorem (`key`). A map satisfying the parity equations agrees on bottoms of cones with the scan-value of the parity map.

Proof. Induction on the string, using the three equations and comparison of bottoms under prefixing. ■

Theorem (`eq_parityMap_of_satisfies`). Any approximable map satisfying the parity equations equals the parity map.

Proof. Compare neighbourhood relations via evaluation at principal filters and the preceding key lemma. ■

Theorem (`parityMap_satisfies`). The parity map satisfies the parity equations.

Proof. Unpack the elementwise formula for the parity map on prefixed elements against the scan-append rules. ■

Theorem (`toElementMap_sigmaMap`). The elementwise action of the prefixing map is prefixing of ideal elements.

Proof. Membership rearranges between the defining relation and the definition of prefixed elements. ■

`sigmaMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Truth
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    sigmaBot_le_iff

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.SigmaMap
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem BitDom

structure SatisfiesParityEquations (g : ApproximableMap B Truth.T) : Prop where
  one : ∀ x : B.Element, g.toElementMap (sigmaElt [true] x) = Truth.trueElt
  zeroOne : ∀ x : B.Element, g.toElementMap (sigmaElt [false, true] x) = Truth.falseElt
  zeroZero : ∀ x : B.Element, g.toElementMap (sigmaElt [false, false] x) = g.
    toElementMap x

/-- Approximable map that prefixes neighbourhoods by a fixed string. -/
def sigmaMap (σ : Str) : ApproximableMap B B where
  rel X Y := B.mem X ∧ B.mem Y ∧ prepend σ X ⊆ Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨B.master_mem, B.master_mem, Set.subset_univ _⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY, hsub⟩ ⟨_, hY', hsub'⟩
    have hpre : B.mem (prepend σ X) := memB_prepend σ hX
    have hsubInter : prepend σ X ⊆ Y ∩ Y' := Set.subset_inter hsub hsub'
    exact ⟨hX, B.inter_mem hY hY' hpre hsubInter, hsubInter⟩
  mono := by
    rintro X X' Y Y' ⟨_, _, hsub⟩ hX'X hYY' hX' hY'
    exact ⟨hX', hY', ((prepend_mono σ hX'X).trans hsub).trans hYY'⟩
```

```
theorem key (g : ApproximableMap B Truth.T) (hg : SatisfiesParityEquations g) :
    ∀ σ : Str, g.toElementMap (sigmaElt σ B.bot) = Truth.valElt (Truth.scan σ)
  | [] => by
      rw [sigmaElt_nil]
      have h1 : g.toElementMap B.bot ≤ Truth.trueElt := by
        have hm := g.toElementMap_mono (B.bot_le (sigmaElt [true] B.bot))
        rwa [hg.one] at hm
      have h2 : g.toElementMap B.bot ≤ Truth.falseElt := by
        have hm := g.toElementMap_mono (B.bot_le (sigmaElt [false, true] B.bot))
        rwa [hg.zeroOne] at hm
      exact Truth.eq_botElt_of_le h1 h2
  | [false] => by
      have hbase : g.toElementMap B.bot = Truth.botElt := by
        have h0 := key g hg []
        rwa [sigmaElt_nil] at h0
      have hb : sigmaElt [false] B.bot ≤ sigmaElt [false, false] B.bot := by
        rw [sigmaElt_bot, sigmaElt_bot]
        exact (sigmaBot_le_iff _ _).mpr ⟨[false], rfl⟩
      have hm : g.toElementMap (sigmaElt [false] B.bot) ≤ Truth.botElt := by
        have := g.toElementMap_mono hb
        rwa [hg.zeroZero, hbase] at this
      have hm2 : Truth.botElt ≤ g.toElementMap (sigmaElt [false] B.bot) :=
        Truth.botElt_le _
      exact (le_antisymm hm hm2).trans rfl
  | (true :: t) => by
      rw [show (true :: t : Str) = [true] ++ t from rfl, sigmaElt_append, hg.one,
        Truth.scan_append_true]
      rfl
  | (false :: true :: t) => by
      rw [show (false :: true :: t : Str) = [false, true] ++ t from rfl,
    sigmaElt_append,
        hg.zeroOne, Truth.scan_append_falseTrue]
      rfl
  | (false :: false :: t) => by
      rw [show (false :: false :: t : Str) = [false, false] ++ t from rfl,
    sigmaElt_append,
        hg.zeroZero, key g hg t, Truth.scan_append_falseFalse]

theorem eq_parityMap_of_satisfies {g : ApproximableMap B Truth.T}
    (hg : SatisfiesParityEquations g) : g = Truth.parityMap := by
  apply ApproximableMap.ext
  intro X Y
  by_cases hX : B.mem X
  · obtain ⟨σ, rfl⟩ := hX
    rw [g.rel_iff_mem_principal (memB_cone σ)]
    show (g.toElementMap (sigmaBot σ)).mem Y ↔ Truth.parityMap.rel (cone σ) Y
    rw [← sigmaElt_bot, key g hg σ, Truth.parityMap_rel_cone]
  · exact ⟨fun hr => absurd (g.rel_dom hr) hX,
      fun hr => absurd (Truth.parityMap.rel_dom hr) hX⟩

theorem parityMap_satisfies : SatisfiesParityEquations Truth.parityMap where
  one x := by
    apply Element.ext
    intro Y
    rw [Truth.parityMap_toElementMap_sigmaElt]
    constructor
```

```
    · rintro ⟨τ, _, hval⟩
      rwa [Truth.scan_append_true] at hval
    · intro hval
      exact ⟨[], by rw [cone.nil]; exact x.master_mem, by rwa [Truth.scan_append_true]⟩
  zeroOne x := by
    apply Element.ext
    intro Y
    rw [Truth.parityMap_toElementMap_sigmaElt]
    constructor
    · rintro ⟨τ, _, hval⟩
      rwa [Truth.scan_append_falseTrue] at hval
    · intro hval
      exact ⟨[], by rw [cone.nil]; exact x.master_mem, by rwa [Truth.
    scan_append_falseTrue]⟩
  zeroZero x := by
    apply Element.ext
    intro Y
    rw [Truth.parityMap_toElementMap_sigmaElt, Truth.parityMap_toElementMap_mem]
    constructor
    · rintro ⟨τ, hτ, hval⟩
      exact ⟨τ, hτ, by rwa [Truth.scan_append_falseFalse] at hval⟩
    · rintro ⟨τ, hτ, hval⟩
      exact ⟨τ, hτ, by rwa [Truth.scan_append_falseFalse]⟩

theorem toElementMap_sigmaMap (σ : Str) (x : B.Element) :
    (sigmaMap σ).toElementMap x = sigmaElt σ x := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨X, hxX, _, hY, hsub⟩
    exact ⟨hY, X, hxX, hsub⟩
  · rintro ⟨hY, X, hxX, hsub⟩
    exact ⟨X, hxX, x.sub hxX, hY, hsub⟩

end DomainTheory.NeighbourhoodSystems.SigmaMap
```

4.3.31 Exercise 2.16 — prefixing is approximable

In the discussion of $\mathbb{B}$ in Chapter 1 we defined a mapping $x \mapsto \sigma x$ for any given $\sigma \in \Sigma^*$. Is this (elementwise) mapping approximable? Show in addition that the mapping $f : \mathbb{B} \to \mathbb{T}$ of 2.3 is uniquely determined among approximable mappings by the equations:

$$f(1x) = \text{true},$$
$$f(01x) = \text{false, and}$$
$$f(00x) = f(x).$$

Theorem (`example_216`). The parity map is the unique approximable map satisfying the three parity equations, and prefixing by a fixed string is the elementwise action of an approximable endomap.

Proof. Uniqueness among solutions of the parity equations, and recovery of prefixed ideal elements from the prefixing map. ■

`Exercise_2_16.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.sigmaMap

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.SigmaMap
/-- Scott's Exercise 2.16: the parity map is uniquely determined by its three
    equations, and
prefixing by a fixed string is the elementwise action of an approximable map. -/
theorem example_216 :
    (∀ {g : ApproximableMap BitDom.B Truth.T},
      SatisfiesParityEquations g → g = Truth.parityMap) ∧
    (∀ (σ : BitDom.Str) (x : BitDom.B.Element),
      (sigmaMap σ).toElementMap x = BitDom.sigmaElt σ x) :=
  ⟨@eq_parityMap_of_satisfies, toElementMap_sigmaMap⟩
```

4.3.32 Uniqueness of the erase-ones map

Uniqueness of the erase-ones approximable map among solutions of the run equations, and verification that the erase-ones map satisfies those equations.

Theorem (`eq_runMap_of_satisfies`). Any approximable endomap of the binary-tree system that satisfies the run equations equals the erase-ones map.

Proof. Compare neighbourhood relations via evaluation at principal filters and the key lemma relating solutions of the equations to the erase-ones string map. ■

Theorem (`runMap_toElementMap_mem`). Membership in the elementwise image of the erase-ones map is witnessed by a cone whose erased output is contained in the neighbourhood.

Proof. Unpack the defining relation on cones. ■

Theorem (`runMap_toElementMap_sigmaElt`). Membership in the image of a prefixed ideal element under the erase-ones map is membership after erasing ones from the concatenated string.

Proof. Transport the cone formula through prefixing of neighbourhoods and monotonicity of the erase-ones string map. ■

Theorem (`sigmaElt_runMap_toElementMap`). Membership after prefixing the erase-ones image is membership for a cone whose output is prefixed by the fixed string.

Proof. Reassociate prefixing of neighbourhoods with the elementwise membership formula. ■

Theorem (`runMap_satisfies`). The erase-ones map satisfies the run equations.

Proof. Compare the prefixed membership formulae against the string identities for leading zero, two ones, one-zero, and a lone one on bottom. ■

`eq_runMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.RunMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.RunMap
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem BitDom
```

```
theorem eq_runMap_of_satisfies {g : ApproximableMap B B} (hg : SatisfiesRunEquations g
    ) :
    g = runMap := by
  apply ApproximableMap.ext
  intro X Y
  by_cases hX : B.mem X
  · obtain ⟨σ, rfl⟩ := hX
    rw [g.rel_iff_mem_principal (memB_cone σ)]
    show (g.toElementMap (sigmaBot σ)).mem Y ↔ runMap.rel (cone σ) Y
    rw [← sigmaElt_bot, key g hg σ, sigmaElt_bot, runMap_rel_cone]
    show (B.principal (memB_cone (out σ))).mem Y ↔ B.mem Y ∧ cone (out σ) ⊆ Y
    exact B.mem_principal (memB_cone (out σ))
  · exact ⟨fun hr => absurd (g.rel_dom hr) hX, fun hr => absurd (runMap.rel_dom hr) hX⟩

theorem runMap_toElementMap_mem (x : B.Element) (Y : Set Str) :
    (runMap.toElementMap x).mem Y ↔ ∃ τ, x.mem (cone τ) ∧ B.mem Y ∧ cone (out τ) ⊆ Y
    := by
  constructor
  · rintro ⟨X, hX, hrel⟩
    obtain ⟨τ, rfl⟩ := x.sub hX
    exact ⟨τ, hX, (runMap_rel_cone τ Y).mp hrel⟩
  · rintro ⟨τ, hτ, hYmem, hsub⟩
    exact ⟨cone τ, hτ, (runMap_rel_cone τ Y).mpr ⟨hYmem, hsub⟩⟩

theorem runMap_toElementMap_sigmaElt (σ : Str) (x : B.Element) (Y : Set Str) :
    (runMap.toElementMap (sigmaElt σ x)).mem Y ↔
      ∃ τ, x.mem (cone τ) ∧ B.mem Y ∧ cone (out (σ ++ τ)) ⊆ Y := by
  constructor
  · rintro ⟨Z, ⟨_, X, hX, hsub⟩, ?, rfl, hYmem, hsubOut⟩
    obtain ⟨τ, rfl⟩ := x.sub hX
    rw [prepend_cone] at hsub
    exact ⟨τ, hX, hYmem, (cone.subset_cone.mpr (out_mono (cone.subset_cone.mp hsub))).
    trans hsubOut⟩
  · rintro ⟨τ, hτ, hYmem, hsub⟩
    exact ⟨cone (σ ++ τ), ⟨memB_cone _, cone τ, hτ, (prepend_cone σ τ).le⟩,
      (runMap_rel_cone (σ ++ τ) Y).mpr ⟨hYmem, hsub⟩⟩

theorem sigmaElt_runMap_toElementMap (τ₀ : Str) (x : B.Element) (Y : Set Str) :
    (sigmaElt τ₀ (runMap.toElementMap x)).mem Y ↔
      ∃ τ, x.mem (cone τ) ∧ B.mem Y ∧ cone (τ₀ ++ out τ) ⊆ Y := by
  constructor
  · rintro ⟨hYmem, Z, hZ, hsub⟩
    obtain ⟨τ, hτ, _, hZsub⟩ := (runMap_toElementMap_mem x Z).mp hZ
    refine ⟨τ, hτ, hYmem, ?_⟩
    rw [← prepend_cone]
    exact (prepend_mono τ₀ hZsub).trans hsub
  · rintro ⟨τ, hτ, hYmem, hsub⟩
    refine ⟨hYmem, cone (out τ),
      (runMap_toElementMap_mem x _).mpr ⟨τ, hτ, memB_cone _, subset_rfl⟩, ?_⟩
    rw [prepend_cone]
    exact hsub

theorem runMap_satisfies : SatisfiesRunEquations runMap where
  zero x := by
    apply Element.ext
```

```
    intro Y
    rw [runMap_toElementMap_sigmaElt, sigmaElt_runMap_toElementMap]
    constructor
    · rintro ⟨τ, hτ, hYmem, hsub⟩
      exact ⟨τ, hτ, hYmem, by rwa [out_append_false] at hsub⟩
    · rintro ⟨τ, hτ, hYmem, hsub⟩
      exact ⟨τ, hτ, hYmem, by rwa [out_append_false]⟩
  oneOne x := by
    apply Element.ext
    intro Y
    rw [runMap_toElementMap_sigmaElt, runMap_toElementMap_sigmaElt]
    constructor
    · rintro ⟨τ, hτ, hYmem, hsub⟩
      exact ⟨τ, hτ, hYmem, by rwa [out_append_trueTrue] at hsub⟩
    · rintro ⟨τ, hτ, hYmem, hsub⟩
      exact ⟨τ, hτ, hYmem, by rwa [← out_append_trueTrue] at hsub⟩
  oneZero x := by
    apply Element.ext
    intro Y
    rw [runMap_toElementMap_sigmaElt]
    constructor
    · rintro ⟨τ, hτ, hYmem, hsub⟩
      rw [out_append_trueFalse] at hsub
      exact ⟨hYmem, cone τ, hτ, by rw [prepend_cone]; exact hsub⟩
    · rintro ⟨hYmem, X, hX, hsub⟩
      obtain ⟨τ, rfl⟩ := x.sub hX
      rw [prepend_cone] at hsub
      exact ⟨τ, hX, hYmem, by rwa [out_append_trueFalse]⟩
  one := by
    apply Element.ext
    intro Y
    rw [runMap_toElementMap_sigmaElt]
    constructor
    · rintro ⟨τ, hτ, hYmem, hsub⟩
      obtain rfl : τ = [] := bot_mem_cone_iff.mp hτ
      rw [show out ([true] ++ ([] : Str)) = ([] : Str) from rfl, cone.nil] at hsub
      rw [mem_bot]
      exact Set.eq_univ_of_univ_subset hsub
    · intro hY
      rw [mem_bot] at hY
      refine ⟨[], bot_mem_cone_iff.mpr rfl, hY ▸ B.master_mem, ?_⟩
      rw [show out ([true] ++ ([] : Str)) = ([] : Str) from rfl, cone.nil, hY,
    B_master]

end DomainTheory.NeighbourhoodSystems.RunMap
```

4.3.33 Exercise 2.17 — erase-ones map

Establish in detail that the mapping $g : \mathbb{B} \to \mathbb{B}$ of Exercise 2.4 is approximable. Is it uniquely determined by these equations:

$$g(0x) = 0g(x),$$
$$g(11x) = g(1x),$$

$$g(10x) = 0x,$$
$$g(1) = \bot$$

or are some missing?

Theorem (`exercise_2_17`). The erase-ones map is the unique approximable endomap of the binary-tree system satisfying the run equations, and those equations hold of the erase-ones map.

Proof. Uniqueness among solutions of the run equations, and verification that the erase-ones map satisfies them. Approximability is the construction of the erase-ones map. ■

Exercise_2_17.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.eq_runMap

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open RunMap

/-- Scott's Exercise 2.17: the erase-ones map is uniquely determined by the run
    equations, and
those equations hold of the erase-ones map. -/
theorem exercise_2_17 :
    (∀ {g : ApproximableMap BitDom.B BitDom.B},
      SatisfiesRunEquations g → g = runMap) ∧
    SatisfiesRunEquations runMap :=
  ⟨@eq_runMap_of_satisfies, runMap_satisfies⟩
```

4.3.34 Insert-zero map and left inverse

The insert-zero approximable map on binary trees, its left inverse that drops alternate bits, and the failure of surjectivity and injectivity for those maps.

Definition (`hOut`). Insert a zero after each bit of a string.

Theorem (`hOut_nil`). Inserting zeros into the empty string yields empty.

Theorem (`hOut_cons`). Inserting zeros after a leading bit yields that bit, a zero, and the recursively expanded tail.

Theorem (`hOut_append`). Inserting zeros is monotone for string prefix.

Proof. Induction on the prefix. ■

Theorem (`hOut_mono`). Prefix-comparable strings remain so after inserting zeros.

Proof. Unpack the prefix witness and apply the append form. ■

Definition (`kOut`). Drop every other bit of a string, truncating a lone leftover bit to empty.

Theorem (`kOut_nil`). Dropping alternate bits of empty yields empty.

Theorem (`kOut_single`). A singleton string drops to empty.

Theorem (`kOut_cons`). Dropping alternate bits after two leading bits keeps the first and continues on the remainder.

Definition (`hMap`). The approximable endomap of the binary-tree system that relates a cone on σ to neighbourhoods containing the cone on the zero-inserted string.

Proof. Master, intersection, and monotonicity follow from injectivity of cones and monotonicity of the insert-zero string map. ■

Theorem (`kOut_append`). Dropping alternate bits is monotone for string prefix.

Proof. Induction on the shape of the prefix. ■

Theorem (`kOut_mono`). Prefix-comparable strings remain so after dropping alternate bits.

Proof. Unpack the prefix witness and apply the append form. ■

Definition (`kMap`). The approximable endomap that relates a cone on σ to neighbourhoods containing the cone on the alternate-bit drop of σ.

Proof. As for the insert-zero map, using monotonicity of the drop string map. ■

Theorem (`kOut_hOut`). Dropping alternate bits after inserting zeros recovers the original string.

Proof. Induction on the string. ■

Theorem (`kMap_comp_hMap`). Composition of the drop map after the insert-zero map is the identity approximable map.

Proof. Compare neighbourhood relations using the string identity and monotonicity of the drop map. ■

Theorem (`toElementMap_hMap_cone`). The elementwise action of the insert-zero map on a principal cone is the principal cone of the zero-inserted string.

Proof. Unpack the defining relation at the cone. ■

Theorem (`toElementMap_kMap_cone`). The elementwise action of the drop map on a principal cone is the principal cone of the alternate-bit drop.

Proof. Unpack the defining relation at the cone. ■

Theorem (`hMap_not_surjective`). The elementwise action of the insert-zero map is not surjective.

Proof. A left inverse would force an impossible string identity on a two-bit cone in the image. ■

Theorem (`kMap_not_injective`). The elementwise action of the drop map is not injective.

Proof. Distinct one-bit principals collapse under the drop map. ■

`hMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.idMap
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ExampleB
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_injective

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.HMap
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem BitDom ApproximableMap

def hOut : Str → Str
  | [] => []
  | b :: t => b :: false :: hOut t

@[simp] theorem hOut_nil : hOut [] = [] := rfl

@[simp] theorem hOut_cons (b : Bool) (t : Str) : hOut (b :: t) = b :: false :: hOut t
    := rfl

theorem hOut_append (σ t : Str) : hOut σ <+: hOut (σ ++ t) := by
```

```
  induction σ with
  | nil => simp
  | cons b s ih =>
    simp only [List.cons_append, hOut_cons]
    obtain ⟨u, hu⟩ := ih
    exact ⟨u, by rw [List.cons_append, List.cons_append, hu]⟩

theorem hOut_mono {σ σ' : Str} (h : σ <+: σ') : hOut σ <+: hOut σ' := by
  obtain ⟨t, rfl⟩ := h
  exact hOut_append σ t

def kOut : Str → Str
  | [] => []
  | [_] => []
  | b :: _ :: t => b :: kOut t

@[simp] theorem kOut_nil : kOut [] = [] := rfl

@[simp] theorem kOut_single (b : Bool) : kOut [b] = [] := rfl

@[simp] theorem kOut_cons (b c : Bool) (t : Str) : kOut (b :: c :: t) = b :: kOut t :=
    rfl

/-- Approximable map inserting a zero after each bit. -/
def hMap : ApproximableMap B B where
  rel X Y := ∃ σ, X = cone σ ∧ B.mem Y ∧ cone (hOut σ) ⊆ Y
  rel_dom := fun ⟨σ, hX, _, _⟩ => ⟨σ, hX⟩
  rel_cod := fun ⟨_, _, hYmem, _⟩ => hYmem
  master_rel := by
    refine ⟨[], (B_master).trans cone.nil.symm, B.master_mem, ?_⟩
    rw [hOut_nil, B_master, cone.nil]
  inter_right := by
    rintro X Y Y' ⟨σ, hX, hYmem, hYsub⟩ ⟨σ', hX', hY'mem, hY'sub⟩
    have hσ : σ = σ' := cone.injective (hX ▶ hX')
    subst hσ
    have hsub : cone (hOut σ) ⊆ Y ∩ Y' := Set.subset_inter hYsub hY'sub
    exact ⟨σ, hX, B.inter_mem hYmem hY'mem (memB_cone (hOut σ)) hsub, hsub⟩
  mono := by
    rintro X X' Y Y' ⟨σ, hX, _, hYsub⟩ hX'X hYY' hX'mem hY'mem
    obtain ⟨σ', hX'cone⟩ := hX'mem
    have hpre : σ <+: σ' := by
      apply cone.subset_cone.mp; rw [← hX'cone, ← hX]; exact hX'X
    have hcone : cone (hOut σ') ⊆ cone (hOut σ) := cone.subset_cone.mpr (hOut_mono
    hpre)
    exact ⟨σ', hX'cone, hY'mem, (hcone.trans hYsub).trans hYY'⟩

theorem kOut_append (σ t : Str) : kOut σ <+: kOut (σ ++ t) := by
  induction σ using kOut.induct with
  | case1 => simp
  | case2 _ => simp
  | case3 b c t' ih =>
    simp only [List.cons_append, kOut_cons]
    obtain ⟨u, hu⟩ := ih
    exact ⟨u, by rw [List.cons_append, hu]⟩

theorem kOut_mono {σ σ' : Str} (h : σ <+: σ') : kOut σ <+: kOut σ' := by
```

```
  obtain ⟨t, rfl⟩ := h
  exact kOut_append σ t

/-- Left inverse (on the image of `hMap`) that drops every other bit. -/
def kMap : ApproximableMap B B where
  rel X Y := ∃ σ, X = cone σ ∧ B.mem Y ∧ cone (kOut σ) ⊆ Y
  rel_dom := fun ⟨σ, hX, _, _⟩ => ⟨σ, hX⟩
  rel_cod := fun ⟨_, _, hYmem, _⟩ => hYmem
  master_rel := by
    refine ⟨[], (B_master).trans cone.nil.symm, B.master_mem, ?_⟩
    rw [kOut_nil, B_master, cone.nil]
  inter_right := by
    rintro X Y Y' ⟨σ, hX, hYmem, hYsub⟩ ⟨σ', hX', hY'mem, hY'sub⟩
    have hσ : σ = σ' := cone.injective (hX ▶ hX')
    subst hσ
    have hsub : cone (kOut σ) ⊆ Y ∩ Y' := Set.subset_inter hYsub hY'sub
    exact ⟨σ, hX, B.inter_mem hYmem hY'mem (memB_cone (kOut σ)) hsub, hsub⟩
  mono := by
    rintro X X' Y Y' ⟨σ, hX, _, hYsub⟩ hX'X hYY' hX'mem hY'mem
    obtain ⟨σ', hX'cone⟩ := hX'mem
    have hpre : σ <+: σ' := by
      apply cone.subset_cone.mp; rw [← hX'cone, ← hX]; exact hX'X
    have hcone : cone (kOut σ') ⊆ cone (kOut σ) := cone.subset_cone.mpr (kOut_mono
    hpre)
    exact ⟨σ', hX'cone, hY'mem, (hcone.trans hYsub).trans hYY'⟩

theorem kOut_hOut (σ : Str) : kOut (hOut σ) = σ := by
  induction σ with
  | nil => rfl
  | cons b t ih => simp only [hOut_cons, kOut_cons, ih]

theorem kMap_comp_hMap : kMap.comp hMap = idMap B := by
  apply ApproximableMap.ext
  intro X Z
  rw [comp_rel, idMap_rel]
  constructor
  · rintro ⟨Y, ⟨σ, hXcone, _, hYsub⟩, τ, hYcone, hZmem, hZsub⟩
    refine ⟨⟨σ, hXcone⟩, hZmem, ?_⟩
    have hτ : τ <+: hOut σ := cone.subset_cone.mp (hYcone ▶ hYsub)
    have hτσ : kOut τ <+: σ := by have := kOut_mono hτ; rwa [kOut_hOut] at this
    calc X = cone σ := hXcone
      _ ⊆ cone (kOut τ) := cone.subset_cone.mpr hτσ
      _ ⊆ Z := hZsub
  · rintro ⟨⟨σ, hXcone⟩, hZmem, hXZ⟩
    refine ⟨cone (hOut σ), ⟨σ, hXcone, memB_cone (hOut σ), subset_rfl⟩, hOut σ, rfl,
    hZmem, ?_⟩
    rw [kOut_hOut, hXcone] at *
    exact hXZ

theorem toElementMap_hMap_cone (σ : Str) :
    hMap.toElementMap (B.principal (memB_cone σ)) = B.principal (memB_cone (hOut σ))
    := by
  apply Element.ext
  intro Y
  rw [← hMap.rel_iff_mem_principal (memB_cone σ), mem_principal]
  constructor
```

```
  · rintro ⟨?, hcone, hYmem, hsub⟩
    have : σ = ? := cone.injective hcone
    subst this
    exact ⟨hYmem, hsub⟩
  · rintro ⟨hYmem, hsub⟩
    exact ⟨σ, rfl, hYmem, hsub⟩

theorem toElementMap_kMap_cone (σ : Str) :
    kMap.toElementMap (B.principal (memB_cone σ)) = B.principal (memB_cone (kOut σ))
    := by
  apply Element.ext
  intro Y
  rw [← kMap.rel_iff_mem_principal (memB_cone σ), mem_principal]
  constructor
  · rintro ⟨?, hcone, hYmem, hsub⟩
    have : σ = ? := cone.injective hcone
    subst this
    exact ⟨hYmem, hsub⟩
  · rintro ⟨hYmem, hsub⟩
    exact ⟨σ, rfl, hYmem, hsub⟩

theorem hMap_not_surjective : ¬ Function.Surjective hMap.toElementMap := by
  intro hsurj
  obtain ⟨x, hx⟩ := hsurj (B.principal (memB_cone [false, true]))
  have hkhx : kMap.toElementMap (hMap.toElementMap x) = x := by
    rw [← toElementMap_comp, kMap_comp_hMap, toElementMap_idMap]
  rw [hx, toElementMap_kMap_cone] at hkhx
  rw [← hkhx, toElementMap_hMap_cone] at hx
  have hcone := cone.injective (B.principal_injective (memB_cone _) (memB_cone _) hx)
  simp at hcone

theorem kMap_not_injective : ¬ Function.Injective kMap.toElementMap := by
  intro hinj
  have h1 : kMap.toElementMap (B.principal (memB_cone [false]))
      = kMap.toElementMap (B.principal (memB_cone [true])) := by
    rw [toElementMap_kMap_cone, toElementMap_kMap_cone]; rfl
  have h2 := hinj h1
  have hcone := cone.injective (B.principal_injective (memB_cone _) (memB_cone _) h2)
  simp at hcone

end DomainTheory.NeighbourhoodSystems.HMap
```

4.3.35 Exercise 2.18 — doubling binary digits

What is the meaning in words of the approximable mapping $h : \mathbb{B} \to \mathbb{B}$, where

$$h(0x) = 00h(x), \text{ and}$$
$$h(1x) = 10h(x),$$

for all elements $x \in |\mathbb{B}|$? Is h an isomorphism? Does there exist a map $k : \mathbb{B} \to \mathbb{B}$ where

$$k \circ h = I_{\mathbb{B}},$$

and is k one-one?

Theorem (`example_218`). The insert-zero map is not surjective on ideal elements, admits a left inverse under composition of approximable maps, and that left inverse is not injective.

Proof. Failure of surjectivity, the left-inverse identity, and failure of injectivity. ■

`Exercise_2_18.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.hMap

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.HMap
/-- Scott's Exercise 2.18: the insert-zero map is not surjective, has a left inverse,
    and that
left inverse is not injective. -/
theorem example_218 :
    (¬ Function.Surjective hMap.toElementMap) ∧
    (kMap.comp hMap = ApproximableMap.idMap BitDom.B) ∧
    (¬ Function.Injective kMap.toElementMap) :=
  ⟨hMap_not_surjective, kMap_comp_hMap, kMap_not_injective⟩
```

4.3.36 Approximable maps of two arguments

Approximable mappings of two arguments as ternary neighbourhood relations, their elementwise action on pairs of ideal elements, and the correspondence with approximable maps out of a product.

Definition (`ApproximableMap2`). An approximable mapping of two arguments is a ternary relation on neighbourhoods that relates only neighbourhoods of the respective systems, relates the three masters, is closed under intersection in the third argument, and is monotone when the first two neighbourhoods shrink and the third expands.

Definition (`toElementMap2`). The elementwise action of a binary approximable map on a pair of ideal elements collects those neighbourhoods related to some neighbourhoods of each argument.

Proof. Codomain membership, master, intersection, and upward closure follow from the corresponding axioms of the ternary relation. ■

Theorem (`mem_toElementMap2`). Membership in the elementwise image is existence of witnessing neighbourhoods of each argument related to the given neighbourhood.

Proof. By definition. ■

Theorem (`rel2_iff_mem_principal`). The ternary relation holds of neighbourhoods if and only if the corresponding neighbourhood belongs to the elementwise image of their principal filters.

Proof. One direction evaluates at the principals; the converse uses monotonicity to shrink to the given neighbourhoods. ■

Theorem (`toElementMap2_mono`). The elementwise action is monotone in each argument for the approximation order.

Proof. Membership witnesses transport along the inequalities. ■

Theorem (`ApproximableMap2.ext`). Ternary relations that agree on all triples of neighbourhoods determine equal binary approximable maps.

Proof. Unpack the structure and compare the relation fields. ■

Definition (`ofMap2`). A binary approximable map determines an approximable map out of the product by relating a product neighbourhood to a codomain neighbourhood when the ternary relation holds of the two untagged factors.

Proof. Approximable-map axioms follow from the ternary axioms and the product neighbourhood laws. ■

Definition (`toMap2`). An approximable map out of a product determines a binary approximable map by evaluating at products of neighbourhoods.

Proof. The ternary axioms follow from the unary ones and nesting of product neighbourhoods. ■

Theorem (`ofMap2_toMap2`). Converting a product map to a binary map and back recovers the original.

Proof. Compare relations using recovery of a product neighbourhood from its untaggings. ■

Theorem (`toMap2_ofMap2`). Converting a binary map to a product map and back recovers the original.

Proof. Compare ternary relations using untagging of a product of neighbourhoods. ■

Definition (`map2Equiv`). Approximable maps out of a product are equivalent to binary approximable maps via the two conversions.

Proof. The round-trip identities. ■

Theorem (`toElementMap2_toMap2`). The elementwise action of the binary form of a product map at a pair of ideal elements equals the unary elementwise action at their pairing.

Proof. Compare membership by unpacking product neighbourhoods into factors. ■

`ApproximableMap2.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodNbhd
public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Image

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {α β ? : Type*}

/-- Approximable mapping of two arguments: a ternary neighbourhood relation. -/
structure ApproximableMap₂ (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (V₂ : NeighborhoodSystem ?) where

  rel : Set α → Set β → Set ? → Prop
  rel_dom₀ : ∀ {X Y Z}, rel X Y Z → V₀.mem X
  rel_dom₁ : ∀ {X Y Z}, rel X Y Z → V₁.mem Y
  rel_cod : ∀ {X Y Z}, rel X Y Z → V₂.mem Z

  master_rel : rel V₀.master V₁.master V₂.master
```

```
  inter_right : ∀ {X Y Z Z'}, rel X Y Z → rel X Y Z' → rel X Y (Z ∩ Z')

  mono : ∀ {X X' Y Y' Z Z'}, rel X Y Z → X' ⊆ X → Y' ⊆ Y → Z ⊆ Z' →
    V₀.mem X' → V₁.mem Y' → V₂.mem Z' → rel X' Y' Z'

namespace ApproximableMap₂

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}

/-- Elementwise action of a binary approximable map on a pair of ideal elements. -/
def toElementMap₂ (f : ApproximableMap₂ V₀ V₁ V₂) (x : V₀.Element) (y : V₁.Element) : V₂
    .Element where
  mem Z := ∃ X Y, x.mem X ∧ y.mem Y ∧ f.rel X Y Z
  sub := fun ⟨_, _, _, _, hrel⟩ => f.rel_cod hrel
  master_mem := ⟨V₀.master, V₁.master, x.master_mem, y.master_mem, f.master_rel⟩
  inter_mem := by
    rintro Z Z' ⟨X, Y, hX, hY, hrel⟩ ⟨X', Y', hX', hY', hrel'⟩
    have hXX' : x.mem (X ∩ X') := x.inter_mem hX hX'
    have hYY' : y.mem (Y ∩ Y') := y.inter_mem hY hY'
    have hXX'm : V₀.mem (X ∩ X') := x.sub hXX'
    have hYY'm : V₁.mem (Y ∩ Y') := y.sub hYY'
    refine ⟨X ∩ X', Y ∩ Y', hXX', hYY', ?_⟩
    have h1 : f.rel (X ∩ X') (Y ∩ Y') Z :=
      f.mono hrel Set.inter_subset_left Set.inter_subset_left subset_rfl hXX'm hYY'm (
    f.rel_cod hrel)
    have h2 : f.rel (X ∩ X') (Y ∩ Y') Z' :=
      f.mono hrel' Set.inter_subset_right Set.inter_subset_right subset_rfl hXX'm hYY'
    m
        (f.rel_cod hrel')
    exact f.inter_right h1 h2
  up_mem := by
    rintro Z Z' ⟨X, Y, hX, hY, hrel⟩ hZ' hZZ'
    exact ⟨X, Y, hX, hY, f.mono hrel subset_rfl subset_rfl hZZ' (x.sub hX) (y.sub hY)
    hZ'⟩

@[simp] theorem mem_toElementMap₂ (f : ApproximableMap₂ V₀ V₁ V₂) (x : V₀.Element) (y :
    V₁.Element)
    {Z : Set ?} : (f.toElementMap₂ x y).mem Z ↔ ∃ X Y, x.mem X ∧ y.mem Y ∧ f.rel X Y Z
     := Iff.rfl

theorem rel₂_iff_mem_principal (f : ApproximableMap₂ V₀ V₁ V₂) {X : Set α} (hX : V₀.mem
    X)
    {Y : Set β} (hY : V₁.mem Y) {Z : Set ?} :
    f.rel X Y Z ↔ (f.toElementMap₂ (V₀.principal hX) (V₁.principal hY)).mem Z := by
  constructor
  · intro hrel
    exact ⟨X, Y, ⟨hX, subset_rfl⟩, ⟨hY, subset_rfl⟩, hrel⟩
  · rintro ⟨X', Y', ⟨_, hXX'⟩, ⟨_, hYY'⟩, hrel⟩
    exact f.mono hrel hXX' hYY' subset_rfl hX hY (f.rel_cod hrel)

theorem toElementMap₂_mono (f : ApproximableMap₂ V₀ V₁ V₂) {x x' : V₀.Element} {y y' : V
    ₁.Element}
    (hx : x ≤ x') (hy : y ≤ y') : f.toElementMap₂ x y ≤ f.toElementMap₂ x' y' := by
  rintro Z ⟨X, Y, hX, hY, hrel⟩
  exact ⟨X, Y, hx X hX, hy Y hY, hrel⟩
```

```
theorem ext {f g : ApproximableMap₂ V₀ V₁ V₂}
    (h : ∀ X Y Z, f.rel X Y Z ↔ g.rel X Y Z) : f = g := by
  obtain ⟨rf, _, _, _, _, _, _⟩ := f
  obtain ⟨rg, _, _, _, _, _, _⟩ := g
  have : rf = rg := by funext X Y Z; exact propext (h X Y Z)
  subst this; rfl

end ApproximableMap₂

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}
open ApproximableMap

def ofMap₂ (f : ApproximableMap₂ V₀ V₁ V₂) : ApproximableMap (prod V₀ V₁) V₂ where
  rel W Z := (prod V₀ V₁).mem W ∧ f.rel (Sum.inl ^-1' W) (Sum.inr ^-1' W) Z
  rel_dom h := h.1
  rel_cod h := f.rel_cod h.2
  master_rel := by
    refine ⟨(prod V₀ V₁).master_mem, ?_⟩
    simpa using f.master_rel
  inter_right := by rintro W Z Z' ⟨hW, hrel⟩ ⟨_, hrel'⟩; exact ⟨hW, f.inter_right hrel
    hrel'⟩
  mono := by
    rintro W W₂ Z Z' ⟨_, hrel⟩ hW₂W hZZ' hW₂ hZ'
    have hinl : Sum.inl ^-1' W₂ ⊆ Sum.inl ^-1' W := Set.preimage_mono hW₂W
    have hinr : Sum.inr ^-1' W₂ ⊆ Sum.inr ^-1' W := Set.preimage_mono hW₂W
    obtain ⟨A, B, hA, hB, rfl⟩ := hW₂
    rw [prodNbhd.inl_preimage] at hinl
    rw [prodNbhd.inr_preimage] at hinr
    refine ⟨⟨A, B, hA, hB, rfl⟩, ?_⟩
    rw [prodNbhd.inl_preimage, prodNbhd.inr_preimage]
    exact f.mono hrel hinl hinr hZZ' hA hB hZ'

def toMap₂ (f : ApproximableMap (prod V₀ V₁) V₂) : ApproximableMap₂ V₀ V₁ V₂ where
  rel X Y Z := f.rel (prodNbhd X Y) Z
  rel_dom₀ h := (prod_mem_prodNbhd_iff.mp (f.rel_dom h)).1
  rel_dom₁ h := (prod_mem_prodNbhd_iff.mp (f.rel_dom h)).2
  rel_cod h := f.rel_cod h
  master_rel := f.master_rel
  inter_right h h' := f.inter_right h h'
  mono := by
    rintro X X' Y Y' Z Z' hrel hX'X hY'Y hZZ' hX' hY' hZ'
    exact f.mono hrel (prodNbhd.subset_iff.mpr ⟨hX'X, hY'Y⟩) hZZ'
      (prod_mem_prodNbhd hX' hY') hZ'

theorem ofMap₂_toMap₂ (f : ApproximableMap (prod V₀ V₁) V₂) : ofMap₂ (toMap₂ f) = f :=
    by
  apply ApproximableMap.ext
  intro W Z
  show (prod V₀ V₁).mem W ∧ f.rel (prodNbhd (Sum.inl ^-1' W) (Sum.inr ^-1' W)) Z ↔ _
  constructor
  · rintro ⟨hW, hrel⟩; rwa [← prodNbhd.preimage hW] at hrel
  · intro hrel
    exact ⟨f.rel_dom hrel, by rwa [← prodNbhd.preimage (f.rel_dom hrel)]⟩
```

```
theorem toMap₂_ofMap₂ (f : ApproximableMap₂ V₀ V₁ V₂) : toMap₂ (ofMap₂ f) = f := by
  apply ApproximableMap₂.ext
  intro X Y Z
  show (prod V₀ V₁).mem (prodNbhd X Y) ∧ f.rel _ _ Z ↔ _
  rw [prodNbhd.inl_preimage, prodNbhd.inr_preimage]
  constructor
  · rintro ⟨_, hrel⟩; exact hrel
  · intro hrel
    exact ⟨prod_mem_prodNbhd (f.rel_dom₀ hrel) (f.rel_dom₁ hrel), hrel⟩

def map₂Equiv (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
    NeighborhoodSystem ?) :
    ApproximableMap (prod V₀ V₁) V₂ ≃ ApproximableMap₂ V₀ V₁ V₂ where
  toFun := toMap₂
  invFun := ofMap₂
  left_inv := ofMap₂_toMap₂
  right_inv := toMap₂_ofMap₂

theorem toElementMap₂_toMap₂ (f : ApproximableMap (prod V₀ V₁) V₂) (x : V₀.Element) (y :
     V₁.Element) :
    (toMap₂ f).toElementMap₂ x y = f.toElementMap (pair x y) := by
  apply Element.ext
  intro Z
  constructor
  · rintro ⟨X, Y, hX, hY, hrel⟩
    exact ⟨prodNbhd X Y, ⟨X, Y, hX, hY, rfl⟩, hrel⟩
  · rintro ⟨W, ⟨X, Y, hX, hY, rfl⟩, hrel⟩
    exact ⟨X, Y, hX, hY, hrel⟩

end

end DomainTheory.NeighbourhoodSystems
```

4.3.37 Exercise 2.19 — maps of two arguments

Generalize Definition 2.1 in an appropriate way in order to define the concept of *an approximable mapping*

$$f : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2$$

of two variables. (Hint: f can be taken to be a certain kind of ternary relation

$$f \subseteq \mathcal{D}_0 \times \mathcal{D}_1 \times \mathcal{D}_2,$$

where we can write

$$X, Y \; f \; Z$$

for the relationship among neighbourhoods.) What is the corresponding version of Proposition 2.2 for functions of two variables?

Theorem (`exercise_2_19`). Membership in the elementwise image of a binary approximable map is witnessed by related neighbourhoods of each argument; the ternary relation is recovered by evaluating at principals; and the elementwise action is monotone in each argument.

Proof. The membership characterisation, the principal-recovery law, and monotonicity of the elementwise action. The ternary relation itself is the two-variable generalisation of approximable maps. ■

`Exercise_2_19.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.
    ApproximableMap2

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open ApproximableMap₂

/-- Scott's Exercise 2.19: the elementwise action of a binary approximable map
    recovers the
ternary neighbourhood relation on principals and is monotone in each argument. -/
theorem exercise_2_19 :
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (f : ApproximableMap₂ V₀ V₁ V₂)
      (x : V₀.Element) (y : V₁.Element) {Z : Set ?},
      (f.toElementMap₂ x y).mem Z ↔ ∃ X Y, x.mem X ∧ y.mem Y ∧ f.rel X Y Z) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (f : ApproximableMap₂ V₀ V₁ V₂)
      {X : Set α} (hX : V₀.mem X) {Y : Set β} (hY : V₁.mem Y) {Z : Set ?},
      f.rel X Y Z ↔ (f.toElementMap₂ (V₀.principal hX) (V₁.principal hY)).mem Z) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (f : ApproximableMap₂ V₀ V₁ V₂)
      {x x' : V₀.Element} {y y' : V₁.Element},
      x ≤ x' → y ≤ y' → f.toElementMap₂ x y ≤ f.toElementMap₂ x' y') :=
  ⟨@ApproximableMap₂.mem_toElementMap₂, @ApproximableMap₂.rel₂_iff_mem_principal,
    @ApproximableMap₂.toElementMap₂_mono⟩
```

4.3.38 Powerset neighbourhood system

The powerset neighbourhood system on the naturals, its identification with arbitrary subsets, and approximable union, intersection, successor, and predecessor maps.

Definition (`powerSet`). The powerset neighbourhood system on the naturals takes as neighbourhoods those sets whose complements are finite, with master the universe.

Proof. The empty complement is finite; intersecting neighbourhoods unions their finite complements; every neighbourhood sits in the universe. ■

Theorem (`mem_powerSet`). A set is a powerset neighbourhood if and only if its complement is finite.

Proof. By definition. ■

Theorem (`powerSet_master`). The master is the universe.

Proof. By definition. ■

Theorem (`mem_compl_singleton`). The complement of a singleton is a powerset neighbourhood.

Proof. Its complement is a singleton, hence finite. ■

Theorem (`powerSet_inter_mem`). The intersection of two powerset neighbourhoods is a powerset neighbourhood.

Proof. Complements of intersections are unions of finite sets. ■

Definition (`toSet`). The set of naturals corresponding to an ideal element consists of those n for which the complement of $\{n\}$ belongs to the element.

Theorem (`mem_toSet`). Membership of n in the corresponding set is membership of the complement of $\{n\}$ in the ideal element.

Proof. By definition. ■

Theorem (`mem_compl_of_finite`). If a finite set of naturals each have their singleton-complements in an ideal element, then the complement of that finite set belongs to the element.

Proof. Induction on the finite set, using intersection of filters. ■

Definition (`ofSet`). An arbitrary subset of the naturals determines an ideal element consisting of those neighbourhoods whose complements are contained in the subset.

Proof. Master, intersection, and upward closure follow from the corresponding set operations on complements. ■

Theorem (`ofSet_mono`). Larger subsets determine larger ideal elements in the approximation order.

Proof. Containment of complements is preserved by enlarging the ambient set. ■

Theorem (`ofSet_toSet`). Passing from an ideal element to its set and back recovers the element.

Proof. Membership of a neighbourhood rearranges via the finite-complement characterisation against the corresponding set. ■

Definition (`predSet`). The predecessor of a set of naturals consists of those n with $n + 1$ in the set.

Theorem (`mem_predSet`). Membership in the predecessor set is membership of the successor in the original set.

Proof. By definition. ■

Definition (`interMap2`). Intersection of ideal elements is the binary approximable map relating neighbourhoods when their union is contained in the third.

Proof. Domain, codomain, master, intersection, and monotonicity follow from the powerset neighbourhood axioms and union of sets. ■

Theorem (`predSet_mono`). Predecessor is monotone for set inclusion.

Proof. Immediate. ■

Definition (`predMap`). The predecessor transformation is the approximable map relating neighbourhoods when the complement of the codomain neighbourhood is contained in the predecessor of the complement of the domain neighbourhood.

Proof. As for other complement-based maps on the powerset system. ■

Definition (`succSet`). The successor of a set of naturals is the image under $n \mapsto n + 1$.

Theorem (`succSet_mono`). Successor is monotone for set inclusion.

Proof. Images preserve inclusions. ■

Definition (`succMap`). The successor transformation is the approximable map relating neighbourhoods when the complement of the codomain neighbourhood is contained in the successor of the complement of the domain neighbourhood.

Proof. As for the predecessor map. ■

Theorem (`toSet_ofSet`). Passing from a set to an ideal element and back recovers the set.

Proof. Membership of a natural rearranges against singleton-complements. ■

Definition (`equivSetNat`). Ideal elements of the powerset system are order-isomorphic to arbitrary subsets of the naturals via the to-set / of-set correspondence.

Proof. The two round-trips and the order correspondence follow from the preceding lemmas. ■

Definition (`unionMap`). Union of ideal elements is the binary approximable map relating neighbourhoods when their intersection is contained in the third.

Proof. Dual to the intersection map. ■

Theorem (`toSet_interMap2`). Under the set identification, the elementwise action of the intersection map is intersection of sets.

Proof. Compare membership of a natural via singleton-complements and the defining relation. ■

Theorem (`toSet_predMap`). Under the set identification, the elementwise action of the predecessor map is the predecessor of sets.

Proof. Compare membership via singleton-complements and the defining relation. ■

Theorem (`toSet_succMap`). Under the set identification, the elementwise action of the successor map is the successor of sets.

Proof. Compare membership via singleton-complements and the defining relation. ■

Theorem (`toSet_unionMap`). Under the set identification, the elementwise action of the union map is union of sets.

Proof. Compare membership via singleton-complements and the defining relation. ■

powerSet.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.
    ApproximableMap2
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import Mathlib.Order.Hom.Basic
public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Finite.Basic
public import Mathlib.Order.BooleanAlgebra.Set
public import Mathlib.Data.Set.Image

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.PowerSet
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

def powerSet : NeighborhoodSystem ℕ where
  mem X := X?.Finite
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := by rw [Set.compl_univ]; exact Set.finite_empty
  inter_mem := by
    intro X Y _ hX hY _ _
    rw [Set.compl_inter]; exact hX.union hY
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem mem_powerSet {X : Set ℕ} : powerSet.mem X ↔ X?.Finite := Iff.rfl

@[simp] theorem powerSet_master : powerSet.master = (Set.univ : Set ℕ) := rfl

theorem mem_compl_singleton (n : ℕ) : powerSet.mem ({n}?) := by
  rw [mem_powerSet, compl_compl]; exact Set.finite_singleton n

theorem powerSet_inter_mem {A B : Set ℕ} (hA : powerSet.mem A) (hB : powerSet.mem B) :
    powerSet.mem (A ∩ B) := by
  rw [mem_powerSet, Set.compl_inter]; exact hA.union hB

def toSet (x : powerSet.Element) : Set ℕ := {n | x.mem ({n}?)}
```

```
@[simp] theorem mem_toSet {x : powerSet.Element} {n : ℕ} : n ∈ toSet x ↔ x.mem ({n}?)
     := Iff.rfl

theorem mem_compl_of_finite (x : powerSet.Element) {t : Set ℕ} (ht : t.Finite)
    (h : ∀ n ∈ t, x.mem ({n}?)) : x.mem (t?) := by
  revert h
  induction t, ht using Set.Finite.induction_on with
  | empty => intro _; rw [Set.compl_empty]; exact x.master_mem
  | @insert a s _ha _hsfin ih =>
    intro h
    have hcompl : (insert a s)? = ({a}?) ∩ (s?) := by rw [Set.insert_eq, Set.
    compl_union]
    rw [hcompl]
    exact x.inter_mem (h a (Set.mem_insert a s))
      (ih (fun n hn => h n (Set.mem_insert_of_mem a hn)))

def ofSet (S : Set ℕ) : powerSet.Element where
  mem Z := powerSet.mem Z ∧ Z? ⊆ S
  sub h := h.1
  master_mem := ⟨powerSet.master_mem, by rw [powerSet_master, Set.compl_univ]; exact
    Set.empty_subset S⟩
  inter_mem := by
    rintro Z W ⟨hZ, hZS⟩ ⟨hW, hWS⟩
    refine ⟨powerSet_inter_mem hZ hW, ?_⟩
    rw [Set.compl_inter]; exact Set.union_subset hZS hWS
  up_mem := by
    rintro Z W ⟨hZ, hZS⟩ hW hZW
    exact ⟨hW, (Set.compl_subset_compl.mpr hZW).trans hZS⟩

theorem ofSet_mono {S S' : Set ℕ} (h : S ⊆ S') : ofSet S ≤ ofSet S' :=
  fun _ hZ => ⟨hZ.1, hZ.2.trans h⟩

theorem ofSet_toSet (x : powerSet.Element) : ofSet (toSet x) = x := by
  apply Element.ext
  intro Z
  constructor
  · rintro ⟨hZ, hZsub⟩
    have hmem := mem_compl_of_finite x hZ (fun n hn => hZsub hn)
    rwa [compl_compl] at hmem
  · intro hZ
    refine ⟨x.sub hZ, ?_⟩
    intro n hn
    rw [mem_toSet]
    exact x.up_mem hZ (mem_compl_singleton n) (Set.subset_compl_singleton_iff.mpr hn)

def predSet (S : Set ℕ) : Set ℕ := {n | n + 1 ∈ S}

@[simp] theorem mem_predSet {S : Set ℕ} {n : ℕ} : n ∈ predSet S ↔ n + 1 ∈ S := Iff.
    rfl

def interMap₂ : ApproximableMap₂ powerSet powerSet powerSet where
  rel X Y Z := powerSet.mem X ∧ powerSet.mem Y ∧ powerSet.mem Z ∧ X ∪ Y ⊆ Z
  rel_dom₀ h := h.1
  rel_dom₁ h := h.2.1
  rel_cod h := h.2.2.1
```

```
  master_rel := ⟨powerSet.master_mem, powerSet.master_mem, powerSet.master_mem,
    Set.union_subset (Set.subset_univ _) (Set.subset_univ _)⟩
  inter_right := by
    rintro X Y Z Z' ⟨hX, hY, hZ, hsub⟩ ⟨_, _, hZ', hsub'⟩
    exact ⟨hX, hY, powerSet_inter_mem hZ hZ', Set.subset_inter hsub hsub'⟩
  mono := by
    rintro X X' Y Y' Z Z' ⟨_, _, _, hsub⟩ hX'X hY'Y hZZ' hX' hY' hZ'
    exact ⟨hX', hY', hZ', (Set.union_subset_union hX'X hY'Y).trans (hsub.trans hZZ')⟩

theorem predSet_mono {S T : Set ℕ} (h : S ⊆ T) : predSet S ⊆ predSet T := fun _ hn =>
    h hn

def predMap : ApproximableMap powerSet powerSet where
  rel X Z := powerSet.mem X ∧ powerSet.mem Z ∧ Z? ⊆ predSet X?
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine ⟨powerSet.master_mem, powerSet.master_mem, ?_⟩
    rw [powerSet_master, Set.compl_univ]; exact Set.empty_subset _
  inter_right := by
    rintro X Z Z' ⟨hX, hZ, hsub⟩ ⟨_, hZ', hsub'⟩
    refine ⟨hX, powerSet_inter_mem hZ hZ', ?_⟩
    rw [Set.compl_inter]; exact Set.union_subset hsub hsub'
  mono := by
    rintro X X' Z Z' ⟨_, _, hsub⟩ hX'X hZZ' hX' hZ'
    refine ⟨hX', hZ', ?_⟩
    exact (Set.compl_subset_compl.mpr hZZ').trans
      (hsub.trans (predSet_mono (Set.compl_subset_compl.mpr hX'X)))

def succSet (S : Set ℕ) : Set ℕ := (· + 1) '' S

theorem succSet_mono {S T : Set ℕ} (h : S ⊆ T) : succSet S ⊆ succSet T := Set.
    image_mono h

def succMap : ApproximableMap powerSet powerSet where
  rel X Z := powerSet.mem X ∧ powerSet.mem Z ∧ Z? ⊆ succSet X?
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine ⟨powerSet.master_mem, powerSet.master_mem, ?_⟩
    rw [powerSet_master, Set.compl_univ]; exact Set.empty_subset _
  inter_right := by
    rintro X Z Z' ⟨hX, hZ, hsub⟩ ⟨_, hZ', hsub'⟩
    refine ⟨hX, powerSet_inter_mem hZ hZ', ?_⟩
    rw [Set.compl_inter]; exact Set.union_subset hsub hsub'
  mono := by
    rintro X X' Z Z' ⟨_, _, hsub⟩ hX'X hZZ' hX' hZ'
    refine ⟨hX', hZ', ?_⟩
    exact (Set.compl_subset_compl.mpr hZZ').trans
      (hsub.trans (succSet_mono (Set.compl_subset_compl.mpr hX'X)))

theorem toSet_ofSet (S : Set ℕ) : toSet (ofSet S) = S := by
  ext n
  simp only [mem_toSet]
  constructor
  · rintro ⟨_, hsub⟩
```

```
    rw [compl_compl] at hsub
    exact hsub (Set.mem_singleton n)
  · intro hn
    exact ⟨mem_compl_singleton n, by rw [compl_compl]; exact Set.singleton_subset_iff.
    mpr hn⟩

def equivSetNat : powerSet.Element ≃o Set ℕ where
  toFun := toSet
  invFun := ofSet
  left_inv := ofSet_toSet
  right_inv := toSet_ofSet
  map_rel_iff' := by
    intro x y
    show toSet x ≤ toSet y ↔ x ≤ y
    constructor
    · intro h
      have hmono := ofSet_mono h
      rwa [ofSet_toSet, ofSet_toSet] at hmono
    · intro h n hn
      exact h _ hn

def unionMap : ApproximableMap₂ powerSet powerSet powerSet where
  rel X Y Z := powerSet.mem X ∧ powerSet.mem Y ∧ powerSet.mem Z ∧ X ∩ Y ⊆ Z
  rel_dom₀ h := h.1
  rel_dom₁ h := h.2.1
  rel_cod h := h.2.2.1
  master_rel := ⟨powerSet.master_mem, powerSet.master_mem, powerSet.master_mem, Set.
    inter_subset_left⟩
  inter_right := by
    rintro X Y Z Z' ⟨hX, hY, hZ, hsub⟩ ⟨_, _, hZ', hsub'⟩
    exact ⟨hX, hY, powerSet_inter_mem hZ hZ', Set.subset_inter hsub hsub'⟩
  mono := by
    rintro X X' Y Y' Z Z' ⟨_, _, _, hsub⟩ hX'X hY'Y hZZ' hX' hY' hZ'
    exact ⟨hX', hY', hZ', (Set.inter_subset_inter hX'X hY'Y).trans (hsub.trans hZZ')⟩

theorem toSet_interMap₂ (x y : powerSet.Element) :
    toSet (interMap₂.toElementMap₂ x y) = toSet x ∩ toSet y := by
  ext k
  simp only [mem_toSet, ApproximableMap₂.mem_toElementMap₂, Set.mem_inter_iff]
  constructor
  · rintro ⟨X, Y, hxX, hyY, _, _, _, hsub⟩
    have hkX : k ∈ X := fun h => (hsub (Or.inl h)) rfl
    have hkY : k ∈ Y := fun h => (hsub (Or.inr h)) rfl
    exact ⟨x.up_mem hxX (mem_compl_singleton k) (Set.subset_compl_singleton_iff.mpr hkX
    ),
      y.up_mem hyY (mem_compl_singleton k) (Set.subset_compl_singleton_iff.mpr hkY)⟩
  · rintro ⟨hkx, hky⟩
    exact ⟨{k}?, {k}?, hkx, hky, mem_compl_singleton k, mem_compl_singleton k,
      mem_compl_singleton k, Set.union_subset subset_rfl subset_rfl⟩

theorem toSet_predMap (x : powerSet.Element) :
    toSet (predMap.toElementMap x) = predSet (toSet x) := by
  ext k
  simp only [mem_toSet, ApproximableMap.mem_toElementMap, mem_predSet]
  constructor
  · rintro ⟨X, hxX, _, _, hsub⟩
```

```
    rw [compl_compl] at hsub
    have hk : k + 1 ∈ X? := hsub (Set.mem_singleton k)
    exact x.up_mem hxX (mem_compl_singleton (k + 1)) (Set.subset_compl_singleton_iff.
    mpr hk)
  · intro hk
    refine ⟨{k + 1}?, hk, mem_compl_singleton (k + 1), mem_compl_singleton k, ?_⟩
    rw [compl_compl]
    intro a ha
    rw [Set.mem_singleton_iff] at ha; subst ha
    rw [mem_predSet, compl_compl]
    exact Set.mem_singleton (a + 1)

theorem toSet_succMap (x : powerSet.Element) :
    toSet (succMap.toElementMap x) = succSet (toSet x) := by
  ext k
  simp only [mem_toSet, ApproximableMap.mem_toElementMap]
  constructor
  · rintro ⟨X, hxX, _, _, hsub⟩
    rw [compl_compl] at hsub
    obtain ⟨m, hm, rfl⟩ := hsub (Set.mem_singleton k)
    exact ⟨m, x.up_mem hxX (mem_compl_singleton m) (Set.subset_compl_singleton_iff.mpr
    hm), rfl⟩
  · rintro ⟨m, hm, rfl⟩
    refine ⟨{m}?, hm, mem_compl_singleton m, mem_compl_singleton (m + 1), ?_⟩
    rw [compl_compl]
    intro a ha
    rw [Set.mem_singleton_iff] at ha; subst ha
    exact ⟨m, by rw [compl_compl]; exact Set.mem_singleton m, rfl⟩

theorem toSet_unionMap (x y : powerSet.Element) :
    toSet (unionMap.toElementMap₂ x y) = toSet x ∪ toSet y := by
  ext k
  simp only [mem_toSet, ApproximableMap₂.mem_toElementMap₂, Set.mem_union]
  constructor
  · rintro ⟨X, Y, hxX, hyY, _, _, _, hsub⟩
    by_cases hkX : k ∈ X
    · refine Or.inr ?_
      have hkY : k ∈ Y := fun hkY => (hsub ⟨hkX, hkY⟩) rfl
      exact y.up_mem hyY (mem_compl_singleton k) (Set.subset_compl_singleton_iff.mpr
    hkY)
    · exact Or.inl (x.up_mem hxX (mem_compl_singleton k) (Set.
    subset_compl_singleton_iff.mpr hkX))
  · rintro (hkx | hky)
    · exact ⟨{k}?, Set.univ, hkx, y.master_mem, mem_compl_singleton k, powerSet.
    master_mem,
        mem_compl_singleton k, Set.inter_subset_left⟩
    · exact ⟨Set.univ, {k}?, x.master_mem, hky, powerSet.master_mem,
    mem_compl_singleton k,
        mem_compl_singleton k, Set.inter_subset_right⟩

end DomainTheory.NeighbourhoodSystems.PowerSet
```

4.3.39 Exercise 2.20 — powerset of the integers

Discuss again the example of Exercise 1.15 where the domain turns out to be the powerset (set of all subsets) of $\mathbf{N}$. Show how the finite elements can be taken to be the finite subsets of $\mathbf{N}$ and can be identified with the tokens of a suitable neighbourhood system $\mathcal{P}$. (Hint: Define $\uparrow F$ for finite sets $F \subseteq \mathbf{N}$.) Show that both union and intersection ($x \cup y$ and $x \cap y$) are functions on $|\mathcal{P}|$ that are approximable in the sense of Exercise 2.19. (The elements of $|\mathcal{P}|$ are being identified with arbitrary sets $x \subseteq \mathbf{N}$.) Show also that the following transformations are approximable:

$$x + 1 = \{n + 1 \mid n \in x\}, \text{ and}$$
$$x - 1 = \{n \mid n + 1 \in x\}.$$

Theorem (`example_220`). Under the identification of ideal elements with subsets of the naturals, the elementwise images of the binary union and intersection maps recover set-theoretic union and intersection, and the elementwise images of the successor and predecessor maps recover the corresponding set transformations.

Proof. The four recovery laws for the powerset maps. ■

`Exercise_2_20.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.powerSet

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.PowerSet
/-- Scott's Exercise 2.20: under the identification of ideal elements with subsets of
    ℕ,
union and intersection are binary approximable maps, and successor/predecessor of sets
     are
approximable. -/
theorem example_220 :
    (∀ (x y : powerSet.Element),
      toSet (unionMap.toElementMap₂ x y) = toSet x ∪ toSet y) ∧
    (∀ (x y : powerSet.Element),
      toSet (interMap₂.toElementMap₂ x y) = toSet x ∩ toSet y) ∧
    (∀ (x : powerSet.Element), toSet (succMap.toElementMap x) = succSet (toSet x)) ∧
    (∀ (x : powerSet.Element), toSet (predMap.toElementMap x) = predSet (toSet x)) :=
  ⟨toSet_unionMap, toSet_interMap₂, toSet_succMap, toSet_predMap⟩
```

4.3.40 Finite and infinite sequence neighbourhood system

The neighbourhood system of finite and infinite binary sequences, with cones and terminator singletons as tokens, the empty total sequence above bottom, and left-biased approximable juxtaposition.

Theorem (`cone_not_subset_singleton`). No cone is contained in a singleton.

Proof. A cone contains both a string and a proper extension, which cannot both lie in one singleton. ■

Theorem (`cone_ne_singleton`). No cone equals a singleton.

Proof. Equality would force the cone into the singleton. ■

Theorem (`cone_singleton_nd`). A cone and a singleton are nested or disjoint.

Proof. If the cone tip prefixes the singleton string then the singleton sits in the cone; otherwise their intersection is empty. ■

Definition (`memC`). A set of strings is a neighbourhood of the enlarged system when it is a cone or a singleton.

Theorem (`singleton_cone_nd`). A singleton and a cone are nested or disjoint.

Proof. Dual to the cone–singleton case via the prefix relation. ■

Theorem (`singleton_singleton_nd`). Two singletons are nested or disjoint.

Proof. Equal strings give nesting; unequal strings give empty intersection. ■

Theorem (`nestedOrDisjoint_C`). Any two neighbourhoods of the enlarged system are nested or disjoint.

Proof. Case analysis on cones versus singletons, using the four nested-or-disjoint lemmas. ■

Definition (`C`). The enlarged neighbourhood system on binary strings has master the universe and neighbourhoods the cones together with the terminator singletons.

Proof. Nested-or-disjoint construction with the empty-cone master. ■

Theorem (`C_mem`). Membership in the enlarged system is the cone-or-singleton predicate.

Proof. By definition. ■

Theorem (`C_master`). The master of the enlarged system is the universe.

Proof. By definition. ■

Theorem (`memC_cone`). Every cone is a neighbourhood of the enlarged system.

Proof. One of the two alternatives in the neighbourhood predicate. ■

Theorem (`memC_singleton`). Every singleton is a neighbourhood of the enlarged system.

Proof. The other alternative in the neighbourhood predicate. ■

Theorem (`prepend_mono`). Prepending a fixed string is monotone for set inclusion.

Proof. Immediate from the definition of prepend. ■

Theorem (`prepend_singleton`). Prepending a string to a singleton yields the singleton of the concatenation.

Proof. Unpack membership in the prepended set. ■

Theorem (`memC_prepend`). Prepending a string to a neighbourhood of the enlarged system yields a neighbourhood.

Proof. Prepending to a cone yields a cone; prepending to a singleton yields a singleton. ■

Theorem (`prepend_subset_cone`). The prepend of any set under a string sits inside that string's cone.

Proof. Concatenation witnesses the prefix relation. ■

Definition (`singletonElt`). The ideal element associated to a finite total sequence is the principal filter of its terminator singleton.

Definition (`Lambda`). The empty total sequence is the singleton element of the empty string.

Theorem (`subset_singleton_eq`). A neighbourhood contained in a singleton equals that singleton.

Proof. Cones never sit in singletons; a singleton contained in another forces equality of tips. ■

Definition (`juxtapose`). Juxtaposition is the binary approximable map relating neighbourhoods when either the left is a cone contained in the result, or the left is a singleton whose prepend of the right neighbourhood sits in the result.

Proof. Domain, codomain, master, intersection, and monotonicity follow from the neighbourhood axioms, injectivity of cones, and the prepend lemmas. ■

Theorem (`bot_lt_Lambda`). Bottom is strictly below the empty total sequence.

Proof. Bottom's only neighbourhood is the master, which cannot equal the empty singleton. ■

Theorem (`isTotal_singletonElt`). Every finite terminator element is total.

Proof. Any neighbourhood of a larger element meeting the terminator singleton must recover that singleton by the subset-singleton law. ■

Theorem (`juxtapose_cone`). Juxtaposing a cone principal on the left with any right argument recovers that cone principal.

Proof. Unpack the defining relation: the cone branch forces containment in the result, and the singleton branch is impossible for a cone left argument. ■

Theorem (`juxtapose_singleton_mem`). Membership in the juxtaposition of a terminator element with a right argument is witnessed by a neighbourhood of the right whose prepend under the terminator tip sits in the result.

Proof. Compare the defining relation against the singleton left argument. ■

C.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.
    ApproximableMap2
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ExampleB
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    sigmaBot_le_iff
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.IsTotal
import Mathlib.Data.Set.Insert

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem BitDom

theorem cone_not_subset_singleton (σ τ : Str) : ¬ cone σ ⊆ ({τ} : Set Str) := by
  intro h
  have h1 : σ ∈ ({τ} : Set Str) := h (cone.mem.mpr List.prefix_rfl)
  have h2 : (σ ++ [false]) ∈ ({τ} : Set Str) := h (cone.mem.mpr (List.prefix_append σ
    [false]))
  rw [Set.mem_singleton_iff] at h1 h2
  rw [← h1] at h2
  have hlen := congrArg List.length h2
  simp only [List.length_append, List.length_cons, List.length_nil] at hlen
  omega

theorem cone_ne_singleton (σ τ : Str) : cone σ = ({τ} : Set Str) := by
  intro h
  refine cone_not_subset_singleton σ τ ?_
  intro x hx
  rw [← h]; exact hx

theorem cone_singleton_nd (σ τ : Str) :
    cone σ ⊆ ({τ} : Set Str) ∨ ({τ} : Set Str) ⊆ cone σ ∨ cone σ ∩ ({τ} : Set Str) = ∅
     :=
```

```
  if h : σ <+: τ then
    Or.inr (Or.inl (Set.singleton_subset_iff.mpr (cone.mem.mpr h)))
  else
    Or.inr (Or.inr (by
      ext w
      simp only [Set.mem_inter_iff, cone.mem, Set.mem_singleton_iff, Set.
    mem_empty_iff_false,
        iff_false, not_and]
      rintro hw rfl
      exact h hw))

def memC (X : Set Str) : Prop := (∃ σ, X = cone σ) ∨ (∃ σ, X = {σ})

theorem singleton_cone_nd (σ τ : Str) :
    ({σ} : Set Str) ⊆ cone τ ∨ cone τ ⊆ ({σ} : Set Str) ∨ ({σ} : Set Str) ∩ cone τ = ∅
     :=
  if h : τ <+: σ then
    Or.inl (Set.singleton_subset_iff.mpr (cone.mem.mpr h))
  else
    Or.inr (Or.inr (by
      ext w
      simp only [Set.mem_inter_iff, Set.mem_singleton_iff, cone.mem, Set.
    mem_empty_iff_false,
        iff_false, not_and]
      rintro rfl hw
      exact h hw))

theorem singleton_singleton_nd (σ τ : Str) :
    ({σ} : Set Str) ⊆ ({τ} : Set Str) ∨ ({τ} : Set Str) ⊆ ({σ} : Set Str) ∨
      ({σ} : Set Str) ∩ ({τ} : Set Str) = ∅ := by
  by_cases h : σ = τ
  · subst h; exact Or.inl subset_rfl
  · refine Or.inr (Or.inr ?_)
    ext w
    simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false,
      not_and]
    rintro rfl h2
    exact h h2

theorem nestedOrDisjoint_C : NestedOrDisjoint memC := by
  rintro X Y hX hY
  rcases hX with ⟨σ, rfl⟩ | ⟨σ, rfl⟩ <;> rcases hY with ⟨τ, rfl⟩ | ⟨τ, rfl⟩
  · exact cone.trichotomy σ τ
  · exact cone_singleton_nd σ τ
  · exact singleton_cone_nd σ τ
  · exact singleton_singleton_nd σ τ

def C : NeighborhoodSystem Str :=
  ofNestedOrDisjoint memC Set.univ ⟨[], Set.mem_univ _⟩
    (Or.inl ⟨[], cone.nil.symm⟩) nestedOrDisjoint_C
    (by rintro X (⟨σ, rfl⟩ | ⟨σ, rfl⟩) <;> exact Set.subset_univ _)

namespace C

@[simp] theorem C_mem {X : Set Str} : C.mem X ↔ memC X := Iff.rfl
```

```
@[simp] theorem C_master : C.master = Set.univ := rfl

theorem memC_cone (σ : Str) : C.mem (cone σ) := Or.inl ⟨σ, rfl⟩

theorem memC_singleton (σ : Str) : C.mem ({σ} : Set Str) := Or.inr ⟨σ, rfl⟩

theorem prepend_mono (σ : Str) {Y Y' : Set Str} (h : Y' ⊆ Y) : prepend σ Y' ⊆ prepend
    σ Y := by
  rintro w ⟨τ, hτ, rfl⟩
  exact ⟨τ, h hτ, rfl⟩

theorem prepend_singleton (σ ? : Str) : prepend σ ({?} : Set Str) = ({σ ++ ?} : Set
    Str) := by
  ext w
  simp only [mem_prepend, Set.mem_singleton_iff]
  constructor
  · rintro ⟨τ, rfl, rfl⟩; rfl
  · rintro rfl; exact ⟨?, rfl, rfl⟩

theorem memC_prepend (σ : Str) {Y : Set Str} (hY : C.mem Y) : C.mem (prepend σ Y) :=
    by
  rcases hY with ⟨?, rfl⟩ | ⟨?, rfl⟩
  · rw [prepend_cone]; exact memC_cone (σ ++ ?)
  · rw [prepend_singleton]; exact memC_singleton (σ ++ ?)

theorem prepend_subset_cone (σ : Str) (Y : Set Str) : prepend σ Y ⊆ cone σ := by
  rintro w ⟨τ, _, rfl⟩
  exact cone.mem.mpr (List.prefix_append σ τ)

def singletonElt (σ : Str) : C.Element := C.principal (memC_singleton σ)

def Lambda : C.Element := singletonElt []

theorem subset_singleton_eq {W : Set Str} {σ : Str} (hW : C.mem W) (hsub : W ⊆ ({σ} :
    Set Str)) :
    W = ({σ} : Set Str) := by
  rcases hW with ⟨?, rfl⟩ | ⟨?, rfl⟩
  · exact absurd hsub (cone_not_subset_singleton ? σ)
  · rw [Set.singleton_subset_iff, Set.mem_singleton_iff] at hsub
    rw [hsub]

def juxtapose : ApproximableMap₂ C C C where
  rel X Y Z := C.mem Y ∧ C.mem Z ∧
    ((∃ σ, X = cone σ ∧ cone σ ⊆ Z) ∨ (∃ σ, X = ({σ} : Set Str) ∧ prepend σ Y ⊆ Z))
  rel_dom₀ := by
    rintro X Y Z ⟨_, _, (⟨σ, rfl, _⟩ | ⟨σ, rfl, _⟩)⟩
    · exact memC_cone σ
    · exact memC_singleton σ
  rel_dom₁ := fun ⟨hY, _, _⟩ => hY
  rel_cod := fun ⟨_, hZ, _⟩ => hZ
  master_rel := ⟨C.master_mem, C.master_mem,
    Or.inl ⟨[], C_master.trans cone.nil.symm, by rw [C_master]; exact Set.subset_univ _
    ⟩⟩
  inter_right := by
    rintro X Y Z Z' ⟨hY, hZ, hd⟩ ⟨_, hZ', hd'⟩
```

```
    rcases hd with ⟨σ, rfl, hsub⟩ | ⟨σ, rfl, hsub⟩
    · rcases hd' with ⟨σ', hX', hsub'⟩ | ⟨σ', hX', _⟩
      · obtain rfl : σ = σ' := cone.injective hX'
        exact ⟨hY, C.inter_mem hZ hZ' (memC_cone σ) (Set.subset_inter hsub hsub'),
          Or.inl ⟨σ, rfl, Set.subset_inter hsub hsub'⟩⟩
      · exact absurd hX' (cone_ne_singleton σ σ')
    · rcases hd' with ⟨σ', hX', _⟩ | ⟨σ', hX', hsub'⟩
      · exact absurd hX'.symm (cone_ne_singleton σ' σ)
      · obtain rfl : σ = σ' := by
          have := hX'; rw [Set.singleton_eq_singleton_iff] at this; exact this
        exact ⟨hY, C.inter_mem hZ hZ' (memC_prepend σ hY) (Set.subset_inter hsub hsub')
    ,
          Or.inr ⟨σ, rfl, Set.subset_inter hsub hsub'⟩⟩
  mono := by
    rintro X X' Y Y' Z Z' ⟨hY, _, hd⟩ hX'X hY'Y hZZ' hX' hY' hZ'
    refine ⟨hY', hZ', ?_⟩
    rcases hd with ⟨σ, rfl, hsub⟩ | ⟨σ, rfl, hsub⟩
    · rcases hX' with ⟨σ', rfl⟩ | ⟨σ', rfl⟩
      · exact Or.inl ⟨σ', rfl, (hX'X.trans hsub).trans hZZ'⟩
      · refine Or.inr ⟨σ', rfl, ?_⟩
        have hσσ' : σ <+: σ' := cone.mem.mp (Set.singleton_subset_iff.mp hX'X)
        have h2 : cone σ' ⊆ cone σ := cone.subset_cone.mpr hσσ'
        exact ((prepend_subset_cone σ' Y').trans (h2.trans hsub)).trans hZZ'
    · obtain rfl : X' = ({σ} : Set Str) := subset_singleton_eq hX' hX'X
      exact Or.inr ⟨σ, rfl, ((prepend_mono σ hY'Y).trans hsub).trans hZZ'⟩

theorem bot_lt_Lambda : C.bot < Lambda := by
  refine lt_of_le_of_ne (C.bot_le _) ?_
  intro h
  have hmem : C.bot.mem ({([] : Str)} : Set Str) := by
    rw [h]; exact ⟨memC_singleton [], subset_rfl⟩
  rw [C.mem_bot, C_master] at hmem
  have h2 : ([true] : Str) ∈ ({([] : Str)} : Set Str) := by rw [hmem]; trivial
  rw [Set.mem_singleton_iff] at h2
  exact absurd h2 (by decide)

theorem isTotal_singletonElt (σ : Str) : C.IsTotal (singletonElt σ) := by
  intro y hy Z hZ
  have hσy : y.mem ({σ} : Set Str) := hy {σ} ⟨memC_singleton σ, subset_rfl⟩
  have hinter : y.mem (({σ} : Set Str) ∩ Z) := y.inter_mem hσy hZ
  have hmemC : C.mem (({σ} : Set Str) ∩ Z) := y.sub hinter
  have heq : ({σ} : Set Str) ∩ Z = ({σ} : Set Str) :=
    subset_singleton_eq hmemC Set.inter_subset_left
  exact ⟨y.sub hZ, by rw [← heq]; exact Set.inter_subset_right⟩

theorem juxtapose_cone (σ : Str) (y : C.Element) :
    juxtapose.toElementMap₂ (C.principal (memC_cone σ)) y = C.principal (memC_cone σ)
    := by
  apply Element.ext
  intro Z
  constructor
  · rintro ⟨X, Y, ⟨_, hconeX⟩, _, hY, hZ, hd⟩
    rcases hd with ⟨?, rfl, hsub⟩ | ⟨?, rfl, _⟩
    · exact ⟨hZ, hconeX.trans hsub⟩
    · exact absurd hconeX (cone_not_subset_singleton σ ?)
  · rintro ⟨hZ, hconeσZ⟩
```

```
    exact ⟨cone σ, C.master, ⟨memC_cone σ, subset_rfl⟩, y.master_mem, C.master_mem, hZ,
      Or.inl ⟨σ, rfl, hconeσZ⟩⟩

theorem juxtapose_singleton_mem (σ : Str) (y : C.Element) {Z : Set Str} :
    (juxtapose.toElementMap₂ (singletonElt σ) y).mem Z ↔
      C.mem Z ∧ ∃ Y', y.mem Y' ∧ prepend σ Y' ⊆ Z := by
  constructor
  · rintro ⟨X, Y, ⟨_, hsingleX⟩, hYmem, _, hZ, hd⟩
    refine ⟨hZ, ?_⟩
    rcases hd with ⟨?, rfl, hsub⟩ | ⟨?, rfl, hsub⟩
    · have h?σ : ? <+: σ := cone.mem.mp (Set.singleton_subset_iff.mp hsingleX)
      refine ⟨C.master, y.master_mem, ?_⟩
      rw [C_master, prepend_univ]
      exact (cone.subset_cone.mpr h?σ).trans hsub
    · obtain rfl : σ = ? := by
        have := Set.singleton_subset_iff.mp hsingleX
        rwa [Set.mem_singleton_iff] at this
      exact ⟨Y, hYmem, hsub⟩
  · rintro ⟨hZ, Y', hY'mem, hsub⟩
    exact ⟨{σ}, Y', ⟨memC_singleton σ, subset_rfl⟩, hY'mem, y.sub hY'mem, hZ,
      Or.inr ⟨σ, rfl, hsub⟩⟩

end C
end DomainTheory.NeighbourhoodSystems
```

4.3.41 Exercise 2.21 — finite and infinite sequences

The system $\mathcal{B}$ of 2.3 has as its total elements only the infinite sequences. Modify the construction of $\mathcal{B}$ to another neighbourhood system $\mathcal{C}$ which has *both* the finite and infinite sequences as total elements. (Hint: $\mathcal{B} \subseteq \mathcal{C}$.) Show that there is an approximable map xy on elements naturally extending ordinary juxtaposition of sequences. (Hint: Write 01001 for a *total* finite sequence and $01001\,\bot$ for the corresponding finite partial element. Remember to distinguish between Λ (the total empty sequence) and $\bot$ (the undefined sequence). The definition should work out so that if x is an infinite sequence (hence, total), then $xy = x$ for all y. What will xy equal if x is not total? In other words, the construction possesses a rather strong left-to-right bias.)

Theorem (`example_221`). Bottom is strictly below the empty total sequence; every finite terminator element is total; juxtaposition of a cone principal on the left recovers that principal; and membership in the juxtaposition of a terminator element is witnessed by prepending a neighbourhood of the right argument.

Proof. Strictness of bottom below the empty total, totality of terminators, left-bias on cones, and the prepend membership law. ■

`Exercise_2_21.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.C

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem BitDom
open DomainTheory.NeighbourhoodSystems
open C
```

```
/-- Scott's Exercise 2.21: system `C` has finite totals strictly above `⊥`,
    juxtaposition is
    left-biased on cones, and on total finite left arguments it prepends
    neighbourhoods of the
    right argument. -/
theorem example_221 :
    (C.bot < Lambda) ∧
    (∀ σ, C.IsTotal (singletonElt σ)) ∧
    (∀ σ y, juxtapose.toElementMap₂ (C.principal (memC_cone σ)) y =
      C.principal (memC_cone σ)) ∧
    (∀ σ y {Z}, (juxtapose.toElementMap₂ (singletonElt σ) y).mem Z ↔
      C.mem Z ∧ ∃ Y', y.mem Y' ∧ prepend σ Y' ⊆ Z) :=
  ⟨bot_lt_Lambda, isTotal_singletonElt, juxtapose_cone, juxtapose_singleton_mem⟩
```

4.3.42 Token neighbourhood system of a closed family

The token neighbourhood system of a family closed under nonempty intersections (and, for the representation, directed unions), with finite tokens, closure, and the order isomorphism between ideal elements and members of the family.

Definition (`Cl`). The closure of a set of tokens is the intersection of all members of the family that contain it.

Theorem (`Cl_mem`). If a set is contained in some member of an intersection-closed family, then its closure belongs to the family.

Proof. The family of containing members is nonempty and contained in the ambient family, so the intersection axiom applies. ■

Theorem (`Cl_subset`). The closure of a set contained in a family member is contained in that member.

Proof. The member belongs to the intersecting family defining the closure. ■

Definition (`IsTok`). A set is a token for the family when it is finite and contained in some member of the family.

Definition (`Tok`). Tokens are the finite sets contained in members of the family, packaged as a subtype.

Definition (`nbhd`). The basic neighbourhood of a set of tokens consists of those tokens whose closures contain the set.

Theorem (`mem_nbhd`). Membership in a basic neighbourhood is containment of the generating set in the closure of the token.

Proof. By definition. ■

Definition (`botTok`). The empty token is the empty set, which is finite and contained in any nonempty family's chosen member.

Theorem (`nbhd_empty`). The basic neighbourhood of the empty set is the universe of tokens.

Proof. Every closure contains the empty set. ■

Theorem (`subset_Cl`). Every set is contained in its closure.

Proof. Each intersecting member contains the set by construction. ■

Theorem (`nbhd_subset_iff`). Basic neighbourhoods nest exactly when the larger generator sits in the closure of the smaller token.

Proof. One direction evaluates at the token itself; the other uses monotonicity of closure under containment in a family member. ■

Definition (`reprSystem`). The token neighbourhood system has as neighbourhoods the basic neighbourhoods of tokens, with master the universe of tokens.

Proof. The empty token witnesses a nonempty master; intersecting basic neighbourhoods is the basic neighbourhood of the union of tokens, using closure under nonempty intersections. ■

Theorem (`mem_reprSystem`). A set of tokens is a neighbourhood if and only if it is a basic neighbourhood of some token.

Proof. By definition. ■

Theorem (`reprSystem_master`). The master of the token neighbourhood system is the universe of tokens.

Proof. By definition. ■

Theorem (`directed_step`). Two basic neighbourhoods of an ideal element admit a common refinement whose closures dominate both closures.

Proof. Their intersection is again a basic neighbourhood, and evaluating at the refining token yields the two containments. ■

Definition (`famC`). The family associated to an ideal element consists of the closures of tokens belonging to that element.

Theorem (`mem_nbhd_bot`). Every ideal element contains the basic neighbourhood of the empty token.

Proof. That neighbourhood is the master. ■

Definition (`ofC`). A member of the ambient family determines the ideal element of neighbourhoods containing a basic neighbourhood of some token inside the member.

Proof. Master, intersection, and upward closure follow from the corresponding token and neighbourhood operations. ■

Definition (`toC`). An ideal element determines the union of the closures of its tokens.

Theorem (`mem_toC`). A point belongs to the set associated to an ideal element if and only if it lies in the closure of some token of that element.

Proof. Unpack the union. ■

Theorem (`exists_tok_of_finite_subset`). Every finite subset of the set associated to an ideal element sits in the closure of some token of that element.

Proof. Induction on the finite set, joining tokens via the directed-step lemma. ■

Theorem (`mem_nbhd_iff`). An ideal element contains the basic neighbourhood of a token if and only if that token sits in the associated set.

Proof. One direction uses containment in the closure; the converse extracts a refining token from the finite-subset lemma and applies neighbourhood monotonicity. ■

Theorem (`toC_mem`). Under directed-union closure of the ambient family, the set associated to an ideal element belongs to the family.

Proof. The family of closures of tokens of the element is nonempty and directed under inclusion, so the directed-union axiom applies. ■

Theorem (`ofC_toC`). Passing from an ideal element to its associated set and back recovers the element.

Proof. Compare neighbourhood membership via the basic-neighbourhood characterisation against the associated set. ■

Theorem (`toC_ofC`). Passing from a family member to an ideal element and back recovers the member.

Proof. One inclusion uses containment of tokens in the member; the other uses singleton tokens. ■

Definition (`reprIso`). Ideal elements of the token neighbourhood system are order-isomorphic to members of the ambient family via the to-set / of-set correspondence.

Proof. The two round-trips and the order correspondence follow from the preceding lemmas. ■

reprSystem.lean

```
module

public import Mathlib.Order.Directed

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import Mathlib.Order.SetNotation
public import Mathlib.Order.Hom.Basic
import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Finite.Basic
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.ReprSystem
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Set

section
variable {τ : Type*} (C : Set (Set τ))

def Cl (F : Set τ) : Set τ := ⋂₀ {X | X ∈ C ∧ F ⊆ X}
end

section
variable {τ : Type*} (C : Set (Set τ))
variable (hInter : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → ⋂₀ ? ∈ C)

include hInter in

theorem Cl_mem {F : Set τ} (h : ∃ X ∈ C, F ⊆ X) : Cl C F ∈ C := by
  show ⋂₀ {X | X ∈ C ∧ F ⊆ X} ∈ C
  apply hInter
  · obtain ⟨X, hX, hFX⟩ := h
    exact ⟨X, hX, hFX⟩
  · intro X hX
    exact hX.1
end

section
variable {τ : Type*} (C : Set (Set τ))

theorem Cl_subset {F X : Set τ} (hX : X ∈ C) (hFX : F ⊆ X) : Cl C F ⊆ X := by
  intro t ht
  exact (Set.mem_sInter.mp ht) X ⟨hX, hFX⟩

class IsTok (F : Set τ) : Prop where
  out : F.Finite ∧ ∃ X ∈ C, F ⊆ X

abbrev Tok := {F : Set τ // IsTok C F}
end

section
```

```
variable {τ : Type*} (C : Set (Set τ))
variable (hInter : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → ⋂₀ ? ∈ C)

def nbhd (F : Set τ) : Set (Tok C) := {G | F ⊆ Cl C G.1}

@[simp] theorem mem_nbhd {F : Set τ} {G : Tok C} : G ∈ nbhd C F ↔ F ⊆ Cl C G.1 := Iff
    .rfl
end

section
variable {τ : Type*} (C : Set (Set τ))
variable (hInter : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → ⋂₀ ? ∈ C)
variable (hne : C.Nonempty)

include hne in

def botTok : Tok C := ⟨∅, ⟨⟨Set.finite_empty, hne.choose, hne.choose_spec, Set.
    empty_subset _⟩⟩⟩
end

section
variable {τ : Type*} (C : Set (Set τ))
variable (hInter : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → ⋂₀ ? ∈ C)

theorem nbhd_empty : nbhd C (∅ : Set τ) = Set.univ := by
  ext G
  simp [nbhd]
end

section
variable {τ : Type*} (C : Set (Set τ))

theorem subset_Cl (F : Set τ) : F ⊆ Cl C F := by
  intro t ht
  rw [Cl, Set.mem_sInter]
  intro X hX
  exact hX.2 ht
end

section
variable {τ : Type*} (C : Set (Set τ))
variable (hInter : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → ⋂₀ ? ∈ C)

include hInter in

theorem nbhd_subset_iff {F G : Set τ} (hF : IsTok C F) :
    nbhd C F ⊆ nbhd C G ↔ G ⊆ Cl C F := by
  constructor
  · intro h
    have hmem : (⟨F, hF⟩ : Tok C) ∈ nbhd C F := subset_Cl C F
    exact h hmem
  · intro hG H hH
    exact hG.trans (Cl_subset C (Cl_mem C hInter H.2.out.2) hH)
end

section
```

```
variable {τ : Type*} (C : Set (Set τ))
variable (hInter : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → ⋂₀ ? ∈ C)
variable (hne : C.Nonempty)

include hInter hne in

def reprSystem : NeighborhoodSystem (Tok C) where
  mem N := ∃ F : Tok C, N = nbhd C F.1
  master := Set.univ
  master_nonempty := ⟨botTok C hne, Set.mem_univ _⟩
  master_mem := ⟨botTok C hne, (nbhd_empty C).symm⟩
  inter_mem := by
    rintro X Y Z ⟨F, rfl⟩ ⟨F', rfl⟩ ⟨F'', rfl⟩ hZsub
    have hF''mem : (F'' : Tok C) ∈ nbhd C F''.1 := subset_Cl C F''.1
    have h := hZsub hF''mem
    have hClF'' : Cl C F''.1 ∈ C := Cl_mem C hInter F''.2.out.2
    have hunionsub : F.1 ∪ F'.1 ⊆ Cl C F''.1 := Set.union_subset h.1 h.2
    refine ⟨⟨F.1 ∪ F'.1, ⟨⟨F.2.out.1.union F'.2.out.1, Cl C F''.1, hClF'', hunionsub⟩⟩⟩,
    ?_⟩
    ext G
    simp only [mem_nbhd, Set.mem_inter_iff, Set.union_subset_iff]
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem mem_reprSystem {N : Set (Tok C)} :
    (reprSystem C hInter hne).mem N ↔ ∃ F : Tok C, N = nbhd C F.1 := Iff.rfl

@[simp] theorem reprSystem_master : (reprSystem C hInter hne).master = Set.univ := rfl

theorem directed_step (x : (reprSystem C hInter hne).Element) {F F' : Tok C}
    (hF : x.mem (nbhd C F.1)) (hF' : x.mem (nbhd C F'.1)) :
    ∃ F₃ : Tok C, x.mem (nbhd C F₃.1) ∧ Cl C F.1 ⊆ Cl C F₃.1 ∧ Cl C F'.1 ⊆ Cl C F₃.1 :=
     by
  have hinter : x.mem (nbhd C F.1 ∩ nbhd C F'.1) := x.inter_mem hF hF'
  have hmemD : (reprSystem C hInter hne).mem (nbhd C F.1 ∩ nbhd C F'.1) := x.sub
    hinter
  obtain ⟨F₃, hF₃eq⟩ := hmemD
  rw [hF₃eq] at hinter
  have hF₃self : (F₃ : Tok C) ∈ nbhd C F₃.1 := subset_Cl C F₃.1
  rw [← hF₃eq] at hF₃self
  refine ⟨F₃, hinter, ?_, ?_⟩
  · exact Cl_subset C (Cl_mem C hInter F₃.2.out.2) hF₃self.1
  · exact Cl_subset C (Cl_mem C hInter F₃.2.out.2) hF₃self.2

def famC (x : (reprSystem C hInter hne).Element) : Set (Set τ) :=
  {Y | ∃ F : Tok C, x.mem (nbhd C F.1) ∧ Y = Cl C F.1}

theorem mem_nbhd_bot (x : (reprSystem C hInter hne).Element) :
    x.mem (nbhd C (botTok C hne).1) := by
  have h : (botTok C hne).1 = (∅ : Set τ) := rfl
  rw [h, nbhd_empty C]
  exact x.master_mem

def ofC (X : Set τ) (hX : X ∈ C) : (reprSystem C hInter hne).Element where
  mem N := (reprSystem C hInter hne).mem N ∧ ∃ F : Tok C, F.1 ⊆ X ∧ nbhd C F.1 ⊆ N
  sub h := h.1
  master_mem :=
```

```
    ⟨(reprSystem C hInter hne).master_mem, botTok C hne, Set.empty_subset X, Set.
    subset_univ _⟩
  inter_mem := by
    rintro N M ⟨hN, F, hFX, hFN⟩ ⟨hM, F', hF'X, hF'M⟩
    have htok : IsTok C (F.1 ∪ F'.1) :=
      ⟨⟨F.2.out.1.union F'.2.out.1, X, hX, Set.union_subset hFX hF'X⟩⟩
    have hsub : nbhd C (F.1 ∪ F'.1) ⊆ N ∩ M := by
      intro G hG
      rw [mem_nbhd, Set.union_subset_iff] at hG
      exact ⟨hFN hG.1, hF'M hG.2⟩
    exact ⟨(reprSystem C hInter hne).inter_mem hN hM ⟨⟨_, htok⟩, rfl⟩ hsub,
      ⟨F.1 ∪ F'.1, htok⟩, Set.union_subset hFX hF'X, hsub⟩
  up_mem := by
    rintro N M ⟨_, F, hFX, hFN⟩ hM hNM
    exact ⟨hM, F, hFX, hFN.trans hNM⟩

def toC (x : (reprSystem C hInter hne).Element) : Set τ := ⋃₀ famC C hInter hne x

theorem mem_toC (x : (reprSystem C hInter hne).Element) {t : τ} :
    t ∈ toC C hInter hne x ↔ ∃ F : Tok C, x.mem (nbhd C F.1) ∧ t ∈ Cl C F.1 := by
  simp only [toC, famC, Set.mem_sUnion, Set.mem_ofPred_eq]
  constructor
  · rintro ⟨Y, ⟨F, hF, rfl⟩, ht⟩
    exact ⟨F, hF, ht⟩
  · rintro ⟨F, hF, ht⟩
    exact ⟨Cl C F.1, ⟨F, hF, rfl⟩, ht⟩

theorem exists_tok_of_finite_subset (x : (reprSystem C hInter hne).Element) {s : Set τ}
    (hs : s.Finite) (hsub : s ⊆ toC C hInter hne x) :
    ∃ F : Tok C, x.mem (nbhd C F.1) ∧ s ⊆ Cl C F.1 := by
  revert hsub
  induction s, hs using Set.Finite.induction_on with
  | empty =>
    intro _
    exact ⟨botTok C hne, mem_nbhd_bot C hInter hne x, Set.empty_subset _⟩
  | @insert a s _ha _hsfin ih =>
    intro hsub
    have ha' : a ∈ toC C hInter hne x := hsub (Set.mem_insert a s)
    rw [mem_toC] at ha'
    obtain ⟨Fa, hFa, haCl⟩ := ha'
    obtain ⟨Fs, hFs, hsCl⟩ := ih (fun y hy => hsub (Set.mem_insert_of_mem a hy))
    obtain ⟨F₃, hF₃, hFaF₃, hFsF₃⟩ := directed_step C hInter hne x hFa hFs
    refine ⟨F₃, hF₃, ?_⟩
    rw [Set.insert_subset_iff]
    exact ⟨hFaF₃ haCl, hsCl.trans hFsF₃⟩

theorem mem_nbhd_iff (x : (reprSystem C hInter hne).Element) (G : Tok C) :
    x.mem (nbhd C G.1) ↔ G.1 ⊆ toC C hInter hne x := by
  constructor
  · intro hx t ht
    rw [mem_toC]
    exact ⟨G, hx, subset_Cl C G.1 ht⟩
  · intro hsub
    obtain ⟨F, hF, hGF⟩ := exists_tok_of_finite_subset C hInter hne x G.2.out.1 hsub
    have hnsub : nbhd C F.1 ⊆ nbhd C G.1 := (nbhd_subset_iff C hInter F.2).mpr hGF
    exact x.up_mem hF ⟨G, rfl⟩ hnsub
```

```
end

section
variable {τ : Type*} (C : Set (Set τ))
variable (hInter : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → ⋂₀ ? ∈ C)
variable (hne : C.Nonempty)
variable (hDir : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → DirectedOn (· ⊆ ·) ? → ⋃₀ ?
    ∈ C)

include hDir in

theorem toC_mem (x : (reprSystem C hInter hne).Element) : toC C hInter hne x ∈ C := by
  show ⋃₀ famC C hInter hne x ∈ C
  apply hDir (famC C hInter hne x)
  · exact ⟨Cl C (botTok C hne).1, botTok C hne, mem_nbhd_bot C hInter hne x, rfl⟩
  · rintro Y ⟨F, _, rfl⟩
    exact Cl_mem C hInter F.2.out.2
  · rintro Y ⟨F, hF, rfl⟩ Y' ⟨F', hF', rfl⟩
    obtain ⟨F₃, hF₃, hFF₃, hF'F₃⟩ := directed_step C hInter hne x hF hF'
    exact ⟨Cl C F₃.1, ⟨F₃, hF₃, rfl⟩, hFF₃, hF'F₃⟩

theorem ofC_toC (x : (reprSystem C hInter hne).Element) :
    ofC C hInter hne (toC C hInter hne x) (toC_mem C hInter hne hDir x) = x := by
  apply Element.ext
  intro N
  constructor
  · rintro ⟨hN, F, hFtoC, hFN⟩
    have hxF : x.mem (nbhd C F.1) := (mem_nbhd_iff C hInter hne x F).mpr hFtoC
    exact x.up_mem hxF hN hFN
  · intro hxN
    have hN : (reprSystem C hInter hne).mem N := x.sub hxN
    obtain ⟨G, rfl⟩ := hN
    have hGtoC : G.1 ⊆ toC C hInter hne x := (mem_nbhd_iff C hInter hne x G).mp hxN
    exact ⟨⟨G, rfl⟩, G, hGtoC, subset_rfl⟩
end

section
variable {τ : Type*} (C : Set (Set τ))
variable (hInter : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → ⋂₀ ? ∈ C)
variable (hne : C.Nonempty)

theorem toC_ofC (X : Set τ) (hX : X ∈ C) : toC C hInter hne (ofC C hInter hne X hX) =
    X := by
  apply Set.Subset.antisymm
  · intro t ht
    rw [mem_toC] at ht
    obtain ⟨F, hF, htCl⟩ := ht
    obtain ⟨_, F', hF'X, hF'F⟩ := hF
    have h1 : F.1 ⊆ Cl C F'.1 := (nbhd_subset_iff C hInter F'.2).mp hF'F
    have hFX : F.1 ⊆ X := h1.trans (Cl_subset C hX hF'X)
    exact Cl_subset C hX hFX htCl
  · intro t ht
    have hst : IsTok C {t} := ⟨⟨Set.finite_singleton t, X, hX, Set.singleton_subset_iff
    .mpr ht⟩⟩
    rw [mem_toC]
    refine ⟨⟨{t}, hst⟩, ?_, subset_Cl C {t} rfl⟩
```

```
    exact ⟨⟨{t}, hst⟩, rfl⟩, ⟨{t}, hst⟩, Set.singleton_subset_iff.mpr ht, subset_rfl⟩
end

section
variable {τ : Type*} (C : Set (Set τ))
variable (hInter : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → ⋂₀ ? ∈ C)
variable (hne : C.Nonempty)
variable (hDir : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → DirectedOn (· ⊆ ·) ? → ⋃₀ ?
    ∈ C)

include hDir in

def reprIso : (reprSystem C hInter hne).Element ≃o {X : Set τ // X ∈ C} where
  toFun x := ⟨toC C hInter hne x, toC_mem C hInter hne hDir x⟩
  invFun X := ofC C hInter hne X.1 X.2
  left_inv x := ofC_toC C hInter hne hDir x
  right_inv X := Subtype.ext (toC_ofC C hInter hne X.1 X.2)
  map_rel_iff' := by
    intro a b
    show toC C hInter hne a ⊆ toC C hInter hne b ↔ a ≤ b
    constructor
    · intro hsub N hN
      have hmemD : (reprSystem C hInter hne).mem N := a.sub hN
      obtain ⟨G, rfl⟩ := hmemD
      have hGa : G.1 ⊆ toC C hInter hne a := (mem_nbhd_iff C hInter hne a G).mp hN
      exact (mem_nbhd_iff C hInter hne b G).mpr (hGa.trans hsub)
    · intro hab t ht
      rw [mem_toC] at ht ⊢
      obtain ⟨F, hF, htCl⟩ := ht
      exact ⟨F, hab _ hF, htCl⟩

end

end DomainTheory.NeighbourhoodSystems.ReprSystem
```

4.3.43 Exercise 2.22 — domains from closure properties

(For set theorists). We have remarked in Exercise 1.18 and in Exercise 2.11 that any domain $|D|$, as a family of sets (in fact, a family of subsets of the set D itself), is closed under the intersection of an arbitrary non-empty sub family and under the union of any directed sub family. For those familiar with the subject matter, the example of the (proper) ideals of a commutative ring (with unit) is also seen to be such a family. What is the abstract situation? Let $\mathbf{C}$ be *any* family of sets with these closure properties. It is to be shown that $\mathbf{C}$ is inclusion-isomorphic to a domain. (Hint: Let Δ be the set of finite sets included in sets in $\mathbf{C}$. For $F \in \Delta$, define its "closure" by the equation:

$$\overline{F} = \bigcap \{X \in \mathbf{C} \mid F \subseteq X\}.$$

Every $\overline{F} \in \mathbf{C}$, and these will prove to be the "finite" elements of $\mathbf{C}$. The neighbourhood system $\mathcal{D}$ over Δ can be taken to be the sets of the form

$$C(F) = \{G \in \Delta \mid F \subseteq \overline{G}\}$$

for $F \in \Delta$. Notice that for all $X \in \mathbf{C}$

$$X = \bigcup \{\overline{F} \mid F \subseteq X \text{ and } F \in \Delta\}.$$

)

Check that approximable functions on these families are just those preserving directed unions.

Theorem (`example_222`). A nonempty family of sets closed under nonempty intersections and directed unions is inclusion-order-isomorphic to the domain of ideal elements of its token neighbourhood system.

Proof. The representation order isomorphism. ■

`Exercise_2_22.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.reprSystem

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Set
open ReprSystem

/-- Scott's Exercise 2.22: a family closed under nonempty intersections and directed
    unions is
    inclusion-isomorphic to the domain of its token neighbourhood system. -/
theorem example_222 {τ : Type*} (C : Set (Set τ))
    (hInter : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → ⋂₀ ? ∈ C)
    (hne : C.Nonempty)
    (hDir : ∀ ? : Set (Set τ), ?.Nonempty → ? ⊆ C → DirectedOn (· ⊆ ·) ? → ⋃₀ ? ∈ C)
    :
    Nonempty ((reprSystem C hInter hne).Element ≃o {X : Set τ // X ∈ C}) :=
  ⟨reprIso C hInter hne hDir⟩
```

4.4 Domain constructs

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

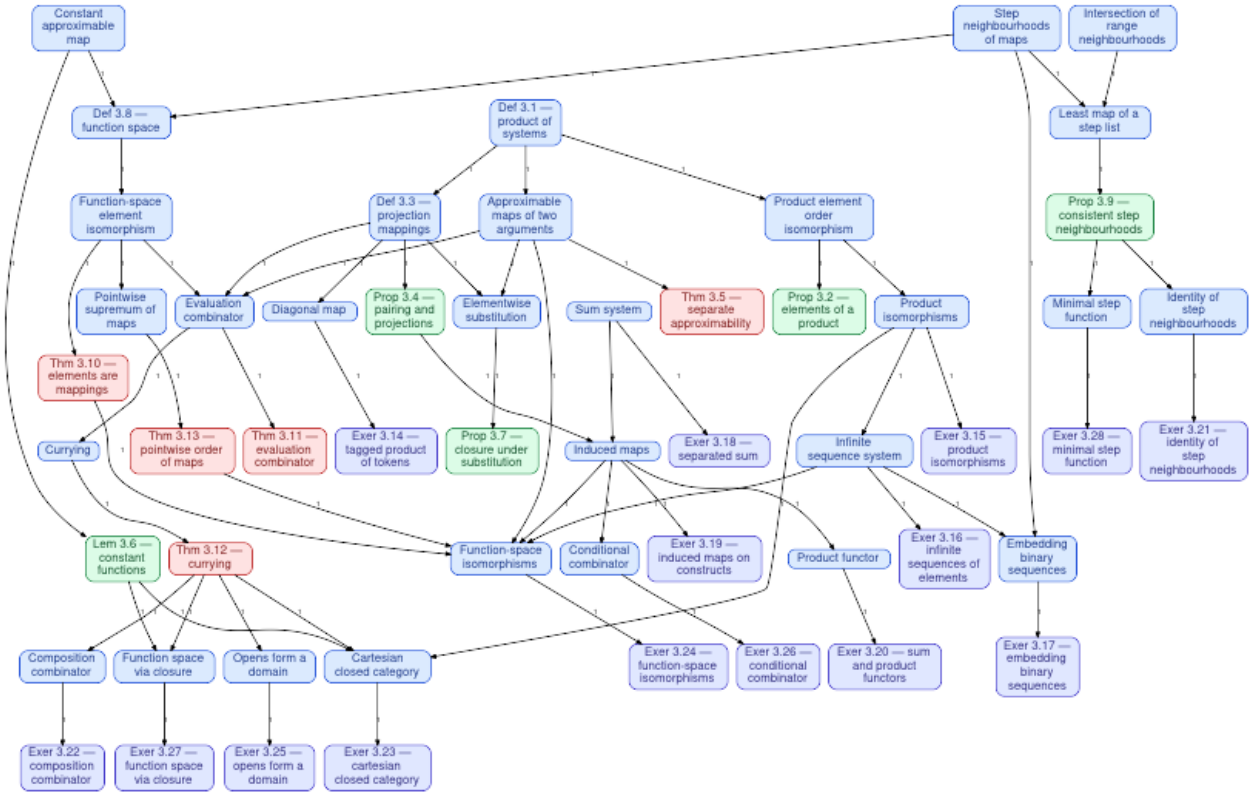


Figure 48: Diagram 13

4.4.1 Introduction

LECTURE III

DOMAIN CONSTRUCTS

Having now seen a number of domains presented through their neighbourhood systems, we need next to introduce general constructs for forming new domains from old. There are an unlimited number of such constructs (technically called *functors*), but we have time only to single out a few of the more important ones. Outstanding among all of them is the notion of product of systems, which in our chosen category has all the expected properties. For the time being in order to simplify notation we assume of the underlying sets Δ_0 and Δ_1 of systems $\mathcal{D}_0$ and $\mathcal{D}_1$ that they are disjoint. There is no loss of generality as $\mathcal{D}_1$ can always be replaced by an isomorphic system disjoint from $\mathcal{D}_0$ in the required sense.

4.4.2 Definition 3.1 — product of systems

DEFINITION 3.1. Let neighbourhood systems $\mathcal{D}_0$ and $\mathcal{D}_1$ be given over disjoint sets Δ_0 and Δ_1. The *product system* over $\Delta_0 \cup \Delta_1$ is defined by:

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{X \cup Y \mid X \in \mathcal{D}_0 \text{ and } Y \in \mathcal{D}_1\}.$$

For elements $x \in |\mathcal{D}_0|$ and $y \in |\mathcal{D}_1|$ we also define:

$$\langle x, y \rangle = \{X \cup Y \mid X \in x \text{ and } Y \in y\}.$$

□

Definition (`prodNbhd`). The product of two sets of tokens is the disjoint union of their tagged copies in the sum type.

Theorem (`prodNbhd.mem_inl`). A left-tagged token belongs to a product neighbourhood if and only if the untagged token belongs to the left factor.

Proof. Unpack the image-union definition. ■

Theorem (`prodNbhd.mem_inr`). A right-tagged token belongs to a product neighbourhood if and only if the untagged token belongs to the right factor.

Proof. Unpack the image-union definition. ■

Theorem (`prodNbhd.inl_preimage`). The left untagging of a product neighbourhood recovers the left factor.

Proof. By the left-membership characterisation. ■

Theorem (`prodNbhd.inr_preimage`). The right untagging of a product neighbourhood recovers the right factor.

Proof. By the right-membership characterisation. ■

Theorem (`prodNbhd.injective`). Equal product neighbourhoods have equal factors.

Proof. Apply left and right untagging. ■

Theorem (`prodNbhd.inter`). Intersection of product neighbourhoods is the product of the intersections of the factors.

Proof. Case analysis on left versus right tags. ■

Theorem (`prodNbhd.subset_iff`). Product neighbourhoods nest if and only if both factors nest.

Proof. One direction evaluates at tagged tokens; the other uses monotonicity of the images. ■

Definition (`prod`). The product of two neighbourhood systems has as neighbourhoods the products of neighbourhoods from each factor, with master the product of the masters.

Proof. Master nonempty and membership follow from the left factor; intersection and containment in the master use the product intersection and nesting laws. ■

Theorem (`prod_mem_iff`). A set is a product neighbourhood if and only if it is the product of neighbourhoods from each factor.

Proof. By definition. ■

Theorem (`prod_master`). The master of a product system is the product of the masters.

Proof. By definition. ■

Theorem (`prod_mem_prodNbhd`). The product of two neighbourhoods is a neighbourhood of the product system.

Proof. Immediate from the membership characterisation. ■

Theorem (`prod_mem_prodNbhd_iff`). A product of sets is a product neighbourhood if and only if both factors are neighbourhoods.

Proof. Injectivity of the product of sets recovers the factors from any witnessing decomposition. ■

Theorem (`prodNbhd.preimage`). Every neighbourhood of a product system equals the product of its left and right untaggings.

Proof. Unpack the witnessing factors and apply the untagging identities. ■

Definition (`NeighborhoodSystem.Element.fst`). The first projection of a product ideal element consists of those left neighbourhoods whose product with the right master belongs to the element.

Proof. Filter axioms follow from intersection and nesting of product neighbourhoods. ■

Theorem (`mem_fst`). Membership in the first projection is neighbourhood membership of the left factor together with membership of its product with the right master.

Proof. By definition. ■

Definition (`NeighborhoodSystem.Element.snd`). The second projection of a product ideal element consists of those right neighbourhoods whose product with the left master belongs to the element.

Proof. Dual to the first projection. ■

Theorem (`mem_snd`). Membership in the second projection is neighbourhood membership of the right factor together with membership of its product with the left master.

Proof. By definition. ■

Definition (`pair`). The pairing of two ideal elements consists of those product neighbourhoods whose factors belong to the respective elements.

Proof. Filter axioms follow from the corresponding operations on each factor and on product neighbourhoods. ■

Theorem (`mem_pair`). Membership in a pairing is witnessed by factors belonging to each argument.

Proof. By definition. ■

Theorem (`mem_pair_prodNbhd`). A product of sets belongs to a pairing if and only if each factor belongs to the corresponding argument.

Proof. Injectivity of the product of sets. ■

Theorem (`fst_pair`). The first projection of a pairing recovers the left argument.

Proof. Compare via the product-membership characterisation of the pairing. ■

Theorem (`prod_mem_split`). Membership of a product neighbourhood in an ideal element splits as membership of the two products against the opposite masters.

Proof. One direction uses upward closure toward the masters; the converse intersects and recovers the factors. ■

Theorem (`pair_fst_snd`). Pairing the two projections of a product ideal element recovers the element.

Proof. Compare via the splitting law. ■

Theorem (`snd_pair`). The second projection of a pairing recovers the right argument.

Proof. Dual to the first-projection recovery. ■

Theorem (`prod_mem_inl`). The left untagging of a product neighbourhood is a neighbourhood of the left system.

Proof. Unpack the witnessing factors. ■

Theorem (`prod_mem_inr`). The right untagging of a product neighbourhood is a neighbourhood of the right system.

Proof. Unpack the witnessing factors. ■

`prodNbhd.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
import Mathlib.Data.Set.Basic
import Mathlib.Data.Set.Image
public import Mathlib.Data.Set.Operations

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
```

```
variable {α β ? ? : Type*}

def prodNbhd (X : Set α) (Y : Set β) : Set (α ? β) := Sum.inl '' X ∪ Sum.inr '' Y

namespace prodNbhd

@[simp] theorem mem_inl {X : Set α} {Y : Set β} {a : α} :
    (Sum.inl a : α ? β) ∈ prodNbhd X Y ↔ a ∈ X := by
  simp [prodNbhd]

@[simp] theorem mem_inr {X : Set α} {Y : Set β} {b : β} :
    (Sum.inr b : α ? β) ∈ prodNbhd X Y ↔ b ∈ Y := by
  simp [prodNbhd]

@[simp] theorem inl_preimage (X : Set α) (Y : Set β) :
    Sum.inl ^-1' prodNbhd X Y = X := by ext a; simp

@[simp] theorem inr_preimage (X : Set α) (Y : Set β) :
    Sum.inr ^-1' prodNbhd X Y = Y := by ext b; simp

theorem injective {X X' : Set α} {Y Y' : Set β}
    (h : prodNbhd X Y = prodNbhd X' Y') : X = X' ∧ Y = Y' := by
  refine ⟨?_, ?_⟩
  · rw [← inl_preimage X Y, ← inl_preimage X' Y', h]
  · rw [← inr_preimage X Y, ← inr_preimage X' Y', h]

theorem inter (X X' : Set α) (Y Y' : Set β) :
    prodNbhd X Y ∩ prodNbhd X' Y' = prodNbhd (X ∩ X') (Y ∩ Y') := by
  ext (a | b) <;> simp [Set.mem_inter_iff]

theorem subset_iff {X X' : Set α} {Y Y' : Set β} :
    prodNbhd X Y ⊆ prodNbhd X' Y' ↔ X ⊆ X' ∧ Y ⊆ Y' := by
  constructor
  · intro h
    refine ⟨fun a ha => ?_, fun b hb => ?_⟩
    · have : (Sum.inl a : α ? β) ∈ prodNbhd X' Y' := h (by simpa using ha)
      simpa using this
    · have : (Sum.inr b : α ? β) ∈ prodNbhd X' Y' := h (by simpa using hb)
      simpa using this
  · rintro ⟨hX, hY⟩ (a | b) hs
    · simp only [mem_inl] at hs ⊢; exact hX hs
    · simp only [mem_inr] at hs ⊢; exact hY hs

end prodNbhd

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def prod (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) : NeighborhoodSystem (
    α ? β) where
  mem W := ∃ X Y, V₀.mem X ∧ V₁.mem Y ∧ W = prodNbhd X Y
  master := prodNbhd V₀.master V₁.master
  master_nonempty := by
    obtain ⟨a, ha⟩ := V₀.master_nonempty
    exact ⟨Sum.inl a, prodNbhd.mem_inl.mpr ha⟩
  master_mem := ⟨V₀.master, V₁.master, V₀.master_mem, V₁.master_mem, rfl⟩
  inter_mem := by
```

```
    rintro W W' Z ⟨X, Y, hX, hY, rfl⟩ ⟨X', Y', hX', hY', rfl⟩ ⟨Z₀, Z₁, hZ₀, hZ₁, rfl⟩
    hsub
    rw [prodNbhd.inter] at hsub ⊢
    obtain ⟨hsub₀, hsub₁⟩ := prodNbhd.subset_iff.mp hsub
    exact ⟨X ∩ X', Y ∩ Y', V₀.inter_mem hX hX' hZ₀ hsub₀, V₁.inter_mem hY hY' hZ₁ hsub₁,
     rfl⟩
  sub_master := by
    rintro W ⟨X, Y, hX, hY, rfl⟩
    exact prodNbhd.subset_iff.mpr ⟨V₀.sub_master hX, V₁.sub_master hY⟩

@[simp] theorem prod_mem_iff {W : Set (α ? β)} :
    (prod V₀ V₁).mem W ↔ ∃ X Y, V₀.mem X ∧ V₁.mem Y ∧ W = prodNbhd X Y := Iff.rfl

@[simp] theorem prod_master : (prod V₀ V₁).master = prodNbhd V₀.master V₁.master := rfl

theorem prod_mem_prodNbhd {X : Set α} {Y : Set β} (hX : V₀.mem X) (hY : V₁.mem Y) :
    (prod V₀ V₁).mem (prodNbhd X Y) := ⟨X, Y, hX, hY, rfl⟩

theorem prod_mem_prodNbhd_iff {X : Set α} {Y : Set β} :
    (prod V₀ V₁).mem (prodNbhd X Y) ↔ V₀.mem X ∧ V₁.mem Y := by
  constructor
  · rintro ⟨A, B, hA, hB, heq⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact ⟨hA, hB⟩
  · rintro ⟨hX, hY⟩; exact prod_mem_prodNbhd hX hY

namespace prodNbhd

theorem preimage {W : Set (α ? β)} (hW : (prod V₀ V₁).mem W) :
    W = prodNbhd (Sum.inl ^-1' W) (Sum.inr ^-1' W) := by
  obtain ⟨X, Y, _, _, rfl⟩ := hW
  rw [inl_preimage, inr_preimage]

end prodNbhd

def NeighborhoodSystem.Element.fst (z : (prod V₀ V₁).Element) : V₀.Element where
  mem X := V₀.mem X ∧ z.mem (prodNbhd X V₁.master)
  sub h := h.1
  master_mem := ⟨V₀.master_mem, z.master_mem⟩
  inter_mem := by
    rintro X X' ⟨_, hzX⟩ ⟨_, hzX'⟩
    have hz := z.inter_mem hzX hzX'
    rw [prodNbhd.inter, Set.inter_self] at hz
    obtain ⟨A, B, hA, _, heq⟩ := z.sub hz
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact ⟨hA, hz⟩
  up_mem := by
    rintro X X' ⟨_, hzX⟩ hX' hXX'
    refine ⟨hX', z.up_mem hzX (prod_mem_prodNbhd hX' V₁.master_mem) ?_⟩
    exact prodNbhd.subset_iff.mpr ⟨hXX', subset_rfl⟩

@[simp] theorem mem_fst {z : (prod V₀ V₁).Element} {X : Set α} :
    z.fst.mem X ↔ V₀.mem X ∧ z.mem (prodNbhd X V₁.master) := Iff.rfl

def NeighborhoodSystem.Element.snd (z : (prod V₀ V₁).Element) : V₁.Element where
  mem Y := V₁.mem Y ∧ z.mem (prodNbhd V₀.master Y)
```

```
  sub h := h.1
  master_mem := ⟨V₁.master_mem, z.master_mem⟩
  inter_mem := by
    rintro Y Y' ⟨_, hzY⟩ ⟨_, hzY'⟩
    have hz := z.inter_mem hzY hzY'
    rw [prodNbhd.inter, Set.inter_self] at hz
    obtain ⟨A, B, _, hB, heq⟩ := z.sub hz
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact ⟨hB, hz⟩
  up_mem := by
    rintro Y Y' ⟨_, hzY⟩ hY' hYY'
    refine ⟨hY', z.up_mem hzY (prod_mem_prodNbhd V₀.master_mem hY') ?_⟩
    exact prodNbhd.subset_iff.mpr ⟨subset_rfl, hYY'⟩

@[simp] theorem mem_snd {z : (prod V₀ V₁).Element} {Y : Set β} :
    z.snd.mem Y ↔ V₁.mem Y ∧ z.mem (prodNbhd V₀.master Y) := Iff.rfl

def pair (x : V₀.Element) (y : V₁.Element) : (prod V₀ V₁).Element where
  mem W := ∃ X Y, x.mem X ∧ y.mem Y ∧ W = prodNbhd X Y
  sub := by rintro W ⟨X, Y, hX, hY, rfl⟩; exact prod_mem_prodNbhd (x.sub hX) (y.sub hY)
  master_mem := ⟨V₀.master, V₁.master, x.master_mem, y.master_mem, rfl⟩
  inter_mem := by
    rintro W W' ⟨X, Y, hX, hY, rfl⟩ ⟨X', Y', hX', hY', rfl⟩
    exact ⟨X ∩ X', Y ∩ Y', x.inter_mem hX hX', y.inter_mem hY hY', prodNbhd.inter X X'
    Y Y'⟩
  up_mem := by
    rintro W W' ⟨X, Y, hX, hY, rfl⟩ ⟨X', Y', hX', hY', rfl⟩ hsub
    obtain ⟨hXX', hYY'⟩ := prodNbhd.subset_iff.mp hsub
    exact ⟨X', Y', x.up_mem hX hX' hXX', y.up_mem hY hY' hYY', rfl⟩

@[simp] theorem mem_pair {x : V₀.Element} {y : V₁.Element} {W : Set (α ? β)} :
    (pair x y).mem W ↔ ∃ X Y, x.mem X ∧ y.mem Y ∧ W = prodNbhd X Y := Iff.rfl

theorem mem_pair_prodNbhd {x : V₀.Element} {y : V₁.Element} {X : Set α} {Y : Set β} :
    (pair x y).mem (prodNbhd X Y) ↔ x.mem X ∧ y.mem Y := by
  constructor
  · rintro ⟨X', Y', hX', hY', heq⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact ⟨hX', hY'⟩
  · rintro ⟨hx, hy⟩; exact ⟨X, Y, hx, hy, rfl⟩

@[simp] theorem fst_pair (x : V₀.Element) (y : V₁.Element) : (pair x y).fst = x := by
  apply Element.ext
  intro X
  constructor
  · rintro ⟨hX, hmem⟩
    exact (mem_pair_prodNbhd.mp hmem).1
  · intro hX
    exact ⟨x.sub hX, mem_pair_prodNbhd.mpr ⟨hX, y.master_mem⟩⟩

theorem prod_mem_split {z : (prod V₀ V₁).Element} {X : Set α} {Y : Set β}
    (hX : V₀.mem X) (hY : V₁.mem Y) :
    z.mem (prodNbhd X Y) ↔ z.mem (prodNbhd X V₁.master) ∧ z.mem (prodNbhd V₀.master Y)
     := by
  constructor
  · intro h
```

```
    refine ⟨z.up_mem h (prod_mem_prodNbhd hX V₁.master_mem) ?_,
            z.up_mem h (prod_mem_prodNbhd V₀.master_mem hY) ?_⟩
    · exact prodNbhd.subset_iff.mpr ⟨subset_rfl, V₁.sub_master hY⟩
    · exact prodNbhd.subset_iff.mpr ⟨V₀.sub_master hX, subset_rfl⟩
  · rintro ⟨h1, h2⟩
    have := z.inter_mem h1 h2
    rwa [prodNbhd.inter, Set.inter_eq_left.mpr (V₀.sub_master hX),
      Set.inter_eq_right.mpr (V₁.sub_master hY)] at this

theorem pair_fst_snd (z : (prod V₀ V₁).Element) : pair z.fst z.snd = z := by
  apply Element.ext
  intro W
  constructor
  · rintro ⟨X, Y, ⟨hX, hzX⟩, ⟨hY, hzY⟩, rfl⟩
    exact (prod_mem_split hX hY).mpr ⟨hzX, hzY⟩
  · intro hW
    obtain ⟨X, Y, hX, hY, rfl⟩ := z.sub hW
    obtain ⟨h1, h2⟩ := (prod_mem_split hX hY).mp hW
    exact ⟨X, Y, ⟨hX, h1⟩, ⟨hY, h2⟩, rfl⟩

@[simp] theorem snd_pair (x : V₀.Element) (y : V₁.Element) : (pair x y).snd = y := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨hY, hmem⟩
    exact (mem_pair_prodNbhd.mp hmem).2
  · intro hY
    exact ⟨y.sub hY, mem_pair_prodNbhd.mpr ⟨x.master_mem, hY⟩⟩

theorem prod_mem_inl {W : Set (α ? β)} (hW : (prod V₀ V₁).mem W) : V₀.mem (Sum.inl ^-1'
    W) := by
  obtain ⟨X, Y, hX, _, rfl⟩ := hW; rwa [prodNbhd.inl_preimage]

theorem prod_mem_inr {W : Set (α ? β)} (hW : (prod V₀ V₁).mem W) : V₁.mem (Sum.inr ^-1'
    W) := by
  obtain ⟨X, Y, _, hY, rfl⟩ := hW; rwa [prodNbhd.inr_preimage]

end DomainTheory.NeighbourhoodSystems
```

4.4.3 Product element order isomorphism

Order isomorphism between product ideal elements and pairs of factor ideal elements.

Definition (`prodEquiv`). Ideal elements of a product neighbourhood system are order-isomorphic to pairs of ideal elements of the factors: forward projection sends an element to its two projections, and the inverse pairs them again.

Proof. The round-trip identities are the pairing–projection recovery laws; the order equivalence unpacks membership of product neighbourhoods via the splitting characterisation. ■

Theorem (`prodEquiv.apply`). The isomorphism sends a product ideal element to the pair of its projections.

Proof. By definition. ■

Theorem (`prodEquiv.symm_apply`). The inverse isomorphism sends a pair of ideal elements to their pairing.

Proof. By definition. ■

Theorem (`pair_le_pair_iff`). A pairing approximates another if and only if each factor approximates the corresponding factor.

Proof. Forward, evaluate at products against opposite masters and recover the factors by injectivity; reverse, apply the two inequalities to the witnessing factors of a product neighbourhood. ■

`prodEquiv.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodNbhd
public import Mathlib.Order.Hom.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

variable {α β : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def prodEquiv (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    (prod V₀ V₁).Element ≃o V₀.Element × V₁.Element where
  toFun z := (z.fst, z.snd)
  invFun p := pair p.1 p.2
  left_inv z := pair_fst_snd z
  right_inv p := by simp
  map_rel_iff' := by
    intro z z'
    constructor
    · rintro ⟨h1, h2⟩ W hW
      obtain ⟨X, Y, hX, hY, rfl⟩ := z.sub hW
      obtain ⟨ha, hb⟩ := (prod_mem_split hX hY).mp hW
      have hX' : z'.fst.mem X := h1 X ⟨hX, ha⟩
      have hY' : z'.snd.mem Y := h2 Y ⟨hY, hb⟩
      exact (prod_mem_split hX hY).mpr ⟨hX'.2, hY'.2⟩
    · intro h
      exact ⟨fun X ⟨hX, hzX⟩ => ⟨hX, h _ hzX⟩, fun Y ⟨hY, hzY⟩ => ⟨hY, h _ hzY⟩⟩

namespace prodEquiv

@[simp] theorem apply (z : (prod V₀ V₁).Element) :
    prodEquiv V₀ V₁ z = (z.fst, z.snd) := rfl

@[simp] theorem symm_apply (p : V₀.Element × V₁.Element) :
    (prodEquiv V₀ V₁).symm p = pair p.1 p.2 := rfl

end prodEquiv

theorem pair_le_pair_iff {x x' : V₀.Element} {y y' : V₁.Element} :
    pair x y ≤ pair x' y' ↔ x ≤ x' ∧ y ≤ y' := by
  constructor
  · intro h
    refine ⟨fun X hX => ?_, fun Y hY => ?_⟩
    · obtain ⟨X', Y', hX', hY', heq⟩ :=
```

```
        h (prodNbhd X V₁.master) ⟨X, V₁.master, hX, y.master_mem, rfl⟩
      obtain ⟨rfl, _⟩ := prodNbhd.injective heq
      exact hX'
    · obtain ⟨X', Y', hX', hY', heq⟩ :=
        h (prodNbhd V₀.master Y) ⟨V₀.master, Y, x.master_mem, hY, rfl⟩
      obtain ⟨_, rfl⟩ := prodNbhd.injective heq
      exact hY'
  · rintro ⟨hx, hy⟩ W ⟨X, Y, hX, hY, rfl⟩
    exact ⟨X, Y, hx X hX, hy Y hY, rfl⟩

end DomainTheory.NeighbourhoodSystems
```

4.4.4 Proposition 3.2 — elements of a product

PROPOSITION 3.2. The construct $\mathcal{D}_0 \times \mathcal{D}_1$ always gives a neighbourhood system where for elements $x, x' \in |\mathcal{D}_0|$ and $y, y' \in |\mathcal{D}_1|$ we have

(i) $\langle x, y\rangle \subseteq \langle x', y'\rangle$ iff $x \subseteq x'$ and $y \subseteq y'$.

Moreover, there is a one-one correspondence between the elements of $|\mathcal{D}_0 \times \mathcal{D}_1|$ and pairs of elements of $|\mathcal{D}_0|$ and $|\mathcal{D}_1|$ since all elements of $|\mathcal{D}_0 \times \mathcal{D}_1|$ are of the form $\langle x, y\rangle$.

Proof: Owing to the disjointness of Δ_0 and Δ_1, we note that for $X, X' \in \mathcal{D}_0$ and $Y, Y' \in \mathcal{D}_1$ we have

$$(1)\ X \cup Y \subseteq X' \cup Y' \text{ iff } X \subseteq X' \text{ and } Y \subseteq Y'.$$

Thus, $\{X \cup Y, X' \cup Y'\}$ is consistent in $\mathcal{D}_0 \times \mathcal{D}_1$ iff $\{X, X'\}$ is consistent in $\mathcal{D}_0$ and $\{Y, Y'\}$ is consistent in $\mathcal{D}_1$. In the consistent case we find

$$(2) \qquad (X \cup Y) \cap (X' \cup Y') = (X \cap X') \cup (Y \cap Y'),$$

and so $\mathcal{D}_0 \times \mathcal{D}_1$ is closed under consistent intersection. As $\Delta_0 \cup \Delta_1 \in \mathcal{D}_0 \times \mathcal{D}_1$, it is certainly a neighbourhood system.

It is easy to check by the previous calculations that $\langle x, y\rangle \in |\mathcal{D}_0 \times \mathcal{D}_1|$ if $x \in |\mathcal{D}_0|$ and $y \in |\mathcal{D}_1|$. The proof of 3.2(i) follows directly from the definition and (1).

Suppose $z \in |\mathcal{D}_0 \times \mathcal{D}_1|$. Define as a temporary notation:

$$z_0 = \{X \in \mathcal{D}_0 \mid X \cup \Delta_1 \in z\}, \quad \text{and} \quad z_1 = \{Y \in \mathcal{D}_1 \mid \Delta_0 \cup Y \in z\}.$$

Clearly, both $z_0 \in |\mathcal{D}_0|$ and $z_1 \in |\mathcal{D}_1|$. In view of the formula

$$(3) \qquad (X \cup \Delta_1) \cap (\Delta_0 \cup Y) = X \cup Y,$$

we can calculate that

$$z = \langle z_0, z_1\rangle.$$

Moreover, if $z = \langle x, y\rangle$, then $\langle x, y\rangle_0 = x$ and $\langle x, y\rangle_1 = y$.

The one-one correspondence required is thus established. □

There is more going on in the proof of 3.2 than just a one-one correspondence between elements and pairs. The extra information is best formalized by introducing a notation for mappings.

Theorem (`proposition_3_2`). Pairing is order-equivalent to the product of the factor orders, and every product system admits an order isomorphism between its ideal elements and pairs of factor ideal elements.

Proof. Assemble the pairing order law with the product element order isomorphism. ■

`Proposition_3_2.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodEquiv

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

/-- Scott's Proposition 3.2: pairing is monotone and order-reflecting in each factor,
    and every
    product ideal element is uniquely a pairing of its projections. -/
theorem proposition_3_2 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {x x' : V₀.Element} {y y' : V₁.Element},
      pair x y ≤ pair x' y' ↔ x ≤ x' ∧ y ≤ y') ∧
    (∀ {α β : Type*} (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β),
      Nonempty ((prod V₀ V₁).Element ≃o V₀.Element × V₁.Element)) :=
  ⟨@pair_le_pair_iff, fun {_ _} V₀ V₁ => ⟨prodEquiv V₀ V₁⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.4.5 Definition 3.3 — projection mappings

DEFINITION 3.3. *Projection mappings*

$$p_0 : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_0 \quad \text{and} \quad p_1 : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_1$$

are defined as relations where

$$(X \cup Y)\ p_0\ X' \text{ iff } X \subseteq X', \quad \text{and} \quad (X \cup Y)\ p_1\ Y' \text{ iff } Y \subseteq Y'$$

hold for all $X, X' \in \mathcal{D}_0$ and $Y, Y' \in \mathcal{D}_1$. Given $f : \mathcal{D}_2 \to \mathcal{D}_0$ and $g : \mathcal{D}_2 \to \mathcal{D}_1$, the *paired mapping*

$$\langle f, g \rangle : \mathcal{D}_2 \to \mathcal{D}_0 \times \mathcal{D}_1$$

is defined as a relation where

$$Z\ \langle f, g \rangle\ (X \cup Y) \text{ iff } Z\ f\ X \text{ and } Z\ g\ Y$$

holds for all $X \in \mathcal{D}_0$, $Y \in \mathcal{D}_1$, and $Z \in \mathcal{D}_2$. □

Definition (`paired`). The pairing of two approximable maps into the factors of a product is the approximable map into the product that relates a neighbourhood to a product neighbourhood precisely when each component map relates it to the corresponding untagged factor.

Proof. Domain and codomain membership, master, intersection on the right, and monotonicity follow from the corresponding axioms of the components together with the product neighbourhood laws. ■

Definition (`proj0`). The left projection from a product system relates a product neighbourhood to a left neighbourhood precisely when the left untagging of the product neighbourhood is contained in that neighbourhood.

Proof. Approximable-map axioms follow from the product neighbourhood structure and left untagging. ■

Theorem (`proj0_rel`). Membership in the left-projection relation is product membership, left neighbourhood membership, and containment of the left untagging.

Proof. By definition. ■

Definition (`proj1`). The right projection from a product system relates a product neighbourhood to a right neighbourhood precisely when the right untagging of the product neighbourhood is contained in that neighbourhood.

Proof. Dual to the left projection. ■

Theorem (`proj1_rel`). Membership in the right-projection relation is product membership, right neighbourhood membership, and containment of the right untagging.

Proof. By definition. ■

Theorem (`paired_rel`). Membership in the paired-map relation is product membership together with the two component relations on the untagged factors.

Proof. By definition. ■

Theorem (`toElementMap_proj0`). The elementwise action of the left projection recovers the first projection of a product ideal element.

Proof. Compare membership via upward closure toward products with the right master, and conversely evaluate at that product. ■

Theorem (`toElementMap_proj1`). The elementwise action of the right projection recovers the second projection of a product ideal element.

Proof. Dual to the left projection. ■

Theorem (`toElementMap_proj0_pair`). The left projection sends a pairing of ideal elements to the left argument.

Proof. Compose the elementwise projection law with recovery of the left factor from a pairing. ■

Theorem (`toElementMap_proj1_pair`). The right projection sends a pairing of ideal elements to the right argument.

Proof. Dual to the left case. ■

Theorem (`toElementMap_paired`). The elementwise action of a paired map is the pairing of the elementwise actions of the components.

Proof. Compare membership by unpacking and reconstructing product neighbourhoods from the two component witnesses. ■

Theorem (`proj0_comp_paired`). Composing the left projection with a paired map recovers the left component.

Proof. Compare elementwise actions using the paired and projection laws. ■

Theorem (`proj1_comp_paired`). Composing the right projection with a paired map recovers the right component.

Proof. Dual to the left case. ■

Theorem (`paired_proj`). Pairing the two projections composed with a map into a product recovers that map.

Proof. Compare elementwise actions using the preceding laws and the pairing–projection recovery. ■

ProjectionMappings.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodNbhd
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Image
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
variable {α β ? ? : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {V₂ : NeighborhoodSystem ?}
open ApproximableMap

def paired (f : ApproximableMap V₂ V₀) (g : ApproximableMap V₂ V₁) :
    ApproximableMap V₂ (prod V₀ V₁) where
  rel Z P := (prod V₀ V₁).mem P ∧ f.rel Z (Sum.inl ^-1' P) ∧ g.rel Z (Sum.inr ^-1' P)
  rel_dom h := f.rel_dom h.2.1
  rel_cod h := h.1
  master_rel := by
    refine ⟨(prod V₀ V₁).master_mem, ?_, ?_⟩
    · simpa using f.master_rel
    · simpa using g.master_rel
  inter_right := by
    rintro Z P P' ⟨hP, hfP, hgP⟩ ⟨hP', hfP', hgP'⟩
    have hfX := f.inter_right hfP hfP'
    have hgY := g.inter_right hgP hgP'
    refine ⟨?_, ?_, ?_⟩
    · rw [prodNbhd.preimage hP, prodNbhd.preimage hP', prodNbhd.inter]
      exact prod_mem_prodNbhd (f.rel_cod hfX) (g.rel_cod hgY)
    · rw [Set.preimage_inter]; exact hfX
    · rw [Set.preimage_inter]; exact hgY
  mono := by
    rintro Z Z₂ P P₂ ⟨_, hfP, hgP⟩ hZ₂Z hPP₂ hZ₂ hP₂
    obtain ⟨A, B, hA, hB, rfl⟩ := hP₂
    have hinl : Sum.inl ^-1' P ⊆ A := by
      have := Set.preimage_mono (f := Sum.inl) hPP₂; rwa [prodNbhd.inl_preimage] at
    this
    have hinr : Sum.inr ^-1' P ⊆ B := by
      have := Set.preimage_mono (f := Sum.inr) hPP₂; rwa [prodNbhd.inr_preimage] at
    this
    refine ⟨⟨A, B, hA, hB, rfl⟩, ?_, ?_⟩
    · rw [prodNbhd.inl_preimage]; exact f.mono hfP hZ₂Z hinl hZ₂ hA
    · rw [prodNbhd.inr_preimage]; exact g.mono hgP hZ₂Z hinr hZ₂ hB

def proj₀ (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    ApproximableMap (prod V₀ V₁) V₀ where
  rel W X' := (prod V₀ V₁).mem W ∧ V₀.mem X' ∧ Sum.inl ^-1' W ⊆ X'
```

```
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(prod V₀ V₁).master_mem, V₀.master_mem, by simp⟩
  inter_right := by
    rintro W X' X'' ⟨hW, hX', hsub⟩ ⟨_, hX'', hsub'⟩
    obtain ⟨A, B, hA, _, rfl⟩ := hW
    rw [prodNbhd.inl_preimage] at hsub hsub' ⊢
    exact ⟨⟨A, B, hA, ?_?, rfl⟩, V₀.inter_mem hX' hX'' hA (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'⟩
  mono := by
    rintro W W₂ X' X₂' ⟨_, _, hsub⟩ hW₂W hX'X₂' hW₂ hX₂'
    exact ⟨hW₂, hX₂', ((Set.preimage_mono hW₂W).trans hsub).trans hX'X₂'⟩

@[simp] theorem proj₀_rel {W : Set (α ? β)} {X' : Set α} :
    (proj₀ V₀ V₁).rel W X' ↔ (prod V₀ V₁).mem W ∧ V₀.mem X' ∧ Sum.inl ^-1' W ⊆ X' :=
    Iff.rfl

def proj₁ (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    ApproximableMap (prod V₀ V₁) V₁ where
  rel W Y' := (prod V₀ V₁).mem W ∧ V₁.mem Y' ∧ Sum.inr ^-1' W ⊆ Y'
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(prod V₀ V₁).master_mem, V₁.master_mem, by simp⟩
  inter_right := by
    rintro W Y' Y'' ⟨hW, hY', hsub⟩ ⟨_, hY'', hsub'⟩
    obtain ⟨A, B, _, hB, rfl⟩ := hW
    rw [prodNbhd.inr_preimage] at hsub hsub' ⊢
    exact ⟨⟨A, B, ?_?, hB, rfl⟩, V₁.inter_mem hY' hY'' hB (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'⟩
  mono := by
    rintro W W₂ Y' Y₂' ⟨_, _, hsub⟩ hW₂W hY'Y₂' hW₂ hY₂'
    exact ⟨hW₂, hY₂', ((Set.preimage_mono hW₂W).trans hsub).trans hY'Y₂'⟩

@[simp] theorem proj₁_rel {W : Set (α ? β)} {Y' : Set β} :
    (proj₁ V₀ V₁).rel W Y' ↔ (prod V₀ V₁).mem W ∧ V₁.mem Y' ∧ Sum.inr ^-1' W ⊆ Y' :=
    Iff.rfl

@[simp] theorem paired_rel {f : ApproximableMap V₂ V₀} {g : ApproximableMap V₂ V₁}
    {Z : Set ?} {P : Set (α ? β)} :
    (paired f g).rel Z P ↔
      (prod V₀ V₁).mem P ∧ f.rel Z (Sum.inl ^-1' P) ∧ g.rel Z (Sum.inr ^-1' P) := Iff.
    rfl

@[simp] theorem toElementMap_proj₀ (z : (prod V₀ V₁).Element) :
    (proj₀ V₀ V₁).toElementMap z = z.fst := by
  apply Element.ext
  intro X'
  constructor
  · rintro ⟨W, hzW, hW, hX', hsub⟩
    obtain ⟨A, B, _, hB, rfl⟩ := hW
    rw [prodNbhd.inl_preimage] at hsub
    refine ⟨hX', z.up_mem hzW (prod_mem_prodNbhd hX' V₁.master_mem) ?_⟩
    exact prodNbhd.subset_iff.mpr ⟨hsub, V₁.sub_master hB⟩
  · rintro ⟨hX', hz⟩
    exact ⟨prodNbhd X' V₁.master, hz, prod_mem_prodNbhd hX' V₁.master_mem, hX', by simp⟩
```

```
@[simp] theorem toElementMap_proj₁ (z : (prod V₀ V₁).Element) :
    (proj₁ V₀ V₁).toElementMap z = z.snd := by
  apply Element.ext
  intro Y'
  constructor
  · rintro ⟨W, hzW, hW, hY', hsub⟩
    obtain ⟨A, B, hA, _, rfl⟩ := hW
    rw [prodNbhd.inr_preimage] at hsub
    refine ⟨hY', z.up_mem hzW (prod_mem_prodNbhd V₀.master_mem hY') ?_⟩
    exact prodNbhd.subset_iff.mpr ⟨V₀.sub_master hA, hsub⟩
  · rintro ⟨hY', hz⟩
    exact ⟨prodNbhd V₀.master Y', hz, prod_mem_prodNbhd V₀.master_mem hY', hY', by simp⟩

theorem toElementMap_proj₀_pair (x : V₀.Element) (y : V₁.Element) :
    (proj₀ V₀ V₁).toElementMap (pair x y) = x := by
  rw [toElementMap_proj₀, fst_pair]

theorem toElementMap_proj₁_pair (x : V₀.Element) (y : V₁.Element) :
    (proj₁ V₀ V₁).toElementMap (pair x y) = y := by
  rw [toElementMap_proj₁, snd_pair]

theorem toElementMap_paired (f : ApproximableMap V₂ V₀) (g : ApproximableMap V₂ V₁)
    (w : V₂.Element) :
    (paired f g).toElementMap w = pair (f.toElementMap w) (g.toElementMap w) := by
  apply Element.ext
  intro P
  constructor
  · rintro ⟨Z, hwZ, hP, hfZ, hgZ⟩
    exact ⟨Sum.inl ^-1' P, Sum.inr ^-1' P, ⟨Z, hwZ, hfZ⟩, ⟨Z, hwZ, hgZ⟩, prodNbhd.
    preimage hP⟩
  · rintro ⟨X, Y, ⟨Z₁, hwZ₁, hfZ₁⟩, ⟨Z₂, hwZ₂, hgZ₂⟩, rfl⟩
    refine ⟨Z₁ ∩ Z₂, w.inter_mem hwZ₁ hwZ₂, prod_mem_prodNbhd (f.rel_cod hfZ₁) (g.
    rel_cod hgZ₂),
      ?_, ?_⟩
    · rw [prodNbhd.inl_preimage]
      exact f.mono hfZ₁ Set.inter_subset_left subset_rfl (w.sub (w.inter_mem hwZ₁ hwZ₂)
    )
        (f.rel_cod hfZ₁)
    · rw [prodNbhd.inr_preimage]
      exact g.mono hgZ₂ Set.inter_subset_right subset_rfl (w.sub (w.inter_mem hwZ₁ hwZ₂
    ))
        (g.rel_cod hgZ₂)

theorem proj₀_comp_paired (f : ApproximableMap V₂ V₀) (g : ApproximableMap V₂ V₁) :
    (proj₀ V₀ V₁).comp (paired f g) = f := by
  apply ext_of_toElementMap
  intro w
  rw [toElementMap_comp, toElementMap_paired, toElementMap_proj₀, fst_pair]

theorem proj₁_comp_paired (f : ApproximableMap V₂ V₀) (g : ApproximableMap V₂ V₁) :
    (proj₁ V₀ V₁).comp (paired f g) = g := by
  apply ext_of_toElementMap
  intro w
  rw [toElementMap_comp, toElementMap_paired, toElementMap_proj₁, snd_pair]

theorem paired_proj (h : ApproximableMap V₂ (prod V₀ V₁)) :
```

```
    paired ((proj₀ V₀ V₁).comp h) ((proj₁ V₀ V₁).comp h) = h := by
  apply ext_of_toElementMap
  intro w
  rw [toElementMap_paired, toElementMap_comp, toElementMap_comp, toElementMap_proj₀,
    toElementMap_proj₁, pair_fst_snd]

end
end DomainTheory.NeighbourhoodSystems
end
```

4.4.6 Proposition 3.4 — pairing and projections

PROPOSITION 3.4. The mappings p_0, p_1 and $\langle f, g\rangle$ are approximable mappings, provided f and g are, and we have:

(i) $p_0 \circ \langle f, g\rangle = f$ and $p_1 \circ \langle f, g\rangle = g$.

Moreover, for $z \in |\mathcal{D}_0 \times \mathcal{D}_1|$, we have:

(ii) $p_0(z) = z_0$ and $p_1(z) = z_1$,

in the notation of the proof of 3.2. Further if $h : \mathcal{D}_2 \to \mathcal{D}_0 \times \mathcal{D}_1$ is any approximable mapping, then

(iii) $h = \langle p_0 \circ h, p_1 \circ h\rangle$.

Moreover, for all $w \in |\mathcal{D}_2|$, we have:

(iv) $\langle f, g\rangle(w) = \langle f(w), g(w)\rangle$,

where again on the right-hand side the notation of the proof of 3.2 is used. □

The proof of this result is left as an exercise. Note the consequence that there is a one-one correspondence between pairs of approximable mappings $f : \mathcal{D}_2 \to \mathcal{D}_0$ and $g : \mathcal{D}_2 \to \mathcal{D}_1$ and mappings $h : \mathcal{D}_2 \to \mathcal{D}_0 \times \mathcal{D}_1$. It is clear that we generalize all this to products

$$\mathcal{D}_0 \times \mathcal{D}_1 \times \cdots \times \mathcal{D}_{n-1}$$

of several systems.

The product construct also neatly explains functions of several variables. In Exercise 2.19 we used the informal notation

$$f : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2$$

and suggested regarding f as a ternary relation

$$X, Y \; f \; Z.$$

But now with $\mathcal{D}_0 \times \mathcal{D}_1$ given an independent meaning, all we have to do is to regard f as a binary relation with

$$(X, Y) \; f \; Z$$

equivalent to the old relationship. We can also employ an element-wise notation as in $f(\langle x, y\rangle)$, which can more easily be written $f(x, y)$. Similar remarks apply to functions of more than two arguments.

We have discussed several times what it means for a function $f(x)$ to come from an approximable mapping. It is interesting to ask the analogous question for functions of several arguments.

Theorem (`proposition_3_4`). Projections invert pairing; pairing the projections of a map into a product recovers that map; and pairing acts elementwise by pairing the images.

Proof. Assemble the left and right composition laws, the pairing–projection recovery, and the element-wise pairing law. ■

`Proposition_3_4.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.ProjectionMappings

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
open ApproximableMap

variable {α β ? : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {V₂ : NeighborhoodSystem ?}

/-- Scott's Proposition 3.4: projections invert pairing, pairing recovers maps into
    products,
    and pairing acts elementwise by pairing the images. -/
theorem proposition_3_4 :
    (∀ (f : ApproximableMap V₂ V₀) (g : ApproximableMap V₂ V₁),
      (proj₀ V₀ V₁).comp (paired f g) = f ∧ (proj₁ V₀ V₁).comp (paired f g) = g) ∧
    (∀ (h : ApproximableMap V₂ (prod V₀ V₁)),
      paired ((proj₀ V₀ V₁).comp h) ((proj₁ V₀ V₁).comp h) = h) ∧
    (∀ (f : ApproximableMap V₂ V₀) (g : ApproximableMap V₂ V₁) (w : V₂.Element),
      (paired f g).toElementMap w = pair (f.toElementMap w) (g.toElementMap w)) :=
  ⟨fun f g => ⟨proj₀_comp_paired f g, proj₁_comp_paired f g⟩, paired_proj,
    toElementMap_paired⟩

end DomainTheory.NeighbourhoodSystems
```

(Already included: `1980_ApproximableMap2.md`.)

4.4.7 Theorem 3.5 — separate approximability

THEOREM 3.5. An elementwise function

$$f : |D_0 \times D_1| \to |D_2|$$

of two arguments comes from an approximable mapping iff for each fixed $a \in |D_0|$ and each fixed $b \in |D_1|$ the transformations

$$x \mapsto f(x, b) \quad \text{and} \quad y \mapsto f(a, y)$$

come from approximable mappings of one argument.

Proof: As this is the first time we have had to deal with constants in functions, a lemma is useful.

The product $\leftrightarrow$ binary approximable-map correspondence is with the approximable maps of two arguments peer.

Theorem (`theorem_3_5`). Joint approximable maps out of a product correspond bijectively to binary approximable maps, and the correspondence agrees elementwise with pairing.

Proof. Assemble the product–binary equivalence with the elementwise pairing law. ■

`Theorem_3_5.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.
    ApproximableMap2

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
open ApproximableMap

/-- Scott's Theorem 3.5: joint approximable maps out of a product correspond to binary
    approximable maps, and the correspondence agrees elementwise with pairing. -/
theorem theorem_3_5 :
    (∀ {α β ? : Type*} (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
      (V₂ : NeighborhoodSystem ?),
      Nonempty (ApproximableMap (prod V₀ V₁) V₂ ≃ ApproximableMap₂ V₀ V₁ V₂)) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (f : ApproximableMap (prod V₀ V₁) V₂)
      (x : V₀.Element) (y : V₁.Element),
      (toMap₂ f).toElementMap₂ x y = f.toElementMap (pair x y)) :=
  ⟨fun {_ _ _} V₀ V₁ V₂ => ⟨map₂Equiv V₀ V₁ V₂⟩, toElementMap₂_toMap₂⟩

end DomainTheory.NeighbourhoodSystems
```

4.4.8 Constant approximable map

Constant approximable maps valued at a fixed ideal element.

Definition (`constMap`). The constant approximable map at an ideal element relates a domain neighbourhood to a codomain neighbourhood precisely when the latter belongs to that element.

Proof. Domain and codomain membership, master, intersection on the right, and monotonicity follow from the filter axioms of the value. ■

Theorem (`constMap_rel`). Membership in the constant-map relation is domain neighbourhood membership together with membership of the value in the codomain neighbourhood.

Proof. By definition. ■

Theorem (`toElementMap_constMap`). The elementwise action of a constant map is the constant function with that value.

Proof. Compare membership: the forward direction unpacks a witnessing domain neighbourhood; the converse evaluates at the domain master. ■

`constMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
open ApproximableMap

variable {α β : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def constMap (V₀ : NeighborhoodSystem α) (b : V₁.Element) : ApproximableMap V₀ V₁ where
  rel X Y := V₀.mem X ∧ b.mem Y
  rel_dom h := h.1
  rel_cod h := b.sub h.2
  master_rel := ⟨V₀.master_mem, b.master_mem⟩
  inter_right := by rintro X Y Y' ⟨hX, hY⟩ ⟨_, hY'⟩; exact ⟨hX, b.inter_mem hY hY'⟩
  mono := by
    rintro X X' Y Y' ⟨_, hY⟩ _ hYY' hX' hY'
    exact ⟨hX', b.up_mem hY hY' hYY'⟩

@[simp] theorem constMap_rel {b : V₁.Element} {X : Set α} {Y : Set β} :
    (constMap V₀ b).rel X Y ↔ V₀.mem X ∧ b.mem Y := Iff.rfl

@[simp] theorem toElementMap_constMap (b : V₁.Element) (x : V₀.Element) :
    (constMap V₀ b).toElementMap x = b := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨X, _, _, hbY⟩; exact hbY
  · intro hbY; exact ⟨V₀.master, x.master_mem, V₀.master_mem, hbY⟩

end DomainTheory.NeighbourhoodSystems
```

4.4.9 Lemma 3.6 — constant functions

LEMMA 3.6. Given $b \in |D_1|$, the constant function

$$b : |D_0| \to |D_1|$$

where $b(x) = b$ for all $x \in |D_0|$, comes from the approximable mapping such that

$$X \; b \; Y \quad \text{iff} \quad Y \in b,$$

for all $X \in D_0$ and $Y \in D_1$. □

There is no real confusion here in using "b" both for function and value. Returning, then, to the proof of 3.5, we see that the reason that $x \mapsto f(x, b)$ comes from an approximable mapping is that the mapping in question is the composition of two approximable mappings, namely $f \circ \langle I_{D_0}, b \rangle$. Clearly we can interchange the rôles of D_0 and D_1 to get at $y \mapsto f(a, y)$.

Conversely, assume that both these functions come from approximable mappings no matter the choice of a and b. Clearly the mapping to determine f is the relation from $X \cup Y$ to Z where

$$Z \in f(\uparrow X, \uparrow Y) = f(\uparrow (X \cup Y)).$$

To prove that this determines f we calculate by the formula of Exercise 2.9:

$$\begin{aligned} f(x,y) &= \bigcup\{f(\uparrow X, y) \mid X \in x\} \\ &= \bigcup\{\bigcup\{f(\uparrow X, \uparrow Y) \mid Y \in y\} \mid X \in x\} \\ &= \bigcup\{f(\uparrow X, \uparrow Y) \mid X \in x \text{ and } Y \in y\} \\ &= \bigcup\{f(\uparrow (X \cup Y)) \mid (X \cup Y) \in \langle x, y\rangle\}. \end{aligned}$$

And, again by 2.9, this is what was needed. □

Said more informarily, a function of several arguments is approximable in all the variables *jointly* if it is approximable in each of the variables *separately*.

The type of argument used in 3.5 in the first half of the proof also provides a generalization of 2.6 to functions of several arguments. When we form a function like

$$f(g(x,z,\ldots), h(y,x,\ldots), k(z,w,\ldots), \ldots)$$

from given functions $f, g, h, k, \ldots$; we call the process *substitution*.

Theorem (`lemma_3_6`). The constant approximable map at an ideal element has the membership relation of that element, and acts elementwise as the constant function.

Proof. Assemble the relation characterisation with the elementwise constant law. ■

`Lemma_3_6.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.constMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
open ApproximableMap

/-- Scott's Lemma 3.6: the constant approximable map at an ideal element has the
    membership
    relation of that element, and acts elementwise as the constant function. -/
theorem lemma_3_6 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {b : V₁.Element} {X : Set α} {Y : Set β},
      (constMap V₀ b).rel X Y ↔ V₀.mem X ∧ b.mem Y) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (b : V₁.Element) (x : V₀.Element),
      (constMap V₀ b).toElementMap x = b) :=
  ⟨@constMap_rel, @toElementMap_constMap⟩

end DomainTheory.NeighbourhoodSystems
```

4.4.10 Elementwise substitution

Elementwise action of substituting approximable maps into a joint map out of a product.

Theorem (`substitution_toElementMap`). Composing a joint approximable map with a pairing of two maps, then evaluating elementwise, equals the binary elementwise action of the joint map at the images of the two components.

Proof. Expand by the composition, pairing, and product–binary elementwise laws. ■

`substitution_toElementMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.ProjectionMappings
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.
    ApproximableMap2
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
open ApproximableMap

variable {α β ? ? : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {V₂ : NeighborhoodSystem ?} {V₃ : NeighborhoodSystem ?}

theorem substitution_toElementMap (F : ApproximableMap (prod V₀ V₁) V₂)
    (a : ApproximableMap V₃ V₀) (b : ApproximableMap V₃ V₁) (w : V₃.Element) :
    (F.comp (paired a b)).toElementMap w
      = (toMap₂ F).toElementMap₂ (a.toElementMap w) (b.toElementMap w) := by
  rw [toElementMap_comp, toElementMap_paired, toElementMap₂_toMap₂]

end DomainTheory.NeighbourhoodSystems
```

4.4.11 Proposition 3.7 — closure under substitution

PROPOSITION 3.7. The functions of several arguments between domains coming from approximable mappings are closed under substitution.

Proof: An example will establish the method. Suppose there are four variables involved taking values in domains provided by systems D_0, D_1, D_2, D_3. We might have a substitution like:

$$f(g(x_0, x_1), h(x_1, x_2), k(x_3, x_0, x_2)).$$

Here it might be that the values of the functions inside come from quite other systems; for instance,

$$k : D_3 \times D_0 \times D_2 \to D_4$$

might be possible. By using projections

$$p_i : D_0 \times D_1 \times D_2 \times D_3 \to D_i,$$

where $i < 4$, we can assure that we have several functions all on the same product; thus,

$$k \cdot \langle p_3, p_0, p_2 \rangle : D_0 \times D_1 \times D_2 \times D_3 \to D_4.$$

Now no matter on what domains f is defined, the following composition makes sense:

$$f \circ \langle g \circ \langle p_0, p_1 \rangle, h \circ \langle p_1, p_2 \rangle, k \circ \langle p_3, p_0, p_2 \rangle \rangle;$$

and in fact this is the desired function. Writing it this way makes it clear that the function comes from an approximable mapping: we apply 3.3 (generalized, of course, to products with several terms) to construe the parts between brackets $\langle$ and $\rangle$ as approximable mappings, and then by this trick the composition $\circ$ is the ordinary composition of 2.6. □

It has to be admitted that there is a slight point overlooked in forming products like $D \times D$ with two identical domains. This is discussed in Exercise 3.14, invoking explicit isomorphisms.

The construct that makes the whole theory of domains work so smoothly is the function-space construct: it is possible to regard functions as *objects* which form a domain. Look back at Definition 2.1 and compare it with the original definition of element in 1.6. There are obvious formal similarities, except that filters are sets of neighbourhoods and mappings are sets of pairs of neighbourhoods (relations). But as we saw in 1.10 it is possible to turn the filters into tokens via a simple definition of neighbourhood. We apply the same kind of definition to the mappings.

Theorem (`proposition_3_7`). Substituting approximable maps into a joint map is composition with a pairing, and agrees elementwise with the binary action on the substituted images.

Proof. Assemble the elementwise substitution law. ■

`Proposition_3_7.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.
    substitution_toElementMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
open ApproximableMap

/-- Scott's Proposition 3.7: substituting approximable maps into a joint map is
    composition
    with a pairing, and agrees elementwise with the binary action on the substituted
    images. -/
theorem proposition_3_7 :
    ∀ {α β ? ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} {V₃ : NeighborhoodSystem ?}
      (F : ApproximableMap (prod V₀ V₁) V₂) (a : ApproximableMap V₃ V₀)
      (b : ApproximableMap V₃ V₁) (w : V₃.Element),
      (F.comp (paired a b)).toElementMap w
        = (toMap₂ F).toElementMap₂ (a.toElementMap w) (b.toElementMap w) :=
  substitution_toElementMap

end DomainTheory.NeighbourhoodSystems
```

4.4.12 Step neighbourhoods of maps

Step neighbourhoods of approximable maps, finite intersections of steps, and the pointwise order on maps.

Instance (`instPartialOrderApproximableMap`). Approximable maps are partially ordered by inclusion of their neighbourhood relations.

Proof. Reflexivity, transitivity, and antisymmetry follow from the corresponding properties of implication, with equality of maps by agreement of relations. ■

Definition (`step`). The step neighbourhood at a pair of neighbourhoods is the set of approximable maps relating those neighbourhoods.

Theorem (`mem_step`). Membership in a step is the corresponding relation of the map.

Proof. By definition. ■

Definition (`stepFun`). The step function of a finite list of neighbourhood pairs is the set of maps relating every pair in the list.

Theorem (`mem_stepFun`). Membership in a step function is relation of every listed pair.

Proof. By definition. ■

Theorem (`stepFun_nil`). The empty list yields the universal set of maps.

Proof. There are no pairs to satisfy. ■

Theorem (`stepFun_append`). Concatenation of lists yields the intersection of the corresponding step functions.

Proof. Unpack membership in the concatenated list. ■

Theorem (`stepFun_cons`). Prepending a pair yields the intersection of that step with the step function of the tail.

Proof. Unpack membership in the cons list. ■

Theorem (`stepFun_singleton`). A singleton list recovers the corresponding step.

Proof. Reduce via cons and the empty-list law. ■

Theorem (`step_master_eq`). The step at the two masters is the universal set of maps.

Proof. Every approximable map relates the masters. ■

Theorem (`stepFun_up_closed`). Step functions are upward closed in the pointwise order on maps.

Proof. Relations of a smaller map transport along the inequality. ■

Theorem (`step_inter_right`). Intersection of two steps with the same domain neighbourhood is the step at the intersection of the range neighbourhoods.

Proof. One direction uses intersection on the right for the relation; the converse uses monotonicity toward each factor. ■

Theorem (`step_subset`). Steps nest when the domain neighbourhood shrinks and the range neighbourhood expands.

Proof. Apply monotonicity of the relation. ■

`step.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
```

```
instance : PartialOrder (ApproximableMap V₀ V₁) where
  le f g := ∀ X Y, f.rel X Y → g.rel X Y
  le_refl _ _ _ h := h
  le_trans _ _ _ h1 h2 X Y h := h2 X Y (h1 X Y h)
  le_antisymm f g h1 h2 := ApproximableMap.ext fun X Y => ⟨h1 X Y, h2 X Y⟩

def step (X : Set α) (Y : Set β) : Set (ApproximableMap V₀ V₁) := {f | f.rel X Y}

@[simp] theorem mem_step {X : Set α} {Y : Set β} {f : ApproximableMap V₀ V₁} :
    f ∈ step X Y ↔ f.rel X Y := Iff.rfl

def stepFun (L : List (Set α × Set β)) : Set (ApproximableMap V₀ V₁) :=
  {f | ∀ p ∈ L, f.rel p.1 p.2}

@[simp] theorem mem_stepFun {L : List (Set α × Set β)} {f : ApproximableMap V₀ V₁} :
    f ∈ stepFun L ↔ ∀ p ∈ L, f.rel p.1 p.2 := Iff.rfl

@[simp] theorem stepFun_nil : (stepFun [] : Set (ApproximableMap V₀ V₁)) = Set.univ :=
    by
  ext f; simp

theorem stepFun_append (L L' : List (Set α × Set β)) :
    (stepFun (L ++ L') : Set (ApproximableMap V₀ V₁)) = stepFun L ∩ stepFun L' := by
  ext f
  simp only [mem_stepFun, List.mem_append, Set.mem_inter_iff]
  constructor
  · intro h; exact ⟨fun p hp => h p (Or.inl hp), fun p hp => h p (Or.inr hp)⟩
  · rintro ⟨hL, hL'⟩ p (hp | hp)
    · exact hL p hp
    · exact hL' p hp

theorem stepFun_cons (p : Set α × Set β) (L : List (Set α × Set β)) :
    (stepFun (p :: L) : Set (ApproximableMap V₀ V₁)) = step p.1 p.2 ∩ stepFun L := by
  ext f
  simp only [mem_stepFun, List.mem_cons, Set.mem_inter_iff, mem_step]
  constructor
  · intro h; exact ⟨h p (Or.inl rfl), fun q hq => h q (Or.inr hq)⟩
  · rintro ⟨hp, hrest⟩ q (rfl | hq)
    · exact hp
    · exact hrest q hq

theorem stepFun_singleton (X : Set α) (Y : Set β) :
    (stepFun [(X, Y)] : Set (ApproximableMap V₀ V₁)) = step X Y := by
  rw [stepFun_cons, stepFun_nil, Set.inter_univ]

@[simp] theorem step_master_eq :
    (step V₀.master V₁.master : Set (ApproximableMap V₀ V₁)) = Set.univ := by
  ext f; simpa using f.master_rel

theorem stepFun_up_closed {L : List (Set α × Set β)} {f f' : ApproximableMap V₀ V₁}
    (hf : f ∈ stepFun L) (hff' : f ≤ f') : f' ∈ stepFun L := by
  intro p hp
  exact hff' p.1 p.2 (hf p hp)

theorem step_inter_right {X : Set α} {Y Y' : Set β} (hY : V₁.mem Y) (hY' : V₁.mem Y') :
```

```
    (step X Y ∩ step X Y' : Set (ApproximableMap V₀ V₁)) = step X (Y ∩ Y') := by
  ext f
  simp only [Set.mem_inter_iff, mem_step]
  constructor
  · rintro ⟨h, h'⟩; exact f.inter_right h h'
  · intro h
    exact ⟨f.mono h subset_rfl Set.inter_subset_left (f.rel_dom h) hY,
           f.mono h subset_rfl Set.inter_subset_right (f.rel_dom h) hY'⟩

theorem step_subset {X X' : Set α} {Y Y' : Set β} (hX' : V₀.mem X') (hY' : V₁.mem Y')
    (hX'X : X' ⊆ X) (hYY' : Y ⊆ Y') : (step X Y : Set (ApproximableMap V₀ V₁)) ⊆ step
    X' Y' := by
  intro f hf
  exact f.mono hf hX'X hYY' hX' hY'

end DomainTheory.NeighbourhoodSystems
```

4.4.13 Intersection of range neighbourhoods

The intersection of range neighbourhoods forced by a finite list of domain–range pairs, relative to a master range neighbourhood.

Definition (`interYs`). Relative to a master range neighbourhood, a list of domain–range pairs sends a domain neighbourhood to those master tokens that lie in every listed range whenever the domain neighbourhood is contained in the corresponding domain factor.

Theorem (`interYs_nil`). The empty list returns the master range neighbourhood.

Proof. By definition. ■

Theorem (`interYs_cons`). Prepending a pair intersects the implication-set for that pair with the value on the tail.

Proof. By definition. ■

Theorem (`mem_interYs`). A token belongs to the intersection precisely when it belongs to the master and, for every listed pair containing the domain neighbourhood, belongs to that pair's range.

Proof. By induction on the list. ■

Theorem (`interYs_antitone`). Shrinking the domain neighbourhood enlarges the resulting range intersection.

Proof. Containment hypotheses become harder when the domain neighbourhood grows. ■

Theorem (`interYs_subset_master`). The intersection is always contained in the master.

Proof. Unpack the membership characterisation. ■

Theorem (`interYs_subset_of_mem`). Evaluating at a listed domain factor recovers containment in that pair's range.

Proof. Instantiate the membership characterisation at that pair with reflexive containment. ■

`interYs.lean`

```
module

public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
```

```
variable {α β : Type*}

def interYs (m : Set β) : List (Set α × Set β) → Set α → Set β
  | [], _ => m
  | p :: L, X => {z | X ⊆ p.1 → z ∈ p.2} ∩ interYs m L X

@[simp] theorem interYs_nil (m : Set β) (X : Set α) : interYs m [] X = m := rfl

theorem interYs_cons (m : Set β) (p : Set α × Set β) (L : List (Set α × Set β)) (X :
    Set α) :
    interYs m (p :: L) X = {z | X ⊆ p.1 → z ∈ p.2} ∩ interYs m L X := rfl

theorem mem_interYs {m : Set β} {L : List (Set α × Set β)} {X : Set α} {z : β} :
    z ∈ interYs m L X ↔ z ∈ m ∧ ∀ p ∈ L, X ⊆ p.1 → z ∈ p.2 := by
  induction L with
  | nil => simp
  | cons p L ih =>
    rw [interYs_cons]
    simp only [Set.mem_inter_iff, Set.mem_ofPred_eq, ih, List.mem_cons]
    constructor
    · rintro ⟨hp, hm, hL⟩
      refine ⟨hm, ?_⟩
      rintro q (rfl | hq) hXq
      · exact hp hXq
      · exact hL q hq hXq
    · rintro ⟨hm, hall⟩
      exact ⟨fun hXp => hall p (Or.inl rfl) hXp, hm,
        fun q hq hXq => hall q (Or.inr hq) hXq⟩

theorem interYs_antitone {m : Set β} {L : List (Set α × Set β)} {X X' : Set α} (h : X'
     ⊆ X) :
    interYs m L X' ⊆ interYs m L X := by
  intro z hz
  rw [mem_interYs] at hz ⊢
  exact ⟨hz.1, fun p hp hXp => hz.2 p hp (h.trans hXp)⟩

theorem interYs_subset_master {m : Set β} {L : List (Set α × Set β)} {X : Set α} :
    interYs m L X ⊆ m := fun _ hz => (mem_interYs.mp hz).1

theorem interYs_subset_of_mem {m : Set β} {L : List (Set α × Set β)} {p : Set α × Set
    β}
    (hp : p ∈ L) : interYs m L p.1 ⊆ p.2 :=
  fun _ hz => (mem_interYs.mp hz).2 p hp subset_rfl

end DomainTheory.NeighbourhoodSystems
```

4.4.14 Least map of a step list

The least approximable map forced by a consistent finite list of domain–range neighbourhood pairs, and its minimality among maps in the corresponding step neighbourhood.

Definition (`leastMap`). Given a list of neighbourhood pairs that are neighbourhoods of the respective systems, and a consistency hypothesis that the associated range intersections are always neighbourhoods, the

least map relates a domain neighbourhood to a range neighbourhood precisely when the range intersection for that domain neighbourhood is contained in the range neighbourhood.

Proof. Domain and codomain membership are immediate; the masters relate by containment in the master; intersection on the right uses the consistency hypothesis; monotonicity uses antitonicity of the range intersection. ■

Theorem (`leastMap_rel`). Membership in the least-map relation is domain and range neighbourhood membership together with containment of the range intersection.

Proof. By definition. ■

Theorem (`leastMap_mem_stepFun`). The least map belongs to the step function of its defining list.

Proof. Evaluate the relation at each listed pair using containment of the range intersection in that pair's range. ■

Theorem (`rel_interYs`). Every map in a step function relates a domain neighbourhood to the associated range intersection.

Proof. By induction on the list, using monotonicity and intersection on the right when the domain neighbourhood sits under a listed domain factor. ■

Theorem (`leastMap_le`). The least map is below every map in its step function.

Proof. Transport the range-intersection relation along containment into the target neighbourhood. ■

Theorem (`stepFun_subset_step_iff`). Under consistency, a step function is contained in a step if and only if the associated range intersection is contained in the step's range neighbourhood.

Proof. Forward, evaluate at the least map; reverse, use the range-intersection relation for an arbitrary map in the step function. ■

`leastMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.interYs
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.step

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def leastMap (L : List (Set α × Set β)) (_hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2)
    (hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X)) : ApproximableMap V₀ V₁
   where
  rel X Y := V₀.mem X ∧ V₁.mem Y ∧ interYs V₁.master L X ⊆ Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨V₀.master_mem, V₁.master_mem, interYs_subset_master⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY, hsub⟩ ⟨_, hY', hsub'⟩
    exact ⟨hX, V₁.inter_mem hY hY' (hcons hX) (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'⟩
  mono := by
    rintro X X' Y Y' ⟨_, _, hsub⟩ hX'X hYY' hX' hY'
    exact ⟨hX', hY', (interYs_antitone hX'X).trans (hsub.trans hYY')⟩
```

```
@[simp] theorem leastMap_rel {L : List (Set α × Set β)}
    {hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2}
    {hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X)} {X : Set α} {Y : Set β}
     :
    (leastMap L hL hcons).rel X Y ↔ V₀.mem X ∧ V₁.mem Y ∧ interYs V₁.master L X ⊆ Y :=
     Iff.rfl

theorem leastMap_mem_stepFun {L : List (Set α × Set β)} (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁
    .mem p.2)
    (hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X)) : leastMap L hL hcons ∈
     stepFun L := by
  intro p hp
  exact ⟨(hL p hp).1, (hL p hp).2, interYs_subset_of_mem hp⟩

theorem rel_interYs {L : List (Set α × Set β)} {f : ApproximableMap V₀ V₁} (hf : f ∈
    stepFun L)
    {X : Set α} (hX : V₀.mem X) : f.rel X (interYs V₁.master L X) := by
  induction L with
  | nil =>
    rw [interYs_nil]
    exact f.mono f.master_rel (V₀.sub_master hX) subset_rfl hX V₁.master_mem
  | cons p L ih =>
    have hftail : f ∈ stepFun L := fun q hq => hf q (List.mem_cons.mpr (Or.inr hq))
    have htail := ih hftail
    by_cases hXp : X ⊆ p.1
    · have hp : f.rel p.1 p.2 := hf p (List.mem_cons.mpr (Or.inl rfl))
      have hXp2 : f.rel X p.2 := f.mono hp hXp subset_rfl hX (f.rel_cod hp)
      have heq : interYs V₁.master (p :: L) X = p.2 ∩ interYs V₁.master L X := by
        rw [interYs_cons]; ext z
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq]
        exact ⟨fun ⟨h1, h2⟩ => ⟨h1 hXp, h2⟩, fun ⟨h1, h2⟩ => ⟨fun _ => h1, h2⟩⟩
      rw [heq]; exact f.inter_right hXp2 htail
    · have heq : interYs V₁.master (p :: L) X = interYs V₁.master L X := by
        rw [interYs_cons]; ext z
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq]
        exact ⟨fun h => h.2, fun h => ⟨fun hc => absurd hc hXp, h⟩⟩
      rw [heq]; exact htail

theorem leastMap_le {L : List (Set α × Set β)} (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2)
    (hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X)) {f : ApproximableMap V₀
    V₁}
    (hf : f ∈ stepFun L) : leastMap L hL hcons ≤ f := by
  rintro X Y ⟨hX, hY, hsub⟩
  exact f.mono (rel_interYs hf hX) subset_rfl hsub hX hY

theorem stepFun_subset_step_iff {L : List (Set α × Set β)} (hL : ∀ p ∈ L, V₀.mem p.1 ∧
     V₁.mem p.2)
    (hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X)) {X : Set α} {Y : Set β}
    (hX : V₀.mem X) (hY : V₁.mem Y) :
    (stepFun L : Set (ApproximableMap V₀ V₁)) ⊆ step X Y ↔ interYs V₁.master L X ⊆ Y
    := by
  constructor
  · intro hsub
    have := hsub (leastMap_mem_stepFun hL hcons)
    exact (mem_step.mp this).2.2
  · intro hsub f hf
```

```
    exact f.mono (rel_interYs hf hX) subset_rfl hsub hX hY

end DomainTheory.NeighbourhoodSystems
```

4.4.15 Definition 3.8 — function space

DEFINITION 3.8. Given neighbourhood systems D_0 and D_1, the *function space* $(D_0 \to D_1)$ is the system whose set of tokens is the set of approximable mappings of Definition 2.1 and whose neighbourhoods are finite non-empty intersections of sets of the form

$$[X, Y] = \{f : D_0 \to D_1 \mid X\, f Y\},$$

where $X \in D_0$ and $Y \in D_1$. □

We have been calling our mappings "approximable" for a long time now without saying exactly how they can be approximated! Definition 3.8 supplies the missing key, because once a domain has been defined, then the general theory gives an explicit meaning to the word approximation. We still have to verify, however, that the mappings do correspond to the elements of the domain.

Definition (`funSpace`). The function space of two neighbourhood systems has as tokens the approximable maps between them, and as neighbourhoods the nonempty step functions of finite lists of neighbourhood pairs from the respective systems, with master the universal set.

Proof. The master is nonempty via the constant map at the bottom; intersection of neighbourhoods concatenates the witnessing lists; every neighbourhood sits in the master. ■

Theorem (`funSpace_master`). The master of a function space is the universal set of maps.

Proof. By definition. ■

Theorem (`funSpace_mem_iff`). A set of maps is a function-space neighbourhood if and only if it is a nonempty step function of neighbourhood pairs.

Proof. By definition. ■

Theorem (`funSpace_mem_up_closed`). Function-space neighbourhoods are upward closed in the pointwise order on maps.

Proof. Unpack as a step function and apply upward closure of step functions. ■

Theorem (`step_mem`). Every step at neighbourhoods of the factor systems is a function-space neighbourhood.

Proof. Realise it as a singleton step function, witnessed nonempty by the constant map at the principal filter of the range neighbourhood. ■

Theorem (`mem_stepFun_iff`). An ideal element of the function space meets a step function if and only if it meets every corresponding step.

Proof. By induction on the list, using intersection and upward closure in the filter of neighbourhoods. ■

`funSpace.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.constMap
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.step

@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def funSpace (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    NeighborhoodSystem (ApproximableMap V₀ V₁) where
  mem W := (∃ L : List (Set α × Set β), (∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2) ∧ W =
    stepFun L)
    ∧ W.Nonempty
  master := Set.univ
  master_nonempty := ⟨constMap V₀ V₁.bot, Set.mem_univ _⟩
  master_mem := ⟨⟨[], by simp, stepFun_nil.symm⟩, ⟨constMap V₀ V₁.bot, Set.mem_univ _⟩⟩
  inter_mem := by
    rintro W W' Z ⟨⟨L, hL, rfl⟩, _⟩ ⟨⟨L', hL', rfl⟩, _⟩ ⟨_, hZne⟩ hZsub
    refine ⟨⟨L ++ L', ?_, (stepFun_append _ _).symm⟩, hZne.mono hZsub⟩
    intro p hp
    rcases List.mem_append.mp hp with h | h
    · exact hL p h
    · exact hL' p h
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem funSpace_master : (funSpace V₀ V₁).master = Set.univ := rfl

theorem funSpace_mem_iff {W : Set (ApproximableMap V₀ V₁)} :
    (funSpace V₀ V₁).mem W ↔
      (∃ L : List (Set α × Set β), (∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2) ∧ W = stepFun L)
        ∧ W.Nonempty := Iff.rfl

theorem funSpace_mem_up_closed {W : Set (ApproximableMap V₀ V₁)} (hW : (funSpace V₀ V₁)
    .mem W)
    {f f' : ApproximableMap V₀ V₁} (hf : f ∈ W) (hff' : f ≤ f') : f' ∈ W := by
  obtain ⟨⟨L, _, rfl⟩, _⟩ := hW
  exact stepFun_up_closed hf hff'

theorem step_mem {X : Set α} {Y : Set β} (hX : V₀.mem X) (hY : V₁.mem Y) :
    (funSpace V₀ V₁).mem (step X Y) := by
  refine ⟨⟨[(X, Y)], ?_, (stepFun_singleton X Y).symm⟩,
    ⟨constMap V₀ (V₁.principal hY), ?_⟩⟩
  · intro p hp; rw [List.mem_singleton] at hp; subst hp; exact ⟨hX, hY⟩
  · show (constMap V₀ (V₁.principal hY)).rel X Y
    exact ⟨hX, hY, subset_rfl⟩

theorem mem_stepFun_iff (φ : (funSpace V₀ V₁).Element) {L : List (Set α × Set β)}
    (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2) :
    φ.mem (stepFun L) ↔ ∀ p ∈ L, φ.mem (step p.1 p.2) := by
  induction L with
  | nil => simp only [stepFun_nil, List.not_mem_nil, IsEmpty.forall_iff, implies_true,
     iff_true]
           exact φ.master_mem
  | cons p L ih =>
    rw [stepFun_cons]
    have hp := hL p (List.mem_cons.mpr (Or.inl rfl))
    have hLtail : ∀ q ∈ L, V₀.mem q.1 ∧ V₁.mem q.2 :=
      fun q hq => hL q (List.mem_cons.mpr (Or.inr hq))
    constructor
```

```
    · intro hmem
      have hstep : φ.mem (step p.1 p.2) :=
        φ.up_mem hmem (step_mem hp.1 hp.2) Set.inter_subset_left
      have hne : (step p.1 p.2 ∩ stepFun L).Nonempty := (φ.sub hmem).2
      have htail : φ.mem (stepFun L) :=
        φ.up_mem hmem ⟨⟨L, hLtail, rfl⟩, hne.mono Set.inter_subset_right⟩ Set.
    inter_subset_right
      intro q hq
      rcases List.mem_cons.mp hq with rfl | hq
      · exact hstep
      · exact (ih hLtail).mp htail q hq
    · intro hall
      have hstep : φ.mem (step p.1 p.2) := hall p (List.mem_cons.mpr (Or.inl rfl))
      have htail : φ.mem (stepFun L) :=
        (ih hLtail).mpr (fun q hq => hall q (List.mem_cons.mpr (Or.inr hq)))
      exact φ.inter_mem hstep htail

end DomainTheory.NeighbourhoodSystems
```

4.4.16 Proposition 3.9 — consistent step neighbourhoods

PROPOSITION 3.9. Let neighbourhoods $X_i \in D_0$ and $Y_i \in D_1$ be given for $i < n$. Then the set of $[X_i, Y_i]$ for $i < n$ is consistent in $(D_0 \to D_1)$ iff the following condition holds:

(i) whenever $I \subseteq \{0, 1, \ldots, n-1\}$ and $\{X_i \mid i \in I\}$ is consistent in D_0, then $\{Y_i \mid i \in I\}$ must be consistent in D_1.

Moreover, when consistency holds, the least approximable mapping f_0 belonging to the intersection of the $[X_i, Y_i]$ is defined by:

(ii) $X f_0 Y$ iff $\bigcap\{Y_i \mid X \subseteq X_i\} \subseteq Y$ for $X \in D_0$ and $Y \in D_1$.

Proof: Suppose the $[X_i, Y_i]$ are consistent in $(D_0 \to D_1)$. Since the function space is being defined outright as a positive system, consistency means

$$f \in \bigcap\{[X_i, Y_i] \mid i < n\}$$

for some $f : D_0 \to D_1$. Now, with f in hand, let us check condition (i). Suppose $\{X_i \mid i \in I\}$ is consistent. This means

$$x \in \bigcap\{[X_i] \mid i \in I\}$$

for some $x \in |D_0|$. Suppose $i \in I$, so $x \in [X_i]$. Since $X_i f Y_i$ holds, $f(x) \in [Y_i]$. This means, therefore, that

$$f(x) \in \bigcap\{[Y_i] \mid i \in I\},$$

and so $\{Y_i \mid i \in I\}$ is consistent.

For the converse, suppose (i) is the case. We take (ii) as the definition of a mapping and remark that for an arbitrary $X \in D_0$, the set $\{X_i \mid X \subseteq X_i\}$ is automatically consistent in D_0. By our assumption, the set $\{Y_i \mid X \subseteq X_i\}$ is therefore consistent. This means that

$$\bigcap\{Y_i \mid X \subseteq X_i\} \in D_1.$$

(Keep in mind that i is restricted to those $i < n$, and there are only finitely many neighbourhoods being considered here.) It is thus almost immediate that the relation f_0 defined by (ii) satisfies conditions of 2.1 and so is an approximable mapping $f_0 : D_0 \to D_1$. By construction

$$X_i f_0 Y_i$$

holds trivially for all $i < n$; therefore,

$$f_0 \in \bigcap\{[X_i, Y_i] \mid i < n\}$$

and the desired consistency is established.

Finally suppose that f is any mapping in the neighbourhood under discussion; this means $X_i f Y_i$ holds for all $i < n$. Suppose $X f_0 Y$ holds. We have for $X \subseteq X_i$, $X f Y_i$; so

$$X f \bigcap\{Y_i \mid X \subseteq X_i\} \subseteq Y.$$

Thus, $X f Y$ follows; hence, as relations, $f_0 \subseteq f$. In other words f_0 is the minimal element of the neighbourhood. □

We note that, as a consequence of what we have just proved, when the neighbourhood is consistent, then

$$\bigcap\{[X_i, Y_i] \mid i < n\} \subseteq [X, Y]$$

is exactly equivalent to

$$\bigcap\{Y_i \mid X \subseteq X_i\} \subseteq Y.$$

Note also that a single neighbourhood $[X_0, Y_0]$ is always consistent since it contains the *constant mapping* k where

$$X k Y \text{ iff } Y_0 \subseteq Y,$$

for all $X \in \mathcal{D}_0$ and $Y \in \mathcal{D}_1$. Some other simple observations about these neighbourhoods are just translations of the conditions of Definition 2.1:

$$[\Delta_0, \Delta_1] = |\mathcal{D}_0 \to \mathcal{D}_1|;$$

$$[X, Y] \cap [X, Y'] = [X, Y \cap Y']; \text{ and}$$

$$X' \subseteq X \text{ and } Y \subseteq Y' \text{ imply } [X, Y] \subseteq [X', Y'],$$

for all $X, X' \in \mathcal{D}_0$ and $Y, Y' \in \mathcal{D}_1$. We are now ready to prove the main result about the construct.

Theorem (`proposition_3_9`). Under consistency of a finite step list, the least map lies in the corresponding step function, is minimal among maps in that neighbourhood, and containment of the step function in a step is equivalent to containment of the associated range intersection.

Proof. Assemble membership of the least map in the step function, its minimality, and the step-containment equivalence. ■

Proposition_3_9.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.leastMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

/-- Scott's Proposition 3.9: under consistency of a finite step list, the least map
    lies in the
    step function, is minimal among maps in that neighbourhood, and containment of the
     step
    function in a step is equivalent to containment of the associated range
    intersection. -/
theorem proposition_3_9 :
    (∀ {L : List (Set α × Set β)} (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2)
      (hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X)),
      leastMap L hL hcons ∈ stepFun L) ∧
    (∀ {L : List (Set α × Set β)} (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2)
      (hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X))
      {f : ApproximableMap V₀ V₁}, f ∈ stepFun L → leastMap L hL hcons ≤ f) ∧
    (∀ {L : List (Set α × Set β)} (_hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2)
      (_hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X))
      {X : Set α} {Y : Set β} (_hX : V₀.mem X) (_hY : V₁.mem Y),
      (stepFun L : Set (ApproximableMap V₀ V₁)) ⊆ step X Y ↔
        interYs V₁.master L X ⊆ Y) :=
  ⟨leastMap_mem_stepFun, leastMap_le, stepFun_subset_step_iff⟩

end DomainTheory.NeighbourhoodSystems
```

4.4.17 Function-space element isomorphism

Order isomorphism between ideal elements of a function space and approximable maps.

Definition (`toApproxMap`). An ideal element of a function space determines an approximable map by relating neighbourhoods precisely when the corresponding step belongs to the element.

Proof. Domain and codomain membership unpack a witnessing map in the step; the masters relate via the master step being universal; intersection on the right and monotonicity use the filter axioms with the step intersection and nesting laws. ■

Theorem (`toApproxMap_rel`). Membership in the recovered relation is membership of the corresponding step in the ideal element.

Proof. By definition. ■

Definition (`toFilter`). An approximable map determines an ideal element of the function space consisting of those function-space neighbourhoods that contain the map.

Proof. Filter axioms follow from the neighbourhood structure of the function space and membership of the map. ■

Theorem (`mem_toFilter`). Membership in the filter of a map is function-space neighbourhood membership together with containment of the map.

Proof. By definition. ■

Definition (`funSpaceEquiv`). Ideal elements of a function space are order-isomorphic to approximable maps via the two conversions.

Proof. Round trips compare membership via step-function generation of neighbourhoods; the order equivalence unpacks approximation as containment of steps. ■

Theorem (`funSpaceEquiv_apply`). The isomorphism sends an ideal element to its recovered approximable map.

Proof. By definition. ■

Theorem (`funSpaceEquiv_symm_apply`). The inverse isomorphism sends a map to its principal filter of neighbourhoods containing it.

Proof. By definition. ■

Theorem (`funSpace_mem_inter`). The intersection of two nonempty function-space neighbourhoods is again a neighbourhood.

Proof. Concatenate the witnessing step-function lists. ■

`funSpaceEquiv.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.funSpace
public import Mathlib.Data.Set.Basic
public import Mathlib.Order.Hom.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def toApproxMap (φ : (funSpace V₀ V₁).Element) : ApproximableMap V₀ V₁ where
  rel X Y := φ.mem (step X Y)
  rel_dom := by intro X Y h; obtain ⟨f, hf⟩ := (φ.sub h).2; exact f.rel_dom hf
  rel_cod := by intro X Y h; obtain ⟨f, hf⟩ := (φ.sub h).2; exact f.rel_cod hf
  master_rel := by show φ.mem (step V₀.master V₁.master); rw [step_master_eq]; exact φ.
    master_mem
  inter_right := by
    intro X Y Y' h h'
    obtain ⟨f, hf⟩ := (φ.sub h).2
    obtain ⟨f', hf'⟩ := (φ.sub h').2
    have hY : V₁.mem Y := f.rel_cod hf
    have hY' : V₁.mem Y' := f'.rel_cod hf'
    show φ.mem (step X (Y ∩ Y'))
    rw [← step_inter_right hY hY']
    exact φ.inter_mem h h'
  mono := by
    intro X X' Y Y' h hX'X hYY' hX' hY'
    show φ.mem (step X' Y')
    exact φ.up_mem h (step_mem hX' hY') (step_subset hX' hY' hX'X hYY')

@[simp] theorem toApproxMap_rel {φ : (funSpace V₀ V₁).Element} {X : Set α} {Y : Set β}
    :
    (toApproxMap φ).rel X Y ↔ φ.mem (step X Y) := Iff.rfl
```

```
def toFilter (f : ApproximableMap V₀ V₁) : (funSpace V₀ V₁).Element where
  mem W := (funSpace V₀ V₁).mem W ∧ f ∈ W
  sub h := h.1
  master_mem := ⟨(funSpace V₀ V₁).master_mem, Set.mem_univ f⟩
  inter_mem := by
    rintro W W' ⟨hW, hfW⟩ ⟨hW', hfW'⟩
    obtain ⟨⟨L, hL, rfl⟩, _⟩ := hW
    obtain ⟨⟨L', hL', rfl⟩, _⟩ := hW'
    refine ⟨⟨L ++ L', ?_, (stepFun_append _ _).symm⟩, ⟨f, ?_⟩⟩, ?_⟩
    · intro p hp; rcases List.mem_append.mp hp with h | h
      · exact hL p h
      · exact hL' p h
    · exact Set.mem_inter hfW hfW'
    · exact Set.mem_inter hfW hfW'
  up_mem := by rintro W W' ⟨hW, hfW⟩ hW' hWW'; exact ⟨hW', hWW' hfW⟩

@[simp] theorem mem_toFilter {f : ApproximableMap V₀ V₁} {W : Set (ApproximableMap V₀ V
    ₁)} :
    (toFilter f).mem W ↔ (funSpace V₀ V₁).mem W ∧ f ∈ W := Iff.rfl

def funSpaceEquiv (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    (funSpace V₀ V₁).Element ≃o ApproximableMap V₀ V₁ where
  toFun := toApproxMap
  invFun := toFilter
  left_inv φ := by
    apply Element.ext
    intro W
    constructor
    · rintro ⟨hWmem, hfW⟩
      obtain ⟨⟨L, hL, rfl⟩, _⟩ := hWmem
      exact (mem_stepFun_iff φ hL).mpr (fun p hp => hfW p hp)
    · intro hW
      refine ⟨φ.sub hW, ?_⟩
      obtain ⟨⟨L, hL, rfl⟩, _⟩ := φ.sub hW
      intro p hp
      exact (mem_stepFun_iff φ hL).mp hW p hp
  right_inv f := by
    apply ApproximableMap.ext
    intro X Y
    constructor
    · rintro ⟨_, hf⟩; exact hf
    · intro hf; exact ⟨step_mem (f.rel_dom hf) (f.rel_cod hf), hf⟩
  map_rel_iff' := by
    intro φ φ'
    constructor
    · intro h W hW
      obtain ⟨⟨L, hL, rfl⟩, _⟩ := φ.sub hW
      refine (mem_stepFun_iff φ' hL).mpr (fun p hp => ?_)
      exact h p.1 p.2 ((mem_stepFun_iff φ hL).mp hW p hp)
    · intro h X Y hrel
      exact h _ hrel

@[simp] theorem funSpaceEquiv_apply (φ : (funSpace V₀ V₁).Element) :
    funSpaceEquiv V₀ V₁ φ = toApproxMap φ := rfl
```

```
@[simp] theorem funSpaceEquiv_symm_apply (f : ApproximableMap V₀ V₁) :
    (funSpaceEquiv V₀ V₁).symm f = toFilter f := rfl

theorem funSpace_mem_inter {W W' : Set (ApproximableMap V₀ V₁)}
    (hW : (funSpace V₀ V₁).mem W) (hW' : (funSpace V₀ V₁).mem W') (hne : (W ∩ W').
    Nonempty) :
    (funSpace V₀ V₁).mem (W ∩ W') := by
  obtain ⟨⟨L, hL, rfl⟩, _⟩ := hW
  obtain ⟨⟨L', hL', rfl⟩, _⟩ := hW'
  refine ⟨⟨L ++ L', ?_, (stepFun_append _ _).symm⟩, hne⟩
  intro p hp
  rcases List.mem_append.mp hp with h | h
  · exact hL p h
  · exact hL' p h

end DomainTheory.NeighbourhoodSystems
```

4.4.18 Theorem 3.10 — elements are mappings

THEOREM 3.10. Given neighbourhood systems $\mathcal{D}_0$ and $\mathcal{D}_1$, the function space system $(\mathcal{D}_0 \to \mathcal{D}_1)$ is complete in the sense that every filter in $|\mathcal{D}_0 \to \mathcal{D}_1|$ is fixed by a unique approximable mapping.

Proof: Let $f : \mathcal{D}_0 \to \mathcal{D}_1$ be an approximable mapping. By the very definition of $(\mathcal{D}_0 \to \mathcal{D}_1)$ it determines a filter by the definition:

$$\hat{f} = \{F \in (D_0 \to D_1) \mid f \in F\}.$$

Trivially $[X, Y] \in \hat{f}$ iff $f \in [X, Y]$ iff XfY; so this filter uniquely determines the relation f. What we have to show is that every filter in $|D_0 \to D_1|$ is of this form.

Suppose $\varphi \in |D_0 \to D_1|$ is any filter. A relation can be defined at once by

$$X\hat{\varphi}Y \quad \text{iff} \quad [X, Y] \in \varphi.$$

In view of the remarks we made just before stating this theorem, there is no problem in showing that $\hat{\varphi}$ is an approximable mapping. Since the neighbourhoods of the function space are in any case finite intersections of sets like $[X, Y]$, those $[X, Y] \in \varphi$ generate φ. This means that $\hat{\hat{\varphi}} = \varphi$. By definition $\hat{\hat{f}} = f$, so there is a one-one correspondence between mappings and filters. (This correspondence is obviously inclusion preserving, too.) □

We now know just about everything about $|D_0 \to D_1|$ as a domain: the elements correspond isomorphically to the approximable mappings; the finite elements are explained completely by 3.9; and we have seen how to calculate with neighbourhoods. The final step is to relate the function space to other domains by appropriate mappings. In doing this we shall freely construe elements of $|D_0 \to D_1|$ as approximable mappings in view of 3.10.

The function-space element isomorphism is with the function-space element isomorphism peer.

Theorem (`theorem_3_10`). Ideal elements of a function space correspond bijectively and order-preservingly to approximable maps.

Proof. Invoke the function-space element isomorphism. ■

`Theorem_3_10.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.funSpaceEquiv

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

/-- Scott's Theorem 3.10: every ideal element of a function space is fixed by a unique
    approximable mapping, via the order isomorphism with maps. -/
theorem theorem_3_10 :
    ∀ {α β : Type*} (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β),
      Nonempty ((funSpace V₀ V₁).Element ≃o ApproximableMap V₀ V₁) :=
  fun {_ _} V₀ V₁ => ⟨funSpaceEquiv V₀ V₁⟩

end DomainTheory.NeighbourhoodSystems
```

4.4.19 Evaluation combinator

Evaluation of an approximable map at an argument, as a binary approximable map on the function space.

Definition (`eval`). Evaluation relates a function-space neighbourhood, a domain neighbourhood, and a codomain neighbourhood precisely when every map in the former relates the latter two.

Proof. Domain and codomain membership unpack the conjuncts; the masters relate by the masters of each system and the master law for each map; intersection on the right and monotonicity use the corresponding laws for each map in the function-space neighbourhood. ■

Definition (`evalMap`). Evaluation as a joint approximable map out of the product of a function space with its domain is the product form of binary evaluation.

Proof. Apply the product encoding of binary approximable maps. ■

Theorem (`toElementMap2_eval`). Binary evaluation of an ideal element of a function space at an ideal element of the domain recovers application of the corresponding approximable map.

Proof. Compare membership: upward closure with a step neighbourhood yields one inclusion; the converse witnesses evaluation at that step. ■

Theorem (`evalMap_apply`). Joint evaluation at a paired ideal element equals application of the recovered approximable map.

Proof. Unpack the product encoding and apply the binary evaluation identity. ■

Theorem (`toElementMap_evalMap_comp_paired`). Evaluation after pairing a map into a function space with a map into the domain acts elementwise by applying the recovered value of the first to the value of the second.

Proof. Compose the elementwise pairing and evaluation identities. ■

`eval.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.funSpaceEquiv
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.
    ApproximableMap2
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.ProjectionMappings
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {V₂ : NeighborhoodSystem ?}

def eval (V₁ : NeighborhoodSystem β) (V₂ : NeighborhoodSystem ?) :
    ApproximableMap₂ (funSpace V₁ V₂) V₁ V₂ where
  rel F X Y := (funSpace V₁ V₂).mem F ∧ V₁.mem X ∧ V₂.mem Y ∧ ∀ f ∈ F, f.rel X Y
  rel_dom₀ h := h.1
  rel_dom₁ h := h.2.1
  rel_cod h := h.2.2.1
  master_rel := ⟨(funSpace V₁ V₂).master_mem, V₁.master_mem, V₂.master_mem,
    fun f _ => f.master_rel⟩
  inter_right := by
    rintro F X Y Y' ⟨hF, hX, hY, hrel⟩ ⟨_, _, hY', hrel'⟩
    obtain ⟨f, hf⟩ := (funSpace_mem_iff.mp hF).2
    refine ⟨hF, hX, ?_, fun g hg => g.inter_right (hrel g hg) (hrel' g hg)⟩
    exact f.rel_cod (f.inter_right (hrel f hf) (hrel' f hf))
  mono := by
    rintro F F' X X' Y Y' ⟨hF, hX, hY, hrel⟩ hF'F hX'X hYY' hF' hX' hY'
    exact ⟨hF', hX', hY', fun f hf => f.mono (hrel f (hF'F hf)) hX'X hYY' hX' hY'⟩

def evalMap (V₁ : NeighborhoodSystem β) (V₂ : NeighborhoodSystem ?) :
    ApproximableMap (prod (funSpace V₁ V₂) V₁) V₂ := ofMap₂ (eval V₁ V₂)

theorem toElementMap₂_eval (φ : (funSpace V₁ V₂).Element) (x : V₁.Element) :
    (eval V₁ V₂).toElementMap₂ φ x = (toApproxMap φ).toElementMap x := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨F, X, hφF, hxX, _, hX, hY, hrel⟩
    refine ⟨X, hxX, ?_⟩
    show φ.mem (step X Y)
    exact φ.up_mem hφF (step_mem hX hY) (fun f hf => hrel f hf)
  · rintro ⟨X, hxX, hrel⟩
    have hstep : (funSpace V₁ V₂).mem (step X Y) := φ.sub hrel
    obtain ⟨f, hf⟩ := (funSpace_mem_iff.mp hstep).2
    exact ⟨step X Y, X, hrel, hxX, hstep, f.rel_dom hf, f.rel_cod hf, fun g hg => hg⟩

theorem evalMap_apply (φ : (funSpace V₁ V₂).Element) (x : V₁.Element) :
    (evalMap V₁ V₂).toElementMap (pair φ x) = (toApproxMap φ).toElementMap x := by
  rw [evalMap, ← toElementMap₂_toMap₂, toMap₂_ofMap₂, toElementMap₂_eval]

theorem toElementMap_evalMap_comp_paired (F : ApproximableMap V₀ (funSpace V₁ V₂))
    (G : ApproximableMap V₀ V₁) (w : V₀.Element) :
    ((evalMap V₁ V₂).comp (paired F G)).toElementMap w =
      (toApproxMap (F.toElementMap w)).toElementMap (G.toElementMap w) := by
  rw [toElementMap_comp, toElementMap_paired, evalMap_apply]

end DomainTheory.NeighbourhoodSystems
```

4.4.20 Theorem 3.11 — evaluation combinator

THEOREM 3.11. Given neighbourhood systems D_1 and D_2, there is a uniquely determined approximable mapping

$$\text{eval} : (D_1 \to D_2) \times D_1 \to D_2,$$

where for all $f : D_1 \to D_2$ and all $x \in |D_1|$ we have

(i) $\text{eval}(f, x) = f(x)$.

Proof: For $F \in (D_1 \to D_2)$ and $X \in D_1$ and $Y \in D_2$ define eval as a relation by:

$$F, X \text{ eval } Y \quad \text{iff} \quad X f Y \text{ for all } f \in F.$$

Remember that neighbourhoods in the function space are sets of approximable mappings. It is easily checked that this definition makes eval approximable. We now calculate the function values by the formula of 2.2 (i):

$$\text{eval}(f, x) = \{Y \in D_2 \mid \exists F \in (D_1 \to D_2)\, \exists X \in x.\ f \in F \text{ and } F \cup X \text{ eval } Y\}$$

Because, again by 2.2 (i), we have

$$f(x) = \{Y \in D_2 \mid \exists X \in x.\ X f Y\},$$

we can see from the definition of eval that $\text{eval}(f, x) \subseteq f(x)$. Suppose that $Y \in f(x)$. Then $X f Y$ holds for some $X \in x$. We can write $f \in [X, Y] \in (D_1 \to D_2)$ and it is clear that

$$[X, Y] \cup X \text{ eval } Y$$

holds by definition. Therefore, $Y \in \text{eval}(f, x)$, and so $f(x) \subseteq \text{eval}(f, x)$. □

This theorem is essential for our programme: it shows that in taking functions as objects the very basic operation of forming the function value is an approximable mapping. In other words we can treat the expression $f(x)$ not just as a function of x, as we have done from the start, but also as a function of f as well. The result also indicates that there are useful maps defined on domains that themselves are function spaces; we shall meet many more of these. The next theorem provides further examples.

The evaluation combinator is with the evaluation combinator peer.

Theorem (`theorem_3_11`). Joint evaluation of an ideal element of a function space at an ideal element of the domain equals application of the recovered approximable map.

Proof. Invoke the joint evaluation identity. ■

Theorem_3_11.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.eval

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
```

```
/-- Scott's Theorem 3.11: evaluation is an approximable map satisfying eval(f, x) = f(
    x). -/
theorem theorem_3_11 :
    ∀ {β ? : Type*} {V₁ : NeighborhoodSystem β} {V₂ : NeighborhoodSystem ?}
      (φ : (funSpace V₁ V₂).Element) (x : V₁.Element),
      (evalMap V₁ V₂).toElementMap (pair φ x) = (toApproxMap φ).toElementMap x :=
  @evalMap_apply

end DomainTheory.NeighbourhoodSystems
```

4.4.21 Currying

Currying and uncurrying for approximable maps, with the adjunction to evaluation.

Definition (`gSection`). Fixing a domain neighbourhood of a product map yields an approximable map on the second factor by relating neighbourhoods precisely when the product neighbourhood of the fixed factor with the second relates under the original map.

Proof. Domain and codomain membership, master, intersection on the right, and monotonicity inherit from the product map via the product-neighbourhood laws. ■

Theorem (`gSection_rel`). Membership in a section relation is membership of the corresponding product neighbourhood in the original relation.

Proof. By definition. ■

Theorem (`gSection_le`). Enlarging the fixed domain neighbourhood yields a smaller section.

Proof. Monotonicity of the product map in the product neighbourhood. ■

Definition (`curry`). The curry of a product map relates a domain neighbourhood to a function-space neighbourhood precisely when the section at that neighbourhood belongs to the latter.

Proof. Domain and codomain membership unpack the witnesses; the masters relate by the masters of each system; intersection on the right uses intersection of function-space neighbourhoods; monotonicity uses upward closure and the section comparison. ■

Theorem (`curry_rel`). Membership in the curry relation is domain membership, function-space membership, and section membership.

Proof. By definition. ■

Theorem (`rel_stepFun_iff`). Relating a domain neighbourhood to a step-function neighbourhood is equivalent to relating it to each constituent step.

Proof. By induction on the witnessing list, using the master law, intersection on the right, and monotonicity. ■

Definition (`uncurry`). The uncurry of a map into a function space relates a product neighbourhood to a codomain neighbourhood precisely when the map relates the left projection to the step of the right projection with that codomain neighbourhood.

Proof. Domain membership unpacks the product conjunct; codomain membership unpacks a witnessing map in the step; the masters relate via the product master and the master step; intersection on the right and monotonicity use the corresponding laws for the map into the function space together with step intersection and nesting. ■

Theorem (`uncurry_rel`). Membership in the uncurry relation is product membership together with the projected step relation.

Proof. By definition. ■

Theorem (`curry_uncurry`). Currying after uncurrying recovers the original map into the function space.

Proof. Compare relations via the step-function characterisation. ■

Theorem (`uncurry_curry`). Uncurrying after currying recovers the original product map.

Proof. Compare relations via the product-neighbourhood preimage law. ■

Definition (`curryEquiv`). Product maps are order-isomorphic to maps into the function space via curry and uncurry.

Proof. The round trips are the previous identities; order equivalence unpacks approximation through sections and steps. ■

Theorem (`toElementMap_curry_apply`). Currying acts elementwise by sending a pair of ideal elements to the value of the product map at their pairing.

Proof. Compare membership of codomain neighbourhoods via product neighbourhoods and steps. ■

Theorem (`uncurry_eq`). Uncurry is evaluation after pairing with the projections.

Proof. Compare elementwise via the pairing, projection, and evaluation identities. ■

Theorem (`curry_eval_comp`). Currying evaluation after pairing with the projections recovers the original map into the function space.

Proof. Rewrite via the uncurry equation and the curry–uncurry round trip. ■

Theorem (`eval_comp_curry`). Evaluation after pairing the curry of a product map with the projections recovers that product map.

Proof. Rewrite via the uncurry equation and the uncurry–curry round trip. ■

Theorem (`ApproximableMap.le_iff`). Approximation of approximable maps is pointwise containment of relations.

Proof. By definition. ■

curry.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.eval

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {V₂ : NeighborhoodSystem ?}

def gSection (g : ApproximableMap (prod V₀ V₁) V₂) {X : Set α} (hX : V₀.mem X) :
    ApproximableMap V₁ V₂ where
  rel Y Z := g.rel (prodNbhd X Y) Z
  rel_dom h := (prod_mem_prodNbhd_iff.mp (g.rel_dom h)).2
  rel_cod h := g.rel_cod h
  master_rel := g.rel_master (prod_mem_prodNbhd hX V₁.master_mem)
  inter_right h h' := g.inter_right h h'
  mono := by
    intro Y Y' Z Z' h hY'Y hZZ' hY' hZ'
    exact g.mono h (prodNbhd.subset_iff.mpr ⟨subset_rfl, hY'Y⟩) hZZ'
      (prod_mem_prodNbhd hX hY') hZ'
```

```
@[simp] theorem gSection_rel {g : ApproximableMap (prod V₀ V₁) V₂} {X : Set α} (hX : V₀
    .mem X)
    {Y : Set β} {Z : Set ?} : (gSection g hX).rel Y Z ↔ g.rel (prodNbhd X Y) Z := Iff.
    rfl

theorem gSection_le {g : ApproximableMap (prod V₀ V₁) V₂} {X X' : Set α}
    (hX : V₀.mem X) (hX' : V₀.mem X') (hX'X : X' ⊆ X) : gSection g hX ≤ gSection g hX'
     := by
  intro Y Z h
  have hY := (prod_mem_prodNbhd_iff.mp (g.rel_dom h)).2
  exact g.mono h (prodNbhd.subset_iff.mpr ⟨hX'X, subset_rfl⟩) subset_rfl
    (prod_mem_prodNbhd hX' hY) (g.rel_cod h)

def curry (g : ApproximableMap (prod V₀ V₁) V₂) : ApproximableMap V₀ (funSpace V₁ V₂)
    where
  rel X W := ∃ hX : V₀.mem X, (funSpace V₁ V₂).mem W ∧ gSection g hX ∈ W
  rel_dom := fun ⟨hX, _⟩ => hX
  rel_cod := fun ⟨_, hW, _⟩ => hW
  master_rel := ⟨V₀.master_mem, (funSpace V₁ V₂).master_mem, Set.mem_univ _⟩
  inter_right := by
    rintro X W W' ⟨hX, hW, hmem⟩ ⟨_, hW', hmem'⟩
    exact ⟨hX, funSpace_mem_inter hW hW' ⟨gSection g hX, hmem, hmem'⟩,
      Set.mem_inter hmem hmem'⟩
  mono := by
    rintro X X' W W' ⟨hX, hW, hmem⟩ hX'X hWW' hX' hW'
    exact ⟨hX', hW', funSpace_mem_up_closed hW' (hWW' hmem) (gSection_le hX hX' hX'X)⟩

@[simp] theorem curry_rel {g : ApproximableMap (prod V₀ V₁) V₂} {X : Set α}
    {W : Set (ApproximableMap V₁ V₂)} :
    (curry g).rel X W ↔ ∃ hX : V₀.mem X, (funSpace V₁ V₂).mem W ∧ gSection g hX ∈ W :=
     Iff.rfl

theorem rel_stepFun_iff (h : ApproximableMap V₀ (funSpace V₁ V₂)) {X : Set α} (hX : V₀.
    mem X)
    {L : List (Set β × Set ?)} (hL : ∀ p ∈ L, V₁.mem p.1 ∧ V₂.mem p.2) :
    h.rel X (stepFun L) ↔ ∀ p ∈ L, h.rel X (step p.1 p.2) := by
  induction L with
  | nil =>
    simp only [stepFun_nil, List.not_mem_nil, IsEmpty.forall_iff, implies_true,
    iff_true]
    show h.rel X (funSpace V₁ V₂).master
    exact h.rel_master hX
  | cons p L ih =>
    rw [stepFun_cons]
    have hp := hL p (List.mem_cons.mpr (Or.inl rfl))
    have hLtail : ∀ q ∈ L, V₁.mem q.1 ∧ V₂.mem q.2 :=
      fun q hq => hL q (List.mem_cons.mpr (Or.inr hq))
    constructor
    · intro hmem
      have hne : (step p.1 p.2 ∩ stepFun L).Nonempty := (h.rel_cod hmem).2
      have hstep : h.rel X (step p.1 p.2) :=
        h.mono hmem subset_rfl Set.inter_subset_left hX (step_mem hp.1 hp.2)
      have htail : h.rel X (stepFun L) :=
        h.mono hmem subset_rfl Set.inter_subset_right hX
          ⟨⟨L, hLtail, rfl⟩, hne.mono Set.inter_subset_right⟩
      intro q hq
```

```
      rcases List.mem_cons.mp hq with rfl | hq
      · exact hstep
      · exact (ih hLtail).mp htail q hq
    · intro hall
      have hstep : h.rel X (step p.1 p.2) := hall p (List.mem_cons.mpr (Or.inl rfl))
      have htail : h.rel X (stepFun L) :=
        (ih hLtail).mpr (fun q hq => hall q (List.mem_cons.mpr (Or.inr hq)))
      exact h.inter_right hstep htail

def uncurry (h : ApproximableMap V₀ (funSpace V₁ V₂)) : ApproximableMap (prod V₀ V₁) V₂
    where
  rel W Z := (prod V₀ V₁).mem W ∧ h.rel (Sum.inl ^-1' W) (step (Sum.inr ^-1' W) Z)
  rel_dom h' := h'.1
  rel_cod := by
    rintro W Z ⟨_, hrel⟩
    obtain ⟨f, hf⟩ := (funSpace_mem_iff.mp (h.rel_cod hrel)).2
    exact f.rel_cod hf
  master_rel := by
    refine ⟨(prod V₀ V₁).master_mem, ?_⟩
    rw [show (prod V₀ V₁).master = prodNbhd V₀.master V₁.master from rfl,
      prodNbhd.inl_preimage, prodNbhd.inr_preimage, step_master_eq]
    exact h.master_rel
  inter_right := by
    rintro W Z Z' ⟨hW, hrel⟩ ⟨_, hrel'⟩
    obtain ⟨f, hf⟩ := (funSpace_mem_iff.mp (h.rel_cod hrel)).2
    obtain ⟨f', hf'⟩ := (funSpace_mem_iff.mp (h.rel_cod hrel')).2
    refine ⟨hW, ?_⟩
    rw [← step_inter_right (f.rel_cod hf) (f'.rel_cod hf')]
    exact h.inter_right hrel hrel'
  mono := by
    rintro W W₂ Z Z' ⟨_, hrel⟩ hW₂W hZZ' hW₂ hZ'
    have hinl : Sum.inl ^-1' W₂ ⊆ Sum.inl ^-1' W := Set.preimage_mono hW₂W
    have hinr : Sum.inr ^-1' W₂ ⊆ Sum.inr ^-1' W := Set.preimage_mono hW₂W
    obtain ⟨A, B, hA, hB, rfl⟩ := hW₂
    rw [prodNbhd.inl_preimage] at hinl ⊢
    rw [prodNbhd.inr_preimage] at hinr ⊢
    refine ⟨prod_mem_prodNbhd hA hB, ?_⟩
    exact h.mono hrel hinl (step_subset hB hZ' hinr hZZ') hA (step_mem hB hZ')

@[simp] theorem uncurry_rel {h : ApproximableMap V₀ (funSpace V₁ V₂)}
    {W : Set (α ? β)} {Z : Set ?} :
    (uncurry h).rel W Z ↔
      (prod V₀ V₁).mem W ∧ h.rel (Sum.inl ^-1' W) (step (Sum.inr ^-1' W) Z) := Iff.rfl

theorem curry_uncurry (h : ApproximableMap V₀ (funSpace V₁ V₂)) : curry (uncurry h) = h
    := by
  apply ApproximableMap.ext
  intro X W
  constructor
  · rintro ⟨hX, hW, hmem⟩
    obtain ⟨⟨L, hL, rfl⟩, _⟩ := hW
    refine (rel_stepFun_iff h hX hL).mpr (fun p hp => ?_)
    have := hmem p hp
    have hrel : (uncurry h).rel (prodNbhd X p.1) p.2 := this
    obtain ⟨_, hrel'⟩ := hrel
    rw [prodNbhd.inl_preimage, prodNbhd.inr_preimage] at hrel'
```

```
    exact hrel'
  · intro hrel
    have hX : V₀.mem X := h.rel_dom hrel
    have hW : (funSpace V₁ V₂).mem W := h.rel_cod hrel
    refine ⟨hX, hW, ?_⟩
    obtain ⟨⟨L, hL, rfl⟩, _⟩ := hW
    intro p hp
    show (uncurry h).rel (prodNbhd X p.1) p.2
    refine ⟨prod_mem_prodNbhd hX (hL p hp).1, ?_⟩
    rw [prodNbhd.inl_preimage, prodNbhd.inr_preimage]
    exact (rel_stepFun_iff h hX hL).mp hrel p hp

theorem uncurry_curry (g : ApproximableMap (prod V₀ V₁) V₂) : uncurry (curry g) = g :=
    by
  apply ApproximableMap.ext
  intro W Z
  constructor
  · rintro ⟨hW, _, _, hmem⟩
    rw [prodNbhd.preimage hW]; exact hmem
  · intro hrel
    have hW := g.rel_dom hrel
    refine ⟨hW, prod_mem_inl hW, step_mem (prod_mem_inr hW) (g.rel_cod hrel), ?_⟩
    show g.rel (prodNbhd (Sum.inl ^-1' W) (Sum.inr ^-1' W)) Z
    rwa [← prodNbhd.preimage hW]

def curryEquiv (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (V₂ : NeighborhoodSystem ?) :
    ApproximableMap (prod V₀ V₁) V₂ ≃o ApproximableMap V₀ (funSpace V₁ V₂) where
  toFun := curry
  invFun := uncurry
  left_inv := uncurry_curry
  right_inv := curry_uncurry
  map_rel_iff' := by
    intro g g'
    constructor
    · intro hcurry W Z hrel
      have h1 : (curry g).rel (Sum.inl ^-1' W) (step (Sum.inr ^-1' W) Z) := by
        have hu : (uncurry (curry g)).rel W Z := by rw [uncurry_curry]; exact hrel
        exact hu.2
      have h2 := hcurry _ _ h1
      have hu' : (uncurry (curry g')).rel W Z := ⟨g.rel_dom hrel, h2⟩
      rw [uncurry_curry] at hu'
      exact hu'
    · intro hg X W hrel
      obtain ⟨hX, hW, hmem⟩ := hrel
      refine ⟨hX, hW, ?_⟩
      obtain ⟨⟨L, hL, rfl⟩, _⟩ := hW
      intro p hp
      have hgrel : g.rel (prodNbhd X p.1) p.2 := hmem p hp
      exact hg _ _ hgrel

theorem toElementMap_curry_apply (g : ApproximableMap (prod V₀ V₁) V₂)
    (x : V₀.Element) (y : V₁.Element) :
    (toApproxMap ((curry g).toElementMap x)).toElementMap y = g.toElementMap (pair x y
    ) := by
  apply Element.ext
```

```
  intro Z
  constructor
  · rintro ⟨Y, hyY, X, hxX, hX, _, hrel⟩
    exact ⟨prodNbhd X Y, ⟨X, Y, hxX, hyY, rfl⟩, hrel⟩
  · rintro ⟨W, ⟨X, Y, hxX, hyY, rfl⟩, hrel⟩
    exact ⟨Y, hyY, X, hxX, x.sub hxX, step_mem (y.sub hyY) (g.rel_cod hrel), hrel⟩

theorem uncurry_eq (h : ApproximableMap V₀ (funSpace V₁ V₂)) :
    uncurry h = (evalMap V₁ V₂).comp (paired (h.comp (proj₀ V₀ V₁)) (proj₁ V₀ V₁)) := by
  apply ext_of_toElementMap
  intro w
  rw [toElementMap_comp, toElementMap_paired, toElementMap_comp, toElementMap_proj₀,
    toElementMap_proj₁, evalMap_apply]
  apply Element.ext
  intro Z
  constructor
  · rintro ⟨W₀, hwW₀, hW₀, hrel⟩
    obtain ⟨X, Y, hX, hY, rfl⟩ := hW₀
    rw [prodNbhd.inl_preimage] at hrel
    rw [prodNbhd.inr_preimage] at hrel
    obtain ⟨hwX, hwY⟩ := (prod_mem_split hX hY).mp hwW₀
    exact ⟨Y, ⟨hY, hwY⟩, X, ⟨hX, hwX⟩, hrel⟩
  · rintro ⟨Y, ⟨hY, hwY⟩, X, ⟨hX, hwX⟩, hrel⟩
    refine ⟨prodNbhd X Y, (prod_mem_split hX hY).mpr ⟨hwX, hwY⟩, prod_mem_prodNbhd hX
    hY, ?_⟩
    rw [prodNbhd.inl_preimage, prodNbhd.inr_preimage]
    exact hrel

theorem curry_eval_comp (h : ApproximableMap V₀ (funSpace V₁ V₂)) :
    curry ((evalMap V₁ V₂).comp (paired (h.comp (proj₀ V₀ V₁)) (proj₁ V₀ V₁))) = h := by
  rw [← uncurry_eq, curry_uncurry]

theorem eval_comp_curry (g : ApproximableMap (prod V₀ V₁) V₂) :
    (evalMap V₁ V₂).comp (paired ((curry g).comp (proj₀ V₀ V₁)) (proj₁ V₀ V₁)) = g := by
  rw [← uncurry_eq]; exact uncurry_curry g

theorem ApproximableMap.le_iff {f g : ApproximableMap V₀ V₁} :
    f ≤ g ↔ ∀ X Y, f.rel X Y → g.rel X Y := Iff.rfl

end DomainTheory.NeighbourhoodSystems
```

4.4.22 Theorem 3.12 — currying

THEOREM 3.12. Given neighbourhood systems D_0, D_1, D_2 there is associated with every approximable mapping $g : D_0 \times D_1 \to D_2$ a uniquely determined approximable mapping

$$\mathrm{curry}(g) : D_0 \to (D_1 \to D_2)$$

such that for $x \in |D_0|$ and $y \in |D_1|$

(i) $\mathrm{curry}(g)(x)(y) = g(x, y)$.

Moreover we have these functional equations:

(ii) $\text{eval} \circ \langle \text{curry}(g) \circ p_0,\ p_1 \rangle = g$, and

(iii) $\text{curry}(\text{eval} \circ \langle h \circ p_0,\ p_1 \rangle) = h$,

where the $p_i : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_i$ are the projection mappings and $h : \mathcal{D}_0 \to (\mathcal{D}_1 \to \mathcal{D}_2)$ is any approximable mapping. This provides an isomorphism between the domains $|\mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2|$ and $|\mathcal{D}_0 \to (\mathcal{D}_1 \to \mathcal{D}_2)|$ and so we can regard

$$\text{curry} : (\mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2) \to (\mathcal{D}_0 \to (\mathcal{D}_1 \to \mathcal{D}_2))$$

as itself being an approximable mapping.

Proof: Given g as indicated, we can define $\text{curry}(g)$ as a relation and as an approximable mapping by:

$$X\ \text{curry}(g)\ [Y, Z] \text{ iff } X \cup Y\ g\ Z \quad \text{(but see Ex. 3.21)}$$

for all $X \in \mathcal{D}_0$, $Y \in \mathcal{D}_1$, $Z \in \mathcal{D}_2$. This is sufficient because an approximable mapping is intersective in the right-hand neighbourhood, so we know from the above exactly what $X\ \text{curry}(g)\ \bigcap\{[Y_i, Z_i] \mid i < n\}$ means for all finite intersections. The remark after 3.9 is then helpful in checking that by this definition $\text{curry}(g)$ satisfies the monotonicity condition and so is indeed approximable. We now calculate:

$$\begin{aligned}
\text{curry}(g)(x)(y) &= \{Z \in \mathcal{D}_2 \mid \exists Y \in y.\ Y\ \text{curry}(g)\ (x)\ Z\} \\
&= \{Z \in \mathcal{D}_2 \mid \exists Y \in y\ \exists X \in x.\ X\ \text{curry}(g)\ [Y, Z]\} \\
&= \{Z \in \mathcal{D}_2 \mid \exists Y \in y\ \exists X \in x.\ X \cup Y\ g\ Z\} \\
&= \{Z \in \mathcal{D}_2 \mid \exists W \in \langle x, y \rangle.\ W\ g\ Z\} \\
&= g(\langle x, y \rangle) = g(x, y).
\end{aligned}$$

This proves (i). We also see, that if we take the left-hand side of (ii) and apply it to a pair $\langle x, y \rangle$, it reduces to $g(x, y)$ by virtue of (i). Thus, the two functions in (ii) are the same.

Turning to (iii), call the left-hand side k. Using (i) again, we find

$$\begin{aligned}
k(x)(y) &= \text{eval} \circ \langle h \circ p_0, p_1 \rangle(\langle x, y \rangle) \\
&= \text{eval}(\langle h \circ p_0(\langle x, y \rangle), p_1(\langle x, y \rangle) \rangle) \\
&= \text{eval}(\langle h(x), y \rangle) \\
&= h(x)(y).
\end{aligned}$$

As this is true for all $y \in |\mathcal{D}_1|$, then $k(x) = h(x)$ follows. As this is true for all $x \in |\mathcal{D}_0|$, then $k = h$ follows, and (iii) is proved.

Taking (ii) and (iii) together, it is clear that the domains $|D_0 \times D_1 \to D_2|$ and $|D_0 \to (D_1 \to D_2)|$ are in a one-one correspondence.

$$\text{curry}(g) \sqsubseteq \text{curry}(g') \text{ iff } g \sqsubseteq g'.$$

Hence, curry is an isomorphism, and we can invoke 2.7 to conclude that it comes from an approximable mapping. □

We close this lecture with some order-theoretic properties of function spaces that characterize inclusion and upper bounds of functions in a "pointwise" manner.

The currying library is with the currying peer.

Theorem (`theorem_3_12`). Currying acts elementwise by pairing, evaluation after pairing with the projections inverts curry and conversely, and product maps are order-isomorphic to maps into the function space.

Proof. Assemble the elementwise curry law, the two eval/curry equations, and the curry order isomorphism. ■

Theorem_3_12.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.curry

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Theorem 3.12: curry packages the elementwise law, the eval/curry equations
    ,
    and the order isomorphism with maps out of a product. -/
theorem theorem_3_12 :
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (g : ApproximableMap (prod V₀ V₁) V₂)
      (x : V₀.Element) (y : V₁.Element),
      (toApproxMap ((curry g).toElementMap x)).toElementMap y = g.toElementMap (pair x
     y)) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (g : ApproximableMap (prod V₀ V₁) V₂),
      (evalMap V₁ V₂).comp (paired ((curry g).comp (proj₀ V₀ V₁)) (proj₁ V₀ V₁)) = g) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (h : ApproximableMap V₀ (funSpace V₁ V₂)),
      curry ((evalMap V₁ V₂).comp (paired (h.comp (proj₀ V₀ V₁)) (proj₁ V₀ V₁))) = h) ∧
    (∀ {α β ? : Type*} (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
      (V₂ : NeighborhoodSystem ?),
      Nonempty (ApproximableMap (prod V₀ V₁) V₂ ≃o ApproximableMap V₀ (funSpace V₁ V₂)))
    :=
  ⟨@toElementMap_curry_apply, @eval_comp_curry, @curry_eval_comp,
    fun {_ _ _} V₀ V₁ V₂ => ⟨curryEquiv V₀ V₁ V₂⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.4.23 Pointwise supremum of maps

Pointwise order and bounded supremum of approximable maps.

Definition (`MapsBounded`). A set of approximable maps is bounded when some approximable map dominates every member.

Proof. By definition. ■

Definition (`PointwiseBounded`). A set of approximable maps is pointwise bounded when, at every ideal element of the domain, the set of values is bounded in the codomain.

Proof. By definition. ■

Theorem (`mapsBounded_principal`). Pointwise boundedness implies boundedness of the values at every principal ideal element.

Proof. Specialise pointwise boundedness at the principal. ■

Definition (`supOnPrincipal`). At a domain neighbourhood, the supremum on principals of a pointwise-bounded family is the codomain supremum of the values of the family at that principal.

Proof. Apply the codomain supremum using the principal boundedness. ■

Theorem (`supOnPrincipal_mono`). Shrinking the domain neighbourhood raises the supremum on principals.

Proof. Bound the larger supremum below by comparing values via monotonicity of each map at nested principals. ■

Definition (`sSupMaps`). The pointwise supremum of a pointwise-bounded family is the monotone approximable map determined by the supremum on principals.

Proof. Apply the monotone-to-approximable encoding with the principal-supremum comparison. ■

Theorem (`le_iff_toElementMap_le`). Approximation of approximable maps is equivalent to pointwise approximation of their values.

Proof. Forward, apply a relating neighbourhood at each argument; converse, evaluate at the principal of a relating domain neighbourhood. ■

Theorem (`le_sSupMaps`). Each member of a pointwise-bounded family is below the pointwise supremum.

Proof. A relating neighbourhood of a member belongs to its value at the corresponding principal, hence to the principal supremum. ■

Theorem (`sSupMaps_le`). The pointwise supremum is below every common upper bound of the family.

Proof. Bound the principal supremum by the value of the upper bound at that principal, using pointwise approximation. ■

Theorem (`mapsBounded_iff_pointwiseBounded`). Boundedness of a family of maps is equivalent to pointwise boundedness of its values.

Proof. Forward, apply pointwise approximation at each argument; converse, take the pointwise supremum as a common upper bound. ■

Theorem (`toElementMap_sSupMaps_principal`). The value of the pointwise supremum at a principal is the supremum on that principal.

Proof. Unpack the monotone encoding at principals. ■

Theorem (`toElementMap_sSupMaps`). The value of the pointwise supremum at an ideal element is the codomain supremum of the values of the family there.

Proof. Compare via principals below the argument and the membership of each family member under the pointwise supremum. ■

Theorem (`toElementMap_sSupMaps'`). Under mere boundedness of the family, the same pointwise-supremum identity holds after converting to pointwise boundedness.

Proof. Apply the previous identity via the boundedness equivalence. ■

Theorem (`toFilter_le_iff`). Approximation of maps is equivalent to approximation of their principal filters in the function space.

Proof. Unpack the order isomorphism between maps and function-space elements. ■

Theorem (`mapsBounded_to_filters`). Boundedness of a family of maps implies boundedness of the corresponding filters in the function space.

Proof. The filter of a common upper bound dominates each converted filter. ■

`sSupMaps.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofMono
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.funSpaceEquiv

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def MapsBounded (F : Set (ApproximableMap V₀ V₁)) : Prop := ∃ h, ∀ f ∈ F, f ≤ h

def PointwiseBounded (F : Set (ApproximableMap V₀ V₁)) : Prop :=
  ∀ x : V₀.Element, V₁.Bounded (Set.image (fun f => f.toElementMap x) F)

theorem mapsBounded_principal {F : Set (ApproximableMap V₀ V₁)} (hF : PointwiseBounded
    F)
    {X : Set α} (hX : V₀.mem X) :
    V₁.Bounded (Set.image (fun f => f.toElementMap (V₀.principal hX)) F) :=
  hF (V₀.principal hX)

def supOnPrincipal (F : Set (ApproximableMap V₀ V₁)) (hF : PointwiseBounded F)
    (X : Set α) (hX : V₀.mem X) : V₁.Element :=
  V₁.sSup (Set.image (fun f => f.toElementMap (V₀.principal hX)) F) (
    mapsBounded_principal hF hX)

theorem supOnPrincipal_mono (F : Set (ApproximableMap V₀ V₁)) (hF : PointwiseBounded F)
    (X X' : Set α) (hX : V₀.mem X) (hX' : V₀.mem X') (hX'X : X' ⊆ X) :
    supOnPrincipal F hF X hX ≤ supOnPrincipal F hF X' hX' :=
  V₁.sSup_le _ (mapsBounded_principal hF hX) fun s hs => by
    obtain ⟨f, hf, rfl⟩ := hs
    exact (toElementMap_mono f ((V₀.principal_le_iff hX hX').mpr hX'X)).trans
      (V₁.le_sSup _ (mapsBounded_principal hF hX') ⟨f, hf, rfl⟩)

def sSupMaps (F : Set (ApproximableMap V₀ V₁)) (hF : PointwiseBounded F) :
    ApproximableMap V₀ V₁ :=
  ofMono (fun X hX => supOnPrincipal F hF X hX) (supOnPrincipal_mono F hF)

theorem le_iff_toElementMap_le {f g : ApproximableMap V₀ V₁} :
    f ≤ g ↔ ∀ x, f.toElementMap x ≤ g.toElementMap x := by
  constructor
  · intro h x Y ⟨X, hXx, hrel⟩
    exact ⟨X, hXx, h X Y hrel⟩
  · intro h X Y hrel
    have hX : V₀.mem X := f.rel_dom hrel
    rw [f.rel_iff_mem_principal hX] at hrel
    rw [g.rel_iff_mem_principal hX]
    exact h (V₀.principal hX) Y hrel
```

```
theorem le_sSupMaps {F : Set (ApproximableMap V₀ V₁)} (hF : PointwiseBounded F)
    {f : ApproximableMap V₀ V₁} (hf : f ∈ F) : f ≤ sSupMaps F hF := by
  intro X Y hrel
  have hX : V₀.mem X := f.rel_dom hrel
  have hmem : (f.toElementMap (V₀.principal hX)).mem Y := (f.rel_iff_mem_principal hX).
    mp hrel
  exact ⟨hX, (V₁.le_sSup _ (mapsBounded_principal hF hX) ⟨f, hf, rfl⟩) Y hmem⟩

theorem sSupMaps_le {F : Set (ApproximableMap V₀ V₁)} (hF : PointwiseBounded F)
    {h : ApproximableMap V₀ V₁} (hh : ∀ f ∈ F, f ≤ h) : sSupMaps F hF ≤ h := by
  intro X Y hrel
  obtain ⟨hX, hYmem⟩ := hrel
  have hle : supOnPrincipal F hF X hX ≤ h.toElementMap (V₀.principal hX) :=
    V₁.sSup_le _ (mapsBounded_principal hF hX) fun s hs => by
      obtain ⟨f, hf, rfl⟩ := hs
      exact (le_iff_toElementMap_le.mp (hh f hf)) (V₀.principal hX)
  exact (h.rel_iff_mem_principal hX).mpr (hle Y hYmem)

theorem mapsBounded_iff_pointwiseBounded {F : Set (ApproximableMap V₀ V₁)} :
    MapsBounded F ↔ PointwiseBounded F := by
  constructor
  · intro ⟨h, hh⟩ x
    refine ⟨h.toElementMap x, fun z hz => ?_⟩
    obtain ⟨f, hf, rfl⟩ := hz
    exact (le_iff_toElementMap_le.mp (hh f hf)) x
  · intro hpb
    refine ⟨sSupMaps F hpb, fun f hf X Y hrel => ?_⟩
    have hX : V₀.mem X := f.rel_dom hrel
    have hmem : (f.toElementMap (V₀.principal hX)).mem Y := (f.rel_iff_mem_principal hX
    ).mp hrel
    exact ⟨hX, (V₁.le_sSup _ (mapsBounded_principal hpb hX) ⟨f, hf, rfl⟩) Y hmem⟩

theorem toElementMap_sSupMaps_principal {F : Set (ApproximableMap V₀ V₁)} (hF :
    PointwiseBounded F)
    {X : Set α} (hX : V₀.mem X) :
    (sSupMaps F hF).toElementMap (V₀.principal hX) = supOnPrincipal F hF X hX :=
  toElementMap_ofMono_principal _ (supOnPrincipal_mono F hF) X hX

theorem toElementMap_sSupMaps {F : Set (ApproximableMap V₀ V₁)} (hF : PointwiseBounded
    F)
    (x : V₀.Element) :
    (sSupMaps F hF).toElementMap x =
      V₁.sSup (Set.image (fun f => f.toElementMap x) F) (hF x) := by
  apply le_antisymm
  · intro Y hY
    rw [toElementMap_mem_iff_principal (sSupMaps F hF) x] at hY
    obtain ⟨X, hxX, hY'⟩ := hY
    have hX : V₀.mem X := x.sub hxX
    rw [toElementMap_sSupMaps_principal hF] at hY'
    have hprinc : V₀.principal hX ≤ x := fun Z hZ => x.up_mem hxX hZ.1 hZ.2
    have hsub : supOnPrincipal F hF X hX ≤ V₁.sSup (Set.image (fun f => f.toElementMap
     x) F) (hF x) :=
      V₁.sSup_le _ (mapsBounded_principal hF hX) fun s hs => by
        obtain ⟨f, hf, rfl⟩ := hs
        exact (toElementMap_mono f hprinc).trans (V₁.le_sSup _ (hF x) ⟨f, hf, rfl⟩)
    exact hsub Y hY'
```

```
  · refine V₁.sSup_le _ (hF x) fun s hs => ?_
    obtain ⟨f, hf, rfl⟩ := hs
    exact le_iff_toElementMap_le.mp (le_sSupMaps hF hf) x

theorem toElementMap_sSupMaps' {F : Set (ApproximableMap V₀ V₁)} (hF : MapsBounded F) (
    x : V₀.Element) :
    (sSupMaps F (mapsBounded_iff_pointwiseBounded.mp hF)).toElementMap x =
      V₁.sSup (Set.image (fun f => f.toElementMap x) F)
        (mapsBounded_iff_pointwiseBounded.mp hF x) :=
  toElementMap_sSupMaps (mapsBounded_iff_pointwiseBounded.mp hF) x

theorem toFilter_le_iff {f g : ApproximableMap V₀ V₁} : toFilter f ≤ toFilter g ↔ f ≤
    g :=
  (funSpaceEquiv V₀ V₁).symm.map_rel_iff'

theorem mapsBounded_to_filters {F : Set (ApproximableMap V₀ V₁)} (h : MapsBounded F) :
    (funSpace V₀ V₁).Bounded (Set.image toFilter F) := by
  obtain ⟨h, hh⟩ := h
  refine ⟨toFilter h, fun φ hφ => ?_⟩
  obtain ⟨f, hf, rfl⟩ := hφ
  exact (toFilter_le_iff).mpr (hh f hf)

end DomainTheory.NeighbourhoodSystems
```

4.4.24 Theorem 3.13 — pointwise order of maps

THEOREM 3.13. For approximable functions $f, g : D_0 \to D_1$ we have

(i) $f \sqsubseteq g$ iff $f(x) \sqsubseteq g(x)$ for all $x \in |D_0|$.

For subsets $F \subseteq |D_0 \to D_1|$ we have

(ii) F is bounded in $|D_0 \to D_1|$ iff $\{f(x) \mid f \in F\}$ is bounded in $|D_1|$ for each $x \in |D_0|$;

and in that case for all $x \in |D_0|$:

(iii) $(\bigsqcup F)(x) = \bigsqcup\{f(x) \mid f \in F\}$.

Proof. The implication in (i) from left to right follows because evaluation is monotone in the function as well as the argument. The converse implication is a consequence of 2.2(ii).

For the proof of (ii) and (iii) we see that by (i) if F is bounded, so is every set $\{f(x) \mid f \in F\}$. For the converse direction, it is clear that (iii) defines *some* pointwise mapping; we have only to prove that it is *approximable*. The calculation that $\sqcup F$ preserves directed unions (see 2.9 and 2.11) is probably the simplest way to reach the conclusion. □

The pointwise-supremum library is with the pointwise supremum of maps peer.

Theorem (`theorem_3_13`). Approximation of approximable maps is pointwise; boundedness of a family is equivalent to pointwise boundedness of values; and the pointwise supremum acts by taking values pointwise.

Proof. Assemble the pointwise-order law, the boundedness equivalence, and the pointwise-supremum identity. ■

Theorem_3_13.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.sSupMaps

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Theorem 3.13: approximation is pointwise, boundedness of maps is pointwise
    boundedness of values, and the supremum of maps acts pointwise. -/
theorem theorem_3_13 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (f g : ApproximableMap V₀ V₁),
      f ≤ g ↔ ∀ x, f.toElementMap x ≤ g.toElementMap x) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (F : Set (ApproximableMap V₀ V₁)),
      MapsBounded F ↔ PointwiseBounded F) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (F : Set (ApproximableMap V₀ V₁)) (hF : PointwiseBounded F) (x : V₀.Element),
      (sSupMaps F hF).toElementMap x =
        V₁.sSup (Set.image (fun f => f.toElementMap x) F) (hF x)) :=
  ⟨@le_iff_toElementMap_le, @mapsBounded_iff_pointwiseBounded, @toElementMap_sSupMaps⟩

end DomainTheory.NeighbourhoodSystems
```

4.4.25 Diagonal map

The diagonal approximable map into a product of a system with itself.

Definition (`diag`). The diagonal map of a neighbourhood system is the pairing of the identity with itself.

Proof. By definition. ■

Theorem (`diag_rel`). The diagonal relates a neighbourhood to a product neighbourhood precisely when both projections of that product neighbourhood relate under the identity.

Proof. Unpack the pairing relation. ■

Theorem (`toElementMap_diag`). Elementwise, the diagonal sends each ideal element to its self-pair.

Proof. Apply the pairing and identity evaluations. ■

Theorem (`proj0_comp_diag`). The first projection after the diagonal recovers the identity.

Proof. Invoke the corresponding law for pairing with identities. ■

`diag.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.ProjectionMappings

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
```

```
variable {α : Type*} (V : NeighborhoodSystem α)

def diag : ApproximableMap V (prod V V) := paired (idMap V) (idMap V)

@[simp] theorem diag_rel {Z : Set α} {P : Set (α ? α)} :
    (diag V).rel Z P ↔
      (prod V V).mem P ∧ (idMap V).rel Z (Sum.inl ^-1' P) ∧ (idMap V).rel Z (Sum.inr
    ^-1' P) :=
  Iff.rfl

@[simp] theorem toElementMap_diag (x : V.Element) :
    (diag V).toElementMap x = pair x x := by
  rw [diag, toElementMap_paired, toElementMap_idMap]

theorem proj₀_comp_diag : (proj₀ V V).comp (diag V) = idMap V :=
  proj₀_comp_paired (idMap V) (idMap V)

end DomainTheory.NeighbourhoodSystems
```

4.4.26 Exercise 3.14 — tagged product of tokens

For the most part we can assume that there is at most a countable number of tokens; thus, without loss of generality the underlying sets Δ_i of given systems $\mathcal{D}_i$ could be assumed to be subsets of Σ^* where $\Sigma = \{0, 1\}$. (Any denumerable set would do.) Show that the product $\mathcal{D}_0 \times \mathcal{D}_1$ could be defined as the system over the set $0\Delta_0 \cup 1\Delta_1$ where

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{0X \cup 1Y \mid X \in \mathcal{D}_0 \text{ and } Y \in \mathcal{D}_1\}.$$

In other words, the assumption of the disjointness of Δ_0 and Δ_1 is unnecessary. Give, therefore, the revised definition of $\langle x, y\rangle$ for elements, and prove that for a single system $\mathcal{D}$, there exists an approximable mapping

$$\mathrm{diag} : \mathcal{D} \to \mathcal{D} \times \mathcal{D}$$

where $\mathrm{diag}(x) = \langle x, x\rangle$ for all $x \in |\mathcal{D}|$. Also extend the definition to a product of n-factors

$$\mathcal{D}_0 \times \mathcal{D}_1 \times \cdots \times \mathcal{D}_{n-1}$$

which will be a system over the set

$$\bigcup_{i<n} 1^i 0 \Delta_i.$$

Note that for a 2-termed product we simplify $10\Delta_1$ to $1\Delta_1$.

The diagonal map is with the diagonal map peer. Tagged products are already realized by the product of systems construction via disjoint union of token types.

Theorem (`example_314`). The first projection recovers the identity after the diagonal.

Proof. Invoke the first-projection law for the diagonal. ■

Exercise_3_14.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.diag

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 3.14: the first projection recovers the identity after the
    diagonal. -/
theorem example_314 :
    ∀ {α : Type*} (V : NeighborhoodSystem α),
      (proj₀ V V).comp (diag V) = idMap V :=
  @proj₀_comp_diag
```

4.4.27 Product isomorphisms

Usual product isomorphisms, including the empty product as the unit system.

Definition (`prodCommD`). Ideal elements of a product are order-isomorphic to those of the swapped product.

Proof. Transport the cartesian swap of ordered sets through the product-element isomorphism. ■

Definition (`prodCongrOrderIso`). The product of two order isomorphisms is an order isomorphism of products.

Proof. Componentwise application; order equivalence is componentwise. ■

Definition (`prodAssocD`). Ideal elements of a left-associated iterated product are order-isomorphic to those of the right-associated iteration.

Proof. Transport the cartesian associator through the product-element isomorphisms and product congruence. ■

Definition (`prodUniqueOrderIso`). A product with a unique right factor is order-isomorphic to the left factor.

Proof. Project and reinstate the unique default; order comparison ignores the unique coordinate. ■

Definition (`uniqueProdOrderIso`). A product with a unique left factor is order-isomorphic to the right factor.

Proof. Symmetric to the previous. ■

Definition (`unitSys`). The unit neighbourhood system on a singleton has only the universal neighbourhood.

Proof. The filter axioms hold for the unique neighbourhood. ■

Theorem (`unitSys_element_eq`). Every ideal element of the unit system equals the bottom element.

Proof. Compare membership: each neighbourhood of an element contains the master, hence equals the unique neighbourhood. ■

Instance (`Unique unitSys.Element`). The unit system has a unique ideal element.

Proof. The bottom is the unique element by the previous identity. ■

Definition (`prodCongrD`). Ideal elements of products are order-isomorphic when the factors are, via the product of the factor isomorphisms.

Proof. Transport product congruence through the product-element isomorphisms. ■

Definition (`prodUnitD`). Ideal elements of a product with the unit system on the right are order-isomorphic to those of the left factor.

Proof. Apply the unique-right-factor isomorphism after the product-element isomorphism. ■

Definition (`unitProdD`). Ideal elements of a product with the unit system on the left are order-isomorphic to those of the right factor.

Proof. Apply the unique-left-factor isomorphism after the product-element isomorphism. ■

Theorem (`Isomorphic.prod`). Domain isomorphism is congruent under products.

Proof. Unpack witnessing order isomorphisms and apply product congruence. ■

Theorem (`prod_assoc_isomorphic`). Iterated products are associative up to domain isomorphism.

Proof. Invoke the associated element order isomorphism. ■

Theorem (`prod_comm_isomorphic`). Products are commutative up to domain isomorphism.

Proof. Invoke the swapped-product element order isomorphism. ■

Theorem (`prod_unit_isomorphic`). A product with the unit system on the right is isomorphic to the left factor.

Proof. Invoke the corresponding element order isomorphism. ■

Theorem (`unit_prod_isomorphic`). A product with the unit system on the left is isomorphic to the right factor.

Proof. Invoke the corresponding element order isomorphism. ■

prodIso.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodEquiv
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

variable {α β ? α' β' : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}
variable {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}

def prodCommD (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    (prod V₀ V₁).Element ≃o (prod V₁ V₀).Element :=
  (prodEquiv V₀ V₁).trans (OrderIso.prodComm.trans (prodEquiv V₁ V₀).symm)

def prodCongrOrderIso {A B C D : Type*} [Preorder A] [Preorder B] [Preorder C] [
    Preorder D]
    (e₀ : A ≃o B) (e₁ : C ≃o D) : A × C ≃o B × D where
  toFun p := (e₀ p.1, e₁ p.2)
  invFun q := (e₀.symm q.1, e₁.symm q.2)
  left_inv p := by simp
  right_inv q := by simp
  map_rel_iff' := by
    rintro ⟨a, c⟩ ⟨a', c'⟩
    show (e₀ a, e₁ c) ≤ (e₀ a', e₁ c') ↔ (a, c) ≤ (a', c')
```

```
    rw [Prod.mk_le_mk, Prod.mk_le_mk, e₀.le_iff_le, e₁.le_iff_le]

def prodAssocD (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
    NeighborhoodSystem ?) :
    (prod V₀ (prod V₁ V₂)).Element ≃o (prod (prod V₀ V₁) V₂).Element :=
  (prodEquiv V₀ (prod V₁ V₂)).trans <|
    (prodCongrOrderIso (OrderIso.refl V₀.Element) (prodEquiv V₁ V₂)).trans <|
      (OrderIso.prodAssoc V₀.Element V₁.Element V₂.Element).symm.trans <|
        (prodCongrOrderIso (prodEquiv V₀ V₁).symm (OrderIso.refl V₂.Element)).trans
          (prodEquiv (prod V₀ V₁) V₂).symm

def prodUniqueOrderIso (A C : Type*) [Preorder A] [Preorder C] [Unique C] : A × C ≃o A
    where
  toFun p := p.1
  invFun a := (a, default)
  left_inv p := by
    have : (default : C) = p.2 := Subsingleton.elim _ _
    simp [this]
  right_inv _ := rfl
  map_rel_iff' := by
    rintro ⟨a, c⟩ ⟨a', c'⟩
    simp only [Prod.mk_le_mk]
    exact ⟨fun h => ⟨h, le_of_eq (Subsingleton.elim c c')⟩, And.left⟩

def uniqueProdOrderIso (A C : Type*) [Preorder A] [Preorder C] [Unique C] : C × A ≃o A
    where
  toFun p := p.2
  invFun a := (default, a)
  left_inv p := by
    have : (default : C) = p.1 := Subsingleton.elim _ _
    simp [this]
  right_inv _ := rfl
  map_rel_iff' := by
    rintro ⟨c, a⟩ ⟨c', a'⟩
    simp only [Prod.mk_le_mk]
    exact ⟨fun h => ⟨le_of_eq (Subsingleton.elim c c'), h⟩, And.right⟩

def unitSys : NeighborhoodSystem Unit where
  mem X := X = Set.univ
  master := Set.univ
  master_nonempty := ⟨(), Set.mem_univ _⟩
  master_mem := rfl
  inter_mem := by rintro X Y Z rfl rfl _ _; simp
  sub_master := by rintro X rfl; exact subset_rfl

theorem unitSys_element_eq (x : unitSys.Element) : x = unitSys.bot := by
  apply Element.ext
  intro Y
  constructor
  · intro hY; rw [mem_bot]; exact x.sub hY
  · intro hY; rw [mem_bot] at hY; subst hY; exact x.master_mem

instance : Unique unitSys.Element where
  default := unitSys.bot
  uniq := unitSys_element_eq
```

```
def prodCongrD (e₀ : V₀.Element ≃o V₀'.Element) (e₁ : V₁.Element ≃o V₁'.Element) :
    (prod V₀ V₁).Element ≃o (prod V₀' V₁').Element :=
  (prodEquiv V₀ V₁).trans ((prodCongrOrderIso e₀ e₁).trans (prodEquiv V₀' V₁').symm)

def prodUnitD (V₀ : NeighborhoodSystem α) :
    (prod V₀ unitSys).Element ≃o V₀.Element :=
  (prodEquiv V₀ unitSys).trans (prodUniqueOrderIso _ _)

def unitProdD (V₀ : NeighborhoodSystem α) :
    (prod unitSys V₀).Element ≃o V₀.Element :=
  (prodEquiv unitSys V₀).trans (uniqueProdOrderIso _ _)

theorem Isomorphic.prod (h₀ : V₀ ≅? V₀') (h₁ : V₁ ≅? V₁') : prod V₀ V₁ ≅? prod V₀' V₁'
    :=
  h₀.elim fun e₀ => h₁.elim fun e₁ => ⟨prodCongrD e₀ e₁⟩

theorem prod_assoc_isomorphic : prod V₀ (prod V₁ V₂) ≅? prod (prod V₀ V₁) V₂ :=
  ⟨prodAssocD V₀ V₁ V₂⟩

theorem prod_comm_isomorphic : prod V₀ V₁ ≅? prod V₁ V₀ := ⟨prodCommD V₀ V₁⟩

theorem prod_unit_isomorphic : prod V₀ unitSys ≅? V₀ := ⟨prodUnitD V₀⟩

theorem unit_prod_isomorphic : prod unitSys V₀ ≅? V₀ := ⟨unitProdD V₀⟩

end DomainTheory.NeighbourhoodSystems
```

4.4.28 Exercise 3.15 — product isomorphisms

Establish the usual isomorphisms:

(i) $\mathcal{D}_0 \times \mathcal{D}_1 \cong \mathcal{D}_1 \times \mathcal{D}_0$;

(ii) $\mathcal{D}_0 \times (\mathcal{D}_1 \times \mathcal{D}_2) \cong (\mathcal{D}_0 \times \mathcal{D}_1) \times \mathcal{D}_2 \cong \mathcal{D}_0 \times \mathcal{D}_1 \times \mathcal{D}_2$.

How does the product of no factors fit in? Prove also:

(iii) $\mathcal{D}_0 \cong \mathcal{D}_0'$ and $\mathcal{D}_1 \cong \mathcal{D}_1'$ imply $\mathcal{D}_0 \times \mathcal{D}_1 \cong \mathcal{D}_0' \times \mathcal{D}_1'$.

The product-isomorphism library is with the product isomorphisms peer. The product of no factors is the unit system there.

Theorem (`example_315`). Products are commutative, congruent under domain isomorphism of factors, and associative, up to domain isomorphism.

Proof. Assemble the commutation, congruence, and association laws. ■

`Exercise_3_15.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodIso

@[expose] public section
```

```
open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem

/-- Scott's Exercise 3.15: product is commutative, congruent, and associative up to
    isomorphism. -/
theorem example_315 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β},
      prod V₀ V₁ ≅? prod V₁ V₀) ∧
    (∀ {α β α' β' : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'},
      V₀ ≅? V₀' → V₁ ≅? V₁' → prod V₀ V₁ ≅? prod V₀' V₁') ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?},
      prod V₀ (prod V₁ V₂) ≅? prod (prod V₀ V₁) V₂) :=
  ⟨@prod_comm_isomorphic, @Isomorphic.prod, @prod_assoc_isomorphic⟩
```

4.4.29 Infinite sequence system

The infinite sequence neighbourhood system, its sequence elements, and the isomorphism with a product.

Definition (`fiber`). The fibre of a set of indexed tokens at an index is the set of tokens appearing with that index.

Proof. By definition. ■

Theorem (`mem_fiber`). Membership in a fibre is membership of the indexed pair in the underlying set.

Proof. By definition. ■

Theorem (`eq_of_fiber_eq`). Two sets of indexed tokens are equal when all fibres agree.

Proof. Compare membership of indexed pairs via the fibres. ■

Theorem (`fiber_inter`). Fibres commute with intersection.

Proof. By definition. ■

Theorem (`fiber_mono`). Fibres preserve inclusion.

Proof. Apply the ambient inclusion to an indexed pair. ■

Definition (`single`). The single-index neighbourhood at an index with a domain neighbourhood is the set of indexed tokens that lie in that neighbourhood at the chosen index and in the master elsewhere.

Proof. By definition. ■

Theorem (`fiber_single_self`). The fibre of a single-index neighbourhood at its own index recovers the domain neighbourhood.

Proof. Unpack the definition at that index. ■

Theorem (`fiber_single_ne`). A fibre of a single-index neighbourhood at a different index is the master.

Proof. Unpack the definition off that index. ■

Definition (`iterSys`). The infinite sequence system consists of those sets of indexed tokens whose fibres are neighbourhoods and that are eventually the master.

Proof. The master is the constant master fibre; intersection preserves fibrewise membership and eventual masters; subsets of neighbourhoods remain below the master fibrewise. ■

Theorem (`iterSys_master`). The master of the sequence system is the set of pairs whose token lies in the base master.

Proof. By definition. ■

Theorem (`fiber_iterSys_master`). Every fibre of the sequence master is the base master.

Proof. By definition. ■

Theorem (`mem_iterSys`). Membership in the sequence system is fibrewise neighbourhood membership together with an eventual master.

Proof. By definition. ■

Theorem (`interUpTo_subset_master`). Finite initial intersections of a sequence of sets in the sequence system lie below the master.

Proof. By induction on the number of factors. ■

Theorem (`reconstruct_subset`). The finite initial intersection of single-index neighbourhoods built from the fibres of a sequence neighbourhood is contained in that neighbourhood.

Proof. On indices below the cutoff, unpack the corresponding single; beyond the cutoff, use the eventual master. ■

Theorem (`single_inter`). Intersection of single-index neighbourhoods at a common index is the single at the intersection of the domains.

Proof. Compare fibres. ■

Theorem (`single_master`). The single-index neighbourhood at the master is the sequence master.

Proof. Compare fibres. ■

Theorem (`single_mem`). A single-index neighbourhood at a neighbourhood is itself a sequence neighbourhood.

Proof. Fibres are the given neighbourhood or the master; after that index they are the master. ■

Theorem (`single_mono`). Singles are monotone in the domain neighbourhood.

Proof. Compare the defining cases. ■

Definition (`component`). The component of a sequence element at an index is the ideal element whose neighbourhoods are those for which the corresponding single belongs to the sequence element.

Proof. Filter axioms follow from those of the sequence element together with the single intersection and nesting laws. ■

Theorem (`mem_component`). Membership in a component is base neighbourhood membership together with membership of the corresponding single in the sequence element.

Proof. By definition. ■

Definition (`ofSeq`). A sequence of ideal elements determines a sequence-system element consisting of those sequence neighbourhoods whose fibres belong to the corresponding terms.

Proof. Filter axioms follow fibrewise from the terms, using fibre intersection and monotonicity. ■

Theorem (`mem_ofSeq`). Membership in the sequence of a family is sequence-neighbourhood membership together with fibrewise membership in the terms.

Proof. By definition. ■

Theorem (`component_ofSeq`). Components of a sequence recover the terms.

Proof. Compare membership via the fibre of the single at the chosen index. ■

Theorem (`ofSeq_mono`). Pointwise approximation of sequences yields approximation of the corresponding sequence elements.

Proof. Apply each term to the corresponding fibre. ■

Theorem (`subset_single`). A sequence neighbourhood is contained in the single built from each of its fibres.

Proof. On the chosen index unpack the fibre; off it use the master bound. ■

Theorem (`ofSeq_component`). Reassembling the components of a sequence element recovers that element.

Proof. One inclusion uses finite reconstruction from singles; the other embeds via singles containing the neighbourhood. ■

Theorem (`le_of_component_le`). Pointwise approximation of components yields approximation of sequence elements.

Proof. Transport through the sequence reconstruction. ■

Definition (`iterSeqEquiv`). Ideal elements of the sequence system are order-isomorphic to arbitrary sequences of base ideal elements.

Proof. The round trips are the component and sequence reconstructions; order equivalence is the pointwise comparison. ■

Definition (`natShiftEquiv`). Sequences are order-isomorphic to a head paired with the tail sequence.

Proof. Split and reassemble by cases on zero; order equivalence compares the head and the shifted terms. ■

Definition (`iterProdIso`). Ideal elements of the sequence system are order-isomorphic to those of the product of the base with the sequence system.

Proof. Compose the sequence isomorphism, the head–tail shift, product congruence, and the product-element isomorphism. ■

Definition (`projN`). Projection at an index relates a sequence neighbourhood to a base neighbourhood precisely when the fibre at that index is contained in the latter.

Proof. Domain and codomain membership unpack the conjuncts; the masters relate by the master fibre; intersection on the right and monotonicity use the base laws with fibre monotonicity. ■

Theorem (`subset_of_fiber_subset`). Fibrewise inclusion implies inclusion of indexed-token sets.

Proof. Apply the fibre inclusion at the index of a pair. ■

Theorem (`projN_rel`). Membership in a projection relation is sequence membership, base membership, and fibre containment.

Proof. By definition. ■

Theorem (`toElementMap_projN`). Projection at an index acts elementwise as the corresponding component.

Proof. Compare membership via singles and fibre containment. ■

Theorem (`iter_isomorphic`). The sequence system is isomorphic to its product with the base.

Proof. Invoke the corresponding element order isomorphism. ■

iterSys.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodIso
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_5a

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
variable {α : Type*}
```

```
def fiber (W : Set (ℕ × α)) (i : ℕ) : Set α := {a | (i, a) ∈ W}

@[simp] theorem mem_fiber {W : Set (ℕ × α)} {i : ℕ} {a : α} : a ∈ fiber W i ↔ (i, a)
    ∈ W := Iff.rfl

theorem eq_of_fiber_eq {W W' : Set (ℕ × α)} (h : ∀ i, fiber W i = fiber W' i) : W = W'
     := by
  ext ⟨i, a⟩
  exact Set.ext_iff.mp (h i) a

theorem fiber_inter (W W' : Set (ℕ × α)) (i : ℕ) :
    fiber (W ∩ W') i = fiber W i ∩ fiber W' i := rfl

theorem fiber_mono {W W' : Set (ℕ × α)} (h : W ⊆ W') (i : ℕ) : fiber W i ⊆ fiber W' i
     :=
  fun _ ha => h ha
end

section
open NeighborhoodSystem
variable {α : Type*}
variable (V : NeighborhoodSystem α)

def single (n : ℕ) (X : Set α) : Set (ℕ × α) :=
  {p | if p.1 = n then p.2 ∈ X else p.2 ∈ V.master}
end

section
open NeighborhoodSystem
variable {α : Type*}
variable (V : NeighborhoodSystem α)
variable {V}

@[simp] theorem fiber_single_self (n : ℕ) (X : Set α) : fiber (single V n X) n = X :=
    by
  ext a; simp [fiber, single]

theorem fiber_single_ne {n i : ℕ} (h : i = n) (X : Set α) : fiber (single V n X) i = V
    .master := by
  ext a; simp [fiber, single, h]

end

section
open NeighborhoodSystem
variable {α : Type*}
variable (V : NeighborhoodSystem α)
variable {V}

def iterSys (V : NeighborhoodSystem α) : NeighborhoodSystem (ℕ × α) where
  mem W := (∀ i, V.mem (fiber W i)) ∧ ∃ N, ∀ i, N ≤ i → fiber W i = V.master
  master := {p | p.2 ∈ V.master}
  master_nonempty := by
    obtain ⟨a, ha⟩ := V.master_nonempty
    exact ⟨(0, a), ha⟩
```

```
  master_mem := ⟨fun _ => V.master_mem, 0, fun _ _ => rfl⟩
  inter_mem := by
    rintro W W' Z ⟨hWf, NW, hNW⟩ ⟨hW'f, NW', hNW'⟩ ⟨hZf, _⟩ hsub
    refine ⟨fun i => ?_, max NW NW', fun i hi => ?_⟩
    · rw [fiber_inter]
      exact V.inter_mem (hWf i) (hW'f i) (hZf i) (fiber_mono hsub i)
    · rw [fiber_inter, hNW i (le_trans (le_max_left _ _) hi),
        hNW' i (le_trans (le_max_right _ _) hi), Set.inter_self]
  sub_master := by
    rintro W ⟨hWf, _⟩ ⟨i, a⟩ ha
    exact V.sub_master (hWf i) ha

@[simp] theorem iterSys_master : (iterSys V).master = {p : ℕ × α | p.2 ∈ V.master} :=
    rfl

theorem fiber_iterSys_master (i : ℕ) : fiber ((iterSys V).master) i = V.master := rfl

@[simp] theorem mem_iterSys {W : Set (ℕ × α)} :
    (iterSys V).mem W ↔ (∀ i, V.mem (fiber W i)) ∧ ∃ N, ∀ i, N ≤ i → fiber W i = V.
    master := Iff.rfl

theorem interUpTo_subset_master (F : ℕ → Set (ℕ × α)) (N : ℕ) :
    (iterSys V).interUpTo F N ⊆ (iterSys V).master := by
  induction N with
  | zero => exact subset_rfl
  | succ n ih => rw [NeighborhoodSystem.interUpTo.succ]; exact Set.inter_subset_left.
    trans ih

theorem reconstruct_subset {W : Set (ℕ × α)} (_hW : (iterSys V).mem W) {N : ℕ}
    (hN : ∀ i, N ≤ i → fiber W i = V.master) :
    (iterSys V).interUpTo (fun i => single V i (fiber W i)) N ⊆ W := by
  rintro ⟨j, a⟩ ha
  by_cases h : j < N
  · have hsub := interUpTo.subset (iterSys V) (fun i => single V i (fiber W i)) h
    have hmem : (j, a) ∈ single V j (fiber W j) := hsub ha
    simpa [single] using hmem
  · have haM : a ∈ V.master := interUpTo_subset_master (fun i => single V i (fiber W i)
    ) N ha
    have : a ∈ fiber W j := by rw [hN j (not_lt.mp h)]; exact haM
    exact this

theorem single_inter {n : ℕ} (X X' : Set α) :
    single V n X ∩ single V n X' = single V n (X ∩ X') := by
  apply eq_of_fiber_eq
  intro i
  by_cases h : i = n
  · subst h; rw [fiber_inter, fiber_single_self, fiber_single_self, fiber_single_self]
  · rw [fiber_inter, fiber_single_ne h, fiber_single_ne h, fiber_single_ne h, Set.
    inter_self]

theorem single_master (n : ℕ) : single V n V.master = (iterSys V).master := by
  apply eq_of_fiber_eq
  intro i
  rw [fiber_iterSys_master]
  by_cases h : i = n
  · subst h; rw [fiber_single_self]
```

```
  · rw [fiber_single_ne h]

theorem single_mem {n : ℕ} {X : Set α} (hX : V.mem X) : (iterSys V).mem (single V n X)
    := by
  refine ⟨fun i => ?_, n + 1, fun i hi => ?_⟩
  · by_cases h : i = n
    · subst h; rw [fiber_single_self]; exact hX
    · rw [fiber_single_ne h]; exact V.master_mem
  · exact fiber_single_ne (by omega : i = n) X

theorem single_mono {n : ℕ} {X X' : Set α} (h : X ⊆ X') : single V n X ⊆ single V n X'
    := by
  intro p hp
  simp only [single, Set.mem_ofPred_eq] at hp ⊢
  by_cases hc : p.1 = n
  · rw [ite_eq_left hc] at hp ⊢; exact h hp
  · rw [ite_eq_right hc] at hp ⊢; exact hp

def component (z : (iterSys V).Element) (n : ℕ) : V.Element where
  mem X := V.mem X ∧ z.mem (single V n X)
  sub h := h.1
  master_mem := ⟨V.master_mem, by rw [single_master]; exact z.master_mem⟩
  inter_mem := by
    rintro X X' ⟨_, hzX⟩ ⟨_, hzX'⟩
    have hz : z.mem (single V n (X ∩ X')) := by rw [← single_inter]; exact z.inter_mem
     hzX hzX'
    have hmem : V.mem (X ∩ X') := by
      have := (z.sub hz).1 n; rwa [fiber_single_self] at this
    exact ⟨hmem, hz⟩
  up_mem := by
    rintro X X' ⟨_, hzX⟩ hX' hXX'
    exact ⟨hX', z.up_mem hzX (single_mem hX') (single_mono hXX')⟩

@[simp] theorem mem_component {z : (iterSys V).Element} {n : ℕ} {X : Set α} :
    (component z n).mem X ↔ V.mem X ∧ z.mem (single V n X) := Iff.rfl

def ofSeq (seq : ℕ → V.Element) : (iterSys V).Element where
  mem W := (iterSys V).mem W ∧ ∀ i, (seq i).mem (fiber W i)
  sub h := h.1
  master_mem := ⟨(iterSys V).master_mem, fun i => by
    rw [fiber_iterSys_master]; exact (seq i).master_mem⟩
  inter_mem := by
    rintro W W' ⟨hW, hWf⟩ ⟨hW', hW'f⟩
    refine ⟨⟨fun i => ?_, ?_⟩, fun i => ?_⟩
    · rw [fiber_inter]; exact (seq i).sub ((seq i).inter_mem (hWf i) (hW'f i))
    · obtain ⟨NW, hNW⟩ := hW.2
      obtain ⟨NW', hNW'⟩ := hW'.2
      refine ⟨max NW NW', fun i hi => ?_⟩
      rw [fiber_inter, hNW i (le_trans (le_max_left _ _) hi),
        hNW' i (le_trans (le_max_right _ _) hi), Set.inter_self]
    · rw [fiber_inter]; exact (seq i).inter_mem (hWf i) (hW'f i)
  up_mem := by
    rintro W W' ⟨_, hWf⟩ hW' hWW'
    exact ⟨hW', fun i => (seq i).up_mem (hWf i) (hW'.1 i) (fiber_mono hWW' i)⟩

@[simp] theorem mem_ofSeq {seq : ℕ → V.Element} {W : Set (ℕ × α)} :
```

```
    (ofSeq seq).mem W ↔ (iterSys V).mem W ∧ ∀ i, (seq i).mem (fiber W i) := Iff.rfl

@[simp] theorem component_ofSeq (seq : ℕ → V.Element) (n : ℕ) :
    component (ofSeq seq) n = seq n := by
  apply Element.ext
  intro X
  constructor
  · rintro ⟨_, _, hfib⟩
    have := hfib n; rwa [fiber_single_self] at this
  · intro hX
    refine ⟨(seq n).sub hX, single_mem ((seq n).sub hX), fun i => ?_⟩
    by_cases h : i = n
    · subst h; rw [fiber_single_self]; exact hX
    · rw [fiber_single_ne h]; exact (seq i).master_mem

theorem ofSeq_mono {seq seq' : ℕ → V.Element} (h : ∀ n, seq n ≤ seq' n) :
    ofSeq seq ≤ ofSeq seq' := by
  rintro W ⟨hW, hf⟩
  exact ⟨hW, fun i => h i _ (hf i)⟩

theorem subset_single {W : Set (ℕ × α)} (hW : (iterSys V).mem W) (i : ℕ) :
    W ⊆ single V i (fiber W i) := by
  rintro ⟨j, a⟩ ha
  simp only [single, Set.mem_ofPred_eq]
  by_cases h : j = i
  · rw [ite_eq_left h]; subst h; exact ha
  · rw [ite_eq_right h]; exact V.sub_master (hW.1 j) ha

@[simp] theorem ofSeq_component (z : (iterSys V).Element) :
    ofSeq (fun n => component z n) = z := by
  apply Element.ext
  intro W
  constructor
  · rintro ⟨hW, hfib⟩
    obtain ⟨N, hN⟩ := hW.2
    exact z.up_mem (z.mem_interUpTo _ (n := N) (fun i _ => (hfib i).2)) hW
      (reconstruct_subset hW hN)
  · intro hzW
    refine ⟨z.sub hzW, fun i => ?_⟩
    exact ⟨(z.sub hzW).1 i, z.up_mem hzW (single_mem ((z.sub hzW).1 i)) (subset_single
    (z.sub hzW) i)⟩

theorem le_of_component_le {z z' : (iterSys V).Element}
    (h : ∀ n, component z n ≤ component z' n) : z ≤ z' := by
  have hmono := ofSeq_mono h
  rwa [ofSeq_component, ofSeq_component] at hmono

def iterSeqEquiv (V : NeighborhoodSystem α) : (iterSys V).Element ≃o (∀ _ : ℕ, V.
    Element) where
  toFun z := fun n => component z n
  invFun seq := ofSeq seq
  left_inv z := ofSeq_component z
  right_inv seq := by funext n; exact component_ofSeq seq n
  map_rel_iff' := by
    intro z z'
    constructor
```

```
    · intro h
      exact le_of_component_le (fun n => h n)
    · intro h n X hX
      exact ⟨hX.1, h (single V n X) hX.2⟩

def natShiftEquiv (E : Type*) [Preorder E] : (ℕ → E) ≃o E × (ℕ → E) where
  toFun f := (f 0, fun n => f (n + 1))
  invFun p := fun n => Nat.casesOn n p.1 (fun m => p.2 m)
  left_inv f := by funext n; cases n <;> rfl
  right_inv p := rfl
  map_rel_iff' := by
    intro f g
    constructor
    · rintro ⟨h0, hs⟩
      intro n
      cases n with
      | zero => exact h0
      | succ m => exact hs m
    · intro h
      exact ⟨h 0, fun m => h (m + 1)⟩

def iterProdIso (V : NeighborhoodSystem α) :
    (iterSys V).Element ≃o (prod V (iterSys V)).Element :=
  (iterSeqEquiv V).trans <|
    (natShiftEquiv V.Element).trans <|
      (prodCongrOrderIso (OrderIso.refl V.Element) (iterSeqEquiv V).symm).trans
        (prodEquiv V (iterSys V)).symm
end

section
open NeighborhoodSystem
variable {α : Type*}
variable (V : NeighborhoodSystem α)
variable {V}
open ApproximableMap

def projN (V : NeighborhoodSystem α) (n : ℕ) : ApproximableMap (iterSys V) V where
  rel W X := (iterSys V).mem W ∧ V.mem X ∧ fiber W n ⊆ X
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(iterSys V).master_mem, V.master_mem, by rw [fiber_iterSys_master]⟩
  inter_right := by
    rintro W X X' ⟨hW, hX, hsub⟩ ⟨_, hX', hsub'⟩
    exact ⟨hW, V.inter_mem hX hX' (hW.1 n) (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'⟩
  mono := by
    rintro W W₂ X X₂ ⟨_, _, hsub⟩ hW₂W hXX₂ hW₂ hX₂
    exact ⟨hW₂, hX₂, (fiber_mono hW₂W n).trans (hsub.trans hXX₂)⟩
end

section
open NeighborhoodSystem
variable {α : Type*}

theorem subset_of_fiber_subset {W W' : Set (ℕ × α)} (h : ∀ i, fiber W i ⊆ fiber W' i)
    :
```

```
    W ⊆ W' := fun ⟨i, _⟩ ha => h i ha

variable (V : NeighborhoodSystem α)

variable {V}

open ApproximableMap

@[simp] theorem projN_rel {n : ℕ} {W : Set (ℕ × α)} {X : Set α} :
    (projN V n).rel W X ↔ (iterSys V).mem W ∧ V.mem X ∧ fiber W n ⊆ X := Iff.rfl

@[simp] theorem toElementMap_projN (z : (iterSys V).Element) (n : ℕ) :
    (projN V n).toElementMap z = component z n := by
  apply Element.ext
  intro X
  constructor
  · rintro ⟨W, hzW, hW, hX, hsub⟩
    refine ⟨hX, z.up_mem hzW (single_mem hX) ?_⟩
    refine subset_of_fiber_subset (fun i => ?_)
    by_cases h : i = n
    · subst h; rw [fiber_single_self]; exact hsub
    · rw [fiber_single_ne h]; exact V.sub_master (hW.1 i)
  · rintro ⟨hX, hz⟩
    exact ⟨single V n X, hz, single_mem hX, hX, by rw [fiber_single_self]⟩
end

section
open NeighborhoodSystem
variable {α : Type*}
variable (V : NeighborhoodSystem α)
variable {V}

theorem iter_isomorphic (V : NeighborhoodSystem α) : iterSys V ≅? prod V (iterSys V)
    :=
  ⟨iterProdIso V⟩
end
end DomainTheory.NeighbourhoodSystems
end
```

4.4.30 Exercise 3.16 — infinite sequences of elements

Let $\mathcal{D}$ be a given neighbourhood system over $\Delta \subseteq \Sigma^*$. Define

$$\Delta^\infty = \bigcup_{n=0}^{\infty} 1^n 0 \Delta,$$

so that Δ^∞ is split into infinitely many disjoint copies of Δ. Let $\mathcal{D}^\infty$ be the least family of subsets of Σ^* where

1. $\Delta^\infty \in \mathcal{D}^\infty$, and
2. whenever $X \in \mathcal{D}$ and $Y \in \mathcal{D}^\infty$, then $0X \cup 1Y \in \mathcal{D}^\infty$.

Show that $\mathcal{D}^\infty$ is a neighbourhood system over Δ^∞. Prove the isomorphism

$$\mathcal{D}^\infty \cong \mathcal{D} \times \mathcal{D}^\infty.$$

Show, moreover, that the elements of $|\mathcal{D}^\infty|$ are in a one-one correspondence with arbitrary infinite sequences $\langle x_n \rangle_{n=0}^\infty$ of elements $x_n \in |\mathcal{D}|$ by using combinations of neighbourhoods

$$0X_0 \cup 10X_1 \cup \cdots \cup 1^n 0X_n \cup \cdots,$$

where from some point on all the X_m are equal to Δ.

The infinite sequence system is with the infinite sequence system peer.

Theorem (`example_316`). The infinite sequence system is isomorphic to its product with the base system.

Proof. Invoke the sequence–product isomorphism. ■

`Exercise_3_16.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.iterSys

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem

/-- Scott's Exercise 3.16: the infinite sequence system is isomorphic to its product
    with the base. -/
theorem example_316 :
    ∀ {α : Type*} (V : NeighborhoodSystem α),
      iterSys V ≅? prod V (iterSys V) :=
  @iter_isomorphic
```

4.4.31 Embedding binary sequences

Embedding of binary sequences into the infinite token sequence system, with a left inverse.

Definition (`T`). Write T for the token neighbourhood system of Example 1.2.

Proof. By abbreviation. ■

Definition (`Token`). Write Token for the underlying token type of Example 1.2.

Proof. By abbreviation. ■

Theorem (`T_nbhd_nonempty`). Every neighbourhood in T is nonempty.

Proof. Inspect the three neighbourhoods of Example 1.2. ■

Definition (`bitNbhd`). The bit neighbourhood of a boolean is the zero neighbourhood or the one neighbourhood according as the bit is false or true.

Proof. By definition. ■

Theorem (`bitNbhd_mem`). Each bit neighbourhood belongs to T.

Proof. Invoke the corresponding membership facts of Example 1.2. ■

Definition (`encSet`). The encoding of a finite boolean string is the set of indexed tokens that, whenever the string is defined at that index, lie in the corresponding bit neighbourhood.

Proof. By definition. ■

Theorem (`mem_encSet`). Membership in an encoding is the defining bit constraint at each index.

Proof. By definition. ■

Theorem (`encSet_nil`). The encoding of the empty string is the universal set.

Proof. The defining constraints are vacuous. ■

Theorem (`comparable_of_agree`). Two finite boolean strings that never disagree at a common defined index are comparable under the prefix order.

Proof. By induction on the strings, matching heads when both are nonempty. ■

Theorem (`encSet_agree`). If a sequence neighbourhood is contained in two encodings, then those encodings agree on every commonly defined bit.

Proof. Pick a token from the fibre at that index and observe that it must lie in both bit neighbourhoods. ■

Theorem (`encSet_anti`). Extending a string shrinks its encoding.

Proof. A defined bit of the shorter string remains defined with the same value after concatenation. ■

Theorem (`fiber_encSet_none`). Beyond the length of a string, every fibre of its encoding is universal.

Proof. The defining constraint at that index is vacuous. ■

Theorem (`fiber_encSet_some`). At a defined bit, the fibre of the encoding is the corresponding bit neighbourhood.

Proof. Compare membership of tokens with the defining constraint. ■

Theorem (`encSet_mem`). Every encoding belongs to the infinite sequence system on T.

Proof. Fibres are either the master or a bit neighbourhood; after the string length they are the master. ■

Definition (`f`). The forward embedding relates a cone neighbourhood of a string to a sequence neighbourhood precisely when the encoding of that string is contained in the latter.

Proof. Domain membership recovers cone membership in $\mathcal{B}$; codomain membership is the sequence witness; the masters relate via the empty encoding; intersection on the right uses uniqueness of cones; monotonicity uses anti-monotonicity of encodings under prefixes. ■

Definition (`g`). The reverse map relates a sequence neighbourhood to a cone neighbourhood of a string precisely when the former is contained in the encoding of that string.

Proof. Domain and codomain membership unpack the witnesses; the masters relate via the empty encoding; intersection on the right uses agreement of encodings under a common lower bound together with comparability of agreeing strings; monotonicity uses anti-monotonicity of encodings. ■

Theorem (`exists_not_mem_bitNbhd`). Each bit neighbourhood omits some token.

Proof. The opposite bit token of Example 1.2 lies outside. ■

Theorem (`fg_le_id`). Composing the forward embedding after the reverse map stays below the identity on the sequence system.

Proof. Unpack the composite relation and transport containment through a common encoding. ■

Theorem (`prefix_of_encSet_subset`). Inclusion of encodings implies the reverse prefix order on the corresponding strings.

Proof. Agreeing encodings are comparable; the longer-first case is ruled out by placing a forbidden token at the first undefined index of the shorter string. ■

Theorem (`gf_eq_id`). Composing the reverse map after the forward embedding recovers the identity on $\mathcal{B}$.

Proof. Compare relations: one direction uses the prefix recovery from encodings; the other embeds via the encoding of the larger cone. ■

Theorem (`f_injective`). The forward embedding is injective on ideal elements.

Proof. Apply the left-inverse identity after transporting equality of values through the reverse map. ■

embBinSeq.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.iterSys
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.step
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ExampleB
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.TwoToken
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.EmbBinSeq

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem BitDom ApproximableMap

abbrev T : NeighborhoodSystem TwoToken.Token := TwoToken.neighborhoodSystem

abbrev Token := TwoToken.Token

theorem T_nbhd_nonempty {F : Set Token} (hF : T.mem F) : F.Nonempty := by
  rcases (TwoToken.mem_iff F).mp hF with rfl | rfl | rfl
  · exact ⟨0, by simp [TwoToken.master]⟩
  · exact ⟨0, by simp [TwoToken.zero]⟩
  · exact ⟨1, by simp [TwoToken.one]⟩

def bitNbhd : Bool → Set Token
  | false => TwoToken.zero
  | true => TwoToken.one

theorem bitNbhd_mem (b : Bool) : T.mem (bitNbhd b) := by
  cases b
  · exact TwoToken.mem_zero
  · exact TwoToken.mem_one

def encSet (σ : Str) : Set (ℕ × Token) := {p | ∀ b, σ[p.1]? = some b → p.2 ∈ bitNbhd
    b}

@[simp] theorem mem_encSet {σ : Str} {p : ℕ × Token} :
    p ∈ encSet σ ↔ ∀ b, σ[p.1]? = some b → p.2 ∈ bitNbhd b := Iff.rfl

@[simp] theorem encSet_nil : encSet [] = Set.univ := by
  ext p; simp [encSet]

theorem comparable_of_agree : ∀ {τ τ' : Str},
    (∀ (i : ℕ) (b b' : Bool), τ[i]? = some b → τ'[i]? = some b' → b = b') → τ <+: τ'
     ∨ τ' <+: τ
  | [], _, _ => Or.inl (List.nil_prefix)
  | _ :: _, [], _ => Or.inr (List.nil_prefix)
  | a :: s, a' :: s', hag => by
      have ha : a = a' := hag 0 a a' (by simp) (by simp)
      subst ha
      have hrec : ∀ (i : ℕ) (b b' : Bool), s[i]? = some b → s'[i]? = some b' → b = b'
     := by
        intro i b b' hb hb'
```

```
        exact hag (i + 1) b b' (by simpa using hb) (by simpa using hb')
      rcases comparable_of_agree hrec with h | h
      · exact Or.inl (List.cons_prefix_cons.mpr ⟨rfl, h⟩)
      · exact Or.inr (List.cons_prefix_cons.mpr ⟨rfl, h⟩)

theorem encSet_agree {τ τ' : Str} {W : Set (ℕ × Token)} (hW : (iterSys T).mem W)
    (h : W ⊆ encSet τ) (h' : W ⊆ encSet τ') {i : ℕ} {b b' : Bool}
    (hb : τ[i]? = some b) (hb' : τ'[i]? = some b') : b = b' := by
  obtain ⟨t, ht⟩ := T_nbhd_nonempty (hW.1 i)
  have h1 : t ∈ bitNbhd b := (h ht) b hb
  have h2 : t ∈ bitNbhd b' := (h' ht) b' hb'
  by_contra hne
  cases b <;> cases b' <;> simp_all [bitNbhd, TwoToken.zero, TwoToken.one]

theorem encSet_anti {σ σ' : Str} (h : σ <+: σ') : encSet σ' ⊆ encSet σ := by
  obtain ⟨t, rfl⟩ := h
  rintro ⟨i, x⟩ hp b hb
  have hi : i < σ.length := by
    obtain ⟨hlt, -⟩ := List.getElem?_eq_some_iff.mp hb; exact hlt
  exact hp b (by rw [List.getElem?_append_left hi]; exact hb)

theorem fiber_encSet_none {σ : Str} {i : ℕ} (h : σ[i]? = none) :
    fiber (encSet σ) i = Set.univ := by
  ext t
  simp only [mem_fiber, mem_encSet, Set.mem_univ, iff_true]
  intro b hb; rw [h] at hb; exact absurd hb (by simp)

theorem fiber_encSet_some {σ : Str} {i : ℕ} {b : Bool} (h : σ[i]? = some b) :
    fiber (encSet σ) i = bitNbhd b := by
  ext t
  simp only [mem_fiber, mem_encSet]
  constructor
  · intro hh; exact hh b h
  · intro ht b' hb'
    rw [h] at hb'
    obtain rfl := Option.some.inj hb'
    exact ht

theorem encSet_mem (σ : Str) : (iterSys T).mem (encSet σ) := by
  refine ⟨fun i => ?_, σ.length, fun i hi => ?_⟩
  · cases h : σ[i]? with
    | none => rw [fiber_encSet_none h]; exact T.master_mem
    | some b => rw [fiber_encSet_some h]; exact bitNbhd_mem b
  · rw [fiber_encSet_none (List.getElem?_eq_none_iff.mpr hi)]; rfl

def f : ApproximableMap B (iterSys T) where
  rel X W := ∃ σ, X = cone σ ∧ (iterSys T).mem W ∧ encSet σ ⊆ W
  rel_dom := fun ⟨σ, hX, _⟩ => hX ▶ memB_cone σ
  rel_cod := fun ⟨_, _, hW, _⟩ => hW
  master_rel := ⟨[], cone.nil.symm, (iterSys T).master_mem, by rw [encSet_nil]; exact
    Set.subset_univ _⟩
  inter_right := by
    rintro X W W' ⟨σ, hX, hWmem, hWsub⟩ ⟨σ', hX', hW'mem, hW'sub⟩
    obtain rfl : σ = σ' := cone.injective (hX.symm.trans hX')
    exact ⟨σ, hX, (iterSys T).inter_mem hWmem hW'mem (encSet_mem σ)
      (Set.subset_inter hWsub hW'sub), Set.subset_inter hWsub hW'sub⟩
```

```
  mono := by
    rintro X X' W W' ⟨σ, hX, _, hWsub⟩ hX'X hWW' hX' hW'mem
    obtain ⟨σ', rfl⟩ := hX'
    subst hX
    have hpre : σ <+: σ' := cone.subset_cone.mp hX'X
    exact ⟨σ', rfl, hW'mem, (encSet_anti hpre).trans (hWsub.trans hWW')⟩

def g : ApproximableMap (iterSys T) B where
  rel W X := (iterSys T).mem W ∧ ∃ τ, X = cone τ ∧ W ⊆ encSet τ
  rel_dom h := h.1
  rel_cod := fun ⟨_, τ, hX, _⟩ => hX ▶ memB_cone τ
  master_rel := ⟨(iterSys T).master_mem, [], cone.nil.symm, by rw [encSet_nil]; exact
    Set.subset_univ _⟩
  inter_right := by
    rintro W X X' ⟨hW, τ, hX, hWτ⟩ ⟨_, τ', hX', hWτ'⟩
    subst hX; subst hX'
    rcases comparable_of_agree (fun i b b' => encSet_agree hW hWτ hWτ') with hpre |
    hpre
    · exact ⟨hW, τ', by rw [Set.inter_eq_right.mpr (cone.subset_cone.mpr hpre)], hWτ'⟩
    · exact ⟨hW, τ, by rw [Set.inter_eq_left.mpr (cone.subset_cone.mpr hpre)], hWτ⟩
  mono := by
    rintro W W₂ X X₂ ⟨_, τ, hX, hWτ⟩ hW₂W hXX₂ hW₂ hX₂
    obtain ⟨τ₂, rfl⟩ := hX₂
    subst hX
    have hpre : τ₂ <+: τ := cone.subset_cone.mp hXX₂
    exact ⟨hW₂, τ₂, rfl, hW₂W.trans (hWτ.trans (encSet_anti hpre))⟩

theorem exists_not_mem_bitNbhd (b : Bool) : ∃ t : Token, t ∈ bitNbhd b := by
  cases b
  · exact ⟨1, by simp [bitNbhd, TwoToken.zero]⟩
  · exact ⟨0, by simp [bitNbhd, TwoToken.one]⟩

theorem fg_le_id : f.comp g ≤ idMap (iterSys T) := by
  intro W Z
  rw [comp_rel, idMap_rel]
  rintro ⟨X, ⟨hW, τ, hX, hWτ⟩, σ, hXσ, hZmem, hσZ⟩
  obtain rfl : τ = σ := cone.injective (hX.symm.trans hXσ)
  exact ⟨hW, hZmem, hWτ.trans hσZ⟩

theorem prefix_of_encSet_subset {σ τ : Str} (h : encSet σ ⊆ encSet τ) : τ <+: σ := by
  have hag : ∀ (i : ℕ) (b b' : Bool), σ[i]? = some b → τ[i]? = some b' → b = b' :=
    fun i b b' => encSet_agree (encSet_mem σ) subset_rfl h
  rcases comparable_of_agree hag with hστ | hτσ
  · by_cases hlen : σ.length = τ.length
    · exact (hστ.eq_of_length hlen) ▶ List.prefix_rfl
    · exfalso
      have hlt : σ.length < τ.length := lt_of_le_of_ne hστ.length_le hlen
      have hb' : τ[σ.length]? = some τ[σ.length] := List.getElem?_eq_getElem hlt
      obtain ⟨t, ht⟩ := exists_not_mem_bitNbhd τ[σ.length]
      have hmemσ : (σ.length, t) ∈ encSet σ := by
        intro b hb
        rw [List.getElem?_eq_none_iff.mpr (le_refl _)] at hb
        exact absurd hb (by simp)
      exact ht ((h hmemσ) _ hb')
  · exact hτσ
```

```
theorem gf_eq_id : g.comp f = idMap B := by
  apply ApproximableMap.ext
  intro X Y
  rw [comp_rel, idMap_rel]
  constructor
  · rintro ⟨W, ⟨σ, hX, _, hσW⟩, _, τ, hY, hWτ⟩
    refine ⟨hX ▸ memB_cone σ, hY ▸ memB_cone τ, ?_⟩
    have hsub : encSet σ ⊆ encSet τ := hσW.trans hWτ
    rw [hX, hY]
    exact cone.subset_cone.mpr (prefix_of_encSet_subset hsub)
  · rintro ⟨hX, hY, hXY⟩
    obtain ⟨σ, rfl⟩ := hX
    obtain ⟨τ, rfl⟩ := hY
    have hpre : τ <+: σ := cone.subset_cone.mp hXY
    exact ⟨encSet σ, ⟨σ, rfl, encSet_mem σ, subset_rfl⟩,
      encSet_mem σ, τ, rfl, encSet_anti hpre⟩

theorem f_injective : Function.Injective f.toElementMap := by
  intro x y hxy
  have : (g.comp f).toElementMap x = (g.comp f).toElementMap y := by
    rw [toElementMap_comp, toElementMap_comp, hxy]
  rwa [gf_eq_id, toElementMap_idMap, toElementMap_idMap] at this

end DomainTheory.NeighbourhoodSystems.EmbBinSeq
```

4.4.32 Exercise 3.17 — embedding binary sequences

Using the $\mathcal{B}$ and $\mathcal{T}$ of Example 2.3 show there is a one-one approximable mapping

$$\{f : \mathcal{B} \to \mathcal{T}^\infty\}$$

and another approximable mapping

$$\{g : \mathcal{T}^\infty \to \mathcal{B}\}$$

such that

$$g \circ f = I_{\mathcal{B}} \quad \text{and} \quad f \circ g \subseteq I_{\mathcal{T}^\infty}.$$

Are $\mathcal{B}$ and $\mathcal{T}^\infty$ isomorphic? Are $\mathcal{B}$ and $\mathcal{T} \times \mathcal{B}$ isomorphic?

The embedding and left inverse are with the embedding binary sequences peer.

Theorem (`example_317`). The forward embedding after the reverse map stays below the identity on the sequence system, and the reverse after the forward recovers the identity on $\mathcal{B}$.

Proof. Invoke the corresponding composition laws. ■

`Exercise_3_17.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.embBinSeq

@[expose] public section
```

```
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem BitDom ApproximableMap
open DomainTheory.NeighbourhoodSystems.EmbBinSeq
/-- Scott's Exercise 3.17: embedding of binary sequences with a left inverse. -/
theorem example_317 :
    f.comp g ≤ idMap (iterSys T) ∧ g.comp f = idMap B :=
  ⟨fg_le_id, gf_eq_id⟩
```

4.4.33 Sum system

The separated sum of two neighbourhood systems, with injections and left-inverse projections.

Definition (`il`). The left tag of a token is that token placed in the left summand, then wrapped as a defined optional token.

Proof. By definition. ■

Definition (`inj0`). The left injection of a set is the image of that set under left tagging.

Proof. By definition. ■

Theorem (`il_mem_inj0`). A left-tagged token lies in a left injection precisely when the token lies in the underlying set.

Proof. Unpack the image. ■

Definition (`ir`). The right tag of a token is that token placed in the right summand, then wrapped as a defined optional token.

Proof. By definition. ■

Definition (`inj1`). The right injection of a set is the image of that set under right tagging.

Proof. By definition. ■

Theorem (`ir_mem_inj0`). A right-tagged token never lies in a left injection.

Proof. The tags disagree. ■

Theorem (`none_mem_inj0`). The undefined token never lies in a left injection.

Proof. Left injections contain only left-tagged tokens. ■

Theorem (`ir_mem_inj1`). A right-tagged token lies in a right injection precisely when the token lies in the underlying set.

Proof. Unpack the image. ■

Theorem (`il_mem_inj1`). A left-tagged token never lies in a right injection.

Proof. The tags disagree. ■

Theorem (`none_mem_inj1`). The undefined token never lies in a right injection.

Proof. Right injections contain only right-tagged tokens. ■

Theorem (`inj0_inter`). Left injection commutes with intersection.

Proof. Compare membership by cases on optional tagged tokens. ■

Theorem (`inj0_inter_inj1`). A left injection meets a right injection emptily.

Proof. No optional tagged token can belong to both. ■

Theorem (`inj0_nonempty`). Left injection preserves nonempty sets.

Proof. Map a witness under left tagging. ■

Theorem (`inj0_subset_inj0`). Inclusion of left injections is equivalent to inclusion of the underlying sets.

Proof. Transport membership through left tagging. ■

Theorem (`inj1_inter`). Right injection commutes with intersection.

Proof. Compare membership by cases on optional tagged tokens. ■

Theorem (`inj1_nonempty`). Right injection preserves nonempty sets.

Proof. Map a witness under right tagging. ■

Theorem (`inj1_subset_inj1`). Inclusion of right injections is equivalent to inclusion of the underlying sets.

Proof. Transport membership through right tagging. ■

Definition (`sumMaster`). The sum master is the undefined token together with the left and right injections of the two masters.

Proof. By definition. ■

Theorem (`none_mem_sumMaster`). The undefined token belongs to the sum master.

Proof. It is inserted by definition. ■

Theorem (`il_injective`). Left tagging is injective.

Proof. Optional and summand constructors are injective. ■

Theorem (`il_preimage_inj0`). Preimaging a left injection under left tagging recovers the underlying set.

Proof. Apply injectivity of left tagging. ■

Theorem (`inj0_subset_sumMaster`). The left injection of a neighbourhood lies below the sum master.

Proof. Bound the underlying set by the left master and embed. ■

Theorem (`inj1_subset_sumMaster`). The right injection of a neighbourhood lies below the sum master.

Proof. Bound the underlying set by the right master and embed. ■

Theorem (`ir_injective`). Right tagging is injective.

Proof. Optional and summand constructors are injective. ■

Theorem (`ir_preimage_inj1`). Preimaging a right injection under right tagging recovers the underlying set.

Proof. Apply injectivity of right tagging. ■

Definition (`leftPart`). The left part of a sum set relative to the left system is the set of left tokens that are either present as left tags or belong to the left master whenever a right tag or the undefined token appears.

Proof. By definition. ■

Theorem (`mem_leftPart`). Membership in a left part unpacks as left-tag membership or the master-and-cross-witness clause.

Proof. By definition. ■

Definition (`rightPart`). The right part of a sum set relative to the right system is symmetric to the left part.

Proof. By definition. ■

Theorem (`mem_rightPart`). Membership in a right part unpacks as right-tag membership or the master-and-cross-witness clause.

Proof. By definition. ■

Theorem (`leftPart_inj0`). The left part of a left injection recovers the underlying set.

Proof. Cross witnesses are absent from left injections. ■

Theorem (`leftPart_mono`). Left parts are monotone in the sum set.

Proof. Apply the ambient inclusion to each clause. ■

Theorem (`rightPart_inj1`). The right part of a right injection recovers the underlying set.

Proof. Cross witnesses are absent from right injections. ■

Theorem (`leftPart_inj1`). The left part of a nonempty right injection is the left master.

Proof. A right-tag witness forces the master clause for every left token in the master. ■

Theorem (`leftPart_sumMaster`). The left part of the sum master is the left master.

Proof. The undefined token forces the master clause; left tags recover the left master injection. ■

Theorem (`rightPart_inj0`). The right part of a nonempty left injection is the right master.

Proof. Symmetric to the left-part fact for right injections. ■

Theorem (`rightPart_mono`). Right parts are monotone in the sum set.

Proof. Apply the ambient inclusion to each clause. ■

Theorem (`rightPart_sumMaster`). The right part of the sum master is the right master.

Proof. Symmetric to the left-part fact for the sum master. ■

Theorem (`sumMaster_inter_inj0`). Intersecting the sum master with a left-injected neighbourhood recovers that injection.

Proof. The injection already lies below the sum master. ■

Theorem (`sumMaster_inter_inj1`). Intersecting the sum master with a right-injected neighbourhood recovers that injection.

Proof. The injection already lies below the sum master. ■

Definition (`sum`). Under the standing nonempty-neighbourhood hypotheses, the separated sum consists of the sum master together with all left and right injections of neighbourhoods.

Proof. The master is the sum master; subsets of neighbourhoods stay below it via the injection bounds; intersection is checked by cases on the three kinds of sum neighbourhoods, using emptiness of mixed injections and nonempty fibres to rule out impossible lower bounds. ■

Definition (`inMap0`). The left injection map relates a left neighbourhood to a sum neighbourhood precisely when the left injection of the former is contained in the latter.

Proof. Domain and codomain membership unpack the witnesses; the masters relate by the left-master bound; intersection on the right and monotonicity use the sum intersection law and monotonicity of left injection. ■

Definition (`inMap1`). The right injection map is symmetric to the left injection map.

Proof. Symmetric to the left case. ■

Theorem (`leftPart_mem`). The left part of a sum neighbourhood is a left neighbourhood.

Proof. Case on the three kinds of sum neighbourhoods and apply the left-part identities. ■

Definition (`outMap0`). The left projection relates a sum neighbourhood to a left neighbourhood precisely when the left part of the former is contained in the latter.

Proof. Domain and codomain membership unpack the witnesses; the masters relate by the left-part identity for the sum master; intersection on the right and monotonicity use left-part membership and monotonicity. ■

Theorem (`rightPart_mem`). The right part of a sum neighbourhood is a right neighbourhood.

Proof. Case on the three kinds of sum neighbourhoods and apply the right-part identities. ■

Definition (`outMap1`). The right projection is symmetric to the left projection.

Proof. Symmetric to the left case. ■

Theorem (`inj0_injective`). Left injection of sets is injective.

Proof. Compare inclusions both ways. ■

Theorem (`inj1_injective`). Right injection of sets is injective.

Proof. Compare inclusions both ways. ■

Theorem (`outMap0_comp_inMap0`). The left projection after the left injection recovers the identity on the left system.

Proof. Compare relations via the left-part identity for left injections. ■

Theorem (`outMap1_comp_inMap1`). The right projection after the right injection recovers the identity on the right system.

Proof. Compare relations via the right-part identity for right injections. ■

Theorem (`eq_sumMaster_of_subset`). A sum neighbourhood containing the sum master equals the sum master.

Proof. Apply the master bound in the other direction. ■

Theorem (`mem_subset_inj0`). A sum neighbourhood contained in a left injection is itself a left injection of a left neighbourhood.

Proof. The master and right-injection cases are impossible by tag mismatches. ■

Theorem (`mem_subset_inj1`). A sum neighbourhood contained in a right injection is itself a right injection of a right neighbourhood.

Proof. Symmetric to the left case. ■

Theorem (`not_inj0_subset_inj1`). A nonempty left injection cannot lie below a right injection.

Proof. A left-tagged witness cannot belong to a right injection. ■

Theorem (`not_inj1_subset_inj0`). A nonempty right injection cannot lie below a left injection.

Proof. A right-tagged witness cannot belong to a left injection. ■

`sumSys.lean`

```
module
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.idMap

public import Mathlib.Data.Set.Operations
public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Image
public import Mathlib.Data.Set.Insert
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {α β : Type*}

def il (a : α) : Option (α ? β) := some (Sum.inl a)

def inj₀ (X : Set α) : Set (Option (α ? β)) := il '' X

@[simp] theorem il_mem_inj₀ {X : Set α} {a : α} : (il a : Option (α ? β)) ∈ inj₀ X ↔ a
    ∈ X := by
  simp only [inj₀, Set.mem_image, il]
  constructor
  · rintro ⟨a', ha', hb⟩; simp only [Option.some.injEq, Sum.inl.injEq] at hb; exact hb
    ▸ ha'
  · intro ha; exact ⟨a, ha, rfl⟩

def ir (b : β) : Option (α ? β) := some (Sum.inr b)

def inj₁ (Y : Set β) : Set (Option (α ? β)) := ir '' Y

@[simp] theorem ir_mem_inj₀ {X : Set α} {b : β} : (ir b : Option (α ? β)) ∈ inj₀ X :=
   by
  rintro ⟨a, _, hb⟩; exact absurd hb (by simp [il, ir])
```

```
@[simp] theorem none_mem_inj₀ {X : Set α} : (none : Option (α ? β)) ∈ inj₀ X := by
  rintro ⟨a, _, hb⟩; exact absurd hb (by simp [il])

@[simp] theorem ir_mem_inj₁ {Y : Set β} {b : β} : (ir b : Option (α ? β)) ∈ inj₁ Y ↔ b
    ∈ Y := by
  simp only [inj₁, Set.mem_image, ir]
  constructor
  · rintro ⟨b', hb', hb⟩; simp only [Option.some.injEq, Sum.inr.injEq] at hb; exact hb
    ▸ hb'
  · intro hb; exact ⟨b, hb, rfl⟩

@[simp] theorem il_mem_inj₁ {Y : Set β} {a : α} : (il a : Option (α ? β)) ∈ inj₁ Y :=
   by
  rintro ⟨b, _, hb⟩; exact absurd hb (by simp [il, ir])

@[simp] theorem none_mem_inj₁ {Y : Set β} : (none : Option (α ? β)) ∈ inj₁ Y := by
  rintro ⟨b, _, hb⟩; exact absurd hb (by simp [ir])

theorem inj₀_inter (X X' : Set α) :
    (inj₀ X ∩ inj₀ X' : Set (Option (α ? β))) = inj₀ (X ∩ X') := by
  ext t; rcases t with _ | (a | b) <;>
    simp [Set.mem_inter_iff, il, inj₀]

theorem inj₀_inter_inj₁ (X : Set α) (Y : Set β) :
    (inj₀ X ∩ inj₁ Y : Set (Option (α ? β))) = ∅ := by
  rw [Set.eq_empty_iff_forall_notMem]
  rintro t ⟨ht0, ht1⟩
  rcases t with _ | (a | b)
  · exact none_mem_inj₀ ht0
  · exact il_mem_inj₁ ht1
  · exact ir_mem_inj₀ ht0

theorem inj₀_nonempty {X : Set α} (hX : X.Nonempty) : (inj₀ X : Set (Option (α ? β))).
   Nonempty :=
  Set.Nonempty.image il hX

theorem inj₀_subset_inj₀ {X X' : Set α} :
    (inj₀ X : Set (Option (α ? β))) ⊆ inj₀ X' ↔ X ⊆ X' := by
  constructor
  · intro h a ha; exact il_mem_inj₀.mp (h (il_mem_inj₀.mpr ha))
  · intro h t ht
    rw [inj₀, Set.mem_image] at ht
    obtain ⟨a, ha, rfl⟩ := ht
    exact il_mem_inj₀.mpr (h ha)

theorem inj₁_inter (Y Y' : Set β) :
    (inj₁ Y ∩ inj₁ Y' : Set (Option (α ? β))) = inj₁ (Y ∩ Y') := by
  ext t; rcases t with _ | (a | b) <;>
    simp [Set.mem_inter_iff, ir, inj₁]

theorem inj₁_nonempty {Y : Set β} (hY : Y.Nonempty) : (inj₁ Y : Set (Option (α ? β))).
   Nonempty :=
  Set.Nonempty.image ir hY

theorem inj₁_subset_inj₁ {Y Y' : Set β} :
```

```
    (inj₁ Y : Set (Option (α ? β))) ⊆ inj₁ Y' ↔ Y ⊆ Y' := by
  constructor
  · intro h b hb; exact ir_mem_inj₁.mp (h (ir_mem_inj₁.mpr hb))
  · intro h t ht
    rw [inj₁, Set.mem_image] at ht
    obtain ⟨b, hb, rfl⟩ := ht
    exact ir_mem_inj₁.mpr (h hb)
end

section
open NeighborhoodSystem ApproximableMap
variable {α β : Type*}
variable (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)

def sumMaster : Set (Option (α ? β)) := insert none (inj₀ V₀.master ∪ inj₁ V₁.master)
end

section
open NeighborhoodSystem ApproximableMap
variable {α β : Type*}
variable (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
variable {V₀ V₁}

@[simp] theorem none_mem_sumMaster : (none : Option (α ? β)) ∈ sumMaster V₀ V₁ :=
  Set.mem_insert _ _

theorem il_injective : Function.Injective (il : α → Option (α ? β)) := by
  intro a a' h; simpa [il] using h

@[simp] theorem il_preimage_inj₀ (X : Set α) :
    ((il : α → Option (α ? β)) ^-1' inj₀ X) = X :=
  Set.preimage_image_eq X il_injective

theorem inj₀_subset_sumMaster {X : Set α} (hX : V₀.mem X) :
    (inj₀ X : Set (Option (α ? β))) ⊆ sumMaster V₀ V₁ := by
  intro t ht
  refine Set.mem_insert_iff.mpr (Or.inr (Set.mem_union_left _ ?_))
  exact (inj₀_subset_inj₀.mpr (V₀.sub_master hX)) ht

theorem inj₁_subset_sumMaster {Y : Set β} (hY : V₁.mem Y) :
    (inj₁ Y : Set (Option (α ? β))) ⊆ sumMaster V₀ V₁ := by
  intro t ht
  refine Set.mem_insert_iff.mpr (Or.inr (Set.mem_union_right _ ?_))
  exact (inj₁_subset_inj₁.mpr (V₁.sub_master hY)) ht

theorem ir_injective : Function.Injective (ir : β → Option (α ? β)) := by
  intro b b' h; simpa [ir] using h

@[simp] theorem ir_preimage_inj₁ (Y : Set β) :
    ((ir : β → Option (α ? β)) ^-1' inj₁ Y) = Y :=
  Set.preimage_image_eq Y ir_injective

def leftPart (V₀ : NeighborhoodSystem α) (W : Set (Option (α ? β))) : Set α :=
  il ^-1' W ∪ {a | a ∈ V₀.master ∧ ((∃ b : β, ir b ∈ W) ∨ (none : Option (α ? β)) ∈ W)
    }
```

```
@[simp] theorem mem_leftPart {V₀ : NeighborhoodSystem α} {W : Set (Option (α ? β))} {a
     : α} :
    a ∈ leftPart V₀ W ↔ il a ∈ W ∨ (a ∈ V₀.master ∧ ((∃ b : β, ir b ∈ W) ∨ none ∈ W))
     := by
  simp only [leftPart, Set.mem_union, Set.mem_preimage, Set.mem_ofPred_eq]

def rightPart (V₁ : NeighborhoodSystem β) (W : Set (Option (α ? β))) : Set β :=
  ir ^-1' W ∪ {b | b ∈ V₁.master ∧ ((∃ a : α, il a ∈ W) ∨ (none : Option (α ? β)) ∈ W)
    }

@[simp] theorem mem_rightPart {V₁ : NeighborhoodSystem β} {W : Set (Option (α ? β))} {b
     : β} :
    b ∈ rightPart V₁ W ↔ ir b ∈ W ∨ (b ∈ V₁.master ∧ ((∃ a : α, il a ∈ W) ∨ none ∈ W)
    ) := by
  simp only [rightPart, Set.mem_union, Set.mem_preimage, Set.mem_ofPred_eq]

@[simp] theorem leftPart_inj₀ (V₀ : NeighborhoodSystem α) (X : Set α) :
    leftPart V₀ (inj₀ X : Set (Option (α ? β))) = X := by
  ext a
  simp only [mem_leftPart, il_mem_inj₀, ir_mem_inj₀, none_mem_inj₀, exists_false,
    or_self,
    and_false]
  exact ⟨fun h => h.resolve_right (by simp), fun h => Or.inl h⟩

theorem leftPart_mono (V₀ : NeighborhoodSystem α) {W W' : Set (Option (α ? β))} (h : W
     ⊆ W') :
    leftPart V₀ W ⊆ leftPart V₀ W' := by
  intro a ha
  rw [mem_leftPart] at ha ⊢
  exact ha.imp (fun h' => h h') (fun ⟨hm, hc⟩ => ⟨hm, hc.imp (fun ⟨b, hb⟩ => ⟨b, h hb⟩) (
    fun hn => h hn)⟩)

@[simp] theorem rightPart_inj₁ (V₁ : NeighborhoodSystem β) (Y : Set β) :
    rightPart V₁ (inj₁ Y : Set (Option (α ? β))) = Y := by
  ext b
  simp only [mem_rightPart, ir_mem_inj₁, il_mem_inj₁, none_mem_inj₁, exists_false,
    or_self,
    and_false]
  exact ⟨fun h => h.resolve_right (by simp), fun h => Or.inl h⟩

theorem leftPart_inj₁ (V₀ : NeighborhoodSystem α) {Y : Set β} (hY : Y.Nonempty) :
    leftPart V₀ (inj₁ Y : Set (Option (α ? β))) = V₀.master := by
  ext a
  simp only [mem_leftPart, il_mem_inj₁, none_mem_inj₁, or_false, false_or]
  constructor
  · rintro ⟨ha, _⟩; exact ha
  · intro ha; obtain ⟨b, hb⟩ := hY; exact ⟨ha, ⟨b, ir_mem_inj₁.mpr hb⟩⟩

theorem leftPart_sumMaster (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    leftPart V₀ (sumMaster V₀ V₁) = V₀.master := by
  ext a
  simp only [mem_leftPart, none_mem_sumMaster, or_true, and_true]
  constructor
  · rintro (h | h)
    · have : (il a : Option (α ? β)) ∈ sumMaster V₀ V₁ := h
      rcases Set.mem_insert_iff.mp this with h' | h'
```

```
      · exact absurd h' (by simp [il])
      · rcases h' with h'' | h''
        · exact (il_mem_inj₀).mp h''
        · exact absurd h'' (by simp)
    · exact h
  · intro ha; exact Or.inr ha

theorem rightPart_inj₀ (V₁ : NeighborhoodSystem β) {X : Set α} (hX : X.Nonempty) :
    rightPart V₁ (inj₀ X : Set (Option (α ⊕ β))) = V₁.master := by
  ext b
  simp only [mem_rightPart, ir_mem_inj₀, none_mem_inj₀, or_false, false_or]
  constructor
  · rintro ⟨hb, _⟩; exact hb
  · intro hb; obtain ⟨a, ha⟩ := hX; exact ⟨hb, ⟨a, il_mem_inj₀.mpr ha⟩⟩

theorem rightPart_mono (V₁ : NeighborhoodSystem β) {W W' : Set (Option (α ⊕ β))} (h : W
    ⊆ W') :
    rightPart V₁ W ⊆ rightPart V₁ W' := by
  intro b hb
  rw [mem_rightPart] at hb ⊢
  exact hb.imp (fun h' => h h') (fun ⟨hm, hc⟩ => ⟨hm, hc.imp (fun ⟨a, ha⟩ => ⟨a, h ha⟩) (
    fun hn => h hn)⟩)

theorem rightPart_sumMaster (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    rightPart V₁ (sumMaster V₀ V₁) = V₁.master := by
  ext b
  simp only [mem_rightPart, none_mem_sumMaster, or_true, and_true]
  constructor
  · rintro (h | h)
    · have : (ir b : Option (α ⊕ β)) ∈ sumMaster V₀ V₁ := h
      rcases Set.mem_insert_iff.mp this with h' | h'
      · exact absurd h' (by simp [ir])
      · rcases h' with h'' | h''
        · exact absurd h'' (by simp)
        · exact (ir_mem_inj₁).mp h''
    · exact h
  · intro hb; exact Or.inr hb

theorem sumMaster_inter_inj₀ {X : Set α} (hX : V₀.mem X) :
    (sumMaster V₀ V₁ ∩ inj₀ X : Set (Option (α ⊕ β))) = inj₀ X :=
  Set.inter_eq_right.mpr (inj₀_subset_sumMaster hX)

theorem sumMaster_inter_inj₁ {Y : Set β} (hY : V₁.mem Y) :
    (sumMaster V₀ V₁ ∩ inj₁ Y : Set (Option (α ⊕ β))) = inj₁ Y :=
  Set.inter_eq_right.mpr (inj₁_subset_sumMaster hY)

def sum (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty) :
    NeighborhoodSystem (Option (α ⊕ β)) where
  mem W := W = sumMaster V₀ V₁ ∨ (∃ X, V₀.mem X ∧ W = inj₀ X) ∨ (∃ Y, V₁.mem Y ∧ W =
    inj₁ Y)
  master := sumMaster V₀ V₁
  master_nonempty := ⟨none, none_mem_sumMaster⟩
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩)
```

```
    · exact subset_rfl
    · exact inj₀_subset_sumMaster hX
    · exact inj₁_subset_sumMaster hY
  inter_mem := by

    have hne : ∀ W, (W = sumMaster V₀ V₁ ∨ (∃ X, V₀.mem X ∧ W = inj₀ X) ∨
        (∃ Y, V₁.mem Y ∧ W = inj₁ Y)) → (W : Set (Option (α ? β))).Nonempty := by
      rintro W (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩)
      · exact ⟨none, none_mem_sumMaster⟩
      · exact inj₀_nonempty (h₀ X hX)
      · exact inj₁_nonempty (h₁ Y hY)
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · rw [Set.inter_self]; exact Or.inl rfl
      · rw [sumMaster_inter_inj₀ hX']; exact Or.inr (Or.inl ⟨X', hX', rfl⟩)
      · rw [sumMaster_inter_inj₁ hY']; exact Or.inr (Or.inr ⟨Y', hY', rfl⟩)
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · rw [Set.inter_comm, sumMaster_inter_inj₀ hX]; exact Or.inr (Or.inl ⟨X, hX, rfl⟩)
      · rw [inj₀_inter] at hZsub ⊢

        rcases hZ with rfl | ⟨Z₀, hZ₀, rfl⟩ | ⟨Z₁, hZ₁, rfl⟩
        · exact absurd (hZsub none_mem_sumMaster) none_mem_inj₀
        · refine Or.inr (Or.inl ⟨X ∩ X', V₀.inter_mem hX hX' hZ₀ (inj₀_subset_inj₀.mp
    hZsub), rfl⟩)
        · obtain ⟨b, hb⟩ := h₁ Z₁ hZ₁
          exact absurd (hZsub (ir_mem_inj₁.mpr hb)) ir_mem_inj₀
      · rw [inj₀_inter_inj₁] at hZsub ⊢
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · rw [Set.inter_comm, sumMaster_inter_inj₁ hY]; exact Or.inr (Or.inr ⟨Y, hY, rfl⟩)
      · rw [Set.inter_comm, inj₀_inter_inj₁] at hZsub ⊢
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      · rw [inj₁_inter] at hZsub ⊢
        rcases hZ with rfl | ⟨Z₀, hZ₀, rfl⟩ | ⟨Z₁, hZ₁, rfl⟩
        · exact absurd (hZsub none_mem_sumMaster) none_mem_inj₁
        · obtain ⟨a, ha⟩ := h₀ Z₀ hZ₀
          exact absurd (hZsub (il_mem_inj₀.mpr ha)) il_mem_inj₁
        · refine Or.inr (Or.inr ⟨Y ∩ Y', V₁.inter_mem hY hY' hZ₁ (inj₁_subset_inj₁.mp
    hZsub), rfl⟩)
end

section
open NeighborhoodSystem ApproximableMap
variable {α β : Type*}
variable (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
variable {V₀ V₁}
variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}

def inMap₀ : ApproximableMap V₀ (sum V₀ V₁ h₀ h₁) where
  rel X W := V₀.mem X ∧ (sum V₀ V₁ h₀ h₁).mem W ∧ inj₀ X ⊆ W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨V₀.master_mem, (sum V₀ V₁ h₀ h₁).master_mem, inj₀_subset_sumMaster V₀.
    master_mem⟩
  inter_right := by
```

```
    rintro X W W' ⟨hX, hW, hsub⟩ ⟨_, hW', hsub'⟩
    exact ⟨hX, (sum V₀ V₁ h₀ h₁).inter_mem hW hW' (Or.inr (Or.inl ⟨X, hX, rfl⟩))
      (Set.subset_inter hsub hsub'), Set.subset_inter hsub hsub'⟩
  mono := by
    rintro X X' W W' ⟨_, _, hsub⟩ hX'X hWW' hX' hW'
    exact ⟨hX', hW', (inj₀_subset_inj₀.mpr hX'X).trans (hsub.trans hWW')⟩

def inMap₁ : ApproximableMap V₁ (sum V₀ V₁ h₀ h₁) where
  rel Y W := V₁.mem Y ∧ (sum V₀ V₁ h₀ h₁).mem W ∧ inj₁ Y ⊆ W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨V₁.master_mem, (sum V₀ V₁ h₀ h₁).master_mem, inj₁_subset_sumMaster V₁.
    master_mem⟩
  inter_right := by
    rintro Y W W' ⟨hY, hW, hsub⟩ ⟨_, hW', hsub'⟩
    exact ⟨hY, (sum V₀ V₁ h₀ h₁).inter_mem hW hW' (Or.inr (Or.inr ⟨Y, hY, rfl⟩))
      (Set.subset_inter hsub hsub'), Set.subset_inter hsub hsub'⟩
  mono := by
    rintro Y Y' W W' ⟨_, _, hsub⟩ hY'Y hWW' hY' hW'
    exact ⟨hY', hW', (inj₁_subset_inj₁.mpr hY'Y).trans (hsub.trans hWW')⟩

theorem leftPart_mem {W : Set (Option (α ? β))} (hW : (sum V₀ V₁ h₀ h₁).mem W) :
    V₀.mem (leftPart V₀ W) := by
  rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
  · rw [leftPart_sumMaster]; exact V₀.master_mem
  · rw [leftPart_inj₀]; exact hX
  · rw [leftPart_inj₁ V₀ (h₁ Y hY)]; exact V₀.master_mem

def outMap₀ : ApproximableMap (sum V₀ V₁ h₀ h₁) V₀ where
  rel W X := (sum V₀ V₁ h₀ h₁).mem W ∧ V₀.mem X ∧ leftPart V₀ W ⊆ X
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(sum V₀ V₁ h₀ h₁).master_mem, V₀.master_mem, (leftPart_sumMaster V₀ V₁).
    subset⟩
  inter_right := by
    rintro W X X' ⟨hW, hX, hsub⟩ ⟨_, hX', hsub'⟩
    exact ⟨hW, V₀.inter_mem hX hX' (leftPart_mem hW) (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'⟩
  mono := by
    rintro W W' X X' ⟨_, _, hsub⟩ hW'W hXX' hW' hX'
    exact ⟨hW', hX', (leftPart_mono V₀ hW'W).trans (hsub.trans hXX')⟩

theorem rightPart_mem {W : Set (Option (α ? β))} (hW : (sum V₀ V₁ h₀ h₁).mem W) :
    V₁.mem (rightPart V₁ W) := by
  rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
  · rw [rightPart_sumMaster]; exact V₁.master_mem
  · rw [rightPart_inj₀ V₁ (h₀ X hX)]; exact V₁.master_mem
  · rw [rightPart_inj₁]; exact hY

def outMap₁ : ApproximableMap (sum V₀ V₁ h₀ h₁) V₁ where
  rel W Y := (sum V₀ V₁ h₀ h₁).mem W ∧ V₁.mem Y ∧ rightPart V₁ W ⊆ Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(sum V₀ V₁ h₀ h₁).master_mem, V₁.master_mem, (rightPart_sumMaster V₀ V₁)
    .subset⟩
  inter_right := by
```

```
    rintro W Y Y' ⟨hW, hY, hsub⟩ ⟨_, hY', hsub'⟩
    exact ⟨hW, V₁.inter_mem hY hY' (rightPart_mem hW) (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'⟩
  mono := by
    rintro W W' Y Y' ⟨_, _, hsub⟩ hW'W hYY' hW' hY'
    exact ⟨hW', hY', (rightPart_mono V₁ hW'W).trans (hsub.trans hYY')⟩
end

section
open NeighborhoodSystem ApproximableMap
variable {α β : Type*}

theorem inj₀_injective {X X' : Set α} (h : (inj₀ X : Set (Option (α ⊕ β))) = inj₀ X') :
    X = X' :=
  Set.Subset.antisymm (inj₀_subset_inj₀.mp h.subset) (inj₀_subset_inj₀.mp h.symm.subset
    )

theorem inj₁_injective {Y Y' : Set β} (h : (inj₁ Y : Set (Option (α ⊕ β))) = inj₁ Y') :
    Y = Y' :=
  Set.Subset.antisymm (inj₁_subset_inj₁.mp h.subset) (inj₁_subset_inj₁.mp h.symm.subset
    )
end

section
open NeighborhoodSystem ApproximableMap
variable {α β : Type*}
variable (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
variable {V₀ V₁}
variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}

theorem outMap₀_comp_inMap₀ :
    (outMap₀ (V₀ := V₀) (V₁ := V₁) (h₀ := h₀) (h₁ := h₁)).comp inMap₀ = idMap V₀ := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  · rintro ⟨W, ⟨hX, _, hinj⟩, _, hZ, hsub⟩
    refine ⟨hX, hZ, ?_⟩
    have hXW : X ⊆ leftPart V₀ W := by
      intro a ha
      rw [mem_leftPart]; exact Or.inl (hinj (il_mem_inj₀.mpr ha))
    exact hXW.trans hsub
  · rintro ⟨hX, hZ, hXZ⟩
    refine ⟨inj₀ X, ⟨hX, Or.inr (Or.inl ⟨X, hX, rfl⟩)), subset_rfl⟩,
      Or.inr (Or.inl ⟨X, hX, rfl⟩)), hZ, ?_⟩
    rw [leftPart_inj₀]; exact hXZ

theorem outMap₁_comp_inMap₁ :
    (outMap₁ (V₀ := V₀) (V₁ := V₁) (h₀ := h₀) (h₁ := h₁)).comp inMap₁ = idMap V₁ := by
  apply ApproximableMap.ext
  intro Y Z
  constructor
  · rintro ⟨W, ⟨hY, _, hinj⟩, _, hZ, hsub⟩
    refine ⟨hY, hZ, ?_⟩
    have hYW : Y ⊆ rightPart V₁ W := by
      intro b hb
      rw [mem_rightPart]; exact Or.inl (hinj (ir_mem_inj₁.mpr hb))
```

```
    exact hYW.trans hsub
  · rintro ⟨hY, hZ, hYZ⟩
    refine ⟨inj₁ Y, ⟨hY, Or.inr (Or.inr ⟨Y, hY, rfl⟩), subset_rfl⟩,
      Or.inr (Or.inr ⟨Y, hY, rfl⟩), hZ, ?_⟩
    rw [rightPart_inj₁]; exact hYZ

end

section
open NeighborhoodSystem ApproximableMap
variable {α β α' β' : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
variable {h₀' : ∀ X, V₀'.mem X → X.Nonempty} {h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty}

theorem eq_sumMaster_of_subset {W : Set (Option (α ? β))}
    (hW : (sum V₀ V₁ h₀ h₁).mem W) (hsub : sumMaster V₀ V₁ ⊆ W) :
    W = sumMaster V₀ V₁ :=
  Set.Subset.antisymm ((sum V₀ V₁ h₀ h₁).sub_master hW) hsub

theorem mem_subset_inj₀ {W : Set (Option (α ? β))} {X : Set α}
    (hW : (sum V₀ V₁ h₀ h₁).mem W) (hsub : W ⊆ inj₀ X) :
    ∃ X₂, V₀.mem X₂ ∧ W = inj₀ X₂ := by
  rcases hW with rfl | ⟨X₂, hX₂, rfl⟩ | ⟨Y₂, hY₂, rfl⟩
  · exact absurd (hsub none_mem_sumMaster) none_mem_inj₀
  · exact ⟨X₂, hX₂, rfl⟩
  · obtain ⟨b, hb⟩ := h₁ Y₂ hY₂
    exact absurd (hsub (ir_mem_inj₁.mpr hb)) ir_mem_inj₀

theorem mem_subset_inj₁ {W : Set (Option (α ? β))} {Y : Set β}
    (hW : (sum V₀ V₁ h₀ h₁).mem W) (hsub : W ⊆ inj₁ Y) :
    ∃ Y₂, V₁.mem Y₂ ∧ W = inj₁ Y₂ := by
  rcases hW with rfl | ⟨X₂, hX₂, rfl⟩ | ⟨Y₂, hY₂, rfl⟩
  · exact absurd (hsub none_mem_sumMaster) none_mem_inj₁
  · obtain ⟨a, ha⟩ := h₀ X₂ hX₂
    exact absurd (hsub (il_mem_inj₀.mpr ha)) il_mem_inj₁
  · exact ⟨Y₂, hY₂, rfl⟩

theorem not_inj₀_subset_inj₁ {X : Set α} {Y : Set β} (hX : X.Nonempty)
    (hsub : (inj₀ X : Set (Option (α ? β))) ⊆ inj₁ Y) : False := by
  obtain ⟨a, ha⟩ := hX
  exact il_mem_inj₁ (hsub (il_mem_inj₀.mpr ha))

theorem not_inj₁_subset_inj₀ {X : Set α} {Y : Set β} (hY : Y.Nonempty)
    (hsub : (inj₁ Y : Set (Option (α ? β))) ⊆ inj₀ X) : False := by
  obtain ⟨b, hb⟩ := hY
  exact ir_mem_inj₀ (hsub (ir_mem_inj₁.mpr hb))

end
end DomainTheory.NeighbourhoodSystems
end
```

4.4.34 Exercise 3.18 — separated sum

Let $\mathcal{D}_0$ and $\mathcal{D}_1$ be neighbourhood systems over Δ_0 and Δ_1, where we again assume that these are subsets of Σ^*. We assume that in addition *no neighbourhood is empty*. Why is this possible without loss of generality?

Define the *sum system* by:

$$\mathcal{D}_0 + \mathcal{D}_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{0X \mid X \in \mathcal{D}_0\} \cup \{1Y \mid Y \in \mathcal{D}_1\}.$$

Prove that this is a neighbourhood system over $\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1$. (Throwing in $\{\Lambda\}$ was not all that necessary, but note that $\mathcal{B} = \mathcal{B} + \mathcal{B}$, and this is an equality of sets not just an isomorphism of systems.)

Prove that in general there are mappings

$$in_i : \mathcal{D}_i \to \mathcal{D}_0 + \mathcal{D}_1 \quad \text{and} \quad out_i : \mathcal{D}_0 + \mathcal{D}_1 \to \mathcal{D}_i,$$

where $out_i \circ in_i = I_{\mathcal{D}_i}$. Where does the assumption $\emptyset \notin \mathcal{D}_i$ come in? How can these sums be generalized to n-terms? (Hint: As for products use sets $1^i 0\Delta_i$.) Draw some pictures.

The sum system and the injection/projection laws are with the sum system peer.

Theorem (`example_318`). For nonempty-neighbourhood systems, each projection after the corresponding injection recovers the identity.

Proof. Invoke the left and right composition laws. ■

`Exercise_3_18.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.sumSys

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 3.18: injections into a separated sum have left inverses. -/
theorem example_318 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty},
      (outMap₀ (V₀ := V₀) (V₁ := V₁) (h₀ := h₀) (h₁ := h₁)).comp inMap₀ = idMap V₀) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty},
      (outMap₁ (V₀ := V₀) (V₁ := V₁) (h₀ := h₀) (h₁ := h₁)).comp inMap₁ = idMap V₁) :=
  ⟨@outMap₀_comp_inMap₀, @outMap₁_comp_inMap₁⟩
```

4.4.35 Induced maps

Induced maps of a pair of approximable maps on products and separated sums.

Definition (`sumMap`). The sum of two approximable maps relates a sum neighbourhood to a sum neighbourhood of the targets by sending the sum master to the sum master and routing left and right injections through the corresponding factor maps.

Proof. Domain and codomain membership unpack the witnesses; the masters relate by the sum masters; intersection on the right and monotonicity are checked by cases on the three kinds of sum neighbourhoods, using injectivity of the tagged copies and the impossibility of mixed inclusions. ■

Theorem (`outMap0_comp_sumMap_comp_inMap0`). The left projection after the sum of maps after the left injection recovers the left factor.

Proof. Compare relations via the left-part identities for left injections and the sum master. ■

Theorem (`outMap1_comp_sumMap_comp_inMap1`). The right projection after the sum of maps after the right injection recovers the right factor.

Proof. Compare relations via the right-part identities for right injections and the sum master. ■

Definition (`prodMap`). The product of two approximable maps is the pairing of each factor after the corresponding projection.

Proof. By definition. ■

Theorem (`toElementMap_prodMap`). Elementwise, the product of maps sends a product element to the pair of the factor values at its components.

Proof. Apply the pairing and composition evaluations with the projection laws. ■

Theorem (`toElementMap_prodMap_pair`). Elementwise, the product of maps sends a pair to the pair of the factor values.

Proof. Specialise the previous identity at a pairing. ■

`inducedMaps.lean`

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.sumSys
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.ProjectionMappings
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {α β α' β' : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
variable {h₀' : ∀ X, V₀'.mem X → X.Nonempty} {h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty}

def sumMap (f : ApproximableMap V₀ V₀') (g : ApproximableMap V₁ V₁') :
    ApproximableMap (sum V₀ V₁ h₀ h₁) (sum V₀' V₁' h₀' h₁') where
  rel W W' := (sum V₀ V₁ h₀ h₁).mem W ∧ (sum V₀' V₁' h₀' h₁').mem W' ∧
    (W' = sumMaster V₀' V₁' ∨
      (∃ X Y', W = inj₀ X ∧ W' = inj₀ Y' ∧ f.rel X Y') ∨
      (∃ Y Y', W = inj₁ Y ∧ W' = inj₁ Y' ∧ g.rel Y Y'))
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(sum V₀ V₁ h₀ h₁).master_mem, (sum V₀' V₁' h₀' h₁').master_mem, Or.inl
    rfl⟩
  inter_right := by
    rintro W W'₁ W'₂ ⟨hW, hW'₁, hd₁⟩ ⟨_, hW'₂, hd₂⟩
```

```
  have hmem : ∀ W'' : Set (Option (α' ? β')),
      (W'' = sumMaster V₀' V₁' ∨
        (∃ X Y', W = inj₀ X ∧ W'' = inj₀ Y' ∧ f.rel X Y') ∨
        (∃ Y Y', W = inj₁ Y ∧ W'' = inj₁ Y' ∧ g.rel Y Y')) →
        (sum V₀' V₁' h₀' h₁').mem W'' := by
    rintro W'' (rfl | ⟨_, Y', _, rfl, hf⟩ | ⟨_, Y', _, rfl, hg⟩)
    · exact (sum V₀' V₁' h₀' h₁').master_mem
    · exact Or.inr (Or.inl ⟨Y', f.rel_cod hf, rfl⟩)
    · exact Or.inr (Or.inr ⟨Y', g.rel_cod hg, rfl⟩)

  have key : (W'₁ ∩ W'₂ = sumMaster V₀' V₁' ∨
      (∃ X Y', W = inj₀ X ∧ W'₁ ∩ W'₂ = inj₀ Y' ∧ f.rel X Y') ∨
      (∃ Y Y', W = inj₁ Y ∧ W'₁ ∩ W'₂ = inj₁ Y' ∧ g.rel Y Y')) := by
    rcases hd₁ with rfl | ⟨X, Y'₁, hWX₁, rfl, hf₁⟩ | ⟨Y, Y'₁, hWY₁, rfl, hg₁⟩
    · rw [Set.inter_eq_right.mpr
        (show W'₂ ⊆ sumMaster V₀' V₁' from (sum V₀' V₁' h₀' h₁').sub_master hW'₂)]
      exact hd₂
    · rcases hd₂ with rfl | ⟨X', Y'₂, hWX₂, rfl, hf₂⟩ | ⟨Y', Y'₂, hWY₂, rfl, hg₂⟩
      · rw [Set.inter_eq_left.mpr (inj₀_subset_sumMaster (f.rel_cod hf₁))]
        exact Or.inr (Or.inl ⟨X, Y'₁, hWX₁, rfl, hf₁⟩)
      · obtain rfl : X = X' := inj₀_injective (hWX₁.symm.trans hWX₂)
        rw [inj₀_inter]
        exact Or.inr (Or.inl ⟨X, Y'₁ ∩ Y'₂, hWX₁, rfl, f.inter_right hf₁ hf₂⟩)
      · exact absurd ((hWX₁.symm.trans hWY₂)) (fun h =>
          not_inj₀_subset_inj₁ (h₀ X (f.rel_dom hf₁)) h.subset)
    · rcases hd₂ with rfl | ⟨X', Y'₂, hWX₂, rfl, hf₂⟩ | ⟨Y', Y'₂, hWY₂, rfl, hg₂⟩
      · rw [Set.inter_eq_left.mpr (inj₁_subset_sumMaster (g.rel_cod hg₁))]
        exact Or.inr (Or.inr ⟨Y, Y'₁, hWY₁, rfl, hg₁⟩)
      · exact absurd ((hWY₁.symm.trans hWX₂)) (fun h =>
          not_inj₁_subset_inj₀ (h₁ Y (g.rel_dom hg₁)) h.subset)
      · obtain rfl : Y = Y' := inj₁_injective (hWY₁.symm.trans hWY₂)
        rw [inj₁_inter]
        exact Or.inr (Or.inr ⟨Y, Y'₁ ∩ Y'₂, hWY₁, rfl, g.inter_right hg₁ hg₂⟩)
  exact ⟨hW, hmem _ key, key⟩
mono := by
  rintro W W₂ W' W'₂ ⟨hW, hW', hd⟩ hW₂W hW'W'₂ hW₂mem hW'₂mem
  refine ⟨hW₂mem, hW'₂mem, ?_⟩
  rcases hd with rfl | ⟨X, Y', rfl, rfl, hf⟩ | ⟨Y, Y', rfl, rfl, hg⟩
  ·
    left; exact eq_sumMaster_of_subset hW'₂mem hW'W'₂
  ·
    obtain ⟨X₂, hX₂, rfl⟩ := mem_subset_inj₀ hW₂mem hW₂W
    have hXX₂ : X₂ ⊆ X := inj₀_subset_inj₀.mp hW₂W
    rcases hW'₂mem with rfl | ⟨Y'₂, hY'₂, rfl⟩ | ⟨Z'₂, hZ'₂, rfl⟩
    · left; rfl
    · have hY'Y'₂ : Y' ⊆ Y'₂ := inj₀_subset_inj₀.mp hW'W'₂
      exact Or.inr (Or.inl ⟨X₂, Y'₂, rfl, rfl,
        f.mono hf hXX₂ hY'Y'₂ hX₂ hY'₂⟩)
    · exact (not_inj₀_subset_inj₁ (h₀' Y' (f.rel_cod hf)) hW'W'₂).elim
  ·
    obtain ⟨Y₂, hY₂, rfl⟩ := mem_subset_inj₁ hW₂mem hW₂W
    have hYY₂ : Y₂ ⊆ Y := inj₁_subset_inj₁.mp hW₂W
    rcases hW'₂mem with rfl | ⟨X'₂, hX'₂, rfl⟩ | ⟨Y'₂, hY'₂, rfl⟩
    · left; rfl
    · exact (not_inj₁_subset_inj₀ (h₁' Y' (g.rel_cod hg)) hW'W'₂).elim
    · have hY'Y'₂ : Y' ⊆ Y'₂ := inj₁_subset_inj₁.mp hW'W'₂
```

```
        exact Or.inr (Or.inr ⟨Y₂, Y'₂, rfl, rfl,
          g.mono hg hYY₂ hY'Y'₂ hY₂ hY'₂⟩)

theorem outMap₀_comp_sumMap_comp_inMap₀ (f : ApproximableMap V₀ V₀') (g :
    ApproximableMap V₁ V₁') :
    (outMap₀ (h₀ := h₀') (h₁ := h₁')).comp
      ((sumMap (h₀ := h₀) (h₁ := h₁) (h₀' := h₀') (h₁' := h₁') f g).comp
        (inMap₀ (h₀ := h₀) (h₁ := h₁))) = f := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  · rintro ⟨W', ⟨W, ⟨hX, _, hinj⟩, hWmem, _, hd⟩, _, hZ, hleft⟩
    rcases hd with rfl | ⟨X₀, Y', hWX₀, rfl, hf⟩ | ⟨Y₀, Y', hWY₀, rfl, hg⟩
    ·
      rw [leftPart_sumMaster] at hleft
      have : Z = V₀'.master := Set.Subset.antisymm (V₀'.sub_master hZ) hleft
      subst this
      exact f.mono f.master_rel (V₀.sub_master hX) subset_rfl hX V₀'.master_mem
    ·
      rw [leftPart_inj₀] at hleft
      have hXX₀ : X ⊆ X₀ := inj₀_subset_inj₀.mp (hWX₀ ▸ hinj)
      exact f.mono hf hXX₀ hleft hX hZ
    ·
      exact (not_inj₀_subset_inj₁ (h₀ X hX) (hWY₀ ▸ hinj)).elim
  · intro hf
    refine ⟨inj₀ Z, ⟨inj₀ X, ⟨f.rel_dom hf, ?_, subset_rfl⟩, ?_, ?_, ?_⟩, ?_, f.rel_cod
    hf, ?_⟩
    · exact Or.inr (Or.inl ⟨X, f.rel_dom hf, rfl⟩)
    · exact Or.inr (Or.inl ⟨X, f.rel_dom hf, rfl⟩)
    · exact Or.inr (Or.inl ⟨Z, f.rel_cod hf, rfl⟩)
    · exact Or.inr (Or.inl ⟨X, Z, rfl, rfl, hf⟩)
    · exact Or.inr (Or.inl ⟨Z, f.rel_cod hf, rfl⟩)
    · exact (leftPart_inj₀ V₀' Z).subset

theorem outMap₁_comp_sumMap_comp_inMap₁ (f : ApproximableMap V₀ V₀') (g :
    ApproximableMap V₁ V₁') :
    (outMap₁ (h₀ := h₀') (h₁ := h₁')).comp
      ((sumMap (h₀ := h₀) (h₁ := h₁) (h₀' := h₀') (h₁' := h₁') f g).comp
        (inMap₁ (h₀ := h₀) (h₁ := h₁))) = g := by
  apply ApproximableMap.ext
  intro Y Z
  constructor
  · rintro ⟨W', ⟨W, ⟨hY, _, hinj⟩, hWmem, _, hd⟩, _, hZ, hright⟩
    rcases hd with rfl | ⟨X₀, Y', hWX₀, rfl, hf⟩ | ⟨Y₀, Y', hWY₀, rfl, hg⟩
    · rw [rightPart_sumMaster] at hright
      have : Z = V₁'.master := Set.Subset.antisymm (V₁'.sub_master hZ) hright
      subst this
      exact g.mono g.master_rel (V₁.sub_master hY) subset_rfl hY V₁'.master_mem
    · exact (not_inj₁_subset_inj₀ (h₁ Y hY) (hWX₀ ▸ hinj)).elim
    · rw [rightPart_inj₁] at hright
      have hYY₀ : Y ⊆ Y₀ := inj₁_subset_inj₁.mp (hWY₀ ▸ hinj)
      exact g.mono hg hYY₀ hright hY hZ
  · intro hg
    refine ⟨inj₁ Z, ⟨inj₁ Y, ⟨g.rel_dom hg, ?_, subset_rfl⟩, ?_, ?_, ?_⟩, ?_, g.rel_cod
    hg, ?_⟩
    · exact Or.inr (Or.inr ⟨Y, g.rel_dom hg, rfl⟩)
```

```
    · exact Or.inr (Or.inr ⟨Y, g.rel_dom hg, rfl⟩)
    · exact Or.inr (Or.inr ⟨Z, g.rel_cod hg, rfl⟩)
    · exact Or.inr (Or.inr ⟨Y, Z, rfl, rfl, hg⟩)
    · exact Or.inr (Or.inr ⟨Z, g.rel_cod hg, rfl⟩)
    · exact (rightPart_inj₁ V₁' Z).subset
end

section
open NeighborhoodSystem ApproximableMap
variable {α β ? α' β' : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}
variable {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}

def prodMap (f : ApproximableMap V₀ V₀') (g : ApproximableMap V₁ V₁') :
    ApproximableMap (prod V₀ V₁) (prod V₀' V₁') :=
  paired (f.comp (proj₀ V₀ V₁)) (g.comp (proj₁ V₀ V₁))

theorem toElementMap_prodMap (f : ApproximableMap V₀ V₀') (g : ApproximableMap V₁ V₁')
    (w : (prod V₀ V₁).Element) :
    (prodMap f g).toElementMap w = pair (f.toElementMap w.fst) (g.toElementMap w.snd)
    := by
  rw [prodMap, toElementMap_paired, toElementMap_comp, toElementMap_comp,
    toElementMap_proj₀,
    toElementMap_proj₁]

theorem toElementMap_prodMap_pair (f : ApproximableMap V₀ V₀') (g : ApproximableMap V₁
    V₁')
    (x : V₀.Element) (y : V₁.Element) :
    (prodMap f g).toElementMap (pair x y) = pair (f.toElementMap x) (g.toElementMap y)
     := by
  rw [toElementMap_prodMap, fst_pair, snd_pair]
end

end DomainTheory.NeighbourhoodSystems
end
```

4.4.36 Exercise 3.19 — induced maps on constructs

Suppose we are given systems and approximable mappings

$$f : \mathcal{D}_0 \to \mathcal{D}'_0 \quad \text{and} \quad g : \mathcal{D}_1 \to \mathcal{D}'_1.$$

Prove there are approximable mappings

$$f \times g : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}'_0 \times \mathcal{D}'_1 \quad \text{and} \quad f + g : \mathcal{D}_0 + \mathcal{D}_1 \to \mathcal{D}'_0 + \mathcal{D}'_1$$

such that

(i) $(f \times g)(x, y) = \langle f(x), g(y) \rangle$ for all $x \in |\mathcal{D}_0|$ and $y \in |\mathcal{D}_1|$, and rewrite this as:

(ii) $f \times g = \langle f \circ p_0, g \circ p_1 \rangle$.

In addition prove that

(iii) $out_0 \circ (f + g) \circ in_0 = f$, and

(iv) $out_1 \circ (f + g) \circ in_1 = g$.

Do equations (iii) and (iv) uniquely determine $f + g$?

The product and sum of maps are with the induced maps peer.

Theorem (`example_319_sum`). The left and right projections after the sum of maps after the corresponding injections recover the factor maps.

Proof. Invoke the corresponding composition laws. ■

Theorem (`example_319`). The product of maps acts componentwise on pairs and on product elements.

Proof. Invoke the corresponding elementwise product-map identities. ■

Exercise_3_19.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.inducedMaps

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 3.19 (iii)-(iv): projections recover the summands after sum-map.
    -/
theorem example_319_sum :
    (∀ {α β α' β' : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
      {h₀' : ∀ X, V₀'.mem X → X.Nonempty} {h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty}
      (f : ApproximableMap V₀ V₀') (g : ApproximableMap V₁ V₁'),
      (outMap₀ (h₀ := h₀') (h₁ := h₁')).comp
        ((sumMap (h₀ := h₀) (h₁ := h₁) (h₀' := h₀') (h₁' := h₁') f g).comp
          (inMap₀ (h₀ := h₀) (h₁ := h₁))) = f) ∧
    (∀ {α β α' β' : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
      {h₀' : ∀ X, V₀'.mem X → X.Nonempty} {h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty}
      (f : ApproximableMap V₀ V₀') (g : ApproximableMap V₁ V₁'),
      (outMap₁ (h₀ := h₀') (h₁ := h₁')).comp
        ((sumMap (h₀ := h₀) (h₁ := h₁) (h₀' := h₀') (h₁' := h₁') f g).comp
          (inMap₁ (h₀ := h₀) (h₁ := h₁))) = g) :=
  ⟨@outMap₀_comp_sumMap_comp_inMap₀, @outMap₁_comp_sumMap_comp_inMap₁⟩

/-- Scott's Exercise 3.19 (i)-(ii): product of maps acts componentwise. -/
theorem example_319 :
    (∀ {α β α' β' : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
      (f : ApproximableMap V₀ V₀') (g : ApproximableMap V₁ V₁')
      (x : V₀.Element) (y : V₁.Element),
      (prodMap f g).toElementMap (pair x y) = pair (f.toElementMap x) (g.toElementMap
    y)) ∧
    (∀ {α β α' β' : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
```

```
    {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
    (f : ApproximableMap V₀ V₀') (g : ApproximableMap V₁ V₁') (w : (prod V₀ V₁).
  Element),
    (prodMap f g).toElementMap w =
      pair (f.toElementMap w.fst) (g.toElementMap w.snd)) :=
⟨@toElementMap_prodMap_pair, @toElementMap_prodMap⟩
```

4.4.37 Product functor

Functoriality of the product of approximable maps, and uniqueness of the categorical pairing.

Theorem (`paired_unique`). A map into a product that recovers given factors after the two projections equals the pairing of those factors.

Proof. Rewrite via the projection–pairing round trip. ■

Theorem (`proj_paired`). The projections after a pairing recover the paired factors.

Proof. Invoke the projection laws for pairing. ■

Theorem (`prodMap_comp`). The product of maps preserves composition in each factor.

Proof. Compare elementwise via the product-map evaluation. ■

Theorem (`prodMap_id`). The product of identity maps is the identity on the product.

Proof. Compare elementwise via the product-map evaluation and the pairing of components. ■

`prodFunctor.lean`

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.inducedMaps
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {α β ? α' β' : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}
variable {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}

theorem paired_unique (h₀ : ApproximableMap V₂ V₀) (h₁ : ApproximableMap V₂ V₁)
    (k : ApproximableMap V₂ (prod V₀ V₁)) (hk₀ : (proj₀ V₀ V₁).comp k = h₀)
    (hk₁ : (proj₁ V₀ V₁).comp k = h₁) : k = paired h₀ h₁ := by
  rw [← hk₀, ← hk₁, paired_proj]

theorem proj_paired (h₀ : ApproximableMap V₂ V₀) (h₁ : ApproximableMap V₂ V₁) :
    (proj₀ V₀ V₁).comp (paired h₀ h₁) = h₀ ∧ (proj₁ V₀ V₁).comp (paired h₀ h₁) = h₁ :=
  ⟨proj₀_comp_paired h₀ h₁, proj₁_comp_paired h₀ h₁⟩

theorem prodMap_comp {α'' β'' : Type*} {V₀'' : NeighborhoodSystem α''} {V₁'' :
    NeighborhoodSystem β''}
    (f' : ApproximableMap V₀' V₀'') (f : ApproximableMap V₀ V₀')
    (g' : ApproximableMap V₁' V₁'') (g : ApproximableMap V₁ V₁') :
    prodMap (f'.comp f) (g'.comp g) = (prodMap f' g').comp (prodMap f g) := by
  apply ext_of_toElementMap
  intro w
```

```
  rw [toElementMap_prodMap, toElementMap_comp, toElementMap_comp, toElementMap_comp,
    toElementMap_prodMap, toElementMap_prodMap, fst_pair, snd_pair]

theorem prodMap_id : prodMap (idMap V₀) (idMap V₁) = idMap (prod V₀ V₁) := by
  apply ext_of_toElementMap
  intro w
  rw [toElementMap_prodMap, toElementMap_idMap, toElementMap_idMap, toElementMap_idMap
    ,
    pair_fst_snd]
end
end DomainTheory.NeighbourhoodSystems
end
```

4.4.38 Exercise 3.20 — sum and product functors

(For category theorists). Show that the result of 3.19 can be used to prove that $+$ and $\times$ on the category of domains and approximable maps are indeed functors. Show further that $\times$ is the categorical product for this category.

The product functoriality and pairing uniqueness are with the product functor peer.

Theorem (`example_320`). The product of maps preserves composition and identities, and a map into a product is uniquely determined by its projections.

Proof. Invoke the composition law, the identity law, and the pairing uniqueness law. ■

`Exercise_3_20.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodFunctor

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 3.20: product of maps is functorial and is the categorical
    product. -/
theorem example_320 :
    (∀ {α β α' β' : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
      {α'' β'' : Type*} {V₀'' : NeighborhoodSystem α''} {V₁'' : NeighborhoodSystem β''}
      (f' : ApproximableMap V₀' V₀'') (f : ApproximableMap V₀ V₀')
      (g' : ApproximableMap V₁' V₁'') (g : ApproximableMap V₁ V₁'),
      prodMap (f'.comp f) (g'.comp g) = (prodMap f' g').comp (prodMap f g)) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β},
      prodMap (idMap V₀) (idMap V₁) = idMap (prod V₀ V₁)) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (h₀ : ApproximableMap V₂ V₀) (h₁ : ApproximableMap V₂
    V₁)
      (k : ApproximableMap V₂ (prod V₀ V₁)),
      (proj₀ V₀ V₁).comp k = h₀ → (proj₁ V₀ V₁).comp k = h₁ → k = paired h₀ h₁) :=
  ⟨@prodMap_comp, @prodMap_id,
   fun {_} {_} {_} {_} {_} {_} h₀ h₁ k => paired_unique h₀ h₁ k⟩
```

4.4.39 Identity of step neighbourhoods

Criterion for equality of step neighbourhoods, via the least map of a singleton step.

Theorem (`interYs_single_not_subset_eq`). If a domain neighbourhood is not contained in the left factor of a singleton step list, the associated intersection of range neighbourhoods is the master.

Proof. The defining constraints are vacuous off that factor. ■

Theorem (`interYs_single_subset_eq`). If a domain neighbourhood is contained in the left factor of a singleton step list, the associated intersection of range neighbourhoods recovers the right factor.

Proof. Unpack the singleton constraint. ■

Theorem (`stepCons`). The intersection of range neighbourhoods for a singleton step list is always a neighbourhood.

Proof. Case on containment in the left factor and apply the previous identities. ■

Definition (`leastStep`). The least map of a singleton step neighbourhood is the least map of the corresponding singleton list.

Proof. By definition, using the consistency law just proved. ■

Theorem (`leastStep_le`). The least map of a step neighbourhood is below every member.

Proof. Specialise the least-map comparison at a singleton. ■

Theorem (`leastStep_mem`). The least map of a step neighbourhood belongs to that neighbourhood.

Proof. Specialise membership of the least map in a step list at a singleton. ■

Theorem (`leastStep_eq_of_step_eq`). Equal step neighbourhoods have equal least maps.

Proof. Compare both ways via membership of each least map in the other neighbourhood. ■

Theorem (`leastStep_rel`). The least map of a step relates neighbourhoods precisely when the associated intersection of range neighbourhoods is contained in the codomain neighbourhood.

Proof. Unpack the least-map relation at a singleton. ■

Theorem (`step_master_right`). A step neighbourhood with master codomain is the universal set of maps.

Proof. Every map relates the domain neighbourhood to the master by monotonicity of the master law. ■

Theorem (`step_eq_univ_iff`). A step neighbourhood is universal precisely when the codomain neighbourhood is the master.

Proof. Forward, the empty least map forces the master below the codomain; converse is the previous identity. ■

Theorem (`step_eq_of_ne_master`). Distinct from the master on the right, equal step neighbourhoods have equal factors.

Proof. Compare least maps and recover the factors from the least-map relation. ■

Theorem (`step_eq_iff`). Two step neighbourhoods agree precisely when both have master codomain, or the factors agree.

Proof. Combine the universal characterisation with the non-master uniqueness. ■

`stepEq.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
```

```
open NeighborhoodSystem ApproximableMap
section
variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

theorem interYs_single_not_subset_eq {YX : Set α} {Z : Set β} {X : Set α}
    (hXY : ¬ X ⊆ YX) : interYs V₁.master [(YX, Z)] X = V₁.master := by
  ext z
  rw [mem_interYs]
  constructor
  · rintro ⟨hm, _⟩; exact hm
  · intro hz
    exact ⟨hz, fun p hp hXp => by rw [List.mem_singleton] at hp; subst hp; exact absurd
     hXp hXY⟩

theorem interYs_single_subset_eq {YX : Set α} {Z : Set β} {X : Set α} (hZ : V₁.mem Z)
    (hXY : X ⊆ YX) : interYs V₁.master [(YX, Z)] X = Z := by
  ext z
  rw [mem_interYs]
  constructor
  · rintro ⟨_, hall⟩; exact hall (YX, Z) (List.mem_singleton.mpr rfl) hXY
  · intro hz
    exact ⟨V₁.sub_master hZ hz, fun p hp _ => by rw [List.mem_singleton] at hp; subst
    hp; exact hz⟩

theorem stepCons {Y : Set α} {Z : Set β} (hZ : V₁.mem Z) :
    ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master [(Y, Z)] X) := by
  intro X _
  by_cases hXY : X ⊆ Y
  · rw [interYs_single_subset_eq hZ hXY]; exact hZ
  · rw [interYs_single_not_subset_eq hXY]; exact V₁.master_mem

def leastStep (Y : Set α) (Z : Set β) (hY : V₀.mem Y) (hZ : V₁.mem Z) :
    ApproximableMap V₀ V₁ :=
  leastMap [(Y, Z)] (by rintro p hp; rw [List.mem_singleton] at hp; subst hp; exact ⟨hY
    , hZ⟩)
    (stepCons hZ)

theorem leastStep_le {Y : Set α} {Z : Set β} (hY : V₀.mem Y) (hZ : V₁.mem Z)
    {f : ApproximableMap V₀ V₁} (hf : f ∈ step Y Z) : leastStep Y Z hY hZ ≤ f := by
  apply leastMap_le
  rw [stepFun_singleton]; exact hf

theorem leastStep_mem {Y : Set α} {Z : Set β} (hY : V₀.mem Y) (hZ : V₁.mem Z) :
    leastStep Y Z hY hZ ∈ step Y Z := by
  have := leastMap_mem_stepFun (V₀ := V₀) (V₁ := V₁)
    (L := [(Y, Z)]) (by rintro p hp; rw [List.mem_singleton] at hp; subst hp; exact ⟨hY
    , hZ⟩)
    (stepCons hZ)
  rw [stepFun_singleton] at this
  exact this

theorem leastStep_eq_of_step_eq {Y : Set α} {Z : Set β} (hY : V₀.mem Y) (hZ : V₁.mem Z)
    {Y' : Set α} {Z' : Set β} (hY' : V₀.mem Y') (hZ' : V₁.mem Z')
    (h : (step Y Z : Set (ApproximableMap V₀ V₁)) = step Y' Z') :
    leastStep Y Z hY hZ = leastStep Y' Z' hY' hZ' := by
  apply le_antisymm
```

```
  · exact leastStep_le hY hZ (h ▶ leastStep_mem hY' hZ')
  · exact leastStep_le hY' hZ' (h.symm ▶ leastStep_mem hY hZ)

theorem leastStep_rel {Y : Set α} {Z : Set β} (hY : V₀.mem Y) (hZ : V₁.mem Z)
    {Y' : Set α} {Z' : Set β} (hY' : V₀.mem Y') (hZ' : V₁.mem Z') :
    (leastStep Y Z hY hZ).rel Y' Z' ↔ interYs V₁.master [(Y, Z)] Y' ⊆ Z' := by
  rw [leastStep, leastMap_rel]
  exact ⟨fun h => h.2.2, fun h => ⟨hY', hZ', h⟩⟩

theorem step_master_right (Y : Set α) (hY : V₀.mem Y) :
    (step Y V₁.master : Set (ApproximableMap V₀ V₁)) = Set.univ := by
  ext f
  simp only [mem_step, Set.mem_univ, iff_true]
  exact f.mono f.master_rel (V₀.sub_master hY) subset_rfl hY V₁.master_mem

theorem step_eq_univ_iff {Y : Set α} {Z : Set β} (hY : V₀.mem Y) (hZ : V₁.mem Z) :
    (step Y Z : Set (ApproximableMap V₀ V₁)) = Set.univ ↔ Z = V₁.master := by
  constructor
  · intro h

    have hmem : leastMap (V₀ := V₀) (V₁ := V₁) ([] : List (Set α × Set β))
        (by simp) (fun {X} _ => by rw [interYs_nil]; exact V₁.master_mem) ∈ step Y Z :=
     by
      rw [h]; exact Set.mem_univ _
    rw [mem_step, leastMap_rel] at hmem
    have : V₁.master ⊆ Z := by rw [← interYs_nil V₁.master Y]; exact hmem.2.2
    exact Set.Subset.antisymm (V₁.sub_master hZ) this
  · rintro rfl; exact step_master_right Y hY

theorem step_eq_of_ne_master {Y : Set α} {Z : Set β} (hY : V₀.mem Y) (hZ : V₁.mem Z)
    {Y' : Set α} {Z' : Set β} (hY' : V₀.mem Y') (hZ' : V₁.mem Z') (hZne : Z = V₁.master
    )
    (h : (step Y Z : Set (ApproximableMap V₀ V₁)) = step Y' Z') : Y = Y' ∧ Z = Z' := by

  have hZ'ne : Z' = V₁.master := by
    intro hZ'eq
    rw [hZ'eq, step_master_right Y' hY'] at h
    exact hZne ((step_eq_univ_iff hY hZ).mp h)
  have hfeq := leastStep_eq_of_step_eq hY hZ hY' hZ' h

  have h1 : (leastStep Y Z hY hZ).rel Y' Z' := by
    rw [hfeq]; exact (leastStep_mem hY' hZ')
  have h2 : (leastStep Y' Z' hY' hZ').rel Y Z := by
    rw [← hfeq]; exact (leastStep_mem hY hZ)
  rw [leastStep_rel hY hZ hY' hZ'] at h1
  rw [leastStep_rel hY' hZ' hY hZ] at h2

  have hY'Y : Y' ⊆ Y := by
    by_contra hns
    rw [interYs_single_not_subset_eq hns] at h1
    exact hZ'ne (Set.Subset.antisymm (V₁.sub_master hZ') h1)
  have hZZ' : Z ⊆ Z' := by
    rw [interYs_single_subset_eq hZ hY'Y] at h1; exact h1

  have hYY' : Y ⊆ Y' := by
    by_contra hns
```

```
    rw [interYs_single_not_subset_eq hns] at h2
    exact hZne (Set.Subset.antisymm (V₁.sub_master hZ) h2)
  have hZ'Z : Z' ⊆ Z := by
    rw [interYs_single_subset_eq hZ' hYY'] at h2; exact h2
  exact ⟨Set.Subset.antisymm hYY' hY'Y, Set.Subset.antisymm hZZ' hZ'Z⟩

theorem step_eq_iff {Y : Set α} {Z : Set β} (hY : V₀.mem Y) (hZ : V₁.mem Z)
    {Y' : Set α} {Z' : Set β} (hY' : V₀.mem Y') (hZ' : V₁.mem Z') :
    (step Y Z : Set (ApproximableMap V₀ V₁)) = step Y' Z' ↔
      (Z = V₁.master ∧ Z' = V₁.master) ∨ (Y = Y' ∧ Z = Z') := by
  constructor
  · intro h
    by_cases hZeq : Z = V₁.master
    · refine Or.inl ⟨hZeq, ?_⟩
      rw [hZeq, step_master_right Y hY] at h
      exact (step_eq_univ_iff hY' hZ').mp h.symm
    · exact Or.inr (step_eq_of_ne_master hY hZ hY' hZ' hZeq h)
  · rintro (⟨rfl, rfl⟩ | ⟨rfl, rfl⟩)
    · rw [step_master_right Y hY, step_master_right Y' hY']
    · rfl
end

end
end DomainTheory.NeighbourhoodSystems
end
```

4.4.40 Exercise 3.21 — identity of step neighbourhoods

In the proofs of 3.12 in the definition of curry (g) it is rather cavalierly assumed that the neighbourhood $[Y, Z]$ uniquely determines Y and Z. Show that this is true if $Z \neq \Delta_2$. (Hint: Find explicitly the least of $f \in [Y, Z]$.) Show that if $Z = \Delta_2$ the biconditional stated at the start of the proof is still valid even though Y is not uniquely determined. (Hint: Remember that $\Delta_1\ g\ \Delta_2$ must hold.) For arbitrary pairs of neighbourhoods of $(D_1 \to D_2)$ is there a simple criterion for identity?

The step-neighbourhood criterion is with the identity of step neighbourhoods peer.

Theorem (`example_321`). Two step neighbourhoods agree precisely when both have master codomain or the factors agree; in particular, off the master the factors are unique, and a master codomain yields the universal neighbourhood of maps.

Proof. Invoke the equality criterion, the non-master uniqueness, and the master-codomain identity. ■

`Exercise_3_21.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.stepEq

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 3.21: criterion for identity of step neighbourhoods. -/
theorem example_321 :
```

```
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {Y : Set α} {Z : Set β} (_hY : V₀.mem Y) (_hZ : V₁.mem Z)
      {Y' : Set α} {Z' : Set β} (_hY' : V₀.mem Y') (_hZ' : V₁.mem Z'),
      (step Y Z : Set (ApproximableMap V₀ V₁)) = step Y' Z' ↔
        (Z = V₁.master ∧ Z' = V₁.master) ∨ (Y = Y' ∧ Z = Z')) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {Y : Set α} {Z : Set β} (_hY : V₀.mem Y) (_hZ : V₁.mem Z)
      {Y' : Set α} {Z' : Set β} (_hY' : V₀.mem Y') (_hZ' : V₁.mem Z'),
      Z = V₁.master →
        (step Y Z : Set (ApproximableMap V₀ V₁)) = step Y' Z' → Y = Y' ∧ Z = Z') ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (Y : Set α) (_hY : V₀.mem Y),
      (step Y V₁.master : Set (ApproximableMap V₀ V₁)) = Set.univ) :=
  ⟨@step_eq_iff, @step_eq_of_ne_master, @step_master_right⟩
```

4.4.41 Composition combinator

The composition combinator on function spaces, built from evaluation and currying.

Definition (`compApp`). Composition applied to an argument is evaluation of the outer map at the evaluation of the inner map, organised by projections and pairing.

Proof. Assemble evaluation and pairing along the indicated product projections. ■

Definition (`compMap`). The composition combinator is the curry of composition-at-an-argument.

Proof. By definition. ■

Theorem (`toElementMap_compApp`). Elementwise, composition-at-an-argument evaluates the outer map at the value of the inner map.

Proof. Unpack the defining composition of evaluations and projections. ■

Theorem (`toElementMap_compMap_apply`). Elementwise, the composition combinator acts by ordinary composition of the corresponding maps at each argument.

Proof. Apply the curry evaluation identity and the previous identity. ■

Theorem (`toApproxMap_compMap`). The approximable map of a composed pair is the composition of the approximable maps.

Proof. Compare elementwise via the previous identity. ■

`compMap.lean`

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
section
variable {α β ? : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}

def compApp (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
    NeighborhoodSystem ?) :
```

```
    ApproximableMap (prod (prod (funSpace V₁ V₂) (funSpace V₀ V₁)) V₀) V₂ :=
  (evalMap V₁ V₂).comp
    (paired
      ((proj₀ (funSpace V₁ V₂) (funSpace V₀ V₁)).comp
        (proj₀ (prod (funSpace V₁ V₂) (funSpace V₀ V₁)) V₀))
      ((evalMap V₀ V₁).comp
        (paired
          ((proj₁ (funSpace V₁ V₂) (funSpace V₀ V₁)).comp
            (proj₀ (prod (funSpace V₁ V₂) (funSpace V₀ V₁)) V₀))
          (proj₁ (prod (funSpace V₁ V₂) (funSpace V₀ V₁)) V₀))))

def compMap (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
    NeighborhoodSystem ?) :
    ApproximableMap (prod (funSpace V₁ V₂) (funSpace V₀ V₁)) (funSpace V₀ V₂) :=
  curry (compApp V₀ V₁ V₂)

theorem toElementMap_compApp (Gφ : (funSpace V₁ V₂).Element) (Fφ : (funSpace V₀ V₁).
    Element)
    (x : V₀.Element) :
    (compApp V₀ V₁ V₂).toElementMap (pair (pair Gφ Fφ) x)
      = (toApproxMap Gφ).toElementMap ((toApproxMap Fφ).toElementMap x) := by
  rw [compApp]
  simp only [toElementMap_comp, toElementMap_paired, toElementMap_proj₀,
    toElementMap_proj₁,
    fst_pair, snd_pair, evalMap_apply]

theorem toElementMap_compMap_apply (Gφ : (funSpace V₁ V₂).Element) (Fφ : (funSpace V₀ V
    ₁).Element)
    (x : V₀.Element) :
    (toApproxMap ((compMap V₀ V₁ V₂).toElementMap (pair Gφ Fφ))).toElementMap x
      = (toApproxMap Gφ).toElementMap ((toApproxMap Fφ).toElementMap x) := by
  rw [compMap, toElementMap_curry_apply, toElementMap_compApp]

theorem toApproxMap_compMap (Gφ : (funSpace V₁ V₂).Element) (Fφ : (funSpace V₀ V₁).
    Element) :
    toApproxMap ((compMap V₀ V₁ V₂).toElementMap (pair Gφ Fφ))
      = (toApproxMap Gφ).comp (toApproxMap Fφ) := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_compMap_apply, toElementMap_comp]
end

end
end DomainTheory.NeighbourhoodSystems
end
```

4.4.42 Exercise 3.22 — composition combinator

Prove that there is an approximable mapping

$$\text{comp} : (D_1 \to D_2) \times (D_0 \to D_1) \to (D_0 \to D_2)$$

where for all $g : D_1 \to D_2$ and $f : D_0 \to D_1$ we have

$$\mathrm{comp}(g, f) = g \circ f.$$

Show this directly by writing down the neighbourhood relation and by building the mapping up from eval and curry (on suitable domains) using $\circ$ and $\langle , \rangle$. (Hint: Fill in maps in the following sequence of domains:

1. $(D_0 \to D_1) \times D_0 \to D_1$
2. $(D_1 \to D_2) \times ((D_0 \to D_1) \times D_0) \to (D_1 \to D_2) \times D_1$
3. $((D_1 \to D_2) \times (D_0 \to D_1)) \times D_0 \to (D_1 \to D_2) \times D_1$
4. $((D_1 \to D_2) \times (D_0 \to D_1)) \times D_0 \to D_2$
5. $(D_1 \to D_2) \times (D_0 \to D_1) \to (D_0 \to D_2)$.

The maps are of course not uniquely determined, but the shifting of brackets ought to suggest the right choice.)

The composition combinator is with the composition combinator peer.

Theorem (`example_322`). The composition combinator acts by ordinary composition of the corresponding approximable maps, and agrees elementwise with that composition at each argument.

Proof. Invoke the corresponding composition identities. ■

`Exercise_3_22.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.compMap

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 3.22: the composition combinator acts by ordinary composition. -/
theorem example_322 :
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (Gφ : (funSpace V₁ V₂).Element) (Fφ : (funSpace V₀ V₁
    ).Element),
      toApproxMap ((compMap V₀ V₁ V₂).toElementMap (pair Gφ Fφ))
        = (toApproxMap Gφ).comp (toApproxMap Fφ)) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (Gφ : (funSpace V₁ V₂).Element) (Fφ : (funSpace V₀ V₁
    ).Element)
      (x : V₀.Element),
      (toApproxMap ((compMap V₀ V₁ V₂).toElementMap (pair Gφ Fφ))).toElementMap x
        = (toApproxMap Gφ).toElementMap ((toApproxMap Fφ).toElementMap x)) :=
  ⟨@toApproxMap_compMap, @toElementMap_compMap_apply⟩
```

4.4.43 Cartesian closed category

Cartesian closed structure: the curry adjunction and the terminal unit system.

Instance (`Subsingleton (ApproximableMap V unitSys)`). There is at most one approximable map into the unit system.

Proof. Compare elementwise using uniqueness of the unit element. ■

Instance (`Unique (ApproximableMap V unitSys)`). There is a unique approximable map into the unit system, namely the constant at the unique unit element.

Proof. Take the constant map as default and invoke the subsingleton law. ■

Definition (`homAdjunction`). Maps out of a product are order-isomorphic to maps into the function space via curry.

Proof. Invoke the curry equivalence. ■

Definition (`toUnit`). The unique map into the unit system is the default of that unique type.

Proof. By definition. ■

Theorem (`toUnit_unique`). Every map into the unit system equals the unique map.

Proof. Invoke the subsingleton law. ■

`cartClosed.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodIso
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
section
variable {α β ? : Type*} (V : NeighborhoodSystem α)
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}

instance : Subsingleton (ApproximableMap V unitSys) :=
  ⟨fun _ _ => ext_of_toElementMap fun _ => Subsingleton.elim _ _⟩

instance : Unique (ApproximableMap V unitSys) where
  default := constMap V (default : unitSys.Element)
  uniq _ := Subsingleton.elim _ _

def homAdjunction (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (V₂ : NeighborhoodSystem ?) :
    ApproximableMap (prod V₀ V₁) V₂ ≃o ApproximableMap V₀ (funSpace V₁ V₂) :=
  curryEquiv V₀ V₁ V₂

def toUnit : ApproximableMap V unitSys := default

theorem toUnit_unique (f : ApproximableMap V unitSys) : f = toUnit V := Subsingleton.
    elim _ _
end
```

```
end

end DomainTheory.NeighbourhoodSystems
end
```

4.4.44 Exercise 3.23 — cartesian closed category

(For category theorists.) Show that the results of 3.11 and 3.12 prove that the category of domains and approximable mappings is a *cartesian closed category*. ([142, pp. 95–96] may be consulted for a very brief introduction.) What is the *terminal domain* in this category? What sort of functor is $(D_0 \to D_1)$?

The terminal map and curry adjunction are with the cartesian closed category peer.

Theorem (`example_323`). Every approximable map into the unit system equals the unique map to the unit.

Proof. Invoke uniqueness of the map to the unit. ■

`Exercise_3_23.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.cartClosed

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 3.23: the unit system is terminal. -/
theorem example_323 :
    ∀ {α : Type*} (V : NeighborhoodSystem α) (f : ApproximableMap V unitSys),
      f = toUnit V :=
  @toUnit_unique
```

4.4.45 Function-space isomorphisms

Function-space isomorphisms for products, infinite sequences, and sums, including distribution of products over sums.

Definition (`funProdEquiv`). Maps into a product are order-isomorphic to pairs of maps into the factors.

Proof. Pair and project via the pairing–projection laws; order equivalence is componentwise. ■

Definition (`funProdIso`). Function-space elements into a product are order-isomorphic to the product of the function spaces.

Proof. Transport the previous equivalence through the function-space element isomorphisms and the product-element isomorphism. ■

Theorem (`funProd_isomorphic`). The function space into a product is isomorphic to the product of the function spaces.

Proof. Invoke the corresponding element order isomorphism. ■

Theorem (`exists_forall_swap`). A sequence neighbourhood belongs fibrewise to the values of a family of maps at an argument if and only if the assembled sequence of values belongs to that neighbourhood.

Proof. Compare fibre membership with membership of indexed pairs. ■

Theorem (`le_of_projComp_le`). Pointwise approximation of projections after maps into the sequence system yields approximation of the maps.

Proof. Compare via the sequence reconstruction from components. ■

Definition (`mapOfSeq`). A sequence of maps into a base determines a map into the sequence system by requiring fibrewise membership of the values.

Proof. Filter axioms follow fibrewise from the terms. ■

Definition (`piCongrOrderIso`). A family of order isomorphisms induces an order isomorphism of dependent products.

Proof. Apply componentwise; order equivalence is componentwise. ■

Theorem (`projComp_mono`). Approximation of maps into the sequence system yields approximation after each projection.

Proof. Apply monotonicity of the projection. ■

Theorem (`toElementMap_mapOfSeq`). Elementwise, a sequence of maps sends an argument to the sequence of its values.

Proof. Compare membership via fibres. ■

Definition (`funIterEquiv`). Maps into the sequence system are order-isomorphic to sequences of maps into the base.

Proof. The round trips are the sequence of projections and the map of a sequence; order equivalence uses the projection comparison. ■

Definition (`funIterIso`). Function-space elements into the sequence system are order-isomorphic to the sequence system on the function space.

Proof. Transport the previous equivalence through the function-space and sequence-element isomorphisms. ■

Theorem (`funIter_isomorphic`). The function space into the sequence system is isomorphic to the sequence system on the function space.

Proof. Invoke the corresponding element order isomorphism. ■

Definition (`copair`). The copairing of two maps into a common codomain relates a sum neighbourhood by routing left and right injections through the corresponding maps.

Proof. Domain and codomain membership, master, intersection on the right, and monotonicity follow by cases on sum neighbourhoods. ■

Definition (`copairProj`). The pair of projections of a map out of a sum is the pair of that map after the two injections.

Proof. By definition. ■

Theorem (`prod_mem_nonempty`). Under nonempty-neighbourhood hypotheses on the factors, every product neighbourhood is nonempty.

Proof. Pick witnesses in each projected neighbourhood. ■

Definition (`distribMap`). Distribution of a product over a sum pairs the first projection with the left injection after the second projection.

Proof. Assemble pairing and composition of the product projections with the left injection. ■

Theorem (`copair_comp_inMap0`). The copair after the left injection recovers the left summand map.

Proof. Compare relations via left injections. ■

Theorem (`copair_comp_inMap1`). The copair after the right injection recovers the right summand map.

Proof. Compare relations via right injections. ■

Theorem (`copairProj_copair`). Projecting a copair recovers the original pair of maps.

Proof. Apply the two injection laws. ■

funSpaceIso.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_10
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.
    ApproximableMap2
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.iterSys
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.sumSys
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.inducedMaps
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_5a
@[expose] public section

section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

variable {α β ? : Type*}

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}

def funProdEquiv (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (V₂ : NeighborhoodSystem ?) :
    ApproximableMap V₀ (prod V₁ V₂) ≃o (ApproximableMap V₀ V₁ × ApproximableMap V₀ V₂)
    where
  toFun h := ((proj₀ V₁ V₂).comp h, (proj₁ V₁ V₂).comp h)
  invFun p := paired p.1 p.2
  left_inv h := paired_proj h
  right_inv p := by
    ext1
    · exact proj₀_comp_paired p.1 p.2
    · exact proj₁_comp_paired p.1 p.2
  map_rel_iff' := by
    intro h h'
    constructor
    · rintro ⟨h0, h1⟩
      rw [← paired_proj h, ← paired_proj h']
      intro Z P hrel
      obtain ⟨hP, hfP, hgP⟩ := hrel
      exact ⟨hP, h0 _ _ hfP, h1 _ _ hgP⟩
    · intro hle
```

```
      refine ⟨fun X Y hrel => ?_, fun X Y hrel => ?_⟩
      · obtain ⟨W, hW, hYW⟩ := hrel
        exact ⟨W, hle X W hW, hYW⟩
      · obtain ⟨W, hW, hYW⟩ := hrel
        exact ⟨W, hle X W hW, hYW⟩

end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
section

open NeighborhoodSystem ApproximableMap

section
variable {α β ? : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}

def funProdIso (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (V₂ : NeighborhoodSystem ?) :
    (funSpace V₀ (prod V₁ V₂)).Element ≃o (prod (funSpace V₀ V₁) (funSpace V₀ V₂)).
    Element :=
  (funSpaceEquiv V₀ (prod V₁ V₂)).trans <|
    (funProdEquiv V₀ V₁ V₂).trans <|
      (prodCongrOrderIso (funSpaceEquiv V₀ V₁).symm (funSpaceEquiv V₀ V₂).symm).trans
        (prodEquiv (funSpace V₀ V₁) (funSpace V₀ V₂)).symm

theorem funProd_isomorphic :
    funSpace V₀ (prod V₁ V₂) ≅? prod (funSpace V₀ V₁) (funSpace V₀ V₂) :=
  ⟨funProdIso V₀ V₁ V₂⟩
end

section
variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

theorem exists_forall_swap {x : V₀.Element} {W : Set (ℕ × β)}
    {g : ℕ → ApproximableMap V₀ V₁} (hW : (iterSys V₁).mem W) :
    (∃ X, x.mem X ∧ ∀ i, (g i).rel X (fiber W i)) ↔
      (∀ i, ∃ X, x.mem X ∧ (g i).rel X (fiber W i)) := by
  constructor
  · rintro ⟨X, hXx, hall⟩ i; exact ⟨X, hXx, hall i⟩
  · intro hall
    obtain ⟨N, hN⟩ := hW.2
    choose Xs hXx hXrel using hall
    have hbase : x.mem (V₀.interUpTo Xs N) := x.mem_interUpTo Xs (n := N) (fun i _ =>
    hXx i)
    refine ⟨V₀.interUpTo Xs N, hbase, fun i => ?_⟩
    by_cases h : i < N
    · exact (g i).mono (hXrel i) (interUpTo.subset V₀ Xs h) subset_rfl (x.sub hbase)
        ((g i).rel_cod (hXrel i))
    · rw [hN i (not_lt.mp h)]; exact (g i).rel_master (x.sub hbase)

theorem le_of_projComp_le {f f' : ApproximableMap V₀ (iterSys V₁)}
    (h : ∀ n, (projN V₁ n).comp f ≤ (projN V₁ n).comp f') : f ≤ f' := by
```

```
  rw [le_iff_toElementMap_le]
  intro x
  apply le_of_component_le
  intro n
  have hx := (le_iff_toElementMap_le.mp (h n)) x
  rwa [toElementMap_comp, toElementMap_comp, toElementMap_projN, toElementMap_projN]
    at hx

def mapOfSeq (g : ℕ → ApproximableMap V₀ V₁) : ApproximableMap V₀ (iterSys V₁) where
  rel X W := V₀.mem X ∧ (iterSys V₁).mem W ∧ ∀ i, (g i).rel X (fiber W i)
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨V₀.master_mem, (iterSys V₁).master_mem, fun i => by
    rw [fiber_iterSys_master]; exact (g i).master_rel⟩
  inter_right := by
    rintro X W W' ⟨hX, hW, hrel⟩ ⟨_, hW', hrel'⟩
    refine ⟨hX, ⟨fun i => (g i).rel_cod ((g i).inter_right (hrel i) (hrel' i)), ?_⟩,
      fun i => (g i).inter_right (hrel i) (hrel' i)⟩
    obtain ⟨NW, hNW⟩ := hW.2
    obtain ⟨NW', hNW'⟩ := hW'.2
    refine ⟨max NW NW', fun i hi => ?_⟩
    rw [fiber_inter, hNW i (le_trans (le_max_left _ _) hi),
      hNW' i (le_trans (le_max_right _ _) hi), Set.inter_self]
  mono := by
    rintro X X' W W' ⟨_, _, hrel⟩ hX'X hWW' hX' hW'
    exact ⟨hX', hW', fun i => (g i).mono (hrel i) hX'X (fiber_mono hWW' i) hX' (hW'.1 i
    )⟩

def piCongrOrderIso {ι : Type*} {A B : ι → Type*} [∀ i, Preorder (A i)] [∀ i, Preorder
    (B i)]
    (e : ∀ i, A i ≃o B i) : (∀ i, A i) ≃o (∀ i, B i) where
  toFun f := fun i => e i (f i)
  invFun g := fun i => (e i).symm (g i)
  left_inv f := by funext i; simp
  right_inv g := by funext i; simp
  map_rel_iff' := by
    intro f g
    exact ⟨fun h i => (e i).le_iff_le.mp (h i), fun h i => (e i).le_iff_le.mpr (h i)⟩

theorem projComp_mono {f f' : ApproximableMap V₀ (iterSys V₁)} (h : f ≤ f') (n : ℕ) :
    (projN V₁ n).comp f ≤ (projN V₁ n).comp f' := by
  rintro X Y ⟨W, hfXW, hWY⟩
  exact ⟨W, h X W hfXW, hWY⟩

theorem toElementMap_mapOfSeq (g : ℕ → ApproximableMap V₀ V₁) (x : V₀.Element) :
    (mapOfSeq g).toElementMap x = ofSeq (fun i => (g i).toElementMap x) := by
  apply Element.ext
  intro W
  constructor
  · rintro ⟨X, hXx, _, hW, hrel⟩
    exact ⟨hW, fun i => ⟨X, hXx, hrel i⟩⟩
  · rintro ⟨hW, hfib⟩
    obtain ⟨X, hXx, hrel⟩ := (exists_forall_swap hW).mpr (fun i => hfib i)
    exact ⟨X, hXx, x.sub hXx, hW, hrel⟩

def funIterEquiv (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
```

```
    ApproximableMap V₀ (iterSys V₁) ≃o (∀ _ : ℕ, ApproximableMap V₀ V₁) where
  toFun f := fun n => (projN V₁ n).comp f
  invFun g := mapOfSeq g
  left_inv f := by
    apply ext_of_toElementMap
    intro x
    rw [toElementMap_mapOfSeq]
    have hfun : (fun i => ((projN V₁ i).comp f).toElementMap x)
        = (fun i => component (f.toElementMap x) i) := by
      funext i; rw [toElementMap_comp, toElementMap_projN]
    rw [hfun, ofSeq_component]
  right_inv g := by
    funext n
    apply ext_of_toElementMap
    intro x
    rw [toElementMap_comp, toElementMap_mapOfSeq, toElementMap_projN, component_ofSeq]
  map_rel_iff' := by
    intro f f'
    constructor
    · intro h; exact le_of_projComp_le (fun n => h n)
    · intro h n; exact projComp_mono h n

def funIterIso (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    (funSpace V₀ (iterSys V₁)).Element ≃o (iterSys (funSpace V₀ V₁)).Element :=
  (funSpaceEquiv V₀ (iterSys V₁)).trans <|
    (funIterEquiv V₀ V₁).trans <|
      (piCongrOrderIso (fun _ : ℕ => (funSpaceEquiv V₀ V₁).symm)).trans
        (iterSeqEquiv (funSpace V₀ V₁)).symm

theorem funIter_isomorphic : funSpace V₀ (iterSys V₁) ≅? iterSys (funSpace V₀ V₁) :=
  ⟨funIterIso V₀ V₁⟩
end

section
open NeighborhoodSystem ApproximableMap
variable {α β ? : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ :
    NeighborhoodSystem ?}
variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}

def copair (a : ApproximableMap V₀ V₂) (b : ApproximableMap V₁ V₂) :
    ApproximableMap (sum V₀ V₁ h₀ h₁) V₂ where
  rel W Z := (sum V₀ V₁ h₀ h₁).mem W ∧ V₂.mem Z ∧
    (Z = V₂.master ∨
      (∃ X, W = inj₀ X ∧ a.rel X Z) ∨
      (∃ Y, W = inj₁ Y ∧ b.rel Y Z))
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(sum V₀ V₁ h₀ h₁).master_mem, V₂.master_mem, Or.inl rfl⟩
  inter_right := by
    rintro W Z Z' ⟨hW, _, hd⟩ ⟨_, _, hd'⟩
    have hmem : ∀ Z'' : Set ?,
        (Z'' = V₂.master ∨ (∃ X, W = inj₀ X ∧ a.rel X Z'') ∨ (∃ Y, W = inj₁ Y ∧ b.rel Y
    Z'')) →
        V₂.mem Z'' := by
      rintro Z'' (rfl | ⟨_, _, hf⟩ | ⟨_, _, hg⟩)
```

```
      · exact V₂.master_mem
      · exact a.rel_cod hf
      · exact b.rel_cod hg
    have key : Z ∩ Z' = V₂.master ∨ (∃ X, W = inj₀ X ∧ a.rel X (Z ∩ Z')) ∨
        (∃ Y, W = inj₁ Y ∧ b.rel Y (Z ∩ Z')) := by
      rcases hd with rfl | ⟨X, hWX, hf⟩ | ⟨Y, hWY, hg⟩
      · rw [Set.inter_eq_right.mpr (V₂.sub_master (hmem _ hd'))]; exact hd'
      · rcases hd' with rfl | ⟨X', hWX', hf'⟩ | ⟨Y', hWY', hg'⟩
        · rw [Set.inter_eq_left.mpr (V₂.sub_master (a.rel_cod hf))]
          exact Or.inr (Or.inl ⟨X, hWX, hf⟩)
        · obtain rfl : X = X' := inj₀_injective (hWX ▶ hWX')
          exact Or.inr (Or.inl ⟨X, hWX, a.inter_right hf hf'⟩)
        · exact absurd (hWX ▶ hWY' : (inj₀ X : Set _) = inj₁ Y')
            (fun h => not_inj₀_subset_inj₁ (h₀ X (a.rel_dom hf)) h.subset)
      · rcases hd' with rfl | ⟨X', hWX', hf'⟩ | ⟨Y', hWY', hg'⟩
        · rw [Set.inter_eq_left.mpr (V₂.sub_master (b.rel_cod hg))]
          exact Or.inr (Or.inr ⟨Y, hWY, hg⟩)
        · exact absurd (hWY ▶ hWX' : (inj₁ Y : Set _) = inj₀ X')
            (fun h => not_inj₁_subset_inj₀ (h₁ Y (b.rel_dom hg)) h.subset)
        · obtain rfl : Y = Y' := inj₁_injective (hWY ▶ hWY')
          exact Or.inr (Or.inr ⟨Y, hWY, b.inter_right hg hg'⟩)
    exact ⟨hW, hmem _ key, key⟩
  mono := by
    rintro W W₂ Z Z' ⟨_, _, hd⟩ hW₂W hZZ' hW₂ hZ'
    refine ⟨hW₂, hZ', ?_⟩
    rcases hd with rfl | ⟨X, rfl, hf⟩ | ⟨Y, rfl, hg⟩
    · left; exact Set.Subset.antisymm (V₂.sub_master hZ') hZZ'
    · obtain ⟨X₂, hX₂, rfl⟩ := mem_subset_inj₀ hW₂ hW₂W
      exact Or.inr (Or.inl ⟨X₂, rfl, a.mono hf (inj₀_subset_inj₀.mp hW₂W) hZZ' hX₂ hZ'⟩)
    · obtain ⟨Y₂, hY₂, rfl⟩ := mem_subset_inj₁ hW₂ hW₂W
      exact Or.inr (Or.inr ⟨Y₂, rfl, b.mono hg (inj₁_subset_inj₁.mp hW₂W) hZZ' hY₂ hZ'⟩)

@[simp] theorem copair_rel {a : ApproximableMap V₀ V₂} {b : ApproximableMap V₁ V₂}
    {W : Set (Option (α ? β))} {Z : Set ?} :
    (copair (h₀ := h₀) (h₁ := h₁) a b).rel W Z ↔ (sum V₀ V₁ h₀ h₁).mem W ∧ V₂.mem Z ∧
      (Z = V₂.master ∨ (∃ X, W = inj₀ X ∧ a.rel X Z) ∨ (∃ Y, W = inj₁ Y ∧ b.rel Y Z))
    := Iff.rfl

def copairProj (h : ApproximableMap (sum V₀ V₁ h₀ h₁) V₂) :
    ApproximableMap V₀ V₂ × ApproximableMap V₁ V₂ :=
  (h.comp inMap₀, h.comp inMap₁)

theorem prod_mem_nonempty (hn₀ : ∀ X, V₀.mem X → X.Nonempty) (_hn₁ : ∀ Y, V₁.mem Y → Y
    .Nonempty)
    (W : Set (α ? β)) (hW : (prod V₀ V₁).mem W) : W.Nonempty := by
  obtain ⟨X, Y, hX, _, rfl⟩ := hW
  obtain ⟨a, ha⟩ := hn₀ X hX
  exact ⟨Sum.inl a, prodNbhd.mem_inl.mpr ha⟩

def distribMap (hn₀ : ∀ X, V₀.mem X → X.Nonempty) (hn₁ : ∀ Y, V₁.mem Y → Y.Nonempty)
    (hn₂ : ∀ Z, V₂.mem Z → Z.Nonempty) :
    ApproximableMap
      (sum (prod V₀ V₁) (prod V₀ V₂) (prod_mem_nonempty hn₀ hn₁) (prod_mem_nonempty hn₀
    hn₂))
      (prod V₀ (sum V₁ V₂ hn₁ hn₂)) :=
  copair
```

```
    (paired (proj₀ V₀ V₁) ((inMap₀ (h₀ := hn₁) (h₁ := hn₂)).comp (proj₁ V₀ V₁)))
    (paired (proj₀ V₀ V₂) ((inMap₁ (h₀ := hn₁) (h₁ := hn₂)).comp (proj₁ V₀ V₂)))

theorem copair_comp_inMap₀ (a : ApproximableMap V₀ V₂) (b : ApproximableMap V₁ V₂) :
    (copair (h₀ := h₀) (h₁ := h₁) a b).comp inMap₀ = a := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  · rintro ⟨W, ⟨hX, _, hinj⟩, _, hZ, hd⟩
    rcases hd with rfl | ⟨X', hWX', hf⟩ | ⟨Y', hWY', hg⟩
    · exact a.rel_master hX
    · exact a.mono hf (inj₀_subset_inj₀.mp (hWX' ▸ hinj)) subset_rfl hX hZ
    · exact (not_inj₀_subset_inj₁ (h₀ X hX) (hWY' ▸ hinj)).elim
  · intro hf
    exact ⟨inj₀ X, ⟨a.rel_dom hf, Or.inr (Or.inl ⟨X, a.rel_dom hf, rfl⟩), subset_rfl⟩,
      Or.inr (Or.inl ⟨X, a.rel_dom hf, rfl⟩), a.rel_cod hf,
      Or.inr (Or.inl ⟨X, rfl, hf⟩)⟩

theorem copair_comp_inMap₁ (a : ApproximableMap V₀ V₂) (b : ApproximableMap V₁ V₂) :
    (copair (h₀ := h₀) (h₁ := h₁) a b).comp inMap₁ = b := by
  apply ApproximableMap.ext
  intro Y Z
  constructor
  · rintro ⟨W, ⟨hY, _, hinj⟩, _, hZ, hd⟩
    rcases hd with rfl | ⟨X', hWX', hf⟩ | ⟨Y', hWY', hg⟩
    · exact b.rel_master hY
    · exact (not_inj₁_subset_inj₀ (h₁ Y hY) (hWX' ▸ hinj)).elim
    · exact b.mono hg (inj₁_subset_inj₁.mp (hWY' ▸ hinj)) subset_rfl hY hZ
  · intro hg
    exact ⟨inj₁ Y, ⟨b.rel_dom hg, Or.inr (Or.inr ⟨Y, b.rel_dom hg, rfl⟩), subset_rfl⟩,
      Or.inr (Or.inr ⟨Y, b.rel_dom hg, rfl⟩), b.rel_cod hg,
      Or.inr (Or.inr ⟨Y, rfl, hg⟩)⟩

theorem copairProj_copair (a : ApproximableMap V₀ V₂) (b : ApproximableMap V₁ V₂) :
    copairProj (h₀ := h₀) (h₁ := h₁) (copair a b) = (a, b) := by
  show ((copair (h₀ := h₀) (h₁ := h₁) a b).comp inMap₀,
        (copair (h₀ := h₀) (h₁ := h₁) a b).comp inMap₁) = (a, b)
  rw [copair_comp_inMap₀, copair_comp_inMap₁]
end

end
end DomainTheory.NeighbourhoodSystems
end
```

4.4.46 Exercise 3.24 — function-space isomorphisms

Establish some more isomorphisms :

(i) $(D_0 \to (D_1 \times D_2)) \cong (D_0 \to D_1) \times (D_0 \to D_2)$

(ii) $(D_0 \to D_1^{\infty}) \cong (D_0 \to D_1)^{\infty}$

(iii) $D_0 \times (D_1 + D_2) \cong (D_0 \times D_1) + (D_0 \times D_2)$

(iv) $(D_0 + D_1) \to D_2 \cong (D_0 \to D_2) \times (D_1 \to D_2) \ldots$

If some of the above are not true, perhaps at least some mapping relationships can be established.

The function-space isomorphisms and copairing laws are with the function-space isomorphisms peer.

Theorem (`example_324`). The function space into a product is isomorphic to the product of the function spaces.

Proof. Invoke the product function-space isomorphism. ■

Theorem (`example_324_iter`). The function space into the sequence system is isomorphic to the sequence system on the function space.

Proof. Invoke the sequence function-space isomorphism. ■

Theorem (`example_324_distrib`). Copairing after the injections recovers the summand maps, and projecting a copair recovers the original pair.

Proof. Invoke the corresponding copairing laws. ■

Exercise_3_24.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.funSpaceIso

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 3.24 (i): function space into a product. -/
theorem example_324 :
    ∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?},
      funSpace V₀ (prod V₁ V₂) ≅? prod (funSpace V₀ V₁) (funSpace V₀ V₂) :=
  @funProd_isomorphic

/-- Scott's Exercise 3.24 (ii): function space into the sequence system. -/
theorem example_324_iter :
    ∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β},
      funSpace V₀ (iterSys V₁) ≅? iterSys (funSpace V₀ V₁) :=
  @funIter_isomorphic

/-- Scott's Exercise 3.24 (iv): copairing recovers summand maps. -/
theorem example_324_distrib :
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
      (a : ApproximableMap V₀ V₂) (b : ApproximableMap V₁ V₂),
      copairProj (h₀ := h₀) (h₁ := h₁) (copair a b) = (a, b)) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
      (a : ApproximableMap V₀ V₂) (b : ApproximableMap V₁ V₂),
      (copair (h₀ := h₀) (h₁ := h₁) a b).comp inMap₀ = a) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
      (a : ApproximableMap V₀ V₂) (b : ApproximableMap V₁ V₂),
      (copair (h₀ := h₀) (h₁ := h₁) a b).comp inMap₁ = b) :=
```

```
⟨@copairProj_copair, @copair_comp_inMap₀, @copair_comp_inMap₁⟩
```

4.4.47 Opens form a domain

Open sets of a domain as maps into the Sierpiński system.

Definition (`sierpinski`). The Sierpiński neighbourhood system on a singleton has the empty set and the singleton as neighbourhoods, with master the singleton.

Proof. The filter axioms hold for these two sets. ■

Definition (`mapOfOpen`). An open set of ideal elements determines an approximable map into Sierpiński by relating a domain neighbourhood to the singleton precisely when the corresponding basic open lies in the open set, and relating freely to the empty neighbourhood.

Proof. Domain and codomain membership, master, intersection on the right, and monotonicity unpack the open-set axioms. ■

Theorem (`mapOfOpen_rel_empty`). Relating to the empty Sierpiński neighbourhood is domain membership together with membership of some principal in the open set.

Proof. Unpack the defining cases. ■

Definition (`openOfMap`). The open set of a map into Sierpiński consists of those ideal elements for which some principal neighbourhood relates to the singleton.

Proof. By definition. ■

Theorem (`openOfMap_isOpen`). The open set of a map into Sierpiński is open.

Proof. Witness basic opens via relating neighbourhoods. ■

Definition (`openIso`). Open subsets are order-isomorphic to approximable maps into Sierpiński.

Proof. The round trips are the previous constructions; order equivalence compares membership via principals. ■

Definition (`opensReprIso`). Open subsets are order-isomorphic to ideal elements of the function space into Sierpiński.

Proof. Compose the previous isomorphism with the function-space element isomorphism. ■

opensDomain.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.OpensDomain
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Set

def sierpinski : NeighborhoodSystem Unit where
  mem N := N = Set.univ ∨ N = ∅
  master := Set.univ
  master_nonempty := ⟨(), Set.mem_univ _⟩
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y Z hX hY hZ _
```

```
    rcases hX with rfl | rfl
    · rcases hY with rfl | rfl
      · left; rw [Set.univ_inter]
      · right; rw [Set.univ_inter]
    · right; rw [Set.empty_inter]
  sub_master := fun _ => Set.subset_univ _

section
variable {α : Type*} (V : NeighborhoodSystem α)

def mapOfOpen (U : Set V.Element) : ApproximableMap V sierpinski where
  rel X Y := V.mem X ∧ (Y = Set.univ ∨ (Y = ∅ ∧ V.basicOpen X ⊆ U))
  rel_dom h := h.1
  rel_cod h := by
    rcases h.2 with rfl | ⟨rfl, _⟩
    · exact Or.inl rfl
    · exact Or.inr rfl
  master_rel := ⟨V.master_mem, Or.inl rfl⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY⟩ ⟨_, hY'⟩
    refine ⟨hX, ?_⟩
    rcases hY with rfl | ⟨rfl, hsub⟩
    · rcases hY' with rfl | ⟨rfl, hsub'⟩
      · left; rw [Set.univ_inter]
      · right; rw [Set.univ_inter]; exact ⟨rfl, hsub'⟩
    · rcases hY' with rfl | ⟨rfl, _⟩
      · right; rw [Set.inter_univ]; exact ⟨rfl, hsub⟩
      · right; rw [Set.inter_self]; exact ⟨rfl, hsub⟩
  mono := by
    rintro X X' Y Y' ⟨hX, hY⟩ hX'X hYY' hX' hY'
    refine ⟨hX', ?_⟩
    rcases hY' with rfl | rfl
    · left; rfl
    · right
      refine ⟨rfl, ?_⟩
      rcases hY with rfl | ⟨rfl, hsub⟩
      · exact absurd (hYY' (Set.mem_univ ())) (by simp)
      · have hbo : V.basicOpen X' ⊆ V.basicOpen X := fun z hz => z.up_mem hz hX hX'X
        exact subset_trans hbo hsub

@[simp] theorem mapOfOpen_rel (U : Set V.Element) (X : Set α) (Y : Set Unit) :
    (mapOfOpen V U).rel X Y ↔ V.mem X ∧ (Y = Set.univ ∨ (Y = ∅ ∧ V.basicOpen X ⊆ U))
    := Iff.rfl

theorem mapOfOpen_rel_empty (U : Set V.Element) (X : Set α) :
    (mapOfOpen V U).rel X ∅ ↔ V.mem X ∧ V.basicOpen X ⊆ U := by
  rw [mapOfOpen_rel]
  constructor
  · rintro ⟨hX, h⟩
    refine ⟨hX, ?_⟩
    rcases h with h | ⟨_, h⟩
    · exact absurd h Set.empty_ne_univ
    · exact h
  · rintro ⟨hX, h⟩
    exact ⟨hX, Or.inr ⟨rfl, h⟩⟩
```

```
def openOfMap (f : ApproximableMap V sierpinski) : Set V.Element :=
  {x | ∃ X, x.mem X ∧ f.rel X ∅}

theorem openOfMap_isOpen (f : ApproximableMap V sierpinski) : IsOpen (openOfMap V f)
    := by
  constructor
  intro x hx
  obtain ⟨X, hxX, hf⟩ := hx
  exact ⟨X, hxX, fun z hz => ⟨X, hz, hf⟩⟩

end

section
variable {α : Type*} (V : NeighborhoodSystem α)

def openIso (V : NeighborhoodSystem α) :
    {U : Set V.Element // IsOpen U} ≃o ApproximableMap V sierpinski where
  toFun U := mapOfOpen V U.1
  invFun f := ⟨openOfMap V f, openOfMap_isOpen V f⟩
  left_inv U := by
    apply Subtype.ext
    ext x
    constructor
    · rintro ⟨X, hxX, hrel⟩
      rw [mapOfOpen_rel_empty] at hrel
      exact hrel.2 hxX
    · intro hx
      obtain ⟨X, hxX, hsub⟩ := U.2.out x hx
      exact ⟨X, hxX, (mapOfOpen_rel_empty V U.1 X).mpr ⟨x.sub hxX, hsub⟩⟩
  right_inv f := by
    apply ApproximableMap.ext
    intro X Y
    rw [mapOfOpen_rel]
    constructor
    · rintro ⟨hX, hd⟩
      rcases hd with rfl | ⟨rfl, hsub⟩
      · exact f.mono f.master_rel (V.sub_master hX) (Set.subset_univ _) hX (Or.inl rfl)
      · have hpr : V.principal hX ∈ V.basicOpen X := ⟨hX, subset_rfl⟩
        obtain ⟨X', hX', hf⟩ := hsub hpr
        exact f.mono hf hX'.2 subset_rfl hX (Or.inr rfl)
    · intro hf
      refine ⟨f.rel_dom hf, ?_⟩
      rcases f.rel_cod hf with rfl | rfl
      · left; rfl
      · exact Or.inr ⟨rfl, fun z hz => ⟨X, hz, hf⟩⟩
  map_rel_iff' := by
    intro a b
    show mapOfOpen V a.1 ≤ mapOfOpen V b.1 ↔ a ≤ b
    rw [ApproximableMap.le_iff]
    constructor
    · intro h x hx
      obtain ⟨X, hxX, hsub⟩ := a.2.out x hx
      have hrelB := h X ∅ ((mapOfOpen_rel_empty V a.1 X).mpr ⟨x.sub hxX, hsub⟩)
      rw [mapOfOpen_rel_empty] at hrelB
      exact hrelB.2 hxX
    · intro h X Y hrel
```

```
      rw [mapOfOpen_rel] at hrel ⊢
      obtain ⟨hX, hd⟩ := hrel
      refine ⟨hX, ?_⟩
      rcases hd with hu | ⟨he, hsub⟩
      · exact Or.inl hu
      · exact Or.inr ⟨he, fun z hz => h (hsub hz)⟩

def opensReprIso (V : NeighborhoodSystem α) :
    {U : Set V.Element // IsOpen U} ≃o (funSpace V sierpinski).Element :=
  (openIso V).trans (funSpaceEquiv V sierpinski).symm
end
end

end DomainTheory.NeighbourhoodSystems.OpensDomain
end
```

4.4.48 Exercise 3.25 — opens form a domain

(For topologists.) Recall from Exercises 1.21 and 2.13 on how to regard a domain $|D|$ as a topological space. Using 3.10 show that the family of open subsets of $|D|$ is isomorphic to a domain.

The Sierpiński representation of opens is with the opens form a domain peer.

Theorem (`example_325`). The open set associated to a map into Sierpiński is open.

Proof. Invoke openness of the open-of-map construction. ■

`Exercise_3_25.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.opensDomain

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem
open DomainTheory.NeighbourhoodSystems.OpensDomain
/-- Scott's Exercise 3.25: opens of a domain arise from maps into Sierpinski. -/
theorem example_325 :
    ∀ {α : Type*} (V : NeighborhoodSystem α) (f : ApproximableMap V sierpinski),
      IsOpen (openOfMap V f) :=
  @openOfMap_isOpen
```

4.4.49 Conditional combinator

The conditional operator on a domain, its sum-valued variant, and the which map from a separated sum.

Definition (`TD`). Write T for the token neighbourhood system of Example 1.2.

Proof. By abbreviation. ■

Theorem (`Tmem_eq_one`). A neighbourhood contained in the one neighbourhood equals the one neighbourhood.

Proof. Inspect the three neighbourhoods of Example 1.2. ■

Theorem (`Tmem_eq_zero`). A neighbourhood contained in the zero neighbourhood equals the zero neighbourhood.

Proof. Inspect the three neighbourhoods of Example 1.2. ■

Definition (`condGuard`). A conditional guard on a product neighbourhood and a codomain neighbourhood holds according to Scott's three cases on the token factor.

Proof. By definition. ■

Theorem (`cond_components`). A product neighbourhood for the conditional unpacks as a token neighbourhood paired with a product of domain neighbourhoods.

Proof. Unpack the product membership criterion. ■

Theorem (`one_ne_master`). The one neighbourhood is not the master.

Proof. Compare the underlying sets of Example 1.2. ■

Theorem (`zero_ne_master`). The zero neighbourhood is not the master.

Proof. Compare the underlying sets of Example 1.2. ■

Theorem (`condGuard_master`). A guard with master token factor forces the codomain neighbourhood to be the master.

Proof. Apply the master case of the guard. ■

Theorem (`zero_ne_one`). The zero and one neighbourhoods are distinct.

Proof. Compare the underlying sets of Example 1.2. ■

Theorem (`condGuard_one`). A guard with one token factor reduces to containment of the first domain neighbourhood in the codomain.

Proof. Apply the one case of the guard. ■

Theorem (`condGuard_zero`). A guard with zero token factor reduces to containment of the second domain neighbourhood in the codomain.

Proof. Apply the zero case of the guard. ■

Definition (`cond`). The conditional relates a product neighbourhood of the token system with a product of a domain with itself to a domain neighbourhood precisely when the conditional guard holds.

Proof. Domain and codomain membership unpack the witnesses; the masters relate by the master guard; intersection on the right and monotonicity follow by cases on the token factor. ■

Theorem (`cond_bot`). The conditional at bottom returns bottom.

Proof. Compare membership via the master token factor. ■

Theorem (`cond_false`). The conditional at false returns the second argument.

Proof. Compare membership via the zero token factor. ■

Theorem (`cond_toElementMap_mem`). Membership of the conditional value in a neighbourhood unpacks according to the three token cases.

Proof. Apply the guard characterisation at principals. ■

Theorem (`cond_true`). The conditional at true returns the first argument.

Proof. Compare membership via the one token factor. ■

Theorem (`Tmem_ge_master`). Every neighbourhood of the token system is contained in the master.

Proof. Apply the master bound. ■

Theorem (`Tmem_ge_one`). A neighbourhood contained in the one neighbourhood equals the one neighbourhood or is handled by the containment criterion.

Proof. Inspect the three neighbourhoods. ■

Theorem (`Tmem_ge_zero`). Symmetric to the one case for the zero neighbourhood.

Proof. Inspect the three neighbourhoods. ■

Definition (`condSumEmb`). The sum-valued conditional embedding pairs a token with the left and right injections of the two arguments.

Proof. Assemble pairing with the injections. ■

Definition (`condSum`). The sum-valued conditional is the conditional of the sum after the sum-valued embedding.

Proof. By definition. ■

Theorem (`inter_in_oneset`). Intersection of neighbourhoods from the master–one family stays in that family.

Proof. Case on the two possibilities. ■

Theorem (`inter_in_zeroset`). Intersection of neighbourhoods from the master–zero family stays in that family.

Proof. Case on the two possibilities. ■

Definition (`whichGuard`). A which-guard relates a sum neighbourhood to a token neighbourhood by the three cases distinguishing left injections, right injections, and the sum master.

Proof. By definition. ■

Theorem (`whichGuard_left`). A left-injection guard forces the token neighbourhood into the master–one family.

Proof. Unpack the left case. ■

Theorem (`whichGuard_masterC`). A sum-master guard forces the token neighbourhood to be the master.

Proof. Unpack the master case. ■

Theorem (`whichGuard_right`). A right-injection guard forces the token neighbourhood into the master–zero family.

Proof. Unpack the right case. ■

Definition (`whichMap`). The which map relates a sum neighbourhood to a token neighbourhood precisely when the which-guard holds.

Proof. Domain and codomain membership, master, intersection on the right, and monotonicity follow by cases on sum neighbourhoods. ■

Theorem (`condSumEmb_toElementMap`). Elementwise, the sum-valued embedding pairs a token with the two injections.

Proof. Unpack the defining pairing. ■

Theorem (`condSum_bot`). The sum-valued conditional at bottom returns bottom.

Proof. Rewrite via the embedding and the ordinary conditional at bottom. ■

Theorem (`condSum_false`). The sum-valued conditional at false returns the right injection of the second argument.

Proof. Rewrite via the embedding and the ordinary conditional at false. ■

Theorem (`condSum_true`). The sum-valued conditional at true returns the left injection of the first argument.

Proof. Rewrite via the embedding and the ordinary conditional at true. ■

Theorem (`not_sum_mem_empty`). The empty set is not a sum neighbourhood.

Proof. Each of the three kinds of sum neighbourhood is nonempty under the standing hypotheses. ■

Theorem (`inMap0_outMap0_eq_of_left`). On left summands, the left injection after the left projection recovers the element.

Proof. Compare membership via left injections and left parts. ■

Theorem (`inMap1_outMap1_eq_of_right`). On right summands, the right injection after the right projection recovers the element.

Proof. Compare membership via right injections and right parts. ■

Theorem (`sum_element_trichotomy`). Every sum element is a left summand, a right summand, or bottom.

Proof. Case on membership of distinguishing neighbourhoods. ■

Theorem (`which_mem_one`). The which value of a left summand belongs to the one neighbourhood.

Proof. Apply the left case of the which-guard. ■

Theorem (`which_mem_zero`). The which value of a right summand belongs to the zero neighbourhood.

Proof. Apply the right case of the which-guard. ■

Theorem (`cond_which`). Conditioning on which, with the two reinjected projections, recovers every sum element.

Proof. Case on the trichotomy and apply the conditional and injection–projection laws. ■

`cond.lean`

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.sumSys
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.inducedMaps
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.ProjectionMappings
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Truth
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Cond
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

section
variable {α : Type*}

abbrev TD : NeighborhoodSystem TwoToken.Token := TwoToken.neighborhoodSystem

theorem Tmem_eq_one {C : Set TwoToken.Token} (hC : TwoToken.mem C) (h : C ⊆ TwoToken.
    one) :
    C = TwoToken.one := by
  rcases (TwoToken.mem_iff C).mp hC with rfl | rfl | rfl
  · exact absurd (h (Set.mem_univ 0)) (by simp [TwoToken.one])
  · exact absurd (h (by simp [TwoToken.zero] : (0 : TwoToken.Token) ∈ TwoToken.zero))
      (by simp [TwoToken.one])
  · rfl

theorem Tmem_eq_zero {C : Set TwoToken.Token} (hC : TwoToken.mem C) (h : C ⊆ TwoToken.
    zero) :
    C = TwoToken.zero := by
  rcases (TwoToken.mem_iff C).mp hC with rfl | rfl | rfl
  · exact absurd (h (Set.mem_univ 1)) (by simp [TwoToken.zero])
  · rfl
  · exact absurd (h (by simp [TwoToken.one] : (1 : TwoToken.Token) ∈ TwoToken.one))
      (by simp [TwoToken.zero])

def condGuard (V : NeighborhoodSystem α) (W : Set (TwoToken.Token ? (α ? α))) (Z : Set
    α) : Prop :=
  (Sum.inl ^-1' W = TwoToken.zero ∧ Sum.inl ^-1' (Sum.inr ^-1' W) ⊆ Z) ∨
```

```
  (Sum.inl ^-1' W = TwoToken.one ∧ Sum.inr ^-1' (Sum.inr ^-1' W) ⊆ Z) ∨
  (Sum.inl ^-1' W = TwoToken.master ∧ V.master ⊆ Z)
end

section
variable {α : Type*}
variable (V : NeighborhoodSystem α)

theorem cond_components {W : Set (TwoToken.Token ? (α ? α))}
    (hW : (prod TD (prod V V)).mem W) :
    TwoToken.mem (Sum.inl ^-1' W) ∧
      V.mem (Sum.inl ^-1' (Sum.inr ^-1' W)) ∧ V.mem (Sum.inr ^-1' (Sum.inr ^-1' W)) :=
     by
  obtain ⟨C, P, hC, hP, rfl⟩ := hW
  obtain ⟨X, Y, hX, hY, rfl⟩ := hP
  simp only [prodNbhd.inl_preimage, prodNbhd.inr_preimage]
  exact ⟨hC, hX, hY⟩
end

section
variable {α : Type*}

theorem one_ne_master : (TwoToken.one : Set TwoToken.Token) = TwoToken.master := by
  intro h
  have h0 := Set.ext_iff.mp h 0
  simp [TwoToken.one, TwoToken.master] at h0

theorem zero_ne_master : (TwoToken.zero : Set TwoToken.Token) = TwoToken.master := by
  intro h
  have h1 := Set.ext_iff.mp h 1
  simp [TwoToken.zero, TwoToken.master] at h1
end

section
variable {α : Type*}
variable (V : NeighborhoodSystem α)

theorem condGuard_master {W : Set (TwoToken.Token ? (α ? α))} {Z : Set α} (hg :
    condGuard V W Z)
    (hC : Sum.inl ^-1' W = TwoToken.master) : V.master ⊆ Z := by
  rcases hg with ⟨hC', _⟩ | ⟨hC', _⟩ | ⟨_, h⟩
  · exact absurd (hC.symm.trans hC') zero_ne_master.symm
  · exact absurd (hC.symm.trans hC') one_ne_master.symm
  · exact h
end

section
variable {α : Type*}

theorem zero_ne_one : (TwoToken.zero : Set TwoToken.Token) = TwoToken.one := by
  intro h
  have h0 := Set.ext_iff.mp h 0
  simp [TwoToken.zero, TwoToken.one] at h0
end

section
```

```
variable {α : Type*}
variable (V : NeighborhoodSystem α)

theorem condGuard_one {W : Set (TwoToken.Token ? (α ? α))} {Z : Set α} (hg : condGuard
    V W Z)
    (hC : Sum.inl ^-1' W = TwoToken.one) : Sum.inr ^-1' (Sum.inr ^-1' W) ⊆ Z := by
  rcases hg with ⟨hC', _⟩ | ⟨_, h⟩ | ⟨hC', _⟩
  · exact absurd (hC.symm.trans hC') zero_ne_one.symm
  · exact h
  · exact absurd (hC.symm.trans hC') one_ne_master

theorem condGuard_zero {W : Set (TwoToken.Token ? (α ? α))} {Z : Set α} (hg :
   condGuard V W Z)
    (hC : Sum.inl ^-1' W = TwoToken.zero) : Sum.inl ^-1' (Sum.inr ^-1' W) ⊆ Z := by
  rcases hg with ⟨_, h⟩ | ⟨hC', _⟩ | ⟨hC', _⟩
  · exact h
  · exact absurd (hC.symm.trans hC') zero_ne_one
  · exact absurd (hC.symm.trans hC') zero_ne_master

def cond (V : NeighborhoodSystem α) : ApproximableMap (prod TD (prod V V)) V where
  rel W Z := (prod TD (prod V V)).mem W ∧ V.mem Z ∧ condGuard V W Z
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine ⟨(prod TD (prod V V)).master_mem, V.master_mem, Or.inr (Or.inr ⟨?_,
    subset_rfl⟩)⟩
    show Sum.inl ^-1' (prod TD (prod V V)).master = TwoToken.master
    rw [prod_master, prodNbhd.inl_preimage]; rfl
  inter_right := by
    rintro W Z Z' ⟨hW, hZ, hg⟩ ⟨_, hZ', hg'⟩
    obtain ⟨_, hX, hY⟩ := cond_components V hW
    refine ⟨hW, ?_, ?_⟩
    · rcases hg with ⟨hCz, hXZ⟩ | ⟨hCo, hYZ⟩ | ⟨hCm, hmZ⟩
      · exact V.inter_mem hZ hZ' hX (Set.subset_inter hXZ (condGuard_zero V hg' hCz))
      · exact V.inter_mem hZ hZ' hY (Set.subset_inter hYZ (condGuard_one V hg' hCo))
      · exact V.inter_mem hZ hZ' V.master_mem (Set.subset_inter hmZ (condGuard_master V
     hg' hCm))
    · rcases hg with ⟨hCz, hXZ⟩ | ⟨hCo, hYZ⟩ | ⟨hCm, hmZ⟩
      · exact Or.inl ⟨hCz, Set.subset_inter hXZ (condGuard_zero V hg' hCz)⟩
      · exact Or.inr (Or.inl ⟨hCo, Set.subset_inter hYZ (condGuard_one V hg' hCo)⟩)
      · exact Or.inr (Or.inr ⟨hCm, Set.subset_inter hmZ (condGuard_master V hg' hCm)⟩)
  mono := by
    rintro W W₂ Z Z₂ ⟨hW, hZ, hg⟩ hW₂W hZZ₂ hW₂ hZ₂
    obtain ⟨hC₂, hX₂, hY₂⟩ := cond_components V hW₂
    refine ⟨hW₂, hZ₂, ?_⟩
    have hCsub : Sum.inl ^-1' W₂ ⊆ Sum.inl ^-1' W := Set.preimage_mono hW₂W
    have hXsub : Sum.inl ^-1' (Sum.inr ^-1' W₂) ⊆ Sum.inl ^-1' (Sum.inr ^-1' W) :=
      Set.preimage_mono (Set.preimage_mono hW₂W)
    have hYsub : Sum.inr ^-1' (Sum.inr ^-1' W₂) ⊆ Sum.inr ^-1' (Sum.inr ^-1' W) :=
      Set.preimage_mono (Set.preimage_mono hW₂W)
    rcases hg with ⟨hCz, hXZ⟩ | ⟨hCo, hYZ⟩ | ⟨hCm, hmZ⟩
    · exact Or.inl ⟨Tmem_eq_zero hC₂ (hCsub.trans hCz.subset), (hXsub.trans hXZ).trans
    hZZ₂⟩
    · exact Or.inr (Or.inl ⟨Tmem_eq_one hC₂ (hCsub.trans hCo.subset),
        (hYsub.trans hYZ).trans hZZ₂⟩)
    · have hmZ₂ : V.master ⊆ Z₂ := hmZ.trans hZZ₂
```

```
      rcases (TwoToken.mem_iff (Sum.inl ^-1' W₂)).mp hC₂ with hC₂m | hC₂z | hC₂o
      · exact Or.inr (Or.inr ⟨hC₂m, hmZ₂⟩)
      · exact Or.inl ⟨hC₂z, (V.sub_master hX₂).trans hmZ₂⟩
      · exact Or.inr (Or.inl ⟨hC₂o, (V.sub_master hY₂).trans hmZ₂⟩)

@[simp] theorem cond_rel {W : Set (TwoToken.Token ? (α ? α))} {Z : Set α} :
    (cond V).rel W Z ↔ (prod TD (prod V V)).mem W ∧ V.mem Z ∧ condGuard V W Z := Iff.
    rfl

theorem cond_bot (x y : V.Element) :
    (cond V).toElementMap (pair Truth.botElt (pair x y)) = V.bot := by
  apply Element.ext
  intro Z
  constructor
  · rintro ⟨W, hWmem, _, hZ, hg⟩
    obtain ⟨C, P, hCbot, hPmem, rfl⟩ := hWmem
    obtain ⟨X, Y, hXx, hYy, rfl⟩ := hPmem
    simp only [condGuard, prodNbhd.inl_preimage, prodNbhd.inr_preimage] at hg
    have hCm : C = TwoToken.master := hCbot
    rcases hg with ⟨hCz, _⟩ | ⟨hCo, _⟩ | ⟨_, hmZ⟩
    · exact absurd (hCm.symm.trans hCz) zero_ne_master.symm
    · exact absurd (hCm.symm.trans hCo) one_ne_master.symm
    · rw [mem_bot, Set.Subset.antisymm (V.sub_master hZ) hmZ]
  · intro hbZ
    rw [mem_bot] at hbZ
    subst hbZ
    refine ⟨prodNbhd TwoToken.master (prodNbhd V.master V.master),
      ⟨TwoToken.master, prodNbhd V.master V.master, rfl,
        ⟨V.master, V.master, x.master_mem, y.master_mem, rfl⟩, rfl⟩,
      prod_mem_prodNbhd TwoToken.mem_master (prod_mem_prodNbhd V.master_mem V.
    master_mem),
      V.master_mem, ?_⟩
    exact Or.inr (Or.inr ⟨prodNbhd.inl_preimage _ _, subset_rfl⟩)

theorem cond_false (x y : V.Element) :
    (cond V).toElementMap (pair Truth.falseElt (pair x y)) = y := by
  apply Element.ext
  intro Z
  constructor
  · rintro ⟨W, hWmem, _, hZ, hg⟩
    obtain ⟨C, P, hCfalse, hPmem, rfl⟩ := hWmem
    obtain ⟨X, Y, hXx, hYy, rfl⟩ := hPmem
    simp only [condGuard, prodNbhd.inl_preimage, prodNbhd.inr_preimage] at hg
    rcases hg with ⟨hCz, _⟩ | ⟨_, hYZ⟩ | ⟨_, hmZ⟩
    · rcases hCfalse with hCm | hCo
      · exact absurd (hCm.symm.trans hCz) zero_ne_master.symm
      · exact absurd (hCo.symm.trans hCz) zero_ne_one.symm
    · exact y.up_mem hYy hZ hYZ
    · rw [Set.Subset.antisymm (V.sub_master hZ) hmZ]; exact y.master_mem
  · intro hyZ
    have hZ : V.mem Z := y.sub hyZ
    refine ⟨prodNbhd TwoToken.one (prodNbhd V.master Z), ⟨TwoToken.one, prodNbhd V.
    master Z,
      Or.inr rfl, ⟨V.master, Z, x.master_mem, hyZ, rfl⟩, rfl⟩,
      prod_mem_prodNbhd TwoToken.mem_one (prod_mem_prodNbhd V.master_mem hZ), hZ, ?_⟩
    refine Or.inr (Or.inl ⟨prodNbhd.inl_preimage _ _, ?_⟩)
```

```
    rw [prodNbhd.inr_preimage, prodNbhd.inr_preimage]

theorem cond_toElementMap_mem (t : TD.Element) (x y : V.Element) {Z : Set α} :
    ((cond V).toElementMap (pair t (pair x y))).mem Z ↔
      (t.mem TwoToken.zero ∧ x.mem Z) ∨ (t.mem TwoToken.one ∧ y.mem Z) ∨ Z = V.master
    := by
  constructor
  · rintro ⟨W, hWmem, _, hZ, hg⟩
    obtain ⟨C, P, hCt, hPmem, rfl⟩ := hWmem
    obtain ⟨X, Y, hXx, hYy, rfl⟩ := hPmem
    simp only [condGuard, prodNbhd.inl_preimage, prodNbhd.inr_preimage] at hg
    rcases hg with ⟨hCz, hXZ⟩ | ⟨hCo, hYZ⟩ | ⟨_, hmZ⟩
    · exact Or.inl ⟨hCz ▸ hCt, x.up_mem hXx hZ hXZ⟩
    · exact Or.inr (Or.inl ⟨hCo ▸ hCt, y.up_mem hYy hZ hYZ⟩)
    · exact Or.inr (Or.inr (Set.Subset.antisymm (V.sub_master hZ) hmZ))
  · intro h
    rcases h with ⟨ht0, hxZ⟩ | ⟨ht1, hyZ⟩ | rfl
    · have hZ : V.mem Z := x.sub hxZ
      refine ⟨prodNbhd TwoToken.zero (prodNbhd Z V.master), ⟨TwoToken.zero, prodNbhd Z
    V.master,
        ht0, ⟨Z, V.master, hxZ, y.master_mem, rfl⟩, rfl⟩,
        prod_mem_prodNbhd TwoToken.mem_zero (prod_mem_prodNbhd hZ V.master_mem), hZ, ?
    _⟩
      refine Or.inl ⟨prodNbhd.inl_preimage _ _, ?_⟩
      rw [prodNbhd.inr_preimage, prodNbhd.inl_preimage]
    · have hZ : V.mem Z := y.sub hyZ
      refine ⟨prodNbhd TwoToken.one (prodNbhd V.master Z), ⟨TwoToken.one, prodNbhd V.
    master Z,
        ht1, ⟨V.master, Z, x.master_mem, hyZ, rfl⟩, rfl⟩,
        prod_mem_prodNbhd TwoToken.mem_one (prod_mem_prodNbhd V.master_mem hZ), hZ, ?_⟩
      refine Or.inr (Or.inl ⟨prodNbhd.inl_preimage _ _, ?_⟩)
      rw [prodNbhd.inr_preimage, prodNbhd.inr_preimage]
    · refine ⟨prodNbhd TwoToken.master (prodNbhd V.master V.master),
        ⟨TwoToken.master, prodNbhd V.master V.master, t.master_mem,
          ⟨V.master, V.master, x.master_mem, y.master_mem, rfl⟩, rfl⟩,
        prod_mem_prodNbhd TwoToken.mem_master (prod_mem_prodNbhd V.master_mem V.
    master_mem),
        V.master_mem, ?_⟩
      exact Or.inr (Or.inr ⟨prodNbhd.inl_preimage _ _, subset_rfl⟩)

theorem cond_true (x y : V.Element) :
    (cond V).toElementMap (pair Truth.trueElt (pair x y)) = x := by
  apply Element.ext
  intro Z
  constructor
  · rintro ⟨W, hWmem, _, hZ, hg⟩
    obtain ⟨C, P, hCtrue, hPmem, rfl⟩ := hWmem
    obtain ⟨X, Y, hXx, hYy, rfl⟩ := hPmem
    simp only [condGuard, prodNbhd.inl_preimage, prodNbhd.inr_preimage] at hg
    rcases hg with ⟨_, hXZ⟩ | ⟨hCo, _⟩ | ⟨_, hmZ⟩
    · exact x.up_mem hXx hZ hXZ
    · rcases hCtrue with hCm | hCz
      · exact absurd (hCm.symm.trans hCo) one_ne_master.symm
      · exact absurd (hCz.symm.trans hCo) zero_ne_one
    · rw [Set.Subset.antisymm (V.sub_master hZ) hmZ]; exact x.master_mem
  · intro hxZ
```

```
    have hZ : V.mem Z := x.sub hxZ
    refine ⟨prodNbhd TwoToken.zero (prodNbhd Z V.master), ⟨TwoToken.zero, prodNbhd Z V.
    master,
      Or.inr rfl, ⟨Z, V.master, hxZ, y.master_mem, rfl⟩, rfl⟩,
      prod_mem_prodNbhd TwoToken.mem_zero (prod_mem_prodNbhd hZ V.master_mem), hZ, ?_⟩
    refine Or.inl ⟨prodNbhd.inl_preimage _ _, ?_⟩
    rw [prodNbhd.inr_preimage, prodNbhd.inl_preimage]
end

section
variable {α β : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}

theorem Tmem_ge_master {C : Set TwoToken.Token} (hC : TwoToken.mem C)
    (h : TwoToken.master ⊆ C) : C = TwoToken.master :=
  Set.Subset.antisymm (TwoToken.neighborhoodSystem.sub_master hC) h

theorem Tmem_ge_one {C : Set TwoToken.Token} (hC : TwoToken.mem C)
    (h : TwoToken.one ⊆ C) : C = TwoToken.one ∨ C = TwoToken.master := by
  rcases (TwoToken.mem_iff C).mp hC with rfl | rfl | rfl
  · exact Or.inr rfl
  · exact absurd (h (by simp [TwoToken.one] : (1 : TwoToken.Token) ∈ TwoToken.one))
      (by simp [TwoToken.zero])
  · exact Or.inl rfl

theorem Tmem_ge_zero {C : Set TwoToken.Token} (hC : TwoToken.mem C)
    (h : TwoToken.zero ⊆ C) : C = TwoToken.zero ∨ C = TwoToken.master := by
  rcases (TwoToken.mem_iff C).mp hC with rfl | rfl | rfl
  · exact Or.inr rfl
  · exact Or.inl rfl
  · exact absurd (h (by simp [TwoToken.zero] : (0 : TwoToken.Token) ∈ TwoToken.zero))
      (by simp [TwoToken.one])

def condSumEmb (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty) :
    ApproximableMap (prod TD (prod V₀ V₁)) (prod TD (prod (sum V₀ V₁ h₀ h₁) (sum V₀ V₁ h₀
     h₁))) :=
  paired (proj₀ TD (prod V₀ V₁))
    (paired ((inMap₀ (h₀ := h₀) (h₁ := h₁)).comp ((proj₀ V₀ V₁).comp (proj₁ TD (prod V₀ V
    ₁))))
      ((inMap₁ (h₀ := h₀) (h₁ := h₁)).comp ((proj₁ V₀ V₁).comp (proj₁ TD (prod V₀ V₁)))))

def condSum (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty) :
    ApproximableMap (prod TD (prod V₀ V₁)) (sum V₀ V₁ h₀ h₁) :=
  (cond (sum V₀ V₁ h₀ h₁)).comp (condSumEmb V₀ V₁ h₀ h₁)

theorem inter_in_oneset {C C' : Set TwoToken.Token} (hc : C = TwoToken.master ∨ C =
    TwoToken.one)
    (hc' : C' = TwoToken.master ∨ C' = TwoToken.one) :
    C ∩ C' = TwoToken.master ∨ C ∩ C' = TwoToken.one := by
  rcases hc with rfl | rfl <;> rcases hc' with rfl | rfl
  · exact Or.inl (Set.inter_self _)
  · exact Or.inr (Set.univ_inter _)
  · exact Or.inr (Set.inter_univ _)
```

```
  · exact Or.inr (Set.inter_self _)

theorem inter_in_zeroset {C C' : Set TwoToken.Token} (hc : C = TwoToken.master ∨ C =
    TwoToken.zero)
    (hc' : C' = TwoToken.master ∨ C' = TwoToken.zero) :
    C ∩ C' = TwoToken.master ∨ C ∩ C' = TwoToken.zero := by
  rcases hc with rfl | rfl <;> rcases hc' with rfl | rfl
  · exact Or.inl (Set.inter_self _)
  · exact Or.inr (Set.univ_inter _)
  · exact Or.inr (Set.inter_univ _)
  · exact Or.inr (Set.inter_self _)

def whichGuard (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (W : Set (Option (α ⊕ β))) (C : Set TwoToken.Token) : Prop :=
  ((∃ X, V₀.mem X ∧ W = inj₀ X) ∧ (C = TwoToken.master ∨ C = TwoToken.zero)) ∨
  ((∃ Y, V₁.mem Y ∧ W = inj₁ Y) ∧ (C = TwoToken.master ∨ C = TwoToken.one)) ∨
  (W = sumMaster V₀ V₁ ∧ C = TwoToken.master)

theorem whichGuard_left {W : Set (Option (α ⊕ β))} {C : Set TwoToken.Token} {X : Set α
    }
    (hg : whichGuard V₀ V₁ W C) (hW : W = inj₀ X) (hX : X.Nonempty) :
    C = TwoToken.master ∨ C = TwoToken.zero := by
  rcases hg with ⟨_, hc⟩ | ⟨⟨Y, _, hWY⟩, _⟩ | ⟨hWsm, _⟩
  · exact hc
  · refine absurd (hW.symm.trans hWY) ?_
    intro h; exact not_inj₀_subset_inj₁ hX h.subset
  · refine absurd (hW.symm.trans hWsm) ?_
    intro h; exact none_mem_inj₀ (h ▸ none_mem_sumMaster)

theorem whichGuard_masterC {W : Set (Option (α ⊕ β))} {C : Set TwoToken.Token}
    (hg : whichGuard V₀ V₁ W C) (hW : W = sumMaster V₀ V₁) : C = TwoToken.master := by
  rcases hg with ⟨⟨X, _, hWX⟩, _⟩ | ⟨⟨Y, _, hWY⟩, _⟩ | ⟨_, hc⟩
  · refine absurd (hW.symm.trans hWX) ?_
    intro h; exact none_mem_inj₀ (h ▸ none_mem_sumMaster)
  · refine absurd (hW.symm.trans hWY) ?_
    intro h; exact none_mem_inj₁ (h ▸ none_mem_sumMaster)
  · exact hc

theorem whichGuard_right {W : Set (Option (α ⊕ β))} {C : Set TwoToken.Token} {Y : Set
    β}
    (hg : whichGuard V₀ V₁ W C) (hW : W = inj₁ Y) (hY : Y.Nonempty) :
    C = TwoToken.master ∨ C = TwoToken.one := by
  rcases hg with ⟨⟨X, _, hWX⟩, _⟩ | ⟨_, hc⟩ | ⟨hWsm, _⟩
  · refine absurd (hW.symm.trans hWX) ?_
    intro h; exact not_inj₁_subset_inj₀ hY h.subset
  · exact hc
  · refine absurd (hW.symm.trans hWsm) ?_
    intro h; exact none_mem_inj₁ (h ▸ none_mem_sumMaster)

def whichMap (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty) :
    ApproximableMap (sum V₀ V₁ h₀ h₁) TD where
  rel W C := (sum V₀ V₁ h₀ h₁).mem W ∧ TwoToken.mem C ∧ whichGuard V₀ V₁ W C
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(sum V₀ V₁ h₀ h₁).master_mem, TwoToken.mem_master, Or.inr (Or.inr ⟨rfl,
```

```
    rfl⟩)⟩
  inter_right := by
    rintro W C C' ⟨hW, _, hg⟩ ⟨_, _, hg'⟩
    rcases hg with ⟨⟨X, hX, rfl⟩, hc⟩ | ⟨⟨Y, hY, rfl⟩, hc⟩ | ⟨rfl, hc⟩
    · have hor := inter_in_zeroset hc (whichGuard_left hg' rfl (h₀ X hX))
      refine ⟨Or.inr (Or.inl ⟨X, hX, rfl⟩), ?_, Or.inl ⟨⟨X, hX, rfl⟩, hor⟩⟩
      rcases hor with h | h
      · rw [h]; exact TwoToken.mem_master
      · rw [h]; exact TwoToken.mem_zero
    · have hor := inter_in_oneset hc (whichGuard_right hg' rfl (h₁ Y hY))
      refine ⟨Or.inr (Or.inr ⟨Y, hY, rfl⟩), ?_, Or.inr (Or.inl ⟨⟨Y, hY, rfl⟩, hor⟩)⟩
      rcases hor with h | h
      · rw [h]; exact TwoToken.mem_master
      · rw [h]; exact TwoToken.mem_one
    · have hc' := whichGuard_masterC hg' rfl
      subst hc; subst hc'
      rw [Set.inter_self]
      exact ⟨Or.inl rfl, TwoToken.mem_master, Or.inr (Or.inr ⟨rfl, rfl⟩)⟩
  mono := by
    rintro W W₂ C C₂ ⟨hW, hC, hg⟩ hW₂W hCC₂ hW₂ hC₂
    refine ⟨hW₂, hC₂, ?_⟩
    rcases hg with ⟨⟨X, hX, rfl⟩, hc⟩ | ⟨⟨Y, hY, rfl⟩, hc⟩ | ⟨rfl, hc⟩
    · obtain ⟨X₂, hX₂, rfl⟩ := mem_subset_inj₀ hW₂ hW₂W
      refine Or.inl ⟨⟨X₂, hX₂, rfl⟩, ?_⟩
      rcases hc with rfl | rfl
      · exact Or.inl (Tmem_ge_master hC₂ hCC₂)
      · exact Or.symm (Tmem_ge_zero hC₂ hCC₂)
    · obtain ⟨Y₂, hY₂, rfl⟩ := mem_subset_inj₁ hW₂ hW₂W
      refine Or.inr (Or.inl ⟨⟨Y₂, hY₂, rfl⟩, ?_⟩)
      rcases hc with rfl | rfl
      · exact Or.inl (Tmem_ge_master hC₂ hCC₂)
      · exact Or.symm (Tmem_ge_one hC₂ hCC₂)
    · subst hc
      have hC₂m : C₂ = TwoToken.master := Tmem_ge_master hC₂ hCC₂
      rcases hW₂ with rfl | ⟨X₂, hX₂, rfl⟩ | ⟨Y₂, hY₂, rfl⟩
      · exact Or.inr (Or.inr ⟨rfl, hC₂m⟩)
      · exact Or.inl ⟨⟨X₂, hX₂, rfl⟩, Or.inl hC₂m⟩
      · exact Or.inr (Or.inl ⟨⟨Y₂, hY₂, rfl⟩, Or.inl hC₂m⟩)

theorem condSumEmb_toElementMap (t : TD.Element) (x : V₀.Element) (y : V₁.Element) :
    (condSumEmb V₀ V₁ h₀ h₁).toElementMap (pair t (pair x y))
      = pair t (pair ((inMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap x)
          ((inMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap y)) := by
  simp only [condSumEmb, toElementMap_paired, toElementMap_comp, toElementMap_proj₀,
    toElementMap_proj₁, fst_pair, snd_pair]

end
end

section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
variable {α β : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
```

```
theorem condSum_bot (x : V₀.Element) (y : V₁.Element) :
    (condSum V₀ V₁ h₀ h₁).toElementMap (pair Truth.botElt (pair x y)) = (sum V₀ V₁ h₀ h₁
    ).bot := by
  rw [condSum, toElementMap_comp, condSumEmb_toElementMap, cond_bot]

theorem condSum_false (x : V₀.Element) (y : V₁.Element) :
    (condSum V₀ V₁ h₀ h₁).toElementMap (pair Truth.falseElt (pair x y))
      = (inMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap y := by
  rw [condSum, toElementMap_comp, condSumEmb_toElementMap, cond_false]

theorem condSum_true (x : V₀.Element) (y : V₁.Element) :
    (condSum V₀ V₁ h₀ h₁).toElementMap (pair Truth.trueElt (pair x y))
      = (inMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap x := by
  rw [condSum, toElementMap_comp, condSumEmb_toElementMap, cond_true]

theorem not_sum_mem_empty : ¬ (sum V₀ V₁ h₀ h₁).mem ∅ := by
  rintro (h | ⟨X, hX, h⟩ | ⟨Y, hY, h⟩)
  · exact absurd (h.symm ▸ none_mem_sumMaster) (Set.notMem_empty _)
  · obtain ⟨a, ha⟩ := inj₀_nonempty (h₀ X hX)
    exact absurd (h ▸ ha) (Set.notMem_empty _)
  · obtain ⟨b, hb⟩ := inj₁_nonempty (h₁ Y hY)
    exact absurd (h ▸ hb) (Set.notMem_empty _)

theorem inMap₀_outMap₀_eq_of_left {x : (sum V₀ V₁ h₀ h₁).Element}
    (hL : ∃ X, V₀.mem X ∧ x.mem (inj₀ X)) :
    (inMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap ((outMap₀ (h₀ := h₀) (h₁ := h₁)).
   toElementMap x) = x := by
  obtain ⟨X₀, hX₀, hxX₀⟩ := hL
  apply Element.ext
  intro Z
  constructor
  · rintro ⟨X, ⟨W, hxW, _, _, hLWX⟩, _, hSZ, hinjXZ⟩

    have hxWX₀ : x.mem (W ∩ inj₀ X₀) := x.inter_mem hxW hxX₀
    obtain ⟨X₁, _, hX₁eq⟩ := mem_subset_inj₀ (x.sub hxWX₀) Set.inter_subset_right
    have hX₁X : X₁ ⊆ X := by
      have : leftPart V₀ (W ∩ inj₀ X₀) ⊆ X :=
        (leftPart_mono V₀ Set.inter_subset_left).trans hLWX
      rwa [hX₁eq, leftPart_inj₀] at this
    have : (W ∩ inj₀ X₀) ⊆ Z :=
      hX₁eq ▸ (inj₀_subset_inj₀.mpr hX₁X).trans hinjXZ
    exact x.up_mem hxWX₀ hSZ this
  · intro hxZ
    rcases x.sub hxZ with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
    ·
      exact ⟨V₀.master, ⟨sumMaster V₀ V₁, x.master_mem, (sum V₀ V₁ h₀ h₁).master_mem,
        V₀.master_mem, (leftPart_sumMaster V₀ V₁).subset⟩, V₀.master_mem,
        (sum V₀ V₁ h₀ h₁).master_mem, inj₀_subset_sumMaster V₀.master_mem⟩
    ·
      exact ⟨X', ⟨inj₀ X', hxZ, Or.inr (Or.inl ⟨X', hX', rfl⟩)⟩, hX', (leftPart_inj₀ V₀ X
    ').subset⟩,
        hX', Or.inr (Or.inl ⟨X', hX', rfl⟩)⟩, subset_rfl⟩
    ·
      have := x.inter_mem hxX₀ hxZ
      rw [inj₀_inter_inj₁] at this
      exact absurd (x.sub this) not_sum_mem_empty
```

```
theorem inMap₁_outMap₁_eq_of_right {x : (sum V₀ V₁ h₀ h₁).Element}
    (hR : ∃ Y, V₁.mem Y ∧ x.mem (inj₁ Y)) :
    (inMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap ((outMap₁ (h₀ := h₀) (h₁ := h₁)).
   toElementMap x) = x := by
  obtain ⟨Y₀, hY₀, hxY₀⟩ := hR
  apply Element.ext
  intro Z
  constructor
  · rintro ⟨Y, ⟨W, hxW, _, _, hRWY⟩, _, hSZ, hinjYZ⟩
    have hxWY₀ : x.mem (W ∩ inj₁ Y₀) := x.inter_mem hxW hxY₀
    obtain ⟨Y₁, _, hY₁eq⟩ := mem_subset_inj₁ (x.sub hxWY₀) Set.inter_subset_right
    have hY₁Y : Y₁ ⊆ Y := by
      have : rightPart V₁ (W ∩ inj₁ Y₀) ⊆ Y :=
        (rightPart_mono V₁ Set.inter_subset_left).trans hRWY
      rwa [hY₁eq, rightPart_inj₁] at this
    have : (W ∩ inj₁ Y₀) ⊆ Z :=
      hY₁eq ▸ (inj₁_subset_inj₁.mpr hY₁Y).trans hinjYZ
    exact x.up_mem hxWY₀ hSZ this
  · intro hxZ
    rcases x.sub hxZ with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
    · exact ⟨V₁.master, ⟨sumMaster V₀ V₁, x.master_mem, (sum V₀ V₁ h₀ h₁).master_mem,
        V₁.master_mem, (rightPart_sumMaster V₀ V₁).subset⟩, V₁.master_mem,
        (sum V₀ V₁ h₀ h₁).master_mem, inj₁_subset_sumMaster V₁.master_mem⟩
    · have := x.inter_mem hxY₀ hxZ
      rw [Set.inter_comm, inj₀_inter_inj₁] at this
      exact absurd (x.sub this) not_sum_mem_empty
    · exact ⟨Y', ⟨inj₁ Y', hxZ, Or.inr (Or.inr ⟨Y', hY', rfl⟩), hY', (rightPart_inj₁ V₁ Y
    ').subset⟩,
        hY', Or.inr (Or.inr ⟨Y', hY', rfl⟩), subset_rfl⟩

theorem sum_element_trichotomy (x : (sum V₀ V₁ h₀ h₁).Element) :
    (∃ X, V₀.mem X ∧ x.mem (inj₀ X)) ∨ (∃ Y, V₁.mem Y ∧ x.mem (inj₁ Y)) ∨
      (∀ W, x.mem W → W = (sum V₀ V₁ h₀ h₁).master) := by
  by_cases hL : ∃ X, V₀.mem X ∧ x.mem (inj₀ X)
  · exact Or.inl hL
  by_cases hR : ∃ Y, V₁.mem Y ∧ x.mem (inj₁ Y)
  · exact Or.inr (Or.inl hR)
  refine Or.inr (Or.inr ?_)
  intro W hxW
  rcases x.sub hxW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
  · rfl
  · exact absurd ⟨X, hX, hxW⟩ hL
  · exact absurd ⟨Y, hY, hxW⟩ hR

theorem which_mem_one {x : (sum V₀ V₁ h₀ h₁).Element} :
    ((whichMap V₀ V₁ h₀ h₁).toElementMap x).mem TwoToken.one
      ↔ ∃ Y, V₁.mem Y ∧ x.mem (inj₁ Y) := by
  constructor
  · rintro ⟨W, hWx, _, _, hg⟩
    rcases hg with ⟨_, hc⟩ | ⟨⟨Y, hY, rfl⟩, _⟩ | ⟨_, hc⟩
    · rcases hc with h | h
      · exact absurd h one_ne_master
      · exact absurd h zero_ne_one.symm
    · exact ⟨Y, hY, hWx⟩
    · exact absurd hc one_ne_master
```

```
  · rintro ⟨Y, hY, hWx⟩
    exact ⟨inj₁ Y, hWx, Or.inr (Or.inr ⟨Y, hY, rfl⟩), TwoToken.mem_one,
      Or.inr (Or.inl ⟨⟨Y, hY, rfl⟩, Or.inr rfl⟩)⟩

theorem which_mem_zero {x : (sum V₀ V₁ h₀ h₁).Element} :
    ((whichMap V₀ V₁ h₀ h₁).toElementMap x).mem TwoToken.zero
      ↔ ∃ X, V₀.mem X ∧ x.mem (inj₀ X) := by
  constructor
  · rintro ⟨W, hWx, _, _, hg⟩
    rcases hg with ⟨⟨X, hX, rfl⟩, _⟩ | ⟨_, hc⟩ | ⟨_, hc⟩
    · exact ⟨X, hX, hWx⟩
    · rcases hc with h | h
      · exact absurd h zero_ne_master
      · exact absurd h zero_ne_one
    · exact absurd hc zero_ne_master
  · rintro ⟨X, hX, hWx⟩
    exact ⟨inj₀ X, hWx, Or.inr (Or.inl ⟨X, hX, rfl⟩), TwoToken.mem_zero,
      Or.inl ⟨⟨X, hX, rfl⟩, Or.inr rfl⟩⟩

theorem cond_which (x : (sum V₀ V₁ h₀ h₁).Element) :
    (cond (sum V₀ V₁ h₀ h₁)).toElementMap
        (pair ((whichMap V₀ V₁ h₀ h₁).toElementMap x)
          (pair ((inMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap
                  ((outMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap x))
                ((inMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap
                  ((outMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap x)))) = x := by
  apply Element.ext
  intro Z
  rw [cond_toElementMap_mem, which_mem_zero, which_mem_one]
  constructor
  · rintro (⟨hL, ha⟩ | ⟨hR, hb⟩ | rfl)
    · rwa [inMap₀_outMap₀_eq_of_left hL] at ha
    · rwa [inMap₁_outMap₁_eq_of_right hR] at hb
    · exact x.master_mem
  · intro hxZ
    rcases sum_element_trichotomy x with hL | hR | hN
    · exact Or.inl ⟨hL, by rw [inMap₀_outMap₀_eq_of_left hL]; exact hxZ⟩
    · exact Or.inr (Or.inl ⟨hR, by rw [inMap₁_outMap₁_eq_of_right hR]; exact hxZ⟩)
    · exact Or.inr (Or.inr (hN Z hxZ))
end

end DomainTheory.NeighbourhoodSystems.Cond
end
```

4.4.50 Exercise 3.26 — conditional combinator

Show that for every domain D there is an approximable mapping

$$\mathrm{cond} : T \times D \times D \to D,$$

called the *conditional operator*, satisfying

(i) $\mathrm{cond}(\mathrm{true}, x, y) = x$

(ii) $\mathrm{cond}(\mathrm{false}, x, y) = y$

(iii) $\mathrm{cond}(\bot, x, y) = \bot$.

(Hint: Recalling that $T = \{\{0\}, \{1\}, \{0, 1\}\}$, define cond as a relation by

$$\begin{aligned} 0C \cup 10X \cup 110Y \ \mathrm{cond}\ Z \quad \text{iff} \quad & 0 \in C \text{ and } X \subseteq Z \text{ or} \\ & 1 \in C \text{ and } Y \subseteq Z \text{ or} \\ & 0, 1 \in C \text{ and } \Delta \subseteq Z, \end{aligned}$$

where $C \in T$ and $X \in D$ and $Y \in D$ and where we are using the construction of Exercise 3.14.) Find a similar operator in the domain

$$T \times D_0 \times D_1 \to D_0 + D_1.$$

Show also there is an approximable mapping

$$\mathrm{which} : D_0 + D_1 \to T$$

such that for all $x \in |D_0 + D_1|$

$$\mathrm{cond}(\mathrm{which}(x), \mathrm{in}_0(\mathrm{out}_0(x)), \mathrm{in}_1(\mathrm{out}_1(x))) = x.$$

The conditional, sum-valued conditional, and which map are with the conditional combinator peer.

Theorem (`example_326`). The conditional returns the first argument at true, the second at false, and bottom at bottom.

Proof. Invoke the corresponding conditional evaluation laws. ■

Theorem (`example_326_sum`). The sum-valued conditional returns the reinjected arguments at true and false and bottom at bottom, and conditioning on which recovers every sum element.

Proof. Invoke the corresponding sum-conditional and which laws. ■

`Exercise_3_26.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.cond

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap Truth
open DomainTheory.NeighbourhoodSystems.Cond
/-- Scott's Exercise 3.26: conditional at true, false, and bottom. -/
theorem example_326 :
    (∀ {α : Type*} (V : NeighborhoodSystem α) (x y : V.Element),
      (cond V).toElementMap (pair trueElt (pair x y)) = x) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (x y : V.Element),
      (cond V).toElementMap (pair falseElt (pair x y)) = y) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (x y : V.Element),
      (cond V).toElementMap (pair botElt (pair x y)) = V.bot) :=
  ⟨@cond_true, @cond_false, @cond_bot⟩

/-- Scott's Exercise 3.26: sum-valued conditional and recovery via which. -/
theorem example_326_sum :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
```

```
    (x : V₀.Element) (y : V₁.Element),
    (condSum V₀ V₁ h₀ h₁).toElementMap (pair botElt (pair x y)) = (sum V₀ V₁ h₀ h₁).
  bot) ∧
  (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
    {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
    (x : V₀.Element) (y : V₁.Element),
    (condSum V₀ V₁ h₀ h₁).toElementMap (pair falseElt (pair x y))
      = (inMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap y) ∧
  (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
    {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
    (x : V₀.Element) (y : V₁.Element),
    (condSum V₀ V₁ h₀ h₁).toElementMap (pair trueElt (pair x y))
      = (inMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap x) ∧
  (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
    {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
    (x : (sum V₀ V₁ h₀ h₁).Element),
    (cond (sum V₀ V₁ h₀ h₁)).toElementMap
        (pair ((whichMap V₀ V₁ h₀ h₁).toElementMap x)
          (pair ((inMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap
                  ((outMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap x))
                ((inMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap
                  ((outMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap x)))) = x) :=
⟨@condSum_bot, @condSum_false, @condSum_true, @cond_which⟩
```

4.4.51 Function space via closure

Approximable maps as a directed-complete family of graphs, yielding a representation of the function space.

Definition (`graph`). The graph of an approximable map is the set of related pairs of neighbourhoods.

Proof. By definition. ■

Theorem (`mem_graph`). Membership in a graph is the relating relation.

Proof. By definition. ■

Definition (`C`). The family of graphs is the range of the graph map.

Proof. By definition. ■

Theorem (`C_nonempty`). The family of graphs is nonempty.

Proof. The constant map at bottom supplies a graph. ■

Theorem (`graph_injective`). Distinct approximable maps have distinct graphs.

Proof. Extensionality of relations. ■

Definition (`graphEquiv`). Approximable maps are order-isomorphic to their graphs as members of the family of graphs.

Proof. The round trip chooses a witnessing map; order equivalence is inclusion of graphs. ■

Definition (`joinMap`). The directed join of a nonempty directed family of graphs is the approximable map whose relation is the union of the graphs.

Proof. Filter axioms follow by directedness, picking a common upper graph. ■

Definition (`meetMap`). The meet of a nonempty family of graphs is the approximable map whose relation is the intersection of the graphs.

Proof. Filter axioms hold pointwise in the family. ■

Theorem (`sInter_eq_graph_meetMap`). The intersection of a nonempty family of graphs is the graph of the meet map.

Proof. Compare membership. ■

Theorem (`C_inter`). The family of graphs is closed under nonempty intersections.

Proof. The intersection is the graph of the meet. ■

Theorem (`sUnion_eq_graph_joinMap`). The union of a nonempty directed family of graphs is the graph of the join map.

Proof. Compare membership. ■

Theorem (`C_dir`). The family of graphs is closed under nonempty directed unions.

Proof. The union is the graph of the join. ■

Definition (`funSpaceReprIso`). Ideal elements of the representation system on the family of graphs are order-isomorphic to function-space elements.

Proof. Compose the representation isomorphism, the graph equivalence, and the function-space element isomorphism. ■

graphClosure.lean

```
module
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.reprSystem
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6
public import Mathlib.Order.Directed
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.GraphClosure
section
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

section
variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def graph (f : ApproximableMap V₀ V₁) : Set (Set α × Set β) := {p | f.rel p.1 p.2}

@[simp] theorem mem_graph {f : ApproximableMap V₀ V₁} {p : Set α × Set β} :
    p ∈ graph f ↔ f.rel p.1 p.2 := Iff.rfl

def C (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) : Set (Set (Set α × Set
    β)) :=
  Set.range (graph (V₀ := V₀) (V₁ := V₁))

theorem C_nonempty : (C V₀ V₁).Nonempty :=
  ⟨graph (constMap V₀ V₁.bot), constMap V₀ V₁.bot, rfl⟩

theorem graph_injective : Function.Injective (graph (V₀ := V₀) (V₁ := V₁)) :=
  fun _ _ h => ApproximableMap.ext fun X Y => Set.ext_iff.mp h (X, Y)

noncomputable def graphEquiv (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    ApproximableMap V₀ V₁ ≃o {X : Set (Set α × Set β) // X ∈ C V₀ V₁} where
  toFun f := ⟨graph f, f, rfl⟩
  invFun X := X.2.choose
  left_inv f := graph_injective (⟨graph f, f, rfl⟩ : {X // X ∈ C V₀ V₁}).2.choose_spec
  right_inv X := Subtype.ext X.2.choose_spec
  map_rel_iff' := by
    intro f g
    show graph f ⊆ graph g ↔ f ≤ g
    rw [ApproximableMap.le_iff]
```

```
    exact ⟨fun h X Y hf => h (a := (X, Y)) hf, fun h p hp => h p.1 p.2 hp⟩

def joinMap (? : Set (Set (Set α × Set β))) (h? : ? ⊆ C V₀ V₁) (hne : ?.Nonempty)
    (hdir : DirectedOn (· ⊆ ·) ?) : ApproximableMap V₀ V₁ where
  rel X Y := ∃ S ∈ ?, (X, Y) ∈ S
  rel_dom := by rintro X Y ⟨S, hS, hp⟩; obtain ⟨f, rfl⟩ := h? hS; exact f.rel_dom hp
  rel_cod := by rintro X Y ⟨S, hS, hp⟩; obtain ⟨f, rfl⟩ := h? hS; exact f.rel_cod hp
  master_rel := by
    obtain ⟨S, hS⟩ := hne; obtain ⟨f, rfl⟩ := h? hS
    exact ⟨graph f, hS, f.master_rel⟩
  inter_right := by
    rintro X Y Y' ⟨S, hS, hp⟩ ⟨S', hS', hp'⟩
    obtain ⟨S₃, hS₃, hSS₃, hS'S₃⟩ := hdir S hS S' hS'
    obtain ⟨f, rfl⟩ := h? hS₃
    exact ⟨graph f, hS₃, f.inter_right (hSS₃ hp) (hS'S₃ hp')⟩
  mono := by
    rintro X X' Y Y' ⟨S, hS, hp⟩ hX'X hYY' hX' hY'
    obtain ⟨f, rfl⟩ := h? hS
    exact ⟨graph f, hS, f.mono hp hX'X hYY' hX' hY'⟩

def meetMap (? : Set (Set (Set α × Set β))) (h? : ? ⊆ C V₀ V₁) (hne : ?.Nonempty) :
    ApproximableMap V₀ V₁ where
  rel X Y := ∀ S ∈ ?, (X, Y) ∈ S
  rel_dom := by
    intro X Y h
    obtain ⟨S, hS⟩ := hne; obtain ⟨f, rfl⟩ := h? hS
    exact f.rel_dom (h _ hS)
  rel_cod := by
    intro X Y h
    obtain ⟨S, hS⟩ := hne; obtain ⟨f, rfl⟩ := h? hS
    exact f.rel_cod (h _ hS)
  master_rel := by
    intro S hS; obtain ⟨f, rfl⟩ := h? hS; exact f.master_rel
  inter_right := by
    intro X Y Y' hY hY' S hS
    obtain ⟨f, rfl⟩ := h? hS
    exact f.inter_right (hY _ hS) (hY' _ hS)
  mono := by
    intro X X' Y Y' h hX'X hYY' hX' hY' S hS
    obtain ⟨f, rfl⟩ := h? hS
    exact f.mono (h _ hS) hX'X hYY' hX' hY'

theorem sInter_eq_graph_meetMap (? : Set (Set (Set α × Set β))) (h? : ? ⊆ C V₀ V₁)
    (hne : ?.Nonempty) : ⋂₀ ? = graph (meetMap ? h? hne) := by
  ext p
  simp only [Set.mem_sInter, mem_graph]
  exact ⟨fun h S hS => h S hS, fun h S hS => h S hS⟩

theorem C_inter : ∀ ? : Set (Set (Set α × Set β)), ?.Nonempty → ? ⊆ C V₀ V₁ → ⋂₀ ? ∈
    C V₀ V₁ := by
  intro ? hne h?
  rw [sInter_eq_graph_meetMap ? h? hne]
  exact ⟨meetMap ? h? hne, rfl⟩

theorem sUnion_eq_graph_joinMap (? : Set (Set (Set α × Set β))) (h? : ? ⊆ C V₀ V₁)
    (hne : ?.Nonempty) (hdir : DirectedOn (· ⊆ ·) ?) :
```

```
    ⋃₀ ? = graph (joinMap ? h? hne hdir) := by
  ext p
  simp only [Set.mem_sUnion, mem_graph]
  exact ⟨fun ⟨S, hS, hp⟩ => ⟨S, hS, hp⟩, fun ⟨S, hS, hp⟩ => ⟨S, hS, hp⟩⟩

theorem C_dir : ∀ ? : Set (Set (Set α × Set β)), ?.Nonempty → ? ⊆ C V₀ V₁ →
    DirectedOn (· ⊆ ·) ? → ⋃₀ ? ∈ C V₀ V₁ := by
  intro ? hne h? hdir
  rw [sUnion_eq_graph_joinMap ? h? hne hdir]
  exact ⟨joinMap ? h? hne hdir, rfl⟩

end

section
variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

noncomputable def funSpaceReprIso (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem
    β) :
    (ReprSystem.reprSystem (C V₀ V₁) C_inter C_nonempty).Element ≃o (funSpace V₀ V₁).
    Element :=
  (ReprSystem.reprIso (C V₀ V₁) C_inter C_nonempty C_dir).trans
    ((graphEquiv V₀ V₁).symm.trans (funSpaceEquiv V₀ V₁).symm)
end
end

end DomainTheory.NeighbourhoodSystems.GraphClosure
end
```

4.4.52 Exercise 3.27 — function space via closure

(For set theorists.) Give another proof that the family of approximable mappings $f : D_0 \to D_1$ is isomorphic to a domain by employing the general argument of Exercise 2.22. How does this compare with the proof method of 3.9 and 3.10? Can the general remarks also be employed to show that

$$\mathrm{eval} : (D_1 \to D_2) \times D_1 \to D_2$$

is approximable without bringing in the neighbourhoods in such an explicit way? (Hint: Use 3.5 and the idea of Exercise 2.12.)

The graph-closure representation is with the function space via closure peer.

Theorem (`example_327`). The family of graphs is closed under nonempty directed unions.

Proof. Invoke directed-union closure for the family of graphs. ■

`Exercise_3_27.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.graphClosure

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem
open DomainTheory.NeighbourhoodSystems.GraphClosure
```

```
/-- Scott's Exercise 3.27: graphs of approximable maps are closed under directed
    unions. -/
theorem example_327 :
    ∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (? : Set (Set (Set α × Set β))),
      ?.Nonempty → ? ⊆ C V₀ V₁ → DirectedOn (· ⊆ ·) ? → ⋃₀ ? ∈ C V₀ V₁ :=
  @C_dir
```

4.4.53 Minimal step function

Elementwise characterisation of the least map of a step list as a pointwise supremum of principals.

Definition (`leastMapVals`). At an ideal element, the value set of a step list is the set of principals of those right factors whose left factors contain the argument.

Proof. By definition. ■

Theorem (`Element.mem_interYs_of`). Membership of an ideal element in an intersection of range neighbourhoods is membership in the master together with membership in each active right factor.

Proof. Unpack the defining quantification. ■

Theorem (`leastMap_toElementMap_le_of_ub`). The least map of a consistent step list is below every map that relates each listed pair.

Proof. Bound the value via the relating neighbourhoods of the upper bound. ■

Theorem (`val_le_leastMap_toElementMap`). Each principal arising from an active right factor lies below the value of the least map.

Proof. The least map relates the corresponding left factor to that right factor. ■

Theorem (`leastMapVals_bounded`). Under consistency of the step list, the value set at each argument is bounded.

Proof. The value of the least map is a common upper bound. ■

Theorem (`toElementMap_leastMap_eq_sSup`). The value of the least map at an argument is the supremum of the corresponding value set.

Proof. The least map is an upper bound, and every upper bound of the step list dominates that supremum. ■

`leastStepMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
section
variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def leastMapVals (L : List (Set α × Set β)) (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2)
    (x : V₀.Element) : Set V₁.Element :=
  {z | ∃ (p : Set α × Set β) (hp : p ∈ L), x.mem p.1 ∧ z = V₁.principal (hL p hp).2}
```

```
theorem Element.mem_interYs_of (z : V₁.Element) {L : List (Set α × Set β)} {X : Set α}
    (hall : ∀ p ∈ L, X ⊆ p.1 → z.mem p.2) : z.mem (interYs V₁.master L X) := by
  induction L with
  | nil => exact z.master_mem
  | cons p L ih =>
    have htail : ∀ q ∈ L, X ⊆ q.1 → z.mem q.2 :=
      fun q hq => hall q (List.mem_cons.mpr (Or.inr hq))
    by_cases hXp : X ⊆ p.1
    · have heq : interYs V₁.master (p :: L) X = p.2 ∩ interYs V₁.master L X := by
        rw [interYs_cons]; ext w
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq]
        exact ⟨fun ⟨h1, h2⟩ => ⟨h1 hXp, h2⟩, fun ⟨h1, h2⟩ => ⟨fun _ => h1, h2⟩⟩
      rw [heq]
      exact z.inter_mem (hall p (List.mem_cons.mpr (Or.inl rfl)) hXp) (ih htail)
    · have heq : interYs V₁.master (p :: L) X = interYs V₁.master L X := by
        rw [interYs_cons]; ext w
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq]
        exact ⟨fun h => h.2, fun h => ⟨fun hc => absurd hc hXp, h⟩⟩
      rw [heq]
      exact ih htail

theorem leastMap_toElementMap_le_of_ub {L : List (Set α × Set β)}
    (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2)
    (hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X)) {x : V₀.Element} {z : V₁
    .Element}
    (hz : ∀ w ∈ leastMapVals L hL x, w ≤ z) :
    (leastMap L hL hcons).toElementMap x ≤ z := by
  rintro Y' ⟨X', hX'x, hX'mem, hY'mem, hsub⟩
  have hzint : z.mem (interYs V₁.master L X') := by
    apply Element.mem_interYs_of
    intro p hp hX'p
    have hp1x : x.mem p.1 := x.up_mem hX'x (hL p hp).1 hX'p
    have hle : V₁.principal (hL p hp).2 ≤ z := hz _ ⟨p, hp, hp1x, rfl⟩
    exact hle p.2 ⟨(hL p hp).2, subset_rfl⟩
  exact z.up_mem hzint hY'mem hsub

theorem val_le_leastMap_toElementMap {L : List (Set α × Set β)}
    (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2)
    (hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X)) {x : V₀.Element}
    {p : Set α × Set β} (hp : p ∈ L) (hp1x : x.mem p.1) :
    V₁.principal (hL p hp).2 ≤ (leastMap L hL hcons).toElementMap x := by
  have hmem : ((leastMap L hL hcons).toElementMap x).mem p.2 :=
    ⟨p.1, hp1x, leastMap_mem_stepFun hL hcons p hp⟩
  intro Z hZ
  exact ((leastMap L hL hcons).toElementMap x).up_mem hmem hZ.1 hZ.2

theorem leastMapVals_bounded {L : List (Set α × Set β)} (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁
    .mem p.2)
    (hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X)) (x : V₀.Element) :
    V₁.Bounded (leastMapVals L hL x) := by
  refine ⟨(leastMap L hL hcons).toElementMap x, ?_⟩
  rintro w ⟨p, hp, hp1x, rfl⟩
  exact val_le_leastMap_toElementMap hL hcons hp hp1x

theorem toElementMap_leastMap_eq_sSup (L : List (Set α × Set β))
```

```
    (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2)
    (hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X)) (x : V₀.Element) :
    (leastMap L hL hcons).toElementMap x
      = V₁.sSup (leastMapVals L hL x) (leastMapVals_bounded hL hcons x) := by
  apply le_antisymm
  · apply leastMap_toElementMap_le_of_ub hL hcons
    intro w hw
    exact V₁.le_sSup _ (leastMapVals_bounded hL hcons x) hw
  · apply V₁.sSup_le
    rintro w ⟨p, hp, hp1x, rfl⟩
    exact val_le_leastMap_toElementMap hL hcons hp hp1x
end

end

end DomainTheory.NeighbourhoodSystems
end
```

4.4.54 Exercise 3.28 — minimal step function

In the function space $(D_0 \to D_1)$ let

$$\bigcap\{[X_i, Y_i] \mid i < n\}$$

be a (non-empty) neighbourhood. In 3.9 the minimal element of this neighbourhood is characterized as a relation f_0. Show that as an elementwise mapping it can be defined by the formula

$$f_0(x) = \bigsqcup\{\uparrow Y_i \mid x \in [X_i]\},$$

for $x \in |D_0|$. Try to draw a picture of $|D_0|$ with neighbourhoods $[X_i]$ and the corresponding values of the function f_0.

The pointwise supremum characterisation is with the minimal step function peer.

Theorem (`example_328`). The value of the least map of a consistent step list at an argument is the supremum of the principals of the active right factors.

Proof. Invoke the pointwise-supremum identity for the least map. ■

`Exercise_3_28.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.leastStepMap

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 3.28: the least map acts by a pointwise supremum of principals.
    -/
theorem example_328 :
    ∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (L : List (Set α × Set β)) (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2)
```

```
    (hcons : ∀ {X}, V₀.mem X → V₁.mem (interYs V₁.master L X)) (x : V₀.Element),
    (leastMap L hL hcons).toElementMap x
      = V₁.sSup (leastMapVals L hL x) (leastMapVals_bounded hL hcons x) :=
@toElementMap_leastMap_eq_sSup
```

4.5 Fixed points and recursion

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

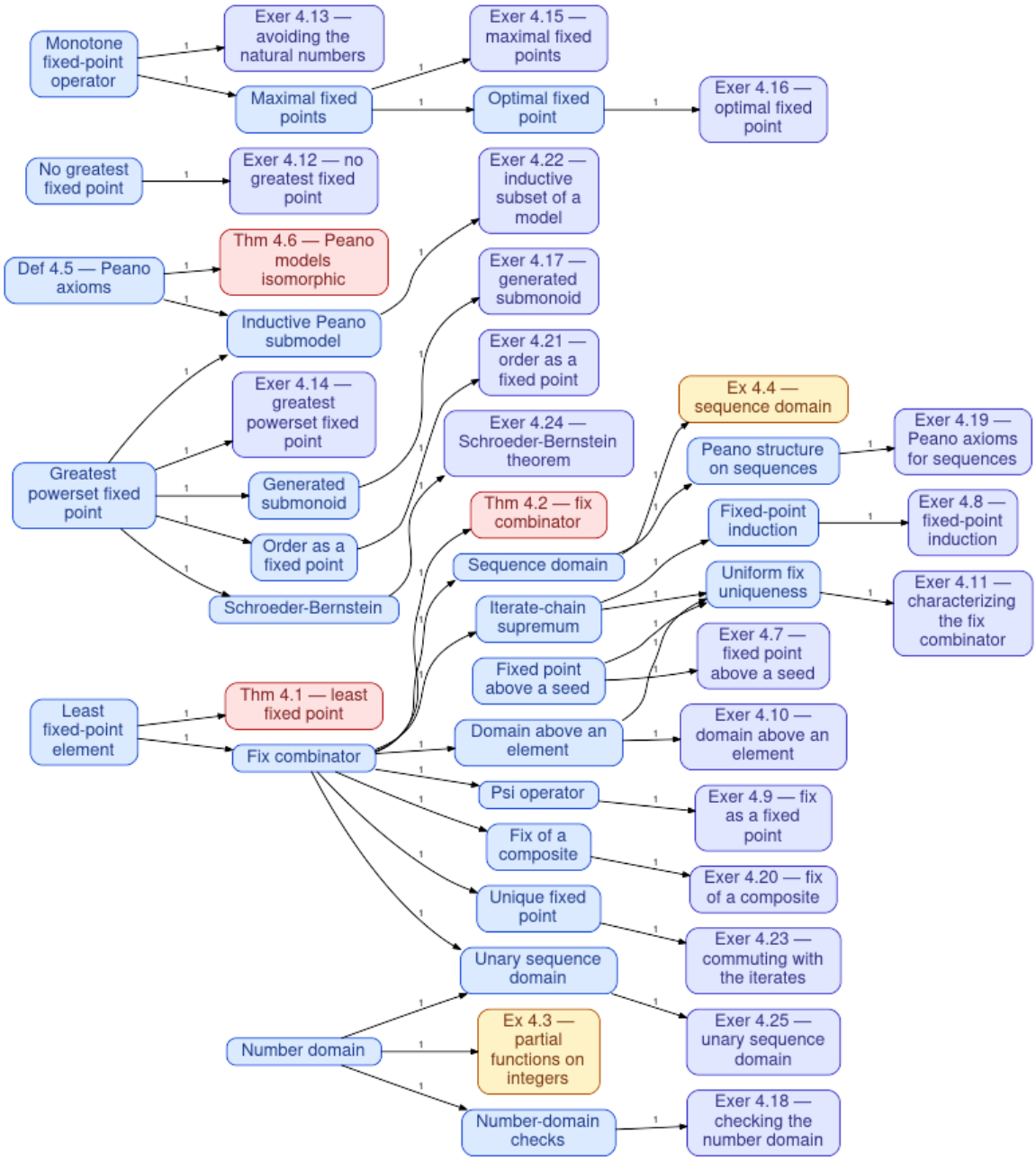


Figure 49: Diagram 14

4.5.1 Introduction

Having at this point a large supply of examples of domains (and further constructs of new domains), we now have to consider some other ways of defining functions — other than by explicit compositions of the very basic functions already mentioned. One of the most fruitful techniques is an infinitely iterated composition that is at the back of the idea of recursion. We will use the process over and over again in these lectures, not only to define new functions but also to define new domains. The heart of the matter lies in the so-called

"Fixed-point Theorem":

4.5.2 Least fixed-point element

Iterates of an endomap and the least fixed-point element they generate.

Definition (`iterMap`). The iterates of an approximable endomap are the identity at stage zero and composition of the map with the preceding iterate at each successor stage.

Proof. By recursion on the stage. ■

Theorem (`iterMap_zero`). The zero-th iterate is the identity map.

Proof. By definition. ■

Theorem (`iterMap_succ`). The successor iterate is the map composed with the preceding iterate.

Proof. By definition. ■

Theorem (`comp_mono`). Composition of approximable endomaps is monotone in each factor.

Proof. Unpack a composite relation into an intermediate neighbourhood and apply the two inequalities. ■

Theorem (`iterMap_mono_map`). Larger endomaps have larger iterates at every stage.

Proof. Induct on the stage, using monotonicity of composition at successor steps. ■

Theorem (`iter_comm`). An endomap commutes with each of its iterates under composition.

Proof. Induct on the stage, using the unit laws for the identity and associativity of composition. ■

Theorem (`rel_master_succ`). A neighbourhood related to the master by an iterate remains related after one further iterate.

Proof. Rewrite the successor iterate via the commutation law and insert the master-to-master relation of the map. ■

Theorem (`rel_master_mono`). Master-relatedness under iterates is preserved when the stage increases.

Proof. Induct along the inequality of stages, using the successor step. ■

Definition (`fixElement`). The least fixed-point element of an endomap consists of those neighbourhoods related to the master by some iterate of the map.

Proof. The master belongs via the identity iterate; upward closure uses monotonicity of the relation; intersection closure equalises stages by the master-relatedness monotonicity and closes under the right-intersection law of the iterate. ■

Theorem (`mem_fixElement`). Membership in the least fixed-point element is existence of a witnessing iterate relating the master to the neighbourhood.

Proof. By definition. ■

Theorem (`fixElement_mono`). Larger endomaps have larger least fixed-point elements.

Proof. Apply monotonicity of iterates stagewise. ■

Theorem (`fixElement_le_of_toElementMap_le`). If the map sends an ideal element into itself, then the least fixed-point element lies below that element.

Proof. Show by induction on the stage that every neighbourhood related to the master by an iterate belongs to the given upper bound, using the identity case for the master and the postfixedness hypothesis at successors. ■

Theorem (`toElementMap_fixElement`). The least fixed-point element is a fixed point of the endomap.

Proof. Compare membership: a neighbourhood reached by a positive iterate witnesses membership after applying the map, and conversely a successor witness unpacks to a related neighbourhood already in the fixed-point element; the zero stage recovers the master via the master relation. ■

fixElement.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.idMap
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.step

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α : Type*} {V : NeighborhoodSystem α}

namespace ApproximableMap

def iterMap (f : ApproximableMap V V) : ℕ → ApproximableMap V V
  | 0 => idMap V
  | (n + 1) => f.comp (f.iterMap n)

@[simp] theorem iterMap_zero (f : ApproximableMap V V) : f.iterMap 0 = idMap V := rfl

@[simp] theorem iterMap_succ (f : ApproximableMap V V) (n : ℕ) :
    f.iterMap (n + 1) = f.comp (f.iterMap n) := rfl

theorem comp_mono {f g a b : ApproximableMap V V} (hfg : f ≤ g) (hab : a ≤ b) :
    f.comp a ≤ g.comp b := by
  intro X Z h
  obtain ⟨Y, hXY, hYZ⟩ := h
  exact ⟨Y, hab X Y hXY, hfg Y Z hYZ⟩

theorem iterMap_mono_map {f g : ApproximableMap V V} (hfg : f ≤ g) (n : ℕ) :
    f.iterMap n ≤ g.iterMap n := by
  induction n with
  | zero => show (idMap V) ≤ (idMap V); exact le_refl _
  | succ k ih => exact comp_mono hfg ih

theorem iter_comm (f : ApproximableMap V V) (n : ℕ) :
    f.comp (f.iterMap n) = (f.iterMap n).comp f := by
  induction n with
  | zero => rw [show f.iterMap 0 = idMap V from rfl, comp_idMap, idMap_comp]
  | succ n ih =>
    show f.comp (f.comp (f.iterMap n)) = (f.comp (f.iterMap n)).comp f
    rw [comp_assoc, ← ih]

theorem rel_master_succ (f : ApproximableMap V V) {n : ℕ} {X : Set α}
    (h : (f.iterMap n).rel V.master X) : (f.iterMap (n + 1)).rel V.master X := by
  have hcomm : f.iterMap (n + 1) = (f.iterMap n).comp f := iter_comm f n
  rw [hcomm]
  exact ⟨V.master, f.master_rel, h⟩

theorem rel_master_mono (f : ApproximableMap V V) {n m : ℕ} (hnm : n ≤ m) {X : Set α}
    (h : (f.iterMap n).rel V.master X) : (f.iterMap m).rel V.master X := by
  induction hnm with
  | refl => exact h
  | step _ ih => exact rel_master_succ f ih
```

```
def fixElement (f : ApproximableMap V V) : V.Element where
  mem X := ∃ n, (f.iterMap n).rel V.master X
  sub := fun ⟨n, h⟩ => (f.iterMap n).rel_cod h
  master_mem := ⟨0, show (idMap V).rel V.master V.master from (idMap V).master_rel⟩
  inter_mem := by
    rintro X Y ⟨n, hn⟩ ⟨m, hm⟩
    refine ⟨max n m, ?_⟩
    have hX : (f.iterMap (max n m)).rel V.master X := rel_master_mono f (le_max_left n
     m) hn
    have hY : (f.iterMap (max n m)).rel V.master Y := rel_master_mono f (le_max_right
    n m) hm
    exact (f.iterMap (max n m)).inter_right hX hY
  up_mem := by
    rintro X Y ⟨n, hn⟩ hYmem hXY
    exact ⟨n, (f.iterMap n).mono hn subset_rfl hXY V.master_mem hYmem⟩

@[simp] theorem mem_fixElement (f : ApproximableMap V V) {X : Set α} :
    f.fixElement.mem X ↔ ∃ n, (f.iterMap n).rel V.master X := Iff.rfl

theorem fixElement_mono {f g : ApproximableMap V V} (hfg : f ≤ g) :
    f.fixElement ≤ g.fixElement := by
  rintro X ⟨n, hn⟩
  exact ⟨n, iterMap_mono_map hfg n V.master X hn⟩

theorem fixElement_le_of_toElementMap_le (f : ApproximableMap V V) {z : V.Element}
    (hz : f.toElementMap z ≤ z) : f.fixElement ≤ z := by
  have key : ∀ n X, (f.iterMap n).rel V.master X → z.mem X := by
    intro n
    induction n with
    | zero =>
      intro X hn
      obtain ⟨_, hXmem, hmX⟩ := hn
      have hXmaster : X = V.master := Set.Subset.antisymm (V.sub_master hXmem) hmX
      subst hXmaster
      exact z.master_mem
    | succ k ih =>
      intro X hn
      obtain ⟨W, hW, hWX⟩ := hn
      exact hz X ⟨W, ih W hW, hWX⟩
  rintro X ⟨n, hn⟩
  exact key n X hn

theorem toElementMap_fixElement (f : ApproximableMap V V) :
    f.toElementMap f.fixElement = f.fixElement := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨X, ⟨n, hn⟩, hXY⟩
    exact ⟨n + 1, ⟨X, hn, hXY⟩⟩
  · rintro ⟨n, hn⟩
    cases n with
    | zero =>
      obtain ⟨_, hYmem, hmY⟩ := hn
      have hYmaster : Y = V.master := Set.Subset.antisymm (V.sub_master hYmem) hmY
      subst hYmaster
```

```
      exact ⟨V.master, f.fixElement.master_mem, f.master_rel⟩
    | succ k =>
      obtain ⟨Z, hZ, hZY⟩ := hn
      exact ⟨Z, ⟨k, hZ⟩, hZY⟩

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

4.5.3 Theorem 4.1 — least fixed point

THEOREM 4.1. For any approximable mapping $f : D \to D$ on any domain, there exists a least element $x \in |D|$ where

$$f(x) = x.$$

Proof: Let f^n for $n \in \mathbf{N}$ stand for the n-fold composition of f with itself. That is,

$$f^0 = I_D, \text{ and}$$

$$f^{n+1} = f \circ f^n.$$

Define

$$x = \{X \in D \mid \Delta \; f^n \; X, \text{ for some } n \in \mathbf{N}\}.$$

We see $X \in x$ iff there is a finite sequence $\Delta = X_0, X_1, \ldots, X_n = X$ where $X_i \; f \; X_{i+1}$ holds for all $i < n$. Now since $\Delta \; f \; \Delta$ automatically holds, a sequence for an $X \in x$ can always be extended to a longer sequence just by adding more Δ's on the front.

We want to prove $x \in |D|$. Clearly $\Delta \in x$; and if $X \subseteq Y$ and $X \in x$, then $Y \in x$. All that remains to be shown is the closure of x under intersection. Note that if

$$U \; f \; V \quad \text{and} \quad U' \; f \; V'$$

hold and U, U' are consistent in D, then V and V' are consistent and
$(U \cap U') \; f \; (V \cap V')$
must hold. Generalizing this to sequences, if

$$\Delta = X_0 \; f \; X_1 \; f \; \cdots \; f \; X_n = X, \text{ and}$$

$$\Delta = Y_0 \; f \; Y_1 \; f \; \cdots \; f \; Y_n = Y$$

both hold (and note we have arranged the lengths of the two sequences to be equal), then each pair X_i, Y_i is consistent and we have

$$\Delta = (X_0 \cap Y_0) \; f \; (X_1 \cap Y_1) \; f \; \cdots \; f \; (X_n \cap Y_n) = X \cap Y.$$

This establishes the desired closure.

We also note that if $X \in x$ and $X \; f \; Y$ then $Y \in x$. Therefore, $f(x) \subseteq x$ and indeed by its very construction x is the *least* element of $|D|$ with this property. (Why?) But f is monotone, so $f(f(x)) \subseteq f(x)$; hence, $x = f(x)$. By what we have already said it must be the least such element. □

Because the element we have shown to exist in 4.1 is a least element, it is unique. That is, we have associated with each $f : D \to D$ a special element $x_f \in |D|$ determined by the choice of f. A function has therefore been defined mapping the set $|D \to D|$ into $|D|$. The next result shows that this function, or operator on functions, is in fact approximable.

The least fixed-point construction and its fixed-point / leastness laws are with the least fixed-point element peer.

Theorem (`theorem_41`). Every approximable endomap has a least fixed point: its least fixed-point element is fixed by the map and lies below every postfixed point.

Proof. Invoke the fixed-point law and the leastness law for the least fixed-point element. ■

`Theorem_4_1.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixElement

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Theorem 4.1: every endomap has a least fixed point. -/
theorem theorem_41 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : ApproximableMap V V),
      f.toElementMap f.fixElement = f.fixElement) ∧
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : ApproximableMap V V) {z : V.Element
    },
      f.toElementMap z ≤ z → f.fixElement ≤ z) :=
  ⟨@toElementMap_fixElement, @fixElement_le_of_toElementMap_le⟩

end DomainTheory.NeighbourhoodSystems
```

4.5.4 Fix combinator

The fix combinator as an approximable map on the function space, characterised as a directed supremum of iterates from bottom.

Definition (`fixMap`). The fix combinator sends each function-space neighbourhood to the least fixed-point element of its principal map, and extends by the monotone-extension principle to an approximable map from the function space into the domain.

Proof. Monotonicity of the assignment on principals follows from monotonicity of least fixed-point elements under the function-space order isomorphism. ■

Definition (`iterElem`). The n-th iterate element of an endomap is the elementwise action of the n-th iterate map on the bottom element.

Proof. By definition. ■

Theorem (`exists_principal_iterMap`). If an iterate of the recovered map relates the master to a neighbourhood, some neighbourhood of the function-space element has the same relation for the principal of that neighbourhood.

Proof. Induct on the stage: the master neighbourhood works at stage zero; at a successor, intersect the inductive neighbourhood with the witnessing step and compare iterates by monotonicity. ■

Theorem (`fixMap_toElementMap_principal`). On a principal function-space element, the fix combinator evaluates to the least fixed-point element of the recovered principal map.

Proof. Invoke the evaluation law for the monotone extension on principals. ■

Theorem (`fixMap_toElementMap`). On an arbitrary function-space element, the fix combinator evaluates to the least fixed-point element of the recovered map.

Proof. Compare membership via the principal-witness characterisation of the elementwise image, using monotonicity of least fixed-point elements one way and the principal-iterate existence lemma the other. ■

Theorem (`fixMap_least`). If the recovered map sends an ideal element into itself, the value of the fix combinator lies below that element.

Proof. Rewrite via the evaluation law and apply leastness of the least fixed-point element. ■

Theorem (`toElementMap_fixMap`). The value of the fix combinator is a fixed point of the recovered map.

Proof. Rewrite via the evaluation law and apply the fixed-point law for the least fixed-point element. ■

Theorem (`fixMap_toElementMap_toFilter`). On the principal filter of an endomap, the fix combinator recovers that map's least fixed-point element.

Proof. Rewrite via the evaluation law and cancel the round trip through the function-space isomorphism. ■

Theorem (`iterElem_eq_iterate`). Iterate elements agree with ordinary iteration of the elementwise map on the bottom element.

Proof. Induct on the stage, using the identity law at zero and the composition law for elementwise images at successors. ■

Theorem (`mem_iterElem`). Membership in an iterate element is master-relatedness under the corresponding iterate map.

Proof. Unpack the elementwise image of bottom, whose only neighbourhood is the master. ■

Theorem (`iterElem_mono`). Iterate elements increase with the stage.

Proof. Transport master-relatedness monotonicity of iterates through the membership characterisation. ■

Theorem (`fixElement_eq_iSupDirected`). The least fixed-point element is the directed supremum of the iterate elements.

Proof. Compare membership with the directed-supremum membership law via the iterate-element characterisation. ■

Theorem (`fixMap_eq_iSup`). The value of the fix combinator is the directed supremum of the iterate elements of the recovered map.

Proof. Rewrite via the evaluation law and apply the directed-supremum characterisation of the least fixed-point element. ■

Theorem (`fixMap_fixed`). The value of the fix combinator is a fixed point of the map recovered from each function-space element.

Proof. Rewrite via the evaluation law and apply the fixed-point law for the least fixed-point element of the recovered map. ■

Theorem (`fixMap_unique`). Any approximable map on the function space that is fixed-point and least on each principal agrees with the fix combinator.

Proof. Compare below using the fix combinator's fixed-point and leastness laws against the assumed laws pointwise on principals. ■

fixMap.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupDirected
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofMono
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixElement

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_10
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff
public import Mathlib.Tactic.Bound

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α : Type*} {V : NeighborhoodSystem α}

def fixMap (V : NeighborhoodSystem α) : ApproximableMap (funSpace V V) V :=
  ofMono (fun W hW => (toApproxMap ((funSpace V V).principal hW)).fixElement)
    (fun W W' hW hW' hW'W => by
      apply fixElement_mono
      exact (funSpaceEquiv V V).monotone
        (((funSpace V V).principal_le_iff hW hW').mpr hW'W))

namespace ApproximableMap

def iterElem (f : ApproximableMap V V) (n : ℕ) : V.Element := (f.iterMap n).
    toElementMap V.bot

end ApproximableMap

open ApproximableMap

theorem exists_principal_iterMap (V : NeighborhoodSystem α) (φ : (funSpace V V).
    Element) :
    ∀ (n : ℕ) (X : Set α), ((toApproxMap φ).iterMap n).rel V.master X →
      ∃ (W : Set (ApproximableMap V V)) (hw : φ.mem W),
        ((toApproxMap ((funSpace V V).principal (φ.sub hw))).iterMap n).rel V.master X
     := by
  intro n
  induction n with
  | zero =>
    intro X hX
    exact ⟨(funSpace V V).master, φ.master_mem, hX⟩
  | succ k ih =>
    intro X hX
    obtain ⟨Y, hY, hYX⟩ := hX
    obtain ⟨W₁, hw₁, hW₁⟩ := ih Y hY
    have hVY : V.mem Y := ((toApproxMap φ).iterMap k).rel_cod hY
    have hVX : V.mem X := (toApproxMap φ).rel_cod hYX
    have hw₂ : φ.mem (step Y X) := toApproxMap_rel.mp hYX
```

```
    have hwInter : φ.mem (W₁ ∩ step Y X) := φ.inter_mem hw₁ hw₂
    refine ⟨W₁ ∩ step Y X, hwInter, ?_⟩
    have hg₁g : toApproxMap ((funSpace V V).principal (φ.sub hw₁))
        ≤ toApproxMap ((funSpace V V).principal (φ.sub hwInter)) :=
      (funSpaceEquiv V V).monotone
        (((funSpace V V).principal_le_iff (φ.sub hw₁) (φ.sub hwInter)).mpr Set.
    inter_subset_left)
    have hYg : ((toApproxMap ((funSpace V V).principal (φ.sub hwInter))).iterMap k).
    rel V.master Y :=
      iterMap_mono_map hg₁g k V.master Y hW₁
    have hgYX : (toApproxMap ((funSpace V V).principal (φ.sub hwInter))).rel Y X := by
      show ((funSpace V V).principal (φ.sub hwInter)).mem (step Y X)
      exact ⟨step_mem hVY hVX, Set.inter_subset_right⟩
    exact ⟨Y, hYg, hgYX⟩

theorem fixMap_toElementMap_principal (V : NeighborhoodSystem α)
    {W : Set (ApproximableMap V V)} (hW : (funSpace V V).mem W) :
    (fixMap V).toElementMap ((funSpace V V).principal hW) =
      (toApproxMap ((funSpace V V).principal hW)).fixElement :=
  toElementMap_ofMono_principal _ _ W hW

theorem fixMap_toElementMap (V : NeighborhoodSystem α) (φ : (funSpace V V).Element) :
    (fixMap V).toElementMap φ = (toApproxMap φ).fixElement := by
  apply Element.ext
  intro X
  rw [toElementMap_mem_iff_principal]
  constructor
  · rintro ⟨W, hw, hmem⟩
    rw [fixMap_toElementMap_principal] at hmem
    have hle : (funSpace V V).principal (φ.sub hw) ≤ φ :=
      fun Z hZ => φ.up_mem hw hZ.1 hZ.2
    exact fixElement_mono ((funSpaceEquiv V V).monotone hle) X hmem
  · rintro ⟨n, hn⟩
    obtain ⟨W, hw, hWn⟩ := exists_principal_iterMap V φ n X hn
    refine ⟨W, hw, ?_⟩
    rw [fixMap_toElementMap_principal]
    exact ⟨n, hWn⟩

theorem fixMap_least (V : NeighborhoodSystem α) (φ : (funSpace V V).Element) {z : V.
    Element}
    (hz : (toApproxMap φ).toElementMap z ≤ z) : (fixMap V).toElementMap φ ≤ z := by
  rw [fixMap_toElementMap]
  exact fixElement_le_of_toElementMap_le (toApproxMap φ) hz

theorem toElementMap_fixMap (V : NeighborhoodSystem α) (φ : (funSpace V V).Element) :
    (toApproxMap φ).toElementMap ((fixMap V).toElementMap φ) = (fixMap V).toElementMap
    φ := by
  rw [fixMap_toElementMap]
  exact toElementMap_fixElement _

theorem fixMap_toElementMap_toFilter (V : NeighborhoodSystem α) (f : ApproximableMap V
    V) :
    (fixMap V).toElementMap (toFilter f) = f.fixElement := by
  rw [fixMap_toElementMap]
  have h : toApproxMap (toFilter f) = f := by
    have he := (funSpaceEquiv V V).apply_symm_apply f
```

```
    rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he
  rw [h]

namespace ApproximableMap

theorem iterElem_eq_iterate (f : ApproximableMap V V) (n : ℕ) :
    f.iterElem n = (f.toElementMap)^[n] V.bot := by
  induction n with
  | zero =>
    show (f.iterMap 0).toElementMap V.bot = V.bot
    exact toElementMap_idMap V.bot
  | succ k ih =>
    have hstep : f.iterElem (k + 1) = f.toElementMap (f.iterElem k) := by
      show (f.comp (f.iterMap k)).toElementMap V.bot
          = f.toElementMap ((f.iterMap k).toElementMap V.bot)
      rw [toElementMap_comp]
    rw [hstep, ih, Function.iterate_succ', Function.comp_apply]

theorem mem_iterElem (f : ApproximableMap V V) (n : ℕ) {X : Set α} :
    (f.iterElem n).mem X ↔ (f.iterMap n).rel V.master X := by
  constructor
  · rintro ⟨W, hW, hWX⟩
    rw [mem_bot] at hW; subst hW; exact hWX
  · intro h; exact ⟨V.master, by rw [mem_bot], h⟩

theorem iterElem_mono (f : ApproximableMap V V) {n m : ℕ} (hnm : n ≤ m) :
    f.iterElem n ≤ f.iterElem m := by
  intro X hX
  rw [mem_iterElem] at hX ⊢
  exact rel_master_mono f hnm hX

theorem fixElement_eq_iSupDirected (f : ApproximableMap V V) :
    f.fixElement =
      NeighborhoodSystem.iSupDirected (f.iterElem)
        (fun i j => ⟨max i j, iterElem_mono f (le_max_left i j),
          iterElem_mono f (le_max_right i j)⟩) := by
  apply Element.ext
  intro X
  rw [NeighborhoodSystem.mem_iSupDirected]
  constructor
  · rintro ⟨n, hn⟩; exact ⟨n, (mem_iterElem f n).mpr hn⟩
  · rintro ⟨n, hn⟩; exact ⟨n, (mem_iterElem f n).mp hn⟩

end ApproximableMap

open ApproximableMap

theorem fixMap_eq_iSup (V : NeighborhoodSystem α) (φ : (funSpace V V).Element) :
    (fixMap V).toElementMap φ =
      NeighborhoodSystem.iSupDirected ((toApproxMap φ).iterElem)
        (fun i j => ⟨max i j, iterElem_mono _ (le_max_left i j),
          iterElem_mono _ (le_max_right i j)⟩) := by
  rw [fixMap_toElementMap]
  exact fixElement_eq_iSupDirected (toApproxMap φ)

theorem fixMap_fixed (V : NeighborhoodSystem α) (φ : (funSpace V V).Element) :
```

```
    (toApproxMap φ).toElementMap ((fixMap V).toElementMap φ) = (fixMap V).toElementMap
    φ := by
  rw [fixMap_toElementMap]
  exact toElementMap_fixElement (toApproxMap φ)

theorem fixMap_unique (V : NeighborhoodSystem α) (fax : ApproximableMap (funSpace V V)
    V)
    (h_fix : ∀ φ, (toApproxMap φ).toElementMap (fax.toElementMap φ) = fax.toElementMap
    φ)
    (h_least : ∀ (φ : (funSpace V V).Element) (z : V.Element),
      (toApproxMap φ).toElementMap z ≤ z → fax.toElementMap φ ≤ z) :
    fax = fixMap V := by
  apply ext_of_toElementMap
  intro φ
  apply le_antisymm
  · exact h_least φ _ (le_of_eq (fixMap_fixed V φ))
  · exact fixMap_least V φ (le_of_eq (h_fix φ))

end DomainTheory.NeighbourhoodSystems
```

4.5.5 Theorem 4.2 — fix combinator

THEOREM 4.2. For any domain D, there is an approximable mapping

$$\mathrm{fix} : (D \to D) \to D$$

such that if $f : D \to D$ is any approximable mapping, then

(i) $\mathrm{fix}(f) = f(\mathrm{fix}(f))$.

Furthermore, if $x \in |D|$, then

(ii) $f(x) \subseteq x$ implies $\mathrm{fix}(f) \subseteq x$.

And this last property implies that fix is unique. Explicitly we can characterize fix by the equation:

(iii)

$$\mathrm{fix}(f) = \bigcup_{n=0}^{\infty} f^n(\bot),$$

for all $f : D \to D$.

Proof: Formula (iii) can be put in a more elementary form:

$$\mathrm{fix}(f) = \{X \mid \Delta f^n X, \text{ for some } n \in \mathbf{N}\}.$$

To show an elementwise mapping approximable we can use the formula of Exercise 2.9, applied to the above as the definition of fix:

$$(*) \quad \mathrm{fix}(f) = \bigcup\{\mathrm{fix}(\uparrow F) \mid f \in [F]\},$$

where F ranges over the neighbourhoods of $(D \to D)$, and where $\uparrow F$ can be considered to be the least element of F as calculated in 3.9.

Now from the definition of fix, it is clear that whenever $f \subseteq g$, then $\mathrm{fix}(f) \subseteq \mathrm{fix}(g)$, because $f^n \subseteq g^n$. (That is, fix is obviously monotone.) Next, if $f \in F$, then $\uparrow F$ is a (finite) approximation to f; so $\uparrow F \subseteq f$ and $\mathrm{fix}(\uparrow F) \subseteq \mathrm{fix}(f)$. This means that half of equation (*) already holds by monotonicity. All that is left is to prove the other half.

So suppose $X \in \mathrm{fix}(f)$. Then, as we have already remarked, there is a finite sequence of neighbourhoods where

$$\Delta = X_0 \; f \; X_1 \; \ldots \; X_{n-1} \; f \; X_n = X.$$

Let the function-space neighbourhood be defined as

$$F = \bigcap \{[X_i, X_{i+1}] \mid i < n\},$$

and note that since $f \in [F]$ we have at once consistency. But, by 3.9, $\uparrow F \in [F]$, so the *same* sequence of X_i is sufficient to show that

$$X \in \mathrm{fix}(\uparrow F).$$

In other words, if X belongs to the left-hand side of (*), it also belongs to the right-hand side. This completes the proof of (*).

Formula (i) is just a restatement of what we proved in 4.1. And (ii) follows easily, because $f(x) \subseteq x$ implies that $\Delta \in x$ and whenever $X \in x$ and $X \; f \; Y$, then $Y \in x$. Thus, by induction, if $\Delta f^n X$, then $X \in x$. So $\mathrm{fix}(f) \subseteq x$.

Finally, if $\mathrm{fax} : (D \to D) \to D$ were any other operator satisfying (i) and (ii), we would prove at once that

$$\mathrm{fix}(f) \subseteq \mathrm{fax}(f) \quad \text{and}$$

$$\mathrm{fax}(f) \subseteq \mathrm{fix}(f).$$

That is to say, the two operators are identical. □

The reader may have noticed that we used recursion in the proof of 4.1 (we had to define f^n for all $n \in \mathbf{N}$). But 4.1 and 4.2 can be used to justify definitions by recursion on a large number of domains — definitions where the process of iteration is far from being as straightforward. In discussing this point, let us start with some basic examples.

The fix combinator and its approximability / leastness / iterate-supremum laws are with the fix combinator peer.

Theorem (`theorem_42`). The fix combinator sends each function-space element to a fixed point of the recovered map, is least among postfixed points, and equals the directed supremum of the iterate elements from bottom.

Proof. Invoke the fixed-point law, the leastness law, and the directed-supremum law for the fix combinator. ■

`Theorem_4_2.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Theorem 4.2: the fix combinator is approximable, least, and an iterate
    supremum. -/
theorem theorem_42 :
    (∀ {α : Type*} (V : NeighborhoodSystem α) (φ : (funSpace V V).Element),
      (toApproxMap φ).toElementMap ((fixMap V).toElementMap φ) = (fixMap V).
    toElementMap φ) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (φ : (funSpace V V).Element) {z : V.
    Element},
      (toApproxMap φ).toElementMap z ≤ z → (fixMap V).toElementMap φ ≤ z) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (φ : (funSpace V V).Element),
      (fixMap V).toElementMap φ =
        NeighborhoodSystem.iSupDirected ((toApproxMap φ).iterElem)
          (fun i j => ⟨max i j, iterElem_mono _ (le_max_left i j),
            iterElem_mono _ (le_max_right i j)⟩)) :=
  ⟨@toElementMap_fixMap, @fixMap_least, @fixMap_eq_iSup⟩

end DomainTheory.NeighbourhoodSystems
```

4.5.6 Number domain

The flat neighbourhood system of natural numbers, with successor, predecessor, and zero-test as approximable maps.

Definition (`memN`). A subset of the naturals is a neighbourhood precisely when it is the universe or a singleton.

Proof. By definition. ■

Theorem (`memN_singleton`). Every singleton is a neighbourhood.

Proof. By definition. ■

Theorem (`memN_univ`). The universe is a neighbourhood.

Proof. By definition. ■

Theorem (`nestedOrDisjoint`). Any two neighbourhoods are nested or disjoint.

Proof. Case on the neighbourhood predicates: universals nest; a singleton nests in the universe; distinct singletons are disjoint. ■

Definition (`N`). The number domain is the neighbourhood system on the naturals generated by that nested-or-disjoint predicate with universe as master.

Proof. Assemble the nested-or-disjoint instance. ■

Theorem (`N_mem`). Membership in the number domain is the neighbourhood predicate.

Proof. By definition. ■

Theorem (`N_master`). The master neighbourhood is the universe.

Proof. By definition. ■

Definition (`T`). The two-token domain of truth values is the ambient truth domain.

Proof. By definition. ■

Definition (`natElem`). The numeral of a natural is the principal ideal of its singleton.

Proof. By definition. ■

Definition (`predVal`). The predecessor value sends zero to bottom and a successor numeral to the preceding numeral.

Proof. By recursion on the natural. ■

Theorem (`singleton_nat_inj`). Equal singletons determine equal naturals.

Proof. Membership of the left witness in the right singleton. ■

Theorem (`univ_ne_singleton`). The universe is not a singleton.

Proof. The successor of the singleton's element lies in the universe but not in the singleton. ■

Theorem (`singleton_ne_univ`). A singleton is not the universe.

Proof. Symmetry of the previous inequality. ■

Definition (`constLiftN`). A value assignment on naturals lifts to an approximable map from the number domain by relating the universe to the target master and each singleton to neighbourhoods of the assigned value.

Proof. Domain and codomain membership unpack the two clauses; the masters relate via the universe clause; right intersection and monotonicity case on those clauses using the singleton–universe inequalities. ■

Definition (`predMap`). Predecessor is the constant lift of the predecessor value.

Proof. By definition. ■

Definition (`succMap`). Successor is the constant lift of the successor-numeral assignment.

Proof. By definition. ■

Definition (`zeroElt`). Zero as an ideal element is the numeral of zero.

Proof. By definition. ■

Definition (`zeroVal`). The zero-test value sends zero to true and positive naturals to false.

Proof. By recursion on the natural. ■

Definition (`zeroMap`). The zero-test map is the constant lift of the zero-test value into the truth domain.

Proof. By definition. ■

Theorem (`N_bot_mem`). The bottom element of the number domain contains precisely the universe.

Proof. Invoke the general bottom-membership law. ■

Theorem (`constLiftN_bot`). A constant lift sends bottom to the target bottom.

Proof. Compare membership: a witnessing neighbourhood of bottom must be the universe, which forces the target master; conversely the target bottom is witnessed by the universe clause. ■

Theorem (`mem_natElem_iff`). A neighbourhood belongs to a numeral precisely when it is the universe or that numeral's singleton.

Proof. Unpack principal membership and the neighbourhood predicate. ■

Theorem (`constLiftN_natElem`). A constant lift evaluates at a numeral to the assigned value.

Proof. Compare membership using the numeral-membership characterisation and the singleton clause of the lift. ■

Theorem (`predMap_bot`). Predecessor sends bottom to bottom.

Proof. Invoke the constant-lift bottom law. ■

Theorem (`predMap_natElem_succ`). Predecessor of a successor numeral is the preceding numeral.

Proof. Invoke the constant-lift evaluation law. ■

Theorem (`predMap_natElem_zero`). Predecessor of the zero numeral is bottom.

Proof. Invoke the constant-lift evaluation law. ■

Theorem (`succMap_bot`). Successor sends bottom to bottom.

Proof. Invoke the constant-lift bottom law. ■

Theorem (`succMap_natElem`). Successor of a numeral is the next numeral.

Proof. Invoke the constant-lift evaluation law. ■

Theorem (`predMap_succMap_natElem`). Predecessor after successor recovers a numeral.

Proof. Compose the successor and predecessor evaluation laws. ■

Theorem (`zeroMap_bot`). The zero-test sends bottom to the truth-domain bottom.

Proof. Invoke the constant-lift bottom law. ■

Theorem (`zeroMap_natElem_succ`). The zero-test of a positive numeral is false.

Proof. Invoke the constant-lift evaluation law. ■

Theorem (`zeroMap_natElem_zero`). The zero-test of the zero numeral is true.

Proof. Invoke the constant-lift evaluation law. ■

`numberDomain.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Truth
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.NatDom
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

def memN (X : Set ℕ) : Prop := X = Set.univ ∨ ∃ n, X = {n}

theorem memN_singleton (n : ℕ) : memN ({n} : Set ℕ) := Or.inr ⟨n, rfl⟩

theorem memN_univ : memN (Set.univ : Set ℕ) := Or.inl rfl

theorem nestedOrDisjoint : NestedOrDisjoint memN := by
  rintro X Y (rfl | ⟨n, rfl⟩) (rfl | ⟨m, rfl⟩)
  · exact Or.inl (Set.Subset.refl _)
  · exact Or.inr (Or.inl (Set.subset_univ _))
  · exact Or.inl (Set.subset_univ _)
  · by_cases h : n = m
    · subst h; exact Or.inl (Set.Subset.refl _)
    · refine Or.inr (Or.inr ?_)
      ext k
      simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false,
        not_and]
      rintro rfl hkm
      exact h hkm

end DomainTheory.NeighbourhoodSystems.NatDom

namespace DomainTheory.NeighbourhoodSystems.NatDom
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

abbrev N : NeighborhoodSystem ℕ :=
  NeighborhoodSystem.ofNestedOrDisjoint memN Set.univ Set.univ_nonempty memN_univ
    nestedOrDisjoint
```

```
    (fun _ => Set.subset_univ _)

@[simp] theorem N_mem {X : Set ℕ} : N.mem X ↔ memN X := Iff.rfl

@[simp] theorem N_master : N.master = (Set.univ : Set ℕ) := rfl

abbrev T : NeighborhoodSystem TwoToken.Token := Truth.T

def natElem (n : ℕ) : N.Element := N.principal (N_mem.mpr (memN_singleton n))

def predVal : ℕ → N.Element
  | 0 => N.bot
  | (k + 1) => natElem k

theorem singleton_nat_inj {n m : ℕ} (h : ({n} : Set ℕ) = {m}) : n = m := by
  have h1 : n ∈ ({n} : Set ℕ) := rfl
  rw [h] at h1
  exact h1

theorem univ_ne_singleton (n : ℕ) : (Set.univ : Set ℕ) = {n} := by
  intro h
  have h1 : (n + 1) ∈ ({n} : Set ℕ) := h ▸ Set.mem_univ (n + 1)
  exact Nat.succ_ne_self n h1

theorem singleton_ne_univ (n : ℕ) : ({n} : Set ℕ) = Set.univ := fun h =>
    univ_ne_singleton n h.symm

def constLiftN {β : Type*} (V : NeighborhoodSystem β) (val : ℕ → V.Element) :
    ApproximableMap N V where
  rel X Y := (X = Set.univ ∧ Y = V.master) ∨ (∃ n, X = {n} ∧ (val n).mem Y)
  rel_dom := by
    rintro X Y (⟨rfl, _⟩ | ⟨n, rfl, _⟩)
    · exact memN_univ
    · exact memN_singleton n
  rel_cod := by
    rintro X Y (⟨_, rfl⟩ | ⟨n, _, hY⟩)
    · exact V.master_mem
    · exact (val n).sub hY
  master_rel := Or.inl ⟨rfl, rfl⟩
  inter_right := by
    rintro X Y Y' (⟨rfl, rfl⟩ | ⟨n, rfl, hY⟩) hc'
    · rcases hc' with ⟨_, rfl⟩ | ⟨n, hX, _⟩
      · exact Or.inl ⟨rfl, by rw [Set.inter_self]⟩
      · exact (univ_ne_singleton n hX).elim
    · rcases hc' with ⟨hX, _⟩ | ⟨n', hX', hY'⟩
      · exact (singleton_ne_univ n hX).elim
      · have hnn' : n = n' := singleton_nat_inj hX'
        subst hnn'
        exact Or.inr ⟨n, rfl, (val n).inter_mem hY hY'⟩
  mono := by
    rintro X X' Y Y' (⟨rfl, rfl⟩ | ⟨n, rfl, hY⟩) hX'X hYY' hX' hY'
    · have hY'master : Y' = V.master := Set.Subset.antisymm (V.sub_master hY') hYY'
      subst hY'master
      rcases hX' with rfl | ⟨k, rfl⟩
      · exact Or.inl ⟨rfl, rfl⟩
      · exact Or.inr ⟨k, rfl, (val k).master_mem⟩
```

```
    · rcases hX' with rfl | ⟨k, rfl⟩
      · exact (singleton_ne_univ n (Set.univ_subset_iff.mp hX'X)).elim
      · have hkn : k = n := Set.mem_singleton_iff.mp (Set.singleton_subset_iff.mp hX'X)
        subst hkn
        exact Or.inr ⟨k, rfl, (val k).up_mem hY hY' hYY'⟩

def predMap : ApproximableMap N N := constLiftN N predVal

def succMap : ApproximableMap N N := constLiftN N (fun n => natElem (n + 1))

def zeroElt : N.Element := natElem 0

def zeroVal : ℕ → T.Element
  | 0 => Truth.trueElt
  | (_ + 1) => Truth.falseElt

def zeroMap : ApproximableMap N T := constLiftN T zeroVal

theorem N_bot_mem {X : Set ℕ} : N.bot.mem X ↔ X = Set.univ := NeighborhoodSystem.
    mem_bot N

theorem constLiftN_bot {β : Type*} (V : NeighborhoodSystem β) (val : ℕ → V.Element) :
    (constLiftN V val).toElementMap N.bot = V.bot := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨X, hXmem, hrel⟩
    rw [N_bot_mem] at hXmem
    subst hXmem
    rcases hrel with ⟨_, rfl⟩ | ⟨k, hXk, _⟩
    · exact V.mem_bot.mpr rfl
    · exact (univ_ne_singleton k hXk).elim
  · intro hY
    rw [V.mem_bot] at hY
    subst hY
    exact ⟨Set.univ, N_bot_mem.mpr rfl, Or.inl ⟨rfl, rfl⟩⟩

theorem mem_natElem_iff {n : ℕ} {Y : Set ℕ} :
    (natElem n).mem Y ↔ Y = Set.univ ∨ Y = {n} := by
  unfold natElem
  rw [mem_principal]
  constructor
  · rintro ⟨hY, hsub⟩
    rcases hY with rfl | ⟨m, rfl⟩
    · exact Or.inl rfl
    · have hnm : n = m := Set.mem_singleton_iff.mp (Set.singleton_subset_iff.mp hsub)
      subst hnm; exact Or.inr rfl
  · rintro (rfl | rfl)
    · exact ⟨memN_univ, Set.subset_univ _⟩
    · exact ⟨memN_singleton n, subset_rfl⟩

theorem constLiftN_natElem {β : Type*} (V : NeighborhoodSystem β) (val : ℕ → V.
    Element) (n : ℕ) :
    (constLiftN V val).toElementMap (natElem n) = val n := by
  apply Element.ext
  intro Y
```

```
  constructor
  · rintro ⟨X, hXmem, hrel⟩
    rw [mem_natElem_iff] at hXmem
    rcases hrel with ⟨_, rfl⟩ | ⟨k, hXk, hY⟩
    · exact (val n).master_mem
    · rcases hXmem with rfl | rfl
      · exact (univ_ne_singleton k hXk).elim
      · have hnk : n = k := singleton_nat_inj hXk
        subst hnk; exact hY
  · intro hY
    exact ⟨{n}, mem_natElem_iff.mpr (Or.inr rfl), Or.inr ⟨n, rfl, hY⟩⟩

theorem predMap_bot : predMap.toElementMap N.bot = N.bot :=
  constLiftN_bot N predVal

theorem predMap_natElem_succ (k : ℕ) : predMap.toElementMap (natElem (k + 1)) =
    natElem k :=
  constLiftN_natElem N predVal (k + 1)

theorem predMap_natElem_zero : predMap.toElementMap (natElem 0) = N.bot :=
  constLiftN_natElem N predVal 0

theorem succMap_bot : succMap.toElementMap N.bot = N.bot :=
  constLiftN_bot N (fun n => natElem (n + 1))

theorem succMap_natElem (n : ℕ) : succMap.toElementMap (natElem n) = natElem (n + 1)
    :=
  constLiftN_natElem N (fun n => natElem (n + 1)) n

theorem predMap_succMap_natElem (n : ℕ) :
    predMap.toElementMap (succMap.toElementMap (natElem n)) = natElem n := by
  rw [succMap_natElem, predMap_natElem_succ]

theorem zeroMap_bot : zeroMap.toElementMap N.bot = T.bot :=
  constLiftN_bot T zeroVal

theorem zeroMap_natElem_succ (n : ℕ) : zeroMap.toElementMap (natElem (n + 1)) = Truth.
    falseElt :=
  constLiftN_natElem T zeroVal (n + 1)

theorem zeroMap_natElem_zero : zeroMap.toElementMap (natElem 0) = Truth.trueElt :=
  constLiftN_natElem T zeroVal 0

end DomainTheory.NeighbourhoodSystems.NatDom
```

4.5.7 Example 4.3 — partial functions on integers

EXAMPLE 4.3. The infinite generalization of our original example 1.2 is the system

$$N = \{\{n\} \mid n \in \mathbf{N}\} \cup \{\mathbf{N}\}.$$

The total elements are clearly in a one-one correspondence with the integers in $\mathbf{N}$. We can apply the construction of Exercise 3.16 to obtain a domain

$$F = N^{\infty}.$$

So we already know quite a bit about this domain — but it has a much more familiar presentation.

Let Φ be the set of all *finite partial functions* $\varphi \subseteq \mathbf{N} \times \mathbf{N}$ (that is, finite sets of ordered pairs of integers where, if $(n, m) \in \varphi$ and $(n, m') \in \varphi$, then $m = m'$). Define

$$\uparrow \varphi = \{\psi \in \Phi \mid \varphi \subseteq \psi\}.$$

Consider the neighbourhood system

$$F' = \{\uparrow \varphi \mid \varphi \in \Phi\}.$$

It is an easy exercise to show that F and F' are isomorphic and that the elements of these domains correspond exactly to the (possibly infinite) *partial functions* $\pi \subseteq \mathbf{N} \times \mathbf{N}$. Moreover, the *total* elements just correspond to the *total* functions $\tau : \mathbf{N} \to \mathbf{N}$ ("function" in the ordinary, set-theoretical sense of the word).

Another easy exercise is to show that the domains

$$F \text{ and } (\mathbf{N} \to \mathbf{N})$$

by our definitions are *NOT* isomorphic; though the two domains are closely related. We can define a mapping

$$\mathrm{val} : F \times N \to N$$

by the relationship

$$\uparrow \phi \cup \{n\} \text{ val } \{m\} \quad \text{iff} \quad (n, m) \in \phi.$$

(Of course val has to relate other neighbourhoods such as:

$$\uparrow \phi \cup N \text{ val } N,$$

but these are all.) It is then simple to prove that if $\pi \in |F|$ is regarded as a partial function $\pi : N \to N$ and if for $n \in N$ we define $\hat{n} \in |N|$ by

$$\hat{n} = \{\{n\}, N\},$$

then we have

$$\mathrm{val}(\pi, \hat{n}) = \widehat{\pi(n)}, \text{ if } \pi \text{ is defined at } n;$$

$$= \{N\}, \text{ otherwise.}$$

(Remember that $\{N\} \in |N|$ is the "undefined" element.) This means that

$$\mathrm{curry}(\mathrm{val}) : F \to (N \to N)$$

is a one-one function on elements. (The rather slight trouble with $(N \to N)$ is that it has *more* elements than F.)

So much for the construction of F, we now wish to consider mappings

$$f : F \to F$$

and their uses. Consider the possibility

$$f(\pi)(n) = 0, \qquad \text{if } n = 0;$$

$$= \pi(n-1) + n - 1, \text{ if } n > 0.$$

If π were a total function, then $f(\pi)$ would be total. But if π is partial, and if it is, say, undefined at k, then $f(\pi)$ becomes undefined at $k+1$. Note that $f(\pi)$ is always defined at 0. Note, too, that f is an approximable mapping because it is completely determined by what it does to finite (partial) functions. Indeed,

$$f(\pi) = \bigcup\{f(\phi) \mid \phi \subseteq \pi\},$$

where ϕ ranges over Φ.

Well, we have proved that every approximable map of a domain into itself has a (*least*) fixed point. What is the least fixed point of this f? Suppose $\sigma = f(\sigma)$. Then $\sigma(0) = 0$, and

$$\sigma(n+1) = f(\sigma)(n+1)$$

$$= \sigma(n) + n.$$

By induction, then

$$\sigma(n) = \sum_{i<n} i$$

and σ is a total function. (Therefore, f has a *unique* fixed point.)

Actually, we can make the procedure more systematic by defining as fixed points elements of $(N \to N)$ rather than F. In the first place we have $\hat{0} \in |N|$, and from now on we will not distinguish between n and $\hat{n}$. Next we have two mappings:

$$\text{succ}, \text{pred} : N \to N$$

where, as approximable mappings we have

$$X \text{ succ } Y \text{ iff } \exists n \in N.\ n \in X \text{ and } n+1 \in Y,$$

$$X \text{ pred } Y \text{ iff } \exists n \in N.\ n+1 \in X \text{ and } n \in Y,$$

for all $X, Y \in N$. This is *correct*, but what we mean in more understandable terms is:

$$\text{succ}(n) = n + 1;$$

$$\text{pred}(n) = n - 1, \text{ if } n > 0;$$

$$= \perp, \text{ if } n = 0.$$

Here, n has been identified with $\hat{n} \in |N|$ and $\perp = \{N\} \in |N|$. Moreover, we have a mapping

$$\text{zero} : N \to T$$

which is such that

$$\text{zero}(n) = \text{true, if } n = 0;$$

$$= \text{false, if } n > 0.$$

The *structured domain*

$$\langle N, 0, \text{succ}, \text{pred}, \text{zero} \rangle$$

can be called "THE domain of integers" for our present theory. We shall meet many other structured domains in the sequel.

Now the iterated summation function σ can be completely characterized — as a map $\sigma : N \to N$ rather than as an element $\sigma \in |F|$ — by the following equation:

$$\sigma(n) = \text{cond}(\text{zero}(n),\ 0,\ \sigma(\text{pred}(n)) + \text{pred}(n)).$$

The only problem is that we have not defined $+ : N \times N \to N$. (A direct definition is left to the reader; general remarks are given later.) But $+$ could be any function of two variables in order to make the point about the form of the definition of σ. Remember

$$\text{cond} : T \times N \times N \to N,$$

as defined in Exercise 3.26. We do not put cond in as part of the structure of N because (as should be clear from 3.26) it is part of the structure of T.

The above equation for σ is properly called a *functional equation*; it will be written as a fixed-point equation in Lecture V when we have the notation for the λ-calculus. □

The flat number system and its successor / predecessor / zero-test maps are with the number domain peer.

Theorem (`example_43`). On numerals, successor advances the index, predecessor recovers the predecessor or bottom at zero, both maps send bottom to bottom, the zero-test distinguishes zero from positive numerals and sends bottom to the truth-domain bottom, and predecessor after successor is the identity on numerals.

Proof. Invoke the corresponding evaluation and bottom laws for successor, predecessor, and the zero-test. ■

Example_4_3.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.numberDomain

@[expose] public section
```

```
open DomainTheory.NeighbourhoodSystems
open NatDom ApproximableMap

/-- Scott's Example 4.3: the structured number domain with succ, pred, and zero. -/
theorem example_43 :
    (∀ n : ℕ, succMap.toElementMap (natElem n) = natElem (n + 1)) ∧
    (∀ k : ℕ, predMap.toElementMap (natElem (k + 1)) = natElem k) ∧
    (predMap.toElementMap (natElem 0) = N.bot) ∧
    (succMap.toElementMap N.bot = N.bot) ∧
    (predMap.toElementMap N.bot = N.bot) ∧
    (zeroMap.toElementMap (natElem 0) = Truth.trueElt) ∧
    (∀ n : ℕ, zeroMap.toElementMap (natElem (n + 1)) = Truth.falseElt) ∧
    (zeroMap.toElementMap N.bot = T.bot) ∧
    (∀ n : ℕ, predMap.toElementMap (succMap.toElementMap (natElem n)) = natElem n) :=
  ⟨@succMap_natElem, @predMap_natElem_succ, predMap_natElem_zero,
   succMap_bot, predMap_bot, zeroMap_natElem_zero, @zeroMap_natElem_succ, zeroMap_bot,
   @predMap_succMap_natElem⟩
```

4.5.8 Sequence domain

The neighbourhood system of finite or infinite binary sequences, with bit-cons maps and the alternating fixed-point element.

Definition (`memC`). A set of strings is a sequence neighbourhood precisely when it is a cone or a singleton.

Proof. By definition. ■

Theorem (`memC_cone`). Every cone is a sequence neighbourhood.

Proof. By definition. ■

Theorem (`memC_singleton`). Every singleton is a sequence neighbourhood.

Proof. By definition. ■

Theorem (`prepend_mono`). Prepending a fixed string preserves inclusion of sets of strings.

Proof. Unpack membership in the prepended set and apply the given inclusion to the suffix. ■

Theorem (`prepend_singleton`). Prepending a string to a singleton yields the singleton of the concatenated string.

Proof. Compare membership in both directions. ■

Theorem (`singleton_subset_cone`). A singleton of a string lies in a cone precisely when the cone's prefix is a prefix of that string.

Proof. Unpack singleton inclusion and cone membership. ■

Theorem (`singleton_cone_nd`). A singleton and a cone are nested or disjoint: either the singleton lies in the cone, or the cone lies in the singleton, or their intersection is empty.

Proof. Case on whether the cone prefix is a prefix of the singleton's string; otherwise no common element can exist. ■

Theorem (`nestedOrDisjoint`). Any two sequence neighbourhoods are nested or disjoint.

Proof. Case on cones versus singletons, invoking cone trichotomy or the singleton–cone law, and for two singletons either they coincide or they are disjoint. ■

Definition (`C`). The sequence domain is the neighbourhood system on strings generated by that nested-or-disjoint predicate with the universe as master.

Proof. Assemble the nested-or-disjoint instance with the empty-string cone as a neighbourhood. ■

Theorem (`C_mem`). Membership in the sequence domain is the sequence-neighbourhood predicate.

Proof. By definition. ■

Theorem (`C_master`). The master neighbourhood is the universe of strings.

Proof. By definition. ■

Theorem (`memC_prepend`). Prepending a fixed string to a sequence neighbourhood yields a sequence neighbourhood.

Proof. Case on cones versus singletons and use the prepend–cone and prepend–singleton identities. ■

Definition (`consMap`). Bit-cons for a boolean relates neighbourhoods when the bit-prepended domain neighbourhood is contained in the codomain neighbourhood.

Proof. Domain and codomain membership are the first two conjuncts; the masters relate via the empty cone; right intersection and monotonicity use prepend monotonicity and that prepending preserves neighbourhoods. ■

Definition (`altElt`). The alternating element is the least fixed point of false-cons after true-cons.

Proof. By definition. ■

Definition (`strBot`). The partial string element of a finite string is the principal ideal of its cone.

Proof. By definition. ■

Definition (`strElem`). The total string element of a finite string is the principal ideal of its singleton.

Proof. By definition. ■

Theorem (`altElt_eq`). False-cons of true-cons of the alternating element recovers the alternating element.

Proof. Invoke the least-fixed-point law for the composite and unpack composition of element maps. ■

Theorem (`consMap_strBot`). Bit-cons prepends the bit on partial string elements.

Proof. Compare membership: a witnessing neighbourhood of the source cone forces the bit-prepended cone into the target; conversely that cone witnesses the relation. ■

Theorem (`consMap_strElem`). Bit-cons prepends the bit on total string elements.

Proof. Compare membership as for partial strings, using the prepend–singleton identity. ■

Theorem (`prepend_subset_self`). The trivial assertion holds.

Proof. Immediate. ■

`seqDomain.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ExampleB
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.SeqDom
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap BitDom

def memC (X : Set Str) : Prop := (∃ σ, X = cone σ) ∨ (∃ σ, X = {σ})

theorem memC_cone (σ : Str) : memC (cone σ) := Or.inl ⟨σ, rfl⟩

theorem memC_singleton (σ : Str) : memC ({σ} : Set Str) := Or.inr ⟨σ, rfl⟩
```

```
theorem prepend_mono (σ : Str) {X X' : Set Str} (h : X' ⊆ X) : prepend σ X' ⊆ prepend
    σ X := by
  rintro w ⟨τ, hτ, rfl⟩
  exact ⟨τ, h hτ, rfl⟩

theorem prepend_singleton (σ τ : Str) : prepend σ {τ} = {σ ++ τ} := by
  ext w
  simp only [mem_prepend, Set.mem_singleton_iff]
  constructor
  · rintro ⟨t, rfl, rfl⟩; rfl
  · rintro rfl; exact ⟨τ, rfl, rfl⟩

theorem singleton_subset_cone {σ τ : Str} : ({τ} : Set Str) ⊆ cone σ ↔ σ <+: τ := by
  rw [Set.singleton_subset_iff, cone.mem]

theorem singleton_cone_nd (σ τ : Str) :
    ({τ} : Set Str) ⊆ cone σ ∨ cone σ ⊆ {τ} ∨ ({τ} : Set Str) ∩ cone σ = ∅ := by
  by_cases h : σ <+: τ
  · exact Or.inl (singleton_subset_cone.mpr h)
  · refine Or.inr (Or.inr ?_)
    ext w
    simp only [Set.mem_inter_iff, Set.mem_singleton_iff, cone.mem, Set.
    mem_empty_iff_false,
      iff_false, not_and]
    rintro rfl hτ
    exact h hτ

theorem nestedOrDisjoint : NestedOrDisjoint memC := by
  rintro X Y (⟨σ, rfl⟩ | ⟨σ, rfl⟩) (⟨τ, rfl⟩ | ⟨τ, rfl⟩)
  · exact cone.trichotomy σ τ
  · rcases singleton_cone_nd σ τ with h | h | h
    · exact Or.inr (Or.inl h)
    · exact Or.inl h
    · exact Or.inr (Or.inr (by rw [Set.inter_comm]; exact h))
  · rcases singleton_cone_nd τ σ with h | h | h
    · exact Or.inl h
    · exact Or.inr (Or.inl h)
    · exact Or.inr (Or.inr h)
  · by_cases h : σ = τ
    · subst h; exact Or.inl (Set.Subset.refl _)
    · refine Or.inr (Or.inr ?_)
      ext w
      simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false,
        not_and]
      rintro rfl h2
      exact h h2

end DomainTheory.NeighbourhoodSystems.SeqDom

namespace DomainTheory.NeighbourhoodSystems.SeqDom
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap BitDom

abbrev C : NeighborhoodSystem Str :=
  NeighborhoodSystem.ofNestedOrDisjoint memC Set.univ
    ⟨[], Set.mem_univ _⟩ (Or.inl ⟨[], cone.nil.symm⟩) nestedOrDisjoint
```

```
    (fun _ => Set.subset_univ _)

@[simp] theorem C_mem {X : Set Str} : C.mem X ↔ memC X := Iff.rfl

@[simp] theorem C_master : C.master = (Set.univ : Set Str) := rfl

theorem memC_prepend (σ : Str) {X : Set Str} (hX : memC X) : memC (prepend σ X) := by
  rcases hX with ⟨?, rfl⟩ | ⟨?, rfl⟩
  · exact Or.inl ⟨σ ++ ?, prepend_cone σ ?⟩
  · exact Or.inr ⟨σ ++ ?, prepend_singleton σ ?⟩

def consMap (b : Bool) : ApproximableMap C C where
  rel X Y := memC X ∧ memC Y ∧ prepend [b] X ⊆ Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine ⟨Or.inl ⟨[], cone.nil.symm⟩, Or.inl ⟨[], cone.nil.symm⟩, ?_⟩
    exact Set.subset_univ _
  inter_right := by
    rintro X Y Y' ⟨hX, hY, hsub⟩ ⟨_, hY', hsub'⟩
    have hsubInter : prepend [b] X ⊆ Y ∩ Y' := Set.subset_inter hsub hsub'
    have hZ : memC (prepend [b] X) := memC_prepend [b] hX
    exact ⟨hX, C.inter_mem hY hY' hZ hsubInter, hsubInter⟩
  mono := by
    rintro X X' Y Y' ⟨hX, hY, hsub⟩ hX'X hYY' hX' hY'
    exact ⟨hX', hY', (prepend_mono [b] hX'X).trans (hsub.trans hYY')⟩

def altElt : C.Element := ((consMap false).comp (consMap true)).fixElement

def strBot (σ : Str) : C.Element := C.principal (C_mem.mpr (memC_cone σ))

def strElem (σ : Str) : C.Element := C.principal (C_mem.mpr (memC_singleton σ))

theorem altElt_eq : (consMap false).toElementMap ((consMap true).toElementMap altElt)
    = altElt := by
  have h := toElementMap_fixElement ((consMap false).comp (consMap true))
  rwa [toElementMap_comp] at h

theorem consMap_strBot (b : Bool) (σ : Str) :
    (consMap b).toElementMap (strBot σ) = strBot (b :: σ) := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨X', ⟨_, hXX'⟩, _, hY, hsub⟩
    refine ⟨hY, ?_⟩
    have hpre : prepend [b] (cone σ) ⊆ Y := (prepend_mono [b] hXX').trans hsub
    rwa [prepend_cone] at hpre
  · rintro ⟨hY, hsub⟩
    refine ⟨cone σ, ⟨memC_cone σ, subset_rfl⟩, memC_cone σ, hY, ?_⟩
    rw [prepend_cone]; exact hsub

theorem consMap_strElem (b : Bool) (σ : Str) :
    (consMap b).toElementMap (strElem σ) = strElem (b :: σ) := by
  apply Element.ext
  intro Y
  constructor
```

```
  · rintro ⟨X', ⟨_, hXX'⟩, _, hY, hsub⟩
    refine ⟨hY, ?_⟩
    have hpre : prepend [b] {σ} ⊆ Y := (prepend_mono [b] hXX').trans hsub
    rwa [prepend_singleton] at hpre
  · rintro ⟨hY, hsub⟩
    refine ⟨{σ}, ⟨memC_singleton σ, subset_rfl⟩, memC_singleton σ, hY, ?_⟩
    rw [prepend_singleton]; exact hsub

theorem prepend_subset_self : True := trivial

end DomainTheory.NeighbourhoodSystems.SeqDom
```

4.5.9 Example 4.4 — sequence domain

EXAMPLE 4.4. The domain C of finite or infinite binary sequences mentioned in Exercise 2.21 may be regarded as a generalization of N. This can be made plain by saying how we wish to regard C as a structured domain. To do this we should recall what C is as a neighbourhood system. In the first place

$$B = \{\sigma\Sigma^* \mid \sigma \in \Sigma^*\}$$

where $\Sigma = \{0, 1\}$. To form the system C we have

$$C = B \cup \{\{\sigma\} \mid \sigma \in \Sigma^*\}.$$

The total elements of B correspond to *infinite binary sequences*; while the total elements of C to *finite or infinite sequences*. To simplify notation let us write for $\sigma \in \Sigma^*$

$$\sigma = \uparrow \{\sigma\} \quad \text{(a total element);}$$

$$\sigma \perp = \uparrow \sigma\Sigma^* \quad \text{(a partial element).}$$

In other words we identify σ with the corresponding total element in $|C|$.

We wish now to think of C as a structured domain seen as a kind of generalization of N. The empty sequence Λ will play the rôle of $0 \in |N|$; the map succ has two different analogues for C, however. Just as for B we define for $x \in |C|$ and $\sigma \in \Sigma^*$:

$$\sigma x = \{Y \mid \sigma X \subseteq Y \text{ some } X \in x\},$$

where of course now X and Y range over C. It should be checked that $\sigma\tau$ has the right meaning whether we think of $\tau \in \Sigma^*$ or $\tau \in |C|$. The two "successor" mappings we are looking for are

$$x \mapsto 0x \quad \text{and} \quad x \mapsto 1x.$$

All the maps $x \mapsto \sigma x$ can be obtained as compositions of these iterated as many times as needed.

Here are two questions which we now should ask:

What plays the rôle of pred?

The mapping will be called **tail**, and it is characterized by:

$$\text{tail}(0x) = x,$$

$$\mathrm{tail}(1x) = x, \text{ and}$$

$$\mathrm{tail}(\Lambda) = \perp .$$

It is left to the reader to show that **tail** exists as an approximable mapping.

What plays the rôle of zero?

The answer is not unique, because in C there are several distinctions that have to be made; in fact we will define three maps:

$$\mathrm{empty}, \mathrm{zero}, \mathrm{one} : C \to T$$

where the three maps take on truth-values to distinguish various kinds of elements in $|C|$ as follows:

$$\begin{aligned}
\mathrm{empty}\,(\Lambda) &= \mathrm{true},\\
\mathrm{empty}\,(0x) &= \mathrm{false},\\
\mathrm{empty}\,(1x) &= \mathrm{false},\\
\mathrm{zero}\,(\Lambda) &= \mathrm{false}\\
\mathrm{zero}\,(0x) &= \mathrm{true}\\
\mathrm{zero}\,(1x) &= \mathrm{false}\\
\mathrm{one}\,(\Lambda) &= \mathrm{false}\\
\mathrm{one}\,(0x) &= \mathrm{false}\\
\mathrm{one}\,(1x) &= \mathrm{true}.
\end{aligned}$$

Again, it is an exercise to show these are approximable. The structured domain is therefore

$$\langle C, \Lambda, 0, 1, \mathrm{tail}, \mathrm{empty}, \mathrm{zero}, \mathrm{one}\rangle.$$

Note that we have changed the meaning of some of the symbols in passing from N to C. Note too that there is a confusion between 0 as an element and 0 as the map $x \mapsto 0x$. There are just too few symbols! In any case this is only an example and not a philosophy of life, so the reader can be expected not to suffer too much.

An example of a definition of an *element* of $|C|$ by a fixed-point equation is:

$$a = 0\,1\,a.$$

This equation has one and only one solution in $|C|$, the infinite sequence that alternates 0's and 1's. Note that a is also characterized by:

$$a = 0101a.$$

Another element is

$$b = 010\,b,$$

which is quite different from a.

An example of a *map* in $|C \to C|$ has the characterization

$$d(\Lambda) = \Lambda$$
$$d(0x) = 00d(x), \quad \text{and}$$
$$d(1x) = 11d(x).$$

We can write:

$$d(x) = \mathrm{cond}(\mathrm{empty}(x),\ \Lambda,\ \mathrm{cond}(\mathrm{zero}(x),\ 00d(\mathrm{tail}(x)),\ 11d(\mathrm{tail}(x)))).$$

As we shall see in due course, this can be regarded as a fixed-point definition of d.

An example of a map in $|C \times C \to C|$ was suggested in 2.21. We can write:

$$xy = \mathrm{cond}(\mathrm{empty}(x),\ y,\ \mathrm{cond}(\mathrm{zero}(x),\ 0(\mathrm{tail}(x)\,y),\ 1(\mathrm{tail}(x)\,y))).$$

It should be checked that this equation exactly characterizes the intended mapping. □

The examples we have given with N and C are examples of definitions of functions by *recursion*. The literal meaning of "recursion" is "running backwards", and a look at the equations for our examples will show that the functions are characterized by giving their values either *outright* (e.g. at 0 or at Λ) or at *earlier* arguments (e.g. at $\mathrm{pred}(x)$ or at $\mathrm{tail}(x)$). The reader should keep in mind that a recursive "definition" is not really a definition in the sense of *explicit definition* but rather is a characterization; a theorem has to be proved to show that such functions exist. Now we have a general definition of domain and a general theorem on fixed points and a general construction of function-space domain; THEREFORE, we know that there are solutions to our equations PROVIDED THAT the variables range over elements of a domain and that the other, given functions that appear in the equations are already known to be approximable (continuous). This proviso is very important, and we shall remark on it time after time.

But, as is well known, recursion also can be done over sets like $\mathbf{N}$, and we should examine now the connection between the familiar kind of recursion and what we are doing over domains. Of course, one simple connection is already provided by the way we regard $\mathbf{N}$ as a subset of N. But there are other useful connections that can be employed in a way that may seem more direct.

The sequence system with cons maps and the alternating fixed point are with the sequence domain peer.

Theorem (`example_44`). False-cons of true-cons of the alternating element recovers that element, and bit-cons prepends the bit on both partial and total string elements.

Proof. Invoke the alternating fixed-point law and the two bit-cons evaluation laws. ■

`Example_4_4.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.seqDomain

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open SeqDom ApproximableMap BitDom

/-- Scott's Example 4.4: cons maps act by prepending on cones and total strings;
    alternating
fixed point of false-then-true cons. -/
```

```
theorem example_44 :
    ((consMap false).toElementMap ((consMap true).toElementMap altElt) = altElt) ∧
    (∀ (b : Bool) (σ : Str), (consMap b).toElementMap (strBot σ) = strBot (b :: σ)) ∧
    (∀ (b : Bool) (σ : Str), (consMap b).toElementMap (strElem σ) = strElem (b :: σ))
    :=
  ⟨altElt_eq, @consMap_strBot, @consMap_strElem⟩
```

4.5.10 Definition 4.5 — Peano axioms

DEFINITION 4.5. A structured set $\langle \mathbf{N}, 0, {}^+ \rangle$, where $0 \in \mathbf{N}$ and ${}^+ : \mathbf{N} \to \mathbf{N}$ is a unary function, is said to be a *model for Peano's Axioms* if the following conditions are satisfied:

(i) $0 \neq n^+$, for all $n \in \mathbf{N}$;

(ii) $n^+ = m^+$ implies $n = m$, for all $n, m \in \mathbf{N}$;

(iii) whenever $x \subseteq \mathbf{N}$ and $0 \in x$ and $x^+ \subseteq x$, then $x = \mathbf{N}$.

Here $x^+ = \{n^+ \mid n \in x\}$. □

Clause (iii) is recognized as the principle of mathematical induction stated in terms of sets. We usually think of $\mathbf{N}$ as being "God given", and (i)–(iii) as known without question. Suppose God, however, decides to withdraw His set of integers and substitute another. We can ask: "Oh! Why did You take from us our beloved numbers? Why must we now live with these strange new beasts?" God will probably reply "Trust Me!" Perhaps we should in view of the theorem:

Definition (`PeanoModel`). A Peano model on a type carries a zero element and a successor operation such that zero is never a successor, successor is injective, and every set containing zero and closed under successor is the whole carrier.

Proof. By definition. ■

Definition (`PeanoModel.Graph`). The Peano graph of two models is the inductive relation generated by pairing the two zeros and closed under simultaneous succession.

Proof. By the inductive clauses. ■

Theorem (`PeanoModel.Graph.swap`). The Peano graph is symmetric: a related pair yields the swapped pair for the swapped models.

Proof. Induct on the graph derivation, swapping the base and step clauses. ■

Theorem (`PeanoModel.graph_succ_right`). If a successor on the left is related to an element on the right, that right element is the successor of a uniquely determined predecessor related to the left predecessor.

Proof. Case on the graph derivation: the base clause contradicts that zero is never a successor; the step clause and injectivity of successor recover the predecessor pair. ■

Theorem (`PeanoModel.graph_zero_right`). If the left zero is related to a right element, that element is the right zero.

Proof. Case on the graph derivation: the step clause would force zero to be a successor. ■

Theorem (`PeanoModel.exists_unique_right`). For each left element there is a unique right element related by the Peano graph.

Proof. Induct on the left model: uniqueness at zero uses the zero-right law; uniqueness at a successor uses the successor-right law and uniqueness at the predecessor. ■

Theorem (`PeanoModel.exists_unique_left`). For each right element there is a unique left element related by the Peano graph.

Proof. Apply uniqueness on the right to the swapped models and swap the witnessing pairs. ■

Definition_4_5.lean

```
module

public import Mathlib.Tactic.Attr.Core
public import Aesop.BuiltinRules
public import Mathlib.Basic.ExistsUnique
public import Mathlib.Data.Set.Defs

@[expose] public section

section
universe u
namespace DomainTheory.NeighbourhoodSystems

structure PeanoModel (N : Type u) where

  zero : N

  succ : N → N

  zero_ne_succ : ∀ n, zero = succ n

  succ_injective : Function.Injective succ

  induction : ∀ (s : Set N), zero ∈ s → (∀ n ∈ s, succ n ∈ s) → ∀ n, n ∈ s

end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
section
universe u v

namespace PeanoModel

variable {M : Type u} {N : Type v}

inductive Graph (P : PeanoModel M) (Q : PeanoModel N) : M → N → Prop
  | base : Graph P Q P.zero Q.zero
  | step : ∀ {m n}, Graph P Q m n → Graph P Q (P.succ m) (Q.succ n)

theorem Graph.swap {P : PeanoModel M} {Q : PeanoModel N} {m : M} {n : N}
    (h : Graph P Q m n) : Graph Q P n m := by
  induction h with
  | base => exact Graph.base
  | step _ ih => exact Graph.step ih

theorem graph_succ_right (P : PeanoModel M) (Q : PeanoModel N) {m : M} {k : N}
    (h : Graph P Q (P.succ m) k) : ∃ n, Graph P Q m n ∧ k = Q.succ n := by
  generalize hz : P.succ m = z at h
```

```
  cases h with
  | base => exact absurd hz.symm (P.zero_ne_succ _)
  | step h' =>
    rename_i m0 n0
    have hm : m = m0 := P.succ_injective hz
    subst hm
    exact ⟨n0, h', rfl⟩

theorem graph_zero_right (P : PeanoModel M) (Q : PeanoModel N) {k : N}
    (h : Graph P Q P.zero k) : k = Q.zero := by
  generalize hz : P.zero = z at h
  cases h with
  | base => rfl
  | step _ => exact absurd hz (P.zero_ne_succ _)

theorem exists_unique_right (P : PeanoModel M) (Q : PeanoModel N) (m : M) :
    ∃! n, Graph P Q m n := by
  refine P.induction {m | ∃! n, Graph P Q m n} ?_ ?_ m
  · exact ⟨Q.zero, Graph.base, fun k hk => graph_zero_right P Q hk⟩
  · rintro m' ⟨n', hn', huniq⟩
    refine ⟨Q.succ n', Graph.step hn', fun k hk => ?_⟩
    obtain ⟨n'', hn'', rfl⟩ := graph_succ_right P Q hk
    rw [huniq n'' hn'']

theorem exists_unique_left (P : PeanoModel M) (Q : PeanoModel N) (n : N) :
    ∃! m, Graph P Q m n := by
  obtain ⟨m, hm, huniq⟩ := exists_unique_right Q P n
  exact ⟨m, hm.swap, fun y hy => huniq y hy.swap⟩

end PeanoModel
end
end DomainTheory.NeighbourhoodSystems
end
```

4.5.11 Theorem 4.6 — Peano models isomorphic

THEOREM 4.6. All models of Peano's Axioms are isomorphic.

Proof: There are several ways to give the proof, but, for the sake of illustration, an application of the fixed-point theorem is appropriate here. Let $\langle \mathbf{N}, 0, {}^{+} \rangle$ be one model, and let $\langle \mathbf{M}, \Box, {}^{\#} \rangle$ be another. Let $\mathbf{N} \times \mathbf{M}$ be the ordinary cartesian product of the two sets and let

$$P(\mathbf{N} \times \mathbf{M})$$

be the powerset (set of all subsets) of $\mathbf{N} \times \mathbf{N}$. As in Exercises 1.15 and 2.20, we regard this set of elements as a domain, whose finite elements are just the finite subsets of the given set $\mathbf{N} \times \mathbf{M}$. The following mapping on $u \subseteq \mathbf{N} \times \mathbf{M}$ is easily proved approximable:

$$u \mapsto \{(0, \Box)\} \cup \{(n^{+}, m^{\#}) \mid (n, m) \in u\}.$$

(This assertion should be checked as an exercise.) We thus let r be the (least) fixed point:

$$r = \{(0, \Box)\} \cup \{(n^{+}, m^{\#}) \mid (n, m) \in r\}.$$

This $r \subseteq \mathbf{N} \times \mathbf{M}$ as a binary relation will turn out to be a one-one correspondence giving the required isomorphism.

First of all we see by construction that

(i) $0\ r\ \square$;

(ii) $n\ r\ m$ implies $n^+\ r\ m^\#$.

So, if r proves to be a one-one correspondence, it will then be the desired isomorphism. Now, the two sets shown in the equation

$$\{(0, \square)\} \cap \{(n^+, m^\#) \mid (n, m) \in r\} = \emptyset$$

are disjoint by virtue of axiom 4.5(i). Therefore, 0 in $\mathbf{N}$ corresponds by r to one and only one element of $\mathbf{M}$, namely the element $\square$. Let $x \subseteq \mathbf{N}$ be the set of all elements of $\mathbf{N}$ corresponding by r to a unique element of $\mathbf{M}$. We have just shown $0 \in x$. Suppose $n \in x$, and let $m \in \mathbf{M}$ be the unique element with $n\ r\ m$. Now $n^+\ r\ m^\#$ holds, so n^+ corresponds to at least one element of $\mathbf{M}$. If $n^+\ r\ k$ also holds, then since $(n^+, k) \neq (0, \square)$, the fixed-point equation implies

$$n^+ = n_0^+ \quad \text{and} \quad k = m_0^\#$$

for some $(n_0, m_0) \in r$. By axiom 4.5(ii), $n = n_0$, and, by uniqueness (remember $n \in x$), $m = m_0$; thus, $m^\#$ is the unique correspondent for n^+. We have proved $n^+ \in x$. Therefore, $x^+ \subseteq x$; so by axiom 4.5(iii), $x = \mathbf{N}$ holds. Otherwise said, every element in $\mathbf{N}$ corresponds to a unique element of $\mathbf{M}$.

Note that the rôles of $\mathbf{N}$ and $\mathbf{M}$ are completely symmetric, *and* they satisfy the same axioms as structured sets. It follows, then, that every element of $\mathbf{M}$ corresponds to a unique element of $\mathbf{N}$. The proof that r is a one-one correspondence is now complete. $\square$

Theorem (`PeanoModel.peano_models_isomorphic`). Any two Peano models are isomorphic by an equivalence sending zero to zero and commuting with successor.

Proof. Build the equivalence from the unique Peano-graph partners on each side, using uniqueness to check the inverse laws, and the base and step clauses to check that zero and successor are preserved. ■

Theorem (`PeanoModel.theorem_46`). All models of Peano's axioms are isomorphic by a zero-and-successor-preserving equivalence.

Proof. Invoke the Peano-model isomorphism theorem. ■

`Theorem_4_6.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Definition_4_5

public import Mathlib.Algebra.Group.End

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
universe u v
namespace PeanoModel
variable {M : Type u} {N : Type v}
```

```
theorem peano_models_isomorphic (P : PeanoModel M) (Q : PeanoModel N) :
    ∃ e : M ≃ N, e P.zero = Q.zero ∧ ∀ m, e (P.succ m) = Q.succ (e m) := by
  classical
  refine ⟨⟨fun m => (exists_unique_right P Q m).choose,
          fun n => (exists_unique_left P Q n).choose, ?_, ?_⟩, ?_, ?_⟩
  ·
    intro m
    have hf : Graph P Q m (exists_unique_right P Q m).choose :=
      (exists_unique_right P Q m).choose_spec.1
    exact ((exists_unique_left P Q _).choose_spec.2 m hf).symm
  ·
    intro n
    have hg : Graph P Q (exists_unique_left P Q n).choose n :=
      (exists_unique_left P Q n).choose_spec.1
    exact ((exists_unique_right P Q _).choose_spec.2 n hg).symm
  ·
    exact ((exists_unique_right P Q P.zero).choose_spec.2 Q.zero Graph.base).symm
  ·
    intro m
    have hf : Graph P Q m (exists_unique_right P Q m).choose :=
      (exists_unique_right P Q m).choose_spec.1
    exact ((exists_unique_right P Q (P.succ m)).choose_spec.2 _ (Graph.step hf)).symm

end PeanoModel
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.PeanoModel

theorem theorem_46 :
    ∀ {M N : Type*} (P : PeanoModel M) (Q : PeanoModel N),
      ∃ e : M ≃ N, e P.zero = Q.zero ∧ ∀ m, e (P.succ m) = Q.succ (e m) :=
  @peano_models_isomorphic

end DomainTheory.NeighbourhoodSystems.PeanoModel
end
```

4.5.12 Fixed point above a seed

Least fixed point of an endomap above a seed that lies below its image, obtained as the directed supremum of the iterates from that seed.

Definition (`ApproximableMap.iterFrom`). The n-fold iterate of an endomap from a seed is the n-fold application of its element map to that seed.

Proof. By definition. ■

Theorem (`ApproximableMap.iterFrom_zero`). The zero-fold iterate is the seed.

Proof. By definition. ■

Theorem (`ApproximableMap.iterFrom_succ`). The successor iterate applies the element map once more.

Proof. Unpack iteration at a successor. ■

Theorem (`ApproximableMap.iterFrom_step`). If the seed lies below its image, successive iterates are increasing.

Proof. Induct on the step index, using monotonicity of the element map. ■

Theorem (`ApproximableMap.iterFrom_mono`). If the seed lies below its image, iterates are monotone in the step index.

Proof. Induct along the inequality of indices, chaining the one-step increase. ■

Definition (`ApproximableMap.fixAbove`). The least fixed point above a seed below its image is the directed supremum of the iterates from that seed.

Proof. Directedness of the iterates follows from monotonicity in the step index. ■

Theorem (`ApproximableMap.fixAbove_isFixed`). The directed supremum of the iterates from such a seed is a fixed point of the endomap.

Proof. Compare both directions: applying the map sends each iterate to the next, hence into the supremum; each iterate lies below the image of the supremum by monotonicity of the map and of the iterates. ■

Theorem (`ApproximableMap.fixAbove_least`). Among postfixed points above the seed, that directed supremum is least.

Proof. Induct on the iterate index: the seed lies below the postfixed point, and the inductive step uses monotonicity of the map together with the postfix inequality. ■

Theorem (`ApproximableMap.le_fixAbove`). The seed lies below the directed supremum of its iterates.

Proof. The zero-fold iterate is the seed and lies below every directed supremum of the iterates. ■

`fixAbove.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupDirected
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {α : Type*} {V : NeighborhoodSystem α}

namespace ApproximableMap

def iterFrom (f : ApproximableMap V V) (a : V.Element) (n : ℕ) : V.Element :=
  (f.toElementMap)^[n] a

@[simp] theorem iterFrom_zero (f : ApproximableMap V V) (a : V.Element) :
    f.iterFrom a 0 = a := rfl

theorem iterFrom_succ (f : ApproximableMap V V) (a : V.Element) (n : ℕ) :
    f.iterFrom a (n + 1) = f.toElementMap (f.iterFrom a n) := by
  rw [iterFrom, iterFrom, Function.iterate_succ', Function.comp_apply]

theorem iterFrom_step (f : ApproximableMap V V) {a : V.Element} (ha : a ≤ f.
    toElementMap a)
    (n : ℕ) : f.iterFrom a n ≤ f.iterFrom a (n + 1) := by
  induction n with
  | zero => rw [iterFrom_succ f a 0, iterFrom_zero]; exact ha
```

```
  | succ k ih =>
    rw [iterFrom_succ f a k, iterFrom_succ f a (k + 1)]
    exact f.toElementMap_mono ih

theorem iterFrom_mono (f : ApproximableMap V V) {a : V.Element} (ha : a ≤ f.
    toElementMap a)
    {n m : ℕ} (hnm : n ≤ m) : f.iterFrom a n ≤ f.iterFrom a m := by
  induction hnm with
  | refl => exact le_refl _
  | step _ ih => exact le_trans ih (iterFrom_step f ha _)

def fixAbove (f : ApproximableMap V V) {a : V.Element} (ha : a ≤ f.toElementMap a) : V
    .Element :=
  NeighborhoodSystem.iSupDirected (f.iterFrom a)
    (fun i j => ⟨max i j, iterFrom_mono f ha (le_max_left i j),
      iterFrom_mono f ha (le_max_right i j)⟩)

theorem fixAbove_isFixed (f : ApproximableMap V V) {a : V.Element} (ha : a ≤ f.
    toElementMap a) :
    f.toElementMap (f.fixAbove ha) = f.fixAbove ha := by
  let hdir : ∀ i j, ∃ k, f.iterFrom a i ≤ f.iterFrom a k ∧ f.iterFrom a j ≤ f.iterFrom
     a k :=
    fun i j => ⟨max i j, iterFrom_mono f ha (le_max_left i j),
      iterFrom_mono f ha (le_max_right i j)⟩
  apply le_antisymm
  ·
    rw [fixAbove, toElementMap_iSupDirected]
    apply NeighborhoodSystem.iSupDirected_le
    intro n
    have : f.toElementMap (f.iterFrom a n) = f.iterFrom a (n + 1) := (iterFrom_succ f
    a n).symm
    rw [this]
    exact NeighborhoodSystem.le_iSupDirected (f.iterFrom a) hdir (n + 1)
  ·
    apply NeighborhoodSystem.iSupDirected_le
    intro n
    calc f.iterFrom a n
        ≤ f.iterFrom a (n + 1) := iterFrom_mono f ha (Nat.le_succ n)
      _ = f.toElementMap (f.iterFrom a n) := iterFrom_succ f a n
      _ ≤ f.toElementMap (f.fixAbove ha) :=
          f.toElementMap_mono (NeighborhoodSystem.le_iSupDirected (f.iterFrom a) hdir
    n)

theorem fixAbove_least (f : ApproximableMap V V) {a : V.Element} (ha : a ≤ f.
    toElementMap a)
    {z : V.Element} (haz : a ≤ z) (hz : f.toElementMap z ≤ z) : f.fixAbove ha ≤ z :=
    by
  apply NeighborhoodSystem.iSupDirected_le
  intro n
  induction n with
  | zero => rwa [iterFrom_zero]
  | succ k ih =>
    rw [iterFrom_succ]
    exact le_trans (f.toElementMap_mono ih) hz

theorem le_fixAbove (f : ApproximableMap V V) {a : V.Element} (ha : a ≤ f.toElementMap
```

```
    a) :
    a ≤ f.fixAbove ha := by
  have := NeighborhoodSystem.le_iSupDirected (f.iterFrom a)
    (fun i j => ⟨max i j, iterFrom_mono f ha (le_max_left i j),
      iterFrom_mono f ha (le_max_right i j)⟩) 0
  rwa [iterFrom_zero] at this

end ApproximableMap
end
end DomainTheory.NeighbourhoodSystems
```

4.5.13 Exercise 4.7 — fixed point above a seed

Formula 4.2(iii) shows how to find the *least* fixed point of $f : D \to D$. Suppose on the other hand that $a \in |D|$ is such that $a \subseteq f(a)$. Will there be a fixed point $x = f(x)$ with $a \subseteq x$?

(Hint: How do we know $\bigsqcup_{n=0}^{\infty} f^n(\bot) \in |D|$?)

The fixed point above a seed and its leastness laws are with the fixed point above a seed peer.

Theorem (`example_407`). If a seed lies below its image under an endomap, the directed supremum of the iterates from that seed is a fixed point, and it is least among postfixed points above the seed.

Proof. Invoke the fixed-point and leastness laws for the fixed point above a seed. ■

`Exercise_4_7.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixAbove

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 4.7: fixAbove is a fixed point and the least one above a. -/
theorem example_407 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : ApproximableMap V V)
      {a : V.Element} (ha : a ≤ f.toElementMap a),
      f.toElementMap (f.fixAbove ha) = f.fixAbove ha) ∧
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : ApproximableMap V V)
      {a : V.Element} (ha : a ≤ f.toElementMap a) {z : V.Element},
      a ≤ z → f.toElementMap z ≤ z → f.fixAbove ha ≤ z) :=
  ⟨@fixAbove_isFixed, @fixAbove_least⟩
```

4.5.14 Iterate-chain supremum

Directed supremum of a monotone $\mathbb{N}$-indexed chain of elements, and the characterisation of the least fixed-point element as the supremum of the iterate elements from bottom.

Definition (`ApproximableMap.supChain`). The supremum of a monotone $\mathbb{N}$-indexed chain of elements is their directed supremum along the maximum-index directedness.

Proof. By definition. ■

Theorem (`ApproximableMap.iterElem_monotone`). The iterate elements of an endomap from bottom form a monotone sequence.

Proof. Invoke monotonicity of iterate elements in the step index. ■

Theorem (`ApproximableMap.iterElem_succ`). The successor iterate element is the image of the previous under the endomap.

Proof. Unpack the iterate-map composition at a successor and the element-map composition law. ■

Theorem (`ApproximableMap.iterElem_zero`). The zero-fold iterate element is bottom.

Proof. The zero iterate map is the identity, which sends bottom to bottom. ■

Theorem (`ApproximableMap.comp_self_iterElem`). The n-fold iterate element of an endomap composed with itself equals the $2n$-fold iterate element of the original map.

Proof. Induct on n, using the successor and zero laws for iterate elements together with composition of element maps. ■

Theorem (`ApproximableMap.mem_supChain`). A neighbourhood belongs to a chain supremum precisely when it belongs to some stage of the chain.

Proof. Invoke the membership law for directed suprema. ■

Theorem (`ApproximableMap.fixElement_eq_supChain`). The least fixed-point element is the chain supremum of the iterate elements from bottom.

Proof. Compare membership using the directed-supremum and least-fixed-point membership characterisations together with the iterate-element membership law. ■

supChain.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupDirected
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import Mathlib.Tactic.Ring

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {α : Type*} {V : NeighborhoodSystem α}

namespace ApproximableMap

def supChain (s : ℕ → V.Element) (hmono : Monotone s) : V.Element :=
  NeighborhoodSystem.iSupDirected s
    (fun i j => ⟨max i j, hmono (le_max_left i j), hmono (le_max_right i j)⟩)

theorem iterElem_monotone (f : ApproximableMap V V) : Monotone f.iterElem :=
  fun _ _ hab => iterElem_mono f hab

theorem iterElem_succ (f : ApproximableMap V V) (n : ℕ) :
    f.iterElem (n + 1) = f.toElementMap (f.iterElem n) := by
  show (f.comp (f.iterMap n)).toElementMap V.bot
      = f.toElementMap ((f.iterMap n).toElementMap V.bot)
  rw [toElementMap_comp]

theorem iterElem_zero (f : ApproximableMap V V) : f.iterElem 0 = V.bot :=
  toElementMap_idMap V.bot
```

```
theorem comp_self_iterElem (f : ApproximableMap V V) (n : ℕ) :
    (f.comp f).iterElem n = f.iterElem (2 * n) := by
  induction n with
  | zero => rw [iterElem_zero, Nat.mul_zero, iterElem_zero]
  | succ k ih =>
    have h2 : 2 * (k + 1) = 2 * k + 1 + 1 := by ring
    rw [iterElem_succ, ih, toElementMap_comp, h2, iterElem_succ, iterElem_succ]

theorem mem_supChain (s : ℕ → V.Element) (hmono : Monotone s) {Z : Set α} :
    (supChain s hmono).mem Z ↔ ∃ n, (s n).mem Z :=
  NeighborhoodSystem.mem_iSupDirected s _

theorem fixElement_eq_supChain (f : ApproximableMap V V) :
    f.fixElement = supChain f.iterElem (iterElem_monotone f) := by
  apply Element.ext
  intro X
  rw [mem_supChain, mem_fixElement]
  constructor
  · rintro ⟨n, hn⟩; exact ⟨n, (mem_iterElem f n).mpr hn⟩
  · rintro ⟨n, hn⟩; exact ⟨n, (mem_iterElem f n).mp hn⟩

end ApproximableMap
end
end DomainTheory.NeighbourhoodSystems
```

4.5.15 Fixed-point induction

Fixed-point induction along iterate chains from bottom, and equality of commuting maps at the least fixed point.

Theorem (`ApproximableMap.fix_induction`). A property of elements that holds at bottom, is preserved by an endomap, and is closed under directed chain-suprema holds at the least fixed point of that map.

Proof. The iterate elements from bottom satisfy the property by induction on the step index; the least fixed point is their chain supremum, so the supremum hypothesis applies. ■

Theorem (`ApproximableMap.fix_induction_eq`). Two maps that agree at bottom and each commute with an endomap agree at that endomap's least fixed point.

Proof. Apply fixed-point induction to equality of the two element maps, using the commutation laws at the successor step and directed-supremum continuity for the chain-supremum step. ■

`fixInduction.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.supChain

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
```

```
variable {α : Type*} {V : NeighborhoodSystem α}

namespace ApproximableMap

theorem fix_induction (f : ApproximableMap V V) (P : V.Element → Prop)
    (hbot : P V.bot)
    (hstep : ∀ x, P x → P (f.toElementMap x))
    (hsup : ∀ (s : ℕ → V.Element) (hmono : Monotone s), (∀ n, P (s n)) → P (supChain
    s hmono)) :
    P f.fixElement := by
  have hmono : Monotone f.iterElem := iterElem_monotone f
  have hP : ∀ n, P (f.iterElem n) := by
    intro n
    induction n with
    | zero => rw [iterElem_zero]; exact hbot
    | succ k ih => rw [iterElem_succ]; exact hstep _ ih
  rw [fixElement_eq_supChain f]
  exact hsup f.iterElem hmono hP

theorem fix_induction_eq (f a b : ApproximableMap V V)
    (hbot : a.toElementMap V.bot = b.toElementMap V.bot)
    (hfa : f.comp a = a.comp f) (hfb : f.comp b = b.comp f) :
    a.toElementMap f.fixElement = b.toElementMap f.fixElement := by
  have hca : ∀ x, a.toElementMap (f.toElementMap x) = f.toElementMap (a.toElementMap x)
     := by
    intro x
    have h1 : (a.comp f).toElementMap x = (f.comp a).toElementMap x := by rw [hfa]
    rwa [toElementMap_comp, toElementMap_comp] at h1
  have hcb : ∀ x, b.toElementMap (f.toElementMap x) = f.toElementMap (b.toElementMap x)
     := by
    intro x
    have h1 : (b.comp f).toElementMap x = (f.comp b).toElementMap x := by rw [hfb]
    rwa [toElementMap_comp, toElementMap_comp] at h1
  refine fix_induction f (fun x => a.toElementMap x = b.toElementMap x) hbot ?_ ?_
  · intro x hx
    rw [hca, hcb, hx]
  · intro s hmono hs
    show a.toElementMap (supChain s hmono) = b.toElementMap (supChain s hmono)
    rw [supChain, toElementMap_iSupDirected, toElementMap_iSupDirected]
    apply Element.ext
    intro Z
    rw [NeighborhoodSystem.mem_iSupDirected, NeighborhoodSystem.mem_iSupDirected]
    constructor
    · rintro ⟨n, hn⟩; exact ⟨n, by rw [← hs n]; exact hn⟩
    · rintro ⟨n, hn⟩; exact ⟨n, by rw [hs n]; exact hn⟩

end ApproximableMap
end
end DomainTheory.NeighbourhoodSystems
```

4.5.16 Exercise 4.8 — fixed-point induction

Suppose $f : D \to D$ and $S \subseteq |D|$ are such that

(i) $\bot \in S$;

(ii) $x \in S$ always implies $f(x) \in S$;

(iii) whenever $\{x_n\}_{n=0}^{\infty} \subseteq S$ and $x_n \sqsubseteq x_{n+1}$ for all n, then $\bigsqcup_{n=0}^{\infty} x_n \in S$.

Conclude that $\mathrm{fix}(f) \in S$. (This could be called the principle of *fixed-point induction*.) Apply the method to a set of the form

$$S = \{x \in |D| \mid a(x) = b(x)\},$$

where $a, b : D \to D$ are approximable, and where we know $a(\bot) = b(\bot)$, and $f \circ a = a \circ f$ and $f \circ b = b \circ f$.

The induction principle and its equality application are with the fixed-point induction peer.

Theorem (`example_408`). Fixed-point induction holds: a property true at bottom, preserved by an endomap, and closed under monotone chain-suprema holds at the least fixed point; and two maps that agree at bottom and commute with the endomap agree at that least fixed point.

Proof. Invoke the fixed-point induction theorem and its equality corollary. ■

`Exercise_4_8.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixInduction

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

theorem example_408 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f a b : ApproximableMap V V),
      a.toElementMap V.bot = b.toElementMap V.bot →
        f.comp a = a.comp f → f.comp b = b.comp f →
          a.toElementMap f.fixElement = b.toElementMap f.fixElement) ∧
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : ApproximableMap V V)
      (P : V.Element → Prop),
      P V.bot →
        (∀ x, P x → P (f.toElementMap x)) →
          (∀ (s : ℕ → V.Element) (hmono : Monotone s),
            (∀ n, P (s n)) → P (supChain s hmono)) →
            P f.fixElement) :=
  ⟨@fix_induction_eq, @fix_induction⟩
```

4.5.17 Psi operator

The approximable operator Ψ that evaluates a functional at a map by applying that map to the functional's value, and the characterisation of the fix combinator as Ψ's least fixed point.

Definition (`bigPsi`). The operator Ψ on functionals is the curry of evaluation composed with pairing the first projection against evaluation of function-space maps.

Proof. By definition. ■

Theorem (`bigPsi_apply`). Applying Ψ to a functional and then to a map yields the map applied to the functional at that map.

Proof. Unpack the curry, composition, pairing, projection, and evaluation laws. ■

Theorem (`toApproxMap_toFilter_fixMap`). The approximable map recovered from the filtered fix combinator is the fix combinator itself.

Proof. Invoke the function-space equivalence unit. ■

Theorem (`bigPsi_fix`). The filtered fix combinator is a fixed point of Ψ.

Proof. Compare recovered maps via the evaluation law for Ψ and the fixed-point law for the fix combinator. ■

Theorem (`bigPsi_least`). Among postfixed points of Ψ, the filtered fix combinator is least.

Proof. A postfix inequality for Ψ yields a postfix inequality at each map, so the least-fixed-point law for each map places the fix combinator below the functional. ■

Theorem (`fix_eq_fixElement_bigPsi`). The filtered fix combinator is the least fixed-point element of Ψ.

Proof. Combine leastness of the filtered fix combinator among postfixed points of Ψ with leastness of the least fixed-point element. ■

`bigPsi.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α : Type*}

def bigPsi (V : NeighborhoodSystem α) :
    ApproximableMap (funSpace (funSpace V V) V) (funSpace (funSpace V V) V) :=
  curry ((evalMap V V).comp
    (paired (proj₁ (funSpace (funSpace V V) V) (funSpace V V)) (evalMap (funSpace V V)
    V)))

theorem bigPsi_apply (V : NeighborhoodSystem α)
    (? : (funSpace (funSpace V V) V).Element) (f : (funSpace V V).Element) :
    (toApproxMap ((bigPsi V).toElementMap ?)).toElementMap f
      = (toApproxMap f).toElementMap ((toApproxMap ?).toElementMap f) := by
  rw [bigPsi, toElementMap_curry_apply, toElementMap_comp, toElementMap_paired,
    toElementMap_proj₁, snd_pair, evalMap_apply, evalMap_apply]

theorem toApproxMap_toFilter_fixMap (V : NeighborhoodSystem α) :
    toApproxMap (toFilter (fixMap V)) = fixMap V := by
  have he := (funSpaceEquiv (funSpace V V) V).apply_symm_apply (fixMap V)
  rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he

theorem bigPsi_fix (V : NeighborhoodSystem α) :
    (bigPsi V).toElementMap (toFilter (fixMap V)) = toFilter (fixMap V) := by
  apply (funSpaceEquiv (funSpace V V) V).injective
```

```
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply]
  apply ext_of_toElementMap
  intro f
  rw [bigPsi_apply, toApproxMap_toFilter_fixMap, fixMap_toElementMap,
    toElementMap_fixElement]

theorem bigPsi_least (V : NeighborhoodSystem α) (? : (funSpace (funSpace V V) V).
    Element)
    (h? : (bigPsi V).toElementMap ? ≤ ?) : toFilter (fixMap V) ≤ ? := by
  have h?' : toApproxMap ((bigPsi V).toElementMap ?) ≤ toApproxMap ? := by
    rw [← funSpaceEquiv_apply, ← funSpaceEquiv_apply]
    exact (funSpaceEquiv (funSpace V V) V).monotone h?
  have hpre : ∀ f, (toApproxMap f).toElementMap ((toApproxMap ?).toElementMap f)
      ≤ (toApproxMap ?).toElementMap f := by
    intro f
    have h := (le_iff_toElementMap_le.mp h?') f
    rwa [bigPsi_apply] at h
  apply (funSpaceEquiv (funSpace V V) V).le_iff_le.mp
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toApproxMap_toFilter_fixMap,
    le_iff_toElementMap_le]
  intro f
  rw [fixMap_toElementMap]
  exact fixElement_le_of_toElementMap_le (toApproxMap f) (hpre f)

theorem fix_eq_fixElement_bigPsi (V : NeighborhoodSystem α) :
    toFilter (fixMap V) = (bigPsi V).fixElement := by
  apply le_antisymm
  · exact bigPsi_least V _ (le_of_eq (toElementMap_fixElement (bigPsi V)))
  · exact fixElement_le_of_toElementMap_le (bigPsi V) (le_of_eq (bigPsi_fix V))

end DomainTheory.NeighbourhoodSystems
```

4.5.18 Exercise 4.9 — fix as a fixed point

Show that there is an approximable operator

$$\Psi : ((D \to D) \to D) \to ((D \to D) \to D)$$

such that for $\theta : (D \to D) \to D$ and $f : D \to D$ we have

$$\Psi(\theta)(f) = f(\theta(f)).$$

Prove further that $\mathrm{fix} : (D \to D) \to D$ is the *least* fixed point of Ψ.

The operator Ψ and its least-fixed-point characterisation of fix are with the Psi operator peer.

Theorem (`example_409`). The operator Ψ evaluates a functional at a map by applying that map to the functional's value; the filtered fix combinator is a fixed point of Ψ, least among postfixed points, and equals Ψ's least fixed-point element.

Proof. Invoke the evaluation, fixed-point, leastness, and least-fixed-point characterisation laws for Ψ. ■

`Exercise_4_9.lean`

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.bigPsi

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

theorem example_409 :
    (∀ {α : Type*} (V : NeighborhoodSystem α)
      (? : (funSpace (funSpace V V) V).Element) (f : (funSpace V V).Element),
      (toApproxMap ((bigPsi V).toElementMap ?)).toElementMap f
        = (toApproxMap f).toElementMap ((toApproxMap ?).toElementMap f)) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α),
      (bigPsi V).toElementMap (toFilter (fixMap V)) = toFilter (fixMap V)) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α)
      (? : (funSpace (funSpace V V) V).Element),
      (bigPsi V).toElementMap ? ≤ ? → toFilter (fixMap V) ≤ ?) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α),
      toFilter (fixMap V) = (bigPsi V).fixElement) :=
  ⟨@bigPsi_apply, @bigPsi_fix, @bigPsi_least, @fix_eq_fixElement_bigPsi⟩
```

4.5.19 Domain above an element

The neighbourhood system of neighbourhoods of a fixed element, with embedding, restriction, the induced endomap, and the inclusion into the ambient domain.

Definition (`ApproximableMap.relSystem`). The relative neighbourhood system of an element has the same master as the ambient system and neighbourhoods precisely those belonging to that element.

Proof. By definition. ■

Theorem (`ApproximableMap.relSystem_mem`). Membership in the relative system is membership in the element.

Proof. By definition. ■

Theorem (`ApproximableMap.relSystem_master`). The relative master is the ambient master.

Proof. By definition. ■

Definition (`ApproximableMap.embed`). Embedding a relative element into the ambient domain records ambient neighbourhoods that contain a relative neighbourhood of the element.

Proof. Intersection and upward closure unpack the ambient and relative element laws. ■

Theorem (`ApproximableMap.embed_le`). An embedded relative element lies below the seed element.

Proof. Upward-close a witnessing neighbourhood of the seed. ■

Theorem (`ApproximableMap.embed_mono`). Embedding is monotone.

Proof. Apply the relative inequality to the witnessing neighbourhood. ■

Theorem (`ApproximableMap.le_of_embed_le`). An inequality of embeddings recovers an inequality of relative elements.

Proof. Test membership by embedding a principal witness and unpacking. ■

Definition (`ApproximableMap.relMap`). An endomap that fixes a seed induces an endomap of the relative system by relating only neighbourhoods of that seed.

Proof. Codomain membership uses the fixed-point hypothesis; the remaining approximability laws unpack those of the ambient map. ■

Definition (`ApproximableMap.restrict`). Restricting an ambient element below the seed yields a relative element with the same neighbourhoods.

Proof. By definition. ■

Theorem (`ApproximableMap.embed_restrict`). Embedding the restriction of an element below the seed recovers that element.

Proof. Compare membership both ways. ■

Theorem (`ApproximableMap.restrict_embed`). Restricting an embedding recovers the relative element.

Proof. Compare membership both ways. ■

Definition (`ApproximableMap.relIso`). Relative elements are order-isomorphic to ambient elements below the seed via embed and restrict.

Proof. Assemble the mutual inverses and the embedding monotonicity laws. ■

Theorem (`ApproximableMap.fixElement_below_unique`). A fixed point below the least fixed-point element equals that least fixed point.

Proof. Leastness of the least fixed-point element yields the reverse inequality. ■

Theorem (`ApproximableMap.relMap_toElementMap_embed`). Embedding intertwines the relative endomap with the ambient endomap.

Proof. Compare membership of neighbourhoods under the two composite evaluations. ■

Theorem (`ApproximableMap.relMap_unique_fixed`). The unique fixed point of the relative endomap on the least fixed-point seed is the restricted least fixed-point element.

Proof. The embedded relative fixed point is an ambient fixed point below the least fixed point, hence equals it; transport back by restriction. ■

Definition (`ApproximableMap.inclMap`). Inclusion from the relative system to the ambient domain relates a relative neighbourhood to an ambient neighbourhood containing it.

Proof. Approximability unpacks ambient intersection and inclusion. ■

Theorem (`ApproximableMap.inclMap_toElementMap`). Inclusion evaluates as embedding.

Proof. Compare membership. ■

Theorem (`ApproximableMap.inclMap_bot`). Inclusion sends relative bottom to ambient bottom.

Proof. Unpack embedding of bottom. ■

Theorem (`ApproximableMap.inclMap_intertwine`). Inclusion intertwines the relative endomap with the ambient endomap.

Proof. Compose the inclusion-as-embedding law with the relative–ambient intertwining law. ■

`relSystem.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import Mathlib.Tactic.Monotonicity.Lemmas
public import Mathlib.Tactic.Bound

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {α : Type*} {V : NeighborhoodSystem α}
namespace ApproximableMap
```

```
def relSystem (a : V.Element) : NeighborhoodSystem α where
  mem X := a.mem X
  master := V.master
  master_nonempty := V.master_nonempty
  master_mem := a.master_mem
  sub_master := fun hX => V.sub_master (a.sub hX)
  inter_mem := fun hX hY _ _ => a.inter_mem hX hY

@[simp] theorem relSystem_mem (a : V.Element) {X : Set α} :
    (relSystem a).mem X ↔ a.mem X := Iff.rfl

@[simp] theorem relSystem_master (a : V.Element) : (relSystem a).master = V.master :=
    rfl

def embed (a : V.Element) (g : (relSystem a).Element) : V.Element where
  mem X := V.mem X ∧ ∃ W, a.mem W ∧ g.mem W ∧ W ⊆ X
  sub := fun h => h.1
  master_mem := ⟨V.master_mem, V.master, a.master_mem, g.master_mem, subset_rfl⟩
  inter_mem := by
    rintro X Y ⟨hVX, Wx, haWx, hgWx, hWxX⟩ ⟨hVY, Wy, haWy, hgWy, hWyY⟩
    have haWxy : a.mem (Wx ∩ Wy) := a.inter_mem haWx haWy
    have hgWxy : g.mem (Wx ∩ Wy) := g.inter_mem hgWx hgWy
    have hsub : Wx ∩ Wy ⊆ X ∩ Y := Set.inter_subset_inter hWxX hWyY
    refine ⟨V.inter_mem hVX hVY (a.sub haWxy) hsub, Wx ∩ Wy, haWxy, hgWxy, hsub⟩
  up_mem := by
    rintro X Y ⟨hVX, W, haW, hgW, hWX⟩ hVY hXY
    exact ⟨hVY, W, haW, hgW, subset_trans hWX hXY⟩

theorem embed_le (a : V.Element) (g : (relSystem a).Element) : embed a g ≤ a := by
  rintro X ⟨hVX, W, haW, _, hWX⟩
  exact a.up_mem haW hVX hWX

theorem embed_mono (a : V.Element) {g₁ g₂ : (relSystem a).Element} (h : g₁ ≤ g₂) :
    embed a g₁ ≤ embed a g₂ := by
  rintro X ⟨hVX, W, haW, hgW, hWX⟩
  exact ⟨hVX, W, haW, h _ hgW, hWX⟩

theorem le_of_embed_le (a : V.Element) {g₁ g₂ : (relSystem a).Element}
    (h : embed a g₁ ≤ embed a g₂) : g₁ ≤ g₂ := by
  intro X hg₁X
  have hVX : V.mem X := a.sub (g₁.sub hg₁X)
  have : (embed a g₂).mem X := h X ⟨hVX, X, g₁.sub hg₁X, hg₁X, subset_rfl⟩
  obtain ⟨_, W, haW, hg₂W, hWX⟩ := this
  exact g₂.up_mem hg₂W (g₁.sub hg₁X) hWX

def relMap (f : ApproximableMap V V) {a : V.Element} (ha : f.toElementMap a = a) :
    ApproximableMap (relSystem a) (relSystem a) where
  rel X Y := a.mem X ∧ f.rel X Y
  rel_dom := fun h => h.1
  rel_cod := by
    rintro X Y ⟨haX, hXY⟩
    have hVX : V.mem X := a.sub haX
    have hpr : V.principal hVX ≤ a := fun Z ⟨hVZ, hXZ⟩ => a.up_mem haX hVZ hXZ
    have hmemfX : (f.toElementMap (V.principal hVX)).mem Y := (f.rel_iff_mem_principal
     hVX).mp hXY
```

```
    have hle : f.toElementMap (V.principal hVX) ≤ f.toElementMap a := f.
    toElementMap_mono hpr
    have : (f.toElementMap a).mem Y := hle Y hmemfX
    rw [ha] at this
    exact this
  master_rel := ⟨a.master_mem, f.master_rel⟩
  inter_right := by
    rintro X Y Y' ⟨haX, hXY⟩ ⟨_, hXY'⟩
    exact ⟨haX, f.inter_right hXY hXY'⟩
  mono := by
    rintro X X' Y Y' ⟨_, hXY⟩ hX'X hYY' haX' haY'
    exact ⟨haX', f.mono hXY hX'X hYY' (a.sub haX') (a.sub haY')⟩

def restrict (a : V.Element) (x : V.Element) (hx : x ≤ a) : (relSystem a).Element
    where
  mem X := x.mem X
  sub := fun h => hx _ h
  master_mem := x.master_mem
  inter_mem := fun hX hY => x.inter_mem hX hY
  up_mem := fun hX hY hXY => x.up_mem hX (a.sub hY) hXY

theorem embed_restrict (a : V.Element) (x : V.Element) (hx : x ≤ a) :
    embed a (restrict a x hx) = x := by
  apply Element.ext
  intro X
  constructor
  · rintro ⟨hVX, W, _, hxW, hWX⟩
    exact x.up_mem hxW hVX hWX
  · intro hxX
    exact ⟨x.sub hxX, X, hx _ hxX, hxX, subset_rfl⟩

theorem restrict_embed (a : V.Element) (g : (relSystem a).Element) :
    restrict a (embed a g) (embed_le a g) = g := by
  apply Element.ext
  intro X
  constructor
  · rintro ⟨hVX, W, haW, hgW, hWX⟩
    exact g.up_mem hgW (a.up_mem haW hVX hWX) hWX
  · intro hgX
    exact ⟨a.sub (g.sub hgX), X, g.sub hgX, hgX, subset_rfl⟩

def relIso (a : V.Element) : (relSystem a).Element ≃o {x : V.Element // x ≤ a} where
  toFun g := ⟨embed a g, embed_le a g⟩
  invFun x := restrict a x.1 x.2
  left_inv g := restrict_embed a g
  right_inv x := Subtype.ext (embed_restrict a x.1 x.2)
  map_rel_iff' := by
    intro g₁ g₂
    constructor
    · intro h; exact le_of_embed_le a h
    · intro h; exact embed_mono a h

theorem fixElement_below_unique (f : ApproximableMap V V) {x : V.Element}
    (hxle : x ≤ f.fixElement) (hxfix : f.toElementMap x = x) : x = f.fixElement := by
  have hge : f.fixElement ≤ x := fixElement_le_of_toElementMap_le f (le_of_eq hxfix)
  exact le_antisymm hxle hge
```

```
theorem relMap_toElementMap_embed (f : ApproximableMap V V) {a : V.Element}
    (ha : f.toElementMap a = a) (g : (relSystem a).Element) :
    embed a ((relMap f ha).toElementMap g) = f.toElementMap (embed a g) := by
  apply Element.ext
  intro X
  constructor
  · rintro ⟨hVX, W, haW, ⟨U, hgU, haU, hUW⟩, hWX⟩

    refine ⟨U, ⟨a.sub haU, U, haU, hgU, subset_rfl⟩, ?_⟩
    exact f.mono hUW subset_rfl hWX (a.sub haU) hVX
  · rintro ⟨U, ⟨hVU, W, haW, hgW, hWU⟩, hUX⟩

    have hVX : V.mem X := f.rel_cod hUX
    have hWX : f.rel W X := f.mono hUX hWU subset_rfl (a.sub haW) hVX
    have haX : a.mem X := by
      have hpr : V.principal (a.sub haW) ≤ a := fun Z ⟨hVZ, hXZ⟩ => a.up_mem haW hVZ
    hXZ
      have hmemfW : (f.toElementMap (V.principal (a.sub haW))).mem X :=
        (f.rel_iff_mem_principal (a.sub haW)).mp hWX
      have := (f.toElementMap_mono hpr) X hmemfW
      rw [ha] at this; exact this
    exact ⟨hVX, X, haX, ⟨W, hgW, haW, hWX⟩, subset_rfl⟩

theorem relMap_unique_fixed (f : ApproximableMap V V)
    (ha : f.toElementMap f.fixElement = f.fixElement) (g : (relSystem f.fixElement).
    Element)
    (hg : (relMap f ha).toElementMap g = g) :
    g = restrict f.fixElement f.fixElement (le_refl _) := by

  have hembfix : f.toElementMap (embed f.fixElement g) = embed f.fixElement g := by
    rw [← relMap_toElementMap_embed f ha g, hg]
  have heq : embed f.fixElement g = f.fixElement :=
    fixElement_below_unique f (embed_le f.fixElement g) hembfix

  have h1 : restrict f.fixElement (embed f.fixElement g) (embed_le f.fixElement g) = g
     :=
    restrict_embed f.fixElement g

  have h2 : restrict f.fixElement (embed f.fixElement g) (embed_le f.fixElement g)
      = restrict f.fixElement f.fixElement (le_refl _) := by
    apply Element.ext
    intro X
    show (embed f.fixElement g).mem X ↔ f.fixElement.mem X
    rw [heq]
  rw [← h1, h2]

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {α : Type*} {V : NeighborhoodSystem α}
```

```
namespace ApproximableMap

def inclMap (a : V.Element) : ApproximableMap (relSystem a) V where
  rel X Y := a.mem X ∧ V.mem Y ∧ X ⊆ Y
  rel_dom := fun h => h.1
  rel_cod := fun h => h.2.1
  master_rel := ⟨a.master_mem, V.master_mem, subset_rfl⟩
  inter_right := by
    rintro X Y Y' ⟨haX, hVY, hXY⟩ ⟨_, hVY', hXY'⟩
    exact ⟨haX, V.inter_mem hVY hVY' (a.sub haX) (Set.subset_inter hXY hXY'),
      Set.subset_inter hXY hXY'⟩
  mono := by
    rintro X X' Y Y' ⟨_, _, hXY⟩ hX'X hYY' haX' hVY'
    exact ⟨haX', hVY', (hX'X.trans hXY).trans hYY'⟩

theorem inclMap_toElementMap (a : V.Element) (g : (relSystem a).Element) :
    (inclMap a).toElementMap g = embed a g := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨X, hgX, haX, hVY, hXY⟩
    exact ⟨hVY, X, haX, hgX, hXY⟩
  · rintro ⟨hVY, W, haW, hgW, hWY⟩
    exact ⟨W, hgW, haW, hVY, hWY⟩

theorem inclMap_bot (a : V.Element) : (inclMap a).toElementMap (relSystem a).bot = V.
    bot := by
  rw [inclMap_toElementMap]
  apply Element.ext
  intro Y
  rw [mem_bot]
  constructor
  · rintro ⟨hVY, W, _, hbW, hWY⟩
    rw [mem_bot] at hbW
    rw [hbW] at hWY
    exact Set.Subset.antisymm (V.sub_master hVY) hWY
  · rintro rfl
    exact ⟨V.master_mem, V.master, a.master_mem, (relSystem a).bot.master_mem,
    subset_rfl⟩

theorem inclMap_intertwine (f : ApproximableMap V V) {a : V.Element} (ha : f.
    toElementMap a = a)
    (g : (relSystem a).Element) :
    (inclMap a).toElementMap ((relMap f ha).toElementMap g) =
      f.toElementMap ((inclMap a).toElementMap g) := by
  rw [inclMap_toElementMap, inclMap_toElementMap, relMap_toElementMap_embed]

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
```

4.5.20 Exercise 4.10 — domain above an element

Given a domain D and an element $a \in |D|$, construct a domain D_a where

$$|D_a| = \{x \in |D| \mid x \sqsubseteq a\}.$$

Show that if $f : D \to D$ is approximable, then f can be restricted to an approximable map $f' : D_{\mathrm{fix}(f)} \to D_{\mathrm{fix}(f)}$ where $f'(x) = f(x)$ for all $x \in |D_{\mathrm{fix}(f)}|$. How many fixed points does f' have in $|D_{\mathrm{fix}(f)}|$?

The relative domain, induced endomap, and inclusion are with the domain above an element peer.

Theorem (`example_410`). A fixed point below the least fixed-point element equals that least fixed point, and the relative endomap on the least fixed-point seed has a unique fixed point, namely the restricted least fixed-point element.

Proof. Invoke uniqueness of fixed points below the least fixed-point element and uniqueness of relative fixed points on that seed. ■

`Exercise_4_10.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.relSystem

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

theorem example_410 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : ApproximableMap V V)
      {x : V.Element},
      x ≤ f.fixElement → f.toElementMap x = x → x = f.fixElement) ∧
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : ApproximableMap V V)
      (ha : f.toElementMap f.fixElement = f.fixElement)
      (g : (relSystem f.fixElement).Element),
      (relMap f ha).toElementMap g = g →
        g = restrict f.fixElement f.fixElement (le_refl _)) :=
  ⟨@fixElement_below_unique, @relMap_unique_fixed⟩
```

4.5.21 Uniform fix uniqueness

Uniform characterisation of the least fixed-point assignment across domains, with commuting-map and iterate lemmas used in the uniqueness argument.

Theorem (`fix_unique_of_uniform`). Any assignment of fixed points to endomaps that is natural under strict intertwining maps equals the least fixed-point element on every domain.

Proof. Apply the assignment on the relative system of the least fixed-point element, use uniqueness of the relative fixed point, and transport along inclusion. ■

Theorem (`ApproximableMap.iterElem_le_fixElement`). Every iterate element from bottom lies below the least fixed-point element.

Proof. Unpack least-fixed-point membership. ■

Theorem (`ApproximableMap.fixElement_uniform`). A strict intertwining map sends least fixed points to least fixed points.

Proof. Compare images of iterates and use leastness on both sides. ■

Theorem (`ApproximableMap.commuting_least_common_fixed`). Commuting endomaps have a least common fixed point.

Proof. Take the fixed point of one map above the least fixed point of the other, using the fixed-point-above-a-seed construction. ■

Theorem (`ApproximableMap.fixElement_eq_of_commuting_bot`). Commuting endomaps that agree at bottom have the same least fixed point.

Proof. Show each least fixed point is fixed by the other map via iterate shifting. ■

Theorem (`ApproximableMap.fixElement_iterTwice`). The least fixed point of a map equals that of the map composed with itself.

Proof. Compare iterate chains using the double-step identity for iterate elements. ■

`fixUniform.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.relSystem
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixAbove
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.supChain
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {α : Type*} {V : NeighborhoodSystem α}
open ApproximableMap
universe u

theorem fix_unique_of_uniform
    (F : ∀ {β : Type u} (W : NeighborhoodSystem β), ApproximableMap W W → W.Element)
    (hfixed : ∀ {β : Type u} (W : NeighborhoodSystem β) (g : ApproximableMap W W),
      g.toElementMap (F W g) = F W g)
    (huniform : ∀ {β ? : Type u} (W₀ : NeighborhoodSystem β) (W₁ : NeighborhoodSystem
    ?)
      (g₀ : ApproximableMap W₀ W₀) (g₁ : ApproximableMap W₁ W₁) (h : ApproximableMap W₀
    W₁),
      h.toElementMap W₀.bot = W₁.bot →
      (∀ x, h.toElementMap (g₀.toElementMap x) = g₁.toElementMap (h.toElementMap x)) →
      h.toElementMap (F W₀ g₀) = F W₁ g₁)
    {α : Type u} (V : NeighborhoodSystem α) (f : ApproximableMap V V) :
    F V f = f.fixElement := by
  have ha : f.toElementMap f.fixElement = f.fixElement := toElementMap_fixElement f

  have hfix' : (relMap f ha).toElementMap (F (relSystem f.fixElement) (relMap f ha)) =
      F (relSystem f.fixElement) (relMap f ha) := hfixed (relSystem f.fixElement) (
    relMap f ha)

  have huniq : F (relSystem f.fixElement) (relMap f ha) =
      restrict f.fixElement f.fixElement (le_refl _) :=
```

```
    relMap_unique_fixed f ha _ hfix'

  have huse := huniform (relSystem f.fixElement) V (relMap f ha) f (inclMap f.
    fixElement)
    (inclMap_bot f.fixElement) (inclMap_intertwine f ha)
  rw [huniq, inclMap_toElementMap, embed_restrict] at huse
  exact huse.symm
end

section
variable {α : Type*} {V : NeighborhoodSystem α}

namespace ApproximableMap

theorem iterElem_le_fixElement (f : ApproximableMap V V) (n : ℕ) :
    f.iterElem n ≤ f.fixElement :=
  fun _ hX => ⟨n, (mem_iterElem f n).mp hX⟩

theorem fixElement_uniform {β ? : Type*} {W₀ : NeighborhoodSystem β} {W₁ :
    NeighborhoodSystem ?}
    (f₀ : ApproximableMap W₀ W₀) (f₁ : ApproximableMap W₁ W₁) (h : ApproximableMap W₀ W₁
    )
    (hbot : h.toElementMap W₀.bot = W₁.bot)
    (hintw : ∀ x, h.toElementMap (f₀.toElementMap x) = f₁.toElementMap (h.toElementMap
    x)) :
    h.toElementMap f₀.fixElement = f₁.fixElement := by

  have hiter : ∀ n, h.toElementMap (f₀.iterElem n) = f₁.iterElem n := by
    intro n
    rw [iterElem_eq_iterate f₀ n, iterElem_eq_iterate f₁ n]
    induction n with
    | zero => simpa using hbot
    | succ k ih =>
      rw [Function.iterate_succ', Function.comp_apply, Function.iterate_succ',
        Function.comp_apply, hintw, ih]
  apply le_antisymm
  .
    rw [fixElement_eq_iSupDirected f₀, toElementMap_iSupDirected]
    apply NeighborhoodSystem.iSupDirected_le
    intro n
    rw [hiter n]
    exact iterElem_le_fixElement f₁ n
  .
    have hfp : f₁.toElementMap (h.toElementMap f₀.fixElement) = h.toElementMap f₀.
    fixElement := by
      rw [← hintw f₀.fixElement, toElementMap_fixElement]
    exact fixElement_le_of_toElementMap_le f₁ (le_of_eq hfp)

theorem commuting_least_common_fixed {f g : ApproximableMap V V} (hcomm : f.comp g = g
    .comp f) :
    ∃ x : V.Element, f.toElementMap x = x ∧ g.toElementMap x = x ∧
      ∀ w : V.Element, f.toElementMap w = w → g.toElementMap w = w → x ≤ w := by
  set a := f.fixElement with ha_def
  have hfa : f.toElementMap a = a := toElementMap_fixElement f

  have ha_le : a ≤ g.toElementMap a := by
```

```
    apply fixElement_le_of_toElementMap_le f
    rw [commute_apply hcomm, hfa]
  set x := g.fixAbove ha_le with hx_def
  have hgx : g.toElementMap x = x := fixAbove_isFixed g ha_le
  have hax : a ≤ x := le_fixAbove g ha_le

  have hfix_each : ∀ n, f.toElementMap (g.iterFrom a n) = g.iterFrom a n := by
    intro n
    induction n with
    | zero => rw [iterFrom_zero]; exact hfa
    | succ k ih =>
      rw [iterFrom_succ g a k, commute_apply hcomm, ih]

  have hfx : f.toElementMap x = x := by
    apply le_antisymm
    · rw [hx_def, fixAbove, toElementMap_iSupDirected]
      apply NeighborhoodSystem.iSupDirected_le
      intro n
      rw [hfix_each n]
      exact NeighborhoodSystem.le_iSupDirected (g.iterFrom a) _ n
    · rw [hx_def, fixAbove]
      apply NeighborhoodSystem.iSupDirected_le
      intro n
      rw [← hfix_each n]
      exact f.toElementMap_mono (NeighborhoodSystem.le_iSupDirected (g.iterFrom a) _ n
    )
  refine ⟨x, hfx, hgx, ?_⟩
  intro w hfw hgw
  have haw : a ≤ w := fixElement_le_of_toElementMap_le f (le_of_eq hfw)
  exact fixAbove_least g ha_le haw (le_of_eq hgw)

theorem fixElement_eq_of_commuting_bot {f g : ApproximableMap V V} (hcomm : f.comp g =
    g.comp f)
    (hbot : f.toElementMap V.bot = g.toElementMap V.bot) :
    f.fixElement = g.fixElement := by

  have hshift : ∀ n, g.toElementMap (f.iterElem n) = f.iterElem (n + 1) := by
    intro n
    induction n with
    | zero =>
      rw [iterElem_zero, iterElem_succ, iterElem_zero, ← hbot]
    | succ k ih =>
      rw [iterElem_succ, ← commute_apply hcomm, ih]
      exact (iterElem_succ f (k + 1)).symm

  have hgfixf : g.toElementMap f.fixElement = f.fixElement := by
    rw [fixElement_eq_iSupDirected f, toElementMap_iSupDirected]
    apply le_antisymm
    · apply NeighborhoodSystem.iSupDirected_le
      intro n
      rw [hshift n]
      exact NeighborhoodSystem.le_iSupDirected f.iterElem _ (n + 1)
    · apply NeighborhoodSystem.iSupDirected_le
      intro n
      calc f.iterElem n ≤ f.iterElem (n + 1) := iterElem_mono f (Nat.le_succ n)
        _ = g.toElementMap (f.iterElem n) := (hshift n).symm
```

```
        _ ≤ _ := NeighborhoodSystem.le_iSupDirected (fun m => g.toElementMap (f.
    iterElem m)) _ n

  have hshift' : ∀ n, f.toElementMap (g.iterElem n) = g.iterElem (n + 1) := by
    intro n
    induction n with
    | zero =>
      rw [iterElem_zero, iterElem_succ, iterElem_zero, hbot]
    | succ k ih =>
      rw [iterElem_succ, commute_apply hcomm, ih]
      exact (iterElem_succ g (k + 1)).symm
  have hffixg : f.toElementMap g.fixElement = g.fixElement := by
    rw [fixElement_eq_iSupDirected g, toElementMap_iSupDirected]
    apply le_antisymm
    · apply NeighborhoodSystem.iSupDirected_le
      intro n
      rw [hshift' n]
      exact NeighborhoodSystem.le_iSupDirected g.iterElem _ (n + 1)
    · apply NeighborhoodSystem.iSupDirected_le
      intro n
      calc g.iterElem n ≤ g.iterElem (n + 1) := iterElem_mono g (Nat.le_succ n)
        _ = f.toElementMap (g.iterElem n) := (hshift' n).symm
        _ ≤ _ := NeighborhoodSystem.le_iSupDirected (fun m => f.toElementMap (g.
    iterElem m)) _ n
  apply le_antisymm
  · exact fixElement_le_of_toElementMap_le f (le_of_eq hffixg)
  · exact fixElement_le_of_toElementMap_le g (le_of_eq hgfixf)

theorem fixElement_iterTwice (f : ApproximableMap V V) :
    f.fixElement = (f.comp f).fixElement := by
  apply le_antisymm
  ·
    rw [fixElement_eq_iSupDirected f]
    apply NeighborhoodSystem.iSupDirected_le
    intro n
    calc f.iterElem n ≤ f.iterElem (2 * n) := iterElem_mono f (by omega)
      _ = (f.comp f).iterElem n := (comp_self_iterElem f n).symm
      _ ≤ (f.comp f).fixElement := iterElem_le_fixElement (f.comp f) n
  ·
    refine fixElement_le_of_toElementMap_le (f.comp f) (le_of_eq ?_)
    rw [toElementMap_comp, toElementMap_fixElement, toElementMap_fixElement]

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
end
```

4.5.22 Exercise 4.11 — characterizing the fix combinator

(Suggested by G. Plotkin). We can regard fix as assigning a fixed-point operator to each domain D. Show that fix is uniquely determined by the following general conditions on an assignment $D \rightsquigarrow F_D$:

(i) $F_D : (D \to D) \to D$;

(ii) $F_D(f) = f(F_D(f))$ for all $f : D \to D$;

(iii) whenever $f_0 : D_0 \to D_0$ and $f_1 : D_1 \to D_1$ are given and $h : D_0 \to D_1$ is such that $h(\bot) = \bot$ and $h \circ f_0 = f_1 \circ h$, then $h(F_{D_0}(f_0)) = F_{D_1}(f_1)$.

(Hint: Apply 4.7 to prove fix satisfies (iii). In the other direction use 4.10.)

The uniform uniqueness theorem and its supporting lemmas are with the uniform fix uniqueness peer.

Theorem (`example_411`). Any assignment of fixed points to endomaps that is natural under strict intertwining maps equals the least fixed-point element on every domain.

Proof. Invoke the uniform uniqueness theorem. ■

`Exercise_4_11.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixUniform

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

universe u

theorem example_411 :
    ∀ (F : ∀ {β : Type u} (W : NeighborhoodSystem β), ApproximableMap W W → W.Element)
    ,
       (∀ {β : Type u} (W : NeighborhoodSystem β) (g : ApproximableMap W W),
         g.toElementMap (F W g) = F W g) →
       (∀ {β ? : Type u} (W₀ : NeighborhoodSystem β) (W₁ : NeighborhoodSystem ?)
         (g₀ : ApproximableMap W₀ W₀) (g₁ : ApproximableMap W₁ W₁) (h : ApproximableMap W
    ₀ W₁),
         h.toElementMap W₀.bot = W₁.bot →
         (∀ x, h.toElementMap (g₀.toElementMap x) = g₁.toElementMap (h.toElementMap x))
    →
         h.toElementMap (F W₀ g₀) = F W₁ g₁) →
       ∀ {α : Type u} (V : NeighborhoodSystem α) (f : ApproximableMap V V),
         F V f = f.fixElement :=
  fun F hfixed huniform => fix_unique_of_uniform F hfixed huniform
```

4.5.23 No greatest fixed point

The identity endomap on Example 1.2 has every element fixed but no greatest among those fixed points.

Definition (`noGreatestFixedDomain`). The neighbourhood system for Exercise 4.12 is the Example 1.2 token domain.

Proof. By definition. ■

Theorem (`one_ne_zero`). The one and zero tokens are distinct.

Proof. Distinctness of numeric tokens. ■

Theorem (`one_ne_master`). The one token is not the master token.

Proof. Membership in the one token neighbourhood refutes zero lying in it. ■

Theorem (`zero_ne_master`). The zero token is not the master token.

Proof. Membership in the zero token neighbourhood refutes one lying in it. ■

Theorem (`elemOne_not_le_elemZero`). The one element does not lie below the zero element.

Proof. Any way-one below relation at the one token forces incompatibility with master or zero classification. ■

Theorem (`elemZero_not_le_elemOne`). The zero element does not lie below the one element.

Proof. Any way-zero below relation at the zero token forces incompatibility with master or one classification. ■

Theorem (`idMap_fixed`). Every element is fixed by the identity approximable map.

Proof. The identity element map fixes every element. ■

Theorem (`no_greatest_fixedPoint`). No element is a greatest common bound of all fixed points of the identity map.

Proof. Any purported greatest fixed point would have to dominate both the zero and one elements; token classification of elements rules out each of the three cases. ■

`noGreatestFixed.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.idMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.
    TwoTokenElements

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap TwoToken TwoToken.neighborhoodSystem

abbrev noGreatestFixedDomain : NeighborhoodSystem TwoToken.Token := TwoToken.
    neighborhoodSystem

theorem one_ne_zero : (TwoToken.one) = TwoToken.zero := by
  intro h
  rw [TwoToken.one, TwoToken.zero, Set.ext_iff] at h
  have h1 : (1 : TwoToken.Token) = 0 := (h 1).mp rfl
  exact absurd h1 (by decide)

theorem one_ne_master : (TwoToken.one) = TwoToken.master := by
  intro h
  rw [TwoToken.one, TwoToken.master, Set.ext_iff] at h
  have h0 : (0 : TwoToken.Token) ∈ ({1} : Set TwoToken.Token) := (h 0).mpr (Set.
    mem_univ 0)
  simp only [Set.mem_singleton_iff] at h0
  exact absurd h0 (by decide)

theorem zero_ne_master : (TwoToken.zero) = TwoToken.master := by
  intro h
  rw [TwoToken.zero, TwoToken.master, Set.ext_iff] at h
  have h1 : (1 : TwoToken.Token) ∈ ({0} : Set TwoToken.Token) := (h 1).mpr (Set.
    mem_univ 1)
  simp only [Set.mem_singleton_iff] at h1
  exact absurd h1 (by decide)
```

```
theorem elemOne_not_le_elemZero : ¬ elemOne ≤ elemZero := by
  intro h
  rcases h TwoToken.one (Or.inr rfl) with hm | hz
  · exact one_ne_master hm
  · exact one_ne_zero hz

theorem elemZero_not_le_elemOne : ¬ elemZero ≤ elemOne := by
  intro h
  rcases h TwoToken.zero (Or.inr rfl) with hm | ho
  · exact zero_ne_master hm
  · exact one_ne_zero ho.symm

theorem idMap_fixed (x : noGreatestFixedDomain.Element) :
    (idMap noGreatestFixedDomain).toElementMap x = x := toElementMap_idMap x

theorem no_greatest_fixedPoint :
    ¬ ∃ z : noGreatestFixedDomain.Element, (idMap noGreatestFixedDomain).toElementMap
    z = z ∧
      ∀ x, (idMap noGreatestFixedDomain).toElementMap x = x → x ≤ z := by
  rintro ⟨z, _, hz⟩
  have h0 : elemZero ≤ z := hz elemZero (idMap_fixed elemZero)
  have h1 : elemOne ≤ z := hz elemOne (idMap_fixed elemOne)
  rcases element_classification z with rfl | rfl | rfl
  · exact (bot_lt_elemZero).2 h0
  · exact elemOne_not_le_elemZero h1
  · exact elemZero_not_le_elemOne h0

end DomainTheory.NeighbourhoodSystems
```

4.5.24 Exercise 4.12 — no greatest fixed point

Need an approximable $f : D \to D$ have a maximum fixed point? Give an example where there are many fixed points.

The counterexample and non-existence of a greatest fixed point are with the No greatest fixed point peer.

Theorem (`example_412`). On Example 1.2, the identity map fixes every element, yet no element is a greatest fixed point.

Proof. Invoke the universal fixed-point law for the identity and the non-existence of a greatest common fixed point. ■

Exercise_4_12.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.
    noGreatestFixed

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap TwoToken TwoToken.neighborhoodSystem

theorem example_412 :
```

```
    (∀ (x : noGreatestFixedDomain.Element),
      (idMap noGreatestFixedDomain).toElementMap x = x) ∧
    (¬ ∃ z : noGreatestFixedDomain.Element,
      (idMap noGreatestFixedDomain).toElementMap z = z ∧
        ∀ x, (idMap noGreatestFixedDomain).toElementMap x = x → x ≤ z) :=
⟨@idMap_fixed, @no_greatest_fixedPoint⟩
```

4.5.25 Monotone fixed-point operator

Monotone endomaps admit a least fixed point as the infimum of pre-fixed points; primitive recursion on $\mathbb{N}$ is unique; chain unions lie below upper bounds.

Definition (`NeighborhoodSystem.preFix`). For an endomap on elements, the pre-fixed points are those with image below the point.

Proof. By definition. ■

Theorem (`NeighborhoodSystem.mem_preFix`). Membership in the pre-fixed set is equivalent to the pre-fix inequality.

Proof. By definition. ■

Definition (`NeighborhoodSystem.monoFix`). Given a pre-fixed seed, the monotone fixed point is the infimum of all pre-fixed points.

Proof. Nonempty pre-fixed set supplies the seed for the infimum. ■

Theorem (`NeighborhoodSystem.exists_unique_nat_rec`). For any set with seed and unary operation, there is a unique $\mathbb{N}$-indexed sequence starting at the seed and stepping by the operation.

Proof. Recursion defines a witness; extensionality on $\mathbb{N}$ proves uniqueness. ■

Theorem (`NeighborhoodSystem.monoFix_le_preFix`). The monotone fixed point lies below every pre-fixed point.

Proof. Infimum comparison against the pre-fixed set. ■

Theorem (`NeighborhoodSystem.monoFix_le`). The monotone fixed point lies below the chosen seed pre-fixed point.

Proof. Specialise the pre-fixed bound with the seed. ■

Theorem (`NeighborhoodSystem.monoFix_least`). The monotone fixed point is below every exact fixed point.

Proof. A fixed point is pre-fixed; apply the pre-fixed bound. ■

Theorem (`NeighborhoodSystem.monoFix_preFix`). Under monotonicity, the image of the monotone fixed point is again pre-fixed.

Proof. Every pre-fixed point dominates the fixed point; monotonicity transports the inequality into the pre-fixed set, which the fixed point infimum bounds from below. ■

Theorem (`NeighborhoodSystem.monoFix_isFixed`). Under monotonicity, the monotone fixed point from a pre-fixed seed is a fixed point.

Proof. Image below and above the fixed point from pre-fix lemmas, then antisymmetry. ■

Theorem (`NeighborhoodSystem.nat_iterate_unique`). The successor iteration starting at zero that adds one each step equals the identity on $\mathbb{N}$.

Proof. Induction on the index using the step and base hypotheses. ■

Theorem (`NeighborhoodSystem.chainUnion_le`). The chain union of a nonempty chain lies below any upper bound of its members.

Proof. Each member of the union comes from some chain element below the bound. ■

monoFix.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {α : Type*} {V : NeighborhoodSystem α}

namespace NeighborhoodSystem

def preFix (f : V.Element → V.Element) : Set V.Element := {x | f x ≤ x}

@[simp] theorem mem_preFix {f : V.Element → V.Element} {x : V.Element} :
    x ∈ preFix f ↔ f x ≤ x := Iff.rfl

def monoFix (f : V.Element → V.Element) {a : V.Element} (ha : f a ≤ a) : V.Element :=
  V.sInf (preFix f) ⟨a, ha⟩

theorem exists_unique_nat_rec {Z : Type*} (z : Z) (op : Z → Z) :
    ∃! s : ℕ → Z, s 0 = z ∧ ∀ n, s (n + 1) = op (s n) := by
  refine ⟨fun n => Nat.rec z (fun _ x => op x) n, ⟨rfl, fun _ => rfl⟩, ?_⟩
  rintro s ⟨h0, hsucc⟩
  funext n
  induction n with
  | zero => exact h0
  | succ k ih => rw [hsucc k, ih]

theorem monoFix_le_preFix (f : V.Element → V.Element) {a : V.Element} (ha : f a ≤ a)
    {x : V.Element} (hx : f x ≤ x) : monoFix f ha ≤ x :=
  V.sInf_le (preFix f) ⟨a, ha⟩ hx

theorem monoFix_le (f : V.Element → V.Element) {a : V.Element} (ha : f a ≤ a) :
    monoFix f ha ≤ a := monoFix_le_preFix f ha ha

theorem monoFix_least (f : V.Element → V.Element) {a : V.Element} (ha : f a ≤ a)
    {y : V.Element} (hy : f y = y) : monoFix f ha ≤ y :=
  monoFix_le_preFix f ha (le_of_eq hy)

theorem monoFix_preFix (f : V.Element → V.Element) (hf : Monotone f) {a : V.Element}
    (ha : f a ≤ a) : f (monoFix f ha) ≤ monoFix f ha := by
  apply V.le_sInf (preFix f) ⟨a, ha⟩
  intro x hx
  exact le_trans (hf (monoFix_le_preFix f ha hx)) hx

theorem monoFix_isFixed (f : V.Element → V.Element) (hf : Monotone f) {a : V.Element}
    (ha : f a ≤ a) : f (monoFix f ha) = monoFix f ha := by
  have hfb : f (monoFix f ha) ≤ monoFix f ha := monoFix_preFix f hf ha
  have hbf : monoFix f ha ≤ f (monoFix f ha) :=
    monoFix_le_preFix f ha (hf hfb)
```

```
  exact le_antisymm hfb hbf

theorem nat_iterate_unique {s : ℕ → ℕ} (h0 : s 0 = 0) (hsucc : ∀ n, s (n + 1) = s n +
    1) :
    ∀ n, s n = n := by
  intro n
  induction n with
  | zero => exact h0
  | succ k ih => rw [hsucc k, ih]

theorem chainUnion_le (C : Set V.Element) (hne : C.Nonempty) (hchain : IsChain (· ≤ ·)
    C)
    {y : V.Element} (hy : ∀ x ∈ C, x ≤ y) : V.chainUnion C hne hchain ≤ y := by
  rintro Z ⟨x, hxC, hxZ⟩
  exact hy x hxC Z hxZ

end NeighborhoodSystem
end
end DomainTheory.NeighbourhoodSystems
```

4.5.26 Exercise 4.13 — avoiding the natural numbers

The proof of 4.1 uses the integers, whereas the proof of 4.6 uses 4.1. There is a hint of circularity here! It can be eliminated by the following steps:

(1) If a domain D has an element a where, for $f : D \to D$, the relation $f(a) \sqsubseteq a$ holds, then the least fixed point can be defined by

$$\mathrm{fix}(f) = \bigcap \{x \in |D| \mid f(x) \sqsubseteq x\}.$$

Note that $\mathrm{fix}(f) \sqsubseteq a$. (Hint: Remark that by 1.17 the formula gives a well-defined element. Call the element b. Prove that $f(b) \sqsubseteq b$ by showing that $f(b) \sqsubseteq x$ whenever $f(x) \sqsubseteq x$. Then note that $f(f(b)) \sqsubseteq f(b)$ so that $b \sqsubseteq f(b)$ also. Conclude $b = \mathrm{fix}(f)$ as least fixed point.)

(2) Remark that this proof uses only the monotonicity property of $f : |D| \to |D|$. Remark, too, that (1) can always be applied to power-set domains PA for any set A.

(3) Review the proof of 4.6 and establish by a fixed-point method that for any structured set $\langle Z, z, \cdot \rangle$ there is a unique function $s : \mathbf{N} \to Z$ such that

(i) $s(0) = z$;

(ii) $s(n^+) = s(n)^{\cdot}$, for $n \in \mathbf{N}$.

(4) Employ (3) for the proof of 4.1 by identifying $\langle \mathbf{N}, 0, {}^+ \rangle$.

The monotone intersection fixed point, primitive recursion, and chain-union bound are with the Monotone fixed-point operator peer.

Theorem (`example_413`). A monotone endomap's infimum-of-pre-fixed-points is a fixed point; primitive recursion is unique; that fixed point is least among exact fixed points; and the standard $\mathbb{N}$ successor walk from zero is the identity.

Proof. Invoke the fixed-point, uniqueness, leastness, and standard iteration laws from the monotone fixed-point operator. ■

Exercise_4_13.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.monoFix

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

/-- Scott's Exercise 4.13: monotone fixpoints and natural-number recursion uniqueness.
    -/
theorem example_413 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : V.Element → V.Element),
      Monotone f → ∀ {a : V.Element} (ha : f a ≤ a),
        f (monoFix f ha) = monoFix f ha) ∧
    (∀ {Z : Type*} (z : Z) (op : Z → Z),
      ∃! s : ℕ → Z, s 0 = z ∧ ∀ n, s (n + 1) = op (s n)) ∧
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : V.Element → V.Element)
      {a : V.Element} (ha : f a ≤ a) {y : V.Element},
      f y = y → monoFix f ha ≤ y) ∧
    (∀ {s : ℕ → ℕ}, s 0 = 0 → (∀ n, s (n + 1) = s n + 1) → ∀ n, s n = n) :=
  ⟨@monoFix_isFixed, @exists_unique_nat_rec, @monoFix_least, @nat_iterate_unique⟩
```

4.5.27 Greatest powerset fixed point

Least and greatest fixed points of monotone set operators as intersections of pre-fixed points and unions of post-fixed points.

Definition (`gfpSet`). The greatest fixed-point set of a power-set endomap is the union of all post-fixed sets.

Proof. By definition. ■

Definition (`lfpSet`). The least fixed-point set of a power-set endomap is the intersection of all pre-fixed sets.

Proof. By definition. ■

Theorem (`lfpSet_subset`). The least fixed-point set lies inside every pre-fixed set.

Proof. Immediate from the universal quantifier in the definition. ■

Theorem (`f_subset_lfpSet`). Under monotonicity, the image of the least fixed-point set lies inside it.

Proof. Any pre-fixed set absorbs its image via monotonicity, so membership in the least fixed-point set propagates. ■

Theorem (`lfpSet_isFixed`). Under monotonicity, the least fixed-point set is a fixed point.

Proof. Subset inclusions both ways from the two preceding lemmas. ■

Theorem (`lfpSet_least`). The least fixed-point set lies inside every exact fixed set.

Proof. A fixed set is pre-fixed. ■

Theorem (`subset_gfpSet`). Every post-fixed set lies inside the greatest fixed-point set.

Proof. Exhibit the post-fixed set as witness in the union. ■

Theorem (`gfpSet_greatest`). Every exact fixed set lies inside the greatest fixed-point set.

Proof. A fixed set is post-fixed. ■

Theorem (`gfpSet_subset_f`). Under monotonicity, the greatest fixed-point set lies inside its image.

Proof. Witness membership via a post-fixed set contained in its image. ■

Theorem (`gfpSet_isFixed`). Under monotonicity, the greatest fixed-point set is a fixed point.

Proof. Subset inclusions both ways using post-fixed absorption and monotonicity. ■

gfpSet.lean

```
module

public import Mathlib.Order.BourbakiWitt

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

section
variable {A : Type*}

def gfpSet (f : Set A → Set A) : Set A := {a | ∃ x : Set A, x ⊆ f x ∧ a ∈ x}

def lfpSet (f : Set A → Set A) : Set A := {a | ∀ x : Set A, f x ⊆ x → a ∈ x}

theorem lfpSet_subset (f : Set A → Set A) {x : Set A} (hx : f x ⊆ x) : lfpSet f ⊆ x
    :=
  fun _ ha => ha x hx

theorem f_subset_lfpSet (f : Set A → Set A) (hf : Monotone f) :
    f (lfpSet f) ⊆ lfpSet f := by
  intro a ha x hx
  exact hx (hf (lfpSet_subset f hx) ha)

theorem lfpSet_isFixed (f : Set A → Set A) (hf : Monotone f) :
    f (lfpSet f) = lfpSet f := by
  have hsub : f (lfpSet f) ⊆ lfpSet f := f_subset_lfpSet f hf
  have hsup : lfpSet f ⊆ f (lfpSet f) := lfpSet_subset f (hf hsub)
  exact Set.Subset.antisymm hsub hsup

theorem lfpSet_least (f : Set A → Set A) {y : Set A} (hy : f y = y) :
    lfpSet f ⊆ y := lfpSet_subset f (le_of_eq hy)

theorem subset_gfpSet (f : Set A → Set A) {x : Set A} (hx : x ⊆ f x) : x ⊆ gfpSet f
    :=
  fun _ ha => ⟨x, hx, ha⟩

theorem gfpSet_greatest (f : Set A → Set A) {y : Set A} (hy : f y = y) :
    y ⊆ gfpSet f := subset_gfpSet f (le_of_eq hy.symm)

theorem gfpSet_subset_f (f : Set A → Set A) (hf : Monotone f) :
    gfpSet f ⊆ f (gfpSet f) := by
```

```
  rintro a ⟨x, hx, hax⟩
  exact hf (subset_gfpSet f hx) (hx hax)

theorem gfpSet_isFixed (f : Set A → Set A) (hf : Monotone f) :
    f (gfpSet f) = gfpSet f := by
  have hsub : gfpSet f ⊆ f (gfpSet f) := gfpSet_subset_f f hf
  have hsup : f (gfpSet f) ⊆ gfpSet f := subset_gfpSet f (hf hsub)
  exact Set.Subset.antisymm hsup hsub

end
end DomainTheory.NeighbourhoodSystems
```

4.5.28 Exercise 4.14 — greatest powerset fixed point

Need a monotone function $f : PA \to PA$ always have a *maximum* fixed point?

The greatest powerset fixed point and its fixed-point characterisation are with the Greatest powerset fixed point peer.

Theorem (`example_414`). Every exact fixed set lies inside the greatest fixed-point set, and under monotonicity that set is itself fixed.

Proof. Invoke the greatestness and fixed-point laws for the greatest powerset fixed point. ■

`Exercise_4_14.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.gfpSet

@[expose] public section

open DomainTheory.NeighbourhoodSystems

theorem example_414 :
    (∀ {A : Type*} (f : Set A → Set A) {y : Set A}, f y = y → y ⊆ gfpSet f) ∧
    (∀ {A : Type*} (f : Set A → Set A), Monotone f → f (gfpSet f) = gfpSet f) :=
  ⟨@gfpSet_greatest, @gfpSet_isFixed⟩
```

4.5.29 Maximal fixed points

Zorn's lemma on post-fixed points yields maximal fixed points; least fixed points follow from the monotone intersection operator; an infimum of fixed points is mapped below itself.

Theorem (`NeighborhoodSystem.exists_maximal_fixedPoint`). Every monotone endomap has a fixed point maximal among fixed points that lie above it.

Proof. Zorn on the set of post-fixed points, using chain unions for upper bounds and monotonicity to close under f. ■

Theorem (`NeighborhoodSystem.exists_least_fixedPoint`). Every monotone endomap has a least fixed point.

Proof. Build the monotone intersection fixed point from a maximal fixed point seed. ■

Theorem (`NeighborhoodSystem.f_sInf_le`). Under monotonicity, if every member of a nonempty fixed-point set is exactly fixed, then the image of their infimum lies below the infimum.

Proof. Compare the image to each member via monotonicity and the fixed-point identities. ■

`maximalFixed.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.monoFix
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {α : Type*} {V : NeighborhoodSystem α}
namespace NeighborhoodSystem

theorem exists_maximal_fixedPoint (f : V.Element → V.Element) (hf : Monotone f) :
    ∃ m, f m = m ∧ ∀ y, f y = y → m ≤ y → y ≤ m := by

  set S : Set V.Element := {x | x ≤ f x} with hS
  have ih : ∀ c ⊆ S, IsChain (· ≤ ·) c → ∃ ub ∈ S, ∀ z ∈ c, z ≤ ub := by
    intro c hcS hchain
    rcases c.eq_empty_or_nonempty with rfl | hne
    · refine ⟨⊥, ?_, fun z hz => ((Set.mem_empty_iff_false z).mp hz).elim⟩
      show (⊥ : V.Element) ≤ f ⊥
      exact _root_.bot_le
    · refine ⟨V.chainUnion c hne hchain, ?_, fun z hz => V.le_chainUnion c hne hchain
    hz⟩

      show V.chainUnion c hne hchain ≤ f (V.chainUnion c hne hchain)
      intro Z hZ
      obtain ⟨x, hxc, hxZ⟩ := hZ
      have hxle : x ≤ V.chainUnion c hne hchain := V.le_chainUnion c hne hchain hxc
      exact (le_trans (hcS hxc) (hf hxle)) Z hxZ
  obtain ⟨m, hm⟩ := zorn_le₀ S ih
  have hms : m ≤ f m := hm.1
  have hfm_le_m : f m ≤ m := hm.2 (hf hms) hms
  refine ⟨m, le_antisymm hfm_le_m hms, fun y hy hmy => hm.2 ?_ hmy⟩
  exact le_of_eq hy.symm

theorem exists_least_fixedPoint (f : V.Element → V.Element) (hf : Monotone f) :
    ∃ b, f b = b ∧ ∀ y, f y = y → b ≤ y := by
  obtain ⟨m, hfm, _⟩ := exists_maximal_fixedPoint f hf
  exact ⟨monoFix f (le_of_eq hfm), monoFix_isFixed f hf (le_of_eq hfm),
    fun y hy => monoFix_least f (le_of_eq hfm) hy⟩

theorem f_sInf_le (f : V.Element → V.Element) (hf : Monotone f) (S : Set V.Element)
    (hS : S.Nonempty) (hfix : ∀ s ∈ S, f s = s) : f (V.sInf S hS) ≤ V.sInf S hS := by
  apply V.le_sInf
  intro s hsS
  calc f (V.sInf S hS) ≤ f s := hf (V.sInf_le S hS hsS)
    _ = s := hfix s hsS
```

```
end NeighborhoodSystem
end
end DomainTheory.NeighbourhoodSystems
```

4.5.30 Exercise 4.15 — maximal fixed points

(For set theorists.) Let $f : |D| \to |D|$ be a monotone function on (the elements of) a domain. Show that f has a *maximal* fixed point (i.e. a fixed point that cannot be extended to a larger fixed point). (Hint: By Zorn's Lemma consider a maximal chain

$$C \subseteq \{x \in |D| \mid x \sqsubseteq f(x)\},$$

and use 2.11 to remark that $\bigsqcup C \in |D|$.) Now argue that f has a least fixed point.

Maximal and least fixed points for monotone endomaps, and the fixed-point inequality below an infimum of fixed points, are with the Maximal fixed points peer.

Theorem (`example_415`). Every monotone endomap has a least fixed point and a maximal fixed point that cannot be extended upward among fixed points.

Proof. Invoke the least and maximal existence laws from the maximal fixed-points peer. ■

`Exercise_4_15.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.maximalFixed

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

/-- Scott's Exercise 4.15: least and maximal fixed points of monotone maps. -/
theorem example_415 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : V.Element → V.Element),
      Monotone f → ∃ b, f b = b ∧ ∀ y, f y = y → b ≤ y) ∧
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : V.Element → V.Element),
      Monotone f → ∃ m, f m = m ∧ ∀ y, f y = y → m ≤ y → y ≤ m) :=
  ⟨@exists_least_fixedPoint, @exists_maximal_fixedPoint⟩
```

4.5.31 Optimal fixed point

The optimal fixed point over a nonempty set of exact fixed points is the monotone least fixed point above the infimum fixed-point bound.

Definition (`NeighborhoodSystem.optimalFix`). Given a nonempty family of exact fixed points, the optimal fixed point is the monotone fixed point built from the infimum fixed-point inequality.

Proof. Well-defined from the seed inequality supplied for the infimum. ■

Theorem (`NeighborhoodSystem.optimalFix_isFixed`). The optimal fixed point is an exact fixed point.

Proof. Apply the monotone fixed-point fixed-point law. ■

Theorem (`NeighborhoodSystem.optimalFix_le_sInf`). The optimal fixed point lies below the infimum of the given fixed-point set.

Proof. The monotone fixed point lies below its seed. ■

Theorem (`NeighborhoodSystem.optimalFix_le`). The optimal fixed point lies below every member of the fixed-point set.

Proof. Transitivity through the infimum bound. ■

Theorem (`NeighborhoodSystem.optimalFix_consistent`). The optimal fixed point shares an upper bound with every member of the fixed-point set.

Proof. Each member bounds the optimal point from above. ■

`optimalFix.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.maximalFixed
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.monoFix

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {α : Type*} {V : NeighborhoodSystem α}
namespace NeighborhoodSystem

def optimalFix (f : V.Element → V.Element) (hf : Monotone f) (S : Set V.Element) (hS :
     S.Nonempty)
    (hfix : ∀ s ∈ S, f s = s) : V.Element :=
  monoFix f (f_sInf_le f hf S hS hfix)

theorem optimalFix_isFixed (f : V.Element → V.Element) (hf : Monotone f) (S : Set V.
    Element)
    (hS : S.Nonempty) (hfix : ∀ s ∈ S, f s = s) :
    f (optimalFix f hf S hS hfix) = optimalFix f hf S hS hfix :=
  monoFix_isFixed f hf _

theorem optimalFix_le_sInf (f : V.Element → V.Element) (hf : Monotone f) (S : Set V.
    Element)
    (hS : S.Nonempty) (hfix : ∀ s ∈ S, f s = s) :
    optimalFix f hf S hS hfix ≤ V.sInf S hS :=
  monoFix_le f _

theorem optimalFix_le (f : V.Element → V.Element) (hf : Monotone f) (S : Set V.Element
    )
    (hS : S.Nonempty) (hfix : ∀ s ∈ S, f s = s) {s : V.Element} (hs : s ∈ S) :
    optimalFix f hf S hS hfix ≤ s :=
  le_trans (optimalFix_le_sInf f hf S hS hfix) (V.sInf_le S hS hs)

theorem optimalFix_consistent (f : V.Element → V.Element) (hf : Monotone f) (S : Set V
    .Element)
    (hS : S.Nonempty) (hfix : ∀ s ∈ S, f s = s) {s : V.Element} (hs : s ∈ S) :
    ∃ ub, optimalFix f hf S hS hfix ≤ ub ∧ s ≤ ub :=
  ⟨s, optimalFix_le f hf S hS hfix hs, le_refl s⟩
```

```
end NeighborhoodSystem
end
end DomainTheory.NeighbourhoodSystems
```

4.5.32 Exercise 4.16 — optimal fixed point

(For fixed-point nuts). Show that a monotone function as in 4.15 has an "optimal" fixed point in the sense that it is the greatest fixed point below all the maximal fixed points and at the same time it is the largest fixed point consistent with all other fixed points. *Consistency* for sets of elements means having a common upper bound. (Hint: Follow these steps:

(1) Show that any non-empty set S of fixed points has a largest fixed point below by using the formula

$$f(\bigcap S) \sqsubseteq \bigcap S$$

and finding the least fixed point over $\bigcap S$.

(2) Letting a be the fixed point of (1) constructed from the set of maximal fixed points, remark that a is consistent with any other fixed point $x = f(x)$, since x can be extended to a maximal one. Suppose b is consistent with all fixed points, then $b \sqsubseteq y$ if y is maximal. (Why?).)

The infimum-of-fixed-points inequality and the optimal fixed point below every member of a fixed-point set are with the Optimal fixed point peer.

Theorem (`example_416`). For a nonempty set of fixed points of a monotone endomap, the image of the infimum lies below the infimum; the optimal fixed point is consistent with each member and lies below each.

Proof. Invoke the infimum fixed-point bound and the optimal fixed-point consistency and comparison laws. ■

`Exercise_4_16.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.optimalFix

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

/-- Scott's Exercise 4.16: optimal fixed points of monotone maps on nonempty fixed
    sets. -/
theorem example_416 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : V.Element → V.Element),
      Monotone f → ∀ (S : Set V.Element) (hS : S.Nonempty),
        (∀ s ∈ S, f s = s) → f (V.sInf S hS) ≤ V.sInf S hS) ∧
    (∀ {α : Type*} {V : NeighborhoodSystem α} (f : V.Element → V.Element)
      (hf : Monotone f) (S : Set V.Element) (hS : S.Nonempty)
      (hfix : ∀ s ∈ S, f s = s) {s : V.Element},
```

```
    s ∈ S → ∃ ub, optimalFix f hf S hS hfix ≤ ub ∧ s ≤ ub) ∧
  (∀ {α : Type*} {V : NeighborhoodSystem α} (f : V.Element → V.Element)
    (hf : Monotone f) (S : Set V.Element) (hS : S.Nonempty)
    (hfix : ∀ s ∈ S, f s = s) {s : V.Element},
    s ∈ S → optimalFix f hf S hS hfix ≤ s) :=
⟨@f_sInf_le, @optimalFix_consistent, @optimalFix_le⟩
```

4.5.33 Generated submonoid

The set operator $F\,a\,b\,x = \{1\} \cup \{a, b\} \cup x \cdot x$ on a monoid's power set is monotone; its least fixed point is the submonoid generated by $\{a, b\}$, and fixed points need not be unique.

Definition (`F`). The generated-submonoid operator unions the unit, $\{a, b\}$, and the set product $x \cdot x$.

Proof. By definition. ■

Theorem (`base_subset_F`). The unit and generators sit inside the operator image.

Proof. Left-hand inclusion of the union. ■

Theorem (`one_mem_F`). The unit lies in the operator image.

Proof. Via the base inclusion. ■

Theorem (`self_mul_subset_F`). The set square $x \cdot x$ lies inside the operator image.

Proof. Right-hand inclusion of the union. ■

Definition (`preFixSubmonoid`). A pre-fixed carrier yields a submonoid structure.

Proof. Closure under unit and multiplication from pre-fixed absorption. ■

Theorem (`F_monotone`). The operator is monotone on subsets.

Proof. Union and pointwise product respect inclusion. ■

Theorem (`F_one_one_singleton`). On $\mathbb{N}$, $F\,1\,1$ fixes the singleton $\{1\}$.

Proof. Evaluate the union and singleton product. ■

Theorem (`F_univ`). On any monoid, $F\,a\,b$ fixes the full set.

Proof. Every element lies in $x \cdot x$ when x is universal. ■

Theorem (`a_mem_F`). Generator a lies in the operator image.

Proof. Membership in the middle union piece. ■

Theorem (`b_mem_F`). Generator b lies in the operator image.

Proof. Membership in the middle union piece. ■

Theorem (`fixedPoint_not_unique`). Distinct fixed sets $\{1\}$ and $\mathbb{N}$ both satisfy $F\,1\,1\,x = x$ on $\mathbb{N}$.

Proof. Direct evaluation and separation at 2. ■

Theorem (`leastSolution_eq`). The least fixed-point set is fixed by $F\,a\,b$.

Proof. Apply the least fixed-point identity under monotonicity. ■

Theorem (`lfpSet_eq_closure`). The least fixed set equals the submonoid closure of $\{a, b\}$.

Proof. Closure absorbs the operator; the least fixed set contains the generators and hence the closure. ■

`genSubmonoid.lean`

```
module

public import Mathlib.Algebra.Group.Nat.Defs
public import Mathlib.Algebra.Group.Pointwise.Set.Basic
public import Mathlib.Algebra.Group.Submonoid.Basic
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.gfpSet
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open Pointwise

section
variable {S : Type*} [Monoid S]

def F (a b : S) (x : Set S) : Set S := {1} ∪ {a, b} ∪ x * x

theorem base_subset_F (a b : S) (x : Set S) : {1} ∪ {a, b} ⊆ F a b x :=
  Set.subset_union_left

theorem one_mem_F (a b : S) (x : Set S) : (1 : S) ∈ F a b x :=
  base_subset_F a b x (Set.mem_union_left _ rfl)

theorem self_mul_subset_F (a b : S) (x : Set S) : x * x ⊆ F a b x :=
  Set.subset_union_right

def preFixSubmonoid (a b : S) {x : Set S} (hx : F a b x ⊆ x) : Submonoid S where
  carrier := x
  one_mem' := hx (one_mem_F a b x)
  mul_mem' := by
    intro s t hs ht
    exact hx (self_mul_subset_F a b x (Set.mul_mem_mul hs ht))

theorem F_monotone (a b : S) : Monotone (F a b) := by
  intro x y hxy
  exact Set.union_subset_union (le_refl _) (Set.mul_subset_mul hxy hxy)

theorem F_one_one_singleton : F (1 : ℕ) 1 ({1} : Set ℕ) = {1} := by
  unfold F
  rw [Set.singleton_mul_singleton]
  simp

theorem F_univ (a b : S) : F a b (Set.univ : Set S) = Set.univ := by
  apply Set.Subset.antisymm (Set.subset_univ _)
  intro s _
  exact self_mul_subset_F a b _ (Set.mem_mul.mpr ⟨1, trivial, s, trivial, one_mul s⟩)

theorem a_mem_F (a b : S) (x : Set S) : a ∈ F a b x :=
  base_subset_F a b x (Set.mem_union_right _ (Set.mem_insert _ _))

theorem b_mem_F (a b : S) (x : Set S) : b ∈ F a b x :=
  base_subset_F a b x (Set.mem_union_right _ (Set.mem_insert_of_mem _ rfl))

theorem fixedPoint_not_unique :
    ∃ x y : Set ℕ, F 1 1 x = x ∧ F 1 1 y = y ∧ x = y := by
  refine ⟨{1}, Set.univ, F_one_one_singleton, F_univ 1 1, ?_⟩
  intro h
  have h2 : (2 : ℕ) ∈ ({1} : Set ℕ) := by rw [h]; exact Set.mem_univ 2
  simp at h2

theorem leastSolution_eq (a b : S) :
    F a b (lfpSet (F a b)) = lfpSet (F a b) :=
  lfpSet_isFixed (F a b) (F_monotone a b)
```

```
theorem lfpSet_eq_closure (a b : S) :
    lfpSet (F a b) = (Submonoid.closure ({a, b} : Set S) : Set S) := by
  apply Set.Subset.antisymm
  ·
    have hclosed : F a b (Submonoid.closure ({a, b} : Set S)) ⊆
        (Submonoid.closure ({a, b} : Set S) : Set S) := by
      apply Set.union_subset
      · apply Set.union_subset
        · rintro s rfl; exact (Submonoid.closure ({a, b} : Set S)).one_mem
        · intro s hs
          rcases hs with rfl | rfl
          · exact Submonoid.subset_closure (Set.mem_insert _ _)
          · exact Submonoid.subset_closure (Set.mem_insert_of_mem _ rfl)
      · rintro s ⟨t, ht, u, hu, rfl⟩
        exact (Submonoid.closure ({a, b} : Set S)).mul_mem ht hu
    exact lfpSet_subset (F a b) hclosed
  ·
    have hfix : F a b (lfpSet (F a b)) ⊆ lfpSet (F a b) :=
      le_of_eq (leastSolution_eq a b)
    have ha : a ∈ lfpSet (F a b) := hfix (a_mem_F a b _)
    have hb : b ∈ lfpSet (F a b) := hfix (b_mem_F a b _)
    have hmem : ({a, b} : Set S) ⊆ lfpSet (F a b) := by
      intro s hs
      rcases Set.mem_insert_iff.mp hs with h | h
      · rw [h]; exact ha
      · rw [Set.mem_singleton_iff.mp h]; exact hb
    exact (Submonoid.closure_le (S := preFixSubmonoid a b hfix)).mpr hmem

end
end DomainTheory.NeighbourhoodSystems
```

4.5.34 Exercise 4.17 — generated submonoid

(For algebraists). Suppose $\langle S, 1, \cdot \rangle$ is a semi-group with unit (sometimes called a monoid). Remark that PS is a domain. For $a, b \in S$, what is the least $x \in PS$ such that

$$x = \{1\} \cup \{a, b\} \cup x \cdot x,$$

where in general for $x, y \subseteq S$

$$x \cdot y = \{t \cdot u \mid t \in x \text{ and } u \in y\}?$$

Need the fixed point be unique?

The powerset operator, non-uniqueness of fixed points, and identification of the least solution with the submonoid generated by $\{a, b\}$ are with the Generated submonoid peer.

Theorem (`example_417`). On $\mathbb{N}$, the operator $F\,1\,1$ has distinct fixed points $\{1\}$ and $\mathbb{N}$; for any monoid, the least fixed set of $F\,a\,b$ equals the submonoid closure of $\{a, b\}$.

Proof. Exhibit the two fixed points on $\mathbb{N}$, then invoke the closure identification for general a, b. ■

`Exercise_4_17.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.genSubmonoid

@[expose] public section

open DomainTheory.NeighbourhoodSystems

theorem example_417 :
    (∃ x y : Set ℕ, F 1 1 x = x ∧ F 1 1 y = y ∧ x = y) ∧
    (∀ {S : Type*} [Monoid S] (a b : S),
      lfpSet (F a b) = (Submonoid.closure ({a, b} : Set S) : Set S)) :=
  ⟨fixedPoint_not_unique, @lfpSet_eq_closure⟩
```

4.5.35 Number-domain checks

Elements of the number neighbourhood domain are bottom or principal numerals; numerals and the successor approximable map are injective on indices; zero is distinct from successors.

Theorem (`singleton_inter_eq_empty`). Distinct singleton neighbourhood sets have empty intersection.

Proof. Extensionality on indices. ■

Theorem (`natElem_injective`). Equal numerals determine equal natural numbers.

Proof. Read membership in one numeral's singleton neighbourhood. ■

Theorem (`natElem_zero_ne_succ`). Zero and any positive numeral are unequal.

Proof. Transfer injectivity through successor indices. ■

Theorem (`not_N_mem_empty`). The empty set is not a valid neighbourhood.

Proof. Cases on the membership predicate. ■

Theorem (`element_classification`). Every element is bottom or a numeral.

Proof. Case on presence of a singleton neighbourhood; otherwise master and bot axioms force bottom. ■

Theorem (`succMap_injective`). Equal successor images imply equal indices.

Proof. Compare numerals after unfolding successor on numerals. ■

Theorem (`zero_ne_succMap`). Zero is not the successor image of a numeral.

Proof. Reduce to the zero-versus-successor numeral distinction. ■

`numberDomainCheck.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.numberDomain

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap NatDom

theorem singleton_inter_eq_empty {n m : ℕ} (hmn : m = n) :
    ({n} : Set ℕ) ∩ {m} = ∅ := by
  ext k
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false, not_and]
```

```
  rintro rfl h2
  exact hmn h2.symm

theorem natElem_injective {n m : ℕ} (h : natElem n = natElem m) : n = m := by
  have hmem : (natElem m).mem {n} := h ▸ (mem_natElem_iff.mpr (Or.inr rfl))
  rw [mem_natElem_iff] at hmem
  rcases hmem with hu | hs
  · exact absurd hu (singleton_ne_univ n)
  · exact singleton_nat_inj hs

theorem natElem_zero_ne_succ (k : ℕ) : natElem 0 = natElem (k + 1) := fun h =>
  Nat.succ_ne_zero k (natElem_injective h).symm

theorem not_N_mem_empty : ¬ N.mem (∅ : Set ℕ) := by
  rw [N_mem]
  rintro (h | ⟨n, h⟩)
  · exact Set.empty_ne_univ h
  · exact (Set.singleton_ne_empty n) h.symm

theorem element_classification (x : N.Element) :
    x = N.bot ∨ ∃ n, x = natElem n := by
  by_cases h : ∃ n, x.mem {n}
  · obtain ⟨n, hn⟩ := h
    refine Or.inr ⟨n, ?_⟩
    apply Element.ext
    intro Y
    rw [mem_natElem_iff]
    constructor
    · intro hY
      rcases N_mem.mp (x.sub hY) with rfl | ⟨m, rfl⟩
      · exact Or.inl rfl
      · by_cases hmn : m = n
        · subst hmn; exact Or.inr rfl
        · exfalso
          have hmem := x.inter_mem hn hY
          rw [singleton_inter_eq_empty hmn] at hmem
          exact not_N_mem_empty (x.sub hmem)
    · rintro (rfl | rfl)
      · exact x.master_mem
      · exact hn
  · refine Or.inl ?_
    apply Element.ext
    intro Y
    rw [N_bot_mem]
    constructor
    · intro hY
      rcases N_mem.mp (x.sub hY) with rfl | ⟨m, rfl⟩
      · rfl
      · exact absurd hY (fun hm => h ⟨m, hm⟩)
    · rintro rfl
      exact x.master_mem

theorem succMap_injective {n m : ℕ}
    (h : succMap.toElementMap (natElem n) = succMap.toElementMap (natElem m)) : n = m
    := by
  rw [succMap_natElem, succMap_natElem] at h
```

```
  have := natElem_injective h
  omega

theorem zero_ne_succMap (n : ℕ) : natElem 0 = succMap.toElementMap (natElem n) := by
  rw [succMap_natElem]
  exact natElem_zero_ne_succ n

end DomainTheory.NeighbourhoodSystems
```

4.5.36 Exercise 4.18 — checking the number domain

In Example 4.3 there are many unproved assertions about N and F. These should be checked. In particular, the isomorphism theorem of 4.6 could be proved by constructing a simple domain M from $\mathbf{M}$ in the way N is constructed from $\mathbf{N}$.

Classification of number-domain elements, injectivity of numerals and the successor map, and distinguished constants are with the Number-domain checks peer.

Theorem (`example_418`). Each element of the number domain is bottom or a numeral; numerals and successor action are injective; zero is not a successor numeral nor the successor image of a numeral.

Proof. Combine element classification with the numeral, successor, and constant distinction lemmas from the number-domain checks. ■

`Exercise_4_18.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.
    numberDomainCheck

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap NatDom

theorem example_418 :
    (∀ x : N.Element, x = N.bot ∨ ∃ n, x = natElem n) ∧
    (∀ {n m : ℕ}, natElem n = natElem m → n = m) ∧
    (∀ k : ℕ, natElem 0 = natElem (k + 1)) ∧
    (∀ {n m : ℕ},
      succMap.toElementMap (natElem n) = succMap.toElementMap (natElem m) → n = m) ∧
    (∀ n : ℕ, natElem 0 = succMap.toElementMap (natElem n)) :=
  ⟨element_classification, @natElem_injective, natElem_zero_ne_succ,
    @succMap_injective, zero_ne_succMap⟩
```

4.5.37 Peano structure on sequences

Head-test maps, unary numerals, and Peano axioms on the sequence domain.

Definition (`headValC`). Head-test evaluation on strings for cone versus singleton branches.

Proof. As in the formal module. ■ **Theorem (`headValC_nil`).** Head-test on the empty string returns the default.

Proof. As in the formal module. ■ **Theorem (`headValC_false`).** Head-test on a false prefix returns the first branch.

Proof. As in the formal module. ■ **Theorem (`headValC_true`).** Head-test on a true prefix returns the second branch.

Proof. As in the formal module. ■ **Definition (`T`).** The two-element token domain for sequence values.

Proof. As in the formal module. ■ **Theorem (`headValC_bot_le`).** Head-test on bottom prefixes is monotone in the prefix order.

Proof. As in the formal module. ■ **Theorem (`not_cone_subset_singleton`).** A cone neighbourhood is never contained in a singleton.

Proof. As in the formal module. ■ **Theorem (`cone_ne_singleton`).** Cone and singleton neighbourhoods are distinct.

Proof. As in the formal module. ■ **Definition (`liftC`).** Universal approximable map from the sequence domain using head-test values.

Proof. As in the formal module. ■ **Definition (`emptyMap`).** Constant false head-tests with bottom on cones.

Proof. As in the formal module. ■ **Theorem (`liftC_strBot`).** The lift sends stratum bottoms to cone values.

Proof. As in the formal module. ■ **Theorem (`liftC_strElem`).** The lift sends stratum elements to singleton values.

Proof. As in the formal module. ■ **Theorem (`emptyMap_strElem`).** Empty map values on stratum elements.

Proof. As in the formal module. ■ **Theorem (`emptyMap_strBot`).** Empty map values on stratum bottoms.

Proof. As in the formal module. ■ **Definition (`oneMap`).** Map sending cones to bottom/false/true pattern.

Proof. As in the formal module. ■ **Definition (`zeroMap`).** Map with true on singleton tails and mixed cones.

Proof. As in the formal module. ■ **Theorem (`zeroMap_strElem`).** Zero map on stratum elements.

Proof. As in the formal module. ■ **Theorem (`zeroMap_strBot`).** Zero map on stratum bottoms.

Proof. As in the formal module. ■ **Theorem (`oneMap_strElem`).** One map on stratum elements.

Proof. As in the formal module. ■ **Theorem (`oneMap_strBot`).** One map on stratum bottoms.

Proof. As in the formal module. ■ **Definition (`oneDef`).** Fixed-point-style definition of the numeral one map.

Proof. As in the formal module. ■ **Theorem (`condT_bot`).** Conditional with bottom first argument is bottom.

Proof. As in the formal module. ■ **Theorem (`one_def_strBot`).** The simple one map agrees with the fixed-point definition on stratum bottoms.

Proof. As in the formal module. ■ **Theorem (`one_def_strElem`).** The simple one map agrees with the fixed-point definition on stratum elements.

Proof. As in the formal module. ■ **Theorem (`peano_cons_disjoint`).** Prefixes false and true yield distinct strings.

Proof. As in the formal module. ■ **Theorem (`peano_cons_injective`).** Equality of conses implies equality of tails.

Proof. As in the formal module. ■ **Theorem (`peano_induction`).** Induction on finite bit-strings.

Proof. As in the formal module. ■ **Theorem (`peano_nil_ne_cons`).** The empty string is not a cons.

Proof. As in the formal module. ■

seqPeano.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.cond
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.seqDomain

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.SeqPeano
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap BitDom
    SeqDom

section
variable {β : Type*}

notation "?" => Truth.trueElt

notation "?" => Truth.falseElt

def headValC (V : NeighborhoodSystem β) (z a0 a1 : V.Element) : Str → V.Element
  | [] => z
  | false :: _ => a0
  | true :: _ => a1

@[simp] theorem headValC_nil (V : NeighborhoodSystem β) (z a0 a1 : V.Element) :
    headValC V z a0 a1 [] = z := rfl

@[simp] theorem headValC_false (V : NeighborhoodSystem β) (z a0 a1 : V.Element) (σ :
    Str) :
    headValC V z a0 a1 (false :: σ) = a0 := rfl

@[simp] theorem headValC_true (V : NeighborhoodSystem β) (z a0 a1 : V.Element) (σ :
    Str) :
    headValC V z a0 a1 (true :: σ) = a1 := rfl

abbrev T : NeighborhoodSystem TwoToken.Token := Truth.T

theorem headValC_bot_le (V : NeighborhoodSystem β) (z' a0 a1 : V.Element) {σ τ : Str}
    (h : σ <+: τ) : headValC V V.bot a0 a1 σ ≤ headValC V z' a0 a1 τ := by
  cases σ with
  | nil => exact bot_le
  | cons b σ0 =>
    obtain ⟨s, rfl⟩ := h
    cases b
    · rw [List.cons_append]; exact le_of_eq rfl
    · rw [List.cons_append]; exact le_of_eq rfl
end

theorem not_cone_subset_singleton (τ σ : Str) : ¬ cone τ ⊆ ({σ} : Set Str) := by
```

```
  intro h
  have h1 : τ ∈ ({σ} : Set Str) := h (by simp [cone.mem])
  have h2 : (τ ++ [true]) ∈ ({σ} : Set Str) := h (by simp [cone.mem])
  rw [Set.mem_singleton_iff] at h1 h2
  have : τ = τ ++ [true] := h1.trans h2.symm
  simp at this

theorem cone_ne_singleton (τ σ : Str) : cone τ = ({σ} : Set Str) := by
  intro h
  exact not_cone_subset_singleton τ σ (h ▶ subset_rfl)

section
variable {β : Type*}

def liftC (V : NeighborhoodSystem β) (coneVal singVal : Str → V.Element)
    (hcone : ∀ {σ τ : Str}, σ <+: τ → coneVal σ ≤ coneVal τ)
    (hsing : ∀ {σ τ : Str}, σ <+: τ → coneVal σ ≤ singVal τ) :
    ApproximableMap C V where
  rel X Y := (∃ σ, X = cone σ ∧ (coneVal σ).mem Y) ∨ (∃ σ, X = {σ} ∧ (singVal σ).mem Y
    )
  rel_dom := by
    rintro X Y (⟨σ, rfl, _⟩ | ⟨σ, rfl, _⟩)
    · exact memC_cone σ
    · exact memC_singleton σ
  rel_cod := by
    rintro X Y (⟨σ, _, hY⟩ | ⟨σ, _, hY⟩)
    · exact (coneVal σ).sub hY
    · exact (singVal σ).sub hY
  master_rel := by
    refine Or.inl ⟨[], ?_, (coneVal []).master_mem⟩
    rw [C_master]; exact cone.nil.symm
  inter_right := by
    rintro X Y Y' (⟨σ, rfl, hY⟩ | ⟨σ, rfl, hY⟩) (⟨σ', hX', hY'⟩ | ⟨σ', hX', hY'⟩)
    · have : σ = σ' := cone.injective hX'
      subst this
      exact Or.inl ⟨σ, rfl, (coneVal σ).inter_mem hY hY'⟩
    · exact absurd hX' (cone_ne_singleton σ σ')
    · exact absurd hX'.symm (cone_ne_singleton σ' σ)
    · have : σ = σ' := by rw [Set.singleton_eq_singleton_iff] at hX'; exact hX'
      subst this
      exact Or.inr ⟨σ, rfl, (singVal σ).inter_mem hY hY'⟩
  mono := by
    rintro X X' Y Y' (⟨σ, rfl, hY⟩ | ⟨σ, rfl, hY⟩) hX'X hYY' hX' hY'
    ·
      rcases hX' with ⟨τ, rfl⟩ | ⟨τ, rfl⟩
      ·
        have hpre : σ <+: τ := cone.subset_cone.mp hX'X
        exact Or.inl ⟨τ, rfl, (coneVal τ).up_mem (hcone hpre Y hY) hY' hYY'⟩
      ·
        have hpre : σ <+: τ := singleton_subset_cone.mp hX'X
        exact Or.inr ⟨τ, rfl, (singVal τ).up_mem (hsing hpre Y hY) hY' hYY'⟩
    ·
      rcases hX' with ⟨τ, rfl⟩ | ⟨τ, rfl⟩
      · exact absurd hX'X (not_cone_subset_singleton τ σ)
      · have hτσ : τ = σ := by
          have := Set.singleton_subset_iff.mp hX'X
```

```
          rwa [Set.mem_singleton_iff] at this
        subst hτσ
        exact Or.inr ⟨τ, rfl, (singVal τ).up_mem hY hY' hYY'⟩

def emptyMap : ApproximableMap C T :=
  liftC T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?)

theorem liftC_strBot (V : NeighborhoodSystem β) (coneVal singVal : Str → V.Element)
    (hcone : ∀ {σ τ : Str}, σ <+: τ → coneVal σ ≤ coneVal τ)
    (hsing : ∀ {σ τ : Str}, σ <+: τ → coneVal σ ≤ singVal τ) (σ : Str) :
    (liftC V coneVal singVal hcone hsing).toElementMap (strBot σ) = coneVal σ := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨X, ⟨_, hsub⟩, hrel⟩
    rcases hrel with ⟨σ', hXcone, hY⟩ | ⟨σ', hXsing, hY⟩
    · have hpre : σ' <+: σ := cone.subset_cone.mp (hXcone ▸ hsub)
      exact hcone hpre Y hY
    · exact absurd (hXsing ▸ hsub) (not_cone_subset_singleton σ σ')
  · intro hY
    exact ⟨cone σ, ⟨memC_cone σ, subset_rfl⟩, Or.inl ⟨σ, rfl, hY⟩⟩

theorem liftC_strElem (V : NeighborhoodSystem β) (coneVal singVal : Str → V.Element)
    (hcone : ∀ {σ τ : Str}, σ <+: τ → coneVal σ ≤ coneVal τ)
    (hsing : ∀ {σ τ : Str}, σ <+: τ → coneVal σ ≤ singVal τ) (σ : Str) :
    (liftC V coneVal singVal hcone hsing).toElementMap (strElem σ) = singVal σ := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨X, ⟨_, hsub⟩, hrel⟩
    rcases hrel with ⟨σ', hXcone, hY⟩ | ⟨σ', hXsing, hY⟩
    · have hpre : σ' <+: σ := by
        apply singleton_subset_cone.mp
        rw [← hXcone]; exact hsub
      exact hsing hpre Y hY
    · have hσσ' : σ = σ' := by
        have := Set.singleton_subset_iff.mp (hXsing ▸ hsub)
        rwa [Set.mem_singleton_iff] at this
      subst hσσ'; exact hY
  · intro hY
    exact ⟨{σ}, ⟨memC_singleton σ, subset_rfl⟩, Or.inr ⟨σ, rfl, hY⟩⟩

@[simp] theorem emptyMap_strElem (σ : Str) :
    emptyMap.toElementMap (strElem σ) = headValC T ? ? ? σ :=
  liftC_strElem T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) σ

@[simp] theorem emptyMap_strBot (σ : Str) :
    emptyMap.toElementMap (strBot σ) = headValC T T.bot ? ? σ :=
  liftC_strBot T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) σ

def oneMap : ApproximableMap C T :=
  liftC T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?)
```

```
def zeroMap : ApproximableMap C T :=
  liftC T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?)

@[simp] theorem zeroMap_strElem (σ : Str) :
    zeroMap.toElementMap (strElem σ) = headValC T ? ? ? σ :=
  liftC_strElem T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) σ

@[simp] theorem zeroMap_strBot (σ : Str) :
    zeroMap.toElementMap (strBot σ) = headValC T T.bot ? ? σ :=
  liftC_strBot T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) σ

@[simp] theorem oneMap_strElem (σ : Str) :
    oneMap.toElementMap (strElem σ) = headValC T ? ? ? σ :=
  liftC_strElem T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) σ

@[simp] theorem oneMap_strBot (σ : Str) :
    oneMap.toElementMap (strBot σ) = headValC T T.bot ? ? σ :=
  liftC_strBot T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) σ

def oneDef (x : C.Element) : T.Element :=
  (Cond.cond T).toElementMap
    (pair (emptyMap.toElementMap x)
      (pair ?
        ((Cond.cond T).toElementMap (pair (zeroMap.toElementMap x) (pair ? ?)))))

theorem condT_bot (x y : T.Element) :
    (Cond.cond T).toElementMap (pair T.bot (pair x y)) = T.bot := by
  apply Element.ext
  intro Z
  rw [Cond.cond_toElementMap_mem]
  constructor
  · rintro (⟨h0, _⟩ | ⟨h1, _⟩ | rfl)
    · rw [NeighborhoodSystem.mem_bot] at h0; exact absurd h0 Cond.zero_ne_master
    · rw [NeighborhoodSystem.mem_bot] at h1; exact absurd h1 Cond.one_ne_master
    · exact T.bot.master_mem
  · intro h
    rw [NeighborhoodSystem.mem_bot] at h
    exact Or.inr (Or.inr h)
end
end DomainTheory.NeighbourhoodSystems.SeqPeano
namespace DomainTheory.NeighbourhoodSystems.SeqPeano
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap BitDom
    SeqDom
section
variable {β : Type*}

theorem one_def_strBot (σ : Str) : oneMap.toElementMap (strBot σ) = oneDef (strBot σ)
    := by
  cases σ with
  | nil =>
```

```
    rw [oneDef, emptyMap_strBot, headValC_nil, condT_bot, oneMap_strBot, headValC_nil]
  | cons b σ0 =>
    cases b with
    | false =>
      rw [oneDef, emptyMap_strBot, headValC_false, Cond.cond_false,
        zeroMap_strBot, headValC_false, Cond.cond_true, oneMap_strBot, headValC_false]
    | true =>
      rw [oneDef, emptyMap_strBot, headValC_true, Cond.cond_false,
        zeroMap_strBot, headValC_true, Cond.cond_false, oneMap_strBot, headValC_true]

theorem one_def_strElem (σ : Str) : oneMap.toElementMap (strElem σ) = oneDef (strElem
    σ) := by
  cases σ with
  | nil =>

    rw [oneDef, emptyMap_strElem, headValC_nil, Cond.cond_true,
      oneMap_strElem, headValC_nil]
  | cons b σ0 =>
    cases b with
    | false =>

      rw [oneDef, emptyMap_strElem, headValC_false, Cond.cond_false,
        zeroMap_strElem, headValC_false, Cond.cond_true, oneMap_strElem,
    headValC_false]
    | true =>
      rw [oneDef, emptyMap_strElem, headValC_true, Cond.cond_false,
        zeroMap_strElem, headValC_true, Cond.cond_false, oneMap_strElem, headValC_true
    ]

end

theorem peano_cons_disjoint (σ τ : Str) : (false :: σ) = (true :: τ) := by
  simp

theorem peano_cons_injective {b : Bool} {σ τ : Str} (h : b :: σ = b :: τ) : σ = τ :=
  (List.cons.injEq b σ b τ).mp h |>.2

theorem peano_induction (P : Str → Prop) (hnil : P []) (hcons : ∀ b σ, P σ → P (b ::
    σ)) :
    ∀ σ, P σ := by
  intro σ
  induction σ with
  | nil => exact hnil
  | cons b σ ih => exact hcons b σ ih

theorem peano_nil_ne_cons (b : Bool) (σ : Str) : ([] : Str) = b :: σ := by
  simp

end DomainTheory.NeighbourhoodSystems.SeqPeano
```

4.5.38 Exercise 4.19 — Peano axioms for sequences

There are many unproved assertions in Example 4.4! In particular discuss "Peano's Axioms" for $\{0,1\}^*$. Show, moreover, that $\mathrm{one} : C \to T$ can be defined from the rest of the structure by a fixed-point equation.

The development is with the Peano structure on sequences peer.

Theorem (`example_419`). Finite bit-strings satisfy Peano's axioms; the explicit one-map agrees with the fixed-point definition of the numeral one on each stratum.

Proof. Combine the Peano lemmas and the stratum comparison theorems from the sequence Peano peer. ■

`Exercise_4_19.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.seqPeano

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap BitDom SeqDom DomainTheory.
    NeighbourhoodSystems.SeqPeano

theorem example_419 :
    (∀ (b : Bool) (σ : Str), ([] : Str) = b :: σ) ∧
    (∀ (σ : Str), oneMap.toElementMap (strBot σ) = oneDef (strBot σ)) ∧
    (∀ (σ : Str), oneMap.toElementMap (strElem σ) = oneDef (strElem σ)) ∧
    (∀ (σ τ : Str), (false :: σ) = (true :: τ)) ∧
    (∀ {b : Bool} {σ τ : Str}, b :: σ = b :: τ → σ = τ) ∧
    (∀ (P : Str → Prop), P [] → (∀ b σ, P σ → P (b :: σ)) → ∀ σ, P σ) :=
  ⟨peano_nil_ne_cons, one_def_strBot, one_def_strElem,
    peano_cons_disjoint, @peano_cons_injective, peano_induction⟩
```

4.5.39 Fix of a composite

For approximable endomaps f and g, the least fixed point of $f \circ g$ maps to that of $g \circ f$ under f, and conversely by symmetry.

Theorem (`comp_fixElement_isFixed`). The composite $f \circ g$ fixes the image under f of the least fixed point of $g \circ f$.

Proof. Apply the composition law for elementwise images and the fixed-point law for the least fixed point of $g \circ f$. ■

Theorem (`fixElement_comp_comm`). The least fixed points of $f \circ g$ and $g \circ f$ are related by f on the elementwise order.

Proof. Use leastness and the preceding fixed-point transfer, then symmetry with roles reversed. ■

`fixComp.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
```

```
section
variable {α : Type*} {V : NeighborhoodSystem α}

namespace ApproximableMap

theorem comp_fixElement_isFixed (f g : ApproximableMap V V) :
    (f.comp g).toElementMap (f.toElementMap (g.comp f).fixElement)
      = f.toElementMap (g.comp f).fixElement := by
  rw [toElementMap_comp]
  have h : g.toElementMap (f.toElementMap (g.comp f).fixElement) = (g.comp f).
    fixElement := by
    rw [← toElementMap_comp]
    exact toElementMap_fixElement (g.comp f)
  rw [h]

theorem fixElement_comp_comm (f g : ApproximableMap V V) :
    (f.comp g).fixElement = f.toElementMap (g.comp f).fixElement := by
  apply le_antisymm
  · exact fixElement_le_of_toElementMap_le (f.comp g)
      (le_of_eq (comp_fixElement_isFixed f g))
  ·
    have hga : (g.comp f).fixElement ≤ g.toElementMap (f.comp g).fixElement := by
      apply fixElement_le_of_toElementMap_le (g.comp f)
      have : (g.comp f).toElementMap (g.toElementMap (f.comp g).fixElement)
          = g.toElementMap (f.comp g).fixElement := comp_fixElement_isFixed g f
      exact le_of_eq this
    calc f.toElementMap (g.comp f).fixElement
        ≤ f.toElementMap (g.toElementMap (f.comp g).fixElement) := f.toElementMap_mono
     hga
      _ = (f.comp g).toElementMap (f.comp g).fixElement := by rw [toElementMap_comp]
      _ = (f.comp g).fixElement := toElementMap_fixElement (f.comp g)

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
```

4.5.40 Exercise 4.20 — fix of a composite

For approximable $f, g : D \to D$ prove that

$$\mathrm{fix}(f \circ g) = f(\mathrm{fix}(g \circ f)).$$

The development is with the Fix of a composite peer.

Theorem (`example_420`). For approximable f, g,
$mathrmfix(f$
$circg) = f($
$mathrmfix(g$
$circf))$.

Proof. Apply the composite fixed-point commutation law. ■

`Exercise_4_20.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixComp

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

open ApproximableMap

theorem example_420 {α : Type*} {V : NeighborhoodSystem α}
    (f g : ApproximableMap V V) : (f.comp g).fixElement = f.toElementMap (g.comp f).
    fixElement :=
  @fixElement_comp_comm α V f g
```

4.5.41 Order as a fixed point

Order, successor towers, and multiplication as powerset least fixed points.

Definition (`leOp`). The order operator unions the diagonal with one-step successor pairs from a relation.

Proof. As in the formal module. ■ **Definition (`leRel`).** The standard less-than-or-equal relation on $\mathbb{N} \times \mathbb{N}$.

Proof. As in the formal module. ■ **Definition (`mulOp`).** For fixed n, multiply a set by adjoining 0 and all $n + m$ with m in the set.

Proof. As in the formal module. ■ **Definition (`succImage`).** Successor image $x^+ = \{k \mid \exists j \in x,\ k = j + 1\}$.

Proof. As in the formal module. ■ **Definition (`upSet`).** The upward-closed set $\{k \mid m \leq k\}$.

Proof. As in the formal module. ■ **Definition (`addIso`).** Isomorphism between $\mathbb{N}$ and integers at least m.

Proof. As in the formal module. ■ **Theorem (`addIso_apply`).** The addition isomorphism sends n to $m + n$.

Proof. As in the formal module. ■ **Theorem (`addIso_succ`).** The addition isomorphism commutes with successor.

Proof. As in the formal module. ■ **Theorem (`addIso_zero`).** The addition isomorphism sends 0 to m.

Proof. As in the formal module. ■ **Theorem (`leOp_monotone`).** The order operator is monotone on subsets of pairs.

Proof. As in the formal module. ■ **Theorem (`leRel_isFixed`).** The standard order is a fixed point of the order operator.

Proof. As in the formal module. ■ **Theorem (`mem_fixedPoint_iff`).** Membership in a fixed order relation is equivalent to $n \leq m$.

Proof. As in the formal module. ■ **Theorem (`leOp_unique`).** Every fixed point of the order operator equals the standard order.

Proof. As in the formal module. ■ **Theorem (`mulOp_monotone`).** The multiplication operator is monotone.

Proof. As in the formal module. ■ **Theorem (`mulOp_lfp_eq_multiples`).** The least fixed set for n is the set of multiples of n.

Proof. As in the formal module. ■ **Theorem (`upSet_succ`).** Successor shift of an upward set is the successor image.

Proof. As in the formal module. ■ **Theorem (`upSet_zero`).** The upward set from 0 is the universe.

Proof. As in the formal module. ■ **Theorem (`upSet_unique`).** The upward-set tower is uniquely determined by zero and successor image.

Proof. As in the formal module. ■

`orderFixed.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.gfpSet
public import Mathlib.Algebra.Group.End
public import Mathlib.Tactic.Ring

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems

def leOp (u : Set (ℕ × ℕ)) : Set (ℕ × ℕ) :=
  {p | p.1 = p.2} ∪ {p | ∃ m, (p.1, m) ∈ u ∧ p.2 = m + 1}

def leRel : Set (ℕ × ℕ) := {p | p.1 ≤ p.2}

def mulOp (n : ℕ) (u : Set ℕ) : Set ℕ := {0} ∪ {k | ∃ m ∈ u, k = n + m}

def succImage (x : Set ℕ) : Set ℕ := {k | ∃ j ∈ x, k = j + 1}

def upSet (m : ℕ) : Set ℕ := {k | m ≤ k}

end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems

def addIso (m : ℕ) : ℕ ≃ {k : ℕ // k ∈ upSet m} where
  toFun n := ⟨m + n, Nat.le_add_right m n⟩
  invFun k := k.1 - m
  left_inv n := by show m + n - m = n; omega
  right_inv := by
    rintro ⟨k, hk⟩
    exact Subtype.ext (Nat.add_sub_of_le hk)

theorem addIso_apply (m n : ℕ) : (addIso m n : ℕ) = m + n := rfl

theorem addIso_succ (m n : ℕ) : (addIso m (n + 1) : ℕ) = (addIso m n : ℕ) + 1 := by
  simp only [addIso_apply]; omega

theorem addIso_zero (m : ℕ) : (addIso m 0 : ℕ) = m := by simp [addIso]

theorem leOp_monotone : Monotone leOp := by
  intro u v huv p hp
  rcases hp with hp | ⟨m, hm, hpm⟩
  · exact Or.inl hp
```

```
  · exact Or.inr ⟨m, huv hm, hpm⟩

theorem leRel_isFixed : leOp leRel = leRel := by
  apply Set.Subset.antisymm
  · rintro ⟨n, m⟩ (h | ⟨k, hk, rfl⟩)
    · exact le_of_eq (h : n = m)
    · exact Nat.le_succ_of_le (hk : n ≤ k)
  · rintro ⟨n, m⟩ (h : n ≤ m)
    rcases Nat.eq_or_lt_of_le h with h' | h'
    · exact Or.inl h'
    · refine Or.inr ⟨m - 1, ?_, ?_⟩
      · change n ≤ m - 1; omega
      · omega

theorem mem_fixedPoint_iff {u : Set (ℕ × ℕ)} (hu : leOp u = u) (n m : ℕ) :
    (n, m) ∈ u ↔ n ≤ m := by
  induction m with
  | zero =>
    rw [← hu]
    constructor
    · rintro (h | ⟨k, _, hk⟩)
      · exact le_of_eq h
      · exact absurd hk (Nat.succ_ne_zero k).symm
    · intro h
      exact Or.inl (Nat.le_zero.mp h)
  | succ k ih =>
    rw [← hu]
    constructor
    · rintro (h | ⟨j, hj, hjk⟩)
      · exact le_of_eq h
      · have hjk' : j = k := by omega
        subst hjk'
        exact Nat.le_succ_of_le (ih.mp hj)
    · intro h
      rcases Nat.eq_or_lt_of_le h with h' | h'
      · exact Or.inl h'
      · exact Or.inr ⟨k, ih.mpr (Nat.lt_succ_iff.mp h'), rfl⟩

theorem leOp_unique {u : Set (ℕ × ℕ)} (hu : leOp u = u) : u = leRel := by
  ext ⟨n, m⟩
  rw [mem_fixedPoint_iff hu]
  rfl

theorem mulOp_monotone (n : ℕ) : Monotone (mulOp n) := by
  rintro u v huv k (hk | ⟨m, hm, rfl⟩)
  · exact Or.inl hk
  · exact Or.inr ⟨m, huv hm, rfl⟩

theorem mulOp_lfp_eq_multiples (n : ℕ) :
    lfpSet (mulOp n) = {k | ∃ i, k = n * i} := by
  apply Set.Subset.antisymm
  ·
    have hpre : mulOp n {k | ∃ i, k = n * i} ⊆ {k | ∃ i, k = n * i} := by
      rintro k (rfl | ⟨m, ⟨i, rfl⟩, rfl⟩)
      · exact ⟨0, by simp⟩
      · exact ⟨i + 1, by ring⟩
```

```
    exact lfpSet_subset (mulOp n) hpre
  ·
    have hfix : mulOp n (lfpSet (mulOp n)) = lfpSet (mulOp n) :=
      lfpSet_isFixed (mulOp n) (mulOp_monotone n)
    rintro k ⟨i, rfl⟩
    induction i with
    | zero =>
      rw [← hfix]; exact Or.inl (by simp)
    | succ j ih =>
      rw [← hfix]
      exact Or.inr ⟨n * j, ih, by ring⟩

theorem upSet_succ (m : ℕ) : upSet (m + 1) = succImage (upSet m) := by
  ext k
  simp only [upSet, succImage, Set.mem_ofPred_eq]
  constructor
  · intro h
    exact ⟨k - 1, by omega, by omega⟩
  · rintro ⟨j, hj, rfl⟩
    omega

theorem upSet_zero : upSet 0 = Set.univ := by
  ext k; simp [upSet]

theorem upSet_unique (s : ℕ → Set ℕ) (h0 : s 0 = Set.univ)
    (hsucc : ∀ m, s (m + 1) = succImage (s m)) : s = upSet := by
  funext m
  induction m with
  | zero => rw [h0, upSet_zero]
  | succ k ih => rw [hsucc k, ih, upSet_succ]

end DomainTheory.NeighbourhoodSystems
```

4.5.42 Exercise 4.21 — order as a fixed point

Show that the less-than-or-equal-to relation $\ell \subseteq \mathbf{N} \times \mathbf{N}$ is uniquely determined by the fixed point equation

$$\ell = \{(n, n) \mid n \in \mathbf{N}\} \cup \{(n, m^+) \mid (n, m) \in \ell\}.$$

Consider the structured set $\langle P\mathbf{N}, \mathbf{N}, {}^+\rangle$ where, as before, $x^+ = \{n^+ \mid n \in x\}$.

What is the unique function $[\cdot] : \mathbf{N} \to P\mathbf{N}$ given by 4.13(3)? Prove that the structures $\langle \mathbf{N}, 0, {}^+\rangle$ and $\langle [m], m, {}^+\rangle$ are uniquely isomorphic for each $m \in \mathbf{N}$, and connect the isomorphism with ordinary addition of integers. Can the same be done for multiplication? (Hint: Consider the fixed-point equation

$$n \cdot \mathbf{N} = \{0\} \cup \{n + m \mid m \in n \cdot \mathbf{N}\},$$

where $n \in \mathbf{N}$ is fixed.)

The development is with the Order as a fixed point peer.

Theorem (`example_421`). The standard order on
$mathbbN$
$times$
$mathbbN$ is the unique fixed point of the order operator; successor towers on

$mathbbN$ are uniquely determined; and for each n the least solution of the multiplication equation is the set of multiples of n.

Proof. Apply the uniqueness, fixed-point, and least-fixed-point lemmas of the order peer. ■

Exercise_4_21.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.orderFixed

@[expose] public section

open DomainTheory.NeighbourhoodSystems

theorem example_421 :
    (∀ (s : ℕ → Set ℕ), s 0 = Set.univ →
      (∀ m, s (m + 1) = succImage (s m)) → s = upSet) ∧
    (∀ {u : Set (ℕ × ℕ)}, leOp u = u → u = leRel) ∧
    (leOp leRel = leRel) ∧
    (∀ (n : ℕ), lfpSet (mulOp n) = {k | ∃ i, k = n * i}) :=
  ⟨upSet_unique, @leOp_unique, leRel_isFixed, mulOp_lfp_eq_multiples⟩
```

4.5.43 Inductive Peano submodel

Least-fixed-point generated subsets carrying full Peano axioms.

Definition (`genOp`). Generated set $\{0\} \cup s''x$ for a carrier with successor s.

Proof. As in the formal module. ■ **Definition (`natPeano`).** The standard Peano model on $\mathbb{N}$.

Proof. As in the formal module. ■ **Definition (`nats`).** Least fixed set of the generator operator.

Proof. As in the formal module. ■ **Definition (`Nat'`).** Subtype of elements in the least generated set.

Proof. As in the formal module. ■ **Theorem (`nats_induction`).** Induction principle for membership in the least generated set.

Proof. As in the formal module. ■ **Theorem (`succ_mem_nats`).** Successor of a generated element is generated.

Proof. As in the formal module. ■ **Definition (`subSucc`).** Successor on the generated subtype.

Proof. As in the formal module. ■ **Theorem (`zero_mem_nats`).** Zero lies in the least generated set.

Proof. As in the formal module. ■ **Definition (`subZero`).** Zero as an element of the generated subtype.

Proof. As in the formal module. ■ **Definition (`peanoSub`).** Peano structure on the generated subtype from partial data.

Proof. As in the formal module. ■ **Theorem (`exists_peano_submodel`).** A full Peano model exists on the generated subtype.

Proof. As in the formal module. ■ **Theorem (`genOp_monotone`).** The generator operator is monotone.

Proof. As in the formal module. ■ **Theorem (`genOp_nats`).** The least generated set is fixed by the generator operator.

Proof. As in the formal module. ■

peanoSub.lean

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Definition_4_5
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.gfpSet

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems

section
variable {M : Type*} (zero : M) (succ : M → M)

def genOp (x : Set M) : Set M := {zero} ∪ succ '' x

def natPeano : PeanoModel ℕ where
  zero := 0
  succ := Nat.succ
  zero_ne_succ n := (Nat.succ_ne_zero n).symm
  succ_injective := Nat.succ_injective
  induction s h0 hstep := by
    intro n
    induction n with
    | zero => exact h0
    | succ k ih => exact hstep k ih

def nats : Set M := lfpSet (genOp zero succ)

abbrev Nat' : Type _ := {m : M // m ∈ nats zero succ}

theorem nats_induction (s : Set M) (h0 : zero ∈ s) (hsucc : ∀ n ∈ s, succ n ∈ s) :
    nats zero succ ⊆ s := by
  apply lfpSet_subset (genOp zero succ)
  rintro x (rfl | ⟨n, hn, rfl⟩)
  · exact h0
  · exact hsucc n hn

theorem succ_mem_nats {n : M} (hn : n ∈ nats zero succ) : succ n ∈ nats zero succ :=
    by
  intro x hx
  exact hx (Or.inr ⟨n, hn x hx, rfl⟩)

def subSucc (n : Nat' zero succ) : Nat' zero succ := ⟨succ n.1, succ_mem_nats zero succ
    n.2⟩

theorem zero_mem_nats : zero ∈ nats zero succ := by
  intro x hx
  exact hx (Or.inl rfl)

def subZero : Nat' zero succ := ⟨zero, zero_mem_nats zero succ⟩
end

section
variable {M : Type*} (zero : M) (succ : M → M)

def peanoSub (hzero : ∀ n, zero = succ n) (hinj : Function.Injective succ) :
    PeanoModel (Nat' zero succ) where
  zero := subZero zero succ
```

```
  succ := subSucc zero succ
  zero_ne_succ n h := hzero n.1 (Subtype.ext_iff.mp h)
  succ_injective a b h := Subtype.ext (hinj (Subtype.ext_iff.mp h))
  induction s h0 hstep := by

    intro n
    have hsub : nats zero succ ⊆
        {m | ∃ h : m ∈ nats zero succ, (⟨m, h⟩ : Nat' zero succ) ∈ s} := by
      apply nats_induction zero succ
      · exact ⟨zero_mem_nats zero succ, h0⟩
      · rintro m ⟨hmnat, hms⟩
        exact ⟨succ_mem_nats zero succ hmnat, hstep ⟨m, hmnat⟩ hms⟩
    obtain ⟨_, hns⟩ := hsub n.2
    exact hns

theorem exists_peano_submodel (hzero : ∀ n, zero = succ n) (hinj : Function.Injective
    succ) :
    Nonempty (PeanoModel (Nat' zero succ)) :=
  ⟨peanoSub zero succ hzero hinj⟩

end

section
variable {M : Type*} (zero : M) (succ : M → M)

theorem genOp_monotone : Monotone (genOp zero succ) := by
  intro x y hxy z hz
  rcases hz with hz | ⟨a, ha, rfl⟩
  · exact Or.inl hz
  · exact Or.inr ⟨a, hxy ha, rfl⟩

theorem genOp_nats : genOp zero succ (nats zero succ) = nats zero succ :=
  lfpSet_isFixed (genOp zero succ) (genOp_monotone zero succ)

end

end DomainTheory.NeighbourhoodSystems
```

4.5.44 Exercise 4.22 — inductive subset of a model

Suppose $\mathbf{N}^*$ is a structured set satisfying only axioms (i) and (ii) of 4.5. Must there be a subset $\mathbf{N} \subseteq \mathbf{N}^*$ that satisfies (i), (ii), and (iii)? (Hint: Use a least fixed point in $P\mathbf{N}^*$.) (For set theorists): How do we know from the axioms of set theory that there exists such a set $\mathbf{N}^*$?

The development is with the Inductive Peano submodel peer.

Theorem (`example_422`). Inductive submodel of a partial Peano structure via a powerset least fixed point.

Proof. Invoke the peer library facts cited in the packaging theorem. ■

`Exercise_4_22.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.peanoSub
```

```
@[expose] public section

open DomainTheory.NeighbourhoodSystems

theorem example_422 :
    ∀ {M : Type*} (zero : M) (succ : M → M),
      (∀ n, zero = succ n) → Function.Injective succ →
        Nonempty (PeanoModel (Nat' zero succ)) :=
  @exists_peano_submodel
```

4.5.45 Unique fixed point

Under Eilenberg's hypotheses on approximating maps below the identity, an approximable endomap has exactly one fixed point, namely its least fixed point.

Theorem (`f_unique_fixedPoint`). If approximable maps a_n rise from bottom to the identity and commute with f on the stated scheme, there is a unique element x with $f(x) = x$.

Proof. Show every fixed x lies below and above the least fixed point of f by induction on stages using the LUB hypothesis and the commutation law. ■

`uniqueFixed.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot
public import Mathlib.Order.Bounds.Defs
public import Mathlib.Basic.ExistsUnique

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {α : Type*} {V : NeighborhoodSystem α}
namespace ApproximableMap

theorem f_unique_fixedPoint (f : ApproximableMap V V) (a : ℕ → ApproximableMap V V)
    (ha0 : ∀ x, (a 0).toElementMap x = V.bot)
    (hlub : ∀ x, IsLUB {y | ∃ n, y = (a n).toElementMap x} x)
    (hcomm : ∀ n x, (a (n + 1)).toElementMap (f.toElementMap x) =
      (a (n + 1)).toElementMap (f.toElementMap ((a n).toElementMap x))) :
    ∃! x, f.toElementMap x = x := by
  set fix := f.fixElement with hfixdef
  have hfixed : f.toElementMap fix = fix := toElementMap_fixElement f
  refine ⟨fix, hfixed, ?_⟩
  intro x hx

  have hle : fix ≤ x := fixElement_le_of_toElementMap_le f (le_of_eq hx)

  have approx_le : ∀ n, (a n).toElementMap x ≤ (a n).toElementMap fix := by
```

```
    intro n
    induction n with
    | zero => rw [ha0 x]; exact bot_le
    | succ k ih =>
      have h1 : (a (k + 1)).toElementMap x =
          (a (k + 1)).toElementMap (f.toElementMap ((a k).toElementMap x)) := by
        rw [← hcomm k x, hx]
      have h2 : (a (k + 1)).toElementMap (f.toElementMap ((a k).toElementMap fix)) =
          (a (k + 1)).toElementMap fix := by
        rw [← hcomm k fix, hfixed]
      rw [h1, ← h2]
      exact toElementMap_mono (a (k + 1)) (toElementMap_mono f ih)

  have hfix_ub : ∀ n, (a n).toElementMap fix ≤ fix := fun n => (hlub fix).1 ⟨n, rfl⟩

  have hge : x ≤ fix := by
    apply (hlub x).2
    rintro y ⟨n, rfl⟩
    exact le_trans (approx_le n) (hfix_ub n)
  exact le_antisymm hge hle

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
```

4.5.46 Exercise 4.23 — commuting with the iterates

(Suggested by S. Eilenberg). Suppose $f : \mathcal{D} \to \mathcal{D}$ is approximable on a given domain $\mathcal{D}$. Suppose $a_n : \mathcal{D} \to \mathcal{D}$ is a sequence of approximable maps where

(i) $a_0(x) = \bot$, for all $x \in |\mathcal{D}|$;

(ii) $a_n \sqsubseteq a_{n+1}$ in $\mathcal{D} \to \mathcal{D}$, for all $n \in \mathbf{N}$;

(iii) $\bigsqcup_{n=0}^{\infty} a_n = I_{\mathcal{D}}$ in $\mathcal{D} \to \mathcal{D}$;

(iv) $a_{n+1} \circ f = a_{n+1} \circ f \circ a_n$, for all $n \in \mathbf{N}$.

Prove that f has a unique fixed point. (Hint: Show that if $x = f(x)$, then $a_n(x) \sqsubseteq a_n(\mathrm{fix}(f))$ for all $n \in \mathbf{N}$ by induction on n.)

The development is with the Unique fixed point peer.

Theorem (`example_423`). Unique fixed point under Eilenberg iterate hypotheses.

Proof. Invoke the peer library facts cited in the packaging theorem. ■

`Exercise_4_23.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.uniqueFixed

@[expose] public section
```

```
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

open ApproximableMap

theorem example_423 :
    ∀ {α : Type*} {V : NeighborhoodSystem α}
      (f : ApproximableMap V V) (a : ℕ → ApproximableMap V V),
      (∀ x, (a 0).toElementMap x = V.bot) →
      (∀ x, IsLUB {y | ∃ n, y = (a n).toElementMap x} x) →
      (∀ n x, (a (n + 1)).toElementMap (f.toElementMap x) =
        (a (n + 1)).toElementMap (f.toElementMap ((a n).toElementMap x))) →
      ∃! x, f.toElementMap x = x :=
  @f_unique_fixedPoint
```

4.5.47 Schroeder-Bernstein

Schroeder–Bernstein via Tarski's least fixed set.

Definition (`sbOp`). Tarski operator: outside the range of g union $g(f(X))$.

Proof. As in the formal module. ■ **Theorem (`sbOp_monotone`).** The Tarski operator is monotone.

Proof. As in the formal module. ■ **Definition (`sbSet`).** Least fixed set of the Tarski operator.

Proof. As in the formal module. ■ **Theorem (`sbSet_isFixed`).** The Schroeder set is a fixed point of the operator.

Proof. As in the formal module. ■ **Theorem (`g_f_mem_of_mem`).** If a is in the fixed set then $g(f(a))$ is too.

Proof. As in the formal module. ■ **Theorem (`mem_range_of_not_mem`).** Elements outside the fixed set lie in the range of g.

Proof. As in the formal module. ■ **Definition (`sbFun`).** Bijection candidate: f on the fixed set, chosen preimage elsewhere.

Proof. As in the formal module. ■ **Theorem (`g_sbFun_not_mem`).** Off the fixed set, g inverts the chosen map.

Proof. As in the formal module. ■ **Theorem (`sbFun_mem`).** On the fixed set the candidate map is f.

Proof. As in the formal module. ■ **Theorem (`sbFun_injective`).** The candidate map is injective if f is.

Proof. As in the formal module. ■ **Theorem (`sbFun_surjective`).** The candidate map is surjective if g is injective.

Proof. As in the formal module. ■ **Definition (`schroeder_bernstein_equiv`).** Equivalence from the candidate bijection.

Proof. As in the formal module. ■ **Theorem (`schroeder_bernstein`).** Injective embeddings yield a bijection $A \to B$.

Proof. As in the formal module. ■

`schroederBernstein.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.gfpSet
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
```

```
public import Mathlib.Algebra.Group.End

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems Function

section
variable {A B : Type*} {f : A → B} {g : B → A}

def sbOp (f : A → B) (g : B → A) (X : Set A) : Set A := (Set.range g)? ∪ g '' (f '' X
    )

theorem sbOp_monotone (f : A → B) (g : B → A) : Monotone (sbOp f g) := by
  intro X Y hXY a ha
  rcases ha with ha | ⟨b, ⟨x, hx, rfl⟩, rfl⟩
  · exact Or.inl ha
  · exact Or.inr ⟨f x, ⟨x, hXY hx, rfl⟩, rfl⟩

def sbSet (f : A → B) (g : B → A) : Set A := lfpSet (sbOp f g)

theorem sbSet_isFixed (f : A → B) (g : B → A) : sbOp f g (sbSet f g) = sbSet f g :=
  lfpSet_isFixed (sbOp f g) (sbOp_monotone f g)

theorem g_f_mem_of_mem {a : A} (ha : a ∈ sbSet f g) : g (f a) ∈ sbSet f g := by
  rw [← sbSet_isFixed]
  exact Or.inr ⟨f a, ⟨a, ha, rfl⟩, rfl⟩

theorem mem_range_of_not_mem {a : A} (ha : a ∈ sbSet f g) : a ∈ Set.range g := by
  have hne : a ∈ sbOp f g (sbSet f g) := by rw [sbSet_isFixed]; exact ha
  have : ¬ a ∈ Set.range g := fun h => hne (Or.inl h)
  exact not_not.mp this

open Classical in
end
end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems Function
section
variable {A B : Type*} {f : A → B} {g : B → A}

open Classical in

noncomputable def sbFun (f : A → B) (g : B → A) (a : A) : B :=
  if ha : a ∈ sbSet f g then f a else Classical.choose (mem_range_of_not_mem ha)

theorem g_sbFun_not_mem {a : A} (ha : a ∈ sbSet f g) : g (sbFun f g a) = a := by
  rw [sbFun, dite_eq_right ha]
  exact Classical.choose_spec (mem_range_of_not_mem ha)

theorem sbFun_mem {a : A} (ha : a ∈ sbSet f g) : sbFun f g a = f a := dite_eq_left ha

theorem sbFun_injective (hf : Injective f) : Injective (sbFun f g) := by
  intro a₁ a₂ heq
  by_cases h1 : a₁ ∈ sbSet f g <;> by_cases h2 : a₂ ∈ sbSet f g
```

```
  · rw [sbFun_mem h1, sbFun_mem h2] at heq
    exact hf heq
  ·
    exfalso
    rw [sbFun_mem h1] at heq
    have : a₂ = g (f a₁) := by rw [heq, g_sbFun_not_mem h2]
    exact h2 (this ▸ g_f_mem_of_mem h1)
  · exfalso
    rw [sbFun_mem h2] at heq
    have : a₁ = g (f a₂) := by rw [← heq, g_sbFun_not_mem h1]
    exact h1 (this ▸ g_f_mem_of_mem h2)
  ·
    have : g (sbFun f g a₁) = g (sbFun f g a₂) := by rw [heq]
    rw [g_sbFun_not_mem h1, g_sbFun_not_mem h2] at this
    exact this

theorem sbFun_surjective (hg : Injective g) : Surjective (sbFun f g) := by
  intro b
  by_cases hb : g b ∈ sbSet f g
  ·
    rw [← sbSet_isFixed] at hb
    rcases hb with hb | ⟨y, ⟨a, ha, rfl⟩, hgy⟩
    · exact absurd ⟨b, rfl⟩ hb
    · refine ⟨a, ?_⟩
      rw [sbFun_mem ha, hg hgy]
  ·
    refine ⟨g b, ?_⟩
    exact hg (g_sbFun_not_mem hb)

noncomputable def schroeder_bernstein_equiv (hf : Injective f) (hg : Injective g) : A
    ≃ B :=
  Equiv.ofBijective (sbFun f g) ⟨sbFun_injective hf, sbFun_surjective hg⟩

open Classical in

theorem schroeder_bernstein (hf : Injective f) (hg : Injective g) :
    ∃ h : A → B, Bijective h :=
  ⟨sbFun f g, sbFun_injective hf, sbFun_surjective hg⟩

end

end DomainTheory.NeighbourhoodSystems
```

4.5.48 Exercise 4.24 — Schroeder-Bernstein theorem

(For set theorists). Let $f : A \to B$ and $g : B \to A$ be one-one functions (into, not necessarily onto!) Prove the Schroeder - Bernstein theorem to the effect that there exists a one-one correspondence $h : A \leftrightarrow B$. (Hint: (Suggested by A. Tarski). By the fixed-point theorem find $X \subseteq A$ where

$$X = (A - g(B)) \cup g(f(X)),$$

where $f(X) =$ the image of the set X under the function f. Define $h \subseteq A \times B$ as a union of two restrictions:

$$h = f \restriction X \cup g^{-1} \restriction (A - X).$$

A picture helps.)

The development is with the Schroeder-Bernstein peer.

Theorem (`example_424`). Schroeder–Bernstein from a Tarski-style least fixed set.

Proof. Invoke the peer library facts cited in the packaging theorem. ■

Exercise_4_24.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.
    schroederBernstein

@[expose] public section

open DomainTheory.NeighbourhoodSystems

theorem example_424 :
    ∀ {A B : Type*} {f : A → B} {g : B → A},
      Function.Injective f → Function.Injective g → ∃ h : A → B, Function.Bijective
    h :=
  @schroeder_bernstein
```

4.5.49 Unary sequence domain

Unary neighbourhood system, cons map, and comparison with the number domain.

Definition (`shift`). Successor neighbourhood shift on subsets of $\mathbb{N}$.

Proof. As in the formal module. ■ **Definition (`tail`).** Tail neighbourhood $\{m \mid n \leq m\}$.

Proof. As in the formal module. ■ **Theorem (`mem_tail`).** Characterisation of tail membership.

Proof. As in the formal module. ■ **Definition (`memC1`).** Neighbourhoods are tails or singletons.

Proof. As in the formal module. ■ **Theorem (`memC1_singleton`).** Singletons are valid neighbourhoods.

Proof. As in the formal module. ■ **Theorem (`memC1_tail`).** Tails are valid neighbourhoods.

Proof. As in the formal module. ■ **Theorem (`shift_mono`).** Shift is monotone in neighbourhood inclusion.

Proof. As in the formal module. ■ **Theorem (`shift_singleton`).** Shift sends $\{n\}$ to $\{n+1\}$.

Proof. As in the formal module. ■ **Theorem (`shift_tail`).** Shift sends tail n to tail $n+1$.

Proof. As in the formal module. ■ **Theorem (`memC1_shift`).** Shift preserves the unary neighbourhood class.

Proof. As in the formal module. ■ **Theorem (`singleton_tail_nd`).** Singleton and tail are nested, reversed, or disjoint.

Proof. As in the formal module. ■ **Theorem (`nestedOrDisjoint`).** Unary neighbourhoods form a nested-or-disjoint family.

Proof. As in the formal module. ■ **Theorem (`tail_zero`).** Tail from zero is the universe.

Proof. As in the formal module. ■ **Theorem (`memC1_univ`).** The universe is a unary neighbourhood.

Proof. As in the formal module. ■ **Definition (`C1`).** Unary sequence neighbourhood system on $\mathbb{N}$.

Proof. As in the formal module. ■ **Theorem (`C1_mem`).** Membership characterisation for unary neighbourhoods.

Proof. As in the formal module. ■ **Theorem (`C1_master`).** Master neighbourhood is the universe.

Proof. As in the formal module. ■ **Definition (`consMap`).** Approximable shift-by-successor on unary neighbourhoods.

Proof. As in the formal module. ■ **Definition (`infElt`).** Least fixed point of the cons map.

Proof. As in the formal module. ■ **Definition (`oneBot`).** Principal element for a tail neighbourhood.

Proof. As in the formal module. ■ **Definition (`oneElem`).** Principal element for a singleton neighbourhood.

Proof. As in the formal module. ■ **Definition (`relateNToC1`).** Constant numeral lift from the number domain.

Proof. As in the formal module. ■ **Theorem (`consMap_oneBot`).** Cons map increments the tail index.

Proof. As in the formal module. ■ **Theorem (`consMap_oneElem`).** Cons map increments the singleton index.

Proof. As in the formal module. ■ **Theorem (`infElt_eq`).** The infinite element is fixed by the cons map.

Proof. As in the formal module. ■ **Theorem (`relateNToC1_bot`).** The lift sends bottom to bottom.

Proof. As in the formal module. ■ **Theorem (`relateNToC1_natElem`).** The lift sends numerals to singleton principals.

Proof. As in the formal module. ■

`unarySeq.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.numberDomain
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.UnarySeq
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

def shift (X : Set ℕ) : Set ℕ := {k | ∃ m ∈ X, k = m + 1}

def tail (n : ℕ) : Set ℕ := {m | n ≤ m}

@[simp] theorem mem_tail {n k : ℕ} : k ∈ tail n ↔ n ≤ k := Iff.rfl

def memC1 (X : Set ℕ) : Prop := (∃ n, X = tail n) ∨ (∃ n, X = {n})

theorem memC1_singleton (n : ℕ) : memC1 ({n} : Set ℕ) := Or.inr ⟨n, rfl⟩

theorem memC1_tail (n : ℕ) : memC1 (tail n) := Or.inl ⟨n, rfl⟩

theorem shift_mono {X X' : Set ℕ} (h : X' ⊆ X) : shift X' ⊆ shift X := by
  rintro k ⟨m, hm, rfl⟩; exact ⟨m, h hm, rfl⟩

theorem shift_singleton (n : ℕ) : shift ({n} : Set ℕ) = {n + 1} := by
```

```
  ext k
  simp only [shift, Set.mem_singleton_iff, Set.mem_ofPred_eq]
  constructor
  · rintro ⟨m, rfl, rfl⟩; rfl
  · rintro rfl; exact ⟨n, rfl, rfl⟩

theorem shift_tail (n : ℕ) : shift (tail n) = tail (n + 1) := by
  ext k
  simp only [shift, tail, Set.mem_ofPred_eq]
  constructor
  · rintro ⟨m, hm, rfl⟩; omega
  · intro h; exact ⟨k - 1, by omega, by omega⟩

theorem memC1_shift {X : Set ℕ} (hX : memC1 X) : memC1 (shift X) := by
  rcases hX with ⟨n, rfl⟩ | ⟨n, rfl⟩
  · exact Or.inl ⟨n + 1, shift_tail n⟩
  · exact Or.inr ⟨n + 1, shift_singleton n⟩

theorem singleton_tail_nd (n k : ℕ) :
    ({k} : Set ℕ) ⊆ tail n ∨ tail n ⊆ {k} ∨ ({k} : Set ℕ) ∩ tail n = ∅ := by
  by_cases h : n ≤ k
  · exact Or.inl (by intro x hx; rw [Set.mem_singleton_iff] at hx; subst hx; exact h)
  · refine Or.inr (Or.inr ?_)
    ext w
    simp only [Set.mem_inter_iff, Set.mem_singleton_iff, mem_tail, Set.
    mem_empty_iff_false,
      iff_false, not_and]
    rintro rfl hw; exact h hw

theorem nestedOrDisjoint : NestedOrDisjoint memC1 := by
  rintro X Y (⟨n, rfl⟩ | ⟨n, rfl⟩) (⟨m, rfl⟩ | ⟨m, rfl⟩)
  · rcases le_total n m with h | h
    · exact Or.inr (Or.inl (fun k hk => le_trans h hk))
    · exact Or.inl (fun k hk => le_trans h hk)
  · rcases singleton_tail_nd n m with h | h | h
    · exact Or.inr (Or.inl h)
    · exact Or.inl h
    · exact Or.inr (Or.inr (by rw [Set.inter_comm]; exact h))
  · rcases singleton_tail_nd m n with h | h | h
    · exact Or.inl h
    · exact Or.inr (Or.inl h)
    · exact Or.inr (Or.inr h)
  · by_cases h : n = m
    · subst h; exact Or.inl (Set.Subset.refl _)
    · refine Or.inr (Or.inr ?_)
      ext w
      simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false,
        not_and]
      rintro rfl h2; exact h h2

theorem tail_zero : tail 0 = Set.univ := by ext k; simp [tail]

theorem memC1_univ : memC1 (Set.univ : Set ℕ) := Or.inl ⟨0, tail_zero.symm⟩

end DomainTheory.NeighbourhoodSystems.UnarySeq
```

```
namespace DomainTheory.NeighbourhoodSystems.UnarySeq
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

def C1 : NeighborhoodSystem ℕ :=
  NeighborhoodSystem.ofNestedOrDisjoint memC1 Set.univ Set.univ_nonempty
    memC1_univ nestedOrDisjoint
    (fun _ => Set.subset_univ _)

@[simp] theorem C1_mem {X : Set ℕ} : C1.mem X ↔ memC1 X := Iff.rfl

@[simp] theorem C1_master : C1.master = (Set.univ : Set ℕ) := rfl

def consMap : ApproximableMap C1 C1 where
  rel X Y := memC1 X ∧ memC1 Y ∧ shift X ⊆ Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨memC1_univ, memC1_univ, Set.subset_univ _⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY, hsub⟩ ⟨_, hY', hsub'⟩
    have hsubInter : shift X ⊆ Y ∩ Y' := Set.subset_inter hsub hsub'
    exact ⟨hX, C1.inter_mem hY hY' (memC1_shift hX) hsubInter, hsubInter⟩
  mono := by
    rintro X X' Y Y' ⟨hX, hY, hsub⟩ hX'X hYY' hX' hY'
    exact ⟨hX', hY', (shift_mono hX'X).trans (hsub.trans hYY')⟩

def infElt : C1.Element := consMap.fixElement

def oneBot (n : ℕ) : C1.Element := C1.principal (memC1_tail n)

def oneElem (n : ℕ) : C1.Element := C1.principal (memC1_singleton n)

def relateNToC1 : ApproximableMap NatDom.N C1 := NatDom.constLiftN C1 oneElem

theorem consMap_oneBot (n : ℕ) : consMap.toElementMap (oneBot n) = oneBot (n + 1) :=
    by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨X', ⟨_, hXX'⟩, _, hY, hsub⟩
    refine ⟨hY, ?_⟩
    have hpre : shift (tail n) ⊆ Y := (shift_mono hXX').trans hsub
    rwa [shift_tail] at hpre
  · rintro ⟨hY, hsub⟩
    refine ⟨tail n, ⟨memC1_tail n, subset_rfl⟩, memC1_tail n, hY, ?_⟩
    rw [shift_tail]; exact hsub

theorem consMap_oneElem (n : ℕ) : consMap.toElementMap (oneElem n) = oneElem (n + 1)
    := by
  apply Element.ext
  intro Y
  constructor
  · rintro ⟨X', ⟨_, hXX'⟩, _, hY, hsub⟩
    refine ⟨hY, ?_⟩
    have hpre : shift {n} ⊆ Y := (shift_mono hXX').trans hsub
    rwa [shift_singleton] at hpre
  · rintro ⟨hY, hsub⟩
```

```
    refine ⟨{n}, ⟨memC1_singleton n, subset_rfl⟩, memC1_singleton n, hY, ?_⟩
    rw [shift_singleton]; exact hsub

theorem infElt_eq : consMap.toElementMap infElt = infElt :=
  toElementMap_fixElement consMap

theorem relateNToC1_bot : relateNToC1.toElementMap NatDom.N.bot = C1.bot :=
  NatDom.constLiftN_bot C1 oneElem

theorem relateNToC1_natElem (n : ℕ) :
    relateNToC1.toElementMap (NatDom.natElem n) = oneElem n :=
  NatDom.constLiftN_natElem C1 oneElem n

end DomainTheory.NeighbourhoodSystems.UnarySeq
```

4.5.50 Exercise 4.25 — unary sequence domain

Perhaps the domains $\mathbf{N}$ and C are not exactly analogous? C was based on $\{0,1\}$ as the underlying set of tokens. Construct a system C_1 based on $\{1\}^*$ (= finite strings of 1's) with neighbourhoods:

$$C_1 = \{\, \{1^m \mid m > n\} \mid n \in \mathbf{N} \,\} \cup \{\, \{1^n\} \mid n \in \mathbf{N} \,\}.$$

What structure should be put on C_1 strictly analogous to that on $C (= C_2)$? What kinds of approximable maps relate $\mathbf{N}$, C_1, and C_2? Draw some pictures.

The development is with the Unary sequence domain peer.

Theorem (`example_425`). Unary neighbourhood system, cons map, and comparison with the number domain.

Proof. Invoke the peer library facts cited in the packaging theorem. ■

`Exercise_4_25.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.unarySeq

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap NatDom DomainTheory.NeighbourhoodSystems.
    UnarySeq

theorem example_425 :
    relateNToC1.toElementMap NatDom.N.bot = C1.bot ∧
    (∀ n : ℕ, relateNToC1.toElementMap (NatDom.natElem n) = oneElem n) ∧
    consMap.toElementMap infElt = infElt ∧
    (∀ n : ℕ, consMap.toElementMap (oneBot n) = oneBot (n + 1)) ∧
    (∀ n : ℕ, consMap.toElementMap (oneElem n) = oneElem (n + 1)) :=
  ⟨relateNToC1_bot, relateNToC1_natElem, infElt_eq, consMap_oneBot, consMap_oneElem⟩
```

4.6 Typed λ-calculus

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

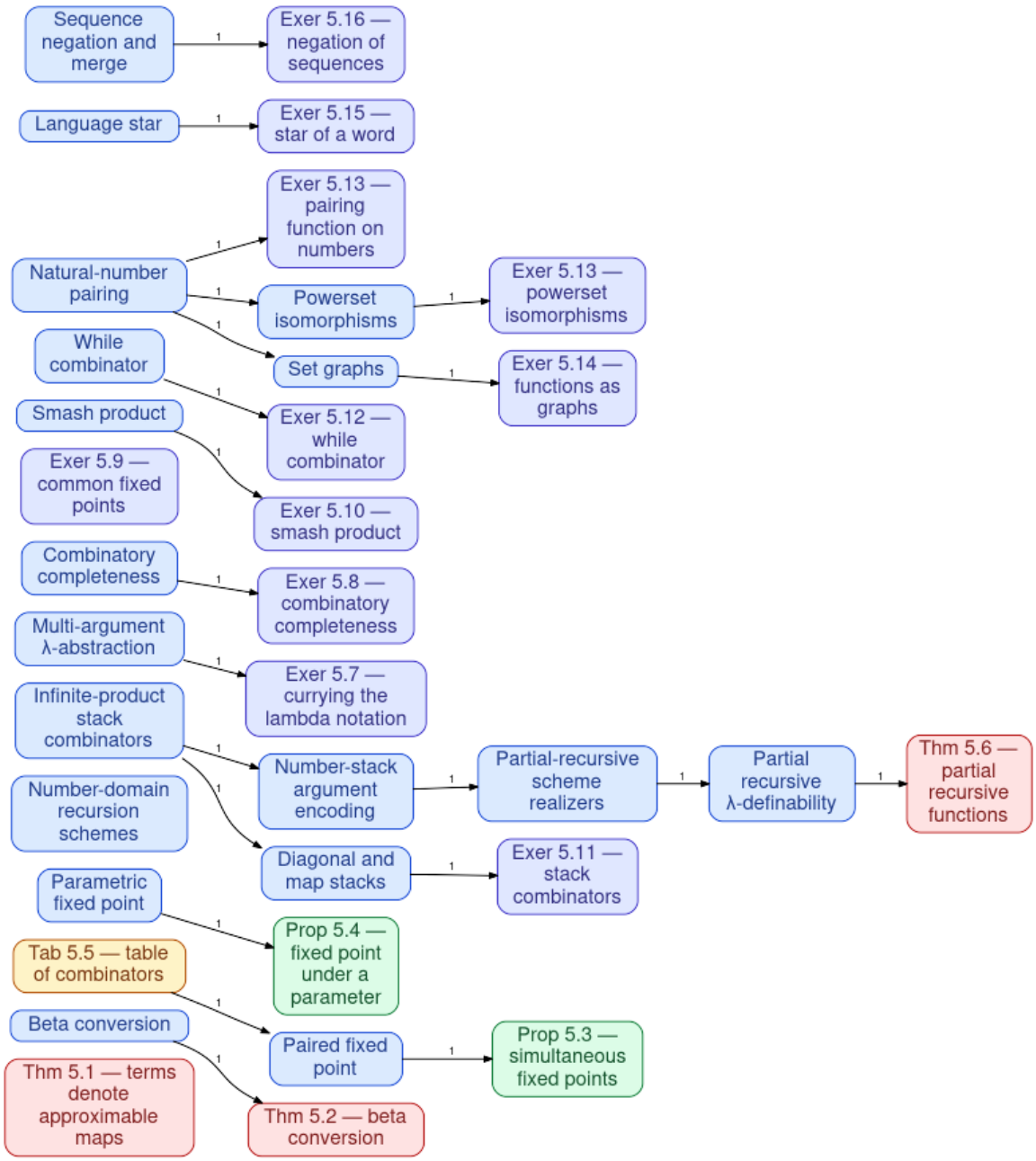


Figure 50: Diagram 15

4.6.1 Introduction

In Examples 4.3 and 4.4, after suitable domains have been constructed, functions are characterized by recursion equations whose form of expression is — basically — a composition or substitution of known functions

together with the function to be defined. This method can be made more precise and more easily usable by expanding our notation for functions — particularly by inventing a "temporary" notation for a function as a thing in itself without having to have special letters for functions. The device is called λ-*abstraction*. It is related to ordinary set abstraction (the $\{x \mid \ldots\}$ — notation already much used in these lectures), but we gear the approach to domains and their elements, and especially to function spaces.

At this stage it would not be so helpful to produce a rigorously formal definition of the syntax of the typed λ-calculus; we shall try to suggest what is needed by example. There are so many examples at hand, the less formal discussion ought to be sufficient.

In the first place we should set aside, in the notational store room as it were, a stock of variables

$$x, y, z, w, \ldots$$

These variables will be required in different "sizes" or "types". Roughly speaking there should be an infinite number of variables to range over the elements of each domain $\mathcal{D}$. We could perhaps write

$$x_0^{\mathcal{D}}, x_1^{\mathcal{D}}, x_2^{\mathcal{D}}, \ldots,$$

but the subscripts to insure an infinity of variables and the superscripts to record the typing of the variables lead to a notation as

tiresome to write as it is to read. We simply agree that we can have as many variables as we need and that they come in all the types.

Strictly speaking we should also introduce *type symbols* and not confuse types with domains. But if the reader will simply keep in mind that *form* in language has always to be kept distinct from *content*, the confusion at the type level will not matter so very much. A point at which the confusion might cause a real confusion concerns *compound types*. Given $\mathcal{D}_0$ and $\mathcal{D}_1$ we can form such compounds as

$$\mathcal{D}_0 + \mathcal{D}_1, \quad \mathcal{D}_0 \times \mathcal{D}_1, \quad \mathcal{D}_0 \to \mathcal{D}_1$$

What has to be remembered is that a compound domain (neighbourhood system), $\mathcal{D}_0 \times \mathcal{D}_1$ say, does not uniquely determine the "parts" $\mathcal{D}_0$ and $\mathcal{D}_1$. (We could make it do so, but it would cost some effort.) Of course, the symbol "$\mathcal{D}_0 \times \mathcal{D}_1$" has well defined parts. The point is that *different* ways of forming a compound domain could lead to the *same* result, meaning that a domain does not let us retrace its exact history of construction. Compound symbols, however, always carry their histories around with them, since otherwise they would not be readable. What we want, of course, are *both* domain symbols *and* domains, the latter being the *meanings* of the former. Most of the time we can happily pretend that it is only the domains themselves we have to think about.

Besides variables, we will also need certain *constants*. For instance, the symbol 0 (perhaps, better $0^{\mathbf{N}}$) denotes a certain element of $|\mathbf{N}|$. Similarly, in view of Theorem 4.2, for each domain $\mathcal{D}$ there is a well-determined element $\mathrm{fix}^{\mathcal{D}}$ of the compound type $((\mathcal{D} \to \mathcal{D}) \to \mathcal{D})$ denoting the least fixed-point operator. We have considered any number of similar constants of a great variety of types already (cf. 4.3 and 4.4; cond is an especially good one). We can say that the variables and constants are *atomic terms*, where "atomic" here means non-compound.

To form compound terms, there are several means: for example, if $\tau, \ldots, \sigma$ is a list of already obtained terms (including variables or constants), then we can form an ordered *tuple*

We have already done so in 3.1. If the types of $\tau, \ldots, \sigma$ are $D, \ldots, D'$, respectively, then the type of the tuple is the product domain

$$D \times \cdots \times D',$$

because we intend that the tuple denote an element of this domain. (The tuple notation for *functions* as in 3.3 is being forgotten for the time being.)

Next suppose that τ has type $(D_0 \to D_1)$ and σ has type D_0, then the usual *function-value* notation

$$\tau(\sigma)$$

is a compound term of type D_1. We also use

$$\tau(\sigma_0, \ldots, \sigma_{n-1})$$

as an abbreviation of

$$\tau(\langle \sigma_0, \ldots, \sigma_{n-1} \rangle),$$

where, if the types of $\sigma_0, \ldots, \sigma_{n-1}$ are $D_0, \ldots, D_{n-1}$, then the type of τ has to be of the form

$$((D_0 \times \cdots \times D_{n-1}) \to D_n)$$

where D_n is the type of the compound. In this manner, with functions applied to tuples, we have the full facility of substitution into functions of many variables just by iterating the notation.

Having taken into account *function value*, it remains to provide for *function definition*. Suppose that $x_0, \ldots, x_{n-1}$ is a list of distinct variables of types $D_0, \ldots, D_{n-1}$. Suppose further that τ is a term — no matter how complicated — of type D_n. Then we can regard τ as defining a function of n-variables of type

$$((D_0 \times \cdots \times D_{n-1}) \to D_n).$$

What we have not done is to reward our regard by, as yet, providing a quick-to-write "name" for that function. This we now do; it is called

$$\lambda x_0, \ldots, x_{n-1} \cdot \tau,$$

where we stress that the x_i must be *distinct* variables and that this
expression denotes the *whole function*. That is why we provide it with a special symbol.
Here is an example of the λ - notation

$$\lambda x, y.x,$$

which is read "lambda ex wye ... (pause) ... ex". If the types of x and y are D_0 and D_1, then the type of the above is

$$((D_0 \times D_1) \to D_0).$$

Indeed, we know this function very well: it is the *first projection function* p_0 of 3.3 and the equation

$$p_0 = \lambda x, y.x$$

is true, as is the equation

$$p_1 = \lambda x, y.y.$$

In the notation of 3.3, we also find the true equation

$$\langle f, g\rangle = \lambda w.\langle f(w), g(w)\rangle,$$

where on the right-hand side we are using "official" λ - notation for a function of type

$$(D_2 \to (D_0 \times D_1)).$$

The notation on the left is just an *abbreviation* and it should not be confused with the pair (2-tuple) of type

$$((D_2 \to D_0) \times (D_2 \to D_1)).$$

(Since the two domains just mentioned are *isomorphic*, the possible confusion is not all that serious. On the other hand, one confusion we will completely overlook is that between 1-tuples $\langle x\rangle$ and elements x. Strictly speaking they are different, but we shall not bother to make the distinction.)

Here are some other examples of true equations:

$$\mathrm{eval} = \lambda f, x.f(x) \qquad \text{(cf. 3.11)}$$

$$\mathrm{curry} = \lambda g\lambda x\lambda y.g(x, y) \qquad \text{(cf. 3.12)}$$

The first should be immediately clear; while the second is particularly *instructive*. What is being illustrated is that the λ - notation can

be *iterated*. The distinction being drawn is between

$$\lambda x_0, x_1, \ldots, x_{n-1}.\tau \quad \text{and} \quad \lambda x_0\lambda x_1 \ldots \lambda x_{n-1}.\tau.$$

The first has type

$$((D_0 \times D_1 \times \cdots \times D_{n-1}) \to D_n);$$

while the second has type

$$(D_0 \to (D_1 \to (\ldots (D_{n-1} \to D_n) \ldots))).$$

This is related also to the true equation

$$\mathrm{curry}(\lambda x, y.\tau) = \lambda x\lambda y.\tau,$$

which shows that there are operators relating to the two notations. The first is the *multivariate* form; the second is the *curried* form.

Here is another true equation

$$\mathrm{fix} = \mathrm{fix}(\lambda F\lambda f.f(F(f))),$$

where the fix on the left has type $((D \to D) \to D)$ and that on the right type

$$((((D \to D) \to D) \to ((D \to D) \to D)) \to ((D \to D) \to D)).$$

This is the content of Exercise 4.9. (This also shows why type superscripts are tiresome.)
The combination

$$\mathrm{fix}(\lambda x.\tau)$$

occurs so often, that from time to time we abbreviate it as

$$!x.\tau,$$

but remember it only makes sense if x and τ have the *same* type. For example in 4.3 we could have written

$$\sigma = !f\lambda n.\mathrm{cond}(\mathrm{zero}(n), 0, f(\mathrm{pred}(n)) + \mathrm{pred}(n))$$

and read this as

"σ is the least (recursively defined) function f whose value at n is cond $(\dots)$."

We note that in the so-called "body" of the expression inside the
cond-part the variable f occurs again. That is just the point! This is a *recursive definition*; it is made into an *explicit* definition by invoking the least fixed-point operator.

In a λ-expression, $\lambda x, y, z.\tau$, say, the variables x, y, z are being *bound* in τ; but τ may have other variables that are nowhere bound in τ and these remain *free variables* of the whole expression. Bound variables are *dummy variables* and may be rewritten by other variables; thus

$$\lambda x.\tau = \lambda y.\tau[y/x]$$

is a true equation *PROVIDED* the variable y does not occur in τ. In the equation the notation $\tau[y/x]$ means the result of *substituting* (*rewriting*) the variable y for the variable x throughout the term τ. We can also write $\tau[\sigma/x]$ for substituting a whole term σ for a variable in the other term.

We have already spoken of "true equations", but how do we know that these curious equations are meaningful at all? They are, but this is something that has to be proved.

4.6.2 Theorem 5.1 — terms denote approximable maps

THEOREM 5.1. Every typed λ-term τ defines an approximable function of its free variables.

Proof: We argue by an induction on the complexity of τ; there will only be a few cases to consider since the "syntax" of λ-terms is limited — even though terms can be of any length.

If τ is a variable or a constant there is nothing to prove. We already know that

$$x \mapsto x \qquad \text{and} \qquad x \mapsto k$$

are approximable functions.

Suppose τ has the form

$$\langle \sigma_0, \dots, \sigma_{n-1} \rangle.$$

Then the σ_i are less complex terms, and so we can assume — as our induction hypothesis — that they define approximable functions of the free variables. Having said this, we just apply the already

proved 3.4 to conclude (after a suitable generalization to the multivariate case) that τ, which takes on tuples as values, also defines an approximable function.

Next, suppose τ has the form

$$\sigma_0(\sigma_1),$$

where we are sure that the types of all the terms match properly. Again we can assume the σ_i to be well behaved. But the values we seek can also be written as

$$\mathrm{eval}(\sigma_0, \sigma_1).$$

Since eval is approximable by 3.11, we just have to invoke an instance of 3.7 to gain the desired conclusion.

Finally, suppose that τ has the form

$$\lambda x.\sigma.$$

By a judicious choice of the order of the variables in σ (including x), we can assume that σ defines an approximable function

$$g : D_0 \times \cdots \times D_{n-1} \times D_n \to D'$$

where D' is the type of σ, D_n is the type of x, and $D_0, \ldots, D_{n-1}$ are the types of the remaining free variables of σ. We apply 3.12 and obtain an approximable function

$$\mathrm{curry}(g) : D_0 \times \cdots \times D_{n-1} \to (D_n \to D').$$

But, this is just exactly the function defined by τ.

We leave as an exercise the more general case of a term τ of the form

$$\lambda x_0, \ldots, x_{k-1}.\sigma,$$

which has a string of bound variables. □

We can now say more precisely what it means to call $\sigma = \tau$ a "true equation". This means that, if we employ the method of the proof of 5.1, the two terms define the *same function* of the free variables. For example,

$$\lambda x.\tau = \lambda y.\tau[y/x]$$

is true, provided y does not occur free in the term τ, since the systematic generation of the function defined by $\lambda x.\tau$ does not depend on what the variable x *looks like* but only on its *position* in the term τ. Some other obviously desirable rules for generating true equations are stated in the exercises. But one rule is so basic that we state it here in full generality.

The curry, evaluation, constant, pairing, and projection elementwise laws used in the inductive cases are with the Domain constructs peers (notably Theorems 3.5 / 3.11–3.12, Proposition 3.4, and Lemma 3.6).

Theorem (`theorem_51`). The elementwise laws for currying, evaluation of a paired function-and-argument, constant maps, pairing, and the first projection of a pair hold for approximable maps.

Proof. Invoke the corresponding Domain-constructs lemmas for curry, evaluation of a paired pair, constants, pairing, and the first projection. ■

Theorem_5_1.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.ProjectionMappings
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Theorem 5.1: the construct facts used to interpret typed λ-terms. -/
theorem theorem_51 :
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
        {V₂ : NeighborhoodSystem ?} (g : ApproximableMap (prod V₀ V₁) V₂)
        (v : V₀.Element) (x : V₁.Element),
      (toApproxMap ((curry g).toElementMap v)).toElementMap x = g.toElementMap (pair v
    x)) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
        {V₂ : NeighborhoodSystem ?} (F : ApproximableMap V₂ (funSpace V₀ V₁))
        (G : ApproximableMap V₂ V₀) (w : V₂.Element),
      ((evalMap V₀ V₁).comp (paired F G)).toElementMap w =
        (toApproxMap (F.toElementMap w)).toElementMap (G.toElementMap w)) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
        (k : V₁.Element) (x : V₀.Element), (constMap V₀ k).toElementMap x = k) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
        {V₂ : NeighborhoodSystem ?} (f : ApproximableMap V₂ V₀) (g : ApproximableMap V₂
    V₁)
        (w : V₂.Element),
      (paired f g).toElementMap w = pair (f.toElementMap w) (g.toElementMap w)) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
        (x : V₀.Element) (y : V₁.Element), (proj₀ V₀ V₁).toElementMap (pair x y) = x) :=
  ⟨fun g v x => toElementMap_curry_apply g v x,
   fun F G w => toElementMap_evalMap_comp_paired F G w,
   fun k x => toElementMap_constMap k x,
   fun f g w => toElementMap_paired f g w,
   fun x y => toElementMap_proj₀_pair x y⟩

end DomainTheory.NeighbourhoodSystems
```

4.6.3 Beta conversion

Beta-conversion laws for approximable maps: evaluating a curried map at a constant argument, double currying, and currying a paired map.

Theorem (`evalMap_comp_paired_curry_const`). Evaluating the curry of a binary map paired with a constant argument equals substituting that constant into the uncurried map along the identity on the remaining argument.

Proof. Compare elementwise images, using the evaluation, curry, constant, pairing, and identity laws.

■

Theorem (`toElementMap_curry_curry`). A twice-curried map evaluates by nesting pairs of arguments.

Proof. Apply the curry evaluation law twice. ■

Theorem (`toElementMap_curry_paired`). The curry of a paired pair of maps evaluates to the pair of the curried evaluations.

Proof. Apply the curry and pairing evaluation laws. ■

`beta.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α β ? ? : Type*}
  {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ : NeighborhoodSystem ?}
  {V₃ : NeighborhoodSystem ?}

/-- Beta for a curried map against a constant argument: evaluation of the
curried pair equals substitution into the uncurried map. -/
theorem evalMap_comp_paired_curry_const (g : ApproximableMap (prod V₀ V₁) V₂)
    (σ : V₁.Element) :
    (evalMap V₁ V₂).comp (paired (curry g) (constMap V₀ σ))
      = g.comp (paired (idMap V₀) (constMap V₀ σ)) := by
  apply ext_of_toElementMap
  intro v
  rw [toElementMap_comp, toElementMap_paired, evalMap_apply, toElementMap_curry_apply,
    toElementMap_constMap, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap]

/-- Double curry evaluates by nesting pairs. -/
theorem toElementMap_curry_curry (g : ApproximableMap (prod (prod V₀ V₁) V₂) V₃)
    (v : V₀.Element) (σ : V₁.Element) (y : V₂.Element) :
    (toApproxMap ((toApproxMap ((curry (curry g)).toElementMap v)).toElementMap σ)).
    toElementMap y
      = g.toElementMap (pair (pair v σ) y) := by
  rw [toElementMap_curry_apply, toElementMap_curry_apply]

/-- Curry of a paired map evaluates componentwise. -/
theorem toElementMap_curry_paired (g₀ : ApproximableMap (prod V₀ V₁) V₂)
    (g₁ : ApproximableMap (prod V₀ V₁) V₃) (v : V₀.Element) (x : V₁.Element) :
    (toApproxMap ((curry (paired g₀ g₁)).toElementMap v)).toElementMap x
      = pair ((toApproxMap ((curry g₀).toElementMap v)).toElementMap x)
          ((toApproxMap ((curry g₁).toElementMap v)).toElementMap x) := by
  rw [toElementMap_curry_apply, toElementMap_paired, toElementMap_curry_apply,
    toElementMap_curry_apply]
```

```
end DomainTheory.NeighbourhoodSystems
```

4.6.4 Theorem 5.2 — beta conversion

THEOREM 5.2. For suitably typed λ-terms the following equation is true:

$$(\lambda x_0, \ldots, x_{n-1} \cdot \tau)(\sigma_0, \ldots, \sigma_{n-1}) = \tau[\sigma_0/x_0, \ldots, \sigma_{n-1}/x_{n-1}].$$

Proof: It will be sufficient to carry out the proof for $n = 1$. The proof proceeds by induction on the complexity of the term τ.

In case τ is a *constant* k, the result reads

$$(\lambda x.k)(\sigma) = k,$$

and this is a true equation.

In case τ is a *variable* (in particular, the variable x), the result reads

$$(\lambda x.x)(\sigma) = \sigma,$$

and again this is a true equation.

In case τ is a *tuple* (say, $\langle \tau_0, \tau_1 \rangle$) the result reads

$$(\lambda x.\langle \tau_0, \tau_1 \rangle)(\sigma) = \langle \tau_0[\sigma/x], \tau_1[\sigma/x] \rangle.$$

This is true, because the left-hand side can be transformed by the true equation

$$(\lambda x.\langle \tau_0, \tau_1 \rangle)(\sigma) = \langle (\lambda x.\tau_0)(\sigma), (\lambda x.\tau_1)(\sigma) \rangle;$$

and then we apply the inductive assumption for τ_0 and for τ_1.

In case τ is an *application*, we want (supposing the term is $\tau_0(\tau_1)$),

$$(\lambda x.\tau_0(\tau_1))(\sigma) = \tau_0[\sigma/x](\tau_1[\sigma/x]).$$

We can proceed as in the last case, noting that the left-hand side equals

$$\text{eval } ((\lambda x.\langle \tau_0, \tau_1 \rangle)\ (\sigma))\ .$$

In case τ is an *abstract* (say, $\lambda y.\tau_0$), we want

$$(\lambda x.\lambda y.\tau_0)\ (\sigma) = \lambda y.\tau_0[\sigma/x]$$

PROVIDED the variable y is not free in σ. For this we require the true equation

$$(\lambda x.\lambda y.\tau)\ (\sigma) = \lambda y.(\lambda x.\tau)\ (\sigma)\ .$$

We argue for this by letting g be the function of $n + 2$ free variables defined by τ. Then, by 5.1, the λ-term $\lambda x.\lambda y.\tau$ defines the function $\text{curry } (\text{curry } (g))$ of n arguments. We can call this function h for the moment. We can write

$$h(v)\ (\sigma)\ (y) = g(v, \sigma, y)\ ,$$

where v is a list of arguments. But, with an appropriate combinator inv, which applied to g inverts the order of the last two arguments, we can write

$$h(v)\ (\sigma)\ (y) = \mathrm{curry}\ (\mathrm{inv}\ (g))\ (v, y)\ (\sigma)\ .$$

But, $\mathrm{curry}\ (\mathrm{inv}\ (g))$ is just the function defined by $(\lambda x.\tau)$. So what we have proved as true is

$$(\lambda x.\lambda y.\tau)\ (\sigma)\ (y) = (\lambda x.\tau)\ (\sigma)\ .$$

But if y is not free in α and

$$\alpha(y) = \beta$$

is true, then so is

$$\alpha = \lambda y.\beta\ .$$

This completes the proof. □

We note that if τ' is the term $\lambda x, y.\tau$, then $\tau'(x, y)$ means the same as τ. This gives a convenient way of indicating free variables: we just write $\sigma(x, y)$ — where x, y are not free in σ — and this will have the same values as any term τ which does involve the extra free variables x and y. We use this notational device in the next theorem.

The beta-conversion laws for currying against a constant, double currying, and currying a paired map are with the beta conversion peer.

Theorem (`theorem_52`). Currying a map against a constant argument agrees with substitution into the uncurried map; a twice-curried map evaluates by nesting pairs; and the curry of a paired pair of maps evaluates componentwise.

Proof. Invoke the three beta-conversion laws. ■

Theorem_5_2.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.beta

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Theorem 5.2: beta conversion for currying, abstraction, and tuples. -/
theorem theorem_52 :
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
       {V₂ : NeighborhoodSystem ?} (g : ApproximableMap (prod V₀ V₁) V₂) (σ : V₁.
   Element),
      (evalMap V₁ V₂).comp (paired (curry g) (constMap V₀ σ))
        = g.comp (paired (idMap V₀) (constMap V₀ σ))) ∧
    (∀ {α β ? ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
       {V₂ : NeighborhoodSystem ?} {V₃ : NeighborhoodSystem ?}
       (g : ApproximableMap (prod (prod V₀ V₁) V₂) V₃)
       (v : V₀.Element) (σ : V₁.Element) (y : V₂.Element),
      (toApproxMap ((toApproxMap ((curry (curry g)).toElementMap v)).toElementMap σ)).
   toElementMap y
```

```
        = g.toElementMap (pair (pair v σ) y)) ∧
    (∀ {α β ? ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
        {V₂ : NeighborhoodSystem ?} {V₃ : NeighborhoodSystem ?}
        (g₀ : ApproximableMap (prod V₀ V₁) V₂) (g₁ : ApproximableMap (prod V₀ V₁) V₃)
        (v : V₀.Element) (x : V₁.Element),
      (toApproxMap ((curry (paired g₀ g₁)).toElementMap v)).toElementMap x
        = pair ((toApproxMap ((curry g₀).toElementMap v)).toElementMap x)
            ((toApproxMap ((curry g₁).toElementMap v)).toElementMap x)) :=
  ⟨fun g σ => evalMap_comp_paired_curry_const g σ,
   fun g v σ y => toElementMap_curry_curry g v σ y,
   fun g₀ g₁ v x => toElementMap_curry_paired g₀ g₁ v x⟩

end DomainTheory.NeighbourhoodSystems
```

4.6.5 Paired fixed point

Sectional least fixed points and the least fixed point of a paired pair of binary maps.

Definition (`ApproximableMap.secFixX`). The sectional least-fixed-point map of a binary endomap in the first argument composes the fix combinator with the curry of that map after swapping arguments.

Proof. By definition. ■

Definition (`ApproximableMap.outerOp`). The outer operator of a pair of binary maps composes the second with the pairing of the sectional least-fixed-point map and the identity.

Proof. By definition. ■

Definition (`ApproximableMap.sectionX`). The section of a binary map at a second-argument value is the recovered map after currying the swapped composite.

Proof. By definition. ■

Theorem (`ApproximableMap.outerOp_apply`). The outer operator evaluates by feeding the sectional least fixed point paired with the second argument into the second map.

Proof. Unfold the definition and apply the composition, pairing, and identity laws. ■

Theorem (`ApproximableMap.secFixX_apply`). The sectional least-fixed-point map evaluates to the least fixed-point element of the corresponding section.

Proof. Unfold the definitions and apply the composition and fix-combinator evaluation laws. ■

Theorem (`ApproximableMap.sectionX_apply`). A section evaluates by pairing the free argument with the fixed second argument into the original map.

Proof. Unfold the definition and apply the curry, composition, and swap laws. ■

Theorem (`ApproximableMap.secFixX_isFixed`). The sectional least fixed point is fixed by the original map when paired with the section parameter.

Proof. Rewrite via the sectional evaluation laws and apply the fixed-point law for least fixed-point elements. ■

Theorem (`ApproximableMap.fixElement_paired_eq`). The least fixed point of a paired pair of binary maps is the pair whose second coordinate is the least fixed point of the outer operator and whose first coordinate is the sectional least fixed point at that second coordinate.

Proof. Show the sectional pair is a fixed point of the paired map, then that it lies below the least fixed point by comparing sections and the outer operator against the least paired fixed point's components. ■

`pairedFix.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Combinators
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodEquiv

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

namespace ApproximableMap

def secFixX (τ : ApproximableMap (prod V₀ V₁) V₀) : ApproximableMap V₁ V₀ :=
  (fixMap V₀).comp (curry (τ.comp (swapC V₁ V₀)))

def outerOp (τ : ApproximableMap (prod V₀ V₁) V₀) (σ : ApproximableMap (prod V₀ V₁) V₁)
    :
    ApproximableMap V₁ V₁ :=
  σ.comp (paired (secFixX τ) (idMap V₁))

def sectionX (τ : ApproximableMap (prod V₀ V₁) V₀) (y : V₁.Element) : ApproximableMap V₀
     V₀ :=
  toApproxMap ((curry (τ.comp (swapC V₁ V₀))).toElementMap y)

theorem outerOp_apply (τ : ApproximableMap (prod V₀ V₁) V₀) (σ : ApproximableMap (prod
    V₀ V₁) V₁)
    (y : V₁.Element) :
    (outerOp τ σ).toElementMap y = σ.toElementMap (pair ((secFixX τ).toElementMap y) y)
     := by
  rw [outerOp, toElementMap_comp, toElementMap_paired, toElementMap_idMap]

theorem secFixX_apply (τ : ApproximableMap (prod V₀ V₁) V₀) (y : V₁.Element) :
    (secFixX τ).toElementMap y = (sectionX τ y).fixElement := by
  rw [secFixX, toElementMap_comp, fixMap_toElementMap, sectionX]

theorem sectionX_apply (τ : ApproximableMap (prod V₀ V₁) V₀) (y : V₁.Element) (x : V₀.
    Element) :
    (sectionX τ y).toElementMap x = τ.toElementMap (pair x y) := by
  rw [sectionX, toElementMap_curry_apply, toElementMap_comp, swapC_apply]

theorem secFixX_isFixed (τ : ApproximableMap (prod V₀ V₁) V₀) (y : V₁.Element) :
    τ.toElementMap (pair ((secFixX τ).toElementMap y) y) = (secFixX τ).toElementMap y
    := by
  rw [secFixX_apply, ← sectionX_apply, toElementMap_fixElement]

/-- Least fixed point of a paired pair of binary maps is the sectional pair
`(secFixX τ (outerOp τ σ).fixElement, (outerOp τ σ).fixElement)`. -/
theorem fixElement_paired_eq (τ : ApproximableMap (prod V₀ V₁) V₀)
    (σ : ApproximableMap (prod V₀ V₁) V₁) :
    (paired τ σ).fixElement =
      pair ((secFixX τ).toElementMap (outerOp τ σ).fixElement) (outerOp τ σ).fixElement
     := by
  set ystar := (outerOp τ σ).fixElement with hystar
  set xstar := (secFixX τ).toElementMap ystar with hxstar
```

```
  have hσ : σ.toElementMap (pair xstar ystar) = ystar := by
    have := toElementMap_fixElement (outerOp τ σ)
    rw [← hystar] at this
    rw [outerOp_apply] at this
    rw [hxstar]; exact this
  have hτ : τ.toElementMap (pair xstar ystar) = xstar := by
    rw [hxstar]; exact secFixX_isFixed τ ystar
  apply le_antisymm
  · apply fixElement_le_of_toElementMap_le
    rw [toElementMap_paired, hτ, hσ]
  · set fp := (paired τ σ).fixElement with hfp
    set a := fp.fst with ha
    set b := fp.snd with hb
    have hfpeq : pair (τ.toElementMap fp) (σ.toElementMap fp) = fp := by
      have := toElementMap_fixElement (paired τ σ)
      rw [← hfp, toElementMap_paired] at this
      exact this
    have hfp_pair : fp = pair a b := (pair_fst_snd fp).symm
    have hτa : τ.toElementMap (pair a b) = a := by
      have : (pair (τ.toElementMap fp) (σ.toElementMap fp)).fst = fp.fst := by rw [
  hfpeq]
      rw [fst_pair] at this
      rw [← hfp_pair, ha]; exact this
    have hσb : σ.toElementMap (pair a b) = b := by
      have : (pair (τ.toElementMap fp) (σ.toElementMap fp)).snd = fp.snd := by rw [
  hfpeq]
      rw [snd_pair] at this
      rw [← hfp_pair, hb]; exact this
    have hxfix_le : (secFixX τ).toElementMap b ≤ a := by
      rw [secFixX_apply]
      apply fixElement_le_of_toElementMap_le
      rw [sectionX_apply]
      exact le_of_eq hτa
    have houter_le : (outerOp τ σ).toElementMap b ≤ b := by
      rw [outerOp_apply]
      calc σ.toElementMap (pair ((secFixX τ).toElementMap b) b)
          ≤ σ.toElementMap (pair a b) :=
            σ.toElementMap_mono (pair_le_pair_iff.mpr ⟨hxfix_le, le_refl b⟩)
        _ = b := hσb
    have hystar_le : ystar ≤ b := by
      rw [hystar]
      exact fixElement_le_of_toElementMap_le (outerOp τ σ) houter_le
    have hxstar_le : xstar ≤ a := by
      rw [hxstar]
      calc (secFixX τ).toElementMap ystar
          ≤ (secFixX τ).toElementMap b := (secFixX τ).toElementMap_mono hystar_le
        _ ≤ a := hxfix_le
    rw [hfp_pair]
    exact pair_le_pair_iff.mpr ⟨hxstar_le, hystar_le⟩

end ApproximableMap
end DomainTheory.NeighbourhoodSystems
```

4.6.6 Proposition 5.3 — simultaneous fixed points

PROPOSITION 5.3. The least fixed point of $\lambda x, y.\ \langle \tau(x,y), \sigma(x,y)\rangle$ is the pair with coordinates
$!x.\ \tau(x,\ !y.\ \sigma(x,y))$ and
$!y.\ \sigma(!x.\ \tau(x,y), y)$.

Proof: (We are assuming that x and y are not free in τ and σ.) The purpose of the fixed-point search is to find the least solution of the pair of equations $x = \tau(x,y)$ and $y = \sigma(x,y)$. In other words, we are generalizing the fixed-point equation from one to two variables—and, of course, we could go much further to any number of variables. To this end, let
$y_* = !y.\ \sigma(!x.\ \tau(x,y), y)$, and
$x_* = !x.\ \tau(x, y_*)$.
Then
$x_* = \tau(x_*, y_*)$,
and
$y_* = \sigma(!x.\ \tau(x, y_*), y_*)$
$= \sigma(x_*, y_*)$.
This proves that $\langle x_*, y_*\rangle$ is one fixed-point pair.

Suppose, then, that $\langle x_0, y_0\rangle$ is the least solution. (Why does a least solution have to exist? Hint: Consider a suitable mapping of type $D_0 \times D_1 \to D_0 \times D_1$, where D_0 is the type of x and D_1 the type of y.) Then we know $x_0 = \tau(x_0, y_0)$ and $y_0 = \sigma(x_0, y_0)$, and also $x_0 \sqsubseteq x_*$ and $y_0 \sqsubseteq y_*$. But from
$\tau(x_0, y_0) \sqsubseteq x_0$,
it follows that
$!x.\ \tau(x, y_0) \sqsubseteq x_0$.
Consequently,

$$\sigma(!x.\tau(x, y_0), y_0) \sqsubseteq \sigma(x_0, y_0) \sqsubseteq y_0.$$

By the fixed-point definition of y_*, we have $y_* \sqsubseteq y_0$, so $y_* = y_0$; whence,

$$x_* = !x.\tau(x, y_*) = !x.\tau(x, y_0) \sqsubseteq x_0.$$

So also $x_* = x_0$. □ We have the right formula for y_0, and a similar argument gives x_0.

The purpose of giving the above proof was to illustrate the use of the least fixed-point operator in proofs. We have such true principles as:

$$!x.\tau(x) = \tau(!x.\tau(x));$$

and

$$\tau(y) \sqsubseteq y \text{ implies } !x.\tau(x) \sqsubseteq y,$$

provided, of course, that x is not free in τ. These, together with the monotonicity of all the functions, were just the methods used in the above proof. Here is another example.

The sectional least-fixed-point / outer-operator characterisation of the least paired fixed point is with the paired fixed point peer.

Theorem (`proposition_53`). The least fixed point of a paired pair of binary maps is the sectional pair built from the outer operator's least fixed point.

Proof. Invoke the paired least-fixed-point law. ■

Proposition_5_3.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.pairedFix

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Proposition 5.3: least fixed point of a paired pair of binary maps. -/
theorem proposition_53 :
    ∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (τ : ApproximableMap (prod V₀ V₁) V₀) (σ : ApproximableMap (prod V₀ V₁) V₁),
      (paired τ σ).fixElement =
        pair ((secFixX τ).toElementMap (outerOp τ σ).fixElement) (outerOp τ σ).
    fixElement :=
  @fixElement_paired_eq

end DomainTheory.NeighbourhoodSystems
```

4.6.7 Parametric fixed point

Parametric least fixed points and the recursion operator on the function space.

Definition (`ApproximableMap.pfix`). The parametric least-fixed-point map composes the fix combinator with the curry of a binary endomap.

Proof. By definition. ■

Definition (`ApproximableMap.recOp`). The recursion operator curries the composite of a binary endomap with the pairing of the second projection and evaluation.

Proof. By definition. ■

Definition (`ApproximableMap.section2`). The section of a binary endomap at a first-argument value is the recovered map after currying.

Proof. By definition. ■

Theorem (`ApproximableMap.pfix_apply`). The parametric least-fixed-point map evaluates to the least fixed-point element of the corresponding section.

Proof. Unfold the definitions and apply the composition and fix-combinator evaluation laws. ■

Theorem (`ApproximableMap.recOp_apply`). The recursion operator evaluates by pairing an argument with the recovered function's value at that argument.

Proof. Unfold curry and apply the composition, pairing, projection, and evaluation laws. ■

Theorem (`ApproximableMap.section2_apply`). A section evaluates by pairing the fixed first argument with the free second argument into the original map.

Proof. Unfold the definition and apply the curry evaluation law. ■

Theorem (`ApproximableMap.pfix_isFixed`). The parametric least fixed point is fixed by the original map when paired with the section parameter.

Proof. Rewrite via the sectional evaluation laws and apply the fixed-point law for least fixed-point elements. ■

Theorem (`ApproximableMap.pfix_eq_fixElement_recOp`). The parametric least-fixed-point map equals the map recovered from the least fixed-point element of the recursion operator.

Proof. Show the recursion-operator fix yields a postfixed point for each section, hence lies above the parametric fix; conversely, the filter of the parametric fix is a postfixed point of the recursion operator, so the recursion-operator fix lies below it after transport through the function-space equivalence. ■

paramFix.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α : Type*} {V : NeighborhoodSystem α}

namespace ApproximableMap

def pfix (τ : ApproximableMap (prod V V) V) : ApproximableMap V V :=
  (fixMap V).comp (curry τ)

def recOp (τ : ApproximableMap (prod V V) V) : ApproximableMap (funSpace V V) (funSpace
    V V) :=
  curry (τ.comp (paired (proj₁ (funSpace V V) V) (evalMap V V)))

def section₂ (τ : ApproximableMap (prod V V) V) (x : V.Element) : ApproximableMap V V
    :=
  toApproxMap ((curry τ).toElementMap x)

theorem pfix_apply (τ : ApproximableMap (prod V V) V) (x : V.Element) :
    (pfix τ).toElementMap x = (section₂ τ x).fixElement := by
  rw [pfix, toElementMap_comp, fixMap_toElementMap, section₂]

theorem recOp_apply (τ : ApproximableMap (prod V V) V) (φ : (funSpace V V).Element) (x
    : V.Element) :
    (toApproxMap ((recOp τ).toElementMap φ)).toElementMap x
      = τ.toElementMap (pair x ((toApproxMap φ).toElementMap x)) := by
  rw [recOp, toElementMap_curry_apply, toElementMap_comp, toElementMap_paired,
    toElementMap_proj₁, snd_pair, evalMap_apply]

theorem section₂_apply (τ : ApproximableMap (prod V V) V) (x y : V.Element) :
    (section₂ τ x).toElementMap y = τ.toElementMap (pair x y) := by
  rw [section₂, toElementMap_curry_apply]

theorem pfix_isFixed (τ : ApproximableMap (prod V V) V) (x : V.Element) :
    τ.toElementMap (pair x ((pfix τ).toElementMap x)) = (pfix τ).toElementMap x := by
  rw [pfix_apply, ← section₂_apply, toElementMap_fixElement]

/-- Parametric least fixed point equals the least fixed point of the recursion
    operator. -/
theorem pfix_eq_fixElement_recOp (τ : ApproximableMap (prod V V) V) :
```

```
    pfix τ = toApproxMap (recOp τ).fixElement := by
  set g₀ := (recOp τ).fixElement with hg₀
  set G := toApproxMap g₀ with hG
  have hG_fixed : ∀ x, G.toElementMap x = τ.toElementMap (pair x (G.toElementMap x)) :=
    by
    intro x
    have hfix : (recOp τ).toElementMap g₀ = g₀ := toElementMap_fixElement (recOp τ)
    have := recOp_apply τ g₀ x
    rw [hfix] at this
    rw [hG]; exact this
  apply le_antisymm
  · rw [le_iff_toElementMap_le]
    intro x
    rw [pfix_apply]
    apply fixElement_le_of_toElementMap_le
    rw [section₂_apply]
    exact le_of_eq (hG_fixed x).symm
  · have hpre : (recOp τ).toElementMap (toFilter (pfix τ)) ≤ toFilter (pfix τ) := by
      apply le_of_eq
      have hmap : toApproxMap ((recOp τ).toElementMap (toFilter (pfix τ))) = pfix τ :=
    by
        apply ext_of_toElementMap
        intro x
        rw [recOp_apply]
        have hround : toApproxMap (toFilter (pfix τ)) = pfix τ := by
          have he := (funSpaceEquiv V V).apply_symm_apply (pfix τ)
          rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he
        rw [hround, pfix_isFixed]
      have h1 : toFilter (toApproxMap ((recOp τ).toElementMap (toFilter (pfix τ)))) =
          toFilter (pfix τ) := by rw [hmap]
      have hround2 : toFilter (toApproxMap ((recOp τ).toElementMap (toFilter (pfix τ)))
    ) =
          (recOp τ).toElementMap (toFilter (pfix τ)) := by
        have he := (funSpaceEquiv V V).symm_apply_apply ((recOp τ).toElementMap (
    toFilter (pfix τ)))
        rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he
      rw [hround2] at h1
      exact h1
    have hle : g₀ ≤ toFilter (pfix τ) := fixElement_le_of_toElementMap_le (recOp τ)
    hpre
    have := (funSpaceEquiv V V).monotone hle
    rw [funSpaceEquiv_apply, funSpaceEquiv_apply] at this
    have hround : toApproxMap (toFilter (pfix τ)) = pfix τ := by
      have he := (funSpaceEquiv V V).apply_symm_apply (pfix τ)
      rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he
    rw [hG, ← hround]
    exact this

end ApproximableMap
end DomainTheory.NeighbourhoodSystems
```

4.6.8 Proposition 5.4 — fixed point under a parameter

PROPOSITION 5.4. Let x, y, and $\tau(x, y)$ be of the same type D and let g be of type $(D \to D)$, then the equation

$$\lambda x.!y.\tau(x, y) =!g.\lambda x.\tau(x, g(x))$$

is true.

Proof : Let f be the function on the left-hand side. We can write

$$f(x) =!y.\tau(x, y) = \tau(x, f(x)).$$

Therefore

$$f = \lambda x.\tau(x, f(x)),$$

and it follows that

$$g_0 =!g.\lambda x.\tau(x, g(x)) \sqsubseteq f.$$

Then we have at once, by definition of g_0,

$$g_0(x) = \tau(x, g_0(x)),$$

for any given x. But by definition of f we find

$$f(x) =!y.\tau(x, y) \sqsubseteq g_0(x).$$

As this holds for all x, then $f \sqsubseteq g_0$ follows. So the equation is true. □

The last proof is instructive as it uses equations and inclusions between functions. In particular we have just made use of the principle:

> if $\tau \sqsubseteq \sigma$ holds for all values of x,
> then $\lambda x.\tau \sqsubseteq \lambda x.\sigma$ holds.

This is another form of Theorem 3.13(i).

The parametric least-fixed-point / recursion-operator characterisation is with the parametric fixed point peer.

Theorem (`proposition_54`). The parametric least-fixed-point map equals the map recovered from the least fixed-point element of the recursion operator.

Proof. Invoke the parametric least-fixed-point law. ■

`Proposition_5_4.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.paramFix

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
```

```
/-- Scott's Proposition 5.4: parametric least fixed point equals recursion-operator
    fix. -/
theorem proposition_54 :
    ∀ {α : Type*} {V : NeighborhoodSystem α} (τ : ApproximableMap (prod V V) V),
      pfix τ = toApproxMap (recOp τ).fixElement :=
  @pfix_eq_fixElement_recOp

end DomainTheory.NeighbourhoodSystems
```

4.6.9 Table 5.5 — table of combinators

TABLE 5.5. In the displayed table we give a summary of uses of the λ-notation to define various combinators. We have mentioned some of these equations before, and there are some combinators here we have not mentioned before — their meanings, however, should be clear.

$$
\begin{aligned}
P_0 &= \lambda x, y.x \\
P_1 &= \lambda x, y.y \\
\text{pair} &= \lambda x \lambda y.\langle x, y\rangle \\
\text{n-tuple} &= \lambda x_0 \lambda x_1 \ldots \lambda x_{n-1}.\langle x_0, x_1, \ldots, x_{n-1}\rangle \\
\text{diag} &= \lambda x.\langle x, x\rangle \\
\text{funpair} &= \lambda f \lambda g \lambda x.\langle f(x), g(x)\rangle \\
\text{proj}_i^n &= \lambda x_0, x_1, \ldots, x_{n-1}.x_i \\
\text{inv}_{i,j}^n &= \lambda x_0, \ldots, x_i, \ldots, x_j, \ldots, x_{n-1}.\langle x_0, \ldots, x_j, \ldots, x_i, \ldots, x_{n-1}\rangle
\end{aligned}
$$

$$
\begin{aligned}
\text{eval} &= \lambda f, x.f(x) \\
\text{curry} &= \lambda g \lambda x \lambda y.g(x, y) \\
\text{comp} &= \lambda g, f \lambda x.g(f(x)) \\
\text{const} &= \lambda k \lambda x.k \\
\text{fix} &= \lambda f! x.f(x)
\end{aligned}
$$

A TABLE OF COMBINATORS

It is important to note that since we have not typed the variables, these equations are ambiguous: they only become precise when the types are specified. It follows, therefore, that what we find in the table are *schemes* for combinators; there are actually infinitely many distinct combinators corresponding to any one equation depending on how the variables have types chosen for them. Clearly it is better to imagine this variety of combinators than it is to try to notate them with type superscripts.

One interest of combinators is that it is often possible to write expressions without variables — if enough combinators are used. This is sometimes useful, but it can become clumsy. On the other hand, if the same combination occurs over and over, it is sometimes useful to give it a name. This is what we do with, say, *composition* where

$$\text{comp}\ (g, f) = g \circ f.$$

On the one side we have the prefix notation, and on the other, the more common infix notation. With either notation the variable seen in $\lambda x.\ g(f(x))$ has been got rid of. The choice between equivalent notations ought to be based on a desire for readability. □

The reader will have noted that there are some combinators not appearing in Table 5.5. The reason is that combinators like cond, succ, pred, zero, 0 cannot be defined in the pure λ-notation but are specific to domains like T and $\mathbf{N}$; we, thus, have to regard them as primitive. But once they are in hand, a very large number of other functions can be defined from these combined with λ-expressions. The next theorem gives an indication of the possibilities.

Definition (`P0`). The first-projection combinator is the product first projection.

Proof. By definition. ■

Theorem (`P0_apply`). The first-projection combinator recovers the first component of a pair.

Proof. Unfold the definition and apply the projection and pairing laws. ■

Definition (`P1`). The second-projection combinator is the product second projection.

Proof. By definition. ■

Theorem (`P1_apply`). The second-projection combinator recovers the second component of a pair.

Proof. Unfold the definition and apply the projection and pairing laws. ■

Definition (`diagC`). The diagonal combinator pairs the identity with itself.

Proof. By definition. ■

Theorem (`diagC_apply`). The diagonal combinator sends an element to its pair with itself.

Proof. Unfold the definition and apply the pairing and identity laws. ■

Definition (`pairC`). The pairing combinator is the curry of the product identity.

Proof. By definition. ■

Definition (`swapC`). The swap combinator pairs the second projection with the first.

Proof. By definition. ■

Theorem (`swapC_apply`). The swap combinator exchanges the components of a pair.

Proof. Unfold the definition and apply the pairing and projection laws. ■

Definition (`compMapTbl`). The uncurried composition table evaluates the outer function at the value of the inner function on the remaining argument, assembled from projections and evaluations.

Proof. By definition. ■

Definition (`compC`). The composition combinator is the curry of that uncurried composition table.

Proof. By definition. ■

Definition (`constC`). The constant combinator is the curry of the first projection out of a product with a dummy argument domain.

Proof. By definition. ■

Definition (`curryIso`). The curry order isomorphism transports function-space filters through the curry equivalence of approximable maps.

Proof. Compose the function-space filter equivalence with the curry equivalence and the inverse filter equivalence. ■

Definition (`curryC`). The curry combinator is the approximable map induced by that order isomorphism.

Proof. By definition. ■

Definition (`evalC`). The evaluation combinator is the evaluation map of the function space.

Proof. By definition. ■

Definition (`fixC`). The fix combinator of the table is the least-fixed-point combinator on the function space.

Proof. By definition. ■

Definition (`funpairMapTbl`). The uncurried function-pairing table evaluates two function arguments at a shared input and pairs the results.

Proof. By definition. ■

Definition (`funpairC`). The function-pairing combinator is the double curry of that uncurried table.

Proof. By definition. ■

Theorem (`compC_apply`). Composition of function-space elements under the composition combinator evaluates as ordinary composition of the recovered maps.

Proof. Unfold curry and the composition table; simplify with projection, pairing, and evaluation laws. ■

Theorem (`compC_eq_comp`). The map recovered from the composition combinator is the composite of the recovered maps.

Proof. Compare elementwise images via the previous evaluation law. ■

Theorem (`constC_apply`). The constant combinator recovers the constant value at every argument.

Proof. Unfold curry and apply the first-projection law. ■

Theorem (`constC_eq_constMap`). The map recovered from the constant combinator is the constant map.

Proof. Compare elementwise images via the previous evaluation law and the constant-map law. ■

Theorem (`curryC_toApproxMap`). The map recovered from the curry combinator is the curry of the recovered map.

Proof. Unfold the isomorphism-induced map and cancel the filter round trip through the function-space equivalence. ■

Theorem (`curryC_apply`). The curry combinator evaluates by pairing arguments into the original map.

Proof. Rewrite via the previous recovery law and apply the curry evaluation law. ■

Theorem (`evalC_apply`). The evaluation combinator evaluates a function-space element at an argument.

Proof. Unfold the definition and apply the evaluation law. ■

Theorem (`fixC_apply`). The fix combinator of the table recovers the least fixed-point element of the recovered map.

Proof. Unfold the definition and apply the fix-combinator evaluation law. ■

Theorem (`fixC_fixed`). The value of the fix combinator is a fixed point of the recovered map.

Proof. Unfold the definition and apply the fixed-point law for the fix combinator. ■

Theorem (`funpairC_apply`). The function-pairing combinator evaluates to the pair of the two recovered values at a shared argument.

Proof. Unfold the double curry and the pairing table; simplify with projection, pairing, and evaluation laws. ■

Theorem (`funpairC_eq_paired`). The map recovered from the function-pairing combinator is the pairing of the recovered maps.

Proof. Compare elementwise images via the previous evaluation law and the pairing law. ■

Theorem (`pairC_apply`). The pairing combinator evaluates to the pair of its two arguments.

Proof. Unfold curry and apply the identity law. ■

`Combinators.lean`

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.unarySeq
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofIso
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {α β ? : Type*}
  {V : NeighborhoodSystem α} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  {V₂ : NeighborhoodSystem ?}

def P₀ (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) : ApproximableMap (prod
    V₀ V₁) V₀ :=
  proj₀ V₀ V₁

@[simp] theorem P₀_apply (x : V₀.Element) (y : V₁.Element) :
    (P₀ V₀ V₁).toElementMap (pair x y) = x := by
  rw [P₀, toElementMap_proj₀, fst_pair]

def P₁ (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) : ApproximableMap (prod
    V₀ V₁) V₁ :=
  proj₁ V₀ V₁

@[simp] theorem P₁_apply (x : V₀.Element) (y : V₁.Element) :
    (P₁ V₀ V₁).toElementMap (pair x y) = y := by
  rw [P₁, toElementMap_proj₁, snd_pair]

def diagC (V : NeighborhoodSystem α) : ApproximableMap V (prod V V) := paired (idMap V
    ) (idMap V)

@[simp] theorem diagC_apply (x : V.Element) : (diagC V).toElementMap x = pair x x :=
    by
  rw [diagC, toElementMap_paired, toElementMap_idMap]

def pairC (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    ApproximableMap V₀ (funSpace V₁ (prod V₀ V₁)) := curry (idMap (prod V₀ V₁))

def swapC (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    ApproximableMap (prod V₀ V₁) (prod V₁ V₀) := paired (proj₁ V₀ V₁) (proj₀ V₀ V₁)

@[simp] theorem swapC_apply (x : V₀.Element) (y : V₁.Element) :
    (swapC V₀ V₁).toElementMap (pair x y) = pair y x := by
  rw [swapC, toElementMap_paired, toElementMap_proj₁, toElementMap_proj₀, snd_pair,
    fst_pair]

def compMapTbl (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
    NeighborhoodSystem ?) :
    ApproximableMap (prod (prod (funSpace V₁ V₂) (funSpace V₀ V₁)) V₀) V₂ :=
  (evalMap V₁ V₂).comp
```

```
    (paired
      ((proj₀ (funSpace V₁ V₂) (funSpace V₀ V₁)).comp
        (proj₀ (prod (funSpace V₁ V₂) (funSpace V₀ V₁)) V₀))
      ((evalMap V₀ V₁).comp
        (paired
          ((proj₁ (funSpace V₁ V₂) (funSpace V₀ V₁)).comp
            (proj₀ (prod (funSpace V₁ V₂) (funSpace V₀ V₁)) V₀))
          (proj₁ (prod (funSpace V₁ V₂) (funSpace V₀ V₁)) V₀))))

def compC (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
    NeighborhoodSystem ?) :
    ApproximableMap (prod (funSpace V₁ V₂) (funSpace V₀ V₁)) (funSpace V₀ V₂) :=
  curry (compMapTbl V₀ V₁ V₂)

def constC (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    ApproximableMap V₁ (funSpace V₀ V₁) := curry (proj₀ V₁ V₀)

def curryIso (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
    NeighborhoodSystem ?) :
    (funSpace (prod V₀ V₁) V₂).Element ≃o (funSpace V₀ (funSpace V₁ V₂)).Element :=
  (funSpaceEquiv (prod V₀ V₁) V₂).trans
    ((curryEquiv V₀ V₁ V₂).trans (funSpaceEquiv V₀ (funSpace V₁ V₂)).symm)

def curryC (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
    NeighborhoodSystem ?) :
    ApproximableMap (funSpace (prod V₀ V₁) V₂) (funSpace V₀ (funSpace V₁ V₂)) :=
  ofIso (curryIso V₀ V₁ V₂)

def evalC (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    ApproximableMap (prod (funSpace V₀ V₁) V₀) V₁ := evalMap V₀ V₁

def fixC (V : NeighborhoodSystem α) : ApproximableMap (funSpace V V) V := fixMap V

def funpairMapTbl (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (V₂ : NeighborhoodSystem ?) :
    ApproximableMap (prod (prod (funSpace V₂ V₀) (funSpace V₂ V₁)) V₂) (prod V₀ V₁) :=
  paired
    ((evalMap V₂ V₀).comp
      (paired
        ((proj₀ (funSpace V₂ V₀) (funSpace V₂ V₁)).comp
          (proj₀ (prod (funSpace V₂ V₀) (funSpace V₂ V₁)) V₂))
        (proj₁ (prod (funSpace V₂ V₀) (funSpace V₂ V₁)) V₂)))
    ((evalMap V₂ V₁).comp
      (paired
        ((proj₁ (funSpace V₂ V₀) (funSpace V₂ V₁)).comp
          (proj₀ (prod (funSpace V₂ V₀) (funSpace V₂ V₁)) V₂))
        (proj₁ (prod (funSpace V₂ V₀) (funSpace V₂ V₁)) V₂)))

def funpairC (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
    NeighborhoodSystem ?) :
    ApproximableMap (funSpace V₂ V₀)
      (funSpace (funSpace V₂ V₁) (funSpace V₂ (prod V₀ V₁))) :=
  curry (curry (funpairMapTbl V₀ V₁ V₂))

end
```

```
section
variable {α β ? : Type*}
  {V : NeighborhoodSystem α} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

theorem compC_apply (φ : (funSpace V₁ V₂).Element) (ψ : (funSpace V₀ V₁).Element) (x :
    V₀.Element) :
    (toApproxMap ((compC V₀ V₁ V₂).toElementMap (pair φ ψ))).toElementMap x
      = (toApproxMap φ).toElementMap ((toApproxMap ψ).toElementMap x) := by
  rw [compC, toElementMap_curry_apply, compMapTbl]
  simp only [toElementMap_comp, toElementMap_paired, toElementMap_proj₀,
    toElementMap_proj₁,
    fst_pair, snd_pair, evalMap_apply]

theorem compC_eq_comp (φ : (funSpace V₁ V₂).Element) (ψ : (funSpace V₀ V₁).Element) :
    toApproxMap ((compC V₀ V₁ V₂).toElementMap (pair φ ψ)) = (toApproxMap φ).comp (
    toApproxMap ψ) := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, compC_apply]

theorem constC_apply (k : V₁.Element) (x : V₀.Element) :
    (toApproxMap ((constC V₀ V₁).toElementMap k)).toElementMap x = k := by
  rw [constC, toElementMap_curry_apply, toElementMap_proj₀, fst_pair]

theorem constC_eq_constMap (k : V₁.Element) :
    toApproxMap ((constC V₀ V₁).toElementMap k) = constMap V₀ k := by
  apply ext_of_toElementMap
  intro x
  rw [constC_apply, toElementMap_constMap]

theorem curryC_toApproxMap (φ : (funSpace (prod V₀ V₁) V₂).Element) :
    toApproxMap ((curryC V₀ V₁ V₂).toElementMap φ) = curry (toApproxMap φ) := by
  rw [curryC, toElementMap_ofIso]
  show toApproxMap (toFilter (curry (toApproxMap φ))) = curry (toApproxMap φ)
  have he := (funSpaceEquiv V₀ (funSpace V₁ V₂)).apply_symm_apply (curry (toApproxMap φ
    ))
  rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he

theorem curryC_apply (φ : (funSpace (prod V₀ V₁) V₂).Element) (x : V₀.Element) (y : V₁.
    Element) :
    (toApproxMap ((toApproxMap ((curryC V₀ V₁ V₂).toElementMap φ)).toElementMap x)).
    toElementMap y
      = (toApproxMap φ).toElementMap (pair x y) := by
  rw [curryC_toApproxMap, toElementMap_curry_apply]

theorem evalC_apply (φ : (funSpace V₀ V₁).Element) (x : V₀.Element) :
    (evalC V₀ V₁).toElementMap (pair φ x) = (toApproxMap φ).toElementMap x := by
  rw [evalC, evalMap_apply]

theorem fixC_apply (φ : (funSpace V V).Element) :
    (fixC V).toElementMap φ = (toApproxMap φ).fixElement := by
  rw [fixC, fixMap_toElementMap]

theorem fixC_fixed (φ : (funSpace V V).Element) :
    (toApproxMap φ).toElementMap ((fixC V).toElementMap φ) = (fixC V).toElementMap φ
    := by
```

```
  rw [fixC]; exact fixMap_fixed V φ

theorem funpairC_apply (φ : (funSpace V₂ V₀).Element) (ψ : (funSpace V₂ V₁).Element)
    (x : V₂.Element) :
    (toApproxMap ((toApproxMap ((funpairC V₀ V₁ V₂).toElementMap φ)).toElementMap ψ)).
   toElementMap x
     = pair ((toApproxMap φ).toElementMap x) ((toApproxMap ψ).toElementMap x) := by
  rw [funpairC, toElementMap_curry_apply, toElementMap_curry_apply, funpairMapTbl]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj₀,
    toElementMap_proj₁,
    fst_pair, snd_pair, evalMap_apply]

theorem funpairC_eq_paired (φ : (funSpace V₂ V₀).Element) (ψ : (funSpace V₂ V₁).Element
    ) :
    toApproxMap ((toApproxMap ((funpairC V₀ V₁ V₂).toElementMap φ)).toElementMap ψ)
      = paired (toApproxMap φ) (toApproxMap ψ) := by
  apply ext_of_toElementMap
  intro x
  rw [funpairC_apply, toElementMap_paired]

theorem pairC_apply (x : V₀.Element) (y : V₁.Element) :
    (toApproxMap ((pairC V₀ V₁).toElementMap x)).toElementMap y = pair x y := by
  rw [pairC, toElementMap_curry_apply, toElementMap_idMap]

end
end DomainTheory.NeighbourhoodSystems
end
```

4.6.10 Number-domain recursion schemes

Number-domain recursion schemes: the μ-operator and primitive recursion on pairs of numbers, together with strict identity and projection.

Definition (`kSel`). The construction `kSel` as used in the number-domain / partial-recursive λ-definability development.

Proof. By definition. ■

Definition (`qSel`). The construction `qSel` as used in the number-domain / partial-recursive λ-definability development.

Proof. By definition. ■

Definition (`argX`). The construction `argX` as used in the number-domain / partial-recursive λ-definability development.

Proof. By definition. ■

Definition (`argY`). The construction `argY` as used in the number-domain / partial-recursive λ-definability development.

Proof. By definition. ■

Definition (`muBody`). The construction `muBody` as used in the number-domain / partial-recursive λ-definability development.

Proof. By definition. ■

Definition (`muOp`). The construction `muOp` as used in the number-domain / partial-recursive λ-definability development.

Proof. By definition. ■

Definition (`muRec`). Binary μ-recursion is the least fixed point of the μ-operator on the binary function space.

Proof. By definition. ■

Definition (`muMap`). The unary μ-map searches from zero along the second argument of a binary number-domain map.

Proof. Compose the binary μ-recursion with the pairing of the zero constant and the identity. ■

Definition (`primBody`). The construction `primBody` as used in the number-domain / partial-recursive λ-definability development.

Proof. By definition. ■

Definition (`primOp`). The construction `primOp` as used in the number-domain / partial-recursive λ-definability development.

Proof. By definition. ■

Definition (`primRec`). Binary primitive recursion is the least fixed point of the corresponding primitive-recursion operator.

Proof. By definition. ■

Definition (`strictId`). The strict identity on the number domain, defined via the conditional on the zero-test.

Proof. By definition. ■

Definition (`strictProj0`). The strict first projection on pairs of numbers.

Proof. By definition. ■

Theorem (`T_bot_eq`). The bottom of the truth domain agrees with the two-token bottom element.

Proof. Antisymmetry of the bottoms. ■

Theorem (`muBody_apply`). The stated property of `muBody_apply`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`muMap_natElem`). The stated property of `muMap_natElem`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`muRec_rec`). The stated property of `muRec_rec`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`muRec_bot`). The stated property of `muRec_bot`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`muRec_found`). The stated property of `muRec_found`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`muRec_step`). The stated property of `muRec_step`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`muRec_climb`). The stated property of `muRec_climb`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`muMap_eq_least`). When the predicate first hits zero at n0 with positive values before, the μ-map returns n0.

Proof. Climb the binary μ-recursion from zero and apply the found-zero clause. ■

Theorem (`primBody_apply`). The stated property of `primBody_apply`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`primRec_rec`). The stated property of `primRec_rec`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`primRec_bot`). The stated property of `primRec_bot`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`primRec_succ`). The stated property of `primRec_succ`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`primRec_zero`). The stated property of `primRec_zero`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`strictId_bot`). The stated property of `strictId_bot`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`strictId_natElem`). The stated property of `strictId_natElem`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`strictProj0_apply`). The stated property of `strictProj0_apply`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`strictProj0_bot_left`). The stated property of `strictProj0_bot_left`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`strictProj0_bot_right`). The stated property of `strictProj0_bot_right`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`strictProj0_natElem`). The stated property of `strictProj0_natElem`.
Proof. Follows from the surrounding construct and fixed-point laws. ■

numRecSchemes.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.numberDomain
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.cond
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.NumRecSchemes
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.NatDom (N natElem zeroElt succMap predMap
    zeroMap
  succMap_natElem succMap_bot predMap_natElem_succ predMap_natElem_zero predMap_bot
  zeroMap_natElem_zero zeroMap_natElem_succ zeroMap_bot constLiftN constLiftN_natElem)
open DomainTheory.NeighbourhoodSystems.Cond (cond)

def kSel : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) (funSpace (prod N
     N) N) :=
  proj₀ (funSpace (prod N N) N) (prod N N)

def qSel : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) (prod N N) :=
  proj₁ (funSpace (prod N N) N) (prod N N)

def argX : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) N := (proj₀ N N).
    comp qSel

def argY : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) N := (proj₁ N N).
    comp qSel
```

```
def strictId : ApproximableMap N N :=
  (cond N).comp (paired zeroMap (paired (idMap N) (idMap N)))

def strictProj₀ : ApproximableMap (prod N N) N :=
  (cond N).comp
    (paired (zeroMap.comp (proj₁ N N))
      (paired (proj₀ N N) (proj₀ N N)))

theorem T_bot_eq : (NatDom.T).bot = Truth.botElt :=
  le_antisymm ((NatDom.T).bot_le _) (Truth.botElt_le _)

section Mu
variable (f : ApproximableMap (prod N N) N)

def muBody : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) N :=
  (cond N).comp
    (paired (zeroMap.comp (f.comp qSel))
      (paired argX
        ((evalMap (prod N N) N).comp
          (paired kSel (paired (succMap.comp argX) argY)))))

def muOp : ApproximableMap (funSpace (prod N N) N) (funSpace (prod N N) N) :=
  curry (muBody f)

def muRec : ApproximableMap (prod N N) N := toApproxMap (muOp f).fixElement

def muMap : ApproximableMap N N := muRec f |>.comp (paired (constLiftN N (fun _ =>
    zeroElt)) (idMap N))

theorem muBody_apply (k : (funSpace (prod N N) N).Element) (x y : N.Element) :
    (muBody f).toElementMap (pair k (pair x y)) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap (f.toElementMap (pair x y)))
          (pair x ((toApproxMap k).toElementMap (pair (succMap.toElementMap x) y))))
    := by
  rw [muBody, toElementMap_comp]
  simp only [argX, argY, kSel, qSel, toElementMap_paired, toElementMap_comp,
    toElementMap_proj₀,
    toElementMap_proj₁, fst_pair, snd_pair, evalMap_apply]

theorem muMap_natElem (m : ℕ) :
    (muMap f).toElementMap (natElem m) = (muRec f).toElementMap (pair zeroElt (natElem
     m)) := by
  rw [muMap, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    constLiftN_natElem]

theorem muRec_rec (x y : N.Element) :
    (muRec f).toElementMap (pair x y) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap (f.toElementMap (pair x y)))
          (pair x ((muRec f).toElementMap (pair (succMap.toElementMap x) y)))) := by
  have hfix : (muOp f).toElementMap (muOp f).fixElement = (muOp f).fixElement :=
    toElementMap_fixElement (muOp f)
  have hval := toElementMap_curry_apply (muBody f) (muOp f).fixElement (pair x y)
  rw [← muOp, hfix] at hval
```

```
  rw [muRec, hval, muBody_apply]

theorem muRec_bot {n m : ℕ}
    (h : f.toElementMap (pair (natElem n) (natElem m)) = N.bot) :
    (muRec f).toElementMap (pair (natElem n) (natElem m)) = N.bot := by
  rw [muRec_rec, h, zeroMap_bot, T_bot_eq, Cond.cond_bot]

theorem muRec_found {n m : ℕ}
    (h : f.toElementMap (pair (natElem n) (natElem m)) = natElem 0) :
    (muRec f).toElementMap (pair (natElem n) (natElem m)) = natElem n := by
  rw [muRec_rec, h, zeroMap_natElem_zero, Cond.cond_true]

theorem muRec_step {n m j : ℕ}
    (h : f.toElementMap (pair (natElem n) (natElem m)) = natElem (j + 1)) :
    (muRec f).toElementMap (pair (natElem n) (natElem m)) =
      (muRec f).toElementMap (pair (natElem (n + 1)) (natElem m)) := by
  rw [muRec_rec, h, zeroMap_natElem_succ, Cond.cond_false, succMap_natElem]

theorem muRec_climb (m : ℕ) :
    ∀ (k n : ℕ), (∀ i < k, ∃ j, f.toElementMap (pair (natElem (n + i)) (natElem m)) =
    natElem (j + 1)) →
      (muRec f).toElementMap (pair (natElem n) (natElem m)) =
        (muRec f).toElementMap (pair (natElem (n + k)) (natElem m)) := by
  intro k
  induction k with
  | zero => intro n _; rw [Nat.add_zero]
  | succ k ih =>
    intro n h
    obtain ⟨j, hj⟩ := h 0 (Nat.succ_pos k)
    rw [Nat.add_zero] at hj
    have key := ih (n + 1) (fun i hi => by
      obtain ⟨j', hj'⟩ := h (i + 1) (by omega)
      exact ⟨j', by rw [show n + 1 + i = n + (i + 1) by omega]; exact hj'⟩)
    rw [muRec_step f hj, key, show n + 1 + k = n + (k + 1) by omega]

theorem muMap_eq_least {m n₀ : ℕ}
    (hzero : f.toElementMap (pair (natElem n₀) (natElem m)) = natElem 0)
    (hpos : ∀ i < n₀, ∃ j, f.toElementMap (pair (natElem i) (natElem m)) = natElem (j +
    1)) :
    (muMap f).toElementMap (natElem m) = natElem n₀ := by
  rw [muMap_natElem]
  have hclimb := muRec_climb f m n₀ 0 (by simpa using hpos)
  rw [show (zeroElt : N.Element) = natElem 0 from rfl, hclimb, Nat.zero_add]
  exact muRec_found f hzero

end Mu

section PrimRec
variable (f : ApproximableMap N N) (g : ApproximableMap (prod N (prod N N)) N)

def primBody : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) N :=
  (cond N).comp
    (paired (zeroMap.comp argX)
      (paired (f.comp argY)
        (g.comp
          (paired (predMap.comp argX)
```

```
            (paired argY
              ((evalMap (prod N N) N).comp
                (paired kSel (paired (predMap.comp argX) argY))))))))

def primOp : ApproximableMap (funSpace (prod N N) N) (funSpace (prod N N) N) :=
  curry (primBody f g)

def primRec : ApproximableMap (prod N N) N := toApproxMap (primOp f g).fixElement

theorem primBody_apply (k : (funSpace (prod N N) N).Element) (x y : N.Element) :
    (primBody f g).toElementMap (pair k (pair x y)) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap x)
          (pair (f.toElementMap y)
            (g.toElementMap
              (pair (predMap.toElementMap x)
                (pair y ((toApproxMap k).toElementMap
                  (pair (predMap.toElementMap x) y))))))) := by
  rw [primBody, toElementMap_comp]
  simp only [argX, argY, kSel, qSel, toElementMap_paired, toElementMap_comp,
    toElementMap_proj₀,
    toElementMap_proj₁, fst_pair, snd_pair, evalMap_apply]

theorem primRec_rec (x y : N.Element) :
    (primRec f g).toElementMap (pair x y) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap x)
          (pair (f.toElementMap y)
            (g.toElementMap
              (pair (predMap.toElementMap x)
                (pair y ((primRec f g).toElementMap
                  (pair (predMap.toElementMap x) y))))))) := by
  have hfix : (primOp f g).toElementMap (primOp f g).fixElement = (primOp f g).
    fixElement :=
    toElementMap_fixElement (primOp f g)
  have hval := toElementMap_curry_apply (primBody f g) (primOp f g).fixElement (pair x
     y)
  rw [← primOp, hfix] at hval
  rw [primRec, hval, primBody_apply]

theorem primRec_bot (m : ℕ) :
    (primRec f g).toElementMap (pair N.bot (natElem m)) = N.bot := by
  rw [primRec_rec, zeroMap_bot, T_bot_eq, Cond.cond_bot]

theorem primRec_succ (n m : ℕ) :
    (primRec f g).toElementMap (pair (natElem (n + 1)) (natElem m)) =
      g.toElementMap
        (pair (natElem n)
          (pair (natElem m) ((primRec f g).toElementMap (pair (natElem n) (natElem m))
    ))) := by
  rw [primRec_rec, zeroMap_natElem_succ, Cond.cond_false, predMap_natElem_succ]

theorem primRec_zero (m : ℕ) :
    (primRec f g).toElementMap (pair (natElem 0) (natElem m)) = f.toElementMap (
    natElem m) := by
  rw [primRec_rec, zeroMap_natElem_zero, Cond.cond_true]
```

```
end PrimRec

theorem strictId_bot : strictId.toElementMap N.bot = N.bot := by
  rw [strictId, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_idMap]
  rw [zeroMap_bot, T_bot_eq, Cond.cond_bot]

theorem strictId_natElem (n : ℕ) : strictId.toElementMap (natElem n) = natElem n := by
  rw [strictId, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_idMap]
  cases n with
  | zero => rw [zeroMap_natElem_zero, Cond.cond_true]
  | succ k => rw [zeroMap_natElem_succ, Cond.cond_false]

theorem strictProj₀_apply (x y : N.Element) :
    strictProj₀.toElementMap (pair x y) =
      (cond N).toElementMap (pair (zeroMap.toElementMap y) (pair x x)) := by
  rw [strictProj₀, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj₀,
    toElementMap_proj₁,
    fst_pair, snd_pair]

theorem strictProj₀_bot_left (m : ℕ) :
    strictProj₀.toElementMap (pair N.bot (natElem m)) = N.bot := by
  rw [strictProj₀_apply]
  cases m with
  | zero => rw [zeroMap_natElem_zero, Cond.cond_true]
  | succ k => rw [zeroMap_natElem_succ, Cond.cond_false]

theorem strictProj₀_bot_right (n : ℕ) :
    strictProj₀.toElementMap (pair (natElem n) N.bot) = N.bot := by
  rw [strictProj₀_apply, zeroMap_bot, T_bot_eq, Cond.cond_bot]

theorem strictProj₀_natElem (n m : ℕ) :
    strictProj₀.toElementMap (pair (natElem n) (natElem m)) = natElem n := by
  rw [strictProj₀_apply]
  cases m with
  | zero => rw [zeroMap_natElem_zero, Cond.cond_true]
  | succ k => rw [zeroMap_natElem_succ, Cond.cond_false]

end DomainTheory.NeighbourhoodSystems.NumRecSchemes
```

4.6.11 Infinite-product stack combinators

Stack combinators on the infinite product: fold, unfold, push, head, and tail, with the usual head/tail/push laws.

Definition (`fold`). Folding a pair consisting of an element and a sequence into a single sequence is the inverse of the product–sequence isomorphism.

Proof. By definition as the inverse isomorphism. ■

Theorem (`iterProdIso_apply`). The product–sequence isomorphism sends a sequence to the pair of its zeroth component with the sequence of remaining components.

Proof. By the definition of the isomorphism. ■

Definition (`push`). The push combinator places an element at the head of a sequence; it is the same map as fold.

Proof. By definition. ■

Definition (`unfold`). Unfolding a sequence recovers the pair of its head with its tail sequence, via the product–sequence isomorphism.

Proof. By definition as the forward isomorphism. ■

Definition (`head`). The head of a sequence is the first factor of its unfolding.

Proof. Compose the first projection with unfold. ■

Theorem (`head_apply`). Evaluating head at a sequence yields the zeroth component.

Proof. Unpack the composition and the isomorphism, then take the first factor of the resulting pair. ■

Definition (`tail`). The tail of a sequence is the second factor of its unfolding.

Proof. Compose the second projection with unfold. ■

Theorem (`tail_apply`). Evaluating tail at a sequence yields the sequence of components shifted by one.

Proof. Unpack the composition and the isomorphism, then take the second factor of the resulting pair. ■

Theorem (`component_tail`). The n-th component of a tail is the $(n{+}1)$-st component of the original sequence.

Proof. Apply the component formula for sequences built from a function of indices. ■

Theorem (`fold_unfold`). Folding the unfolding of a sequence recovers the sequence.

Proof. The two directions of the product–sequence isomorphism cancel. ■

Theorem (`unfold_fold`). Unfolding the folding of a pair recovers the pair.

Proof. The two directions of the product–sequence isomorphism cancel. ■

Theorem (`push_head_tail`). Pushing the head and tail of a sequence recovers the sequence.

Proof. Reduce push to fold, reassemble the pair of projections from the unfolding, and apply fold–unfold. ■

Theorem (`head_push`). The head of a pushed pair is the pushed element.

Proof. Reduce to unfold–fold and the first projection of a pair. ■

Theorem (`tail_push`). The tail of a pushed pair is the pushed sequence.

Proof. Reduce to unfold–fold and the second projection of a pair. ■

`iterStack.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofIso
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.iterSys

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {α : Type*} (V : NeighborhoodSystem α)
```

```
def fold : ApproximableMap (prod V (iterSys V)) (iterSys V) := ofIso (iterProdIso V).
    symm

theorem iterProdIso_apply (z : (iterSys V).Element) :
    iterProdIso V z = pair (component z 0) (ofSeq (fun n => component z (n + 1))) :=
    rfl

def push : ApproximableMap (prod V (iterSys V)) (iterSys V) := fold V

def unfold : ApproximableMap (iterSys V) (prod V (iterSys V)) := ofIso (iterProdIso V)

def head : ApproximableMap (iterSys V) V := (proj₀ V (iterSys V)).comp (unfold V)

@[simp] theorem head_apply (z : (iterSys V).Element) :
    (head V).toElementMap z = component z 0 := by
  rw [head, toElementMap_comp, unfold, toElementMap_ofIso, toElementMap_proj₀,
    iterProdIso_apply, fst_pair]

def tail : ApproximableMap (iterSys V) (iterSys V) := (proj₁ V (iterSys V)).comp (
    unfold V)

@[simp] theorem tail_apply (z : (iterSys V).Element) :
    (tail V).toElementMap z = ofSeq (fun n => component z (n + 1)) := by
  rw [tail, toElementMap_comp, unfold, toElementMap_ofIso, toElementMap_proj₁,
    iterProdIso_apply, snd_pair]

theorem component_tail (z : (iterSys V).Element) (n : ℕ) :
    component ((tail V).toElementMap z) n = component z (n + 1) := by
  rw [tail_apply, component_ofSeq]

theorem fold_unfold (z : (iterSys V).Element) :
    (fold V).toElementMap ((unfold V).toElementMap z) = z := by
  rw [unfold, fold, toElementMap_ofIso, toElementMap_ofIso, OrderIso.symm_apply_apply]

theorem unfold_fold (p : (prod V (iterSys V)).Element) :
    (unfold V).toElementMap ((fold V).toElementMap p) = p := by
  rw [unfold, fold, toElementMap_ofIso, toElementMap_ofIso, OrderIso.apply_symm_apply]

theorem push_head_tail (z : (iterSys V).Element) :
    (push V).toElementMap (pair ((head V).toElementMap z) ((tail V).toElementMap z)) =
     z := by
  rw [head, tail, toElementMap_comp, toElementMap_comp, toElementMap_proj₀,
    toElementMap_proj₁,
    pair_fst_snd, push, fold_unfold]

theorem head_push (x : V.Element) (s : (iterSys V).Element) :
    (head V).toElementMap ((push V).toElementMap (pair x s)) = x := by
  rw [head, toElementMap_comp, push, unfold_fold, toElementMap_proj₀, fst_pair]

theorem tail_push (x : V.Element) (s : (iterSys V).Element) :
    (tail V).toElementMap ((push V).toElementMap (pair x s)) = s := by
  rw [tail, toElementMap_comp, push, unfold_fold, toElementMap_proj₁, snd_pair]

end
end DomainTheory.NeighbourhoodSystems
```

4.6.12 Number-stack argument encoding

Number-stack argument encoding: finite (partial) vectors as sequences over the number domain, together with the λ-definability predicate and the guard/tuple scaffolding used to realize base and composition schemes.

Definition (`optElem`). An optional natural number is interpreted as bottom when absent and as the corresponding number-domain atom when present.

Proof. By definition. ■

Definition (`[U+1D4A9]`). The number-stack is the infinite product of the number domain with itself.

Proof. By definition as the iterated system on the number domain. ■

Definition (`ArgLike`). A sequence is argument-like when every component is either bottom or a number-domain atom.

Proof. By definition. ■

Definition (`argElem`). A finite partial vector is encoded by placing optional atoms in the initial components and bottom thereafter.

Proof. By definition via the sequence constructor. ■

Definition (`vecElem`). A total finite vector is the argument encoding of its pointwise-defined optional values.

Proof. By definition. ■

Definition (`LamDef`). An approximable map λ-defines a partial function when it is strict on argument-like ideals, agrees on defined values, and returns bottom on undefined ones.

Proof. By the structure fields. ■

Theorem (`component_argElem`). The stated property of `component_argElem`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`component_vecElem`). The stated property of `component_vecElem`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Definition (`guard1`). A one-place guard returns bottom when the chosen component is bottom, and otherwise agrees with the guarded map.

Proof. By definition via the conditional on the zero-test of that component. ■

Definition (`oneArg`). A single number is pushed onto the empty stack to form a one-argument sequence.

Proof. Compose push with the pairing of the identity and the constant empty stack. ■

Definition (`strictGuardN`). An n-place strict guard nests one-place guards on the first n components.

Proof. By recursion on n. ■

Definition (`tupleMap`). A tuple of maps into the number domain is assembled by successively pushing their values onto the stack.

Proof. By recursion on the tuple length. ■

Theorem (`T_bot_eq`). The stated property of `T_bot_eq`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`argElem_argLike`). The stated property of `argElem_argLike`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`component_argElem_none`). The stated property of `component_argElem_none`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`component_bot`). The stated property of `component_bot`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`component_push_succ`). The stated property of `component_push_succ`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`component_push_zero`). The stated property of `component_push_zero`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`component_vecElem_lt`). The stated property of `component_vecElem_lt`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`eq_of_component_eq`). The stated property of `eq_of_component_eq`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`guard1_apply`). The stated property of `guard1_apply`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`guard1_bot`). The stated property of `guard1_bot`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`guard1_inner_bot`). The stated property of `guard1_inner_bot`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`guard1_total`). The stated property of `guard1_total`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`lamDef_coe_val`). The stated property of `lamDef_coe_val`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`lamDef_congr`). The stated property of `lamDef_congr`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`mem_mOfFn`). The stated property of `mem_mOfFn`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`oneArg_bot`). The stated property of `oneArg_bot`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`push_natElem_vecElem`). The stated property of `push_natElem_vecElem`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`oneArg_natElem`). The stated property of `oneArg_natElem`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`realizer_value`). The stated property of `realizer_value`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`strictGuardN_strict`). The stated property of `strictGuardN_strict`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`strictGuardN_total`). The stated property of `strictGuardN_total`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`tupleMap_succ`). The stated property of `tupleMap_succ`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`component_tupleMap`). The stated property of `component_tupleMap`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`tupleMap_zero`). The stated property of `tupleMap_zero`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`component_tupleMap_ge`). The stated property of `component_tupleMap_ge`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`tupleMap_argLike`). The stated property of `tupleMap_argLike`.
Proof. Follows from the surrounding construct and fixed-point laws. ■
Theorem (`tupleMap_vecElem_eq`). The stated property of `tupleMap_vecElem_eq`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`vecElem_totals`). The stated property of `vecElem_totals`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

lamArg.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.iterStack
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.numberDomain
public import Mathlib.Computability.PartrecBasis
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.cond
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.PartrecLam
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.NatDom (N natElem zeroElt succMap predMap
    zeroMap
  succMap_natElem succMap_bot predMap_natElem_succ predMap_natElem_zero predMap_bot
  zeroMap_natElem_zero zeroMap_natElem_succ zeroMap_bot constLiftN)
open DomainTheory.NeighbourhoodSystems.Cond (cond)

def optElem : Option ℕ → N.Element
  | none => N.bot
  | some n => natElem n

abbrev ? : NeighborhoodSystem (ℕ × ℕ) := iterSys N

def ArgLike (z : ?.Element) : Prop := ∀ j, component z j = N.bot ∨ ∃ k, component z j
    = natElem k

def argElem {n : ℕ} (a : Fin n → Option ℕ) : ?.Element :=
  ofSeq (fun i => if h : i < n then optElem (a ⟨i, h⟩) else N.bot)

def vecElem {n : ℕ} (v : List.Vector ℕ n) : ?.Element :=
  argElem (fun i => some (v.get i))

structure LamDef {n : ℕ} (φ : ApproximableMap ? N) (f : List.Vector ℕ n →. ℕ) : Prop
    where
  defined : ∀ (v : List.Vector ℕ n) (c : ℕ), c ∈ f v → φ.toElementMap (vecElem v) =
    natElem c
  undef : ∀ (v : List.Vector ℕ n), ¬ (f v).Dom → φ.toElementMap (vecElem v) = N.bot
  strict : ∀ (z : ?.Element), ArgLike z → ∀ i, i < n → component z i = N.bot →
    φ.toElementMap z = N.bot

@[simp] theorem component_argElem {n : ℕ} (a : Fin n → Option ℕ) (i : ℕ) :
    component (argElem a) i = if h : i < n then optElem (a ⟨i, h⟩) else N.bot := by
  rw [argElem, component_ofSeq]
```

```
@[simp] theorem component_vecElem {n : ℕ} (v : List.Vector ℕ n) (i : ℕ) :
    component (vecElem v) i = if h : i < n then natElem (v.get ⟨i, h⟩) else N.bot := by
  rw [vecElem, component_argElem]; split <;> rfl

def guard1 (j : ℕ) (φ : ApproximableMap ? N) : ApproximableMap ? N :=
  (cond N).comp (paired (zeroMap.comp (projN N j)) (paired φ φ))

def oneArg : ApproximableMap N ? := (push N).comp (paired (idMap N) (constMap N ?.bot)
    )

def strictGuardN : ℕ → ApproximableMap ? N → ApproximableMap ? N
  | 0, φ => φ
  | n + 1, φ => guard1 n (strictGuardN n φ)

def tupleMap : (n : ℕ) → (Fin n → ApproximableMap ? N) → ApproximableMap ? ?
  | 0, _ => constMap ? ?.bot
  | n + 1, gs => (push N).comp (paired (gs 0) (tupleMap n (fun i => gs i.succ)))

theorem T_bot_eq : (NatDom.T).bot = Truth.botElt :=
  le_antisymm ((NatDom.T).bot_le _) (Truth.botElt_le _)

theorem argElem_argLike {n : ℕ} (a : Fin n → Option ℕ) : ArgLike (argElem a) := by
  intro j
  rw [component_argElem]
  split
  · cases h : a _ with
    | none => exact Or.inl rfl
    | some k => exact Or.inr ⟨k, rfl⟩
  · exact Or.inl rfl

theorem component_argElem_none {n : ℕ} (a : Fin n → Option ℕ) (i : Fin n) (hi : a i =
     none) :
    component (argElem a) (i : ℕ) = N.bot := by
  rw [component_argElem, dite_eq_left i.isLt, Fin.eta, hi]; rfl

theorem component_bot (i : ℕ) : component (?.bot) i = N.bot := by
  have hbot : (?.bot) = ofSeq (fun _ => N.bot) :=
    le_antisymm (?.bot_le _)
      (le_of_component_le (fun i => by rw [component_ofSeq]; exact N.bot_le _))
  rw [hbot, component_ofSeq]

theorem component_push_succ (x : N.Element) (s : ?.Element) (j : ℕ) :
    component ((push N).toElementMap (pair x s)) (j + 1) = component s j := by
  rw [← component_tail, tail_push]

theorem component_push_zero (x : N.Element) (s : ?.Element) :
    component ((push N).toElementMap (pair x s)) 0 = x := by
  rw [← head_apply, head_push]

theorem component_vecElem_lt {n : ℕ} (v : List.Vector ℕ n) {j : ℕ} (hj : j < n) :
    component (vecElem v) j = natElem (v.get ⟨j, hj⟩) := by
  rw [component_vecElem, dite_eq_left hj]

theorem eq_of_component_eq {z z' : ?.Element}
    (h : ∀ n, component z n = component z' n) : z = z' :=
  le_antisymm (le_of_component_le (fun n => (h n).le))
```

```
    (le_of_component_le (fun n => (h n).ge))

theorem guard1_apply (j : ℕ) (φ : ApproximableMap ? N) (z : ?.Element) :
    (guard1 j φ).toElementMap z =
      (cond N).toElementMap (pair (zeroMap.toElementMap (component z j))
        (pair (φ.toElementMap z) (φ.toElementMap z))) := by
  rw [guard1, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_projN]

theorem guard1_bot (j : ℕ) (φ : ApproximableMap ? N) (z : ?.Element)
    (h : component z j = N.bot) : (guard1 j φ).toElementMap z = N.bot := by
  rw [guard1_apply, h, zeroMap_bot, T_bot_eq, Cond.cond_bot]

theorem guard1_inner_bot (j : ℕ) (φ : ApproximableMap ? N) (z : ?.Element)
    (hz : component z j = N.bot ∨ ∃ k, component z j = natElem k)
    (h : φ.toElementMap z = N.bot) : (guard1 j φ).toElementMap z = N.bot := by
  rcases hz with hb | ⟨k, hk⟩
  · exact guard1_bot j φ z hb
  · rw [guard1_apply, hk, h]
    cases k with
    | zero => rw [zeroMap_natElem_zero, Cond.cond_true]
    | succ m => rw [zeroMap_natElem_succ, Cond.cond_false]

theorem guard1_total (j : ℕ) (φ : ApproximableMap ? N) (z : ?.Element) {k : ℕ}
    (h : component z j = natElem k) : (guard1 j φ).toElementMap z = φ.toElementMap z
    := by
  rw [guard1_apply, h]
  cases k with
  | zero => rw [zeroMap_natElem_zero, Cond.cond_true]
  | succ m => rw [zeroMap_natElem_succ, Cond.cond_false]

theorem lamDef_coe_val {n : ℕ} {φ : ApproximableMap ? N} {g : List.Vector ℕ n → ℕ}
    (h : LamDef φ (g : List.Vector ℕ n →. ℕ)) (v : List.Vector ℕ n) :
    φ.toElementMap (vecElem v) = natElem (g v) :=
  h.defined v (g v) (by rw [PFun.coe_val]; exact Part.mem_some _)

theorem lamDef_congr {n : ℕ} {φ : ApproximableMap ? N} {f f' : List.Vector ℕ n →. ℕ}
    (h : ∀ v, f v = f' v) (hf : LamDef φ f) : LamDef φ f' where
  defined v c hc := hf.defined v c (by rw [h v]; exact hc)
  undef v hv := hf.undef v (by rw [h v]; exact hv)
  strict := hf.strict

theorem mem_mOfFn : ∀ {n : ℕ} (p : Fin n → Part ℕ) (w : List.Vector ℕ n),
    w ∈ List.Vector.mOfFn p ↔ ∀ i, w.get i ∈ p i
  | 0, p, w => by
      constructor
      · intro _ i; exact i.elim0
      · intro _
        show w ∈ (pure List.Vector.nil : Part _)
        rw [Part.pure_eq_some, Part.mem_some_iff]
        exact List.Vector.eq_nil w
  | n + 1, p, w => by
      have hmof : List.Vector.mOfFn p
          = (p 0).bind (fun a => (List.Vector.mOfFn fun i => p i.succ).bind
              (fun u => Part.some (a ::? u))) := by
        simp only [List.Vector.mOfFn, Part.bind_eq_bind, Part.pure_eq_some]
```

```
      rw [hmof, Part.mem_bind_iff]
      constructor
      · rintro ⟨a, ha, hrest⟩
        rw [Part.mem_bind_iff] at hrest
        obtain ⟨u, hu, hw⟩ := hrest
        rw [Part.mem_some_iff] at hw
        subst hw
        have hu' := (mem_mOfFn (fun i => p i.succ) u).mp hu
        intro i
        refine Fin.cases ?_ ?_ i
        · simpa using ha
        · intro k; simpa using hu' k
      · intro hall
        refine ⟨w.head, ?_, ?_⟩
        · have := hall 0; rwa [List.Vector.get_zero] at this
        · rw [Part.mem_bind_iff]
          refine ⟨w.tail, (mem_mOfFn (fun i => p i.succ) w.tail).mpr (fun k => ?_), ?_⟩
          · have h := hall k.succ
            rwa [← List.Vector.get_tail_succ w k] at h
          · rw [Part.mem_some_iff]; exact (List.Vector.cons_head_tail w).symm

theorem oneArg_bot : (oneArg).toElementMap N.bot = ?.bot := by
  rw [oneArg, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap]
  apply eq_of_component_eq
  intro i
  cases i with
  | zero => rw [component_push_zero, component_bot]
  | succ k => rw [component_push_succ, component_bot, component_bot]

theorem push_natElem_vecElem {n : ℕ} (a : ℕ) (u : List.Vector ℕ n) :
    (push N).toElementMap (pair (natElem a) (vecElem u)) = vecElem (a ::? u) := by
  apply eq_of_component_eq
  intro j
  cases j with
  | zero =>
    rw [component_push_zero, component_vecElem, dite_eq_left (Nat.succ_pos n)]
    exact congrArg natElem (List.Vector.get_cons_zero a u).symm
  | succ k =>
    rw [component_push_succ, component_vecElem, component_vecElem]
    by_cases hk : k < n
    · rw [dite_eq_left hk, dite_eq_left (show k + 1 < n + 1 by omega)]
      exact congrArg natElem (List.Vector.get_cons_succ a u ⟨k, hk⟩).symm
    · rw [dite_eq_right hk, dite_eq_right (show ¬ k + 1 < n + 1 by omega)]

theorem oneArg_natElem (a : ℕ) :
    (oneArg).toElementMap (natElem a) = vecElem (a ::? (List.Vector.nil : List.Vector
    ℕ 0)) := by
  rw [oneArg, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap]
  rw [show ?.bot = vecElem (List.Vector.nil : List.Vector ℕ 0) from ?_,
    push_natElem_vecElem]
  exact eq_of_component_eq (fun i => by rw [component_bot, component_vecElem,
    dite_eq_right (by omega)])

theorem realizer_value {n : ℕ} {φ : ApproximableMap ? N} {f : List.Vector ℕ n →. ℕ}
```

```
    (h : LamDef φ f) (v : List.Vector ℕ n) :
    φ.toElementMap (vecElem v) = N.bot ∨ ∃ k, φ.toElementMap (vecElem v) = natElem k
    := by
  by_cases hd : (f v).Dom
  · exact Or.inr ⟨(f v).get hd, h.defined v _ (Part.get_mem hd)⟩
  · exact Or.inl (h.undef v hd)

theorem strictGuardN_strict (n : ℕ) (φ : ApproximableMap ? N) (z : ?.Element)
    (hz : ArgLike z) {i : ℕ} (hi : i < n) (hb : component z i = N.bot) :
    (strictGuardN n φ).toElementMap z = N.bot := by
  induction n with
  | zero => exact absurd hi (Nat.not_lt_zero i)
  | succ m ih =>
    rw [strictGuardN]
    rcases Nat.lt_succ_iff_lt_or_eq.mp hi with hlt | heq
    · exact guard1_inner_bot m _ z (hz m) (ih hlt)
    · exact guard1_bot m _ z (heq ▸ hb)

theorem strictGuardN_total (n : ℕ) (φ : ApproximableMap ? N) (z : ?.Element)
    (h : ∀ j, j < n → ∃ k, component z j = natElem k) :
    (strictGuardN n φ).toElementMap z = φ.toElementMap z := by
  induction n with
  | zero => rfl
  | succ m ih =>
    obtain ⟨k, hk⟩ := h m (Nat.lt_succ_self m)
    rw [strictGuardN, guard1_total m _ z hk]
    exact ih (fun j hj => h j (Nat.lt_succ_of_lt hj))

theorem tupleMap_succ (n : ℕ) (gs : Fin (n + 1) → ApproximableMap ? N) :
    tupleMap (n + 1) gs = (push N).comp (paired (gs 0) (tupleMap n (fun i => gs i.succ
    ))) := rfl

theorem component_tupleMap : ∀ (n : ℕ) (gs : Fin n → ApproximableMap ? N) (z : ?.
    Element)
    (i : ℕ) (hi : i < n),
    component ((tupleMap n gs).toElementMap z) i = (gs ⟨i, hi⟩).toElementMap z
  | 0, _, _, i, hi => absurd hi (Nat.not_lt_zero i)
  | n + 1, gs, z, 0, hi => by
      rw [tupleMap_succ, toElementMap_comp, toElementMap_paired, component_push_zero]
      exact congrArg (fun w => (gs w).toElementMap z) (Fin.ext (by simp))
  | n + 1, gs, z, i + 1, hi => by
      rw [tupleMap_succ, toElementMap_comp, toElementMap_paired, component_push_succ,
        component_tupleMap n (fun j => gs j.succ) z i (Nat.lt_of_succ_lt_succ hi)]
      rfl

theorem tupleMap_zero (gs : Fin 0 → ApproximableMap ? N) :
    tupleMap 0 gs = constMap ? ?.bot := rfl

theorem component_tupleMap_ge : ∀ (n : ℕ) (gs : Fin n → ApproximableMap ? N) (z : ?.
    Element)
    (i : ℕ), n ≤ i → component ((tupleMap n gs).toElementMap z) i = N.bot
  | 0, gs, z, i, _ => by rw [tupleMap_zero, toElementMap_constMap, component_bot]
  | n + 1, gs, z, i, hi => by
      obtain ⟨j, rfl⟩ : ∃ j, i = j + 1 := ⟨i - 1, by omega⟩
      rw [tupleMap_succ, toElementMap_comp, toElementMap_paired, component_push_succ]
      exact component_tupleMap_ge n (fun k => gs k.succ) z j (by omega)
```

```
theorem tupleMap_argLike {m nn : ℕ} (φg : Fin nn → ApproximableMap ? N)
    (g : Fin nn → List.Vector ℕ m →. ℕ) (hg : ∀ i, LamDef (φg i) (g i)) (v : List.
    Vector ℕ m) :
    ArgLike ((tupleMap nn φg).toElementMap (vecElem v)) := by
  intro i
  by_cases hi : i < nn
  · rw [component_tupleMap nn φg (vecElem v) i hi]; exact realizer_value (hg ⟨i, hi⟩) v
  · rw [component_tupleMap_ge nn φg (vecElem v) i (Nat.le_of_not_lt hi)]; exact Or.inl
    rfl

theorem tupleMap_vecElem_eq {m nn : ℕ} (φg : Fin nn → ApproximableMap ? N)
    (g : Fin nn → List.Vector ℕ m →. ℕ) (hg : ∀ i, LamDef (φg i) (g i))
    (v : List.Vector ℕ m) (w : List.Vector ℕ nn) (hw : ∀ i, w.get i ∈ g i v) :
    (tupleMap nn φg).toElementMap (vecElem v) = vecElem w := by
  apply eq_of_component_eq
  intro i
  by_cases hi : i < nn
  · rw [component_tupleMap nn φg (vecElem v) i hi, component_vecElem_lt w hi]
    exact (hg ⟨i, hi⟩).defined v (w.get ⟨i, hi⟩) (hw ⟨i, hi⟩)
  · rw [component_tupleMap_ge nn φg (vecElem v) i (Nat.le_of_not_lt hi),
    component_vecElem,
      dite_eq_right hi]

theorem vecElem_totals {n : ℕ} (v : List.Vector ℕ n) :
    ∀ j, j < n → ∃ k, component (vecElem v) j = natElem k :=
  fun j hj => ⟨v.get ⟨j, hj⟩, component_vecElem_lt v hj⟩

end DomainTheory.NeighbourhoodSystems.PartrecLam
```

4.6.13 Partial-recursive scheme realizers

Least-fixed-point realizers on the number-stack for unbounded search and primitive recursion, with their recursion and step lemmas.

Definition (`evalAt`). Evaluation at a fixed stack argument is evaluation of the function-space map at that constant.

Proof. Pair the identity on the function space with the constant argument and compose with evaluation. ■

Definition (`findBody`). The search body returns the head when the predicate vanishes, and otherwise continues at the successor head on the tail.

Proof. By definition via the conditional on the zero-test of the predicate at the current stack. ■

Definition (`findOp`). The search operator is the curry of the search body.

Proof. By definition. ■

Definition (`precBody`). The primitive-recursion body branches on whether the head vanishes, applying the base map on the tail or the step map at the predecessor.

Proof. By definition via the conditional on the zero-test of the head. ■

Definition (`recOp`). The primitive-recursion operator is the curry of the primitive-recursion body.

Proof. By definition. ■

Definition (`recMap`). Primitive recursion is the least fixed point of the primitive-recursion operator.

Proof. By definition. ■

Definition (`rfindZeroPred`). The Mathlib zero-finding predicate for a total vector function is the decision of whether the value at a candidate is zero.

Proof. By definition. ■

Definition (`searchMap`). Unbounded search is the least fixed point of the search operator.

Proof. By definition. ■

Definition (`findMap`). Search from zero prepends a zero head and then applies unbounded search.

Proof. Compose search with the push of the zero constant. ■

Theorem (`evalAt_apply`). The stated property of `evalAt_apply`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`findBody_apply`). The stated property of `findBody_apply`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`findOp_iterElem_zero`). The stated property of `findOp_iterElem_zero`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`iterVal_succ`). The stated property of `iterVal_succ`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`precBody_apply`). The stated property of `precBody_apply`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`findMap_vecElem`). The stated property of `findMap_vecElem`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`recMap_rec`). The stated property of `recMap_rec`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`recMap_eval`). The stated property of `recMap_eval`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`searchMap_rec`). The stated property of `searchMap_rec`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`searchMap_step_found`). The stated property of `searchMap_step_found`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`searchMap_step_next`). The stated property of `searchMap_step_next`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`searchMap_climb`). The stated property of `searchMap_climb`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`toApproxMap_bot`). The stated property of `toApproxMap_bot`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`iterVal_zero`). The stated property of `iterVal_zero`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`iterVal_bot`). The stated property of `iterVal_bot`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`searchMap_diverge`). The stated property of `searchMap_diverge`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

partrecSchemes.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.lamArg
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.PartrecLam
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.NatDom (N natElem zeroElt succMap predMap
    zeroMap
  succMap_natElem succMap_bot predMap_natElem_succ predMap_natElem_zero predMap_bot
  zeroMap_natElem_zero zeroMap_natElem_succ zeroMap_bot constLiftN)
open DomainTheory.NeighbourhoodSystems.Cond (cond)

def evalAt (z : ?.Element) : ApproximableMap (funSpace ? N) N :=
  (evalMap ? N).comp (paired (idMap (funSpace ? N)) (constMap (funSpace ? N) z))

def rfindZeroPred {n : ℕ} (f : List.Vector ℕ (n + 1) → ℕ) (v : List.Vector ℕ n) : ℕ
    →. Bool :=
  fun k => Part.some (decide (f (k ::? v) = 0))

theorem evalAt_apply (z : ?.Element) (φ : (funSpace ? N).Element) :
    (evalAt z).toElementMap φ = (toApproxMap φ).toElementMap z := by
  rw [evalAt, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap,
    evalMap_apply]

theorem toApproxMap_bot (z : ?.Element) :
    (toApproxMap ((funSpace ? N).bot)).toElementMap z = N.bot := by
  refine le_antisymm ?_ (N.bot_le _)
  have h2 := (funSpaceEquiv ? N).monotone
    ((funSpace ? N).bot_le ((funSpaceEquiv ? N).symm (constMap ? N.bot)))
  rw [OrderIso.apply_symm_apply, funSpaceEquiv_apply] at h2
  have h3 := (le_iff_toElementMap_le.mp h2) z
  rwa [toElementMap_constMap] at h3

section Rfind
variable (φf : ApproximableMap ? N)

def findBody : ApproximableMap (prod (funSpace ? N) ?) N :=
  (cond N).comp
    (paired (zeroMap.comp (φf.comp (proj₁ (funSpace ? N) ?)))
      (paired ((head N).comp (proj₁ (funSpace ? N) ?))
        ((evalMap ? N).comp
          (paired (proj₀ (funSpace ? N) ?)
            ((push N).comp
              (paired (succMap.comp ((head N).comp (proj₁ (funSpace ? N) ?)))
                ((tail N).comp (proj₁ (funSpace ? N) ?)))))))))

def findOp : ApproximableMap (funSpace ? N) (funSpace ? N) := curry (findBody φf)

def searchMap : ApproximableMap ? N := toApproxMap (findOp φf).fixElement

def findMap : ApproximableMap ? N :=
  (searchMap φf).comp ((push N).comp (paired (constMap ? (natElem 0)) (idMap ?)))

theorem findBody_apply (k : (funSpace ? N).Element) (z : ?.Element) :
    (findBody φf).toElementMap (pair k z) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap (φf.toElementMap z))
```

```
          (pair ((head N).toElementMap z)
            ((toApproxMap k).toElementMap
              ((push N).toElementMap
                (pair (succMap.toElementMap ((head N).toElementMap z))
                  ((tail N).toElementMap z)))))) := by
  rw [findBody, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj₀,
    toElementMap_proj₁,
    fst_pair, snd_pair, evalMap_apply]

theorem findOp_iterElem_zero : (findOp φf).iterElem 0 = (funSpace ? N).bot := by
  rw [iterElem_eq_iterate, Function.iterate_zero_apply]

theorem iterVal_succ (m : ℕ) (z : ?.Element) :
    (toApproxMap ((findOp φf).iterElem (m + 1))).toElementMap z =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap (φf.toElementMap z))
          (pair ((head N).toElementMap z)
            ((toApproxMap ((findOp φf).iterElem m)).toElementMap
              ((push N).toElementMap
                (pair (succMap.toElementMap ((head N).toElementMap z))
                  ((tail N).toElementMap z)))))) := by
  have h1 : (findOp φf).iterElem (m + 1) = (findOp φf).toElementMap ((findOp φf).
    iterElem m) := by
    rw [iterElem_eq_iterate, iterElem_eq_iterate, Function.iterate_succ', Function.
    comp_apply]
  rw [h1]
  have hval := toElementMap_curry_apply (findBody φf) ((findOp φf).iterElem m) z
  rw [← findOp] at hval
  rw [hval, findBody_apply]

theorem findMap_vecElem {n : ℕ} (v : List.Vector ℕ n) :
    (findMap φf).toElementMap (vecElem v) = (searchMap φf).toElementMap (vecElem (0
    ::? v)) := by
  rw [findMap, toElementMap_comp, toElementMap_comp, toElementMap_paired,
    toElementMap_constMap,
    toElementMap_idMap, push_natElem_vecElem]

theorem searchMap_rec (z : ?.Element) :
    (searchMap φf).toElementMap z =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap (φf.toElementMap z))
          (pair ((head N).toElementMap z)
            ((searchMap φf).toElementMap
              ((push N).toElementMap
                (pair (succMap.toElementMap ((head N).toElementMap z))
                  ((tail N).toElementMap z)))))) := by
  have hfix : (findOp φf).toElementMap (findOp φf).fixElement = (findOp φf).fixElement
    :=
    toElementMap_fixElement (findOp φf)
  have hval := toElementMap_curry_apply (findBody φf) (findOp φf).fixElement z
  rw [← findOp, hfix] at hval
  rw [searchMap, hval, findBody_apply]

theorem searchMap_step_found {n : ℕ} {f : List.Vector ℕ (n + 1) → ℕ}
    (hf : LamDef φf (f : List.Vector ℕ (n + 1) →. ℕ)) (v : List.Vector ℕ n) (k : ℕ)
```

```
    (h0 : f (k ::? v) = 0) :
    (searchMap φf).toElementMap (vecElem (k ::? v)) = natElem k := by
  rw [← push_natElem_vecElem, searchMap_rec, head_push, tail_push,
    push_natElem_vecElem, lamDef_coe_val hf (k ::? v), h0, zeroMap_natElem_zero,
    Cond.cond_true]

theorem searchMap_step_next {n : ℕ} {f : List.Vector ℕ (n + 1) → ℕ}
    (hf : LamDef φf (f : List.Vector ℕ (n + 1) →. ℕ)) (v : List.Vector ℕ n) (k : ℕ)
    (hne : f (k ::? v) = 0) :
    (searchMap φf).toElementMap (vecElem (k ::? v)) =
      (searchMap φf).toElementMap (vecElem ((k + 1) ::? v)) := by
  obtain ⟨m, hm⟩ : ∃ m, f (k ::? v) = m + 1 := ⟨f (k ::? v) - 1, by omega⟩
  rw [← push_natElem_vecElem, searchMap_rec, head_push, tail_push,
    push_natElem_vecElem, lamDef_coe_val hf (k ::? v), hm, zeroMap_natElem_succ,
    Cond.cond_false, succMap_natElem, push_natElem_vecElem]

theorem searchMap_climb {n : ℕ} {f : List.Vector ℕ (n + 1) → ℕ}
    (hf : LamDef φf (f : List.Vector ℕ (n + 1) →. ℕ)) (v : List.Vector ℕ n) (m : ℕ)
    (hm0 : f (m ::? v) = 0) (hmin : ∀ i, i < m → f (i ::? v) = 0) :
    ∀ d j, j + d = m → (searchMap φf).toElementMap (vecElem (j ::? v)) = natElem m :=
    by
  intro d
  induction d with
  | zero =>
    intro j hj
    have hjm : j = m := by omega
    rw [hjm]
    exact searchMap_step_found φf hf v m hm0
  | succ d ih =>
    intro j hj
    have hjm : j < m := by omega
    rw [searchMap_step_next φf hf v j (hmin j hjm)]
    exact ih (j + 1) (by omega)

theorem iterVal_zero (z : ?.Element) :
    (toApproxMap ((findOp φf).iterElem 0)).toElementMap z = N.bot := by
  rw [findOp_iterElem_zero, toApproxMap_bot]

theorem iterVal_bot {n : ℕ} {f : List.Vector ℕ (n + 1) → ℕ}
    (hf : LamDef φf (f : List.Vector ℕ (n + 1) →. ℕ)) (v : List.Vector ℕ n)
    (hdiv : ∀ k, f (k ::? v) = 0) :
    ∀ m k, (toApproxMap ((findOp φf).iterElem m)).toElementMap (vecElem (k ::? v)) = N
    .bot := by
  intro m
  induction m with
  | zero => intro k; exact iterVal_zero φf (vecElem (k ::? v))
  | succ m ih =>
    intro k
    have hne := hdiv k
    obtain ⟨j, hj⟩ : ∃ j, f (k ::? v) = j + 1 := ⟨f (k ::? v) - 1, by omega⟩
    rw [← push_natElem_vecElem, iterVal_succ, head_push, tail_push,
      push_natElem_vecElem, lamDef_coe_val hf (k ::? v), hj, zeroMap_natElem_succ,
      Cond.cond_false, succMap_natElem, push_natElem_vecElem]
    exact ih (k + 1)

theorem searchMap_diverge {n : ℕ} {f : List.Vector ℕ (n + 1) → ℕ}
```

```
    (hf : LamDef φf (f : List.Vector ℕ (n + 1) →. ℕ)) (v : List.Vector ℕ n)
    (hdiv : ∀ k, f (k ::? v) = 0) (k : ℕ) :
    (searchMap φf).toElementMap (vecElem (k ::? v)) = N.bot := by
  refine le_antisymm ?_ (N.bot_le _)
  rw [searchMap, ← evalAt_apply, fixElement_eq_iSupDirected, toElementMap_iSupDirected
    ]
  apply NeighborhoodSystem.iSupDirected_le
  intro m
  rw [evalAt_apply]
  exact le_of_eq (iterVal_bot φf hf v hdiv m k)

end Rfind

section Prec
variable (φf φg : ApproximableMap ? N)

def precBody : ApproximableMap (prod (funSpace ? N) ?) N :=
  (cond N).comp
    (paired (zeroMap.comp ((head N).comp (proj₁ (funSpace ? N) ?)))
      (paired (φf.comp ((tail N).comp (proj₁ (funSpace ? N) ?)))
        (φg.comp
          ((push N).comp
            (paired (predMap.comp ((head N).comp (proj₁ (funSpace ? N) ?)))
              ((push N).comp
                (paired
                  ((evalMap ? N).comp
                    (paired (proj₀ (funSpace ? N) ?)
                      ((push N).comp
                        (paired (predMap.comp ((head N).comp (proj₁ (funSpace ? N) ?)))
                          ((tail N).comp (proj₁ (funSpace ? N) ?))))))
                  ((tail N).comp (proj₁ (funSpace ? N) ?)))))))))

def recOp : ApproximableMap (funSpace ? N) (funSpace ? N) := curry (precBody φf φg)

def recMap : ApproximableMap ? N := toApproxMap (recOp φf φg).fixElement

theorem precBody_apply (k : (funSpace ? N).Element) (z : ?.Element) :
    (precBody φf φg).toElementMap (pair k z) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap ((head N).toElementMap z))
          (pair (φf.toElementMap ((tail N).toElementMap z))
            (φg.toElementMap
              ((push N).toElementMap
                (pair (predMap.toElementMap ((head N).toElementMap z))
                  ((push N).toElementMap
                    (pair ((toApproxMap k).toElementMap
                            ((push N).toElementMap
                              (pair (predMap.toElementMap ((head N).toElementMap z))
                                ((tail N).toElementMap z))))
                      ((tail N).toElementMap z)))))))) := by
  rw [precBody, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj₀,
    toElementMap_proj₁,
    fst_pair, snd_pair, evalMap_apply]

theorem recMap_rec (z : ?.Element) :
```

```
    (recMap φf φg).toElementMap z =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap ((head N).toElementMap z))
          (pair (φf.toElementMap ((tail N).toElementMap z))
            (φg.toElementMap
              ((push N).toElementMap
                (pair (predMap.toElementMap ((head N).toElementMap z))
                  ((push N).toElementMap
                    (pair ((recMap φf φg).toElementMap
                            ((push N).toElementMap
                              (pair (predMap.toElementMap ((head N).toElementMap z))
                                ((tail N).toElementMap z))))
                      ((tail N).toElementMap z))))))))) := by
  have hfix : (recOp φf φg).toElementMap (recOp φf φg).fixElement = (recOp φf φg).
    fixElement :=
    toElementMap_fixElement (recOp φf φg)
  have hval := toElementMap_curry_apply (precBody φf φg) (recOp φf φg).fixElement z
  rw [← recOp, hfix] at hval
  rw [recMap, hval, precBody_apply]

theorem recMap_eval {n : ℕ} (f₀ : List.Vector ℕ n → ℕ) (g₀ : List.Vector ℕ (n + 2) →
    ℕ)
    (hf : LamDef φf (f₀ : List.Vector ℕ n →. ℕ)) (hg : LamDef φg (g₀ : List.Vector ℕ
    (n + 2) →. ℕ))
    (u : List.Vector ℕ n) (d : ℕ) :
    (recMap φf φg).toElementMap (vecElem (d ::? u))
      = natElem (Nat.rec (motive := fun _ => ℕ) (f₀ u)
          (fun y IH => g₀ (y ::? IH ::? u)) d) := by
  induction d with
  | zero =>
    rw [← push_natElem_vecElem, recMap_rec, head_push, tail_push,
      zeroMap_natElem_zero, Cond.cond_true]
    exact lamDef_coe_val hf u
  | succ d ih =>
    rw [← push_natElem_vecElem, recMap_rec, head_push, tail_push,
      zeroMap_natElem_succ, Cond.cond_false, predMap_natElem_succ,
    push_natElem_vecElem, ih,
      push_natElem_vecElem, push_natElem_vecElem]
    exact lamDef_coe_val hg _

end Prec

end DomainTheory.NeighbourhoodSystems.PartrecLam
```

4.6.14 Partial recursive λ-definability

Closure of λ-definability under Mathlib's generators for primitive and partial recursive functions, culminating in the one-variable representation theorem.

Theorem (`lamDef_of_inner`). The stated property of `lamDef_of_inner`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`lamDef_comp`). The stated property of `lamDef_comp`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`lamDef_prec`). The stated property of `lamDef_prec`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`lamDef_primComp`). The stated property of `lamDef_primComp`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`lamDef_rfind`). The stated property of `lamDef_rfind`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`lamDef_total`). The stated property of `lamDef_total`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`lamDef_get`). The stated property of `lamDef_get`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`lamDef_succ`). The stated property of `lamDef_succ`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`lamDef_zero`). The stated property of `lamDef_zero`.

Proof. Follows from the surrounding construct and fixed-point laws. ■

Theorem (`primrec_lamDef`). Every Mathlib primitive-recursive vector function admits a `LamDef` witness.

Proof. Induct on the primitive-recursive generation. ■

Theorem (`partrec_lamDef`). Every Mathlib partial-recursive vector function admits a `LamDef` witness.

Proof. Induct on the partial-recursive generation, using the primitive, composition, and μ-find cases. ■

Theorem (`partrec_one`). Every unary partial recursive function is realized by a strict approximable endomap on the number domain.

Proof. Apply the general `LamDef` existence theorem and compose with the one-argument embedding. ■

`partrecLam.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.partrecSchemes
public import Mathlib.Computability.PartrecBasis

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.PartrecLam
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.NatDom (N natElem zeroElt succMap predMap
    zeroMap
  succMap_natElem succMap_bot predMap_natElem_succ predMap_natElem_zero predMap_bot
  zeroMap_natElem_zero zeroMap_natElem_succ zeroMap_bot constLiftN)
open DomainTheory.NeighbourhoodSystems.Cond (cond)

theorem lamDef_of_inner {n : ℕ} (inner : ApproximableMap ? N) (f : List.Vector ℕ n →.
    ℕ)
    (hdef : ∀ (v : List.Vector ℕ n) (c : ℕ), c ∈ f v → inner.toElementMap (vecElem v)
    = natElem c)
    (hundef : ∀ (v : List.Vector ℕ n), ¬ (f v).Dom → inner.toElementMap (vecElem v) =
    N.bot) :
    LamDef (strictGuardN n inner) f where
  defined v c hc := by
    rw [strictGuardN_total n inner (vecElem v) (vecElem_totals v)]; exact hdef v c hc
  undef v hv := by
```

```
    rw [strictGuardN_total n inner (vecElem v) (vecElem_totals v)]; exact hundef v hv
  strict z hz i hi hb := strictGuardN_strict n inner z hz hi hb

theorem lamDef_comp {m nn : ℕ} (f : List.Vector ℕ nn →. ℕ) (g : Fin nn → List.Vector
    ℕ m →. ℕ)
    (φf : ApproximableMap ? N) (hf : LamDef φf f)
    (φg : Fin nn → ApproximableMap ? N) (hg : ∀ i, LamDef (φg i) (g i)) :
    LamDef (strictGuardN m (φf.comp (tupleMap nn φg)))
      (fun v => (List.Vector.mOfFn fun i => g i v) >>= f) := by
  refine lamDef_of_inner (φf.comp (tupleMap nn φg)) _ ?_ ?_
  · intro v c hc
    have hc' : c ∈ (List.Vector.mOfFn fun i => g i v).bind (fun w => f w) := hc
    rw [Part.mem_bind_iff] at hc'
    obtain ⟨w, hw, hcw⟩ := hc'
    rw [toElementMap_comp, tupleMap_vecElem_eq φg g hg v w ((mem_mOfFn _ w).mp hw)]
    exact hf.defined w c hcw
  · intro v hv
    rw [toElementMap_comp]
    by_cases hall : ∀ i, (g i v).Dom
    · set w : List.Vector ℕ nn := List.Vector.ofFn (fun i => (g i v).get (hall i)) with
     hw_def
      have hwget : ∀ i, w.get i ∈ g i v := by
        intro i; rw [hw_def, List.Vector.get_ofFn]; exact Part.get_mem (hall i)
      rw [tupleMap_vecElem_eq φg g hg v w hwget]
      refine hf.undef w (fun hfw => hv ?_)
      have hmem : (f w).get hfw ∈ (List.Vector.mOfFn fun i => g i v).bind (fun w => f
    w) := by
        rw [Part.mem_bind_iff]
        exact ⟨w, (mem_mOfFn _ w).mpr hwget, Part.get_mem hfw⟩
      exact Part.dom_iff_mem.mpr ⟨_, hmem⟩
    · rw [not_forall] at hall
      obtain ⟨j, hj⟩ := hall
      refine hf.strict ((tupleMap nn φg).toElementMap (vecElem v))
        (tupleMap_argLike φg g hg v) j j.isLt ?_
      rw [component_tupleMap nn φg (vecElem v) j j.isLt, Fin.eta]
      exact (hg j).undef v hj

theorem lamDef_prec {n : ℕ} (φf φg : ApproximableMap ? N)
    (f₀ : List.Vector ℕ n → ℕ) (g₀ : List.Vector ℕ (n + 2) → ℕ)
    (hf : LamDef φf (f₀ : List.Vector ℕ n →. ℕ))
    (hg : LamDef φg (g₀ : List.Vector ℕ (n + 2) →. ℕ)) :
    LamDef (strictGuardN (n + 1) (recMap φf φg))
      (((fun v : List.Vector ℕ (n + 1) =>
        v.head.rec (f₀ v.tail) fun y IH => g₀ (y ::? IH ::? v.tail)) :
        List.Vector ℕ (n + 1) → ℕ) : List.Vector ℕ (n + 1) →. ℕ) := by
  refine lamDef_of_inner (recMap φf φg) _ (fun v c hc => ?_)
    (fun v hv => by rw [PFun.coe_val] at hv; exact absurd trivial hv)
  rw [PFun.coe_val, Part.mem_some_iff] at hc
  subst hc
  obtain ⟨d, u, rfl⟩ := List.Vector.exists_eq_cons v
  rw [List.Vector.head_cons, List.Vector.tail_cons]
  exact recMap_eval φf φg f₀ g₀ hf hg u d

theorem lamDef_primComp {m nn : ℕ} (f₀ : List.Vector ℕ nn → ℕ)
    (g₀ : Fin nn → List.Vector ℕ m → ℕ)
    (φf : ApproximableMap ? N) (hf : LamDef φf (f₀ : List.Vector ℕ nn →. ℕ))
```

```
    (φg : Fin nn → ApproximableMap ? N) (hg : ∀ i, LamDef (φg i) (g₀ i : List.Vector ℕ
     m →. ℕ)) :
    LamDef (strictGuardN m (φf.comp (tupleMap nn φg)))
      ((fun a => f₀ (List.Vector.ofFn fun i => g₀ i a) : List.Vector ℕ m → ℕ) :
        List.Vector ℕ m →. ℕ) := by
  refine lamDef_congr (fun v => ?_)
    (lamDef_comp (f₀ : _ →. ℕ) (fun i => (g₀ i : _ →. ℕ)) φf hf φg hg)
  simp only [PFun.coe_val, Vector.mOfFn_part_some]
  change (Part.some (List.Vector.ofFn fun i => g₀ i v)).bind (fun w => Part.some (f₀ w)
    )
      = Part.some (f₀ (List.Vector.ofFn fun i => g₀ i v))
  exact Part.bind_some _ _

theorem lamDef_rfind {n : ℕ} (φf : ApproximableMap ? N) {f : List.Vector ℕ (n + 1) →
    ℕ}
    (hf : LamDef φf (f : List.Vector ℕ (n + 1) →. ℕ)) :
    LamDef (strictGuardN n (findMap φf))
      (fun v : List.Vector ℕ n => Nat.rfind (rfindZeroPred f v)) := by
  refine lamDef_of_inner (findMap φf) _ ?_ ?_
  · intro v c hc
    rw [findMap_vecElem]
    have hc' : c ∈ Nat.rfind (rfindZeroPred f v) := hc
    rw [Nat.mem_rfind] at hc'
    obtain ⟨hcs, hcm⟩ := hc'
    have h0 : f (c ::? v) = 0 := of_decide_eq_true (Part.mem_some_iff.mp hcs).symm
    have hmin : ∀ i, i < c → f (i ::? v) = 0 :=
      fun i hi => of_decide_eq_false (Part.mem_some_iff.mp (hcm hi)).symm
    exact searchMap_climb φf hf v c h0 hmin c 0 (by omega)
  · intro v hv
    rw [findMap_vecElem]
    have hdiv : ∀ k, f (k ::? v) = 0 := by
      intro k h0
      refine hv ?_
      have hdom : (Nat.rfind (rfindZeroPred f v)).Dom := by
        rw [Nat.rfind_dom]
        refine ⟨k, ?_, fun {m} _ => trivial⟩
        change true ∈ Part.some (decide (f (k ::? v) = 0))
        rw [Part.mem_some_iff]
        exact (decide_eq_true_iff.mpr h0).symm
      exact hdom
    exact searchMap_diverge φf hf v hdiv 0

theorem lamDef_total {n : ℕ} (ψ : ApproximableMap ? N) (g : List.Vector ℕ n → ℕ)
    (hval : ∀ v, ψ.toElementMap (vecElem v) = natElem (g v)) :
    LamDef (strictGuardN n ψ) (g : List.Vector ℕ n →. ℕ) :=
  lamDef_of_inner ψ _
    (fun v c hc => by rw [PFun.coe_val, Part.mem_some_iff] at hc; rw [hval, ← hc])
    (fun v hv => by rw [PFun.coe_val] at hv; exact absurd trivial hv)

theorem lamDef_get {n : ℕ} (i : Fin n) :
    LamDef (strictGuardN n (projN N i))
      ((fun v => v.get i : List.Vector ℕ n → ℕ) : List.Vector ℕ n →. ℕ) :=
  lamDef_total _ _ (fun v => by
    rw [toElementMap_projN, component_vecElem_lt v i.isLt, Fin.eta])

theorem lamDef_succ : LamDef (strictGuardN 1 (succMap.comp (projN N 0)))
```

```
    ((fun v => Nat.succ v.head : List.Vector ℕ 1 → ℕ) : List.Vector ℕ 1 →. ℕ) :=
  lamDef_total _ _ (fun v => by
    rw [toElementMap_comp, toElementMap_projN, component_vecElem_lt v (by norm_num),
      succMap_natElem]
    congr 1
    simp [List.Vector.get_zero])

theorem lamDef_zero : LamDef (strictGuardN 0 (constMap ? (natElem 0)))
    ((fun _ => 0 : List.Vector ℕ 0 → ℕ) : List.Vector ℕ 0 →. ℕ) :=
  lamDef_total (constMap ? (natElem 0)) _ (fun v => by rw [toElementMap_constMap])

theorem primrec_lamDef {n : ℕ} {f : List.Vector ℕ n → ℕ} (h : Nat.Primrec' f) :
    ∃ φ : ApproximableMap ? N, LamDef φ (f : List.Vector ℕ n →. ℕ) := by
  induction h with
  | zero => exact ⟨_, lamDef_zero⟩
  | succ => exact ⟨_, lamDef_succ⟩
  | get i => exact ⟨_, lamDef_get i⟩
  | comp g _ _ ihf ihg =>
      obtain ⟨φf, hφf⟩ := ihf
      choose φg hφg using ihg
      exact ⟨_, lamDef_primComp _ g φf hφf φg hφg⟩
  | prec _ _ ihf ihg =>
      obtain ⟨φf, hφf⟩ := ihf
      obtain ⟨φg, hφg⟩ := ihg
      exact ⟨_, lamDef_prec φf φg _ _ hφf hφg⟩

theorem partrec_lamDef {n : ℕ} {f : List.Vector ℕ n →. ℕ} (h : Nat.Partrec' f) :
    ∃ φ : ApproximableMap ? N, LamDef φ f := by
  induction h with
  | prim hp => exact primrec_lamDef hp
  | comp g _ _ ihf ihg =>
      obtain ⟨φf, hφf⟩ := ihf
      choose φg hφg using ihg
      exact ⟨_, lamDef_comp _ g φf hφf φg hφg⟩
  | rfind _ ih =>
      obtain ⟨φf, hφf⟩ := ih
      exact ⟨_, lamDef_rfind φf hφf⟩

theorem partrec_one {h : ℕ →. ℕ}
    (H : Nat.Partrec' (fun v : List.Vector ℕ 1 => h v.head)) :
    ∃ τ : ApproximableMap N N,
      (∀ a m, m ∈ h a → τ.toElementMap (natElem a) = natElem m) ∧
      (∀ a, ¬ (h a).Dom → τ.toElementMap (natElem a) = N.bot) ∧
      τ.toElementMap N.bot = N.bot := by
  obtain ⟨φ, hφ⟩ := partrec_lamDef H
  refine ⟨φ.comp oneArg, ?_, ?_, ?_⟩
  · intro a m hm
    rw [toElementMap_comp, oneArg_natElem]
    refine hφ.defined (a ::? List.Vector.nil) m ?_
    rwa [List.Vector.head_cons]
  · intro a hdom
    rw [toElementMap_comp, oneArg_natElem]
    refine hφ.undef (a ::? List.Vector.nil) ?_
    rwa [List.Vector.head_cons]
  · rw [toElementMap_comp, oneArg_bot]
    refine hφ.strict ?.bot (fun j => Or.inl (component_bot j)) 0 (by omega) (
```

```
    component_bot 0)

end DomainTheory.NeighbourhoodSystems.PartrecLam
```

4.6.15 Theorem 5.6 — partial recursive functions

THEOREM 5.6. For every partial recursive function $h : \mathbf{N} \to \mathbf{N}$, there is a λ-term τ of type $(\mathbf{N} \to \mathbf{N})$ such that the only constants occurring in τ are

$$\text{cond},\ \text{succ},\ \text{pred},\ \text{zero},\ 0$$

and where if $h(n) = m$, then

$$\tau(n) = m$$

is true; and if $h(n)$ is undefined, then

$$\tau(n) = \bot$$

is true. The equation $\tau(\bot) = \bot$ is also true.

Proof : We have only formulated the theorem for functions of one variable — but to give the proof, it is convenient to pass through functions of any number of (integer) variables. We shall also have to recall the precise definition of the notion of partial recursive function.

It is also convenient to work with (*very*)*strict* functions

$$f : \mathbf{N}^k \to \mathbf{N}.$$

These are functions such that if $n_0, \ldots, n_{k-1} \in \mathbf{N}$ and $n_i = \bot$ for at least one $i < k$, then

$$f(n_0, \ldots, n_{k-1}) = \bot.$$

It is easy to check that compositions of strict functions are strict. It is also easy to see that any *partial* function

$$g : \mathbf{N}^k \to \mathbf{N}$$

extends to a strict (*approximable*) function

$$\bar{g} : \mathbf{N}^k \to \mathbf{N},$$

which takes the same values as g as long as g is defined; otherwise $\bar{g}$ takes the value $\bot$. What we want to show for *partial recursive* g is that the corresponding $\bar{g}$ is defined by a λ - expression.

In the first place we have to check that *primitive recursive* functions have λ - definitions in this sense. We recall that *primitive recursive* functions are generated from certain elementary *starting* functions by multi-variate composition and the scheme of primitive recrusion. The *starting* functions are the constant function with value zero and the "identity" or "projection" functions. For example, $g(n_0, n_1, n_2) = n_1$ for all $n_0, n_1, n_2 \in \mathbf{N}$ is one of the *starting* functions. Now we cannot just use the λ-term

$$\lambda x_0, x_1, x_2.x_1$$

to represent $\bar{g}$, because the function so defined is not strict. But any function in $\mathbf{N}^k \to \mathbf{N}$ can be cut down to a strict function by a simple device. Consider

$$\lambda x.\ \mathrm{cond}(\mathrm{zero}(x), x, x)$$

with x of type $\mathbf{N}$. This is the strict version of the identity function of one argument. The strict projection function of two arguments can be defined by

$$\lambda x_0, x_1.\ \mathrm{cond}(\mathrm{zero}(x_1), x_0, x_0).$$

The one of three arguments by:

$$\lambda x_0, x_1, x_2.\ \mathrm{cond}(\mathrm{zero}(x_0), \mathrm{cond}(\mathrm{zero}(x_2), x_1, x_1), \mathrm{cond}(\mathrm{zero}(x_2), x_1, x_1)).$$

This is not done very elegantly, and the reader can find for himself a general solution based on perhaps a better notation for the required compositions of functions.

As we remarked, strict functions are closed under substitution, and any substitution of a batch of functions into another function can be given by a λ-term, if the various functions can themselves be so defined. It only remains to λ-define functions obtained by primitive recursion. Thus, suppose, for the sake of argument, that

$$\bar{f} : \mathbf{N} \to \mathbf{N} \qquad \text{and} \qquad \bar{g} : \mathbf{N}^3 \to \mathbf{N}$$

are given as total functions with $\bar{f}$ and $\bar{g}$ being λ-definable. From them, we obtain by primitive recursion $\bar{h} : \mathbf{N}^2 \to \mathbf{N}$ where

$$\bar{h}(0, m) = \bar{f}(m),$$

$$\bar{h}(n+1, m) = \bar{g}(n, m, \bar{h}(n, m))$$

for all $n, m \in \mathbf{N}$. The λ-term defining $\bar{h}$ is

$$!k\ \lambda x, y.\ \mathrm{cond}(\mathrm{zero}(x), \bar{f}(y), \bar{g}(\mathrm{pred}(x), y, k(\mathrm{pred}(x), y))).$$

Here we have had to use the fixed-point operator on a variable k of type $(\mathbf{N}^2 \to \mathbf{N})$. The variables x, y are of type $\mathbf{N}$ and the cond-construction puts the two traditional equations into two clauses of one expression. It is easy to see that the fixed-point function *is* strict and is nothing more than $\bar{h}$.

That completes the representation of *primitive recursive* functions. To obtain the *partial recursive* functions, the idea is to use the so-called μ-scheme (least number operator) and, further, to close up under substitution. We need only treat the μ-scheme. Suppose, by way of example, $f(n, m)$ is given as a

primitive recursive function. We then define h (generally, a partial function) by

$$h(m) = \text{the least } n \text{ where } f(n, m) = 0.$$

This is often written

$$h(m) = \mu n.\ f(n, m) = 0.$$

Supposing, as we may, $\bar{f}$ is λ-definable, we introduce first

$$\bar{g} = !g\, \lambda x, y.\, \mathrm{cond}(\mathrm{zero}(\bar{f}(x,y)), x, g(\mathrm{succ}(x), y)).$$

Then $\bar{h} = \lambda y.\, \bar{g}(0, y)$. This is easily seen to be strict. Also easy to see is that if $h(m)$ is defined, then $\bar{g}(0, m) = h(m)$. But, if $h(m)$ is not defined, it takes some argument to make sure that the least fixed-point construction forces $\bar{g}(0, m) = \bot$. However, the argument is not very difficult. □

What is *not* said in 5.6 is that every λ-term defines a partial recursive function. This is true (with suitable control over the constants and types in the expression), but the proof requires a full analysis of computability properties of domain constructions. This is the topic of Lecture VII.

It should be remarked that the types of variables needed for the proof of 5.6 never get very high. In fact, types like N, N^k, and $(N^k \to N)$ were the only ones needed (with perhaps T thrown in also).

Recursion on N was the topic of 5.6; further examples of recursion on other domains are included in the exercises.

The binary μ- and primitive-recursion schemes on the number domain are with the number-domain recursion schemes peer. The `LamDef` completeness development and the one-variable representation theorem are with the partial recursive λ-definability peer.

Theorem (`theorem_56`). Every unary partial recursive function is realized by a strict approximable endomap on the number domain that returns numerals on defined values and bottom on undefined values and at bottom.

Proof. Invoke the one-variable partial-recursive λ-definability theorem. ■

`Theorem_5_6.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.partrecLam

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.NatDom (N natElem)
open DomainTheory.NeighbourhoodSystems.PartrecLam

/-- Scott's Theorem 5.6: every unary partial recursive function is realized by a
strict approximable endomap on the number domain. -/
theorem theorem_56 {h : ℕ →. ℕ}
    (H : Nat.Partrec' (fun v : List.Vector ℕ 1 => h v.head)) :
    ∃ τ : ApproximableMap N N,
      (∀ a m, m ∈ h a → τ.toElementMap (natElem a) = natElem m) ∧
      (∀ a, ¬ (h a).Dom → τ.toElementMap (natElem a) = N.bot) ∧
      τ.toElementMap N.bot = N.bot :=
  partrec_one H

end DomainTheory.NeighbourhoodSystems
```

4.6.16 Multi-argument λ-abstraction

Multi-argument λ-abstraction and application via iterated curry and uncurry, together with the pairing reconstruction used to simulate simultaneous arguments.

Definition (`curry3`). Ternary curry is the double curry of a map on a nested product.

Proof. By definition. ■

Theorem (`app_two_args`). Applying a binary map to a pair equals evaluating its curry at the two components in succession.

Proof. The curry evaluation law. ■

Theorem (`curry3_apply`). A thrice-curried map evaluates by nesting three arguments as a pair of pairs.

Proof. Apply the curry evaluation law twice. ■

Theorem (`lam_two_vars`). The single-variable λ-abstraction of a binary map recovers the map on pairs when applied to both arguments.

Proof. The curry evaluation law. ■

Theorem (`surjective_pairing`). Pairing the two projections of a product element recovers that element.

Proof. Unpack the projections and reassemble with the pairing identity. ■

Theorem (`uncurry_apply`). Uncurry evaluates by applying the curried map to the two components of a pair.

Proof. Expand uncurry as evaluation against the paired projections. ■

Theorem (`uncurry_lam_two_vars`). Uncurry is a left inverse of curry on binary maps.

Proof. The standard uncurry–curry law. ■

`multiLam.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.MultiLam
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

variable {α β ? ? : Type*}
  {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  {V₂ : NeighborhoodSystem ?} {V₃ : NeighborhoodSystem ?}

def curry₃ (g : ApproximableMap (prod (prod V₀ V₁) V₂) V₃) :
    ApproximableMap V₀ (funSpace V₁ (funSpace V₂ V₃)) :=
  curry (curry g)

theorem app_two_args (σ : ApproximableMap (prod V₀ V₁) V₂) (x : V₀.Element) (y : V₁.
    Element) :
    σ.toElementMap (pair x y) =
      (toApproxMap ((curry σ).toElementMap x)).toElementMap y := by
  rw [toElementMap_curry_apply]

theorem curry₃_apply (g : ApproximableMap (prod (prod V₀ V₁) V₂) V₃)
    (x : V₀.Element) (y : V₁.Element) (z : V₂.Element) :
    (toApproxMap ((toApproxMap ((curry₃ g).toElementMap x)).toElementMap y)).
    toElementMap z =
      g.toElementMap (pair (pair x y) z) := by
```

```
  rw [curry₃, toElementMap_curry_apply, toElementMap_curry_apply]

theorem lam_two_vars (g : ApproximableMap (prod V₀ V₁) V₂) (x : V₀.Element) (y : V₁.
    Element) :
    (toApproxMap ((curry g).toElementMap x)).toElementMap y = g.toElementMap (pair x y
    ) :=
  toElementMap_curry_apply g x y

theorem surjective_pairing (z : (prod V₀ V₁).Element) :
    pair ((proj₀ V₀ V₁).toElementMap z) ((proj₁ V₀ V₁).toElementMap z) = z := by
  rw [toElementMap_proj₀, toElementMap_proj₁, pair_fst_snd]

theorem uncurry_apply (h : ApproximableMap V₀ (funSpace V₁ V₂))
    (x : V₀.Element) (y : V₁.Element) :
    (uncurry h).toElementMap (pair x y) =
      (toApproxMap (h.toElementMap x)).toElementMap y := by
  rw [uncurry_eq, toElementMap_comp, toElementMap_paired, toElementMap_comp,
    toElementMap_proj₀, toElementMap_proj₁, fst_pair, snd_pair, evalMap_apply]

theorem uncurry_lam_two_vars (g : ApproximableMap (prod V₀ V₁) V₂) : uncurry (curry g)
    = g :=
  uncurry_curry g

end DomainTheory.NeighbourhoodSystems.MultiLam
```

4.6.17 Exercise 5.7 — currying the lambda notation

Find definitions of

$$\lambda x, y.\, \tau \quad \text{and} \quad \sigma(x, y)$$

which use only λv with one variable and applications only to one argument at a time. Note that use must be made of the combinators p_0, p_1, pair. Generalize the result to functions of many variables.

The development is with the multi-argument λ-abstraction peer.

Theorem (`example_507`). Binary λ-abstraction and application are curry and successive evaluation; uncurry inverts curry; pairing is recovered from its projections; ternary curry evaluates by nested pairs.

Proof. Assemble `lam_two_vars`, `app_two_args`, `uncurry_lam_two_vars`, `surjective_pairing`, and `curry3_apply` from the multi-argument λ peer. ■

`Exercise_5_7.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.multiLam

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap MultiLam

/-- Scott's Exercise 5.7: multi-argument λ and application via single-variable curry/
    uncurry
```

```
and the pairing projections; ternary curry as the many-variable pattern. -/
theorem example_507 :
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (g : ApproximableMap (prod V₀ V₁) V₂)
      (x : V₀.Element) (y : V₁.Element),
      (toApproxMap ((curry g).toElementMap x)).toElementMap y = g.toElementMap (pair x
    y)) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (σ : ApproximableMap (prod V₀ V₁) V₂)
      (x : V₀.Element) (y : V₁.Element),
      σ.toElementMap (pair x y) =
        (toApproxMap ((curry σ).toElementMap x)).toElementMap y) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} (g : ApproximableMap (prod V₀ V₁) V₂),
      uncurry (curry g) = g) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (z : (prod V₀ V₁).Element),
      pair ((proj₀ V₀ V₁).toElementMap z) ((proj₁ V₀ V₁).toElementMap z) = z) ∧
    (∀ {α β ? ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} {V₃ : NeighborhoodSystem ?}
      (g : ApproximableMap (prod (prod V₀ V₁) V₂) V₃)
      (x : V₀.Element) (y : V₁.Element) (z : V₂.Element),
      (toApproxMap ((toApproxMap ((curry₃ g).toElementMap x)).toElementMap y)).
   toElementMap z =
       g.toElementMap (pair (pair x y) z)) :=
  ⟨@lam_two_vars, @app_two_args, @uncurry_lam_two_vars, @surjective_pairing, @curry₃
    _apply⟩
```

4.6.18 Combinatory completeness

Combinatory completeness: I, K, and S realize every polynomial λ-term via bracket abstraction.

Definition (`Dom`). A domain package is a carrier type with a neighbourhood system.

Proof. By definition. ■

Abbreviation (`Dom.arrow`). The function-space domain between two packaged domains.

Proof. By definition. ■

Definition (`CL`). Combinatory terms are constants or applications.

Proof. By definition. ■

Definition (`CL.denote`). Denotation of a combinatory term evaluates applications via approximable maps.

Proof. By definition. ■

Definition (`Poly`). Polynomials over a parameter domain are variables, constants, or applications.

Proof. By definition. ■

Definition (`Poly.denote`). Denotation of a polynomial is a map from the parameter element.

Proof. By definition. ■

Definition (`Ielem`). The I combinator is the identity map as a filter element.

Proof. By definition. ■

Definition (`Kelem`). The K combinator is the curry of the first projection.

Proof. By definition. ■

Definition (`Sbody`). The S-body evaluates two curried maps at a shared argument.

Proof. By definition. ■

Definition (`Smap`). The S combinator map is the double curry of the S-body.
Proof. By definition. ■
Definition (`Selem`). The S combinator as a filter element on packaged domains.
Proof. By definition. ■
Definition (`bracket`). Bracket abstraction translates a polynomial into an I/K/S combinatory term.
Proof. By definition. ■
Theorem (`Smap_apply`). The S map evaluates by applying both branches at the shared argument.
Proof. Follows from the surrounding construct and evaluation laws. ■
Theorem (`toApproxMap_toFilter`). toApproxMap inverts toFilter on approximable maps.
Proof. Follows from the surrounding construct and evaluation laws. ■
Theorem (`Ielem_apply`). I acts as the identity.
Proof. Follows from the surrounding construct and evaluation laws. ■
Theorem (`Kelem_apply`). K returns its first argument.
Proof. Follows from the surrounding construct and evaluation laws. ■
Theorem (`Selem_apply`). S acts by the S-map evaluation law.
Proof. Follows from the surrounding construct and evaluation laws. ■
Theorem (`bracket_spec`). Bracket abstraction preserves denotation on every argument.
Proof. Induction on the polynomial, using the I, K, and S evaluation laws. ■

`combinatoryComplete.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.CombinatoryComplete
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

variable {α β ? : Type*}
variable {αx αa αb : Type*}
  {Vx : NeighborhoodSystem αx} {Va : NeighborhoodSystem αa} {Vb : NeighborhoodSystem α
    b}

structure Dom where

  carrier : Type

  sys : NeighborhoodSystem carrier

abbrev Dom.arrow (A B : Dom) : Dom := ⟨ApproximableMap A.sys B.sys, funSpace A.sys B.
    sys⟩

inductive CL : Dom → Type 1 where
  | con {A : Dom} (c : A.sys.Element) : CL A
  | app {A B : Dom} (f : CL (A.arrow B)) (a : CL A) : CL B

def CL.denote : {A : Dom} → CL A → A.sys.Element
```

```
  | _, .con c => c
  | _, .app f a => (toApproxMap f.denote).toElementMap a.denote

inductive Poly (X : Dom) : Dom → Type 1 where
  | var : Poly X X
  | con {A : Dom} (c : A.sys.Element) : Poly X A
  | app {A B : Dom} (f : Poly X (A.arrow B)) (a : Poly X A) : Poly X B

def Poly.denote {X : Dom} : {A : Dom} → Poly X A → X.sys.Element → A.sys.Element
  | _, .var, x => x
  | _, .con c, _ => c
  | _, .app f a, x => (toApproxMap (f.denote x)).toElementMap (a.denote x)

def Ielem (X : Dom) : (X.arrow X).sys.Element := toFilter (idMap X.sys)

def Kelem (A X : Dom) : (A.arrow (X.arrow A)).sys.Element := toFilter (curry (proj₀ A.
    sys X.sys))

def Sbody (Vx : NeighborhoodSystem αx) (Va : NeighborhoodSystem αa) (Vb :
    NeighborhoodSystem αb) :
    ApproximableMap
      (prod (prod (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)) Vx) Vb :=
  (evalMap Va Vb).comp
    (paired
      ((evalMap Vx (funSpace Va Vb)).comp
        (paired
          ((proj₀ (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)).comp
            (proj₀ (prod (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)) Vx))
          (proj₁ (prod (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)) Vx)))
      ((evalMap Vx Va).comp
        (paired
          ((proj₁ (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)).comp
            (proj₀ (prod (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)) Vx))
          (proj₁ (prod (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)) Vx))))

def Smap (Vx : NeighborhoodSystem αx) (Va : NeighborhoodSystem αa) (Vb :
    NeighborhoodSystem αb) :
    ApproximableMap (funSpace Vx (funSpace Va Vb))
      (funSpace (funSpace Vx Va) (funSpace Vx Vb)) :=
  curry (curry (Sbody Vx Va Vb))

def Selem (X A B : Dom) :
    ((X.arrow (A.arrow B)).arrow ((X.arrow A).arrow (X.arrow B))).sys.Element :=
  toFilter (Smap X.sys A.sys B.sys)

def bracket {X : Dom} : {A : Dom} → Poly X A → CL (X.arrow A)
  | _, .var => .con (Ielem X)
  | _, .con c => .app (.con (Kelem _ X)) (.con c)
  | _, .app f a => .app (.app (.con (Selem X _ _)) (bracket f)) (bracket a)

theorem Smap_apply (F : (funSpace Vx (funSpace Va Vb)).Element) (G : (funSpace Vx Va).
    Element)
    (x : Vx.Element) :
    (toApproxMap ((toApproxMap ((Smap Vx Va Vb).toElementMap F)).toElementMap G)).
    toElementMap x
      = (toApproxMap ((toApproxMap F).toElementMap x)).toElementMap ((toApproxMap G).
```

```
    toElementMap x) := by
  rw [Smap, toElementMap_curry_apply, toElementMap_curry_apply]
  simp only [Sbody, toElementMap_comp, toElementMap_paired, toElementMap_proj₀,
    toElementMap_proj₁, fst_pair, snd_pair, evalMap_apply]

theorem toApproxMap_toFilter {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
    (f : ApproximableMap V₀ V₁) : toApproxMap (toFilter f) = f := by
  have he := (funSpaceEquiv V₀ V₁).apply_symm_apply f
  rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he

theorem Ielem_apply (X : Dom) (x : X.sys.Element) :
    (toApproxMap (Ielem X)).toElementMap x = x := by
  rw [Ielem, toApproxMap_toFilter, toElementMap_idMap]

theorem Kelem_apply (A X : Dom) (c : A.sys.Element) (x : X.sys.Element) :
    (toApproxMap ((toApproxMap (Kelem A X)).toElementMap c)).toElementMap x = c := by
  rw [Kelem, toApproxMap_toFilter]
  have h := toElementMap_curry_apply (proj₀ A.sys X.sys) c x
  rw [toElementMap_proj₀, fst_pair] at h
  exact h

theorem Selem_apply (X A B : Dom)
    (F : (X.arrow (A.arrow B)).sys.Element) (G : (X.arrow A).sys.Element) (x : X.sys.
    Element) :
    (toApproxMap ((toApproxMap ((toApproxMap (Selem X A B)).toElementMap F)).
    toElementMap G)).toElementMap x
      = (toApproxMap ((toApproxMap F).toElementMap x)).toElementMap ((toApproxMap G).
    toElementMap x) := by
  rw [Selem, toApproxMap_toFilter]
  exact Smap_apply F G x

theorem bracket_spec {X : Dom} {A : Dom} (t : Poly X A) (x : X.sys.Element) :
    (toApproxMap (bracket t).denote).toElementMap x = t.denote x := by
  induction t with
  | var =>
      simp only [bracket, CL.denote, Poly.denote]
      exact Ielem_apply X x
  | @con A c =>
      simp only [bracket, CL.denote, Poly.denote]
      exact Kelem_apply A X c x
  | @app A B f a ihf iha =>
      simp only [bracket, CL.denote, Poly.denote]
      have hS := Selem_apply X A B (CL.denote (bracket f)) (CL.denote (bracket a)) x
      rw [ihf, iha] at hS
      exact hS

end DomainTheory.NeighbourhoodSystems.CombinatoryComplete
```

4.6.19 Exercise 5.8 — combinatory completeness

(For combinator nuts.) Table 5.5 was meant to show how combinators could be defined in terms of λ-expressions. Can the tables be turned to show that with enough combinators available, every λ-expression can be defined by combining combinators, using $\sigma(\tau)$ as the *only* mode of combination?

The development is with the combinatory completeness peer.

Theorem (`example_508`). Every polynomial λ-term over a packaged domain is realized by an I/K/S combinatory term whose denotation agrees at every argument.

Proof. The bracket-abstraction specification from the combinatory-completeness peer. ■

`Exercise_5_8.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.
    combinatoryComplete

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open CombinatoryComplete

/-- Scott's Exercise 5.8: every polynomial λ-term is realized by an I/K/S combinatory
term whose denotation agrees on every argument. -/
theorem example_508 :
    ∀ {X A : Dom} (t : Poly X A) (x : X.sys.Element),
      (toApproxMap (bracket t).denote).toElementMap x = t.denote x :=
  @bracket_spec
```

4.6.20 Exercise 5.9 — common fixed points

Suppose that $f, g : \mathcal{D} \to \mathcal{D}$ are approximable and $f \circ g = g \circ f$. Show that f and g have a *least common fixed point* $x = f(x) = g(x)$. (Hint: Refer back to Exercise 4.20.) If in addition $f(\bot) = g(\bot)$, show that $\mathrm{fix}(f) = \mathrm{fix}(g)$. In particular, will $\mathrm{fix}(f) = \mathrm{fix}(f^2)$? What if we only assume $f \circ g = g^2 \circ f$?

The development uses the commuting-fixed-point lemmas packaged with Exercise 4.11.

Theorem (`example_509`). Commuting endomaps share a least common fixed point; if they also agree at bottom then their least fixed points coincide; in particular the least fixed point of a map equals that of its square.

Proof. Assemble the commuting least-common-fixed-point law, the equal-bottoms uniqueness law, and the iterate-twice law from the Exercise 4.11 development. ■

`Exercise_5_9.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_11

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 5.9: commuting endomaps share a least common fixed point;
equal bottoms yield equal least fixed points; in particular fix(f) = fix(fof). -/
theorem example_509 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} {f g : ApproximableMap V V},
      f.comp g = g.comp f →
```

```
    ∃ x : V.Element, f.toElementMap x = x ∧ g.toElementMap x = x ∧
      ∀ w : V.Element, f.toElementMap w = w → g.toElementMap w = w → x ≤ w) ∧
  (∀ {α : Type*} {V : NeighborhoodSystem α} {f g : ApproximableMap V V},
    f.comp g = g.comp f →
    f.toElementMap V.bot = g.toElementMap V.bot →
      f.fixElement = g.fixElement) ∧
  (∀ {α : Type*} {V : NeighborhoodSystem α} (f : ApproximableMap V V),
    f.fixElement = (f.comp f).fixElement) :=
⟨@commuting_least_common_fixed, @fixElement_eq_of_commuting_bot,
  @fixElement_iterTwice⟩
```

4.6.21 Smash product

Smash product of neighbourhood systems, strict function spaces, and the curry isomorphism with smash.

Definition (`SmashProper`). The construction `SmashProper` as used in the smash-product development.

Proof. By definition. ■

Theorem (`inter_ne_master_left`). The stated property of `inter_ne_master_left`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`inter_ne_master_right`). The stated property of `inter_ne_master_right`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`smash`). The smash product of two neighbourhood systems consists of the joined masters together with unions of proper neighbourhoods from each factor.

Proof. The master and intersection axioms follow from those of the factors, using properness of the summands. ■

Theorem (`smash_master`). The stated property of `smash_master`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`smash_mem_bot`). The stated property of `smash_mem_bot`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`smash_mem_proper`). The stated property of `smash_mem_proper`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`smashPair`). The construction `smashPair` as used in the smash-product development.

Proof. By definition. ■

Theorem (`mem_smashPair`). The stated property of `mem_smashPair`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`IsStrict`). A map is strict when it sends bottom to bottom in the sense that proper neighbourhoods cannot pull back only the master.

Proof. By definition. ■

Abbreviation (`StrictMap`). A strict map is an approximable map that is strict.

Proof. By definition. ■

Theorem (`eq_bot_of_no_proper`). The stated property of `eq_bot_of_no_proper`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`exists_proper_of_ne_bot`). The stated property of `exists_proper_of_ne_bot`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`isStrict_constBot`). The stated property of `isStrict_constBot`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`isStrict_iff_apply_bot`). The stated property of `isStrict_iff_apply_bot`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`IsStrict.mono`). The stated property of `IsStrict.mono`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`principal_master_eq_bot`). The stated property of `principal_master_eq_bot`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`smashPair_bot_left`). The stated property of `smashPair_bot_left`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`smashPair_bot_right`). The stated property of `smashPair_bot_right`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`smashPair_eq_bot_iff`). The stated property of `smashPair_eq_bot_iff`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`smashPair_mono`). The stated property of `smashPair_mono`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`sstep`). The construction `sstep` as used in the smash-product development.

Proof. By definition. ■

Theorem (`mem_sstep`). The stated property of `mem_sstep`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`sstepFun`). The construction `sstepFun` as used in the smash-product development.

Proof. By definition. ■

Theorem (`mem_sstepFun`). The stated property of `mem_sstepFun`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`sstepFun_nil`). The stated property of `sstepFun_nil`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`sstepFun_append`). The stated property of `sstepFun_append`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`sstepFun_cons`). The stated property of `sstepFun_cons`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`sstepFun_singleton`). The stated property of `sstepFun_singleton`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`sstep_master_eq`). The stated property of `sstep_master_eq`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`sstep_inter_right`). The stated property of `sstep_inter_right`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`sstep_subset`). The stated property of `sstep_subset`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`strictFun`). The strict function space has neighbourhoods generated by finite lists of proper relation steps between strict maps.

Proof. By definition as a neighbourhood system on strict maps. ■

Theorem (`strictFun_master`). The stated property of `strictFun_master`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`sstepFun_up_closed`). The stated property of `sstepFun_up_closed`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`sstep_mem_of_mem`). The stated property of `sstep_mem_of_mem`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`mem_sstepFun_iff`). The stated property of `mem_sstepFun_iff`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`strictFun_mem_iff`). The stated property of `strictFun_mem_iff`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`strictFun_mem_inter`). The stated property of `strictFun_mem_inter`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`strictFun_mem_up_closed`). The stated property of `strictFun_mem_up_closed`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`toStrictMap`). The construction `toStrictMap` as used in the smash-product development.

Proof. By definition. ■

Theorem (`toStrictMap_rel`). The stated property of `toStrictMap_rel`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`toStrictFilter`). The construction `toStrictFilter` as used in the smash-product development.

Proof. By definition. ■

Theorem (`mem_toStrictFilter`). The stated property of `mem_toStrictFilter`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`rel_sstepFun_iff`). The stated property of `rel_sstepFun_iff`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`smashPair_principal_apply`). The stated property of `smashPair_principal_apply`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`smashSection`). The construction `smashSection` as used in the smash-product development.

Proof. By definition. ■

Theorem (`isStrict_smashSection`). The stated property of `isStrict_smashSection`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`smashSection_mono`). The stated property of `smashSection_mono`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`smashSection_rel`). The stated property of `smashSection_rel`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`smashCurryMap`). The construction `smashCurryMap` as used in the smash-product development.

Proof. By definition. ■

Theorem (`smash_mem_prodNbhd_form`). The stated property of `smash_mem_prodNbhd_form`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`smashUncurryMap`). The construction `smashUncurryMap` as used in the smash-product development.

Proof. By definition. ■

Theorem (`smashUncurry_curry`). The stated property of `smashUncurry_curry`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`sstep_cod_master`). The stated property of `sstep_cod_master`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`section_uncurry_rel`). The stated property of `section_uncurry_rel`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`smashCurry_uncurry`). The stated property of `smashCurry_uncurry`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Definition (`smashCurryEquiv`). Currying against the smash product yields an order isomorphism between strict maps out of smash and strict maps into the strict function space.

Proof. The forward and reverse maps are smashCurryMap and smashUncurryMap; the inverse laws and relation preservation supply the isomorphism. ■

Definition (`strictFunEquiv`). The construction `strictFunEquiv` as used in the smash-product development.

Proof. By definition. ■

Theorem (`isStrict_idMap`). The stated property of `isStrict_idMap`.

Proof. Follows from the surrounding smash and strict-map laws. ■

Theorem (`smash_mem_iff`). Membership in the smash product is either the joined master or a proper smash neighbourhood.

Proof. By definition. ■

smashProduct.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofMono

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.cond
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.funSpace
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.
    ApproximableMap2
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.SmashProduct
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

variable {α β ? : Type*}

def SmashProper (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (W : Set (α ? β
    )) : Prop :=
  ∃ X Y, V₀.mem X ∧ X = V₀.master ∧ V₁.mem Y ∧ Y = V₁.master ∧ W = prodNbhd X Y

variable {α β ? : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

theorem inter_ne_master_left {X X' : Set α} (hX : V₀.mem X) (hXne : X = V₀.master) :
    X ∩ X' = V₀.master := by
  intro h
  apply hXne
```

```
  refine Set.Subset.antisymm (V₀.sub_master hX) ?_
  rw [← h]; exact Set.inter_subset_left

theorem inter_ne_master_right {Y Y' : Set β} (hY : V₁.mem Y) (hYne : Y = V₁.master) :
    Y ∩ Y' = V₁.master :=
  inter_ne_master_left (V₀ := V₁) hY hYne
end DomainTheory.NeighbourhoodSystems.SmashProduct

namespace DomainTheory.NeighbourhoodSystems.SmashProduct
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
variable {α β ? : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def smash (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) : NeighborhoodSystem
    (α ? β) where
  mem W := W = prodNbhd V₀.master V₁.master ∨ SmashProper V₀ V₁ W
  master := prodNbhd V₀.master V₁.master
  master_nonempty := by
    obtain ⟨a, ha⟩ := V₀.master_nonempty
    exact ⟨Sum.inl a, prodNbhd.mem_inl.mpr ha⟩
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' Z (rfl | ⟨X, Y, hX, hXne, hY, hYne, rfl⟩) hW' hZ hZsub
    ·
      rcases hW' with rfl | ⟨X', Y', hX', hX'ne, hY', hY'ne, rfl⟩
      · left; rw [Set.inter_self]
      · right
        refine ⟨X', Y', hX', hX'ne, hY', hY'ne, ?_⟩
        rw [prodNbhd.inter, Set.inter_eq_right.mpr (V₀.sub_master hX'),
          Set.inter_eq_right.mpr (V₁.sub_master hY')]
    ·
      rcases hW' with rfl | ⟨X', Y', hX', hX'ne, hY', hY'ne, rfl⟩
      · right
        refine ⟨X, Y, hX, hXne, hY, hYne, ?_⟩
        rw [prodNbhd.inter, Set.inter_eq_left.mpr (V₀.sub_master hX),
          Set.inter_eq_left.mpr (V₁.sub_master hY)]
      ·
        right
        rw [prodNbhd.inter] at hZsub ⊢
        rcases hZ with hZeq | ⟨U, V, hU, _, hV, _, rfl⟩
        ·
          exfalso
          rw [hZeq] at hZsub
          obtain ⟨hΔ₀, _⟩ := prodNbhd.subset_iff.mp hZsub
          exact hXne (Set.Subset.antisymm (V₀.sub_master hX) (hΔ₀.trans Set.
    inter_subset_left))
        · obtain ⟨hUsub, hVsub⟩ := prodNbhd.subset_iff.mp hZsub
          refine ⟨X ∩ X', Y ∩ Y', ?_, ?_, ?_, ?_, rfl⟩
          · exact V₀.inter_mem hX hX' hU hUsub
          · exact inter_ne_master_left hX hXne
          · exact V₁.inter_mem hY hY' hV hVsub
          · exact inter_ne_master_right hY hYne
  sub_master := by
    rintro W (rfl | ⟨X, Y, hX, _, hY, _, rfl⟩)
    · exact subset_rfl
    · exact prodNbhd.subset_iff.mpr ⟨V₀.sub_master hX, V₁.sub_master hY⟩
```

```
@[simp] theorem smash_master :
    (smash V₀ V₁).master = prodNbhd V₀.master V₁.master := rfl

@[simp] theorem smash_mem_bot {W : Set (α ? β)} :
    (smash V₀ V₁).bot.mem W ↔ W = prodNbhd V₀.master V₁.master := by
  rw [NeighborhoodSystem.mem_bot, smash_master]

theorem smash_mem_proper {X : Set α} {Y : Set β} (hX : V₀.mem X) (hXne : X = V₀.master)
    (hY : V₁.mem Y) (hYne : Y = V₁.master) : (smash V₀ V₁).mem (prodNbhd X Y) :=
  Or.inr ⟨X, Y, hX, hXne, hY, hYne, rfl⟩

def smashPair (x : V₀.Element) (y : V₁.Element) : (smash V₀ V₁).Element where
  mem W := W = prodNbhd V₀.master V₁.master ∨
    ∃ X Y, x.mem X ∧ X = V₀.master ∧ y.mem Y ∧ Y = V₁.master ∧ W = prodNbhd X Y
  sub := by
    rintro W (rfl | ⟨X, Y, hX, hXne, hY, hYne, rfl⟩)
    · exact (smash V₀ V₁).master_mem
    · exact smash_mem_proper (x.sub hX) hXne (y.sub hY) hYne
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, Y, hX, hXne, hY, hYne, rfl⟩) hW'
    · rcases hW' with rfl | ⟨X', Y', hX', hX'ne, hY', hY'ne, rfl⟩
      · left; rw [Set.inter_self]
      · right
        exact ⟨X', Y', hX', hX'ne, hY', hY'ne, by
          rw [prodNbhd.inter, Set.inter_eq_right.mpr (V₀.sub_master (x.sub hX')),
            Set.inter_eq_right.mpr (V₁.sub_master (y.sub hY'))]⟩
    · rcases hW' with rfl | ⟨X', Y', hX', hX'ne, hY', hY'ne, rfl⟩
      · right
        exact ⟨X, Y, hX, hXne, hY, hYne, by
          rw [prodNbhd.inter, Set.inter_eq_left.mpr (V₀.sub_master (x.sub hX)),
            Set.inter_eq_left.mpr (V₁.sub_master (y.sub hY))]⟩
      · right
        refine ⟨X ∩ X', Y ∩ Y', x.inter_mem hX hX', inter_ne_master_left (x.sub hX)
    hXne,
          y.inter_mem hY hY', inter_ne_master_right (y.sub hY) hYne, ?_⟩
        rw [prodNbhd.inter]
  up_mem := by
    rintro W W' (rfl | ⟨X, Y, hX, hXne, hY, hYne, rfl⟩) hW' hsub
    · left
      exact Set.Subset.antisymm ((smash V₀ V₁).sub_master hW') hsub
    · rcases hW' with rfl | ⟨X', Y', hX', hX'ne, hY', hY'ne, rfl⟩
      · left; rfl
      · obtain ⟨hXX', hYY'⟩ := prodNbhd.subset_iff.mp hsub
        right
        exact ⟨X', Y', x.up_mem hX hX' hXX', hX'ne, y.up_mem hY hY' hYY', hY'ne, rfl⟩

@[simp] theorem mem_smashPair {x : V₀.Element} {y : V₁.Element} {W : Set (α ? β)} :
    (smashPair x y).mem W ↔ W = prodNbhd V₀.master V₁.master ∨
      ∃ X Y, x.mem X ∧ X = V₀.master ∧ y.mem Y ∧ Y = V₁.master ∧ W = prodNbhd X Y :=
    Iff.rfl

def IsStrict (f : ApproximableMap V₀ V₁) : Prop :=
  ∀ {{Y}}, f.rel V₀.master Y → Y = V₁.master
```

```
abbrev StrictMap (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :=
  {f : ApproximableMap V₀ V₁ // IsStrict f}

theorem eq_bot_of_no_proper {x : V₀.Element} (hx : ∀ X, x.mem X → X = V₀.master) :
    x = V₀.bot :=
  le_antisymm (fun W hW => by rw [NeighborhoodSystem.mem_bot]; exact hx W hW)
    (V₀.bot_le x)

theorem exists_proper_of_ne_bot {x : V₀.Element} (hx : x = V₀.bot) :
    ∃ X, x.mem X ∧ X = V₀.master := by
  by_contra hc
  refine hx (le_antisymm (fun W hW => ?_) (V₀.bot_le x))
  rw [NeighborhoodSystem.mem_bot]
  by_contra hWne
  exact hc ⟨W, hW, hWne⟩

theorem isStrict_constBot : IsStrict (constMap V₀ (V₁.bot)) := by
  rintro Y ⟨_, hY⟩
  rwa [NeighborhoodSystem.mem_bot] at hY

theorem isStrict_iff_apply_bot {f : ApproximableMap V₀ V₁} :
    IsStrict f ↔ f.toElementMap V₀.bot = V₁.bot := by
  constructor
  · intro h
    apply Element.ext
    intro Y
    rw [NeighborhoodSystem.mem_bot]
    constructor
    · rintro ⟨X, hX, hrel⟩
      rw [NeighborhoodSystem.mem_bot] at hX; subst hX
      exact h hrel
    · rintro rfl
      exact ⟨V₀.master, V₀.bot.master_mem, f.master_rel⟩
  · intro h Y hrel
    have : (f.toElementMap V₀.bot).mem Y := ⟨V₀.master, V₀.bot.master_mem, hrel⟩
    rw [h, NeighborhoodSystem.mem_bot] at this
    exact this

theorem IsStrict.mono {f g : ApproximableMap V₀ V₁} (hf : IsStrict f) (hgf : g ≤ f) :
    IsStrict g :=
  fun _ hrel => hf (hgf _ _ hrel)

theorem principal_master_eq_bot {X : Set α} (hX : V₀.mem X) (hXm : X = V₀.master) :
    V₀.principal hX = V₀.bot := by
  subst hXm; rfl

theorem smashPair_bot_left (y : V₁.Element) : smashPair V₀.bot y = (smash V₀ V₁).bot :=
    by
  apply eq_bot_of_no_proper
  rintro W (rfl | ⟨X, Y, hxX, hXne, hyY, hYne, rfl⟩)
  · rfl
  · rw [NeighborhoodSystem.mem_bot] at hxX; exact absurd hxX hXne

theorem smashPair_bot_right (x : V₀.Element) : smashPair x V₁.bot = (smash V₀ V₁).bot
    := by
  apply eq_bot_of_no_proper
```

```
  rintro W (rfl | ⟨X, Y, hxX, hXne, hyY, hYne, rfl⟩)
  · rfl
  · rw [NeighborhoodSystem.mem_bot] at hyY; exact absurd hyY hYne

theorem smashPair_eq_bot_iff {x : V₀.Element} {y : V₁.Element} :
    smashPair x y = (smash V₀ V₁).bot ↔ x = V₀.bot ∨ y = V₁.bot := by
  constructor
  · intro h
    by_contra hcon
    obtain ⟨hx, hy⟩ := not_or.mp hcon
    obtain ⟨X, hxX, hXne⟩ := exists_proper_of_ne_bot hx
    obtain ⟨Y, hyY, hYne⟩ := exists_proper_of_ne_bot hy

    have hmem : (smashPair x y).mem (prodNbhd X Y) :=
      Or.inr ⟨X, Y, hxX, hXne, hyY, hYne, rfl⟩
    rw [h, smash_mem_bot] at hmem
    obtain ⟨hX, _⟩ := prodNbhd.injective hmem
    exact hXne hX
  · intro h
    apply eq_bot_of_no_proper
    rintro W (rfl | ⟨X, Y, hxX, hXne, hyY, hYne, rfl⟩)
    · rfl
    · exfalso
      rcases h with hx | hy
      · rw [hx, NeighborhoodSystem.mem_bot] at hxX; exact hXne hxX
      · rw [hy, NeighborhoodSystem.mem_bot] at hyY; exact hYne hyY

theorem smashPair_mono {x x' : V₀.Element} {y y' : V₁.Element} (hx : x ≤ x') (hy : y ≤
    y') :
    smashPair x y ≤ smashPair x' y' := by
  rintro W (rfl | ⟨X, Y, hxX, hXne, hyY, hYne, rfl⟩)
  · exact Or.inl rfl
  · exact Or.inr ⟨X, Y, hx X hxX, hXne, hy Y hyY, hYne, rfl⟩

def sstep (X : Set α) (Y : Set β) : Set (StrictMap V₀ V₁) := {f | f.1.rel X Y}

@[simp] theorem mem_sstep {X : Set α} {Y : Set β} {f : StrictMap V₀ V₁} :
    f ∈ sstep X Y ↔ f.1.rel X Y := Iff.rfl

def sstepFun (L : List (Set α × Set β)) : Set (StrictMap V₀ V₁) :=
  {f | ∀ p ∈ L, f.1.rel p.1 p.2}

@[simp] theorem mem_sstepFun {L : List (Set α × Set β)} {f : StrictMap V₀ V₁} :
    f ∈ sstepFun L ↔ ∀ p ∈ L, f.1.rel p.1 p.2 := Iff.rfl

@[simp] theorem sstepFun_nil : (sstepFun [] : Set (StrictMap V₀ V₁)) = Set.univ := by
  ext f; simp

theorem sstepFun_append (L L' : List (Set α × Set β)) :
    (sstepFun (L ++ L') : Set (StrictMap V₀ V₁)) = sstepFun L ∩ sstepFun L' := by
  ext f
  simp only [mem_sstepFun, List.mem_append, Set.mem_inter_iff]
  constructor
  · intro h; exact ⟨fun p hp => h p (Or.inl hp), fun p hp => h p (Or.inr hp)⟩
  · rintro ⟨hL, hL'⟩ p (hp | hp)
    · exact hL p hp
```

```
    · exact hL' p hp

theorem sstepFun_cons (p : Set α × Set β) (L : List (Set α × Set β)) :
    (sstepFun (p :: L) : Set (StrictMap V₀ V₁)) = sstep p.1 p.2 ∩ sstepFun L := by
  ext f
  simp only [mem_sstepFun, List.mem_cons, Set.mem_inter_iff, mem_sstep]
  constructor
  · intro h; exact ⟨h p (Or.inl rfl), fun q hq => h q (Or.inr hq)⟩
  · rintro ⟨hp, hrest⟩ q (rfl | hq)
    · exact hp
    · exact hrest q hq

theorem sstepFun_singleton (X : Set α) (Y : Set β) :
    (sstepFun [(X, Y)] : Set (StrictMap V₀ V₁)) = sstep X Y := by
  rw [sstepFun_cons, sstepFun_nil, Set.inter_univ]

@[simp] theorem sstep_master_eq :
    (sstep V₀.master V₁.master : Set (StrictMap V₀ V₁)) = Set.univ := by
  ext f; simpa using f.1.master_rel

theorem sstep_inter_right {X : Set α} {Y Y' : Set β} (hY : V₁.mem Y) (hY' : V₁.mem Y')
    :
    (sstep X Y ∩ sstep X Y' : Set (StrictMap V₀ V₁)) = sstep X (Y ∩ Y') := by
  ext f
  simp only [Set.mem_inter_iff, mem_sstep]
  constructor
  · rintro ⟨h, h'⟩; exact f.1.inter_right h h'
  · intro h
    exact ⟨f.1.mono h subset_rfl Set.inter_subset_left (f.1.rel_dom h) hY,
           f.1.mono h subset_rfl Set.inter_subset_right (f.1.rel_dom h) hY'⟩

theorem sstep_subset {X X' : Set α} {Y Y' : Set β} (hX' : V₀.mem X') (hY' : V₁.mem Y')
    (hX'X : X' ⊆ X) (hYY' : Y ⊆ Y') : (sstep X Y : Set (StrictMap V₀ V₁)) ⊆ sstep X' Y
    ' := by
  intro f hf
  exact f.1.mono hf hX'X hYY' hX' hY'

def strictFun (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    NeighborhoodSystem (StrictMap V₀ V₁) where
  mem W := (∃ L : List (Set α × Set β), (∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2) ∧ W =
    sstepFun L)
    ∧ W.Nonempty
  master := Set.univ
  master_nonempty := ⟨⟨constMap V₀ V₁.bot, isStrict_constBot⟩, Set.mem_univ _⟩
  master_mem := ⟨⟨[], by simp, sstepFun_nil.symm⟩,
    ⟨⟨constMap V₀ V₁.bot, isStrict_constBot⟩, Set.mem_univ _⟩⟩
  inter_mem := by
    rintro W W' Z ⟨⟨L, hL, rfl⟩, _⟩ ⟨⟨L', hL', rfl⟩, _⟩ ⟨_, hZne⟩ hZsub
    refine ⟨⟨L ++ L', ?_, (sstepFun_append _ _).symm⟩, hZne.mono hZsub⟩
    intro p hp
    rcases List.mem_append.mp hp with h | h
    · exact hL p h
    · exact hL' p h
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem strictFun_master : (strictFun V₀ V₁).master = Set.univ := rfl
```

```
theorem sstepFun_up_closed {L : List (Set α × Set β)} {f f' : StrictMap V₀ V₁}
    (hf : f ∈ sstepFun L) (hff' : f ≤ f') : f' ∈ sstepFun L := by
  intro p hp
  exact hff' p.1 p.2 (hf p hp)

theorem sstep_mem_of_mem {g : StrictMap V₀ V₁} {X : Set α} {Y : Set β} (h : g.1.rel X Y
    ) :
    (strictFun V₀ V₁).mem (sstep X Y) := by
  refine ⟨⟨[(X, Y)], ?_, (sstepFun_singleton X Y).symm⟩, ⟨g, h⟩⟩
  intro p hp; rw [List.mem_singleton] at hp; subst hp
  exact ⟨g.1.rel_dom h, g.1.rel_cod h⟩

theorem mem_sstepFun_iff (φ : (strictFun V₀ V₁).Element) {L : List (Set α × Set β)}
    (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2) :
    φ.mem (sstepFun L) ↔ ∀ p ∈ L, φ.mem (sstep p.1 p.2) := by
  induction L with
  | nil => simp only [sstepFun_nil, List.not_mem_nil, IsEmpty.forall_iff, implies_true
    , iff_true]
           exact φ.master_mem
  | cons p L ih =>
    rw [sstepFun_cons]
    have hLtail : ∀ q ∈ L, V₀.mem q.1 ∧ V₁.mem q.2 :=
      fun q hq => hL q (List.mem_cons.mpr (Or.inr hq))
    constructor
    · intro hmem
      obtain ⟨g, hg⟩ := (φ.sub hmem).2
      have hstep : φ.mem (sstep p.1 p.2) :=
        φ.up_mem hmem (sstep_mem_of_mem (g := g) (hg.1)) Set.inter_subset_left
      have hne : (sstep p.1 p.2 ∩ sstepFun L).Nonempty := (φ.sub hmem).2
      have htail : φ.mem (sstepFun L) :=
        φ.up_mem hmem ⟨⟨L, hLtail, rfl⟩, hne.mono Set.inter_subset_right⟩ Set.
    inter_subset_right
      intro q hq
      rcases List.mem_cons.mp hq with rfl | hq
      · exact hstep
      · exact (ih hLtail).mp htail q hq
    · intro hall
      have hstep : φ.mem (sstep p.1 p.2) := hall p (List.mem_cons.mpr (Or.inl rfl))
      have htail : φ.mem (sstepFun L) :=
        (ih hLtail).mpr (fun q hq => hall q (List.mem_cons.mpr (Or.inr hq)))
      exact φ.inter_mem hstep htail

theorem strictFun_mem_iff {W : Set (StrictMap V₀ V₁)} :
    (strictFun V₀ V₁).mem W ↔
      (∃ L : List (Set α × Set β), (∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem p.2) ∧ W = sstepFun L
    )
        ∧ W.Nonempty := Iff.rfl

theorem strictFun_mem_inter {W W' : Set (StrictMap V₀ V₁)}
    (hW : (strictFun V₀ V₁).mem W) (hW' : (strictFun V₀ V₁).mem W') (hne : (W ∩ W').
    Nonempty) :
    (strictFun V₀ V₁).mem (W ∩ W') := by
  obtain ⟨⟨L, hL, rfl⟩, _⟩ := hW
  obtain ⟨⟨L', hL', rfl⟩, _⟩ := hW'
  refine ⟨⟨L ++ L', ?_, (sstepFun_append _ _).symm⟩, hne⟩
```

```
  intro p hp
  rcases List.mem_append.mp hp with h | h
  · exact hL p h
  · exact hL' p h

theorem strictFun_mem_up_closed {W : Set (StrictMap V₀ V₁)} (hW : (strictFun V₀ V₁).mem
    W)
    {f f' : StrictMap V₀ V₁} (hf : f ∈ W) (hff' : f ≤ f') : f' ∈ W := by
  obtain ⟨⟨L, _, rfl⟩, _⟩ := hW
  exact sstepFun_up_closed hf hff'

def toStrictMap (φ : (strictFun V₀ V₁).Element) : StrictMap V₀ V₁ :=
  ⟨{ rel := fun X Y => φ.mem (sstep X Y)
     rel_dom := by intro X Y h; obtain ⟨f, hf⟩ := (φ.sub h).2; exact f.1.rel_dom hf
     rel_cod := by intro X Y h; obtain ⟨f, hf⟩ := (φ.sub h).2; exact f.1.rel_cod hf
     master_rel := by
       show φ.mem (sstep V₀.master V₁.master); rw [sstep_master_eq]; exact φ.master_mem
     inter_right := by
       intro X Y Y' h h'
       obtain ⟨f, hf⟩ := (φ.sub h).2
       obtain ⟨f', hf'⟩ := (φ.sub h').2
       have hY : V₁.mem Y := f.1.rel_cod hf
       have hY' : V₁.mem Y' := f'.1.rel_cod hf'
       show φ.mem (sstep X (Y ∩ Y'))
       rw [← sstep_inter_right hY hY']
       exact φ.inter_mem h h'
     mono := by
       intro X X' Y Y' h hX'X hYY' hX' hY'
       obtain ⟨g, hg⟩ := (φ.sub h).2
       have hg' : g.1.rel X' Y' := g.1.mono hg hX'X hYY' hX' hY'
       show φ.mem (sstep X' Y')
       exact φ.up_mem h (sstep_mem_of_mem (g := g) hg') (sstep_subset hX' hY' hX'X hYY
   ') },
   by
     intro Y h
     obtain ⟨g, hg⟩ := (φ.sub h).2
     exact g.2 hg⟩

@[simp] theorem toStrictMap_rel {φ : (strictFun V₀ V₁).Element} {X : Set α} {Y : Set β}
    :
    (toStrictMap φ).1.rel X Y ↔ φ.mem (sstep X Y) := Iff.rfl

def toStrictFilter (f : StrictMap V₀ V₁) : (strictFun V₀ V₁).Element where
  mem W := (strictFun V₀ V₁).mem W ∧ f ∈ W
  sub h := h.1
  master_mem := ⟨(strictFun V₀ V₁).master_mem, Set.mem_univ f⟩
  inter_mem := by
    rintro W W' ⟨hW, hfW⟩ ⟨hW', hfW'⟩
    exact ⟨strictFun_mem_inter hW hW' ⟨f, Set.mem_inter hfW hfW'⟩, Set.mem_inter hfW
    hfW'⟩
  up_mem := by rintro W W' ⟨hW, hfW⟩ hW' hWW'; exact ⟨hW', hWW' hfW⟩

@[simp] theorem mem_toStrictFilter {f : StrictMap V₀ V₁} {W : Set (StrictMap V₀ V₁)} :
    (toStrictFilter f).mem W ↔ (strictFun V₀ V₁).mem W ∧ f ∈ W := Iff.rfl

variable {α β ? : Type*}
```

```
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {V₂ : NeighborhoodSystem ?}

theorem rel_sstepFun_iff (h : ApproximableMap V₀ (strictFun V₁ V₂)) {X : Set α} (hX : V
    ₀.mem X)
    {L : List (Set β × Set ?)} (hL : ∀ p ∈ L, V₁.mem p.1 ∧ V₂.mem p.2) :
    h.rel X (sstepFun L) ↔ ∀ p ∈ L, h.rel X (sstep p.1 p.2) := by
  induction L with
  | nil =>
    simp only [sstepFun_nil, List.not_mem_nil, IsEmpty.forall_iff, implies_true,
    iff_true]
    show h.rel X (strictFun V₁ V₂).master
    exact h.rel_master hX
  | cons p L ih =>
    rw [sstepFun_cons]
    have hp := hL p (List.mem_cons.mpr (Or.inl rfl))
    have hLtail : ∀ q ∈ L, V₁.mem q.1 ∧ V₂.mem q.2 :=
      fun q hq => hL q (List.mem_cons.mpr (Or.inr hq))
    constructor
    · intro hmem
      obtain ⟨f, hf⟩ := (strictFun_mem_iff.mp (h.rel_cod hmem)).2
      have hstep : h.rel X (sstep p.1 p.2) :=
        h.mono hmem subset_rfl Set.inter_subset_left hX (sstep_mem_of_mem (g := f) (hf
    .1))
      have hne : (sstep p.1 p.2 ∩ sstepFun L).Nonempty := (h.rel_cod hmem).2
      have htail : h.rel X (sstepFun L) :=
        h.mono hmem subset_rfl Set.inter_subset_right hX
          ⟨⟨L, hLtail, rfl⟩, hne.mono Set.inter_subset_right⟩
      intro q hq
      rcases List.mem_cons.mp hq with rfl | hq
      · exact hstep
      · exact (ih hLtail).mp htail q hq
    · intro hall
      have hstep : h.rel X (sstep p.1 p.2) := hall p (List.mem_cons.mpr (Or.inl rfl))
      have htail : h.rel X (sstepFun L) :=
        (ih hLtail).mpr (fun q hq => hall q (List.mem_cons.mpr (Or.inr hq)))
      exact h.inter_right hstep htail

theorem smashPair_principal_apply (g : ApproximableMap (smash V₀ V₁) V₂)
    {X : Set α} {Y : Set β} (hX : V₀.mem X) (hXne : X = V₀.master)
    (hY : V₁.mem Y) (hYne : Y = V₁.master) {Z : Set ?} :
    (g.toElementMap (smashPair (V₀.principal hX) (V₁.principal hY))).mem Z
      ↔ g.rel (prodNbhd X Y) Z := by
  constructor
  · rintro ⟨W, hW, hrel⟩
    rcases hW with rfl | ⟨A, B, ⟨_, hXA⟩, _, ⟨_, hYB⟩, _, rfl⟩
    · exact g.mono hrel (prodNbhd.subset_iff.mpr ⟨V₀.sub_master hX, V₁.sub_master hY⟩)
    subset_rfl
        (smash_mem_proper hX hXne hY hYne) (g.rel_cod hrel)
    · exact g.mono hrel (prodNbhd.subset_iff.mpr ⟨hXA, hYB⟩) subset_rfl
        (smash_mem_proper hX hXne hY hYne) (g.rel_cod hrel)
  · intro hrel
    exact ⟨prodNbhd X Y, Or.inr ⟨X, Y, ⟨hX, subset_rfl⟩, hXne, ⟨hY, subset_rfl⟩, hYne,
    rfl⟩, hrel⟩

def smashSection (g : ApproximableMap (smash V₀ V₁) V₂) {X : Set α} (hX : V₀.mem X) :
```

```
    ApproximableMap V₁ V₂ :=
  ofMono (fun Y hY => g.toElementMap (smashPair (V₀.principal hX) (V₁.principal hY)))
    (by
      intro Y Y' hY hY' hY'Y
      exact toElementMap_mono g (smashPair_mono le_rfl ((V₁.principal_le_iff hY hY').
    mpr hY'Y)))

theorem isStrict_smashSection {g : ApproximableMap (smash V₀ V₁) V₂} (hg : IsStrict g)
    {X : Set α} (hX : V₀.mem X) : IsStrict (smashSection g hX) := by
  rw [isStrict_iff_apply_bot, smashSection,
    show (V₁.bot) = V₁.principal V₁.master_mem from rfl, toElementMap_ofMono_principal,
    show smashPair (V₀.principal hX) (V₁.principal V₁.master_mem) = (smash V₀ V₁).bot
    from
      smashPair_bot_right _]
  exact isStrict_iff_apply_bot.mp hg

theorem smashSection_mono {g : ApproximableMap (smash V₀ V₁) V₂} {X X' : Set α}
    (hX : V₀.mem X) (hX' : V₀.mem X') (hX'X : X' ⊆ X) :
    smashSection g hX ≤ smashSection g hX' := by
  intro Y Z hrel
  obtain ⟨hY, hmem⟩ := hrel
  exact ⟨hY, toElementMap_mono g
    (smashPair_mono ((V₀.principal_le_iff hX hX').mpr hX'X) le_rfl) Z hmem⟩

theorem smashSection_rel {g : ApproximableMap (smash V₀ V₁) V₂} {X : Set α} (hX : V₀.
    mem X)
    {Y : Set β} {Z : Set ?} :
    (smashSection g hX).rel Y Z ↔
      ∃ hY : V₁.mem Y, (g.toElementMap (smashPair (V₀.principal hX) (V₁.principal hY)))
    .mem Z :=
  Iff.rfl

def smashCurryMap (g : StrictMap (smash V₀ V₁) V₂) : StrictMap V₀ (strictFun V₁ V₂) :=
  ⟨{ rel := fun X N => ∃ hX : V₀.mem X, (strictFun V₁ V₂).mem N ∧
       (⟨smashSection g.1 hX, isStrict_smashSection g.2 hX⟩ : StrictMap V₁ V₂) ∈ N
     rel_dom := fun ⟨hX, _⟩ => hX
     rel_cod := fun ⟨_, hN, _⟩ => hN
     master_rel := ⟨V₀.master_mem, (strictFun V₁ V₂).master_mem, Set.mem_univ _⟩
     inter_right := by
       rintro X N N' ⟨hX, hN, hmem⟩ ⟨_, hN', hmem'⟩
       exact ⟨hX, strictFun_mem_inter hN hN' ⟨_, hmem, hmem'⟩, Set.mem_inter hmem hmem'⟩
     mono := by
       rintro X X' N N' ⟨hX, hN, hmem⟩ hX'X hNN' hX' hN'
       refine ⟨hX', hN', strictFun_mem_up_closed hN' (hNN' hmem) ?_⟩
       exact Subtype.coe_le_coe.mp (smashSection_mono hX hX' hX'X) },
  by
    rintro N ⟨hΔ₀, hN, hmem⟩
    obtain ⟨⟨L, hL, rfl⟩, _⟩ := hN
    have hall : ∀ p ∈ L, p.2 = V₂.master := by
      intro p hp
      have hpr : (smashSection g.1 hΔ₀).rel p.1 p.2 := hmem p hp
      rw [smashSection_rel] at hpr
      obtain ⟨hp1, hpmem⟩ := hpr
      have hbot : smashPair (V₀.principal hΔ₀) (V₁.principal hp1) = (smash V₀ V₁).bot
    := by
        rw [principal_master_eq_bot hΔ₀ rfl]; exact smashPair_bot_left _
```

```
      rw [hbot, isStrict_iff_apply_bot.mp g.2, NeighborhoodSystem.mem_bot] at hpmem
      exact hpmem
    apply Set.eq_univ_of_forall
    intro f q hq
    rw [hall q hq]
    exact (f.1).rel_master (hL q hq).1⟩

theorem smash_mem_prodNbhd_form {W : Set (α ? β)} (hW : (smash V₀ V₁).mem W) :
    ∃ A B, V₀.mem A ∧ V₁.mem B ∧ W = prodNbhd A B := by
  rcases hW with rfl | ⟨X, Y, hX, _, hY, _, rfl⟩
  · exact ⟨_, _, V₀.master_mem, V₁.master_mem, rfl⟩
  · exact ⟨X, Y, hX, hY, rfl⟩

def smashUncurryMap (h : StrictMap V₀ (strictFun V₁ V₂)) : StrictMap (smash V₀ V₁) V₂ :=
  ⟨{ rel := fun W Z => (smash V₀ V₁).mem W ∧ h.1.rel (Sum.inl ^-1' W) (sstep (Sum.inr
    ^-1' W) Z)
     rel_dom := fun hh => hh.1
     rel_cod := by
       rintro W Z ⟨_, hrel⟩
       obtain ⟨f, hf⟩ := (strictFun_mem_iff.mp (h.1.rel_cod hrel)).2
       exact f.1.rel_cod hf
     master_rel := by
       refine ⟨(smash V₀ V₁).master_mem, ?_⟩
       rw [show (smash V₀ V₁).master = prodNbhd V₀.master V₁.master from rfl,
         prodNbhd.inl_preimage, prodNbhd.inr_preimage, sstep_master_eq]
       exact h.1.master_rel
     inter_right := by
       rintro W Z Z' ⟨hW, hrel⟩ ⟨_, hrel'⟩
       obtain ⟨f, hf⟩ := (strictFun_mem_iff.mp (h.1.rel_cod hrel)).2
       obtain ⟨f', hf'⟩ := (strictFun_mem_iff.mp (h.1.rel_cod hrel')).2
       refine ⟨hW, ?_⟩
       rw [← sstep_inter_right (f.1.rel_cod hf) (f'.1.rel_cod hf')]
       exact h.1.inter_right hrel hrel'
     mono := by
       rintro W W₂ Z Z' ⟨_, hrel⟩ hW₂W hZZ' hW₂ hZ'
       have hinl : Sum.inl ^-1' W₂ ⊆ Sum.inl ^-1' W := Set.preimage_mono hW₂W
       have hinr : Sum.inr ^-1' W₂ ⊆ Sum.inr ^-1' W := Set.preimage_mono hW₂W
       obtain ⟨A, B, hA, hB, rfl⟩ := smash_mem_prodNbhd_form hW₂
       refine ⟨hW₂, ?_⟩
       rw [prodNbhd.inl_preimage] at hinl
       rw [prodNbhd.inr_preimage] at hinr
       rw [prodNbhd.inl_preimage, prodNbhd.inr_preimage]
       obtain ⟨f, hf⟩ := (strictFun_mem_iff.mp (h.1.rel_cod hrel)).2
       have hfB : f.1.rel B Z' := f.1.mono hf hinr hZZ' hB hZ'
       exact h.1.mono hrel hinl (sstep_subset hB hZ' hinr hZZ') hA
         (sstep_mem_of_mem (g := f) hfB) },
   by
     rintro Z ⟨_, hrel⟩
     rw [show (smash V₀ V₁).master = prodNbhd V₀.master V₁.master from rfl,
       prodNbhd.inl_preimage, prodNbhd.inr_preimage] at hrel
     have huniv : sstep V₁.master Z = (Set.univ : Set (StrictMap V₁ V₂)) := h.2 hrel
     have hcb : (⟨constMap V₁ V₂.bot, isStrict_constBot⟩ : StrictMap V₁ V₂) ∈ sstep V₁.
    master Z := by
       rw [huniv]; exact Set.mem_univ _
     obtain ⟨_, hZ⟩ := hcb
     rwa [NeighborhoodSystem.mem_bot] at hZ⟩
```

```
theorem smashUncurry_curry (g : StrictMap (smash V₀ V₁) V₂) :
    smashUncurryMap (smashCurryMap g) = g := by
  apply Subtype.ext
  apply ApproximableMap.ext
  intro W Z
  constructor
  · rintro ⟨hW, hcurry⟩
    rcases hW with rfl | ⟨X, Y, hXp, hXne, hYp, hYne, rfl⟩
    · simp only [prodNbhd.inl_preimage, prodNbhd.inr_preimage] at hcurry
      obtain ⟨hXm, _, hsec⟩ := hcurry
      have hZ : Z = V₂.master := isStrict_smashSection g.2 hXm hsec
      rw [hZ]; exact g.1.master_rel
    · simp only [prodNbhd.inl_preimage, prodNbhd.inr_preimage] at hcurry
      obtain ⟨hX, _, hY, hmem⟩ := hcurry
      exact (smashPair_principal_apply g.1 hXp hXne hYp hYne).mp hmem
  · intro hrel
    have hW : (smash V₀ V₁).mem W := g.1.rel_dom hrel
    rcases hW with rfl | ⟨X, Y, hXp, hXne, hYp, hYne, rfl⟩
    · have hZ : Z = V₂.master := g.2 hrel
      subst hZ
      refine ⟨Or.inl rfl, ?_⟩
      simp only [prodNbhd.inl_preimage, prodNbhd.inr_preimage, sstep_master_eq]
      exact ⟨V₀.master_mem, (strictFun V₁ V₂).master_mem, Set.mem_univ _⟩
    · refine ⟨smash_mem_proper hXp hXne hYp hYne, ?_⟩
      simp only [prodNbhd.inl_preimage, prodNbhd.inr_preimage]
      have hsec : (smashSection g.1 hXp).rel Y Z :=
        ⟨hYp, (smashPair_principal_apply g.1 hXp hXne hYp hYne).mpr hrel⟩
      exact ⟨hXp, sstep_mem_of_mem
        (g := ⟨smashSection g.1 hXp, isStrict_smashSection g.2 hXp⟩) hsec, hsec⟩

theorem sstep_cod_master {Y : Set α} (hY : V₀.mem Y) :
    (sstep Y V₁.master : Set (StrictMap V₀ V₁)) = Set.univ := by
  ext f
  simp only [mem_sstep, Set.mem_univ, iff_true]
  exact f.1.mono f.1.master_rel (V₀.sub_master hY) subset_rfl hY V₁.master_mem

theorem section_uncurry_rel (h : StrictMap V₀ (strictFun V₁ V₂))
    {X : Set α} (hX : V₀.mem X) {Y : Set β} (hY : V₁.mem Y) {Z : Set ?} :
    (smashSection (smashUncurryMap h).1 hX).rel Y Z ↔ h.1.rel X (sstep Y Z) := by
  rw [smashSection_rel]
  by_cases hXm : X = V₀.master
  · subst hXm
    constructor
    · rintro ⟨hY', hmem⟩
      rw [principal_master_eq_bot hX rfl,
        show smashPair V₀.bot (V₁.principal hY') = (smash V₀ V₁).bot from
          smashPair_eq_bot_iff.mpr (Or.inl rfl),
        isStrict_iff_apply_bot.mp (smashUncurryMap h).2, NeighborhoodSystem.mem_bot]
    at hmem
      subst hmem
      rw [sstep_cod_master hY]
      exact h.1.rel_master hX
    · intro hrel
      have huniv : sstep Y Z = (Set.univ : Set (StrictMap V₁ V₂)) := h.2 hrel
      have hZ : Z = V₂.master := by
```

```
        have hcb : (⟨constMap V₁ V₂.bot, isStrict_constBot⟩ : StrictMap V₁ V₂) ∈ sstep Y
     Z := by
          rw [huniv]; exact Set.mem_univ _
        obtain ⟨_, hbz⟩ := hcb
        rwa [NeighborhoodSystem.mem_bot] at hbz
      subst hZ
      exact ⟨hY, ((smashUncurryMap h).1.toElementMap _).master_mem⟩
  · by_cases hYm : Y = V₁.master
    · subst hYm
      constructor
      · rintro ⟨hY', hmem⟩
        rw [principal_master_eq_bot hY' rfl,
          show smashPair (V₀.principal hX) V₁.bot = (smash V₀ V₁).bot from
            smashPair_eq_bot_iff.mpr (Or.inr rfl),
          isStrict_iff_apply_bot.mp (smashUncurryMap h).2, NeighborhoodSystem.mem_bot]
     at hmem
        subst hmem
        rw [sstep_master_eq]
        exact h.1.rel_master hX
      · intro hrel
        obtain ⟨f, hf⟩ := (strictFun_mem_iff.mp (h.1.rel_cod hrel)).2
        have hZ : Z = V₂.master := f.2 hf
        subst hZ
        exact ⟨hY, ((smashUncurryMap h).1.toElementMap _).master_mem⟩
    · constructor
      · rintro ⟨hY', hmem⟩
        obtain ⟨_, hrel⟩ :=
          (smashPair_principal_apply (smashUncurryMap h).1 hX hXm hY hYm).mp hmem
        simpa only [prodNbhd.inl_preimage, prodNbhd.inr_preimage] using hrel
      · intro hrel
        refine ⟨hY, (smashPair_principal_apply (smashUncurryMap h).1 hX hXm hY hYm).mpr
     ?_⟩
        refine ⟨smash_mem_proper hX hXm hY hYm, ?_⟩
        simpa only [prodNbhd.inl_preimage, prodNbhd.inr_preimage] using hrel

theorem smashCurry_uncurry (h : StrictMap V₀ (strictFun V₁ V₂)) :
    smashCurryMap (smashUncurryMap h) = h := by
  apply Subtype.ext
  apply ApproximableMap.ext
  intro X N
  constructor
  · rintro ⟨hX, hN, hmem⟩
    obtain ⟨⟨L, hL, rfl⟩, _⟩ := hN
    refine (rel_sstepFun_iff h.1 hX hL).mpr (fun p hp => ?_)
    exact (section_uncurry_rel h hX (hL p hp).1).mp (hmem p hp)
  · intro hrel
    have hX : V₀.mem X := h.1.rel_dom hrel
    have hN : (strictFun V₁ V₂).mem N := h.1.rel_cod hrel
    refine ⟨hX, hN, ?_⟩
    obtain ⟨⟨L, hL, rfl⟩, _⟩ := hN
    intro p hp
    exact (section_uncurry_rel h hX (hL p hp).1).mpr
      ((rel_sstepFun_iff h.1 hX hL).mp hrel p hp)

def smashCurryEquiv (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (V₂ : NeighborhoodSystem ?) :
```

```
    StrictMap (smash V₀ V₁) V₂ ≃o StrictMap V₀ (strictFun V₁ V₂) where
  toFun := smashCurryMap
  invFun := smashUncurryMap
  left_inv := smashUncurry_curry
  right_inv := smashCurry_uncurry
  map_rel_iff' := by
    intro g g'
    constructor
    · intro hcurry W Z hrel
      have h1 : (smashCurryMap g).1.rel (Sum.inl ^-1' W) (sstep (Sum.inr ^-1' W) Z) :=
     by
        have hu : (smashUncurryMap (smashCurryMap g)).1.rel W Z := by
          rw [smashUncurry_curry]; exact hrel
        exact hu.2
      have h2 := hcurry _ _ h1
      have hu' : (smashUncurryMap (smashCurryMap g')).1.rel W Z := ⟨g.1.rel_dom hrel,
    h2⟩
      rw [smashUncurry_curry] at hu'
      exact hu'
    · intro hg X N hrel
      obtain ⟨hX, hN, hmem⟩ := hrel
      refine ⟨hX, hN, ?_⟩
      obtain ⟨⟨L, hL, rfl⟩, _⟩ := hN
      intro p hp
      obtain ⟨hY, hmm⟩ := hmem p hp
      obtain ⟨W, hWmem, hWrel⟩ := hmm
      exact ⟨hY, W, hWmem, hg W p.2 hWrel⟩

variable {α β ? : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def strictFunEquiv (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    (strictFun V₀ V₁).Element ≃o StrictMap V₀ V₁ where
  toFun := toStrictMap
  invFun := toStrictFilter
  left_inv φ := by
    apply Element.ext
    intro W
    constructor
    · rintro ⟨hWmem, hfW⟩
      obtain ⟨⟨L, hL, rfl⟩, _⟩ := hWmem
      exact (mem_sstepFun_iff φ hL).mpr (fun p hp => hfW p hp)
    · intro hW
      refine ⟨φ.sub hW, ?_⟩
      obtain ⟨⟨L, hL, rfl⟩, _⟩ := φ.sub hW
      intro p hp
      exact (mem_sstepFun_iff φ hL).mp hW p hp
  right_inv f := by
    apply Subtype.ext
    apply ApproximableMap.ext
    intro X Y
    constructor
    · rintro ⟨_, hf⟩; exact hf
    · intro hf; exact ⟨sstep_mem_of_mem (g := f) hf, hf⟩
  map_rel_iff' := by
    intro φ φ'
```

```
    constructor
    · intro h W hW
      obtain ⟨⟨L, hL, rfl⟩, _⟩ := φ.sub hW
      refine (mem_sstepFun_iff φ' hL).mpr (fun p hp => ?_)
      exact h p.1 p.2 ((mem_sstepFun_iff φ hL).mp hW p hp)
    · intro h X Y hrel
      exact h _ hrel

theorem isStrict_idMap : IsStrict (idMap V₀) := by
  rintro Y ⟨_, hY, hsub⟩
  exact Set.Subset.antisymm (V₀.sub_master hY) hsub

variable {α β ? : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

theorem smash_mem_iff {W : Set (α ? β)} :
    (smash V₀ V₁).mem W ↔
      W = prodNbhd V₀.master V₁.master ∨ SmashProper V₀ V₁ W := Iff.rfl

end DomainTheory.NeighbourhoodSystems.SmashProduct
```

4.6.22 Exercise 5.10 — smash product

Suppose $\mathcal{D}_0$ and $\mathcal{D}_1$ are neighbourhood systems over disjoint sets Δ_0 and Δ_1. Define the *smash product* $\mathcal{D}_0 \otimes \mathcal{D}_1$ with neighbourhoods

$$\{\Delta_0 \cup \Delta_1\} \cup \{X \cup Y \mid X \in \mathcal{D}_0 \setminus \{\Delta_0\} \text{ and } Y \in \mathcal{D}_1 \setminus \{\Delta_1\}\}.$$

Show that this *is* a neighbourhood system. Define $(\mathcal{D}_0 \to_\perp \mathcal{D}_1)$ so that $|\mathcal{D}_0 \to_\perp \mathcal{D}_1|$ consists exactly of the *strict functions*. By introducing appropriate combinators, show that

$$(\mathcal{D}_0 \to_\perp (\mathcal{D}_1 \to_\perp \mathcal{D}_2)) \quad \text{and} \quad ((\mathcal{D}_0 \otimes \mathcal{D}_1) \to_\perp \mathcal{D}_2)$$

are isomorphic.

The development is with the smash product peer.

Theorem (`example_510`). Membership in the smash product is the joined master or a proper smash neighbourhood, and currying yields an order isomorphism between strict maps out of smash and strict maps into the strict function space.

Proof. The smash membership characterisation and the smash–curry equivalence from the smash-product peer. ■

`Exercise_5_10.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.smashProduct

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct
```

```
/-- Scott's Exercise 5.10: the smash product is a neighbourhood system; strict
    function
spaces are isomorphic under currying against smash. -/
theorem example_510 :
    (∀ {α β : Type*} (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
      {W : Set (α ? β)},
      (smash V₀ V₁).mem W ↔
        W = prodNbhd V₀.master V₁.master ∨ SmashProper V₀ V₁ W) ∧
    (∀ {α β ? : Type*} (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
      (V₂ : NeighborhoodSystem ?),
      Nonempty (StrictMap (smash V₀ V₁) V₂ ≃o StrictMap V₀ (strictFun V₁ V₂))) :=
  ⟨fun V₀ V₁ => smash_mem_iff (V₀ := V₀) (V₁ := V₁),
    fun V₀ V₁ V₂ => ⟨smashCurryEquiv V₀ V₁ V₂⟩⟩
```

4.6.23 Diagonal and map stacks

Diagonal and componentwise map combinators on infinite-product stacks, obtained by fixed-point recursion from the stack push.

Definition (`Arg`). The argument domain for the map combinator is the product of a stack of endomaps with the underlying domain.

Proof. By definition. ■

Definition (`diag`). The diagonal combinator is the fixed point of push, curried in the element being stacked.

Proof. By definition. ■

Definition (`mapBody`). The body of the map recursion pushes the head map applied to the element onto the recursive map of the tail.

Proof. By definition. ■

Definition (`mapOp`). The map operator is the curry of the map body.

Proof. By definition. ■

Definition (`mapMap`). The map combinator is the fixed point of the map operator, viewed as an approximable map.

Proof. By definition. ■

Theorem (`diag_rec`). The diagonal of an element equals the push of that element onto its own diagonal.

Proof. Unpack the fixed-point and curry evaluation laws for push. ■

Theorem (`mapBody_apply`). The map body at a functional and a stack–element pair pushes the head map on the element onto the functional applied to the tail stack with the same element.

Proof. Expand the body through projections, evaluation, and pairing. ■

Theorem (`map_rec`). Mapping a stack of maps on an element equals pushing the head application onto the map of the tail.

Proof. The fixed-point law for the map operator together with the map-body evaluation. ■

Theorem (`component_diag`). Every component of the diagonal of an element is that element.

Proof. Induct on the index, using the diagonal recursion and the head/tail laws for push. ■

Theorem (`component_map`). The nth component of mapping a stack of maps on an element is the nth map applied to that element.

Proof. Induct on the index, using the map recursion and the head/tail laws for push. ■

`diagMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.iterStack
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.DiagMap
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

variable {α : Type*} (V : NeighborhoodSystem α)

abbrev Arg (V : NeighborhoodSystem α) := prod (iterSys (funSpace V V)) V

def diag : ApproximableMap V (iterSys V) :=
  (fixMap (iterSys V)).comp (curry (push V))

def mapBody : ApproximableMap (prod (funSpace (Arg V) (iterSys V)) (Arg V)) (iterSys V
    ) :=
  (push V).comp
    (paired
      ((evalMap V V).comp
        (paired
          ((head (funSpace V V)).comp ((proj₀ (iterSys (funSpace V V)) V).comp
            (proj₁ (funSpace (Arg V) (iterSys V)) (Arg V))))
          ((proj₁ (iterSys (funSpace V V)) V).comp
            (proj₁ (funSpace (Arg V) (iterSys V)) (Arg V)))))
      ((evalMap (Arg V) (iterSys V)).comp
        (paired (proj₀ (funSpace (Arg V) (iterSys V)) (Arg V))
          (paired
            ((tail (funSpace V V)).comp ((proj₀ (iterSys (funSpace V V)) V).comp
              (proj₁ (funSpace (Arg V) (iterSys V)) (Arg V))))
            ((proj₁ (iterSys (funSpace V V)) V).comp
              (proj₁ (funSpace (Arg V) (iterSys V)) (Arg V)))))))

def mapOp : ApproximableMap (funSpace (Arg V) (iterSys V)) (funSpace (Arg V) (iterSys
    V)) :=
  curry (mapBody V)

def mapMap : ApproximableMap (Arg V) (iterSys V) := toApproxMap (mapOp V).fixElement

theorem diag_rec (x : V.Element) :
    (diag V).toElementMap x =
      (push V).toElementMap (pair x ((diag V).toElementMap x)) := by
  rw [diag, toElementMap_comp]
  set φ := (curry (push V)).toElementMap x with hφ
  have hfix := fixMap_fixed (iterSys V) φ
  have hval := toElementMap_curry_apply (push V) x ((fixMap (iterSys V)).toElementMap
    φ)
  rw [← hφ] at hval
  rw [hval] at hfix
  exact hfix.symm
```

```
theorem mapBody_apply (W : (funSpace (Arg V) (iterSys V)).Element)
    (s : (iterSys (funSpace V V)).Element) (x : V.Element) :
    (mapBody V).toElementMap (pair W (pair s x)) =
      (push V).toElementMap
        (pair ((toApproxMap ((head (funSpace V V)).toElementMap s)).toElementMap x)
          ((toApproxMap W).toElementMap (pair ((tail (funSpace V V)).toElementMap s) x
   ))) := by
  rw [mapBody, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj₀,
    toElementMap_proj₁,
    fst_pair, snd_pair, evalMap_apply]

theorem map_rec (s : (iterSys (funSpace V V)).Element) (x : V.Element) :
    (mapMap V).toElementMap (pair s x) =
      (push V).toElementMap
        (pair ((toApproxMap ((head (funSpace V V)).toElementMap s)).toElementMap x)
          ((mapMap V).toElementMap (pair ((tail (funSpace V V)).toElementMap s) x)))
    := by
  have hfix : (mapOp V).toElementMap (mapOp V).fixElement = (mapOp V).fixElement :=
    toElementMap_fixElement (mapOp V)
  have hval := toElementMap_curry_apply (mapBody V) (mapOp V).fixElement (pair s x)
  rw [← mapOp, hfix] at hval
  rw [mapMap, hval, mapBody_apply]

theorem component_diag (x : V.Element) (n : ℕ) :
    component ((diag V).toElementMap x) n = x := by
  induction n with
  | zero =>
    rw [← head_apply, diag_rec, head_push]
  | succ k ih =>
    have ht : (tail V).toElementMap ((diag V).toElementMap x) = (diag V).toElementMap
    x := by
      conv_lhs => rw [diag_rec]
      rw [tail_push]
    rw [← component_tail, ht, ih]

theorem component_map (x : V.Element) :
    ∀ (n : ℕ) (s : (iterSys (funSpace V V)).Element),
      component ((mapMap V).toElementMap (pair s x)) n =
        (toApproxMap (component s n)).toElementMap x := by
  intro n
  induction n with
  | zero =>
    intro s
    rw [← head_apply, map_rec, head_push, head_apply]
  | succ k ih =>
    intro s
    have ht : (tail V).toElementMap ((mapMap V).toElementMap (pair s x)) =
        (mapMap V).toElementMap (pair ((tail (funSpace V V)).toElementMap s) x) := by
      conv_lhs => rw [map_rec]
      rw [tail_push]
    rw [← component_tail, ht, ih, component_tail]

end DomainTheory.NeighbourhoodSystems.DiagMap
```

4.6.24 Exercise 5.11 — stack combinators

For any domain $\mathcal{D}$ we may regard $\mathcal{D}^\infty$ as consisting of (bottomless) *stacks* of elements of $\mathcal{D}$. With this image in mind, define appropriate combinators with the obvious meanings:

$$\{\mathrm{head} : \mathcal{D}^\infty \to \mathcal{D}\}; \quad \{\mathrm{tail} : \mathcal{D}^\infty \to \mathcal{D}^\infty\}; \quad \{\mathrm{push} : \mathcal{D} \times \mathcal{D}^\infty \to \mathcal{D}^\infty\}.$$

Using the fixed-point theorem argue that there is a combinator

$$\{\mathrm{diag} : \mathcal{D} \to \mathcal{D}^\infty\}$$

where for all $x \in |\mathcal{D}|$ we have

$$\mathrm{diag}(x) = \langle x \rangle_{n=0}^{\infty}.$$

(Hint: Try a recursive definition, say

$$\mathrm{diag}(x) = \mathrm{push}(x, \mathrm{diag}(x)),$$

but be sure to prove *all* terms of $\mathrm{diag}(x)$ equal x.) Also introduce by an appropriate recursion a combinator

$$\{\mathrm{map} : (D \to D)^\infty \times D \to D^\infty\}$$

where for elements of the suitable types:

$$\mathrm{map}(\langle f_n \rangle_{n=0}^{\infty}, x) = \langle f_n(x) \rangle_{n=0}^{\infty}.$$

The stack combinators are with the infinite-product stack peer; the diagonal and map recursion are with the diagonal and map stacks peer.

Theorem (`example_511`). Every component of the diagonal of an element is that element, and every component of mapping a stack of maps on an element is the corresponding map applied to that element.

Proof. The component laws for the diagonal and map combinators from the diagonal-and-map peer. ■

`Exercise_5_11.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.diagMap

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DiagMap

/-- Scott's Exercise 5.11: diagonal stack of a constant element, and componentwise
application of a stack of maps (stack combinators live in the infinite-product peer).
    -/
theorem example_511 :
    (∀ {α : Type*} (V : NeighborhoodSystem α) (x : V.Element) (n : ℕ),
      component ((diag V).toElementMap x) n = x) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (x : V.Element) (n : ℕ)
```

```
      (s : (iterSys (funSpace V V)).Element),
      component ((mapMap V).toElementMap (pair s x)) n =
        (toApproxMap (component s n)).toElementMap x) :=
  ⟨@component_diag, fun V x n s => component_map V x n s⟩
```

4.6.25 While combinator

The while combinator as the least fixed point of a conditional body that either recurses after applying the step map or returns the current element.

Definition (`bodyMap`). The while body conditions on the predicate at the current element, pairing the recursive application after the step with the current element.

Proof. By definition. ■

Definition (`Wop`). The while operator is the curry of the while body.

Proof. By definition. ■

Definition (`whileFix`). The while fixed point is the least fixed point of the while operator.

Proof. By definition. ■

Definition (`whileMap`). The while combinator is that fixed point viewed as an approximable map.

Proof. By definition. ■

Theorem (`bodyMap_apply`). The while body at a functional and an element conditions on the predicate, pairing the functional on the step with the element.

Proof. Expand through projections, evaluation, and pairing. ■

Theorem (`Wop_apply`). Applying the while operator at a functional recovers the conditional body.

Proof. Curry evaluation with the body law. ■

Theorem (`whileMap_least`). Any functional satisfying the while recursion dominates the while fixed point.

Proof. The least-fixed-point characterisation of the while operator. ■

Theorem (`whileMap_rec`). While satisfies the conditional recursion at every element.

Proof. The fixed-point law for the while operator. ■

Theorem (`whileMap_bot`). When the predicate is bottom, while returns bottom.

Proof. The recursion together with the bottom case of conditional. ■

Theorem (`whileMap_false`). When the predicate is false, while returns the current element.

Proof. The recursion together with the false case of conditional. ■

Theorem (`whileMap_true`). When the predicate is true, while recurses after the step map.

Proof. The recursion together with the true case of conditional. ■

`whileMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.cond
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.WhileMap
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
```

```
open DomainTheory.NeighbourhoodSystems.Cond (cond TD)

variable {α : Type*} (V : NeighborhoodSystem α)
  (p : ApproximableMap V TD) (f : ApproximableMap V V)

def bodyMap : ApproximableMap (prod (funSpace V V) V) V :=
  (cond V).comp
    (paired (p.comp (proj₁ (funSpace V V) V))
      (paired
        ((evalMap V V).comp
          (paired (proj₀ (funSpace V V) V) (f.comp (proj₁ (funSpace V V) V))))
        (proj₁ (funSpace V V) V)))

def Wop : ApproximableMap (funSpace V V) (funSpace V V) := curry (bodyMap V p f)

def whileFix : (funSpace V V).Element := (Wop V p f).fixElement

def whileMap : ApproximableMap V V := toApproxMap (whileFix V p f)

theorem bodyMap_apply (w : (funSpace V V).Element) (x : V.Element) :
    (bodyMap V p f).toElementMap (pair w x) =
      (cond V).toElementMap
        (pair (p.toElementMap x)
          (pair ((toApproxMap w).toElementMap (f.toElementMap x)) x)) := by
  rw [bodyMap, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj₀,
    toElementMap_proj₁,
    fst_pair, snd_pair, evalMap_apply]

theorem Wop_apply (w : (funSpace V V).Element) (x : V.Element) :
    (toApproxMap ((Wop V p f).toElementMap w)).toElementMap x =
      (cond V).toElementMap
        (pair (p.toElementMap x)
          (pair ((toApproxMap w).toElementMap (f.toElementMap x)) x)) := by
  rw [Wop, toElementMap_curry_apply, bodyMap_apply]

theorem whileMap_least (w' : (funSpace V V).Element)
    (hw' : ∀ x : V.Element, (toApproxMap w').toElementMap x =
      (cond V).toElementMap
        (pair (p.toElementMap x)
          (pair ((toApproxMap w').toElementMap (f.toElementMap x)) x))) :
    whileFix V p f ≤ w' := by
  apply fixElement_le_of_toElementMap_le
  apply le_of_eq
  apply (funSpaceEquiv V V).injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply]
  apply ext_of_toElementMap
  intro x
  rw [Wop_apply]
  exact (hw' x).symm

theorem whileMap_rec (x : V.Element) :
    (whileMap V p f).toElementMap x =
      (cond V).toElementMap
        (pair (p.toElementMap x)
          (pair ((whileMap V p f).toElementMap (f.toElementMap x)) x)) := by
```

```
  have hfix : (Wop V p f).toElementMap (whileFix V p f) = whileFix V p f :=
    toElementMap_fixElement (Wop V p f)
  have := Wop_apply V p f (whileFix V p f) x
  rw [hfix] at this
  exact this

theorem whileMap_bot {x : V.Element} (hx : p.toElementMap x = Truth.botElt) :
    (whileMap V p f).toElementMap x = V.bot := by
  rw [whileMap_rec, hx, Cond.cond_bot]

theorem whileMap_false {x : V.Element} (hx : p.toElementMap x = Truth.falseElt) :
    (whileMap V p f).toElementMap x = x := by
  rw [whileMap_rec, hx, Cond.cond_false]

theorem whileMap_true {x : V.Element} (hx : p.toElementMap x = Truth.trueElt) :
    (whileMap V p f).toElementMap x = (whileMap V p f).toElementMap (f.toElementMap x)
     := by
  rw [whileMap_rec, hx, Cond.cond_true]

end DomainTheory.NeighbourhoodSystems.WhileMap
```

4.6.26 Exercise 5.12 — while combinator

On any domain D introduce (as a least fixed point) a combinator

$$\mathrm{while} : (D \to T) \times (D \to D) \to (D \to D)$$

by the recursion

$$\mathrm{while}(p, f)(x) = \mathrm{cond}(p(x), \mathrm{while}(p, f)(f(x)), x).$$

Prove that

$$\mathrm{while}(p, \mathrm{while}(p, f)) = \mathrm{while}(p, f).$$

Show how while could have been used to obtain the least number operator mentioned in the proof of 5.6. Generalize the idea to define a combinator

$$\{\mathrm{find} : D^{\infty} \times (D \to T) \to D\}$$

with the meaning "find the first term of the sequence (if any) which satisfies the given precicate."

The development is with the while combinator peer.

Theorem (`example_512`). The while combinator satisfies the conditional recursion, and the three truth-value cases return bottom, the current element, or a recursive call after the step, respectively.

Proof. The while recursion and case laws from the while-combinator peer. ■

`Exercise_5_12.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.whileMap
```

```
@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.WhileMap
open DomainTheory.NeighbourhoodSystems.Cond (cond TD)
open Truth

/-- Scott's Exercise 5.12: the while combinator as a least fixed point satisfying the
conditional recursion, with the three truth-value cases. -/
theorem example_512 :
    (∀ {α : Type*} (V : NeighborhoodSystem α) (p : ApproximableMap V TD)
      (f : ApproximableMap V V) (x : V.Element),
      (whileMap V p f).toElementMap x =
        (cond V).toElementMap
          (pair (p.toElementMap x)
            (pair ((whileMap V p f).toElementMap (f.toElementMap x)) x))) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (p : ApproximableMap V TD)
      (f : ApproximableMap V V) {x : V.Element},
      p.toElementMap x = Truth.botElt →
        (whileMap V p f).toElementMap x = V.bot) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (p : ApproximableMap V TD)
      (f : ApproximableMap V V) {x : V.Element},
      p.toElementMap x = Truth.falseElt →
        (whileMap V p f).toElementMap x = x) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (p : ApproximableMap V TD)
      (f : ApproximableMap V V) {x : V.Element},
      p.toElementMap x = Truth.trueElt →
        (whileMap V p f).toElementMap x =
          (whileMap V p f).toElementMap (f.toElementMap x)) :=
  ⟨@whileMap_rec, @whileMap_bot, @whileMap_false, @whileMap_true⟩
```

4.6.27 Natural-number pairing

Cantor pairing of natural numbers via triangular numbers, with the diagonal-step recurrence and a two-sided inverse.

Definition (`nextCell`). The next cell on the antidiagonal enumeration steps left-and-up, or wraps to the next antidiagonal.

Proof. By definition. ■

Definition (`tri`). The kth triangular number is $k(k+1)/2$.

Proof. By definition. ■

Definition (`num`). The pairing of n and m is the triangular number of their sum plus m.

Proof. By definition. ■

Definition (`numP`). Pairing on a product is pairing of the two components.

Proof. By definition. ■

Theorem (`num_succ_right`). Incrementing the second argument equals stepping left on the antidiagonal and adding one.

Proof. Compare triangular numbers after rewriting the sum. ■

Theorem (`num_zero_zero`). The origin pairs to zero.

Proof. Immediate from the definitions. ■

Theorem (`two_dvd_mul_succ`). Twice divides the product of a number and its successor.

Proof. Induction on the number. ■

Theorem (`tri_mul_two`). Twice a triangular number recovers the product of the index with its successor.

Proof. Cancel the division using the previous divisibility. ■

Theorem (`tri_succ`). The next triangular number adds the next index.

Proof. Compare doubles of both sides. ■

Theorem (`num_lt_tri_succ`). A pairing lies strictly below the next triangular number after its antidiagonal.

Proof. Expand via the triangular successor law. ■

Theorem (`num_succ_left`). Starting a new antidiagonal equals wrapping from the previous row and adding one.

Proof. Expand via the triangular successor law. ■

Theorem (`numP_nextCell`). Stepping to the next cell increments the pairing value by one.

Proof. Case on the first coordinate and apply the two successor laws. ■

Theorem (`tri_le_succ`). Triangular numbers are monotone in the successor.

Proof. The triangular successor law. ■

Theorem (`tri_mono`). Triangular numbers are monotone.

Proof. Induction along the inequality. ■

Theorem (`num_injective`). The pairing on products is injective.

Proof. Equal pairings force equal antidiagonals by the triangular bounds, then equal second coordinates. ■

Definition (`unnum`). The inverse enumeration walks the next-cell map from the origin.

Proof. By definition. ■

Theorem (`numP_unnum`). Pairing after unpairing recovers the number.

Proof. Induction, using the next-cell increment. ■

Theorem (`unnum_numP`). Unpairing after pairing recovers the cell.

Proof. Injectivity of pairing with the previous round-trip. ■

natPair.lean

```
module

public import Aesop.BuiltinRules
public import Mathlib.Algebra.Ring.Parity
public import Mathlib.Data.Nat.Basic
public import Mathlib.Tactic.NormNum
public import Mathlib.Tactic.Ring
public import Mathlib.Tactic.ToDual
public import Mathlib.Tactic.ToAdditive

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.NatPair

def nextCell : ℕ × ℕ → ℕ × ℕ
  | (n + 1, m) => (n, m + 1)
  | (0, m) => (m + 1, 0)

def tri (k : ℕ) : ℕ := k * (k + 1) / 2
```

```
def num (n m : ℕ) : ℕ := tri (n + m) + m

def numP (p : ℕ × ℕ) : ℕ := num p.1 p.2

theorem num_succ_right (n m : ℕ) : num n (m + 1) = num (n + 1) m + 1 := by
  unfold num
  have h : n + (m + 1) = (n + 1) + m := by omega
  rw [h]; omega

theorem num_zero_zero : num 0 0 = 0 := rfl

theorem two_dvd_mul_succ (k : ℕ) : 2 ∣ k * (k + 1) := by
  induction k with
  | zero => exact ⟨0, by ring⟩
  | succ n ih =>
      obtain ⟨c, hc⟩ := ih
      refine ⟨c + (n + 1), ?_⟩
      have hexp : (n + 1) * (n + 1 + 1) = n * (n + 1) + 2 * (n + 1) := by ring
      rw [hexp, hc]; ring

theorem tri_mul_two (k : ℕ) : tri k * 2 = k * (k + 1) := by
  unfold tri
  exact Nat.div_mul_cancel (two_dvd_mul_succ k)

theorem tri_succ (k : ℕ) : tri (k + 1) = tri k + (k + 1) := by
  have e1 : tri k * 2 = k * (k + 1) := tri_mul_two k
  have e2 : tri (k + 1) * 2 = (k + 1) * (k + 1 + 1) := tri_mul_two (k + 1)
  have key : tri (k + 1) * 2 = (tri k + (k + 1)) * 2 := by
    have hexp : (tri k + (k + 1)) * 2 = tri k * 2 + (k + 1) * 2 := by ring
    rw [hexp, e1, e2]; ring
  exact Nat.eq_of_mul_eq_mul_right (by norm_num) key

theorem num_lt_tri_succ (n m : ℕ) : num n m < tri (n + m + 1) := by
  rw [tri_succ]; unfold num; omega

theorem num_succ_left (n : ℕ) : num (n + 1) 0 = num 0 n + 1 := by
  unfold num
  have h0 : (n + 1) + 0 = n + 1 := by omega
  have h1 : 0 + n = n := by omega
  rw [h0, h1, tri_succ]; omega

theorem numP_nextCell (c : ℕ × ℕ) : numP (nextCell c) = numP c + 1 := by
  obtain ⟨n, m⟩ := c
  cases n with
  | zero => exact num_succ_left m
  | succ k => exact num_succ_right k m

theorem tri_le_succ (k : ℕ) : tri k ≤ tri (k + 1) := by
  rw [tri_succ]; omega

theorem tri_mono {a b : ℕ} (h : a ≤ b) : tri a ≤ tri b := by
  induction h with
  | refl => exact le_refl _
  | step _ ih => exact le_trans ih (tri_le_succ _)

theorem num_injective : Function.Injective numP := by
```

```
  rintro ⟨n₁, m₁⟩ ⟨n₂, m₂⟩ h
  have h' : num n₁ m₁ = num n₂ m₂ := h
  have hs : n₁ + m₁ = n₂ + m₂ := by
    rcases lt_trichotomy (n₁ + m₁) (n₂ + m₂) with hlt | heq | hgt
    · exfalso
      have h1 : num n₁ m₁ < tri (n₁ + m₁ + 1) := num_lt_tri_succ n₁ m₁
      have h2 : tri (n₁ + m₁ + 1) ≤ tri (n₂ + m₂) := tri_mono (by omega)
      have h3 : tri (n₂ + m₂) ≤ num n₂ m₂ := by unfold num; omega
      omega
    · exact heq
    · exfalso
      have h1 : num n₂ m₂ < tri (n₂ + m₂ + 1) := num_lt_tri_succ n₂ m₂
      have h2 : tri (n₂ + m₂ + 1) ≤ tri (n₁ + m₁) := tri_mono (by omega)
      have h3 : tri (n₁ + m₁) ≤ num n₁ m₁ := by unfold num; omega
      omega
  have hbase : tri (n₁ + m₁) + m₁ = tri (n₂ + m₂) + m₂ := h'
  rw [hs] at hbase
  have hm : m₁ = m₂ := by omega
  have hn : n₁ = n₂ := by omega
  subst hm; subst hn; rfl

def unnum : ℕ → ℕ × ℕ
  | 0 => (0, 0)
  | v + 1 => nextCell (unnum v)

theorem numP_unnum (v : ℕ) : numP (unnum v) = v := by
  induction v with
  | zero => exact num_zero_zero
  | succ k ih =>
      show numP (nextCell (unnum k)) = k + 1
      rw [numP_nextCell, ih]

theorem unnum_numP (c : ℕ × ℕ) : unnum (numP c) = c :=
  num_injective (by rw [numP_unnum])

end DomainTheory.NeighbourhoodSystems.NatPair
```

4.6.28 Exercise 5.13 — pairing function on numbers

Prove the existence of a one-one function $\mathrm{num} : \mathbf{N} \times \mathbf{N} \to \mathbf{N}$ such that

$$\mathrm{num}(0,0) = 0;$$

$$\mathrm{num}(n, m+1) = \mathrm{num}(n+1, m) + 1;$$

$$\mathrm{num}(n+1, 0) = \mathrm{num}(0, n) + 1.$$

Draw a picture (i.e. an infinite matrix) for the function and find a closed form for its values, if possible. Use the function to prove the isomorphism of the domains

$$P\mathbf{N},\ P(\mathbf{N} \times \mathbf{N}),\ P\mathbf{N} \times P\mathbf{N}.$$

The pairing development is with the natural-number pairing peer; the powerset isomorphisms continue in Exercise 5.13 (powerset).

Theorem (`example_513`). The Cantor pairing sends the origin to zero, obeys the two antidiagonal successor laws, and is inverted by the next-cell enumeration.

Proof. The zero, successor, and inverse laws from the natural-number pairing peer. ■

Exercise_5_13.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.natPair

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NatPair

/-- Scott's Exercise 5.13: the Cantor pairing `num` with the three recurrence laws and
a two-sided inverse `unnum`. -/
theorem example_513 :
    num 0 0 = 0 ∧
    (∀ n m : ℕ, num n (m + 1) = num (n + 1) m + 1) ∧
    (∀ n : ℕ, num (n + 1) 0 = num 0 n + 1) ∧
    (∀ v : ℕ, numP (unnum v) = v) ∧
    (∀ c : ℕ × ℕ, unnum (numP c) = c) :=
  ⟨num_zero_zero, num_succ_right, num_succ_left, numP_unnum, unnum_numP⟩
```

4.6.29 Powerset isomorphisms

Order isomorphisms among power sets of the naturals, of pairs of naturals, and of the product of two power sets, induced by the Cantor pairing and the sum–product equivalence.

Definition (`numEquiv`). The Cantor pairing and its inverse form an equivalence between pairs of naturals and naturals.

Proof. The two-sided inverse laws for the pairing. ■

Theorem (`numEquiv_apply`). The equivalence on a pair is the pairing of its components.

Proof. By definition. ■

Definition (`setCongr`). An equivalence of types induces an order isomorphism of power sets by direct image.

Proof. Direct and inverse images are mutual inverses; inclusion is preserved both ways by injectivity. ■

Definition (`PN_orderIso_PNN`). Power sets of naturals are order-isomorphic to power sets of pairs via the inverse pairing.

Proof. Apply the set congruence to the symmetric pairing equivalence. ■

Definition (`PN_orderIso_prod`). Power sets of naturals are order-isomorphic to a product of two power sets via the sum–natural equivalence.

Proof. Compose the set congruence of the sum equivalence with the sum–product isomorphism of sets. ■

Definition (`PNN_orderIso_prod`). Power sets of pairs are order-isomorphic to a product of two power sets.

Proof. Transport the previous isomorphism along the pairing isomorphism. ■

powersetIso.lean

```
module

public import Mathlib.Logic.Equiv.Nat
public import Mathlib.Algebra.Group.End

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.natPair

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.PowersetIso
open NatPair

def numEquiv : ℕ × ℕ ≃ ℕ where
  toFun := numP
  invFun := unnum
  left_inv := unnum_numP
  right_inv := numP_unnum

@[simp] theorem numEquiv_apply (p : ℕ × ℕ) : numEquiv p = num p.1 p.2 := rfl

variable {α β : Type*}

def setCongr (e : α ≃ β) : Set α ≃o Set β where
  toFun S := e '' S
  invFun T := e.symm '' T
  left_inv := e.symm_image_image
  right_inv := e.symm.symm_image_image
  map_rel_iff' := by
    intro a b
    constructor
    · intro h x hx
      obtain ⟨y, hy, hey⟩ := h ⟨x, hx, rfl⟩
      rwa [e.injective hey] at hy
    · rintro h _ ⟨x, hx, rfl⟩
      exact ⟨x, h hx, rfl⟩

def PN_orderIso_PNN : Set ℕ ≃o Set (ℕ × ℕ) := setCongr numEquiv.symm

def PN_orderIso_prod : Set ℕ ≃o Set ℕ × Set ℕ :=
  (setCongr Equiv.natSumNatEquivNat.symm).trans Set.sumEquiv

def PNN_orderIso_prod : Set (ℕ × ℕ) ≃o Set ℕ × Set ℕ :=
  PN_orderIso_PNN.symm.trans PN_orderIso_prod

end DomainTheory.NeighbourhoodSystems.PowersetIso
```

4.6.30 Exercise 5.13 — powerset isomorphisms

Prove the existence of a one-one function $\mathrm{num} : \mathbf{N} \times \mathbf{N} \to \mathbf{N}$ such that

$$\mathrm{num}(0,0) = 0;$$

$$\mathrm{num}(n, m+1) = \mathrm{num}(n+1, m) + 1;$$

$$\mathrm{num}(n+1, 0) = \mathrm{num}(0, n) + 1.$$

Draw a picture (i.e. an infinite matrix) for the function and find a closed form for its values, if possible. Use the function to prove the isomorphism of the domains

$$P\mathbf{N},\ P(\mathbf{N} \times \mathbf{N}),\ P\mathbf{N} \times P\mathbf{N}.$$

The pairing is with the natural-number pairing peer; the powerset isomorphisms are with the powerset isomorphisms peer.

Theorem (`example_513b`). There are order isomorphisms among the power set of the naturals, the power set of pairs of naturals, and the product of two power sets of naturals.

Proof. The three order isomorphisms from the powerset-isomorphisms peer. ■

`Exercise_5_13b.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.powersetIso

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open PowersetIso

/-- Scott's Exercise 5.13 (powerset clause): order isomorphisms among
`Set ℕ`, `Set (ℕ × ℕ)`, and `Set ℕ × Set ℕ`. -/
theorem example_513b :
    Nonempty (Set ℕ ≃o Set (ℕ × ℕ)) ∧
    Nonempty (Set ℕ ≃o Set ℕ × Set ℕ) ∧
    Nonempty (Set (ℕ × ℕ) ≃o Set ℕ × Set ℕ) :=
  ⟨⟨PN_orderIso_PNN⟩, ⟨PN_orderIso_prod⟩, ⟨PNN_orderIso_prod⟩⟩
```

4.6.31 Set graphs

Tagging finite sequences by iterated Cantor pairing, and the Fun/Graph correspondence between subsets of naturals and approximable set-maps.

Definition (`tag`). Tagging a list against a payload iterates the Cantor pairing with successors on the list entries and zero on the empty list.

Proof. By definition. ■

Theorem (`tag_nil`). Tagging the empty list is pairing zero with the payload.

Proof. By definition. ■

Theorem (`tag_cons`). Tagging a cons cell pairs the successor of the head with the tag of the tail.

Proof. By definition. ■

Definition (`entries`). The entry set of a list is the set of its members.

Proof. By definition. ■

Theorem (`mem_entries`). Membership in the entry set is list membership.

Proof. By definition. ■

Definition (`Fun`). The function of a code set sends an input set to those payloads whose tagged finite witnesses lie in the code and whose entries lie in the input.

Proof. By definition. ■

Definition (`Graph`). The graph of a set-map collects tags of finite witnesses whose payload lies in the map of their entry set.

Proof. By definition. ■

Definition (`IsApprox`). A set-map is approximable when it is monotone and every output has a finite witness from the input.

Proof. By definition. ■

Theorem (`Fun_isApprox`). The function of any code is approximable.

Proof. Monotonicity uses input enlargement; finite witnesses are the same lists used in the membership. ■

Theorem (`Fun_mono`). Enlarging the code or the input enlarges the function value.

Proof. Transport the finite witness. ■

Theorem (`num_succ_left_gt`). A second component is strictly below a pairing whose first component is a successor.

Proof. Expand via the triangular successor. ■

Theorem (`tag_injective`). Equal tags force equal lists and equal payloads.

Proof. Induction on the first list, using injectivity of pairing. ■

Theorem (`Fun_Graph`). For an approximable set-map, the function of its graph recovers the map.

Proof. One inclusion uses injectivity of tags and monotonicity; the other uses the finite-witness property. ■

Theorem (`tag_surjective`). Every natural is a tag of some list and payload.

Proof. Strong induction via unpairing and the strict decrease of the second component on successors. ■

Theorem (`id_le_Graph_Fun`). Every code is contained in the graph of its function.

Proof. Surjectivity of tags supplies a witness whose membership is preserved. ■

`setGraph.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.natPair

public import Mathlib.Data.Set.Defs

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.SetGraph
open DomainTheory.NeighbourhoodSystems.NatPair

def tag : List ℕ → ℕ → ℕ
  | [], m => num 0 m
  | (n :: ns), m => num (n + 1) (tag ns m)

@[simp] theorem tag_nil (m : ℕ) : tag [] m = num 0 m := rfl
```

```
@[simp] theorem tag_cons (n : ℕ) (ns : List ℕ) (m : ℕ) :
    tag (n :: ns) m = num (n + 1) (tag ns m) := rfl

def entries (ns : List ℕ) : Set ℕ := {n | n ∈ ns}

@[simp] theorem mem_entries {n : ℕ} {ns : List ℕ} : n ∈ entries ns ↔ n ∈ ns := Iff.
    rfl

def Fun (u : Set ℕ) (x : Set ℕ) : Set ℕ :=
  {m | ∃ ns, (∀ n ∈ ns, n ∈ x) ∧ tag ns m ∈ u}

def Graph (f : Set ℕ → Set ℕ) : Set ℕ :=
  {c | ∃ ns m, c = tag ns m ∧ m ∈ f (entries ns)}

def IsApprox (f : Set ℕ → Set ℕ) : Prop :=
  (∀ {{x x' : Set ℕ}}, x ⊆ x' → f x ⊆ f x') ∧
    ∀ x m, m ∈ f x → ∃ ns, (∀ n ∈ ns, n ∈ x) ∧ m ∈ f (entries ns)

theorem Fun_isApprox (u : Set ℕ) : IsApprox (Fun u) := by
  refine ⟨?_, ?_⟩
  · intro x x' hxx' m hm
    obtain ⟨ns, hsub, htag⟩ := hm
    exact ⟨ns, fun n hn => hxx' (hsub n hn), htag⟩
  · intro x m hm
    obtain ⟨ns, hsub, htag⟩ := hm
    exact ⟨ns, hsub, ns, fun n hn => hn, htag⟩

theorem Fun_mono {u u' x x' : Set ℕ} (hu : u ⊆ u') (hx : x ⊆ x') : Fun u x ⊆ Fun u' x
    ' := by
  rintro m ⟨ns, hns, htag⟩
  exact ⟨ns, fun n hn => hx (hns n hn), hu htag⟩

theorem num_succ_left_gt (n b : ℕ) : b < num (n + 1) b := by
  unfold num
  have e : n + 1 + b = (n + b) + 1 := by omega
  rw [e, tri_succ]
  omega

theorem tag_injective : ∀ {ns₁ ns₂ : List ℕ} {m₁ m₂ : ℕ},
    tag ns₁ m₁ = tag ns₂ m₂ → ns₁ = ns₂ ∧ m₁ = m₂ := by
  intro ns₁
  induction ns₁ with
  | nil =>
      intro ns₂ m₁ m₂ h
      cases ns₂ with
      | nil =>
          have hp := num_injective (show numP (0, m₁) = numP (0, m₂) from h)
          injection hp with _ hm
          exact ⟨rfl, hm⟩
      | cons n₂ ns₂' =>
          exfalso
          have hp := num_injective (show numP (0, m₁) = numP (n₂ + 1, tag ns₂' m₂) from
    h)
          injection hp with h1 _
          omega
```

```
  | cons n₁ ns₁' ih =>
      intro ns₂ m₁ m₂ h
      cases ns₂ with
      | nil =>
          exfalso
          have hp := num_injective (show numP (n₁ + 1, tag ns₁' m₁) = numP (0, m₂) from
    h)
          injection hp with h1 _
          omega
      | cons n₂ ns₂' =>
          have hp := num_injective
            (show numP (n₁ + 1, tag ns₁' m₁) = numP (n₂ + 1, tag ns₂' m₂) from h)
          injection hp with h1 h2
          have hn : n₁ = n₂ := by omega
          obtain ⟨hns, hm⟩ := ih h2
          exact ⟨by rw [hn, hns], hm⟩

theorem Fun_Graph {f : Set ℕ → Set ℕ} (hf : IsApprox f) (x : Set ℕ) :
    Fun (Graph f) x = f x := by
  apply Set.Subset.antisymm
  · intro m hm
    obtain ⟨ns, hsub, htag⟩ := hm
    obtain ⟨ns', m', heq, hmem⟩ := htag
    obtain ⟨hns, hmm⟩ := tag_injective heq
    subst hns; subst hmm
    have hsubset : entries ns ⊆ x := fun n hn => hsub n hn
    exact hf.1 hsubset hmem
  · intro m hm
    obtain ⟨ns, hsub, hmem⟩ := hf.2 x m hm
    exact ⟨ns, hsub, ns, m, rfl, hmem⟩

theorem tag_surjective (c : ℕ) : ∃ ns m, tag ns m = c := by
  induction c using Nat.strong_induction_on with
  | _ c ih =>
      obtain ⟨a, b, hab⟩ : ∃ a b, num a b = c :=
        ⟨(unnum c).1, (unnum c).2, numP_unnum c⟩
      cases a with
      | zero => exact ⟨[], b, hab⟩
      | succ k =>
          have hlt : b < c := by rw [← hab]; exact num_succ_left_gt k b
          obtain ⟨ns, m, hns⟩ := ih b hlt
          exact ⟨k :: ns, m, by rw [tag_cons, hns]; exact hab⟩

theorem id_le_Graph_Fun (u : Set ℕ) : u ⊆ Graph (Fun u) := by
  intro c hc
  obtain ⟨ns, m, hceq⟩ := tag_surjective c
  refine ⟨ns, m, hceq.symm, ?_⟩
  exact ⟨ns, fun n hn => hn, by rw [hceq]; exact hc⟩

end DomainTheory.NeighbourhoodSystems.SetGraph
```

4.6.32 Exercise 5.14 — functions as graphs

Show that there are approximable mappings

$$\mathrm{graph} : (P\mathbf{N} \to P\mathbf{N}) \to P\mathbf{N} \quad \text{and} \quad \mathrm{fun} : P\mathbf{N} \to (P\mathbf{N} \to P\mathbf{N}),$$

where we have

$$\mathrm{fun} \circ \mathrm{graph} = \lambda f.\, f, \quad \text{and} \quad \mathrm{graph} \circ \mathrm{fun} \supseteq \lambda x.\, x.$$

(Hint: Using the notation

$$[n_0, n_1, \ldots, n_k] = \mathrm{num}(n_0, [n_1, \ldots, n_k])$$

two such combinators can be given by formulae

$$\mathrm{fun}(u)(x) = \{m \mid \exists n_0, \ldots, n_{k-1} \in x.\, [n_0 + 1, \ldots, n_{k-1} + 1, 0, m] \in u\}$$

$$\mathrm{graph}(f) = \{[n_0 + 1, \ldots, n_{k-1} + 1, 0, m] \mid m \in f(\{n_0, \ldots, n_{k-1}\})\}$$

where k is variable — meaning all finite sequences are to be considered.)

The development is with the set graphs peer.

Theorem (`example_514`). The function of any code is approximable; for approximable set-maps, the function of the graph recovers the map; and every code sits inside the graph of its function.

Proof. The Fun/Graph laws from the set-graphs peer. ■

`Exercise_5_14.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.setGraph

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open SetGraph

/-- Scott's Exercise 5.14: `Fun` and `Graph` recover approximable set-maps and
dominate the identity on graphs. -/
theorem example_514 :
    (∀ u : Set ℕ, IsApprox (Fun u)) ∧
    (∀ {f : Set ℕ → Set ℕ}, IsApprox f → ∀ x : Set ℕ, Fun (Graph f) x = f x) ∧
    (∀ u : Set ℕ, u ⊆ Graph (Fun u)) :=
  ⟨Fun_isApprox, @Fun_Graph, id_le_Graph_Fun⟩
```

4.6.33 Language star

Kleene star of languages in a monoid, Arden's least-fixed-point law, and Park's least solution of the two-variable language equations.

Definition (`kpow`). The nth power of a set of monoid elements is the unit for zero and left multiplication by the set for successors.

Proof. By definition. ■

Theorem (`kpow_zero`). The zeroth power is the unit set.

Proof. By definition. ■

Theorem (`kpow_succ`). The successor power left-multiplies by the set.

Proof. By definition. ■

Definition (`G`). The Arden operator left-multiplies by a fixed set and unions a fixed remainder.

Proof. By definition. ■

Definition (`star`). The star of a set collects all finite powers.

Proof. By definition. ■

Definition (`parkA`). Park's A coefficient is a union b times star of a times b.

Proof. By definition. ■

Definition (`parkC`). Park's C coefficient is c union b times star of a times d.

Proof. By definition. ■

Definition (`parkX`). Park's x-solution is the star of A times C.

Proof. By definition. ■

Definition (`parkY`). Park's y-solution is the star of a times the union of b times x with d.

Proof. By definition. ■

Theorem (`mem_kpow_singleton`). Membership in a power of a singleton is equality with the corresponding power of the generator.

Proof. Induction on the exponent. ■

Theorem (`mem_star`). Membership in the star is existence of a power membership.

Proof. By definition. ■

Theorem (`mem_star_singleton`). Membership in the star of a singleton is equality with some power of the generator.

Proof. Transport the singleton power characterisation. ■

Theorem (`smul_assoc`). Pointwise multiplication of sets is associative.

Proof. Unpack products and reassociate the monoid multiplication. ■

Theorem (`smul_union`). Left multiplication distributes over union.

Proof. Case on which summand supplies the right factor. ■

Theorem (`star_eq`). The star equals the unit union the set times the star.

Proof. Compare powers with the unit and successor cases. ■

Theorem (`star_mul_subset_prefixed`). If a set absorbs left multiplication by z after union with v, then star of z times v sits inside it.

Proof. Induct on the power, using absorption of left multiplication. ■

Theorem (`sunion_mul`). Right multiplication distributes over union.

Proof. Case on which summand supplies the left factor. ■

Theorem (`star_mul_isFixed`). Star of z times v is a fixed point of the Arden operator for z and v.

Proof. Expand the star equation and redistribute. ■

Theorem (`arden`). The least fixed point of the Arden operator is star of z times v.

Proof. The star product is a fixed point and least among prefixed points. ■

Theorem (`parkX_eq`). Park's x satisfies the first language equation.

Proof. The star fixed-point law for Park's A and C, redistributed. ■

Theorem (`parkY_eq`). Park's y satisfies the second language equation.

Proof. The star fixed-point law for a against the y-body, redistributed. ■

Theorem (`park_least`). Any solution of the two language equations dominates Park's solution.

Proof. Prefixedness of the Park coefficients under any solution, then Arden. ■

Theorem (`park_solves`). Park's pair solves the two language equations.

Proof. The two Park equation laws. ■

Theorem (`part1`). For singletons, the least fixed point of left multiplication by e union $\{e'\}$ is the star of $\{e\}$ times $\{e'\}$.

Proof. Arden's lemma on singletons. ■

Theorem (`part1_freeMonoid`). The singleton Arden law specialises to the free monoid on booleans.

Proof. Specialise the singleton Arden law. ■

`langStar.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.gfpSet
public import Mathlib.Algebra.FreeMonoid.Basic
public import Mathlib.Algebra.Group.Pointwise.Set.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.LangStar
open Pointwise
open DomainTheory.NeighbourhoodSystems

variable {S : Type*} [Monoid S]

def kpow (z : Set S) : ℕ → Set S
  | 0 => 1
  | n + 1 => z * kpow z n

@[simp] theorem kpow_zero (z : Set S) : kpow z 0 = 1 := rfl

@[simp] theorem kpow_succ (z : Set S) (n : ℕ) : kpow z (n + 1) = z * kpow z n := rfl

def G (z v : Set S) (w : Set S) : Set S := z * w ∪ v

def star (z : Set S) : Set S := {s | ∃ n, s ∈ kpow z n}

def parkA (a b : Set S) : Set S := a ∪ b * star a * b

def parkC (a b c d : Set S) : Set S := c ∪ b * star a * d

def parkX (a b c d : Set S) : Set S := star (parkA a b) * parkC a b c d

def parkY (a b c d : Set S) : Set S := star a * (b * parkX a b c d ∪ d)

theorem mem_kpow_singleton (e s : S) (n : ℕ) : s ∈ kpow ({e} : Set S) n ↔ s = e ^ n
    := by
  induction n generalizing s with
  | zero => rw [kpow_zero, pow_zero]; exact Set.mem_one
  | succ m ih =>
      rw [kpow_succ, pow_succ', Set.mem_mul]
      constructor
      · rintro ⟨a, ha, t, ht, rfl⟩
        rw [Set.mem_singleton_iff] at ha
        rw [(ih t).mp ht, ha]
      · rintro rfl
```

```
        exact ⟨e, Set.mem_singleton_iff.mpr rfl, e ^ m, (ih (e ^ m)).mpr rfl, rfl⟩

theorem mem_star {z : Set S} {s : S} : s ∈ star z ↔ ∃ n, s ∈ kpow z n := Iff.rfl

theorem mem_star_singleton (e s : S) : s ∈ star ({e} : Set S) ↔ ∃ n, s = e ^ n := by
  rw [mem_star]
  constructor
  · rintro ⟨n, hn⟩; exact ⟨n, (mem_kpow_singleton e s n).mp hn⟩
  · rintro ⟨n, rfl⟩; exact ⟨n, (mem_kpow_singleton e (e ^ n) n).mpr rfl⟩

theorem smul_assoc (a b c : Set S) : a * b * c = a * (b * c) := by
  ext s; simp only [Set.mem_mul]
  constructor
  · rintro ⟨u, ⟨p, hp, q, hq, rfl⟩, w, hw, rfl⟩
    exact ⟨p, hp, q * w, ⟨q, hq, w, hw, rfl⟩, by rw [mul_assoc]⟩
  · rintro ⟨p, hp, u, ⟨q, hq, w, hw, rfl⟩, rfl⟩
    exact ⟨p * q, ⟨p, hp, q, hq, rfl⟩, w, hw, by rw [mul_assoc]⟩

theorem smul_union (a b c : Set S) : a * (b ∪ c) = a * b ∪ a * c := by
  ext s; simp only [Set.mem_mul, Set.mem_union]
  constructor
  · rintro ⟨u, hu, w, (hw | hw), rfl⟩
    · exact Or.inl ⟨u, hu, w, hw, rfl⟩
    · exact Or.inr ⟨u, hu, w, hw, rfl⟩
  · rintro (⟨u, hu, w, hw, rfl⟩ | ⟨u, hu, w, hw, rfl⟩)
    · exact ⟨u, hu, w, Or.inl hw, rfl⟩
    · exact ⟨u, hu, w, Or.inr hw, rfl⟩

theorem star_eq (z : Set S) : star z = (1 : Set S) ∪ z * star z := by
  apply Set.Subset.antisymm
  · intro s hs
    obtain ⟨n, hn⟩ := mem_star.mp hs
    cases n with
    | zero =>
        rw [kpow_zero] at hn
        rw [Set.mem_union]; exact Or.inl hn
    | succ m =>
        rw [kpow_succ, Set.mem_mul] at hn
        obtain ⟨a, ha, t, ht, rfl⟩ := hn
        rw [Set.mem_union]; right
        rw [Set.mem_mul]
        exact ⟨a, ha, t, mem_star.mpr ⟨m, ht⟩, rfl⟩
  · intro s hs
    rw [Set.mem_union] at hs
    rcases hs with h1 | h2
    · exact mem_star.mpr ⟨0, by rw [kpow_zero]; exact h1⟩
    · rw [Set.mem_mul] at h2
      obtain ⟨a, ha, t, ht, rfl⟩ := h2
      obtain ⟨m, hm⟩ := mem_star.mp ht
      exact mem_star.mpr ⟨m + 1, by rw [kpow_succ, Set.mem_mul]; exact ⟨a, ha, t, hm,
    rfl⟩⟩

theorem star_mul_subset_prefixed (z v x : Set S) (hx : z * x ∪ v ⊆ x) : star z * v ⊆ x
     := by
  have hvx : v ⊆ x := Set.subset_union_right.trans hx
  have hzx : z * x ⊆ x := Set.subset_union_left.trans hx
```

```
  have hpow : ∀ n, kpow z n * v ⊆ x := by
    intro n
    induction n with
    | zero => rw [kpow_zero, one_mul]; exact hvx
    | succ m ih =>
        rw [kpow_succ, smul_assoc]
        exact (Set.mul_subset_mul_left ih).trans hzx
  intro s hs
  rw [Set.mem_mul] at hs
  obtain ⟨a, ha, t, ht, rfl⟩ := hs
  obtain ⟨n, hn⟩ := mem_star.mp ha
  exact hpow n (by rw [Set.mem_mul]; exact ⟨a, hn, t, ht, rfl⟩)

theorem sunion_mul (a b c : Set S) : (a ∪ b) * c = a * c ∪ b * c := by
  ext s; simp only [Set.mem_mul, Set.mem_union]
  constructor
  · rintro ⟨u, (hu | hu), w, hw, rfl⟩
    · exact Or.inl ⟨u, hu, w, hw, rfl⟩
    · exact Or.inr ⟨u, hu, w, hw, rfl⟩
  · rintro (⟨u, hu, w, hw, rfl⟩ | ⟨u, hu, w, hw, rfl⟩)
    · exact ⟨u, Or.inl hu, w, hw, rfl⟩
    · exact ⟨u, Or.inr hu, w, hw, rfl⟩

theorem star_mul_isFixed (z v : Set S) : z * (star z * v) ∪ v = star z * v := by
  conv_rhs => rw [star_eq z]
  rw [sunion_mul, one_mul, smul_assoc]
  exact Set.union_comm _ _

theorem arden (z v : Set S) : lfpSet (G z v) = star z * v := by
  apply Set.Subset.antisymm
  · exact lfpSet_least (G z v) (star_mul_isFixed z v)
  · intro s hs x hx
    exact star_mul_subset_prefixed z v x hx hs

theorem parkX_eq (a b c d : Set S) :
    parkX a b c d = a * parkX a b c d ∪ b * parkY a b c d ∪ c := by
  have hfix : parkA a b * parkX a b c d ∪ parkC a b c d = parkX a b c d :=
    star_mul_isFixed (parkA a b) (parkC a b c d)
  conv_lhs => rw [← hfix]
  simp only [parkA, parkC, parkY, sunion_mul, smul_union, smul_assoc,
    Set.union_comm, Set.union_left_comm]

theorem parkY_eq (a b c d : Set S) :
    parkY a b c d = b * parkX a b c d ∪ a * parkY a b c d ∪ d := by
  have hfix : a * parkY a b c d ∪ (b * parkX a b c d ∪ d) = parkY a b c d :=
    star_mul_isFixed a (b * parkX a b c d ∪ d)
  conv_lhs => rw [← hfix]
  simp only [Set.union_assoc, Set.union_comm]

theorem park_least (a b c d : Set S) {x y : Set S}
    (hx : x = a * x ∪ b * y ∪ c) (hy : y = b * x ∪ a * y ∪ d) :
    parkX a b c d ⊆ x ∧ parkY a b c d ⊆ y := by
  have hax : a * x ⊆ x := by
    calc a * x ⊆ a * x ∪ b * y ∪ c := Set.subset_union_left.trans Set.
    subset_union_left
      _ = x := hx.symm
```

```
  have hby : b * y ⊆ x := by
    calc b * y ⊆ a * x ∪ b * y ∪ c := Set.subset_union_right.trans Set.
    subset_union_left
      _ = x := hx.symm
  have hcx : c ⊆ x := by
    calc c ⊆ a * x ∪ b * y ∪ c := Set.subset_union_right
      _ = x := hx.symm
  have hay : a * y ⊆ y := by
    calc a * y ⊆ b * x ∪ a * y ∪ d := Set.subset_union_right.trans Set.
    subset_union_left
      _ = y := hy.symm
  have hbx : b * x ⊆ y := by
    calc b * x ⊆ b * x ∪ a * y ∪ d := Set.subset_union_left.trans Set.
    subset_union_left
      _ = y := hy.symm
  have hdy : d ⊆ y := by
    calc d ⊆ b * x ∪ a * y ∪ d := Set.subset_union_right
      _ = y := hy.symm

  have hpre : a * y ∪ (b * x ∪ d) ⊆ y := Set.union_subset hay (Set.union_subset hbx
    hdy)
  have hyy : star a * (b * x ∪ d) ⊆ y := by
    rw [← arden a (b * x ∪ d)]
    exact lfpSet_subset (G a (b * x ∪ d)) hpre

  have key : b * star a * b * x ∪ b * star a * d = b * (star a * (b * x ∪ d)) := by
    simp only [smul_assoc, smul_union]

  have hprefixX : parkA a b * x ∪ parkC a b c d ⊆ x := by
    have hmid : b * star a * b * x ∪ b * star a * d ⊆ x := by
      rw [key]; exact (Set.mul_subset_mul_left hyy).trans hby
    have hm1 : b * star a * b * x ⊆ x := Set.subset_union_left.trans hmid
    have hm2 : b * star a * d ⊆ x := Set.subset_union_right.trans hmid
    unfold parkA parkC
    rw [sunion_mul]
    exact Set.union_subset (Set.union_subset hax hm1) (Set.union_subset hcx hm2)
  have hPX : parkX a b c d ⊆ x := by
    have harden : parkX a b c d = lfpSet (G (parkA a b) (parkC a b c d)) :=
      (arden (parkA a b) (parkC a b c d)).symm
    rw [harden]
    exact lfpSet_subset (G (parkA a b) (parkC a b c d)) hprefixX
  have hPY : parkY a b c d ⊆ y := by
    show star a * (b * parkX a b c d ∪ d) ⊆ y
    have h1 : b * parkX a b c d ∪ d ⊆ b * x ∪ d :=
      Set.union_subset_union_left d (Set.mul_subset_mul_left hPX)
    exact (Set.mul_subset_mul_left h1).trans hyy
  exact ⟨hPX, hPY⟩

theorem park_solves (a b c d : Set S) :
    parkX a b c d = a * parkX a b c d ∪ b * parkY a b c d ∪ c ∧
      parkY a b c d = b * parkX a b c d ∪ a * parkY a b c d ∪ d :=
  ⟨parkX_eq a b c d, parkY_eq a b c d⟩

theorem part1 (e e' : S) :
    lfpSet (fun w => ({e} : Set S) * w ∪ {e'}) = star ({e} : Set S) * {e'} := by
  have hG : (fun w => ({e} : Set S) * w ∪ {e'}) = G ({e} : Set S) {e'} := rfl
```

```
  rw [hG]; exact arden {e} {e'}

theorem part1_freeMonoid (e e' : FreeMonoid Bool) :
    lfpSet (fun w => ({e} : Set (FreeMonoid Bool)) ⋆ w ∪ {e'})
      = star ({e} : Set (FreeMonoid Bool)) ⋆ {e'} :=
  part1 e e'

end DomainTheory.NeighbourhoodSystems.LangStar
```

4.6.34 Exercise 5.15 — star of a word

(For algebraists.) We can regard $\langle \{0,1\}^*, \Lambda, \cdot \rangle$ as the free semigroup on two generators 0 and 1. The powerset $P\{0,1\}^*$ is taken as a domain as in Exercise 4.17. For "words" $e \in \{0,1\}^*$ define

$$e^* = \{\Lambda, e, e^2, e^3, \ldots, e^n, \ldots\}.$$

Show that the least fixed point of

$$z = \{e\} \cdot z \cup \{e'\}$$

in $P\{0,1\}^*$ is $z = e^* \cdot \{e'\}$.

Show further (as suggested by David Park) that the least solution of

$$x = a \cdot x \cup b \cdot y \cup c,$$
$$y = b \cdot x \cup a \cdot y \cup d$$

has

$$x = (a \cup b \cdot a^* \cdot b)^* \cdot (c \cup b \cdot a^* \cdot d),$$

where the $\{\cdot\}$ has been dropped off $\{a\}$, $\{b\}$ etc., and where the *-notation has been extended to the whole domain, so that $z^* = \Lambda \cup z^* \cdot z$.

(Hint: Apply 5.3.)

The development is with the language star peer.

Theorem (`example_515`). On the free monoid of boolean words, the least fixed point of left multiplication by a singleton union another singleton is the star product; Park's pair solves the two language equations and is least among solutions.

Proof. The free-monoid Arden law and the Park solution/leastness laws from the language-star peer. ■

`Exercise_5_15.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.langStar

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open LangStar Pointwise
```

```
/-- Scott's Exercise 5.15: Arden's lemma for singleton generators, and Park's least
solution of the two-variable language equations. -/
theorem example_515 :
    (∀ (e e' : FreeMonoid Bool),
      lfpSet (fun w => ({e} : Set (FreeMonoid Bool)) * w ∪ {e'}) =
        star ({e} : Set (FreeMonoid Bool)) * {e'}) ∧
    (∀ {S : Type*} [Monoid S] (a b c d : Set S),
      parkX a b c d = a * parkX a b c d ∪ b * parkY a b c d ∪ c ∧
        parkY a b c d = b * parkX a b c d ∪ a * parkY a b c d ∪ d) ∧
    (∀ {S : Type*} [Monoid S] (a b c d : Set S) {x y : Set S},
      x = a * x ∪ b * y ∪ c → y = b * x ∪ a * y ∪ d →
        parkX a b c d ⊆ x ∧ parkY a b c d ⊆ y) :=
  ⟨part1_freeMonoid, @park_solves, fun a b c d _ _ hx hy => park_least a b c d hx hy⟩
```

4.6.35 Sequence negation and merge

Negation, doubling, merge, and the Thue–Morse sequence on the domain of binary streams.

Definition (`flip`). Bitwise negation of a finite boolean string.

Proof. By definition. ■

Theorem (`flip_nil`). The named base or successor case.

Proof. By unfolding the definition. ■

Theorem (`flip_cons`). The named base or successor case.

Proof. By unfolding the definition. ■

Theorem (`flip_flip`). The named law for flip, double, or tail.

Proof. Unfold the string operations and the map laws. ■

Definition (`double`). Doubling inserts a copy of each bit.

Proof. By definition. ■

Theorem (`double_nil`). The named base or successor case.

Proof. By unfolding the definition. ■

Theorem (`double_cons`). The named base or successor case.

Proof. By unfolding the definition. ■

Theorem (`double_append`). The named comparison or prefix law for stream principals.

Proof. Unpack principal order via cones and singletons. ■

Theorem (`double_prefix`). The named comparison or prefix law for stream principals.

Proof. Unpack principal order via cones and singletons. ■

Theorem (`flip_prefix`). The named comparison or prefix law for stream principals.

Proof. Unpack principal order via cones and singletons. ■

Theorem (`not_strElem_le_strBot`). The named comparison or prefix law for stream principals.

Proof. Unpack principal order via cones and singletons. ■

Theorem (`strBot_le_strBot_iff`). The named comparison or prefix law for stream principals.

Proof. Unpack principal order via cones and singletons. ■

Theorem (`strBot_le_strElem_iff`). The named comparison or prefix law for stream principals.

Proof. Unpack principal order via cones and singletons. ■

Theorem (`strElem_le_strElem_iff`). The named comparison or prefix law for stream principals.

Proof. Unpack principal order via cones and singletons. ■

Definition (`tailSing`). The singular tail of a string, bottom on empty.

Proof. By definition. ■
Theorem (`tail_hsing`). The named law for flip, double, or tail.
Proof. Unfold the string operations and the map laws. ■
Theorem (`tail_prefix`). The named comparison or prefix law for stream principals.
Proof. Unpack principal order via cones and singletons. ■
Theorem (`tail_hcone`). The named law for flip, double, or tail.
Proof. Unfold the string operations and the map laws. ■
Definition (`tailMap`). The approximable tail map on the stream domain.
Proof. By definition. ■
Theorem (`tailMap_strBot`). The named law for flip, double, or tail.
Proof. Unfold the string operations and the map laws. ■
Theorem (`tailMap_strElem`). The named law for flip, double, or tail.
Proof. Unfold the string operations and the map laws. ■
Theorem (`neg_hcone`). The named law for stream negation.
Proof. Use the approximable-map presentation of negation. ■
Theorem (`neg_hsing`). The named law for stream negation.
Proof. Use the approximable-map presentation of negation. ■
Definition (`negMap`). The approximable negation map on streams.
Proof. By definition. ■
Theorem (`negMap_strBot`). The named law for stream negation.
Proof. Use the approximable-map presentation of negation. ■
Theorem (`negMap_strElem`). The named law for stream negation.
Proof. Use the approximable-map presentation of negation. ■
Theorem (`d_hcone`). The named law for flip, double, or tail.
Proof. Unfold the string operations and the map laws. ■
Theorem (`d_hsing`). The named law for flip, double, or tail.
Proof. Unfold the string operations and the map laws. ■
Definition (`dMap`). The approximable doubling map on streams.
Proof. By definition. ■
Theorem (`dMap_strBot`). The named law for flip, double, or tail.
Proof. Unfold the string operations and the map laws. ■
Theorem (`dMap_strElem`). The named law for flip, double, or tail.
Proof. Unfold the string operations and the map laws. ■
Definition (`shape`). The neighbourhood shape for a total/partial flag and a finite prefix.
Proof. By definition. ■
Theorem (`memC_shape`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Definition (`shapeElem`). The principal element of a shape neighbourhood.
Proof. By definition. ■
Theorem (`shapeElem_true`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Theorem (`shapeElem_false`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Definition (`SLe`). The comparison relation used to merge two shaped streams.
Proof. By definition. ■

Definition (`mergeVal`). The finite merge of two flagged strings.
Proof. By definition. ■
Theorem (`mergeVal_nil`). The named base or successor case.
Proof. By unfolding the definition. ■
Theorem (`mergeVal_cons_nil`). The named base or successor case.
Proof. By unfolding the definition. ■
Theorem (`mergeVal_cons_cons`). The named base or successor case.
Proof. By unfolding the definition. ■
Theorem (`SLe_cons2`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Theorem (`SLe_cons_inv`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Definition (`mergeElem`). The stream element of a merged pair of shapes.
Proof. By definition. ■
Theorem (`mergeVal_SLe`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Theorem (`shapeElem_le_iff`). The named comparison or prefix law for stream principals.
Proof. Unpack principal order via cones and singletons. ■
Theorem (`mergeElem_mono`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Theorem (`shape_injective`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Theorem (`shape_refine`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Definition (`mergeMap`). The approximable merge of a product of streams.
Proof. By definition. ■
Theorem (`consMap_shapeElem`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Theorem (`map_ext_C`). The named extensionality or constructor law.
Proof. Compare approximable maps on principals. ■
Theorem (`memC_eq_shape`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Theorem (`mergeMap_pair`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Theorem (`mergeVal_diag`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Theorem (`mergeElem_diag`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Theorem (`mergeMap_diag`). The named law for merge or shapes.
Proof. Case on the merge/shape definitions. ■
Theorem (`negMap_comp_negMap`). The named law for stream negation.
Proof. Use the approximable-map presentation of negation. ■
Theorem (`negMap_negMap`). The named law for stream negation.
Proof. Use the approximable-map presentation of negation. ■
Theorem (`neg_cons_false`). The named law for stream negation.

Proof. Use the approximable-map presentation of negation. ■

Theorem (`neg_cons_true`). The named law for stream negation.

Proof. Use the approximable-map presentation of negation. ■

Theorem (`prod_principal_pair`). The named law for flip, double, or tail.

Proof. Unfold the string operations and the map laws. ■

Theorem (`prodMap_ext`). The named extensionality or constructor law.

Proof. Compare approximable maps on principals. ■

Theorem (`mergeMap_cons`). The named base or successor case.

Proof. By unfolding the definition. ■

Theorem (`tailMap_consMap_strBot`). The named law for flip, double, or tail.

Proof. Unfold the string operations and the map laws. ■

Theorem (`tailMap_consMap_strElem`). The named law for flip, double, or tail.

Proof. Unfold the string operations and the map laws. ■

Definition (`expand`). The Thue–Morse expansion step on a finite string.

Proof. By definition. ■

Theorem (`expand_nil`). The named base or successor case.

Proof. By unfolding the definition. ■

Theorem (`expand_cons`). The named base or successor case.

Proof. By unfolding the definition. ■

Definition (`tm`). The Thue–Morse bit of a natural number.

Proof. By definition. ■

Definition (`tmOp`). The operator whose fixed point is the Thue–Morse stream.

Proof. By definition. ■

Theorem (`tm_zero`). The named base or successor case.

Proof. By unfolding the definition. ■

Definition (`tmList`). The finite Thue–Morse prefix of a given length.

Proof. By definition. ■

Theorem (`tmList_zero`). The named base or successor case.

Proof. By unfolding the definition. ■

Definition (`tElt`). The Thue–Morse stream element.

Proof. By definition. ■

Theorem (`cone_univ_iff`). The named stream fact.

Proof. Follows from the surrounding stream and fixed-point laws. ■

Theorem (`expand_append`). The named law for the Thue–Morse stream.

Proof. Use the binary-bit and fixed-point characterisations. ■

Theorem (`iterate_expand_cons`). The named base or successor case.

Proof. By unfolding the definition. ■

Theorem (`mergeVal_snd_false`). The named law for merge or shapes.

Proof. Case on the merge/shape definitions. ■

Theorem (`strBot_mem_cone`). The named comparison or prefix law for stream principals.

Proof. Unpack principal order via cones and singletons. ■

Theorem (`strBot_nil`). The named base or successor case.

Proof. By unfolding the definition. ■

Theorem (`tElt_unfold`). The named law for the Thue–Morse stream.

Proof. Use the binary-bit and fixed-point characterisations. ■

Theorem (`tmList_length`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`tmList_succ`). The named base or successor case.
Proof. By unfolding the definition. ■
Theorem (`tmList_prefix`). The named comparison or prefix law for stream principals.
Proof. Unpack principal order via cones and singletons. ■
Theorem (`tmOp_strBot`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`tm_two_mul`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`tm_two_mul_add_one`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`expand_tmList`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`expand_iterate_eq`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`weave_head`). The named stream fact.
Proof. Follows from the surrounding stream and fixed-point laws. ■
Theorem (`tmOp_strBot_expand`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`iterElem_succ_eq`). The named stream fact.
Proof. Follows from the surrounding stream and fixed-point laws. ■
Definition (`Overlap`). A periodic overlap predicate on Thue–Morse bits.
Proof. By definition. ■
Theorem (`append_three_period`). The named law for flip, double, or tail.
Proof. Unfold the string operations and the map laws. ■
Theorem (`tmList_getElem?`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`tm_of_eq_two_mul`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`tm_of_eq_two_mul_add_one`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`tm_two_mul_ne`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`odd_of_consec_eq`). The named law for flip, double, or tail.
Proof. Unfold the string operations and the map laws. ■
Theorem (`no_three_consec`). The named stream fact.
Proof. Follows from the surrounding stream and fixed-point laws. ■
Theorem (`no_overlap`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`no_cube`). The named law for the Thue–Morse stream.
Proof. Use the binary-bit and fixed-point characterisations. ■
Theorem (`tmList_le_tElt`). The named comparison or prefix law for stream principals.
Proof. Unpack principal order via cones and singletons. ■
Theorem (`tElt_digit`). The named law for the Thue–Morse stream.

Proof. Use the binary-bit and fixed-point characterisations. ■

Theorem (`tElt_mem_cone_iff`). The named law for the Thue–Morse stream.

Proof. Use the binary-bit and fixed-point characterisations. ■

Theorem (`tElt_cube_free`). The named law for the Thue–Morse stream.

Proof. Use the binary-bit and fixed-point characterisations. ■

seqNegMerge.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_11
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.seqPeano
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import Mathlib.Data.Nat.Bits
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.iterSys
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.SeqNegMerge
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap BitDom
    SeqDom SeqPeano

abbrev flip (σ : Str) : Str := σ.map not

@[simp] theorem flip_nil : flip [] = [] := rfl

@[simp] theorem flip_cons (b : Bool) (σ : Str) : flip (b :: σ) = (!b) :: flip σ := rfl

@[simp] theorem flip_flip (σ : Str) : flip (flip σ) = σ := by
  induction σ with
  | nil => rfl
  | cons b σ ih => simp [ih]

def double : Str → Str
  | [] => []
  | b :: σ => b :: b :: double σ

@[simp] theorem double_nil : double [] = [] := rfl

@[simp] theorem double_cons (b : Bool) (σ : Str) : double (b :: σ) = b :: b :: double
    σ := rfl

theorem double_append (σ τ : Str) : double (σ ++ τ) = double σ ++ double τ := by
  induction σ with
  | nil => rfl
  | cons b σ ih => simp [double, ih]

theorem double_prefix {σ τ : Str} (h : σ <+: τ) : double σ <+: double τ := by
  obtain ⟨?, rfl⟩ := h
```

```
  exact ⟨double ?, (double_append σ ?).symm⟩

theorem flip_prefix {σ τ : Str} (h : σ <+: τ) : flip σ <+: flip τ := h.map _

theorem not_strElem_le_strBot {σ τ : Str} : ¬ strElem σ ≤ strBot τ := by
  rw [strElem, strBot, C.principal_le_iff]
  exact not_cone_subset_singleton τ σ

theorem strBot_le_strBot_iff {σ τ : Str} : strBot σ ≤ strBot τ ↔ σ <+: τ := by
  rw [strBot, strBot, C.principal_le_iff, cone.subset_cone]

theorem strBot_le_strElem_iff {σ τ : Str} : strBot σ ≤ strElem τ ↔ σ <+: τ := by
  rw [strBot, strElem, C.principal_le_iff]
  exact singleton_subset_cone

theorem strElem_le_strElem_iff {σ τ : Str} : strElem σ ≤ strElem τ ↔ σ = τ := by
  rw [strElem, strElem, C.principal_le_iff, Set.singleton_subset_iff, Set.
    mem_singleton_iff,
    eq_comm]

variable {β : Type*}

def tailSing : Str → C.Element
  | [] => strBot []
  | _ :: σ' => strElem σ'

theorem tail_hsing {σ τ : Str} (h : σ <+: τ) : strBot σ.tail ≤ tailSing τ := by
  cases τ with
  | nil => obtain rfl := List.prefix_nil.mp h; exact le_refl _
  | cons a τ' =>
    refine strBot_le_strElem_iff.mpr ?_
    cases σ with
    | nil => exact List.nil_prefix
    | cons b σ' =>
      obtain ⟨rfl, h'⟩ := List.cons_prefix_cons.mp h
      exact h'

theorem tail_prefix {σ τ : Str} (h : σ <+: τ) : σ.tail <+: τ.tail := by
  obtain ⟨?, rfl⟩ := h
  cases σ with
  | nil => simp
  | cons a σ' => exact List.prefix_append σ' ?

variable {β : Type*}

theorem tail_hcone {σ τ : Str} (h : σ <+: τ) : strBot σ.tail ≤ strBot τ.tail :=
  strBot_le_strBot_iff.mpr (tail_prefix h)

variable {β : Type*}

def tailMap : ApproximableMap C C :=
  liftC C (fun σ => strBot σ.tail) tailSing tail_hcone tail_hsing

@[simp] theorem tailMap_strBot (σ : Str) :
    tailMap.toElementMap (strBot σ) = strBot σ.tail :=
  liftC_strBot C (fun σ => strBot σ.tail) tailSing tail_hcone tail_hsing σ
```

```
@[simp] theorem tailMap_strElem (σ : Str) :
    tailMap.toElementMap (strElem σ) = tailSing σ :=
  liftC_strElem C (fun σ => strBot σ.tail) tailSing tail_hcone tail_hsing σ

theorem neg_hcone {σ τ : Str} (h : σ <+: τ) : strBot (flip σ) ≤ strBot (flip τ) :=
  strBot_le_strBot_iff.mpr (flip_prefix h)

theorem neg_hsing {σ τ : Str} (h : σ <+: τ) : strBot (flip σ) ≤ strElem (flip τ) :=
  strBot_le_strElem_iff.mpr (flip_prefix h)

def negMap : ApproximableMap C C :=
  liftC C (fun σ => strBot (flip σ)) (fun σ => strElem (flip σ)) neg_hcone neg_hsing

@[simp] theorem negMap_strBot (σ : Str) :
    negMap.toElementMap (strBot σ) = strBot (flip σ) :=
  liftC_strBot C (fun σ => strBot (flip σ)) (fun σ => strElem (flip σ)) neg_hcone
    neg_hsing σ

@[simp] theorem negMap_strElem (σ : Str) :
    negMap.toElementMap (strElem σ) = strElem (flip σ) :=
  liftC_strElem C (fun σ => strBot (flip σ)) (fun σ => strElem (flip σ)) neg_hcone
    neg_hsing σ

theorem d_hcone {σ τ : Str} (h : σ <+: τ) : strBot (double σ) ≤ strBot (double τ) :=
  strBot_le_strBot_iff.mpr (double_prefix h)

theorem d_hsing {σ τ : Str} (h : σ <+: τ) : strBot (double σ) ≤ strElem (double τ) :=
  strBot_le_strElem_iff.mpr (double_prefix h)

def dMap : ApproximableMap C C :=
  liftC C (fun σ => strBot (double σ)) (fun σ => strElem (double σ)) d_hcone d_hsing

@[simp] theorem dMap_strBot (σ : Str) :
    dMap.toElementMap (strBot σ) = strBot (double σ) :=
  liftC_strBot C (fun σ => strBot (double σ)) (fun σ => strElem (double σ)) d_hcone
    d_hsing σ

@[simp] theorem dMap_strElem (σ : Str) :
    dMap.toElementMap (strElem σ) = strElem (double σ) :=
  liftC_strElem C (fun σ => strBot (double σ)) (fun σ => strElem (double σ)) d_hcone
    d_hsing σ

def shape : Bool → Str → Set Str
  | true, σ => {σ}
  | false, σ => cone σ

theorem memC_shape : ∀ (b : Bool) (σ : Str), memC (shape b σ)
  | true, σ => memC_singleton σ
  | false, σ => memC_cone σ

def shapeElem (b : Bool) (σ : Str) : C.Element := C.principal (memC_shape b σ)

@[simp] theorem shapeElem_true (σ : Str) : shapeElem true σ = strElem σ := rfl

@[simp] theorem shapeElem_false (σ : Str) : shapeElem false σ = strBot σ := rfl
```

```
def SLe : Bool → Str → Bool → Str → Prop
  | true, σ, b', σ' => b' = true ∧ σ = σ'
  | false, σ, _, σ' => σ <+: σ'

def mergeVal : Str → Bool → Str → Bool → Str × Bool
  | [], b₀, _, _ => ([], b₀)
  | a :: _, _, [], _ => ([a], false)
  | a :: σ, b₀, b :: τ, b₁ => (a :: b :: (mergeVal σ b₀ τ b₁).1, (mergeVal σ b₀ τ b₁).2)

@[simp] theorem mergeVal_nil (b₀ : Bool) (τ : Str) (b₁ : Bool) :
    mergeVal [] b₀ τ b₁ = ([], b₀) := rfl

@[simp] theorem mergeVal_cons_nil (a : Bool) (σ : Str) (b₀ b₁ : Bool) :
    mergeVal (a :: σ) b₀ [] b₁ = ([a], false) := rfl

@[simp] theorem mergeVal_cons_cons (a : Bool) (σ : Str) (b₀ b : Bool) (τ : Str) (b₁ :
    Bool) :
    mergeVal (a :: σ) b₀ (b :: τ) b₁ =
      (a :: b :: (mergeVal σ b₀ τ b₁).1, (mergeVal σ b₀ τ b₁).2) := rfl

theorem SLe_cons2 {p p' : Bool} {? ?' : Str} (c d : Bool) (h : SLe p ? p' ?') :
    SLe p (c :: d :: ?) p' (c :: d :: ?') := by
  cases p with
  | true => obtain ⟨rfl, rfl⟩ := h; exact ⟨rfl, rfl⟩
  | false =>
    exact List.cons_prefix_cons.mpr ⟨rfl, List.cons_prefix_cons.mpr ⟨rfl, h⟩⟩

theorem SLe_cons_inv {b₀ b₀' : Bool} {a : Bool} {σ₀ σ' : Str} (h : SLe b₀ (a :: σ₀) b₀'
    σ') :
    ∃ σ₀', σ' = a :: σ₀' ∧ SLe b₀ σ₀ b₀' σ₀' := by
  cases b₀ with
  | true =>
    obtain ⟨rfl, rfl⟩ := h
    exact ⟨σ₀, rfl, rfl, rfl⟩
  | false =>
    cases σ' with
    | nil => exact absurd h (by simp [SLe])
    | cons a' σ₀' =>
      obtain ⟨rfl, h'⟩ := List.cons_prefix_cons.mp h
      exact ⟨σ₀', rfl, h'⟩

def mergeElem (σ : Str) (b₀ : Bool) (τ : Str) (b₁ : Bool) : C.Element :=
  shapeElem (mergeVal σ b₀ τ b₁).2 (mergeVal σ b₀ τ b₁).1

theorem mergeVal_SLe : ∀ (σ : Str) (b₀ : Bool) (σ' : Str) (b₀' : Bool)
    (τ : Str) (b₁ : Bool) (τ' : Str) (b₁' : Bool),
    SLe b₀ σ b₀' σ' → SLe b₁ τ b₁' τ' →
    SLe (mergeVal σ b₀ τ b₁).2 (mergeVal σ b₀ τ b₁).1
      (mergeVal σ' b₀' τ' b₁').2 (mergeVal σ' b₀' τ' b₁').1
  | [], b₀, σ', b₀', τ, b₁, τ', b₁', h0, _ => by
    cases b₀ with
    | true =>
      obtain ⟨rfl, rfl⟩ := h0
      simp only [mergeVal_nil]; exact ⟨rfl, rfl⟩
    | false =>
```

```
      simp only [mergeVal_nil]; exact List.nil_prefix
  | a :: σ₀, b₀, σ', b₀', [], b₁, τ', b₁', h0, _ => by
    obtain ⟨σ₀', rfl, _⟩ := SLe_cons_inv h0
    simp only [mergeVal_cons_nil]
    cases τ' with
    | nil => simp only [mergeVal_cons_nil]; exact List.prefix_rfl
    | cons c τ₀' => simp only [mergeVal_cons_cons]; exact ⟨c :: _, rfl⟩
  | a :: σ₀, b₀, σ', b₀', b :: τ₀, b₁, τ', b₁', h0, h1 => by
    obtain ⟨σ₀', rfl, h0'⟩ := SLe_cons_inv h0
    obtain ⟨τ₀', rfl, h1'⟩ := SLe_cons_inv h1
    simp only [mergeVal_cons_cons]
    exact SLe_cons2 a b (mergeVal_SLe σ₀ b₀ σ₀' b₀' τ₀ b₁ τ₀' b₁' h0' h1')

theorem shapeElem_le_iff {b b' : Bool} {σ σ' : Str} :
    shapeElem b σ ≤ shapeElem b' σ' ↔ SLe b σ b' σ' := by
  cases b <;> cases b' <;>
    simp only [shapeElem_true, shapeElem_false, SLe, strElem_le_strElem_iff,
      strBot_le_strBot_iff, strBot_le_strElem_iff, true_and]
  · exact ⟨fun h => absurd h not_strElem_le_strBot, fun h => absurd h (by simp)⟩

theorem mergeElem_mono {σ σ' τ τ' : Str} {b₀ b₀' b₁ b₁' : Bool}
    (h0 : shapeElem b₀ σ ≤ shapeElem b₀' σ') (h1 : shapeElem b₁ τ ≤ shapeElem b₁' τ') :
    mergeElem σ b₀ τ b₁ ≤ mergeElem σ' b₀' τ' b₁' :=
  shapeElem_le_iff.mpr
    (mergeVal_SLe σ b₀ σ' b₀' τ b₁ τ' b₁' (shapeElem_le_iff.mp h0) (shapeElem_le_iff.mp
     h1))

theorem shape_injective : ∀ {b b' : Bool} {σ σ' : Str}, shape b σ = shape b' σ' → b =
    b' ∧ σ = σ'
  | true, true, σ, σ', h => ⟨rfl, by rwa [shape, shape, Set.singleton_eq_singleton_iff]
     at h⟩
  | true, false, σ, σ', h => absurd h.symm (cone_ne_singleton σ' σ)
  | false, true, σ, σ', h => absurd h (cone_ne_singleton σ σ')
  | false, false, σ, σ', h => ⟨rfl, cone.injective h⟩

theorem shape_refine {b : Bool} {σ : Str} {P : Set Str} (hP : memC P) (hsub : P ⊆
    shape b σ) :
    ∃ (b' : Bool) (σ' : Str), P = shape b' σ' ∧ shapeElem b σ ≤ shapeElem b' σ' := by
  rcases hP with ⟨?, rfl⟩ | ⟨?, rfl⟩
  · exact ⟨false, ?, rfl, (C.principal_le_iff (memC_shape b σ) (memC_shape false ?)).
    mpr hsub⟩
  · exact ⟨true, ?, rfl, (C.principal_le_iff (memC_shape b σ) (memC_shape true ?)).mpr
    hsub⟩

def mergeMap : ApproximableMap (prod C C) C where
  rel W Z := ∃ (b₀ : Bool) (σ : Str) (b₁ : Bool) (τ : Str),
    W = prodNbhd (shape b₀ σ) (shape b₁ τ) ∧ (mergeElem σ b₀ τ b₁).mem Z
  rel_dom := by
    rintro W Z ⟨b₀, σ, b₁, τ, rfl, _⟩
    exact prod_mem_prodNbhd (memC_shape b₀ σ) (memC_shape b₁ τ)
  rel_cod := by
    rintro W Z ⟨b₀, σ, b₁, τ, _, hZ⟩
    exact (mergeElem σ b₀ τ b₁).sub hZ
  master_rel := by
    refine ⟨false, [], false, [], ?_, (mergeElem [] false [] false).master_mem⟩
    show (prod C C).master = prodNbhd (shape false []) (shape false [])
```

```
    simp only [prod_master, shape, C_master, cone.nil]
  inter_right := by
    rintro W Z Z' ⟨b₀, σ, b₁, τ, rfl, hZ⟩ ⟨b₀', σ', b₁', τ', heq, hZ'⟩
    obtain ⟨hX, hY⟩ := prodNbhd.injective heq
    obtain ⟨rfl, rfl⟩ := shape_injective hX
    obtain ⟨rfl, rfl⟩ := shape_injective hY
    exact ⟨b₀, σ, b₁, τ, rfl, (mergeElem σ b₀ τ b₁).inter_mem hZ hZ'⟩
  mono := by
    rintro W W₂ Z Z' ⟨b₀, σ, b₁, τ, rfl, hZ⟩ hW₂W hZZ' hW₂ hZ'
    obtain ⟨P, Q, hP, hQ, rfl⟩ := hW₂
    obtain ⟨hPsub, hQsub⟩ := prodNbhd.subset_iff.mp hW₂W
    obtain ⟨b₀', σ', hPeq, hle0⟩ := shape_refine hP hPsub
    obtain ⟨b₁', τ', hQeq, hle1⟩ := shape_refine hQ hQsub
    subst hPeq; subst hQeq
    refine ⟨b₀', σ', b₁', τ', rfl, ?_⟩
    have hmono := mergeElem_mono hle0 hle1
    exact (mergeElem σ' b₀' τ' b₁').up_mem (hmono Z hZ) hZ' hZZ'

@[simp] theorem consMap_shapeElem (b c : Bool) (σ : Str) :
    (consMap b).toElementMap (shapeElem c σ) = shapeElem c (b :: σ) := by
  cases c with
  | false => rw [shapeElem_false, consMap_strBot]; rfl
  | true => rw [shapeElem_true, consMap_strElem]; rfl

theorem map_ext_C {V : NeighborhoodSystem β} {f g : ApproximableMap C V}
    (hbot : ∀ σ, f.toElementMap (strBot σ) = g.toElementMap (strBot σ))
    (helem : ∀ σ, f.toElementMap (strElem σ) = g.toElementMap (strElem σ)) : f = g :=
   by
  apply eq_of_toElementMap_principal
  intro X hX
  obtain (⟨σ, rfl⟩ | ⟨σ, rfl⟩) := (C_mem.mp hX)
  · exact hbot σ
  · exact helem σ

theorem memC_eq_shape {X : Set Str} (hX : memC X) : ∃ (b : Bool) (σ : Str), X = shape b
    σ := by
  rcases hX with ⟨σ, rfl⟩ | ⟨σ, rfl⟩
  · exact ⟨false, σ, rfl⟩
  · exact ⟨true, σ, rfl⟩

theorem mergeMap_pair (b₀ : Bool) (σ : Str) (b₁ : Bool) (τ : Str) :
    mergeMap.toElementMap (pair (shapeElem b₀ σ) (shapeElem b₁ τ)) = mergeElem σ b₀ τ b
    ₁ := by
  apply Element.ext
  intro Z
  constructor
  · rintro ⟨W, hWmem, c₀, ?, c₁, ?, rfl, hZ⟩
    rw [mem_pair_prodNbhd] at hWmem
    obtain ⟨hmσ, hmτ⟩ := hWmem
    have hle0 : shapeElem c₀ ? ≤ shapeElem b₀ σ :=
      (C.principal_le_iff (memC_shape c₀ ?) (memC_shape b₀ σ)).mpr hmσ.2
    have hle1 : shapeElem c₁ ? ≤ shapeElem b₁ τ :=
      (C.principal_le_iff (memC_shape c₁ ?) (memC_shape b₁ τ)).mpr hmτ.2
    exact mergeElem_mono hle0 hle1 Z hZ
  · intro hZ
    refine ⟨prodNbhd (shape b₀ σ) (shape b₁ τ), ?_, b₀, σ, b₁, τ, rfl, hZ⟩
```

```
    exact mem_pair_prodNbhd.mpr ⟨⟨memC_shape b₀ σ, subset_rfl⟩, ⟨memC_shape b₁ τ,
    subset_rfl⟩⟩

theorem mergeVal_diag (s : Bool) (σ : Str) : mergeVal σ s σ s = (double σ, s) := by
  induction σ with
  | nil => rfl
  | cons a σ ih => simp [mergeVal_cons_cons, ih]

theorem mergeElem_diag (s : Bool) (σ : Str) : mergeElem σ s σ s = shapeElem s (double
    σ) := by
  simp [mergeElem, mergeVal_diag]

theorem mergeMap_diag (x : C.Element) :
    mergeMap.toElementMap (pair x x) = dMap.toElementMap x := by
  have key : mergeMap.comp (paired (idMap C) (idMap C)) = dMap := by
    apply map_ext_C
    · intro σ
      rw [toElementMap_comp, toElementMap_paired, toElementMap_idMap,
        show pair (strBot σ) (strBot σ) = pair (shapeElem false σ) (shapeElem false σ)
     from rfl,
        mergeMap_pair, mergeElem_diag, shapeElem_false, dMap_strBot]
    · intro σ
      rw [toElementMap_comp, toElementMap_paired, toElementMap_idMap,
        show pair (strElem σ) (strElem σ) = pair (shapeElem true σ) (shapeElem true σ)
     from rfl,
        mergeMap_pair, mergeElem_diag, shapeElem_true, dMap_strElem]
  have hx := congrArg (fun m : ApproximableMap C C => m.toElementMap x) key
  simp only [toElementMap_comp, toElementMap_paired, toElementMap_idMap] at hx
  exact hx

theorem negMap_comp_negMap : negMap.comp negMap = idMap C := by
  apply map_ext_C
  · intro σ
    rw [toElementMap_comp, negMap_strBot, negMap_strBot, flip_flip, toElementMap_idMap
    ]
  · intro σ
    rw [toElementMap_comp, negMap_strElem, negMap_strElem, flip_flip,
    toElementMap_idMap]

theorem negMap_negMap (x : C.Element) : negMap.toElementMap (negMap.toElementMap x) =
    x := by
  have h := negMap_comp_negMap
  rw [← toElementMap_comp, h, toElementMap_idMap]

theorem neg_cons_false : negMap.comp (consMap false) = (consMap true).comp negMap :=
    by
  apply map_ext_C
  · intro σ
    rw [toElementMap_comp, consMap_strBot, negMap_strBot, toElementMap_comp,
    negMap_strBot,
      consMap_strBot]
    rfl
  · intro σ
    rw [toElementMap_comp, consMap_strElem, negMap_strElem, toElementMap_comp,
    negMap_strElem,
      consMap_strElem]
```

```
    rfl

theorem neg_cons_true : negMap.comp (consMap true) = (consMap false).comp negMap := by
  apply map_ext_C
  · intro σ
    rw [toElementMap_comp, consMap_strBot, negMap_strBot, toElementMap_comp,
    negMap_strBot,
      consMap_strBot]
    rfl
  · intro σ
    rw [toElementMap_comp, consMap_strElem, negMap_strElem, toElementMap_comp,
    negMap_strElem,
      consMap_strElem]
    rfl

theorem prod_principal_pair (b₀ : Bool) (σ : Str) (b₁ : Bool) (τ : Str) :
    (prod C C).principal (prod_mem_prodNbhd (memC_shape b₀ σ) (memC_shape b₁ τ))
      = pair (shapeElem b₀ σ) (shapeElem b₁ τ) := by
  apply Element.ext
  intro P
  rw [mem_principal]
  constructor
  · rintro ⟨hP, hsub⟩
    obtain ⟨X', Y', hX', hY', rfl⟩ := hP
    obtain ⟨hsX, hsY⟩ := prodNbhd.subset_iff.mp hsub
    exact ⟨X', Y', ⟨hX', hsX⟩, ⟨hY', hsY⟩, rfl⟩
  · rintro ⟨X', Y', ⟨hX', hsX⟩, ⟨hY', hsY⟩, rfl⟩
    exact ⟨prod_mem_prodNbhd hX' hY', prodNbhd.subset_iff.mpr ⟨hsX, hsY⟩⟩

theorem prodMap_ext {f g : ApproximableMap (prod C C) C}
    (h : ∀ b₀ σ b₁ τ, f.toElementMap (pair (shapeElem b₀ σ) (shapeElem b₁ τ))
      = g.toElementMap (pair (shapeElem b₀ σ) (shapeElem b₁ τ))) : f = g := by
  apply eq_of_toElementMap_principal
  intro W hW
  obtain ⟨b₀, σ, b₁, τ, rfl⟩ :
      ∃ b₀ σ b₁ τ, W = prodNbhd (shape b₀ σ) (shape b₁ τ) := by
    obtain ⟨X, Y, hX, hY, rfl⟩ := hW
    obtain ⟨b₀, σ, rfl⟩ := memC_eq_shape hX
    obtain ⟨b₁, τ, rfl⟩ := memC_eq_shape hY
    exact ⟨b₀, σ, b₁, τ, rfl⟩
  have heq : (prod C C).principal hW = pair (shapeElem b₀ σ) (shapeElem b₁ τ) :=
    prod_principal_pair b₀ σ b₁ τ
  rw [heq]; exact h b₀ σ b₁ τ

theorem mergeMap_cons (? ? : Bool) (x y : C.Element) :
    mergeMap.toElementMap
        (pair ((consMap ?).toElementMap x) ((consMap ?).toElementMap y))
      = (consMap ?).toElementMap

          ((consMap ?).toElementMap (mergeMap.toElementMap (pair x y))) := by
  have key :
      mergeMap.comp (paired ((consMap ?).comp (proj₀ C C)) ((consMap ?).comp (proj₁ C C
    )))
        = ((consMap ?).comp (consMap ?)).comp mergeMap := by
    apply prodMap_ext
    intro b₀ σ b₁ τ
```

```
    simp only [toElementMap_comp, toElementMap_paired, toElementMap_proj₀,
    toElementMap_proj₁,
      fst_pair, snd_pair, consMap_shapeElem, mergeMap_pair, mergeElem,
    mergeVal_cons_cons]
  have hx := congrArg (fun m : ApproximableMap (prod C C) C => m.toElementMap (pair x
    y)) key
  simp only [toElementMap_comp, toElementMap_paired, toElementMap_proj₀,
    toElementMap_proj₁,
    fst_pair, snd_pair] at hx
  exact hx

variable {β : Type*}

theorem tailMap_consMap_strBot (b : Bool) (σ : Str) :
    tailMap.toElementMap ((consMap b).toElementMap (strBot σ)) = strBot σ := by
  rw [consMap_strBot, tailMap_strBot]; rfl

theorem tailMap_consMap_strElem (b : Bool) (σ : Str) :
    tailMap.toElementMap ((consMap b).toElementMap (strElem σ)) = strElem σ := by
  rw [consMap_strElem, tailMap_strElem]; rfl

def expand : Str → Str
  | [] => []
  | b :: σ => b :: (!b) :: expand σ

@[simp] theorem expand_nil : expand [] = [] := rfl

@[simp] theorem expand_cons (b : Bool) (σ : Str) : expand (b :: σ) = b :: (!b) ::
    expand σ := rfl

def tm (n : ℕ) : Bool := (Nat.bits n).foldr xor false

def tmOp : ApproximableMap C C :=
  (consMap false).comp (mergeMap.comp (paired negMap tailMap))

@[simp] theorem tm_zero : tm 0 = false := by simp [tm, Nat.zero_bits]

def tmList (n : ℕ) : Str := (List.range n).map tm

@[simp] theorem tmList_zero : tmList 0 = [] := rfl

def tElt : C.Element := tmOp.fixElement

theorem cone_univ_iff {σ : Str} : cone σ = Set.univ ↔ σ = [] := by
  constructor
  · intro h
    exact List.prefix_nil.mp (cone.mem.mp (h ▸ Set.mem_univ ([] : Str)))
  · rintro rfl; exact cone.nil

theorem expand_append (σ τ : Str) : expand (σ ++ τ) = expand σ ++ expand τ := by
  induction σ with
  | nil => rfl
  | cons b σ ih => simp [ih]

theorem iterate_expand_cons (k : ℕ) : ∃ ?, (expand^[k] [false] : Str) = false :: ? :=
    by
```

```
  induction k with
  | zero => exact ⟨[], rfl⟩
  | succ k ih =>
    obtain ⟨?, h?⟩ := ih
    exact ⟨true :: expand ?, by rw [Function.iterate_succ_apply', h?]; rfl⟩

theorem mergeVal_snd_false : ∀ (σ τ : Str) (b₁ : Bool), (mergeVal σ false τ b₁).2 =
    false
  | [], _, _ => rfl
  | _ :: _, [], _ => rfl
  | _ :: σ, _ :: τ, b₁ => by simp only [mergeVal_cons_cons]; exact mergeVal_snd_false σ
     τ b₁

theorem strBot_mem_cone {σ τ : Str} : (strBot τ).mem (cone σ) ↔ σ <+: τ := by
  rw [strBot, mem_principal]
  exact ⟨fun h => cone.subset_cone.mp h.2, fun h => ⟨memC_cone σ, cone.subset_cone.mpr
    h⟩⟩

theorem strBot_nil : strBot [] = C.bot := by
  apply Element.ext
  intro Y
  rw [strBot, mem_principal, mem_bot, cone.nil, C_master]
  exact ⟨fun h => Set.Subset.antisymm (Set.subset_univ _) h.2,
    fun h => ⟨h ▶ C.master_mem, h ▶ Set.Subset.refl _⟩⟩

theorem tElt_unfold :
    (consMap false).toElementMap
        (mergeMap.toElementMap (pair (negMap.toElementMap tElt) (tailMap.toElementMap
    tElt)))
      = tElt := by
  have h := toElementMap_fixElement tmOp
  rw [tmOp, toElementMap_comp, toElementMap_comp, toElementMap_paired] at h
  exact h

theorem tmList_length (n : ℕ) : (tmList n).length = n := by
  rw [tmList, List.length_map, List.length_range]

theorem tmList_succ (n : ℕ) : tmList (n + 1) = tmList n ++ [tm n] := by
  simp only [tmList, List.range_succ, List.map_append, List.map_cons, List.map_nil]

theorem tmList_prefix {n m : ℕ} (h : n ≤ m) : tmList n <+: tmList m := by
  induction m with
  | zero => rw [Nat.le_zero.mp h]
  | succ m ih =>
    rcases Nat.lt_succ_iff_lt_or_eq.mp (Nat.lt_succ_of_le h) with hlt | rfl
    · exact (ih (Nat.lt_succ_iff.mp hlt)).trans (by rw [tmList_succ]; exact List.
    prefix_append _ _)
    · exact List.prefix_rfl

theorem tmOp_strBot (σ : Str) :
    tmOp.toElementMap (strBot σ)
      = strBot (false :: (mergeVal (flip σ) false σ.tail false).1) := by
  unfold tmOp
  rw [toElementMap_comp, toElementMap_comp, toElementMap_paired, negMap_strBot,
    tailMap_strBot,
    show (strBot (flip σ)) = shapeElem false (flip σ) from rfl,
```

```
    show (strBot σ.tail) = shapeElem false σ.tail from rfl, mergeMap_pair, mergeElem,
    mergeVal_snd_false, shapeElem_false, consMap_strBot]

theorem tm_two_mul (n : ℕ) : tm (2 * n) = tm n := by
  rcases Nat.eq_zero_or_pos n with rfl | hn
  · rfl
  · simp only [tm, Nat.bit0_bits n (by omega : n = 0), List.foldr_cons, Bool.false_xor]

theorem tm_two_mul_add_one (n : ℕ) : tm (2 * n + 1) = !tm n := by
  simp only [tm, Nat.bit1_bits n, List.foldr_cons, Bool.true_xor]

theorem expand_tmList (m : ℕ) : expand (tmList m) = tmList (2 * m) := by
  induction m with
  | zero => rfl
  | succ m ih =>
    have key : tmList (2 * (m + 1)) = tmList (2 * m) ++ [tm m, !tm m] := by
      rw [show 2 * (m + 1) = 2 * m + 1 + 1 by ring, tmList_succ, tmList_succ,
        tm_two_mul_add_one, tm_two_mul, List.append_assoc]
      rfl
    rw [tmList_succ, expand_append, ih, key, expand_cons, expand_nil]

theorem expand_iterate_eq (k : ℕ) : (expand^[k] [false] : Str) = tmList (2 ^ k) := by
  induction k with
  | zero => rfl
  | succ k ih =>
    rw [Function.iterate_succ_apply', ih, expand_tmList, show 2 * 2 ^ k = 2 ^ (k + 1)
   by ring]

theorem weave_head : ∀ (h : Bool) (? : Str), (mergeVal (h :: flip ?) false ? false).1 =
     h :: expand ?
  | h, [] => rfl
  | h, c :: ? => by
    simp only [flip_cons, mergeVal_cons_cons, expand_cons]
    rw [weave_head (!c) ?]

theorem tmOp_strBot_expand (? : Str) :
    tmOp.toElementMap (strBot (false :: ?)) = strBot (expand (false :: ?)) := by
  rw [tmOp_strBot]
  simp only [flip_cons, List.tail_cons, expand_cons]
  rw [weave_head (!false) ?]

theorem iterElem_succ_eq (k : ℕ) :
    tmOp.iterElem (k + 1) = strBot (expand^[k] [false]) := by
  induction k with
  | zero =>
    rw [iterElem_succ, iterElem_zero, ← strBot_nil, tmOp_strBot]
    rfl
  | succ k ih =>
    obtain ⟨?, h?⟩ := iterate_expand_cons k
    rw [iterElem_succ, ih, h?, tmOp_strBot_expand, ← h?, Function.iterate_succ_apply']

open DomainTheory.NeighbourhoodSystems

def Overlap (i p : ℕ) : Prop := 1 ≤ p ∧ ∀ k, k ≤ p → tm (i + k) = tm (i + p + k)

open DomainTheory.NeighbourhoodSystems
```

```
open SeqDom BitDom NeighborhoodSystem

theorem append_three_period {α : Type*} (a : List α) (m : ℕ) (hm : m < 2 * a.length) :
    (a ++ (a ++ a))[m]? = (a ++ (a ++ a))[m + a.length]? := by
  rcases lt_or_ge m a.length with h | h
  · rw [List.getElem?_append_left h,
      List.getElem?_append_right (by omega : a.length ≤ m + a.length),
      show m + a.length - a.length = m by omega, List.getElem?_append_left h]
  · rw [List.getElem?_append_right h,
      List.getElem?_append_right (by omega : a.length ≤ m + a.length),
      show m + a.length - a.length = m by omega,
      List.getElem?_append_left (by omega : m - a.length < a.length),
      List.getElem?_append_right (by omega : a.length ≤ m)]

theorem tmList_getElem? {n j : ℕ} (h : j < n) : (tmList n)[j]? = some (tm j) := by
  rw [tmList, List.getElem?_map, List.getElem?_range h]
  rfl

open DomainTheory.NeighbourhoodSystems

theorem tm_of_eq_two_mul {x y : ℕ} (h : x = 2 * y) : tm x = tm y := by
  rw [h, tm_two_mul]

theorem tm_of_eq_two_mul_add_one {x y : ℕ} (h : x = 2 * y + 1) : tm x = !tm y := by
  rw [h, tm_two_mul_add_one]

theorem tm_two_mul_ne (x : ℕ) : tm (2 * x) = tm (2 * x + 1) := by
  rw [tm_two_mul, tm_two_mul_add_one]
  cases tm x <;> simp

theorem odd_of_consec_eq {x : ℕ} (h : tm x = tm (x + 1)) : Odd x := by
  rcases Nat.even_or_odd x with ⟨m, hm⟩ | hodd
  · exfalso; rw [hm, ← two_mul] at h; exact tm_two_mul_ne m h
  · exact hodd

theorem no_three_consec (j : ℕ) : ¬ (tm j = tm (j + 1) ∧ tm (j + 1) = tm (j + 2)) :=
    by
  rintro ⟨h1, h2⟩
  have hj : Odd j := odd_of_consec_eq h1
  have hj1 : Odd (j + 1) := odd_of_consec_eq h2
  rcases hj with ⟨a, rfl⟩
  rcases hj1 with ⟨b, hb⟩
  omega

theorem no_overlap (p : ℕ) : ∀ i, ¬ Overlap i p := by
  induction p using Nat.strong_induction_on with
  | _ p IH =>
    intro i hov
    obtain ⟨hp1, hrel⟩ := hov
    obtain ⟨q, hq | hq⟩ := Nat.even_or_odd' p
    ·
      subst hq
      have hq1 : 1 ≤ q := by omega
      obtain ⟨a, hi | hi⟩ := Nat.even_or_odd' i
      ·
        refine IH q (by omega) a ⟨hq1, fun c hc => ?_⟩
```

```
    have hk : 2 * c ≤ 2 * q := by omega
    have := hrel (2 * c) hk
    rw [tm_of_eq_two_mul (show i + 2 * c = 2 * (a + c) by omega),
      tm_of_eq_two_mul (show i + 2 * q + 2 * c = 2 * (a + q + c) by omega)] at
this
    exact this
  .
    refine IH q (by omega) a ⟨hq1, fun c hc => ?_⟩
    have hk : 2 * c ≤ 2 * q := by omega
    have := hrel (2 * c) hk
    rw [tm_of_eq_two_mul_add_one (show i + 2 * c = 2 * (a + c) + 1 by omega),
      tm_of_eq_two_mul_add_one (show i + 2 * q + 2 * c = 2 * (a + q + c) + 1 by
omega)] at this
    exact Bool.not_inj this
.
  subst hq
  rcases Nat.eq_zero_or_pos q with rfl | hq1
  .
    refine no_three_consec i ⟨?_, ?_⟩
    · have := hrel 0 (by omega); simpa using this
    · have := hrel 1 (by omega); simpa [show i + 1 + 1 = i + 2 by omega] using this
  .
    obtain ⟨a, hi | hi⟩ := Nat.even_or_odd' i
    .
      subst hi
      have r0 := hrel 0 (by omega)
      have r1 := hrel 1 (by omega)
      have r2 := hrel 2 (by omega)
      have r3 := hrel 3 (by omega)
      rw [tm_of_eq_two_mul (show 2 * a + 0 = 2 * a by omega),
        tm_of_eq_two_mul_add_one (show 2 * a + (2 * q + 1) + 0 = 2 * (a + q) + 1
by omega)] at r0
      rw [tm_of_eq_two_mul_add_one (show 2 * a + 1 = 2 * a + 1 by omega),
        tm_of_eq_two_mul (show 2 * a + (2 * q + 1) + 1 = 2 * (a + q + 1) by omega)
] at r1
      rw [tm_of_eq_two_mul (show 2 * a + 2 = 2 * (a + 1) by omega),
        tm_of_eq_two_mul_add_one
          (show 2 * a + (2 * q + 1) + 2 = 2 * (a + q + 1) + 1 by omega)] at r2
      rw [tm_of_eq_two_mul_add_one (show 2 * a + 3 = 2 * (a + 1) + 1 by omega),
        tm_of_eq_two_mul (show 2 * a + (2 * q + 1) + 3 = 2 * (a + q + 2) by omega)
] at r3
      refine no_three_consec (a + q) ⟨?_, ?_⟩
      · revert r0 r1; cases tm a <;> cases tm (a + q) <;> cases tm (a + q + 1) <;>
          simp_all
      · revert r1 r2 r3
        cases tm (a + 1) <;> cases tm (a + q + 1) <;> cases tm (a + q + 2) <;>
cases tm a <;>
          simp_all
    .
      subst hi
      rcases Nat.eq_or_lt_of_le hq1 with hq2 | hq2
      .
        subst hq2
        have r0 := hrel 0 (by omega)
        have r1 := hrel 1 (by omega)
        have r2 := hrel 2 (by omega)
```

```
          have r3 := hrel 3 (by omega)
          rw [tm_of_eq_two_mul_add_one (show 2 * a + 1 + 0 = 2 * a + 1 by omega),
            tm_of_eq_two_mul (show 2 * a + 1 + (2 * 1 + 1) + 0 = 2 * (a + 2) by
    omega)] at r0
          rw [tm_of_eq_two_mul (show 2 * a + 1 + 1 = 2 * (a + 1) by omega),
            tm_of_eq_two_mul_add_one
              (show 2 * a + 1 + (2 * 1 + 1) + 1 = 2 * (a + 2) + 1 by omega)] at r1
          rw [tm_of_eq_two_mul_add_one (show 2 * a + 1 + 2 = 2 * (a + 1) + 1 by
    omega),
            tm_of_eq_two_mul (show 2 * a + 1 + (2 * 1 + 1) + 2 = 2 * (a + 3) by
    omega)] at r2
          rw [tm_of_eq_two_mul (show 2 * a + 1 + 3 = 2 * (a + 2) by omega),
            tm_of_eq_two_mul_add_one
              (show 2 * a + 1 + (2 * 1 + 1) + 3 = 2 * (a + 3) + 1 by omega)] at r3
          revert r0 r1 r2 r3
          cases tm a <;> cases tm (a + 1) <;> cases tm (a + 2) <;> cases tm (a + 3)
    <;> simp_all
        .
          have r1 := hrel 1 (by omega)
          have r2 := hrel 2 (by omega)
          have r3 := hrel 3 (by omega)
          have r4 := hrel 4 (by omega)
          rw [tm_of_eq_two_mul (show 2 * a + 1 + 1 = 2 * (a + 1) by omega),
            tm_of_eq_two_mul_add_one
              (show 2 * a + 1 + (2 * q + 1) + 1 = 2 * (a + q + 1) + 1 by omega)] at
    r1
          rw [tm_of_eq_two_mul_add_one (show 2 * a + 1 + 2 = 2 * (a + 1) + 1 by
    omega),
            tm_of_eq_two_mul
              (show 2 * a + 1 + (2 * q + 1) + 2 = 2 * (a + q + 2) by omega)] at r2
          rw [tm_of_eq_two_mul (show 2 * a + 1 + 3 = 2 * (a + 2) by omega),
            tm_of_eq_two_mul_add_one
              (show 2 * a + 1 + (2 * q + 1) + 3 = 2 * (a + q + 2) + 1 by omega)] at
    r3
          rw [tm_of_eq_two_mul_add_one (show 2 * a + 1 + 4 = 2 * (a + 2) + 1 by
    omega),
            tm_of_eq_two_mul
              (show 2 * a + 1 + (2 * q + 1) + 4 = 2 * (a + q + 3) by omega)] at r4
          refine no_three_consec (a + q + 1) ⟨?_, ?_⟩
          · revert r1 r2; cases tm (a + 1) <;> cases tm (a + q + 1) <;> cases tm (a +
     q + 2) <;>
              simp_all
          · revert r3 r4; cases tm (a + 2) <;> cases tm (a + q + 2) <;> cases tm (a +
     q + 3) <;>
              simp_all

theorem no_cube (i p : ℕ) (hp : 1 ≤ p) :
    ¬ ∀ k, k < 2 * p → tm (i + k) = tm (i + p + k) := by
  intro hcube
  exact no_overlap p i ⟨hp, fun k hk => hcube k (by omega)⟩

theorem tmList_le_tElt (n : ℕ) : strBot (tmList n) ≤ tElt := by
  have hpre : tmList n <+: expand^[n] [false] := by
    rw [expand_iterate_eq]; exact tmList_prefix (Nat.le_of_lt (Nat.lt_two_pow_self))
  calc strBot (tmList n) ≤ strBot (expand^[n] [false]) :=
       strBot_le_strBot_iff.mpr hpre
```

```
    _ = tmOp.iterElem (n + 1) := (iterElem_succ_eq n).symm
    _ ≤ tElt := iterElem_le_fixElement tmOp (n + 1)

theorem tElt_digit (n : ℕ) : strBot (tmList n ++ [tm n]) ≤ tElt := by
  have h := tmList_le_tElt (n + 1)
  rwa [tmList_succ] at h

theorem tElt_mem_cone_iff (σ : Str) : tElt.mem (cone σ) ↔ σ = tmList σ.length := by
  constructor
  · intro hσ
    rw [show tElt = tmOp.fixElement from rfl, tmOp.fixElement_eq_iSupDirected,
      mem_iSupDirected] at hσ
    obtain ⟨n, hn⟩ := hσ

    obtain ⟨N, hpre⟩ : ∃ N, σ <+: tmList N := by
      cases n with
      | zero =>
        rw [iterElem_zero, mem_bot, C_master, cone_univ_iff] at hn
        exact ⟨0, by rw [hn]; exact List.nil_prefix⟩
      | succ k =>
        rw [iterElem_succ_eq, strBot_mem_cone, expand_iterate_eq] at hn
        exact ⟨2 ^ k, hn⟩

    have hlen : σ.length ≤ N := hpre.length_le.trans_eq (tmList_length _)
    have h1 : σ = (tmList N).take σ.length := List.prefix_iff_eq_take.mp hpre
    have h2 : tmList σ.length = (tmList N).take σ.length := by
      have := List.prefix_iff_eq_take.mp (tmList_prefix hlen)
      rwa [tmList_length] at this
    exact h1.trans h2.symm
  · intro hσ
    have h := tmList_le_tElt σ.length
    have hmem : (strBot (tmList σ.length)).mem (cone σ) :=
      strBot_mem_cone.mpr (hσ ▸ List.prefix_rfl)
    exact h _ hmem

open DomainTheory.NeighbourhoodSystems
open SeqDom BitDom NeighborhoodSystem

theorem tElt_cube_free (u a : Str) (ha : a = []) :
    ¬ strBot (u ++ a ++ a ++ a) ≤ tElt := by
  intro hle

  have hmem : tElt.mem (cone (u ++ a ++ a ++ a)) :=
    hle _ (strBot_mem_cone.mpr List.prefix_rfl)
  have heq : u ++ a ++ a ++ a = tmList (u ++ a ++ a ++ a).length :=
    (tElt_mem_cone_iff _).mp hmem

  set N := (u ++ a ++ a ++ a).length with hN
  have hlen : N = u.length + 3 * a.length := by simp only [hN, List.length_append];
    omega
  have hp : 1 ≤ a.length := by
    cases a with
    | nil => exact (ha rfl).elim
    | cons _ _ => simp
  refine no_cube u.length a.length hp (fun k hk => ?_)
```

```
  have hk3 : u.length + k < N := by omega
  have hk3' : u.length + a.length + k < N := by omega
  have e1 : (tmList N)[u.length + k]? = some (tm (u.length + k)) := tmList_getElem?
    hk3
  have e2 : (tmList N)[u.length + a.length + k]? = some (tm (u.length + a.length + k))
     :=
    tmList_getElem? hk3'

  have hassoc : u ++ a ++ a ++ a = u ++ (a ++ (a ++ a)) := by
    simp [List.append_assoc]
  rw [← heq, hassoc] at e1 e2
  rw [List.getElem?_append_right (by omega : u.length ≤ u.length + k),
    show u.length + k - u.length = k by omega] at e1
  rw [List.getElem?_append_right (by omega : u.length ≤ u.length + a.length + k),
    show u.length + a.length + k - u.length = k + a.length by omega] at e2

  have hper := append_three_period a k (by omega)
  have : some (tm (u.length + k)) = some (tm (u.length + a.length + k)) := by
    rw [← e1, hper, e2]
  exact Option.some.inj this

end DomainTheory.NeighbourhoodSystems.SeqNegMerge
```

4.6.36 Exercise 5.16 — negation of sequences

Return to the discussion of Example 4.4 and the construction of the domain of finite and infinite binary sequences. Give a fixed-point definition of $\mathrm{neg} : C \to C$, where

$$\mathrm{neg}(0x) = 1\,\mathrm{neg}(x); \qquad \mathrm{neg}(1x) = 0\,\mathrm{neg}(x).$$

Prove that $\mathrm{neg}(\mathrm{neg}(x)) = x$ for all $x \in |C|$. Also define $\mathrm{merge} : C \times C \to C$, where for $\epsilon, \delta \in \{0, 1\}$ we have:

$$\mathrm{merge}(\epsilon x, \delta y) = \epsilon\,\delta\,\mathrm{merge}(x, y).$$

(Note: There may be a little trouble with $\mathrm{merge}(x, y)$ when x is finite and total and y is infinite—you have to decide what you want in e.g. $\mathrm{merge}(\Lambda, y)$.) Prove that

$$\mathrm{merge}(x, x) = d(x),$$

in the notation of 4.4. Consider also the infinite non-periodic sequence

$$t = 0\,\mathrm{merge}(\mathrm{neg}(t), \mathrm{tail}(t)).$$

Prove that the nth digit of t is the sum mod 2 of the digits of the number n written in the binary scale (a suggestion of J. Lambek). Show also that $t \neq u\,a\,a\,a\,v$ where a is any finite sequence $\neq \Lambda$, and where u is finite.

The development is with the sequence negation and merge peer.

Theorem (`example_516`). Negation is an involution on the stream domain; merge on the diagonal recovers doubling; and no principal of a cube prefix with nonempty period lies below the Thue–Morse stream.

Proof. The involution, diagonal-merge, and cube-freeness laws from the sequence-negation-and-merge peer. ■

Exercise_5_16.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.seqNegMerge

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap BitDom SeqDom SeqNegMerge

/-- Scott's Exercise 5.16: negation is an involution, merge on the diagonal is
    doubling,
and the Thue–Morse stream is cube-free. -/
theorem example_516 :
    (∀ x : C.Element, negMap.toElementMap (negMap.toElementMap x) = x) ∧
    (∀ x : C.Element, mergeMap.toElementMap (pair x x) = dMap.toElementMap x) ∧
    (∀ u a : Str, a = [] → ¬ strBot (u ++ a ++ a ++ a) ≤ tElt) :=
  ⟨negMap_negMap, mergeMap_diag, tElt_cube_free⟩
```

4.7 Domain equations

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>



Figure 51: Diagram 16

4.7.1 Introduction

The major reason for introducing the theory of domains is to have a notion of *computability* incorporating both finite and infinite elements. In our many examples already explored we have seen how functions (functionals, operators, combinators) can be defined on domains; owing to the property of *approximability (continuity)* of these functions, we have also seen how they can be "calculated" by finite approximation. In this lecture further examples of domains will be constructed – especially domains having infinite elements,

which can be introduced in a variety of ways giving rise to interesting structural possibilities. The next lecture then treats a precise notion of *computability* appropriate to these domains; while the last lecture opens up new methods of domain construction.

4.7.2 Indefinite product

Indefinite product domain: embeddings of atoms and pairs, the generated neighbourhood system, and its characterisation as Scott's D♯.

Definition (`Gamma`). As defined.
Proof. By definition. ■
Definition (`embL`). As defined.
Proof. By definition. ■
Definition (`embR`). As defined.
Proof. By definition. ■
Definition (`embPair`). As defined.
Proof. By definition. ■
Definition (`embZero`). As defined.
Proof. By definition. ■
Definition (`MemS`). As defined.
Proof. By definition. ■
Theorem (`mem_embZero`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`mem_embL`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`mem_embR`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`mem_embPair`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`mem_Gamma`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`Gamma_nonempty`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`embPair_inter`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`embPair_nonempty`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`embPair_subset`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`embPair_injective`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`embPair_subset_Gamma`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`embZero_inter`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`embZero_inter_embPair`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■

Theorem (`embZero_nonempty`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`embZero_subset`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`embZero_injective`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`embZero_subset_Gamma`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`memS_nonempty`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`memS_subset_gamma`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`memS_inter`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Definition (`Dsharp`). As defined.
Proof. By definition. ■
Theorem (`Dsharp_mem`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`Dsharp_master`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`memS_embPair_inv`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`memS_embZero_inv`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■
Theorem (`prod_dsharp_nonempty`). The named law for embeddings, MemS, or D♯.
Proof. Unpack the definitions and the neighbourhood-system axioms. ■

`indefProduct.lean`

```
module

public import Mathlib.Data.Set.Basic
public import Mathlib.Tactic.Tauto

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodNbhd

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.IndefProduct
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

variable {α : Type*}

variable {α : Type*}

def Gamma (D : NeighborhoodSystem α) : Set (List Bool × α) := {t | t.2 ∈ D.master}
```

```
def embL (P : Set (List Bool × α)) : Set (List Bool × α) := {t | ∃ p', t.1 = true :: p
    ' ∧ (p', t.2) ∈ P}

def embR (Q : Set (List Bool × α)) : Set (List Bool × α) := {t | ∃ q', t.1 = false ::
    q' ∧ (q', t.2) ∈ Q}

def embPair (P Q : Set (List Bool × α)) : Set (List Bool × α) := embL P ∪ embR Q

def embZero (X : Set α) : Set (List Bool × α) := {t | t.1 = [] ∧ t.2 ∈ X}

inductive MemS (D : NeighborhoodSystem α) : Set (List Bool × α) → Prop
  | gamma : MemS D (Gamma D)
  | zero {X : Set α} : D.mem X → MemS D (embZero X)
  | pair {P Q : Set (List Bool × α)} : MemS D P → MemS D Q → MemS D (embPair P Q)

@[simp] theorem mem_embZero {X : Set α} {p : List Bool} {a : α} :
    (p, a) ∈ embZero X ↔ p = [] ∧ a ∈ X := Iff.rfl

@[simp] theorem mem_embL {P : Set (List Bool × α)} {p : List Bool} {a : α} :
    (p, a) ∈ embL P ↔ ∃ p', p = true :: p' ∧ (p', a) ∈ P := Iff.rfl

@[simp] theorem mem_embR {Q : Set (List Bool × α)} {p : List Bool} {a : α} :
    (p, a) ∈ embR Q ↔ ∃ q', p = false :: q' ∧ (q', a) ∈ Q := Iff.rfl

@[simp] theorem mem_embPair {P Q : Set (List Bool × α)} {p : List Bool} {a : α} :
    (p, a) ∈ embPair P Q ↔
      (∃ p', p = true :: p' ∧ (p', a) ∈ P) ∨ (∃ q', p = false :: q' ∧ (q', a) ∈ Q) :=
    Iff.rfl

@[simp] theorem mem_Gamma {D : NeighborhoodSystem α} {p : List Bool} {a : α} :
    (p, a) ∈ Gamma D ↔ a ∈ D.master := Iff.rfl

theorem Gamma_nonempty {D : NeighborhoodSystem α} (hD : ∀ X, D.mem X → X.Nonempty) :
    (Gamma D).Nonempty := by
  obtain ⟨a, ha⟩ := hD D.master D.master_mem; exact ⟨([], a), ha⟩

theorem embPair_inter (P Q P' Q' : Set (List Bool × α)) :
    embPair P Q ∩ embPair P' Q' = embPair (P ∩ P') (Q ∩ Q') := by
  ext ⟨p, a⟩
  simp only [Set.mem_inter_iff, mem_embPair]
  rcases p with _ | ⟨b, ps⟩
  · simp
  · cases b <;> simp [List.cons.injEq]

theorem embPair_nonempty {P Q : Set (List Bool × α)} (hP : P.Nonempty) :
    (embPair P Q).Nonempty := by
  obtain ⟨⟨p', a⟩, hP⟩ := hP; exact ⟨(true :: p', a), Or.inl ⟨p', rfl, hP⟩⟩

theorem embPair_subset {P Q P' Q' : Set (List Bool × α)} :
    embPair P Q ⊆ embPair P' Q' ↔ P ⊆ P' ∧ Q ⊆ Q' := by
  constructor
  · intro h
    refine ⟨fun t ht => ?_, fun t ht => ?_⟩
    · obtain ⟨p', a⟩ := t
      have hmem : ((true :: p', a) : List Bool × α) ∈ embPair P' Q' :=
        h (Or.inl ⟨p', rfl, ht⟩)
```

```
      rcases hmem with ⟨p'', hp'', hP'⟩ | ⟨q'', hq'', -⟩
      · simp only [List.cons.injEq, true_and] at hp''; exact hp'' ▸ hP'
      · exact absurd hq'' (by simp)
    · obtain ⟨q', a⟩ := t
      have hmem : ((false :: q', a) : List Bool × α) ∈ embPair P' Q' :=
        h (Or.inr ⟨q', rfl, ht⟩)
      rcases hmem with ⟨p'', hp'', -⟩ | ⟨q'', hq'', hQ'⟩
      · exact absurd hp'' (by simp)
      · simp only [List.cons.injEq, true_and] at hq''; exact hq'' ▸ hQ'
  · rintro ⟨hP, hQ⟩ t ht
    rcases ht with ⟨p', hp', hPmem⟩ | ⟨q', hq', hQmem⟩
    · exact Or.inl ⟨p', hp', hP hPmem⟩
    · exact Or.inr ⟨q', hq', hQ hQmem⟩

theorem embPair_injective {P Q P' Q' : Set (List Bool × α)}
    (h : embPair P Q = embPair P' Q') : P = P' ∧ Q = Q' := by
  obtain ⟨hP, hQ⟩ := embPair_subset.mp h.subset
  obtain ⟨hP', hQ'⟩ := embPair_subset.mp h.symm.subset
  exact ⟨Set.Subset.antisymm hP hP', Set.Subset.antisymm hQ hQ'⟩

theorem embPair_subset_Gamma {D : NeighborhoodSystem α} {P Q : Set (List Bool × α)}
    (hP : P ⊆ Gamma D) (hQ : Q ⊆ Gamma D) : embPair P Q ⊆ Gamma D := by
  rintro ⟨p, a⟩ (⟨p', -, hPmem⟩ | ⟨q', -, hQmem⟩)
  · have h := hP hPmem; exact h
  · have h := hQ hQmem; exact h

theorem embZero_inter (X X' : Set α) :
    embZero X ∩ embZero X' = embZero (X ∩ X' : Set α) := by
  ext ⟨p, a⟩
  simp only [Set.mem_inter_iff, mem_embZero]
  tauto

theorem embZero_inter_embPair (X : Set α) (P Q : Set (List Bool × α)) :
    embZero X ∩ embPair P Q = ∅ := by
  ext ⟨p, a⟩
  simp only [Set.mem_inter_iff, mem_embZero, mem_embPair, Set.mem_empty_iff_false,
    iff_false,
    not_and, not_or]
  rintro ⟨rfl, -⟩
  exact ⟨by rintro ⟨p', hp', -⟩; exact absurd hp' (by simp), by rintro ⟨q', hq', -⟩;
    exact absurd hq' (by simp)⟩

theorem embZero_nonempty {X : Set α} (hX : X.Nonempty) : (embZero X).Nonempty := by
  obtain ⟨a, ha⟩ := hX; exact ⟨([], a), rfl, ha⟩

theorem embZero_subset {X X' : Set α} : embZero X ⊆ embZero X' ↔ X ⊆ X' := by
  constructor
  · intro h a ha
    have : (([], a) : List Bool × α) ∈ embZero X' := h ⟨rfl, ha⟩
    exact this.2
  · intro h t ht; exact ⟨ht.1, h ht.2⟩

theorem embZero_injective {X X' : Set α} (h : embZero X = embZero X') : X = X' :=
  Set.Subset.antisymm (embZero_subset.mp h.subset) (embZero_subset.mp h.symm.subset)

theorem embZero_subset_Gamma {D : NeighborhoodSystem α} {X : Set α} (hX : D.mem X) :
```

```
    embZero X ⊆ Gamma D := by
  rintro ⟨p, a⟩ ⟨-, ha⟩; exact D.sub_master hX ha

theorem memS_nonempty {D : NeighborhoodSystem α} (hD : ∀ X, D.mem X → X.Nonempty)
    {W : Set (List Bool × α)} (hW : MemS D W) : W.Nonempty := by
  induction hW with
  | gamma => exact Gamma_nonempty hD
  | zero hX => exact embZero_nonempty (hD _ hX)
  | pair _ _ ihP _ => exact embPair_nonempty ihP

theorem memS_subset_gamma {D : NeighborhoodSystem α} {W : Set (List Bool × α)}
    (hW : MemS D W) : W ⊆ Gamma D := by
  induction hW with
  | gamma => exact subset_rfl
  | zero hX => exact embZero_subset_Gamma hX
  | pair _ _ ihP ihQ => exact embPair_subset_Gamma ihP ihQ

theorem memS_inter {D : NeighborhoodSystem α} (hD : ∀ X, D.mem X → X.Nonempty) :
    ∀ {X : Set (List Bool × α)}, MemS D X → ∀ {Y : Set (List Bool × α)}, MemS D Y →
      ∀ {Z : Set (List Bool × α)}, MemS D Z → Z ⊆ X ∩ Y → MemS D (X ∩ Y) := by
  intro X hX
  induction hX with
  | gamma =>
    intro Y hY Z _ _
    rw [Set.inter_eq_right.mpr (memS_subset_gamma hY)]; exact hY
  | @zero A hA =>
    intro Y hY Z hZ hsub
    cases hY with
    | gamma =>
      rw [Set.inter_eq_left.mpr (embZero_subset_Gamma hA)]; exact MemS.zero hA
    | @zero B hB =>
      rw [embZero_inter]

      rw [embZero_inter] at hsub
      cases hZ with
      | gamma =>
        obtain ⟨a, ha⟩ := hD D.master D.master_mem
        exact absurd (hsub (show ((true :: [], a) : List Bool × α) ∈ Gamma D from ha))
     (by simp)
      | @zero C hC =>
        exact MemS.zero (D.inter_mem hA hB hC (embZero_subset.mp hsub))
      | @pair P Q hP hQ =>
        obtain ⟨t, hmem⟩ := embPair_nonempty (memS_nonempty hD hP) (Q := Q)
        have hempty : t.1 = [] := (hsub hmem).1
        rcases hmem with ⟨p', hp', -⟩ | ⟨q', hq', -⟩
        · rw [hp'] at hempty; exact absurd hempty (by simp)
        · rw [hq'] at hempty; exact absurd hempty (by simp)
    | @pair P Q _ _ =>
      rw [embZero_inter_embPair] at hsub
      obtain ⟨z, hz⟩ := memS_nonempty hD hZ
      exact absurd (hsub hz) (Set.notMem_empty z)
  | @pair P Q hP hQ ihP ihQ =>
    intro Y hY Z hZ hsub
    cases hY with
    | gamma =>
      rw [Set.inter_eq_left.mpr (embPair_subset_Gamma (memS_subset_gamma hP) (
```

```
    memS_subset_gamma hQ))]
      exact MemS.pair hP hQ
    | @zero B hB =>
      rw [Set.inter_comm, embZero_inter_embPair] at hsub
      obtain ⟨z, hz⟩ := memS_nonempty hD hZ
      exact absurd (hsub hz) (Set.notMem_empty z)
    | @pair P' Q' hP' hQ' =>
      rw [embPair_inter]
      rw [embPair_inter] at hsub
      cases hZ with
      | gamma =>
        obtain ⟨a, ha⟩ := hD D.master D.master_mem
        exact absurd (hsub (show (([], a) : List Bool × α) ∈ Gamma D from ha)) (by
    simp)
      | @zero C hC =>
        obtain ⟨a, ha⟩ := hD C hC
        exact absurd (hsub (show (([], a) : List Bool × α) ∈ embZero C from ⟨rfl, ha⟩))
     (by simp)
      | @pair Z₀ Z₁ hZ₀ hZ₁ =>
        obtain ⟨hsub₀, hsub₁⟩ := embPair_subset.mp hsub
        exact MemS.pair (ihP hP' hZ₀ hsub₀) (ihQ hQ' hZ₁ hsub₁)

abbrev Dsharp (D : NeighborhoodSystem α) (hD : ∀ X, D.mem X → X.Nonempty) :
    NeighborhoodSystem (List Bool × α) where
  mem := MemS D
  master := Gamma D
  master_nonempty := by
    obtain ⟨a, ha⟩ := D.master_nonempty
    exact ⟨([], a), ha⟩
  master_mem := MemS.gamma
  inter_mem := fun hX hY hZ hsub => memS_inter hD hX hY hZ hsub
  sub_master := memS_subset_gamma

@[simp] theorem Dsharp_mem {D : NeighborhoodSystem α} {hD : ∀ X, D.mem X → X.Nonempty}
    {W : Set (List Bool × α)} : (Dsharp D hD).mem W ↔ MemS D W := Iff.rfl

@[simp] theorem Dsharp_master {D : NeighborhoodSystem α} {hD : ∀ X, D.mem X → X.
    Nonempty} :
    (Dsharp D hD).master = Gamma D := rfl

theorem memS_embPair_inv {D : NeighborhoodSystem α} (hD : ∀ X, D.mem X → X.Nonempty)
    {P Q : Set (List Bool × α)} (h : MemS D (embPair P Q)) : MemS D P ∧ MemS D Q := by
  generalize hW : embPair P Q = W at h
  cases h with
  | gamma =>
    exfalso
    obtain ⟨a, ha⟩ := hD D.master D.master_mem
    have h1 : (([], a) : List Bool × α) ∈ Gamma D := ha
    rw [← hW] at h1
    rcases h1 with ⟨p', hp', -⟩ | ⟨q', hq', -⟩
    · simp at hp'
    · simp at hq'
  | @zero X hX =>
    exfalso
    obtain ⟨t, ht⟩ := embZero_nonempty (hD X hX)
    have hempty : t.1 = [] := ht.1
```

```
    rw [← hW] at ht
    rcases ht with ⟨p', hp', -⟩ | ⟨q', hq', -⟩
    · rw [hp'] at hempty; simp at hempty
    · rw [hq'] at hempty; simp at hempty
  | @pair P' Q' hP' hQ' =>
    obtain ⟨hPP, hQQ⟩ := embPair_injective hW
    rw [hPP, hQQ]; exact ⟨hP', hQ'⟩

theorem memS_embZero_inv {D : NeighborhoodSystem α} (hD : ∀ X, D.mem X → X.Nonempty) {
    X : Set α}
    (h : MemS D (embZero X)) : D.mem X := by
  generalize hW : embZero X = W at h
  cases h with
  | gamma =>
    exfalso
    obtain ⟨a, ha⟩ := hD D.master D.master_mem
    have h1 : ((true :: [], a) : List Bool × α) ∈ Gamma D := ha
    rw [← hW] at h1; exact absurd h1.1 (by simp)
  | @zero X' hX' => rw [embZero_injective hW]; exact hX'
  | @pair P Q hP hQ =>
    exfalso
    obtain ⟨t, ht⟩ := embPair_nonempty (memS_nonempty hD hP) (Q := Q)
    have hcons : t.1 = [] := by
      rcases ht with ⟨p', hp', -⟩ | ⟨q', hq', -⟩
      · simp [hp']
      · simp [hq']
    have hz : t ∈ embZero X := by rw [← hW] at ht; exact ht
    exact hcons hz.1

variable (D : NeighborhoodSystem α) (hD : ∀ X, D.mem X → X.Nonempty)

include hD

theorem prod_dsharp_nonempty :
    ∀ V, (prod (Dsharp D hD) (Dsharp D hD)).mem V → V.Nonempty := by
  rintro V ⟨P, Q, hP, hQ, rfl⟩
  obtain ⟨t, ht⟩ := memS_nonempty hD hP
  exact ⟨Sum.inl t, prodNbhd.mem_inl.mpr ht⟩

theorem embZero_ne_Gamma (X : Set α) : embZero X = Gamma D := by
  intro h
  obtain ⟨a, ha⟩ := hD D.master D.master_mem
  have hmem : ((true :: [], a) : List Bool × α) ∈ Gamma D := ha
  rw [← h] at hmem; exact absurd hmem.1 (by simp)

theorem embPair_ne_Gamma (P Q : Set (List Bool × α)) : embPair P Q = Gamma D := by
  intro h
  obtain ⟨a, ha⟩ := hD D.master D.master_mem
  have hmem : (([], a) : List Bool × α) ∈ Gamma D := ha
  rw [← h] at hmem
  rcases hmem with ⟨p', hp', -⟩ | ⟨q', hq', -⟩
  · simp at hp'
  · simp at hq'

theorem embZero_ne_embPair {X : Set α} (hX : D.mem X) (P Q : Set (List Bool × α)) :
    embZero X = embPair P Q := by
```

```
  intro h
  obtain ⟨a, ha⟩ := hD X hX
  have hmem : (([], a) : List Bool × α) ∈ embPair P Q := h ▶ (⟨rfl, ha⟩ : ([], a) ∈
    embZero X)
  rcases hmem with ⟨p', hp', -⟩ | ⟨q', hq', -⟩
  · simp at hp'
  · simp at hq'

/-- Scott's injection `x^§ = {Γ} ∪ {0X ? X ∈ x}` of a `D`-element into `D^§`. -/
def inSharp (x : D.Element) : (Dsharp D hD).Element where
  mem W := W = Gamma D ∨ ∃ X, x.mem X ∧ W = embZero X
  sub := by
    rintro W (rfl | ⟨X, hX, rfl⟩)
    · exact MemS.gamma
    · exact MemS.zero (x.sub hX)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl⟩) (rfl | ⟨X', hX', rfl⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr ⟨X', hX', by rw [Set.inter_eq_right.mpr (embZero_subset_Gamma (x.sub
      hX'))]⟩
    · exact Or.inr ⟨X, hX, by rw [Set.inter_eq_left.mpr (embZero_subset_Gamma (x.sub hX
    ))]⟩
    · exact Or.inr ⟨X ∩ X', x.inter_mem hX hX', by rw [embZero_inter]⟩
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl⟩) hW' hsub
    · exact Or.inl (Set.Subset.antisymm (memS_subset_gamma hW') hsub)
    · cases hW' with
      | gamma => exact Or.inl rfl
      | @zero X' hX' => exact Or.inr ⟨X', x.up_mem hX hX' (embZero_subset.mp hsub), rfl
    ⟩
      | @pair P Q hP hQ =>
        exfalso
        obtain ⟨a, ha⟩ := hD X (x.sub hX)
        have hmem : (([], a) : List Bool × α) ∈ embPair P Q := hsub ⟨rfl, ha⟩
        rcases hmem with ⟨p', hp', -⟩ | ⟨q', hq', -⟩
        · simp at hp'
        · simp at hq'

theorem inSharp_le_iff {x x' : D.Element} : inSharp D hD x ≤ inSharp D hD x' ↔ x ≤ x'
     := by
  constructor
  · intro h X hX
    rcases h (embZero X) (Or.inr ⟨X, hX, rfl⟩) with h0 | ⟨X', hX', heq⟩
    · exact absurd h0 (embZero_ne_Gamma D hD X)
    · rw [← embZero_injective heq] at hX'; exact hX'
  · intro h W hW
    rcases hW with rfl | ⟨X, hX, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr ⟨X, h _ hX, rfl⟩

/-- Scott's pairing `⟨x, y⟩ = {Γ} ∪ {1P ∪ 2Q ? P ∈ x, Q ∈ y}` of two `D^§`-elements. -/
def pairSharp (x y : (Dsharp D hD).Element) : (Dsharp D hD).Element where
  mem W := W = Gamma D ∨ ∃ P Q, x.mem P ∧ y.mem Q ∧ W = embPair P Q
  sub := by
    rintro W (rfl | ⟨P, Q, hP, hQ, rfl⟩)
```

```
    · exact MemS.gamma
    · exact MemS.pair (x.sub hP) (y.sub hQ)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨P, Q, hP, hQ, rfl⟩) (rfl | ⟨P', Q', hP', hQ', rfl⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr ⟨P', Q', hP', hQ', by
        rw [Set.inter_eq_right.mpr (embPair_subset_Gamma (memS_subset_gamma (x.sub hP
    '))
          (memS_subset_gamma (y.sub hQ'))))]⟩
    · exact Or.inr ⟨P, Q, hP, hQ, by
        rw [Set.inter_eq_left.mpr (embPair_subset_Gamma (memS_subset_gamma (x.sub hP))
          (memS_subset_gamma (y.sub hQ))))]⟩
    · exact Or.inr ⟨P ∩ P', Q ∩ Q', x.inter_mem hP hP', y.inter_mem hQ hQ', by rw [
    embPair_inter]⟩
  up_mem := by
    rintro W W' (rfl | ⟨P, Q, hP, hQ, rfl⟩) hW' hsub
    · exact Or.inl (Set.Subset.antisymm (memS_subset_gamma hW') hsub)
    · cases hW' with
      | gamma => exact Or.inl rfl
      | @zero X' hX' =>
        exfalso
        obtain ⟨t, ht⟩ := embPair_nonempty (memS_nonempty hD (x.sub hP)) (Q := Q)
        have hcons : t.1 = [] := by
          rcases ht with ⟨p', hp', -⟩ | ⟨q', hq', -⟩
          · simp [hp']
          · simp [hq']
        exact hcons (hsub ht).1
      | @pair P' Q' hP' hQ' =>
        obtain ⟨hPP, hQQ⟩ := embPair_subset.mp hsub
        exact Or.inr ⟨P', Q', x.up_mem hP hP' hPP, y.up_mem hQ hQ' hQQ, rfl⟩

theorem pairSharp_le_iff {x x' y y' : (Dsharp D hD).Element} :
    pairSharp D hD x y ≤ pairSharp D hD x' y' ↔ x ≤ x' ∧ y ≤ y' := by
  constructor
  · intro h
    refine ⟨fun P hP => ?_, fun Q hQ => ?_⟩
    · rcases h (embPair P (Gamma D)) (Or.inr ⟨P, Gamma D, hP, y.master_mem, rfl⟩) with
        h0 | ⟨P', Q', hP', hQ', heq⟩
      · exact absurd h0 (embPair_ne_Gamma D hD P (Gamma D))
      · obtain ⟨hPP, -⟩ := embPair_injective heq; rw [← hPP] at hP'; exact hP'
    · rcases h (embPair (Gamma D) Q) (Or.inr ⟨Gamma D, Q, x.master_mem, hQ, rfl⟩) with
        h0 | ⟨P', Q', hP', hQ', heq⟩
      · exact absurd h0 (embPair_ne_Gamma D hD (Gamma D) Q)
      · obtain ⟨-, hQQ⟩ := embPair_injective heq; rw [← hQQ] at hQ'; exact hQ'
  · rintro ⟨hx, hy⟩ W hW
    rcases hW with rfl | ⟨P, Q, hP, hQ, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr ⟨P, Q, hx _ hP, hy _ hQ, rfl⟩

end DomainTheory.NeighbourhoodSystems.IndefProduct
```

4.7.3 Example 6.1 — indefinite product

EXAMPLE 6.1. Let D be fixed as a given domain. We are now familiar with a useful construct like $D \times D$ whose elements are ordered pairs $\langle x, y\rangle$ of elements x, y of D. The question is: can this construct be iterated? The answer is obviously yes, since $D \times (D \times D)$ and $(D \times D) \times (D \times D)$ and so on can be formed with elements $\langle x, \langle y, z\rangle\rangle$ and $\langle\langle u, v\rangle, \langle x, y\rangle\rangle$ and the like. But the real question is: can the construct be iterated *indefinitely*? AND can the results be collected together into a *single* domain? The answer is yes, but it requires a bit of work to get it right. The method to be introduced will be open to many variations, so more than one answer is possible, giving non-isomorphic domains.

In order to collect all the iterates into one large domain we give ourselves first a very big domain inside of which the desired family of neighbourhoods will be found. There are many ways to make this choice, and we are fixing on one that will keep the notation simple. We have often used binary sequences for examples and constructions, but for this example let us use

ternary sequences. Let $\Sigma = \{0, 1, 2\}$ and let Σ^* be all finite sequences from this three-letter alphabet. We will select subsets of Σ^* for our neighbourhoods. As Σ^* is countably infinite, it is without much loss of generality to assume that $\mathcal{D}$ is a neighbourhood system over Δ, where we take $\Delta \subseteq \Sigma^*$. Also without loss of generality we can assume $\emptyset \notin \mathcal{D}$. (Why?)

We wish to find another set $\Gamma \subseteq \Sigma^*$ to be the set of tokens for the new domain. After we find it, we will still have to say just which $X \subseteq \Gamma$ are appropriate for the structure we want.

The totality $\{X \mid X \subseteq \Sigma^*\}$ is, as a powerset, isomorphic to the set of elements of a domain: a point we have remarked several times. So, by the Fixed-Point Theorem we know there is a set $\Gamma \subseteq \Sigma^*$ where

$$\Gamma = 0\Delta \cup 1\Gamma \cup 2\Gamma.$$

In fact $\Gamma = \{1, 2\}^*0\Delta$, because we can say:

$$\{1, 2\}^* = \{\Lambda\} \cup 1\{1, 2\}^* \cup 2\{1, 2\}^*.$$

The domain we are looking for will be found as a domain $\mathcal{D}^\S$ over Γ. The reason for splitting Γ up, as shown in the equation above, is to ensure that if $X, Y \in \mathcal{D}^\S$ are two neighbourhoods in the system $\mathcal{D}^\S$, then $1X \cup 2Y$ has a chance of being also in $\mathcal{D}^\S$ because

$$1X \cup 2Y \subseteq \Gamma.$$

This will make $\mathcal{D}^\S \times \mathcal{D}^\S$ isomorphic to a part of $\mathcal{D}^\S$. If we make $\mathcal{D}$ also isomorphic to a part of $\mathcal{D}^\S$, then all the iterated products will be contained in $\mathcal{D}^\S$.

What is a neighbourhood system? Just a set of sets. But $\mathcal{P}\mathcal{P}\Sigma^*$ is a domain (as a power set) and because $\Gamma \subseteq \Sigma^*$, we find

$$\mathcal{D}^\S \in \mathcal{P}\mathcal{P}\Sigma^*$$

as an element. But elements of domains can often be defined by fixed-point equations. Indeed we will introduce $\mathcal{D}^\S$ this way:

$$\mathcal{D}^\S = \{\Gamma\} \cup \{0X \mid X \in \mathcal{D}\} \cup \{1X \cup 2Y \mid X, Y \in \mathcal{D}^\S\}.$$

The reader should stop to think why $\mathcal{D}^\S$ can be immediately seen to exist by writing such an equation. Of course another way to describe $\mathcal{D}^\S$ is to say it is the least family of sets containing (i) the set Γ, (ii) the sets $0X$ for X in the given system $\mathcal{D}$, and (iii) sets $1X \cup 2Y$ whenever it already contains X and Y (closure

under a set-forming operation). By saying "least" we mean (iv) nothing else belongs to $\mathcal{D}^\S$ except as allowed by (i)–(iii); this makes the truth of the equation for $\mathcal{D}^\S$ clear. So $\mathcal{D}^\S$ exists as a family of sets, but what good is it?

By our construction of Γ, all the sets we put into $\mathcal{D}^\S$ are subsets of Γ (why?); so $\mathcal{D}^\S$ has a chance of being a system over Γ if we can check the closure under intersection. So suppose $Z \subseteq X \cap Y$ where $Z, X, Y \in \mathcal{D}^\S$; we want to show $X \cap Y \in \mathcal{D}^\S$. We argue by induction on the number of steps required to put X and Y into $\mathcal{D}^\S$ by (i)–(iii). There are several cases.

If $X = \Gamma$ or $Y = \Gamma$, there is nothing to prove, because both sets are subsets of Γ. We note that $\emptyset \notin \mathcal{D}^\S$, because (i)–(iii) cannot introduce $\emptyset$ as a member of $\mathcal{D}^\S$. So, if $X = 0A$ for $A \in \mathcal{D}$, then Y must have this form also (if it is not Γ), because

$$0A \cap (1B \cup 2C) = \emptyset.$$

(That is, if Y had the form (iii), then $Z = \emptyset$ would be a consequence, which is impossible.) Thus, if $X = 0A$ for $A \in \mathcal{D}$, then $Y = 0B$ for some $B \in \mathcal{D}$. But by the same reasoning $Z = 0C$ for some $C \in \mathcal{D}$ also. But the relationship $0C \subseteq 0A \cap 0B$ is equivalent to $C \subseteq A \cap B$. We see, therefore, that $A \cap B \in \mathcal{D}$, and so

$$X \cap Y = 0A \cap 0B = 0(A \cap B)$$

must belong to $\mathcal{D}^\S$.

The final case has X, Y, Z all of the form (iii):

$$X = 1A_1 \cup 2A_2,$$

$$Y = 1B_1 \cup 2B_2, \text{ and}$$

$$Z = 1C_1 \cup 2C_2.$$

We can think of the A_i and B_i put into $\mathcal{D}^\S$ earlier and the intersection result as being already established for them. But the relationship $Z \subseteq X \cap Y$ is equivalent to $C_i \subseteq A_i \cap B_i$ for $i = 1, 2$. Therefore $A_i \cap B_i \in \mathcal{D}^\S$, and so does

$$X \cap Y = (1A_1 \cup 2A_2) \cap (1B_1 \cup 2B_2) = 1(A_1 \cap B_1) \cup 2(A_2 \cap B_2).$$

We have now seen that $D^\S$ is a neighbourhood system, but why was it constructed that way? The reason is simply this isomorphism (or domain equation):

$$D^\S \cong D + (D^\S \times D^\S),$$

as can be seen by reference to the equation for $D^\S$ and the definitions of $+$ and $\times$. What are the elements of $D^\S$? There is always

$$\perp = \{\Gamma\}.$$

Next if $x \in |D|$ we define

$$x^\S = \{\Gamma\} \cup \{0X \mid X \in x\}.$$

That gives an isomorphic injection

$$\lambda x.\, x^{\S} : D \to D^{\S}.$$

Then for $x, y \in |D^{\S}|$ we can define

$$\langle x, y\rangle = \{\Gamma\} \cup \{1X \cup 2Y \mid X \in x \text{ and } Y \in y\}.$$

We have another isomorphic injection

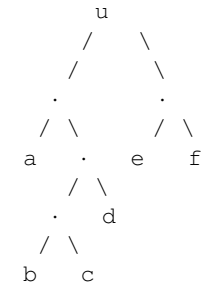
$$\lambda x, y.\, \langle x, y\rangle : D^{\S} \times D^{\S} \to D^{\S}.$$

Indeed by looking at the neighbourhood definition of $D^{\S}$ we conclude that the *finite* elements of $D^{\S}$ are exactly those that are either of the form (i) $\bot$, or (ii) $a^{\S}$, where a is finite in $|D|$, or (iii) $\langle a, b\rangle$, where a and b are previously obtained finite elements of $|D^{\S}|$.

Suppose $a, \ldots, f$ are finite in $|D|$. We can picture the element

$$u = \langle\langle a^{\S}, \langle\langle b^{\S}, c^{\S}\rangle, d^{\S}\rangle\rangle, \langle e^{\S}, f^{\S}\rangle\rangle$$

in $|D^{\S}|$ as a tree:

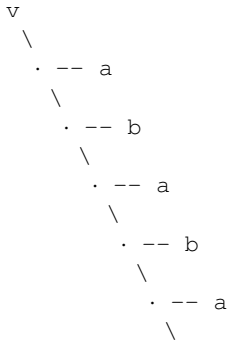


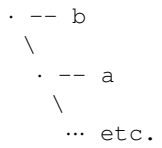
Note that the tree has binary branching with the elements of $|D|$ at the ends of the branches. Any such tree could be given a notation as an element of $|D^{\S}|$. The finite elements of $|D^{\S}|$ correspond exactly to such finite trees.

What of the infinite elements of $|D^{\S}|$? Are there infinite trees? Let $a, b \in |D^{\S}|$ be any elements of $|D^{\S}|$. Since pairing is an approximable mapping, we can solve the fixed-point equation

$$v = \langle a, \langle b, v\rangle\rangle.$$

In pictures we can diagram v roughly as:





The word is "roughly" here, since if a or b were not in the $|D|$ part of $|D^{\S}|$, then in the diagram the letters "a" and "b" should be replaced by the corresponding tree diagrams for a and b.

Suppose that a and b are finite. Then we can easily see that the infinite tree v is the limit of the following sequence of finite trees:

$$v_0 = \bot,$$

$$v_{n+1} = \langle a, \langle b, v_n \rangle \rangle,$$

and

$$v = \bigcup_{n=0}^{\infty} v_n.$$

The reader should think how to explain from tree diagrams the approximation relation $v_n \sqsubseteq v$ and more general such relationships.

We could call $D^{\S}$ a tree algebra over D. There may be others. A general one is a structure of the form

$$\langle E, \text{in}, \text{pair} \rangle,$$

where

$$\text{in} : D \to E, \text{ and}$$

$$\text{pair} : E \times E \to E.$$

The algebra

$$\langle D^{\S}, \lambda x.\, x^{\S}, \lambda x, y.\, \langle x, y \rangle \rangle,$$

however, is a very special one: it is "minimal" among all tree algebras over D in a sense we shall have to make precise.

To do this think of how E and $D^{\S}$ can differ. In view of the isomorphism that $D^{\S}$ satisfies, the injection of D and the pairing are one-one, so no "information" is lost by these mappings. The same may not at all be true of E, but it is reasonable to think that at least we can define an approximable mapping $g : D^{\S} \to E$ where

$$(1) \quad g(\bot) = \bot_E,$$

$$(2) \quad g(x^{\S}) = \text{in}(x), \text{ for } x \in |D|, \text{ and}$$

$$(3) \quad g(\langle x, y \rangle) = \text{pair}(g(x), g(y)), \text{ for } x, y \in |D^{\S}|.$$

By what we said earlier, g will be uniquely determined by (1)–(3), because these equations tell us exactly how to calculate g on all finite elements of $|D^{\S}|$. An approximable mapping is always determined by its action on the finite elements. But why does g exist?

It would not be too hard to give an inductive construction of g as a neighbourhood relation, but a fixed-point equation is easier to write down for the same purpose. We need, though, to have the inverse ("predecessor") functions:

$$\text{out} : D^{\S} \to D$$

$$\text{proj}_i : D^{\S} \to D^{\S}, \text{ for } i = 0, 1,$$

where

$$\text{out}(x^{\S}) = x,$$

$$\text{proj}_0(\langle x, y\rangle) = x, \text{ and}$$

$$\text{proj}_1(\langle x, y\rangle) = y.$$

We also need

$$\text{atom} : D^{\S} \to T,$$

where

$$\text{atom}(x^{\S}) = \text{true}, \text{ and}$$

$$\text{atom}(\langle x, y\rangle) = \text{false}.$$

We can then write

$$g(x) = \text{cond}\big(\text{atom}(x),\ \text{in}(\text{out}(x)),\ \text{pair}(g(\text{proj}_0(x)),\ g(\text{proj}_1(x)))\big).$$

This g exists by fixed-point theory, and it satisfies (1)–(3) by what we know about the structure of $|D^{\S}|$. As we said, g is unique because the values on finite elements are fixed.

In algebraic language g is a *homomorphism* of tree algebras; and $D^{\S}$ is called an *initial algebra*, because for any tree algebra E there is a unique homomorphism $g : D^{\S} \to E$. We note at once that any two initial algebras are isomorphic. For if D^* were another, there would exist homomorphisms in both directions between $D^{\S}$ and D^*. But the compositions of homomorphisms are again homomorphisms, and in the case of $D^{\S}$ if we go from $D^{\S}$ to D^* and back to $D^{\S}$, the result must be the identity. The reason is that the identity can be the only homomorphism of an initial algebra into itself. We thus have a precise meaning of the minimal character of $D^{\S}$. But note it still took a construction to show that the domain $D^{\S}$ exists. □

The development is with the indefinite product peer.

Theorem (`example_61`). For a neighbourhood system whose neighbourhoods are nonempty, the indefinite product is a neighbourhood system whose membership is the MemS inductive family and whose master is the Gamma set of tagged master tokens.

Proof. The membership and master characterisations of the indefinite-product peer. ■

Example_6_1.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.indefProduct

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem IndefProduct

/-- Scott's Example 6.1: the indefinite product domain `D?` is a neighbourhood system
whose neighbourhoods are generated by the master, atomic embeddings, and pairing. -/
theorem example_61 :
    ∀ {α : Type*} (D : NeighborhoodSystem α) (hD : ∀ X, D.mem X → X.Nonempty),
      (∀ {W : Set (List Bool × α)}, (Dsharp D hD).mem W ↔ MemS D W) ∧
        (Dsharp D hD).master = Gamma D :=
  fun D hD => ⟨Dsharp_mem (D := D) (hD := hD), Dsharp_master (D := D) (hD := hD)⟩
```

4.7.4 seqEqB — B equation

Sequence domain B: bit embeddings and the equation B ≅ B + B.

Definition (`embBit`). As in the library.

Proof. By definition. ■

Theorem (`embBit_eq_prepend`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embBit_cone`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`memB_embBit`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`nil_not_mem_embBit`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embBit_ne_univ`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embBit_inter`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embBit_inter_ne`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embBit_subset`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embBit_injective`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embBit_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`memB_embBit_inv`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embBit_ne`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`B_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`memB_cases`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`BB`). As in the library.

Proof. By definition. ■

Theorem (`sum_mem_inj`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`sum_mem_inj`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`sum_mem_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`toBB`). As in the library.

Proof. By definition. ■

Definition (`fromBB`). As in the library.

Proof. By definition. ■

Theorem (`fromBB_toBB`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`toBB_fromBB`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`bbEquiv`). As in the library.

Proof. By definition. ■

Theorem (`B_domain_equation`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

seqEqB.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.seqDomain
public import Mathlib.Computability.MyhillNerode
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ExampleB
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodIso
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.sumSys
import Mathlib.Data.Set.Finite.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap BitDom

namespace SeqEqB

/-! ### Prepending a single bit: `bX = {bw' ? w' ∈ X}`. -/

/-- `bX = {b :: w' ? w' ∈ X}`: the `b`-prefixed copy of a neighbourhood `X` (Scott's `0
    X` for
`b = false` and `1X` for `b = true`). -/
def embBit (b : Bool) (X : Set Str) : Set Str := {w | ∃ w', w = b :: w' ∧ w' ∈ X}

@[simp] theorem mem_embBit {b : Bool} {X : Set Str} {w : Str} :
    w ∈ embBit b X ↔ ∃ w', w = b :: w' ∧ w' ∈ X := Iff.rfl

/-- `bX = b·X` is exactly the single-bit prepend of Example 1.B. -/
theorem embBit_eq_prepend (b : Bool) (X : Set Str) : embBit b X = prepend [b] X := by
  ext w
  simp only [mem_embBit, mem_prepend]
  constructor
  · rintro ⟨w', rfl, hX⟩; exact ⟨w', hX, rfl⟩
  · rintro ⟨τ, hX, rfl⟩; exact ⟨τ, rfl, hX⟩

/-- `b(σΣ⋆) = (bσ)Σ⋆`: prepending a bit to a cone gives the cone of the longer prefix.
    -/
theorem embBit_cone (b : Bool) (σ : Str) : embBit b (cone σ) = cone (b :: σ) := by
  rw [embBit_eq_prepend, prepend_cone]; rfl

/-- Prepending a bit lands back in `B`. -/
theorem memB_embBit (b : Bool) {X : Set Str} (hX : B.mem X) : B.mem (embBit b X) := by
  rw [embBit_eq_prepend]; exact memB_prepend [b] hX

theorem nil_not_mem_embBit {b : Bool} {X : Set Str} : ([] : Str) ∈ embBit b X := by
  rintro ⟨w', heq, -⟩; exact absurd heq (by simp)

theorem embBit_ne_univ (b : Bool) (X : Set Str) : embBit b X = Set.univ := by
  intro h; exact nil_not_mem_embBit (X := X) (b := b) (by rw [h]; trivial)

theorem embBit_inter (b : Bool) (X X' : Set Str) :
    embBit b X ∩ embBit b X' = embBit b (X ∩ X') := by
```

```
  ext w
  simp only [Set.mem_inter_iff, mem_embBit]
  constructor
  · rintro ⟨⟨w', rfl, hX⟩, w'', heq, hX'⟩
    rw [List.cons.injEq] at heq
    obtain ⟨-, rfl⟩ := heq
    exact ⟨w', rfl, hX, hX'⟩
  · rintro ⟨w', rfl, hX, hX'⟩
    exact ⟨⟨w', rfl, hX⟩, ⟨w', rfl, hX'⟩⟩

theorem embBit_inter_ne {b b' : Bool} (h : b = b') (X Y : Set Str) :
    embBit b X ∩ embBit b' Y = ∅ := by
  ext w
  simp only [Set.mem_inter_iff, mem_embBit, Set.mem_empty_iff_false, iff_false,
    not_and]
  rintro ⟨w', rfl, -⟩ ⟨w'', heq, -⟩
  rw [List.cons.injEq] at heq
  exact h heq.1

theorem embBit_subset {b : Bool} {X X' : Set Str} :
    embBit b X ⊆ embBit b X' ↔ X ⊆ X' := by
  constructor
  · intro h w' hw'
    obtain ⟨w'', heq, hX'⟩ := h ⟨w', rfl, hw'⟩
    rw [List.cons.injEq] at heq
    obtain ⟨-, rfl⟩ := heq
    exact hX'
  · rintro h w ⟨w', rfl, hX⟩
    exact ⟨w', rfl, h hX⟩

theorem embBit_injective {b : Bool} {X X' : Set Str} (h : embBit b X = embBit b X') :
    X = X' :=
  Set.Subset.antisymm (embBit_subset.mp h.subset) (embBit_subset.mp h.symm.subset)

theorem embBit_nonempty {b : Bool} {X : Set Str} (hX : X.Nonempty) : (embBit b X).
    Nonempty := by
  obtain ⟨w', hw'⟩ := hX; exact ⟨b :: w', w', rfl, hw'⟩

/-- If `bW ∈ B` then `W ∈ B`: a `b`-prefixed neighbourhood that lands in `B` must be `b·
    (cone w')`,
so `W = cone w'`. -/
theorem memB_embBit_inv {b : Bool} {W : Set Str} (h : B.mem (embBit b W)) : B.mem W :=
     by
  obtain ⟨σ, hσ⟩ := h
  have hmem : σ ∈ embBit b W := hσ ▸ (show σ ∈ cone σ from List.prefix_rfl)
  obtain ⟨w', rfl, -⟩ := hmem
  rw [← embBit_cone] at hσ
  rw [embBit_injective hσ]
  exact memB_cone w'

theorem embBit_ne {b b' : Bool} (h : b = b') {X Y : Set Str} (hX : X.Nonempty) :
    embBit b X = embBit b' Y := by
  intro heq
  obtain ⟨w', hw'⟩ := hX
  have hmem : (b :: w') ∈ embBit b' Y := heq ▸ (⟨w', rfl, hw'⟩ : (b :: w') ∈ embBit b X
    )
```

```
  obtain ⟨w'', he, -⟩ := hmem
  rw [List.cons.injEq] at he
  exact h he.1

/-! ### `B` is positive (no empty neighbourhood) and its neighbourhood-shape
    classification. -/

/-- Scott's standing assumption `∅ ∈ B`: every neighbourhood of `B` is nonempty (cones
    contain their
generating prefix). -/
theorem B_nonempty : ∀ X, B.mem X → X.Nonempty := by
  rintro X ⟨σ, rfl⟩; exact ⟨σ, List.prefix_rfl⟩

/-- **Example 6.2 -- the shape of a `B`-neighbourhood.** Every neighbourhood of `B` is
    either the
master `Σ⋆ = cone []`, a `0`-copy `0X` with `X ∈ B`, or a `1`-copy `1X` with `X ∈ B`. -/
theorem memB_cases {W : Set Str} (hW : B.mem W) :
    W = Set.univ ∨ (∃ X, B.mem X ∧ W = embBit false X) ∨ (∃ Y, B.mem Y ∧ W = embBit
    true Y) := by
  obtain ⟨σ, rfl⟩ := hW
  cases σ with
  | nil => exact Or.inl cone.nil
  | cons b σ' =>
    cases b with
    | false => exact Or.inr (Or.inl ⟨cone σ', memB_cone σ', (embBit_cone false σ').symm
    ⟩)
    | true => exact Or.inr (Or.inr ⟨cone σ', memB_cone σ', (embBit_cone true σ').symm⟩)

/-! ### The sum target `B + B` and its inversion lemmas. -/

/-- The right-hand side of Scott's domain equation: the sum system `B + B` (Exercise
    3.18). -/
abbrev BB : NeighborhoodSystem (Option (Str ? Str)) := sum B B B_nonempty B_nonempty

theorem sum_mem_inj₀_inv {X : Set Str} (h : BB.mem (inj₀ X)) : B.mem X := by
  rcases h with h0 | ⟨X', hX', heq⟩ | ⟨Y', hY', heq⟩
  · exact absurd (h0 ▶ none_mem_sumMaster) none_mem_inj₀
  · rw [inj₀_injective heq]; exact hX'
  · obtain ⟨b, hb⟩ := B_nonempty Y' hY'
    exact absurd (heq ▶ ir_mem_inj₁.mpr hb) ir_mem_inj₀

theorem sum_mem_inj₁_inv {Y : Set Str} (h : BB.mem (inj₁ Y)) : B.mem Y := by
  rcases h with h0 | ⟨X', hX', heq⟩ | ⟨Y', hY', heq⟩
  · exact absurd (h0 ▶ none_mem_sumMaster) none_mem_inj₁
  · obtain ⟨a, ha⟩ := B_nonempty X' hX'
    exact absurd (heq ▶ il_mem_inj₀.mpr ha) il_mem_inj₁
  · rw [inj₁_injective heq]; exact hY'

theorem sum_mem_nonempty {W : Set (Option (Str ? Str))} (h : BB.mem W) : W.Nonempty :=
    by
  rcases h with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
  · exact ⟨none, none_mem_sumMaster⟩
  · exact inj₀_nonempty (B_nonempty X hX)
  · exact inj₁_nonempty (B_nonempty Y hY)

/-! ### The forward half `toBB : |B| → |B + B|`. -/
```

```
/-- **Example 6.2 -- forward half of `B ≅ B + B`.** An element `x` of `B` is sent to
    the sum element
recording, for each branch, whether `x` reaches the `0`-copy `0X` (left summand) or the
     `1`-copy `1Y`
(right summand). -/
def toBB (x : B.Element) : BB.Element where
  mem W := W = sumMaster B B
    ∨ (∃ X, B.mem X ∧ W = inj₀ X ∧ x.mem (embBit false X))
    ∨ (∃ Y, B.mem Y ∧ W = inj₁ Y ∧ x.mem (embBit true Y))
  sub := by
    rintro W (rfl | ⟨X, hX, rfl, -⟩ | ⟨Y, hY, rfl, -⟩)
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨X, hX, rfl⟩)
    · exact Or.inr (Or.inr ⟨Y, hY, rfl⟩)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩)
      (rfl | ⟨X', hX', rfl, hzX'⟩ | ⟨Y', hY', rfl, hzY'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (Or.inl ⟨X', hX', by rw [sumMaster_inter_inj₀ hX'], hzX'⟩)
    · exact Or.inr (Or.inr ⟨Y', hY', by rw [sumMaster_inter_inj₁ hY'], hzY'⟩)
    · exact Or.inr (Or.inl ⟨X, hX, by rw [Set.inter_comm, sumMaster_inter_inj₀ hX], hzX⟩
    )
    · refine Or.inr (Or.inl ⟨X ∩ X', ?_, by rw [inj₀_inter], ?_⟩)
      · have hz := x.inter_mem hzX hzX'; rw [embBit_inter] at hz; exact memB_embBit_inv
     (x.sub hz)
      · have hz := x.inter_mem hzX hzX'; rwa [embBit_inter] at hz
    · exfalso
      have hz := x.inter_mem hzX hzY'
      rw [embBit_inter_ne (show (false : Bool) = true by decide)] at hz
      obtain ⟨t, ht⟩ := B_nonempty _ (x.sub hz); exact Set.notMem_empty t ht
    · exact Or.inr (Or.inr ⟨Y, hY, by rw [Set.inter_comm, sumMaster_inter_inj₁ hY], hzY⟩
    )
    · exfalso
      have hz := x.inter_mem hzY hzX'
      rw [embBit_inter_ne (show (true : Bool) = false by decide)] at hz
      obtain ⟨t, ht⟩ := B_nonempty _ (x.sub hz); exact Set.notMem_empty t ht
    · refine Or.inr (Or.inr ⟨Y ∩ Y', ?_, by rw [inj₁_inter], ?_⟩)
      · have hz := x.inter_mem hzY hzY'; rw [embBit_inter] at hz; exact memB_embBit_inv
     (x.sub hz)
      · have hz := x.inter_mem hzY hzY'; rwa [embBit_inter] at hz
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩) hW' hsub
    · exact Or.inl (eq_sumMaster_of_subset hW' hsub)
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · exact Or.inl rfl
      · refine Or.inr (Or.inl ⟨X', hX', rfl, ?_⟩)
        exact x.up_mem hzX (memB_embBit false hX') (embBit_subset.mpr (inj₀_subset_inj₀
    .mp hsub))
      · exfalso
        obtain ⟨a, ha⟩ := B_nonempty X hX
        exact absurd (hsub (il_mem_inj₀.mpr ha)) il_mem_inj₁
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · exact Or.inl rfl
      · exfalso
```

```
        obtain ⟨b, hb⟩ := B_nonempty Y hY
        exact absurd (hsub (ir_mem_inj₁.mpr hb)) ir_mem_inj₀
      · refine Or.inr (Or.inr ⟨Y', hY', rfl, ?_⟩)
        exact x.up_mem hzY (memB_embBit true hY') (embBit_subset.mpr (inj₁_subset_inj₁.
    mp hsub))

@[simp] theorem toBB_mem_inj₀ {x : B.Element} {X : Set Str} (hX : B.mem X) :
    (toBB x).mem (inj₀ X) ↔ x.mem (embBit false X) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hz⟩ | ⟨Y', hY', heq, hz⟩)
    · exact absurd (h0 ▶ none_mem_sumMaster) none_mem_inj₀
    · rwa [inj₀_injective heq]
    · obtain ⟨a, ha⟩ := B_nonempty X hX
      exact absurd (heq ▶ il_mem_inj₀.mpr ha) il_mem_inj₁
  · intro hz; exact Or.inr (Or.inl ⟨X, hX, rfl, hz⟩)

@[simp] theorem toBB_mem_inj₁ {x : B.Element} {Y : Set Str} (hY : B.mem Y) :
    (toBB x).mem (inj₁ Y) ↔ x.mem (embBit true Y) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hz⟩ | ⟨Y', hY', heq, hz⟩)
    · exact absurd (h0 ▶ none_mem_sumMaster) none_mem_inj₁
    · obtain ⟨a, ha⟩ := B_nonempty X' hX'
      exact absurd (heq ▶ il_mem_inj₀.mpr ha) il_mem_inj₁
    · rwa [inj₁_injective heq]
  · intro hz; exact Or.inr (Or.inr ⟨Y, hY, rfl, hz⟩)

/-! ### The inverse half `fromBB : |B + B| → |B|`. -/

/-- **Example 6.2 -- inverse half of `B ≅ B + B`.** -/
def fromBB (s : BB.Element) : B.Element where
  mem W := W = Set.univ
    ∨ (∃ X, B.mem X ∧ W = embBit false X ∧ s.mem (inj₀ X))
    ∨ (∃ Y, B.mem Y ∧ W = embBit true Y ∧ s.mem (inj₁ Y))
  sub := by
    rintro W (rfl | ⟨X, hX, rfl, -⟩ | ⟨Y, hY, rfl, -⟩)
    · exact ⟨[], cone.nil.symm⟩
    · exact memB_embBit false hX
    · exact memB_embBit true hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hsX⟩ | ⟨Y, hY, rfl, hsY⟩)
      (rfl | ⟨X', hX', rfl, hsX'⟩ | ⟨Y', hY', rfl, hsY'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (Or.inl ⟨X', hX', by rw [Set.univ_inter], hsX'⟩)
    · exact Or.inr (Or.inr ⟨Y', hY', by rw [Set.univ_inter], hsY'⟩)
    · exact Or.inr (Or.inl ⟨X, hX, by rw [Set.inter_univ], hsX⟩)
    · refine Or.inr (Or.inl ⟨X ∩ X', ?_, by rw [embBit_inter], ?_⟩)
      · have hs := s.inter_mem hsX hsX'; rw [inj₀_inter] at hs
        exact sum_mem_inj₀_inv (s.sub hs)
      · have hs := s.inter_mem hsX hsX'; rwa [inj₀_inter] at hs
    · exfalso
      have hs := s.inter_mem hsX hsY'; rw [inj₀_inter_inj₁] at hs
      obtain ⟨t, ht⟩ := sum_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    · exact Or.inr (Or.inr ⟨Y, hY, by rw [Set.inter_univ], hsY⟩)
    · exfalso
      have hs := s.inter_mem hsY hsX'; rw [Set.inter_comm, inj₀_inter_inj₁] at hs
```

```
      obtain ⟨t, ht⟩ := sum_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    · refine Or.inr (Or.inr ⟨Y ∩ Y', ?_, by rw [embBit_inter], ?_⟩)
      · have hs := s.inter_mem hsY hsY'; rw [inj₁_inter] at hs
        exact sum_mem_inj₁_inv (s.sub hs)
      · have hs := s.inter_mem hsY hsY'; rwa [inj₁_inter] at hs
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hsX⟩ | ⟨Y, hY, rfl, hsY⟩) hW' hsub
    · exact Or.inl (Set.univ_subset_iff.mp hsub)
    · rcases memB_cases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · exact Or.inl rfl
      · refine Or.inr (Or.inl ⟨X', hX', rfl, ?_⟩)
        exact s.up_mem hsX (Or.inr (Or.inl ⟨X', hX', rfl⟩))
          (inj₀_subset_inj₀.mpr (embBit_subset.mp hsub))
      · exfalso
        obtain ⟨a, ha⟩ := B_nonempty X hX
        obtain ⟨w', he, -⟩ := hsub (⟨a, rfl, ha⟩ : (false :: a) ∈ embBit false X)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
    · rcases memB_cases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · exact Or.inl rfl
      · exfalso
        obtain ⟨b, hb⟩ := B_nonempty Y hY
        obtain ⟨w', he, -⟩ := hsub (⟨b, rfl, hb⟩ : (true :: b) ∈ embBit true Y)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
      · refine Or.inr (Or.inr ⟨Y', hY', rfl, ?_⟩)
        exact s.up_mem hsY (Or.inr (Or.inr ⟨Y', hY', rfl⟩))
          (inj₁_subset_inj₁.mpr (embBit_subset.mp hsub))

@[simp] theorem fromBB_mem_embF {s : BB.Element} {X : Set Str} (hX : B.mem X) :
    (fromBB s).mem (embBit false X) ↔ s.mem (inj₀ X) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hs⟩ | ⟨Y', hY', heq, hs⟩)
    · exact absurd h0 (embBit_ne_univ false X)
    · rwa [embBit_injective heq]
    · exact absurd heq (embBit_ne (show (false : Bool) = true by decide) (B_nonempty X
    hX))
  · intro hs; exact Or.inr (Or.inl ⟨X, hX, rfl, hs⟩)

@[simp] theorem fromBB_mem_embT {s : BB.Element} {Y : Set Str} (hY : B.mem Y) :
    (fromBB s).mem (embBit true Y) ↔ s.mem (inj₁ Y) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hs⟩ | ⟨Y', hY', heq, hs⟩)
    · exact absurd h0 (embBit_ne_univ true Y)
    · exact absurd heq.symm (embBit_ne (show (false : Bool) = true by decide) (
    B_nonempty X' hX'))
    · rwa [embBit_injective heq]
  · intro hs; exact Or.inr (Or.inr ⟨Y, hY, rfl, hs⟩)

/-! ### The two halves are mutually inverse. -/

theorem fromBB_toBB (x : B.Element) : fromBB (toBB x) = x := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨X, hX, rfl, hs⟩ | ⟨Y, hY, rfl, hs⟩)
    · exact x.master_mem
    · exact (toBB_mem_inj₀ hX).mp hs
```

```
    · exact (toBB_mem_inj₁ hY).mp hs
  · intro hW
    rcases memB_cases (x.sub hW) with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨X, hX, rfl, (toBB_mem_inj₀ hX).mpr hW⟩)
    · exact Or.inr (Or.inr ⟨Y, hY, rfl, (toBB_mem_inj₁ hY).mpr hW⟩)

theorem toBB_fromBB (s : BB.Element) : toBB (fromBB s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨X, hX, rfl, hs⟩ | ⟨Y, hY, rfl, hs⟩)
    · exact s.master_mem
    · exact (fromBB_mem_embF hX).mp hs
    · exact (fromBB_mem_embT hY).mp hs
  · intro hW
    rcases s.sub hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨X, hX, rfl, (fromBB_mem_embF hX).mpr hW⟩)
    · exact Or.inr (Or.inr ⟨Y, hY, rfl, (fromBB_mem_embT hY).mpr hW⟩)

/-! ### The domain equation `B ≅ B + B`. -/

/-- **Example 6.2 (Scott 1981, PRG-19) -- the isomorphism `|B| ≃o |B + B|`.** -/
def bbEquiv : B.Element ≃o BB.Element where
  toFun := toBB
  invFun := fromBB
  left_inv := fromBB_toBB
  right_inv := toBB_fromBB
  map_rel_iff' := by
    intro x x'
    constructor
    · intro h X hX
      rcases memB_cases (x.sub hX) with rfl | ⟨A, hA, rfl⟩ | ⟨A, hA, rfl⟩
      · exact x'.master_mem
      · exact (toBB_mem_inj₀ hA).mp (h _ (Or.inr (Or.inl ⟨A, hA, rfl, hX⟩)))
      · exact (toBB_mem_inj₁ hA).mp (h _ (Or.inr (Or.inr ⟨A, hA, rfl, hX⟩)))
    · intro h W hW
      rcases hW with rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨X, hX, rfl, h _ hzX⟩)
      · exact Or.inr (Or.inr ⟨Y, hY, rfl, h _ hzY⟩)

/-- **Example 6.2 (Scott 1981, PRG-19) -- the domain equation `B ≅ B + B`.** Scott's
    binary-sequence
domain `B` is, as a domain, isomorphic to `B + B`: a sequence is bottom, or begins with
     `0`, or begins
with `1`, the two non-bottom cases giving the two summands. -/
theorem B_domain_equation : B ≅? sum B B B_nonempty B_nonempty :=
  ⟨bbEquiv⟩

end SeqEqB
end DomainTheory.NeighbourhoodSystems
```

4.7.5 seqEqA — A^n equation

Sequence domain A^n: tuple embeddings and the equation A^n ≅ A^n + A^n.

Definition (`slotPre`). As in the library.

Proof. By definition. ■

Definition (`embTuple`). As in the library.

Proof. By definition. ■

Definition (`MemA`). As in the library.

Proof. By definition. ■

Definition (`prodN`). As in the library.

Proof. By definition. ■

Theorem (`prodN_injective`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`prodN_inter`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`prodN_subset`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`npow`). As in the library.

Proof. By definition. ■

Theorem (`embTuple_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`memA_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`nil_not_mem_embTuple`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embTuple_ne_univ`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`npow_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`slot_list_inj`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`slotPre_inj`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embTuple_inter`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embTuple_inter_ne`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embTuple_ne`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embTuple_subset`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embTuple_injective`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`memA_tuple_inv`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`tag_eq_of_subset`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`memA_inter`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`Asys`). As in the library.

Proof. By definition. ■

Definition (`Apow`). As in the library.

Proof. By definition. ■

Theorem (`apowNe`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`AAsys`). As in the library.

Proof. By definition. ■

Theorem (`aa_mem_inj`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`aa_mem_inj`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`aa_mem_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`apow_mem_prodN`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`aa_mem_inj`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`aa_mem_inj`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`apow_mem_prodN_inv`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`toAA`). As in the library.

Proof. By definition. ■

Definition (`fromAA`). As in the library.

Proof. By definition. ■

Theorem (`fromAA_toAA`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`toAA_fromAA`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`aaEquiv`). As in the library.

Proof. By definition. ■

Theorem (`A_domain_equation`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

seqEqA.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.seqEqB

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

namespace SeqEqA
open NeighborhoodSystem ApproximableMap BitDom

open NeighborhoodSystem ApproximableMap BitDom

variable {β : Type*} {n : ℕ}
variable {n : ℕ}

def slotPre (i : Bool) (j : ℕ) : Str := i :: (List.replicate j true ++ [false])

def embTuple (i : Bool) (X : Fin n → Set Str) : Set Str :=
  {w | ∃ j : Fin n, ∃ w', w = slotPre i (j : ℕ) ++ w' ∧ w' ∈ X j}

inductive MemA (n : ℕ) : Set Str → Prop
  | univ : MemA n Set.univ
  | tuple (i : Bool) {X : Fin n → Set Str} : (∀ j, MemA n (X j)) → MemA n (embTuple i
    X)

variable {β : Type*} {n : ℕ}
```

```
def prodN (X : Fin n → Set β) : Set (Fin n × β) := {p | p.2 ∈ X p.1}

@[simp] theorem mem_prodN {X : Fin n → Set β} {p : Fin n × β} : p ∈ prodN X ↔ p.2 ∈
    X p.1 := Iff.rfl

theorem prodN_injective {X X' : Fin n → Set β} (h : prodN X = prodN X') : ∀ j, X j = X
    ' j := by
  intro j; ext b
  have := Set.ext_iff.mp h (j, b)
  simpa only [mem_prodN] using this

theorem prodN_inter {X X' : Fin n → Set β} :
    prodN X ∩ prodN X' = prodN (fun j => X j ∩ X' j) := by
  ext p; simp only [Set.mem_inter_iff, mem_prodN]

theorem prodN_subset {X X' : Fin n → Set β} : prodN X ⊆ prodN X' ↔ ∀ j, X j ⊆ X' j
    := by
  constructor
  · intro h j b hb; exact h (show (j, b) ∈ prodN X from hb)
  · intro h p hp; exact h p.1 hp

def npow (V : NeighborhoodSystem β) (n : ℕ) (hn : 0 < n) : NeighborhoodSystem (Fin n ×
     β) where
  mem W := ∃ X : Fin n → Set β, (∀ j, V.mem (X j)) ∧ W = prodN X
  master := prodN (fun _ => V.master)
  master_nonempty := by
    obtain ⟨b, hb⟩ := V.master_nonempty
    exact ⟨(⟨0, hn⟩, b), hb⟩
  master_mem := ⟨_, fun _ => V.master_mem, rfl⟩
  inter_mem := by
    rintro W W' Z ⟨X, hX, rfl⟩ ⟨X', hX', rfl⟩ ⟨Zw, hZw, rfl⟩ hsub
    refine ⟨fun j => X j ∩ X' j, ?_, prodN_inter⟩
    intro j
    rw [prodN_inter] at hsub
    exact V.inter_mem (hX j) (hX' j) (hZw j) (prodN_subset.mp hsub j)
  sub_master := by
    rintro W ⟨X, hX, rfl⟩
    exact prodN_subset.mpr (fun j => V.sub_master (hX j))

@[simp] theorem npow_mem {V : NeighborhoodSystem β} {W : Set (Fin n × β)} {hn : 0 < n}
     :
    (npow V n hn).mem W ↔ ∃ X : Fin n → Set β, (∀ j, V.mem (X j)) ∧ W = prodN X :=
    Iff.rfl

variable {β : Type*} {n : ℕ}
variable {n : ℕ}

@[simp] theorem mem_embTuple {i : Bool} {X : Fin n → Set Str} {w : Str} :
    w ∈ embTuple i X ↔ ∃ j : Fin n, ∃ w', w = slotPre i (j : ℕ) ++ w' ∧ w' ∈ X j :=
    Iff.rfl

theorem embTuple_nonempty {i : Bool} {X : Fin n → Set Str} (j : Fin n) (h : (X j).
    Nonempty) :
    (embTuple i X).Nonempty := by
  obtain ⟨a, ha⟩ := h
  exact ⟨slotPre i (j : ℕ) ++ a, j, a, rfl, ha⟩
```

```
theorem memA_nonempty (hn : 0 < n) : ∀ {W}, MemA n W → W.Nonempty := by
  intro W hW
  induction hW with
  | univ => exact ⟨[], Set.mem_univ []⟩
  | tuple i hX ih => exact embTuple_nonempty ⟨0, hn⟩ (ih ⟨0, hn⟩)

theorem nil_not_mem_embTuple {i : Bool} {X : Fin n → Set Str} : ([] : Str) ∈ embTuple
    i X := by
  rintro ⟨j, a, heq, -⟩
  rw [slotPre, List.cons_append] at heq
  exact List.cons_ne_nil _ _ heq.symm

theorem embTuple_ne_univ {i : Bool} (X : Fin n → Set Str) : embTuple i X = Set.univ :=
    by
  intro h
  have : ([] : Str) ∈ embTuple i X := by rw [h]; exact Set.mem_univ []
  exact nil_not_mem_embTuple this

variable {β : Type*} {n : ℕ}

theorem npow_nonempty {V : NeighborhoodSystem β} (hn : 0 < n)
    (hV : ∀ X, V.mem X → X.Nonempty) : ∀ W, (npow V n hn).mem W → W.Nonempty := by
  rintro W ⟨X, hX, rfl⟩
  obtain ⟨b, hb⟩ := hV (X ⟨0, hn⟩) (hX ⟨0, hn⟩)
  exact ⟨(⟨0, hn⟩, b), hb⟩

theorem slot_list_inj : ∀ (j j' : ℕ) (u v : Str),
    List.replicate j true ++ false :: u = List.replicate j' true ++ false :: v → j = j
    ' ∧ u = v
  | 0, 0, u, v, h => by simpa using h
  | 0, j' + 1, u, v, h => by simp [List.replicate_succ] at h
  | j + 1, 0, u, v, h => by simp [List.replicate_succ] at h
  | j + 1, j' + 1, u, v, h => by
    rw [List.replicate_succ, List.cons_append, List.replicate_succ, List.cons_append
    ,
      List.cons.injEq] at h
    obtain ⟨hj, hu⟩ := slot_list_inj j j' u v h.2
    exact ⟨by rw [hj], hu⟩

theorem slotPre_inj {i i' : Bool} {j j' : ℕ} {u v : Str}
    (h : slotPre i j ++ u = slotPre i' j' ++ v) : i = i' ∧ j = j' ∧ u = v := by
  rw [slotPre, slotPre, List.cons_append, List.cons_append, List.cons.injEq] at h
  obtain ⟨hi, hrest⟩ := h
  rw [List.append_assoc, List.append_assoc, List.singleton_append, List.
    singleton_append] at hrest
  obtain ⟨hj, huv⟩ := slot_list_inj j j' u v hrest
  exact ⟨hi, hj, huv⟩

variable {β : Type*} {n : ℕ}
variable {n : ℕ}

theorem embTuple_inter {i : Bool} (X X' : Fin n → Set Str) :
    embTuple i X ∩ embTuple i X' = embTuple i (fun j => X j ∩ X' j) := by
  ext w
  simp only [Set.mem_inter_iff, mem_embTuple]
```

```
  constructor
  · rintro ⟨⟨j, a, rfl, ha⟩, ⟨j', a', heq, ha'⟩⟩
    obtain ⟨-, hjj, rfl⟩ := slotPre_inj heq
    obtain rfl : j = j' := Fin.ext hjj
    exact ⟨j, a, rfl, ha, ha'⟩
  · rintro ⟨j, a, rfl, ha, ha'⟩
    exact ⟨⟨j, a, rfl, ha⟩, ⟨j, a, rfl, ha'⟩⟩

theorem embTuple_inter_ne {i i' : Bool} (h : i = i') (X X' : Fin n → Set Str) :
    embTuple i X ∩ embTuple i' X' = ∅ := by
  ext w
  simp only [Set.mem_inter_iff, mem_embTuple, Set.mem_empty_iff_false, iff_false,
    not_and]
  rintro ⟨j, a, rfl, ha⟩ ⟨j', a', heq, ha'⟩
  exact h (slotPre_inj heq).1

theorem embTuple_ne {i i' : Bool} (h : i = i') {X Y : Fin n → Set Str} (j : Fin n)
    (hX : (X j).Nonempty) : embTuple i X = embTuple i' Y := by
  intro heq
  obtain ⟨a, ha⟩ := hX
  have hmem : slotPre i (j : ℕ) ++ a ∈ embTuple i' Y := heq ▸ (⟨j, a, rfl, ha⟩ : _ ∈
    embTuple i X)
  obtain ⟨j', a', he, -⟩ := hmem
  exact h (slotPre_inj he).1

theorem embTuple_subset {i : Bool} {X X' : Fin n → Set Str} :
    embTuple i X ⊆ embTuple i X' ↔ ∀ j, X j ⊆ X' j := by
  constructor
  · intro h j a ha
    obtain ⟨j', a', heq, ha'⟩ := h ⟨j, a, rfl, ha⟩
    obtain ⟨-, hjj, rfl⟩ := slotPre_inj heq
    obtain rfl : j = j' := Fin.ext hjj
    exact ha'
  · rintro h w ⟨j, a, rfl, ha⟩
    exact ⟨j, a, rfl, h j ha⟩

theorem embTuple_injective {i : Bool} {X X' : Fin n → Set Str}
    (h : embTuple i X = embTuple i X') : ∀ j, X j = X' j := by
  intro j
  exact Set.Subset.antisymm (embTuple_subset.mp h.subset j) (embTuple_subset.mp h.symm
    .subset j)

theorem memA_tuple_inv (hn : 0 < n) {i : Bool} {X : Fin n → Set Str}
    (h : MemA n (embTuple i X)) : ∀ j, MemA n (X j) := by
  generalize hW : embTuple i X = W at h
  cases h with
  | univ =>
    exfalso
    have : ([] : Str) ∈ embTuple i X := by rw [hW]; exact Set.mem_univ []
    exact nil_not_mem_embTuple this
  | @tuple i' X' hX' =>
    have hne : (embTuple i' X').Nonempty :=
      embTuple_nonempty ⟨0, hn⟩ (memA_nonempty hn (hX' ⟨0, hn⟩))
    rw [← hW] at hne
    obtain ⟨w, j, a, rfl, ha⟩ := hne
    have hmem : slotPre i (j : ℕ) ++ a ∈ embTuple i' X' := hW ▸ (⟨j, a, rfl, ha⟩ : _ ∈
```

```
     embTuple i X)
    obtain ⟨j', a', he, -⟩ := hmem
    have hii : i = i' := (slotPre_inj he).1
    subst hii
    intro k; rw [embTuple_injective hW k]; exact hX' k

theorem tag_eq_of_subset (hn : 0 < n) {i i' : Bool} {Z V : Fin n → Set Str}
    (hZ : ∀ j, MemA n (Z j)) (hsub : embTuple i Z ⊆ embTuple i' V) : i = i' := by
  obtain ⟨a, ha⟩ := memA_nonempty hn (hZ ⟨0, hn⟩)
  obtain ⟨j', v, he, -⟩ := hsub ⟨⟨0, hn⟩, a, rfl, ha⟩
  exact (slotPre_inj he).1

theorem memA_inter (hn : 0 < n) :
    ∀ {X}, MemA n X → ∀ {Y}, MemA n Y → ∀ {Z}, MemA n Z → Z ⊆ X ∩ Y → MemA n (X ∩ Y
    ) := by
  intro X hX
  induction hX with
  | univ =>
    intro Y hY Z _ _
    rw [Set.univ_inter]; exact hY
  | @tuple i X hX ih =>
    intro Y hY Z hZ hsub
    cases hY with
    | univ =>
      rw [Set.inter_univ]; exact MemA.tuple i hX
    | @tuple i' X' hX' =>
      by_cases hii : i = i'
      · rw [hii, embTuple_inter] at hsub ⊢
        cases hZ with
        | univ => exact absurd (hsub (Set.mem_univ [])) nil_not_mem_embTuple
        | @tuple i'' ZW hZW =>
          have htag : i'' = i' := tag_eq_of_subset hn hZW hsub
          rw [htag] at hsub
          have hZsub := embTuple_subset.mp hsub
          exact MemA.tuple i' (fun j => ih j (hX' j) (hZW j) (hZsub j))
      · rw [embTuple_inter_ne hii] at hsub ⊢
        exfalso
        obtain ⟨z, hz⟩ := memA_nonempty hn hZ
        exact Set.notMem_empty z (hsub hz)

def Asys (n : ℕ) (hn : 0 < n) : NeighborhoodSystem Str where
  mem := MemA n
  master := Set.univ
  master_nonempty := ⟨[], Set.mem_univ _⟩
  master_mem := MemA.univ
  inter_mem := fun hX hY hZ hsub => memA_inter hn hX hY hZ hsub
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem Asys_mem {hn : 0 < n} {W : Set Str} : (Asys n hn).mem W ↔ MemA n W :=
    Iff.rfl

@[simp] theorem Asys_master {hn : 0 < n} : (Asys n hn).master = Set.univ := rfl

abbrev Apow (hn : 0 < n) : NeighborhoodSystem (Fin n × Str) := npow (Asys n hn) n hn

theorem apowNe (hn : 0 < n) : ∀ W, (Apow hn).mem W → W.Nonempty :=
```

```
  npow_nonempty hn (fun _ h => memA_nonempty hn h)

abbrev AAsys (hn : 0 < n) : NeighborhoodSystem (Option ((Fin n × Str) ? (Fin n × Str))
    ) :=
  sum (Apow hn) (Apow hn) (apowNe hn) (apowNe hn)

theorem aa_mem_inj₀_inv (hn : 0 < n) {V : Set (Fin n × Str)}
    (h : (AAsys hn).mem (inj₀ V)) : (Apow hn).mem V := by
  rcases h with h0 | ⟨X', hX', heq⟩ | ⟨Y', hY', heq⟩
  · exact absurd (h0 ▶ none_mem_sumMaster) none_mem_inj₀
  · rw [inj₀_injective heq]; exact hX'
  · obtain ⟨b, hb⟩ := apowNe hn _ hY'
    exact absurd (heq ▶ ir_mem_inj₁.mpr hb) ir_mem_inj₀

theorem aa_mem_inj₁_inv (hn : 0 < n) {V : Set (Fin n × Str)}
    (h : (AAsys hn).mem (inj₁ V)) : (Apow hn).mem V := by
  rcases h with h0 | ⟨X', hX', heq⟩ | ⟨Y', hY', heq⟩
  · exact absurd (h0 ▶ none_mem_sumMaster) none_mem_inj₁
  · obtain ⟨a, ha⟩ := apowNe hn _ hX'
    exact absurd (heq ▶ il_mem_inj₀.mpr ha) il_mem_inj₁
  · rw [inj₁_injective heq]; exact hY'

theorem aa_mem_nonempty (hn : 0 < n) {W : Set (Option ((Fin n × Str) ? (Fin n × Str)))
    }
    (h : (AAsys hn).mem W) : W.Nonempty := by
  rcases h with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
  · exact ⟨none, none_mem_sumMaster⟩
  · exact inj₀_nonempty (apowNe hn _ hX)
  · exact inj₁_nonempty (apowNe hn _ hY)

theorem apow_mem_prodN (hn : 0 < n) {X : Fin n → Set Str} (hX : ∀ j, MemA n (X j)) :
    (Apow hn).mem (prodN X) := ⟨X, hX, rfl⟩

theorem aa_mem_inj₀ (hn : 0 < n) {X : Fin n → Set Str} (hX : ∀ j, MemA n (X j)) :
    (AAsys hn).mem (inj₀ (prodN X)) := Or.inr (Or.inl ⟨prodN X, apow_mem_prodN hn hX,
    rfl⟩)

theorem aa_mem_inj₁ (hn : 0 < n) {Y : Fin n → Set Str} (hY : ∀ j, MemA n (Y j)) :
    (AAsys hn).mem (inj₁ (prodN Y)) := Or.inr (Or.inr ⟨prodN Y, apow_mem_prodN hn hY,
    rfl⟩)

theorem apow_mem_prodN_inv (hn : 0 < n) {X : Fin n → Set Str} (h : (Apow hn).mem (
    prodN X)) :
    ∀ j, MemA n (X j) := by
  obtain ⟨X', hX', heq⟩ := h
  intro j; rw [prodN_injective heq j]; exact hX' j

def toAA (hn : 0 < n) (z : (Asys n hn).Element) : (AAsys hn).Element where
  mem W := W = sumMaster (Apow hn) (Apow hn)
    ∨ (∃ X, (∀ j, MemA n (X j)) ∧ W = inj₀ (prodN X) ∧ z.mem (embTuple false X))
    ∨ (∃ Y, (∀ j, MemA n (Y j)) ∧ W = inj₁ (prodN Y) ∧ z.mem (embTuple true Y))
  sub := by
    rintro W (rfl | ⟨X, hX, rfl, -⟩ | ⟨Y, hY, rfl, -⟩)
    · exact (AAsys hn).master_mem
    · exact aa_mem_inj₀ hn hX
    · exact aa_mem_inj₁ hn hY
```

```
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩)
      (rfl | ⟨X', hX', rfl, hzX'⟩ | ⟨Y', hY', rfl, hzY'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (Or.inl ⟨X', hX', by rw [sumMaster_inter_inj₀ (apow_mem_prodN hn hX')
    ], hzX'⟩)
    · exact Or.inr (Or.inr ⟨Y', hY', by rw [sumMaster_inter_inj₁ (apow_mem_prodN hn hY')
    ], hzY'⟩)
    · exact Or.inr (Or.inl ⟨X, hX,
        by rw [Set.inter_comm, sumMaster_inter_inj₀ (apow_mem_prodN hn hX)], hzX⟩)
    · refine Or.inr (Or.inl ⟨fun j => X j ∩ X' j, ?_, by rw [inj₀_inter, prodN_inter],
    ?_⟩)
      · have hz := z.inter_mem hzX hzX'; rw [embTuple_inter] at hz
        exact memA_tuple_inv hn (z.sub hz)
      · have hz := z.inter_mem hzX hzX'; rwa [embTuple_inter] at hz
    · exfalso
      have hz := z.inter_mem hzX hzY'
      rw [embTuple_inter_ne (by decide : (false : Bool) = true)] at hz
      obtain ⟨t, ht⟩ := memA_nonempty hn (z.sub hz); exact Set.notMem_empty t ht
    · exact Or.inr (Or.inr ⟨Y, hY,
        by rw [Set.inter_comm, sumMaster_inter_inj₁ (apow_mem_prodN hn hY)], hzY⟩)
    · exfalso
      have hz := z.inter_mem hzY hzX'
      rw [embTuple_inter_ne (by decide : (true : Bool) = false)] at hz
      obtain ⟨t, ht⟩ := memA_nonempty hn (z.sub hz); exact Set.notMem_empty t ht
    · refine Or.inr (Or.inr ⟨fun j => Y j ∩ Y' j, ?_, by rw [inj₁_inter, prodN_inter],
    ?_⟩)
      · have hz := z.inter_mem hzY hzY'; rw [embTuple_inter] at hz
        exact memA_tuple_inv hn (z.sub hz)
      · have hz := z.inter_mem hzY hzY'; rwa [embTuple_inter] at hz
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩) hW' hsub
    · exact Or.inl (eq_sumMaster_of_subset hW' hsub)
    · rcases hW' with rfl | ⟨V', hV', rfl⟩ | ⟨V', hV', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨X', hX', rfl⟩ := hV'
        refine Or.inr (Or.inl ⟨X', hX', rfl, ?_⟩)
        exact z.up_mem hzX (MemA.tuple false hX')
          (embTuple_subset.mpr (prodN_subset.mp (inj₀_subset_inj₀.mp hsub)))
      · obtain ⟨b, hb⟩ := apowNe hn (prodN X) (apow_mem_prodN hn hX)
        exact absurd (hsub (il_mem_inj₀.mpr hb)) il_mem_inj₁
    · rcases hW' with rfl | ⟨V', hV', rfl⟩ | ⟨V', hV', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨b, hb⟩ := apowNe hn (prodN Y) (apow_mem_prodN hn hY)
        exact absurd (hsub (ir_mem_inj₁.mpr hb)) ir_mem_inj₀
      · obtain ⟨Y', hY', rfl⟩ := hV'
        refine Or.inr (Or.inr ⟨Y', hY', rfl, ?_⟩)
        exact z.up_mem hzY (MemA.tuple true hY')
          (embTuple_subset.mpr (prodN_subset.mp (inj₁_subset_inj₁.mp hsub)))

@[simp] theorem toAA_mem_inj₀ (hn : 0 < n) {z : (Asys n hn).Element} {X : Fin n → Set
    Str}
    (hX : ∀ j, MemA n (X j)) :
    (toAA hn z).mem (inj₀ (prodN X)) ↔ z.mem (embTuple false X) := by
  constructor
```

```
  · rintro (h0 | ⟨X', hX', heq, hz⟩ | ⟨Y', hY', heq, hz⟩)
    · exact absurd (h0 ▸ none_mem_sumMaster) none_mem_inj₀
    · rw [funext (prodN_injective (inj₀_injective heq))]; exact hz
    · obtain ⟨b, hb⟩ := apowNe hn (prodN X) (apow_mem_prodN hn hX)
      exact absurd (heq ▸ il_mem_inj₀.mpr hb) il_mem_inj₁
  · intro hz; exact Or.inr (Or.inl ⟨X, hX, rfl, hz⟩)

@[simp] theorem toAA_mem_inj₁ (hn : 0 < n) {z : (Asys n hn).Element} {Y : Fin n → Set
    Str}
    (hY : ∀ j, MemA n (Y j)) :
    (toAA hn z).mem (inj₁ (prodN Y)) ↔ z.mem (embTuple true Y) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hz⟩ | ⟨Y', hY', heq, hz⟩)
    · exact absurd (h0 ▸ none_mem_sumMaster) none_mem_inj₁
    · obtain ⟨a, ha⟩ := apowNe hn (prodN X') (apow_mem_prodN hn hX')
      exact absurd (heq ▸ il_mem_inj₀.mpr ha) il_mem_inj₁
    · rw [funext (prodN_injective (inj₁_injective heq))]; exact hz
  · intro hz; exact Or.inr (Or.inr ⟨Y, hY, rfl, hz⟩)

def fromAA (hn : 0 < n) (s : (AAsys hn).Element) : (Asys n hn).Element where
  mem W := W = Set.univ
    ∨ (∃ X, (∀ j, MemA n (X j)) ∧ W = embTuple false X ∧ s.mem (inj₀ (prodN X)))
    ∨ (∃ Y, (∀ j, MemA n (Y j)) ∧ W = embTuple true Y ∧ s.mem (inj₁ (prodN Y)))
  sub := by
    rintro W (rfl | ⟨X, hX, rfl, -⟩ | ⟨Y, hY, rfl, -⟩)
    · exact MemA.univ
    · exact MemA.tuple false hX
    · exact MemA.tuple true hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hsX⟩ | ⟨Y, hY, rfl, hsY⟩)
      (rfl | ⟨X', hX', rfl, hsX'⟩ | ⟨Y', hY', rfl, hsY'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (Or.inl ⟨X', hX', by rw [Set.univ_inter], hsX'⟩)
    · exact Or.inr (Or.inr ⟨Y', hY', by rw [Set.univ_inter], hsY'⟩)
    · exact Or.inr (Or.inl ⟨X, hX, by rw [Set.inter_univ], hsX⟩)
    · have hs := s.inter_mem hsX hsX'
      rw [inj₀_inter, prodN_inter] at hs
      exact Or.inr (Or.inl ⟨fun j => X j ∩ X' j,
        apow_mem_prodN_inv hn (aa_mem_inj₀_inv hn (s.sub hs)), by rw [embTuple_inter],
    hs⟩)
    · exfalso
      have hs := s.inter_mem hsX hsY'; rw [inj₀_inter_inj₁] at hs
      obtain ⟨t, ht⟩ := aa_mem_nonempty hn (s.sub hs); exact Set.notMem_empty t ht
    · exact Or.inr (Or.inr ⟨Y, hY, by rw [Set.inter_univ], hsY⟩)
    · exfalso
      have hs := s.inter_mem hsY hsX'; rw [Set.inter_comm, inj₀_inter_inj₁] at hs
      obtain ⟨t, ht⟩ := aa_mem_nonempty hn (s.sub hs); exact Set.notMem_empty t ht
    · have hs := s.inter_mem hsY hsY'
      rw [inj₁_inter, prodN_inter] at hs
      exact Or.inr (Or.inr ⟨fun j => Y j ∩ Y' j,
        apow_mem_prodN_inv hn (aa_mem_inj₁_inv hn (s.sub hs)), by rw [embTuple_inter],
    hs⟩)
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hsX⟩ | ⟨Y, hY, rfl, hsY⟩) hW' hsub
    · exact Or.inl (Set.univ_subset_iff.mp hsub)
```

```
    · cases hW' with
      | univ => exact Or.inl rfl
      | @tuple i'' V hV =>
        have htag : false = i'' := tag_eq_of_subset hn hX hsub
        subst htag
        refine Or.inr (Or.inl ⟨V, hV, rfl, ?_⟩)
        exact s.up_mem hsX (aa_mem_inj₀ hn hV)
          (inj₀_subset_inj₀.mpr (prodN_subset.mpr (embTuple_subset.mp hsub)))
    · cases hW' with
      | univ => exact Or.inl rfl
      | @tuple i'' V hV =>
        have htag : true = i'' := tag_eq_of_subset hn hY hsub
        subst htag

        refine Or.inr (Or.inr ⟨V, hV, rfl, ?_⟩)
        exact s.up_mem hsY (aa_mem_inj₁ hn hV)
          (inj₁_subset_inj₁.mpr (prodN_subset.mpr (embTuple_subset.mp hsub)))

@[simp] theorem fromAA_mem_embF (hn : 0 < n) {s : (AAsys hn).Element} {X : Fin n → Set
    Str}
    (hX : ∀ j, MemA n (X j)) :
    (fromAA hn s).mem (embTuple false X) ↔ s.mem (inj₀ (prodN X)) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hs⟩ | ⟨Y', hY', heq, hs⟩)
    · exact absurd h0 (embTuple_ne_univ X)
    · rw [funext (embTuple_injective heq)]; exact hs
    · exact absurd heq
        (embTuple_ne (by decide : (false : Bool) = true) ⟨0, hn⟩ (memA_nonempty hn (hX
    ⟨0, hn⟩)))
  · intro hs; exact Or.inr (Or.inl ⟨X, hX, rfl, hs⟩)

@[simp] theorem fromAA_mem_embT (hn : 0 < n) {s : (AAsys hn).Element} {Y : Fin n → Set
    Str}
    (hY : ∀ j, MemA n (Y j)) :
    (fromAA hn s).mem (embTuple true Y) ↔ s.mem (inj₁ (prodN Y)) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hs⟩ | ⟨Y', hY', heq, hs⟩)
    · exact absurd h0 (embTuple_ne_univ Y)
    · exact absurd heq.symm
        (embTuple_ne (by decide : (false : Bool) = true) ⟨0, hn⟩ (memA_nonempty hn (hX'
    ⟨0, hn⟩)))
    · rw [funext (embTuple_injective heq)]; exact hs
  · intro hs; exact Or.inr (Or.inr ⟨Y, hY, rfl, hs⟩)

theorem fromAA_toAA (hn : 0 < n) (z : (Asys n hn).Element) : fromAA hn (toAA hn z) = z
    := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨X, hX, rfl, hs⟩ | ⟨Y, hY, rfl, hs⟩)
    · exact z.master_mem
    · exact (toAA_mem_inj₀ hn hX).mp hs
    · exact (toAA_mem_inj₁ hn hY).mp hs
  · intro hW
    cases z.sub hW with
    | univ => exact Or.inl rfl
```

```
    | @tuple i X hX =>
      cases i with
      | false => exact Or.inr (Or.inl ⟨X, hX, rfl, (toAA_mem_inj₀ hn hX).mpr hW⟩)
      | true => exact Or.inr (Or.inr ⟨X, hX, rfl, (toAA_mem_inj₁ hn hX).mpr hW⟩)

theorem toAA_fromAA (hn : 0 < n) (s : (AAsys hn).Element) : toAA hn (fromAA hn s) = s
    := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨X, hX, rfl, hs⟩ | ⟨Y, hY, rfl, hs⟩)
    · exact s.master_mem
    · exact (fromAA_mem_embF hn hX).mp hs
    · exact (fromAA_mem_embT hn hY).mp hs
  · intro hW
    rcases s.sub hW with rfl | ⟨V, hV, rfl⟩ | ⟨V, hV, rfl⟩
    · exact Or.inl rfl
    · obtain ⟨X, hX, rfl⟩ := hV
      exact Or.inr (Or.inl ⟨X, hX, rfl, (fromAA_mem_embF hn hX).mpr hW⟩)
    · obtain ⟨Y, hY, rfl⟩ := hV
      exact Or.inr (Or.inr ⟨Y, hY, rfl, (fromAA_mem_embT hn hY).mpr hW⟩)

def aaEquiv (hn : 0 < n) : (Asys n hn).Element ≃o (AAsys hn).Element where
  toFun := toAA hn
  invFun := fromAA hn
  left_inv := fromAA_toAA hn
  right_inv := toAA_fromAA hn
  map_rel_iff' := by
    intro z z'
    constructor
    · intro h W hW
      cases z.sub hW with
      | univ => exact z'.master_mem
      | @tuple i X hX =>
        cases i with
        | false => exact (toAA_mem_inj₀ hn hX).mp (h _ (Or.inr (Or.inl ⟨X, hX, rfl, hW⟩
    )))
        | true => exact (toAA_mem_inj₁ hn hX).mp (h _ (Or.inr (Or.inr ⟨X, hX, rfl, hW⟩))
    ))
    · intro h W hW
      rcases hW with rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨X, hX, rfl, h _ hzX⟩)
      · exact Or.inr (Or.inr ⟨Y, hY, rfl, h _ hzY⟩)

theorem A_domain_equation (hn : 0 < n) : Asys n hn ≅? AAsys hn := ⟨aaEquiv hn⟩

end SeqEqA
end DomainTheory.NeighbourhoodSystems
```

4.7.6 seqEqC — C equation

Sequence domain C: three-way sum tags and the equation C ≅ [U+1D7D9] + C + C.

Definition (`t0`). As in the library.

Proof. By definition. ■

Definition (`j0`). As in the library.

Proof. By definition. ■

Definition (`t1`). As in the library.

Proof. By definition. ■

Definition (`j1`). As in the library.

Proof. By definition. ■

Definition (`t2`). As in the library.

Proof. By definition. ■

Definition (`j2`). As in the library.

Proof. By definition. ■

Theorem (`j0_inter_j0`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j0_inter_j1`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j0_inter_j2`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j0_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j0_subset_j0`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j1_inter_j1`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j1_inter_j2`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j1_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j1_subset_j1`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j1_injective`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j2_inter_j2`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j2_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j2_subset_j2`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j2_injective`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`master3`). As in the library.

Proof. By definition. ■

Theorem (`C_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`embBit_singleton`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`eq_master3_of_subset`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j0_subset_master3`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j1_subset_master3`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j2_subset_master3`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`master3_inter_j0`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`master3_inter_j1`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`master3_inter_j2`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`memC_cases`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`memC_embBit`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`memC_embBit_inv`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`singleton_nil_inter_embBit`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`singleton_nil_ne_embBit`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`singleton_nil_ne_univ`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`sum3`). As in the library.

Proof. By definition. ■

Theorem (`unitSys_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`CC`). As in the library.

Proof. By definition. ■

Theorem (`sum3_mem_j1_inv`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`sum3_mem_j2_inv`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`sum3_mem_nonempty`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`toCC`). As in the library.

Proof. By definition. ■

Definition (`fromCC`). As in the library.

Proof. By definition. ■

Theorem (`fromCC_toCC`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`toCC_fromCC`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Definition (`ccEquiv`). As in the library.

Proof. By definition. ■

Theorem (`C_domain_equation`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`j0_injective`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

seqEqC.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.seqEqB
import Mathlib.Data.Set.Finite.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap BitDom SeqDom SeqEqB

def t0 {α β ? : Type*} (a : α) : Option (α ? β ? ?) := some (Sum.inl a)

def j0 {α β ? : Type*} (X : Set α) : Set (Option (α ? β ? ?)) := {w | ∃ a, w = t0 a ∧
    a ∈ X}

variable {α β ? : Type*}

@[simp] theorem t0_mem_j0 {X : Set α} {a : α} : (t0 a : Option (α ? β ? ?)) ∈ j0 X ↔
    a ∈ X := by
  constructor
  · rintro ⟨a', heq, ha'⟩; simp only [t0, Option.some.injEq, Sum.inl.injEq] at heq;
    exact heq ▶ ha'
  · intro ha; exact ⟨a, rfl, ha⟩

def t1 {α β ? : Type*} (b : β) : Option (α ? β ? ?) := some (Sum.inr (Sum.inl b))

def j1 {α β ? : Type*} (Y : Set β) : Set (Option (α ? β ? ?)) := {w | ∃ b, w = t1 b ∧
    b ∈ Y}

variable {α β ? : Type*}

@[simp] theorem t1_mem_j1 {Y : Set β} {b : β} : (t1 b : Option (α ? β ? ?)) ∈ j1 Y ↔
    b ∈ Y := by
  constructor
  · rintro ⟨b', heq, hb'⟩
    simp only [t1, Option.some.injEq, Sum.inr.injEq, Sum.inl.injEq] at heq; exact heq
    ▶ hb'
  · intro hb; exact ⟨b, rfl, hb⟩

def t2 {α β ? : Type*} (c : ?) : Option (α ? β ? ?) := some (Sum.inr (Sum.inr c))

def j2 {α β ? : Type*} (Z : Set ?) : Set (Option (α ? β ? ?)) := {w | ∃ c, w = t2 c ∧
    c ∈ Z}

variable {α β ? : Type*}

@[simp] theorem t2_mem_j2 {Z : Set ?} {c : ?} : (t2 c : Option (α ? β ? ?)) ∈ j2 Z ↔
    c ∈ Z := by
  constructor
  · rintro ⟨c', heq, hc'⟩
    simp only [t2, Option.some.injEq, Sum.inr.injEq] at heq; exact heq ▶ hc'
  · intro hc; exact ⟨c, rfl, hc⟩

@[simp] theorem none_not_mem_j0 {X : Set α} : (none : Option (α ? β ? ?)) ∈ j0 X := by
```

```
  rintro ⟨a, heq, -⟩; exact absurd heq (by simp [t0])

@[simp] theorem none_not_mem_j1 {Y : Set β} : (none : Option (α ? β ? ?)) ∈ j1 Y := by
  rintro ⟨b, heq, -⟩; exact absurd heq (by simp [t1])

@[simp] theorem none_not_mem_j2 {Z : Set ?} : (none : Option (α ? β ? ?)) ∈ j2 Z := by
  rintro ⟨c, heq, -⟩; exact absurd heq (by simp [t2])

@[simp] theorem t1_not_mem_j0 {X : Set α} {b : β} : (t1 b : Option (α ? β ? ?)) ∈ j0 X
    := by
  rintro ⟨a, heq, -⟩; exact absurd heq (by simp [t0, t1])

@[simp] theorem t2_not_mem_j0 {X : Set α} {c : ?} : (t2 c : Option (α ? β ? ?)) ∈ j0 X
    := by
  rintro ⟨a, heq, -⟩; exact absurd heq (by simp [t0, t2])

@[simp] theorem t0_not_mem_j1 {Y : Set β} {a : α} : (t0 a : Option (α ? β ? ?)) ∈ j1 Y
    := by
  rintro ⟨b, heq, -⟩; exact absurd heq (by simp [t0, t1])

@[simp] theorem t2_not_mem_j1 {Y : Set β} {c : ?} : (t2 c : Option (α ? β ? ?)) ∈ j1 Y
    := by
  rintro ⟨b, heq, -⟩; exact absurd heq (by simp [t1, t2])

@[simp] theorem t0_not_mem_j2 {Z : Set ?} {a : α} : (t0 a : Option (α ? β ? ?)) ∈ j2 Z
    := by
  rintro ⟨c, heq, -⟩; exact absurd heq (by simp [t0, t2])

@[simp] theorem t1_not_mem_j2 {Z : Set ?} {b : β} : (t1 b : Option (α ? β ? ?)) ∈ j2 Z
    := by
  rintro ⟨c, heq, -⟩; exact absurd heq (by simp [t1, t2])

theorem j0_inter_j0 (X X' : Set α) :
    (j0 X ∩ j0 X' : Set (Option (α ? β ? ?))) = j0 (X ∩ X') := by
  ext w; rcases w with _ | (a | b | c) <;>
    simp [j0, t0, Set.mem_inter_iff]

theorem j0_inter_j1 (X : Set α) (Y : Set β) :
    (j0 X ∩ j1 Y : Set (Option (α ? β ? ?))) = ∅ := by
  ext w; rcases w with _ | (a | b | c) <;>
    simp [j0, j1, t0, t1, Set.mem_inter_iff]

theorem j0_inter_j2 (X : Set α) (Z : Set ?) :
    (j0 X ∩ j2 Z : Set (Option (α ? β ? ?))) = ∅ := by
  ext w; rcases w with _ | (a | b | c) <;>
    simp [j0, j2, t0, t2, Set.mem_inter_iff]

theorem j0_nonempty {X : Set α} (hX : X.Nonempty) : (j0 X : Set (Option (α ? β ? ?))).
    Nonempty := by
  obtain ⟨a, ha⟩ := hX; exact ⟨t0 a, a, rfl, ha⟩

theorem j0_subset_j0 {X X' : Set α} :
    (j0 X : Set (Option (α ? β ? ?))) ⊆ j0 X' ↔ X ⊆ X' := by
  constructor
  · intro h a ha; exact t0_mem_j0.mp (h (t0_mem_j0.mpr ha))
  · rintro h w ⟨a, rfl, ha⟩; exact t0_mem_j0.mpr (h ha)
```

```
theorem j1_inter_j1 (Y Y' : Set β) :
    (j1 Y ∩ j1 Y' : Set (Option (α ? β ? ?))) = j1 (Y ∩ Y') := by
  ext w; rcases w with _ | (a | b | c) <;>
    simp [j1, t1, Set.mem_inter_iff]

theorem j1_inter_j2 (Y : Set β) (Z : Set ?) :
    (j1 Y ∩ j2 Z : Set (Option (α ? β ? ?))) = ∅ := by
  ext w; rcases w with _ | (a | b | c) <;>
    simp [j1, j2, t1, t2, Set.mem_inter_iff]

theorem j1_nonempty {Y : Set β} (hY : Y.Nonempty) : (j1 Y : Set (Option (α ? β ? ?))).
    Nonempty := by
  obtain ⟨b, hb⟩ := hY; exact ⟨t1 b, b, rfl, hb⟩

theorem j1_subset_j1 {Y Y' : Set β} :
    (j1 Y : Set (Option (α ? β ? ?))) ⊆ j1 Y' ↔ Y ⊆ Y' := by
  constructor
  · intro h b hb; exact t1_mem_j1.mp (h (t1_mem_j1.mpr hb))
  · rintro h w ⟨b, rfl, hb⟩; exact t1_mem_j1.mpr (h hb)

theorem j1_injective {Y Y' : Set β}
    (h : (j1 Y : Set (Option (α ? β ? ?))) = j1 Y') : Y = Y' :=
  Set.Subset.antisymm (j1_subset_j1.mp h.subset) (j1_subset_j1.mp h.symm.subset)

theorem j2_inter_j2 (Z Z' : Set ?) :
    (j2 Z ∩ j2 Z' : Set (Option (α ? β ? ?))) = j2 (Z ∩ Z') := by
  ext w; rcases w with _ | (a | b | c) <;>
    simp [j2, t2, Set.mem_inter_iff]

theorem j2_nonempty {Z : Set ?} (hZ : Z.Nonempty) : (j2 Z : Set (Option (α ? β ? ?))).
    Nonempty := by
  obtain ⟨c, hc⟩ := hZ; exact ⟨t2 c, c, rfl, hc⟩

theorem j2_subset_j2 {Z Z' : Set ?} :
    (j2 Z : Set (Option (α ? β ? ?))) ⊆ j2 Z' ↔ Z ⊆ Z' := by
  constructor
  · intro h c hc; exact t2_mem_j2.mp (h (t2_mem_j2.mpr hc))
  · rintro h w ⟨c, rfl, hc⟩; exact t2_mem_j2.mpr (h hc)

theorem j2_injective {Z Z' : Set ?}
    (h : (j2 Z : Set (Option (α ? β ? ?))) = j2 Z') : Z = Z' :=
  Set.Subset.antisymm (j2_subset_j2.mp h.subset) (j2_subset_j2.mp h.symm.subset)

variable {α β ? : Type*}
variable (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
    NeighborhoodSystem ?)

def master3 : Set (Option (α ? β ? ?)) :=
  insert none (j0 V₀.master ∪ j1 V₁.master ∪ j2 V₂.master)

variable {α β ? : Type*}
variable (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
    NeighborhoodSystem ?)
variable {V₀ V₁ V₂}
```

```
@[simp] theorem none_mem_master3 : (none : Option (α ? β ? ?)) ∈ master3 V₀ V₁ V₂ :=
  Set.mem_insert _ _

namespace SeqEqC

theorem C_nonempty : ∀ X, C.mem X → X.Nonempty := by
  rintro X (⟨σ, rfl⟩ | ⟨σ, rfl⟩)
  · exact ⟨σ, List.prefix_rfl⟩
  · exact ⟨σ, rfl⟩

theorem embBit_singleton (b : Bool) (σ : Str) : embBit b ({σ} : Set Str) = {(b :: σ :
    Str)} := by
  rw [embBit_eq_prepend, SeqDom.prepend_singleton]; rfl
end SeqEqC

theorem eq_master3_of_subset {W : Set (Option (α ? β ? ?))}
    (hsub : master3 V₀ V₁ V₂ ⊆ W) (hsub' : W ⊆ master3 V₀ V₁ V₂) : W = master3 V₀ V₁ V₂
    :=
  Set.Subset.antisymm hsub' hsub

theorem j0_subset_master3 {X : Set α} (hX : V₀.mem X) :
    (j0 X : Set (Option (α ? β ? ?))) ⊆ master3 V₀ V₁ V₂ := by
  rintro w ⟨a, rfl, ha⟩
  exact Set.mem_insert_iff.mpr (Or.inr (Set.mem_union_left _
    (Set.mem_union_left _ (t0_mem_j0.mpr (V₀.sub_master hX ha)))))

theorem j1_subset_master3 {Y : Set β} (hY : V₁.mem Y) :
    (j1 Y : Set (Option (α ? β ? ?))) ⊆ master3 V₀ V₁ V₂ := by
  rintro w ⟨b, rfl, hb⟩
  exact Set.mem_insert_iff.mpr (Or.inr (Set.mem_union_left _
    (Set.mem_union_right _ (t1_mem_j1.mpr (V₁.sub_master hY hb)))))

theorem j2_subset_master3 {Z : Set ?} (hZ : V₂.mem Z) :
    (j2 Z : Set (Option (α ? β ? ?))) ⊆ master3 V₀ V₁ V₂ := by
  rintro w ⟨c, rfl, hc⟩
  exact Set.mem_insert_iff.mpr (Or.inr (Set.mem_union_right _ (t2_mem_j2.mpr (V₂.
    sub_master hZ hc))))

theorem master3_inter_j0 {X : Set α} (hX : V₀.mem X) :
    (master3 V₀ V₁ V₂ ∩ j0 X : Set (Option (α ? β ? ?))) = j0 X :=
  Set.inter_eq_right.mpr (j0_subset_master3 hX)

theorem master3_inter_j1 {Y : Set β} (hY : V₁.mem Y) :
    (master3 V₀ V₁ V₂ ∩ j1 Y : Set (Option (α ? β ? ?))) = j1 Y :=
  Set.inter_eq_right.mpr (j1_subset_master3 hY)

theorem master3_inter_j2 {Z : Set ?} (hZ : V₂.mem Z) :
    (master3 V₀ V₁ V₂ ∩ j2 Z : Set (Option (α ? β ? ?))) = j2 Z :=
  Set.inter_eq_right.mpr (j2_subset_master3 hZ)

namespace SeqEqC

theorem memC_cases {W : Set Str} (hW : C.mem W) :
    W = Set.univ ∨ W = ({[]} : Set Str)
      ∨ (∃ X, C.mem X ∧ W = embBit false X) ∨ (∃ Y, C.mem Y ∧ W = embBit true Y) := by
  rcases hW with ⟨σ, rfl⟩ | ⟨σ, rfl⟩
```

```
  · cases σ with
    | nil => exact Or.inl cone.nil
    | cons b σ' => cases b with
      | false =>
        exact Or.inr (Or.inr (Or.inl ⟨cone σ', memC_cone σ', (embBit_cone false σ').
    symm⟩))
      | true =>
        exact Or.inr (Or.inr (Or.inr ⟨cone σ', memC_cone σ', (embBit_cone true σ').symm
    ⟩))
  · cases σ with
    | nil => exact Or.inr (Or.inl rfl)
    | cons b σ' => cases b with
      | false =>
        exact Or.inr (Or.inr (Or.inl ⟨{σ'}, memC_singleton σ', (embBit_singleton false
    σ').symm⟩))
      | true =>
        exact Or.inr (Or.inr (Or.inr ⟨{σ'}, memC_singleton σ', (embBit_singleton true σ
    ').symm⟩))

theorem memC_embBit (b : Bool) {X : Set Str} (hX : C.mem X) : C.mem (embBit b X) := by
  rw [embBit_eq_prepend]; exact SeqDom.memC_prepend [b] hX

theorem memC_embBit_inv {b : Bool} {W : Set Str} (h : C.mem (embBit b W)) : C.mem W :=
     by
  rcases h with ⟨σ, hσ⟩ | ⟨σ, hσ⟩
  · have hmem : σ ∈ embBit b W := hσ ▶ (show σ ∈ cone σ from List.prefix_rfl)
    obtain ⟨w', rfl, -⟩ := hmem
    rw [← embBit_cone] at hσ; rw [embBit_injective hσ]; exact memC_cone w'
  · have hmem : σ ∈ embBit b W := hσ ▶ (Set.mem_singleton_iff.mpr rfl : σ ∈ ({σ} : Set
     Str))
    obtain ⟨w', rfl, -⟩ := hmem
    rw [← embBit_singleton] at hσ; rw [embBit_injective hσ]; exact memC_singleton w'

theorem singleton_nil_inter_embBit (b : Bool) (X : Set Str) :
    (({[]} : Set Str) ∩ embBit b X) = ∅ := by
  ext w
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, mem_embBit, Set.
    mem_empty_iff_false,
    iff_false, not_and]
  rintro rfl ⟨w', heq, -⟩
  exact absurd heq (by simp)

theorem singleton_nil_ne_embBit (b : Bool) (X : Set Str) : ({[]} : Set Str) = embBit b
     X := by
  intro h
  exact nil_not_mem_embBit (h ▶ (Set.mem_singleton_iff.mpr rfl : ([] : Str) ∈ ({[]} :
    Set Str)))

theorem singleton_nil_ne_univ : ({[]} : Set Str) = Set.univ := by
  intro h
  have hmem : ([true] : Str) ∈ ({[]} : Set Str) := h ▶ Set.mem_univ _
  rw [Set.mem_singleton_iff] at hmem; exact absurd hmem (by simp)

end SeqEqC

def sum3 (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) (V₂ :
```

```
  NeighborhoodSystem ?)
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)
  (h₂ : ∀ Z, V₂.mem Z → Z.Nonempty) : NeighborhoodSystem (Option (α ? β ? ?)) where
mem W := W = master3 V₀ V₁ V₂ ∨ (∃ X, V₀.mem X ∧ W = j0 X)
  ∨ (∃ Y, V₁.mem Y ∧ W = j1 Y) ∨ (∃ Z, V₂.mem Z ∧ W = j2 Z)
master := master3 V₀ V₁ V₂
master_nonempty := ⟨none, none_mem_master3⟩
master_mem := Or.inl rfl
sub_master := by
  rintro W (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩ | ⟨Z, hZ, rfl⟩)
  · exact subset_rfl
  · exact j0_subset_master3 hX
  · exact j1_subset_master3 hY
  · exact j2_subset_master3 hZ
inter_mem := by
  have hne : ∀ W, (W = master3 V₀ V₁ V₂ ∨ (∃ X, V₀.mem X ∧ W = j0 X)
      ∨ (∃ Y, V₁.mem Y ∧ W = j1 Y) ∨ (∃ Z, V₂.mem Z ∧ W = j2 Z)) →
      (W : Set (Option (α ? β ? ?))).Nonempty := by
    rintro W (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩ | ⟨Z, hZ, rfl⟩)
    · exact ⟨none, none_mem_master3⟩
    · exact j0_nonempty (h₀ X hX)
    · exact j1_nonempty (h₁ Y hY)
    · exact j2_nonempty (h₂ Z hZ)
  rintro W W' Z hW hW' hZ hZsub
  rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩ | ⟨Zc, hZc, rfl⟩
  · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
    · rw [Set.inter_self]; exact Or.inl rfl
    · rw [master3_inter_j0 hX']; exact Or.inr (Or.inl ⟨X', hX', rfl⟩)
    · rw [master3_inter_j1 hY']; exact Or.inr (Or.inr (Or.inl ⟨Y', hY', rfl⟩))
    · rw [master3_inter_j2 hZ']; exact Or.inr (Or.inr (Or.inr ⟨Z', hZ', rfl⟩))
  · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
    · rw [Set.inter_comm, master3_inter_j0 hX]; exact Or.inr (Or.inl ⟨X, hX, rfl⟩)
    · rw [j0_inter_j0] at hZsub ⊢
      rcases hZ with rfl | ⟨Z0, hZ0, rfl⟩ | ⟨Z1, hZ1, rfl⟩ | ⟨Z2, hZ2, rfl⟩
      · exact absurd (hZsub none_mem_master3) (by simp)
      · exact Or.inr (Or.inl ⟨X ∩ X', V₀.inter_mem hX hX' hZ0 (j0_subset_j0.mp hZsub)
  , rfl⟩)
      · obtain ⟨b, hb⟩ := h₁ Z1 hZ1; exact absurd (hZsub (t1_mem_j1.mpr hb)) (by simp)
      · obtain ⟨c, hc⟩ := h₂ Z2 hZ2; exact absurd (hZsub (t2_mem_j2.mpr hc)) (by simp)
    · rw [j0_inter_j1] at hZsub ⊢
      obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    · rw [j0_inter_j2] at hZsub ⊢
      obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
  · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
    · rw [Set.inter_comm, master3_inter_j1 hY]; exact Or.inr (Or.inr (Or.inl ⟨Y, hY,
  rfl⟩))
    · rw [Set.inter_comm, j0_inter_j1] at hZsub ⊢
      obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    · rw [j1_inter_j1] at hZsub ⊢
      rcases hZ with rfl | ⟨Z0, hZ0, rfl⟩ | ⟨Z1, hZ1, rfl⟩ | ⟨Z2, hZ2, rfl⟩
      · exact absurd (hZsub none_mem_master3) (by simp)
      · obtain ⟨a, ha⟩ := h₀ Z0 hZ0; exact absurd (hZsub (t0_mem_j0.mpr ha)) (by simp)
      · exact Or.inr (Or.inr (Or.inl ⟨Y ∩ Y', V₁.inter_mem hY hY' hZ1 (j1_subset_j1.
  mp hZsub), rfl⟩))
      · obtain ⟨c, hc⟩ := h₂ Z2 hZ2; exact absurd (hZsub (t2_mem_j2.mpr hc)) (by simp)
    · rw [j1_inter_j2] at hZsub ⊢
```

```
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
      · rw [Set.inter_comm, master3_inter_j2 hZc]; exact Or.inr (Or.inr (Or.inr ⟨Zc,
    hZc, rfl⟩))
      · rw [Set.inter_comm, j0_inter_j2] at hZsub ⊢
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      · rw [Set.inter_comm, j1_inter_j2] at hZsub ⊢
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      · rw [j2_inter_j2] at hZsub ⊢
        rcases hZ with rfl | ⟨Z0, hZ0, rfl⟩ | ⟨Z1, hZ1, rfl⟩ | ⟨Z2, hZ2, rfl⟩
        · exact absurd (hZsub none_mem_master3) (by simp)
        · obtain ⟨a, ha⟩ := h₀ Z0 hZ0; exact absurd (hZsub (t0_mem_j0.mpr ha)) (by simp)
        · obtain ⟨b, hb⟩ := h₁ Z1 hZ1; exact absurd (hZsub (t1_mem_j1.mpr hb)) (by simp)
        · exact Or.inr (Or.inr (Or.inr ⟨Zc ∩ Z', V₂.inter_mem hZc hZ' hZ2 (j2_subset_j2
    .mp hZsub), rfl⟩))

namespace SeqEqC

theorem unitSys_nonempty : ∀ X, unitSys.mem X → X.Nonempty := by
  rintro X rfl; exact Set.univ_nonempty

abbrev CC : NeighborhoodSystem (Option (Unit ? Str ? Str)) :=
  sum3 unitSys C C unitSys_nonempty C_nonempty C_nonempty

theorem sum3_mem_j1_inv {X : Set Str} (h : CC.mem (j1 X)) : C.mem X := by
  rcases h with h0 | ⟨U, hU, heq⟩ | ⟨Y, hY, heq⟩ | ⟨Z, hZ, heq⟩
  · exact absurd (h0 ▶ none_mem_master3) none_not_mem_j1
  · obtain ⟨a, ha⟩ := unitSys_nonempty U hU; exact absurd (heq ▶ (t0_mem_j0.mpr ha))
    t0_not_mem_j1
  · rw [j1_injective heq]; exact hY
  · obtain ⟨c, hc⟩ := C_nonempty Z hZ; exact absurd (heq ▶ (t2_mem_j2.mpr hc))
    t2_not_mem_j1

theorem sum3_mem_j2_inv {Y : Set Str} (h : CC.mem (j2 Y)) : C.mem Y := by
  rcases h with h0 | ⟨U, hU, heq⟩ | ⟨Z, hZ, heq⟩ | ⟨Y', hY', heq⟩
  · exact absurd (h0 ▶ none_mem_master3) none_not_mem_j2
  · obtain ⟨a, ha⟩ := unitSys_nonempty U hU; exact absurd (heq ▶ (t0_mem_j0.mpr ha))
    t0_not_mem_j2
  · obtain ⟨c, hc⟩ := C_nonempty Z hZ; exact absurd (heq ▶ (t1_mem_j1.mpr hc))
    t1_not_mem_j2
  · rw [j2_injective heq]; exact hY'

theorem sum3_mem_nonempty {W : Set (Option (Unit ? Str ? Str))} (h : CC.mem W) : W.
    Nonempty := by
  rcases h with rfl | ⟨U, hU, rfl⟩ | ⟨Y, hY, rfl⟩ | ⟨Z, hZ, rfl⟩
  · exact ⟨none, none_mem_master3⟩
  · exact j0_nonempty (unitSys_nonempty U hU)
  · exact j1_nonempty (C_nonempty Y hY)
  · exact j2_nonempty (C_nonempty Z hZ)

def toCC (x : C.Element) : CC.Element where
  mem W := W = master3 unitSys C C
    ∨ (W = j0 (Set.univ : Set Unit) ∧ x.mem ({[]} : Set Str))
    ∨ (∃ X, C.mem X ∧ W = j1 X ∧ x.mem (embBit false X))
    ∨ (∃ Y, C.mem Y ∧ W = j2 Y ∧ x.mem (embBit true Y))
  sub := by
```

```
  rintro W (rfl | ⟨rfl, -⟩ | ⟨X, hX, rfl, -⟩ | ⟨Y, hY, rfl, -⟩)
  · exact Or.inl rfl
  · exact Or.inr (Or.inl ⟨Set.univ, rfl, rfl⟩)
  · exact Or.inr (Or.inr (Or.inl ⟨X, hX, rfl⟩))
  · exact Or.inr (Or.inr (Or.inr ⟨Y, hY, rfl⟩))
master_mem := Or.inl rfl
inter_mem := by
  rintro W W' (rfl | ⟨rfl, hzU⟩ | ⟨X, hX, rfl, hzF⟩ | ⟨Y, hY, rfl, hzT⟩)
    (rfl | ⟨rfl, hzU'⟩ | ⟨X', hX', rfl, hzF'⟩ | ⟨Y', hY', rfl, hzT'⟩)
  · exact Or.inl (by rw [Set.inter_self])
  · exact Or.inr (Or.inl ⟨master3_inter_j0 rfl, hzU'⟩)
  · exact Or.inr (Or.inr (Or.inl ⟨X', hX', master3_inter_j1 hX', hzF'⟩))
  · exact Or.inr (Or.inr (Or.inr ⟨Y', hY', master3_inter_j2 hY', hzT'⟩))
  · exact Or.inr (Or.inl ⟨by rw [Set.inter_comm, master3_inter_j0 rfl], hzU⟩)
  · refine Or.inr (Or.inl ⟨?_, hzU⟩)
    rw [j0_inter_j0, Set.inter_self]
  · exfalso
    have hx := x.inter_mem hzU hzF'
    rw [singleton_nil_inter_embBit] at hx
    obtain ⟨t, ht⟩ := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
  · exfalso
    have hx := x.inter_mem hzU hzT'
    rw [singleton_nil_inter_embBit] at hx
    obtain ⟨t, ht⟩ := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
  · exact Or.inr (Or.inr (Or.inl ⟨X, hX, by rw [Set.inter_comm, master3_inter_j1 hX],
   hzF⟩))
  · exfalso
    have hx := x.inter_mem hzF hzU'
    rw [Set.inter_comm, singleton_nil_inter_embBit] at hx
    obtain ⟨t, ht⟩ := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
  · refine Or.inr (Or.inr (Or.inl ⟨X ∩ X', ?_, j1_inter_j1 X X', ?_⟩))
    · have hx := x.inter_mem hzF hzF'; rw [embBit_inter] at hx; exact memC_embBit_inv
   (x.sub hx)
    · have hx := x.inter_mem hzF hzF'; rwa [embBit_inter] at hx
  · exfalso
    have hx := x.inter_mem hzF hzT'
    rw [embBit_inter_ne (show (false : Bool) = true by decide)] at hx
    obtain ⟨t, ht⟩ := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
  · exact Or.inr (Or.inr (Or.inr ⟨Y, hY, by rw [Set.inter_comm, master3_inter_j2 hY],
   hzT⟩))
  · exfalso
    have hx := x.inter_mem hzT hzU'
    rw [Set.inter_comm, singleton_nil_inter_embBit] at hx
    obtain ⟨t, ht⟩ := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
  · exfalso
    have hx := x.inter_mem hzT hzF'
    rw [embBit_inter_ne (show (true : Bool) = false by decide)] at hx
    obtain ⟨t, ht⟩ := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
  · refine Or.inr (Or.inr (Or.inr ⟨Y ∩ Y', ?_, j2_inter_j2 Y Y', ?_⟩))
    · have hx := x.inter_mem hzT hzT'; rw [embBit_inter] at hx; exact memC_embBit_inv
   (x.sub hx)
    · have hx := x.inter_mem hzT hzT'; rwa [embBit_inter] at hx
up_mem := by
  rintro W W' (rfl | ⟨rfl, hzU⟩ | ⟨X, hX, rfl, hzF⟩ | ⟨Y, hY, rfl, hzT⟩) hW' hsub
  · exact Or.inl (eq_master3_of_subset hsub (CC.sub_master hW'))
  · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
```

```
      · exact Or.inl rfl
      · obtain rfl := hX'; exact Or.inr (Or.inl ⟨rfl, hzU⟩)
      · exact absurd (hsub (t0_mem_j0.mpr (Set.mem_univ ()))) t0_not_mem_j1
      · exact absurd (hsub (t0_mem_j0.mpr (Set.mem_univ ()))) t0_not_mem_j2
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨b, hb⟩ := C_nonempty X hX
        exact absurd (hsub (t1_mem_j1.mpr hb)) t1_not_mem_j0
      · refine Or.inr (Or.inr (Or.inl ⟨Y', hY', rfl, ?_⟩))
        exact x.up_mem hzF (memC_embBit false hY') (embBit_subset.mpr (j1_subset_j1.mp
     hsub))
      · obtain ⟨b, hb⟩ := C_nonempty X hX
        exact absurd (hsub (t1_mem_j1.mpr hb)) t1_not_mem_j2
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨c, hc⟩ := C_nonempty Y hY
        exact absurd (hsub (t2_mem_j2.mpr hc)) t2_not_mem_j0
      · obtain ⟨c, hc⟩ := C_nonempty Y hY
        exact absurd (hsub (t2_mem_j2.mpr hc)) t2_not_mem_j1
      · refine Or.inr (Or.inr (Or.inr ⟨Z', hZ', rfl, ?_⟩))
        exact x.up_mem hzT (memC_embBit true hZ') (embBit_subset.mpr (j2_subset_j2.mp
    hsub))

@[simp] theorem toCC_mem_j0 {x : C.Element} :
    (toCC x).mem (j0 (Set.univ : Set Unit)) ↔ x.mem ({[]} : Set Str) := by
  constructor
  · rintro (h0 | ⟨-, hz⟩ | ⟨X', hX', heq, hz⟩ | ⟨Y', hY', heq, hz⟩)
    · exact absurd (h0 ▶ none_mem_master3) none_not_mem_j0
    · exact hz
    · exact absurd (heq ▶ (t0_mem_j0.mpr (Set.mem_univ ()))) t0_not_mem_j1
    · exact absurd (heq ▶ (t0_mem_j0.mpr (Set.mem_univ ()))) t0_not_mem_j2
  · intro hz; exact Or.inr (Or.inl ⟨rfl, hz⟩)

@[simp] theorem toCC_mem_j1 {x : C.Element} {X : Set Str} (hX : C.mem X) :
    (toCC x).mem (j1 X) ↔ x.mem (embBit false X) := by
  constructor
  · rintro (h0 | ⟨heq, hz⟩ | ⟨X', hX', heqj, hz⟩ | ⟨Y', hY', heqj, hz⟩)
    · exact absurd (h0 ▶ none_mem_master3) none_not_mem_j1
    · obtain ⟨b, hb⟩ := C_nonempty X hX
      exact absurd (heq ▶ (t1_mem_j1.mpr hb)) t1_not_mem_j0
    · rw [j1_injective heqj]; exact hz
    · obtain ⟨b, hb⟩ := C_nonempty X hX
      exact absurd (heqj ▶ (t1_mem_j1.mpr hb)) t1_not_mem_j2
  · intro hz; exact Or.inr (Or.inr (Or.inl ⟨X, hX, rfl, hz⟩))

@[simp] theorem toCC_mem_j2 {x : C.Element} {Y : Set Str} (hY : C.mem Y) :
    (toCC x).mem (j2 Y) ↔ x.mem (embBit true Y) := by
  constructor
  · rintro (h0 | ⟨heq, hz⟩ | ⟨X', hX', heqj, hz⟩ | ⟨Y', hY', heqj, hz⟩)
    · exact absurd (h0 ▶ none_mem_master3) none_not_mem_j2
    · obtain ⟨c, hc⟩ := C_nonempty Y hY
      exact absurd (heq ▶ (t2_mem_j2.mpr hc)) t2_not_mem_j0
    · obtain ⟨c, hc⟩ := C_nonempty Y hY
      exact absurd (heqj ▶ (t2_mem_j2.mpr hc)) t2_not_mem_j1
    · rw [j2_injective heqj]; exact hz
  · intro hz; exact Or.inr (Or.inr (Or.inr ⟨Y, hY, rfl, hz⟩))
```

```
def fromCC (s : CC.Element) : C.Element where
  mem W := W = Set.univ
    ∨ (W = ({[]} : Set Str) ∧ s.mem (j0 (Set.univ : Set Unit)))
    ∨ (∃ X, C.mem X ∧ W = embBit false X ∧ s.mem (j1 X))
    ∨ (∃ Y, C.mem Y ∧ W = embBit true Y ∧ s.mem (j2 Y))
  sub := by
    rintro W (rfl | ⟨rfl, -⟩ | ⟨X, hX, rfl, -⟩ | ⟨Y, hY, rfl, -⟩)
    · exact Or.inl ⟨[], cone.nil.symm⟩
    · exact memC_singleton []
    · exact memC_embBit false hX
    · exact memC_embBit true hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨rfl, hsU⟩ | ⟨X, hX, rfl, hsF⟩ | ⟨Y, hY, rfl, hsT⟩)
      (rfl | ⟨rfl, hsU'⟩ | ⟨X', hX', rfl, hsF'⟩ | ⟨Y', hY', rfl, hsT'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (Or.inl ⟨by rw [Set.univ_inter], hsU'⟩)
    · exact Or.inr (Or.inr (Or.inl ⟨X', hX', by rw [Set.univ_inter], hsF'⟩))
    · exact Or.inr (Or.inr (Or.inr ⟨Y', hY', by rw [Set.univ_inter], hsT'⟩))
    · exact Or.inr (Or.inl ⟨by rw [Set.inter_univ], hsU⟩)
    · exact Or.inr (Or.inl ⟨by rw [Set.inter_self], hsU⟩)
    · exfalso
      have hs := s.inter_mem hsU hsF'; rw [j0_inter_j1] at hs
      obtain ⟨t, ht⟩ := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    · exfalso
      have hs := s.inter_mem hsU hsT'; rw [j0_inter_j2] at hs
      obtain ⟨t, ht⟩ := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    · exact Or.inr (Or.inr (Or.inl ⟨X, hX, by rw [Set.inter_univ], hsF⟩))
    · exfalso
      have hs := s.inter_mem hsF hsU'; rw [Set.inter_comm, j0_inter_j1] at hs
      obtain ⟨t, ht⟩ := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    · refine Or.inr (Or.inr (Or.inl ⟨X ∩ X', ?_, embBit_inter false X X', ?_⟩))
      · have hs := s.inter_mem hsF hsF'; rw [j1_inter_j1] at hs
        exact sum3_mem_j1_inv (s.sub hs)
      · have hs := s.inter_mem hsF hsF'; rw [j1_inter_j1] at hs; exact hs
    · exfalso
      have hs := s.inter_mem hsF hsT'; rw [j1_inter_j2] at hs
      obtain ⟨t, ht⟩ := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    · exact Or.inr (Or.inr (Or.inr ⟨Y, hY, by rw [Set.inter_univ], hsT⟩))
    · exfalso
      have hs := s.inter_mem hsT hsU'; rw [Set.inter_comm, j0_inter_j2] at hs
      obtain ⟨t, ht⟩ := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    · exfalso
      have hs := s.inter_mem hsT hsF'; rw [Set.inter_comm, j1_inter_j2] at hs
      obtain ⟨t, ht⟩ := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    · refine Or.inr (Or.inr (Or.inr ⟨Y ∩ Y', ?_, embBit_inter true Y Y', ?_⟩))
      · have hs := s.inter_mem hsT hsT'; rw [j2_inter_j2] at hs
        exact sum3_mem_j2_inv (s.sub hs)
      · have hs := s.inter_mem hsT hsT'; rw [j2_inter_j2] at hs; exact hs
  up_mem := by
    rintro W W' (rfl | ⟨rfl, hsU⟩ | ⟨X, hX, rfl, hsF⟩ | ⟨Y, hY, rfl, hsT⟩) hW' hsub
    · exact Or.inl (Set.univ_subset_iff.mp hsub)
    · rcases memC_cases hW' with rfl | rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨rfl, hsU⟩)
```

```
      · exact absurd (hsub (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embBit
      · exact absurd (hsub (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embBit
    · rcases memC_cases hW' with rfl | rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · exact Or.inl rfl
      · exfalso
        obtain ⟨a, ha⟩ := C_nonempty X hX
        have hm := hsub (⟨a, rfl, ha⟩ : (false :: a) ∈ embBit false X)
        rw [Set.mem_singleton_iff] at hm; exact absurd hm (by simp)
      · refine Or.inr (Or.inr (Or.inl ⟨X', hX', rfl, ?_⟩))
        exact s.up_mem hsF (Or.inr (Or.inr (Or.inl ⟨X', hX', rfl⟩)))
          (j1_subset_j1.mpr (embBit_subset.mp hsub))
      · exfalso
        obtain ⟨a, ha⟩ := C_nonempty X hX
        obtain ⟨w', he, -⟩ := hsub (⟨a, rfl, ha⟩ : (false :: a) ∈ embBit false X)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
    · rcases memC_cases hW' with rfl | rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · exact Or.inl rfl
      · exfalso
        obtain ⟨a, ha⟩ := C_nonempty Y hY
        have hm := hsub (⟨a, rfl, ha⟩ : (true :: a) ∈ embBit true Y)
        rw [Set.mem_singleton_iff] at hm; exact absurd hm (by simp)
      · exfalso
        obtain ⟨a, ha⟩ := C_nonempty Y hY
        obtain ⟨w', he, -⟩ := hsub (⟨a, rfl, ha⟩ : (true :: a) ∈ embBit true Y)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
      · refine Or.inr (Or.inr (Or.inr ⟨Y', hY', rfl, ?_⟩))
        exact s.up_mem hsT (Or.inr (Or.inr (Or.inr ⟨Y', hY', rfl⟩)))
          (j2_subset_j2.mpr (embBit_subset.mp hsub))

@[simp] theorem fromCC_mem_nil {s : CC.Element} :
    (fromCC s).mem ({[]} : Set Str) ↔ s.mem (j0 (Set.univ : Set Unit)) := by
  constructor
  · rintro (h0 | ⟨-, hs⟩ | ⟨X', hX', heq, hs⟩ | ⟨Y', hY', heq, hs⟩)
    · exact absurd h0 singleton_nil_ne_univ
    · exact hs
    · exact absurd heq (singleton_nil_ne_embBit false X')
    · exact absurd heq (singleton_nil_ne_embBit true Y')
  · intro hs; exact Or.inr (Or.inl ⟨rfl, hs⟩)

@[simp] theorem fromCC_mem_embF {s : CC.Element} {X : Set Str} (hX : C.mem X) :
    (fromCC s).mem (embBit false X) ↔ s.mem (j1 X) := by
  constructor
  · rintro (h0 | ⟨heq, hs⟩ | ⟨X', hX', heqj, hs⟩ | ⟨Y', hY', heqj, hs⟩)
    · exact absurd h0 (embBit_ne_univ false X)
    · exact absurd heq.symm (singleton_nil_ne_embBit false X)
    · rw [embBit_injective heqj]; exact hs
    · exact absurd heqj (embBit_ne (show (false : Bool) = true by decide) (C_nonempty X
     hX))
  · intro hs; exact Or.inr (Or.inr (Or.inl ⟨X, hX, rfl, hs⟩))

@[simp] theorem fromCC_mem_embT {s : CC.Element} {Y : Set Str} (hY : C.mem Y) :
    (fromCC s).mem (embBit true Y) ↔ s.mem (j2 Y) := by
  constructor
  · rintro (h0 | ⟨heq, hs⟩ | ⟨X', hX', heqj, hs⟩ | ⟨Y', hY', heqj, hs⟩)
    · exact absurd h0 (embBit_ne_univ true Y)
    · exact absurd heq.symm (singleton_nil_ne_embBit true Y)
```

```
    · exact absurd heqj.symm (embBit_ne (show (false : Bool) = true by decide) (
    C_nonempty X' hX'))
    · rw [embBit_injective heqj]; exact hs
  · intro hs; exact Or.inr (Or.inr (Or.inr ⟨Y, hY, rfl, hs⟩))

theorem fromCC_toCC (x : C.Element) : fromCC (toCC x) = x := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨rfl, hs⟩ | ⟨X, hX, rfl, hs⟩ | ⟨Y, hY, rfl, hs⟩)
    · exact x.master_mem
    · exact toCC_mem_j0.mp hs
    · exact (toCC_mem_j1 hX).mp hs
    · exact (toCC_mem_j2 hY).mp hs
  · intro hW
    rcases memC_cases (x.sub hW) with rfl | rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨rfl, toCC_mem_j0.mpr hW⟩)
    · exact Or.inr (Or.inr (Or.inl ⟨X, hX, rfl, (toCC_mem_j1 hX).mpr hW⟩))
    · exact Or.inr (Or.inr (Or.inr ⟨Y, hY, rfl, (toCC_mem_j2 hY).mpr hW⟩))

theorem toCC_fromCC (s : CC.Element) : toCC (fromCC s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨rfl, hs⟩ | ⟨X, hX, rfl, hs⟩ | ⟨Y, hY, rfl, hs⟩)
    · exact s.master_mem
    · exact fromCC_mem_nil.mp hs
    · exact (fromCC_mem_embF hX).mp hs
    · exact (fromCC_mem_embT hY).mp hs
  · intro hW
    rcases s.sub hW with rfl | ⟨U, hU, rfl⟩ | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
    · exact Or.inl rfl
    · obtain rfl := hU
      exact Or.inr (Or.inl ⟨rfl, fromCC_mem_nil.mpr hW⟩)
    · exact Or.inr (Or.inr (Or.inl ⟨X, hX, rfl, (fromCC_mem_embF hX).mpr hW⟩))
    · exact Or.inr (Or.inr (Or.inr ⟨Y, hY, rfl, (fromCC_mem_embT hY).mpr hW⟩))

def ccEquiv : C.Element ≃o CC.Element where
  toFun := toCC
  invFun := fromCC
  left_inv := fromCC_toCC
  right_inv := toCC_fromCC
  map_rel_iff' := by
    intro x x'
    constructor
    · intro h W hW
      rcases memC_cases (x.sub hW) with rfl | rfl | ⟨A, hA, rfl⟩ | ⟨A, hA, rfl⟩
      · exact x'.master_mem
      · exact toCC_mem_j0.mp (h _ (Or.inr (Or.inl ⟨rfl, hW⟩)))
      · exact (toCC_mem_j1 hA).mp (h _ (Or.inr (Or.inr (Or.inl ⟨A, hA, rfl, hW⟩)))))
      · exact (toCC_mem_j2 hA).mp (h _ (Or.inr (Or.inr (Or.inr ⟨A, hA, rfl, hW⟩)))))
    · intro h W hW
      rcases hW with rfl | ⟨rfl, hz⟩ | ⟨X, hX, rfl, hz⟩ | ⟨Y, hY, rfl, hz⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨rfl, h _ hz⟩)
```

```
      · exact Or.inr (Or.inr (Or.inl ⟨X, hX, rfl, h _ hz⟩))
      · exact Or.inr (Or.inr (Or.inr ⟨Y, hY, rfl, h _ hz⟩))

theorem C_domain_equation :
    C ≅? sum3 unitSys C C unitSys_nonempty C_nonempty C_nonempty :=
  ⟨ccEquiv⟩

end SeqEqC

theorem j0_injective {X X' : Set α}
    (h : (j0 X : Set (Option (α ? β ? ?))) = j0 X') : X = X' :=
  Set.Subset.antisymm (j0_subset_j0.mp h.subset) (j0_subset_j0.mp h.symm.subset)

end DomainTheory.NeighbourhoodSystems
```

4.7.7 seqEqRegular — regular trees

Eventually periodic selector trees and regular languages.

Definition (`Tree`). As in the library.

Proof. By definition. ■

Definition (`pos`). As in the library.

Proof. By definition. ■

Definition (`select`). As in the library.

Proof. By definition. ■

Definition (`child`). As in the library.

Proof. By definition. ■

Definition (`treeLang`). As in the library.

Proof. By definition. ■

Definition (`EventuallyPeriodic`). As in the library.

Proof. By definition. ■

Theorem (`select_append`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`select_cons`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`treeLang_eq_pos`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`treeLang_injective`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`treeLang_select`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`eventuallyPeriodic_iff_isRegular`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

Theorem (`isRegular_iff_exists_eventuallyPeriodic`). As in the library.

Proof. Assemble the preceding lemmas on embeddings, neighbourhood shape, and the order-isomorphism. ■

`seqEqRegular.lean`

```
module

public import Mathlib.Computability.Language
public import Mathlib.Computability.MyhillNerode
public import Mathlib.Data.Set.Finite.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

namespace PeriodicTree
open Language

variable {n : ℕ}

abbrev Tree (n : ℕ) : Type := List (Fin n) → Bool

def pos (a : Tree n) : Bool := a []

def select (a : Tree n) (σ : List (Fin n)) : Tree n := fun τ => a (σ ++ τ)

@[simp] theorem select_apply (a : Tree n) (σ τ : List (Fin n)) : select a σ τ = a (σ
    ++ τ) := rfl

def child (a : Tree n) (i : Fin n) : Tree n := select a [i]

@[simp] theorem select_nil (a : Tree n) : select a [] = a := by
  funext τ; simp

@[simp] theorem pos_select (a : Tree n) (σ : List (Fin n)) : pos (select a σ) = a σ :=
     by
  simp [pos]

def treeLang (a : Tree n) : Language (Fin n) := {σ | a σ = true}

@[simp] theorem mem_treeLang {a : Tree n} {σ : List (Fin n)} : σ ∈ treeLang a ↔ a σ =
    true := Iff.rfl

def EventuallyPeriodic (a : Tree n) : Prop := (Set.range (select a)).Finite

theorem select_append (a : Tree n) (σ τ : List (Fin n)) :
    select a (σ ++ τ) = select (select a σ) τ := by
  funext ?; simp [List.append_assoc]

theorem select_cons (a : Tree n) (i : Fin n) (σ : List (Fin n)) :
    select a (i :: σ) = select (child a i) σ := by
  funext τ; simp [child]

theorem treeLang_eq_pos (a : Tree n) : treeLang a = {σ | pos (select a σ) = true} :=
    by
```

```
  simp only [pos_select]; rfl

theorem treeLang_injective : Function.Injective (treeLang (n := n)) := by
  intro a a' h
  funext σ
  have hmem : (σ ∈ treeLang a) ↔ (σ ∈ treeLang a') := by rw [h]
  simp only [mem_treeLang] at hmem
  cases ha : a σ <;> cases ha' : a' σ <;> simp_all

theorem treeLang_select (a : Tree n) (σ : List (Fin n)) :
    treeLang (select a σ) = (treeLang a).leftQuotient σ := by
  ext τ
  simp only [mem_treeLang, select_apply, Language.mem_leftQuotient]

theorem eventuallyPeriodic_iff_isRegular (a : Tree n) :
    EventuallyPeriodic a ↔ (treeLang a).IsRegular := by
  have hcomp : (treeLang a).leftQuotient = treeLang ∘ select a :=
    funext fun σ => (treeLang_select a σ).symm
  rw [Language.isRegular_iff_finite_range_leftQuotient, hcomp, Set.range_comp]
  exact (Set.finite_image_iff (treeLang_injective.injOn)).symm

theorem isRegular_iff_exists_eventuallyPeriodic (L : Language (Fin n)) :
    L.IsRegular ↔ ∃ a : Tree n, EventuallyPeriodic a ∧ treeLang a = L := by
  classical
  constructor
  · intro hL
    refine ⟨fun σ => decide (σ ∈ L), ?_, ?_⟩
    · have hlang : treeLang (fun σ => decide (σ ∈ L)) = L := by
        ext σ; simp only [mem_treeLang, decide_eq_true_eq]
      rw [eventuallyPeriodic_iff_isRegular, hlang]; exact hL
    · ext σ; simp only [mem_treeLang, decide_eq_true_eq]
  · rintro ⟨a, hEP, rfl⟩
    exact (eventuallyPeriodic_iff_isRegular a).mp hEP

end PeriodicTree
end DomainTheory.NeighbourhoodSystems
```

4.7.8 Example 6.2 — sequence domain equations

EXAMPLE 6.2. Our staple examples B and C satisfy "domain equations" in the form of isomorphisms as we have previously seen. Indeed

$$B \cong B + B, \text{ and}$$

$$C \cong \{\{\Lambda\}\} + C + C,$$

where if we liked we could construct both systems over $\{0,1\}^*$ and have:

$$B = \{\{0,1\}^*\} \cup \{0X \mid X \in B\} \cup \{1X \mid X \in B\}, \text{ and}$$

$$C = \{\{0,1\}^*\} \cup \{\{\Lambda\}\} \cup \{0X \mid X \in C\} \cup \{1X \mid X \in C\}.$$

We leave to the exercises the explanations of what kinds of algebras B and C are and why they are initial. Here we want to propose a simple, yet interesting generalization of B.

Consider this domain equation:

$$A \cong A^n + A^n,$$

where A^n stands for the n-fold cartesian power of A. We can, with the aid of some encoding solve this equation as a neighbourhood system over $\{0,1\}^*$ as follows:

$$A = \{\{0,1\}^*\} \cup \bigcup_{i=0,1} \left\{ i \bigcup_{j<n} 1^j 0 X_j \mid X_j \in A \text{ for all } j < n \right\}.$$

For instance, if $n = 3$, then a typical neighbourhood in A is something like

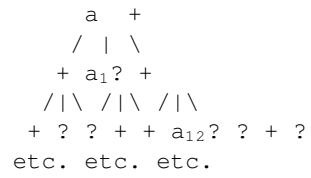
$$00X_0 \cup 010X_1 \cup 0110X_2,$$

where $X_0, X_1, X_2 \in A$. The first '0' could also be a '1' in front of each of the terms.

In words, an element of A (other than $\perp$) is an n-tuple of elements of A: but there are two separate copies of these, the left one and the right one. We can write for $a \in |A|$

$$a = \pm\langle a_0, a_1, \ldots, a_{n-1}\rangle,$$

where $+$ is chosen if a is on the right, and $-$ if on the left. As a tree diagram a might look like this for $n = 3$:



That is, a is an infinite ternary tree with $+$ or $-$ labels at each node. If each node (subtree) is truly infinite, the element is *total*; if $\perp$ is ever encountered, it is only *partial*; if every branch ends with $\perp$, the tree is a *finite element* of $|A|$.

What can be done with such trees? Let $\sigma \in \{0,1,\ldots,n-1\}^*$ be a finite sequence of "digits" each less than n. We let $\Sigma = \{0,1,\ldots,n-1\}$. We can define for $a \in |A|$ the operation $\sigma \mapsto a\sigma$ by recursion on σ:

$$a\Lambda = a, \text{ and}$$

$$ai\sigma = (a_i)\sigma.$$

The $a\sigma$ are just the *subtrees* of a with σ as a *selector*. We also have a map

$$\text{pos} : A \to T$$

where

$$\text{pos}(+\langle a_0, a_1, \ldots, a_{n-1}\rangle) = \text{true}, \text{ and}$$

$$\mathrm{pos}(-\langle a_0, a_1, \ldots, a_{n-1}\rangle) = \mathrm{false}.$$

We say that a (total) tree a is *eventually periodic* iff the set $\{a\sigma \mid \sigma \in \Sigma^*\}$ is finite. The result is that the "language"

$$L_a = \{\sigma \in \Sigma^* \mid \mathrm{pos}(a\sigma) = \mathrm{true}\}$$

corresponding to an eventually periodic tree is always a *regular event* of automata theory, and every such language has this form. In fact, a just represents the initial state of an automaton, and $a\sigma$ represents the state after "reading" a tape σ. □

Theorem (`example_62`). Scott's Example 6.2: the domains B and C satisfy their domain equations; for positive n the system A^n satisfies A^n ≅ A^n + A^n; and languages over Fin n are regular exactly when they arise from eventually periodic selector trees.

Proof. Combine B_domain_equation, C_domain_equation, A_domain_equation, and isRegular_iff_exists_eventuallyPeriodic from the companion libraries. ■

Example_6_2.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.seqEqB
public import DomainTheory.NeighbourhoodSystems.DomainEquations.seqEqA
public import DomainTheory.NeighbourhoodSystems.DomainEquations.seqEqC
public import DomainTheory.NeighbourhoodSystems.DomainEquations.seqEqRegular

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap BitDom SeqDom SeqEqB SeqEqA SeqEqC
    PeriodicTree

/-- Scott's Example 6.2: the staple sequence domains satisfy domain equations
`B ≅ B + B`, `C ≅ {{?}} + C + C`, and (for `n > 0`) `A^n ≅ A^n + A^n`; eventually
    periodic
trees yield regular languages. -/
theorem example_62 :
    (B ≅? sum B B B_nonempty B_nonempty) ∧
      (C ≅? sum3 unitSys C C unitSys_nonempty C_nonempty C_nonempty) ∧
      (∀ {n : ℕ} (hn : 0 < n), Asys n hn ≅? AAsys hn) ∧
      (∀ {n : ℕ} (L : Language (Fin n)), L.IsRegular ↔ ∃ a : Tree n,
    EventuallyPeriodic a ∧ treeLang a = L) :=
  ⟨B_domain_equation, C_domain_equation,
    fun hn => A_domain_equation hn,
    fun L => isRegular_iff_exists_eventuallyPeriodic L⟩
```

4.7.9 Endofunctor

Endofunctors and isomorphisms on the bespoke category of domains (Def 6.3 library).

Definition (`Endofunctor`). An endofunctor assigns to each object an object and to each morphism a morphism, such that identities and compositions are preserved.

Proof. By definition. ■

Definition (`Iso`). An isomorphism consists of mutually inverse morphisms between two objects.

Proof. By definition. ■

`endofunctor.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_28

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

universe u

variable {Obj : Type u} [Category Obj]

structure Endofunctor (Obj : Type u) [Category Obj] where
  obj : Obj → Obj
  map : {X Y : Obj} → (X ⟶ Y) → ((obj X) ⟶ (obj Y))
  map_id : ∀ (X : Obj), map ((? X)) = (? (obj X))
  map_comp : ∀ {X Y Z : Obj} (g : (Y ⟶ Z)) (f : (X ⟶ Y)),
    map (g ? f) = (map g) ? (map f)

structure Iso (X Y : Obj) where
  hom : (X ⟶ Y)
  inv : (Y ⟶ X)
  hom_inv_id : inv ? hom = (? X)
  inv_hom_id : hom ? inv = (? Y)

end DomainTheory.NeighbourhoodSystems
```

4.7.10 Definition 6.3 — functor

DEFINITION 6.3. A *functor* on a category (into itself) associates with every domain X in the category another domain $T(X)$ and to every map

$$f : X \to Y$$

another map

$$T(f) : T(X) \to T(Y)$$

in such a way that identity maps and compositions are preserved:

$$T(I_X) = I_{T(X)}, \text{ and}$$

$$T(g \circ f) = T(g) \circ T(f),$$

whenever $f : X \to Y$ and $g : Y \to Z$. □

In the example from 6.1 we have not checked how the special **T** is a functor. The hint is that whenever $f : X \to Y$, then there is a map

$$f \times f : X \times X \to Y \times Y.$$

But there is also a map

$$I_D + f \times f : D + (X \times X) \to D + (Y \times Y)$$

and this suggests the definition of $T(f)$. The details are left to the exercises. Note that the map $T(f)$ just suggested is always strict, so **T** is a functor also for the category of strict maps.

One good reason for a little of the category-theoretic language is that the next definition becomes very neat indeed.

The endofunctor peer records the library names.

Theorem (`definition_63`). For each endofunctor, identity morphisms and composition are preserved under the object and morphism actions.

Proof. The packaged equalities are exactly the endofunctor axioms. ■

`Definition_6_3.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.endofunctor

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

/-- Scott's Definition 6.3: an endofunctor assigns `T(X)` and `T(f)`, preserving
    identity
and composition. -/
theorem definition_63 {Obj : Type u} [Category Obj] (T : Endofunctor Obj) :
    (∀ X : Obj, T.map (? X) = ? (T.obj X)) ∧
      (∀ {X Y Z : Obj} (f : X ⟶ Y) (g : Y ⟶ Z), T.map (g ? f) = T.map g ? T.map f)
    :=
  ⟨fun X => T.map_id X, @fun _ _ _ f g => T.map_comp (g := g) (f := f)⟩

end DomainTheory.NeighbourhoodSystems
```

4.7.11 T-algebra

T-algebras and homomorphisms (Def 6.4 library).

Definition (`TAlgebra`). A T-algebra is an object together with a structure map from the image under the functor to the object.

Proof. By definition. ■

Definition (`AlgHom`). A homomorphism of T-algebras is a morphism between carriers making the structure square commute.

Proof. By definition. ■

`talgebra.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.endofunctor
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

universe u

variable {Obj : Type u} [Category Obj]

variable {T : Endofunctor Obj}

structure TAlgebra (T : Endofunctor Obj) where
  carrier : Obj
  str : ((T.obj carrier) ⟶ carrier)

structure AlgHom (A B : TAlgebra T) where
  hom : (A.carrier ⟶ B.carrier)
  comm : hom ? A.str = B.str ? T.map hom

end DomainTheory.NeighbourhoodSystems
```

4.7.12 Definition 6.4 — algebra for a functor

DEFINITION 6.4. A ***T*-algebra** is a domain E in the category together with a map

$$k : T(E) \to E.$$

If $m : T(F) \to F$ is another ***T*-algebra**, then a *homomorphism* is a map $h : E \to F$ in the category such that the diagram

$$\begin{array}{ccc} T(E) & \xrightarrow{k} & E \\ \downarrow{\scriptstyle T(h)} & & \downarrow{\scriptstyle h} \\ T(F) & \xrightarrow{m} & F \end{array}$$

commutes; that is, the equation

$$h \circ k = m \circ T(h)$$

holds. □

In our example from 6.1 a T-algebra is a *strict* map

$$k : D + (E \times E) \to E.$$

But such strict maps are in a one-one correspondence with pairs of (not necessarily strict) maps

$$n : D \to E \quad \text{and} \quad p : E \times E \to E.$$

And the structure $\langle E, n, p \rangle$ is what we called a tree algebra. Definition 6.4 just makes this abstract. The reader should also work out the details showing that 6.4's definition of homomorphism is just what we ought to expect.

Note that the T-algebras and homomorphisms form a category. (Why?) The following definition is so abstract that it could be given for any category.

The T-algebra peer records the library names.

Theorem (`definition_64`). Every algebra homomorphism satisfies the commuting-square equation.

Proof. This is the defining commutativity field of an algebra homomorphism. ■

`Definition_6_4.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.talgebra

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

universe u

variable {Obj : Type u} [Category Obj]

variable {T : Endofunctor Obj}

/-- Scott's Definition 6.4: a homomorphism of `T`-algebras satisfies
`h ∘ k = m ∘ T(h)`. -/
theorem definition_64 {A B : TAlgebra T} (h : AlgHom A B) :
    h.hom ? A.str = B.str ? T.map h.hom := h.comm

end DomainTheory.NeighbourhoodSystems
```

4.7.13 Initial algebra

Initial T-algebras, the category of homomorphisms, and the identity endofunctor (Def 6.5 library).

Definition (`IsInitial`). A T-algebra is initial when it has a chosen homomorphism into every algebra and that homomorphism is unique.

Proof. By definition. ■

Definition (`AlgHom.comp`). Composition of algebra homomorphisms is composition of the underlying morphisms, with the commutativity law inherited from functoriality and the squares for the factors.

Proof. Apply associativity, the commutative squares, and preservation of composition by the functor. ■

Definition (`AlgHom.id`). The identity morphism on the carrier is an algebra homomorphism.

Proof. Uses identity laws and the functor's preservation of identity. ■

Theorem (`AlgHom.id_hom`). The underlying morphism of the identity homomorphism is the identity on the carrier.

Proof. By definition. ■

Theorem (`AlgHom.comp_hom`). The underlying morphism of a composite homomorphism is the composite of the underlying morphisms.

Proof. By definition. ■

Definition (`idEndofunctor`). The identity endofunctor fixes objects and morphisms.

Proof. By definition. ■

Theorem (`idEndofunctor_obj`). The identity endofunctor leaves each object unchanged.

Proof. By definition. ■

Theorem (`idEndofunctor_map`). The identity endofunctor leaves each morphism unchanged.

Proof. By definition. ■

initialAlgebra.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.talgebra
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.DomainObj
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.smashProduct

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

universe u

variable {Obj : Type u} [Category Obj]

variable {T : Endofunctor Obj}

structure IsInitial (A : TAlgebra T) where
  desc : (B : TAlgebra T) → AlgHom A B
  uniq : ∀ (B : TAlgebra T) (h : AlgHom A B), h = desc B

namespace AlgHom

variable {T : Endofunctor Obj}

def comp {A B C : TAlgebra T} (β : AlgHom B C) (α : AlgHom A B) : AlgHom A C where
  hom := β.hom ? α.hom
  comm := by
    have hα := α.comm
    have hβ := β.comm
    have hmap := T.map_comp β.hom α.hom
    dsimp only [compAfter] at hα hβ hmap ⊢
    rw [← CategoryTheory.Category.assoc, hα, CategoryTheory.Category.assoc, hβ,
      ← CategoryTheory.Category.assoc, hmap]

def id (A : TAlgebra T) : AlgHom A A where
  hom := (? A.carrier)
  comm := by
    simp only [compAfter]
    rw [CategoryTheory.Category.comp_id, T.map_id, CategoryTheory.Category.id_comp]

@[simp] theorem id_hom (A : TAlgebra T) : (AlgHom.id A).hom = (? A.carrier) := rfl

@[simp] theorem comp_hom {A B C : TAlgebra T} (β : AlgHom B C) (α : AlgHom A B) :
    (β.comp α).hom = β.hom ? α.hom := rfl

end AlgHom

def idEndofunctor (Obj : Type u) [Category Obj] : Endofunctor Obj where
```

```
  obj X := X
  map f := f
  map_id _ := rfl
  map_comp _ _ := rfl

@[simp] theorem idEndofunctor_obj {Obj : Type u} [Category Obj] (X : Obj) :
    (idEndofunctor Obj).obj X = X := rfl

@[simp] theorem idEndofunctor_map {Obj : Type u} [Category Obj] {X Y : Obj}
    (f : (X ⟶ Y)) : (idEndofunctor Obj).map f = f := rfl

end DomainTheory.NeighbourhoodSystems
```

4.7.14 Definition 6.5 — initial algebra

DEFINITION 6.5. A T-algebra is *initial* if and only if there is a unique homomorphism from it into any other T-algebra. □

The word "other" here is not meant to imply "distinct". For an initial algebra there is one and only one homomorphism into itself: the identity map. As we already indicated in 6.1 it is a general fact that the next proposition holds.

The initial algebra peer records the library names and the homomorphism category.

Theorem (`definition_65`). An initial algebra has a homomorphism into every algebra, and any two such homomorphisms agree.

Proof. By the description and uniqueness fields packaged with initiality. ■

`Definition_6_5.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.initialAlgebra

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

/-- Scott's Definition 6.5: an initial `T`-algebra admits a unique homomorphism into
every other `T`-algebra. -/
theorem definition_65 {Obj : Type u} [Category Obj] {T : Endofunctor Obj} {A :
    TAlgebra T}
    (hA : IsInitial A) (B : TAlgebra T) :
    ∃ h : AlgHom A B, ∀ h' : AlgHom A B, h' = h :=
  ⟨hA.desc B, hA.uniq B⟩

end DomainTheory.NeighbourhoodSystems
```

4.7.15 Initial iso

Uniqueness and isomorphism of initial algebras (Proposition 6.6 library).

Theorem (`comp_desc_eq_id`). Composing the two canonical homomorphisms between initial algebras in opposite directions yields the identity.

Proof. Both composites and the identity are homomorphisms into an initial algebra; uniqueness forces agreement. ■

Definition (`initialIso`). For initial algebras, the forward and backward canonical homomorphisms assemble an isomorphism of carriers.

Proof. The inverse laws reduce to the round-trip identities from the previous lemma. ■

Theorem (`iso_hom_unique`). Every homomorphism from an initial algebra equals the canonical one.

Proof. By initiality. ■

initialIso.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_5

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

universe u

variable {Obj : Type u} [Category Obj] {T : Endofunctor Obj} {A B : TAlgebra T}

theorem comp_desc_eq_id (hA : IsInitial A) (hB : IsInitial B) :
    (hB.desc A).comp (hA.desc B) = AlgHom.id A := by
  rw [hA.uniq A ((hB.desc A).comp (hA.desc B)), hA.uniq A (AlgHom.id A)]

def initialIso (hA : IsInitial A) (hB : IsInitial B) : Iso A.carrier B.carrier where
  hom := (hA.desc B).hom
  inv := (hB.desc A).hom
  hom_inv_id := by
    have h := comp_desc_eq_id hA hB
    have := congrArg AlgHom.hom h
    simpa using this
  inv_hom_id := by
    have h := comp_desc_eq_id hB hA
    have := congrArg AlgHom.hom h
    simpa using this

theorem iso_hom_unique (hA : IsInitial A) (h : AlgHom A B) : h = hA.desc B :=
  hA.uniq B h

end DomainTheory.NeighbourhoodSystems
```

4.7.16 Proposition 6.6 — uniqueness of initial algebras

PROPOSITION 6.6. Any two initial T-algebras are uniquely isomorphic. □

Slightly more interesting is the behaviour of T on initial algebras.

The initial iso peer records the library names for the isomorphism and uniqueness lemmas.

Theorem (`proposition_66`). Every homomorphism from an initial algebra agrees with the canonical homomorphism furnished by initiality.

Proof. Invoke uniqueness of homomorphisms from an initial algebra. ■

Proposition_6_6.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.initialIso

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

/-- Scott's Proposition 6.6: the homomorphism from an initial algebra is unique. -/
theorem proposition_66 :
    ∀ {Obj : Type*} [Category Obj] {T : Endofunctor Obj} {A B : TAlgebra T}
      (hA : IsInitial A) (h : AlgHom A B), h = hA.desc B :=
  @iso_hom_unique

end DomainTheory.NeighbourhoodSystems
```

4.7.17 Lambek

Lambek's lemma: the structure map of an initial algebra is an isomorphism (Proposition 6.7 library).

Definition (`tStr`). Applying the functor to an algebra yields the algebra with carrier and structure map given by functorial action.

Proof. By definition. ■

Definition (`strHom`). The structure map is a homomorphism from the algebra obtained by functorial rearrangement.

Proof. The commutativity square holds by reflexivity. ■

Theorem (`str_comp_desc`). Composing the structure map with the canonical descent homomorphism yields the identity on the carrier.

Proof. Both sides are homomorphisms into an initial algebra; uniqueness identifies them. ■

Definition (`lambek`). The structure map and its descent inverse form an isomorphism between the functor image of the carrier and the carrier itself.

Proof. One inverse law is the preceding theorem; the other uses the homomorphism square and functoriality. ■

lambek.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_5

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

universe u

variable {Obj : Type u} [Category Obj] {T : Endofunctor Obj}

abbrev tStr (A : TAlgebra T) : TAlgebra T where
  carrier := T.obj A.carrier
  str := T.map A.str
```

```
def strHom (A : TAlgebra T) : AlgHom (tStr A) A where
  hom := A.str
  comm := rfl

theorem str_comp_desc (A : TAlgebra T) (hA : IsInitial A) :
    A.str ? (hA.desc (tStr A)).hom = (AlgHom.id A).hom := by
  have h : (strHom A).comp (hA.desc (tStr A)) = AlgHom.id A := by
    rw [hA.uniq A ((strHom A).comp (hA.desc (tStr A))), hA.uniq A (AlgHom.id A)]
  exact congrArg AlgHom.hom h

def lambek (A : TAlgebra T) (hA : IsInitial A) : Iso (T.obj A.carrier) A.carrier where
  hom := A.str
  inv := (hA.desc (tStr A)).hom
  inv_hom_id := str_comp_desc A hA
  hom_inv_id :=
    calc (hA.desc (tStr A)).hom ? A.str
        = (tStr A).str ? T.map (hA.desc (tStr A)).hom := (hA.desc (tStr A)).comm
      _ = T.map A.str ? T.map (hA.desc (tStr A)).hom := rfl
      _ = T.map (A.str ? (hA.desc (tStr A)).hom) := (T.map_comp _ _).symm
      _ = T.map (AlgHom.id A).hom := by rw [str_comp_desc A hA]
      _ = T.map (? A.carrier) := rfl
      _ = (? (T.obj A.carrier)) := T.map_id _

end DomainTheory.NeighbourhoodSystems
```

4.7.18 Proposition 6.7 — functor preserves initiality

PROPOSITION 6.7. If $i : T(D) \to D$ is an initial T-algebra, then so is $T(i) : T^2(D) \to T(D)$ and i is the isomorphism from $T(D)$ to D.

Proof: Clearly since T is a functor, the map $T(i)$ has the right mapping character to make $T(D)$ a T-algebra. Since D is initial, we have a commuting diagram:

$$\begin{array}{ccc} T(D) & \xrightarrow{i} & D \\ \downarrow{\scriptstyle T(j)} & & \downarrow{\scriptstyle j} \\ T^2(D) & \xrightarrow{T(i)} & T(D) \end{array}$$

But we also have the trivial diagram:

$$\begin{array}{ccc} T^2(D) & \xrightarrow{T(i)} & T(D) \\ \downarrow{\scriptstyle T(i)} & & \downarrow{\scriptstyle i} \\ T(D) & \xrightarrow{i} & D \end{array}$$

It follows that $i \circ j$ is a homomorphism, so

$$i \circ j = I_D.$$

But then because T is a functor we find:

$$T(i) \circ T(j) = I_{T(D)},$$

and, since j is a homomorphism, we have

$$j \circ i = I_{T(D)}.$$

This shows that i is an isomorphism. □

From 6.7 we see that if we are going to have initial algebras at all we have to satisfy the domain equation

$$D \cong T(D).$$

But generally that is not enough to assure that D is initial. There is a condition that our functors satisfy, however, which guarantees the existence of homomorphisms.

The Lambek peer records the library names for the isomorphism and its auxiliary maps.

Theorem (`proposition_67`). For an initial algebra, the structure map followed by the canonical descent homomorphism is the identity on the carrier.

Proof. Apply the round-trip identity packaged with the peer library. ■

`Proposition_6_7.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.lambek

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

universe u

variable {Obj : Type u} [Category Obj] {T : Endofunctor Obj}

/-- Scott's Proposition 6.7 (Lambek): the structure map of an initial algebra is an
    isomorphism. -/
theorem proposition_67 (A : TAlgebra T) (hA : IsInitial A) :
    A.str ? (hA.desc (tStr A)).hom = (AlgHom.id A).hom :=
  str_comp_desc A hA

end DomainTheory.NeighbourhoodSystems
```

4.7.19 Continuous on maps

Definition 6.8 library: functors continuous on strict function spaces.

Definition (`ContinuousOnMaps`). For every pair of domains, the induced map on strict approximable functions is itself approximable, with action agreeing with the functor on underlying maps.

Proof. By definition. ■

Theorem (`ContinuousOnMaps.isStrict_map`). A functor continuous on maps sends strict maps to strict maps.

Proof. From the witnessing equation, whose value is a strict map. ■

Theorem (`continuousOnMaps_id`). The identity functor is continuous on maps.

Proof. The identity on the strict function space witnesses continuity. ■

`continuousOnMaps.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.smashProduct
public import DomainTheory.NeighbourhoodSystems.DomainEquations.initialAlgebra
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.DomainObj

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

def ContinuousOnMaps (T : Endofunctor DomainObj) : Prop :=
  ∀ D E : DomainObj,
    ∃ ? : ApproximableMap (strictFun D.sys E.sys) (strictFun (T.obj D).sys (T.obj E).
    sys),
      ∀ f : StrictMap D.sys E.sys,
        (toStrictMap (?.toElementMap (toStrictFilter f))).1 = T.map (X := D) (Y := E)
    f.1

theorem ContinuousOnMaps.isStrict_map {T : Endofunctor DomainObj} (h :
    ContinuousOnMaps T)
    {D E : DomainObj} (f : StrictMap D.sys E.sys) :
    IsStrict (T.map (X := D) (Y := E) f.1) := by
  obtain ⟨?, h?⟩ := h D E
  rw [← h? f]
  exact (toStrictMap (?.toElementMap (toStrictFilter f))).2

theorem toStrictMap_toStrictFilter {α β : Type*} {V₀ : NeighborhoodSystem α}
    {V₁ : NeighborhoodSystem β} (f : StrictMap V₀ V₁) :
    toStrictMap (toStrictFilter f) = f :=
  (strictFunEquiv V₀ V₁).right_inv f

theorem continuousOnMaps_id : ContinuousOnMaps (idEndofunctor DomainObj) := by
  intro D E
  refine ⟨idMap (strictFun D.sys E.sys), fun f => ?_⟩
  show (toStrictMap ((idMap (strictFun D.sys E.sys)).toElementMap (toStrictFilter f)))
    .1 = f.1
  rw [toElementMap_idMap, toStrictMap_toStrictFilter]

end DomainTheory.NeighbourhoodSystems
```

4.7.20 Definition 6.8 — continuity on maps

DEFINITION 6.8. On the category of domains and strict approximable maps a functor T is *continuous on maps* if for any systems D and E the induced mapping

$$\lambda f.\, T(f) : (D \to_\perp E) \to (T(D) \to_\perp T(E))$$

is approximable.

The continuous on maps peer records the library names.

Theorem (`definition_68`). A functor is continuous on maps exactly when, for all domains D and E, the induced map on strict approximable functions is approximable with action $T(f)$.

Proof. This is the definition packaged as a biconditional. ■

Definition_6_8.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.continuousOnMaps

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

/-- Scott's Definition 6.8: a functor is continuous on strict function spaces. -/
theorem definition_68 (T : Endofunctor DomainObj) :
    ContinuousOnMaps T ↔
      ∀ D E : DomainObj,
        ∃ ? : ApproximableMap (strictFun D.sys E.sys) (strictFun (T.obj D).sys (T.obj E
    ).sys),
          ∀ f : StrictMap D.sys E.sys,
            (toStrictMap (?.toElementMap (toStrictFilter f))).1 = T.map (X := D) (Y :=
     E) f.1 :=
  Iff.rfl

end DomainTheory.NeighbourhoodSystems
```

4.7.21 Initial homomorphism

Theorem 6.9 library: post/pre-composition on strict function spaces and existence of algebra homomorphisms.

Definition (`homOpComp`, `homOp`). For strict post- and pre-composition with domain maps, the operator $g \mapsto k \circ g \circ j$ is an approximable self-map of the strict function space.

Proof. Built by monotonic extension from its action on principal neighbourhoods. ■

Theorem (`homOp_apply_filter`). The approximable operator realizes strict post/pre-composition on filters of strict maps.

Proof. By comparison on single-step neighbourhoods through the strict function-space representation. ■

Theorem (`nonempty_algHom_of_continuousOnMaps`). If T is continuous on maps and $D \cong T(D)$, then every strict T-algebra admits a strict homomorphism from D.

Proof. The homomorphism equation is a fixed-point equation for an approximable operator; a least fixed point yields the desired homomorphism. ■

initialHom.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import DomainTheory.NeighbourhoodSystems.DomainEquations.continuousOnMaps
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

variable {α β ? : Type*}
  {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ : NeighborhoodSystem ?}

theorem comp_mono_gen {a a' : ApproximableMap V₁ V₂} {b b' : ApproximableMap V₀ V₁}
    (ha : a ≤ a') (hb : b ≤ b') : a.comp b ≤ a'.comp b' := by
  intro X Z h
  obtain ⟨Y, hXY, hYZ⟩ := h
  exact ⟨Y, hb _ _ hXY, ha _ _ hYZ⟩

theorem isStrict_comp {a : ApproximableMap V₁ V₂} {b : ApproximableMap V₀ V₁}
    (ha : IsStrict a) (hb : IsStrict b) : IsStrict (a.comp b) := by
  rw [isStrict_iff_apply_bot, toElementMap_comp, isStrict_iff_apply_bot.mp hb,
    isStrict_iff_apply_bot.mp ha]

theorem isStrict_of_comp_eq_id {a : ApproximableMap V₁ V₀} {b : ApproximableMap V₀ V₁}
    (h : a.comp b = idMap V₀) : IsStrict a := by
  rw [isStrict_iff_apply_bot]
  refine le_antisymm ?_ (V₀.bot_le _)
  calc a.toElementMap V₁.bot
      ≤ a.toElementMap (b.toElementMap V₀.bot) := toElementMap_mono a (V₁.bot_le _)
    _ = (a.comp b).toElementMap V₀.bot := (toElementMap_comp a b V₀.bot).symm
    _ = V₀.bot := by rw [h, toElementMap_idMap]

theorem toStrictFilter_mono {f f' : StrictMap V₀ V₁} (h : f ≤ f') :
    toStrictFilter f ≤ toStrictFilter f' := by
  intro W hW
  rw [mem_toStrictFilter] at hW ⊢
  exact ⟨hW.1, strictFun_mem_up_closed hW.1 hW.2 h⟩

theorem toStrictMap_mono {φ φ' : (strictFun V₀ V₁).Element} (h : φ ≤ φ') :
    toStrictMap φ ≤ toStrictMap φ' := by
  intro X Y hrel
  rw [toStrictMap_rel] at hrel ⊢
  exact h _ hrel

theorem toStrictFilter_toStrictMap (φ : (strictFun V₀ V₁).Element) :
    toStrictFilter (toStrictMap φ) = φ := by
  apply Element.ext
  intro W
  constructor
  · rintro ⟨hWmem, hfW⟩
    obtain ⟨⟨L, hL, rfl⟩, _⟩ := hWmem
    exact (mem_sstepFun_iff φ hL).mpr (fun p hp => hfW p hp)
  · intro hW
    refine ⟨φ.sub hW, ?_⟩
    obtain ⟨⟨L, hL, rfl⟩, _⟩ := φ.sub hW
    intro p hp
    exact (mem_sstepFun_iff φ hL).mp hW p hp

variable (T : Endofunctor DomainObj) (D E : DomainObj)
```

```
  (j : ApproximableMap D.sys (T.obj D).sys)
  (k : ApproximableMap (T.obj E).sys E.sys)
  (hj : IsStrict j) (hk : IsStrict k)

def homOpComp (g : StrictMap (T.obj D).sys (T.obj E).sys) : StrictMap D.sys E.sys :=
  ⟨k.comp (g.1.comp j), isStrict_comp hk (isStrict_comp g.2 hj)⟩

theorem homOpComp_mono {g g' : StrictMap (T.obj D).sys (T.obj E).sys} (hgg : g ≤ g') :
    homOpComp T D E j k hj hk g ≤ homOpComp T D E j k hj hk g' := by
  show k.comp (g.1.comp j) ≤ k.comp (g'.1.comp j)
  exact comp_mono_gen le_rfl (comp_mono_gen (Subtype.coe_le_coe.mpr hgg) le_rfl)

def homOp : ApproximableMap (strictFun (T.obj D).sys (T.obj E).sys) (strictFun D.sys E
    .sys) :=
  ofMono
    (fun N hN => toStrictFilter (homOpComp T D E j k hj hk
      (toStrictMap ((strictFun (T.obj D).sys (T.obj E).sys).principal hN))))
    (by
      intro N N' hN hN' hN'N
      apply toStrictFilter_mono
      apply homOpComp_mono
      apply toStrictMap_mono
      exact ((strictFun (T.obj D).sys (T.obj E).sys).principal_le_iff hN hN').mpr hN'N
    )

theorem homOp_rel {N : Set (StrictMap (T.obj D).sys (T.obj E).sys)}
    {M : Set (StrictMap D.sys E.sys)} :
    (homOp T D E j k hj hk).rel N M ↔
      ∃ hN : (strictFun (T.obj D).sys (T.obj E).sys).mem N,
        (toStrictFilter (homOpComp T D E j k hj hk
          (toStrictMap ((strictFun (T.obj D).sys (T.obj E).sys).principal hN)))).mem M
    :=
  Iff.rfl

theorem homOp_apply_filter (g : StrictMap (T.obj D).sys (T.obj E).sys) :
    (homOp T D E j k hj hk).toElementMap (toStrictFilter g)
      = toStrictFilter (homOpComp T D E j k hj hk g) := by
  have key : ∀ X Z,
      ((homOp T D E j k hj hk).toElementMap (toStrictFilter g)).mem (sstep X Z)
        ↔ (homOpComp T D E j k hj hk g).1.rel X Z := by
    intro X Z
    constructor
    · rintro ⟨N, hgN, hrel⟩
      rw [homOp_rel] at hrel
      obtain ⟨hN, hmem⟩ := hrel
      rw [mem_toStrictFilter] at hmem
      obtain ⟨_, hsstep⟩ := hmem
      rw [mem_sstep] at hsstep
      obtain ⟨Y2, ⟨Y1, hXY1, hY1Y2⟩, hY2Z⟩ := hsstep
      rw [toStrictMap_rel, mem_principal] at hY1Y2
      obtain ⟨_, hNsub⟩ := hY1Y2
      exact ⟨Y2, ⟨Y1, hXY1, hNsub (mem_toStrictFilter.mp hgN).2⟩, hY2Z⟩
    · intro hrel
      obtain ⟨Y2, ⟨Y1, hXY1, hgY1Y2⟩, hY2Z⟩ := hrel
      refine ⟨sstep Y1 Y2, ?_, ?_⟩
      · rw [mem_toStrictFilter]
```

```
        exact ⟨sstep_mem_of_mem (g := g) hgY1Y2, hgY1Y2⟩
      · rw [homOp_rel]
        refine ⟨sstep_mem_of_mem (g := g) hgY1Y2, ?_⟩
        rw [mem_toStrictFilter]
        refine ⟨sstep_mem_of_mem (g := homOpComp T D E j k hj hk g)
          ⟨Y2, ⟨Y1, hXY1, hgY1Y2⟩, hY2Z⟩, ?_⟩
        rw [mem_sstep]
        refine ⟨Y2, ⟨Y1, hXY1, ?_⟩, hY2Z⟩
        rw [toStrictMap_rel, mem_principal]
        exact ⟨sstep_mem_of_mem (g := g) hgY1Y2, subset_rfl⟩
  have hmap : toStrictMap ((homOp T D E j k hj hk).toElementMap (toStrictFilter g))
      = homOpComp T D E j k hj hk g := by
    apply Subtype.ext
    apply ApproximableMap.ext
    intro X Z
    rw [toStrictMap_rel]
    exact key X Z
  have hL := toStrictFilter_toStrictMap
    ((homOp T D E j k hj hk).toElementMap (toStrictFilter g))
  rw [hmap] at hL
  exact hL.symm

theorem nonempty_algHom_of_continuousOnMaps
    (T : Endofunctor DomainObj) (hT : ContinuousOnMaps T)
    {D : DomainObj} (iso : Iso (T.obj D) D)
    (B : TAlgebra T) (hk : IsStrict B.str) :
    Nonempty {g : AlgHom (⟨D, iso.hom⟩ : TAlgebra T) B // IsStrict g.hom} := by

  have hji : iso.inv.comp iso.hom = idMap (T.obj D).sys := iso.hom_inv_id
  have hj : IsStrict iso.inv := isStrict_of_comp_eq_id hji

  obtain ⟨?, h?⟩ := hT D B.carrier

  set Op := (homOp T D B.carrier iso.inv B.str hj hk).comp ? with hOp
  set x := Op.fixElement with hx
  set h := toStrictMap x with hh
  let Tg : StrictMap (T.obj D).sys (T.obj B.carrier).sys :=
    ⟨T.map (X := D) (Y := B.carrier) h.1, hT.isStrict_map (D := D) (E := B.carrier) h⟩
  have hfix : Op.toElementMap x = x := toElementMap_fixElement Op
  have hxh : toStrictFilter h = x := toStrictFilter_toStrictMap x

  have hTg : toStrictMap (?.toElementMap (toStrictFilter h)) = Tg := Subtype.ext (h? h
    )
  have hφ : ?.toElementMap (toStrictFilter h) = toStrictFilter Tg := by
    rw [← hTg]; exact (toStrictFilter_toStrictMap _).symm

  have hOpx : Op.toElementMap x
      = toStrictFilter (homOpComp T D B.carrier iso.inv B.str hj hk Tg) := by
    rw [hOp, toElementMap_comp, ← hxh, hφ, homOp_apply_filter]
  have hcompeq : toStrictFilter (homOpComp T D B.carrier iso.inv B.str hj hk Tg)
      = toStrictFilter h := by
    rw [← hOpx, hfix, hxh]
  have hh' : homOpComp T D B.carrier iso.inv B.str hj hk Tg = h := by
    have h2 := congrArg toStrictMap hcompeq
    rwa [toStrictMap_toStrictFilter, toStrictMap_toStrictFilter] at h2
```

```
  have hcore : B.str.comp ((T.map (X := D) (Y := B.carrier) h.1).comp iso.inv) = h.1
    :=
    congrArg Subtype.val hh'

  refine ⟨⟨{ hom := h.1, comm := ?_ }, h.2⟩⟩
  show h.1.comp iso.hom = B.str.comp (T.map (X := D) (Y := B.carrier) h.1)
  conv_lhs => rw [← hcore]
  rw [comp_assoc, comp_assoc, hji, comp_idMap]

end DomainTheory.NeighbourhoodSystems
```

4.7.22 Theorem 6.9 — homomorphism from initial algebra

THEOREM 6.9. If the functor T is continuous on maps and if $D \cong T(D)$, so in particular D is a T-algebra, then for any T-algebra $k : T(E) \to E$ there is a homomorphism $h : D \to E$.

Proof: Let $i : T(D) \to D$ make D a T-algebra, where $j : D \to T(D)$ is the inverse so that i is an isomorphism of domains. Suppose that $k : T(E) \to E$ is any T-algebra. A homomorphism $h : D \to E$ would satisfy

$$h \circ i = k \circ T(h).$$

Rewrite this equation as

$$h = k \circ T(h) \circ j.$$

In the domain of strict maps $(D \to E)$ this is a fixed-point equation for an approximable map

$$\lambda h.\, k \circ T(h) \circ j$$

by our assumption on T. Thus, the desired homomorphism exists. □

The initial homomorphism peer records the library development and existence proof.

Theorem (`theorem_69`). Under continuity on maps and $D \cong T(D)$, every strict T-algebra receives a strict homomorphism from D.

Proof. By the fixed-point construction in the initial-homomorphism library. ■

The final question we have to answer is why in our category the minimal D exist. The reason is that the functors T that we have in mind possess further continuity properties on domains. This is conveniently expressed in terms of a notion of "subdomain".

Theorem_6_9.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.initialHom

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open DomainTheory.NeighbourhoodSystems.SmashProduct

/-- Scott's Theorem 6.9: continuous functors yield a strict algebra homomorphism. -/
```

```
theorem theorem_69 :
    ∀ (T : Endofunctor DomainObj), ContinuousOnMaps T →
      ∀ {D : DomainObj} (iso : Iso (T.obj D) D) (B : TAlgebra T),
        IsStrict B.str →
        Nonempty {g : AlgHom (⟨D, iso.hom⟩ : TAlgebra T) B // IsStrict g.hom} :=
  @nonempty_algHom_of_continuousOnMaps

end DomainTheory.NeighbourhoodSystems
```

4.7.23 Subdomain

Definition 6.10 library: the subdomain relation $\triangleleft$ on neighbourhood systems over a common token set.

Definition (`Subsystem`). $D \triangleleft E$ means D and E share the same master set, every neighbourhood of D belongs to E, and whenever $X, Y \in D$ with $X \cap Y \in E$, then $X \cap Y \in D$.

Proof. This is the intended mathematical content of Scott's clause (ii) together with $D \subseteq E$. ■

Definition (`?`). Infix notation for the subdomain proposition `Subsystem D E`.

Proof. Notation only. ■

Theorem (`NeighborhoodSystem.ext`). Two systems agreeing on membership and master are equal.

Proof. Extensionality on the membership field and the master token. ■

Theorem (`Subsystem.antisymm`). Mutual subdomain forces equality of systems.

Proof. Compare memberships via each inclusion map and identify masters. ■

Theorem (`Subsystem.refl`). Every system is a subdomain of itself.

Proof. Reflexivity of inclusion and intersection closure. ■

Theorem (`Subsystem.trans`). Subdomain is transitive.

Proof. Chain master equalities and neighbourhood inclusions; intersection closure passes along the chain. ■

Theorem (`Subsystem.subsystem_iff_subset_of_common`). For two subdomains of a common superdomain, subdomain order coincides with neighbourhood inclusion.

Proof. One direction is the subdomain map; the converse builds the intersection-closure clause from shared context. ■

`subdomain.lean`

```
module

public import Mathlib.CategoryTheory.Category.Basic

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.DomainObj
public import Mathlib.Tactic.ToAdditive

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
export CategoryTheory (Category)

open scoped CategoryTheory
```

```
scoped notation "?" => CategoryTheory.CategoryStruct.id

variable {α : Type*}

structure Subsystem (D E : NeighborhoodSystem α) : Prop where
  master_eq : D.master = E.master
  sub : ∀ {X : Set α}, D.mem X → E.mem X
  inter_closed : ∀ {X Y : Set α}, D.mem X → D.mem Y → E.mem (X ∩ Y) → D.mem (X ∩ Y)

@[inherit_doc] infix:50 " ? " => Subsystem

namespace Subsystem

theorem _root_.DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.ext {D E :
    NeighborhoodSystem α}
    (hmem : ∀ X, D.mem X ↔ E.mem X) (hmaster : D.master = E.master) : D = E := by
  rcases D with ⟨Dmem, Dmaster, _, _, _⟩
  rcases E with ⟨Emem, Emaster, _, _, _⟩
  have hm : Dmem = Emem := funext fun X => propext (hmem X)
  subst hm
  subst hmaster
  rfl

theorem antisymm {D E : NeighborhoodSystem α} (h₁ : D ? E) (h₂ : E ? D) : D = E :=
  NeighborhoodSystem.ext (fun _ => ⟨fun h => h₁.sub h, fun h => h₂.sub h⟩) h₁.master_eq

theorem refl (D : NeighborhoodSystem α) : D ? D where
  master_eq := rfl
  sub h := h
  inter_closed _ _ h := h

theorem trans {D E F : NeighborhoodSystem α} (h₁ : D ? E) (h₂ : E ? F) : D ? F where
  master_eq := h₁.master_eq.trans h₂.master_eq
  sub h := h₂.sub (h₁.sub h)
  inter_closed hX hY hXY :=
    h₁.inter_closed hX hY (h₂.inter_closed (h₁.sub hX) (h₁.sub hY) hXY)

theorem subsystem_iff_subset_of_common {D₀ D₁ E : NeighborhoodSystem α}
    (h₀ : D₀ ? E) (h₁ : D₁ ? E) :
    D₀ ? D₁ ↔ ∀ {X : Set α}, D₀.mem X → D₁.mem X := by
  constructor
  · intro h _ hX; exact h.sub hX
  · intro hsub
    refine ⟨h₀.master_eq.trans h₁.master_eq.symm, hsub, ?_⟩
    intro X Y hX hY hXY
    exact h₀.inter_closed hX hY (h₁.sub hXY)

end Subsystem

end DomainTheory.NeighbourhoodSystems
```

4.7.24 Definition 6.10 — subdomain

DEFINITION 6.10. For two neighbourhood systems D and E we write

$$D \triangleleft E$$

to mean that these are neighbourhood systems over the same set of tokens Δ and not only is $D \subseteq E$ but whenever $X, Y \in D$ and $X \cap Y \in E$, then $X \cap Y \in D$. $\square$

For the subdomain relation $D \triangleleft E$ to hold, D has to be a smaller family of neighbourhoods, but the notion of consistency in D also has to be the same as in E. Note that if $D_0 \triangleleft E$ and $D_1 \triangleleft E$ then

$$D_0 \triangleleft D_1 \text{ iff } D_0 \subseteq D_1.$$

It is also easy to prove that the union of a directed family of subdomains of E is again a subdomain. As a consequence of this remark we have:

The subdomain peer records the library names.

Theorem (`definition_610`). The subdomain relation is equivalent to equality of masters, inclusion of neighbourhood families, and inheritance of consistency for intersections admitted by the superdomain.

Proof. Unfold the subdomain structure in both directions. ■

`Definition_6_10.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.subdomain

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem Set

variable {α : Type*}

/-- Scott's Definition 6.10: `D ? E` -- same tokens, fewer neighbourhoods, inherited
    consistency. -/
theorem definition_610 {D E : NeighborhoodSystem α} :
    D ? E ↔
      D.master = E.master ∧
        (∀ {X}, D.mem X → E.mem X) ∧
          (∀ {X Y}, D.mem X → D.mem Y → E.mem (X ∩ Y) → D.mem (X ∩ Y)) :=
  ⟨fun h => ⟨h.master_eq, h.sub, h.inter_closed⟩,
    fun ⟨hm, hsub, hinter⟩ => ⟨hm, hsub, hinter⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.7.25 Subsystem lattice

Proposition 6.11 library: subsystems of a fixed neighbourhood system form a domain.

Definition (`subFam`). Families of neighbourhoods arising as $\{X \mid D.\mathrm{mem}\, X\}$ for some $D \triangleleft E$.

Proof. Definition from the subdomain/data pairing. ■

Theorem (`subFam_inter_closed`). Members of `subFam` are closed under intersections consistent in E.

Proof. Apply the subdomain intersection clause of the witness system. ■

Theorem (`subFam_master_mem`). Every member of `subFam` contains the master neighbourhood of E.

Proof. Master membership in the witness system, rewound along master equality. ■

Theorem (`subFam_mem_E`). Neighbourhoods in a member of `subFam` lie in E.

Proof. Subdomain inclusion from the witness. ■

Definition (`interSys`). System whose neighbourhoods lie in every family of a nonempty family from `subFam`.

Proof. Pointwise intersection membership defines a neighbourhood system over E's tokens. ■

Theorem (`interSys_nbset`). The neighbourhood family of `interSys` is the set-theoretic intersection of the family.

Proof. Characterization of membership in a global intersection. ■

Theorem (`interSys_subsystem`). `interSys` is a subdomain of E.

Proof. Master fixed to E; inclusion and intersection inheritance from `subFam` lemmas. ■

Definition (`ofMem`). Rebuild a neighbourhood system from a family in `subFam`.

Proof. Membership is literal membership in the family; axioms from `subFam` closure. ■

Theorem (`ofMem_subsystem`). `ofMem` recovers a subdomain of E.

Proof. Reflexivity of the data packaged with `subFam` membership. ■

Definition (`subPartialOrder`). Subsystems of E ordered by subdomain.

Proof. Reflexivity, transitivity, and antisymmetry from subdomain lemmas. ■

Theorem (`subFam_nonempty`). `subFam E` is nonempty.

Proof. The full family $\{X \mid E.\mathrm{mem}\, X\}$ arises from E itself. ■

Theorem (`subFam_sInter_mem`). Nonempty families in `subFam` have their intersection in `subFam`.

Proof. Realize the intersection as `interSys`. ■

Definition (`subIso`). Order isomorphism between subsystems of E and families in `subFam`.

Proof. Forward and backward maps are $\{X \mid D.\mathrm{mem}\, X\}$ and ofMem; order preserved by Scott's subset remark. ■

Definition (`unionSys`). System whose neighbourhoods appear in some member of a directed subfamily.

Proof. Existential membership over the directed family; intersection uses directedness. ■

Theorem (`unionSys_nbset`). Neighbourhood family equals the set-theoretic directed union.

Proof. Membership reformulation of $\bigcup$`0`. ■

Theorem (`unionSys_subsystem`). Directed unions of subdomains yield a subdomain.

Proof. Scott's remark realized via `unionSys`. ■

Theorem (`subFam_sUnion_mem`). Directed unions of families in `subFam` remain in `subFam`.

Proof. Witness `unionSys` and its subdomain law. ■

Definition (`subsystemReprIso`). Subsystems of E are order-isomorphic to the representable domain built from `subFam E`.

Proof. Compose `subIso` with the representation isomorphism of Exercise 2.22 using intersection and union closure. ■

`subsystemLattice.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.reprSystem
```

```
public import DomainTheory.NeighbourhoodSystems.DomainEquations.subdomain

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.SubsystemLattice
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Set DomainTheory.
    NeighbourhoodSystems.ReprSystem

variable {α : Type*}

def subFam (E : NeighborhoodSystem α) : Set (Set (Set α)) :=
  {? | ∃ D : NeighborhoodSystem α, D ? E ∧ ? = {X | D.mem X}}

theorem subFam_inter_closed (E : NeighborhoodSystem α) {? : Set (Set α)} {X Y : Set α}
    (h : ? ∈ subFam E) (hX : X ∈ ?) (hY : Y ∈ ?) (hXY : E.mem (X ∩ Y)) : X ∩ Y ∈ ? :=
   by
  obtain ⟨D, hD, rfl⟩ := h
  exact hD.inter_closed hX hY hXY

theorem subFam_master_mem (E : NeighborhoodSystem α) {? : Set (Set α)}
    (h : ? ∈ subFam E) : E.master ∈ ? := by
  obtain ⟨D, hD, rfl⟩ := h
  rw [← hD.master_eq]
  exact D.master_mem

theorem subFam_mem_E (E : NeighborhoodSystem α) {? : Set (Set α)} {X : Set α}
    (h : ? ∈ subFam E) (hX : X ∈ ?) : E.mem X := by
  obtain ⟨D, hD, rfl⟩ := h
  exact hD.sub hX

def interSys (E : NeighborhoodSystem α) (? : Set (Set (Set α)))
    (hne : ?.Nonempty) (h? : ? ⊆ subFam E) : NeighborhoodSystem α where
  mem X := ∀ ? ∈ ?, X ∈ ?
  master := E.master
  master_nonempty := E.master_nonempty
  master_mem := fun ? h? => subFam_master_mem E (h? h?)
  inter_mem := by
    intro X Y Z hX hY hZ hsub ? h?
    have hEX : E.mem X := subFam_mem_E E (h? h?) (hX ? h?)
    have hEY : E.mem Y := subFam_mem_E E (h? h?) (hY ? h?)
    have hEZ : E.mem Z := subFam_mem_E E (h? h?) (hZ ? h?)
    exact subFam_inter_closed E (h? h?) (hX ? h?) (hY ? h?) (E.inter_mem hEX hEY hEZ
    hsub)
  sub_master := by
    intro X hX
    obtain ⟨?, h?⟩ := hne
    exact E.sub_master (subFam_mem_E E (h? h?) (hX ? h?))

theorem interSys_nbset (E : NeighborhoodSystem α) (? : Set (Set (Set α)))
    (hne : ?.Nonempty) (h? : ? ⊆ subFam E) :
    {X | (interSys E ? hne h?).mem X} = ⋂₀ ? := by
  ext X
  exact Set.mem_sInter.symm

theorem interSys_subsystem (E : NeighborhoodSystem α) (? : Set (Set (Set α)))
    (hne : ?.Nonempty) (h? : ? ⊆ subFam E) : interSys E ? hne h? ? E where
```

```
  master_eq := rfl
  sub := by
    intro X hX
    obtain ⟨?, h?⟩ := hne
    exact subFam_mem_E E (h? h?) (hX ? h?)
  inter_closed := by
    intro X Y hX hY hEXY ? h?
    exact subFam_inter_closed E (h? h?) (hX ? h?) (hY ? h?) hEXY

def ofMem (E : NeighborhoodSystem α) (? : Set (Set α)) (h : ? ∈ subFam E) :
    NeighborhoodSystem α where
  mem X := X ∈ ?
  master := E.master
  master_nonempty := E.master_nonempty
  master_mem := subFam_master_mem E h
  inter_mem := by
    intro X Y Z hX hY hZ hsub
    exact subFam_inter_closed E h hX hY
      (E.inter_mem (subFam_mem_E E h hX) (subFam_mem_E E h hY) (subFam_mem_E E h hZ)
    hsub)
  sub_master := fun hX => E.sub_master (subFam_mem_E E h hX)

theorem ofMem_subsystem (E : NeighborhoodSystem α) (? : Set (Set α)) (h : ? ∈ subFam E
    ) :
    ofMem E ? h ? E where
  master_eq := rfl
  sub := subFam_mem_E E h
  inter_closed := subFam_inter_closed E h

instance subPartialOrder (E : NeighborhoodSystem α) :
    PartialOrder {D : NeighborhoodSystem α // D ? E} where
  le D₀ D₁ := D₀.1 ? D₁.1
  le_refl D := Subsystem.refl D.1
  le_trans _ _ _ h₁ h₂ := h₁.trans h₂
  le_antisymm _ _ h₁ h₂ := Subtype.ext (h₁.antisymm h₂)

theorem subFam_nonempty (E : NeighborhoodSystem α) : (subFam E).Nonempty :=
  ⟨{X | E.mem X}, E, Subsystem.refl E, rfl⟩

theorem subFam_sInter_mem (E : NeighborhoodSystem α) (? : Set (Set (Set α)))
    (hne : ?.Nonempty) (h? : ? ⊆ subFam E) : ⋂₀ ? ∈ subFam E :=
  ⟨interSys E ? hne h?, interSys_subsystem E ? hne h?, (interSys_nbset E ? hne h?).symm
    ⟩

def subIso (E : NeighborhoodSystem α) :
    {D : NeighborhoodSystem α // D ? E} ≃o {? : Set (Set α) // ? ∈ subFam E} where
  toFun D := ⟨{X | D.1.mem X}, ⟨D.1, D.2, rfl⟩⟩
  invFun ? := ⟨ofMem E ?.1 ?.2, ofMem_subsystem E ?.1 ?.2⟩
  left_inv := by
    intro D
    apply Subtype.ext
    apply NeighborhoodSystem.ext
    · intro X; exact Iff.rfl
    · exact D.2.master_eq.symm
  right_inv := by
    intro ?
```

```
    apply Subtype.ext
    ext X
    exact Iff.rfl
  map_rel_iff' := by
    intro a b
    show ({X | a.1.mem X} : Set (Set α)) ⊆ {X | b.1.mem X} ↔ a.1 ? b.1
    constructor
    · intro hsub
      refine (Subsystem.subsystem_iff_subset_of_common a.2 b.2).mpr ?_
      intro X hX
      exact hsub hX
    · intro hsub X hX
      exact hsub.sub hX

def unionSys (E : NeighborhoodSystem α) (? : Set (Set (Set α)))
    (hne : ?.Nonempty) (h? : ? ⊆ subFam E) (hdir : DirectedOn (· ⊆ ·) ?) :
    NeighborhoodSystem α where
  mem X := ∃ ? ∈ ?, X ∈ ?
  master := E.master
  master_nonempty := E.master_nonempty
  master_mem := by
    obtain ⟨?, h?⟩ := hne
    exact ⟨?, h?, subFam_master_mem E (h? h?)⟩
  inter_mem := by
    intro X Y Z hX hY hZ hsub
    obtain ⟨?x, h?x, hXx⟩ := hX
    obtain ⟨?y, h?y, hYy⟩ := hY
    obtain ⟨?z, h?z, hZz⟩ := hZ
    obtain ⟨?s, h?s, hxs, hys⟩ := hdir ?x h?x ?y h?y
    have hXs : X ∈ ?s := hxs hXx
    have hYs : Y ∈ ?s := hys hYy
    have hEX : E.mem X := subFam_mem_E E (h? h?s) hXs
    have hEY : E.mem Y := subFam_mem_E E (h? h?s) hYs
    have hEZ : E.mem Z := subFam_mem_E E (h? h?z) hZz
    exact ⟨?s, h?s, subFam_inter_closed E (h? h?s) hXs hYs (E.inter_mem hEX hEY hEZ
     hsub)⟩
  sub_master := by
    intro X hX
    obtain ⟨?, h?, hX?⟩ := hX
    exact E.sub_master (subFam_mem_E E (h? h?) hX?)

theorem unionSys_nbset (E : NeighborhoodSystem α) (? : Set (Set (Set α)))
    (hne : ?.Nonempty) (h? : ? ⊆ subFam E) (hdir : DirectedOn (· ⊆ ·) ?) :
    {X | (unionSys E ? hne h? hdir).mem X} = ⋃₀ ? := by
  ext X
  exact Set.mem_sUnion.symm

theorem unionSys_subsystem (E : NeighborhoodSystem α) (? : Set (Set (Set α)))
    (hne : ?.Nonempty) (h? : ? ⊆ subFam E) (hdir : DirectedOn (· ⊆ ·) ?) :
    unionSys E ? hne h? hdir ? E where
  master_eq := rfl
  sub := by
    intro X hX
    obtain ⟨?, h?, hX?⟩ := hX
    exact subFam_mem_E E (h? h?) hX?
  inter_closed := by
```

```
    intro X Y hX hY hEXY
    obtain ⟨?x, h?x, hXx⟩ := hX
    obtain ⟨?y, h?y, hYy⟩ := hY
    obtain ⟨?s, h?s, hxs, hys⟩ := hdir ?x h?x ?y h?y
    exact ⟨?s, h?s, subFam_inter_closed E (h? h?s) (hxs hXx) (hys hYy) hEXY⟩

theorem subFam_sUnion_mem (E : NeighborhoodSystem α) (? : Set (Set (Set α)))
    (hne : ?.Nonempty) (h? : ? ⊆ subFam E) (hdir : DirectedOn (· ⊆ ·) ?) :
    ⋃₀ ? ∈ subFam E :=
  ⟨unionSys E ? hne h? hdir, unionSys_subsystem E ? hne h? hdir,
    (unionSys_nbset E ? hne h? hdir).symm⟩

def subsystemReprIso (E : NeighborhoodSystem α) :
    {D : NeighborhoodSystem α // D ? E} ≃o
      (reprSystem (subFam E) (subFam_sInter_mem E) (subFam_nonempty E)).Element :=
  (subIso E).trans
    (reprIso (subFam E) (subFam_sInter_mem E) (subFam_nonempty E) (subFam_sUnion_mem E
    )).symm

end DomainTheory.NeighbourhoodSystems.SubsystemLattice
```

4.7.26 Proposition 6.11 — lattice of subsystems

PROPOSITION 6.11. For a given neighbourhood system E, the set of subsystems

$$\{D \mid D \triangleleft E\}$$

forms a domain in its own right. □

The subdomain relationship implies a mapping relationship between the domains.

The subsystem lattice peer records the library names.

Theorem (`proposition_611`). For each E, the subtype of subsystems ordered by $\triangleleft$ is nonempty-order-isomorphic to the domain element type given by the abstract representation of `subFam E`.

Proof. Exhibit `subsystemReprIso`. ■

`Proposition_6_11.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.subsystemLattice

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem SubsystemLattice ReprSystem

variable {α : Type*}

/-- Scott's Proposition 6.11: subsystems of `E` form a domain. -/
theorem proposition_611 (E : NeighborhoodSystem α) :
    Nonempty ({D : NeighborhoodSystem α // D ? E} ≃o
      (reprSystem (subFam E) (subFam_sInter_mem E) (subFam_nonempty E)).Element) :=
  ⟨subsystemReprIso E⟩
```

```
end DomainTheory.NeighbourhoodSystems
```

4.7.27 Projection pair library

Proposition 6.12 library: injection and projection maps for a subdomain.

Definition (`ProjectionPair`). A pair of approximable maps with $j \circ i = I_D$ and $i \circ j \subseteq I_E$.

Proof. Structure recording the laws. ■

Definition (`Subsystem.inj`). Relation $X\, i\, Y \leftrightarrow X \in D \wedge Y \in E \wedge X \subseteq Y$.

Proof. Standard approximable-map axioms; intersection uses subdomain inclusion into E. ■

Theorem (`Subsystem.inj_rel`). Unfolding lemma for the injection relation.

Proof. Definition. ■

Definition (`Subsystem.proj`). Relation $Y\, j\, X \leftrightarrow Y \in E \wedge X \in D \wedge Y \subseteq X$; intersection output uses subdomain consistency.

Proof. Approximable-map axioms with `inter_closed` on D. ■

Theorem (`Subsystem.proj_rel`). Unfolding lemma for the projection relation.

Proof. Definition. ■

Theorem (`Subsystem.inj_comp_proj_le`). $i \circ j \subseteq I_E$.

Proof. A round trip through a common D-neighbourhood refines $Y \subseteq Y'$ on E. ■

Theorem (`Subsystem.proj_comp_inj`). $j \circ i = I_D$.

Proof. Relational extensionality; round trips collapse to $\subseteq$ on D. ■

Definition (`Subsystem.projectionPair`). Bundle the maps and laws for $D \triangleleft E$.

Proof. Fields filled from `inj`, `proj`, and the two composition laws. ■

`projectionPair.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.subdomain
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.idMap
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.step

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

variable {α : Type*}

namespace Subsystem

variable {D E : NeighborhoodSystem α}

structure ProjectionPair (D E : NeighborhoodSystem α) where
  inj : ApproximableMap D E
  proj : ApproximableMap E D
  proj_comp_inj : proj.comp inj = idMap D
  inj_comp_proj_le : inj.comp proj ≤ idMap E

def inj (h : D ? E) : ApproximableMap D E where
  rel X Y := D.mem X ∧ E.mem Y ∧ X ⊆ Y
```

```
  rel_dom hr := hr.1
  rel_cod hr := hr.2.1
  master_rel := ⟨D.master_mem, E.master_mem, h.master_eq.subset⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY, hXY⟩ ⟨_, hY', hXY'⟩
    exact ⟨hX, E.inter_mem hY hY' (h.sub hX) (Set.subset_inter hXY hXY'),
      Set.subset_inter hXY hXY'⟩
  mono := by
    rintro X X' Y Y' ⟨_, _, hXY⟩ hX'X hYY' hX' hY'
    exact ⟨hX', hY', (hX'X.trans hXY).trans hYY'⟩

@[simp] theorem inj_rel (h : D ? E) {X Y : Set α} :
    (h.inj).rel X Y ↔ D.mem X ∧ E.mem Y ∧ X ⊆ Y := Iff.rfl

def proj (h : D ? E) : ApproximableMap E D where
  rel Y X := E.mem Y ∧ D.mem X ∧ Y ⊆ X
  rel_dom hr := hr.1
  rel_cod hr := hr.2.1
  master_rel := ⟨E.master_mem, D.master_mem, h.master_eq.symm.subset⟩
  inter_right := by
    rintro Y X X' ⟨hY, hX, hYX⟩ ⟨_, hX', hYX'⟩
    have hEinter : E.mem (X ∩ X') :=
      E.inter_mem (h.sub hX) (h.sub hX') hY (Set.subset_inter hYX hYX')
    exact ⟨hY, h.inter_closed hX hX' hEinter, Set.subset_inter hYX hYX'⟩
  mono := by
    rintro Y Y' X X' ⟨_, _, hYX⟩ hY'Y hXX' hY' hX'
    exact ⟨hY', hX', (hY'Y.trans hYX).trans hXX'⟩

@[simp] theorem proj_rel (h : D ? E) {Y X : Set α} :
    (h.proj).rel Y X ↔ E.mem Y ∧ D.mem X ∧ Y ⊆ X := Iff.rfl

theorem inj_comp_proj_le (h : D ? E) : h.inj.comp h.proj ≤ idMap E := by
  intro Y Y' hr
  obtain ⟨X, ⟨hEY, _, hYX⟩, _, hEY', hXY'⟩ := hr
  exact ⟨hEY, hEY', hYX.trans hXY'⟩

theorem proj_comp_inj (h : D ? E) : h.proj.comp h.inj = idMap D := by
  apply ApproximableMap.ext
  intro X Z
  rw [comp_rel, idMap_rel]
  constructor
  · rintro ⟨Y, ⟨hX, _, hXY⟩, _, hZ, hYZ⟩
    exact ⟨hX, hZ, hXY.trans hYZ⟩
  · rintro ⟨hX, hZ, hXZ⟩
    exact ⟨Z, ⟨hX, h.sub hZ, hXZ⟩, h.sub hZ, hZ, subset_rfl⟩

def projectionPair (h : D ? E) : ProjectionPair D E where
  inj := h.inj
  proj := h.proj
  proj_comp_inj := h.proj_comp_inj
  inj_comp_proj_le := h.inj_comp_proj_le

end Subsystem

end DomainTheory.NeighbourhoodSystems
```

4.7.28 Proposition 6.12 — projection pair

PROPOSITION 6.12. If $D \triangleleft E$, then there exists a projection pair of approximable mappings:

$$i : D \to E \quad \text{and} \quad j : E \to D$$

where $j \circ i = I_D$ and $i \circ j \subseteq I_E$, which are determined as element-wise functions by these equations:

$$i(x) = \{Y \in E \mid \exists X \in x.\, X \subseteq Y\},$$

and

$$j(y) = y \cap D,$$

for all $x \in |D|$ and $y \in |E|$. □

The proof is left for the exercises.

The projection pair library peer records the library names.

Theorem (`proposition_612`). For every subdomain $h : D \triangleleft E$, the standard injection and projection satisfy $i \circ j \subseteq I_E$ and $j \circ i = I_D$.

Proof. Both clauses are the peer composition laws. ■

Theorem (`Subsystem.trianglelefteq`). A subdomain yields the embedding $D \trianglelefteq E$ using D itself as witness.

Proof. Reflexivity of domain isomorphism on the subtype. ■

`Proposition_6_12.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.projectionPair
public import DomainTheory.NeighbourhoodSystems.DomainEquations.projPairSubdomain

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Subsystem

variable {α : Type*}

/-- Scott's Proposition 6.12: a subdomain yields injection and projection with the two
    laws. -/
theorem proposition_612 :
    (∀ {α : Type*} {D E : NeighborhoodSystem α} (h : D ? E),
      h.inj.comp h.proj ≤ idMap E) ∧
    (∀ {α : Type*} {D E : NeighborhoodSystem α} (h : D ? E),
      h.proj.comp h.inj = idMap D) :=
  ⟨@inj_comp_proj_le, @proj_comp_inj⟩

theorem Subsystem.trianglelefteq {D E : NeighborhoodSystem α} (h : D ? E) : D ? E :=
  ⟨D, h, Isomorphic.refl D⟩

end DomainTheory.NeighbourhoodSystems
```

4.7.29 Continuous on domains

continuousOnDomains.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_10
public import DomainTheory.NeighbourhoodSystems.DomainEquations.endofunctor
public import DomainTheory.NeighbourhoodSystems.DomainEquations.initialAlgebra
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.DomainObj

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

universe w

structure MonotoneAt (T : Endofunctor DomainObj.{w}) {α : Type w}
    {D E : NeighborhoodSystem α} (h : D ? E) : Prop where
  carrier_eq : (T.obj ⟨α, E⟩).carrier = (T.obj ⟨α, D⟩).carrier
  sub : (T.obj ⟨α, D⟩).sys ?
    (carrier_eq ▸ (T.obj ⟨α, E⟩).sys : NeighborhoodSystem (T.obj ⟨α, D⟩).carrier)
  inj_heq : HEq (T.map (X := ⟨α, D⟩) (Y := ⟨α, E⟩) h.inj) sub.inj
  proj_heq : HEq (T.map (X := ⟨α, E⟩) (Y := ⟨α, D⟩) h.proj) sub.proj

def MonotoneOnDomains (T : Endofunctor DomainObj.{w}) : Prop :=
  ∀ {α : Type w} {D E : NeighborhoodSystem α} (h : D ? E), MonotoneAt T h

def targetFam (T : Endofunctor DomainObj.{w}) (hmono : MonotoneOnDomains T)
    {α : Type w} {D E : NeighborhoodSystem α} (h : D ? E) :
    Set (Set (T.obj ⟨α, E⟩).carrier) :=
  {Y | (T.obj ⟨α, D⟩).sys.mem ((hmono h).carrier_eq ▸ Y)}

def ContinuousOnDomains (T : Endofunctor DomainObj.{w}) : Prop :=
  ∃ hmono : MonotoneOnDomains T,
    ∀ {α : Type w} {E : NeighborhoodSystem α}
      (? : Set (NeighborhoodSystem α)) (h? : ∀ {{D}}, D ∈ ? → D ? E)
      (_hne : ?.Nonempty) (_hdir : DirectedOn (· ? ·) ?)
      {U : NeighborhoodSystem α} (hUE : U ? E)
      (_hU : ∀ X, U.mem X ↔ ∃ D ∈ ?, D.mem X),
      targetFam T hmono hUE = ⋃ D, ⋃ hD : D ∈ ?, targetFam T hmono (h? hD)

theorem monotoneOnDomains_id : MonotoneOnDomains (idEndofunctor DomainObj.{w}) := by
  intro α D E h
  exact ⟨rfl, h, HEq.rfl, HEq.rfl⟩

theorem continuousOnDomains_id : ContinuousOnDomains (idEndofunctor DomainObj.{w}) :=
    by
  refine ⟨monotoneOnDomains_id, ?_⟩
  intro α E ? h? _hne _hdir U hUE hU
  apply Set.ext
  intro Y
  simp only [targetFam, Set.mem_iUnion, exists_prop]
  exact hU Y
```

```
end DomainTheory.NeighbourhoodSystems
```

4.7.30 Definition 6.13 — continuity on domains

```
##### Continuous on domains

**[continuousOnDomains.lean](DomainTheory/NeighbourhoodSystems/DomainEquations/
    continuousOnDomains.lean)**

@import "continuousOnDomains.lean"

<p style="color:#15803d;margin-top:1em;">? <strong>Revised</strong> (2026-11-07) -- <a
     href="../1980_REVISION.md">1980 revision progress</a></p>
```

Definition_6_13.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.continuousOnDomains

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

universe w

/-- Scott's Definition 6.13: monotone and continuous functors on subdomains (`
    MonotoneOnDomains`,
`ContinuousOnDomains`, and `targetFam`). -/
theorem definition_613 (T : Endofunctor DomainObj.{w}) :
    ContinuousOnDomains T ↔
      ∃ hmono : MonotoneOnDomains T,
        ∀ {α : Type w} {E : NeighborhoodSystem α}
          (? : Set (NeighborhoodSystem α)) (h? : ∀ {{D}}, D ∈ ? → D ? E)
          (_hne : ?.Nonempty) (_hdir : DirectedOn (· ? ·) ?)
          {U : NeighborhoodSystem α} (hUE : U ? E)
          (_hU : ∀ X, U.mem X ↔ ∃ D ∈ ?, D.mem X),
          targetFam T hmono hUE = ⋃ D, ⋃ hD : D ∈ ?, targetFam T hmono (h? hD) :=
  Iff.rfl

end DomainTheory.NeighbourhoodSystems
```

DEFINITION 6.13. A functor T is *monotone on domains* iff whenever $D \triangleleft E$, then not only do we have $T(D) \triangleleft T(E)$ but the projection pair i, j of 6.12 is mapped to the same kind of projection pair $T(i), T(j)$. A monotone functor is *continuous on domains* iff whenever E is a domain, then the mapping

$$\lambda D.\, T(D) : \{D \mid D \triangleleft E\} \to \{D' \mid D' \triangleleft T(E)\}$$

is approximable. □

We can now state an existence theorem that covers in fairly wide generality the examples of this lecture.

4.7.31 Exists initial

existsInitial.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.indefProduct
public import DomainTheory.NeighbourhoodSystems.DomainEquations.embeddingCompare
public import DomainTheory.NeighbourhoodSystems.DomainEquations.continuousOnMaps
public import DomainTheory.NeighbourhoodSystems.DomainEquations.initialHom
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

universe w

namespace Theorem614
variable {α : Type w}

theorem exists_algebra_with_hom (s : Setup.{w}) :
    ∃ A : TAlgebra s.T, Nonempty (Iso (s.T.obj A.carrier) A.carrier) ∧
      ∀ B : TAlgebra s.T, IsStrict B.str → Nonempty (AlgHom A B) :=
  ⟨colimAlg s, ⟨colimIso s⟩, fun B hk => nonempty_algHom s B hk⟩

theorem rho_zero_rel (s : Setup.{w}) (hΓ : ∀ X, s.Γ.mem X → X = s.Γ.master)
    {X Y : Set s.Tok} :
    (rho s 0).rel X Y ↔ (colim s).mem X ∧ Y = (colim s).master := by
  rw [rho_rel]
  constructor
  · rintro ⟨hcX, hcY, z, hz, _, hzY⟩
    have hzm : z = s.Γ.master := hΓ z hz
    subst hzm
    refine ⟨hcX, Set.Subset.antisymm ((colim s).sub_master hcY) ?_⟩
    rw [colim_master]; exact hzY
  · rintro ⟨hcX, rfl⟩
    refine ⟨hcX, (colim s).master_mem, s.Γ.master, s.Γ.master_mem, ?_, ?_⟩
    · have h := (colim s).sub_master hcX; rwa [colim_master] at h
    · rw [colim_master]

theorem strict_rel_master {β₀ β₁ : Type w} {V₀ : NeighborhoodSystem β₀}
    {V₁ : NeighborhoodSystem β₁} {g : ApproximableMap V₀ V₁} (hg : IsStrict g) {Z : Set
     β₁} :
    g.rel V₀.master Z ↔ Z = V₁.master :=
  ⟨fun h => hg h, fun h => h ▸ g.master_rel⟩

theorem gcomp_rho_zero_rel (s : Setup.{w}) (hΓ : ∀ X, s.Γ.mem X → X = s.Γ.master)
    (B : TAlgebra s.T) {g : ApproximableMap (colim s) B.carrier.sys} (hg : IsStrict g)
    {X : Set s.Tok} {Z : Set B.carrier.carrier} :
    (g.comp (rho s 0)).rel X Z ↔ (colim s).mem X ∧ Z = B.carrier.sys.master := by
  rw [comp_rel]
  constructor
```

```
  · rintro ⟨Y, hrho, hgYZ⟩
    rw [rho_zero_rel s hΓ] at hrho
    obtain ⟨hcX, rfl⟩ := hrho
    exact ⟨hcX, (strict_rel_master hg).mp hgYZ⟩
  · rintro ⟨hcX, rfl⟩
    exact ⟨(colim s).master, (rho_zero_rel s hΓ).mpr ⟨hcX, rfl⟩, g.master_rel⟩

theorem gcomp_rho_zero_indep (s : Setup.{w}) (hΓ : ∀ X, s.Γ.mem X → X = s.Γ.master)
    (B : TAlgebra s.T) {g g' : ApproximableMap (colim s) B.carrier.sys}
    (hg : IsStrict g) (hg' : IsStrict g') :
    g.comp (rho s 0) = g'.comp (rho s 0) := by
  apply ApproximableMap.ext
  intro X Z
  rw [gcomp_rho_zero_rel s hΓ B hg, gcomp_rho_zero_rel s hΓ B hg']

theorem transport_heq {Obj : Type*} [Category Obj] {X Y : Obj} (e : X = Y)
    (f : (X ⟶ X)) : HEq (e ▶ f : (Y ⟶ Y)) f := by
  cases e; rfl

theorem key_rho (s : Setup.{w}) (n : ℕ) :
    rho s (n + 1) = (colimIso s).hom ? Tmap_rho s n ? (colimIso s).inv := by
  rw [show (colimIso s).hom = (isoOfEq (colimObj_eq s)).hom from rfl,
      show (colimIso s).inv = (isoOfEq (colimObj_eq s)).inv from rfl,
      isoOfEq_conj (colimObj_eq s) (Tmap_rho s n)]
  apply eq_of_heq
  exact HEq.trans (map_rho_heq s n).symm
    (transport_heq (colimObj_eq s) (Tmap_rho s n)).symm

theorem gcomp_rho_succ (s : Setup.{w}) (B : TAlgebra s.T) (g : AlgHom (colimAlg s) B)
    (n : ℕ) :
    g.hom.comp (rho s (n + 1))
      = B.str.comp ((s.T.map (X := objColim s) (Y := B.carrier)
          (g.hom.comp (rho s n))).comp (colimIso s).inv) := by
  have hcomm : g.hom ? (colimIso s).hom
      = B.str ? s.T.map (X := objColim s) (Y := B.carrier) g.hom := g.comm
  show g.hom ? rho s (n + 1)
      = B.str ? (s.T.map (X := objColim s) (Y := B.carrier) (g.hom ? rho s n)) ? (
    colimIso s).inv
  calc g.hom ? rho s (n + 1)
      = g.hom ? ((colimIso s).hom ? (Tmap_rho s n ? (colimIso s).inv)) :=
          congrArg (fun x => g.hom ? x) (key_rho s n)
    _ = (g.hom ? (colimIso s).hom) ? (Tmap_rho s n ? (colimIso s).inv) :=
          CategoryTheory.Category.assoc (Tmap_rho s n ? (colimIso s).inv) (colimIso s)
    .hom g.hom
    _ = (B.str ? s.T.map (X := objColim s) (Y := B.carrier) g.hom)
          ? (Tmap_rho s n ? (colimIso s).inv) := by rw [hcomm]
    _ = B.str ? (s.T.map (X := objColim s) (Y := B.carrier) g.hom
          ? (Tmap_rho s n ? (colimIso s).inv)) :=
        (CategoryTheory.Category.assoc (Tmap_rho s n ? (colimIso s).inv)
          (s.T.map (X := objColim s) (Y := B.carrier) g.hom) B.str).symm
    _ = B.str ? ((s.T.map (X := objColim s) (Y := B.carrier) g.hom ? Tmap_rho s n)
          ? (colimIso s).inv) :=
        congrArg (B.str ? ·)
          (CategoryTheory.Category.assoc (colimIso s).inv (Tmap_rho s n)
            (s.T.map (X := objColim s) (Y := B.carrier) g.hom))
    _ = B.str ? (s.T.map (X := objColim s) (Y := B.carrier) (g.hom ? rho s n)
```

```
            ? (colimIso s).inv) :=
          congrArg (fun m => B.str ? (m ? (colimIso s).inv))
            (s.T.map_comp (X := objColim s) (Y := objColim s) (Z := B.carrier) g.hom
              (rho s n)).symm

theorem gcomp_rho_indep (s : Setup.{w}) (hΓ : ∀ X, s.Γ.mem X → X = s.Γ.master)
    (B : TAlgebra s.T) (g g' : AlgHom (colimAlg s) B)
    (hg : IsStrict g.hom) (hg' : IsStrict g'.hom) (n : ℕ) :
    g.hom.comp (rho s n) = g'.hom.comp (rho s n) := by
  induction n with
  | zero => exact gcomp_rho_zero_indep s hΓ B hg hg'
  | succ k ih => rw [gcomp_rho_succ s B g k, gcomp_rho_succ s B g' k, ih]

theorem gcomp_eq (s : Setup.{w}) (hΓ : ∀ X, s.Γ.mem X → X = s.Γ.master)
    (B : TAlgebra s.T) (g g' : AlgHom (colimAlg s) B)
    (hg : IsStrict g.hom) (hg' : IsStrict g'.hom) :
    g.hom = g'.hom := by
  have key : g.hom.comp (iSupRho s) = g'.hom.comp (iSupRho s) := by
    apply ApproximableMap.ext
    intro X Z
    rw [comp_rel, comp_rel]
    constructor
    · rintro ⟨Y, ⟨n, hrho⟩, hgYZ⟩
      have hin : (g.hom.comp (rho s n)).rel X Z := ⟨Y, hrho, hgYZ⟩
      rw [gcomp_rho_indep s hΓ B g g' hg hg' n] at hin
      obtain ⟨Y', hrho', hgYZ'⟩ := hin
      exact ⟨Y', ⟨n, hrho'⟩, hgYZ'⟩
    · rintro ⟨Y, ⟨n, hrho⟩, hgYZ⟩
      have hin : (g'.hom.comp (rho s n)).rel X Z := ⟨Y, hrho, hgYZ⟩
      rw [← gcomp_rho_indep s hΓ B g g' hg hg' n] at hin
      obtain ⟨Y', hrho', hgYZ'⟩ := hin
      exact ⟨Y', ⟨n, hrho'⟩, hgYZ'⟩
  have e : g.hom.comp (idMap (colim s)) = g.hom := comp_idMap g.hom
  have e' : g'.hom.comp (idMap (colim s)) = g'.hom := comp_idMap g'.hom
  rw [← e, ← e', ← iSupRho_eq_id]
  exact key

theorem algHom_unique (s : Setup.{w}) (hΓ : ∀ X, s.Γ.mem X → X = s.Γ.master)
    (B : TAlgebra s.T) (g g' : AlgHom (colimAlg s) B)
    (hg : IsStrict g.hom) (hg' : IsStrict g'.hom) : g = g' :=
  algHom_ext (gcomp_eq s hΓ B g g' hg hg')

theorem exists_unique_strict_algHom (s : Setup.{w}) (hΓ : ∀ X, s.Γ.mem X → X = s.Γ.
    master)
    (B : TAlgebra s.T) (hk : IsStrict B.str) :
    ∃ g : AlgHom (colimAlg s) B, IsStrict g.hom ∧
      ∀ g' : AlgHom (colimAlg s) B, IsStrict g'.hom → g' = g := by
  obtain ⟨⟨g, hg⟩⟩ := nonempty_strict_algHom s B hk
  exact ⟨g, hg, fun g' hg' => algHom_unique s hΓ B g' g hg' hg⟩

end Theorem614

end DomainTheory.NeighbourhoodSystems
```

4.7.32 Theorem 6.14 — existence of initial algebras

```
##### Exists initial

**[existsInitial.lean](DomainTheory/NeighbourhoodSystems/DomainEquations/existsInitial
    .lean)**

@import "existsInitial.lean"

<p style="color:#15803d;margin-top:1em;">? <strong>Revised</strong> (2026-11-07) -- <a
     href="../1980_REVISION.md">1980 revision progress</a></p>
```

Theorem_6_14.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.existsInitial

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Theorem614

universe w

open CategoryTheory NeighborhoodSystem SmashProduct Theorem614

/-- Scott's Theorem 6.14: existence and uniqueness of homomorphisms into strict
    algebras. -/
theorem theorem_614 :
    (∀ s : Setup.{w}, ∃ A : TAlgebra s.T,
        Nonempty (Iso (s.T.obj A.carrier) A.carrier) ∧
          ∀ B : TAlgebra s.T, IsStrict B.str → Nonempty (AlgHom A B)) ∧
    (∀ (s : Setup.{w}) (_hΓ : ∀ X, s.Γ.mem X → X = s.Γ.master) (B : TAlgebra s.T) (_hk
     : IsStrict B.str),
      ∃ g : AlgHom (colimAlg s) B, IsStrict g.hom ∧
        ∀ g' : AlgHom (colimAlg s) B, IsStrict g'.hom → g' = g) :=
  ⟨@exists_algebra_with_hom, @exists_unique_strict_algHom⟩

end DomainTheory.NeighbourhoodSystems.Theorem614
```

THEOREM 6.14. If the functor T is continuous on maps and monotone and continuous on domains, and if there is a set Γ such that

$$\{\Gamma\} \triangleleft T(\{\Gamma\}),$$

then there exists an initial T-algebra.

Proof: We proceed as in the proof of the fixed-point theorem by iterating the functor. The assumption about Γ means that, as a neighbourhood system, $T(\{\Gamma\})$ is a system over the same set Γ. Thus, if we iterate T to form $T^n(\{\Gamma\})$, all these systems are over Γ and indeed

$$T^n(\{\Gamma\}) \triangleleft T^{n+1}(\{\Gamma\})$$

for all n. We can thus introduce

$$\mathcal{D} = \bigcup_{n=0}^{\infty} T^n(\{\Gamma\}),$$

and it is easy to check that $\mathcal{D}$ is a system over Γ and

$$T^n(\{\Gamma\}) \lhd \mathcal{D}$$

holds for all n. But then we have for all n:

$$T^n(\{\Gamma\}) \lhd T^{n+1}(\{\Gamma\}) \lhd T(\mathcal{D}),$$

which implies $\mathcal{D} \lhd T(\mathcal{D})$. But T is continuous on domains, so

$$\begin{aligned} T(\mathcal{D}) &= T\left(\bigcup_{n=0}^{\infty} T^n(\{\Gamma\})\right) \\ &= \bigcup_{n=0}^{\infty} T^{n+1}(\{\Gamma\}) \\ &= \mathcal{D}. \end{aligned}$$

Thus, not only is $\mathcal{D}$ a T-algebra, but the isomorphism we get for $\mathcal{D}$ and $T(\mathcal{D})$ is just the identity mapping. We know by 6.9 that homomorphisms exist; what remains to show is that homomorphism from $\mathcal{D}$ are unique. As in the examples, we will show in effect they are determined uniquely on the finite elements of $\mathcal{D}$.

Since each $T^n(\{\Gamma\}) \lhd \mathcal{D}$, there are projection mappings $i_n : T^n(\{\Gamma\}) \to \mathcal{D}$ and $j_n : \mathcal{D} \to T^n(\{\Gamma\})$.

Define $\rho_n : \mathcal{D} \to \mathcal{D}$ by $\rho_n = i_n \circ j_n$. Projection pairs are always pairs of strict mappings (Why?), and so are in the category. By assumption and 6.13, the functor T preserves these maps, so we have

$$T(\rho_n) = T(i_n) \circ T(j_n) = i_{n+1} \circ j_{n+1} = \rho_{n+1}.$$

As a neighbourhood relation ρ_n can be characterized by :

$$X \rho_n Y \quad \text{iff} \quad \exists z \in T^n(\{\Gamma\}).\ X \subseteq z \subseteq Y.$$

We thus see that $\rho_n \subseteq \rho_{n+1}$ and

$$\bigcup_{n=0}^{\infty} \rho_n = I_{\mathcal{D}}.$$

Now suppose $k : T(E) \to E$ is any T-algebra and $h : \mathcal{D} \to E$ is a homomorphism. The mapping will satisfy the fixed-point equation

$$h = k \circ T(h),$$

where no other mappings need be written in because $\mathcal{D} = T(\mathcal{D})$ and so

$$T(h) : \mathcal{D} \to T(E).$$

We wish to show that h really is the least fixed point of this equation.

Define $h_n = h \circ \rho_n : \mathcal{D} \to E$. For $n = 0$, the map ρ_0 is the trivial map where $\rho_0(x) = \bot_{\mathcal{D}}$ for all $x \in |\mathcal{D}|$. But h must be strict, so $h_0(x) = \bot_E$ for all $x \in |\mathcal{D}|$; that is, h_0 is the least element of $|\mathcal{D} \to_\bot E|$. Now calculate :

$$\begin{aligned} k \circ T(h_n) &= k \circ T(h) \circ T(\rho_n) \\ &= h \circ \rho_{n+1} \\ &= h_{n+1} \,. \end{aligned}$$

This shows that the union of the h_n is the least fixed point of $\lambda h.\, k \circ T(h)$. But

$$\begin{aligned} \bigcup_{n=0}^{\infty} h_n &= \bigcup_{n=0}^{\infty} h \circ \rho_n \\ &= h \circ \bigcup_{n=0}^{\infty} \rho_n \\ &= h \circ I_D = h, \end{aligned}$$

so the given h is in fact the least fixed point. The homomorphism is uniquely determined, and D is the initial T-algebra. □

Having the existence of initial T-algebras, we can prove one more result that shows just how minimal they are. We need a lemma about projection pairs, first, that shows where sub-domains fit in. We write $D \trianglelefteq E$ as short for $D \cong D'$ for some $D' \triangleleft E$ in the following. The lemma gives a converse to 6.12.

4.7.33 Proj pair subdomain

projPairSubdomain.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.projectionPair
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Subsystem

variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

def Trianglelefteq (D : NeighborhoodSystem α) (E : NeighborhoodSystem β) : Prop :=
  ∃ D' : NeighborhoodSystem β, D' ? E ∧ (D ≅? D')

@[inherit_doc] infix:50 " ? " => Trianglelefteq

section ProjectionPairFromMaps

variable (i : ApproximableMap D E) (j : ApproximableMap E D)

def IsGen (X : Set α) (Y : Set β) : Prop := i.rel X Y ∧ j.rel Y X

theorem isGen_exists (hji : j.comp i = idMap D) {X : Set α} (hX : D.mem X) :
    ∃ Y, IsGen i j X Y := by
```

```
  have hrel : (j.comp i).rel X X := by rw [hji]; exact ⟨hX, hX, subset_rfl⟩
  obtain ⟨Y, hiXY, hjYX⟩ := hrel
  exact ⟨Y, hiXY, hjYX⟩

theorem isGen_inter {X X' : Set α} {Y Y' : Set β}
    (h : IsGen i j X Y) (h' : IsGen i j X' Y') (hE : E.mem (Y ∩ Y')) :
    IsGen i j (X ∩ X') (Y ∩ Y') := by
  obtain ⟨hiXY, hjYX⟩ := h
  obtain ⟨hiX'Y', hjY'X'⟩ := h'
  have hj1 : j.rel (Y ∩ Y') X :=
    j.mono hjYX Set.inter_subset_left subset_rfl hE (j.rel_cod hjYX)
  have hj2 : j.rel (Y ∩ Y') X' :=
    j.mono hjY'X' Set.inter_subset_right subset_rfl hE (j.rel_cod hjY'X')
  have hjInter : j.rel (Y ∩ Y') (X ∩ X') := j.inter_right hj1 hj2
  have hDXX' : D.mem (X ∩ X') := j.rel_cod hjInter
  have hi1 : i.rel (X ∩ X') Y :=
    i.mono hiXY Set.inter_subset_left subset_rfl hDXX' (i.rel_cod hiXY)
  have hi2 : i.rel (X ∩ X') Y' :=
    i.mono hiX'Y' Set.inter_subset_right subset_rfl hDXX' (i.rel_cod hiX'Y')
  exact ⟨i.inter_right hi1 hi2, hjInter⟩

theorem isGen_master : IsGen i j D.master E.master :=
  ⟨i.master_rel, j.master_rel⟩

def Dprime : NeighborhoodSystem β where
  mem Y := ∃ X, IsGen i j X Y
  master := E.master
  master_nonempty := E.master_nonempty
  master_mem := ⟨D.master, isGen_master i j⟩
  inter_mem := by
    rintro Y₁ Y₂ Z ⟨X₁, hg₁⟩ ⟨X₂, hg₂⟩ ⟨_, hgz⟩ hZsub
    have hEY₁ : E.mem Y₁ := i.rel_cod hg₁.1
    have hEY₂ : E.mem Y₂ := i.rel_cod hg₂.1
    have hEZ : E.mem Z := i.rel_cod hgz.1
    have hEinter : E.mem (Y₁ ∩ Y₂) := E.inter_mem hEY₁ hEY₂ hEZ hZsub
    exact ⟨X₁ ∩ X₂, isGen_inter i j hg₁ hg₂ hEinter⟩
  sub_master := by
    rintro Y ⟨X, hg⟩
    exact E.sub_master (i.rel_cod hg.1)

theorem isGen_mono (hji : j.comp i = idMap D) {X X' : Set α} {Z W : Set β}
    (h : IsGen i j X Z) (h' : IsGen i j X' W) (hZW : Z ⊆ W) : X ⊆ X' := by
  obtain ⟨hiXZ, _⟩ := h
  obtain ⟨_, hjWX'⟩ := h'
  have hiXW : i.rel X W :=
    i.mono hiXZ subset_rfl hZW (i.rel_dom hiXZ) (j.rel_dom hjWX')
  have hrel : (j.comp i).rel X X' := ⟨W, hiXW, hjWX'⟩
  rw [hji] at hrel
  exact hrel.2.2

theorem isGen_fst_unique (hji : j.comp i = idMap D) {X X' : Set α} {Y : Set β}
    (h : IsGen i j X Y) (h' : IsGen i j X' Y) : X = X' :=
  Set.Subset.antisymm (isGen_mono i j hji h h' subset_rfl)
    (isGen_mono i j hji h' h subset_rfl)

theorem isGen_mono' (hij : i.comp j ≤ idMap E) {X X' : Set α} {Z W : Set β}
```

```
    (h : IsGen i j X Z) (h' : IsGen i j X' W) (hXX' : X ⊆ X') : Z ⊆ W := by
  obtain ⟨_, hjZX⟩ := h
  obtain ⟨hiX'W, _⟩ := h'
  have hjZX' : j.rel Z X' :=
    j.mono hjZX subset_rfl hXX' (j.rel_dom hjZX) (i.rel_dom hiX'W)
  have hrel : (i.comp j).rel Z W := ⟨X', hjZX', hiX'W⟩
  exact (hij Z W hrel).2.2

theorem isGen_snd_unique (hij : i.comp j ≤ idMap E) {X : Set α} {Y Y' : Set β}
    (h : IsGen i j X Y) (h' : IsGen i j X Y') : Y = Y' :=
  Set.Subset.antisymm (isGen_mono' i j hij h h' subset_rfl)
    (isGen_mono' i j hij h' h subset_rfl)

@[simp] theorem mem_Dprime {Y : Set β} : (Dprime i j).mem Y ↔ ∃ X, IsGen i j X Y :=
    Iff.rfl

theorem Dprime_subsystem : Dprime i j ? E where
  master_eq := rfl
  sub := by rintro Y ⟨X, hg⟩; exact i.rel_cod hg.1
  inter_closed := by
    rintro Y₁ Y₂ ⟨X₁, hg₁⟩ ⟨X₂, hg₂⟩ hE
    exact ⟨X₁ ∩ X₂, isGen_inter i j hg₁ hg₂ hE⟩

def ofEl (hji : j.comp i = idMap D) (hij : i.comp j ≤ idMap E)
    (y : (Dprime i j).Element) : D.Element where
  mem X := ∃ Y, y.mem Y ∧ IsGen i j X Y
  sub := by rintro X ⟨Y, _, hg⟩; exact i.rel_dom hg.1
  master_mem := ⟨E.master, y.master_mem, isGen_master i j⟩
  inter_mem := by
    rintro X₁ X₂ ⟨Y₁, hY₁y, hg₁⟩ ⟨Y₂, hY₂y, hg₂⟩
    have hyInter : y.mem (Y₁ ∩ Y₂) := y.inter_mem hY₁y hY₂y
    have hEInter : E.mem (Y₁ ∩ Y₂) := (Dprime_subsystem i j).sub (y.sub hyInter)
    exact ⟨Y₁ ∩ Y₂, hyInter, isGen_inter i j hg₁ hg₂ hEInter⟩
  up_mem := by
    rintro X X' ⟨Y, hYy, hg⟩ hDX' hXX'
    obtain ⟨Y', hg'⟩ := isGen_exists i j hji hDX'
    have hYY' : Y ⊆ Y' := isGen_mono' i j hij hg hg' hXX'
    exact ⟨Y', y.up_mem hYy ⟨X', hg'⟩ hYY', hg'⟩

def toEl (hji : j.comp i = idMap D) (x : D.Element) : (Dprime i j).Element where
  mem Y := ∃ X, x.mem X ∧ IsGen i j X Y
  sub := by rintro Y ⟨X, _, hg⟩; exact ⟨X, hg⟩
  master_mem := ⟨D.master, x.master_mem, isGen_master i j⟩
  inter_mem := by
    rintro Y₁ Y₂ ⟨X₁, hX₁x, hg₁⟩ ⟨X₂, hX₂x, hg₂⟩
    have hxInter : x.mem (X₁ ∩ X₂) := x.inter_mem hX₁x hX₂x
    have hDInter : D.mem (X₁ ∩ X₂) := x.sub hxInter
    have hi1 : i.rel (X₁ ∩ X₂) Y₁ :=
      i.mono hg₁.1 Set.inter_subset_left subset_rfl hDInter (i.rel_cod hg₁.1)
    have hi2 : i.rel (X₁ ∩ X₂) Y₂ :=
      i.mono hg₂.1 Set.inter_subset_right subset_rfl hDInter (i.rel_cod hg₂.1)
    have hEinter : E.mem (Y₁ ∩ Y₂) := i.rel_cod (i.inter_right hi1 hi2)
    exact ⟨X₁ ∩ X₂, hxInter, isGen_inter i j hg₁ hg₂ hEinter⟩
  up_mem := by
    rintro Y Y' ⟨X, hXx, hg⟩ ⟨X', hg'⟩ hYY'
    have hXX' : X ⊆ X' := isGen_mono i j hji hg hg' hYY'
```

```
    exact ⟨X', x.up_mem hXx (i.rel_dom hg'.1) hXX', hg'⟩

def dprimeEquiv (hji : j.comp i = idMap D) (hij : i.comp j ≤ idMap E) :
    D.Element ≃o (Dprime i j).Element where
  toFun := toEl i j hji
  invFun := ofEl i j hji hij
  left_inv := by
    intro x
    apply Element.ext
    intro X
    constructor
    · rintro ⟨Y, ⟨X₁, hX₁x, hg₁⟩, hg⟩
      rw [isGen_fst_unique i j hji hg hg₁]; exact hX₁x
    · intro hXx
      obtain ⟨Y, hg⟩ := isGen_exists i j hji (x.sub hXx)
      exact ⟨Y, ⟨X, hXx, hg⟩, hg⟩
  right_inv := by
    intro y
    apply Element.ext
    intro Y
    constructor
    · rintro ⟨X, ⟨Y₁, hY₁y, hg₁⟩, hg⟩
      rw [isGen_snd_unique i j hij hg hg₁]; exact hY₁y
    · intro hYy
      obtain ⟨X, hg⟩ := y.sub hYy
      exact ⟨X, ⟨Y, hYy, hg⟩, hg⟩
  map_rel_iff' := by
    intro x x'
    constructor
    · intro h X hXx
      obtain ⟨Y, hg⟩ := isGen_exists i j hji (x.sub hXx)
      obtain ⟨X₁, hX₁x', hg₁⟩ := h Y ⟨X, hXx, hg⟩
      rw [isGen_fst_unique i j hji hg hg₁]; exact hX₁x'
    · rintro h Y ⟨X, hXx, hg⟩
      exact ⟨X, h X hXx, hg⟩

end ProjectionPairFromMaps

theorem trianglelefteq_of_projectionPair (i : ApproximableMap D E) (j :
    ApproximableMap E D)
    (hji : j.comp i = idMap D) (hij : i.comp j ≤ idMap E) : D ? E :=
  ⟨Dprime i j, Dprime_subsystem i j, ⟨dprimeEquiv i j hji hij⟩⟩

namespace Subsystem.ProjectionPair

variable {α : Type*} {D E : NeighborhoodSystem α}

def lowerSet (P : ProjectionPair D E) (y : E.Element) : Set D.Element :=
  {x | P.inj.toElementMap x ≤ y}

@[simp] theorem mem_lowerSet (P : ProjectionPair D E) {y : E.Element} {x : D.Element}
    :
    x ∈ P.lowerSet y ↔ P.inj.toElementMap x ≤ y := Iff.rfl

def upperSet (P : ProjectionPair D E) (x : D.Element) : Set E.Element :=
  {y | x ≤ P.proj.toElementMap y}
```

```
@[simp] theorem mem_upperSet (P : ProjectionPair D E) {x : D.Element} {y : E.Element}
    :
    y ∈ P.upperSet x ↔ x ≤ P.proj.toElementMap y := Iff.rfl

end Subsystem.ProjectionPair

end DomainTheory.NeighbourhoodSystems
```

4.7.34 Lemma 6.15 — subdomains from projection pairs

```
##### Proj pair subdomain

**[projPairSubdomain.lean](DomainTheory/NeighbourhoodSystems/DomainEquations/
    projPairSubdomain.lean)**

@import "projPairSubdomain.lean"

<p style="color:#15803d;margin-top:1em;">? <strong>Revised</strong> (2026-11-07) -- <a
     href="../1980_REVISION.md">1980 revision progress</a></p>
```

Lemma_6_15.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.projPairSubdomain

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Subsystem

variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

/-- Scott's Lemma 6.15: a projection pair yields a domain embedding. -/
theorem lemma_615 :
    ∀ {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}
      (i : ApproximableMap D E) (j : ApproximableMap E D),
      j.comp i = idMap D → i.comp j ≤ idMap E → D ? E :=
  @trianglelefteq_of_projectionPair

end DomainTheory.NeighbourhoodSystems
```

LEMMA 6.15. For two neighbourhood systems D and E, if there exist a projection pair

$$i : D \to E \quad \text{and} \quad j : E \to D$$

with $j \circ i = I_D$ and $i \circ j \subseteq I_E$, then $D \trianglelefteq E$.

Proof. What we want to show is that i maps finite elements to finite elements, and that the desired D' is the image of D in E.

Suppose $X \in D$. We can write:

$$i(\uparrow X) = \bigcup\{\uparrow Y \mid Y \in i(\uparrow X)\}\ .$$

Applying j to both sides we have:

$$\uparrow X = j \circ i(\uparrow X) = \bigcup\{j(\uparrow Y) \mid Y \in i(\uparrow X)\}.$$

But then, since $X \in \uparrow X$, we find $X \in j(\uparrow Y)$ for some $Y \in i(\uparrow X)$. This implies

$$\uparrow X \subseteq j(\uparrow Y);\ \text{and so}\ i(\uparrow X) \subseteq i \circ j(\uparrow Y) \subseteq \uparrow Y.$$

Since $\uparrow Y \subseteq i(\uparrow X)$ in any case, we conclude $i(\uparrow X) = \uparrow Y$. This proves finite elements are mapped to finite elements.

What of Δ; that is, what is $i(\uparrow \Delta)$? We find, supposing E to be a neighbourhood system over a set Δ', that since $\uparrow \Delta \subseteq j(\uparrow \Delta')$, then $i(\uparrow \Delta) \subseteq \uparrow \Delta'$ and so $i(\uparrow \Delta) = \uparrow \Delta'$. This means that Δ corresponds to Δ'. So we have established that $\mathcal{D}$ is in an inclusion preserving one-one correspondence with a subset $\mathcal{D}'$ of E where $\Delta' \in \mathcal{D}'$. But it remains to show that $\mathcal{D}'$ is a neighbourhood system and that $\mathcal{D}' \triangleleft E$ holds. All we really have to show is that $\mathcal{D}'$ is closed under intersection whenever the intersection belongs to E.

Suppose $Y', Z' \in \mathcal{D}'$ and $Y' \cap Z' \in E$. Let $X' = Y' \cap Z'$. We have, for suitable $Y, Z \in \mathcal{D}$,

$$i(\uparrow Y) = \uparrow Y',\ \text{and so}\ \uparrow Y = j(\uparrow Y');\ \text{and}$$

$$i(\uparrow Z) = \uparrow Z',\ \text{and so}\ \uparrow Z = j(\uparrow Z').$$

But $\uparrow Y' \subseteq \uparrow X'$ and $j(\uparrow Y') \subseteq j(\uparrow X')$; thus $Y \in j(\uparrow X')$. For similar reasons $Z \in j(\uparrow X')$. But then $X = Y \cap Z \in j(\uparrow X')$, and therefore $Y \cap Z \in \mathcal{D}$. (The element $j(\uparrow X')$ must be a filter.) Notice, however, that

$$\uparrow Y \subseteq \uparrow X,\ \text{and so}\ \uparrow Y' \subseteq i(\uparrow X);\ \text{and}$$

$$\uparrow Z \subseteq \uparrow X,\ \text{and so}\ \uparrow Z' \subseteq i(\uparrow X).$$

It follows that $Y' \cap Z' = X' \in i(\uparrow X)$. On the other hand we already knew $X \in j(\uparrow X')$, which implies $i(\uparrow X) \subseteq \uparrow X'$. We may thus conclude that $i(\uparrow X) = \uparrow X'$. In other words $X' \in \mathcal{D}'$. □

4.7.35 Minimal initial

minimalInitial.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.supChain
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_7
public import DomainTheory.NeighbourhoodSystems.DomainEquations.existsInitial
public import DomainTheory.NeighbourhoodSystems.DomainEquations.continuousOnMaps
public import DomainTheory.NeighbourhoodSystems.DomainEquations.initialHom
public import DomainTheory.NeighbourhoodSystems.DomainEquations.strictDomainObj
public import DomainTheory.NeighbourhoodSystems.DomainEquations.projPairSubdomain

@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

universe w

theorem botStrict_rel {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ :
    NeighborhoodSystem β}
    {X : Set α} {Y : Set β} :
    (toStrictMap (strictFun V₀ V₁).bot).1.rel X Y ↔ V₀.mem X ∧ Y = V₁.master := by
  rw [toStrictMap_rel, NeighborhoodSystem.mem_bot, strictFun_master]
  constructor
  · intro h
    have hcb : (⟨constMap V₀ V₁.bot, isStrict_constBot⟩ : StrictMap V₀ V₁) ∈ sstep X Y
    := by
      rw [h]; exact Set.mem_univ _
    rw [mem_sstep, constMap_rel, NeighborhoodSystem.mem_bot] at hcb
    exact hcb
  · rintro ⟨hX, rfl⟩
    exact sstep_cod_master hX

theorem opStep (T : Endofunctor DomainObj.{w}) (D E : DomainObj.{w})
    (j : ApproximableMap D.sys (T.obj D).sys) (k : ApproximableMap (T.obj E).sys E.sys
    )
    (hj : IsStrict j) (hk : IsStrict k)
    (? : ApproximableMap (strictFun D.sys E.sys) (strictFun (T.obj D).sys (T.obj E).
    sys))
    (h? : ∀ f : StrictMap D.sys E.sys,
      (toStrictMap (?.toElementMap (toStrictFilter f))).1 = T.map (X := D) (Y := E) f
    .1)
    (x : (strictFun D.sys E.sys).Element) :
    (toStrictMap (((homOp T D E j k hj hk).comp ?).toElementMap x)).1
      = k.comp ((T.map (X := D) (Y := E) (toStrictMap x).1).comp j) := by
  set h := toStrictMap x with hh
  have hx : toStrictFilter h = x := toStrictFilter_toStrictMap x
  have hφ : ?.toElementMap x
      = toStrictFilter (toStrictMap (?.toElementMap (toStrictFilter h))) := by
    rw [← hx]; exact (toStrictFilter_toStrictMap _).symm
  rw [toElementMap_comp, hφ, homOp_apply_filter, toStrictMap_toStrictFilter]
  show k.comp ((toStrictMap (?.toElementMap (toStrictFilter h))).1.comp j)
      = k.comp ((T.map (X := D) (Y := E) h.1).comp j)
  rw [h? h]

theorem trianglelefteq_of_isInitial
    (T : Endofunctor DomainObj.{w}) (hT : ContinuousOnMaps T)
    (Dalg : TAlgebra T) (hinit : IsInitial Dalg)
    (E : DomainObj.{w}) (isoE : Iso (T.obj E) E) :
    Dalg.carrier.sys ? E.sys := by

  let isoD : Iso (T.obj Dalg.carrier) Dalg.carrier := lambek Dalg hinit

  have hi : IsStrict isoD.hom := isStrict_of_comp_eq_id isoD.inv_hom_id
  have hj : IsStrict isoD.inv := isStrict_of_comp_eq_id isoD.hom_inv_id
  have hu : IsStrict isoE.hom := isStrict_of_comp_eq_id isoE.inv_hom_id
  have hv : IsStrict isoE.inv := isStrict_of_comp_eq_id isoE.hom_inv_id
```

```
have hji : isoD.inv.comp isoD.hom = idMap (T.obj Dalg.carrier).sys := isoD.
  hom_inv_id
have hvu : isoE.inv.comp isoE.hom = idMap (T.obj E).sys := isoE.hom_inv_id
have huv : isoE.hom.comp isoE.inv = idMap E.sys := isoE.inv_hom_id

obtain ⟨?DE, h?DE⟩ := hT Dalg.carrier E
obtain ⟨?ED, h?ED⟩ := hT E Dalg.carrier
obtain ⟨?EE, h?EE⟩ := hT E E

let Oph := (homOp T Dalg.carrier E isoD.inv isoE.hom hj hu).comp ?DE
let Opg := (homOp T E Dalg.carrier isoE.inv isoD.hom hv hi).comp ?ED
let Opk := (homOp T E E isoE.inv isoE.hom hv hu).comp ?EE

let H : ℕ → ApproximableMap Dalg.carrier.sys E.sys := fun n => (toStrictMap (Oph.
  iterElem n)).1
let G : ℕ → ApproximableMap E.sys Dalg.carrier.sys := fun n => (toStrictMap (Opg.
  iterElem n)).1
let K : ℕ → ApproximableMap E.sys E.sys := fun n => (toStrictMap (Opk.iterElem n)).1

have iterElem_zero : ∀ {? : Type w} {V : NeighborhoodSystem ?} (f : ApproximableMap V
   V),
    f.iterElem 0 = V.bot := by
  intro ? V f
  show (f.iterMap 0).toElementMap V.bot = V.bot
  rw [iterMap_zero, toElementMap_idMap]

have H_succ : ∀ n, H (n + 1)
    = isoE.hom.comp ((T.map (X := Dalg.carrier) (Y := E) (H n)).comp isoD.inv) := by
  intro n
  show (toStrictMap (Oph.iterElem (n + 1))).1 = _
  rw [iterElem_succ]
  exact opStep T Dalg.carrier E isoD.inv isoE.hom hj hu ?DE h?DE (Oph.iterElem n)
have G_succ : ∀ n, G (n + 1)
    = isoD.hom.comp ((T.map (X := E) (Y := Dalg.carrier) (G n)).comp isoE.inv) := by
  intro n
  show (toStrictMap (Opg.iterElem (n + 1))).1 = _
  rw [iterElem_succ]
  exact opStep T E Dalg.carrier isoE.inv isoD.hom hv hi ?ED h?ED (Opg.iterElem n)
have K_succ : ∀ n, K (n + 1)
    = isoE.hom.comp ((T.map (X := E) (Y := E) (K n)).comp isoE.inv) := by
  intro n
  show (toStrictMap (Opk.iterElem (n + 1))).1 = _
  rw [iterElem_succ]
  exact opStep T E E isoE.inv isoE.hom hv hu ?EE h?EE (Opk.iterElem n)

have H_mono : ∀ {n m : ℕ}, n ≤ m → H n ≤ H m := fun hnm =>
  toStrictMap_mono (iterElem_mono Oph hnm)
have G_mono : ∀ {n m : ℕ}, n ≤ m → G n ≤ G m := fun hnm =>
  toStrictMap_mono (iterElem_mono Opg hnm)

have key : ∀ (a : ApproximableMap (T.obj Dalg.carrier).sys (T.obj E).sys)
    (b : ApproximableMap (T.obj E).sys (T.obj Dalg.carrier).sys),
    (isoE.hom.comp (a.comp isoD.inv)).comp (isoD.hom.comp (b.comp isoE.inv))
      = isoE.hom.comp ((a.comp b).comp isoE.inv) := by
  intro a b
  rw [comp_assoc isoE.hom (a.comp isoD.inv) (isoD.hom.comp (b.comp isoE.inv)),
```

```
      comp_assoc a isoD.inv (isoD.hom.comp (b.comp isoE.inv)),
      ← comp_assoc isoD.inv isoD.hom (b.comp isoE.inv), hji, idMap_comp,
      ← comp_assoc a b isoE.inv]

have hTcomp : ∀ (p : ApproximableMap Dalg.carrier.sys E.sys)
    (q : ApproximableMap E.sys Dalg.carrier.sys),
    (T.map (X := Dalg.carrier) (Y := E) p).comp (T.map (X := E) (Y := Dalg.carrier)
  q)
      = T.map (X := E) (Y := E) (p.comp q) :=
  fun p q => (T.map_comp (X := E) (Y := Dalg.carrier) (Z := E) p q).symm

have ladder : ∀ n, (H n).comp (G n) = K n := by
  intro n
  induction n with
  | zero =>
    apply ApproximableMap.ext
    intro X Z
    have hH0 : ∀ {P Q}, (H 0).rel P Q ↔ Dalg.carrier.sys.mem P ∧ Q = E.sys.master :=
   by
      intro P Q
      show (toStrictMap (Oph.iterElem 0)).1.rel P Q ↔ _
      rw [iterElem_zero]; exact botStrict_rel
    have hG0 : ∀ {P Q}, (G 0).rel P Q ↔ E.sys.mem P ∧ Q = Dalg.carrier.sys.master :=
   by
      intro P Q
      show (toStrictMap (Opg.iterElem 0)).1.rel P Q ↔ _
      rw [iterElem_zero]; exact botStrict_rel
    have hK0 : ∀ {P Q}, (K 0).rel P Q ↔ E.sys.mem P ∧ Q = E.sys.master := by
      intro P Q
      show (toStrictMap (Opk.iterElem 0)).1.rel P Q ↔ _
      rw [iterElem_zero]; exact botStrict_rel
    constructor
    · rintro ⟨Y, hG, hHr⟩
      rw [hG0] at hG; rw [hH0] at hHr
      obtain ⟨hEX, rfl⟩ := hG
      obtain ⟨_, rfl⟩ := hHr
      rw [hK0]; exact ⟨hEX, rfl⟩
    · intro hK
      rw [hK0] at hK
      obtain ⟨hEX, rfl⟩ := hK
      exact ⟨Dalg.carrier.sys.master,
        (hG0).mpr ⟨hEX, rfl⟩, (hH0).mpr ⟨Dalg.carrier.sys.master_mem, rfl⟩⟩
  | succ n ih =>
    rw [H_succ n, G_succ n, key, K_succ n, hTcomp (H n) (G n), ih]

let hh := (toStrictMap Oph.fixElement).1
let gg := (toStrictMap Opg.fixElement).1
let kk := (toStrictMap Opk.fixElement).1
have H_fix_rel : ∀ X Y, hh.rel X Y ↔ ∃ n, (H n).rel X Y := by
  intro X Y
  show (toStrictMap Oph.fixElement).1.rel X Y ↔ _
  rw [toStrictMap_rel, Oph.fixElement_eq_iSupDirected, NeighborhoodSystem.
  mem_iSupDirected]
  constructor
  · rintro ⟨n, hn⟩; exact ⟨n, hn⟩
  · rintro ⟨n, hn⟩; exact ⟨n, hn⟩
```

```
have G_fix_rel : ∀ X Y, gg.rel X Y ↔ ∃ n, (G n).rel X Y := by
  intro X Y
  show (toStrictMap Opg.fixElement).1.rel X Y ↔ _
  rw [toStrictMap_rel, Opg.fixElement_eq_iSupDirected, NeighborhoodSystem.
  mem_iSupDirected]
  constructor
  · rintro ⟨n, hn⟩; exact ⟨n, hn⟩
  · rintro ⟨n, hn⟩; exact ⟨n, hn⟩
have K_fix_rel : ∀ X Y, kk.rel X Y ↔ ∃ n, (K n).rel X Y := by
  intro X Y
  show (toStrictMap Opk.fixElement).1.rel X Y ↔ _
  rw [toStrictMap_rel, Opk.fixElement_eq_iSupDirected, NeighborhoodSystem.
  mem_iSupDirected]
  constructor
  · rintro ⟨n, hn⟩; exact ⟨n, hn⟩
  · rintro ⟨n, hn⟩; exact ⟨n, hn⟩

have hgk : hh.comp gg = kk := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  · rintro ⟨Y, hgXY, hhYZ⟩
    rw [G_fix_rel] at hgXY
    rw [H_fix_rel] at hhYZ
    obtain ⟨m, hm⟩ := hgXY
    obtain ⟨n, hn⟩ := hhYZ
    rw [K_fix_rel]
    refine ⟨max m n, ?_⟩
    rw [← ladder (max m n)]
    exact ⟨Y, G_mono (le_max_left m n) X Y hm, H_mono (le_max_right m n) Y Z hn⟩
  · intro hk
    rw [K_fix_rel] at hk
    obtain ⟨p, hp⟩ := hk
    rw [← ladder p] at hp
    obtain ⟨Y, hG, hHr⟩ := hp
    exact ⟨Y, (G_fix_rel X Y).mpr ⟨p, hG⟩, (H_fix_rel Y Z).mpr ⟨p, hHr⟩⟩

have hk_le : kk ≤ idMap E.sys := by
  have hstepeq : (toStrictMap (Opk.toElementMap
      (toStrictFilter (⟨idMap E.sys, isStrict_idMap⟩ : StrictMap E.sys E.sys)))).1
        = idMap E.sys := by
    have hs := opStep T E E isoE.inv isoE.hom hv hu ?EE h?EE
      (toStrictFilter (⟨idMap E.sys, isStrict_idMap⟩ : StrictMap E.sys E.sys))
    have hmapid : T.map (X := E) (Y := E) (idMap E.sys) = idMap (T.obj E).sys := T.
  map_id E
    rw [hs, show (toStrictMap (toStrictFilter
        (⟨idMap E.sys, isStrict_idMap⟩ : StrictMap E.sys E.sys))).1 = idMap E.sys
  from
        congrArg Subtype.val (toStrictMap_toStrictFilter _),
      hmapid, idMap_comp, huv]
  have hfp : Opk.toElementMap (toStrictFilter (⟨idMap E.sys, isStrict_idMap⟩ :
  StrictMap E.sys E.sys))
      = toStrictFilter (⟨idMap E.sys, isStrict_idMap⟩ : StrictMap E.sys E.sys) := by
    calc Opk.toElementMap (toStrictFilter ⟨idMap E.sys, isStrict_idMap⟩)
        = toStrictFilter (toStrictMap (Opk.toElementMap
            (toStrictFilter ⟨idMap E.sys, isStrict_idMap⟩))) :=
```

```
            (toStrictFilter_toStrictMap _).symm
        _ = toStrictFilter (⟨idMap E.sys, isStrict_idMap⟩ : StrictMap E.sys E.sys) :=
    by
            congr 1
            apply Subtype.ext
            rw [hstepeq]
        _ = toStrictFilter ⟨idMap E.sys, isStrict_idMap⟩ := rfl
    have hle : Opk.fixElement ≤ toStrictFilter (⟨idMap E.sys, isStrict_idMap⟩ :
    StrictMap E.sys E.sys) :=
      fixElement_le_of_toElementMap_le Opk (le_of_eq hfp)
    have hmono := toStrictMap_mono hle
    show (toStrictMap Opk.fixElement).1 ≤ idMap E.sys
    refine le_of_le_of_eq hmono ?_
    exact congrArg Subtype.val (toStrictMap_toStrictFilter _)

  let Balg : TAlgebra T := ⟨E, isoE.hom⟩
  have h_fixeq : hh = isoE.hom.comp ((T.map (X := Dalg.carrier) (Y := E) hh).comp isoD
    .inv) := by
    have hs := opStep T Dalg.carrier E isoD.inv isoE.hom hj hu ?DE h?DE Oph.fixElement
    rw [toElementMap_fixElement] at hs
    exact hs
  have h_comm : hh.comp isoD.hom = isoE.hom.comp (T.map (X := Dalg.carrier) (Y := E)
    hh) := by
    conv_lhs => rw [h_fixeq]
    rw [comp_assoc, comp_assoc, hji, comp_idMap]
  let h_alg : AlgHom Dalg Balg := { hom := hh, comm := h_comm }
  have g_fixeq : gg = isoD.hom.comp ((T.map (X := E) (Y := Dalg.carrier) gg).comp isoE
    .inv) := by
    have hs := opStep T E Dalg.carrier isoE.inv isoD.hom hv hi ?ED h?ED Opg.fixElement
    rw [toElementMap_fixElement] at hs
    exact hs
  have g_comm : gg.comp isoE.hom = isoD.hom.comp (T.map (X := E) (Y := Dalg.carrier)
    gg) := by
    conv_lhs => rw [g_fixeq]
    rw [comp_assoc, comp_assoc, hvu, comp_idMap]
  let g_alg : AlgHom Balg Dalg := { hom := gg, comm := g_comm }

  have hgh_id : gg.comp hh = idMap Dalg.carrier.sys := by
    have huniq : g_alg.comp h_alg = AlgHom.id Dalg := by
      rw [hinit.uniq Dalg (g_alg.comp h_alg), hinit.uniq Dalg (AlgHom.id Dalg)]
    exact congrArg AlgHom.hom huniq

  exact trianglelefteq_of_projectionPair hh gg hgh_id (le_of_eq_of_le hgk hk_le)

end DomainTheory.NeighbourhoodSystems
```

4.7.36 Theorem 6.16 — initial algebra is minimal

```
##### Minimal initial

**[minimalInitial.lean](DomainTheory/NeighbourhoodSystems/DomainEquations/
    minimalInitial.lean)**

@import "minimalInitial.lean"
```

? Revised (2026-11-07) -- 1980 revision progress

Theorem_6_16.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.minimalInitial
public import DomainTheory.NeighbourhoodSystems.DomainEquations.strictDomainObj
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.seqNegMerge
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.smashProduct

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

universe w

/-- Scott's Theorem 6.16: the initial algebra embeds in any carrier with a coalgebra
    isomorphism. -/
theorem theorem_616 (T : Endofunctor DomainObj.{w}) (hT : ContinuousOnMaps T) (Dalg :
    TAlgebra T)
    (hinit : IsInitial Dalg) (E : DomainObj.{w}) (isoE : Iso (T.obj E) E) :
    Dalg.carrier.sys ? E.sys :=
  trianglelefteq_of_isInitial T hT Dalg hinit E isoE

open SeqEqC

theorem sum3_nonempty {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ :
    NeighborhoodSystem β}
    {V₂ : NeighborhoodSystem ?} {h₀ : ∀ X, V₀.mem X → X.Nonempty}
    {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty} {h₂ : ∀ Z, V₂.mem Z → Z.Nonempty} :
    ∀ W, (sum3 V₀ V₁ V₂ h₀ h₁ h₂).mem W → W.Nonempty := by
  rintro W (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩ | ⟨Z, hZ, rfl⟩)
  · exact ⟨none, none_mem_master3⟩
  · exact j0_nonempty (h₀ X hX)
  · exact j1_nonempty (h₁ Y hY)
  · exact j2_nonempty (h₂ Z hZ)

abbrev tcObj (D : StrictDomainObj.{w}) : StrictDomainObj.{w} where
  carrier := Option (Unit ? D.carrier ? D.carrier)
  sys := sum3 unitSys D.sys D.sys SeqEqC.unitSys_nonempty D.nonempty D.nonempty
  nonempty := sum3_nonempty

@[simp] theorem tcObj_sys (D : StrictDomainObj.{w}) :
    (tcObj D).sys = sum3 unitSys D.sys D.sys SeqEqC.unitSys_nonempty D.nonempty D.
    nonempty := rfl

section ShapeLemmas

open SeqEqC

variable {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  {V₂ : NeighborhoodSystem ?} {h₀ : ∀ X, V₀.mem X → X.Nonempty}
```

```
  {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty} {h₂ : ∀ Z, V₂.mem Z → Z.Nonempty}

theorem mem_subset_j0_inv {W : Set (Option (α ? β ? ?))} {X : Set α}
    (hW : (sum3 V₀ V₁ V₂ h₀ h₁ h₂).mem W) (hsub : W ⊆ j0 X) :
    ∃ X₂, V₀.mem X₂ ∧ W = j0 X₂ := by
  rcases hW with rfl | ⟨X₂, hX₂, rfl⟩ | ⟨Y₂, hY₂, rfl⟩ | ⟨Z₂, hZ₂, rfl⟩
  · exact absurd (hsub none_mem_master3) none_not_mem_j0
  · exact ⟨X₂, hX₂, rfl⟩
  · obtain ⟨b, hb⟩ := h₁ Y₂ hY₂; exact absurd (hsub (t1_mem_j1.mpr hb)) t1_not_mem_j0
  · obtain ⟨c, hc⟩ := h₂ Z₂ hZ₂; exact absurd (hsub (t2_mem_j2.mpr hc)) t2_not_mem_j0

theorem mem_subset_j1_inv {W : Set (Option (α ? β ? ?))} {Y : Set β}
    (hW : (sum3 V₀ V₁ V₂ h₀ h₁ h₂).mem W) (hsub : W ⊆ j1 Y) :
    ∃ Y₂, V₁.mem Y₂ ∧ W = j1 Y₂ := by
  rcases hW with rfl | ⟨X₂, hX₂, rfl⟩ | ⟨Y₂, hY₂, rfl⟩ | ⟨Z₂, hZ₂, rfl⟩
  · exact absurd (hsub none_mem_master3) none_not_mem_j1
  · obtain ⟨a, ha⟩ := h₀ X₂ hX₂; exact absurd (hsub (t0_mem_j0.mpr ha)) t0_not_mem_j1
  · exact ⟨Y₂, hY₂, rfl⟩
  · obtain ⟨c, hc⟩ := h₂ Z₂ hZ₂; exact absurd (hsub (t2_mem_j2.mpr hc)) t2_not_mem_j1

theorem mem_subset_j2_inv {W : Set (Option (α ? β ? ?))} {Z : Set ?}
    (hW : (sum3 V₀ V₁ V₂ h₀ h₁ h₂).mem W) (hsub : W ⊆ j2 Z) :
    ∃ Z₂, V₂.mem Z₂ ∧ W = j2 Z₂ := by
  rcases hW with rfl | ⟨X₂, hX₂, rfl⟩ | ⟨Y₂, hY₂, rfl⟩ | ⟨Z₂, hZ₂, rfl⟩
  · exact absurd (hsub none_mem_master3) none_not_mem_j2
  · obtain ⟨a, ha⟩ := h₀ X₂ hX₂; exact absurd (hsub (t0_mem_j0.mpr ha)) t0_not_mem_j2
  · obtain ⟨b, hb⟩ := h₁ Y₂ hY₂; exact absurd (hsub (t1_mem_j1.mpr hb)) t1_not_mem_j2
  · exact ⟨Z₂, hZ₂, rfl⟩

end ShapeLemmas

section SumInj

open SeqEqC

variable {α β ? : Type*}
  {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ : NeighborhoodSystem ?}
  {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
  {h₂ : ∀ Z, V₂.mem Z → Z.Nonempty}

def sinj0 (x₀ : V₀.Element) : (sum3 V₀ V₁ V₂ h₀ h₁ h₂).Element where
  mem W := W = master3 V₀ V₁ V₂ ∨ ∃ X, V₀.mem X ∧ W = j0 X ∧ x₀.mem X
  sub := by
    rintro W (rfl | ⟨X, hX, rfl, -⟩)
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨X, hX, rfl⟩)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hx⟩) (rfl | ⟨X', hX', rfl, hx'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr ⟨X', hX', by rw [master3_inter_j0 hX'], hx'⟩
    · exact Or.inr ⟨X, hX, by rw [Set.inter_comm, master3_inter_j0 hX], hx⟩
    · exact Or.inr ⟨X ∩ X', x₀.sub (x₀.inter_mem hx hx'), j0_inter_j0 X X', x₀.inter_mem
      hx hx'⟩
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hx⟩) hW' hsub
```

```
    · exact Or.inl (eq_master3_of_subset hsub ((sum3 V₀ V₁ V₂ h₀ h₁ h₂).sub_master hW'))
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
      · exact Or.inl rfl
      · exact Or.inr ⟨X', hX', rfl, x₀.up_mem hx hX' (j0_subset_j0.mp hsub)⟩
      · obtain ⟨a, ha⟩ := h₀ X (x₀.sub hx); exact absurd (hsub (t0_mem_j0.mpr ha))
    t0_not_mem_j1
      · obtain ⟨a, ha⟩ := h₀ X (x₀.sub hx); exact absurd (hsub (t0_mem_j0.mpr ha))
    t0_not_mem_j2

@[simp] theorem sinj0_mem_j0 {x₀ : V₀.Element} {X : Set α} (hX : V₀.mem X) :
    (sinj0 (V₁ := V₁) (V₂ := V₂) (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) x₀).mem (j0 X) ↔ x₀.mem
     X := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hx⟩)
    · exact absurd (h0 ▶ none_mem_master3) none_not_mem_j0
    · rw [j0_injective heq]; exact hx
  · intro hx; exact Or.inr ⟨X, hX, rfl, hx⟩

def sinj1 (x₁ : V₁.Element) : (sum3 V₀ V₁ V₂ h₀ h₁ h₂).Element where
  mem W := W = master3 V₀ V₁ V₂ ∨ ∃ Y, V₁.mem Y ∧ W = j1 Y ∧ x₁.mem Y
  sub := by
    rintro W (rfl | ⟨Y, hY, rfl, -⟩)
    · exact Or.inl rfl
    · exact Or.inr (Or.inr (Or.inl ⟨Y, hY, rfl⟩))
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨Y, hY, rfl, hx⟩) (rfl | ⟨Y', hY', rfl, hx'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr ⟨Y', hY', by rw [master3_inter_j1 hY'], hx'⟩
    · exact Or.inr ⟨Y, hY, by rw [Set.inter_comm, master3_inter_j1 hY], hx⟩
    · exact Or.inr ⟨Y ∩ Y', x₁.sub (x₁.inter_mem hx hx'), j1_inter_j1 Y Y', x₁.inter_mem
     hx hx'⟩
  up_mem := by
    rintro W W' (rfl | ⟨Y, hY, rfl, hx⟩) hW' hsub
    · exact Or.inl (eq_master3_of_subset hsub ((sum3 V₀ V₁ V₂ h₀ h₁ h₂).sub_master hW'))
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨b, hb⟩ := h₁ Y (x₁.sub hx); exact absurd (hsub (t1_mem_j1.mpr hb))
    t1_not_mem_j0
      · exact Or.inr ⟨Y', hY', rfl, x₁.up_mem hx hY' (j1_subset_j1.mp hsub)⟩
      · obtain ⟨b, hb⟩ := h₁ Y (x₁.sub hx); exact absurd (hsub (t1_mem_j1.mpr hb))
    t1_not_mem_j2

@[simp] theorem sinj1_mem_j1 {x₁ : V₁.Element} {Y : Set β} (hY : V₁.mem Y) :
    (sinj1 (V₀ := V₀) (V₂ := V₂) (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) x₁).mem (j1 Y) ↔ x₁.mem
     Y := by
  constructor
  · rintro (h0 | ⟨Y', hY', heq, hx⟩)
    · exact absurd (h0 ▶ none_mem_master3) none_not_mem_j1
    · rw [j1_injective heq]; exact hx
  · intro hx; exact Or.inr ⟨Y, hY, rfl, hx⟩

def sinj2 (x₂ : V₂.Element) : (sum3 V₀ V₁ V₂ h₀ h₁ h₂).Element where
  mem W := W = master3 V₀ V₁ V₂ ∨ ∃ Z, V₂.mem Z ∧ W = j2 Z ∧ x₂.mem Z
  sub := by
    rintro W (rfl | ⟨Z, hZ, rfl, -⟩)
```

```
    · exact Or.inl rfl
    · exact Or.inr (Or.inr (Or.inr ⟨Z, hZ, rfl⟩))
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨Z, hZ, rfl, hx⟩) (rfl | ⟨Z', hZ', rfl, hx'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr ⟨Z', hZ', by rw [master3_inter_j2 hZ'], hx'⟩
    · exact Or.inr ⟨Z, hZ, by rw [Set.inter_comm, master3_inter_j2 hZ], hx⟩
    · exact Or.inr ⟨Z ∩ Z', x₂.sub (x₂.inter_mem hx hx'), j2_inter_j2 Z Z', x₂.inter_mem
     hx hx'⟩
  up_mem := by
    rintro W W' (rfl | ⟨Z, hZ, rfl, hx⟩) hW' hsub
    · exact Or.inl (eq_master3_of_subset hsub ((sum3 V₀ V₁ V₂ h₀ h₁ h₂).sub_master hW'))
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨c, hc⟩ := h₂ Z (x₂.sub hx); exact absurd (hsub (t2_mem_j2.mpr hc))
    t2_not_mem_j0
      · obtain ⟨c, hc⟩ := h₂ Z (x₂.sub hx); exact absurd (hsub (t2_mem_j2.mpr hc))
    t2_not_mem_j1
      · exact Or.inr ⟨Z', hZ', rfl, x₂.up_mem hx hZ' (j2_subset_j2.mp hsub)⟩

@[simp] theorem sinj2_mem_j2 {x₂ : V₂.Element} {Z : Set ?} (hZ : V₂.mem Z) :
    (sinj2 (V₀ := V₀) (V₁ := V₁) (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) x₂).mem (j2 Z) ↔ x₂.mem
     Z := by
  constructor
  · rintro (h0 | ⟨Z', hZ', heq, hx⟩)
    · exact absurd (h0 ▸ none_mem_master3) none_not_mem_j2
    · rw [j2_injective heq]; exact hx
  · intro hx; exact Or.inr ⟨Z, hZ, rfl, hx⟩

end SumInj

section SumInjMap

open SeqEqC

variable {α β ? α' β' ?' : Type*}
  {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ : NeighborhoodSystem ?}
  {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'} {V₂' : NeighborhoodSystem
    ?'}
  {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
  {h₂ : ∀ Z, V₂.mem Z → Z.Nonempty}
  {h₀' : ∀ X, V₀'.mem X → X.Nonempty} {h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty}
  {h₂' : ∀ Z, V₂'.mem Z → Z.Nonempty}

theorem sinj1_mono {x x' : V₁.Element} (hx : x ≤ x') :
    sinj1 (V₀ := V₀) (V₂ := V₂) (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) x ≤ sinj1 x' := by
  rintro W (rfl | ⟨Y, hY, rfl, hm⟩)
  · exact Or.inl rfl
  · exact Or.inr ⟨Y, hY, rfl, hx Y hm⟩

theorem sinj2_mono {x x' : V₂.Element} (hx : x ≤ x') :
    sinj2 (V₀ := V₀) (V₁ := V₁) (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) x ≤ sinj2 x' := by
  rintro W (rfl | ⟨Z, hZ, rfl, hm⟩)
  · exact Or.inl rfl
  · exact Or.inr ⟨Z, hZ, rfl, hx Z hm⟩
```

```
end SumInjMap

section SumMap3

open SeqEqC

variable {α β ? α' β' ?' : Type*}
  {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ : NeighborhoodSystem ?}
  {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'} {V₂' : NeighborhoodSystem
    ?'}
  {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
  {h₂ : ∀ Z, V₂.mem Z → Z.Nonempty}
  {h₀' : ∀ X, V₀'.mem X → X.Nonempty} {h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty}
  {h₂' : ∀ Z, V₂'.mem Z → Z.Nonempty}

def sumMap3 (f₀ : ApproximableMap V₀ V₀') (f₁ : ApproximableMap V₁ V₁')
    (f₂ : ApproximableMap V₂ V₂') :
    ApproximableMap (sum3 V₀ V₁ V₂ h₀ h₁ h₂) (sum3 V₀' V₁' V₂' h₀' h₁' h₂') where
  rel W W' := (sum3 V₀ V₁ V₂ h₀ h₁ h₂).mem W ∧ (sum3 V₀' V₁' V₂' h₀' h₁' h₂').mem W' ∧
    (W' = master3 V₀' V₁' V₂' ∨
      (∃ X Y', W = j0 X ∧ W' = j0 Y' ∧ f₀.rel X Y') ∨
      (∃ X Y', W = j1 X ∧ W' = j1 Y' ∧ f₁.rel X Y') ∨
      (∃ X Y', W = j2 X ∧ W' = j2 Y' ∧ f₂.rel X Y'))
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(sum3 V₀ V₁ V₂ h₀ h₁ h₂).master_mem, (sum3 V₀' V₁' V₂' h₀' h₁' h₂').
    master_mem,
    Or.inl rfl⟩
  inter_right := by
    rintro W W'₁ W'₂ ⟨hW, hW'₁, hd₁⟩ ⟨_, hW'₂, hd₂⟩
    have hmem : ∀ W'' : Set (Option (α' ? β' ? ?')),
        (W'' = master3 V₀' V₁' V₂' ∨
          (∃ X Y', W = j0 X ∧ W'' = j0 Y' ∧ f₀.rel X Y') ∨
          (∃ X Y', W = j1 X ∧ W'' = j1 Y' ∧ f₁.rel X Y') ∨
          (∃ X Y', W = j2 X ∧ W'' = j2 Y' ∧ f₂.rel X Y')) →
          (sum3 V₀' V₁' V₂' h₀' h₁' h₂').mem W'' := by
      rintro W'' (rfl | ⟨_, Y', _, rfl, hf⟩ | ⟨_, Y', _, rfl, hf⟩ | ⟨_, Y', _, rfl, hf⟩)
      · exact (sum3 V₀' V₁' V₂' h₀' h₁' h₂').master_mem
      · exact Or.inr (Or.inl ⟨Y', f₀.rel_cod hf, rfl⟩)
      · exact Or.inr (Or.inr (Or.inl ⟨Y', f₁.rel_cod hf, rfl⟩))
      · exact Or.inr (Or.inr (Or.inr ⟨Y', f₂.rel_cod hf, rfl⟩))
    have key : (W'₁ ∩ W'₂ = master3 V₀' V₁' V₂' ∨
        (∃ X Y', W = j0 X ∧ W'₁ ∩ W'₂ = j0 Y' ∧ f₀.rel X Y') ∨
        (∃ X Y', W = j1 X ∧ W'₁ ∩ W'₂ = j1 Y' ∧ f₁.rel X Y') ∨
        (∃ X Y', W = j2 X ∧ W'₁ ∩ W'₂ = j2 Y' ∧ f₂.rel X Y')) := by
      rcases hd₁ with rfl | ⟨X, Y'₁, hWX₁, rfl, hf₁⟩ | ⟨Y, Y'₁, hWY₁, rfl, hf₁⟩
        | ⟨Z, Y'₁, hWZ₁, rfl, hf₁⟩
      · rw [Set.inter_eq_right.mpr (show W'₂ ⊆ master3 V₀' V₁' V₂' from
          (sum3 V₀' V₁' V₂' h₀' h₁' h₂').sub_master hW'₂)]; exact hd₂
      · rcases hd₂ with rfl | ⟨X', Y'₂, hWX₂, rfl, hf₂⟩ | ⟨Y', Y'₂, hWY₂, rfl, hf₂⟩
          | ⟨Z', Y'₂, hWZ₂, rfl, hf₂⟩
        · rw [Set.inter_eq_left.mpr (j0_subset_master3 (f₀.rel_cod hf₁))]
          exact Or.inr (Or.inl ⟨X, Y'₁, hWX₁, rfl, hf₁⟩)
        · obtain rfl : X = X' := j0_injective (hWX₁.symm.trans hWX₂)
          rw [j0_inter_j0]
```

```
        exact Or.inr (Or.inl ⟨X, Y'₁ ∩ Y'₂, hWX₁, rfl, f₀.inter_right hf₁ hf₂⟩)
      · obtain ⟨a, ha⟩ := h₀ X (f₀.rel_dom hf₁)
        exact absurd ((hWX₁.symm.trans hWY₂) ▸ t0_mem_j0.mpr ha) t0_not_mem_j1
      · obtain ⟨a, ha⟩ := h₀ X (f₀.rel_dom hf₁)
        exact absurd ((hWX₁.symm.trans hWZ₂) ▸ t0_mem_j0.mpr ha) t0_not_mem_j2
    · rcases hd₂ with rfl | ⟨X', Y'₂, hWX₂, rfl, hf₂⟩ | ⟨Y', Y'₂, hWY₂, rfl, hf₂⟩
        | ⟨Z', Y'₂, hWZ₂, rfl, hf₂⟩
      · rw [Set.inter_eq_left.mpr (j1_subset_master3 (f₁.rel_cod hf₁))]
        exact Or.inr (Or.inr (Or.inl ⟨Y, Y'₁, hWY₁, rfl, hf₁⟩))
      · obtain ⟨b, hb⟩ := h₁ Y (f₁.rel_dom hf₁)
        exact absurd ((hWY₁.symm.trans hWX₂) ▸ t1_mem_j1.mpr hb) t1_not_mem_j0
      · obtain rfl : Y = Y' := j1_injective (hWY₁.symm.trans hWY₂)
        rw [j1_inter_j1]
        exact Or.inr (Or.inr (Or.inl ⟨Y, Y'₁ ∩ Y'₂, hWY₁, rfl, f₁.inter_right hf₁ hf₂⟩
  ))
      · obtain ⟨b, hb⟩ := h₁ Y (f₁.rel_dom hf₁)
        exact absurd ((hWY₁.symm.trans hWZ₂) ▸ t1_mem_j1.mpr hb) t1_not_mem_j2
    · rcases hd₂ with rfl | ⟨X', Y'₂, hWX₂, rfl, hf₂⟩ | ⟨Y', Y'₂, hWY₂, rfl, hf₂⟩
        | ⟨Z', Y'₂, hWZ₂, rfl, hf₂⟩
      · rw [Set.inter_eq_left.mpr (j2_subset_master3 (f₂.rel_cod hf₁))]
        exact Or.inr (Or.inr (Or.inr ⟨Z, Y'₁, hWZ₁, rfl, hf₁⟩))
      · obtain ⟨c, hc⟩ := h₂ Z (f₂.rel_dom hf₁)
        exact absurd ((hWZ₁.symm.trans hWX₂) ▸ t2_mem_j2.mpr hc) t2_not_mem_j0
      · obtain ⟨c, hc⟩ := h₂ Z (f₂.rel_dom hf₁)
        exact absurd ((hWZ₁.symm.trans hWY₂) ▸ t2_mem_j2.mpr hc) t2_not_mem_j1
      · obtain rfl : Z = Z' := j2_injective (hWZ₁.symm.trans hWZ₂)
        rw [j2_inter_j2]
        exact Or.inr (Or.inr (Or.inr ⟨Z, Y'₁ ∩ Y'₂, hWZ₁, rfl, f₂.inter_right hf₁ hf₂⟩
  ))
  exact ⟨hW, hmem _ key, key⟩
mono := by
  rintro W W₂ W' W'₂ ⟨hW, hW', hd⟩ hW₂W hW'W'₂ hW₂mem hW'₂mem
  refine ⟨hW₂mem, hW'₂mem, ?_⟩
  rcases hd with rfl | ⟨X, Y', rfl, rfl, hf⟩ | ⟨Y, Y', rfl, rfl, hf⟩ | ⟨Z, Y', rfl,
  rfl, hf⟩
  · exact Or.inl (eq_master3_of_subset hW'W'₂ ((sum3 V₀' V₁' V₂' h₀' h₁' h₂').
  sub_master hW'₂mem))
  · obtain ⟨X₂, hX₂, rfl⟩ := mem_subset_j0_inv hW₂mem hW₂W
    have hX₂X : X₂ ⊆ X := j0_subset_j0.mp hW₂W
    rcases hW'₂mem with rfl | ⟨Y'₂, hY'₂, rfl⟩ | ⟨Y'₂, hY'₂, rfl⟩ | ⟨Y'₂, hY'₂, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨X₂, Y'₂, rfl, rfl,
        f₀.mono hf hX₂X (j0_subset_j0.mp hW'W'₂) hX₂ hY'₂⟩)
    · obtain ⟨a, ha⟩ := h₀' Y' (f₀.rel_cod hf)
      exact absurd (hW'W'₂ (t0_mem_j0.mpr ha)) t0_not_mem_j1
    · obtain ⟨a, ha⟩ := h₀' Y' (f₀.rel_cod hf)
      exact absurd (hW'W'₂ (t0_mem_j0.mpr ha)) t0_not_mem_j2
  · obtain ⟨Y₂, hY₂, rfl⟩ := mem_subset_j1_inv hW₂mem hW₂W
    have hY₂Y : Y₂ ⊆ Y := j1_subset_j1.mp hW₂W
    rcases hW'₂mem with rfl | ⟨Y'₂, hY'₂, rfl⟩ | ⟨Y'₂, hY'₂, rfl⟩ | ⟨Y'₂, hY'₂, rfl⟩
    · exact Or.inl rfl
    · obtain ⟨b, hb⟩ := h₁' Y' (f₁.rel_cod hf)
      exact absurd (hW'W'₂ (t1_mem_j1.mpr hb)) t1_not_mem_j0
    · exact Or.inr (Or.inr (Or.inl ⟨Y₂, Y'₂, rfl, rfl,
        f₁.mono hf hY₂Y (j1_subset_j1.mp hW'W'₂) hY₂ hY'₂⟩))
    · obtain ⟨b, hb⟩ := h₁' Y' (f₁.rel_cod hf)
```

```
        exact absurd (hW'W'₂ (t1_mem_j1.mpr hb)) t1_not_mem_j2
    · obtain ⟨Z₂, hZ₂, rfl⟩ := mem_subset_j2_inv hW₂mem hW₂W
      have hZ₂Z : Z₂ ⊆ Z := j2_subset_j2.mp hW₂W
      rcases hW'₂mem with rfl | ⟨Y'₂, hY'₂, rfl⟩ | ⟨Y'₂, hY'₂, rfl⟩ | ⟨Y'₂, hY'₂, rfl⟩
      · exact Or.inl rfl
      · obtain ⟨c, hc⟩ := h₂' Y' (f₂.rel_cod hf)
        exact absurd (hW'W'₂ (t2_mem_j2.mpr hc)) t2_not_mem_j0
      · obtain ⟨c, hc⟩ := h₂' Y' (f₂.rel_cod hf)
        exact absurd (hW'W'₂ (t2_mem_j2.mpr hc)) t2_not_mem_j1
      · exact Or.inr (Or.inr (Or.inr ⟨Z₂, Y'₂, rfl, rfl,
          f₂.mono hf hZ₂Z (j2_subset_j2.mp hW'W'₂) hZ₂ hY'₂⟩))

theorem isStrict_sumMap3 (f₀ : ApproximableMap V₀ V₀') (f₁ : ApproximableMap V₁ V₁')
    (f₂ : ApproximableMap V₂ V₂') :
    IsStrict (sumMap3 (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) (h₀' := h₀') (h₁' := h₁') (h₂' :=
    h₂')
      f₀ f₁ f₂) := by
  rintro Y ⟨-, -, hd⟩
  have h0 : (none : Option (α ? β ? ?)) ∈ (sum3 V₀ V₁ V₂ h₀ h₁ h₂).master :=
    none_mem_master3
  rcases hd with rfl | ⟨X, Y', hWX, -, -⟩ | ⟨X, Y', hWX, -, -⟩ | ⟨X, Y', hWX, -, -⟩
  · rfl
  · exact absurd (hWX ▶ h0) none_not_mem_j0
  · exact absurd (hWX ▶ h0) none_not_mem_j1
  · exact absurd (hWX ▶ h0) none_not_mem_j2

theorem sumMap3_comp {α'' β'' ?'' : Type*} {V₀'' : NeighborhoodSystem α''}
    {V₁'' : NeighborhoodSystem β''} {V₂'' : NeighborhoodSystem ?''}
    {h₀'' : ∀ X, V₀''.mem X → X.Nonempty} {h₁'' : ∀ Y, V₁''.mem Y → Y.Nonempty}
    {h₂'' : ∀ Z, V₂''.mem Z → Z.Nonempty}
    (g₀ : ApproximableMap V₀' V₀'') (g₁ : ApproximableMap V₁' V₁'') (g₂ :
    ApproximableMap V₂' V₂'')
    (f₀ : ApproximableMap V₀ V₀') (f₁ : ApproximableMap V₁ V₁') (f₂ : ApproximableMap V₂
     V₂') :
    sumMap3 (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) (h₀' := h₀'') (h₁' := h₁'') (h₂' := h₂'')
        (g₀.comp f₀) (g₁.comp f₁) (g₂.comp f₂)
      = (sumMap3 (h₀ := h₀') (h₁ := h₁') (h₂ := h₂') (h₀' := h₀'') (h₁' := h₁'') (h₂' :=
    h₂'')
          g₀ g₁ g₂).comp
        (sumMap3 (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) (h₀' := h₀') (h₁' := h₁') (h₂' := h₂')
          f₀ f₁ f₂) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  · rintro ⟨hW, hW'', hd⟩
    rcases hd with rfl | ⟨X, Z'', rfl, rfl, Y', hf, hg⟩ | ⟨Y, Z'', rfl, rfl, Y', hf, hg⟩
      | ⟨Z, Z'', rfl, rfl, Y', hf, hg⟩
    · exact ⟨master3 V₀' V₁' V₂', ⟨hW, (sum3 V₀' V₁' V₂' h₀' h₁' h₂').master_mem, Or.inl
    rfl⟩,
        (sum3 V₀' V₁' V₂' h₀' h₁' h₂').master_mem, hW'', Or.inl rfl⟩
    · exact ⟨j0 Y', ⟨hW, Or.inr (Or.inl ⟨Y', f₀.rel_cod hf, rfl⟩),
        Or.inr (Or.inl ⟨X, Y', rfl, rfl, hf⟩)⟩,
        Or.inr (Or.inl ⟨Y', f₀.rel_cod hf, rfl⟩)⟩, hW'', Or.inr (Or.inl ⟨Y', Z'', rfl,
    rfl, hg⟩)⟩
    · exact ⟨j1 Y', ⟨hW, Or.inr (Or.inr (Or.inl ⟨Y', f₁.rel_cod hf, rfl⟩)),
        Or.inr (Or.inr (Or.inl ⟨Y, Y', rfl, rfl, hf⟩))⟩,
```

```
        Or.inr (Or.inr (Or.inl ⟨Y', f₁.rel_cod hf, rfl⟩)), hW'',
        Or.inr (Or.inr (Or.inl ⟨Y', Z'', rfl, rfl, hg⟩))⟩
    · exact ⟨j2 Y', ⟨hW, Or.inr (Or.inr (Or.inr ⟨Y', f₂.rel_cod hf, rfl⟩)),
        Or.inr (Or.inr (Or.inr ⟨Z, Y', rfl, rfl, hf⟩))⟩,
        Or.inr (Or.inr (Or.inr ⟨Y', f₂.rel_cod hf, rfl⟩)), hW'',
        Or.inr (Or.inr (Or.inr ⟨Y', Z'', rfl, rfl, hg⟩))⟩
  · rintro ⟨W', ⟨hW, hW', hdf⟩, _, hW'', hdg⟩
    refine ⟨hW, hW'', ?_⟩
    rcases hdg with rfl | ⟨X', Z'', hW'X', rfl, hg⟩ | ⟨Y', Z'', hW'Y', rfl, hg⟩
      | ⟨Z', Z'', hW'Z', rfl, hg⟩
    · exact Or.inl rfl
    · rcases hdf with rfl | ⟨X, Y'₀, rfl, hW'eq, hf⟩ | ⟨Y, Y'₀, rfl, hW'eq, hf⟩
        | ⟨Z, Y'₀, rfl, hW'eq, hf⟩
      · exact absurd ((hW'X'.symm) ▸ none_mem_master3) none_not_mem_j0
      · obtain rfl : Y'₀ = X' := j0_injective (hW'eq.symm.trans hW'X')
        exact Or.inr (Or.inl ⟨X, Z'', rfl, rfl, ⟨Y'₀, hf, hg⟩⟩)
      · obtain ⟨b, hb⟩ := h₁' Y'₀ (f₁.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'X') ▸ t1_mem_j1.mpr hb) t1_not_mem_j0
      · obtain ⟨c, hc⟩ := h₂' Y'₀ (f₂.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'X') ▸ t2_mem_j2.mpr hc) t2_not_mem_j0
    · rcases hdf with rfl | ⟨X, Y'₀, rfl, hW'eq, hf⟩ | ⟨Y, Y'₀, rfl, hW'eq, hf⟩
        | ⟨Z, Y'₀, rfl, hW'eq, hf⟩
      · exact absurd ((hW'Y'.symm) ▸ none_mem_master3) none_not_mem_j1
      · obtain ⟨a, ha⟩ := h₀' Y'₀ (f₀.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'Y') ▸ t0_mem_j0.mpr ha) t0_not_mem_j1
      · obtain rfl : Y'₀ = Y' := j1_injective (hW'eq.symm.trans hW'Y')
        exact Or.inr (Or.inr (Or.inl ⟨Y, Z'', rfl, rfl, ⟨Y'₀, hf, hg⟩⟩))
      · obtain ⟨c, hc⟩ := h₂' Y'₀ (f₂.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'Y') ▸ t2_mem_j2.mpr hc) t2_not_mem_j1
    · rcases hdf with rfl | ⟨X, Y'₀, rfl, hW'eq, hf⟩ | ⟨Y, Y'₀, rfl, hW'eq, hf⟩
        | ⟨Z, Y'₀, rfl, hW'eq, hf⟩
      · exact absurd ((hW'Z'.symm) ▸ none_mem_master3) none_not_mem_j2
      · obtain ⟨a, ha⟩ := h₀' Y'₀ (f₀.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'Z') ▸ t0_mem_j0.mpr ha) t0_not_mem_j2
      · obtain ⟨b, hb⟩ := h₁' Y'₀ (f₁.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'Z') ▸ t1_mem_j1.mpr hb) t1_not_mem_j2
      · obtain rfl : Y'₀ = Z' := j2_injective (hW'eq.symm.trans hW'Z')
        exact Or.inr (Or.inr (Or.inr ⟨Z, Z'', rfl, rfl, ⟨Y'₀, hf, hg⟩⟩))

theorem sumMap3_id :
    sumMap3 (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) (h₀' := h₀) (h₁' := h₁) (h₂' := h₂)
      (idMap V₀) (idMap V₁) (idMap V₂) = idMap (sum3 V₀ V₁ V₂ h₀ h₁ h₂) := by
  apply ApproximableMap.ext
  intro W W'

  constructor
  · rintro ⟨hW, hW', hd⟩
    refine ⟨hW, hW', ?_⟩
    rcases hd with rfl | ⟨X, Y', rfl, rfl, _, _, hXY⟩ | ⟨Y, Y', rfl, rfl, _, _, hXY⟩
      | ⟨Z, Y', rfl, rfl, _, _, hXY⟩
    · exact (sum3 V₀ V₁ V₂ h₀ h₁ h₂).sub_master hW
    · exact j0_subset_j0.mpr hXY
    · exact j1_subset_j1.mpr hXY
    · exact j2_subset_j2.mpr hXY
  · rintro ⟨hW, hW', hsub⟩
    refine ⟨hW, hW', ?_⟩
```

```
    rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩ | ⟨Z, hZ, rfl⟩
    · left; exact eq_master3_of_subset hsub ((sum3 V₀ V₁ V₂ h₀ h₁ h₂).sub_master hW')
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨X, X', rfl, rfl, hX, hX', j0_subset_j0.mp hsub⟩)
      · obtain ⟨a, ha⟩ := h₀ X hX; exact absurd (hsub (t0_mem_j0.mpr ha)) t0_not_mem_j1
      · obtain ⟨a, ha⟩ := h₀ X hX; exact absurd (hsub (t0_mem_j0.mpr ha)) t0_not_mem_j2
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨b, hb⟩ := h₁ Y hY; exact absurd (hsub (t1_mem_j1.mpr hb)) t1_not_mem_j0
      · exact Or.inr (Or.inr (Or.inl ⟨Y, Y', rfl, rfl, hY, hY', j1_subset_j1.mp hsub⟩))
      · obtain ⟨b, hb⟩ := h₁ Y hY; exact absurd (hsub (t1_mem_j1.mpr hb)) t1_not_mem_j2
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩ | ⟨Z', hZ', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨c, hc⟩ := h₂ Z hZ; exact absurd (hsub (t2_mem_j2.mpr hc)) t2_not_mem_j0
      · obtain ⟨c, hc⟩ := h₂ Z hZ; exact absurd (hsub (t2_mem_j2.mpr hc)) t2_not_mem_j1
      · exact Or.inr (Or.inr (Or.inr ⟨Z, Z', rfl, rfl, hZ, hZ', j2_subset_j2.mp hsub⟩))

end SumMap3

section SumInjMap

open SeqEqC

variable {α β ? α' β' ?' : Type*}
  {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β} {V₂ : NeighborhoodSystem ?}
  {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'} {V₂' : NeighborhoodSystem
    ?'}
  {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
  {h₂ : ∀ Z, V₂.mem Z → Z.Nonempty}
  {h₀' : ∀ X, V₀'.mem X → X.Nonempty} {h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty}
  {h₂' : ∀ Z, V₂'.mem Z → Z.Nonempty}

theorem sumMap3_sinj0 (f₀ : ApproximableMap V₀ V₀') (f₁ : ApproximableMap V₁ V₁')
    (f₂ : ApproximableMap V₂ V₂') (x₀ : V₀.Element) :
    (sumMap3 (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) (h₀' := h₀') (h₁' := h₁') (h₂' := h₂') f₀ f₁
     f₂).toElementMap
        (sinj0 (V₁ := V₁) (V₂ := V₂) (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) x₀)
      = sinj0 (V₁ := V₁') (V₂ := V₂') (h₀ := h₀') (h₁ := h₁') (h₂ := h₂') (f₀.
    toElementMap x₀) := by
  apply Element.ext
  intro W'
  constructor
  · rintro ⟨U, hU, hUmem, hU'mem, hd⟩
    rcases hd with rfl | ⟨X, Y', hUj, rfl, hf⟩ | ⟨X, Y', hUj, rfl, hf⟩ | ⟨X, Y', hUj,
    rfl, hf⟩
    · exact Or.inl rfl
    · rcases hU with hUm | ⟨X₀, hX₀, hUeq, hx⟩
      · exact absurd ((hUm.symm.trans hUj) ▸ none_mem_master3) none_not_mem_j0
      · have hXX : X = X₀ := j0_injective (hUj.symm.trans hUeq)
        exact Or.inr ⟨Y', f₀.rel_cod hf, rfl, ⟨X₀, hx, hXX ▸ hf⟩⟩
    · rcases hU with hUm | ⟨X₀, hX₀, hUeq, hx⟩
      · exact absurd ((hUm.symm.trans hUj) ▸ none_mem_master3) none_not_mem_j1
      · obtain ⟨a, ha⟩ := h₀ X₀ hX₀; exact absurd ((hUeq.symm.trans hUj) ▸ t0_mem_j0.mpr
     ha) t0_not_mem_j1
    · rcases hU with hUm | ⟨X₀, hX₀, hUeq, hx⟩
```

```
      · exact absurd ((hUm.symm.trans hUj) ▶ none_mem_master3) none_not_mem_j2
      · obtain ⟨a, ha⟩ := h₀ X₀ hX₀; exact absurd ((hUeq.symm.trans hUj) ▶ t0_mem_j0.mpr
    ha) t0_not_mem_j2
  · rintro (rfl | ⟨Y', hY', rfl, hm⟩)
    · exact ⟨master3 V₀ V₁ V₂, Or.inl rfl, (sum3 V₀ V₁ V₂ h₀ h₁ h₂).master_mem,
        (sum3 V₀' V₁' V₂' h₀' h₁' h₂').master_mem, Or.inl rfl⟩
    · obtain ⟨X, hx, hf⟩ := hm
      exact ⟨j0 X, Or.inr ⟨X, x₀.sub hx, rfl, hx⟩, Or.inr (Or.inl ⟨X, x₀.sub hx, rfl⟩),
        Or.inr (Or.inl ⟨Y', f₀.rel_cod hf, rfl⟩), Or.inr (Or.inl ⟨X, Y', rfl, rfl, hf⟩)⟩

theorem sumMap3_sinj1 (f₀ : ApproximableMap V₀ V₀') (f₁ : ApproximableMap V₁ V₁')
    (f₂ : ApproximableMap V₂ V₂') (x₁ : V₁.Element) :
    (sumMap3 (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) (h₀' := h₀') (h₁' := h₁') (h₂' := h₂') f₀ f₁
    f₂).toElementMap
        (sinj1 (V₀ := V₀) (V₂ := V₂) (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) x₁)
      = sinj1 (V₀ := V₀') (V₂ := V₂') (h₀ := h₀') (h₁ := h₁') (h₂ := h₂') (f₁.
    toElementMap x₁) := by
  apply Element.ext
  intro W'
  constructor
  · rintro ⟨U, hU, hUmem, hU'mem, hd⟩
    rcases hd with rfl | ⟨X, Y', hUj, rfl, hf⟩ | ⟨X, Y', hUj, rfl, hf⟩ | ⟨X, Y', hUj,
    rfl, hf⟩
    · exact Or.inl rfl
    · rcases hU with hUm | ⟨Y₀, hY₀, hUeq, hx⟩
      · exact absurd ((hUm.symm.trans hUj) ▶ none_mem_master3) none_not_mem_j0
      · obtain ⟨b, hb⟩ := h₁ Y₀ hY₀; exact absurd ((hUeq.symm.trans hUj) ▶ t1_mem_j1.mpr
    hb) t1_not_mem_j0
    · rcases hU with hUm | ⟨Y₀, hY₀, hUeq, hx⟩
      · exact absurd ((hUm.symm.trans hUj) ▶ none_mem_master3) none_not_mem_j1
      · have hXX : X = Y₀ := j1_injective (hUj.symm.trans hUeq)
        exact Or.inr ⟨Y', f₁.rel_cod hf, rfl, ⟨Y₀, hx, hXX ▶ hf⟩⟩
    · rcases hU with hUm | ⟨Y₀, hY₀, hUeq, hx⟩
      · exact absurd ((hUm.symm.trans hUj) ▶ none_mem_master3) none_not_mem_j2
      · obtain ⟨b, hb⟩ := h₁ Y₀ hY₀; exact absurd ((hUeq.symm.trans hUj) ▶ t1_mem_j1.mpr
    hb) t1_not_mem_j2
  · rintro (rfl | ⟨Y', hY', rfl, hm⟩)
    · exact ⟨master3 V₀ V₁ V₂, Or.inl rfl, (sum3 V₀ V₁ V₂ h₀ h₁ h₂).master_mem,
        (sum3 V₀' V₁' V₂' h₀' h₁' h₂').master_mem, Or.inl rfl⟩
    · obtain ⟨X, hx, hf⟩ := hm
      exact ⟨j1 X, Or.inr ⟨X, x₁.sub hx, rfl, hx⟩, Or.inr (Or.inr (Or.inl ⟨X, x₁.sub hx,
    rfl⟩)),
        Or.inr (Or.inr (Or.inl ⟨Y', f₁.rel_cod hf, rfl⟩)), Or.inr (Or.inr (Or.inl ⟨X, Y
  ', rfl, rfl, hf⟩))⟩

theorem sumMap3_sinj2 (f₀ : ApproximableMap V₀ V₀') (f₁ : ApproximableMap V₁ V₁')
    (f₂ : ApproximableMap V₂ V₂') (x₂ : V₂.Element) :
    (sumMap3 (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) (h₀' := h₀') (h₁' := h₁') (h₂' := h₂') f₀ f₁
    f₂).toElementMap
        (sinj2 (V₀ := V₀) (V₁ := V₁) (h₀ := h₀) (h₁ := h₁) (h₂ := h₂) x₂)
      = sinj2 (V₀ := V₀') (V₁ := V₁') (h₀ := h₀') (h₁ := h₁') (h₂ := h₂') (f₂.
    toElementMap x₂) := by
  apply Element.ext
  intro W'
  constructor
  · rintro ⟨U, hU, hUmem, hU'mem, hd⟩
```

```
    rcases hd with rfl | ⟨X, Y', hUj, rfl, hf⟩ | ⟨X, Y', hUj, rfl, hf⟩ | ⟨X, Y', hUj,
    rfl, hf⟩
    · exact Or.inl rfl
    · rcases hU with hUm | ⟨Z₀, hZ₀, hUeq, hx⟩
      · exact absurd ((hUm.symm.trans hUj) ▶ none_mem_master3) none_not_mem_j0
      · obtain ⟨c, hc⟩ := h₂ Z₀ hZ₀; exact absurd ((hUeq.symm.trans hUj) ▶ t2_mem_j2.mpr
     hc) t2_not_mem_j0
    · rcases hU with hUm | ⟨Z₀, hZ₀, hUeq, hx⟩
      · exact absurd ((hUm.symm.trans hUj) ▶ none_mem_master3) none_not_mem_j1
      · obtain ⟨c, hc⟩ := h₂ Z₀ hZ₀; exact absurd ((hUeq.symm.trans hUj) ▶ t2_mem_j2.mpr
     hc) t2_not_mem_j1
    · rcases hU with hUm | ⟨Z₀, hZ₀, hUeq, hx⟩
      · exact absurd ((hUm.symm.trans hUj) ▶ none_mem_master3) none_not_mem_j2
      · have hXX : X = Z₀ := j2_injective (hUj.symm.trans hUeq)
        exact Or.inr ⟨Y', f₂.rel_cod hf, rfl, ⟨Z₀, hx, hXX ▶ hf⟩⟩
  · rintro (rfl | ⟨Y', hY', rfl, hm⟩)
    · exact ⟨master3 V₀ V₁ V₂, Or.inl rfl, (sum3 V₀ V₁ V₂ h₀ h₁ h₂).master_mem,
        (sum3 V₀' V₁' V₂' h₀' h₁' h₂').master_mem, Or.inl rfl⟩
    · obtain ⟨X, hx, hf⟩ := hm
      exact ⟨j2 X, Or.inr ⟨X, x₂.sub hx, rfl, hx⟩, Or.inr (Or.inr (Or.inr ⟨X, x₂.sub hx,
     rfl⟩)),
        Or.inr (Or.inr (Or.inr ⟨Y', f₂.rel_cod hf, rfl⟩)), Or.inr (Or.inr (Or.inr ⟨X, Y
    ', rfl, rfl, hf⟩))⟩

end SumInjMap

open SeqEqC in

def tcMapHom {D E : StrictDomainObj.{w}} (f : (D ⟶ E)) :
    ((tcObj D) ⟶ (tcObj E)) :=
  ⟨sumMap3 (h₀ := SeqEqC.unitSys_nonempty) (h₁ := D.nonempty) (h₂ := D.nonempty)
      (h₀' := SeqEqC.unitSys_nonempty) (h₁' := E.nonempty) (h₂' := E.nonempty)
      (idMap unitSys) f.1 f.1, isStrict_sumMap3 _ _ _⟩

end DomainTheory.NeighbourhoodSystems
```

THEOREM 6.16. If on the category of domains and strict approximable maps the functor T is continuous on maps, and if D is an initial T-algebra, then for any system $E \cong T(E)$ we have $D \trianglelefteq E$.

Proof: There is a homomorphism $h : D \to E$. By 6.9 there is a homomorphism $g : E \to D$. Now $g \circ h : D \to D$ is also a homomorphism, so $g \circ h = I_D$ because D is initial. In view of 6.15, all we have to prove now is that $h \circ g \sqsubseteq I_E$.

Let the maps $i : T(D) \to D$ and $j : D \to T(D)$ give the isomorphism for D, and let $u : T(E) \to E$ and $v : E \to T(E)$ do the same for E. By the proof of 6.9 we know

$$g = i \circ T(g) \circ v \quad \text{and} \quad h = u \circ T(h) \circ j$$

and each of these maps is the least fixed point of its respective equation. Let

$$g_0 = \bot_{E \to D} \quad \text{and} \quad h_0 = \bot_{D \to E}$$

and define by recursion

$$g_{n+1} = i \circ T(g_n) \circ v \quad \text{and} \quad h_{n+1} = u \circ T(h_n) \circ j.$$

By the fixed-point calculation

$$g = \bigsqcup_{n=0}^{\infty} g_n \quad \text{and} \quad h = \bigsqcup_{n=0}^{\infty} h_n.$$

Now we see that

$$h_0 \circ g_0 = \bot_{E \to E},$$

and for each n that

$$\begin{aligned} h_{n+1} \circ g_{n+1} &= u \circ T(h_n) \circ j \circ i \circ T(g_n) \circ v \\ &= u \circ T(h_n) \circ T(g_n) \circ v \\ &= u \circ T(h_n \circ g_n) \circ v. \end{aligned}$$

But this means that

$$h \circ g = \bigsqcup_{n=0}^{\infty} (h_n \circ g_n)$$

is the least fixed point for the equation

$$k = u \circ T(k) \circ v.$$

But I_E is one of the fixed points; whence $h \circ g \sqsubseteq I_E$ must follow. □

4.7.37 Lecture VI exercises

EXERCISES

4.7.38 cAlgebra — C initial algebra

cAlgebra.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.seqPeano
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.seqNegMerge
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofIso
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Theorem_6_16
public import DomainTheory.NeighbourhoodSystems.DomainEquations.continuousOnMaps
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.iterSys
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodFunctor

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
```

```
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

open SeqEqC in

abbrev Tc : Endofunctor StrictDomainObj.{w} where
  obj := tcObj
  map := tcMapHom
  map_id D := Subtype.ext (by
    show sumMap3 (idMap unitSys) (idMap D.sys) (idMap D.sys) = idMap (tcObj D).sys
    exact sumMap3_id)
  map_comp {D E F} g f := Subtype.ext (by
    show sumMap3 (idMap unitSys) (g.1.comp f.1) (g.1.comp f.1)
      = (sumMap3 (idMap unitSys) g.1 g.1).comp (sumMap3 (idMap unitSys) f.1 f.1)
    have h := sumMap3_comp (h₀ := SeqEqC.unitSys_nonempty) (h₁ := D.nonempty) (h₂ := D.
    nonempty)
      (h₀' := SeqEqC.unitSys_nonempty) (h₁' := E.nonempty) (h₂' := E.nonempty)
      (h₀'' := SeqEqC.unitSys_nonempty) (h₁'' := F.nonempty) (h₂'' := F.nonempty)
      (idMap unitSys) g.1 g.1 (idMap unitSys) f.1 f.1
    rw [idMap_comp] at h
    exact h)

@[simp] theorem Tc_obj (D : StrictDomainObj.{w}) : Tc.obj D = tcObj D := rfl

@[simp] theorem Tc_map_val {D E : StrictDomainObj.{w}} (f : (D ⟶ E)) :
    (Tc.map f).1 = sumMap3 (h₀ := SeqEqC.unitSys_nonempty) (h₁ := D.nonempty) (h₂ := D.
    nonempty)
      (h₀' := SeqEqC.unitSys_nonempty) (h₁' := E.nonempty) (h₂' := E.nonempty)
      (idMap unitSys) f.1 f.1 := rfl

open SeqDom SeqEqC BitDom in

abbrev Cobj : StrictDomainObj.{0} := ⟨Str, C, C_nonempty⟩

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

namespace SeqEqC

open SeqDom BitDom SeqEqB

@[simp] theorem ccEquiv_apply (x : C.Element) : ccEquiv x = toCC x := rfl

theorem consMap_mem_embBit {b : Bool} {z : C.Element} {X : Set Str} (hX : C.mem X) :
    ((consMap b).toElementMap z).mem (embBit b X) ↔ z.mem X := by
  constructor
  · rintro ⟨X', hzX', _, _, hsub⟩
    rw [← embBit_eq_prepend] at hsub
    exact z.up_mem hzX' hX (embBit_subset.mp hsub)
  · intro hz
    refine ⟨X, hz, z.sub hz, memC_embBit b hX, ?_⟩
    rw [← embBit_eq_prepend]

theorem consMap_not_mem_embBit_ne {b c : Bool} (hbc : b = c) {z : C.Element} {X : Set
    Str} :
    ¬ ((consMap b).toElementMap z).mem (embBit c X) := by
  rintro ⟨X', hzX', hX'mem, _, hsub⟩
  obtain ⟨a, ha⟩ := C_nonempty X' hX'mem
```

```
  rw [← embBit_eq_prepend] at hsub
  obtain ⟨w, hw, heq⟩ := hsub ⟨a, rfl, ha⟩
  rw [List.cons.injEq] at hw; exact hbc hw.1

theorem consMap_not_mem_nil {b : Bool} {z : C.Element} :
    ¬ ((consMap b).toElementMap z).mem ({[]} : Set Str) := by
  rintro ⟨X', hzX', hX'mem, _, hsub⟩
  obtain ⟨a, ha⟩ := C_nonempty X' hX'mem
  rw [← embBit_eq_prepend] at hsub
  have := hsub ⟨a, rfl, ha⟩
  rw [Set.mem_singleton_iff] at this; exact absurd this (by simp)

end SeqEqC

section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

def descE : B.carrier.sys.Element :=
  B.str.1.toElementMap (sinj0 (h₀ := SeqEqC.unitSys_nonempty) (h₁ := B.carrier.nonempty
    )
    (h₂ := B.carrier.nonempty) unitSys.bot)

def descF (b : Bool) (y : B.carrier.sys.Element) : B.carrier.sys.Element :=
  B.str.1.toElementMap (cond b
    (sinj2 (h₀ := SeqEqC.unitSys_nonempty) (h₁ := B.carrier.nonempty) (h₂ := B.carrier.
    nonempty) y)
    (sinj1 (h₀ := SeqEqC.unitSys_nonempty) (h₁ := B.carrier.nonempty) (h₂ := B.carrier.
    nonempty) y))

theorem descF_mono (b : Bool) {y y' : B.carrier.sys.Element} (h : y ≤ y') :
    descF B b y ≤ descF B b y' := by
  cases b
  · exact B.str.1.toElementMap_mono (sinj1_mono h)
  · exact B.str.1.toElementMap_mono (sinj2_mono h)

def descVal (z : B.carrier.sys.Element) : Str → B.carrier.sys.Element
  | [] => z
  | b :: σ => descF B b (descVal z σ)

theorem descVal_append (z : B.carrier.sys.Element) (σ ? : Str) :
    descVal B z (σ ++ ?) = descVal B (descVal B z ?) σ := by
  induction σ with
  | nil => rfl
  | cons b σ ih => exact congrArg (descF B b) ih

theorem descVal_mono_z {z z' : B.carrier.sys.Element} (h : z ≤ z') :
    ∀ σ, descVal B z σ ≤ descVal B z' σ
  | [] => h
  | _ :: σ => descF_mono B _ (descVal_mono_z h σ)
```

```
theorem descMap_hcone {σ τ : Str} (h : σ <+: τ) :
    descVal B B.carrier.sys.bot σ ≤ descVal B B.carrier.sys.bot τ := by
  obtain ⟨?, rfl⟩ := h
  rw [descVal_append]
  exact descVal_mono_z B (B.carrier.sys.bot_le _) σ

theorem descMap_hsing {σ τ : Str} (h : σ <+: τ) :
    descVal B B.carrier.sys.bot σ ≤ descVal B (descE B) τ := by
  obtain ⟨?, rfl⟩ := h
  rw [descVal_append]
  exact descVal_mono_z B (B.carrier.sys.bot_le _) σ

end Initial

end

end DomainTheory.NeighbourhoodSystems
end

section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

def descMap : ApproximableMap C B.carrier.sys :=
  liftC B.carrier.sys (descVal B B.carrier.sys.bot) (descVal B (descE B))
    (fun {_ _} => descMap_hcone B) (fun {_ _} => descMap_hsing B)

end Initial

end DomainTheory.NeighbourhoodSystems
end

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

@[simp] theorem descMap_strBot (σ : Str) :
    (descMap B).toElementMap (strBot σ) = descVal B B.carrier.sys.bot σ :=
  liftC_strBot _ _ _ _ _ σ

end Initial
end
```

```
section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

@[simp] theorem descMap_strElem (σ : Str) :
    (descMap B).toElementMap (strElem σ) = descVal B (descE B) σ :=
  liftC_strElem _ _ _ _ _ σ

end Initial
end

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

theorem C_bot_eq_strBot_nil : C.bot = strBot [] := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  show (C.mem Y ∧ C.master ⊆ Y) ↔ (C.mem Y ∧ cone [] ⊆ Y)
  rw [C_master, cone.nil]

end Initial
end

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

theorem ccEquiv_comp_symm :
    (ofIso ccEquiv).comp (ofIso ccEquiv.symm) = idMap (sum3 unitSys C C SeqEqC.
    unitSys_nonempty
      SeqEqC.C_nonempty SeqEqC.C_nonempty) := by
  apply ext_of_toElementMap
  intro s
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap]
  exact ccEquiv.apply_symm_apply s

end Initial
end

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)
```

```
theorem ccEquiv_symm_comp : (ofIso ccEquiv.symm).comp (ofIso ccEquiv) = idMap C := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap]
  exact ccEquiv.symm_apply_apply x

end Initial
end

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

theorem descMap_strict : IsStrict (descMap B) := by
  rw [isStrict_iff_apply_bot, C_bot_eq_strBot_nil, descMap_strBot]
  rfl

end Initial
end

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

def descStrict : (Cobj ⟶ B.carrier) := ⟨descMap B, descMap_strict B⟩

end Initial
end

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

theorem genKeyBot (g : ApproximableMap C B.carrier.sys) :
    B.str.1.toElementMap ((sumMap3 (h₀ := SeqEqC.unitSys_nonempty) (h₁ := SeqEqC.
    C_nonempty)
        (h₂ := SeqEqC.C_nonempty) (h₀' := SeqEqC.unitSys_nonempty) (h₁' := B.carrier.
    nonempty)
        (h₂' := B.carrier.nonempty) (idMap unitSys) g g).toElementMap (toCC (strBot [])
    ))
      = B.carrier.sys.bot := by
  have hb : toCC (strBot []) = (sum3 unitSys C C SeqEqC.unitSys_nonempty SeqEqC.
    C_nonempty
      SeqEqC.C_nonempty).bot := by
    rw [← C_bot_eq_strBot_nil, ← SeqEqC.ccEquiv_apply]; exact ccEquiv.map_bot
  rw [hb, isStrict_iff_apply_bot.mp (isStrict_sumMap3 (h₀ := SeqEqC.unitSys_nonempty)
```

```
    (h₁ := SeqEqC.C_nonempty) (h₂ := SeqEqC.C_nonempty) (idMap unitSys) g g)]
  exact isStrict_iff_apply_bot.mp B.str.2

end Initial
end

theorem isStrict_ofIso {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ :
    NeighborhoodSystem β}
    (e : V₀.Element ≃o V₁.Element) : IsStrict (ofIso e) := by
  rw [isStrict_iff_apply_bot, toElementMap_ofIso]
  exact e.map_bot

open SeqDom SeqEqC in

abbrev cStr : ((Tc.obj Cobj) ⟶ Cobj) :=
  ⟨ofIso (by exact ccEquiv.symm), isStrict_ofIso _⟩

open SeqDom SeqEqC in

abbrev Calg : TAlgebra Tc := ⟨Cobj, cStr⟩

namespace SeqEqC

open SeqDom BitDom SeqEqB

theorem toCC_consMap (b : Bool) (z : C.Element) :
    toCC ((consMap b).toElementMap z)
      = cond b
          (sinj2 (V₀ := unitSys) (V₁ := C) (h₀ := unitSys_nonempty) (h₁ := C_nonempty)
            (h₂ := C_nonempty) z)
          (sinj1 (V₀ := unitSys) (V₂ := C) (h₀ := unitSys_nonempty) (h₁ := C_nonempty)
            (h₂ := C_nonempty) z) := by
  apply NeighborhoodSystem.Element.ext
  intro W
  cases b
  · simp only [Bool.cond_false]
    constructor
    · rintro (rfl | ⟨rfl, hz⟩ | ⟨X, hX, rfl, hz⟩ | ⟨Y, hY, rfl, hz⟩)
      · exact Or.inl rfl
      · exact absurd hz consMap_not_mem_nil
      · exact Or.inr ⟨X, hX, rfl, (consMap_mem_embBit hX).mp hz⟩
      · exact absurd hz (consMap_not_mem_embBit_ne (by decide))
    · rintro (rfl | ⟨Y, hY, rfl, hz⟩)
      · exact Or.inl rfl
      · exact Or.inr (Or.inr (Or.inl ⟨Y, hY, rfl, (consMap_mem_embBit hY).mpr hz⟩))
  · simp only [Bool.cond_true]
    constructor
    · rintro (rfl | ⟨rfl, hz⟩ | ⟨X, hX, rfl, hz⟩ | ⟨Y, hY, rfl, hz⟩)
      · exact Or.inl rfl
      · exact absurd hz consMap_not_mem_nil
      · exact absurd hz (consMap_not_mem_embBit_ne (by decide))
      · exact Or.inr ⟨Y, hY, rfl, (consMap_mem_embBit hY).mp hz⟩
    · rintro (rfl | ⟨Y, hY, rfl, hz⟩)
      · exact Or.inl rfl
      · exact Or.inr (Or.inr (Or.inr ⟨Y, hY, rfl, (consMap_mem_embBit hY).mpr hz⟩))
```

```
end SeqEqC

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

theorem genKey (g : ApproximableMap C B.carrier.sys) (b : Bool) (w : C.Element) :
    B.str.1.toElementMap ((sumMap3 (h₀ := SeqEqC.unitSys_nonempty) (h₁ := SeqEqC.
    C_nonempty)
        (h₂ := SeqEqC.C_nonempty) (h₀' := SeqEqC.unitSys_nonempty) (h₁' := B.carrier.
    nonempty)
        (h₂' := B.carrier.nonempty) (idMap unitSys) g g).toElementMap
      (toCC ((consMap b).toElementMap w)))
      = descF B b (g.toElementMap w) := by
  rw [toCC_consMap]
  cases b
  · simp only [Bool.cond_false]; rw [sumMap3_sinj1]; rfl
  · simp only [Bool.cond_true]; rw [sumMap3_sinj2]; rfl

end Initial
end

namespace SeqEqC

open SeqDom BitDom SeqEqB

theorem toCC_strElem_nil :
    toCC (strElem []) = sinj0 (V₁ := C) (V₂ := C) (h₀ := unitSys_nonempty) (h₁ :=
    C_nonempty)
      (h₂ := C_nonempty) unitSys.bot := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨rfl, hz⟩ | ⟨X, hX, rfl, hz⟩ | ⟨Y, hY, rfl, hz⟩)
    · exact Or.inl rfl
    · exact Or.inr ⟨Set.univ, rfl, rfl, unitSys.bot.master_mem⟩
    · exact absurd (hz.2 (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embBit
    · exact absurd (hz.2 (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embBit
  · rintro (rfl | ⟨X, hX, rfl, hz⟩)
    · exact Or.inl rfl
    · obtain rfl : X = Set.univ := hX
      exact Or.inr (Or.inl ⟨rfl, memC_singleton [], subset_rfl⟩)

end SeqEqC

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

theorem genKey0 (g : ApproximableMap C B.carrier.sys) :
```

```
    B.str.1.toElementMap ((sumMap3 (h₀ := SeqEqC.unitSys_nonempty) (h₁ := SeqEqC.
    C_nonempty)
        (h₂ := SeqEqC.C_nonempty) (h₀' := SeqEqC.unitSys_nonempty) (h₁' := B.carrier.
    nonempty)
        (h₂' := B.carrier.nonempty) (idMap unitSys) g g).toElementMap (toCC (strElem
    [])))
      = descE B := by
  rw [toCC_strElem_nil, sumMap3_sinj0, toElementMap_idMap]

  rfl

end Initial
end

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

theorem descMap_satisfiesRec :
    descMap B = (B.str.1.comp (sumMap3 (h₀ := SeqEqC.unitSys_nonempty)
        (h₁ := SeqEqC.C_nonempty) (h₂ := SeqEqC.C_nonempty) (h₀' := SeqEqC.
    unitSys_nonempty)
        (h₁' := B.carrier.nonempty) (h₂' := B.carrier.nonempty) (idMap unitSys) (
    descMap B)
        (descMap B))).comp (ofIso ccEquiv) := by
  apply map_ext_C
  · intro σ
    rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, descMap_strBot]
    cases σ with
    | nil => exact (genKeyBot B (descMap B)).symm
    | cons b σ =>
      rw [SeqEqC.ccEquiv_apply, ← consMap_strBot]
      have h := genKey B (descMap B) b (strBot σ)
      rw [descMap_strBot] at h
      exact h.symm
  · intro σ
    rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, descMap_strElem]
    cases σ with
    | nil => exact (genKey0 B (descMap B)).symm
    | cons b σ =>
      rw [SeqEqC.ccEquiv_apply, ← consMap_strElem]
      have h := genKey B (descMap B) b (strElem σ)
      rw [descMap_strElem] at h
      exact h.symm

end Initial
end

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge
```

```
variable (B : TAlgebra Tc)

theorem descComm : (descMap B).comp (ofIso ccEquiv.symm)
    = B.str.1.comp (sumMap3 (h₀ := SeqEqC.unitSys_nonempty) (h₁ := SeqEqC.C_nonempty)
        (h₂ := SeqEqC.C_nonempty) (h₀' := SeqEqC.unitSys_nonempty) (h₁' := B.carrier.
    nonempty)
        (h₂' := B.carrier.nonempty) (idMap unitSys) (descMap B) (descMap B)) := by
  conv_lhs => rw [descMap_satisfiesRec B]
  rw [comp_assoc, ccEquiv_comp_symm, comp_idMap]

end Initial
end

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

def descAlgHom : AlgHom Calg B where
  hom := descStrict B
  comm := by
    apply Subtype.ext
    simp only [StrictDomainObj.comp_val]
    exact descComm B

end Initial
end

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

theorem rec_determines (g : ApproximableMap C B.carrier.sys)
    (hg : g = (B.str.1.comp (sumMap3 (h₀ := SeqEqC.unitSys_nonempty) (h₁ := SeqEqC.
    C_nonempty)
        (h₂ := SeqEqC.C_nonempty) (h₀' := SeqEqC.unitSys_nonempty) (h₁' := B.carrier.
    nonempty)
        (h₂' := B.carrier.nonempty) (idMap unitSys) g g)).comp (ofIso ccEquiv)) :
    g = descMap B := by
  have hbot : ∀ σ, g.toElementMap (strBot σ) = descVal B B.carrier.sys.bot σ := by
    intro σ
    induction σ with
    | nil =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, SeqEqC.
    ccEquiv_apply]
      exact genKeyBot B g
    | cons b σ ih =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, SeqEqC.
    ccEquiv_apply,
```

```
        ← consMap_strBot]
      have h := genKey B g b (strBot σ)
      rw [ih] at h
      exact h
  have helem : ∀ σ, g.toElementMap (strElem σ) = descVal B (descE B) σ := by
    intro σ
    induction σ with
    | nil =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, SeqEqC.
    ccEquiv_apply]
      exact genKey0 B g
    | cons b σ ih =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, SeqEqC.
    ccEquiv_apply,
        ← consMap_strElem]
      have h := genKey B g b (strElem σ)
      rw [ih] at h
      exact h
  apply map_ext_C
  · intro σ; rw [hbot, descMap_strBot]
  · intro σ; rw [helem, descMap_strElem]

end Initial
end

section
section Initial

open SeqDom SeqEqC BitDom SeqPeano SeqNegMerge

variable (B : TAlgebra Tc)

theorem descAlgHom_uniq (h' : AlgHom Calg B) : h' = descAlgHom B := by
  obtain ⟨hom, comm⟩ := h'
  have hg : hom.1 = descMap B := by
    refine rec_determines B hom.1 ?_
    have hc : hom.1.comp (ofIso ccEquiv.symm)
        = B.str.1.comp (sumMap3 (h₀ := SeqEqC.unitSys_nonempty) (h₁ := SeqEqC.
    C_nonempty)
          (h₂ := SeqEqC.C_nonempty) (h₀' := SeqEqC.unitSys_nonempty) (h₁' := B.carrier.
    nonempty)
          (h₂' := B.carrier.nonempty) (idMap unitSys) hom.1 hom.1) :=
      congrArg Subtype.val comm
    have h2 := congrArg (fun m => m.comp (ofIso ccEquiv)) hc
    simp only at h2
    rw [comp_assoc] at h2
    erw [ccEquiv_symm_comp, comp_idMap] at h2
    exact h2
  have hhom : hom = descStrict B := Subtype.ext hg
  subst hhom
  rfl

end Initial
end
```

```
def CisInitial : IsInitial Calg where
  desc := descAlgHom
  uniq := fun B h => descAlgHom_uniq B h
```

4.7.39 cAlgebraGen — Cn over A

cAlgebraGen.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.cAlgebra
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.seqPeano
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofIso
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Theorem_6_16
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.CAlgebraGen
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap DomainTheory
    .NeighbourhoodSystems.SmashProduct

section
variable {A : Type} [DecidableEq A]

abbrev Strn (A : Type) := List A

def coneN (σ : Strn A) : Set (Strn A) := {w | σ <+: w}

omit [DecidableEq A] in

@[simp] theorem mem_coneN {σ w : Strn A} : w ∈ coneN σ ↔ σ <+: w := Iff.rfl

omit [DecidableEq A] in

theorem coneN_nil : coneN ([] : Strn A) = Set.univ := by ext w; simp [coneN]

omit [DecidableEq A] in

theorem coneN_subset_coneN {σ τ : Strn A} : coneN σ ⊆ coneN τ ↔ τ <+: σ := by
  constructor
  · intro h; exact h (show σ ∈ coneN σ from List.prefix_rfl)
  · intro hτσ w hw; exact hτσ.trans hw

omit [DecidableEq A] in

theorem coneN_injective {σ τ : Strn A} (h : coneN σ = coneN τ) : σ = τ := by
  have h1 : τ <+: σ := coneN_subset_coneN.mp (le_of_eq h)
  have h2 : σ <+: τ := coneN_subset_coneN.mp (le_of_eq h.symm)
  exact h2.eq_of_length (h2.length_le.antisymm h1.length_le)
```

```
theorem coneN_trichotomy (σ τ : Strn A) :
    coneN σ ⊆ coneN τ ∨ coneN τ ⊆ coneN σ ∨ coneN σ ∩ coneN τ = ∅ :=
  if hστ : σ <+: τ then Or.inr (Or.inl (coneN_subset_coneN.mpr hστ))
  else if hτσ : τ <+: σ then Or.inl (coneN_subset_coneN.mpr hτσ)
  else Or.inr (Or.inr (by
    ext w
    simp only [Set.mem_inter_iff, mem_coneN, Set.mem_empty_iff_false, iff_false,
    not_and]
    intro h1 h2
    rcases List.prefix_or_prefix_of_prefix h1 h2 with h | h
    · exact hστ h
    · exact hτσ h))

def memCn (X : Set (Strn A)) : Prop := (∃ σ, X = coneN σ) ∨ (∃ σ, X = {σ})

omit [DecidableEq A] in

theorem memCn_coneN (σ : Strn A) : memCn (coneN σ) := Or.inl ⟨σ, rfl⟩

omit [DecidableEq A] in

theorem memCn_singleton (σ : Strn A) : memCn ({σ} : Set (Strn A)) := Or.inr ⟨σ, rfl⟩

omit [DecidableEq A] in

theorem singleton_subset_coneN {σ τ : Strn A} : ({τ} : Set (Strn A)) ⊆ coneN σ ↔ σ
    <+: τ := by
  rw [Set.singleton_subset_iff, mem_coneN]

theorem singleton_coneN_nd (σ τ : Strn A) :
    ({τ} : Set (Strn A)) ⊆ coneN σ ∨ coneN σ ⊆ {τ} ∨ ({τ} : Set (Strn A)) ∩ coneN σ =
    ∅ := by
  by_cases h : σ <+: τ
  · exact Or.inl (singleton_subset_coneN.mpr h)
  · refine Or.inr (Or.inr ?_)
    ext w
    simp only [Set.mem_inter_iff, Set.mem_singleton_iff, mem_coneN, Set.
    mem_empty_iff_false,
      iff_false, not_and]
    rintro rfl hτ
    exact h hτ

theorem nestedOrDisjointN : NestedOrDisjoint (memCn (A := A)) := by
  rintro X Y (⟨σ, rfl⟩ | ⟨σ, rfl⟩) (⟨τ, rfl⟩ | ⟨τ, rfl⟩)
  · exact coneN_trichotomy σ τ
  · rcases singleton_coneN_nd σ τ with h | h | h
    · exact Or.inr (Or.inl h)
    · exact Or.inl h
    · exact Or.inr (Or.inr (by rw [Set.inter_comm]; exact h))
  · rcases singleton_coneN_nd τ σ with h | h | h
    · exact Or.inl h
    · exact Or.inr (Or.inl h)
    · exact Or.inr (Or.inr h)
  · by_cases h : σ = τ
    · subst h; exact Or.inl (Set.Subset.refl _)
    · refine Or.inr (Or.inr ?_)
```

```
      ext w
      simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false,
        not_and]
      rintro rfl h2
      exact h h2

abbrev Cn (A : Type) [DecidableEq A] : NeighborhoodSystem (Strn A) :=
  NeighborhoodSystem.ofNestedOrDisjoint memCn Set.univ
    ⟨[], Set.mem_univ _⟩ (Or.inl ⟨[], coneN_nil.symm⟩) nestedOrDisjointN
    (fun _ => Set.subset_univ _)

@[simp] theorem Cn_mem {X : Set (Strn A)} : (Cn A).mem X ↔ memCn X := Iff.rfl

@[simp] theorem Cn_master : (Cn A).master = (Set.univ : Set (Strn A)) := rfl

theorem Cn_nonempty : ∀ X, (Cn A).mem X → X.Nonempty := by
  rintro X (⟨σ, rfl⟩ | ⟨σ, rfl⟩)
  · exact ⟨σ, List.prefix_rfl⟩
  · exact ⟨σ, rfl⟩

def prependN (σ : Strn A) (X : Set (Strn A)) : Set (Strn A) := {w | ∃ τ, τ ∈ X ∧ w = σ
     ++ τ}

omit [DecidableEq A] in

@[simp] theorem mem_prependN {σ : Strn A} {X : Set (Strn A)} {w : Strn A} :
    w ∈ prependN σ X ↔ ∃ τ, τ ∈ X ∧ w = σ ++ τ := Iff.rfl

omit [DecidableEq A] in

theorem prependN_coneN (σ ? : Strn A) : prependN σ (coneN ?) = coneN (σ ++ ?) := by
  ext w
  simp only [mem_prependN, mem_coneN]
  constructor
  · rintro ⟨τ, hτ, rfl⟩; exact (List.prefix_append_right_inj σ).mpr hτ
  · rintro ⟨t, ht⟩
    exact ⟨? ++ t, List.prefix_append ? t, by rw [← ht, List.append_assoc]⟩

omit [DecidableEq A] in

theorem prependN_mono (σ : Strn A) {X X' : Set (Strn A)} (h : X' ⊆ X) :
    prependN σ X' ⊆ prependN σ X := by
  rintro w ⟨τ, hτ, rfl⟩; exact ⟨τ, h hτ, rfl⟩

omit [DecidableEq A] in

theorem prependN_singleton (σ τ : Strn A) : prependN σ {τ} = {σ ++ τ} := by
  ext w
  simp only [mem_prependN, Set.mem_singleton_iff]
  constructor
  · rintro ⟨t, rfl, rfl⟩; rfl
  · rintro rfl; exact ⟨τ, rfl, rfl⟩

omit [DecidableEq A] in
```

```
theorem memCn_prependN (σ : Strn A) {X : Set (Strn A)} (hX : memCn X) : memCn (
    prependN σ X) := by
  rcases hX with ⟨?, rfl⟩ | ⟨?, rfl⟩
  · exact Or.inl ⟨σ ++ ?, prependN_coneN σ ?⟩
  · exact Or.inr ⟨σ ++ ?, prependN_singleton σ ?⟩

def consMapN (a : A) : ApproximableMap (Cn A) (Cn A) where
  rel X Y := memCn X ∧ memCn Y ∧ prependN [a] X ⊆ Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨Or.inl ⟨[], coneN_nil.symm⟩, Or.inl ⟨[], coneN_nil.symm⟩,
    fun _ _ => trivial⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY, hXY⟩ ⟨_, hY', hXY'⟩
    have hsubInter : prependN [a] X ⊆ Y ∩ Y' := Set.subset_inter hXY hXY'
    have hZ : memCn (prependN [a] X) := memCn_prependN [a] hX
    exact ⟨hX, (Cn A).inter_mem hY hY' hZ hsubInter, hsubInter⟩
  mono := by
    rintro X X' Y Y' ⟨_, _, hXY⟩ hX'X hYY' hX' hY'
    exact ⟨hX', hY', (prependN_mono [a] hX'X).trans (hXY.trans hYY')⟩

@[simp] theorem consMapN_rel {a : A} {X Y : Set (Strn A)} :
    (consMapN a).rel X Y ↔ memCn X ∧ memCn Y ∧ prependN [a] X ⊆ Y := Iff.rfl

section SumSig

universe v

variable {β : Type v} {? : Type v}

abbrev SigTok (A : Type) (β : Type v) := Option (Unit ? (A × β))

end SumSig

def strBotN (σ : Strn A) : (Cn A).Element := (Cn A).principal (Cn_mem.mpr (memCn_coneN
    σ))

theorem consMapN_strBot (a : A) (σ : Strn A) :
    (consMapN a).toElementMap (strBotN σ) = strBotN (a :: σ) := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  constructor
  · rintro ⟨X, ⟨_, hXcone⟩, _, hY, hsub⟩
    refine ⟨hY, ?_⟩
    calc coneN (a :: σ) = prependN [a] (coneN σ) := by rw [prependN_coneN]; rfl
      _ ⊆ prependN [a] X := prependN_mono [a] hXcone
      _ ⊆ Y := hsub
  · rintro ⟨hY, hsub⟩
    refine ⟨coneN σ, ⟨memCn_coneN σ, subset_rfl⟩, memCn_coneN σ, hY, ?_⟩
    rw [show prependN [a] (coneN σ) = coneN (a :: σ) by rw [prependN_coneN]; rfl]
    exact hsub

def strElemN (σ : Strn A) : (Cn A).Element := (Cn A).principal (Cn_mem.mpr (
    memCn_singleton σ))

theorem consMapN_strElem (a : A) (σ : Strn A) :
```

```
    (consMapN a).toElementMap (strElemN σ) = strElemN (a :: σ) := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  constructor
  · rintro ⟨X, ⟨_, hXsing⟩, _, hY, hsub⟩
    refine ⟨hY, ?_⟩
    calc ({a :: σ} : Set (Strn A)) = prependN [a] {σ} := by rw [prependN_singleton];
    rfl
      _ ⊆ prependN [a] X := prependN_mono [a] hXsing
      _ ⊆ Y := hsub
  · rintro ⟨hY, hsub⟩
    refine ⟨{σ}, ⟨memCn_singleton σ, subset_rfl⟩, memCn_singleton σ, hY, ?_⟩
    rw [show prependN [a] ({σ} : Set (Strn A)) = {a :: σ} by rw [prependN_singleton];
    rfl]
    exact hsub

section SumSig

universe v

variable {β : Type v} {? : Type v}

def tc (a : A) (t : β) : SigTok A β := some (Sum.inr (a, t))

def jc (a : A) (X : Set β) : Set (SigTok A β) := {w | ∃ t, w = tc a t ∧ t ∈ X}

omit [DecidableEq A] in

theorem tc_injective {a a' : A} {t t' : β} (h : (tc a t : SigTok A β) = tc a' t') :
    a = a' ∧ t = t' := by
  simp only [tc, Option.some.injEq, Sum.inr.injEq, Prod.mk.injEq] at h; exact h

omit [DecidableEq A] in

@[simp] theorem tc_mem_jc {a : A} {X : Set β} {t : β} :
    (tc a t : SigTok A β) ∈ jc a X ↔ t ∈ X := by
  constructor
  · rintro ⟨t', heq, ht'⟩; obtain ⟨-, rfl⟩ := tc_injective heq; exact ht'
  · intro ht; exact ⟨t, rfl, ht⟩

def tu : SigTok A β := some (Sum.inl ())

def jU : Set (SigTok A β) := {tu}

omit [DecidableEq A] in

@[simp] theorem mem_jU {w : SigTok A β} : w ∈ (jU : Set (SigTok A β)) ↔ w = tu := Iff.
    rfl

omit [DecidableEq A] in

@[simp] theorem none_not_mem_jU : (none : SigTok A β) ∈ (jU : Set (SigTok A β)) := by
  simp [jU, tu]

omit [DecidableEq A] in
```

```
@[simp] theorem none_not_mem_jc {a : A} {X : Set β} : (none : SigTok A β) ∈ jc a X :=
    by
  rintro ⟨t, heq, -⟩; exact absurd heq (by simp [tc])

omit [DecidableEq A] in

@[simp] theorem tu_not_mem_jc {a : A} {X : Set β} : (tu : SigTok A β) ∈ jc a X := by
  rintro ⟨t, heq, -⟩; exact absurd heq (by simp [tu, tc])

omit [DecidableEq A] in

@[simp] theorem tc_not_mem_jU {a : A} {t : β} : (tc a t : SigTok A β) ∈ (jU : Set (
    SigTok A β)) := by
  simp [jU, tu, tc]

omit [DecidableEq A] in

theorem jU_inter_jc (a : A) (X : Set β) : (jU ∩ jc a X : Set (SigTok A β)) = ∅ := by
  ext w
  simp only [Set.mem_inter_iff, mem_jU, Set.mem_empty_iff_false, iff_false, not_and]
  rintro rfl; exact tu_not_mem_jc

omit [DecidableEq A] in

theorem jU_nonempty : (jU : Set (SigTok A β)).Nonempty := ⟨tu, rfl⟩

omit [DecidableEq A] in

theorem jc_inter_jc_ne {a a' : A} (h : a = a') (X X' : Set β) :
    (jc a X ∩ jc a' X' : Set (SigTok A β)) = ∅ := by
  ext w
  simp only [Set.mem_inter_iff, Set.mem_empty_iff_false, iff_false, not_and]
  rintro ⟨t, rfl, -⟩ ⟨t', heq, -⟩
  exact h (tc_injective heq).1

omit [DecidableEq A] in

theorem jc_inter_jc_same (a : A) (X X' : Set β) :
    (jc a X ∩ jc a X' : Set (SigTok A β)) = jc a (X ∩ X') := by
  ext w
  constructor
  · rintro ⟨⟨t, rfl, ht⟩, ⟨t', heq, ht'⟩⟩
    obtain ⟨-, rfl⟩ := tc_injective heq; exact ⟨t, rfl, ht, ht'⟩
  · rintro ⟨t, rfl, ht, ht'⟩; exact ⟨⟨t, rfl, ht⟩, ⟨t, rfl, ht'⟩⟩

omit [DecidableEq A] in

theorem jc_nonempty {a : A} {X : Set β} (hX : X.Nonempty) : (jc a X : Set (SigTok A β)
    ).Nonempty := by
  obtain ⟨t, ht⟩ := hX; exact ⟨tc a t, t, rfl, ht⟩

omit [DecidableEq A] in

theorem jc_subset_jc {a : A} {X X' : Set β} :
    (jc a X : Set (SigTok A β)) ⊆ jc a X' ↔ X ⊆ X' := by
  constructor
```

```
  · intro h t ht; exact tc_mem_jc.mp (h (tc_mem_jc.mpr ht))
  · rintro h w ⟨t, rfl, ht⟩; exact tc_mem_jc.mpr (h ht)

omit [DecidableEq A] in

theorem jc_injective {a : A} {X X' : Set β} (h : (jc a X : Set (SigTok A β)) = jc a X
    ') : X = X' :=
  Set.Subset.antisymm (jc_subset_jc.mp h.subset) (jc_subset_jc.mp h.symm.subset)

omit [DecidableEq A] in

theorem jc_eq_jc {a a' : A} {X X' : Set β} (hXne : X.Nonempty)
    (heq : (jc a X : Set (SigTok A β)) = jc a' X') : a = a' ∧ X = X' := by
  obtain ⟨t, ht⟩ := hXne
  obtain ⟨t', he, -⟩ := heq ▸ (tc_mem_jc.mpr ht)
  obtain ⟨rfl, -⟩ := tc_injective he
  exact ⟨rfl, jc_injective heq⟩

end SumSig

section SumSig

universe v

variable {β : Type v} {? : Type v}

variable (V : NeighborhoodSystem β)

def masterSig : Set (SigTok A β) :=
  {w | w = none ∨ w = tu ∨ ∃ a t, t ∈ V.master ∧ w = tc a t}

end SumSig

section SumSig

universe v

variable {β : Type v} {? : Type v}

variable (V : NeighborhoodSystem β)

variable {V}

omit [DecidableEq A] in

@[simp] theorem none_mem_masterSig : (none : SigTok A β) ∈ masterSig V := Or.inl rfl

end SumSig

section SumSig

universe v

variable {β : Type v} {? : Type v}

omit [DecidableEq A] in
```

```
theorem tc_mem_jc_ne {a a' : A} (h : a = a') {X : Set β} {t : β} :
    (tc a t : SigTok A β) ∈ jc a' X := by
  rintro ⟨t', heq, -⟩; exact h (tc_injective heq).1

end SumSig

section SumSig

universe v

variable {β : Type v} {? : Type v}

variable (V : NeighborhoodSystem β)

variable {V}

omit [DecidableEq A] in

@[simp] theorem tu_mem_masterSig : (tu : SigTok A β) ∈ masterSig V := Or.inr (Or.inl
    rfl)

omit [DecidableEq A] in

theorem eq_masterSig_of_subset {W : Set (SigTok A β)}
    (hsub : masterSig V ⊆ W) (hsub' : W ⊆ masterSig V) : W = masterSig V :=
  Set.Subset.antisymm hsub' hsub

omit [DecidableEq A] in

theorem jU_subset_masterSig : (jU : Set (SigTok A β)) ⊆ masterSig V := by
  rintro w rfl; exact tu_mem_masterSig

omit [DecidableEq A] in

theorem jc_subset_masterSig {a : A} {X : Set β} (hX : V.mem X) :
    (jc a X : Set (SigTok A β)) ⊆ masterSig V := by


  rintro w ⟨t, rfl, ht⟩; exact Or.inr (Or.inr ⟨a, t, V.sub_master hX ht, rfl⟩)

omit [DecidableEq A] in

theorem masterSig_inter_jU : (masterSig V ∩ jU : Set (SigTok A β)) = jU :=
  Set.inter_eq_right.mpr jU_subset_masterSig

omit [DecidableEq A] in

theorem masterSig_inter_jc {a : A} {X : Set β} (hX : V.mem X) :
    (masterSig V ∩ jc a X : Set (SigTok A β)) = jc a X :=
  Set.inter_eq_right.mpr (jc_subset_masterSig hX)

def sumSig (A : Type) [DecidableEq A] (V : NeighborhoodSystem β)
    (h : ∀ X, V.mem X → X.Nonempty) :
    NeighborhoodSystem (SigTok A β) where
  mem W := W = masterSig V ∨ W = jU ∨ ∃ a X, V.mem X ∧ W = jc a X
  master := masterSig V
```

```
  master_nonempty := ⟨none, none_mem_masterSig⟩
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | rfl | ⟨a, X, hX, rfl⟩)
    · exact subset_rfl
    · exact jU_subset_masterSig
    · exact jc_subset_masterSig hX
  inter_mem := by
    have hne : ∀ W, (W = masterSig V ∨ W = jU V ∨ ∃ a X, V.mem X ∧ W = jc a X) →
        (W : Set (SigTok A β)).Nonempty := by
      rintro W (rfl | rfl | ⟨a, X, hX, rfl⟩)
      · exact ⟨none, none_mem_masterSig⟩
      · exact jU_nonempty
      · exact jc_nonempty (h X hX)
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | rfl | ⟨a, X, hX, rfl⟩
    · rcases hW' with rfl | rfl | ⟨a', X', hX', rfl⟩
      · rw [Set.inter_self]; exact Or.inl rfl
      · rw [masterSig_inter_jU]; exact Or.inr (Or.inl rfl)
      · rw [masterSig_inter_jc hX']; exact Or.inr (Or.inr ⟨a', X', hX', rfl⟩)
    · rcases hW' with rfl | rfl | ⟨a', X', hX', rfl⟩
      · rw [Set.inter_comm, masterSig_inter_jU]; exact Or.inr (Or.inl rfl)
      · rw [Set.inter_self]; exact Or.inr (Or.inl rfl)
      · rw [jU_inter_jc] at hZsub ⊢
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    · rcases hW' with rfl | rfl | ⟨a', X', hX', rfl⟩
      · rw [Set.inter_comm, masterSig_inter_jc hX]; exact Or.inr (Or.inr ⟨a, X, hX, rfl⟩
    )
      · rw [Set.inter_comm, jU_inter_jc] at hZsub ⊢
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      · by_cases haa : a = a'
        · subst haa
          rw [jc_inter_jc_same] at hZsub ⊢
          rcases hZ with rfl | rfl | ⟨a₂, Z₂, hZ₂, rfl⟩
          · exact absurd (hZsub none_mem_masterSig) (by simp)
          · exact absurd (hZsub (show tu ∈ jU from rfl)) tu_not_mem_jc
          · by_cases ha₂ : a = a₂
            · subst ha₂
              exact Or.inr (Or.inr ⟨a, X ∩ X', V.inter_mem hX hX' hZ₂ (jc_subset_jc.mp
    hZsub), rfl⟩)
            · obtain ⟨t, ht⟩ := h Z₂ hZ₂
              exact absurd (hZsub (tc_mem_jc.mpr ht)) (tc_mem_jc_ne (Ne.symm ha₂))
        · rw [jc_inter_jc_ne haa] at hZsub ⊢
          obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)

@[simp] theorem sumSig_master {h : ∀ X, V.mem X → X.Nonempty} :
    (sumSig A V h).master = masterSig V := rfl

end SumSig

section SumSig

universe v

variable {β : Type v} {? : Type v}
```

```
variable (V : NeighborhoodSystem β)

variable {V}

variable {h : ∀ X, V.mem X → X.Nonempty}

theorem mem_subset_jU_inv {W : Set (SigTok A β)}
    (hW : (sumSig A V h).mem W) (hsub : W ⊆ jU) : W = jU := by
  rcases hW with rfl | rfl | ⟨a', X₂, hX₂, rfl⟩
  · exact absurd (hsub none_mem_masterSig) none_not_mem_jU
  · rfl
  · obtain ⟨t, ht⟩ := h X₂ hX₂
    exact absurd (hsub (tc_mem_jc.mpr ht)) tc_not_mem_jU

theorem mem_subset_jc_inv {W : Set (SigTok A β)} {a : A} {X : Set β}
    (hW : (sumSig A V h).mem W) (hsub : W ⊆ jc a X) : ∃ X₂, V.mem X₂ ∧ W = jc a X₂ :=
   by
  rcases hW with rfl | rfl | ⟨a', X₂, hX₂, rfl⟩
  · exact absurd (hsub none_mem_masterSig) none_not_mem_jc
  · exact absurd (hsub (show tu ∈ jU from rfl)) tu_not_mem_jc
  · obtain ⟨t, ht⟩ := h X₂ hX₂
    obtain ⟨t', heq, -⟩ := hsub (tc_mem_jc.mpr ht)
    obtain ⟨rfl, -⟩ := tc_injective heq
    exact ⟨X₂, hX₂, rfl⟩

def sinjC (a : A) (x : V.Element) : (sumSig A V h).Element where
  mem W := W = masterSig V ∨ ∃ X, V.mem X ∧ W = jc a X ∧ x.mem X
  sub := by
    rintro W (rfl | ⟨X, hX, rfl, -⟩)
    · exact Or.inl rfl
    · exact Or.inr (Or.inr ⟨a, X, hX, rfl⟩)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hx⟩) (rfl | ⟨X', hX', rfl, hx'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr ⟨X', hX', by rw [masterSig_inter_jc hX'], hx'⟩
    · exact Or.inr ⟨X, hX, by rw [Set.inter_comm, masterSig_inter_jc hX], hx⟩
    · exact Or.inr ⟨X ∩ X', x.sub (x.inter_mem hx hx'), jc_inter_jc_same a X X', x.
     inter_mem hx hx'⟩
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hx⟩) hW' hsub
    · exact Or.inl (eq_masterSig_of_subset hsub ((sumSig A V h).sub_master hW'))
    · rcases hW' with rfl | rfl | ⟨a', X', hX', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨t, ht⟩ := h X (x.sub hx)
        exact absurd (hsub (tc_mem_jc.mpr ht)) tc_not_mem_jU
      · obtain ⟨t, ht⟩ := h X (x.sub hx)
        obtain ⟨t', heq, -⟩ := hsub (tc_mem_jc.mpr ht)
        obtain ⟨rfl, -⟩ := tc_injective heq
        exact Or.inr ⟨X', hX', rfl, x.up_mem hx hX' (jc_subset_jc.mp hsub)⟩

def sinjU : (sumSig A V h).Element where
  mem W := W = masterSig V ∨ W = jU
  sub := by
    rintro W (rfl | rfl)
    · exact Or.inl rfl
```

```
    · exact Or.inr (Or.inl rfl)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | rfl) (rfl | rfl)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (by rw [masterSig_inter_jU])
    · exact Or.inr (by rw [Set.inter_comm, masterSig_inter_jU])
    · exact Or.inr (by rw [Set.inter_self])
  up_mem := by
    rintro W W' (rfl | rfl) hW' hsub
    · exact Or.inl (eq_masterSig_of_subset hsub ((sumSig A V h).sub_master hW'))
    · rcases hW' with rfl | rfl | ⟨a', X', hX', rfl⟩
      · exact Or.inl rfl
      · exact Or.inr rfl
      · exact absurd (hsub (show tu ∈ jU from rfl)) tu_not_mem_jc

@[simp] theorem sinjU_mem_jU : (sinjU (h := h)).mem (jU : Set (SigTok A β)) := Or.inr
    rfl

@[simp] theorem sinjC_mem_jc {a : A} {x : V.Element} {X : Set β} (hX : V.mem X) :
    (sinjC (h := h) a x).mem (jc a X) ↔ x.mem X := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hx⟩)
    · exact absurd (h0 ▸ none_mem_masterSig) none_not_mem_jc
    · rw [jc_injective heq]; exact hx
  · intro hx; exact Or.inr ⟨X, hX, rfl, hx⟩

theorem sinjC_mono {a : A} {x x' : V.Element} (hxle : x ≤ x') :
    sinjC (h := h) a x ≤ sinjC a x' := by
  rintro W (rfl | ⟨X, hX, rfl, hm⟩)
  · exact Or.inl rfl
  · exact Or.inr ⟨X, hX, rfl, hxle X hm⟩

end SumSig

section SumMapSig

universe v

variable {β₀ β₁ β₂ : Type v}
  {V₀ : NeighborhoodSystem β₀} {V₁ : NeighborhoodSystem β₁} {V₂ : NeighborhoodSystem β₂}
  {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
  {h₂ : ∀ Z, V₂.mem Z → Z.Nonempty}

def sumMapSig (f : ApproximableMap V₀ V₁) :
    ApproximableMap (sumSig A V₀ h₀) (sumSig A V₁ h₁) where
  rel W W' := (sumSig A V₀ h₀).mem W ∧ (sumSig A V₁ h₁).mem W' ∧
    (W' = masterSig V₁ ∨ (W = jU ∧ W' = jU) ∨
      ∃ a X Y', W = jc a X ∧ W' = jc a Y' ∧ f.rel X Y')
  rel_dom hr := hr.1
  rel_cod hr := hr.2.1
  master_rel := ⟨(sumSig A V₀ h₀).master_mem, (sumSig A V₁ h₁).master_mem, Or.inl rfl⟩
  inter_right := by
    rintro W W'₁ W'₂ ⟨hW, hW'₁, hd₁⟩ ⟨-, hW'₂, hd₂⟩
    have hmem : ∀ W'' : Set (SigTok A β₁),
        (W'' = masterSig V₁ ∨ (W = jU ∧ W'' = jU) ∨
```

```
        ∃ a X Y', W = jc a X ∧ W'' = jc a Y' ∧ f.rel X Y') → (sumSig A V₁ h₁).mem W
    '' := by
      rintro W'' (rfl | ⟨-, rfl⟩ | ⟨a, X, Y', -, rfl, hf⟩)
      · exact (sumSig A V₁ h₁).master_mem
      · exact Or.inr (Or.inl rfl)
      · exact Or.inr (Or.inr ⟨a, Y', f.rel_cod hf, rfl⟩)
    have key : (W'₁ ∩ W'₂ = masterSig V₁ ∨ (W = jU ∧ W'₁ ∩ W'₂ = jU) ∨
        ∃ a X Y', W = jc a X ∧ W'₁ ∩ W'₂ = jc a Y' ∧ f.rel X Y') := by
      rcases hd₁ with rfl | ⟨hWU, rfl⟩ | ⟨a, X, Y'₁, hWX, rfl, hf₁⟩
      · rw [Set.inter_eq_right.mpr (show W'₂ ⊆ masterSig V₁ from
          (sumSig A V₁ h₁).sub_master hW'₂)]; exact hd₂
      · rcases hd₂ with rfl | ⟨-, rfl⟩ | ⟨a, X, Y', hWX, rfl, hf⟩
        · rw [Set.inter_eq_left.mpr jU_subset_masterSig]; exact Or.inr (Or.inl ⟨hWU,
    rfl⟩)
        · rw [Set.inter_self]; exact Or.inr (Or.inl ⟨hWU, rfl⟩)
        · obtain ⟨t, ht⟩ := h₀ X (f.rel_dom hf)
          exact absurd ((hWX.symm.trans hWU) ▸ tc_mem_jc.mpr ht) tc_not_mem_jU
      · rcases hd₂ with rfl | ⟨hWU2, rfl⟩ | ⟨a', X', Y'₂, hWX', rfl, hf₂⟩
        · rw [Set.inter_eq_left.mpr (jc_subset_masterSig (f.rel_cod hf₁))]
          exact Or.inr (Or.inr ⟨a, X, Y'₁, hWX, rfl, hf₁⟩)
        · obtain ⟨t, ht⟩ := h₀ X (f.rel_dom hf₁)
          exact absurd ((hWX.symm.trans hWU2) ▸ tc_mem_jc.mpr ht) tc_not_mem_jU
        · obtain ⟨rfl, rfl⟩ := jc_eq_jc (h₀ X (f.rel_dom hf₁)) (hWX.symm.trans hWX')
          rw [jc_inter_jc_same]
          exact Or.inr (Or.inr ⟨a, X, Y'₁ ∩ Y'₂, hWX, rfl, f.inter_right hf₁ hf₂⟩)
    exact ⟨hW, hmem _ key, key⟩
  mono := by
    rintro W W₂ W' W'₂ ⟨hW, hW', hd⟩ hW₂W hW'W'₂ hW₂mem hW'₂mem
    refine ⟨hW₂mem, hW'₂mem, ?_⟩
    rcases hd with rfl | ⟨rfl, rfl⟩ | ⟨a, X, Y', rfl, rfl, hf⟩
    · exact Or.inl (eq_masterSig_of_subset hW'W'₂ ((sumSig A V₁ h₁).sub_master hW'₂mem))
    · have hW₂jU : W₂ = jU := mem_subset_jU_inv hW₂mem hW₂W
      rcases hW'₂mem with rfl | rfl | ⟨a', Y'₂, hY'₂, rfl⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨hW₂jU, rfl⟩)
      · exact absurd (hW'W'₂ (show tu ∈ jU from rfl)) tu_not_mem_jc
    · obtain ⟨X₂, hX₂, rfl⟩ := mem_subset_jc_inv hW₂mem hW₂W
      have hX₂X : X₂ ⊆ X := jc_subset_jc.mp hW₂W
      rcases hW'₂mem with rfl | rfl | ⟨a', Y'₂, hY'₂, rfl⟩
      · exact Or.inl rfl
      · obtain ⟨t, ht⟩ := h₁ Y' (f.rel_cod hf)
        exact absurd (hW'W'₂ (tc_mem_jc.mpr ht)) tc_not_mem_jU
      · obtain ⟨t, ht⟩ := h₁ Y' (f.rel_cod hf)
        obtain ⟨t', he, -⟩ := hW'W'₂ (tc_mem_jc.mpr ht)
        obtain ⟨rfl, -⟩ := tc_injective he
        exact Or.inr (Or.inr ⟨a, X₂, Y'₂, rfl, rfl,
          f.mono hf hX₂X (jc_subset_jc.mp hW'W'₂) hX₂ hY'₂⟩)

theorem isStrict_sumMapSig (f : ApproximableMap V₀ V₁) :
    IsStrict (sumMapSig (A := A) (h₀ := h₀) (h₁ := h₁) f) := by
  rintro W' ⟨-, -, hd⟩
  rcases hd with rfl | ⟨hWU, -⟩ | ⟨a, X, Y', hWX, -, -⟩
  · rfl
  · exact absurd (hWU ▸ (show (none : SigTok A β₀) ∈ (sumSig A V₀ h₀).master from
      none_mem_masterSig)) none_not_mem_jU
  · exact absurd (hWX ▸ (show (none : SigTok A β₀) ∈ (sumSig A V₀ h₀).master from
```

```
      none_mem_masterSig)) none_not_mem_jc

@[simp] theorem sumMapSig_rel {f : ApproximableMap V₀ V₁} {W W'} :
    (sumMapSig (A := A) (h₀ := h₀) (h₁ := h₁) f).rel W W' ↔
      (sumSig A V₀ h₀).mem W ∧ (sumSig A V₁ h₁).mem W' ∧
        (W' = masterSig V₁ ∨ (W = jU ∧ W' = jU) ∨
          ∃ a X Y', W = jc a X ∧ W' = jc a Y' ∧ f.rel X Y') := Iff.rfl

theorem sumMapSig_comp (g : ApproximableMap V₁ V₂) (f : ApproximableMap V₀ V₁) :
    sumMapSig (A := A) (h₀ := h₀) (h₁ := h₂) (g.comp f)
      = (sumMapSig (A := A) (h₀ := h₁) (h₁ := h₂) g).comp
          (sumMapSig (A := A) (h₀ := h₀) (h₁ := h₁) f) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  · rintro ⟨hW, hW'', hd⟩
    rcases hd with rfl | ⟨hWU, rfl⟩ | ⟨a, X, Z'', rfl, rfl, Y', hf, hg⟩
    · exact ⟨masterSig V₁, ⟨hW, (sumSig A V₁ h₁).master_mem, Or.inl rfl⟩,
        (sumSig A V₁ h₁).master_mem, hW'', Or.inl rfl⟩
    · exact ⟨jU, ⟨hW, Or.inr (Or.inl rfl), Or.inr (Or.inl ⟨hWU, rfl⟩)⟩,
        Or.inr (Or.inl rfl), hW'', Or.inr (Or.inl ⟨rfl, rfl⟩)⟩
    · exact ⟨jc a Y', ⟨hW, Or.inr (Or.inr ⟨a, Y', f.rel_cod hf, rfl⟩),
        Or.inr (Or.inr ⟨a, X, Y', rfl, rfl, hf⟩)⟩,
        Or.inr (Or.inr ⟨a, Y', f.rel_cod hf, rfl⟩)), hW'',
        Or.inr (Or.inr ⟨a, Y', Z'', rfl, rfl, hg⟩)⟩
  · rintro ⟨W', ⟨hW, hW', hdf⟩, -, hW'', hdg⟩
    refine ⟨hW, hW'', ?_⟩
    rcases hdg with rfl | ⟨hW'U, rfl⟩ | ⟨a, Y', Z'', hW'Y', rfl, hg⟩
    · exact Or.inl rfl
    · rcases hdf with rfl | ⟨hWU, -⟩ | ⟨a, X, Y'₀, -, hW'eq, hf⟩
      · exact absurd ((hW'U.symm) ▸ none_mem_masterSig) none_not_mem_jU
      · exact Or.inr (Or.inl ⟨hWU, rfl⟩)
      · obtain ⟨t, ht⟩ := h₁ Y'₀ (f.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'U) ▸ tc_mem_jc.mpr ht) tc_not_mem_jU
    · rcases hdf with rfl | ⟨-, hW'U⟩ | ⟨a', X, Y'₀, rfl, hW'eq, hf⟩
      · exact absurd ((hW'Y'.symm) ▸ none_mem_masterSig) none_not_mem_jc
      · obtain ⟨t, ht⟩ := h₁ Y' (g.rel_dom hg)
        exact absurd ((hW'U.symm.trans hW'Y') ▸ tc_mem_jc.mpr ht) tc_not_mem_jU
      · obtain ⟨rfl, rfl⟩ := jc_eq_jc (h₁ Y'₀ (f.rel_cod hf)) (hW'eq.symm.trans hW'Y')
        exact Or.inr (Or.inr ⟨a', X, Z'', rfl, rfl, ⟨Y'₀, hf, hg⟩⟩)

theorem sumMapSig_id :
    sumMapSig (A := A) (V₀ := V₀) (V₁ := V₀) (h₀ := h₀) (h₁ := h₀) (idMap V₀)
      = idMap (sumSig A V₀ h₀) := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  · rintro ⟨hW, hW', hd⟩
    refine ⟨hW, hW', ?_⟩
    rcases hd with rfl | ⟨rfl, rfl⟩ | ⟨a, X, Y', rfl, rfl, -, -, hXY⟩
    · exact (sumSig A V₀ h₀).sub_master hW
    · exact subset_rfl
    · exact jc_subset_jc.mpr hXY
  · rintro ⟨hW, hW', hsub⟩
    refine ⟨hW, hW', ?_⟩
    rcases hW with rfl | rfl | ⟨a, X, hX, rfl⟩
```

```
    · exact Or.inl (eq_masterSig_of_subset hsub ((sumSig A V₀ h₀).sub_master hW'))
    · rcases hW' with rfl | rfl | ⟨a', X', hX', rfl⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨rfl, rfl⟩)
      · exact absurd (hsub (show tu ∈ jU from rfl)) tu_not_mem_jc
    · rcases hW' with rfl | rfl | ⟨a', X', hX', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨t, ht⟩ := h₀ X hX; exact absurd (hsub (tc_mem_jc.mpr ht)) tc_not_mem_jU
      · obtain ⟨t, ht⟩ := h₀ X hX
        obtain ⟨t', he, -⟩ := hsub (tc_mem_jc.mpr ht)
        obtain ⟨rfl, -⟩ := tc_injective he
        exact Or.inr (Or.inr ⟨a, X, X', rfl, rfl, hX, hX', jc_subset_jc.mp hsub⟩)

theorem sumMapSig_sinjC (f : ApproximableMap V₀ V₁) (a : A) (x : V₀.Element) :
    (sumMapSig (A := A) (h₀ := h₀) (h₁ := h₁) f).toElementMap (sinjC (h := h₀) a x)
      = sinjC (h := h₁) a (f.toElementMap x) := by
  apply NeighborhoodSystem.Element.ext
  intro W'
  constructor
  · rintro ⟨U, hU, -, -, hd⟩
    rcases hd with rfl | ⟨hUjU, rfl⟩ | ⟨a', X, Y', hUj, rfl, hf⟩
    · exact Or.inl rfl
    · rcases hU with hUm | ⟨X₀, hX₀, hUeq, hx⟩
      · exact absurd ((hUm.symm.trans hUjU) ▸ none_mem_masterSig) none_not_mem_jU
      · obtain ⟨t, ht⟩ := h₀ X₀ hX₀
        exact absurd ((hUeq.symm.trans hUjU) ▸ tc_mem_jc.mpr ht) tc_not_mem_jU
    · rcases hU with hUm | ⟨X₀, hX₀, hUeq, hx⟩
      · exact absurd ((hUm.symm.trans hUj) ▸ none_mem_masterSig) none_not_mem_jc
      · obtain ⟨rfl, rfl⟩ := jc_eq_jc (h₀ X₀ hX₀) (hUeq.symm.trans hUj)
        exact Or.inr ⟨Y', f.rel_cod hf, rfl, ⟨X₀, hx, hf⟩⟩
  · rintro (rfl | ⟨Y', hY', rfl, hm⟩)
    · exact ⟨masterSig V₀, Or.inl rfl, (sumSig A V₀ h₀).master_mem,
        (sumSig A V₁ h₁).master_mem, Or.inl rfl⟩
    · obtain ⟨X, hx, hf⟩ := hm
      exact ⟨jc a X, Or.inr ⟨X, x.sub hx, rfl, hx⟩, Or.inr (Or.inr ⟨a, X, x.sub hx, rfl⟩
    ),
        Or.inr (Or.inr ⟨a, Y', f.rel_cod hf, rfl⟩), Or.inr (Or.inr ⟨a, X, Y', rfl, rfl,
     hf⟩)⟩

theorem sumMapSig_sinjU (f : ApproximableMap V₀ V₁) :
    (sumMapSig (A := A) (h₀ := h₀) (h₁ := h₁) f).toElementMap (sinjU (h := h₀))
      = sinjU (h := h₁) := by
  apply NeighborhoodSystem.Element.ext
  intro W'
  constructor
  · rintro ⟨U, hU, -, -, hd⟩
    rcases hd with rfl | ⟨-, rfl⟩ | ⟨a, X, Y', hUj, rfl, hf⟩
    · exact Or.inl rfl
    · exact Or.inr rfl
    · rcases hU with hUm | rfl
      · exact absurd ((hUm.symm.trans hUj) ▸ none_mem_masterSig) none_not_mem_jc
      · exact absurd (hUj ▸ (show tu ∈ jU from rfl)) tu_not_mem_jc
  · rintro (rfl | rfl)
    · exact ⟨masterSig V₀, Or.inl rfl, (sumSig A V₀ h₀).master_mem,
        (sumSig A V₁ h₁).master_mem, Or.inl rfl⟩
    · exact ⟨jU, Or.inr rfl, Or.inr (Or.inl rfl), Or.inr (Or.inl rfl),
```

```
        Or.inr (Or.inl ⟨rfl, rfl⟩)⟩

end SumMapSig

section SumSig

universe v

variable {β : Type v} {? : Type v}

variable (V : NeighborhoodSystem β)

variable {V}

theorem sumSig_nonempty {h : ∀ X, V.mem X → X.Nonempty} :
    ∀ W, (sumSig A V h).mem W → W.Nonempty := by
  rintro W (rfl | rfl | ⟨a, X, hX, rfl⟩)
  · exact ⟨none, none_mem_masterSig⟩
  · exact jU_nonempty
  · exact jc_nonempty (h X hX)

end SumSig

abbrev tsigObj (A : Type) [DecidableEq A] (D : StrictDomainObj.{0}) : StrictDomainObj
    .{0} where
  carrier := SigTok A D.carrier
  sys := sumSig A D.sys D.nonempty
  nonempty := sumSig_nonempty

@[simp] theorem tsigObj_sys (A : Type) [DecidableEq A] (D : StrictDomainObj.{0}) :
    (tsigObj A D).sys = sumSig A D.sys D.nonempty := rfl

def tsigMapHom (A : Type) [DecidableEq A] {D E : StrictDomainObj.{0}} (f : (D ⟶ E)) :
    ((tsigObj A D) ⟶ (tsigObj A E)) :=
  ⟨sumMapSig (A := A) (h₀ := D.nonempty) (h₁ := E.nonempty) f.1, isStrict_sumMapSig _⟩

@[simp] theorem tsigMapHom_val (A : Type) [DecidableEq A] {D E : StrictDomainObj.{0}}
    (f : (D ⟶ E)) :
    (tsigMapHom A f).1 = sumMapSig (A := A) (h₀ := D.nonempty) (h₁ := E.nonempty) f.1
    := rfl

abbrev Tsig (A : Type) [DecidableEq A] : Endofunctor StrictDomainObj.{0} where
  obj := tsigObj A
  map := tsigMapHom A
  map_id _ := Subtype.ext sumMapSig_id
  map_comp {_ _ _} g f := Subtype.ext (sumMapSig_comp g.1 f.1)

@[simp] theorem Tsig_obj (A : Type) [DecidableEq A] (D : StrictDomainObj.{0}) :
    (Tsig A).obj D = tsigObj A D := rfl

@[simp] theorem Tsig_map_val (A : Type) [DecidableEq A] {D E : StrictDomainObj.{0}}
    (f : (D ⟶ E)) :
    ((Tsig A).map f).1 = sumMapSig (A := A) (h₀ := D.nonempty) (h₁ := E.nonempty) f.1
    := rfl

section Iso
```

```
variable {A : Type} [DecidableEq A] [Inhabited A]


def embA (a : A) (X : Set (Strn A)) : Set (Strn A) := {w | ∃ w', w = a :: w' ∧ w' ∈ X}

omit [DecidableEq A] [Inhabited A] in

@[simp] theorem mem_embA {a : A} {X : Set (Strn A)} {w : Strn A} :
    w ∈ embA a X ↔ ∃ w', w = a :: w' ∧ w' ∈ X := Iff.rfl

abbrev CCn (A : Type) [DecidableEq A] : NeighborhoodSystem (SigTok A (Strn A)) :=
  sumSig A (Cn A) Cn_nonempty

omit [DecidableEq A] [Inhabited A] in

theorem embA_eq_prependN (a : A) (X : Set (Strn A)) : embA a X = prependN [a] X := by
  ext w
  simp only [mem_embA, mem_prependN]
  constructor
  · rintro ⟨w', rfl, hX⟩; exact ⟨w', hX, rfl⟩
  · rintro ⟨t, hX, rfl⟩; exact ⟨t, rfl, hX⟩

omit [DecidableEq A] [Inhabited A] in

theorem embA_coneN (a : A) (σ : Strn A) : embA a (coneN σ) = coneN (a :: σ) := by
  rw [embA_eq_prependN, prependN_coneN]; rfl

omit [DecidableEq A] [Inhabited A] in

theorem embA_inter (a : A) (X X' : Set (Strn A)) : embA a X ∩ embA a X' = embA a (X ∩
    X') := by
  ext w
  simp only [Set.mem_inter_iff, mem_embA]
  constructor
  · rintro ⟨⟨w', rfl, hX⟩, w'', heq, hX'⟩
    rw [List.cons.injEq] at heq; obtain ⟨-, rfl⟩ := heq; exact ⟨w', rfl, hX, hX'⟩
  · rintro ⟨w', rfl, hX, hX'⟩; exact ⟨⟨w', rfl, hX⟩, ⟨w', rfl, hX'⟩⟩

omit [DecidableEq A] [Inhabited A] in

theorem embA_inter_ne {a a' : A} (h : a = a') (X Y : Set (Strn A)) :
    embA a X ∩ embA a' Y = ∅ := by
  ext w
  simp only [Set.mem_inter_iff, mem_embA, Set.mem_empty_iff_false, iff_false, not_and]
  rintro ⟨w', rfl, -⟩ ⟨w'', heq, -⟩
  rw [List.cons.injEq] at heq; exact h heq.1

omit [DecidableEq A] [Inhabited A] in

theorem embA_singleton (a : A) (σ : Strn A) : embA a ({σ} : Set (Strn A)) = {a :: σ}
    := by
  rw [embA_eq_prependN, prependN_singleton]; rfl

omit [DecidableEq A] [Inhabited A] in
```

```
theorem embA_subset {a : A} {X X' : Set (Strn A)} : embA a X ⊆ embA a X' ↔ X ⊆ X' :=
    by
  constructor
  · intro h w' hw'
    obtain ⟨w'', heq, hX'⟩ := h ⟨w', rfl, hw'⟩
    rw [List.cons.injEq] at heq; obtain ⟨-, rfl⟩ := heq; exact hX'
  · rintro h w ⟨w', rfl, hX⟩; exact ⟨w', rfl, h hX⟩

omit [DecidableEq A] [Inhabited A] in

theorem embA_injective {a : A} {X X' : Set (Strn A)} (h : embA a X = embA a X') : X =
    X' :=
  Set.Subset.antisymm (embA_subset.mp h.subset) (embA_subset.mp h.symm.subset)

omit [DecidableEq A] [Inhabited A] in

theorem memCn_cases {W : Set (Strn A)} (hW : memCn W) :
    W = Set.univ ∨ W = ({[]} : Set (Strn A)) ∨ ∃ a X, memCn X ∧ W = embA a X := by
  rcases hW with ⟨σ, rfl⟩ | ⟨σ, rfl⟩
  · cases σ with
    | nil => exact Or.inl coneN_nil
    | cons a σ' => exact Or.inr (Or.inr ⟨a, coneN σ', memCn_coneN σ', (embA_coneN a σ')
    .symm⟩)
  · cases σ with
    | nil => exact Or.inr (Or.inl rfl)
    | cons a σ' => exact Or.inr (Or.inr ⟨a, {σ'}, memCn_singleton σ', (embA_singleton a
     σ').symm⟩)

omit [Inhabited A] in

omit [DecidableEq A] in

theorem memCn_embA (a : A) {X : Set (Strn A)} (hX : memCn X) : memCn (embA a X) := by
  rw [embA_eq_prependN]; exact memCn_prependN [a] hX

omit [DecidableEq A] [Inhabited A] in

theorem memCn_embA_inv {a : A} {W : Set (Strn A)} (h : memCn (embA a W)) : memCn W :=
    by
  rcases h with ⟨σ, hσ⟩ | ⟨σ, hσ⟩
  · have hmem : σ ∈ embA a W := hσ ▸ (show σ ∈ coneN σ from List.prefix_rfl)
    obtain ⟨w', rfl, -⟩ := hmem
    rw [← embA_coneN] at hσ; rw [embA_injective hσ]; exact memCn_coneN w'
  · have hmem : σ ∈ embA a W := hσ ▸ (Set.mem_singleton_iff.mpr rfl : σ ∈ ({σ} : Set (
    Strn A)))
    obtain ⟨w', rfl, -⟩ := hmem
    rw [← embA_singleton] at hσ; rw [embA_injective hσ]; exact memCn_singleton w'

omit [DecidableEq A] [Inhabited A] in

theorem nil_not_mem_embA {a : A} {X : Set (Strn A)} : ([] : Strn A) ∈ embA a X := by
  rintro ⟨w', heq, -⟩; exact absurd heq (by simp)

omit [DecidableEq A] [Inhabited A] in

theorem embA_ne_univ (a : A) (X : Set (Strn A)) : embA a X = Set.univ := by
```

```
  intro h; exact nil_not_mem_embA (X := X) (a := a) (by rw [h]; trivial)

omit [DecidableEq A] [Inhabited A] in

theorem singleton_nil_inter_embA (a : A) (X : Set (Strn A)) :
    (({[]} : Set (Strn A)) ∩ embA a X) = ∅ := by
  ext w
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, mem_embA, Set.
    mem_empty_iff_false,
    iff_false, not_and]
  rintro rfl ⟨w', heq, -⟩; exact absurd heq (by simp)

omit [DecidableEq A] [Inhabited A] in

theorem singleton_nil_ne_embA (a : A) (X : Set (Strn A)) :
    ({[]} : Set (Strn A)) = embA a X := by
  intro h
  exact nil_not_mem_embA (h ▶ (Set.mem_singleton_iff.mpr rfl : ([] : Strn A) ∈ ({[]} :
     Set (Strn A))))

omit [DecidableEq A] in

theorem singleton_nil_ne_univ : ({[]} : Set (Strn A)) = Set.univ := by
  intro h
  have hmem : ([default] : Strn A) ∈ ({[]} : Set (Strn A)) := by rw [h]; trivial
  rw [Set.mem_singleton_iff] at hmem; exact absurd hmem (by simp)

omit [Inhabited A] in

theorem sumSig_mem_jc_inv {a : A} {X : Set (Strn A)} (h : (CCn A).mem (jc a X)) :
    (Cn A).mem X := by
  rcases h with h0 | hU | ⟨a', X', hX', heq⟩
  · exact absurd (h0 ▶ none_mem_masterSig) none_not_mem_jc
  · have : (tu : SigTok A (Strn A)) ∈ jc a X := by rw [hU]; rfl
    exact absurd this tu_not_mem_jc
  · by_cases haa : a = a'
    · subst haa; rw [jc_injective heq]; exact hX'
    · obtain ⟨t, ht⟩ := Cn_nonempty X' hX'
      exact absurd (heq.symm ▶ (tc_mem_jc.mpr ht)) (tc_mem_jc_ne (Ne.symm haa))

def toCC (x : (Cn A).Element) : (CCn A).Element where
  mem W := W = masterSig (Cn A)
    ∨ (W = jU ∧ x.mem ({[]} : Set (Strn A)))
    ∨ (∃ a X, (Cn A).mem X ∧ W = jc a X ∧ x.mem (embA a X))
  sub := by
    rintro W (rfl | ⟨rfl, -⟩ | ⟨a, X, hX, rfl, -⟩)
    · exact Or.inl rfl
    · exact Or.inr (Or.inl rfl)
    · exact Or.inr (Or.inr ⟨a, X, hX, rfl⟩)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨rfl, hzU⟩ | ⟨a, X, hX, rfl, hzF⟩)
      (rfl | ⟨rfl, hzU'⟩ | ⟨a', X', hX', rfl, hzF'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (Or.inl ⟨by rw [masterSig_inter_jU], hzU'⟩)
    · exact Or.inr (Or.inr ⟨a', X', hX', by rw [masterSig_inter_jc hX'], hzF'⟩)
```

```
    · exact Or.inr (Or.inl ⟨by rw [Set.inter_comm, masterSig_inter_jU], hzU⟩)
    · exact Or.inr (Or.inl ⟨by rw [Set.inter_self], hzU⟩)
    · exfalso
      have hx := x.inter_mem hzU hzF'; rw [singleton_nil_inter_embA] at hx
      obtain ⟨t, ht⟩ := Cn_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
    · exact Or.inr (Or.inr ⟨a, X, hX, by rw [Set.inter_comm, masterSig_inter_jc hX],
    hzF⟩)
    · exfalso
      have hx := x.inter_mem hzF hzU'; rw [Set.inter_comm, singleton_nil_inter_embA]
    at hx
      obtain ⟨t, ht⟩ := Cn_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
    · by_cases haa : a = a'
      · subst haa
        have hx := x.inter_mem hzF hzF'; rw [embA_inter] at hx
        exact Or.inr (Or.inr ⟨a, X ∩ X', memCn_embA_inv (x.sub hx), jc_inter_jc_same a
    X X', hx⟩)
      · exfalso
        have hx := x.inter_mem hzF hzF'; rw [embA_inter_ne haa] at hx
        obtain ⟨t, ht⟩ := Cn_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
  up_mem := by
    rintro W W' (rfl | ⟨rfl, hzU⟩ | ⟨a, X, hX, rfl, hzF⟩) hW' hsub
    · exact Or.inl (eq_masterSig_of_subset hsub ((CCn A).sub_master hW'))
    · rcases hW' with rfl | rfl | ⟨a', X', hX', rfl⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨rfl, hzU⟩)
      · exact absurd (hsub (show tu ∈ jU from rfl)) tu_not_mem_jc
    · rcases hW' with rfl | rfl | ⟨a', X', hX', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨t, ht⟩ := Cn_nonempty X hX
        exact absurd (hsub (tc_mem_jc.mpr ht)) tc_not_mem_jU
      · obtain ⟨t, ht⟩ := Cn_nonempty X hX
        obtain ⟨t', he, -⟩ := hsub (tc_mem_jc.mpr ht)
        obtain ⟨rfl, -⟩ := tc_injective he
        exact Or.inr (Or.inr ⟨a, X', hX', rfl,
          x.up_mem hzF (memCn_embA a hX') (embA_subset.mpr (jc_subset_jc.mp hsub))⟩)

omit [Inhabited A] in

@[simp] theorem toCC_mem_jU {x : (Cn A).Element} :
    (toCC x).mem (jU : Set (SigTok A (Strn A))) ↔ x.mem ({[]} : Set (Strn A)) := by
  constructor
  · rintro (h0 | ⟨-, hz⟩ | ⟨a', X', hX', heq, hz⟩)
    · exact absurd (h0.symm ▸ none_mem_masterSig) none_not_mem_jU
    · exact hz
    · exact absurd (heq ▸ (show tu ∈ jU from rfl)) tu_not_mem_jc
  · intro hz; exact Or.inr (Or.inl ⟨rfl, hz⟩)

omit [Inhabited A] in

@[simp] theorem toCC_mem_jc {x : (Cn A).Element} {a : A} {X : Set (Strn A)} (hX : (Cn
    A).mem X) :
    (toCC x).mem (jc a X) ↔ x.mem (embA a X) := by
  constructor
  · rintro (h0 | ⟨heq, hz⟩ | ⟨a', X', hX', heqj, hz⟩)
    · exact absurd (h0 ▸ none_mem_masterSig) none_not_mem_jc
    · obtain ⟨t, ht⟩ := Cn_nonempty X hX
```

```
      exact absurd (heq ▶ (tc_mem_jc.mpr ht)) tc_not_mem_jU
    · obtain ⟨rfl, rfl⟩ := jc_eq_jc (Cn_nonempty X hX) heqj
      exact hz
  · intro hz; exact Or.inr (Or.inr ⟨a, X, hX, rfl, hz⟩)

omit [DecidableEq A] [Inhabited A] in

theorem embA_eq_embA {a a' : A} {X X' : Set (Strn A)} (hXne : X.Nonempty)
    (h : embA a X = embA a' X') : a = a' ∧ X = X' := by
  obtain ⟨w', hw'⟩ := hXne
  have hmem : (a :: w') ∈ embA a X := ⟨w', rfl, hw'⟩
  rw [h] at hmem
  obtain ⟨u, he, -⟩ := hmem
  rw [List.cons.injEq] at he; obtain ⟨rfl, -⟩ := he
  exact ⟨rfl, embA_injective h⟩

def fromCC (s : (CCn A).Element) : (Cn A).Element where
  mem W := W = Set.univ
    ∨ (W = ({[]} : Set (Strn A)) ∧ s.mem jU)
    ∨ (∃ a X, (Cn A).mem X ∧ W = embA a X ∧ s.mem (jc a X))
  sub := by
    rintro W (rfl | ⟨rfl, -⟩ | ⟨a, X, hX, rfl, -⟩)
    · exact Or.inl ⟨[], coneN_nil.symm⟩
    · exact memCn_singleton []
    · exact memCn_embA a hX
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨rfl, hsU⟩ | ⟨a, X, hX, rfl, hsF⟩)
      (rfl | ⟨rfl, hsU'⟩ | ⟨a', X', hX', rfl, hsF'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (Or.inl ⟨by rw [Set.univ_inter], hsU'⟩)
    · exact Or.inr (Or.inr ⟨a', X', hX', by rw [Set.univ_inter], hsF'⟩)
    · exact Or.inr (Or.inl ⟨by rw [Set.inter_univ], hsU⟩)
    · exact Or.inr (Or.inl ⟨by rw [Set.inter_self], hsU⟩)
    · exfalso
      have hs := s.inter_mem hsU hsF'; rw [jU_inter_jc] at hs
      obtain ⟨t, ht⟩ := sumSig_nonempty _ (s.sub hs); exact Set.notMem_empty t ht
    · exact Or.inr (Or.inr ⟨a, X, hX, by rw [Set.inter_univ], hsF⟩)
    · exfalso
      have hs := s.inter_mem hsF hsU'; rw [Set.inter_comm, jU_inter_jc] at hs
      obtain ⟨t, ht⟩ := sumSig_nonempty _ (s.sub hs); exact Set.notMem_empty t ht
    · by_cases haa : a = a'
      · subst haa
        have hs := s.inter_mem hsF hsF'; rw [jc_inter_jc_same] at hs
        exact Or.inr (Or.inr ⟨a, X ∩ X', sumSig_mem_jc_inv (s.sub hs), embA_inter a X X
    ', hs⟩)
      · exfalso
        have hs := s.inter_mem hsF hsF'; rw [jc_inter_jc_ne haa] at hs
        obtain ⟨t, ht⟩ := sumSig_nonempty _ (s.sub hs); exact Set.notMem_empty t ht
  up_mem := by
    rintro W W' (rfl | ⟨rfl, hsU⟩ | ⟨a, X, hX, rfl, hsF⟩) hW' hsub
    · exact Or.inl (Set.univ_subset_iff.mp hsub)
    · rcases memCn_cases hW' with rfl | rfl | ⟨a', X', hX', rfl⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨rfl, hsU⟩)
      · exact absurd (hsub (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embA
```

```
    · rcases memCn_cases hW' with rfl | rfl | ⟨a', X', hX', rfl⟩
      · exact Or.inl rfl
      · obtain ⟨t, ht⟩ := Cn_nonempty X hX
        have hm := hsub (⟨t, rfl, ht⟩ : (a :: t) ∈ embA a X)
        rw [Set.mem_singleton_iff] at hm; exact absurd hm (by simp)
      · obtain ⟨t, ht⟩ := Cn_nonempty X hX
        obtain ⟨w', he, -⟩ := hsub (⟨t, rfl, ht⟩ : (a :: t) ∈ embA a X)
        rw [List.cons.injEq] at he; obtain ⟨rfl, -⟩ := he
        refine Or.inr (Or.inr ⟨a, X', hX', rfl, ?_⟩)
        exact s.up_mem hsF (Or.inr (Or.inr ⟨a, X', hX', rfl⟩))
          (jc_subset_jc.mpr (embA_subset.mp hsub))

@[simp] theorem fromCC_mem_nil {s : (CCn A).Element} :
    (fromCC s).mem ({[]} : Set (Strn A)) ↔ s.mem (jU : Set (SigTok A (Strn A))) := by
  constructor
  · rintro (h0 | ⟨-, hs⟩ | ⟨a', X', hX', heq, hs⟩)
    · exact absurd h0 singleton_nil_ne_univ
    · exact hs
    · exact absurd heq (singleton_nil_ne_embA a' X')
  · intro hs; exact Or.inr (Or.inl ⟨rfl, hs⟩)

omit [Inhabited A] in

@[simp] theorem fromCC_mem_embA {s : (CCn A).Element} {a : A} {X : Set (Strn A)} (hX :
     (Cn A).mem X) :
    (fromCC s).mem (embA a X) ↔ s.mem (jc a X) := by
  constructor
  · rintro (h0 | ⟨heq, hs⟩ | ⟨a', X', hX', heqj, hs⟩)
    · exact absurd h0 (embA_ne_univ a X)
    · exact absurd heq.symm (singleton_nil_ne_embA a X)
    · obtain ⟨rfl, rfl⟩ := embA_eq_embA (Cn_nonempty X hX) heqj
      exact hs
  · intro hs; exact Or.inr (Or.inr ⟨a, X, hX, rfl, hs⟩)

omit [Inhabited A] in

theorem fromCC_toCC (x : (Cn A).Element) : fromCC (toCC x) = x := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨rfl, hs⟩ | ⟨a, X, hX, rfl, hs⟩)
    · exact x.master_mem
    · exact toCC_mem_jU.mp hs
    · exact (toCC_mem_jc hX).mp hs
  · intro hW
    rcases memCn_cases (x.sub hW) with rfl | rfl | ⟨a, X, hX, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨rfl, toCC_mem_jU.mpr hW⟩)
    · exact Or.inr (Or.inr ⟨a, X, hX, rfl, (toCC_mem_jc hX).mpr hW⟩)

theorem toCC_fromCC (s : (CCn A).Element) : toCC (fromCC s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨rfl, hs⟩ | ⟨a, X, hX, rfl, hs⟩)
    · exact s.master_mem
```

```
    · exact fromCC_mem_nil.mp hs
    · exact (fromCC_mem_embA hX).mp hs
  · intro hW
    rcases s.sub hW with rfl | rfl | ⟨a, X, hX, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨rfl, fromCC_mem_nil.mpr hW⟩)
    · exact Or.inr (Or.inr ⟨a, X, hX, rfl, (fromCC_mem_embA hX).mpr hW⟩)

def ccEquiv : (Cn A).Element ≃o (CCn A).Element where
  toFun := toCC
  invFun := fromCC
  left_inv := fromCC_toCC
  right_inv := toCC_fromCC
  map_rel_iff' := by
    intro x x'
    constructor
    · intro hle W hW
      rcases memCn_cases (x.sub hW) with rfl | rfl | ⟨a, X, hX, rfl⟩
      · exact x'.master_mem
      · exact toCC_mem_jU.mp (hle _ (Or.inr (Or.inl ⟨rfl, hW⟩)))
      · exact (toCC_mem_jc hX).mp (hle _ (Or.inr (Or.inr ⟨a, X, hX, rfl, hW⟩)))
    · intro hle W hW
      rcases hW with rfl | ⟨rfl, hz⟩ | ⟨a, X, hX, rfl, hz⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨rfl, hle _ hz⟩)
      · exact Or.inr (Or.inr ⟨a, X, hX, rfl, hle _ hz⟩)

@[simp] theorem ccEquiv_apply (x : (Cn A).Element) : ccEquiv x = toCC x := rfl

end Iso

section Algebra

variable {A : Type} [DecidableEq A] [Inhabited A]

abbrev Cnobj (A : Type) [DecidableEq A] : StrictDomainObj.{0} := ⟨Strn A, Cn A,
    Cn_nonempty⟩

@[simp] theorem Cnobj_sys (A : Type) [DecidableEq A] : (Cnobj A).sys = Cn A := rfl

abbrev cnStr : (((Tsig A).obj (Cnobj A)) ⟶ (Cnobj A)) :=
  ⟨ofIso ccEquiv.symm, isStrict_ofIso _⟩

abbrev Cnalg (A : Type) [DecidableEq A] [Inhabited A] : TAlgebra (Tsig A) := ⟨Cnobj A,
    cnStr⟩

end Algebra

section Iso

variable {A : Type} [DecidableEq A] [Inhabited A]

omit [Inhabited A] in

theorem consMapN_mem_embA {a : A} {z : (Cn A).Element} {X : Set (Strn A)} (hX : (Cn A)
    .mem X) :
```

```
    ((consMapN a).toElementMap z).mem (embA a X) ↔ z.mem X := by
  constructor
  · rintro ⟨X', hzX', -, -, hsub⟩
    rw [← embA_eq_prependN] at hsub
    exact z.up_mem hzX' hX (embA_subset.mp hsub)
  · intro hz
    refine ⟨X, hz, z.sub hz, memCn_embA a hX, ?_⟩
    rw [← embA_eq_prependN]

omit [Inhabited A] in

theorem consMapN_not_mem_embA_ne {a c : A} (hac : a = c) {z : (Cn A).Element} {X : Set
     (Strn A)} :
    ¬ ((consMapN a).toElementMap z).mem (embA c X) := by
  rintro ⟨X', hzX', hX'mem, -, hsub⟩
  obtain ⟨t, ht⟩ := Cn_nonempty X' hX'mem
  rw [← embA_eq_prependN] at hsub
  obtain ⟨w, hw, -⟩ := hsub ⟨t, rfl, ht⟩
  rw [List.cons.injEq] at hw; exact hac hw.1

omit [Inhabited A] in

theorem consMapN_not_mem_nil {a : A} {z : (Cn A).Element} :
    ¬ ((consMapN a).toElementMap z).mem ({[]} : Set (Strn A)) := by
  rintro ⟨X', hzX', hX'mem, -, hsub⟩
  obtain ⟨t, ht⟩ := Cn_nonempty X' hX'mem
  rw [← embA_eq_prependN] at hsub
  have hmem := hsub ⟨t, rfl, ht⟩
  rw [Set.mem_singleton_iff] at hmem; exact absurd hmem (by simp)

end Iso


section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

def descE : B.carrier.sys.Element :=
  B.str.1.toElementMap (sinjU (h := B.carrier.nonempty))

def descF (a : A) (y : B.carrier.sys.Element) : B.carrier.sys.Element :=
  B.str.1.toElementMap (sinjC (h := B.carrier.nonempty) a y)

omit [Inhabited A] in

theorem descF_mono (a : A) {y y' : B.carrier.sys.Element} (h : y ≤ y') :
    descF B a y ≤ descF B a y' :=
  B.str.1.toElementMap_mono (sinjC_mono h)

def descVal (z : B.carrier.sys.Element) : Strn A → B.carrier.sys.Element
  | [] => z
  | a :: σ => descF B a (descVal z σ)

omit [Inhabited A] in

theorem descVal_append (z : B.carrier.sys.Element) (σ ? : Strn A) :
```

```
    descVal B z (σ ++ ?) = descVal B (descVal B z ?) σ := by
  induction σ with
  | nil => rfl
  | cons a σ ih => exact congrArg (descF B a) ih

omit [Inhabited A] in

theorem descVal_mono_z {z z' : B.carrier.sys.Element} (h : z ≤ z') :
    ∀ σ, descVal B z σ ≤ descVal B z' σ
  | [] => h
  | _ :: σ => descF_mono B _ (descVal_mono_z h σ)

omit [Inhabited A] in

theorem descMap_hcone {σ τ : Strn A} (h : σ <+: τ) :
    descVal B B.carrier.sys.bot σ ≤ descVal B B.carrier.sys.bot τ := by
  obtain ⟨?, rfl⟩ := h
  rw [descVal_append]
  exact descVal_mono_z B (B.carrier.sys.bot_le _) σ

omit [Inhabited A] in

theorem descMap_hsing {σ τ : Strn A} (h : σ <+: τ) :
    descVal B B.carrier.sys.bot σ ≤ descVal B (descE B) τ := by
  obtain ⟨?, rfl⟩ := h
  rw [descVal_append]
  exact descVal_mono_z B (B.carrier.sys.bot_le _) σ

end Initial

section Lift

variable {A : Type} [DecidableEq A] [Inhabited A] {β : Type}

omit [Inhabited A] in

theorem map_ext_Cn {V : NeighborhoodSystem β} {f g : ApproximableMap (Cn A) V}
    (hbot : ∀ σ, f.toElementMap (strBotN σ) = g.toElementMap (strBotN σ))
    (helem : ∀ σ, f.toElementMap (strElemN σ) = g.toElementMap (strElemN σ)) : f = g
   := by
  apply eq_of_toElementMap_principal
  intro X hX
  obtain (⟨σ, rfl⟩ | ⟨σ, rfl⟩) := (Cn_mem.mp hX)
  · exact hbot σ
  · exact helem σ

omit [DecidableEq A] in

theorem not_coneN_subset_singleton (τ σ : Strn A) : ¬ coneN τ ⊆ ({σ} : Set (Strn A))
    := by
  intro h
  have h1 : τ ∈ ({σ} : Set (Strn A)) := h (show τ ∈ coneN τ from List.prefix_rfl)
  have h2 : (τ ++ [default]) ∈ ({σ} : Set (Strn A)) := h (List.prefix_append τ [
    default])
  rw [Set.mem_singleton_iff] at h1 h2
  have : τ = τ ++ [default] := h1.trans h2.symm
```

```
  simp at this

omit [DecidableEq A] in

theorem coneN_ne_singleton (τ σ : Strn A) : coneN τ = ({σ} : Set (Strn A)) := fun h =>
  not_coneN_subset_singleton τ σ (h ▶ subset_rfl)

def liftCn (V : NeighborhoodSystem β) (coneVal singVal : Strn A → V.Element)
    (hcone : ∀ {σ τ : Strn A}, σ <+: τ → coneVal σ ≤ coneVal τ)
    (hsing : ∀ {σ τ : Strn A}, σ <+: τ → coneVal σ ≤ singVal τ) :
    ApproximableMap (Cn A) V where
  rel X Y := (∃ σ, X = coneN σ ∧ (coneVal σ).mem Y) ∨ (∃ σ, X = {σ} ∧ (singVal σ).mem
    Y)
  rel_dom := by
    rintro X Y (⟨σ, rfl, _⟩ | ⟨σ, rfl, _⟩)
    · exact memCn_coneN σ
    · exact memCn_singleton σ
  rel_cod := by
    rintro X Y (⟨σ, _, hY⟩ | ⟨σ, _, hY⟩)
    · exact (coneVal σ).sub hY
    · exact (singVal σ).sub hY
  master_rel := by
    refine Or.inl ⟨[], ?_, (coneVal []).master_mem⟩
    rw [Cn_master]; exact coneN_nil.symm
  inter_right := by
    rintro X Y Y' (⟨σ, rfl, hY⟩ | ⟨σ, rfl, hY⟩) (⟨σ', hX', hY'⟩ | ⟨σ', hX', hY'⟩)
    · have hσσ : σ = σ' := coneN_injective hX'
      subst hσσ
      exact Or.inl ⟨σ, rfl, (coneVal σ).inter_mem hY hY'⟩
    · exact absurd hX' (coneN_ne_singleton σ σ')
    · exact absurd hX'.symm (coneN_ne_singleton σ' σ)
    · have hσσ : σ = σ' := by rw [Set.singleton_eq_singleton_iff] at hX'; exact hX'
      subst hσσ
      exact Or.inr ⟨σ, rfl, (singVal σ).inter_mem hY hY'⟩
  mono := by
    rintro X X' Y Y' (⟨σ, rfl, hY⟩ | ⟨σ, rfl, hY⟩) hX'X hYY' hX' hY'
    · rcases hX' with ⟨τ, rfl⟩ | ⟨τ, rfl⟩
      · have hpre : σ <+: τ := coneN_subset_coneN.mp hX'X
        exact Or.inl ⟨τ, rfl, (coneVal τ).up_mem (hcone hpre Y hY) hY' hYY'⟩
      · have hpre : σ <+: τ := singleton_subset_coneN.mp hX'X
        exact Or.inr ⟨τ, rfl, (singVal τ).up_mem (hsing hpre Y hY) hY' hYY'⟩
    · rcases hX' with ⟨τ, rfl⟩ | ⟨τ, rfl⟩
      · exact absurd hX'X (not_coneN_subset_singleton τ σ)
      · have hτσ : τ = σ := by
          have hmem := Set.singleton_subset_iff.mp hX'X
          rwa [Set.mem_singleton_iff] at hmem
        subst hτσ
        exact Or.inr ⟨τ, rfl, (singVal τ).up_mem hY hY' hYY'⟩

end Lift

section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

def descMap : ApproximableMap (Cn A) B.carrier.sys :=
```

```
  liftCn B.carrier.sys (descVal B B.carrier.sys.bot) (descVal B (descE B))
    (fun {_ _} => descMap_hcone B) (fun {_ _} => descMap_hsing B)

end Initial

section Lift

variable {A : Type} [DecidableEq A] [Inhabited A] {β : Type}

theorem liftCn_strBot (V : NeighborhoodSystem β) (coneVal singVal : Strn A → V.Element
    )
    (hcone : ∀ {σ τ : Strn A}, σ <+: τ → coneVal σ ≤ coneVal τ)
    (hsing : ∀ {σ τ : Strn A}, σ <+: τ → coneVal σ ≤ singVal τ) (σ : Strn A) :
    (liftCn V coneVal singVal hcone hsing).toElementMap (strBotN σ) = coneVal σ := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  constructor
  · rintro ⟨X, ⟨_, hsub⟩, hrel⟩
    rcases hrel with ⟨σ', hXcone, hY⟩ | ⟨σ', hXsing, hY⟩
    · have hpre : σ' <+: σ := coneN_subset_coneN.mp (hXcone ▸ hsub)
      exact hcone hpre Y hY
    · exact absurd (hXsing ▸ hsub) (not_coneN_subset_singleton σ σ')
  · intro hY
    exact ⟨coneN σ, ⟨memCn_coneN σ, subset_rfl⟩, Or.inl ⟨σ, rfl, hY⟩⟩

end Lift

section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

@[simp] theorem descMap_strBot (σ : Strn A) :
    (descMap B).toElementMap (strBotN σ) = descVal B B.carrier.sys.bot σ :=
  liftCn_strBot _ _ _ _ _ σ

end Initial

section Lift

variable {A : Type} [DecidableEq A] [Inhabited A] {β : Type}

theorem liftCn_strElem (V : NeighborhoodSystem β) (coneVal singVal : Strn A → V.
    Element)
    (hcone : ∀ {σ τ : Strn A}, σ <+: τ → coneVal σ ≤ coneVal τ)
    (hsing : ∀ {σ τ : Strn A}, σ <+: τ → coneVal σ ≤ singVal τ) (σ : Strn A) :
    (liftCn V coneVal singVal hcone hsing).toElementMap (strElemN σ) = singVal σ := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  constructor
  · rintro ⟨X, ⟨_, hsub⟩, hrel⟩
    rcases hrel with ⟨σ', hXcone, hY⟩ | ⟨σ', hXsing, hY⟩
    · have hpre : σ' <+: σ := by
        apply singleton_subset_coneN.mp; rw [← hXcone]; exact hsub
      exact hsing hpre Y hY
    · have hσσ' : σ = σ' := by
        have hmem := Set.singleton_subset_iff.mp (hXsing ▸ hsub)
```

```
        rwa [Set.mem_singleton_iff] at hmem
      subst hσσ'; exact hY
  · intro hY
    exact ⟨{σ}, ⟨memCn_singleton σ, subset_rfl⟩, Or.inr ⟨σ, rfl, hY⟩⟩

end Lift

section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

@[simp] theorem descMap_strElem (σ : Strn A) :
    (descMap B).toElementMap (strElemN σ) = descVal B (descE B) σ :=
  liftCn_strElem _ _ _ _ _ σ

omit [Inhabited A] in

theorem Cn_bot_eq_strBotN_nil : (Cn A).bot = strBotN ([] : Strn A) := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  show ((Cn A).mem Y ∧ (Cn A).master ⊆ Y) ↔ ((Cn A).mem Y ∧ coneN [] ⊆ Y)
  rw [Cn_master, coneN_nil]

theorem ccEquiv_comp_symm :
    (ofIso (ccEquiv (A := A))).comp (ofIso ccEquiv.symm) = idMap (CCn A) := by
  apply ext_of_toElementMap
  intro s
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap]
  exact ccEquiv.apply_symm_apply s

theorem ccEquiv_symm_comp :
    (ofIso (ccEquiv (A := A)).symm).comp (ofIso ccEquiv) = idMap (Cn A) := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap]
  exact ccEquiv.symm_apply_apply x

theorem descMap_strict : IsStrict (descMap B) := by
  rw [isStrict_iff_apply_bot, Cn_bot_eq_strBotN_nil, descMap_strBot]
  rfl

def descStrict : ((Cnobj A) ⟶ B.carrier) := ⟨descMap B, descMap_strict B⟩

theorem genKeyBot (g : ApproximableMap (Cn A) B.carrier.sys) :
    B.str.1.toElementMap ((sumMapSig (A := A) (h₀ := Cn_nonempty) (h₁ := B.carrier.
    nonempty)
        g).toElementMap (toCC (strBotN ([] : Strn A))))
      = B.carrier.sys.bot := by
  have hb : toCC (strBotN ([] : Strn A)) = (CCn A).bot := by
    rw [← Cn_bot_eq_strBotN_nil, ← ccEquiv_apply]; exact ccEquiv.map_bot
  rw [hb, isStrict_iff_apply_bot.mp (isStrict_sumMapSig (A := A) (h₀ := Cn_nonempty)
    (h₁ := B.carrier.nonempty) g)]
  exact isStrict_iff_apply_bot.mp B.str.2

end Initial
```

```
section Iso

variable {A : Type} [DecidableEq A] [Inhabited A]
omit [Inhabited A] in

theorem toCC_consMapN (a : A) (z : (Cn A).Element) :
    toCC ((consMapN a).toElementMap z) = sinjC (h := Cn_nonempty) a z := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨rfl, hz⟩ | ⟨c, X, hX, rfl, hz⟩)
    · exact Or.inl rfl
    · exact absurd hz consMapN_not_mem_nil
    · by_cases hac : a = c
      · subst hac; exact Or.inr ⟨X, hX, rfl, (consMapN_mem_embA hX).mp hz⟩
      · exact absurd hz (consMapN_not_mem_embA_ne hac)
  · rintro (rfl | ⟨X, hX, rfl, hm⟩)
    · exact Or.inl rfl
    · exact Or.inr (Or.inr ⟨a, X, hX, rfl, (consMapN_mem_embA hX).mpr hm⟩)

end Iso

section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

omit [Inhabited A] in

theorem genKey (g : ApproximableMap (Cn A) B.carrier.sys) (a : A) (w : (Cn A).Element)
     :
    B.str.1.toElementMap ((sumMapSig (A := A) (h₀ := Cn_nonempty) (h₁ := B.carrier.
    nonempty)
        g).toElementMap (toCC ((consMapN a).toElementMap w)))
      = descF B a (g.toElementMap w) := by
  rw [toCC_consMapN, sumMapSig_sinjC]; rfl

end Initial

section Iso

variable {A : Type} [DecidableEq A] [Inhabited A]
omit [Inhabited A] in

theorem toCC_strElemN_nil :
    toCC (strElemN ([] : Strn A)) = sinjU (h := Cn_nonempty) := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨rfl, hz⟩ | ⟨a, X, hX, rfl, hz⟩)
    · exact Or.inl rfl
    · exact Or.inr rfl
    · exact absurd (hz.2 (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embA
  · rintro (rfl | rfl)
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨rfl, memCn_singleton [], subset_rfl⟩)
```

```
end Iso

section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

omit [Inhabited A] in

theorem genKey0 (g : ApproximableMap (Cn A) B.carrier.sys) :
    B.str.1.toElementMap ((sumMapSig (A := A) (h₀ := Cn_nonempty) (h₁ := B.carrier.
    nonempty)
        g).toElementMap (toCC (strElemN ([] : Strn A))))
      = descE B := by
  rw [toCC_strElemN_nil, sumMapSig_sinjU]; rfl

theorem descMap_satisfiesRec :
    descMap B = (B.str.1.comp (sumMapSig (A := A) (h₀ := Cn_nonempty) (h₁ := B.carrier.
    nonempty)
        (descMap B))).comp (ofIso ccEquiv) := by
  apply map_ext_Cn
  · intro σ
    rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, descMap_strBot]
    cases σ with
    | nil => exact (genKeyBot B (descMap B)).symm
    | cons a σ =>
      rw [ccEquiv_apply, ← consMapN_strBot]
      have h := genKey B (descMap B) a (strBotN σ)
      rw [descMap_strBot] at h
      exact h.symm
  · intro σ
    rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, descMap_strElem]
    cases σ with
    | nil => exact (genKey0 B (descMap B)).symm
    | cons a σ =>
      rw [ccEquiv_apply, ← consMapN_strElem]
      have h := genKey B (descMap B) a (strElemN σ)
      rw [descMap_strElem] at h
      exact h.symm

theorem descComm : (descMap B).comp (ofIso ccEquiv.symm)
    = B.str.1.comp (sumMapSig (A := A) (h₀ := Cn_nonempty) (h₁ := B.carrier.nonempty)
        (descMap B)) := by
  conv_lhs => rw [descMap_satisfiesRec B]
  rw [comp_assoc, ccEquiv_comp_symm, comp_idMap]

def descAlgHom : AlgHom (Cnalg A) B where
  hom := descStrict B
  comm := by
    apply Subtype.ext
    simp only [StrictDomainObj.comp_val]
    exact descComm B

theorem rec_determines (g : ApproximableMap (Cn A) B.carrier.sys)
    (hg : g = (B.str.1.comp (sumMapSig (A := A) (h₀ := Cn_nonempty) (h₁ := B.carrier.
    nonempty)
        g)).comp (ofIso ccEquiv)) :
```

```
    g = descMap B := by
  have hbot : ∀ σ, g.toElementMap (strBotN σ) = descVal B B.carrier.sys.bot σ := by
    intro σ
    induction σ with
    | nil =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, ccEquiv_apply]
      exact genKeyBot B g
    | cons a σ ih =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, ccEquiv_apply,
        ← consMapN_strBot]
      have h := genKey B g a (strBotN σ)
      rw [ih] at h
      exact h
  have helem : ∀ σ, g.toElementMap (strElemN σ) = descVal B (descE B) σ := by
    intro σ
    induction σ with
    | nil =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, ccEquiv_apply]
      exact genKey0 B g
    | cons a σ ih =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, ccEquiv_apply,
        ← consMapN_strElem]
      have h := genKey B g a (strElemN σ)
      rw [ih] at h
      exact h
  apply map_ext_Cn
  · intro σ; rw [hbot, descMap_strBot]
  · intro σ; rw [helem, descMap_strElem]

theorem descAlgHom_uniq (h' : AlgHom (Cnalg A) B) : h' = descAlgHom B := by
  obtain ⟨hom, comm⟩ := h'
  have hg : hom.1 = descMap B := by
    refine rec_determines B hom.1 ?_
    have hc : hom.1.comp (ofIso ccEquiv.symm)
        = B.str.1.comp (sumMapSig (A := A) (h₀ := Cn_nonempty) (h₁ := B.carrier.
    nonempty) hom.1) := by
      exact congrArg Subtype.val comm
    have h2 := congrArg (fun m => m.comp (ofIso ccEquiv)) hc
    simp only at h2
    rw [comp_assoc] at h2
    erw [ccEquiv_symm_comp, comp_idMap] at h2
    exact h2
  have hhom : hom = descStrict B := Subtype.ext hg
  subst hhom
  rfl

end Initial

def CnisInitial (A : Type) [DecidableEq A] [Inhabited A] : IsInitial (Cnalg A) where

  desc := descAlgHom
  uniq := fun B h => descAlgHom_uniq B h
```

```
def CfinIsInitial (n : ℕ) : IsInitial (Cnalg (Fin (n + 1))) := CnisInitial (Fin (n +
    1))

theorem Cn_domain_equation (A : Type) [DecidableEq A] [Inhabited A] :
    Cn A ≅? CCn A := ⟨ccEquiv⟩

end

section
variable {A : Type} [DecidableEq A]

theorem Cfin_domain_equation (n : ℕ) : Cn (Fin (n + 1)) ≅? CCn (Fin (n + 1)) :=
  Cn_domain_equation (Fin (n + 1))
end

section
variable {A : Type}

section Iso

theorem embA_ne {a a' : A} (h : a = a') {X Y : Set (Strn A)} (hX : X.Nonempty) :
    embA a X = embA a' Y := by
  intro heq
  obtain ⟨w', hw'⟩ := hX
  have hmem : (a :: w') ∈ embA a' Y := heq ▸ (⟨w', rfl, hw'⟩ : (a :: w') ∈ embA a X)
  obtain ⟨w'', he, -⟩ := hmem
  rw [List.cons.injEq] at he; exact h he.1

end Iso

section Iso

theorem embA_nonempty {a : A} {X : Set (Strn A)} (hX : X.Nonempty) : (embA a X).
    Nonempty := by
  obtain ⟨w', hw'⟩ := hX; exact ⟨a :: w', w', rfl, hw'⟩

end Iso

end

end DomainTheory.NeighbourhoodSystems.CAlgebraGen
```

4.7.40 Exercise 6.17 — algebras for sequence domains

NeighbourhoodSystems/DomainEquations/Exercise_6_17.lean

Exercise_6_17.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.cAlgebra
public import DomainTheory.NeighbourhoodSystems.DomainEquations.cAlgebraGen

@[expose] public section
```

```
open DomainTheory.NeighbourhoodSystems

open SeqDom SeqEqC BitDom DomainTheory.NeighbourhoodSystems.SeqPeano SeqNegMerge
open CAlgebraGen

/-- Scott's Exercise 6.17: the initial `Tc`-algebra is unique. -/
theorem example_617 (B : TAlgebra Tc) (h' : AlgHom Calg B) : h' = descAlgHom B :=
  @descAlgHom_uniq B h'

/-- Scott's Exercise 6.17 (general `A`): `Cn A` and `CCn A` are isomorphic domains. -/
theorem example_617_gen (A : Type) [DecidableEq A] [Inhabited A] : Cn A ≅? CCn A :=
  @Cn_domain_equation A _ _
```

EXERCISE 6.17. What are the algebras for which C is initial? If A of 6.2 is a generalization of B, what is the corresponding generalization of C? Prove that it exists and explain what are the algebras involved.

4.7.41 iterProdInitial — D^∞ initial

iterProdInitial.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.strictDomainObj
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.iterSys
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodFunctor
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofIso
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Theorem_6_16
public import DomainTheory.NeighbourhoodSystems.DomainEquations.initialAlgebra
public import DomainTheory.NeighbourhoodSystems.DomainEquations.talgebra
public import DomainTheory.NeighbourhoodSystems.DomainEquations.endofunctor
public import DomainTheory.NeighbourhoodSystems.DomainEquations.cAlgebra
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.supChain

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

section
namespace IterProdInitial

variable {α : Type*} {V : NeighborhoodSystem α}

def consSeq (a : V.Element) (b : (iterSys V).Element) : ℕ → V.Element :=
  fun i => Nat.casesOn i a (fun k => component b k)

@[simp] theorem consSeq_zero (a : V.Element) (b : (iterSys V).Element) : consSeq a b 0
    = a := rfl

@[simp] theorem consSeq_succ (a : V.Element) (b : (iterSys V).Element) (k : ℕ) :
    consSeq a b (k + 1) = component b k := rfl

theorem iterBot_eq : (iterSys V).bot = ofSeq (fun _ : ℕ => V.bot) := by
```

```
  apply Element.ext
  intro W
  rw [mem_bot, mem_ofSeq]
  constructor
  · rintro rfl
    refine ⟨(iterSys V).master_mem, fun i => ?_⟩
    rw [fiber_iterSys_master, mem_bot]
  · rintro ⟨_, hfib⟩
    apply eq_of_fiber_eq
    intro i
    have hi := hfib i
    rw [fiber_iterSys_master]
    rwa [mem_bot] at hi

@[simp] theorem component_bot (n : ℕ) : component (iterSys V).bot n = V.bot := by
  rw [iterBot_eq, component_ofSeq]

theorem chain_le_of_succ {β : Type*} {W : NeighborhoodSystem β} {a : ℕ → W.Element}
    (h : ∀ n, a n ≤ a (n + 1)) {i j : ℕ} (hij : i ≤ j) : a i ≤ a j := by
  induction hij with
  | refl => exact le_rfl
  | step _ ih => exact ih.trans (h _)

section Target

variable {? : Type*} {E : NeighborhoodSystem ?}

theorem constBot_le (f : ApproximableMap (iterSys V) E) :
    constMap (iterSys V) E.bot ≤ f := by
  intro X Y hrel
  obtain ⟨hX, hY⟩ := hrel
  rw [mem_bot] at hY; subst hY
  exact f.rel_master hX

end Target

def imap (V : NeighborhoodSystem α) : ApproximableMap (prod V (iterSys V)) (iterSys V)
    :=
  ofIso (iterProdIso V).symm

theorem isStrict_imap : IsStrict (imap V) := isStrict_ofIso _

theorem iterProdIso_apply (z : (iterSys V).Element) :
    iterProdIso V z = pair (component z 0) (ofSeq (fun n => component z (n + 1))) :=
    rfl

theorem iterProdIso_symm_pair (a : V.Element) (b : (iterSys V).Element) :
    (iterProdIso V).symm (pair a b) = ofSeq (consSeq a b) := by
  have hkey : iterProdIso V (ofSeq (consSeq a b)) = pair a b := by
    rw [iterProdIso_apply]
    congr 1
    · rw [component_ofSeq, consSeq_zero]
    · have : (fun n => component (ofSeq (consSeq a b)) (n + 1)) = fun n => component b
    n := by
        funext n; rw [component_ofSeq, consSeq_succ]
      rw [this, ofSeq_component]
```

```
  rw [← hkey, OrderIso.symm_apply_apply]

end IterProdInitial
end

namespace IterProdInitial

theorem iterSys_nonempty {α : Type*} {V : NeighborhoodSystem α}
    (h : ∀ X, V.mem X → X.Nonempty) :
    ∀ W, (iterSys V).mem W → W.Nonempty := by
  rintro W ⟨hfib, _⟩
  obtain ⟨a, ha⟩ := h (fiber W 0) (hfib 0)
  exact ⟨(0, a), ha⟩

end IterProdInitial

section
namespace IterProdInitial

variable {α : Type*} {V : NeighborhoodSystem α}

def iterObj (Dom : StrictDomainObj.{w}) : StrictDomainObj.{w} where
  carrier := ℕ × Dom.carrier
  sys := iterSys Dom.sys
  nonempty := iterSys_nonempty Dom.nonempty

def jmap (V : NeighborhoodSystem α) : ApproximableMap (iterSys V) (prod V (iterSys V))
    :=
  ofIso (iterProdIso V)

section Target

variable {? : Type*} {E : NeighborhoodSystem ?}

def descOp (k : ApproximableMap (prod V E) E) (f : ApproximableMap (iterSys V) E) :
    ApproximableMap (iterSys V) E :=
  k.comp ((prodMap (idMap V) f).comp (jmap V))

theorem descOp_apply (k : ApproximableMap (prod V E) E) (f : ApproximableMap (iterSys
    V) E)
    (z : (iterSys V).Element) :
    (descOp k f).toElementMap z
      = k.toElementMap (pair (component z 0)
          (f.toElementMap (ofSeq (fun n => component z (n + 1))))) := by
  simp only [descOp, jmap]
  rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, iterProdIso_apply,
    toElementMap_prodMap_pair, toElementMap_idMap]

def descSeq (k : ApproximableMap (prod V E) E) : ℕ → ApproximableMap (iterSys V) E
  | 0 => constMap (iterSys V) E.bot
  | (n + 1) => descOp k (descSeq k n)

end Target

section Uniq
```

```
variable {? : Type*} {E : NeighborhoodSystem ?}

theorem gcomp_rho_succ (k : ApproximableMap (prod V E) E) (g : ApproximableMap (
    iterSys V) E)
    (hc : g.comp (imap V) = k.comp (prodMap (idMap V) g)) (n : ℕ) :
    g.comp (descSeq (imap V) (n + 1)) = descOp k (g.comp (descSeq (imap V) n)) := by
  show g.comp ((imap V).comp ((prodMap (idMap V) (descSeq (imap V) n)).comp (jmap V)))
     = k.comp ((prodMap (idMap V) (g.comp (descSeq (imap V) n))).comp (jmap V))
  rw [← comp_assoc, hc, comp_assoc]
  congr 1
  rw [← comp_assoc, ← prodMap_comp, idMap_comp]

theorem gcomp_rho_zero (g : ApproximableMap (iterSys V) E) (hg : IsStrict g) :
    g.comp (descSeq (imap V) 0) = constMap (iterSys V) E.bot := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_constMap,
    show descSeq (imap V) 0 = constMap (iterSys V) (iterSys V).bot from rfl,
    toElementMap_constMap, isStrict_iff_apply_bot.mp hg]

end Uniq

theorem jmap_comp_imap : (jmap V).comp (imap V) = idMap (prod V (iterSys V)) := by
  apply ext_of_toElementMap
  intro w
  simp only [jmap, imap]
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap,
    OrderIso.apply_symm_apply]

section Target

variable {? : Type*} {E : NeighborhoodSystem ?}

def kHead (k : ApproximableMap (prod V E) E) (a : V.Element) :
    ApproximableMap E E :=
  k.comp (paired (constMap E a) (idMap E))

theorem kHead_apply (k : ApproximableMap (prod V E) E) (a : V.Element) (u : E.Element)
    :
    (kHead k a).toElementMap u = k.toElementMap (pair a u) := by
  rw [kHead, toElementMap_comp, toElementMap_paired, toElementMap_constMap,
    toElementMap_idMap]

end Target

theorem pair_bot {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
    :
    pair V₀.bot V₁.bot = (prod V₀ V₁).bot := by
  apply Element.ext
  intro W
  rw [mem_bot, prod_master]
  constructor
  · rintro ⟨X, Y, hX, hY, rfl⟩
    rw [mem_bot] at hX hY; subst hX; subst hY; rfl
  · rintro rfl
    exact ⟨V₀.master, V₁.master, V₀.bot.master_mem, V₁.bot.master_mem, rfl⟩
```

```
section Target

variable {? : Type*} {E : NeighborhoodSystem ?}

theorem descSeq_strict (k : ApproximableMap (prod V E) E) (hk : IsStrict k) :
    ∀ n, IsStrict (descSeq k n)
  | 0 => isStrict_constBot
  | (n + 1) => by
      have ih := descSeq_strict k hk n
      rw [isStrict_iff_apply_bot] at ih ⊢
      rw [show descSeq k (n + 1) = descOp k (descSeq k n) from rfl, descOp_apply]
      have htl : ofSeq (fun m => component (iterSys V).bot (m + 1)) = (iterSys V).bot
    := by
        have hconst : (fun m => component (iterSys V).bot (m + 1)) = (fun _ => V.bot)
    := by
          funext m; rw [component_bot]
        rw [hconst, ← iterBot_eq]
      rw [component_bot, htl, ih, pair_bot]
      exact isStrict_iff_apply_bot.mp hk

end Target

theorem isStrict_prodMap {α β α' β' : Type*} {V₀ : NeighborhoodSystem α} {V₁ :
    NeighborhoodSystem β}
    {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
    {f₀ : ApproximableMap V₀ V₀'} {f₁ : ApproximableMap V₁ V₁'}
    (hf₀ : IsStrict f₀) (hf₁ : IsStrict f₁) : IsStrict (prodMap f₀ f₁) := by
  rw [isStrict_iff_apply_bot, show (prod V₀ V₁).bot = pair V₀.bot V₁.bot from pair_bot.
    symm,
    toElementMap_prodMap_pair, isStrict_iff_apply_bot.mp hf₀, isStrict_iff_apply_bot.mp
     hf₁]
  exact pair_bot

end IterProdInitial
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

namespace IterProdInitial

variable {α : Type*} {V : NeighborhoodSystem α}

theorem prodMap_idMap_mono {? : Type*} {E : NeighborhoodSystem ?}
    {f f' : ApproximableMap (iterSys V) E} (h : f ≤ f') :
    prodMap (idMap V) f ≤ prodMap (idMap V) f' := by
  intro W P hrel
  simp only [prodMap, paired_rel] at hrel ⊢
  exact ⟨hrel.1, hrel.2.1, comp_mono_gen h le_rfl _ _ hrel.2.2⟩

section Target

variable {? : Type*} {E : NeighborhoodSystem ?}
```

```
theorem descOp_mono (k : ApproximableMap (prod V E) E) {f f' : ApproximableMap (
    iterSys V) E}
    (h : f ≤ f') : descOp k f ≤ descOp k f' :=
  comp_mono_gen le_rfl (comp_mono_gen (prodMap_idMap_mono h) le_rfl)

theorem descSeq_mono_succ (k : ApproximableMap (prod V E) E) (n : ℕ) :
    descSeq k n ≤ descSeq k (n + 1) := by
  induction n with
  | zero => exact constBot_le _
  | succ m ih => exact descOp_mono k ih

theorem descSeq_mono (k : ApproximableMap (prod V E) E) {n m : ℕ} (h : n ≤ m) :
    descSeq k n ≤ descSeq k m := by
  induction h with
  | refl => exact le_rfl
  | step _ ih => exact ih.trans (descSeq_mono_succ k _)

theorem descSeq_dir (k : ApproximableMap (prod V E) E) :
    ∀ i j, ∃ l, (∀ X Y, (descSeq k i).rel X Y → (descSeq k l).rel X Y) ∧
      (∀ X Y, (descSeq k j).rel X Y → (descSeq k l).rel X Y) :=
  fun i j => ⟨max i j, fun _ _ => descSeq_mono k (le_max_left i j) _ _,
    fun _ _ => descSeq_mono k (le_max_right i j) _ _⟩

def descMap (k : ApproximableMap (prod V E) E) : ApproximableMap (iterSys V) E :=
  iSupMap (descSeq k) (descSeq_dir k)

theorem descMap_toElementMap (k : ApproximableMap (prod V E) E) (z : (iterSys V).
    Element)
    {Y : Set ?} :
    ((descMap k).toElementMap z).mem Y ↔ ∃ n, ((descSeq k n).toElementMap z).mem Y :=
  mem_toElementMap_iSupMap (descSeq k) (descSeq_dir k) z

theorem descMap_strict (k : ApproximableMap (prod V E) E) (hk : IsStrict k) :
    IsStrict (descMap k) := by
  rw [isStrict_iff_apply_bot]
  apply Element.ext
  intro Y
  rw [mem_bot, descMap_toElementMap]
  constructor
  · rintro ⟨n, hn⟩
    have hs := descSeq_strict k hk n
    rw [isStrict_iff_apply_bot] at hs
    rw [hs, mem_bot] at hn
    exact hn
  · rintro rfl
    have hs := descSeq_strict k hk 0
    rw [isStrict_iff_apply_bot] at hs
    exact ⟨0, by rw [hs]; exact E.bot.master_mem⟩

end Target

end IterProdInitial
end

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct
```

```
namespace IterProdInitial

theorem prod_nonempty {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ :
    NeighborhoodSystem β}
    (h₀ : ∀ X, V₀.mem X → X.Nonempty) (_h₁ : ∀ Y, V₁.mem Y → Y.Nonempty) :
    ∀ W, (prod V₀ V₁).mem W → W.Nonempty := by
  rintro W ⟨X, Y, hX, _, rfl⟩
  obtain ⟨a, ha⟩ := h₀ X hX
  exact ⟨Sum.inl a, prodNbhd.mem_inl.mpr ha⟩

end IterProdInitial

section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

namespace IterProdInitial

variable {α : Type*} {V : NeighborhoodSystem α}

def prodObj (Dom X : StrictDomainObj.{w}) : StrictDomainObj.{w} where
  carrier := Dom.carrier ? X.carrier
  sys := prod Dom.sys X.sys
  nonempty := prod_nonempty Dom.nonempty X.nonempty

def prodMapHom (Dom : StrictDomainObj.{w}) {X Y : StrictDomainObj.{w}} (f : (X ⟶ Y))
    :
    ((prodObj Dom X) ⟶ (prodObj Dom Y)) :=
  ⟨prodMap (idMap Dom.sys) f.1, isStrict_prodMap isStrict_idMap f.2⟩

def prodFunctor (Dom : StrictDomainObj.{w}) : Endofunctor StrictDomainObj.{w} where
  obj := prodObj Dom
  map := prodMapHom Dom
  map_id X := Subtype.ext (by
    show prodMap (idMap Dom.sys) (idMap X.sys) = idMap (prod Dom.sys X.sys)
    exact prodMap_id)
  map_comp {X Y Z} g f := Subtype.ext (by
    show prodMap (idMap Dom.sys) (g.1.comp f.1)
       = (prodMap (idMap Dom.sys) g.1).comp (prodMap (idMap Dom.sys) f.1)
    have h := prodMap_comp (idMap Dom.sys) (idMap Dom.sys) g.1 f.1
    rw [idMap_comp] at h
    exact h)

def iterAlg (Dom : StrictDomainObj.{w}) : TAlgebra (prodFunctor Dom) where
  carrier := iterObj Dom
  str := ⟨imap Dom.sys, isStrict_imap⟩

theorem rho_apply (n : ℕ) (z : (iterSys V).Element) :
    (descSeq (imap V) n).toElementMap z
      = ofSeq (fun i => if i < n then component z i else V.bot) := by
  induction n generalizing z with
  | zero =>
    show (constMap (iterSys V) (iterSys V).bot).toElementMap z = _
    rw [toElementMap_constMap, iterBot_eq]
    congr 1
  | succ n ih =>
```

```
    rw [show descSeq (imap V) (n + 1) = descOp (imap V) (descSeq (imap V) n) from rfl,
      descOp_apply, ih]
    simp only [imap]
    rw [toElementMap_ofIso, iterProdIso_symm_pair]
    congr 1
    funext j
    cases j with
    | zero =>
      show component z 0 = if (0 : ℕ) < n + 1 then component z 0 else V.bot
      rw [ite_eq_left (Nat.zero_lt_succ n)]
    | succ k =>
      rw [consSeq_succ, component_ofSeq, component_ofSeq]
      show (if k < n then component z (k + 1) else V.bot)
         = if k + 1 < n + 1 then component z (k + 1) else V.bot
      by_cases h : k < n
      · rw [ite_eq_left h, ite_eq_left (Nat.succ_lt_succ h)]
      · rw [ite_eq_right h, ite_eq_right (fun hc => h (Nat.lt_of_succ_lt_succ hc))]

theorem iSupRho_eq_id : descMap (imap V) = idMap (iterSys V) := by
  apply ext_of_toElementMap
  intro z
  rw [toElementMap_idMap]
  apply Element.ext
  intro Y
  rw [descMap_toElementMap]
  constructor
  · rintro ⟨n, hn⟩
    have hle : (descSeq (imap V) n).toElementMap z ≤ z := by
      rw [rho_apply]
      have hz : z = ofSeq (fun i => component z i) := (ofSeq_component z).symm
      conv_rhs => rw [hz]
      apply ofSeq_mono
      intro i
      show (if i < n then component z i else V.bot) ≤ component z i
      split
      · exact le_rfl
      · exact V.bot_le _
    exact hle Y hn
  · intro hzY
    have hY : (iterSys V).mem Y := z.sub hzY
    obtain ⟨N, hN⟩ := hY.2
    have hcomp : ∀ i, (component z i).mem (fiber Y i) := by
      have h := hzY
      rw [← ofSeq_component z] at h
      rw [mem_ofSeq] at h
      exact h.2
    refine ⟨N, ?_⟩
    rw [rho_apply, mem_ofSeq]
    refine ⟨hY, fun i => ?_⟩

    show (if i < N then component z i else V.bot).mem (fiber Y i)
    by_cases h : i < N
    · rw [ite_eq_left h]; exact hcomp i
    · rw [ite_eq_right h, hN i (not_lt.mp h)]; exact V.bot.master_mem

section Uniq
```

```
variable {? : Type*} {E : NeighborhoodSystem ?}

theorem comm_unique (k : ApproximableMap (prod V E) E)
    {g g' : ApproximableMap (iterSys V) E} (hg : IsStrict g) (hg' : IsStrict g')
    (hc : g.comp (imap V) = k.comp (prodMap (idMap V) g))
    (hc' : g'.comp (imap V) = k.comp (prodMap (idMap V) g')) : g = g' := by
  have hindep : ∀ n, g.comp (descSeq (imap V) n) = g'.comp (descSeq (imap V) n) := by
    intro n
    induction n with
    | zero => rw [gcomp_rho_zero g hg, gcomp_rho_zero g' hg']
    | succ m ih => rw [gcomp_rho_succ k g hc m, gcomp_rho_succ k g' hc' m, ih]
  have key : g.comp (descMap (imap V)) = g'.comp (descMap (imap V)) := by
    apply ApproximableMap.ext
    intro X Z
    simp only [descMap, comp_rel]
    constructor
    · rintro ⟨Y, ⟨i, hXY⟩, hYZ⟩
      have hg : (g.comp (descSeq (imap V) i)).rel X Z := ⟨Y, hXY, hYZ⟩
      rw [hindep i] at hg
      obtain ⟨Y', hXY', hYZ'⟩ := hg
      exact ⟨Y', ⟨i, hXY'⟩, hYZ'⟩
    · rintro ⟨Y, ⟨i, hXY⟩, hYZ⟩
      have hg' : (g'.comp (descSeq (imap V) i)).rel X Z := ⟨Y, hXY, hYZ⟩
      rw [← hindep i] at hg'
      obtain ⟨Y', hXY', hYZ'⟩ := hg'
      exact ⟨Y', ⟨i, hXY'⟩, hYZ'⟩
  rw [iSupRho_eq_id] at key
  rwa [comp_idMap, comp_idMap] at key

end Uniq

theorem succChainDir {β : Type*} {W : NeighborhoodSystem β} (a : ℕ → W.Element)
    (h : ∀ n, a n ≤ a (n + 1)) : ∀ i j, ∃ l, a i ≤ a l ∧ a j ≤ a l :=
  fun i j => ⟨max i j, chain_le_of_succ h (le_max_left i j), chain_le_of_succ h (
    le_max_right i j)⟩

theorem toElementMap_le_of_le {β₀ β₁ : Type*} {W₀ : NeighborhoodSystem β₀}
    {W₁ : NeighborhoodSystem β₁} {f g : ApproximableMap W₀ W₁} (h : f ≤ g) (x : W₀.
    Element) :
    f.toElementMap x ≤ g.toElementMap x := by
  rintro Y ⟨X, hX, hrel⟩
  exact ⟨X, hX, h _ _ hrel⟩

section Target

variable {? : Type*} {E : NeighborhoodSystem ?}

theorem descSeqEltMono (k : ApproximableMap (prod V E) E) (x : (iterSys V).Element) :
    ∀ n, (descSeq k n).toElementMap x ≤ (descSeq k (n + 1)).toElementMap x :=
  fun n => toElementMap_le_of_le (descSeq_mono_succ k n) x

theorem descMap_eq (k : ApproximableMap (prod V E) E) (x : (iterSys V).Element) :
    (descMap k).toElementMap x
      = iSupDirected (fun n => (descSeq k n).toElementMap x)
          (succChainDir _ (descSeqEltMono k x)) := by
```

```
  apply Element.ext
  intro Y
  rw [descMap_toElementMap, mem_iSupDirected]

theorem descMap_fix (k : ApproximableMap (prod V E) E) : descMap k = descOp k (descMap
    k) := by
  apply ext_of_toElementMap
  intro z
  rw [descOp_apply, descMap_eq k (ofSeq (fun n => component z (n + 1))),
    ← kHead_apply k (component z 0), toElementMap_iSupDirected, descMap_eq k z]

  apply Element.ext
  intro Y
  rw [mem_iSupDirected, mem_iSupDirected]
  have hstep : ∀ m, (kHead k (component z 0)).toElementMap
        ((descSeq k m).toElementMap (ofSeq (fun n => component z (n + 1))))
      = (descSeq k (m + 1)).toElementMap z := by
    intro m
    rw [kHead_apply, show descSeq k (m + 1) = descOp k (descSeq k m) from rfl,
    descOp_apply]
  constructor
  · rintro ⟨n, hn⟩
    cases n with
    | zero => exact ⟨0, by rw [hstep 0]; exact descSeqEltMono k z 0 _ hn⟩
    | succ m => exact ⟨m, by rw [hstep m]; exact hn⟩
  · rintro ⟨m, hm⟩
    rw [hstep m] at hm
    exact ⟨m + 1, hm⟩

theorem descMap_comm (k : ApproximableMap (prod V E) E) :
    (descMap k).comp (imap V) = k.comp (prodMap (idMap V) (descMap k)) := by
  conv_lhs => rw [descMap_fix k]
  show (k.comp ((prodMap (idMap V) (descMap k)).comp (jmap V))).comp (imap V)
     = k.comp (prodMap (idMap V) (descMap k))
  rw [comp_assoc, comp_assoc, jmap_comp_imap, comp_idMap]

end Target

def descAlgHom (Dom : StrictDomainObj.{w}) (B : TAlgebra (prodFunctor Dom)) :
    AlgHom (iterAlg Dom) B where
  hom := ⟨descMap B.str.1, descMap_strict B.str.1 B.str.2⟩
  comm := by
    apply Subtype.ext
    show (descMap B.str.1).comp (imap Dom.sys)
       = B.str.1.comp (prodMap (idMap Dom.sys) (descMap B.str.1))
    exact descMap_comm B.str.1

def iterIsInitial (Dom : StrictDomainObj) : IsInitial (iterAlg Dom) where
  desc := descAlgHom Dom
  uniq B h := by
    obtain ⟨hom, comm⟩ := h
    have hcomm : hom.1.comp (imap Dom.sys)
        = B.str.1.comp (prodMap (idMap Dom.sys) hom.1) := congrArg Subtype.val comm
    have hg : hom.1 = descMap B.str.1 :=
      comm_unique B.str.1 hom.2 (descMap_strict B.str.1 B.str.2) hcomm (descMap_comm B
    .str.1)
```

```
    have hhom : hom = ⟨descMap B.str.1, descMap_strict B.str.1 B.str.2⟩ := Subtype.ext
    hg
    subst hhom
    rfl

end IterProdInitial
end
end DomainTheory.NeighbourhoodSystems
```

4.7.42 Exercise 6.18 — infinite product as initial

NeighbourhoodSystems/DomainEquations/Exercise_6_18.lean

`Exercise_6_18.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.iterProdInitial
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Theorem_6_14

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct
open DomainTheory.NeighbourhoodSystems.IterProdInitial
/-- Scott's Exercise 6.18: strictness is preserved by the product of two approximable
    maps. -/
theorem example_618 {α β α' β' : Type*} {V₀ : NeighborhoodSystem α} {V₁ :
    NeighborhoodSystem β}
    {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'} {f₀ : ApproximableMap V
    ₀ V₀'}
    {f₁ : ApproximableMap V₁ V₁'} :
    IsStrict f₀ → IsStrict f₁ → IsStrict (prodMap f₀ f₁) :=
  isStrict_prodMap
```

EXERCISE 6.18. With reference back to Exercise 3.16 discuss the construction of $\mathcal{D}^\infty$ as an initial algebra and as a solution to the domain equation

$$\mathcal{D}^\infty \cong \mathcal{D} \times \mathcal{D}^\infty.$$

(I do not know whether all solutions must be of the form $\mathcal{D}^\infty \times E$.)

4.7.43 sumProdTok — strict sum and product

`sumProdTok.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Example_6_2
public import DomainTheory.NeighbourhoodSystems.DomainEquations.seqEqB

@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap SeqEqB BitDom

namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

def sumTokMaster (D₀ D₁ : NeighborhoodSystem Str) : Set Str :=
  insert [] (embBit false D₀.master ∪ embBit true D₁.master)

theorem embF_subset_sumTokMaster {X : Set Str} (hX : D₀.mem X) :
    embBit false X ⊆ sumTokMaster D₀ D₁ :=
  (embBit_subset.mpr (D₀.sub_master hX)).trans (Set.subset_union_left.trans (Set.
    subset_insert _ _))

theorem embT_subset_sumTokMaster {Y : Set Str} (hY : D₁.mem Y) :
    embBit true Y ⊆ sumTokMaster D₀ D₁ :=
  (embBit_subset.mpr (D₁.sub_master hY)).trans (Set.subset_union_right.trans (Set.
    subset_insert _ _))

theorem nil_mem_sumTokMaster : ([] : Str) ∈ sumTokMaster D₀ D₁ := Set.mem_insert _ _

theorem embF_ne_sumTokMaster {X : Set Str} : embBit false X = sumTokMaster D₀ D₁ := fun
     h =>
  nil_not_mem_embBit (h.symm ▶ nil_mem_sumTokMaster)

theorem embT_ne_sumTokMaster {Y : Set Str} : embBit true Y = sumTokMaster D₀ D₁ := fun
    h =>
  nil_not_mem_embBit (h.symm ▶ nil_mem_sumTokMaster)

theorem sumTokMaster_inter_embF {X : Set Str} (hX : D₀.mem X) :
    sumTokMaster D₀ D₁ ∩ embBit false X = embBit false X :=
  Set.inter_eq_right.mpr (embF_subset_sumTokMaster hX)

theorem sumTokMaster_inter_embT {Y : Set Str} (hY : D₁.mem Y) :
    sumTokMaster D₀ D₁ ∩ embBit true Y = embBit true Y :=
  Set.inter_eq_right.mpr (embT_subset_sumTokMaster hY)

end FunctorTokens
end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap BitDom SeqDom SeqEqB
open NeighborhoodSystem ApproximableMap SeqEqB BitDom
section
namespace FunctorTokens
variable {D₀ D₁ : NeighborhoodSystem Str}

def sumTok (D₀ D₁ : NeighborhoodSystem Str)
    (h₀ : ∀ X, D₀.mem X → X.Nonempty) (h₁ : ∀ Y, D₁.mem Y → Y.Nonempty) :
    NeighborhoodSystem Str where
  mem W := W = sumTokMaster D₀ D₁ ∨ (∃ X, D₀.mem X ∧ W = embBit false X) ∨
    (∃ Y, D₁.mem Y ∧ W = embBit true Y)
  master := sumTokMaster D₀ D₁
  master_nonempty := ⟨[], nil_mem_sumTokMaster⟩
  master_mem := Or.inl rfl
```

```
  sub_master := by
    rintro W (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩)
    · exact subset_rfl
    · exact embF_subset_sumTokMaster hX
    · exact embT_subset_sumTokMaster hY
  inter_mem := by
    have hne : ∀ W, (W = sumTokMaster D₀ D₁ ∨ (∃ X, D₀.mem X ∧ W = embBit false X) ∨
        (∃ Y, D₁.mem Y ∧ W = embBit true Y)) → (W : Set Str).Nonempty := by
      rintro W (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩)
      · exact ⟨[], nil_mem_sumTokMaster⟩
      · exact embBit_nonempty (h₀ X hX)
      · exact embBit_nonempty (h₁ Y hY)
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · rw [Set.inter_self]; exact Or.inl rfl
      · rw [sumTokMaster_inter_embF hX']; exact Or.inr (Or.inl ⟨X', hX', rfl⟩)
      · rw [sumTokMaster_inter_embT hY']; exact Or.inr (Or.inr ⟨Y', hY', rfl⟩)
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · rw [Set.inter_comm, sumTokMaster_inter_embF hX]; exact Or.inr (Or.inl ⟨X, hX,
    rfl⟩)
      · rw [embBit_inter] at hZsub ⊢
        rcases hZ with rfl | ⟨Z₀, hZ₀, rfl⟩ | ⟨Z₁, hZ₁, rfl⟩
        · exact absurd (hZsub nil_mem_sumTokMaster) nil_not_mem_embBit
        · exact Or.inr (Or.inl ⟨X ∩ X', D₀.inter_mem hX hX' hZ₀ (embBit_subset.mp hZsub)
    , rfl⟩)
        · obtain ⟨b, hb⟩ := h₁ Z₁ hZ₁
          obtain ⟨w', he, -⟩ := hZsub (⟨b, rfl, hb⟩ : (true :: b) ∈ embBit true Z₁)
          simp only [List.cons.injEq] at he; exact absurd he.1 (by decide)
      · rw [embBit_inter_ne (show (false : Bool) = true by decide)] at hZsub ⊢
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · rw [Set.inter_comm, sumTokMaster_inter_embT hY]; exact Or.inr (Or.inr ⟨Y, hY,
    rfl⟩)
      · rw [embBit_inter_ne (show (true : Bool) = false by decide)] at hZsub ⊢
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      · rw [embBit_inter] at hZsub ⊢
        rcases hZ with rfl | ⟨Z₀, hZ₀, rfl⟩ | ⟨Z₁, hZ₁, rfl⟩
        · exact absurd (hZsub nil_mem_sumTokMaster) nil_not_mem_embBit
        · obtain ⟨a, ha⟩ := h₀ Z₀ hZ₀
          obtain ⟨w', he, -⟩ := hZsub (⟨a, rfl, ha⟩ : (false :: a) ∈ embBit false Z₀)
          simp only [List.cons.injEq] at he; exact absurd he.1 (by decide)
        · exact Or.inr (Or.inr ⟨Y ∩ Y', D₁.inter_mem hY hY' hZ₁ (embBit_subset.mp hZsub)
    , rfl⟩)

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

theorem sumTok_mem_embF {X : Set Str} (hX : D₀.mem X) :
```

```
    (sumTok D₀ D₁ h₀ h₁).mem (embBit false X) := Or.inr (Or.inl ⟨X, hX, rfl⟩)

theorem sumTok_mem_embF_inv {W : Set Str} (h : (sumTok D₀ D₁ h₀ h₁).mem (embBit false W
    )) :
    D₀.mem W := by
  rcases h with h0 | ⟨X, hX, heq⟩ | ⟨Y, hY, heq⟩
  · exact absurd (h0.symm ▸ nil_mem_sumTokMaster) nil_not_mem_embBit
  · rw [embBit_injective heq]; exact hX
  · exact absurd heq.symm (embBit_ne (show (true : Bool) = false by decide) (h₁ Y hY))

theorem sumTok_mem_embT {Y : Set Str} (hY : D₁.mem Y) :
    (sumTok D₀ D₁ h₀ h₁).mem (embBit true Y) := Or.inr (Or.inr ⟨Y, hY, rfl⟩)

theorem sumTok_mem_embT_inv {W : Set Str} (h : (sumTok D₀ D₁ h₀ h₁).mem (embBit true W)
    ) :
    D₁.mem W := by
  rcases h with h0 | ⟨X, hX, heq⟩ | ⟨Y, hY, heq⟩
  · exact absurd (h0.symm ▸ nil_mem_sumTokMaster) nil_not_mem_embBit
  · exact absurd heq.symm (embBit_ne (show (false : Bool) = true by decide) (h₀ X hX))
  · rw [embBit_injective heq]; exact hY

theorem sumTok_mem_nonempty {W : Set Str} (h : (sumTok D₀ D₁ h₀ h₁).mem W) : W.Nonempty
    := by
  rcases h with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
  · exact ⟨[], nil_mem_sumTokMaster⟩
  · exact embBit_nonempty (h₀ X hX)
  · exact embBit_nonempty (h₁ Y hY)

theorem sum_mem_inj₀_inv {X : Set Str} (h : (sum D₀ D₁ h₀ h₁).mem (inj₀ X)) : D₀.mem X
    := by
  rcases h with h0 | ⟨X', hX', heq⟩ | ⟨Y', hY', heq⟩
  · exact absurd (h0 ▸ none_mem_sumMaster) none_mem_inj₀
  · rw [inj₀_injective heq]; exact hX'
  · obtain ⟨b, hb⟩ := h₁ Y' hY'
    exact absurd (heq ▸ ir_mem_inj₁.mpr hb) ir_mem_inj₀

theorem sum_mem_inj₁_inv {Y : Set Str} (h : (sum D₀ D₁ h₀ h₁).mem (inj₁ Y)) : D₁.mem Y
    := by
  rcases h with h0 | ⟨X', hX', heq⟩ | ⟨Y', hY', heq⟩
  · exact absurd (h0 ▸ none_mem_sumMaster) none_mem_inj₁
  · obtain ⟨a, ha⟩ := h₀ X' hX'
    exact absurd (heq ▸ il_mem_inj₀.mpr ha) il_mem_inj₁
  · rw [inj₁_injective heq]; exact hY'

theorem sum_mem_nonempty {W : Set (Option (Str ? Str))} (h : (sum D₀ D₁ h₀ h₁).mem W) :
    W.Nonempty := by
  rcases h with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
  · exact ⟨none, none_mem_sumMaster⟩
  · exact inj₀_nonempty (h₀ X hX)
  · exact inj₁_nonempty (h₁ Y hY)

def toSum (x : (sumTok D₀ D₁ h₀ h₁).Element) : (sum D₀ D₁ h₀ h₁).Element where
  mem W := W = sumMaster D₀ D₁
    ∨ (∃ X, D₀.mem X ∧ W = inj₀ X ∧ x.mem (embBit false X))
    ∨ (∃ Y, D₁.mem Y ∧ W = inj₁ Y ∧ x.mem (embBit true Y))
  sub := by
```

```
    rintro W (rfl | ⟨X, hX, rfl, -⟩ | ⟨Y, hY, rfl, -⟩)
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨X, hX, rfl⟩)
    · exact Or.inr (Or.inr ⟨Y, hY, rfl⟩)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩)
      (rfl | ⟨X', hX', rfl, hzX'⟩ | ⟨Y', hY', rfl, hzY'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (Or.inl ⟨X', hX', by rw [sumMaster_inter_inj₀ hX'], hzX'⟩)
    · exact Or.inr (Or.inr ⟨Y', hY', by rw [sumMaster_inter_inj₁ hY'], hzY'⟩)
    · exact Or.inr (Or.inl ⟨X, hX, by rw [Set.inter_comm, sumMaster_inter_inj₀ hX], hzX⟩
    )
    · refine Or.inr (Or.inl ⟨X ∩ X', ?_, by rw [inj₀_inter], ?_⟩)
      · have hz := x.inter_mem hzX hzX'; rw [embBit_inter] at hz; exact
    sumTok_mem_embF_inv (x.sub hz)
      · have hz := x.inter_mem hzX hzX'; rwa [embBit_inter] at hz
    · exfalso
      have hz := x.inter_mem hzX hzY'
      rw [embBit_inter_ne (show (false : Bool) = true by decide)] at hz
      obtain ⟨t, ht⟩ := sumTok_mem_nonempty (x.sub hz); exact Set.notMem_empty t ht
    · exact Or.inr (Or.inr ⟨Y, hY, by rw [Set.inter_comm, sumMaster_inter_inj₁ hY], hzY⟩
    )
    · exfalso
      have hz := x.inter_mem hzY hzX'
      rw [embBit_inter_ne (show (true : Bool) = false by decide)] at hz
      obtain ⟨t, ht⟩ := sumTok_mem_nonempty (x.sub hz); exact Set.notMem_empty t ht
    · refine Or.inr (Or.inr ⟨Y ∩ Y', ?_, by rw [inj₁_inter], ?_⟩)
      · have hz := x.inter_mem hzY hzY'; rw [embBit_inter] at hz; exact
    sumTok_mem_embT_inv (x.sub hz)
      · have hz := x.inter_mem hzY hzY'; rwa [embBit_inter] at hz
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩) hW' hsub
    · exact Or.inl (eq_sumMaster_of_subset hW' hsub)
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · exact Or.inl rfl
      · refine Or.inr (Or.inl ⟨X', hX', rfl, ?_⟩)
        exact x.up_mem hzX (sumTok_mem_embF hX') (embBit_subset.mpr (inj₀_subset_inj₀.
    mp hsub))
      · exfalso
        obtain ⟨a, ha⟩ := h₀ X hX
        exact absurd (hsub (il_mem_inj₀.mpr ha)) il_mem_inj₁
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · exact Or.inl rfl
      · exfalso
        obtain ⟨b, hb⟩ := h₁ Y hY
        exact absurd (hsub (ir_mem_inj₁.mpr hb)) ir_mem_inj₀
      · refine Or.inr (Or.inr ⟨Y', hY', rfl, ?_⟩)
        exact x.up_mem hzY (sumTok_mem_embT hY') (embBit_subset.mpr (inj₁_subset_inj₁.
    mp hsub))

@[simp] theorem toSum_mem_inj₀ {x : (sumTok D₀ D₁ h₀ h₁).Element} {X : Set Str} (hX : D₀
    .mem X) :
    (toSum x).mem (inj₀ X) ↔ x.mem (embBit false X) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hz⟩ | ⟨Y', hY', heq, hz⟩)
```

```
    · exact absurd (h0 ▶ none_mem_sumMaster) none_mem_inj₀
    · rwa [inj₀_injective heq]
    · obtain ⟨a, ha⟩ := h₀ X hX
      exact absurd (heq ▶ il_mem_inj₀.mpr ha) il_mem_inj₁
  · intro hz; exact Or.inr (Or.inl ⟨X, hX, rfl, hz⟩)

@[simp] theorem toSum_mem_inj₁ {x : (sumTok D₀ D₁ h₀ h₁).Element} {Y : Set Str} (hY : D₁
    .mem Y) :
    (toSum x).mem (inj₁ Y) ↔ x.mem (embBit true Y) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hz⟩ | ⟨Y', hY', heq, hz⟩)
    · exact absurd (h0 ▶ none_mem_sumMaster) none_mem_inj₁
    · obtain ⟨a, ha⟩ := h₀ X' hX'
      exact absurd (heq ▶ il_mem_inj₀.mpr ha) il_mem_inj₁
    · rwa [inj₁_injective heq]
  · intro hz; exact Or.inr (Or.inr ⟨Y, hY, rfl, hz⟩)

def fromSum (s : (sum D₀ D₁ h₀ h₁).Element) : (sumTok D₀ D₁ h₀ h₁).Element where
  mem W := W = sumTokMaster D₀ D₁
    ∨ (∃ X, D₀.mem X ∧ W = embBit false X ∧ s.mem (inj₀ X))
    ∨ (∃ Y, D₁.mem Y ∧ W = embBit true Y ∧ s.mem (inj₁ Y))
  sub := by
    rintro W (rfl | ⟨X, hX, rfl, -⟩ | ⟨Y, hY, rfl, -⟩)
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨X, hX, rfl⟩)
    · exact Or.inr (Or.inr ⟨Y, hY, rfl⟩)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hsX⟩ | ⟨Y, hY, rfl, hsY⟩)
      (rfl | ⟨X', hX', rfl, hsX'⟩ | ⟨Y', hY', rfl, hsY'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (Or.inl ⟨X', hX', by rw [sumTokMaster_inter_embF hX'], hsX'⟩)
    · exact Or.inr (Or.inr ⟨Y', hY', by rw [sumTokMaster_inter_embT hY'], hsY'⟩)
    · exact Or.inr (Or.inl ⟨X, hX, by rw [Set.inter_comm, sumTokMaster_inter_embF hX],
    hsX⟩)
    · refine Or.inr (Or.inl ⟨X ∩ X', ?_, by rw [embBit_inter], ?_⟩)
      · have hs := s.inter_mem hsX hsX'; rw [inj₀_inter] at hs
        exact sum_mem_inj₀_inv (s.sub hs)
      · have hs := s.inter_mem hsX hsX'; rwa [inj₀_inter] at hs
    · exfalso
      have hs := s.inter_mem hsX hsY'; rw [inj₀_inter_inj₁] at hs
      obtain ⟨t, ht⟩ := sum_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    · exact Or.inr (Or.inr ⟨Y, hY, by rw [Set.inter_comm, sumTokMaster_inter_embT hY],
    hsY⟩)
    · exfalso
      have hs := s.inter_mem hsY hsX'; rw [Set.inter_comm, inj₀_inter_inj₁] at hs
      obtain ⟨t, ht⟩ := sum_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    · refine Or.inr (Or.inr ⟨Y ∩ Y', ?_, by rw [embBit_inter], ?_⟩)
      · have hs := s.inter_mem hsY hsY'; rw [inj₁_inter] at hs
        exact sum_mem_inj₁_inv (s.sub hs)
      · have hs := s.inter_mem hsY hsY'; rwa [inj₁_inter] at hs
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hsX⟩ | ⟨Y, hY, rfl, hsY⟩) hW' hsub
    · exact Or.inl (Set.Subset.antisymm ((sumTok D₀ D₁ h₀ h₁).sub_master hW') hsub)
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · exact Or.inl rfl
```

```
      · refine Or.inr (Or.inl ⟨X', hX', rfl, ?_⟩)
        exact s.up_mem hsX (Or.inr (Or.inl ⟨X', hX', rfl⟩))
          (inj₀_subset_inj₀.mpr (embBit_subset.mp hsub))
      · exfalso
        obtain ⟨a, ha⟩ := h₀ X hX
        obtain ⟨w', he, -⟩ := hsub (⟨a, rfl, ha⟩ : (false :: a) ∈ embBit false X)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
    · rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
      · exact Or.inl rfl
      · exfalso
        obtain ⟨b, hb⟩ := h₁ Y hY
        obtain ⟨w', he, -⟩ := hsub (⟨b, rfl, hb⟩ : (true :: b) ∈ embBit true Y)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
      · refine Or.inr (Or.inr ⟨Y', hY', rfl, ?_⟩)
        exact s.up_mem hsY (Or.inr (Or.inr ⟨Y', hY', rfl⟩))
          (inj₁_subset_inj₁.mpr (embBit_subset.mp hsub))

@[simp] theorem fromSum_mem_embF {s : (sum D₀ D₁ h₀ h₁).Element} {X : Set Str} (hX : D₀.
    mem X) :
    (fromSum s).mem (embBit false X) ↔ s.mem (inj₀ X) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hs⟩ | ⟨Y', hY', heq, hs⟩)
    · exact absurd h0 embF_ne_sumTokMaster
    · rwa [embBit_injective heq]
    · exact absurd heq (embBit_ne (show (false : Bool) = true by decide) (h₀ X hX))
  · intro hs; exact Or.inr (Or.inl ⟨X, hX, rfl, hs⟩)

@[simp] theorem fromSum_mem_embT {s : (sum D₀ D₁ h₀ h₁).Element} {Y : Set Str} (hY : D₁.
    mem Y) :
    (fromSum s).mem (embBit true Y) ↔ s.mem (inj₁ Y) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hs⟩ | ⟨Y', hY', heq, hs⟩)
    · exact absurd h0 embT_ne_sumTokMaster
    · exact absurd heq.symm (embBit_ne (show (false : Bool) = true by decide) (h₀ X' hX
    '))
    · rwa [embBit_injective heq]
  · intro hs; exact Or.inr (Or.inr ⟨Y, hY, rfl, hs⟩)

theorem fromSum_toSum (x : (sumTok D₀ D₁ h₀ h₁).Element) : fromSum (toSum x) = x := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨X, hX, rfl, hs⟩ | ⟨Y, hY, rfl, hs⟩)
    · exact x.master_mem
    · exact (toSum_mem_inj₀ hX).mp hs
    · exact (toSum_mem_inj₁ hY).mp hs
  · intro hW
    rcases x.sub hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨X, hX, rfl, (toSum_mem_inj₀ hX).mpr hW⟩)
    · exact Or.inr (Or.inr ⟨Y, hY, rfl, (toSum_mem_inj₁ hY).mpr hW⟩)

def prodTokNbhd (X Y : Set Str) : Set Str := insert [] (embBit false X ∪ embBit true Y
    )

@[simp] theorem mem_prodTokNbhd_nil {X Y : Set Str} : ([] : Str) ∈ prodTokNbhd X Y :=
```

```
  Set.mem_insert _ _

@[simp] theorem mem_prodTokNbhd_false {X Y : Set Str} {w' : Str} :
    (false :: w') ∈ prodTokNbhd X Y ↔ w' ∈ X := by
  simp only [prodTokNbhd, Set.mem_insert_iff, Set.mem_union, mem_embBit]
  constructor
  · rintro (h | ⟨u, hu, hX⟩ | ⟨u, hu, hY⟩)
    · exact absurd h (by simp)
    · rw [List.cons.injEq] at hu; rw [hu.2]; exact hX
    · rw [List.cons.injEq] at hu; exact absurd hu.1 (by decide)
  · intro h; exact Or.inr (Or.inl ⟨w', rfl, h⟩)

@[simp] theorem mem_prodTokNbhd_true {X Y : Set Str} {w' : Str} :
    (true :: w') ∈ prodTokNbhd X Y ↔ w' ∈ Y := by
  simp only [prodTokNbhd, Set.mem_insert_iff, Set.mem_union, mem_embBit]
  constructor
  · rintro (h | ⟨u, hu, hX⟩ | ⟨u, hu, hY⟩)
    · exact absurd h (by simp)
    · rw [List.cons.injEq] at hu; exact absurd hu.1 (by decide)
    · rw [List.cons.injEq] at hu; rw [hu.2]; exact hY
  · intro h; exact Or.inr (Or.inr ⟨w', rfl, h⟩)

theorem prodTokNbhd_inter (X X' Y Y' : Set Str) :
    prodTokNbhd X Y ∩ prodTokNbhd X' Y' = prodTokNbhd (X ∩ X') (Y ∩ Y') := by
  ext w
  cases w with
  | nil => simp [Set.mem_inter_iff]
  | cons b w' => cases b <;> simp [Set.mem_inter_iff]

theorem prodTokNbhd_subset_iff {X X' Y Y' : Set Str} :
    prodTokNbhd X Y ⊆ prodTokNbhd X' Y' ↔ X ⊆ X' ∧ Y ⊆ Y' := by
  constructor
  · intro h
    refine ⟨fun a ha => ?_, fun b hb => ?_⟩
    · exact mem_prodTokNbhd_false.mp (h (mem_prodTokNbhd_false.mpr ha))
    · exact mem_prodTokNbhd_true.mp (h (mem_prodTokNbhd_true.mpr hb))
  · rintro ⟨hX, hY⟩ w hw
    cases w with
    | nil => exact mem_prodTokNbhd_nil
    | cons b w' =>
      cases b with
      | false => exact mem_prodTokNbhd_false.mpr (hX (mem_prodTokNbhd_false.mp hw))
      | true => exact mem_prodTokNbhd_true.mpr (hY (mem_prodTokNbhd_true.mp hw))

def prodTok (D₀ D₁ : NeighborhoodSystem Str) : NeighborhoodSystem Str where
  mem W := ∃ X Y, D₀.mem X ∧ D₁.mem Y ∧ W = prodTokNbhd X Y
  master := prodTokNbhd D₀.master D₁.master
  master_nonempty := ⟨[], mem_prodTokNbhd_nil⟩
  master_mem := ⟨D₀.master, D₁.master, D₀.master_mem, D₁.master_mem, rfl⟩
  inter_mem := by
    rintro W W' Z ⟨X, Y, hX, hY, rfl⟩ ⟨X', Y', hX', hY', rfl⟩ ⟨Z₀, Z₁, hZ₀, hZ₁, rfl⟩
    hsub
    rw [prodTokNbhd_inter] at hsub ⊢
    obtain ⟨hsub₀, hsub₁⟩ := prodTokNbhd_subset_iff.mp hsub
    exact ⟨X ∩ X', Y ∩ Y', D₀.inter_mem hX hX' hZ₀ hsub₀, D₁.inter_mem hY hY' hZ₁ hsub₁,
     rfl⟩
```

```
  sub_master := by
    rintro W ⟨X, Y, hX, hY, rfl⟩
    exact prodTokNbhd_subset_iff.mpr ⟨D₀.sub_master hX, D₁.sub_master hY⟩

theorem prodTokNbhd_injective {X X' Y Y' : Set Str} (h : prodTokNbhd X Y = prodTokNbhd
    X' Y') :
    X = X' ∧ Y = Y' :=
  ⟨Set.Subset.antisymm (prodTokNbhd_subset_iff.mp h.subset).1
      (prodTokNbhd_subset_iff.mp h.symm.subset).1,
   Set.Subset.antisymm (prodTokNbhd_subset_iff.mp h.subset).2
      (prodTokNbhd_subset_iff.mp h.symm.subset).2⟩

@[simp] theorem prodTok_mem_iff {W : Set Str} :
    (prodTok D₀ D₁).mem W ↔ ∃ X Y, D₀.mem X ∧ D₁.mem Y ∧ W = prodTokNbhd X Y := Iff.rfl

@[simp] theorem prodTok_master : (prodTok D₀ D₁).master = prodTokNbhd D₀.master D₁.
    master := rfl

theorem prodTok_mem_prodTokNbhd {X Y : Set Str} (hX : D₀.mem X) (hY : D₁.mem Y) :
    (prodTok D₀ D₁).mem (prodTokNbhd X Y) := ⟨X, Y, hX, hY, rfl⟩

def fstTok (z : (prodTok D₀ D₁).Element) : D₀.Element where
  mem X := D₀.mem X ∧ z.mem (prodTokNbhd X D₁.master)
  sub h := h.1
  master_mem := ⟨D₀.master_mem, z.master_mem⟩
  inter_mem := by
    rintro X X' ⟨_, hzX⟩ ⟨_, hzX'⟩
    have hz := z.inter_mem hzX hzX'
    rw [prodTokNbhd_inter, Set.inter_self] at hz
    obtain ⟨A, B, hA, _, heq⟩ := z.sub hz
    obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective heq
    exact ⟨hA, hz⟩

  up_mem := by
    rintro X X' ⟨_, hzX⟩ hX' hXX'
    exact ⟨hX', z.up_mem hzX (prodTok_mem_prodTokNbhd hX' D₁.master_mem)
      (prodTokNbhd_subset_iff.mpr ⟨hXX', subset_rfl⟩)⟩

@[simp] theorem mem_fstTok {z : (prodTok D₀ D₁).Element} {X : Set Str} :
    (fstTok z).mem X ↔ D₀.mem X ∧ z.mem (prodTokNbhd X D₁.master) := Iff.rfl

def sndTok (z : (prodTok D₀ D₁).Element) : D₁.Element where
  mem Y := D₁.mem Y ∧ z.mem (prodTokNbhd D₀.master Y)
  sub h := h.1
  master_mem := ⟨D₁.master_mem, z.master_mem⟩
  inter_mem := by
    rintro Y Y' ⟨_, hzY⟩ ⟨_, hzY'⟩
    have hz := z.inter_mem hzY hzY'
    rw [prodTokNbhd_inter, Set.inter_self] at hz
    obtain ⟨A, B, _, hB, heq⟩ := z.sub hz
    obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective heq
    exact ⟨hB, hz⟩
  up_mem := by
    rintro Y Y' ⟨_, hzY⟩ hY' hYY'
    exact ⟨hY', z.up_mem hzY (prodTok_mem_prodTokNbhd D₀.master_mem hY')
      (prodTokNbhd_subset_iff.mpr ⟨subset_rfl, hYY'⟩)⟩
```

```
@[simp] theorem mem_sndTok {z : (prodTok D₀ D₁).Element} {Y : Set Str} :
    (sndTok z).mem Y ↔ D₁.mem Y ∧ z.mem (prodTokNbhd D₀.master Y) := Iff.rfl

def pairTok (x : D₀.Element) (y : D₁.Element) : (prodTok D₀ D₁).Element where
  mem W := ∃ X Y, x.mem X ∧ y.mem Y ∧ W = prodTokNbhd X Y
  sub := by rintro W ⟨X, Y, hX, hY, rfl⟩; exact prodTok_mem_prodTokNbhd (x.sub hX) (y.
    sub hY)
  master_mem := ⟨D₀.master, D₁.master, x.master_mem, y.master_mem, rfl⟩
  inter_mem := by
    rintro W W' ⟨X, Y, hX, hY, rfl⟩ ⟨X', Y', hX', hY', rfl⟩
    exact ⟨X ∩ X', Y ∩ Y', x.inter_mem hX hX', y.inter_mem hY hY', prodTokNbhd_inter X
    X' Y Y'⟩
  up_mem := by
    rintro W W' ⟨X, Y, hX, hY, rfl⟩ ⟨X', Y', hX', hY', rfl⟩ hsub
    obtain ⟨hXX', hYY'⟩ := prodTokNbhd_subset_iff.mp hsub
    exact ⟨X', Y', x.up_mem hX hX' hXX', y.up_mem hY hY' hYY', rfl⟩

theorem mem_pairTok_prodTokNbhd {x : D₀.Element} {y : D₁.Element} {X Y : Set Str} :
    (pairTok x y).mem (prodTokNbhd X Y) ↔ x.mem X ∧ y.mem Y := by
  constructor
  · rintro ⟨X', Y', hX', hY', heq⟩
    obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective heq
    exact ⟨hX', hY'⟩
  · rintro ⟨hx, hy⟩; exact ⟨X, Y, hx, hy, rfl⟩

@[simp] theorem fstTok_pairTok (x : D₀.Element) (y : D₁.Element) : fstTok (pairTok x y)
    = x := by
  apply NeighborhoodSystem.Element.ext
  intro X
  constructor
  · rintro ⟨hX, hmem⟩; exact (mem_pairTok_prodTokNbhd.mp hmem).1
  · intro hX; exact ⟨x.sub hX, mem_pairTok_prodTokNbhd.mpr ⟨hX, y.master_mem⟩⟩

theorem prodTok_mem_split {z : (prodTok D₀ D₁).Element} {X Y : Set Str}
    (hX : D₀.mem X) (hY : D₁.mem Y) :
    z.mem (prodTokNbhd X Y) ↔
      z.mem (prodTokNbhd X D₁.master) ∧ z.mem (prodTokNbhd D₀.master Y) := by
  constructor
  · intro h
    refine ⟨z.up_mem h (prodTok_mem_prodTokNbhd hX D₁.master_mem) ?_,
            z.up_mem h (prodTok_mem_prodTokNbhd D₀.master_mem hY) ?_⟩
    · exact prodTokNbhd_subset_iff.mpr ⟨subset_rfl, D₁.sub_master hY⟩
    · exact prodTokNbhd_subset_iff.mpr ⟨D₀.sub_master hX, subset_rfl⟩
  · rintro ⟨h1, h2⟩
    have := z.inter_mem h1 h2
    rwa [prodTokNbhd_inter, Set.inter_eq_left.mpr (D₀.sub_master hX),
      Set.inter_eq_right.mpr (D₁.sub_master hY)] at this

theorem pairTok_fstTok_sndTok (z : (prodTok D₀ D₁).Element) : pairTok (fstTok z) (
    sndTok z) = z := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro ⟨X, Y, ⟨hX, hzX⟩, ⟨hY, hzY⟩, rfl⟩
    exact (prodTok_mem_split hX hY).mpr ⟨hzX, hzY⟩
```

```
  · intro hW
    obtain ⟨X, Y, hX, hY, rfl⟩ := z.sub hW
    obtain ⟨h1, h2⟩ := (prodTok_mem_split hX hY).mp hW
    exact ⟨X, Y, ⟨hX, h1⟩, ⟨hY, h2⟩, rfl⟩

@[simp] theorem sndTok_pairTok (x : D₀.Element) (y : D₁.Element) : sndTok (pairTok x y)
     = y := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  constructor
  · rintro ⟨hY, hmem⟩; exact (mem_pairTok_prodTokNbhd.mp hmem).2
  · intro hY; exact ⟨y.sub hY, mem_pairTok_prodTokNbhd.mpr ⟨x.master_mem, hY⟩⟩

def prodTokEquiv : (prodTok D₀ D₁).Element ≃o D₀.Element × D₁.Element where
  toFun z := (fstTok z, sndTok z)
  invFun p := pairTok p.1 p.2
  left_inv z := pairTok_fstTok_sndTok z
  right_inv p := by simp
  map_rel_iff' := by
    intro z z'
    constructor
    · rintro ⟨h1, h2⟩ W hW
      obtain ⟨X, Y, hX, hY, rfl⟩ := z.sub hW
      obtain ⟨ha, hb⟩ := (prodTok_mem_split hX hY).mp hW
      have hX' : (fstTok z').mem X := h1 X ⟨hX, ha⟩
      have hY' : (sndTok z').mem Y := h2 Y ⟨hY, hb⟩
      exact (prodTok_mem_split hX hY).mpr ⟨hX'.2, hY'.2⟩
    · intro h
      exact ⟨fun X ⟨hX, hzX⟩ => ⟨hX, h _ hzX⟩, fun Y ⟨hY, hzY⟩ => ⟨hY, h _ hzY⟩⟩

theorem toSum_fromSum (s : (sum D₀ D₁ h₀ h₁).Element) : toSum (fromSum s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨X, hX, rfl, hs⟩ | ⟨Y, hY, rfl, hs⟩)
    · exact s.master_mem
    · exact (fromSum_mem_embF hX).mp hs
    · exact (fromSum_mem_embT hY).mp hs
  · intro hW
    rcases s.sub hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨X, hX, rfl, (fromSum_mem_embF hX).mpr hW⟩)
    · exact Or.inr (Or.inr ⟨Y, hY, rfl, (fromSum_mem_embT hY).mpr hW⟩)

def sumTokEquiv : (sumTok D₀ D₁ h₀ h₁).Element ≃o (sum D₀ D₁ h₀ h₁).Element where
  toFun := toSum
  invFun := fromSum
  left_inv := fromSum_toSum
  right_inv := toSum_fromSum
  map_rel_iff' := by
    intro x x'
    constructor
    · intro h X hX
      rcases x.sub hX with rfl | ⟨A, hA, rfl⟩ | ⟨A, hA, rfl⟩
      · exact x'.master_mem
      · exact (toSum_mem_inj₀ hA).mp (h _ (Or.inr (Or.inl ⟨A, hA, rfl, hX⟩)))
```

```
      · exact (toSum_mem_inj₁ hA).mp (h _ (Or.inr (Or.inr ⟨A, hA, rfl, hX⟩)))
    · intro h W hW
      rcases hW with rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨X, hX, rfl, h _ hzX⟩)
      · exact Or.inr (Or.inr ⟨Y, hY, rfl, h _ hzY⟩)

theorem prodTokNbhd_master_eq : prodTokNbhd D₀.master D₁.master = sumTokMaster D₀ D₁ :=
    rfl

theorem prodTok_iso_prod : prodTok D₀ D₁ ≅? prod D₀ D₁ :=
  ⟨prodTokEquiv.trans (prodEquiv D₀ D₁).symm⟩

theorem prodTok_nonempty : ∀ W, (prodTok D₀ D₁).mem W → W.Nonempty := by
  rintro W ⟨X, Y, _, _, rfl⟩; exact ⟨[], mem_prodTokNbhd_nil⟩

theorem sumTok_iso_sum : sumTok D₀ D₁ h₀ h₁ ≅? sum D₀ D₁ h₀ h₁ := ⟨sumTokEquiv⟩

theorem sumTok_mem_master {D₀ D₁ : NeighborhoodSystem Str}
    {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty} :
    (sumTok D₀ D₁ h₀ h₁).mem (sumTokMaster D₀ D₁) := Or.inl rfl

theorem sumTok_nonempty {D₀ D₁ : NeighborhoodSystem Str}
    {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty} :
    ∀ W, (sumTok D₀ D₁ h₀ h₁).mem W → W.Nonempty :=
  fun _ h => sumTok_mem_nonempty h

end FunctorTokens
end

end DomainTheory.NeighbourhoodSystems
```

4.7.44 Exercise 6.19 — strict sum and product

For the sake of uniformity restrict attention to systems $\mathcal{D}$ on sets $\Delta \subseteq \{0,1\}^*$, where $\Lambda \in \Delta$ and $\emptyset \notin \mathcal{D}$, and to the category of strict maps. Define sum and product by:

$$\mathcal{D}_0 + \mathcal{D}_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{0X \mid X \in \mathcal{D}_0\} \cup \{1Y \mid Y \in \mathcal{D}_1\},$$

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{\{\Lambda\} \cup 0X \cup 1Y \mid X \in \mathcal{D}_0 \text{ and } Y \in \mathcal{D}_1\}.$$

Are these correct up to isomorphism? Now generate all constructs $T(X)$ formed by the constants (that is, $T(X) = \mathcal{D}$ for a fixed $\mathcal{D}$), by the identity ($T(X) = X$), and by sums and products ($T_0(X) + T_1(X)$, etc.). Show that these are all functors, continuous on maps, and monotone and continuous on domains.

NeighbourhoodSystems/DomainEquations/Exercise_6_19.lean

Exercise_6_19.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.sumProdTok

@[expose] public section
```

```
open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap BitDom DomainTheory.NeighbourhoodSystems.
    FunctorTokens

/-- Scott's Exercise 6.19: product and sum token neighbourhood systems are isomorphic
    to `prod` and `sum`. -/
theorem example_619 :
    (∀ {D₀ D₁ : NeighborhoodSystem Str}, prodTok D₀ D₁ ≅? prod D₀ D₁) ∧
    (∀ {D₀ D₁ : NeighborhoodSystem Str} {h₀ : ∀ X, D₀.mem X → X.Nonempty}
        {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty},
      sumTok D₀ D₁ h₀ h₁ ≅? sum D₀ D₁ h₀ h₁) :=
  ⟨@prodTok_iso_prod, @sumTok_iso_sum⟩
```

4.7.45 functorTokens — FExpr functor algebra

functorTokens.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.sumProdTok
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Example_6_2
public import DomainTheory.NeighbourhoodSystems.DomainEquations.subdomain
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.smashProduct
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap BitDom SeqEqB
open DomainTheory.NeighbourhoodSystems.FunctorTokens
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
open DomainTheory.NeighbourhoodSystems.SmashProduct

namespace FunctorTokens
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.
    FunctorTokens
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
open DomainTheory.NeighbourhoodSystems.SmashProduct

structure ScottSys where

  sys : NeighborhoodSystem Str

  ne : ∀ X, sys.mem X → X.Nonempty

end FunctorTokens
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.
    FunctorTokens
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
open DomainTheory.NeighbourhoodSystems.SmashProduct
```

```
section
namespace FunctorTokens

variable {A₀ A₁ B₀ B₁ C₀ C₁ : ScottSys}

inductive FExpr where
  | const : ScottSys → FExpr
  | var : FExpr
  | sum : FExpr → FExpr → FExpr
  | prod : FExpr → FExpr → FExpr

theorem embBit_not_subset_cross {b b' : Bool} (h : b = b') {X Y : Set Str} (hX : X.
    Nonempty)
    (hsub : embBit b X ⊆ embBit b' Y) : False := by
  obtain ⟨t, ht⟩ := hX
  obtain ⟨w', he, -⟩ := hsub ⟨t, rfl, ht⟩
  simp only [List.cons.injEq] at he
  exact h he.1

end FunctorTokens
end

namespace FunctorTokens

def ScottSys.prod (A₀ A₁ : ScottSys) : ScottSys :=
  ⟨prodTok A₀.sys A₁.sys, prodTok_nonempty⟩

end FunctorTokens

section
namespace FunctorTokens

variable {A₀ A₁ B₀ B₁ C₀ C₁ : ScottSys}

def prodMapTok (f₀ : ApproximableMap A₀.sys B₀.sys) (f₁ : ApproximableMap A₁.sys B₁.sys)
    :
    ApproximableMap (A₀.prod A₁).sys (B₀.prod B₁).sys where
  rel W W' := ∃ X Y X' Y', f₀.rel X X' ∧ f₁.rel Y Y' ∧
    W = prodTokNbhd X Y ∧ W' = prodTokNbhd X' Y'
  rel_dom := by
    rintro W W' ⟨X, Y, X', Y', h0, h1, rfl, -⟩
    exact prodTok_mem_prodTokNbhd (f₀.rel_dom h0) (f₁.rel_dom h1)
  rel_cod := by
    rintro W W' ⟨X, Y, X', Y', h0, h1, -, rfl⟩
    exact prodTok_mem_prodTokNbhd (f₀.rel_cod h0) (f₁.rel_cod h1)
  master_rel :=
    ⟨A₀.sys.master, A₁.sys.master, B₀.sys.master, B₁.sys.master,
      f₀.master_rel, f₁.master_rel, rfl, rfl⟩
  inter_right := by
    rintro W W'₁ W'₂ ⟨X, Y, X', Y', h0, h1, rfl, rfl⟩ ⟨X₂, Y₂, X'₂, Y'₂, h0₂, h1₂, hWeq,
    rfl⟩
    obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective hWeq
    exact ⟨X, Y, X' ∩ X'₂, Y' ∩ Y'₂, f₀.inter_right h0 h0₂, f₁.inter_right h1 h1₂, rfl,
      prodTokNbhd_inter X' X'₂ Y' Y'₂⟩
  mono := by
    rintro W W'' Z Z' ⟨X, Y, X', Y', h0, h1, rfl, rfl⟩ hWW hZZ' hW'' hZ'
```

```
    obtain ⟨X₂, Y₂, hX₂, hY₂, rfl⟩ := hW''
    obtain ⟨X'₃, Y'₃, hX'₃, hY'₃, rfl⟩ := hZ'
    obtain ⟨hsX, hsY⟩ := prodTokNbhd_subset_iff.mp hWW
    obtain ⟨hsX', hsY'⟩ := prodTokNbhd_subset_iff.mp hZZ'
    exact ⟨X₂, Y₂, X'₃, Y'₃, f₀.mono h0 hsX hsX' hX₂ hX'₃, f₁.mono h1 hsY hsY' hY₂ hY'₃,
    rfl, rfl⟩

def singletonSys (Γ : Set Str) (h : Γ.Nonempty) : ScottSys where
  sys :=
    { mem := fun X => X = Γ
      master := Γ
      master_nonempty := h
      master_mem := rfl
      inter_mem := by
        intro X Y Z hX hY _ _
        show X ∩ Y = Γ
        rw [hX, hY, Set.inter_self]
      sub_master := by intro X hX; rw [show X = Γ from hX] }
  ne := by intro X hX; rw [show X = Γ from hX]; exact h

end FunctorTokens
end

namespace FunctorTokens

def ScottSys.sum (A₀ A₁ : ScottSys) : ScottSys :=
  ⟨sumTok A₀.sys A₁.sys A₀.ne A₁.ne, sumTok_nonempty⟩

end FunctorTokens

section
namespace FunctorTokens

variable {A₀ A₁ B₀ B₁ C₀ C₁ : ScottSys}

def FExpr.RootedConst : FExpr → Prop
  | .const C => ([] : Str) ∈ C.sys.master
  | .var => True
  | .sum a b => a.RootedConst ∧ b.RootedConst
  | .prod a b => a.RootedConst ∧ b.RootedConst

def mFun : FExpr → Set Str → Set Str
  | .const C, _ => C.sys.master
  | .var, Γ => Γ
  | .sum T₀ T₁, Γ => insert ([] : Str) (embBit false (mFun T₀ Γ) ∪ embBit true (mFun T₁
    Γ))
  | .prod T₀ T₁, Γ => insert ([] : Str) (embBit false (mFun T₀ Γ) ∪ embBit true (mFun T
    ₁ Γ))

def mIter (T : FExpr) : ℕ → Set Str
  | 0 => {([] : Str)}
  | n + 1 => mFun T (mIter T n)

def FExpr.obj : FExpr → ScottSys → ScottSys
  | .const D, _ => D
  | .var, X => X
```

```
  | .sum T₀ T₁, X => (T₀.obj X).sum (T₁.obj X)
  | .prod T₀ T₁, X => (T₀.obj X).prod (T₁.obj X)

def sumMapTok (f₀ : ApproximableMap A₀.sys B₀.sys) (f₁ : ApproximableMap A₁.sys B₁.sys)
    :
    ApproximableMap (A₀.sum A₁).sys (B₀.sum B₁).sys where
  rel W W' :=
    ((sumTok A₀.sys A₁.sys A₀.ne A₁.ne).mem W ∧ W' = sumTokMaster B₀.sys B₁.sys) ∨
    (∃ X X', f₀.rel X X' ∧ W = embBit false X ∧ W' = embBit false X') ∨
    (∃ Y Y', f₁.rel Y Y' ∧ W = embBit true Y ∧ W' = embBit true Y')
  rel_dom := by
    rintro W W' (⟨hW, -⟩ | ⟨X, X', hrel, rfl, -⟩ | ⟨Y, Y', hrel, rfl, -⟩)
    · exact hW
    · exact Or.inr (Or.inl ⟨X, f₀.rel_dom hrel, rfl⟩)
    · exact Or.inr (Or.inr ⟨Y, f₁.rel_dom hrel, rfl⟩)
  rel_cod := by
    rintro W W' (⟨-, rfl⟩ | ⟨X, X', hrel, -, rfl⟩ | ⟨Y, Y', hrel, -, rfl⟩)
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨X', f₀.rel_cod hrel, rfl⟩)
    · exact Or.inr (Or.inr ⟨Y', f₁.rel_cod hrel, rfl⟩)
  master_rel := Or.inl ⟨(A₀.sum A₁).sys.master_mem, rfl⟩
  inter_right := by
    rintro W W'₁ W'₂ h1 h2
    rcases h1 with ⟨hW, rfl⟩ | ⟨X, X', hrel, rfl, rfl⟩ | ⟨Y, Y', hrel, rfl, rfl⟩
    · rcases h2 with ⟨-, rfl⟩ | ⟨X, X', hrel, hWeq, rfl⟩ | ⟨Y, Y', hrel, hWeq, rfl⟩
      · exact Or.inl ⟨hW, by rw [Set.inter_self]⟩
      · exact Or.inr (Or.inl ⟨X, X', hrel, hWeq, by rw [sumTokMaster_inter_embF (f₀.
    rel_cod hrel)]⟩)
      · exact Or.inr (Or.inr ⟨Y, Y', hrel, hWeq, by rw [sumTokMaster_inter_embT (f₁.
    rel_cod hrel)]⟩)
    · rcases h2 with ⟨-, rfl⟩ | ⟨X₂, X'₂, hrel₂, hWeq, rfl⟩ | ⟨Y₂, Y'₂, hrel₂, hWeq, rfl⟩
      · refine Or.inr (Or.inl ⟨X, X', hrel, rfl, ?_⟩)
        rw [Set.inter_comm, sumTokMaster_inter_embF (f₀.rel_cod hrel)]
      · obtain rfl := embBit_injective hWeq
        exact Or.inr (Or.inl ⟨X, X' ∩ X'₂, f₀.inter_right hrel hrel₂, rfl, embBit_inter
     false X' X'₂⟩)
      · exact absurd hWeq (embBit_ne (by decide) (A₀.ne X (f₀.rel_dom hrel)))
    · rcases h2 with ⟨-, rfl⟩ | ⟨X₂, X'₂, hrel₂, hWeq, rfl⟩ | ⟨Y₂, Y'₂, hrel₂, hWeq, rfl⟩
      · refine Or.inr (Or.inr ⟨Y, Y', hrel, rfl, ?_⟩)
        rw [Set.inter_comm, sumTokMaster_inter_embT (f₁.rel_cod hrel)]
      · exact absurd hWeq (embBit_ne (by decide) (A₁.ne Y (f₁.rel_dom hrel)))
      · obtain rfl := embBit_injective hWeq
        exact Or.inr (Or.inr ⟨Y, Y' ∩ Y'₂, f₁.inter_right hrel hrel₂, rfl, embBit_inter
     true Y' Y'₂⟩)
  mono := by
    rintro W W'' Z Z' h hWW hZZ' hW'' hZ'
    rcases h with ⟨-, rfl⟩ | ⟨X, X', hrel, rfl, rfl⟩ | ⟨Y, Y', hrel, rfl, rfl⟩
    ·
      exact Or.inl ⟨hW'', Set.Subset.antisymm ((B₀.sum B₁).sys.sub_master hZ') hZZ'⟩
    ·
      rcases hZ' with rfl | ⟨X₃, hX₃, rfl⟩ | ⟨Y₃, hY₃, rfl⟩
      · exact Or.inl ⟨hW'', rfl⟩
      · rcases hW'' with rfl | ⟨X₂, hX₂, rfl⟩ | ⟨Y₂, hY₂, rfl⟩
        · exact absurd (hWW nil_mem_sumTokMaster) nil_not_mem_embBit
        · exact Or.inr (Or.inl ⟨X₂, X₃,
```

```
            f₀.mono hrel (embBit_subset.mp hWW) (embBit_subset.mp hZZ') hX₂ hX₃, rfl,
    rfl⟩)
        · exact absurd hWW (fun hsub => embBit_not_subset_cross (by decide) (A₁.ne Y₂
    hY₂) hsub)
      · exact absurd hZZ' (fun hsub =>
          embBit_not_subset_cross (by decide) (B₀.ne X' (f₀.rel_cod hrel)) hsub)
    ·
      rcases hZ' with rfl | ⟨X₃, hX₃, rfl⟩ | ⟨Y₃, hY₃, rfl⟩
      · exact Or.inl ⟨hW'', rfl⟩
      · exact absurd hZZ' (fun hsub =>
          embBit_not_subset_cross (by decide) (B₁.ne Y' (f₁.rel_cod hrel)) hsub)
      · rcases hW'' with rfl | ⟨X₂, hX₂, rfl⟩ | ⟨Y₂, hY₂, rfl⟩
        · exact absurd (hWW nil_mem_sumTokMaster) nil_not_mem_embBit
        · exact absurd hWW (fun hsub => embBit_not_subset_cross (by decide) (A₀.ne X₂
    hX₂) hsub)
        · exact Or.inr (Or.inr ⟨Y₂, Y₃,
            f₁.mono hrel (embBit_subset.mp hWW) (embBit_subset.mp hZZ') hY₂ hY₃, rfl,
    rfl⟩)

def FExpr.map : (T : FExpr) → {X Y : ScottSys} → ApproximableMap X.sys Y.sys →
    ApproximableMap (T.obj X).sys (T.obj Y).sys
  | .const D, _, _, _ => idMap D.sys
  | .var, _, _, f => f
  | .sum T₀ T₁, _, _, f => sumMapTok (T₀.map f) (T₁.map f)
  | .prod T₀ T₁, _, _, f => prodMapTok (T₀.map f) (T₁.map f)

def ScottSys.tok (D : ScottSys) : Set Str := D.sys.master

theorem mFun_eq_master : (T : FExpr) → {Γ : Set Str} → (h : Γ.Nonempty) →
    mFun T Γ = (T.obj (singletonSys Γ h)).sys.master
  | .const _, _, _ => rfl
  | .var, _, _ => rfl
  | .sum T₀ T₁, Γ, h => by
      show insert ([] : Str) (embBit false (mFun T₀ Γ) ∪ embBit true (mFun T₁ Γ))
        = insert ([] : Str) (embBit false ((T₀.obj (singletonSys Γ h)).sys.master)
            ∪ embBit true ((T₁.obj (singletonSys Γ h)).sys.master))
      rw [mFun_eq_master T₀ h, mFun_eq_master T₁ h]
  | .prod T₀ T₁, Γ, h => by
      show insert ([] : Str) (embBit false (mFun T₀ Γ) ∪ embBit true (mFun T₁ Γ))
        = insert ([] : Str) (embBit false ((T₀.obj (singletonSys Γ h)).sys.master)
            ∪ embBit true ((T₁.obj (singletonSys Γ h)).sys.master))
      rw [mFun_eq_master T₀ h, mFun_eq_master T₁ h]

theorem insertTag_mono {p q p' q' : Set Str} (hp : p ⊆ p') (hq : q ⊆ q') :
    insert ([] : Str) (embBit false p ∪ embBit true q)
      ⊆ insert ([] : Str) (embBit false p' ∪ embBit true q') := by
  rintro w (rfl | hw | hw)
  · exact Or.inl rfl
  · obtain ⟨w', rfl, hw'⟩ := hw
    exact Or.inr (Or.inl ⟨w', rfl, hp hw'⟩)
  · obtain ⟨w', rfl, hw'⟩ := hw
    exact Or.inr (Or.inr ⟨w', rfl, hq hw'⟩)

theorem mFun_mono (T : FExpr) {Γ Γ' : Set Str} (h : Γ ⊆ Γ') : mFun T Γ ⊆ mFun T Γ' :=
    by
  induction T with
```

```
  | const C => exact subset_rfl
  | var => exact h
  | sum T₀ T₁ ih₀ ih₁ => exact insertTag_mono ih₀ ih₁
  | prod T₀ T₁ ih₀ ih₁ => exact insertTag_mono ih₀ ih₁

theorem insertTag_continuous {? : Set (Set Str)} {U : Set Str} (hne : ?.Nonempty)
    {p q : Set Str → Set Str}
    (hp : ∀ w, w ∈ p U ↔ ∃ Γ ∈ ?, w ∈ p Γ)
    (hq : ∀ w, w ∈ q U ↔ ∃ Γ ∈ ?, w ∈ q Γ) (w : Str) :
    (w ∈ insert ([] : Str) (embBit false (p U) ∪ embBit true (q U)))
      ↔ ∃ Γ ∈ ?, w ∈ insert ([] : Str) (embBit false (p Γ) ∪ embBit true (q Γ)) := by
  simp only [Set.mem_insert_iff, Set.mem_union]
  constructor
  · rintro (rfl | hw | hw)
    · obtain ⟨Γ, hΓ⟩ := hne; exact ⟨Γ, hΓ, Or.inl rfl⟩
    · obtain ⟨w', rfl, hw'⟩ := hw
      obtain ⟨Γ, hΓ, hpΓ⟩ := (hp w').mp hw'
      exact ⟨Γ, hΓ, Or.inr (Or.inl ⟨w', rfl, hpΓ⟩)⟩
    · obtain ⟨w', rfl, hw'⟩ := hw
      obtain ⟨Γ, hΓ, hqΓ⟩ := (hq w').mp hw'
      exact ⟨Γ, hΓ, Or.inr (Or.inr ⟨w', rfl, hqΓ⟩)⟩
  · rintro ⟨Γ, hΓ, (rfl | hw | hw)⟩
    · exact Or.inl rfl
    · obtain ⟨w', rfl, hw'⟩ := hw
      exact Or.inr (Or.inl ⟨w', rfl, (hp w').mpr ⟨Γ, hΓ, hw'⟩⟩)
    · obtain ⟨w', rfl, hw'⟩ := hw
      exact Or.inr (Or.inr ⟨w', rfl, (hq w').mpr ⟨Γ, hΓ, hw'⟩⟩)

theorem mFun_continuous (T : FExpr) {? : Set (Set Str)} {U : Set Str}
    (_hdir : DirectedOn (· ⊆ ·) ?) (hne : ?.Nonempty)
    (hU : ∀ w, w ∈ U ↔ ∃ Γ ∈ ?, w ∈ Γ) :
    ∀ w, w ∈ mFun T U ↔ ∃ Γ ∈ ?, w ∈ mFun T Γ := by
  induction T with
  | const C =>
      intro w
      exact ⟨fun hw => let ⟨Γ, hΓ⟩ := hne; ⟨Γ, hΓ, hw⟩, fun ⟨_, _, hw⟩ => hw⟩
  | var => intro w; exact hU w
  | sum T₀ T₁ ih₀ ih₁ => intro w; exact insertTag_continuous hne ih₀ ih₁ w
  | prod T₀ T₁ ih₀ ih₁ => intro w; exact insertTag_continuous hne ih₀ ih₁ w

theorem mFun_nil_mem : ∀ (T : FExpr), T.RootedConst → {Γ : Set Str} →
    ([] : Str) ∈ Γ → ([] : Str) ∈ mFun T Γ
  | .const _, hC, _, _ => hC
  | .var, _, _, hΓ => hΓ
  | .sum _ _, _, _, _, _ => Set.mem_insert _ _
  | .prod _ _, _, _, _, _ => Set.mem_insert _ _

theorem nil_mem_mIter (T : FExpr) (hT : T.RootedConst) : ∀ n, ([] : Str) ∈ mIter T n
  | 0 => rfl
  | n + 1 => mFun_nil_mem T hT (nil_mem_mIter T hT n)

theorem mIter_mono_step (T : FExpr) (hT : T.RootedConst) :
    ∀ n, mIter T n ⊆ mIter T (n + 1)
  | 0 => by
      intro w hw
      have hw' : w = [] := hw
```

```
      subst hw'
      exact mFun_nil_mem T hT rfl
  | n + 1 => mFun_mono T (mIter_mono_step T hT n)

theorem mIter_mono (T : FExpr) (hT : T.RootedConst) {m n : ℕ} (hmn : m ≤ n) :
    mIter T m ⊆ mIter T n := by
  induction hmn with
  | refl => exact subset_rfl
  | step _ ih => intro x hx; exact mIter_mono_step T hT _ (ih hx)

theorem mFun_iter_fixed (T : FExpr) (hT : T.RootedConst) :
    mFun T (⋃ n, mIter T n) = ⋃ n, mIter T n := by
  have hstep := mIter_mono_step T hT
  have hne : (Set.range (mIter T)).Nonempty := ⟨mIter T 0, 0, rfl⟩
  have hdir : DirectedOn (· ⊆ ·) (Set.range (mIter T)) := by
    rintro _ ⟨i, rfl⟩ _ ⟨j, rfl⟩
    exact ⟨mIter T (max i j), ⟨max i j, rfl⟩,
      mIter_mono T hT (le_max_left i j), mIter_mono T hT (le_max_right i j)⟩
  have hU : ∀ v, v ∈ (⋃ n, mIter T n) ↔ ∃ S ∈ Set.range (mIter T), v ∈ S := by
    intro v
    constructor
    · intro hv; rw [Set.mem_iUnion] at hv; obtain ⟨n, hn⟩ := hv
      exact ⟨mIter T n, ⟨n, rfl⟩, hn⟩
    · rintro ⟨S, ⟨n, rfl⟩, hv⟩; exact Set.mem_iUnion.mpr ⟨n, hv⟩
  apply Set.ext; intro w
  rw [mFun_continuous T hdir hne hU w]
  constructor
  · rintro ⟨S, ⟨n, rfl⟩, hwS⟩; exact Set.mem_iUnion.mpr ⟨n + 1, hwS⟩
  · intro hw
    rw [Set.mem_iUnion] at hw; obtain ⟨n, hn⟩ := hw
    exact ⟨mIter T n, ⟨n, rfl⟩, hstep n hn⟩

theorem exists_tok_fixedPoint (T : FExpr) (hT : T.RootedConst) :
    ∃ Γ : Set Str, ([] : Str) ∈ Γ ∧ mFun T Γ = Γ :=
  ⟨⋃ n, mIter T n, Set.mem_iUnion.mpr ⟨0, nil_mem_mIter T hT 0⟩, mFun_iter_fixed T hT⟩

theorem exists_singleton_subsystem (T : FExpr) (hT : T.RootedConst) :
    ∃ (Γ : Set Str) (h : Γ.Nonempty),
      (singletonSys Γ h).sys ? (T.obj (singletonSys Γ h)).sys := by
  obtain ⟨Γ, hnil, hfix⟩ := exists_tok_fixedPoint T hT
  have hne : Γ.Nonempty := ⟨[], hnil⟩

  have hmaster : (T.obj (singletonSys Γ hne)).sys.master = Γ :=
    (mFun_eq_master T hne).symm.trans hfix
  refine ⟨Γ, hne, ?_, ?_, ?_⟩
  ·
    exact hmaster.symm
  ·
    intro X hX
    have heq : X = (T.obj (singletonSys Γ hne)).sys.master := (hX : X = Γ).trans
    hmaster.symm
    rw [heq]
    exact (T.obj (singletonSys Γ hne)).sys.master_mem
  ·
    intro X Y hX hY _
    show X ∩ Y = Γ
```

```
    rw [show X = Γ from hX, show Y = Γ from hY, Set.inter_self]

theorem prodMapTok_comp (f₀ : ApproximableMap A₀.sys B₀.sys) (f₁ : ApproximableMap A₁.
    sys B₁.sys)
    (g₀ : ApproximableMap B₀.sys C₀.sys) (g₁ : ApproximableMap B₁.sys C₁.sys) :
    prodMapTok (g₀.comp f₀) (g₁.comp f₁) = (prodMapTok g₀ g₁).comp (prodMapTok f₀ f₁) :=
     by
  apply ApproximableMap.ext
  intro W W''
  constructor
  · rintro ⟨X, Y, X'', Y'', ⟨X', hf0, hg0⟩, ⟨Y', hf1, hg1⟩, rfl, rfl⟩
    exact ⟨prodTokNbhd X' Y', ⟨X, Y, X', Y', hf0, hf1, rfl, rfl⟩,
      ⟨X', Y', X'', Y'', hg0, hg1, rfl, rfl⟩⟩
  · rintro ⟨W', ⟨X, Y, X', Y', hf0, hf1, rfl, rfl⟩, ⟨X₂, Y₂, X'', Y'', hg0, hg1, hWeq,
    rfl⟩⟩
    obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective hWeq
    exact ⟨X, Y, X'', Y'', ⟨X', hf0, hg0⟩, ⟨Y', hf1, hg1⟩, rfl, rfl⟩

theorem prodMapTok_id :
    prodMapTok (idMap A₀.sys) (idMap A₁.sys) = idMap (A₀.prod A₁).sys := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  · rintro ⟨X, Y, X', Y', ⟨hX, hX', hsX⟩, ⟨hY, hY', hsY⟩, rfl, rfl⟩
    exact ⟨prodTok_mem_prodTokNbhd hX hY, prodTok_mem_prodTokNbhd hX' hY',
      prodTokNbhd_subset_iff.mpr ⟨hsX, hsY⟩⟩
  · rintro ⟨⟨X, Y, hX, hY, rfl⟩, ⟨X', Y', hX', hY', rfl⟩, hsub⟩
    obtain ⟨hsX, hsY⟩ := prodTokNbhd_subset_iff.mp hsub
    exact ⟨X, Y, X', Y', ⟨hX, hX', hsX⟩, ⟨hY, hY', hsY⟩, rfl, rfl⟩

theorem prodMapTok_isStrict {f₀ : ApproximableMap A₀.sys B₀.sys}
    {f₁ : ApproximableMap A₁.sys B₁.sys} (hf₀ : IsStrict f₀) (hf₁ : IsStrict f₁) :
    IsStrict (prodMapTok f₀ f₁) := by
  rintro Y ⟨X, Y0, X', Y', h0, h1, hWeq, rfl⟩
  obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective hWeq
  rw [hf₀ h0, hf₁ h1]
  rfl

theorem prodMapTok_mono {f₀ f₀' : ApproximableMap A₀.sys B₀.sys}
    {f₁ f₁' : ApproximableMap A₁.sys B₁.sys} (h0 : f₀ ≤ f₀') (h1 : f₁ ≤ f₁') :
    prodMapTok f₀ f₁ ≤ prodMapTok f₀' f₁' := by
  rw [ApproximableMap.le_iff]
  rintro W W' ⟨X, Y, X', Y', hr0, hr1, rfl, rfl⟩
  exact ⟨X, Y, X', Y', h0 X X' hr0, h1 Y Y' hr1, rfl, rfl⟩

theorem prodTok_subsystem (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    (A₀.prod A₁).sys ? (B₀.prod B₁).sys := by
  have heqm : prodTokNbhd A₀.sys.master A₁.sys.master
      = prodTokNbhd B₀.sys.master B₁.sys.master := by rw [h0.master_eq, h1.master_eq]
  refine ⟨heqm, ?_, ?_⟩
  · rintro W ⟨X, Y, hX, hY, rfl⟩
    exact ⟨X, Y, h0.sub hX, h1.sub hY, rfl⟩

  · rintro W W' ⟨X, Y, hX, hY, rfl⟩ ⟨X', Y', hX', hY', rfl⟩ hInt
    rw [prodTokNbhd_inter] at hInt ⊢
    obtain ⟨X'', Y'', hX'', hY'', heq⟩ := hInt
```

```
    obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective heq
    exact ⟨X ∩ X', Y ∩ Y', h0.inter_closed hX hX' hX'', h1.inter_closed hY hY' hY'',
    rfl⟩

theorem sumMapTok_comp (f₀ : ApproximableMap A₀.sys B₀.sys) (f₁ : ApproximableMap A₁.sys
     B₁.sys)
    (g₀ : ApproximableMap B₀.sys C₀.sys) (g₁ : ApproximableMap B₁.sys C₁.sys) :
    sumMapTok (g₀.comp f₀) (g₁.comp f₁) = (sumMapTok g₀ g₁).comp (sumMapTok f₀ f₁) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, X'', ⟨X', hf, hg⟩, rfl, rfl⟩ | ⟨Y, Y'', ⟨Y', hf, hg⟩, rfl, rfl
    ⟩)
    · exact ⟨sumTokMaster B₀.sys B₁.sys, Or.inl ⟨hW, rfl⟩,
        Or.inl ⟨(B₀.sum B₁).sys.master_mem, rfl⟩⟩
    · exact ⟨embBit false X', Or.inr (Or.inl ⟨X, X', hf, rfl, rfl⟩),
        Or.inr (Or.inl ⟨X', X'', hg, rfl, rfl⟩)⟩
    · exact ⟨embBit true Y', Or.inr (Or.inr ⟨Y, Y', hf, rfl, rfl⟩),
        Or.inr (Or.inr ⟨Y', Y'', hg, rfl, rfl⟩)⟩
  · rintro ⟨W', hWW', hW'W''⟩
    rcases hWW' with ⟨hW, rfl⟩ | ⟨X, X', hf, rfl, rfl⟩ | ⟨Y, Y', hf, rfl, rfl⟩
    · rcases hW'W'' with ⟨-, rfl⟩ | ⟨X, X', -, heq, -⟩ | ⟨Y, Y', -, heq, -⟩
      · exact Or.inl ⟨hW, rfl⟩
      · exact absurd (heq ▶ nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact absurd (heq ▶ nil_mem_sumTokMaster) nil_not_mem_embBit
    · rcases hW'W'' with ⟨-, rfl⟩ | ⟨X₂, X'', hg, heq, rfl⟩ | ⟨Y₂, Y'', hg, heq, -⟩
      · exact Or.inl ⟨Or.inr (Or.inl ⟨X, f₀.rel_dom hf, rfl⟩), rfl⟩
      · obtain rfl := embBit_injective heq
        exact Or.inr (Or.inl ⟨X, X'', ⟨X', hf, hg⟩, rfl, rfl⟩)
      · exact absurd heq (embBit_ne (by decide) (B₀.ne X' (f₀.rel_cod hf)))
    · rcases hW'W'' with ⟨-, rfl⟩ | ⟨X₂, X'', hg, heq, -⟩ | ⟨Y₂, Y'', hg, heq, rfl⟩
      · exact Or.inl ⟨Or.inr (Or.inr ⟨Y, f₁.rel_dom hf, rfl⟩), rfl⟩
      · exact absurd heq (embBit_ne (by decide) (B₁.ne Y' (f₁.rel_cod hf)))
      · obtain rfl := embBit_injective heq
        exact Or.inr (Or.inr ⟨Y, Y'', ⟨Y', hf, hg⟩, rfl, rfl⟩)

theorem FExpr.map_comp : (T : FExpr) → {X Y Z : ScottSys} → (f : ApproximableMap X.
    sys Y.sys) →
    (g : ApproximableMap Y.sys Z.sys) → T.map (g.comp f) = (T.map g).comp (T.map f)
  | .const D, _, _, _, _, _ => (idMap_comp (idMap D.sys)).symm
  | .var, _, _, _, _, _ => rfl
  | .sum T₀ T₁, _, _, _, f, g => by
      show sumMapTok (T₀.map (g.comp f)) (T₁.map (g.comp f))
          = (sumMapTok (T₀.map g) (T₁.map g)).comp (sumMapTok (T₀.map f) (T₁.map f))
      rw [T₀.map_comp f g, T₁.map_comp f g, sumMapTok_comp]
  | .prod T₀ T₁, _, _, _, f, g => by
      show prodMapTok (T₀.map (g.comp f)) (T₁.map (g.comp f))
          = (prodMapTok (T₀.map g) (T₁.map g)).comp (prodMapTok (T₀.map f) (T₁.map f))
      rw [T₀.map_comp f g, T₁.map_comp f g, prodMapTok_comp]

theorem sumMapTok_id :
    sumMapTok (idMap A₀.sys) (idMap A₁.sys) = idMap (A₀.sum A₁).sys := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, X', ⟨hX, hX', hsub⟩, rfl, rfl⟩ | ⟨Y, Y', ⟨hY, hY', hsub⟩, rfl,
```

```
     rfl⟩)
    · exact ⟨hW, (A₀.sum A₁).sys.master_mem, (A₀.sum A₁).sys.sub_master hW⟩
    · exact ⟨Or.inr (Or.inl ⟨X, hX, rfl⟩), Or.inr (Or.inl ⟨X', hX', rfl⟩), embBit_subset.
    mpr hsub⟩
    · exact ⟨Or.inr (Or.inr ⟨Y, hY, rfl⟩), Or.inr (Or.inr ⟨Y', hY', rfl⟩), embBit_subset.
    mpr hsub⟩
  · rintro ⟨hW, hW', hsub⟩
    rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
    · exact Or.inl ⟨hW, rfl⟩
    · rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact Or.inr (Or.inl ⟨X, X', ⟨hX, hX', embBit_subset.mp hsub⟩, rfl, rfl⟩)
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A₁.ne Y hY) h)
    · rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A₀.ne X hX) h)
      · exact Or.inr (Or.inr ⟨Y, Y', ⟨hY, hY', embBit_subset.mp hsub⟩, rfl, rfl⟩)

theorem FExpr.map_id : (T : FExpr) → (X : ScottSys) → T.map (idMap X.sys) = idMap (T.
    obj X).sys
  | .const D, _ => rfl
  | .var, _ => rfl
  | .sum T₀ T₁, X => by
      show sumMapTok (T₀.map (idMap X.sys)) (T₁.map (idMap X.sys))
          = idMap ((T₀.obj X).sum (T₁.obj X)).sys
      rw [T₀.map_id X, T₁.map_id X, sumMapTok_id]
  | .prod T₀ T₁, X => by
      show prodMapTok (T₀.map (idMap X.sys)) (T₁.map (idMap X.sys))
          = idMap ((T₀.obj X).prod (T₁.obj X)).sys
      rw [T₀.map_id X, T₁.map_id X, prodMapTok_id]

theorem sumMapTok_isStrict (f₀ : ApproximableMap A₀.sys B₀.sys)
    (f₁ : ApproximableMap A₁.sys B₁.sys) : IsStrict (sumMapTok f₀ f₁) := by
  rintro Y (⟨-, rfl⟩ | ⟨X, X', -, heq, -⟩ | ⟨Y0, Y', -, heq, -⟩)
  · rfl
  · have heq' : sumTokMaster A₀.sys A₁.sys = embBit false X := heq
    exact absurd (heq' ▶ nil_mem_sumTokMaster) nil_not_mem_embBit
  · have heq' : sumTokMaster A₀.sys A₁.sys = embBit true Y0 := heq
    exact absurd (heq' ▶ nil_mem_sumTokMaster) nil_not_mem_embBit

theorem FExpr.map_isStrict : (T : FExpr) → {X Y : ScottSys} → (f : ApproximableMap X.
    sys Y.sys) →
    IsStrict f → IsStrict (T.map f)
  | .const _, _, _, _, _ => isStrict_idMap
  | .var, _, _, _, hf => hf
  | .sum T₀ T₁, _, _, f, _ => sumMapTok_isStrict (T₀.map f) (T₁.map f)
  | .prod T₀ T₁, _, _, f, hf =>
      prodMapTok_isStrict (T₀.map_isStrict f hf) (T₁.map_isStrict f hf)

theorem sumMapTok_mono {f₀ f₀' : ApproximableMap A₀.sys B₀.sys}
    {f₁ f₁' : ApproximableMap A₁.sys B₁.sys} (h0 : f₀ ≤ f₀') (h1 : f₁ ≤ f₁') :
    sumMapTok f₀ f₁ ≤ sumMapTok f₀' f₁' := by
  rw [ApproximableMap.le_iff]
  rintro W W' (⟨hW, rfl⟩ | ⟨X, X', hrel, rfl, rfl⟩ | ⟨Y, Y', hrel, rfl, rfl⟩)
  · exact Or.inl ⟨hW, rfl⟩
  · exact Or.inr (Or.inl ⟨X, X', h0 X X' hrel, rfl, rfl⟩)
```

```
  · exact Or.inr (Or.inr ⟨Y, Y', h1 Y Y' hrel, rfl, rfl⟩)

theorem FExpr.map_mono : (T : FExpr) → {X Y : ScottSys} → {f f' : ApproximableMap X.
    sys Y.sys} →
    f ≤ f' → T.map f ≤ T.map f'
  | .const _, _, _, _, _, _ => le_rfl
  | .var, _, _, _, _, h => h
  | .sum T₀ T₁, _, _, _, _, h => sumMapTok_mono (T₀.map_mono h) (T₁.map_mono h)
  | .prod T₀ T₁, _, _, _, _, h => prodMapTok_mono (T₀.map_mono h) (T₁.map_mono h)

theorem FExpr.map_continuous_mp : (T : FExpr) → {I : Type} → {X Y : ScottSys} →
    {g : I → ApproximableMap X.sys Y.sys} → {f : ApproximableMap X.sys Y.sys} →
    [Nonempty I] → (∀ i j, ∃ k, g i ≤ g k ∧ g j ≤ g k) →
    (∀ A B, f.rel A B ↔ ∃ i, (g i).rel A B) →
    {A B : Set Str} → (T.map f).rel A B → ∃ i, (T.map (g i)).rel A B
  | .const _, _, _, _, _, _, _, _, _, _, _, hrel => by
      obtain ⟨i⟩ := ?Nonempty _?; exact ⟨i, hrel⟩
  | .var, _, _, _, _, _, _, hf, A, B, hrel => (hf A B).mp hrel
  | .sum T₀ T₁, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      rcases hrel with ⟨hA, rfl⟩ | ⟨P, P', hr, rfl, rfl⟩ | ⟨Q, Q', hr, rfl, rfl⟩
      · obtain ⟨i⟩ := ?Nonempty _?; exact ⟨i, Or.inl ⟨hA, rfl⟩⟩
      · obtain ⟨i, hi⟩ := T₀.map_continuous_mp hdir hf hr
        exact ⟨i, Or.inr (Or.inl ⟨P, P', hi, rfl, rfl⟩)⟩
      · obtain ⟨i, hi⟩ := T₁.map_continuous_mp hdir hf hr
        exact ⟨i, Or.inr (Or.inr ⟨Q, Q', hi, rfl, rfl⟩)⟩
  | .prod T₀ T₁, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      obtain ⟨P, Q, P', Q', hr0, hr1, rfl, rfl⟩ := hrel
      obtain ⟨i, hi⟩ := T₀.map_continuous_mp hdir hf hr0
      obtain ⟨j, hj⟩ := T₁.map_continuous_mp hdir hf hr1
      obtain ⟨k, hik, hjk⟩ := hdir i j
      exact ⟨k, P, Q, P', Q', (T₀.map_mono hik) P P' hi, (T₁.map_mono hjk) Q Q' hj, rfl
    , rfl⟩

theorem FExpr.map_continuous (T : FExpr) {I : Type} [Nonempty I] {X Y : ScottSys}
    (g : I → ApproximableMap X.sys Y.sys) (f : ApproximableMap X.sys Y.sys)
    (hdir : ∀ i j, ∃ k, g i ≤ g k ∧ g j ≤ g k)
    (hf : ∀ A B, f.rel A B ↔ ∃ i, (g i).rel A B) (A B : Set Str) :
    (T.map f).rel A B ↔ ∃ i, (T.map (g i)).rel A B := by
  refine ⟨T.map_continuous_mp hdir hf, ?_⟩
  rintro ⟨i, hi⟩
  have hgif : g i ≤ f := by
    rw [ApproximableMap.le_iff]; intro A' B' h; exact (hf A' B').mpr ⟨i, h⟩
  exact (T.map_mono hgif) A B hi

theorem sumTok_subsystem (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    (A₀.sum A₁).sys ? (B₀.sum B₁).sys := by
  have heqm : sumTokMaster A₀.sys A₁.sys = sumTokMaster B₀.sys B₁.sys := by
    unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]
  refine ⟨heqm, ?_, ?_⟩
  · rintro W (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩)
    · exact Or.inl heqm
    · exact Or.inr (Or.inl ⟨X, h0.sub hX, rfl⟩)
    · exact Or.inr (Or.inr ⟨Y, h1.sub hY, rfl⟩)
  · rintro W W' (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩) (rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
    ) hInt
    · rw [Set.inter_self]; exact Or.inl rfl
```

```
    · rw [sumTokMaster_inter_embF hX']; exact Or.inr (Or.inl ⟨X', hX', rfl⟩)
    · rw [sumTokMaster_inter_embT hY']; exact Or.inr (Or.inr ⟨Y', hY', rfl⟩)
    · rw [Set.inter_comm, sumTokMaster_inter_embF hX]; exact Or.inr (Or.inl ⟨X, hX, rfl⟩
    )
    · rw [embBit_inter] at hInt ⊢
      exact Or.inr (Or.inl ⟨X ∩ X',
        h0.inter_closed hX hX' (sumTok_mem_embF_inv (h₀ := B₀.ne) (h₁ := B₁.ne) hInt),
    rfl⟩)
    · rw [embBit_inter_ne (show (false : Bool) = true by decide)] at hInt
      exact absurd ((B₀.sum B₁).ne _ hInt) Set.not_nonempty_empty
    · rw [Set.inter_comm, sumTokMaster_inter_embT hY]; exact Or.inr (Or.inr ⟨Y, hY, rfl⟩
    )
    · rw [embBit_inter_ne (show (true : Bool) = false by decide)] at hInt
      exact absurd ((B₀.sum B₁).ne _ hInt) Set.not_nonempty_empty
    · rw [embBit_inter] at hInt ⊢
      exact Or.inr (Or.inr ⟨Y ∩ Y',
        h1.inter_closed hY hY' (sumTok_mem_embT_inv (h₀ := B₀.ne) (h₁ := B₁.ne) hInt),
    rfl⟩)

theorem FExpr.obj_subsystem : (T : FExpr) → {X Y : ScottSys} → X.sys ? Y.sys →
    (T.obj X).sys ? (T.obj Y).sys
  | .const D, _, _, _ => Subsystem.refl D.sys
  | .var, _, _, h => h
  | .sum T₀ T₁, _, _, h => sumTok_subsystem (T₀.obj_subsystem h) (T₁.obj_subsystem h)
  | .prod T₀ T₁, _, _, h => prodTok_subsystem (T₀.obj_subsystem h) (T₁.obj_subsystem h)

theorem FExpr.obj_continuous_mp : (T : FExpr) → {? : Set ScottSys} → {U : ScottSys}
    →
    DirectedOn (fun a b => a.sys ? b.sys) ? → ?.Nonempty →
    (∀ D ∈ ?, D.sys ? U.sys) → (∀ X, U.sys.mem X ↔ ∃ D ∈ ?, D.sys.mem X) →
    {W : Set Str} → (T.obj U).sys.mem W → ∃ D ∈ ?, (T.obj D).sys.mem W
  | .const _, _, _, _, hne, _, _, _, hmem => by
      obtain ⟨D, hD⟩ := hne; exact ⟨D, hD, hmem⟩
  | .var, _, _, _, _, hU, W, hmem => (hU W).mp hmem
  | .sum T₀ T₁, _, _, hdir, hne, hsub, hU, _, hmem => by
      rcases hmem with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
      · obtain ⟨D, hD⟩ := hne
        exact ⟨D, hD, Or.inl ((FExpr.sum T₀ T₁).obj_subsystem (hsub D hD)).master_eq.
    symm⟩
      · obtain ⟨D, hD, hXD⟩ := T₀.obj_continuous_mp hdir hne hsub hU hX
        exact ⟨D, hD, Or.inr (Or.inl ⟨X, hXD, rfl⟩)⟩
      · obtain ⟨D, hD, hYD⟩ := T₁.obj_continuous_mp hdir hne hsub hU hY
        exact ⟨D, hD, Or.inr (Or.inr ⟨Y, hYD, rfl⟩)⟩
  | .prod T₀ T₁, _, _, hdir, hne, hsub, hU, _, hmem => by
      obtain ⟨X, Y, hX, hY, rfl⟩ := hmem
      obtain ⟨D₁, hD₁, hXD⟩ := T₀.obj_continuous_mp hdir hne hsub hU hX
      obtain ⟨D₂, hD₂, hYD⟩ := T₁.obj_continuous_mp hdir hne hsub hU hY
      obtain ⟨D₃, hD₃, hr1, hr2⟩ := hdir D₁ hD₁ D₂ hD₂
      exact ⟨D₃, hD₃, X, Y, (T₀.obj_subsystem hr1).sub hXD, (T₁.obj_subsystem hr2).sub
    hYD, rfl⟩

theorem FExpr.obj_continuous (T : FExpr) {? : Set ScottSys} {U : ScottSys}
    (hdir : DirectedOn (fun a b => a.sys ? b.sys) ?) (hne : ?.Nonempty)
    (hsub : ∀ D ∈ ?, D.sys ? U.sys) (hU : ∀ X, U.sys.mem X ↔ ∃ D ∈ ?, D.sys.mem X)
    (W : Set Str) : (T.obj U).sys.mem W ↔ ∃ D ∈ ?, (T.obj D).sys.mem W := by
  refine ⟨T.obj_continuous_mp hdir hne hsub hU, ?_⟩
```

```
  rintro ⟨D, hD, hmem⟩
  exact (T.obj_subsystem (hsub D hD)).sub hmem

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}
variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}
variable {A₀ A₁ B₀ B₁ : ScottSys}
variable {C₀ C₁ : ScottSys}

inductive GExpr where
  | const : ScottSys → GExpr
  | var : GExpr
  | sum : GExpr → GExpr → GExpr
  | prod : GExpr → GExpr → GExpr
  | oplus : GExpr → GExpr → GExpr
  | otimes : GExpr → GExpr → GExpr

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

theorem inter_ne_of_ne_left {X X' Δ : Set Str} (hX : X ⊆ Δ) (hne : X = Δ) : X ∩ X' =
    Δ := by
  intro h
  exact hne (Set.Subset.antisymm hX (by rw [← h]; exact Set.inter_subset_left))

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

variable {A₀ A₁ B₀ B₁ : ScottSys}

variable {C₀ C₁ : ScottSys}

def GExpr.naryOp (op : GExpr → GExpr → GExpr) (a : GExpr) : List GExpr → GExpr
  | [] => a
  | b :: l => op a (GExpr.naryOp op b l)

def GExpr.naryProd : GExpr → List GExpr → GExpr := GExpr.naryOp GExpr.prod

def GExpr.narySum : GExpr → List GExpr → GExpr := GExpr.naryOp GExpr.sum
```

```
end FunctorTokens
end

end DomainTheory.NeighbourhoodSystems
end
```

4.7.46 Exercise 6.20 — tokens of a functor

NeighbourhoodSystems/DomainEquations/Exercise_6_20.lean

`Exercise_6_20.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.functorTokens

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap BitDom DomainTheory.NeighbourhoodSystems.
    FunctorTokens

/-- Scott's Exercise 6.20: `λΓ. tok(T({Γ}))` is continuous, hence a fixed point exists.
    -/
theorem example_620 :
    (∀ (T : FExpr), T.RootedConst → ∃ (Γ : Set Str), ([] : Str) ∈ Γ ∧ mFun T Γ = Γ) ∧
    (∀ (T : FExpr), T.RootedConst →
      ∃ (Γ : Set Str) (h : Γ.Nonempty),
        (singletonSys Γ h).sys ? (T.obj (singletonSys Γ h)).sys) :=
  ⟨@exists_tok_fixedPoint, @exists_singleton_subsystem⟩
```

EXERCISE 6.20. For any system $\mathcal{D}$ let $\mathrm{tok}(\mathcal{D})$ be the underlying set of tokens, so that $\mathcal{D}$ is a system over $\mathrm{tok}(\mathcal{D})$. For the category of Exercise 6.19 show that the function

$$\lambda\Gamma.\,\mathrm{tok}(T(\{\Gamma\}))$$

is continuous on the domain $\{\Gamma \subseteq \{0,1\}^* \mid \Lambda \in \Gamma\}$, where T is any of the functors generated in 6.19. Conclude that there must exist a set

$$\Gamma = \mathrm{tok}(T(\{\Gamma\})),$$

so that $\{\Gamma\} \triangleleft T(\{\Gamma\})$, and so 6.14 applies.

4.7.47 coalesceSmash — coalesced sum and smash

`coalesceSmash.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.functorTokens
public import DomainTheory.NeighbourhoodSystems.DomainEquations.continuousOnDomains
```

```
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.
    FunctorTokens
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
open DomainTheory.NeighbourhoodSystems.SmashProduct
section
namespace FunctorTokens
variable {D₀ D₁ : NeighborhoodSystem Str}

def oplusTok (D₀ D₁ : NeighborhoodSystem Str)
    (h₀ : ∀ X, D₀.mem X → X.Nonempty) (h₁ : ∀ Y, D₁.mem Y → Y.Nonempty) :
    NeighborhoodSystem Str where
  mem W := W = sumTokMaster D₀ D₁ ∨ (∃ X, D₀.mem X ∧ X = D₀.master ∧ W = embBit false X
    ) ∨
    (∃ Y, D₁.mem Y ∧ Y = D₁.master ∧ W = embBit true Y)
  master := sumTokMaster D₀ D₁
  master_nonempty := ⟨[], nil_mem_sumTokMaster⟩
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | ⟨X, hX, -, rfl⟩ | ⟨Y, hY, -, rfl⟩)
    · exact subset_rfl
    · exact embF_subset_sumTokMaster hX
    · exact embT_subset_sumTokMaster hY
  inter_mem := by
    have hne : ∀ W, (W = sumTokMaster D₀ D₁ ∨ (∃ X, D₀.mem X ∧ X = D₀.master ∧ W =
    embBit false X) ∨
        (∃ Y, D₁.mem Y ∧ Y = D₁.master ∧ W = embBit true Y)) → (W : Set Str).Nonempty
    := by
      rintro W (rfl | ⟨X, hX, -, rfl⟩ | ⟨Y, hY, -, rfl⟩)
      · exact ⟨[], nil_mem_sumTokMaster⟩
      · exact embBit_nonempty (h₀ X hX)
      · exact embBit_nonempty (h₁ Y hY)
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩
    · rcases hW' with rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩
      · rw [Set.inter_self]; exact Or.inl rfl
      · rw [sumTokMaster_inter_embF hX']; exact Or.inr (Or.inl ⟨X', hX', hX'ne, rfl⟩)
      · rw [sumTokMaster_inter_embT hY']; exact Or.inr (Or.inr ⟨Y', hY', hY'ne, rfl⟩)
    · rcases hW' with rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩
      · rw [Set.inter_comm, sumTokMaster_inter_embF hX]
        exact Or.inr (Or.inl ⟨X, hX, hXne, rfl⟩)
      · rw [embBit_inter] at hZsub ⊢
        rcases hZ with rfl | ⟨Z₀, hZ₀, -, rfl⟩ | ⟨Z₁, hZ₁, -, rfl⟩
        · exact absurd (hZsub nil_mem_sumTokMaster) nil_not_mem_embBit
        · exact Or.inr (Or.inl ⟨X ∩ X', D₀.inter_mem hX hX' hZ₀ (embBit_subset.mp hZsub)
    ,
            inter_ne_of_ne_left (D₀.sub_master hX) hXne, rfl⟩)
        · obtain ⟨b, hb⟩ := h₁ Z₁ hZ₁
          obtain ⟨w', he, -⟩ := hZsub (⟨b, rfl, hb⟩ : (true :: b) ∈ embBit true Z₁)
          simp only [List.cons.injEq] at he; exact absurd he.1 (by decide)
      · rw [embBit_inter_ne (show (false : Bool) = true by decide)] at hZsub ⊢
```

```
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    · rcases hW' with rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩
      · rw [Set.inter_comm, sumTokMaster_inter_embT hY]
        exact Or.inr (Or.inr ⟨Y, hY, hYne, rfl⟩)
      · rw [embBit_inter_ne (show (true : Bool) = false by decide)] at hZsub ⊢
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      · rw [embBit_inter] at hZsub ⊢
        rcases hZ with rfl | ⟨Z₀, hZ₀, -, rfl⟩ | ⟨Z₁, hZ₁, -, rfl⟩
        · exact absurd (hZsub nil_mem_sumTokMaster) nil_not_mem_embBit
        · obtain ⟨a, ha⟩ := h₀ Z₀ hZ₀
          obtain ⟨w', he, -⟩ := hZsub (⟨a, rfl, ha⟩ : (false :: a) ∈ embBit false Z₀)
          simp only [List.cons.injEq] at he; exact absurd he.1 (by decide)
        · exact Or.inr (Or.inr ⟨Y ∩ Y', D₁.inter_mem hY hY' hZ₁ (embBit_subset.mp hZsub)
    ,
            inter_ne_of_ne_left (D₁.sub_master hY) hYne, rfl⟩)

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

theorem oplusTok_mem_embF {X : Set Str} (hX : D₀.mem X) (hXne : X = D₀.master) :
    (oplusTok D₀ D₁ h₀ h₁).mem (embBit false X) := Or.inr (Or.inl ⟨X, hX, hXne, rfl⟩)

theorem oplusTok_mem_embT {Y : Set Str} (hY : D₁.mem Y) (hYne : Y = D₁.master) :
    (oplusTok D₀ D₁ h₀ h₁).mem (embBit true Y) := Or.inr (Or.inr ⟨Y, hY, hYne, rfl⟩)

theorem oplusTok_nonempty : ∀ W, (oplusTok D₀ D₁ h₀ h₁).mem W → W.Nonempty := by
  rintro W (rfl | ⟨X, hX, -, rfl⟩ | ⟨Y, hY, -, rfl⟩)
  · exact ⟨[], nil_mem_sumTokMaster⟩
  · exact embBit_nonempty (h₀ X hX)
  · exact embBit_nonempty (h₁ Y hY)

def ScottSys.oplus (A₀ A₁ : ScottSys) : ScottSys :=
  ⟨oplusTok A₀.sys A₁.sys A₀.ne A₁.ne, oplusTok_nonempty⟩

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

variable {A₀ A₁ B₀ B₁ : ScottSys}

variable {C₀ C₁ : ScottSys}

def GExpr.naryOplus : GExpr → List GExpr → GExpr := GExpr.naryOp GExpr.oplus
```

```
def oplusMapTok (f₀ : ApproximableMap A₀.sys B₀.sys) (f₁ : ApproximableMap A₁.sys B₁.sys
    ) :
    ApproximableMap (A₀.oplus A₁).sys (B₀.oplus B₁).sys where
  rel W W' :=
    ((A₀.oplus A₁).sys.mem W ∧ W' = sumTokMaster B₀.sys B₁.sys) ∨
    (∃ X X', f₀.rel X X' ∧ X = A₀.sys.master ∧ X' = B₀.sys.master ∧
      W = embBit false X ∧ W' = embBit false X') ∨
    (∃ Y Y', f₁.rel Y Y' ∧ Y = A₁.sys.master ∧ Y' = B₁.sys.master ∧
      W = embBit true Y ∧ W' = embBit true Y')
  rel_dom := by
    rintro W W' (⟨hW, -⟩ | ⟨X, X', hrel, hXne, -, rfl, -⟩ | ⟨Y, Y', hrel, hYne, -, rfl,
    -⟩)
    · exact hW
    · exact oplusTok_mem_embF (h₀ := A₀.ne) (h₁ := A₁.ne) (f₀.rel_dom hrel) hXne
    · exact oplusTok_mem_embT (h₀ := A₀.ne) (h₁ := A₁.ne) (f₁.rel_dom hrel) hYne
  rel_cod := by
    rintro W W' (⟨-, rfl⟩ | ⟨X, X', hrel, -, hX'ne, -, rfl⟩ | ⟨Y, Y', hrel, -, hY'ne, -,
     rfl⟩)
    · exact Or.inl rfl
    · exact oplusTok_mem_embF (h₀ := B₀.ne) (h₁ := B₁.ne) (f₀.rel_cod hrel) hX'ne
    · exact oplusTok_mem_embT (h₀ := B₀.ne) (h₁ := B₁.ne) (f₁.rel_cod hrel) hY'ne
  master_rel := Or.inl ⟨(A₀.oplus A₁).sys.master_mem, rfl⟩
  inter_right := by
    rintro W W'₁ W'₂ h1 h2
    rcases h1 with ⟨hW, rfl⟩ | ⟨X, X', hrel, hXne, hX'ne, rfl, rfl⟩ | ⟨Y, Y', hrel, hYne
    , hY'ne, rfl, rfl⟩
    · rcases h2 with ⟨-, rfl⟩ | ⟨X, X', hrel, hXne, hX'ne, hWeq, rfl⟩ | ⟨Y, Y', hrel,
    hYne, hY'ne, hWeq, rfl⟩
      · exact Or.inl ⟨hW, by rw [Set.inter_self]⟩
      · exact Or.inr (Or.inl ⟨X, X', hrel, hXne, hX'ne, hWeq,
          by rw [sumTokMaster_inter_embF (f₀.rel_cod hrel)]⟩)
      · exact Or.inr (Or.inr ⟨Y, Y', hrel, hYne, hY'ne, hWeq,
          by rw [sumTokMaster_inter_embT (f₁.rel_cod hrel)]⟩)
    · rcases h2 with ⟨-, rfl⟩ | ⟨X₂, X'₂, hrel₂, hX₂ne, hX'₂ne, hWeq, rfl⟩ | ⟨Y₂, Y'₂,
    hrel₂, -, -, hWeq, rfl⟩
      · refine Or.inr (Or.inl ⟨X, X', hrel, hXne, hX'ne, rfl, ?_⟩)
        rw [Set.inter_comm, sumTokMaster_inter_embF (f₀.rel_cod hrel)]
      · obtain rfl := embBit_injective hWeq
        exact Or.inr (Or.inl ⟨X, X' ∩ X'₂, f₀.inter_right hrel hrel₂, hXne,
          inter_ne_of_ne_left (B₀.sys.sub_master (f₀.rel_cod hrel)) hX'ne, rfl,
          embBit_inter false X' X'₂⟩)
      · exact absurd hWeq (embBit_ne (by decide) (A₀.ne X (f₀.rel_dom hrel)))
    · rcases h2 with ⟨-, rfl⟩ | ⟨X₂, X'₂, hrel₂, -, -, hWeq, rfl⟩ | ⟨Y₂, Y'₂, hrel₂, hY₂ne
    , hY'₂ne, hWeq, rfl⟩
      · refine Or.inr (Or.inr ⟨Y, Y', hrel, hYne, hY'ne, rfl, ?_⟩)
        rw [Set.inter_comm, sumTokMaster_inter_embT (f₁.rel_cod hrel)]
      · exact absurd hWeq (embBit_ne (by decide) (A₁.ne Y (f₁.rel_dom hrel)))
      · obtain rfl := embBit_injective hWeq
        exact Or.inr (Or.inr ⟨Y, Y' ∩ Y'₂, f₁.inter_right hrel hrel₂, hYne,
          inter_ne_of_ne_left (B₁.sys.sub_master (f₁.rel_cod hrel)) hY'ne, rfl,
          embBit_inter true Y' Y'₂⟩)
  mono := by
    rintro W W'' Z Z' h hWW hZZ' hW'' hZ'
    rcases h with ⟨-, rfl⟩ | ⟨X, X', hrel, hXne, hX'ne, rfl, rfl⟩ | ⟨Y, Y', hrel, hYne,
    hY'ne, rfl, rfl⟩
```

```
    · exact Or.inl ⟨hW'', Set.Subset.antisymm ((B₀.oplus B₁).sys.sub_master hZ') hZZ'⟩
    · rcases hZ' with rfl | ⟨X₃, hX₃, hX₃ne, rfl⟩ | ⟨Y₃, hY₃, hY₃ne, rfl⟩
      · exact Or.inl ⟨hW'', rfl⟩
      · rcases hW'' with rfl | ⟨X₂, hX₂, hX₂ne, rfl⟩ | ⟨Y₂, hY₂, hY₂ne, rfl⟩
        · exact absurd (hWW nil_mem_sumTokMaster) nil_not_mem_embBit
        · exact Or.inr (Or.inl ⟨X₂, X₃,
            f₀.mono hrel (embBit_subset.mp hWW) (embBit_subset.mp hZZ') hX₂ hX₃,
            hX₂ne, hX₃ne, rfl, rfl⟩)
        · exact absurd hWW (fun hsub => embBit_not_subset_cross (by decide) (A₁.ne Y₂
  hY₂) hsub)
      · exact absurd hZZ' (fun hsub =>
          embBit_not_subset_cross (by decide) (B₀.ne X' (f₀.rel_cod hrel)) hsub)
    · rcases hZ' with rfl | ⟨X₃, hX₃, hX₃ne, rfl⟩ | ⟨Y₃, hY₃, hY₃ne, rfl⟩
      · exact Or.inl ⟨hW'', rfl⟩
      · exact absurd hZZ' (fun hsub =>
          embBit_not_subset_cross (by decide) (B₁.ne Y' (f₁.rel_cod hrel)) hsub)
      · rcases hW'' with rfl | ⟨X₂, hX₂, hX₂ne, rfl⟩ | ⟨Y₂, hY₂, hY₂ne, rfl⟩
        · exact absurd (hWW nil_mem_sumTokMaster) nil_not_mem_embBit
        · exact absurd hWW (fun hsub => embBit_not_subset_cross (by decide) (A₀.ne X₂
  hX₂) hsub)
        · exact Or.inr (Or.inr ⟨Y₂, Y₃,
            f₁.mono hrel (embBit_subset.mp hWW) (embBit_subset.mp hZZ') hY₂ hY₃,
            hY₂ne, hY₃ne, rfl, rfl⟩)

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

def otimesTok (D₀ D₁ : NeighborhoodSystem Str) : NeighborhoodSystem Str where
  mem W := W = prodTokNbhd D₀.master D₁.master ∨
    (∃ X Y, D₀.mem X ∧ D₁.mem Y ∧ X = D₀.master ∧ Y = D₁.master ∧ W = prodTokNbhd X Y)
  master := prodTokNbhd D₀.master D₁.master
  master_nonempty := ⟨[], mem_prodTokNbhd_nil⟩
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | ⟨X, Y, hX, hY, -, -, rfl⟩)
    · exact subset_rfl
    · exact prodTokNbhd_subset_iff.mpr ⟨D₀.sub_master hX, D₁.sub_master hY⟩
  inter_mem := by
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | ⟨X, Y, hX, hY, hXne, hYne, rfl⟩
    · rcases hW' with rfl | ⟨X', Y', hX', hY', hX'ne, hY'ne, rfl⟩
      · rw [Set.inter_self]; exact Or.inl rfl
      · rw [prodTokNbhd_inter, Set.inter_eq_right.mpr (D₀.sub_master hX'),
          Set.inter_eq_right.mpr (D₁.sub_master hY')]
        exact Or.inr ⟨X', Y', hX', hY', hX'ne, hY'ne, rfl⟩
    · rcases hW' with rfl | ⟨X', Y', hX', hY', hX'ne, hY'ne, rfl⟩
      · rw [Set.inter_comm, prodTokNbhd_inter, Set.inter_eq_right.mpr (D₀.sub_master hX
  ),
          Set.inter_eq_right.mpr (D₁.sub_master hY)]
```

```
        exact Or.inr ⟨X, Y, hX, hY, hXne, hYne, rfl⟩
      · rw [prodTokNbhd_inter] at hZsub ⊢
        rcases hZ with rfl | ⟨Z₀, Z₁, hZ₀, hZ₁, -, -, rfl⟩
        · obtain ⟨hsub₀, -⟩ := prodTokNbhd_subset_iff.mp hZsub
          exact absurd (Set.Subset.antisymm (D₀.sub_master hX)
            (hsub₀.trans Set.inter_subset_left)) hXne
        · obtain ⟨hsub₀, hsub₁⟩ := prodTokNbhd_subset_iff.mp hZsub
          exact Or.inr ⟨X ∩ X', Y ∩ Y', D₀.inter_mem hX hX' hZ₀ hsub₀,
            D₁.inter_mem hY hY' hZ₁ hsub₁, inter_ne_of_ne_left (D₀.sub_master hX) hXne,
            inter_ne_of_ne_left (D₁.sub_master hY) hYne, rfl⟩

theorem otimesTok_mem_master : (otimesTok D₀ D₁).mem (prodTokNbhd D₀.master D₁.master)
    := Or.inl rfl

theorem otimesTok_mem_prod {X Y : Set Str} (hX : D₀.mem X) (hY : D₁.mem Y)
    (hXne : X = D₀.master) (hYne : Y = D₁.master) :
    (otimesTok D₀ D₁).mem (prodTokNbhd X Y) := Or.inr ⟨X, Y, hX, hY, hXne, hYne, rfl⟩

theorem otimesTok_nonempty : ∀ W, (otimesTok D₀ D₁).mem W → W.Nonempty := by
  rintro W (rfl | ⟨X, Y, -, -, -, -, rfl⟩) <;> exact ⟨[], mem_prodTokNbhd_nil⟩

def ScottSys.otimes (A₀ A₁ : ScottSys) : ScottSys :=
  ⟨otimesTok A₀.sys A₁.sys, otimesTok_nonempty⟩

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

variable {A₀ A₁ B₀ B₁ : ScottSys}

variable {C₀ C₁ : ScottSys}

def GExpr.RootedConst : GExpr → Prop
  | .const C => ([] : Str) ∈ C.sys.master
  | .var => True
  | .sum a b => a.RootedConst ∧ b.RootedConst
  | .prod a b => a.RootedConst ∧ b.RootedConst
  | .oplus a b => a.RootedConst ∧ b.RootedConst
  | .otimes a b => a.RootedConst ∧ b.RootedConst

def gFun : GExpr → Set Str → Set Str
  | .const C, _ => C.sys.master
  | .var, Γ => Γ
  | .sum a b, Γ => insert ([] : Str) (embBit false (gFun a Γ) ∪ embBit true (gFun b Γ)
    )
  | .prod a b, Γ => insert ([] : Str) (embBit false (gFun a Γ) ∪ embBit true (gFun b Γ
    ))
  | .oplus a b, Γ => insert ([] : Str) (embBit false (gFun a Γ) ∪ embBit true (gFun b
    Γ))
```

```
  | .otimes a b, Γ => insert ([] : Str) (embBit false (gFun a Γ) ∪ embBit true (gFun b
    Γ))

def gIter (T : GExpr) : ℕ → Set Str
  | 0 => {([] : Str)}
  | n + 1 => gFun T (gIter T n)

def GExpr.naryOtimes : GExpr → List GExpr → GExpr := GExpr.naryOp GExpr.otimes

def GExpr.obj : GExpr → ScottSys → ScottSys
  | .const D, _ => D
  | .var, X => X
  | .sum a b, X => (a.obj X).sum (b.obj X)
  | .prod a b, X => (a.obj X).prod (b.obj X)
  | .oplus a b, X => (a.obj X).oplus (b.obj X)
  | .otimes a b, X => (a.obj X).otimes (b.obj X)

def otimesMapTok (f₀ : ApproximableMap A₀.sys B₀.sys) (f₁ : ApproximableMap A₁.sys B₁.
    sys) :
    ApproximableMap (A₀.otimes A₁).sys (B₀.otimes B₁).sys where
  rel W W' :=
    ((A₀.otimes A₁).sys.mem W ∧ W' = prodTokNbhd B₀.sys.master B₁.sys.master) ∨
    (∃ X Y X' Y', f₀.rel X X' ∧ f₁.rel Y Y' ∧ X = A₀.sys.master ∧ Y = A₁.sys.master ∧
      X' = B₀.sys.master ∧ Y' = B₁.sys.master ∧ W = prodTokNbhd X Y ∧ W' = prodTokNbhd
    X' Y')
  rel_dom := by
    rintro W W' (⟨hW, -⟩ | ⟨X, Y, X', Y', h0, h1, hXne, hYne, -, -, rfl, -⟩)
    · exact hW
    · exact otimesTok_mem_prod (f₀.rel_dom h0) (f₁.rel_dom h1) hXne hYne
  rel_cod := by
    rintro W W' (⟨-, rfl⟩ | ⟨X, Y, X', Y', h0, h1, -, -, hX'ne, hY'ne, -, rfl⟩)
    · exact otimesTok_mem_master
    · exact otimesTok_mem_prod (f₀.rel_cod h0) (f₁.rel_cod h1) hX'ne hY'ne
  master_rel := Or.inl ⟨(A₀.otimes A₁).sys.master_mem, rfl⟩
  inter_right := by
    rintro W W'₁ W'₂ h1 h2
    rcases h1 with ⟨hW, rfl⟩ | ⟨X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne, rfl,
    rfl⟩
    · rcases h2 with ⟨-, rfl⟩ | ⟨X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne, hWeq,
     rfl⟩
      · exact Or.inl ⟨hW, by rw [Set.inter_self]⟩
      · refine Or.inr ⟨X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne, hWeq, ?_⟩
        rw [prodTokNbhd_inter, Set.inter_eq_right.mpr (B₀.sys.sub_master (f₀.rel_cod
    hr0)),
          Set.inter_eq_right.mpr (B₁.sys.sub_master (f₁.rel_cod hr1))]
    · rcases h2 with ⟨-, rfl⟩ | ⟨X₂, Y₂, X'₂, Y'₂, hr0₂, hr1₂, hX₂ne, hY₂ne, hX'₂ne, hY'₂
    ne, hWeq, rfl⟩
      · refine Or.inr ⟨X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne, rfl, ?_⟩
        rw [Set.inter_comm, prodTokNbhd_inter,
          Set.inter_eq_right.mpr (B₀.sys.sub_master (f₀.rel_cod hr0)),
          Set.inter_eq_right.mpr (B₁.sys.sub_master (f₁.rel_cod hr1))]
      · obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective hWeq
        refine Or.inr ⟨X, Y, X' ∩ X'₂, Y' ∩ Y'₂, f₀.inter_right hr0 hr0₂,
          f₁.inter_right hr1 hr1₂, hXne, hYne,
          inter_ne_of_ne_left (B₀.sys.sub_master (f₀.rel_cod hr0)) hX'ne,
          inter_ne_of_ne_left (B₁.sys.sub_master (f₁.rel_cod hr1)) hY'ne, rfl, ?_⟩
```

```
        rw [prodTokNbhd_inter]
  mono := by
    rintro W W'' Z Z' h hWW hZZ' hZmem hZ'mem
    rcases h with ⟨-, rfl⟩ | ⟨X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne, rfl,
    rfl⟩
    · exact Or.inl ⟨hZmem, Set.Subset.antisymm ((B₀.otimes B₁).sys.sub_master hZ'mem)
    hZZ'⟩
    · rcases hZmem with rfl | ⟨Xz, Yz, hXz, hYz, hXzne, hYzne, rfl⟩
      · obtain ⟨hsubX, -⟩ := prodTokNbhd_subset_iff.mp hWW
        exact absurd (Set.Subset.antisymm (A₀.sys.sub_master (f₀.rel_dom hr0)) hsubX)
    hXne
      · rcases hZ'mem with rfl | ⟨Xz', Yz', hXz', hYz', hXz'ne, hYz'ne, rfl⟩
        · exact Or.inl ⟨otimesTok_mem_prod hXz hYz hXzne hYzne, rfl⟩
        · obtain ⟨hXzX, hYzY⟩ := prodTokNbhd_subset_iff.mp hWW
          obtain ⟨hX'Xz', hY'Yz'⟩ := prodTokNbhd_subset_iff.mp hZZ'
          exact Or.inr ⟨Xz, Yz, Xz', Yz', f₀.mono hr0 hXzX hX'Xz' hXz hXz',
            f₁.mono hr1 hYzY hY'Yz' hYz hYz', hXzne, hYzne, hXz'ne, hYz'ne, rfl, rfl⟩

def GExpr.map : (T : GExpr) → {X Y : ScottSys} → ApproximableMap X.sys Y.sys →
    ApproximableMap (T.obj X).sys (T.obj Y).sys
  | .const D, _, _, _ => idMap D.sys
  | .var, _, _, f => f
  | .sum a b, _, _, f => sumMapTok (a.map f) (b.map f)
  | .prod a b, _, _, f => prodMapTok (a.map f) (b.map f)
  | .oplus a b, _, _, f => oplusMapTok (a.map f) (b.map f)
  | .otimes a b, _, _, f => otimesMapTok (a.map f) (b.map f)

theorem gFun_eq_master : (T : GExpr) → {Γ : Set Str} → (h : Γ.Nonempty) →
    gFun T Γ = (T.obj (singletonSys Γ h)).sys.master
  | .const _, _, _ => rfl
  | .var, _, _ => rfl
  | .sum a b, Γ, h => by
      show insert ([] : Str) (embBit false (gFun a Γ) ∪ embBit true (gFun b Γ))
        = insert ([] : Str) (embBit false ((a.obj (singletonSys Γ h)).sys.master)
            ∪ embBit true ((b.obj (singletonSys Γ h)).sys.master))
      rw [gFun_eq_master a h, gFun_eq_master b h]
  | .prod a b, Γ, h => by
      show insert ([] : Str) (embBit false (gFun a Γ) ∪ embBit true (gFun b Γ))
        = insert ([] : Str) (embBit false ((a.obj (singletonSys Γ h)).sys.master)
            ∪ embBit true ((b.obj (singletonSys Γ h)).sys.master))
      rw [gFun_eq_master a h, gFun_eq_master b h]
  | .oplus a b, Γ, h => by
      show insert ([] : Str) (embBit false (gFun a Γ) ∪ embBit true (gFun b Γ))
        = insert ([] : Str) (embBit false ((a.obj (singletonSys Γ h)).sys.master)
            ∪ embBit true ((b.obj (singletonSys Γ h)).sys.master))
      rw [gFun_eq_master a h, gFun_eq_master b h]
  | .otimes a b, Γ, h => by
      show insert ([] : Str) (embBit false (gFun a Γ) ∪ embBit true (gFun b Γ))
        = insert ([] : Str) (embBit false ((a.obj (singletonSys Γ h)).sys.master)
            ∪ embBit true ((b.obj (singletonSys Γ h)).sys.master))
      rw [gFun_eq_master a h, gFun_eq_master b h]

theorem gFun_mono (T : GExpr) {Γ Γ' : Set Str} (h : Γ ⊆ Γ') : gFun T Γ ⊆ gFun T Γ' :=
    by
  induction T with
  | const C => exact subset_rfl
```

```
  | var => exact h
  | sum a b ih₀ ih₁ => exact insertTag_mono ih₀ ih₁
  | prod a b ih₀ ih₁ => exact insertTag_mono ih₀ ih₁
  | oplus a b ih₀ ih₁ => exact insertTag_mono ih₀ ih₁
  | otimes a b ih₀ ih₁ => exact insertTag_mono ih₀ ih₁

theorem gFun_continuous (T : GExpr) {? : Set (Set Str)} {U : Set Str}
    (_hdir : DirectedOn (· ⊆ ·) ?) (hne : ?.Nonempty)
    (hU : ∀ w, w ∈ U ↔ ∃ Γ ∈ ?, w ∈ Γ) :
    ∀ w, w ∈ gFun T U ↔ ∃ Γ ∈ ?, w ∈ gFun T Γ := by
  induction T with
  | const C =>
      intro w
      exact ⟨fun hw => let ⟨Γ, hΓ⟩ := hne; ⟨Γ, hΓ, hw⟩, fun ⟨_, _, hw⟩ => hw⟩
  | var => intro w; exact hU w
  | sum a b ih₀ ih₁ => intro w; exact insertTag_continuous hne ih₀ ih₁ w
  | prod a b ih₀ ih₁ => intro w; exact insertTag_continuous hne ih₀ ih₁ w
  | oplus a b ih₀ ih₁ => intro w; exact insertTag_continuous hne ih₀ ih₁ w
  | otimes a b ih₀ ih₁ => intro w; exact insertTag_continuous hne ih₀ ih₁ w

theorem gFun_nil_mem : ∀ (T : GExpr), T.RootedConst → {Γ : Set Str} →
    ([] : Str) ∈ Γ → ([] : Str) ∈ gFun T Γ
  | .const _, hC, _, _ => hC
  | .var, _, _, hΓ => hΓ
  | .sum _ _, _, _, _ => Set.mem_insert _ _
  | .prod _ _, _, _, _ => Set.mem_insert _ _
  | .oplus _ _, _, _, _ => Set.mem_insert _ _
  | .otimes _ _, _, _, _ => Set.mem_insert _ _

theorem nil_mem_gIter (T : GExpr) (hT : T.RootedConst) : ∀ n, ([] : Str) ∈ gIter T n
  | 0 => rfl
  | n + 1 => gFun_nil_mem T hT (nil_mem_gIter T hT n)

theorem gIter_mono_step (T : GExpr) (hT : T.RootedConst) :
    ∀ n, gIter T n ⊆ gIter T (n + 1)
  | 0 => by
      intro w hw


      have hw' : w = [] := hw
      subst hw'
      exact gFun_nil_mem T hT rfl
  | n + 1 => gFun_mono T (gIter_mono_step T hT n)

theorem gIter_mono (T : GExpr) (hT : T.RootedConst) {m n : ℕ} (hmn : m ≤ n) :
    gIter T m ⊆ gIter T n := by
  induction hmn with
  | refl => exact subset_rfl
  | step _ ih => intro x hx; exact gIter_mono_step T hT _ (ih hx)

theorem gFun_iter_fixed (T : GExpr) (hT : T.RootedConst) :
    gFun T (⋃ n, gIter T n) = ⋃ n, gIter T n := by
  have hstep := gIter_mono_step T hT
  have hne : (Set.range (gIter T)).Nonempty := ⟨gIter T 0, 0, rfl⟩
  have hdir : DirectedOn (· ⊆ ·) (Set.range (gIter T)) := by
    rintro _ ⟨i, rfl⟩ _ ⟨j, rfl⟩
    exact ⟨gIter T (max i j), ⟨max i j, rfl⟩,
```

```
      gIter_mono T hT (le_max_left i j), gIter_mono T hT (le_max_right i j)⟩
  have hU : ∀ v, v ∈ (⋃ n, gIter T n) ↔ ∃ S ∈ Set.range (gIter T), v ∈ S := by
    intro v
    constructor
    · intro hv; rw [Set.mem_iUnion] at hv; obtain ⟨n, hn⟩ := hv
      exact ⟨gIter T n, ⟨n, rfl⟩, hn⟩
    · rintro ⟨S, ⟨n, rfl⟩, hv⟩; exact Set.mem_iUnion.mpr ⟨n, hv⟩
  apply Set.ext; intro w
  rw [gFun_continuous T hdir hne hU w]
  constructor
  · rintro ⟨S, ⟨n, rfl⟩, hwS⟩; exact Set.mem_iUnion.mpr ⟨n + 1, hwS⟩
  · intro hw
    rw [Set.mem_iUnion] at hw; obtain ⟨n, hn⟩ := hw
    exact ⟨gIter T n, ⟨n, rfl⟩, hstep n hn⟩

theorem gExists_tok_fixedPoint (T : GExpr) (hT : T.RootedConst) :
    ∃ Γ : Set Str, ([] : Str) ∈ Γ ∧ gFun T Γ = Γ :=
  ⟨⋃ n, gIter T n, Set.mem_iUnion.mpr ⟨0, nil_mem_gIter T hT 0⟩, gFun_iter_fixed T hT⟩

theorem gExists_singleton_subsystem (T : GExpr) (hT : T.RootedConst) :
    ∃ (Γ : Set Str) (h : Γ.Nonempty),
      (singletonSys Γ h).sys ? (T.obj (singletonSys Γ h)).sys := by
  obtain ⟨Γ, hnil, hfix⟩ := gExists_tok_fixedPoint T hT
  have hne : Γ.Nonempty := ⟨[], hnil⟩
  have hmaster : (T.obj (singletonSys Γ hne)).sys.master = Γ :=
    (gFun_eq_master T hne).symm.trans hfix
  refine ⟨Γ, hne, ?_, ?_, ?_⟩
  · exact hmaster.symm
  · intro X hX
    have heq : X = (T.obj (singletonSys Γ hne)).sys.master := (hX : X = Γ).trans
    hmaster.symm
    rw [heq]
    exact (T.obj (singletonSys Γ hne)).sys.master_mem
  · intro X Y hX hY _
    show X ∩ Y = Γ
    rw [show X = Γ from hX, show Y = Γ from hY, Set.inter_self]

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

variable {A₀ A₁ B₀ B₁ : ScottSys}

variable {C₀ C₁ : ScottSys}

theorem GExpr.naryOp_rootedConst {op : GExpr → GExpr → GExpr}
    (hop : ∀ x y, (op x y).RootedConst ↔ x.RootedConst ∧ y.RootedConst) :
    ∀ (a : GExpr) (l : List GExpr), a.RootedConst → (∀ b ∈ l, b.RootedConst) →
      (GExpr.naryOp op a l).RootedConst
  | a, [], ha, _ => ha
  | a, b :: l, ha, hl => by
      rw [GExpr.naryOp, hop]
      exact ⟨ha, GExpr.naryOp_rootedConst hop b l (hl b (List.mem_cons_self ..))
        (fun c hc => hl c (List.mem_cons_of_mem _ hc))⟩

end FunctorTokens
```

```
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

theorem inter_ne_of_ne_right {X X' Δ : Set Str} (hX' : X' ⊆ Δ) (hne : X' = Δ) : X ∩ X
    ' = Δ := by
  intro h
  exact hne (Set.Subset.antisymm hX' (by rw [← h]; exact Set.inter_subset_right))

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

variable {A₀ A₁ B₀ B₁ : ScottSys}

variable {C₀ C₁ : ScottSys}

theorem naryOplus_singleton_subsystem (a : GExpr) (l : List GExpr)
    (ha : a.RootedConst) (hl : ∀ b ∈ l, b.RootedConst) :
    ∃ (Γ : Set Str) (h : Γ.Nonempty),
      (singletonSys Γ h).sys ? ((GExpr.naryOplus a l).obj (singletonSys Γ h)).sys :=
  gExists_singleton_subsystem _ (GExpr.naryOp_rootedConst (fun _ _ => Iff.rfl) a l ha
    hl)

theorem naryOtimes_singleton_subsystem (a : GExpr) (l : List GExpr)
    (ha : a.RootedConst) (hl : ∀ b ∈ l, b.RootedConst) :
    ∃ (Γ : Set Str) (h : Γ.Nonempty),
      (singletonSys Γ h).sys ? ((GExpr.naryOtimes a l).obj (singletonSys Γ h)).sys :=
  gExists_singleton_subsystem _ (GExpr.naryOp_rootedConst (fun _ _ => Iff.rfl) a l ha
    hl)

theorem naryProd_singleton_subsystem (a : GExpr) (l : List GExpr)
    (ha : a.RootedConst) (hl : ∀ b ∈ l, b.RootedConst) :
    ∃ (Γ : Set Str) (h : Γ.Nonempty),
      (singletonSys Γ h).sys ? ((GExpr.naryProd a l).obj (singletonSys Γ h)).sys :=
  gExists_singleton_subsystem _ (GExpr.naryOp_rootedConst (fun _ _ => Iff.rfl) a l ha
    hl)

theorem narySum_singleton_subsystem (a : GExpr) (l : List GExpr)
    (ha : a.RootedConst) (hl : ∀ b ∈ l, b.RootedConst) :
    ∃ (Γ : Set Str) (h : Γ.Nonempty),
      (singletonSys Γ h).sys ? ((GExpr.narySum a l).obj (singletonSys Γ h)).sys :=
  gExists_singleton_subsystem _ (GExpr.naryOp_rootedConst (fun _ _ => Iff.rfl) a l ha
    hl)

theorem oplusMapTok_comp (f₀ : ApproximableMap A₀.sys B₀.sys) (f₁ : ApproximableMap A₁.
    sys B₁.sys)
```

```
    {g₀ : ApproximableMap B₀.sys C₀.sys} {g₁ : ApproximableMap B₁.sys C₁.sys}
    (hg₀ : IsStrict g₀) (hg₁ : IsStrict g₁) :
    oplusMapTok (g₀.comp f₀) (g₁.comp f₁) = (oplusMapTok g₀ g₁).comp (oplusMapTok f₀ f₁)
     := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, X'', ⟨X', hf, hg⟩, hXne, hX''ne, rfl, rfl⟩ |
      ⟨Y, Y'', ⟨Y', hf, hg⟩, hYne, hY''ne, rfl, rfl⟩)
    · exact ⟨sumTokMaster B₀.sys B₁.sys, Or.inl ⟨hW, rfl⟩,
        Or.inl ⟨(B₀.oplus B₁).sys.master_mem, rfl⟩⟩
    · have hX'ne : X' = B₀.sys.master := fun h => hX''ne (hg₀ (h ▶ hg))
      exact ⟨embBit false X', Or.inr (Or.inl ⟨X, X', hf, hXne, hX'ne, rfl, rfl⟩),
        Or.inr (Or.inl ⟨X', X'', hg, hX'ne, hX''ne, rfl, rfl⟩)⟩
    · have hY'ne : Y' = B₁.sys.master := fun h => hY''ne (hg₁ (h ▶ hg))
      exact ⟨embBit true Y', Or.inr (Or.inr ⟨Y, Y', hf, hYne, hY'ne, rfl, rfl⟩),
        Or.inr (Or.inr ⟨Y', Y'', hg, hY'ne, hY''ne, rfl, rfl⟩)⟩
  · rintro ⟨W', hWW', hW'W''⟩
    rcases hWW' with ⟨hW, rfl⟩ | ⟨X, X', hf, hXne, hX'ne, rfl, rfl⟩ | ⟨Y, Y', hf, hYne,
    hY'ne, rfl, rfl⟩
    · rcases hW'W'' with ⟨-, rfl⟩ | ⟨X, X', -, -, -, heq, -⟩ | ⟨Y, Y', -, -, -, heq, -⟩
      · exact Or.inl ⟨hW, rfl⟩
      · exact absurd (heq ▶ nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact absurd (heq ▶ nil_mem_sumTokMaster) nil_not_mem_embBit
    · rcases hW'W'' with ⟨-, rfl⟩ | ⟨X₂, X'', hg, -, hX''ne, heq, rfl⟩ | ⟨Y₂, Y'', hg, -,
     -, heq, -⟩
      · exact Or.inl ⟨oplusTok_mem_embF (h₀ := A₀.ne) (h₁ := A₁.ne) (f₀.rel_dom hf) hXne,
     rfl⟩
      · obtain rfl := embBit_injective heq
        exact Or.inr (Or.inl ⟨X, X'', ⟨X', hf, hg⟩, hXne, hX''ne, rfl, rfl⟩)
      · exact absurd heq (embBit_ne (by decide) (B₀.ne X' (f₀.rel_cod hf)))
    · rcases hW'W'' with ⟨-, rfl⟩ | ⟨X₂, X'', hg, -, -, heq, -⟩ | ⟨Y₂, Y'', hg, -, hY''ne
    , heq, rfl⟩
      · exact Or.inl ⟨oplusTok_mem_embT (h₀ := A₀.ne) (h₁ := A₁.ne) (f₁.rel_dom hf) hYne,
     rfl⟩
      · exact absurd heq (embBit_ne (by decide) (B₁.ne Y' (f₁.rel_cod hf)))
      · obtain rfl := embBit_injective heq
        exact Or.inr (Or.inr ⟨Y, Y'', ⟨Y', hf, hg⟩, hYne, hY''ne, rfl, rfl⟩)

theorem oplusMapTok_id :
    oplusMapTok (idMap A₀.sys) (idMap A₁.sys) = idMap (A₀.oplus A₁).sys := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, X', ⟨hX, hX', hsub⟩, hXne, hX'ne, rfl, rfl⟩ |
      ⟨Y, Y', ⟨hY, hY', hsub⟩, hYne, hY'ne, rfl, rfl⟩)
    · exact ⟨hW, (A₀.oplus A₁).sys.master_mem, (A₀.oplus A₁).sys.sub_master hW⟩
    · exact ⟨oplusTok_mem_embF (h₀ := A₀.ne) (h₁ := A₁.ne) hX hXne,
        oplusTok_mem_embF (h₀ := A₀.ne) (h₁ := A₁.ne) hX' hX'ne, embBit_subset.mpr hsub⟩
    · exact ⟨oplusTok_mem_embT (h₀ := A₀.ne) (h₁ := A₁.ne) hY hYne,
        oplusTok_mem_embT (h₀ := A₀.ne) (h₁ := A₁.ne) hY' hY'ne, embBit_subset.mpr hsub⟩
  · rintro ⟨hW, hW', hsub⟩
    rcases hW' with rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩
    · exact Or.inl ⟨hW, rfl⟩
    · rcases hW with rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩
      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
```

```
      · exact Or.inr (Or.inl ⟨X, X', ⟨hX, hX', embBit_subset.mp hsub⟩, hXne, hX'ne, rfl,
     rfl⟩)
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A₁.ne Y hY) h)
    · rcases hW with rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩
      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A₀.ne X hX) h)
      · exact Or.inr (Or.inr ⟨Y, Y', ⟨hY, hY', embBit_subset.mp hsub⟩, hYne, hY'ne, rfl,
     rfl⟩)

theorem oplusMapTok_isStrict (f₀ : ApproximableMap A₀.sys B₀.sys)
    (f₁ : ApproximableMap A₁.sys B₁.sys) : IsStrict (oplusMapTok f₀ f₁) := by
  rintro Y (⟨-, rfl⟩ | ⟨X, X', -, -, -, heq, -⟩ | ⟨Y0, Y', -, -, -, heq, -⟩)
  · rfl
  · have heq' : sumTokMaster A₀.sys A₁.sys = embBit false X := heq
    exact absurd (heq' ▶ nil_mem_sumTokMaster) nil_not_mem_embBit
  · have heq' : sumTokMaster A₀.sys A₁.sys = embBit true Y0 := heq
    exact absurd (heq' ▶ nil_mem_sumTokMaster) nil_not_mem_embBit

theorem oplusMapTok_mono {f₀ f₀' : ApproximableMap A₀.sys B₀.sys}
    {f₁ f₁' : ApproximableMap A₁.sys B₁.sys} (h0 : f₀ ≤ f₀') (h1 : f₁ ≤ f₁') :
    oplusMapTok f₀ f₁ ≤ oplusMapTok f₀' f₁' := by
  rw [ApproximableMap.le_iff]
  rintro W W' (⟨hW, rfl⟩ | ⟨X, X', hrel, hXne, hX'ne, rfl, rfl⟩ |
    ⟨Y, Y', hrel, hYne, hY'ne, rfl, rfl⟩)
  · exact Or.inl ⟨hW, rfl⟩
  · exact Or.inr (Or.inl ⟨X, X', h0 X X' hrel, hXne, hX'ne, rfl, rfl⟩)
  · exact Or.inr (Or.inr ⟨Y, Y', h1 Y Y' hrel, hYne, hY'ne, rfl, rfl⟩)

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

theorem oplusTok_mem_embF_inv {W : Set Str} (h : (oplusTok D₀ D₁ h₀ h₁).mem (embBit
    false W)) :
    D₀.mem W := by
  rcases h with h0 | ⟨X, hX, -, heq⟩ | ⟨Y, hY, -, heq⟩
  · exact absurd (h0.symm ▶ nil_mem_sumTokMaster) nil_not_mem_embBit
  · rw [embBit_injective heq]; exact hX
  · exact absurd heq.symm (embBit_ne (show (true : Bool) = false by decide) (h₁ Y hY))

theorem oplusTok_mem_embT_inv {W : Set Str} (h : (oplusTok D₀ D₁ h₀ h₁).mem (embBit
    true W)) :
    D₁.mem W := by
  rcases h with h0 | ⟨X, hX, -, heq⟩ | ⟨Y, hY, -, heq⟩
  · exact absurd (h0.symm ▶ nil_mem_sumTokMaster) nil_not_mem_embBit
  · exact absurd heq.symm (embBit_ne (show (false : Bool) = true by decide) (h₀ X hX))
  · rw [embBit_injective heq]; exact hY

theorem oplusTok_mem_master : (oplusTok D₀ D₁ h₀ h₁).mem (sumTokMaster D₀ D₁) := Or.inl
    rfl
```

```
end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

variable {A₀ A₁ B₀ B₁ : ScottSys}

theorem oplusTok_subsystem (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    (A₀.oplus A₁).sys ? (B₀.oplus B₁).sys := by
  have heqm : sumTokMaster A₀.sys A₁.sys = sumTokMaster B₀.sys B₁.sys := by
    unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]
  refine ⟨heqm, ?_, ?_⟩
  · rintro W (rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩)
    · exact Or.inl heqm
    · exact Or.inr (Or.inl ⟨X, h0.sub hX, h0.master_eq ▸ hXne, rfl⟩)
    · exact Or.inr (Or.inr ⟨Y, h1.sub hY, h1.master_eq ▸ hYne, rfl⟩)
  · rintro W W' (rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩)
      (rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩) hInt
    · rw [Set.inter_self]; exact Or.inl rfl
    · rw [sumTokMaster_inter_embF hX']; exact Or.inr (Or.inl ⟨X', hX', hX'ne, rfl⟩)
    · rw [sumTokMaster_inter_embT hY']; exact Or.inr (Or.inr ⟨Y', hY', hY'ne, rfl⟩)
    · rw [Set.inter_comm, sumTokMaster_inter_embF hX]; exact Or.inr (Or.inl ⟨X, hX,
    hXne, rfl⟩)
    · rw [embBit_inter] at hInt ⊢
      exact Or.inr (Or.inl ⟨X ∩ X',
        h0.inter_closed hX hX' (oplusTok_mem_embF_inv (h₀ := B₀.ne) (h₁ := B₁.ne) hInt)
    ,
        inter_ne_of_ne_left (A₀.sys.sub_master hX) hXne, rfl⟩)
    · rw [embBit_inter_ne (show (false : Bool) = true by decide)] at hInt
      exact absurd ((B₀.oplus B₁).ne _ hInt) Set.not_nonempty_empty
    · rw [Set.inter_comm, sumTokMaster_inter_embT hY]; exact Or.inr (Or.inr ⟨Y, hY,
    hYne, rfl⟩)
    · rw [embBit_inter_ne (show (true : Bool) = false by decide)] at hInt
      exact absurd ((B₀.oplus B₁).ne _ hInt) Set.not_nonempty_empty
    · rw [embBit_inter] at hInt ⊢
      exact Or.inr (Or.inr ⟨Y ∩ Y',
        h1.inter_closed hY hY' (oplusTok_mem_embT_inv (h₀ := B₀.ne) (h₁ := B₁.ne) hInt)
    ,
        inter_ne_of_ne_left (A₁.sys.sub_master hY) hYne, rfl⟩)

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}
```

```
variable {A₀ A₁ B₀ B₁ : ScottSys}

variable {C₀ C₁ : ScottSys}

theorem otimesMapTok_comp (f₀ : ApproximableMap A₀.sys B₀.sys) (f₁ : ApproximableMap A₁.
    sys B₁.sys)
    {g₀ : ApproximableMap B₀.sys C₀.sys} {g₁ : ApproximableMap B₁.sys C₁.sys}
    (hg₀ : IsStrict g₀) (hg₁ : IsStrict g₁) :
    otimesMapTok (g₀.comp f₀) (g₁.comp f₁) = (otimesMapTok g₀ g₁).comp (otimesMapTok f₀
    f₁) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, Y, X'', Y'', ⟨X', hf0, hg0⟩, ⟨Y', hf1, hg1⟩,
      hXne, hYne, hX''ne, hY''ne, rfl, rfl⟩)
    · exact ⟨prodTokNbhd B₀.sys.master B₁.sys.master, Or.inl ⟨hW, rfl⟩,
        Or.inl ⟨(B₀.otimes B₁).sys.master_mem, rfl⟩⟩
    · have hX'ne : X' = B₀.sys.master := fun h => hX''ne (hg₀ (h ▶ hg0))
      have hY'ne : Y' = B₁.sys.master := fun h => hY''ne (hg₁ (h ▶ hg1))
      exact ⟨prodTokNbhd X' Y',
        Or.inr ⟨X, Y, X', Y', hf0, hf1, hXne, hYne, hX'ne, hY'ne, rfl, rfl⟩,
        Or.inr ⟨X', Y', X'', Y'', hg0, hg1, hX'ne, hY'ne, hX''ne, hY''ne, rfl, rfl⟩⟩
  · rintro ⟨W', hWW', hW'W''⟩
    rcases hWW' with ⟨hW, rfl⟩ | ⟨X, Y, X', Y', hf0, hf1, hXne, hYne, hX'ne, hY'ne, rfl
    , rfl⟩
    · rcases hW'W'' with ⟨-, rfl⟩ | ⟨Xg, Yg, X'', Y'', -, -, hXgne, -, -, -, heq, -⟩
      · exact Or.inl ⟨hW, rfl⟩
      · have heq' : prodTokNbhd B₀.sys.master B₁.sys.master = prodTokNbhd Xg Yg := heq
        obtain ⟨h, -⟩ := prodTokNbhd_injective heq'
        exact absurd h.symm hXgne
    · rcases hW'W'' with ⟨-, rfl⟩ |
        ⟨Xg, Yg, X'', Y'', hg0, hg1, hXgne, hYgne, hX''ne, hY''ne, heq, rfl⟩
      · exact Or.inl ⟨otimesTok_mem_prod (f₀.rel_dom hf0) (f₁.rel_dom hf1) hXne hYne,
    rfl⟩
      · obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective heq
        exact Or.inr ⟨X, Y, X'', Y'', ⟨X', hf0, hg0⟩, ⟨Y', hf1, hg1⟩,
          hXne, hYne, hX''ne, hY''ne, rfl, rfl⟩

theorem otimesMapTok_id :
    otimesMapTok (idMap A₀.sys) (idMap A₁.sys) = idMap (A₀.otimes A₁).sys := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, Y, X', Y', ⟨hX, hX', hsubX⟩, ⟨hY, hY', hsubY⟩,
      hXne, hYne, hX'ne, hY'ne, rfl, rfl⟩)
    · exact ⟨hW, (A₀.otimes A₁).sys.master_mem, (A₀.otimes A₁).sys.sub_master hW⟩
    · exact ⟨otimesTok_mem_prod hX hY hXne hYne, otimesTok_mem_prod hX' hY' hX'ne hY'ne
    ,
        prodTokNbhd_subset_iff.mpr ⟨hsubX, hsubY⟩⟩
  · rintro ⟨hW, hW', hsub⟩
    rcases hW' with rfl | ⟨X', Y', hX', hY', hX'ne, hY'ne, rfl⟩
    · exact Or.inl ⟨hW, rfl⟩
    · rcases hW with rfl | ⟨X, Y, hX, hY, hXne, hYne, rfl⟩
      · obtain ⟨hsubX, -⟩ := prodTokNbhd_subset_iff.mp hsub
        exact absurd (Set.Subset.antisymm (A₀.sys.sub_master hX') hsubX) hX'ne
      · obtain ⟨hsX, hsY⟩ := prodTokNbhd_subset_iff.mp hsub
```

```
        exact Or.inr ⟨X, Y, X', Y', ⟨hX, hX', hsX⟩, ⟨hY, hY', hsY⟩,
          hXne, hYne, hX'ne, hY'ne, rfl, rfl⟩

theorem GExpr.map_id : (T : GExpr) → (X : ScottSys) → T.map (idMap X.sys) = idMap (T.
    obj X).sys
  | .const _, _ => rfl
  | .var, _ => rfl
  | .sum a b, X => by
      show sumMapTok (a.map (idMap X.sys)) (b.map (idMap X.sys)) = idMap ((a.obj X).
    sum (b.obj X)).sys
      rw [a.map_id X, b.map_id X, sumMapTok_id]
  | .prod a b, X => by
      show prodMapTok (a.map (idMap X.sys)) (b.map (idMap X.sys)) = idMap ((a.obj X).
    prod (b.obj X)).sys
      rw [a.map_id X, b.map_id X, prodMapTok_id]
  | .oplus a b, X => by
      show oplusMapTok (a.map (idMap X.sys)) (b.map (idMap X.sys))
          = idMap ((a.obj X).oplus (b.obj X)).sys
      rw [a.map_id X, b.map_id X, oplusMapTok_id]
  | .otimes a b, X => by
      show otimesMapTok (a.map (idMap X.sys)) (b.map (idMap X.sys))
          = idMap ((a.obj X).otimes (b.obj X)).sys
      rw [a.map_id X, b.map_id X, otimesMapTok_id]

theorem otimesMapTok_isStrict (f₀ : ApproximableMap A₀.sys B₀.sys)
    (f₁ : ApproximableMap A₁.sys B₁.sys) : IsStrict (otimesMapTok f₀ f₁) := by
  rintro Y (⟨-, rfl⟩ | ⟨X, Y0, X', Y', -, -, hXne, -, -, -, heq, -⟩)
  · rfl
  · have heq' : prodTokNbhd A₀.sys.master A₁.sys.master = prodTokNbhd X Y0 := heq
    obtain ⟨h, -⟩ := prodTokNbhd_injective heq'
    exact absurd h.symm hXne

theorem GExpr.map_isStrict : (T : GExpr) → {X Y : ScottSys} → (f : ApproximableMap X.
    sys Y.sys) →
    IsStrict f → IsStrict (T.map f)
  | .const _, _, _, _, _ => isStrict_idMap
  | .var, _, _, _, hf => hf
  | .sum a b, _, _, f, _ => sumMapTok_isStrict (a.map f) (b.map f)
  | .prod a b, _, _, f, hf => prodMapTok_isStrict (a.map_isStrict f hf) (b.
    map_isStrict f hf)
  | .oplus a b, _, _, f, _ => oplusMapTok_isStrict (a.map f) (b.map f)
  | .otimes a b, _, _, f, _ => otimesMapTok_isStrict (a.map f) (b.map f)

theorem GExpr.map_comp : (T : GExpr) → {X Y Z : ScottSys} → (f : ApproximableMap X.
    sys Y.sys) →
    {g : ApproximableMap Y.sys Z.sys} → IsStrict g → T.map (g.comp f) = (T.map g).
    comp (T.map f)
  | .const D, _, _, _, _, _, _ => (idMap_comp (idMap D.sys)).symm
  | .var, _, _, _, _, _, _ => rfl
  | .sum a b, _, _, _, f, g, hg => by
      show sumMapTok (a.map (g.comp f)) (b.map (g.comp f))
          = (sumMapTok (a.map g) (b.map g)).comp (sumMapTok (a.map f) (b.map f))
      rw [a.map_comp f hg, b.map_comp f hg]
      · exact sumMapTok_comp _ _ _ _
  | .prod a b, _, _, _, f, g, hg => by
      show prodMapTok (a.map (g.comp f)) (b.map (g.comp f))
```

```
          = (prodMapTok (a.map g) (b.map g)).comp (prodMapTok (a.map f) (b.map f))
      rw [a.map_comp f hg, b.map_comp f hg]
      · exact prodMapTok_comp _ _ _ _
  | .oplus a b, _, _, _, f, g, hg => by
      show oplusMapTok (a.map (g.comp f)) (b.map (g.comp f))
          = (oplusMapTok (a.map g) (b.map g)).comp (oplusMapTok (a.map f) (b.map f))
      rw [a.map_comp f hg, b.map_comp f hg]
      · exact oplusMapTok_comp _ _ (a.map_isStrict g hg) (b.map_isStrict g hg)
  | .otimes a b, _, _, _, f, g, hg => by
      show otimesMapTok (a.map (g.comp f)) (b.map (g.comp f))
          = (otimesMapTok (a.map g) (b.map g)).comp (otimesMapTok (a.map f) (b.map f))
      rw [a.map_comp f hg, b.map_comp f hg]
      · exact otimesMapTok_comp _ _ (a.map_isStrict g hg) (b.map_isStrict g hg)

theorem otimesMapTok_mono {f₀ f₀' : ApproximableMap A₀.sys B₀.sys}
    {f₁ f₁' : ApproximableMap A₁.sys B₁.sys} (h0 : f₀ ≤ f₀') (h1 : f₁ ≤ f₁') :
    otimesMapTok f₀ f₁ ≤ otimesMapTok f₀' f₁' := by
  rw [ApproximableMap.le_iff]
  rintro W W' (⟨hW, rfl⟩ | ⟨X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne, rfl, rfl⟩)
  · exact Or.inl ⟨hW, rfl⟩
  · exact Or.inr ⟨X, Y, X', Y', h0 X X' hr0, h1 Y Y' hr1, hXne, hYne, hX'ne, hY'ne, rfl
    , rfl⟩

theorem GExpr.map_mono : (T : GExpr) → {X Y : ScottSys} → {f f' : ApproximableMap X.
    sys Y.sys} →
    f ≤ f' → T.map f ≤ T.map f'


  | .const _, _, _, _, _, _ => le_rfl
  | .var, _, _, _, _, h => h
  | .sum a b, _, _, _, _, h => sumMapTok_mono (a.map_mono h) (b.map_mono h)
  | .prod a b, _, _, _, _, h => prodMapTok_mono (a.map_mono h) (b.map_mono h)
  | .oplus a b, _, _, _, _, h => oplusMapTok_mono (a.map_mono h) (b.map_mono h)
  | .otimes a b, _, _, _, _, h => otimesMapTok_mono (a.map_mono h) (b.map_mono h)

theorem GExpr.map_continuous_mp : (T : GExpr) → {I : Type} → {X Y : ScottSys} →
    {g : I → ApproximableMap X.sys Y.sys} → {f : ApproximableMap X.sys Y.sys} →
    [Nonempty I] → (∀ i j, ∃ k, g i ≤ g k ∧ g j ≤ g k) →
    (∀ A B, f.rel A B ↔ ∃ i, (g i).rel A B) →
    {A B : Set Str} → (T.map f).rel A B → ∃ i, (T.map (g i)).rel A B
  | .const _, _, _, _, _, _, _, _, _, _, _, hrel => by
      obtain ⟨i⟩ := ?Nonempty _?; exact ⟨i, hrel⟩
  | .var, _, _, _, _, _, _, _, hf, A, B, hrel => (hf A B).mp hrel
  | .sum a b, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      rcases hrel with ⟨hA, rfl⟩ | ⟨P, P', hr, rfl, rfl⟩ | ⟨Q, Q', hr, rfl, rfl⟩
      · obtain ⟨i⟩ := ?Nonempty _?; exact ⟨i, Or.inl ⟨hA, rfl⟩⟩
      · obtain ⟨i, hi⟩ := a.map_continuous_mp hdir hf hr
        exact ⟨i, Or.inr (Or.inl ⟨P, P', hi, rfl, rfl⟩)⟩
      · obtain ⟨i, hi⟩ := b.map_continuous_mp hdir hf hr
        exact ⟨i, Or.inr (Or.inr ⟨Q, Q', hi, rfl, rfl⟩)⟩
  | .prod a b, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      obtain ⟨P, Q, P', Q', hr0, hr1, rfl, rfl⟩ := hrel
      obtain ⟨i, hi⟩ := a.map_continuous_mp hdir hf hr0
      obtain ⟨j, hj⟩ := b.map_continuous_mp hdir hf hr1
      obtain ⟨k, hik, hjk⟩ := hdir i j
      exact ⟨k, P, Q, P', Q', (a.map_mono hik) P P' hi, (b.map_mono hjk) Q Q' hj, rfl,
    rfl⟩
```

```
  | .oplus a b, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      rcases hrel with ⟨hA, rfl⟩ | ⟨P, P', hr, hPne, hP'ne, rfl, rfl⟩ |
        ⟨Q, Q', hr, hQne, hQ'ne, rfl, rfl⟩
      · obtain ⟨i⟩ := ?Nonempty _?; exact ⟨i, Or.inl ⟨hA, rfl⟩⟩
      · obtain ⟨i, hi⟩ := a.map_continuous_mp hdir hf hr
        exact ⟨i, Or.inr (Or.inl ⟨P, P', hi, hPne, hP'ne, rfl, rfl⟩)⟩
      · obtain ⟨i, hi⟩ := b.map_continuous_mp hdir hf hr
        exact ⟨i, Or.inr (Or.inr ⟨Q, Q', hi, hQne, hQ'ne, rfl, rfl⟩)⟩
  | .otimes a b, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      rcases hrel with ⟨hA, rfl⟩ | ⟨P, Q, P', Q', hr0, hr1, hPne, hQne, hP'ne, hQ'ne,
    rfl, rfl⟩
      · obtain ⟨i⟩ := ?Nonempty _?; exact ⟨i, Or.inl ⟨hA, rfl⟩⟩
      · obtain ⟨i, hi⟩ := a.map_continuous_mp hdir hf hr0
        obtain ⟨j, hj⟩ := b.map_continuous_mp hdir hf hr1
        obtain ⟨k, hik, hjk⟩ := hdir i j
        exact ⟨k, Or.inr ⟨P, Q, P', Q', (a.map_mono hik) P P' hi, (b.map_mono hjk) Q Q'
    hj,
          hPne, hQne, hP'ne, hQ'ne, rfl, rfl⟩⟩

theorem GExpr.map_continuous (T : GExpr) {I : Type} [Nonempty I] {X Y : ScottSys}
    (g : I → ApproximableMap X.sys Y.sys) (f : ApproximableMap X.sys Y.sys)
    (hdir : ∀ i j, ∃ k, g i ≤ g k ∧ g j ≤ g k)
    (hf : ∀ A B, f.rel A B ↔ ∃ i, (g i).rel A B) (A B : Set Str) :
    (T.map f).rel A B ↔ ∃ i, (T.map (g i)).rel A B := by
  refine ⟨T.map_continuous_mp hdir hf, ?_⟩
  rintro ⟨i, hi⟩
  have hgif : g i ≤ f := by
    rw [ApproximableMap.le_iff]; intro A' B' h; exact (hf A' B').mpr ⟨i, h⟩
  exact (T.map_mono hgif) A B hi

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

theorem otimesTok_mem_prod_inv {X Y : Set Str} (h : (otimesTok D₀ D₁).mem (prodTokNbhd
    X Y))
    (hX : X = D₀.master) : D₀.mem X ∧ D₁.mem Y := by
  rcases h with heq | ⟨X', Y', hX', hY', -, -, heq⟩
  · obtain ⟨rfl, -⟩ := prodTokNbhd_injective heq; exact absurd rfl hX
  · obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective heq; exact ⟨hX', hY'⟩

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}
```

```
variable {A₀ A₁ B₀ B₁ : ScottSys}

theorem otimesTok_subsystem (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    (A₀.otimes A₁).sys ? (B₀.otimes B₁).sys := by
  have heqm : prodTokNbhd A₀.sys.master A₁.sys.master
      = prodTokNbhd B₀.sys.master B₁.sys.master := by rw [h0.master_eq, h1.master_eq]
  refine ⟨heqm, ?_, ?_⟩
  · rintro W (rfl | ⟨X, Y, hX, hY, hXne, hYne, rfl⟩)
    · exact Or.inl heqm
    · exact Or.inr ⟨X, Y, h0.sub hX, h1.sub hY, h0.master_eq ▸ hXne, h1.master_eq ▸
    hYne, rfl⟩
  · rintro W W' (rfl | ⟨X, Y, hX, hY, hXne, hYne, rfl⟩)
      (rfl | ⟨X', Y', hX', hY', hX'ne, hY'ne, rfl⟩) hInt
    · rw [Set.inter_self]; exact Or.inl rfl
    · rw [prodTokNbhd_inter, Set.inter_eq_right.mpr (A₀.sys.sub_master hX'),
        Set.inter_eq_right.mpr (A₁.sys.sub_master hY')]
      exact Or.inr ⟨X', Y', hX', hY', hX'ne, hY'ne, rfl⟩
    · rw [Set.inter_comm, prodTokNbhd_inter, Set.inter_eq_right.mpr (A₀.sys.sub_master
    hX),
        Set.inter_eq_right.mpr (A₁.sys.sub_master hY)]
      exact Or.inr ⟨X, Y, hX, hY, hXne, hYne, rfl⟩
    · rw [prodTokNbhd_inter] at hInt ⊢
      have hXne' : X ∩ X' = B₀.sys.master := by
        rw [← h0.master_eq]; exact inter_ne_of_ne_left (A₀.sys.sub_master hX) hXne
      obtain ⟨hmemX, hmemY⟩ := otimesTok_mem_prod_inv hInt hXne'
      exact Or.inr ⟨X ∩ X', Y ∩ Y', h0.inter_closed hX hX' hmemX, h1.inter_closed hY hY
    ' hmemY,
        inter_ne_of_ne_left (A₀.sys.sub_master hX) hXne,
        inter_ne_of_ne_left (A₁.sys.sub_master hY) hYne, rfl⟩

end FunctorTokens
end

section
namespace FunctorTokens

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

variable {A₀ A₁ B₀ B₁ : ScottSys}

variable {C₀ C₁ : ScottSys}

theorem GExpr.obj_subsystem : (T : GExpr) → {X Y : ScottSys} → X.sys ? Y.sys →
    (T.obj X).sys ? (T.obj Y).sys
  | .const D, _, _, _ => Subsystem.refl D.sys
  | .var, _, _, h => h
  | .sum a b, _, _, h => sumTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)
  | .prod a b, _, _, h => prodTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)
  | .oplus a b, _, _, h => oplusTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)
  | .otimes a b, _, _, h => otimesTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h
    )

theorem GExpr.obj_continuous_mp : (T : GExpr) → {? : Set ScottSys} → {U : ScottSys}
```

```
    →
    DirectedOn (fun a b => a.sys ? b.sys) ? → ?.Nonempty →
    (∀ D ∈ ?, D.sys ? U.sys) → (∀ X, U.sys.mem X ↔ ∃ D ∈ ?, D.sys.mem X) →
    {W : Set Str} → (T.obj U).sys.mem W → ∃ D ∈ ?, (T.obj D).sys.mem W
  | .const _, _, _, _, hne, _, _, _, hmem => by
      obtain ⟨D, hD⟩ := hne; exact ⟨D, hD, hmem⟩
  | .var, _, _, _, _, _, hU, W, hmem => (hU W).mp hmem
  | .sum a b, _, _, hdir, hne, hsub, hU, _, hmem => by
      rcases hmem with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
      · obtain ⟨D, hD⟩ := hne
        exact ⟨D, hD, Or.inl ((GExpr.sum a b).obj_subsystem (hsub D hD)).master_eq.symm
    ⟩
      · obtain ⟨D, hD, hXD⟩ := a.obj_continuous_mp hdir hne hsub hU hX
        exact ⟨D, hD, Or.inr (Or.inl ⟨X, hXD, rfl⟩)⟩
      · obtain ⟨D, hD, hYD⟩ := b.obj_continuous_mp hdir hne hsub hU hY
        exact ⟨D, hD, Or.inr (Or.inr ⟨Y, hYD, rfl⟩)⟩
  | .prod a b, _, _, hdir, hne, hsub, hU, _, hmem => by
      obtain ⟨X, Y, hX, hY, rfl⟩ := hmem
      obtain ⟨D₁, hD₁, hXD⟩ := a.obj_continuous_mp hdir hne hsub hU hX
      obtain ⟨D₂, hD₂, hYD⟩ := b.obj_continuous_mp hdir hne hsub hU hY
      obtain ⟨D₃, hD₃, hr1, hr2⟩ := hdir D₁ hD₁ D₂ hD₂
      exact ⟨D₃, hD₃, X, Y, (a.obj_subsystem hr1).sub hXD, (b.obj_subsystem hr2).sub
    hYD, rfl⟩
  | .oplus a b, _, _, hdir, hne, hsub, hU, _, hmem => by
      rcases hmem with rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩
      · obtain ⟨D, hD⟩ := hne
        exact ⟨D, hD, Or.inl ((GExpr.oplus a b).obj_subsystem (hsub D hD)).master_eq.
    symm⟩
      · obtain ⟨D, hD, hXD⟩ := a.obj_continuous_mp hdir hne hsub hU hX
        refine ⟨D, hD, Or.inr (Or.inl ⟨X, hXD, ?_, rfl⟩)⟩
        exact fun heq => hXne (heq.trans (a.obj_subsystem (hsub D hD)).master_eq)
      · obtain ⟨D, hD, hYD⟩ := b.obj_continuous_mp hdir hne hsub hU hY
        refine ⟨D, hD, Or.inr (Or.inr ⟨Y, hYD, ?_, rfl⟩)⟩
        exact fun heq => hYne (heq.trans (b.obj_subsystem (hsub D hD)).master_eq)
  | .otimes a b, _, _, hdir, hne, hsub, hU, _, hmem => by
      rcases hmem with rfl | ⟨X, Y, hX, hY, hXne, hYne, rfl⟩
      · obtain ⟨D, hD⟩ := hne
        exact ⟨D, hD, Or.inl ((GExpr.otimes a b).obj_subsystem (hsub D hD)).master_eq.
    symm⟩
      · obtain ⟨D₁, hD₁, hXD⟩ := a.obj_continuous_mp hdir hne hsub hU hX
        obtain ⟨D₂, hD₂, hYD⟩ := b.obj_continuous_mp hdir hne hsub hU hY
        obtain ⟨D₃, hD₃, hr1, hr2⟩ := hdir D₁ hD₁ D₂ hD₂
        refine ⟨D₃, hD₃, Or.inr ⟨X, Y, (a.obj_subsystem hr1).sub hXD,
          (b.obj_subsystem hr2).sub hYD, ?_, ?_, rfl⟩⟩
        · exact fun heq => hXne (heq.trans (a.obj_subsystem (hsub D₃ hD₃)).master_eq)
        · exact fun heq => hYne (heq.trans (b.obj_subsystem (hsub D₃ hD₃)).master_eq)

theorem GExpr.obj_continuous (T : GExpr) {? : Set ScottSys} {U : ScottSys}
    (hdir : DirectedOn (fun a b => a.sys ? b.sys) ?) (hne : ?.Nonempty)
    (hsub : ∀ D ∈ ?, D.sys ? U.sys) (hU : ∀ X, U.sys.mem X ↔ ∃ D ∈ ?, D.sys.mem X)
    (W : Set Str) : (T.obj U).sys.mem W ↔ ∃ D ∈ ?, (T.obj D).sys.mem W := by
  refine ⟨T.obj_continuous_mp hdir hne hsub hU, ?_⟩
  rintro ⟨D, hD, hmem⟩
  exact (T.obj_subsystem (hsub D hD)).sub hmem

end FunctorTokens
```

```
end

end DomainTheory.NeighbourhoodSystems
end
```

4.7.48 Exercise 6.21 — coalesced sum and smash

NeighbourhoodSystems/DomainEquations/Exercise_6_21.lean

`Exercise_6_21.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.coalesceSmash

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap SeqEqB BitDom DomainTheory.
    NeighbourhoodSystems.SmashProduct DomainTheory.NeighbourhoodSystems.FunctorTokens

/-- Scott's Exercise 6.21: rooted grammars admit a subsystem solution and a fixed-
    point set containing `[]`. -/
theorem example_621 :
    (∀ (T : GExpr), T.RootedConst →
      ∃ (Γ : Set Str) (h : Γ.Nonempty),
        (singletonSys Γ h).sys ? (T.obj (singletonSys Γ h)).sys) ∧
    (∀ (T : GExpr), T.RootedConst → ∃ (Γ : Set Str), ([] : Str) ∈ Γ ∧ gFun T Γ = Γ)
    :=
  ⟨@gExists_singleton_subsystem, @gExists_tok_fixedPoint⟩
```

EXERCISE 6.21. Do the same as 6.19 and 6.20 when the functors are also allowed to be generated by the operations:

$$D_0 \oplus D_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{0X \mid X \in D_0 \setminus \{\Delta_0\}\} \cup \{1Y \mid Y \in D_1 \setminus \{\Delta_1\}\},$$

$$D_0 \otimes D_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{\{\Lambda\} \cup 0X \cup 1Y \mid X \in D_0 \setminus \{\Delta_0\} \text{ and } Y \in D_1 \setminus \{\Delta_1\}\}.$$

Generalize all of $+$, $\times$, $\oplus$, $\otimes$ to combinations of several terms, not just the binary sums and products.

4.7.49 intExprDomains — integer equations

`intExprDomains.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.coalesceSmash
public import DomainTheory.NeighbourhoodSystems.DomainEquations.strictDomainObj
public import DomainTheory.NeighbourhoodSystems.DomainEquations.initialHom
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_6
```

```
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.
    FunctorTokens

open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom

namespace FunctorTokens

def Cnat : ScottSys where
  sys :=
    { mem := fun X => X = {([false] : Str)} ∨ X = {([false] : Str), ([] : Str)}
      master := {([false] : Str), ([] : Str)}
      master_nonempty := ⟨[false], Set.mem_insert _ _⟩
      master_mem := Or.inr rfl
      inter_mem := by
        have hAB : ({([false] : Str)} : Set Str) ⊆ {([false] : Str), ([] : Str)} :=
          Set.singleton_subset_iff.mpr (Set.mem_insert _ _)
        rintro X Y Z (rfl | rfl) (rfl | rfl) _ _
        · exact Or.inl (Set.inter_self _)
        · exact Or.inl (Set.inter_eq_self_of_subset_left hAB)
        · exact Or.inl (Set.inter_eq_self_of_subset_right hAB)
        · exact Or.inr (Set.inter_self _)
      sub_master := by
        have hAB : ({([false] : Str)} : Set Str) ⊆ {([false] : Str), ([] : Str)} :=
          Set.singleton_subset_iff.mpr (Set.mem_insert _ _)
        rintro X (rfl | rfl)
        · exact hAB
        · exact subset_rfl }
  ne := by
    rintro X (rfl | rfl)
    · exact ⟨[false], rfl⟩
    · exact ⟨[false], Set.mem_insert _ _⟩

end FunctorTokens

end DomainTheory.NeighbourhoodSystems

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.
    FunctorTokens
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom

namespace FunctorTokens

def Cone : ScottSys := singletonSys ({([] : Str)} : Set Str) ⟨[], rfl⟩

def MExpr : GExpr := .sum (.const Cone) .var

def NExpr : GExpr := .oplus (.const Cnat) .var
```

```
def NStarExpr (N : ScottSys) : GExpr := .oplus (.const N) (.otimes (.const N) .var)

theorem NStarExpr_rooted {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master) :
    (NStarExpr N).RootedConst := ⟨hN, hN, trivial⟩

theorem NStar_eq_solution (N : ScottSys) (hN : ([] : Str) ∈ N.sys.master) :
    ∃ (Γ : Set Str) (h : Γ.Nonempty),
      (singletonSys Γ h).sys ? ((NStarExpr N).obj (singletonSys Γ h)).sys :=
  gExists_singleton_subsystem (NStarExpr N) (NStarExpr_rooted hN)

theorem nil_mem_Cnat : ([] : Str) ∈ Cnat.sys.master := Set.mem_insert_iff.mpr (Or.inr
    rfl)

theorem NExpr_rooted : NExpr.RootedConst := ⟨nil_mem_Cnat, trivial⟩

theorem NStar_over_N_exists :
    ∃ N : ScottSys, ([] : Str) ∈ N.sys.master ∧
      ∃ (Γ : Set Str) (h : Γ.Nonempty),
        (singletonSys Γ h).sys ? ((NStarExpr N).obj (singletonSys Γ h)).sys := by
  obtain ⟨Γ₁, hnil₁, _⟩ := gExists_tok_fixedPoint NExpr NExpr_rooted
  exact ⟨singletonSys Γ₁ ⟨[], hnil₁⟩, hnil₁, NStar_eq_solution _ hnil₁⟩

theorem N_eq_solution :
    ∃ (Γ : Set Str) (h : Γ.Nonempty),
      (singletonSys Γ h).sys ? (NExpr.obj (singletonSys Γ h)).sys :=
  gExists_singleton_subsystem NExpr NExpr_rooted

theorem nil_mem_Cone : ([] : Str) ∈ Cone.sys.master := rfl

theorem MExpr_rooted : MExpr.RootedConst := ⟨nil_mem_Cone, trivial⟩

theorem M_eq_solution :
    ∃ (Γ : Set Str) (h : Γ.Nonempty),
      (singletonSys Γ h).sys ? (MExpr.obj (singletonSys Γ h)).sys :=
  gExists_singleton_subsystem MExpr MExpr_rooted

end FunctorTokens

section
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
    DomainTheory.NeighbourhoodSystems.SmashProduct

namespace FunctorTokens

def gFix (T : GExpr) : Set Str := ⋃ n, gIter T n

theorem gFix_fixed (T : GExpr) (hT : T.RootedConst) : gFun T (gFix T) = gFix T :=
  gFun_iter_fixed T hT

theorem gFix_nil_mem (T : GExpr) : ([] : Str) ∈ gFix T :=
  Set.mem_iUnion.mpr ⟨0, rfl⟩

theorem gFix_nonempty (T : GExpr) : (gFix T).Nonempty := ⟨[], gFix_nil_mem T⟩

def gGen (T : GExpr) : ScottSys := singletonSys (gFix T) (gFix_nonempty T)
```

```
@[simp] theorem gGen_master (T : GExpr) : (gGen T).sys.master = gFix T := rfl

theorem gBase (T : GExpr) (hT : T.RootedConst) : (gGen T).sys ? (T.obj (gGen T)).sys
    := by
  have hmaster : (T.obj (gGen T)).sys.master = gFix T :=
    (gFun_eq_master T (gFix_nonempty T)).symm.trans (gFix_fixed T hT)
  refine ⟨hmaster.symm, ?_, ?_⟩
  · intro X hX
    have heq : X = (T.obj (gGen T)).sys.master := (hX : X = gFix T).trans hmaster.symm
    rw [heq]; exact (T.obj (gGen T)).sys.master_mem
  · intro X Y hX hY _
    show X ∩ Y = gFix T
    rw [show X = gFix T from hX, show Y = gFix T from hY, Set.inter_self]

def gTower (T : GExpr) : ℕ → ScottSys
  | 0 => gGen T
  | n + 1 => T.obj (gTower T n)

@[simp] theorem gTower_zero (T : GExpr) : gTower T 0 = gGen T := rfl

@[simp] theorem gTower_succ (T : GExpr) (n : ℕ) : gTower T (n + 1) = T.obj (gTower T n
    ) := rfl

theorem gChain (T : GExpr) (hT : T.RootedConst) :
    ∀ n, (gTower T n).sys ? (gTower T (n + 1)).sys
  | 0 => gBase T hT
  | n + 1 => T.obj_subsystem (gChain T hT n)

theorem gTower_le (T : GExpr) (hT : T.RootedConst) {n m : ℕ} (h : n ≤ m) :
    (gTower T n).sys ? (gTower T m).sys := by
  induction h with
  | refl => exact Subsystem.refl _
  | step _ ih => exact ih.trans (gChain T hT _)

theorem gTower_master (T : GExpr) (hT : T.RootedConst) :
    ∀ n, (gTower T n).sys.master = gFix T
  | 0 => rfl
  | n + 1 => ((gChain T hT n).master_eq).symm.trans (gTower_master T hT n)

def gColim (T : GExpr) (hT : T.RootedConst) : ScottSys where
  sys :=
    { mem := fun X => ∃ n, (gTower T n).sys.mem X
      master := gFix T
      master_nonempty := (gTower T 0).sys.master_nonempty
      master_mem := ⟨0, (gTower T 0).sys.master_mem⟩
      inter_mem := by
        rintro X Y Z ⟨n, hX⟩ ⟨m, hY⟩ ⟨p, hZ⟩ hsub
        set N := max n (max m p) with hN
        have hXN : (gTower T N).sys.mem X := (gTower_le T hT (le_max_left n _)).sub hX
        have hYN : (gTower T N).sys.mem Y :=
          (gTower_le T hT ((le_max_left m p).trans (le_max_right n _))).sub hY
        have hZN : (gTower T N).sys.mem Z :=
          (gTower_le T hT ((le_max_right m p).trans (le_max_right n _))).sub hZ
        exact ⟨N, (gTower T N).sys.inter_mem hXN hYN hZN hsub⟩
      sub_master := by
        rintro X ⟨n, hX⟩
```

```
        rw [← gTower_master T hT n]
        exact (gTower T n).sys.sub_master hX }
  ne := by rintro X ⟨n, hX⟩; exact (gTower T n).ne X hX

@[simp] theorem mem_gColim (T : GExpr) (hT : T.RootedConst) {X : Set Str} :
    (gColim T hT).sys.mem X ↔ ∃ n, (gTower T n).sys.mem X := Iff.rfl

@[simp] theorem gColim_master (T : GExpr) (hT : T.RootedConst) :
    (gColim T hT).sys.master = gFix T := rfl

def Texp (N : ScottSys) : GExpr :=
  .oplus (.const N) (.sum (.prod .var .var) (.prod .var .var))

theorem Texp_rooted {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master) : (Texp N).
    RootedConst :=
  ⟨hN, ⟨trivial, trivial⟩, ⟨trivial, trivial⟩⟩

def Exp (N : ScottSys) (hN : ([] : Str) ∈ N.sys.master) : ScottSys :=
  gColim (Texp N) (Texp_rooted hN)

theorem ScottSys.ext {A B : ScottSys} (h : A.sys = B.sys) : A = B := by
  cases A; cases B; cases h; rfl

instance : Category ScottSys where
  Hom A B := StrictMap A.sys B.sys
  id A := ⟨idMap A.sys, isStrict_idMap⟩
  comp f g := ⟨g.1.comp f.1, isStrict_comp g.2 f.2⟩
  id_comp f := Subtype.ext (comp_idMap f.1)
  comp_id f := Subtype.ext (idMap_comp f.1)
  assoc f g h := Subtype.ext (comp_assoc h.1 g.1 f.1).symm

theorem gTower_sub_colim (T : GExpr) (hT : T.RootedConst) (n : ℕ) :
    (gTower T n).sys ? (gColim T hT).sys where
  master_eq := by rw [gColim_master, gTower_master T hT]
  sub hX := ⟨n, hX⟩
  inter_closed := by
    rintro X Y hX hY ⟨m, hXY⟩
    have hN : (gTower T (max n m)).sys.mem (X ∩ Y) :=
      (gTower_le T hT (le_max_right n m)).sub hXY
    exact (gTower_le T hT (le_max_left n m)).inter_closed hX hY hN

theorem gColim_obj_sys_eq (T : GExpr) (hT : T.RootedConst) :
    (T.obj (gColim T hT)).sys = (gColim T hT).sys := by
  set ? : Set ScottSys := Set.range (gTower T) with h?
  have hne : ?.Nonempty := ⟨gTower T 0, 0, rfl⟩
  have hsub : ∀ D ∈ ?, D.sys ? (gColim T hT).sys := by
    rintro D ⟨n, rfl⟩; exact gTower_sub_colim T hT n
  have hdir : DirectedOn (fun a b => a.sys ? b.sys) ? := by
    rintro _ ⟨n, rfl⟩ _ ⟨m, rfl⟩
    exact ⟨gTower T (max n m), ⟨max n m, rfl⟩,
      gTower_le T hT (le_max_left n m), gTower_le T hT (le_max_right n m)⟩
  have hU : ∀ X, (gColim T hT).sys.mem X ↔ ∃ D ∈ ?, D.sys.mem X := by
    intro X; constructor
    · rintro ⟨n, hn⟩; exact ⟨gTower T n, ⟨n, rfl⟩, hn⟩
    · rintro ⟨D, ⟨n, rfl⟩, hn⟩; exact ⟨n, hn⟩
  apply NeighborhoodSystem.ext
```

```
  · intro W
    rw [T.obj_continuous hdir hne hsub hU W]
    constructor
    · rintro ⟨D, ⟨n, rfl⟩, hn⟩

      exact ⟨n + 1, hn⟩
    · rintro ⟨n, hn⟩

      exact ⟨gTower T n, ⟨n, rfl⟩, (gChain T hT n).sub hn⟩
  ·
    have h := (T.obj_subsystem (gTower_sub_colim T hT 0)).master_eq
    rw [gColim_master]
    rw [show (T.obj (gTower T 0)) = gTower T 1 from rfl] at h
    rw [← h, gTower_master T hT]

theorem gColim_obj_eq (T : GExpr) (hT : T.RootedConst) : T.obj (gColim T hT) = gColim
    T hT :=
  ScottSys.ext (gColim_obj_sys_eq T hT)

theorem Exp_structure_eq (N : ScottSys) (hN : ([] : Str) ∈ N.sys.master) :
    (Texp N).obj (Exp N hN) = Exp N hN :=
  gColim_obj_eq (Texp N) (Texp_rooted hN)

@[simp] theorem ScottSys.id_val (A : ScottSys) :
    ((? A) : StrictMap A.sys A.sys).1 = idMap A.sys := rfl

@[simp] theorem ScottSys.comp_val {A B C : ScottSys} (g : (B ⟶ C)) (f : (A ⟶ B)) :
    ((g ? f : StrictMap A.sys C.sys)).1 = g.1.comp f.1 := rfl

def gFunctorMap (T : GExpr) {X Y : ScottSys} (f : StrictMap X.sys Y.sys) :
    StrictMap (T.obj X).sys (T.obj Y).sys :=
  ⟨T.map f.1, T.map_isStrict f.1 f.2⟩

abbrev gFunctor (T : GExpr) : Endofunctor ScottSys where
  obj := T.obj
  map := gFunctorMap T
  map_id X := Subtype.ext (T.map_id X)
  map_comp {_ _ _} g f :=
    Subtype.ext (T.map_comp (f : StrictMap _ _).1 (g : StrictMap _ _).2)

@[simp] theorem gFunctor_obj (T : GExpr) (X : ScottSys) : (gFunctor T).obj X = T.obj X
    := rfl

@[simp] theorem gFunctorMap_val (T : GExpr) {X Y : ScottSys} (f : StrictMap X.sys Y.
    sys) :
    (gFunctorMap T f).1 = T.map f.1 := rfl

abbrev TexpF (N : ScottSys) : Endofunctor ScottSys := gFunctor (Texp N)

def algStr (B : TAlgebra (TexpF N)) :
    ApproximableMap ((TexpF N).obj B.carrier).sys B.carrier.sys :=
  (B.str : StrictMap ((TexpF N).obj B.carrier).sys B.carrier.sys).1

theorem algStr_strict (B : TAlgebra (TexpF N)) : IsStrict (algStr B) :=
  (B.str : StrictMap ((TexpF N).obj B.carrier).sys B.carrier.sys).2
```

```
def isoOfObjEq {Obj : Type*} [Category Obj] {X Y : Obj} (h : X = Y) : Iso X Y := by
  cases h
  exact ⟨(? X), (? X), CategoryTheory.Category.id_comp _, CategoryTheory.Category.
    id_comp _⟩

def ExpIso (N : ScottSys) (hN : ([] : Str) ∈ N.sys.master) :
    Iso ((TexpF N).obj (Exp N hN)) (Exp N hN) :=
  isoOfObjEq (Exp_structure_eq N hN)

abbrev ExpAlg (N : ScottSys) (hN : ([] : Str) ∈ N.sys.master) : TAlgebra (TexpF N)
    where
  carrier := Exp N hN
  str := (ExpIso N hN).hom

def expHom (N : ScottSys) (hN : ([] : Str) ∈ N.sys.master) :
    ApproximableMap ((TexpF N).obj (Exp N hN)).sys (Exp N hN).sys :=
  ((ExpIso N hN).hom : StrictMap ((TexpF N).obj (Exp N hN)).sys (Exp N hN).sys).1

def expInv (N : ScottSys) (hN : ([] : Str) ∈ N.sys.master) :
    ApproximableMap (Exp N hN).sys ((TexpF N).obj (Exp N hN)).sys :=
  ((ExpIso N hN).inv : StrictMap (Exp N hN).sys ((TexpF N).obj (Exp N hN)).sys).1

section Existence

variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master) (B : TAlgebra (TexpF N))

def descRel : ℕ → ApproximableMap (Exp N hN).sys B.carrier.sys
  | 0 => constMap (Exp N hN).sys B.carrier.sys.bot
  | n + 1 => (algStr B).comp (((Texp N).map (descRel n)).comp (expInv N hN))

@[simp] theorem descRel_succ (n : ℕ) :
    descRel hN B (n + 1) = (algStr B).comp (((Texp N).map (descRel hN B n)).comp (
    expInv N hN)) :=
  rfl

end Existence

end FunctorTokens
end

section
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
    DomainTheory.NeighbourhoodSystems.SmashProduct

namespace FunctorTokens

variable {A₀ A₁ B₀ B₁ : ScottSys}

section Uniqueness

variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master)

variable (B : TAlgebra (TexpF N))

theorem algHom_ext {A C : TAlgebra (TexpF N)} {g g' : AlgHom A C} (h : g.hom = g'.hom)
    : g = g' := by
```

```
  cases g; cases g'; cases h; rfl

end Uniqueness

end FunctorTokens
end

section
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
     DomainTheory.NeighbourhoodSystems.SmashProduct

namespace FunctorTokens

section Existence

variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master) (B : TAlgebra (TexpF N))

theorem constBot_le {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem
    β}
    (g : ApproximableMap V₀ V₁) : constMap V₀ V₁.bot ≤ g := by
  intro X Y hr
  obtain ⟨hX, hY⟩ := hr
  rw [NeighborhoodSystem.mem_bot] at hY
  subst hY
  exact g.mono g.master_rel (V₀.sub_master hX) subset_rfl hX V₁.master_mem

theorem descRel_le_succ : ∀ n, descRel hN B n ≤ descRel hN B (n + 1)
  | 0 => constBot_le _
  | n + 1 => by
      rw [descRel_succ, descRel_succ]
      exact comp_mono_gen le_rfl
        (comp_mono_gen ((Texp N).map_mono (descRel_le_succ n)) le_rfl)

theorem descRel_mono {i j : ℕ} (h : i ≤ j) : descRel hN B i ≤ descRel hN B j := by
  induction h with
  | refl => exact le_rfl
  | step _ ih => exact ih.trans (descRel_le_succ hN B _)

theorem descDir (i j : ℕ) : ∃ k, (∀ X Y, (descRel hN B i).rel X Y → (descRel hN B k).
    rel X Y) ∧
    (∀ X Y, (descRel hN B j).rel X Y → (descRel hN B k).rel X Y) :=
  ⟨max i j, descRel_mono hN B (le_max_left i j), descRel_mono hN B (le_max_right i j)⟩

theorem descDirLe (i j : ℕ) :
    ∃ k, descRel hN B i ≤ descRel hN B k ∧ descRel hN B j ≤ descRel hN B k :=
  ⟨max i j, descRel_mono hN B (le_max_left i j), descRel_mono hN B (le_max_right i j)⟩

def descMap : ApproximableMap (Exp N hN).sys B.carrier.sys :=
  iSupMap (descRel hN B) (descDir hN B)

theorem descMap_is_sup (A E : Set Str) :
    (descMap hN B).rel A E ↔ ∃ n, (descRel hN B n).rel A E := Iff.rfl

theorem descMap_fix :
    descMap hN B = (algStr B).comp (((Texp N).map (descMap hN B)).comp (expInv N hN))
    := by
```

```
  have hmc : ∀ Y C, ((Texp N).map (descMap hN B)).rel Y C
      ↔ ∃ n, ((Texp N).map (descRel hN B n)).rel Y C :=
    fun Y C => (Texp N).map_continuous (descRel hN B) (descMap hN B) (descDirLe hN B)
      (descMap_is_sup hN B) Y C
  apply ApproximableMap.ext
  intro A E
  rw [comp_rel]
  constructor
  · rintro ⟨n, hn⟩
    have hn1 : (descRel hN B (n + 1)).rel A E := descRel_le_succ hN B n A E hn
    rw [descRel_succ, comp_rel] at hn1
    obtain ⟨C, hAC, hCE⟩ := hn1
    rw [comp_rel] at hAC
    obtain ⟨Y, hAY, hYC⟩ := hAC
    exact ⟨C, ⟨Y, hAY, (hmc Y C).mpr ⟨n, hYC⟩⟩, hCE⟩
  · rintro ⟨C, hAC, hCE⟩
    rw [comp_rel] at hAC
    obtain ⟨Y, hAY, hYC⟩ := hAC
    obtain ⟨n, hn⟩ := (hmc Y C).mp hYC
    refine ⟨n + 1, ?_⟩
    rw [descRel_succ, comp_rel]

    exact ⟨C, by rw [comp_rel]; exact ⟨Y, hAY, hn⟩, hCE⟩

theorem descMap_rel {A E : Set Str} :
    (descMap hN B).rel A E ↔ ∃ n, (descRel hN B n).rel A E := Iff.rfl

end Existence

theorem expInv_comp_expHom (N : ScottSys) (hN : ([] : Str) ∈ N.sys.master) :
    (expInv N hN).comp (expHom N hN) = idMap ((TexpF N).obj (Exp N hN)).sys := by
  have h := congrArg (Subtype.val) (ExpIso N hN).hom_inv_id
  exact h

section Existence

variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master) (B : TAlgebra (TexpF N))

theorem descComm :
    (descMap hN B).comp (expHom N hN) = (algStr B).comp ((Texp N).map (descMap hN B))
    := by
  calc (descMap hN B).comp (expHom N hN)
      = ((algStr B).comp (((Texp N).map (descMap hN B)).comp (expInv N hN))).comp (
    expHom N hN) := by
        rw [← descMap_fix]
    _ = (algStr B).comp ((((Texp N).map (descMap hN B)).comp (expInv N hN)).comp (
    expHom N hN)) := by
        rw [comp_assoc]
    _ = (algStr B).comp (((Texp N).map (descMap hN B)).comp ((expInv N hN).comp (
    expHom N hN))) := by
        rw [comp_assoc]
    _ = (algStr B).comp (((Texp N).map (descMap hN B)).comp (idMap _)) := by
        rw [expInv_comp_expHom]
    _ = (algStr B).comp ((Texp N).map (descMap hN B)) := by rw [comp_idMap]

end Existence
```

```
theorem expInv_strict (N : ScottSys) (hN : ([] : Str) ∈ N.sys.master) : IsStrict (
    expInv N hN) :=
  ((ExpIso N hN).inv : StrictMap (Exp N hN).sys ((TexpF N).obj (Exp N hN)).sys).2

section Existence

variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master) (B : TAlgebra (TexpF N))

theorem descRel_isStrict : ∀ n, IsStrict (descRel hN B n)
  | 0 => isStrict_constBot
  | n + 1 => by
      rw [descRel_succ]
      exact isStrict_comp (algStr_strict B)
        (isStrict_comp ((Texp N).map_isStrict _ (descRel_isStrict n)) (expInv_strict N
     hN))

theorem descMap_isStrict : IsStrict (descMap hN B) := by
  rintro Y ⟨n, hn⟩
  exact descRel_isStrict hN B n hn

def descAlgHom : AlgHom (ExpAlg N hN) B where
  hom := ⟨descMap hN B, descMap_isStrict hN B⟩
  comm := by
    apply Subtype.ext
    show (descMap hN B).comp (expHom N hN) = (algStr B).comp ((Texp N).map (descMap hN
     B))
    exact descComm hN B

end Existence

end FunctorTokens
end

section
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
     DomainTheory.NeighbourhoodSystems.SmashProduct

namespace FunctorTokens

variable {A₀ A₁ B₀ B₁ : ScottSys}

section Uniqueness

variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master)

theorem expSub (n : ℕ) : (gTower (Texp N) n).sys ? (Exp N hN).sys :=
  gTower_sub_colim (Texp N) (Texp_rooted hN) n

end Uniqueness

end FunctorTokens
end

open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
     DomainTheory.NeighbourhoodSystems.SmashProduct
```

```
namespace FunctorTokens

theorem Subsystem.inj_isStrict {α : Type*} {D E : NeighborhoodSystem α} (h : D ? E) :
    IsStrict h.inj := by
  intro Y hrel
  rw [Subsystem.inj_rel] at hrel
  obtain ⟨_, hYE, hsub⟩ := hrel
  exact Set.Subset.antisymm (E.sub_master hYE) (by rw [← h.master_eq]; exact hsub)

end FunctorTokens

section
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
     DomainTheory.NeighbourhoodSystems.SmashProduct

namespace FunctorTokens

variable {A₀ A₁ B₀ B₁ : ScottSys}

theorem isoOfObjEq_hom_rel {X Y : ScottSys} (e : X = Y) {A E : Set Str} :
    ((isoOfObjEq e).hom).1.rel A E ↔ X.sys.mem A ∧ Y.sys.mem E ∧ A ⊆ E := by
  cases e; exact idMap_rel

theorem expHom_rel {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master) {A E : Set Str} :
    (expHom N hN).rel A E ↔
      ((Texp N).obj (Exp N hN)).sys.mem A ∧ (Exp N hN).sys.mem E ∧ A ⊆ E :=
  isoOfObjEq_hom_rel (Exp_structure_eq N hN)

theorem isoOfObjEq_inv_rel {X Y : ScottSys} (e : X = Y) {A E : Set Str} :
    ((isoOfObjEq e).inv).1.rel A E ↔ Y.sys.mem A ∧ X.sys.mem E ∧ A ⊆ E := by
  cases e; exact idMap_rel

theorem expInv_rel {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master) {A E : Set Str} :
    (expInv N hN).rel A E ↔
      (Exp N hN).sys.mem A ∧ ((Texp N).obj (Exp N hN)).sys.mem E ∧ A ⊆ E :=
  isoOfObjEq_inv_rel (Exp_structure_eq N hN)

theorem oplusMapTok_inj (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    oplusMapTok h0.inj h1.inj = (oplusTok_subsystem h0 h1).inj := by
  have hsubM : ∀ {W : Set Str}, (A₀.oplus A₁).sys.mem W → W ⊆ sumTokMaster B₀.sys B₁.
    sys := by
    rintro W (rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩)
    · exact (show sumTokMaster A₀.sys A₁.sys = sumTokMaster B₀.sys B₁.sys by
        unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]).subset
    · exact embF_subset_sumTokMaster (h0.sub hX)
    · exact embT_subset_sumTokMaster (h1.sub hY)
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.inj_rel]
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, X', ⟨hX, hX', hXs⟩, hXne, hX'ne, rfl, rfl⟩ |
      ⟨Y, Y', ⟨hY, hY', hYs⟩, hYne, hY'ne, rfl, rfl⟩)
    · exact ⟨hW, (B₀.oplus B₁).sys.master_mem, hsubM hW⟩
    · exact ⟨Or.inr (Or.inl ⟨X, hX, hXne, rfl⟩), Or.inr (Or.inl ⟨X', hX', hX'ne, rfl⟩),
        embBit_subset.mpr hXs⟩
```

```
    · exact ⟨Or.inr (Or.inr ⟨Y, hY, hYne, rfl⟩), Or.inr (Or.inr ⟨Y', hY', hY'ne, rfl⟩),
        embBit_subset.mpr hYs⟩
  · rintro ⟨hW, hW', hsub⟩
    rcases hW' with rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩
    · exact Or.inl ⟨hW, rfl⟩
    · rcases hW with rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩
      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact Or.inr (Or.inl ⟨X, X', ⟨hX, hX', embBit_subset.mp hsub⟩, hXne, hX'ne, rfl,
     rfl⟩)
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A₁.ne Y hY) h)
    · rcases hW with rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩
      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A₀.ne X hX) h)
      · exact Or.inr (Or.inr ⟨Y, Y', ⟨hY, hY', embBit_subset.mp hsub⟩, hYne, hY'ne, rfl,
     rfl⟩)

theorem oplusMapTok_proj (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    oplusMapTok h0.proj h1.proj = (oplusTok_subsystem h0 h1).proj := by
  have hsubM : ∀ {W : Set Str}, (B₀.oplus B₁).sys.mem W → W ⊆ sumTokMaster A₀.sys A₁.
    sys := by
    rintro W (rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩)
    · exact (show sumTokMaster B₀.sys B₁.sys = sumTokMaster A₀.sys A₁.sys by
        unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]).subset
    · exact (embBit_subset.mpr (by rw [h0.master_eq]; exact B₀.sys.sub_master hX)).
    trans
        (embF_subset_sumTokMaster A₀.sys.master_mem)
    · exact (embBit_subset.mpr (by rw [h1.master_eq]; exact B₁.sys.sub_master hY)).
    trans
        (embT_subset_sumTokMaster A₁.sys.master_mem)
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.proj_rel]
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, X', ⟨hX, hX', hXs⟩, hXne, hX'ne, rfl, rfl⟩ |
      ⟨Y, Y', ⟨hY, hY', hYs⟩, hYne, hY'ne, rfl, rfl⟩)
    · exact ⟨hW, (A₀.oplus A₁).sys.master_mem, hsubM hW⟩
    · exact ⟨Or.inr (Or.inl ⟨X, hX, hXne, rfl⟩), Or.inr (Or.inl ⟨X', hX', hX'ne, rfl⟩),
        embBit_subset.mpr hXs⟩
    · exact ⟨Or.inr (Or.inr ⟨Y, hY, hYne, rfl⟩), Or.inr (Or.inr ⟨Y', hY', hY'ne, rfl⟩),
        embBit_subset.mpr hYs⟩
  · rintro ⟨hW, hW', hsub⟩
    rcases hW' with rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩
    · exact Or.inl ⟨hW, rfl⟩
    · rcases hW with rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩
      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact Or.inr (Or.inl ⟨X, X', ⟨hX, hX', embBit_subset.mp hsub⟩, hXne, hX'ne, rfl,
     rfl⟩)
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (B₁.ne Y hY) h)
    · rcases hW with rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩
      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (B₀.ne X hX) h)
      · exact Or.inr (Or.inr ⟨Y, Y', ⟨hY, hY', embBit_subset.mp hsub⟩, hYne, hY'ne, rfl,
     rfl⟩)

theorem otimesMapTok_inj (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    otimesMapTok h0.inj h1.inj = (otimesTok_subsystem h0 h1).inj := by
```

```
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.inj_rel]
  constructor
  · rintro (⟨hW, rfl⟩ |
      ⟨X, Y, X', Y', ⟨hX, hX', hXs⟩, ⟨hY, hY', hYs⟩, hXne, hYne, hX'ne, hY'ne, rfl, rfl⟩
    )
    · refine ⟨hW, (B₀.otimes B₁).sys.master_mem, ?_⟩
      rcases hW with rfl | ⟨P, Q, hP, hQ, hPne, hQne, rfl⟩
      · exact (show prodTokNbhd A₀.sys.master A₁.sys.master
            = prodTokNbhd B₀.sys.master B₁.sys.master by rw [h0.master_eq, h1.master_eq
    ]).subset
      · exact prodTokNbhd_subset_iff.mpr ⟨B₀.sys.sub_master (h0.sub hP),
          B₁.sys.sub_master (h1.sub hQ)⟩
    · exact ⟨Or.inr ⟨X, Y, hX, hY, hXne, hYne, rfl⟩, Or.inr ⟨X', Y', hX', hY', hX'ne, hY
    'ne, rfl⟩,
        prodTokNbhd_subset_iff.mpr ⟨hXs, hYs⟩⟩
  · rintro ⟨hW, hW', hsub⟩
    rcases hW' with rfl | ⟨X', Y', hX', hY', hX'ne, hY'ne, rfl⟩
    · exact Or.inl ⟨hW, rfl⟩
    · rcases hW with rfl | ⟨X, Y, hX, hY, hXne, hYne, rfl⟩
      · obtain ⟨hsX, _⟩ := prodTokNbhd_subset_iff.mp hsub
        exact absurd (Set.Subset.antisymm (B₀.sys.sub_master hX')
          (by rw [← h0.master_eq]; exact hsX)) hX'ne
      · obtain ⟨hXs, hYs⟩ := prodTokNbhd_subset_iff.mp hsub
        exact Or.inr ⟨X, Y, X', Y', ⟨hX, hX', hXs⟩, ⟨hY, hY', hYs⟩,
          hXne, hYne, hX'ne, hY'ne, rfl, rfl⟩

theorem otimesMapTok_proj (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    otimesMapTok h0.proj h1.proj = (otimesTok_subsystem h0 h1).proj := by
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.proj_rel]
  constructor
  · rintro (⟨hW, rfl⟩ |
      ⟨X, Y, X', Y', ⟨hX, hX', hXs⟩, ⟨hY, hY', hYs⟩, hXne, hYne, hX'ne, hY'ne, rfl, rfl⟩
    )
    · refine ⟨hW, (A₀.otimes A₁).sys.master_mem, ?_⟩
      rcases hW with rfl | ⟨P, Q, hP, hQ, hPne, hQne, rfl⟩
      · exact (show prodTokNbhd B₀.sys.master B₁.sys.master
            = prodTokNbhd A₀.sys.master A₁.sys.master by rw [h0.master_eq, h1.master_eq
    ]).subset
      · exact prodTokNbhd_subset_iff.mpr ⟨by rw [h0.master_eq]; exact B₀.sys.sub_master
    hP,
          by rw [h1.master_eq]; exact B₁.sys.sub_master hQ⟩
    · exact ⟨Or.inr ⟨X, Y, hX, hY, hXne, hYne, rfl⟩, Or.inr ⟨X', Y', hX', hY', hX'ne, hY
    'ne, rfl⟩,
        prodTokNbhd_subset_iff.mpr ⟨hXs, hYs⟩⟩
  · rintro ⟨hW, hW', hsub⟩
    rcases hW' with rfl | ⟨X', Y', hX', hY', hX'ne, hY'ne, rfl⟩
    · exact Or.inl ⟨hW, rfl⟩
    · rcases hW with rfl | ⟨X, Y, hX, hY, hXne, hYne, rfl⟩
      · obtain ⟨hsX, _⟩ := prodTokNbhd_subset_iff.mp hsub
        exact absurd (Set.Subset.antisymm (A₀.sys.sub_master hX')
          (by rw [h0.master_eq]; exact hsX)) hX'ne
      · obtain ⟨hXs, hYs⟩ := prodTokNbhd_subset_iff.mp hsub
```

```
        exact Or.inr ⟨X, Y, X', Y', ⟨hX, hX', hXs⟩, ⟨hY, hY', hYs⟩,
          hXne, hYne, hX'ne, hY'ne, rfl, rfl⟩

theorem prodMapTok_inj (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    prodMapTok h0.inj h1.inj = (prodTok_subsystem h0 h1).inj := by
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.inj_rel]
  constructor
  · rintro ⟨X, Y, X', Y', ⟨hX, hX', hXs⟩, ⟨hY, hY', hYs⟩, rfl, rfl⟩
    exact ⟨prodTok_mem_prodTokNbhd hX hY, prodTok_mem_prodTokNbhd hX' hY',
      prodTokNbhd_subset_iff.mpr ⟨hXs, hYs⟩⟩
  · rintro ⟨⟨X, Y, hX, hY, rfl⟩, ⟨X', Y', hX', hY', rfl⟩, hsub⟩
    obtain ⟨hXs, hYs⟩ := prodTokNbhd_subset_iff.mp hsub
    exact ⟨X, Y, X', Y', ⟨hX, hX', hXs⟩, ⟨hY, hY', hYs⟩, rfl, rfl⟩

theorem prodMapTok_proj (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    prodMapTok h0.proj h1.proj = (prodTok_subsystem h0 h1).proj := by
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.proj_rel]
  constructor
  · rintro ⟨X, Y, X', Y', ⟨hX, hX', hXs⟩, ⟨hY, hY', hYs⟩, rfl, rfl⟩
    exact ⟨prodTok_mem_prodTokNbhd hX hY, prodTok_mem_prodTokNbhd hX' hY',
      prodTokNbhd_subset_iff.mpr ⟨hXs, hYs⟩⟩
  · rintro ⟨⟨X, Y, hX, hY, rfl⟩, ⟨X', Y', hX', hY', rfl⟩, hsub⟩
    obtain ⟨hXs, hYs⟩ := prodTokNbhd_subset_iff.mp hsub
    exact ⟨X, Y, X', Y', ⟨hX, hX', hXs⟩, ⟨hY, hY', hYs⟩, rfl, rfl⟩

section Uniqueness

variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master)

def rho (n : ℕ) : ApproximableMap (Exp N hN).sys (Exp N hN).sys :=
  (expSub hN n).inj.comp (expSub hN n).proj

theorem rho_rel (n : ℕ) {A E : Set Str} :
    (rho hN n).rel A E ↔ (Exp N hN).sys.mem A ∧ (Exp N hN).sys.mem E ∧
      ∃ z, (gTower (Texp N) n).sys.mem z ∧ A ⊆ z ∧ z ⊆ E := by
  unfold rho
  rw [comp_rel]
  constructor
  · rintro ⟨z, hproj, hinj⟩
    rw [Subsystem.proj_rel] at hproj
    rw [Subsystem.inj_rel] at hinj
    obtain ⟨hcA, hTz, hAz⟩ := hproj
    obtain ⟨_, hcE, hzE⟩ := hinj
    exact ⟨hcA, hcE, z, hTz, hAz, hzE⟩
  · rintro ⟨hcA, hcE, z, hTz, hAz, hzE⟩
    exact ⟨z, by rw [Subsystem.proj_rel]; exact ⟨hcA, hTz, hAz⟩,
      by rw [Subsystem.inj_rel]; exact ⟨hTz, hcE, hzE⟩⟩

theorem rho_mono {n m : ℕ} (h : n ≤ m) {A E : Set Str} (hr : (rho hN n).rel A E) :
    (rho hN m).rel A E := by
  rw [rho_rel] at hr ⊢
  obtain ⟨hcA, hcE, z, hTz, hAz, hzE⟩ := hr
```

```
  exact ⟨hcA, hcE, z, (gTower_le (Texp N) (Texp_rooted hN) h).sub hTz, hAz, hzE⟩

def iSupRho : ApproximableMap (Exp N hN).sys (Exp N hN).sys :=
  iSupMap (rho hN) (fun i j => ⟨max i j,
    fun _ _ h => rho_mono hN (le_max_left i j) h,
    fun _ _ h => rho_mono hN (le_max_right i j) h⟩)

theorem iSupRho_eq_id : iSupRho hN = idMap (Exp N hN).sys := by
  apply ApproximableMap.ext
  intro A E
  rw [idMap_rel]
  constructor
  · rintro ⟨n, hr⟩
    rw [rho_rel] at hr
    obtain ⟨hcA, hcE, z, _, hAz, hzE⟩ := hr
    exact ⟨hcA, hcE, hAz.trans hzE⟩
  · rintro ⟨hcA, hcE, hAE⟩
    obtain ⟨n, hA⟩ := hcA
    exact ⟨n, (rho_rel hN n).mpr ⟨⟨n, hA⟩, hcE, A, hA, subset_rfl, hAE⟩⟩

theorem rho_zero_rel {A E : Set Str} :
    (rho hN 0).rel A E ↔ (Exp N hN).sys.mem A ∧ E = (Exp N hN).sys.master := by
  rw [rho_rel]
  constructor
  · rintro ⟨hcA, hcE, z, hz, _, hzE⟩
    have hzm : z = (Exp N hN).sys.master :=
      (hz : z = gFix (Texp N)).trans (gColim_master (Texp N) (Texp_rooted hN)).symm
    subst hzm
    exact ⟨hcA, Set.Subset.antisymm ((Exp N hN).sys.sub_master hcE) hzE⟩
  · rintro ⟨hcA, rfl⟩
    exact ⟨hcA, (Exp N hN).sys.master_mem, (Exp N hN).sys.master,
      gColim_master (Texp N) (Texp_rooted hN), (Exp N hN).sys.sub_master hcA,
    subset_rfl⟩

end Uniqueness

section Uniqueness

variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master)

variable (B : TAlgebra (TexpF N))

theorem gcomp_rho_zero (g : AlgHom (ExpAlg N hN) B) :
    g.hom.1.comp (rho hN 0) = descRel hN B 0 := by
  apply ApproximableMap.ext
  intro A Z
  rw [comp_rel]
  constructor
  · rintro ⟨E, hrho, hg⟩
    rw [rho_zero_rel] at hrho
    obtain ⟨hcA, rfl⟩ := hrho
    have hZ : Z = B.carrier.sys.master := g.hom.2 hg
    exact ⟨hcA, by rw [NeighborhoodSystem.mem_bot]; exact hZ⟩
  · rintro ⟨hcA, hZ⟩
    rw [NeighborhoodSystem.mem_bot] at hZ
    subst hZ
```

```
    exact ⟨(Exp N hN).sys.master, (rho_zero_rel hN).mpr ⟨hcA, rfl⟩, g.hom.1.master_rel⟩

end Uniqueness

end FunctorTokens
end

open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
     DomainTheory.NeighbourhoodSystems.SmashProduct

namespace FunctorTokens

theorem Subsystem.self_inj {α : Type*} {D : NeighborhoodSystem α} (h : D ? D) :
    h.inj = idMap D := by
  apply ApproximableMap.ext
  intro X Y
  rw [Subsystem.inj_rel, idMap_rel]

theorem Subsystem.self_proj {α : Type*} {D : NeighborhoodSystem α} (h : D ? D) :
    h.proj = idMap D := by
  apply ApproximableMap.ext
  intro X Y
  rw [Subsystem.proj_rel, idMap_rel]

end FunctorTokens

section
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
     DomainTheory.NeighbourhoodSystems.SmashProduct

namespace FunctorTokens

variable {A₀ A₁ B₀ B₁ : ScottSys}

theorem sumMapTok_inj (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    sumMapTok h0.inj h1.inj = (sumTok_subsystem h0 h1).inj := by
  have hsubM : ∀ {W : Set Str}, (A₀.sum A₁).sys.mem W → W ⊆ sumTokMaster B₀.sys B₁.sys
     := by
    rintro W (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩)
    · exact (show sumTokMaster A₀.sys A₁.sys = sumTokMaster B₀.sys B₁.sys by
        unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]).subset
    · exact embF_subset_sumTokMaster (h0.sub hX)
    · exact embT_subset_sumTokMaster (h1.sub hY)
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.inj_rel]
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, X', ⟨hX, hX', hXsub⟩, rfl, rfl⟩ | ⟨Y, Y', ⟨hY, hY', hYsub⟩,
    rfl, rfl⟩)
    · exact ⟨hW, (B₀.sum B₁).sys.master_mem, hsubM hW⟩
    · exact ⟨Or.inr (Or.inl ⟨X, hX, rfl⟩), Or.inr (Or.inl ⟨X', hX', rfl⟩), embBit_subset.
    mpr hXsub⟩
    · exact ⟨Or.inr (Or.inr ⟨Y, hY, rfl⟩), Or.inr (Or.inr ⟨Y', hY', rfl⟩), embBit_subset.
    mpr hYsub⟩
  · rintro ⟨hW, hW', hsub⟩
    rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
```

```
    · exact Or.inl ⟨hW, rfl⟩
    · rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact Or.inr (Or.inl ⟨X, X', ⟨hX, hX', embBit_subset.mp hsub⟩, rfl, rfl⟩)
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A₁.ne Y hY) h)
    · rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩

      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A₀.ne X hX) h)
      · exact Or.inr (Or.inr ⟨Y, Y', ⟨hY, hY', embBit_subset.mp hsub⟩, rfl, rfl⟩)

theorem GExpr.map_inj : (T : GExpr) → {X Y : ScottSys} → (h : X.sys ? Y.sys) →
    T.map h.inj = (T.obj_subsystem h).inj
  | .const D, _, _, _ => (Subsystem.self_inj (Subsystem.refl D.sys)).symm
  | .var, _, _, _ => rfl
  | .sum a b, _, _, h => by
      show sumMapTok (a.map h.inj) (b.map h.inj)
          = (sumTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).inj
      rw [a.map_inj h, b.map_inj h, sumMapTok_inj]
  | .prod a b, _, _, h => by
      show prodMapTok (a.map h.inj) (b.map h.inj)
          = (prodTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).inj
      rw [a.map_inj h, b.map_inj h, prodMapTok_inj]
  | .oplus a b, _, _, h => by
      show oplusMapTok (a.map h.inj) (b.map h.inj)
          = (oplusTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).inj
      rw [a.map_inj h, b.map_inj h, oplusMapTok_inj]
  | .otimes a b, _, _, h => by
      show otimesMapTok (a.map h.inj) (b.map h.inj)
          = (otimesTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).inj
      rw [a.map_inj h, b.map_inj h, otimesMapTok_inj]

theorem sumMapTok_proj (h0 : A₀.sys ? B₀.sys) (h1 : A₁.sys ? B₁.sys) :
    sumMapTok h0.proj h1.proj = (sumTok_subsystem h0 h1).proj := by
  have hsubM : ∀ {W : Set Str}, (B₀.sum B₁).sys.mem W → W ⊆ sumTokMaster A₀.sys A₁.sys
    := by
    rintro W (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩)
    · exact (show sumTokMaster B₀.sys B₁.sys = sumTokMaster A₀.sys A₁.sys by
        unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]).subset
    · exact (embBit_subset.mpr (by rw [h0.master_eq]; exact B₀.sys.sub_master hX)).
    trans
        (embF_subset_sumTokMaster A₀.sys.master_mem)
    · exact (embBit_subset.mpr (by rw [h1.master_eq]; exact B₁.sys.sub_master hY)).
    trans
        (embT_subset_sumTokMaster A₁.sys.master_mem)
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.proj_rel]
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, X', ⟨hX, hX', hXsub⟩, rfl, rfl⟩ | ⟨Y, Y', ⟨hY, hY', hYsub⟩,
    rfl, rfl⟩)
    · exact ⟨hW, (A₀.sum A₁).sys.master_mem, hsubM hW⟩
    · exact ⟨Or.inr (Or.inl ⟨X, hX, rfl⟩), Or.inr (Or.inl ⟨X', hX', rfl⟩), embBit_subset.
    mpr hXsub⟩
    · exact ⟨Or.inr (Or.inr ⟨Y, hY, rfl⟩), Or.inr (Or.inr ⟨Y', hY', rfl⟩), embBit_subset.
    mpr hYsub⟩
```

```
  · rintro ⟨hW, hW', hsub⟩
    rcases hW' with rfl | ⟨X', hX', rfl⟩ | ⟨Y', hY', rfl⟩
    · exact Or.inl ⟨hW, rfl⟩
    · rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact Or.inr (Or.inl ⟨X, X', ⟨hX, hX', embBit_subset.mp hsub⟩, rfl, rfl⟩)
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (B₁.ne Y hY) h)
    · rcases hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
      · exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      · exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (B₀.ne X hX) h)
      · exact Or.inr (Or.inr ⟨Y, Y', ⟨hY, hY', embBit_subset.mp hsub⟩, rfl, rfl⟩)

theorem GExpr.map_proj : (T : GExpr) → {X Y : ScottSys} → (h : X.sys ? Y.sys) →
    T.map h.proj = (T.obj_subsystem h).proj
  | .const D, _, _, _ => (Subsystem.self_proj (Subsystem.refl D.sys)).symm
  | .var, _, _, _ => rfl
  | .sum a b, _, _, h => by
      show sumMapTok (a.map h.proj) (b.map h.proj)
          = (sumTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).proj
      rw [a.map_proj h, b.map_proj h, sumMapTok_proj]
  | .prod a b, _, _, h => by
      show prodMapTok (a.map h.proj) (b.map h.proj)
          = (prodTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).proj
      rw [a.map_proj h, b.map_proj h, prodMapTok_proj]
  | .oplus a b, _, _, h => by
      show oplusMapTok (a.map h.proj) (b.map h.proj)
          = (oplusTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).proj
      rw [a.map_proj h, b.map_proj h, oplusMapTok_proj]
  | .otimes a b, _, _, h => by
      show otimesMapTok (a.map h.proj) (b.map h.proj)
          = (otimesTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).proj
      rw [a.map_proj h, b.map_proj h, otimesMapTok_proj]

section Uniqueness

variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master)

theorem map_rho_eq (n : ℕ) :
    (Texp N).map (rho hN n)
      = ((Texp N).obj_subsystem (expSub hN n)).inj.comp
        ((Texp N).obj_subsystem (expSub hN n)).proj := by
  unfold rho
  rw [(Texp N).map_comp (expSub hN n).proj (Subsystem.inj_isStrict (expSub hN n)),
      (Texp N).map_inj, (Texp N).map_proj]

theorem key_rho (n : ℕ) :
    rho hN (n + 1)
      = (expHom N hN).comp (((Texp N).map (rho hN n)).comp (expInv N hN)) := by
  have hsyseq : ((Texp N).obj (Exp N hN)).sys = (Exp N hN).sys :=
    gColim_obj_sys_eq (Texp N) (Texp_rooted hN)
  apply ApproximableMap.ext
  intro A E
  rw [map_rho_eq]
  simp only [comp_rel, rho_rel, expInv_rel, expHom_rel, Subsystem.proj_rel,
    Subsystem.inj_rel, hsyseq]
  constructor
```

```
  · rintro ⟨hcA, hcE, z, hTz, hAz, hzE⟩
    exact ⟨E, ⟨A, ⟨hcA, hcA, subset_rfl⟩, z, ⟨hcA, hTz, hAz⟩, hTz, hcE, hzE⟩,
      hcE, hcE, subset_rfl⟩
  · rintro ⟨Y, ⟨C, ⟨hcA, _, hAC⟩, z, ⟨_, hTz, hCz⟩, _, _, hzY⟩, _, hcE, hYE⟩
    exact ⟨hcA, hcE, z, hTz, hAC.trans hCz, hzY.trans hYE⟩

end Uniqueness

section Uniqueness

variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master)

variable (B : TAlgebra (TexpF N))

theorem gcomp_rho_succ (g : AlgHom (ExpAlg N hN) B) (n : ℕ) :
    g.hom.1.comp (rho hN (n + 1))
      = (algStr B).comp (((Texp N).map (g.hom.1.comp (rho hN n))).comp (expInv N hN))
    := by
  have hcomm : (g.hom.1).comp (expHom N hN) = (algStr B).comp ((Texp N).map g.hom.1)
    :=
    congrArg Subtype.val g.comm
  calc g.hom.1.comp (rho hN (n + 1))
      = g.hom.1.comp ((expHom N hN).comp
          (((Texp N).map (rho hN n)).comp (expInv N hN))) := by rw [key_rho]
    _ = (g.hom.1.comp (expHom N hN)).comp
          (((Texp N).map (rho hN n)).comp (expInv N hN)) :=
        (comp_assoc _ _ _).symm
    _ = ((algStr B).comp ((Texp N).map g.hom.1)).comp
          (((Texp N).map (rho hN n)).comp (expInv N hN)) :=
        congrArg (fun m => m.comp (((Texp N).map (rho hN n)).comp (expInv N hN)))
    hcomm
    _ = (algStr B).comp (((Texp N).map g.hom.1).comp
          (((Texp N).map (rho hN n)).comp (expInv N hN))) := comp_assoc _ _ _
    _ = (algStr B).comp ((((Texp N).map g.hom.1).comp ((Texp N).map (rho hN n))).comp
          (expInv N hN)) :=
        congrArg ((algStr B).comp ·)
          (comp_assoc ((Texp N).map g.hom.1) ((Texp N).map (rho hN n)) (expInv N hN)).
    symm
    _ = (algStr B).comp (((Texp N).map (g.hom.1.comp (rho hN n))).comp (expInv N hN))
    :=
        congrArg (fun m => (algStr B).comp (m.comp (expInv N hN)))
          ((Texp N).map_comp (rho hN n) g.hom.2).symm

theorem gcomp_rho_eq (g : AlgHom (ExpAlg N hN) B) :
    ∀ n, g.hom.1.comp (rho hN n) = descRel hN B n
  | 0 => gcomp_rho_zero hN B g
  | n + 1 => by rw [gcomp_rho_succ hN B g n, gcomp_rho_eq g n, ← descRel_succ]

theorem descMap_eq_algHom (g : AlgHom (ExpAlg N hN) B) : g.hom.1 = descMap hN B := by
  have hcomp : g.hom.1.comp (iSupRho hN) = descMap hN B := by
    apply ApproximableMap.ext
    intro A E
    rw [comp_rel, descMap_rel]
    constructor
    · rintro ⟨Y, ⟨n, hrho⟩, hg⟩
      refine ⟨n, ?_⟩
```

```
      rw [← gcomp_rho_eq hN B g n, comp_rel]
      exact ⟨Y, hrho, hg⟩
    · rintro ⟨n, hn⟩
      rw [← gcomp_rho_eq hN B g n, comp_rel] at hn
      obtain ⟨Y, hrho, hg⟩ := hn
      exact ⟨Y, ⟨n, hrho⟩, hg⟩
  calc g.hom.1 = g.hom.1.comp (iSupRho hN) := by
        rw [iSupRho_eq_id hN]; exact (comp_idMap g.hom.1).symm
    _ = descMap hN B := hcomp

end Uniqueness

end FunctorTokens
end
end DomainTheory.NeighbourhoodSystems
end
```

4.7.50 Exercise 6.22 — equations for the integers

NeighbourhoodSystems/DomainEquations/Exercise_6_22.lean

`Exercise_6_22.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.intExprDomains

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open DomainTheory.NeighbourhoodSystems.FunctorTokens BitDom

/-- Scott's Exercise 6.22: an N? solution over a rooted N. -/
theorem example_622 :
    ∃ N : ScottSys, ([] : Str) ∈ N.sys.master ∧
      ∃ (Γ : Set Str) (h : Γ.Nonempty),
        (singletonSys Γ h).sys ? ((NStarExpr N).obj (singletonSys Γ h)).sys :=
  @NStar_over_N_exists
```

EXERCISE 6.22. Comment on these domain equations:

$$N \cong \{\{0\}, \{0, \Lambda\}\} \oplus N,$$

$$M \cong \{\{\Lambda\}\} + M,$$

$$N^* \cong N \oplus (N \otimes N^*).$$

4.7.51 exprDomain — syntactic Exp

`exprDomain.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.intExprDomains
public import DomainTheory.NeighbourhoodSystems.DomainEquations.continuousOnMaps

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.
    FunctorTokens
section
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
     DomainTheory.NeighbourhoodSystems.SmashProduct
namespace FunctorTokens
variable {A₀ A₁ B₀ B₁ : ScottSys}
section Uniqueness
variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master)
variable (B : TAlgebra (TexpF N))

def ExpInitial : IsInitial (ExpAlg N hN) where
  desc B := descAlgHom hN B
  uniq B g := algHom_ext (Subtype.ext (descMap_eq_algHom hN B g))

end Uniqueness

end FunctorTokens
end

section
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
     DomainTheory.NeighbourhoodSystems.SmashProduct

namespace FunctorTokens

theorem Subsystem.proj_isStrict {α : Type*} {D E : NeighborhoodSystem α} (h : D ? E) :
    IsStrict h.proj := by
  intro X hrel
  rw [Subsystem.proj_rel] at hrel
  obtain ⟨_, hXD, hsub⟩ := hrel
  exact Set.Subset.antisymm (D.sub_master hXD) (by rw [h.master_eq]; exact hsub)

theorem expHom_comp_expInv (N : ScottSys) (hN : ([] : Str) ∈ N.sys.master) :
    (expHom N hN).comp (expInv N hN) = idMap (Exp N hN).sys := by
  have h := congrArg (Subtype.val) (ExpIso N hN).inv_hom_id
  exact h

section Existence

variable {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master) (B : TAlgebra (TexpF N))

theorem algHom_fix (g : AlgHom (ExpAlg N hN) B) :
    g.hom.1 = (algStr B).comp (((Texp N).map g.hom.1).comp (expInv N hN)) := by
  have hcomm : (g.hom.1).comp (expHom N hN) = (algStr B).comp ((Texp N).map g.hom.1)
    :=
    congrArg Subtype.val g.comm
  calc g.hom.1
```

```
      = (g.hom.1).comp (idMap (Exp N hN).sys) := (comp_idMap _).symm
    _ = (g.hom.1).comp ((expHom N hN).comp (expInv N hN)) := by rw [expHom_comp_expInv
    ]
    _ = ((g.hom.1).comp (expHom N hN)).comp (expInv N hN) := (comp_assoc _ _ _).symm
    _ = ((algStr B).comp ((Texp N).map g.hom.1)).comp (expInv N hN) :=
          congrArg (fun m => m.comp (expInv N hN)) hcomm
    _ = (algStr B).comp (((Texp N).map g.hom.1).comp (expInv N hN)) := comp_assoc _ _
    _

theorem descRel_le_algHom (g : AlgHom (ExpAlg N hN) B) : ∀ n, descRel hN B n ≤ g.hom.1
  | 0 => constBot_le _
  | n + 1 => by
      rw [descRel_succ, algHom_fix hN B g]
      exact comp_mono_gen le_rfl
        (comp_mono_gen ((Texp N).map_mono (descRel_le_algHom g n)) le_rfl)

theorem descMap_le_algHom (g : AlgHom (ExpAlg N hN) B) : descMap hN B ≤ g.hom.1 := by
  intro X Y hr
  obtain ⟨n, hn⟩ := hr
  exact descRel_le_algHom hN B g n X Y hn

end Existence

end FunctorTokens
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.FunctorTokens
open NeighborhoodSystem ApproximableMap SeqEqB BitDom SmashProduct
```

4.7.52 Exercise 6.23 — syntactic expression domain

NeighbourhoodSystems/DomainEquations/Exercise_6_23.lean

Exercise_6_23.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.exprDomain

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap SeqEqB BitDom DomainTheory.
    NeighbourhoodSystems.SmashProduct DomainTheory.NeighbourhoodSystems.FunctorTokens

/-- Scott's Exercise 6.23: the descending approximable map is below any algebra
    homomorphism. -/
theorem example_623 {N : ScottSys} (hN : ([] : Str) ∈ N.sys.master) (B : TAlgebra (
    TexpF N))
    (g : AlgHom (ExpAlg N hN) B) :
    descMap hN B ≤ g.hom.1 :=
  @descMap_le_algHom _ hN B g
```

EXERCISE 6.23. Construe the initial solution to

$$Exp \cong N \oplus ((Exp \times Exp) + (Exp \times Exp))$$

as a "syntactical domain" of expressions generated from infinitely many "variables" by means of two binary "operation symbols". Given an algebra D with two operations

$$u : D \times D \to D \quad \text{and} \quad v : D \times D \to D,$$

show how any strict map $s : N \to D$ determines a unique map

$$val(s) : Exp \to D$$

that can be regarded as the "evaluation of an expression".

4.7.53 recDomainEq — simultaneous solutions

recDomainEq.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.exprDomain
public import DomainTheory.NeighbourhoodSystems.DomainEquations.functorTokens

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem DomainTheory.NeighbourhoodSystems.FunctorTokens
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom

namespace RecDomainEq

def Fsol (D E : ScottSys) : ScottSys := D.sum (D.prod E)

def Gsol (D E : ScottSys) : ScottSys := D.sum E

def gTok (p q : Set Str) : Set Str := insert ([] : Str) (embBit false p ∪ embBit true
    q)

def fTok (p q : Set Str) : Set Str := gTok p (gTok p q)

def pIter : ℕ → Set Str × Set Str
  | 0 => ({([] : Str)}, {([] : Str)})
  | n + 1 => (fTok (pIter n).1 (pIter n).2, gTok (pIter n).1 (pIter n).2)

def GammaD : Set Str := ⋃ n, (pIter n).1

def GammaE : Set Str := ⋃ n, (pIter n).2

theorem nil_mem_GammaD : ([] : Str) ∈ GammaD := Set.mem_iUnion.mpr ⟨0, rfl⟩

def Dsol : ScottSys := singletonSys GammaD ⟨[], nil_mem_GammaD⟩

theorem nil_mem_GammaE : ([] : Str) ∈ GammaE := Set.mem_iUnion.mpr ⟨0, rfl⟩
```

```
def Esol : ScottSys := singletonSys GammaE ⟨[], nil_mem_GammaE⟩

theorem gTok_mono {p p' q q' : Set Str} (hp : p ⊆ p') (hq : q ⊆ q') : gTok p q ⊆ gTok
    p' q' := by
  rintro w (rfl | ⟨w', rfl, hw'⟩ | ⟨w', rfl, hw'⟩)
  · exact Set.mem_insert _ _
  · exact Or.inr (Or.inl ⟨w', rfl, hp hw'⟩)
  · exact Or.inr (Or.inr ⟨w', rfl, hq hw'⟩)

theorem fTok_mono {p p' q q' : Set Str} (hp : p ⊆ p') (hq : q ⊆ q') : fTok p q ⊆ fTok
    p' q' :=
  gTok_mono hp (gTok_mono hp hq)

theorem master_Fsol : (Fsol Dsol Esol).sys.master = fTok GammaD GammaE := rfl

theorem master_Gsol : (Gsol Dsol Esol).sys.master = gTok GammaD GammaE := rfl

theorem nil_mem_fTok (p q : Set Str) : ([] : Str) ∈ fTok p q := Set.mem_insert _ _

theorem nil_mem_gTok (p q : Set Str) : ([] : Str) ∈ gTok p q := Set.mem_insert _ _

theorem mem_fTok_iUnion {a b : ℕ → Set Str} {w : Str}
    (hw : w ∈ fTok (⋃ n, a n) (⋃ n, b n)) : ∃ n, w ∈ fTok (a n) (b n) := by
  rcases hw with rfl | ⟨w', rfl, hw'⟩ | ⟨u, rfl, hu⟩
  · exact ⟨0, nil_mem_fTok _ _⟩
  · obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hw'
    exact ⟨n, Or.inr (Or.inl ⟨w', rfl, hn⟩)⟩
  · rcases hu with rfl | ⟨u', rfl, hu'⟩ | ⟨u', rfl, hu'⟩
    · exact ⟨0, Or.inr (Or.inr ⟨[], rfl, nil_mem_gTok _ _⟩)⟩
    · obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hu'
      exact ⟨n, Or.inr (Or.inr ⟨false :: u', rfl, Or.inr (Or.inl ⟨u', rfl, hn⟩)⟩)⟩
    · obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hu'
      exact ⟨n, Or.inr (Or.inr ⟨true :: u', rfl, Or.inr (Or.inr ⟨u', rfl, hn⟩)⟩)⟩

theorem mem_gTok_iUnion {a b : ℕ → Set Str} {w : Str}
    (hw : w ∈ gTok (⋃ n, a n) (⋃ n, b n)) : ∃ n, w ∈ gTok (a n) (b n) := by
  rcases hw with rfl | ⟨w', rfl, hw'⟩ | ⟨w', rfl, hw'⟩
  · exact ⟨0, nil_mem_gTok _ _⟩
  · obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hw'
    exact ⟨n, Or.inr (Or.inl ⟨w', rfl, hn⟩)⟩
  · obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hw'
    exact ⟨n, Or.inr (Or.inr ⟨w', rfl, hn⟩)⟩

theorem pIter_fst_subset_GammaD (m : ℕ) : (pIter m).1 ⊆ GammaD :=
  fun _ hx => Set.mem_iUnion.mpr ⟨m, hx⟩

theorem pIter_snd_subset_GammaE (m : ℕ) : (pIter m).2 ⊆ GammaE :=
  fun _ hx => Set.mem_iUnion.mpr ⟨m, hx⟩

theorem fTok_GammaD_GammaE : fTok GammaD GammaE = GammaD := by
  apply Set.Subset.antisymm
  · intro w hw
    obtain ⟨n, hn⟩ :=
      mem_fTok_iUnion (a := fun n => (pIter n).1) (b := fun n => (pIter n).2) hw
    exact Set.mem_iUnion.mpr ⟨n + 1, hn⟩
```

```
  · intro w hw
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hw
    cases n with
    | zero =>
        have hw0 : w = [] := hn; subst hw0; exact nil_mem_fTok _ _
    | succ m =>
        exact fTok_mono (pIter_fst_subset_GammaD m) (pIter_snd_subset_GammaE m) hn

theorem Dsol_subsystem : Dsol.sys ? (Fsol Dsol Esol).sys := by
  have hmaster : (Fsol Dsol Esol).sys.master = GammaD := master_Fsol.trans
    fTok_GammaD_GammaE
  refine ⟨hmaster.symm, ?_, ?_⟩
  · intro X hX
    have hXeq : X = GammaD := hX
    rw [hXeq, ← hmaster]
    exact (Fsol Dsol Esol).sys.master_mem
  · intro X Y hX hY _
    show X ∩ Y = GammaD
    rw [show X = GammaD from hX, show Y = GammaD from hY, Set.inter_self]

theorem gTok_GammaD_GammaE : gTok GammaD GammaE = GammaE := by
  apply Set.Subset.antisymm
  · intro w hw
    obtain ⟨n, hn⟩ :=
      mem_gTok_iUnion (a := fun n => (pIter n).1) (b := fun n => (pIter n).2) hw
    exact Set.mem_iUnion.mpr ⟨n + 1, hn⟩
  · intro w hw
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hw
    cases n with
    | zero =>
        have hw0 : w = [] := hn; subst hw0; exact nil_mem_gTok _ _
    | succ m =>
        exact gTok_mono (pIter_fst_subset_GammaD m) (pIter_snd_subset_GammaE m) hn

theorem Esol_subsystem : Esol.sys ? (Gsol Dsol Esol).sys := by
  have hmaster : (Gsol Dsol Esol).sys.master = GammaE := master_Gsol.trans
    gTok_GammaD_GammaE
  refine ⟨hmaster.symm, ?_, ?_⟩
  · intro X hX
    have hXeq : X = GammaE := hX
    rw [hXeq, ← hmaster]
    exact (Gsol Dsol Esol).sys.master_mem
  · intro X Y hX hY _
    show X ∩ Y = GammaE
    rw [show X = GammaE from hX, show Y = GammaE from hY, Set.inter_self]

theorem exists_double_fixedPoint :
    ∃ Γd Γe : Set Str, ([] : Str) ∈ Γd ∧ ([] : Str) ∈ Γe ∧
      fTok Γd Γe = Γd ∧ gTok Γd Γe = Γe :=
  ⟨GammaD, GammaE, nil_mem_GammaD, nil_mem_GammaE, fTok_GammaD_GammaE,
    gTok_GammaD_GammaE⟩

end RecDomainEq
end DomainTheory.NeighbourhoodSystems
end
```

```
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem DomainTheory.NeighbourhoodSystems.FunctorTokens
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
namespace RecDomainEq

theorem exists_simultaneous_subsystems :
    ∃ D E : ScottSys, D.sys ? (D.sum (D.prod E)).sys ∧ E.sys ? (D.sum E).sys :=
  ⟨Dsol, Esol, Dsol_subsystem, Esol_subsystem⟩

end RecDomainEq

end DomainTheory.NeighbourhoodSystems
```

4.7.54 Exercise 6.24 — recursive domain equations

NeighbourhoodSystems/DomainEquations/Exercise_6_24.lean

`Exercise_6_24.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.recDomainEq

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open DomainTheory.NeighbourhoodSystems.FunctorTokens
/-- Scott's Exercise 6.24: simultaneous subsystem solutions. -/
theorem example_624 :
    ∃ D E : ScottSys, D.sys ? (D.sum (D.prod E)).sys ∧ E.sys ? (D.sum E).sys :=
  ⟨RecDomainEq.Dsol, RecDomainEq.Esol, RecDomainEq.Dsol_subsystem, RecDomainEq.
    Esol_subsystem⟩
```

EXERCISE 6.24. Show that there must exist domains satisfying:

$$D \cong D + (D \times E), \quad \text{and}$$

$$E \cong D + E,$$

by using a double fixed-point method. First decide what the underlying set of tokens should be, and then define D and E by simultaneous fixed points. (Syntactical domains as in 6.23 may very well require several simultaneous equations.)

4.7.55 projPairLaws — projection pair laws

`projPairLaws.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Lemma_6_15
```

```
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more
import Mathlib.Data.Fintype.Powerset

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap Subsystem
section
variable {α : Type*} {D E : NeighborhoodSystem α}
namespace Subsystem.ProjectionPair

theorem inj_bounded (P : ProjectionPair D E) {S : Set D.Element} (hS : D.Bounded S) :
    E.Bounded (P.inj.toElementMap '' S) := by
  obtain ⟨b, hb⟩ := hS
  refine ⟨P.inj.toElementMap b, ?_⟩
  rintro _ ⟨s, hs, rfl⟩
  exact P.inj.toElementMap_mono (hb s hs)

theorem inj_proj_apply_le (P : ProjectionPair D E) (y : E.Element) :
    P.inj.toElementMap (P.proj.toElementMap y) ≤ y := by
  have h := (le_iff_toElementMap_le.mp P.inj_comp_proj_le) y
  rwa [toElementMap_comp, toElementMap_idMap] at h

theorem proj_inj_apply (P : ProjectionPair D E) (x : D.Element) :
    P.proj.toElementMap (P.inj.toElementMap x) = x := by
  rw [← toElementMap_comp, P.proj_comp_inj, toElementMap_idMap]

theorem galois (P : ProjectionPair D E) (x : D.Element) (y : E.Element) :
    P.inj.toElementMap x ≤ y ↔ x ≤ P.proj.toElementMap y := by
  constructor
  · intro hxy
    have := P.proj.toElementMap_mono hxy
    rwa [P.proj_inj_apply x] at this
  · intro hxy
    exact (P.inj.toElementMap_mono hxy).trans (P.inj_proj_apply_le y)

theorem inj_sSup (P : ProjectionPair D E) {S : Set D.Element} (hS : D.Bounded S) :
    P.inj.toElementMap (D.sSup S hS) =
      E.sSup (P.inj.toElementMap '' S) (P.inj_bounded hS) := by
  apply le_antisymm
  · refine (P.galois _ _).mpr (D.sSup_le S hS fun s hs => ?_)
    refine (P.galois s _).mp ?_
    exact E.le_sSup _ (P.inj_bounded hS) ⟨s, hs, rfl⟩
  · refine E.sSup_le _ (P.inj_bounded hS) ?_
    rintro _ ⟨s, hs, rfl⟩
    exact P.inj.toElementMap_mono (D.le_sSup S hS hs)

theorem lowerSet_bounded (P : ProjectionPair D E) (y : E.Element) :
    D.Bounded (P.lowerSet y) :=
  ⟨P.proj.toElementMap y, fun _ hx => (P.galois _ y).mp hx⟩

theorem lowerSet_directed (P : ProjectionPair D E) (y : E.Element) :
    ∀ a ∈ P.lowerSet y, ∀ b ∈ P.lowerSet y,
```

```
      ∃ c ∈ P.lowerSet y, a ≤ c ∧ b ≤ c := by
  intro a ha b hb
  exact ⟨P.proj.toElementMap y, P.inj_proj_apply_le y,
    (P.galois a y).mp ha, (P.galois b y).mp hb⟩

theorem proj_eq_sSup (P : ProjectionPair D E) (y : E.Element) :
    P.proj.toElementMap y = D.sSup (P.lowerSet y) (P.lowerSet_bounded y) := by
  apply le_antisymm
  · exact D.le_sSup _ (P.lowerSet_bounded y) (P.inj_proj_apply_le y)
  · exact D.sSup_le _ (P.lowerSet_bounded y) fun x hx => (P.galois x y).mp hx

theorem upperSet_nonempty (P : ProjectionPair D E) (x : D.Element) :
    (P.upperSet x).Nonempty :=
  ⟨P.inj.toElementMap x, by rw [mem_upperSet, P.proj_inj_apply x]⟩

theorem inj_eq_sInf (P : ProjectionPair D E) (x : D.Element) :
    P.inj.toElementMap x = E.sInf (P.upperSet x) (P.upperSet_nonempty x) := by
  apply le_antisymm
  · exact E.le_sInf _ (P.upperSet_nonempty x) _ fun y hy => (P.galois x y).mpr hy
  · refine E.sInf_le _ (P.upperSet_nonempty x) ?_
    rw [mem_upperSet, P.proj_inj_apply x]

end Subsystem.ProjectionPair
end
end DomainTheory.NeighbourhoodSystems
```

4.7.56 Exercise 6.25 — projection pair laws

NeighbourhoodSystems/DomainEquations/Exercise_6_25.lean

Exercise_6_25.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.projPairLaws

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
open Subsystem
open Subsystem.ProjectionPair

/-- Scott's Exercise 6.25: Galois connection and continuity laws for projection pairs.
    -/
theorem example_625 :
    (∀ {α : Type*} {D E : NeighborhoodSystem α} (P : ProjectionPair D E) (x : D.
    Element) (y : E.Element),
        P.inj.toElementMap x ≤ y ↔ x ≤ P.proj.toElementMap y) ∧
    (∀ {α : Type*} {D E : NeighborhoodSystem α} (P : ProjectionPair D E) {S : Set D.
    Element},
        D.Bounded S → E.Bounded (P.inj.toElementMap '' S)) ∧
    (∀ {α : Type*} {D E : NeighborhoodSystem α} (P : ProjectionPair D E) (x : D.
    Element),
        P.inj.toElementMap x = E.sInf (P.upperSet x) (P.upperSet_nonempty x)) ∧
```

```
    (∀ {α : Type*} {D E : NeighborhoodSystem α} (P : ProjectionPair D E) {S : Set D.
    Element} (hS : D.Bounded S),
        P.inj.toElementMap (D.sSup S hS) = E.sSup (P.inj.toElementMap '' S) (P.
    inj_bounded hS)) ∧
    (∀ {α : Type*} {D E : NeighborhoodSystem α} (P : ProjectionPair D E) (y : E.
    Element),
        ∀ a ∈ P.lowerSet y, ∀ b ∈ P.lowerSet y, ∃ c ∈ P.lowerSet y, a ≤ c ∧ b ≤ c) ∧
    (∀ {α : Type*} {D E : NeighborhoodSystem α} (P : ProjectionPair D E) (y : E.
    Element),
        P.proj.toElementMap y = D.sSup (P.lowerSet y) (P.lowerSet_bounded y)) ∧
    (∀ {α : Type*} {D E : NeighborhoodSystem α} (P : ProjectionPair D E) (x : D.
    Element),
        (P.upperSet x).Nonempty) :=
  ⟨@galois, ⟨@inj_bounded, ⟨@inj_eq_sInf, ⟨@inj_sSup, ⟨@lowerSet_directed, ⟨
    @proj_eq_sSup, @upperSet_nonempty⟩⟩⟩⟩⟩⟩
```

EXERCISE 6.25. For a projection pair $g : \mathcal{D} \to \mathcal{E}$ and $h : \mathcal{E} \to \mathcal{D}$ show that for $x \in |\mathcal{D}|$ and $y \in |\mathcal{E}|$ we have:

$$g(x) \sqsubseteq y \text{ iff } x \sqsubseteq h(y).$$

Thus, conclude that:

$$h(y) = \bigsqcup\{x \in |\mathcal{D}| \mid g(x) \sqsubseteq y\}, \quad \text{and}$$

$$g(x) = \bigsqcap\{y \in |\mathcal{E}| \mid x \sqsubseteq h(y)\},$$

for all $x \in |\mathcal{D}|$ and $y \in |\mathcal{E}|$. So each of the functions determines the other. In the first equation check that the set on the right is directed, and in the second equation that the set on the right is non empty. Prove also that g maps consistent sets to consistent sets and preserves $\bigsqcup$ (not just directed unions).

4.7.57 liftDomain — lifting a domain

liftDomain.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.recDomainEq

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_21
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_10
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.
    FunctorTokens
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
open DomainTheory.NeighbourhoodSystems.SmashProduct
section
```

```
namespace FunctorTokens
variable {D : NeighborhoodSystem Str}

def liftTokMaster (D : NeighborhoodSystem Str) : Set Str := insert [] (embBit false D.
    master)

theorem embF_subset_liftTokMaster {X : Set Str} (hX : D.mem X) :
    embBit false X ⊆ liftTokMaster D :=
  (embBit_subset.mpr (D.sub_master hX)).trans (Set.subset_insert _ _)

theorem liftTokMaster_inter_embF {X : Set Str} (hX : D.mem X) :
    liftTokMaster D ∩ embBit false X = embBit false X :=
  Set.inter_eq_right.mpr (embF_subset_liftTokMaster hX)

theorem nil_mem_liftTokMaster : ([] : Str) ∈ liftTokMaster D := Set.mem_insert _ _

theorem embF_ne_liftTokMaster {X : Set Str} : embBit false X = liftTokMaster D := fun
    h =>
  nil_not_mem_embBit (h.symm ▶ nil_mem_liftTokMaster)

def liftTok (D : NeighborhoodSystem Str) (_hD : ∀ X, D.mem X → X.Nonempty) :
    NeighborhoodSystem Str where
  mem W := W = liftTokMaster D ∨ ∃ X, D.mem X ∧ W = embBit false X
  master := liftTokMaster D
  master_nonempty := ⟨[], nil_mem_liftTokMaster⟩
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | ⟨X, hX, rfl⟩)
    · exact subset_rfl
    · exact embF_subset_liftTokMaster hX
  inter_mem := by
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | ⟨X, hX, rfl⟩
    · rcases hW' with rfl | ⟨X', hX', rfl⟩
      · rw [Set.inter_self]; exact Or.inl rfl
      · rw [liftTokMaster_inter_embF hX']; exact Or.inr ⟨X', hX', rfl⟩
    · rcases hW' with rfl | ⟨X', hX', rfl⟩
      · rw [Set.inter_comm, liftTokMaster_inter_embF hX]; exact Or.inr ⟨X, hX, rfl⟩
      · rw [embBit_inter] at hZsub ⊢
        rcases hZ with rfl | ⟨Z', hZ', rfl⟩
        · exact absurd (hZsub nil_mem_liftTokMaster) nil_not_mem_embBit
        · exact Or.inr ⟨X ∩ X', D.inter_mem hX hX' hZ' (embBit_subset.mp hZsub), rfl⟩

end FunctorTokens
end

section
namespace FunctorTokens

variable {D : NeighborhoodSystem Str}

variable {hD : ∀ X, D.mem X → X.Nonempty}

def liftBot (D : NeighborhoodSystem Str) (hD : ∀ X, D.mem X → X.Nonempty) :
    (liftTok D hD).Element where
  mem W := W = liftTokMaster D
```

```
  sub := by rintro W rfl; exact Or.inl rfl
  master_mem := rfl
  inter_mem := by rintro W W' rfl rfl; rw [Set.inter_self]
  up_mem := by
    rintro W W' rfl hW' hsub
    exact Set.Subset.antisymm ((liftTok D hD).sub_master hW') hsub

theorem liftTok_mem_embF {X : Set Str} (hX : D.mem X) :
    (liftTok D hD).mem (embBit false X) := Or.inr ⟨X, hX, rfl⟩

theorem liftTok_mem_embF_inv {W : Set Str} (h : (liftTok D hD).mem (embBit false W)) :
    D.mem W := by
  rcases h with h0 | ⟨X, hX, heq⟩
  · exact absurd (h0.symm ▸ nil_mem_liftTokMaster) nil_not_mem_embBit
  · rw [embBit_injective heq]; exact hX

end FunctorTokens
end

section
namespace FunctorTokens

variable {D : NeighborhoodSystem Str}

theorem liftTok_nonempty (hD : ∀ X, D.mem X → X.Nonempty) :
    ∀ W, (liftTok D hD).mem W → W.Nonempty := by
  rintro W (rfl | ⟨X, hX, rfl⟩)
  · exact ⟨[], nil_mem_liftTokMaster⟩
  · exact embBit_nonempty (hD X hX)

def ScottSys.lift (A : ScottSys) : ScottSys := ⟨liftTok A.sys A.ne, liftTok_nonempty A.
    ne⟩

end FunctorTokens
end

section
namespace FunctorTokens

variable {D : NeighborhoodSystem Str}

variable {hD : ∀ X, D.mem X → X.Nonempty}

def liftUp {D : NeighborhoodSystem Str} {hD : ∀ X, D.mem X → X.Nonempty} (x : D.
    Element) :
    (liftTok D hD).Element where
  mem W := W = liftTokMaster D ∨ ∃ X, x.mem X ∧ W = embBit false X
  sub := by
    rintro W (rfl | ⟨X, hX, rfl⟩)
    · exact Or.inl rfl
    · exact Or.inr ⟨X, x.sub hX, rfl⟩
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl⟩) (rfl | ⟨X', hX', rfl⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr ⟨X', hX', by rw [liftTokMaster_inter_embF (x.sub hX')]⟩
```

```
    · exact Or.inr ⟨X, hX, by rw [Set.inter_comm, liftTokMaster_inter_embF (x.sub hX)]⟩
    · exact Or.inr ⟨X ∩ X', x.inter_mem hX hX', by rw [embBit_inter]⟩
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl⟩) hW' hsub
    · exact Or.inl (Set.Subset.antisymm ((liftTok D hD).sub_master hW') hsub)
    · rcases hW' with rfl | ⟨X', hX', rfl⟩
      · exact Or.inl rfl
      · exact Or.inr ⟨X', x.up_mem hX hX' (embBit_subset.mp hsub), rfl⟩

@[simp] theorem mem_liftBot {W : Set Str} : (liftBot D hD).mem W ↔ W = liftTokMaster D
    := Iff.rfl

@[simp] theorem mem_liftUp {x : D.Element} {W : Set Str} :
    (liftUp (hD := hD) x).mem W ↔ W = liftTokMaster D ∨ ∃ X, x.mem X ∧ W = embBit
    false X := Iff.rfl

end FunctorTokens
end

section
namespace FunctorTokens

variable {D : NeighborhoodSystem Str}

variable {hD : ∀ X, D.mem X → X.Nonempty}

variable {A B C : ScottSys}

def liftMapTok (f : ApproximableMap A.sys B.sys) :
    ApproximableMap (ScottSys.lift A).sys (ScottSys.lift B).sys where
  rel W W' :=
    ((liftTok A.sys A.ne).mem W ∧ W' = liftTokMaster B.sys) ∨
    (∃ X X', f.rel X X' ∧ W = embBit false X ∧ W' = embBit false X')
  rel_dom := by
    rintro W W' (⟨hW, -⟩ | ⟨X, X', hrel, rfl, -⟩)
    · exact hW
    · exact liftTok_mem_embF (hD := A.ne) (f.rel_dom hrel)
  rel_cod := by
    rintro W W' (⟨-, rfl⟩ | ⟨X, X', hrel, -, rfl⟩)
    · exact Or.inl rfl
    · exact liftTok_mem_embF (hD := B.ne) (f.rel_cod hrel)
  master_rel := Or.inl ⟨(ScottSys.lift A).sys.master_mem, rfl⟩
  inter_right := by
    rintro W W'₁ W'₂ h1 h2
    rcases h1 with ⟨hW, rfl⟩ | ⟨X, X', hrel, rfl, rfl⟩
    · rcases h2 with ⟨-, rfl⟩ | ⟨X, X', hrel, hWeq, rfl⟩
      · exact Or.inl ⟨hW, by rw [Set.inter_self]⟩
      · exact Or.inr ⟨X, X', hrel, hWeq, by rw [liftTokMaster_inter_embF (f.rel_cod
    hrel)]⟩
    · rcases h2 with ⟨-, rfl⟩ | ⟨X₂, X'₂, hrel₂, hWeq, rfl⟩
      · refine Or.inr ⟨X, X', hrel, rfl, ?_⟩
        rw [Set.inter_comm, liftTokMaster_inter_embF (f.rel_cod hrel)]
      · obtain rfl := embBit_injective hWeq
        exact Or.inr ⟨X, X' ∩ X'₂, f.inter_right hrel hrel₂, rfl, embBit_inter false X'
    X'₂⟩
  mono := by
```

```
    rintro W W'' Z Z' h hWW hZZ' hZmem hZ'mem
    rcases h with ⟨-, rfl⟩ | ⟨X, X', hrel, rfl, rfl⟩
    · exact Or.inl ⟨hZmem, Set.Subset.antisymm ((ScottSys.lift B).sys.sub_master hZ'mem
    ) hZZ'⟩
    · rcases hZ'mem with rfl | ⟨X₃, hX₃, rfl⟩
      · exact Or.inl ⟨hZmem, rfl⟩
      · rcases hZmem with rfl | ⟨X₂, hX₂, rfl⟩
        · exact absurd (hWW nil_mem_liftTokMaster) nil_not_mem_embBit
        · exact Or.inr ⟨X₂, X₃,
            f.mono hrel (embBit_subset.mp hWW) (embBit_subset.mp hZZ') hX₂ hX₃, rfl,
    rfl⟩

end FunctorTokens
end

section
namespace FunctorTokens

variable {D : NeighborhoodSystem Str}

variable {hD : ∀ X, D.mem X → X.Nonempty}

variable {A B C : ScottSys}

variable {D E : ScottSys}

theorem o_embFF_inv {W : Set Str}
    (h : (D.lift.oplus E.lift).sys.mem (embBit false (embBit false W))) : D.sys.mem W
    :=
  liftTok_mem_embF_inv (hD := D.ne)
    (oplusTok_mem_embF_inv (D₀ := D.lift.sys) (D₁ := E.lift.sys)
      (h₀ := D.lift.ne) (h₁ := E.lift.ne) h)

theorem o_embTF_inv {W : Set Str}
    (h : (D.lift.oplus E.lift).sys.mem (embBit true (embBit false W))) : E.sys.mem W
    :=
  liftTok_mem_embF_inv (hD := E.ne)
    (oplusTok_mem_embT_inv (D₀ := D.lift.sys) (D₁ := E.lift.sys)
      (h₀ := D.lift.ne) (h₁ := E.lift.ne) h)

theorem o_mem_embFF {X' : Set Str} (hX' : D.sys.mem X') :
    (D.lift.oplus E.lift).sys.mem (embBit false (embBit false X')) :=
  oplusTok_mem_embF (h₀ := D.lift.ne) (h₁ := E.lift.ne)
    (liftTok_mem_embF (hD := D.ne) hX') (embF_ne_liftTokMaster (D := D.sys))

theorem o_mem_embTF {Y' : Set Str} (hY' : E.sys.mem Y') :
    (D.lift.oplus E.lift).sys.mem (embBit true (embBit false Y')) :=
  oplusTok_mem_embT (h₀ := D.lift.ne) (h₁ := E.lift.ne)
    (liftTok_mem_embF (hD := E.ne) hY') (embF_ne_liftTokMaster (D := E.sys))

def toSumLift (z : (D.lift.oplus E.lift).sys.Element) : (D.sum E).sys.Element where
  mem W := W = sumTokMaster D.sys E.sys
    ∨ (∃ X, D.sys.mem X ∧ W = embBit false X ∧ z.mem (embBit false (embBit false X)))
    ∨ (∃ Y, E.sys.mem Y ∧ W = embBit true Y ∧ z.mem (embBit true (embBit false Y)))
  sub := by
    rintro W (rfl | ⟨X, hX, rfl, -⟩ | ⟨Y, hY, rfl, -⟩)
```

```
    · exact Or.inl rfl
    · exact sumTok_mem_embF (h₀ := D.ne) (h₁ := E.ne) hX
    · exact sumTok_mem_embT (h₀ := D.ne) (h₁ := E.ne) hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩)
      (rfl | ⟨X', hX', rfl, hzX'⟩ | ⟨Y', hY', rfl, hzY'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (Or.inl ⟨X', hX', by rw [sumTokMaster_inter_embF hX'], hzX'⟩)
    · exact Or.inr (Or.inr ⟨Y', hY', by rw [sumTokMaster_inter_embT hY'], hzY'⟩)
    · exact Or.inr (Or.inl ⟨X, hX, by rw [Set.inter_comm, sumTokMaster_inter_embF hX],
    hzX⟩)
    · refine Or.inr (Or.inl ⟨X ∩ X', ?_, by rw [embBit_inter], ?_⟩)
      · have hz := z.inter_mem hzX hzX'; rw [embBit_inter, embBit_inter] at hz
        exact o_embFF_inv (z.sub hz)
      · have hz := z.inter_mem hzX hzX'; rwa [embBit_inter, embBit_inter] at hz
    · exfalso
      have hz := z.inter_mem hzX hzY'
      rw [embBit_inter_ne (show (false : Bool) = true by decide)] at hz
      obtain ⟨t, ht⟩ := (D.lift.oplus E.lift).ne _ (z.sub hz); exact Set.notMem_empty t
     ht
    · exact Or.inr (Or.inr ⟨Y, hY, by rw [Set.inter_comm, sumTokMaster_inter_embT hY],
    hzY⟩)
    · exfalso
      have hz := z.inter_mem hzY hzX'
      rw [embBit_inter_ne (show (true : Bool) = false by decide)] at hz
      obtain ⟨t, ht⟩ := (D.lift.oplus E.lift).ne _ (z.sub hz); exact Set.notMem_empty t
     ht
    · refine Or.inr (Or.inr ⟨Y ∩ Y', ?_, by rw [embBit_inter], ?_⟩)
      · have hz := z.inter_mem hzY hzY'; rw [embBit_inter, embBit_inter] at hz
        exact o_embTF_inv (z.sub hz)
      · have hz := z.inter_mem hzY hzY'; rwa [embBit_inter, embBit_inter] at hz
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩) hW' hsub
    · exact Or.inl (Set.Subset.antisymm ((D.sum E).sys.sub_master hW') hsub)
    · rcases hW' with rfl | ⟨X'', hX'', rfl⟩ | ⟨Y'', hY'', rfl⟩
      · exact Or.inl rfl
      · refine Or.inr (Or.inl ⟨X'', hX'', rfl, ?_⟩)
        exact z.up_mem hzX (o_mem_embFF hX'')
          (embBit_subset.mpr (embBit_subset.mpr (embBit_subset.mp hsub)))
      · exact absurd hsub
          (fun hs => embBit_not_subset_cross (show (false : Bool) = true by decide) (D
    .ne X hX) hs)
    · rcases hW' with rfl | ⟨X'', hX'', rfl⟩ | ⟨Y'', hY'', rfl⟩
      · exact Or.inl rfl
      · exact absurd hsub
          (fun hs => embBit_not_subset_cross (show (true : Bool) = false by decide) (E
    .ne Y hY) hs)
      · refine Or.inr (Or.inr ⟨Y'', hY'', rfl, ?_⟩)
        exact z.up_mem hzY (o_mem_embTF hY'')
          (embBit_subset.mpr (embBit_subset.mpr (embBit_subset.mp hsub)))

@[simp] theorem toSumLift_mem_embF {z : (D.lift.oplus E.lift).sys.Element} {X : Set
    Str}
    (hX : D.sys.mem X) :
    (toSumLift z).mem (embBit false X) ↔ z.mem (embBit false (embBit false X)) := by
```

```
  constructor
  · rintro (h0 | ⟨X', hX', heq, hz⟩ | ⟨Y', hY', heq, hz⟩)
    · exact absurd h0 embF_ne_sumTokMaster
    · rwa [embBit_injective heq]
    · exact absurd heq (embBit_ne (show (false : Bool) = true by decide) (D.ne X hX))
  · intro hz; exact Or.inr (Or.inl ⟨X, hX, rfl, hz⟩)

@[simp] theorem toSumLift_mem_embT {z : (D.lift.oplus E.lift).sys.Element} {Y : Set
    Str}
    (hY : E.sys.mem Y) :
    (toSumLift z).mem (embBit true Y) ↔ z.mem (embBit true (embBit false Y)) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hz⟩ | ⟨Y', hY', heq, hz⟩)
    · exact absurd h0 embT_ne_sumTokMaster
    · exact absurd heq (embBit_ne (show (true : Bool) = false by decide) (E.ne Y hY))
    · rwa [embBit_injective heq]
  · intro hz; exact Or.inr (Or.inr ⟨Y, hY, rfl, hz⟩)

def fromSumLift (s : (D.sum E).sys.Element) : (D.lift.oplus E.lift).sys.Element where
  mem W := W = sumTokMaster D.lift.sys E.lift.sys
    ∨ (∃ X, D.sys.mem X ∧ W = embBit false (embBit false X) ∧ s.mem (embBit false X))
    ∨ (∃ Y, E.sys.mem Y ∧ W = embBit true (embBit false Y) ∧ s.mem (embBit true Y))
  sub := by
    rintro W (rfl | ⟨X, hX, rfl, -⟩ | ⟨Y, hY, rfl, -⟩)
    · exact Or.inl rfl
    · exact o_mem_embFF hX
    · exact o_mem_embTF hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hsX⟩ | ⟨Y, hY, rfl, hsY⟩)
      (rfl | ⟨X', hX', rfl, hsX'⟩ | ⟨Y', hY', rfl, hsY'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · refine Or.inr (Or.inl ⟨X', hX', ?_, hsX'⟩)
      rw [sumTokMaster_inter_embF (D₀ := D.lift.sys) (D₁ := E.lift.sys)
        (liftTok_mem_embF (hD := D.ne) hX')]
    · refine Or.inr (Or.inr ⟨Y', hY', ?_, hsY'⟩)
      rw [sumTokMaster_inter_embT (D₀ := D.lift.sys) (D₁ := E.lift.sys)
        (liftTok_mem_embF (hD := E.ne) hY')]
    · refine Or.inr (Or.inl ⟨X, hX, ?_, hsX⟩)
      rw [Set.inter_comm, sumTokMaster_inter_embF (D₀ := D.lift.sys) (D₁ := E.lift.sys)
        (liftTok_mem_embF (hD := D.ne) hX)]
    · refine Or.inr (Or.inl ⟨X ∩ X', ?_, by rw [embBit_inter, embBit_inter], ?_⟩)
      · have hs := s.inter_mem hsX hsX'; rw [embBit_inter] at hs
        exact sumTok_mem_embF_inv (h₀ := D.ne) (h₁ := E.ne) (s.sub hs)
      · have hs := s.inter_mem hsX hsX'; rwa [embBit_inter] at hs
    · exfalso
      have hs := s.inter_mem hsX hsY'
      rw [embBit_inter_ne (show (false : Bool) = true by decide)] at hs
      obtain ⟨t, ht⟩ := sumTok_mem_nonempty (h₀ := D.ne) (h₁ := E.ne) (s.sub hs)
      exact Set.notMem_empty t ht
    · refine Or.inr (Or.inr ⟨Y, hY, ?_, hsY⟩)
      rw [Set.inter_comm, sumTokMaster_inter_embT (D₀ := D.lift.sys) (D₁ := E.lift.sys)
        (liftTok_mem_embF (hD := E.ne) hY)]
    · exfalso
      have hs := s.inter_mem hsY hsX'
      rw [embBit_inter_ne (show (true : Bool) = false by decide)] at hs
```

```
      obtain ⟨t, ht⟩ := sumTok_mem_nonempty (h₀ := D.ne) (h₁ := E.ne) (s.sub hs)
      exact Set.notMem_empty t ht
    · refine Or.inr (Or.inr ⟨Y ∩ Y', ?_, by rw [embBit_inter, embBit_inter], ?_⟩)
      · have hs := s.inter_mem hsY hsY'; rw [embBit_inter] at hs
        exact sumTok_mem_embT_inv (h₀ := D.ne) (h₁ := E.ne) (s.sub hs)
      · have hs := s.inter_mem hsY hsY'; rwa [embBit_inter] at hs
  up_mem := by
    rintro W W' (rfl | ⟨X, hX, rfl, hsX⟩ | ⟨Y, hY, rfl, hsY⟩) hW' hsub
    · exact Or.inl (Set.Subset.antisymm ((D.lift.oplus E.lift).sys.sub_master hW') hsub
    )
    · rcases hW' with rfl | ⟨V, hV, hVne, rfl⟩ | ⟨V, hV, hVne, rfl⟩
      · exact Or.inl rfl
      · rcases hV with rfl | ⟨X'', hX''D, rfl⟩
        · exact absurd rfl hVne
        · refine Or.inr (Or.inl ⟨X'', hX''D, rfl, ?_⟩)
          exact s.up_mem hsX (sumTok_mem_embF (h₀ := D.ne) (h₁ := E.ne) hX''D)
            (embBit_subset.mpr (embBit_subset.mp (embBit_subset.mp hsub)))
      · exact absurd hsub
          (fun hs => embBit_not_subset_cross (show (false : Bool) = true by decide)
            (embBit_nonempty (D.ne X hX)) hs)
    · rcases hW' with rfl | ⟨V, hV, hVne, rfl⟩ | ⟨V, hV, hVne, rfl⟩
      · exact Or.inl rfl
      · exact absurd hsub
          (fun hs => embBit_not_subset_cross (show (true : Bool) = false by decide)
            (embBit_nonempty (E.ne Y hY)) hs)
      · rcases hV with rfl | ⟨Y'', hY''E, rfl⟩
        · exact absurd rfl hVne
        · refine Or.inr (Or.inr ⟨Y'', hY''E, rfl, ?_⟩)
          exact s.up_mem hsY (sumTok_mem_embT (h₀ := D.ne) (h₁ := E.ne) hY''E)
            (embBit_subset.mpr (embBit_subset.mp (embBit_subset.mp hsub)))

@[simp] theorem fromSumLift_mem_embFF {s : (D.sum E).sys.Element} {X : Set Str} (hX :
    D.sys.mem X) :
    (fromSumLift s).mem (embBit false (embBit false X)) ↔ s.mem (embBit false X) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hs⟩ | ⟨Y', hY', heq, hs⟩)
    · exact absurd h0 embF_ne_sumTokMaster
    · rwa [embBit_injective (embBit_injective heq)]
    · exact absurd heq (embBit_ne (show (false : Bool) = true by decide)
        (embBit_nonempty (D.ne X hX)))
  · intro hs; exact Or.inr (Or.inl ⟨X, hX, rfl, hs⟩)

@[simp] theorem fromSumLift_mem_embTF {s : (D.sum E).sys.Element} {Y : Set Str} (hY :
    E.sys.mem Y) :
    (fromSumLift s).mem (embBit true (embBit false Y)) ↔ s.mem (embBit true Y) := by
  constructor
  · rintro (h0 | ⟨X', hX', heq, hs⟩ | ⟨Y', hY', heq, hs⟩)
    · exact absurd h0 embT_ne_sumTokMaster
    · exact absurd heq (embBit_ne (show (true : Bool) = false by decide)
        (embBit_nonempty (E.ne Y hY)))
    · rwa [embBit_injective (embBit_injective heq)]
  · intro hs; exact Or.inr (Or.inr ⟨Y, hY, rfl, hs⟩)

theorem fromSumLift_toSumLift (z : (D.lift.oplus E.lift).sys.Element) :
    fromSumLift (toSumLift z) = z := by
  apply NeighborhoodSystem.Element.ext
```

```
  intro W
  constructor
  · rintro (rfl | ⟨X, hX, rfl, hs⟩ | ⟨Y, hY, rfl, hs⟩)
    · exact z.master_mem
    · exact (toSumLift_mem_embF hX).mp hs
    · exact (toSumLift_mem_embT hY).mp hs
  · intro hW
    rcases z.sub hW with rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩
    · exact Or.inl rfl
    · rcases hX with rfl | ⟨X', hX'D, rfl⟩
      · exact absurd rfl hXne
      · exact Or.inr (Or.inl ⟨X', hX'D, rfl, (toSumLift_mem_embF hX'D).mpr hW⟩)
    · rcases hY with rfl | ⟨Y', hY'E, rfl⟩
      · exact absurd rfl hYne
      · exact Or.inr (Or.inr ⟨Y', hY'E, rfl, (toSumLift_mem_embT hY'E).mpr hW⟩)

theorem lp_mem_embF {X' Y' : Set Str} (hX' : D.sys.mem X') (hY' : E.sys.mem Y') :


    (D.prod E).lift.sys.mem (embBit false (prodTokNbhd X' Y')) :=
  liftTok_mem_embF (hD := (D.prod E).ne) (prodTok_mem_prodTokNbhd hX' hY')

theorem lp_prod_inv {X' Y' : Set Str}
    (h : (D.prod E).lift.sys.mem (embBit false (prodTokNbhd X' Y'))) :
    D.sys.mem X' ∧ E.sys.mem Y' := by
  obtain ⟨A, B, hA, hB, heq⟩ := liftTok_mem_embF_inv (hD := (D.prod E).ne) h
  obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective heq
  exact ⟨hA, hB⟩

theorem ot_mem_prod {X' Y' : Set Str} (hX' : D.sys.mem X') (hY' : E.sys.mem Y') :
    (D.lift.otimes E.lift).sys.mem (prodTokNbhd (embBit false X') (embBit false Y'))
    :=
  otimesTok_mem_prod (liftTok_mem_embF (hD := D.ne) hX') (liftTok_mem_embF (hD := E.ne
    ) hY')
    (embF_ne_liftTokMaster (D := D.sys)) (embF_ne_liftTokMaster (D := E.sys))

theorem ot_mem_prod_inv {X' Y' : Set Str}
    (h : (D.lift.otimes E.lift).sys.mem (prodTokNbhd (embBit false X') (embBit false Y
    '))) :
    D.sys.mem X' ∧ E.sys.mem Y' := by
  obtain ⟨h1, h2⟩ := otimesTok_mem_prod_inv (D₀ := D.lift.sys) (D₁ := E.lift.sys) h
    (embF_ne_liftTokMaster (D := D.sys))
  exact ⟨liftTok_mem_embF_inv (hD := D.ne) h1, liftTok_mem_embF_inv (hD := E.ne) h2⟩

def toLiftProd (z : (D.lift.otimes E.lift).sys.Element) : (D.prod E).lift.sys.Element
    where
  mem W := W = liftTokMaster (prodTok D.sys E.sys)
    ∨ (∃ X Y, D.sys.mem X ∧ E.sys.mem Y ∧ W = embBit false (prodTokNbhd X Y) ∧
        z.mem (prodTokNbhd (embBit false X) (embBit false Y)))
  sub := by
    rintro W (rfl | ⟨X, Y, hX, hY, rfl, -⟩)
    · exact Or.inl rfl
    · exact lp_mem_embF hX hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, Y, hX, hY, rfl, hzXY⟩) (rfl | ⟨X', Y', hX', hY', rfl, hzXY'⟩)
    · exact Or.inl (by rw [Set.inter_self])
```

```
    · refine Or.inr ⟨X', Y', hX', hY', ?_, hzXY'⟩
      rw [liftTokMaster_inter_embF (prodTok_mem_prodTokNbhd hX' hY')]
    · refine Or.inr ⟨X, Y, hX, hY, ?_, hzXY⟩
      rw [Set.inter_comm, liftTokMaster_inter_embF (prodTok_mem_prodTokNbhd hX hY)]
    · have hz := z.inter_mem hzXY hzXY'
      rw [prodTokNbhd_inter, embBit_inter, embBit_inter] at hz
      obtain ⟨hXi, hYi⟩ := ot_mem_prod_inv (z.sub hz)
      refine Or.inr ⟨X ∩ X', Y ∩ Y', hXi, hYi, ?_, hz⟩
      rw [embBit_inter, prodTokNbhd_inter]
  up_mem := by
    rintro W W' (rfl | ⟨X, Y, hX, hY, rfl, hzXY⟩) hW' hsub
    · exact Or.inl (Set.Subset.antisymm ((D.prod E).lift.sys.sub_master hW') hsub)
    · rcases hW' with rfl | ⟨Z, hZ, rfl⟩
      · exact Or.inl rfl
      · obtain ⟨X'', Y'', hX'', hY'', rfl⟩ := hZ
        refine Or.inr ⟨X'', Y'', hX'', hY'', rfl, ?_⟩
        obtain ⟨hsX, hsY⟩ := prodTokNbhd_subset_iff.mp (embBit_subset.mp hsub)
        exact z.up_mem hzXY (ot_mem_prod hX'' hY'')
          (prodTokNbhd_subset_iff.mpr ⟨embBit_subset.mpr hsX, embBit_subset.mpr hsY⟩)

@[simp] theorem toLiftProd_mem_embF {z : (D.lift.otimes E.lift).sys.Element} {X Y :
    Set Str}
    (hX : D.sys.mem X) (hY : E.sys.mem Y) :
    (toLiftProd z).mem (embBit false (prodTokNbhd X Y)) ↔
      z.mem (prodTokNbhd (embBit false X) (embBit false Y)) := by
  constructor
  · rintro (h0 | ⟨X', Y', hX', hY', heq, hz⟩)
    · exact absurd h0 (embF_ne_liftTokMaster (D := prodTok D.sys E.sys))
    · obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective (embBit_injective heq); exact hz
  · intro hz; exact Or.inr ⟨X, Y, hX, hY, rfl, hz⟩

def fromLiftProd (s : (D.prod E).lift.sys.Element) : (D.lift.otimes E.lift).sys.
    Element where
  mem W := W = prodTokNbhd (liftTokMaster D.sys) (liftTokMaster E.sys)
    ∨ (∃ X Y, D.sys.mem X ∧ E.sys.mem Y ∧ W = prodTokNbhd (embBit false X) (embBit
    false Y) ∧
        s.mem (embBit false (prodTokNbhd X Y)))
  sub := by
    rintro W (rfl | ⟨X, Y, hX, hY, rfl, -⟩)
    · exact Or.inl rfl
    · exact ot_mem_prod hX hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | ⟨X, Y, hX, hY, rfl, hsXY⟩) (rfl | ⟨X', Y', hX', hY', rfl, hsXY'⟩)
    · exact Or.inl (by rw [Set.inter_self])
    · refine Or.inr ⟨X', Y', hX', hY', ?_, hsXY'⟩
      rw [prodTokNbhd_inter, liftTokMaster_inter_embF hX', liftTokMaster_inter_embF hY
    ']
    · refine Or.inr ⟨X, Y, hX, hY, ?_, hsXY⟩
      rw [Set.inter_comm, prodTokNbhd_inter, liftTokMaster_inter_embF hX,
        liftTokMaster_inter_embF hY]
    · have hs := s.inter_mem hsXY hsXY'
      rw [embBit_inter, prodTokNbhd_inter] at hs
      obtain ⟨hXi, hYi⟩ := lp_prod_inv (s.sub hs)
      refine Or.inr ⟨X ∩ X', Y ∩ Y', hXi, hYi, ?_, hs⟩
      rw [prodTokNbhd_inter, embBit_inter, embBit_inter]
```

```
  up_mem := by
    rintro W W' (rfl | ⟨X, Y, hX, hY, rfl, hsXY⟩) hW' hsub
    · exact Or.inl (Set.Subset.antisymm ((D.lift.otimes E.lift).sys.sub_master hW')
    hsub)
    · rcases hW' with rfl | ⟨U, V, hU, hV, hUne, hVne, rfl⟩
      · exact Or.inl rfl
      · rcases hU with rfl | ⟨X'', hX''D, rfl⟩
        · exact absurd rfl hUne
        · rcases hV with rfl | ⟨Y'', hY''E, rfl⟩
          · exact absurd rfl hVne
          · refine Or.inr ⟨X'', Y'', hX''D, hY''E, rfl, ?_⟩
            obtain ⟨hsX, hsY⟩ := prodTokNbhd_subset_iff.mp hsub
            exact s.up_mem hsXY (lp_mem_embF hX''D hY''E)
              (embBit_subset.mpr (prodTokNbhd_subset_iff.mpr
                ⟨embBit_subset.mp hsX, embBit_subset.mp hsY⟩))

@[simp] theorem fromLiftProd_mem_prod {s : (D.prod E).lift.sys.Element} {X Y : Set Str
    }
    (hX : D.sys.mem X) (hY : E.sys.mem Y) :
    (fromLiftProd s).mem (prodTokNbhd (embBit false X) (embBit false Y)) ↔
      s.mem (embBit false (prodTokNbhd X Y)) := by
  constructor
  · rintro (h0 | ⟨X', Y', hX', hY', heq, hs⟩)
    · obtain ⟨hX0, -⟩ := prodTokNbhd_injective h0
      exact absurd hX0 (embF_ne_liftTokMaster (D := D.sys))
    · obtain ⟨hXe, hYe⟩ := prodTokNbhd_injective heq
      rw [embBit_injective hXe, embBit_injective hYe]; exact hs
  · intro hs; exact Or.inr ⟨X, Y, hX, hY, rfl, hs⟩

theorem fromLiftProd_toLiftProd (z : (D.lift.otimes E.lift).sys.Element) :
    fromLiftProd (toLiftProd z) = z := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨X, Y, hX, hY, rfl, hs⟩)
    · exact z.master_mem
    · exact (toLiftProd_mem_embF hX hY).mp hs
  · intro hW
    rcases z.sub hW with rfl | ⟨X, Y, hX, hY, hXne, hYne, rfl⟩
    · exact Or.inl rfl
    · rcases hX with rfl | ⟨X'', hX''D, rfl⟩
      · exact absurd rfl hXne
      · rcases hY with rfl | ⟨Y'', hY''E, rfl⟩
        · exact absurd rfl hYne
        · exact Or.inr ⟨X'', Y'', hX''D, hY''E, rfl, (toLiftProd_mem_embF hX''D hY''E).
    mpr hW⟩

theorem toLiftProd_fromLiftProd (s : (D.prod E).lift.sys.Element) :
    toLiftProd (fromLiftProd s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨X, Y, hX, hY, rfl, hs⟩)
    · exact s.master_mem
    · exact (fromLiftProd_mem_prod hX hY).mp hs
  · intro hW
```

```
    rcases s.sub hW with rfl | ⟨Z, hZ, rfl⟩
    · exact Or.inl rfl
    · obtain ⟨X, Y, hX, hY, rfl⟩ := hZ
      exact Or.inr ⟨X, Y, hX, hY, rfl, (fromLiftProd_mem_prod hX hY).mpr hW⟩

def liftProdEquiv : (D.lift.otimes E.lift).sys.Element ≃o (D.prod E).lift.sys.Element
    where
  toFun := toLiftProd
  invFun := fromLiftProd
  left_inv := fromLiftProd_toLiftProd
  right_inv := toLiftProd_fromLiftProd
  map_rel_iff' := by
    intro z z'
    constructor
    · intro h W hW
      rcases z.sub hW with rfl | ⟨X, Y, hX, hY, hXne, hYne, rfl⟩
      · exact z'.master_mem
      · rcases hX with rfl | ⟨X'', hX''D, rfl⟩
        · exact absurd rfl hXne
        · rcases hY with rfl | ⟨Y'', hY''E, rfl⟩
          · exact absurd rfl hYne
          · exact (toLiftProd_mem_embF hX''D hY''E).mp
              (h _ ((toLiftProd_mem_embF hX''D hY''E).mpr hW))
    · intro h W hW
      rcases hW with rfl | ⟨X, Y, hX, hY, rfl, hzXY⟩
      · exact Or.inl rfl
      · exact Or.inr ⟨X, Y, hX, hY, rfl, h _ hzXY⟩

theorem toSumLift_fromSumLift (s : (D.sum E).sys.Element) :
    toSumLift (fromSumLift s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨X, hX, rfl, hs⟩ | ⟨Y, hY, rfl, hs⟩)
    · exact s.master_mem
    · exact (fromSumLift_mem_embFF hX).mp hs
    · exact (fromSumLift_mem_embTF hY).mp hs
  · intro hW
    rcases s.sub hW with rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr (Or.inl ⟨X, hX, rfl, (fromSumLift_mem_embFF hX).mpr hW⟩)
    · exact Or.inr (Or.inr ⟨Y, hY, rfl, (fromSumLift_mem_embTF hY).mpr hW⟩)

def sumLiftEquiv : (D.lift.oplus E.lift).sys.Element ≃o (D.sum E).sys.Element where
  toFun := toSumLift
  invFun := fromSumLift
  left_inv := fromSumLift_toSumLift
  right_inv := toSumLift_fromSumLift
  map_rel_iff' := by
    intro z z'
    constructor
    · intro h W hW
      rcases z.sub hW with rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩
      · exact z'.master_mem
      · rcases hX with rfl | ⟨X', hX'D, rfl⟩
        · exact absurd rfl hXne
```

```
        · exact (toSumLift_mem_embF hX'D).mp (h _ ((toSumLift_mem_embF hX'D).mpr hW))
      · rcases hY with rfl | ⟨Y', hY'E, rfl⟩
        · exact absurd rfl hYne
        · exact (toSumLift_mem_embT hY'E).mp (h _ ((toSumLift_mem_embT hY'E).mpr hW))
    · intro h W hW
      rcases hW with rfl | ⟨X, hX, rfl, hzX⟩ | ⟨Y, hY, rfl, hzY⟩
      · exact Or.inl rfl
      · exact Or.inr (Or.inl ⟨X, hX, rfl, h _ hzX⟩)
      · exact Or.inr (Or.inr ⟨Y, hY, rfl, h _ hzY⟩)

end FunctorTokens
end

section
namespace FunctorTokens

variable {D : NeighborhoodSystem Str}

variable {hD : ∀ X, D.mem X → X.Nonempty}

def unlift (z : (liftTok D hD).Element) (hz : z.mem (embBit false D.master)) : D.
    Element where
  mem X := z.mem (embBit false X)
  sub := fun hX => liftTok_mem_embF_inv (z.sub hX)
  master_mem := hz
  inter_mem := by
    intro X X' hX hX'
    have hz' := z.inter_mem hX hX'
    rwa [embBit_inter] at hz'
  up_mem := by
    intro X Y hX hY hXY
    exact z.up_mem hX (liftTok_mem_embF hY) (embBit_subset.mpr hXY)

theorem liftBot_le (z : (liftTok D hD).Element) : liftBot D hD ≤ z := by
  rintro W rfl; exact z.master_mem

theorem liftBot_lt_liftUp (x : D.Element) : liftBot D hD < liftUp (hD := hD) x := by
  refine lt_of_le_of_ne (liftBot_le _) (fun heq => ?_)
  have hmem : (liftBot D hD).mem (embBit false D.master) := by
    rw [heq]; exact Or.inr ⟨D.master, x.master_mem, rfl⟩
  exact embF_ne_liftTokMaster hmem

end FunctorTokens
end

section
namespace FunctorTokens

variable {D : NeighborhoodSystem Str}

variable {hD : ∀ X, D.mem X → X.Nonempty}

variable {A B C : ScottSys}

theorem liftMapTok_comp (f : ApproximableMap A.sys B.sys) (g : ApproximableMap B.sys C
    .sys) :
```

```
    liftMapTok (g.comp f) = (liftMapTok g).comp (liftMapTok f) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, X'', ⟨X', hf, hg⟩, rfl, rfl⟩)
    · exact ⟨liftTokMaster B.sys, Or.inl ⟨hW, rfl⟩,
        Or.inl ⟨(ScottSys.lift B).sys.master_mem, rfl⟩⟩
    · exact ⟨embBit false X', Or.inr ⟨X, X', hf, rfl, rfl⟩, Or.inr ⟨X', X'', hg, rfl,
    rfl⟩⟩
  · rintro ⟨W', hWW', hW'W''⟩
    rcases hWW' with ⟨hW, rfl⟩ | ⟨X, X', hf, rfl, rfl⟩
    · rcases hW'W'' with ⟨-, rfl⟩ | ⟨X, X', -, heq, -⟩
      · exact Or.inl ⟨hW, rfl⟩
      · exact absurd (heq ▸ nil_mem_liftTokMaster) nil_not_mem_embBit
    · rcases hW'W'' with ⟨-, rfl⟩ | ⟨X₂, X'', hg, heq, rfl⟩
      · exact Or.inl ⟨liftTok_mem_embF (hD := A.ne) (f.rel_dom hf), rfl⟩
      · obtain rfl := embBit_injective heq
        exact Or.inr ⟨X, X'', ⟨X', hf, hg⟩, rfl, rfl⟩

theorem liftMapTok_id : liftMapTok (idMap A.sys) = idMap (ScottSys.lift A).sys := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  · rintro (⟨hW, rfl⟩ | ⟨X, X', ⟨hX, hX', hsub⟩, rfl, rfl⟩)
    · exact ⟨hW, (ScottSys.lift A).sys.master_mem, (ScottSys.lift A).sys.sub_master hW⟩
    · exact ⟨liftTok_mem_embF (hD := A.ne) hX, liftTok_mem_embF (hD := A.ne) hX',
        embBit_subset.mpr hsub⟩
  · rintro ⟨hW, hW', hsub⟩
    rcases hW' with rfl | ⟨X', hX', rfl⟩
    · exact Or.inl ⟨hW, rfl⟩
    · rcases hW with rfl | ⟨X, hX, rfl⟩
      · exact absurd (hsub nil_mem_liftTokMaster) nil_not_mem_embBit
      · exact Or.inr ⟨X, X', ⟨hX, hX', embBit_subset.mp hsub⟩, rfl, rfl⟩

theorem liftMapTok_isStrict (f : ApproximableMap A.sys B.sys) : IsStrict (liftMapTok f
    ) := by
  rintro Y (⟨-, rfl⟩ | ⟨X, X', -, heq, -⟩)
  · rfl
  · have hnil : ([] : Str) ∈ embBit false X := by
      rw [← heq]; exact nil_mem_liftTokMaster
    exact absurd hnil nil_not_mem_embBit

end FunctorTokens
end

section
namespace FunctorTokens

variable {D : NeighborhoodSystem Str}

variable {hD : ∀ X, D.mem X → X.Nonempty}

theorem liftTok_mem_master : (liftTok D hD).mem (liftTokMaster D) := Or.inl rfl

theorem liftUp_le_liftUp_iff {x y : D.Element} :
    liftUp (hD := hD) x ≤ liftUp (hD := hD) y ↔ x ≤ y := by
```

```
  constructor
  · intro h X hX
    have hmem := h (embBit false X) (Or.inr ⟨X, hX, rfl⟩)
    rcases hmem with h0 | ⟨X', hX', heq⟩
    · exact absurd (h0.symm ▸ nil_mem_liftTokMaster) nil_not_mem_embBit
    · rw [embBit_injective heq]; exact hX'
  · rintro h W (rfl | ⟨X, hX, rfl⟩)
    · exact Or.inl rfl
    · exact Or.inr ⟨X, h X hX, rfl⟩

theorem liftUp_unlift (z : (liftTok D hD).Element) (hz : z.mem (embBit false D.master)
    ) :
    liftUp (hD := hD) (unlift z hz) = z := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (rfl | ⟨X, hX, rfl⟩)
    · exact z.master_mem
    · exact hX
  · intro hW
    rcases z.sub hW with rfl | ⟨X, hX, rfl⟩
    · exact Or.inl rfl
    · exact Or.inr ⟨X, hW, rfl⟩

theorem eq_liftBot_or_exists_liftUp (z : (liftTok D hD).Element) :
    z = liftBot D hD ∨ ∃ x : D.Element, z = liftUp (hD := hD) x := by
  by_cases hz : z.mem (embBit false D.master)
  · exact Or.inr ⟨unlift z hz, (liftUp_unlift z hz).symm⟩
  · refine Or.inl ?_
    apply NeighborhoodSystem.Element.ext
    intro W
    constructor
    · intro hW
      rcases z.sub hW with rfl | ⟨X, hX, rfl⟩
      · rfl
      · exact absurd
          (z.up_mem hW (liftTok_mem_embF D.master_mem) (embBit_subset.mpr (D.
    sub_master hX))) hz
    · rintro rfl; exact z.master_mem

end FunctorTokens
end

section
namespace FunctorTokens

variable {D : NeighborhoodSystem Str}

variable {hD : ∀ X, D.mem X → X.Nonempty}

variable {A B C : ScottSys}

variable {D E : ScottSys}

theorem lift_oplus_lift_iso_sum :
    (D.lift.oplus E.lift).sys ≅? (D.sum E).sys := ⟨sumLiftEquiv⟩
```

```
theorem lift_otimes_lift_iso_lift_prod :
    (D.lift.otimes E.lift).sys ≅? (D.prod E).lift.sys := ⟨liftProdEquiv⟩

end FunctorTokens
end

end DomainTheory.NeighbourhoodSystems
```

4.7.58 Exercise 6.26 — lifting a domain

NeighbourhoodSystems/DomainEquations/Exercise_6_26.lean

`Exercise_6_26.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.liftDomain

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem BitDom
open DomainTheory.NeighbourhoodSystems.FunctorTokens
theorem example_626 :
    (∀ {D : NeighborhoodSystem Str} {hD : ∀ X, D.mem X → X.Nonempty} (z : (liftTok D
    hD).Element),
        z = liftBot D hD ∨ ∃ x : D.Element, z = liftUp (hD := hD) x) ∧
    (∀ {D E : ScottSys}, (D.lift.oplus E.lift).sys ≅? (D.sum E).sys) ∧
    (∀ {D E : ScottSys}, (D.lift.otimes E.lift).sys ≅? (D.prod E).lift.sys) :=
  ⟨@eq_liftBot_or_exists_liftUp, @lift_oplus_lift_iso_sum,
    @lift_otimes_lift_iso_lift_prod⟩
```

EXERCISE 6.26. For systems $\mathcal{D}$ as in 6.19 define

$$\mathcal{D}_\perp = \{\{\Lambda\} \cup 0\Delta\} \cup \{0X \mid X \in \mathcal{D}\}.$$

Describe the construct in terms of elements. Is this a suitable functor? Prove that

$$\mathcal{D}_\perp \oplus \mathcal{E}_\perp \cong \mathcal{D} + \mathcal{E}.$$

What is

$$\mathcal{D}_\perp \otimes \mathcal{E}_\perp \cong ??$$

4.7.59 embeddingCompare — which embeddings hold

`embeddingCompare.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupMap
```

```
public import DomainTheory.NeighbourhoodSystems.DomainEquations.liftDomain

public import DomainTheory.NeighbourhoodSystems.DomainEquations.initialHom
public import DomainTheory.NeighbourhoodSystems.DomainEquations.continuousOnDomains
public import DomainTheory.NeighbourhoodSystems.DomainEquations.subdomain
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.
    FunctorTokens
open DomainTheory.NeighbourhoodSystems.SeqEqB DomainTheory.NeighbourhoodSystems.BitDom
open DomainTheory.NeighbourhoodSystems.SmashProduct

section
namespace EmbeddingCompare

variable {D₀ D₁ : NeighborhoodSystem Str}

def fstInj (D₀ D₁ : NeighborhoodSystem Str) : ApproximableMap D₀ (prodTok D₀ D₁) where
  rel X W := D₀.mem X ∧ (prodTok D₀ D₁).mem W ∧ prodTokNbhd X D₁.master ⊆ W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨D₀.master_mem, (prodTok D₀ D₁).master_mem, subset_rfl⟩
  inter_right := by
    rintro X W W' ⟨hX, hW, hsub⟩ ⟨_, hW', hsub'⟩
    exact ⟨hX, (prodTok D₀ D₁).inter_mem hW hW' (prodTok_mem_prodTokNbhd hX D₁.
    master_mem)
      (Set.subset_inter hsub hsub'), Set.subset_inter hsub hsub'⟩
  mono := by
    rintro X X' W W' ⟨_, _, hsub⟩ hX'X hWW' hX' hW'
    exact ⟨hX', hW', ((prodTokNbhd_subset_iff.mpr ⟨hX'X, subset_rfl⟩).trans hsub).trans
     hWW'⟩

def fstProj (D₀ D₁ : NeighborhoodSystem Str) : ApproximableMap (prodTok D₀ D₁) D₀ where
  rel W X := (prodTok D₀ D₁).mem W ∧ D₀.mem X ∧ W ⊆ prodTokNbhd X D₁.master
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(prodTok D₀ D₁).master_mem, D₀.master_mem, subset_rfl⟩
  inter_right := by
    rintro W X X' ⟨hW, hX, hsub⟩ ⟨_, hX', hsub'⟩
    obtain ⟨A, B, hA, hB, rfl⟩ := hW
    have hAX : A ⊆ X := (prodTokNbhd_subset_iff.mp hsub).1
    have hAX' : A ⊆ X' := (prodTokNbhd_subset_iff.mp hsub').1
    exact ⟨prodTok_mem_prodTokNbhd hA hB, D₀.inter_mem hX hX' hA (Set.subset_inter hAX
    hAX'),
      prodTokNbhd_subset_iff.mpr ⟨Set.subset_inter hAX hAX', D₁.sub_master hB⟩⟩
  mono := by
    rintro W W' X X' ⟨_, _, hsub⟩ hW'W hXX' hW' hX'
    exact ⟨hW', hX', hW'W.trans (hsub.trans (prodTokNbhd_subset_iff.mpr ⟨hXX',
    subset_rfl⟩))⟩
```

```
end EmbeddingCompare
end

section
namespace EmbeddingCompare

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

section Part5

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def incl (φ : (strictFun V₀ V₁).Element) : (funSpace V₀ V₁).Element :=
  toFilter (toStrictMap φ).1

theorem incl_mono : Monotone (incl (V₀ := V₀) (V₁ := V₁)) := by
  intro a b hab M hM
  obtain ⟨hMmem, haM⟩ := hM
  obtain ⟨⟨L, hL, rfl⟩, hne⟩ := hMmem
  refine ⟨⟨⟨L, hL, rfl⟩, hne⟩, ?_⟩
  intro p hp
  have hle : (toStrictMap a).1 ≤ (toStrictMap b).1 := (strictFunEquiv V₀ V₁).monotone
    hab
  exact hle p.1 p.2 (haM p hp)

def inclMap : ApproximableMap (strictFun V₀ V₁) (funSpace V₀ V₁) :=
  ofMono (fun _ hW => incl ((strictFun V₀ V₁).principal hW))
    (fun _ _ hW hW' hW'W => incl_mono (((strictFun V₀ V₁).principal_le_iff hW hW').mpr
    hW'W))

end Part5

def leftN (D₀ D₁ : NeighborhoodSystem Str) (X : Set Str) : Set Str :=
  embBit false X ∪ {w | w ∈ sumTokMaster D₀ D₁ ∧ X = D₀.master}

theorem X_eq_master_of_embT_subset_leftN {B X : Set Str} (hB : B.Nonempty)
    (h : embBit true B ⊆ leftN D₀ D₁ X) : X = D₀.master := by
  obtain ⟨σ, hσ⟩ := hB
  rcases h ⟨σ, rfl, hσ⟩ with ⟨w', hw', -⟩ | ⟨-, hXm⟩
  · rw [List.cons.injEq] at hw'; exact absurd hw'.1 (by decide)
  · exact hXm

theorem X_eq_master_of_nil_mem_leftN {X : Set Str} (h : ([] : Str) ∈ leftN D₀ D₁ X) :
    X = D₀.master := by
  rcases h with h | ⟨-, h⟩
  · exact absurd h nil_not_mem_embBit
  · exact h

theorem embF_subset_leftN {X : Set Str} : embBit false X ⊆ leftN D₀ D₁ X :=
  Set.subset_union_left

theorem leftN_master : leftN D₀ D₁ D₀.master = sumTokMaster D₀ D₁ := by
  apply Set.eq_of_subset_of_subset
  · rintro w (hw | ⟨hw, -⟩)
```

```
    · exact embF_subset_sumTokMaster D₀.master_mem hw
    · exact hw
  · exact fun w hw => Or.inr ⟨hw, rfl⟩

theorem leftN_proper {X : Set Str} (hne : X = D₀.master) : leftN D₀ D₁ X = embBit false
    X := by
  apply Set.eq_of_subset_of_subset
  · rintro w (hw | ⟨-, hXm⟩)
    · exact hw
    · exact absurd hXm hne
  · exact embF_subset_leftN

theorem oplus_mem_leftN {X : Set Str} (hX : D₀.mem X) :
    (oplusTok D₀ D₁ h₀ h₁).mem (leftN D₀ D₁ X) := by
  by_cases h : X = D₀.master
  · rw [h, leftN_master]; exact (oplusTok D₀ D₁ h₀ h₁).master_mem
  · rw [leftN_proper h]; exact oplusTok_mem_embF hX h

section Part5

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def strictifyMap (g : ApproximableMap V₀ V₁) : ApproximableMap V₀ V₁ where
  rel X Y := g.rel X Y ∧ (X = V₀.master → Y = V₁.master)
  rel_dom h := g.rel_dom h.1
  rel_cod h := g.rel_cod h.1
  master_rel := ⟨g.master_rel, fun _ => rfl⟩
  inter_right := by
    rintro X Y Y' ⟨hg, hs⟩ ⟨hg', hs'⟩
    exact ⟨g.inter_right hg hg', fun hX => by rw [hs hX, hs' hX, Set.inter_self]⟩
  mono := by
    rintro X X' Y Y' ⟨hg, hs⟩ hX'X hYY' hX' hY'
    refine ⟨g.mono hg hX'X hYY' hX' hY', fun hX'm => ?_⟩
    have hX : V₀.mem X := g.rel_dom hg
    have hXm : X = V₀.master := Set.Subset.antisymm (V₀.sub_master hX) (hX'm ▸ hX'X)
    exact Set.Subset.antisymm (V₁.sub_master hY') ((hs hXm) ▸ hYY')

theorem strictifyMap_isStrict (g : ApproximableMap V₀ V₁) : IsStrict (strictifyMap g)
    :=
  fun _ h => h.2 rfl

def strictify (g : ApproximableMap V₀ V₁) : StrictMap V₀ V₁ := ⟨strictifyMap g,
    strictifyMap_isStrict g⟩

def strct (ψ : (funSpace V₀ V₁).Element) : (strictFun V₀ V₁).Element :=
  toStrictFilter (strictify (toApproxMap ψ))

theorem strct_mono : Monotone (strct (V₀ := V₀) (V₁ := V₁)) := by
  intro a b hab W hW
  obtain ⟨hWmem, haW⟩ := hW
  obtain ⟨⟨L, hL, rfl⟩, hne⟩ := hWmem
  refine ⟨⟨⟨L, hL, rfl⟩, hne⟩, ?_⟩
  intro p hp
  have hle : toApproxMap a ≤ toApproxMap b := (funSpaceEquiv V₀ V₁).monotone hab
  exact ⟨hle p.1 p.2 (haW p hp).1, (haW p hp).2⟩
```

```
def strctMap : ApproximableMap (funSpace V₀ V₁) (strictFun V₀ V₁) :=
  ofMono (fun _ hM => strct ((funSpace V₀ V₁).principal hM))
    (fun _ _ hM hM' hM'M => strct_mono (((funSpace V₀ V₁).principal_le_iff hM hM').mpr
    hM'M))

end Part5

theorem subset_of_embF_subset_leftN {A X : Set Str} (hA : A ⊆ D₀.master)
    (h : embBit false A ⊆ leftN D₀ D₁ X) : A ⊆ X := by
  intro σ hσ
  rcases h ⟨σ, rfl, hσ⟩ with ⟨w', hw', hw'X⟩ | ⟨-, hXm⟩
  · rw [List.cons.injEq] at hw'; rw [hw'.2]; exact hw'X
  · rw [hXm]; exact hA hσ

theorem leftN_subset_iff {X X' : Set Str} (hX : D₀.mem X) (hX' : D₀.mem X') :
    leftN D₀ D₁ X ⊆ leftN D₀ D₁ X' ↔ X ⊆ X' := by
  constructor
  · intro h
    exact subset_of_embF_subset_leftN (D₀.sub_master hX) (embF_subset_leftN.trans h)
  · intro hXX'
    rintro w (⟨w', rfl, hw'⟩ | ⟨hw, hXm⟩)
    · exact embF_subset_leftN ⟨w', rfl, hXX' hw'⟩
    · exact Or.inr ⟨hw, Set.Subset.antisymm (D₀.sub_master hX') (hXm ▶ hXX')⟩

def inlInj : ApproximableMap D₀ (oplusTok D₀ D₁ h₀ h₁) where
  rel X W := D₀.mem X ∧ (oplusTok D₀ D₁ h₀ h₁).mem W ∧ leftN D₀ D₁ X ⊆ W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨D₀.master_mem, (oplusTok D₀ D₁ h₀ h₁).master_mem, leftN_master.le⟩
  inter_right := by
    rintro X W W' ⟨hX, hW, hsub⟩ ⟨_, hW', hsub'⟩
    exact ⟨hX, (oplusTok D₀ D₁ h₀ h₁).inter_mem hW hW' (oplus_mem_leftN hX)
      (Set.subset_inter hsub hsub'), Set.subset_inter hsub hsub'⟩
  mono := by
    rintro X X' W W' ⟨hX, _, hsub⟩ hX'X hWW' hX' hW'
    exact ⟨hX', hW', ((leftN_subset_iff hX' hX).mpr hX'X).trans (hsub.trans hWW')⟩

def inlProj : ApproximableMap (oplusTok D₀ D₁ h₀ h₁) D₀ where
  rel W X := (oplusTok D₀ D₁ h₀ h₁).mem W ∧ D₀.mem X ∧ W ⊆ leftN D₀ D₁ X
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(oplusTok D₀ D₁ h₀ h₁).master_mem, D₀.master_mem, leftN_master.ge⟩
  inter_right := by
    rintro W X X' ⟨hW, hX, hsub⟩ ⟨_, hX', hsub'⟩
    rcases hW with rfl | ⟨A, hA, hAne, rfl⟩ | ⟨B, hB, hBne, rfl⟩
    · have hXm := X_eq_master_of_nil_mem_leftN (hsub nil_mem_sumTokMaster)
      have hX'm := X_eq_master_of_nil_mem_leftN (hsub' nil_mem_sumTokMaster)
      subst hXm; subst hX'm; rw [Set.inter_self]
      exact ⟨Or.inl rfl, D₀.master_mem, leftN_master.ge⟩
    · have hAX : A ⊆ X := subset_of_embF_subset_leftN (D₀.sub_master hA) hsub
      have hAX' : A ⊆ X' := subset_of_embF_subset_leftN (D₀.sub_master hA) hsub'
      exact ⟨oplusTok_mem_embF hA hAne, D₀.inter_mem hX hX' hA (Set.subset_inter hAX
    hAX'),
        (embBit_subset.mpr (Set.subset_inter hAX hAX')).trans embF_subset_leftN⟩
    · have hXm := X_eq_master_of_embT_subset_leftN (h₁ B hB) hsub
      have hX'm := X_eq_master_of_embT_subset_leftN (h₁ B hB) hsub'
```

```
      subst hXm; subst hX'm; rw [Set.inter_self]
      exact ⟨oplusTok_mem_embT hB hBne, D₀.master_mem, by
        rw [leftN_master]; exact embT_subset_sumTokMaster hB⟩
  mono := by
    rintro W W' X X' ⟨_, hX, hsub⟩ hW'W hXX' hW' hX'
    exact ⟨hW', hX', hW'W.trans (hsub.trans ((leftN_subset_iff hX hX').mpr hXX'))⟩

def twoPt : NeighborhoodSystem Str where
  mem X := X = Set.univ ∨ X = {[]}
  master := Set.univ
  master_nonempty := ⟨[], Set.mem_univ _⟩
  master_mem := Or.inl rfl
  sub_master := by
    rintro X (rfl | rfl)
    · exact subset_rfl
    · exact Set.subset_univ _
  inter_mem := by
    rintro X Y Z (rfl | rfl) (rfl | rfl) _ _
    · rw [Set.inter_self]; exact Or.inl rfl
    · rw [Set.univ_inter]; exact Or.inr rfl
    · rw [Set.inter_univ]; exact Or.inr rfl
    · rw [Set.inter_self]; exact Or.inr rfl

def unitPt : NeighborhoodSystem Str := (singletonSys ({[]} : Set Str) ⟨[], rfl⟩).sys

theorem inlProj_comp_inlInj :
    (inlProj (D₀ := D₀) (D₁ := D₁) (h₀ := h₀) (h₁ := h₁)).comp inlInj = idMap D₀ := by
  apply ApproximableMap.ext
  intro X Z
  rw [comp_rel, idMap_rel]
  constructor
  · rintro ⟨W, ⟨hX, _, hsub⟩, _, hZ, hsub'⟩
    exact ⟨hX, hZ, (leftN_subset_iff hX hZ).mp (hsub.trans hsub')⟩
  · rintro ⟨hX, hZ, hXZ⟩
    exact ⟨leftN D₀ D₁ X, ⟨hX, oplus_mem_leftN hX, subset_rfl⟩,
      oplus_mem_leftN hX, hZ, (leftN_subset_iff hX hZ).mpr hXZ⟩

theorem inlInj_comp_inlProj_le :
    (inlInj (D₀ := D₀) (D₁ := D₁) (h₀ := h₀) (h₁ := h₁)).comp inlProj
      ≤ idMap (oplusTok D₀ D₁ h₀ h₁) := by
  intro W W' hr
  obtain ⟨X, ⟨hW, _, hsub⟩, _, hW', hsub'⟩ := hr
  exact ⟨hW, hW', hsub.trans hsub'⟩

end EmbeddingCompare
end
section
namespace EmbeddingCompare
variable {D₀ D₁ : NeighborhoodSystem Str}
variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

theorem inl_trianglelefteq_oplus :
    D₀ ? oplusTok D₀ D₁ h₀ h₁ :=
  trianglelefteq_of_projectionPair inlInj inlProj inlProj_comp_inlInj
    inlInj_comp_inlProj_le
```

```
section Part5

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

theorem ext_of_principal {? ? : Type*} {W₀ : NeighborhoodSystem ?} {W₁ :
    NeighborhoodSystem ?}
    {f g : ApproximableMap W₀ W₁}
    (h : ∀ (X : Set ?) (hX : W₀.mem X),
      f.toElementMap (W₀.principal hX) = g.toElementMap (W₀.principal hX)) : f = g :=
    by
  apply ApproximableMap.ext
  intro X Y
  constructor
  · intro hr
    have hX := f.rel_dom hr
    rw [g.rel_iff_mem_principal hX, ← h X hX, ← f.rel_iff_mem_principal hX]; exact hr
  · intro hr
    have hX := g.rel_dom hr
    rw [f.rel_iff_mem_principal hX, h X hX, ← g.rel_iff_mem_principal hX]; exact hr

end Part5

end EmbeddingCompare
end

section
namespace EmbeddingCompare

variable {D₀ D₁ : NeighborhoodSystem Str}

theorem fstInj_comp_fstProj_le (D₀ D₁ : NeighborhoodSystem Str) :
    (fstInj D₀ D₁).comp (fstProj D₀ D₁) ≤ idMap (prodTok D₀ D₁) := by
  intro W W' hr
  obtain ⟨X, ⟨hW, _, hsub⟩, _, hW', hsub'⟩ := hr
  exact ⟨hW, hW', hsub.trans hsub'⟩

theorem fstProj_comp_fstInj (D₀ D₁ : NeighborhoodSystem Str) :
    (fstProj D₀ D₁).comp (fstInj D₀ D₁) = idMap D₀ := by
  apply ApproximableMap.ext
  intro X Z
  rw [comp_rel, idMap_rel]
  constructor
  · rintro ⟨W, ⟨hX, _, hsub⟩, _, hZ, hsub'⟩
    exact ⟨hX, hZ, (prodTokNbhd_subset_iff.mp (hsub.trans hsub')).1⟩
  · rintro ⟨hX, hZ, hXZ⟩
    exact ⟨prodTokNbhd Z D₁.master,
      ⟨hX, prodTok_mem_prodTokNbhd hZ D₁.master_mem,
        prodTokNbhd_subset_iff.mpr ⟨hXZ, subset_rfl⟩⟩,
      prodTok_mem_prodTokNbhd hZ D₁.master_mem, hZ, subset_rfl⟩

theorem fst_trianglelefteq_prod (D₀ D₁ : NeighborhoodSystem Str) :
    D₀ ◁ prodTok D₀ D₁ :=
  trianglelefteq_of_projectionPair (fstInj D₀ D₁) (fstProj D₀ D₁)
    (fstProj_comp_fstInj D₀ D₁) (fstInj_comp_fstProj_le D₀ D₁)

end EmbeddingCompare
```

```
end

section
namespace EmbeddingCompare

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

theorem oplusTok_subsystem_sumTok (h₀ : ∀ X, D₀.mem X → X.Nonempty)
    (h₁ : ∀ Y, D₁.mem Y → Y.Nonempty) :
    oplusTok D₀ D₁ h₀ h₁ ? sumTok D₀ D₁ h₀ h₁ where
  master_eq := rfl
  sub := by
    rintro W (rfl | ⟨X, hX, -, rfl⟩ | ⟨Y, hY, -, rfl⟩)
    · exact Or.inl rfl
    · exact sumTok_mem_embF hX
    · exact sumTok_mem_embT hY
  inter_closed := by
    rintro W W' (rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩)
      (rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩) hInt
    · rw [Set.inter_self]; exact Or.inl rfl
    · rw [sumTokMaster_inter_embF hX']; exact oplusTok_mem_embF hX' hX'ne
    · rw [sumTokMaster_inter_embT hY']; exact oplusTok_mem_embT hY' hY'ne
    · rw [Set.inter_comm, sumTokMaster_inter_embF hX]; exact oplusTok_mem_embF hX hXne
    · rw [embBit_inter] at hInt ⊢
      exact oplusTok_mem_embF (sumTok_mem_embF_inv hInt)
        (inter_ne_of_ne_left (D₀.sub_master hX) hXne)
    · rw [embBit_inter_ne (show (false : Bool) = true by decide)] at hInt
      exact absurd (sumTok_mem_nonempty hInt) Set.not_nonempty_empty
    · rw [Set.inter_comm, sumTokMaster_inter_embT hY]; exact oplusTok_mem_embT hY hYne
    · rw [embBit_inter_ne (show (true : Bool) = false by decide)] at hInt
      exact absurd (sumTok_mem_nonempty hInt) Set.not_nonempty_empty
    · rw [embBit_inter] at hInt ⊢
      exact oplusTok_mem_embT (sumTok_mem_embT_inv hInt)
        (inter_ne_of_ne_left (D₁.sub_master hY) hYne)

theorem oplus_trianglelefteq_sum (h₀ : ∀ X, D₀.mem X → X.Nonempty)
    (h₁ : ∀ Y, D₁.mem Y → Y.Nonempty) :
    oplusTok D₀ D₁ h₀ h₁ ? sumTok D₀ D₁ h₀ h₁ :=
  (oplusTok_subsystem_sumTok h₀ h₁).trianglelefteq

end EmbeddingCompare
end

section
namespace EmbeddingCompare

variable {D₀ D₁ : NeighborhoodSystem Str}

theorem otimesTok_subsystem_prodTok (D₀ D₁ : NeighborhoodSystem Str) :
    otimesTok D₀ D₁ ? prodTok D₀ D₁ where
  master_eq := rfl
  sub := by
    rintro W (rfl | ⟨X, Y, hX, hY, -, -, rfl⟩)
    · exact ⟨D₀.master, D₁.master, D₀.master_mem, D₁.master_mem, rfl⟩
```

```
    · exact ⟨X, Y, hX, hY, rfl⟩
  inter_closed := by
    rintro W W' (rfl | ⟨X, Y, hX, hY, hXne, hYne, rfl⟩)
      (rfl | ⟨X', Y', hX', hY', hX'ne, hY'ne, rfl⟩) hInt
    · rw [Set.inter_self]; exact Or.inl rfl
    · rw [prodTokNbhd_inter, Set.inter_eq_right.mpr (D₀.sub_master hX'),
        Set.inter_eq_right.mpr (D₁.sub_master hY')]
      exact Or.inr ⟨X', Y', hX', hY', hX'ne, hY'ne, rfl⟩
    · rw [Set.inter_comm, prodTokNbhd_inter, Set.inter_eq_right.mpr (D₀.sub_master hX),
        Set.inter_eq_right.mpr (D₁.sub_master hY)]
      exact Or.inr ⟨X, Y, hX, hY, hXne, hYne, rfl⟩
    · rw [prodTokNbhd_inter] at hInt ⊢
      obtain ⟨A, B, hA, hB, heq⟩ := hInt
      obtain ⟨rfl, rfl⟩ := prodTokNbhd_injective heq
      exact Or.inr ⟨X ∩ X', Y ∩ Y', hA, hB,
        inter_ne_of_ne_left (D₀.sub_master hX) hXne,
        inter_ne_of_ne_left (D₁.sub_master hY) hYne, rfl⟩

theorem otimes_trianglelefteq_prod (D₀ D₁ : NeighborhoodSystem Str) :
    otimesTok D₀ D₁ ? prodTok D₀ D₁ :=


  (otimesTok_subsystem_prodTok D₀ D₁).trianglelefteq

end EmbeddingCompare
end

section
namespace EmbeddingCompare

variable {D₀ D₁ : NeighborhoodSystem Str}

variable {h₀ : ∀ X, D₀.mem X → X.Nonempty} {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty}

theorem otimes_unitPt_collapse :
    ∀ W, (otimesTok twoPt unitPt).mem W → W = (otimesTok twoPt unitPt).master := by
  rintro W (rfl | ⟨X, Y, _, hY, _, hYne, rfl⟩)
  · rfl
  · exact absurd hY hYne

section Part5

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

theorem strictifyMap_le (g : ApproximableMap V₀ V₁) : strictifyMap g ≤ g := fun _ _ h =
    > h.1

theorem strictifyMap_of_isStrict {f : ApproximableMap V₀ V₁} (hf : IsStrict f) :
    strictifyMap f = f := by
  apply ApproximableMap.ext
  intro X Y
  exact ⟨fun h => h.1, fun h => ⟨h, fun hXm => hf (hXm ▶ h)⟩⟩

end Part5

theorem subsingleton_element_of_only_master {? : Type*} {V : NeighborhoodSystem ?}
    (h : ∀ X, V.mem X → X = V.master) : Subsingleton V.Element := by
```

```
  refine ⟨fun x y => ?_⟩
  apply Element.ext
  intro X
  constructor
  · intro hx; have hXm := h X (x.sub hx); rw [hXm]; exact y.master_mem
  · intro hy; have hXm := h X (y.sub hy); rw [hXm]; exact x.master_mem

theorem not_trianglelefteq_otimes :
    ¬ (twoPt ? otimesTok twoPt unitPt) := by
  rintro ⟨D', hsub, ⟨e⟩⟩
  have hD'only : ∀ X, D'.mem X → X = D'.master := by
    intro X hX
    rw [otimes_unitPt_collapse X (hsub.sub hX)]; exact hsub.master_eq.symm
  have hss : Subsingleton D'.Element := subsingleton_element_of_only_master hD'only
  have hPmem : twoPt.mem ({[]} : Set Str) := Or.inr rfl
  have hne : twoPt.principal hPmem = twoPt.principal twoPt.master_mem := by
    intro heq
    have h1 : (twoPt.principal hPmem).mem ({[]} : Set Str) := ⟨hPmem, subset_rfl⟩
    rw [heq] at h1
    obtain ⟨-, h2⟩ := h1
    exact absurd (Set.mem_singleton_iff.mp (h2 (Set.mem_univ ([false] : Str))) :
      ([false] : Str) = []) (by decide)
  exact hne (e.injective (hss.allEq (e (twoPt.principal hPmem))
    (e (twoPt.principal twoPt.master_mem))))

section Part5

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

theorem toApproxMap_toFilter (g : ApproximableMap V₀ V₁) : toApproxMap (toFilter g) = g
    :=
  (funSpaceEquiv V₀ V₁).apply_symm_apply g

theorem toElementMap_inclMap (φ : (strictFun V₀ V₁).Element) :
    inclMap.toElementMap φ = incl φ := by
  apply Element.ext
  intro M
  constructor
  · rintro ⟨W, hφW, _, hincl⟩
    obtain ⟨hMmem, hg⟩ := hincl
    obtain ⟨⟨L, hL, rfl⟩, hne⟩ := hMmem
    refine ⟨⟨⟨L, hL, rfl⟩, hne⟩, ?_⟩
    intro p hp
    obtain ⟨hsstep, hWsub⟩ := toStrictMap_rel.mp (hg p hp)
    exact φ.up_mem hφW hsstep hWsub
  · rintro ⟨hMmem, hgφ⟩
    obtain ⟨⟨L, hL, rfl⟩, hne⟩ := hMmem
    have hφL : φ.mem (sstepFun L) :=
      (mem_sstepFun_iff φ hL).mpr (fun p hp => toStrictMap_rel.mp (hgφ p hp))
    refine ⟨sstepFun L, hφL, φ.sub hφL, ⟨⟨L, hL, rfl⟩, hne⟩, ?_⟩
    intro p hp
    exact ⟨φ.sub (toStrictMap_rel.mp (hgφ p hp)), fun f hf => hf p hp⟩

theorem toElementMap_strctMap (ψ : (funSpace V₀ V₁).Element) :
    strctMap.toElementMap ψ = strct ψ := by
  apply Element.ext
```

```
  intro W
  constructor
  · rintro ⟨M, hψM, _, hstrct⟩
    obtain ⟨hWmem, hg⟩ := hstrct
    obtain ⟨⟨L, hL, rfl⟩, hne⟩ := hWmem
    refine ⟨⟨⟨L, hL, rfl⟩, hne⟩, ?_⟩
    intro p hp
    obtain ⟨hstepmem, hMsub⟩ := (hg p hp).1
    exact ⟨ψ.up_mem hψM hstepmem hMsub, (hg p hp).2⟩
  · rintro ⟨hWmem, hgψ⟩
    obtain ⟨⟨L, hL, rfl⟩, hne⟩ := hWmem
    have hψL : ψ.mem (stepFun L) :=
      (mem_stepFun_iff ψ hL).mpr (fun p hp => (hgψ p hp).1)
    refine ⟨stepFun L, hψL, ψ.sub hψL, ⟨⟨L, hL, rfl⟩, hne⟩, ?_⟩
    intro p hp
    exact ⟨⟨ψ.sub (hgψ p hp).1, fun f hf => hf p hp⟩, (hgψ p hp).2⟩

theorem toFilter_toApproxMap (ψ : (funSpace V₀ V₁).Element) : toFilter (toApproxMap ψ)
    = ψ :=
  (funSpaceEquiv V₀ V₁).symm_apply_apply ψ

theorem toStrictFilter_toStrictMap (φ : (strictFun V₀ V₁).Element) :
    toStrictFilter (toStrictMap φ) = φ :=
  (strictFunEquiv V₀ V₁).symm_apply_apply φ

theorem strct_incl (φ : (strictFun V₀ V₁).Element) : strct (incl φ) = φ := by
  show toStrictFilter (strictify (toApproxMap (toFilter (toStrictMap φ).1))) = φ
  rw [toApproxMap_toFilter,
    show strictify (toStrictMap φ).1 = toStrictMap φ from
      Subtype.ext (strictifyMap_of_isStrict (toStrictMap φ).2)]
  exact toStrictFilter_toStrictMap φ

theorem strctMap_comp_inclMap :
    (strctMap (V₀ := V₀) (V₁ := V₁)).comp inclMap = idMap (strictFun V₀ V₁) := by
  apply ext_of_principal
  intro X hX
  rw [toElementMap_comp, toElementMap_inclMap, toElementMap_strctMap, strct_incl,
    toElementMap_idMap]

theorem toStrictMap_toStrictFilter (g : StrictMap V₀ V₁) : toStrictMap (toStrictFilter
    g) = g :=
  (strictFunEquiv V₀ V₁).apply_symm_apply g

theorem incl_strct_le (ψ : (funSpace V₀ V₁).Element) : incl (strct ψ) ≤ ψ := by
  have key : incl (strct ψ) = toFilter (strictifyMap (toApproxMap ψ)) := by
    show toFilter (toStrictMap (strct ψ)).1 = toFilter (strictifyMap (toApproxMap ψ))
    rw [show toStrictMap (strct ψ) = strictify (toApproxMap ψ) from
    toStrictMap_toStrictFilter _]
    rfl
  rw [key]
  conv_rhs => rw [← toFilter_toApproxMap ψ]
  exact (funSpaceEquiv V₀ V₁).symm.monotone (strictifyMap_le (toApproxMap ψ))

theorem inclMap_comp_strctMap_le :
    (inclMap (V₀ := V₀) (V₁ := V₁)).comp strctMap ≤ idMap (funSpace V₀ V₁) := by
  rw [le_iff_toElementMap_le]
```

```
  intro ψ
  rw [toElementMap_comp, toElementMap_strctMap, toElementMap_inclMap,
    toElementMap_idMap]
  exact incl_strct_le ψ

theorem strictFun_trianglelefteq_funSpace (V₀ : NeighborhoodSystem α) (V₁ :
    NeighborhoodSystem β) :
    strictFun V₀ V₁ ? funSpace V₀ V₁ :=
  trianglelefteq_of_projectionPair inclMap strctMap strctMap_comp_inclMap
    inclMap_comp_strctMap_le

end Part5

end EmbeddingCompare
end
end DomainTheory.NeighbourhoodSystems
end

section

universe w

namespace DomainTheory.NeighbourhoodSystems
namespace Theorem614

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

variable {α : Type w}

structure Setup where

  T : Endofunctor DomainObj.{w}

  hmaps : ContinuousOnMaps T

  hmono : MonotoneOnDomains T

  hcont : ContinuousOnDomains T
  {Tok : Type w}

  Γ : NeighborhoodSystem Tok

  ceq : (T.obj ⟨Tok, Γ⟩).carrier = Tok

  hsub : Γ ? (ceq ▶ (T.obj ⟨Tok, Γ⟩).sys : NeighborhoodSystem Tok)

end Theorem614
end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.SmashProduct

section
universe w
```

```
namespace Theorem614

variable {α : Type w}

theorem mem_cast {β : Type w} (e : β = α) (V : NeighborhoodSystem β) (X : Set α) :
    (e ▶ V : NeighborhoodSystem α).mem X ↔ V.mem (e.symm ▶ X : Set β) := by
  cases e; rfl

theorem rec_trans {β ? : Type w} (e : β = ?) (e' : ? = α) (x : NeighborhoodSystem β) :
    (e' ▶ (e ▶ x : NeighborhoodSystem ?) : NeighborhoodSystem α) = (e.trans e') ▶ x :=
     by
  cases e; cases e'; rfl

theorem set_rec_trans {β ? : Type w} (e : β = ?) (e' : ? = α) (X : Set β) :
    (e' ▶ (e ▶ X : Set ?) : Set α) = (e.trans e') ▶ X := by
  cases e; cases e'; rfl

theorem subsystem_cast {β : Type w} (e : β = α) {D E : NeighborhoodSystem β} (h : D ?
    E) :
    (e ▶ D : NeighborhoodSystem α) ? (e ▶ E : NeighborhoodSystem α) := by
  cases e; exact h

def iter (s : Setup.{w}) : (n : ℕ) →
    Σ' (S : NeighborhoodSystem s.Tok), Σ' (ceq : (s.T.obj ⟨s.Tok, S⟩).carrier = s.Tok),
      S ? (ceq ▶ (s.T.obj ⟨s.Tok, S⟩).sys : NeighborhoodSystem s.Tok)
  | 0 => ⟨s.Γ, s.ceq, s.hsub⟩
  | (n + 1) =>
      let p := iter s n
      ⟨p.2.1 ▶ (s.T.obj ⟨s.Tok, p.1⟩).sys,
        (s.hmono p.2.2).carrier_eq.trans p.2.1,
        by
          have hsub := subsystem_cast p.2.1 (s.hmono p.2.2).sub
          rwa [rec_trans] at hsub⟩

def Dsys (s : Setup.{w}) (n : ℕ) : NeighborhoodSystem s.Tok := (iter s n).1

theorem Dceq (s : Setup.{w}) (n : ℕ) : (s.T.obj ⟨s.Tok, Dsys s n⟩).carrier = s.Tok := (
    iter s n).2.1

theorem Dchain (s : Setup.{w}) (n : ℕ) : Dsys s n ? Dsys s (n + 1) := (iter s n).2.2

theorem Dsys_master (s : Setup.{w}) (n : ℕ) : (Dsys s n).master = s.Γ.master := by
  induction n with
  | zero => rfl
  | succ k ih => rw [← (Dchain s k).master_eq]; exact ih

theorem Dsys_succ (s : Setup.{w}) (n : ℕ) :
    Dsys s (n + 1) = (Dceq s n ▶ (s.T.obj ⟨s.Tok, Dsys s n⟩).sys : NeighborhoodSystem s
    .Tok) :=
  rfl

theorem chain_le (s : Setup.{w}) {n m : ℕ} (h : n ≤ m) : Dsys s n ? Dsys s m := by
  induction h with
  | refl => exact Subsystem.refl _
  | step _ ih => exact ih.trans (Dchain s _)
```

```
def colim (s : Setup.{w}) : NeighborhoodSystem s.Tok where
  mem X := ∃ n, (Dsys s n).mem X
  master := s.Γ.master
  master_nonempty := s.Γ.master_nonempty
  master_mem := ⟨0, s.Γ.master_mem⟩
  inter_mem := by
    rintro X Y Z ⟨n, hX⟩ ⟨m, hY⟩ ⟨p, hZ⟩ hsub
    set N := max n (max m p) with hN
    have hXN : (Dsys s N).mem X := (chain_le s (le_max_left n _)).sub hX
    have hYN : (Dsys s N).mem Y :=
      (chain_le s ((le_max_left m p).trans (le_max_right n _))).sub hY
    have hZN : (Dsys s N).mem Z :=
      (chain_le s ((le_max_right m p).trans (le_max_right n _))).sub hZ
    exact ⟨N, (Dsys s N).inter_mem hXN hYN hZN hsub⟩
  sub_master := by
    rintro X ⟨n, hX⟩
    rw [← Dsys_master s n]
    exact (Dsys s n).sub_master hX

@[simp] theorem mem_colim (s : Setup.{w}) {X : Set s.Tok} :
    (colim s).mem X ↔ ∃ n, (Dsys s n).mem X := Iff.rfl

@[simp] theorem colim_master (s : Setup.{w}) : (colim s).master = s.Γ.master := rfl

theorem Dsys_sub_colim (s : Setup.{w}) (n : ℕ) : Dsys s n ? colim s where
  master_eq := by rw [colim_master, Dsys_master]
  sub hX := ⟨n, hX⟩
  inter_closed := by
    rintro X Y hX hY ⟨m, hXY⟩
    have hN : (Dsys s (max n m)).mem (X ∩ Y) := by
      have hle : (Dsys s m) ? (Dsys s (max n m)) := chain_le s (le_max_right n m)
      exact hle.sub hXY

    exact (chain_le s (le_max_left n m)).inter_closed hX hY hN

theorem colimCeq (s : Setup.{w}) : (s.T.obj ⟨s.Tok, colim s⟩).carrier = s.Tok :=
  (s.hmono (Dsys_sub_colim s 0)).carrier_eq.trans (Dceq s 0)

def Tcolim (s : Setup.{w}) : NeighborhoodSystem s.Tok :=
  colimCeq s ▸ (s.T.obj ⟨s.Tok, colim s⟩).sys

theorem Dsys_sub_Tcolim (s : Setup.{w}) (n : ℕ) : Dsys s (n + 1) ? Tcolim s := by
  have h := subsystem_cast (Dceq s n) (s.hmono (Dsys_sub_colim s n)).sub
  rw [rec_trans] at h
  exact h

theorem Tcolim_master (s : Setup.{w}) : (Tcolim s).master = s.Γ.master := by
  rw [← (Dsys_sub_Tcolim s 0).master_eq, Dsys_master]

theorem Tcolim_sub_colim (s : Setup.{w}) {X : Set s.Tok} (hX : (Tcolim s).mem X) :
    (colim s).mem X := by
  obtain ⟨hmono', hC⟩ := s.hcont
  set ? : Set (NeighborhoodSystem s.Tok) := Set.range (Dsys s) with h?def
  have h? : ∀ {{D}}, D ∈ ? → D ? colim s := by rintro D ⟨n, rfl⟩; exact Dsys_sub_colim
     s n
```

```
  have hne : ?.Nonempty := ⟨Dsys s 0, ⟨0, rfl⟩⟩
  have hdir : DirectedOn (· ? ·) ? := by
    rintro _ ⟨n, rfl⟩ _ ⟨m, rfl⟩
    exact ⟨Dsys s (max n m), ⟨max n m, rfl⟩,
      chain_le s (le_max_left n m), chain_le s (le_max_right n m)⟩
  have hU : ∀ Y, (colim s).mem Y ↔ ∃ D ∈ ?, D.mem Y := by
    intro Y; constructor
    · rintro ⟨n, hn⟩; exact ⟨Dsys s n, ⟨n, rfl⟩, hn⟩
    · rintro ⟨D, ⟨n, rfl⟩, hn⟩; exact ⟨n, hn⟩
  have heq := hC ? h? hne hdir (Subsystem.refl (colim s)) hU
  set Y₀ : Set (s.T.obj ⟨s.Tok, colim s⟩).carrier := (colimCeq s).symm ▸ X with hY₀

  have hmem : Y₀ ∈ targetFam s.T hmono' (Subsystem.refl (colim s)) :=
    (mem_cast (colimCeq s) _ X).mp hX
  rw [heq, Set.mem_iUnion] at hmem
  obtain ⟨D, hmem⟩ := hmem
  rw [Set.mem_iUnion] at hmem
  obtain ⟨hD, hmemD⟩ := hmem
  obtain ⟨n, rfl⟩ := hD
  simp only [targetFam, Set.mem_ofPred_eq] at hmemD

  refine ⟨n + 1, ?_⟩
  rw [Dsys_succ s n, mem_cast (Dceq s n)]
  have key : ((Dceq s n).symm ▸ X : Set (s.T.obj ⟨s.Tok, Dsys s n⟩).carrier)
      = (s.hmono (h? ⟨n, rfl⟩)).carrier_eq ▸ Y₀ := by
    rw [hY₀, set_rec_trans]
  rw [key]
  exact hmemD

theorem colim_sub_Tcolim (s : Setup.{w}) {X : Set s.Tok} (hX : (colim s).mem X) :
    (Tcolim s).mem X := by
  obtain ⟨n, hn⟩ := hX
  exact (Dsys_sub_Tcolim s n).sub ((Dchain s n).sub hn)

theorem Tcolim_eq_colim (s : Setup.{w}) : Tcolim s = colim s :=
  NeighborhoodSystem.ext
    (fun _ => ⟨fun h => Tcolim_sub_colim s h, fun h => colim_sub_Tcolim s h⟩)
    (by rw [Tcolim_master, colim_master])

theorem domainObj_ext {c : Type w} (σ : NeighborhoodSystem c) (e : c = α)
    {V : NeighborhoodSystem α} (h : (e ▸ σ : NeighborhoodSystem α) = V) :
    (⟨c, σ⟩ : DomainObj) = ⟨α, V⟩ := by
  cases e; cases h; rfl

theorem colimObj_eq (s : Setup.{w}) :
    s.T.obj ⟨s.Tok, colim s⟩ = (⟨s.Tok, colim s⟩ : DomainObj) :=
  domainObj_ext (s.T.obj ⟨s.Tok, colim s⟩).sys (colimCeq s) (Tcolim_eq_colim s)

def isoOfEq {Obj : Type*} [Category Obj] {X Y : Obj} (h : X = Y) : Iso X Y := by
  cases h
  exact ⟨(? X), (? X), CategoryTheory.Category.id_comp _, CategoryTheory.Category.
    id_comp _⟩

def colimIso (s : Setup.{w}) : Iso (s.T.obj ⟨s.Tok, colim s⟩) (⟨s.Tok, colim s⟩ :
    DomainObj) :=
  isoOfEq (colimObj_eq s)
```

```
def rho (s : Setup.{w}) (n : ℕ) : ApproximableMap (colim s) (colim s) :=
  (Dsys_sub_colim s n).inj.comp (Dsys_sub_colim s n).proj

theorem rho_rel (s : Setup.{w}) (n : ℕ) {X Y : Set s.Tok} :
    (rho s n).rel X Y ↔
      (colim s).mem X ∧ (colim s).mem Y ∧ ∃ z, (Dsys s n).mem z ∧ X ⊆ z ∧ z ⊆ Y := by
  unfold rho
  rw [comp_rel]
  constructor
  · rintro ⟨z, ⟨hcX, hDz, hXz⟩, _, hcY, hzY⟩
    exact ⟨hcX, hcY, z, hDz, hXz, hzY⟩
  · rintro ⟨hcX, hcY, z, hDz, hXz, hzY⟩
    exact ⟨z, ⟨hcX, hDz, hXz⟩, hDz, hcY, hzY⟩

theorem rho_mono (s : Setup.{w}) {n m : ℕ} (h : n ≤ m) {X Y : Set s.Tok}
    (hr : (rho s n).rel X Y) : (rho s m).rel X Y := by
  rw [rho_rel] at hr ⊢
  obtain ⟨hcX, hcY, z, hDz, hXz, hzY⟩ := hr
  exact ⟨hcX, hcY, z, (chain_le s h).sub hDz, hXz, hzY⟩

def iSupRho (s : Setup.{w}) : ApproximableMap (colim s) (colim s) :=
  iSupMap (rho s) (fun i j => ⟨max i j,
    fun _ _ h => rho_mono s (le_max_left i j) h,
    fun _ _ h => rho_mono s (le_max_right i j) h⟩)

abbrev colimAlg (s : Setup.{w}) : TAlgebra s.T :=
  ⟨⟨s.Tok, colim s⟩, (colimIso s).hom⟩

abbrev objColim (s : Setup.{w}) : DomainObj := ⟨s.Tok, colim s⟩

abbrev objDsys (s : Setup.{w}) (n : ℕ) : DomainObj := ⟨s.Tok, Dsys s n⟩

abbrev Tmap_rho (s : Setup.{w}) (n : ℕ) :
    ApproximableMap (s.T.obj (objColim s)).sys (s.T.obj (objColim s)).sys :=
  s.T.map (X := objColim s) (Y := objColim s) (rho s n)

@[simp] theorem Dsys_zero (s : Setup.{w}) : Dsys s 0 = s.Γ := rfl

theorem algHom_ext {Obj : Type*} [Category Obj] {T : Endofunctor Obj} {A B : TAlgebra
    T}
    {g g' : AlgHom A B} (h : g.hom = g'.hom) : g = g' := by
  cases g; cases g'; cases h; rfl

theorem cat_comp_eq {X Y Z : DomainObj} (g : (Y ⟶ Z)) (f : (X ⟶ Y)) :
    g ≫ f = g.comp f := rfl


theorem iSupRho_eq_id (s : Setup.{w}) : iSupRho s = idMap (colim s) := by
  apply ApproximableMap.ext
  intro X Y
  rw [idMap_rel]
  constructor
  · rintro ⟨n, hr⟩
    rw [rho_rel] at hr
    obtain ⟨hcX, hcY, z, _, hXz, hzY⟩ := hr
```

```
    exact ⟨hcX, hcY, hXz.trans hzY⟩
  · rintro ⟨hcX, hcY, hXY⟩
    obtain ⟨n, hX⟩ := hcX
    exact ⟨n, (rho_rel s n).mpr ⟨⟨n, hX⟩, hcY, X, hX, subset_rfl, hXY⟩⟩

theorem isoOfEq_conj {Obj : Type*} [Category Obj] {X Y : Obj} (e : X = Y)
    (f : (X ⟶ X)) :
    (isoOfEq e).hom ? f ? (isoOfEq e).inv = (e ▶ f : (Y ⟶ Y)) := by
  cases e
  change (? X) ? f ? (? X) = f
  dsimp only [compAfter]
  rw [CategoryTheory.Category.comp_id, CategoryTheory.Category.id_comp]

theorem map_comp_proj_heq {Tok : Type w} {Pc Qc : Type w} (cn : Pc = Tok) (cc : Qc =
    Tok)
    {Ps : NeighborhoodSystem Pc} {Qs : NeighborhoodSystem Qc} (ce : Qc = Pc)
    (sub : Ps ? (ce ▶ Qs : NeighborhoodSystem Pc))
    {Dn1 Col : NeighborhoodSystem Tok}
    (hDn1 : (cn ▶ Ps : NeighborhoodSystem Tok) = Dn1)
    (hCol : (cc ▶ Qs : NeighborhoodSystem Tok) = Col)
    (hsub' : Dn1 ? Col)
    (Tmi : ApproximableMap Ps Qs) (Tmj : ApproximableMap Qs Ps)
    (hi : HEq Tmi sub.inj) (hj : HEq Tmj sub.proj) :
    HEq (Tmi.comp Tmj) (hsub'.inj.comp hsub'.proj) := by
  subst cn
  subst cc
  obtain rfl := hDn1
  obtain rfl := hCol
  have e1 : Tmi = sub.inj := eq_of_heq hi
  have e2 : Tmj = sub.proj := eq_of_heq hj
  rw [e1, e2]

theorem map_rho_heq (s : Setup.{w}) (n : ℕ) :
    HEq (Tmap_rho s n) (rho s (n + 1)) := by
  have hcomp : Tmap_rho s n
      = (s.T.map (X := objDsys s n) (Y := objColim s) (Dsys_sub_colim s n).inj).comp
          (s.T.map (X := objColim s) (Y := objDsys s n) (Dsys_sub_colim s n).proj) :=
    s.T.map_comp (X := objColim s) (Y := objDsys s n) (Z := objColim s)
      (Dsys_sub_colim s n).inj (Dsys_sub_colim s n).proj
  rw [hcomp]
  exact map_comp_proj_heq (Dceq s n) (colimCeq s) (s.hmono (Dsys_sub_colim s n)).
    carrier_eq
    (s.hmono (Dsys_sub_colim s n)).sub (Dsys_succ s n).symm (Tcolim_eq_colim s)
    (Dsys_sub_colim s (n + 1)) (s.T.map (X := objDsys s n) (Y := objColim s) (
    Dsys_sub_colim s n).inj)
    (s.T.map (X := objColim s) (Y := objDsys s n) (Dsys_sub_colim s n).proj)
    (s.hmono (Dsys_sub_colim s n)).inj_heq (s.hmono (Dsys_sub_colim s n)).proj_heq

theorem nonempty_strict_algHom (s : Setup.{w}) (B : TAlgebra s.T) (hk : IsStrict B.str
    ) :
    Nonempty {g : AlgHom (colimAlg s) B // IsStrict g.hom} :=
  nonempty_algHom_of_continuousOnMaps s.T s.hmaps (colimIso s) B hk

theorem nonempty_algHom (s : Setup.{w}) (B : TAlgebra s.T) (hk : IsStrict B.str) :
    Nonempty (AlgHom (colimAlg s) B) :=
  (nonempty_strict_algHom s B hk).map (·.1)
```

```
end Theorem614
end
end DomainTheory.NeighbourhoodSystems
end
```

4.7.60 Exercise 6.27 — which embeddings hold

NeighbourhoodSystems/DomainEquations/Exercise_6_27.lean

Exercise_6_27.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.embeddingCompare

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem DomainTheory.NeighbourhoodSystems.FunctorTokens DomainTheory.
    NeighbourhoodSystems.SmashProduct BitDom
open DomainTheory.NeighbourhoodSystems.EmbeddingCompare
theorem example_627 :
    (∀ (D₀ D₁ : NeighborhoodSystem Str), otimesTok D₀ D₁ ? prodTok D₀ D₁) ∧
    (∀ {D₀ D₁ : NeighborhoodSystem Str} {h₀ : ∀ X, D₀.mem X → X.Nonempty}
        {h₁ : ∀ Y, D₁.mem Y → Y.Nonempty},
      D₀ ? oplusTok D₀ D₁ h₀ h₁) ∧
    (∀ (D₀ D₁ : NeighborhoodSystem Str), D₀ ? prodTok D₀ D₁) ∧
    (¬ (twoPt ? otimesTok twoPt unitPt)) ∧
    (∀ {D₀ D₁ : NeighborhoodSystem Str} (h₀ : ∀ X, D₀.mem X → X.Nonempty)
        (h₁ : ∀ Y, D₁.mem Y → Y.Nonempty),
      oplusTok D₀ D₁ h₀ h₁ ? sumTok D₀ D₁ h₀ h₁) ∧
    (∀ {α β : Type*} (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β),
      strictFun V₀ V₁ ? funSpace V₀ V₁) :=
  ⟨@otimes_trianglelefteq_prod, @inl_trianglelefteq_oplus, @fst_trianglelefteq_prod,
    @not_trianglelefteq_otimes, @oplus_trianglelefteq_sum,
    @strictFun_trianglelefteq_funSpace⟩
```

EXERCISE 6.27. Which of the following relationships are true:

$$(\mathcal{D} \otimes \mathcal{E}) \trianglelefteq (\mathcal{D} \times \mathcal{E})\,; \quad \mathcal{D} \trianglelefteq \mathcal{D} \times \mathcal{E}\,;$$

$$(\mathcal{D} \oplus \mathcal{E}) \trianglelefteq (\mathcal{D} + \mathcal{E})\,; \quad \mathcal{D} \trianglelefteq \mathcal{D} \oplus \mathcal{E}\,;$$

$$(\mathcal{D} \to_{\perp} \mathcal{E}) \trianglelefteq (\mathcal{D} \to \mathcal{E})\,; \quad \mathcal{D} \trianglelefteq \mathcal{D} \otimes \mathcal{E}\,?$$

4.7.61 finiteEmbedCancel — finite mutual embeddings

finiteEmbedCancel.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.projPairSubdomain

public import Mathlib.Data.Fintype.Powerset

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

def NeighborhoodSystem.IsFinite {α : Type*} (D : NeighborhoodSystem α) : Prop :=
  Finite {X : Set α // D.mem X}

def projElementEmbedding {α β : Type*} {D : NeighborhoodSystem α} {E :
    NeighborhoodSystem β}
    (i : ApproximableMap D E) (j : ApproximableMap E D) (hji : j.comp i = idMap D) :
    D.Element ↪o E.Element :=
  OrderEmbedding.ofMapLEIff i.toElementMap (by
    have hround : ∀ x : D.Element, j.toElementMap (i.toElementMap x) = x := by
      intro x; rw [← toElementMap_comp, hji, toElementMap_idMap]
    intro a b
    constructor
    · intro h
      have hjm := toElementMap_mono j h
      rwa [hround a, hround b] at hjm
    · intro h
      exact toElementMap_mono i h)

theorem Trianglelefteq.elementEmbedding (h : D ◁ E) :
    Nonempty (D.Element ↪o E.Element) := by
  obtain ⟨D', hsub, ⟨e⟩⟩ := h
  exact ⟨e.toOrderEmbedding.trans
    (projElementEmbedding hsub.inj hsub.proj hsub.proj_comp_inj)⟩

theorem finite_element_of_isFinite {α : Type*} {D : NeighborhoodSystem α} (h : D.
    IsFinite) :
    Finite D.Element := by
  have : Finite {X : Set α // D.mem X} := h
  apply Finite.of_injective
    (β := Set {X : Set α // D.mem X}) (fun x => {p | x.mem p.1})
  intro x y hxy
  apply Element.ext
  intro X
  by_cases hX : D.mem X
  · have := Set.ext_iff.mp hxy ⟨X, hX⟩
    simpa using this
  · exact ⟨fun hmem => absurd (x.sub hmem) hX, fun hmem => absurd (y.sub hmem) hX⟩

theorem orderIso_of_embeddings {P Q : Type*} [PartialOrder P] [PartialOrder Q]
    [Finite P] [Finite Q] (f : P ↪o Q) (g : Q ↪o P) : Nonempty (P ≃o Q) := by
  classical
  obtain ⟨_⟩ := nonempty_fintype P
```

```
  obtain ⟨_⟩ := nonempty_fintype Q
  have hcard : Fintype.card P = Fintype.card Q :=
    le_antisymm (Fintype.card_le_of_injective f f.injective)
      (Fintype.card_le_of_injective g g.injective)
  have hbij : Function.Bijective f :=
    (Fintype.bijective_iff_injective_and_card f).mpr ⟨f.injective, hcard⟩
  exact ⟨{ toEquiv := Equiv.ofBijective f hbij, map_rel_iff' := f.map_rel_iff' }⟩

theorem isomorphic_of_trianglelefteq_both [Finite D.Element] [Finite E.Element]
    (h₁ : D ? E) (h₂ : E ? D) : D ≅? E := by
  obtain ⟨f⟩ := h₁.elementEmbedding
  obtain ⟨g⟩ := h₂.elementEmbedding
  exact orderIso_of_embeddings f g

theorem isomorphic_of_finite_system (hD : D.IsFinite) (hE : E.IsFinite)
    (h₁ : D ? E) (h₂ : E ? D) : D ≅? E := by
  have := finite_element_of_isFinite hD
  have := finite_element_of_isFinite hE
  exact isomorphic_of_trianglelefteq_both h₁ h₂

end

namespace Subsystem

variable {α : Type*} {D E : NeighborhoodSystem α}

theorem toElementMap_inj (h : D ? E) (x : D.Element) {Y : Set α} :
    (h.inj.toElementMap x).mem Y ↔ E.mem Y ∧ ∃ X, x.mem X ∧ X ⊆ Y := by
  constructor
  · rintro ⟨X, hX, _, hY, hXY⟩
    exact ⟨hY, X, hX, hXY⟩
  · rintro ⟨hY, X, hX, hXY⟩
    exact ⟨X, hX, x.sub hX, hY, hXY⟩

theorem toElementMap_proj (h : D ? E) (y : E.Element) {X : Set α} :
    (h.proj.toElementMap y).mem X ↔ y.mem X ∧ D.mem X := by
  constructor
  · rintro ⟨Y, hY, hEY, hX, hYX⟩
    exact ⟨y.up_mem hY (h.sub hX) hYX, hX⟩
  · rintro ⟨hX, hDX⟩
    exact ⟨X, hX, y.sub hX, hDX, subset_rfl⟩

end Subsystem

export CategoryTheory (Category)

open scoped CategoryTheory

universe u

abbrev compAfter {Obj : Type u} [Category.{v} Obj] {X Y Z : Obj} (g : Y ⟶ Z) (f : X
    ⟶ Y) : X ⟶ Z :=
  f ? g

scoped infixr:80 " ? " => compAfter
```

```
end DomainTheory.NeighbourhoodSystems
```

4.7.62 Exercise 6.28 — finite embeddings cancel

NeighbourhoodSystems/DomainEquations/Exercise_6_28.lean

`Exercise_6_28.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.finiteEmbedCancel

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

theorem example_628 :
    (∀ {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β},
      D.IsFinite → E.IsFinite → D ? E → E ? D → D ≅? E) ∧
    (∀ {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}
      [Finite D.Element] [Finite E.Element],
      D ? E → E ? D → D ≅? E) :=
  ⟨@isomorphic_of_finite_system, @isomorphic_of_trianglelefteq_both⟩
```

EXERCISE 6.28. (Suggested by G. Plotkin). Show that if $\mathcal{D}$ and $\mathcal{E}$ are *finite* systems and

$$\mathcal{D} \trianglelefteq \mathcal{E} \trianglelefteq \mathcal{D}\,,$$

then $\mathcal{D} \cong \mathcal{E}$. Need the same be true of infinite systems?

4.7.63 infSumProd — infinitary sum and product

`infSumProd.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.sumProdTok
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_10
import Mathlib.Data.Set.Finite.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
namespace InfSumProd
universe u v
variable {ι : Type u} {α : ι → Type v}
```

```
def iprodNbhd (X : ∀ i, Set (α i)) : Set (Σ i, α i) := {p | p.2 ∈ X p.1}

@[simp] theorem mem_iprodNbhd {X : ∀ i, Set (α i)} {i : ι} {a : α i} :
    (⟨i, a⟩ : Σ i, α i) ∈ iprodNbhd X ↔ a ∈ X i := Iff.rfl

variable {D : ∀ i, NeighborhoodSystem (α i)}

def FinSupp (D : ∀ i, NeighborhoodSystem (α i)) (X : ∀ i, Set (α i)) : Prop :=
  ∃ l : List ι, ∀ i, i ∈ l → X i = (D i).master

theorem FinSupp.inter {X X' : ∀ i, Set (α i)}
    (hX : FinSupp D X) (hX' : FinSupp D X') : FinSupp D (fun i => X i ∩ X' i) := by
  obtain ⟨l, hl⟩ := hX; obtain ⟨l', hl'⟩ := hX'
  refine ⟨l ++ l', fun i hi => ?_⟩
  rw [List.mem_append, not_or] at hi
  simp only [hl i hi.1, hl' i hi.2, Set.inter_self]

theorem iprodNbhd_injective {X X' : ∀ i, Set (α i)} (h : iprodNbhd X = iprodNbhd X') :
    X = X' := by
  funext i; apply Set.ext; intro a
  simpa only [mem_iprodNbhd] using Set.ext_iff.mp h ⟨i, a⟩

theorem iprodNbhd_inter (X X' : ∀ i, Set (α i)) :
    iprodNbhd X ∩ iprodNbhd X' = iprodNbhd (fun i => X i ∩ X' i) := by
  apply Set.ext; rintro ⟨i, a⟩; simp [iprodNbhd, Set.mem_inter_iff]

theorem iprodNbhd_subset_iff {X X' : ∀ i, Set (α i)} :
    iprodNbhd X ⊆ iprodNbhd X' ↔ ∀ i, X i ⊆ X' i := by
  constructor
  · intro h i a ha; exact h (show ⟨i, a⟩ : Σ i, α i) ∈ iprodNbhd X from ha)
  · rintro h ⟨i, a⟩ ha; exact h i ha

theorem FinSupp.master : FinSupp D (fun i => (D i).master) := ⟨[], fun _ _ => rfl⟩

section IndexedProduct
variable [Nonempty ι]
variable (D) in

def iprod : NeighborhoodSystem (Σ i, α i) where
  mem W := ∃ X : ∀ i, Set (α i), (∀ i, (D i).mem (X i)) ∧ FinSupp D X ∧ W = iprodNbhd
    X
  master := iprodNbhd (fun i => (D i).master)
  master_nonempty := by
    cases ?Nonempty ι? with
    | intro i =>
      obtain ⟨a, ha⟩ := (D i).master_nonempty
      exact ⟨⟨i, a⟩, ha⟩
  master_mem := ⟨fun i => (D i).master, fun i => (D i).master_mem, FinSupp.master, rfl⟩
  sub_master := by
    rintro W ⟨X, hX, _, rfl⟩
    exact iprodNbhd_subset_iff.mpr fun i => (D i).sub_master (hX i)
  inter_mem := by
    rintro W W' Z ⟨X, hX, hXf, rfl⟩ ⟨X', hX', hX'f, rfl⟩ ⟨ZZ, hZZ, _, rfl⟩ hsub
    rw [iprodNbhd_inter] at hsub ⊢
    refine ⟨fun i => X i ∩ X' i, fun i => ?_, hXf.inter hX'f, rfl⟩
    have : ZZ i ⊆ X i ∩ X' i := iprodNbhd_subset_iff.mp hsub i
```

```
    exact (D i).inter_mem (hX i) (hX' i) (hZZ i) this

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

def updTuple (D : ∀ i, NeighborhoodSystem (α i)) (i : ι) (U : Set (α i)) : ∀ j, Set (α
    j) :=
  Function.update (fun j => (D j).master) i U

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

omit [Nonempty ι] in
@[simp] theorem updTuple_apply_self (i : ι) (U : Set (α i)) : updTuple D i U i = U :=
    by
  simp [updTuple]

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

def fromPi (D : ∀ i, NeighborhoodSystem (α i)) (x : ∀ i, (D i).Element) : (iprod D).
    Element where
  mem W := ∃ X : ∀ i, Set (α i), (∀ i, (x i).mem (X i)) ∧ FinSupp D X ∧ W = iprodNbhd
    X
  sub := by rintro W ⟨X, hX, hXf, rfl⟩; exact ⟨X, fun i => (x i).sub (hX i), hXf, rfl⟩
  master_mem := ⟨fun i => (D i).master, fun i => (x i).master_mem, FinSupp.master, rfl⟩
  inter_mem := by
    rintro W W' ⟨X, hX, hXf, rfl⟩ ⟨X', hX', hX'f, rfl⟩
    exact ⟨fun i => X i ∩ X' i, fun i => (x i).inter_mem (hX i) (hX' i), hXf.inter hX'f
    ,
      iprodNbhd_inter X X'⟩
```

```
  up_mem := by
    rintro W W' ⟨X, hX, _, rfl⟩ hW' hsub
    obtain ⟨X', hX'mem, hX'f, rfl⟩ := hW'
    rw [iprodNbhd_subset_iff] at hsub
    exact ⟨X', fun i => (x i).up_mem (hX i) (hX'mem i) (hsub i), hX'f, rfl⟩

end Iso

end IndexedProduct

def injI (i : ι) (X : Set (α i)) : Set (Option (Σ i, α i)) :=
  (fun a => (some ⟨i, a⟩ : Option (Σ i, α i))) '' X

theorem none_not_mem_injI {i : ι} {X : Set (α i)} : (none : Option (Σ i, α i)) ∈ injI
    i X := by
  rintro ⟨a, _, heq⟩; simp at heq

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

def restrictTo (D : ∀ i, NeighborhoodSystem (α i)) (l : List ι) (X : ∀ i, Set (α i)) :
    ∀ j, Set (α j) := fun j => if j ∈ l then X j else (D j).master

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

def slice (D : ∀ i, NeighborhoodSystem (α i)) (i : ι) (U : Set (α i)) : Set (Σ i, α i)
    :=
  iprodNbhd (updTuple D i U)

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]
```

```
theorem iprod_mem_slice_inv {i : ι} {U : Set (α i)} (h : (iprod D).mem (slice D i U)) :
    (D i).mem U := by
  obtain ⟨X, hX, _, heq⟩ := h
  have hXeq : updTuple D i U = X := iprodNbhd_injective heq
  have hi := hX i
  rwa [← hXeq, updTuple_apply_self] at hi

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

omit [Nonempty ι] in
theorem slice_eq (i : ι) (U : Set (α i)) : slice D i U = iprodNbhd (updTuple D i U) :=
    rfl

end Iso

end IndexedProduct

@[simp] theorem some_mem_injI {i : ι} {a : α i} {X : Set (α i)} :
    (some ⟨i, a⟩ : Option (Σ i, α i)) ∈ injI i X ↔ a ∈ X := by
  constructor
  · rintro ⟨b, hb, heq⟩
    simp only [Option.some.injEq, Sigma.mk.injEq, heq_eq_eq, true_and] at heq
    rwa [heq] at hb
  · intro ha; exact ⟨a, ha, rfl⟩

theorem index_of_some_mem_injI {i k : ι} {a : α k} {X : Set (α i)}
    (hmem : (some ⟨k, a⟩ : Option (Σ i, α i)) ∈ injI i X) : k = i := by
  obtain ⟨b, _, heq⟩ := hmem
  simp only [Option.some.injEq] at heq
  exact (congrArg Sigma.fst heq).symm

theorem injI_inter_same (i : ι) (X Y : Set (α i)) : injI i X ∩ injI i Y = injI i (X ∩ Y
    ) := by
  apply Set.ext
  rintro (_ | ⟨k, a⟩)
  · constructor
    · rintro ⟨h, _⟩; exact absurd h none_not_mem_injI
    · intro h; exact absurd h none_not_mem_injI
  · constructor
    · rintro ⟨hx, hy⟩
      obtain rfl := index_of_some_mem_injI hx
      exact some_mem_injI.mpr ⟨some_mem_injI.mp hx, some_mem_injI.mp hy⟩
    · intro h
      obtain rfl := index_of_some_mem_injI h
      have h' := some_mem_injI.mp h
      exact ⟨some_mem_injI.mpr h'.1, some_mem_injI.mpr h'.2⟩
```

```
theorem injI_subset_iff {i : ι} {X Y : Set (α i)} : injI i X ⊆ injI i Y ↔ X ⊆ Y := by
  constructor
  · intro h a ha; exact some_mem_injI.mp (h (some_mem_injI.mpr ha))
  · intro h; rintro w ⟨a, ha, rfl⟩; exact ⟨a, h ha, rfl⟩

def iotimes [Nonempty ι] (D : ∀ i, NeighborhoodSystem (α i)) : NeighborhoodSystem (Σ i,
     α i) where
  mem W := W = iprodNbhd (fun i => (D i).master) ∨
    ∃ X : ∀ i, Set (α i), (∀ i, (D i).mem (X i)) ∧ (∀ i, X i = (D i).master) ∧
      FinSupp D X ∧ W = iprodNbhd X
  master := iprodNbhd (fun i => (D i).master)
  master_nonempty := by
    cases ?Nonempty ι? with
    | intro i =>
      obtain ⟨a, ha⟩ := (D i).master_nonempty
      exact ⟨⟨i, a⟩, ha⟩
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | ⟨X, hX, _, _, rfl⟩)
    · exact subset_rfl
    · exact iprodNbhd_subset_iff.mpr fun i => (D i).sub_master (hX i)
  inter_mem := by
    rintro W W' Z hW hW' hZ hsub
    rcases hW with rfl | ⟨X, hX, hXne, hXf, rfl⟩
    · have hW'sub : W' ⊆ iprodNbhd (fun i => (D i).master) := by
        rcases hW' with rfl | ⟨X', hX', _, _, rfl⟩
        · exact subset_rfl
        · exact iprodNbhd_subset_iff.mpr fun i => (D i).sub_master (hX' i)
      rw [Set.inter_eq_right.mpr hW'sub]; exact hW'
    · rcases hW' with rfl | ⟨X', hX', hX'ne, hX'f, rfl⟩
      · rw [Set.inter_eq_left.mpr (iprodNbhd_subset_iff.mpr fun i => (D i).sub_master (
    hX i))]
        exact Or.inr ⟨X, hX, hXne, hXf, rfl⟩
      · rw [iprodNbhd_inter] at hsub ⊢
        obtain ⟨ZZ, hZZmem, rfl⟩ : ∃ ZZ, (∀ i, (D i).mem (ZZ i)) ∧ Z = iprodNbhd ZZ :=
    by
          rcases hZ with rfl | ⟨ZZ, hZZ, _, _, rfl⟩
          · exact ⟨fun i => (D i).master, fun i => (D i).master_mem, rfl⟩
          · exact ⟨ZZ, hZZ, rfl⟩
        refine Or.inr ⟨fun i => X i ∩ X' i, fun i => ?_, fun i => ?_, ?_, rfl⟩
        · exact (D i).inter_mem (hX i) (hX' i) (hZZmem i) (iprodNbhd_subset_iff.mp hsub
     i)
        · intro heq
          exact hXne i (Set.Subset.antisymm ((D i).sub_master (hX i))
            (by rw [← heq]; exact Set.inter_subset_left))
        · exact hXf.inter hX'f

def sumMasterI (D : ∀ i, NeighborhoodSystem (α i)) : Set (Option (Σ i, α i)) :=
  insert none ((fun p => (some p : Option (Σ i, α i))) '' iprodNbhd (fun i => (D i).
    master))

theorem injI_subset_sumMasterI {i : ι} {X : Set (α i)} (hX : (D i).mem X) :
    injI i X ⊆ sumMasterI D := by
  rintro w ⟨a, ha, rfl⟩
  exact Set.mem_insert_iff.mpr (Or.inr ⟨⟨i, a⟩, (D i).sub_master hX ha, rfl⟩)
```

```
theorem none_mem_sumMasterI : (none : Option (Σ i, α i)) ∈ sumMasterI D := Set.
    mem_insert _ _

def ioplus (D : ∀ i, NeighborhoodSystem (α i))
    (hne : ∀ i, ∀ X : Set (α i), (D i).mem X → X.Nonempty) :
    NeighborhoodSystem (Option (Σ i, α i)) where
  mem W := W = sumMasterI D ∨ ∃ i, ∃ X : Set (α i), (D i).mem X ∧ X = (D i).master ∧ W
     = injI i X
  master := sumMasterI D
  master_nonempty := ⟨none, none_mem_sumMasterI⟩
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | ⟨i, X, hX, _, rfl⟩)
    · exact subset_rfl
    · exact injI_subset_sumMasterI hX
  inter_mem := by
    rintro W W' Z hW hW' hZ hsub
    rcases hW with rfl | ⟨i, X, hX, hXne, rfl⟩
    · have hsub' : W' ⊆ sumMasterI D := by
        rcases hW' with rfl | ⟨j, Y, hY, _, rfl⟩
        · exact subset_rfl
        · exact injI_subset_sumMasterI hY
      rw [Set.inter_eq_right.mpr hsub']; exact hW'
    · rcases hW' with rfl | ⟨j, Y, hY, hYne, rfl⟩
      · rw [Set.inter_eq_left.mpr (injI_subset_sumMasterI hX)]
        exact Or.inr ⟨i, X, hX, hXne, rfl⟩
      · rcases hZ with rfl | ⟨k, Wk, hWk, _, rfl⟩
        · exact absurd (hsub none_mem_sumMasterI).1 none_not_mem_injI
        · obtain ⟨w, hw⟩ := hne k Wk hWk
          have hmemZ := hsub (some_mem_injI.mpr hw)
          obtain rfl := index_of_some_mem_injI hmemZ.1
          obtain rfl := index_of_some_mem_injI hmemZ.2
          rw [injI_inter_same] at hsub ⊢
          exact Or.inr ⟨k, X ∩ Y, (D k).inter_mem hX hY hWk (injI_subset_iff.mp hsub),
            fun heq => hXne (Set.Subset.antisymm ((D k).sub_master hX)
              (heq ▶ Set.inter_subset_left)), rfl⟩

def isum (D : ∀ i, NeighborhoodSystem (α i))
    (hne : ∀ i, ∀ X : Set (α i), (D i).mem X → X.Nonempty) :
    NeighborhoodSystem (Option (Σ i, α i)) where
  mem W := W = sumMasterI D ∨ ∃ i, ∃ X : Set (α i), (D i).mem X ∧ W = injI i X
  master := sumMasterI D
  master_nonempty := ⟨none, none_mem_sumMasterI⟩
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | ⟨i, X, hX, rfl⟩)
    · exact subset_rfl
    · exact injI_subset_sumMasterI hX
  inter_mem := by
    rintro W W' Z hW hW' hZ hsub
    rcases hW with rfl | ⟨i, X, hX, rfl⟩
    · have hsub' : W' ⊆ sumMasterI D := by
        rcases hW' with rfl | ⟨j, Y, hY, rfl⟩
        · exact subset_rfl
        · exact injI_subset_sumMasterI hY
```

```
      rw [Set.inter_eq_right.mpr hsub']; exact hW'
    · rcases hW' with rfl | ⟨j, Y, hY, rfl⟩
      · rw [Set.inter_eq_left.mpr (injI_subset_sumMasterI hX)]
        exact Or.inr ⟨i, X, hX, rfl⟩
      ·
        rcases hZ with rfl | ⟨k, Wk, hWk, rfl⟩
        · exact absurd (hsub none_mem_sumMasterI).1 none_not_mem_injI
        · obtain ⟨w, hw⟩ := hne k Wk hWk
          have hmemZ := hsub (some_mem_injI.mpr hw)
          obtain rfl := index_of_some_mem_injI hmemZ.1
          obtain rfl := index_of_some_mem_injI hmemZ.2
          rw [injI_inter_same] at hsub ⊢
          exact Or.inr ⟨k, X ∩ Y, (D k).inter_mem hX hY hWk (injI_subset_iff.mp hsub),
    rfl⟩

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

omit [Nonempty ι] in
theorem updTuple_apply_ne {i j : ι} (U : Set (α i)) (h : j = i) :
    updTuple D i U j = (D j).master := by
  simp [updTuple, Function.update_of_ne h]

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

theorem fromPi_mem_slice (x : ∀ i, (D i).Element) (i : ι) (U : Set (α i)) :
    (fromPi D x).mem (slice D i U) ↔ (x i).mem U := by
  constructor
  · rintro ⟨X, hX, _, heq⟩
    have hXeq : updTuple D i U = X := iprodNbhd_injective heq
    have hi := hX i
    rwa [← hXeq, updTuple_apply_self] at hi
  · intro hU
    refine ⟨updTuple D i U, fun j => ?_, ⟨[i], fun j hj => ?_⟩, rfl⟩
    · by_cases h : j = i
      · subst h; rw [updTuple_apply_self]; exact hU
      · rw [updTuple_apply_ne U h]; exact (x j).master_mem
    · exact updTuple_apply_ne U fun h => hj (List.mem_singleton.mpr h)

end Iso
```

```
end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

omit [Nonempty ι] in
theorem iprodNbhd_restrictTo_cons {X : ∀ i, Set (α i)} (hXsub : ∀ i, X i ⊆ (D i).
    master)

    (a : ι) (l : List ι) :
    iprodNbhd (restrictTo D (a :: l) X) = slice D a (X a) ∩ iprodNbhd (restrictTo D l
    X) := by
  rw [slice_eq, iprodNbhd_inter]
  congr 1; funext j
  show (if j ∈ a :: l then X j else (D j).master)
      = updTuple D a (X a) j ∩ (if j ∈ l then X j else (D j).master)
  by_cases hja : j = a
  · subst hja
    rw [updTuple_apply_self]
    by_cases hjl : j ∈ l
    · rw [ite_eq_left List.mem_cons_self, ite_eq_left hjl, Set.inter_self]
    · rw [ite_eq_left List.mem_cons_self, ite_eq_right hjl, Set.inter_eq_left.mpr (
    hXsub j)]
  · rw [updTuple_apply_ne (X a) hja]
    by_cases hjl : j ∈ l
    · rw [ite_eq_left (List.mem_cons_of_mem a hjl), ite_eq_left hjl, Set.inter_eq_right
    .mpr (hXsub j)]
    · rw [ite_eq_right fun h => (List.mem_cons.mp h).elim hja hjl, ite_eq_right hjl,
    Set.inter_self]

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

omit [Nonempty ι] in

theorem iprodNbhd_subset_slice {X : ∀ i, Set (α i)} (hX : ∀ i, (D i).mem (X i)) (i : ι)
     :
    iprodNbhd X ⊆ slice D i (X i) := by
  rw [slice_eq, iprodNbhd_subset_iff]
  intro j
  by_cases h : j = i
  · subst h; rw [updTuple_apply_self]
```

```
  · rw [updTuple_apply_ne (X i) h]; exact (D j).sub_master (hX j)

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

theorem iprod_mem_slice {i : ι} {U : Set (α i)} (hU : (D i).mem U) :
    (iprod D).mem (slice D i U) := by
  refine ⟨updTuple D i U, fun j => ?_, ⟨[i], fun j hj => ?_⟩, rfl⟩
  · by_cases h : j = i
    · subst h; rw [updTuple_apply_self]; exact hU
    · rw [updTuple_apply_ne U h]; exact (D j).master_mem
  · exact updTuple_apply_ne U fun h => hj (List.mem_singleton.mpr h)

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

omit [Nonempty ι] in

theorem slice_inter (i : ι) (U U' : Set (α i)) :
    slice D i U ∩ slice D i U' = slice D i (U ∩ U') := by
  rw [slice_eq, slice_eq, slice_eq, iprodNbhd_inter]
  congr 1; funext j
  by_cases h : j = i
  · subst h; rw [updTuple_apply_self, updTuple_apply_self, updTuple_apply_self]
  · rw [updTuple_apply_ne U h, updTuple_apply_ne U' h, updTuple_apply_ne (U ∩ U') h,
      Set.inter_self]

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]
```

```
omit [Nonempty ι] in

theorem slice_subset (i : ι) {U U' : Set (α i)} (hUU' : U ⊆ U') : slice D i U ⊆ slice
    D i U' := by
  rw [slice_eq, slice_eq, iprodNbhd_subset_iff]
  intro j
  by_cases h : j = i
  · subst h; rw [updTuple_apply_self, updTuple_apply_self]; exact hUU'
  · rw [updTuple_apply_ne U h, updTuple_apply_ne U' h]

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

def proj (z : (iprod D).Element) (i : ι) : (D i).Element where
  mem U := (D i).mem U ∧ z.mem (slice D i U)
  sub h := h.1
  master_mem := ⟨(D i).master_mem, by
    have : slice D i (D i).master = (iprod D).master := by
      show iprodNbhd (updTuple D i (D i).master) = iprodNbhd (fun j => (D j).master)
      congr 1; funext j
      by_cases h : j = i
      · subst h; rw [updTuple_apply_self]
      · rw [updTuple_apply_ne (D i).master h]
    rw [this]; exact z.master_mem⟩
  inter_mem := by
    rintro U U' ⟨_, hzU⟩ ⟨_, hzU'⟩
    have hz := z.inter_mem hzU hzU'
    rw [slice_inter] at hz
    exact ⟨iprod_mem_slice_inv (z.sub hz), hz⟩
  up_mem := by
    rintro U U' ⟨_, hzU⟩ hU' hUU'
    exact ⟨hU', z.up_mem hzU (iprod_mem_slice hU') (slice_subset i hUU')⟩

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

theorem proj_fromPi (x : ∀ i, (D i).Element) (i : ι) : proj (fromPi D x) i = x i := by
```

```
  apply Element.ext
  intro U
  constructor
  · rintro ⟨_, hz⟩; exact (fromPi_mem_slice x i U).mp hz
  · intro hU; exact ⟨(x i).sub hU, (fromPi_mem_slice x i U).mpr hU⟩

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

theorem z_mem_iprodNbhd_restrictTo (z : (iprod D).Element) {X : ∀ i, Set (α i)}
    (hXsub : ∀ i, X i ⊆ (D i).master) (hslice : ∀ i, z.mem (slice D i (X i))) (l :
    List ι) :
    z.mem (iprodNbhd (restrictTo D l X)) := by
  induction l with
  | nil =>
    have : restrictTo D [] X = fun j => (D j).master := by
      funext j; rw [restrictTo]; exact ite_eq_right List.not_mem_nil
    rw [this]; exact z.master_mem
  | cons a l ih =>
    rw [iprodNbhd_restrictTo_cons hXsub a l]
    exact z.inter_mem (hslice a) ih

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

theorem z_mem_of_slices (z : (iprod D).Element) {X : ∀ i, Set (α i)}
    (hXmem : ∀ i, (D i).mem (X i)) (hXf : FinSupp D X)
    (hslice : ∀ i, z.mem (slice D i (X i))) : z.mem (iprodNbhd X) := by
  obtain ⟨l, hl⟩ := hXf
  have hsub : ∀ i, X i ⊆ (D i).master := fun i => (D i).sub_master (hXmem i)
  have key := z_mem_iprodNbhd_restrictTo z hsub hslice l
  have heq : restrictTo D l X = X := by
    funext j
    rw [restrictTo]
    by_cases h : j ∈ l
    · rw [ite_eq_left h]
    · rw [ite_eq_right h]; exact (hl j h).symm
  rwa [heq] at key
```

```
end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

theorem fromPi_toPi (z : (iprod D).Element) : fromPi D (fun i => proj z i) = z := by
  apply Element.ext
  intro W
  constructor
  · rintro ⟨X, hX, hXf, rfl⟩
    exact z_mem_of_slices z (fun i => (hX i).1) hXf (fun i => (hX i).2)
  · intro hW
    obtain ⟨X, hXmem, hXf, rfl⟩ := z.sub hW
    exact ⟨X, fun i => ⟨hXmem i, z.up_mem hW (iprod_mem_slice (hXmem i))
      (iprodNbhd_subset_slice hXmem i)⟩, hXf, rfl⟩

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ι]

section Iso

variable [DecidableEq ι]

def iprodEquiv (D : ∀ i, NeighborhoodSystem (α i)) :
    (iprod D).Element ≃o (∀ i, (D i).Element) where
  toFun z := fun i => proj z i
  invFun x := fromPi D x
  left_inv z := fromPi_toPi z
  right_inv x := funext fun i => proj_fromPi x i
  map_rel_iff' := by
    intro z z'
    constructor
    · intro h W hzW
      obtain ⟨X, hXmem, hXf, rfl⟩ := z.sub hzW
      refine z_mem_of_slices z' hXmem hXf fun i => ?_
      have hzi : (proj z i).mem (X i) :=
        ⟨hXmem i, z.up_mem hzW (iprod_mem_slice (hXmem i)) (iprodNbhd_subset_slice
    hXmem i)⟩
      exact ((h i) (X i) hzi).2
    · intro h i U hU
      exact ⟨hU.1, h _ hU.2⟩

end Iso
```

```
end IndexedProduct

theorem injI_inter_ne {i j : ι} (h : i = j) (X : Set (α i)) (Y : Set (α j)) :
    injI i X ∩ injI j Y = ∅ := by
  apply Set.eq_empty_iff_forall_notMem.mpr
  rintro (_ | ⟨k, a⟩) ⟨hw1, hw2⟩
  · exact none_not_mem_injI hw1
  · exact h ((index_of_some_mem_injI hw1).symm.trans (index_of_some_mem_injI hw2))

theorem injI_nonempty {i : ι} {X : Set (α i)} (hX : X.Nonempty) : (injI i X).Nonempty
    := by
  obtain ⟨a, ha⟩ := hX; exact ⟨some ⟨i, a⟩, some_mem_injI.mpr ha⟩

theorem iotimes_only_master [Infinite ι] (D : ∀ i, NeighborhoodSystem (α i)) :
    ∀ W, (iotimes D).mem W → W = (iotimes D).master := by
  rintro W (rfl | ⟨X, _, hXne, ⟨l, hl⟩, rfl⟩)
  · rfl
  ·

    refine absurd ((List.finite_toSet l).subset (fun i _ => ?_)) Set.infinite_univ
    exact not_not.mp fun hni => hXne i (hl i hni)

theorem iotimes_subsingleton [Infinite ι] (D : ∀ i, NeighborhoodSystem (α i)) :
    Subsingleton (iotimes D).Element := by
  refine ⟨fun x y => ?_⟩
  apply Element.ext
  intro W
  constructor
  · intro hx; rw [iotimes_only_master D W (x.sub hx)]; exact y.master_mem
  · intro hy; rw [iotimes_only_master D W (y.sub hy)]; exact x.master_mem

section IndexedProduct

variable [Nonempty ι]

theorem iprod_mem_iprodNbhd {X : ∀ i, Set (α i)} (hX : ∀ i, (D i).mem (X i))
    (hXf : FinSupp D X) : (iprod D).mem (iprodNbhd X) :=
  ⟨X, hX, hXf, rfl⟩

end IndexedProduct

theorem isum_summand_unique {hne : ∀ i, ∀ X : Set (α i), (D i).mem X → X.Nonempty}
    (z : (isum D hne).Element) {i j : ι} {X : Set (α i)} {Y : Set (α j)}
    (hX : z.mem (injI i X)) (hY : z.mem (injI j Y)) : i = j := by

  have hz := z.inter_mem hX hY
  rcases z.sub hz with h0 | ⟨k, W, hWmem, hWeq⟩
  · rw [Set.ext_iff] at h0
    exact absurd ((h0 none).mpr none_mem_sumMasterI).1 none_not_mem_injI
  · obtain ⟨w, hw⟩ := hne k W hWmem
    rw [Set.ext_iff] at hWeq
    have hmem := (hWeq (some ⟨k, w⟩)).mpr (some_mem_injI.mpr hw)
    obtain rfl := index_of_some_mem_injI hmem.1
    obtain rfl := index_of_some_mem_injI hmem.2
    rfl
```

```
theorem isum_trichotomy {hne : ∀ i, ∀ X : Set (α i), (D i).mem X → X.Nonempty}
    (z : (isum D hne).Element) :
    (∃ i, ∃ X : Set (α i), (D i).mem X ∧ z.mem (injI i X)) ∨
      (∀ W, z.mem W → W = (isum D hne).master) := by
  by_cases h : ∃ i, ∃ X : Set (α i), (D i).mem X ∧ z.mem (injI i X)
  · exact Or.inl h
  · refine Or.inr fun W hW => ?_
    rcases z.sub hW with rfl | ⟨i, X, hX, rfl⟩
    · rfl
    · exact absurd ⟨i, X, hX, hW⟩ h

end InfSumProd
end

end DomainTheory.NeighbourhoodSystems
```

4.7.64 Exercise 6.29 — infinitary sum and product

Generalize $+$ and $\times$ to infinitary operations on domains:

$$\sum_{n=0}^{\infty} D_n \quad \text{and} \quad \prod_{n=0}^{\infty} D_n.$$

Would a similar generalization be possible for $\oplus$ and $\otimes$?

Exercise_6_29.lean

Exercise_6_29.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.infSumProd

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem
open DomainTheory.NeighbourhoodSystems.InfSumProd
theorem example_629 :
    (∀ {ι : Type*} {α : ι → Type*} [Infinite ι] (D : ∀ i, NeighborhoodSystem (α i)) (W
    : Set (Σ i, α i)),
        (iotimes D).mem W → W = (iotimes D).master) ∧
    (∀ {ι : Type*} {α : ι → Type*} [Infinite ι] (D : ∀ i, NeighborhoodSystem (α i)),
      Subsingleton (iotimes D).Element) ∧
    (∀ {ι : Type*} {α : ι → Type*} {D : ∀ i, NeighborhoodSystem (α i)}
        {hne : ∀ i, ∀ X : Set (α i), (D i).mem X → X.Nonempty} (z : (isum D hne).
    Element)
        {i j : ι} {X : Set (α i)} {Y : Set (α j)},
      z.mem (injI i X) → z.mem (injI j Y) → i = j) ∧
    (∀ {ι : Type*} {α : ι → Type*} {D : ∀ i, NeighborhoodSystem (α i)}
        {hne : ∀ i, ∀ X : Set (α i), (D i).mem X → X.Nonempty} (z : (isum D hne).
    Element),
      (∃ i, ∃ X : Set (α i), (D i).mem X ∧ z.mem (injI i X)) ∨
        (∀ W, z.mem W → W = (isum D hne).master)) :=
```

⟨@iotimes_only_master, @iotimes_subsingleton, @isum_summand_unique, @isum_trichotomy⟩

4.8 Computability in effectively given domains

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>



Figure 52: Diagram 17

4.8.1 Introduction

For the domain N the strict functions from N into N, the strict maps $f : N \to N$, correspond exactly to the partial functions $g : N \to N$ (as we wrote in 5.6 we had $f = \bar{g}$). For such functions there is a standard theory of computability: g is called computable if it can be defined as a partial recursive function with its "program" written down in a certain standard form. The non-strict maps $h : N \to N$ are all constant, and so are intuitively computable; so we know all about computable maps in $|N \to N|$ in general. The question is: what are the computable maps on (elements of) other domains?

The answer will of course depend on how the domain is presented to us. Even with N, there are continuum many isomorphisms $\pi : N \to N$ of N onto itself, not all of which can be computable. That is, if we permute N and, so to speak, present the integers in a different order, then a well-behaved computable function $f : N \to N$ may well be transformed into a non-computable function,

$$\pi \circ f \circ \pi^{-1} : N \to N.$$

(Hint: Consider the characteristic function e of the even numbers. Take $f = \bar{e}$ and let π be very horrid.) The reason we imagined we knew which were the computable $f : N \to N$ is that N is always thought of in a standard presentation. We must thus define "in general" a concept of an *effectively given domain*, that is to say, one with a sufficiently computable presentation to represent the additional knowledge about the domain.

The main idea will be that the *finite elements* of $|D|$ should be regarded as the ones initially known. Abstractly, to know a finite element is to know how it is *related* to other finite elements.

Of course, this will mean that we will allow at most a countable infinity of finite elements — but this restriction well accords with intuition. To make precise the terminology "related to" it proves most convenient to go back to the neighbourhoods (in any case they are in a one-one correspondence with the finite elements).

4.8.2 Recursive predicates

Recursively enumerable and recursively decidable predicates on the natural numbers, together with the primitive-recursive coding of lists used to present neighbourhoods.

Theorem (`amGM`). For natural numbers, four times their product is at most the square of their sum.

Proof. Expanding the square reduces the inequality to nonnegativity of the square of the difference of the two numbers. ■

Theorem (`iter_sq_le`). Every value returned by the square-root iteration has square at most the input.

Proof. Induction over the square-root iteration uses the arithmetic–geometric mean inequality to show that each positive update remains below the input after squaring; the zero branch is immediate. ■

Theorem (`lt_of_mul_lt_mul_left'`). A strict inequality after multiplication by a common natural factor implies the corresponding strict inequality of the other factors.

Proof. A strict product inequality forces the common factor to be positive; cancellation of that positive factor then gives the strict inequality between the remaining factors. ■

Theorem (`lt_iter_succ_sq`). If the input lies below the square of one more than an initial guess, it also lies below the square of one more than the resulting square-root iterate.

Proof. Induction on the iteration compares the arithmetic-mean update with the preceding guess; the arithmetic–geometric mean estimate and cancellation of a positive factor preserve the strict upper-square bound. ■

Theorem (`lt_succ_sqrt`). Every natural number is strictly below the square of one more than its square root.

Proof. The square root satisfies the lower-square bound, and the next consecutive square is therefore strictly above the input by the interval characterization. ■

Theorem (`sqrt_le`). The square of the natural square root of a number is at most that number.

Proof. The square-root iteration already has square at most the input, and the natural square root is the iteration begun from the input itself. ■

Theorem (`sqrt_eq_of`). If a number lies between two consecutive squares beginning at the square of a given integer, its natural square root is that integer.

Proof. The two hypotheses place the number in the unique interval from one square up to the next consecutive square. ■

Theorem (`sqrt_add_eq`). Adding at most twice an integer to its square does not change its natural square root.

Proof. The hypothesis places the added term below twice the base, so the result lies from the base square up to, but not including, the next square; the square-root interval characterization applies. ■

Theorem (`unpair_pair`). Unpairing a paired pair of natural numbers recovers the original ordered pair.

Proof. Substitution of the explicit pairing formula identifies its square-root block; the resulting quotient and offset calculations recover both entries. ■

Theorem (`unpair_pair_fst`). The first component obtained by unpairing a paired pair is its first entry.

Proof. Taking first components in the pairing inverse identity recovers the first entry. ■

Theorem (`unpair_pair_snd`). The second component obtained by unpairing a paired pair is its second entry.

Proof. Taking second components in the pairing inverse identity recovers the second entry. ■

Definition (`selectFn`). This selector returns its first value when its numerical control is one and its second value when the control is zero.

Theorem (`selectFn_one`). A selector controlled by one returns its first value.

Proof. Substitution of control one cancels the second summand and leaves the first value. ■

Theorem (`selectFn_zero`). A selector controlled by zero returns its second value.

Proof. Substitution of control zero cancels the first summand and leaves the second value. ■

Theorem (`selectFn_ite`). A zero-one control obtained from a proposition makes numerical selection agree with ordinary case distinction.

Proof. In the true case the Boolean control is one and in the false case it is zero; the two selector equations settle the cases. ■

Definition (`RecDecidable`). A predicate on natural numbers is recursively decidable when a primitive-recursive characteristic function takes the value one exactly on its true instances.

Definition (`REPred`). A predicate on natural numbers is recursively enumerable when membership is the existence of a numerical witness satisfying a recursively decidable relation.

Definition (`REPred2`). A binary predicate is recursively enumerable when its pullback along the fixed pairing of natural numbers is recursively enumerable.

Definition (`RecDecidable2`). A binary predicate is recursively decidable when its pullback along the fixed pairing of natural numbers is recursively decidable.

Definition (`RecDecidable3`). A ternary predicate is recursively decidable when the predicate obtained by nested pairing is recursively decidable.

Definition (`encodeList`). The empty list is coded by zero, and a nonempty list is coded by pairing its head with the code of its tail and then taking the successor.

Definition (`foldStep`). A coded fold step either leaves a terminal state fixed or removes the next coded list entry and updates the accumulator with the given step operation.

Definition (`foldCode`). The fold of a coded list iterates the coded fold step for a bounded number of stages and returns the resulting accumulator.

Definition (`halfParity`). This primitive recursion simultaneously records the quotient and remainder of a natural number on division by two.

Definition (`isOne`). This zero-one characteristic function has value one exactly at the natural number one.

Theorem (`isOne_eq_one_iff`). The characteristic of one has value one exactly at the input one.

Proof. Expanding the two truncated differences shows that their sum vanishes exactly when the input is neither below nor above one. ■

Theorem (`isOne_one`). The characteristic of one evaluates to one at one.

Proof. Substituting one makes both truncated differences zero, leaving one. ■

Theorem (`isOne_zero`). The characteristic of one evaluates to zero at zero.

Proof. Substituting zero makes the second truncated difference one, which is subtracted from one. ■

Definition (`allListStp`). This fold step preserves a success flag precisely when the current coded list entry satisfies the given characteristic function.

Definition (`allListChar`). This characteristic function tests whether every entry of a coded list satisfies a given numerical predicate.

Definition (`bExistsFn`). This bounded search characteristic records whether some index below the bound satisfies the given numerical predicate.

Definition (`bForallFn`). This bounded search characteristic records whether every index below the bound satisfies the given numerical predicate.

Definition (`existsListStp`). This fold step records whether the current coded list entry or an earlier entry satisfies the given characteristic function.

Definition (`existsListChar`). This characteristic function tests whether some entry of a coded list satisfies a given numerical predicate.

Definition (`isBinDigit`). This characteristic function recognizes exactly the two binary digits zero and one.

Definition (`allBinDigitsChar`). This characteristic function tests whether every entry of a coded list is a binary digit.

Definition (`isZero`). This zero-one characteristic function has value one exactly at zero.

Theorem (`le_pair_right`). Each component of a paired natural number is at most the pair code.

Proof. Expanding the pairing formula leaves the second component plus nonnegative terms. ■

Definition (`listLenStp`). This fold step increments the accumulated length by one.

Definition (`listLenChar`). This numerical function computes the length of the list represented by a code.

Definition (`lowOr`). This operation computes the lowest binary digit of the bitwise disjunction of two natural numbers.

Definition (`lorStep`). This state transition removes the lowest bits of two numbers while accumulating their bitwise disjunction.

Definition (`mapPairStep`). This operation adjoins the pair formed from a fixed parameter and a list entry to an accumulated list code.

Definition (`mapPairStp`). This packed fold step applies the operation that pairs every list entry with a fixed parameter.

Definition (`mapPairCode`). This operation codes the list obtained by pairing a fixed natural number with every entry of a coded list.

Definition (`mulBit`). This operation combines two zero-one values by multiplication, thereby representing conjunction.

Definition (`decodeFuelOkCharBody`). This one-stage characteristic interprets a tagged code using previously computed values for its recursive subcodes.

Definition (`decodeFuelOkChar`). This fuel-bounded characteristic evaluates well-formedness of coded expressions by recursion on the available fuel.

Definition (`myLor`). This operation computes binary disjunction by repeatedly consuming the low bits of its two arguments.

Definition (`natEqChar`). This zero-one characteristic function recognizes equality of two natural numbers.

Theorem (`natEqChar_eq_one_iff`). The natural-number equality characteristic has value one exactly when its two arguments are equal.

Proof. The two opposite truncated differences vanish simultaneously exactly when the arguments are equal. ■

Definition (`codeBound`). This recursively defined bound dominates every code of a sufficiently short list of binary digits.

Definition (`listEqStpNonzero`). This comparison step consumes one entry from a nonempty remaining list code and preserves the equality flag exactly when the entries agree.

Definition (`listEqStp`). This comparison step handles the empty-list case and otherwise advances the coded list comparison.

Definition (`listEqChar`). This characteristic function tests equality of the two lists represented by given codes.

Definition (`listEqStep`). This unpacked comparison step applies one coded-list equality update to a state and a new entry.

Definition (`nthStp`). This fold step counts list positions and retains the entry at the requested index, using a prescribed default if none is found.

Definition (`nthCode`). This operation returns the requested entry of a coded list, or a prescribed default beyond its length.

Definition (`appendListTabFn`). This table entry function reads first from one coded list and then from a second, with the split determined by the first length.

Definition (`decodeFuelOkCharLookup`). This table-based evaluator performs one tagged well-formedness step using prior values stored in a coded table.

Definition (`dropListTabFn`). This table entry function reads a coded list after shifting its index by the number of entries to discard.

Definition (`reForallF`). This update consumes one witness code and preserves a success flag when the corresponding decidable relation holds.

Definition (`reForallStp`). This packed update advances the witness-list test for universal membership in a recursively enumerable predicate.

Definition (`reForallChar`). This characteristic checks a list of proposed witnesses against the corresponding entries of another coded list.

Theorem (`sqrt_le_add`). Every natural number is at most the square of its natural square root plus twice that square root.

Proof. The input is below the next square, whose expansion is the square-root square plus twice the square root plus one; natural-number discreteness removes the final one. ■

Theorem (`pair_unpair`). Pairing the two components obtained by unpairing a natural number recovers that number.

Proof. The square-root bounds locate the number in its triangular pairing block; substituting the resulting quotient and offset reconstructs it. ■

Definition (`tabStep`). This recursion step adjoins the next value of a finite numerical table to its code.

Definition (`tabCode`). This operation codes the finite table of values of a numerical function below a given bound.

Definition (`appendListCode`). This operation produces a code for the concatenation of two coded lists.

Definition (`dropCode`). This operation produces a code for the list obtained after deleting a prescribed initial segment.

Definition (`fuelTableStep`). This operation constructs the next finite table of fuel-bounded values from the preceding table.

Definition (`fuelTable`). This primitive recursion builds the finite table of all fuel-bounded values below a prescribed code bound.

Definition (`takeListTabFn`). This table entry function reads an entry of a coded list at the table index.

Definition (`takeCode`). This operation produces a code for the prescribed initial segment of a coded list.

Theorem (`unpair_snd_le`). The second component obtained by unpairing a natural number is at most that number.

Proof. The second component is bounded by its reconstructed pair, and pairing the two unpaired components returns the original number. ■

Definition (`decodeList`). Zero decodes to the empty list, while a successor code decodes by unpairing its predecessor into the head and the code of the tail.

Theorem (`primrec_id`). The numerical function defining id is primitive recursive.

Proof. The identity is the first projection after pairing the input with zero, hence is primitive recursive. ■

Theorem (`rec_add`). The displayed primitive recursion computes add on natural numbers.

Proof. Induction on the second argument matches zero with the initial value and each successor with one further increment. ■

Theorem (`rec_mul`). The displayed primitive recursion computes mul on natural numbers.

Proof. Induction on the second argument turns the successor equation into one further addition, matching multiplication by a successor. ■

Theorem (`primrec_add`). The numerical function defining add is primitive recursive.

Proof. Primitive recursion from the first input, adding one at each successor of the second, computes addition. ■

Theorem (`primrec_mul`). The numerical function defining mul is primitive recursive.

Proof. The mul construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`rec_pred`). The displayed primitive recursion computes pred on natural numbers.

Proof. The zero clause returns zero, while at a successor the recursion returns the preceding index. ■

Theorem (`primrec_pred`). The numerical function defining pred is primitive recursive.

Proof. Primitive recursion returns the predecessor stage at each successor and zero at the base. ■

Theorem (`rec_sub`). The displayed primitive recursion computes sub on natural numbers.

Proof. Induction on the second argument matches zero subtraction and one further predecessor at each successor. ■

Theorem (`primrec_sub`). The numerical function defining sub is primitive recursive.

Proof. Primitive recursion from the first input applies predecessor once for each unit of the second input. ■

Theorem (`nat_mul_eq_one`). A product of natural numbers is one exactly when both factors are one.

Proof. Unique factorization of one in the natural numbers forces each factor to be one, and the converse is immediate by multiplication. ■

Theorem (`primrec_add2`). The numerical function defining add2 is primitive recursive.

Proof. The add2 construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_mul2`). The numerical function defining mul2 is primitive recursive.

Proof. The mul2 construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_sub2`). The numerical function defining sub2 is primitive recursive.

Proof. The sub2 construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_selectFn`). The numerical function defining selectFn is primitive recursive.

Proof. The select function construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`RecDecidable.of_iff`). A predicate pointwise equivalent to a recursively decidable predicate is recursively decidable.

Proof. Use the same primitive-recursive characteristic and replace its truth specification by the given pointwise equivalence. ■

Theorem (`RecDecidable.comp`). The pullback of a recursively decidable predicate along a primitive-recursive function is recursively decidable.

Proof. Compose the witnessing zero–one characteristic with the primitive-recursive input function. ■

Theorem (`RecDecidable.and`). The conjunction of two recursively decidable predicates is recursively decidable.

Proof. Multiply the two zero–one characteristic functions, so the result is one exactly when both original predicates hold. ■

Theorem (`RecDecidable.natEq`). Equality of the values of two primitive-recursive natural-number functions is recursively decidable.

Proof. Compare the two primitive-recursive values in both directions by truncated subtraction; both differences vanish exactly when the values are equal. ■

Theorem (`RecDecidable.not`). The complement of a recursively decidable predicate is recursively decidable.

Proof. Subtract the zero–one characteristic from one; this exchanges its accepting and rejecting values. ■

Theorem (`RecDecidable.em`). Every instance of a recursively decidable predicate either holds or fails.

Proof. The witnessing characteristic has a definite natural value; its specification decides whether the predicate holds. ■

Theorem (`RecDecidable.or`). The disjunction of two recursively decidable predicates is recursively decidable.

Proof. Use the zero–one test of the sum of the two characteristics; the sum is positive exactly when at least one predicate holds. ■

Theorem (`RecDecidable.re`). Every recursively decidable predicate is recursively enumerable.

Proof. Introduce a dummy existential witness and use the original decidable predicate as the witness relation. ■

Theorem (`RecDecidable2.re`). Every recursively decidable binary relation is recursively enumerable.

Proof. Apply the decidable-to-enumerable conversion after encoding the two arguments by pairing. ■

Theorem (`REPred.of_iff`). A predicate pointwise equivalent to a recursively enumerable predicate is recursively enumerable.

Proof. Keep the same decidable witness relation and transport its existential characterization through the pointwise equivalence. ■

Theorem (`REPred.comp`). The pullback of a recursively enumerable predicate along a primitive-recursive function is recursively enumerable.

Proof. Apply the primitive-recursive input map to the object coordinate while retaining the existential witness coordinate. ■

Theorem (`REPred.and`). The conjunction of two recursively enumerable predicates is recursively enumerable.

Proof. Multiply the two zero–one characteristic functions, so the result is one exactly when both original predicates hold. ■

Theorem (`REPred.proj`). Existential projection of a recursively enumerable predicate along the first paired coordinate is recursively enumerable.

Proof. Pair the input with an existential witness and absorb the witness into the existential quantifier already present in recursive enumerability. ■

Theorem (`REPred.or`). The disjunction of two recursively enumerable predicates is recursively enumerable.

Proof. Combine the two zero–one characteristics by bounded addition, which is positive exactly when at least one predicate holds. ■

Theorem (`le_pair_left`). Each component of a paired natural number is at most the pair code.

Proof. Expanding the pairing formula leaves the first component plus nonnegative terms. ■

Theorem (`encodeList_length_le`). The length of a finite list is at most its natural-number code.

Proof. Induction on the list uses the tail bound and the fact that a pair code dominates its second component. ■

Theorem (`foldStep_zero`). The coded finite fold agrees with repeated application of its step function to the decoded list.

Proof. A zero remaining code activates the terminal branch, which preserves the packed accumulator state. ■

Theorem (`foldStep_pos`). The coded finite fold agrees with repeated application of its step function to the decoded list.

Proof. For a successor list code, subtracting one exposes its paired head and tail; the nonempty branch advances the tail and updates the accumulator. ■

Theorem (`rec_const_iterate`). The displayed primitive recursion computes const iterate on natural numbers.

Proof. Induction on the iteration count identifies the base state and one further application of the function. ■

Theorem (`foldStep_iterate`). The coded finite fold agrees with repeated application of its step function to the decoded list.

Proof. Induction on the fuel follows the empty-list fixed point or consumes one encoded head; the length bound guarantees enough fuel for the tail. ■

Theorem (`foldCode_eq`). The coded finite fold agrees with repeated application of its step function to the decoded list.

Proof. The code supplies at least the decoded list length as fuel, so the fold-step iteration theorem yields the ordinary left fold. ■

Theorem (`unpair_left_le`). The first component obtained by unpairing a natural number is at most that number.

Proof. The first component is bounded by its pair code, and pairing the two unpaired components reconstructs the original number. ■

Theorem (`pair_le_pair_left`). Increasing the first component while fixing the second cannot decrease the pair code.

Proof. Strict monotonicity of pairing in the first coordinate handles the strict case, while equality is immediate. ■

Theorem (`pair_le_pair_right'`). Increasing the second component while fixing the first cannot decrease the pair code.

Proof. Strict monotonicity of pairing in the second coordinate handles the strict case, while equality is immediate. ■

Theorem (`pair_le_pair`). Increasing both components cannot decrease the pair code.

Proof. First increase the first coordinate and then the second, composing the two coordinatewise monotonicity results. ■

Theorem (`decodeList_zero`). Zero decodes to the empty list.

Proof. The zero clause of decoding is the empty list. ■

Theorem (`decodeList_succ`). A successor code decodes to the first unpaired component of its predecessor followed by the decoding of the second component.

Proof. The successor clause unpairs the predecessor into the head and recursively decoded tail. ■

Theorem (`encodeList_decodeList`). Encoding the list decoded from a natural number recovers that number.

Proof. Strong induction on the code uses the strictly smaller tail component and reconstruction by pairing. ■

Theorem (`foldCode_eq'`). The coded finite fold agrees with repeated application of its step function to the decoded list.

Proof. Encode the decoded list without changing the code, then apply the coded-fold theorem to that list. ■

Theorem (`primrec_foldStepPacked`). The numerical function defining foldStepPacked is primitive recursive.

Proof. The foldStepPacked construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_foldCode`). The numerical function defining foldCode is primitive recursive.

Proof. The foldCode construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`isOne_le_one`). The characteristic of one is always at most one.

Proof. Its definition subtracts a natural number from one, so truncated subtraction cannot exceed one. ■

Theorem (`isOne_of_ne_one`). At every input other than one, the characteristic of one is zero.

Proof. If the input is below or above one, one of the two opposite truncated differences is positive, forcing the final subtraction to zero. ■

Theorem (`primrec_isOne`). The numerical function defining isOne is primitive recursive.

Proof. The isOne construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`isZero_eq_one_iff`). The zero characteristic has value one exactly at zero.

Proof. The minimum with one is zero exactly at zero; subtracting it from one gives the claimed truth condition. ■

Theorem (`isZero_le_one`). The zero characteristic is always at most one.

Proof. It is obtained by truncated subtraction from one, and therefore cannot exceed one. ■

Theorem (`primrec_isZero`). The numerical function defining isZero is primitive recursive.

Proof. The isZero construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_le`). The numerical function defining le is primitive recursive.

Proof. The le construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`bForallFn_le_one`). The bounded universal characteristic is always at most one.

Proof. Induction on the bound starts at one and each new selector returns either zero or a one-characteristic, both at most one. ■

Theorem (`bForallFn_eq_one_iff`). The bounded universal characteristic is one exactly when the tested function is one at every index below the bound.

Proof. Induction on the bound separates the previous indices from the new final index; the binary accumulator remains one precisely when both parts succeed. ■

Theorem (`bExistsFn_le_one`). The bounded existential characteristic is always at most one.

Proof. Induction on the bound observes that an already successful search stays at one, while an unsuccessful search takes the next one-characteristic, which is binary. ■

Theorem (`bExistsFn_congr`). Two bounded existential searches agree when their tested values agree at every index below the bound.

Proof. Induction on the bound uses agreement on all earlier indices for the recursive value and agreement at the new final index for the next update. ■

Theorem (`bExistsFn_eq_one_iff`). The bounded existential characteristic is one exactly when some index below the bound has tested value one.

Proof. Induction on the bound splits a witness below the previous bound from the newly added final index and follows the two selector cases. ■

Theorem (`primrec_bExistsFn`). The numerical function defining bExistsFn is primitive recursive.

Proof. The bExists function construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_bExistsFn_param`). The numerical function defining bExistsFn param is primitive recursive.

Proof. The bExists function param construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`decodeList_encodeList`). Decoding the code of a finite list recovers the list.

Proof. Induction on the list or code uses the successor coding equation and the fact that unpairing a freshly paired head and tail recovers both components. ■

Theorem (`reForallStp_eq`). Unpacking one universal-witness step yields the update that consumes the next witness and list entry.

Proof. Unpairing the current list entry, witness code and flag gives exactly the unpacked witness-checking update. ■

Theorem (`reForallChar_eq`). The coded universal-witness characteristic is the final flag of the corresponding fold over the decoded list.

Proof. The coded-fold equation turns the packed step into the unpacked witness-checking update over the decoded list. ■

Theorem (`reForallF_foldl_eq_one_iff`). The characteristic value is one exactly when every finite condition encoded in its arguments holds.

Proof. Induction on the candidate list tracks the binary flag and aligns each entry with the same-position witness obtained by decoding the witness code. ■

Theorem (`reForallChar_eq_one_iff`). The characteristic value is one exactly when every finite condition encoded in its arguments holds.

Proof. Specializing the fold characterization to initial flag one leaves exactly the pointwise witness checks below the decoded length. ■

Theorem (`primrec_reForallStp`). The numerical function defining reForallStp is primitive recursive.

Proof. The reForall step construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_reForallChar`). The numerical function defining reForallChar is primitive recursive.

Proof. The reForall characteristic construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`REPred.forall_mem_decodeList`). If a predicate is recursively enumerable, then the property that every member of a decoded finite list satisfies it is recursively enumerable.

Proof. Encode one witness for each list position into a witness list; the universal witness checker verifies all positions, and conversely each accepted position supplies a witness for the corresponding member. ■

Theorem (`lowOr_eq_mod`). The recursive binary-digit calculation agrees with ordinary quotient, remainder, or bitwise disjunction.

Proof. The four possible pairs of low bits agree with the low bit of binary disjunction. ■

Theorem (`lor_low_rec`). The recursive binary-digit calculation agrees with ordinary quotient, remainder, or bitwise disjunction.

Proof. Division by two shifts every test bit down one place, while the remainder formula restores the low disjunction bit. ■

Theorem (`lorStep_iter_spec`). After the specified number of binary-disjunction steps, both inputs have been divided by the corresponding power of two, the place value equals that power, and the accumulator plus the unprocessed high part equals the original disjunction.

Proof. Induction on the iteration count tracks the halved inputs, doubling place value, and accumulated low bits; the one-step binary recurrence preserves the invariant. ■

Theorem (`myLor_eq_lor`). The recursive binary-digit calculation agrees with ordinary quotient, remainder, or bitwise disjunction.

Proof. After enough halvings both remaining inputs are zero, so the iteration invariant identifies the accumulator with the full binary disjunction. ■

Theorem (`allListStp_eq`). Unpacking one universal-list step yields conjunction of the old flag with the current entry test.

Proof. Unpairing the packed entry, accumulator and parameter gives the selector that conjoins the old flag with the current test. ■

Theorem (`allListChar_eq`). The universal-list characteristic is the conjunction fold of the entry tests over the decoded list.

Proof. The coded-fold equation and the unpacked step equation identify the characteristic with the displayed conjunction fold. ■

Theorem (`allList_foldl_eq_one_iff`). The characteristic value is one exactly when every finite condition encoded in its arguments holds.

Proof. Induction on the list uses the binary accumulator: success after the head and tail means initial success and every entry test equals one. ■

Theorem (`allListChar_eq_one_iff`). The characteristic value is one exactly when every finite condition encoded in its arguments holds.

Proof. Apply the conjunction-fold characterization with initial accumulator one. ■

Theorem (`allListChar_le_one`). The characteristic function always takes a value at most one.

Proof. Induction on the decoded list preserves the bound because each update returns either zero or a one-characteristic. ■

Theorem (`primrec_allListStp`). The numerical function defining allListStp is primitive recursive.

Proof. The allList step construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`isBinDigit_eq_one_iff`). The characteristic value is one exactly when every finite condition encoded in its arguments holds.

Proof. The cases zero and one activate an accepting selector branch, while every value at least two reaches the rejecting branch. ■

Theorem (`primrec_isBinDigit`). The numerical function defining isBinDigit is primitive recursive.

Proof. The isBinDigit construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`allBinDigitsChar_eq_one_iff`). The characteristic value is one exactly when every finite condition encoded in its arguments holds.

Proof. The universal-list characterization says every decoded entry passes the binary-digit test, whose truth condition is being zero or one. ■

Theorem (`allBinDigitsChar_le_one`). The characteristic function always takes a value at most one.

Proof. This is the universal-list bound specialized to the binary-digit characteristic. ■

Theorem (`primrec_allBinDigitsChar`). The numerical function defining allBinDigitsChar is primitive recursive.

Proof. The allBinDigits characteristic construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`nthStp_val`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. Unpairing the state increments the running position and selects the current entry exactly when that position equals the requested index. ■

Theorem (`nthCode_foldl`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. Induction on the list compares the requested index with the running position, preserving the previous result before the position and replacing it exactly at equality. ■

Theorem (`nthCode_eq`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. The coded-fold equation starts the lookup fold at position zero, so the fold specification is ordinary list lookup with the given default. ■

Theorem (`primrec_nthStp`). The numerical function defining nthStp is primitive recursive.

Proof. The nth step construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_nthCode`). The numerical function defining nthCode is primitive recursive.

Proof. The nthCode construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`tabStep_val`). One table-building step adjoins the value at the next descending index to the accumulated table code.

Proof. Unpairing the bound, parameter, stage and prior code exposes the next descending table index and adjoins its value. ■

Theorem (`decodeList_tabCode_aux`). Decoding a partially built table yields the mapped interval of indices from the remaining bound through the processed entries.

Proof. Induction on the number of table entries shows that each step prepends the next descending index value to the already decoded suffix. ■

Theorem (`decodeList_tabCode`). Decoding a complete table code yields the values of the function on all indices below the bound.

Proof. Specialize the auxiliary decoding formula to the full bound, where the descending range becomes the ordinary initial range. ■

Theorem (`getD_eq_getElem_cf`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. Induction on the list and index reduces in-range defaulted lookup to ordinary indexed lookup. ■

Theorem (`getD_eq_default_cf`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. Induction on the list and index shows that an index at least the length repeatedly passes the head and eventually returns the default. ■

Theorem (`getD_map_range_cf`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. The mapped range has the prescribed length, and in-range lookup first selects its index from the range and then applies the map. ■

Theorem (`tabCode_nth_lt`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. Decode the table as the mapped initial range and apply the in-range lookup formula. ■

Theorem (`tabCode_nth_ge`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. Decode the table as a list of the bound length and apply the out-of-range default formula. ■

Theorem (`primrec_tabStep`). The numerical function defining tabStep is primitive recursive.

Proof. The tabStep construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_tabCode`). The numerical function defining tabCode is primitive recursive.

Proof. The tabCode construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_tagCase4`). The numerical function defining tagCase4 is primitive recursive.

Proof. The tagCase4 construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`mulBit_eq_one_iff`). The product of two binary characteristics is one exactly when both are one.

Proof. A product of natural numbers equals one only when both factors equal one, and the converse follows by multiplication. ■

Theorem (`mulBit_le_one`). The product of two characteristics bounded by one is itself bounded by one.

Proof. Each factor is zero or one; checking the four possible pairs gives a product no larger than one. ■

Theorem (`primrec_mulBit`). The numerical function defining mulBit is primitive recursive.

Proof. The mulBit construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`decodeFuelOkCharBody_eq`). The fuel-decoding body recognizes the four tags as the base, binary-list, conjunction and conjunction cases, and rejects every larger tag.

Proof. A case split on tags zero through three yields the base test, binary-list test and two recursive conjunction cases; all larger tags return zero. ■

Theorem (`primrec_decodeFuelOkCharBody`). The numerical function defining decodeFuelOkCharBody is primitive recursive.

Proof. The decodeFuelOk characteristicBody construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`decodeFuelOkCharBody_local`). The fuel-decoding body has the same value for two prior-stage functions that agree on every code no larger than the current code.

Proof. Only the two recursive subcodes can inspect the supplied functions, and both subcodes are no larger than the current code. ■

Theorem (`listLenStp_eq`). One packed length step increments the accumulated length by one.

Proof. Unpairing the packed state exposes the accumulated length, to which the step adds one. ■

Theorem (`listLenChar_eq`). The coded length function returns the length of the decoded list.

Proof. The coded fold becomes a fold that increments once per entry; induction on the decoded list shows that this fold counts its entries. ■

Theorem (`primrec_listLenStp`). The numerical function defining listLenStp is primitive recursive.

Proof. The listLen step construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_listLenChar`). The numerical function defining listLenChar is primitive recursive.

Proof. The listLen characteristic construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`natEqChar_le_one`). The natural-number equality characteristic is always at most one.

Proof. It is a truncated subtraction from one, so its value cannot exceed one. ■

Theorem (`natEqChar_eq_zero_or_one`). The natural-number equality characteristic is always zero or one.

Proof. The sum of opposite differences is either zero, leaving one, or positive, making the truncated subtraction zero. ■

Theorem (`primrec_natEqChar`). The numerical function defining natEqChar is primitive recursive.

Proof. The natEq characteristic construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`listEqStp_acc`). A packed list-comparison step fails on an empty remainder and otherwise compares the next entries while advancing the remainder.

Proof. Unpairing the state and separating whether the remaining code is zero gives exactly the failure branch and the nonempty comparison branch. ■

Theorem (`decodeList_eq_nil_iff`). A natural number decodes to the empty list exactly when it is zero.

Proof. Encoding both sides shows that an empty decoded list has code zero; the converse is the zero decoding equation. ■

Theorem (`selectFn_eq_one_iff_zero`). The characteristic value is one exactly when every finite condition encoded in its arguments holds.

Proof. The assumed binary control leaves the zero and one cases; direct evaluation gives conjunction in the one case and failure in the zero case. ■

Theorem (`listEqStep_zero`). Comparing another entry after the second coded list is exhausted produces a failed empty state.

Proof. The zero remainder activates the empty-remainder branch and returns the failed empty state. ■

Theorem (`listEqStep_succ`). With a nonempty second coded list, one comparison step combines the old flag with equality of the heads and advances to the tail code.

Proof. The zero test rejects the successor remainder; unpairing its predecessor supplies the next head and tail, and the equality characteristic updates the flag. ■

Theorem (`listEq_foldl_zero`). The coded finite fold agrees with repeated application of its step function to the decoded list.

Proof. After the second list is exhausted the next comparison sets the flag to zero, and induction shows that every remaining step preserves failure. ■

Theorem (`listEq_foldl_end_iff`). The coded finite fold agrees with repeated application of its step function to the decoded list.

Proof. Induction on the first list compares heads while both lists are nonempty, fails if the second ends early, and requires both successful flag and empty remainder at the end. ■

Theorem (`listEqChar_eq_one_iff`). The characteristic value is one exactly when every finite condition encoded in its arguments holds.

Proof. The final product is one exactly when the comparison flag is one and the remainder is empty; the fold characterization turns this into equality of decoded lists. ■

Theorem (`listEq_foldl_flag_le_one`). The characteristic function always takes a value at most one.

Proof. Induction on the first list preserves the binary flag because every comparison update uses binary selection and equality characteristic. ■

Theorem (`listEqChar_le_one`). The characteristic function always takes a value at most one.

Proof. Both the final comparison flag and empty-remainder test are at most one, so their product is at most one. ■

Theorem (`primrec_listEqStpNonzero`). The numerical function defining listEqStpNonzero is primitive recursive.

Proof. The listEq stepNonzero construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_listEqStp`). The numerical function defining listEqStp is primitive recursive.

Proof. The listEq step construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_listEqChar`). The numerical function defining listEqChar is primitive recursive.

Proof. The listEq characteristic construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`list_eq_of_getD`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. List extensionality uses equality of lengths and the assumed equality of defaulted lookups at every in-range position. ■

Theorem (`getD_take_cf`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. Below the take length, lookup in the prefix agrees with lookup in the original list; beyond it, both use the default. ■

Theorem (`getD_drop_cf`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. Lookup after dropping shifts the original index by the number discarded, with the default covering indices beyond the remaining suffix. ■

Theorem (`isZero_succ_sub_len1`). The zero test of one plus a truncated length difference records whether the first list is no longer than the second.

Proof. The truncated length difference is zero exactly when the first length does not exceed the second, and adding one makes the zero test reject precisely the opposite case. ■

Theorem (`isZero_succ_sub_len1_zero`). The same length test is zero precisely when the first list is strictly longer than the second.

Proof. Natural-number subtraction is positive exactly in the strict-length case, and the zero characteristic rejects every positive successor. ■

Theorem (`appendListTabFn_eq`). The coding operation agrees with concatenation or adjoining an entry to a finite list.

Proof. Below the first length the table reads the first list; at and above the split it reads the second list at the shifted index. ■

Theorem (`takeCode_eq`). Decoding the prefix code gives the requested initial segment of the decoded list.

Proof. Decode the generated table and compare each position with lookup in the original list to identify the requested prefix. ■

Theorem (`dropCode_eq`). Decoding the suffix code gives the decoded list after deleting the requested number of initial entries.

Proof. Decode the generated table and use shifted lookup to identify the suffix remaining after the requested deletion. ■

Theorem (`encodeList_take_le`). The code of an initial segment of a list is at most the code of the whole list.

Proof. A prefix is a sublist of the original list, and induction on the coding shows that encoding is monotone under removal of a tail. ■

Theorem (`encodeList_drop_le`). The code of a final segment of a list is at most the code of the whole list.

Proof. A suffix is obtained by repeated removal of heads, each of which cannot increase the list code. ■

Theorem (`takeCode_le`). A prefix code is at most the original list code.

Proof. Replace the generated code by the encoding of the decoded prefix and apply the prefix-code bound. ■

Theorem (`dropCode_le`). A suffix code is at most the original list code.

Proof. Replace the generated code by the encoding of the decoded suffix and apply the suffix-code bound. ■

Theorem (`primrec_takeListTabFn`). The numerical function defining takeListTabFn is primitive recursive.

Proof. The takeListTab function construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_min`). The numerical function defining min is primitive recursive.

Proof. The min construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_takeCode`). The numerical function defining takeCode is primitive recursive.

Proof. The takeCode construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_dropListTabFn`). The numerical function defining dropListTabFn is primitive recursive.

Proof. The dropListTab function construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_dropCode`). The numerical function defining dropCode is primitive recursive.

Proof. The dropCode construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`pair_lt_pair_zero_right`). Pairing zero with a strictly smaller second component gives a strictly smaller code.

Proof. Strict monotonicity of pairing in the second coordinate applies with first coordinate zero. ■

Theorem (`codeBound_ge`). The recursive code bound dominates every binary-list code of the prescribed length.

Proof. Induction on the length bounds the head binary digit and invokes the previous bound on the tail code before applying monotonicity of pairing. ■

Theorem (`RecDecidable2.swap`). Swapping the arguments of a recursively decidable binary relation preserves recursive decidability.

Proof. Precompose the paired characteristic with the primitive-recursive permutation that exchanges the two coordinates. ■

Theorem (`RecDecidable.bExists`). Bounded existential quantification of a recursively decidable relation is recursively decidable when the bound is primitive recursive.

Proof. A primitive-recursive bounded search accumulates disjunction of the characteristic values below the primitive-recursive bound. ■

Theorem (`RecDecidable.bForall`). Bounded universal quantification of a recursively decidable relation is recursively decidable when the bound is primitive recursive.

Proof. Encode the finitely many witnesses in one list, then use the decidable list checker to verify each corresponding entry; existential quantification over that witness list gives recursive enumerability. ■

Theorem (`RecDecidable.of_zero_one_char`). A primitive-recursive zero–one function that equals one exactly on a predicate makes that predicate recursively decidable.

Proof. Take the supplied primitive-recursive zero–one function as the characteristic; its assumed one-value condition is exactly the required specification. ■

Theorem (`RecDecidable2.of_paired_zero_one_char`). A primitive-recursive characteristic on paired arguments makes the corresponding binary relation recursively decidable.

Proof. Use the supplied primitive-recursive function directly as the characteristic after pairing the two arguments. ■

Theorem (`RecDecidable3.of_triple_zero_one_char`). A primitive-recursive characteristic on nested triples makes the corresponding ternary relation recursively decidable.

Proof. Use the supplied primitive-recursive function directly after encoding the three arguments by nested pairing. ■

Theorem (`codeBound_succ`). The code bound at a successor is obtained by pairing one with the previous bound and then taking the successor.

Proof. The successor equation is the defining recursive step of the code bound. ■

Theorem (`codeBound_zero`). The code bound at zero is zero.

Proof. The zero equation is the base clause of the code bound. ■

Theorem (`decodeFuelOkChar_le_one`). The characteristic function always takes a value at most one.

Proof. Induction on fuel separates the tag cases; constants and binary-list tests are bounded by one, while recursive conjunction multiplies two values already bounded by one. ■

Theorem (`decodeList_length_le`). The length of the list decoded from a natural number is at most that number.

Proof. Strong induction on the code removes the strictly smaller tail code and adds one entry, while the pairing bounds control the resulting length. ■

Theorem (`decodeList_mapPairStep`). Decoding one map-pair step prepends the pair of the fixed parameter and current entry to the decoded accumulator.

Proof. The adjoining decoding equation shows that one map step prepends the pair of the fixed parameter and current entry. ■

Theorem (`eqZero_bit`). The displayed binary control is one exactly in its zero case.

Proof. A value bounded by one is zero or one, and direct evaluation of the one-characteristic of its complement distinguishes the cases. ■

Theorem (`eq_of_le_one_iff_one`). For a natural number bounded by one, having one-characteristic equal to one is equivalent to being one.

Proof. The bound leaves only zero and one, and evaluation of the characteristic distinguishes those two values. ■

Theorem (`existsListStp_eq`). Unpacking one existential-list step yields disjunction of the old flag with the current entry test.

Proof. Unpairing the packed arguments yields the selector that preserves prior success or tests the current entry. ■

Theorem (`existsListChar_eq`). The existential-list characteristic is the disjunction fold of the entry tests over the decoded list.

Proof. The coded-fold equation and unpacked existential step identify the characteristic with a disjunction fold over the decoded list. ■

Theorem (`existsListChar_le_one`). The characteristic function always takes a value at most one.

Proof. Induction on the decoded list preserves binary values because a successful accumulator remains one and failure defers to the next one-characteristic. ■

Theorem (`existsList_foldl_eq_one_iff`). The characteristic value is one exactly when every finite condition encoded in its arguments holds.

Proof. Induction on the list splits a witness at the head from a witness in the tail while tracking whether success was already present. ■

Theorem (`existsListChar_eq_one_iff`). The characteristic value is one exactly when every finite condition encoded in its arguments holds.

Proof. Apply the existential-fold characterization with initial flag zero. ■

Theorem (`fuelTableStep_nth`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. Decode the newly built table and use in-range lookup to identify each entry with one application of the body to the corresponding prior-table lookup. ■

Theorem (`fuelTable_succ_nth`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. The successor recursion equation replaces the old table by one table-building step, so the preceding lookup formula applies. ■

Theorem (`fuelTable_zero_nth`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. At zero fuel the table code is zero, whose decoded list is empty, so every lookup returns the default. ■

Theorem (`fuelTable_eq_of_recursion`). Any table family with the same zero table and the same successor lookup recurrence as the fuel table agrees with it at every fuel and bounded index.

Proof. Induction on fuel uses the supplied zero table at the base and the supplied lookup recurrence to identify each successor table extensionally. ■

Theorem (`decodeFuelOkChar_fuelTable_eq`). Lookup in the fuel table equals the fuel-bounded decoding characteristic.

Proof. The fuel table satisfies the same base and successor lookup equations as fuel-bounded decoding, so uniqueness of that recursion identifies them. ■

Theorem (`geTwo_bit`). The displayed binary control recognizes values at least two.

Proof. Separate values below two from successors at least two; truncated subtraction and the one-characteristic give the corresponding binary values. ■

Theorem (`getD_append_cf`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. Split whether the index lies below the first length; concatenated lookup uses the first list there and shifted lookup in the second list otherwise. ■

Theorem (`getD_append_right_cf`). The coded lookup returns the corresponding table entry in range and the prescribed default outside the range.

Proof. An index at least the first length passes through the first list, and lookup continues in the second list at the shifted index. ■

Theorem (`appendListCode_eq`). The coding operation agrees with concatenation or adjoining an entry to a finite list.

Proof. Decode the generated table and split each lookup at the first length; the left and shifted-right formulas agree with concatenation. ■

Theorem (`halfParity_spec`). The recursive binary-digit calculation agrees with ordinary quotient, remainder, or bitwise disjunction.

Proof. Induction on the input follows the two-state parity recursion, alternating the remainder and incrementing the quotient every second step. ■

Theorem (`mapPairStp_eq`). Unpacking a coded map step adjoins the pair of the fixed parameter and current entry to the accumulated list code.

Proof. Unpairing the packed state recovers the fixed parameter, accumulator and current entry, after which one paired entry is adjoined. ■

Theorem (`decodeList_foldl_mapPairStp`). Decoding the map-pair fold gives the reversed mapped input followed by the decoded accumulator.

Proof. Induction on the input list uses the one-step decoding equation and records reversal because each mapped pair is prepended. ■

Theorem (`decodeList_mapPairCode`). Decoding a map-pair code gives the decoded input list with the fixed parameter paired to every entry.

Proof. The coded-fold equation and the map-step decoding formula show by induction that every decoded entry is paired with the fixed parameter. ■

Theorem (`primrec_allListChar`). The numerical function defining allListChar is primitive recursive.

Proof. The allList characteristic construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`RecDecidable2.bForallList`). Universal satisfaction of a recursively decidable binary relation over every entry of a decoded finite list is recursively decidable.

Proof. Encode the finitely many witnesses in one list, then use the decidable list checker to verify each corresponding entry; existential quantification over that witness list gives recursive enumerability. ■

Theorem (`primrec_appendListTabFn`). The numerical function defining appendListTabFn is primitive recursive.

Proof. The appendListTab function construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_appendListCode`). The numerical function defining appendListCode is primitive recursive.

Proof. The appendListCode construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_bForallFn_param`). The numerical function defining bForallFn param is primitive recursive.

Proof. The bForall function param construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_codeBound`). The numerical function defining codeBound is primitive recursive.

Proof. The codeBound construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_decodeFuelOkChar`). The numerical function defining decodeFuelOkChar is primitive recursive.

Proof. The decodeFuelOk characteristic construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_decodeFuelOkCharLookup`). The numerical function defining decodeFuelOkCharLookup is primitive recursive.

Proof. The decodeFuelOk characteristicLookup construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_existsListStp`). The numerical function defining existsListStp is primitive recursive.

Proof. The existsList step construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_existsListChar`). The numerical function defining existsListChar is primitive recursive.

Proof. The existsList characteristic construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`RecDecidable2.bExistsList`). Existence of an entry in a decoded finite list satisfying a recursively decidable binary relation is recursively decidable.

Proof. A bounded primitive recursion accumulates the disjunction of the characteristic values below the bound, giving a recursive decision procedure for bounded existence. ■

Theorem (`primrec_fuelTable`). The numerical function defining fuelTable is primitive recursive.

Proof. The fuelTable construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_decodeFuelOkChar2`). The numerical function defining decodeFuelOkChar2 is primitive recursive.

Proof. The decodeFuelOk characteristic2 construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_halfParity`). The numerical function defining halfParity is primitive recursive.

Proof. The halfParity construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_div2`). The numerical function defining div2 is primitive recursive.

Proof. The div2 construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_ite`). The numerical function defining ite is primitive recursive.

Proof. The ite construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_mapPairStp`). The numerical function defining mapPairStp is primitive recursive.

Proof. The mapPair step construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_mapPairCode`). The numerical function defining mapPairCode is primitive recursive.

Proof. The mapPairCode construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`REPred.forall_mem_decodeList2`). If a binary relation is recursively enumerable, then requiring it between a fixed input and every member of a decoded finite list is recursively enumerable.

Proof. Pair the fixed outer argument into the binary relation, apply finite-list universal closure, and use primitive-recursive reassociation of the paired inputs. ■

Theorem (`primrec_max`). The numerical function defining max is primitive recursive.

Proof. The max construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_max_one`). The numerical function defining max one is primitive recursive.

Proof. The max one construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_mod2`). The numerical function defining mod2 is primitive recursive.

Proof. The mod2 construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_lowOr`). The numerical function defining lowOr is primitive recursive.

Proof. The lowOr construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_lorStep`). The numerical function defining lorStep is primitive recursive.

Proof. The lorStep construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_myLor`). The numerical function defining myLor is primitive recursive.

Proof. The myLor construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`recDecidable_of_forall`). An everywhere-true predicate on natural numbers is recursively decidable.

Proof. The constant-one characteristic is primitive recursive and accepts every input, matching the hypothesis pointwise. ■

Theorem (`rePred_of_forall`). An everywhere-true predicate on natural numbers is recursively enumerable.

Proof. First decide it by the constant-one characteristic, then use the standard inclusion of decidable predicates among recursively enumerable predicates. ■

Theorem (`recDecidable2_of_forall`). An everywhere-true binary predicate on natural numbers is recursively decidable.

Proof. After pairing the two arguments, the predicate remains universally true and is decided by the constant-one characteristic. ■

Theorem (`recPow_eq`). The primitive recursion that repeatedly multiplies by the base computes natural-number exponentiation.

Proof. Induction on the exponent uses the zero exponent equation and one further multiplication in the successor case. ■

Theorem (`primrec_pow`). The numerical function defining pow is primitive recursive.

Proof. The pow construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_pow2`). The numerical function defining pow2 is primitive recursive.

Proof. The pow2 construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`primrec_two_pow`). The numerical function defining two pow is primitive recursive.

Proof. The two pow construction is assembled from primitive-recursive projections, constants, pairing, composition and its displayed bounded fold or primitive recursion. ■

Theorem (`selectFn_isOne_one_sub_sigma`). For binary inputs, the nested selector returns zero exactly in the exceptional control configuration and one otherwise.

Proof. The binary input sum is zero, one or at least two; evaluating truncated subtraction and the one-characteristic in these cases gives the selected value. ■

`rePred.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodIso

public import Mathlib.Computability.Primrec.Basic

public import Mathlib.Data.Nat.Bitwise

import Mathlib.Tactic.Ring

@[expose] public section

namespace Domain.Recursive
open DomainTheory.NeighbourhoodSystems

theorem amGM : ∀ {a b : ℕ}, 4 * a * b ≤ (a + b) * (a + b)
  | 0, _ => by rw [Nat.mul_zero, Nat.zero_mul]; exact Nat.zero_le _
  | _, 0 => by rw [Nat.mul_zero]; exact Nat.zero_le _
  | a + 1, b + 1 => by
    simpa only [Nat.mul_add, Nat.add_mul, show (4 : ℕ) = 1 + 1 + 1 + 1 from rfl, Nat.
    one_mul,
      Nat.mul_one, Nat.add_assoc, Nat.add_left_comm, Nat.add_le_add_iff_left]
      using Nat.add_le_add_right (@amGM a b) 4

theorem iter_sq_le (n guess : ℕ) : Nat.sqrt.iter n guess * Nat.sqrt.iter n guess ≤ n
    := by
  unfold Nat.sqrt.iter
  if h : (guess + n / guess) / 2 < guess then
    rw [dite_eq_left h]
    exact iter_sq_le n ((guess + n / guess) / 2)
  else
    rw [dite_eq_right h]
    apply Nat.mul_le_of_le_div
    omega
  termination_by guess
```

```
  decreasing_by exact h

theorem lt_of_mul_lt_mul_left' {a b c : ℕ} (h : a * b < a * c) : b < c := by
  rcases Nat.lt_or_ge b c with hbc | hbc
  · exact hbc
  · exact absurd h (Nat.not_lt.mpr (Nat.mul_le_mul_left a hbc))

theorem lt_iter_succ_sq (n guess : ℕ) (hn : n < (guess + 1) * (guess + 1)) :
    n < (Nat.sqrt.iter n guess + 1) * (Nat.sqrt.iter n guess + 1) := by
  unfold Nat.sqrt.iter
  if h : (guess + n / guess) / 2 < guess then
    rw [dite_eq_left h]
    suffices hsuff : n < ((guess + n / guess) / 2 + 1) * ((guess + n / guess) / 2 + 1)
     from
      lt_iter_succ_sq n ((guess + n / guess) / 2) hsuff
    refine lt_of_mul_lt_mul_left' (a := 4 * (guess * guess)) ?_
    apply Nat.lt_of_le_of_lt amGM
    rw [show (4 : ℕ) = 2 * 2 from rfl]
    rw [Nat.mul_mul_mul_comm 2, Nat.mul_mul_mul_comm (2 * guess)]
    refine Nat.mul_self_lt_mul_self (?_ : _ < _ * ((_ / 2) + 1))
    rw [← Nat.add_div_right _ (by decide), Nat.mul_comm 2, Nat.mul_assoc,
      show guess + n / guess + 2 = (guess + n / guess + 1) + 1 from rfl]
    have aux : (guess + n / guess + 1) ≤ 2 * ((guess + n / guess + 1 + 1) / 2) := by
    omega
    refine lt_of_lt_of_le ?_ (Nat.mul_le_mul_left _ aux)
    rw [Nat.add_assoc, Nat.mul_add]
    exact Nat.add_lt_add_left
      (Nat.lt_mul_div_succ _ (lt_of_le_of_lt (Nat.zero_le _) h)) _
  else
    rw [dite_eq_right h]
    exact hn
  termination_by guess
  decreasing_by exact h

theorem lt_succ_sqrt (n : ℕ) : n < (Nat.sqrt n + 1) * (Nat.sqrt n + 1) := by
  rcases Nat.lt_or_ge 1 n with h | h
  · rw [Nat.sqrt, ite_eq_right (Nat.not_le.mpr h)]
    refine lt_iter_succ_sq _ _ ?_
    set g := 1 <<< (n.log2 / 2 + 1) with hg
    have hshift : g = 2 ^ (n.log2 / 2 + 1) := by rw [hg, Nat.shiftLeft_eq, Nat.one_mul
    ]
    have hgg : g * g = 2 ^ (2 * (n.log2 / 2 + 1)) := by
      rw [hshift, ← pow_add]; congr 1; omega
    calc n < 2 ^ (n.log2 + 1) := Nat.lt_log2_self
      _ ≤ 2 ^ (2 * (n.log2 / 2 + 1)) := Nat.pow_le_pow_right (by decide) (by omega)
      _ = g * g := hgg.symm
      _ ≤ (g + 1) * (g + 1) := Nat.mul_le_mul (Nat.le_succ g) (Nat.le_succ g)
  · rw [Nat.sqrt, ite_eq_left h]
    exact Nat.lt_of_lt_of_le (Nat.lt_succ_self n)
      (Nat.le_mul_of_pos_right (n + 1) (Nat.succ_pos n))

theorem sqrt_le (n : ℕ) : Nat.sqrt n * Nat.sqrt n ≤ n := by
  rcases Nat.lt_or_ge 1 n with h | h
  · rw [Nat.sqrt, ite_eq_right (Nat.not_le.mpr h)]
    exact iter_sq_le _ _
  · rw [Nat.sqrt, ite_eq_left h]
```

```
    calc n * n ≤ 1 * n := Nat.mul_le_mul_right n h
      _ = n := Nat.one_mul n

theorem sqrt_eq_of {m q : ℕ} (h1 : q * q ≤ m) (h2 : m < (q + 1) * (q + 1)) : Nat.sqrt
    m = q := by
  have hle := sqrt_le m
  have hlt := lt_succ_sqrt m
  have hq_le : q ≤ Nat.sqrt m := by
    rcases Nat.lt_or_ge (Nat.sqrt m) q with hh | hh
    · exfalso
      have ha : Nat.sqrt m + 1 ≤ q := hh
      have hmul : (Nat.sqrt m + 1) * (Nat.sqrt m + 1) ≤ q * q := Nat.mul_le_mul ha ha
      omega
    · exact hh
  have hle_q : Nat.sqrt m ≤ q := by
    rcases Nat.lt_or_ge q (Nat.sqrt m) with hh | hh
    · exfalso
      have ha : q + 1 ≤ Nat.sqrt m := hh
      have hmul : (q + 1) * (q + 1) ≤ Nat.sqrt m * Nat.sqrt m := Nat.mul_le_mul ha ha
      omega
    · exact hh
  omega

theorem sqrt_add_eq (q a : ℕ) (h : a ≤ q + q) : Nat.sqrt (q * q + a) = q := by
  refine sqrt_eq_of (Nat.le_add_right _ _) ?_
  have expand : (q + 1) * (q + 1) = q * q + q + q + 1 := Nat.succ_mul_succ q q
  omega

theorem unpair_pair (a b : ℕ) : Nat.unpair (Nat.pair a b) = (a, b) := by
  rw [Nat.pair]
  if h : a < b then
    rw [ite_eq_left h]
    show Nat.unpair (b * b + a) = (a, b)
    have be : Nat.sqrt (b * b + a) = b := sqrt_add_eq b a (by omega)
    simp only [Nat.unpair, be, Nat.add_sub_cancel_left, ite_eq_left h]
  else
    rw [ite_eq_right h]
    show Nat.unpair (a * a + a + b) = (a, b)
    have ae : Nat.sqrt (a * a + a + b) = a := by
      have e : a * a + a + b = a * a + (a + b) := by omega
      rw [e]; exact sqrt_add_eq a (a + b) (by omega)
    have e1 : a * a + a + b - a * a = a + b := by omega
    have e2 : ¬ a + b < a := by omega
    have e3 : a + b - a = b := by omega
    simp only [Nat.unpair, ae, e1, ite_eq_right e2, e3]

@[simp] theorem unpair_pair_fst (a b : ℕ) : (Nat.pair a b).unpair.1 = a := by rw [
    unpair_pair]

@[simp] theorem unpair_pair_snd (a b : ℕ) : (Nat.pair a b).unpair.2 = b := by rw [
    unpair_pair]

def selectFn (c a b : ℕ) : ℕ := c * a + (1 - c) * b

@[simp] theorem selectFn_one (a b : ℕ) : selectFn 1 a b = a := by simp [selectFn]
```

```
@[simp] theorem selectFn_zero (a b : ℕ) : selectFn 0 a b = b := by simp [selectFn]

@[simp] theorem selectFn_ite {c : Prop} [Decidable c] (a b : ℕ) :
    selectFn (if c then 1 else 0) a b = if c then a else b := by
  split <;> simp [selectFn]

def RecDecidable (p : ℕ → Prop) : Prop :=
  ∃ f : ℕ → ℕ, Nat.Primrec f ∧ ∀ n, p n ↔ f n = 1

def REPred (p : ℕ → Prop) : Prop :=
  ∃ q : ℕ → Prop, RecDecidable q ∧ ∀ n, p n ↔ ∃ i, q (Nat.pair i n)

def REPred₂ (r : ℕ → ℕ → Prop) : Prop :=
  REPred fun t => r t.unpair.1 t.unpair.2

def RecDecidable₂ (r : ℕ → ℕ → Prop) : Prop :=
  RecDecidable fun t => r t.unpair.1 t.unpair.2

def RecDecidable₃ (r : ℕ → ℕ → ℕ → Prop) : Prop :=
  RecDecidable fun t => r t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2

def encodeList : List ℕ → ℕ
  | [] => 0
  | a :: l => Nat.pair a (encodeList l) + 1

def foldStep (stp : ℕ → ℕ) (params s : ℕ) : ℕ :=
  selectFn (1 - s.unpair.1) s
    (Nat.pair (s.unpair.1 - 1).unpair.2
      (stp (Nat.pair (s.unpair.1 - 1).unpair.1 (Nat.pair s.unpair.2 params))))

def foldCode (stp : ℕ → ℕ) (params z c : ℕ) : ℕ :=
  ((foldStep stp params)^[c] (Nat.pair c z)).unpair.2

def halfParity (n : ℕ) : ℕ :=
  Nat.rec (motive := fun _ => ℕ) 0
    (fun _ ih => Nat.pair (ih.unpair.1 + ih.unpair.2) (1 - ih.unpair.2)) n

def isOne (v : ℕ) : ℕ := 1 - ((v - 1) + (1 - v))

theorem isOne_eq_one_iff (v : ℕ) : isOne v = 1 ↔ v = 1 := by
  unfold isOne; constructor <;> (intro h; omega)

@[simp] theorem isOne_one : isOne 1 = 1 := (isOne_eq_one_iff 1).2 rfl

@[simp] theorem isOne_zero : isOne 0 = 0 := by unfold isOne; omega

def allListStp (g : ℕ → ℕ) (w : ℕ) : ℕ :=
  selectFn w.unpair.2.unpair.1 (isOne (g (Nat.pair w.unpair.1 w.unpair.2.unpair.2))) 0

def allListChar (g : ℕ → ℕ) (p c : ℕ) : ℕ := foldCode (allListStp g) p 1 c

def bExistsFn (g : ℕ → ℕ) (n N : ℕ) : ℕ :=
  Nat.rec (motive := fun _ => ℕ) 0 (fun i ih => selectFn ih 1 (isOne (g (Nat.pair i n)
    ))) N

def bForallFn (g : ℕ → ℕ) (n N : ℕ) : ℕ :=
```

```
  Nat.rec (motive := fun _ => ℕ) 1 (fun i ih => selectFn ih (isOne (g (Nat.pair i n)))
    0) N

def existsListStp (g : ℕ → ℕ) (w : ℕ) : ℕ :=
  selectFn w.unpair.2.unpair.1 1 (isOne (g (Nat.pair w.unpair.1 w.unpair.2.unpair.2)))

def existsListChar (g : ℕ → ℕ) (p c : ℕ) : ℕ := foldCode (existsListStp g) p 0 c

def isBinDigit (n : ℕ) : ℕ :=
  selectFn (isOne (1 - n)) 1 (selectFn (isOne (2 - n)) 1 0)

def allBinDigitsChar (c : ℕ) : ℕ :=
  allListChar (fun t => isBinDigit t.unpair.1) 0 c

def isZero (n : ℕ) : ℕ := 1 - min n 1

theorem le_pair_right (a b : ℕ) : b ≤ Nat.pair a b := by
  have hbb : b ≤ b * b := by
    rcases Nat.eq_zero_or_pos b with h | h
    · simp [h]
    · exact Nat.le_mul_of_pos_left b h
  unfold Nat.pair
  split <;> omega

set_option maxHeartbeats 800000 in

def listLenStp (w : ℕ) : ℕ := w.unpair.2.unpair.1 + 1

set_option maxHeartbeats 800000 in

def listLenChar (c : ℕ) : ℕ := foldCode listLenStp 0 0 c

def lowOr (x y : ℕ) : ℕ := 1 - (1 - (x % 2 + y % 2))

def lorStep (s : ℕ) : ℕ :=
  Nat.pair (Nat.pair (s.unpair.1.unpair.1 / 2) (s.unpair.1.unpair.2 / 2))
    (Nat.pair (2 * s.unpair.2.unpair.1)
      (s.unpair.2.unpair.2 + s.unpair.2.unpair.1 * lowOr s.unpair.1.unpair.1 s.unpair
    .1.unpair.2))

def mapPairStep (n acc x : ℕ) : ℕ := Nat.pair (Nat.pair n x) acc + 1

def mapPairStp (w : ℕ) : ℕ :=
  mapPairStep w.unpair.2.unpair.2 w.unpair.2.unpair.1 w.unpair.1

def mapPairCode (n c : ℕ) : ℕ := foldCode mapPairStp n 0 c

set_option maxHeartbeats 800000 in

def mulBit (a b : ℕ) : ℕ := a * b

set_option maxHeartbeats 800000 in

def decodeFuelOkCharBody (prev : ℕ → ℕ) (c : ℕ) : ℕ :=
  let sub := mulBit (prev c.unpair.2.unpair.1) (prev c.unpair.2.unpair.2)
  selectFn (isOne (1 - c.unpair.1))
```

```
    (selectFn (isOne (1 - c.unpair.2)) 1 0)
    (selectFn (isOne (2 - c.unpair.1)) (allBinDigitsChar c.unpair.2)
      (selectFn (isOne (3 - c.unpair.1)) sub
        (selectFn (isOne (4 - c.unpair.1)) sub 0)))

set_option maxHeartbeats 800000 in

def decodeFuelOkChar : ℕ → ℕ → ℕ
  | 0, _ => 0
  | fuel + 1, c => decodeFuelOkCharBody (decodeFuelOkChar fuel) c

def myLor (a b : ℕ) : ℕ :=
  (lorStep^[a + b] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.2

set_option maxHeartbeats 800000 in

def natEqChar (a b : ℕ) : ℕ := 1 - ((a - b) + (b - a))

set_option maxHeartbeats 800000 in

@[simp] theorem natEqChar_eq_one_iff (a b : ℕ) : natEqChar a b = 1 ↔ a = b := by
  unfold natEqChar; omega

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def codeBound : ℕ → ℕ
  | 0 => 1
  | n + 1 => Nat.pair 1 (codeBound n) + 1

set_option maxHeartbeats 800000 in

def listEqStpNonzero (w : ℕ) : ℕ :=
  Nat.pair
    (selectFn w.unpair.2.unpair.1.unpair.1
      (natEqChar w.unpair.1 ((w.unpair.2.unpair.1.unpair.2 - 1).unpair.1)) 0)
    ((w.unpair.2.unpair.1.unpair.2 - 1).unpair.2)

set_option maxHeartbeats 800000 in

def listEqStp (w : ℕ) : ℕ :=
  selectFn (isZero w.unpair.2.unpair.1.unpair.2) (Nat.pair 0 0) (listEqStpNonzero w)

set_option maxHeartbeats 800000 in

def listEqChar (c1 c2 : ℕ) : ℕ :=
  let r := foldCode listEqStp 0 (Nat.pair 1 c2) c1
```

```
  mulBit r.unpair.1 (isZero r.unpair.2)

set_option maxHeartbeats 800000 in

def listEqStep (st x : ℕ) : ℕ := listEqStp (Nat.pair x (Nat.pair st 0))

set_option maxHeartbeats 800000 in

def nthStp (w : ℕ) : ℕ :=
  Nat.pair (w.unpair.2.unpair.1.unpair.1 + 1)
    (selectFn (1 - ((w.unpair.2.unpair.1.unpair.1 - w.unpair.2.unpair.2)
        + (w.unpair.2.unpair.2 - w.unpair.2.unpair.1.unpair.1)))
      w.unpair.1 w.unpair.2.unpair.1.unpair.2)

set_option maxHeartbeats 800000 in

def nthCode (c i d : ℕ) : ℕ := (foldCode nthStp i (Nat.pair 0 d) c).unpair.2

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def appendListTabFn (w : ℕ) : ℕ :=
  let i := w.unpair.1
  let p := w.unpair.2
  let len1 := p.unpair.2.unpair.2
  selectFn (isZero ((i + 1) - len1))
    (nthCode p.unpair.1 i 0)
    (nthCode p.unpair.2.unpair.1 (i - len1) 0)

set_option maxHeartbeats 800000 in

def decodeFuelOkCharLookup (w : ℕ) : ℕ :=
  decodeFuelOkCharBody (fun c' => nthCode w.unpair.1 c' 0) w.unpair.2

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def dropListTabFn (w : ℕ) : ℕ :=
  nthCode (w.unpair.2.unpair.2) (w.unpair.1 + w.unpair.2.unpair.1) 0

def reForallF (qc : ℕ → ℕ) (acc x : ℕ) : ℕ :=
  Nat.pair (acc.unpair.1 - 1).unpair.2
    (selectFn acc.unpair.2 (isOne (qc (Nat.pair (acc.unpair.1 - 1).unpair.1 x))) 0)

def reForallStp (qc : ℕ → ℕ) (w : ℕ) : ℕ :=
  Nat.pair (w.unpair.2.unpair.1.unpair.1 - 1).unpair.2
    (selectFn w.unpair.2.unpair.1.unpair.2
      (isOne (qc (Nat.pair (w.unpair.2.unpair.1.unpair.1 - 1).unpair.1 w.unpair.1)))
    0)
```

```
def reForallChar (qc : ℕ → ℕ) (w c : ℕ) : ℕ :=
  (foldCode (reForallStp qc) 0 (Nat.pair w 1) c).unpair.2

theorem sqrt_le_add (n : ℕ) : n ≤ Nat.sqrt n * Nat.sqrt n + Nat.sqrt n + Nat.sqrt n :=
    by
  have h := lt_succ_sqrt n
  have e : (Nat.sqrt n + 1) * (Nat.sqrt n + 1)
      = Nat.sqrt n * Nat.sqrt n + Nat.sqrt n + Nat.sqrt n + 1 := Nat.succ_mul_succ _ _
  omega

theorem pair_unpair (n : ℕ) : Nat.pair (Nat.unpair n).1 (Nat.unpair n).2 = n := by
  have sm : Nat.sqrt n * Nat.sqrt n + (n - Nat.sqrt n * Nat.sqrt n) = n :=
    Nat.add_sub_cancel' (sqrt_le _)
  have hadd := sqrt_le_add n
  rw [Nat.unpair]
  if h : n - Nat.sqrt n * Nat.sqrt n < Nat.sqrt n then
    rw [ite_eq_left h]
    show Nat.pair (n - Nat.sqrt n * Nat.sqrt n) (Nat.sqrt n) = n
    rw [Nat.pair, ite_eq_left h]
    omega
  else
    rw [ite_eq_right h]
    have hge : Nat.sqrt n ≤ n - Nat.sqrt n * Nat.sqrt n := Nat.le_of_not_lt h
    have hlt : ¬ Nat.sqrt n < n - Nat.sqrt n * Nat.sqrt n - Nat.sqrt n := by omega
    show Nat.pair (Nat.sqrt n) (n - Nat.sqrt n * Nat.sqrt n - Nat.sqrt n) = n
    rw [Nat.pair, ite_eq_right hlt]
    omega

set_option maxHeartbeats 800000 in

def tabStep (g : ℕ → ℕ) (w : ℕ) : ℕ :=
  Nat.pair (g (Nat.pair (w.unpair.1.unpair.1 - 1 - w.unpair.2.unpair.1) w.unpair.1.
    unpair.2))
    w.unpair.2.unpair.2 + 1

set_option maxHeartbeats 800000 in

def tabCode (g : ℕ → ℕ) (a B : ℕ) : ℕ :=
  Nat.rec (motive := fun _ => ℕ) 0 (fun y IH => tabStep g (Nat.pair a (Nat.pair y IH))
    ) B

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in


set_option maxHeartbeats 800000 in

def appendListCode (c1 c2 : ℕ) : ℕ :=
  let len1 := listLenChar c1
  let len2 := listLenChar c2
  tabCode appendListTabFn (Nat.pair (len1 + len2) (Nat.pair c1 (Nat.pair c2 len1))) (
    len1 + len2)

set_option maxHeartbeats 800000 in
```

```
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def dropCode (n c : ℕ) : ℕ :=
  let len := listLenChar c
  tabCode dropListTabFn (Nat.pair (len - n) (Nat.pair n c)) (len - n)

set_option maxHeartbeats 800000 in

def fuelTableStep (bodyLookup : ℕ → ℕ) (bound prevTable : ℕ) : ℕ :=
  tabCode (fun w => bodyLookup (Nat.pair w.unpair.2 w.unpair.1))
    (Nat.pair (bound + 1) prevTable) (bound + 1)

set_option maxHeartbeats 800000 in

def fuelTable (bodyLookup : ℕ → ℕ) (bound fuel : ℕ) : ℕ :=
  Nat.rec (motive := fun _ => ℕ) (tabCode (fun _ => 0) (Nat.pair (bound + 1) 0) (bound
    + 1))
    (fun _ prevTable => fuelTableStep bodyLookup bound prevTable) fuel

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def takeListTabFn (w : ℕ) : ℕ := nthCode (w.unpair.2.unpair.2) w.unpair.1 0

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def takeCode (n c : ℕ) : ℕ :=
  let len := listLenChar c
  let B := Nat.min n len
  tabCode takeListTabFn (Nat.pair B (Nat.pair n c)) B

theorem unpair_snd_le (n : ℕ) : n.unpair.2 ≤ n := by
  have h := le_pair_right n.unpair.1 n.unpair.2
  rwa [pair_unpair] at h

def decodeList : ℕ → List ℕ
  | 0 => []
  | (c + 1) => c.unpair.1 :: decodeList c.unpair.2
decreasing_by exact Nat.lt_succ_of_le (unpair_snd_le c)

theorem primrec_id : Nat.Primrec id :=
  (Nat.Primrec.left.pair Nat.Primrec.right).of_eq fun n => pair_unpair n

theorem rec_add (a b : ℕ) : Nat.rec a (fun _ IH => IH + 1) b = a + b := by
  induction b with
  | zero => rfl
  | succ k ih => show Nat.rec a (fun _ IH => IH + 1) k + 1 = a + (k + 1); omega
```

```
private theorem rec_mul (a b : ℕ) : Nat.rec 0 (fun _ IH => a + IH) b = a * b := by
  induction b with
  | zero => rfl
  | succ k ih =>
    show a + Nat.rec 0 (fun _ IH => a + IH) k = a * (k + 1)
    rw [Nat.mul_succ]; omega

theorem primrec_add : Nat.Primrec (Nat.unpaired (· + ·)) :=
  (Nat.Primrec.prec primrec_id
      ((Nat.Primrec.succ.comp Nat.Primrec.right).comp Nat.Primrec.right)).of_eq fun p
    => by
    simp only [unpair_pair_snd, id_eq]
    exact rec_add _ _

theorem primrec_mul : Nat.Primrec (Nat.unpaired (· * ·)) :=
  (Nat.Primrec.prec Nat.Primrec.zero
      (primrec_add.comp (Nat.Primrec.pair Nat.Primrec.left
        (Nat.Primrec.right.comp Nat.Primrec.right)))).of_eq fun p => by
    simp only [unpair_pair_fst, unpair_pair_snd, Nat.unpaired]
    exact rec_mul _ _

private theorem rec_pred (n : ℕ) : Nat.rec 0 (fun y _ => y) n = n - 1 := by
  cases n with
  | zero => rfl
  | succ k => rfl

theorem primrec_pred : Nat.Primrec (fun n => n - 1) :=
  ((Nat.Primrec.prec Nat.Primrec.zero (Nat.Primrec.left.comp Nat.Primrec.right)).comp
    (primrec_id.pair primrec_id)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
    exact rec_pred n

private theorem rec_sub (a b : ℕ) : Nat.rec a (fun _ IH => IH - 1) b = a - b := by
  induction b with
  | zero => rfl
  | succ k ih => show Nat.rec a (fun _ IH => IH - 1) k - 1 = a - (k + 1); rw [ih];
    omega

theorem primrec_sub : Nat.Primrec (Nat.unpaired (· - ·)) :=
  (Nat.Primrec.prec primrec_id
    ((primrec_pred.comp Nat.Primrec.right).comp Nat.Primrec.right)).of_eq fun p => by
    simp only [unpair_pair_snd, id_eq]
    exact rec_sub _ _

theorem nat_mul_eq_one {a b : ℕ} : a * b = 1 ↔ a = 1 ∧ b = 1 := by
  refine ⟨fun h => ⟨Nat.dvd_one.mp ⟨b, h.symm⟩, Nat.dvd_one.mp ⟨a, ?_⟩⟩, fun h => by rw [
    h.1, h.2]⟩
  rw [Nat.mul_comm] at h; exact h.symm

theorem primrec_add₂ {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => f n + g n) :=
  (primrec_add.comp (hf.pair hg)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]

theorem primrec_mul₂ {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
```

```
    Nat.Primrec (fun n => f n * g n) :=
  (primrec_mul.comp (hf.pair hg)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]

theorem primrec_sub₂ {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => f n - g n) :=
  (primrec_sub.comp (hf.pair hg)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]

theorem primrec_selectFn {c a b : ℕ → ℕ} (hc : Nat.Primrec c) (ha : Nat.Primrec a)
    (hb : Nat.Primrec b) : Nat.Primrec (fun n => selectFn (c n) (a n) (b n)) :=
  primrec_add₂ (primrec_mul₂ hc ha) (primrec_mul₂ (primrec_sub₂ (Nat.Primrec.const 1)
    hc) hb)

theorem RecDecidable.of_iff {p q : ℕ → Prop} (h : ∀ n, q n ↔ p n) (hp : RecDecidable
    p) :
    RecDecidable q := by
  obtain ⟨f, hf, hfe⟩ := hp
  exact ⟨f, hf, fun n => (h n).trans (hfe n)⟩

theorem RecDecidable.comp {p : ℕ → Prop} (hp : RecDecidable p) {g : ℕ → ℕ}
    (hg : Nat.Primrec g) : RecDecidable (fun n => p (g n)) := by
  obtain ⟨f, hf, hfe⟩ := hp
  exact ⟨fun n => f (g n), hf.comp hg, fun n => hfe (g n)⟩

theorem RecDecidable.and {p q : ℕ → Prop} (hp : RecDecidable p) (hq : RecDecidable q)
    :
    RecDecidable (fun n => p n ∧ q n) := by
  obtain ⟨f, hf, hfe⟩ := hp
  obtain ⟨g, hg, hge⟩ := hq
  refine ⟨fun n => f n * g n, ?_, fun n => ?_⟩
  · exact (primrec_mul.comp (hf.pair hg)).of_eq fun n => by
      simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  · rw [nat_mul_eq_one]; exact and_congr (hfe n) (hge n)

theorem RecDecidable.natEq {a b : ℕ → ℕ} (ha : Nat.Primrec a) (hb : Nat.Primrec b) :
    RecDecidable (fun t => a t = b t) := by
  refine ⟨fun t => Nat.unpaired (· - ·) (Nat.pair 1 (Nat.unpaired (· + ·)
      (Nat.pair (Nat.unpaired (· - ·) (Nat.pair (a t) (b t)))
        (Nat.unpaired (· - ·) (Nat.pair (b t) (a t)))))), ?_, fun t => ?_⟩
  · exact primrec_sub.comp ((Nat.Primrec.const 1).pair (primrec_add.comp
      ((primrec_sub.comp (ha.pair hb)).pair (primrec_sub.comp (hb.pair ha)))))
  · simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
    constructor <;> intro h <;> omega

theorem RecDecidable.not {p : ℕ → Prop} (hp : RecDecidable p) : RecDecidable (fun n =>
    ¬ p n) := by
  obtain ⟨f, hf, hfe⟩ := hp
  refine ⟨fun n => Nat.unpaired (· - ·) (Nat.pair 1 (Nat.unpaired (· - ·) (Nat.pair 1
      (Nat.unpaired (· + ·) (Nat.pair (Nat.unpaired (· - ·) (Nat.pair (f n) 1))
        (Nat.unpaired (· - ·) (Nat.pair 1 (f n)))))))), ?_, fun n => ?_⟩
  · exact primrec_sub.comp ((Nat.Primrec.const 1).pair (primrec_sub.comp
      ((Nat.Primrec.const 1).pair (primrec_add.comp
        ((primrec_sub.comp (hf.pair (Nat.Primrec.const 1))).pair
          (primrec_sub.comp ((Nat.Primrec.const 1).pair hf)))))))
  · show ¬ p n ↔ _
```

```
    rw [hfe n]
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
    constructor <;> intro h <;> omega

theorem RecDecidable.em {p : ℕ → Prop} (hp : RecDecidable p) (n : ℕ) : p n ∨ ¬ p n :=
    by
  obtain ⟨f, _, hfe⟩ := hp
  by_cases h : f n = 1
  · exact Or.inl ((hfe n).mpr h)
  · exact Or.inr (fun hp => h ((hfe n).mp hp))

theorem RecDecidable.or {p q : ℕ → Prop} (hp : RecDecidable p) (hq : RecDecidable q) :
    RecDecidable (fun n => p n ∨ q n) := by
  refine RecDecidable.of_iff (fun n => ?_) (hp.not.and hq.not).not
  constructor
  · rintro (h | h) ⟨hnp, hnq⟩
    · exact hnp h
    · exact hnq h
  · intro h
    rcases hp.em n with hp' | hp'
    · exact Or.inl hp'
    · rcases hq.em n with hq' | hq'
      · exact Or.inr hq'
      · exact absurd ⟨hp', hq'⟩ h

theorem RecDecidable.re {p : ℕ → Prop} (hp : RecDecidable p) : REPred p := by
  refine ⟨fun t => p t.unpair.2, hp.comp Nat.Primrec.right, fun n => ?_⟩
  simp only [unpair_pair_snd]
  exact ⟨fun h => ⟨0, h⟩, fun ⟨_, h⟩ => h⟩

theorem RecDecidable₂.re {r : ℕ → ℕ → Prop} (hr : RecDecidable₂ r) : REPred₂ r :=
  RecDecidable.re hr

theorem REPred.of_iff {p q : ℕ → Prop} (h : ∀ n, q n ↔ p n) (hp : REPred p) : REPred
    q := by
  obtain ⟨r, hr, hre⟩ := hp
  exact ⟨r, hr, fun n => (h n).trans (hre n)⟩

theorem REPred.comp {p : ℕ → Prop} (hp : REPred p) {g : ℕ → ℕ} (hg : Nat.Primrec g)
    :
    REPred (fun n => p (g n)) := by
  obtain ⟨q, hq, hqe⟩ := hp
  refine ⟨fun t => q (Nat.pair t.unpair.1 (g t.unpair.2)),
    hq.comp (Nat.Primrec.left.pair (hg.comp Nat.Primrec.right)), fun n => ?_⟩
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact hqe (g n)

theorem REPred.and {p q : ℕ → Prop} (hp : REPred p) (hq : REPred q) :
    REPred (fun n => p n ∧ q n) := by
  obtain ⟨a, ha, hae⟩ := hp
  obtain ⟨b, hb, hbe⟩ := hq
  refine ⟨fun u => a (Nat.pair u.unpair.1.unpair.1 u.unpair.2)
      ∧ b (Nat.pair u.unpair.1.unpair.2 u.unpair.2),
    (ha.comp ((Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right)).and
      (hb.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right)),
    fun n => ?_⟩
```

```
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hae n, hbe n]
  constructor
  · rintro ⟨⟨i, hi⟩, ⟨j, hj⟩⟩
    exact ⟨Nat.pair i j, by simp only [unpair_pair_fst, unpair_pair_snd]; exact ⟨hi, hj
    ⟩⟩
  · rintro ⟨w, hw1, hw2⟩
    exact ⟨⟨w.unpair.1, hw1⟩, ⟨w.unpair.2, hw2⟩⟩

theorem REPred.proj {p : ℕ → Prop} (hp : REPred p) :
    REPred (fun n => ∃ i, p (Nat.pair i n)) := by
  obtain ⟨q, hq, hqe⟩ := hp
  refine ⟨fun u => q (Nat.pair u.unpair.1.unpair.2 (Nat.pair u.unpair.1.unpair.1 u.
    unpair.2)),
    hq.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
      ((Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right)), fun n => ?_⟩
  simp only [unpair_pair_fst, unpair_pair_snd]
  constructor
  · rintro ⟨i, hi⟩
    rw [hqe (Nat.pair i n)] at hi
    obtain ⟨j, hj⟩ := hi
    exact ⟨Nat.pair i j, by simpa only [unpair_pair_fst, unpair_pair_snd] using hj⟩
  · rintro ⟨w, hw⟩
    refine ⟨w.unpair.1, ?_⟩
    rw [hqe (Nat.pair w.unpair.1 n)]
    exact ⟨w.unpair.2, hw⟩

theorem REPred.or {p q : ℕ → Prop} (hp : REPred p) (hq : REPred q) :
    REPred (fun n => p n ∨ q n) := by
  obtain ⟨a, ha, hae⟩ := hp
  obtain ⟨b, hb, hbe⟩ := hq
  refine ⟨fun u => (a (Nat.pair u.unpair.1.unpair.2 u.unpair.2) ∧ u.unpair.1.unpair.1 =
    0)
      ∨ (b (Nat.pair u.unpair.1.unpair.2 u.unpair.2) ∧ u.unpair.1.unpair.1 = 1), ?_,
    fun n => ?_⟩
  · refine RecDecidable.or (RecDecidable.and ?_ ?_) (RecDecidable.and ?_ ?_)
    · exact ha.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right)
    · exact RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.
    const 0)
    · exact hb.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right)
    · exact RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.
    const 1)
  · show p n ∨ q n ↔ _
    rw [hae n, hbe n]
    constructor
    · rintro (⟨i, hi⟩ | ⟨i, hi⟩)
      · refine ⟨Nat.pair 0 i, Or.inl ⟨?_, ?_⟩⟩
        · simpa only [unpair_pair_fst, unpair_pair_snd] using hi
        · simp only [unpair_pair_fst]
      · refine ⟨Nat.pair 1 i, Or.inr ⟨?_, ?_⟩⟩
        · simpa only [unpair_pair_fst, unpair_pair_snd] using hi
        · simp only [unpair_pair_fst]
    · rintro ⟨w, hw⟩
      simp only [unpair_pair_fst, unpair_pair_snd] at hw
      rcases hw with ⟨hi, _⟩ | ⟨hi, _⟩
      · exact Or.inl ⟨w.unpair.2, hi⟩
```

```
      · exact Or.inr ⟨w.unpair.2, hi⟩

theorem le_pair_left (a b : ℕ) : a ≤ Nat.pair a b := by
  unfold Nat.pair
  split <;> omega

theorem encodeList_length_le : ∀ l : List ℕ, l.length ≤ encodeList l
  | [] => Nat.le_refl 0
  | a :: l => by
    simp only [encodeList, List.length_cons]
    have hrec := encodeList_length_le l
    have hle : encodeList l ≤ Nat.pair a (encodeList l) := le_pair_right a (encodeList
     l)
    omega

theorem foldStep_zero (stp : ℕ → ℕ) (params acc : ℕ) :
    foldStep stp params (Nat.pair 0 acc) = Nat.pair 0 acc := by
  unfold foldStep
  simp only [unpair_pair_fst, unpair_pair_snd, Nat.sub_zero, selectFn_one]

theorem foldStep_pos (stp : ℕ → ℕ) (params a t acc : ℕ) :
    foldStep stp params (Nat.pair (Nat.pair a t + 1) acc)
      = Nat.pair t (stp (Nat.pair a (Nat.pair acc params))) := by
  unfold foldStep
  simp only [unpair_pair_fst, unpair_pair_snd]
  have h1 : 1 - (Nat.pair a t + 1) = 0 := by omega
  have h2 : Nat.pair a t + 1 - 1 = Nat.pair a t := by omega
  rw [h1, h2, selectFn_zero, unpair_pair_fst, unpair_pair_snd]

theorem rec_const_iterate (f : ℕ → ℕ) (s : ℕ) :
    ∀ k : ℕ, Nat.rec (motive := fun _ => ℕ) s (fun _ ih => f ih) k = f^[k] s
  | 0 => rfl
  | (k + 1) => by
      rw [Function.iterate_succ_apply']
      exact congrArg f (rec_const_iterate f s k)

theorem foldStep_iterate (stp : ℕ → ℕ) (params : ℕ) :
    ∀ (k : ℕ) (l : List ℕ) (acc : ℕ), l.length ≤ k →
      ((foldStep stp params)^[k] (Nat.pair (encodeList l) acc)).unpair.2
        = List.foldl (fun acc x => stp (Nat.pair x (Nat.pair acc params))) acc l := by
  intro k
  induction k with
  | zero =>
    intro l acc hlen
    cases l with
    | nil => simp only [Function.iterate_zero_apply, encodeList, unpair_pair_snd, List
    .foldl_nil]
    | cons a l' => simp only [List.length_cons] at hlen; omega
  | succ k ih =>
    intro l acc hlen
    rw [Function.iterate_succ_apply]
    cases l with
    | nil =>
      rw [show encodeList ([] : List ℕ) = 0 from rfl, foldStep_zero]
      exact ih [] acc (Nat.zero_le k)
    | cons a l' =>
```

```
      have hlen' : l'.length ≤ k := by simp only [List.length_cons] at hlen; omega
      rw [show encodeList (a :: l') = Nat.pair a (encodeList l') + 1 from rfl,
    foldStep_pos,
        ih l' (stp (Nat.pair a (Nat.pair acc params))) hlen', List.foldl_cons]

theorem foldCode_eq (stp : ℕ → ℕ) (params z : ℕ) (l : List ℕ) :
    foldCode stp params z (encodeList l)
      = List.foldl (fun acc x => stp (Nat.pair x (Nat.pair acc params))) z l := by
  unfold foldCode
  exact foldStep_iterate stp params (encodeList l) l z (encodeList_length_le l)

theorem unpair_left_le (n : ℕ) : n.unpair.1 ≤ n := by
  have h := le_pair_left n.unpair.1 n.unpair.2
  rwa [pair_unpair] at h

theorem pair_le_pair_left (b : ℕ) {a a' : ℕ} (h : a ≤ a') : Nat.pair a b ≤ Nat.pair a
    ' b := by
  rcases eq_or_lt_of_le h with rfl | hlt
  · exact le_refl _
  · exact le_of_lt (Nat.pair_lt_pair_left b hlt)

theorem pair_le_pair_right' (a : ℕ) {b b' : ℕ} (h : b ≤ b') : Nat.pair a b ≤ Nat.pair
     a b' := by
  rcases eq_or_lt_of_le h with rfl | hlt
  · exact le_refl _
  · exact le_of_lt (Nat.pair_lt_pair_right a hlt)

theorem pair_le_pair {a b a' b' : ℕ} (ha : a ≤ a') (hb : b ≤ b') :
    Nat.pair a b ≤ Nat.pair a' b' :=
  le_trans (pair_le_pair_left b ha) (pair_le_pair_right' a' hb)

theorem decodeList_zero : decodeList 0 = [] := by rw [decodeList]

theorem decodeList_succ (c : ℕ) :
    decodeList (c + 1) = c.unpair.1 :: decodeList c.unpair.2 := by
  rw [decodeList]

theorem encodeList_decodeList (c : ℕ) : encodeList (decodeList c) = c := by
  induction c using Nat.strong_induction_on with
  | _ c ih =>
    cases c with
    | zero => simp only [decodeList_zero, encodeList]
    | succ d =>
      rw [decodeList_succ, encodeList, ih d.unpair.2 (Nat.lt_succ_of_le (unpair_snd_le
     d)),
        pair_unpair]

theorem foldCode_eq' (stp : ℕ → ℕ) (params z c : ℕ) :
    foldCode stp params z c
      = List.foldl (fun acc x => stp (Nat.pair x (Nat.pair acc params))) z (decodeList
     c) := by
  conv_lhs => rw [← encodeList_decodeList c]
  rw [foldCode_eq]

theorem primrec_foldStepPacked {stp : ℕ → ℕ} (hstp : Nat.Primrec stp) :
    Nat.Primrec (fun w => foldStep stp w.unpair.1 w.unpair.2) := by
```

```
  have hparams : Nat.Primrec (fun w => w.unpair.1) := Nat.Primrec.left
  have hstate : Nat.Primrec (fun w => w.unpair.2) := Nat.Primrec.right
  have hrc : Nat.Primrec (fun w => w.unpair.2.unpair.1) := Nat.Primrec.left.comp Nat.
    Primrec.right
  have hacc : Nat.Primrec (fun w => w.unpair.2.unpair.2) := Nat.Primrec.right.comp Nat
    .Primrec.right
  have hcond : Nat.Primrec (fun w => 1 - w.unpair.2.unpair.1) :=
    primrec_sub₂ (Nat.Primrec.const 1) hrc
  have hrcm1 : Nat.Primrec (fun w => w.unpair.2.unpair.1 - 1) :=
    primrec_sub₂ hrc (Nat.Primrec.const 1)
  have hhead : Nat.Primrec (fun w => (w.unpair.2.unpair.1 - 1).unpair.1) :=
    Nat.Primrec.left.comp hrcm1
  have htail : Nat.Primrec (fun w => (w.unpair.2.unpair.1 - 1).unpair.2) :=
    Nat.Primrec.right.comp hrcm1
  have hinner : Nat.Primrec (fun w => Nat.pair w.unpair.2.unpair.2 w.unpair.1) := hacc
    .pair hparams
  have harg : Nat.Primrec
      (fun w => Nat.pair (w.unpair.2.unpair.1 - 1).unpair.1
        (Nat.pair w.unpair.2.unpair.2 w.unpair.1)) := hhead.pair hinner
  have hstpv : Nat.Primrec
      (fun w => stp (Nat.pair (w.unpair.2.unpair.1 - 1).unpair.1
        (Nat.pair w.unpair.2.unpair.2 w.unpair.1))) := hstp.comp harg
  have helse : Nat.Primrec
      (fun w => Nat.pair (w.unpair.2.unpair.1 - 1).unpair.2
        (stp (Nat.pair (w.unpair.2.unpair.1 - 1).unpair.1
          (Nat.pair w.unpair.2.unpair.2 w.unpair.1)))) := htail.pair hstpv

  exact (primrec_selectFn hcond hstate helse).of_eq fun _ => rfl

theorem primrec_foldCode {stp : ℕ → ℕ} (hstp : Nat.Primrec stp)
    {params z c : ℕ → ℕ} (hp : Nat.Primrec params) (hz : Nat.Primrec z) (hc : Nat.
    Primrec c) :
    Nat.Primrec (fun n => foldCode stp (params n) (z n) (c n)) := by
  have hfoldw : Nat.Primrec (fun w => foldStep stp w.unpair.1 w.unpair.2) :=
    primrec_foldStepPacked hstp
  have hg : Nat.Primrec (fun x => foldStep stp x.unpair.1.unpair.1 x.unpair.2.unpair
    .2) :=
    (hfoldw.comp ((Nat.Primrec.left.comp Nat.Primrec.left).pair
      (Nat.Primrec.right.comp Nat.Primrec.right))).of_eq fun _ => by
        simp only [unpair_pair_fst, unpair_pair_snd]
  have hprec := Nat.Primrec.prec Nat.Primrec.right hg
  refine (Nat.Primrec.right.comp
    (hprec.comp ((hp.pair (hc.pair hz)).pair hc))).of_eq fun n => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  rw [rec_const_iterate]
  rfl

theorem isOne_le_one (v : ℕ) : isOne v ≤ 1 := by unfold isOne; omega

theorem isOne_of_ne_one {v : ℕ} (h : v = 1) : isOne v = 0 := by
  unfold isOne
  omega

theorem primrec_isOne : Nat.Primrec isOne :=
  primrec_sub₂ (Nat.Primrec.const 1)
    (primrec_add₂ (primrec_sub₂ primrec_id (Nat.Primrec.const 1))
```

```
      (primrec_sub₂ (Nat.Primrec.const 1) primrec_id))

theorem isZero_eq_one_iff (n : ℕ) : isZero n = 1 ↔ n = 0 := by
  unfold isZero; omega

theorem isZero_le_one (n : ℕ) : isZero n ≤ 1 := by unfold isZero; omega

theorem primrec_isZero : Nat.Primrec isZero := by
  have hmin : Nat.Primrec (fun n => min n 1) :=
    (primrec_sub₂ primrec_id (primrec_sub₂ primrec_id (Nat.Primrec.const 1))).of_eq fun
     n => by
      rcases n with _ | n
      · rfl
      · show n + 1 - (n + 1 - 1) = min (n + 1) 1
        rw [Nat.min_eq_right (Nat.le_add_left 1 n)]
        omega
  exact primrec_sub₂ (Nat.Primrec.const 1) hmin

theorem primrec_le {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => isZero (f n - g n)) :=
  primrec_isZero.comp (primrec_sub₂ hf hg)

theorem bForallFn_le_one (g : ℕ → ℕ) (n N : ℕ) : bForallFn g n N ≤ 1 := by
  induction N with
  | zero => exact Nat.le_refl 1
  | succ N ih =>
    show selectFn (bForallFn g n N) (isOne (g (Nat.pair N n))) 0 ≤ 1
    rcases (show bForallFn g n N = 0 ∨ bForallFn g n N = 1 by omega) with h | h
    · rw [h, selectFn_zero]; exact Nat.zero_le 1
    · rw [h, selectFn_one]; exact isOne_le_one _

theorem bForallFn_eq_one_iff (g : ℕ → ℕ) (n N : ℕ) :
    bForallFn g n N = 1 ↔ ∀ i, i < N → g (Nat.pair i n) = 1 := by
  induction N with
  | zero =>
    constructor
    · intro _ i hi; exact absurd hi (Nat.not_lt_zero i)
    · intro _; rfl
  | succ N ih =>
    have hstep : bForallFn g n (N + 1)
        = selectFn (bForallFn g n N) (isOne (g (Nat.pair N n))) 0 := rfl
    have hle := bForallFn_le_one g n N
    rw [hstep]
    rcases (show bForallFn g n N = 0 ∨ bForallFn g n N = 1 by omega) with h0 | h1
    · rw [h0, selectFn_zero]
      constructor
      · intro hcontra; exact absurd hcontra (by decide)
      · intro hall
        have hb : bForallFn g n N = 1 := ih.mpr (fun i hi => hall i (Nat.lt_succ_of_lt
     hi))
        rw [h0] at hb; exact hb
    · rw [h1, selectFn_one, isOne_eq_one_iff]
      constructor
      · intro hgN i hi
        rcases (show i < N ∨ i = N by omega) with hlt | heq
        · exact (ih.mp h1) i hlt
```

```
        · subst heq; exact hgN
      · intro hall; exact hall N (Nat.lt_succ_self N)

theorem bExistsFn_le_one (g : ℕ → ℕ) (n N : ℕ) : bExistsFn g n N ≤ 1 := by
  induction N with
  | zero => simp [bExistsFn]
  | succ N ih =>
    show selectFn (bExistsFn g n N) 1 (isOne (g (Nat.pair N n))) ≤ 1
    rcases (show bExistsFn g n N = 0 ∨ bExistsFn g n N = 1 by omega) with h | h
    · rw [h, selectFn_zero]
      exact isOne_le_one _
    · rw [h, selectFn_one]

theorem bExistsFn_congr {g1 g2 : ℕ → ℕ} {n N : ℕ}
    (h : ∀ i < N, g1 (Nat.pair i n) = g2 (Nat.pair i n)) : bExistsFn g1 n N = bExistsFn
     g2 n N := by
  induction N with
  | zero => rfl
  | succ N ih =>
    show selectFn (bExistsFn g1 n N) 1 (isOne (g1 (Nat.pair N n)))
      = selectFn (bExistsFn g2 n N) 1 (isOne (g2 (Nat.pair N n)))
    rw [ih (fun i hi => h i (Nat.lt_succ_of_lt hi)), h N (Nat.lt_succ_self N)]

theorem bExistsFn_eq_one_iff (g : ℕ → ℕ) (n N : ℕ) :
    bExistsFn g n N = 1 ↔ ∃ i, i < N ∧ g (Nat.pair i n) = 1 := by
  induction N with
  | zero =>
    simp [bExistsFn]
  | succ N ih =>
    have hstep : bExistsFn g n (N + 1)
        = selectFn (bExistsFn g n N) 1 (isOne (g (Nat.pair N n))) := rfl
    rw [hstep]
    have hle := bExistsFn_le_one g n N
    rcases (show bExistsFn g n N = 0 ∨ bExistsFn g n N = 1 by omega) with h0 | h1
    · rw [h0, selectFn_zero, isOne_eq_one_iff]
      constructor
      · intro hgN; exact ⟨N, Nat.lt_succ_self N, hgN⟩
      · intro ⟨i, hi, hg⟩
        rcases (show i < N ∨ i = N by omega) with hlt | heq
        · have hne : bExistsFn g n N = 1 := ih.mpr ⟨i, hlt, hg⟩
          omega
        · subst heq; exact hg
    · rw [h1, selectFn_one]
      constructor
      · intro _
        obtain ⟨i, hi, hg⟩ := ih.mp h1
        exact ⟨i, Nat.lt_succ_of_lt hi, hg⟩
      · intro _; rfl

theorem primrec_bExistsFn {g : ℕ → ℕ} (hg : Nat.Primrec g) :
    Nat.Primrec (fun t => bExistsFn g t.unpair.1 t.unpair.2) := by
  have hGfn : Nat.Primrec (fun w =>
      selectFn w.unpair.2.unpair.2 1 (isOne (g (Nat.pair w.unpair.2.unpair.1 w.unpair
    .1)))) :=
    primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right)
      (Nat.Primrec.const 1)
```

```
      (primrec_isOne.comp (hg.comp
        ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left)))
  have hprec := Nat.Primrec.prec (Nat.Primrec.const 0) hGfn
  refine (hprec.comp (Nat.Primrec.left.pair Nat.Primrec.right)).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  rfl

theorem primrec_bExistsFn_param {g : ℕ → ℕ} (hg : Nat.Primrec g) :
    Nat.Primrec (fun t => bExistsFn (fun p => g (Nat.pair t.unpair.1.unpair.1 p))
      t.unpair.1.unpair.2 t.unpair.2) := by
  have hGfn : Nat.Primrec (fun w =>
      selectFn w.unpair.2.unpair.2 1
        (isOne (g (Nat.pair w.unpair.1.unpair.1
          (Nat.pair w.unpair.2.unpair.1 w.unpair.1.unpair.2))))) :=
    primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right)
      (Nat.Primrec.const 1)
      (primrec_isOne.comp (hg.comp
        (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.left)
          (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.right)
            (Nat.Primrec.right.comp Nat.Primrec.left)))))
  have hprec := Nat.Primrec.prec (Nat.Primrec.const 0) hGfn
  refine (hprec.comp (Nat.Primrec.left.pair Nat.Primrec.right)).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  rfl

theorem decodeList_encodeList : ∀ l : List ℕ, decodeList (encodeList l) = l
  | [] => by rw [encodeList, decodeList_zero]
  | a :: l => by
    rw [encodeList, decodeList_succ, unpair_pair_fst, unpair_pair_snd,
    decodeList_encodeList l]

theorem reForallStp_eq (qc : ℕ → ℕ) (acc x : ℕ) :
    reForallStp qc (Nat.pair x (Nat.pair acc 0)) = reForallF qc acc x := by
  unfold reForallStp reForallF
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem reForallChar_eq (qc : ℕ → ℕ) (w c : ℕ) :
    reForallChar qc w c
      = (List.foldl (reForallF qc) (Nat.pair w 1) (decodeList c)).unpair.2 := by
  have hfun : (fun (acc x : ℕ) => reForallStp qc (Nat.pair x (Nat.pair acc 0))) =
    reForallF qc := by
    funext acc x; exact reForallStp_eq qc acc x
  unfold reForallChar
  rw [foldCode_eq', hfun]

theorem reForallF_foldl_eq_one_iff (qc : ℕ → ℕ) :
    ∀ (l : List ℕ) (w flag : ℕ), flag ≤ 1 →
      ((List.foldl (reForallF qc) (Nat.pair w flag) l).unpair.2 = 1 ↔
        flag = 1 ∧ ∀ k, k < l.length →
          qc (Nat.pair ((decodeList w).getD k 0) (l.getD k 0)) = 1) := by
  intro l
  induction l with
  | nil =>
    intro w flag _
    simp only [List.foldl_nil, unpair_pair_snd, List.length_nil, Nat.not_lt_zero,
      false_implies, implies_true, and_true]
```

```
  | cons x l ih =>
    intro w flag hflag
    have hi : (decodeList w).getD 0 0 = (w - 1).unpair.1 := by
      rcases w with _ | c
      · have hz : ((0 : ℕ) - 1).unpair.1 = 0 := by
          rw [Nat.zero_sub]; exact congrArg Prod.fst Nat.unpair_zero
        rw [decodeList_zero, List.getD_nil, hz]
      · rw [decodeList_succ, List.getD_cons_zero, Nat.add_sub_cancel]
    have htail : ∀ k, (decodeList w).getD (k + 1) 0 = (decodeList (w - 1).unpair.2).
    getD k 0 := by
      intro k
      rcases w with _ | c
      · have hz : ((0 : ℕ) - 1).unpair.2 = 0 := by
          rw [Nat.zero_sub]; exact congrArg Prod.snd Nat.unpair_zero
        rw [decodeList_zero, hz, decodeList_zero, List.getD_nil, List.getD_nil]
      · rw [decodeList_succ, List.getD_cons_succ, Nat.add_sub_cancel]
    have hstep : List.foldl (reForallF qc) (Nat.pair w flag) (x :: l)
        = List.foldl (reForallF qc) (Nat.pair (w - 1).unpair.2
            (selectFn flag (isOne (qc (Nat.pair (w - 1).unpair.1 x))) 0)) l := by
      rw [List.foldl_cons]
      congr 1
      show reForallF qc (Nat.pair w flag) x = _
      unfold reForallF
      rw [unpair_pair_fst, unpair_pair_snd]
    rw [hstep]
    set flag' := selectFn flag (isOne (qc (Nat.pair (w - 1).unpair.1 x))) 0 with hflag
    'def
    have hflag'le : flag' ≤ 1 := by
      rw [hflag'def]
      rcases (show flag = 0 ∨ flag = 1 by omega) with h | h
      · rw [h, selectFn_zero]; exact Nat.zero_le 1
      · rw [h, selectFn_one]; exact isOne_le_one _
    rw [ih (w - 1).unpair.2 flag' hflag'le]
    constructor
    · rintro ⟨hf', hrest⟩
      have hsplit : flag = 1 ∧ qc (Nat.pair (w - 1).unpair.1 x) = 1 := by
        rw [hflag'def] at hf'
        rcases (show flag = 0 ∨ flag = 1 by omega) with h | h
        · rw [h, selectFn_zero] at hf'; exact absurd hf' (by decide)
        · rw [h, selectFn_one, isOne_eq_one_iff] at hf'; exact ⟨h, hf'⟩
      refine ⟨hsplit.1, fun k hk => ?_⟩
      rcases k with _ | k'
      · rw [List.getD_cons_zero, hi]; exact hsplit.2
      · rw [List.getD_cons_succ, htail]
        exact hrest k' (by simp only [List.length_cons] at hk; omega)
    · rintro ⟨hflag1, hall⟩
      have hhead : qc (Nat.pair (w - 1).unpair.1 x) = 1 := by
        have := hall 0 (by simp only [List.length_cons]; omega)
        rwa [List.getD_cons_zero, hi] at this
      refine ⟨?_, fun k hk => ?_⟩
      · rw [hflag'def, hflag1, selectFn_one, isOne_eq_one_iff]; exact hhead
      · have := hall (k + 1) (by simp only [List.length_cons]; omega)
        rwa [List.getD_cons_succ, htail] at this

theorem reForallChar_eq_one_iff (qc : ℕ → ℕ) (w c : ℕ) :
    reForallChar qc w c = 1 ↔
```

```
      ∀ k, k < (decodeList c).length →
        qc (Nat.pair ((decodeList w).getD k 0) ((decodeList c).getD k 0)) = 1 := by
  rw [reForallChar_eq, reForallF_foldl_eq_one_iff qc (decodeList c) w 1 (Nat.le_refl
    1)]
  simp only [true_and]

theorem primrec_reForallStp {qc : ℕ → ℕ} (hqc : Nat.Primrec qc) :
    Nat.Primrec (reForallStp qc) := by
  have hx : Nat.Primrec (fun w : ℕ => w.unpair.1) := Nat.Primrec.left
  have hacc : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hrw : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp hacc
  have hflag : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp hacc
  have hpm : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1.unpair.1 - 1) :=
    primrec_sub₂ hrw (Nat.Primrec.const 1)
  have hi : Nat.Primrec (fun w : ℕ => (w.unpair.2.unpair.1.unpair.1 - 1).unpair.1) :=
    Nat.Primrec.left.comp hpm
  have hrw' : Nat.Primrec (fun w : ℕ => (w.unpair.2.unpair.1.unpair.1 - 1).unpair.2)
    :=
    Nat.Primrec.right.comp hpm
  have hcall : Nat.Primrec (fun w : ℕ =>
      qc (Nat.pair (w.unpair.2.unpair.1.unpair.1 - 1).unpair.1 w.unpair.1)) :=
    hqc.comp (hi.pair hx)
  have hB : Nat.Primrec (fun w : ℕ =>
      selectFn w.unpair.2.unpair.1.unpair.2
        (isOne (qc (Nat.pair (w.unpair.2.unpair.1.unpair.1 - 1).unpair.1 w.unpair.1)))
     0) :=
    primrec_selectFn hflag (primrec_isOne.comp hcall) (Nat.Primrec.const 0)
  exact (hrw'.pair hB).of_eq (fun _ => rfl)

theorem primrec_reForallChar {qc : ℕ → ℕ} (hqc : Nat.Primrec qc) :
    Nat.Primrec (fun t => reForallChar qc t.unpair.1 t.unpair.2) := by
  have hfold := primrec_foldCode (primrec_reForallStp hqc) (Nat.Primrec.const 0)
    (Nat.Primrec.left.pair (Nat.Primrec.const 1)) Nat.Primrec.right
  exact (Nat.Primrec.right.comp hfold).of_eq (fun _ => rfl)

theorem REPred.forall_mem_decodeList {p : ℕ → Prop} (hp : REPred p) :
    REPred (fun c => ∀ e ∈ decodeList c, p e) := by
  obtain ⟨q, hq, hqe⟩ := hp
  obtain ⟨qc, hqcp, hqcs⟩ := hq

  have hpe : ∀ e, p e ↔ ∃ j, qc (Nat.pair j e) = 1 := by
    intro e; rw [hqe e]; exact exists_congr (fun j => hqcs (Nat.pair j e))

  have hmemgetD : ∀ (l : List ℕ) (e : ℕ), e ∈ l → ∃ k, k < l.length ∧ l.getD k 0 = e
    := by
    intro l
    induction l with
    | nil => intro e he; cases he
    | cons a l ih =>
      intro e he
      rcases List.mem_cons.mp he with rfl | he'
      · exact ⟨0, by simp only [List.length_cons]; omega, by rw [List.getD_cons_zero]⟩
      · obtain ⟨k, hk, hek⟩ := ih e he'
```

```
        exact ⟨k + 1, by simp only [List.length_cons]; omega, by rw [List.
    getD_cons_succ]; exact hek⟩
  refine ⟨fun t => reForallChar qc t.unpair.1 t.unpair.2 = 1,
    ⟨fun t => reForallChar qc t.unpair.1 t.unpair.2, primrec_reForallChar hqcp,
      fun _ => Iff.rfl⟩, fun c => ?_⟩
  simp only [unpair_pair_fst, unpair_pair_snd]
  constructor
  · intro hall

    have hwit : ∀ (l : List ℕ), (∀ e ∈ l, ∃ j, qc (Nat.pair j e) = 1) →
        ∃ iws : List ℕ, ∀ k, k < l.length →
          qc (Nat.pair (iws.getD k 0) (l.getD k 0)) = 1 := by
      intro l
      induction l with
      | nil => intro _; exact ⟨[], fun k hk => absurd hk (Nat.not_lt_zero k)⟩
      | cons e l ih =>
        intro hh
        obtain ⟨j, hj⟩ := hh e (List.mem_cons.mpr (Or.inl rfl))
        obtain ⟨iws, hiws⟩ := ih (fun e' he' => hh e' (List.mem_cons.mpr (Or.inr he')))
        refine ⟨j :: iws, fun k hk => ?_⟩
        rcases k with _ | k'
        · rw [List.getD_cons_zero, List.getD_cons_zero]; exact hj
        · rw [List.getD_cons_succ, List.getD_cons_succ]
          exact hiws k' (by simp only [List.length_cons] at hk; omega)
    obtain ⟨iws, hiws⟩ := hwit (decodeList c) (fun e he => (hpe e).mp (hall e he))
    refine ⟨encodeList iws, (reForallChar_eq_one_iff qc _ c).mpr (fun k hk => ?_)⟩
    rw [decodeList_encodeList]; exact hiws k hk
  · rintro ⟨w, hw⟩
    rw [reForallChar_eq_one_iff] at hw
    intro e he
    obtain ⟨k, hk, hek⟩ := hmemgetD (decodeList c) e he
    rw [hpe e]
    refine ⟨(decodeList w).getD k 0, ?_⟩
    have := hw k hk
    rwa [hek] at this

theorem lowOr_eq_mod (x y : ℕ) : lowOr x y = (x ||| y) % 2 := by
  have key : ((x ||| y) % 2 = 1) ↔ (x % 2 = 1 ∨ y % 2 = 1) := by
    have hb := Nat.testBit_lor x y 0
    rw [Nat.testBit_zero, Nat.testBit_zero, Nat.testBit_zero] at hb
    rw [← decide_eq_decide, hb, Bool.decide_or]
  have e1 : (x ||| y) % 2 < 2 := Nat.mod_lt _ (by decide)
  unfold lowOr

  rcases Nat.mod_two_eq_zero_or_one x with hx | hx <;>
    rcases Nat.mod_two_eq_zero_or_one y with hy | hy <;> rw [hx, hy]
  ·
    have hne : (x ||| y) % 2 = 1 := fun h => by rcases key.mp h with h' | h' <;> omega
    omega
  ·
    rw [key.mpr (Or.inr hy)]
  ·
    rw [key.mpr (Or.inl hx)]
  ·
    rw [key.mpr (Or.inl hx)]
```

```
theorem lor_low_rec (x y : ℕ) : x ||| y = 2 * (x / 2 ||| y / 2) + lowOr x y := by
  have hdiv : (x ||| y) / 2 = x / 2 ||| y / 2 := by
    apply Nat.eq_of_testBit_eq
    intro i
    rw [← Nat.testBit_add_one, Nat.testBit_lor, Nat.testBit_lor, Nat.testBit_add_one,
      Nat.testBit_add_one]
  have hmod := lowOr_eq_mod x y
  conv_lhs => rw [← Nat.div_add_mod (x ||| y) 2]
  rw [hdiv, hmod]

theorem lorStep_iter_spec (a b : ℕ) : ∀ k,
    (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.unpair.1 = a / 2 ^
     k ∧
    (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.unpair.2 = b / 2 ^
     k ∧
    (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.1 = 2 ^ k ∧
    (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.2 +
        2 ^ k * (a / 2 ^ k ||| b / 2 ^ k) = a ||| b := by
  intro k
  induction k with
  | zero =>
    simp only [Function.iterate_zero_apply, unpair_pair_fst, unpair_pair_snd, pow_zero
    ,
      Nat.div_one, Nat.one_mul, Nat.zero_add, true_and]
  | succ k ih =>
    obtain ⟨hA, hB, hW, hAcc⟩ := ih
    rw [Function.iterate_succ_apply']
    have hdd : a / 2 ^ (k + 1) = (a / 2 ^ k) / 2 := by rw [Nat.div_div_eq_div_mul, ←
    pow_succ]
    have hdd' : b / 2 ^ (k + 1) = (b / 2 ^ k) / 2 := by rw [Nat.div_div_eq_div_mul, ←
    pow_succ]

    have p11 : (lorStep (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0)))).unpair
    .1.unpair.1
        = (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.unpair.1 / 2
     := by
      unfold lorStep; rw [unpair_pair_fst, unpair_pair_fst]
    have p12 : (lorStep (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0)))).unpair
    .1.unpair.2
        = (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.unpair.2 / 2
     := by
      unfold lorStep; rw [unpair_pair_fst, unpair_pair_snd]
    have p21 : (lorStep (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0)))).unpair
    .2.unpair.1
        = 2 * (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.1
     := by
      unfold lorStep; rw [unpair_pair_snd, unpair_pair_fst]
    have p22 : (lorStep (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0)))).unpair
    .2.unpair.2
        = (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.2
          + (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.1
            * lowOr (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.
    unpair.1
                (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.unpair
    .2 := by
      unfold lorStep; rw [unpair_pair_snd, unpair_pair_snd]
```

```
    refine ⟨?_, ?_, ?_, ?_⟩
    · rw [p11, hA, Nat.div_div_eq_div_mul, ← pow_succ]
    · rw [p12, hB, Nat.div_div_eq_div_mul, ← pow_succ]
    · rw [p21, hW, ← pow_succ']
    · rw [p22, hA, hB, hW, hdd, hdd', pow_succ, ← hAcc, lor_low_rec (a / 2 ^ k) (b / 2
     ^ k)]
      ring

theorem myLor_eq_lor (a b : ℕ) : myLor a b = a ||| b := by


  unfold myLor
  obtain ⟨_, _, _, hAcc⟩ := lorStep_iter_spec a b (a + b)
  have ha0 : a / 2 ^ (a + b) = 0 :=
    Nat.div_eq_of_lt (Nat.lt_of_lt_of_le Nat.lt_two_pow_self (Nat.pow_le_pow_right (by
     decide)
      (Nat.le_add_right a b)))
  have hb0 : b / 2 ^ (a + b) = 0 :=
    Nat.div_eq_of_lt (Nat.lt_of_lt_of_le Nat.lt_two_pow_self (Nat.pow_le_pow_right (by
     decide)
      (Nat.le_add_left b a)))
  rw [ha0, hb0] at hAcc
  simpa using hAcc

theorem allListStp_eq (g : ℕ → ℕ) (acc x p : ℕ) :
    allListStp g (Nat.pair x (Nat.pair acc p)) = selectFn acc (isOne (g (Nat.pair x p)
    )) 0 := by
  unfold allListStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem allListChar_eq (g : ℕ → ℕ) (p c : ℕ) :
    allListChar g p c
      = List.foldl (fun acc x => selectFn acc (isOne (g (Nat.pair x p))) 0) 1 (
    decodeList c) := by
  have hfun : (fun (acc x : ℕ) => allListStp g (Nat.pair x (Nat.pair acc p)))
      = (fun acc x => selectFn acc (isOne (g (Nat.pair x p))) 0) := by
    funext acc x; exact allListStp_eq g acc x p
  unfold allListChar; rw [foldCode_eq', hfun]

theorem allList_foldl_eq_one_iff (g : ℕ → ℕ) (p : ℕ) :
    ∀ (l : List ℕ) (acc : ℕ), acc ≤ 1 →
      (List.foldl (fun acc x => selectFn acc (isOne (g (Nat.pair x p))) 0) acc l = 1 ↔
        acc = 1 ∧ ∀ x ∈ l, g (Nat.pair x p) = 1) := by
  intro l
  induction l with
  | nil => intro acc _; simp only [List.foldl_nil, List.not_mem_nil, false_implies,
    implies_true,
      and_true]
  | cons x l ih =>
    intro acc hacc
    rw [List.foldl_cons]
    set acc' := selectFn acc (isOne (g (Nat.pair x p))) 0 with hacc'
    have hacc'le : acc' ≤ 1 := by
      rw [hacc']
      rcases (show acc = 0 ∨ acc = 1 by omega) with h | h
      · rw [h, selectFn_zero]; exact Nat.zero_le 1
      · rw [h, selectFn_one]; exact isOne_le_one _
    rw [ih acc' hacc'le]
```

```
    constructor
    · rintro ⟨hacc'1, hrest⟩
      have hsplit : acc = 1 ∧ g (Nat.pair x p) = 1 := by
        rw [hacc'] at hacc'1
        rcases (show acc = 0 ∨ acc = 1 by omega) with h | h
        · rw [h, selectFn_zero] at hacc'1; exact absurd hacc'1 (by decide)
        · rw [h, selectFn_one, isOne_eq_one_iff] at hacc'1; exact ⟨h, hacc'1⟩
      refine ⟨hsplit.1, fun y hy => ?_⟩
      rcases List.mem_cons.mp hy with rfl | hy'
      · exact hsplit.2
      · exact hrest y hy'
    · rintro ⟨hacc1, hall⟩
      have hhead : g (Nat.pair x p) = 1 := hall x (List.mem_cons.mpr (Or.inl rfl))
      refine ⟨?_, fun y hy => hall y (List.mem_cons.mpr (Or.inr hy))⟩
      rw [hacc', hacc1, selectFn_one, isOne_eq_one_iff]; exact hhead

theorem allListChar_eq_one_iff (g : ℕ → ℕ) (p c : ℕ) :
    allListChar g p c = 1 ↔ ∀ x ∈ decodeList c, g (Nat.pair x p) = 1 := by
  rw [allListChar_eq, allList_foldl_eq_one_iff g p (decodeList c) 1 (Nat.le_refl 1)]
  simp only [true_and]

theorem allListChar_le_one (g : ℕ → ℕ) (p c : ℕ) : allListChar g p c ≤ 1 := by
  have h : ∀ (l : List ℕ) (z : ℕ), z ≤ 1 →
      List.foldl (fun acc x => allListStp g (Nat.pair x (Nat.pair acc p))) z l ≤ 1 :=
    by
    intro l
    induction l with
    | nil => intro z hz; simpa using hz
    | cons x xs ih =>
      intro z hz
      simp only [List.foldl_cons]
      apply ih
      rw [allListStp_eq]
      rcases (show z = 0 ∨ z = 1 by omega) with h0 | h1
      · rw [h0, selectFn_zero]; exact Nat.zero_le 1
      · rw [h1, selectFn_one]; exact isOne_le_one _
  unfold allListChar
  rw [foldCode_eq']
  exact h (decodeList c) 1 (le_refl 1)

theorem primrec_allListStp {g : ℕ → ℕ} (hg : Nat.Primrec g) : Nat.Primrec (allListStp
    g) := by
  have hacc : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hx : Nat.Primrec (fun w : ℕ => w.unpair.1) := Nat.Primrec.left
  have hp : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hcall : Nat.Primrec (fun w : ℕ => isOne (g (Nat.pair w.unpair.1 w.unpair.2.
    unpair.2))) :=
    primrec_isOne.comp (hg.comp (hx.pair hp))
  exact (primrec_selectFn hacc hcall (Nat.Primrec.const 0)).of_eq fun _ => rfl

theorem isBinDigit_eq_one_iff (n : ℕ) : isBinDigit n = 1 ↔ n = 0 ∨ n = 1 := by
  unfold isBinDigit selectFn isOne
  match n with | 0 | 1 => simp | n + 2 => simp
```

```
theorem primrec_isBinDigit : Nat.Primrec isBinDigit := by
  have h01 := primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 1) primrec_id)
  have h12 := primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 2) primrec_id)
  refine (primrec_selectFn h01 (Nat.Primrec.const 1)
    (primrec_selectFn h12 (Nat.Primrec.const 1) (Nat.Primrec.const 0))).of_eq fun _ =>
     rfl

theorem allBinDigitsChar_eq_one_iff (c : ℕ) :
    allBinDigitsChar c = 1 ↔ ∀ x ∈ decodeList c, x = 0 ∨ x = 1 := by
  unfold allBinDigitsChar
  rw [allListChar_eq_one_iff]
  simp [isBinDigit_eq_one_iff]

theorem allBinDigitsChar_le_one (c : ℕ) : allBinDigitsChar c ≤ 1 := by
  unfold allBinDigitsChar; exact allListChar_le_one _ _ _

set_option maxHeartbeats 800000 in
theorem primrec_allBinDigitsChar : Nat.Primrec allBinDigitsChar := by
  unfold allBinDigitsChar
  exact (primrec_foldCode (primrec_allListStp (primrec_isBinDigit.comp Nat.Primrec.
    left))
    (Nat.Primrec.const 0) (Nat.Primrec.const 1) primrec_id).of_eq fun _ => rfl

theorem nthStp_val (i x s r : ℕ) :
    nthStp (Nat.pair x (Nat.pair (Nat.pair s r) i))
      = Nat.pair (s + 1) (selectFn (1 - ((s - i) + (i - s))) x r) := by
  unfold nthStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem nthCode_foldl (i : ℕ) : ∀ (l : List ℕ) (s r : ℕ),
    (List.foldl (fun acc x => nthStp (Nat.pair x (Nat.pair acc i))) (Nat.pair s r) l).
    unpair.2
      = if i < s then r else l.getD (i - s) r := by
  intro l
  induction l with
  | nil => intro s r; simp only [List.foldl_nil, unpair_pair_snd, List.getD_nil];
    split <;> rfl
  | cons x l ih =>
    intro s r
    rw [List.foldl_cons, nthStp_val, ih (s + 1) (selectFn (1 - ((s - i) + (i - s))) x
    r)]
    by_cases h : i = s
    · subst h
      have he : (1 : ℕ) - ((i - i) + (i - i)) = 1 := by omega
      rw [he, selectFn_one, ite_eq_right (lt_irrefl i), ite_eq_left (Nat.lt_succ_self
    i), Nat.sub_self,
        List.getD_cons_zero]
    · by_cases h2 : i < s
      · have he : (1 : ℕ) - ((s - i) + (i - s)) = 0 := by omega
        rw [he, selectFn_zero, ite_eq_left h2, ite_eq_left (Nat.lt_succ_of_lt h2)]
      · have he : (1 : ℕ) - ((s - i) + (i - s)) = 0 := by omega
        rw [he, selectFn_zero, ite_eq_right h2, ite_eq_right (show ¬ i < s + 1 by
    omega),
          show i - s = (i - (s + 1)) + 1 by omega, List.getD_cons_succ]

theorem nthCode_eq (c i d : ℕ) : nthCode c i d = (decodeList c).getD i d := by
  unfold nthCode
```

```
  rw [foldCode_eq']
  have h := nthCode_foldl i (decodeList c) 0 d
  rw [ite_eq_right (Nat.not_lt_zero i), Nat.sub_zero] at h
  exact h

theorem primrec_nthStp : Nat.Primrec nthStp := by
  have hcount : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hi : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hx : Nat.Primrec (fun w : ℕ => w.unpair.1) := Nat.Primrec.left
  have hres : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hbit : Nat.Primrec (fun w : ℕ => 1 - ((w.unpair.2.unpair.1.unpair.1 - w.unpair
    .2.unpair.2)
      + (w.unpair.2.unpair.2 - w.unpair.2.unpair.1.unpair.1))) :=
    primrec_sub₂ (Nat.Primrec.const 1)
      (primrec_add₂ (primrec_sub₂ hcount hi) (primrec_sub₂ hi hcount))
  exact ((Nat.Primrec.succ.comp hcount).pair (primrec_selectFn hbit hx hres)).of_eq
    fun _ => rfl

theorem primrec_nthCode : Nat.Primrec
    (fun t => nthCode t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2) := by
  have hfold := primrec_foldCode primrec_nthStp (Nat.Primrec.left.comp Nat.Primrec.
    right)
    ((Nat.Primrec.const 0).pair (Nat.Primrec.right.comp Nat.Primrec.right)) Nat.
    Primrec.left
  exact (Nat.Primrec.right.comp hfold).of_eq fun _ => rfl

theorem tabStep_val (g : ℕ → ℕ) (a y IH : ℕ) :
    tabStep g (Nat.pair a (Nat.pair y IH))
      = Nat.pair (g (Nat.pair (a.unpair.1 - 1 - y) a.unpair.2)) IH + 1 := by
  unfold tabStep; simp only [unpair_pair_fst, unpair_pair_snd]

theorem decodeList_tabCode_aux (g : ℕ → ℕ) (B p : ℕ) :
    ∀ K, K ≤ B → decodeList (Nat.rec (motive := fun _ => ℕ) 0
        (fun y IH => tabStep g (Nat.pair (Nat.pair B p) (Nat.pair y IH))) K)
      = (List.range' (B - K) K).map (fun j => g (Nat.pair j p)) := by
  intro K
  induction K with
  | zero => intro _; simp [decodeList_zero]
  | succ K ih =>
    intro hK
    have hKB : K ≤ B := Nat.le_of_succ_le hK
    show decodeList (tabStep g (Nat.pair (Nat.pair B p) (Nat.pair K
      (Nat.rec (motive := fun _ => ℕ) 0
        (fun y IH => tabStep g (Nat.pair (Nat.pair B p) (Nat.pair y IH))) K)))) = _
    rw [tabStep_val]
    simp only [unpair_pair_fst, unpair_pair_snd]
    rw [decodeList_succ, unpair_pair_fst, unpair_pair_snd, ih hKB,
      show B - (K + 1) = B - 1 - K by omega, List.range'_succ, List.map_cons,
      show B - 1 - K + 1 = B - K by omega]

theorem decodeList_tabCode (g : ℕ → ℕ) (B p : ℕ) :
    decodeList (tabCode g (Nat.pair B p) B) = (List.range B).map (fun j => g (Nat.pair
     j p)) := by
```

```
  have h := decodeList_tabCode_aux g B p B (le_refl B)
  rw [Nat.sub_self, ← List.range_eq_range'] at h
  exact h

theorem getD_eq_getElem_cf {β : Type*} (l : List β) (d : β) :
    ∀ {n : ℕ} (h : n < l.length), l.getD n d = l[n] := by
  induction l with
  | nil => intro n h; exact absurd h (Nat.not_lt_zero n)
  | cons a t ih =>
    intro n h
    cases n with
    | zero => rw [List.getD_cons_zero, List.getElem_cons_zero]
    | succ m => rw [List.getD_cons_succ, List.getElem_cons_succ]; exact ih _

theorem getD_eq_default_cf {β : Type*} (l : List β) (d : β) :
    ∀ {n : ℕ}, l.length ≤ n → l.getD n d = d := by
  induction l with
  | nil => intro n _; exact List.getD_nil
  | cons a t ih =>
    intro n h
    cases n with
    | zero => exact absurd h (by rw [List.length_cons]; exact Nat.not_succ_le_zero _)
    | succ m => rw [List.getD_cons_succ]; exact ih (by rw [List.length_cons] at h;
    omega)

theorem getD_map_range_cf {β : Type*} (h : ℕ → β) (d : β) {B i : ℕ} (hi : i < B) :
    ((List.range B).map h).getD i d = h i := by
  have hlen : ((List.range B).map h).length = B := by rw [List.length_map, List.
    length_range]
  rw [getD_eq_getElem_cf _ d (by rw [hlen]; exact hi), List.getElem_map, List.
    getElem_range]

theorem tabCode_nth_lt {g : ℕ → ℕ} {B p i : ℕ} (hi : i < B) (d : ℕ) :
    nthCode (tabCode g (Nat.pair B p) B) i d = g (Nat.pair i p) := by
  rw [nthCode_eq, decodeList_tabCode, getD_map_range_cf]; exact hi

theorem tabCode_nth_ge {g : ℕ → ℕ} {B p i : ℕ} (hi : B ≤ i) (d : ℕ) :
    nthCode (tabCode g (Nat.pair B p) B) i d = d := by
  rw [nthCode_eq, decodeList_tabCode, getD_eq_default_cf]
  rw [List.length_map, List.length_range]; exact hi

theorem primrec_tabStep {g : ℕ → ℕ} (hg : Nat.Primrec g) : Nat.Primrec (tabStep g) :=
    by
  have hB : Nat.Primrec (fun w : ℕ => w.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.left
  have hp : Nat.Primrec (fun w : ℕ => w.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have hk : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hih : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hidx : Nat.Primrec (fun w : ℕ => w.unpair.1.unpair.1 - 1 - w.unpair.2.unpair.1)
    :=
    primrec_sub₂ (primrec_sub₂ hB (Nat.Primrec.const 1)) hk
  exact (Nat.Primrec.succ.comp ((hg.comp (hidx.pair hp)).pair hih)).of_eq fun _ => rfl
```

```
theorem primrec_tabCode {g : ℕ → ℕ} (hg : Nat.Primrec g) :
    Nat.Primrec (fun t => tabCode g t.unpair.1 t.unpair.2) :=
  (Nat.Primrec.prec (Nat.Primrec.const 0) (primrec_tabStep hg)).of_eq fun t => by
    simp only [Nat.unpaired]; rfl

theorem primrec_tagCase4 {f0 f1 f2 f3 fdef : ℕ → ℕ}
    (hf0 : Nat.Primrec f0) (hf1 : Nat.Primrec f1) (hf2 : Nat.Primrec f2) (hf3 : Nat.
    Primrec f3)
    (hfdef : Nat.Primrec fdef) :
    Nat.Primrec (fun e =>
      let t := e.unpair.1
      selectFn (isOne (1 - t)) (f0 e)
        (selectFn (isOne (2 - t)) (f1 e)
          (selectFn (isOne (3 - t)) (f2 e)
            (selectFn (isOne (4 - t)) (f3 e) (fdef e))))) := by
  have ht : Nat.Primrec (fun e => e.unpair.1) := Nat.Primrec.left
  have h01 : Nat.Primrec (fun e => isOne (1 - e.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 1) ht)
  have h12 : Nat.Primrec (fun e => isOne (2 - e.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 2) ht)
  have h23 : Nat.Primrec (fun e => isOne (3 - e.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 3) ht)
  have h34 : Nat.Primrec (fun e => isOne (4 - e.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 4) ht)
  refine (primrec_selectFn h01 hf0
    (primrec_selectFn h12 hf1
      (primrec_selectFn h23 hf2
        (primrec_selectFn h34 hf3 hfdef)))).of_eq fun e => rfl

theorem mulBit_eq_one_iff (a b : ℕ) : mulBit a b = 1 ↔ a = 1 ∧ b = 1 := by
  simp [mulBit, nat_mul_eq_one]

theorem mulBit_le_one {a b : ℕ} (ha : a ≤ 1) (hb : b ≤ 1) : mulBit a b ≤ 1 := by
  unfold mulBit
  rcases (show a = 0 ∨ a = 1 by omega) with rfl | rfl <;>
    rcases (show b = 0 ∨ b = 1 by omega) with rfl | rfl <;> simp

theorem primrec_mulBit : Nat.Primrec (fun t => mulBit t.unpair.1 t.unpair.2) :=
  primrec_mul₂ Nat.Primrec.left Nat.Primrec.right

theorem decodeFuelOkCharBody_eq (prev : ℕ → ℕ) (c : ℕ) :
    decodeFuelOkCharBody prev c =
      match c.unpair.1 with
      | 0 => selectFn (isOne (1 - c.unpair.2)) 1 0
      | 1 => allBinDigitsChar c.unpair.2
      | 2 | 3 => mulBit (prev c.unpair.2.unpair.1) (prev c.unpair.2.unpair.2)
      | _ => 0 := by
  unfold decodeFuelOkCharBody mulBit
  match tag : c.unpair.1 with
  | 0 =>
    have h10 : (1 - 0 : ℕ) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01 : (1 - 1 : ℕ) = 0 := rfl
    have h12 : (2 - 1 : ℕ) = 1 := rfl
    simp only [h01, h12, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
```

```
  | 2 =>
    have h01 : (1 - 2 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 2 : ℕ) = 0 := rfl
    have h23 : (3 - 2 : ℕ) = 1 := rfl
    simp only [h01, h23, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 3 =>
    have h01 : (1 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23 : (3 - 3 : ℕ) = 0 := rfl
    have h34 : (4 - 3 : ℕ) = 1 := rfl
    simp only [h01, h34, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | t + 4 =>
    have hne1 : isOne (1 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne2 : isOne (2 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne3 : isOne (3 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne4 : isOne (4 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    simp only [hne1, hne2, hne3, hne4, selectFn_zero]

theorem primrec_decodeFuelOkCharBody {prev : ℕ → ℕ} (hprev : Nat.Primrec prev) :
    Nat.Primrec (fun c => decodeFuelOkCharBody prev c) := by
  unfold decodeFuelOkCharBody
  have hleft : Nat.Primrec (fun c => prev c.unpair.2.unpair.1) :=
    (hprev.comp (Nat.Primrec.left.comp Nat.Primrec.right)).of_eq fun c => by
      simp only
  have hright : Nat.Primrec (fun c => prev c.unpair.2.unpair.2) :=
    (hprev.comp (Nat.Primrec.right.comp Nat.Primrec.right)).of_eq fun c => by
      simp only
  have hsub : Nat.Primrec (fun c => mulBit (prev c.unpair.2.unpair.1) (prev c.unpair
    .2.unpair.2)) :=
    primrec_mul₂ hleft hright
  have hf0 : Nat.Primrec (fun c => selectFn (isOne (1 - c.unpair.2)) 1 0) :=
    primrec_selectFn
      (primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 1) Nat.Primrec.right))
      (Nat.Primrec.const 1) (Nat.Primrec.const 0)
  have hf1 : Nat.Primrec (fun c => allBinDigitsChar c.unpair.2) :=
    primrec_allBinDigitsChar.comp Nat.Primrec.right
  exact primrec_tagCase4 hf0 hf1 hsub hsub (Nat.Primrec.const 0)

theorem decodeFuelOkCharBody_local (f g : ℕ → ℕ) (c : ℕ) (h : ∀ c' ≤ c, f c' = g c')
    :
    decodeFuelOkCharBody f c = decodeFuelOkCharBody g c := by
  rw [decodeFuelOkCharBody_eq, decodeFuelOkCharBody_eq]
  have h1 : c.unpair.2.unpair.1 ≤ c := le_trans (unpair_left_le c.unpair.2) (
    unpair_snd_le c)
  have h2 : c.unpair.2.unpair.2 ≤ c := le_trans (unpair_snd_le c.unpair.2) (
    unpair_snd_le c)
  rw [h _ h1, h _ h2]

theorem listLenStp_eq (acc x p : ℕ) :
    listLenStp (Nat.pair x (Nat.pair acc p)) = acc + 1 := by
  unfold listLenStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem listLenChar_eq (c : ℕ) : listLenChar c = (decodeList c).length := by
  have hfun : (fun acc x => listLenStp (Nat.pair x (Nat.pair acc 0))) = fun acc _ =>
    acc + 1 := by
    funext acc x; simp [listLenStp_eq]
```

```
  have hfold : ∀ (k : ℕ) (l : List ℕ),
      List.foldl (fun acc _ => acc + 1) k l = k + l.length := by
    intro k l
    induction l generalizing k with
    | nil => simp
    | cons _ xs ih => simp [List.foldl_cons, ih, List.length_cons]; omega
  unfold listLenChar
  rw [foldCode_eq', hfun]
  simp [hfold]

theorem primrec_listLenStp : Nat.Primrec listLenStp :=
  primrec_add₂ (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)

theorem primrec_listLenChar : Nat.Primrec listLenChar := by
  unfold listLenChar
  exact (primrec_foldCode primrec_listLenStp (Nat.Primrec.const 0) (Nat.Primrec.const
    0)
    primrec_id).of_eq fun _ => rfl

theorem natEqChar_le_one (a b : ℕ) : natEqChar a b ≤ 1 := by unfold natEqChar; omega

theorem natEqChar_eq_zero_or_one (a b : ℕ) : natEqChar a b = 0 ∨ natEqChar a b = 1 :=
    by
  unfold natEqChar; omega

theorem primrec_natEqChar : Nat.Primrec (fun t => natEqChar t.unpair.1 t.unpair.2) :=
  (primrec_sub₂ (Nat.Primrec.const 1)
    (primrec_add₂ (primrec_sub₂ Nat.Primrec.left Nat.Primrec.right)
      (primrec_sub₂ Nat.Primrec.right Nat.Primrec.left))).of_eq fun _ => rfl

theorem listEqStp_acc (flag rem x : ℕ) :
    listEqStp (Nat.pair x (Nat.pair (Nat.pair flag rem) 0)) =
      selectFn (isZero rem) (Nat.pair 0 0)
        (Nat.pair (selectFn flag (natEqChar x ((rem - 1).unpair.1)) 0) ((rem - 1).
    unpair.2)) := by
  unfold listEqStp listEqStpNonzero
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem decodeList_eq_nil_iff (c : ℕ) : decodeList c = [] ↔ c = 0 := by
  constructor
  · intro h
    have := congrArg encodeList h
    rwa [encodeList, encodeList_decodeList] at this
  · intro hc; simp [hc, decodeList_zero]

private theorem selectFn_eq_one_iff_zero {a b : ℕ} (ha : a = 0 ∨ a = 1) :
    selectFn a b 0 = 1 ↔ a = 1 ∧ b = 1 := by
  rcases ha with ha | ha <;> simp [selectFn, ha]

private theorem listEqStep_zero (flag x : ℕ) :
    listEqStep (Nat.pair flag 0) x = Nat.pair 0 0 := by
  simp [listEqStep, listEqStp_acc, isZero, min]


private theorem listEqStep_succ (flag x m : ℕ) :
    listEqStep (Nat.pair flag (m + 1)) x =
```

```
      Nat.pair (selectFn flag (natEqChar x m.unpair.1) 0) m.unpair.2 := by
  unfold listEqStep listEqStp listEqStpNonzero
  simp only [unpair_pair_fst, unpair_pair_snd]
  have hzero : isZero (m + 1) = 0 := by
    unfold isZero; simp
  rw [hzero, selectFn_zero, Nat.add_sub_cancel]

private theorem listEq_foldl_zero (flag x : ℕ) (xs : List ℕ) :
    (List.foldl listEqStep (Nat.pair flag 0) (x :: xs)).unpair.1 = 0 := by
  rw [List.foldl_cons, listEqStep_zero]
  induction xs with
  | nil => simp [List.foldl_nil]
  | cons y ys ih =>
    rw [List.foldl_cons, listEqStep_zero, ih]

private theorem listEq_foldl_end_iff (l1 : List ℕ) (flag rem : ℕ) (hflag : flag = 0 ∨
    flag = 1) :
    (let r := List.foldl listEqStep (Nat.pair flag rem) l1
     r.unpair.1 = 1 ∧ isZero r.unpair.2 = 1) ↔
      flag = 1 ∧ l1 = decodeList rem := by
  induction l1 generalizing flag rem with
  | nil =>
    simp only [List.foldl_nil, unpair_pair_fst, unpair_pair_snd]
    constructor
    · intro ⟨hflag', hrem0⟩
      have hrem0' : rem = 0 := (isZero_eq_one_iff rem).1 hrem0
      exact ⟨hflag', ((decodeList_eq_nil_iff rem).2 hrem0').symm⟩
    · intro ⟨hflag', heq⟩
      exact ⟨hflag', (isZero_eq_one_iff rem).2 ((decodeList_eq_nil_iff rem).1 heq.symm)
    ⟩
  | cons x xs ih =>
    by_cases hrem : rem = 0
    · subst hrem
      simp only [decodeList_zero]
      constructor
      · intro ⟨h1, _⟩
        have h0 : (let r := List.foldl listEqStep (Nat.pair flag 0) (x :: xs); r.
    unpair.1) = 0 := by
          show (List.foldl listEqStep (Nat.pair flag 0) (x :: xs)).unpair.1 = 0
          exact listEq_foldl_zero flag x xs
        rw [h0] at h1
        exact absurd h1 (by decide)
      · intro h
        cases h.2
    · rcases rem with _ | m
      · omega
      · set acc' := selectFn flag (natEqChar x m.unpair.1) 0 with hacc'
        have hacc01 : acc' = 0 ∨ acc' = 1 := by
          rcases hflag with h | h
          · left; rw [hacc', h, selectFn_zero]
          · have hz := natEqChar_eq_zero_or_one x m.unpair.1
            rw [hacc', h, selectFn_one]
            rcases hz with hz0 | hz1
            · exact Or.inl hz0
            · exact Or.inr hz1
        rw [List.foldl_cons, listEqStep_succ, ih acc' m.unpair.2 hacc01,
```

```
    decodeList_succ]
        constructor
        · intro ⟨hacc'1, hxs⟩
          have hsel : selectFn flag (natEqChar x m.unpair.1) 0 = 1 := hacc'.trans hacc
    '1
          rcases (selectFn_eq_one_iff_zero hflag).1 hsel with ⟨hflag1, hxeq⟩
          rw [natEqChar_eq_one_iff] at hxeq
          exact ⟨hflag1, by rw [hxeq, hxs]⟩
        · intro ⟨hflag1, heq⟩
          rcases List.cons.inj heq with ⟨hx, hxs⟩
          refine ⟨hacc'.trans ((selectFn_eq_one_iff_zero hflag).2 ⟨hflag1, by
            rw [natEqChar_eq_one_iff, hx]⟩), hxs⟩

theorem listEqChar_eq_one_iff (c1 c2 : ℕ) :
    listEqChar c1 c2 = 1 ↔ decodeList c1 = decodeList c2 := by
  unfold listEqChar
  rw [foldCode_eq', show (fun acc x => listEqStp (Nat.pair x (Nat.pair acc 0))) =
    listEqStep from rfl,
    mulBit_eq_one_iff]
  exact (listEq_foldl_end_iff (decodeList c1) 1 c2 (Or.inr rfl)).trans (by simp)

private theorem listEq_foldl_flag_le_one : ∀ (l : List ℕ) (z : ℕ), z.unpair.1 ≤ 1 →
    (List.foldl listEqStep z l).unpair.1 ≤ 1 := by
  intro l
  induction l with
  | nil => intro z hz; exact hz
  | cons x xs ih =>
    intro z hz
    simp only [List.foldl_cons]
    apply ih
    show (listEqStp (Nat.pair x (Nat.pair z 0))).unpair.1 ≤ 1
    have hkey := listEqStp_acc z.unpair.1 z.unpair.2 x
    rw [Nat.pair_unpair] at hkey
    rw [hkey]
    rcases (show isZero z.unpair.2 = 0 ∨ isZero z.unpair.2 = 1 from by
      have := isZero_le_one z.unpair.2; omega) with h0 | h1
    · rw [h0, selectFn_zero, unpair_pair_fst]
      rcases (show z.unpair.1 = 0 ∨ z.unpair.1 = 1 from by omega) with hf0 | hf1
      · rw [hf0, selectFn_zero]; exact Nat.zero_le 1
      · rw [hf1, selectFn_one]
        have := natEqChar_le_one x (z.unpair.2 - 1).unpair.1
        omega
    · rw [h1, selectFn_one, unpair_pair_fst]; exact Nat.zero_le 1

theorem listEqChar_le_one (c1 c2 : ℕ) : listEqChar c1 c2 ≤ 1 := by
  unfold listEqChar
  rw [foldCode_eq', show (fun acc x => listEqStp (Nat.pair x (Nat.pair acc 0))) =
    listEqStep from rfl]
  exact mulBit_le_one
    (listEq_foldl_flag_le_one (decodeList c1) (Nat.pair 1 c2) (by simp))
    (isZero_le_one _)

set_option maxHeartbeats 800000 in
theorem primrec_listEqStpNonzero : Nat.Primrec listEqStpNonzero := by
  have hflag := Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hrem := Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)
```

```
  have hhead := Nat.Primrec.left
  have hrem1 := primrec_sub₂ hrem (Nat.Primrec.const 1)
  have hcmp := primrec_natEqChar.comp (hhead.pair (Nat.Primrec.left.comp hrem1))
  have htail := Nat.Primrec.right.comp hrem1
  exact (Nat.Primrec.pair (primrec_selectFn hflag hcmp (Nat.Primrec.const 0)) htail).
    of_eq fun w => by
    simp [listEqStpNonzero, selectFn]

set_option maxHeartbeats 800000 in
theorem primrec_listEqStp : Nat.Primrec listEqStp := by
  have hrem := Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hzero := Nat.Primrec.pair (Nat.Primrec.const 0) (Nat.Primrec.const 0)
  exact (primrec_selectFn (primrec_isZero.comp hrem) hzero primrec_listEqStpNonzero).
    of_eq fun w => by
    simp [listEqStp, selectFn, isZero, min]

theorem primrec_listEqChar : Nat.Primrec (fun t => listEqChar t.unpair.1 t.unpair.2)
    := by
  have hfold := primrec_foldCode primrec_listEqStp (Nat.Primrec.const 0)
    (Nat.Primrec.pair (Nat.Primrec.const 1) Nat.Primrec.right) Nat.Primrec.left
  have hflag : Nat.Primrec (fun t =>
      (foldCode listEqStp 0 (Nat.pair 1 t.unpair.2) t.unpair.1).unpair.1) :=
    Nat.Primrec.left.comp hfold
  have hrem : Nat.Primrec (fun t =>
      (foldCode listEqStp 0 (Nat.pair 1 t.unpair.2) t.unpair.1).unpair.2) :=
    Nat.Primrec.right.comp hfold
  exact (primrec_mulBit.comp (hflag.pair (primrec_isZero.comp hrem))).of_eq fun t =>
    by
    simp [listEqChar]

theorem list_eq_of_getD {l1 l2 : List ℕ} (h : ∀ i d, l1.getD i d = l2.getD i d) : l1 =
    l2 := by
  have hlen : l1.length = l2.length := by
    rcases Nat.lt_trichotomy l1.length l2.length with hlt | heq | hgt
    · have h1 := h l1.length 1
      have h2 := h l1.length 2
      rw [getD_eq_default_cf l1 _ (Nat.le_refl _), getD_eq_getElem_cf l2 _ (by omega)]
      at h1 h2
      omega
    · exact heq
    · have h1 := h l2.length 1
      have h2 := h l2.length 2
      rw [getD_eq_getElem_cf l1 _ (by omega), getD_eq_default_cf l2 _ (Nat.le_refl _)]
      at h1 h2
      omega
  apply List.ext_getElem hlen
  intro i hi _
  have hi' : i < l2.length := by rw [← hlen]; exact hi
  rw [← getD_eq_getElem_cf l1 0 hi, ← getD_eq_getElem_cf l2 0 hi', h i 0]

theorem getD_take_cf (l : List ℕ) (n i d : ℕ) (hi : i < (l.take n).length) :
    (l.take n).getD i d = l.getD i d := by
  have hlen : i < l.length := by
    rw [List.length_take] at hi
    exact Nat.lt_of_lt_of_le hi (Nat.min_le_right n l.length)
  rw [getD_eq_getElem_cf (l.take n) d hi, getD_eq_getElem_cf l d hlen, List.
```

```
    getElem_take]

theorem getD_drop_cf (l : List ℕ) (n i d : ℕ) (hi : i < (l.drop n).length) :
    (l.drop n).getD i d = l.getD (i + n) d := by
  induction n generalizing l i d with
  | zero =>
    simp [List.drop_zero] at hi ⊢
  | succ n ih =>
    cases l with
    | nil => simp [List.drop, List.length_nil] at hi
    | cons a tl =>
      cases i with
      | zero =>
        simp [List.drop]
      | succ i =>
        simp only [List.drop]
        exact ih tl (i + 1) d (by simpa [List.length_drop] using hi)

private theorem isZero_succ_sub_len1 (i len1 : ℕ) :
    isZero ((i + 1) - len1) = 1 ↔ i < len1 := by
  unfold isZero
  constructor
  · intro h
    by_contra hge
    have hpos : 0 < (i + 1) - len1 := Nat.sub_pos_of_lt (Nat.lt_succ_of_le (Nat.
    le_of_not_gt hge))
    have hmin : min ((i + 1) - len1) 1 = 1 := Nat.min_eq_right (Nat.succ_le_iff.mpr
    hpos)
    omega
  · intro hlt
    have : (i + 1) - len1 = 0 := Nat.sub_eq_zero_iff_le.mpr (Nat.succ_le_iff.mpr hlt)
    simp [this, min]

private theorem isZero_succ_sub_len1_zero (i len1 : ℕ) (hge : len1 ≤ i) :
    isZero ((i + 1) - len1) = 0 := by
  unfold isZero
  have hpos : 0 < (i + 1) - len1 := Nat.sub_pos_of_lt (Nat.lt_succ_of_le hge)
  have hmin : min ((i + 1) - len1) 1 = 1 := Nat.min_eq_right (Nat.succ_le_iff.mpr hpos
    )
  simp [hmin]

theorem appendListTabFn_eq (c1 c2 len1 i : ℕ) :
    appendListTabFn (Nat.pair i (Nat.pair c1 (Nat.pair c2 len1))) =
      if i < len1 then
        (decodeList c1).getD i 0
      else
        (decodeList c2).getD (i - len1) 0 := by
  unfold appendListTabFn
  simp only [unpair_pair_fst, unpair_pair_snd]
  by_cases hlt : i < len1
  · rw [ite_eq_left hlt, (isZero_succ_sub_len1 i len1).2 hlt, selectFn_one, nthCode_eq]
  · rw [ite_eq_right hlt, isZero_succ_sub_len1_zero i len1 (Nat.le_of_not_gt hlt),
    selectFn_zero,
      nthCode_eq]

theorem takeCode_eq (n c : ℕ) :
```

```
    decodeList (takeCode n c) = (decodeList c).take n := by
  unfold takeCode
  rw [decodeList_tabCode, listLenChar_eq c]
  apply list_eq_of_getD
  intro i d
  by_cases hi : i < Nat.min n (decodeList c).length
  · have hic : i < (decodeList c).length := Nat.lt_of_lt_of_le hi (Nat.min_le_right n _
    )
    rw [getD_map_range_cf (fun j => takeListTabFn (Nat.pair j (Nat.pair n c))) d hi,
      show takeListTabFn (Nat.pair i (Nat.pair n c)) = (decodeList c).getD i d by
        have hc : (Nat.pair i (Nat.pair n c)).unpair.2.unpair.2 = c := by simp
        have hi' : (Nat.pair i (Nat.pair n c)).unpair.1 = i := by simp
        rw [takeListTabFn, hc, hi', nthCode_eq,
          show (decodeList c).getD i 0 = (decodeList c).getD i d from by
            rw [getD_eq_getElem_cf _ 0 hic, getD_eq_getElem_cf _ d hic]],
      getD_take_cf (decodeList c) n i d (by rwa [List.length_take])]
  · have hge : Nat.min n (decodeList c).length ≤ i := Nat.le_of_not_gt hi
    rw [getD_eq_default_cf _ d (by rw [List.length_map, List.length_range]; exact hge)
    ,
      getD_eq_default_cf _ d (by rw [List.length_take]; exact hge)]

theorem dropCode_eq (n c : ℕ) :
    decodeList (dropCode n c) = (decodeList c).drop n := by
  unfold dropCode
  rw [decodeList_tabCode, listLenChar_eq c]
  apply list_eq_of_getD
  intro i d
  by_cases hi : i < (decodeList c).length - n
  · have hic : i + n < (decodeList c).length := Nat.add_lt_of_lt_sub hi
    rw [getD_map_range_cf (fun j => dropListTabFn (Nat.pair j (Nat.pair n c))) d hi,
      show dropListTabFn (Nat.pair i (Nat.pair n c)) = (decodeList c).getD (i + n) d
    by
        have hc : (Nat.pair i (Nat.pair n c)).unpair.2.unpair.2 = c := by simp
        have hn : (Nat.pair i (Nat.pair n c)).unpair.2.unpair.1 = n := by simp
        have hi' : (Nat.pair i (Nat.pair n c)).unpair.1 = i := by simp
        rw [dropListTabFn, hi', hn, hc, nthCode_eq,
          show (decodeList c).getD (i + n) 0 = (decodeList c).getD (i + n) d from by
            rw [getD_eq_getElem_cf _ 0 hic, getD_eq_getElem_cf _ d hic]],
      getD_drop_cf (decodeList c) n i d (by rw [List.length_drop]; exact hi)]
  · have hge : (decodeList c).length - n ≤ i := Nat.le_of_not_gt hi
    rw [getD_eq_default_cf _ d (by rw [List.length_map, List.length_range]; exact hge)
    ,
      getD_eq_default_cf _ d (by rw [List.length_drop]; exact hge)]

theorem encodeList_take_le (l : List ℕ) : ∀ n, encodeList (l.take n) ≤ encodeList l :=
     by
  induction l with
  | nil => intro n; simp [encodeList]
  | cons a l ih =>
    intro n
    cases n with
    | zero => simp [encodeList]
    | succ n =>
      show encodeList (a :: l.take n) ≤ encodeList (a :: l)
      simp only [encodeList]
      exact Nat.add_le_add_right (pair_le_pair_right' a (ih n)) 1
```

```
theorem encodeList_drop_le (l : List ℕ) : ∀ n, encodeList (l.drop n) ≤ encodeList l :=
     by
  induction l with
  | nil => intro n; simp [encodeList]
  | cons a l ih =>
    intro n
    cases n with
    | zero => simp [encodeList]
    | succ n =>
      show encodeList (l.drop n) ≤ encodeList (a :: l)
      calc encodeList (l.drop n) ≤ encodeList l := ih n
        _ ≤ Nat.pair a (encodeList l) := le_pair_right a (encodeList l)
        _ ≤ Nat.pair a (encodeList l) + 1 := Nat.le_succ _

theorem takeCode_le (n c : ℕ) : takeCode n c ≤ c := by
  have h1 : takeCode n c = encodeList ((decodeList c).take n) := by
    rw [← takeCode_eq, encodeList_decodeList]
  rw [h1]
  exact le_trans (encodeList_take_le _ n) (le_of_eq (encodeList_decodeList c))

theorem dropCode_le (n c : ℕ) : dropCode n c ≤ c := by
  have h1 : dropCode n c = encodeList ((decodeList c).drop n) := by
    rw [← dropCode_eq, encodeList_decodeList]
  rw [h1]
  exact le_trans (encodeList_drop_le _ n) (le_of_eq (encodeList_decodeList c))

theorem primrec_takeListTabFn : Nat.Primrec takeListTabFn :=
  (primrec_nthCode.comp
    ((Nat.Primrec.right.comp Nat.Primrec.right).pair
      (Nat.Primrec.left.pair (Nat.Primrec.const 0)))).of_eq fun w => by
    simp [takeListTabFn]

theorem primrec_min {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => Nat.min (f n) (g n)) :=
  (primrec_selectFn (primrec_le hf hg) hf hg).of_eq fun n => by
    unfold selectFn isZero Nat.min
    by_cases h : f n ≤ g n
    · simp [Nat.sub_eq_zero_iff_le.mpr h, Nat.min_eq_left h]
    · have hlt : g n < f n := Nat.lt_of_not_ge h
      have hpos : 0 < f n - g n := Nat.sub_pos_of_lt hlt
      have hmin : min (f n - g n) 1 = 1 := Nat.min_eq_right (Nat.succ_le_iff.mpr hpos)
      simp [hmin, Nat.min_eq_right (Nat.le_of_lt hlt)]

theorem primrec_takeCode : Nat.Primrec (fun t => takeCode t.unpair.1 t.unpair.2) := by
  have hn := Nat.Primrec.left
  have hc := Nat.Primrec.right
  have hlen := primrec_listLenChar.comp hc
  have hB := primrec_min hn hlen
  have hp := hn.pair hc
  have hr := (hB.pair hp).pair hB
  refine ((primrec_tabCode primrec_takeListTabFn).comp hr).of_eq fun t => by
    simp [takeCode]

theorem primrec_dropListTabFn : Nat.Primrec dropListTabFn :=
  (primrec_nthCode.comp
```

```
    ((Nat.Primrec.right.comp Nat.Primrec.right).pair
      (Nat.Primrec.pair (primrec_add₂ Nat.Primrec.left (Nat.Primrec.left.comp Nat.
    Primrec.right))
        (Nat.Primrec.const 0)))).of_eq fun w => by
    simp [dropListTabFn]

theorem primrec_dropCode : Nat.Primrec (fun t => dropCode t.unpair.1 t.unpair.2) := by
  have hn := Nat.Primrec.left
  have hc := Nat.Primrec.right
  have hlen := primrec_listLenChar.comp hc
  have hB := primrec_sub₂ hlen hn
  have hp := hn.pair hc
  have hr := (hB.pair hp).pair hB
  refine ((primrec_tabCode primrec_dropListTabFn).comp hr).of_eq fun t => by
    simp [dropCode]

private theorem pair_lt_pair_zero_right {t b : ℕ} (ht : t < b) :
    Nat.pair 0 t < Nat.pair 1 b :=
  lt_trans (Nat.pair_lt_pair_right 0 ht) (Nat.pair_lt_pair_left b Nat.zero_lt_one)

theorem codeBound_ge (n c : ℕ) (hlen : (decodeList c).length ≤ n)
    (hbin : ∀ x ∈ decodeList c, x ≤ 1) : c < codeBound n := by
  induction n generalizing c with
  | zero =>
    have hc : c = 0 := by
      cases c with
      | zero => rfl
      | succ c' => simp [decodeList_succ, List.length_cons] at hlen
    simp [hc, codeBound]
  | succ n ih =>
    cases c with
    | zero => simp [codeBound]
    | succ d =>
      simp only [decodeList_succ, List.length_cons] at hlen
      simp only [decodeList_succ] at hbin
      have htlen : (decodeList d.unpair.2).length ≤ n := by omega
      have htbin : ∀ x ∈ decodeList d.unpair.2, x ≤ 1 := fun x hx =>
        hbin x (List.mem_cons.mpr (Or.inr hx))
      have ht := ih d.unpair.2 htlen htbin
      have hh : d.unpair.1 ≤ 1 := hbin d.unpair.1 (List.mem_cons.mpr (Or.inl rfl))
      have hpair : Nat.pair d.unpair.1 d.unpair.2 < Nat.pair 1 (codeBound n) := by
        rcases (show d.unpair.1 = 0 ∨ d.unpair.1 = 1 by omega) with h0 | h1
        · rw [h0]; exact pair_lt_pair_zero_right ht
        · rw [h1]; exact Nat.pair_lt_pair_right 1 ht
      rw [Nat.pair_unpair] at hpair
      simp only [codeBound]
      omega

theorem RecDecidable₂.swap {r : ℕ → ℕ → Prop} (hr : RecDecidable₂ r) :
    RecDecidable₂ (fun a b => r b a) := by
  have hswap : Nat.Primrec (fun t => Nat.pair t.unpair.2 t.unpair.1) :=
    Nat.Primrec.right.pair Nat.Primrec.left
  refine RecDecidable.of_iff (fun t => ?_) (hr.comp hswap)
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem RecDecidable.bExists {p : ℕ → Prop} (hp : RecDecidable p) {bound : ℕ → ℕ}
```

```
    (hb : Nat.Primrec bound) :
    RecDecidable (fun n => ∃ i, i < bound n ∧ p (Nat.pair i n)) := by
  obtain ⟨f, hf, hfspec⟩ := hp
  refine ⟨fun n => bExistsFn f n (bound n), ?_, ?_⟩
  · have hGfn : Nat.Primrec (fun w => selectFn w.unpair.2.unpair.2 1
        (isOne (f (Nat.pair w.unpair.2.unpair.1 w.unpair.1)))) :=
      primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right) (Nat.Primrec.const
    1)
        (primrec_isOne.comp (hf.comp
          ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left)))
    have hprec := Nat.Primrec.prec (Nat.Primrec.const 0) hGfn
    refine (hprec.comp (primrec_id.pair hb)).of_eq (fun n => ?_)
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
    rfl
  · intro n
    show (∃ i, i < bound n ∧ p (Nat.pair i n)) ↔ bExistsFn f n (bound n) = 1
    rw [bExistsFn_eq_one_iff]
    exact ⟨fun ⟨i, hi, h⟩ => ⟨i, hi, (hfspec _).mp h⟩, fun ⟨i, hi, h⟩ => ⟨i, hi, (hfspec
    _).mpr h⟩⟩

theorem RecDecidable.bForall {p : ℕ → Prop} (hp : RecDecidable p) {bound : ℕ → ℕ}
    (hb : Nat.Primrec bound) :
    RecDecidable (fun n => ∀ i, i < bound n → p (Nat.pair i n)) := by
  obtain ⟨f, hf, hfspec⟩ := hp
  refine ⟨fun n => bForallFn f n (bound n), ?_, ?_⟩
  · have hGfn : Nat.Primrec (fun w => selectFn w.unpair.2.unpair.2
        (isOne (f (Nat.pair w.unpair.2.unpair.1 w.unpair.1))) 0) :=
      primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right)
        (primrec_isOne.comp (hf.comp
          ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left)))
        (Nat.Primrec.const 0)
    have hprec := Nat.Primrec.prec (Nat.Primrec.const 1) hGfn
    refine (hprec.comp (primrec_id.pair hb)).of_eq (fun n => ?_)


    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
    rfl
  · intro n
    show (∀ i, i < bound n → p (Nat.pair i n)) ↔ bForallFn f n (bound n) = 1
    rw [bForallFn_eq_one_iff]
    exact ⟨fun h i hi => (hfspec _).mp (h i hi), fun h i hi => (hfspec _).mpr (h i hi)⟩

theorem RecDecidable.of_zero_one_char {p : ℕ → Prop} {f : ℕ → ℕ} (hf : Nat.Primrec f
    )
    (_h01 : ∀ n, f n = 0 ∨ f n = 1) (hfe : ∀ n, p n ↔ f n = 1) : RecDecidable p :=
  ⟨f, hf, hfe⟩

theorem RecDecidable₂.of_paired_zero_one_char {r : ℕ → ℕ → Prop} {f : ℕ → ℕ} (hf :
    Nat.Primrec f)
    (h01 : ∀ t, f t = 0 ∨ f t = 1) (hfe : ∀ n m, r n m ↔ f (Nat.pair n m) = 1) :
    RecDecidable₂ r := by
  unfold RecDecidable₂
  refine RecDecidable.of_zero_one_char hf h01 (fun t => ?_)
  rw [hfe, pair_unpair]

theorem RecDecidable₃.of_triple_zero_one_char {r : ℕ → ℕ → ℕ → Prop} {f : ℕ → ℕ}
    (hf : Nat.Primrec f) (h01 : ∀ t, f t = 0 ∨ f t = 1)
```

```
    (hfe : ∀ n m k, r n m k ↔ f (Nat.pair n (Nat.pair m k)) = 1) :
    RecDecidable₃ r := by
  unfold RecDecidable₃
  refine RecDecidable.of_zero_one_char hf h01 (fun t => ?_)
  rw [hfe, pair_unpair, pair_unpair]

theorem codeBound_succ (n : ℕ) : codeBound (n + 1) = Nat.pair 1 (codeBound n) + 1 :=
    rfl

theorem codeBound_zero : codeBound 0 = 1 := rfl

theorem decodeFuelOkChar_le_one : ∀ fuel c, decodeFuelOkChar fuel c ≤ 1
  | 0, _ => by simp [decodeFuelOkChar]
  | fuel + 1, c => by
    rw [decodeFuelOkChar, decodeFuelOkCharBody_eq]
    match tag : c.unpair.1 with
    | 0 =>
      simp
      rcases (show isOne (1 - c.unpair.2) = 0 ∨ isOne (1 - c.unpair.2) = 1 by
        have := isOne_le_one (1 - c.unpair.2); omega) with h0 | h1
      · rw [h0, selectFn_zero]; exact Nat.zero_le 1
      · rw [h1, selectFn_one]
    | 1 =>
      simp
      exact allBinDigitsChar_le_one _
    | 2 | 3 =>
      simp
      exact mulBit_le_one (decodeFuelOkChar_le_one fuel c.unpair.2.unpair.1)
        (decodeFuelOkChar_le_one fuel c.unpair.2.unpair.2)
    | _ + 4 =>
      simp

theorem decodeList_length_le (c : ℕ) : (decodeList c).length ≤ c := by
  induction c using Nat.strong_induction_on with
  | _ c ih =>
    cases c with
    | zero => simp [decodeList_zero]
    | succ d =>
      rw [decodeList_succ, List.length_cons]
      have hle := unpair_snd_le d
      have := ih d.unpair.2 (Nat.lt_succ_of_le hle)
      omega

theorem decodeList_mapPairStep (n acc x : ℕ) :
    decodeList (mapPairStep n acc x) = Nat.pair n x :: decodeList acc := by
  unfold mapPairStep; rw [decodeList_succ, unpair_pair_fst, unpair_pair_snd]

theorem eqZero_bit (a : ℕ) : 1 - a = if a = 0 then 1 else 0 := by split <;> omega

theorem eq_of_le_one_iff_one {x y : ℕ} (hx : x ≤ 1) (hy : y ≤ 1) (h : x = 1 ↔ y = 1)
    : x = y := by
  by_cases hx1 : x = 1
  · have hy1 := h.mp hx1; omega
  · have hy1 : y = 1 := fun hy1 => hx1 (h.mpr hy1)
    omega
```

```
theorem existsListStp_eq (g : ℕ → ℕ) (acc x p : ℕ) :
    existsListStp g (Nat.pair x (Nat.pair acc p)) = selectFn acc 1 (isOne (g (Nat.pair
     x p))) := by
  unfold existsListStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem existsListChar_eq (g : ℕ → ℕ) (p c : ℕ) :
    existsListChar g p c
      = List.foldl (fun acc x => selectFn acc 1 (isOne (g (Nat.pair x p)))) 0 (
    decodeList c) := by
  have hfun : (fun (acc x : ℕ) => existsListStp g (Nat.pair x (Nat.pair acc p)))
      = (fun acc x => selectFn acc 1 (isOne (g (Nat.pair x p)))) := by
    funext acc x; exact existsListStp_eq g acc x p
  unfold existsListChar; rw [foldCode_eq', hfun]

theorem existsListChar_le_one (g : ℕ → ℕ) (p c : ℕ) : existsListChar g p c ≤ 1 := by
  have h : ∀ (l : List ℕ) (z : ℕ), z ≤ 1 →
      List.foldl (fun acc x => existsListStp g (Nat.pair x (Nat.pair acc p))) z l ≤ 1
    := by
    intro l
    induction l with
    | nil => intro z hz; simpa using hz
    | cons x xs ih =>
      intro z hz
      simp only [List.foldl_cons]
      apply ih
      rw [existsListStp_eq]
      rcases (show z = 0 ∨ z = 1 by omega) with h0 | h1
      · rw [h0, selectFn_zero]; exact isOne_le_one _
      · rw [h1, selectFn_one]
  unfold existsListChar
  rw [foldCode_eq']
  exact h (decodeList c) 0 (Nat.zero_le 1)

theorem existsList_foldl_eq_one_iff (g : ℕ → ℕ) (p : ℕ) :
    ∀ (l : List ℕ) (acc : ℕ), acc ≤ 1 →
      (List.foldl (fun acc x => selectFn acc 1 (isOne (g (Nat.pair x p)))) acc l = 1 ↔
        acc = 1 ∨ ∃ x ∈ l, g (Nat.pair x p) = 1) := by
  intro l
  induction l with
  | nil => intro acc _; simp only [List.foldl_nil, List.not_mem_nil, false_and,
    exists_false,
      or_false]
  | cons x l ih =>
    intro acc hacc
    rw [List.foldl_cons]
    set acc' := selectFn acc 1 (isOne (g (Nat.pair x p))) with hacc'
    have hacc'le : acc' ≤ 1 := by
      rw [hacc']
      rcases (show acc = 0 ∨ acc = 1 by omega) with h | h
      · rw [h, selectFn_zero]; exact isOne_le_one _
      · rw [h, selectFn_one]
    rw [ih acc' hacc'le]
    constructor
    · rintro (hacc'1 | ⟨y, hy, hyg⟩)
      · rw [hacc'] at hacc'1
        rcases (show acc = 0 ∨ acc = 1 by omega) with h | h
```

```
        · rw [h, selectFn_zero, isOne_eq_one_iff] at hacc'1
          exact Or.inr ⟨x, List.mem_cons.mpr (Or.inl rfl), hacc'1⟩
        · exact Or.inl h
      · exact Or.inr ⟨y, List.mem_cons.mpr (Or.inr hy), hyg⟩
    · rintro (hacc1 | ⟨y, hy, hyg⟩)
      · exact Or.inl (by rw [hacc', hacc1, selectFn_one])
      · rcases List.mem_cons.mp hy with rfl | hy'
        · refine Or.inl ?_
          rw [hacc']
          rcases (show acc = 0 ∨ acc = 1 by omega) with h | h
          · rw [h, selectFn_zero, isOne_eq_one_iff]; exact hyg
          · rw [h, selectFn_one]
        · exact Or.inr ⟨y, hy', hyg⟩

theorem existsListChar_eq_one_iff (g : ℕ → ℕ) (p c : ℕ) :
    existsListChar g p c = 1 ↔ ∃ x ∈ decodeList c, g (Nat.pair x p) = 1 := by
  rw [existsListChar_eq, existsList_foldl_eq_one_iff g p (decodeList c) 0 (Nat.zero_le
    1)]
  simp only [Nat.zero_ne_one, false_or]

theorem fuelTableStep_nth {bodyLookup : ℕ → ℕ} {bound prevTable i : ℕ} (hi : i ≤
    bound) (d : ℕ) :
    nthCode (fuelTableStep bodyLookup bound prevTable) i d = bodyLookup (Nat.pair
    prevTable i) := by
  unfold fuelTableStep
  rw [tabCode_nth_lt (show i < bound + 1 by omega)]
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem fuelTable_succ_nth {bodyLookup : ℕ → ℕ} {bound fuel i : ℕ} (hi : i ≤ bound) (
    d : ℕ) :
    nthCode (fuelTable bodyLookup bound (fuel + 1)) i d =
      bodyLookup (Nat.pair (fuelTable bodyLookup bound fuel) i) :=
  fuelTableStep_nth hi d

theorem fuelTable_zero_nth {bodyLookup : ℕ → ℕ} {bound i : ℕ} (hi : i ≤ bound) (d : ℕ
    ) :
    nthCode (fuelTable bodyLookup bound 0) i d = 0 := by
  show nthCode (tabCode (fun _ => 0) (Nat.pair (bound + 1) 0) (bound + 1)) i d = 0
  rw [tabCode_nth_lt (show i < bound + 1 by omega)]

theorem fuelTable_eq_of_recursion {F : ℕ → ℕ → ℕ} {Body : (ℕ → ℕ) → ℕ → ℕ} {
    bodyLookup : ℕ → ℕ}
    (hF0 : ∀ c, F 0 c = 0) (hFsucc : ∀ fuel c, F (fuel + 1) c = Body (F fuel) c)
    (hlookup : ∀ table c, bodyLookup (Nat.pair table c) = Body (fun c' => nthCode table
     c' 0) c)
    (hlocal : ∀ (f g : ℕ → ℕ) c, (∀ c' ≤ c, f c' = g c') → Body f c = Body g c) :
    ∀ fuel bound c, c ≤ bound → nthCode (fuelTable bodyLookup bound fuel) c 0 = F
    fuel c := by
  intro fuel
  induction fuel with
  | zero => intro bound c hc; rw [fuelTable_zero_nth hc, hF0]
  | succ fuel ih =>
    intro bound c hc
    rw [fuelTable_succ_nth hc, hlookup, hFsucc]
    exact hlocal _ _ c fun c' hc' => ih bound c' (le_trans hc' hc)
```

```
theorem decodeFuelOkChar_fuelTable_eq :
    ∀ fuel bound c, c ≤ bound →
      nthCode (fuelTable decodeFuelOkCharLookup bound fuel) c 0 = decodeFuelOkChar
    fuel c :=
  fuelTable_eq_of_recursion (F := decodeFuelOkChar) (Body := decodeFuelOkCharBody)
    (fun _ => rfl) (fun _ _ => rfl)
    (fun table c => by unfold decodeFuelOkCharLookup; simp)
    decodeFuelOkCharBody_local

theorem geTwo_bit (a : ℕ) : 1 - (2 - a) = if 2 ≤ a then 1 else 0 := by split <;> omega

theorem getD_append_cf {β : Type*} (l l' : List β) (d : β) :
    ∀ {n : ℕ}, n < l.length → (l ++ l').getD n d = l.getD n d := by
  induction l with
  | nil => intro n h; exact absurd h (Nat.not_lt_zero n)
  | cons a t ih =>
    intro n h
    cases n with
    | zero => rw [List.cons_append, List.getD_cons_zero, List.getD_cons_zero]
    | succ m =>
      rw [List.cons_append, List.getD_cons_succ, List.getD_cons_succ]
      exact ih (by rw [List.length_cons] at h; omega)

theorem getD_append_right_cf {β : Type*} (l l' : List β) (d : β) :
    ∀ {n : ℕ}, l.length ≤ n → (l ++ l').getD n d = l'.getD (n - l.length) d := by
  induction l with
  | nil => intro n _; rw [List.nil_append, List.length_nil, Nat.sub_zero]
  | cons a t ih =>
    intro n h
    cases n with
    | zero => exact absurd h (by rw [List.length_cons]; exact Nat.not_succ_le_zero _)
    | succ m =>
      rw [List.cons_append, List.getD_cons_succ, List.length_cons, Nat.succ_sub_succ]
      exact ih (by rw [List.length_cons] at h; omega)

theorem appendListCode_eq (c1 c2 : ℕ) :
    decodeList (appendListCode c1 c2) = decodeList c1 ++ decodeList c2 := by
  unfold appendListCode
  rw [decodeList_tabCode, listLenChar_eq c1, listLenChar_eq c2]
  apply list_eq_of_getD
  intro i d
  by_cases hi : i < (decodeList c1).length + (decodeList c2).length
  · have hf := appendListTabFn_eq c1 c2 (decodeList c1).length i
    rw [getD_map_range_cf
        (fun j => appendListTabFn (Nat.pair j (Nat.pair c1 (Nat.pair c2 (decodeList c1
  ).length)))) d hi,
      hf]
    by_cases hlt : i < (decodeList c1).length
    · rw [ite_eq_left hlt,
        show (decodeList c1).getD i 0 = (decodeList c1).getD i d from by
          rw [getD_eq_getElem_cf _ 0 hlt, getD_eq_getElem_cf _ d hlt],
        (getD_append_cf (decodeList c1) (decodeList c2) d hlt).symm]
    · rw [ite_eq_right hlt,
        show (decodeList c2).getD (i - (decodeList c1).length) 0 =
            (decodeList c2).getD (i - (decodeList c1).length) d from by
          have hi2 : i - (decodeList c1).length < (decodeList c2).length := by
```

```
            have hle : (decodeList c1).length ≤ i := Nat.le_of_not_gt hlt
            omega
          rw [getD_eq_getElem_cf _ 0 hi2, getD_eq_getElem_cf _ d hi2],
        (getD_append_right_cf (decodeList c1) (decodeList c2) d (Nat.le_of_not_gt hlt)
    ).symm]
  · have hge : (decodeList c1).length + (decodeList c2).length ≤ i := Nat.le_of_not_gt
    hi
    rw [getD_eq_default_cf _ d (by rw [List.length_map, List.length_range]; exact hge)
    ,
      getD_eq_default_cf _ d (by rw [List.length_append]; exact hge)]

theorem halfParity_spec (n : ℕ) : halfParity n = Nat.pair (n / 2) (n % 2) := by
  induction n with
  | zero => rfl
  | succ n ih =>
    show Nat.pair ((halfParity n).unpair.1 + (halfParity n).unpair.2) (1 - (halfParity
     n).unpair.2)
        = Nat.pair ((n + 1) / 2) ((n + 1) % 2)
    rw [ih, unpair_pair_fst, unpair_pair_snd]
    congr 1 <;> omega

theorem mapPairStp_eq (n acc x : ℕ) :
    mapPairStp (Nat.pair x (Nat.pair acc n)) = mapPairStep n acc x := by
  unfold mapPairStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem decodeList_foldl_mapPairStp (n : ℕ) (el : List ℕ) (acc : ℕ) :
    decodeList (List.foldl (fun acc x => mapPairStp (Nat.pair x (Nat.pair acc n))) acc
     el)
      = (el.map (Nat.pair n ·)).reverse ++ decodeList acc := by
  induction el generalizing acc with
  | nil => simp
  | cons e el ih =>
    rw [List.foldl_cons, ih, mapPairStp_eq, decodeList_mapPairStep, List.map_cons,
      List.reverse_cons, List.append_assoc, List.singleton_append]

theorem decodeList_mapPairCode (n c : ℕ) :
    decodeList (mapPairCode n c) = ((decodeList c).map (Nat.pair n ·)).reverse := by
  unfold mapPairCode
  rw [foldCode_eq']
  have h := decodeList_foldl_mapPairStp n (decodeList c) 0
  rwa [decodeList_zero, List.append_nil] at h

theorem primrec_allListChar {g : ℕ → ℕ} (hg : Nat.Primrec g) :
    Nat.Primrec (fun t => allListChar g t.unpair.2 t.unpair.1) :=
  (primrec_foldCode (primrec_allListStp hg) Nat.Primrec.right (Nat.Primrec.const 1)
    Nat.Primrec.left).of_eq fun _ => rfl

theorem RecDecidable₂.bForallList {q : ℕ → ℕ → Prop} (hq : RecDecidable₂ q) :
    RecDecidable₂ (fun c p => ∀ e ∈ decodeList c, q e p) := by
  obtain ⟨g, hgp, hgs⟩ := hq
  refine ⟨fun t => allListChar g t.unpair.2 t.unpair.1, primrec_allListChar hgp, fun t
    => ?_⟩
  show (∀ e ∈ decodeList t.unpair.1, q e t.unpair.2) ↔ allListChar g t.unpair.2 t.
    unpair.1 = 1
  rw [allListChar_eq_one_iff]
  refine forall_congr' fun e => forall_congr' fun _ => ?_
```

```
  have := hgs (Nat.pair e t.unpair.2)
  simp only [unpair_pair_fst, unpair_pair_snd] at this
  exact this

theorem primrec_appendListTabFn : Nat.Primrec appendListTabFn := by
  have hi := Nat.Primrec.left
  have hp := Nat.Primrec.right
  have hlen1 := Nat.Primrec.right.comp (Nat.Primrec.right.comp hp)
  have hc1 := Nat.Primrec.left.comp hp
  have hc2 := Nat.Primrec.left.comp (Nat.Primrec.right.comp hp)
  have hlt := primrec_isZero.comp
    (primrec_sub₂ (primrec_add₂ hi (Nat.Primrec.const 1)) hlen1)
  have hleft := primrec_nthCode.comp (hc1.pair (Nat.Primrec.pair hi (Nat.Primrec.const
     0)))
  have hright := primrec_nthCode.comp
    (hc2.pair (Nat.Primrec.pair (primrec_sub₂ hi hlen1) (Nat.Primrec.const 0)))
  exact (primrec_selectFn hlt hleft hright).of_eq fun w => by
    unfold appendListTabFn
    simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_appendListCode : Nat.Primrec (fun t => appendListCode t.unpair.1 t.
    unpair.2) := by
  have hc1 := Nat.Primrec.left
  have hc2 := Nat.Primrec.right
  have hlen1 := primrec_listLenChar.comp hc1
  have hlen2 := primrec_listLenChar.comp hc2
  have hsum := primrec_add₂ hlen1 hlen2
  have hp := hc1.pair (hc2.pair hlen1)
  have hr := (hsum.pair hp).pair hsum
  refine ((primrec_tabCode primrec_appendListTabFn).comp hr).of_eq fun t => by
    unfold appendListCode
    simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_bForallFn_param {g : ℕ → ℕ} (hg : Nat.Primrec g) :
    Nat.Primrec (fun t => bForallFn (fun p => g (Nat.pair t.unpair.1.unpair.1 p))
      t.unpair.1.unpair.2 t.unpair.2) := by
  have hGfn : Nat.Primrec (fun w =>
      selectFn w.unpair.2.unpair.2
        (isOne (g (Nat.pair w.unpair.1.unpair.1
          (Nat.pair w.unpair.2.unpair.1 w.unpair.1.unpair.2)))) 0) :=
    primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right)
      (primrec_isOne.comp (hg.comp
        (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.left)
          (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.right)
            (Nat.Primrec.right.comp Nat.Primrec.left)))))
      (Nat.Primrec.const 0)
  have hprec := Nat.Primrec.prec (Nat.Primrec.const 1) hGfn
  refine (hprec.comp (Nat.Primrec.left.pair Nat.Primrec.right)).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  rfl

theorem primrec_codeBound : Nat.Primrec codeBound := by
  have hstep : Nat.Primrec (fun w => Nat.pair 1 w.unpair.2 + 1) :=
    primrec_add₂ (Nat.Primrec.pair (Nat.Primrec.const 1) Nat.Primrec.right)
      (Nat.Primrec.const 1)
  have haux : ∀ n, Nat.rec (motive := fun _ => ℕ) 1
```

```
      (fun y IH => Nat.pair 1 (Nat.pair y IH).unpair.2 + 1) n = codeBound n := by
    intro n
    induction n with
    | zero => simp [codeBound]
    | succ n ih =>
      simp only [unpair_pair_snd] at ih ⊢
      simp [codeBound, ih]
  exact (Nat.Primrec.prec1 1 hstep).of_eq haux

theorem primrec_decodeFuelOkChar : ∀ fuel, Nat.Primrec (fun c => decodeFuelOkChar fuel
    c)
  | 0 => Nat.Primrec.const 0
  | fuel + 1 => by
    simpa [decodeFuelOkChar] using
      primrec_decodeFuelOkCharBody (primrec_decodeFuelOkChar fuel)

theorem primrec_decodeFuelOkCharLookup : Nat.Primrec decodeFuelOkCharLookup := by
  have hleft : Nat.Primrec (fun w => nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.1
    0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.
    right))
        (Nat.Primrec.const 0)))).of_eq fun w => by simp
  have hright : Nat.Primrec (fun w => nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.2
    0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.
    right))
        (Nat.Primrec.const 0)))).of_eq fun w => by simp
  have hsub : Nat.Primrec (fun w =>
      mulBit (nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.1 0)
        (nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.2 0)) :=
    primrec_mul₂ hleft hright
  have hf0 : Nat.Primrec (fun w => selectFn (isOne (1 - w.unpair.2.unpair.2)) 1 0) :=
    primrec_selectFn
      (primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 1) (Nat.Primrec.right.comp
    Nat.Primrec.right)))
      (Nat.Primrec.const 1) (Nat.Primrec.const 0)
  have hf1 : Nat.Primrec (fun w => allBinDigitsChar w.unpair.2.unpair.2) :=
    primrec_allBinDigitsChar.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have ht : Nat.Primrec (fun w => w.unpair.2.unpair.1) := Nat.Primrec.left.comp Nat.
    Primrec.right
  have h01 : Nat.Primrec (fun w => isOne (1 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 1) ht)
  have h12 : Nat.Primrec (fun w => isOne (2 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 2) ht)
  have h23 : Nat.Primrec (fun w => isOne (3 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 3) ht)
  have h34 : Nat.Primrec (fun w => isOne (4 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 4) ht)
  refine (primrec_selectFn h01 hf0
    (primrec_selectFn h12 hf1
      (primrec_selectFn h23 hsub
        (primrec_selectFn h34 hsub (Nat.Primrec.const 0))))).of_eq fun w => ?_
  show _ = decodeFuelOkCharBody (fun c' => nthCode w.unpair.1 c' 0) w.unpair.2
  rw [decodeFuelOkCharBody_eq]
```

```
  match tag : w.unpair.2.unpair.1 with
  | 0 =>
    have h10 : (1 - 0 : ℕ) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01' : (1 - 1 : ℕ) = 0 := rfl
    have h12' : (2 - 1 : ℕ) = 1 := rfl
    simp only [h01', h12', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 2 =>
    have h01' : (1 - 2 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 2 : ℕ) = 0 := rfl
    have h23' : (3 - 2 : ℕ) = 1 := rfl
    simp only [h01', h23', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 3 =>
    have h01' : (1 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - 3 : ℕ) = 0 := rfl
    have h34' : (4 - 3 : ℕ) = 1 := rfl
    simp only [h01', h34', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | t + 4 =>


    have h01' : (1 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h34' : (4 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    simp only [h01', h12', h23', h34', isOne_zero, selectFn_zero]

theorem primrec_existsListStp {g : ℕ → ℕ} (hg : Nat.Primrec g) :
    Nat.Primrec (existsListStp g) := by
  have hacc : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hx : Nat.Primrec (fun w : ℕ => w.unpair.1) := Nat.Primrec.left
  have hp : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hcall : Nat.Primrec (fun w : ℕ => isOne (g (Nat.pair w.unpair.1 w.unpair.2.
    unpair.2))) :=
    primrec_isOne.comp (hg.comp (hx.pair hp))
  exact (primrec_selectFn hacc (Nat.Primrec.const 1) hcall).of_eq fun _ => rfl

theorem primrec_existsListChar {g : ℕ → ℕ} (hg : Nat.Primrec g) :
    Nat.Primrec (fun t => existsListChar g t.unpair.2 t.unpair.1) :=
  (primrec_foldCode (primrec_existsListStp hg) Nat.Primrec.right (Nat.Primrec.const 0)
    Nat.Primrec.left).of_eq fun _ => rfl

theorem RecDecidable₂.bExistsList {q : ℕ → ℕ → Prop} (hq : RecDecidable₂ q) :
    RecDecidable₂ (fun c p => ∃ e ∈ decodeList c, q e p) := by
  obtain ⟨g, hgp, hgs⟩ := hq
  refine ⟨fun t => existsListChar g t.unpair.2 t.unpair.1, primrec_existsListChar hgp,
    fun t => ?_⟩
  show (∃ e ∈ decodeList t.unpair.1, q e t.unpair.2) ↔ existsListChar g t.unpair.2 t.
    unpair.1 = 1
  rw [existsListChar_eq_one_iff]
  refine exists_congr fun e => and_congr_right fun _ => ?_
  have := hgs (Nat.pair e t.unpair.2)
  simp only [unpair_pair_fst, unpair_pair_snd] at this
  exact this
```

```
set_option maxHeartbeats 1000000 in

theorem primrec_fuelTable {bodyLookup : ℕ → ℕ} (hbodyLookup : Nat.Primrec bodyLookup)
     :
    Nat.Primrec (fun t => fuelTable bodyLookup t.unpair.1 t.unpair.2) := by
  have hgLookup : Nat.Primrec (fun v => bodyLookup (Nat.pair v.unpair.2 v.unpair.1))
    :=
    hbodyLookup.comp (Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left)
  have htab := primrec_tabCode hgLookup
  have hf : Nat.Primrec (fun bound => tabCode (fun _ => 0) (Nat.pair (bound + 1) 0) (
    bound + 1)) :=
    ((primrec_tabCode (Nat.Primrec.const 0)).comp
      (Nat.Primrec.pair
        (Nat.Primrec.pair (primrec_add₂ Nat.Primrec.id (Nat.Primrec.const 1)) (Nat.
    Primrec.const 0))
        (primrec_add₂ Nat.Primrec.id (Nat.Primrec.const 1)))).of_eq fun bound => by
    simp
  have hpack : Nat.Primrec (fun w =>
      Nat.pair (Nat.pair (w.unpair.1 + 1) w.unpair.2.unpair.2) (w.unpair.1 + 1)) :=
    Nat.Primrec.pair
      (Nat.Primrec.pair (primrec_add₂ Nat.Primrec.left (Nat.Primrec.const 1))
        (Nat.Primrec.right.comp Nat.Primrec.right))
      (primrec_add₂ Nat.Primrec.left (Nat.Primrec.const 1))
  have hg : Nat.Primrec (fun w => fuelTableStep bodyLookup w.unpair.1 w.unpair.2.
    unpair.2) :=
    (htab.comp hpack).of_eq fun w => by simp [fuelTableStep]
  have haux : ∀ bound fuel, Nat.rec (motive := fun _ => ℕ)
      (tabCode (fun _ => 0) (Nat.pair (bound + 1) 0) (bound + 1))
      (fun _ IH => fuelTableStep bodyLookup bound IH) fuel = fuelTable bodyLookup
    bound fuel := by
    intro bound fuel
    induction fuel with
    | zero => rfl
    | succ fuel ih => show fuelTableStep bodyLookup bound _ = fuelTableStep bodyLookup
     bound _; rw [ih]
  refine (Nat.Primrec.prec hf hg).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  exact haux t.unpair.1 t.unpair.2

theorem primrec_decodeFuelOkChar2 : Nat.Primrec (fun s => decodeFuelOkChar s.unpair.1
    s.unpair.2) := by
  have hft := primrec_fuelTable primrec_decodeFuelOkCharLookup
  have hpack : Nat.Primrec (fun s : ℕ => Nat.pair s.unpair.2 s.unpair.1) :=
    Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left
  have hcomp : Nat.Primrec (fun s => fuelTable decodeFuelOkCharLookup s.unpair.2 s.
    unpair.1) :=
    (hft.comp hpack).of_eq fun s => by simp
  have hnth : Nat.Primrec
      (fun s => nthCode (fuelTable decodeFuelOkCharLookup s.unpair.2 s.unpair.1) s.
    unpair.2 0) :=
    (primrec_nthCode.comp
      (Nat.Primrec.pair hcomp (Nat.Primrec.pair Nat.Primrec.right (Nat.Primrec.const
    0)))).of_eq
      fun s => by simp
  exact hnth.of_eq fun s => decodeFuelOkChar_fuelTable_eq s.unpair.1 s.unpair.2 s.
```

```
    unpair.2 (le_refl _)

theorem primrec_halfParity : Nat.Primrec halfParity := by
  have hIH : Nat.Primrec (fun w => w.unpair.2.unpair.2) := Nat.Primrec.right.comp Nat.
    Primrec.right
  have hstep : Nat.Primrec (fun w => Nat.pair (w.unpair.2.unpair.2.unpair.1 + w.unpair
    .2.unpair.2.unpair.2)
      (1 - w.unpair.2.unpair.2.unpair.2)) :=
    (primrec_add₂ (Nat.Primrec.left.comp hIH) (Nat.Primrec.right.comp hIH)).pair
      (primrec_sub₂ (Nat.Primrec.const 1) (Nat.Primrec.right.comp hIH))
  refine ((Nat.Primrec.prec (Nat.Primrec.const 0) hstep).comp
    ((Nat.Primrec.const 0).pair primrec_id)).of_eq (fun n => ?_)
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rfl

theorem primrec_div2 : Nat.Primrec (fun n => n / 2) :=
  (Nat.Primrec.left.comp primrec_halfParity).of_eq fun n => by
    rw [halfParity_spec, unpair_pair_fst]

theorem primrec_ite {c t f : ℕ → ℕ} (hc : Nat.Primrec c) (ht : Nat.Primrec t) (hf :
    Nat.Primrec f) :
    Nat.Primrec (fun n => selectFn (c n) (t n) (f n)) :=
  primrec_selectFn hc ht hf

theorem primrec_mapPairStp : Nat.Primrec mapPairStp := by
  have h1 : Nat.Primrec (fun w : ℕ => w.unpair.1) := Nat.Primrec.left
  have h21 : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have h22 : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  exact (Nat.Primrec.succ.comp ((h22.pair h1).pair h21)).of_eq (fun _ => rfl)

theorem primrec_mapPairCode : Nat.Primrec (fun t => mapPairCode t.unpair.1 t.unpair.2)
     :=
  (primrec_foldCode primrec_mapPairStp Nat.Primrec.left (Nat.Primrec.const 0)
    Nat.Primrec.right).of_eq (fun _ => rfl)

theorem REPred.forall_mem_decodeList₂ {p : ℕ → ℕ → Prop} (hp : REPred₂ p) :
    REPred (fun t => ∀ e ∈ decodeList t.unpair.2, p t.unpair.1 e) := by
  have hp' : REPred (fun s => p s.unpair.1 s.unpair.2) := hp
  have hbase : REPred (fun c => ∀ e' ∈ decodeList c, p e'.unpair.1 e'.unpair.2) :=
    hp'.forall_mem_decodeList
  refine REPred.of_iff (fun t => ?_) (hbase.comp primrec_mapPairCode)
  rw [decodeList_mapPairCode]
  simp only [List.mem_reverse, List.mem_map]
  constructor
  · rintro hall e' ⟨e, he, rfl⟩
    simp only [unpair_pair_fst, unpair_pair_snd]
    exact hall e he
  · intro hall e he
    simpa only [unpair_pair_fst, unpair_pair_snd]
      using hall (Nat.pair t.unpair.1 e) ⟨e, he, rfl⟩

theorem primrec_max {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => Nat.max (f n) (g n)) := by
  refine (primrec_selectFn (primrec_le hf hg) hg hf).of_eq fun n => by
```

```
    unfold selectFn isZero Nat.max
    by_cases h : f n ≤ g n
    · simp [Nat.sub_eq_zero_iff_le.mpr h, Nat.max_eq_right h]
    · have hlt : g n < f n := Nat.lt_of_not_ge h
      have hpos : 0 < f n - g n := Nat.sub_pos_of_lt hlt
      have hmin : min (f n - g n) 1 = 1 := Nat.min_eq_right (Nat.succ_le_iff.mpr hpos)
      simp [hmin, Nat.max_eq_left (Nat.le_of_lt hlt)]

theorem primrec_max_one : Nat.Primrec (fun n => Nat.max n 1) := by
  refine (primrec_selectFn (primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 1)
    primrec_id))
    (Nat.Primrec.const 1) primrec_id).of_eq fun n => by
    unfold selectFn isOne Nat.max
    rcases n with _ | n <;> simp

theorem primrec_mod2 : Nat.Primrec (fun n => n % 2) :=
  (Nat.Primrec.right.comp primrec_halfParity).of_eq fun n => by
    rw [halfParity_spec, unpair_pair_snd]

theorem primrec_lowOr : Nat.Primrec (fun t => lowOr t.unpair.1 t.unpair.2) := by
  have hx : Nat.Primrec (fun t : ℕ => t.unpair.1 % 2) := primrec_mod2.comp Nat.Primrec
    .left
  have hy : Nat.Primrec (fun t : ℕ => t.unpair.2 % 2) := primrec_mod2.comp Nat.Primrec
    .right
  exact (primrec_sub₂ (Nat.Primrec.const 1)
    (primrec_sub₂ (Nat.Primrec.const 1) (primrec_add₂ hx hy))).of_eq fun _ => rfl

theorem primrec_lorStep : Nat.Primrec lorStep := by
  have hA : Nat.Primrec (fun s : ℕ => s.unpair.1.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.left
  have hB : Nat.Primrec (fun s : ℕ => s.unpair.1.unpair.2) := Nat.Primrec.right.comp
    Nat.Primrec.left
  have hW : Nat.Primrec (fun s : ℕ => s.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have hAcc : Nat.Primrec (fun s : ℕ => s.unpair.2.unpair.2) := Nat.Primrec.right.comp
    Nat.Primrec.right
  have hlow : Nat.Primrec (fun s : ℕ => lowOr s.unpair.1.unpair.1 s.unpair.1.unpair.2)
    :=
    (primrec_lowOr.comp (hA.pair hB)).of_eq fun s => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (((primrec_div2.comp hA).pair (primrec_div2.comp hB)).pair
    ((primrec_mul₂ (Nat.Primrec.const 2) hW).pair
      (primrec_add₂ hAcc (primrec_mul₂ hW hlow)))).of_eq fun _ => rfl

theorem primrec_myLor : Nat.Primrec (fun t => myLor t.unpair.1 t.unpair.2) := by
  have hbase : Nat.Primrec
      (fun z => Nat.pair (Nat.pair z.unpair.1 z.unpair.2) (Nat.pair 1 0)) :=
    (Nat.Primrec.left.pair Nat.Primrec.right).pair
      ((Nat.Primrec.const 1).pair (Nat.Primrec.const 0))
  have hstep : Nat.Primrec (fun w => lorStep w.unpair.2.unpair.2) :=
    primrec_lorStep.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hprec := Nat.Primrec.prec hbase hstep
  have hcount : Nat.Primrec (fun t => t.unpair.1 + t.unpair.2) :=
    primrec_add₂ Nat.Primrec.left Nat.Primrec.right
  refine ((Nat.Primrec.right.comp Nat.Primrec.right).comp
    (hprec.comp (primrec_id.pair hcount))).of_eq fun t => ?_
```

```
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rw [rec_const_iterate]
  rfl

theorem recDecidable_of_forall {p : ℕ → Prop} (h : ∀ n, p n) : RecDecidable p :=
  ⟨fun _ => 1, Nat.Primrec.const 1, fun n => ⟨fun _ => rfl, fun _ => h n⟩⟩

theorem rePred_of_forall {p : ℕ → Prop} (h : ∀ n, p n) : REPred p :=
  (recDecidable_of_forall h).re

theorem recDecidable₂_of_forall {r : ℕ → ℕ → Prop} (h : ∀ n m, r n m) : RecDecidable₂
    r :=
  recDecidable_of_forall (fun t => h t.unpair.1 t.unpair.2)

theorem recPow_eq (b : ℕ) :
    ∀ e, Nat.rec (motive := fun _ => ℕ) 1 (fun _ ih => ih * b) e = b ^ e
  | 0 => (pow_zero b).symm
  | e + 1 => by rw [pow_succ]; exact congrArg (· * b) (recPow_eq b e)

theorem primrec_pow : Nat.Primrec (Nat.unpaired fun b e => b ^ e) := by
  have hg : Nat.Primrec (fun w => w.unpair.2.unpair.2 * w.unpair.1) :=
    primrec_mul₂ (Nat.Primrec.right.comp Nat.Primrec.right) Nat.Primrec.left
  refine (Nat.Primrec.prec (Nat.Primrec.const 1) hg).of_eq (fun p => ?_)
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  exact recPow_eq p.unpair.1 p.unpair.2

theorem primrec_pow₂ {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => f n ^ g n) :=
  (primrec_pow.comp (hf.pair hg)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]

theorem primrec_two_pow {g : ℕ → ℕ} (hg : Nat.Primrec g) : Nat.Primrec (fun n => 2 ^
    g n) :=
  (primrec_pow.comp ((Nat.Primrec.const 2).pair hg)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]

theorem selectFn_isOne_one_sub_sigma (u : ℕ) :
    selectFn (isOne (1 - u)) 1 0 = 1 ↔ u = 0 := by
  constructor
  · intro h
    by_cases hu : u = 0
    · exact hu
    · have : isOne (1 - u) = 0 := isOne_of_ne_one (by omega)
      simp [this, selectFn_zero] at h
  · intro hu
    subst hu
    simp [isOne_one, selectFn_one]

end Domain.Recursive
```

4.8.3 Computable presentation

A computable presentation of a neighbourhood system: an enumeration of the neighbourhoods for which intersection and consistency are recursively decidable.

Definition (`ComputablePresentation`). A computable presentation enumerates all neighbourhoods, decides equality with intersections and consistency of pairs, computes intersections whenever they are consistent, and supplies an index for the master neighbourhood.

Definition (`NeighborhoodSystem.IsEffectivelyGiven`). A neighbourhood system is effectively given when it admits a computable presentation.

Definition (`ComputablePresentation.inclShuffle`). This reindexing sends a pair of indices to the triple that compares the first neighbourhood intersected with the second against the first.

Definition (`ComputablePresentation.reindexInvolutive`). An involutive primitive-recursive permutation of indices transports a computable presentation to an equivalent presentation.

Definition (`ComputablePresentation.swapPair`). This operation exchanges the two components of a paired natural number.

Definition (`unitPresentation`). The one-point neighbourhood system is presented by the constant enumeration of its unique neighbourhood.

Theorem (`ComputablePresentation.primrec_inclShuffle`). The reindexing used to reduce inclusion to intersection equality is primitive recursive.

Proof. The two coordinate projections are primitive recursive, and pairing them first with one another and then with the repeated first coordinate preserves primitive recursiveness. ■

Theorem (`ComputablePresentation.primrec_swapPair`). Exchanging the components of a paired natural number is primitive recursive.

Proof. The first and second projections are primitive recursive, and pairing them in reverse order gives the required permutation. ■

Theorem (`ComputablePresentation.incl_computable`). Inclusion between two enumerated neighbourhoods is recursively decidable.

Proof. A neighbourhood is contained in another exactly when intersecting it with the latter returns the former; the shuffled triple is therefore decided by the presentation's intersection-equality test. ■

Theorem (`ComputablePresentation.eq_computable`). Equality between two enumerated neighbourhoods is recursively decidable.

Proof. Two neighbourhoods are equal exactly when each contains the other, so conjunction of the two decidable inclusion tests decides equality. ■

Theorem (`unitSys_isEffectivelyGiven`). The one-point neighbourhood system is effectively given.

Proof. The constant enumeration by the whole one-point set is surjective; every intersection and consistency question is automatically true, and the constant index zero names both intersections and the master neighbourhood. ■

`computablePresentation.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .rePred
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodIso

public import Mathlib.Computability.Primrec.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
```

```
open NeighborhoodSystem Domain.Recursive

section
variable {α : Type*}

structure ComputablePresentation (V : NeighborhoodSystem α) where

  X : ℕ → Set α

  mem_X : ∀ n, V.mem (X n)

  surj : ∀ {Y : Set α}, V.mem Y → ∃ n, X n = Y

  interEq_computable : RecDecidable₃ (fun n m k => X n ∩ X m = X k)

  cons_computable : RecDecidable₂ (fun n m => ∃ k, X k ⊆ X n ∩ X m)

  inter : ℕ → ℕ → ℕ

  inter_primrec : Nat.Primrec (fun t => inter t.unpair.1 t.unpair.2)

  inter_spec : ∀ {n m : ℕ}, (∃ k, X k ⊆ X n ∩ X m) → X (inter n m) = X n ∩ X m

  masterIdx : ℕ

  masterIdx_spec : X masterIdx = V.master

def NeighborhoodSystem.IsEffectivelyGiven (V : NeighborhoodSystem α) : Prop :=
  Nonempty (ComputablePresentation V)

namespace ComputablePresentation

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

def inclShuffle (t : ℕ) : ℕ := Nat.pair t.unpair.1 (Nat.pair t.unpair.2 t.unpair.1)

def reindexInvolutive (P : ComputablePresentation V) (φ : ℕ → ℕ)
    (hφinv : Function.Involutive φ) (hφp : Nat.Primrec φ) : ComputablePresentation V
    where
  X n := P.X (φ n)
  mem_X n := P.mem_X (φ n)
  surj := fun {Y} hY => by
    obtain ⟨k, hk⟩ := P.surj hY
    exact ⟨φ k, by rw [hφinv k]; exact hk⟩
  interEq_computable := by
    have hreindex : Nat.Primrec (fun t : ℕ =>
        Nat.pair (φ t.unpair.1) (Nat.pair (φ t.unpair.2.unpair.1) (φ t.unpair.2.unpair
    .2))) :=
      (hφp.comp Nat.Primrec.left).pair
        ((hφp.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
          (hφp.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
    refine RecDecidable.of_iff (fun t => ?_) (P.interEq_computable.comp hreindex)
    simp only [unpair_pair_fst, unpair_pair_snd]
  cons_computable := by
    have hreindex : Nat.Primrec (fun t : ℕ =>
        Nat.pair (φ t.unpair.1) (φ t.unpair.2)) :=
```

```
      (hφp.comp Nat.Primrec.left).pair (hφp.comp Nat.Primrec.right)
    refine RecDecidable.of_iff (fun t => ?_) (P.cons_computable.comp hreindex)
    simp only [unpair_pair_fst, unpair_pair_snd]
    constructor
    · rintro ⟨k, hk⟩; exact ⟨φ k, hk⟩
    · rintro ⟨k, hk⟩; exact ⟨φ k, by rwa [hφinv]⟩
  inter n m := φ (P.inter (φ n) (φ m))
  inter_primrec := by
    have hreindex : Nat.Primrec (fun t : ℕ => Nat.pair (φ t.unpair.1) (φ t.unpair.2))
    :=
      (hφp.comp Nat.Primrec.left).pair (hφp.comp Nat.Primrec.right)
    exact (hφp.comp ((P.inter_primrec.comp hreindex))).of_eq
      (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
  inter_spec {n m} h := by
    show P.X (φ (φ (P.inter (φ n) (φ m)))) = P.X (φ n) ∩ P.X (φ m)
    rw [hφinv]
    obtain ⟨k, hk⟩ := h
    exact P.inter_spec ⟨φ k, hk⟩
  masterIdx := φ P.masterIdx
  masterIdx_spec := by show P.X (φ (φ P.masterIdx)) = V.master; rw [hφinv]; exact P.
    masterIdx_spec

def swapPair (t : ℕ) : ℕ := Nat.pair t.unpair.2 t.unpair.1

end ComputablePresentation

end

section
variable {α : Type*}

def unitPresentation : ComputablePresentation unitSys where
  X _ := Set.univ
  mem_X _ := rfl
  surj := by rintro Y rfl; exact ⟨0, rfl⟩
  interEq_computable := recDecidable_of_forall fun _ => Set.inter_self Set.univ
  cons_computable := recDecidable_of_forall fun _ =>
    ⟨0, by rw [Set.inter_self]⟩
  inter _ _ := 0
  inter_primrec := Nat.Primrec.zero
  inter_spec _ := by rw [Set.inter_self]
  masterIdx := 0
  masterIdx_spec := rfl

namespace ComputablePresentation

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

theorem primrec_inclShuffle : Nat.Primrec inclShuffle :=
  Nat.Primrec.pair Nat.Primrec.left (Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.
    left)

private theorem primrec_swapPair : Nat.Primrec swapPair :=
  Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left

theorem incl_computable : RecDecidable₂ (fun n m => P.X n ⊆ P.X m) := by
```

```
  refine RecDecidable.of_iff (fun t => ?_) (P.interEq_computable.comp
    primrec_inclShuffle)
  simp only [inclShuffle, unpair_pair_fst, unpair_pair_snd]
  exact Set.inter_eq_left.symm

theorem eq_computable : RecDecidable₂ (fun n m => P.X n = P.X m) := by
  refine RecDecidable.of_iff (fun t => ?_)
    (P.incl_computable.and (P.incl_computable.comp primrec_swapPair))
  simp only [swapPair, unpair_pair_fst, unpair_pair_snd]
  exact Set.Subset.antisymm_iff

end ComputablePresentation

end

section
variable {α : Type*}

theorem unitSys_isEffectivelyGiven : unitSys.IsEffectivelyGiven :=
  ⟨unitPresentation⟩
end

end DomainTheory.NeighbourhoodSystems
```

4.8.4 Definition 7.1 — computable presentation

DEFINITION 7.1. A neighbourhood system D has a *computable presentation* provided we can write

$$D = \{X_n \mid n \in N\},$$

where the following two relations

(i) $X_n \cap X_m = X_k$; and

(ii) $\exists k \in N.\, X_k \subseteq X_n$ and $X_k \subseteq X_m$

are recursively decidable (in integer indices n, m, k and in n, m, respectively). □

More strictly the sequence,

$$\langle X_n \rangle_{n=0}^{\infty},$$

is the presentation. Even more strictly, when it is required to cope with infinitely many domains at a time, it would be necessary to give the actual Gödel numbers of the recursive relations (i) and (ii) (rather than just saying there exists some way of showing them to be recursively decidable).

The intuitive idea of 7.1 is that the system is effectively given if you know how to do elementary "calculations" with neighbourhoods. The basic calculations are the forming of intersections. The neighbourhoods have to be laid out in a systematic way; and, if we are asked for an intersection of two given neighbourhoods, we have to be able to locate it in the standard sequence. Relation (ii) is the *consistency condition* , which is the necessary and sufficient condition for the intersection to exist in D. When (ii) is true, therefore, we have only to try $k = 0, 1, 2, \ldots$ until we discover that we have found the intersection. We are

assuming that these basic decisions can be carried out in "finite time". Note that the obvious biconditional,

$$X_n \subseteq X_m \text{ iff } X_n \cap X_m = X_n,$$

assures us that the inclusion relation between neighbourhoods is itself decidable in terms of the indices. So in (ii) if k exists, *then* it (or the first one) can indeed be found in finite time. The rub is that if it *does not exist*, no finite number of inclusion checks will determine that fact. That is why we have to *assume* that (ii) is always decidable. The information contained in (ii) is a fundamental part of the neighbourhood structure. (An axiomatic characterization of neighbourhood structures is given in Exercise 7.13, which may make clearer what we are assuming and what a presentation is.)

The development is with the recursive predicates and computable presentation peers.

Theorem (`definition_71`). A neighbourhood system is effectively given precisely when it has a computable presentation, and for any such presentation inclusion and equality of enumerated neighbourhoods are recursively decidable.

Proof. The equivalence is the definition of effective givenness. Inclusion and equality reduce to the decidable intersection relation of the presentation. ■

Definition_7_1.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .rePred
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .computablePresentation

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive

/-- Scott's Definition 7.1: a computable presentation enumerates the neighbourhoods
so that intersection and consistency are recursively decidable, and therefore so
are inclusion and equality of neighbourhoods. -/
theorem definition_71 {α : Type*} (V : NeighborhoodSystem α) :
    (V.IsEffectivelyGiven ↔ Nonempty (ComputablePresentation V)) ∧
      (∀ P : ComputablePresentation V, RecDecidable₂ (fun n m => P.X n ⊆ P.X m)) ∧
        (∀ P : ComputablePresentation V, RecDecidable₂ (fun n m => P.X n = P.X m)) :=
  ⟨Iff.rfl, fun P => P.incl_computable, fun P => P.eq_computable⟩

end DomainTheory.NeighbourhoodSystems
```

4.8.5 Computable map

Computable elements and computable approximable maps on effectively given domains.

Definition (`ComputableElement`). An element is computable relative to a presentation when the set of indices of its neighbourhoods is recursively enumerable.

Definition (`ComputableMap`). An approximable map is computable relative to two presentations when its relation on enumerated neighbourhoods is recursively enumerable.

computableMap.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_1
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap

variable {α β : Type*}

/-- An element of an effectively given domain is computable when the indices of
neighbourhoods it contains are a recursively enumerable set. -/
class ComputableElement {W : NeighborhoodSystem β} (Q : ComputablePresentation W)
    (y : W.Element) : Prop where
  out : REPred (fun m => y.mem (Q.X m))

/-- An approximable map between effectively given domains is computable when the
neighbourhood relation is recursively enumerable in the two indices. -/
class ComputableMap {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
    (P : ComputablePresentation V) (Q : ComputablePresentation W) (f : ApproximableMap
     V W) : Prop where
  out : REPred₂ (fun n m => f.rel (P.X n) (Q.X m))

end DomainTheory.NeighbourhoodSystems
```

4.8.6 Definition 7.2 — computable mapping

DEFINITION 7.2. Given two recursively presented domains,

$$\mathcal{D} = \{X_n \mid n \in \mathbf{N}\} \text{ and } E = \{Y_m \mid m \in \mathbf{N}\},$$

an approximable mapping $f : \mathcal{D} \to E$ is said to be *computable* iff the relation

$$X_n f Y_m$$

is recursively enumerable in n and m. □

The question to ask first is why "recursively enumerable" rather than "recursive" (= "recursively decidable")? The answer will become clear when we let $\mathcal{D}$ degenerate to the one-element domain, $\mathcal{D} = \{\Delta\}$. Then what we are considering is merely a single element

$$y = f(\{\Delta\}) \in |E|.$$

Therefore, 7.2 incorporates the notion of a *computable element* of a domain. And the condition reduces to the statement that the filter $y \in |E|$ is such that the set

$$\{m \in \mathbf{N} \mid Y_m \in y\}$$

is a recursively enumerable set of integers. The point is that the elements of $|E|$ are finite or infinite. If y were finite, the set of indices above would indeed be recursive in view of

our assumptions on E. But an infinite element can in general only be approximated "a little at a time". We cannot expect to know the whole story of its approximations in a flash. What it means to be recursively enumerable is that there is a primitive recursive function (hence, a total function), $r : \mathbf{N} \to \mathbf{N}$, such that

$$y = \{Y_{r(i)} \mid i \in \mathbf{N}\}.$$

That is to say, *all* the approximations to y can eventually be listed. In the case of the mapping f we could write

$$f = \{(X_{s(i)}, Y_{r(i)}) \mid i \in \mathbf{N}\},$$

for a suitable pair of primitive recursive functions s and r.

Definitions 7.1 and 7.2 may very well irritate the person hearing them for the first time: instead of explaining computability in direct terms, the whole question is thrown into the lap of recursion theory! There are several answers. "You have to start somewhere" is one thing I always say. Recursion on the integers is a well-understood theory, and we shall not need the refined parts of the development, fortunately. In any case, our definitions apply to *many* domains of quite different structure, not just to the domain $\mathbf{N}$. And the next step we shall take is to show how to build up computable functions (and also effectively given domains) from simpler ones. Thus, often it will not be necessary to go back to the seemingly over-precise definitions involving the indices but to appeal to some broad general principles.

The development is with the computable map peer.

Theorem (`definition_72`). An approximable map between computably presented domains is computable exactly when its neighbourhood relation is recursively enumerable in the two indices, and an element is computable exactly when the indices of neighbourhoods it contains are recursively enumerable.

Proof. Both directions are the defining recursively enumerable predicates of a computable map and a computable element. ■

Definition_7_2.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .computableMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap

/-- Scott's Definition 7.2: a map is computable when its neighbourhood relation is
recursively enumerable, and an element is computable when the neighbourhoods it
meets are recursively enumerable. -/
theorem definition_72 {α β : Type*} {V : NeighborhoodSystem α} {W : NeighborhoodSystem
    β}
    (P : ComputablePresentation V) (Q : ComputablePresentation W)
    (f : ApproximableMap V W) (y : W.Element) :
    (ComputableMap P Q f ↔ REPred₂ (fun n m => f.rel (P.X n) (Q.X m))) ∧
      (ComputableElement Q y ↔ REPred (fun m => y.mem (Q.X m))) :=
  ⟨⟨fun h => h.out, fun h => ⟨h⟩⟩, ⟨fun h => h.out, fun h => ⟨h⟩⟩⟩
```

```
end DomainTheory.NeighbourhoodSystems
```

4.8.7 Computable maps form a category

Identity, composition, application, and principal elements stay inside the computable maps and computable elements.

Theorem (`apply_isComputableElement`). Applying a computable approximable map to a computable element yields a computable element.

Proof. The required relation is obtained from recursively enumerable or decidable index relations by pairing, composition, conjunction, and projection. ■

Theorem (`comp_isComputable`). The composite of two computable approximable maps is computable.

Proof. The required relation is obtained from recursively enumerable or decidable index relations by pairing, composition, conjunction, and projection. ■

Theorem (`idMap_isComputable`). The identity approximable map of an effectively presented system is computable.

Proof. The required relation is obtained from recursively enumerable or decidable index relations by pairing, composition, conjunction, and projection. ■

Theorem (`principal_isComputableElement`). Every principal element generated by an enumerated neighbourhood is computable.

Proof. The required relation is obtained from recursively enumerable or decidable index relations by pairing, composition, conjunction, and projection. ■

`computableCategory.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_2
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.idMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap

variable {α β ? : Type*}
variable {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}

theorem apply_isComputableElement {P : ComputablePresentation V} {Q :
    ComputablePresentation W}
    {f : ApproximableMap V W} (hf : ComputableMap P Q f) {x : V.Element}
    (hx : ComputableElement P x) : ComputableElement Q (f.toElementMap x) := by
  have hf' : REPred (fun s => f.rel (P.X s.unpair.1) (Q.X s.unpair.2)) := hf.out
  have hx' : REPred (fun n => x.mem (P.X n)) := hx.out
  constructor
  refine REPred.of_iff (fun m => ?_) ((hx'.comp Nat.Primrec.left).and hf').proj
  simp only [mem_toElementMap, unpair_pair_fst, unpair_pair_snd]
  constructor
  · rintro ⟨X, hX, hfX⟩
```

```
    obtain ⟨n, rfl⟩ := P.surj (x.sub hX)
    exact ⟨n, hX, hfX⟩
  · rintro ⟨n, hxn, hfn⟩
    exact ⟨P.X n, hxn, hfn⟩

theorem comp_isComputable {U : NeighborhoodSystem ?}
    {P : ComputablePresentation V} {Q : ComputablePresentation W} {R :
    ComputablePresentation U}
    {f : ApproximableMap V W} {g : ApproximableMap W U}
    (hf : ComputableMap P Q f) (hg : ComputableMap Q R g) :
    ComputableMap P R (g.comp f) := by
  constructor
  have hf' : REPred (fun s => f.rel (P.X s.unpair.1) (Q.X s.unpair.2)) := hf.out
  have hg' : REPred (fun s => g.rel (Q.X s.unpair.1) (R.X s.unpair.2)) := hg.out
  have hgf : Nat.Primrec (fun u => Nat.pair u.unpair.2.unpair.1 u.unpair.1) :=
    Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.right) Nat.Primrec.left
  have hgg : Nat.Primrec (fun u => Nat.pair u.unpair.1 u.unpair.2.unpair.2) :=
    Nat.Primrec.pair Nat.Primrec.left (Nat.Primrec.right.comp Nat.Primrec.right)
  refine REPred.of_iff (fun t => ?_) ((hf'.comp hgf).and (hg'.comp hgg)).proj
  simp only [comp_rel, unpair_pair_fst, unpair_pair_snd]
  constructor
  · rintro ⟨Y, hfY, hgY⟩
    obtain ⟨l, rfl⟩ := Q.surj (g.rel_dom hgY)
    exact ⟨l, hfY, hgY⟩
  · rintro ⟨l, hfl, hgl⟩
    exact ⟨Q.X l, hfl, hgl⟩

theorem idMap_isComputable (P : ComputablePresentation V) :
    ComputableMap P P (idMap V) :=
  ⟨(RecDecidable.of_iff (fun t => by
    simp only [idMap_rel]
    exact ⟨fun h => h.2.2, fun h => ⟨P.mem_X _, P.mem_X _, h⟩⟩)
    P.incl_computable).re⟩

theorem principal_isComputableElement (P : ComputablePresentation V) (n : ℕ) :
    ComputableElement P (V.principal (P.mem_X n)) := by
  have hg : Nat.Primrec (fun m => Nat.pair n m) :=
    ((Nat.Primrec.const n).pair primrec_id).of_eq (fun _ => rfl)
  have hrec : RecDecidable (fun m => P.X n ⊆ P.X m) :=
    RecDecidable.of_iff (fun m => by simp only [unpair_pair_fst, unpair_pair_snd])
      (P.incl_computable.comp hg)
  constructor
  refine (RecDecidable.of_iff (fun m => ?_) hrec).re
  simp only [mem_principal]
  exact ⟨fun h => h.2, fun h => ⟨P.mem_X m, h⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.8.8 Proposition 7.3 — computable maps form a category

PROPOSITION 7.3. The identity map on an effectively given domain is computable; the composition of computable mappings on effectively given domains is again computable. □

The proofs for 7.3 are so trivial they are hardly worth an exercise. Note the immediate and useful consequence: if $f : D \to E$ is computable and $x \in |D|$ is computable, then $f(x) \in |E|$ is also computable.

The next result is, however, worth working out even though it is quite easy.

The development is with the computable maps form a category peer.

Theorem (`proposition_73`). Application of a computable map to a computable element is computable; composition of computable maps is computable; the identity map of an effectively given domain is computable; and each principal neighbourhood of a presentation is a computable element.

Proof. The four facts of the computable-category peer. ■

`Proposition_7_3.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .computableCategory

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Proposition 7.3: identities and composition of computable maps are
computable, so the image of a computable element is computable, and principal
elements of a presentation are computable. -/
theorem proposition_73 {α β ? : Type*} {V : NeighborhoodSystem α} {W :
    NeighborhoodSystem β}
    {U : NeighborhoodSystem ?} (P : ComputablePresentation V) (Q :
    ComputablePresentation W)
    (R : ComputablePresentation U) :
    (∀ {f : ApproximableMap V W} {x : V.Element},
        ComputableMap P Q f → ComputableElement P x →
          ComputableElement Q (f.toElementMap x)) ∧
      (∀ {f : ApproximableMap V W} {g : ApproximableMap W U},
        ComputableMap P Q f → ComputableMap Q R g → ComputableMap P R (g.comp f)) ∧
        ComputableMap P P (idMap V) ∧
          (∀ n, ComputableElement P (V.principal (P.mem_X n))) :=
  ⟨fun hf hx => apply_isComputableElement hf hx,
    ⟨fun hf hg => comp_isComputable hf hg,
      ⟨idMap_isComputable P, fun n => principal_isComputableElement P n⟩⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.8.9 Effective sum

The sum of effectively given neighbourhood systems is effectively given, and the sum combinators are computable.

Definition (`sumEnum`). The sum enumeration uses tag zero for left neighbourhoods, tag one for right neighbourhoods, and every other tag for the master neighbourhood.

Theorem (`sumEnum_zero`). An index tagged by zero denotes the corresponding left neighbourhood.

Proof. The outer tag test takes its zero branch, which returns the left injection at the second component of the index. ■

Theorem (`sumEnum_one`). An index tagged by one denotes the corresponding right neighbourhood.

Proof. The zero branch is excluded and the next tag test takes its one branch, returning the right injection at the second component. ■

Theorem (`inj0_eq_iff`). Two left-injected neighbourhoods are equal exactly when their underlying left neighbourhoods are equal.

Proof. Injectivity of the corresponding tagged embedding gives the forward implication, and substitution gives the converse. ■

Theorem (`inj0_eq_inj1_elim`). A nonempty left-injected neighbourhood cannot equal a right-injected neighbourhood.

Proof. Choose an element of the assumed nonempty summand; its left and right tags cannot coincide, contradicting the proposed equality. ■

Theorem (`inj0_ne_inj1_of_nonempty`). A left-injected neighbourhood cannot equal a nonempty right-injected neighbourhood.

Proof. Choose an element of the assumed nonempty summand; its left and right tags cannot coincide, contradicting the proposed equality. ■

Theorem (`inj0_ne_sumMaster`). No left-injected neighbourhood equals the master neighbourhood of the sum.

Proof. The sum master contains the distinguished untagged point, whereas an injected summand contains only tagged points. ■

Theorem (`inj1_eq_iff`). Two right-injected neighbourhoods are equal exactly when their underlying right neighbourhoods are equal.

Proof. Injectivity of the corresponding tagged embedding gives the forward implication, and substitution gives the converse. ■

Theorem (`inj1_ne_sumMaster`). No right-injected neighbourhood equals the master neighbourhood of the sum.

Proof. The sum master contains the distinguished untagged point, whereas an injected summand contains only tagged points. ■

Theorem (`recDec_setEq0`). Equality of left neighbourhoods whose indices are computed primitively is recursively decidable.

Proof. Set equality is mutual inclusion, so compose the presentation's decidable inclusion relation with the two primitive-recursive index functions in both orders and conjoin the results. ■

Theorem (`recDec_setEq1`). Equality of right neighbourhoods whose indices are computed primitively is recursively decidable.

Proof. Set equality is mutual inclusion, so compose the presentation's decidable inclusion relation with the two primitive-recursive index functions in both orders and conjoin the results. ■

Theorem (`sumEnum_master`). Every index tagged by a number other than zero or one denotes the master neighbourhood of the sum.

Proof. Both distinguished tag tests fail, so the remaining branch is the sum master neighbourhood. ■

Theorem (`sumEnum_mem`). Every set produced by the tagged sum enumeration is a sum neighbourhood.

Proof. The three tag cases are respectively a valid left neighbourhood, a valid right neighbourhood, and the master neighbourhood, all admitted by the definition of the sum system. ■

Theorem (`sumEnum_nonempty`). Every neighbourhood produced by the tagged sum enumeration is nonempty.

Proof. Inspect the tag: a left or right neighbourhood is nonempty by the corresponding factor hypothesis, while the master case contains the distinguished untagged point. ■

Theorem (`sumEnum_subset_sumMaster`). Every neighbourhood in the sum enumeration is contained in the master neighbourhood.

Proof. In each tag case the injected factor neighbourhood lies in the sum master; the remaining tag already denotes the master itself. ■

Theorem (`sumEnum_inter_sumMaster`). Intersecting an enumerated sum neighbourhood with the master neighbourhood leaves it unchanged.

Proof. Every enumerated sum neighbourhood is contained in the master neighbourhood, so the left-hand absorption law for intersection applies. ■

Theorem (`sumMaster_inter_sumEnum`). Intersecting the master neighbourhood with an enumerated sum neighbourhood leaves that neighbourhood unchanged.

Proof. Every enumerated sum neighbourhood lies inside the master neighbourhood, so the right-hand absorption law for intersection applies. ■

Theorem (`tag_trichotomy`). Every natural-number tag is zero, one, or distinct from both zero and one.

Proof. Decidable equality first separates zero; in the remaining case it separates one, leaving a tag unequal to both. ■

Theorem (`sumEnum_eq_iff`). Two tagged sum indices denote the same neighbourhood exactly when both are master tags, or both have the same summand tag and equal underlying factor neighbourhoods.

Proof. The nine combinations of left, right, and master tags are disjoint: equal tags reduce by injectivity to equality in one factor, and two master tags are automatically equal. ■

Theorem (`eqSEdec`). Equality between two neighbourhoods in the tagged sum enumeration is recursively decidable.

Proof. Decide the tags of both indices. Master tags agree with one another, matching left or right tags invoke decidable equality in that factor, and all mixed tags are unequal. ■

Definition (`sumPresentation`). The tagged enumeration of left, right, and master neighbourhoods forms a computable presentation of the sum.

Theorem (`sum_isEffectivelyGiven`). The sum neighbourhood system is effectively given.

Proof. Choose computable presentations of the two factors; their tagged sum presentation supplies a computable presentation of the sum. ■

Theorem (`sumPresentation_X`). The neighbourhood family of the sum presentation is exactly the tagged sum enumeration.

Proof. This is the corresponding defining equation after substituting the displayed arguments. ■

Theorem (`inMap0_isComputable`). The left injection into an effectively presented sum is computable.

Proof. Split the target index into left, right and master tags. The left case is exactly decidable inclusion in the first factor, the right case is impossible by nonemptiness, and the master case always holds. ■

Theorem (`inMap1_isComputable`). The right injection into an effectively presented sum is computable.

Proof. Split the target index into left, right and master tags. The right case is exactly decidable inclusion in the second factor, the left case is impossible by nonemptiness, and the master case always holds. ■

Theorem (`outMap0_isComputable`). The map extracting the left component of an effectively presented sum is computable.

Proof. Choose an index for the first master neighbourhood. A left-tagged source reduces to factor inclusion, while right-tagged and master sources both reduce to inclusion of that chosen master index. ■

Theorem (`outMap1_isComputable`). The map extracting the right component of an effectively presented sum is computable.

Proof. Choose an index for the second master neighbourhood. A right-tagged source reduces to factor inclusion, while left-tagged and master sources both reduce to inclusion of that chosen master index. ■

Theorem (`sumMap_isComputable`). The map induced componentwise by two computable maps between sums is computable.

Proof. The target master case is automatic. Matching left tags use the recursively enumerable relation for the first component map, matching right tags use that for the second, and all mismatched nonmaster tags are excluded by disjointness and nonemptiness. ■

`sumEffective.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_7
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_3
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.sumSys
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.inducedMaps

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive IndefProduct ApproximableMap

section
variable {α β ? α' β' : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

section Sum

variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}

def sumEnum (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁) (t : ℕ) :
    Set (Option (α ? β)) :=
  if t.unpair.1 = 0 then inj₀ (P₀.X t.unpair.2)
  else if t.unpair.1 = 1 then inj₁ (P₁.X t.unpair.2)
  else sumMaster V₀ V₁

end Sum

section Sum

variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}

variable {P₀ : ComputablePresentation V₀} {P₁ : ComputablePresentation V₁}

@[simp] theorem sumEnum_zero {t : ℕ} (h : t.unpair.1 = 0) :
    sumEnum P₀ P₁ t = inj₀ (P₀.X t.unpair.2) := by unfold sumEnum; rw [ite_eq_left h]

@[simp] theorem sumEnum_one {t : ℕ} (h : t.unpair.1 = 1) :
    sumEnum P₀ P₁ t = inj₁ (P₁.X t.unpair.2) := by
  have h0 : t.unpair.1 = 0 := by omega
  unfold sumEnum; rw [ite_eq_right h0, ite_eq_left h]

theorem inj₀_eq_iff {A B : Set α} : (inj₀ A : Set (Option (α ? β))) = inj₀ B ↔ A = B
```

```
    :=
  ⟨inj₀_injective, fun h => h ▶ rfl⟩

theorem inj₀_eq_inj₁_elim {A : Set α} {B : Set β} (hA : A.Nonempty)
    (h : (inj₀ A : Set (Option (α ⊕ β))) = inj₁ B) : False := by
  obtain ⟨a, ha⟩ := hA; exact il_mem_inj₁ (h ▶ il_mem_inj₀.mpr ha)

theorem inj₀_ne_inj₁_of_nonempty {A : Set α} {B : Set β} (hB : B.Nonempty) :
    (inj₀ A : Set (Option (α ⊕ β))) = inj₁ B := by
  rintro h; obtain ⟨b, hb⟩ := hB
  exact ir_mem_inj₀ (h.symm ▶ ir_mem_inj₁.mpr hb)

theorem inj₀_ne_sumMaster {A : Set α} : (inj₀ A : Set (Option (α ⊕ β))) = sumMaster V₀
    V₁ :=
  fun h => none_mem_inj₀ (h.symm ▶ none_mem_sumMaster)

theorem inj₁_eq_iff {A B : Set β} : (inj₁ A : Set (Option (α ⊕ β))) = inj₁ B ↔ A = B
    :=
  ⟨inj₁_injective, fun h => h ▶ rfl⟩

theorem inj₁_ne_sumMaster {B : Set β} : (inj₁ B : Set (Option (α ⊕ β))) = sumMaster V₀
    V₁ :=
  fun h => none_mem_inj₁ (h.symm ▶ none_mem_sumMaster)

theorem recDec_setEq₀ {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    RecDecidable (fun u => P₀.X (f u) = P₀.X (g u)) := by
  have hincl : RecDecidable (fun s => P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) := P₀.
    incl_computable
  refine RecDecidable.of_iff (fun u => ?_) ((hincl.comp (hf.pair hg)).and (hincl.comp
    (hg.pair hf)))
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact ⟨fun h => ⟨h.subset, h.symm.subset⟩, fun ⟨h1, h2⟩ => Set.Subset.antisymm h1 h2⟩

theorem recDec_setEq₁ {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    RecDecidable (fun u => P₁.X (f u) = P₁.X (g u)) := by
  have hincl : RecDecidable (fun s => P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) := P₁.
    incl_computable
  refine RecDecidable.of_iff (fun u => ?_) ((hincl.comp (hf.pair hg)).and (hincl.comp
    (hg.pair hf)))
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact ⟨fun h => ⟨h.subset, h.symm.subset⟩, fun ⟨h1, h2⟩ => Set.Subset.antisymm h1 h2⟩

theorem sumEnum_master {t : ℕ} (h0 : t.unpair.1 = 0) (h1 : t.unpair.1 = 1) :
    sumEnum P₀ P₁ t = sumMaster V₀ V₁ := by unfold sumEnum; rw [ite_eq_right h0,
    ite_eq_right h1]

theorem sumEnum_mem (t : ℕ) : (sum V₀ V₁ h₀ h₁).mem (sumEnum P₀ P₁ t) := by
  unfold sumEnum
  split
  · exact Or.inr (Or.inl ⟨_, P₀.mem_X _, rfl⟩)
  · split
    · exact Or.inr (Or.inr ⟨_, P₁.mem_X _, rfl⟩)
    · exact Or.inl rfl

include h₀ h₁ in
theorem sumEnum_nonempty (t : ℕ) : (sumEnum P₀ P₁ t).Nonempty := by
```

```
  unfold sumEnum
  split
  · exact inj₀_nonempty (h₀ _ (P₀.mem_X _))
  · split
    · exact inj₁_nonempty (h₁ _ (P₁.mem_X _))
    · exact ⟨none, none_mem_sumMaster⟩

theorem sumEnum_subset_sumMaster (n : ℕ) :
    sumEnum P₀ P₁ n ⊆ sumMaster V₀ V₁ := by
  unfold sumEnum
  split
  · exact inj₀_subset_sumMaster (P₀.mem_X _)
  · split
    · exact inj₁_subset_sumMaster (P₁.mem_X _)
    · exact subset_rfl

theorem sumEnum_inter_sumMaster (n : ℕ) :
    sumEnum P₀ P₁ n ∩ sumMaster V₀ V₁ = sumEnum P₀ P₁ n :=
  Set.inter_eq_left.mpr (sumEnum_subset_sumMaster n)

theorem sumMaster_inter_sumEnum (m : ℕ) :
    sumMaster V₀ V₁ ∩ sumEnum P₀ P₁ m = sumEnum P₀ P₁ m :=
  Set.inter_eq_right.mpr (sumEnum_subset_sumMaster m)

end Sum

theorem tag_trichotomy (n : ℕ) : n = 0 ∨ n = 1 ∨ (n = 0 ∧ n = 1) := by
  have h_dec : (n = 0) ∨ (n = 0) := by
    by_cases h : n = 0
    · exact Or.inr h
    · exact Or.inl h
  rcases h_dec with h | h
  · have h'_dec : (n = 1) ∨ (n = 1) := by
      by_cases h' : n = 1
      · exact Or.inr h'
      · exact Or.inl h'
    rcases h'_dec with h' | h'
    · exact Or.inr (Or.inr ⟨h, h'⟩)
    · exact Or.inr (Or.inl h')
  · exact Or.inl h

section Sum

variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}

variable {P₀ : ComputablePresentation V₀} {P₁ : ComputablePresentation V₁}

include h₁ in

theorem sumEnum_eq_iff (x y : ℕ) :
    sumEnum P₀ P₁ x = sumEnum P₀ P₁ y ↔
      ((x.unpair.1 = 0 ∧ x.unpair.1 = 1) ∧ (y.unpair.1 = 0 ∧ y.unpair.1 = 1))
        ∨ (((x.unpair.1 = 0 ∧ y.unpair.1 = 0) ∧ P₀.X x.unpair.2 = P₀.X y.unpair.2)
          ∨ ((x.unpair.1 = 1 ∧ y.unpair.1 = 1) ∧ P₁.X x.unpair.2 = P₁.X y.unpair.2)) :=
   by
  rcases tag_trichotomy x.unpair.1 with hx | hx | ⟨hx0, hx1⟩ <;>
```

```
    rcases tag_trichotomy y.unpair.1 with hy | hy | ⟨hy0, hy1⟩
  ·
    rw [sumEnum_zero hx, sumEnum_zero hy, inj₀_eq_iff]
    constructor
    · intro he; exact Or.inr (Or.inl ⟨⟨hx, hy⟩, he⟩)
    · rintro (⟨⟨e1, e2⟩, _⟩ | ⟨_, he⟩ | ⟨⟨e1, e2⟩, _⟩)
      · exfalso; omega
      · exact he
      · exfalso; omega
  ·
    rw [sumEnum_zero hx, sumEnum_one hy]
    constructor
    · intro h; exact absurd h (inj₀_ne_inj₁_of_nonempty (h₁ _ (P₁.mem_X _)))
    · rintro (⟨⟨e1, e2⟩, _⟩ | ⟨⟨e1, e2⟩, _⟩ | ⟨⟨e1, e2⟩, _⟩) <;> exfalso <;> omega
  ·
    rw [sumEnum_zero hx, sumEnum_master hy0 hy1]
    constructor
    · intro h; exact absurd h inj₀_ne_sumMaster
    · rintro (⟨⟨e1, e2⟩, _⟩ | ⟨⟨e1, e2⟩, _⟩ | ⟨⟨e1, e2⟩, _⟩) <;> exfalso <;> omega
  ·
    rw [sumEnum_one hx, sumEnum_zero hy]
    constructor
    · intro h; exact absurd h.symm (inj₀_ne_inj₁_of_nonempty (h₁ _ (P₁.mem_X _)))
    · rintro (⟨⟨e1, e2⟩, _⟩ | ⟨⟨e1, e2⟩, _⟩ | ⟨⟨e1, e2⟩, _⟩) <;> exfalso <;> omega
  ·
    rw [sumEnum_one hx, sumEnum_one hy, inj₁_eq_iff]
    constructor
    · intro he; exact Or.inr (Or.inr ⟨⟨hx, hy⟩, he⟩)
    · rintro (⟨⟨e1, e2⟩, _⟩ | ⟨⟨e1, e2⟩, _⟩ | ⟨_, he⟩)
      · exfalso; omega
      · exfalso; omega
      · exact he
  ·
    rw [sumEnum_one hx, sumEnum_master hy0 hy1]
    constructor
    · intro h; exact absurd h inj₁_ne_sumMaster
    · rintro (⟨⟨e1, e2⟩, _⟩ | ⟨⟨e1, e2⟩, _⟩ | ⟨⟨e1, e2⟩, _⟩) <;> exfalso <;> omega
  ·
    rw [sumEnum_master hx0 hx1, sumEnum_zero hy]
    constructor
    · intro h; exact absurd h.symm inj₀_ne_sumMaster
    · rintro (⟨⟨e1, e2⟩, ⟨e3, e4⟩⟩ | ⟨⟨e1, e2⟩, _⟩ | ⟨⟨e1, e2⟩, _⟩) <;> exfalso <;> omega
  ·
    rw [sumEnum_master hx0 hx1, sumEnum_one hy]
    constructor
    · intro h; exact absurd h.symm inj₁_ne_sumMaster
    · rintro (⟨⟨e1, e2⟩, ⟨e3, e4⟩⟩ | ⟨⟨e1, e2⟩, _⟩ | ⟨⟨e1, e2⟩, _⟩) <;> exfalso <;> omega
  ·
    rw [sumEnum_master hx0 hx1, sumEnum_master hy0 hy1]
    exact ⟨fun _ => Or.inl ⟨⟨hx0, hx1⟩, ⟨hy0, hy1⟩⟩, fun _ => rfl⟩

include h₁ in

theorem eqSEdec :
    RecDecidable (fun u => sumEnum P₀ P₁ u.unpair.1 = sumEnum P₀ P₁ u.unpair.2) := by
  have hX0 : RecDecidable (fun u => u.unpair.1.unpair.1 = 0) :=
```

```
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have hX1 : RecDecidable (fun u => u.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 1)
  have hY0 : RecDecidable (fun u => u.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
  have hY1 : RecDecidable (fun u => u.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  have hE0 : RecDecidable (fun u => P₀.X u.unpair.1.unpair.2 = P₀.X u.unpair.2.unpair
    .2) :=
    recDec_setEq₀ (Nat.Primrec.right.comp Nat.Primrec.left) (Nat.Primrec.right.comp Nat
    .Primrec.right)
  have hE1 : RecDecidable (fun u => P₁.X u.unpair.1.unpair.2 = P₁.X u.unpair.2.unpair
    .2) :=
    recDec_setEq₁ (Nat.Primrec.right.comp Nat.Primrec.left) (Nat.Primrec.right.comp Nat
    .Primrec.right)
  exact RecDecidable.of_iff (fun u => sumEnum_eq_iff (h₁ := h₁) u.unpair.1 u.unpair.2)
    (((hX0.not.and hX1.not).and (hY0.not.and hY1.not)).or
      (((hX0.and hY0).and hE0).or ((hX1.and hY1).and hE1)))

include h₀ h₁

def sumPresentation (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁) :
    ComputablePresentation (sum V₀ V₁ h₀ h₁) where
  X := sumEnum P₀ P₁
  mem_X t := sumEnum_mem t
  surj := by
    rintro W (rfl | ⟨X, hX, rfl⟩ | ⟨Y, hY, rfl⟩)
    · exact ⟨Nat.pair 2 0, sumEnum_master (by rw [unpair_pair_fst]; decide)
        (by rw [unpair_pair_fst]; decide)⟩
    · obtain ⟨n, rfl⟩ := P₀.surj hX
      exact ⟨Nat.pair 0 n, by rw [sumEnum_zero (by rw [unpair_pair_fst]),
    unpair_pair_snd]⟩
    · obtain ⟨n, rfl⟩ := P₁.surj hY
      exact ⟨Nat.pair 1 n, by rw [sumEnum_one (by rw [unpair_pair_fst]),
    unpair_pair_snd]⟩
  interEq_computable := by
    have hMK : RecDecidable (fun t => sumEnum P₀ P₁ t.unpair.2.unpair.1
        = sumEnum P₀ P₁ t.unpair.2.unpair.2) := by
      refine RecDecidable.of_iff (fun t => ?_) ((eqSEdec (P₀ := P₀) (P₁ := P₁) (h₁ := h₁
    )).comp
        ((Nat.Primrec.left.comp Nat.Primrec.right).pair (Nat.Primrec.right.comp Nat.
    Primrec.right)))
      simp only [unpair_pair_fst, unpair_pair_snd]
    have hNK : RecDecidable (fun t => sumEnum P₀ P₁ t.unpair.1
        = sumEnum P₀ P₁ t.unpair.2.unpair.2) := by
      refine RecDecidable.of_iff (fun t => ?_) ((eqSEdec (P₀ := P₀) (P₁ := P₁) (h₁ := h₁
    )).comp
        (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)))
      simp only [unpair_pair_fst, unpair_pair_snd]
    have hA0 : RecDecidable (fun t => t.unpair.1.unpair.1 = 0) :=
      RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
    0)
    have hA1 : RecDecidable (fun t => t.unpair.1.unpair.1 = 1) :=
      RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
    1)
    have hB0 : RecDecidable (fun t => t.unpair.2.unpair.1.unpair.1 = 0) :=
```

```
  RecDecidable.natEq (Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.
right))
    (Nat.Primrec.const 0)
have hB1 : RecDecidable (fun t => t.unpair.2.unpair.1.unpair.1 = 1) :=
  RecDecidable.natEq (Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.
right))
    (Nat.Primrec.const 1)
have hC0 : RecDecidable (fun t => t.unpair.2.unpair.2.unpair.1 = 0) :=
  RecDecidable.natEq (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.
right))
    (Nat.Primrec.const 0)
have hC1 : RecDecidable (fun t => t.unpair.2.unpair.2.unpair.1 = 1) :=
  RecDecidable.natEq (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.
right))
    (Nat.Primrec.const 1)
have hg : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2
    (Nat.pair t.unpair.2.unpair.1.unpair.2 t.unpair.2.unpair.2.unpair.2)) :=
  (Nat.Primrec.right.comp Nat.Primrec.left).pair
    ((Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
      (Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
have hIE0 : RecDecidable (fun t => P₀.X t.unpair.1.unpair.2 ∩ P₀.X t.unpair.2.
unpair.1.unpair.2
    = P₀.X t.unpair.2.unpair.2.unpair.2) := by
  refine RecDecidable.of_iff (fun t => ?_) (P₀.interEq_computable.comp hg)
  simp only [unpair_pair_fst, unpair_pair_snd]
have hIE1 : RecDecidable (fun t => P₁.X t.unpair.1.unpair.2 ∩ P₁.X t.unpair.2.
unpair.1.unpair.2
    = P₁.X t.unpair.2.unpair.2.unpair.2) := by
  refine RecDecidable.of_iff (fun t => ?_) (P₁.interEq_computable.comp hg)
  simp only [unpair_pair_fst, unpair_pair_snd]
refine RecDecidable.of_iff (fun t => ?_)
  (((hA0.not.and hA1.not).and hMK).or
    (((hB0.not.and hB1.not).and hNK).or
      ((((hA0.and hB0).and hC0).and hIE0).or
        (((hA1.and hB1).and hC1).and hIE1))))
dsimp only
rcases tag_trichotomy t.unpair.1.unpair.1 with hna | hna | ⟨hna0, hna1⟩
· rcases tag_trichotomy t.unpair.2.unpair.1.unpair.1 with hnb | hnb | ⟨hnb0, hnb1⟩
  ·
    rw [sumEnum_zero hna, sumEnum_zero hnb, inj₀_inter]
    rcases tag_trichotomy t.unpair.2.unpair.2.unpair.1 with hnc | hnc | ⟨hnc0, hnc1
⟩
    · rw [sumEnum_zero hnc, inj₀_eq_iff]
      constructor
      · intro h; exact Or.inr (Or.inr (Or.inl ⟨⟨⟨hna, hnb⟩, hnc⟩, h⟩))
      · rintro (⟨⟨e1, _⟩, _⟩ | ⟨⟨e1, _⟩, _⟩ | ⟨⟨⟨_, _⟩, _⟩, h⟩ | ⟨⟨⟨e1, _⟩, _⟩, _⟩)
        · exfalso; omega
        · exfalso; omega
        · exact h
        · exfalso; omega
    · rw [sumEnum_one hnc]
      constructor
      · intro h; exact absurd h (inj₀_ne_inj₁_of_nonempty (h₁ _ (P₁.mem_X _)))
      · rintro (⟨⟨e1, _⟩, _⟩ | ⟨⟨e1, _⟩, _⟩ | ⟨⟨⟨_, _⟩, e3⟩, _⟩ | ⟨⟨⟨e1, _⟩, _⟩, _⟩) <;>
exfalso <;> omega
    · rw [sumEnum_master hnc0 hnc1]
```

```
      constructor
      · intro h; exact absurd h inj₀_ne_sumMaster
      · rintro (⟨⟨e1, _⟩, _⟩ | ⟨⟨e1, _⟩, _⟩ | ⟨⟨⟨_, _⟩, e3⟩, _⟩ | ⟨⟨⟨e1, _⟩, _⟩, _⟩) <;>
exfalso <;> omega
  ·
    rw [sumEnum_zero hna, sumEnum_one hnb, inj₀_inter_inj₁]
    constructor
    · intro h
      obtain ⟨x, hx⟩ := sumEnum_nonempty (h₀ := h₀) (h₁ := h₁) t.unpair.2.unpair.2
      rw [← h] at hx
      exact absurd hx (Set.notMem_empty x)
    · rintro (⟨⟨e1, _⟩, _⟩ | ⟨⟨_, e2⟩, _⟩ | ⟨⟨⟨_, e2⟩, _⟩, _⟩ | ⟨⟨⟨e1, _⟩, _⟩, _⟩) <;>
exfalso <;> omega
  ·
    rw [sumEnum_master hnb0 hnb1, sumEnum_inter_sumMaster]
    constructor
    · intro h; exact Or.inr (Or.inl ⟨⟨hnb0, hnb1⟩, h⟩)
    · rintro (⟨⟨e1, _⟩, _⟩ | ⟨_, h⟩ | ⟨⟨⟨_, e2⟩, _⟩, _⟩ | ⟨⟨⟨e1, _⟩, _⟩, _⟩)
      · exfalso; omega
      · exact h
      · exfalso; omega
      · exfalso; omega
· rcases tag_trichotomy t.unpair.2.unpair.1.unpair.1 with hnb | hnb | ⟨hnb0, hnb1⟩
  ·
    rw [sumEnum_one hna, sumEnum_zero hnb, Set.inter_comm, inj₀_inter_inj₁]
    constructor
    · intro h
      obtain ⟨x, hx⟩ := sumEnum_nonempty (h₀ := h₀) (h₁ := h₁) t.unpair.2.unpair.2
      rw [← h] at hx
      exact absurd hx (Set.notMem_empty x)
    · rintro (⟨⟨_, e2⟩, _⟩ | ⟨⟨e1, _⟩, _⟩ | ⟨⟨⟨e1, _⟩, _⟩, _⟩ | ⟨⟨⟨_, e2⟩, _⟩, _⟩) <;>
exfalso <;> omega
  ·
    rw [sumEnum_one hna, sumEnum_one hnb, inj₁_inter]
    rcases tag_trichotomy t.unpair.2.unpair.2.unpair.1 with hnc | hnc | ⟨hnc0, hnc1
⟩
    · rw [sumEnum_zero hnc]
      constructor
      · intro h; exact (inj₀_eq_inj₁_elim (h₀ _ (P₀.mem_X _)) h.symm).elim
      · rintro (⟨⟨_, e2⟩, _⟩ | ⟨⟨_, e2⟩, _⟩ | ⟨⟨⟨e1, _⟩, _⟩, _⟩ | ⟨⟨⟨_, _⟩, e3⟩, _⟩) <;>
exfalso <;> omega
    · rw [sumEnum_one hnc, inj₁_eq_iff]
      constructor
      · intro h; exact Or.inr (Or.inr (Or.inr ⟨⟨hna, hnb⟩, hnc⟩, h⟩))
      · rintro (⟨⟨_, e2⟩, _⟩ | ⟨⟨_, e2⟩, _⟩ | ⟨⟨⟨e1, _⟩, _⟩, _⟩ | ⟨⟨⟨_, _⟩, _⟩, h⟩)
        · exfalso; omega
        · exfalso; omega
        · exfalso; omega
        · exact h
    · rw [sumEnum_master hnc0 hnc1]
      constructor
      · intro h; exact absurd h inj₁_ne_sumMaster
      · rintro (⟨⟨_, e2⟩, _⟩ | ⟨⟨_, e2⟩, _⟩ | ⟨⟨⟨e1, _⟩, _⟩, _⟩ | ⟨⟨⟨_, _⟩, e3⟩, _⟩) <;>
exfalso <;> omega
  ·
    rw [sumEnum_master hnb0 hnb1, sumEnum_inter_sumMaster]
```

```
      constructor
      · intro h; exact Or.inr (Or.inl ⟨⟨hnb0, hnb1⟩, h⟩)
      · rintro (⟨⟨_, e2⟩, _⟩ | ⟨_, h⟩ | ⟨⟨⟨_, e2⟩, _⟩, _⟩ | ⟨⟨⟨_, e2⟩, _⟩, _⟩)
        · exfalso; omega
        · exact h
        · exfalso; omega
        · exfalso; omega
  .
    rw [sumEnum_master hna0 hna1, sumMaster_inter_sumEnum]
    constructor
    · intro h; exact Or.inl ⟨⟨hna0, hna1⟩, h⟩
    · rintro (⟨_, h⟩ | ⟨⟨e1, e2⟩, h⟩ | ⟨⟨⟨e1, _⟩, _⟩, _⟩ | ⟨⟨⟨e1, _⟩, _⟩, _⟩)
      · exact h
      · rw [sumEnum_master e1 e2]; exact h
      · exfalso; omega
      · exfalso; omega
cons_computable := by
  have hA0 : RecDecidable (fun s => s.unpair.1.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
  0)
  have hA1 : RecDecidable (fun s => s.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
  1)
  have hB0 : RecDecidable (fun s => s.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const
  0)
  have hB1 : RecDecidable (fun s => s.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const
  1)
  have hgc : Nat.Primrec (fun s => Nat.pair s.unpair.1.unpair.2 s.unpair.2.unpair.2)
   :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
  Primrec.right)
  have hC0 : RecDecidable (fun s => ∃ k, P₀.X k ⊆ P₀.X s.unpair.1.unpair.2 ∩ P₀.X s.
  unpair.2.unpair.2)
      := by
    refine RecDecidable.of_iff (fun s => ?_) (P₀.cons_computable.comp hgc)
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hC1 : RecDecidable (fun s => ∃ k, P₁.X k ⊆ P₁.X s.unpair.1.unpair.2 ∩ P₁.X s.
  unpair.2.unpair.2)
      := by
    refine RecDecidable.of_iff (fun s => ?_) (P₁.cons_computable.comp hgc)
    simp only [unpair_pair_fst, unpair_pair_snd]
  refine RecDecidable.of_iff (fun s => ?_)
    ((hA0.not.and hA1.not).or
      ((hB0.not.and hB1.not).or
        (((hA0.and hB0).and hC0).or ((hA1.and hB1).and hC1))))
  dsimp only
  rcases tag_trichotomy s.unpair.1.unpair.1 with hna | hna | ⟨hna0, hna1⟩
  · rcases tag_trichotomy s.unpair.2.unpair.1 with hnb | hnb | ⟨hnb0, hnb1⟩
    .
      rw [sumEnum_zero hna, sumEnum_zero hnb, inj₀_inter]
      constructor
      · rintro ⟨k, hk⟩
        obtain ⟨X₂, hX₂mem, hX₂eq⟩ := mem_subset_inj₀ (sumEnum_mem (h₀ := h₀) (h₁ := h₁
  ) k) hk
```

```
        obtain ⟨j, hj⟩ := P₀.surj hX₂mem
        refine Or.inr (Or.inr (Or.inl ⟨⟨hna, hnb⟩, j, ?_⟩))
        rw [hj]
        rw [hX₂eq] at hk
        exact inj₀_subset_inj₀.mp hk
      · rintro (⟨e1, _⟩ | ⟨e1, _⟩ | ⟨⟨_, _⟩, j, hj⟩ | ⟨⟨e1, _⟩, _⟩)
        · exfalso; omega
        · exfalso; omega
        · refine ⟨Nat.pair 0 j, ?_⟩

          rw [sumEnum_zero (by rw [unpair_pair_fst]), unpair_pair_snd]
          exact inj₀_subset_inj₀.mpr hj
        · exfalso; omega
    .
      rw [sumEnum_zero hna, sumEnum_one hnb, inj₀_inter_inj₁]
      constructor
      · rintro ⟨k, hk⟩
        obtain ⟨x, hx⟩ := sumEnum_nonempty (h₀ := h₀) (h₁ := h₁) k
        exact absurd (hk hx) (Set.notMem_empty x)
      · rintro (⟨e1, _⟩ | ⟨_, e2⟩ | ⟨⟨_, e2⟩, _⟩ | ⟨⟨e1, _⟩, _⟩) <;> exfalso <;> omega
    .
      rw [sumEnum_master hnb0 hnb1, sumEnum_inter_sumMaster]
      exact ⟨fun _ => Or.inr (Or.inl ⟨hnb0, hnb1⟩), fun _ => ⟨s.unpair.1, subset_rfl⟩⟩
  · rcases tag_trichotomy s.unpair.2.unpair.1 with hnb | hnb | ⟨hnb0, hnb1⟩
    .
      rw [sumEnum_one hna, sumEnum_zero hnb, Set.inter_comm, inj₀_inter_inj₁]
      constructor
      · rintro ⟨k, hk⟩
        obtain ⟨x, hx⟩ := sumEnum_nonempty (h₀ := h₀) (h₁ := h₁) k
        exact absurd (hk hx) (Set.notMem_empty x)
      · rintro (⟨_, e2⟩ | ⟨e1, _⟩ | ⟨⟨e1, _⟩, _⟩ | ⟨⟨_, e2⟩, _⟩) <;> exfalso <;> omega
    .
      rw [sumEnum_one hna, sumEnum_one hnb, inj₁_inter]
      constructor
      · rintro ⟨k, hk⟩
        obtain ⟨Y₂, hY₂mem, hY₂eq⟩ := mem_subset_inj₁ (sumEnum_mem (h₀ := h₀) (h₁ := h₁
  ) k) hk
        obtain ⟨j, hj⟩ := P₁.surj hY₂mem
        refine Or.inr (Or.inr (Or.inr ⟨⟨hna, hnb⟩, j, ?_⟩))
        rw [hj]
        rw [hY₂eq] at hk
        exact inj₁_subset_inj₁.mp hk
      · rintro (⟨_, e2⟩ | ⟨_, e2⟩ | ⟨⟨e1, _⟩, _⟩ | ⟨⟨_, _⟩, j, hj⟩)
        · exfalso; omega
        · exfalso; omega
        · exfalso; omega
        · refine ⟨Nat.pair 1 j, ?_⟩
          rw [sumEnum_one (by rw [unpair_pair_fst]), unpair_pair_snd]
          exact inj₁_subset_inj₁.mpr hj
    .
      rw [sumEnum_master hnb0 hnb1, sumEnum_inter_sumMaster]
      exact ⟨fun _ => Or.inr (Or.inl ⟨hnb0, hnb1⟩), fun _ => ⟨s.unpair.1, subset_rfl⟩⟩
  .
    rw [sumEnum_master hna0 hna1, sumMaster_inter_sumEnum]
    exact ⟨fun _ => Or.inl ⟨hna0, hna1⟩, fun _ => ⟨s.unpair.2, subset_rfl⟩⟩
inter n m :=
```

```
  selectFn (1 - (2 - n.unpair.1)) m
    (selectFn (1 - (2 - m.unpair.1)) n
      (selectFn (1 - n.unpair.1)
        (Nat.pair 0 (P₀.inter n.unpair.2 m.unpair.2))
        (Nat.pair 1 (P₁.inter n.unpair.2 m.unpair.2))))
inter_primrec := by
  have hnTag : Nat.Primrec (fun t => t.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.left
  have hmTag : Nat.Primrec (fun t => t.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hnIdx : Nat.Primrec (fun t => t.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have hmIdx : Nat.Primrec (fun t => t.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hP0 : Nat.Primrec (fun t => Nat.pair 0 (P₀.inter t.unpair.1.unpair.2 t.unpair
  .2.unpair.2)) :=
    (Nat.Primrec.const 0).pair ((P₀.inter_primrec.comp (hnIdx.pair hmIdx)).of_eq fun
  t => by
      simp only [unpair_pair_fst, unpair_pair_snd])
  have hP1 : Nat.Primrec (fun t => Nat.pair 1 (P₁.inter t.unpair.1.unpair.2 t.unpair
  .2.unpair.2)) :=
    (Nat.Primrec.const 1).pair ((P₁.inter_primrec.comp (hnIdx.pair hmIdx)).of_eq fun
  t => by
      simp only [unpair_pair_fst, unpair_pair_snd])
  have hcnMaster : Nat.Primrec (fun t => 1 - (2 - t.unpair.1.unpair.1)) :=
    primrec_sub₂ (Nat.Primrec.const 1) (primrec_sub₂ (Nat.Primrec.const 2) hnTag)
  have hcmMaster : Nat.Primrec (fun t => 1 - (2 - t.unpair.2.unpair.1)) :=
    primrec_sub₂ (Nat.Primrec.const 1) (primrec_sub₂ (Nat.Primrec.const 2) hmTag)
  have hcnZero : Nat.Primrec (fun t => 1 - t.unpair.1.unpair.1) :=
    primrec_sub₂ (Nat.Primrec.const 1) hnTag
  exact primrec_selectFn hcnMaster Nat.Primrec.right
    (primrec_selectFn hcmMaster Nat.Primrec.left
      (primrec_selectFn hcnZero hP0 hP1))
inter_spec := by
  rintro n m ⟨k, hk⟩
  rcases tag_trichotomy n.unpair.1 with hna | hna | ⟨hna0, hna1⟩
  ·
    rw [show 1 - (2 - n.unpair.1) = 0 by omega, selectFn_zero]
    rcases tag_trichotomy m.unpair.1 with hmb | hmb | ⟨hmb0, hmb1⟩
    ·
      rw [show 1 - (2 - m.unpair.1) = 0 by omega, selectFn_zero,
        show 1 - n.unpair.1 = 1 by omega, selectFn_one,
        sumEnum_zero (by rw [unpair_pair_fst]), unpair_pair_snd,
        sumEnum_zero hna, sumEnum_zero hmb, inj₀_inter]
      rw [sumEnum_zero hna, sumEnum_zero hmb, inj₀_inter] at hk
      obtain ⟨X₂, hX₂mem, hX₂eq⟩ := mem_subset_inj₀ (sumEnum_mem (h₀ := h₀) (h₁ := h₁)
  k) hk
      obtain ⟨j, hj⟩ := P₀.surj hX₂mem
      rw [hX₂eq] at hk
      have hjsub : P₀.X j ⊆ P₀.X n.unpair.2 ∩ P₀.X m.unpair.2 := by
        rw [hj]; exact inj₀_subset_inj₀.mp hk
      rw [P₀.inter_spec ⟨j, hjsub⟩]
    ·
      exfalso
      rw [sumEnum_zero hna, sumEnum_one hmb, inj₀_inter_inj₁] at hk
      obtain ⟨x, hx⟩ := sumEnum_nonempty (h₀ := h₀) (h₁ := h₁) k
```

```
        exact absurd (hk hx) (Set.notMem_empty x)
      .
        rw [show 1 - (2 - m.unpair.1) = 1 by omega, selectFn_one,
          sumEnum_master hmb0 hmb1, sumEnum_inter_sumMaster]
    .
      rw [show 1 - (2 - n.unpair.1) = 0 by omega, selectFn_zero]
      rcases tag_trichotomy m.unpair.1 with hmb | hmb | ⟨hmb0, hmb1⟩
      .
        exfalso
        rw [sumEnum_one hna, sumEnum_zero hmb, Set.inter_comm, inj₀_inter_inj₁] at hk
        obtain ⟨x, hx⟩ := sumEnum_nonempty (h₀ := h₀) (h₁ := h₁) k
        exact absurd (hk hx) (Set.notMem_empty x)
      .
        rw [show 1 - (2 - m.unpair.1) = 0 by omega, selectFn_zero,
          show 1 - n.unpair.1 = 0 by omega, selectFn_zero,
          sumEnum_one (by rw [unpair_pair_fst]), unpair_pair_snd,
          sumEnum_one hna, sumEnum_one hmb, inj₁_inter]
        rw [sumEnum_one hna, sumEnum_one hmb, inj₁_inter] at hk
        obtain ⟨Y₂, hY₂mem, hY₂eq⟩ := mem_subset_inj₁ (sumEnum_mem (h₀ := h₀) (h₁ := h₁)
    k) hk
        obtain ⟨j, hj⟩ := P₁.surj hY₂mem
        rw [hY₂eq] at hk
        have hjsub : P₁.X j ⊆ P₁.X n.unpair.2 ∩ P₁.X m.unpair.2 := by
          rw [hj]; exact inj₁_subset_inj₁.mp hk
        rw [P₁.inter_spec ⟨j, hjsub⟩]
      .
        rw [show 1 - (2 - m.unpair.1) = 1 by omega, selectFn_one,
          sumEnum_master hmb0 hmb1, sumEnum_inter_sumMaster]
    .
      rw [show 1 - (2 - n.unpair.1) = 1 by omega, selectFn_one,
        sumEnum_master hna0 hna1, sumMaster_inter_sumEnum]
  masterIdx := Nat.pair 2 0
  masterIdx_spec := by
    rw [sumEnum_master (by rw [unpair_pair_fst]; decide) (by rw [unpair_pair_fst];
    decide)]
    rfl

theorem sum_isEffectivelyGiven (g₀ : V₀.IsEffectivelyGiven) (g₁ : V₁.IsEffectivelyGiven
    ) :
    (sum V₀ V₁ h₀ h₁).IsEffectivelyGiven := by
  obtain ⟨P₀⟩ := g₀; obtain ⟨P₁⟩ := g₁
  exact ⟨sumPresentation P₀ P₁⟩

@[simp] theorem sumPresentation_X (P₀ : ComputablePresentation V₀) (P₁ :
    ComputablePresentation V₁)
    (t : ℕ) : (sumPresentation P₀ P₁ (h₀ := h₀) (h₁ := h₁)).X t = sumEnum P₀ P₁ t := rfl

theorem inMap₀_isComputable (P₀ : ComputablePresentation V₀) (P₁ :
    ComputablePresentation V₁) :
    ComputableMap P₀ (sumPresentation P₀ P₁) (inMap₀ (h₀ := h₀) (h₁ := h₁)) := by
  constructor
  have hincl : RecDecidable (fun s => P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) := P₀.
    incl_computable
  have hmtag0 : RecDecidable (fun s => s.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
  have hmtag1 : RecDecidable (fun s => s.unpair.2.unpair.1 = 1) :=
```

```
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  have hincl0 : RecDecidable (fun s => P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2.unpair.2) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)))
  refine (RecDecidable.of_iff (fun s => ?_)
    ((hmtag0.and hincl0).or (hmtag0.not.and hmtag1.not))).re
  simp only [sumPresentation_X]
  rcases tag_trichotomy s.unpair.2.unpair.1 with hm | hm | ⟨hm0, hm1⟩
  · constructor
    · rintro ⟨_, _, hsub⟩
      rw [sumEnum_zero hm] at hsub
      exact Or.inl ⟨hm, inj₀_subset_inj₀.mp hsub⟩
    · rintro (⟨_, hsub⟩ | ⟨e1, _⟩)
      · refine ⟨P₀.mem_X _, sumEnum_mem _, ?_⟩
        rw [sumEnum_zero hm]; exact inj₀_subset_inj₀.mpr hsub
      · exfalso; omega
  · constructor
    · rintro ⟨_, _, hsub⟩
      rw [sumEnum_one hm] at hsub
      exact (not_inj₀_subset_inj₁ (h₀ _ (P₀.mem_X _)) hsub).elim
    · rintro (⟨e1, _⟩ | ⟨_, e2⟩) <;> exfalso <;> omega
  · constructor
    · intro _; exact Or.inr ⟨hm0, hm1⟩
    · intro _
      refine ⟨P₀.mem_X _, sumEnum_mem _, ?_⟩
      rw [sumEnum_master hm0 hm1]; exact inj₀_subset_sumMaster (P₀.mem_X _)

theorem inMap₁_isComputable (P₀ : ComputablePresentation V₀) (P₁ :
    ComputablePresentation V₁) :
    ComputableMap P₁ (sumPresentation P₀ P₁) (inMap₁ (h₀ := h₀) (h₁ := h₁)) := by
  constructor
  have hincl : RecDecidable (fun s => P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) := P₁.
    incl_computable
  have hmtag0 : RecDecidable (fun s => s.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
  have hmtag1 : RecDecidable (fun s => s.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  have hincl1 : RecDecidable (fun s => P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2.unpair.2) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)))
  refine (RecDecidable.of_iff (fun s => ?_)
    ((hmtag1.and hincl1).or (hmtag0.not.and hmtag1.not))).re
  simp only [sumPresentation_X]
  rcases tag_trichotomy s.unpair.2.unpair.1 with hm | hm | ⟨hm0, hm1⟩
  · constructor
    · rintro ⟨_, _, hsub⟩
      rw [sumEnum_zero hm] at hsub
      exact (not_inj₁_subset_inj₀ (h₁ _ (P₁.mem_X _)) hsub).elim
    · rintro (⟨e1, _⟩ | ⟨_, e2⟩) <;> exfalso <;> omega
  · constructor
    · rintro ⟨_, _, hsub⟩
      rw [sumEnum_one hm] at hsub
      exact Or.inl ⟨hm, inj₁_subset_inj₁.mp hsub⟩
    · rintro (⟨_, hsub⟩ | ⟨e1, _⟩)
      · refine ⟨P₁.mem_X _, sumEnum_mem _, ?_⟩
        rw [sumEnum_one hm]; exact inj₁_subset_inj₁.mpr hsub
```

```
      · exfalso; omega
  · constructor
    · intro _; exact Or.inr ⟨hm0, hm1⟩
    · intro _
      refine ⟨P₁.mem_X _, sumEnum_mem _, ?_⟩
      rw [sumEnum_master hm0 hm1]; exact inj₁_subset_sumMaster (P₁.mem_X _)

theorem outMap₀_isComputable (P₀ : ComputablePresentation V₀) (P₁ :
    ComputablePresentation V₁) :
    ComputableMap (sumPresentation P₀ P₁) P₀ (outMap₀ (h₀ := h₀) (h₁ := h₁)) := by
  constructor
  obtain ⟨k0, hk0⟩ := P₀.surj V₀.master_mem
  have hincl : RecDecidable (fun s => P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) := P₀.
    incl_computable
  have hntag0 : RecDecidable (fun s => s.unpair.1.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have hincl_left : RecDecidable (fun s => P₀.X s.unpair.1.unpair.2 ⊆ P₀.X s.unpair.2)
    :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right))
  have hincl_master : RecDecidable (fun s => P₀.X k0 ⊆ P₀.X s.unpair.2) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp ((Nat.Primrec.const k0).pair Nat.Primrec.right))
  refine (RecDecidable.of_iff (fun s => ?_)
    ((hntag0.and hincl_left).or (hntag0.not.and hincl_master))).re
  simp only [sumPresentation_X]
  rcases tag_trichotomy s.unpair.1.unpair.1 with hn | hn | ⟨hn0, hn1⟩
  · constructor
    · rintro ⟨_, _, hsub⟩
      rw [sumEnum_zero hn, leftPart_inj₀] at hsub
      exact Or.inl ⟨hn, hsub⟩
    · rintro (⟨_, hsub⟩ | ⟨e1, _⟩)
      · refine ⟨sumEnum_mem _, P₀.mem_X _, ?_⟩
        rw [sumEnum_zero hn, leftPart_inj₀]; exact hsub
      · exfalso; omega
  · constructor
    · rintro ⟨_, _, hsub⟩
      rw [sumEnum_one hn, leftPart_inj₁ V₀ (h₁ _ (P₁.mem_X _))] at hsub
      refine Or.inr ⟨by omega, ?_⟩
      rw [hk0]; exact hsub
    · rintro (⟨e1, _⟩ | ⟨_, hsub⟩)
      · exfalso; omega
      · refine ⟨sumEnum_mem _, P₀.mem_X _, ?_⟩
        rw [sumEnum_one hn, leftPart_inj₁ V₀ (h₁ _ (P₁.mem_X _)), ← hk0]; exact hsub
  · constructor
    · rintro ⟨_, _, hsub⟩
      rw [sumEnum_master hn0 hn1, leftPart_sumMaster] at hsub
      refine Or.inr ⟨hn0, ?_⟩
      rw [hk0]; exact hsub
    · rintro (⟨e1, _⟩ | ⟨_, hsub⟩)
      · exfalso; omega
      · refine ⟨sumEnum_mem _, P₀.mem_X _, ?_⟩
        rw [sumEnum_master hn0 hn1, leftPart_sumMaster, ← hk0]; exact hsub

theorem outMap₁_isComputable (P₀ : ComputablePresentation V₀) (P₁ :
    ComputablePresentation V₁) :
```

```
    ComputableMap (sumPresentation P₀ P₁) P₁ (outMap₁ (h₀ := h₀) (h₁ := h₁)) := by
  constructor
  obtain ⟨k1, hk1⟩ := P₁.surj V₁.master_mem
  have hincl : RecDecidable (fun s => P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) := P₁.
    incl_computable
  have hntag1 : RecDecidable (fun s => s.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 1)
  have hincl_right : RecDecidable (fun s => P₁.X s.unpair.1.unpair.2 ⊆ P₁.X s.unpair.2)
     :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right))
  have hincl_master : RecDecidable (fun s => P₁.X k1 ⊆ P₁.X s.unpair.2) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp ((Nat.Primrec.const k1).pair Nat.Primrec.right))
  refine (RecDecidable.of_iff (fun s => ?_)
    ((hntag1.and hincl_right).or (hntag1.not.and hincl_master))).re
  simp only [sumPresentation_X]
  rcases tag_trichotomy s.unpair.1.unpair.1 with hn | hn | ⟨hn0, hn1⟩
  · constructor
    · rintro ⟨_, _, hsub⟩
      rw [sumEnum_zero hn, rightPart_inj₀ V₁ (h₀ _ (P₀.mem_X _))] at hsub
      refine Or.inr ⟨by omega, ?_⟩
      rw [hk1]; exact hsub
    · rintro (⟨e1, _⟩ | ⟨_, hsub⟩)
      · exfalso; omega
      · refine ⟨sumEnum_mem _, P₁.mem_X _, ?_⟩
        rw [sumEnum_zero hn, rightPart_inj₀ V₁ (h₀ _ (P₀.mem_X _)), ← hk1]; exact hsub
  · constructor
    · rintro ⟨_, _, hsub⟩
      rw [sumEnum_one hn, rightPart_inj₁] at hsub
      exact Or.inl ⟨hn, hsub⟩
    · rintro (⟨_, hsub⟩ | ⟨e1, _⟩)
      · refine ⟨sumEnum_mem _, P₁.mem_X _, ?_⟩
        rw [sumEnum_one hn, rightPart_inj₁]; exact hsub
      · exfalso; omega
  · constructor
    · rintro ⟨_, _, hsub⟩
      rw [sumEnum_master hn0 hn1, rightPart_sumMaster] at hsub
      refine Or.inr ⟨hn1, ?_⟩
      rw [hk1]; exact hsub
    · rintro (⟨e1, _⟩ | ⟨_, hsub⟩)
      · exfalso; omega
      · refine ⟨sumEnum_mem _, P₁.mem_X _, ?_⟩
        rw [sumEnum_master hn0 hn1, rightPart_sumMaster, ← hk1]; exact hsub

theorem sumMap_isComputable {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
    {h₀' : ∀ X, V₀'.mem X → X.Nonempty} {h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty}
    (Q₀ : ComputablePresentation V₀') (Q₁ : ComputablePresentation V₁')
    {f : ApproximableMap V₀ V₀'} {g : ApproximableMap V₁ V₁'}
    (hf : ComputableMap P₀ Q₀ f) (hg : ComputableMap P₁ Q₁ g) :
    ComputableMap (sumPresentation P₀ P₁ (h₀ := h₀) (h₁ := h₁))
      (sumPresentation Q₀ Q₁ (h₀ := h₀') (h₁ := h₁'))
      (sumMap f g) := by
  constructor
  have hf' : REPred (fun u => f.rel (P₀.X u.unpair.1) (Q₀.X u.unpair.2)) := hf.out
  have hg' : REPred (fun u => g.rel (P₁.X u.unpair.1) (Q₁.X u.unpair.2)) := hg.out
```

```
have hNtag0 : RecDecidable (fun s => s.unpair.1.unpair.1 = 0) :=
  RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 0)
have hNtag1 : RecDecidable (fun s => s.unpair.1.unpair.1 = 1) :=
  RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 1)
have hMtag0 : RecDecidable (fun s => s.unpair.2.unpair.1 = 0) :=
  RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
have hMtag1 : RecDecidable (fun s => s.unpair.2.unpair.1 = 1) :=
  RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
have hfr : REPred (fun s => f.rel (P₀.X s.unpair.1.unpair.2) (Q₀.X s.unpair.2.unpair
  .2)) :=
  REPred.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
    (hf'.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
      (Nat.Primrec.right.comp Nat.Primrec.right)))
have hgr : REPred (fun s => g.rel (P₁.X s.unpair.1.unpair.2) (Q₁.X s.unpair.2.unpair
  .2)) :=
  REPred.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
    (hg'.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
      (Nat.Primrec.right.comp Nat.Primrec.right)))
have hA : REPred (fun s => s.unpair.2.unpair.1 = 0 ∧ s.unpair.2.unpair.1 = 1) :=
  (hMtag0.not.and hMtag1.not).re
have hB : REPred (fun s => (s.unpair.1.unpair.1 = 0 ∧ s.unpair.2.unpair.1 = 0)
    ∧ f.rel (P₀.X s.unpair.1.unpair.2) (Q₀.X s.unpair.2.unpair.2)) :=
  ((hNtag0.and hMtag0).re).and hfr
have hC : REPred (fun s => (s.unpair.1.unpair.1 = 1 ∧ s.unpair.2.unpair.1 = 1)
    ∧ g.rel (P₁.X s.unpair.1.unpair.2) (Q₁.X s.unpair.2.unpair.2)) :=
  ((hNtag1.and hMtag1).re).and hgr
refine REPred.of_iff (fun s => ?_) (hA.or (hB.or hC))
simp only [sumPresentation_X]
constructor
· rintro ⟨_, _, hdisj⟩
  rcases hdisj with hM | ⟨X, Y', hWX, hWY, hfXY⟩ | ⟨Y, Y', hWY, hWY', hgYY⟩
  · left
    rcases tag_trichotomy s.unpair.2.unpair.1 with hm | hm | hm
    · rw [sumEnum_zero hm] at hM; exact absurd hM inj₀_ne_sumMaster
    · rw [sumEnum_one hm] at hM; exact absurd hM inj₁_ne_sumMaster
    · exact hm
  · right; left
    have ha : s.unpair.1.unpair.1 = 0 ∧ X = P₀.X s.unpair.1.unpair.2 := by
      rcases tag_trichotomy s.unpair.1.unpair.1 with hn | hn | ⟨h0, h1⟩
      · rw [sumEnum_zero hn] at hWX; exact ⟨hn, (inj₀_eq_iff.mp hWX).symm⟩
      · rw [sumEnum_one hn] at hWX
        exact absurd hWX.symm (inj₀_ne_inj₁_of_nonempty (h₁ _ (P₁.mem_X _)))
      · rw [sumEnum_master h0 h1] at hWX; exact absurd hWX.symm inj₀_ne_sumMaster
    have hb : s.unpair.2.unpair.1 = 0 ∧ Y' = Q₀.X s.unpair.2.unpair.2 := by
      rcases tag_trichotomy s.unpair.2.unpair.1 with hm | hm | ⟨h0, h1⟩
      · rw [sumEnum_zero hm] at hWY; exact ⟨hm, (inj₀_eq_iff.mp hWY).symm⟩
      · rw [sumEnum_one hm] at hWY
        exact absurd hWY.symm (inj₀_ne_inj₁_of_nonempty (h₁' _ (Q₁.mem_X _)))
      · rw [sumEnum_master h0 h1] at hWY; exact absurd hWY.symm inj₀_ne_sumMaster
    obtain ⟨ha0, hX⟩ := ha; obtain ⟨hb0, hY⟩ := hb
    subst hX; subst hY
    exact ⟨⟨ha0, hb0⟩, hfXY⟩
  · right; right
    have ha : s.unpair.1.unpair.1 = 1 ∧ Y = P₁.X s.unpair.1.unpair.2 := by
      rcases tag_trichotomy s.unpair.1.unpair.1 with hn | hn | ⟨h0, h1⟩
      · rw [sumEnum_zero hn] at hWY
```

```
          exact (inj₀_eq_inj₁_elim (h₀ _ (P₀.mem_X _)) hWY).elim
        · rw [sumEnum_one hn] at hWY; exact ⟨hn, (inj₁_eq_iff.mp hWY).symm⟩
        · rw [sumEnum_master h0 h1] at hWY; exact absurd hWY.symm inj₁_ne_sumMaster
      have hb : s.unpair.2.unpair.1 = 1 ∧ Y' = Q₁.X s.unpair.2.unpair.2 := by
        rcases tag_trichotomy s.unpair.2.unpair.1 with hm | hm | ⟨h0, h1⟩
        · rw [sumEnum_zero hm] at hWY'
          exact (inj₀_eq_inj₁_elim (h₀' _ (Q₀.mem_X _)) hWY').elim
        · rw [sumEnum_one hm] at hWY'; exact ⟨hm, (inj₁_eq_iff.mp hWY').symm⟩
        · rw [sumEnum_master h0 h1] at hWY'; exact absurd hWY'.symm inj₁_ne_sumMaster
      obtain ⟨ha1, hYeq⟩ := ha; obtain ⟨hb1, hY'eq⟩ := hb
      subst hYeq; subst hY'eq
      exact ⟨⟨ha1, hb1⟩, hgYY⟩
  · rintro (⟨hm0, hm1⟩ | ⟨⟨hn0, hm0⟩, hfXY⟩ | ⟨⟨hn1, hm1⟩, hgYY⟩)
    · exact ⟨sumEnum_mem _, sumEnum_mem _, Or.inl (sumEnum_master hm0 hm1)⟩
    · exact ⟨sumEnum_mem _, sumEnum_mem _,
        Or.inr (Or.inl ⟨_, _, sumEnum_zero hn0, sumEnum_zero hm0, hfXY⟩)⟩
    · exact ⟨sumEnum_mem _, sumEnum_mem _,
        Or.inr (Or.inr ⟨_, _, sumEnum_one hn1, sumEnum_one hm1, hgYY⟩)⟩

end Sum

end

end DomainTheory.NeighbourhoodSystems
```

4.8.10 Effective product

The product of effectively given neighbourhood systems is effectively given, and the product combinators are computable.

Definition (`prodEnum`). The product enumeration pairs an index from each factor and returns the corresponding product neighbourhood.

Theorem (`prodEnum_apply`). The neighbourhood at a paired product index is the product of the two neighbourhoods at its component indices.

Proof. Both product coordinates reduce to the corresponding facts in the two factor presentations. ■

Theorem (`prodNbhd_eq_iff`). Two product neighbourhoods are equal exactly when their left components and their right components are respectively equal.

Proof. Both product coordinates reduce to the corresponding facts in the two factor presentations. ■

Definition (`prodPresentation`). Pairing the two factor enumerations gives a computable presentation of the product neighbourhood system.

Theorem (`prodPresentation_X`). The product presentation enumerates the product of the neighbourhoods named by the two components of an index.

Proof. Both product coordinates reduce to the corresponding facts in the two factor presentations. ■

Theorem (`paired_isComputable`). Pairing two computable approximable maps with common source gives a computable map into the product.

Proof. The required relation is obtained from recursively enumerable or decidable index relations by pairing, composition, conjunction, and projection. ■

Theorem (`prod_isEffectivelyGiven`). The product of two effectively given neighbourhood systems is effectively given.

Proof. Both product coordinates reduce to the corresponding facts in the two factor presentations. ■

Theorem (`proj0_isComputable`). The first projection from an effectively presented product is computable.

Proof. The required relation is obtained from recursively enumerable or decidable index relations by pairing, composition, conjunction, and projection. ■

Theorem (`proj1_isComputable`). The second projection from an effectively presented product is computable.

Proof. The required relation is obtained from recursively enumerable or decidable index relations by pairing, composition, conjunction, and projection. ■

Theorem (`prodMap_isComputable`). The product of two computable approximable maps is computable.

Proof. The required relation is obtained from recursively enumerable or decidable index relations by pairing, composition, conjunction, and projection. ■

prodEffective.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_7
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_3
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.ProjectionMappings
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.inducedMaps
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodNbhd

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive IndefProduct ApproximableMap

section
variable {α β ? α' β' : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def prodEnum (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁) (t : ℕ)
    :
    Set (α ? β) :=
  prodNbhd (P₀.X t.unpair.1) (P₁.X t.unpair.2)

@[simp] theorem prodEnum_apply (P₀ : ComputablePresentation V₀) (P₁ :
    ComputablePresentation V₁)
    (t : ℕ) : prodEnum P₀ P₁ t = prodNbhd (P₀.X t.unpair.1) (P₁.X t.unpair.2) := rfl

theorem prodNbhd_eq_iff {X X' : Set α} {Y Y' : Set β} :
    prodNbhd X Y = prodNbhd X' Y' ↔ X = X' ∧ Y = Y' :=
  ⟨prodNbhd.injective, by rintro ⟨rfl, rfl⟩; rfl⟩

def prodPresentation (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)
    :
    ComputablePresentation (prod V₀ V₁) where
  X := prodEnum P₀ P₁
  mem_X t := prod_mem_prodNbhd (P₀.mem_X _) (P₁.mem_X _)
  surj := by
    rintro Z ⟨X, Y, hX, hY, rfl⟩
```

```
  obtain ⟨n, rfl⟩ := P₀.surj hX
  obtain ⟨m, rfl⟩ := P₁.surj hY
  exact ⟨Nat.pair n m, by simp only [prodEnum_apply, unpair_pair_fst, unpair_pair_snd
  ]⟩
interEq_computable := by
  have hIE0 : RecDecidable (fun s => P₀.X s.unpair.1 ∩ P₀.X s.unpair.2.unpair.1
      = P₀.X s.unpair.2.unpair.2) := P₀.interEq_computable
  have hIE1 : RecDecidable (fun s => P₁.X s.unpair.1 ∩ P₁.X s.unpair.2.unpair.1
      = P₁.X s.unpair.2.unpair.2) := P₁.interEq_computable
  have hg0 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1
      (Nat.pair t.unpair.2.unpair.1.unpair.1 t.unpair.2.unpair.2.unpair.1)) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair
      ((Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
        (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hg1 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2
      (Nat.pair t.unpair.2.unpair.1.unpair.2 t.unpair.2.unpair.2.unpair.2)) :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair
      ((Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
        (Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
  refine RecDecidable.of_iff (fun t => ?_) ((hIE0.comp hg0).and (hIE1.comp hg1))
  simp only [prodEnum_apply, unpair_pair_fst, unpair_pair_snd, prodNbhd.inter,
  prodNbhd_eq_iff]
cons_computable := by
  have hC0 : RecDecidable (fun s => ∃ k, P₀.X k ⊆ P₀.X s.unpair.1 ∩ P₀.X s.unpair.2)
  :=
    P₀.cons_computable
  have hC1 : RecDecidable (fun s => ∃ k, P₁.X k ⊆ P₁.X s.unpair.1 ∩ P₁.X s.unpair.2)
  :=
    P₁.cons_computable
  have hh0 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1 t.unpair.2.unpair.1)
   :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair (Nat.Primrec.left.comp Nat.Primrec
  .right)
  have hh1 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2 t.unpair.2.unpair.2)
   :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
  Primrec.right)
  refine RecDecidable.of_iff (fun t => ?_) ((hC0.comp hh0).and (hC1.comp hh1))
  simp only [prodEnum_apply, unpair_pair_fst, unpair_pair_snd, prodNbhd.inter,
    prodNbhd.subset_iff]
  constructor
  · rintro ⟨k, hk0, hk1⟩
    exact ⟨⟨k.unpair.1, hk0⟩, ⟨k.unpair.2, hk1⟩⟩
  · rintro ⟨⟨k0, hk0⟩, ⟨k1, hk1⟩⟩
    exact ⟨Nat.pair k0 k1, by simpa only [unpair_pair_fst] using hk0,
      by simpa only [unpair_pair_snd] using hk1⟩
inter n m := Nat.pair (P₀.inter n.unpair.1 m.unpair.1) (P₁.inter n.unpair.2 m.unpair
  .2)
inter_primrec := by
  have h0 : Nat.Primrec (fun t => P₀.inter t.unpair.1.unpair.1 t.unpair.2.unpair.1)
  :=
    (P₀.inter_primrec.comp ((Nat.Primrec.left.comp Nat.Primrec.left).pair
      (Nat.Primrec.left.comp Nat.Primrec.right))).of_eq fun t => by
        simp only [unpair_pair_fst, unpair_pair_snd]
  have h1 : Nat.Primrec (fun t => P₁.inter t.unpair.1.unpair.2 t.unpair.2.unpair.2)
  :=
```

```
      (P₁.inter_primrec.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
        (Nat.Primrec.right.comp Nat.Primrec.right))).of_eq fun t => by
          simp only [unpair_pair_fst, unpair_pair_snd]
    exact h0.pair h1
  inter_spec := by
    rintro n m ⟨k, hk⟩
    simp only [prodEnum_apply, unpair_pair_fst, unpair_pair_snd] at hk ⊢
    rw [prodNbhd.inter] at hk ⊢
    obtain ⟨h0, h1⟩ := prodNbhd.subset_iff.mp hk
    rw [P₀.inter_spec ⟨_, h0⟩, P₁.inter_spec ⟨_, h1⟩]
  masterIdx := Nat.pair P₀.masterIdx P₁.masterIdx
  masterIdx_spec := by
    simp only [prodEnum_apply, unpair_pair_fst, unpair_pair_snd, P₀.masterIdx_spec,
      P₁.masterIdx_spec, prod_master]

@[simp] theorem prodPresentation_X (P₀ : ComputablePresentation V₀) (P₁ :
    ComputablePresentation V₁)
    (t : ℕ) :
    (prodPresentation P₀ P₁).X t = prodNbhd (P₀.X t.unpair.1) (P₁.X t.unpair.2) := rfl

theorem paired_isComputable {V₂ : NeighborhoodSystem ?}
    {P₂ : ComputablePresentation V₂} {P₀ : ComputablePresentation V₀}
    {P₁ : ComputablePresentation V₁} {a : ApproximableMap V₂ V₀} {b : ApproximableMap V₂
     V₁}
    (ha : ComputableMap P₂ P₀ a) (hb : ComputableMap P₂ P₁ b) :
    ComputableMap P₂ (prodPresentation P₀ P₁) (paired a b) := by
  constructor
  have ha' : REPred (fun s => a.rel (P₂.X s.unpair.1) (P₀.X s.unpair.2)) := ha.out
  have hb' : REPred (fun s => b.rel (P₂.X s.unpair.1) (P₁.X s.unpair.2)) := hb.out
  have hra : Nat.Primrec (fun t => Nat.pair t.unpair.1 t.unpair.2.unpair.1) :=
    Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right)
  have hrb : Nat.Primrec (fun t => Nat.pair t.unpair.1 t.unpair.2.unpair.2) :=
    Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)
  refine REPred.of_iff (fun t => ?_) ((ha'.comp hra).and (hb'.comp hrb))
  simp only [prodPresentation_X, paired_rel, prodNbhd.inl_preimage, prodNbhd.
    inr_preimage,
    unpair_pair_fst, unpair_pair_snd]
  exact ⟨fun h => ⟨h.2.1, h.2.2⟩,
    fun h => ⟨prod_mem_prodNbhd (P₀.mem_X _) (P₁.mem_X _), h.1, h.2⟩⟩

theorem prod_isEffectivelyGiven (h₀ : V₀.IsEffectivelyGiven) (h₁ : V₁.
    IsEffectivelyGiven) :
    (prod V₀ V₁).IsEffectivelyGiven := by
  obtain ⟨P₀⟩ := h₀; obtain ⟨P₁⟩ := h₁
  exact ⟨prodPresentation P₀ P₁⟩

theorem proj₀_isComputable (P₀ : ComputablePresentation V₀) (P₁ :
    ComputablePresentation V₁) :
    ComputableMap (prodPresentation P₀ P₁) P₀ (proj₀ V₀ V₁) := by
  constructor
  have hincl : RecDecidable (fun s => P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) := P₀.
    incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1 t.unpair.2) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [prodPresentation_X, proj₀_rel, prodNbhd.inl_preimage, unpair_pair_fst,
```

```
    unpair_pair_snd]
  exact ⟨fun h => h.2.2,
    fun h => ⟨prod_mem_prodNbhd (P₀.mem_X _) (P₁.mem_X _), P₀.mem_X _, h⟩⟩

theorem proj₁_isComputable (P₀ : ComputablePresentation V₀) (P₁ :
    ComputablePresentation V₁) :
    ComputableMap (prodPresentation P₀ P₁) P₁ (proj₁ V₀ V₁) := by
  constructor
  have hincl : RecDecidable (fun s => P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) := P₁.
    incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2 t.unpair.2) :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [prodPresentation_X, proj₁_rel, prodNbhd.inr_preimage, unpair_pair_fst,
    unpair_pair_snd]
  exact ⟨fun h => h.2.2,
    fun h => ⟨prod_mem_prodNbhd (P₀.mem_X _) (P₁.mem_X _), P₁.mem_X _, h⟩⟩

theorem prodMap_isComputable {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β
    '}
    {P₀ : ComputablePresentation V₀} {P₁ : ComputablePresentation V₁}
    {P₀' : ComputablePresentation V₀'} {P₁' : ComputablePresentation V₁'}
    {f : ApproximableMap V₀ V₀'} {g : ApproximableMap V₁ V₁'}
    (hf : ComputableMap P₀ P₀' f) (hg : ComputableMap P₁ P₁' g) :
    ComputableMap (prodPresentation P₀ P₁) (prodPresentation P₀' P₁') (prodMap f g) :=
    by
  rw [prodMap]
  exact paired_isComputable (comp_isComputable (proj₀_isComputable P₀ P₁) hf)
    (comp_isComputable (proj₁_isComputable P₀ P₁) hg)

end

end DomainTheory.NeighbourhoodSystems
```

4.8.11 Theorem 7.4 — sum and product effective

THEOREM 7.4. If $\mathcal{D}_0$ and $\mathcal{D}_1$ are effectively given, then so are $(\mathcal{D}_0 + \mathcal{D}_1)$ and $(\mathcal{D}_0 \times \mathcal{D}_1)$.

Moreover the combinators in_i and out_i and proj_i^2 are all computable; further, if f and g are computable maps, then so are $f + g$ and $f \times g$.

Proof: Let the computable presentations be given as:

$$\mathcal{D}_i = \{X_n^i \mid n \in \mathbf{N}\}.$$

We can assume that the sets of tokens Δ_0 and Δ_1 are disjoint and $\emptyset \notin \mathcal{D}_i$. Then the construction of the sum is just

$$\mathcal{D}_0 + \mathcal{D}_1 = \{\Delta_0 \cup \Delta_1\} \cup \mathcal{D}_0 \cup \mathcal{D}_1.$$

As an enumeration we define for $n \in \mathbf{N}$:

$$Z_0 = \Delta_0 \cup \Delta_1 \; ; \; Z_{2n+1} = X_n^0 \; ; \; Z_{2n+2} = X_n^1 \; .$$

We leave as an exercise the check of 7.1(i)–(ii).

For the product we want:

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{X^0_n \cup X^1_m \mid n, m \in \mathbf{N}\}.$$

What we then need are recursive functions $p : \mathbf{N} \to \mathbf{N}$, $q : \mathbf{N} \to \mathbf{N}$, and $r : \mathbf{N} \times \mathbf{N} \to \mathbf{N}$ where for $m, n, k \in \mathbf{N}$ we have:

$$p(r(n, m)) = n \text{ and } q(r(n, m)) = m, \text{ and } r(p(k), q(k)) = k.$$

Thus r is a "one-one pairing function"; there are many ways to find such functions (see Exercise 5.13). We can then define for $k \in \mathbf{N}$:

$$W_k = X^0_{p(k)} \cup X^1_{q(k)} .$$

Again we leave as an exercise the check that this provides a computable presentation of $\mathcal{D}_0 \times \mathcal{D}_1$.

As for the combinators, the neighbourhood relations have to be worked out in terms of the indices. For example

$$X^0_n \text{ in}_0 \ Z_m \text{ iff either } m = 0 \text{ or for some } k$$

$$m = 2k + 1 \text{ and } X^0_n \subseteq X^0_k .$$

and

$$W_k \text{ proj}^2_1 \ X^1_m \text{ iff } X^1_{q(k)} \subseteq X^1_m .$$

The reader needs to check that these are recursively enumerable

Relations in the indices. For this purpose it may be convenient to recall some closure properties of these relations: taking conjunctions, disjunctions, substituting recursive functions, applying an existential quantifier to the front. □

Products give us a way of providing an immediate meaning to the notion of a computable function of several variables. Note that the proof of 3.7 is "effective" and shows that substitution of computable functions of several variables into each other always gives computable functions. We turn next to the function spaces.

The development is with the effective sum and effective product peers.

Theorem (`theorem_74`). If two neighbourhood systems are effectively given, then so are their sum and their product. The injections, the projections, and the sum and product of computable maps are computable.

Proof. By the effective sum and the effective product. ■

`Theorem_7_4.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .sumEffective
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .prodEffective

@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap IndefProduct

/-- Scott's Theorem 7.4: effective givenness and computability for sum and product. -/
theorem theorem_74 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty},
      V₀.IsEffectivelyGiven → V₁.IsEffectivelyGiven → (sum V₀ V₁ h₀ h₁).
    IsEffectivelyGiven) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁),
      ComputableMap P₀ (sumPresentation P₀ P₁ (h₀ := h₀) (h₁ := h₁))
        (inMap₀ (h₀ := h₀) (h₁ := h₁))) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁),
      ComputableMap P₁ (sumPresentation P₀ P₁ (h₀ := h₀) (h₁ := h₁))
        (inMap₁ (h₀ := h₀) (h₁ := h₁))) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁),
      ComputableMap (sumPresentation P₀ P₁ (h₀ := h₀) (h₁ := h₁)) P₀
        (outMap₀ (h₀ := h₀) (h₁ := h₁))) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁),
      ComputableMap (sumPresentation P₀ P₁ (h₀ := h₀) (h₁ := h₁)) P₁
        (outMap₁ (h₀ := h₀) (h₁ := h₁))) ∧
    (∀ {α β α' β' : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
      {P₀ : ComputablePresentation V₀} {P₁ : ComputablePresentation V₁}
      {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
      {h₀' : ∀ X, V₀'.mem X → X.Nonempty} {h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty}
      (Q₀ : ComputablePresentation V₀') (Q₁ : ComputablePresentation V₁')
      {f : ApproximableMap V₀ V₀'} {g : ApproximableMap V₁ V₁'},
      ComputableMap P₀ Q₀ f → ComputableMap P₁ Q₁ g →
        ComputableMap (sumPresentation P₀ P₁ (h₀ := h₀) (h₁ := h₁))
          (sumPresentation Q₀ Q₁ (h₀ := h₀') (h₁ := h₁'))
          (sumMap (h₀ := h₀) (h₁ := h₁) (h₀' := h₀') (h₁' := h₁') f g)) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?} {P₂ : ComputablePresentation V₂}
      {P₀ : ComputablePresentation V₀} {P₁ : ComputablePresentation V₁}
      {a : ApproximableMap V₂ V₀} {b : ApproximableMap V₂ V₁},
      ComputableMap P₂ P₀ a → ComputableMap P₂ P₁ b →
        ComputableMap P₂ (prodPresentation P₀ P₁) (paired a b)) ∧
    (∀ {α β α' β' : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
      {P₀ : ComputablePresentation V₀} {P₁ : ComputablePresentation V₁}
      {P₀' : ComputablePresentation V₀'} {P₁' : ComputablePresentation V₁'}
      {f : ApproximableMap V₀ V₀'} {g : ApproximableMap V₁ V₁'},
      ComputableMap P₀ P₀' f → ComputableMap P₁ P₁' g →
        ComputableMap (prodPresentation P₀ P₁) (prodPresentation P₀' P₁') (prodMap f g)
    ) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β},
```

```
      V₀.IsEffectivelyGiven → V₁.IsEffectivelyGiven → (prod V₀ V₁).IsEffectivelyGiven)
    ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁),
      ComputableMap (prodPresentation P₀ P₁) P₀ (proj₀ V₀ V₁)) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁),
      ComputableMap (prodPresentation P₀ P₁) P₁ (proj₁ V₀ V₁)) :=
  ⟨@sum_isEffectivelyGiven, ⟨@inMap₀_isComputable, ⟨@inMap₁_isComputable,
    ⟨@outMap₀_isComputable, ⟨@outMap₁_isComputable, ⟨@sumMap_isComputable,
      ⟨@paired_isComputable, ⟨@prodMap_isComputable, ⟨@prod_isEffectivelyGiven,
        ⟨@proj₀_isComputable, @proj₁_isComputable⟩⟩⟩⟩⟩⟩⟩⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.8.12 Function-space presentation

The function space of effectively given systems is effectively given, and evaluation and currying are computable.

Definition (`interList`). Starting from a base set, this operation intersects all sets in a finite list.

Theorem (`interList_nil`). Intersecting no listed sets leaves the base set.

Proof. The empty list contributes no intersections, so the recursive definition returns the base set. ■

Definition (`funPair`). A paired index denotes the pair consisting of the corresponding source and target neighbourhoods.

Theorem (`funPair_fst`). The first component of a paired index is the source neighbourhood named by its first numerical component.

Proof. The first projection of the interpreted pair is the source neighbourhood indexed by the first unpaired component. ■

Theorem (`funPair_snd`). The second component of a paired index is the target neighbourhood named by its second numerical component.

Proof. The second projection of the interpreted pair is the target neighbourhood indexed by the second unpaired component. ■

Definition (`bitSelect`). A natural number is read as a bit mask selecting a sublist from a given finite list.

Theorem (`bitSelect_nil`). Every mask selects the empty sublist from the empty list.

Proof. There are no entries to inspect, so every mask selects the empty list. ■

Theorem (`andFoldl_eq_one_iff`). The characteristic value is one exactly when every finite condition encoded in its arguments holds.

Proof. Induction on the list uses the binary bound on the accumulator; each update is one exactly when the old flag and the current test are both one. ■

Theorem (`andFoldl_one`). Starting with success, the finite conjunction fold returns one exactly when every listed value is one.

Proof. Specializing the preceding fold characterization to initial flag one removes the redundant initial conjunct. ■

Definition (`appendStep`). This operation adjoins one natural number to the front of an accumulated list code.

Definition (`appendStp`). This packed fold step adjoins the current entry to an accumulated list code.

Definition (`appendCode`). This operation combines two list codes by folding the entries of one into the other.

Theorem (`appendStp_eq`). The coding operation agrees with concatenation or adjoining an entry to a finite list.

Proof. Unpairing the packed entry and accumulator recovers both arguments of the adjoining operation. ■

Definition (`botMap`). The least approximable map relates every source neighbourhood only as forced by the master target neighbourhood.

Theorem (`botMap_rel`). The least approximable map relates a source neighbourhood to a target neighbourhood exactly when both are valid and the master target neighbourhood is contained in the latter.

Proof. The least map generated by an empty constraint list contains exactly the mandatory master-target relation, together with validity of both neighbourhoods. ■

Definition (`condSet`). This set consists of the points in a base target neighbourhood satisfying every consequent whose antecedent contains a fixed source neighbourhood.

Theorem (`condSet_nil`). With no constraints, the conditional set is the initial target set.

Proof. Set extensionality identifies both sides pointwise: each asks for membership in the base and in every consequent whose antecedent contains the fixed neighbourhood. ■

Theorem (`condSet_subset`). Every conditional intersection is contained in its initial target neighbourhood.

Proof. Membership in the conditional set includes membership in the initial target set as its first conjunct, which gives the inclusion. ■

Definition (`consPairChar`). This zero-one characteristic tests whether two enumerated function-space neighbourhoods have a consistent intersection.

Definition (`consUpd`). This state update scans a selected sublist, intersects the chosen neighbourhood indices, and records whether each required intersection remains consistent.

Definition (`consStp`). This packed fold step applies the consistency update to one coded entry.

Definition (`consCharAt`). This characteristic tests whether the neighbourhoods selected by a bit mask have a consistent finite intersection.

Theorem (`decodeList_appendStep`). Decoding an adjoining step gives the new entry followed by the decoded accumulator.

Proof. The successor decoding equation unpairs the predecessor into the newly adjoined head and the former accumulator code. ■

Theorem (`decodeList_encodeList`). Decoding the code of a finite list recovers the list.

Proof. Induction on the list or code uses the successor coding equation and the fact that unpairing a freshly paired head and tail recovers both components. ■

Theorem (`decodeList_foldl_appendStp`). Decoding an adjoining fold gives the reverse of the folded entries followed by the decoded accumulator.

Proof. Induction on the folded list uses the adjoining equation at each head; because entries are prepended, the processed prefix appears in reverse order. ■

Theorem (`decodeList_appendCode`). Decoding the combined code gives the reversed second decoded list followed by the first.

Proof. The coded-fold equation and the preceding decoding formula identify the result as the reversed second list followed by the first. ■

Definition (`funImpChar`). This characteristic represents the implication from consistency of selected antecedents to consistency of selected consequents.

Definition (`funConsChar`). This bounded universal characteristic tests consistency of a finite family of step constraints over all selected subfamilies.

Definition (`funListOf`). A list of paired indices is interpreted as a list of source-target neighbourhood pairs.

Definition (`Xenum`). A consistent code denotes its finite step-function neighbourhood; an inconsistent code denotes the master function-space neighbourhood.

Theorem (`Xenum_neg`). A code rejected by the consistency characteristic denotes the whole function space.

Proof. A failed consistency test takes the second branch of the enumeration, which is the whole function space. ■

Theorem (`Xenum_pos`). A code accepted by the consistency characteristic denotes the finite step-function neighbourhood determined by its decoded constraints.

Proof. A successful consistency test takes the first branch, the step neighbourhood generated by the decoded constraints. ■

Theorem (`funListOf_cons`). Interpreting a nonempty index list gives the interpreted head pair followed by the interpreted tail.

Proof. Mapping interpreted paired indices over a cons list preserves its head and maps the tail. ■

Theorem (`exists_funListOf`). Every finite list of valid source-target neighbourhood pairs is represented by a list of paired presentation indices.

Proof. Induction on the finite pair list chooses source and target indices by surjectivity of the two presentations, pairs those indices, and applies the induction hypothesis to the tail. ■

Theorem (`funListOf_valid`). Every pair represented by a list of paired indices consists of valid source and target neighbourhoods.

Proof. Every list entry comes from a paired index, and each coordinate enumeration always names a valid neighbourhood. ■

Theorem (`Xenum_mem`). Every set produced by the function-space enumeration is a function-space neighbourhood.

Proof. In the successful case, consistency supplies nonemptiness of a valid finite step neighbourhood; in the failed case, the whole function space is its master neighbourhood. ■

Theorem (`Xenum_nonempty`). Every neighbourhood in the function-space enumeration is nonempty.

Proof. Every enumerated set is a neighbourhood, and nonemptiness is part of the neighbourhood axiom in the function-space system. ■

Definition (`idxchain`). This index is obtained by successively intersecting a finite list of neighbourhood indices, beginning with the master index.

Definition (`interFrom`). This operation successively intersects a starting set with the neighbourhoods named by a list of indices.

Definition (`interIdx`). This index denotes the intersection of two enumerated function-space neighbourhoods whenever both are proper, and otherwise selects the proper one.

Definition (`interYsStp`). This fold step intersects a target accumulator with a consequent exactly when the fixed antecedent is contained in the current antecedent.

Definition (`interYsIdx`). This operation computes an index for the intersection of all consequents applicable to a fixed antecedent.

Theorem (`mem_condSet`). A point belongs to the conditional set exactly when it lies in the base and in every consequent whose antecedent contains the fixed source neighbourhood.

Proof. This is the membership condition defining the conditional set: membership in the base together with every applicable consequent. ■

Theorem (`condSet_cons_neg`). A constraint whose antecedent does not contain the fixed source neighbourhood contributes no condition to the conditional intersection.

Proof. The head implication has a false antecedent, hence imposes no condition; the remaining universal conditions are exactly those from the tail. ■

Theorem (`condSet_cons_pos`). A constraint whose antecedent contains the fixed source neighbourhood contributes intersection with its consequent.

Proof. When the fixed antecedent is contained in the head antecedent, the head condition requires membership in its consequent, producing one additional intersection before processing the tail. ■

Theorem (`condFoldl_spec`). Folding the computable intersection indices yields an index for the conditional intersection determined by the finite constraint list.

Proof. Induction on the constraint list skips a head when inclusion fails and otherwise computes the intersection with its consequent; neighbourhood consistency licenses the presentation's intersection index. ■

Theorem (`condSet_eq_interYs`). Starting from the master target neighbourhood, the conditional intersection equals the intersection of all consequents applicable to the fixed antecedent.

Proof. Set extensionality identifies both sides pointwise: each asks for membership in the base and in every consequent whose antecedent contains the fixed neighbourhood. ■

Theorem (`primrec_appendCode`). The numerical function defining appendCode is primitive recursive.

Proof. The append code function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`primrec_consPairChar`). The numerical function defining consPairChar is primitive recursive.

Proof. The cons pair characteristic function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`primrec_interIdx`). The numerical function defining interIdx is primitive recursive.

Proof. The inter idx function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`primrec_interYsStp`). The numerical function defining interYsStp is primitive recursive.

Proof. The inter ys step function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`primrec_interYsIdx`). The numerical function defining interYsIdx is primitive recursive.

Proof. The inter ys idx function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`rel_interYs_funList`). Every map satisfying a finite list of step constraints relates each source neighbourhood to the intersection of all applicable consequents.

Proof. Induction on the constraint list ignores heads whose antecedent does not contain the chosen source and otherwise combines the head relation with the induction hypothesis by target intersection. ■

Theorem (`interYs_funList_mem_of_nonempty`). If a finite step-function neighbourhood is nonempty, then its intersection of applicable consequents is a target neighbourhood.

Proof. The inclusion characteristic selects exactly the applicable consequents, and induction over the decoded list shows that the computed presentation intersections denote their set-theoretic intersection. ■

Theorem (`selectFn_isOne`). Selection controlled by the characteristic of one chooses the first value exactly when the tested value is one.

Proof. Substitution of the control value into the arithmetic definition cancels one summand and leaves the selected argument. ■

Theorem (`interYsIdx_spec`). The computed target index denotes exactly the intersection of all consequents applicable to the chosen antecedent.

Proof. The inclusion characteristic selects exactly the applicable consequents, and induction over the decoded list shows that the computed presentation intersections denote their set-theoretic intersection. ■

Theorem (`selectFn_mul_iff`). Selection with default zero has value one exactly when both its control and selected value are one.

Proof. The selector with zero default is the product of its two inputs, and a product of natural numbers is one precisely when both factors are one. ■

Theorem (`stepFun_funListOf_appendCode`). The finite step neighbourhood represented by the combined code is the intersection of the neighbourhoods represented by the two original codes.

Proof. Interpretation is list mapping by paired indices, and mapping distributes over concatenation. ■

Theorem (`Xenum_inter_eq`). Whenever the intersection is nonempty, the computed intersection index denotes exactly the intersection of the two enumerated function-space neighbourhoods.

Proof. Split on the two consistency bits. If both codes are proper, concatenating their constraint lists represents intersection; if either is improper, its neighbourhood is the whole space and intersection returns the other. ■

Theorem (`consPairChar_spec`). The pair-consistency characteristic is one exactly when some enumerated function-space neighbourhood is contained in the intersection.

Proof. Existence of such a refinement is equivalent to nonemptiness of the intersection. When nonempty, the computed intersection index is a witness; conversely every enumerated refinement is itself nonempty. ■

Theorem (`stepFun_funListOf_eq_univ_iff`). A finite step-function neighbourhood is the whole function space exactly when every listed consequent is the master neighbourhood.

Proof. Testing the alleged equality on the least approximable map forces each consequent to contain the master and hence equal it; conversely, master consequents are satisfied by every map through its mandatory master relation. ■

Theorem (`stepFun_funListOf_subset_iff`). Inclusion of the two finite step-function neighbourhoods is equivalent to containment of every required consequent by the intersection generated from the first family.

Proof. The least map generated by the first constraints supplies necessity; conversely, each map satisfying those constraints relates an antecedent to their applicable-consequent intersection, and monotonicity yields every constraint in the second family. ■

Definition (`subStp`). This fold step preserves a success flag precisely when the computed applicable-consequent intersection is contained in the current consequent.

Theorem (`primrec_subStp`). The numerical function defining subStp is primitive recursive.

Proof. The sub step function is built from primitive-recursive projections, constants, pairing and composition. ■

Definition (`subChar`). This characteristic tests inclusion between two finite step-function neighbourhoods.

Theorem (`primrec_subChar`). The numerical function defining subChar is primitive recursive.

Proof. The sub characteristic function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`subChar_spec`). The inclusion characteristic has value one exactly when the first finite step-function neighbourhood is contained in the second.

Proof. The least map generated by the first constraints supplies necessity; conversely, each map satisfying those constraints relates an antecedent to their applicable-consequent intersection, and monotonicity yields every constraint in the second family. ■

Definition (`trivStp`). This fold step preserves a success flag precisely when the current consequent is the master neighbourhood.

Theorem (`primrec_trivStp`). The numerical function defining trivStp is primitive recursive.

Proof. The triv step function is built from primitive-recursive projections, constants, pairing and composition. ■

Definition (`trivialChar`). This characteristic tests whether a finite step-function neighbourhood is the entire function space.

Definition (`eqEnumChar`). This characteristic tests equality of two enumerated function-space neighbourhoods, treating inconsistent codes as the master neighbourhood.

Definition (`interEqChar`). This characteristic tests whether one enumerated function-space neighbourhood is the intersection of two others.

Theorem (`primrec_trivialChar`). The numerical function defining trivialChar is primitive recursive.

Proof. The trivial characteristic function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`primrec_eqEnumChar`). The numerical function defining eqEnumChar is primitive recursive.

Proof. The equality enum characteristic function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`primrec_interEqChar`). The numerical function defining interEqChar is primitive recursive.

Proof. The interequality characteristic function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`trivialChar_spec`). The triviality characteristic has value one exactly when the finite step-function neighbourhood is the whole function space.

Proof. The finite conjunction accepts precisely when every consequent in the decoded constraints equals the master neighbourhood, which is exactly the criterion for the step neighbourhood to be the whole function space. ■

Theorem (`eqEnumChar_spec`). The equality characteristic has value one exactly when the two enumerated function-space neighbourhoods are equal.

Proof. Separate the four consistency cases. Two proper neighbourhoods are equal exactly by mutual inclusion; a proper and an improper code are equal exactly when the proper one is the whole space; two improper codes both denote the whole space. ■

Theorem (`interEqChar_spec`). The intersection-equality characteristic has value one exactly when the first two enumerated neighbourhoods intersect to the third.

Proof. The consistency test first guarantees that the computed intersection index denotes the actual intersection, and the equality characteristic then compares that neighbourhood with the third one. ■

Definition (`funPresentation`). The enumeration by consistent finite step constraints, with inconsistent codes assigned to the master neighbourhood, gives a computable presentation of the function space.

Theorem (`interList_cons`). Intersecting a nonempty list is intersection with its head followed by intersection of its tail.

Proof. The recursive definition intersects the head set with the intersection accumulated from the tail. ■

Theorem (`mem_interList`). A point belongs to a finite list intersection exactly when it lies in the base and in every listed set.

Proof. Induction on the list expands the head intersection, converting point membership into membership in the base and in every listed set. ■

Theorem (`rel_interList_of_selection`). A map satisfying a finite selection of constraints relates a common stronger antecedent to the intersection of their consequents.

Proof. Induction on the selected constraints first restricts each relation to the common stronger antecedent, then combines the head consequent with the tail intersection using closure under target intersection. ■

Theorem (`interList_mem_of_stepFun_nonempty`). A compatible finite selection of consequents has a neighbourhood intersection whenever the ambient step-function neighbourhood is nonempty.

Proof. Choose a map from the nonempty step neighbourhood; it satisfies every selected constraint, and the preceding finite-intersection relation makes the intersection a target neighbourhood. ■

Theorem (`stepFun_funListOf_nonempty_iff`). The finite step-function neighbourhood is nonempty exactly when every consistent selected family of antecedents has a consistent family of consequents.

Proof. Necessity follows by choosing a map satisfying the finite constraints and intersecting the selected consequents; sufficiency constructs the least map after filtering exactly the antecedents containing each source neighbourhood. ■

Theorem (`mem_interFrom`). A point belongs to the indexed iterated intersection exactly when it lies in the initial set and every enumerated factor.

Proof. Induction on the index list expands each successive intersection, giving membership in the base and in every enumerated factor. ■

Theorem (`interFrom_subset`). An iterated intersection is contained in its initial set.

Proof. The membership characterization includes membership in the initial set as its first conjunct. ■

Theorem (`interFrom_mem_of_witness`). An iterated intersection is a neighbourhood whenever it contains a neighbourhood witness and its initial set is a neighbourhood.

Proof. Induction on the index list applies neighbourhood closure under a finite intersection, using the common neighbourhood witness to certify consistency at each new factor. ■

Theorem (`interFrom_eq_of_foldl`). The coded finite fold agrees with repeated application of its step function to the decoded list.

Proof. Induction on the indices computes the head intersection using the presentation's intersection specification and continues from that neighbourhood on the tail. ■

Theorem (`idxchain_spec`). When the iterated intersection is a neighbourhood, the successively computed index denotes exactly that intersection.

Proof. Induction on the index list uses the presentation's computed-intersection specification at the head and continues with the resulting neighbourhood index on the tail. ■

Theorem (`bitSelect_cons`). The low mask bit decides whether the head is retained, and the halved mask selects from the tail.

Proof. The low bit decides whether to retain the head, and division by two supplies the mask for the tail. ■

Theorem (`bitSelect_sublist`). Selection by a bit mask always produces a sublist of the original list.

Proof. Induction on the source list either keeps or skips its head according to the low bit; both choices preserve the sublist relation for the recursively selected tail. ■

Theorem (`exists_bitSelect_lt`). Every sublist is selected by some bit mask smaller than the corresponding power of two.

Proof. Induction on the sublist derivation assigns an even mask when the ambient head is skipped and an odd mask when it is retained; the power-of-two bound doubles at each new position. ■

Theorem (`forall_sublist_iff_forall_bitmask`). A property holds for every sublist exactly when it holds for every sublist selected by a bounded bit mask.

Proof. Every mask selects a sublist, giving one direction; for the converse, encode an arbitrary sublist by its bounded even-and-odd mask and apply the assumed masked case. ■

Theorem (`consUpd_eval`). On an explicitly packed state, the consistency update either processes the selected entry or skips it according to the low mask bit.

Proof. Unpairing the state exposes the mask, current index and flag; the low bit then chooses between processing the entry and carrying the state forward. ■

Theorem (`consUpd_foldl_flag_zero`). The coded finite fold agrees with repeated application of its step function to the decoded list.

Proof. Induction on the remaining list shows that a zero success flag is absorbing: whether the next entry is selected or skipped, the recursive state still carries flag zero. ■

Theorem (`consUpd_foldl_spec`). The coded finite fold agrees with repeated application of its step function to the decoded list.

Proof. Induction on the scanned list follows the low mask bit. Skipping preserves the state; selecting intersects the current neighbourhood and updates the flag with the consistency characteristic. ■

Theorem (`consCharAt_spec`). The consistency characteristic has value one exactly when the bit-selected family has a neighbourhood intersection.

Proof. The coded-fold equation reduces the characteristic to the consistency-update fold, whose specification identifies acceptance with the selected finite intersection being a neighbourhood. ■

Theorem (`primrec_consStp`). The numerical function defining consStp is primitive recursive.

Proof. The cons step function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`primrec_consCharAt`). The numerical function defining consCharAt is primitive recursive.

Proof. The cons characteristic at function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`consFold_decidable`). Neighbourhood status of every bit-selected finite intersection is recursively decidable.

Proof. Decide consistency for each bit-selected antecedent and consequent intersection, then apply bounded universal quantification over all masks below the power-of-two bound. ■

Theorem (`interList_X_eq_interFrom`). Intersecting a list of enumerated neighbourhoods from the master set agrees with iterated intersection by their indices.

Proof. Pointwise membership on both sides is membership in the master set and in every neighbourhood named by the index list. ■

Theorem (`antecedent_cons_iff`). A finite family of antecedents has a common neighbourhood refinement exactly when its iterated intersection is a neighbourhood.

Proof. A common neighbourhood witness is contained in the iterated intersection; conversely, when that intersection is a neighbourhood, its computed presentation index supplies a common refinement of every listed antecedent. ■

Theorem (`funConsequent_eq`). The intersection of selected consequent neighbourhoods equals their iterated intersection from the master neighbourhood.

Proof. The general equality between list intersection and indexed iteration applies after mapping each selected pair to its target index. ■

Theorem (`funCons_iff`). A finite step-function neighbourhood is nonempty exactly when consistency of every selected antecedent family implies consistency of its consequent family.

Proof. The finite-consistency criterion is first expressed over every sublist, then bounded masks are used to enumerate exactly those sublists; antecedent and consequent intersections are converted to indexed iterations. ■

Theorem (`funCons_decidable`). Nonemptiness of the finite step-function neighbourhood represented by a code is recursively decidable.

Proof. Decide consistency for each bit-selected antecedent and consequent intersection, then apply bounded universal quantification over all masks below the power-of-two bound. ■

Theorem (`selectFn_one_iff_imp`). For a zero-one antecedent, selection with default one returns one exactly when value one of the antecedent forces value one of the consequent.

Proof. Since the control is at most one, it is either zero or one; the two selector equations turn these cases into the truth table for implication. ■

Theorem (`funConsChar_spec`). The function-space consistency characteristic has value one exactly when the represented finite step-function neighbourhood is nonempty.

Proof. The coded fold specification turns each selected mask into a finite neighbourhood intersection, and bounded universal quantification ranges over exactly all sublists. ■

Theorem (`primrec_funImpChar`). The numerical function defining funImpChar is primitive recursive.

Proof. The fun imp characteristic function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`primrec_funConsChar`). The numerical function defining funConsChar is primitive recursive.

Proof. The fun cons characteristic function is built from primitive-recursive projections, constants, pairing and composition. ■

Theorem (`funSpace_isEffectivelyGiven`). The funSpace neighbourhood system is effectively given.

Proof. Choose computable presentations for source and target, build the finite-step enumeration with its primitive-recursive consistency characteristic, and use it as the required function-space presentation. ■

Theorem (`Xenum_singleton`). The code for a singleton source-target constraint denotes the corresponding one-step function-space neighbourhood.

Proof. The successor list code decodes to the single paired index; consistency of a valid one-step constraint selects its step neighbourhood rather than the master case. ■

Theorem (`consChain_iff`). The computed intersection index lies below every index in the chain exactly when the corresponding iterated intersection is a neighbourhood.

Proof. If the computed neighbourhood lies below every factor, it witnesses that their intersection is a

neighbourhood; conversely, the intersection specification identifies the computed neighbourhood with that intersection. ■

Theorem (`evalMap_rel_prodNbhd_iff`). Evaluation relates a product neighbourhood to a target neighbourhood exactly when every function in the function component sends the argument neighbourhood into that target neighbourhood.

Proof. The definition of evaluation reduces the product coordinates to the function neighbourhood and argument neighbourhood; its relation is precisely pointwise membership in the target step. ■

Theorem (`evalMap_isComputable`). Evaluation from the product of a presented function space and its source is computable.

Proof. Evaluation on a product neighbourhood is equivalent to inclusion of its function component in a one-step neighbourhood; the function-space presentation makes this inclusion decidable. ■

Theorem (`funListOf_append`). Interpreting a concatenation of index lists gives the concatenation of their interpreted neighbourhood-pair lists.

Proof. Interpretation is list mapping by paired indices, and mapping distributes over concatenation. ■

Theorem (`interList_subset_base`). The iterated intersection is contained in its initial base set.

Proof. Induction on the list expands the head intersection, converting point membership into membership in the base and in every listed set. ■

Theorem (`mem_Xenum_iff`). A function-space element contains an enumerated neighbourhood exactly when it satisfies every decoded step constraint whenever the code is consistent.

Proof. A successful code reduces membership to all decoded step constraints; a failed code denotes the master neighbourhood, which every function-space element contains. ■

Theorem (`Xenum_isComputableElement_iff`). Computability of the function-space element is equivalent to computability of its associated approximable map.

Proof. Singleton constraint codes recover the relation of the associated approximable map in one direction; in the other, finite universal closure combines that recursively enumerable relation over every decoded list. ■

Theorem (`isComputableElement_funPresentation_iff`). Computability of the function-space element is equivalent to computability of its associated approximable map.

Proof. This is the preceding equivalence after unpacking the definitions of computable element and computable map. ■

Theorem (`mem_Xenum_iff_map`). An approximable map belongs to an enumerated function neighbourhood exactly when every decoded relation holds whenever the code is consistent.

Proof. A successful code is membership in the finite step neighbourhood, hence satisfaction of every decoded relation; a failed code is the whole space. ■

Theorem (`curry_rel_Xenum_iff`). A curried map relates an outer antecedent to an enumerated function neighbourhood exactly when the original map satisfies every decoded product constraint.

Proof. The section of the original map at the outer antecedent satisfies exactly the decoded inner step constraints; recursive enumerability follows by finite universal quantification over the decoded list. ■

Theorem (`curry_isComputable`). Currying a computable map on the product presentation yields a computable map into the presented function space.

Proof. Rewrite the curried relation as an implication from consistency to all decoded product constraints; recursive enumerability follows from finite universal closure and disjunction with failure of the primitive-recursive consistency test. ■

Definition (`curryStepCode`). This code represents the singleton step constraint obtained by adjoining a fixed outer antecedent to an inner antecedent-consequent pair.

Theorem (`curryIso_apply`). The currying isomorphism sends an element to the filter generated by the curried associated approximable map.

Proof. The equality is the defining action of the currying isomorphism on an element represented by its associated approximable map. ■

Theorem (`primrec_curryStepCode`). The numerical function defining curryStepCode is primitive recursive.

Proof. The curry step code function is built from primitive-recursive projections, constants, pairing and composition. ■

`funPresentation.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Combinators
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_4
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_11
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import Mathlib.Data.Fin.VecNotation

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

def interList (base : Set β) : List (Set β) → Set β
  | [] => base
  | Y :: M => Y ∩ interList base M

@[simp] theorem interList_nil (base : Set β) : interList base [] = base := rfl

end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

def funPair (e : ℕ) : Set α × Set β := (P₀.X e.unpair.1, P₁.X e.unpair.2)

@[simp] theorem funPair_fst (e : ℕ) : (funPair P₀ P₁ e).1 = P₀.X e.unpair.1 := rfl

@[simp] theorem funPair_snd (e : ℕ) : (funPair P₀ P₁ e).2 = P₁.X e.unpair.2 := rfl

def bitSelect : List ℕ → ℕ → List ℕ
  | [], _ => []
  | e :: L, b => if b % 2 = 1 then e :: bitSelect L (b / 2) else bitSelect L (b / 2)

@[simp] theorem bitSelect_nil (b : ℕ) : bitSelect [] b = [] := rfl

theorem andFoldl_eq_one_iff (g : ℕ → ℕ) :
```

```
    ∀ (el : List ℕ) (a : ℕ), a ≤ 1 →
      (List.foldl (fun acc x => selectFn acc (isOne (g x)) 0) a el = 1 ↔
        a = 1 ∧ ∀ e ∈ el, g e = 1)
  | [], a, _ => by simp
  | e :: el, a, ha => by
    rw [List.foldl_cons]
    have hstep : selectFn a (isOne (g e)) 0 ≤ 1 := by
      rcases (show a = 0 ∨ a = 1 by omega) with h | h <;> subst h
      · rw [selectFn_zero]; exact Nat.zero_le 1
      · rw [selectFn_one]; exact isOne_le_one _
    rw [andFoldl_eq_one_iff g el _ hstep]
    rcases (show a = 0 ∨ a = 1 by omega) with h | h <;> subst h
    · rw [selectFn_zero]; simp
    · rw [selectFn_one, List.forall_mem_cons, isOne_eq_one_iff]
      constructor
      · rintro ⟨hge, hall⟩; exact ⟨rfl, hge, hall⟩
      · rintro ⟨_, hge, hall⟩; exact ⟨hge, hall⟩

theorem andFoldl_one (g : ℕ → ℕ) (el : List ℕ) :
    List.foldl (fun acc x => selectFn acc (isOne (g x)) 0) 1 el = 1 ↔ ∀ e ∈ el, g e =
    1 := by
  rw [andFoldl_eq_one_iff g el 1 (le_refl 1)]; simp

def appendStep (acc x : ℕ) : ℕ := Nat.pair x acc + 1

def appendStp (w : ℕ) : ℕ := appendStep w.unpair.2.unpair.1 w.unpair.1

def appendCode (a b : ℕ) : ℕ := foldCode appendStp 0 a b

theorem appendStp_eq (x acc : ℕ) : appendStp (Nat.pair x (Nat.pair acc 0)) =
    appendStep acc x := by
  unfold appendStp; simp only [unpair_pair_fst, unpair_pair_snd]

def botMap : ApproximableMap V₀ V₁ :=
  leastMap [] (by simp) (by intro X _; rw [interYs_nil]; exact V₁.master_mem)

theorem botMap_rel {X : Set α} {Y : Set β} :
    (botMap : ApproximableMap V₀ V₁).rel X Y ↔ V₀.mem X ∧ V₁.mem Y ∧ V₁.master ⊆ Y :=
    Iff.rfl

def condSet (k : ℕ) (A : Set β) (el : List ℕ) : Set β :=
  {z | z ∈ A ∧ ∀ e ∈ el, P₀.X k ⊆ P₀.X e.unpair.1 → z ∈ P₁.X e.unpair.2}

theorem condSet_nil (k : ℕ) (A : Set β) : condSet P₀ P₁ k A [] = A := by
  ext z; simp [condSet]

theorem condSet_subset (k : ℕ) (A : Set β) (el : List ℕ) : condSet P₀ P₁ k A el ⊆ A :=
  fun _ hz => hz.1

def consPairChar (gN : ℕ → ℕ) (n m : ℕ) : ℕ :=
  selectFn (isOne (gN n)) (selectFn (isOne (gN m)) (gN (appendCode n m)) 1) 1

section ConsFold

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)
```

```
def consUpd (projFn fc : ℕ → ℕ) (s x : ℕ) : ℕ :=
  selectFn (s.unpair.1 % 2)
    (Nat.pair (s.unpair.1 / 2)
      (Nat.pair (P.inter s.unpair.2.unpair.1 (projFn x))
        (selectFn s.unpair.2.unpair.2 (isOne (fc (Nat.pair s.unpair.2.unpair.1 (projFn
     x)))) 0)))
    (Nat.pair (s.unpair.1 / 2) (Nat.pair s.unpair.2.unpair.1 s.unpair.2.unpair.2))

def consStp (projFn fc : ℕ → ℕ) (w : ℕ) : ℕ :=
  consUpd P projFn fc w.unpair.2.unpair.1 w.unpair.1

def consCharAt (projFn fc : ℕ → ℕ) (w : ℕ) : ℕ :=
  (foldCode (consStp P projFn fc) 0 (Nat.pair w.unpair.1 (Nat.pair P.masterIdx 1))
    w.unpair.2).unpair.2.unpair.2

end ConsFold

theorem decodeList_appendStep (acc x : ℕ) :
    decodeList (appendStep acc x) = x :: decodeList acc := by
  unfold appendStep; rw [decodeList_succ, unpair_pair_fst, unpair_pair_snd]

theorem decodeList_encodeList (l : List ℕ) : decodeList (encodeList l) = l := by
  induction l with
  | nil => rw [show encodeList ([] : List ℕ) = 0 from rfl, decodeList_zero]
  | cons a l ih =>
    rw [show encodeList (a :: l) = Nat.pair a (encodeList l) + 1 from rfl,
      decodeList_succ, unpair_pair_fst, unpair_pair_snd, ih]

theorem decodeList_foldl_appendStp (el : List ℕ) (acc : ℕ) :
    decodeList (List.foldl (fun acc x => appendStp (Nat.pair x (Nat.pair acc 0))) acc
    el)
      = el.reverse ++ decodeList acc := by
  induction el generalizing acc with
  | nil => simp
  | cons e el ih =>
    rw [List.foldl_cons, ih, appendStp_eq, decodeList_appendStep, List.reverse_cons,
      List.append_assoc, List.singleton_append]

theorem decodeList_appendCode (a b : ℕ) :
    decodeList (appendCode a b) = (decodeList b).reverse ++ decodeList a := by
  unfold appendCode; rw [foldCode_eq']; exact decodeList_foldl_appendStp (decodeList b
    ) a

def funImpChar (fc0 fc1 : ℕ → ℕ) (w : ℕ) : ℕ :=
  selectFn (isOne (consCharAt P₀ (fun e => e.unpair.1) fc0 w))
    (isOne (consCharAt P₁ (fun e => e.unpair.2) fc1 w)) 1

def funConsChar (fc0 fc1 : ℕ → ℕ) (c : ℕ) : ℕ :=
  bForallFn (funImpChar P₀ P₁ fc0 fc1) c (2 ^ c)

def funListOf (el : List ℕ) : List (Set α × Set β) := el.map (funPair P₀ P₁)

def Xenum (gN : ℕ → ℕ) (c : ℕ) : Set (ApproximableMap V₀ V₁) :=
  if gN c = 1 then stepFun (funListOf P₀ P₁ (decodeList c)) else Set.univ

theorem Xenum_neg {gN : ℕ → ℕ} {c : ℕ} (h : gN c = 1) : Xenum P₀ P₁ gN c = Set.univ :=
```

```
    ite_eq_right h

theorem Xenum_pos {gN : ℕ → ℕ} {c : ℕ} (h : gN c = 1) :
    Xenum P₀ P₁ gN c = stepFun (funListOf P₀ P₁ (decodeList c)) := ite_eq_left h

theorem funListOf_cons (e : ℕ) (el : List ℕ) :
    funListOf P₀ P₁ (e :: el) = funPair P₀ P₁ e :: funListOf P₀ P₁ el := rfl

theorem exists_funListOf {L : List (Set α × Set β)} (hL : ∀ p ∈ L, V₀.mem p.1 ∧ V₁.mem
    p.2) :
    ∃ el, funListOf P₀ P₁ el = L := by
  induction L with
  | nil => exact ⟨[], rfl⟩
  | cons p L ih =>
    obtain ⟨X, Y⟩ := p
    obtain ⟨i, hi⟩ := P₀.surj (hL (X, Y) (List.mem_cons.mpr (Or.inl rfl))).1
    obtain ⟨j, hj⟩ := P₁.surj (hL (X, Y) (List.mem_cons.mpr (Or.inl rfl))).2
    obtain ⟨el, hel⟩ := ih (fun q hq => hL q (List.mem_cons.mpr (Or.inr hq)))
    refine ⟨Nat.pair i j :: el, ?_⟩
    rw [funListOf_cons, hel]
    congr 1
    show (P₀.X (Nat.pair i j).unpair.1, P₁.X (Nat.pair i j).unpair.2) = (X, Y)
    rw [unpair_pair_fst, unpair_pair_snd, hi, hj]

theorem funListOf_valid (el : List ℕ) :
    ∀ p ∈ funListOf P₀ P₁ el, V₀.mem p.1 ∧ V₁.mem p.2 := by
  intro p hp
  rw [funListOf, List.mem_map] at hp
  obtain ⟨e, _, rfl⟩ := hp
  exact ⟨P₀.mem_X _, P₁.mem_X _⟩

theorem Xenum_mem (gN : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
      : Set (ApproximableMap V₀ V₁)).Nonempty) (c : ℕ) :
    (funSpace V₀ V₁).mem (Xenum P₀ P₁ gN c) := by
  by_cases h : gN c = 1
  · rw [Xenum_pos P₀ P₁ h]
    exact ⟨⟨funListOf P₀ P₁ (decodeList c), funListOf_valid P₀ P₁ _, rfl⟩, (hgN c).mp h⟩
  · rw [Xenum_neg P₀ P₁ h]; exact funSpace_master ▸ (funSpace V₀ V₁).master_mem

theorem Xenum_nonempty (gN : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
      : Set (ApproximableMap V₀ V₁)).Nonempty) (c : ℕ) :
    (Xenum P₀ P₁ gN c).Nonempty := (Xenum_mem P₀ P₁ gN hgN c).2

section ConsChain

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

def idxchain (js : List ℕ) : ℕ := js.foldl (fun acc j => P.inter acc j) P.masterIdx

def interFrom (A : Set α) : List ℕ → Set α
  | [] => A
  | j :: js => interFrom (A ∩ P.X j) js

end ConsChain
```

```
def interIdx (gN : ℕ → ℕ) (n m : ℕ) : ℕ :=
  selectFn (isOne (gN n)) (selectFn (isOne (gN m)) (appendCode n m) n) m

def interYsStp (incl0 : ℕ → ℕ) (w : ℕ) : ℕ :=
  selectFn (isOne (incl0 (Nat.pair w.unpair.2.unpair.2 w.unpair.1.unpair.1)))
    (P₁.inter w.unpair.2.unpair.1 w.unpair.1.unpair.2) w.unpair.2.unpair.1

def interYsIdx (incl0 : ℕ → ℕ) (a k : ℕ) : ℕ :=
  foldCode (interYsStp P₁ incl0) k P₁.masterIdx a

theorem mem_condSet {k : ℕ} {A : Set β} {el : List ℕ} {z : β} :
    z ∈ condSet P₀ P₁ k A el ↔ z ∈ A ∧ ∀ e ∈ el, P₀.X k ⊆ P₀.X e.unpair.1 → z ∈ P₁.X
    e.unpair.2 :=
  Iff.rfl

theorem condSet_cons_neg {k : ℕ} {A : Set β} {e : ℕ} {el : List ℕ}
    (h : ¬ P₀.X k ⊆ P₀.X e.unpair.1) :
    condSet P₀ P₁ k A (e :: el) = condSet P₀ P₁ k A el := by
  ext z
  simp only [mem_condSet, List.mem_cons]
  constructor
  · rintro ⟨hzA, hall⟩
    exact ⟨hzA, fun e' he' hsub => hall e' (Or.inr he') hsub⟩
  · rintro ⟨hzA, hall⟩
    exact ⟨hzA, fun e' he' hsub => he'.elim (fun heq => absurd (heq ▸ hsub) h) (fun
    hmem => hall e' hmem hsub)⟩

theorem condSet_cons_pos {k : ℕ} {A : Set β} {e : ℕ} {el : List ℕ}
    (h : P₀.X k ⊆ P₀.X e.unpair.1) :
    condSet P₀ P₁ k A (e :: el) = condSet P₀ P₁ k (A ∩ P₁.X e.unpair.2) el := by
  ext z
  simp only [mem_condSet, List.mem_cons, Set.mem_inter_iff]
  constructor
  · rintro ⟨hzA, hall⟩
    exact ⟨⟨hzA, hall e (Or.inl rfl) h⟩, fun e' he' hsub => hall e' (Or.inr he') hsub⟩
  · rintro ⟨⟨hzA, hzE⟩, hall⟩
    exact ⟨hzA, fun e' he' hsub => he'.elim (fun heq => heq ▸ hzE) (fun hmem => hall e'
     hmem hsub)⟩

theorem condFoldl_spec (incl0 : ℕ → ℕ)
    (hincl0 : ∀ s, incl0 s = 1 ↔ P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) (k : ℕ) :
    ∀ (el : List ℕ) (A : Set β) (a : ℕ), P₁.X a = A → V₁.mem A →
      V₁.mem (condSet P₀ P₁ k A el) →
      P₁.X (List.foldl (fun acc x => if incl0 (Nat.pair k x.unpair.1) = 1
              then P₁.inter acc x.unpair.2 else acc) a el) = condSet P₀ P₁ k A el
  | [], A, a, hXa, _, _ => by rw [List.foldl_nil, condSet_nil]; exact hXa
  | e :: el, A, a, hXa, hA, hcons => by
    rw [List.foldl_cons]
    by_cases hpred : incl0 (Nat.pair k e.unpair.1) = 1
    · rw [ite_eq_left hpred]
      have hincl : P₀.X k ⊆ P₀.X e.unpair.1 := by
        have h := (hincl0 (Nat.pair k e.unpair.1)).mp hpred
        rwa [unpair_pair_fst, unpair_pair_snd] at h
      rw [condSet_cons_pos P₀ P₁ hincl] at hcons ⊢
      have hwit : V₁.mem (A ∩ P₁.X e.unpair.2) :=
```

```
        V₁.inter_mem hA (P₁.mem_X _) hcons (condSet_subset P₀ P₁ k (A ∩ P₁.X e.unpair.2)
     el)
      have hk : ∃ j, P₁.X j ⊆ P₁.X a ∩ P₁.X e.unpair.2 := by
        obtain ⟨j, hj⟩ := P₁.surj hwit; exact ⟨j, by rw [hj, hXa]⟩
      have hXinter : P₁.X (P₁.inter a e.unpair.2) = A ∩ P₁.X e.unpair.2 := by
        rw [P₁.inter_spec hk, hXa]
      exact condFoldl_spec incl0 hincl0 k el (A ∩ P₁.X e.unpair.2) (P₁.inter a e.unpair
    .2)
        hXinter hwit hcons
    · rw [ite_eq_right hpred]
      have hincl : ¬ P₀.X k ⊆ P₀.X e.unpair.1 := fun hc =>
        hpred ((hincl0 (Nat.pair k e.unpair.1)).mpr (by rwa [unpair_pair_fst,
    unpair_pair_snd]))
      rw [condSet_cons_neg P₀ P₁ hincl] at hcons ⊢
      exact condFoldl_spec incl0 hincl0 k el A a hXa hA hcons

theorem condSet_eq_interYs (k : ℕ) (el : List ℕ) :
    condSet P₀ P₁ k V₁.master el = interYs V₁.master (funListOf P₀ P₁ el) (P₀.X k) := by
  ext z
  rw [mem_interYs, mem_condSet]
  constructor
  · rintro ⟨hz, hall⟩
    refine ⟨hz, ?_⟩
    intro p hp hsub
    rw [funListOf, List.mem_map] at hp
    obtain ⟨e, he, rfl⟩ := hp
    exact hall e he hsub
  · rintro ⟨hz, hall⟩
    exact ⟨hz, fun e he hsub => hall (funPair P₀ P₁ e) (List.mem_map.mpr ⟨e, he, rfl⟩)
    hsub⟩

theorem primrec_appendCode : Nat.Primrec (fun t => appendCode t.unpair.1 t.unpair.2)
    := by
  have hstp : Nat.Primrec appendStp :=
    (primrec_add₂ (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right))
      (Nat.Primrec.const 1)).of_eq (fun w => rfl)
  exact (primrec_foldCode hstp (Nat.Primrec.const 0) Nat.Primrec.left Nat.Primrec.
    right).of_eq
    (fun t => rfl)

theorem primrec_consPairChar (gN : ℕ → ℕ) (hgNp : Nat.Primrec gN) :
    Nat.Primrec (fun t => consPairChar gN t.unpair.1 t.unpair.2) := by
  have hn := hgNp.comp Nat.Primrec.left
  have hm := hgNp.comp Nat.Primrec.right
  have happ := hgNp.comp primrec_appendCode
  have hinner := primrec_selectFn (primrec_isOne.comp hm) happ (Nat.Primrec.const 1)
  exact (primrec_selectFn (primrec_isOne.comp hn) hinner (Nat.Primrec.const 1)).of_eq
    (fun _ => rfl)

theorem primrec_interIdx (gN : ℕ → ℕ) (hgNp : Nat.Primrec gN) :
    Nat.Primrec (fun t => interIdx gN t.unpair.1 t.unpair.2) := by
  have hn := hgNp.comp Nat.Primrec.left
  have hm := hgNp.comp Nat.Primrec.right
  have hinner := primrec_selectFn (primrec_isOne.comp hm) primrec_appendCode Nat.
    Primrec.left
  exact (primrec_selectFn (primrec_isOne.comp hn) hinner Nat.Primrec.right).of_eq (fun
```

```
    _ => rfl)

theorem primrec_interYsStp (incl0 : ℕ → ℕ) (hincl0p : Nat.Primrec incl0) :
    Nat.Primrec (interYsStp P₁ incl0) := by
  have he1 : Nat.Primrec (fun w : ℕ => w.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.left
  have he2 : Nat.Primrec (fun w : ℕ => w.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have hacc : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hk : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hcond := primrec_isOne.comp (hincl0p.comp (hk.pair he1))
  have hinter := P₁.inter_primrec.comp (hacc.pair he2)
  refine (primrec_selectFn hcond hinter hacc).of_eq (fun w => ?_)
  unfold interYsStp
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_interYsIdx (incl0 : ℕ → ℕ) (hincl0p : Nat.Primrec incl0) :
    Nat.Primrec (fun t => interYsIdx P₁ incl0 t.unpair.1 t.unpair.2) :=
  (primrec_foldCode (primrec_interYsStp P₁ incl0 hincl0p) Nat.Primrec.right
    (Nat.Primrec.const P₁.masterIdx) Nat.Primrec.left).of_eq (fun _ => rfl)

theorem rel_interYs_funList {f : ApproximableMap V₀ V₁} {el : List ℕ}
    (hf : f ∈ stepFun (funListOf P₀ P₁ el)) (n' : ℕ) :
    f.rel (P₀.X n') (interYs V₁.master (funListOf P₀ P₁ el) (P₀.X n')) := by
  induction el with
  | nil =>
    rw [funListOf, List.map_nil, interYs_nil]
    exact f.mono f.master_rel (V₀.sub_master (P₀.mem_X n')) subset_rfl (P₀.mem_X n') V₁
    .master_mem
  | cons e el ih =>
    rw [funListOf_cons] at hf
    have hftail : f ∈ stepFun (funListOf P₀ P₁ el) :=
      fun p hp => hf p (List.mem_cons_of_mem _ hp)
    have htail := ih hftail
    rw [funListOf_cons]
    rcases P₀.incl_computable.em (Nat.pair n' e.unpair.1) with hinc | hinc <;>
      simp only [unpair_pair_fst, unpair_pair_snd] at hinc
    · have hp : f.rel (P₀.X e.unpair.1) (P₁.X e.unpair.2) :=
        hf (funPair P₀ P₁ e) (List.mem_cons.mpr (Or.inl rfl))
      have hXp2 : f.rel (P₀.X n') (P₁.X e.unpair.2) :=
        f.mono hp hinc subset_rfl (P₀.mem_X n') (f.rel_cod hp)
      have heq : interYs V₁.master (funPair P₀ P₁ e :: funListOf P₀ P₁ el) (P₀.X n')
          = P₁.X e.unpair.2 ∩ interYs V₁.master (funListOf P₀ P₁ el) (P₀.X n') := by
        rw [interYs_cons]; ext z
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq, funPair]
        exact ⟨fun ⟨h1, h2⟩ => ⟨h1 hinc, h2⟩, fun ⟨h1, h2⟩ => ⟨fun _ => h1, h2⟩⟩
      rw [heq]; exact f.inter_right hXp2 htail
    · have heq : interYs V₁.master (funPair P₀ P₁ e :: funListOf P₀ P₁ el) (P₀.X n')
          = interYs V₁.master (funListOf P₀ P₁ el) (P₀.X n') := by
        rw [interYs_cons]; ext z
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq, funPair]
        exact ⟨fun h => h.2, fun h => ⟨fun hc => absurd hc hinc, h⟩⟩
      rw [heq]; exact htail
```

```
theorem interYs_funList_mem_of_nonempty {ea : List ℕ}
    (h : (stepFun (funListOf P₀ P₁ ea) : Set (ApproximableMap V₀ V₁)).Nonempty)
    {X : Set α} (hX : V₀.mem X) :
    V₁.mem (interYs V₁.master (funListOf P₀ P₁ ea) X) := by
  obtain ⟨n', hn'⟩ := P₀.surj hX
  obtain ⟨f, hf⟩ := h
  have hr := rel_interYs_funList P₀ P₁ hf n'
  rw [hn'] at hr
  exact f.rel_cod hr

theorem selectFn_isOne (v a b : ℕ) : selectFn (isOne v) a b = if v = 1 then a else b
    := by
  by_cases h : v = 1
  · rw [ite_eq_left h, h, (isOne_eq_one_iff 1).mpr rfl, selectFn_one]
  · rw [ite_eq_right h]
    have h0 : isOne v = 0 := by
      rcases (show isOne v = 0 ∨ isOne v = 1 from by have := isOne_le_one v; omega)
    with h0 | h1
      · exact h0
      · exact absurd ((isOne_eq_one_iff v).mp h1) h
    rw [h0, selectFn_zero]

theorem interYsIdx_spec (incl0 : ℕ → ℕ)
    (hincl0 : ∀ s, incl0 s = 1 ↔ P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) (a k : ℕ)
    (hcons : V₁.mem (interYs V₁.master (funListOf P₀ P₁ (decodeList a)) (P₀.X k))) :
    P₁.X (interYsIdx P₁ incl0 a k)
      = interYs V₁.master (funListOf P₀ P₁ (decodeList a)) (P₀.X k) := by
  unfold interYsIdx
  rw [foldCode_eq']
  have hstep : (fun acc x => interYsStp P₁ incl0 (Nat.pair x (Nat.pair acc k)))
      = (fun acc x => if incl0 (Nat.pair k x.unpair.1) = 1
            then P₁.inter acc x.unpair.2 else acc) := by
    funext acc x
    show interYsStp P₁ incl0 (Nat.pair x (Nat.pair acc k)) = _
    unfold interYsStp
    simp only [unpair_pair_fst, unpair_pair_snd]
    exact selectFn_isOne _ _ _
  rw [hstep]
  rw [← condSet_eq_interYs P₀ P₁ k (decodeList a)] at hcons ⊢
  exact condFoldl_spec P₀ P₁ incl0 hincl0 k (decodeList a) V₁.master P₁.masterIdx
    P₁.masterIdx_spec V₁.master_mem hcons

theorem selectFn_mul_iff (c d : ℕ) : selectFn c d 0 = 1 ↔ c = 1 ∧ d = 1 := by
  have h : selectFn c d 0 = c * d := by simp only [selectFn, Nat.mul_zero, Nat.
    add_zero]
  rw [h, nat_mul_eq_one]


theorem stepFun_funListOf_appendCode (a b : ℕ) :
    (stepFun (funListOf P₀ P₁ (decodeList (appendCode a b))) : Set (ApproximableMap V₀
    V₁)) =
      stepFun (funListOf P₀ P₁ (decodeList a)) ∩ stepFun (funListOf P₀ P₁ (decodeList b
    )) := by
  ext f
  rw [decodeList_appendCode]
  simp only [funListOf, List.map_append, List.map_reverse, mem_stepFun, Set.
```

```
    mem_inter_iff,
    List.mem_append, List.mem_reverse]
  constructor
  · intro h; exact ⟨fun p hp => h p (Or.inr hp), fun p hp => h p (Or.inl hp)⟩
  · rintro ⟨ha, hb⟩ p (hp | hp)
    · exact hb p hp
    · exact ha p hp

theorem Xenum_inter_eq (gN : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
      : Set (ApproximableMap V₀ V₁)).Nonempty) (n m : ℕ)
    (hne : (Xenum P₀ P₁ gN n ∩ Xenum P₀ P₁ gN m).Nonempty) :
    Xenum P₀ P₁ gN (interIdx gN n m) = Xenum P₀ P₁ gN n ∩ Xenum P₀ P₁ gN m := by
  unfold interIdx
  rw [selectFn_isOne]
  by_cases hn : gN n = 1
  · rw [ite_eq_left hn, selectFn_isOne]
    by_cases hm : gN m = 1
    · rw [ite_eq_left hm, Xenum_pos P₀ P₁ hn, Xenum_pos P₀ P₁ hm, ←
    stepFun_funListOf_appendCode P₀ P₁ n m]
      rw [Xenum_pos P₀ P₁ hn, Xenum_pos P₀ P₁ hm] at hne
      have hgne : gN (appendCode n m) = 1 :=
        (hgN _).mpr (by rw [stepFun_funListOf_appendCode P₀ P₁ n m]; exact hne)
      rw [Xenum_pos P₀ P₁ hgne]
    · rw [ite_eq_right hm, Xenum_neg P₀ P₁ hm, Set.inter_univ]
  · rw [ite_eq_right hn, Xenum_neg P₀ P₁ hn, Set.univ_inter]

theorem consPairChar_spec (gN : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
      : Set (ApproximableMap V₀ V₁)).Nonempty) (n m : ℕ) :
    consPairChar gN n m = 1 ↔
      ∃ k, Xenum P₀ P₁ gN k ⊆ Xenum P₀ P₁ gN n ∩ Xenum P₀ P₁ gN m := by
  have hcons_iff : (∃ k, Xenum P₀ P₁ gN k ⊆ Xenum P₀ P₁ gN n ∩ Xenum P₀ P₁ gN m) ↔
      (Xenum P₀ P₁ gN n ∩ Xenum P₀ P₁ gN m).Nonempty := by
    constructor
    · rintro ⟨k, hk⟩; exact (Xenum_nonempty P₀ P₁ gN hgN k).mono hk
    · intro hne
      exact ⟨interIdx gN n m, (Xenum_inter_eq P₀ P₁ gN hgN n m hne).subset⟩
  rw [hcons_iff]
  unfold consPairChar
  by_cases hn : gN n = 1
  · rw [selectFn_isOne, ite_eq_left hn, selectFn_isOne]
    by_cases hm : gN m = 1
    · rw [ite_eq_left hm, hgN (appendCode n m), Xenum_pos P₀ P₁ hn, Xenum_pos P₀ P₁ hm,
        ← stepFun_funListOf_appendCode P₀ P₁ n m]
    · rw [ite_eq_right hm, Xenum_pos P₀ P₁ hn, Xenum_neg P₀ P₁ hm, Set.inter_univ]
      exact iff_of_true rfl ((hgN n).mp hn)
  · rw [selectFn_isOne, ite_eq_right hn, Xenum_neg P₀ P₁ hn, Set.univ_inter]
    exact iff_of_true rfl (Xenum_nonempty P₀ P₁ gN hgN m)

theorem stepFun_funListOf_eq_univ_iff (el : List ℕ) :
    (stepFun (funListOf P₀ P₁ el) : Set (ApproximableMap V₀ V₁)) = Set.univ ↔
      ∀ e ∈ el, P₁.X e.unpair.2 = V₁.master := by
  constructor
  · intro h e he
    have hbot : (botMap : ApproximableMap V₀ V₁) ∈ stepFun (funListOf P₀ P₁ el) := by
```

```
      rw [h]; exact Set.mem_univ _
    have hr := hbot (funPair P₀ P₁ e) (List.mem_map.mpr ⟨e, he, rfl⟩)
    exact Set.Subset.antisymm (V₁.sub_master (P₁.mem_X _)) (botMap_rel.mp hr).2.2
  · intro h
    ext f
    simp only [Set.mem_univ, iff_true, mem_stepFun]
    intro p hp
    rw [funListOf, List.mem_map] at hp
    obtain ⟨e, he, rfl⟩ := hp
    show f.rel (P₀.X e.unpair.1) (P₁.X e.unpair.2)
    rw [h e he]
    exact f.mono f.master_rel (V₀.sub_master (P₀.mem_X _)) subset_rfl (P₀.mem_X _) V₁.
    master_mem

theorem stepFun_funListOf_subset_iff {ea eb : List ℕ}
    (hcons : ∀ {X : Set α}, V₀.mem X → V₁.mem (interYs V₁.master (funListOf P₀ P₁ ea) X
    )) :
    (stepFun (funListOf P₀ P₁ ea) : Set (ApproximableMap V₀ V₁)) ⊆ stepFun (funListOf P
    ₀ P₁ eb) ↔
      ∀ e' ∈ eb, interYs V₁.master (funListOf P₀ P₁ ea) (P₀.X e'.unpair.1) ⊆ P₁.X e'.
    unpair.2 := by
  have hL : ∀ p ∈ funListOf P₀ P₁ ea, V₀.mem p.1 ∧ V₁.mem p.2 := funListOf_valid P₀ P₁
    ea
  constructor
  · intro hsub e' he'
    have hmem := hsub (leastMap_mem_stepFun hL hcons)
    have hrel := hmem (funPair P₀ P₁ e') (List.mem_map.mpr ⟨e', he', rfl⟩)
    exact (leastMap_rel.mp hrel).2.2
  · intro hcond f hf p hp
    rw [funListOf, List.mem_map] at hp
    obtain ⟨e', he', rfl⟩ := hp
    show f.rel (P₀.X e'.unpair.1) (P₁.X e'.unpair.2)
    exact f.mono (rel_interYs_funList P₀ P₁ hf e'.unpair.1) subset_rfl (hcond e' he')
      (P₀.mem_X _) (P₁.mem_X _)

def subStp (incl0 incl1 : ℕ → ℕ) (w : ℕ) : ℕ :=
  selectFn w.unpair.2.unpair.1
    (isOne (incl1 (Nat.pair (interYsIdx P₁ incl0 w.unpair.2.unpair.2 w.unpair.1.unpair
    .1)
      w.unpair.1.unpair.2))) 0

theorem primrec_subStp (incl0 incl1 : ℕ → ℕ) (hincl0p : Nat.Primrec incl0)
    (hincl1p : Nat.Primrec incl1) : Nat.Primrec (subStp P₁ incl0 incl1) := by
  have hflag : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have ha : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hk : Nat.Primrec (fun w : ℕ => w.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.left
  have he2 : Nat.Primrec (fun w : ℕ => w.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have hidx : Nat.Primrec
      (fun w : ℕ => interYsIdx P₁ incl0 w.unpair.2.unpair.2 w.unpair.1.unpair.1) :=
    ((primrec_interYsIdx P₁ incl0 hincl0p).comp (ha.pair hk)).of_eq (fun w => by
      show interYsIdx P₁ incl0 (Nat.pair w.unpair.2.unpair.2 w.unpair.1.unpair.1).
    unpair.1
```

```
        (Nat.pair w.unpair.2.unpair.2 w.unpair.1.unpair.1).unpair.2 = _
      rw [unpair_pair_fst, unpair_pair_snd])
  have hincl := hincl1p.comp (hidx.pair he2)
  refine (primrec_selectFn hflag (primrec_isOne.comp hincl) (Nat.Primrec.const 0)).
    of_eq (fun w => ?_)
  unfold subStp
  rfl

def subChar (incl0 incl1 : ℕ → ℕ) (a b : ℕ) : ℕ := foldCode (subStp P₁ incl0 incl1) a
     1 b

theorem primrec_subChar (incl0 incl1 : ℕ → ℕ) (hincl0p : Nat.Primrec incl0)
    (hincl1p : Nat.Primrec incl1) :
    Nat.Primrec (fun t => subChar P₁ incl0 incl1 t.unpair.1 t.unpair.2) :=
  (primrec_foldCode (primrec_subStp P₁ incl0 incl1 hincl0p hincl1p) Nat.Primrec.left
    (Nat.Primrec.const 1) Nat.Primrec.right).of_eq (fun _ => rfl)

theorem subChar_spec (incl0 incl1 : ℕ → ℕ)
    (hincl0 : ∀ s, incl0 s = 1 ↔ P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2)
    (hincl1 : ∀ s, incl1 s = 1 ↔ P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2)
    {a : ℕ}
    (hane : (stepFun (funListOf P₀ P₁ (decodeList a)) : Set (ApproximableMap V₀ V₁)).
    Nonempty)
    (b : ℕ) :
    subChar P₁ incl0 incl1 a b = 1 ↔
      (stepFun (funListOf P₀ P₁ (decodeList a)) : Set (ApproximableMap V₀ V₁)) ⊆
        stepFun (funListOf P₀ P₁ (decodeList b)) := by
  have hcons : ∀ {X : Set α},
      V₀.mem X → V₁.mem (interYs V₁.master (funListOf P₀ P₁ (decodeList a)) X) :=
    fun {X} hX => interYs_funList_mem_of_nonempty P₀ P₁ hane hX
  rw [stepFun_funListOf_subset_iff P₀ P₁ hcons]
  unfold subChar
  rw [foldCode_eq']
  have hstep : (fun acc x => subStp P₁ incl0 incl1 (Nat.pair x (Nat.pair acc a)))
      = (fun acc x => selectFn acc
          (isOne (incl1 (Nat.pair (interYsIdx P₁ incl0 a x.unpair.1) x.unpair.2))) 0)
    := by
    funext acc x
    show subStp P₁ incl0 incl1 (Nat.pair x (Nat.pair acc a)) = _
    unfold subStp
    simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hstep, andFoldl_one (fun x => incl1 (Nat.pair (interYsIdx P₁ incl0 a x.unpair.1)
    x.unpair.2))]
  refine forall_congr' (fun e' => ?_)
  refine imp_congr_right (fun he' => ?_)
  rw [hincl1]
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [interYsIdx_spec P₀ P₁ incl0 hincl0 a e'.unpair.1 (hcons (P₀.mem_X e'.unpair.1))]

def trivStp (eq1 : ℕ → ℕ) (w : ℕ) : ℕ :=
  selectFn w.unpair.2.unpair.1
    (isOne (eq1 (Nat.pair w.unpair.1.unpair.2 P₁.masterIdx))) 0

theorem primrec_trivStp (eq1 : ℕ → ℕ) (heq1p : Nat.Primrec eq1) :
    Nat.Primrec (trivStp P₁ eq1) := by
  have hflag : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
```

```
    Nat.Primrec.left.comp Nat.Primrec.right
  have he2 : Nat.Primrec (fun w : ℕ => w.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have harg := heq1p.comp (he2.pair (Nat.Primrec.const P₁.masterIdx))
  refine (primrec_selectFn hflag (primrec_isOne.comp harg) (Nat.Primrec.const 0)).
    of_eq (fun w => ?_)
  unfold trivStp
  rfl

def trivialChar (eq1 : ℕ → ℕ) (a : ℕ) : ℕ := foldCode (trivStp P₁ eq1) 0 1 a

def eqEnumChar (gN incl0 incl1 eq1 : ℕ → ℕ) (a b : ℕ) : ℕ :=
  selectFn (isOne (gN a))
    (selectFn (isOne (gN b))
      (selectFn (subChar P₁ incl0 incl1 a b) (subChar P₁ incl0 incl1 b a) 0)
      (trivialChar P₁ eq1 a))
    (selectFn (isOne (gN b)) (trivialChar P₁ eq1 b) 1)

def interEqChar (gN incl0 incl1 eq1 : ℕ → ℕ) (n m k : ℕ) : ℕ :=
  selectFn (consPairChar gN n m) (eqEnumChar P₁ gN incl0 incl1 eq1 (interIdx gN n m) k)
    0

theorem primrec_trivialChar (eq1 : ℕ → ℕ) (heq1p : Nat.Primrec eq1) :
    Nat.Primrec (trivialChar P₁ eq1) :=
  (primrec_foldCode (primrec_trivStp P₁ eq1 heq1p) (Nat.Primrec.const 0)
    (Nat.Primrec.const 1) primrec_id).of_eq (fun _ => rfl)

theorem primrec_eqEnumChar (gN incl0 incl1 eq1 : ℕ → ℕ) (hgNp : Nat.Primrec gN)
    (hincl0p : Nat.Primrec incl0) (hincl1p : Nat.Primrec incl1) (heq1p : Nat.Primrec
    eq1) :
    Nat.Primrec (fun t => eqEnumChar P₁ gN incl0 incl1 eq1 t.unpair.1 t.unpair.2) := by
  have ha := primrec_isOne.comp (hgNp.comp Nat.Primrec.left)
  have hb := primrec_isOne.comp (hgNp.comp Nat.Primrec.right)
  have hsubab : Nat.Primrec (fun t => subChar P₁ incl0 incl1 t.unpair.1 t.unpair.2) :=
    primrec_subChar P₁ incl0 incl1 hincl0p hincl1p
  have hsubba : Nat.Primrec (fun t => subChar P₁ incl0 incl1 t.unpair.2 t.unpair.1) :=
    ((primrec_subChar P₁ incl0 incl1 hincl0p hincl1p).comp
      (Nat.Primrec.right.pair Nat.Primrec.left)).of_eq (fun t => by
        show subChar P₁ incl0 incl1 (Nat.pair t.unpair.2 t.unpair.1).unpair.1
          (Nat.pair t.unpair.2 t.unpair.1).unpair.2 = _
        rw [unpair_pair_fst, unpair_pair_snd])
  have htriva := (primrec_trivialChar P₁ eq1 heq1p).comp Nat.Primrec.left
  have htrivb := (primrec_trivialChar P₁ eq1 heq1p).comp Nat.Primrec.right
  have hbothcons := primrec_selectFn hsubab hsubba (Nat.Primrec.const 0)
  have hacons := primrec_selectFn hb hbothcons htriva
  have hanot := primrec_selectFn hb htrivb (Nat.Primrec.const 1)
  exact (primrec_selectFn ha hacons hanot).of_eq (fun _ => rfl)

theorem primrec_interEqChar (gN incl0 incl1 eq1 : ℕ → ℕ) (hgNp : Nat.Primrec gN)
    (hincl0p : Nat.Primrec incl0) (hincl1p : Nat.Primrec incl1) (heq1p : Nat.Primrec
    eq1) :
    Nat.Primrec (fun t => interEqChar P₁ gN incl0 incl1 eq1 t.unpair.1
      t.unpair.2.unpair.1 t.unpair.2.unpair.2) := by
  have hn : Nat.Primrec (fun t : ℕ => t.unpair.1) := Nat.Primrec.left
  have hm : Nat.Primrec (fun t : ℕ => t.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
```

```
  have hk : Nat.Primrec (fun t : ℕ => t.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hcons := (primrec_consPairChar gN hgNp).comp (hn.pair hm)
  have hidx := (primrec_interIdx gN hgNp).comp (hn.pair hm)
  have heqc := (primrec_eqEnumChar P₁ gN incl0 incl1 eq1 hgNp hincl0p hincl1p heq1p).
    comp
     (hidx.pair hk)
  refine (primrec_selectFn hcons heqc (Nat.Primrec.const 0)).of_eq (fun t => ?_)
  show _ = interEqChar P₁ gN incl0 incl1 eq1 t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.
    unpair.2
  unfold interEqChar
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem trivialChar_spec (eq1 : ℕ → ℕ)
    (heq1 : ∀ s, eq1 s = 1 ↔ P₁.X s.unpair.1 = P₁.X s.unpair.2) (a : ℕ) :
    trivialChar P₁ eq1 a = 1 ↔
      (stepFun (funListOf P₀ P₁ (decodeList a)) : Set (ApproximableMap V₀ V₁)) = Set.
    univ := by
  rw [stepFun_funListOf_eq_univ_iff P₀ P₁]
  unfold trivialChar
  rw [foldCode_eq']
  have hstep : (fun acc x => trivStp P₁ eq1 (Nat.pair x (Nat.pair acc 0)))
      = (fun acc x => selectFn acc (isOne (eq1 (Nat.pair x.unpair.2 P₁.masterIdx))) 0)
    := by
    funext acc x
    show trivStp P₁ eq1 (Nat.pair x (Nat.pair acc 0)) = _
    unfold trivStp
    simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hstep, andFoldl_one (fun x => eq1 (Nat.pair x.unpair.2 P₁.masterIdx))]
  refine forall_congr' (fun e => ?_)
  refine imp_congr_right (fun he => ?_)
  rw [heq1]
  simp only [unpair_pair_fst, unpair_pair_snd, P₁.masterIdx_spec]

theorem eqEnumChar_spec (gN incl0 incl1 eq1 : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
      : Set (ApproximableMap V₀ V₁)).Nonempty)
    (hincl0 : ∀ s, incl0 s = 1 ↔ P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2)
    (hincl1 : ∀ s, incl1 s = 1 ↔ P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2)
    (heq1 : ∀ s, eq1 s = 1 ↔ P₁.X s.unpair.1 = P₁.X s.unpair.2) (a b : ℕ) :
    eqEnumChar P₁ gN incl0 incl1 eq1 a b = 1 ↔ Xenum P₀ P₁ gN a = Xenum P₀ P₁ gN b :=
    by
  unfold eqEnumChar
  by_cases ha : gN a = 1
  · rw [selectFn_isOne, ite_eq_left ha, selectFn_isOne]
    have hanea := (hgN a).mp ha
    by_cases hb : gN b = 1
    · rw [ite_eq_left hb, Xenum_pos P₀ P₁ ha, Xenum_pos P₀ P₁ hb, selectFn_mul_iff,
        subChar_spec P₀ P₁ incl0 incl1 hincl0 hincl1 hanea b,
        subChar_spec P₀ P₁ incl0 incl1 hincl0 hincl1 ((hgN b).mp hb) a]
      exact ⟨fun ⟨h1, h2⟩ => Set.Subset.antisymm h1 h2, fun h => ⟨h.subset, h.superset⟩⟩
    · rw [ite_eq_right hb, Xenum_pos P₀ P₁ ha, Xenum_neg P₀ P₁ hb, trivialChar_spec P₀ P
    ₁ eq1 heq1 a]
  · rw [selectFn_isOne, ite_eq_right ha, selectFn_isOne]
    by_cases hb : gN b = 1
    · rw [ite_eq_left hb, Xenum_neg P₀ P₁ ha, Xenum_pos P₀ P₁ hb, trivialChar_spec P₀ P₁
```

```
      eq1 heq1 b]
      exact eq_comm
    · rw [ite_eq_right hb, Xenum_neg P₀ P₁ ha, Xenum_neg P₀ P₁ hb]
      exact iff_of_true rfl rfl

theorem interEqChar_spec (gN incl0 incl1 eq1 : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
      : Set (ApproximableMap V₀ V₁)).Nonempty)
    (hincl0 : ∀ s, incl0 s = 1 ↔ P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2)
    (hincl1 : ∀ s, incl1 s = 1 ↔ P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2)
    (heq1 : ∀ s, eq1 s = 1 ↔ P₁.X s.unpair.1 = P₁.X s.unpair.2) (n m k : ℕ) :
    interEqChar P₁ gN incl0 incl1 eq1 n m k = 1 ↔
      Xenum P₀ P₁ gN n ∩ Xenum P₀ P₁ gN m = Xenum P₀ P₁ gN k := by
  unfold interEqChar
  rw [selectFn_mul_iff, consPairChar_spec P₀ P₁ gN hgN n m,
    eqEnumChar_spec P₀ P₁ gN incl0 incl1 eq1 hgN hincl0 hincl1 heq1 (interIdx gN n m) k
    ]
  constructor
  · rintro ⟨⟨k', hk'⟩, heq⟩
    have hne : (Xenum P₀ P₁ gN n ∩ Xenum P₀ P₁ gN m).Nonempty :=
      (Xenum_nonempty P₀ P₁ gN hgN k').mono hk'
    rw [← Xenum_inter_eq P₀ P₁ gN hgN n m hne]; exact heq
  · intro heq
    have hne : (Xenum P₀ P₁ gN n ∩ Xenum P₀ P₁ gN m).Nonempty := by
      rw [heq]; exact Xenum_nonempty P₀ P₁ gN hgN k
    exact ⟨⟨k, heq.superset⟩, by rw [Xenum_inter_eq P₀ P₁ gN hgN n m hne]; exact heq⟩

def funPresentation (gN incl0 incl1 eq1 : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
      : Set (ApproximableMap V₀ V₁)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : ∀ s, incl0 s = 1 ↔ P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) (hincl0p : Nat.
    Primrec incl0)
    (hincl1 : ∀ s, incl1 s = 1 ↔ P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) (hincl1p : Nat.
    Primrec incl1)
    (heq1 : ∀ s, eq1 s = 1 ↔ P₁.X s.unpair.1 = P₁.X s.unpair.2) (heq1p : Nat.Primrec
    eq1) :
    ComputablePresentation (funSpace V₀ V₁) where
  X := Xenum P₀ P₁ gN
  mem_X := Xenum_mem P₀ P₁ gN hgN
  surj := by
    rintro W ⟨⟨L, hL, rfl⟩, hne⟩
    obtain ⟨el, hel⟩ := exists_funListOf P₀ P₁ hL
    have hc : gN (encodeList el) = 1 := by rw [hgN, decodeList_encodeList, hel]; exact
     hne
    exact ⟨encodeList el, by rw [Xenum_pos P₀ P₁ hc, decodeList_encodeList, hel]⟩
  interEq_computable :=
    ⟨fun t => interEqChar P₁ gN incl0 incl1 eq1 t.unpair.1 t.unpair.2.unpair.1 t.unpair
    .2.unpair.2,
      primrec_interEqChar P₁ gN incl0 incl1 eq1 hgNp hincl0p hincl1p heq1p,
      fun t => (interEqChar_spec P₀ P₁ gN incl0 incl1 eq1 hgN hincl0 hincl1 heq1
        t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2).symm⟩
  cons_computable :=
    ⟨fun t => consPairChar gN t.unpair.1 t.unpair.2, primrec_consPairChar gN hgNp,
      fun t => (consPairChar_spec P₀ P₁ gN hgN t.unpair.1 t.unpair.2).symm⟩
  inter := interIdx gN
  inter_primrec := primrec_interIdx gN hgNp
```

```
  inter_spec := by
    intro n m h
    obtain ⟨k, hk⟩ := h
    exact Xenum_inter_eq P₀ P₁ gN hgN n m ((Xenum_nonempty P₀ P₁ gN hgN k).mono hk)
  masterIdx := 0
  masterIdx_spec := by
    show Xenum P₀ P₁ gN 0 = Set.univ
    unfold Xenum
    rw [decodeList_zero]
    simp only [funListOf, List.map_nil, stepFun_nil, ite_self]

end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

theorem interList_cons (base : Set β) (Y : Set β) (M : List (Set β)) :
    interList base (Y :: M) = Y ∩ interList base M := rfl

theorem mem_interList {base : Set β} {M : List (Set β)} {z : β} :
    z ∈ interList base M ↔ z ∈ base ∧ ∀ Y ∈ M, z ∈ Y := by
  induction M with
  | nil => simp
  | cons Y M ih =>
    rw [interList_cons]
    simp only [Set.mem_inter_iff, ih, List.mem_cons]
    constructor
    · rintro ⟨hY, hbase, hM⟩
      exact ⟨hbase, fun W hW => hW.elim (fun h => h ▸ hY) (hM W)⟩
    · rintro ⟨hbase, hall⟩
      exact ⟨hall Y (Or.inl rfl), hbase, fun W hW => hall W (Or.inr hW)⟩

theorem rel_interList_of_selection {f : ApproximableMap V₀ V₁}
    {Z : Set α} (hZ : V₀.mem Z) {sub : List (Set α × Set β)}
    (hmem : ∀ p ∈ sub, f.rel p.1 p.2) (hZle : ∀ p ∈ sub, Z ⊆ p.1) :
    f.rel Z (interList V₁.master (sub.map Prod.snd)) := by
  induction sub with
  | nil =>
    simp only [List.map_nil, interList_nil]
    exact f.mono f.master_rel (V₀.sub_master hZ) subset_rfl hZ V₁.master_mem
  | cons p sub ih =>
    have hp : f.rel p.1 p.2 := hmem p (List.mem_cons.mpr (Or.inl rfl))
    have hZp : f.rel Z p.2 :=
      f.mono hp (hZle p (List.mem_cons.mpr (Or.inl rfl))) subset_rfl hZ (f.rel_cod hp)
    have htail : f.rel Z (interList V₁.master (sub.map Prod.snd)) :=
      ih (fun q hq => hmem q (List.mem_cons.mpr (Or.inr hq)))
        (fun q hq => hZle q (List.mem_cons.mpr (Or.inr hq)))
    rw [List.map_cons, interList_cons]
    exact f.inter_right hZp htail

theorem interList_mem_of_stepFun_nonempty {L : List (Set α × Set β)}
    (h : (stepFun L : Set (ApproximableMap V₀ V₁)).Nonempty) {sub : List (Set α × Set
    β)}
    (hsub : sub.Sublist L) {Z : Set α} (hZ : V₀.mem Z) (hZle : ∀ p ∈ sub, Z ⊆ p.1) :
    V₁.mem (interList V₁.master (sub.map Prod.snd)) := by
```

```
  obtain ⟨f, hf⟩ := h
  have hmem : ∀ p ∈ sub, f.rel p.1 p.2 := fun p hp => hf p (hsub.subset hp)
  exact f.rel_cod (rel_interList_of_selection hZ hmem hZle)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

theorem stepFun_funListOf_nonempty_iff (el : List ℕ) :
    (stepFun (funListOf P₀ P₁ el) : Set (ApproximableMap V₀ V₁)).Nonempty ↔
      ∀ sub : List ℕ, sub.Sublist el →
        (∃ Z, V₀.mem Z ∧ ∀ e ∈ sub, Z ⊆ P₀.X e.unpair.1) →
          V₁.mem (interList V₁.master (sub.map (fun e => P₁.X e.unpair.2))) := by
  constructor
  ·
    intro h sub hsub hcons
    obtain ⟨Z, hZ, hZle⟩ := hcons
    have hsub' : (sub.map (funPair P₀ P₁)).Sublist (funListOf P₀ P₁ el) := hsub.map _
    have hZle' : ∀ p ∈ sub.map (funPair P₀ P₁), Z ⊆ p.1 := by
      intro p hp
      rw [List.mem_map] at hp
      obtain ⟨e, he, rfl⟩ := hp
      exact hZle e he
    have hres := interList_mem_of_stepFun_nonempty h hsub' hZ hZle'
    have hlist : (sub.map (funPair P₀ P₁)).map Prod.snd = sub.map (fun e => P₁.X e.
    unpair.2) := by
      rw [List.map_map]; rfl
    rwa [hlist] at hres
  ·
    intro hcond
    have hL := funListOf_valid P₀ P₁ el
    have hcons : ∀ {X' : Set α}, V₀.mem X' →
        V₁.mem (interYs V₁.master (funListOf P₀ P₁ el) X') := by
      intro X' hX'
      obtain ⟨nx, hnx⟩ := P₀.surj hX'
      obtain ⟨finc, _, hfinc⟩ := P₀.incl_computable

      let : DecidablePred (fun e : ℕ => X' ⊆ P₀.X e.unpair.1) := fun e =>
        decidable_of_iff (finc (Nat.pair nx e.unpair.1) = 1) (by
          have h := hfinc (Nat.pair nx e.unpair.1)
          simp only [unpair_pair_fst, unpair_pair_snd] at h
          rw [hnx] at h
          exact h.symm)
      set sub : List ℕ := el.filter (fun e => decide (X' ⊆ P₀.X e.unpair.1)) with
    hsubdef
      have hsub : sub.Sublist el := List.filter_sublist
      have hmem_iff : ∀ e, e ∈ sub ↔ e ∈ el ∧ X' ⊆ P₀.X e.unpair.1 := by
        intro e
        rw [hsubdef, List.mem_filter, decide_eq_true_iff]
      have heq : interYs V₁.master (funListOf P₀ P₁ el) X'
          = interList V₁.master (sub.map (fun e => P₁.X e.unpair.2)) := by
        apply Set.ext; intro z
        rw [mem_interYs, mem_interList]
        constructor
        · rintro ⟨hzb, hall⟩
          refine ⟨hzb, ?_⟩
          intro Y hY
          rw [List.mem_map] at hY
```

```
          obtain ⟨e, hesub, rfl⟩ := hY
          have he := (hmem_iff e).mp hesub
          have hpair : funPair P₀ P₁ e ∈ funListOf P₀ P₁ el := List.mem_map.mpr ⟨e, he
    .1, rfl⟩
          exact hall (funPair P₀ P₁ e) hpair he.2
        · rintro ⟨hzb, hall⟩
          refine ⟨hzb, ?_⟩
          intro p hp hXp
          rw [funListOf, List.mem_map] at hp
          obtain ⟨e, hee, rfl⟩ := hp
          have hesub : e ∈ sub := (hmem_iff e).mpr ⟨hee, hXp⟩
          exact hall (P₁.X e.unpair.2) (List.mem_map.mpr ⟨e, hesub, rfl⟩)
      rw [heq]
      refine hcond sub hsub ⟨X', hX', ?_⟩
      intro e he
      exact ((hmem_iff e).mp he).2
    exact ⟨leastMap (funListOf P₀ P₁ el) hL hcons, leastMap_mem_stepFun hL hcons⟩

section ConsChain

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

theorem mem_interFrom {A : Set α} {js : List ℕ} {z : α} :
    z ∈ interFrom P A js ↔ z ∈ A ∧ ∀ j ∈ js, z ∈ P.X j := by
  induction js generalizing A with
  | nil => simp [interFrom]
  | cons j js ih =>
    rw [interFrom, ih]
    simp only [Set.mem_inter_iff, List.mem_cons]
    constructor
    · rintro ⟨⟨hA, hj⟩, hrest⟩
      exact ⟨hA, fun j' hj' => hj'.elim (fun h => h ▸ hj) (hrest j')⟩
    · rintro ⟨hA, hall⟩
      exact ⟨⟨hA, hall j (Or.inl rfl)⟩, fun j' hj' => hall j' (Or.inr hj')⟩

theorem interFrom_subset {A : Set α} {js : List ℕ} : interFrom P A js ⊆ A :=
  fun _ hz => (mem_interFrom P |>.mp hz).1

theorem interFrom_mem_of_witness {A : Set α} {js : List ℕ} {Z : Set α}
    (hZ : V.mem Z) (hZsub : Z ⊆ interFrom P A js) (hA : V.mem A) :
    V.mem (interFrom P A js) := by
  induction js generalizing A with
  | nil => simpa [interFrom] using hA
  | cons j js ih =>
    rw [interFrom] at hZsub ⊢
    have hAXj : V.mem (A ∩ P.X j) :=
      V.inter_mem hA (P.mem_X j) hZ (hZsub.trans (interFrom_subset P))
    exact ih hZsub hAXj

theorem interFrom_eq_of_foldl : ∀ (js : List ℕ) (A : Set α) (a : ℕ),
    P.X a = A → V.mem A → V.mem (interFrom P A js) →
    P.X (js.foldl (fun acc j => P.inter acc j) a) = interFrom P A js
  | [], A, a, hXa, _, _ => by simpa [interFrom] using hXa
  | j :: js, A, a, hXa, hA, hmem => by
    have hmem' : V.mem (interFrom P (A ∩ P.X j) js) := by rwa [interFrom] at hmem
    have hAXj : V.mem (A ∩ P.X j) :=
```

```
      V.inter_mem hA (P.mem_X j) hmem' (interFrom_subset P)
    have hk : ∃ k, P.X k ⊆ P.X a ∩ P.X j := by
      obtain ⟨k, hk⟩ := P.surj hAXj
      exact ⟨k, by rw [hk, hXa]⟩
    have hXinter : P.X (P.inter a j) = A ∩ P.X j := by rw [P.inter_spec hk, hXa]
    rw [List.foldl_cons, interFrom]
    exact interFrom_eq_of_foldl js (A ∩ P.X j) (P.inter a j) hXinter hAXj hmem'

theorem idxchain_spec {js : List ℕ} (h : V.mem (interFrom P V.master js)) :
    P.X (idxchain P js) = interFrom P V.master js :=
  interFrom_eq_of_foldl P js V.master P.masterIdx P.masterIdx_spec V.master_mem h

end ConsChain

theorem bitSelect_cons (e : ℕ) (L : List ℕ) (b : ℕ) :
    bitSelect (e :: L) b =
      if b % 2 = 1 then e :: bitSelect L (b / 2) else bitSelect L (b / 2) := rfl

theorem bitSelect_sublist (L : List ℕ) (b : ℕ) : (bitSelect L b).Sublist L := by
  induction L generalizing b with
  | nil => simp
  | cons e L ih =>
    rw [bitSelect_cons]
    split
    · exact (ih (b / 2)).cons_cons e
    · exact (ih (b / 2)).cons e

theorem exists_bitSelect_lt {L sub : List ℕ} (hsub : sub.Sublist L) :
    ∃ b, b < 2 ^ L.length ∧ bitSelect L b = sub := by
  induction hsub with
  | slnil => exact ⟨0, by rw [List.length_nil, pow_zero]; exact Nat.one_pos, rfl⟩
  | @cons l₁ l₂ e _h ih =>
    obtain ⟨b, hb, heq⟩ := ih
    refine ⟨2 * b, ?_, ?_⟩
    · have hpow : 2 ^ (l₂.length + 1) = 2 ^ l₂.length * 2 := pow_succ 2 l₂.length
      rw [List.length_cons]; omega
    · rw [bitSelect_cons, ite_eq_right (by omega), show 2 * b / 2 = b by omega]; exact
    heq
  | @cons_cons l₁ l₂ e _h ih =>
    obtain ⟨b, hb, heq⟩ := ih
    refine ⟨2 * b + 1, ?_, ?_⟩
    · have hpow : 2 ^ (l₂.length + 1) = 2 ^ l₂.length * 2 := pow_succ 2 l₂.length
      rw [List.length_cons]; omega
    · rw [bitSelect_cons, ite_eq_left (by omega), show (2 * b + 1) / 2 = b by omega];
    rw [heq]

theorem forall_sublist_iff_forall_bitmask (c : ℕ) (p : List ℕ → Prop) :
    (∀ sub, sub.Sublist (decodeList c) → p sub) ↔
      ∀ b, b < 2 ^ c → p (bitSelect (decodeList c) b) := by
  constructor
  · intro hall b _; exact hall _ (bitSelect_sublist _ _)
  · intro hall sub hsub
    obtain ⟨b, hb, heq⟩ := exists_bitSelect_lt hsub
    have hb' : b < 2 ^ c :=
      Nat.lt_of_lt_of_le hb (Nat.pow_le_pow_right (Nat.le_succ 1) (
    decodeList_length_le c))
```

```
    rw [← heq]; exact hall b hb'

section ConsFold

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

theorem consUpd_eval (projFn fc : ℕ → ℕ) (b idx flag x : ℕ) :
    consUpd P projFn fc (Nat.pair b (Nat.pair idx flag)) x =
      selectFn (b % 2)
        (Nat.pair (b / 2)
          (Nat.pair (P.inter idx (projFn x))
            (selectFn flag (isOne (fc (Nat.pair idx (projFn x)))) 0)))
        (Nat.pair (b / 2) (Nat.pair idx flag)) := by
  unfold consUpd
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem consUpd_foldl_flag_zero (projFn fc : ℕ → ℕ) (el : List ℕ) (b a : ℕ) :
    (List.foldl (fun s x => consUpd P projFn fc s x)
      (Nat.pair b (Nat.pair a 0)) el).unpair.2.unpair.2 = 0 := by
  induction el generalizing b a with
  | nil => simp only [List.foldl_nil, unpair_pair_snd]
  | cons e el ih =>
    rw [List.foldl_cons, consUpd_eval]
    by_cases hb : b % 2 = 1
    · rw [hb, selectFn_one, selectFn_zero]
      exact ih (b / 2) (P.inter a (projFn e))
    · rw [show b % 2 = 0 by omega, selectFn_zero]
      exact ih (b / 2) a

theorem consUpd_foldl_spec (projFn fc : ℕ → ℕ)
    (hfc : ∀ s, fc s = 1 ↔ ∃ k, P.X k ⊆ P.X s.unpair.1 ∩ P.X s.unpair.2)
    (el : List ℕ) (b a : ℕ) :
    (List.foldl (fun s x => consUpd P projFn fc s x)
        (Nat.pair b (Nat.pair a 1)) el).unpair.2.unpair.2 = 1 ↔
      V.mem (interFrom P (P.X a) (List.map projFn (bitSelect el b))) := by
  induction el generalizing b a with
  | nil =>
    have hL : (List.foldl (fun s x => consUpd P projFn fc s x)
        (Nat.pair b (Nat.pair a 1)) []).unpair.2.unpair.2 = 1 := by
      simp only [List.foldl_nil, unpair_pair_snd]
    rw [bitSelect_nil, List.map_nil, show interFrom P (P.X a) [] = P.X a from rfl]
    exact iff_of_true hL (P.mem_X a)
  | cons e el ih =>
    rw [List.foldl_cons, consUpd_eval, bitSelect_cons]
    by_cases hb : b % 2 = 1
    · rw [ite_eq_left hb, List.map_cons, hb, selectFn_one, selectFn_one,
        show interFrom P (P.X a) (projFn e :: List.map projFn (bitSelect el (b / 2)))
          = interFrom P (P.X a ∩ P.X (projFn e)) (List.map projFn (bitSelect el (b /
    2))) from rfl]
      by_cases hfce : fc (Nat.pair a (projFn e)) = 1
      · have hcons : ∃ k, P.X k ⊆ P.X a ∩ P.X (projFn e) := by
          have h := (hfc (Nat.pair a (projFn e))).mp hfce
          rwa [unpair_pair_fst, unpair_pair_snd] at h
        have hXa' : P.X (P.inter a (projFn e)) = P.X a ∩ P.X (projFn e) := P.
    inter_spec hcons
        rw [(isOne_eq_one_iff _).mpr hfce, ih (b / 2) (P.inter a (projFn e)), hXa']
```

```
      · have his0 : isOne (fc (Nat.pair a (projFn e))) = 0 := by
          have hle := isOne_le_one (fc (Nat.pair a (projFn e)))
          rcases (show isOne (fc (Nat.pair a (projFn e))) = 0 ∨
              isOne (fc (Nat.pair a (projFn e))) = 1 by omega) with h | h
          · exact h
          · exact absurd ((isOne_eq_one_iff _).mp h) hfce
        rw [his0]
        constructor
        · intro h
          rw [consUpd_foldl_flag_zero P projFn fc el (b / 2) (P.inter a (projFn e))]
    at h
          exact absurd h (by decide)
        · intro h
          exact absurd
            ((hfc (Nat.pair a (projFn e))).mpr (by
              obtain ⟨k, hk⟩ := P.surj h
              rw [unpair_pair_fst, unpair_pair_snd]
              exact ⟨k, by rw [hk]; exact interFrom_subset P⟩))
            hfce
    · rw [ite_eq_right hb, show b % 2 = 0 by omega, selectFn_zero]
      exact ih (b / 2) a

theorem consCharAt_spec (projFn fc : ℕ → ℕ)
    (hfc : ∀ s, fc s = 1 ↔ ∃ k, P.X k ⊆ P.X s.unpair.1 ∩ P.X s.unpair.2) (w : ℕ) :
    consCharAt P projFn fc w = 1 ↔
      V.mem (interFrom P V.master
        (List.map projFn (bitSelect (decodeList w.unpair.2) w.unpair.1))) := by
  unfold consCharAt
  rw [foldCode_eq']
  have hstep : (fun acc x => consStp P projFn fc (Nat.pair x (Nat.pair acc 0)))
      = (fun acc x => consUpd P projFn fc acc x) := by
    funext acc x
    show consStp P projFn fc (Nat.pair x (Nat.pair acc 0)) = _
    unfold consStp
    simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hstep, consUpd_foldl_spec P projFn fc hfc (decodeList w.unpair.2) w.unpair.1 P.
    masterIdx,
    P.masterIdx_spec]

theorem primrec_consStp {projFn fc : ℕ → ℕ} (hprojp : Nat.Primrec projFn) (hfcp : Nat
    .Primrec fc) :
    Nat.Primrec (consStp P projFn fc) := by
  have hb : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hidx : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.
    right))
  have hflag : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.
    right))
  have hpx : Nat.Primrec (fun w : ℕ => projFn w.unpair.1) := hprojp.comp Nat.Primrec.
    left
  have hpair := hidx.pair hpx
  have hinter := P.inter_primrec.comp hpair
  have hfcv := primrec_isOne.comp (hfcp.comp hpair)
  have hsel := primrec_selectFn hflag hfcv (Nat.Primrec.const 0)
```

```
  have hyes := (primrec_div2.comp hb).pair (hinter.pair hsel)
  have hno := (primrec_div2.comp hb).pair (hidx.pair hflag)
  refine (primrec_selectFn (primrec_mod2.comp hb) hyes hno).of_eq (fun w => ?_)
  unfold consStp consUpd
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_consCharAt {projFn fc : ℕ → ℕ} (hprojp : Nat.Primrec projFn)
    (hfcp : Nat.Primrec fc) : Nat.Primrec (consCharAt P projFn fc) := by
  have hfold := primrec_foldCode (primrec_consStp P hprojp hfcp) (Nat.Primrec.const 0)
    (Nat.Primrec.left.pair ((Nat.Primrec.const P.masterIdx).pair (Nat.Primrec.const 1)
    ))
    Nat.Primrec.right
  exact ((Nat.Primrec.right.comp Nat.Primrec.right).comp hfold).of_eq (fun _ => rfl)

theorem consFold_decidable (projFn fc : ℕ → ℕ)
    (hfc : ∀ s, fc s = 1 ↔ ∃ k, P.X k ⊆ P.X s.unpair.1 ∩ P.X s.unpair.2)
    (hprojp : Nat.Primrec projFn) (hfcp : Nat.Primrec fc) :
    RecDecidable (fun w =>
      V.mem (interFrom P V.master
        (List.map projFn (bitSelect (decodeList w.unpair.2) w.unpair.1)))) :=
  ⟨consCharAt P projFn fc, primrec_consCharAt P hprojp hfcp,
    fun w => (consCharAt_spec P projFn fc hfc w).symm⟩

end ConsFold

theorem interList_X_eq_interFrom {V : NeighborhoodSystem α} (P :
    ComputablePresentation V)
    (js : List ℕ) : interList V.master (js.map (fun j => P.X j)) = interFrom P V.
    master js := by
  apply Set.ext; intro z
  rw [mem_interList, mem_interFrom]
  constructor
  · rintro ⟨hb, hall⟩
    exact ⟨hb, fun j hj => hall (P.X j) (List.mem_map.mpr ⟨j, hj, rfl⟩)⟩
  · rintro ⟨hb, hall⟩
    refine ⟨hb, fun Y hY => ?_⟩
    rw [List.mem_map] at hY
    obtain ⟨j, hj, rfl⟩ := hY
    exact hall j hj

theorem antecedent_cons_iff (sub : List ℕ) :
    (∃ Z, V₀.mem Z ∧ ∀ e ∈ sub, Z ⊆ P₀.X e.unpair.1) ↔
      V₀.mem (interFrom P₀ V₀.master (sub.map (fun e => e.unpair.1))) := by
  constructor
  · rintro ⟨Z, hZ, hZle⟩
    refine interFrom_mem_of_witness P₀ hZ (fun z hz => ?_) V₀.master_mem
    rw [mem_interFrom]
    refine ⟨V₀.sub_master hZ hz, fun j hj => ?_⟩
    rw [List.mem_map] at hj
    obtain ⟨e, he, rfl⟩ := hj
    exact hZle e he hz
  · intro hmem
    refine ⟨P₀.X (idxchain P₀ (sub.map (fun e => e.unpair.1))), P₀.mem_X _, fun e he z
    hz => ?_⟩
    have hj : e.unpair.1 ∈ sub.map (fun e => e.unpair.1) := List.mem_map.mpr ⟨e, he,
    rfl⟩
```

```
    rw [idxchain_spec P₀ hmem] at hz
    exact (mem_interFrom P₀ |>.mp hz).2 _ hj

theorem funConsequent_eq (sub : List ℕ) :
    interList V₁.master (sub.map (fun e => P₁.X e.unpair.2))
      = interFrom P₁ V₁.master (sub.map (fun e => e.unpair.2)) := by
  rw [← interList_X_eq_interFrom P₁, List.map_map]; rfl

theorem funCons_iff (c : ℕ) :
    (stepFun (funListOf P₀ P₁ (decodeList c)) : Set (ApproximableMap V₀ V₁)).Nonempty ↔
      ∀ b, b < 2 ^ c →
        V₀.mem (interFrom P₀ V₀.master ((bitSelect (decodeList c) b).map (fun e => e.
    unpair.1))) →
          V₁.mem (interFrom P₁ V₁.master ((bitSelect (decodeList c) b).map (fun e => e.
    unpair.2))) := by
  rw [stepFun_funListOf_nonempty_iff]
  constructor
  · intro h b hb hant0
    have hsub : (bitSelect (decodeList c) b).Sublist (decodeList c) :=
    bitSelect_sublist _ _
    have hcons := h _ hsub ((antecedent_cons_iff P₀ (bitSelect (decodeList c) b)).mpr
    hant0)
    rwa [funConsequent_eq P₁] at hcons
  · intro h sub hsub hant
    obtain ⟨b, hb, heq⟩ := exists_bitSelect_lt hsub
    have hb' : b < 2 ^ c :=
      Nat.lt_of_lt_of_le hb (Nat.pow_le_pow_right (Nat.le_succ 1) (
    decodeList_length_le c))
    have hcons := h b hb' (by rw [heq]; exact (antecedent_cons_iff P₀ sub).mp hant)
    rw [heq, ← funConsequent_eq P₁] at hcons
    exact hcons

theorem funCons_decidable (fc0 fc1 : ℕ → ℕ)
    (hfc0 : ∀ s, fc0 s = 1 ↔ ∃ k, P₀.X k ⊆ P₀.X s.unpair.1 ∩ P₀.X s.unpair.2)
    (hfc1 : ∀ s, fc1 s = 1 ↔ ∃ k, P₁.X k ⊆ P₁.X s.unpair.1 ∩ P₁.X s.unpair.2)
    (hfc0p : Nat.Primrec fc0) (hfc1p : Nat.Primrec fc1) :
    RecDecidable (fun c =>
      (stepFun (funListOf P₀ P₁ (decodeList c)) : Set (ApproximableMap V₀ V₁)).Nonempty
    ) := by
  have hc0 := consFold_decidable P₀ (fun e => e.unpair.1) fc0 hfc0 Nat.Primrec.left
    hfc0p
  have hc1 := consFold_decidable P₁ (fun e => e.unpair.2) fc1 hfc1 Nat.Primrec.right
    hfc1p
  have himp : RecDecidable (fun w =>
      V₀.mem (interFrom P₀ V₀.master
          (List.map (fun e => e.unpair.1) (bitSelect (decodeList w.unpair.2) w.unpair
    .1))) →
        V₁.mem (interFrom P₁ V₁.master
          (List.map (fun e => e.unpair.2) (bitSelect (decodeList w.unpair.2) w.unpair
    .1)))) := by
    refine RecDecidable.of_iff (fun w => ?_) (hc0.not.or hc1)
    constructor
    · intro himp
      rcases hc0.em w with h0 | h0
      · exact Or.inr (himp h0)
      · exact Or.inl h0
```

```
    · rintro (h0 | h1) hp0
      · exact absurd hp0 h0
      · exact h1
  refine RecDecidable.of_iff (fun c => ?_)
    (himp.bForall (bound := fun n => 2 ^ n) (primrec_two_pow primrec_id))
  rw [funCons_iff P₀ P₁ c]
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem selectFn_one_iff_imp {A B : ℕ} (hA : A ≤ 1) :
    selectFn A B 1 = 1 ↔ (A = 1 → B = 1) := by
  rcases (show A = 0 ∨ A = 1 by omega) with h | h <;> subst h
  · rw [selectFn_zero]; simp
  · rw [selectFn_one]; simp

theorem funConsChar_spec (fc0 fc1 : ℕ → ℕ)
    (hfc0 : ∀ s, fc0 s = 1 ↔ ∃ k, P₀.X k ⊆ P₀.X s.unpair.1 ∩ P₀.X s.unpair.2)
    (hfc1 : ∀ s, fc1 s = 1 ↔ ∃ k, P₁.X k ⊆ P₁.X s.unpair.1 ∩ P₁.X s.unpair.2) (c : ℕ)
    :
    funConsChar P₀ P₁ fc0 fc1 c = 1 ↔
      (stepFun (funListOf P₀ P₁ (decodeList c)) : Set (ApproximableMap V₀ V₁)).Nonempty
     := by
  rw [funCons_iff P₀ P₁ c]
  unfold funConsChar
  rw [bForallFn_eq_one_iff]
  refine forall_congr' (fun b => ?_)
  refine imp_congr_right (fun hb => ?_)
  unfold funImpChar
  rw [selectFn_one_iff_imp (isOne_le_one _), isOne_eq_one_iff, isOne_eq_one_iff,
    consCharAt_spec P₀ (fun e => e.unpair.1) fc0 hfc0,
    consCharAt_spec P₁ (fun e => e.unpair.2) fc1 hfc1]
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_funImpChar (fc0 fc1 : ℕ → ℕ) (hfc0p : Nat.Primrec fc0) (hfc1p : Nat.
    Primrec fc1) :
    Nat.Primrec (funImpChar P₀ P₁ fc0 fc1) :=
  primrec_selectFn (primrec_isOne.comp (primrec_consCharAt P₀ Nat.Primrec.left hfc0p))
    (primrec_isOne.comp (primrec_consCharAt P₁ Nat.Primrec.right hfc1p)) (Nat.Primrec.
    const 1)

theorem primrec_funConsChar (fc0 fc1 : ℕ → ℕ) (hfc0p : Nat.Primrec fc0) (hfc1p : Nat.
    Primrec fc1) :
    Nat.Primrec (funConsChar P₀ P₁ fc0 fc1) := by
  have hgp := primrec_funImpChar P₀ P₁ fc0 fc1 hfc0p hfc1p
  have hGfn : Nat.Primrec (fun w => selectFn w.unpair.2.unpair.2
      (isOne (funImpChar P₀ P₁ fc0 fc1 (Nat.pair w.unpair.2.unpair.1 w.unpair.1))) 0)
    :=
    primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right)
      (primrec_isOne.comp (hgp.comp
        ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left)))
      (Nat.Primrec.const 0)
  have hprec := Nat.Primrec.prec (Nat.Primrec.const 1) hGfn
  refine (hprec.comp (primrec_id.pair (primrec_two_pow primrec_id))).of_eq (fun c => ?
    _)
  show _ = funConsChar P₀ P₁ fc0 fc1 c
  unfold funConsChar bForallFn
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
```

```
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

theorem funSpace_isEffectivelyGiven (h₀ : V₀.IsEffectivelyGiven) (h₁ : V₁.
    IsEffectivelyGiven) :
    (funSpace V₀ V₁).IsEffectivelyGiven := by
  obtain ⟨P₀⟩ := h₀
  obtain ⟨P₁⟩ := h₁
  obtain ⟨incl0, hincl0p, hincl0s⟩ := P₀.incl_computable
  obtain ⟨incl1, hincl1p, hincl1s⟩ := P₁.incl_computable
  obtain ⟨eq1, heq1p, heq1s⟩ := P₁.eq_computable
  obtain ⟨fc0, hfc0p, hfc0s⟩ := P₀.cons_computable
  obtain ⟨fc1, hfc1p, hfc1s⟩ := P₁.cons_computable
  exact ⟨funPresentation P₀ P₁ (funConsChar P₀ P₁ fc0 fc1) incl0 incl1 eq1
    (funConsChar_spec P₀ P₁ fc0 fc1 (fun s => (hfc0s s).symm) (fun s => (hfc1s s).symm)
    )
    (primrec_funConsChar P₀ P₁ fc0 fc1 hfc0p hfc1p)
    (fun s => (hincl0s s).symm) hincl0p
    (fun s => (hincl1s s).symm) hincl1p


    (fun s => (heq1s s).symm) heq1p⟩

theorem Xenum_singleton (gN : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
      : Set (ApproximableMap V₀ V₁)).Nonempty) (j m : ℕ) :
    Xenum P₀ P₁ gN (Nat.pair (Nat.pair j m) 0 + 1) = step (P₀.X j) (P₁.X m) := by
  have hfp : funPair P₀ P₁ (Nat.pair j m) = (P₀.X j, P₁.X m) := by
    show (P₀.X (Nat.pair j m).unpair.1, P₁.X (Nat.pair j m).unpair.2) = (P₀.X j, P₁.X m
    )
    rw [unpair_pair_fst, unpair_pair_snd]
  have hdec : decodeList (Nat.pair (Nat.pair j m) 0 + 1) = [Nat.pair j m] := by
    rw [decodeList_succ, unpair_pair_fst, unpair_pair_snd, decodeList_zero]
  have hstepeq : (stepFun (funListOf P₀ P₁ [Nat.pair j m]) : Set (ApproximableMap V₀ V₁
    ))
      = step (P₀.X j) (P₁.X m) := by
    show stepFun [funPair P₀ P₁ (Nat.pair j m)] = step (P₀.X j) (P₁.X m)
    rw [hfp, stepFun_singleton]
  have hcons : gN (Nat.pair (Nat.pair j m) 0 + 1) = 1 := by
    rw [hgN, hdec, hstepeq]; exact (step_mem (P₀.mem_X j) (P₁.mem_X m)).2
  rw [Xenum_pos P₀ P₁ hcons, hdec, hstepeq]

section ConsChain

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

theorem consChain_iff (js : List ℕ) :
    (∀ j ∈ js, P.X (idxchain P js) ⊆ P.X j) ↔ V.mem (interFrom P V.master js) := by
  constructor
  · intro hle
    have hsub : P.X (idxchain P js) ⊆ interFrom P V.master js := by
      intro z hz
```

```
      rw [mem_interFrom]
      exact ⟨V.sub_master (P.mem_X _) hz, fun j hj => hle j hj hz⟩
    exact interFrom_mem_of_witness P (P.mem_X _) hsub V.master_mem
  · intro hmem j hj
    rw [idxchain_spec P hmem]
    exact fun z hz => (mem_interFrom P |>.mp hz).2 j hj

end ConsChain

theorem evalMap_rel_prodNbhd_iff {F : Set (ApproximableMap V₀ V₁)} (hF : (funSpace V₀ V
    ₁).mem F)
    {X : Set α} (hX : V₀.mem X) {Y : Set β} (hY : V₁.mem Y) :
    (evalMap V₀ V₁).rel (prodNbhd F X) Y ↔ F ⊆ step X Y := by
  show (prod (funSpace V₀ V₁) V₀).mem (prodNbhd F X) ∧
    (eval V₀ V₁).rel (Sum.inl ^-1' prodNbhd F X) (Sum.inr ^-1' prodNbhd F X) Y ↔ _
  rw [prodNbhd.inl_preimage, prodNbhd.inr_preimage]
  constructor
  · rintro ⟨_, _, _, _, hrel⟩ f hf; exact mem_step.mpr (hrel f hf)
  · intro hsub
    exact ⟨prod_mem_prodNbhd hF hX, hF, hX, hY, fun f hf => mem_step.mp (hsub hf)⟩

theorem evalMap_isComputable (gN incl0 incl1 eq1 : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
      : Set (ApproximableMap V₀ V₁)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : ∀ s, incl0 s = 1 ↔ P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) (hincl0p : Nat.
    Primrec incl0)
    (hincl1 : ∀ s, incl1 s = 1 ↔ P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) (hincl1p : Nat.
    Primrec incl1)
    (heq1 : ∀ s, eq1 s = 1 ↔ P₁.X s.unpair.1 = P₁.X s.unpair.2) (heq1p : Nat.Primrec
    eq1) :
    ComputableMap
      (prodPresentation (funPresentation P₀ P₁ gN incl0 incl1 eq1 hgN hgNp hincl0
    hincl0p
        hincl1 hincl1p heq1 heq1p) P₀) P₁ (evalMap V₀ V₁) := by
  constructor

  have hsub : RecDecidable (fun s => Xenum P₀ P₁ gN s.unpair.1 ⊆ Xenum P₀ P₁ gN s.
    unpair.2) :=
    (funPresentation P₀ P₁ gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p
      hincl1 hincl1p heq1 heq1p).incl_computable

  have h11 : Nat.Primrec (fun t : ℕ => t.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.left
  have h12 : Nat.Primrec (fun t : ℕ => t.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have hg : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1
      (Nat.pair (Nat.pair t.unpair.1.unpair.2 t.unpair.2) 0 + 1)) :=
    h11.pair (Nat.Primrec.succ.comp ((h12.pair Nat.Primrec.right).pair (Nat.Primrec.
    const 0)))
  refine (RecDecidable.of_iff (fun t => ?_) (hsub.comp hg)).re
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [Xenum_singleton P₀ P₁ gN hgN t.unpair.1.unpair.2 t.unpair.2,
    show (prodPresentation (funPresentation P₀ P₁ gN incl0 incl1 eq1 hgN hgNp hincl0
    hincl0p
        hincl1 hincl1p heq1 heq1p) P₀).X t.unpair.1
      = prodNbhd (Xenum P₀ P₁ gN t.unpair.1.unpair.1) (P₀.X t.unpair.1.unpair.2) from
```

```
    rfl,
    evalMap_rel_prodNbhd_iff (Xenum_mem P₀ P₁ gN hgN _) (P₀.mem_X _) (P₁.mem_X _)]

theorem funListOf_append (la lb : List ℕ) :
    funListOf P₀ P₁ (la ++ lb) = funListOf P₀ P₁ la ++ funListOf P₀ P₁ lb := by
  simp [funListOf, List.map_append]

end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}

theorem interList_subset_base {base : Set β} {M : List (Set β)} : interList base M ⊆
    base :=
  fun _ hz => (mem_interList.mp hz).1
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

theorem mem_Xenum_iff (gN : ℕ → ℕ) (φ : (funSpace V₀ V₁).Element) (c : ℕ) :
    φ.mem (Xenum P₀ P₁ gN c) ↔
      (gN c = 1 → ∀ e ∈ decodeList c, φ.mem (step (P₀.X e.unpair.1) (P₁.X e.unpair.2))
    ) := by
  by_cases h : gN c = 1
  · rw [Xenum_pos P₀ P₁ h, mem_stepFun_iff φ (funListOf_valid P₀ P₁ (decodeList c))]
    constructor
    · intro hp _ e he
      exact hp (funPair P₀ P₁ e) (List.mem_map.mpr ⟨e, he, rfl⟩)
    · intro himp p hp
      rw [funListOf, List.mem_map] at hp
      obtain ⟨e, he, rfl⟩ := hp
      exact himp h e he
  · rw [Xenum_neg P₀ P₁ h]
    constructor
    · intro _ hcon; exact absurd hcon h
    · intro _; rw [← funSpace_master]; exact φ.master_mem

theorem Xenum_isComputableElement_iff (gN : ℕ → ℕ) (hgNp : Nat.Primrec gN)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
      : Set (ApproximableMap V₀ V₁)).Nonempty) (φ : (funSpace V₀ V₁).Element) :
    REPred (fun c => φ.mem (Xenum P₀ P₁ gN c)) ↔ ComputableMap P₀ P₁ (toApproxMap φ) :=
    by

  have hbridge := mem_Xenum_iff P₀ P₁ gN φ
  constructor
  ·
    intro helem
    have hc0p : Nat.Primrec (fun s => Nat.pair s 0 + 1) :=
      Nat.Primrec.succ.comp (primrec_id.pair (Nat.Primrec.const 0))
    have key : REPred (fun s => φ.mem (Xenum P₀ P₁ gN (Nat.pair s 0 + 1))) := helem.
    comp hc0p
    constructor
```

```
    show REPred (fun s => φ.mem (step (P₀.X s.unpair.1) (P₁.X s.unpair.2)))
    refine REPred.of_iff (fun s => ?_) key
    have hdec : decodeList (Nat.pair s 0 + 1) = [s] := by
      rw [decodeList_succ, unpair_pair_fst, unpair_pair_snd, decodeList_zero]
    have hcons : gN (Nat.pair s 0 + 1) = 1 := by
      rw [hgN, hdec]
      show (stepFun [(P₀.X s.unpair.1, P₁.X s.unpair.2)]
        : Set (ApproximableMap V₀ V₁)).Nonempty
      rw [stepFun_singleton]
      exact (step_mem (P₀.mem_X _) (P₁.mem_X _)).2
    rw [hbridge (Nat.pair s 0 + 1), hdec]
    constructor
    · intro hRs _ e he; rw [List.mem_singleton] at he; subst he; exact hRs
    · intro h; exact h hcons s (List.mem_singleton.mpr rfl)
  .
    intro hmap
    have hR' : REPred (fun s => φ.mem (step (P₀.X s.unpair.1) (P₁.X s.unpair.2))) :=
    hmap.out
    have hforall : REPred (fun c => ∀ e ∈ decodeList c,
        φ.mem (step (P₀.X e.unpair.1) (P₁.X e.unpair.2))) := hR'.forall_mem_decodeList
    have hne1 : REPred (fun c => ¬ gN c = 1) :=
      ((RecDecidable.natEq hgNp (Nat.Primrec.const 1)).not).re
    refine REPred.of_iff (fun c => ?_) (hne1.or hforall)
    rw [hbridge c]; exact Decidable.imp_iff_not_or

theorem isComputableElement_funPresentation_iff (gN incl0 incl1 eq1 : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
      : Set (ApproximableMap V₀ V₁)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : ∀ s, incl0 s = 1 ↔ P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) (hincl0p : Nat.
    Primrec incl0)
    (hincl1 : ∀ s, incl1 s = 1 ↔ P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) (hincl1p : Nat.
    Primrec incl1)
    (heq1 : ∀ s, eq1 s = 1 ↔ P₁.X s.unpair.1 = P₁.X s.unpair.2) (heq1p : Nat.Primrec
    eq1)
    (φ : (funSpace V₀ V₁).Element) :
    ComputableElement (funPresentation P₀ P₁ gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p
        hincl1 hincl1p heq1 heq1p) φ ↔ ComputableMap P₀ P₁ (toApproxMap φ) :=
  ⟨fun h => (Xenum_isComputableElement_iff P₀ P₁ gN hgNp hgN φ).mp h.out,
    fun h => ⟨(Xenum_isComputableElement_iff P₀ P₁ gN hgNp hgN φ).mpr h⟩⟩

theorem mem_Xenum_iff_map (gN : ℕ → ℕ) (f : ApproximableMap V₀ V₁) (c : ℕ) :
    f ∈ Xenum P₀ P₁ gN c ↔
      (gN c = 1 → ∀ e ∈ decodeList c, f.rel (P₀.X e.unpair.1) (P₁.X e.unpair.2)) := by
  by_cases h : gN c = 1
  · rw [Xenum_pos P₀ P₁ h, mem_stepFun]
    constructor
    · intro hp _ e he
      exact hp (funPair P₀ P₁ e) (List.mem_map.mpr ⟨e, he, rfl⟩)
    · intro himp p hp
      rw [funListOf, List.mem_map] at hp
      obtain ⟨e, he, rfl⟩ := hp
      exact himp h e he
  · rw [Xenum_neg P₀ P₁ h]
    constructor
    · intro _ hcon; exact absurd hcon h
    · intro _; exact Set.mem_univ f
```

```
theorem curry_rel_Xenum_iff {V₂ : NeighborhoodSystem ?} (P₂ : ComputablePresentation V₂
    )
    (gN : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₁ P₂ (decodeList c))
      : Set (ApproximableMap V₁ V₂)).Nonempty)
    (g : ApproximableMap (prod V₀ V₁) V₂) (n c : ℕ) :
    (curry g).rel (P₀.X n) (Xenum P₁ P₂ gN c) ↔
      (gN c = 1 → ∀ e ∈ decodeList c,
        g.rel (prodNbhd (P₀.X n) (P₁.X e.unpair.1)) (P₂.X e.unpair.2)) := by
  rw [curry_rel]
  constructor
  · rintro ⟨hX, _, hmem⟩
    rw [mem_Xenum_iff_map P₁ P₂ gN (gSection g hX) c] at hmem
    intro hc e he
    have hrel := hmem hc e he
    rwa [gSection_rel] at hrel
  · intro himp
    refine ⟨P₀.mem_X n, Xenum_mem P₁ P₂ gN hgN c, ?_⟩
    rw [mem_Xenum_iff_map P₁ P₂ gN (gSection g (P₀.mem_X n)) c]
    intro hc e he
    rw [gSection_rel]
    exact himp hc e he

theorem curry_isComputable {V₂ : NeighborhoodSystem ?} (P₂ : ComputablePresentation V₂)
    (gN incl0 incl1 eq1 : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₁ P₂ (decodeList c))
      : Set (ApproximableMap V₁ V₂)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : ∀ s, incl0 s = 1 ↔ P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) (hincl0p : Nat.
    Primrec incl0)
    (hincl1 : ∀ s, incl1 s = 1 ↔ P₂.X s.unpair.1 ⊆ P₂.X s.unpair.2) (hincl1p : Nat.
    Primrec incl1)
    (heq1 : ∀ s, eq1 s = 1 ↔ P₂.X s.unpair.1 = P₂.X s.unpair.2) (heq1p : Nat.Primrec
    eq1)
    {g : ApproximableMap (prod V₀ V₁) V₂}
    (hg : ComputableMap (prodPresentation P₀ P₁) P₂ g) :
    ComputableMap P₀ (funPresentation P₁ P₂ gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p
      hincl1 hincl1p heq1 heq1p) (curry g) := by
  constructor
  have hg' : REPred (fun s => g.rel ((prodPresentation P₀ P₁).X s.unpair.1) (P₂.X s.
    unpair.2)) := hg.out
  have hre : Nat.Primrec (fun s => Nat.pair (Nat.pair s.unpair.1 s.unpair.2.unpair.1)
      s.unpair.2.unpair.2) :=
    (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right)).pair
      (Nat.Primrec.right.comp Nat.Primrec.right)

  have hS : REPred₂ (fun n e =>
      g.rel (prodNbhd (P₀.X n) (P₁.X e.unpair.1)) (P₂.X e.unpair.2)) := by
    refine REPred.of_iff (fun s => ?_) (hg'.comp hre)
    simp only [prodPresentation_X, unpair_pair_fst, unpair_pair_snd]

  have hforall : REPred (fun t => ∀ e ∈ decodeList t.unpair.2,
      g.rel (prodNbhd (P₀.X t.unpair.1) (P₁.X e.unpair.1)) (P₂.X e.unpair.2)) :=
    REPred.forall_mem_decodeList₂ hS
  have hne1 : REPred (fun t => ¬ gN t.unpair.2 = 1) :=
    ((RecDecidable.natEq (hgNp.comp Nat.Primrec.right) (Nat.Primrec.const 1)).not).re
```

```
  refine REPred.of_iff (fun t => ?_) (hne1.or hforall)
  show (curry g).rel (P₀.X t.unpair.1) (Xenum P₁ P₂ gN t.unpair.2) ↔ _
  rw [curry_rel_Xenum_iff P₀ P₁ P₂ gN hgN g t.unpair.1 t.unpair.2]
  exact Decidable.imp_iff_not_or

end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  {V₂ : NeighborhoodSystem ?}

def curryStepCode (a e' : ℕ) : ℕ :=
  Nat.pair (Nat.pair (Nat.pair a e'.unpair.1) e'.unpair.2) 0 + 1

theorem curryIso_apply (x : (funSpace (prod V₀ V₁) V₂).Element) :
    curryIso V₀ V₁ V₂ x = toFilter (curry (toApproxMap x)) := rfl

theorem primrec_curryStepCode {a e : ℕ → ℕ} (ha : Nat.Primrec a) (he : Nat.Primrec e)
    :
    Nat.Primrec (fun w => curryStepCode (a w) (e w)) := by
  have h1 : Nat.Primrec (fun w => (e w).unpair.1) := Nat.Primrec.left.comp he
  have h2 : Nat.Primrec (fun w => (e w).unpair.2) := Nat.Primrec.right.comp he
  refine (Nat.Primrec.succ.comp (((ha.pair h1).pair h2).pair (Nat.Primrec.const 0))).
    of_eq
    (fun w => ?_)
  rfl

end
end DomainTheory.NeighbourhoodSystems
```

4.8.13 Theorem 7.5 — function space effective

THEOREM 7.5. If $\mathcal{D}_0$ and $\mathcal{D}_1$ are effectively given, then so is $(\mathcal{D}_0 \to \mathcal{D}_1)$. The combinators eval and curry are computable, provided all the domains involved are effectively given. The computable elements $f \in |\mathcal{D}_0 \to \mathcal{D}_1|$ are exactly the computable maps $f : \mathcal{D}_0 \to \mathcal{D}_1$.

Proof: The proofs of 3.9, 3.11, and 3.12 were set up with this theorem in mind. If

$$\mathcal{D}_0 = \{X_n \mid n \in \mathbf{N}\} \quad \text{and} \quad \mathcal{D}_1 = \{Y_m \mid m \in \mathbf{N}\}$$

are two effectively given neighbourhood systems, then the neighbourhoods of $(\mathcal{D}_0 \to \mathcal{D}_1)$, by Definition 3.8, are non-empty intersections like

$$\bigcap_{i<q} [X_{n_i}, Y_{m_i}],$$

where $\langle n_0, n_1, \ldots, n_{q-1}\rangle$ and $\langle m_0, m_1, \ldots, m_{q-1}\rangle$ are two finite sequences of integers determining the choice of the function-space neighbourhood. In 3.9(i) the test for nonemptiness is given. Assuming the decidability of relations in $\mathcal{D}_0$ and $\mathcal{D}_1$, one remarks that the consistency of *finite sequences* of neighbourhoods is also decidable. (Hint: Test the first *two*, then form their intersection. Next test the third given neighbourhood against this one set; if consistent, form the intersection, and carry on.) By 3.9(i) at most $2 \cdot 2^q$ such sequential checks must be carried out to determine whether the function-space neighbourhood is non empty.

It may not be fun, but the checks can be carried out in finite time. Owing to this decidability, we can therefore enumerate in a systematic way *all* the pairs of finite sequences $\langle n_0, \ldots\rangle$ and $\langle m_0, \ldots\rangle$ that determine neighbourhoods: that is the way that $(\mathcal{D}_0 \to \mathcal{D}_1)$ obtains its enumeration.

Concerning the decidability of the required relations on $(\mathcal{D}_0 \to \mathcal{D}_1)$, we remark first off that consistency is more of the same: to test two finite intersections against each other, just form one big intersection and test it for non-emptiness as before. Secondly, the testing for intersection comes down in the end to testing one typical intersection of $[X, Y]$-neighbourhoods for equality with another. But equality amounts to two inclusions; inclusion in an intersection amounts to inclusion in each term. Therefore, what we need to do is to check a finite number of statements of the form:

$$\bigcap_{i<q}[X_{n_i}, Y_{m_i}] \subseteq [X_k, Y_\ell].$$

As we pointed out after the proof of 3.9, this inclusion is equivalent to

$$\bigcap\{Y_{m_i} \mid X_k \subseteq X_{n_i}\} \subseteq Y_\ell.$$

By decidability in $\mathcal{D}_0$, we can effectively find the n_i that are needed. Then in $\mathcal{D}_1$, we form the intersection of the corresponding Y_{m_i}. Finally, we check the inclusion. Again, one check in $(\mathcal{D}_0 \to \mathcal{D}_1)$ requires a whole sequence of checks in $\mathcal{D}_0$ and in $\mathcal{D}_1$, but the process is finite. So we have argued that $(\mathcal{D}_0 \to \mathcal{D}_1)$ is effectively given.

In showing that the combinators are computable, we refer first to the proof of 3.11. The typical pair of neighbourhoods possibly belonging to eval is

$$\bigcap_{i<q}[X_{n_i}, Y_{m_i}],\ X_k \text{ eval } Y_\ell.$$

As we needed not to be so specific, we expressed the holding of this relationship in terms of *all* the functions in the function-

space neighbourhood. But we know that the neighbourhood, by 3.9(ii), has a minimal element; it is then sufficient to test for the holding of $X_k f_0 Y_\ell$ at this minimal function f_0. But this test, we have already seen, is decidable. So the pairs in eval actually form a recursive set, not just a recursively enumerable set; thus, eval is a computable function.

The case of curry involves three domains and is a bit more messy. But again, if the required neighbourhoods are written out in full, it will be seen that curry, too, is computable. We leave this minor struggle to the exercises.

The final statement is an easy consequence of the fundamental connection between approximable $f : \mathcal{D}_0 \to \mathcal{D}_1$ as relations and as elements. Recall, as in the proof of 3.10, that we have

$$f \in [X, Y] \text{ iff } X f Y,$$

for all $X \in \mathcal{D}_0$ and $Y \in \mathcal{D}_1$. Therefore,

$$f \in \bigcap_{i<q}[X_{n_i}, Y_{m_i}] \text{ iff } \forall i < q.\ X_{n_i} f Y_{m_i}.$$

It follows that if f is recursively enumerable as a set of pairs, then, by forming all the non-empty intersections (as shown), we get an enumeration of all the neighbourhoods to which f belongs; and this is

the same as the filter corresponding to f as an element of the function space. The converse direction is clear. □

We have nearly all our favourite combinators computable, but perhaps the most important one - since it is the key to recursive definitions - is the fixed-point combinator. It is not left out.

The development is with the function-space presentation peer.

Theorem (`theorem_75`). If two neighbourhood systems are effectively given, then so is their function space. Evaluation and currying are computable, and the computable elements of the function space are exactly the computable maps.

Proof. By the function-space presentation. ■

Theorem_7_5.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .funPresentation

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap

/-- Scott's Theorem 7.5: function space effective givenness and computability. -/
theorem theorem_75 :
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁) (gN : ℕ → ℕ),
      Nat.Primrec gN →
        (∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
          : Set (ApproximableMap V₀ V₁)).Nonempty) →
          ∀ (φ : (funSpace V₀ V₁).Element),
            REPred (fun c => φ.mem (Xenum P₀ P₁ gN c)) ↔
              ComputableMap P₀ P₁ (toApproxMap φ)) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β},
      V₀.IsEffectivelyGiven → V₁.IsEffectivelyGiven → (funSpace V₀ V₁).
    IsEffectivelyGiven) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)
      {V₂ : NeighborhoodSystem ?} (P₂ : ComputablePresentation V₂)
      (gN incl0 incl1 eq1 : ℕ → ℕ)
      (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₁ P₂ (decodeList c))
        : Set (ApproximableMap V₁ V₂)).Nonempty) (hgNp : Nat.Primrec gN)
      (hincl0 : ∀ s, incl0 s = 1 ↔ P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) (hincl0p : Nat.
    Primrec incl0)
      (hincl1 : ∀ s, incl1 s = 1 ↔ P₂.X s.unpair.1 ⊆ P₂.X s.unpair.2) (hincl1p : Nat.
    Primrec incl1)
      (heq1 : ∀ s, eq1 s = 1 ↔ P₂.X s.unpair.1 = P₂.X s.unpair.2) (heq1p : Nat.Primrec
     eq1)
      {g : ApproximableMap (prod V₀ V₁) V₂},
      ComputableMap (prodPresentation P₀ P₁) P₂ g →
        ComputableMap P₀ (funPresentation P₁ P₂ gN incl0 incl1 eq1 hgN hgNp hincl0
    hincl0p
          hincl1 hincl1p heq1 heq1p) (curry g)) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)
```

```
      (gN incl0 incl1 eq1 : ℕ → ℕ)
      (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
        : Set (ApproximableMap V₀ V₁)).Nonempty) (hgNp : Nat.Primrec gN)
      (hincl0 : ∀ s, incl0 s = 1 ↔ P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) (hincl0p : Nat.
    Primrec incl0)
      (hincl1 : ∀ s, incl1 s = 1 ↔ P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) (hincl1p : Nat.
    Primrec incl1)
      (heq1 : ∀ s, eq1 s = 1 ↔ P₁.X s.unpair.1 = P₁.X s.unpair.2) (heq1p : Nat.Primrec
     eq1),
      ComputableMap
        (prodPresentation (funPresentation P₀ P₁ gN incl0 incl1 eq1 hgN hgNp hincl0
    hincl0p
          hincl1 hincl1p heq1 heq1p) P₀) P₁ (evalMap V₀ V₁)) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)
      (gN incl0 incl1 eq1 : ℕ → ℕ)
      (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P₀ P₁ (decodeList c))
        : Set (ApproximableMap V₀ V₁)).Nonempty) (hgNp : Nat.Primrec gN)
      (hincl0 : ∀ s, incl0 s = 1 ↔ P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) (hincl0p : Nat.
    Primrec incl0)
      (hincl1 : ∀ s, incl1 s = 1 ↔ P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) (hincl1p : Nat.
    Primrec incl1)
      (heq1 : ∀ s, eq1 s = 1 ↔ P₁.X s.unpair.1 = P₁.X s.unpair.2) (heq1p : Nat.Primrec
     eq1)
      (φ : (funSpace V₀ V₁).Element),
      ComputableElement (funPresentation P₀ P₁ gN incl0 incl1 eq1 hgN hgNp hincl0
    hincl0p
          hincl1 hincl1p heq1 heq1p) φ ↔ ComputableMap P₀ P₁ (toApproxMap φ)) :=
  ⟨@Xenum_isComputableElement_iff, ⟨@funSpace_isEffectivelyGiven, ⟨@curry_isComputable,
    ⟨@evalMap_isComputable, @isComputableElement_funPresentation_iff⟩⟩⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.8.14 Computable fixpoint

The fixpoint combinator on an effectively given domain is a computable map.

Theorem (`fixMap_isComputable`). The least-fixpoint approximable map on an effectively presented function space is computable.

Proof. The required relation is obtained from recursively enumerable or decidable index relations by pairing, composition, conjunction, and projection. ■

`fixComputable.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_24
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
```

```
open NeighborhoodSystem Domain.Recursive ApproximableMap
section
variable {α : Type*} {V : NeighborhoodSystem α}
variable (P : ComputablePresentation V)

theorem fixMap_isComputable
    (gN incl eq : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P P (decodeList c))
      : Set (ApproximableMap V V)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl : ∀ s, incl s = 1 ↔ P.X s.unpair.1 ⊆ P.X s.unpair.2) (hinclp : Nat.Primrec
     incl)
    (heq : ∀ s, eq s = 1 ↔ P.X s.unpair.1 = P.X s.unpair.2) (heqp : Nat.Primrec eq) :
    ComputableMap
      (funPresentation P P gN incl incl eq hgN hgNp hincl hinclp hincl hinclp heq heqp
    ) P
      (fixMap V) := by
  constructor
  obtain ⟨fincl, hfinclp, hfincls⟩ :=
    (funPresentation P P gN incl incl eq hgN hgNp hincl hinclp hincl hinclp heq heqp).
    incl_computable
  have hfincl : ∀ s, fincl s = 1 ↔ Xenum P P gN s.unpair.1 ⊆ Xenum P P gN s.unpair.2
    :=
    fun s => (hfincls s).symm
  show REPred₂ (fun c ? => (fixMap V).rel (Xenum P P gN c) (P.X ?))
  refine ⟨fun w =>
      (fixChainChar P fincl w.unpair.2.unpair.1 w.unpair.1).unpair.2 = 1
      ∧ incl (Nat.pair (fixChainChar P fincl w.unpair.2.unpair.1 w.unpair.1).unpair.1
          w.unpair.2.unpair.2) = 1,
    ?_, ?_⟩
  ·
    have hcc : Nat.Primrec (fun w => fixChainChar P fincl w.unpair.2.unpair.1 w.unpair
    .1) :=
      ((primrec_fixChainChar P fincl hfinclp).comp
        ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left)).of_eq (fun
    w => by
        simp only [unpair_pair_fst, unpair_pair_snd])
    have hA : Nat.Primrec (fun w =>
        (fixChainChar P fincl w.unpair.2.unpair.1 w.unpair.1).unpair.2) :=
      Nat.Primrec.right.comp hcc
    have hB : Nat.Primrec (fun w => incl (Nat.pair
        (fixChainChar P fincl w.unpair.2.unpair.1 w.unpair.1).unpair.1 w.unpair.2.
    unpair.2)) :=
      hinclp.comp ((Nat.Primrec.left.comp hcc).pair (Nat.Primrec.right.comp Nat.
    Primrec.right))
    exact (RecDecidable.natEq hA (Nat.Primrec.const 1)).and
      (RecDecidable.natEq hB (Nat.Primrec.const 1))
  ·
    intro t
    show (fixMap V).rel (Xenum P P gN t.unpair.1) (P.X t.unpair.2) ↔ _
    rw [fixMap_rel_iff P gN hgN t.unpair.1 t.unpair.2]
    constructor
    · rintro ⟨full, hsteps, hlast⟩
      refine ⟨encodeList full, ?_⟩
      simp only [unpair_pair_fst, unpair_pair_snd]
      have hdec : decodeList (encodeList full) = full := decodeList_encodeList full
      obtain ⟨hfst, hsnd⟩ := fixChainChar_spec P fincl t.unpair.1 (encodeList full)
```

```
      rw [hdec] at hfst hsnd
      refine ⟨hsnd.mpr
        ((chainDec_iff_gStepsOK P gN hgN fincl hfincl t.unpair.1 full P.masterIdx).mpr
     hsteps), ?_⟩
      rw [hfst, hincl, unpair_pair_fst, unpair_pair_snd]
      exact hlast
    · rintro ⟨i, hi⟩
      simp only [unpair_pair_fst, unpair_pair_snd] at hi
      obtain ⟨hi1, hi2⟩ := hi
      obtain ⟨hfst, hsnd⟩ := fixChainChar_spec P fincl t.unpair.1 i
      refine ⟨decodeList i, ?_, ?_⟩
      · exact (chainDec_iff_gStepsOK P gN hgN fincl hfincl t.unpair.1 (decodeList i)
          P.masterIdx).mp (hsnd.mp hi1)
      · rw [← hfst]
        have hi2' := (hincl (Nat.pair (fixChainChar P fincl t.unpair.1 i).unpair.1
          t.unpair.2)).mp hi2
        rwa [unpair_pair_fst, unpair_pair_snd] at hi2'
end

end DomainTheory.NeighbourhoodSystems
```

4.8.15 Theorem 7.6 — fix is computable

THEOREM 7.6. For any effectively given domain $\mathcal{D}$, the combinator fix : $(\mathcal{D} \to \mathcal{D}) \to \mathcal{D}$ is computable.

Proof : Referring back to the proof of Theorem 4.2 and thinking of

$$\mathcal{D} = \{X_n \mid n \in \mathbb{N}\}$$

as effectively given, fix as a relation comes down to

$$\bigcap_{i<q} [X_{n_i}, X_{m_i}] \text{ fix } X_\ell \text{ iff for some finite sequence}$$

$$\Delta = X_{k_0}, \ldots, X_{k_p} = X_\ell,$$

we have, for each $j < p$,

$$\bigcap \{X_{m_i} \mid X_{k_j} \subseteq X_{n_i}\} \subseteq X_{k_{j+1}}.$$

Inside the "for some finite sequence" all the checks are decidable by assumption on D. But the existential quantification of a decidable predicate always gives a recursively enumerable predicate. (And, as there is no implied bound on the size of the finite sequence we are looking for, this really *is* an enumerable set and not generally a recursive set.) □

The major consequence of what we have done up to this point concerns typed λ-calculus. Any expression involving only *effectively given types* and, perhaps, some *basic computable constants* using only the $\lambda, !$-notation defines a computable function of its free variables. And such functions applied to computable arguments give computable values. And such functions have computable least fixed points. Etc., etc. In a definite sense then we have in the "metalanguage", as people say, a quite precise and fully *mathematical programming language* for defining computable operators. It is not a machine implemented language, but it is a mathematically well-defined and easy-to-use language. And when we combine the usual type-definition facility together with *domain equations*, we have an especially powerful language.

The development is with the computable fixpoint peer.

Theorem (`theorem_76`). On an effectively given domain, the fixpoint combinator is computable.

Proof. By the computable fixpoint map. ■

`Theorem_7_6.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .fixComputable

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap

/-- Scott's Theorem 7.6: the fixpoint map is computable. -/
theorem theorem_76 :
    ∀ {α : Type*} {V : NeighborhoodSystem α} (P : ComputablePresentation V)
      (gN incl eq : ℕ → ℕ)
      (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P P (decodeList c))
        : Set (ApproximableMap V V)).Nonempty) (hgNp : Nat.Primrec gN)
      (hincl : ∀ s, incl s = 1 ↔ P.X s.unpair.1 ⊆ P.X s.unpair.2) (hinclp : Nat.
    Primrec incl)
      (heq : ∀ s, eq s = 1 ↔ P.X s.unpair.1 = P.X s.unpair.2) (heqp : Nat.Primrec eq),
      ComputableMap
        (funPresentation P P gN incl incl eq hgN hgNp hincl hinclp hincl hinclp heq
    heqp) P
        (fixMap V) :=
  @fixMap_isComputable

end DomainTheory.NeighbourhoodSystems
```

4.8.16 Effectively given lists

The list domain over an effectively given neighbourhood system is effectively given.

Definition (`Proposition77.consB`). The consistency component of a packed triple is the first coordinate of its second coordinate.

Definition (`Proposition77.eqB`). The equality component of a packed triple is its first coordinate.

Definition (`Proposition77.intI`). The intersection-index component of a packed triple is the second coordinate of its second coordinate.

Theorem (`Proposition77.le_pair_left`). The left coordinate of a paired natural number does not exceed the pair code.

Proof. This follows directly from the quadratic pairing formula. ■

Definition (`Proposition77.listGetStp`). The list lookup step decrements a counter and, exactly when the counter is one, replaces the stored value by the current list entry.

Definition (`Proposition77.listGet`). The coded-list lookup function folds the lookup step through a coded list and returns the stored value, using zero as the default.

Theorem (`Proposition77.listGetStp_pair`). On a state with counter c, value v, and current entry x, the lookup step returns counter $c - 1$ and selects x precisely when $c = 1$.

Proof. Unpack the state and apply the definition of the step. ■

Theorem (`Proposition77.listGet_foldl_zero`). Folding the lookup step from counter zero leaves the stored value unchanged.

Proof. Induct on the list; a zero counter never selects a list entry. ■

Theorem (`Proposition77.listGet_foldl`). Folding from counter $c + 1$ returns the list entry at position c, or the supplied default if that position is absent.

Proof. Induct on the list, separating the zero and successor cases of c. ■

Theorem (`Proposition77.listGet_encodeList`). Looking up index i in an encoded list returns its i-th entry, with default zero.

Proof. Coded folding agrees with list folding, to which the preceding lookup identity applies. ■

Theorem (`Proposition77.max_sq_le_pair`). The square of the larger of two natural numbers is at most their pair code.

Proof. Expand the pairing formula and separate the two possible coordinate orders. ■

Theorem (`Proposition77.memS_sub_embPair`). A list neighbourhood contained in an embedded pair is itself an embedded pair of list neighbourhoods.

Proof. The master and zero-embedded forms contain paths incompatible with a pair; only the paired constructor remains. ■

Theorem (`Proposition77.memS_sub_embZero`). A list neighbourhood contained in an embedded zero neighbourhood is itself the zero embedding of an original neighbourhood.

Proof. Nonemptiness rules out the master and paired forms by their incompatible path shapes. ■

Theorem (`Proposition77.nat_shape`). Every natural number is either zero, positive odd, or positive even.

Proof. A positive number is classified by its remainder modulo two. ■

Definition (`Proposition77.packT`). Three natural numbers are packed as a pair whose second coordinate is itself a pair.

Theorem (`Proposition77.eqB_packT`). The equality projection of a packed triple returns its first entry.

Proof. Unpair twice. ■

Theorem (`Proposition77.consB_packT`). The consistency projection of a packed triple returns its second entry.

Proof. Unpair twice. ■

Theorem (`Proposition77.intI_packT`). The intersection-index projection of a packed triple returns its third entry.

Proof. Unpair twice. ■

Definition (`Proposition77.dsharpStep`). The list-domain decision step classifies two neighbourhood indices as master, zero-embedded, or paired, and packs equality, consistency, and intersection information using supplied base-system procedures and earlier table entries.

Theorem (`Proposition77.dsharpStep_i0`). If the first index is zero, the decision step reports consistency, returns the second index as intersection, and reports equality exactly when the second index is zero.

Proof. The first master-index branch is selected. ■

Theorem (`Proposition77.dsharpStep_j0`). If the second index is zero and the first is nonzero, the decision step reports consistency, returns the first index as intersection, and reports inequality.

Proof. The second master-index branch is selected. ■

Theorem (`Proposition77.dsharpStep_ll`). For two odd indices, the decision step applies the supplied base equality, consistency, and intersection procedures to their decoded indices and repacks the

intersection as odd.

Proof. Both parity tests select the zero-embedded branch. ■

Theorem (`Proposition77.dsharpStep_ln`). For an odd first index and a positive even second index, the decision step returns three zero components.

Proof. The two neighbourhoods have incompatible zero and pair shapes. ■

Theorem (`Proposition77.dsharpStep_nl`). For a positive even first index and an odd second index, the decision step returns three zero components.

Proof. The two neighbourhoods have incompatible pair and zero shapes. ■

Theorem (`Proposition77.dsharpStep_nn`). For two positive even indices, the decision step combines the two recursively tabulated component results: equality and consistency are products, and the intersection index pairs the two component intersection indices.

Proof. Both parity tests select the paired branch and the decoded child pairs address earlier table entries. ■

Theorem (`Proposition77.pair_lt_succ_sq`). A pair code is smaller than the square of one plus its largest coordinate.

Proof. Expand the pairing formula in each coordinate order. ■

Theorem (`Proposition77.pair_lt_pair_of_lt`). Strict coordinatewise inequalities imply a strict inequality between pair codes.

Proof. Bound the smaller code by the next square and the larger code below by the square of its largest coordinate. ■

Theorem (`Proposition77.primrec_listGet`). Coded-list lookup is primitive recursive.

Proof. Its step, initial state, coded fold, and final projection are all primitive recursive. ■

Theorem (`Proposition77.primrec_listGet2`). Substituting primitive-recursive code and index functions into coded-list lookup yields a primitive-recursive function.

Proof. Pair the two arguments and compose with primitive-recursive lookup. ■

Theorem (`Proposition77.primrec_dsharpStep`). If the supplied base consistency, equality, and intersection functions are primitive recursive, then the list-domain decision step is primitive recursive.

Proof. Every branch condition, table address, projection, product, pairing, and packed result is primitive recursive, and finite numerical selection preserves this property. ■

Definition (`Proposition77.rtbl`). The recursion table at stage zero is empty; at stage $w + 1$ it prepends the result of applying the step at w to the previous table.

Definition (`Proposition77.gOf`). The value generated at stage w is the step applied to w together with the table of earlier values.

Definition (`Proposition77.gList`). The generated list at stage $w + 1$ has the stage-w value at its head followed by the list from stage w.

Theorem (`Proposition77.gOf_def`). The generated value is the step evaluated at its stage and recursion table.

Proof. This is the definition. ■

Theorem (`Proposition77.getD_gList`). For $p < w$, position p in the generated list at stage w contains the value generated at stage $w - 1 - p$.

Proof. Induct on w, separating the head position from successor positions. ■

Theorem (`Proposition77.rtbl_eq_encode`). The recursion table at stage w is the code of the generated list at that stage.

Proof. Induct on w; both constructions prepend the newly generated value. ■

Theorem (`Proposition77.listGet_rtbl`). If $v < w$, lookup at reverse position $w - 1 - v$ in the stage-w table returns the value generated at stage v.

Proof. Replace the table by its encoded generated list and use the reverse-position formula. ■

Theorem (`Proposition77.rtbl_eq_rec`). The recursion table agrees with ordinary natural-number recursion that prepends each new step result.

Proof. Induct on the stage. ■

Theorem (`Proposition77.primrec_rtbl`). A primitive-recursive step has a primitive-recursive recursion table.

Proof. The table is obtained by primitive recursion with primitive-recursive pairing and successor operations. ■

Theorem (`Proposition77.primrec_gOf`). A primitive-recursive step has a primitive-recursive generated-value function.

Proof. Pair the stage with its primitive-recursive table and apply the step. ■

Theorem (`Proposition77.unpair_fst_le`). The first coordinate decoded from a natural number does not exceed that number.

Proof. The number is the pair of its decoded coordinates, and the left coordinate is bounded by its pair code. ■

Definition (`Proposition77.Vsharp`). The list neighbourhood at index zero is the master neighbourhood; positive odd indices encode zero embeddings of base neighbourhoods, and positive even indices encode pairs of recursively indexed list neighbourhoods.

Theorem (`Proposition77.Vsharp_succ`). The successor case of the enumeration is the odd zero-embedding branch or the even paired branch according to parity.

Proof. This is the recursive defining equation. ■

Theorem (`Proposition77.Vsharp_even`). The neighbourhood indexed by $2n + 2$ is the embedded pair of the neighbourhoods at the two coordinates decoded from n.

Proof. The index is positive even, and its predecessor decodes back to n. ■

Theorem (`Proposition77.Vsharp_even_inter_even`). The intersection of two even-indexed neighbourhoods is the embedded pair of the intersections of their corresponding components.

Proof. Expand both even indices and distribute intersection through embedded pairing. ■

Theorem (`Proposition77.Vsharp_odd`). The neighbourhood indexed by $2n + 1$ is the zero embedding of the base neighbourhood indexed by n.

Proof. The index is positive odd. ■

Theorem (`Proposition77.Vsharp_odd_inter_even`). An odd-indexed neighbourhood and an even-indexed neighbourhood have empty intersection.

Proof. Zero embeddings and pair embeddings have disjoint path shapes. ■

Theorem (`Proposition77.Vsharp_even_inter_odd`). An even-indexed neighbourhood and an odd-indexed neighbourhood have empty intersection.

Proof. Commute the intersection and use the odd-even case. ■

Theorem (`Proposition77.Vsharp_odd_inter_odd`). The intersection of two odd-indexed neighbourhoods is the zero embedding of the intersection of their base neighbourhoods.

Proof. Expand both odd indices and distribute intersection through zero embedding. ■

Theorem (`Proposition77.Vsharp_zero`). Index zero enumerates the master list neighbourhood.

Proof. This is the base case of the enumeration. ■

Theorem (`Proposition77.Vsharp_eq_Gamma_iff`). When all base neighbourhoods are nonempty, an enumerated list neighbourhood is master exactly when its index is zero.

Proof. Odd and positive even indices have zero or pair shape, each distinct from the master shape. ■

Theorem (`Proposition77.Vsharp_mem`). Every enumerated $V^\sharp$ family is a list neighbourhood.

Proof. Strong induction follows the zero, odd zero-embedding, and even paired clauses of the enumeration. ■

Theorem (`Proposition77.Vsharp_inter_zero`). Intersecting any enumerated neighbourhood on the right with index zero leaves it unchanged.

Proof. Index zero is master and every list neighbourhood lies within the master neighbourhood. ■

Theorem (`Proposition77.Vsharp_nonempty`). If every base neighbourhood is nonempty, every enumerated list neighbourhood is nonempty.

Proof. Every list neighbourhood built from a nonempty base system is nonempty. ■

Theorem (`Proposition77.Vsharp_surj`). Every list neighbourhood occurs in the $V^\sharp$ enumeration.

Proof. Induct over its master, zero-embedded, and paired construction, using surjectivity of the base presentation and pairing of the recursive indices. ■

Theorem (`Proposition77.Vsharp_zero_inter`). Intersecting index zero on the left with any enumerated neighbourhood leaves it unchanged.

Proof. Index zero is master. ■

Theorem (`Proposition77.exists_sub_embPair_iff`). An enumerated list neighbourhood lies inside an embedded pair of A, B exactly when some enumerated neighbourhood lies inside A and some enumerated neighbourhood lies inside B.

Proof. Any list neighbourhood inside a pair has paired components, and surjectivity enumerates them; conversely pair two witnessing neighbourhoods. ■

Theorem (`Proposition77.exists_sub_embZero_iff`). An enumerated list neighbourhood lies inside the zero embedding of Y exactly when some enumerated base neighbourhood lies inside Y.

Proof. Any list neighbourhood inside a zero embedding has zero shape, and base-presentation surjectivity identifies its generator; the converse uses the corresponding odd index. ■

Theorem (`Proposition77.dsharp_decider_spec`). The generated packed decision value correctly reports consistency, supplies an intersection index when consistent, and reports equality for every pair of enumerated list neighbourhoods.

Proof. Strong induction on the pair code follows the master, odd, and even shape cases; paired inputs reduce to two strictly smaller component pairs, while mixed shapes are disjoint. ■

Theorem (`Proposition77.dsharp_intI_correct`). Whenever two enumerated list neighbourhoods are consistent, the generated intersection component indexes their intersection.

Proof. The same strong induction on shape uses the supplied base intersection specification and recursively combines paired components. ■

Theorem (`Proposition77.dsharp_interEq_iff`). Equality of an intersection $V_n^\sharp \cap V_m^\sharp$ with $V_k^\sharp$ is equivalent to the product of the generated consistency bit and the equality bit comparing the generated intersection index with k being one.

Proof. Consistency supplies the correct intersection index, after which the generated equality decision tests it against k. ■

Definition (`Proposition77.dsharpPresentation`). The effective presentation of the list system uses $V^\sharp$, the generated decision procedure for equality and consistency, the generated intersection index, and zero as master index.

Proof. The enumeration is surjective, all decision functions are primitive recursive, the intersection component is correct, and index zero denotes the master neighbourhood. ■

Theorem (`Proposition77.dsharp_isEffectivelyGiven`). If the base system has a computable presentation and all its neighbourhoods are nonempty, then its list system is effectively given.

Proof. Use the constructed list-system computable presentation. ■

Theorem (`Proposition77.dsharpPresentation_X`). The neighbourhood enumerated at n by the list-system presentation is $V^{\sharp}_{n}$.

Proof. This is the presentation's enumeration. ■

Definition (`Proposition77.inSharpMap`). The list insertion relation sends a base neighbourhood X to list neighbourhoods containing its zero embedding.

Proof. The relation contains the master case, is closed under target intersections, and is monotone. ■

Theorem (`Proposition77.inSharpMap_rel`). Insertion relates X to W exactly when both are valid neighbourhoods and the zero embedding of X is contained in W.

Proof. This is the defining relation. ■

Definition (`Proposition77.proj0Map`). The first projection relation sends the master target universally and sends a paired source to targets containing its left component.

Proof. Component uniqueness gives intersection closure, and refinement of a paired source refines its left component. ■

Theorem (`Proposition77.proj0Map_rel`). The first projection relates W to Z exactly when both are list neighbourhoods and either Z is master or W is a pair whose left component lies inside Z.

Proof. This is the defining relation. ■

Theorem (`Proposition77.inSharpMap_toElementMap`). On elements, the insertion approximable map is the canonical zero-branch insertion into the list domain.

Proof. A list neighbourhood containing a zero-embedded source neighbourhood is either master or a zero embedding of a neighbourhood of the source element. ■

Theorem (`Proposition77.inSharp_isComputable`). The insertion approximable map is computable.

Proof. At presentation indices its relation is decidable inclusion from the corresponding odd-indexed zero embedding. ■

Theorem (`Proposition77.proj0_rel_Vsharp_iff`). The first projection relates indices m, k exactly when $k = 0$, or m is nonzero even and the decoded left component of m is contained in k.

Proof. Separate the master, odd zero-embedded, and even paired forms of the source. ■

Theorem (`Proposition77.proj0_isComputable`). The first projection approximable map is computable.

Proof. Its indexed criterion combines decidable zero, parity, and nonzero tests with decidable inclusion of the decoded left component. ■

Theorem (`Proposition77.proj0_toElementMap_pairSharp`). Applying the first projection to a paired list-domain element returns its first component.

Proof. Target membership is equivalent to containing a left-component neighbourhood of the pair. ■

Theorem (`Proposition77.gList_length`). The generated list at stage w has length w.

Proof. Induct on w; each successor stage prepends one value. ■

`dsharpEffective.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.indefProduct
```

```
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .computableMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (P : ComputablePresentation D)

def consB (r : ℕ) : ℕ := r.unpair.2.unpair.1

def eqB (r : ℕ) : ℕ := r.unpair.1

def intI (r : ℕ) : ℕ := r.unpair.2.unpair.2

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

theorem le_pair_left (a b : ℕ) : a ≤ Nat.pair a b := by
  unfold Nat.pair; split <;> omega

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (P : ComputablePresentation D)

def listGetStp (t : ℕ) : ℕ :=
  Nat.pair (t.unpair.2.unpair.1.unpair.1 - 1)
    (selectFn (isOne t.unpair.2.unpair.1.unpair.1) t.unpair.1 t.unpair.2.unpair.1.
    unpair.2)

def listGet (c i : ℕ) : ℕ :=
  (foldCode listGetStp 0 (Nat.pair (i + 1) 0) c).unpair.2

theorem listGetStp_pair (cd val x : ℕ) :
    listGetStp (Nat.pair x (Nat.pair (Nat.pair cd val) 0))
      = Nat.pair (cd - 1) (selectFn (isOne cd) x val) := by
  unfold listGetStp
  simp only [unpair_pair_fst, unpair_pair_snd]
```

```
theorem listGet_foldl_zero (l : List ℕ) (val : ℕ) :
    (List.foldl (fun acc x => listGetStp (Nat.pair x (Nat.pair acc 0))) (Nat.pair 0
    val) l).unpair.2
      = val := by
  induction l generalizing val with
  | nil => simp
  | cons x xs ih =>
      rw [List.foldl_cons, listGetStp_pair, Nat.zero_sub,
        show isOne 0 = 0 from rfl, selectFn_zero, ih]

theorem listGet_foldl (l : List ℕ) (c val : ℕ) :
    (List.foldl (fun acc x => listGetStp (Nat.pair x (Nat.pair acc 0)))
        (Nat.pair (c + 1) val) l).unpair.2 = l.getD c val := by
  induction l generalizing c val with
  | nil => simp
  | cons x xs ih =>
      rw [List.foldl_cons, listGetStp_pair, Nat.add_sub_cancel]
      rcases Nat.eq_zero_or_pos c with rfl | hc
      · rw [show isOne (0 + 1) = 1 from rfl, selectFn_one, listGet_foldl_zero,
          List.getD_cons_zero]
      · obtain ⟨c', rfl⟩ : ∃ c', c = c' + 1 := ⟨c - 1, by omega⟩
        rw [show isOne (c' + 1 + 1) = 0 by unfold isOne; omega, selectFn_zero, ih,
          List.getD_cons_succ]

theorem listGet_encodeList (l : List ℕ) (i : ℕ) : listGet (encodeList l) i = l.getD i
    0 := by
  unfold listGet
  rw [foldCode_eq]
  exact listGet_foldl l i 0

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

theorem max_sq_le_pair (i j : ℕ) : max i j * max i j ≤ Nat.pair i j := by
  unfold Nat.pair
  rcases lt_or_ge i j with h | h
  · rw [ite_eq_left h, show max i j = j by omega]; omega
  · rw [ite_eq_right (by omega), show max i j = i by omega]; omega

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (P : ComputablePresentation D)
```

```
theorem memS_sub_embPair (hD : ∀ X, D.mem X → X.Nonempty)
    {W A B : Set (List Bool × α)} (hW : MemS D W) (hsub : W ⊆ embPair A B) :
    ∃ Pp Qq, MemS D Pp ∧ MemS D Qq ∧ W = embPair Pp Qq := by
  cases hW with
  | gamma =>
      obtain ⟨a, ha⟩ := hD D.master D.master_mem
      rcases hsub (show (([], a) : List Bool × α) ∈ Gamma D from ha) with ⟨p', hp', -⟩
    | ⟨q', hq', -⟩
      · simp at hp'
      · simp at hq'
  | @zero Z hZ =>
      obtain ⟨a, ha⟩ := hD Z hZ
      rcases hsub (show (([], a) : List Bool × α) ∈ embZero Z from ⟨rfl, ha⟩) with
        ⟨p', hp', -⟩ | ⟨q', hq', -⟩
      · simp at hp'
      · simp at hq'
  | @pair Pp Qq hP hQ => exact ⟨Pp, Qq, hP, hQ, rfl⟩

theorem memS_sub_embZero (hD : ∀ X, D.mem X → X.Nonempty) {W : Set (List Bool × α)} {
    Y : Set α}
    (hW : MemS D W) (hsub : W ⊆ embZero Y) : ∃ Z, D.mem Z ∧ W = embZero Z := by
  cases hW with
  | gamma =>
      obtain ⟨a, ha⟩ := hD D.master D.master_mem
      exact absurd (hsub (show ((true :: [], a) : List Bool × α) ∈ Gamma D from ha)).1
     (by simp)
  | @zero Z hZ => exact ⟨Z, hZ, rfl⟩
  | @pair Pp Qq hP hQ =>
      obtain ⟨⟨pth, a⟩, ht⟩ := embPair_nonempty (memS_nonempty hD hP) (Q := Qq)
      have hmem := hsub ht
      rcases ht with ⟨p', hp', -⟩ | ⟨q', hq', -⟩
      · exact absurd (hp'.symm.trans hmem.1) (by simp)
      · exact absurd (hq'.symm.trans hmem.1) (by simp)

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

theorem nat_shape (n : ℕ) : n = 0 ∨ (∃ a, n = 2 * a + 1) ∨ (∃ a, n = 2 * a + 2) := by
  rcases Nat.eq_zero_or_pos n with rfl | hn
  · exact Or.inl rfl
  · rcases Nat.lt_or_ge (n % 2) 1 with h | h
    · exact Or.inr (Or.inr ⟨n / 2 - 1, by omega⟩)
    · exact Or.inr (Or.inl ⟨n / 2, by omega⟩)

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
```

```
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (P : ComputablePresentation D)

def packT (e c ii : ℕ) : ℕ := Nat.pair e (Nat.pair c ii)

@[simp] theorem eqB_packT (e c ii : ℕ) : eqB (packT e c ii) = e := by
  simp only [eqB, packT, unpair_pair_fst]

@[simp] theorem consB_packT (e c ii : ℕ) : consB (packT e c ii) = c := by
  simp only [consB, packT, unpair_pair_fst, unpair_pair_snd]

@[simp] theorem intI_packT (e c ii : ℕ) : intI (packT e c ii) = ii := by
  simp only [intI, packT, unpair_pair_snd]

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (P : ComputablePresentation D)

variable (fcons feq finter : ℕ → ℕ)

def dsharpStep (p : ℕ) : ℕ :=
  let i := p.unpair.1.unpair.1
  let j := p.unpair.1.unpair.2
  let tbl := p.unpair.2
  let w := p.unpair.1
  let na := i / 2 - 1
  let nb := j / 2 - 1
  let r1 := listGet tbl (w - 1 - Nat.pair na.unpair.1 nb.unpair.1)
  let r2 := listGet tbl (w - 1 - Nat.pair na.unpair.2 nb.unpair.2)
  selectFn (1 - i) (packT (1 - j) 1 j)
    (selectFn (1 - j) (packT (1 - i) 1 i)
      (selectFn (i % 2)
        (selectFn (j % 2)
          (packT (feq (Nat.pair (i / 2) (j / 2))) (fcons (Nat.pair (i / 2) (j / 2)))
            (2 * finter (Nat.pair (i / 2) (j / 2)) + 1))
          (packT 0 0 0))
        (selectFn (j % 2) (packT 0 0 0)
          (packT (eqB r1 * eqB r2) (consB r1 * consB r2)
            (2 * Nat.pair (intI r1) (intI r2) + 2)))))

theorem dsharpStep_i0 (j tbl : ℕ) :
    dsharpStep fcons feq finter (Nat.pair (Nat.pair 0 j) tbl) = packT (1 - j) 1 j :=
    by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd, Nat.sub_zero, selectFn_one]

theorem dsharpStep_j0 {i : ℕ} (tbl : ℕ) (hi : i = 0) :
```

```
    dsharpStep fcons feq finter (Nat.pair (Nat.pair i 0) tbl) = packT (1 - i) 1 i :=
    by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : ℕ) - i = 0 by omega, selectFn_zero, Nat.sub_zero, selectFn_one]

theorem dsharpStep_ll (a b tbl : ℕ) :
    dsharpStep fcons feq finter (Nat.pair (Nat.pair (2 * a + 1) (2 * b + 1)) tbl)
      = packT (feq (Nat.pair a b)) (fcons (Nat.pair a b)) (2 * finter (Nat.pair a b) +
     1) := by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : ℕ) - (2 * a + 1) = 0 by omega, selectFn_zero,
    show (1 : ℕ) - (2 * b + 1) = 0 by omega, selectFn_zero,
    show (2 * a + 1) % 2 = 1 by omega, selectFn_one,
    show (2 * b + 1) % 2 = 1 by omega, selectFn_one,
    show (2 * a + 1) / 2 = a by omega, show (2 * b + 1) / 2 = b by omega]

theorem dsharpStep_ln (a b tbl : ℕ) :
    dsharpStep fcons feq finter (Nat.pair (Nat.pair (2 * a + 1) (2 * b + 2)) tbl) =
    packT 0 0 0 := by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : ℕ) - (2 * a + 1) = 0 by omega, selectFn_zero,
    show (1 : ℕ) - (2 * b + 2) = 0 by omega, selectFn_zero,
    show (2 * a + 1) % 2 = 1 by omega, selectFn_one,
    show (2 * b + 2) % 2 = 0 by omega, selectFn_zero]

theorem dsharpStep_nl (a b tbl : ℕ) :
    dsharpStep fcons feq finter (Nat.pair (Nat.pair (2 * a + 2) (2 * b + 1)) tbl) =
    packT 0 0 0 := by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : ℕ) - (2 * a + 2) = 0 by omega, selectFn_zero,
    show (1 : ℕ) - (2 * b + 1) = 0 by omega, selectFn_zero,
    show (2 * a + 2) % 2 = 0 by omega, selectFn_zero,
    show (2 * b + 1) % 2 = 1 by omega, selectFn_one]

theorem dsharpStep_nn (a b tbl : ℕ) :
    dsharpStep fcons feq finter (Nat.pair (Nat.pair (2 * a + 2) (2 * b + 2)) tbl)
      = packT
          (eqB (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.unpair
    .1 b.unpair.1))
            * eqB (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.
    unpair.2 b.unpair.2)))
          (consB (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.
    unpair.1 b.unpair.1))
            * consB (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.
    unpair.2 b.unpair.2)))
          (2 * Nat.pair
            (intI (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.
    unpair.1 b.unpair.1)))
            (intI (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.
    unpair.2 b.unpair.2)))
            + 2) := by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : ℕ) - (2 * a + 2) = 0 by omega, selectFn_zero,
    show (1 : ℕ) - (2 * b + 2) = 0 by omega, selectFn_zero,
    show (2 * a + 2) % 2 = 0 by omega, selectFn_zero,
    show (2 * b + 2) % 2 = 0 by omega, selectFn_zero,
```

```
    show (2 * a + 2) / 2 - 1 = a by omega, show (2 * b + 2) / 2 - 1 = b by omega]

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

theorem pair_lt_succ_sq (x y : ℕ) : Nat.pair x y < (max x y + 1) * (max x y + 1) := by
  have hm : ∀ z : ℕ, (z + 1) * (z + 1) = z * z + 2 * z + 1 := fun z => by
    rw [Nat.succ_mul, Nat.mul_succ]; omega
  unfold Nat.pair
  rcases lt_or_ge x y with h | h
  · rw [ite_eq_left h, show max x y = y by omega, hm]; omega
  · rw [ite_eq_right (by omega), show max x y = x by omega, hm]; omega

theorem pair_lt_pair_of_lt {x y i j : ℕ} (hx : x < i) (hy : y < j) :
    Nat.pair x y < Nat.pair i j := by
  have h1 : Nat.pair x y < (max x y + 1) * (max x y + 1) := pair_lt_succ_sq x y
  have h2 : max i j * max i j ≤ Nat.pair i j := max_sq_le_pair i j
  have h3 : max x y + 1 ≤ max i j := by
    rcases le_total x y with h | h
    · rw [max_eq_right h]; have := le_max_right i j; omega
    · rw [max_eq_left h]; have := le_max_left i j; omega
  have h4 : (max x y + 1) * (max x y + 1) ≤ max i j * max i j := Nat.mul_le_mul h3 h3
  omega

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (P : ComputablePresentation D)

theorem primrec_listGet : Nat.Primrec (fun p => listGet p.unpair.1 p.unpair.2) := by
  have hstp : Nat.Primrec listGetStp := by
    unfold listGetStp
    have hcd : Nat.Primrec (fun t => t.unpair.2.unpair.1.unpair.1) :=
      Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)
    have hval : Nat.Primrec (fun t => t.unpair.2.unpair.1.unpair.2) :=
      Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)
    have hx : Nat.Primrec (fun t : ℕ => t.unpair.1) := Nat.Primrec.left
    exact (primrec_sub₂ hcd (Nat.Primrec.const 1)).pair
      (primrec_selectFn (primrec_isOne.comp hcd) hx hval)
  have hz : Nat.Primrec (fun p : ℕ => Nat.pair (p.unpair.2 + 1) 0) :=
    (Nat.Primrec.succ.comp Nat.Primrec.right).pair (Nat.Primrec.const 0)
  exact (Nat.Primrec.right.comp
    (primrec_foldCode hstp (Nat.Primrec.const 0) hz Nat.Primrec.left)).of_eq fun p =>
    rfl
```

```
theorem primrec_listGet₂ {c i : ℕ → ℕ} (hc : Nat.Primrec c) (hi : Nat.Primrec i) :
    Nat.Primrec (fun p => listGet (c p) (i p)) :=
  (primrec_listGet.comp (hc.pair hi)).of_eq fun p => by
    simp only [unpair_pair_fst, unpair_pair_snd]

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (P : ComputablePresentation D)

variable (fcons feq finter : ℕ → ℕ)

theorem primrec_dsharpStep (hfcons : Nat.Primrec fcons) (hfeq : Nat.Primrec feq)
    (hfinter : Nat.Primrec finter) : Nat.Primrec (dsharpStep fcons feq finter) := by
  have hi : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.left
  have hj : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.2) := Nat.Primrec.right.comp
    Nat.Primrec.left
  have htbl : Nat.Primrec (fun p : ℕ => p.unpair.2) := Nat.Primrec.right
  have hw : Nat.Primrec (fun p : ℕ => p.unpair.1) := Nat.Primrec.left
  have hidiv : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.1 / 2) := primrec_div2.comp
    hi
  have hjdiv : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.2 / 2) := primrec_div2.comp
    hj
  have hna : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.1 / 2 - 1) :=
    primrec_sub₂ hidiv (Nat.Primrec.const 1)
  have hnb : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.2 / 2 - 1) :=
    primrec_sub₂ hjdiv (Nat.Primrec.const 1)
  have hv1 : Nat.Primrec (fun p : ℕ =>
      Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.1 (p.unpair.1.unpair.2 / 2 - 1).
    unpair.1) :=
    (Nat.Primrec.left.comp hna).pair (Nat.Primrec.left.comp hnb)
  have hv2 : Nat.Primrec (fun p : ℕ =>
      Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.2 (p.unpair.1.unpair.2 / 2 - 1).
    unpair.2) :=
    (Nat.Primrec.right.comp hna).pair (Nat.Primrec.right.comp hnb)
  have hr1 : Nat.Primrec (fun p : ℕ => listGet p.unpair.2
      (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.1
        (p.unpair.1.unpair.2 / 2 - 1).unpair.1)) :=
    primrec_listGet₂ htbl (primrec_sub₂ (primrec_sub₂ hw (Nat.Primrec.const 1)) hv1)
  have hr2 : Nat.Primrec (fun p : ℕ => listGet p.unpair.2
      (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.2
        (p.unpair.1.unpair.2 / 2 - 1).unpair.2)) :=
    primrec_listGet₂ htbl (primrec_sub₂ (primrec_sub₂ hw (Nat.Primrec.const 1)) hv2)
  have hidiv2 : Nat.Primrec (fun p : ℕ => Nat.pair (p.unpair.1.unpair.1 / 2) (p.unpair
    .1.unpair.2 / 2)) :=
    hidiv.pair hjdiv
  have hpackT : ∀ {e c ii : ℕ → ℕ}, Nat.Primrec e → Nat.Primrec c → Nat.Primrec ii
    →
```

```
      Nat.Primrec (fun p => packT (e p) (c p) (ii p)) := by
    intro e c ii he hc hii
    exact (he.pair (hc.pair hii)).of_eq fun p => rfl
  have hleaf : Nat.Primrec (fun p : ℕ =>
      packT (feq (Nat.pair (p.unpair.1.unpair.1 / 2) (p.unpair.1.unpair.2 / 2)))
        (fcons (Nat.pair (p.unpair.1.unpair.1 / 2) (p.unpair.1.unpair.2 / 2)))
        (2 * finter (Nat.pair (p.unpair.1.unpair.1 / 2) (p.unpair.1.unpair.2 / 2)) +
    1)) :=
    hpackT (hfeq.comp hidiv2) (hfcons.comp hidiv2)
      (Nat.Primrec.succ.comp (primrec_mul₂ (Nat.Primrec.const 2) (hfinter.comp hidiv2))
    )
  have hzero : Nat.Primrec (fun _ : ℕ => packT 0 0 0) := Nat.Primrec.const _
  have heqB1 : Nat.Primrec (fun p => eqB _) := Nat.Primrec.left.comp hr1
  have heqB2 : Nat.Primrec (fun p => eqB _) := Nat.Primrec.left.comp hr2
  have hconsB1 : Nat.Primrec (fun p => consB _) := (Nat.Primrec.left.comp Nat.Primrec.
    right).comp hr1
  have hconsB2 : Nat.Primrec (fun p => consB _) := (Nat.Primrec.left.comp Nat.Primrec.
    right).comp hr2
  have hintI1 : Nat.Primrec (fun p => intI _) := (Nat.Primrec.right.comp Nat.Primrec.
    right).comp hr1
  have hintI2 : Nat.Primrec (fun p => intI _) := (Nat.Primrec.right.comp Nat.Primrec.
    right).comp hr2
  have hnode : Nat.Primrec (fun p : ℕ =>
      packT (eqB (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 /
     2 - 1).unpair.1
                (p.unpair.1.unpair.2 / 2 - 1).unpair.1))
            * eqB (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1
    / 2 - 1).unpair.2
                (p.unpair.1.unpair.2 / 2 - 1).unpair.2)))
        (consB (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2
     - 1).unpair.1
                (p.unpair.1.unpair.2 / 2 - 1).unpair.1))
            * consB (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair
    .1 / 2 - 1).unpair.2
                (p.unpair.1.unpair.2 / 2 - 1).unpair.2)))
        (2 * Nat.pair (intI (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair
    .1.unpair.1 / 2 - 1).unpair.1
                (p.unpair.1.unpair.2 / 2 - 1).unpair.1)))
              (intI (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair
    .1 / 2 - 1).unpair.2
                (p.unpair.1.unpair.2 / 2 - 1).unpair.2))) + 2)) :=
    hpackT (primrec_mul₂ heqB1 heqB2) (primrec_mul₂ hconsB1 hconsB2)
      ((Nat.Primrec.succ.comp Nat.Primrec.succ).comp (primrec_mul₂ (Nat.Primrec.const
    2)
        (hintI1.pair hintI2)))

  refine (primrec_selectFn (primrec_sub₂ (Nat.Primrec.const 1) hi)
    (hpackT (primrec_sub₂ (Nat.Primrec.const 1) hj) (Nat.Primrec.const 1) hj)
    (primrec_selectFn (primrec_sub₂ (Nat.Primrec.const 1) hj)
      (hpackT (primrec_sub₂ (Nat.Primrec.const 1) hi) (Nat.Primrec.const 1) hi)
      (primrec_selectFn (primrec_mod2.comp hi)
        (primrec_selectFn (primrec_mod2.comp hj) hleaf hzero)
        (primrec_selectFn (primrec_mod2.comp hj) hzero hnode)))).of_eq fun p => rfl

end Proposition77
end
```

```
section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (P : ComputablePresentation D)

def rtbl (step : ℕ → ℕ) : ℕ → ℕ
  | 0 => 0
  | (w + 1) => Nat.pair (step (Nat.pair w (rtbl step w))) (rtbl step w) + 1

def gOf (step : ℕ → ℕ) (w : ℕ) : ℕ := step (Nat.pair w (rtbl step w))

def gList (step : ℕ → ℕ) : ℕ → List ℕ
  | 0 => []
  | (w + 1) => gOf step w :: gList step w

theorem gOf_def (step : ℕ → ℕ) (w : ℕ) : gOf step w = step (Nat.pair w (rtbl step w))
    := rfl

theorem getD_gList (step : ℕ → ℕ) : ∀ (w p : ℕ), p < w →
    (gList step w).getD p 0 = gOf step (w - 1 - p) := by
  intro w
  induction w with
  | zero => intro p hp; omega
  | succ w ih =>
      intro p hp
      cases p with
      | zero =>
          rw [gList, List.getD_cons_zero]
          have : w + 1 - 1 - 0 = w := by omega
          rw [this]
      | succ p =>
          rw [gList, List.getD_cons_succ, ih p (by omega)]
          have : w + 1 - 1 - (p + 1) = w - 1 - p := by omega
          rw [this]

theorem rtbl_eq_encode (step : ℕ → ℕ) (w : ℕ) : rtbl step w = encodeList (gList step
    w) := by
  induction w with
  | zero => rfl
  | succ w ih =>
      show Nat.pair (step (Nat.pair w (rtbl step w))) (rtbl step w) + 1
        = encodeList (gList step (w + 1))
      rw [gList, encodeList, gOf, ih]

theorem listGet_rtbl (step : ℕ → ℕ) {v w : ℕ} (hv : v < w) :
    listGet (rtbl step w) (w - 1 - v) = gOf step v := by
  rw [rtbl_eq_encode, listGet_encodeList, getD_gList step w (w - 1 - v) (by omega)]
  congr 1; omega

theorem rtbl_eq_rec (step : ℕ → ℕ) (w : ℕ) :
    rtbl step w
      = Nat.rec (motive := fun _ => ℕ) 0
```

```
          (fun y IH => Nat.pair (step (Nat.pair y IH)) IH + 1) w := by
  induction w with
  | zero => rfl
  | succ w ih => rw [rtbl, ih]

theorem primrec_rtbl {step : ℕ → ℕ} (hstep : Nat.Primrec step) : Nat.Primrec (rtbl
    step) := by
  have hg : Nat.Primrec (fun c => Nat.pair (step c.unpair.2) c.unpair.2.unpair.2 + 1)
    :=
    Nat.Primrec.succ.comp ((hstep.comp Nat.Primrec.right).pair
      (Nat.Primrec.right.comp Nat.Primrec.right))
  refine ((Nat.Primrec.prec Nat.Primrec.zero hg).comp
    (Nat.Primrec.zero.pair primrec_id)).of_eq fun w => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rw [rtbl_eq_rec]

theorem primrec_gOf {step : ℕ → ℕ} (hstep : Nat.Primrec step) : Nat.Primrec (gOf step
    ) :=
  (hstep.comp (primrec_id.pair (primrec_rtbl hstep))).of_eq fun _ => rfl

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

theorem unpair_fst_le (n : ℕ) : n.unpair.1 ≤ n := by
  have h := le_pair_left n.unpair.1 n.unpair.2
  rwa [pair_unpair] at h

end Proposition77
end
end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}
namespace Proposition77

def Vsharp (D : NeighborhoodSystem α) (P : ComputablePresentation D) :
    ℕ → Set (List Bool × α)
  | 0 => Gamma D
  | (k + 1) =>
      if (k + 1) % 2 = 1 then
        embZero (P.X (k / 2))
      else
        embPair (Vsharp D P ((k - 1) / 2).unpair.1) (Vsharp D P ((k - 1) / 2).unpair
    .2)
  termination_by k => k
  decreasing_by
    · have := unpair_fst_le ((k - 1) / 2); omega
    · have := unpair_snd_le ((k - 1) / 2); omega
```

```
end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (P : ComputablePresentation D)

theorem Vsharp_succ (k : ℕ) :
    Vsharp D P (k + 1) =
      if (k + 1) % 2 = 1 then embZero (P.X (k / 2))
      else embPair (Vsharp D P ((k - 1) / 2).unpair.1) (Vsharp D P ((k - 1) / 2).
    unpair.2) := by
  rw [Vsharp]

theorem Vsharp_even (n : ℕ) :
    Vsharp D P (2 * n + 2) = embPair (Vsharp D P n.unpair.1) (Vsharp D P n.unpair.2)
    := by
  rw [show 2 * n + 2 = (2 * n + 1) + 1 from rfl, Vsharp_succ, ite_eq_right (by omega),
    show (2 * n + 1 - 1) / 2 = n from by omega]

theorem Vsharp_even_inter_even (a b : ℕ) :
    Vsharp D P (2 * a + 2) ∩ Vsharp D P (2 * b + 2)
      = embPair (Vsharp D P a.unpair.1 ∩ Vsharp D P b.unpair.1)
                (Vsharp D P a.unpair.2 ∩ Vsharp D P b.unpair.2) := by
  rw [Vsharp_even, Vsharp_even, embPair_inter]

theorem Vsharp_odd (n : ℕ) : Vsharp D P (2 * n + 1) = embZero (P.X n) := by
  rw [Vsharp_succ, ite_eq_left (by omega), show 2 * n / 2 = n from by omega]

theorem Vsharp_odd_inter_even (a b : ℕ) :
    Vsharp D P (2 * a + 1) ∩ Vsharp D P (2 * b + 2) = ∅ := by
  rw [Vsharp_odd, Vsharp_even, embZero_inter_embPair]

theorem Vsharp_even_inter_odd (a b : ℕ) :
    Vsharp D P (2 * a + 2) ∩ Vsharp D P (2 * b + 1) = ∅ := by
  rw [Set.inter_comm]; exact Vsharp_odd_inter_even P b a

theorem Vsharp_odd_inter_odd (a b : ℕ) :
    Vsharp D P (2 * a + 1) ∩ Vsharp D P (2 * b + 1) = embZero (P.X a ∩ P.X b) := by
  rw [Vsharp_odd, Vsharp_odd, embZero_inter]

theorem Vsharp_zero : Vsharp D P 0 = Gamma D := by rw [Vsharp]

theorem Vsharp_eq_Gamma_iff (hD : ∀ X, D.mem X → X.Nonempty) (j : ℕ) :
    Vsharp D P j = Gamma D ↔ j = 0 := by
  constructor
  · intro h
    rcases nat_shape j with rfl | ⟨a, rfl⟩ | ⟨a, rfl⟩
    · rfl
    · rw [Vsharp_odd] at h; exact absurd h (embZero_ne_Gamma D hD (P.X a))
    · rw [Vsharp_even] at h; exact absurd h (embPair_ne_Gamma D hD _ _)
```

```
  · rintro rfl; exact Vsharp_zero P

theorem Vsharp_mem (k : ℕ) : MemS D (Vsharp D P k) := by
  induction k using Nat.strong_induction_on with
  | _ k ih =>
    rcases Nat.eq_zero_or_pos k with rfl | hk
    · rw [Vsharp_zero]; exact MemS.gamma
    ·
      rcases (by
        rcases Nat.lt_or_ge (k % 2) 1 with h | h
        · exact Or.inr ⟨k / 2 - 1, by omega⟩
        · exact Or.inl ⟨k / 2, by omega⟩ :
          (∃ n, k = 2 * n + 1) ∨ (∃ n, k = 2 * n + 2)) with ⟨n, rfl⟩ | ⟨n, rfl⟩
      · rw [Vsharp_odd]; exact MemS.zero (P.mem_X n)
      · rw [Vsharp_even]
        exact MemS.pair (ih _ (by have := unpair_fst_le n; omega))
          (ih _ (by have := unpair_snd_le n; omega))

theorem Vsharp_inter_zero (n : ℕ) : Vsharp D P n ∩ Vsharp D P 0 = Vsharp D P n := by
  rw [Vsharp_zero, Set.inter_eq_left]; exact memS_subset_gamma (Vsharp_mem P n)

theorem Vsharp_nonempty (hD : ∀ X, D.mem X → X.Nonempty) (k : ℕ) : (Vsharp D P k).
    Nonempty :=
  memS_nonempty hD (Vsharp_mem P k)

theorem Vsharp_surj {W : Set (List Bool × α)} (hW : MemS D W) : ∃ k, Vsharp D P k = W
    := by
  induction hW with
  | gamma => exact ⟨0, Vsharp_zero P⟩
  | @zero X hX => obtain ⟨n, hn⟩ := P.surj hX; exact ⟨2 * n + 1, by rw [Vsharp_odd, hn]⟩
  | @pair Pp Qq _ _ ihP ihQ =>
      obtain ⟨a, ha⟩ := ihP
      obtain ⟨b, hb⟩ := ihQ
      refine ⟨2 * Nat.pair a b + 2, ?_⟩
      have h1 : (Nat.pair a b).unpair.1 = a := by rw [unpair_pair]
      have h2 : (Nat.pair a b).unpair.2 = b := by rw [unpair_pair]
      rw [Vsharp_even, h1, h2, ha, hb]

theorem Vsharp_zero_inter (m : ℕ) : Vsharp D P 0 ∩ Vsharp D P m = Vsharp D P m := by
  rw [Vsharp_zero, Set.inter_eq_right]; exact memS_subset_gamma (Vsharp_mem P m)

theorem exists_sub_embPair_iff (hD : ∀ X, D.mem X → X.Nonempty)
    (A B : Set (List Bool × α)) :
    (∃ l, Vsharp D P l ⊆ embPair A B) ↔
      (∃ l, Vsharp D P l ⊆ A) ∧ (∃ l, Vsharp D P l ⊆ B) := by
  constructor
  · rintro ⟨l, hl⟩
    obtain ⟨Pp, Qq, hP, hQ, hWeq⟩ := memS_sub_embPair hD (Vsharp_mem P l) hl
    rw [hWeq] at hl
    obtain ⟨hPA, hQB⟩ := embPair_subset.mp hl
    obtain ⟨l1, h1⟩ := Vsharp_surj P hP
    obtain ⟨l2, h2⟩ := Vsharp_surj P hQ
    exact ⟨⟨l1, h1 ▸ hPA⟩, ⟨l2, h2 ▸ hQB⟩⟩
  · rintro ⟨⟨l1, h1⟩, ⟨l2, h2⟩⟩
    obtain ⟨l, hl⟩ := Vsharp_surj P (MemS.pair (Vsharp_mem P l1) (Vsharp_mem P l2))
    exact ⟨l, by rw [hl]; exact embPair_subset.mpr ⟨h1, h2⟩⟩
```

```
theorem exists_sub_embZero_iff (hD : ∀ X, D.mem X → X.Nonempty) (Y : Set α) :
    (∃ l, Vsharp D P l ⊆ embZero Y) ↔ ∃ k, P.X k ⊆ Y := by
  constructor
  · rintro ⟨l, hl⟩
    obtain ⟨Z, hZ, hWZ⟩ := memS_sub_embZero hD (Vsharp_mem P l) hl
    rw [hWZ] at hl
    obtain ⟨k, hk⟩ := P.surj hZ
    exact ⟨k, hk ▸ embZero_subset.mp hl⟩
  · rintro ⟨k, hk⟩
    exact ⟨2 * k + 1, by rw [Vsharp_odd]; exact embZero_subset.mpr hk⟩

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive IndefProduct
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (P : ComputablePresentation D)

variable (fcons feq finter : ℕ → ℕ)

theorem dsharp_decider_spec (hD : ∀ X, D.mem X → X.Nonempty)
    (hcons : ∀ a b, fcons (Nat.pair a b) = 1 ↔ ∃ l, P.X l ⊆ P.X a ∩ P.X b)
    (heq : ∀ a b, feq (Nat.pair a b) = 1 ↔ P.X a = P.X b)
    (hinter : ∀ a b, (∃ l, P.X l ⊆ P.X a ∩ P.X b) → P.X (finter (Nat.pair a b)) = P.X
    a ∩ P.X b) :
    ∀ i j : ℕ,
      (consB (gOf (dsharpStep fcons feq finter) (Nat.pair i j)) = 1
          ↔ ∃ l, Vsharp D P l ⊆ Vsharp D P i ∩ Vsharp D P j) ∧
        ((∃ l, Vsharp D P l ⊆ Vsharp D P i ∩ Vsharp D P j) →
          Vsharp D P (intI (gOf (dsharpStep fcons feq finter) (Nat.pair i j)))
            = Vsharp D P i ∩ Vsharp D P j) ∧
        (eqB (gOf (dsharpStep fcons feq finter) (Nat.pair i j)) = 1
          ↔ Vsharp D P i = Vsharp D P j) := by
  have key : ∀ w : ℕ, ∀ i j : ℕ, Nat.pair i j = w →
      (consB (gOf (dsharpStep fcons feq finter) (Nat.pair i j)) = 1
          ↔ ∃ l, Vsharp D P l ⊆ Vsharp D P i ∩ Vsharp D P j) ∧
        ((∃ l, Vsharp D P l ⊆ Vsharp D P i ∩ Vsharp D P j) →
          Vsharp D P (intI (gOf (dsharpStep fcons feq finter) (Nat.pair i j)))
            = Vsharp D P i ∩ Vsharp D P j) ∧
        (eqB (gOf (dsharpStep fcons feq finter) (Nat.pair i j)) = 1
          ↔ Vsharp D P i = Vsharp D P j) := by
    intro w
    induction w using Nat.strong_induction_on with
    | _ w ih =>
      intro i j hw
      rcases nat_shape i with rfl | ⟨a, rfl⟩ | ⟨a, rfl⟩
      ·
        simp only [gOf_def]
        rw [dsharpStep_i0 fcons feq finter j]
        simp only [eqB_packT, consB_packT, intI_packT]
        refine ⟨?_, ?_, ?_⟩
```

```
    · exact ⟨fun _ => ⟨j, (Vsharp_zero_inter P j).ge⟩, fun _ => trivial⟩
    · intro _; rw [Vsharp_zero_inter P]
    · constructor
      · intro h
        have hj : j = 0 := by omega
        rw [hj]
      · intro h
        have hj : j = 0 := (Vsharp_eq_Gamma_iff P hD j).mp (h.symm.trans (
Vsharp_zero P))
        omega
  ·
    rcases nat_shape j with rfl | ⟨b, rfl⟩ | ⟨b, rfl⟩
    ·
      simp only [gOf_def]
      rw [dsharpStep_j0 fcons feq finter _ (by omega : 2 * a + 1 = 0)]
      simp only [eqB_packT, consB_packT, intI_packT]
      refine ⟨?_, ?_, ?_⟩
      · exact ⟨fun _ => ⟨2 * a + 1, (Vsharp_inter_zero P (2 * a + 1)).ge⟩, fun _ =>
trivial⟩
      · intro _; exact (Vsharp_inter_zero P (2 * a + 1)).symm
      · rw [Vsharp_zero P, Vsharp_eq_Gamma_iff P hD]; omega
    ·
      simp only [gOf_def]
      rw [dsharpStep_ll fcons feq finter a b]
      simp only [eqB_packT, consB_packT, intI_packT]
      refine ⟨?_, ?_, ?_⟩
      · rw [Vsharp_odd_inter_odd P, exists_sub_embZero_iff P hD]; exact hcons a b
      · intro hp
        rw [Vsharp_odd_inter_odd P] at hp ⊢
        rw [exists_sub_embZero_iff P hD] at hp
        rw [Vsharp_odd P, hinter a b hp]
      · rw [Vsharp_odd P, Vsharp_odd P, heq a b]
        exact ⟨fun h => by rw [h], fun h => embZero_injective h⟩
    ·
      simp only [gOf_def]
      rw [dsharpStep_ln fcons feq finter a b]
      simp only [eqB_packT, consB_packT, intI_packT]
      refine ⟨?_, ?_, ?_⟩
      · rw [Vsharp_odd_inter_even P]
        constructor
        · intro h; exact absurd h (by decide)
        · rintro ⟨l, hl⟩
          obtain ⟨x, hx⟩ := Vsharp_nonempty P hD l
          exact absurd (hl hx) (by simp)
      · rw [Vsharp_odd_inter_even P]
        rintro ⟨l, hl⟩
        obtain ⟨x, hx⟩ := Vsharp_nonempty P hD l
        exact absurd (hl hx) (by simp)
      · rw [Vsharp_odd P, Vsharp_even P]
        constructor
        · intro h; exact absurd h (by decide)
        · intro h; exact absurd h (embZero_ne_embPair D hD (P.mem_X a) _ _)
  ·
    rcases nat_shape j with rfl | ⟨b, rfl⟩ | ⟨b, rfl⟩
    ·
      simp only [gOf_def]
```

```
      rw [dsharpStep_j0 fcons feq finter _ (by omega : 2 * a + 2 = 0)]
      simp only [eqB_packT, consB_packT, intI_packT]
      refine ⟨?_, ?_, ?_⟩
      · exact ⟨fun _ => ⟨2 * a + 2, (Vsharp_inter_zero P (2 * a + 2)).ge⟩, fun _ =>
trivial⟩
      · intro _; exact (Vsharp_inter_zero P (2 * a + 2)).symm
      · rw [Vsharp_zero P, Vsharp_eq_Gamma_iff P hD]; omega
    .
      simp only [gOf_def]
      rw [dsharpStep_nl fcons feq finter a b]
      simp only [eqB_packT, consB_packT, intI_packT]
      refine ⟨?_, ?_, ?_⟩
      · rw [Vsharp_even_inter_odd P]
        constructor
        · intro h; exact absurd h (by decide)
        · rintro ⟨l, hl⟩
          obtain ⟨x, hx⟩ := Vsharp_nonempty P hD l
          exact absurd (hl hx) (by simp)
      · rw [Vsharp_even_inter_odd P]
        rintro ⟨l, hl⟩
        obtain ⟨x, hx⟩ := Vsharp_nonempty P hD l
        exact absurd (hl hx) (by simp)
      · rw [Vsharp_even P, Vsharp_odd P]
        constructor
        · intro h; exact absurd h (by decide)
        · intro h; exact absurd h.symm (embZero_ne_embPair D hD (P.mem_X b) _ _)
    .
      have hlt1 : Nat.pair a.unpair.1 b.unpair.1 < Nat.pair (2 * a + 2) (2 * b +
2) :=
        pair_lt_pair_of_lt (by have := unpair_fst_le a; omega) (by have :=
unpair_fst_le b; omega)
      have hlt2 : Nat.pair a.unpair.2 b.unpair.2 < Nat.pair (2 * a + 2) (2 * b +
2) :=
        pair_lt_pair_of_lt (by have := unpair_snd_le a; omega) (by have :=
unpair_snd_le b; omega)
      have hr1 := listGet_rtbl (dsharpStep fcons feq finter) hlt1
      have hr2 := listGet_rtbl (dsharpStep fcons feq finter) hlt2
      obtain ⟨ihc1, ihi1, ihe1⟩ :=
        ih (Nat.pair a.unpair.1 b.unpair.1) (hw ▶ hlt1) a.unpair.1 b.unpair.1 rfl
      obtain ⟨ihc2, ihi2, ihe2⟩ :=
        ih (Nat.pair a.unpair.2 b.unpair.2) (hw ▶ hlt2) a.unpair.2 b.unpair.2 rfl
      simp only [gOf_def]
      rw [dsharpStep_nn fcons feq finter a b]
      simp only [eqB_packT, consB_packT, intI_packT]
      rw [hr1, hr2]
      refine ⟨?_, ?_, ?_⟩
      · rw [Vsharp_even_inter_even P, exists_sub_embPair_iff P hD, nat_mul_eq_one]
        exact and_congr ihc1 ihc2
      · intro hp
        rw [Vsharp_even_inter_even P] at hp ⊢
        rw [exists_sub_embPair_iff P hD] at hp
        obtain ⟨hp1, hp2⟩ := hp
        rw [Vsharp_even P]
        simp only [unpair_pair_fst, unpair_pair_snd]
        rw [ihi1 hp1, ihi2 hp2]
      · rw [Vsharp_even P, Vsharp_even P, nat_mul_eq_one, ihe1, ihe2]
```

```
            constructor
            · rintro ⟨h1, h2⟩; rw [h1, h2]
            · intro h; exact embPair_injective h
  intro i j
  exact key (Nat.pair i j) i j rfl

theorem dsharp_intI_correct (hD : ∀ X, D.mem X → X.Nonempty)
    (hinter : ∀ a b, (∃ l, P.X l ⊆ P.X a ∩ P.X b) → P.X (finter (Nat.pair a b)) = P.X
     a ∩ P.X b) :
    ∀ i j : ℕ, (∃ l, Vsharp D P l ⊆ Vsharp D P i ∩ Vsharp D P j) →
      Vsharp D P (intI (gOf (dsharpStep fcons feq finter) (Nat.pair i j)))
        = Vsharp D P i ∩ Vsharp D P j := by
  have key : ∀ w : ℕ, ∀ i j : ℕ, Nat.pair i j = w →
      (∃ l, Vsharp D P l ⊆ Vsharp D P i ∩ Vsharp D P j) →
      Vsharp D P (intI (gOf (dsharpStep fcons feq finter) (Nat.pair i j)))
        = Vsharp D P i ∩ Vsharp D P j := by
    intro w

    induction w using Nat.strong_induction_on with
    | _ w ih =>
      intro i j hw
      rcases nat_shape i with rfl | ⟨a, rfl⟩ | ⟨a, rfl⟩
      · intro _
        simp only [gOf_def]; rw [dsharpStep_i0 fcons feq finter j]
        simp only [intI_packT]; rw [Vsharp_zero_inter P]
      · rcases nat_shape j with rfl | ⟨b, rfl⟩ | ⟨b, rfl⟩
        · intro _
          simp only [gOf_def]; rw [dsharpStep_j0 fcons feq finter _ (by omega : 2 * a
    + 1 = 0)]
          simp only [intI_packT]; exact (Vsharp_inter_zero P (2 * a + 1)).symm
        · intro hp
          simp only [gOf_def]; rw [dsharpStep_ll fcons feq finter a b]
          simp only [intI_packT]
          rw [Vsharp_odd_inter_odd P] at hp ⊢
          rw [exists_sub_embZero_iff P hD] at hp
          rw [Vsharp_odd P, hinter a b hp]
        · intro hp
          exfalso
          rw [Vsharp_odd_inter_even P] at hp
          obtain ⟨l, hl⟩ := hp
          obtain ⟨x, hx⟩ := Vsharp_nonempty P hD l
          exact absurd (hl hx) (by simp)
      · rcases nat_shape j with rfl | ⟨b, rfl⟩ | ⟨b, rfl⟩
        · intro _
          simp only [gOf_def]; rw [dsharpStep_j0 fcons feq finter _ (by omega : 2 * a
    + 2 = 0)]
          simp only [intI_packT]; exact (Vsharp_inter_zero P (2 * a + 2)).symm
        · intro hp
          exfalso
          rw [Vsharp_even_inter_odd P] at hp
          obtain ⟨l, hl⟩ := hp
          obtain ⟨x, hx⟩ := Vsharp_nonempty P hD l
          exact absurd (hl hx) (by simp)
        · intro hp
          have hlt1 : Nat.pair a.unpair.1 b.unpair.1 < Nat.pair (2 * a + 2) (2 * b +
    2) :=
```

```
            pair_lt_pair_of_lt (by have := unpair_fst_le a; omega) (by have :=
    unpair_fst_le b; omega)
          have hlt2 : Nat.pair a.unpair.2 b.unpair.2 < Nat.pair (2 * a + 2) (2 * b +
    2) :=
            pair_lt_pair_of_lt (by have := unpair_snd_le a; omega) (by have :=
    unpair_snd_le b; omega)
          have hr1 := listGet_rtbl (dsharpStep fcons feq finter) hlt1
          have hr2 := listGet_rtbl (dsharpStep fcons feq finter) hlt2
          rw [Vsharp_even_inter_even P] at hp ⊢
          rw [exists_sub_embPair_iff P hD] at hp
          obtain ⟨hp1, hp2⟩ := hp
          simp only [gOf_def]; rw [dsharpStep_nn fcons feq finter a b]
          simp only [intI_packT]; rw [hr1, hr2, Vsharp_even P]
          simp only [unpair_pair_fst, unpair_pair_snd]
          rw [ih _ (hw ▶ hlt1) a.unpair.1 b.unpair.1 rfl hp1,
              ih _ (hw ▶ hlt2) a.unpair.2 b.unpair.2 rfl hp2]
  intro i j; exact key (Nat.pair i j) i j rfl

theorem dsharp_interEq_iff (hD : ∀ X, D.mem X → X.Nonempty)
    (hcons : ∀ a b, fcons (Nat.pair a b) = 1 ↔ ∃ l, P.X l ⊆ P.X a ∩ P.X b)
    (heq : ∀ a b, feq (Nat.pair a b) = 1 ↔ P.X a = P.X b)
    (hinter : ∀ a b, (∃ l, P.X l ⊆ P.X a ∩ P.X b) → P.X (finter (Nat.pair a b)) = P.X
     a ∩ P.X b)
    (n m k : ℕ) :
    (Vsharp D P n ∩ Vsharp D P m = Vsharp D P k) ↔
      consB (gOf (dsharpStep fcons feq finter) (Nat.pair n m))
        * eqB (gOf (dsharpStep fcons feq finter)
            (Nat.pair (intI (gOf (dsharpStep fcons feq finter) (Nat.pair n m))) k)) =
    1 := by
  obtain ⟨hc0, hi0, _⟩ := dsharp_decider_spec P fcons feq finter hD hcons heq hinter n
    m
  obtain ⟨_, _, he⟩ := dsharp_decider_spec P fcons feq finter hD hcons heq hinter
    (intI (gOf (dsharpStep fcons feq finter) (Nat.pair n m))) k
  rw [nat_mul_eq_one]
  constructor
  · intro hEq
    have hch : ∃ l, Vsharp D P l ⊆ Vsharp D P n ∩ Vsharp D P m := ⟨k, hEq.ge⟩
    exact ⟨hc0.mpr hch, he.mpr ((hi0 hch).trans hEq)⟩
  · rintro ⟨hcb, heb⟩
    rw [← hi0 (hc0.mp hcb)]; exact he.mp heb

def dsharpPresentation (hD : ∀ X, D.mem X → X.Nonempty) :
    ComputablePresentation (Dsharp D hD) where
  X := Vsharp D P
  mem_X := fun n => Vsharp_mem P n
  surj := fun hY => Vsharp_surj P hY
  interEq_computable := by
    obtain ⟨fc, hfc_pr, hfc⟩ := P.cons_computable
    obtain ⟨fe, hfe_pr, hfe⟩ := P.eq_computable
    have hcons : ∀ a b, fc (Nat.pair a b) = 1 ↔ ∃ l, P.X l ⊆ P.X a ∩ P.X b := fun a b
     => by
      have := hfc (Nat.pair a b); simp only [unpair_pair_fst, unpair_pair_snd] at this
    ; exact this.symm
    have heq : ∀ a b, fe (Nat.pair a b) = 1 ↔ P.X a = P.X b := fun a b => by
      have := hfe (Nat.pair a b); simp only [unpair_pair_fst, unpair_pair_snd] at this
    ; exact this.symm
```

```
  have hinter : ∀ a b, (∃ l, P.X l ⊆ P.X a ∩ P.X b) →
      P.X ((fun s => P.inter s.unpair.1 s.unpair.2) (Nat.pair a b)) = P.X a ∩ P.X b
  := fun a b h => by
    simp only [unpair_pair_fst, unpair_pair_snd]; exact P.inter_spec h
  refine ⟨fun t => consB (gOf (dsharpStep fc fe (fun s => P.inter s.unpair.1 s.unpair
  .2))
      (Nat.pair t.unpair.1 t.unpair.2.unpair.1))
      * eqB (gOf (dsharpStep fc fe (fun s => P.inter s.unpair.1 s.unpair.2))
          (Nat.pair (intI (gOf (dsharpStep fc fe (fun s => P.inter s.unpair.1 s.
  unpair.2))
            (Nat.pair t.unpair.1 t.unpair.2.unpair.1))) t.unpair.2.unpair.2)), ?_,
  fun t => ?_⟩
  · have hg := primrec_gOf (primrec_dsharpStep fc fe (fun s => P.inter s.unpair.1 s.
  unpair.2)
      hfc_pr hfe_pr P.inter_primrec)
    have hc0 := hg.comp (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.
  right))
    exact primrec_mul₂ ((Nat.Primrec.left.comp Nat.Primrec.right).comp hc0)
      (Nat.Primrec.left.comp (hg.comp
        (((Nat.Primrec.right.comp Nat.Primrec.right).comp hc0).pair
          (Nat.Primrec.right.comp Nat.Primrec.right))))
  · exact dsharp_interEq_iff P fc fe (fun s => P.inter s.unpair.1 s.unpair.2) hD
  hcons heq hinter
      t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2
cons_computable := by
  obtain ⟨fc, hfc_pr, hfc⟩ := P.cons_computable
  obtain ⟨fe, hfe_pr, hfe⟩ := P.eq_computable
  have hcons : ∀ a b, fc (Nat.pair a b) = 1 ↔ ∃ l, P.X l ⊆ P.X a ∩ P.X b := fun a b
   => by
    have := hfc (Nat.pair a b); simp only [unpair_pair_fst, unpair_pair_snd] at this
  ; exact this.symm
  have heq : ∀ a b, fe (Nat.pair a b) = 1 ↔ P.X a = P.X b := fun a b => by
    have := hfe (Nat.pair a b); simp only [unpair_pair_fst, unpair_pair_snd] at this
  ; exact this.symm
  have hinter : ∀ a b, (∃ l, P.X l ⊆ P.X a ∩ P.X b) →
      P.X ((fun s => P.inter s.unpair.1 s.unpair.2) (Nat.pair a b)) = P.X a ∩ P.X b
  := fun a b h => by
    simp only [unpair_pair_fst, unpair_pair_snd]; exact P.inter_spec h
  refine ⟨fun t => consB (gOf (dsharpStep fc fe (fun s => P.inter s.unpair.1 s.unpair
  .2))
      (Nat.pair t.unpair.1 t.unpair.2)), ?_, fun t => ?_⟩
  · have hg := primrec_gOf (primrec_dsharpStep fc fe (fun s => P.inter s.unpair.1 s.
  unpair.2)
      hfc_pr hfe_pr P.inter_primrec)
    exact (Nat.Primrec.left.comp Nat.Primrec.right).comp
      (hg.comp (Nat.Primrec.left.pair Nat.Primrec.right))
  · exact (dsharp_decider_spec P fc fe (fun s => P.inter s.unpair.1 s.unpair.2) hD
  hcons heq hinter
      t.unpair.1 t.unpair.2).1.symm
inter := fun n m => intI (gOf (dsharpStep (fun _ => 0) (fun _ => 0)
  (fun s => P.inter s.unpair.1 s.unpair.2)) (Nat.pair n m))
inter_primrec := by
  have hg := primrec_gOf (primrec_dsharpStep (fun _ => 0) (fun _ => 0)
    (fun s => P.inter s.unpair.1 s.unpair.2) (Nat.Primrec.const 0) (Nat.Primrec.
  const 0)
    P.inter_primrec)
```

```
    have hpair : Nat.Primrec (fun t : ℕ => Nat.pair t.unpair.1 t.unpair.2) :=
      Nat.Primrec.left.pair Nat.Primrec.right
    exact (Nat.Primrec.right.comp Nat.Primrec.right).comp (hg.comp hpair)
  inter_spec := by
    intro n m h
    exact dsharp_intI_correct P (fun _ => 0) (fun _ => 0) (fun s => P.inter s.unpair.1
     s.unpair.2) hD
      (fun a b hh => by simp only [unpair_pair_fst, unpair_pair_snd]; exact P.
    inter_spec hh) n m h
  masterIdx := 0
  masterIdx_spec := Vsharp_zero P

theorem dsharp_isEffectivelyGiven (P : ComputablePresentation D)
    (hD : ∀ X, D.mem X → X.Nonempty) :
    (Dsharp D hD).IsEffectivelyGiven :=
  ⟨dsharpPresentation P hD⟩

end Proposition77
end
end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive IndefProduct

section
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (hD : ∀ X, D.mem X → X.Nonempty)

@[simp] theorem dsharpPresentation_X (P : ComputablePresentation D) (n : ℕ) :
    (dsharpPresentation P hD).X n = Vsharp D P n := rfl

def inSharpMap : ApproximableMap D (Dsharp D hD) where
  rel X W := D.mem X ∧ MemS D W ∧ embZero X ⊆ W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨D.master_mem, MemS.gamma, embZero_subset_Gamma D.master_mem⟩
  inter_right := by
    rintro X W W' ⟨hX, hW, hsub⟩ ⟨-, hW', hsub'⟩
    exact ⟨hX, memS_inter hD hW hW' (MemS.zero hX) (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'⟩
  mono := by
    rintro X X' W W' ⟨hX, hW, hsub⟩ hX'X hWW' hX'mem hW'mem
    exact ⟨hX'mem, hW'mem, (embZero_subset.mpr hX'X).trans (hsub.trans hWW')⟩

@[simp] theorem inSharpMap_rel {X : Set α} {W : Set (List Bool × α)} :
    (inSharpMap hD).rel X W ↔ D.mem X ∧ MemS D W ∧ embZero X ⊆ W := Iff.rfl

def proj0Map : ApproximableMap (Dsharp D hD) (Dsharp D hD) where
  rel W Z := MemS D W ∧ MemS D Z ∧ (Z = Gamma D ∨ ∃ Pp Qq, W = embPair Pp Qq ∧ Pp ⊆ Z)
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨MemS.gamma, MemS.gamma, Or.inl rfl⟩
  inter_right := by
```

```
    rintro W Z Z' ⟨hW, hZ, hd⟩ ⟨-, hZ', hd'⟩
    have hgZ : Z ⊆ Gamma D := memS_subset_gamma hZ
    have hgZ' : Z' ⊆ Gamma D := memS_subset_gamma hZ'
    rcases hd with rfl | ⟨Pp, Qq, hWeq, hPZ⟩
    · rw [Set.inter_eq_right.mpr hgZ']; exact ⟨hW, hZ', hd'⟩
    · rcases hd' with rfl | ⟨Pp', Qq', hWeq', hPZ'⟩
      · rw [Set.inter_eq_left.mpr hgZ]; exact ⟨hW, hZ, Or.inr ⟨Pp, Qq, hWeq, hPZ⟩⟩
      · have hPP : Pp = Pp' := (embPair_injective (hWeq.symm.trans hWeq')).1
        have hPsub : Pp ⊆ Z ∩ Z' := Set.subset_inter hPZ (by rw [hPP]; exact hPZ')
        have hMemP : MemS D Pp := (memS_embPair_inv hD (hWeq ▸ hW)).1
        exact ⟨hW, memS_inter hD hZ hZ' hMemP hPsub, Or.inr ⟨Pp, Qq, hWeq, hPsub⟩⟩
  mono := by
    rintro W W' Z Z' ⟨hW, hZ, hd⟩ hW'W hZZ' hW'mem hZ'mem
    refine ⟨hW'mem, hZ'mem, ?_⟩
    rcases hd with rfl | ⟨Pp, Qq, hWeq, hPZ⟩
    · exact Or.inl (Set.Subset.antisymm (memS_subset_gamma hZ'mem) hZZ')
    · subst hWeq
      cases hW'mem with
      | gamma =>
        exact absurd (Set.Subset.antisymm (memS_subset_gamma hW) hW'W)
          (embPair_ne_Gamma D hD Pp Qq)
      | @zero X hX =>
        exfalso
        obtain ⟨a, ha⟩ := hD X hX
        rcases hW'W (show (([], a) : List Bool × α) ∈ embZero X from ⟨rfl, ha⟩) with
          ⟨p', hp', -⟩ | ⟨q', hq', -⟩
        · simp at hp'
        · simp at hq'
      | @pair Pp' Qq' hP' hQ' =>
        obtain ⟨hP'P, -⟩ := embPair_subset.mp hW'W
        exact Or.inr ⟨Pp', Qq', rfl, fun z hz => hZZ' (hPZ (hP'P hz))⟩

@[simp] theorem proj0Map_rel {W Z : Set (List Bool × α)} :
    (proj0Map hD).rel W Z ↔
      MemS D W ∧ MemS D Z ∧ (Z = Gamma D ∨ ∃ Pp Qq, W = embPair Pp Qq ∧ Pp ⊆ Z) := Iff
    .rfl

theorem inSharpMap_toElementMap (x : D.Element) :
    (inSharpMap hD).toElementMap x = inSharp D hD x := by
  apply Element.ext
  intro W
  simp only [ApproximableMap.mem_toElementMap, inSharpMap_rel]
  constructor
  · rintro ⟨X, hXx, hX, hW, hsub⟩
    cases hW with
    | gamma => exact Or.inl rfl
    | @zero Y hY => exact Or.inr ⟨Y, x.up_mem hXx hY (embZero_subset.mp hsub), rfl⟩
    | @pair Pp Qq hP hQ =>
      exfalso
      obtain ⟨a, ha⟩ := hD X hX
      rcases hsub (show (([], a) : List Bool × α) ∈ embZero X from ⟨rfl, ha⟩) with
        ⟨p', hp', -⟩ | ⟨q', hq', -⟩
      · simp at hp'
      · simp at hq'
  · rintro (rfl | ⟨Y, hYx, rfl⟩)
    · exact ⟨D.master, x.master_mem, D.master_mem, MemS.gamma, embZero_subset_Gamma D.
```

```
    master_mem⟩
    · exact ⟨Y, hYx, x.sub hYx, MemS.zero (x.sub hYx), subset_rfl⟩

theorem inSharp_isComputable (P : ComputablePresentation D) :
    ComputableMap P (dsharpPresentation P hD) (inSharpMap hD) := by
  constructor
  show REPred (fun s => (inSharpMap hD).rel (P.X s.unpair.1) (Vsharp D P s.unpair.2))
  have hincl : RecDecidable (fun s => Vsharp D P s.unpair.1 ⊆ Vsharp D P s.unpair.2)
    :=
    (dsharpPresentation P hD).incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair (2 * t.unpair.1 + 1) t.unpair.2) :=
    (primrec_add₂ (primrec_mul₂ (Nat.Primrec.const 2) Nat.Primrec.left)
      (Nat.Primrec.const 1)).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [inSharpMap_rel, unpair_pair_fst, unpair_pair_snd, Vsharp_odd]
  exact ⟨fun h => h.2.2, fun h => ⟨P.mem_X _, Vsharp_mem P _, h⟩⟩

theorem proj0_rel_Vsharp_iff (P : ComputablePresentation D) (m k : ℕ) :
    (proj0Map hD).rel (Vsharp D P m) (Vsharp D P k) ↔
      k = 0 ∨ (m % 2 = 0 ∧ m = 0 ∧ Vsharp D P (m / 2 - 1).unpair.1 ⊆ Vsharp D P k) :=
    by
  rw [proj0Map_rel, and_iff_right (Vsharp_mem P m), and_iff_right (Vsharp_mem P k)]
  refine or_congr (Vsharp_eq_Gamma_iff P hD k) ?_
  rcases nat_shape m with rfl | ⟨a, rfl⟩ | ⟨a, rfl⟩
  · rw [Vsharp_zero]
    constructor
    · rintro ⟨Pp, Qq, heq, -⟩; exact absurd heq.symm (embPair_ne_Gamma D hD Pp Qq)
    · rintro ⟨-, hne, -⟩; exact absurd rfl hne
  · rw [Vsharp_odd]
    constructor
    · rintro ⟨Pp, Qq, heq, -⟩; exact absurd heq (embZero_ne_embPair D hD (P.mem_X a) Pp
    Qq)
    · rintro ⟨hmod, -, -⟩; exact absurd hmod (by omega)
  · rw [Vsharp_even, show (2 * a + 2) / 2 - 1 = a from by omega]
    constructor
    · rintro ⟨Pp, Qq, heq, hPk⟩
      obtain ⟨hP, -⟩ := embPair_injective heq
      exact ⟨by omega, by omega, by rw [hP]; exact hPk⟩
    · rintro ⟨-, -, hsub⟩
      exact ⟨Vsharp D P a.unpair.1, Vsharp D P a.unpair.2, rfl, hsub⟩

theorem proj0_isComputable (P : ComputablePresentation D) :
    ComputableMap (dsharpPresentation P hD) (dsharpPresentation P hD) (proj0Map hD) :=
     by
  constructor
  show REPred (fun s => (proj0Map hD).rel (Vsharp D P s.unpair.1) (Vsharp D P s.unpair
    .2))
  have hk0 : RecDecidable (fun s => s.unpair.2 = 0) :=
    RecDecidable.natEq Nat.Primrec.right (Nat.Primrec.const 0)
  have hmod : RecDecidable (fun s => s.unpair.1 % 2 = 0) :=
    RecDecidable.natEq (primrec_mod2.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have hne : RecDecidable (fun s => s.unpair.1 = 0) :=
    (RecDecidable.natEq Nat.Primrec.left (Nat.Primrec.const 0)).not
  have hg : Nat.Primrec (fun s => Nat.pair (s.unpair.1 / 2 - 1).unpair.1 s.unpair.2)
    :=
    (Nat.Primrec.left.comp (primrec_sub₂ (primrec_div2.comp Nat.Primrec.left)
```

```
      (Nat.Primrec.const 1))).pair Nat.Primrec.right
  have hincl : RecDecidable (fun s => Vsharp D P (s.unpair.1 / 2 - 1).unpair.1 ⊆
      Vsharp D P s.unpair.2) := by
    refine RecDecidable.of_iff (fun s => ?_) ((dsharpPresentation P hD).
    incl_computable.comp hg)
    simp only [dsharpPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact (RecDecidable.of_iff (fun s => proj0_rel_Vsharp_iff hD P s.unpair.1 s.unpair
    .2)
    (hk0.or (hmod.and (hne.and hincl)))).re

theorem proj0_toElementMap_pairSharp (x y : (Dsharp D hD).Element) :
    (proj0Map hD).toElementMap (pairSharp D hD x y) = x := by
  apply Element.ext
  intro Z
  simp only [ApproximableMap.mem_toElementMap, proj0Map_rel]
  constructor
  · rintro ⟨W, hWmem, hMW, hMZ, hdisj⟩
    rcases hWmem with rfl | ⟨Pp, Qq, hxP, hyQ, rfl⟩
    · rcases hdisj with rfl | ⟨Pp', Qq', hPe, -⟩
      · exact x.master_mem
      · exact absurd hPe.symm (embPair_ne_Gamma D hD Pp' Qq')
    · rcases hdisj with rfl | ⟨Pp', Qq', hPe, hP'Z⟩
      · exact x.master_mem
      · obtain ⟨hPP, -⟩ := embPair_injective hPe
        exact x.up_mem hxP hMZ (by rw [hPP]; exact hP'Z)
  · intro hxZ
    exact ⟨embPair Z (Gamma D), Or.inr ⟨Z, Gamma D, hxZ, y.master_mem, rfl⟩,
      MemS.pair (x.sub hxZ) MemS.gamma, x.sub hxZ, Or.inr ⟨Z, Gamma D, rfl, subset_rfl⟩
    ⟩

end Proposition77
end

section
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (P : ComputablePresentation D)

theorem gList_length (step : ℕ → ℕ) (w : ℕ) : (gList step w).length = w := by
  induction w with
  | zero => rfl
  | succ w ih => rw [gList, List.length_cons, ih]

end Proposition77
end
end DomainTheory.NeighbourhoodSystems
```

4.8.17 List combinators

Pairing and the projections on the list domain are computable.

Definition (`Proposition77.proj1Map`). The second projection relation sends the master neighbourhood to every master target and sends a paired neighbourhood to targets containing its right component.

Proof. The relation contains the master pair, is closed under target intersections, and is monotone under refinement of source and enlargement of target. ■

Theorem (`Proposition77.proj1Map_rel`). The second projection relates W to Z exactly when both are list neighbourhoods and either Z is master or W is a pair whose right component is contained in Z.

Proof. This is the defining relation. ■

Definition (`Proposition77.pairSharpMap`). The pairing relation sends a product neighbourhood generated by A, B to every list neighbourhood containing the embedded pair of A and B.

Proof. Pairing master neighbourhoods gives the master case; intersections and monotonicity follow componentwise. ■

Theorem (`Proposition77.pairSharpMap_rel`). A product neighbourhood V is related to W exactly when V is generated by list neighbourhoods A, B and the embedded pair of A, B is contained in W.

Proof. This is the defining relation. ■

Theorem (`Proposition77.pairSharpMap_toElementMap`). On elements, the pairing approximable map is the list-domain pairing operation.

Proof. A target neighbourhood is obtained from component neighbourhoods exactly when it contains their embedded pair. ■

Theorem (`Proposition77.pairSharp_rel_Vsharp_iff`). At presentation indices t, k, the pairing relation holds exactly when the paired list neighbourhood indexed by $2t + 2$ is contained in the neighbourhood indexed by k.

Proof. Decode t into its two product indices and use the even-index formula for list neighbourhoods. ■

Theorem (`Proposition77.pairSharp_isComputable`). The pairing approximable map on the list domain is computable.

Proof. Its indexed relation reduces to decidable inclusion after the primitive-recursive transformation $(t, k) \mapsto (2t + 2, k)$. ■

Theorem (`Proposition77.proj1_rel_Vsharp_iff`). The second projection relates the neighbourhood indexed by m to that indexed by k exactly when k is the master index, or m is nonzero and even and its decoded right component is contained in k.

Proof. Separate the master, zero-embedded, and paired forms of the source; only a paired source has a right component. ■

Theorem (`Proposition77.proj1_isComputable`). The second projection approximable map is computable.

Proof. The criterion uses decidable tests for zero, parity, and nonzeroness, together with decidable inclusion of the decoded right component. ■

Theorem (`Proposition77.proj1_toElementMap_pairSharp`). Applying the second projection to a paired list-domain element returns its second component.

Proof. A target neighbourhood belongs to the image exactly when it contains a right-component neighbourhood of the pair. ■

`listCombinators.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .dsharpEffective
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .prodEffective
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap IndefProduct

section
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (hD : ∀ X, D.mem X → X.Nonempty)

def proj1Map : ApproximableMap (Dsharp D hD) (Dsharp D hD) where
  rel W Z := MemS D W ∧ MemS D Z ∧ (Z = Gamma D ∨ ∃ Pp Qq, W = embPair Pp Qq ∧ Qq ⊆ Z)
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨MemS.gamma, MemS.gamma, Or.inl rfl⟩
  inter_right := by
    rintro W Z Z' ⟨hW, hZ, hd⟩ ⟨-, hZ', hd'⟩
    have hgZ : Z ⊆ Gamma D := memS_subset_gamma hZ
    have hgZ' : Z' ⊆ Gamma D := memS_subset_gamma hZ'
    rcases hd with rfl | ⟨Pp, Qq, hWeq, hQZ⟩
    · rw [Set.inter_eq_right.mpr hgZ']; exact ⟨hW, hZ', hd'⟩
    · rcases hd' with rfl | ⟨Pp', Qq', hWeq', hQZ'⟩
      · rw [Set.inter_eq_left.mpr hgZ]; exact ⟨hW, hZ, Or.inr ⟨Pp, Qq, hWeq, hQZ⟩⟩
      · have hQQ : Qq = Qq' := (embPair_injective (hWeq.symm.trans hWeq')).2
        have hQsub : Qq ⊆ Z ∩ Z' := Set.subset_inter hQZ (by rw [hQQ]; exact hQZ')
        have hMemQ : MemS D Qq := (memS_embPair_inv hD (hWeq ▸ hW)).2
        exact ⟨hW, memS_inter hD hZ hZ' hMemQ hQsub, Or.inr ⟨Pp, Qq, hWeq, hQsub⟩⟩
  mono := by
    rintro W W' Z Z' ⟨hW, hZ, hd⟩ hW'W hZZ' hW'mem hZ'mem
    refine ⟨hW'mem, hZ'mem, ?_⟩
    rcases hd with rfl | ⟨Pp, Qq, hWeq, hQZ⟩
    · exact Or.inl (Set.Subset.antisymm (memS_subset_gamma hZ'mem) hZZ')
    · subst hWeq
      cases hW'mem with
      | gamma =>
        exact absurd (Set.Subset.antisymm (memS_subset_gamma hW) hW'W)
          (embPair_ne_Gamma D hD Pp Qq)
      | @zero X hX =>
        exfalso
        obtain ⟨a, ha⟩ := hD X hX
        rcases hW'W (show (([], a) : List Bool × α) ∈ embZero X from ⟨rfl, ha⟩) with
          ⟨p', hp', -⟩ | ⟨q', hq', -⟩
        · simp at hp'
        · simp at hq'
      | @pair Pp' Qq' hP' hQ' =>
        obtain ⟨-, hQ'Q⟩ := embPair_subset.mp hW'W
        exact Or.inr ⟨Pp', Qq', rfl, fun z hz => hZZ' (hQZ (hQ'Q hz))⟩

@[simp] theorem proj1Map_rel {W Z : Set (List Bool × α)} :
    (proj1Map hD).rel W Z ↔
      MemS D W ∧ MemS D Z ∧ (Z = Gamma D ∨ ∃ Pp Qq, W = embPair Pp Qq ∧ Qq ⊆ Z) := Iff
    .rfl

def pairSharpMap : ApproximableMap (prod (Dsharp D hD) (Dsharp D hD)) (Dsharp D hD)
```

```
    where
  rel V W := (prod (Dsharp D hD) (Dsharp D hD)).mem V ∧ MemS D W ∧
    ∃ A B, MemS D A ∧ MemS D B ∧ V = prodNbhd A B ∧ embPair A B ⊆ W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(prod (Dsharp D hD) (Dsharp D hD)).master_mem, MemS.gamma,
    ⟨Gamma D, Gamma D, MemS.gamma, MemS.gamma, rfl, embPair_subset_Gamma subset_rfl
    subset_rfl⟩⟩
  inter_right := by
    rintro V W W' ⟨hV, hW, A, B, hA, hB, rfl, hsub⟩ ⟨-, hW', A', B', hA', hB', heq,
    hsub'⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact ⟨hV, memS_inter hD hW hW' (MemS.pair hA hB) (Set.subset_inter hsub hsub'),
      A, B, hA, hB, rfl, Set.subset_inter hsub hsub'⟩
  mono := by
    rintro V V' W W' ⟨hV, hW, A, B, hA, hB, rfl, hsub⟩ hV'V hWW' hV'mem hW'mem
    obtain ⟨A', B', hA', hB', rfl⟩ := hV'mem
    obtain ⟨hA'A, hB'B⟩ := prodNbhd.subset_iff.mp hV'V
    exact ⟨prod_mem_prodNbhd hA' hB', hW'mem, A', B', hA', hB', rfl,
      (embPair_subset.mpr ⟨hA'A, hB'B⟩).trans (hsub.trans hWW')⟩

@[simp] theorem pairSharpMap_rel {V : Set ((List Bool × α) ? (List Bool × α))}
    {W : Set (List Bool × α)} :
    (pairSharpMap hD).rel V W ↔
      (prod (Dsharp D hD) (Dsharp D hD)).mem V ∧ MemS D W ∧
        ∃ A B, MemS D A ∧ MemS D B ∧ V = prodNbhd A B ∧ embPair A B ⊆ W := Iff.rfl

theorem pairSharpMap_toElementMap (x y : (Dsharp D hD).Element) :
    (pairSharpMap hD).toElementMap (DomainTheory.NeighbourhoodSystems.pair x y) =
    pairSharp D hD x y := by
  apply Element.ext
  intro W
  simp only [ApproximableMap.mem_toElementMap, pairSharpMap_rel, mem_pair]
  constructor
  · rintro ⟨V, ⟨A0, B0, hxA0, hyB0, rfl⟩, -, hW, A, B, -, -, heq, hsub⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact (pairSharp D hD x y).up_mem (Or.inr ⟨A0, B0, hxA0, hyB0, rfl⟩) hW hsub
  · intro hW
    rcases hW with rfl | ⟨P, Q, hxP, hyQ, rfl⟩
    · exact ⟨prodNbhd (Gamma D) (Gamma D), ⟨Gamma D, Gamma D, x.master_mem, y.master_mem
    , rfl⟩,
        ⟨Gamma D, Gamma D, MemS.gamma, MemS.gamma, rfl⟩, MemS.gamma,
        Gamma D, Gamma D, MemS.gamma, MemS.gamma, rfl,
        embPair_subset_Gamma subset_rfl subset_rfl⟩
    · exact ⟨prodNbhd P Q, ⟨P, Q, hxP, hyQ, rfl⟩, prod_mem_prodNbhd (x.sub hxP) (y.sub
    hyQ),
        MemS.pair (x.sub hxP) (y.sub hyQ), P, Q, x.sub hxP, y.sub hyQ, rfl, subset_rfl⟩

theorem pairSharp_rel_Vsharp_iff (P : ComputablePresentation D) (t k : ℕ) :
    (pairSharpMap hD).rel
        ((prodPresentation (dsharpPresentation P hD) (dsharpPresentation P hD)).X t)
        ((dsharpPresentation P hD).X k) ↔
      Vsharp D P (2 * t + 2) ⊆ Vsharp D P k := by
  rw [pairSharpMap_rel, prodPresentation_X, Vsharp_even]
  constructor
  · rintro ⟨-, -, A, B, -, -, heq, hsub⟩
```

```
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact hsub
  · intro hsub
    exact ⟨prod_mem_prodNbhd (Vsharp_mem P _) (Vsharp_mem P _), Vsharp_mem P k,
      _, _, Vsharp_mem P _, Vsharp_mem P _, rfl, hsub⟩

end Proposition77
end
end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive IndefProduct ApproximableMap
section
variable {α : Type*}
namespace Proposition77
variable {D : NeighborhoodSystem α} (hD : ∀ X, D.mem X → X.Nonempty)

theorem pairSharp_isComputable (P : ComputablePresentation D) :
    ComputableMap (prodPresentation (dsharpPresentation P hD) (dsharpPresentation P hD
    ))
      (dsharpPresentation P hD) (pairSharpMap hD) := by
  constructor
  show REPred (fun s => (pairSharpMap hD).rel
    ((prodPresentation (dsharpPresentation P hD) (dsharpPresentation P hD)).X s.unpair
    .1)
    ((dsharpPresentation P hD).X s.unpair.2))
  have hincl : RecDecidable (fun s => Vsharp D P s.unpair.1 ⊆ Vsharp D P s.unpair.2)
    :=
    (dsharpPresentation P hD).incl_computable
  have hr : Nat.Primrec (fun s => Nat.pair (2 * s.unpair.1 + 2) s.unpair.2) :=
    (primrec_add₂ (primrec_mul₂ (Nat.Primrec.const 2) Nat.Primrec.left)
      (Nat.Primrec.const 2)).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun s => ?_) (hincl.comp hr)).re
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [pairSharp_rel_Vsharp_iff hD P s.unpair.1 s.unpair.2]

theorem proj1_rel_Vsharp_iff (P : ComputablePresentation D) (m k : ℕ) :
    (proj1Map hD).rel (Vsharp D P m) (Vsharp D P k) ↔
      k = 0 ∨ (m % 2 = 0 ∧ m = 0 ∧ Vsharp D P (m / 2 - 1).unpair.2 ⊆ Vsharp D P k) :=
    by
  rw [proj1Map_rel, and_iff_right (Vsharp_mem P m), and_iff_right (Vsharp_mem P k)]
  refine or_congr (Vsharp_eq_Gamma_iff P hD k) ?_
  rcases nat_shape m with rfl | ⟨a, rfl⟩ | ⟨a, rfl⟩
  · rw [Vsharp_zero]
    constructor
    · rintro ⟨Pp, Qq, heq, -⟩; exact absurd heq.symm (embPair_ne_Gamma D hD Pp Qq)
    · rintro ⟨-, hne, -⟩; exact absurd rfl hne
  · rw [Vsharp_odd]
    constructor
    · rintro ⟨Pp, Qq, heq, -⟩; exact absurd heq (embZero_ne_embPair D hD (P.mem_X a) Pp
    Qq)
    · rintro ⟨hmod, -, -⟩; exact absurd hmod (by omega)
  · rw [Vsharp_even, show (2 * a + 2) / 2 - 1 = a from by omega]
    constructor
    · rintro ⟨Pp, Qq, heq, hQk⟩
      obtain ⟨-, hQ⟩ := embPair_injective heq
```

```
      exact ⟨by omega, by omega, by rw [hQ]; exact hQk⟩
    · rintro ⟨-, -, hsub⟩
      exact ⟨Vsharp D P a.unpair.1, Vsharp D P a.unpair.2, rfl, hsub⟩

theorem proj1_isComputable (P : ComputablePresentation D) :
    ComputableMap (dsharpPresentation P hD) (dsharpPresentation P hD) (proj1Map hD) :=
     by
  constructor
  show REPred (fun s => (proj1Map hD).rel (Vsharp D P s.unpair.1) (Vsharp D P s.unpair
    .2))
  have hk0 : RecDecidable (fun s => s.unpair.2 = 0) :=
    RecDecidable.natEq Nat.Primrec.right (Nat.Primrec.const 0)
  have hmod : RecDecidable (fun s => s.unpair.1 % 2 = 0) :=
    RecDecidable.natEq (primrec_mod2.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have hne : RecDecidable (fun s => s.unpair.1 = 0) :=
    (RecDecidable.natEq Nat.Primrec.left (Nat.Primrec.const 0)).not
  have hg : Nat.Primrec (fun s => Nat.pair (s.unpair.1 / 2 - 1).unpair.2 s.unpair.2)
    :=
    (Nat.Primrec.right.comp (primrec_sub₂ (primrec_div2.comp Nat.Primrec.left)
      (Nat.Primrec.const 1))).pair Nat.Primrec.right
  have hincl : RecDecidable (fun s => Vsharp D P (s.unpair.1 / 2 - 1).unpair.2 ⊆
     Vsharp D P s.unpair.2) := by
    refine RecDecidable.of_iff (fun s => ?_) ((dsharpPresentation P hD).
    incl_computable.comp hg)
    simp only [dsharpPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact (RecDecidable.of_iff (fun s => proj1_rel_Vsharp_iff hD P s.unpair.1 s.unpair
    .2)
    (hk0.or (hmod.and (hne.and hincl)))).re

end Proposition77
end
end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap IndefProduct
variable {α : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem α} (hD : ∀ X, D.mem X → X.Nonempty)

theorem proj1_toElementMap_pairSharp (x y : (Dsharp D hD).Element) :
    (proj1Map hD).toElementMap (pairSharp D hD x y) = y := by
  apply Element.ext
  intro Z
  simp only [ApproximableMap.mem_toElementMap, proj1Map_rel]
  constructor
  · rintro ⟨W, hWmem, hMW, hMZ, hdisj⟩
    rcases hWmem with rfl | ⟨Pp, Qq, hxP, hyQ, rfl⟩
    · rcases hdisj with rfl | ⟨Pp', Qq', hPe, -⟩
      · exact y.master_mem
      · exact absurd hPe.symm (embPair_ne_Gamma D hD Pp' Qq')
    · rcases hdisj with rfl | ⟨Pp', Qq', hPe, hQ'Z⟩
      · exact y.master_mem
      · obtain ⟨-, hQQ⟩ := embPair_injective hPe
```

```
        exact y.up_mem hyQ hMZ (by rw [hQQ]; exact hQ'Z)
  · intro hyZ
    exact ⟨embPair (Gamma D) Z, Or.inr ⟨Gamma D, Z, x.master_mem, hyZ, rfl⟩,
      MemS.pair MemS.gamma (y.sub hyZ), y.sub hyZ, Or.inr ⟨Gamma D, Z, rfl, subset_rfl⟩
    ⟩

end Proposition77
end
end DomainTheory.NeighbourhoodSystems
```

4.8.18 Proposition 7.7 — lists effectively given

PROPOSITION 7.7. For any effectively given domain D, the domain $D^{\S}$ is also effectively given, and all the combinators of Example 6.1 prove to be computable.

Proof: This proof is essentially an exercise, but it is useful to have an easy-to-grasp example. Indeed, to make things easy to reason about, we can assume that D is a system over $\Delta = \mathbf{N}$, and that in the presentation where

$$D = \{X_n \mid n \in \mathbf{N}\},$$

the relation $k \in X_n$ is *recursive* in k and n. (It is worth thinking why this is so.) Of course, a lot of other things are recursive also.

Now what kind of a system is $D^{\S}$? The construction of 6.1 made it a system over a certain set of strings Γ. For the sake of checking various assertions about computability, we are transposing everything back to $\mathbf{N}$. (These are all denumerable sets in any case.) The set Γ is divided into three equally big parts, and we can do the same for $\mathbf{N}$. Let us write for any $m, k \in \mathbf{N}$ and subset $X \subseteq \mathbf{N}$:

$$mX + k = \{m \cdot n + k \mid n \in X\}.$$

Then by splitting the integers modulo 3 we have:

$$\mathbf{N} = 3\mathbf{N} \cup (3\mathbf{N} + 1) \cup (3\mathbf{N} + 2),$$

and this equation is quite analogous to that for Γ. We then propose this definition for $D^{\S}$:

$$D^{\S} = \{\mathbf{N}\} \cup \{3X \mid X \in D\} \cup \{(3X + 1) \cup (3Y + 2) \mid X, Y \in D^{\S}\},$$

but this does not make the enumeration of $D^{\S}$ all that obvious. This is one way to do it:

$$V_0 = \mathbf{N}\ ;\ V_{2n+1} = 3X_n\ ;\ V_{2n+2} = (3V_{p(n)} + 1) \cup (3V_{q(n)} + 2).$$

Here p and q are the inverse of the pairing functions mentioned in 7.4. They must be chosen so that $p(n) < n$ and $q(n) < n$ for all $n \in \mathbf{N}$. Thus, in calculating V_k where $k = 2n + 2$ we will be using $V_{p(n)}$ and $V_{q(n)}$ where both subscripts are strictly less than k. This observation is required so that $m \in V_k$ is going to be a recursive relation. What we claim is that

$$D^{\S} = \{V_k \mid k \in \mathbf{N}\}.$$

It should be clear that everything on the right belongs to $D^{\S}$. What needs an inductive argument is that everything in $D^{\S}$ is eventually of the form V_k. But this should be fairly obvious owing to the properties of $r : \mathbf{N} \times \mathbf{N} \leftrightarrow \mathbf{N}$.

The reader also has to check that 7.1(i)–(ii) hold for the V_k. The idea is that any such check is either (1) trivial, or (2) something already assumed about D and the X_n, or (3) can be thrown back to some sets V_m with strictly smaller subscripts. Therefore, the checks will give an answer in finite time according to an effective reduction.

Next for the combinators, we have to translate neighbourhood relations into relations among integer indices. A selection of examples must suffice,

$$X_n\ (\lambda x.\, x^{\S})\ V_k \text{ iff } V_{2n+1} \subseteq V_k$$

$$V_m\ \mathrm{proj}_0\ V_k \text{ iff } k = 0 \text{ or } \exists n \in \mathbf{N}.\, m = 2n+2 \text{ and } V_{p(n)} \subseteq V_k.$$

The reader should write out other cases. □

The development is with the effectively given lists and list combinators peers.

Theorem (`proposition_77`). For an effectively given neighbourhood system, the domain of lists over it is effectively given.

Proof. By the effective presentation of the list domain. ■

`Proposition_7_7.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .dsharpEffective
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_18

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Proposition77
set_option linter.unusedVariables false
open NeighborhoodSystem Domain.Recursive IndefProduct

/-- Scott's Proposition 7.7: D? is effectively given. -/
theorem proposition_77 :
    ∀ {α : Type*} {D : NeighborhoodSystem α} (P : ComputablePresentation D)
      (hD : ∀ X, D.mem X → X.Nonempty),
      (Dsharp D hD).IsEffectivelyGiven :=
  @dsharp_isEffectivelyGiven

end DomainTheory.NeighbourhoodSystems.Proposition77
```

4.8.19 Powerset neighbourhoods

The powerset of the naturals is a neighbourhood system whose neighbourhoods are the cofinite complements of finite sets of bits.

Definition (`nbhd`). The neighbourhood coded by a natural number n consists of the bit positions at which n has value false.

Theorem (`mem_nbhd`). A natural number k belongs to the neighbourhood coded by n exactly when the k-th bit of n is false.

Proof. This is the defining membership condition. ■

Theorem (`nbhd_injective`). Distinct natural-number codes determine distinct neighbourhoods.

Proof. Equality of the neighbourhoods gives equality of the truth value of every bit, and natural numbers are determined by their bits. ■

Theorem (`nbhd_inter`). The intersection of the neighbourhoods coded by n and m is the neighbourhood coded by their bitwise disjunction.

Proof. A bit is false in the disjunction exactly when it is false in both operands. ■

Theorem (`nbhd_zero`). The neighbourhood coded by zero is the whole set of natural numbers.

Proof. Every bit of zero is false. ■

Definition (`PN`). The powerset neighbourhood system on the natural numbers has precisely the coded neighbourhoods, with the whole set as master neighbourhood.

Proof. Zero codes the master neighbourhood, and bitwise disjunction codes intersections. ■

Definition (`PNpres`). The canonical computable presentation enumerates the neighbourhood coded by n at index n, uses bitwise disjunction for intersection, and uses zero for the master index.

Proof. Injectivity of the coding decides equality, bitwise disjunction computes intersections, and every pair of neighbourhoods is consistent. ■

Theorem (`PN_consistent`). Any two coded neighbourhoods have a coded neighbourhood contained in their intersection.

Proof. Their bitwise disjunction codes the intersection itself. ■

Theorem (`PN_isEffectivelyGiven`). The powerset neighbourhood system is effectively given.

Proof. The canonical computable presentation supplies the required effective enumeration and operations. ■

`powersetNbhd.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_1
public import Mathlib.Data.Nat.Bitwise

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open Domain.Recursive NeighborhoodSystem

def nbhd (n : ℕ) : Set ℕ := {k | n.testBit k = false}

@[simp] theorem mem_nbhd {n k : ℕ} : k ∈ nbhd n ↔ n.testBit k = false := Iff.rfl

theorem nbhd_injective {n m : ℕ} (h : nbhd n = nbhd m) : n = m := by
  apply Nat.eq_of_testBit_eq
  intro k
  have hk : (n.testBit k = false) ↔ (m.testBit k = false) := by
    have := Set.ext_iff.mp h k; simpa only [mem_nbhd] using this
  cases hn : n.testBit k <;> cases hm : m.testBit k <;> simp_all

theorem nbhd_inter (n m : ℕ) : nbhd n ∩ nbhd m = nbhd (myLor n m) := by
  ext k
```

```
  simp only [nbhd, Set.mem_inter_iff, Set.mem_ofPred_eq, myLor_eq_lor, Nat.testBit_lor
    ,
    Bool.or_eq_false_iff]

theorem nbhd_zero : nbhd 0 = Set.univ := by
  ext k; simp [nbhd, Nat.zero_testBit]

def PN : NeighborhoodSystem ℕ where
  mem Y := ∃ n, Y = nbhd n
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := ⟨0, nbhd_zero.symm⟩
  inter_mem := by
    rintro X Y Z ⟨a, rfl⟩ ⟨b, rfl⟩ _ _
    exact ⟨myLor a b, nbhd_inter a b⟩
  sub_master := by rintro X _; exact Set.subset_univ X

end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
open Domain.Recursive NeighborhoodSystem

def PNpres : ComputablePresentation PN where
  X := nbhd
  mem_X n := ⟨n, rfl⟩
  surj := by rintro Y ⟨n, rfl⟩; exact ⟨n, rfl⟩
  interEq_computable := by

    refine RecDecidable.of_iff (q := fun t =>
        nbhd t.unpair.1 ∩ nbhd t.unpair.2.unpair.1 = nbhd t.unpair.2.unpair.2)
      (fun t => ?_)
      (RecDecidable.natEq
        (a := fun t => myLor t.unpair.1 t.unpair.2.unpair.1)
        (b := fun t => t.unpair.2.unpair.2)
        ((primrec_myLor.comp
          (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right))).of_eq fun
     s => by
            simp only [unpair_pair_fst, unpair_pair_snd])
        (Nat.Primrec.right.comp Nat.Primrec.right))

    constructor
    · intro h; exact nbhd_injective ((nbhd_inter _ _).symm.trans h)
    · intro h; rw [nbhd_inter]; exact congrArg nbhd h
  cons_computable :=
    recDecidable_of_forall fun t => ⟨myLor t.unpair.1 t.unpair.2, (nbhd_inter _ _).symm
    .subset⟩
  inter := myLor
  inter_primrec := primrec_myLor
  inter_spec := fun {n m} _ => (nbhd_inter n m).symm
  masterIdx := 0
  masterIdx_spec := nbhd_zero

theorem PN_consistent (n m : ℕ) : ∃ k, nbhd k ⊆ nbhd n ∩ nbhd m :=
  ⟨myLor n m, (nbhd_inter n m).symm.subset⟩

theorem PN_isEffectivelyGiven : PN.IsEffectivelyGiven := ⟨PNpres⟩
```

```
end DomainTheory.NeighbourhoodSystems
```

4.8.20 Example 7.8 — powerset effectively given

EXAMPLE 7.8. We have often made reference to the powerset $P\mathbf{N}$ as a domain and we should check here that it is effectively given. One easy way to see this is to note

$$P\mathbf{N} \cong |T^\infty|.$$

The (slight) trouble with $P\mathbf{N}$ is that we usually think of it in terms of *elements* rather than *neighbourhoods*. Going back to Exercise 1.16, we can argue that the neighbourhoods of $P\mathbf{N}$ are ordered not like the finite sets of integers but in the partial ordering *converse* to that. But this is of no trouble, since all will be decidable. What we need first is an enumeration of all finite sets of integers. We can do this by:

$$E_n = \{k \mid \exists i, j.\, i < 2^k \text{ and } n = i + 2^k + j \cdot 2^{k+1}\}.$$

The idea is that $k \in E_n$ means that the exponent k does occur in the binary expansion of n as a sum of powers of 2. All finite subsets of $\mathbf{N}$ are of the form E_n. We then find that as a neighbourhood system

$$(P\mathbf{N}) = \{\mathbf{N} \setminus E_n \mid n \in \mathbf{N}\}.$$

As the relationship $E_n \cup E_m = E_k$ is recursive, there is no trouble in proving that this is a computable presentation. In this system, of course, any two neighbourhoods are consistent. Various combinators on $P\mathbf{N}$ are suggested in Exercise 7.23. □

We end this chapter with an example of another kind of domain construct. This construct is known as the *Smyth Power Domain*. It is defined for any neighbourhood system $\mathcal{D}$ and results in a new system we shall call here $\mathbb{P}\mathcal{D}$. The elements of $\mathbb{P}\mathcal{D}$ behave rather like *sets of elements* of $\mathcal{D}$, but since our elements can be either partial or total, there are certain dangers to pushing the analogy too far. For some purposes a rival construct called the *Plotkin Power Domain* is better, but it leads outside the category of neighbourhood systems as defined in these lectures. Do not confuse $P\mathbf{N}$ with $\mathbb{P}\mathcal{D}$.

The development is with the powerset neighbourhoods peer.

Theorem (`example_78`). The powerset of the natural numbers is effectively given.

Proof. By the powerset presentation. ■

Example_7_8.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .powersetNbhd

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open Domain.Recursive

/-- Scott's Example 7.8: the powerset of the naturals is effectively given. -/
theorem example_78 : PN.IsEffectivelyGiven :=
  PN_isEffectivelyGiven
```

4.8.21 Power domain

The powerdomain of a neighbourhood system consists of the neighbourhoods that are finite unions of upper sets.

Definition (`NeighborhoodSystem.PDmem`). A powerdomain neighbourhood is a finite union of upper sets generated by neighbourhoods of the original system.

Definition (`NeighborhoodSystem.dagger`). The dagger system is the upper-set power system associated with the original neighbourhood system.

Theorem (`NeighborhoodSystem.PDmem_union`). The union of two powerdomain neighbourhoods is a powerdomain neighbourhood.

Proof. Concatenate finite lists of generators; the resulting union is the union of the two original finite unions. ■

Theorem (`NeighborhoodSystem.dagger_isomorphic`). Every neighbourhood system is domain-isomorphic to its dagger system.

Proof. This is the canonical isomorphism between a system and its upper-set power presentation. ■

Theorem (`NeighborhoodSystem.dagger_upSet_inter`). If X, Y, Z are neighbourhoods and $Z \subseteq X \cap Y$, then the intersection of the upper sets generated by X and Y is a dagger neighbourhood.

Proof. The intersection is the upper set generated by $X \cap Y$, which is a neighbourhood because it contains the neighbourhood Z. ■

Theorem (`NeighborhoodSystem.mem_PDunion`). A set belongs to the finite union generated by a list exactly when it belongs to the upper set generated by some member of the list.

Proof. This is the elementwise meaning of a union indexed first by the list and then by membership in it. ■

Theorem (`NeighborhoodSystem.PDmem_empty`). The empty family is a powerdomain neighbourhood.

Proof. It is generated by the empty list. ■

Theorem (`NeighborhoodSystem.PDmem_iff_fin`). A family is a powerdomain neighbourhood exactly when it is a union of a finite indexed family of upper sets generated by neighbourhoods.

Proof. Pass between a finite list and its indexing by the finite set of positions. ■

Theorem (`NeighborhoodSystem.PDmem_upSet`). The upper set generated by any neighbourhood is a powerdomain neighbourhood.

Proof. Use the singleton list of generators. ■

Theorem (`NeighborhoodSystem.PDmem_master`). The upper set generated by the master neighbourhood is a powerdomain neighbourhood.

Proof. Apply the singleton construction to the master neighbourhood. ■

Theorem (`NeighborhoodSystem.upSet_inter_nonempty_iff`). Two generated upper sets have nonempty intersection exactly when some neighbourhood is contained in the intersection of their generators.

Proof. A member of both upper sets is precisely a neighbourhood contained in both generators. ■

powerDomain.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem_more
public import Mathlib.Data.Set.Lattice.Indexed
public import Mathlib.Data.List.Basic
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section
variable {α : Type*} (V : NeighborhoodSystem α)

def PDmem (W : Set (Set α)) : Prop :=
  ∃ L : List (Set α), (∀ X ∈ L, V.mem X) ∧ W = ⋃ X ∈ L, V.upSet X

abbrev dagger : NeighborhoodSystem (Set α) := V.powerSystem

theorem PDmem_union {W₁ W₂ : Set (Set α)} (h₁ : V.PDmem W₁) (h₂ : V.PDmem W₂) :
    V.PDmem (W₁ ∪ W₂) := by
  obtain ⟨L₁, hL₁, rfl⟩ := h₁
  obtain ⟨L₂, hL₂, rfl⟩ := h₂
  refine ⟨L₁ ++ L₂, ?_, ?_⟩
  · intro X hX
    rcases List.mem_append.mp hX with h | h
    · exact hL₁ X h
    · exact hL₂ X h
  · ext z
    simp only [Set.mem_union, Set.mem_iUnion, exists_prop, List.mem_append]
    constructor
    · rintro (⟨X, hX, hz⟩ | ⟨X, hX, hz⟩)
      · exact ⟨X, Or.inl hX, hz⟩
      · exact ⟨X, Or.inr hX, hz⟩
    · rintro ⟨X, hX | hX, hz⟩
      · exact Or.inl ⟨X, hX, hz⟩
      · exact Or.inr ⟨X, hX, hz⟩

theorem dagger_isomorphic : V ≅? V.dagger := V.isomorphic_powerSystem

theorem dagger_upSet_inter {X Y Z : Set α} (hX : V.mem X) (hY : V.mem Y) (hZ : V.mem Z
    )
    (hsub : Z ⊆ X ∩ Y) : V.dagger.mem (V.upSet X ∩ V.upSet Y) := by
  rw [V.upSet_inter]
  exact ⟨X ∩ Y, V.inter_mem hX hY hZ hsub, rfl⟩

theorem mem_PDunion {L : List (Set α)} {z : Set α} :
    z ∈ (⋃ X ∈ L, V.upSet X) ↔ ∃ X ∈ L, z ∈ V.upSet X := by
  simp only [Set.mem_iUnion, exists_prop]

theorem PDmem_empty : V.PDmem ∅ := by
  refine ⟨[], ?_, ?_⟩
  · intro X hX; cases hX
  · ext z
    rw [mem_PDunion]
    constructor
    · intro h; exact absurd h (Set.notMem_empty z)
    · rintro ⟨X, hX, _⟩; cases hX

theorem PDmem_iff_fin {W : Set (Set α)} :
    V.PDmem W ↔ ∃ (n : ℕ) (X : Fin n → Set α),
      (∀ i, V.mem (X i)) ∧ W = ⋃ i, V.upSet (X i) := by
```

```
  constructor
  · rintro ⟨L, hL, rfl⟩
    refine ⟨L.length, L.get, fun i => hL _ (L.get_mem i), ?_⟩
    ext z
    rw [mem_PDunion, Set.mem_iUnion]
    constructor
    · rintro ⟨X, hX, hz⟩
      obtain ⟨i, rfl⟩ := List.get_of_mem hX
      exact ⟨i, hz⟩
    · rintro ⟨i, hz⟩
      exact ⟨L.get i, L.get_mem i, hz⟩
  · rintro ⟨n, X, hX, rfl⟩
    refine ⟨List.ofFn X, ?_, ?_⟩
    · intro Y hY
      obtain ⟨i, rfl⟩ := Set.mem_range.mp (List.mem_ofFn.mp hY)
      exact hX i
    · ext z
      rw [Set.mem_iUnion, mem_PDunion]
      constructor
      · rintro ⟨i, hz⟩
        exact ⟨X i, List.mem_ofFn.mpr (Set.mem_range.mpr ⟨i, rfl⟩), hz⟩
      · rintro ⟨Y, hY, hz⟩
        obtain ⟨i, rfl⟩ := Set.mem_range.mp (List.mem_ofFn.mp hY)
        exact ⟨i, hz⟩

theorem PDmem_upSet {X : Set α} (hX : V.mem X) : V.PDmem (V.upSet X) := by
  refine ⟨[X], ?_, ?_⟩
  · intro Y hY; rw [List.mem_singleton] at hY; subst hY; exact hX
  · ext z
    rw [mem_PDunion]
    constructor
    · intro hz; exact ⟨X, List.mem_singleton.mpr rfl, hz⟩
    · rintro ⟨Y, hY, hz⟩; rw [List.mem_singleton] at hY; subst hY; exact hz

theorem PDmem_master : V.PDmem (V.upSet V.master) := V.PDmem_upSet V.master_mem

theorem upSet_inter_nonempty_iff {X Y : Set α} :
    (V.upSet X ∩ V.upSet Y).Nonempty ↔ ∃ Z, V.mem Z ∧ Z ⊆ X ∩ Y := by
  rw [V.upSet_inter]
  constructor
  · rintro ⟨Z, hZmem, hZsub⟩; exact ⟨Z, hZmem, hZsub⟩
  · rintro ⟨Z, hZmem, hZsub⟩; exact ⟨Z, hZmem, hZsub⟩

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
```

4.8.22 Definition 7.9 — power domain

DEFINITION 7.9. Let $\mathcal{D}$ be any neighbourhood system and define

$$\mathbb{P}\mathcal{D} = \left\{ \bigcup_{i<n} (\downarrow X_i) \mid \forall i < n.\, X_i \in \mathcal{D} \right\}.$$

We recall that for any $X \in \mathcal{D}$

$$\downarrow X = \{Y \in \mathcal{D} \mid Y \subseteq X\}.$$

The finite unions in $\mathbb{P}\mathcal{D}$ can be empty (i.e. if $n = 0$). □

Formally, the system $\mathbb{P}\mathcal{D}$ is just more or less the closure of $\mathcal{D}$ under finite unions; however, this would not be an isomorphism-invariant construct unless $\mathcal{D}$ is "prepared". The preparation consists of replacing $\mathcal{D}$ by the isomorphic domain

$$\mathcal{D}^\dagger = \{\downarrow X \mid X \in \mathcal{D}\}.$$

(In this connection refer back to Exercise 1.20.) We remark that

$$\downarrow X \cap \downarrow Y \neq \emptyset \text{ iff } \{X, Y\} \text{ is consistent in } \mathcal{D},$$

and in that case

$$\downarrow X \cap \downarrow Y = \downarrow (X \cap Y).$$

Definition_7_9.lean

@import "Definition_7_9.lean"

The development is with the power domain peer.

Theorem (`definition_79`). The dagger of a neighbourhood system is isomorphic to it, and the upper set of the master neighbourhood lies in the powerdomain.

Proof. By the powerdomain. ■

Definition_7_9.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .powerDomain

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

/-- Scott's Definition 7.9: the powerdomain and the dagger embedding. -/
theorem definition_79 {α : Type*} (V : NeighborhoodSystem α) :
    (V ≅? V.dagger) ∧ V.PDmem (V.upSet V.master) :=
  ⟨V.dagger_isomorphic, V.PDmem_master⟩

end DomainTheory.NeighbourhoodSystems
```

4.8.23 Effective powerdomain

The powerdomain of an effectively given neighbourhood system is effectively given.

Definition (`NeighborhoodSystem.PowerDomain`). The powerdomain neighbourhood system has finite unions of generated upper sets as neighbourhoods and the upper set of the original master neighbourhood as master.

Proof. Finite unions are closed under the required intersections and every such family lies below the master family. ■

Theorem (`NeighborhoodSystem.PowerDomain_mem`). Membership as a powerdomain neighbourhood is exactly being a finite union of upper sets generated by original neighbourhoods.

Proof. This is the defining neighbourhood predicate. ■

Theorem (`NeighborhoodSystem.PowerDomain_master`). The master powerdomain neighbourhood is the upper set generated by the original master neighbourhood.

Proof. This is the defining master. ■

Definition (`NeighborhoodSystem.UPX`). For a finite list of indices, UPX is the union of the upper sets generated by the enumerated neighbourhoods at those indices.

Theorem (`NeighborhoodSystem.UPX_def`). UPX is the finite union of the generated upper sets named by its index list.

Proof. This restates the definition. ■

Definition (`NeighborhoodSystem.Ypd`). The powerdomain neighbourhood coded by c is UPX of the finite list decoded from c.

Theorem (`NeighborhoodSystem.Ypd_def`). Decoding c and forming its finite generated union gives $Ypd(c)$.

Proof. This restates the definition. ■

Definition (`NeighborhoodSystem.innerInterStp`). For a fixed outer generator, the inner intersection step examines another generator and prepends their computed intersection exactly when the pair is consistent.

Definition (`NeighborhoodSystem.innerInterCode`). The inner intersection code folds the inner step through a coded finite list, beginning with a given accumulator.

Theorem (`NeighborhoodSystem.innerInterStp_eq`). On an explicitly packed state, the inner step either prepends the computed intersection index or leaves the accumulator unchanged, according as the consistency test returns one.

Proof. Unpack the state into the current generator, accumulator, and fixed generator. ■

Theorem (`NeighborhoodSystem.innerInterCode_eq`). The inner code equals the corresponding left fold over the decoded list.

Proof. Coded iteration agrees with list iteration, and the packed step has the preceding explicit form. ■

Theorem (`NeighborhoodSystem.mem_UPX`). A set belongs to $UPX(l)$ exactly when it belongs to the upper set of some enumerated neighbourhood whose index occurs in l.

Proof. This is membership in the finite indexed union. ■

Theorem (`NeighborhoodSystem.UPX_cons`). Adding b to the front of a list adds the upper set generated by the neighbourhood indexed by b to the union.

Proof. Split list membership between the head and tail. ■

Theorem (`NeighborhoodSystem.UPX_nil`). The union generated by the empty list is empty.

Proof. The empty list has no generator. ■

Theorem (`NeighborhoodSystem.Ypd_cons_code`). A nonzero list code obtained by prepending v codes the union of the upper set at v with the family coded by the accumulator.

Proof. Decode the cons code and apply the head-tail formula for UPX. ■

Theorem (`NeighborhoodSystem.Ypd_zero`). The zero code denotes the empty powerdomain neighbourhood.

Proof. Zero decodes to the empty list. ■

Theorem (`NeighborhoodSystem.PDmem_exists_Ypd_aux`). Every finite union generated by a list of original neighbourhoods is equal to $Ypd(c)$ for some code c.

Proof. Induct on the generator list, use surjectivity of the original presentation for the head, and prepend its index to the code for the tail. ■

Theorem (`NeighborhoodSystem.PDmem_exists_Ypd`). Every powerdomain neighbourhood has a Ypd code.

Proof. Choose its finite generator list and apply the coded-list construction. ■

Theorem (`NeighborhoodSystem.mem_Ypd`). A set belongs to $Ypd(c)$ exactly when it belongs to the upper set generated by some index in the list decoded from c.

Proof. Apply the membership characterization of UPX. ■

Theorem (`NeighborhoodSystem.Ypd_isPDmem`). Every $Ypd(c)$ is a powerdomain neighbourhood.

Proof. Map the decoded index list to its enumerated neighbourhoods; their finite union is exactly $Ypd(c)$. ■

Definition (`NeighborhoodSystem.outerInterStp`). The outer intersection step runs the inner intersection code for the current generator against the second coded list.

Definition (`NeighborhoodSystem.interCode`). The intersection code folds the outer step over the first coded list, starting with the empty code.

Theorem (`NeighborhoodSystem.outerInterStp_eq`). On an explicitly packed state, the outer step is the inner intersection code for the unpacked generator, accumulator, and second list.

Proof. This follows by unpacking the state. ■

Theorem (`NeighborhoodSystem.interCode_eq`). The intersection code is the left fold of the inner intersection operation over the decoded first list.

Proof. Coded iteration agrees with list iteration and the outer step has the preceding explicit form. ■

Theorem (`NeighborhoodSystem.primrec_innerInterStp`). If the consistency test is primitive recursive, then the inner intersection step is primitive recursive.

Proof. It is assembled from projections, pairing, the primitive-recursive intersection index, the consistency test, and numerical selection. ■

Theorem (`NeighborhoodSystem.primrec_outerInterStp`). If the consistency test is primitive recursive, then the outer intersection step is primitive recursive.

Proof. Primitive-recursive coded folding preserves primitive recursiveness. ■

Theorem (`NeighborhoodSystem.primrec_interCode`). If the consistency test is primitive recursive, then the binary intersection-code function is primitive recursive.

Proof. It is the coded fold of the primitive-recursive outer step. ■

Theorem (`NeighborhoodSystem.subCode_computable`). It is decidable whether every generator in one decoded list is contained in some generator in another decoded list.

Proof. Original-neighbourhood inclusion is decidable, and bounded existential and universal search over finite decoded lists preserves decidability. ■

Theorem (`NeighborhoodSystem.upSet_inter_eq_empty_of_not_cons`). If no enumerated neighbourhood lies inside the intersection of two enumerated neighbourhoods, then their generated upper sets are disjoint.

Proof. A member of both upper sets would be a neighbourhood inside the intersection and hence, by surjectivity, would yield a forbidden index. ■

Theorem (`NeighborhoodSystem.Ypd_innerstep`). One inner step changes the coded family by adjoining the intersection of the two generated upper sets.

Proof. In the consistent case the computed neighbourhood equals the intersection generator; in the inconsistent case the two upper sets are disjoint and the accumulator is unchanged. ■

Theorem (`NeighborhoodSystem.Ypd_innerfoldl`). Folding the inner step over a list adjoins the intersection of the fixed generator's upper set with the union generated by that list.

Proof. Induct on the list and distribute intersection over union. ■

Theorem (`NeighborhoodSystem.Ypd_innerInterCode`). The inner intersection code denotes the old accumulator union the intersection of the fixed generator's upper set with the family denoted by the input code.

Proof. Replace coded iteration by its list fold and apply the inner-fold identity. ■

Theorem (`NeighborhoodSystem.Ypd_outerfoldl`). Folding inner intersection over an outer list adjoins the intersection of the union generated by that list with the second coded family.

Proof. Induct on the outer list and distribute union over intersection. ■

Theorem (`NeighborhoodSystem.Ypd_interCode`). The intersection code of n, m denotes $Ypd(n) \cap Ypd(m)$.

Proof. Apply the outer-fold identity with the empty initial code. ■

Theorem (`NeighborhoodSystem.upSet_subset_Ypd_iff`). The upper set generated at index a is contained in $Ypd(k)$ exactly when some decoded generator of k contains the neighbourhood at a.

Proof. Test inclusion on the generator itself for one direction; transitivity proves the other. ■

Theorem (`NeighborhoodSystem.Ypd_subset_iff`). $Ypd(c) \subseteq Ypd(k)$ exactly when every decoded generator of c is contained in some decoded generator of k.

Proof. A finite union is contained in a family exactly when each summand is, then apply the generated-upper-set criterion. ■

Theorem (`NeighborhoodSystem.Ypd_eq_iff`). Two coded powerdomain neighbourhoods are equal exactly when their decoded generators mutually refine one another.

Proof. Equality is mutual inclusion, characterized in each direction by the preceding theorem. ■

Theorem (`NeighborhoodSystem.eqCode_computable`). Equality of two coded powerdomain neighbourhoods is decidable.

Proof. Decide the two finite mutual-refinement conditions. ■

Theorem (`NeighborhoodSystem.Ypd_interEq_computable`). If consistency is primitive recursive with the stated specification, the ternary relation $Ypd(n) \cap Ypd(m) = Ypd(k)$ is decidable.

Proof. Compute the intersection code and decide its equality with k. ■

Definition (`NeighborhoodSystem.PDPresentation`). The effective powerdomain presentation enumerates Ypd-families, computes intersections by the coded double fold, and uses the singleton code of the original master index as master.

Proof. Coding is surjective, equality and consistency are decidable, intersection coding is primitive recursive and correct, and the chosen master code denotes the powerdomain master. ■

Theorem (`NeighborhoodSystem.PowerDomain_isEffectivelyGiven`). The powerdomain of an effectively given neighbourhood system is effectively given.

Proof. Choose an effective presentation of the original system and its computable consistency test, then form the effective powerdomain presentation. ■

`powerDomainEffective.lean`

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .computableWitness

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive
section
namespace NeighborhoodSystem
variable {α : Type*} (V : NeighborhoodSystem α)

abbrev PowerDomain : NeighborhoodSystem (Set α) where
  mem := V.PDmem
  master := V.upSet V.master
  master_nonempty := ⟨V.master, V.master_mem, subset_rfl⟩
  master_mem := V.PDmem_master
  inter_mem := fun h₁ h₂ _ _ => V.PDmem_inter h₁ h₂
  sub_master := by
    intro W hW
    obtain ⟨L, hL, rfl⟩ := hW
    intro Z hZ
    rw [V.mem_PDunion] at hZ
    obtain ⟨X, hX, hZmem, hZsub⟩ := hZ
    exact ⟨hZmem, hZsub.trans (V.sub_master (hL X hX))⟩

@[simp] theorem PowerDomain_mem {W : Set (Set α)} : V.PowerDomain.mem W ↔ V.PDmem W :=
     Iff.rfl

@[simp] theorem PowerDomain_master : V.PowerDomain.master = V.upSet V.master := rfl

end NeighborhoodSystem
end

section
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

variable (P : ComputablePresentation V)

def UPX (l : List ℕ) : Set (Set α) := ⋃ b ∈ l, V.upSet (P.X b)

theorem UPX_def (l : List ℕ) : V.UPX P l = ⋃ b ∈ l, V.upSet (P.X b) := rfl

def Ypd (c : ℕ) : Set (Set α) := V.UPX P (decodeList c)

theorem Ypd_def (c : ℕ) : V.Ypd P c = V.UPX P (decodeList c) := rfl

def innerInterStp (cons : ℕ → ℕ) (w : ℕ) : ℕ :=
  selectFn (isOne (cons (Nat.pair w.unpair.2.unpair.2 w.unpair.1)))
    (Nat.pair (P.inter w.unpair.2.unpair.2 w.unpair.1) w.unpair.2.unpair.1 + 1)
    w.unpair.2.unpair.1

def innerInterCode (cons : ℕ → ℕ) (a acc m : ℕ) : ℕ := foldCode (V.innerInterStp P
    cons) a acc m
```

```
theorem innerInterStp_eq (cons : ℕ → ℕ) (a acc b : ℕ) :
    V.innerInterStp P cons (Nat.pair b (Nat.pair acc a))
      = selectFn (isOne (cons (Nat.pair a b))) (Nat.pair (P.inter a b) acc + 1) acc :=
     by
  unfold innerInterStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem innerInterCode_eq (cons : ℕ → ℕ) (a acc m : ℕ) :
    V.innerInterCode P cons a acc m
      = List.foldl (fun acc b => selectFn (isOne (cons (Nat.pair a b)))
          (Nat.pair (P.inter a b) acc + 1) acc) acc (decodeList m) := by
  have hf : (fun (acc x : ℕ) => V.innerInterStp P cons (Nat.pair x (Nat.pair acc a)))
      = (fun acc b => selectFn (isOne (cons (Nat.pair a b))) (Nat.pair (P.inter a b)
    acc + 1) acc) := by
    funext acc b; exact V.innerInterStp_eq P cons a acc b
  unfold innerInterCode; rw [foldCode_eq', hf]

theorem mem_UPX {l : List ℕ} {z : Set α} :
    z ∈ V.UPX P l ↔ ∃ b ∈ l, z ∈ V.upSet (P.X b) := by
  simp only [UPX, Set.mem_iUnion, exists_prop]

theorem UPX_cons (b : ℕ) (l : List ℕ) :
    V.UPX P (b :: l) = V.upSet (P.X b) ∪ V.UPX P l := by
  ext z
  rw [Set.mem_union, V.mem_UPX P, V.mem_UPX P]
  constructor
  · rintro ⟨c, hc, hz⟩
    rcases List.mem_cons.mp hc with rfl | hc'
    · exact Or.inl hz
    · exact Or.inr ⟨c, hc', hz⟩
  · rintro (hz | ⟨c, hc, hz⟩)
    · exact ⟨b, List.mem_cons.mpr (Or.inl rfl), hz⟩
    · exact ⟨c, List.mem_cons.mpr (Or.inr hc), hz⟩

theorem UPX_nil : V.UPX P [] = ∅ := by
  ext z
  rw [V.mem_UPX P]
  simp only [List.not_mem_nil, false_and, exists_false, Set.mem_empty_iff_false]

theorem Ypd_cons_code (v acc : ℕ) :
    V.Ypd P (Nat.pair v acc + 1) = V.upSet (P.X v) ∪ V.Ypd P acc := by
  rw [Ypd_def, decodeList_succ, unpair_pair_fst, unpair_pair_snd, V.UPX_cons P, ← V.
    Ypd_def P]

theorem Ypd_zero : V.Ypd P 0 = ∅ := by rw [Ypd_def, decodeList_zero, V.UPX_nil P]

theorem PDmem_exists_Ypd_aux :
    ∀ (L : List (Set α)), (∀ X ∈ L, V.mem X) → ∃ c, V.Ypd P c = ⋃ X ∈ L, V.upSet X :=
     by
  intro L
  induction L with
  | nil => intro _; exact ⟨0, by rw [V.Ypd_zero P, V.upSetUnion_nil]⟩
  | cons X L ih =>
    intro hL
    obtain ⟨n, hn⟩ := P.surj (hL X (List.mem_cons.mpr (Or.inl rfl)))
    obtain ⟨c', hc'⟩ := ih (fun Y hY => hL Y (List.mem_cons.mpr (Or.inr hY)))
    exact ⟨Nat.pair n c' + 1, by rw [V.Ypd_cons_code P, hn, hc', V.upSetUnion_cons]⟩
```

```
theorem PDmem_exists_Ypd {W : Set (Set α)} (hW : V.PDmem W) : ∃ c, V.Ypd P c = W := by
  obtain ⟨L, hL, rfl⟩ := hW
  exact V.PDmem_exists_Ypd_aux P L hL

theorem mem_Ypd {c : ℕ} {z : Set α} :
    z ∈ V.Ypd P c ↔ ∃ a ∈ decodeList c, z ∈ V.upSet (P.X a) := V.mem_UPX P

theorem Ypd_isPDmem (c : ℕ) : V.PDmem (V.Ypd P c) := by
  refine ⟨(decodeList c).map P.X, ?_, ?_⟩
  · intro X hX
    obtain ⟨a, _, rfl⟩ := List.mem_map.mp hX
    exact P.mem_X a
  · ext z
    rw [V.mem_Ypd P, V.mem_PDunion]
    constructor
    · rintro ⟨a, ha, hz⟩; exact ⟨P.X a, List.mem_map.mpr ⟨a, ha, rfl⟩, hz⟩
    · rintro ⟨X, hX, hz⟩
      obtain ⟨a, ha, rfl⟩ := List.mem_map.mp hX
      exact ⟨a, ha, hz⟩

def outerInterStp (cons : ℕ → ℕ) (w : ℕ) : ℕ :=
  V.innerInterCode P cons w.unpair.1 w.unpair.2.unpair.1 w.unpair.2.unpair.2

def interCode (cons : ℕ → ℕ) (n m : ℕ) : ℕ := foldCode (V.outerInterStp P cons) m 0 n

theorem outerInterStp_eq (cons : ℕ → ℕ) (a acc m : ℕ) :
    V.outerInterStp P cons (Nat.pair a (Nat.pair acc m)) = V.innerInterCode P cons a
    acc m := by
  unfold outerInterStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem interCode_eq (cons : ℕ → ℕ) (n m : ℕ) :
    V.interCode P cons n m
      = List.foldl (fun acc a => V.innerInterCode P cons a acc m) 0 (decodeList n) :=
    by
  have hf : (fun (acc x : ℕ) => V.outerInterStp P cons (Nat.pair x (Nat.pair acc m)))
      = (fun acc a => V.innerInterCode P cons a acc m) := by
    funext acc a; exact V.outerInterStp_eq P cons a acc m
  unfold interCode; rw [foldCode_eq', hf]

theorem primrec_innerInterStp (cons : ℕ → ℕ) (hconsp : Nat.Primrec cons) :
    Nat.Primrec (V.innerInterStp P cons) := by
  have h1 : Nat.Primrec (fun w : ℕ => w.unpair.1) := Nat.Primrec.left
  have hacc : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have ha : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hpair_ab : Nat.Primrec (fun w : ℕ => Nat.pair w.unpair.2.unpair.2 w.unpair.1)
    := ha.pair h1
  have hcons_ab : Nat.Primrec (fun w : ℕ => cons (Nat.pair w.unpair.2.unpair.2 w.
    unpair.1)) :=
    hconsp.comp hpair_ab
  have hinter_ab : Nat.Primrec (fun w : ℕ => P.inter w.unpair.2.unpair.2 w.unpair.1)
    :=
    (P.inter_primrec.comp hpair_ab).of_eq (fun w => by simp only [unpair_pair_fst,
    unpair_pair_snd])
```

```
  have hprepend : Nat.Primrec
      (fun w : ℕ => Nat.pair (P.inter w.unpair.2.unpair.2 w.unpair.1) w.unpair.2.
    unpair.1 + 1) :=
    Nat.Primrec.succ.comp (hinter_ab.pair hacc)
  exact (primrec_selectFn (primrec_isOne.comp hcons_ab) hprepend hacc).of_eq (fun _ =>
     rfl)

theorem primrec_outerInterStp (cons : ℕ → ℕ) (hconsp : Nat.Primrec cons) :
    Nat.Primrec (V.outerInterStp P cons) :=
  (primrec_foldCode (V.primrec_innerInterStp P cons hconsp) Nat.Primrec.left
    (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.right.comp Nat.Primrec.
    right)).of_eq
    (fun _ => rfl)

theorem primrec_interCode (cons : ℕ → ℕ) (hconsp : Nat.Primrec cons) :
    Nat.Primrec (fun t => V.interCode P cons t.unpair.1 t.unpair.2) :=
  (primrec_foldCode (V.primrec_outerInterStp P cons hconsp) Nat.Primrec.right
    (Nat.Primrec.const 0) Nat.Primrec.left).of_eq (fun _ => rfl)

theorem subCode_computable :
    RecDecidable₂ (fun c k => ∀ a ∈ decodeList c, ∃ b ∈ decodeList k, P.X a ⊆ P.X b)
    := by
  have hinner : RecDecidable₂ (fun k a => ∃ b ∈ decodeList k, P.X a ⊆ P.X b) :=
    P.incl_computable.swap.bExistsList
  exact hinner.swap.bForallList

theorem upSet_inter_eq_empty_of_not_cons {a b : ℕ}
    (h : ¬ ∃ k, P.X k ⊆ P.X a ∩ P.X b) :
    V.upSet (P.X a) ∩ V.upSet (P.X b) = ∅ := by
  rw [V.upSet_inter]
  ext Z
  simp only [mem_upSet, Set.mem_empty_iff_false, iff_false, not_and]
  intro hZmem hZsub
  obtain ⟨n, hn⟩ := P.surj hZmem
  exact h ⟨n, by rw [hn]; exact hZsub⟩

theorem Ypd_innerstep (cons : ℕ → ℕ) {a b : ℕ}
    (hcons : cons (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b) (acc : ℕ) :
    V.Ypd P (selectFn (isOne (cons (Nat.pair a b))) (Nat.pair (P.inter a b) acc + 1)
    acc)
      = V.Ypd P acc ∪ (V.upSet (P.X a) ∩ V.upSet (P.X b)) := by
  by_cases h : cons (Nat.pair a b) = 1
  · rw [(isOne_eq_one_iff _).mpr h, selectFn_one, V.Ypd_cons_code P]
    have hXeq : P.X (P.inter a b) = P.X a ∩ P.X b := P.inter_spec (hcons.mp h)
    rw [hXeq, ← V.upSet_inter, Set.union_comm]
  · have hisz : isOne (cons (Nat.pair a b)) = 0 := by
      have hle := isOne_le_one (cons (Nat.pair a b))
      rcases (show isOne (cons (Nat.pair a b)) = 0 ∨ isOne (cons (Nat.pair a b)) = 1
    by omega)
        with h0 | h1
      · exact h0
      · exact absurd ((isOne_eq_one_iff _).mp h1) h
    rw [hisz, selectFn_zero,
      V.upSet_inter_eq_empty_of_not_cons P (fun hk => h (hcons.mpr hk)), Set.
    union_empty]
```

```
theorem Ypd_innerfoldl (cons : ℕ → ℕ) (a : ℕ)
    (hcons : ∀ b, cons (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b) :
    ∀ (bs : List ℕ) (acc : ℕ),
      V.Ypd P (List.foldl (fun acc b => selectFn (isOne (cons (Nat.pair a b)))
          (Nat.pair (P.inter a b) acc + 1) acc) acc bs)
        = V.Ypd P acc ∪ (V.upSet (P.X a) ∩ V.UPX P bs) := by
  intro bs
  induction bs with
  | nil => intro acc; rw [List.foldl_nil, V.UPX_nil P, Set.inter_empty, Set.
    union_empty]
  | cons b bs ih =>
    intro acc
    rw [List.foldl_cons, ih, V.Ypd_innerstep P cons (hcons b), V.UPX_cons P,
      Set.inter_union_distrib_left, Set.union_assoc]

theorem Ypd_innerInterCode (cons : ℕ → ℕ) (a acc m : ℕ)
    (hcons : ∀ b, cons (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b) :
    V.Ypd P (V.innerInterCode P cons a acc m)
      = V.Ypd P acc ∪ (V.upSet (P.X a) ∩ V.Ypd P m) := by
  rw [V.innerInterCode_eq P, V.Ypd_innerfoldl P cons a hcons (decodeList m) acc, ← V.
    Ypd_def P]

theorem Ypd_outerfoldl (cons : ℕ → ℕ) (m : ℕ)
    (hcons : ∀ a b, cons (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b) :
    ∀ (as : List ℕ) (acc : ℕ),
      V.Ypd P (List.foldl (fun acc a => V.innerInterCode P cons a acc m) acc as)
        = V.Ypd P acc ∪ (V.UPX P as ∩ V.Ypd P m) := by
  intro as
  induction as with
  | nil => intro acc; rw [List.foldl_nil, V.UPX_nil P, Set.empty_inter, Set.
    union_empty]
  | cons a as ih =>
    intro acc
    rw [List.foldl_cons, ih, V.Ypd_innerInterCode P cons a acc m (hcons a), V.UPX_cons
     P,
      Set.union_inter_distrib_right, Set.union_assoc]

theorem Ypd_interCode (cons : ℕ → ℕ) (n m : ℕ)
    (hcons : ∀ a b, cons (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b) :
    V.Ypd P (V.interCode P cons n m) = V.Ypd P n ∩ V.Ypd P m := by
  rw [V.interCode_eq P, V.Ypd_outerfoldl P cons m hcons (decodeList n) 0, V.Ypd_zero P
    ,
    Set.empty_union, ← V.Ypd_def P]

theorem upSet_subset_Ypd_iff {a k : ℕ} :
    V.upSet (P.X a) ⊆ V.Ypd P k ↔ ∃ b ∈ decodeList k, P.X a ⊆ P.X b := by
  constructor
  · intro h
    have hmem : P.X a ∈ V.Ypd P k := h ⟨P.mem_X a, subset_rfl⟩
    obtain ⟨b, hb, hPab⟩ := (V.mem_Ypd P).mp hmem
    exact ⟨b, hb, hPab.2⟩
  · rintro ⟨b, hb, hab⟩ z hz
    exact (V.mem_Ypd P).mpr ⟨b, hb, hz.1, hz.2.trans hab⟩

theorem Ypd_subset_iff {c k : ℕ} :
    V.Ypd P c ⊆ V.Ypd P k ↔ ∀ a ∈ decodeList c, ∃ b ∈ decodeList k, P.X a ⊆ P.X b :=
```

```
    by
  rw [Ypd_def, UPX_def, Set.iUnion₂_subset_iff]
  constructor
  · intro h a ha; exact (V.upSet_subset_Ypd_iff P).mp (h a ha)
  · intro h a ha; exact (V.upSet_subset_Ypd_iff P).mpr (h a ha)

theorem Ypd_eq_iff {c k : ℕ} :
    V.Ypd P c = V.Ypd P k ↔
      (∀ a ∈ decodeList c, ∃ b ∈ decodeList k, P.X a ⊆ P.X b) ∧
      (∀ b ∈ decodeList k, ∃ a ∈ decodeList c, P.X b ⊆ P.X a) := by
  rw [Set.Subset.antisymm_iff, V.Ypd_subset_iff P, V.Ypd_subset_iff P]

theorem eqCode_computable : RecDecidable₂ (fun c k => V.Ypd P c = V.Ypd P k) := by
  have h := (V.subCode_computable P).and (V.subCode_computable P).swap
  refine RecDecidable.of_iff (fun t => ?_) h
  exact V.Ypd_eq_iff P

theorem Ypd_interEq_computable (cons : ℕ → ℕ) (hconsp : Nat.Primrec cons)
    (hcons : ∀ a b, cons (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b) :
    RecDecidable₃ (fun n m k => V.Ypd P n ∩ V.Ypd P m = V.Ypd P k) := by
  have hic : Nat.Primrec (fun t => V.interCode P cons t.unpair.1 t.unpair.2.unpair.1)
    :=
    ((V.primrec_interCode P cons hconsp).comp
      (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right))).of_eq
      (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
  have hg : Nat.Primrec
      (fun t => Nat.pair (V.interCode P cons t.unpair.1 t.unpair.2.unpair.1) t.unpair
    .2.unpair.2) :=
    hic.pair (Nat.Primrec.right.comp Nat.Primrec.right)
  refine RecDecidable.of_iff (fun t => ?_) ((V.eqCode_computable P).comp hg)
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [V.Ypd_interCode P cons t.unpair.1 t.unpair.2.unpair.1 hcons]

def PDPresentation (cons : ℕ → ℕ) (hconsp : Nat.Primrec cons)
    (hcons : ∀ a b, cons (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b) :
    ComputablePresentation V.PowerDomain where
  X := V.Ypd P
  mem_X := fun c => V.Ypd_isPDmem P c
  surj := fun hW => V.PDmem_exists_Ypd P hW
  interEq_computable := V.Ypd_interEq_computable P cons hconsp hcons
  cons_computable :=
    recDecidable_of_forall (fun t => ⟨0, by rw [V.Ypd_zero P]; exact Set.empty_subset _
    ⟩)
  inter := V.interCode P cons
  inter_primrec := V.primrec_interCode P cons hconsp
  inter_spec := fun _ => V.Ypd_interCode P cons _ _ hcons
  masterIdx := Nat.pair P.masterIdx 0 + 1
  masterIdx_spec := by
    show V.Ypd P (Nat.pair P.masterIdx 0 + 1) = V.PowerDomain.master
    rw [V.Ypd_cons_code P, V.Ypd_zero P, Set.union_empty, P.masterIdx_spec,
    PowerDomain_master]

end NeighborhoodSystem
end

theorem NeighborhoodSystem.PowerDomain_isEffectivelyGiven {α : Type*} {V :
```

```
    NeighborhoodSystem α}
    (h : V.IsEffectivelyGiven) : V.PowerDomain.IsEffectivelyGiven := by
  obtain ⟨P⟩ := h
  obtain ⟨cons, hconsp, hconss⟩ := P.cons_computable
  refine ⟨V.PDPresentation P cons hconsp (fun a b => ?_)⟩
  have hb := hconss (Nat.pair a b)
  simp only [unpair_pair_fst, unpair_pair_snd] at hb
  exact hb.symm

end DomainTheory.NeighbourhoodSystems
```

4.8.24 Proposition 7.10 — power domain effective

PROPOSITION 7.10. The power domain $\mathbb{P}\mathcal{D}$ is a neighbourhood system if $\mathcal{D}$ is, and it is effectively given if $\mathcal{D}$ is.

Proof : The system $\mathcal{D}^\dagger$ is a neighbourhood system as we just remarked; indeed it is a positive neighbourhood system. It is easy to prove that the closure of any positive system under finite unions is a neighbourhood system, because the resulting family of sets is closed under *all* finite intersections. (If we left out the empty union, the result would be a positive system.) The proof is obvious since intersection of sets distributes over finite union. So $\mathbb{P}\mathcal{D}$ is a neighbourhood system.

For the second half of the proposition, we just have to constructivize the previous argument. Thus, if

$$\mathcal{D} = \{X_n \mid n \in \mathbb{N}\},$$

then the elements of $\mathbb{P}\mathcal{D}$ can be written as:

$$\bigcup_{i<q} (\downarrow X_{n_i}),$$

and hence are indexed by the finite sequences $\langle n_0, \ldots, n_{q-1}\rangle$ of integers. Now one of the standard devices of recursion theory is to put the finite sequences of integers into a recursive one-one correspondence with the integers themselves. This is the start of the recursive presentation of $\mathbb{P}\mathcal{D}$, since it means we can list effectively all the required neighbourhoods.

Next consider an intersection

$$\bigcup_{i<q} (\downarrow X_{n_i}) \cap \bigcup_{j<r} (\downarrow X_{m_j}) = \bigcup_{\substack{i<q \\ j<r}} \downarrow (X_{n_i} \cap X_{m_j})\,.$$

Some of the terms which are $\emptyset$ have to be thrown out — but this requires only a finite number of decisions all computable by assumption. Now we have to rewrite

$$X_{n_i} \cap X_{m_j} = X_{k_{ij}}\,,$$

but the finding of k_{ij} is also computable. *Finally*, we have to re-order the doubly indexed sequence into a singly indexed sequence of length $q \cdot r$, but this is easily seen to be computable also. Therefore, intersections can be "calculated".

It remains to be shown that equality between neighbourhoods in $\mathbb{P}\mathcal{D}$ is decidable. The question really comes down to deciding something like:

$$\downarrow X_k \subseteq \bigcup_{i<q} \downarrow X_{n_i}\,.$$

Now since $X_k \in \downarrow X_k$, we find that the above is just equivalent to:

$$\exists i < q.\, X_k \subseteq X_{n_i}\,.$$

By our assumptions on $\mathcal{D}$, this is decidable. (It is this part of the argument that required the passage to $\mathcal{D}^\dagger$. It does not seem to be generally true that the closure under finite unions of an effectively given system is again effectively given.) □

One of the main reasons that $\mathbb{P}\mathcal{D}$ is like a power domain is the possibility of forming "finite sets".

The development is with the effective powerdomain peer.

Theorem (`proposition_710`). The powerdomain of an effectively given neighbourhood system is effectively given.

Proof. By the effective powerdomain presentation. ■

`Proposition_7_10.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .powerDomainEffective

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
open NeighborhoodSystem

/-- Scott's Proposition 7.10: the powerdomain of an effectively given system is
    effectively given. -/
theorem proposition_710 :
    ∀ {α : Type*} {V : NeighborhoodSystem α},
      V.IsEffectivelyGiven → V.PowerDomain.IsEffectivelyGiven :=
  @PowerDomain_isEffectivelyGiven

end DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
```

4.8.25 Finite powerdomain elements

Finite joins of elements, and singletons, are elements of the powerdomain.

Definition (`NeighborhoodSystem.PDfinJoinZero`). The empty powerdomain join is the least powerdomain element.

Definition (`NeighborhoodSystem.PDmemFinJoin`). A powerdomain neighbourhood belongs to the finite join of x_i when there are neighbourhoods X_i of the respective x_i whose generated upper sets are all contained in it.

Definition (`NeighborhoodSystem.PDfinJoinSucc`). For a nonempty finite family, the preceding membership condition defines a powerdomain element.

Proof. The master neighbourhood is witnessed by master neighbourhoods of all components; intersections are witnessed componentwise; and membership is upward closed. ■

Definition (`NeighborhoodSystem.PDfinJoin`). The finite join is the least element for an empty family and the nonempty finite-join element otherwise.

Theorem (`NeighborhoodSystem.PDmem_finJoinZero`). A neighbourhood belongs to the empty finite join exactly when it is the master powerdomain neighbourhood.

Proof. This is the membership rule for the least element. ■

Definition (`NeighborhoodSystem.PDsingleton`). The singleton powerdomain element is the one-term finite join.

Theorem (`NeighborhoodSystem.PDmem_finJoinSucc`). Membership in a nonempty finite join is exactly the finite-join witness condition.

Proof. This is the defining membership relation. ■

Theorem (`NeighborhoodSystem.PDmem_singleton`). A powerdomain neighbourhood belongs to the singleton generated by x exactly when it contains the upper set of some neighbourhood of x.

Proof. Specialize the nonempty finite-join condition to the unique index of a one-element family. ■

Theorem (`NeighborhoodSystem.PDmem_finJoin`). Membership in a finite join is equality with the master neighbourhood when the family is empty, and the finite-join witness condition otherwise.

Proof. Separate the zero and successor cases of the family size. ■

Theorem (`NeighborhoodSystem.PDmem_finJoin_iUnion`). For a nonempty family, membership in its finite join is equivalent to containing the union of the upper sets generated by suitable component neighbourhoods.

Proof. Containing every generated upper set is equivalent to containing their union. ■

Theorem (`NeighborhoodSystem.PDmem_down`). A powerdomain neighbourhood is downward closed among original neighbourhoods: if it contains Z, $X \subseteq Z$, and X is a neighbourhood, then it contains X.

Proof. Choose a generator whose upper set contains Z; transitivity gives that the same generator's upper set contains X. ■

`finitePower.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .powerDomainEffective

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section
variable {α : Type*} (V : NeighborhoodSystem α)

def PDfinJoinZero : V.PowerDomain.Element := V.PowerDomain.bot

def PDmemFinJoin {n : ℕ} (xs : Fin n → V.Element) (W : Set (Set α)) : Prop :=
  ∃ (X : Fin n → Set α) (_hX : ∀ i, (xs i).mem (X i)),
    V.PowerDomain.mem W ∧ ∀ i, V.upSet (X i) ⊆ W

def PDfinJoinSucc {m : ℕ} (xs : Fin (m + 1) → V.Element) : V.PowerDomain.Element where
  mem := V.PDmemFinJoin xs
  sub hW := by obtain ⟨_, _, hPD, _⟩ := hW; exact hPD
```

```
  master_mem := by
    refine ⟨fun i => V.master, fun i => (xs i).master_mem, V.PDmem_master, fun i Z hz =
    > hz⟩
  inter_mem h1 h2 := by
    obtain ⟨X1, hX1, hPD1, hsub1⟩ := h1
    obtain ⟨X2, hX2, hPD2, hsub2⟩ := h2
    refine ⟨fun i => X1 i ∩ X2 i, fun i => (xs i).inter_mem (hX1 i) (hX2 i),
      V.PDmem_inter hPD1 hPD2, fun i Y hY => ?_⟩
    rw [← V.upSet_inter] at hY
    rcases hY with ⟨hY1, hY2⟩
    exact ⟨hsub1 i hY1, hsub2 i hY2⟩
  up_mem h hW hsub := by
    obtain ⟨X, hX, _, hsub'⟩ := h
    exact ⟨X, hX, hW, fun i => (hsub' i).trans hsub⟩

end
end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section
variable {α : Type*} (V : NeighborhoodSystem α)

def PDfinJoin (n : ℕ) (xs : Fin n → V.Element) : V.PowerDomain.Element :=
  match n with
  | 0 => V.PDfinJoinZero
  | _m + 1 => V.PDfinJoinSucc xs

@[simp] theorem PDmem_finJoinZero {W : Set (Set α)} :
    (V.PDfinJoinZero).mem W ↔ W = V.PowerDomain.master := by
  rw [PDfinJoinZero, mem_bot, PowerDomain_master]

def PDsingleton (x : V.Element) : V.PowerDomain.Element := V.PDfinJoin 1 (fun (_ : Fin
    1) => x)

theorem PDmem_finJoinSucc {m : ℕ} (xs : Fin (m + 1) → V.Element) {W : Set (Set α)} :
    (V.PDfinJoinSucc xs).mem W ↔ V.PDmemFinJoin xs W := Iff.rfl

@[simp] theorem PDmem_singleton {x : V.Element} {W : Set (Set α)} :
    (V.PDsingleton x).mem W ↔
      ∃ X, x.mem X ∧ V.PowerDomain.mem W ∧ V.upSet X ⊆ W := by
  rw [PDsingleton, PDfinJoin, PDmem_finJoinSucc, PDmemFinJoin]
  constructor
  · rintro ⟨X, hX, hPD, hsub⟩
    exact ⟨X 0, hX 0, hPD, hsub 0⟩
  · rintro ⟨X, hX, hPD, hsub⟩
    refine ⟨fun _ => X, fun _ => hX, hPD, fun _ => hsub⟩

theorem PDmem_finJoin {n : ℕ} (xs : Fin n → V.Element) {W : Set (Set α)} :
    (V.PDfinJoin n xs).mem W ↔
      if n = 0 then W = V.PowerDomain.master else V.PDmemFinJoin xs W := by
  match n with
  | 0 => simp [PDfinJoin, PDfinJoinZero, mem_bot]
  | m + 1 => simp [PDfinJoin, PDfinJoinSucc]
```

```
theorem PDmem_finJoin_iUnion {m : ℕ} (xs : Fin (m + 1) → V.Element) {W : Set (Set α)}
    :
    (V.PDfinJoinSucc xs).mem W ↔
      ∃ (X : Fin (m + 1) → Set α) (_hX : ∀ i, (xs i).mem (X i)),
        V.PowerDomain.mem W ∧
          (⋃ i, V.upSet (X i)) ⊆ W := by
  rw [PDmem_finJoinSucc]
  constructor
  · rintro ⟨X, hX, hPD, hsub⟩
    exact ⟨X, hX, hPD, Set.iUnion_subset hsub⟩
  · rintro ⟨X, hX, hPD, hsub⟩
    exact ⟨X, hX, hPD, (Set.iUnion_subset_iff).mp hsub⟩

end

end NeighborhoodSystem
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*}

namespace NeighborhoodSystem

variable (V : NeighborhoodSystem α)

theorem PDmem_down {A : Set (Set α)} (hA : V.PDmem A) {Z X : Set α}
    (hZ : Z ∈ A) (hXZ : X ⊆ Z) (hX : V.mem X) : X ∈ A := by
  obtain ⟨L, hL, rfl⟩ := hA
  obtain ⟨X', hX', _, hZsub⟩ := (V.mem_PDunion).mp hZ
  exact (V.mem_PDunion).mpr ⟨X', hX', hX, hXZ.trans hZsub⟩

end NeighborhoodSystem
end

end DomainTheory.NeighbourhoodSystems
```

4.8.26 Definition 7.11 — finite sets of elements

DEFINITION 7.11. For elements $x_0, \ldots, x_{n-1} \in |D|$ we define

$$\{x_0, \ldots, x_{n-1}\} = \{z \in \mathbb{P}D \mid \exists X_0 \in x_0 \ldots \exists X_{n-1} \in x_{n-1}.\ \bigcup_{i<n} (\uparrow X_i) \subseteq z\}.$$

(Note, we could also write $\forall i < n.\, X_i \in z$.) □

The development is with the finite powerdomain elements peer.

Theorem (`definition_711`). The singleton of an element is the finite join of that one element.

Proof. By the definition of the singleton. ■

`Definition_7_11.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .finitePower
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

/-- Scott's Definition 7.11: finite joins and singletons in the powerdomain. -/
theorem definition_711 {α : Type*} (V : NeighborhoodSystem α) (x : V.Element) :
    V.PDsingleton x = V.PDfinJoin 1 (fun _ : Fin 1 => x) :=
  rfl

end DomainTheory.NeighbourhoodSystems
```

4.8.27 Finite-join approximation

The singleton and binary join maps into the powerdomain are approximable, and the binary join is computable.

Theorem (`NeighborhoodSystem.PDsingleton_mono`). If $X' \subseteq X$ are neighbourhoods, then the singleton generated by the principal element at X lies below the singleton generated at X'.

Proof. Every neighbourhood containing an upper set witnessed above X is also witnessed above the smaller neighbourhood X'. ■

Definition (`NeighborhoodSystem.PDsingletonApproxMap`). The singleton approximable map sends a neighbourhood X to the singleton powerdomain element generated by the principal element at X.

Proof. The preceding antitonicity in neighbourhood refinement gives the required monotonicity of the relation. ■

Theorem (`NeighborhoodSystem.PDsingletonApproxMap_rel`). For a neighbourhood X, the singleton map relates X to W exactly when W belongs to the singleton generated by the principal element at X.

Proof. This is the relation induced by the monotone assignment. ■

Definition (`NeighborhoodSystem.PDsingletonMeet`). The meet of two powerdomain elements consists of the neighbourhoods belonging to both.

Proof. Common membership is closed under intersection and enlargement and contains the master neighbourhood. ■

Theorem (`NeighborhoodSystem.PDsingletonMeet_mem`). A neighbourhood belongs to the singleton meet of a and b exactly when it belongs to both a and b.

Proof. This is the defining membership condition. ■

Definition (`NeighborhoodSystem.PDfinJoinApproxMap2`). The binary finite-join relation holds when each input is related to the target by the singleton approximable map.

Proof. Domain and codomain validity, target intersections, and monotonicity hold componentwise. ■

Definition (`NeighborhoodSystem.finJoinMap_prod`). The product form of the binary finite-join map is obtained by viewing the binary relation as an approximable map out of the product system.

Theorem (`NeighborhoodSystem.finJoinMap_prod_rel`). On a product neighbourhood generated by X, Y, the product join relation is exactly the binary finite-join relation on X, Y.

Proof. The two product preimages are precisely X and Y. ■

Theorem (`NeighborhoodSystem.PDext`). Two powerdomain elements are equal when they have the same neighbourhoods.

Proof. Elements of any neighbourhood system are determined extensionally by membership. ■

Theorem (`NeighborhoodSystem.PDfinJoinApproxMap2_rel_iff`). The binary finite-join relation holds exactly when both singleton relations hold.

Proof. This is its defining condition. ■

Theorem (`NeighborhoodSystem.PDfinJoin_pair`). The two-term finite join of x, y is the meet of their singleton powerdomain elements.

Proof. Both sides contain exactly those powerdomain neighbourhoods that contain suitable generated upper sets for both x and y. ■

Theorem (`NeighborhoodSystem.PDfinJoin_inter_two`). A neighbourhood belongs to the two-term finite join exactly when it belongs to each singleton.

Proof. Combine the two-term join identity with the membership rule for the singleton meet. ■

Theorem (`NeighborhoodSystem.PDfinJoinApproxMap2_rel_PD`). On neighbourhoods X, Y, the binary relation to W holds exactly when W belongs to the two-term join of the corresponding principal elements.

Proof. Each conjunct is singleton membership for one principal element, and two-term join membership is their conjunction. ■

Theorem (`NeighborhoodSystem.PDsingletonApproxMap_rel_Ypd_iff`). The singleton map relates the neighbourhood indexed by n to the coded powerdomain neighbourhood k exactly when some index in the decoded list for k names a neighbourhood containing the one indexed by n.

Proof. Membership of the generator in the coded finite union selects exactly such a list entry. ■

Theorem (`NeighborhoodSystem.PDsingletonApproxMap_toElementMap`). The element map induced by the singleton approximable map sends x to its singleton powerdomain element.

Proof. A witness neighbourhood below x can be enlarged within x, and conversely a singleton witness itself supplies a source neighbourhood. ■

Theorem (`NeighborhoodSystem.PDfinJoinApproxMap2_toElementMap`). The element map induced by the binary finite-join relation sends x, y to their two-term powerdomain join.

Proof. Binary relation membership splits into the two singleton images, whose conjunction is two-term join membership. ■

Theorem (`NeighborhoodSystem.PDsingleton_principal`). The singleton of a principal element generated by X equals the principal powerdomain element generated by the upper set of X.

Proof. On both sides, membership means being a powerdomain neighbourhood containing that generated upper set. ■

Theorem (`NeighborhoodSystem.ext_of_principal`). Two approximable maps are equal if their element maps agree on every principal element.

Proof. A relation at X is equivalent to target membership in the image of the principal element at X. ■

Theorem (`NeighborhoodSystem.finJoinMap_prod_toElementMap`). The product join map sends a pair of elements to their two-term powerdomain join.

Proof. Convert the product map to its binary form and apply the binary element-map identity. ■

Theorem (`NeighborhoodSystem.singleton_isComputable`). Relative to the effective powerdomain presentation, the singleton approximable map is computable.

Proof. Its indexed relation is the decidable bounded search for a decoded generator containing the source neighbourhood. ■

Theorem (`NeighborhoodSystem.PDfinJoinApproxMap2_isComputable`). The product form of the binary finite-join map is computable.

Proof. Its relation is the conjunction of two computable singleton relations after primitive-recursively selecting the two product coordinates. ■

finJoinApprox.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_4
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_11
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import Mathlib.Data.Fin.VecNotation

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

section
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive
namespace NeighborhoodSystem
variable {α : Type*} (V : NeighborhoodSystem α)

theorem PDsingleton_mono {X X' : Set α} (hX : V.mem X) (hX' : V.mem X') (hX'X : X' ⊆ X
    ) :
    V.PDsingleton (V.principal hX) ≤ V.PDsingleton (V.principal hX') := by
  intro W hW
  rw [PDmem_singleton] at hW ⊢
  obtain ⟨Y, hYpr, hPD, hsub⟩ := hW
  rw [mem_principal] at hYpr
  exact ⟨Y, ⟨hYpr.1, hX'X.trans hYpr.2⟩, hPD, hsub⟩

def PDsingletonApproxMap : ApproximableMap V V.PowerDomain :=
  ofMono (fun _X hX => V.PDsingleton (V.principal hX))
    (fun X X' hX hX' hX'X => @PDsingleton_mono α V X X' hX hX' hX'X)

@[simp] theorem PDsingletonApproxMap_rel {X : Set α} {W : Set (Set α)} (hX : V.mem X)
    :
    (PDsingletonApproxMap (V := V)).rel X W ↔ (V.PDsingleton (V.principal hX)).mem W
    := by
  dsimp [PDsingletonApproxMap, ofMono]
  constructor
  · intro ⟨_, h⟩; exact h
  · intro h; exact ⟨hX, h⟩

def PDsingletonMeet (a b : V.PowerDomain.Element) : V.PowerDomain.Element where
  mem W := a.mem W ∧ b.mem W
  sub h := a.sub h.1
  master_mem := ⟨a.master_mem, b.master_mem⟩
  inter_mem h1 h2 := ⟨a.inter_mem h1.1 h2.1, b.inter_mem h1.2 h2.2⟩
  up_mem h hW hsub := ⟨a.up_mem h.1 hW hsub, b.up_mem h.2 hW hsub⟩

@[simp] theorem PDsingletonMeet_mem {a b : V.PowerDomain.Element} {W : Set (Set α)} :
    (V.PDsingletonMeet a b).mem W ↔ a.mem W ∧ b.mem W := Iff.rfl

def PDfinJoinApproxMap₂ : ApproximableMap₂ V V V.PowerDomain where
  rel X Y W := (PDsingletonApproxMap (V := V)).rel X W ∧ (PDsingletonApproxMap (V := V
```

```
    )).rel Y W
  rel_dom₀ h := (PDsingletonApproxMap (V := V)).rel_dom h.1
  rel_dom₁ h := (PDsingletonApproxMap (V := V)).rel_dom h.2
  rel_cod h := (PDsingletonApproxMap (V := V)).rel_cod h.1
  master_rel := ⟨(PDsingletonApproxMap (V := V)).master_rel, (PDsingletonApproxMap (V
    := V)).master_rel⟩
  inter_right := fun ⟨h1, h2⟩ ⟨h1', h2'⟩ =>
    ⟨(PDsingletonApproxMap (V := V)).inter_right h1 h1',
      (PDsingletonApproxMap (V := V)).inter_right h2 h2'⟩
  mono := fun ⟨h1, h2⟩ hXX' hYY' hWW' hX' hY' hW' =>
    ⟨(PDsingletonApproxMap (V := V)).mono h1 hXX' hWW' hX' hW',
      (PDsingletonApproxMap (V := V)).mono h2 hYY' hWW' hY' hW'⟩

def finJoinMap_prod : ApproximableMap (prod V V) V.PowerDomain := ofMap₂ (
    PDfinJoinApproxMap₂ (V := V))

@[simp] theorem finJoinMap_prod_rel {X Y : Set α} (hX : V.mem X) (hY : V.mem Y) {W :
    Set (Set α)} :
    (finJoinMap_prod (V := V)).rel (prodNbhd X Y) W ↔
      (PDfinJoinApproxMap₂ (V := V)).rel X Y W := by
  dsimp [finJoinMap_prod, ofMap₂]
  simp only [prodNbhd.inl_preimage, prodNbhd.inr_preimage]
  exact ⟨fun ⟨_, h⟩ => h, fun h => ⟨prod_mem_prodNbhd hX hY, h⟩⟩

end NeighborhoodSystem
end

open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

section
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem PDext {a b : V.PowerDomain.Element} (h : ∀ W, a.mem W ↔ b.mem W) : a = b :=
  @Element.ext (Set α) V.PowerDomain a b h

end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

variable (P : ComputablePresentation V)

theorem PDfinJoinApproxMap₂_rel_iff {X Y : Set α} (_hX : V.mem X) (_hY : V.mem Y) {W :
    Set (Set α)} :
    (PDfinJoinApproxMap₂ (V := V)).rel X Y W ↔
      (PDsingletonApproxMap (V := V)).rel X W ∧ (PDsingletonApproxMap (V := V)).rel Y
    W := Iff.rfl
```

```
end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem PDfinJoin_pair (x y : V.Element) :
    V.PDfinJoin 2 ![x, y] = V.PDsingletonMeet (V.PDsingleton x) (V.PDsingleton y) :=
    by
  apply PDext
  intro W
  simp only [PDsingletonMeet_mem, PDmem_singleton, PDfinJoin, PDmem_finJoinSucc,
    PDmemFinJoin]
  constructor
  · rintro ⟨X, hX, hPD, hsub⟩
    exact ⟨⟨X 0, hX 0, hPD, hsub 0⟩, ⟨X 1, hX 1, hPD, hsub 1⟩⟩
  · rintro ⟨⟨X0, hx, hPD, hsub0⟩, ⟨X1, hy, _, hsub1⟩⟩
    refine ⟨![X0, X1], ?_, hPD, ?_⟩
    · intro i; fin_cases i <;> assumption
    · intro i; fin_cases i <;> assumption

theorem PDfinJoin_inter_two (x y : V.Element) {W : Set (Set α)} :
    (V.PDfinJoin 2 ![x, y]).mem W ↔
      (V.PDsingleton x).mem W ∧ (V.PDsingleton y).mem W := by
  rw [← PDsingletonMeet_mem, ← PDfinJoin_pair]

end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

variable (P : ComputablePresentation V)

theorem PDfinJoinApproxMap₂_rel_PD {X Y : Set α} (hX : V.mem X) (hY : V.mem Y) {W : Set
    (Set α)} :
    (PDfinJoinApproxMap₂ (V := V)).rel X Y W ↔
      (V.PDfinJoin 2 ![V.principal hX, V.principal hY]).mem W := by
  constructor
  · intro ⟨h1, h2⟩
    rw [PDsingletonApproxMap_rel (hX := hX)] at h1
    rw [PDsingletonApproxMap_rel (hX := hY)] at h2
    exact (PDfinJoin_inter_two (x := V.principal hX) (y := V.principal hY)).2 ⟨h1, h2⟩
  · intro h
    rcases (PDfinJoin_inter_two (x := V.principal hX) (y := V.principal hY)).1 h with ⟨
    h1, h2⟩
    exact ⟨by rw [PDsingletonApproxMap_rel (hX := hX)]; exact h1,
```

```
      by rw [PDsingletonApproxMap_rel (hX := hY)]; exact h2⟩

theorem PDsingletonApproxMap_rel_Ypd_iff {n k : ℕ} :
    ((PDsingletonApproxMap (V := V))).rel (P.X n) (V.Ypd P k) ↔
      ∃ b ∈ decodeList k, P.X n ⊆ P.X b := by
  rw [PDsingletonApproxMap_rel (hX := P.mem_X n), PDmem_singleton, PowerDomain_mem]
  constructor
  · rintro ⟨Y, hYpr, _, hsub⟩
    have hn : P.X n ∈ V.upSet Y := ⟨P.mem_X n, hYpr.2⟩
    obtain ⟨b, hb, hPab⟩ := (V.mem_Ypd P).mp (hsub hn)
    exact ⟨b, hb, hPab.2⟩
  · intro ⟨b, hb, hnb⟩
    refine ⟨P.X b, ⟨P.mem_X b, hnb⟩, V.Ypd_isPDmem P k,
      (V.upSet_subset_Ypd_iff P).mpr ⟨b, hb, subset_rfl⟩⟩

end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem PDsingletonApproxMap_toElementMap (x : V.Element) :
    (PDsingletonApproxMap (V := V)).toElementMap x = V.PDsingleton x := by
  apply PDext
  intro W
  rw [ApproximableMap.mem_toElementMap, PDmem_singleton]
  constructor
  · rintro ⟨X, hxX, hrel⟩
    rw [PDsingletonApproxMap_rel (hX := x.sub hxX)] at hrel
    rw [PDmem_singleton] at hrel
    obtain ⟨Y, ⟨hYmem, hXY⟩, hPD, hsub⟩ := hrel
    exact ⟨Y, x.up_mem hxX hYmem hXY, hPD, hsub⟩
  · intro hW
    obtain ⟨Y, hy, hPD, hsub⟩ := hW
    refine ⟨Y, hy, ?_⟩
    rw [PDsingletonApproxMap_rel (hX := x.sub hy), PDmem_singleton]
    exact ⟨Y, ⟨x.sub hy, subset_rfl⟩, hPD, hsub⟩

theorem PDfinJoinApproxMap₂_toElementMap (x y : V.Element) :
    (PDfinJoinApproxMap₂ (V := V)).toElementMap₂ x y = V.PDfinJoin 2 ![x, y] := by
  apply PDext
  intro W
  rw [PDfinJoin_inter_two, ApproximableMap₂.mem_toElementMap₂]
  constructor
  · rintro ⟨X, Y, hxX, hyY, ⟨h1, h2⟩⟩
    constructor
    · rw [← PDsingletonApproxMap_toElementMap (V := V)]
      exact (ApproximableMap.mem_toElementMap (PDsingletonApproxMap (V := V)) x).mpr ⟨X
    , hxX, h1⟩
    · rw [← PDsingletonApproxMap_toElementMap (V := V)]
      exact (ApproximableMap.mem_toElementMap (PDsingletonApproxMap (V := V)) y).mpr ⟨Y
    , hyY, h2⟩
```

```
  · intro ⟨h1, h2⟩
    rw [← PDsingletonApproxMap_toElementMap (V := V)] at h1 h2
    obtain ⟨X, hxX, hrelX⟩ :=
      (ApproximableMap.mem_toElementMap (PDsingletonApproxMap (V := V)) x).mp h1
    obtain ⟨Y, hyY, hrelY⟩ :=
      (ApproximableMap.mem_toElementMap (PDsingletonApproxMap (V := V)) y).mp h2
    exact ⟨X, Y, hxX, hyY, hrelX, hrelY⟩

theorem PDsingleton_principal {X : Set α} (hX : V.mem X) :
    V.PDsingleton (V.principal hX) =
      V.PowerDomain.principal (V.PDmem_upSet hX) := by
  apply PDext
  intro W
  rw [PDmem_singleton, mem_principal, PowerDomain_mem]
  constructor
  · rintro ⟨Y, ⟨hYmem, hXY⟩, hPD, hsub⟩
    exact ⟨hPD, ((V.upSet_subset_iff hX).mpr hXY).trans hsub⟩
  · rintro ⟨hPD, hsub⟩
    exact ⟨X, ⟨hX, subset_rfl⟩, hPD, hsub⟩

end NeighborhoodSystem
end

open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

section
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem ext_of_principal {β : Type*} {W : NeighborhoodSystem β}
    {f g : ApproximableMap V W}
    (h : ∀ (X : Set α) (hX : V.mem X),
      f.toElementMap (V.principal hX) = g.toElementMap (V.principal hX)) : f = g := by
  apply ApproximableMap.ext
  intro X Y
  constructor
  · intro hr
    have hX := f.rel_dom hr
    rw [g.rel_iff_mem_principal hX, ← h X hX, ← f.rel_iff_mem_principal hX]; exact hr
  · intro hr
    have hX := g.rel_dom hr
    rw [f.rel_iff_mem_principal hX, h X hX, ← g.rel_iff_mem_principal hX]; exact hr

theorem finJoinMap_prod_toElementMap (x y : V.Element) :
    (finJoinMap_prod (V := V)).toElementMap (pair x y) = V.PDfinJoin 2 ![x, y] := by
  have hmap : toMap₂ (finJoinMap_prod (V := V)) = PDfinJoinApproxMap₂ (V := V) := toMap
    ₂_ofMap₂ _
  rw [← toElementMap₂_toMap₂ (finJoinMap_prod (V := V)), hmap, PDfinJoinApproxMap₂
    _toElementMap]

end NeighborhoodSystem
end
```

```
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

section
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

variable (P : ComputablePresentation V)

theorem singleton_isComputable (cons : ℕ → ℕ) (hconsp : Nat.Primrec cons)
    (hcons : ∀ a b, cons (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b) :
    ComputableMap P (V.PDPresentation P cons hconsp hcons) ((PDsingletonApproxMap (V
    := V))) := by
  constructor
  have h := P.incl_computable.swap.bExistsList.swap
  refine (RecDecidable.of_iff (fun s => ?_) h).re
  show PDsingletonApproxMap (V := V).rel (P.X s.unpair.1) ((V.PDPresentation P cons
    hconsp hcons).X s.unpair.2) ↔
      ∃ e ∈ decodeList s.unpair.2, P.X s.unpair.1 ⊆ P.X e
  simp only [PDPresentation]
  apply PDsingletonApproxMap_rel_Ypd_iff (V := V)

theorem PDfinJoinApproxMap₂_isComputable (cons : ℕ → ℕ) (hconsp : Nat.Primrec cons)
    (hcons : ∀ a b, cons (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b) :
    ComputableMap (prodPresentation P P)
      (V.PDPresentation P cons hconsp hcons) (finJoinMap_prod (V := V)) := by
  constructor
  have hs : REPred (fun s => (PDsingletonApproxMap (V := V)).rel (P.X s.unpair.1) (V.
    Ypd P s.unpair.2)) :=
    (singleton_isComputable (V := V) P cons hconsp hcons).out
  have hra : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1 t.unpair.2) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right
  have hrb : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2 t.unpair.2) :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine REPred.of_iff (fun t => by
    dsimp [prodPresentation_X, finJoinMap_prod, ofMap₂]
    simp only [prodNbhd.inl_preimage, prodNbhd.inr_preimage,
      PDPresentation, unpair_pair_fst, unpair_pair_snd]
    constructor
    · rintro ⟨_, ⟨h1, h2⟩⟩
      exact ⟨h1, h2⟩
    · intro ⟨h1, h2⟩
      refine ⟨prod_mem_prodNbhd (P.mem_X _) (P.mem_X _), h1, h2⟩) ((hs.comp hra).and (
    hs.comp hrb))

end NeighborhoodSystem
end

open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

end DomainTheory.NeighbourhoodSystems
```

4.8.28 V-shape counterexample

The V-shaped neighbourhood system does not embed into its powerdomain.

Definition (`Counterexample712C.HasTop`). A neighbourhood system has a top element when some element lies above every element in its domain order.

Theorem (`Counterexample712C.ft_inter`). The singleton sets containing false and true have empty intersection.

Proof. No Boolean value is simultaneously false and true. ■

Definition (`Counterexample712C.improperTop`). If the neighbourhoods of a system are closed under arbitrary binary intersections, the family of all neighbourhoods forms an element of that system.

Proof. Closure gives finite-intersection closure, while the remaining element conditions follow directly from neighbourhood membership. ■

Theorem (`Counterexample712C.tf_inter`). The singleton sets containing true and false have empty intersection.

Proof. No Boolean value is simultaneously true and false. ■

Definition (`Counterexample712C.Vshape`). The V-shaped system on the Booleans has exactly three neighbourhoods: the whole set and the two singleton sets.

Proof. The whole set is master; the only incompatible intersection is empty, which cannot contain any of the three nonempty neighbourhoods required by the intersection axiom. ■

Theorem (`Counterexample712C.Vshape_mem_false`). The singleton containing false is a V-shaped neighbourhood.

Proof. It is one of the three defining neighbourhoods. ■

Theorem (`Counterexample712C.Vshape_mem_true`). The singleton containing true is a V-shaped neighbourhood.

Proof. It is one of the three defining neighbourhoods. ■

Theorem (`Counterexample712C.Vshape_not_mem_empty`). The empty set is not a V-shaped neighbourhood.

Proof. It is neither the whole Boolean set nor either Boolean singleton. ■

Theorem (`Counterexample712C.Vshape_not_hasTop`). The V-shaped domain has no top element.

Proof. A top element would contain both singleton neighbourhoods and hence their empty intersection, forcing the empty set to be a neighbourhood of that element, contrary to the V-shaped neighbourhood list. ■

Theorem (`Counterexample712C.hasTop_of_inter_closed`). A neighbourhood system whose neighbourhoods are closed under binary intersection has a top element.

Proof. The element consisting of all neighbourhoods lies above every other element. ■

Theorem (`Counterexample712C.hasTop_of_iso`). Having a top element is preserved when transported backwards along an order isomorphism.

Proof. The inverse image of a top element is above every element by order reflection. ■

Theorem (`Counterexample712C.powerDomain_hasTop`). Every powerdomain has a top element.

Proof. Powerdomain neighbourhoods are closed under intersection, so the intersection-closed construction applies. ■

Theorem (`Counterexample712C.subsystem_inter_closed`). A subsystem of an intersection-closed neighbourhood system is itself intersection closed.

Proof. The ambient intersection is a neighbourhood, and the subsystem's closure property brings it back into the subsystem. ■

Theorem (`Counterexample712C.hasTop_of_trianglelefteq_powerDomain`). Any neighbourhood system isomorphic to a subsystem of a powerdomain has a top element.

Proof. The subsystem inherits intersection closure from the powerdomain and therefore has a top, which the isomorphism transports to the original system. ■

Theorem (`Counterexample712C.vshape_not_trianglelefteq_powerDomain`). The V-shaped neighbourhood system is not isomorphic to any subsystem of its powerdomain.

Proof. Such an embedding would give the V-shaped domain a top element, contradicting its failure to have one. ■

`vshapeCounterexample.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .finJoinApprox
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_4
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_11
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import Mathlib.Data.Fin.VecNotation

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive

namespace Counterexample712C

open NeighborhoodSystem

variable {? ? : Type*}

def HasTop (E : NeighborhoodSystem ?) : Prop := ∃ t : E.Element, ∀ x : E.Element, x ≤
    t

theorem ft_inter : ({false} : Set Bool) ∩ {true} = ∅ := by
  ext b
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false, not_and]
  rintro rfl h; exact absurd h (by decide)

def improperTop (E : NeighborhoodSystem ?)
    (hcl : ∀ {X Y : Set ?}, E.mem X → E.mem Y → E.mem (X ∩ Y)) : E.Element where
  mem := E.mem
  sub h := h
  master_mem := E.master_mem
  inter_mem h1 h2 := hcl h1 h2
  up_mem _ hY _ := hY

theorem tf_inter : ({true} : Set Bool) ∩ {false} = ∅ := by
```

```
  ext b
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false, not_and]
  rintro rfl h; exact absurd h (by decide)

def Vshape : NeighborhoodSystem Bool where
  mem X := X = Set.univ ∨ X = {true} ∨ X = {false}
  master := Set.univ
  master_nonempty := ⟨true, Set.mem_univ _⟩
  master_mem := Or.inl rfl
  sub_master _ := Set.subset_univ _
  inter_mem := by
    rintro X Y Z hX hY hZ hZsub

    have hZne : Z.Nonempty := by
      rcases hZ with rfl | rfl | rfl
      · exact ⟨true, Set.mem_univ true⟩
      · exact ⟨true, rfl⟩
      · exact ⟨false, rfl⟩
    obtain ⟨z, hz⟩ := hZne
    rcases hX with rfl | rfl | rfl <;> rcases hY with rfl | rfl | rfl
    · exact Or.inl (by rw [Set.inter_self])
    · exact Or.inr (Or.inl (by rw [Set.univ_inter]))
    · exact Or.inr (Or.inr (by rw [Set.univ_inter]))
    · exact Or.inr (Or.inl (by rw [Set.inter_univ]))
    · exact Or.inr (Or.inl (by rw [Set.inter_self]))
    · rw [tf_inter] at hZsub; exact absurd (hZsub hz) (Set.notMem_empty z)
    · exact Or.inr (Or.inr (by rw [Set.inter_univ]))
    · rw [ft_inter] at hZsub; exact absurd (hZsub hz) (Set.notMem_empty z)
    · exact Or.inr (Or.inr (by rw [Set.inter_self]))

end Counterexample712C
namespace Counterexample712C

open NeighborhoodSystem

variable {? ? : Type*}

theorem Vshape_mem_false : Vshape.mem {false} := Or.inr (Or.inr rfl)

theorem Vshape_mem_true : Vshape.mem {true} := Or.inr (Or.inl rfl)

theorem Vshape_not_mem_empty : ¬ Vshape.mem (∅ : Set Bool) := by
  rintro (h | h | h)
  · exact Set.empty_ne_univ h
  · exact absurd h.symm (Set.singleton_ne_empty true)
  · exact absurd h.symm (Set.singleton_ne_empty false)

theorem Vshape_not_hasTop : ¬ HasTop Vshape := by
  rintro ⟨t, ht⟩
  have htt : t.mem {true} :=
    ht (Vshape.principal Vshape_mem_true) {true} ⟨Vshape_mem_true, subset_rfl⟩
  have htf : t.mem {false} :=
    ht (Vshape.principal Vshape_mem_false) {false} ⟨Vshape_mem_false, subset_rfl⟩
  have hint : t.mem (({true} : Set Bool) ∩ {false}) := t.inter_mem htt htf
  rw [tf_inter] at hint
```

```
  exact Vshape_not_mem_empty (t.sub hint)

end Counterexample712C
namespace Counterexample712C

open NeighborhoodSystem

variable {? ? : Type*}

theorem hasTop_of_inter_closed (E : NeighborhoodSystem ?)
    (hcl : ∀ {X Y : Set ?}, E.mem X → E.mem Y → E.mem (X ∩ Y)) : HasTop E :=
  ⟨improperTop E hcl, fun x _X hX => x.sub hX⟩

theorem hasTop_of_iso {D : NeighborhoodSystem ?} {D' : NeighborhoodSystem ?}
    (e : D.Element ≃o D'.Element) (h : HasTop D') : HasTop D := by
  obtain ⟨t, ht⟩ := h
  refine ⟨e.symm t, fun x => ?_⟩
  rw [← e.le_iff_le, e.apply_symm_apply]
  exact ht (e x)

theorem powerDomain_hasTop (V : NeighborhoodSystem ?) : HasTop V.PowerDomain :=
  hasTop_of_inter_closed V.PowerDomain (fun hX hY => V.PDmem_inter hX hY)

end Counterexample712C
namespace Counterexample712C

open NeighborhoodSystem

variable {? ? : Type*}

theorem subsystem_inter_closed {D E : NeighborhoodSystem ?} (h : D ? E)
    (hE : ∀ {X Y : Set ?}, E.mem X → E.mem Y → E.mem (X ∩ Y))
    {X Y : Set ?} (hX : D.mem X) (hY : D.mem Y) : D.mem (X ∩ Y) :=
  h.inter_closed hX hY (hE (h.sub hX) (h.sub hY))

theorem hasTop_of_trianglelefteq_powerDomain {D : NeighborhoodSystem ?} {E :
   NeighborhoodSystem ?}
    (h : D ? E.PowerDomain) : HasTop D := by
  obtain ⟨D', hsub, ⟨e⟩⟩ := h
  have hPcl : ∀ {X Y : Set (Set ?)}, E.PowerDomain.mem X → E.PowerDomain.mem Y →
      E.PowerDomain.mem (X ∩ Y) := fun hX hY => E.PDmem_inter hX hY
  exact hasTop_of_iso e (hasTop_of_inter_closed D' (subsystem_inter_closed hsub hPcl))

theorem vshape_not_trianglelefteq_powerDomain : ¬ (Vshape ? Vshape.PowerDomain) := fun
    h =>
  Vshape_not_hasTop (hasTop_of_trianglelefteq_powerDomain h)

end Counterexample712C

end DomainTheory.NeighbourhoodSystems
```

4.8.29 Proposition 7.12 — finite sets are approximable

PROPOSITION 7.12. The mapping

$$\lambda x_0, \ldots, x_{n-1}. \{x_0, \ldots, x_{n-1}\} : D^n \to \mathbb{P}D$$

is approximable and is computable if D is effectively given. Moreover, the map $\lambda x. \{x\}$ shows that $D \trianglelefteq \mathbb{P}D$, and we also have the law:

$$\{x_0, \ldots, x_{n-1}\} = \{x_0\} \cap \cdots \cap \{x_{n-1}\}$$

as an intersection of filters.

Proof : The second part shows that everything reduces to $\lambda x. \{x\}$. We see that

$$X_k \ (\lambda x. \{x\}) \bigcup_{i<q} (\uparrow X_{n_i}) \text{ iff } \exists i < q. X_k \sqsubseteq X_{n_i}.$$

Thus, $\lambda x. \{x\}$ is an approximable mapping and is computable in the effectively given case.

The proof of the law can be reduced to the special case

$$\{x\} \cap \{y\} = \{x, y\}$$

for the sake of illustration. In terms of finite elements of the two domains D and $\mathbb{P}D$ we find

$$\{\uparrow X\} = \uparrow\uparrow X,$$

and so,

$$\begin{aligned} \{\uparrow X\} \cap \{\uparrow Y\} &= \uparrow\uparrow X \cap \uparrow\uparrow Y \\ &= \uparrow (\uparrow X \cup \uparrow Y) \\ &= \{\uparrow X, \uparrow Y\}. \end{aligned}$$

An equation between approximable functions that checks for finite elements also holds for all elements.

Finally, we note that

$$D \cong D^\dagger \trianglelefteq \mathbb{P}D$$

and that the isomorphism involved is just $\lambda x. \{x\}$ by what we saw on the finite elements. □

Further combinators on the power domain are given in the exercises.

The development is with the finite-join approximation and V-shape counterexample peers.

Theorem (`proposition_712`). Binary join into the powerdomain is the meet of the two singletons and is computable, and the V-shaped system does not embed into its powerdomain.

Proof. By the finite-join approximation and the V-shape counterexample. ■

`Proposition_7_12.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .finJoinApprox
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .vshapeCounterexample
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .funPresentation
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
set_option linter.unusedVariables false
open NeighborhoodSystem ApproximableMap ApproximableMap₂ Domain.Recursive
    Counterexample712C

/-- Scott's Proposition 7.12: singleton/finJoin maps and the Vshape counterexample. -/
theorem proposition_712 :
    (∀ {α : Type*} (V : NeighborhoodSystem α) (x y : V.Element),
      (PDfinJoinApproxMap₂ (V := V)).toElementMap₂ x y = V.PDfinJoin 2 ![x, y]) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (P : ComputablePresentation V)
      (cons : ℕ → ℕ) (hconsp : Nat.Primrec cons)
      (hcons : ∀ a b, cons (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b),
      ComputableMap (prodPresentation P P) (V.PDPresentation P cons hconsp hcons)
        (finJoinMap_prod (V := V))) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) {X Y : Set α} (_hX : V.mem X) (_hY : V.
    mem Y)
      {W : Set (Set α)},
      (PDfinJoinApproxMap₂ (V := V)).rel X Y W ↔
        (PDsingletonApproxMap (V := V)).rel X W ∧ (PDsingletonApproxMap (V := V)).rel
    Y W) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (x y : V.Element) {W : Set (Set α)},
      (V.PDfinJoin 2 ![x, y]).mem W ↔ (V.PDsingleton x).mem W ∧ (V.PDsingleton y).mem
    W) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (x y : V.Element),
      V.PDfinJoin 2 ![x, y] = V.PDsingletonMeet (V.PDsingleton x) (V.PDsingleton y)) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (P : ComputablePresentation V) {n k : ℕ
    },
      (PDsingletonApproxMap (V := V)).rel (P.X n) (V.Ypd P k) ↔
        ∃ b ∈ decodeList k, P.X n ⊆ P.X b) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (x : V.Element),
      (PDsingletonApproxMap (V := V)).toElementMap x = V.PDsingleton x) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) {X X' : Set α} (hX : V.mem X) (hX' : V.
    mem X')
      (hX'X : X' ⊆ X),
      V.PDsingleton (V.principal hX) ≤ V.PDsingleton (V.principal hX')) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) {X : Set α} (hX : V.mem X),
      V.PDsingleton (V.principal hX) = V.PowerDomain.principal (V.PDmem_upSet hX)) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) (P : ComputablePresentation V)
      (cons : ℕ → ℕ) (hconsp : Nat.Primrec cons)
      (hcons : ∀ a b, cons (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b),
      ComputableMap P (V.PDPresentation P cons hconsp hcons) (PDsingletonApproxMap (V
    := V))) ∧
    ¬ (Vshape ? Vshape.PowerDomain) :=
  ⟨@PDfinJoinApproxMap₂_toElementMap, ⟨@PDfinJoinApproxMap₂_isComputable,
    ⟨@PDfinJoinApproxMap₂_rel_iff, ⟨@PDfinJoin_inter_two, ⟨@PDfinJoin_pair,
      ⟨@PDsingletonApproxMap_rel_Ypd_iff, ⟨@PDsingletonApproxMap_toElementMap,
        ⟨@PDsingleton_mono, ⟨@PDsingleton_principal, ⟨@singleton_isComputable,
          vshape_not_trianglelefteq_powerDomain⟩⟩⟩⟩⟩⟩⟩⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.8.30 Lecture VII exercises

EXERCISES

4.8.31 Inclusion structures

An inclusion structure on the integers determines a neighbourhood system, and every computable presentation reconstructs its system.

Definition (`InclStructure`). An inclusion structure is a reflexive and transitive recursive relation on natural numbers, equipped with a greatest index and a primitive-recursive operation choosing meets whenever two indices are consistent; consistency and the universal property of a proposed meet are recursively decidable.

Definition (`InclStructure.CONS`). Two indices are consistent when some index is included in both.

Definition (`InclStructure.MEET`). An index is a meet of two others when the indices included in it are exactly those included in both.

Definition (`InclStructure.toNbhd`). The neighbourhood represented by an index is the set of all indices included in it.

Theorem (`InclStructure.mem_toNbhd`). An index belongs to the neighbourhood represented by another exactly when it is included in that index.

Proof. This is the defining membership condition. ■

Theorem (`InclStructure.toNbhd_inter_eq_iff`). The intersection of two represented neighbourhoods equals a third represented neighbourhood exactly when the third index is their meet.

Proof. Extensional equality says that every index belongs to the third neighbourhood exactly when it belongs to both of the first two. ■

Theorem (`InclStructure.toNbhd_subset_iff`). One represented neighbourhood is included in another exactly when its representing index is included in the other index.

Proof. The forward direction applies inclusion to the reflexive member; the reverse direction uses transitivity. ■

Theorem (`InclStructure.toNbhd_subset_inter_iff`). A represented neighbourhood lies below an intersection exactly when its index is included in both indices.

Proof. Inclusion in an intersection is equivalent to the conjunction of the two inclusions. ■

Theorem (`InclStructure.toNbhd_top`). The neighbourhood represented by the greatest index is the whole set of indices.

Proof. Every index is included in the greatest index. ■

Definition (`InclStructure.toSystem`). An inclusion structure determines the neighbourhood system whose neighbourhoods are precisely its represented sets and whose master is the whole set of indices.

Theorem (`InclStructure.toSystem_mem`). A set is a neighbourhood of the induced system exactly when it is represented by some index.

Proof. This is the defining neighbourhood predicate. ■

Theorem (`meet_iff_interEq`). In a computable presentation, the universal lower-bound property for three numbered neighbourhoods is equivalent to equality of the intersection of the first two with the third.

Proof. Equality immediately gives the lower-bound property. Conversely, choose a numbered representative of the intersection and compare it with the proposed meet in both directions. ■

Definition (`ofPresentation`). A computable presentation determines an inclusion structure by ordering indices according to inclusion of their numbered neighbourhoods, using the presentation's intersection and master indices.

Theorem (`ofPresentation_INCL`). Inclusion in the structure obtained from a presentation is exactly inclusion between the corresponding numbered neighbourhoods.

Proof. This is the defining relation. ■

Theorem (`mem_ofPresentation_toNbhd`). An index belongs to the reconstructed neighbourhood of n exactly when its numbered neighbourhood is included in the neighbourhood numbered by n.

Proof. Expand the two defining membership relations. ■

Theorem (`ofPresentation_toNbhd_eq_iff`). Two reconstructed neighbourhoods are equal exactly when their original numbered neighbourhoods are equal.

Proof. Both equalities are characterized by mutual inclusion. ■

Definition (`reconstruct`). The reconstruction of a computable presentation is the neighbourhood system induced by its inclusion structure.

Definition (`reconElem`). An original element yields a reconstructed element containing a represented index-set exactly when the original element contains the corresponding numbered neighbourhood.

Definition (`reconElemInv`). A reconstructed element yields an original element containing a neighbourhood exactly when some equal numbered neighbourhood has its represented index-set in the reconstructed element.

Definition (`reconIso`). The preceding two transformations form an order isomorphism between the original elements and the reconstructed elements.

Definition (`InclStructure.toPresentation`). The induced neighbourhood system has a computable presentation given by the represented neighbourhoods, the structure's meet operation, and its greatest index.

Theorem (`InclStructure.axiom_i`). Every index is included in itself.

Proof. This is reflexivity of the inclusion structure. ■

Theorem (`InclStructure.axiom_ii`). Inclusion of indices is transitive.

Proof. This is transitivity of the inclusion structure. ■

Theorem (`InclStructure.axiom_iii`). There is an index above every index.

Proof. The greatest index has this property. ■

Theorem (`InclStructure.axiom_iv`). Every consistent pair of indices has a meet.

Proof. The designated meet index satisfies the meet specification whenever consistency has a witness. ■

Theorem (`ofPresentation_toPresentation_INCL`). Passing from an inclusion structure to its presentation and back preserves the original inclusion relation.

Proof. Inclusion between represented neighbourhoods is equivalent to inclusion between their indices. ■

Theorem (`reconstruct_isomorphic`). The reconstruction of a computable presentation is isomorphic to its original neighbourhood system.

Proof. Use the inverse order isomorphism between original and reconstructed elements. ■

Theorem (`InclStructure.toSystem_isEffectivelyGiven`). The neighbourhood system induced by an inclusion structure is effectively given.

Proof. Its represented neighbourhoods carry the computable presentation constructed above. ■

`inclStructure.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_1
public import Mathlib.Order.BourbakiWitt

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive

structure InclStructure where

  INCL : ℕ → ℕ → Prop

  meetIdx : ℕ → ℕ → ℕ

  topIdx : ℕ

  incl_dec : RecDecidable₂ INCL

  cons_dec : RecDecidable₂ (fun n m => ∃ k, INCL k n ∧ INCL k m)

  meet_dec : RecDecidable₃ (fun n m k => ∀ j, INCL j k ↔ (INCL j n ∧ INCL j m))

  meetIdx_primrec : Nat.Primrec (fun t => meetIdx t.unpair.1 t.unpair.2)

  incl_refl : ∀ n, INCL n n

  incl_trans : ∀ {n m k}, INCL n m → INCL m k → INCL n k

  topIdx_spec : ∀ n, INCL n topIdx

  meetIdx_spec : ∀ {n m}, (∃ k, INCL k n ∧ INCL k m) →
    ∀ j, INCL j (meetIdx n m) ↔ (INCL j n ∧ INCL j m)

end DomainTheory.NeighbourhoodSystems

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive

section
namespace InclStructure

variable (I : InclStructure)

def CONS (n m : ℕ) : Prop := ∃ k, I.INCL k n ∧ I.INCL k m

def MEET (n m k : ℕ) : Prop := ∀ j, I.INCL j k ↔ (I.INCL j n ∧ I.INCL j m)

def toNbhd (n : ℕ) : Set ℕ := {m | I.INCL m n}

@[simp] theorem mem_toNbhd {n m : ℕ} : m ∈ I.toNbhd n ↔ I.INCL m n := Iff.rfl

theorem toNbhd_inter_eq_iff (n m k : ℕ) :
```

```
    I.toNbhd n ∩ I.toNbhd m = I.toNbhd k ↔ I.MEET n m k := by
  unfold MEET
  rw [Set.ext_iff]
  refine forall_congr' fun j => ?_
  rw [Set.mem_inter_iff, mem_toNbhd, mem_toNbhd, mem_toNbhd]
  exact iff_comm

theorem toNbhd_subset_iff {n k : ℕ} : I.toNbhd n ⊆ I.toNbhd k ↔ I.INCL n k := by
  constructor
  · intro h
    exact h (I.incl_refl n)
  · intro hnk m hm
    exact I.incl_trans hm hnk

theorem toNbhd_subset_inter_iff (k n m : ℕ) :
    I.toNbhd k ⊆ I.toNbhd n ∩ I.toNbhd m ↔ (I.INCL k n ∧ I.INCL k m) := by
  rw [Set.subset_inter_iff, I.toNbhd_subset_iff, I.toNbhd_subset_iff]

theorem toNbhd_top : I.toNbhd I.topIdx = Set.univ := by
  ext m
  simp only [mem_toNbhd, Set.mem_univ, iff_true]
  exact I.topIdx_spec m

def toSystem : NeighborhoodSystem ℕ where
  mem Y := ∃ n, Y = I.toNbhd n
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := ⟨I.topIdx, I.toNbhd_top.symm⟩
  inter_mem := by
    rintro X Y Z ⟨n, rfl⟩ ⟨m, rfl⟩ ⟨l, rfl⟩ hZsub
    have hln : I.INCL l n := I.toNbhd_subset_iff.mp (hZsub.trans Set.inter_subset_left
    )
    have hlm : I.INCL l m := I.toNbhd_subset_iff.mp (hZsub.trans Set.
    inter_subset_right)
    exact ⟨I.meetIdx n m, (I.toNbhd_inter_eq_iff n m (I.meetIdx n m)).mpr
      (I.meetIdx_spec ⟨l, hln, hlm⟩)⟩
  sub_master := by rintro X _; exact Set.subset_univ X

@[simp] theorem toSystem_mem {Y : Set ℕ} : I.toSystem.mem Y ↔ ∃ n, Y = I.toNbhd n :=
    Iff.rfl

end InclStructure
end

section
variable {α : Type*} {V : NeighborhoodSystem α}

theorem meet_iff_interEq (P : ComputablePresentation V) (n m k : ℕ) :
    (∀ j, P.X j ⊆ P.X k ↔ (P.X j ⊆ P.X n ∧ P.X j ⊆ P.X m)) ↔ (P.X n ∩ P.X m = P.X k)
     := by
  constructor
  · intro h
    have hk : P.X k ⊆ P.X n ∩ P.X m :=
      Set.subset_inter ((h k).mp subset_rfl).1 ((h k).mp subset_rfl).2
    refine Set.Subset.antisymm ?_ hk
    have hmem : V.mem (P.X n ∩ P.X m) := V.inter_mem (P.mem_X n) (P.mem_X m) (P.mem_X
```

```
    k) hk
    obtain ⟨p, hp⟩ := P.surj hmem
    have hpk : P.X p ⊆ P.X k :=
      (h p).mpr (by rw [hp]; exact ⟨Set.inter_subset_left, Set.inter_subset_right⟩)
    exact hp ▸ hpk
  · intro h j
    rw [← h, Set.subset_inter_iff]

def ofPresentation (P : ComputablePresentation V) : InclStructure where
  INCL n m := P.X n ⊆ P.X m
  meetIdx := P.inter
  topIdx := P.masterIdx
  incl_dec := P.incl_computable
  cons_dec :=
    RecDecidable.of_iff
      (fun t => by
        refine exists_congr fun k => ?_
        exact (Set.subset_inter_iff).symm)
      P.cons_computable
  meet_dec :=
    RecDecidable.of_iff
      (fun t => meet_iff_interEq P t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2)
      P.interEq_computable
  meetIdx_primrec := P.inter_primrec
  incl_refl := fun _ => subset_rfl
  incl_trans := fun h1 h2 => h1.trans h2
  topIdx_spec := by
    intro n
    show P.X n ⊆ P.X P.masterIdx
    rw [P.masterIdx_spec]
    exact V.sub_master (P.mem_X n)
  meetIdx_spec := by
    intro n m hcons j
    obtain ⟨k, h1, h2⟩ := hcons
    have hcons' : ∃ k, P.X k ⊆ P.X n ∩ P.X m := ⟨k, Set.subset_inter h1 h2⟩
    show P.X j ⊆ P.X (P.inter n m) ↔ _
    rw [P.inter_spec hcons', Set.subset_inter_iff]

@[simp] theorem ofPresentation_INCL (P : ComputablePresentation V) (n m : ℕ) :
    (ofPresentation P).INCL n m ↔ P.X n ⊆ P.X m := Iff.rfl

@[simp] theorem mem_ofPresentation_toNbhd (P : ComputablePresentation V) (j n : ℕ) :
    j ∈ (ofPresentation P).toNbhd n ↔ P.X j ⊆ P.X n := Iff.rfl

theorem ofPresentation_toNbhd_eq_iff (P : ComputablePresentation V) (n m : ℕ) :
    (ofPresentation P).toNbhd n = (ofPresentation P).toNbhd m ↔ P.X n = P.X m := by
  rw [Set.Subset.antisymm_iff, (ofPresentation P).toNbhd_subset_iff,
    (ofPresentation P).toNbhd_subset_iff]
  show (P.X n ⊆ P.X m ∧ P.X m ⊆ P.X n) ↔ P.X n = P.X m
  exact (Set.Subset.antisymm_iff).symm

abbrev reconstruct (P : ComputablePresentation V) : NeighborhoodSystem ℕ :=
  (ofPresentation P).toSystem

def reconElem (P : ComputablePresentation V) (x : V.Element) : (reconstruct P).Element
    where
```

```
  mem Y := ∃ n, Y = (ofPresentation P).toNbhd n ∧ x.mem (P.X n)
  sub := by rintro Y ⟨n, rfl, _⟩; exact ⟨n, rfl⟩
  master_mem :=
    ⟨P.masterIdx, (ofPresentation P).toNbhd_top.symm, by rw [P.masterIdx_spec]; exact x
    .master_mem⟩
  inter_mem := by
    rintro X Y ⟨n, rfl, hxn⟩ ⟨m, rfl, hxm⟩
    have hmem : V.mem (P.X n ∩ P.X m) := x.sub (x.inter_mem hxn hxm)
    obtain ⟨q, hq⟩ := P.surj hmem
    refine ⟨q, ?_, ?_⟩
    · ext j
      rw [Set.mem_inter_iff, mem_ofPresentation_toNbhd, mem_ofPresentation_toNbhd,
        mem_ofPresentation_toNbhd, hq, Set.subset_inter_iff]
    · have : x.mem (P.X q) := by rw [hq]; exact x.inter_mem hxn hxm
      exact this
  up_mem := by
    rintro X Y ⟨n, rfl, hxn⟩ ⟨m, rfl⟩ hsub
    have hnm : P.X n ⊆ P.X m := (ofPresentation P).toNbhd_subset_iff.mp hsub
    exact ⟨m, rfl, x.up_mem hxn (P.mem_X m) hnm⟩

def reconElemInv (P : ComputablePresentation V) (y : (reconstruct P).Element) : V.
    Element where
  mem W := ∃ n, W = P.X n ∧ y.mem ((ofPresentation P).toNbhd n)
  sub := by rintro W ⟨n, rfl, _⟩; exact P.mem_X n
  master_mem :=
    ⟨P.masterIdx, P.masterIdx_spec.symm, by
      rw [show (ofPresentation P).toNbhd P.masterIdx = Set.univ from (ofPresentation P
    ).toNbhd_top]
      exact y.master_mem⟩
  inter_mem := by
    rintro X Y ⟨n, rfl, hyn⟩ ⟨m, rfl, hym⟩
    obtain ⟨p, hp⟩ := y.sub (y.inter_mem hyn hym)
    have hpmem : p ∈ (ofPresentation P).toNbhd n ∩ (ofPresentation P).toNbhd m :=
      hp ▸ ((ofPresentation P).mem_toNbhd.mpr ((ofPresentation P).incl_refl p))
    rw [Set.mem_inter_iff, mem_ofPresentation_toNbhd, mem_ofPresentation_toNbhd] at
    hpmem
    have hwit : P.X p ⊆ P.X n ∩ P.X m := Set.subset_inter hpmem.1 hpmem.2
    have hmem : V.mem (P.X n ∩ P.X m) := V.inter_mem (P.mem_X n) (P.mem_X m) (P.mem_X
    p) hwit
    obtain ⟨q, hq⟩ := P.surj hmem
    refine ⟨q, hq.symm, ?_⟩
    have htoNbhd : (ofPresentation P).toNbhd q
        = (ofPresentation P).toNbhd n ∩ (ofPresentation P).toNbhd m := by
      ext j
      rw [Set.mem_inter_iff, mem_ofPresentation_toNbhd, mem_ofPresentation_toNbhd,
        mem_ofPresentation_toNbhd, hq, Set.subset_inter_iff]
    rw [htoNbhd]
    exact y.inter_mem hyn hym
  up_mem := by
    rintro W Z ⟨n, rfl, hyn⟩ hZ hWZ
    obtain ⟨p, hp⟩ := P.surj hZ
    refine ⟨p, hp.symm, ?_⟩
    have hnp : (ofPresentation P).toNbhd n ⊆ (ofPresentation P).toNbhd p :=
      (ofPresentation P).toNbhd_subset_iff.mpr (by show P.X n ⊆ P.X p; rw [hp]; exact
    hWZ)
    exact y.up_mem hyn ⟨p, rfl⟩ hnp
```

```
def reconIso (P : ComputablePresentation V) : V.Element ≃o (reconstruct P).Element
    where
  toFun := reconElem P
  invFun := reconElemInv P
  left_inv := by
    intro x
    apply Element.ext
    intro W
    constructor
    · rintro ⟨n, rfl, m, hWm, hxm⟩
      rw [ofPresentation_toNbhd_eq_iff] at hWm
      rw [hWm]; exact hxm
    · intro hxW
      obtain ⟨n, hn⟩ := P.surj (x.sub hxW)
      exact ⟨n, hn.symm, n, rfl, by rw [hn]; exact hxW⟩
  right_inv := by
    intro y
    apply Element.ext
    intro T
    constructor
    · rintro ⟨n, rfl, m, hXm, hym⟩
      have : (ofPresentation P).toNbhd n = (ofPresentation P).toNbhd m :=
        (ofPresentation_toNbhd_eq_iff P n m).mpr hXm
      rw [this]; exact hym
    · intro hyT
      obtain ⟨n, hn⟩ := y.sub hyT
      exact ⟨n, hn, n, rfl, by rw [← hn]; exact hyT⟩
  map_rel_iff' := by
    intro x x'
    constructor
    · intro h W hxW
      obtain ⟨n, hn⟩ := P.surj (x.sub hxW)
      have hmem : (reconElem P x).mem ((ofPresentation P).toNbhd n) :=
        ⟨n, rfl, by rw [hn]; exact hxW⟩
      obtain ⟨m, hm, hx'm⟩ := h ((ofPresentation P).toNbhd n) hmem
      rw [ofPresentation_toNbhd_eq_iff] at hm
      rw [← hn, hm]; exact hx'm
    · intro h Y hY
      obtain ⟨n, rfl, hxn⟩ := hY
      exact ⟨n, rfl, h (P.X n) hxn⟩

end

section
namespace InclStructure

variable (I : InclStructure)

def toPresentation : ComputablePresentation I.toSystem where
  X := I.toNbhd
  mem_X n := ⟨n, rfl⟩
  surj := by rintro Y ⟨n, rfl⟩; exact ⟨n, rfl⟩
  interEq_computable :=
    RecDecidable.of_iff
      (fun t => (I.toNbhd_inter_eq_iff t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.
```

```
    unpair.2))
      I.meet_dec
  cons_computable :=
    RecDecidable.of_iff
      (fun t => by
        refine exists_congr fun k => ?_
        exact I.toNbhd_subset_inter_iff k t.unpair.1 t.unpair.2)
      I.cons_dec
  inter := I.meetIdx
  inter_primrec := I.meetIdx_primrec
  inter_spec := by
    intro n m hcons
    obtain ⟨k, hk⟩ := hcons
    rw [Set.subset_inter_iff] at hk
    have hcons' : I.CONS n m := ⟨k, I.toNbhd_subset_iff.mp hk.1, I.toNbhd_subset_iff.mp
     hk.2⟩
    exact ((I.toNbhd_inter_eq_iff n m (I.meetIdx n m)).mpr (I.meetIdx_spec hcons')).
    symm
  masterIdx := I.topIdx
  masterIdx_spec := I.toNbhd_top

theorem axiom_i (n : ℕ) : I.INCL n n := I.incl_refl n

theorem axiom_ii {n m k : ℕ} (h1 : I.INCL n m) (h2 : I.INCL m k) : I.INCL n k := I.
    incl_trans h1 h2

theorem axiom_iii : ∃ m, ∀ n, I.INCL n m := ⟨I.topIdx, I.topIdx_spec⟩

theorem axiom_iv (n m : ℕ) (h : I.CONS n m) : ∃ k, I.MEET n m k :=
  ⟨I.meetIdx n m, I.meetIdx_spec h⟩

end InclStructure
end

section
variable {α : Type*} {V : NeighborhoodSystem α}

theorem ofPresentation_toPresentation_INCL (I : InclStructure) (n m : ℕ) :
    (ofPresentation I.toPresentation).INCL n m ↔ I.INCL n m := by
  show I.toNbhd n ⊆ I.toNbhd m ↔ I.INCL n m
  exact I.toNbhd_subset_iff

theorem reconstruct_isomorphic (P : ComputablePresentation V) : reconstruct P ≅? V :=
  ⟨(reconIso P).symm⟩

end

section
namespace InclStructure

variable (I : InclStructure)

theorem toSystem_isEffectivelyGiven : I.toSystem.IsEffectivelyGiven := ⟨I.
    toPresentation⟩

end InclStructure
```

```
end

end DomainTheory.NeighbourhoodSystems
end
```

4.8.32 Exercise 7.13 — axiomatic neighbourhood structure

Show that an effectively given domain can always be identified with a relation $\mathrm{INCL}(n, m)$ on integers, where the two derived relations

$$\mathrm{CONS}(n, m) \text{ iff } \exists k.\, \mathrm{INCL}(k, n) \text{ and } \mathrm{INCL}(k, m);$$

$$\mathrm{MEET}(n, m, k) \text{ iff } \forall j\, [\mathrm{INCL}(j, k) \text{ iff } \mathrm{INCL}(j, n) \text{ and } \mathrm{INCL}(j, m)]$$

are both recursively decidable, and where the following axioms hold:

(i) $\forall n.\, \mathrm{INCL}(n, n)$;

(ii) $\forall n, m, k.\, \mathrm{INCL}(n, m)$ and $\mathrm{INCL}(m, k)$ imply $\mathrm{INCL}(n, k)$;

(iii) $\exists m\, \forall n.\, \mathrm{INCL}(n, m)$

(iv) $\forall n, m.\, \mathrm{CONS}(n, m)$ implies $\exists k.\, \mathrm{MEET}(n, m, k)$.

(Hint: Consider the neighbourhood system

$$\mathcal{D} = \big\{\{m \in \mathbf{N} \mid \mathrm{INCL}(m, n)\} \mid n \in \mathbf{N}\big\}.$$

Is this essentially any effectively given system?)

The development is with the inclusion structures peer.

Theorem (`example_713`). Every computable presentation reconstructs a system isomorphic to the original, and every inclusion structure is effectively given.

Proof. By the reconstruction isomorphism and the presentation of an inclusion structure. ■

`Exercise_7_13.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .inclStructure

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem
open DomainTheory.NeighbourhoodSystems.InclStructure
/-- Scott's Exercise 7.13: reconstructing from a presentation; inclusion structures
    are effectively given. -/
theorem example_713 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} (P : ComputablePresentation V),
```

```
    reconstruct P ≅? V) ∧
  (∀ (I : InclStructure), I.toSystem.IsEffectivelyGiven) :=
⟨@reconstruct_isomorphic, @InclStructure.toSystem_isEffectivelyGiven⟩
```

4.8.33 Computable witnesses

A computable element is the decreasing union of the principal elements named by a primitive-recursive sequence of indices.

Definition (`ComputableWitness.tFun`). From a sequence of presentation indices, form a new sequence by taking the first index initially and then repeatedly choosing an index for the intersection with the next neighbourhood.

Theorem (`ComputableWitness.repred_exists_primrec_range`). Every nonempty recursively enumerable predicate on the natural numbers is the range of a primitive-recursive function.

Proof. Enumerate witnesses to the predicate and return a fixed known witness whenever the current search certificate fails. ■

Theorem (`ComputableWitness.tFun_zero`). The initial value of the thinning sequence is the initial value of the original sequence.

Proof. This is the initial clause of the recursion. ■

Theorem (`ComputableWitness.tFun_succ`). Each successor value of the thinning sequence is the chosen intersection index of its preceding value and the next original value.

Proof. This is the successor clause of the recursion. ■

Theorem (`ComputableWitness.primrec_tFun`). Thinning a primitive-recursive sequence along a computable presentation produces a primitive-recursive sequence.

Proof. Primitive recursion applies because the presentation's intersection-index operation and the original sequence are primitive recursive. ■

Theorem (`ComputableWitness.computableElement_eq_decreasing_iUnion_principal`). Every computable element is generated by a primitive-recursive decreasing sequence of principal elements: a neighbourhood belongs to the element exactly when it belongs to one of those principals.

Proof. Enumerate the indices of neighbourhoods containing the element, starting with a master index, and thin this enumeration by successive intersections. Every original neighbourhood occurs above the corresponding thinned neighbourhood, while every thinned neighbourhood still contains the element. ■

Theorem (`ComputableWitness.repred_range_primrec`). The range of a primitive-recursive function is recursively enumerable.

Proof. A number belongs to the range exactly when some coded pair has first component mapped to its second component. ■

Theorem (`ComputableWitness.repred2_exists_primrec_enum`). Every inhabited recursively enumerable binary relation has a primitive-recursive enumeration by two coordinate functions.

Proof. Enumerate the paired relation by one primitive-recursive function and take the two projections of each value. ■

Theorem (`NeighborhoodSystem.PDmem_upSet_inter`). The intersection of the upper sets generated by two neighbourhoods is a power-domain neighbourhood.

Proof. It is the upper set of their intersection when that intersection is a neighbourhood, and is empty otherwise; both forms are power-domain neighbourhoods. ■

Theorem (`NeighborhoodSystem.upSetUnion_cons`). The union of upper sets indexed by a list with head Y is the union of the upper set of Y and the union indexed by the tail.

Proof. Split membership in the list according to whether the chosen entry is the head or lies in the tail. ■

Theorem (`NeighborhoodSystem.upSetUnion_nil`). The union of upper sets indexed by the empty list is empty.

Proof. The empty list has no indexing member. ■

Theorem (`NeighborhoodSystem.PDmem_upSet_inter_biUnion`). The intersection of one principal upper set with any finite union of principal upper sets is a power-domain neighbourhood.

Proof. Induct on the list and distribute intersection over union, using closure under binary union. ■

Theorem (`NeighborhoodSystem.PDmem_biUnion_inter`). The intersection of a finite union of principal upper sets with any power-domain neighbourhood is a power-domain neighbourhood.

Proof. Induct on the first finite list and use the preceding principal-intersection result. ■

Theorem (`NeighborhoodSystem.PDmem_inter`). Power-domain neighbourhoods are closed under intersection.

Proof. Represent the first neighbourhood as a finite union of principal upper sets and apply the preceding theorem. ■

computableWitness.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_2
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_9

@[expose] public section

section

namespace DomainTheory.NeighbourhoodSystems.ComputableWitness

open NeighborhoodSystem Domain.Recursive DomainTheory.NeighbourhoodSystems

variable {β : Type*}

def tFun {W : NeighborhoodSystem β} (Q : ComputablePresentation W) (r : ℕ → ℕ) (n : ℕ
    ) :
    ℕ :=
  Nat.rec (motive := fun _ => ℕ) (r 0) (fun y IH => Q.inter IH (r (y + 1))) n

end DomainTheory.NeighbourhoodSystems.ComputableWitness
end

section
namespace DomainTheory.NeighbourhoodSystems.ComputableWitness
open NeighborhoodSystem Domain.Recursive DomainTheory.NeighbourhoodSystems

section

theorem repred_exists_primrec_range {p : ℕ → Prop} (hp : REPred p) {a : ℕ} (ha : p a)
    :
```

```
    ∃ r : ℕ → ℕ, Nat.Primrec r ∧ ∀ n, p n ↔ ∃ i, r i = n := by
  obtain ⟨q, ⟨qc, hqcp, hqcs⟩, hqe⟩ := hp
  refine ⟨fun w => selectFn (isOne (qc w)) w.unpair.2 a,
    primrec_selectFn (primrec_isOne.comp hqcp) Nat.Primrec.right (Nat.Primrec.const a)
    , ?_⟩

  have hrw_mem : ∀ w, p (selectFn (isOne (qc w)) w.unpair.2 a) := by
    intro w
    rcases (show isOne (qc w) = 0 ∨ isOne (qc w) = 1 by
        have := isOne_le_one (qc w); omega) with h0 | h1
    · rw [h0, selectFn_zero]; exact ha
    · rw [h1, selectFn_one]
      have hqw : q w := (hqcs w).mpr ((isOne_eq_one_iff (qc w)).mp h1)
      rw [hqe w.unpair.2]
      exact ⟨w.unpair.1, by rw [pair_unpair]; exact hqw⟩
  intro n
  constructor
  · intro hpn
    rw [hqe n] at hpn
    obtain ⟨i, hi⟩ := hpn
    refine ⟨Nat.pair i n, ?_⟩
    show selectFn (isOne (qc (Nat.pair i n))) (Nat.pair i n).unpair.2 a = n
    have hqc1 : isOne (qc (Nat.pair i n)) = 1 := (isOne_eq_one_iff _).mpr ((hqcs _).mp
     hi)
    rw [hqc1, selectFn_one, unpair_pair_snd]
  · rintro ⟨w, rfl⟩
    exact hrw_mem w

variable {β : Type*}

theorem tFun_zero {W : NeighborhoodSystem β} (Q : ComputablePresentation W) (r : ℕ →
    ℕ) :
    tFun Q r 0 = r 0 := rfl

private theorem tFun_succ {W : NeighborhoodSystem β} (Q : ComputablePresentation W) (r
     : ℕ → ℕ)
    (i : ℕ) : tFun Q r (i + 1) = Q.inter (tFun Q r i) (r (i + 1)) := rfl

private theorem primrec_tFun {W : NeighborhoodSystem β} (Q : ComputablePresentation W)
     {r : ℕ → ℕ}
    (hr : Nat.Primrec r) : Nat.Primrec (tFun Q r) := by
  have hg : Nat.Primrec (fun w => Q.inter w.unpair.2.unpair.2 (r (w.unpair.2.unpair.1
    + 1))) := by
    have hIH : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2) :=
      Nat.Primrec.right.comp Nat.Primrec.right
    have hy1 : Nat.Primrec (fun w : ℕ => r (w.unpair.2.unpair.1 + 1)) :=
      hr.comp (Nat.Primrec.succ.comp (Nat.Primrec.left.comp Nat.Primrec.right))
    exact (Q.inter_primrec.comp (hIH.pair hy1)).of_eq fun w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  refine ((Nat.Primrec.prec (Nat.Primrec.const (r 0)) hg).comp
    ((Nat.Primrec.const 0).pair primrec_id)).of_eq (fun n => ?_)
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rfl

theorem computableElement_eq_decreasing_iUnion_principal {W : NeighborhoodSystem β}
    (Q : ComputablePresentation W) {y : W.Element} (hy : ComputableElement Q y) :
```

```
    ∃ t : ℕ → ℕ, Nat.Primrec t ∧
      (∀ i, Q.X (t (i + 1)) ⊆ Q.X (t i)) ∧
      (∀ Z, y.mem Z ↔ ∃ i, (W.principal (Q.mem_X (t i))).mem Z) := by
  have hy' : REPred (fun m => y.mem (Q.X m)) := hy.out

  obtain ⟨m₀, hm₀⟩ := Q.surj W.master_mem
  have hwit : y.mem (Q.X m₀) := by rw [hm₀]; exact y.master_mem
  obtain ⟨r₀, hr₀, hr0_spec⟩ := repred_exists_primrec_range hy' hwit

  have hr0_mem : ∀ i, y.mem (Q.X (r₀ i)) := fun i => (hr0_spec (r₀ i)).mpr ⟨i, rfl⟩

  have hcons : ∀ a c : ℕ, y.mem (Q.X a) → y.mem (Q.X c) →
      ∃ k, Q.X k ⊆ Q.X a ∩ Q.X c := by
    intro a c ha hc
    obtain ⟨k, hk⟩ := Q.surj (y.sub (y.inter_mem ha hc))
    exact ⟨k, hk.subset⟩

  have ht_mem : ∀ i, y.mem (Q.X (tFun Q r₀ i)) := by
    intro i
    induction i with
    | zero => rw [tFun_zero]; exact hr0_mem 0
    | succ k ih =>
      have heq : Q.X (tFun Q r₀ (k + 1)) = Q.X (tFun Q r₀ k) ∩ Q.X (r₀ (k + 1)) :=
        Q.inter_spec (hcons (tFun Q r₀ k) (r₀ (k + 1)) ih (hr0_mem (k + 1)))
      rw [heq]; exact y.inter_mem ih (hr0_mem (k + 1))

  have ht_eq : ∀ i, Q.X (tFun Q r₀ (i + 1)) = Q.X (tFun Q r₀ i) ∩ Q.X (r₀ (i + 1)) :=
    fun i =>
    Q.inter_spec (hcons (tFun Q r₀ i) (r₀ (i + 1)) (ht_mem i) (hr0_mem (i + 1)))

  have ht_dec : ∀ i, Q.X (tFun Q r₀ (i + 1)) ⊆ Q.X (tFun Q r₀ i) := by
    intro i; rw [ht_eq i]; exact Set.inter_subset_left

  have ht_sub_r : ∀ i, Q.X (tFun Q r₀ i) ⊆ Q.X (r₀ i) := by
    intro i
    cases i with
    | zero => rw [tFun_zero]
    | succ k => rw [ht_eq k]; exact Set.inter_subset_right
  refine ⟨tFun Q r₀, primrec_tFun Q hr₀, ht_dec, fun Z => ?_⟩
  constructor
  · intro hZ
    obtain ⟨m, hm⟩ := Q.surj (y.sub hZ)
    have hym : y.mem (Q.X m) := by rw [hm]; exact hZ
    obtain ⟨i, hi⟩ := (hr0_spec m).mp hym
    have hZeq : Q.X (r₀ i) = Z := by rw [hi]; exact hm
    exact ⟨i, y.sub hZ, by rw [← hZeq]; exact ht_sub_r i⟩
  · rintro ⟨i, hWZ, hsub⟩
    exact y.up_mem (ht_mem i) hWZ hsub
end

theorem repred_range_primrec {r : ℕ → ℕ} (hr : Nat.Primrec r) :
    REPred (fun n => ∃ i, r i = n) := by
  refine ⟨fun t => r t.unpair.1 = t.unpair.2,
    RecDecidable.natEq (hr.comp Nat.Primrec.left) Nat.Primrec.right, fun n => ?_⟩
  refine exists_congr (fun i => ?_)
  simp only [unpair_pair_fst, unpair_pair_snd]
```

```
theorem repred₂_exists_primrec_enum {p : ℕ → ℕ → Prop} (hp : REPred₂ p) {a b : ℕ}
    (hab : p a b) :
    ∃ s r : ℕ → ℕ, Nat.Primrec s ∧ Nat.Primrec r ∧
      ∀ n m, p n m ↔ ∃ i, s i = n ∧ r i = m := by
  have hp' : REPred (fun t => p t.unpair.1 t.unpair.2) := hp
  have hwit : (fun t => p t.unpair.1 t.unpair.2) (Nat.pair a b) := by
    simp only [unpair_pair_fst, unpair_pair_snd]; exact hab
  obtain ⟨pf, hpf, hspec⟩ := repred_exists_primrec_range hp' hwit
  refine ⟨fun i => (pf i).unpair.1, fun i => (pf i).unpair.2,
    Nat.Primrec.left.comp hpf, Nat.Primrec.right.comp hpf, fun n m => ?_⟩
  have h := hspec (Nat.pair n m)
  simp only [unpair_pair_fst, unpair_pair_snd] at h
  rw [h]
  constructor
  · rintro ⟨i, hi⟩
    refine ⟨i, ?_, ?_⟩
    · show (pf i).unpair.1 = n; rw [hi, unpair_pair_fst]
    · show (pf i).unpair.2 = m; rw [hi, unpair_pair_snd]
  · rintro ⟨i, hs, hr⟩
    refine ⟨i, ?_⟩
    change (pf i).unpair.1 = n at hs
    change (pf i).unpair.2 = m at hr
    rw [← hs, ← hr]; exact (pair_unpair _).symm

end DomainTheory.NeighbourhoodSystems.ComputableWitness
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive

section
namespace NeighborhoodSystem

variable {α : Type*} (V : NeighborhoodSystem α)

theorem PDmem_upSet_inter {X Y : Set α} (hX : V.mem X) (hY : V.mem Y) :
    V.PDmem (V.upSet X ∩ V.upSet Y) := by
  rw [V.upSet_inter]
  by_cases h : V.mem (X ∩ Y)
  · exact V.PDmem_upSet h
  · have hempty : V.upSet (X ∩ Y) = ∅ := by
      ext Z
      simp only [mem_upSet, Set.mem_empty_iff_false, iff_false, not_and]
      intro hZmem hZsub
      exact h (V.inter_mem hX hY hZmem hZsub)
    rw [hempty]; exact V.PDmem_empty

theorem upSetUnion_cons (Y : Set α) (L : List (Set α)) :
    (⋃ X ∈ (Y :: L), V.upSet X) = V.upSet Y ∪ ⋃ X ∈ L, V.upSet X := by
  ext z
  rw [Set.mem_union, V.mem_PDunion, V.mem_PDunion]
  constructor
  · rintro ⟨X, hX, hz⟩
    rcases List.mem_cons.mp hX with rfl | hX'
    · exact Or.inl hz
    · exact Or.inr ⟨X, hX', hz⟩
  · rintro (hz | ⟨X, hX, hz⟩)
```

```
    · exact ⟨Y, List.mem_cons.mpr (Or.inl rfl), hz⟩
    · exact ⟨X, List.mem_cons.mpr (Or.inr hX), hz⟩

theorem upSetUnion_nil : (⋃ X ∈ ([] : List (Set α)), V.upSet X) = ∅ := by
  ext z
  rw [V.mem_PDunion]
  simp only [List.not_mem_nil, false_and, exists_false, Set.mem_empty_iff_false]

theorem PDmem_upSet_inter_biUnion {X : Set α} (hX : V.mem X) :
    ∀ (L : List (Set α)), (∀ Y ∈ L, V.mem Y) → V.PDmem (V.upSet X ∩ ⋃ Y ∈ L, V.upSet
    Y) := by
  intro L
  induction L with
  | nil =>
    intro _
    rw [V.upSetUnion_nil, Set.inter_empty]
    exact V.PDmem_empty
  | cons Y L ih =>
    intro hL
    rw [V.upSetUnion_cons, Set.inter_union_distrib_left]
    exact V.PDmem_union (V.PDmem_upSet_inter hX (hL Y (List.mem_cons.mpr (Or.inl rfl))
    ))
      (ih (fun Z hZ => hL Z (List.mem_cons.mpr (Or.inr hZ))))

theorem PDmem_biUnion_inter :
    ∀ (L₁ : List (Set α)), (∀ X ∈ L₁, V.mem X) → ∀ {W₂ : Set (Set α)}, V.PDmem W₂ →
      V.PDmem ((⋃ X ∈ L₁, V.upSet X) ∩ W₂) := by
  intro L₁
  induction L₁ with
  | nil =>
    intro _ W₂ _
    rw [V.upSetUnion_nil, Set.empty_inter]
    exact V.PDmem_empty
  | cons X L ih =>
    intro hL W₂ hW₂
    rw [V.upSetUnion_cons, Set.union_inter_distrib_right]
    refine V.PDmem_union ?_ (ih (fun Z hZ => hL Z (List.mem_cons.mpr (Or.inr hZ))) hW₂)
    obtain ⟨L₂, hL₂, rfl⟩ := hW₂
    exact V.PDmem_upSet_inter_biUnion (hL X (List.mem_cons.mpr (Or.inl rfl))) L₂ hL₂

theorem PDmem_inter {W₁ W₂ : Set (Set α)} (h₁ : V.PDmem W₁) (h₂ : V.PDmem W₂) :
    V.PDmem (W₁ ∩ W₂) := by
  obtain ⟨L₁, hL₁, rfl⟩ := h₁
  exact V.PDmem_biUnion_inter L₁ hL₁ h₂

end NeighborhoodSystem
end
end DomainTheory.NeighbourhoodSystems
end
```

4.8.34 Exercise 7.14 — primitive recursive witnesses

(For recursive-function theorists.) Prove the statements after definition 7.2 about the existence of primitive recursive functions for showing things recursively enumerable. (Recall that a non-empty set is r.e. iff it

is the range of a primitive recursive function.) Show also that every computable element $y \in |E|$ can be written

$$y = \bigcup \{\uparrow Y_{t(i)} \mid i \in \mathbf{N}\},$$

where $t : \mathbf{N} \to \mathbf{N}$ is primitive recursive and where we may assume
$Y_{t(i+1)} \subseteq Y_{t(i)}$
for all $i \in \mathbf{N}$.

The development is with the computable witnesses peer.

Theorem (`example_714`). A computable element is the decreasing union of principal elements along a primitive-recursive sequence of indices.

Proof. By the computable witness. ■

`Exercise_7_14.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .computableWitness

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem Domain.Recursive DomainTheory.NeighbourhoodSystems
open DomainTheory.NeighbourhoodSystems.ComputableWitness
/-- Scott's Exercise 7.14: computable elements are decreasing unions of principals. -/
theorem example_714 :
    ∀ {β : Type*} {W : NeighborhoodSystem β} (Q : ComputablePresentation W) {y : W.
    Element},
      ComputableElement Q y →
        ∃ t : ℕ → ℕ, Nat.Primrec t ∧
          (∀ i, Q.X (t (i + 1)) ⊆ Q.X (t i)) ∧
          (∀ Z, y.mem Z ↔ ∃ i, (W.principal (Q.mem_X (t i))).mem Z) :=
  @computableElement_eq_decreasing_iUnion_principal
```

4.8.35 Scott presentation

A Scott presentation records intersection equality and consistency as recursively decidable relations.

Definition (`ScottPresentation`). A Scott presentation of a neighbourhood system is a surjective numbering of its neighbourhoods for which equality of a numbered intersection with a third numbered neighbourhood, and the existence of a numbered neighbourhood below two given ones, are recursively decidable.

Definition (`ComputableMapS`). An approximable map between Scott-presented systems is computable when the relation between pairs of numbered basic neighbourhoods is recursively enumerable.

Definition (`NeighborhoodSystem.IsEffectivelyGivenS`). A neighbourhood system is effectively given in the Scott sense when it admits a Scott presentation.

Definition (`ScottPresentation.inclShuffle`). The inclusion shuffle sends a pair of indices (n, m) to the triple (n, m, n).

Theorem (`ScottPresentation.primrec_inclShuffle`). The inclusion shuffle is primitive recursive.

Proof. Pairing, both projections, and their compositions are primitive recursive. ■

Theorem (`ScottPresentation.incl_computable`). Inclusion between numbered neighbourhoods of a Scott presentation is recursively decidable.

Proof. The inclusion $X_n \subseteq X_m$ is equivalent to $X_n \cap X_m = X_n$, an instance of decidable intersection equality. ■

Definition (`ScottPresentation.swapPair`). The pair-swap operation sends (n, m) to (m, n).

Theorem (`ScottPresentation.primrec_swapPair`). Pair-swap is primitive recursive.

Proof. It is obtained by pairing the second and first projections. ■

Theorem (`ScottPresentation.eq_computable`). Equality between numbered neighbourhoods of a Scott presentation is recursively decidable.

Proof. Two sets are equal exactly when each is included in the other, and both inclusions are decidable. ■

Definition (`ComputablePresentation.toScott`). Every computable presentation has an underlying Scott presentation with the same numbering and decision procedures.

Theorem (`NeighborhoodSystem.IsEffectivelyGiven.toS`). Every effectively given neighbourhood system is effectively given in the Scott sense.

Proof. Forget the chosen intersection index and master index from a computable presentation. ■

`scottPresentation.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_7
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_3

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive IndefProduct ApproximableMap

section
variable {α β ? : Type*}

structure ScottPresentation (V : NeighborhoodSystem α) where

  X : ℕ → Set α

  mem_X : ∀ n, V.mem (X n)

  surj : ∀ {Y : Set α}, V.mem Y → ∃ n, X n = Y

  interEq_computable : RecDecidable₃ (fun n m k => X n ∩ X m = X k)

  cons_computable : RecDecidable₂ (fun n m => ∃ k, X k ⊆ X n ∩ X m)

class ComputableMapS {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
    (P : ScottPresentation V) (Q : ScottPresentation W) (f : ApproximableMap V W) :
    Prop where
```

```
  out : REPred₂ (fun n m => f.rel (P.X n) (Q.X m))

def NeighborhoodSystem.IsEffectivelyGivenS (V : NeighborhoodSystem α) : Prop :=
  Nonempty (ScottPresentation V)

namespace ScottPresentation

variable {V : NeighborhoodSystem α} (P : ScottPresentation V)

def inclShuffle (t : ℕ) : ℕ := Nat.pair t.unpair.1 (Nat.pair t.unpair.2 t.unpair.1)

theorem primrec_inclShuffle : Nat.Primrec inclShuffle :=
  Nat.Primrec.pair Nat.Primrec.left (Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.
    left)

theorem incl_computable : RecDecidable₂ (fun n m => P.X n ⊆ P.X m) := by
  refine RecDecidable.of_iff (fun t => ?_) (P.interEq_computable.comp
    primrec_inclShuffle)
  simp only [inclShuffle, unpair_pair_fst, unpair_pair_snd]
  exact Set.inter_eq_left.symm

end ScottPresentation

end

section
variable {α β ? : Type*}

namespace ScottPresentation

variable {V : NeighborhoodSystem α} (P : ScottPresentation V)

def swapPair (t : ℕ) : ℕ := Nat.pair t.unpair.2 t.unpair.1

theorem primrec_swapPair : Nat.Primrec swapPair :=
  Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left

theorem eq_computable : RecDecidable₂ (fun n m => P.X n = P.X m) := by
  refine RecDecidable.of_iff (fun t => ?_)
    (P.incl_computable.and (P.incl_computable.comp primrec_swapPair))
  simp only [swapPair, unpair_pair_fst, unpair_pair_snd]
  exact Set.Subset.antisymm_iff

end ScottPresentation
end

end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive

variable {α β ? : Type*}

def ComputablePresentation.toScott {V : NeighborhoodSystem α} (P :
    ComputablePresentation V) :
    ScottPresentation V where
```

```
  X := P.X
  mem_X := P.mem_X
  surj := P.surj
  interEq_computable := P.interEq_computable
  cons_computable := P.cons_computable

namespace ScottPresentation

variable {V : NeighborhoodSystem α} (P : ScottPresentation V)

end ScottPresentation

section
variable {α β ? : Type*}

theorem NeighborhoodSystem.IsEffectivelyGiven.toS {V : NeighborhoodSystem α}
    (h : V.IsEffectivelyGiven) : V.IsEffectivelyGivenS :=
  h.elim fun P => ⟨P.toScott⟩

end
end DomainTheory.NeighbourhoodSystems
```

4.8.36 Effective smash

The smash product is enumerated by pairing neighbourhoods, collapsing a factor that is the master, and its combinators are computable.

Definition (`smashEnum`). A paired index enumerates the product neighbourhood of its two components when both are proper; otherwise it enumerates the smash master neighbourhood.

Theorem (`smashEnum_master`). If at least one indexed component is the master neighbourhood, the smash enumeration returns the smash master.

Proof. The proper-pair condition fails in the defining alternative. ■

Theorem (`smashEnum_proper`). If both indexed components are proper, the smash enumeration returns their product neighbourhood.

Proof. The proper-pair condition holds in the defining alternative. ■

Theorem (`smashEnum_eq_eff`). Given Boolean characteristic functions for proper component neighbourhoods, the smash enumeration is the product of the selected component indices, where a failed properness test selects the appropriate master index.

Proof. If both tests succeed, both original indices are selected; if either fails, both selections yield master neighbourhoods and hence the smash master. ■

Theorem (`smashEnum_eq_prodNbhd_mem`). Every enumerated smash neighbourhood is a product neighbourhood of two component neighbourhoods.

Proof. Use the indexed components in the proper case and the two masters otherwise. ■

Theorem (`smashEnum_mem`). Every set in the smash enumeration is a smash neighbourhood.

Proof. A proper pair gives a proper smash neighbourhood, while every improper pair gives the master. ■

Theorem (`smashEnum_subset_master`). Every enumerated smash neighbourhood is included in the product of the component masters.

Proof. Every neighbourhood is included in the smash master, which is that product. ■

Theorem (`proper0_dec`). Properness of a numbered neighbourhood in the first component is recursively decidable.

Proof. Equality with the numbered master is decidable, so its negation is decidable. ■

Theorem (`proper1_dec`). Properness of a numbered neighbourhood in the second component is recursively decidable.

Proof. Equality with the numbered master is decidable, so its negation is decidable. ■

Definition (`smashPresentation`). The smash Scott presentation uses the smash enumeration and decides intersection equality and consistency by reducing them to the corresponding component relations and properness tests.

Theorem (`smashPresentation_X`). The neighbourhood numbered by an index in the smash presentation is its smash-enumerated neighbourhood.

Proof. This is the presentation's defining enumeration. ■

Definition (`smashProj0`). The first smash projection relates W to X when both are neighbourhoods and the left preimage of W is included in X.

Theorem (`smashProj0_rel`). The relation of the first smash projection is exactly the stated neighbourhood and left-preimage inclusion condition.

Proof. This is its defining relation. ■

Definition (`smashProj1`). The second smash projection relates W to Y when both are neighbourhoods and the right preimage of W is included in Y.

Theorem (`smashProj1_rel`). The relation of the second smash projection is exactly the stated neighbourhood and right-preimage inclusion condition.

Proof. This is its defining relation. ■

Definition (`smashPaired`). Pairing two approximable maps relates a source neighbourhood to the smash master automatically, and to a proper product neighbourhood when the two maps relate it to the respective factors.

Theorem (`smashPaired_rel`). The relation of smash pairing is precisely the disjunction between the master case and the proper componentwise case.

Proof. This is its defining relation. ■

Definition (`smashMap`). Two approximable maps act componentwise on smash products, with every source related to the target master and proper products related through the two component maps.

Theorem (`smashMap_isComputable`). The componentwise smash map of two computable maps is computable.

Proof. Proper target pairs reduce the relation to the two recursively enumerable component relations after primitive-recursive index selection; an improper target is always related. ■

Theorem (`smashPaired_isComputable`). Smash pairing of two computable maps is computable.

Proof. For a proper target pair the relation is the conjunction of the two component relations; for the target master it holds automatically. ■

Theorem (`smashProj0_isComputable`). The first smash projection is computable.

Proof. On enumerated neighbourhoods its relation is decidable inclusion of the effectively selected left component into the target neighbourhood. ■

Theorem (`smashProj1_isComputable`). The second smash projection is computable.

Proof. On enumerated neighbourhoods its relation is decidable inclusion of the effectively selected right component into the target neighbourhood. ■

Theorem (`smash_isEffectivelyGivenS`). The smash product of two effectively given systems is effectively given in the Scott sense.

Proof. Choose presentations of both factors and use their smash presentation. ■

smashEffective.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .scottPresentation
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_4
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_7
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_3

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive IndefProduct ApproximableMap

section
variable {α β ? : Type*}
open SmashProduct

section Smash

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

open scoped Classical in

noncomputable def smashEnum (t : ℕ) : Set (α ? β) :=
  if P₀.X t.unpair.1 = V₀.master ∧ P₁.X t.unpair.2 = V₁.master then
    prodNbhd (P₀.X t.unpair.1) (P₁.X t.unpair.2)
  else prodNbhd V₀.master V₁.master

end Smash

section Smash

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

open scoped Classical in
theorem smashEnum_master {t : ℕ}
    (h : ¬(P₀.X t.unpair.1 = V₀.master ∧ P₁.X t.unpair.2 = V₁.master)) :
    smashEnum P₀ P₁ t = prodNbhd V₀.master V₁.master := ite_eq_right h

open scoped Classical in
theorem smashEnum_proper {t : ℕ}
    (h : P₀.X t.unpair.1 = V₀.master ∧ P₁.X t.unpair.2 = V₁.master) :
    smashEnum P₀ P₁ t = prodNbhd (P₀.X t.unpair.1) (P₁.X t.unpair.2) := ite_eq_left h

theorem smashEnum_eq_eff {c₀ c₁ : ℕ → ℕ} (hc₀le : ∀ n, c₀ n ≤ 1) (hc₁le : ∀ n, c₁ n ≤
```

```
    1)
    (hc₀ : ∀ n, P₀.X n = V₀.master ↔ c₀ n = 1) (hc₁ : ∀ n, P₁.X n = V₁.master ↔ c₁ n =
    1) (t : ℕ) :
    smashEnum P₀ P₁ t =
      prodNbhd (P₀.X (selectFn (c₀ t.unpair.1 * c₁ t.unpair.2) t.unpair.1 P₀.masterIdx)
    )
        (P₁.X (selectFn (c₀ t.unpair.1 * c₁ t.unpair.2) t.unpair.2 P₁.masterIdx)) := by
  by_cases h : P₀.X t.unpair.1 = V₀.master ∧ P₁.X t.unpair.2 = V₁.master
  · have e0 : c₀ t.unpair.1 = 1 := (hc₀ _).mp h.1
    have e1 : c₁ t.unpair.2 = 1 := (hc₁ _).mp h.2
    rw [smashEnum_proper h, e0, e1, mul_one, selectFn_one, selectFn_one]
  · rw [smashEnum_master h]
    have hpf : c₀ t.unpair.1 * c₁ t.unpair.2 = 0 := by
      rcases not_and_or.mp h with h0 | h1
      · have hz : c₀ t.unpair.1 = 0 := by
          have hle := hc₀le t.unpair.1
          rcases (show c₀ t.unpair.1 = 0 ∨ c₀ t.unpair.1 = 1 by omega) with h | h
          · exact h
          · exact absurd ((hc₀ _).mpr h) h0
        rw [hz, Nat.zero_mul]
      · have hz : c₁ t.unpair.2 = 0 := by
          have hle := hc₁le t.unpair.2
          rcases (show c₁ t.unpair.2 = 0 ∨ c₁ t.unpair.2 = 1 by omega) with h | h
          · exact h
          · exact absurd ((hc₁ _).mpr h) h1
        rw [hz, Nat.mul_zero]
    rw [hpf, selectFn_zero, selectFn_zero, P₀.masterIdx_spec, P₁.masterIdx_spec]

theorem smashEnum_eq_prodNbhd_mem (k : ℕ) :
    ∃ A B, V₀.mem A ∧ V₁.mem B ∧ smashEnum P₀ P₁ k = prodNbhd A B := by
  by_cases h : P₀.X k.unpair.1 = V₀.master ∧ P₁.X k.unpair.2 = V₁.master
  · exact ⟨_, _, P₀.mem_X _, P₁.mem_X _, smashEnum_proper h⟩
  · exact ⟨_, _, V₀.master_mem, V₁.master_mem, smashEnum_master h⟩

theorem smashEnum_mem (t : ℕ) : (smash V₀ V₁).mem (smashEnum P₀ P₁ t) := by
  by_cases h : P₀.X t.unpair.1 = V₀.master ∧ P₁.X t.unpair.2 = V₁.master
  · rw [smashEnum_proper h]; exact smash_mem_proper (P₀.mem_X _) h.1 (P₁.mem_X _) h.2
  · rw [smashEnum_master h]; exact (smash V₀ V₁).master_mem

theorem smashEnum_subset_master (k : ℕ) :
    smashEnum P₀ P₁ k ⊆ prodNbhd V₀.master V₁.master := by
  have h := (smash V₀ V₁).sub_master (smashEnum_mem (P₀ := P₀) (P₁ := P₁) k)
  rwa [smash_master] at h

end Smash

end

section
variable {α β ? : Type*}
open SmashProduct

section Smash

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)
```

```
variable {P₀ P₁}

variable (P₀ P₁)

theorem proper₀_dec : RecDecidable (fun n => P₀.X n = V₀.master) := by
  have heq : RecDecidable (fun n => P₀.X n = V₀.master) := by
    refine RecDecidable.of_iff (fun n => ?_)
      (P₀.eq_computable.comp (primrec_id.pair (Nat.Primrec.const P₀.masterIdx)))
    simp only [unpair_pair_fst, unpair_pair_snd, P₀.masterIdx_spec, id_eq]
  exact heq.not

theorem proper₁_dec : RecDecidable (fun n => P₁.X n = V₁.master) := by
  have heq : RecDecidable (fun n => P₁.X n = V₁.master) := by
    refine RecDecidable.of_iff (fun n => ?_)
      (P₁.eq_computable.comp (primrec_id.pair (Nat.Primrec.const P₁.masterIdx)))
    simp only [unpair_pair_fst, unpair_pair_snd, P₁.masterIdx_spec, id_eq]
  exact heq.not

end Smash

end

end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive IndefProduct ApproximableMap SmashProduct

variable {α β ? : Type*}

section Smash
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)
variable {P₀ P₁}
variable (P₀ P₁)

noncomputable def smashPresentation : ScottPresentation (smash V₀ V₁) where
  X := smashEnum P₀ P₁
  mem_X := smashEnum_mem
  surj := by
    rintro Z (rfl | ⟨X, Y, hX, hXne, hY, hYne, rfl⟩)
    · refine ⟨Nat.pair P₀.masterIdx P₁.masterIdx, ?_⟩
      rw [smashEnum_master (by
        rw [unpair_pair_fst, P₀.masterIdx_spec]; exact fun h => h.1 rfl)]
    · obtain ⟨n, rfl⟩ := P₀.surj hX
      obtain ⟨m, rfl⟩ := P₁.surj hY
      refine ⟨Nat.pair n m, ?_⟩
      rw [smashEnum_proper (by rw [unpair_pair_fst, unpair_pair_snd]; exact ⟨hXne, hYne
    ⟩),
        unpair_pair_fst, unpair_pair_snd]
  interEq_computable := by
    obtain ⟨f₀, hf₀p, hf₀e⟩ := proper₀_dec P₀
    obtain ⟨f₁, hf₁p, hf₁e⟩ := proper₁_dec P₁
    set c₀ := fun n => isOne (f₀ n) with hc₀def
    set c₁ := fun n => isOne (f₁ n) with hc₁def
    have hc₀le : ∀ n, c₀ n ≤ 1 := fun n => isOne_le_one _
```

```
  have hc₁le : ∀ n, c₁ n ≤ 1 := fun n => isOne_le_one _
  have hc₀ : ∀ n, P₀.X n = V₀.master ↔ c₀ n = 1 := fun n => by
    rw [hc₀def, isOne_eq_one_iff]; exact hf₀e n
  have hc₁ : ∀ n, P₁.X n = V₁.master ↔ c₁ n = 1 := fun n => by
    rw [hc₁def, isOne_eq_one_iff]; exact hf₁e n
  have hc₀p : Nat.Primrec c₀ := primrec_isOne.comp hf₀p
  have hc₁p : Nat.Primrec c₁ := primrec_isOne.comp hf₁p
  have hpf : Nat.Primrec (fun n => c₀ n.unpair.1 * c₁ n.unpair.2) :=
    (primrec_mul.comp ((hc₀p.comp Nat.Primrec.left).pair (hc₁p.comp Nat.Primrec.right
))).of_eq
      (fun n => by simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd])

  have heffLp : Nat.Primrec (fun n => selectFn (c₀ n.unpair.1 * c₁ n.unpair.2)
      n.unpair.1 P₀.masterIdx) :=
    primrec_selectFn hpf Nat.Primrec.left (Nat.Primrec.const _)
  have heffRp : Nat.Primrec (fun n => selectFn (c₀ n.unpair.1 * c₁ n.unpair.2)
      n.unpair.2 P₁.masterIdx) :=
    primrec_selectFn hpf Nat.Primrec.right (Nat.Primrec.const _)

  have hgL : Nat.Primrec (fun t => Nat.pair
      (selectFn (c₀ t.unpair.1.unpair.1 * c₁ t.unpair.1.unpair.2) t.unpair.1.unpair.1
   P₀.masterIdx)
      (Nat.pair
        (selectFn (c₀ t.unpair.2.unpair.1.unpair.1 * c₁ t.unpair.2.unpair.1.unpair.2)
          t.unpair.2.unpair.1.unpair.1 P₀.masterIdx)
        (selectFn (c₀ t.unpair.2.unpair.2.unpair.1 * c₁ t.unpair.2.unpair.2.unpair.2)
          t.unpair.2.unpair.2.unpair.1 P₀.masterIdx))) :=
    (heffLp.comp Nat.Primrec.left).pair
      ((heffLp.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
        (heffLp.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hgR : Nat.Primrec (fun t => Nat.pair
      (selectFn (c₀ t.unpair.1.unpair.1 * c₁ t.unpair.1.unpair.2) t.unpair.1.unpair.2
   P₁.masterIdx)
      (Nat.pair
        (selectFn (c₀ t.unpair.2.unpair.1.unpair.1 * c₁ t.unpair.2.unpair.1.unpair.2)
          t.unpair.2.unpair.1.unpair.2 P₁.masterIdx)
        (selectFn (c₀ t.unpair.2.unpair.2.unpair.1 * c₁ t.unpair.2.unpair.2.unpair.2)
          t.unpair.2.unpair.2.unpair.2 P₁.masterIdx))) :=
    (heffRp.comp Nat.Primrec.left).pair
      ((heffRp.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
        (heffRp.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
  refine RecDecidable.of_iff (fun t => ?_)
    ((P₀.interEq_computable.comp hgL).and (P₁.interEq_computable.comp hgR))
  dsimp only
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [smashEnum_eq_eff hc₀le hc₁le hc₀ hc₁, smashEnum_eq_eff hc₀le hc₁le hc₀ hc₁,
    smashEnum_eq_eff hc₀le hc₁le hc₀ hc₁, prodNbhd.inter, prodNbhd_eq_iff]
cons_computable := by
  obtain ⟨f₀, hf₀p, hf₀e⟩ := proper₀_dec P₀
  obtain ⟨f₁, hf₁p, hf₁e⟩ := proper₁_dec P₁

  have hpa : RecDecidable (fun t => ¬(P₀.X t.unpair.1.unpair.1 = V₀.master
      ∧ P₁.X t.unpair.1.unpair.2 = V₁.master)) := by
    have h0 : RecDecidable (fun t => P₀.X t.unpair.1.unpair.1 = V₀.master) :=
      (proper₀_dec P₀).comp (Nat.Primrec.left.comp Nat.Primrec.left)
    have h1 : RecDecidable (fun t => P₁.X t.unpair.1.unpair.2 = V₁.master) :=
```

```
    (proper₁_dec P₁).comp (Nat.Primrec.right.comp Nat.Primrec.left)
  exact (h0.and h1).not
have hpb : RecDecidable (fun t => ¬(P₀.X t.unpair.2.unpair.1 = V₀.master
    ∧ P₁.X t.unpair.2.unpair.2 = V₁.master)) := by
  have h0 : RecDecidable (fun t => P₀.X t.unpair.2.unpair.1 = V₀.master) :=
    (proper₀_dec P₀).comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have h1 : RecDecidable (fun t => P₁.X t.unpair.2.unpair.2 = V₁.master) :=
    (proper₁_dec P₁).comp (Nat.Primrec.right.comp Nat.Primrec.right)
  exact (h0.and h1).not
have hcons0 : RecDecidable (fun t => ∃ k, P₀.X k ⊆ P₀.X t.unpair.1.unpair.1
    ∩ P₀.X t.unpair.2.unpair.1) := by
  refine RecDecidable.of_iff (fun t => ?_) (P₀.cons_computable.comp
    ((Nat.Primrec.left.comp Nat.Primrec.left).pair (Nat.Primrec.left.comp Nat.
Primrec.right)))
  simp only [unpair_pair_fst, unpair_pair_snd]
have hcons1 : RecDecidable (fun t => ∃ k, P₁.X k ⊆ P₁.X t.unpair.1.unpair.2
    ∩ P₁.X t.unpair.2.unpair.2) := by
  refine RecDecidable.of_iff (fun t => ?_) (P₁.cons_computable.comp
    ((Nat.Primrec.right.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
Primrec.right)))
  simp only [unpair_pair_fst, unpair_pair_snd]
refine RecDecidable.of_iff (fun t => ?_) (hpa.or (hpb.or (hcons0.and hcons1)))
constructor
· rintro ⟨k, hk⟩
  by_cases hpa' : P₀.X t.unpair.1.unpair.1 = V₀.master ∧ P₁.X t.unpair.1.unpair.2 =
 V₁.master
  · by_cases hpb' : P₀.X t.unpair.2.unpair.1 = V₀.master ∧ P₁.X t.unpair.2.unpair.2
= V₁.master
    · refine Or.inr (Or.inr ⟨?_, ?_⟩)
      · rw [smashEnum_proper hpa', smashEnum_proper hpb', prodNbhd.inter] at hk
        obtain ⟨A, B, hA, _, hkeq⟩ := smashEnum_eq_prodNbhd_mem (P₀ := P₀) (P₁ := P₁
) k
        rw [hkeq] at hk
        obtain ⟨hAsub, _⟩ := prodNbhd.subset_iff.mp hk
        obtain ⟨j₀, hj₀⟩ := P₀.surj hA
        exact ⟨j₀, by rw [hj₀]; exact hAsub⟩
      · rw [smashEnum_proper hpa', smashEnum_proper hpb', prodNbhd.inter] at hk
        obtain ⟨A, B, _, hB, hkeq⟩ := smashEnum_eq_prodNbhd_mem (P₀ := P₀) (P₁ := P₁
) k
        rw [hkeq] at hk
        obtain ⟨_, hBsub⟩ := prodNbhd.subset_iff.mp hk
        obtain ⟨j₁, hj₁⟩ := P₁.surj hB
        exact ⟨j₁, by rw [hj₁]; exact hBsub⟩
    · exact Or.inr (Or.inl hpb')
  · exact Or.inl hpa'
· intro hp
  by_cases hpa' : P₀.X t.unpair.1.unpair.1 = V₀.master ∧ P₁.X t.unpair.1.unpair.2 =
 V₁.master
  · by_cases hpb' : P₀.X t.unpair.2.unpair.1 = V₀.master ∧ P₁.X t.unpair.2.unpair.2
= V₁.master
    · rcases hp with h | h | ⟨⟨j₀, hj₀⟩, ⟨j₁, hj₁⟩⟩
      · exact absurd hpa' h
      · exact absurd hpb' h
      · have hXint : V₀.mem (P₀.X t.unpair.1.unpair.1 ∩ P₀.X t.unpair.2.unpair.1) :=
          V₀.inter_mem (P₀.mem_X _) (P₀.mem_X _) (P₀.mem_X j₀) hj₀
        have hYint : V₁.mem (P₁.X t.unpair.1.unpair.2 ∩ P₁.X t.unpair.2.unpair.2)
```

```
    :=
              V₁.inter_mem (P₁.mem_X _) (P₁.mem_X _) (P₁.mem_X j₁) hj₁
            have hXne : P₀.X t.unpair.1.unpair.1 ∩ P₀.X t.unpair.2.unpair.1 = V₀.master
     :=
              inter_ne_master_left (P₀.mem_X _) hpa'.1
            have hYne : P₁.X t.unpair.1.unpair.2 ∩ P₁.X t.unpair.2.unpair.2 = V₁.master
     :=
              inter_ne_master_left (P₁.mem_X _) hpa'.2
            obtain ⟨p₀, hp₀⟩ := P₀.surj hXint
            obtain ⟨p₁, hp₁⟩ := P₁.surj hYint
            refine ⟨Nat.pair p₀ p₁, ?_⟩
            rw [smashEnum_proper (by
                  rw [unpair_pair_fst, unpair_pair_snd, hp₀, hp₁]; exact ⟨hXne, hYne⟩),
              unpair_pair_fst, unpair_pair_snd, hp₀, hp₁, smashEnum_proper hpa',
              smashEnum_proper hpb', prodNbhd.inter]
        · refine ⟨t.unpair.1, ?_⟩
          rw [smashEnum_master (t := t.unpair.2) hpb',
            Set.inter_eq_left.mpr (smashEnum_subset_master t.unpair.1)]
      · refine ⟨t.unpair.2, ?_⟩
        rw [smashEnum_master (t := t.unpair.1) hpa',
          Set.inter_eq_right.mpr (smashEnum_subset_master t.unpair.2)]

@[simp] theorem smashPresentation_X (t : ℕ) :
    (smashPresentation P₀ P₁).X t = smashEnum P₀ P₁ t := rfl

def smashProj₀ (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    ApproximableMap (smash V₀ V₁) V₀ where
  rel W X' := (smash V₀ V₁).mem W ∧ V₀.mem X' ∧ Sum.inl ^-1' W ⊆ X'
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(smash V₀ V₁).master_mem, V₀.master_mem, by simp⟩
  inter_right := by
    rintro W X' X'' ⟨hW, hX', hsub⟩ ⟨-, hX'', hsub'⟩
    obtain ⟨A, B, hA, _, rfl⟩ := smash_mem_prodNbhd_form hW
    rw [prodNbhd.inl_preimage] at hsub hsub' ⊢
    exact ⟨hW, V₀.inter_mem hX' hX'' hA (Set.subset_inter hsub hsub'), Set.subset_inter
     hsub hsub'⟩
  mono := by
    rintro W W₂ X' X₂' ⟨_, _, hsub⟩ hW₂W hX'X₂' hW₂ hX₂'
    exact ⟨hW₂, hX₂', ((Set.preimage_mono hW₂W).trans hsub).trans hX'X₂'⟩

@[simp] theorem smashProj₀_rel {W : Set (α ? β)} {X' : Set α} :
    (smashProj₀ V₀ V₁).rel W X' ↔ (smash V₀ V₁).mem W ∧ V₀.mem X' ∧ Sum.inl ^-1' W ⊆ X'
     := Iff.rfl

def smashProj₁ (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β) :
    ApproximableMap (smash V₀ V₁) V₁ where
  rel W Y' := (smash V₀ V₁).mem W ∧ V₁.mem Y' ∧ Sum.inr ^-1' W ⊆ Y'
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(smash V₀ V₁).master_mem, V₁.master_mem, by simp⟩
  inter_right := by
    rintro W Y' Y'' ⟨hW, hY', hsub⟩ ⟨-, hY'', hsub'⟩
    obtain ⟨A, B, _, hB, rfl⟩ := smash_mem_prodNbhd_form hW
    rw [prodNbhd.inr_preimage] at hsub hsub' ⊢
    exact ⟨hW, V₁.inter_mem hY' hY'' hB (Set.subset_inter hsub hsub'), Set.subset_inter
```

```
    hsub hsub'⟩
  mono := by
    rintro W W₂ Y' Y₂' ⟨_, _, hsub⟩ hW₂W hY'Y₂' hW₂ hY₂'
    exact ⟨hW₂, hY₂', ((Set.preimage_mono hW₂W).trans hsub).trans hY'Y₂'⟩

@[simp] theorem smashProj₁_rel {W : Set (α ? β)} {Y' : Set β} :
    (smashProj₁ V₀ V₁).rel W Y' ↔ (smash V₀ V₁).mem W ∧ V₁.mem Y' ∧ Sum.inr ^-1' W ⊆ Y'
    := Iff.rfl

def smashPaired {V₂ : NeighborhoodSystem ?} (a : ApproximableMap V₂ V₀)
    (b : ApproximableMap V₂ V₁) : ApproximableMap V₂ (smash V₀ V₁) where
  rel Z W := V₂.mem Z ∧ (smash V₀ V₁).mem W ∧
    (W = prodNbhd V₀.master V₁.master ∨
      ∃ A B, W = prodNbhd A B ∧ A = V₀.master ∧ B = V₁.master ∧ a.rel Z A ∧ b.rel Z B)
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨V₂.master_mem, (smash V₀ V₁).master_mem, Or.inl rfl⟩
  inter_right := by
    rintro Z W₁ W₂ ⟨hZ, hW₁, hd₁⟩ ⟨-, hW₂, hd₂⟩
    have key : W₁ ∩ W₂ = prodNbhd V₀.master V₁.master ∨
        ∃ A B, W₁ ∩ W₂ = prodNbhd A B ∧ A = V₀.master ∧ B = V₁.master ∧
          a.rel Z A ∧ b.rel Z B := by
      rcases hd₁ with rfl | ⟨A₁, B₁, rfl, hA₁, hB₁, ha₁, hb₁⟩
      · rw [Set.inter_eq_right.mpr (show W₂ ⊆ prodNbhd V₀.master V₁.master from
          (smash V₀ V₁).sub_master hW₂)]
        exact hd₂
      · rcases hd₂ with rfl | ⟨A₂, B₂, rfl, hA₂, hB₂, ha₂, hb₂⟩
        · rw [Set.inter_eq_left.mpr (show prodNbhd A₁ B₁ ⊆ prodNbhd V₀.master V₁.master
    from
            (smash V₀ V₁).sub_master hW₁)]
          exact Or.inr ⟨A₁, B₁, rfl, hA₁, hB₁, ha₁, hb₁⟩
        · rw [prodNbhd.inter]
          exact Or.inr ⟨A₁ ∩ A₂, B₁ ∩ B₂, rfl,
            inter_ne_master_left (a.rel_cod ha₁) hA₁,
            inter_ne_master_left (b.rel_cod hb₁) hB₁,
            a.inter_right ha₁ ha₂, b.inter_right hb₁ hb₂⟩
    refine ⟨hZ, ?_, key⟩
    rcases key with hm | ⟨A, B, hWW, hA, hB, ha, hb⟩
    · rw [hm]; exact (smash V₀ V₁).master_mem
    · rw [hWW]; exact smash_mem_proper (a.rel_cod ha) hA (b.rel_cod hb) hB
  mono := by
    rintro Z Z₂ W W₂ ⟨_, _, hd⟩ hZ₂Z hWW₂ hZ₂mem hW₂mem
    refine ⟨hZ₂mem, hW₂mem, ?_⟩
    rcases hd with rfl | ⟨A, B, rfl, hA, hB, ha, hb⟩
    · left; exact Set.Subset.antisymm ((smash V₀ V₁).sub_master hW₂mem) hWW₂
    · rcases smash_mem_iff.mp hW₂mem with rfl | ⟨A₂, B₂, hA₂, hA₂ne, hB₂, hB₂ne, rfl⟩
      · left; rfl
      · obtain ⟨hAA₂, hBB₂⟩ := prodNbhd.subset_iff.mp hWW₂
        exact Or.inr ⟨A₂, B₂, rfl, hA₂ne, hB₂ne,
          a.mono ha hZ₂Z hAA₂ hZ₂mem hA₂, b.mono hb hZ₂Z hBB₂ hZ₂mem hB₂⟩

@[simp] theorem smashPaired_rel {V₂ : NeighborhoodSystem ?} {a : ApproximableMap V₂ V₀}
    {b : ApproximableMap V₂ V₁} {Z : Set ?} {W : Set (α ? β)} :
    (smashPaired a b).rel Z W ↔ V₂.mem Z ∧ (smash V₀ V₁).mem W ∧
      (W = prodNbhd V₀.master V₁.master ∨
        ∃ A B, W = prodNbhd A B ∧ A = V₀.master ∧ B = V₁.master ∧ a.rel Z A ∧ b.rel Z
```

```
    B) :=
  Iff.rfl

end Smash
section Smash

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

variable (P₀ P₁)

section SmashFunctor

variable {α' β' : Type*} {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}

def smashMap (f : ApproximableMap V₀ V₀') (g : ApproximableMap V₁ V₁') :
    ApproximableMap (smash V₀ V₁) (smash V₀' V₁') where
  rel W W' := (smash V₀ V₁).mem W ∧ (smash V₀' V₁').mem W' ∧
    (W' = prodNbhd V₀'.master V₁'.master ∨
      ∃ A B A' B', W = prodNbhd A B ∧ W' = prodNbhd A' B' ∧
        A' = V₀'.master ∧ B' = V₁'.master ∧ f.rel A A' ∧ g.rel B B')
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(smash V₀ V₁).master_mem, (smash V₀' V₁').master_mem, Or.inl rfl⟩
  inter_right := by
    rintro W W'₁ W'₂ ⟨hW, hW'₁, hd₁⟩ ⟨-, hW'₂, hd₂⟩
    have key : W'₁ ∩ W'₂ = prodNbhd V₀'.master V₁'.master ∨
        ∃ A B A' B', W = prodNbhd A B ∧ W'₁ ∩ W'₂ = prodNbhd A' B' ∧
          A' = V₀'.master ∧ B' = V₁'.master ∧ f.rel A A' ∧ g.rel B B' := by
      rcases hd₁ with rfl | ⟨A, B, A'₁, B'₁, hWAB, rfl, hA'₁, hB'₁, hf₁, hg₁⟩
      · rw [Set.inter_eq_right.mpr (show W'₂ ⊆ prodNbhd V₀'.master V₁'.master from
          (smash V₀' V₁').sub_master hW'₂)]
        exact hd₂
      · rcases hd₂ with rfl | ⟨A₂, B₂, A'₂, B'₂, hWAB₂, rfl, hA'₂, hB'₂, hf₂, hg₂⟩
        · rw [Set.inter_eq_left.mpr (show prodNbhd A'₁ B'₁ ⊆ prodNbhd V₀'.master V₁'.
    master from
            (smash V₀' V₁').sub_master hW'₁)]
          exact Or.inr ⟨A, B, A'₁, B'₁, hWAB, rfl, hA'₁, hB'₁, hf₁, hg₁⟩
        · obtain ⟨rfl, rfl⟩ := prodNbhd.injective (hWAB.symm.trans hWAB₂)

          rw [prodNbhd.inter]
          exact Or.inr ⟨A, B, A'₁ ∩ A'₂, B'₁ ∩ B'₂, hWAB, rfl,
            inter_ne_master_left (f.rel_cod hf₁) hA'₁,
            inter_ne_master_left (g.rel_cod hg₁) hB'₁,
            f.inter_right hf₁ hf₂, g.inter_right hg₁ hg₂⟩
    refine ⟨hW, ?_, key⟩
    rcases key with hm | ⟨A, B, A', B', -, hWW', hA', hB', hf, hg⟩
    · rw [hm]; exact (smash V₀' V₁').master_mem
    · rw [hWW']; exact smash_mem_proper (f.rel_cod hf) hA' (g.rel_cod hg) hB'
  mono := by
    rintro W W₂ W' W'₂ ⟨hW, hW', hd⟩ hW₂W hW'W'₂ hW₂mem hW'₂mem
    refine ⟨hW₂mem, hW'₂mem, ?_⟩
    rcases hd with rfl | ⟨A, B, A', B', rfl, rfl, hA', hB', hf, hg⟩
    · left; exact Set.Subset.antisymm ((smash V₀' V₁').sub_master hW'₂mem) hW'W'₂
```

```
    · obtain ⟨A₂, B₂, hA₂, hB₂, rfl⟩ := smash_mem_prodNbhd_form hW₂mem
      obtain ⟨hA₂A, hB₂B⟩ := prodNbhd.subset_iff.mp hW₂W
      rcases smash_mem_iff.mp hW'₂mem with rfl |
        ⟨A'₂, B'₂, hA'₂, hA'₂ne, hB'₂, hB'₂ne, rfl⟩
      · left; rfl
      · obtain ⟨hA'A'₂, hB'B'₂⟩ := prodNbhd.subset_iff.mp hW'W'₂
        exact Or.inr ⟨A₂, B₂, A'₂, B'₂, rfl, rfl, hA'₂ne, hB'₂ne,
          f.mono hf hA₂A hA'A'₂ hA₂ hA'₂, g.mono hg hB₂B hB'B'₂ hB₂ hB'₂⟩

end SmashFunctor

end Smash
section Smash

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

variable (P₀ P₁)

section SmashFunctor

variable {α' β' : Type*} {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}

theorem smashMap_isComputable (P₀' : ComputablePresentation V₀') (P₁' :
    ComputablePresentation V₁')
    {f : ApproximableMap V₀ V₀'} {g : ApproximableMap V₁ V₁'}
    (hf : ComputableMapS P₀.toScott P₀'.toScott f)
    (hg : ComputableMapS P₁.toScott P₁'.toScott g) :
    ComputableMapS (smashPresentation P₀ P₁) (smashPresentation P₀' P₁') (smashMap f g)
     := by
  constructor
  obtain ⟨f₀, hf₀p, hf₀e⟩ := proper₀_dec P₀
  obtain ⟨f₁, hf₁p, hf₁e⟩ := proper₁_dec P₁
  set c₀ := fun n => isOne (f₀ n) with hc₀def
  set c₁ := fun n => isOne (f₁ n) with hc₁def
  have hc₀le : ∀ n, c₀ n ≤ 1 := fun n => isOne_le_one _
  have hc₁le : ∀ n, c₁ n ≤ 1 := fun n => isOne_le_one _
  have hc₀ : ∀ n, P₀.X n = V₀.master ↔ c₀ n = 1 := fun n => by
    rw [hc₀def, isOne_eq_one_iff]; exact hf₀e n
  have hc₁ : ∀ n, P₁.X n = V₁.master ↔ c₁ n = 1 := fun n => by
    rw [hc₁def, isOne_eq_one_iff]; exact hf₁e n
  have hc₀p : Nat.Primrec c₀ := primrec_isOne.comp hf₀p
  have hc₁p : Nat.Primrec c₁ := primrec_isOne.comp hf₁p
  have hpf : Nat.Primrec (fun n => c₀ n.unpair.1 * c₁ n.unpair.2) :=
    (primrec_mul.comp ((hc₀p.comp Nat.Primrec.left).pair (hc₁p.comp Nat.Primrec.right))
    ).of_eq
      (fun n => by simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd])
  have heffLp : Nat.Primrec (fun n => selectFn (c₀ n.unpair.1 * c₁ n.unpair.2)
      n.unpair.1 P₀.masterIdx) :=
    primrec_selectFn hpf Nat.Primrec.left (Nat.Primrec.const _)
  have heffRp : Nat.Primrec (fun n => selectFn (c₀ n.unpair.1 * c₁ n.unpair.2)
      n.unpair.2 P₁.masterIdx) :=
    primrec_selectFn hpf Nat.Primrec.right (Nat.Primrec.const _)
```

```
  have Hf : REPred (fun t => f.rel (P₀.toScott.X
      (selectFn (c₀ t.unpair.1.unpair.1 * c₁ t.unpair.1.unpair.2)
        t.unpair.1.unpair.1 P₀.masterIdx)) (P₀'.toScott.X t.unpair.2.unpair.1)) :=
    REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hf.out.comp ((heffLp.comp Nat.Primrec.left).pair
        (Nat.Primrec.left.comp Nat.Primrec.right)))
  have Hg : REPred (fun t => g.rel (P₁.toScott.X
      (selectFn (c₀ t.unpair.1.unpair.1 * c₁ t.unpair.1.unpair.2)
        t.unpair.1.unpair.2 P₁.masterIdx)) (P₁'.toScott.X t.unpair.2.unpair.2)) :=
    REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hg.out.comp ((heffRp.comp Nat.Primrec.left).pair
        (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hJP : RecDecidable (fun t => P₀'.X t.unpair.2.unpair.1 = V₀'.master ∧
      P₁'.X t.unpair.2.unpair.2 = V₁'.master) :=
    ((proper₀_dec P₀').comp (Nat.Primrec.left.comp Nat.Primrec.right)).and
      ((proper₁_dec P₁').comp (Nat.Primrec.right.comp Nat.Primrec.right))
  refine REPred.of_iff (fun t => ?_) ((hJP.not.re).or (hJP.re.and (Hf.and Hg)))
  show (smashMap f g).rel (smashEnum P₀ P₁ t.unpair.1) (smashEnum P₀' P₁' t.unpair.2) ↔
    _
  have heq : smashEnum P₀ P₁ t.unpair.1 = prodNbhd
      (P₀.X (selectFn (c₀ t.unpair.1.unpair.1 * c₁ t.unpair.1.unpair.2)
        t.unpair.1.unpair.1 P₀.masterIdx))
      (P₁.X (selectFn (c₀ t.unpair.1.unpair.1 * c₁ t.unpair.1.unpair.2)
        t.unpair.1.unpair.2 P₁.masterIdx)) := smashEnum_eq_eff hc₀le hc₁le hc₀ hc₁ t.
    unpair.1
  by_cases hJPm : P₀'.X t.unpair.2.unpair.1 = V₀'.master ∧ P₁'.X t.unpair.2.unpair.2 =
    V₁'.master
  · constructor
    · rintro ⟨-, -, hd⟩
      rcases hd with hmaster | ⟨A, B, A', B', hWAB, hW'AB, -, -, hfr, hgr⟩
      · rw [smashEnum_proper hJPm] at hmaster
        exact absurd (prodNbhd.injective hmaster).1 hJPm.1
      · rw [smashEnum_proper hJPm] at hW'AB
        obtain ⟨rfl, rfl⟩ := prodNbhd.injective (heq.symm.trans hWAB)
        obtain ⟨rfl, rfl⟩ := prodNbhd.injective hW'AB
        exact Or.inr ⟨hJPm, hfr, hgr⟩
    · rintro (hnp | ⟨-, hfr, hgr⟩)
      · exact absurd hJPm hnp
      · exact ⟨smashEnum_mem (P₀ := P₀) (P₁ := P₁) t.unpair.1,
          smashEnum_mem (P₀ := P₀') (P₁ := P₁') t.unpair.2,
          Or.inr ⟨_, _, _, _, heq, smashEnum_proper hJPm, hJPm.1, hJPm.2, hfr, hgr⟩⟩
  · constructor
    · intro _; exact Or.inl hJPm
    · intro _
      exact ⟨smashEnum_mem (P₀ := P₀) (P₁ := P₁) t.unpair.1,
        smashEnum_mem (P₀ := P₀') (P₁ := P₁') t.unpair.2,
        Or.inl (smashEnum_master hJPm)⟩

end SmashFunctor

theorem smashPaired_isComputable {V₂ : NeighborhoodSystem ?} (P₂ :
    ComputablePresentation V₂)
    {a : ApproximableMap V₂ V₀} {b : ApproximableMap V₂ V₁}
    (ha : ComputableMapS P₂.toScott P₀.toScott a)
    (hb : ComputableMapS P₂.toScott P₁.toScott b) :
    ComputableMapS P₂.toScott (smashPresentation P₀ P₁) (smashPaired a b) := by
```

```
  constructor
  have Ha : REPred (fun t => a.rel (P₂.toScott.X t.unpair.1) (P₀.toScott.X t.unpair.2.
    unpair.1)) :=
    REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (ha.out.comp (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right)))
  have Hb : REPred (fun t => b.rel (P₂.toScott.X t.unpair.1) (P₁.toScott.X t.unpair.2.
    unpair.2)) :=
    REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hb.out.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hJP : RecDecidable (fun t => P₀.X t.unpair.2.unpair.1 = V₀.master ∧
      P₁.X t.unpair.2.unpair.2 = V₁.master) :=
    ((proper₀_dec P₀).comp (Nat.Primrec.left.comp Nat.Primrec.right)).and
      ((proper₁_dec P₁).comp (Nat.Primrec.right.comp Nat.Primrec.right))
  refine REPred.of_iff (fun t => ?_) ((hJP.not.re).or (hJP.re.and (Ha.and Hb)))
  show (smashPaired a b).rel (P₂.X t.unpair.1) (smashEnum P₀ P₁ t.unpair.2) ↔ _
  by_cases hJPm : P₀.X t.unpair.2.unpair.1 = V₀.master ∧ P₁.X t.unpair.2.unpair.2 = V₁.
    master
  · constructor
    · rintro ⟨-, -, hd⟩
      rcases hd with hmaster | ⟨A, B, hWAB, -, -, ha', hb'⟩
      · rw [smashEnum_proper hJPm] at hmaster
        exact absurd (prodNbhd.injective hmaster).1 hJPm.1
      · rw [smashEnum_proper hJPm] at hWAB
        obtain ⟨rfl, rfl⟩ := prodNbhd.injective hWAB
        exact Or.inr ⟨hJPm, ha', hb'⟩
    · rintro (hnp | ⟨-, ha', hb'⟩)
      · exact absurd hJPm hnp
      · exact ⟨P₂.mem_X _, smashEnum_mem (P₀ := P₀) (P₁ := P₁) _,
          Or.inr ⟨_, _, smashEnum_proper hJPm, hJPm.1, hJPm.2, ha', hb'⟩⟩
  · constructor
    · intro _; exact Or.inl hJPm
    · intro _
      exact ⟨P₂.mem_X _, smashEnum_mem (P₀ := P₀) (P₁ := P₁) _,
        Or.inl (smashEnum_master hJPm)⟩

theorem smashProj₀_isComputable :
    ComputableMapS (smashPresentation P₀ P₁) P₀.toScott (smashProj₀ V₀ V₁) := by
  constructor
  obtain ⟨f₀, hf₀p, hf₀e⟩ := proper₀_dec P₀
  obtain ⟨f₁, hf₁p, hf₁e⟩ := proper₁_dec P₁
  set c₀ := fun n => isOne (f₀ n) with hc₀def
  set c₁ := fun n => isOne (f₁ n) with hc₁def
  have hc₀le : ∀ n, c₀ n ≤ 1 := fun n => isOne_le_one _
  have hc₁le : ∀ n, c₁ n ≤ 1 := fun n => isOne_le_one _
  have hc₀ : ∀ n, P₀.X n = V₀.master ↔ c₀ n = 1 := fun n => by
    rw [hc₀def, isOne_eq_one_iff]; exact hf₀e n
  have hc₁ : ∀ n, P₁.X n = V₁.master ↔ c₁ n = 1 := fun n => by
    rw [hc₁def, isOne_eq_one_iff]; exact hf₁e n
  have hc₀p : Nat.Primrec c₀ := primrec_isOne.comp hf₀p
  have hc₁p : Nat.Primrec c₁ := primrec_isOne.comp hf₁p
  have hpf : Nat.Primrec (fun n => c₀ n.unpair.1 * c₁ n.unpair.2) :=
    (primrec_mul.comp ((hc₀p.comp Nat.Primrec.left).pair (hc₁p.comp Nat.Primrec.right))
    ).of_eq
      (fun n => by simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd])
  have heffLp : Nat.Primrec (fun n => selectFn (c₀ n.unpair.1 * c₁ n.unpair.2)
      n.unpair.1 P₀.masterIdx) :=
```

```
    primrec_selectFn hpf Nat.Primrec.left (Nat.Primrec.const _)
  have hincl : RecDecidable (fun s => P₀.X s.unpair.1 ⊆ P₀.X s.unpair.2) := P₀.
    incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair
      (selectFn (c₀ t.unpair.1.unpair.1 * c₁ t.unpair.1.unpair.2) t.unpair.1.unpair.1 P
    ₀.masterIdx)
      t.unpair.2) := (heffLp.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  show (smashProj₀ V₀ V₁).rel (smashEnum P₀ P₁ t.unpair.1) (P₀.X t.unpair.2) ↔ _
  simp only [smashProj₀_rel, unpair_pair_fst, unpair_pair_snd]
  constructor
  · rintro ⟨-, -, hsub⟩
    rw [smashEnum_eq_eff hc₀le hc₁le hc₀ hc₁, prodNbhd.inl_preimage] at hsub
    exact hsub
  · intro h
    refine ⟨smashEnum_mem (P₀ := P₀) (P₁ := P₁) _, P₀.mem_X _, ?_⟩
    rw [smashEnum_eq_eff hc₀le hc₁le hc₀ hc₁, prodNbhd.inl_preimage]
    exact h

theorem smashProj₁_isComputable :
    ComputableMapS (smashPresentation P₀ P₁) P₁.toScott (smashProj₁ V₀ V₁) := by
  constructor
  obtain ⟨f₀, hf₀p, hf₀e⟩ := proper₀_dec P₀
  obtain ⟨f₁, hf₁p, hf₁e⟩ := proper₁_dec P₁
  set c₀ := fun n => isOne (f₀ n) with hc₀def
  set c₁ := fun n => isOne (f₁ n) with hc₁def
  have hc₀le : ∀ n, c₀ n ≤ 1 := fun n => isOne_le_one _
  have hc₁le : ∀ n, c₁ n ≤ 1 := fun n => isOne_le_one _
  have hc₀ : ∀ n, P₀.X n = V₀.master ↔ c₀ n = 1 := fun n => by
    rw [hc₀def, isOne_eq_one_iff]; exact hf₀e n
  have hc₁ : ∀ n, P₁.X n = V₁.master ↔ c₁ n = 1 := fun n => by
    rw [hc₁def, isOne_eq_one_iff]; exact hf₁e n
  have hc₀p : Nat.Primrec c₀ := primrec_isOne.comp hf₀p
  have hc₁p : Nat.Primrec c₁ := primrec_isOne.comp hf₁p
  have hpf : Nat.Primrec (fun n => c₀ n.unpair.1 * c₁ n.unpair.2) :=
    (primrec_mul.comp ((hc₀p.comp Nat.Primrec.left).pair (hc₁p.comp Nat.Primrec.right))
    ).of_eq
      (fun n => by simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd])
  have heffRp : Nat.Primrec (fun n => selectFn (c₀ n.unpair.1 * c₁ n.unpair.2)
      n.unpair.2 P₁.masterIdx) :=
    primrec_selectFn hpf Nat.Primrec.right (Nat.Primrec.const _)
  have hincl : RecDecidable (fun s => P₁.X s.unpair.1 ⊆ P₁.X s.unpair.2) := P₁.
    incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair
      (selectFn (c₀ t.unpair.1.unpair.1 * c₁ t.unpair.1.unpair.2) t.unpair.1.unpair.2 P
    ₁.masterIdx)
      t.unpair.2) := (heffRp.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  show (smashProj₁ V₀ V₁).rel (smashEnum P₀ P₁ t.unpair.1) (P₁.X t.unpair.2) ↔ _
  simp only [smashProj₁_rel, unpair_pair_fst, unpair_pair_snd]
  constructor
  · rintro ⟨-, -, hsub⟩
    rw [smashEnum_eq_eff hc₀le hc₁le hc₀ hc₁, prodNbhd.inr_preimage] at hsub
    exact hsub
  · intro h
    refine ⟨smashEnum_mem (P₀ := P₀) (P₁ := P₁) _, P₁.mem_X _, ?_⟩
```

```
    rw [smashEnum_eq_eff hc₀le hc₁le hc₀ hc₁, prodNbhd.inr_preimage]
    exact h

theorem smash_isEffectivelyGivenS (h₀ : V₀.IsEffectivelyGiven) (h₁ : V₁.
    IsEffectivelyGiven) :
    (smash V₀ V₁).IsEffectivelyGivenS := by
  obtain ⟨Q₀⟩ := h₀; obtain ⟨Q₁⟩ := h₁
  exact ⟨smashPresentation Q₀ Q₁⟩

end Smash

end DomainTheory.NeighbourhoodSystems
```

4.8.37 Coalesced sum

The coalesced sum of effectively given neighbourhood systems is effectively given, and its injections and projections are computable.

Definition (`osum`). For systems whose neighbourhoods are nonempty, the coalesced sum has as neighbourhoods the common master and the proper neighbourhoods injected from either summand.

Theorem (`osum_master`). The master of the coalesced sum is the common sum master.

Proof. This is part of the defining structure. ■

Definition (`osumEnum`). A tagged index enumerates a proper left neighbourhood for tag zero, a proper right neighbourhood for tag one, and the common master otherwise.

Theorem (`osumEnum_left`). A zero-tagged proper left index enumerates its left injection.

Proof. The first defining alternative applies. ■

Theorem (`osumEnum_master`). If neither the left nor right tagged properness condition holds, the enumeration returns the common master.

Proof. Both proper alternatives fail. ■

Theorem (`osumEnum_right`). A one-tagged proper right index enumerates its right injection.

Proof. The left alternative is incompatible with tag one, and the right alternative applies. ■

Theorem (`osumEnum_mem`). Every enumerated set is a coalesced-sum neighbourhood.

Proof. It is either a proper injected neighbourhood or the master. ■

Theorem (`osumEnum_nonempty`). Every enumerated coalesced-sum neighbourhood is nonempty.

Proof. Injected neighbourhoods inherit nonemptiness from their summands, and the master contains the common bottom point. ■

Theorem (`osumEnum_subset_master`). Every enumerated neighbourhood is included in the common sum master.

Proof. Every neighbourhood of a neighbourhood system lies below its master. ■

Definition (`osumOutMap0`). The left projection relates a sum neighbourhood W to X when the left part of W is included in X.

Definition (`osumOutMap1`). The right projection relates a sum neighbourhood W to Y when the right part of W is included in Y.

Definition (`osumPresentation`). The coalesced-sum Scott presentation uses the tagged enumeration and decides intersections and consistency from tags, properness, and the two component presentations.

Theorem (`osumPresentation_X`). The neighbourhood numbered by an index in the coalesced-sum presentation is its tagged enumerated set.

Proof. This is the presentation's defining enumeration. ■

Theorem (`osum_eq_master_of_inj0master`). A coalesced-sum neighbourhood containing the injection of the entire left master must be the common master.

Proof. A proper left injection would force its source to equal the left master, and a right injection cannot contain a point from the nonempty left master. ■

Definition (`osumInMap0`). The left injection map relates X to W when the injection of X is included in W.

Theorem (`osum_eq_master_of_inj1master`). A coalesced-sum neighbourhood containing the injection of the entire right master must be the common master.

Proof. This is the symmetric argument using nonemptiness of the right master. ■

Definition (`osumInMap1`). The right injection map relates Y to W when the injection of Y is included in W.

Theorem (`osum_mem_subset_inj0`). A coalesced-sum neighbourhood included in a left injection is itself the injection of a proper left neighbourhood.

Proof. The master contains the common point and a right injection contains a right point, so neither can lie in a left injection. ■

Theorem (`osum_mem_subset_inj1`). A coalesced-sum neighbourhood included in a right injection is itself the injection of a proper right neighbourhood.

Proof. The symmetric exclusions leave only the proper right case. ■

Definition (`osumMap`). Two approximable maps act componentwise on coalesced sums: every source relates to the target master, while proper left and right targets are related through the corresponding map and matching source tag.

Theorem (`osumInMap0_isComputable`). The left injection map is computable.

Proof. A proper left target reduces the relation to decidable component inclusion; a proper right target is impossible, and the master case always holds. ■

Theorem (`osumInMap1_isComputable`). The right injection map is computable.

Proof. A proper right target reduces the relation to decidable component inclusion; the remaining cases are decided by their tags. ■

Theorem (`osumMap_isComputable`). The componentwise map between coalesced sums of two computable maps is computable.

Proof. The target tag decides whether the relation is automatic at the master or reduces to the recursively enumerable relation of the matching component, together with decidable source and target properness. ■

Theorem (`osumOutMap0_isComputable`). The left projection from the coalesced sum is computable.

Proof. A proper left source uses decidable inclusion of its component; right and master sources contribute the left master, whose inclusion is also decidable. ■

Theorem (`osumOutMap1_isComputable`). The right projection from the coalesced sum is computable.

Proof. This is the symmetric tag analysis for the right component. ■

Theorem (`osum_isEffectivelyGivenS`). The coalesced sum of two effectively given systems with nonempty neighbourhoods is effectively given in the Scott sense.

Proof. Choose component presentations and form the coalesced-sum presentation. ■

Theorem (`osum_mem_iff`). A set is a coalesced-sum neighbourhood exactly when it is the common master, a proper injected left neighbourhood, or a proper injected right neighbourhood.

Proof. This is the defining classification of neighbourhoods. ■

`coalescedSum.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_4
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .scottPresentation
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .smashEffective

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive IndefProduct ApproximableMap SmashProduct

variable {α β ? α' β' : Type*}

section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

def osum : NeighborhoodSystem (Option (α ? β)) where
  mem W := W = sumMaster V₀ V₁ ∨ (∃ X, V₀.mem X ∧ X = V₀.master ∧ W = inj₀ X) ∨
    (∃ Y, V₁.mem Y ∧ Y = V₁.master ∧ W = inj₁ Y)
  master := sumMaster V₀ V₁
  master_nonempty := ⟨none, none_mem_sumMaster⟩
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | ⟨X, hX, -, rfl⟩ | ⟨Y, hY, -, rfl⟩)
    · exact subset_rfl
    · exact inj₀_subset_sumMaster hX
    · exact inj₁_subset_sumMaster hY
  inter_mem := by
    have hne : ∀ W, (W = sumMaster V₀ V₁ ∨ (∃ X, V₀.mem X ∧ X = V₀.master ∧ W = inj₀ X)
     ∨
         (∃ Y, V₁.mem Y ∧ Y = V₁.master ∧ W = inj₁ Y)) → (W : Set (Option (α ? β))).
    Nonempty := by
      rintro W (rfl | ⟨X, hX, -, rfl⟩ | ⟨Y, hY, -, rfl⟩)
      · exact ⟨none, none_mem_sumMaster⟩
      · exact inj₀_nonempty (h₀ X hX)
      · exact inj₁_nonempty (h₁ Y hY)
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩
    · rcases hW' with rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩
      · rw [Set.inter_self]; exact Or.inl rfl
      · rw [sumMaster_inter_inj₀ hX']; exact Or.inr (Or.inl ⟨X', hX', hX'ne, rfl⟩)
      · rw [sumMaster_inter_inj₁ hY']; exact Or.inr (Or.inr ⟨Y', hY', hY'ne, rfl⟩)
    · rcases hW' with rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩
      · rw [Set.inter_comm, sumMaster_inter_inj₀ hX]; exact Or.inr (Or.inl ⟨X, hX, hXne,
     rfl⟩)
      · rw [inj₀_inter] at hZsub ⊢
        rcases hZ with rfl | ⟨Z₀, hZ₀, -, rfl⟩ | ⟨Z₁, hZ₁, -, rfl⟩
        · exact absurd (hZsub none_mem_sumMaster) none_mem_inj₀
        · exact Or.inr (Or.inl ⟨X ∩ X', V₀.inter_mem hX hX' hZ₀ (inj₀_subset_inj₀.mp
    hZsub),
            inter_ne_master_left hX hXne, rfl⟩)
```

```
        · obtain ⟨b, hb⟩ := h₁ Z₁ hZ₁
          exact absurd (hZsub (ir_mem_inj₁.mpr hb)) ir_mem_inj₀
      · rw [inj₀_inter_inj₁] at hZsub
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    · rcases hW' with rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩
      · rw [Set.inter_comm, sumMaster_inter_inj₁ hY]; exact Or.inr (Or.inr ⟨Y, hY, hYne,
     rfl⟩)
      · rw [Set.inter_comm, inj₀_inter_inj₁] at hZsub
        obtain ⟨t, ht⟩ := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      · rw [inj₁_inter] at hZsub ⊢
        rcases hZ with rfl | ⟨Z₀, hZ₀, -, rfl⟩ | ⟨Z₁, hZ₁, -, rfl⟩
        · exact absurd (hZsub none_mem_sumMaster) none_mem_inj₁
        · obtain ⟨a, ha⟩ := h₀ Z₀ hZ₀
          exact absurd (hZsub (il_mem_inj₀.mpr ha)) il_mem_inj₁
        · exact Or.inr (Or.inr ⟨Y ∩ Y', V₁.inter_mem hY hY' hZ₁ (inj₁_subset_inj₁.mp
    hZsub),
            inter_ne_master_right hY hYne, rfl⟩)

@[simp] theorem osum_master : (osum h₀ h₁).master = sumMaster V₀ V₁ := rfl

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

open scoped Classical in

noncomputable def osumEnum (t : ℕ) : Set (Option (α ⊕ β)) :=
  if t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master then inj₀ (P₀.X t.unpair.2)
  else if t.unpair.1 = 1 ∧ P₁.X t.unpair.2 = V₁.master then inj₁ (P₁.X t.unpair.2)
  else sumMaster V₀ V₁

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

open scoped Classical in
theorem osumEnum_left {t : ℕ} (h : t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master) :
    osumEnum P₀ P₁ t = inj₀ (P₀.X t.unpair.2) := ite_eq_left h

open scoped Classical in
theorem osumEnum_master {t : ℕ}
    (h0 : ¬(t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master))
    (h1 : ¬(t.unpair.1 = 1 ∧ P₁.X t.unpair.2 = V₁.master)) :
    osumEnum P₀ P₁ t = sumMaster V₀ V₁ := by
  unfold osumEnum; rw [ite_eq_right h0, ite_eq_right h1]
```

```
open scoped Classical in
theorem osumEnum_right {t : ℕ} (h : t.unpair.1 = 1 ∧ P₁.X t.unpair.2 = V₁.master) :
    osumEnum P₀ P₁ t = inj₁ (P₁.X t.unpair.2) := by
  have h0 : ¬(t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master) := by
    rintro ⟨hc, -⟩; obtain ⟨h1, -⟩ := h; omega
  unfold osumEnum; rw [ite_eq_right h0, ite_eq_left h]

theorem osumEnum_mem (t : ℕ) : (osum h₀ h₁).mem (osumEnum P₀ P₁ t) := by
  by_cases hL : t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master
  · rw [osumEnum_left hL]; exact Or.inr (Or.inl ⟨_, P₀.mem_X _, hL.2, rfl⟩)
  · by_cases hR : t.unpair.1 = 1 ∧ P₁.X t.unpair.2 = V₁.master
    · rw [osumEnum_right hR]; exact Or.inr (Or.inr ⟨_, P₁.mem_X _, hR.2, rfl⟩)
    · rw [osumEnum_master hL hR]; exact Or.inl rfl

include h₀ h₁ in
theorem osumEnum_nonempty (t : ℕ) : (osumEnum P₀ P₁ t).Nonempty := by
  by_cases hL : t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master
  · rw [osumEnum_left hL]; exact inj₀_nonempty (h₀ _ (P₀.mem_X _))
  · by_cases hR : t.unpair.1 = 1 ∧ P₁.X t.unpair.2 = V₁.master
    · rw [osumEnum_right hR]; exact inj₁_nonempty (h₁ _ (P₁.mem_X _))
    · rw [osumEnum_master hL hR]; exact ⟨none, none_mem_sumMaster⟩

include h₀ h₁ in
theorem osumEnum_subset_master (t : ℕ) : osumEnum P₀ P₁ t ⊆ sumMaster V₀ V₁ :=
  (osum h₀ h₁).sub_master (osumEnum_mem h₀ h₁ t)

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

variable {h₀ h₁}

def osumOutMap₀ : ApproximableMap (osum h₀ h₁) V₀ where
  rel W X := (osum h₀ h₁).mem W ∧ V₀.mem X ∧ leftPart V₀ W ⊆ X
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(osum h₀ h₁).master_mem, V₀.master_mem, (leftPart_sumMaster V₀ V₁).
    subset⟩
  inter_right := by
    rintro W X X' ⟨hW, hX, hsub⟩ ⟨-, hX', hsub'⟩
    refine ⟨hW, V₀.inter_mem hX hX' ?_ (Set.subset_inter hsub hsub'), Set.subset_inter
    hsub hsub'⟩
    rcases hW with rfl | ⟨A, hA, -, rfl⟩ | ⟨B, hB, -, rfl⟩
    · rw [leftPart_sumMaster]; exact V₀.master_mem
    · rw [leftPart_inj₀]; exact hA
    · rw [leftPart_inj₁ V₀ (h₁ B hB)]; exact V₀.master_mem
  mono := by
    rintro W W' X X' ⟨_, _, hsub⟩ hW'W hXX' hW' hX'
    exact ⟨hW', hX', (leftPart_mono V₀ hW'W).trans (hsub.trans hXX')⟩
```

```
def osumOutMap₁ : ApproximableMap (osum h₀ h₁) V₁ where
  rel W Y := (osum h₀ h₁).mem W ∧ V₁.mem Y ∧ rightPart V₁ W ⊆ Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(osum h₀ h₁).master_mem, V₁.master_mem, (rightPart_sumMaster V₀ V₁).
    subset⟩
  inter_right := by
    rintro W Y Y' ⟨hW, hY, hsub⟩ ⟨-, hY', hsub'⟩
    refine ⟨hW, V₁.inter_mem hY hY' ?_ (Set.subset_inter hsub hsub'), Set.subset_inter
    hsub hsub'⟩
    rcases hW with rfl | ⟨A, hA, -, rfl⟩ | ⟨B, hB, -, rfl⟩
    · rw [rightPart_sumMaster]; exact V₁.master_mem
    · rw [rightPart_inj₀ V₁ (h₀ A hA)]; exact V₁.master_mem
    · rw [rightPart_inj₁]; exact hB
  mono := by
    rintro W W' Y Y' ⟨_, _, hsub⟩ hW'W hYY' hW' hY'
    exact ⟨hW', hY', (rightPart_mono V₁ hW'W).trans (hsub.trans hYY')⟩

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

noncomputable def osumPresentation : ScottPresentation (osum h₀ h₁) where
  X := osumEnum P₀ P₁
  mem_X := osumEnum_mem h₀ h₁
  surj := by
    rintro W (rfl | ⟨X, hX, hXne, rfl⟩ | ⟨Y, hY, hYne, rfl⟩)
    · refine ⟨Nat.pair 2 0, ?_⟩
      rw [osumEnum_master (by rw [unpair_pair_fst]; rintro ⟨h, -⟩; omega)
        (by rw [unpair_pair_fst]; rintro ⟨h, -⟩; omega)]
    · obtain ⟨n, rfl⟩ := P₀.surj hX
      refine ⟨Nat.pair 0 n, ?_⟩
      rw [osumEnum_left (by rw [unpair_pair_fst, unpair_pair_snd]; exact ⟨rfl, hXne⟩),
        unpair_pair_snd]
    · obtain ⟨n, rfl⟩ := P₁.surj hY
      refine ⟨Nat.pair 1 n, ?_⟩
      rw [osumEnum_right (by rw [unpair_pair_fst, unpair_pair_snd]; exact ⟨rfl, hYne⟩),
        unpair_pair_snd]
  interEq_computable := by
    obtain ⟨f₀, hf₀p, hf₀e⟩ := proper₀_dec P₀
    obtain ⟨f₁, hf₁p, hf₁e⟩ := proper₁_dec P₁
    have hc₀1 : ∀ n, isOne (f₀ n) = 1 ↔ P₀.X n = V₀.master := fun n => by
      rw [isOne_eq_one_iff]; exact (hf₀e n).symm
    have hc₁1 : ∀ n, isOne (f₁ n) = 1 ↔ P₁.X n = V₁.master := fun n => by
      rw [isOne_eq_one_iff]; exact (hf₁e n).symm
    classical
    set r : ℕ → ℕ := fun t =>
      if t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master then Nat.pair 0 t.unpair.2
      else if t.unpair.1 = 1 ∧ P₁.X t.unpair.2 = V₁.master then Nat.pair 1 t.unpair.2
      else Nat.pair 2 0 with hrdef
```

```
have hr_left : ∀ t, (t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master) →
    r t = Nat.pair 0 t.unpair.2 := fun t h => by simp only [hrdef]; exact
ite_eq_left h
have hr_right : ∀ t, ¬(t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master) →
    (t.unpair.1 = 1 ∧ P₁.X t.unpair.2 = V₁.master) → r t = Nat.pair 1 t.unpair.2
:=
  fun t h0 h1 => by simp only [hrdef]; rw [ite_eq_right h0, ite_eq_left h1]
have hr_master : ∀ t, ¬(t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master) →
    ¬(t.unpair.1 = 1 ∧ P₁.X t.unpair.2 = V₁.master) → r t = Nat.pair 2 0 :=
  fun t h0 h1 => by simp only [hrdef]; rw [ite_eq_right h0, ite_eq_right h1]
have hval : ∀ t, osumEnum P₀ P₁ t = sumEnum P₀ P₁ (r t) := by
  intro t
  by_cases hL : t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master
  · rw [osumEnum_left hL, hr_left t hL, sumEnum_zero (by rw [unpair_pair_fst]),
      unpair_pair_snd]
  · by_cases hR : t.unpair.1 = 1 ∧ P₁.X t.unpair.2 = V₁.master
    · rw [osumEnum_right hR, hr_right t hL hR, sumEnum_one (by rw [unpair_pair_fst
]),
        unpair_pair_snd]
    · rw [osumEnum_master hL hR, hr_master t hL hR,
        sumEnum_master (by rw [unpair_pair_fst]; decide) (by rw [unpair_pair_fst];
 decide)]
have hbit0 : ∀ v, isOne v = if v = 1 then 1 else 0 := fun v => by
  by_cases h : v = 1
  · rw [ite_eq_left h, h]; rfl
  · rw [ite_eq_right h]; have h1 := isOne_le_one v; have h2 := isOne_eq_one_iff v;
omega
have hrp : Nat.Primrec r := by
  have hchr1 : Nat.Primrec (fun t : ℕ => (1 - t.unpair.1) * isOne (f₀ t.unpair.2))
:=
    primrec_mul₂ (primrec_sub₂ (Nat.Primrec.const 1) Nat.Primrec.left)
      (primrec_isOne.comp (hf₀p.comp Nat.Primrec.right))
  have hchr2 : Nat.Primrec (fun t : ℕ => isOne t.unpair.1 * isOne (f₁ t.unpair.2))
:=
    primrec_mul₂ (primrec_isOne.comp Nat.Primrec.left)
      (primrec_isOne.comp (hf₁p.comp Nat.Primrec.right))
  have hA : Nat.Primrec (fun t : ℕ => Nat.pair 0 t.unpair.2) :=
    (Nat.Primrec.const 0).pair Nat.Primrec.right
  have hB : Nat.Primrec (fun t : ℕ => Nat.pair 1 t.unpair.2) :=
    (Nat.Primrec.const 1).pair Nat.Primrec.right
  have hC : Nat.Primrec (fun _ : ℕ => Nat.pair 2 0) := Nat.Primrec.const _
  have hchr1eq : ∀ t : ℕ, (1 - t.unpair.1) * isOne (f₀ t.unpair.2)
      = if (t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master) then 1 else 0 := by
    intro t
    by_cases hc : t.unpair.1 = 0 ∧ P₀.X t.unpair.2 = V₀.master
    · rw [ite_eq_left hc]
      have ht0 := hc.1
      have e1 : 1 - t.unpair.1 = 1 := by omega
      rw [e1, (hc₀1 _).mpr hc.2]
    · rw [ite_eq_right hc]
      rcases not_and_or.mp hc with h | h
      · have e1 : 1 - t.unpair.1 = 0 := by omega
        rw [e1, Nat.zero_mul]
      · have e2 : isOne (f₀ t.unpair.2) = 0 := by
          have hle := isOne_le_one (f₀ t.unpair.2)
          by_contra h0; exact h ((hc₀1 _).mp (by omega))
```

```
        rw [e2, Nat.mul_zero]
    have hchr2eq : ∀ t : ℕ, isOne t.unpair.1 ⋆ isOne (f₁ t.unpair.2)
        = if (t.unpair.1 = 1 ∧ P₁.X t.unpair.2 = V₁.master) then 1 else 0 := by
      intro t
      by_cases hc : t.unpair.1 = 1 ∧ P₁.X t.unpair.2 = V₁.master
      · rw [ite_eq_left hc, (by rw [isOne_eq_one_iff]; exact hc.1 : isOne t.unpair.1
  = 1),
          (hc₁1 _).mpr hc.2]
      · rw [ite_eq_right hc]
        rcases not_and_or.mp hc with h | h
        · have e1 : isOne t.unpair.1 = 0 := by rw [hbit0]; exact ite_eq_right h
          rw [e1, Nat.zero_mul]
        · have e2 : isOne (f₁ t.unpair.2) = 0 := by
            have hle := isOne_le_one (f₁ t.unpair.2)
            by_contra h0; exact h ((hc₁1 _).mp (by omega))
          rw [e2, Nat.mul_zero]
    refine (primrec_selectFn hchr1 hA (primrec_selectFn hchr2 hB hC)).of_eq (fun t =
  > ?_)
    simp only [hchr1eq, hchr2eq, selectFn_ite, hrdef]
  have hg : Nat.Primrec (fun t => Nat.pair (r t.unpair.1)
      (Nat.pair (r t.unpair.2.unpair.1) (r t.unpair.2.unpair.2))) :=
    (hrp.comp Nat.Primrec.left).pair
      ((hrp.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
        (hrp.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
  refine RecDecidable.of_iff (fun t => ?_)
    ((sumPresentation (h₀ := h₀) (h₁ := h₁) P₀ P₁).interEq_computable.comp hg)
  have hXeq : ∀ x, (sumPresentation (h₀ := h₀) (h₁ := h₁) P₀ P₁).X x = sumEnum P₀ P₁ x
   :=
    fun _ => rfl
  simp only [unpair_pair_fst, unpair_pair_snd, hXeq]
  rw [hval t.unpair.1, hval t.unpair.2.unpair.1, hval t.unpair.2.unpair.2]
cons_computable := by
  have hLa : RecDecidable (fun t => t.unpair.1.unpair.1 = 0
      ∧ P₀.X t.unpair.1.unpair.2 = V₀.master) :=
    (RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
  0)).and
      ((proper₀_dec P₀).comp (Nat.Primrec.right.comp Nat.Primrec.left))
  have hRa : RecDecidable (fun t => t.unpair.1.unpair.1 = 1
      ∧ P₁.X t.unpair.1.unpair.2 = V₁.master) :=
    (RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
  1)).and
      ((proper₁_dec P₁).comp (Nat.Primrec.right.comp Nat.Primrec.left))
  have hLb : RecDecidable (fun t => t.unpair.2.unpair.1 = 0
      ∧ P₀.X t.unpair.2.unpair.2 = V₀.master) :=
    (RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const
   0)).and
      ((proper₀_dec P₀).comp (Nat.Primrec.right.comp Nat.Primrec.right))
  have hRb : RecDecidable (fun t => t.unpair.2.unpair.1 = 1
      ∧ P₁.X t.unpair.2.unpair.2 = V₁.master) :=
    (RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const
   1)).and
      ((proper₁_dec P₁).comp (Nat.Primrec.right.comp Nat.Primrec.right))
  have hMca : RecDecidable (fun t => ¬(t.unpair.1.unpair.1 = 0
      ∧ P₀.X t.unpair.1.unpair.2 = V₀.master)
      ∧ ¬(t.unpair.1.unpair.1 = 1 ∧ P₁.X t.unpair.1.unpair.2 = V₁.master)) := hLa.not
  .and hRa.not
```

```
have hMcb : RecDecidable (fun t => ¬(t.unpair.2.unpair.1 = 0
    ∧ P₀.X t.unpair.2.unpair.2 = V₀.master)
    ∧ ¬(t.unpair.2.unpair.1 = 1 ∧ P₁.X t.unpair.2.unpair.2 = V₁.master)) := hLb.not
.and hRb.not
have hcons0 : RecDecidable (fun t => ∃ k, P₀.X k ⊆ P₀.X t.unpair.1.unpair.2
    ∩ P₀.X t.unpair.2.unpair.2) := by
  refine RecDecidable.of_iff (fun t => ?_) (P₀.cons_computable.comp
    ((Nat.Primrec.right.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
Primrec.right)))
  simp only [unpair_pair_fst, unpair_pair_snd]
have hcons1 : RecDecidable (fun t => ∃ k, P₁.X k ⊆ P₁.X t.unpair.1.unpair.2
    ∩ P₁.X t.unpair.2.unpair.2) := by
  refine RecDecidable.of_iff (fun t => ?_) (P₁.cons_computable.comp
    ((Nat.Primrec.right.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
Primrec.right)))
  simp only [unpair_pair_fst, unpair_pair_snd]
refine RecDecidable.of_iff (fun t => ?_)
  (hMca.or (hMcb.or ((hLa.and (hLb.and hcons0)).or (hRa.and (hRb.and hcons1)))))
constructor
· rintro ⟨k, hk⟩
  by_cases hLa' : t.unpair.1.unpair.1 = 0 ∧ P₀.X t.unpair.1.unpair.2 = V₀.master
  · by_cases hLb' : t.unpair.2.unpair.1 = 0 ∧ P₀.X t.unpair.2.unpair.2 = V₀.master
    · rw [osumEnum_left hLa', osumEnum_left hLb', inj₀_inter] at hk
      refine Or.inr (Or.inr (Or.inl ⟨hLa', hLb', ?_⟩))
      rcases osumEnum_mem h₀ h₁ k with hkm | ⟨Xk, hXk, -, hkeq⟩ | ⟨Yk, hYk, -, hkeq⟩
      · exact absurd (hk (hkm ▸ none_mem_sumMaster)) none_mem_inj₀
      · rw [hkeq, inj₀_subset_inj₀] at hk
        obtain ⟨j, hj⟩ := P₀.surj hXk
        exact ⟨j, by rw [hj]; exact hk⟩
      · rw [hkeq] at hk
        obtain ⟨y0, hy0⟩ := h₁ Yk hYk
        exact absurd (hk (ir_mem_inj₁.mpr hy0)) ir_mem_inj₀
    · by_cases hRb' : t.unpair.2.unpair.1 = 1 ∧ P₁.X t.unpair.2.unpair.2 = V₁.
master
      · rw [osumEnum_left hLa', osumEnum_right hRb', inj₀_inter_inj₁] at hk
        obtain ⟨m, hm⟩ := osumEnum_nonempty h₀ h₁ k
        exact absurd (hk hm) (Set.notMem_empty m)
      · exact Or.inr (Or.inl ⟨hLb', hRb'⟩)
  · by_cases hRa' : t.unpair.1.unpair.1 = 1 ∧ P₁.X t.unpair.1.unpair.2 = V₁.master
    · by_cases hRb' : t.unpair.2.unpair.1 = 1 ∧ P₁.X t.unpair.2.unpair.2 = V₁.
master
      · rw [osumEnum_right hRa', osumEnum_right hRb', inj₁_inter] at hk
        refine Or.inr (Or.inr (Or.inr ⟨hRa', hRb', ?_⟩))
        rcases osumEnum_mem h₀ h₁ k with hkm | ⟨Xk, hXk, -, hkeq⟩ | ⟨Yk, hYk, -,
hkeq⟩
        · exact absurd (hk (hkm ▸ none_mem_sumMaster)) none_mem_inj₁
        · rw [hkeq] at hk
          obtain ⟨x0, hx0⟩ := h₀ Xk hXk
          exact absurd (hk (il_mem_inj₀.mpr hx0)) il_mem_inj₁
        · rw [hkeq, inj₁_subset_inj₁] at hk
          obtain ⟨j, hj⟩ := P₁.surj hYk
          exact ⟨j, by rw [hj]; exact hk⟩
      · by_cases hLb' : t.unpair.2.unpair.1 = 0 ∧ P₀.X t.unpair.2.unpair.2 = V₀.
master
        · rw [osumEnum_right hRa', osumEnum_left hLb', Set.inter_comm,
            inj₀_inter_inj₁] at hk
```

```
            obtain ⟨m, hm⟩ := osumEnum_nonempty h₀ h₁ k
            exact absurd (hk hm) (Set.notMem_empty m)
          · exact Or.inr (Or.inl ⟨hLb', hRb'⟩)
      · exact Or.inl ⟨hLa', hRa'⟩
  · intro hp
    rcases hp with hMca' | hMcb' | ⟨hLa', hLb', hc0⟩ | ⟨hRa', hRb', hc1⟩
    · refine ⟨t.unpair.2, ?_⟩
      rw [osumEnum_master hMca'.1 hMca'.2]
      exact Set.subset_inter (osumEnum_subset_master h₀ h₁ t.unpair.2) subset_rfl
    · refine ⟨t.unpair.1, ?_⟩
      rw [osumEnum_master hMcb'.1 hMcb'.2]
      exact Set.subset_inter subset_rfl (osumEnum_subset_master h₀ h₁ t.unpair.1)
    · obtain ⟨j, hj⟩ := hc0
      have hRHSmem : V₀.mem (P₀.X t.unpair.1.unpair.2 ∩ P₀.X t.unpair.2.unpair.2) :=
        V₀.inter_mem (P₀.mem_X _) (P₀.mem_X _) (P₀.mem_X j) hj
      have hRHSne : P₀.X t.unpair.1.unpair.2 ∩ P₀.X t.unpair.2.unpair.2 = V₀.master
  :=
        inter_ne_master_left (P₀.mem_X _) hLa'.2
      have hjne : P₀.X j = V₀.master := fun he =>
        hRHSne (Set.Subset.antisymm (V₀.sub_master hRHSmem) (he ▶ hj))
      refine ⟨Nat.pair 0 j, ?_⟩
      rw [osumEnum_left (by rw [unpair_pair_fst, unpair_pair_snd]; exact ⟨rfl, hjne⟩)
  ,
        unpair_pair_snd, osumEnum_left hLa', osumEnum_left hLb', inj₀_inter, inj₀
  _subset_inj₀]
      exact hj
    · obtain ⟨j, hj⟩ := hc1
      have hRHSmem : V₁.mem (P₁.X t.unpair.1.unpair.2 ∩ P₁.X t.unpair.2.unpair.2) :=
        V₁.inter_mem (P₁.mem_X _) (P₁.mem_X _) (P₁.mem_X j) hj
      have hRHSne : P₁.X t.unpair.1.unpair.2 ∩ P₁.X t.unpair.2.unpair.2 = V₁.master
  :=
        inter_ne_master_right (P₁.mem_X _) hRa'.2
      have hjne : P₁.X j = V₁.master := fun he =>
        hRHSne (Set.Subset.antisymm (V₁.sub_master hRHSmem) (he ▶ hj))
      refine ⟨Nat.pair 1 j, ?_⟩
      rw [osumEnum_right (by rw [unpair_pair_fst, unpair_pair_snd]; exact ⟨rfl, hjne⟩)
  ),
        unpair_pair_snd, osumEnum_right hRa', osumEnum_right hRb', inj₁_inter, inj₁
  _subset_inj₁]
      exact hj

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)


variable {P₀ P₁}

variable {h₀ h₁}

variable (P₀ P₁)

@[simp] theorem osumPresentation_X (t : ℕ) :
```

```
    (osumPresentation h₀ h₁ (P₀ := P₀) (P₁ := P₁)).X t = osumEnum P₀ P₁ t := rfl

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

variable {h₀ h₁}

theorem osum_eq_master_of_inj₀master {W : Set (Option (α ? β))} (hW : (osum h₀ h₁).mem
    W)
    (hsub : inj₀ V₀.master ⊆ W) : W = sumMaster V₀ V₁ := by
  rcases hW with rfl | ⟨X', hX', hX'ne, rfl⟩ | ⟨Y', hY', -, rfl⟩
  · rfl
  · exact absurd (Set.Subset.antisymm (V₀.sub_master hX') (inj₀_subset_inj₀.mp hsub)) hX
    'ne
  · obtain ⟨a, ha⟩ := h₀ V₀.master V₀.master_mem
    exact absurd (hsub (il_mem_inj₀.mpr ha)) il_mem_inj₁

def osumInMap₀ : ApproximableMap V₀ (osum h₀ h₁) where
  rel X W := V₀.mem X ∧ (osum h₀ h₁).mem W ∧ inj₀ X ⊆ W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨V₀.master_mem, (osum h₀ h₁).master_mem, inj₀_subset_sumMaster V₀.
    master_mem⟩
  inter_right := by
    rintro X W W' ⟨hX, hW, hsub⟩ ⟨-, hW', hsub'⟩
    refine ⟨hX, ?_, Set.subset_inter hsub hsub'⟩
    by_cases hXm : X = V₀.master
    · subst hXm
      rw [osum_eq_master_of_inj₀master hW hsub, osum_eq_master_of_inj₀master hW' hsub',
        Set.inter_self]
      exact (osum h₀ h₁).master_mem
    · exact (osum h₀ h₁).inter_mem hW hW' (Or.inr (Or.inl ⟨X, hX, hXm, rfl⟩))
        (Set.subset_inter hsub hsub')
  mono := by
    rintro X X' W W' ⟨_, _, hsub⟩ hX'X hWW' hX' hW'
    exact ⟨hX', hW', (inj₀_subset_inj₀.mpr hX'X).trans (hsub.trans hWW')⟩

theorem osum_eq_master_of_inj₁master {W : Set (Option (α ? β))} (hW : (osum h₀ h₁).mem
    W)
    (hsub : inj₁ V₁.master ⊆ W) : W = sumMaster V₀ V₁ := by
  rcases hW with rfl | ⟨X', hX', -, rfl⟩ | ⟨Y', hY', hY'ne, rfl⟩
  · rfl
  · obtain ⟨b, hb⟩ := h₁ V₁.master V₁.master_mem
    exact absurd (hsub (ir_mem_inj₁.mpr hb)) ir_mem_inj₀
  · exact absurd (Set.Subset.antisymm (V₁.sub_master hY') (inj₁_subset_inj₁.mp hsub)) hY
    'ne

def osumInMap₁ : ApproximableMap V₁ (osum h₀ h₁) where
  rel Y W := V₁.mem Y ∧ (osum h₀ h₁).mem W ∧ inj₁ Y ⊆ W
```

```
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨V₁.master_mem, (osum h₀ h₁).master_mem, inj₁_subset_sumMaster V₁.
    master_mem⟩
  inter_right := by
    rintro Y W W' ⟨hY, hW, hsub⟩ ⟨-, hW', hsub'⟩
    refine ⟨hY, ?_, Set.subset_inter hsub hsub'⟩
    by_cases hYm : Y = V₁.master
    · subst hYm
      rw [osum_eq_master_of_inj₁master hW hsub, osum_eq_master_of_inj₁master hW' hsub',
        Set.inter_self]
      exact (osum h₀ h₁).master_mem
    · exact (osum h₀ h₁).inter_mem hW hW' (Or.inr (Or.inr ⟨Y, hY, hYm, rfl⟩))
        (Set.subset_inter hsub hsub')
  mono := by
    rintro Y Y' W W' ⟨_, _, hsub⟩ hY'Y hWW' hY' hW'
    exact ⟨hY', hW', (inj₁_subset_inj₁.mpr hY'Y).trans (hsub.trans hWW')⟩

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

variable {h₀ h₁}

variable (P₀ P₁)

section CoalescedFunctor

variable {α' β' : Type*} {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
  (h₀' : ∀ X, V₀'.mem X → X.Nonempty) (h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty)

theorem osum_mem_subset_inj₀ {W : Set (Option (α ⊕ β))} {X : Set α}
    (hW : (osum h₀ h₁).mem W) (hsub : W ⊆ inj₀ X) :
    ∃ X₂, V₀.mem X₂ ∧ X₂ = V₀.master ∧ W = inj₀ X₂ := by
  rcases hW with rfl | ⟨X₂, hX₂, hne, rfl⟩ | ⟨Y₂, hY₂, hne, rfl⟩
  · exact absurd (hsub none_mem_sumMaster) none_mem_inj₀
  · exact ⟨X₂, hX₂, hne, rfl⟩
  · obtain ⟨b, hb⟩ := h₁ Y₂ hY₂; exact absurd (hsub (ir_mem_inj₁.mpr hb)) ir_mem_inj₀

end CoalescedFunctor

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}
```

```
variable {h₀ h₁}

variable (P₀ P₁)

section CoalescedFunctor

variable {α' β' : Type*} {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
  (h₀' : ∀ X, V₀'.mem X → X.Nonempty) (h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty)

theorem osum_mem_subset_inj₁ {W : Set (Option (α ? β))} {Y : Set β}
    (hW : (osum h₀ h₁).mem W) (hsub : W ⊆ inj₁ Y) :
    ∃ Y₂, V₁.mem Y₂ ∧ Y₂ = V₁.master ∧ W = inj₁ Y₂ := by
  rcases hW with rfl | ⟨X₂, hX₂, hne, rfl⟩ | ⟨Y₂, hY₂, hne, rfl⟩
  · exact absurd (hsub none_mem_sumMaster) none_mem_inj₁
  · obtain ⟨a, ha⟩ := h₀ X₂ hX₂; exact absurd (hsub (il_mem_inj₀.mpr ha)) il_mem_inj₁
  · exact ⟨Y₂, hY₂, hne, rfl⟩

end CoalescedFunctor

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

variable {h₀ h₁}

variable (P₀ P₁)

section CoalescedFunctor

variable {α' β' : Type*} {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
  (h₀' : ∀ X, V₀'.mem X → X.Nonempty) (h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty)

def osumMap (f : ApproximableMap V₀ V₀') (g : ApproximableMap V₁ V₁') :
    ApproximableMap (osum h₀ h₁) (osum h₀' h₁') where
  rel W W' := (osum h₀ h₁).mem W ∧ (osum h₀' h₁').mem W' ∧
    (W' = sumMaster V₀' V₁' ∨
      (∃ X Y', W = inj₀ X ∧ W' = inj₀ Y' ∧ Y' = V₀'.master ∧ f.rel X Y') ∨
      (∃ Y Y', W = inj₁ Y ∧ W' = inj₁ Y' ∧ Y' = V₁'.master ∧ g.rel Y Y'))
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(osum h₀ h₁).master_mem, (osum h₀' h₁').master_mem, Or.inl rfl⟩
  inter_right := by
    rintro W W'₁ W'₂ ⟨hW, hW'₁, hd₁⟩ ⟨-, hW'₂, hd₂⟩
    have hmem : ∀ W'' : Set (Option (α' ? β')),
        (W'' = sumMaster V₀' V₁' ∨
          (∃ X Y', W = inj₀ X ∧ W'' = inj₀ Y' ∧ Y' = V₀'.master ∧ f.rel X Y') ∨
          (∃ Y Y', W = inj₁ Y ∧ W'' = inj₁ Y' ∧ Y' = V₁'.master ∧ g.rel Y Y')) →
          (osum h₀' h₁').mem W'' := by
      rintro W'' (rfl | ⟨_, Y', _, rfl, hY'ne, hf⟩ | ⟨_, Y', _, rfl, hY'ne, hg⟩)
```

```
      · exact (osum h₀' h₁').master_mem
      · exact Or.inr (Or.inl ⟨Y', f.rel_cod hf, hY'ne, rfl⟩)
      · exact Or.inr (Or.inr ⟨Y', g.rel_cod hg, hY'ne, rfl⟩)
    have key : W'₁ ∩ W'₂ = sumMaster V₀' V₁' ∨
        (∃ X Y', W = inj₀ X ∧ W'₁ ∩ W'₂ = inj₀ Y' ∧ Y' = V₀'.master ∧ f.rel X Y') ∨
        (∃ Y Y', W = inj₁ Y ∧ W'₁ ∩ W'₂ = inj₁ Y' ∧ Y' = V₁'.master ∧ g.rel Y Y') := by
      rcases hd₁ with rfl | ⟨X, Y'₁, hWX₁, rfl, hY'₁ne, hf₁⟩ | ⟨Y, Y'₁, hWY₁, rfl, hY'₁ne
    , hg₁⟩
      · have he : sumMaster V₀' V₁' ∩ W'₂ = W'₂ :=
          Set.inter_eq_self_of_subset_right
            ((osum h₀' h₁').sub_master hW'₂ : W'₂ ⊆ sumMaster V₀' V₁')
        rw [he]; exact hd₂
      · rcases hd₂ with rfl | ⟨X', Y'₂, hWX₂, rfl, hY'₂ne, hf₂⟩ | ⟨Y', Y'₂, hWY₂, rfl, hY'
    ₂ne, hg₂⟩
        · rw [Set.inter_eq_left.mpr (inj₀_subset_sumMaster (f.rel_cod hf₁))]
          exact Or.inr (Or.inl ⟨X, Y'₁, hWX₁, rfl, hY'₁ne, hf₁⟩)
        · obtain rfl : X = X' := inj₀_injective (hWX₁.symm.trans hWX₂)
          rw [inj₀_inter]
          exact Or.inr (Or.inl ⟨X, Y'₁ ∩ Y'₂, hWX₁, rfl,
            inter_ne_master_left (f.rel_cod hf₁) hY'₁ne, f.inter_right hf₁ hf₂⟩)
        · exact absurd (hWX₁.symm.trans hWY₂)
            (fun h => not_inj₀_subset_inj₁ (h₀ X (f.rel_dom hf₁)) h.subset)
      · rcases hd₂ with rfl | ⟨X', Y'₂, hWX₂, rfl, hY'₂ne, hf₂⟩ | ⟨Y', Y'₂, hWY₂, rfl, hY'
    ₂ne, hg₂⟩
        · rw [Set.inter_eq_left.mpr (inj₁_subset_sumMaster (g.rel_cod hg₁))]
          exact Or.inr (Or.inr ⟨Y, Y'₁, hWY₁, rfl, hY'₁ne, hg₁⟩)
        · exact absurd (hWY₁.symm.trans hWX₂)
            (fun h => not_inj₁_subset_inj₀ (h₁ Y (g.rel_dom hg₁)) h.subset)
        · obtain rfl : Y = Y' := inj₁_injective (hWY₁.symm.trans hWY₂)
          rw [inj₁_inter]
          exact Or.inr (Or.inr ⟨Y, Y'₁ ∩ Y'₂, hWY₁, rfl,
            inter_ne_master_left (g.rel_cod hg₁) hY'₁ne, g.inter_right hg₁ hg₂⟩)
    exact ⟨hW, hmem _ key, key⟩
  mono := by
    rintro W W₂ W' W'₂ ⟨hW, hW', hd⟩ hW₂W hW'W'₂ hW₂mem hW'₂mem
    refine ⟨hW₂mem, hW'₂mem, ?_⟩
    rcases hd with rfl | ⟨X, Y', rfl, rfl, hY'ne, hf⟩ | ⟨Y, Y', rfl, rfl, hY'ne, hg⟩
    · left; exact Set.Subset.antisymm ((osum h₀' h₁').sub_master hW'₂mem) hW'W'₂
    · obtain ⟨X₂, hX₂, hX₂ne, rfl⟩ := osum_mem_subset_inj₀ hW₂mem hW₂W
      have hXX₂ : X₂ ⊆ X := inj₀_subset_inj₀.mp hW₂W
      rcases hW'₂mem with rfl | ⟨Y'₂, hY'₂, hY'₂ne, rfl⟩ | ⟨Z'₂, hZ'₂, hZ'₂ne, rfl⟩
      · left; rfl
      · exact Or.inr (Or.inl ⟨X₂, Y'₂, rfl, rfl, hY'₂ne,
          f.mono hf hXX₂ (inj₀_subset_inj₀.mp hW'W'₂) hX₂ hY'₂⟩)
      · exact (not_inj₀_subset_inj₁ (h₀' Y' (f.rel_cod hf)) hW'W'₂).elim
    · obtain ⟨Y₂, hY₂, hY₂ne, rfl⟩ := osum_mem_subset_inj₁ hW₂mem hW₂W
      have hYY₂ : Y₂ ⊆ Y := inj₁_subset_inj₁.mp hW₂W
      rcases hW'₂mem with rfl | ⟨X'₂, hX'₂, hX'₂ne, rfl⟩ | ⟨Y'₂, hY'₂, hY'₂ne, rfl⟩
      · left; rfl
      · exact (not_inj₁_subset_inj₀ (h₁' Y' (g.rel_cod hg)) hW'W'₂).elim
      · exact Or.inr (Or.inr ⟨Y₂, Y'₂, rfl, rfl, hY'₂ne,
          g.mono hg hYY₂ (inj₁_subset_inj₁.mp hW'W'₂) hY₂ hY'₂⟩)

end CoalescedFunctor

end Coalesced
```

```
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

variable {h₀ h₁}

variable (P₀ P₁)

theorem osumInMap₀_isComputable :
    ComputableMapS P₀.toScott (osumPresentation h₀ h₁ (P₀ := P₀) (P₁ := P₁)) osumInMap₀
    := by
  constructor
  have hT0 : RecDecidable (fun t => t.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
  have hT1 : RecDecidable (fun t => t.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  have hP0 : RecDecidable (fun t => P₀.X t.unpair.2.unpair.2 = V₀.master) :=
    (proper₀_dec P₀).comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hP1 : RecDecidable (fun t => P₁.X t.unpair.2.unpair.2 = V₁.master) :=
    (proper₁_dec P₁).comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hInc : RecDecidable (fun t => P₀.X t.unpair.1 ⊆ P₀.X t.unpair.2.unpair.2) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (P₀.incl_computable.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.
    Primrec.right)))
  refine (RecDecidable.of_iff (fun t => ?_)
    ((hT0.and (hP0.and hInc)).or ((hT0.and hP0).not.and (hT1.and hP1).not))).re
  show (osumInMap₀ (h₀ := h₀) (h₁ := h₁)).rel (P₀.X t.unpair.1) (osumEnum P₀ P₁ t.unpair
    .2) ↔ _
  rw [show (osumInMap₀ (h₀ := h₀) (h₁ := h₁)).rel (P₀.X t.unpair.1) (osumEnum P₀ P₁ t.
    unpair.2)
      ↔ inj₀ (P₀.X t.unpair.1) ⊆ osumEnum P₀ P₁ t.unpair.2 from
    ⟨fun h => h.2.2, fun h => ⟨P₀.mem_X _, osumEnum_mem h₀ h₁ _, h⟩⟩]
  by_cases hL : t.unpair.2.unpair.1 = 0 ∧ P₀.X t.unpair.2.unpair.2 = V₀.master
  · rw [osumEnum_left hL, inj₀_subset_inj₀]
    exact ⟨fun h => Or.inl ⟨hL.1, hL.2, h⟩,
      fun h => h.elim (fun h => h.2.2) (fun h => absurd hL h.1)⟩
  · by_cases hR : t.unpair.2.unpair.1 = 1 ∧ P₁.X t.unpair.2.unpair.2 = V₁.master
    · rw [osumEnum_right hR]
      refine ⟨fun h => ?_, fun h => h.elim (fun h => absurd ⟨h.1, h.2.1⟩ hL)
        (fun h => absurd hR h.2)⟩
      obtain ⟨a, ha⟩ := h₀ _ (P₀.mem_X t.unpair.1)
      exact absurd (h (il_mem_inj₀.mpr ha)) il_mem_inj₁
    · rw [osumEnum_master hL hR]
      exact ⟨fun _ => Or.inr ⟨hL, hR⟩, fun _ => inj₀_subset_sumMaster (P₀.mem_X _)⟩

theorem osumInMap₁_isComputable :
    ComputableMapS P₁.toScott (osumPresentation h₀ h₁ (P₀ := P₀) (P₁ := P₁)) osumInMap₁
    := by
  constructor
  have hT0 : RecDecidable (fun t => t.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
```

```
  have hT1 : RecDecidable (fun t => t.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  have hP0 : RecDecidable (fun t => P₀.X t.unpair.2.unpair.2 = V₀.master) :=
    (proper₀_dec P₀).comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hP1 : RecDecidable (fun t => P₁.X t.unpair.2.unpair.2 = V₁.master) :=
    (proper₁_dec P₁).comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hInc : RecDecidable (fun t => P₁.X t.unpair.1 ⊆ P₁.X t.unpair.2.unpair.2) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (P₁.incl_computable.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.
    Primrec.right)))
  refine (RecDecidable.of_iff (fun t => ?_)
    ((hT1.and (hP1.and hInc)).or ((hT0.and hP0).not.and (hT1.and hP1).not))).re
  show (osumInMap₁ (h₀ := h₀) (h₁ := h₁)).rel (P₁.X t.unpair.1) (osumEnum P₀ P₁ t.unpair
    .2) ↔ _
  rw [show (osumInMap₁ (h₀ := h₀) (h₁ := h₁)).rel (P₁.X t.unpair.1) (osumEnum P₀ P₁ t.
    unpair.2)
      ↔ inj₁ (P₁.X t.unpair.1) ⊆ osumEnum P₀ P₁ t.unpair.2 from
    ⟨fun h => h.2.2, fun h => ⟨P₁.mem_X _, osumEnum_mem h₀ h₁ _, h⟩⟩]
  by_cases hL : t.unpair.2.unpair.1 = 0 ∧ P₀.X t.unpair.2.unpair.2 = V₀.master
  · rw [osumEnum_left hL]
    refine ⟨fun h => ?_, fun h => ?_⟩
    · obtain ⟨b, hb⟩ := h₁ _ (P₁.mem_X t.unpair.1)
      exact absurd (h (ir_mem_inj₁.mpr hb)) ir_mem_inj₀
    · rcases h with ⟨h1, -⟩ | ⟨hn0, -⟩
      · rw [hL.1] at h1; exact absurd h1 (by decide)
      · exact absurd hL hn0
  · by_cases hR : t.unpair.2.unpair.1 = 1 ∧ P₁.X t.unpair.2.unpair.2 = V₁.master
    · rw [osumEnum_right hR, inj₁_subset_inj₁]
      exact ⟨fun h => Or.inl ⟨hR.1, hR.2, h⟩,
        fun h => h.elim (fun h => h.2.2) (fun h => absurd hR h.2)⟩
    · rw [osumEnum_master hL hR]
      exact ⟨fun _ => Or.inr ⟨hL, hR⟩, fun _ => inj₁_subset_sumMaster (P₁.mem_X _)⟩

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

variable {h₀ h₁}

variable (P₀ P₁)

section CoalescedFunctor

variable {α' β' : Type*} {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
  (h₀' : ∀ X, V₀'.mem X → X.Nonempty) (h₁' : ∀ Y, V₁'.mem Y → Y.Nonempty)

theorem osumMap_isComputable (P₀' : ComputablePresentation V₀') (P₁' :
    ComputablePresentation V₁')
    {f : ApproximableMap V₀ V₀'} {g : ApproximableMap V₁ V₁'}
    (hf : ComputableMapS P₀.toScott P₀'.toScott f)
```

```
  (hg : ComputableMapsS P₁.toScott P₁'.toScott g) :
  ComputableMapsS (osumPresentation h₀ h₁ (P₀ := P₀) (P₁ := P₁))
    (osumPresentation h₀' h₁' (P₀ := P₀') (P₁ := P₁')) (osumMap h₀' h₁' f g) := by
constructor
have mT0 : RecDecidable (fun t => t.unpair.2.unpair.1 = 0) :=
  RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
have mT1 : RecDecidable (fun t => t.unpair.2.unpair.1 = 1) :=
  RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
have mP0 : RecDecidable (fun t => P₀'.X t.unpair.2.unpair.2 = V₀'.master) :=
  (proper₀_dec P₀').comp (Nat.Primrec.right.comp Nat.Primrec.right)
have mP1 : RecDecidable (fun t => P₁'.X t.unpair.2.unpair.2 = V₁'.master) :=
  (proper₁_dec P₁').comp (Nat.Primrec.right.comp Nat.Primrec.right)
have nT0 : RecDecidable (fun t => t.unpair.1.unpair.1 = 0) :=
  RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 0)
have nT1 : RecDecidable (fun t => t.unpair.1.unpair.1 = 1) :=
  RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 1)
have nP0 : RecDecidable (fun t => P₀.X t.unpair.1.unpair.2 = V₀.master) :=
  (proper₀_dec P₀).comp (Nat.Primrec.right.comp Nat.Primrec.left)
have nP1 : RecDecidable (fun t => P₁.X t.unpair.1.unpair.2 = V₁.master) :=
  (proper₁_dec P₁).comp (Nat.Primrec.right.comp Nat.Primrec.left)
have Hf : REPred (fun t => f.rel (P₀.toScott.X t.unpair.1.unpair.2)
    (P₀'.toScott.X t.unpair.2.unpair.2)) :=
  REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
    (hf.out.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
      (Nat.Primrec.right.comp Nat.Primrec.right)))
have Hg : REPred (fun t => g.rel (P₁.toScott.X t.unpair.1.unpair.2)
    (P₁'.toScott.X t.unpair.2.unpair.2)) :=
  REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
    (hg.out.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
      (Nat.Primrec.right.comp Nat.Primrec.right)))
have hImp : RecDecidable (fun t => ¬(t.unpair.2.unpair.1 = 0 ∧ P₀'.X t.unpair.2.
  unpair.2 = V₀'.master)
    ∧ ¬(t.unpair.2.unpair.1 = 1 ∧ P₁'.X t.unpair.2.unpair.2 = V₁'.master)) :=
  (mT0.and mP0).not.and (mT1.and mP1).not
refine REPred.of_iff (fun t => ?_)
  ((hImp.re).or ((((mT0.and mP0).and (nT0.and nP0)).re.and Hf).or
    (((mT1.and mP1).and (nT1.and nP1)).re.and Hg)))
show (osumMap h₀' h₁' f g).rel (osumEnum P₀ P₁ t.unpair.1) (osumEnum P₀' P₁' t.unpair
  .2) ↔ _
by_cases hm0 : t.unpair.2.unpair.1 = 0 ∧ P₀'.X t.unpair.2.unpair.2 = V₀'.master
· rw [osumEnum_left hm0]
  constructor
  · rintro ⟨-, -, hd⟩
    rcases hd with hmaster | ⟨X, Y', hnX, hW'eq, -, hf⟩ | ⟨Y, Y', -, hW'eq, -, -⟩
    · exact absurd hmaster inj₀_ne_sumMaster
    · obtain rfl : Y' = P₀'.X t.unpair.2.unpair.2 := (inj₀_injective hW'eq).symm
      by_cases hn0 : t.unpair.1.unpair.1 = 0 ∧ P₀.X t.unpair.1.unpair.2 = V₀.master
      · rw [osumEnum_left hn0] at hnX
        obtain rfl : P₀.X t.unpair.1.unpair.2 = X := inj₀_injective hnX
        exact Or.inr (Or.inl ⟨⟨hm0, hn0⟩, hf⟩)
      · exfalso
        by_cases hn1 : t.unpair.1.unpair.1 = 1 ∧ P₁.X t.unpair.1.unpair.2 = V₁.master
        · rw [osumEnum_right hn1] at hnX
          obtain ⟨b, hb⟩ := h₁ _ (P₁.mem_X _)
          exact ir_mem_inj₀ (hnX ▸ ir_mem_inj₁.mpr hb)
        · rw [osumEnum_master hn0 hn1] at hnX
```

```
            exact none_mem_inj₀ (hnX ▶ none_mem_sumMaster)
        · obtain ⟨a, ha⟩ := h₀' _ (P₀'.mem_X _)
          exact (il_mem_inj₁ (hW'eq ▶ il_mem_inj₀.mpr ha)).elim
      · rintro (himp | ⟨⟨-, hn0⟩, hf⟩ | ⟨⟨hm1, -⟩, -⟩)
        · exact absurd hm0 himp.1
        · exact ⟨osumEnum_mem h₀ h₁ _, Or.inr (Or.inl ⟨_, P₀'.mem_X _, hm0.2, rfl⟩),
            Or.inr (Or.inl ⟨_, _, osumEnum_left hn0, rfl, hm0.2, hf⟩)⟩
        · obtain ⟨he, -⟩ := hm1; rw [hm0.1] at he; exact absurd he (by decide)
    · by_cases hm1 : t.unpair.2.unpair.1 = 1 ∧ P₁'.X t.unpair.2.unpair.2 = V₁'.master
      · rw [osumEnum_right hm1]
        constructor
        · rintro ⟨-, -, hd⟩
          rcases hd with hmaster | ⟨X, Y', -, hW'eq, -, -⟩ | ⟨Y, Y', hnY, hW'eq, -, hg⟩
          · exact absurd hmaster inj₁_ne_sumMaster
          · exact absurd hW'eq.symm (inj₀_ne_inj₁_of_nonempty (h₁' _ (P₁'.mem_X _)))
          · obtain rfl : Y' = P₁'.X t.unpair.2.unpair.2 := (inj₁_injective hW'eq).symm
            by_cases hn1 : t.unpair.1.unpair.1 = 1 ∧ P₁.X t.unpair.1.unpair.2 = V₁.master
            · rw [osumEnum_right hn1] at hnY
              obtain rfl : P₁.X t.unpair.1.unpair.2 = Y := inj₁_injective hnY
              exact Or.inr (Or.inr ⟨⟨hm1, hn1⟩, hg⟩)
            · exfalso


            by_cases hn0 : t.unpair.1.unpair.1 = 0 ∧ P₀.X t.unpair.1.unpair.2 = V₀.
      master
            · rw [osumEnum_left hn0] at hnY
              obtain ⟨a, ha⟩ := h₀ _ (P₀.mem_X _)
              exact il_mem_inj₁ (hnY ▶ il_mem_inj₀.mpr ha)
            · rw [osumEnum_master hn0 hn1] at hnY
              exact none_mem_inj₁ (hnY ▶ none_mem_sumMaster)
        · rintro (himp | ⟨⟨hm0', -⟩, -⟩ | ⟨⟨-, hn1⟩, hg⟩)
          · exact absurd hm1 himp.2
          · obtain ⟨he, -⟩ := hm0'; rw [hm1.1] at he; exact absurd he (by decide)
          · exact ⟨osumEnum_mem h₀ h₁ _, Or.inr (Or.inr ⟨_, P₁'.mem_X _, hm1.2, rfl⟩),
              Or.inr (Or.inr ⟨_, _, osumEnum_right hn1, rfl, hm1.2, hg⟩)⟩
      · rw [osumEnum_master hm0 hm1]
        constructor
        · intro _; exact Or.inl ⟨hm0, hm1⟩
        · intro _
          exact ⟨osumEnum_mem h₀ h₁ _, (osum h₀' h₁').master_mem, Or.inl rfl⟩

end CoalescedFunctor

theorem osumOutMap₀_isComputable :
    ComputableMapS (osumPresentation h₀ h₁ (P₀ := P₀) (P₁ := P₁)) P₀.toScott osumOutMap₀
     := by
  constructor
  obtain ⟨k0, hk0⟩ := P₀.surj V₀.master_mem
  have hT0 : RecDecidable (fun t => t.unpair.1.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have hP0 : RecDecidable (fun t => P₀.X t.unpair.1.unpair.2 = V₀.master) :=
    (proper₀_dec P₀).comp (Nat.Primrec.right.comp Nat.Primrec.left)
  have hIncL : RecDecidable (fun t => P₀.X t.unpair.1.unpair.2 ⊆ P₀.X t.unpair.2) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (P₀.incl_computable.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.
    Primrec.right))
  have hIncM : RecDecidable (fun t => V₀.master ⊆ P₀.X t.unpair.2) :=
```

```
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd, hk0
    ])
      (P₀.incl_computable.comp ((Nat.Primrec.const k0).pair Nat.Primrec.right))
  refine (RecDecidable.of_iff (fun t => ?_)
    ((hT0.and (hP0.and hIncL)).or ((hT0.and hP0).not.and hIncM))).re
  show (osumOutMap₀ (h₀ := h₀) (h₁ := h₁)).rel (osumEnum P₀ P₁ t.unpair.1) (P₀.X t.
    unpair.2) ↔ _
  rw [show (osumOutMap₀ (h₀ := h₀) (h₁ := h₁)).rel (osumEnum P₀ P₁ t.unpair.1) (P₀.X t.
    unpair.2)
      ↔ leftPart V₀ (osumEnum P₀ P₁ t.unpair.1) ⊆ P₀.X t.unpair.2 from
    ⟨fun h => h.2.2, fun h => ⟨osumEnum_mem h₀ h₁ _, P₀.mem_X _, h⟩⟩]
  by_cases hL : t.unpair.1.unpair.1 = 0 ∧ P₀.X t.unpair.1.unpair.2 = V₀.master
  · rw [osumEnum_left hL, leftPart_inj₀]
    exact ⟨fun h => Or.inl ⟨hL.1, hL.2, h⟩,
      fun h => h.elim (fun h => h.2.2) (fun h => absurd hL h.1)⟩
  · by_cases hR : t.unpair.1.unpair.1 = 1 ∧ P₁.X t.unpair.1.unpair.2 = V₁.master
    · rw [osumEnum_right hR, leftPart_inj₁ V₀ (h₁ _ (P₁.mem_X _))]
      exact ⟨fun h => Or.inr ⟨hL, h⟩,
        fun h => h.elim (fun h => absurd ⟨h.1, h.2.1⟩ hL) (fun h => h.2)⟩
    · rw [osumEnum_master hL hR, leftPart_sumMaster]
      exact ⟨fun h => Or.inr ⟨hL, h⟩,
        fun h => h.elim (fun h => absurd ⟨h.1, h.2.1⟩ hL) (fun h => h.2)⟩

theorem osumOutMap₁_isComputable :
    ComputableMapsS (osumPresentation h₀ h₁ (P₀ := P₀) (P₁ := P₁)) P₁.toScott osumOutMap₁
     := by
  constructor
  obtain ⟨k1, hk1⟩ := P₁.surj V₁.master_mem
  have hT1 : RecDecidable (fun t => t.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 1)
  have hP1 : RecDecidable (fun t => P₁.X t.unpair.1.unpair.2 = V₁.master) :=
    (proper₁_dec P₁).comp (Nat.Primrec.right.comp Nat.Primrec.left)
  have hIncR : RecDecidable (fun t => P₁.X t.unpair.1.unpair.2 ⊆ P₁.X t.unpair.2) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (P₁.incl_computable.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.
    Primrec.right))
  have hIncM : RecDecidable (fun t => V₁.master ⊆ P₁.X t.unpair.2) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd, hk1
    ])
      (P₁.incl_computable.comp ((Nat.Primrec.const k1).pair Nat.Primrec.right))
  refine (RecDecidable.of_iff (fun t => ?_)
    ((hT1.and (hP1.and hIncR)).or ((hT1.and hP1).not.and hIncM))).re
  show (osumOutMap₁ (h₀ := h₀) (h₁ := h₁)).rel (osumEnum P₀ P₁ t.unpair.1) (P₁.X t.
    unpair.2) ↔ _
  rw [show (osumOutMap₁ (h₀ := h₀) (h₁ := h₁)).rel (osumEnum P₀ P₁ t.unpair.1) (P₁.X t.
    unpair.2)
      ↔ rightPart V₁ (osumEnum P₀ P₁ t.unpair.1) ⊆ P₁.X t.unpair.2 from
    ⟨fun h => h.2.2, fun h => ⟨osumEnum_mem h₀ h₁ _, P₁.mem_X _, h⟩⟩]
  by_cases hR : t.unpair.1.unpair.1 = 1 ∧ P₁.X t.unpair.1.unpair.2 = V₁.master
  · rw [osumEnum_right hR, rightPart_inj₁]
    exact ⟨fun h => Or.inl ⟨hR.1, hR.2, h⟩,
      fun h => h.elim (fun h => h.2.2) (fun h => absurd hR h.1)⟩
  · by_cases hL : t.unpair.1.unpair.1 = 0 ∧ P₀.X t.unpair.1.unpair.2 = V₀.master
    · rw [osumEnum_left hL, rightPart_inj₀ V₁ (h₀ _ (P₀.mem_X _))]
      exact ⟨fun h => Or.inr ⟨hR, h⟩,
        fun h => h.elim (fun h => absurd ⟨h.1, h.2.1⟩ hR) (fun h => h.2)⟩
```

```
    · rw [osumEnum_master hL hR, rightPart_sumMaster]
      exact ⟨fun h => Or.inr ⟨hR, h⟩,
        fun h => h.elim (fun h => absurd ⟨h.1, h.2.1⟩ hR) (fun h => h.2)⟩

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

variable (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)

variable {P₀ P₁}

include h₀ h₁ in

theorem osum_isEffectivelyGivenS (e₀ : V₀.IsEffectivelyGiven) (e₁ : V₁.
    IsEffectivelyGiven) :
    (osum h₀ h₁).IsEffectivelyGivenS := by
  obtain ⟨Q₀⟩ := e₀; obtain ⟨Q₁⟩ := e₁
  exact ⟨osumPresentation h₀ h₁ (P₀ := Q₀) (P₁ := Q₁)⟩

end Coalesced
section Coalesced

variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty)

theorem osum_mem_iff {W : Set (Option (α ⊕ β))} :
    (osum h₀ h₁).mem W ↔ W = sumMaster V₀ V₁ ∨
      (∃ X, V₀.mem X ∧ X = V₀.master ∧ W = inj₀ X) ∨
      (∃ Y, V₁.mem Y ∧ Y = V₁.master ∧ W = inj₁ Y) := Iff.rfl

end Coalesced

end DomainTheory.NeighbourhoodSystems
```

4.8.38 Effective iterate

The iterate of an effectively given neighbourhood system is effectively given.

Definition (`iterIdx`). The j-th index represented by a finite code t is its j-th list entry, with the master index used beyond the list.

Definition (`iterEnum`). The neighbourhood enumerated by t in the iterate consists of pairs (j, a) for which a belongs to the neighbourhood indexed by the j-th entry of t.

Theorem (`fiber_iterEnum`). The j-th fibre of the neighbourhood enumerated by t is the neighbourhood indexed by the j-th entry of t.

Proof. This is immediate from the definition of the enumeration. ■

Theorem (`exists_list_fiber`). For every iterate neighbourhood and every length N, its first N fibres can be represented by a list of N presentation indices.

Proof. Choose an index for each successive fibre using surjectivity of the presentation and append it to the list. ■

Definition (`interG`). At a fibre number, the intersection-index function returns the chosen index for the intersection of the two fibre neighbourhoods encoded by a pair of list codes.

Theorem (`iterIdx_ge`). At every position at least the length bound represented by a code, the iterated index is the master index.

Proof. The decoded list has length at most its code, so lookup beyond that bound returns the default. ■

Theorem (`iterEnum_inter_eq_iff`). An intersection of two enumerated iterate neighbourhoods equals a third exactly when the corresponding fibre intersections agree below the sum of the three codes.

Proof. Equality implies equality of every fibre. Conversely, above the stated bound all three fibres are master neighbourhoods, so the finite family of equalities determines the whole set. ■

Theorem (`iterEnum_mem`). Every enumerated iterate set is an iterate neighbourhood.

Proof. Each fibre is a presented neighbourhood, and all fibres beyond the code are the master neighbourhood. ■

Definition (`iterInter`). The intersection code for two iterate indices tabulates their fibrewise intersection indices up to the sum of the two codes.

Theorem (`iterInter_idx_ge`). At positions at least the sum of the two input codes, the intersection code yields the master index.

Proof. Tabulation uses the master index beyond its finite bound. ■

Theorem (`iterInter_idx_lt`). Below the sum of the two input codes, the intersection code yields the chosen intersection index of the two corresponding fibres.

Proof. Finite tabulation agrees with its generating function below the bound. ■

Theorem (`iterEnum_cons_iff`). Two enumerated iterate neighbourhoods have a common numbered refinement exactly when each fibre below the sum of their codes has a common numbered refinement.

Proof. A global refinement gives fibrewise refinements. Conversely, the fibrewise intersection code is a global refinement, with master fibres beyond the finite bound. ■

Theorem (`primrec_iterIdx_proj`). If a code-valued function is primitive recursive, then looking up the position supplied by the first projection in that code is primitive recursive.

Proof. Coded-list lookup is primitive recursive and is closed under composition. ■

Theorem (`primrec_interG`). The fibrewise intersection-index function is primitive recursive.

Proof. Both coded lookups are primitive recursive, as is the presentation's intersection-index operation. ■

Definition (`iterPresentation`). The iterate presentation enumerates finite sequences of basic neighbourhoods, computes intersections fibrewise, and uses the empty code for the master neighbourhood.

Theorem (`iterPresentation_X`). The neighbourhood numbered by t in the iterate presentation is the set enumerated by t.

Proof. This is the defining enumeration of the presentation. ■

Theorem (`iterSys_isEffectivelyGiven`). The iterate of an effectively given neighbourhood system is effectively given.

Proof. Apply the iterate construction to a chosen computable presentation. ■

Theorem (`projN_isComputable`). Projection from an iterate to its n-th coordinate is a computable approximable map.

Proof. On numbered neighbourhoods its relation is inclusion from the n-th decoded fibre into the target neighbourhood; this is recursively decidable. ■

`iterEffective.lean`

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_4
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .scottPresentation
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .smashEffective

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive IndefProduct ApproximableMap SmashProduct

variable {α β ? : Type*}

section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

def iterIdx (t j : ℕ) : ℕ := nthCode t j P.masterIdx

def iterEnum (t : ℕ) : Set (ℕ × α) := {p | p.2 ∈ P.X (iterIdx P t p.1)}

end Iterate
section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

variable {P}

@[simp] theorem fiber_iterEnum (t j : ℕ) : fiber (iterEnum P t) j = P.X (iterIdx P t j
    ) := rfl

theorem exists_list_fiber {W : Set (ℕ × α)} (hW : (iterSys V).mem W) :
    ∀ N, ∃ L : List ℕ, L.length = N ∧ ∀ j, j < N → P.X (L.getD j P.masterIdx) = fiber
    W j := by
  intro N
  induction N with
  | zero => exact ⟨[], rfl, fun j hj => absurd hj (Nat.not_lt_zero j)⟩
  | succ N ih =>
    obtain ⟨L, hlen, hL⟩ := ih
    obtain ⟨n, hn⟩ := P.surj (hW.1 N)
    refine ⟨L ++ [n], by simp [List.length_append, hlen], fun j hj => ?_⟩
    by_cases h : j < N
    · rw [getD_append_cf L [n] P.masterIdx (by rw [hlen]; exact h)]
      exact hL j h
    · have hjN : j = N := by omega
      subst hjN
      rw [getD_append_right_cf L [n] P.masterIdx hlen.le, hlen, Nat.sub_self,
        List.getD_cons_zero]
      exact hn

def interG (P : ComputablePresentation V) (s : ℕ) : ℕ :=
```

```
  P.inter (iterIdx P s.unpair.2.unpair.1 s.unpair.1) (iterIdx P s.unpair.2.unpair.2 s.
    unpair.1)

theorem iterIdx_ge {t j : ℕ} (h : t ≤ j) : iterIdx P t j = P.masterIdx := by
  unfold iterIdx
  rw [nthCode_eq, getD_eq_default_cf]
  exact le_trans (decodeList_length_le t) h

theorem iterEnum_inter_eq_iff {n m k : ℕ} :
    iterEnum P n ∩ iterEnum P m = iterEnum P k ↔
      ∀ j, j < n + m + k →
        P.X (iterIdx P n j) ∩ P.X (iterIdx P m j) = P.X (iterIdx P k j) := by
  constructor
  · intro h j _
    have := congrArg (fun W => fiber W j) h
    simpa only [fiber_inter, fiber_iterEnum] using this
  · intro h
    apply eq_of_fiber_eq
    intro j
    rw [fiber_inter, fiber_iterEnum, fiber_iterEnum, fiber_iterEnum]
    by_cases hj : j < n + m + k
    · exact h j hj
    · rw [iterIdx_ge (show n ≤ j by omega), iterIdx_ge (show m ≤ j by omega),
        iterIdx_ge (show k ≤ j by omega), P.masterIdx_spec, Set.inter_self]

theorem iterEnum_mem (t : ℕ) : (iterSys V).mem (iterEnum P t) := by
  refine ⟨fun j => ?_, t, fun j hj => ?_⟩
  · rw [fiber_iterEnum]; exact P.mem_X _
  · rw [fiber_iterEnum, iterIdx_ge hj, P.masterIdx_spec]

def iterInter (P : ComputablePresentation V) (n m : ℕ) : ℕ :=
  tabCode (interG P) (Nat.pair (n + m) (Nat.pair n m)) (n + m)

theorem iterInter_idx_ge {n m j : ℕ} (hj : n + m ≤ j) :
    iterIdx P (iterInter P n m) j = P.masterIdx := by
  show nthCode (iterInter P n m) j P.masterIdx = _
  rw [iterInter, tabCode_nth_ge hj]

theorem iterInter_idx_lt {n m j : ℕ} (hj : j < n + m) :
    iterIdx P (iterInter P n m) j = P.inter (iterIdx P n j) (iterIdx P m j) := by
  show nthCode (iterInter P n m) j P.masterIdx = _
  rw [iterInter, tabCode_nth_lt hj]
  unfold interG
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem iterEnum_cons_iff {n m : ℕ} :
    (∃ k, iterEnum P k ⊆ iterEnum P n ∩ iterEnum P m) ↔
      ∀ j, j < n + m → ∃ i, P.X i ⊆ P.X (iterIdx P n j) ∩ P.X (iterIdx P m j) := by
  constructor
  · rintro ⟨k, hk⟩ j _
    refine ⟨iterIdx P k j, ?_⟩
    have := fiber_mono hk j
    rwa [fiber_inter, fiber_iterEnum, fiber_iterEnum, fiber_iterEnum] at this
  · intro h
    refine ⟨iterInter P n m, ?_⟩
    apply subset_of_fiber_subset
```

```
    intro j
    rw [fiber_inter, fiber_iterEnum, fiber_iterEnum, fiber_iterEnum]
    by_cases hj : j < n + m
    · rw [iterInter_idx_lt hj]
      obtain ⟨i, hi⟩ := h j hj
      rw [P.inter_spec ⟨i, hi⟩]
    · rw [iterInter_idx_ge (by omega), iterIdx_ge (show n ≤ j by omega),
        iterIdx_ge (show m ≤ j by omega), P.masterIdx_spec, Set.inter_self]

end Iterate
section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

variable {P}

theorem primrec_iterIdx_proj {codeProj : ℕ → ℕ} (hc : Nat.Primrec codeProj) :
    Nat.Primrec (fun w => iterIdx P (codeProj w) w.unpair.1) := by
  refine (primrec_nthCode.comp (hc.pair (Nat.Primrec.left.pair
    (Nat.Primrec.const P.masterIdx)))).of_eq (fun w => ?_)
  simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]

theorem primrec_interG : Nat.Primrec (interG P) := by
  have hA : Nat.Primrec (fun s => iterIdx P s.unpair.2.unpair.1 s.unpair.1) :=
    primrec_iterIdx_proj (Nat.Primrec.left.comp Nat.Primrec.right)
  have hB : Nat.Primrec (fun s => iterIdx P s.unpair.2.unpair.2 s.unpair.1) :=
    primrec_iterIdx_proj (Nat.Primrec.right.comp Nat.Primrec.right)
  refine (P.inter_primrec.comp (hA.pair hB)).of_eq (fun s => ?_)
  unfold interG
  simp only [unpair_pair_fst, unpair_pair_snd]

noncomputable def iterPresentation (P : ComputablePresentation V) :
    ComputablePresentation (iterSys V) where
  X := iterEnum P
  mem_X := iterEnum_mem
  surj := by
    rintro W hW
    obtain ⟨N, hN⟩ := hW.2
    obtain ⟨L, hlen, hL⟩ := exists_list_fiber (P := P) hW N
    refine ⟨encodeList L, ?_⟩
    apply eq_of_fiber_eq
    intro j
    show P.X (nthCode (encodeList L) j P.masterIdx) = fiber W j
    rw [nthCode_eq, decodeList_encodeList]
    by_cases hj : j < N
    · exact hL j hj
    · rw [getD_eq_default_cf L P.masterIdx (by rw [hlen]; omega), P.masterIdx_spec,
        hN j (by omega)]
  interEq_computable := by
    have hbound : Nat.Primrec
        (fun t : ℕ => t.unpair.1 + t.unpair.2.unpair.1 + t.unpair.2.unpair.2) :=
      primrec_add₂ (primrec_add₂ Nat.Primrec.left (Nat.Primrec.left.comp Nat.Primrec.
    right))
        (Nat.Primrec.right.comp Nat.Primrec.right)
```

```
  have hA : Nat.Primrec (fun w => iterIdx P w.unpair.2.unpair.1 w.unpair.1) :=
    primrec_iterIdx_proj (Nat.Primrec.left.comp Nat.Primrec.right)
  have hB : Nat.Primrec (fun w => iterIdx P w.unpair.2.unpair.2.unpair.1 w.unpair.1)
   :=
    primrec_iterIdx_proj (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.
  right))
  have hC : Nat.Primrec (fun w => iterIdx P w.unpair.2.unpair.2.unpair.2 w.unpair.1)
   :=
    primrec_iterIdx_proj (Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec
  .right))
  have hq : RecDecidable (fun w => P.X (iterIdx P w.unpair.2.unpair.1 w.unpair.1)
      ∩ P.X (iterIdx P w.unpair.2.unpair.2.unpair.1 w.unpair.1)
      = P.X (iterIdx P w.unpair.2.unpair.2.unpair.2 w.unpair.1)) := by
    refine RecDecidable.of_iff (fun w => ?_) (P.interEq_computable.comp (hA.pair (hB
  .pair hC)))
    simp only [unpair_pair_fst, unpair_pair_snd]
  refine RecDecidable.of_iff (fun t => ?_) (hq.bForall hbound)
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact iterEnum_inter_eq_iff
cons_computable := by
  have hbound : Nat.Primrec (fun t : ℕ => t.unpair.1 + t.unpair.2) :=
    primrec_add₂ Nat.Primrec.left Nat.Primrec.right
  have hA : Nat.Primrec (fun w => iterIdx P w.unpair.2.unpair.1 w.unpair.1) :=
    primrec_iterIdx_proj (Nat.Primrec.left.comp Nat.Primrec.right)
  have hB : Nat.Primrec (fun w => iterIdx P w.unpair.2.unpair.2 w.unpair.1) :=
    primrec_iterIdx_proj (Nat.Primrec.right.comp Nat.Primrec.right)
  have hq : RecDecidable (fun w => ∃ i, P.X i ⊆ P.X (iterIdx P w.unpair.2.unpair.1 w
  .unpair.1)
      ∩ P.X (iterIdx P w.unpair.2.unpair.2 w.unpair.1)) := by
    refine RecDecidable.of_iff (fun w => ?_) (P.cons_computable.comp (hA.pair hB))
    simp only [unpair_pair_fst, unpair_pair_snd]
  refine RecDecidable.of_iff (fun t => ?_) (hq.bForall hbound)
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact iterEnum_cons_iff
inter := iterInter P
inter_primrec := by
  have hr : Nat.Primrec (fun t => Nat.pair
      (Nat.pair (t.unpair.1 + t.unpair.2) (Nat.pair t.unpair.1 t.unpair.2))
      (t.unpair.1 + t.unpair.2)) :=
    ((primrec_add₂ Nat.Primrec.left Nat.Primrec.right).pair
      (Nat.Primrec.left.pair Nat.Primrec.right)).pair
      (primrec_add₂ Nat.Primrec.left Nat.Primrec.right)
  refine ((primrec_tabCode (primrec_interG (P := P))).comp hr).of_eq (fun t => ?_)
  show tabCode (interG P) _ _ = iterInter P t.unpair.1 t.unpair.2
  unfold iterInter
  simp only [unpair_pair_fst, unpair_pair_snd]
inter_spec := by
  intro n m hcons
  apply eq_of_fiber_eq
  intro j
  rw [fiber_inter, fiber_iterEnum, fiber_iterEnum, fiber_iterEnum]
  by_cases hj : j < n + m
  · rw [iterInter_idx_lt hj, P.inter_spec ((iterEnum_cons_iff (P := P)).mp hcons j hj
  )]
  · rw [iterInter_idx_ge (by omega), iterIdx_ge (show n ≤ j by omega),
      iterIdx_ge (show m ≤ j by omega), P.masterIdx_spec, Set.inter_self]
```

```
  masterIdx := 0
  masterIdx_spec := by
    apply eq_of_fiber_eq
    intro j
    rw [fiber_iterEnum, fiber_iterSys_master, iterIdx_ge (Nat.zero_le j), P.
    masterIdx_spec]

end Iterate
section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

variable {P}

@[simp] theorem iterPresentation_X (t : ℕ) : (iterPresentation P).X t = iterEnum P t
    := rfl

end Iterate
section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

variable {P}

theorem iterSys_isEffectivelyGiven (e : V.IsEffectivelyGiven) :
    (iterSys V).IsEffectivelyGiven := by
  obtain ⟨Q⟩ := e
  exact ⟨iterPresentation (P := Q)⟩

end Iterate
section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem α} (P : ComputablePresentation V)

variable {P}

theorem projN_isComputable (P : ComputablePresentation V) (n : ℕ) :
    ComputableMap (iterPresentation P) P (projN V n) := by
  constructor
  have hincl : RecDecidable (fun s => P.X s.unpair.1 ⊆ P.X s.unpair.2) := P.
    incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair (nthCode t.unpair.1 n P.masterIdx) t.unpair
    .2) :=
    ((primrec_nthCode.comp (Nat.Primrec.left.pair
      ((Nat.Primrec.const n).pair (Nat.Primrec.const P.masterIdx)))).of_eq
        (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])).pair Nat.Primrec.
    right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [iterPresentation_X, projN_rel, fiber_iterEnum, iterIdx, unpair_pair_fst,
    unpair_pair_snd]
```

```
  exact ⟨fun h => h.2.2, fun h => ⟨iterEnum_mem _, P.mem_X _, h⟩⟩

end Iterate

end DomainTheory.NeighbourhoodSystems
```

4.8.39 Exercise 7.15 — further constructs effective

Finish the proof of 7.4 and establish similar results for the constructs $(D_0 \otimes D_1)$, $(D_0 \oplus D_1)$ and D^∞. Take into account the various appropriate combinators.

The development is with the Scott presentation, effective smash, coalesced sum, and effective iterate peers.

Theorem (`example_715`). The iterate, the coalesced sum, and the smash product of effectively given neighbourhood systems are effectively given.

Proof. By the effective iterate, the coalesced sum, and the effective smash. ■

`Exercise_7_15.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .iterEffective
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .coalescedSum
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .smashEffective

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem Domain.Recursive ApproximableMap DomainTheory.
    NeighbourhoodSystems.SmashProduct

/-- Scott's Exercise 7.15: effective givenness for iterate, coalesced sum, and smash.
    -/
theorem example_715 :
    (∀ {α : Type*} {V : NeighborhoodSystem α},
      V.IsEffectivelyGiven → (iterSys V).IsEffectivelyGiven) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      (h₀ : ∀ X, V₀.mem X → X.Nonempty) (h₁ : ∀ Y, V₁.mem Y → Y.Nonempty),
      V₀.IsEffectivelyGiven → V₁.IsEffectivelyGiven → (osum h₀ h₁).IsEffectivelyGivenS
    ) ∧
    (∀ {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β},
      V₀.IsEffectivelyGiven → V₁.IsEffectivelyGiven → (smash V₀ V₁).
    IsEffectivelyGivenS) :=
  ⟨@iterSys_isEffectivelyGiven, ⟨@osum_isEffectivelyGivenS, @smash_isEffectivelyGivenS⟩⟩
```

4.8.40 Curry enumeration

Currying is a computable map of function spaces, and its neighbourhood relation is decidable.

Theorem (`curryComb_rel_recDecidable`). For compatible effective presentations of the uncurried and curried function spaces, the relation of the currying combinator on numbered neighbourhoods is recursively decidable.

Proof. Membership in a curried function neighbourhood reduces to finitely many guarded inclusion tests between numbered uncurried function neighbourhoods. The guards and finite-list quantifiers are decidable, and all index transformations are primitive recursive. ■

Theorem (`curryComb_isComputable`). Given computable presentations of three systems, there are computable presentations of the corresponding uncurried and curried function spaces for which currying is computable.

Proof. Construct the function-space presentations from recursive consistency and inclusion witnesses. The preceding decision theorem supplies the recursively enumerable relation required for currying. ■

Theorem (`toApproxMap_principal_mem`). The approximable map represented by the principal element of a function-space neighbourhood belongs to that neighbourhood.

Proof. A function-space neighbourhood is generated by finitely many steps. The represented map satisfies every generating step by the defining universal property of the principal element. ■

Theorem (`curryComb_rel`). The currying combinator relates function-space neighbourhoods G and H exactly when both are neighbourhoods and every map in G has its curry in H.

Proof. The principal map of G lies in G, and every other member lies above it. Monotonicity of currying and upward closure of H convert the principal-map condition into the stated universal condition. ■

`curryEnum.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Combinators

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
  {V₂ : NeighborhoodSystem ?}

theorem curryComb_rel_recDecidable
    (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)
    (P₂ : ComputablePresentation V₂) (gNa gNb gNc : ℕ → ℕ)
    (PA : ComputablePresentation (funSpace (prod V₀ V₁) V₂))
    (PB : ComputablePresentation (funSpace V₀ (funSpace V₁ V₂)))
    (Pc : ComputablePresentation (funSpace V₁ V₂))
    (hPAX : PA.X = Xenum (prodPresentation P₀ P₁) P₂ gNa)
    (hPBX : PB.X = Xenum P₀ Pc gNb)
    (hPcX : Pc.X = Xenum P₁ P₂ gNc)
    (hgNa : ∀ c, gNa c = 1 ↔
      (stepFun (funListOf (prodPresentation P₀ P₁) P₂ (decodeList c))
```

```
      : Set (ApproximableMap (prod V₀ V₁) V₂)).Nonempty)
  (hgNc : ∀ c, gNc c = 1 ↔
    (stepFun (funListOf P₁ P₂ (decodeList c)) : Set (ApproximableMap V₁ V₂)).Nonempty
  )
  (hgNbp : Nat.Primrec gNb) (hgNcp : Nat.Primrec gNc) :
  RecDecidable₂ (fun n m => (curryC V₀ V₁ V₂).rel (PA.X n) (PB.X m)) := by

have hcomb : ∀ n m, (curryC V₀ V₁ V₂).rel (PA.X n) (PB.X m) ↔
    (gNb m = 1 → ∀ e ∈ decodeList m, gNc e.unpair.2 = 1 →
      ∀ e' ∈ decodeList e.unpair.2, PA.X n ⊆ PA.X (curryStepCode e.unpair.1 e')) :=
  by
  intro n m
  have hstep1 : (curryC V₀ V₁ V₂).rel (PA.X n) (PB.X m) ↔
      curry (toApproxMap ((funSpace (prod V₀ V₁) V₂).principal (PA.mem_X n))) ∈ PB.X
  m := by
    constructor
    · rintro ⟨hG, hH⟩
      rw [curryIso_apply, mem_toFilter] at hH
      exact hH.2
    · intro h
      exact ⟨PA.mem_X n, by rw [curryIso_apply, mem_toFilter]; exact ⟨PB.mem_X m, h⟩⟩
  rw [hstep1, hPBX, mem_Xenum_iff_map P₀ Pc gNb
    (curry (toApproxMap ((funSpace (prod V₀ V₁) V₂).principal (PA.mem_X n)))) m]
  apply imp_congr_right; intro _
  apply forall_congr'; intro e
  apply imp_congr_right; intro _
  rw [hPcX, curry_rel_Xenum_iff P₀ P₁ P₂ gNc hgNc
    (toApproxMap ((funSpace (prod V₀ V₁) V₂).principal (PA.mem_X n))) e.unpair.1 e.
  unpair.2]
  apply imp_congr_right; intro _
  apply forall_congr'; intro e'
  apply imp_congr_right; intro _
  have hstepeq : step (prodNbhd (P₀.X e.unpair.1) (P₁.X e'.unpair.1)) (P₂.X e'.unpair
  .2)
      = PA.X (curryStepCode e.unpair.1 e') := by
    have hs := Xenum_singleton (prodPresentation P₀ P₁) P₂ gNa hgNa
      (Nat.pair e.unpair.1 e'.unpair.1) e'.unpair.2
    rw [prodPresentation_X, unpair_pair_fst, unpair_pair_snd] at hs
    rw [hPAX]; exact hs.symm
  rw [toApproxMap_rel, mem_principal, hstepeq]
  exact ⟨fun h => h.2, fun h => ⟨PA.mem_X _, h⟩⟩

have q1_dec : RecDecidable₂ (fun e' p =>
    PA.X p.unpair.1 ⊆ PA.X (curryStepCode p.unpair.2 e')) := by
  have hr : Nat.Primrec (fun w => Nat.pair w.unpair.2.unpair.1
      (curryStepCode w.unpair.2.unpair.2 w.unpair.1)) :=
    (Nat.Primrec.left.comp Nat.Primrec.right).pair
      (primrec_curryStepCode (Nat.Primrec.right.comp Nat.Primrec.right) Nat.Primrec.
  left)
  refine RecDecidable.of_iff (fun w => ?_) (RecDecidable.comp PA.incl_computable hr)
  simp only [unpair_pair_fst, unpair_pair_snd]
have dInner := q1_dec.bForallList
have dMid : RecDecidable₂ (fun e n =>
    gNc e.unpair.2 = 1 → ∀ e' ∈ decodeList e.unpair.2,
      PA.X n ⊆ PA.X (curryStepCode e.unpair.1 e')) := by
  have hg_mid : Nat.Primrec (fun s => Nat.pair s.unpair.1.unpair.2
```

```
        (Nat.pair s.unpair.2 s.unpair.1.unpair.1)) :=
      (Nat.Primrec.right.comp Nat.Primrec.left).pair
        (Nat.Primrec.right.pair (Nat.Primrec.left.comp Nat.Primrec.left))
    have B1 : RecDecidable (fun s => ∀ e' ∈ decodeList s.unpair.1.unpair.2,
       PA.X s.unpair.2 ⊆ PA.X (curryStepCode s.unpair.1.unpair.1 e')) := by
      refine RecDecidable.of_iff (fun s => ?_) (RecDecidable.comp dInner hg_mid)
      simp only [unpair_pair_fst, unpair_pair_snd]
    have guard1 : RecDecidable (fun s => gNc s.unpair.1.unpair.2 = 1) :=
      RecDecidable.natEq (hgNcp.comp (Nat.Primrec.right.comp Nat.Primrec.left))
        (Nat.Primrec.const 1)
    refine RecDecidable.of_iff (fun s => ?_) (guard1.not.or B1)
    exact Decidable.imp_iff_not_or
  have dOuter := dMid.bForallList
  refine RecDecidable.of_iff (fun t => hcomb t.unpair.1 t.unpair.2) ?_
  have hswap2 : Nat.Primrec (fun t => Nat.pair t.unpair.2 t.unpair.1) :=
    Nat.Primrec.right.pair Nat.Primrec.left
  have B2 : RecDecidable (fun t => ∀ e ∈ decodeList t.unpair.2, gNc e.unpair.2 = 1 →
      ∀ e' ∈ decodeList e.unpair.2, PA.X t.unpair.1 ⊆ PA.X (curryStepCode e.unpair.1 e
    ')) := by
    refine RecDecidable.of_iff (fun t => ?_) (RecDecidable.comp dOuter hswap2)
    simp only [unpair_pair_fst, unpair_pair_snd]
  have guard2 : RecDecidable (fun t => gNb t.unpair.2 = 1) :=
    RecDecidable.natEq (hgNbp.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  refine RecDecidable.of_iff (fun t => ?_) (guard2.not.or B2)
  exact Decidable.imp_iff_not_or

theorem curryComb_isComputable
    (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)
    (P₂ : ComputablePresentation V₂) :
    ∃ (PA : ComputablePresentation (funSpace (prod V₀ V₁) V₂))
      (PB : ComputablePresentation (funSpace V₀ (funSpace V₁ V₂))),
      RecDecidable₂ (fun n m => (curryC V₀ V₁ V₂).rel (PA.X n) (PB.X m)) ∧
        ComputableMap PA PB (curryC V₀ V₁ V₂) := by
  obtain ⟨incl0, hincl0p, hincl0s⟩ := P₀.incl_computable
  obtain ⟨fc0, hfc0p, hfc0s⟩ := P₀.cons_computable
  obtain ⟨incl1, hincl1p, hincl1s⟩ := P₁.incl_computable
  obtain ⟨fc1, hfc1p, hfc1s⟩ := P₁.cons_computable
  obtain ⟨incl2, hincl2p, hincl2s⟩ := P₂.incl_computable
  obtain ⟨eq2, heq2p, heq2s⟩ := P₂.eq_computable
  obtain ⟨fc2, hfc2p, hfc2s⟩ := P₂.cons_computable
  obtain ⟨inclpr, hinclprp, hinclprs⟩ := (prodPresentation P₀ P₁).incl_computable
  obtain ⟨fcpr, hfcprp, hfcprs⟩ := (prodPresentation P₀ P₁).cons_computable

  let Pc : ComputablePresentation (funSpace V₁ V₂) :=
    funPresentation P₁ P₂ (funConsChar P₁ P₂ fc1 fc2) incl1 incl2 eq2
      (funConsChar_spec P₁ P₂ fc1 fc2 (fun s => (hfc1s s).symm) (fun s => (hfc2s s).
    symm))
      (primrec_funConsChar P₁ P₂ fc1 fc2 hfc1p hfc2p)
      (fun s => (hincl1s s).symm) hincl1p (fun s => (hincl2s s).symm) hincl2p
      (fun s => (heq2s s).symm) heq2p
  obtain ⟨inclc, hinclcp, hinclcs⟩ := Pc.incl_computable
  obtain ⟨eqc, heqcp, heqcs⟩ := Pc.eq_computable
  obtain ⟨fcc, hfccp, hfccs⟩ := Pc.cons_computable

  let PA : ComputablePresentation (funSpace (prod V₀ V₁) V₂) :=
    funPresentation (prodPresentation P₀ P₁) P₂
```

```
      (funConsChar (prodPresentation P₀ P₁) P₂ fcpr fc2) inclpr incl2 eq2
      (funConsChar_spec (prodPresentation P₀ P₁) P₂ fcpr fc2
        (fun s => (hfcprs s).symm) (fun s => (hfc2s s).symm))
      (primrec_funConsChar (prodPresentation P₀ P₁) P₂ fcpr fc2 hfcprp hfc2p)
      (fun s => (hinclprs s).symm) hinclprp (fun s => (hincl2s s).symm) hincl2p
      (fun s => (heq2s s).symm) heq2p

  let PB : ComputablePresentation (funSpace V₀ (funSpace V₁ V₂)) :=
    funPresentation P₀ Pc (funConsChar P₀ Pc fc0 fcc) incl0 inclc eqc
      (funConsChar_spec P₀ Pc fc0 fcc (fun s => (hfc0s s).symm) (fun s => (hfccs s).
    symm))
      (primrec_funConsChar P₀ Pc fc0 fcc hfc0p hfccp)
      (fun s => (hincl0s s).symm) hincl0p (fun s => (hinclcs s).symm) hinclcp
      (fun s => (heqcs s).symm) heqcp
  have hRec : RecDecidable₂ (fun n m => (curryC V₀ V₁ V₂).rel (PA.X n) (PB.X m)) :=
    curryComb_rel_recDecidable P₀ P₁ P₂
      (funConsChar (prodPresentation P₀ P₁) P₂ fcpr fc2) (funConsChar P₀ Pc fc0 fcc)
      (funConsChar P₁ P₂ fc1 fc2) PA PB Pc rfl rfl rfl
      (funConsChar_spec (prodPresentation P₀ P₁) P₂ fcpr fc2
        (fun s => (hfcprs s).symm) (fun s => (hfc2s s).symm))
      (funConsChar_spec P₁ P₂ fc1 fc2 (fun s => (hfc1s s).symm) (fun s => (hfc2s s).
    symm))
      (primrec_funConsChar P₀ Pc fc0 fcc hfc0p hfccp)
      (primrec_funConsChar P₁ P₂ fc1 fc2 hfc1p hfc2p)
  exact ⟨PA, PB, hRec, ⟨hRec.re⟩⟩

theorem toApproxMap_principal_mem {G : Set (ApproximableMap V₀ V₁)}
    (hG : (funSpace V₀ V₁).mem G) : toApproxMap ((funSpace V₀ V₁).principal hG) ∈ G :=
    by
  obtain ⟨L, hL, rfl⟩ := hG.1
  intro p hp
  show (toApproxMap ((funSpace V₀ V₁).principal hG)).rel p.1 p.2
  rw [toApproxMap_rel, mem_principal]
  exact ⟨step_mem (hL p hp).1 (hL p hp).2, fun f hf => hf p hp⟩

theorem curryComb_rel {G : Set (ApproximableMap (prod V₀ V₁) V₂)}
    {H : Set (ApproximableMap V₀ (funSpace V₁ V₂))} :
    (curryC V₀ V₁ V₂).rel G H ↔ (funSpace (prod V₀ V₁) V₂).mem G ∧
      (funSpace V₀ (funSpace V₁ V₂)).mem H ∧ ∀ g ∈ G, curry g ∈ H := by
  constructor
  · rintro ⟨hG, hH⟩
    rw [curryIso_apply, mem_toFilter] at hH
    obtain ⟨hHmem, hcurry⟩ := hH
    refine ⟨hG, hHmem, fun g hg => ?_⟩
    have hle : toApproxMap ((funSpace (prod V₀ V₁) V₂).principal hG) ≤ g := by
      rw [ApproximableMap.le_iff]
      intro X Y hrel
      rw [toApproxMap_rel, mem_principal] at hrel
      exact hrel.2 hg
    have hcle : curry (toApproxMap ((funSpace (prod V₀ V₁) V₂).principal hG)) ≤ curry g
     :=
      (curryEquiv V₀ V₁ V₂).monotone hle
    exact funSpace_mem_up_closed hHmem hcurry hcle
  · rintro ⟨hG, hH, hall⟩
    refine ⟨hG, ?_⟩
    rw [curryIso_apply, mem_toFilter]
```

```
    exact ⟨hH, hall _ (toApproxMap_principal_mem hG)⟩

end

end DomainTheory.NeighbourhoodSystems
end
```

4.8.41 Exercise 7.16 — curry as an enumeration

Let $D_0 = \{X_n \mid n \in \mathbf{N}\}$, $D_1 = \{Y_m \mid m \in \mathbf{N}\}$ and $D_2 = \{Z_k \mid k \in \mathbf{N}\}$ be three effectively given domains. Complete the proof of 7.5 by writing out curry as a relation between neighbourhoods. Is it a recursive set or only a recursively enumerable set?

The development is with the curry enumeration peer.

Theorem (`example_716`). Currying between function spaces is computable, and its neighbourhood relation is decidable from the presentations.

Proof. By the curry enumeration. ■

`Exercise_7_16.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .curryEnum

@[expose] public section

open DomainTheory.NeighbourhoodSystems

set_option linter.unusedVariables false
open NeighborhoodSystem Domain.Recursive ApproximableMap

/-- Scott's Exercise 7.16: curry combination is computable with a decidable relation.
    -/
theorem example_716 :
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)
      (P₂ : ComputablePresentation V₂),
      ∃ (PA : ComputablePresentation (funSpace (prod V₀ V₁) V₂))
        (PB : ComputablePresentation (funSpace V₀ (funSpace V₁ V₂))),
        RecDecidable₂ (fun n m => (curryC V₀ V₁ V₂).rel (PA.X n) (PB.X m)) ∧
          ComputableMap PA PB (curryC V₀ V₁ V₂)) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?}
      {G : Set (ApproximableMap (prod V₀ V₁) V₂)}
      {H : Set (ApproximableMap V₀ (funSpace V₁ V₂))},
      (curryC V₀ V₁ V₂).rel G H ↔ (funSpace (prod V₀ V₁) V₂).mem G ∧
        (funSpace V₀ (funSpace V₁ V₂)).mem H ∧ ∀ g ∈ G, curry g ∈ H) ∧
    (∀ {α β ? : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
      {V₂ : NeighborhoodSystem ?}
      (P₀ : ComputablePresentation V₀) (P₁ : ComputablePresentation V₁)
      (P₂ : ComputablePresentation V₂) (gNa gNb gNc : ℕ → ℕ)
      (PA : ComputablePresentation (funSpace (prod V₀ V₁) V₂))
```

```
    (PB : ComputablePresentation (funSpace V₀ (funSpace V₁ V₂)))
    (Pc : ComputablePresentation (funSpace V₁ V₂))
    (hPAX : PA.X = Xenum (prodPresentation P₀ P₁) P₂ gNa)
    (hPBX : PB.X = Xenum P₀ Pc gNb) (hPcX : Pc.X = Xenum P₁ P₂ gNc)
    (hgNa : ∀ c, gNa c = 1 ↔ (stepFun (funListOf (prodPresentation P₀ P₁) P₂ (
  decodeList c))
      : Set (ApproximableMap (prod V₀ V₁) V₂)).Nonempty)
    (hgNc : ∀ c, gNc c = 1 ↔ (stepFun (funListOf P₁ P₂ (decodeList c))
      : Set (ApproximableMap V₁ V₂)).Nonempty)
    (hgNbp : Nat.Primrec gNb) (hgNcp : Nat.Primrec gNc),
    RecDecidable₂ (fun n m => (curryC V₀ V₁ V₂).rel (PA.X n) (PB.X m))) :=
⟨@curryComb_isComputable, ⟨@curryComb_rel, @curryComb_rel_recDecidable⟩⟩
```

4.8.42 List recursion

Pairing, projection, and the recursion map on the list domain are computable.

Definition (`DsharpRecursion.GRel`). The recursion relation is generated by three clauses: the list-domain master relates to the codomain master; an embedded leaf relates through the map u; and an embedded pair relates through recursively related children followed by the binary map v.

Theorem (`DsharpRecursion.embZero_ne_embPair_of_memS`). An embedded leaf cannot equal an embedded pair whose left component is a list-domain neighbourhood.

Proof. The pair has an element with a nonempty path, whereas every element of an embedded leaf has the empty path. ■

Theorem (`DsharpRecursion.gRel_cod`). Every codomain set occurring in the recursion relation is a codomain neighbourhood.

Proof. Induct over the three constructors and use codomain preservation of u and v. ■

Theorem (`DsharpRecursion.gRel_dom`). Every domain set occurring in the recursion relation is a list-domain neighbourhood.

Proof. Induct over the constructors, using the master, leaf, and pair clauses for list-domain neighbourhoods. ■

Theorem (`DsharpRecursion.gRel_embPair_inv`). A recursion derivation from an embedded pair decomposes into derivations from both children and one application of v.

Proof. Master and leaf derivations are excluded by the distinct shapes; injectivity of pair embedding identifies the two children in the remaining case. ■

Theorem (`DsharpRecursion.gRel_embZero_inv`). A recursion derivation from an embedded leaf is exactly a relation of that leaf neighbourhood through u.

Proof. The master and pair shapes are impossible, leaving the leaf constructor. ■

Theorem (`DsharpRecursion.gRel_gamma_inv`). A recursion derivation from the list-domain master has the codomain master as its target.

Proof. Leaf and pair shapes are distinct from the master, so only the master constructor is possible. ■

Theorem (`DsharpRecursion.gRel_interRight`). For a fixed domain neighbourhood, the recursion relation is closed under intersection of targets.

Proof. Induct over the first derivation, invert the second by shape, intersect corresponding child targets, and use intersection closure of u or v. ■

Theorem (`DsharpRecursion.gRel_master`). Every list-domain neighbourhood is related to the codomain master.

Proof. Induct over its master, leaf, and pair construction, using the master relations of u and v. ■

Theorem (`DsharpRecursion.gRel_mono`). The recursion relation is downward monotone in its domain neighbourhood and upward monotone in its codomain neighbourhood.

Proof. Induct over a derivation, decompose smaller leaves or pairs by shape, and apply monotonicity of u and v. ■

Definition (`DsharpRecursion.gMap`). The recursion relation, with its domain, codomain, master, intersection, and monotonicity properties, defines an approximable map from the list domain to the codomain.

Theorem (`DsharpRecursion.gMap_rel`). The relation of the recursion map is the inductively generated recursion relation.

Proof. This is the defining relation. ■

Definition (`DsharpRecursion.gStep`). One coded evaluation step inspects the node shape and certificate: it checks master equality at a master node, a witness for u at a leaf, or two previously tabulated child certificates and a witness for v at a pair node.

Definition (`DsharpRecursion.gEval`). Coded recursion evaluation is course-of-values recursion generated by one evaluation step.

Theorem (`DsharpRecursion.gEval_pair`). Evaluation at a coded node and certificate is one step applied to that input together with its table of preceding evaluations.

Proof. This is the defining equation of course-of-values recursion. ■

Theorem (`DsharpRecursion.gMap_in`). Applying the recursion map after the list-domain leaf injection agrees with applying u.

Proof. Membership on either side decomposes into a leaf neighbourhood and the leaf inversion theorem converts the recursion relation to the relation of u. ■

Theorem (`DsharpRecursion.gMap_pair`). Applying the recursion map to a paired pair of list-domain elements agrees with applying v to the pair of their recursive images.

Proof. Decompose pair neighbourhoods and use pair inversion in one direction and the pair constructor in the other. ■

Theorem (`DsharpRecursion.gMap_strict`). The recursion map is strict.

Proof. Any relation from the list-domain master has codomain master by master inversion. ■

Theorem (`DsharpRecursion.gStep_gamma`). At a master-shaped node, one evaluation step returns the master-equality test and the proposed output index.

Proof. The master branch is selected by the node code. ■

Theorem (`DsharpRecursion.gStep_leaf`). At a leaf-shaped node, one evaluation step returns the u-witness test and the proposed output index.

Proof. The odd-node branch is selected and its leaf index is recovered by halving. ■

Theorem (`DsharpRecursion.gStep_node`). At a pair-shaped node, one evaluation step multiplies the success bits of both tabulated children with the v-witness test, and returns the proposed output index.

Proof. The positive even-node branch is selected and its two child addresses are decoded. ■

Theorem (`DsharpRecursion.gEval_out`). Evaluation preserves the proposed output index stored in the certificate.

Proof. Split into master, leaf, and pair node shapes; every step equation has that same second component. ■

Theorem (`DsharpRecursion.listGet_rtbl_gEval`). Looking up an earlier address in the evaluation table returns the evaluation at that address.

Proof. This is the defining property of the course-of-values table. ■

Theorem (`DsharpRecursion.pair_le_pair_right`). Pair coding is monotone in its second argument.

Proof. Compare the two branches of the pairing formula and use monotonicity of squaring. ■

Theorem (`DsharpRecursion.pair_lt_pair_left`). Pair coding is strictly monotone in its first argument.

Proof. A case analysis on which coordinate dominates in the pairing formula gives the strict inequality. ■

Theorem (`DsharpRecursion.pair_lt_pair_of_lt_le`). If the first coordinate strictly increases and the second weakly increases, the paired code strictly increases.

Proof. Combine strict monotonicity in the first coordinate with monotonicity in the second. ■

Theorem (`DsharpRecursion.gEval_complete`). Every recursion derivation from a numbered list-domain neighbourhood has a certificate whose evaluation succeeds and whose output index names the target neighbourhood.

Proof. Use strong induction on the node code. Master and leaf derivations supply direct witnesses; a pair derivation combines certificates for its smaller children with a witness for v. ■

Theorem (`DsharpRecursion.primrec_gStep`). If the three certificate tests are primitive recursive, then one coded evaluation step is primitive recursive.

Proof. Every decoded field, table lookup, arithmetic test, branch selection, and call to a certificate test is primitive recursive. ■

Theorem (`DsharpRecursion.unpair_snd_snd_fst_le`). The first component reached after two second projections of a code is at most the original code.

Proof. Each pairing projection is bounded by its source. ■

Theorem (`DsharpRecursion.unpair_snd_snd_snd_le`). The component reached after three second projections of a code is at most the original code.

Proof. Compose the bounds for successive second projections. ■

Theorem (`DsharpRecursion.gEval_sound`). Every successful evaluation certificate yields a derivation in the recursion relation for the named output neighbourhood.

Proof. Use strong induction on the paired node-and-certificate address. Master and leaf successes invoke their relation tests; pair success supplies successful smaller child evaluations and a witness for v. ■

Theorem (`DsharpRecursion.gMap_isComputable`). If u and v are computable, then their list-domain recursion map is computable.

Proof. Extract primitive-recursive witness tests for u, v, and equality with the master. Soundness and completeness identify the recursion relation with existence of a successful evaluation certificate, making it recursively enumerable. ■

`dsharpRecursion.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .listCombinators

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
```

```
open NeighborhoodSystem Domain.Recursive IndefProduct
open NeighborhoodSystem Domain.Recursive ApproximableMap
open NeighborhoodSystem Domain.Recursive ApproximableMap IndefProduct
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap IndefProduct
variable {α : Type*}

end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap IndefProduct SmashProduct
    Proposition77
variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

namespace DsharpRecursion

inductive GRel (u : ApproximableMap D E) (v : ApproximableMap (prod E E) E) :
    Set (List Bool × α) → Set β → Prop
  | gamma {Z} : Z = E.master → GRel u v (Gamma D) Z
  | leaf {X Z} : u.rel X Z → GRel u v (embZero X) Z
  | node {P Q Z₁ Z₂ Z} : GRel u v P Z₁ → GRel u v Q Z₂ →
      v.rel (prodNbhd Z₁ Z₂) Z → GRel u v (embPair P Q) Z

end DsharpRecursion
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap IndefProduct SmashProduct
    Proposition77
variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

namespace DsharpRecursion

variable {u : ApproximableMap D E} {v : ApproximableMap (prod E E) E}

theorem embZero_ne_embPair_of_memS (hD : ∀ X, D.mem X → X.Nonempty)
    {X : Set α} {P Q : Set (List Bool × α)} (hP : MemS D P) :
    embZero X = embPair P Q := by
  intro h
  obtain ⟨z, hz⟩ := embPair_nonempty (memS_nonempty hD hP) (Q := Q)
  have hz0 : z ∈ embZero X := h ▶ hz
  have hpath : z.1 = [] := hz0.1
  rcases hz with ⟨p', hp', -⟩ | ⟨q', hq', -⟩
  · rw [hp'] at hpath; simp at hpath
  · rw [hq'] at hpath; simp at hpath

theorem gRel_cod {W Z} (h : GRel u v W Z) : E.mem Z := by
  induction h with
  | gamma hZ => exact hZ ▶ E.master_mem
  | leaf hrel => exact u.rel_cod hrel
  | node _ _ hvrel _ _ => exact v.rel_cod hvrel

theorem gRel_dom {W Z} (h : GRel u v W Z) : MemS D W := by
  induction h with
```

```
  | gamma _ => exact MemS.gamma
  | leaf hrel => exact MemS.zero (u.rel_dom hrel)
  | node _ _ _ ihP ihQ => exact MemS.pair ihP ihQ

theorem gRel_embPair_inv (hD : ∀ X, D.mem X → X.Nonempty) {P Q : Set (List Bool × α)}
    {Z}
    (h : GRel u v (embPair P Q) Z) :
    ∃ Z₁ Z₂, GRel u v P Z₁ ∧ GRel u v Q Z₂ ∧ v.rel (prodNbhd Z₁ Z₂) Z := by
  have hPQ := gRel_dom h
  obtain ⟨hPmem, hQmem⟩ := memS_embPair_inv hD hPQ
  generalize hW : embPair P Q = W at h
  cases h with
  | gamma hZ => exact absurd hW (embPair_ne_Gamma D hD P Q)
  | @leaf X Z hrel => exact absurd hW.symm (embZero_ne_embPair_of_memS hD hPmem)
  | @node P' Q' Z₁ Z₂ Z hP' hQ' hvrel =>
      obtain ⟨rfl, rfl⟩ := embPair_injective hW
      exact ⟨Z₁, Z₂, hP', hQ', hvrel⟩

theorem gRel_embZero_inv (hD : ∀ X, D.mem X → X.Nonempty) {X : Set α} {Z}
    (h : GRel u v (embZero X) Z) : u.rel X Z := by
  generalize hW : embZero X = W at h
  cases h with
  | gamma hZ => exact absurd hW (embZero_ne_Gamma D hD X)
  | @leaf X' Z' hrel => rw [embZero_injective hW]; exact hrel
  | @node P Q Z₁ Z₂ Z hP hQ hvrel =>
      exact absurd hW (embZero_ne_embPair_of_memS hD (gRel_dom hP))

theorem gRel_gamma_inv (hD : ∀ X, D.mem X → X.Nonempty) {Z}
    (h : GRel u v (Gamma D) Z) : Z = E.master := by
  generalize hW : Gamma D = W at h
  cases h with
  | gamma hZ => exact hZ
  | @leaf X Z hrel => exact absurd hW.symm (embZero_ne_Gamma D hD X)
  | @node P Q Z₁ Z₂ Z hP hQ hvrel => exact absurd hW.symm (embPair_ne_Gamma D hD P Q)

theorem gRel_interRight (hD : ∀ X, D.mem X → X.Nonempty) {W Z} (h : GRel u v W Z) :
    ∀ {Z'}, GRel u v W Z' → GRel u v W (Z ∩ Z') := by
  induction h with
  | @gamma Z hZ =>
      intro Z' h'
      subst hZ
      have hZ' := gRel_gamma_inv hD h'
      subst hZ'
      rw [Set.inter_self]
      exact GRel.gamma rfl
  | @leaf X Z hrel =>
      intro Z' h'
      exact GRel.leaf (u.inter_right hrel (gRel_embZero_inv hD h'))
  | @node P Q Z₁ Z₂ Z hP hQ hvrel ihP ihQ =>
      intro Z' h'
      obtain ⟨Z₁', Z₂', hP', hQ', hv'⟩ := gRel_embPair_inv hD h'
      have hgP : GRel u v P (Z₁ ∩ Z₁') := ihP hP'
      have hgQ : GRel u v Q (Z₂ ∩ Z₂') := ihQ hQ'
      refine GRel.node hgP hgQ ?_
      have hmem : (prod E E).mem (prodNbhd (Z₁ ∩ Z₁') (Z₂ ∩ Z₂')) :=
        prod_mem_prodNbhd (gRel_cod hgP) (gRel_cod hgQ)
```

```
      have hv1 : v.rel (prodNbhd (Z₁ ∩ Z₁') (Z₂ ∩ Z₂')) Z :=
        v.mono hvrel (prodNbhd.subset_iff.mpr ⟨Set.inter_subset_left, Set.
    inter_subset_left⟩)
          subset_rfl hmem (v.rel_cod hvrel)
      have hv2 : v.rel (prodNbhd (Z₁ ∩ Z₁') (Z₂ ∩ Z₂')) Z' :=
        v.mono hv' (prodNbhd.subset_iff.mpr ⟨Set.inter_subset_right, Set.
    inter_subset_right⟩)
          subset_rfl hmem (v.rel_cod hv')
      exact v.inter_right hv1 hv2

theorem gRel_master : ∀ {W}, MemS D W → GRel u v W E.master := by
  intro W hW
  induction hW with
  | gamma => exact GRel.gamma rfl
  | @zero X hX =>
      exact GRel.leaf ((u.rel_iff_mem_principal hX).mpr (u.toElementMap _).master_mem)
  | @pair P Q _ _ ihP ihQ =>
      refine GRel.node ihP ihQ ?_
      have h := v.master_rel
      rwa [prod_master] at h

theorem gRel_mono (hD : ∀ X, D.mem X → X.Nonempty) {W Z} (h : GRel u v W Z) :
    ∀ {W' Z'}, W' ⊆ W → Z ⊆ Z' → MemS D W' → E.mem Z' → GRel u v W' Z' := by
  induction h with
  | @gamma Z hZ =>
      intro W' Z' hW'W hZZ' hW'mem hZ'mem
      subst hZ
      have hZ' : Z' = E.master := Set.Subset.antisymm (E.sub_master hZ'mem) hZZ'
      subst hZ'
      exact gRel_master hW'mem
  | @leaf X Z hrel =>
      intro W' Z' hW'W hZZ' hW'mem hZ'mem
      obtain ⟨X', hX'mem, rfl⟩ := memS_sub_embZero hD hW'mem hW'W
      exact GRel.leaf (u.mono hrel (embZero_subset.mp hW'W) hZZ' hX'mem hZ'mem)
  | @node P Q Z₁ Z₂ Z hP hQ hvrel ihP ihQ =>
      intro W' Z' hW'W hZZ' hW'mem hZ'mem
      obtain ⟨P', Q', hP'mem, hQ'mem, rfl⟩ := memS_sub_embPair hD hW'mem hW'W
      obtain ⟨hP'P, hQ'Q⟩ := embPair_subset.mp hW'W
      refine GRel.node (ihP hP'P subset_rfl hP'mem (gRel_cod hP))
        (ihQ hQ'Q subset_rfl hQ'mem (gRel_cod hQ)) ?_
      exact v.mono hvrel subset_rfl hZZ'
        (prod_mem_prodNbhd (gRel_cod hP) (gRel_cod hQ)) hZ'mem

def gMap (u : ApproximableMap D E) (v : ApproximableMap (prod E E) E)
    (hD : ∀ X, D.mem X → X.Nonempty) : ApproximableMap (Dsharp D hD) E where
  rel := GRel u v
  rel_dom h := gRel_dom h
  rel_cod h := gRel_cod h
  master_rel := GRel.gamma rfl
  inter_right h h' := gRel_interRight hD h h'
  mono h hsub hsup hX' hY' := gRel_mono hD h hsub hsup hX' hY'

@[simp] theorem gMap_rel {hD : ∀ X, D.mem X → X.Nonempty} {W : Set (List Bool × α)} {
    Z : Set β} :
    (gMap u v hD).rel W Z ↔ GRel u v W Z := Iff.rfl
```

```
section Computability

variable (fe fU fV : ℕ → ℕ) (mIdx : ℕ)

def gStep (p : ℕ) : ℕ :=
  let w := p.unpair.1
  let tbl := p.unpair.2
  let nn := w.unpair.1
  let cert := w.unpair.2
  let out := cert.unpair.1
  let wit := cert.unpair.2.unpair.1
  let lcert := cert.unpair.2.unpair.2.unpair.1
  let rcert := cert.unpair.2.unpair.2.unpair.2
  let b := nn / 2 - 1
  let childL := listGet tbl (w - 1 - Nat.pair b.unpair.1 lcert)
  let childR := listGet tbl (w - 1 - Nat.pair b.unpair.2 rcert)
  selectFn (1 - nn)
    (Nat.pair (fe (Nat.pair out mIdx)) out)
    (selectFn (nn % 2)
      (Nat.pair (fU (Nat.pair wit (Nat.pair (nn / 2) out))) out)
      (Nat.pair (childL.unpair.1 * childR.unpair.1
          * fV (Nat.pair wit (Nat.pair (Nat.pair childL.unpair.2 childR.unpair.2) out)
   )) out))

def gEval : ℕ → ℕ := gOf (gStep fe fU fV mIdx)

theorem gEval_pair (n cert : ℕ) :
    gEval fe fU fV mIdx (Nat.pair n cert)
      = gStep fe fU fV mIdx
          (Nat.pair (Nat.pair n cert) (rtbl (gStep fe fU fV mIdx) (Nat.pair n cert)))
    := by
  rw [gEval, gOf_def]

end Computability

theorem gMap_in (hD : ∀ X, D.mem X → X.Nonempty) (x : D.Element) :
    (gMap u v hD).toElementMap (inSharp D hD x) = u.toElementMap x := by
  apply Element.ext
  intro Z
  simp only [mem_toElementMap, gMap_rel]
  constructor
  · rintro ⟨W, hWmem, hrel⟩
    rcases hWmem with rfl | ⟨X, hXx, rfl⟩
    · have hZ := gRel_gamma_inv hD hrel; subst hZ
      exact ⟨D.master, x.master_mem, u.master_rel⟩
    · exact ⟨X, hXx, gRel_embZero_inv hD hrel⟩
  · rintro ⟨X, hXx, hrel⟩
    exact ⟨embZero X, Or.inr ⟨X, hXx, rfl⟩, GRel.leaf hrel⟩

theorem gMap_pair (hD : ∀ X, D.mem X → X.Nonempty) (y z : (Dsharp D hD).Element) :
    (gMap u v hD).toElementMap (pairSharp D hD y z)
      = v.toElementMap (pair ((gMap u v hD).toElementMap y) ((gMap u v hD).
   toElementMap z)) := by
  apply Element.ext
  intro Z
  simp only [mem_toElementMap, gMap_rel]
```

```
  constructor
  · rintro ⟨W, hWmem, hrel⟩
    rcases hWmem with rfl | ⟨P, Q, hPy, hQz, rfl⟩
    · have hZ := gRel_gamma_inv hD hrel; subst hZ
      exact ⟨(prod E E).master, (pair _ _).master_mem, v.master_rel⟩
    · obtain ⟨Z₁, Z₂, hP1, hQ2, hvrel⟩ := gRel_embPair_inv hD hrel
      exact ⟨prodNbhd Z₁ Z₂,
        mem_pair.mpr ⟨Z₁, Z₂, ⟨P, hPy, hP1⟩, ⟨Q, hQz, hQ2⟩, rfl⟩, hvrel⟩
  · rintro ⟨V, hVmem, hvrel⟩
    obtain ⟨Z₁, Z₂, ⟨P, hPy, hP1⟩, ⟨Q, hQz, hQ2⟩, rfl⟩ := mem_pair.mp hVmem
    exact ⟨embPair P Q, Or.inr ⟨P, Q, hPy, hQz, rfl⟩, GRel.node hP1 hQ2 hvrel⟩

theorem gMap_strict (hD : ∀ X, D.mem X → X.Nonempty) :
    IsStrict (gMap u v hD) := fun _ h => gRel_gamma_inv hD h

section Computability

variable (fe fU fV : ℕ → ℕ) (mIdx : ℕ)

theorem gStep_gamma (cert tbl : ℕ) :
    gStep fe fU fV mIdx (Nat.pair (Nat.pair 0 cert) tbl)
      = Nat.pair (fe (Nat.pair cert.unpair.1 mIdx)) cert.unpair.1 := by
  simp only [gStep, unpair_pair_fst, unpair_pair_snd, Nat.sub_zero, selectFn_one]

theorem gStep_leaf (a cert tbl : ℕ) :
    gStep fe fU fV mIdx (Nat.pair (Nat.pair (2 * a + 1) cert) tbl)
      = Nat.pair (fU (Nat.pair cert.unpair.2.unpair.1 (Nat.pair a cert.unpair.1)))
          cert.unpair.1 := by
  simp only [gStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : ℕ) - (2 * a + 1) = 0 by omega, selectFn_zero,
    show (2 * a + 1) % 2 = 1 by omega, selectFn_one, show (2 * a + 1) / 2 = a by omega
    ]

theorem gStep_node (a cert tbl : ℕ) :
    gStep fe fU fV mIdx (Nat.pair (Nat.pair (2 * a + 2) cert) tbl)
      = Nat.pair
          ((listGet tbl (Nat.pair (2 * a + 2) cert - 1
                - Nat.pair a.unpair.1 cert.unpair.2.unpair.2.unpair.1)).unpair.1
            * (listGet tbl (Nat.pair (2 * a + 2) cert - 1
                - Nat.pair a.unpair.2 cert.unpair.2.unpair.2.unpair.2)).unpair.1
            * fV (Nat.pair cert.unpair.2.unpair.1
                (Nat.pair (Nat.pair
                    (listGet tbl (Nat.pair (2 * a + 2) cert - 1
                      - Nat.pair a.unpair.1 cert.unpair.2.unpair.2.unpair.1)).unpair.2
                    (listGet tbl (Nat.pair (2 * a + 2) cert - 1
                      - Nat.pair a.unpair.2 cert.unpair.2.unpair.2.unpair.2)).unpair
    .2)
                  cert.unpair.1)))
          cert.unpair.1 := by
  simp only [gStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : ℕ) - (2 * a + 2) = 0 by omega, selectFn_zero,
    show (2 * a + 2) % 2 = 0 by omega, selectFn_zero, show (2 * a + 2) / 2 - 1 = a by
    omega]

theorem gEval_out (n cert : ℕ) :
    (gEval fe fU fV mIdx (Nat.pair n cert)).unpair.2 = cert.unpair.1 := by
```

```
  rw [gEval_pair]
  rcases nat_shape n with rfl | ⟨a, rfl⟩ | ⟨a, rfl⟩
  · rw [gStep_gamma, unpair_pair_snd]
  · rw [gStep_leaf, unpair_pair_snd]
  · rw [gStep_node, unpair_pair_snd]

theorem listGet_rtbl_gEval {w' w : ℕ} (h : w' < w) :
    listGet (rtbl (gStep fe fU fV mIdx) w) (w - 1 - w') = gEval fe fU fV mIdx w' :=
  listGet_rtbl (gStep fe fU fV mIdx) h

end Computability

theorem pair_le_pair_right {a b d : ℕ} (h : b ≤ d) : Nat.pair a b ≤ Nat.pair a d := by
  unfold Nat.pair
  rcases Nat.lt_or_ge a b with hab | hab
  · rw [ite_eq_left hab, ite_eq_left (lt_of_lt_of_le hab h)]
    have : b * b ≤ d * d := Nat.mul_le_mul h h
    omega
  · rw [ite_eq_right (Nat.not_lt.mpr hab)]
    rcases Nat.lt_or_ge a d with had | had
    · rw [ite_eq_left had]
      have h1 : a * a + a ≤ d * d := by
        calc a * a + a = a * (a + 1) := by ring
          _ ≤ a * d := Nat.mul_le_mul (le_refl a) had
          _ ≤ d * d := Nat.mul_le_mul had.le (le_refl d)
      omega
    · rw [ite_eq_right (Nat.not_lt.mpr had)]; omega

theorem pair_lt_pair_left {a c b : ℕ} (h : a < c) : Nat.pair a b < Nat.pair c b := by
  unfold Nat.pair
  rcases Nat.lt_or_ge a b with hab | hab
  · rw [ite_eq_left hab]
    rcases Nat.lt_or_ge c b with hcb | hcb
    · rw [ite_eq_left hcb]; omega
    · rw [ite_eq_right (Nat.not_lt.mpr hcb)]
      have hbc : b * b ≤ c * c := Nat.mul_le_mul hcb hcb
      omega
  · rw [ite_eq_right (Nat.not_lt.mpr hab),
      ite_eq_right (Nat.not_lt.mpr (le_of_lt (lt_of_le_of_lt hab h)))]
    have hcc : (a + 1) * (a + 1) ≤ c * c := Nat.mul_le_mul h h
    have hexp : (a + 1) * (a + 1) = a * a + 2 * a + 1 := by ring
    omega

theorem pair_lt_pair_of_lt_le {a c b d : ℕ} (hac : a < c) (hbd : b ≤ d) :
    Nat.pair a b < Nat.pair c d :=
  lt_of_lt_of_le (pair_lt_pair_left hac) (pair_le_pair_right hbd)

section Computability

variable (fe fU fV : ℕ → ℕ) (mIdx : ℕ)

set_option maxHeartbeats 1000000 in

theorem gEval_complete (hD : ∀ X, D.mem X → X.Nonempty)
    (P : ComputablePresentation D) (Q : ComputablePresentation E)
    (hEq : ∀ m, fe (Nat.pair m mIdx) = 1 ↔ Q.X m = E.master) (hMidx : Q.X mIdx = E.
```

```
  master)
   (hU : ∀ a m, (∃ i, fU (Nat.pair i (Nat.pair a m)) = 1) ↔ u.rel (P.X a) (Q.X m))
   (hV : ∀ k1 k2 m, (∃ i, fV (Nat.pair i (Nat.pair (Nat.pair k1 k2) m)) = 1) ↔
       v.rel (prodNbhd (Q.X k1) (Q.X k2)) (Q.X m)) :
  ∀ n Z, GRel u v (Vsharp D P n) Z →
    ∃ cert, (gEval fe fU fV mIdx (Nat.pair n cert)).unpair.1 = 1
      ∧ Q.X cert.unpair.1 = Z := by
intro n
induction n using Nat.strong_induction_on with
| _ n ih =>
  intro Z hg
  rcases nat_shape n with rfl | ⟨a, rfl⟩ | ⟨a, rfl⟩
  .
    rw [Vsharp_zero] at hg
    have hZ := gRel_gamma_inv hD hg
    refine ⟨Nat.pair mIdx (Nat.pair 0 0), ?_, ?_⟩
    · rw [gEval_pair, gStep_gamma, unpair_pair_fst, unpair_pair_fst]
      exact (hEq mIdx).mpr hMidx
    · rw [unpair_pair_fst, hMidx, hZ]
  .
    rw [Vsharp_odd] at hg
    have hu := gRel_embZero_inv hD hg
    obtain ⟨e, he⟩ := Q.surj (u.rel_cod hu)
    rw [← he] at hu
    obtain ⟨wit, hwit⟩ := (hU a e).mpr hu
    refine ⟨Nat.pair e (Nat.pair wit 0), ?_, ?_⟩
    · rw [gEval_pair, gStep_leaf, unpair_pair_fst, unpair_pair_snd, unpair_pair_fst,
        unpair_pair_fst]
      exact hwit
    · rw [unpair_pair_fst]; exact he
  .
    rw [Vsharp_even] at hg
    obtain ⟨Z₁, Z₂, hg1, hg2, hvrel⟩ := gRel_embPair_inv hD hg
    obtain ⟨certL, hokL, heqL⟩ := ih a.unpair.1 (by have := unpair_fst_le a; omega) Z
  ₁ hg1
    obtain ⟨certR, hokR, heqR⟩ := ih a.unpair.2 (by have := unpair_snd_le a; omega) Z
  ₂ hg2
    obtain ⟨e, he⟩ := Q.surj (v.rel_cod hvrel)
    rw [← heqL, ← heqR, ← he] at hvrel
    obtain ⟨wit, hwit⟩ := (hV certL.unpair.1 certR.unpair.1 e).mpr hvrel
    refine ⟨Nat.pair e (Nat.pair wit (Nat.pair certL certR)), ?_, ?_⟩
    · rw [gEval_pair, gStep_node, unpair_pair_fst]
      have hcertL : certL ≤ Nat.pair e (Nat.pair wit (Nat.pair certL certR)) :=
        le_trans (Domain.Recursive.le_pair_left _ _)
          (le_trans (Domain.Recursive.le_pair_right wit _) (Domain.Recursive.
  le_pair_right e _))
      have hcertR : certR ≤ Nat.pair e (Nat.pair wit (Nat.pair certL certR)) :=
        le_trans (Domain.Recursive.le_pair_right _ _)
          (le_trans (Domain.Recursive.le_pair_right wit _) (Domain.Recursive.
  le_pair_right e _))
      have hlt1 : Nat.pair a.unpair.1 certL < Nat.pair (2 * a + 2)
          (Nat.pair e (Nat.pair wit (Nat.pair certL certR))) :=
        pair_lt_pair_of_lt_le (by have := unpair_fst_le a; omega) hcertL
      have hlt2 : Nat.pair a.unpair.2 certR < Nat.pair (2 * a + 2)
          (Nat.pair e (Nat.pair wit (Nat.pair certL certR))) :=
        pair_lt_pair_of_lt_le (by have := unpair_snd_le a; omega) hcertR
```

```
        simp only [unpair_pair_fst, unpair_pair_snd]
        rw [listGet_rtbl_gEval fe fU fV mIdx hlt1, listGet_rtbl_gEval fe fU fV mIdx
    hlt2,
          gEval_out, gEval_out]
        rw [show (gEval fe fU fV mIdx (Nat.pair a.unpair.1 certL)).unpair.1 = 1 from
    hokL,
          show (gEval fe fU fV mIdx (Nat.pair a.unpair.2 certR)).unpair.1 = 1 from
    hokR]
        simp only [Nat.one_mul]
        exact hwit
      · rw [unpair_pair_fst]; exact he

theorem primrec_gStep (hfe : Nat.Primrec fe) (hfU : Nat.Primrec fU) (hfV : Nat.Primrec
     fV) :
    Nat.Primrec (gStep fe fU fV mIdx) := by
  have hw : Nat.Primrec (fun p : ℕ => p.unpair.1) := Nat.Primrec.left
  have htbl : Nat.Primrec (fun p : ℕ => p.unpair.2) := Nat.Primrec.right
  have hnn : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.left
  have hcert : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left

  have hout : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.2.unpair.1) := Nat.Primrec.
    left.comp hcert
  have hwit : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.2.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp hcert)
  have hlc : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.1)
     :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp (Nat.Primrec.right.comp hcert))
  have hrc : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.2)
     :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp (Nat.Primrec.right.comp hcert))
  have hb : Nat.Primrec (fun p : ℕ => p.unpair.1.unpair.1 / 2 - 1) :=
    primrec_sub₂ (primrec_div2.comp hnn) (Nat.Primrec.const 1)
  have hlpos : Nat.Primrec (fun p : ℕ =>
      p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.1
        p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.1) :=
    primrec_sub₂ (primrec_sub₂ hw (Nat.Primrec.const 1))
      ((Nat.Primrec.left.comp hb).pair hlc)
  have hrpos : Nat.Primrec (fun p : ℕ =>
      p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.2
        p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.2) :=
    primrec_sub₂ (primrec_sub₂ hw (Nat.Primrec.const 1))
      ((Nat.Primrec.right.comp hb).pair hrc)
  have hchildL : Nat.Primrec (fun p : ℕ => listGet p.unpair.2
      (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.1
        p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.1)) :=
    primrec_listGet₂ htbl hlpos
  have hchildR : Nat.Primrec (fun p : ℕ => listGet p.unpair.2
      (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.2
        p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.2)) :=
    primrec_listGet₂ htbl hrpos
  have hgamma : Nat.Primrec (fun p : ℕ =>
      Nat.pair (fe (Nat.pair p.unpair.1.unpair.2.unpair.1 mIdx)) p.unpair.1.unpair.2.
    unpair.1) :=
    (hfe.comp (hout.pair (Nat.Primrec.const mIdx))).pair hout
```

```
  have hleaf : Nat.Primrec (fun p : ℕ =>
      Nat.pair (fU (Nat.pair p.unpair.1.unpair.2.unpair.2.unpair.1
          (Nat.pair (p.unpair.1.unpair.1 / 2) p.unpair.1.unpair.2.unpair.1)))
        p.unpair.1.unpair.2.unpair.1) :=
    (hfU.comp (hwit.pair ((primrec_div2.comp hnn).pair hout))).pair hout
  have hfVcall : Nat.Primrec (fun p : ℕ =>
      fV (Nat.pair p.unpair.1.unpair.2.unpair.2.unpair.1
        (Nat.pair (Nat.pair
            (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 -
    1).unpair.1
              p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.1)).unpair.2
            (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 -
    1).unpair.2
              p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.2)).unpair.2)
          p.unpair.1.unpair.2.unpair.1))) :=
    hfV.comp (hwit.pair
      (((Nat.Primrec.right.comp hchildL).pair (Nat.Primrec.right.comp hchildR)).pair
    hout))
  have hnode : Nat.Primrec (fun p : ℕ =>
      Nat.pair ((listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 /
    2 - 1).unpair.1
              p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.1)).unpair.1
          * (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 -
    1).unpair.2
              p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.2)).unpair.1
          * fV (Nat.pair p.unpair.1.unpair.2.unpair.2.unpair.1
              (Nat.pair (Nat.pair
                  (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1
    / 2 - 1).unpair.1
                    p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.1)).unpair.2
                  (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1
    / 2 - 1).unpair.2
                    p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.2)).unpair.2)
                p.unpair.1.unpair.2.unpair.1)))
        p.unpair.1.unpair.2.unpair.1) :=
    (primrec_mul₂ (primrec_mul₂ (Nat.Primrec.left.comp hchildL) (Nat.Primrec.left.comp
    hchildR))
      hfVcall).pair hout
  refine (primrec_selectFn (primrec_sub₂ (Nat.Primrec.const 1) hnn) hgamma
    (primrec_selectFn (primrec_mod2.comp hnn) hleaf hnode)).of_eq fun p => rfl

end Computability

theorem unpair_snd_snd_fst_le (c : ℕ) : c.unpair.2.unpair.2.unpair.1 ≤ c := by
  have h1 := unpair_snd_le c
  have h2 := unpair_snd_le c.unpair.2
  have h3 := unpair_fst_le c.unpair.2.unpair.2
  omega

theorem unpair_snd_snd_snd_le (c : ℕ) : c.unpair.2.unpair.2.unpair.2 ≤ c := by
  have h1 := unpair_snd_le c
  have h2 := unpair_snd_le c.unpair.2
  have h3 := unpair_snd_le c.unpair.2.unpair.2
  omega

section Computability
```

```
variable (fe fU fV : ℕ → ℕ) (mIdx : ℕ)

theorem gEval_sound
    (P : ComputablePresentation D) (Q : ComputablePresentation E)
    (hEq : ∀ m, fe (Nat.pair m mIdx) = 1 ↔ Q.X m = E.master)
    (hU : ∀ a m, (∃ i, fU (Nat.pair i (Nat.pair a m)) = 1) ↔ u.rel (P.X a) (Q.X m))
    (hV : ∀ k1 k2 m, (∃ i, fV (Nat.pair i (Nat.pair (Nat.pair k1 k2) m)) = 1) ↔
        v.rel (prodNbhd (Q.X k1) (Q.X k2)) (Q.X m)) :
    ∀ n cert, (gEval fe fU fV mIdx (Nat.pair n cert)).unpair.1 = 1 →
      GRel u v (Vsharp D P n) (Q.X cert.unpair.1) := by
  have key : ∀ w n cert, Nat.pair n cert = w →
      (gEval fe fU fV mIdx (Nat.pair n cert)).unpair.1 = 1 →
        GRel u v (Vsharp D P n) (Q.X cert.unpair.1) := by
    intro w
    induction w using Nat.strong_induction_on with
    | _ w ih =>
      intro n cert hw hok
      rcases nat_shape n with rfl | ⟨a, rfl⟩ | ⟨a, rfl⟩
      · rw [gEval_pair, gStep_gamma, unpair_pair_fst] at hok
        rw [Vsharp_zero]
        exact GRel.gamma ((hEq cert.unpair.1).mp hok)
      · rw [gEval_pair, gStep_leaf, unpair_pair_fst] at hok
        rw [Vsharp_odd]
        exact GRel.leaf ((hU a cert.unpair.1).mp ⟨cert.unpair.2.unpair.1, hok⟩)
      · rw [Vsharp_even]
        rw [gEval_pair, gStep_node, unpair_pair_fst] at hok
        have hlt1 : Nat.pair a.unpair.1 cert.unpair.2.unpair.2.unpair.1 < Nat.pair (2
    * a + 2) cert :=
          pair_lt_pair_of_lt_le (by have := unpair_fst_le a; omega) (
    unpair_snd_snd_fst_le cert)
        have hlt2 : Nat.pair a.unpair.2 cert.unpair.2.unpair.2.unpair.2 < Nat.pair (2
    * a + 2) cert :=
          pair_lt_pair_of_lt_le (by have := unpair_snd_le a; omega) (
    unpair_snd_snd_snd_le cert)
        rw [listGet_rtbl_gEval fe fU fV mIdx hlt1,
          listGet_rtbl_gEval fe fU fV mIdx hlt2] at hok
        rw [nat_mul_eq_one, nat_mul_eq_one] at hok
        obtain ⟨⟨hokL, hokR⟩, hokV⟩ := hok
        have hgL := ih _ (hw ▶ hlt1) a.unpair.1 cert.unpair.2.unpair.2.unpair.1 rfl
    hokL
        have hgR := ih _ (hw ▶ hlt2) a.unpair.2 cert.unpair.2.unpair.2.unpair.2 rfl
    hokR
        rw [gEval_out, gEval_out] at hokV
        refine GRel.node hgL hgR ?_
        exact (hV _ _ cert.unpair.1).mp ⟨cert.unpair.2.unpair.1, hokV⟩
  intro n cert; exact key (Nat.pair n cert) n cert rfl

end Computability

theorem gMap_isComputable (hD : ∀ X, D.mem X → X.Nonempty)
    (P : ComputablePresentation D) (Q : ComputablePresentation E)
    (hu : ComputableMap P Q u)
    (hv : ComputableMap (prodPresentation Q Q) Q v) :
    ComputableMap (dsharpPresentation P hD) Q (gMap u v hD) := by
  constructor
```

```
obtain ⟨qU, hqU, hqUe⟩ := hu
obtain ⟨fU, hfUp, hfUe⟩ := hqU
obtain ⟨qV, hqV, hqVe⟩ := hv
obtain ⟨fV, hfVp, hfVe⟩ := hqV
obtain ⟨fe, hfep, hfee⟩ := Q.eq_computable
have hMidx : Q.X Q.masterIdx = E.master := Q.masterIdx_spec
have hEq : ∀ m, fe (Nat.pair m Q.masterIdx) = 1 ↔ Q.X m = E.master := by
  intro m
  have h := hfee (Nat.pair m Q.masterIdx)
  simp only [unpair_pair_fst, unpair_pair_snd] at h
  rw [hMidx] at h; exact h.symm
have hU : ∀ a m, (∃ i, fU (Nat.pair i (Nat.pair a m)) = 1) ↔ u.rel (P.X a) (Q.X m)
  := by
  intro a m
  have h := hqUe (Nat.pair a m)
  simp only [unpair_pair_fst, unpair_pair_snd] at h
  rw [h]; exact exists_congr fun i => (hfUe (Nat.pair i (Nat.pair a m))).symm
have hV : ∀ k1 k2 m, (∃ i, fV (Nat.pair i (Nat.pair (Nat.pair k1 k2) m)) = 1) ↔
    v.rel (prodNbhd (Q.X k1) (Q.X k2)) (Q.X m) := by
  intro k1 k2 m
  have h := hqVe (Nat.pair (Nat.pair k1 k2) m)
  simp only [prodPresentation_X, unpair_pair_fst, unpair_pair_snd] at h
  rw [h]; exact exists_congr fun i => (hfVe (Nat.pair i (Nat.pair (Nat.pair k1 k2) m
  ))).symm

have hgEval : Nat.Primrec (gEval fe fU fV Q.masterIdx) :=
  primrec_gOf (primrec_gStep fe fU fV Q.masterIdx hfep hfUp hfVp)
have hidx : Nat.Primrec (fun s : ℕ => Nat.pair s.unpair.2.unpair.1 s.unpair.1) :=
  (Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left
have hp1 : RecDecidable (fun s : ℕ =>
    (gEval fe fU fV Q.masterIdx (Nat.pair s.unpair.2.unpair.1 s.unpair.1)).unpair.1
  = 1) :=
  ⟨_, Nat.Primrec.left.comp (hgEval.comp hidx), fun _ => Iff.rfl⟩
have hg2 : Nat.Primrec (fun s : ℕ => Nat.pair s.unpair.1.unpair.1 s.unpair.2.unpair
  .2) :=
  (Nat.Primrec.left.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.Primrec.
  right)
have hp2 : RecDecidable (fun s : ℕ => Q.X s.unpair.1.unpair.1 = Q.X s.unpair.2.
  unpair.2) :=
  RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
    (Q.eq_computable.comp hg2)
have hbase : REPred (fun s : ℕ =>
    (gEval fe fU fV Q.masterIdx (Nat.pair s.unpair.2.unpair.1 s.unpair.1)).unpair.1
  = 1
      ∧ Q.X s.unpair.1.unpair.1 = Q.X s.unpair.2.unpair.2) := (hp1.and hp2).re
unfold REPred₂
refine REPred.of_iff (fun t => ?_) (REPred.proj hbase)
simp only [unpair_pair_fst, unpair_pair_snd]
constructor
· intro hg
  obtain ⟨cert, hok, heq⟩ :=
    gEval_complete fe fU fV Q.masterIdx hD P Q hEq hMidx hU hV t.unpair.1 (Q.X t.
  unpair.2) hg
  exact ⟨cert, hok, heq⟩
· rintro ⟨cert, hok, heq⟩
  have hsound := gEval_sound fe fU fV Q.masterIdx P Q hEq hU hV t.unpair.1 cert hok
```

```
    rw [heq] at hsound
    exact hsound

end DsharpRecursion
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems
end DomainTheory.NeighbourhoodSystems
end
```

4.8.43 Exercise 7.17 — recursion over lists

Complete the proof of 7.7 for showing that $D^{\S}$ is effectively given if D is. Include all the combinators of 6.2. Prove also that if E is effectively given and $u : D \to E$ and $v : E \times E \to E$ are computable, then the unique strict mapping $g : D^{\S} \to E$, where, for $x \in |D|$ and $y, z \in |E|$,

$$g(\mathrm{in}(x)) = u(x), \quad \text{and}$$

$$g(\mathrm{pair}(y, z)) = v(g(y), g(z)),$$

is a computable mapping.

The development is with the list recursion peer.

Theorem (`example_717`). Pairing and the second projection on the list domain are computable.

Proof. By the list-domain combinators. ■

Theorem (`example_717_part2`). The recursion map determined by a computable step on the list domain is computable.

Proof. By evaluation of the recursion along a finite chain. ■

`Exercise_7_17.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .dsharpRecursion

@[expose] public section

open DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap IndefProduct
open Proposition77

/-- Scott's Exercise 7.17 (part 1): pairing and second projection on D? are computable
    . -/
theorem example_717 :
    (∀ {α : Type*} {D : NeighborhoodSystem α} (hD : ∀ X, D.mem X → X.Nonempty)
      (P : ComputablePresentation D),
      ComputableMap (prodPresentation (dsharpPresentation P hD) (dsharpPresentation P
    hD))
        (dsharpPresentation P hD) (pairSharpMap hD)) ∧
    (∀ {α : Type*} {D : NeighborhoodSystem α} (hD : ∀ X, D.mem X → X.Nonempty)
      (P : ComputablePresentation D),
```

```
      ComputableMap (dsharpPresentation P hD) (dsharpPresentation P hD) (proj1Map hD))
    :=
  ⟨@pairSharp_isComputable, @proj1_isComputable⟩

open DsharpRecursion

/-- Scott's Exercise 7.17 (part 2): the g-map is computable. -/
theorem example_717_part2 :
    ∀ {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}
      {u : ApproximableMap D E} {v : ApproximableMap (prod E E) E}
      (hD : ∀ X, D.mem X → X.Nonempty)
      (P : ComputablePresentation D) (Q : ComputablePresentation E),
      ComputableMap P Q u →
        ComputableMap (prodPresentation Q Q) Q v →
          ComputableMap (dsharpPresentation P hD) Q (gMap u v hD) :=
  @gMap_isComputable
```

4.8.44 Effective isomorphism

An effective isomorphism is a computable pair of inverse approximable maps, and an iterate is effectively isomorphic to its iterate.

Definition (`EffectiveIso`). An effective isomorphism between two presented systems consists of computable approximable maps in both directions whose two composites are the corresponding identity maps.

Definition (`EffectivelyIsomorphic`). Two computably presented systems are effectively isomorphic when an effective isomorphism between them exists.

Definition (`EffectiveIso.fiber2`). The (i, j)-fibre of a doubly indexed set is the j-th fibre of its i-th fibre.

Theorem (`EffectiveIso.mem_fiber2`). An element belongs to the (i, j)-fibre exactly when the doubly paired element belongs to the original set.

Proof. This is the defining membership condition for two successive fibres. ■

Theorem (`EffectiveIso.fiber2_inter`). Taking a double fibre preserves intersections.

Proof. Fibre membership is pointwise membership, which preserves conjunction. ■

Theorem (`EffectiveIso.fiber2_master`). Every double fibre of the master of a double iterate is the original master.

Proof. Each of the two successive master fibres is a master. ■

Theorem (`EffectiveIso.fiber2_mono`). Inclusion of doubly indexed sets induces inclusion of every double fibre.

Proof. Apply the original inclusion to the paired element. ■

Theorem (`EffectiveIso.mem_fiber2_of_mem`). Every double fibre of a double-iterate neighbourhood is an original neighbourhood.

Proof. The outer fibre is an iterate neighbourhood and its inner fibre is therefore an original neighbourhood. ■

Definition (`EffectiveIso.reindexF`). Forward reindexing replaces a single coordinate coded by a pair with two successive coordinates.

Theorem (`EffectiveIso.fiber2_reindexF`). The (i, j)-fibre after forward reindexing is the original fibre at the paired index (i, j).

Proof. This is immediate from the membership definition. ■

Definition (`EffectiveIso.reindexG`). Backward reindexing replaces two successive coordinates with the two components decoded from one coordinate.

Theorem (`EffectiveIso.fiber_reindexG`). The fibre at k after backward reindexing is the original double fibre at the two components of k.

Proof. This is immediate from the membership definition. ■

Theorem (`EffectiveIso.exists_inner_bound`). For every double-iterate neighbourhood and every finite number of outer fibres, there is one bound beyond which all their inner fibres are master.

Proof. Induct over the number of outer fibres and take the maximum of the previous bound and the new fibre's bound. ■

Theorem (`EffectiveIso.pair_lt_pair_of_lt`). If both coordinates strictly increase, their paired code strictly increases.

Proof. Bound the first pair below the square following its largest coordinate and the second pair above the square of its largest coordinate. ■

Theorem (`EffectiveIso.reindexF_mem`). Forward reindexing sends iterate neighbourhoods to double-iterate neighbourhoods.

Proof. All double fibres are original neighbourhoods, inner fibres become master beyond the original bound, and outer fibres do likewise. ■

Theorem (`EffectiveIso.reindexF_subset_iff`). Forward reindexing of W is included in S exactly when every fibre of W at (i, j) is included in the (i, j)-fibre of S.

Proof. Set inclusion is equivalent to inclusion of all successive fibres. ■

Definition (`EffectiveIso.Fmap`). The forward approximable map relates W to S when both are neighbourhoods and every paired fibre of W is included in the corresponding double fibre of S.

Theorem (`EffectiveIso.Fmap_rel`). The relation of the forward map is exactly its neighbourhood and fibrewise inclusion conditions.

Proof. This is its defining relation. ■

Theorem (`EffectiveIso.reindexG_mem`). Backward reindexing sends double-iterate neighbourhoods to iterate neighbourhoods.

Proof. Use a common inner bound for the finitely many nonmaster outer fibres; any paired index beyond the paired bounds has either a master outer fibre or a master inner fibre. ■

Theorem (`EffectiveIso.reindexG_subset_iff`). Backward reindexing of S is included in W exactly when each decoded double fibre of S is included in the corresponding fibre of W.

Proof. Inclusion is equivalent to inclusion of every single fibre. ■

Definition (`EffectiveIso.Gmap`). The backward approximable map relates S to W when both are neighbourhoods and every decoded double fibre of S is included in the corresponding fibre of W.

Theorem (`EffectiveIso.Gmap_rel`). The relation of the backward map is exactly its neighbourhood and fibrewise inclusion conditions.

Proof. This is its defining relation. ■

Theorem (`EffectiveIso.Fmap_comp_Gmap`). Forward reindexing after backward reindexing is the identity approximable map on the double iterate.

Proof. A mediating reindexed neighbourhood converts the two fibrewise inclusions into ordinary inclusion, and backward reindexing itself supplies a mediator for the converse. ■

Theorem (`EffectiveIso.Gmap_comp_Fmap`). Backward reindexing after forward reindexing is the identity approximable map on the iterate.

Proof. Pairing and unpairing identify the intermediate fibres, reducing the composite relation to inclusion. ■

Theorem (`EffectiveIso.fiber2_iterEnum_iter`). The (i, j)-fibre of a doubly iterated enumerated neighbourhood is the original neighbourhood indexed first by its outer code at i, then by that inner code at j.

Proof. Apply the fibre equation for the iterate enumeration twice. ■

Theorem (`EffectiveIso.Gmap_rel_enum_iff`). On enumerated neighbourhoods, the backward-map relation is equivalent to finitely many inclusions from decoded double fibres into target fibres below the target code.

Proof. The general fibrewise relation gives these inclusions; beyond the target bound the target fibre is master, so no further test is needed. ■

Theorem (`EffectiveIso.Gmap_isComputable`). The backward reindexing map is computable for the iterate presentations.

Proof. The finite characterization uses primitive-recursive nested index lookup and decidable inclusion, followed by bounded universal quantification. ■

Theorem (`EffectiveIso.masterIdx_iterPresentation`). The master index of an iterate presentation is zero.

Proof. This is the chosen master code in the construction. ■

Theorem (`EffectiveIso.Fmap_rel_enum_iff`). On enumerated neighbourhoods, the forward-map relation is equivalent to the finitely bounded family of inclusions from paired source fibres into nested target fibres.

Proof. The general fibrewise relation gives the bounded conditions; outside the outer or inner code bounds the target fibre is master. ■

Theorem (`EffectiveIso.Fmap_isComputable`). The forward reindexing map is computable for the iterate presentations.

Proof. Its finite characterization consists of primitive-recursive nested lookups, decidable inclusion, and two bounded universal quantifiers. ■

Definition (`EffectiveIso.iterIterEffectiveIso`). The forward and backward reindexing maps, with their computability and inverse laws, form an effective isomorphism between an iterate and its double iterate.

Definition (`EffectiveIso.toDomainIso`). An effective isomorphism induces an order isomorphism of element domains by applying its two approximable maps to elements.

Theorem (`EffectivelyIsomorphic.isomorphic`). Effectively isomorphic presentations have isomorphic element domains.

Proof. Choose an effective isomorphism and pass to its induced domain isomorphism. ■

Theorem (`iterSys_effectivelyIsomorphic_iterIter`). Every presented iterate is effectively isomorphic to its double iterate.

Proof. Use the effective isomorphism supplied by forward and backward reindexing. ■

Theorem (`iterSys_isomorphic_iterIter`). An iterate neighbourhood system is isomorphic to its double iterate.

Proof. Forget the computability data from the effective reindexing isomorphism. ■

`effectiveIso.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12
```

```
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_15
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_24
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .regularCode
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .regularLanguage

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap

section
variable {α β : Type*}
variable {V : NeighborhoodSystem α}

structure EffectiveIso {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
    (P : ComputablePresentation V) (Q : ComputablePresentation W) where
  toMap : ApproximableMap V W
  invMap : ApproximableMap W V
  toMap_computable : ComputableMap P Q toMap
  invMap_computable : ComputableMap Q P invMap
  left_inv : invMap.comp toMap = idMap V
  right_inv : toMap.comp invMap = idMap W

def EffectivelyIsomorphic {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
    (P : ComputablePresentation V) (Q : ComputablePresentation W) : Prop :=
  Nonempty (EffectiveIso P Q)
end

namespace EffectiveIso

def fiber2 (S : Set (ℕ × (ℕ × α))) (i j : ℕ) : Set α := fiber (fiber S i) j

@[simp] theorem mem_fiber2 {S : Set (ℕ × (ℕ × α))} {i j : ℕ} {a : α} :
    a ∈ fiber2 S i j ↔ (i, (j, a)) ∈ S := Iff.rfl

theorem fiber2_inter (S S' : Set (ℕ × (ℕ × α))) (i j : ℕ) :
    fiber2 (S ∩ S') i j = fiber2 S i j ∩ fiber2 S' i j := rfl

theorem fiber2_master (i j : ℕ) :
    fiber2 ((iterSys (iterSys V)).master) i j = V.master := by
  show fiber (fiber ((iterSys (iterSys V)).master) i) j = V.master
  rw [fiber_iterSys_master, fiber_iterSys_master]

theorem fiber2_mono {S S' : Set (ℕ × (ℕ × α))} (h : S ⊆ S') (i j : ℕ) :
    fiber2 S i j ⊆ fiber2 S' i j := fun _ ha => h ha

theorem mem_fiber2_of_mem {S : Set (ℕ × (ℕ × α))} (hS : (iterSys (iterSys V)).mem S)
    (i j : ℕ) :
    V.mem (fiber2 S i j) := (hS.1 i).1 j

end EffectiveIso
```

```
namespace EffectiveIso
open NeighborhoodSystem Domain.Recursive ApproximableMap

section
variable {α β : Type*}
variable {V : NeighborhoodSystem α}

def reindexF (W : Set (ℕ × α)) : Set (ℕ × (ℕ × α)) :=
  {p | p.2.2 ∈ fiber W (Nat.pair p.1 p.2.1)}

@[simp] theorem fiber2_reindexF (W : Set (ℕ × α)) (i j : ℕ) :
    fiber2 (reindexF W) i j = fiber W (Nat.pair i j) := rfl

def reindexG (S : Set (ℕ × (ℕ × α))) : Set (ℕ × α) :=
  {p | p.2 ∈ fiber2 S (Nat.unpair p.1).1 (Nat.unpair p.1).2}

@[simp] theorem fiber_reindexG (S : Set (ℕ × (ℕ × α))) (k : ℕ) :
    fiber (reindexG S) k = fiber2 S (Nat.unpair k).1 (Nat.unpair k).2 := rfl

theorem exists_inner_bound {S : Set (ℕ × (ℕ × α))} (hS : (iterSys (iterSys V)).mem S)
    :
    ∀ n, ∃ M, ∀ i, i < n → ∀ j, M ≤ j → fiber2 S i j = V.master := by
  intro n
  induction n with
  | zero => exact ⟨0, fun i hi => absurd hi (Nat.not_lt_zero i)⟩
  | succ n ih =>
    obtain ⟨M, hM⟩ := ih
    obtain ⟨Mn, hMn⟩ := (hS.1 n).2
    refine ⟨max M Mn, fun i hi j hj => ?_⟩
    rcases Nat.lt_succ_iff_lt_or_eq.mp hi with hlt | heq
    · exact hM i hlt j (le_trans (le_max_left _ _) hj)
    · subst heq
      exact hMn j (le_trans (le_max_right _ _) hj)

theorem pair_lt_pair_of_lt {x y i j : ℕ} (hx : x < i) (hy : y < j) :
    Nat.pair x y < Nat.pair i j := by
  have hms : ∀ z : ℕ, (z + 1) * (z + 1) = z * z + 2 * z + 1 := fun z => by
    rw [Nat.succ_mul, Nat.mul_succ]; omega
  have hlt : Nat.pair x y < (max x y + 1) * (max x y + 1) := by
    unfold Nat.pair
    rcases lt_or_ge x y with h | h
    · rw [ite_eq_left h, show max x y = y by omega, hms]; omega
    · rw [ite_eq_right (by omega), show max x y = x by omega, hms]; omega
  have hle : max i j * max i j ≤ Nat.pair i j := by
    unfold Nat.pair
    rcases lt_or_ge i j with h | h
    · rw [ite_eq_left h, show max i j = j by omega]; omega
    · rw [ite_eq_right (by omega), show max i j = i by omega]; omega
  have h3 : max x y + 1 ≤ max i j := by
    rcases le_total x y with h | h
    · rw [max_eq_right h]; have := le_max_right i j; omega
    · rw [max_eq_left h]; have := le_max_left i j; omega
  have h4 : (max x y + 1) * (max x y + 1) ≤ max i j * max i j := Nat.mul_le_mul h3 h3
  omega
```

```
theorem reindexF_mem {W : Set (ℕ × α)} (hW : (iterSys V).mem W) :
    (iterSys (iterSys V)).mem (reindexF W) := by
  obtain ⟨hWf, Nw, hNw⟩ := hW
  refine ⟨fun i => ⟨fun j => ?_, Nw, fun j hj => ?_⟩, Nw, fun i hi => ?_⟩
  ·
    show V.mem (fiber2 (reindexF W) i j)
    rw [fiber2_reindexF]; exact hWf (Nat.pair i j)
  ·
    show fiber2 (reindexF W) i j = V.master
    rw [fiber2_reindexF]
    exact hNw (Nat.pair i j) (le_trans hj (le_pair_right i j))
  ·
    apply eq_of_fiber_eq
    intro j
    show fiber2 (reindexF W) i j = fiber ((iterSys V).master) j
    rw [fiber2_reindexF, fiber_iterSys_master]
    exact hNw (Nat.pair i j) (le_trans hi (le_pair_left i j))

theorem reindexF_subset_iff {W : Set (ℕ × α)} {S : Set (ℕ × (ℕ × α))} :
    reindexF W ⊆ S ↔ ∀ i j, fiber W (Nat.pair i j) ⊆ fiber2 S i j := by
  constructor
  · intro h i j
    have := fiber2_mono h i j
    rwa [fiber2_reindexF] at this
  · intro h
    refine subset_of_fiber_subset (fun i => subset_of_fiber_subset (fun j => ?_))
    show fiber2 (reindexF W) i j ⊆ fiber2 S i j
    rw [fiber2_reindexF]; exact h i j

def Fmap (V : NeighborhoodSystem α) : ApproximableMap (iterSys V) (iterSys (iterSys V)
    ) where
  rel W S := (iterSys V).mem W ∧ (iterSys (iterSys V)).mem S ∧
    ∀ i j, fiber W (Nat.pair i j) ⊆ fiber2 S i j
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(iterSys V).master_mem, (iterSys (iterSys V)).master_mem, fun i j =>
    by
    rw [fiber_iterSys_master, fiber2_master]⟩
  inter_right := by
    rintro W S S' ⟨hW, hS, hf⟩ ⟨-, hS', hf'⟩
    refine ⟨hW, (iterSys (iterSys V)).inter_mem hS hS' (reindexF_mem hW) ?_, fun i j =>
     ?_⟩
    · exact Set.subset_inter (reindexF_subset_iff.mpr hf) (reindexF_subset_iff.mpr hf')
    · rw [fiber2_inter]; exact Set.subset_inter (hf i j) (hf' i j)
  mono := by
    rintro W W' S S' ⟨hW, hS, hf⟩ hW'W hSS' hW' hS'
    exact ⟨hW', hS', fun i j => ((fiber_mono hW'W _).trans (hf i j)).trans (fiber2_mono
     hSS' i j)⟩

@[simp] theorem Fmap_rel {W : Set (ℕ × α)} {S : Set (ℕ × (ℕ × α))} :
    (Fmap V).rel W S ↔ (iterSys V).mem W ∧ (iterSys (iterSys V)).mem S ∧
      ∀ i j, fiber W (Nat.pair i j) ⊆ fiber2 S i j := Iff.rfl

theorem reindexG_mem {S : Set (ℕ × (ℕ × α))} (hS : (iterSys (iterSys V)).mem S) :
    (iterSys V).mem (reindexG S) := by
  obtain ⟨No, hNo⟩ := hS.2
```

```
  obtain ⟨M, hM⟩ := exists_inner_bound hS No
  refine ⟨fun k => ?_, Nat.pair No M, fun k hk => ?_⟩
  ·
    rw [fiber_reindexG]; exact mem_fiber2_of_mem hS _ _
  ·
    rw [fiber_reindexG]
    set i := (Nat.unpair k).1 with hi
    set j := (Nat.unpair k).2 with hj
    by_cases hiN : i < No
    · by_cases hjM : j < M
      · exfalso
        have hklt : Nat.pair i j < Nat.pair No M := pair_lt_pair_of_lt hiN hjM
        rw [pair_unpair] at hklt
        omega
      · exact hM i hiN j (not_lt.mp hjM)
    · have h1 : fiber S i = (iterSys V).master := hNo i (not_lt.mp hiN)
      show fiber2 S i j = V.master
      rw [fiber2, h1, fiber_iterSys_master]

theorem reindexG_subset_iff {S : Set (ℕ × (ℕ × α))} {W : Set (ℕ × α)} :
    reindexG S ⊆ W ↔ ∀ k, fiber2 S (Nat.unpair k).1 (Nat.unpair k).2 ⊆ fiber W k :=
    by
  constructor
  · intro h k
    have := fiber_mono h k
    rwa [fiber_reindexG] at this
  · intro h
    refine subset_of_fiber_subset (fun k => ?_)
    rw [fiber_reindexG]; exact h k

def Gmap (V : NeighborhoodSystem α) : ApproximableMap (iterSys (iterSys V)) (iterSys V
    ) where
  rel S W := (iterSys (iterSys V)).mem S ∧ (iterSys V).mem W ∧
    ∀ k, fiber2 S (Nat.unpair k).1 (Nat.unpair k).2 ⊆ fiber W k
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨(iterSys (iterSys V)).master_mem, (iterSys V).master_mem, fun k => by
    rw [fiber2_master, fiber_iterSys_master]⟩
  inter_right := by
    rintro S W W' ⟨hS, hW, hg⟩ ⟨-, hW', hg'⟩
    refine ⟨hS, (iterSys V).inter_mem hW hW' (reindexG_mem hS) ?_, fun k => ?_⟩
    · exact Set.subset_inter (reindexG_subset_iff.mpr hg) (reindexG_subset_iff.mpr hg')
    · rw [fiber_inter]; exact Set.subset_inter (hg k) (hg' k)
  mono := by
    rintro S S' W W' ⟨hS, hW, hg⟩ hS'S hWW' hS' hW'
    exact ⟨hS', hW', fun k => ((fiber2_mono hS'S _ _).trans (hg k)).trans (fiber_mono
    hWW' k)⟩

@[simp] theorem Gmap_rel {S : Set (ℕ × (ℕ × α))} {W : Set (ℕ × α)} :
    (Gmap V).rel S W ↔ (iterSys (iterSys V)).mem S ∧ (iterSys V).mem W ∧
      ∀ k, fiber2 S (Nat.unpair k).1 (Nat.unpair k).2 ⊆ fiber W k := Iff.rfl

theorem Fmap_comp_Gmap : (Fmap V).comp (Gmap V) = idMap (iterSys (iterSys V)) := by
  apply ApproximableMap.ext
  intro S S'
  rw [comp_rel, idMap_rel]
```

```
  constructor
  · rintro ⟨W, ⟨hS, hW, hg⟩, ⟨-, hS', hf⟩⟩
    refine ⟨hS, hS', subset_of_fiber_subset (fun i => subset_of_fiber_subset (fun j =>
    ?_))⟩
    show fiber2 S i j ⊆ fiber2 S' i j
    have h1 : fiber2 S i j ⊆ fiber W (Nat.pair i j) := by
      have := hg (Nat.pair i j)
      rwa [unpair_pair_fst, unpair_pair_snd] at this
    exact h1.trans (hf i j)
  · rintro ⟨hS, hS', hSS'⟩
    refine ⟨reindexG S, ⟨hS, reindexG_mem hS, ?_⟩, ⟨reindexG_mem hS, hS', ?_⟩⟩
    · intro k; rw [fiber_reindexG]
    · intro i j
      rw [fiber_reindexG, unpair_pair_fst, unpair_pair_snd]
      exact fiber2_mono hSS' i j

theorem Gmap_comp_Fmap : (Gmap V).comp (Fmap V) = idMap (iterSys V) := by
  apply ApproximableMap.ext
  intro W W'
  rw [comp_rel, idMap_rel]
  constructor
  · rintro ⟨S, ⟨hW, hS, hf⟩, ⟨-, hW', hg⟩⟩
    refine ⟨hW, hW', subset_of_fiber_subset (fun k => ?_)⟩
    have h1 : fiber W k ⊆ fiber2 S (Nat.unpair k).1 (Nat.unpair k).2 := by
      have := hf (Nat.unpair k).1 (Nat.unpair k).2
      rwa [pair_unpair] at this
    exact h1.trans (hg k)
  · rintro ⟨hW, hW', hWW'⟩
    refine ⟨reindexF W, ⟨hW, reindexF_mem hW, ?_⟩, ⟨reindexF_mem hW, hW', ?_⟩⟩
    · intro i j; rw [fiber2_reindexF]
    · intro k
      rw [fiber2_reindexF, pair_unpair]
      exact fiber_mono hWW' k

end

section
variable {α β : Type*}
variable {V : NeighborhoodSystem α}
variable (P : ComputablePresentation V)

theorem fiber2_iterEnum_iter (m i j : ℕ) :
    fiber2 (iterEnum (iterPresentation P) m) i j
      = P.X (iterIdx P (iterIdx (iterPresentation P) m i) j) := by
  show fiber (fiber (iterEnum (iterPresentation P) m) i) j = _
  rw [fiber_iterEnum, iterPresentation_X, fiber_iterEnum]
end

section
variable {α β : Type*}
variable {V : NeighborhoodSystem α}
variable (P : ComputablePresentation V)
variable {P}

theorem Gmap_rel_enum_iff (s n : ℕ) :
    (Gmap V).rel (iterEnum (iterPresentation P) s) (iterEnum P n) ↔
```

```
      ∀ k, k < n →
        P.X (iterIdx P (iterIdx (iterPresentation P) s (Nat.unpair k).1) (Nat.unpair k
    ).2)
          ⊆ P.X (iterIdx P n k) := by
  rw [Gmap_rel]
  have hmem₁ := iterEnum_mem (P := iterPresentation P) s
  have hmem₂ := iterEnum_mem (P := P) n
  constructor
  · rintro ⟨-, -, hg⟩ k _
    have := hg k
    rwa [fiber2_iterEnum_iter, fiber_iterEnum] at this
  · intro hbd
    refine ⟨hmem₁, hmem₂, fun k => ?_⟩
    rw [fiber2_iterEnum_iter, fiber_iterEnum]
    by_cases hk : k < n
    · exact hbd k hk
    · rw [iterIdx_ge (not_lt.mp hk), P.masterIdx_spec]
      exact V.sub_master (P.mem_X _)

theorem Gmap_isComputable :
    ComputableMap (iterPresentation (iterPresentation P)) (iterPresentation P) (Gmap V
    ) := by
  constructor

  have hLHS : Nat.Primrec (fun u : ℕ =>
      iterIdx P (iterIdx (iterPresentation P) u.unpair.2.unpair.1 u.unpair.1.unpair.1)
        u.unpair.1.unpair.2) := by
    have hinner : Nat.Primrec (fun u : ℕ =>
        iterIdx (iterPresentation P) u.unpair.2.unpair.1 u.unpair.1.unpair.1) := by
      refine (primrec_nthCode.comp
        ((Nat.Primrec.left.comp Nat.Primrec.right).pair
          ((Nat.Primrec.left.comp Nat.Primrec.left).pair
            (Nat.Primrec.const (iterPresentation P).masterIdx)))).of_eq (fun u => ?_)
      simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
    refine (primrec_nthCode.comp
      (hinner.pair ((Nat.Primrec.right.comp Nat.Primrec.left).pair
        (Nat.Primrec.const P.masterIdx)))).of_eq (fun u => ?_)
    simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
  have hRHS : Nat.Primrec (fun u : ℕ =>
      iterIdx P u.unpair.2.unpair.2 u.unpair.1) := by
    refine (primrec_nthCode.comp
      ((Nat.Primrec.right.comp Nat.Primrec.right).pair
        (Nat.Primrec.left.pair (Nat.Primrec.const P.masterIdx)))).of_eq (fun u => ?_)
    simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
  have hp₂ : RecDecidable (fun u : ℕ =>
      P.X (iterIdx P (iterIdx (iterPresentation P) u.unpair.2.unpair.1 u.unpair.1.
    unpair.1)
        u.unpair.1.unpair.2) ⊆ P.X (iterIdx P u.unpair.2.unpair.2 u.unpair.1)) := by
    refine RecDecidable.of_iff (fun u => ?_) (P.incl_computable.comp (hLHS.pair hRHS))
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hbound₁ : Nat.Primrec (fun t : ℕ => t.unpair.2) := Nat.Primrec.right
  have hp₀ := hp₂.bForall hbound₁
  refine (RecDecidable.of_iff (fun t => ?_) hp₀).re
  simp only [iterPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact Gmap_rel_enum_iff t.unpair.1 t.unpair.2
```

```
end

section
variable {α β : Type*}
variable {V : NeighborhoodSystem α}
variable (P : ComputablePresentation V)

theorem masterIdx_iterPresentation : (iterPresentation P).masterIdx = 0 := rfl
end

section
variable {α β : Type*}
variable {V : NeighborhoodSystem α}
variable (P : ComputablePresentation V)
variable {P}

theorem Fmap_rel_enum_iff (n m : ℕ) :
    (Fmap V).rel (iterEnum P n) (iterEnum (iterPresentation P) m) ↔
      ∀ i, i < m → ∀ j, j < iterIdx (iterPresentation P) m i →
        P.X (iterIdx P n (Nat.pair i j)) ⊆ P.X (iterIdx P (iterIdx (iterPresentation P
    ) m i) j) := by
  rw [Fmap_rel]
  have hmem₁ := iterEnum_mem (P := P) n
  have hmem₂ := iterEnum_mem (P := iterPresentation P) m
  constructor
  · rintro ⟨-, -, hf⟩ i _ j _
    have := hf i j
    rwa [fiber_iterEnum, fiber2_iterEnum_iter] at this
  · intro hbd
    refine ⟨hmem₁, hmem₂, fun i j => ?_⟩
    rw [fiber_iterEnum, fiber2_iterEnum_iter]
    by_cases hi : i < m
    · by_cases hj : j < iterIdx (iterPresentation P) m i
      · exact hbd i hi j hj
      ·
        rw [iterIdx_ge (not_lt.mp hj), P.masterIdx_spec]
        exact V.sub_master (P.mem_X _)
    ·
      have h0 : iterIdx (iterPresentation P) m i = 0 := by
        rw [iterIdx_ge (not_lt.mp hi), masterIdx_iterPresentation]
      rw [h0, iterIdx_ge (Nat.zero_le j), P.masterIdx_spec]
      exact V.sub_master (P.mem_X _)

theorem Fmap_isComputable :
    ComputableMap (iterPresentation P) (iterPresentation (iterPresentation P)) (Fmap V
    ) := by
  constructor

  have hLHS : Nat.Primrec (fun u : ℕ =>
      iterIdx P u.unpair.2.unpair.2.unpair.1 (Nat.pair u.unpair.2.unpair.1 u.unpair.1)
    ) := by
    refine (primrec_nthCode.comp
      ((Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)).pair
        (((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left).pair
          (Nat.Primrec.const P.masterIdx)))).of_eq (fun u => ?_)
    simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
```

```
  have hRHS : Nat.Primrec (fun u : ℕ =>
      iterIdx P (iterIdx (iterPresentation P) u.unpair.2.unpair.2.unpair.2 u.unpair.2.
    unpair.1)
        u.unpair.1) := by
    have hinner : Nat.Primrec (fun u : ℕ =>
        iterIdx (iterPresentation P) u.unpair.2.unpair.2.unpair.2 u.unpair.2.unpair.1)
     := by
      refine (primrec_nthCode.comp
        ((Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)).pair
          ((Nat.Primrec.left.comp Nat.Primrec.right).pair
            (Nat.Primrec.const (iterPresentation P).masterIdx)))).of_eq (fun u => ?_)
      simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
    refine (primrec_nthCode.comp
      (hinner.pair (Nat.Primrec.left.pair (Nat.Primrec.const P.masterIdx)))).of_eq (
    fun u => ?_)
    simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
  have hp₂ : RecDecidable (fun u : ℕ =>
      P.X (iterIdx P u.unpair.2.unpair.2.unpair.1 (Nat.pair u.unpair.2.unpair.1 u.
    unpair.1))
        ⊆ P.X (iterIdx P (iterIdx (iterPresentation P) u.unpair.2.unpair.2.unpair.2
          u.unpair.2.unpair.1) u.unpair.1)) := by
    refine RecDecidable.of_iff (fun u => ?_) (P.incl_computable.comp (hLHS.pair hRHS))
    simp only [unpair_pair_fst, unpair_pair_snd]

  have hbound₂ : Nat.Primrec (fun v : ℕ =>
      iterIdx (iterPresentation P) v.unpair.2.unpair.2 v.unpair.1) := by
    refine (primrec_nthCode.comp
      ((Nat.Primrec.right.comp Nat.Primrec.right).pair
        (Nat.Primrec.left.pair (Nat.Primrec.const (iterPresentation P).masterIdx)))).
    of_eq

      (fun v => ?_)
    simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
  have hbound₁ : Nat.Primrec (fun t : ℕ => t.unpair.2) := Nat.Primrec.right
  have hp₁ := hp₂.bForall hbound₂
  have hp₀ := hp₁.bForall hbound₁
  refine (RecDecidable.of_iff (fun t => ?_) hp₀).re

  simp only [iterPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact Fmap_rel_enum_iff t.unpair.1 t.unpair.2

def iterIterEffectiveIso (P : ComputablePresentation V) :
    EffectiveIso (iterPresentation P) (iterPresentation (iterPresentation P)) where
  toMap := Fmap V
  invMap := Gmap V
  toMap_computable := Fmap_isComputable
  invMap_computable := Gmap_isComputable
  left_inv := Gmap_comp_Fmap
  right_inv := Fmap_comp_Gmap

end

end EffectiveIso

namespace EffectiveIso
open NeighborhoodSystem Domain.Recursive ApproximableMap
```

```
section
variable {α β : Type*}
variable {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
  {P : ComputablePresentation V} {Q : ComputablePresentation W}

def toDomainIso (e : EffectiveIso P Q) : DomainIso V W where
  toFun := e.toMap.toElementMap
  invFun := e.invMap.toElementMap
  left_inv x := by
    rw [← toElementMap_comp, e.left_inv, toElementMap_idMap]
  right_inv y := by
    rw [← toElementMap_comp, e.right_inv, toElementMap_idMap]
  map_rel_iff' := by
    intro x y
    show e.toMap.toElementMap x ≤ e.toMap.toElementMap y ↔ x ≤ y
    refine ⟨fun h => ?_, fun h => e.toMap.toElementMap_mono h⟩
    have hmono := e.invMap.toElementMap_mono h
    rwa [← toElementMap_comp, ← toElementMap_comp, e.left_inv, toElementMap_idMap,
      toElementMap_idMap] at hmono

end
end EffectiveIso

section
variable {α β : Type*}

theorem EffectivelyIsomorphic.isomorphic {V : NeighborhoodSystem α} {W :
    NeighborhoodSystem β}
    {P : ComputablePresentation V} {Q : ComputablePresentation W}
    (h : EffectivelyIsomorphic P Q) : V ≅? W :=
  h.elim fun e => ⟨e.toDomainIso⟩

variable {V : NeighborhoodSystem α}

theorem iterSys_effectivelyIsomorphic_iterIter (P : ComputablePresentation V) :
    EffectivelyIsomorphic (iterPresentation P) (iterPresentation (iterPresentation P))
     :=
  ⟨EffectiveIso.iterIterEffectiveIso P⟩

theorem iterSys_isomorphic_iterIter (P : ComputablePresentation V) :
    iterSys V ≅? iterSys (iterSys V) :=
  (iterSys_effectivelyIsomorphic_iterIter P).isomorphic
end

end DomainTheory.NeighbourhoodSystems
```

4.8.45 Exercise 7.18 — effective isomorphism

Two effectively given systems D and E are *effectively isomorphic* iff … (complete the sentence!). Show that if D is effectively given then the isomorphism

$$D^\infty \cong (D^\infty)^\infty$$

is effective.

The development is with the effective isomorphism peer.

Theorem (`example_718`). An iterate is effectively isomorphic to the iterate of that iterate.

Proof. By the effective isomorphism. ■

`Exercise_7_18.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .effectiveIso

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem Domain.Recursive ApproximableMap
open DomainTheory.NeighbourhoodSystems.EffectiveIso
/-- Scott's Exercise 7.18: iterate is effectively isomorphic to its iterate. -/
theorem example_718 :
    ∀ {α : Type*} {V : NeighborhoodSystem α} (P : ComputablePresentation V),
      EffectivelyIsomorphic (iterPresentation P) (iterPresentation (iterPresentation P
    )) :=
  @iterSys_effectivelyIsomorphic_iterIter
```

4.8.46 Powerdomain functor

The powerdomain construction sends an approximable map to an approximable map, preserving identity and composition, and the result is computable.

Definition (`PFmap`). For an approximable map from one neighbourhood system to another, define its powerdomain action by relating two finite upper families when every source neighbourhood has a related target neighbourhood in the target family.

Theorem (`PFmap_rel`). Two powerdomain neighbourhoods are related by the lifted map exactly when they are valid powerdomain neighbourhoods and every member of the first has a related member of the second.

Proof. This is exactly the relation used to define the powerdomain action. ■

Definition (`PFmapComb`). Define the higher-order powerdomain operator that sends each neighbourhood of approximable maps to a neighbourhood containing every corresponding lifted map.

Theorem (`PFmapComb_rel`). A function-space neighbourhood and a lifted function-space neighbourhood are related exactly when every map in the former has its powerdomain lifting in the latter.

Proof. The equivalence is the defining condition of the higher-order relation. ■

Theorem (`PDmem_mem`). Every member of a powerdomain neighbourhood is itself a neighbourhood of the underlying system.

Proof. A powerdomain neighbourhood is a finite union of principal upper families generated by neighbourhoods. Membership therefore supplies a generator contained in the given member, and upward closure makes that member a neighbourhood. ■

Theorem (`PFmap_idMap`). Lifting the identity approximable map to powerdomains gives the identity map on the powerdomain.

Proof. If every source member is related by the identity to a target member, then each source member lies in the target family by downward closure. Conversely, inclusion of the source family in the target family

lets each member witness itself. ■

Theorem (`PFmap_rel_Ypd_iff`). For presented powerdomain neighbourhoods, the lifted relation holds exactly when every decoded source index has a decoded target index whose basic neighbourhoods are related.

Proof. In the forward direction, apply the lifted relation to each basic source neighbourhood and choose a basic target neighbourhood below the resulting member. In the reverse direction, choose basic generators below arbitrary family members and use monotonicity of the original map. ■

Theorem (`PFmap_rel_fin`). For finite upper families generated by two lists, the lifted relation holds exactly when every generator in the source list has a related generator in the target list.

Proof. Applying the lifted relation to a source generator yields a target-family member, hence a target generator below it; codomain monotonicity gives the required generator relation. Conversely, choose the stipulated target generator below each arbitrary source-family member and use domain monotonicity. ■

Theorem (`bExists_decodeList_re`). If a binary relation is recursively enumerable, then the relation saying that some member of a decoded finite list is related to a given number is recursively enumerable.

Proof. Membership in a decoded finite list is decidable. Conjoining it with the given enumerable relation and projecting away the candidate witness preserves recursive enumerability. ■

Theorem (`PFmap_isComputable_Ypd`). On the canonical enumerations of powerdomain neighbourhoods, the relation induced by the lifted map is recursively enumerable.

Proof. The finite-code characterization reduces the relation to a bounded universal quantifier over source indices followed by a bounded existential quantifier over target indices. Recursive enumerability is preserved by both finite quantifiers and by exchanging the paired inputs. ■

Theorem (`PFmap_isComputable`). The powerdomain lifting of a computable approximable map is computable for the induced computable powerdomain presentations.

Proof. Computability for these presentations is precisely recursive enumerability on their canonical coded neighbourhoods, supplied by the preceding theorem. ■

Theorem (`comp_witness`). Given finitely many source neighbourhoods whose composites reach a target family, one can choose a finite intermediate family witnessing both stages of the composite.

Proof. Induct on the source list. The empty list uses no intermediates. For a new head, choose one of its intermediate witnesses, prepend it to the list obtained for the tail, and retain the corresponding witnesses for both stages. ■

Theorem (`PFmap_comp`). Powerdomain lifting preserves composition of approximable maps.

Proof. For the forward implication, express the source family by finitely many generators and collect finitely many intermediate witnesses; their generated upper family witnesses the composite of the two lifted maps. For the reverse implication, compose the two witnesses chosen successively for each source member. ■

Theorem (`gather_witnesses`). Finite pointwise existential witnesses can be collected into a finite list of related source–target pairs, with at least one pair for each source.

Proof. Induct on the source list, choosing one target witness for the head and adjoining that pair to the collection furnished for the tail. ■

Theorem (`exists_stepFun_forall_PFmap_rel`). Any finite family of lifted constraints satisfied by a map is forced by finitely many ordinary step constraints satisfied by that map.

Proof. For one lifted constraint, write its two powerdomain neighbourhoods using finite generators and collect one related pair for each source generator. For a finite list of constraints, append these pairs to the pairs obtained inductively; any map satisfying all collected pairs satisfies each lifted constraint by the finite-generator characterization. ■

Theorem (`PFmapComb_toElementMap`). On elements, the higher-order lifting operator sends the filter of a map to the filter of its powerdomain lifting.

Proof. One inclusion applies the defining universal condition to the given map. For the other, each finite neighbourhood of the lifted map is forced by finitely many step constraints; the function-space neighbourhood generated by those constraints contains the original map and is sent inside the chosen lifted neighbourhood. ■

`powerFunctor.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_10
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_11
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.funSpace
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.funSpaceEquiv
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.idMap

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*}
variable {V : NeighborhoodSystem α} {W : NeighborhoodSystem β} {U : NeighborhoodSystem
     ?}

def PFmap (f : ApproximableMap V W) : ApproximableMap V.PowerDomain W.PowerDomain
    where
  rel A B := V.PDmem A ∧ W.PDmem B ∧ ∀ X ∈ A, ∃ Y ∈ B, f.rel X Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine ⟨V.PDmem_master, W.PDmem_master, ?_⟩
    rintro X ⟨hXmem, hXsub⟩
    exact ⟨W.master, ⟨W.master_mem, subset_rfl⟩,
      f.mono f.master_rel hXsub subset_rfl hXmem W.master_mem⟩
  inter_right := by
    rintro A B B' ⟨hA, hB, hbody⟩ ⟨-, hB', hbody'⟩
    refine ⟨hA, W.PDmem_inter hB hB', ?_⟩
    intro X hX
    obtain ⟨Y, hYB, hrel⟩ := hbody X hX
    obtain ⟨Y', hY'B', hrel'⟩ := hbody' X hX
    have hinmem : W.mem (Y ∩ Y') := f.rel_cod (f.inter_right hrel hrel')
    exact ⟨Y ∩ Y', Set.mem_inter
      (W.PDmem_down hB hYB Set.inter_subset_left hinmem)
      (W.PDmem_down hB' hY'B' Set.inter_subset_right hinmem),
      f.inter_right hrel hrel'⟩
  mono := by
    rintro A A' B B' ⟨_, _, hbody⟩ hA'A hBB' hA' hB'
    refine ⟨hA', hB', ?_⟩
    intro X hX
    obtain ⟨Y, hYB, hrel⟩ := hbody X (hA'A hX)
```

```
    exact ⟨Y, hBB' hYB, hrel⟩

@[simp] theorem PFmap_rel {f : ApproximableMap V W} {A : Set (Set α)} {B : Set (Set β)
    } :
    (PFmap f).rel A B ↔ V.PDmem A ∧ W.PDmem B ∧ ∀ X ∈ A, ∃ Y ∈ B, f.rel X Y := Iff.
    rfl

def PFmapComb (V : NeighborhoodSystem α) (W : NeighborhoodSystem β) :
    ApproximableMap (funSpace V W) (funSpace V.PowerDomain W.PowerDomain) where
  rel ? ? := (funSpace V W).mem ? ∧ (funSpace V.PowerDomain W.PowerDomain).mem ? ∧
    ∀ f ∈ ?, PFmap f ∈ ?
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel :=
    ⟨(funSpace V W).master_mem, (funSpace V.PowerDomain W.PowerDomain).master_mem,
      fun _ _ => Set.mem_univ _⟩
  inter_right := by
    rintro ? ? ?' ⟨h?, h?, hbody⟩ ⟨-, h?', hbody'⟩
    obtain ⟨f₀, hf₀⟩ := h?.2
    obtain ⟨⟨L₂, hL₂, rfl⟩, -⟩ := h?
    obtain ⟨⟨L₂', hL₂', rfl⟩, -⟩ := h?'
    refine ⟨h?, ⟨⟨L₂ ++ L₂', ?_, (stepFun_append _ _).symm⟩,
        ⟨PFmap f₀, Set.mem_inter (hbody f₀ hf₀) (hbody' f₀ hf₀)⟩⟩,
      fun f hf => Set.mem_inter (hbody f hf) (hbody' f hf)⟩
    intro q hq
    rcases List.mem_append.mp hq with hq' | hq'
    · exact hL₂ q hq'
    · exact hL₂' q hq'
  mono := by
    rintro ? ?' ? ?' ⟨_, _, hbody⟩ h?'? h??' h?' h?'
    exact ⟨h?', h?', fun f hf => h??' (hbody f (h?'? hf))⟩

end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*}

namespace NeighborhoodSystem

variable (V : NeighborhoodSystem α)

end NeighborhoodSystem

variable {V : NeighborhoodSystem α} {W : NeighborhoodSystem β} {U : NeighborhoodSystem
    ?}

@[simp] theorem PFmapComb_rel {? : Set (ApproximableMap V W)}
    {? : Set (ApproximableMap V.PowerDomain W.PowerDomain)} :
    (PFmapComb V W).rel ? ? ↔
      (funSpace V W).mem ? ∧ (funSpace V.PowerDomain W.PowerDomain).mem ? ∧
        ∀ f ∈ ?, PFmap f ∈ ? := Iff.rfl
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
```

```
variable {α β ? : Type*}

namespace NeighborhoodSystem

variable (V : NeighborhoodSystem α)

theorem PDmem_mem {A : Set (Set α)} (hA : V.PDmem A) {X : Set α} (hX : X ∈ A) : V.mem
    X := by
  obtain ⟨L, hL, rfl⟩ := hA
  obtain ⟨X', _, hXmem, _⟩ := (V.mem_PDunion).mp hX
  exact hXmem

end NeighborhoodSystem
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β ? : Type*}
variable {V : NeighborhoodSystem α} {W : NeighborhoodSystem β} {U : NeighborhoodSystem
     ?}

theorem PFmap_idMap (V : NeighborhoodSystem α) : PFmap (idMap V) = idMap V.PowerDomain
     := by
  apply ApproximableMap.ext
  intro A B
  simp only [PFmap_rel, idMap_rel]
  constructor
  · rintro ⟨hA, hB, hbody⟩
    refine ⟨hA, hB, ?_⟩
    intro X hX
    obtain ⟨Y, hYB, hXmem, _, hXY⟩ := hbody X hX
    exact V.PDmem_down hB hYB hXY hXmem
  · rintro ⟨hA, hB, hsub⟩
    refine ⟨hA, hB, ?_⟩
    intro X hX
    exact ⟨X, hsub hX, V.PDmem_mem hA hX, V.PDmem_mem hA hX, subset_rfl⟩

theorem PFmap_rel_Ypd_iff (P : ComputablePresentation V) (Q : ComputablePresentation W
    )
    (f : ApproximableMap V W) {c d : ℕ} :
    (PFmap f).rel (V.Ypd P c) (W.Ypd Q d) ↔
      ∀ a ∈ decodeList c, ∃ b ∈ decodeList d, f.rel (P.X a) (Q.X b) := by
  constructor
  · rintro ⟨-, -, hbody⟩ a ha
    have hXin : P.X a ∈ V.Ypd P c := (V.mem_Ypd P).mpr ⟨a, ha, P.mem_X a, subset_rfl⟩
    obtain ⟨Y, hYin, hrel⟩ := hbody (P.X a) hXin
    obtain ⟨b, hb, hYmem, hYsub⟩ := (W.mem_Ypd Q).mp hYin
    exact ⟨b, hb, f.mono hrel subset_rfl hYsub (P.mem_X a) (Q.mem_X b)⟩
  · intro hgen
    refine ⟨V.Ypd_isPDmem P c, W.Ypd_isPDmem Q d, ?_⟩
    rintro X hXin
    obtain ⟨a, ha, hXmem, hXsub⟩ := (V.mem_Ypd P).mp hXin
    obtain ⟨b, hb, hrel⟩ := hgen a ha
    exact ⟨Q.X b, (W.mem_Ypd Q).mpr ⟨b, hb, Q.mem_X b, subset_rfl⟩,
      f.mono hrel hXsub subset_rfl hXmem (Q.mem_X b)⟩
```

```
theorem PFmap_rel_fin (f : ApproximableMap V W) {L₁ : List (Set α)} {L₂ : List (Set β)}
    (hL₁ : ∀ X ∈ L₁, V.mem X) (hL₂ : ∀ Y ∈ L₂, W.mem Y) :
    (PFmap f).rel (⋃ X ∈ L₁, V.upSet X) (⋃ Y ∈ L₂, W.upSet Y)
      ↔ ∀ X ∈ L₁, ∃ Y ∈ L₂, f.rel X Y := by
  constructor
  · rintro ⟨-, -, hbody⟩ X hX
    have hXin : X ∈ ⋃ X' ∈ L₁, V.upSet X' :=
      (V.mem_PDunion).mpr ⟨X, hX, hL₁ X hX, subset_rfl⟩
    obtain ⟨Z, hZin, hrel⟩ := hbody X hXin
    obtain ⟨Y, hY, hZmem, hZsub⟩ := (W.mem_PDunion).mp hZin
    exact ⟨Y, hY, f.mono hrel subset_rfl hZsub (hL₁ X hX) (hL₂ Y hY)⟩
  · intro hgen
    refine ⟨⟨L₁, hL₁, rfl⟩, ⟨L₂, hL₂, rfl⟩, ?_⟩
    rintro Z hZin
    obtain ⟨X, hX, hZmem, hZsub⟩ := (V.mem_PDunion).mp hZin
    obtain ⟨Y, hY, hrel⟩ := hgen X hX
    exact ⟨Y, (W.mem_PDunion).mpr ⟨Y, hY, hL₂ Y hY, subset_rfl⟩,
      f.mono hrel hZsub subset_rfl hZmem (hL₂ Y hY)⟩

theorem bExists_decodeList_re {R : ℕ → ℕ → Prop} (hR : REPred₂ R) :
    REPred₂ (fun d a => ∃ b ∈ decodeList d, R a b) := by
  have hmem : RecDecidable₂ (fun d b => b ∈ decodeList d) := by
    have heq : RecDecidable₂ (fun e b => e = b) :=
      RecDecidable.natEq Nat.Primrec.left Nat.Primrec.right
    have hbase : RecDecidable₂ (fun d b => ∃ e ∈ decodeList d, e = b) := heq.
    bExistsList
    refine RecDecidable.of_iff (fun t => ?_) hbase
    constructor
    · intro h; exact ⟨_, h, rfl⟩
    · rintro ⟨e, he, rfl⟩; exact he
  have hpart1 : REPred (fun w : ℕ => w.unpair.1 ∈ decodeList w.unpair.2.unpair.1) :=
    by
    have hg : Nat.Primrec (fun w : ℕ => Nat.pair w.unpair.2.unpair.1 w.unpair.1) :=
      (Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left
    refine RecDecidable.re (RecDecidable.of_iff (fun w => ?_) (hmem.comp hg))
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hpart2 : REPred (fun w : ℕ => R w.unpair.2.unpair.2 w.unpair.1) := by
    have hg : Nat.Primrec (fun w : ℕ => Nat.pair w.unpair.2.unpair.2 w.unpair.1) :=
      (Nat.Primrec.right.comp Nat.Primrec.right).pair Nat.Primrec.left
    refine REPred.of_iff (fun w => ?_) (REPred.comp hR hg)
    simp only [unpair_pair_fst, unpair_pair_snd]
  refine REPred.of_iff (fun s => ?_) (hpart1.and hpart2).proj
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem PFmap_isComputable_Ypd (P : ComputablePresentation V) (Q :
    ComputablePresentation W)
    {f : ApproximableMap V W} (hf : ComputableMap P Q f) :
    REPred₂ (fun c d => (PFmap f).rel (V.Ypd P c) (W.Ypd Q d)) := by
  have hR : REPred₂ (fun a b => f.rel (P.X a) (Q.X b)) := hf.out
  have hInner : REPred₂ (fun d a => ∃ b ∈ decodeList d, f.rel (P.X a) (Q.X b)) :=
    bExists_decodeList_re hR
  have hForall := REPred.forall_mem_decodeList₂ hInner
  have hswap : Nat.Primrec (fun s : ℕ => Nat.pair s.unpair.2 s.unpair.1) :=
    Nat.Primrec.right.pair Nat.Primrec.left
  refine REPred.of_iff (fun s => ?_) (hForall.comp hswap)
  simp only [unpair_pair_fst, unpair_pair_snd]
```

```
  exact PFmap_rel_Ypd_iff P Q f

theorem PFmap_isComputable (P : ComputablePresentation V) (Q : ComputablePresentation
    W)
    (consV : ℕ → ℕ) (hconsVp : Nat.Primrec consV)
    (hconsVs : ∀ a b, consV (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b)
    (consW : ℕ → ℕ) (hconsWp : Nat.Primrec consW)
    (hconsWs : ∀ a b, consW (Nat.pair a b) = 1 ↔ ∃ k, Q.X k ⊆ Q.X a ∩ Q.X b)
    {f : ApproximableMap V W} (hf : ComputableMap P Q f) :
    ComputableMap (V.PDPresentation P consV hconsVp hconsVs)
      (W.PDPresentation Q consW hconsWp hconsWs) (PFmap f) := by
  exact ⟨PFmap_isComputable_Ypd P Q hf⟩

theorem comp_witness (f : ApproximableMap V W) (g : ApproximableMap W U) (C : Set (Set
    ?)) :
    ∀ (L : List (Set α)), (∀ X ∈ L, V.mem X) →
      (∀ X ∈ L, ∃ Z, Z ∈ C ∧ ∃ Y, f.rel X Y ∧ g.rel Y Z) →
      ∃ M : List (Set β), (∀ Y ∈ M, W.mem Y) ∧
        (∀ X ∈ L, ∃ Y ∈ M, f.rel X Y) ∧ (∀ Y ∈ M, ∃ Z ∈ C, g.rel Y Z) := by
  intro L
  induction L with
  | nil =>
    intro _ _
    refine ⟨[], ?_, ?_, ?_⟩
    · intro Y hY; cases hY
    · intro X hX; cases hX
    · intro Y hY; cases hY
  | cons X L ih =>
    intro hL hpre
    obtain ⟨Zx, hZxC, Yx, hfx, hgx⟩ := hpre X (List.mem_cons.mpr (Or.inl rfl))
    obtain ⟨M, hMmem, hfM, hgM⟩ :=
      ih (fun X' hX' => hL X' (List.mem_cons.mpr (Or.inr hX')))
         (fun X' hX' => hpre X' (List.mem_cons.mpr (Or.inr hX')))
    refine ⟨Yx :: M, ?_, ?_, ?_⟩
    · intro Y hY
      rcases List.mem_cons.mp hY with rfl | hY'
      · exact f.rel_cod hfx
      · exact hMmem Y hY'
    · intro X' hX'
      rcases List.mem_cons.mp hX' with rfl | hX''
      · exact ⟨Yx, List.mem_cons.mpr (Or.inl rfl), hfx⟩
      · obtain ⟨Y, hYM, hrel⟩ := hfM X' hX''
        exact ⟨Y, List.mem_cons.mpr (Or.inr hYM), hrel⟩
    · intro Y hY
      rcases List.mem_cons.mp hY with rfl | hY'
      · exact ⟨Zx, hZxC, hgx⟩
      · exact hgM Y hY'

theorem PFmap_comp (g : ApproximableMap W U) (f : ApproximableMap V W) :
    PFmap (g.comp f) = (PFmap g).comp (PFmap f) := by
  apply ApproximableMap.ext
  intro A C
  simp only [PFmap_rel, comp_rel]
  constructor
  · rintro ⟨hA, hC, hbody⟩
    obtain ⟨L, hL, rfl⟩ := hA
```

```
    have hpre : ∀ X ∈ L, ∃ Z, Z ∈ C ∧ ∃ Y, f.rel X Y ∧ g.rel Y Z := by
      intro X hX
      obtain ⟨Z, hZC, Y, hfXY, hgYZ⟩ :=
        hbody X ((V.mem_PDunion).mpr ⟨X, hX, hL X hX, subset_rfl⟩)
      exact ⟨Z, hZC, Y, hfXY, hgYZ⟩
    obtain ⟨M, hMmem, hfL, hgM⟩ := comp_witness f g C L hL hpre
    refine ⟨⋃ Y ∈ M, W.upSet Y,
      ⟨⟨L, hL, rfl⟩, ⟨M, hMmem, rfl⟩, ?_⟩, ⟨M, hMmem, rfl⟩, hC, ?_⟩
    · intro X hX
      obtain ⟨X', hX', hXmem, hXsub⟩ := (V.mem_PDunion).mp hX
      obtain ⟨Y, hYM, hrel⟩ := hfL X' hX'
      exact ⟨Y, (W.mem_PDunion).mpr ⟨Y, hYM, hMmem Y hYM, subset_rfl⟩,
        f.mono hrel hXsub subset_rfl hXmem (hMmem Y hYM)⟩
    · intro Y hY
      obtain ⟨Y', hY', hYmem, hYsub⟩ := (W.mem_PDunion).mp hY
      obtain ⟨Z, hZC, hrel⟩ := hgM Y' hY'
      exact ⟨Z, hZC, g.mono hrel hYsub subset_rfl hYmem (g.rel_cod hrel)⟩
  · rintro ⟨B, ⟨hA, hB, hfAB⟩, _, hC, hgBC⟩
    refine ⟨hA, hC, ?_⟩
    intro X hX
    obtain ⟨Y, hYB, hfXY⟩ := hfAB X hX
    obtain ⟨Z, hZC, hgYZ⟩ := hgBC Y hYB
    exact ⟨Z, hZC, Y, hfXY, hgYZ⟩

theorem gather_witnesses (f : ApproximableMap V W) :
    ∀ (L₁ : List (Set α)) (L₂ : List (Set β)), (∀ X ∈ L₁, ∃ Y ∈ L₂, f.rel X Y) →
      ∃ M : List (Set α × Set β), (∀ p ∈ M, p.1 ∈ L₁ ∧ p.2 ∈ L₂ ∧ f.rel p.1 p.2) ∧
        ∀ X ∈ L₁, ∃ Y ∈ L₂, (X, Y) ∈ M := by
  intro L₁
  induction L₁ with
  | nil => intro L₂ _; exact ⟨[], by simp, by simp⟩
  | cons X L₁ ih =>
    intro L₂ h
    obtain ⟨Y, hY, hrel⟩ := h X (List.mem_cons.mpr (Or.inl rfl))
    obtain ⟨M, hMprop, hMwit⟩ := ih L₂ (fun X' hX' => h X' (List.mem_cons.mpr (Or.inr
    hX')))
    refine ⟨(X, Y) :: M, ?_, ?_⟩
    · rintro p hp
      rcases List.mem_cons.mp hp with rfl | hp'
      · exact ⟨List.mem_cons.mpr (Or.inl rfl), hY, hrel⟩
      · obtain ⟨hX1, hY1, hrel1⟩ := hMprop p hp'
        exact ⟨List.mem_cons.mpr (Or.inr hX1), hY1, hrel1⟩
    · intro X' hX'
      rcases List.mem_cons.mp hX' with rfl | hX''
      · exact ⟨Y, hY, List.mem_cons.mpr (Or.inl rfl)⟩
      · obtain ⟨Y', hY', hmem⟩ := hMwit X' hX''
        exact ⟨Y', hY', List.mem_cons.mpr (Or.inr hmem)⟩

theorem exists_stepFun_forall_PFmap_rel (f : ApproximableMap V W) :
    ∀ (L : List (Set (Set α) × Set (Set β))),
      (∀ p ∈ L, V.PDmem p.1 ∧ W.PDmem p.2) → (∀ p ∈ L, (PFmap f).rel p.1 p.2) →
      ∃ M : List (Set α × Set β), (∀ p ∈ M, V.mem p.1 ∧ W.mem p.2) ∧
        (∀ p ∈ M, f.rel p.1 p.2) ∧
        ∀ g : ApproximableMap V W, (∀ p ∈ M, g.rel p.1 p.2) → ∀ p ∈ L, (PFmap g).rel p
    .1 p.2 := by
  intro L
```

```
  induction L with
  | nil => intro _ _; exact ⟨[], by simp, by simp, by simp⟩
  | cons p L ih =>
    intro hL hf
    obtain ⟨hA, hB⟩ := hL p (List.mem_cons.mpr (Or.inl rfl))
    obtain ⟨L₁, hL₁, hAeq⟩ := hA
    obtain ⟨L₂, hL₂, hBeq⟩ := hB
    have hrelAB : (PFmap f).rel p.1 p.2 := hf p (List.mem_cons.mpr (Or.inl rfl))
    rw [hAeq, hBeq] at hrelAB
    have hgen := (PFmap_rel_fin f hL₁ hL₂).mp hrelAB
    obtain ⟨M₀, hM₀prop, hM₀wit⟩ := gather_witnesses f L₁ L₂ hgen
    obtain ⟨M, hMmem, hMrel, hMgen⟩ :=
      ih (fun p' hp' => hL p' (List.mem_cons.mpr (Or.inr hp')))
         (fun p' hp' => hf p' (List.mem_cons.mpr (Or.inr hp')))
    refine ⟨M₀ ++ M, ?_, ?_, ?_⟩
    · intro q hq
      rcases List.mem_append.mp hq with hq' | hq'
      · obtain ⟨hq1, hq2, _⟩ := hM₀prop q hq'
        exact ⟨hL₁ q.1 hq1, hL₂ q.2 hq2⟩
      · exact hMmem q hq'
    · intro q hq
      rcases List.mem_append.mp hq with hq' | hq'
      · exact (hM₀prop q hq').2.2
      · exact hMrel q hq'
    · intro g hg p' hp'
      rcases List.mem_cons.mp hp' with rfl | hp''
      · rw [hAeq, hBeq]
        refine (PFmap_rel_fin g hL₁ hL₂).mpr ?_
        intro X hX
        obtain ⟨Y, hY, hmem⟩ := hM₀wit X hX
        exact ⟨Y, hY, hg (X, Y) (List.mem_append.mpr (Or.inl hmem))⟩
      · exact hMgen g (fun q hq => hg q (List.mem_append.mpr (Or.inr hq))) p' hp''

theorem PFmapComb_toElementMap (f : ApproximableMap V W) :
    (PFmapComb V W).toElementMap (toFilter f) = toFilter (PFmap f) := by
  apply Element.ext
  intro ?
  simp only [mem_toElementMap, mem_toFilter, PFmapComb_rel]
  constructor
  · rintro ⟨?, ⟨_, h?f⟩, _, h?mem, hbody⟩
    exact ⟨h?mem, hbody f h?f⟩
  · rintro ⟨h?mem, hf?⟩
    obtain ⟨⟨L₂, hL₂, rfl⟩, -⟩ := h?mem
    obtain ⟨M, hMmem, hMrel, hMgen⟩ :=
      exists_stepFun_forall_PFmap_rel f L₂ hL₂ (mem_stepFun.mp hf?)
    have h?mem : (funSpace V W).mem (stepFun M) := ⟨⟨M, hMmem, rfl⟩, f, mem_stepFun.mpr
     hMrel⟩
    have h?mem' : (funSpace V.PowerDomain W.PowerDomain).mem (stepFun L₂) :=
      ⟨⟨L₂, hL₂, rfl⟩, PFmap f, hf?⟩
    exact ⟨stepFun M, ⟨h?mem, mem_stepFun.mpr hMrel⟩, h?mem, h?mem',
      fun g hg => mem_stepFun.mpr (hMgen g (mem_stepFun.mp hg))⟩

end

end DomainTheory.NeighbourhoodSystems
end
```

4.8.47 Exercise 7.19 — power domain functor

Prove that $D \mapsto \mathbb{P}D$ is a functor by defining for each $f : D \to E$ a mapping

$$\mathbb{P}f : \mathbb{P}D \to \mathbb{P}E$$

by the formula

$$\bigcup_{i<n} \downarrow X_i \quad \mathbb{P}f \quad \bigcup_{j<m} \downarrow Y_j \quad \text{iff} \quad \forall i < n \exists j < m.\, X_i f Y_j\,.$$

Be sure to check that $\mathbb{P}f$ is approximable and that $\mathbb{P}$ preserves identity maps and composition. If f is computable, is $\mathbb{P}f$? Is there a combinator $\lambda f.\, \mathbb{P}f$? What is

$$\mathbb{P}f(\{x, y\}) = ??$$

The development is with the powerdomain functor peer.

Theorem (`example_719`). The direct-image map on powerdomains preserves composition and identities and is computable.

Proof. By the powerdomain functor. ■

`Exercise_7_19.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .powerFunctor

@[expose] public section

open DomainTheory.NeighbourhoodSystems

set_option linter.unusedVariables false
open NeighborhoodSystem Domain.Recursive ApproximableMap

/-- Scott's Exercise 7.19: powerdomain functoriality of PFmap. -/
theorem example_719 :
    (∀ {α β : Type*} {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
      (f : ApproximableMap V W),
      (PFmapComb V W).toElementMap (toFilter f) = toFilter (PFmap f)) ∧
    (∀ {α β ? : Type*} {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
      {U : NeighborhoodSystem ?} (g : ApproximableMap W U) (f : ApproximableMap V W),
      PFmap (g.comp f) = (PFmap g).comp (PFmap f)) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α),
      PFmap (idMap V) = idMap V.PowerDomain) ∧
    (∀ {α β : Type*} {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
      (P : ComputablePresentation V) (Q : ComputablePresentation W)
      (consV : ℕ → ℕ) (hconsVp : Nat.Primrec consV)
      (hconsVs : ∀ a b, consV (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b)
      (consW : ℕ → ℕ) (hconsWp : Nat.Primrec consW)
      (hconsWs : ∀ a b, consW (Nat.pair a b) = 1 ↔ ∃ k, Q.X k ⊆ Q.X a ∩ Q.X b)
      {f : ApproximableMap V W} (hf : ComputableMap P Q f),
      ComputableMap (V.PDPresentation P consV hconsVp hconsVs)
        (W.PDPresentation Q consW hconsWp hconsWs) (PFmap f)) ∧
    (∀ {α β : Type*} {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
```

```
    (P : ComputablePresentation V) (Q : ComputablePresentation W)
    {f : ApproximableMap V W} (hf : ComputableMap P Q f),
    REPred₂ (fun c d => (PFmap f).rel (V.Ypd P c) (W.Ypd Q d))) ∧
  (∀ {α β : Type*} {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
    (P : ComputablePresentation V) (Q : ComputablePresentation W)
    (f : ApproximableMap V W) {c d : ℕ},
    (PFmap f).rel (V.Ypd P c) (W.Ypd Q d) ↔
      ∀ a ∈ decodeList c, ∃ b ∈ decodeList d, f.rel (P.X a) (Q.X b)) ∧
  (∀ {α β : Type*} {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
    (f : ApproximableMap V W) {L₁ : List (Set α)} {L₂ : List (Set β)}
    (hL₁ : ∀ X ∈ L₁, V.mem X) (hL₂ : ∀ Y ∈ L₂, W.mem Y),
    (PFmap f).rel (⋃ X ∈ L₁, V.upSet X) (⋃ Y ∈ L₂, W.upSet Y) ↔
      ∀ X ∈ L₁, ∃ Y ∈ L₂, f.rel X Y) :=
⟨@PFmapComb_toElementMap, ⟨@PFmap_comp, ⟨@PFmap_idMap,
  ⟨@PFmap_isComputable, ⟨@PFmap_isComputable_Ypd, ⟨@PFmap_rel_Ypd_iff, @PFmap_rel_fin⟩
  ⟩⟩⟩⟩⟩
```

4.8.48 Powerdomain union

Union of neighbourhoods is a computable map on the powerdomain.

Definition (`unionMap`). Define union on the iterated powerdomain by relating a family of families to a target family when every neighbourhood occurring in a source member is contained in some target member.

Theorem (`unionMap_rel`). The union relation holds exactly when its arguments are valid powerdomain neighbourhoods and every neighbourhood occurring in the source is contained in a member of the target.

Proof. This equivalence is the relation by which union on the iterated powerdomain is defined. ■

Theorem (`unionMap_rel_Ypd_iff`). For canonical presented neighbourhoods, union holds exactly when every doubly decoded source index is contained in a basic neighbourhood indexed by the decoded target list.

Proof. Given the union relation, apply it to canonical basic members selected by the two source indices, then choose a basic target generator below the resulting member. Conversely, represent arbitrary source members by decoded basic generators and compose their inclusions with the assumed inclusion into a decoded target generator. ■

Theorem (`ypd_zero_subset_inter`). The zeroth presented powerdomain neighbourhood, which is empty, is contained in the intersection of any two presented powerdomain neighbourhoods.

Proof. The zeroth code decodes to the empty family, which is contained in every set and hence in the stated intersection. ■

Theorem (`unionMap_isComputable`). Union from the iterated powerdomain to the powerdomain is computable for the canonical powerdomain presentations.

Proof. By the canonical-code characterization, the union relation is a finite succession of universal list quantifiers followed by existence of a target index satisfying decidable basic inclusion. This predicate is decidable, and therefore recursively enumerable. ■

Theorem (`unionMap_rel_fin`). For finitely generated upper families, union holds exactly when every generator in every source list is contained in a target generator.

Proof. Forward, apply union to each source generator and then choose a target generator below the resulting target member. Reverse, choose generators below an arbitrary source family and one of its members; the assumed generator inclusion, followed by these two containments, supplies the required target witness. ■

powerUnion.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_11
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_10

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_19

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α : Type*}

def unionMap (V : NeighborhoodSystem α) :
    ApproximableMap V.PowerDomain.PowerDomain V.PowerDomain where
  rel A B := V.PowerDomain.PDmem A ∧ V.PDmem B ∧ ∀ S ∈ A, ∀ X ∈ S, ∃ Y ∈ B, X ⊆ Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine ⟨V.PowerDomain.PDmem_master, V.PDmem_master, ?_⟩
    intro S hS X hX
    simp only [mem_upSet] at hS
    exact ⟨V.master, ⟨V.master_mem, subset_rfl⟩, V.sub_master (V.PDmem_mem hS.1 hX)⟩
  inter_right := by
    rintro A B B' ⟨hA, hB, hbody⟩ ⟨-, hB', hbody'⟩
    refine ⟨hA, V.PDmem_inter hB hB', ?_⟩
    intro S hS X hX
    obtain ⟨Y, hYB, hXY⟩ := hbody S hS X hX
    obtain ⟨Y', hY'B', hXY'⟩ := hbody' S hS X hX
    have hXmem : V.mem X := V.PDmem_mem (V.PowerDomain.PDmem_mem hA hS) hX
    have hYmem : V.mem Y := V.PDmem_mem hB hYB
    have hY'mem : V.mem Y' := V.PDmem_mem hB' hY'B'
    have hsub : X ⊆ Y ∩ Y' := Set.subset_inter hXY hXY'
    have hinmem : V.mem (Y ∩ Y') := V.inter_mem hYmem hY'mem hXmem hsub
    exact ⟨Y ∩ Y', Set.mem_inter
      (V.PDmem_down hB hYB Set.inter_subset_left hinmem)
      (V.PDmem_down hB' hY'B' Set.inter_subset_right hinmem), hsub⟩
  mono := by
    rintro A A' B B' ⟨_, _, hbody⟩ hA'A hBB' hA' hB'
    refine ⟨hA', hB', ?_⟩
    intro S hS X hX
    obtain ⟨Y, hYB, hXY⟩ := hbody S (hA'A hS) X hX
    exact ⟨Y, hBB' hYB, hXY⟩

@[simp] theorem unionMap_rel {V : NeighborhoodSystem α} {A : Set (Set (Set α))} {B :
    Set (Set α)} :
    (unionMap V).rel A B ↔ V.PowerDomain.PDmem A ∧ V.PDmem B ∧ ∀ S ∈ A, ∀ X ∈ S, ∃ Y
    ∈ B, X ⊆ Y :=
  Iff.rfl
```

```
theorem unionMap_rel_Ypd_iff {V : NeighborhoodSystem α} (P : ComputablePresentation V)
    (Q : ComputablePresentation V.PowerDomain) (hQ : ∀ c, Q.X c = V.Ypd P c) {n m : ℕ}
    :
    (unionMap V).rel (V.PowerDomain.Ypd Q n) (V.Ypd P m)
      ↔ ∀ c ∈ decodeList n, ∀ a ∈ decodeList c, ∃ b ∈ decodeList m, P.X a ⊆ P.X b :=
   by
  constructor
  · rintro ⟨-, -, hbody⟩ c hc a ha
    have hSmem : Q.X c ∈ V.PowerDomain.Ypd Q n :=
      (V.PowerDomain.mem_Ypd Q).mpr ⟨c, hc, Q.mem_X c, subset_rfl⟩
    have hXinS : P.X a ∈ Q.X c := by
      rw [hQ c]; exact (V.mem_Ypd P).mpr ⟨a, ha, P.mem_X a, subset_rfl⟩
    obtain ⟨Y, hYB, hXY⟩ := hbody (Q.X c) hSmem (P.X a) hXinS
    obtain ⟨b, hb, hYin⟩ := (V.mem_Ypd P).mp hYB
    exact ⟨b, hb, hXY.trans hYin.2⟩
  · intro hgen
    refine ⟨V.PowerDomain.Ypd_isPDmem Q n, V.Ypd_isPDmem P m, ?_⟩
    intro S hS X hX
    obtain ⟨c, hc, hScod⟩ := (V.PowerDomain.mem_Ypd Q).mp hS
    have hXinQ : X ∈ Q.X c := hScod.2 hX
    rw [hQ c] at hXinQ
    obtain ⟨a, ha, hXin⟩ := (V.mem_Ypd P).mp hXinQ
    obtain ⟨b, hb, hab⟩ := hgen c hc a ha
    exact ⟨P.X b, (V.mem_Ypd P).mpr ⟨b, hb, P.mem_X b, subset_rfl⟩, hXin.2.trans hab⟩

theorem ypd_zero_subset_inter {V : NeighborhoodSystem α} (P : ComputablePresentation V
    ) (a b : ℕ) :
    V.Ypd P 0 ⊆ V.Ypd P a ∩ V.Ypd P b := by
  rw [V.Ypd_zero P]
  exact Set.empty_subset _

theorem unionMap_isComputable {V : NeighborhoodSystem α} (P : ComputablePresentation V
    )
    (consV : ℕ → ℕ) (hconsVp : Nat.Primrec consV)
    (hconsVs : ∀ a b, consV (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b) :
    ComputableMap
      (V.PowerDomain.PDPresentation (V.PDPresentation P consV hconsVp hconsVs)
        (fun _ => 1) (Nat.Primrec.const 1)
        (fun a b => ⟨fun _ => ⟨0, ypd_zero_subset_inter P a b⟩, fun _ => rfl⟩))
      (V.PDPresentation P consV hconsVp hconsVs)
      (unionMap V) := by
  constructor
  have hrd : RecDecidable₂ (fun n m => ∀ c ∈ decodeList n, ∀ a ∈ decodeList c,
      ∃ b ∈ decodeList m, P.X a ⊆ P.X b) := (V.subCode_computable P).bForallList
  have key : RecDecidable₂ (fun n m => (unionMap V).rel
      (V.PowerDomain.Ypd (V.PDPresentation P consV hconsVp hconsVs) n) (V.Ypd P m)) :=
    by
    refine RecDecidable.of_iff (fun t => ?_) hrd
    exact unionMap_rel_Ypd_iff P (V.PDPresentation P consV hconsVp hconsVs) (fun _ =>
    rfl)
  exact key.re

theorem unionMap_rel_fin (V : NeighborhoodSystem α) {LS : List (List (Set α))} {LY :
    List (Set α)}
    (hLS : ∀ l ∈ LS, ∀ X ∈ l, V.mem X) (hLY : ∀ Y ∈ LY, V.mem Y) :
```

```
    (unionMap V).rel (⋃ l ∈ LS, V.PowerDomain.upSet (⋃ X ∈ l, V.upSet X)) (⋃ Y ∈ LY,
    V.upSet Y)
      ↔ ∀ l ∈ LS, ∀ X ∈ l, ∃ Y ∈ LY, X ⊆ Y := by
  constructor
  · rintro ⟨-, -, hbody⟩ l hl X hX
    have hXmem : V.mem X := hLS l hl X hX
    have hSinA : (⋃ X ∈ l, V.upSet X) ∈ ⋃ l ∈ LS, V.PowerDomain.upSet (⋃ X ∈ l, V.
    upSet X) :=
      Set.mem_iUnion₂.mpr ⟨l, hl, (⟨l, hLS l hl, rfl⟩ : V.PDmem _), subset_rfl⟩
    have hXinS : X ∈ ⋃ X' ∈ l, V.upSet X' := (V.mem_PDunion).mpr ⟨X, hX, hXmem,
    subset_rfl⟩
    obtain ⟨Y, hYB, hXY⟩ := hbody _ hSinA X hXinS
    obtain ⟨Y', hY', hYin⟩ := (V.mem_PDunion).mp hYB
    exact ⟨Y', hY', hXY.trans hYin.2⟩
  · intro hgen
    refine ⟨?_, ⟨LY, hLY, rfl⟩, ?_⟩
    · refine ⟨LS.map (fun l => ⋃ X ∈ l, V.upSet X), ?_, ?_⟩
      · intro S hS
        obtain ⟨l, hl, rfl⟩ := List.mem_map.mp hS
        exact ⟨l, hLS l hl, rfl⟩
      · ext z
        simp only [Set.mem_iUnion, List.mem_map, exists_prop]
        constructor
        · rintro ⟨l, hl, hz⟩
          exact ⟨_, ⟨l, hl, rfl⟩, hz⟩
        · rintro ⟨S, ⟨l, hl, rfl⟩, hz⟩
          exact ⟨l, hl, hz⟩
    · intro S hS X hX
      obtain ⟨l, hl, hSin⟩ := Set.mem_iUnion₂.mp hS
      have hXin : X ∈ ⋃ X' ∈ l, V.upSet X' := hSin.2 hX
      obtain ⟨X', hX', hXmem⟩ := (V.mem_PDunion).mp hXin
      obtain ⟨Y, hY, hX'Y⟩ := hgen l hl X' hX'
      exact ⟨Y, (V.mem_PDunion).mpr ⟨Y, hY, hLY Y hY, subset_rfl⟩, hXmem.2.trans hX'Y⟩

end

end DomainTheory.NeighbourhoodSystems
end
```

4.8.49 Exercise 7.20 — union on power domains

Show that there is a combinator

$$\text{union} : \mathbb{P}(\mathbb{P}D) \to \mathbb{P}D$$

where for suitable neighbourhoods

$$\bigcup_{i<n} \downarrow \left(\bigcup_{j<m_i} \downarrow X_{ij} \right) \quad \text{union} \quad \bigcup_{k<q} \downarrow Y_k \quad \text{iff} \quad \forall i < n \forall j < m_i \exists k < q.\, X_{ij} \subseteq Y_k\,.$$

Is union computable if D is effectively given? What is

$$\text{union}(\{\{x\}, \{y, z\}\}) = ??$$

Are $\mathbb{P}(\mathbb{P}D)$ and $\mathbb{P}D$ generally isomorphic??

The development is with the powerdomain union peer.

Theorem (`example_720`). Union is a computable map on the powerdomain.

Proof. By the powerdomain union. ■

`Exercise_7_20.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .powerUnion

@[expose] public section

open DomainTheory.NeighbourhoodSystems

set_option linter.unusedVariables false
open NeighborhoodSystem Domain.Recursive ApproximableMap

/-- Scott's Exercise 7.20: union map on the powerdomain is computable. -/
theorem example_720 :
    (∀ {α : Type*} {V : NeighborhoodSystem α} (P : ComputablePresentation V)
      (consV : ℕ → ℕ) (hconsVp : Nat.Primrec consV)
      (hconsVs : ∀ a b, consV (Nat.pair a b) = 1 ↔ ∃ k, P.X k ⊆ P.X a ∩ P.X b),
      ComputableMap
        (V.PowerDomain.PDPresentation (V.PDPresentation P consV hconsVp hconsVs)
          (fun _ => 1) (Nat.Primrec.const 1)
          (fun a b => ⟨fun _ => ⟨0, ypd_zero_subset_inter P a b⟩, fun _ => rfl⟩))
        (V.PDPresentation P consV hconsVp hconsVs) (unionMap V)) ∧
    (∀ {α : Type*} {V : NeighborhoodSystem α} (P : ComputablePresentation V)
      (Q : ComputablePresentation V.PowerDomain) (hQ : ∀ c, Q.X c = V.Ypd P c) {n m :
    ℕ},
      (unionMap V).rel (V.PowerDomain.Ypd Q n) (V.Ypd P m) ↔
        ∀ c ∈ decodeList n, ∀ a ∈ decodeList c, ∃ b ∈ decodeList m, P.X a ⊆ P.X b) ∧
    (∀ {α : Type*} (V : NeighborhoodSystem α) {LS : List (List (Set α))} {LY : List (
    Set α)}
      (hLS : ∀ l ∈ LS, ∀ X ∈ l, V.mem X) (hLY : ∀ Y ∈ LY, V.mem Y),
      (unionMap V).rel (⋃ l ∈ LS, V.PowerDomain.upSet (⋃ X ∈ l, V.upSet X))
        (⋃ Y ∈ LY, V.upSet Y) ↔ ∀ l ∈ LS, ∀ X ∈ l, ∃ Y ∈ LY, X ⊆ Y) :=
  ⟨@unionMap_isComputable, ⟨@unionMap_rel_Ypd_iff, @unionMap_rel_fin⟩⟩
```

4.8.50 Pointwise application

Evaluation distributes over the powerdomain of a function space.

Definition (`papplyEval`). Define powerdomain evaluation by requiring every function neighbourhood in the first family and every argument neighbourhood in the second to evaluate into some member of the result family.

Theorem (`papplyEval_rel`). Powerdomain evaluation relates three families exactly when they are valid powerdomain neighbourhoods and every function–argument pair has an evaluation witness in the result family.

Proof. This is precisely the relation defining powerdomain evaluation. ■

Definition (`papplyB`). Define uncurried powerdomain application by viewing powerdomain evaluation as a map out of the product of function and argument powerdomains.

Definition (`papply`). Define powerdomain application as the currying of uncurried powerdomain evaluation.

Theorem (`papplyEval_rel_Ypd_iff`). On canonical presentations, powerdomain evaluation holds exactly when every decoded function index and decoded argument index have a decoded result index witnessing ordinary evaluation.

Proof. Forward, apply powerdomain evaluation to the basic function and argument neighbourhoods selected by the decoded indices, then choose a basic result neighbourhood below the witness. Reverse, choose decoded basic neighbourhoods below arbitrary function and argument members and enlarge the resulting basic evaluation witness by monotonicity. ■

Theorem (`papplyEval_step_witness`). A principal family generated by a single step function evaluates its principal argument family into the corresponding principal result family.

Proof. Every function in the principal family contains the generating step, and every argument in the principal argument family contains its input neighbourhood. The step relation therefore sends that argument to the generating result neighbourhood, which itself belongs to the principal result family. ■

Theorem (`re_forallG_forallX_existsY`). Recursive enumerability is preserved by bounded universal quantification over two decoded finite lists followed by bounded existential quantification over a third.

Proof. First combine the enumerable relation with decidable membership in the result list and project out the result witness. Then apply bounded universal quantification over the argument list and over the function list, rearranging the nested pair coding between these stages. ■

Theorem (`papplyEval_isComputable`). If ordinary evaluation on the chosen presentations is recursively enumerable, then powerdomain evaluation on their canonical finite-family presentations is recursively enumerable.

Proof. The canonical-code characterization identifies powerdomain evaluation with two bounded universal quantifiers and one bounded existential quantifier applied to ordinary evaluation. The preceding closure theorem therefore makes this coded relation recursively enumerable. ■

`papplyEval.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_19
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_11
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_10
public import DomainTheory.NeighbourhoodSystems.DomainEquations.indefProduct
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Theorem_6_9

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {α β : Type*}
```

```
def papplyEval (V : NeighborhoodSystem α) (W : NeighborhoodSystem β) :
    ApproximableMap₂ (funSpace V W).PowerDomain V.PowerDomain W.PowerDomain where
  rel ? A B := (funSpace V W).PDmem ? ∧ V.PDmem A ∧ W.PDmem B ∧
    ∀ G ∈ ?, ∀ X ∈ A, ∃ Y ∈ B, (eval V W).rel G X Y
  rel_dom₀ h := h.1
  rel_dom₁ h := h.2.1
  rel_cod h := h.2.2.1
  master_rel := by
    refine ⟨(funSpace V W).PDmem_master, V.PDmem_master, W.PDmem_master, ?_⟩
    intro G hG X hX
    simp only [mem_upSet] at hG hX
    refine ⟨W.master, ⟨W.master_mem, subset_rfl⟩, ?_⟩
    exact ⟨hG.1, hX.1, W.master_mem,
      fun f hf => f.mono f.master_rel (V.sub_master hX.1) subset_rfl hX.1 W.master_mem⟩
  inter_right := by
    rintro ? A B B' ⟨h?, hA, hB, hbody⟩ ⟨-, -, hB', hbody'⟩
    refine ⟨h?, hA, W.PDmem_inter hB hB', ?_⟩
    intro G hG X hX
    obtain ⟨Y, hYB, hrel⟩ := hbody G hG X hX
    obtain ⟨Y', hY'B', hrel'⟩ := hbody' G hG X hX
    have hinmem : W.mem (Y ∩ Y') := (eval V W).rel_cod ((eval V W).inter_right hrel
    hrel')
    exact ⟨Y ∩ Y', Set.mem_inter
      (W.PDmem_down hB hYB Set.inter_subset_left hinmem)
      (W.PDmem_down hB' hY'B' Set.inter_subset_right hinmem),
      (eval V W).inter_right hrel hrel'⟩
  mono := by
    rintro ? ?' A A' B B' ⟨_, _, _, hbody⟩ h?'? hA'A hBB' h?' hA' hB'
    refine ⟨h?', hA', hB', ?_⟩
    intro G hG X hX
    obtain ⟨Y, hYB, hrel⟩ := hbody G (h?'? hG) X (hA'A hX)
    exact ⟨Y, hBB' hYB, hrel⟩

@[simp] theorem papplyEval_rel {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
    {? : Set (Set (ApproximableMap V W))} {A : Set (Set α)} {B : Set (Set β)} :
    (papplyEval V W).rel ? A B ↔ (funSpace V W).PDmem ? ∧ V.PDmem A ∧ W.PDmem B ∧
      ∀ G ∈ ?, ∀ X ∈ A, ∃ Y ∈ B, (eval V W).rel G X Y := Iff.rfl

def papplyB (V : NeighborhoodSystem α) (W : NeighborhoodSystem β) :
    ApproximableMap (prod (funSpace V W).PowerDomain V.PowerDomain) W.PowerDomain :=
  ofMap₂ (papplyEval V W)

def papply (V : NeighborhoodSystem α) (W : NeighborhoodSystem β) :
    ApproximableMap (funSpace V W).PowerDomain (funSpace V.PowerDomain W.PowerDomain)
    :=
  curry (papplyB V W)

theorem papplyEval_rel_Ypd_iff {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
    (Pf : ComputablePresentation (funSpace V W)) (P : ComputablePresentation V)
    (Q : ComputablePresentation W) {?c Ac Bc : ℕ} :
    (papplyEval V W).rel ((funSpace V W).Ypd Pf ?c) (V.Ypd P Ac) (W.Ypd Q Bc)
      ↔ ∀ g ∈ decodeList ?c, ∀ x ∈ decodeList Ac, ∃ y ∈ decodeList Bc,
          (eval V W).rel (Pf.X g) (P.X x) (Q.X y) := by
  constructor
  · rintro ⟨-, -, -, hbody⟩ g hg x hx
```

```
    have hGin : Pf.X g ∈ (funSpace V W).Ypd Pf ?c :=
      ((funSpace V W).mem_Ypd Pf).mpr ⟨g, hg, Pf.mem_X g, subset_rfl⟩
    have hXin : P.X x ∈ V.Ypd P Ac := (V.mem_Ypd P).mpr ⟨x, hx, P.mem_X x, subset_rfl⟩
    obtain ⟨Y, hYin, hrel⟩ := hbody (Pf.X g) hGin (P.X x) hXin
    obtain ⟨b, hb, hYmem, hYsub⟩ := (W.mem_Ypd Q).mp hYin
    exact ⟨b, hb, (eval V W).mono hrel subset_rfl subset_rfl hYsub
      ((eval V W).rel_dom₀ hrel) ((eval V W).rel_dom₁ hrel) (Q.mem_X b)⟩
  · intro hgen
    refine ⟨(funSpace V W).Ypd_isPDmem Pf ?c, V.Ypd_isPDmem P Ac, W.Ypd_isPDmem Q Bc, ?
    _⟩
    intro G hGin X hXin
    obtain ⟨g, hg, hGmem, hGsub⟩ := ((funSpace V W).mem_Ypd Pf).mp hGin
    obtain ⟨x, hx, hXmem, hXsub⟩ := (V.mem_Ypd P).mp hXin
    obtain ⟨y, hy, hrel⟩ := hgen g hg x hx
    exact ⟨Q.X y, (W.mem_Ypd Q).mpr ⟨y, hy, Q.mem_X y, subset_rfl⟩,
      (eval V W).mono hrel hGsub hXsub subset_rfl hGmem hXmem (Q.mem_X y)⟩

theorem papplyEval_step_witness {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
    {X₀ : Set α} {Y₀ : Set β} (hX₀ : V.mem X₀) (hY₀ : W.mem Y₀) :
    (papplyEval V W).rel ((funSpace V W).upSet (step X₀ Y₀)) (V.upSet X₀) (W.upSet Y₀)
    := by
  refine ⟨(funSpace V W).PDmem_upSet (step_mem hX₀ hY₀), V.PDmem_upSet hX₀,
    W.PDmem_upSet hY₀, ?_⟩
  intro G hG X hX
  obtain ⟨hGmem, hGsub⟩ := hG
  obtain ⟨hXmem, hXsub⟩ := hX
  refine ⟨Y₀, ⟨hY₀, subset_rfl⟩, hGmem, hXmem, hY₀, ?_⟩
  intro f hf
  have hfstep : f.rel X₀ Y₀ := mem_step.mp (hGsub hf)
  exact f.mono hfstep hXsub subset_rfl hXmem hY₀

theorem re_forallG_forallX_existsY {R : ℕ → ℕ → ℕ → Prop}
    (hR : REPred (fun s => R s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.unpair.2)) :
    REPred (fun w => ∀ g ∈ decodeList w.unpair.1, ∀ x ∈ decodeList w.unpair.2.unpair
    .1,
      ∃ y ∈ decodeList w.unpair.2.unpair.2, R g x y) := by

  have hm1 : Nat.Primrec (fun s : ℕ => Nat.pair s.unpair.1.unpair.1
      (Nat.pair s.unpair.1.unpair.2 s.unpair.2)) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair
      ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right)
  have hR2 : REPred₂ (fun gx y => R gx.unpair.1 gx.unpair.2 y) := by
    show REPred (fun t => R t.unpair.1.unpair.1 t.unpair.1.unpair.2 t.unpair.2)
    refine REPred.of_iff (fun s => ?_) (hR.comp hm1)
    simp only [unpair_pair_fst, unpair_pair_snd]

  have hE : REPred (fun u => ∃ b ∈ decodeList u.unpair.1,
      R u.unpair.2.unpair.1 u.unpair.2.unpair.2 b) := bExists_decodeList_re hR2

  have hm2 : Nat.Primrec (fun t : ℕ => Nat.pair t.unpair.1.unpair.1
      (Nat.pair t.unpair.1.unpair.2 t.unpair.2)) := hm1
  have hF : REPred₂ (fun param x => ∃ y ∈ decodeList param.unpair.1,
      R param.unpair.2 x y) := by
    show REPred (fun t => ∃ y ∈ decodeList t.unpair.1.unpair.1,
      R t.unpair.1.unpair.2 t.unpair.2 y)
    refine REPred.of_iff (fun t => ?_) (hE.comp hm2)
```

```
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hFfull : REPred (fun t => ∀ x ∈ decodeList t.unpair.2,
      ∃ y ∈ decodeList t.unpair.1.unpair.1, R t.unpair.1.unpair.2 x y) :=
    REPred.forall_mem_decodeList₂ hF

  have hm3 : Nat.Primrec (fun t : ℕ => Nat.pair
      (Nat.pair t.unpair.1.unpair.1 t.unpair.2) t.unpair.1.unpair.2) :=
    ((Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right).pair
      (Nat.Primrec.right.comp Nat.Primrec.left)
  have hG : REPred₂ (fun param g => ∀ x ∈ decodeList param.unpair.2,
      ∃ y ∈ decodeList param.unpair.1, R g x y) := by
    show REPred (fun t => ∀ x ∈ decodeList t.unpair.1.unpair.2,
      ∃ y ∈ decodeList t.unpair.1.unpair.1, R t.unpair.2 x y)
    refine REPred.of_iff (fun t => ?_) (hFfull.comp hm3)
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hGfull : REPred (fun t => ∀ g ∈ decodeList t.unpair.2,
      ∀ x ∈ decodeList t.unpair.1.unpair.2,
        ∃ y ∈ decodeList t.unpair.1.unpair.1, R g x y) :=
    REPred.forall_mem_decodeList₂ hG

  have hm4 : Nat.Primrec (fun w : ℕ => Nat.pair
      (Nat.pair w.unpair.2.unpair.2 w.unpair.2.unpair.1) w.unpair.1) :=
    ((Nat.Primrec.right.comp Nat.Primrec.right).pair
      (Nat.Primrec.left.comp Nat.Primrec.right)).pair Nat.Primrec.left
  refine REPred.of_iff (fun w => ?_) (hGfull.comp hm4)
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem papplyEval_isComputable {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
    (Pf : ComputablePresentation (funSpace V W)) (P : ComputablePresentation V)
    (Q : ComputablePresentation W)
    (heval : REPred (fun s => (eval V W).rel (Pf.X s.unpair.1)
      (P.X s.unpair.2.unpair.1) (Q.X s.unpair.2.unpair.2))) :
    REPred (fun w => (papplyEval V W).rel ((funSpace V W).Ypd Pf w.unpair.1)
      (V.Ypd P w.unpair.2.unpair.1) (W.Ypd Q w.unpair.2.unpair.2)) := by
  refine REPred.of_iff (fun w => ?_)
    (re_forallG_forallX_existsY
      (R := fun g x y => (eval V W).rel (Pf.X g) (P.X x) (Q.X y)) heval)
  exact papplyEval_rel_Ypd_iff Pf P Q

end

end DomainTheory.NeighbourhoodSystems
```

4.8.51 Exercise 7.21 — distributing the power domain

Is there a non-trivial combinator of type

$$\mathbb{P}(D \to E) \to (\mathbb{P}D \to \mathbb{P}E)\,?$$

Are there in general any isomorphisms between the systems

$$(D \to \mathbb{P}E),\ \mathbb{P}(D \times E),\ \mathbb{P}D \times \mathbb{P}E\ ??$$

Is there a non-trivial combinator of type

$$\mathbb{P}(D \times E) \times \mathbb{P}(E \times F) \to \mathbb{P}(D \times F)\ ???$$

Is there any connection between

$$\mathbb{P}\mathbf{N} \text{ and } P\mathbf{N}\ ????$$

The development is with the pointwise application peer.

Theorem (`example_721`). Pointwise application on the powerdomain of a function space is computable, and it holds of enumerated neighbourhoods exactly when every pair from the first two lists meets the relation in some element of the third.

Proof. By pointwise application. ■

`Exercise_7_21.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .papplyEval

@[expose] public section

open DomainTheory.NeighbourhoodSystems

set_option linter.unusedVariables false
open NeighborhoodSystem Domain.Recursive ApproximableMap

/-- Scott's Exercise 7.21: pointwise application/eval on powerdomains. -/
theorem example_721 :
    (∀ {α β : Type*} {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
      (Pf : ComputablePresentation (funSpace V W)) (P : ComputablePresentation V)
      (Q : ComputablePresentation W)
      (heval : REPred (fun s => (eval V W).rel (Pf.X s.unpair.1)
        (P.X s.unpair.2.unpair.1) (Q.X s.unpair.2.unpair.2))),
      REPred (fun w => (papplyEval V W).rel ((funSpace V W).Ypd Pf w.unpair.1)
        (V.Ypd P w.unpair.2.unpair.1) (W.Ypd Q w.unpair.2.unpair.2))) ∧
    (∀ {α β : Type*} {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
      (Pf : ComputablePresentation (funSpace V W)) (P : ComputablePresentation V)
      (Q : ComputablePresentation W) {?c Ac Bc : ℕ},
      (papplyEval V W).rel ((funSpace V W).Ypd Pf ?c) (V.Ypd P Ac) (W.Ypd Q Bc) ↔
        ∀ g ∈ decodeList ?c, ∀ x ∈ decodeList Ac, ∃ y ∈ decodeList Bc,
          (eval V W).rel (Pf.X g) (P.X x) (Q.X y)) ∧
    (∀ {α β : Type*} {V : NeighborhoodSystem α} {W : NeighborhoodSystem β}
      {X₀ : Set α} {Y₀ : Set β} (hX₀ : V.mem X₀) (hY₀ : W.mem Y₀),
      (papplyEval V W).rel ((funSpace V W).upSet (step X₀ Y₀)) (V.upSet X₀) (W.upSet Y₀
    )) :=
  ⟨@papplyEval_isComputable, ⟨@papplyEval_rel_Ypd_iff, @papplyEval_step_witness⟩⟩
```

4.8.52 Regular languages

Expressions, automata, and the neighbourhood system of formal languages.

Definition (`concat`). The concatenation of two languages consists of all words obtained by appending a word of the second language to a word of the first.

Definition (`InS`). The admissible regular languages are generated by the universal language and singleton languages, and are closed under concatenation and nonempty intersection.

Theorem (`mem_concat`). The element described by concat belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for concat is supplied directly by the defining membership condition. ■

Theorem (`Ssys_master_mem`). The master neighbourhood in ssys master mem is the universal set and satisfies the claimed relation.

Proof. Following the defining recursion for ssys master mem and applying the preceding component lemmas yields the claimed result. ■

Theorem (`Ssys_master_nonempty`). The set or language ssys master nonempty has an element under the hypotheses of the theorem.

Proof. Choose witnesses from the nonempty input sets in ssys master nonempty and combine them using the defining construction. ■

Theorem (`Ssys_sub_master`). The master neighbourhood in ssys sub master is the universal set and satisfies the claimed relation.

Proof. Following the defining recursion for ssys sub master and applying the preceding component lemmas yields the claimed result. ■

Theorem (`concat_mono`). Enlarging the input sets in concat mono can only enlarge its resulting set or element.

Proof. Transport every witness for concat mono along the assumed inclusions; the same witness then belongs to the enlarged result. ■

Theorem (`concat_nonempty`). The set or language concat nonempty has an element under the hypotheses of the theorem.

Proof. Choose witnesses from the nonempty input sets in concat nonempty and combine them using the defining construction. ■

Theorem (`InS.nonempty`). The set or language nonempty has an element under the hypotheses of the theorem.

Proof. Choose witnesses from the nonempty input sets in nonempty and combine them using the defining construction. ■

Theorem (`Ssys_inter_mem`). The construction ssys inter mem preserves the specified intersection of sets or neighbourhoods.

Proof. Use the same witnesses in both directions for ssys inter mem, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Definition (`Ssys`). The regular-language neighbourhood system has all Boolean words as master set and the nonempty languages generated from the universal language and singleton languages by concatenation and intersection as neighbourhoods.

Theorem (`Ssys_mem`). For the objects in ssys mem, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for ssys mem and applying the preceding component lemmas yields the claimed result. ■

Definition (`mulElem`). Multiplication of language elements is induced by concatenating every neighbourhood from the first element with every neighbourhood from the second.

Theorem (`mem_mulElem`). The element described by mul elem belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for mul elem is supplied directly by the defining membership condition. ■

Definition (`emb`). A word is embedded as the principal language element generated by its singleton language.

Theorem (`mem_emb`). The element described by emb belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for emb is supplied directly by the defining membership condition. ■

Definition (`prependMap`). Prefixing by a fixed word defines an approximable endomap of the regular-language neighbourhood system.

Definition (`repeatWord`). The nth repetition of a word is obtained by concatenating n copies, with the zeroth repetition equal to the empty word.

Theorem (`repeatWord_zero`). At input zero, repeat word has the initial value specified by its definition.

Proof. The relevant clause of the recursive definition of repeat word zero gives this equation immediately. ■

Definition (`powerLang`). The power language of a word is the set of all of its finite repetitions.

Theorem (`mem_powerLang`). The element described by power lang belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for power lang is supplied directly by the defining membership condition. ■

Definition (`streamArrow`). The stream arrow of a word is the least fixed point of prefixing by that word.

Definition (`streamElemMem`). A language neighbourhood belongs to the stream element of a word when it contains a sufficiently long finite power of that word.

Definition (`streamElem`). The stream element generated by a word consists of all admissible languages containing some finite power of that word.

Theorem (`append_mem_concat`). For the objects in append mem concat, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for append mem concat and applying the preceding component lemmas yields the claimed result. ■

Theorem (`concat_assoc`). The binary operation concat assoc is associative.

Proof. Reassociate the three word concatenations in witnesses for concat assoc; this converts membership on either side into membership on the other. ■

Theorem (`mulElem_assoc`). The binary operation mul elem assoc is associative.

Proof. Reassociate the three word concatenations in witnesses for mul elem assoc; this converts membership on either side into membership on the other. ■

Theorem (`repeatWord_add`). For the objects in repeat word add, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for repeat word add and applying the preceding component lemmas yields the claimed result. ■

Theorem (`repeatWord_eq_empty`). The set or language in repeat word eq empty is empty exactly under the stated absence-of-witness condition.

Proof. An element of the set in repeat word eq empty would provide exactly the forbidden witness, while any such witness produces an element; hence emptiness is equivalent to its absence. ■

Theorem (`InS_powerLang_empty`). The set or language in in s power lang empty is empty exactly under the stated absence-of-witness condition.

Proof. An element of the set in in s power lang empty would provide exactly the forbidden witness, while any such witness produces an element; hence emptiness is equivalent to its absence. ■

Theorem (`mem_streamElem`). The element described by stream elem belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for stream elem is supplied directly by the defining membership condition. ■

Theorem (`streamElem_powers_of_mul`). The hypotheses mul imply stream elem powers.

Proof. Following the defining recursion for stream elem powers of mul and applying the preceding component lemmas yields the claimed result. ■

Theorem (`streamElem_mul_self_mem`). The element in stream elem mul self mem is idempotent under language multiplication.

Proof. Finite powers are closed under addition of exponents, so neighbourhood membership for stream elem mul self mem is unchanged by multiplying the element by itself. ■

Theorem (`streamElem_idempotent`). The element in stream elem idempotent is idempotent under language multiplication.

Proof. Finite powers are closed under addition of exponents, so neighbourhood membership for stream elem idempotent is unchanged by multiplying the element by itself. ■

Theorem (`Ssys_isPositive`). For the objects in ssys is positive, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for ssys is positive and applying the preceding component lemmas yields the claimed result. ■

Theorem (`Ssys_master`). The master neighbourhood in ssys master is the universal set and satisfies the claimed relation.

Proof. Following the defining recursion for ssys master and applying the preceding component lemmas yields the claimed result. ■

Theorem (`concat_singleton`). For the objects in concat singleton, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for concat singleton and applying the preceding component lemmas yields the claimed result. ■

Theorem (`emb_injective`). The construction emb injective sends equal outputs only from equal inputs.

Proof. Equality of two values of emb injective determines their outer form and then each component, so the original inputs coincide. ■

Theorem (`emb_mul`). For the objects in emb mul, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for emb mul and applying the preceding component lemmas yields the claimed result. ■

Theorem (`mulElem_bot_le`). The element or numerical value mul elem bot le lies below the bound in the theorem.

Proof. Following the defining recursion for mul elem bot le and applying the preceding component lemmas yields the claimed result. ■

Theorem (`mulElem_mono_right`). Enlarging the input sets in mul elem mono right can only enlarge its resulting set or element.

Proof. Transport every witness for mul elem mono right along the assumed inclusions; the same witness then belongs to the enlarged result. ■

Theorem (`powerLang_concat`). For the objects in power lang concat, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for power lang concat and applying the preceding component lemmas yields the claimed result. ■

Theorem (`prependMap_toElementMap`). For the objects in prepend map to element map, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for prepend map to element map and applying the preceding component lemmas yields the claimed result. ■

Theorem (`repeatWord_succ`). At a successor input, repeat word succ satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of repeat word succ gives this equation immediately. ■

Theorem (`streamArrow_eq`). The value of stream arrow eq equals the value described by stream arrow eq.

Proof. Compare membership or numerical components in stream arrow eq; the defining equations reduce both sides to the same data. ■

Theorem (`prependMap_iterElem_mul_streamArrow_le`). The element or numerical value prepend map iter elem mul stream arrow le lies below the bound in the theorem.

Proof. Following the defining recursion for prepend map iter elem mul stream arrow le and applying the preceding component lemmas yields the claimed result. ■

Theorem (`streamArrow_le_mul_self`). The element in stream arrow le mul self is idempotent under language multiplication.

Proof. Finite powers are closed under addition of exponents, so neighbourhood membership for stream arrow le mul self is unchanged by multiplying the element by itself. ■

Theorem (`streamArrow_mul_self_le`). The element in stream arrow mul self le is idempotent under language multiplication.

Proof. Finite powers are closed under addition of exponents, so neighbourhood membership for stream arrow mul self le is unchanged by multiplying the element by itself. ■

Theorem (`streamArrow_mul_self`). The element in stream arrow mul self is idempotent under language multiplication.

Proof. Finite powers are closed under addition of exponents, so neighbourhood membership for stream arrow mul self is unchanged by multiplying the element by itself. ■

Theorem (`streamArrow_containsZero_pow_four`). For the objects in stream arrow contains zero pow four, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for stream arrow contains zero pow four and applying the preceding component lemmas yields the claimed result. ■

Theorem (`streamArrow_mul_self_append_true`). The element in stream arrow mul self append true is idempotent under language multiplication.

Proof. Finite powers are closed under addition of exponents, so neighbourhood membership for stream arrow mul self append true is unchanged by multiplying the element by itself. ■

Theorem (`streamArrow_mul_self_self`). The element in stream arrow mul self self is idempotent under language multiplication.

Proof. Finite powers are closed under addition of exponents, so neighbourhood membership for stream arrow mul self self is unchanged by multiplying the element by itself. ■

Definition (`SExpr`). A regular-language expression is either the universal language, a singleton word, a concatenation, or an intersection.

Definition (`sexprOfNatFuel`). The fuel-bounded expression decoder reads a natural-number pairing code and recursively reconstructs a regular-language expression while fuel remains.

Definition (`sexprOfNat`). A natural number is decoded as an expression using one more unit of fuel than its own value.

Definition (`sexprToNat`). Expressions are encoded by assigning constructor tags and pairing the recursively encoded subexpressions.

Theorem (`snd_lt_pair_of_pos_fst`). The hypotheses pos fst imply snd lt pair.

Proof. Following the defining recursion for snd lt pair of pos fst and applying the preceding component lemmas yields the claimed result. ■

Theorem (`sexprOfNatFuel_sexprToNat`). For the objects in sexpr of nat fuel sexpr to nat, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for sexpr of nat fuel sexpr to nat and applying the preceding component lemmas yields the claimed result. ■

Definition (`denote`). The denotation of an expression interprets the four constructors as the universal language, singleton, concatenation, and intersection.

Theorem (`denote_sigma`). For the objects in denote sigma, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for denote sigma and applying the preceding component lemmas yields the claimed result. ■

Theorem (`denote_single`). For the objects in denote single, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for denote single and applying the preceding component lemmas yields the claimed result. ■

Theorem (`denote_cat`). For the objects in denote cat, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for denote cat and applying the preceding component lemmas yields the claimed result. ■

Theorem (`denote_cap`). For the objects in denote cap, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for denote cap and applying the preceding component lemmas yields the claimed result. ■

Definition (`matchesB`). The Boolean matcher decides whether a finite Boolean word belongs to the language denoted by an expression.

Theorem (`mem_concat_iff_split`). Membership in concat iff split is equivalent to the explicit witness condition given by the construction.

Proof. Expanding the membership definition of concat iff split exposes exactly the witnesses and equations occurring on the other side. ■

Theorem (`matchesB_iff`). The condition matches b holds exactly when the finite witness condition holds.

Proof. Expand both conditions in matches b iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Definition (`emptyExpr`). The empty-language expression is the intersection of the singleton languages containing false and true.

Theorem (`denote_emptyExpr`). The set or language in denote empty expr is empty exactly under the stated absence-of-witness condition.

Proof. An element of the set in denote empty expr would provide exactly the forbidden witness, while any such witness produces an element; hence emptiness is equivalent to its absence. ■

Definition (`containsZero`). The expression for words containing false is the concatenation of the universal language, the false singleton, and the universal language.

Theorem (`InS_denote_of_nonempty`). The set or language in s denote of nonempty has an element under the hypotheses of the theorem.

Proof. Choose witnesses from the nonempty input sets in in s denote of nonempty and combine them using the defining construction. ■

Theorem (`InS_exists_denote`). For the objects in in s exists denote, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for in s exists denote and applying the preceding component lemmas yields the claimed result. ■

Theorem (`denote_catSigmaSigma`). For the objects in denote cat sigma sigma, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for denote cat sigma sigma and applying the preceding component lemmas yields the claimed result. ■

Theorem (`empty_iff_equiv_emptyExpr`). The set or language in empty iff equiv empty expr is empty exactly under the stated absence-of-witness condition.

Proof. An element of the set in empty iff equiv empty expr would provide exactly the forbidden witness, while any such witness produces an element; hence emptiness is equivalent to its absence. ■

Theorem (`inS_eq_range_denote`). The value of in s equals the value described by range denote.

Proof. Compare membership or numerical components in in s eq range denote; the defining equations reduce both sides to the same data. ■

Theorem (`inS_iff_exists_denote`). The condition in s holds exactly when exists denote holds.

Proof. Expand both conditions in in s iff exists denote; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`interEq_iff`). The intersection in inter eq iff equals the indexed neighbourhood or language exactly under the stated finite comparison condition.

Proof. Use the same witnesses in both directions for inter eq iff, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`mem_denote_sigma`). The element described by denote sigma belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for denote sigma is supplied directly by the defining membership condition. ■

Theorem (`sigma_ne_containsZero`). At input zero, sigma ne contains has the initial value specified by its definition.

Proof. The relevant clause of the recursive definition of sigma ne contains zero gives this equation immediately. ■

Theorem (`example_722_regular`). Scott's example example 722 regular has the claimed language-theoretic or computability property.

Proof. Following the defining recursion for example 722 regular and applying the preceding component lemmas yields the claimed result. ■

Definition (`catEps`). Concatenation of two nondeterministic automata is represented by an epsilon automaton whose accepting states of the first component have epsilon transitions to the start states of the second.

Theorem (`catEps_step_inl_some`). For the objects in cat eps step inl some, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for cat eps step inl some and applying the preceding component lemmas yields the claimed result. ■

Theorem (`catEps_step_inl_none`). At input one, cat eps step inl has the value specified by its definition.

Proof. Following the defining recursion for cat eps step inl none and applying the preceding component lemmas yields the claimed result. ■

Theorem (`catEps_step_inr_some`). For the objects in cat eps step inr some, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for cat eps step inr some and applying the preceding component lemmas yields the claimed result. ■

Theorem (`catEps_step_inr_none`). At input one, cat eps step inr has the value specified by its definition.

Proof. Following the defining recursion for cat eps step inr none and applying the preceding component lemmas yields the claimed result. ■

Theorem (`catEps_start`). For the objects in cat eps start, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for cat eps start and applying the preceding component lemmas yields the claimed result. ■

Theorem (`catEps_accept`). For the objects in cat eps accept, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for cat eps accept and applying the preceding component lemmas yields the claimed result. ■

Theorem (`catEps_mem_`). For the objects in cat eps mem, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for cat eps mem and applying the preceding component lemmas yields the claimed result. ■

Theorem (`catEps_mem_eval_iff`). Membership in cat eps mem eval iff is equivalent to the explicit witness condition given by the construction.

Proof. Expanding the membership definition of cat eps mem eval iff exposes exactly the witnesses and equations occurring on the other side. ■

Theorem (`catEps_accepts`). The automaton in cat eps accepts accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for cat eps accepts; the accepting-state condition then gives the stated language. ■

Theorem (`example_722_cat`). Scott's example example 722 cat has the claimed language-theoretic or computability property.

Proof. Following the defining recursion for example 722 cat and applying the preceding component lemmas yields the claimed result. ■

Definition (`sigmaDFA`). The automaton sigma dfa has the recursively specified state space, transitions, initial states, and accepting states.

Theorem (`sigmaDFA_accepts`). The automaton in sigma dfa accepts accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for sigma dfa accepts; the accepting-state condition then gives the stated language. ■

Definition (`SingleState`). The state space for recognizing one fixed word consists of its positions together with a failure state.

Definition (`singleDFA`). The automaton single dfa has the recursively specified state space, transitions, initial states, and accepting states.

Theorem (`complDFA_accepts`). The automaton in compl dfa accepts accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for compl dfa accepts; the accepting-state condition then gives the stated language. ■

Theorem (`inter_eval`). Evaluating the automaton or finite transition system in inter eval yields the stated reachable state after reading the word.

Proof. Induction on the input word tracks the reachable states for inter eval; the accepting-state condition then gives the stated language. ■

Theorem (`interDFA_accepts`). The automaton in inter dfa accepts accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for inter dfa accepts; the accepting-state condition then gives the stated language. ■

Theorem (`example_722_dfa`). Scott's example example 722 dfa has the claimed language-theoretic or computability property.

Proof. Following the defining recursion for example 722 dfa and applying the preceding component lemmas yields the claimed result. ■

Theorem (`sigmaDFA_accepts_denote`). The automaton in sigma dfa accepts denote accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for sigma dfa accepts denote; the accepting-state condition then gives the stated language. ■

Theorem (`singleDFA_evalFrom_none`). Evaluating the automaton or finite transition system in single dfa eval from none yields the stated reachable state after reading the word.

Proof. Induction on the input word tracks the reachable states for single dfa eval from none; the accepting-state condition then gives the stated language. ■

Theorem (`singleDFA_evalFrom`). Evaluating the automaton or finite transition system in single dfa eval from yields the stated reachable state after reading the word.

Proof. Induction on the input word tracks the reachable states for single dfa eval from; the accepting-state condition then gives the stated language. ■

Theorem (`singleDFA_accepts`). The automaton in single dfa accepts accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for single dfa accepts; the accepting-state condition then gives the stated language. ■

Theorem (`singleDFA_accepts_denote`). The automaton in single dfa accepts denote accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for single dfa accepts denote; the accepting-state condition then gives the stated language. ■

Definition (`wordsUpTo`). The words of length at most n are enumerated recursively from the empty word by prefixing both Boolean letters.

Definition (`anyMatchesB`). This finite Boolean test reports whether some word of bounded length matches the expression.

Theorem (`mem_wordsUpTo`). The element described by words up to belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for words up to is supplied directly by the defining membership condition. ■

Theorem (`example_722_words`). Scott's example example 722 words has the claimed language-theoretic or computability property.

Proof. Following the defining recursion for example 722 words and applying the preceding component lemmas yields the claimed result. ■

Definition (`pathStateAtAux`). The auxiliary path-state function follows a path up to the requested position, starting from the supplied state.

Definition (`pathStateAt`). The state at a position of a finite automaton path is obtained by the auxiliary path traversal.

Definition (`PathSplit`). A path split records an intermediate state and the two path segments obtained by cutting a path at a chosen position.

Definition (`NFAinter`). The intersection automaton runs two nondeterministic automata in parallel on the product of their state spaces.

Theorem (`NFAinter_mem_eval_iff`). Membership in nfainter mem eval iff is equivalent to the explicit witness condition given by the construction.

Proof. Expanding the membership definition of nfainter mem eval iff exposes exactly the witnesses and equations occurring on the other side. ■

Theorem (`NFAinter_mem_accepts_iff`). Membership in nfainter mem accepts iff is equivalent to the explicit witness condition given by the construction.

Proof. Expanding the membership definition of nfainter mem accepts iff exposes exactly the witnesses and equations occurring on the other side. ■

Definition (`autState`). The automaton state type associated with an expression is defined recursively from the expression constructors.

Theorem (`dfaToNFA_eval`). Evaluating the automaton or finite transition system in dfa to nfa eval yields the stated reachable state after reading the word.

Proof. Induction on the input word tracks the reachable states for dfa to nfa eval; the accepting-state condition then gives the stated language. ■

Theorem (`dfaToNFA_accepts`). The automaton in dfa to nfa accepts accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for dfa to nfa accepts; the accepting-state condition then gives the stated language. ■

Definition (`autStateCard`). The number of automaton states associated with an expression is computed recursively from its constructors.

Theorem (`pathStateAt_zero`). At input zero, path state at has the initial value specified by its definition.

Proof. The relevant clause of the recursive definition of path state at zero gives this equation immediately. ■

Theorem (`accepts_card_zero`). The automaton in accepts card zero accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for accepts card zero; the accepting-state condition then gives the stated language. ■

Theorem (`autStateCard_eq_card`). The value of aut state card equals the value described by card.

Proof. Compare membership or numerical components in aut state card eq card; the defining equations reduce both sides to the same data. ■

Definition (`decideNonemptyB`). Nonemptiness is decided by searching words only up to the finite number of automaton states.

Definition (`decideEmptyB`). Emptiness is the Boolean negation of the nonemptiness decision.

Theorem (`mem_accepts_of_path`). The element described by accepts of path belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for accepts of path is supplied directly by the defining membership condition. ■

Definition (`pathAppend`). Two compatible automaton paths concatenate to a path labelled by the concatenation of their words.

Theorem (`pathStateAt_succ`). At a successor input, path state at succ satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of path state at succ gives this equation immediately. ■

Definition (`pathAppend_take_drop`). Every path is the concatenation of its prefix and suffix obtained by cutting at a bounded position.

Theorem (`accepts_skip_loop`). The automaton in accepts skip loop accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for accepts skip loop; the accepting-state condition then gives the stated language. ■

Theorem (`accepts_shorten_step`). The automaton in accepts shorten step accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for accepts shorten step; the accepting-state condition then gives the stated language. ■

Theorem (`exists_accepted_word_short`). The language in accepted word short is nonempty exactly when it accepts a word shorter than the number of available automaton states.

Proof. Repeatedly remove a loop from any longer accepting path for accepted word short; finiteness of the state set guarantees a repeated state until a short accepted word remains. ■

Definition (`toNFA`). The automaton to nfa has the recursively specified state space, transitions, initial states, and accepting states.

Theorem (`toNFA_accepts`). The automaton in to nfa accepts accepts exactly the words described by the associated language expression.

Proof. Induction on the input word tracks the reachable states for to nfa accepts; the accepting-state condition then gives the stated language. ■

Theorem (`denote_nonempty_iff_short`). The language in denote nonempty iff short is nonempty exactly when it accepts a word shorter than the number of available automaton states.

Proof. Repeatedly remove a loop from any longer accepting path for denote nonempty iff short; finiteness of the state set guarantees a repeated state until a short accepted word remains. ■

Theorem (`decideNonemptyB_iff`). The set or language decide nonempty b iff has an element under the hypotheses of the theorem.

Proof. Choose witnesses from the nonempty input sets in decide nonempty b iff and combine them using the defining construction. ■

Theorem (`decideEmptyB_iff`). The set or language in decide empty b iff is empty exactly under the stated absence-of-witness condition.

Proof. An element of the set in decide empty b iff would provide exactly the forbidden witness, while any such witness produces an element; hence emptiness is equivalent to its absence. ■

Definition (`consistentB`). Two expressions are consistent when the denotation of their intersection is nonempty.

Theorem (`consistentB_iff`). The condition consistent b holds exactly when the finite witness condition holds.

Proof. Expand both conditions in consistent b iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`capNonempty_iff_consistent`). The set or language cap nonempty iff consistent has an element under the hypotheses of the theorem.

Proof. Choose witnesses from the nonempty input sets in cap nonempty iff consistent and combine them using the defining construction. ■

Theorem (`autStateCard_le_card`). The element or numerical value aut state card le card lies below the bound in the theorem.

Proof. Following the defining recursion for aut state card le card and applying the preceding component lemmas yields the claimed result. ■

`regularLanguage.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_19
public import Mathlib.Computability.EpsilonNFA
public import Mathlib.Data.Fintype.Option

public import Mathlib.Logic.Equiv.List
public import Mathlib.Data.Fintype.Prod

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import Mathlib.Data.Fintype.Card
public import Mathlib.Data.Fintype.Pigeonhole
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace RegularLanguage
open NeighborhoodSystem ApproximableMap

def concat (X Y : Set (List Bool)) : Set (List Bool) := {w | ∃ a ∈ X, ∃ b ∈ Y, a ++ b
    = w}

inductive InS : Set (List Bool) → Prop
  | univ : InS Set.univ
```

```
  | singleton (σ : List Bool) : InS {σ}
  | mul {X Y : Set (List Bool)} : InS X → InS Y → InS (concat X Y)
  | inter {X Y : Set (List Bool)} : InS X → InS Y → (X ∩ Y).Nonempty → InS (X ∩ Y)

@[simp] theorem mem_concat {X Y : Set (List Bool)} {w : List Bool} :
    w ∈ concat X Y ↔ ∃ a ∈ X, ∃ b ∈ Y, a ++ b = w := Iff.rfl

theorem Ssys_master_mem : InS Set.univ := InS.univ

theorem Ssys_master_nonempty : (Set.univ : Set (List Bool)).Nonempty := by
  exact ⟨[], Set.mem_univ []⟩

theorem Ssys_sub_master {X : Set (List Bool)} (_h : InS X) : X ⊆ Set.univ :=
  Set.subset_univ _

theorem concat_mono {X X' Y Y' : Set (List Bool)} (hX : X ⊆ X') (hY : Y ⊆ Y') :
    concat X Y ⊆ concat X' Y' := by
  rintro w ⟨a, ha, b, hb, rfl⟩; exact ⟨a, hX ha, b, hY hb, rfl⟩

theorem concat_nonempty {X Y : Set (List Bool)} (hX : X.Nonempty) (hY : Y.Nonempty) :
    (concat X Y).Nonempty := by
  obtain ⟨a, ha⟩ := hX
  obtain ⟨b, hb⟩ := hY
  exact ⟨a ++ b, a, ha, b, hb, rfl⟩

theorem InS.nonempty {X : Set (List Bool)} (h : InS X) : X.Nonempty := by
  induction h with
  | univ => exact ⟨[], trivial⟩
  | singleton σ => exact ⟨σ, rfl⟩
  | mul _ _ ihX ihY => exact concat_nonempty ihX ihY
  | inter _ _ hne _ _ => exact hne

theorem Ssys_inter_mem {X Y Z : Set (List Bool)}
    (hX : InS X) (hY : InS Y) (hZ : InS Z) (hZsub : Z ⊆ X ∩ Y) : InS (X ∩ Y) :=
  InS.inter hX hY (hZ.nonempty.mono hZsub)

end RegularLanguage
end DomainTheory.NeighbourhoodSystems

section
namespace DomainTheory.NeighbourhoodSystems
namespace RegularLanguage
open NeighborhoodSystem ApproximableMap

def Ssys : NeighborhoodSystem (List Bool) where
  mem := InS
  master := Set.univ
  master_nonempty := Ssys_master_nonempty
  master_mem := Ssys_master_mem
  inter_mem := Ssys_inter_mem
  sub_master := Ssys_sub_master

@[simp] theorem Ssys_mem {X : Set (List Bool)} : Ssys.mem X ↔ InS X := Iff.rfl

def mulElem (x y : Ssys.Element) : Ssys.Element where
  mem Z := InS Z ∧ ∃ X, x.mem X ∧ ∃ Y, y.mem Y ∧ concat X Y ⊆ Z
```

```
  sub h := h.1
  master_mem :=
    ⟨InS.univ, Set.univ, x.master_mem, Set.univ, y.master_mem, Set.subset_univ _⟩
  inter_mem := by
    rintro Z1 Z2 ⟨hZ1, X1, hX1, Y1, hY1, hsub1⟩ ⟨hZ2, X2, hX2, Y2, hY2, hsub2⟩
    have hXi : x.mem (X1 ∩ X2) := x.inter_mem hX1 hX2
    have hYi : y.mem (Y1 ∩ Y2) := y.inter_mem hY1 hY2
    have hcsub : concat (X1 ∩ X2) (Y1 ∩ Y2) ⊆ Z1 ∩ Z2 := by
      intro w hw
      exact ⟨hsub1 (concat_mono Set.inter_subset_left Set.inter_subset_left hw),
             hsub2 (concat_mono Set.inter_subset_right Set.inter_subset_right hw)⟩
    have hne : (Z1 ∩ Z2).Nonempty :=
      (concat_nonempty (x.sub hXi).nonempty (y.sub hYi).nonempty).mono hcsub
    exact ⟨InS.inter hZ1 hZ2 hne, X1 ∩ X2, hXi, Y1 ∩ Y2, hYi, hcsub⟩
  up_mem := by
    rintro Z W ⟨_, X, hX, Y, hY, hsub⟩ hW hZW
    exact ⟨hW, X, hX, Y, hY, hsub.trans hZW⟩

@[simp] theorem mem_mulElem {x y : Ssys.Element} {Z : Set (List Bool)} :
    (mulElem x y).mem Z ↔ InS Z ∧ ∃ X, x.mem X ∧ ∃ Y, y.mem Y ∧ concat X Y ⊆ Z := Iff
    .rfl

def emb (σ : List Bool) : Ssys.Element where
  mem X := InS X ∧ σ ∈ X
  sub h := h.1
  master_mem := ⟨InS.univ, Set.mem_univ σ⟩
  inter_mem := by
    rintro X Y ⟨hX, hσX⟩ ⟨hY, hσY⟩
    exact ⟨InS.inter hX hY ⟨σ, hσX, hσY⟩, hσX, hσY⟩
  up_mem := by
    rintro X Y ⟨_, hσ⟩ hY hsub
    exact ⟨hY, hsub hσ⟩

@[simp] theorem mem_emb {σ : List Bool} {X : Set (List Bool)} :
    (emb σ).mem X ↔ InS X ∧ σ ∈ X := Iff.rfl

def prependMap (σ : List Bool) : ApproximableMap Ssys Ssys where
  rel Y Z := InS Y ∧ InS Z ∧ concat {σ} Y ⊆ Z
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨InS.univ, InS.univ, Set.subset_univ _⟩
  inter_right := by
    rintro Y Z Z' ⟨hY, hZ, hsub⟩ ⟨_, hZ', hsub'⟩
    have hcsub : concat {σ} Y ⊆ Z ∩ Z' := Set.subset_inter hsub hsub'
    have hne : (Z ∩ Z').Nonempty :=
      (concat_nonempty (Set.singleton_nonempty σ) hY.nonempty).mono hcsub
    exact ⟨hY, InS.inter hZ hZ' hne, hcsub⟩
  mono := by
    rintro Y Y' Z Z' ⟨_, hZ, hsub⟩ hYY' hZZ' hY' hZ'
    exact ⟨hY', hZ', (concat_mono (Set.Subset.refl _) hYY').trans (hsub.trans hZZ')⟩

def repeatWord (w : List Bool) : ℕ → List Bool
  | 0 => []
  | n + 1 => w ++ repeatWord w n

@[simp] theorem repeatWord_zero (w : List Bool) : repeatWord w 0 = [] := rfl
```

```
def powerLang (w : List Bool) : Set (List Bool) :=
  {u | ∃ n, repeatWord w n = u}

@[simp] theorem mem_powerLang {w u : List Bool} :
    u ∈ powerLang w ↔ ∃ n, repeatWord w n = u := Iff.rfl

def streamArrow (σ : List Bool) : Ssys.Element := (prependMap σ).fixElement

def streamElemMem (w : List Bool) (Z : Set (List Bool)) : Prop :=
  InS Z ∧ ∀ n, repeatWord w n ∈ Z

def streamElem (w : List Bool) : Ssys.Element where
  mem Z := streamElemMem w Z
  sub h := h.1
  master_mem := ⟨InS.univ, fun n => Set.mem_univ (repeatWord w n)⟩
  inter_mem := by
    intro X Y hX hY
    obtain ⟨hXIn, hXw⟩ := hX
    obtain ⟨hYIn, hYw⟩ := hY
    exact ⟨InS.inter hXIn hYIn ⟨repeatWord w 0, hXw 0, hYw 0⟩, fun n => ⟨hXw n, hYw n⟩⟩
  up_mem := by
    intro X Y hX hInSY hsub
    obtain ⟨_hXIn, hXw⟩ := hX
    exact ⟨hInSY, fun n => hsub (hXw n)⟩

theorem append_mem_concat {X Y : Set (List Bool)} {a b : List Bool} (ha : a ∈ X) (hb :
    b ∈ Y) :
    a ++ b ∈ concat X Y := ⟨a, ha, b, hb, rfl⟩

theorem concat_assoc (X Y Z : Set (List Bool)) :
    concat (concat X Y) Z = concat X (concat Y Z) := by
  ext w
  constructor
  · rintro ⟨ab, ⟨a, ha, b, hb, rfl⟩, c, hc, rfl⟩
    exact ⟨a, ha, b ++ c, ⟨b, hb, c, hc, rfl⟩, by rw [List.append_assoc]⟩
  · rintro ⟨a, ha, bc, ⟨b, hb, c, hc, rfl⟩, rfl⟩
    exact ⟨a ++ b, ⟨a, ha, b, hb, rfl⟩, c, hc, by rw [List.append_assoc]⟩

theorem mulElem_assoc (x y z : Ssys.Element) :
    mulElem (mulElem x y) z = mulElem x (mulElem y z) := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro ⟨hW, P, ⟨_, X, hX, Y, hY, hXY⟩, Z, hZ, hPZ⟩
    refine ⟨hW, X, hX, concat Y Z, ⟨InS.mul (y.sub hY) (z.sub hZ), Y, hY, Z, hZ,
      Set.Subset.refl _⟩, ?_⟩
    rw [← concat_assoc]
    exact (concat_mono hXY (Set.Subset.refl _)).trans hPZ
  · rintro ⟨hW, X, hX, Q, ⟨_, Y, hY, Z, hZ, hYZ⟩, hXQ⟩
    refine ⟨hW, concat X Y, ⟨InS.mul (x.sub hX) (y.sub hY), X, hX, Y, hY,
      Set.Subset.refl _⟩, Z, hZ, ?_⟩
    rw [concat_assoc]
    exact (concat_mono (Set.Subset.refl _) hYZ).trans hXQ

theorem repeatWord_add (w : List Bool) (a b : ℕ) :
```

```
    repeatWord w (a + b) = repeatWord w a ++ repeatWord w b := by
  induction a with
  | zero => simp [repeatWord]
  | succ a ih =>
      simp only [Nat.succ_add, repeatWord]
      rw [ih, List.append_assoc]

theorem repeatWord_eq_empty (w : List Bool) (n : ℕ) (hw : w = []) :
    repeatWord w n = [] := by
  subst hw
  induction n with
  | zero => rfl
  | succ n ih => simp [repeatWord, ih]

theorem InS_powerLang_empty : InS (powerLang []) := by
  have h : powerLang [] = {[]} := by
    ext u
    simp only [mem_powerLang, Set.mem_singleton_iff]
    constructor
    · rintro ⟨n, hn⟩; exact hn.symm.trans (repeatWord_eq_empty [] n rfl)
    · intro hu; subst hu; exact ⟨0, rfl⟩
  simpa [h] using InS.singleton []

@[simp] theorem mem_streamElem {w : List Bool} {Z : Set (List Bool)} :
    (streamElem w).mem Z ↔ streamElemMem w Z := Iff.rfl

theorem streamElem_powers_of_mul (w : List Bool) (Z : Set (List Bool))
    (h : (mulElem (streamElem w) (streamElem w)).mem Z) :
    streamElemMem w Z := by
  obtain ⟨hZ, X, hX, Y, hY, hsub⟩ := h
  obtain ⟨_hXIn, hXw⟩ := hX
  obtain ⟨_hYIn, hYw⟩ := hY
  refine ⟨hZ, fun n => ?_⟩
  have h1 := hsub (append_mem_concat (hXw n) (hYw 0))
  simpa [repeatWord_zero, List.append_nil] using h1

theorem streamElem_mul_self_mem (w : List Bool) (Z : Set (List Bool))
    (h : streamElemMem w Z) (hPL : InS (powerLang w)) :
    (mulElem (streamElem w) (streamElem w)).mem Z := by
  refine ⟨h.1, powerLang w, ⟨hPL, fun n => ⟨n, rfl⟩⟩, powerLang w, ⟨hPL, fun n => ⟨n, rfl
    ⟩⟩, ?_⟩
  intro u hu
  obtain ⟨a, ⟨m, hm⟩, b, ⟨n, hn⟩, rfl⟩ := hu
  simpa [hm, hn, repeatWord_add] using h.2 (m + n)

theorem streamElem_idempotent (w : List Bool) (hPL : InS (powerLang w)) :
    mulElem (streamElem w) (streamElem w) = streamElem w := by
  apply NeighborhoodSystem.Element.ext
  intro Z
  exact ⟨streamElem_powers_of_mul w Z, fun h => streamElem_mul_self_mem w Z h hPL⟩

example : mulElem (streamElem []) (streamElem []) = streamElem [] :=
  streamElem_idempotent [] InS_powerLang_empty

example (σ : List Bool) (Z : Set (List Bool)) :
    (mulElem (streamElem σ) (streamElem σ)).mem Z → streamElemMem σ Z :=
```

```
  streamElem_powers_of_mul σ Z

example (σ : List Bool) (h : InS (powerLang σ)) :
    mulElem (mulElem (streamElem σ) (streamElem σ)) (streamElem σ) = streamElem σ :=
    by
  rw [mulElem_assoc, streamElem_idempotent σ h, streamElem_idempotent σ h]

example (σ : List Bool) (h : InS (powerLang (σ ++ [true]))) :
    mulElem (streamElem (σ ++ [true])) (streamElem (σ ++ [true])) =
      streamElem (σ ++ [true]) :=
  streamElem_idempotent (σ ++ [true]) h

example (h : InS (powerLang [false, true])) :
    mulElem
        (mulElem (streamElem [false, true]) (streamElem [false, true]))
        (mulElem (streamElem [false, true]) (streamElem [false, true])) =
      mulElem (streamElem [false, true]) (streamElem [false, true]) := by
  rw [streamElem_idempotent _ h, streamElem_idempotent _ h]

theorem Ssys_isPositive : Ssys.IsPositive := by
  intro X Y hX hY
  exact ⟨fun h => h.nonempty, fun h => InS.inter hX hY h⟩

theorem Ssys_master : Ssys.master = Set.univ := rfl

theorem concat_singleton (a b : List Bool) : concat {a} {b} = {a ++ b} := by
  ext w
  simp only [mem_concat, Set.mem_singleton_iff]
  constructor
  · rintro ⟨a', rfl, b', rfl, rfl⟩; rfl
  · rintro rfl; exact ⟨a, rfl, b, rfl, rfl⟩

theorem emb_injective : Function.Injective emb := by
  intro σ τ h
  have hmem : (emb τ).mem {σ} := h ▸ (⟨InS.singleton σ, rfl⟩ : (emb σ).mem {σ})
  exact (Set.mem_singleton_iff.mp hmem.2).symm

theorem emb_mul (σ τ : List Bool) : emb (σ ++ τ) = mulElem (emb σ) (emb τ) := by
  apply NeighborhoodSystem.Element.ext
  intro Z
  constructor
  · rintro ⟨hZ, hστ⟩
    refine ⟨hZ, {σ}, ⟨InS.singleton σ, rfl⟩, {τ}, ⟨InS.singleton τ, rfl⟩, ?_⟩
    rw [concat_singleton]
    intro w hw
    rw [Set.mem_singleton_iff] at hw
    subst hw; exact hστ
  · rintro ⟨hZ, X, ⟨_, hσX⟩, Y, ⟨_, hτY⟩, hsub⟩
    exact ⟨hZ, hsub (append_mem_concat hσX hτY)⟩

theorem mulElem_bot_le (y : Ssys.Element) : mulElem Ssys.bot y ≤ y := by
  rintro Z ⟨hInS, X, hX, Y, hY, hsub⟩
  rw [NeighborhoodSystem.mem_bot, Ssys_master] at hX
  subst hX
  refine y.up_mem hY hInS fun w hw => hsub ?_
  simpa using append_mem_concat (Set.mem_univ ([] : List Bool)) hw
```

```
theorem mulElem_mono_right (x : Ssys.Element) : Monotone (mulElem x) := by
  rintro y y' hyy' Z ⟨hInS, X, hX, Y, hY, hsub⟩
  exact ⟨hInS, X, hX, Y, hyy' Y hY, hsub⟩

theorem powerLang_concat (w : List Bool) :
    concat (powerLang w) (powerLang w) ⊆ powerLang w := by
  rintro u ⟨a, ⟨m, hm⟩, b, ⟨n, hn⟩, rfl⟩
  exact ⟨m + n, by rw [repeatWord_add, hm, hn]⟩

theorem prependMap_toElementMap (σ : List Bool) (y : Ssys.Element) :
    (prependMap σ).toElementMap y = mulElem (emb σ) y := by
  apply NeighborhoodSystem.Element.ext
  intro Z
  constructor
  · rintro ⟨Y, hY, hInSY, hInSZ, hsub⟩
    exact ⟨hInSZ, {σ}, ⟨InS.singleton σ, rfl⟩, Y, hY, hsub⟩
  · rintro ⟨hInSZ, X, ⟨_, hσX⟩, Y, hY, hsub⟩
    refine ⟨Y, hY, y.sub hY, hInSZ, ?_⟩
    exact (concat_mono (Set.singleton_subset_iff.mpr hσX) (Set.Subset.refl _)).trans
    hsub

theorem repeatWord_succ (w : List Bool) (n : ℕ) :
    repeatWord w (n + 1) = w ++ repeatWord w n := rfl

theorem streamArrow_eq (σ : List Bool) :
    mulElem (emb σ) (streamArrow σ) = streamArrow σ := by
  rw [← prependMap_toElementMap]
  exact toElementMap_fixElement (prependMap σ)

theorem prependMap_iterElem_mul_streamArrow_le (σ : List Bool) :
    ∀ n, mulElem ((prependMap σ).iterElem n) (streamArrow σ) ≤ streamArrow σ
  | 0 => by
      have h0 : (prependMap σ).iterElem 0 = Ssys.bot := by
        show (idMap Ssys).toElementMap Ssys.bot = Ssys.bot
        exact toElementMap_idMap Ssys.bot
      rw [h0]
      exact mulElem_bot_le (streamArrow σ)
  | n + 1 => by
      have hsucc : (prependMap σ).iterElem (n + 1) =
          mulElem (emb σ) ((prependMap σ).iterElem n) := by
        show ((prependMap σ).comp ((prependMap σ).iterMap n)).toElementMap Ssys.bot =
    _
        rw [toElementMap_comp]
        exact prependMap_toElementMap σ _
      rw [hsucc, mulElem_assoc]
      calc mulElem (emb σ) (mulElem ((prependMap σ).iterElem n) (streamArrow σ))
          ≤ mulElem (emb σ) (streamArrow σ) :=
            mulElem_mono_right (emb σ) (prependMap_iterElem_mul_streamArrow_le σ n)
        _ = streamArrow σ := streamArrow_eq σ

theorem streamArrow_le_mul_self (σ : List Bool) :
    streamArrow σ ≤ mulElem (streamArrow σ) (streamArrow σ) := by
  apply fixElement_le_of_toElementMap_le
  have heq : (prependMap σ).toElementMap (mulElem (streamArrow σ) (streamArrow σ)) =
      mulElem (streamArrow σ) (streamArrow σ) := by
```

```
    rw [prependMap_toElementMap, ← mulElem_assoc, streamArrow_eq]
  exact le_of_eq heq

theorem streamArrow_mul_self_le (σ : List Bool) :
    mulElem (streamArrow σ) (streamArrow σ) ≤ streamArrow σ := by
  rintro Z ⟨hInSZ, X, hX, Y, hY, hsub⟩
  obtain ⟨n, hn⟩ := (mem_fixElement (prependMap σ)).mp hX
  have hXn : ((prependMap σ).iterElem n).mem X := (mem_iterElem (prependMap σ) n).mpr
    hn
  exact prependMap_iterElem_mul_streamArrow_le σ n Z ⟨hInSZ, X, hXn, Y, hY, hsub⟩

theorem streamArrow_mul_self (σ : List Bool) :
    mulElem (streamArrow σ) (streamArrow σ) = streamArrow σ :=
  le_antisymm (streamArrow_mul_self_le σ) (streamArrow_le_mul_self σ)

end RegularLanguage
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems
namespace RegularLanguage
open NeighborhoodSystem ApproximableMap

theorem streamArrow_containsZero_pow_four :
    mulElem
        (mulElem (streamArrow [false, true]) (streamArrow [false, true]))
        (mulElem (streamArrow [false, true]) (streamArrow [false, true]))
      = mulElem (streamArrow [false, true]) (streamArrow [false, true]) := by
  rw [streamArrow_mul_self, streamArrow_mul_self]

theorem streamArrow_mul_self_append_true (σ : List Bool) :
    mulElem (streamArrow (σ ++ [true])) (streamArrow (σ ++ [true])) =
      streamArrow (σ ++ [true]) :=
  streamArrow_mul_self (σ ++ [true])

theorem streamArrow_mul_self_self (σ : List Bool) :
    mulElem (mulElem (streamArrow σ) (streamArrow σ)) (streamArrow σ) = streamArrow σ
    := by
  rw [streamArrow_mul_self, streamArrow_mul_self]

inductive SExpr : Type
  | sigma : SExpr
  | single : List Bool → SExpr
  | cat : SExpr → SExpr → SExpr
  | cap : SExpr → SExpr → SExpr
  deriving DecidableEq

def sexprOfNatFuel : ℕ → ℕ → Option SExpr
  | 0, _ => none
  | fuel + 1, n =>


    match n.unpair.1 with
    | 0 => some .sigma
    | 1 => (Encodable.decode n.unpair.2).map SExpr.single
    | 2 =>
      match sexprOfNatFuel fuel n.unpair.2.unpair.1, sexprOfNatFuel fuel n.unpair.2.
    unpair.2 with
      | some a, some b => some (.cat a b)
```

```
      | _, _ => none
    | 3 =>
      match sexprOfNatFuel fuel n.unpair.2.unpair.1, sexprOfNatFuel fuel n.unpair.2.
    unpair.2 with
      | some a, some b => some (.cap a b)
      | _, _ => none
    | _ => none

def sexprOfNat (n : ℕ) : Option SExpr := sexprOfNatFuel (n + 1) n

def sexprToNat : SExpr → ℕ
  | .sigma => Nat.pair 0 0
  | .single σ => Nat.pair 1 (Encodable.encode σ)
  | .cat a b => Nat.pair 2 (Nat.pair (sexprToNat a) (sexprToNat b))
  | .cap a b => Nat.pair 3 (Nat.pair (sexprToNat a) (sexprToNat b))

theorem snd_lt_pair_of_pos_fst {a b : ℕ} (ha : 0 < a) : b < Nat.pair a b := by
  rw [Nat.pair]
  split_ifs with h
  · have hb : 0 < b := Nat.lt_trans ha h
    have hbb : b ≤ b * b := by
      calc
        b = 1 * b := (Nat.one_mul b).symm
        _ ≤ b * b := Nat.mul_le_mul_right b (Nat.succ_le_of_lt hb)
    have : b * b < b * b + a := Nat.lt_add_of_pos_right ha
    omega
  · have : 0 < a * a + a := by
      have : a ≤ a * a + a := Nat.le_add_left _ _
      omega
    omega

theorem sexprOfNatFuel_sexprToNat (e : SExpr) :
    ∀ fuel, sexprToNat e < fuel → sexprOfNatFuel fuel (sexprToNat e) = some e := by
  induction e with
  | sigma =>
    intro fuel hfuel
    cases fuel with
    | zero => omega
    | succ fuel => simp [sexprToNat, sexprOfNatFuel, Nat.unpair_pair]
  | single σ =>
    intro fuel hfuel
    cases fuel with
    | zero => omega
    | succ fuel => simp [sexprToNat, sexprOfNatFuel, Nat.unpair_pair, Encodable.
    encodek]
  | cat a b iha ihb =>
    intro fuel hfuel
    cases fuel with
    | zero => omega
    | succ fuel =>
      have ha : sexprToNat a < fuel := by
        have hinner : sexprToNat a ≤ Nat.pair (sexprToNat a) (sexprToNat b) := Nat.
    left_le_pair _ _
        have houter : Nat.pair (sexprToNat a) (sexprToNat b) < sexprToNat (.cat a b)
    := by
          simpa [sexprToNat] using snd_lt_pair_of_pos_fst (a := 2) (b := Nat.pair (
```

```
    sexprToNat a) (sexprToNat b)) (by omega)
        omega
      have hb : sexprToNat b < fuel := by
        have hinner : sexprToNat b ≤ Nat.pair (sexprToNat a) (sexprToNat b) := Nat.
    right_le_pair _ _
        have houter : Nat.pair (sexprToNat a) (sexprToNat b) < sexprToNat (.cat a b)
    := by
          simpa [sexprToNat] using snd_lt_pair_of_pos_fst (a := 2) (b := Nat.pair (
    sexprToNat a) (sexprToNat b)) (by omega)
        omega
      simp [sexprToNat, sexprOfNatFuel, Nat.unpair_pair, iha _ ha, ihb _ hb]
  | cap a b iha ihb =>
    intro fuel hfuel
    cases fuel with
    | zero => omega
    | succ fuel =>
      have ha : sexprToNat a < fuel := by
        have hinner : sexprToNat a ≤ Nat.pair (sexprToNat a) (sexprToNat b) := Nat.
    left_le_pair _ _
        have houter : Nat.pair (sexprToNat a) (sexprToNat b) < sexprToNat (.cap a b)
    := by
          simpa [sexprToNat] using snd_lt_pair_of_pos_fst (a := 3) (b := Nat.pair (
    sexprToNat a) (sexprToNat b)) (by omega)
        omega
      have hb : sexprToNat b < fuel := by
        have hinner : sexprToNat b ≤ Nat.pair (sexprToNat a) (sexprToNat b) := Nat.
    right_le_pair _ _
        have houter : Nat.pair (sexprToNat a) (sexprToNat b) < sexprToNat (.cap a b)
    := by
          simpa [sexprToNat] using snd_lt_pair_of_pos_fst (a := 3) (b := Nat.pair (
    sexprToNat a) (sexprToNat b)) (by omega)
        omega
      simp [sexprToNat, sexprOfNatFuel, Nat.unpair_pair, iha _ ha, ihb _ hb]

instance : Encodable SExpr where
  encode := sexprToNat
  decode := sexprOfNat
  encodek e := sexprOfNatFuel_sexprToNat e (sexprToNat e + 1) (Nat.lt_succ_self _)

def denote : SExpr → Set (List Bool)
  | .sigma => Set.univ
  | .single σ => {σ}
  | .cat a b => concat (denote a) (denote b)
  | .cap a b => denote a ∩ denote b

@[simp] theorem denote_sigma : denote .sigma = Set.univ := rfl

@[simp] theorem denote_single (σ : List Bool) : denote (.single σ) = {σ} := rfl

@[simp] theorem denote_cat (a b : SExpr) : denote (.cat a b) = concat (denote a) (
    denote b) := rfl

@[simp] theorem denote_cap (a b : SExpr) : denote (.cap a b) = denote a ∩ denote b :=
    rfl

def matchesB : SExpr → List Bool → Bool
```

```
  | .sigma, _ => true
  | .single σ, w => decide (w = σ)
  | .cat a b, w => (List.range (w.length + 1)).any (fun i => matchesB a (w.take i) &&
    matchesB b (w.drop i))
  | .cap a b, w => matchesB a w && matchesB b w

theorem mem_concat_iff_split {X Y : Set (List Bool)} {w : List Bool} :
    w ∈ concat X Y ↔ ∃ i, i ≤ w.length ∧ w.take i ∈ X ∧ w.drop i ∈ Y := by
  constructor
  · rintro ⟨a, ha, b, hb, rfl⟩
    refine ⟨a.length, by simp, ?_, ?_⟩
    · rw [List.take_left]; exact ha
    · rw [List.drop_left]; exact hb
  · rintro ⟨i, _, hpre, hsuf⟩
    exact ⟨w.take i, hpre, w.drop i, hsuf, w.take_append_drop i⟩

theorem matchesB_iff (e : SExpr) (w : List Bool) : matchesB e w = true ↔ w ∈ denote e
    := by
  induction e generalizing w with
  | sigma => simp [matchesB]
  | single σ => simp [matchesB]
  | cat a b iha ihb =>
    rw [denote_cat, mem_concat_iff_split]
    simp only [matchesB, List.any_eq_true, List.mem_range, Bool.and_eq_true]
    constructor
    · rintro ⟨i, hi, ha, hb⟩
      exact ⟨i, Nat.lt_succ_iff.mp hi, (iha _).mp ha, (ihb _).mp hb⟩
    · rintro ⟨i, hi, ha, hb⟩
      exact ⟨i, Nat.lt_succ_iff.mpr hi, (iha _).mpr ha, (ihb _).mpr hb⟩
  | cap a b iha ihb =>
    simp only [matchesB, Bool.and_eq_true, denote_cap, Set.mem_inter_iff]
    rw [iha, ihb]

instance decidableMemDenote (e : SExpr) : DecidablePred (· ∈ denote e) :=
  fun w => decidable_of_iff (matchesB e w = true) (matchesB_iff e w)

instance : Countable SExpr := inferInstance

def emptyExpr : SExpr := .cap (.single [false]) (.single [true])

@[simp] theorem denote_emptyExpr : denote emptyExpr = (∅ : Set (List Bool)) := by
  ext w
  simp only [emptyExpr, denote_cap, denote_single, Set.mem_inter_iff, Set.
    mem_singleton_iff,
    Set.mem_empty_iff_false, iff_false, not_and]
  rintro rfl
  decide

def containsZero : SExpr := .cat .sigma (.cat (.single [false]) .sigma)

theorem InS_denote_of_nonempty : ∀ {e : SExpr}, (denote e).Nonempty → InS (denote e)
  | .sigma, _ => InS.univ
  | .single σ, _ => InS.singleton σ
  | .cat a b, h => by
    rw [denote_cat] at h ⊢
    obtain ⟨w, x, hx, y, hy, _⟩ := h
```

```
    exact InS.mul (InS_denote_of_nonempty ⟨x, hx⟩) (InS_denote_of_nonempty ⟨y, hy⟩)
  | .cap a b, h => by
    rw [denote_cap] at h ⊢
    have ha : (denote a).Nonempty := h.mono Set.inter_subset_left
    have hb : (denote b).Nonempty := h.mono Set.inter_subset_right
    exact InS.inter (InS_denote_of_nonempty ha) (InS_denote_of_nonempty hb) h

theorem InS_exists_denote {X : Set (List Bool)} (h : InS X) : ∃ e : SExpr, denote e = X
    := by
  induction h with
  | univ => exact ⟨.sigma, rfl⟩
  | singleton σ => exact ⟨.single σ, rfl⟩
  | mul _ _ iha ihb =>
    obtain ⟨ea, rfl⟩ := iha
    obtain ⟨eb, rfl⟩ := ihb
    exact ⟨.cat ea eb, rfl⟩
  | inter _ _ _ iha ihb =>
    obtain ⟨ea, rfl⟩ := iha
    obtain ⟨eb, rfl⟩ := ihb
    exact ⟨.cap ea eb, rfl⟩

theorem denote_catSigmaSigma : denote (.cat .sigma .sigma) = denote .sigma := by
  ext w
  simp only [denote_cat, denote_sigma, mem_concat, Set.mem_univ, iff_true]
  exact ⟨[], trivial, w, trivial, rfl⟩

theorem empty_iff_equiv_emptyExpr (a b : SExpr) :
    denote (.cap a b) = (∅ : Set (List Bool)) ↔ denote (.cap a b) = denote emptyExpr
    := by
  rw [denote_emptyExpr]

theorem inS_eq_range_denote : {X | InS X} = denote '' {e | (denote e).Nonempty} := by
  ext X
  simp only [Set.mem_ofPred_eq, Set.mem_image]
  constructor
  · intro h
    obtain ⟨e, he⟩ := InS_exists_denote h
    exact ⟨e, by rw [he]; exact h.nonempty, he⟩
  · rintro ⟨e, hne, rfl⟩
    exact InS_denote_of_nonempty hne

theorem inS_iff_exists_denote {X : Set (List Bool)} :
    InS X ↔ ∃ e : SExpr, denote e = X ∧ X.Nonempty := by
  constructor
  · intro h
    obtain ⟨e, he⟩ := InS_exists_denote h
    exact ⟨e, he, h.nonempty⟩
  · rintro ⟨e, rfl, hne⟩
    exact InS_denote_of_nonempty hne

theorem interEq_iff (a b k : SExpr) :
    denote a ∩ denote b = denote k ↔ denote (.cap a b) = denote k := by
  rw [denote_cap]

theorem mem_denote_sigma (w : List Bool) : w ∈ denote .sigma := Set.mem_univ w
```

```
theorem sigma_ne_containsZero : denote .sigma = denote containsZero := by
  intro h
  have h1 : ([true] : List Bool) ∈ denote .sigma := mem_denote_sigma _
  rw [h] at h1
  exact absurd h1 (by decide)

end RegularLanguage
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.RegularLanguage
open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 7.22: Σ is distinct from the contains-zero language. -/
theorem example_722_regular : denote (.sigma) = denote containsZero :=
  sigma_ne_containsZero

end DomainTheory.NeighbourhoodSystems.RegularLanguage
namespace DomainTheory.NeighbourhoodSystems
namespace RegularLanguage
open scoped Computability
open Sum Set

section
variable {σ₁ σ₂ : Type}

def catEps (M₁ : NFA Bool σ₁) (M₂ : NFA Bool σ₂) : ?NFA Bool (σ₁ ? σ₂) where
  step s a :=
    match s, a with
    | inl s, some c => inl '' M₁.step s c
    | inl s, none   => {t | s ∈ M₁.accept ∧ t ∈ inr '' M₂.start}
    | inr s, some c => inr '' M₂.step s c
    | inr _, none   => ∅
  start := inl '' M₁.start
  accept := inr '' M₂.accept
end

section
variable {σ₁ σ₂ : Type}
variable (M₁ : NFA Bool σ₁) (M₂ : NFA Bool σ₂)

@[simp] theorem catEps_step_inl_some (s : σ₁) (c : Bool) :
    (catEps M₁ M₂).step (inl s) (some c) = inl '' M₁.step s c := rfl
@[simp] theorem catEps_step_inl_none (s : σ₁) :
    (catEps M₁ M₂).step (inl s) none = {t | s ∈ M₁.accept ∧ t ∈ inr '' M₂.start} := rfl
@[simp] theorem catEps_step_inr_some (s : σ₂) (c : Bool) :
    (catEps M₁ M₂).step (inr s) (some c) = inr '' M₂.step s c := rfl
@[simp] theorem catEps_step_inr_none (s : σ₂) :
    (catEps M₁ M₂).step (inr s) none = ∅ := rfl
@[simp] theorem catEps_start : (catEps M₁ M₂).start = inl '' M₁.start := rfl
@[simp] theorem catEps_accept : (catEps M₁ M₂).accept = inr '' M₂.accept := rfl

example [Fintype σ₁] [Fintype σ₂] : Fintype (σ₁ ? σ₂) := inferInstance

theorem catEps_mem_?Closure_iff (T : Set (σ₁ ? σ₂)) (s : σ₁ ? σ₂) :
    s ∈ (catEps M₁ M₂).?Closure T ↔
      s ∈ T ∨ ((∃ q ∈ M₁.accept, (inl q : σ₁ ? σ₂) ∈ T) ∧ s ∈ inr '' M₂.start) := by
  constructor
```

```
  · intro h
    induction h using ?NFA.?Closure_induction with
    | base x hx => exact Or.inl hx
    | step x y hyx _hclos ih =>

      cases x with
      | inl q =>
        rcases ih with hT | ⟨_, hmem⟩
        · simp only [catEps_step_inl_none, mem_ofPred_eq] at hyx
          exact Or.inr ⟨⟨q, hyx.1, hT⟩, hyx.2⟩
        · obtain ⟨w, _, hw⟩ := hmem; exact absurd hw.symm (by simp)
      | inr q => simp only [catEps_step_inr_none] at hyx; exact absurd hyx (by simp)
  · rintro (hs | ⟨⟨q, hq, hqT⟩, hs⟩)
    · exact (catEps M₁ M₂).subset_?Closure T hs
    · obtain ⟨t, ht, rfl⟩ := hs
      have hbase : (inl q : σ₁ ? σ₂) ∈ (catEps M₁ M₂).?Closure T :=
        (catEps M₁ M₂).subset_?Closure T hqT
      refine (catEps M₁ M₂).mem_?Closure_of_mem_step ?_ hbase
      simp only [catEps_step_inl_none, mem_ofPred_eq]
      exact ⟨hq, t, ht, rfl⟩

theorem catEps_mem_eval_iff (x : List Bool) (s : σ₁ ? σ₂) :
    s ∈ (catEps M₁ M₂).eval x ↔
      (∃ s₁, s = inl s₁ ∧ s₁ ∈ M₁.eval x) ∨
      (∃ t, s = inr t ∧ ∃ u v, x = u ++ v ∧ u ∈ M₁.accepts ∧ t ∈ M₂.eval v) := by
  induction x using List.reverseRecOn generalizing s with
  | nil =>
    show s ∈ (catEps M₁ M₂).?Closure (catEps M₁ M₂).start ↔ _
    rw [catEps_start, catEps_mem_?Closure_iff]
    constructor
    · rintro (hs | ⟨⟨q, hq, hqs⟩, hs⟩)
      · obtain ⟨s₁, hs₁, rfl⟩ := hs
        exact Or.inl ⟨s₁, rfl, hs₁⟩
      · obtain ⟨t, ht, rfl⟩ := hs
        refine Or.inr ⟨t, rfl, [], [], rfl, ?_, ht⟩
        obtain ⟨q', hq', hqq⟩ := hqs
        cases hqq
        exact ⟨q, hq, hq'⟩
    · rintro (⟨s₁, rfl, hs₁⟩ | ⟨t, rfl, u, v, hx, hu, hv⟩)
      · exact Or.inl ⟨s₁, hs₁, rfl⟩
      · obtain ⟨rfl, rfl⟩ := List.append_eq_nil_iff.mp hx.symm
        obtain ⟨q, hq, hqstart⟩ := hu
        exact Or.inr ⟨⟨q, hq, q, hqstart, rfl⟩, t, hv, rfl⟩
  | append_singleton x a ih =>
    rw [?NFA.eval_append_singleton, ?NFA.mem_stepSet_iff]
    constructor
    · rintro ⟨t, ht, hs⟩
      rw [ih] at ht
      rcases ht with ⟨s₁, rfl, hs₁⟩ | ⟨t₂, rfl, u, v, hx, hu, hv⟩
      ·
        rw [catEps_step_inl_some, catEps_mem_?Closure_iff] at hs
        rcases hs with hs | ⟨⟨q, hq, hqmem⟩, hs⟩
        ·
          obtain ⟨s', hs', rfl⟩ := hs
          exact Or.inl ⟨s', rfl, by rw [NFA.eval_append_singleton, NFA.mem_stepSet];
    exact ⟨s₁, hs₁, hs'⟩⟩
```

```
      ·
        obtain ⟨q', hq', hqq⟩ := hqmem
        cases hqq
        obtain ⟨t', ht', rfl⟩ := hs
        refine Or.inr ⟨t', rfl, x ++ [a], [], (List.append_nil _).symm, ?_, ht'⟩
        rw [NFA.mem_accepts]
        refine ⟨q, hq, ?_⟩
        change q ∈ M₁.eval (x ++ [a])
        rw [NFA.eval_append_singleton, NFA.mem_stepSet]
        exact ⟨s₁, hs₁, hq'⟩
    ·
      rw [catEps_step_inr_some, catEps_mem_?Closure_iff] at hs
      rcases hs with hs | ⟨⟨q, _, hqmem⟩, _⟩
      · obtain ⟨t', ht', rfl⟩ := hs
        refine Or.inr ⟨t', rfl, u, v ++ [a], by rw [hx, List.append_assoc], hu, ?_⟩
        rw [NFA.eval_append_singleton, NFA.mem_stepSet]; exact ⟨t₂, hv, ht'⟩
      · exact absurd hqmem (by simp)
  · rintro (⟨s', rfl, hs'⟩ | ⟨t', rfl, u, v, hx, hu, hv⟩)
    ·
      rw [NFA.eval_append_singleton, NFA.mem_stepSet] at hs'
      obtain ⟨s₁, hs₁, hstep⟩ := hs'
      exact ⟨inl s₁, (ih (inl s₁)).mpr (Or.inl ⟨s₁, rfl, hs₁⟩),
        by rw [catEps_step_inl_some, catEps_mem_?Closure_iff]; exact Or.inl ⟨s',
  hstep, rfl⟩⟩
    ·
      rcases List.eq_nil_or_concat v with rfl | ⟨v', c, rfl⟩
      ·
        rw [List.append_nil] at hx; subst hx
        rw [NFA.mem_accepts] at hu
        obtain ⟨q, hq, hqe⟩ := hu
        change q ∈ M₁.eval (x ++ [a]) at hqe
        rw [NFA.eval_append_singleton, NFA.mem_stepSet] at hqe
        obtain ⟨s₁, hs₁, hstep⟩ := hqe
        refine ⟨inl s₁, (ih (inl s₁)).mpr (Or.inl ⟨s₁, rfl, hs₁⟩), ?_⟩
        rw [catEps_step_inl_some, catEps_mem_?Closure_iff]
        exact Or.inr ⟨⟨q, hq, q, hstep, rfl⟩, t', hv, rfl⟩
      ·
        rw [List.concat_eq_append] at hx hv
        rw [← List.append_assoc] at hx
        obtain ⟨hxe, hac⟩ := List.append_inj' hx rfl
        have hca : c = a := by simpa using hac.symm
        subst hca
        rw [NFA.eval_append_singleton, NFA.mem_stepSet] at hv
        obtain ⟨t₂, ht₂, hstep⟩ := hv
        refine ⟨inr t₂, (ih (inr t₂)).mpr (Or.inr ⟨t₂, rfl, u, v', hxe, hu, ht₂⟩), ?_⟩
        rw [catEps_step_inr_some, catEps_mem_?Closure_iff]
        exact Or.inl ⟨t', hstep, rfl⟩

theorem catEps_accepts :
    (catEps M₁ M₂).accepts = concat M₁.accepts M₂.accepts := by
  ext x
  constructor
  · rintro ⟨S, hS, hSeval⟩
    obtain ⟨t, ht, rfl⟩ := hS
    rw [catEps_mem_eval_iff] at hSeval
    rcases hSeval with ⟨s₁, hc, _⟩ | ⟨t', heq, u, v, hx, hu, hv⟩
```

```
    · exact absurd hc (by simp)
    · cases heq
      exact ⟨u, hu, v, ⟨t, ht, hv⟩, hx.symm⟩
  · rintro ⟨u, hu, v, hv, rfl⟩
    obtain ⟨t, ht, hteval⟩ := hv
    refine ⟨inr t, ⟨t, ht, rfl⟩, ?_⟩
    rw [catEps_mem_eval_iff]
    exact Or.inr ⟨t, rfl, u, v, rfl, hu, hteval⟩

end

end RegularLanguage
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.RegularLanguage
open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 7.22: ?-NFA concatenation accepts the concatenated language. -/
theorem example_722_cat {σ₁ σ₂ : Type} (M₁ : NFA Bool σ₁) (M₂ : NFA Bool σ₂) :
    (catEps M₁ M₂).accepts = concat M₁.accepts M₂.accepts :=
  catEps_accepts M₁ M₂

end DomainTheory.NeighbourhoodSystems.RegularLanguage
namespace DomainTheory.NeighbourhoodSystems
namespace RegularLanguage
open scoped Computability


def sigmaDFA : DFA Bool Unit where
  step _ _ := ()
  start := ()
  accept := Set.univ

@[simp] theorem sigmaDFA_accepts : sigmaDFA.accepts = Set.univ :=
  Set.eq_univ_of_forall fun _ => Set.mem_univ _

abbrev SingleState (σ : List Bool) : Type := Option (Fin (σ.length + 1))

def singleDFA (σ : List Bool) : DFA Bool (SingleState σ) where
  step s b := s.bind fun i =>
    if h : (i : ℕ) < σ.length then
      if σ.get ⟨i, h⟩ = b then some ⟨i + 1, by omega⟩ else none
    else none
  start := some ⟨0, by omega⟩
  accept := {some ⟨σ.length, by omega⟩}

theorem complDFA_accepts {σ : Type} (M : DFA Bool σ) (a : SExpr) (h : M.accepts =
    denote a) :
    (M?).accepts = (denote a)? := by
  rw [← h]
  ext w
  exact Iff.rfl

example : Fintype (Unit) := inferInstance
example : DecidableEq (Unit) := inferInstance
example (σ : List Bool) : Fintype (SingleState σ) := by unfold SingleState;
    infer_instance
```

```
example (σ : List Bool) : DecidableEq (SingleState σ) := by unfold SingleState;
    infer_instance
example {σ₁ σ₂ : Type} [Fintype σ₁] [Fintype σ₂] : Fintype (σ₁ × σ₂) := inferInstance
example {σ₁ σ₂ : Type} [DecidableEq σ₁] [DecidableEq σ₂] : DecidableEq (σ₁ × σ₂) :=
    inferInstance

theorem inter_eval {σ₁ σ₂ : Type} (M₁ : DFA Bool σ₁) (M₂ : DFA Bool σ₂) (w : List Bool)
    :
    (M₁.inter M₂).eval w = (M₁.eval w, M₂.eval w) := by
  induction w using List.reverseRecOn with
  | nil => rfl
  | append_singleton w a ih =>
    rw [DFA.eval_append_singleton, DFA.eval_append_singleton, DFA.
    eval_append_singleton, ih]
    rfl

theorem interDFA_accepts {σ₁ σ₂ : Type} (M₁ : DFA Bool σ₁) (M₂ : DFA Bool σ₂)
    (a b : SExpr) (h₁ : M₁.accepts = denote a) (h₂ : M₂.accepts = denote b) :
    (M₁.inter M₂).accepts = denote (.cap a b) := by
  rw [denote_cap, ← h₁, ← h₂]
  ext w
  constructor
  · intro hw
    have hp : (M₁.inter M₂).eval w ∈ (M₁.inter M₂).accept := hw
    rw [inter_eval] at hp
    exact hp
  · intro hw
    show (M₁.inter M₂).eval w ∈ (M₁.inter M₂).accept
    rw [inter_eval]
    exact hw

end RegularLanguage
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.RegularLanguage
open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 7.22: DFA intersection matches the cap of denoted expressions. -/
theorem example_722_dfa {σ₁ σ₂ : Type} (M₁ : DFA Bool σ₁) (M₂ : DFA Bool σ₂)
    (a b : SExpr) :
    M₁.accepts = denote a → M₂.accepts = denote b →
      (M₁.inter M₂).accepts = denote (.cap a b) :=
  interDFA_accepts M₁ M₂ a b

end DomainTheory.NeighbourhoodSystems.RegularLanguage
namespace DomainTheory.NeighbourhoodSystems
namespace RegularLanguage
open scoped Computability

theorem sigmaDFA_accepts_denote : sigmaDFA.accepts = denote .sigma := by
  rw [sigmaDFA_accepts, denote_sigma]

theorem singleDFA_evalFrom_none (σ : List Bool) (w : List Bool) :
    (singleDFA σ).evalFrom none w = none := by
  induction w with
  | nil => rfl
  | cons b w ih => rw [DFA.evalFrom_cons]; exact ih
```

```
theorem singleDFA_evalFrom (σ : List Bool) (w : List Bool) (k : Fin (σ.length + 1)) :
    (singleDFA σ).evalFrom (some k) w =
      if h : w <+: List.drop (k : ℕ) σ
      then some ⟨(k : ℕ) + w.length, by
        have hlen := h.length_le
        rw [List.length_drop] at hlen
        have := k.isLt
        omega⟩
      else none := by
  induction w generalizing k with
  | nil =>
    rw [DFA.evalFrom_nil, dite_eq_left List.nil_prefix]
    simp
  | cons b w ih =>
    rw [DFA.evalFrom_cons]
    show (singleDFA σ).evalFrom ((singleDFA σ).step (some k) b) w = _
    by_cases hk : (k : ℕ) < σ.length
    · have hdrop : List.drop (k : ℕ) σ = σ.get ⟨k, hk⟩ :: List.drop ((k : ℕ) + 1) σ :=
    by
        rw [List.get_eq_getElem]; exact List.drop_eq_getElem_cons hk
      have hcondiff : ((b :: w) <+: List.drop (k : ℕ) σ) ↔ (b = σ.get ⟨k, hk⟩ ∧ w <+:
    List.drop ((k : ℕ) + 1) σ) := by
        rw [hdrop, List.cons_prefix_cons]
      by_cases hchar : σ.get ⟨k, hk⟩ = b
      · have hstep : (singleDFA σ).step (some k) b = some ⟨(k : ℕ) + 1, by omega⟩ := by
          simp only [singleDFA, Option.bind_some, dite_eq_left hk, ite_eq_left hchar]
        rw [hstep, ih ⟨(k : ℕ) + 1, by omega⟩]
        by_cases hpre : w <+: List.drop ((k : ℕ) + 1) σ
        · rw [dite_eq_left hpre, dite_eq_left (hcondiff.mpr ⟨hchar.symm, hpre⟩)]
          congr 1
          apply Fin.ext
          simp only [List.length_cons]
          omega
        · rw [dite_eq_right hpre, dite_eq_right (fun hc => hpre (hcondiff.mp hc).2)]
      · have hstep : (singleDFA σ).step (some k) b = none := by
          simp only [singleDFA, Option.bind_some, dite_eq_left hk, ite_eq_right hchar]
        rw [hstep, singleDFA_evalFrom_none, dite_eq_right]
        intro hc
        exact hchar (hcondiff.mp hc).1.symm
    · have hstep : (singleDFA σ).step (some k) b = none := by
        simp only [singleDFA, Option.bind_some, dite_eq_right hk]
      rw [hstep, singleDFA_evalFrom_none, dite_eq_right]
      have hd : List.drop (k : ℕ) σ = [] := by
        rw [List.drop_eq_nil_iff]; have := k.isLt; omega
      rw [hd]; exact (by simp : ¬ (b :: w) <+: [])

theorem singleDFA_accepts (σ : List Bool) : (singleDFA σ).accepts = {σ} := by
  ext w
  show (singleDFA σ).evalFrom (some ⟨0, by omega⟩) w ∈ (singleDFA σ).accept ↔ _
  rw [singleDFA_evalFrom]
  by_cases h : w <+: List.drop (0 : ℕ) (σ : List Bool)
  · rw [dite_eq_left h, List.drop_zero] at *
    show some (⟨0 + w.length, _⟩ : Fin (σ.length + 1)) ∈ ({some ⟨σ.length, _⟩} : Set _)
    ↔ w ∈ ({σ} : Set _)
    rw [Set.mem_singleton_iff, Set.mem_singleton_iff, Option.some.injEq, Fin.mk.injEq,
```

```
     Nat.zero_add]
    constructor
    · intro hlen; exact h.eq_of_length hlen
    · intro hw; subst hw; rfl
  · rw [dite_eq_right h, List.drop_zero] at *
    show (none : SingleState σ) ∈ ({some ⟨σ.length, _⟩} : Set _) ↔ w ∈ ({σ} : Set _)
    rw [Set.mem_singleton_iff, Set.mem_singleton_iff]
    constructor
    · intro hcontra; exact absurd hcontra (by simp)
    · intro hw; subst hw; exact absurd List.prefix_rfl h

theorem singleDFA_accepts_denote (σ : List Bool) : (singleDFA σ).accepts = denote (.
    single σ) := by
  rw [singleDFA_accepts]; exact (denote_single σ).symm

def wordsUpTo : ℕ → List (List Bool)
  | 0 => [[]]
  | n + 1 =>
    (wordsUpTo n) ++ (wordsUpTo n).flatMap fun w => [[false] ++ w, [true] ++ w]

def anyMatchesB (e : SExpr) (ws : List (List Bool)) : Bool := ws.any (matchesB e)

#eval anyMatchesB .sigma (wordsUpTo 0)

theorem mem_wordsUpTo (n : ℕ) {w : List Bool} : w ∈ wordsUpTo n ↔ w.length ≤ n := by
  induction n generalizing w with
  | zero =>
    simp only [wordsUpTo, List.mem_singleton]
    constructor
    · rintro (rfl : w = [])
      simp
    · intro h
      exact (List.eq_nil_of_length_eq_zero (Nat.eq_zero_of_le_zero h)).symm ▸ rfl
  | succ n ih =>
    simp only [wordsUpTo]
    rw [List.mem_append, List.mem_flatMap]
    constructor
    · intro h
      rcases h with h | ⟨w', hw', hmem⟩
      · exact Nat.le_succ_of_le (ih.mp h)
      · rw [List.mem_cons, List.mem_singleton] at hmem
        rcases hmem with rfl | rfl
        · simpa [List.length_cons] using Nat.succ_le_succ (ih.mp hw')
        · simpa [List.length_cons] using Nat.succ_le_succ (ih.mp hw')
    · intro hlen
      by_cases hw : w.length ≤ n
      · exact Or.inl (ih.mpr hw)
      · match w with
        | [] => exfalso; exact hw (by simp)
        | hd :: tl =>
          have htl_len : tl.length ≤ n := by
            have : tl.length + 1 ≤ n + 1 := by simpa [List.length_cons] using hlen
            exact Nat.le_of_succ_le_succ this
          have htl : tl ∈ wordsUpTo n := ih.mpr htl_len
          refine Or.inr ⟨tl, htl, ?_⟩
          rw [List.mem_cons, List.mem_singleton]
```

```
          cases hd with
          | false => exact Or.inl rfl
          | true => exact Or.inr rfl

end RegularLanguage
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.RegularLanguage
open NeighborhoodSystem ApproximableMap

/-- Scott's Exercise 7.22: wordsUpTo characterises length. -/
theorem example_722_words (n : ℕ) {w : List Bool} :
    w ∈ wordsUpTo n ↔ w.length ≤ n :=
  mem_wordsUpTo n

end DomainTheory.NeighbourhoodSystems.RegularLanguage
namespace DomainTheory.NeighbourhoodSystems
namespace RegularLanguage
open scoped Computability
open Sum Set

section
variable {σ₁ σ₂ : Type}
variable {σ : Type}
variable {σ : Type} [Fintype σ] {M : NFA Bool σ}

def pathStateAtAux {s t : σ} {x : List Bool} : M.Path s t x → ∀ n, n ≤ x.length → σ
  | .nil s, 0, _ => s
  | .nil _, n + 1, hn => (Nat.not_succ_le_zero n hn).elim
  | .cons _ s' _ _ _ _ p', 0, _ => s'
  | .cons _ _ _ _ _ _ p', n + 1, hn =>
      pathStateAtAux p' n (Nat.le_of_succ_le_succ hn)

def pathStateAt {s t : σ} {x : List Bool} (p : M.Path s t x) (n : ℕ) (hn : n ≤ x.
    length) : σ :=
  pathStateAtAux p n hn

structure PathSplit {s t : σ} {x : List Bool} (p : M.Path s t x) (n : ℕ) (hn : n ≤ x.
    length) where
  u : σ
  hp : M.Path s u (x.take n)
  ht : M.Path u t (x.drop n)
  hu : pathStateAt p n hn = u

end

section
variable {σ₁ σ₂ : Type}

section NFAinterLemmas

def NFAinter (M₁ : NFA Bool σ₁) (M₂ : NFA Bool σ₂) : NFA Bool (σ₁ × σ₂) where
  step s a := {t | t.1 ∈ M₁.step s.1 a ∧ t.2 ∈ M₂.step s.2 a}
  start := {s | s.1 ∈ M₁.start ∧ s.2 ∈ M₂.start}
  accept := {s | s.1 ∈ M₁.accept ∧ s.2 ∈ M₂.accept}

end NFAinterLemmas
```

```
section NFAinterLemmas

variable (M₁ : NFA Bool σ₁) (M₂ : NFA Bool σ₂)

theorem NFAinter_mem_eval_iff (x : List Bool) (p : σ₁ × σ₂) :
    p ∈ (NFAinter M₁ M₂).eval x ↔ p.1 ∈ M₁.eval x ∧ p.2 ∈ M₂.eval x := by
  induction x using List.reverseRecOn generalizing p with
  | nil => exact Iff.rfl
  | append_singleton x a ih =>
    rw [NFA.eval_append_singleton, NFA.mem_stepSet]
    constructor
    · rintro ⟨q, hq, hstep⟩
      rw [ih] at hq
      rw [NFA.eval_append_singleton, NFA.mem_stepSet, NFA.eval_append_singleton, NFA.
    mem_stepSet]
      exact ⟨⟨q.1, hq.1, hstep.1⟩, ⟨q.2, hq.2, hstep.2⟩⟩
    · rintro ⟨h1, h2⟩
      rw [NFA.eval_append_singleton, NFA.mem_stepSet] at h1 h2
      obtain ⟨q1, hq1, hs1⟩ := h1
      obtain ⟨q2, hq2, hs2⟩ := h2
      exact ⟨(q1, q2), (ih (q1, q2)).mpr ⟨hq1, hq2⟩, hs1, hs2⟩

theorem NFAinter_mem_accepts_iff (x : List Bool) :
    x ∈ (NFAinter M₁ M₂).accepts ↔ x ∈ M₁.accepts ∧ x ∈ M₂.accepts := by
  constructor
  · rintro ⟨p, hp, hpe⟩
    rw [NFAinter_mem_eval_iff] at hpe
    exact ⟨⟨p.1, hp.1, hpe.1⟩, ⟨p.2, hp.2, hpe.2⟩⟩
  · rintro ⟨⟨s, hs, hse⟩, ⟨t, ht, hte⟩⟩
    exact ⟨(s, t), ⟨hs, ht⟩, (NFAinter_mem_eval_iff M₁ M₂ x (s, t)).mpr ⟨hse, hte⟩⟩

end NFAinterLemmas

def autState : SExpr → Type
  | .sigma => Unit
  | .single σ => Option (Fin (σ.length + 1))
  | .cap a b => autState a × autState b
  | .cat a b => autState a ? autState b

theorem dfaToNFA_eval {σ : Type} (M : DFA Bool σ) (x : List Bool) :
    M.toNFA.eval x = {M.eval x} := by
  induction x using List.reverseRecOn with
  | nil => rfl
  | append_singleton x a ih =>
    rw [NFA.eval_append_singleton, ih]
    ext s
    rw [NFA.mem_stepSet]
    constructor
    · rintro ⟨t, ht, hs⟩
      rw [Set.mem_singleton_iff] at ht
      subst ht
      rw [Set.mem_singleton_iff, DFA.eval_append_singleton]
      exact hs
    · intro hs
      rw [Set.mem_singleton_iff, DFA.eval_append_singleton] at hs
```

```
      exact ⟨M.eval x, Set.mem_singleton _, hs⟩

theorem dfaToNFA_accepts {σ : Type} (M : DFA Bool σ) : M.toNFA.accepts = M.accepts :=
    by
  ext x
  constructor
  · rintro ⟨S, hS, hSe⟩
    rw [dfaToNFA_eval, Set.mem_singleton_iff] at hSe
    rw [DFA.mem_accepts, ← hSe]
    exact hS
  · intro hx
    rw [DFA.mem_accepts] at hx
    exact ⟨M.eval x, hx, by rw [dfaToNFA_eval]; exact Set.mem_singleton _⟩

instance instFintypeAutState : (e : SExpr) → Fintype (autState e)
  | .sigma => inferInstanceAs (Fintype Unit)
  | .single σ => inferInstanceAs (Fintype (Option (Fin (σ.length + 1))))
  | .cap a b => by
      letI := instFintypeAutState a; letI := instFintypeAutState b
      exact inferInstanceAs (Fintype (autState a × autState b))
  | .cat a b => by
      letI := instFintypeAutState a; letI := instFintypeAutState b
      exact inferInstanceAs (Fintype (autState a ? autState b))

instance instDecidableEqAutState : (e : SExpr) → DecidableEq (autState e)
  | .sigma => inferInstanceAs (DecidableEq Unit)
  | .single σ => inferInstanceAs (DecidableEq (Option (Fin (σ.length + 1))))
  | .cap a b => by
      letI := instDecidableEqAutState a; letI := instDecidableEqAutState b
      exact inferInstanceAs (DecidableEq (autState a × autState b))
  | .cat a b => by
      letI := instDecidableEqAutState a; letI := instDecidableEqAutState b
      exact inferInstanceAs (DecidableEq (autState a ? autState b))

def autStateCard : SExpr → ℕ
  | .sigma => 1
  | .single σ => σ.length + 2
  | .cap a b => autStateCard a * autStateCard b
  | .cat a b => autStateCard a + autStateCard b

end

section
variable {σ₁ σ₂ : Type}
variable {σ : Type}

section ShortWord

variable {σ : Type} [Fintype σ] {M : NFA Bool σ}

omit [Fintype σ] in
@[simp] theorem pathStateAt_zero {s t : σ} {x : List Bool} (p : M.Path s t x) :
    pathStateAt p 0 (Nat.zero_le _) = s := by
  cases x with
  | nil =>
    cases p; simp [pathStateAt, pathStateAtAux]
```

```
  | cons a xs =>
    cases p with
    | cons _ s' _ _ _ _ _ => simp [pathStateAt, pathStateAtAux]

theorem accepts_card_zero (hσ : Fintype.card σ = 0) {w : List Bool} (hw : w ∈ M.
    accepts) : False := by
  rw [NFA.mem_accepts] at hw
  obtain ⟨t, ht, hte⟩ := hw
  rw [NFA.mem_evalFrom_iff_exists (S := M.start) (s := t) (x := w)] at hte
  obtain ⟨s, hs, _⟩ := hte
  exact (Fintype.card_eq_zero_iff.mp hσ).elim s

theorem autStateCard_eq_card (e : SExpr) : autStateCard e = Fintype.card (autState e)
    := by
  induction e with
  | sigma =>
    simp [autStateCard, autState]
    exact (Fintype.card_unique (α := Unit)).symm
  | single σ =>
    simp only [autStateCard, autState]
    have h1 : Fintype.card (Option (Fin (σ.length + 1))) =
        Fintype.card (Fin (σ.length + 1)) + 1 :=
      Fintype.card_option (α := Fin (σ.length + 1))
    have h2 : Fintype.card (Fin (σ.length + 1)) = σ.length + 1 :=
      Fintype.card_fin (σ.length + 1)
    calc autStateCard (.single σ)
        = σ.length + 2 := rfl
      _ = (σ.length + 1) + 1 := rfl
      _ = Fintype.card (Fin (σ.length + 1)) + 1 := (congrArg (fun n => n + 1) h2).symm
      _ = Fintype.card (Option (Fin (σ.length + 1))) := h1.symm
  | cap a b ih_a ih_b =>
    simp [autStateCard, autState, ih_a, ih_b]
    exact (Fintype.card_prod (autState a) (autState b)).symm
  | cat a b ih_a ih_b =>
    simp [autStateCard, autState, ih_a, ih_b]
    exact (Fintype.card_sum (α := autState a) (β := autState b)).symm

end ShortWord

def decideNonemptyB (e : SExpr) : Bool :=
  anyMatchesB e (wordsUpTo (autStateCard e))

def decideEmptyB (e : SExpr) : Bool := !decideNonemptyB e

section ShortWord

variable {σ : Type} [Fintype σ] {M : NFA Bool σ}

omit [Fintype σ] in
theorem mem_accepts_of_path {s t : σ} {x : List Bool} (hs : s ∈ M.start) (ht : t ∈ M.
    accept)
    (p : M.Path s t x) : x ∈ M.accepts := by

  rw [NFA.accepts_iff_exists_path]
  exact ⟨s, hs, t, ht, ⟨p⟩⟩
```

```
def pathAppend {s u t : σ} {a b : List Bool} (p : M.Path s u a) (p' : M.Path u t b) :
    M.Path s t (a ++ b) :=
  match p with
  | .nil _ => p'
  | .cons sMid s' _ a' xs hstep p'' =>
      .cons sMid s' _ a' (xs ++ b) hstep (pathAppend p'' p')

omit [Fintype σ] in
theorem pathStateAt_succ {sMid s' u : σ} {a : Bool} {xs : List Bool}
    (hstep : sMid ∈ M.step s' a) (p' : M.Path sMid u xs) (n : ℕ) (hn : n ≤ xs.length)
    :
    pathStateAt (.cons sMid s' u a xs hstep p') (n + 1) (Nat.succ_le_succ hn) =
      pathStateAt p' n hn := by
  simp [pathStateAt, pathStateAtAux]

noncomputable def pathAppend_take_drop {s t : σ} {x : List Bool} (p : M.Path s t x) (n
    : ℕ)
    (hn : n ≤ x.length) : PathSplit p n hn := by
  revert n hn
  induction p with
  | nil s =>
    intro n hn
    have hn0 : n = 0 := Nat.eq_zero_of_le_zero hn
    subst hn0
    exact { u := s, hp := .nil s, ht := .nil s, hu := by rfl }
  | cons sMid s' u a xs hstep p' ih =>
    intro n hn
    cases n with
    | zero =>
      exact { u := s', hp := .nil s', ht := .cons sMid s' u a xs hstep p', hu := by
    rfl }
    | succ n' =>
      have hn' : n' ≤ xs.length := by simpa [List.length_cons] using hn
      have split := ih n' hn'
      exact {
        u := split.u
        hp := .cons sMid s' split.u a (xs.take n') hstep split.hp
        ht := split.ht
        hu := by rw [pathStateAt_succ hstep p' n' hn']; exact split.hu
      }

omit [Fintype σ] in
theorem accepts_skip_loop {s t : σ} {x : List Bool} {i j : ℕ}
    (hi : i ≤ x.length) (hj : j ≤ x.length) (_hij : i < j)
    (hs : s ∈ M.start) (ht : t ∈ M.accept) (hp : M.Path s t x)
    (heq : pathStateAt hp i hi = pathStateAt hp j hj) :
    x.take i ++ x.drop j ∈ M.accepts := by
  let splitI := pathAppend_take_drop hp i hi
  let splitJ := pathAppend_take_drop hp j hj
  have hqq' : splitI.u = splitJ.u := by rw [← splitI.hu, heq, splitJ.hu]
  exact mem_accepts_of_path hs ht (pathAppend splitI.hp (hqq' ▸ splitJ.ht))

theorem accepts_shorten_step {x : List Bool} (hx : x ∈ M.accepts)
    (hlen : Fintype.card σ ≤ x.length) :
    ∃ y, y ∈ M.accepts ∧ y.length < x.length := by
  rw [NFA.accepts_iff_exists_path] at hx
```

```
  obtain ⟨s, hs, t, ht, ⟨hp⟩⟩ := hx
  obtain ⟨i, j, hne, heq⟩ :=
    Fintype.exists_ne_map_eq_of_card_lt
      (f := fun k : Fin (x.length + 1) => pathStateAt hp k.val (Nat.le_of_lt_succ k.
    isLt))
      (by rw [Fintype.card_fin]; exact Nat.lt_succ_iff.mpr hlen)
  have hval : i.val = j.val := fun h => hne (Fin.ext h)
  rcases Nat.lt_or_gt_of_ne hval with hij | hij
  · refine ⟨x.take i.val ++ x.drop j.val, ?_, ?_⟩
    · exact accepts_skip_loop (by omega) (by omega) hij hs ht hp heq
    · have : j.val - i.val > 0 := Nat.sub_pos_of_lt hij
      simp [List.length_append, List.length_take, List.length_drop]
      omega
  · refine ⟨x.take j.val ++ x.drop i.val, ?_, ?_⟩
    · exact accepts_skip_loop (by omega) (by omega) hij hs ht hp heq.symm
    · have : i.val - j.val > 0 := Nat.sub_pos_of_lt hij
      simp [List.length_append, List.length_take, List.length_drop]
      omega

theorem exists_accepted_word_short (h : M.accepts.Nonempty) :
    ∃ w, w.length < Fintype.card σ ∧ w ∈ M.accepts := by
  by_cases hσ : Fintype.card σ = 0
  · obtain ⟨x, hx⟩ := h
    exact (accepts_card_zero hσ hx).elim
  · obtain ⟨x, hx⟩ := h
    exact Nat.strongRecOn x.length
      (motive := fun n =>
        ∀ w, w.length = n → w ∈ M.accepts → ∃ u, u.length < Fintype.card σ ∧ u ∈ M.
    accepts)
      (fun n ih w hwlen hw => by
        by_cases hlt : n < Fintype.card σ
        · exact ⟨w, hwlen ▸ hlt, hw⟩
        · have hge : Fintype.card σ ≤ n := Nat.not_lt.mp hlt
          obtain ⟨y, hy, hylt⟩ := accepts_shorten_step hw (hwlen.symm ▸ hge)
          exact ih y.length (Nat.lt_of_lt_of_eq hylt hwlen) y rfl hy)
      x rfl hx

end ShortWord

end

section
variable {σ₁ σ₂ : Type}

def toNFA : (e : SExpr) → NFA Bool (autState e)
  | .sigma => sigmaDFA.toNFA
  | .single σ => (singleDFA σ).toNFA
  | .cap a b => NFAinter (toNFA a) (toNFA b)
  | .cat a b => (catEps (toNFA a) (toNFA b)).toNFA

theorem toNFA_accepts : (e : SExpr) → (toNFA e).accepts = denote e
  | .sigma => by
      change sigmaDFA.toNFA.accepts = denote .sigma
      rw [dfaToNFA_accepts sigmaDFA, sigmaDFA_accepts, denote_sigma]
  | .single σ => by
      change (singleDFA σ).toNFA.accepts = denote (.single σ)
```

```
      rw [dfaToNFA_accepts (singleDFA σ)]
      exact singleDFA_accepts_denote σ
  | .cap a b => by
      ext x
      change x ∈ (NFAinter (toNFA a) (toNFA b)).accepts ↔ _
      rw [NFAinter_mem_accepts_iff, toNFA_accepts a, toNFA_accepts b, denote_cap]
      exact (Set.mem_inter_iff x (denote a) (denote b)).symm
  | .cat a b => by
      change (catEps (toNFA a) (toNFA b)).toNFA.accepts = denote (.cat a b)
      rw [?NFA.toNFA_correct, catEps_accepts, toNFA_accepts a, toNFA_accepts b,
    denote_cat]

end

section
variable {σ₁ σ₂ : Type}
variable {σ : Type}

section ShortWord

variable {σ : Type} [Fintype σ] {M : NFA Bool σ}

theorem denote_nonempty_iff_short (e : SExpr) :
    (denote e).Nonempty ↔ ∃ w ∈ wordsUpTo (autStateCard e), matchesB e w = true := by
  have hc : autStateCard e = Fintype.card (autState e) := autStateCard_eq_card e
  constructor
  · intro hne
    rw [← toNFA_accepts e] at hne
    obtain ⟨w, hwlt, hw⟩ := exists_accepted_word_short hne
    refine ⟨w, ?_, ?_⟩
    · rw [mem_wordsUpTo, hc]; omega
    · rw [matchesB_iff, ← toNFA_accepts e]; exact hw
  · intro ⟨w, hwmem, hmatch⟩
    rw [mem_wordsUpTo, hc] at hwmem
    exact ⟨w, (matchesB_iff e w).mp hmatch⟩

end ShortWord

@[simp] theorem decideNonemptyB_iff (e : SExpr) :
    decideNonemptyB e = true ↔ (denote e).Nonempty := by
  simp [decideNonemptyB, anyMatchesB, List.any_eq_true, denote_nonempty_iff_short]

@[simp] theorem decideEmptyB_iff (e : SExpr) :
    decideEmptyB e = true ↔ denote e = ∅ := by
  rw [decideEmptyB, Bool.not_eq_true_eq_eq_false, ← Bool.eq_false_eq_not_eq_true,
    decideNonemptyB_iff, Set.not_nonempty_iff_eq_empty]

instance decidableEmptyDenote (e : SExpr) : Decidable (denote e = ∅) :=
  if h : decideEmptyB e then .isTrue (decideEmptyB_iff e |>.mp h)
  else .isFalse fun he => h (decideEmptyB_iff e |>.mpr he)

#eval decideEmptyB (.cap (.single [false]) (.single [true]))

def consistentB (a b : SExpr) : Bool := !decideEmptyB (.cap a b)

end
```

```
section
variable {σ₁ σ₂ : Type}

example : DecidableEq (autState (.sigma : SExpr)) := inferInstance
example : DecidableEq (autState (.single [true, false] : SExpr)) := inferInstance

variable {σ : Type}

@[simp] theorem consistentB_iff (a b : SExpr) :
    consistentB a b = true ↔ (denote (.cap a b)).Nonempty := by
  have h : consistentB a b = decideNonemptyB (.cap a b) := by
    simp [consistentB, decideEmptyB, decideNonemptyB]
  rw [h, decideNonemptyB_iff]

@[simp] theorem capNonempty_iff_consistent (a b : SExpr) :
    (denote a ∩ denote b).Nonempty ↔ consistentB a b = true := by
  rw [← denote_cap, consistentB_iff]
end

section
variable {σ₁ σ₂ : Type}

theorem autStateCard_le_card (e : SExpr) : autStateCard e ≤ Fintype.card (autState e)
    := by
  induction e with
  | sigma =>
    simp [autStateCard, autState]
    exact (Fintype.card_unique (α := Unit)).ge
  | single σ =>
    have heq : autStateCard (.single σ) = Fintype.card (autState (.single σ)) := by
      simp only [autStateCard, autState]
      have h1 : Fintype.card (Option (Fin (σ.length + 1))) =
          Fintype.card (Fin (σ.length + 1)) + 1 :=
        Fintype.card_option (α := Fin (σ.length + 1))
      have h2 : Fintype.card (Fin (σ.length + 1)) = σ.length + 1 :=
        Fintype.card_fin (σ.length + 1)
      calc σ.length + 2
          = (σ.length + 1) + 1 := rfl
        _ = Fintype.card (Fin (σ.length + 1)) + 1 := (congrArg (fun n => n + 1) h2).
    symm
        _ = Fintype.card (Option (Fin (σ.length + 1))) := h1.symm
    exact Nat.le_of_eq heq
  | cap a b ih_a ih_b =>
    simp only [autStateCard, autState]
    exact (Nat.mul_le_mul ih_a ih_b).trans (Fintype.card_prod (autState a) (autState b
    )).ge
  | cat a b ih_a ih_b =>
    simp only [autStateCard, autState]
    exact (Nat.add_le_add ih_a ih_b).trans (Fintype.card_sum (α := autState a) (β :=
    autState b)).ge
end

end RegularLanguage
end DomainTheory.NeighbourhoodSystems
end
```

4.8.53 Regular-language codes

Gödel coding and primitive-recursive decision procedures for the regular-language expressions of Exercise 7.22.

Theorem (`consistentB_iff_Ssys`). The condition consistent b holds exactly when ssys holds.

Proof. Expand both conditions in consistent b iff ssys; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`example_722_decide`). Scott's example example 722 decide has the claimed language-theoretic or computability property.

Proof. Following the defining recursion for example 722 decide and applying the preceding component lemmas yields the claimed result. ■

Theorem (`denote_eq_empty_iff`). The set or language in denote eq empty iff is empty exactly under the stated absence-of-witness condition.

Proof. An element of the set in denote eq empty iff would provide exactly the forbidden witness, while any such witness produces an element; hence emptiness is equivalent to its absence. ■

Theorem (`nfa_accepts_nonempty_iff`). The set or language nfa accepts nonempty iff has an element under the hypotheses of the theorem.

Proof. Choose witnesses from the nonempty input sets in nfa accepts nonempty iff and combine them using the defining construction. ■

Theorem (`nfa_accepts_nonempty_iff_short`). The language in nfa accepts nonempty iff short is nonempty exactly when it accepts a word shorter than the number of available automaton states.

Proof. Repeatedly remove a loop from any longer accepting path for nfa accepts nonempty iff short; finiteness of the state set guarantees a repeated state until a short accepted word remains. ■

Theorem (`pathStateAt_last`). For the objects in path state at last, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for path state at last and applying the preceding component lemmas yields the claimed result. ■

Definition (`acceptFin`). The finite accepting-state set of an expression automaton is computed recursively from the expression.

Definition (`startFin`). The finite initial-state set of an expression automaton is computed recursively from the expression.

Definition (`stepFinSingle`). The state transformer step fin single performs one stage of the finite numerical computation encoded in its state.

Definition (`stepFin`). The state transformer step fin performs one stage of the finite numerical computation encoded in its state.

Definition (`diffNFA`). The automaton diff nfa has the recursively specified state space, transitions, initial states, and accepting states.

Definition (`evalFin`). The finite state set reached after reading a word is obtained by folding the finite transition operation from the finite start set.

Definition (`subsetB`). The set subset b consists exactly of the elements admitted by its defining membership condition.

Definition (`interEqB`). Language equality is tested by checking inclusion in both directions.

Theorem (`coe_acceptFin`). For the objects in accept fin, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for accept fin and applying the preceding component lemmas yields the claimed result. ■

Theorem (`coe_startFin`). For the objects in start fin, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for start fin and applying the preceding component lemmas yields the claimed result. ■

Theorem (`coe_stepFinSingle`). For the objects in step fin single, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for step fin single and applying the preceding component lemmas yields the claimed result. ■

Theorem (`coe_stepFin`). For the objects in step fin, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for step fin and applying the preceding component lemmas yields the claimed result. ■

Theorem (`coe_foldl_stepFin`). For the objects in foldl step fin, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for foldl step fin and applying the preceding component lemmas yields the claimed result. ■

Theorem (`coe_evalFin`). Evaluating the automaton or finite transition system in eval fin yields the stated reachable state after reading the word.

Proof. Induction on the input word tracks the reachable states for eval fin; the accepting-state condition then gives the stated language. ■

Theorem (`denote_iff_evalFin`). Evaluating the automaton or finite transition system in denote iff eval fin yields the stated reachable state after reading the word.

Proof. Induction on the input word tracks the reachable states for denote iff eval fin; the accepting-state condition then gives the stated language. ■

Theorem (`diffNFA_evalFrom`). Evaluating the automaton or finite transition system in diff nfa eval from yields the stated reachable state after reading the word.

Proof. Induction on the input word tracks the reachable states for diff nfa eval from; the accepting-state condition then gives the stated language. ■

Theorem (`diffNFA_eval`). Evaluating the automaton or finite transition system in diff nfa eval yields the stated reachable state after reading the word.

Proof. Induction on the input word tracks the reachable states for diff nfa eval; the accepting-state condition then gives the stated language. ■

Theorem (`diffNFA_mem_accepts_iff`). Membership in diff nfa mem accepts iff is equivalent to the explicit witness condition given by the construction.

Proof. Expanding the membership definition of diff nfa mem accepts iff exposes exactly the witnesses and equations occurring on the other side. ■

Theorem (`exists_diff_word_short`). The language in diff word short is nonempty exactly when it accepts a word shorter than the number of available automaton states.

Proof. Repeatedly remove a loop from any longer accepting path for diff word short; finiteness of the state set guarantees a repeated state until a short accepted word remains. ■

Theorem (`subsetB_iff`). The condition subset b holds exactly when the finite witness condition holds.

Proof. Expand both conditions in subset b iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`interEqB_iff`). The intersection in inter eq b iff equals the indexed neighbourhood or language exactly under the stated finite comparison condition.

Proof. Use the same witnesses in both directions for inter eq b iff, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`example_722_equiv`). Scott's example example 722 equiv has the claimed language-theoretic or computability property.

Proof. Compare membership or numerical components in example 722 equiv; the defining equations reduce both sides to the same data. ■

Definition (`autStateCardFuelCharBody`). This is the one-step numerical recurrence computing the number of automaton states represented by an expression code.

Definition (`autStateCardFuelChar`). This fuel-bounded numerical function computes the automaton-state count associated with an expression code.

Definition (`c9b5_boolNat`). A Boolean digit is numerically encoded by zero or one.

Definition (`c9b5_encodeListBool`). The coding function c9b5 encode list bool assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`c9b5_sexprDepth`). The coded expression depth is computed recursively, with leaves of depth one and binary depth one plus the maximum child depth.

Definition (`c9b5_sexprGodelEncode`). The coding function c9b5 sexpr godel encode assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`capCode`). The coding function cap code assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`autStateCardFuelCharLookup`). The numerical characteristic function aut state card fuel char lookup encodes its Boolean or relational test by zero and one.

Definition (`subsetBoundChar`). The numerical characteristic function subset bound char encodes its Boolean or relational test by zero and one.

Definition (`subsetGuardChar`). The numerical characteristic function subset guard char encodes its Boolean or relational test by zero and one.

Definition (`matchesBCatG`). The bounded concatenation-search function tests all splits of a coded word using the preceding matching approximation.

Definition (`matchesBCatGLookup`). The lookup form unpacks a single paired argument and applies the bounded concatenation-search function.

Definition (`matchesBCharBody`). The numerical characteristic function matches bchar body encodes its Boolean or relational test by zero and one.

Definition (`matchesBChar`). This fuel-bounded characteristic function returns one exactly when the coded word matches the coded expression.

Definition (`decideNonemptyBChar`). The numerical characteristic function decide nonempty bchar encodes its Boolean or relational test by zero and one.

Definition (`consistentBChar`). The numerical characteristic function consistent bchar encodes its Boolean or relational test by zero and one.

Definition (`matchesBCharBodyLookup`). The numerical characteristic function matches bchar body lookup encodes its Boolean or relational test by zero and one.

Definition (`subsetBChar`). This bounded characteristic function tests inclusion between the languages denoted by two expression codes.

Definition (`interEqChar`). This bounded characteristic function tests equality between the languages denoted by two expression codes.

Definition (`c9b6_decodeListBool`). The decoding operation c9b6 decode list bool recovers the finite list, expression, or tagged value represented by its numerical input, returning failure when the code is malformed.

Theorem (`c9b5_decodeList_encodeListBool`). For the objects in c9b5 decode list encode list bool, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for c9b5 decode list encode list bool and applying the preceding component lemmas yields the claimed result. ■

Theorem (`c9b5_boolNat_injective`). The construction c9b5 bool nat injective sends equal outputs only from equal inputs.

Proof. Equality of two values of c9b5 bool nat injective determines their outer form and then each component, so the original inputs coincide. ■

Theorem (`c9b5_decodeList_injective`). The construction c9b5 decode list injective sends equal outputs only from equal inputs.

Proof. Equality of two values of c9b5 decode list injective determines their outer form and then each component, so the original inputs coincide. ■

Theorem (`c9b5_map_boolNat_eq_iff`). The condition c9b5 map bool nat eq holds exactly when the finite witness condition holds.

Proof. Expand both conditions in c9b5 map bool nat eq iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`c9b5_takeCode_encodeListBool`). For the objects in c9b5 take code encode list bool, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for c9b5 take code encode list bool and applying the preceding component lemmas yields the claimed result. ■

Theorem (`c9b5_dropCode_encodeListBool`). For the objects in c9b5 drop code encode list bool, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for c9b5 drop code encode list bool and applying the preceding component lemmas yields the claimed result. ■

Theorem (`c9b5_listEqChar_encodeListBool`). For the objects in c9b5 list eq char encode list bool, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for c9b5 list eq char encode list bool and applying the preceding component lemmas yields the claimed result. ■

Theorem (`autStateCardFuelCharBody_eq`). The value of aut state card fuel char body eq equals the value described by aut state card fuel char body eq.

Proof. Compare membership or numerical components in aut state card fuel char body eq; the defining equations reduce both sides to the same data. ■

Theorem (`autStateCardFuelChar_eq_autStateCard`). The value of aut state card fuel char equals the value described by aut state card.

Proof. Compare membership or numerical components in aut state card fuel char eq aut state card; the defining equations reduce both sides to the same data. ■

Theorem (`primrec_autStateCardFuelCharBody`). The numerical function aut state card fuel char body is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of aut state card fuel char body. ■

Theorem (`autStateCardFuelCharBody_local`). For the objects in aut state card fuel char body local, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for aut state card fuel char body local and applying the preceding component lemmas yields the claimed result. ■

Theorem (`matchesBCharBody_eq`). The value of matches bchar body eq equals the value described by matches bchar body eq.

Proof. Compare membership or numerical components in matches bchar body eq; the defining equations reduce both sides to the same data. ■

Theorem (`c9b5_matchesBCatG_char`). For the objects in c9b5 matches bcat g char, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for c9b5 matches bcat g char and applying the preceding component lemmas yields the claimed result. ■

Theorem (`matchesBChar_eq_one_iff`). The zero–one test matches bchar returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in matches bchar eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`primrec_matchesBCatG`). The numerical function matches bcat g is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of matches bcat g. ■

Theorem (`primrec_matchesBCharBody`). The numerical function matches bchar body is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of matches bchar body. ■

Theorem (`matchesBChar_le_one`). For every input, matches bchar takes one of the two values zero and one.

Proof. Each Boolean-valued component of matches bchar le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`matchesBCharBody_local`). For the objects in matches bchar body local, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for matches bchar body local and applying the preceding component lemmas yields the claimed result. ■

Theorem (`c9b6_allBinDigitsChar_encodeListBool`). For the objects in c9b6 all bin digits char encode list bool, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for c9b6 all bin digits char encode list bool and applying the preceding component lemmas yields the claimed result. ■

Theorem (`c9b6_encodeListBool_lt_codeBound`). The natural number c9b6 encode list bool lt code bound lies strictly below its finite bound.

Proof. Following the defining recursion for c9b6 encode list bool lt code bound and applying the preceding component lemmas yields the claimed result. ■

Theorem (`c9b6_boolNat_decide_eq`). The value of c9b6 bool nat decide eq equals the value described by c9b6 bool nat decide eq.

Proof. Compare membership or numerical components in c9b6 bool nat decide eq; the defining equations reduce both sides to the same data. ■

Theorem (`c9b6_encodeListBool_decodeListBool_of_allBin`). The hypotheses all bin imply c9b6 encode list bool decode list bool.

Proof. Following the defining recursion for c9b6 encode list bool decode list bool of all bin and applying the preceding component lemmas yields the claimed result. ■

Theorem (`c9b6_matchesBChar_of_allBin`). The hypotheses all bin imply c9b6 matches bchar.

Proof. Following the defining recursion for c9b6 matches bchar of all bin and applying the preceding component lemmas yields the claimed result. ■

Theorem (`decideNonemptyBChar_eq_one_iff`). The zero–one test decide nonempty bchar returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in decide nonempty bchar eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`c9b6_sexprGodelEncode_cap`). For the objects in c9b6 sexpr godel encode cap, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for c9b6 sexpr godel encode cap and applying the preceding component lemmas yields the claimed result. ■

Theorem (`subsetGuardChar_le_one`). For every input, subset guard char takes one of the two values zero and one.

Proof. Each Boolean-valued component of subset guard char le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`subsetBChar_eq_one_iff`). The zero–one test subset bchar returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in subset bchar eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`consistentBChar_eq_one_iff`). The zero–one test consistent bchar returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in consistent bchar eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`decideNonemptyBChar_le_one`). For every input, decide nonempty bchar takes one of the two values zero and one.

Proof. Each Boolean-valued component of decide nonempty bchar le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`consistentBChar_le_one`). For every input, consistent bchar takes one of the two values zero and one.

Proof. Each Boolean-valued component of consistent bchar le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`autStateCardFuelChar_fuelTable_eq`). The value of aut state card fuel char fuel table eq equals the value described by aut state card fuel char fuel table eq.

Proof. Compare membership or numerical components in aut state card fuel char fuel table eq; the defining equations reduce both sides to the same data. ■

Theorem (`interEqChar_eq_one_iff`). The zero–one test inter eq char returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in inter eq char eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`matchesBChar_fuelTable_eq`). The value of matches bchar fuel table eq equals the value described by matches bchar fuel table eq.

Proof. Compare membership or numerical components in matches bchar fuel table eq; the defining equations reduce both sides to the same data. ■

Theorem (`primrec_autStateCardFuelChar`). The numerical function aut state card fuel char is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of aut state card fuel char. ■

Theorem (`primrec_autStateCardFuelCharLookup`). The numerical function aut state card fuel char lookup is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of aut state card fuel char lookup. ■

Theorem (`primrec_autStateCardFuelChar2`). The numerical function aut state card fuel char2 is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of aut state card fuel char2. ■

Theorem (`primrec_matchesBCatGLookup`). The numerical function matches bcat glookup is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of matches bcat glookup. ■

Theorem (`primrec_matchesBChar`). The numerical function matches bchar is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of matches bchar. ■

Theorem (`primrec_decideNonemptyBChar`). The numerical function decide nonempty bchar is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of decide nonempty bchar. ■

Theorem (`primrec_consistentBChar`). The numerical function consistent bchar is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of consistent bchar. ■

Theorem (`primrec_matchesBCharBodyLookup`). The numerical function matches bchar body lookup is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of matches bchar body lookup. ■

Theorem (`primrec_matchesBChar2`). The numerical function matches bchar2 is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of matches bchar2. ■

Theorem (`primrec_decideNonemptyBChar2`). The numerical function decide nonempty bchar2 is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of decide nonempty bchar2. ■

Theorem (`primrec_consistentBChar2`). The numerical function consistent bchar2 is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of consistent bchar2. ■

Theorem (`primrec_subsetBChar`). The numerical function subset bchar is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of subset bchar. ■

Theorem (`primrec_interEqChar`). The numerical function inter eq char is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of inter eq char. ■

Theorem (`subsetBChar_le_one`). For every input, subset bchar takes one of the two values zero and one.

Proof. Each Boolean-valued component of subset bchar le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`interEqChar_le_one`). For every input, inter eq char takes one of the two values zero and one.

Proof. Each Boolean-valued component of inter eq char le one is zero or one, and the defining Boolean combination preserves that range. ■

Definition (`ConsistencyPresentation`). A consistency presentation consists of an enumeration of all neighbourhoods, proofs that the enumerated sets are neighbourhoods, surjectivity of the enumeration, and a recursive decision procedure for finite consistency.

`regularCode.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .rePred
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .regularLanguage
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.prodIso

public import Mathlib.Computability.Primrec.Basic

public import Mathlib.Data.Nat.Bitwise

import Mathlib.Tactic.Ring

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace RegularLanguage
open scoped Computability
open Sum Set
section
variable {σ₁ σ₂ : Type}
variable {σ : Type}

theorem consistentB_iff_Ssys (a b : SExpr) (ha : InS (denote a)) (hb : InS (denote b))
     :
    consistentB a b = true ↔ InS (denote a ∩ denote b) := by
  rw [← denote_cap, consistentB_iff]
  exact Iff.symm (Ssys_isPositive (Ssys_mem.mpr ha) (Ssys_mem.mpr hb))
end

end RegularLanguage
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.RegularLanguage
set_option linter.unusedVariables false

theorem example_722_decide : ∀ (a b : DomainTheory.NeighbourhoodSystems.RegularLanguage
    .SExpr),
```

```
  DomainTheory.NeighbourhoodSystems.RegularLanguage.InS (DomainTheory.
    NeighbourhoodSystems.RegularLanguage.denote a) →
    DomainTheory.NeighbourhoodSystems.RegularLanguage.InS (DomainTheory.
    NeighbourhoodSystems.RegularLanguage.denote b) →
      Iff (@Eq Bool (DomainTheory.NeighbourhoodSystems.RegularLanguage.consistentB a b
    ) Bool.true)
        (DomainTheory.NeighbourhoodSystems.RegularLanguage.InS
          (@Inter.inter (Set (List Bool)) (@Set.instInter (List Bool))
            (DomainTheory.NeighbourhoodSystems.RegularLanguage.denote a)
            (DomainTheory.NeighbourhoodSystems.RegularLanguage.denote b))) :=
  DomainTheory.NeighbourhoodSystems.RegularLanguage.consistentB_iff_Ssys

end DomainTheory.NeighbourhoodSystems.RegularLanguage
namespace DomainTheory.NeighbourhoodSystems
namespace RegularLanguage
open scoped Computability
open Sum Set

section
variable {σ₁ σ₂ : Type}
variable {σ : Type}

theorem denote_eq_empty_iff (e : SExpr) :
    denote e = ∅ ↔ ∀ s ∈ (toNFA e).accept, ∀ x, s ∈ (toNFA e).eval x := by
  rw [Set.eq_empty_iff_forall_notMem]
  constructor
  · intro h s hs x hse
    exact h x (by rw [← toNFA_accepts e]; exact ⟨s, hs, hse⟩)
  · intro h x hx
    rw [← toNFA_accepts e] at hx
    obtain ⟨s, hs, hse⟩ := hx
    exact h s hs x hse

theorem nfa_accepts_nonempty_iff (M : NFA Bool σ) :
    M.accepts.Nonempty ↔ ∃ s ∈ M.accept, ∃ x, s ∈ M.eval x := by
  constructor
  · rintro ⟨x, S, hS, hSe⟩; exact ⟨S, hS, x, hSe⟩
  · rintro ⟨S, hS, x, hSe⟩; exact ⟨x, S, hS, hSe⟩

section ShortWord

variable {σ : Type} [Fintype σ] {M : NFA Bool σ}

theorem nfa_accepts_nonempty_iff_short :
    M.accepts.Nonempty ↔ ∃ w, w ∈ wordsUpTo (Fintype.card σ) ∧ w ∈ M.accepts := by
  constructor
  · intro h
    obtain ⟨w, hwlt, hw⟩ := exists_accepted_word_short h
    refine ⟨w, ?_, hw⟩
    rw [mem_wordsUpTo]
    omega
  · intro ⟨w, _, hw⟩
    exact ⟨w, hw⟩

omit [Fintype σ] in
theorem pathStateAt_last {s t : σ} {x : List Bool} (p : M.Path s t x) :
```

```
    pathStateAt p x.length (Nat.le_refl _) = t := by
  induction p with
  | nil s => simp [pathStateAt, pathStateAtAux]
  | cons sMid s' u a xs hstep p' ih =>
    simp [pathStateAt, pathStateAtAux, List.length_cons]
    exact ih

end ShortWord

end

def acceptFin : (e : SExpr) → Finset (autState e)
  | .sigma => Finset.univ
  | .single σ => {(some ⟨σ.length, Nat.lt_succ_self _⟩ : SingleState σ)}
  | .cap a b => (acceptFin a) ×ˢ (acceptFin b)
  | .cat a b => (acceptFin b).image (inr : autState b → autState a ? autState b)

def startFin : (e : SExpr) → Finset (autState e)
  | .sigma => Finset.univ
  | .single σ => {(some ⟨0, Nat.succ_pos _⟩ : SingleState σ)}
  | .cap a b => (startFin a) ×ˢ (startFin b)
  | .cat a b =>
      if ((startFin a) ∩ (acceptFin a)).Nonempty
      then (startFin a).image (inl : autState a → autState a ? autState b) ∪
        (startFin b).image (inr : autState b → autState a ? autState b)
      else (startFin a).image (inl : autState a → autState a ? autState b)

def stepFinSingle : (e : SExpr) → autState e → Bool → Finset (autState e)
  | .sigma => fun _ _ => Finset.univ
  | .single σ => fun s c => {(singleDFA σ).step s c}
  | .cap a b => fun s c => (stepFinSingle a s.1 c) ×ˢ (stepFinSingle b s.2 c)
  | .cat a b => fun s c =>
      match s with
      | inl s' =>
          if ((stepFinSingle a s' c) ∩ (acceptFin a)).Nonempty
          then (stepFinSingle a s' c).image (inl : autState a → autState a ? autState
    b) ∪
            (startFin b).image (inr : autState b → autState a ? autState b)
          else (stepFinSingle a s' c).image (inl : autState a → autState a ? autState
    b)
      | inr s' => (stepFinSingle b s' c).image (inr : autState b → autState a ?
    autState b)

def stepFin (e : SExpr) (S : Finset (autState e)) (c : Bool) : Finset (autState e) :=
  S.biUnion (fun s => stepFinSingle e s c)

def diffNFA (e1 e2 : SExpr) : NFA Bool (Finset (autState e1) × Finset (autState e2))
    where
  step S c := {(stepFin e1 S.1 c, stepFin e2 S.2 c)}
  start := {(startFin e1, startFin e2)}
  accept := {S | (S.1 ∩ acceptFin e1).Nonempty ∧ S.2 ∩ acceptFin e2 = ∅}

def evalFin (e : SExpr) (w : List Bool) : Finset (autState e) :=
  w.foldl (stepFin e) (startFin e)

def subsetB (e1 e2 : SExpr) : Bool :=
```

```
  (wordsUpTo (Fintype.card (Finset (autState e1) × Finset (autState e2)))).all
    (fun w => !matchesB e1 w || matchesB e2 w)

def interEqB (e1 e2 : SExpr) : Bool := subsetB e1 e2 && subsetB e2 e1

theorem coe_acceptFin : ∀ e : SExpr, (↑(acceptFin e) : Set (autState e)) = (toNFA e).
    accept
  | .sigma => by
      show (↑(Finset.univ : Finset Unit) : Set Unit) = sigmaDFA.toNFA.accept
      show (↑(Finset.univ : Finset Unit) : Set Unit) = Set.univ
      ext x; cases x; simp
  | .single σ => by
      change (↑({(some ⟨σ.length, Nat.lt_succ_self _⟩ : SingleState σ)} : Finset _) :
    Set _)
        = (singleDFA σ).toNFA.accept
      simp [DFA.toNFA, singleDFA]
  | .cap a b => by
      change (↑((acceptFin a) ×ˢ (acceptFin b)) : Set (autState a × autState b))
        = (NFAinter (toNFA a) (toNFA b)).accept
      rw [Finset.coe_product, coe_acceptFin a, coe_acceptFin b]
      rfl
  | .cat a b => by
      show (↑((acceptFin b).image (inr : autState b → autState a ? autState b)) :
        Set (autState a ? autState b)) = (catEps (toNFA a) (toNFA b)).toNFA.accept
      show (↑((acceptFin b).image (inr : autState b → autState a ? autState b)) :
        Set (autState a ? autState b)) = inr '' (toNFA b).accept
      rw [Finset.coe_image, coe_acceptFin b]

theorem coe_startFin : ∀ e : SExpr, (↑(startFin e) : Set (autState e)) = (toNFA e).
    start
  | .sigma => by
      show (↑(Finset.univ : Finset Unit) : Set Unit) = sigmaDFA.toNFA.start
      show (↑(Finset.univ : Finset Unit) : Set Unit) = {sigmaDFA.start}
      ext x; cases x; simp
  | .single σ => by
      show (↑({(some ⟨0, Nat.succ_pos _⟩ : SingleState σ)} : Finset (SingleState σ)) :
    Set _)
        = (singleDFA σ).toNFA.start
      simp [DFA.toNFA, singleDFA]
  | .cap a b => by
      show (↑((startFin a) ×ˢ (startFin b)) : Set (autState a × autState b))
        = (NFAinter (toNFA a) (toNFA b)).start
      rw [Finset.coe_product, coe_startFin a, coe_startFin b]
      rfl
  | .cat a b => by
      show (↑(if ((startFin a) ∩ (acceptFin a)).Nonempty
          then (startFin a).image (inl : autState a → autState a ? autState b) ∪
            (startFin b).image (inr : autState b → autState a ? autState b)
          else (startFin a).image (inl : autState a → autState a ? autState b)) :
          Set (autState a ? autState b)) = (catEps (toNFA a) (toNFA b)).toNFA.start
      show _ = (catEps (toNFA a) (toNFA b)).?Closure (catEps (toNFA a) (toNFA b)).
    start
      rw [catEps_start]
      by_cases hne : ((startFin a) ∩ (acceptFin a)).Nonempty
      · rw [ite_eq_left hne]
        ext x
```

```
      rw [catEps_mem_?Closure_iff, ← coe_startFin a, ← coe_acceptFin a, ←
    coe_startFin b]
      simp only [Finset.coe_union, Finset.coe_image, mem_union, mem_image, Finset.
    mem_coe]
      constructor
      · rintro (⟨q, hq, rfl⟩ | ⟨q, hq, rfl⟩)
        · exact Or.inl ⟨q, hq, rfl⟩
        · refine Or.inr ⟨?_, q, hq, rfl⟩
          obtain ⟨q0, hq0⟩ := hne
          rw [Finset.mem_inter] at hq0
          exact ⟨q0, hq0.2, q0, hq0.1, rfl⟩
      · rintro (⟨q, hq, rfl⟩ | ⟨_, q, hq, rfl⟩)
        · exact Or.inl ⟨q, hq, rfl⟩
        · exact Or.inr ⟨q, hq, rfl⟩
    · rw [ite_eq_right hne]
      ext x
      rw [catEps_mem_?Closure_iff, ← coe_startFin a, ← coe_acceptFin a, ←
    coe_startFin b]
      simp only [Finset.coe_image, mem_image, Finset.mem_coe]
      constructor
      · rintro ⟨q, hq, rfl⟩; exact Or.inl ⟨q, hq, rfl⟩
      · rintro (⟨q, hq, rfl⟩ | ⟨⟨q, hq1, q', hq2, heq⟩, _⟩)
        · exact ⟨q, hq, rfl⟩
        · cases heq
          exact absurd ⟨q, Finset.mem_inter.mpr ⟨hq2, hq1⟩⟩ hne

theorem coe_stepFinSingle : ∀ (e : SExpr) (s : autState e) (c : Bool),
    (↑(stepFinSingle e s c) : Set (autState e)) = (toNFA e).step s c
  | .sigma, s, c => by
    show (↑(Finset.univ : Finset Unit) : Set Unit) = sigmaDFA.toNFA.step s c
    show (↑(Finset.univ : Finset Unit) : Set Unit) = {sigmaDFA.step s c}
    ext x; cases x; simp
  | .single σ, s, c => by
    show (↑({(singleDFA σ).step s c} : Finset (SingleState σ)) : Set _)
      = (singleDFA σ).toNFA.step s c
    simp [DFA.toNFA]
  | .cap a b, (s1, s2), c => by
    show (↑((stepFinSingle a s1 c) ×ˢ (stepFinSingle b s2 c)) : Set (autState a ×
    autState b))
      = (NFAinter (toNFA a) (toNFA b)).step (s1, s2) c
    rw [Finset.coe_product, coe_stepFinSingle a s1 c, coe_stepFinSingle b s2 c]
    rfl
  | .cat a b, inl s, c => by
    show (↑(if ((stepFinSingle a s c) ∩ (acceptFin a)).Nonempty
        then (stepFinSingle a s c).image (inl : autState a → autState a ? autState b
    ) ∪
          (startFin b).image (inr : autState b → autState a ? autState b)
        else (stepFinSingle a s c).image (inl : autState a → autState a ? autState b
    )) :
        Set (autState a ? autState b)) = (catEps (toNFA a) (toNFA b)).toNFA.step (
    inl s) c
    show _ = (catEps (toNFA a) (toNFA b)).?Closure
      ((catEps (toNFA a) (toNFA b)).step (inl s) (some c))
    rw [catEps_step_inl_some]
    by_cases hne : ((stepFinSingle a s c) ∩ (acceptFin a)).Nonempty
    · rw [ite_eq_left hne]
```

```
        ext x
        rw [catEps_mem_?Closure_iff, ← coe_stepFinSingle a s c, ← coe_acceptFin a,
          ← coe_startFin b]
        simp only [Finset.coe_union, Finset.coe_image, mem_union, mem_image, Finset.
    mem_coe]
        constructor
        · rintro (⟨q, hq, rfl⟩ | ⟨q, hq, rfl⟩)
          · exact Or.inl ⟨q, hq, rfl⟩
          · refine Or.inr ⟨?_, q, hq, rfl⟩
            obtain ⟨q0, hq0⟩ := hne
            rw [Finset.mem_inter] at hq0
            exact ⟨q0, hq0.2, q0, hq0.1, rfl⟩
        · rintro (⟨q, hq, rfl⟩ | ⟨_, q, hq, rfl⟩)
          · exact Or.inl ⟨q, hq, rfl⟩
          · exact Or.inr ⟨q, hq, rfl⟩
      · rw [ite_eq_right hne]
        ext x
        rw [catEps_mem_?Closure_iff, ← coe_stepFinSingle a s c, ← coe_acceptFin a,
          ← coe_startFin b]
        simp only [Finset.coe_image, mem_image, Finset.mem_coe]
        constructor
        · rintro ⟨q, hq, rfl⟩; exact Or.inl ⟨q, hq, rfl⟩
        · rintro (⟨q, hq, rfl⟩ | ⟨⟨q, hq1, q', hq2, heq⟩, _⟩)
          · exact ⟨q, hq, rfl⟩
          · cases heq
            exact absurd ⟨q, Finset.mem_inter.mpr ⟨hq2, hq1⟩⟩ hne
  | .cat a b, inr s, c => by
      show (↑((stepFinSingle b s c).image (inr : autState b → autState a ? autState b)
    ) :
        Set (autState a ? autState b)) = (catEps (toNFA a) (toNFA b)).toNFA.step (inr
    s) c
      show _ = (catEps (toNFA a) (toNFA b)).?Closure
        ((catEps (toNFA a) (toNFA b)).step (inr s) (some c))
      rw [catEps_step_inr_some]
      ext x
      rw [catEps_mem_?Closure_iff, ← coe_stepFinSingle b s c]
      simp only [Finset.coe_image, mem_image, Finset.mem_coe]
      constructor
      · rintro ⟨q, hq, rfl⟩; exact Or.inl ⟨q, hq, rfl⟩
      · rintro (⟨q, hq, rfl⟩ | ⟨⟨q, _, hqmem⟩, _⟩)
        · exact ⟨q, hq, rfl⟩
        · exact absurd hqmem (by simp)

theorem coe_stepFin (e : SExpr) (S : Finset (autState e)) (c : Bool) :
    (↑(stepFin e S c) : Set (autState e)) = (toNFA e).stepSet ↑S c := by
  ext x
  rw [Finset.mem_coe]
  unfold stepFin
  rw [Finset.mem_biUnion, NFA.mem_stepSet]
  constructor
  · rintro ⟨s, hs, hx⟩
    exact ⟨s, hs, by rw [← coe_stepFinSingle e s c]; exact Finset.mem_coe.mpr hx⟩
  · rintro ⟨s, hs, hx⟩
    exact ⟨s, hs, by rw [← coe_stepFinSingle e s c] at hx; exact Finset.mem_coe.mp hx⟩

theorem coe_foldl_stepFin (e : SExpr) (w : List Bool) (S : Finset (autState e)) :
```

```
    (↑(w.foldl (stepFin e) S) : Set (autState e)) = (toNFA e).evalFrom ↑S w := by
  induction w generalizing S with
  | nil => rfl
  | cons c w ih =>
    show (↑(w.foldl (stepFin e) (stepFin e S c)) : Set (autState e)) = _
    rw [ih (stepFin e S c), NFA.evalFrom_cons, coe_stepFin e S c]

theorem coe_evalFin (e : SExpr) (w : List Bool) :
    (↑(evalFin e w) : Set (autState e)) = (toNFA e).eval w := by
  show (↑(w.foldl (stepFin e) (startFin e)) : Set (autState e))
    = (toNFA e).evalFrom (toNFA e).start w
  rw [coe_foldl_stepFin e w (startFin e), coe_startFin e]

theorem denote_iff_evalFin (e : SExpr) (w : List Bool) :
    w ∈ denote e ↔ (evalFin e w ∩ acceptFin e).Nonempty := by
  rw [← toNFA_accepts e]
  constructor
  · rintro ⟨s, hs, hse⟩
    refine ⟨s, Finset.mem_inter.mpr ⟨?_, ?_⟩⟩
    · rw [← Finset.mem_coe, coe_evalFin]; exact hse
    · rw [← Finset.mem_coe, coe_acceptFin]; exact hs
  · rintro ⟨s, hs⟩
    rw [Finset.mem_inter] at hs
    obtain ⟨hs1, hs2⟩ := hs
    refine ⟨s, ?_, ?_⟩
    · rw [← Finset.mem_coe, coe_acceptFin] at hs2; exact hs2
    · rw [← Finset.mem_coe, coe_evalFin] at hs1; exact hs1

theorem diffNFA_evalFrom (e1 e2 : SExpr) (w : List Bool)
    (S1 : Finset (autState e1)) (S2 : Finset (autState e2)) :
    (diffNFA e1 e2).evalFrom {(S1, S2)} w =
      {(w.foldl (stepFin e1) S1, w.foldl (stepFin e2) S2)} := by
  induction w generalizing S1 S2 with
  | nil => rfl
  | cons c w ih =>
    show (diffNFA e1 e2).evalFrom ((diffNFA e1 e2).stepSet {(S1, S2)} c) w = _
    rw [NFA.stepSet_singleton]
    show (diffNFA e1 e2).evalFrom {(stepFin e1 S1 c, stepFin e2 S2 c)} w = _
    exact ih (stepFin e1 S1 c) (stepFin e2 S2 c)

theorem diffNFA_eval (e1 e2 : SExpr) (w : List Bool) :
    (diffNFA e1 e2).eval w = {(evalFin e1 w, evalFin e2 w)} := by
  show (diffNFA e1 e2).evalFrom (diffNFA e1 e2).start w = _
  show (diffNFA e1 e2).evalFrom {(startFin e1, startFin e2)} w = _
  exact diffNFA_evalFrom e1 e2 w (startFin e1) (startFin e2)

theorem diffNFA_mem_accepts_iff (e1 e2 : SExpr) (w : List Bool) :
    w ∈ (diffNFA e1 e2).accepts ↔ w ∈ denote e1 ∧ w ∈ denote e2 := by
  constructor
  · rintro ⟨s, hs, hse⟩
    rw [diffNFA_eval, Set.mem_singleton_iff] at hse
    subst hse
    obtain ⟨hacc1, hacc2⟩ := hs
    refine ⟨(denote_iff_evalFin e1 w).mpr hacc1, fun hmem => ?_⟩
    rw [denote_iff_evalFin] at hmem
    rw [hacc2] at hmem
```

```
    exact Finset.not_nonempty_empty hmem
  · rintro ⟨h1, h2⟩
    refine ⟨(evalFin e1 w, evalFin e2 w), ⟨((denote_iff_evalFin e1 w).mp h1, ?_⟩, ?_⟩
    · show evalFin e2 w ∩ acceptFin e2 = ∅
      by_contra hcontra
      exact h2 ((denote_iff_evalFin e2 w).mpr (Finset.nonempty_iff_ne_empty.mpr
    hcontra))
    · rw [diffNFA_eval]; rfl

theorem exists_diff_word_short {e1 e2 : SExpr} (h : ¬ denote e1 ⊆ denote e2) :
    ∃ w, w.length < Fintype.card (Finset (autState e1) × Finset (autState e2)) ∧
      w ∈ denote e1 ∧ w ∈ denote e2 := by
  rw [Set.not_subset] at h
  have hne : (diffNFA e1 e2).accepts.Nonempty := by
    obtain ⟨x, hx1, hx2⟩ := h
    exact ⟨x, (diffNFA_mem_accepts_iff e1 e2 x).mpr ⟨hx1, hx2⟩⟩
  obtain ⟨w, hlen, hmem⟩ := exists_accepted_word_short hne
  exact ⟨w, hlen, (diffNFA_mem_accepts_iff e1 e2 w).mp hmem⟩

theorem subsetB_iff (e1 e2 : SExpr) : subsetB e1 e2 = true ↔ denote e1 ⊆ denote e2 :=
     by
  unfold subsetB
  rw [List.all_eq_true]
  constructor
  · intro h
    by_contra hsub
    obtain ⟨w, hlen, hw1, hw2⟩ := exists_diff_word_short hsub
    have hwmem : w ∈ wordsUpTo (Fintype.card (Finset (autState e1) × Finset (autState
    e2))) :=
      (mem_wordsUpTo _).mpr (le_of_lt hlen)
    have hor := h w hwmem
    have he1 : matchesB e1 w = true := (matchesB_iff e1 w).mpr hw1
    rw [he1] at hor
    simp only [Bool.not_true, Bool.false_or] at hor
    exact hw2 ((matchesB_iff e2 w).mp hor)
  · intro hsub w _
    by_cases hw1 : w ∈ denote e1
    · have he2 : matchesB e2 w = true := (matchesB_iff e2 w).mpr (hsub hw1)
      simp [he2]
    · have he1 : matchesB e1 w = false := by
        rw [Bool.eq_false_iff]
        intro hc
        exact hw1 ((matchesB_iff e1 w).mp hc)
      simp [he1]

theorem interEqB_iff (e1 e2 : SExpr) : interEqB e1 e2 = true ↔ denote e1 = denote e2
    := by
  unfold interEqB
  rw [Bool.and_eq_true, subsetB_iff, subsetB_iff, ← Set.Subset.antisymm_iff]

end RegularLanguage
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.RegularLanguage
set_option linter.unusedVariables false

theorem example_722_equiv : ∀ (e1 e2 : DomainTheory.NeighbourhoodSystems.
```

```
    RegularLanguage.SExpr),

  Iff (@Eq Bool (DomainTheory.NeighbourhoodSystems.RegularLanguage.interEqB e1 e2)
    Bool.true)
     (@Eq (Set (List Bool)) (DomainTheory.NeighbourhoodSystems.RegularLanguage.denote
    e1)
       (DomainTheory.NeighbourhoodSystems.RegularLanguage.denote e2)) :=
  DomainTheory.NeighbourhoodSystems.RegularLanguage.interEqB_iff

end DomainTheory.NeighbourhoodSystems.RegularLanguage

namespace Domain.Recursive
open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

def autStateCardFuelCharBody (prev : ℕ → ℕ) (c : ℕ) : ℕ :=
  let subAdd := prev c.unpair.2.unpair.1 + prev c.unpair.2.unpair.2
  let subMul := prev c.unpair.2.unpair.1 * prev c.unpair.2.unpair.2
  selectFn (isOne (1 - c.unpair.1))
    1
    (selectFn (isOne (2 - c.unpair.1))
      (listLenChar c.unpair.2 + 2)
      (selectFn (isOne (3 - c.unpair.1))
        subAdd
        (selectFn (isOne (4 - c.unpair.1)) subMul 0)))

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

def autStateCardFuelChar : ℕ → ℕ → ℕ
  | 0, _ => 0
  | fuel + 1, c => autStateCardFuelCharBody (autStateCardFuelChar fuel) c

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage
```

```
def c9b5_boolNat (b : Bool) : ℕ := if b then 1 else 0

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

def c9b5_encodeListBool (σ : List Bool) : ℕ := encodeList (σ.map c9b5_boolNat)

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

def c9b5_sexprDepth : SExpr → ℕ
  | .sigma => 1
  | .single _ => 1
  | .cat a b => 1 + max (c9b5_sexprDepth a) (c9b5_sexprDepth b)
  | .cap a b => 1 + max (c9b5_sexprDepth a) (c9b5_sexprDepth b)

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

def c9b5_sexprGodelEncode : SExpr → ℕ
  | .sigma => Nat.pair 0 0
  | .single σ => Nat.pair 1 (c9b5_encodeListBool σ)
  | .cat a b => Nat.pair 2 (Nat.pair (c9b5_sexprGodelEncode a) (c9b5_sexprGodelEncode
    b))
  | .cap a b => Nat.pair 3 (Nat.pair (c9b5_sexprGodelEncode a) (c9b5_sexprGodelEncode
    b))

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

set_option maxHeartbeats 1600000 in
```

```
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def capCode (a b : ℕ) : ℕ := Nat.pair 3 (Nat.pair a b)

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

def autStateCardFuelCharLookup (w : ℕ) : ℕ :=
  autStateCardFuelCharBody (fun c' => nthCode w.unpair.1 c' 0) w.unpair.2

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def subsetBoundChar (fuel c1 c2 : ℕ) : ℕ :=
  2 ^ (autStateCardFuelChar fuel c1) * 2 ^ (autStateCardFuelChar fuel c2)

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def subsetGuardChar (fuel c1 c2 i : ℕ) : ℕ :=
  mulBit (allBinDigitsChar i) (isOne (1 - (listLenChar i - subsetBoundChar fuel c1 c2)
    ))
```

```
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

def matchesBCatG (prev : ℕ → ℕ) (p : ℕ) : ℕ :=
  let i := p.unpair.1
  let c := p.unpair.2.unpair.1
  let cw := p.unpair.2.unpair.2
  mulBit (prev (Nat.pair c.unpair.2.unpair.1 (takeCode i cw)))
         (prev (Nat.pair c.unpair.2.unpair.2 (dropCode i cw)))

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def matchesBCatGLookup (w : ℕ) : ℕ :=
  matchesBCatG (fun p' => nthCode w.unpair.1 p' 0) w.unpair.2

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

def matchesBCharBody (prev : ℕ → ℕ) (c cw : ℕ) : ℕ :=
  let catExist := bExistsFn (matchesBCatG prev) (Nat.pair c cw) (listLenChar cw + 1)
  let capAnd := mulBit (prev (Nat.pair c.unpair.2.unpair.1 cw))
                       (prev (Nat.pair c.unpair.2.unpair.2 cw))
  selectFn (isOne (1 - c.unpair.1))
    1
    (selectFn (isOne (2 - c.unpair.1))
      (listEqChar cw c.unpair.2)
      (selectFn (isOne (3 - c.unpair.1))
        catExist
        (selectFn (isOne (4 - c.unpair.1)) capAnd 0)))

set_option maxHeartbeats 800000 in
```

```
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

def matchesBChar : ℕ → ℕ → ℕ → ℕ
  | 0, _, _ => 0
  | fuel + 1, c, cw =>
      matchesBCharBody (fun p => matchesBChar fuel p.unpair.1 p.unpair.2) c cw

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def decideNonemptyBChar (fuel : ℕ) (c_e : ℕ) : ℕ :=
  bExistsFn (fun p =>
      mulBit (allBinDigitsChar p.unpair.1) (matchesBChar fuel p.unpair.2 p.unpair.1))
    c_e (codeBound (autStateCardFuelChar fuel c_e))

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def consistentBChar (fuel : ℕ) (c1 c2 : ℕ) : ℕ :=
  decideNonemptyBChar fuel (capCode c1 c2)

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in
```

```
open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def matchesBCharBodyLookup (w : ℕ) : ℕ :=
  matchesBCharBody (fun p' => nthCode w.unpair.1 p' 0) w.unpair.2.unpair.1 w.unpair.2.
    unpair.2

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

@[irreducible] def subsetBChar (fuel c1 c2 : ℕ) : ℕ :=
  bForallFn (fun p =>
      selectFn (subsetGuardChar fuel p.unpair.2.unpair.1 p.unpair.2.unpair.2 p.unpair
    .1)
        (selectFn (matchesBChar fuel p.unpair.2.unpair.1 p.unpair.1)
          (matchesBChar fuel p.unpair.2.unpair.2 p.unpair.1) 1)
        1)
    (Nat.pair c1 c2) (codeBound (subsetBoundChar fuel c1 c2))

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

@[irreducible] def interEqChar (fuel c1 c2 : ℕ) : ℕ :=
```

```
  mulBit (subsetBChar fuel c1 c2) (subsetBChar fuel c2 c1)

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems RegularLanguage
open DomainTheory.NeighbourhoodSystems.RegularLanguage

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def c9b6_decodeListBool (c : ℕ) : List Bool :=
  (decodeList c).map fun n => decide (n = 1)

private theorem c9b5_decodeList_encodeListBool (σ : List Bool) :
    decodeList (c9b5_encodeListBool σ) = σ.map c9b5_boolNat := by
  simp [c9b5_encodeListBool, decodeList_encodeList]

private theorem c9b5_boolNat_injective : Function.Injective c9b5_boolNat := by
  intro b c h
  cases b <;> cases c <;> first | rfl | simp [c9b5_boolNat] at h

private theorem c9b5_decodeList_injective {c1 c2 : ℕ} (h : decodeList c1 = decodeList
    c2) : c1 = c2 := by
  have := congrArg encodeList h
  rwa [encodeList_decodeList, encodeList_decodeList] at this

private theorem c9b5_map_boolNat_eq_iff (w σ : List Bool) :
    w.map c9b5_boolNat = σ.map c9b5_boolNat ↔ w = σ := by
  constructor
  · intro h
    revert σ
    induction w with
    | nil =>
      intro σ h
      cases σ with
      | nil => rfl
      | cons _ _ => simp at h
    | cons b w ih =>
      intro σ h
      cases σ with
      | nil => simp at h
      | cons c σ =>
        simp only [List.map_cons] at h
        obtain ⟨hb, ht⟩ := List.cons.inj h
        have hb' : b = c := by
          cases b <;> cases c <;> first | rfl | simp [c9b5_boolNat] at hb
        exact hb' ▸ congrArg (List.cons c) (ih σ ht)
  · intro h; rw [h]
```

```
private theorem c9b5_takeCode_encodeListBool (w : List Bool) (i : ℕ) :
    takeCode i (c9b5_encodeListBool w) = c9b5_encodeListBool (w.take i) := by
  apply c9b5_decodeList_injective
  rw [takeCode_eq, c9b5_decodeList_encodeListBool, ← List.map_take]
  rw [c9b5_decodeList_encodeListBool (σ := w.take i)]

private theorem c9b5_dropCode_encodeListBool (w : List Bool) (i : ℕ) :
    dropCode i (c9b5_encodeListBool w) = c9b5_encodeListBool (w.drop i) := by
  apply c9b5_decodeList_injective
  rw [dropCode_eq, c9b5_decodeList_encodeListBool, ← List.map_drop]
  rw [c9b5_decodeList_encodeListBool (σ := w.drop i)]

private theorem c9b5_listEqChar_encodeListBool (w σ : List Bool) :
    listEqChar (c9b5_encodeListBool w) (c9b5_encodeListBool σ) = 1 ↔ w = σ := by
  simp only [listEqChar_eq_one_iff, c9b5_decodeList_encodeListBool,
    c9b5_map_boolNat_eq_iff]

theorem autStateCardFuelCharBody_eq (prev : ℕ → ℕ) (c : ℕ) :
    autStateCardFuelCharBody prev c =


      match c.unpair.1 with
      | 0 => 1
      | 1 => listLenChar c.unpair.2 + 2
      | 2 => prev c.unpair.2.unpair.1 + prev c.unpair.2.unpair.2
      | 3 => prev c.unpair.2.unpair.1 * prev c.unpair.2.unpair.2
      | _ => 0 := by
  unfold autStateCardFuelCharBody
  match tag : c.unpair.1 with
  | 0 =>
    have h10 : (1 - 0 : ℕ) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01 : (1 - 1 : ℕ) = 0 := rfl
    have h12 : (2 - 1 : ℕ) = 1 := rfl
    simp only [h01, h12, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 2 =>
    have h01 : (1 - 2 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 2 : ℕ) = 0 := rfl
    have h23 : (3 - 2 : ℕ) = 1 := rfl
    simp only [h01, h23, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 3 =>
    have h01 : (1 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23 : (3 - 3 : ℕ) = 0 := rfl
    have h34 : (4 - 3 : ℕ) = 1 := rfl
    simp only [h01, h34, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | t + 4 =>
    have hne1 : isOne (1 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne2 : isOne (2 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne3 : isOne (3 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne4 : isOne (4 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    simp only [hne1, hne2, hne3, hne4, selectFn_zero]

theorem autStateCardFuelChar_eq_autStateCard {fuel : ℕ} {e : SExpr}
    (h : c9b5_sexprDepth e ≤ fuel) :
    autStateCardFuelChar fuel (c9b5_sexprGodelEncode e) = autStateCard e := by
```

```
  induction e generalizing fuel with
  | sigma =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel =>
      simp [autStateCardFuelChar, autStateCardFuelCharBody_eq, c9b5_sexprGodelEncode,
    autStateCard]
  | single σ =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel =>
      simp [autStateCardFuelChar, autStateCardFuelCharBody_eq, c9b5_sexprGodelEncode,
    autStateCard,
        listLenChar_eq, c9b5_decodeList_encodeListBool, List.length_map]
  | cat a b iha ihb =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel =>
      have ha : c9b5_sexprDepth a ≤ fuel := by
        simp only [c9b5_sexprDepth] at h ⊢; omega
      have hb : c9b5_sexprDepth b ≤ fuel := by
        simp only [c9b5_sexprDepth] at h ⊢; omega
      simp [autStateCardFuelChar, autStateCardFuelCharBody_eq, c9b5_sexprGodelEncode,
    autStateCard,
        iha ha, ihb hb]
  | cap a b iha ihb =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel =>
      have ha : c9b5_sexprDepth a ≤ fuel := by
        simp only [c9b5_sexprDepth] at h ⊢; omega
      have hb : c9b5_sexprDepth b ≤ fuel := by
        simp only [c9b5_sexprDepth] at h ⊢; omega
      simp [autStateCardFuelChar, autStateCardFuelCharBody_eq, c9b5_sexprGodelEncode,
    autStateCard,
        iha ha, ihb hb]

theorem primrec_autStateCardFuelCharBody {prev : ℕ → ℕ} (hprev : Nat.Primrec prev) :
    Nat.Primrec (fun c => autStateCardFuelCharBody prev c) := by
  unfold autStateCardFuelCharBody
  have hleft : Nat.Primrec (fun c => prev c.unpair.2.unpair.1) :=
    (hprev.comp (Nat.Primrec.left.comp Nat.Primrec.right)).of_eq fun c => by
      simp only
  have hright : Nat.Primrec (fun c => prev c.unpair.2.unpair.2) :=
    (hprev.comp (Nat.Primrec.right.comp Nat.Primrec.right)).of_eq fun c => by
      simp only
  have hadd := primrec_add₂ hleft hright
  have hmul := primrec_mul₂ hleft hright
  have hf1 := primrec_add₂ (primrec_listLenChar.comp Nat.Primrec.right) (Nat.Primrec.
    const 2)
  exact primrec_tagCase4 (Nat.Primrec.const 1) hf1 hadd hmul (Nat.Primrec.const 0)

theorem autStateCardFuelCharBody_local (f g : ℕ → ℕ) (c : ℕ) (h : ∀ c' ≤ c, f c' = g
    c') :
    autStateCardFuelCharBody f c = autStateCardFuelCharBody g c := by
  rw [autStateCardFuelCharBody_eq, autStateCardFuelCharBody_eq]
```

```
  have h1 : c.unpair.2.unpair.1 ≤ c := le_trans (unpair_left_le c.unpair.2) (
    unpair_snd_le c)
  have h2 : c.unpair.2.unpair.2 ≤ c := le_trans (unpair_snd_le c.unpair.2) (
    unpair_snd_le c)
  rw [h _ h1, h _ h2]

theorem matchesBCharBody_eq (prev : ℕ → ℕ) (c cw : ℕ) :
    matchesBCharBody prev c cw =
      match c.unpair.1 with
      | 0 => 1
      | 1 => listEqChar cw c.unpair.2
      | 2 =>
        bExistsFn (matchesBCatG prev) (Nat.pair c cw) (listLenChar cw + 1)
      | 3 =>
        mulBit (prev (Nat.pair c.unpair.2.unpair.1 cw))
               (prev (Nat.pair c.unpair.2.unpair.2 cw))
      | _ => 0 := by
  unfold matchesBCharBody matchesBCatG
  match tag : c.unpair.1 with
  | 0 =>
    have h10 : (1 - 0 : ℕ) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01 : (1 - 1 : ℕ) = 0 := rfl
    have h12 : (2 - 1 : ℕ) = 1 := rfl
    simp only [h01, h12, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 2 =>
    have h01 : (1 - 2 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 2 : ℕ) = 0 := rfl
    have h23 : (3 - 2 : ℕ) = 1 := rfl
    simp only [h01, h23, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 3 =>
    have h01 : (1 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23 : (3 - 3 : ℕ) = 0 := rfl
    have h34 : (4 - 3 : ℕ) = 1 := rfl
    simp only [h01, h34, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | t + 4 =>
    have hne1 : isOne (1 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne2 : isOne (2 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne3 : isOne (3 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne4 : isOne (4 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    simp only [hne1, hne2, hne3, hne4, selectFn_zero]

private theorem c9b5_matchesBCatG_char (fuel : ℕ) (a b : SExpr) (w : List Bool) (i : ℕ
    ) :
    matchesBCatG (fun p => matchesBChar fuel p.unpair.1 p.unpair.2)
        (Nat.pair i
          (Nat.pair (Nat.pair 2 (Nat.pair (c9b5_sexprGodelEncode a) (
    c9b5_sexprGodelEncode b)))
            (c9b5_encodeListBool w))) =
      mulBit (matchesBChar fuel (c9b5_sexprGodelEncode a) (takeCode i (
    c9b5_encodeListBool w)))
        (matchesBChar fuel (c9b5_sexprGodelEncode b) (dropCode i (c9b5_encodeListBool
    w))) := by
  simp [matchesBCatG]
```

```
set_option maxHeartbeats 1600000 in
theorem matchesBChar_eq_one_iff {fuel : ℕ} {e : SExpr} {w : List Bool}
    (h : c9b5_sexprDepth e ≤ fuel) :
    matchesBChar fuel (c9b5_sexprGodelEncode e) (c9b5_encodeListBool w) = 1 ↔
      matchesB e w = true := by
  induction e generalizing fuel w with
  | sigma =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel => simp [matchesBChar, matchesBCharBody_eq, matchesB,
    c9b5_sexprGodelEncode]
  | single σ =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel =>
      simp [matchesBChar, matchesBCharBody_eq, c9b5_sexprGodelEncode, matchesB,
        c9b5_listEqChar_encodeListBool w σ, decide_eq_true_eq]
  | cat a b iha ihb =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel =>
      have ha : c9b5_sexprDepth a ≤ fuel := by
        simp only [c9b5_sexprDepth] at h ⊢; omega
      have hb : c9b5_sexprDepth b ≤ fuel := by
        simp only [c9b5_sexprDepth] at h ⊢; omega
      have hbody := matchesBCharBody_eq (fun p => matchesBChar fuel p.unpair.1 p.
    unpair.2)
          (c9b5_sexprGodelEncode (.cat a b)) (c9b5_encodeListBool w)
      rw [matchesBChar, hbody]
      simp only [c9b5_sexprGodelEncode, unpair_pair_fst, matchesB, List.any_eq_true]
      rw [bExistsFn_eq_one_iff]
      have hlen : listLenChar (c9b5_encodeListBool w) + 1 = w.length + 1 := by
        simp [listLenChar_eq, c9b5_decodeList_encodeListBool, List.length_map]
      constructor
      · intro ⟨i, hi, hg⟩
        rw [c9b5_matchesBCatG_char, mulBit_eq_one_iff] at hg
        obtain ⟨h1, h2⟩ := hg
        have hi' : i < w.length + 1 := by rw [← hlen]; exact hi
        refine ⟨i, (List.mem_range).2 hi', ?_⟩
        simp only [Bool.and_eq_true]
        exact ⟨(iha ha).mp (by rw [← c9b5_takeCode_encodeListBool w i]; exact h1),
          (ihb hb).mp (by rw [← c9b5_dropCode_encodeListBool w i]; exact h2)⟩
      · intro ⟨i, hi, hfg⟩
        have hi' : i < listLenChar (c9b5_encodeListBool w) + 1 := by
          rw [hlen]; exact (List.mem_range).1 hi
        refine ⟨i, hi', ?_⟩
        rw [c9b5_matchesBCatG_char, mulBit_eq_one_iff]
        simp [Bool.and_eq_true] at hfg
        refine And.intro ?h1 ?h2
        · simpa [c9b5_takeCode_encodeListBool w i] using (iha ha).mpr hfg.1
        · simpa [c9b5_dropCode_encodeListBool w i] using (ihb hb).mpr hfg.2
  | cap a b iha ihb =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel =>
```

```
      have ha : c9b5_sexprDepth a ≤ fuel := by
        simp only [c9b5_sexprDepth] at h ⊢; omega
      have hb : c9b5_sexprDepth b ≤ fuel := by
        simp only [c9b5_sexprDepth] at h ⊢; omega
      have hbody := matchesBCharBody_eq (fun p => matchesBChar fuel p.unpair.1 p.
    unpair.2)
          (c9b5_sexprGodelEncode (.cap a b)) (c9b5_encodeListBool w)
      rw [matchesBChar, hbody]
      simp only [c9b5_sexprGodelEncode, unpair_pair_fst, unpair_pair_snd, matchesB,
    mulBit_eq_one_iff]
      constructor
      · intro ⟨h1, h2⟩
        rw [Bool.and_eq_true]
        exact ⟨(iha ha).mp h1, (ihb hb).mp h2⟩
      · intro hfg
        rw [Bool.and_eq_true] at hfg
        exact ⟨(iha ha).mpr hfg.1, (ihb hb).mpr hfg.2⟩

set_option maxHeartbeats 800000 in
private theorem primrec_matchesBCatG {prev : ℕ → ℕ} (hprev : Nat.Primrec prev) :
    Nat.Primrec (fun p => matchesBCatG prev p) := by
  have hc := Nat.Primrec.left.comp Nat.Primrec.right
  have ha := Nat.Primrec.left.comp (Nat.Primrec.right.comp hc)
  have hb := Nat.Primrec.right.comp (Nat.Primrec.right.comp hc)
  have hcw := Nat.Primrec.right.comp Nat.Primrec.right
  have hi := Nat.Primrec.left
  have h1 := hprev.comp (Nat.Primrec.pair ha (primrec_takeCode.comp (hi.pair hcw)))
  have h2 := hprev.comp (Nat.Primrec.pair hb (primrec_dropCode.comp (hi.pair hcw)))
  exact (primrec_mul₂ h1 h2).of_eq fun p => by
    dsimp [matchesBCatG, mulBit]
    simp only [unpair_pair_fst, unpair_pair_snd]

set_option maxHeartbeats 800000 in
theorem primrec_matchesBCharBody {prev : ℕ → ℕ} (hprev : Nat.Primrec prev) :
    Nat.Primrec (fun t => matchesBCharBody prev t.unpair.1 t.unpair.2) := by
  have htag : Nat.Primrec (fun t => (t.unpair.1).unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.left
  have h01 := primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 1) htag)
  have h12 := primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 2) htag)
  have h23 := primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 3) htag)
  have h34 := primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 4) htag)
  have hf1 := primrec_listEqChar.comp (Nat.Primrec.pair Nat.Primrec.right
    (Nat.Primrec.right.comp Nat.Primrec.left))
  have hpack : Nat.Primrec (fun t => Nat.pair t (listLenChar t.unpair.2 + 1)) :=
    Nat.Primrec.pair primrec_id
      (primrec_add₂ (primrec_listLenChar.comp Nat.Primrec.right) (Nat.Primrec.const 1))
  have hcat := (primrec_bExistsFn (primrec_matchesBCatG hprev)).comp hpack
  have hc := Nat.Primrec.left
  have ha := Nat.Primrec.left.comp (Nat.Primrec.right.comp hc)
  have hb := Nat.Primrec.right.comp (Nat.Primrec.right.comp hc)
  have hcw := Nat.Primrec.right
  have hcap := primrec_mulBit.comp (Nat.Primrec.pair
    (hprev.comp (Nat.Primrec.pair ha hcw)) (hprev.comp (Nat.Primrec.pair hb hcw)))
  exact (primrec_selectFn h01 (Nat.Primrec.const 1)
    (primrec_selectFn h12 hf1
      (primrec_selectFn h23 hcat
```

```
        (primrec_selectFn h34 hcap (Nat.Primrec.const 0)))))).of_eq fun t => by
    rw [matchesBCharBody_eq]
    match tag : (t.unpair.1).unpair.1 with
    | 0 | 1 | 2 | 3 =>
      simp [selectFn, isOne, mulBit]
    | _ + 4 =>
      simp [selectFn, isOne, mulBit]

theorem matchesBChar_le_one : ∀ fuel c cw, matchesBChar fuel c cw ≤ 1
  | 0, _, _ => by simp [matchesBChar]
  | fuel + 1, c, cw => by
    rw [matchesBChar, matchesBCharBody_eq]
    match tag : c.unpair.1 with
    | 0 => simp
    | 1 => exact listEqChar_le_one _ _
    | 2 => exact bExistsFn_le_one _ _ _
    | 3 =>
      exact mulBit_le_one (matchesBChar_le_one fuel _ _) (matchesBChar_le_one fuel _ _
    )
    | _ + 4 => simp

theorem matchesBCharBody_local (f g : ℕ → ℕ) (c cw : ℕ)
    (h : ∀ p ≤ Nat.pair c cw, f p = g p) : matchesBCharBody f c cw = matchesBCharBody
    g c cw := by
  rw [matchesBCharBody_eq, matchesBCharBody_eq]
  match tag : c.unpair.1 with
  | 0 => rfl
  | 1 => rfl
  | 2 =>
    simp
    apply bExistsFn_congr
    intro i _
    unfold matchesBCatG
    simp only [unpair_pair_fst, unpair_pair_snd]
    have h1 : Nat.pair c.unpair.2.unpair.1 (takeCode i cw) ≤ Nat.pair c cw :=
      pair_le_pair (le_trans (unpair_left_le c.unpair.2) (unpair_snd_le c)) (
    takeCode_le i cw)
    have h2 : Nat.pair c.unpair.2.unpair.2 (dropCode i cw) ≤ Nat.pair c cw :=
      pair_le_pair (le_trans (unpair_snd_le c.unpair.2) (unpair_snd_le c)) (
    dropCode_le i cw)
    rw [h _ h1, h _ h2]
  | 3 =>
    simp
    have h1 : Nat.pair c.unpair.2.unpair.1 cw ≤ Nat.pair c cw :=
      pair_le_pair (le_trans (unpair_left_le c.unpair.2) (unpair_snd_le c)) (le_refl
    cw)
    have h2 : Nat.pair c.unpair.2.unpair.2 cw ≤ Nat.pair c cw :=
      pair_le_pair (le_trans (unpair_snd_le c.unpair.2) (unpair_snd_le c)) (le_refl cw
    )
    rw [h _ h1, h _ h2]
  | _ + 4 => rfl

private theorem c9b6_allBinDigitsChar_encodeListBool (w : List Bool) :
    allBinDigitsChar (c9b5_encodeListBool w) = 1 := by
  rw [allBinDigitsChar_eq_one_iff]
  intro x hx
```

```
  rw [c9b5_decodeList_encodeListBool] at hx
  simp only [List.mem_map] at hx
  obtain ⟨b, _, hb⟩ := hx
  cases b <;> simp [c9b5_boolNat] at hb <;> omega

private theorem c9b6_encodeListBool_lt_codeBound {n : ℕ} {w : List Bool} (hw : w.
    length ≤ n) :
    c9b5_encodeListBool w < codeBound n := by
  apply codeBound_ge n (c9b5_encodeListBool w)
  · rw [c9b5_decodeList_encodeListBool, List.length_map]; exact hw
  · intro x hx
    rw [c9b5_decodeList_encodeListBool] at hx
    simp only [List.mem_map] at hx
    obtain ⟨b, _, hb⟩ := hx
    cases b <;> simp [c9b5_boolNat] at hb <;> omega

private theorem c9b6_boolNat_decide_eq (x : ℕ) (hx : x = 0 ∨ x = 1) :
    c9b5_boolNat (decide (x = 1)) = x := by
  rcases hx with rfl | rfl <;> simp [c9b5_boolNat]

private theorem c9b6_encodeListBool_decodeListBool_of_allBin {c : ℕ}
    (hall : allBinDigitsChar c = 1) :
    c9b5_encodeListBool (c9b6_decodeListBool c) = c := by
  rw [allBinDigitsChar_eq_one_iff] at hall
  apply c9b5_decodeList_injective
  rw [c9b5_decodeList_encodeListBool, c9b6_decodeListBool, List.map_map]
  conv_rhs => rw [← List.map_id (decodeList c)]
  exact List.map_congr_left (fun x hx => c9b6_boolNat_decide_eq x (hall x hx))

private theorem c9b6_matchesBChar_of_allBin {fuel : ℕ} {e : SExpr} {c : ℕ}
    (h : c9b5_sexprDepth e ≤ fuel) (hall : allBinDigitsChar c = 1)
    (hmatch : matchesBChar fuel (c9b5_sexprGodelEncode e) c = 1) :
    matchesB e (c9b6_decodeListBool c) = true := by
  apply (matchesBChar_eq_one_iff (w := c9b6_decodeListBool c) h).1
  rwa [c9b6_encodeListBool_decodeListBool_of_allBin hall]

theorem decideNonemptyBChar_eq_one_iff {fuel : ℕ} {e : SExpr}
    (h : c9b5_sexprDepth e ≤ fuel) :
    decideNonemptyBChar fuel (c9b5_sexprGodelEncode e) = 1 ↔ (denote e).Nonempty := by
  simp only [decideNonemptyBChar, bExistsFn_eq_one_iff, unpair_pair_fst,
    unpair_pair_snd,
    mulBit_eq_one_iff]
  constructor
  · rintro ⟨i, -, hall, hmatch⟩
    exact ⟨c9b6_decodeListBool i, (matchesB_iff e _).mp (c9b6_matchesBChar_of_allBin h
    hall hmatch)⟩
  · intro hne
    obtain ⟨w, hwmem, hmatch⟩ := (denote_nonempty_iff_short e).mp hne
    refine ⟨c9b5_encodeListBool w, ?_, c9b6_allBinDigitsChar_encodeListBool w,
      (matchesBChar_eq_one_iff h).2 hmatch⟩
    rw [autStateCardFuelChar_eq_autStateCard h]
    exact c9b6_encodeListBool_lt_codeBound (by rwa [mem_wordsUpTo] at hwmem)

theorem c9b6_sexprGodelEncode_cap (a b : SExpr) :
    c9b5_sexprGodelEncode (.cap a b) = capCode (c9b5_sexprGodelEncode a) (
    c9b5_sexprGodelEncode b) :=
```

```
  rfl

theorem subsetGuardChar_le_one (fuel c1 c2 i : ℕ) : subsetGuardChar fuel c1 c2 i ≤ 1
    :=
  mulBit_le_one (allBinDigitsChar_le_one _) (isOne_le_one _)

set_option maxHeartbeats 800000 in
theorem subsetBChar_eq_one_iff {fuel : ℕ} {e1 e2 : SExpr}
    (h1 : c9b5_sexprDepth e1 ≤ fuel) (h2 : c9b5_sexprDepth e2 ≤ fuel) :
    subsetBChar fuel (c9b5_sexprGodelEncode e1) (c9b5_sexprGodelEncode e2) = 1 ↔
      denote e1 ⊆ denote e2 := by
  rw [← subsetB_iff]
  have hbound : subsetBoundChar fuel (c9b5_sexprGodelEncode e1) (c9b5_sexprGodelEncode
     e2) =
      Fintype.card (Finset (autState e1) × Finset (autState e2)) := by
    unfold subsetBoundChar
    rw [autStateCardFuelChar_eq_autStateCard h1, autStateCardFuelChar_eq_autStateCard
    h2,
      autStateCard_eq_card, autStateCard_eq_card, Fintype.card_prod, Fintype.
    card_finset,
      Fintype.card_finset]
  unfold subsetB subsetBChar
  rw [List.all_eq_true, bForallFn_eq_one_iff]
  simp only [unpair_pair_fst, unpair_pair_snd, hbound]
  constructor
  · intro h w hwmem
    rw [mem_wordsUpTo] at hwmem
    have hi := h (c9b5_encodeListBool w) (c9b6_encodeListBool_lt_codeBound hwmem)
    have hguard1 : subsetGuardChar fuel (c9b5_sexprGodelEncode e1) (
    c9b5_sexprGodelEncode e2)
        (c9b5_encodeListBool w) = 1 := by
      unfold subsetGuardChar
      rw [c9b6_allBinDigitsChar_encodeListBool, mulBit, one_mul, listLenChar_eq,
        c9b5_decodeList_encodeListBool, List.length_map, hbound,
        Nat.sub_eq_zero_of_le hwmem]
      exact isOne_one
    rw [hguard1, selectFn_one] at hi
    by_cases hc1 : matchesBChar fuel (c9b5_sexprGodelEncode e1) (c9b5_encodeListBool w
    ) = 1
    · rw [hc1, selectFn_one] at hi
      simp only [Bool.or_eq_true]
      exact Or.inr ((matchesBChar_eq_one_iff h2).mp hi)
    · have hc1' : matchesBChar fuel (c9b5_sexprGodelEncode e1) (c9b5_encodeListBool w)
    = 0 := by
        have := matchesBChar_le_one fuel (c9b5_sexprGodelEncode e1) (
    c9b5_encodeListBool w)
        omega
      simp only [Bool.or_eq_true]
      refine Or.inl ?_
      by_contra hcontra
      exact hc1 ((matchesBChar_eq_one_iff h1).mpr (by simpa using hcontra))
  · intro h i _
    rcases (show subsetGuardChar fuel (c9b5_sexprGodelEncode e1) (
    c9b5_sexprGodelEncode e2) i = 0 ∨
        subsetGuardChar fuel (c9b5_sexprGodelEncode e1) (c9b5_sexprGodelEncode e2) i =
     1 from by
```

```
      have := subsetGuardChar_le_one fuel (c9b5_sexprGodelEncode e1) (
    c9b5_sexprGodelEncode e2) i
      omega) with hguard | hguard
    · rw [hguard, selectFn_zero]
    · rw [hguard, selectFn_one]
      unfold subsetGuardChar at hguard
      rw [mulBit_eq_one_iff, isOne_eq_one_iff] at hguard
      obtain ⟨hallbin, hlen0⟩ := hguard
      have hlenle : listLenChar i ≤
          Fintype.card (Finset (autState e1) × Finset (autState e2)) := by omega

      have hieq : c9b5_encodeListBool (c9b6_decodeListBool i) = i :=
        c9b6_encodeListBool_decodeListBool_of_allBin hallbin
      have hwlen : (c9b6_decodeListBool i).length ≤
          Fintype.card (Finset (autState e1) × Finset (autState e2)) := by
        unfold c9b6_decodeListBool
        rw [List.length_map]
        rwa [listLenChar_eq] at hlenle
      have hwmem := (mem_wordsUpTo _).mpr hwlen
      have hspec := h (c9b6_decodeListBool i) hwmem
      rw [← hieq]
      by_cases hc1 : matchesBChar fuel (c9b5_sexprGodelEncode e1) (c9b5_encodeListBool
          (c9b6_decodeListBool i)) = 1
      · rw [hc1, selectFn_one]
        have hc1matches : matchesB e1 (c9b6_decodeListBool i) = true :=
          (matchesBChar_eq_one_iff h1).mp hc1
        simp only [Bool.or_eq_true] at hspec
        rcases hspec with hf | ht
        · rw [hc1matches] at hf; simp at hf
        · exact (matchesBChar_eq_one_iff h2).mpr ht
      · have hc1' : matchesBChar fuel (c9b5_sexprGodelEncode e1)
            (c9b5_encodeListBool (c9b6_decodeListBool i)) = 0 := by
          have := matchesBChar_le_one fuel (c9b5_sexprGodelEncode e1)
            (c9b5_encodeListBool (c9b6_decodeListBool i))
          omega
        rw [hc1', selectFn_zero]

theorem consistentBChar_eq_one_iff {fuel : ℕ} {a b : SExpr}
    (h : c9b5_sexprDepth (.cap a b) ≤ fuel) :
    consistentBChar fuel (c9b5_sexprGodelEncode a) (c9b5_sexprGodelEncode b) = 1 ↔
      (denote (.cap a b)).Nonempty := by
  simp only [consistentBChar, ← c9b6_sexprGodelEncode_cap]
  exact decideNonemptyBChar_eq_one_iff h

theorem decideNonemptyBChar_le_one (fuel c_e : ℕ) : decideNonemptyBChar fuel c_e ≤ 1
    :=
  bExistsFn_le_one _ _ _

theorem consistentBChar_le_one (fuel c1 c2 : ℕ) : consistentBChar fuel c1 c2 ≤ 1 :=
  decideNonemptyBChar_le_one _ _

theorem autStateCardFuelChar_fuelTable_eq :
    ∀ fuel bound c, c ≤ bound →
      nthCode (fuelTable autStateCardFuelCharLookup bound fuel) c 0 =
    autStateCardFuelChar fuel c :=
  fuelTable_eq_of_recursion (F := autStateCardFuelChar) (Body :=
```

```
    autStateCardFuelCharBody)
    (fun _ => rfl) (fun _ _ => rfl)
    (fun table c => by unfold autStateCardFuelCharLookup; simp)
    autStateCardFuelCharBody_local

theorem interEqChar_eq_one_iff {fuel : ℕ} {e1 e2 : SExpr}
    (h1 : c9b5_sexprDepth e1 ≤ fuel) (h2 : c9b5_sexprDepth e2 ≤ fuel) :
    interEqChar fuel (c9b5_sexprGodelEncode e1) (c9b5_sexprGodelEncode e2) = 1 ↔
      denote e1 = denote e2 := by
  unfold interEqChar
  rw [mulBit_eq_one_iff, subsetBChar_eq_one_iff h1 h2, subsetBChar_eq_one_iff h2 h1,
    ← Set.Subset.antisymm_iff]

theorem matchesBChar_fuelTable_eq :
    ∀ fuel bound code, code ≤ bound →
      nthCode (fuelTable matchesBCharBodyLookup bound fuel) code 0 =
        matchesBChar fuel code.unpair.1 code.unpair.2 :=
  fuelTable_eq_of_recursion
    (F := fun fuel code => matchesBChar fuel code.unpair.1 code.unpair.2)
    (Body := fun prev code => matchesBCharBody prev code.unpair.1 code.unpair.2)
    (fun _ => rfl) (fun _ _ => rfl)
    (fun table code => by unfold matchesBCharBodyLookup; simp)
    (fun f g code h => matchesBCharBody_local f g code.unpair.1 code.unpair.2 (by
      simpa [pair_unpair] using h))

theorem primrec_autStateCardFuelChar : ∀ fuel, Nat.Primrec (fun c =>
    autStateCardFuelChar fuel c)
  | 0 => Nat.Primrec.const 0
  | fuel + 1 => by
    simpa [autStateCardFuelChar] using
      primrec_autStateCardFuelCharBody (primrec_autStateCardFuelChar fuel)

theorem primrec_autStateCardFuelCharLookup : Nat.Primrec autStateCardFuelCharLookup :=
     by
  have hleft : Nat.Primrec (fun w => nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.1
    0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.
    right))
        (Nat.Primrec.const 0)))).of_eq fun w => by simp
  have hright : Nat.Primrec (fun w => nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.2
    0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.
    right))
        (Nat.Primrec.const 0)))).of_eq fun w => by simp
  have hadd : Nat.Primrec (fun w =>
      nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.1 0 +
        nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.2 0) :=
    primrec_add₂ hleft hright
  have hmul : Nat.Primrec (fun w =>
      nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.1 0 *
        nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.2 0) :=
    primrec_mul₂ hleft hright
  have hf0 : Nat.Primrec (fun _ : ℕ => (1 : ℕ)) := Nat.Primrec.const 1
  have hf1 : Nat.Primrec (fun w => listLenChar w.unpair.2.unpair.2 + 2) :=
```

```
    primrec_add₂ (primrec_listLenChar.comp (Nat.Primrec.right.comp Nat.Primrec.right))
      (Nat.Primrec.const 2)
  have ht : Nat.Primrec (fun w => w.unpair.2.unpair.1) := Nat.Primrec.left.comp Nat.
    Primrec.right
  have h01 : Nat.Primrec (fun w => isOne (1 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 1) ht)
  have h12 : Nat.Primrec (fun w => isOne (2 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 2) ht)
  have h23 : Nat.Primrec (fun w => isOne (3 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 3) ht)
  have h34 : Nat.Primrec (fun w => isOne (4 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 4) ht)
  refine (primrec_selectFn h01 hf0
    (primrec_selectFn h12 hf1
      (primrec_selectFn h23 hadd
        (primrec_selectFn h34 hmul (Nat.Primrec.const 0))))).of_eq fun w => ?_
  show _ = autStateCardFuelCharBody (fun c' => nthCode w.unpair.1 c' 0) w.unpair.2
  rw [autStateCardFuelCharBody_eq]
  match tag : w.unpair.2.unpair.1 with
  | 0 =>
    have h10 : (1 - 0 : ℕ) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01' : (1 - 1 : ℕ) = 0 := rfl
    have h12' : (2 - 1 : ℕ) = 1 := rfl
    simp only [h01', h12', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 2 =>
    have h01' : (1 - 2 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 2 : ℕ) = 0 := rfl
    have h23' : (3 - 2 : ℕ) = 1 := rfl
    simp only [h01', h23', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 3 =>
    have h01' : (1 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - 3 : ℕ) = 0 := rfl
    have h34' : (4 - 3 : ℕ) = 1 := rfl
    simp only [h01', h34', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | t + 4 =>
    have h01' : (1 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h34' : (4 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    simp only [h01', h12', h23', h34', isOne_zero, selectFn_zero]

theorem primrec_autStateCardFuelChar2 :
    Nat.Primrec (fun s => autStateCardFuelChar s.unpair.1 s.unpair.2) := by
  have hft := primrec_fuelTable primrec_autStateCardFuelCharLookup
  have hpack : Nat.Primrec (fun s : ℕ => Nat.pair s.unpair.2 s.unpair.1) :=
    Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left
  have hcomp : Nat.Primrec (fun s => fuelTable autStateCardFuelCharLookup s.unpair.2 s
    .unpair.1) :=
    (hft.comp hpack).of_eq fun s => by simp
  have hnth : Nat.Primrec
      (fun s => nthCode (fuelTable autStateCardFuelCharLookup s.unpair.2 s.unpair.1) s
    .unpair.2 0) :=
    (primrec_nthCode.comp
```

```
      (Nat.Primrec.pair hcomp (Nat.Primrec.pair Nat.Primrec.right (Nat.Primrec.const
    0)))).of_eq
      fun s => by simp
  exact hnth.of_eq fun s =>
    autStateCardFuelChar_fuelTable_eq s.unpair.1 s.unpair.2 s.unpair.2 (le_refl _)

theorem primrec_matchesBCatGLookup : Nat.Primrec matchesBCatGLookup := by
  have hi : Nat.Primrec (fun w => w.unpair.2.unpair.1) := Nat.Primrec.left.comp Nat.
    Primrec.right
  have hc : Nat.Primrec (fun w => w.unpair.2.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hcw : Nat.Primrec (fun w => w.unpair.2.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have ha : Nat.Primrec (fun w => w.unpair.2.unpair.2.unpair.1.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp hc)
  have hb : Nat.Primrec (fun w => w.unpair.2.unpair.2.unpair.1.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp hc)
  have htake : Nat.Primrec (fun w => takeCode w.unpair.2.unpair.1 w.unpair.2.unpair.2.
    unpair.2) :=
    (primrec_takeCode.comp (Nat.Primrec.pair hi hcw)).of_eq fun w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hdrop : Nat.Primrec (fun w => dropCode w.unpair.2.unpair.1 w.unpair.2.unpair.2.
    unpair.2) :=
    (primrec_dropCode.comp (Nat.Primrec.pair hi hcw)).of_eq fun w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have h1 : Nat.Primrec (fun w => nthCode w.unpair.1
      (Nat.pair w.unpair.2.unpair.2.unpair.1.unpair.2.unpair.1
        (takeCode w.unpair.2.unpair.1 w.unpair.2.unpair.2.unpair.2)) 0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.pair ha htake) (Nat.Primrec.const 0)))).of_eq fun
     w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have h2 : Nat.Primrec (fun w => nthCode w.unpair.1
      (Nat.pair w.unpair.2.unpair.2.unpair.1.unpair.2.unpair.2
        (dropCode w.unpair.2.unpair.1 w.unpair.2.unpair.2.unpair.2)) 0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.pair hb hdrop) (Nat.Primrec.const 0)))).of_eq fun
     w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_mul₂ h1 h2).of_eq fun w => by
    dsimp [matchesBCatGLookup, matchesBCatG, mulBit]

theorem primrec_matchesBChar : ∀ fuel, Nat.Primrec (fun t => matchesBChar fuel t.unpair
    .1 t.unpair.2)
  | 0 => Nat.Primrec.const 0
  | fuel + 1 => by
    simpa [matchesBChar] using
      primrec_matchesBCharBody (primrec_matchesBChar fuel)

theorem primrec_decideNonemptyBChar (fuel : ℕ) :
    Nat.Primrec (fun c_e => decideNonemptyBChar fuel c_e) := by
  have hg : Nat.Primrec (fun w =>
      mulBit (allBinDigitsChar w.unpair.1) (matchesBChar fuel w.unpair.2 w.unpair.1))
    :=
    (primrec_mulBit.comp (Nat.Primrec.pair (primrec_allBinDigitsChar.comp Nat.Primrec.
    left)
```

```
      ((primrec_matchesBChar fuel).comp (Nat.Primrec.pair Nat.Primrec.right Nat.
    Primrec.left)))).of_eq
      fun w => by simp
  have hpack : Nat.Primrec (fun c_e =>
      Nat.pair c_e (codeBound (autStateCardFuelChar fuel c_e))) :=
    Nat.Primrec.pair Nat.Primrec.id
      (primrec_codeBound.comp (primrec_autStateCardFuelChar fuel))
  refine ((primrec_bExistsFn hg).comp hpack).of_eq fun c_e => ?_
  simp [decideNonemptyBChar]

theorem primrec_consistentBChar (fuel : ℕ) :
    Nat.Primrec (fun t => consistentBChar fuel t.unpair.1 t.unpair.2) :=
  (primrec_decideNonemptyBChar fuel).comp
    (Nat.Primrec.pair (Nat.Primrec.const 3)
      (Nat.Primrec.pair Nat.Primrec.left Nat.Primrec.right))

set_option maxHeartbeats 4000000 in
theorem primrec_matchesBCharBodyLookup : Nat.Primrec matchesBCharBodyLookup := by
  have htable : Nat.Primrec (fun w => w.unpair.1) := Nat.Primrec.left
  have hc : Nat.Primrec (fun w => w.unpair.2.unpair.1) := Nat.Primrec.left.comp Nat.
    Primrec.right
  have hcw : Nat.Primrec (fun w => w.unpair.2.unpair.2) := Nat.Primrec.right.comp Nat.
    Primrec.right
  have htag : Nat.Primrec (fun w => w.unpair.2.unpair.1.unpair.1) := Nat.Primrec.left.
    comp hc
  have h01 := primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 1) htag)
  have h12 := primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 2) htag)
  have h23 := primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 3) htag)
  have h34 := primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 4) htag)
  have hf1 : Nat.Primrec (fun w => listEqChar w.unpair.2.unpair.2 w.unpair.2.unpair.1.
    unpair.2) :=
    (primrec_listEqChar.comp (Nat.Primrec.pair hcw (Nat.Primrec.right.comp hc))).of_eq
     fun w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hg : Nat.Primrec (fun w'' => matchesBCatGLookup
      (Nat.pair w''.unpair.2.unpair.1 (Nat.pair w''.unpair.1 w''.unpair.2.unpair.2)))
    :=
    (primrec_matchesBCatGLookup.comp (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.
    Primrec.right)
      (Nat.Primrec.pair Nat.Primrec.left (Nat.Primrec.right.comp Nat.Primrec.right))))
    .of_eq
      fun _ => rfl
  have hcatpack : Nat.Primrec (fun w => Nat.pair
      (Nat.pair w.unpair.1 (Nat.pair w.unpair.2.unpair.1 w.unpair.2.unpair.2))
      (listLenChar w.unpair.2.unpair.2 + 1)) :=
    Nat.Primrec.pair (Nat.Primrec.pair htable (Nat.Primrec.pair hc hcw))
      (primrec_add₂ (primrec_listLenChar.comp hcw) (Nat.Primrec.const 1))
  have hcat : Nat.Primrec (fun w => bExistsFn
      (fun w'' => matchesBCatGLookup
        (Nat.pair w''.unpair.2.unpair.1 (Nat.pair w''.unpair.1 w''.unpair.2.unpair.2))
    )
      (Nat.pair w.unpair.1 (Nat.pair w.unpair.2.unpair.1 w.unpair.2.unpair.2))
      (listLenChar w.unpair.2.unpair.2 + 1)) :=
    ((primrec_bExistsFn hg).comp hcatpack).of_eq fun w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have ha : Nat.Primrec (fun w => w.unpair.2.unpair.1.unpair.2.unpair.1) :=
```

```
    Nat.Primrec.left.comp (Nat.Primrec.right.comp hc)
  have hb : Nat.Primrec (fun w => w.unpair.2.unpair.1.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp hc)
  have hlk1 : Nat.Primrec (fun w => nthCode w.unpair.1
      (Nat.pair w.unpair.2.unpair.1.unpair.2.unpair.1 w.unpair.2.unpair.2) 0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair htable
      (Nat.Primrec.pair (Nat.Primrec.pair ha hcw) (Nat.Primrec.const 0)))).of_eq fun w
     => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hlk2 : Nat.Primrec (fun w => nthCode w.unpair.1
      (Nat.pair w.unpair.2.unpair.1.unpair.2.unpair.2 w.unpair.2.unpair.2) 0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair htable
      (Nat.Primrec.pair (Nat.Primrec.pair hb hcw) (Nat.Primrec.const 0)))).of_eq fun w
     => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hcap := primrec_mul₂ hlk1 hlk2
  refine (primrec_selectFn h01 (Nat.Primrec.const 1)
    (primrec_selectFn h12 hf1
      (primrec_selectFn h23 hcat
        (primrec_selectFn h34 hcap (Nat.Primrec.const 0))))).of_eq fun w => ?_
  show _ = matchesBCharBody (fun p' => nthCode w.unpair.1 p' 0) w.unpair.2.unpair.1 w.
    unpair.2.unpair.2
  rw [matchesBCharBody_eq]
  match tag : w.unpair.2.unpair.1.unpair.1 with
  | 0 =>
    have h10 : (1 - 0 : ℕ) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01' : (1 - 1 : ℕ) = 0 := rfl
    have h12' : (2 - 1 : ℕ) = 1 := rfl
    simp only [h01', h12', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 2 =>
    have h01' : (1 - 2 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 2 : ℕ) = 0 := rfl
    have h23' : (3 - 2 : ℕ) = 1 := rfl
    simp only [h01', h23', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
    apply eq_of_le_one_iff_one (bExistsFn_le_one _ _ _) (bExistsFn_le_one _ _ _)
    rw [bExistsFn_eq_one_iff, bExistsFn_eq_one_iff]
    refine exists_congr fun i => and_congr_right fun _ => ?_
    simp only [matchesBCatGLookup, unpair_pair_fst, unpair_pair_snd]
  | 3 =>
    have h01' : (1 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 3 : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - 3 : ℕ) = 0 := rfl
    have h34' : (4 - 3 : ℕ) = 1 := rfl
    simp only [h01', h34', isOne_zero, isOne_one, selectFn_zero, selectFn_one,
      mulBit]
  | t + 4 =>
    have h01' : (1 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h34' : (4 - (t + 4) : ℕ) = 0 := Nat.sub_eq_zero_of_le (by omega)
    simp only [h01', h12', h23', h34', isOne_zero, selectFn_zero]

theorem primrec_matchesBChar2 :
    Nat.Primrec (fun s => matchesBChar s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.
```

```
    unpair.2) := by
  have hft := primrec_fuelTable primrec_matchesBCharBodyLookup
  have hpack : Nat.Primrec (fun s : ℕ => Nat.pair s.unpair.2 s.unpair.1) :=
    Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left
  have hcomp : Nat.Primrec (fun s => fuelTable matchesBCharBodyLookup s.unpair.2 s.
    unpair.1) :=
    (hft.comp hpack).of_eq fun s => by simp
  have hnth : Nat.Primrec
      (fun s => nthCode (fuelTable matchesBCharBodyLookup s.unpair.2 s.unpair.1) s.
    unpair.2 0) :=
    (primrec_nthCode.comp
      (Nat.Primrec.pair hcomp (Nat.Primrec.pair Nat.Primrec.right (Nat.Primrec.const
    0)))).of_eq
      fun s => by simp
  exact hnth.of_eq fun s => by
    rw [matchesBChar_fuelTable_eq s.unpair.1 s.unpair.2 s.unpair.2 (le_refl _)]

theorem primrec_decideNonemptyBChar2 :
    Nat.Primrec (fun s => decideNonemptyBChar s.unpair.1 s.unpair.2) := by
  have hg : Nat.Primrec (fun u => mulBit (allBinDigitsChar u.unpair.2.unpair.1)
      (matchesBChar u.unpair.1 u.unpair.2.unpair.2 u.unpair.2.unpair.1)) :=
    (primrec_mulBit.comp (Nat.Primrec.pair
      (primrec_allBinDigitsChar.comp (Nat.Primrec.left.comp Nat.Primrec.right))
      (primrec_matchesBChar2.comp (Nat.Primrec.pair Nat.Primrec.left
        (Nat.Primrec.pair (Nat.Primrec.right.comp Nat.Primrec.right)
          (Nat.Primrec.left.comp Nat.Primrec.right)))))).of_eq fun u => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hbe := primrec_bExistsFn_param hg
  have hpack : Nat.Primrec (fun s => Nat.pair (Nat.pair s.unpair.1 s.unpair.2)
      (codeBound (autStateCardFuelChar s.unpair.1 s.unpair.2))) :=
    Nat.Primrec.pair (Nat.Primrec.pair Nat.Primrec.left Nat.Primrec.right)
      (primrec_codeBound.comp primrec_autStateCardFuelChar2)
  exact (hbe.comp hpack).of_eq fun s => by
    simp only [unpair_pair_fst, unpair_pair_snd, decideNonemptyBChar]

theorem primrec_consistentBChar2 :
    Nat.Primrec (fun s => consistentBChar s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.
    unpair.2) := by
  have hpack : Nat.Primrec (fun s => Nat.pair s.unpair.1
      (Nat.pair 3 (Nat.pair s.unpair.2.unpair.1 s.unpair.2.unpair.2))) :=
    Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.const 3)
        (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.right)
          (Nat.Primrec.right.comp Nat.Primrec.right)))
  exact (primrec_decideNonemptyBChar2.comp hpack).of_eq fun s => by
    simp only [unpair_pair_fst, unpair_pair_snd, consistentBChar, capCode]

theorem primrec_subsetBChar :
    Nat.Primrec (fun s => subsetBChar s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.unpair
    .2) := by
  have hg : Nat.Primrec (fun u => selectFn
      (subsetGuardChar u.unpair.1 u.unpair.2.unpair.2.unpair.1 u.unpair.2.unpair.2.
    unpair.2
        u.unpair.2.unpair.1)
      (selectFn (matchesBChar u.unpair.1 u.unpair.2.unpair.2.unpair.1 u.unpair.2.
    unpair.1)
```

```
      (matchesBChar u.unpair.1 u.unpair.2.unpair.2.unpair.2 u.unpair.2.unpair.1) 1)
  1) := by
  have hfuel : Nat.Primrec (fun u : ℕ => u.unpair.1) := Nat.Primrec.left
  have hi : Nat.Primrec (fun u : ℕ => u.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hc1 : Nat.Primrec (fun u : ℕ => u.unpair.2.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hc2 : Nat.Primrec (fun u : ℕ => u.unpair.2.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hguard : Nat.Primrec (fun u => subsetGuardChar u.unpair.1 u.unpair.2.unpair
  .2.unpair.1
      u.unpair.2.unpair.2.unpair.2 u.unpair.2.unpair.1) := by
    have hbound : Nat.Primrec (fun u => subsetBoundChar u.unpair.1
        u.unpair.2.unpair.2.unpair.1 u.unpair.2.unpair.2.unpair.2) := by
      have h1 : Nat.Primrec (fun u => autStateCardFuelChar u.unpair.1
          u.unpair.2.unpair.2.unpair.1) := by
        have := primrec_autStateCardFuelChar2
        exact (this.comp (Nat.Primrec.pair hfuel hc1)).of_eq fun u => by
          simp only [unpair_pair_fst, unpair_pair_snd]
      have h2 : Nat.Primrec (fun u => autStateCardFuelChar u.unpair.1
          u.unpair.2.unpair.2.unpair.2) := by
        have := primrec_autStateCardFuelChar2
        exact (this.comp (Nat.Primrec.pair hfuel hc2)).of_eq fun u => by
          simp only [unpair_pair_fst, unpair_pair_snd]
      exact (primrec_mul₂ (primrec_pow₂ (Nat.Primrec.const 2) h1)
        (primrec_pow₂ (Nat.Primrec.const 2) h2))
    exact (primrec_mulBit.comp (Nat.Primrec.pair
      (primrec_allBinDigitsChar.comp hi)
      (primrec_isOne.comp (primrec_sub₂ (Nat.Primrec.const 1)
        (primrec_sub₂ (primrec_listLenChar.comp hi) hbound))))).of_eq fun u => by
      simp only [unpair_pair_fst, unpair_pair_snd, subsetGuardChar, mulBit]
  have hm1 : Nat.Primrec (fun u => matchesBChar u.unpair.1 u.unpair.2.unpair.2.
  unpair.1
      u.unpair.2.unpair.1) := by
    have := primrec_matchesBChar2
    exact (this.comp (Nat.Primrec.pair hfuel (Nat.Primrec.pair hc1 hi))).of_eq fun u
   => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hm2 : Nat.Primrec (fun u => matchesBChar u.unpair.1 u.unpair.2.unpair.2.
  unpair.2
      u.unpair.2.unpair.1) := by
    have := primrec_matchesBChar2
    exact (this.comp (Nat.Primrec.pair hfuel (Nat.Primrec.pair hc2 hi))).of_eq fun u
   => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact primrec_selectFn hguard (primrec_selectFn hm1 hm2 (Nat.Primrec.const 1))
    (Nat.Primrec.const 1)
have hb := primrec_bForallFn_param hg
have hpack : Nat.Primrec (fun s => Nat.pair
    (Nat.pair s.unpair.1 (Nat.pair s.unpair.2.unpair.1 s.unpair.2.unpair.2))
    (codeBound (subsetBoundChar s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.unpair.2))
  ) := by
  have hfuel : Nat.Primrec (fun s : ℕ => s.unpair.1) := Nat.Primrec.left
  have hc1 : Nat.Primrec (fun s : ℕ => s.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hc2 : Nat.Primrec (fun s : ℕ => s.unpair.2.unpair.2) :=
```

```
      Nat.Primrec.right.comp Nat.Primrec.right
    have h1 : Nat.Primrec (fun s => autStateCardFuelChar s.unpair.1 s.unpair.2.unpair
    .1) := by
      have := primrec_autStateCardFuelChar2
      exact (this.comp (Nat.Primrec.pair hfuel hc1)).of_eq fun s => by
        simp only [unpair_pair_fst, unpair_pair_snd]
    have h2 : Nat.Primrec (fun s => autStateCardFuelChar s.unpair.1 s.unpair.2.unpair
    .2) := by
      have := primrec_autStateCardFuelChar2
      exact (this.comp (Nat.Primrec.pair hfuel hc2)).of_eq fun s => by
        simp only [unpair_pair_fst, unpair_pair_snd]
    have hbnd : Nat.Primrec (fun s => subsetBoundChar s.unpair.1 s.unpair.2.unpair.1
        s.unpair.2.unpair.2) :=
      primrec_mul₂ (primrec_pow₂ (Nat.Primrec.const 2) h1) (primrec_pow₂ (Nat.Primrec.
    const 2) h2)

    exact Nat.Primrec.pair (Nat.Primrec.pair hfuel (Nat.Primrec.pair hc1 hc2))
      (primrec_codeBound.comp hbnd)
  exact (hb.comp hpack).of_eq fun s => by
    simp only [unpair_pair_fst, unpair_pair_snd, subsetBChar]

theorem primrec_interEqChar :
    Nat.Primrec (fun s => interEqChar s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.unpair
    .2) := by
  have hswap : Nat.Primrec (fun s : ℕ =>
      Nat.pair s.unpair.1 (Nat.pair s.unpair.2.unpair.2 s.unpair.2.unpair.1)) :=
    Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.right.comp Nat.Primrec.right)
        (Nat.Primrec.left.comp Nat.Primrec.right))
  have h1 := primrec_subsetBChar
  have h2 : Nat.Primrec (fun s => subsetBChar s.unpair.1 s.unpair.2.unpair.2 s.unpair
    .2.unpair.1) :=
    (primrec_subsetBChar.comp hswap).of_eq fun s => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_mulBit.comp (Nat.Primrec.pair h1 h2)).of_eq fun s => by
    simp only [unpair_pair_fst, unpair_pair_snd, interEqChar]

theorem subsetBChar_le_one (fuel c1 c2 : ℕ) : subsetBChar fuel c1 c2 ≤ 1 := by
  unfold subsetBChar; exact bForallFn_le_one _ _ _

theorem interEqChar_le_one (fuel c1 c2 : ℕ) : interEqChar fuel c1 c2 ≤ 1 := by
  unfold interEqChar; exact mulBit_le_one (subsetBChar_le_one _ _ _) (
    subsetBChar_le_one _ _ _)

end Domain.Recursive

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive

namespace RegularLanguage
open Set Domain.Recursive

variable {α : Type*}

structure ConsistencyPresentation (V : NeighborhoodSystem α) where
```

```
  X : ℕ → Set α

  mem_X : ∀ n, V.mem (X n)

  surj : ∀ {Y : Set α}, V.mem Y → ∃ n, X n = Y

  cons_computable : RecDecidable₂ (fun n m => ∃ k, X k ⊆ X n ∩ X m)

end RegularLanguage
end DomainTheory.NeighbourhoodSystems
```

4.8.54 Language presentation

Formal languages, coded by expressions, form a partially effectively given neighbourhood system.

Definition (`SsysX_default`). The default presented language is the universal language of all finite Boolean words.

Definition (`boolNat`). A Boolean value is represented by zero for false and one for true.

Theorem (`boolNat_eq_one_iff`). The zero–one test bool nat returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in bool nat eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`boolNat_zero_one`). For every input, bool nat takes one of the two values zero and one.

Proof. Each Boolean-valued component of bool nat zero one is zero or one, and the defining Boolean combination preserves that range. ■

Definition (`encodeListBool`). The coding function list bool assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`SExpr.encode`). The coding function encode assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`natBool`). A natural number decodes to false at zero, true at one, and fails at every larger number.

Definition (`decodeListBool`). The decoding operation list bool recovers the finite list, expression, or tagged value represented by its numerical input, returning failure when the code is malformed.

Definition (`decodeFuel`). The decoding operation fuel recovers the finite list, expression, or tagged value represented by its numerical input, returning failure when the code is malformed.

Definition (`SExpr.decode`). The decoding operation decode recovers the finite list, expression, or tagged value represented by its numerical input, returning failure when the code is malformed.

Definition (`SsysX`). An index denotes the decoded nonempty regular language when decoding succeeds and otherwise denotes the universal language.

Theorem (`SsysX_mem`). For the objects in ssys x mem, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for ssys x mem and applying the preceding component lemmas yields the claimed result. ■

Theorem (`decodeFuel_pair_cap_isSome_iff`). The condition decode fuel pair cap is some holds exactly when the finite witness condition holds.

Proof. Expand both conditions in fuel pair cap is some iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`decodeFuel_pair_cat_isSome_iff`). The condition decode fuel pair cat is some holds exactly when the finite witness condition holds.

Proof. Expand both conditions in fuel pair cat is some iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`decodeFuel_succ_cap`). At a successor input, fuel succ cap satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of fuel succ cap gives this equation immediately. ■

Theorem (`decodeFuel_succ_cat`). At a successor input, fuel succ cat satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of fuel succ cat gives this equation immediately. ■

Theorem (`decodeFuel_succ_junk`). At a successor input, fuel succ junk satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of fuel succ junk gives this equation immediately. ■

Theorem (`decodeFuel_succ_not_sigma`). At a successor input, fuel succ not sigma satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of fuel succ not sigma gives this equation immediately. ■

Theorem (`decodeFuel_succ_sigma`). At a successor input, fuel succ sigma satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of fuel succ sigma gives this equation immediately. ■

Theorem (`decodeFuel_succ_single`). At a successor input, fuel succ single satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of fuel succ single gives this equation immediately. ■

Theorem (`decodeListBool_encodeListBool`). For the objects in list bool encode list bool, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for list bool encode list bool and applying the preceding component lemmas yields the claimed result. ■

Theorem (`natBinDigit_of_natBool_some`). The hypotheses nat bool some imply nat bin digit.

Proof. Following the defining recursion for nat bin digit of nat bool some and applying the preceding component lemmas yields the claimed result. ■

Theorem (`natBool_isSome_iff`). The condition nat bool is some holds exactly when the finite witness condition holds.

Proof. Expand both conditions in nat bool is some iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`mapM_natBool_isSome_iff`). The condition map m nat bool is some holds exactly when the finite witness condition holds.

Proof. Expand both conditions in map m nat bool is some iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`decodeListBool_isSome_iff`). The condition decode list bool is some holds exactly when the finite witness condition holds.

Proof. Expand both conditions in list bool is some iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`decodeFuelOkChar_eq_one_iff`). The zero–one test decode fuel ok char returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in fuel ok char eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Definition (`safeDecodeActive`). The decoding operation safe decode active recovers the finite list, expression, or tagged value represented by its numerical input, returning failure when the code is malformed.

Theorem (`SsysX_eq_denote_safe`). The value of ssys x equals the value described by denote safe.

Proof. Compare membership or numerical components in ssys x eq denote safe; the defining equations reduce both sides to the same data. ■

Theorem (`safeDecodeActive_nonempty`). The set or language safe decode active nonempty has an element under the hypotheses of the theorem.

Proof. Choose witnesses from the nonempty input sets in safe decode active nonempty and combine them using the defining construction. ■

Theorem (`consistentB_safe_sigma`). For the objects in consistent b safe sigma, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for consistent b safe sigma and applying the preceding component lemmas yields the claimed result. ■

Theorem (`consistentB_sigma_safe`). For the objects in consistent b sigma safe, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for consistent b sigma safe and applying the preceding component lemmas yields the claimed result. ■

Definition (`sexprDepth`). Expression depth is one at leaves and one plus the maximum depth of the two children at binary constructors.

Theorem (`decodeFuel_encode`). For the objects in fuel encode, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for fuel encode and applying the preceding component lemmas yields the claimed result. ■

Definition (`SExpr.index`). The coding function index assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Theorem (`SExpr.decode_index`). For the objects in index, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for index and applying the preceding component lemmas yields the claimed result. ■

Theorem (`SsysX_surj`). Every object in the target of ssys x surj is represented by some input.

Proof. Choose the finite code or canonical representative supplied by the target object; decoding that choice gives the required preimage for ssys x surj. ■

Definition (`ssysActive`). An index is active exactly when it decodes to an expression with nonempty denotation.

Theorem (`safeDecodeActive_inactive`). For the objects in safe decode active inactive, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for safe decode active inactive and applying the preceding component lemmas yields the claimed result. ■

Definition (`ssysConsistentB`). Two presentation indices are declared consistent automatically if either is inactive, and otherwise by testing consistency of their decoded expressions.

Theorem (`ssysConsistentB_iff`). The condition ssys consistent b holds exactly when the finite witness condition holds.

Proof. Expand both conditions in ssys consistent b iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`ssys_cons_positivity`). For the objects in ssys cons positivity, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for ssys cons positivity and applying the preceding component lemmas yields the claimed result. ■

Theorem (`ssys_inter_nonempty_iff_consistent`). The set or language ssys inter nonempty iff consistent has an element under the hypotheses of the theorem.

Proof. Choose witnesses from the nonempty input sets in ssys inter nonempty iff consistent and combine them using the defining construction. ■

Theorem (`ssys_cons_iff`). The condition ssys cons holds exactly when the finite witness condition holds.

Proof. Expand both conditions in ssys cons iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`ssys_cons_char_iff`). The condition ssys cons char holds exactly when the finite witness condition holds.

Proof. Expand both conditions in ssys cons char iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`example_722_presentation_inst47`). Scott's example example 722 presentation inst47 has the claimed language-theoretic or computability property.

Proof. Following the defining recursion for example 722 presentation inst47 and applying the preceding component lemmas yields the claimed result. ■

Definition (`PartiallyEffectivelyGiven`). A neighbourhood system is partially effectively given when it admits a consistency presentation.

Theorem (`c9b5_encodeListBool_eq`). The value of c9b5 encode list bool eq equals the value described by c9b5 encode list bool eq.

Proof. Compare membership or numerical components in c9b5 encode list bool eq; the defining equations reduce both sides to the same data. ■

Theorem (`c9b5_sexprDepth_eq`). The value of c9b5 sexpr depth eq equals the value described by c9b5 sexpr depth eq.

Proof. Compare membership or numerical components in c9b5 sexpr depth eq; the defining equations reduce both sides to the same data. ■

Theorem (`c9b5_sexprGodelEncode_eq`). The value of c9b5 sexpr godel encode eq equals the value described by c9b5 sexpr godel encode eq.

Proof. Compare membership or numerical components in c9b5 sexpr godel encode eq; the defining equations reduce both sides to the same data. ■

Theorem (`decodeFuel_depth_le`). The element or numerical value fuel depth le lies below the bound in the theorem.

Proof. Following the defining recursion for fuel depth le and applying the preceding component lemmas yields the claimed result. ■

Theorem (`natBool_eq_some_iff`). The condition nat bool eq some holds exactly when the finite witness condition holds.

Proof. Expand both conditions in nat bool eq some iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`mapM_natBool_eq_some`). The value of map m nat bool equals the value described by some.

Proof. Compare membership or numerical components in map m nat bool eq some; the defining equations reduce both sides to the same data. ■

Theorem (`decodeListBool_eq_some`). The value of decode list bool equals the value described by some.

Proof. Compare membership or numerical components in list bool eq some; the defining equations reduce both sides to the same data. ■

Theorem (`decodeFuel_sound`). Every finite witness accepted by fuel sound determines a valid semantic relation.

Proof. Induction over the finite witness for fuel sound composes each valid step, producing the semantic relation at its endpoint. ■

Definition (`ssysActiveChar`). The numerical characteristic function ssys active char encodes its Boolean or relational test by zero and one.

Theorem (`primrec_ssysActiveChar`). The numerical function ssys active char is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of ssys active char. ■

Theorem (`ssysActiveChar_eq_one_iff`). The zero–one test ssys active char returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in ssys active char eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`ssysActiveChar_le_one`). For every input, ssys active char takes one of the two values zero and one.

Proof. Each Boolean-valued component of ssys active char le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`ssysActive_exists_decode`). For the objects in ssys active exists decode, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for ssys active exists decode and applying the preceding component lemmas yields the claimed result. ■

Definition (`ssysCanonicalCode`). The coding function ssys canonical code assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`ssysConsChar`). The numerical characteristic function ssys cons char encodes its Boolean or relational test by zero and one.

Theorem (`ssysConsChar_eq_one_iff`). The zero–one test ssys cons char returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in ssys cons char eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`ssysConsChar_le_one`). For every input, ssys cons char takes one of the two values zero and one.

Proof. Each Boolean-valued component of ssys cons char le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`ssysConsChar_zero_one`). For every input, ssys cons char takes one of the two values zero and one.

Proof. Each Boolean-valued component of ssys cons char zero one is zero or one, and the defining Boolean combination preserves that range. ■

Definition (`ssysConsistentBChar`). The numerical characteristic function ssys consistent bchar encodes its Boolean or relational test by zero and one.

Theorem (`primrec_ssysConsistentBChar`). The numerical function ssys consistent bchar is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of ssys consistent bchar. ■

Theorem (`ssysConsistentBChar_eq_one_iff`). The zero–one test ssys consistent bchar returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in ssys consistent bchar eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`ssysConsistentBChar_le_one`). For every input, ssys consistent bchar takes one of the two values zero and one.

Proof. Each Boolean-valued component of ssys consistent bchar le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`ssysConsChar_eq_ssysConsistentBChar`). The value of ssys cons char equals the value described by ssys consistent bchar.

Proof. Compare membership or numerical components in ssys cons char eq ssys consistent bchar; the defining equations reduce both sides to the same data. ■

Theorem (`primrec_ssysConsChar`). The numerical function ssys cons char is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of ssys cons char. ■

Definition (`ssysInterEqChar`). The numerical characteristic function ssys inter eq char encodes its Boolean or relational test by zero and one.

Theorem (`ssys_cons_char_eq_one_iff`). The zero–one test ssys cons char returns one exactly when the mathematical condition encoded in its definition holds.

Proof. The consistency character is one exactly when the preceding positivity result supplies a common indexed refinement of the two neighbourhoods. ■

Theorem (`Ssys_cons_computable_of_primrec_ssysConsChar`). The relation ssys cons computable of primrec ssys cons char is recursively decidable.

Proof. Reduce the relation for ssys cons computable of primrec ssys cons char to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`Ssys_cons_computable`). The relation ssys cons computable is recursively decidable.

Proof. Reduce the relation for ssys cons computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Definition (`SsysPres`). The ssys pres is the computable presentation whose enumeration and decision procedures are those constructed above.

Theorem (`c9b5_boolNat_eq`). The value of c9b5 bool nat eq equals the value described by c9b5 bool nat eq.

Proof. Compare membership or numerical components in c9b5 bool nat eq; the defining equations reduce both sides to the same data. ■

Theorem (`ssysCanonicalCode_eq`). The value of ssys canonical code eq equals the value described by ssys canonical code eq.

Proof. Compare membership or numerical components in ssys canonical code eq; the defining equations reduce both sides to the same data. ■

Theorem (`sexprDepth_safeDecodeActive_le`). The element or numerical value sexpr depth safe decode active le lies below the bound in the theorem.

Proof. Following the defining recursion for sexpr depth safe decode active le and applying the preceding component lemmas yields the claimed result. ■

Theorem (`ssys_interEq_iff`). The intersection in ssys inter eq iff equals the indexed neighbourhood or language exactly under the stated finite comparison condition.

Proof. Use the same witnesses in both directions for ssys inter eq iff, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`ssysInterEqChar_le_one`). For every input, ssys inter eq char takes one of the two values zero and one.

Proof. Each Boolean-valued component of ssys inter eq char le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`ssysInterEqChar_eq_one_iff`). The zero–one test ssys inter eq char returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in ssys inter eq char eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`primrec_ssysInterEqChar`). The numerical function ssys inter eq char is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of ssys inter eq char. ■

Theorem (`Ssys_interEq_computable`). The relation ssys inter eq computable is recursively decidable.

Proof. Use the same witnesses in both directions for ssys inter eq computable, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`Ssys_partially_effectively_given`). For the objects in ssys partially effectively given, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for ssys partially effectively given and applying the preceding component lemmas yields the claimed result. ■

Theorem (`example_722_presentation`). Scott's example example 722 presentation has the claimed language-theoretic or computability property.

Proof. Following the defining recursion for example 722 presentation and applying the preceding component lemmas yields the claimed result. ■

`languagePresentation.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
```

```
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_15
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_24
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .regularCode
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .regularLanguage

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace RegularLanguage
open Set Domain.Recursive

def SsysX_default : Set (List Bool) := Set.univ

def boolNat (b : Bool) : ℕ := if b then 1 else 0

theorem boolNat_eq_one_iff (b : Bool) : boolNat b = 1 ↔ b := by
  cases b <;> simp [boolNat]

theorem boolNat_zero_one (b : Bool) : boolNat b = 0 ∨ boolNat b = 1 := by
  cases b <;> simp [boolNat]

def encodeListBool (σ : List Bool) : ℕ := encodeList (σ.map boolNat)

def SExpr.encode : SExpr → ℕ
  | .sigma => Nat.pair 0 0
  | .single σ => Nat.pair 1 (encodeListBool σ)
  | .cat a b => Nat.pair 2 (Nat.pair (encode a) (encode b))
  | .cap a b => Nat.pair 3 (Nat.pair (encode a) (encode b))

end RegularLanguage
namespace RegularLanguage
open Set Domain.Recursive

def natBool (n : ℕ) : Option Bool :=
  match n with | 0 => some false | 1 => some true | _ => none

def decodeListBool (n : ℕ) : Option (List Bool) :=
  (decodeList n).mapM natBool

def decodeFuel (fuel : ℕ) (n : ℕ) : Option SExpr :=
  match fuel with
  | 0 => none
  | fuel + 1 =>
    match n.unpair.1 with
    | 0 => if n.unpair.2 = 0 then some .sigma else none
    | 1 => (decodeListBool n.unpair.2).map (.single ·)
    | 2 =>
      match decodeFuel fuel n.unpair.2.unpair.1, decodeFuel fuel n.unpair.2.unpair.2
    with
      | some a, some b => some (.cat a b)
      | _, _ => none
```

```
    | 3 =>
      match decodeFuel fuel n.unpair.2.unpair.1, decodeFuel fuel n.unpair.2.unpair.2
    with
      | some a, some b => some (.cap a b)
      | _, _ => none
    | _ => none

def SExpr.decode (n : ℕ) : Option SExpr := decodeFuel (n.unpair.2 + 1) n.unpair.1

def SsysX (n : ℕ) : Set (List Bool) :=
  match SExpr.decode n with
  | none => SsysX_default
  | some e =>
    if decideNonemptyB e then denote e else SsysX_default

theorem SsysX_mem (n : ℕ) : InS (SsysX n) := by
  unfold SsysX
  cases hdec : SExpr.decode n with
  | none => simp; exact InS.univ
  | some e =>
    by_cases h : decideNonemptyB e = true
    · simp [h]
      exact InS_denote_of_nonempty ((decideNonemptyB_iff e).1 h)
    · simp [h]
      exact InS.univ

theorem decodeFuel_pair_cap_isSome_iff (fuel c : ℕ) :
    (match decodeFuel fuel c.unpair.2.unpair.1, decodeFuel fuel c.unpair.2.unpair.2
    with
      | some a, some b => some (SExpr.cap a b) | _, _ => none).isSome = true ↔
      (decodeFuel fuel c.unpair.2.unpair.1).isSome = true ∧
        (decodeFuel fuel c.unpair.2.unpair.2).isSome = true := by
  cases decodeFuel fuel c.unpair.2.unpair.1 <;> cases decodeFuel fuel c.unpair.2.
    unpair.2 <;> simp

theorem decodeFuel_pair_cat_isSome_iff (fuel c : ℕ) :
    (match decodeFuel fuel c.unpair.2.unpair.1, decodeFuel fuel c.unpair.2.unpair.2
    with
      | some a, some b => some (SExpr.cat a b) | _, _ => none).isSome = true ↔
      (decodeFuel fuel c.unpair.2.unpair.1).isSome = true ∧
        (decodeFuel fuel c.unpair.2.unpair.2).isSome = true := by
  cases decodeFuel fuel c.unpair.2.unpair.1 <;> cases decodeFuel fuel c.unpair.2.
    unpair.2 <;> simp

theorem decodeFuel_succ_cap (fuel c : ℕ) (h : c.unpair.1 = 3) :
    decodeFuel (fuel + 1) c =
      match decodeFuel fuel c.unpair.2.unpair.1, decodeFuel fuel c.unpair.2.unpair.2
    with
      | some a, some b => some (.cap a b)
      | _, _ => none := by
  simp [decodeFuel, h]

theorem decodeFuel_succ_cat (fuel c : ℕ) (h : c.unpair.1 = 2) :
    decodeFuel (fuel + 1) c =
      match decodeFuel fuel c.unpair.2.unpair.1, decodeFuel fuel c.unpair.2.unpair.2
    with
```

```
      | some a, some b => some (.cat a b)
      | _, _ => none := by
  simp [decodeFuel, h]

theorem decodeFuel_succ_junk (fuel c : ℕ) (h : 4 ≤ c.unpair.1) :
    decodeFuel (fuel + 1) c = none := by
  match tag : c.unpair.1 with
  | 0 | 1 | 2 | 3 => omega
  | t + 4 => simp [decodeFuel, tag]

theorem decodeFuel_succ_not_sigma (fuel c : ℕ) (h : c.unpair.1 = 0) (hσ : c.unpair.2 =
    0) :
    decodeFuel (fuel + 1) c = none := by simp [decodeFuel, h, hσ]

theorem decodeFuel_succ_sigma (fuel c : ℕ) (h : c.unpair.1 = 0) (hσ : c.unpair.2 = 0)
    :
    decodeFuel (fuel + 1) c = some .sigma := by simp [decodeFuel, h, hσ]

theorem decodeFuel_succ_single (fuel c : ℕ) (h : c.unpair.1 = 1) :
    decodeFuel (fuel + 1) c = (decodeListBool c.unpair.2).map (.single ·) := by
  simp [decodeFuel, h]

theorem decodeListBool_encodeListBool (σ : List Bool) :
    decodeListBool (encodeListBool σ) = some σ := by
  simp [decodeListBool, encodeListBool, decodeList_encodeList, List.mapM_map]
  induction σ with
  | nil => rfl
  | cons b bs ih =>
    cases b <;> simp [boolNat, natBool, ih]

theorem natBinDigit_of_natBool_some (n : ℕ) {b : Bool} (h : natBool n = some b) :
    n = 0 ∨ n = 1 := by
  match n with
  | 0 => simp [natBool] at h; exact Or.inl rfl
  | 1 => simp [natBool] at h; exact Or.inr rfl
  | n + 2 => simp [natBool] at h

theorem natBool_isSome_iff (n : ℕ) :
    (natBool n).isSome = true ↔ n = 0 ∨ n = 1 := by
  match n with
  | 0 => simp [natBool]
  | 1 => simp [natBool]
  | n + 2 => simp [natBool]

theorem mapM_natBool_isSome_iff (l : List ℕ) :
    (l.mapM natBool).isSome = true ↔ ∀ x ∈ l, x = 0 ∨ x = 1 := by
  induction l with
  | nil => simp
  | cons x xs ih =>
    rw [List.mapM_cons]
    constructor
    · intro h
      cases hnx : natBool x with
      | none => simp [hnx] at h
      | some b =>
        cases htx : xs.mapM natBool with
```

```
        | none => simp [hnx, htx] at h
        | some ts =>
          have hxs' : (xs.mapM natBool).isSome = true := by simp [htx]
          intro y hy
          rcases List.mem_cons.mp hy with hEq | hyTail
          · rw [hEq]
            exact natBinDigit_of_natBool_some x hnx
          · exact ih.mp hxs' y hyTail
    · intro h
      have hx : (natBool x).isSome = true :=
        (natBool_isSome_iff x).2 (h x (List.mem_cons_self ..))
      have hxs : (xs.mapM natBool).isSome = true :=
        ih.mpr fun y hy => h y (List.mem_cons_of_mem x hy)
      cases hnx : natBool x with
      | none =>
        rcases h x (List.mem_cons_self ..) with h0 | h1
        · subst h0; simp [natBool] at hnx
        · subst h1; simp [natBool] at hnx
      | some b =>
        cases htx : xs.mapM natBool with
        | none => simp [htx] at hxs
        | some ts => simp

theorem decodeListBool_isSome_iff (n : ℕ) :
    (decodeListBool n).isSome = true ↔ allBinDigitsChar n = 1 := by
  simp only [decodeListBool, mapM_natBool_isSome_iff, allBinDigitsChar_eq_one_iff]

theorem decodeFuelOkChar_eq_one_iff (fuel c : ℕ) :
    decodeFuelOkChar fuel c = 1 ↔ (decodeFuel fuel c).isSome = true := by
  induction fuel generalizing c with
  | zero =>
    simp [decodeFuelOkChar, decodeFuel]
  | succ fuel ih =>
    simp only [decodeFuelOkChar]
    rw [decodeFuelOkCharBody_eq]
    match tag : c.unpair.1 with
    | 0 =>
      rw [show decodeFuel (fuel + 1) c = if c.unpair.2 = 0 then some .sigma else none
    from by
        by_cases hσ : c.unpair.2 = 0 <;> simp [decodeFuel, tag, hσ]]
      simp [selectFn_isOne_one_sub_sigma]
    | 1 =>
      rw [decodeFuel_succ_single fuel c tag]
      simp [decodeListBool_isSome_iff, Option.isSome_map]
    | 2 =>
      simp
      rw [decodeFuel_succ_cat fuel c tag, mulBit_eq_one_iff]
      have ih1 := ih c.unpair.2.unpair.1
      have ih2 := ih c.unpair.2.unpair.2
      cases hL : decodeFuel fuel c.unpair.2.unpair.1 <;>
        cases hR : decodeFuel fuel c.unpair.2.unpair.2 <;>
        simp [hL, hR, ih1, ih2]
    | 3 =>
      simp
      rw [decodeFuel_succ_cap fuel c tag, mulBit_eq_one_iff]
      have ih1 := ih c.unpair.2.unpair.1
```

```
      have ih2 := ih c.unpair.2.unpair.2
      cases hL : decodeFuel fuel c.unpair.2.unpair.1 <;>
        cases hR : decodeFuel fuel c.unpair.2.unpair.2 <;>
        simp [hL, hR, ih1, ih2]
    | t + 4 =>
      rw [decodeFuel_succ_junk fuel c (by omega)]
      simp

end RegularLanguage
namespace RegularLanguage
open Set Domain.Recursive

def safeDecodeActive (n : ℕ) : SExpr :=
  match SExpr.decode n with
  | none => .sigma
  | some e => if decideNonemptyB e then e else .sigma

theorem SsysX_eq_denote_safe (n : ℕ) : SsysX n = denote (safeDecodeActive n) := by
  unfold SsysX safeDecodeActive
  cases hdec : SExpr.decode n with
  | none => simp [denote_sigma, SsysX_default]
  | some e =>
    by_cases hb : decideNonemptyB e = true
    · simp [hb]
    · simp [hb, denote_sigma, SsysX_default]

theorem safeDecodeActive_nonempty (n : ℕ) : (denote (safeDecodeActive n)).Nonempty :=
    by
  unfold safeDecodeActive
  cases hdec : SExpr.decode n with
  | none => exact Set.univ_nonempty
  | some e =>
    by_cases hb : decideNonemptyB e = true
    · simp [hb]; exact (decideNonemptyB_iff e).mp hb
    · simp [hb, denote_sigma]

theorem consistentB_safe_sigma (n : ℕ) : consistentB (safeDecodeActive n) .sigma =
    true := by
  rw [consistentB_iff, denote_cap, denote_sigma, Set.inter_univ]
  exact safeDecodeActive_nonempty n

theorem consistentB_sigma_safe (m : ℕ) : consistentB .sigma (safeDecodeActive m) =
    true := by
  rw [consistentB_iff, denote_cap, denote_sigma, Set.univ_inter]
  exact safeDecodeActive_nonempty m

def sexprDepth : SExpr → ℕ
  | .sigma => 1
  | .single _ => 1
  | .cat a b => 1 + max (sexprDepth a) (sexprDepth b)
  | .cap a b => 1 + max (sexprDepth a) (sexprDepth b)

theorem decodeFuel_encode {fuel : ℕ} {e : SExpr} (h : sexprDepth e ≤ fuel) :
    decodeFuel fuel (SExpr.encode e) = some e := by
  induction e generalizing fuel with
  | sigma =>
```

```
    cases fuel with
    | zero => simp [sexprDepth] at h
    | succ fuel =>
      simp [decodeFuel, SExpr.encode]
  | single σ =>
    cases fuel with
    | zero => simp [sexprDepth] at h
    | succ fuel =>
      simp [decodeFuel, SExpr.encode, decodeListBool_encodeListBool σ]
  | cat a b iha ihb =>
    cases fuel with
    | zero => simp [sexprDepth] at h
    | succ fuel =>
      have ha : sexprDepth a ≤ fuel := by
        simp only [sexprDepth] at h ⊢
        omega
      have hb : sexprDepth b ≤ fuel := by
        simp only [sexprDepth] at h ⊢
        omega
      simp [decodeFuel, SExpr.encode, iha ha, ihb hb]
  | cap a b iha ihb =>
    cases fuel with
    | zero => simp [sexprDepth] at h
    | succ fuel =>
      have ha : sexprDepth a ≤ fuel := by
        simp only [sexprDepth] at h ⊢
        omega
      have hb : sexprDepth b ≤ fuel := by
        simp only [sexprDepth] at h ⊢
        omega
      simp [decodeFuel, SExpr.encode, iha ha, ihb hb]

def SExpr.index (e : SExpr) : ℕ := Nat.pair (encode e) (sexprDepth e)

theorem SExpr.decode_index (e : SExpr) : decode (index e) = some e := by
  simp [decode, index]
  exact decodeFuel_encode (Nat.le_succ (sexprDepth e))

theorem SsysX_surj {Y : Set (List Bool)} (hY : InS Y) : ∃ n, SsysX n = Y := by
  obtain ⟨e, he, hne⟩ := inS_iff_exists_denote.mp hY
  refine ⟨SExpr.index e, ?_⟩
  unfold SsysX
  rw [SExpr.decode_index e]
  have hne' : (denote e).Nonempty := he ▸ hne
  have hb : decideNonemptyB e = true := (decideNonemptyB_iff e).2 hne'
  simp [hb, he]

def ssysActive (n : ℕ) : Bool :=
  match SExpr.decode n with
  | none => false
  | some e => decideNonemptyB e

theorem safeDecodeActive_inactive (n : ℕ) (h : ssysActive n = false) :
    safeDecodeActive n = .sigma := by
  dsimp only [ssysActive, safeDecodeActive] at h ⊢
  cases hdec : SExpr.decode n with
```

```
  | none => rfl
  | some e =>
    by_cases hb : decideNonemptyB e
    · exfalso
      simp [hdec, hb] at h
    · simp [hb]

def ssysConsistentB (n m : ℕ) : Bool :=
  if !ssysActive n || !ssysActive m then true
  else
    match SExpr.decode n, SExpr.decode m with
    | some a, some b => consistentB a b
    | _, _ => true

theorem ssysConsistentB_iff (n m : ℕ) :
    ssysConsistentB n m = true ↔
      consistentB (safeDecodeActive n) (safeDecodeActive m) = true := by
  by_cases hn : ssysActive n
  · by_cases hm : ssysActive m
    · cases hdecn : SExpr.decode n with
      | none => exfalso; simp [ssysActive, hdecn] at hn
      | some a =>
        cases hdecm : SExpr.decode m with
        | none => exfalso; simp [ssysActive, hdecm] at hm
        | some b =>
          have ha : decideNonemptyB a := by simpa [ssysActive, hdecn] using hn
          have hb : decideNonemptyB b := by simpa [ssysActive, hdecm] using hm
          have hsa : safeDecodeActive n = a := by simp [safeDecodeActive, hdecn, ha]
          have hsb : safeDecodeActive m = b := by simp [safeDecodeActive, hdecm, hb]
          constructor
          · intro h
            rw [hsa, hsb]
            have heq : ssysConsistentB n m = consistentB a b := by
              unfold ssysConsistentB
              simp [hn, hm, hdecn, hdecm]
            rw [heq] at h
            exact h
          · intro hcons
            rw [hsa, hsb] at hcons
            have heq : ssysConsistentB n m = consistentB a b := by
              unfold ssysConsistentB
              simp [hn, hm, hdecn, hdecm]
            rw [heq]
            exact hcons
    · have hm' : ssysActive m = false := by simpa using hm
      constructor
      · intro _; rw [safeDecodeActive_inactive m hm', consistentB_safe_sigma n]
      · intro _; unfold ssysConsistentB; simp [hn, hm']
  · have hn' : ssysActive n = false := by simpa using hn
    constructor
    · intro _; rw [safeDecodeActive_inactive n hn', consistentB_sigma_safe m]
    · intro _; unfold ssysConsistentB; simp [hn']

theorem ssys_cons_positivity (n m : ℕ) :
    (∃ k, SsysX k ⊆ SsysX n ∩ SsysX m) ↔ (SsysX n ∩ SsysX m).Nonempty := by
  constructor
```

```
  · intro ⟨k, hk⟩
    exact Set.Nonempty.mono hk (InS.nonempty (SsysX_mem k))
  · intro hne
    have hIn : InS (SsysX n ∩ SsysX m) :=
      (Ssys_isPositive (Ssys_mem.mpr (SsysX_mem n)) (Ssys_mem.mpr (SsysX_mem m))).2
    hne
    obtain ⟨k, hk⟩ := SsysX_surj hIn
    exact ⟨k, hk.subset⟩

theorem ssys_inter_nonempty_iff_consistent (n m : ℕ) :
    (SsysX n ∩ SsysX m).Nonempty ↔ consistentB (safeDecodeActive n) (safeDecodeActive
    m) = true := by
  rw [SsysX_eq_denote_safe, SsysX_eq_denote_safe, ← denote_cap, consistentB_iff]

theorem ssys_cons_iff (n m : ℕ) :
    (∃ k, SsysX k ⊆ SsysX n ∩ SsysX m) ↔
      consistentB (safeDecodeActive n) (safeDecodeActive m) = true := by
  rw [ssys_cons_positivity, ssys_inter_nonempty_iff_consistent]

theorem ssys_cons_char_iff (n m : ℕ) :
    ssysConsistentB n m = true ↔ ∃ k, SsysX k ⊆ SsysX n ∩ SsysX m := by
  rw [ssysConsistentB_iff, ssys_cons_iff]

instance decidableSsysCons (n m : ℕ) :
    Decidable (∃ k, SsysX k ⊆ SsysX n ∩ SsysX m) :=

  decidable_of_iff (ssysConsistentB n m = true) (ssys_cons_char_iff n m)

end RegularLanguage
end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems.RegularLanguage
open NeighborhoodSystem Domain.Recursive ApproximableMap

/-- Consistency character for the Ssys presentation. -/
theorem example_722_presentation_inst47 (n m : ℕ) :
    ssysConsistentB n m = true ↔ ∃ k, SsysX k ⊆ SsysX n ∩ SsysX m :=
  ssys_cons_char_iff n m

end DomainTheory.NeighbourhoodSystems.RegularLanguage

namespace DomainTheory.NeighbourhoodSystems
namespace RegularLanguage
open Set Domain.Recursive

set_option maxHeartbeats 5000000

class PartiallyEffectivelyGiven (V : NeighborhoodSystem α) : Prop where
  out : Nonempty (ConsistencyPresentation V)

theorem c9b5_encodeListBool_eq (σ : List Bool) :
    Domain.Recursive.c9b5_encodeListBool σ = encodeListBool σ := by
  unfold Domain.Recursive.c9b5_encodeListBool encodeListBool
  congr 1

theorem c9b5_sexprDepth_eq (e : SExpr) : Domain.Recursive.c9b5_sexprDepth e =
```

```
    sexprDepth e := by
  induction e with
  | sigma => rfl
  | single _ => rfl
  | cat a b iha ihb => simp [Domain.Recursive.c9b5_sexprDepth, sexprDepth, iha, ihb]
  | cap a b iha ihb => simp [Domain.Recursive.c9b5_sexprDepth, sexprDepth, iha, ihb]

theorem c9b5_sexprGodelEncode_eq (e : SExpr) :
    Domain.Recursive.c9b5_sexprGodelEncode e = SExpr.encode e := by
  induction e with
  | sigma => rfl
  | single σ => simp [Domain.Recursive.c9b5_sexprGodelEncode, SExpr.encode,
    c9b5_encodeListBool_eq]
  | cat a b iha ihb => simp [Domain.Recursive.c9b5_sexprGodelEncode, SExpr.encode, iha
    , ihb]
  | cap a b iha ihb => simp [Domain.Recursive.c9b5_sexprGodelEncode, SExpr.encode, iha
    , ihb]

theorem decodeFuel_depth_le {fuel c : ℕ} {e : SExpr} (h : decodeFuel fuel c = some e)
    :
    sexprDepth e ≤ fuel := by
  induction fuel generalizing c e with
  | zero => simp [decodeFuel] at h
  | succ fuel ih =>
    match tag : c.unpair.1 with
    | 0 =>
      by_cases hσ : c.unpair.2 = 0
      · rw [decodeFuel_succ_sigma fuel c tag hσ] at h
        simp only [Option.some.injEq] at h
        simp [← h, sexprDepth]
      · rw [decodeFuel_succ_not_sigma fuel c tag hσ] at h; simp at h
    | 1 =>
      rw [decodeFuel_succ_single fuel c tag] at h
      cases hb : decodeListBool c.unpair.2 with
      | none => simp [hb] at h
      | some σ =>
        rw [hb] at h
        simp only [Option.map_some, Option.some.injEq] at h
        simp [← h, sexprDepth]
    | 2 =>
      rw [decodeFuel_succ_cat fuel c tag] at h
      cases ha : decodeFuel fuel c.unpair.2.unpair.1 with
      | none => simp [ha] at h
      | some a =>
        cases hb : decodeFuel fuel c.unpair.2.unpair.2 with
        | none => simp [ha, hb] at h
        | some b =>
          simp only [ha, hb, Option.some.injEq] at h
          have hda := ih ha
          have hdb := ih hb
          simp only [← h, sexprDepth]
          omega
    | 3 =>
      rw [decodeFuel_succ_cap fuel c tag] at h
      cases ha : decodeFuel fuel c.unpair.2.unpair.1 with
      | none => simp [ha] at h
```

```
      | some a =>
        cases hb : decodeFuel fuel c.unpair.2.unpair.2 with
        | none => simp [ha, hb] at h
        | some b =>
          simp only [ha, hb, Option.some.injEq] at h
          have hda := ih ha
          have hdb := ih hb
          simp only [← h, sexprDepth]
          omega
    | _ + 4 =>
      rw [decodeFuel_succ_junk fuel c (by omega)] at h; simp at h

theorem natBool_eq_some_iff (n : ℕ) (b : Bool) : natBool n = some b ↔ n = boolNat b :=
    by
  match n with
  | 0 => cases b <;> simp [natBool, boolNat]
  | 1 => cases b <;> simp [natBool, boolNat]
  | n + 2 => cases b <;> simp [natBool, boolNat]

theorem mapM_natBool_eq_some (l : List ℕ) (σ : List Bool)
    (h : l.mapM natBool = some σ) : l = σ.map boolNat := by
  induction l generalizing σ with
  | nil => rw [List.mapM_nil] at h; simpa using h.symm
  | cons x xs ih =>
    rw [List.mapM_cons] at h
    cases hnx : natBool x with
    | none => simp [hnx] at h
    | some b =>
      cases htx : xs.mapM natBool with
      | none => simp [hnx, htx] at h
      | some ys =>
        simp only [hnx, htx] at h
        obtain rfl : σ = b :: ys := by simpa using h.symm
        simp [ih ys htx, (natBool_eq_some_iff x b).mp hnx]

theorem decodeListBool_eq_some (n : ℕ) (σ : List Bool) (h : decodeListBool n = some σ)
    :
    n = encodeListBool σ := by
  have hl : decodeList n = σ.map boolNat := mapM_natBool_eq_some (decodeList n) σ h
  calc n = encodeList (decodeList n) := (encodeList_decodeList n).symm
    _ = encodeList (σ.map boolNat) := by rw [hl]
    _ = encodeListBool σ := rfl

theorem decodeFuel_sound {fuel c : ℕ} {e : SExpr} (h : decodeFuel fuel c = some e) :
    c = SExpr.encode e := by
  induction fuel generalizing c e with
  | zero => simp [decodeFuel] at h
  | succ fuel ih =>
    match tag : c.unpair.1 with
    | 0 =>
      by_cases hσ : c.unpair.2 = 0
      · have hc : c = Nat.pair 0 0 := by
          have hpu := Nat.pair_unpair c; rw [tag, hσ] at hpu; exact hpu.symm
        rw [decodeFuel_succ_sigma fuel c tag hσ] at h
        simp only [Option.some.injEq] at h
        simp [SExpr.encode, ← h, hc]
```

```
      · rw [decodeFuel_succ_not_sigma fuel c tag hσ] at h; simp at h
    | 1 =>
      have hc1 : c = Nat.pair 1 c.unpair.2 := by
        have hpu := Nat.pair_unpair c; rw [tag] at hpu; exact hpu.symm
      rw [decodeFuel_succ_single fuel c tag] at h
      cases hb : decodeListBool c.unpair.2 with
      | none => simp [hb] at h
      | some σ =>
        rw [hb] at h
        simp only [Option.map_some, Option.some.injEq] at h
        have hσc : c.unpair.2 = encodeListBool σ := decodeListBool_eq_some c.unpair.2
    σ hb
        rw [← h, SExpr.encode, hc1, hσc]
    | 2 =>
      have hc2 : c = Nat.pair 2 c.unpair.2 := by
        have hpu := Nat.pair_unpair c; rw [tag] at hpu; exact hpu.symm
      rw [decodeFuel_succ_cat fuel c tag] at h
      cases ha : decodeFuel fuel c.unpair.2.unpair.1 with
      | none => simp [ha] at h
      | some a =>
        cases hb : decodeFuel fuel c.unpair.2.unpair.2 with
        | none => simp [ha, hb] at h
        | some b =>
          simp only [ha, hb, Option.some.injEq] at h
          have hac : c.unpair.2.unpair.1 = SExpr.encode a := ih ha
          have hbc : c.unpair.2.unpair.2 = SExpr.encode b := ih hb
          have hpc : c.unpair.2 = Nat.pair (SExpr.encode a) (SExpr.encode b) := by
            have hpu := Nat.pair_unpair c.unpair.2; rw [hac, hbc] at hpu; exact hpu.
    symm
          rw [← h, SExpr.encode, hc2, hpc]
    | 3 =>
      have hc3 : c = Nat.pair 3 c.unpair.2 := by
        have hpu := Nat.pair_unpair c; rw [tag] at hpu; exact hpu.symm
      rw [decodeFuel_succ_cap fuel c tag] at h
      cases ha : decodeFuel fuel c.unpair.2.unpair.1 with
      | none => simp [ha] at h
      | some a =>
        cases hb : decodeFuel fuel c.unpair.2.unpair.2 with
        | none => simp [ha, hb] at h
        | some b =>
          simp only [ha, hb, Option.some.injEq] at h
          have hac : c.unpair.2.unpair.1 = SExpr.encode a := ih ha
          have hbc : c.unpair.2.unpair.2 = SExpr.encode b := ih hb
          have hpc : c.unpair.2 = Nat.pair (SExpr.encode a) (SExpr.encode b) := by
            have hpu := Nat.pair_unpair c.unpair.2; rw [hac, hbc] at hpu; exact hpu.
    symm
          rw [← h, SExpr.encode, hc3, hpc]
    | _ + 4 =>
      rw [decodeFuel_succ_junk fuel c (by omega)] at h; simp at h

def ssysActiveChar (n : ℕ) : ℕ :=
  mulBit (decodeFuelOkChar (n.unpair.2 + 1) n.unpair.1)
    (decideNonemptyBChar (n.unpair.2 + 1) n.unpair.1)

theorem primrec_ssysActiveChar : Nat.Primrec ssysActiveChar := by
  have hpack : Nat.Primrec (fun n : ℕ => Nat.pair (n.unpair.2 + 1) n.unpair.1) :=
```

```
    Nat.Primrec.pair (primrec_add₂ Nat.Primrec.right (Nat.Primrec.const 1)) Nat.Primrec
    .left
  have h1 : Nat.Primrec (fun n => decodeFuelOkChar (n.unpair.2 + 1) n.unpair.1) :=
    (primrec_decodeFuelOkChar2.comp hpack).of_eq fun n => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have h2 : Nat.Primrec (fun n => decideNonemptyBChar (n.unpair.2 + 1) n.unpair.1) :=
    (primrec_decideNonemptyBChar2.comp hpack).of_eq fun n => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_mulBit.comp (Nat.Primrec.pair h1 h2)).of_eq fun n => by
    simp only [unpair_pair_fst, unpair_pair_snd, ssysActiveChar]

theorem ssysActiveChar_eq_one_iff (n : ℕ) : ssysActiveChar n = 1 ↔ ssysActive n = true
    := by
  simp only [ssysActiveChar, ssysActive, SExpr.decode, mulBit_eq_one_iff,
    decodeFuelOkChar_eq_one_iff]
  cases hdec : decodeFuel (n.unpair.2 + 1) n.unpair.1 with
  | none => simp
  | some e =>
    have hc : n.unpair.1 = SExpr.encode e := decodeFuel_sound hdec
    have hdepth : Domain.Recursive.c9b5_sexprDepth e ≤ n.unpair.2 + 1 := by
      rw [c9b5_sexprDepth_eq]; exact decodeFuel_depth_le hdec
    simp only [Option.isSome_some, true_and]
    rw [hc, ← c9b5_sexprGodelEncode_eq, decideNonemptyBChar_eq_one_iff hdepth,
    decideNonemptyB_iff]

theorem ssysActiveChar_le_one (n : ℕ) : ssysActiveChar n ≤ 1 :=
  mulBit_le_one (decodeFuelOkChar_le_one _ _) (bExistsFn_le_one _ _ _)

theorem ssysActive_exists_decode {n : ℕ} (hn : ssysActive n = true) :
    ∃ e, SExpr.decode n = some e := by
  unfold ssysActive at hn
  cases hdec : SExpr.decode n with
  | none => simp [hdec] at hn
  | some e => exact ⟨e, rfl⟩

@[irreducible] def ssysCanonicalCode (n : ℕ) : ℕ :=
  Domain.Recursive.selectFn (ssysActiveChar n) n.unpair.1 (SExpr.encode .sigma)

def ssysConsChar (t : ℕ) : ℕ := boolNat (ssysConsistentB t.unpair.1 t.unpair.2)

theorem ssysConsChar_eq_one_iff (t : ℕ) :
    ssysConsChar t = 1 ↔ ssysConsistentB t.unpair.1 t.unpair.2 = true := by
  simp [ssysConsChar, boolNat_eq_one_iff]

theorem ssysConsChar_le_one (t : ℕ) : ssysConsChar t ≤ 1 := by
  have := boolNat_zero_one (ssysConsistentB t.unpair.1 t.unpair.2)
  unfold ssysConsChar
  omega

theorem ssysConsChar_zero_one (t : ℕ) : ssysConsChar t = 0 ∨ ssysConsChar t = 1 :=
  boolNat_zero_one _

def ssysConsistentBChar (n m : ℕ) : ℕ :=
  selectFn (mulBit (ssysActiveChar n) (ssysActiveChar m))
    (consistentBChar (n.unpair.2 + m.unpair.2 + 2) n.unpair.1 m.unpair.1)
    1
```

```
theorem primrec_ssysConsistentBChar :
    Nat.Primrec (fun t => ssysConsistentBChar t.unpair.1 t.unpair.2) := by
  have hcond : Nat.Primrec
      (fun t => mulBit (ssysActiveChar t.unpair.1) (ssysActiveChar t.unpair.2)) :=
    (primrec_mulBit.comp (Nat.Primrec.pair (primrec_ssysActiveChar.comp Nat.Primrec.
    left)
      (primrec_ssysActiveChar.comp Nat.Primrec.right))).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hfuel : Nat.Primrec (fun t : ℕ => t.unpair.1.unpair.2 + t.unpair.2.unpair.2 +
    2) :=
    primrec_add₂ (primrec_add₂ (Nat.Primrec.right.comp Nat.Primrec.left)
      (Nat.Primrec.right.comp Nat.Primrec.right)) (Nat.Primrec.const 2)
  have hpack : Nat.Primrec (fun t : ℕ => Nat.pair (t.unpair.1.unpair.2 + t.unpair.2.
    unpair.2 + 2)
      (Nat.pair t.unpair.1.unpair.1 t.unpair.2.unpair.1)) :=
    Nat.Primrec.pair hfuel (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.left)
      (Nat.Primrec.left.comp Nat.Primrec.right))
  have hcons : Nat.Primrec (fun t => consistentBChar
      (t.unpair.1.unpair.2 + t.unpair.2.unpair.2 + 2) t.unpair.1.unpair.1 t.unpair.2.
    unpair.1) :=
    (primrec_consistentBChar2.comp hpack).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_selectFn hcond hcons (Nat.Primrec.const 1)).of_eq fun t => by
    simp only [ssysConsistentBChar]

theorem ssysConsistentBChar_eq_one_iff (n m : ℕ) :
    ssysConsistentBChar n m = 1 ↔ ssysConsistentB n m = true := by
  unfold ssysConsistentBChar ssysConsistentB
  by_cases hn : ssysActive n = true
  · by_cases hm : ssysActive m = true
    · have hnc : ssysActiveChar n = 1 := (ssysActiveChar_eq_one_iff n).mpr hn
      have hmc : ssysActiveChar m = 1 := (ssysActiveChar_eq_one_iff m).mpr hm
      obtain ⟨a, hdecn⟩ := ssysActive_exists_decode hn
      obtain ⟨b, hdecm⟩ := ssysActive_exists_decode hm
      have hac : n.unpair.1 = SExpr.encode a := decodeFuel_sound hdecn
      have hbc : m.unpair.1 = SExpr.encode b := decodeFuel_sound hdecm
      have hda : sexprDepth a ≤ n.unpair.2 + 1 := decodeFuel_depth_le hdecn
      have hdb : sexprDepth b ≤ m.unpair.2 + 1 := decodeFuel_depth_le hdecm
      have hfuel : Domain.Recursive.c9b5_sexprDepth (.cap a b) ≤ n.unpair.2 + m.unpair
    .2 + 2 := by
        rw [c9b5_sexprDepth_eq]
        simp only [sexprDepth]
        omega
      simp only [hnc, hmc, mulBit, mul_one, selectFn_one]
      rw [hac, hbc, ← c9b5_sexprGodelEncode_eq, ← c9b5_sexprGodelEncode_eq,
        consistentBChar_eq_one_iff hfuel]
      have hnotinactive : (!ssysActive n || !ssysActive m) = false := by simp [hn, hm]
      rw [hnotinactive, ite_eq_right (by decide)]
      simp only [hdecn, hdecm]
      exact (consistentB_iff a b).symm
    · have hm0 : ssysActiveChar m = 0 := by
        have hle := ssysActiveChar_le_one m
        have hne : ssysActiveChar m = 1 := fun h => hm ((ssysActiveChar_eq_one_iff m).
    mp h)
        omega
```

```
      have hmf : ssysActive m = false := by
        cases hAm : ssysActive m with
        | false => rfl
        | true => exact absurd hAm hm
      simp only [hm0, mulBit, mul_zero, selectFn_zero, hmf, Bool.not_false, Bool.
    or_true, ite_true]
  · have hn0 : ssysActiveChar n = 0 := by
      have hle := ssysActiveChar_le_one n
      have hne : ssysActiveChar n = 1 := fun h => hn ((ssysActiveChar_eq_one_iff n).mp
     h)
      omega
    have hnf : ssysActive n = false := by
      cases hAn : ssysActive n with
      | false => rfl
      | true => exact absurd hAn hn
    simp only [hn0, mulBit, zero_mul, selectFn_zero, hnf, Bool.not_false, Bool.true_or
    , ite_true]

theorem ssysConsistentBChar_le_one (n m : ℕ) : ssysConsistentBChar n m ≤ 1 := by
  unfold ssysConsistentBChar
  rcases (show mulBit (ssysActiveChar n) (ssysActiveChar m) = 0 ∨
      mulBit (ssysActiveChar n) (ssysActiveChar m) = 1 by
    have h1 := ssysActiveChar_le_one n
    have h2 := ssysActiveChar_le_one m
    unfold mulBit
    rcases (show ssysActiveChar n = 0 ∨ ssysActiveChar n = 1 by omega) with hn | hn
    <;>
      rcases (show ssysActiveChar m = 0 ∨ ssysActiveChar m = 1 by omega) with hm | hm
    <;>
      simp [hn, hm]) with h0 | h1
  · rw [h0, selectFn_zero]
  · rw [h1, selectFn_one]; exact consistentBChar_le_one _ _ _

theorem ssysConsChar_eq_ssysConsistentBChar (t : ℕ) :
    ssysConsChar t = ssysConsistentBChar t.unpair.1 t.unpair.2 := by
  apply eq_of_le_one_iff_one (ssysConsChar_le_one t) (ssysConsistentBChar_le_one _ _)
  rw [ssysConsChar_eq_one_iff, ssysConsistentBChar_eq_one_iff]

theorem primrec_ssysConsChar : Nat.Primrec ssysConsChar :=
  (primrec_ssysConsistentBChar).of_eq fun t => (ssysConsChar_eq_ssysConsistentBChar t)
    .symm

def ssysInterEqChar (t : ℕ) : ℕ :=
  let n := t.unpair.1
  let m := t.unpair.2.unpair.1
  let k := t.unpair.2.unpair.2
  Domain.Recursive.interEqChar (n.unpair.2 + m.unpair.2 + k.unpair.2 + 3)
    (Domain.Recursive.capCode (ssysCanonicalCode n) (ssysCanonicalCode m)) (
    ssysCanonicalCode k)

theorem ssys_cons_char_eq_one_iff (n m : ℕ) :
    (∃ k, SsysX k ⊆ SsysX n ∩ SsysX m) ↔ ssysConsChar (Nat.pair n m) = 1 := by
  rw [ssysConsChar_eq_one_iff, ssys_cons_char_iff, unpair_pair_fst, unpair_pair_snd]

theorem Ssys_cons_computable_of_primrec_ssysConsChar (hf : Nat.Primrec ssysConsChar) :
    RecDecidable₂ (fun n m => ∃ k, SsysX k ⊆ SsysX n ∩ SsysX m) :=
```

```
  RecDecidable₂.of_paired_zero_one_char hf ssysConsChar_zero_one
    ssys_cons_char_eq_one_iff

theorem Ssys_cons_computable : RecDecidable₂ (fun n m => ∃ k, SsysX k ⊆ SsysX n ∩
    SsysX m) :=
  Ssys_cons_computable_of_primrec_ssysConsChar primrec_ssysConsChar

def SsysPres : ConsistencyPresentation Ssys where
  X := SsysX
  mem_X n := Ssys_mem.mpr (SsysX_mem n)
  surj hY := SsysX_surj (Ssys_mem.mp hY)
  cons_computable := Ssys_cons_computable

theorem c9b5_boolNat_eq (b : Bool) : Domain.Recursive.c9b5_boolNat b = boolNat b := by
  cases b <;> rfl

theorem ssysCanonicalCode_eq (n : ℕ) : ssysCanonicalCode n = SExpr.encode (
    safeDecodeActive n) := by
  unfold ssysCanonicalCode
  by_cases hn : ssysActive n = true
  · rw [(ssysActiveChar_eq_one_iff n).mpr hn, Domain.Recursive.selectFn_one]
    obtain ⟨e, hdec⟩ := ssysActive_exists_decode hn
    have hac : n.unpair.1 = SExpr.encode e := decodeFuel_sound hdec
    rw [hac]
    congr 1
    unfold safeDecodeActive
    rw [hdec]
    have : decideNonemptyB e = true := by
      unfold ssysActive at hn; rw [hdec] at hn; exact hn
    simp [this]
  · have hn0 : ssysActiveChar n = 0 := by
      have hle := ssysActiveChar_le_one n
      have hne : ssysActiveChar n = 1 := fun h => hn ((ssysActiveChar_eq_one_iff n).mp
     h)
      omega
    have hnf : ssysActive n = false := by
      cases hAn : ssysActive n with
      | false => rfl
      | true => exact absurd hAn hn
    rw [hn0, Domain.Recursive.selectFn_zero, safeDecodeActive_inactive n hnf]

theorem sexprDepth_safeDecodeActive_le (n : ℕ) : sexprDepth (safeDecodeActive n) ≤ n.
    unpair.2 + 1 := by
  by_cases hn : ssysActive n = true
  · obtain ⟨e, hdec⟩ := ssysActive_exists_decode hn
    have hda : sexprDepth e ≤ n.unpair.2 + 1 := decodeFuel_depth_le hdec
    have heq : safeDecodeActive n = e := by
      unfold safeDecodeActive
      rw [hdec]
      have : decideNonemptyB e = true := by
        unfold ssysActive at hn; rw [hdec] at hn; exact hn
      simp [this]
    rw [heq]; exact hda
  · have hnf : ssysActive n = false := by
      cases hAn : ssysActive n with
      | false => rfl
```

```
      | true => exact absurd hAn hn
    rw [safeDecodeActive_inactive n hnf]
    simp [sexprDepth]

theorem ssys_interEq_iff (n m k : ℕ) :
    SsysX n ∩ SsysX m = SsysX k ↔
      denote (.cap (safeDecodeActive n) (safeDecodeActive m)) = denote (
    safeDecodeActive k) := by
  rw [SsysX_eq_denote_safe, SsysX_eq_denote_safe, SsysX_eq_denote_safe, denote_cap]

theorem ssysInterEqChar_le_one (t : ℕ) : ssysInterEqChar t ≤ 1 := by
  unfold ssysInterEqChar
  exact Domain.Recursive.interEqChar_le_one _ _ _

theorem ssysInterEqChar_eq_one_iff (n m k : ℕ) :
    ssysInterEqChar (Nat.pair n (Nat.pair m k)) = 1 ↔ SsysX n ∩ SsysX m = SsysX k :=
    by
  unfold ssysInterEqChar
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [ssys_interEq_iff]
  have hfuel : n.unpair.2 + m.unpair.2 + k.unpair.2 + 3 ≥
      c9b5_sexprDepth (.cap (safeDecodeActive n) (safeDecodeActive m)) ∧
      n.unpair.2 + m.unpair.2 + k.unpair.2 + 3 ≥ c9b5_sexprDepth (safeDecodeActive k)
    := by
    rw [c9b5_sexprDepth_eq, c9b5_sexprDepth_eq]
    have hn := sexprDepth_safeDecodeActive_le n
    have hm := sexprDepth_safeDecodeActive_le m
    have hk := sexprDepth_safeDecodeActive_le k
    simp only [sexprDepth]
    omega
  rw [show Domain.Recursive.capCode (ssysCanonicalCode n) (ssysCanonicalCode m) =
      c9b5_sexprGodelEncode (.cap (safeDecodeActive n) (safeDecodeActive m)) from ?_,
    show ssysCanonicalCode k = c9b5_sexprGodelEncode (safeDecodeActive k) from ?_,
    Domain.Recursive.interEqChar_eq_one_iff hfuel.1 hfuel.2]
  · rw [c9b5_sexprGodelEncode_eq]; exact ssysCanonicalCode_eq k
  · unfold Domain.Recursive.capCode
    rw [ssysCanonicalCode_eq, ssysCanonicalCode_eq, ← c9b5_sexprGodelEncode_eq,
      ← c9b5_sexprGodelEncode_eq]
    rfl

theorem primrec_ssysInterEqChar : Nat.Primrec ssysInterEqChar := by
  have hn : Nat.Primrec (fun t : ℕ => t.unpair.1) := Nat.Primrec.left
  have hm : Nat.Primrec (fun t : ℕ => t.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hk : Nat.Primrec (fun t : ℕ => t.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hcanon : Nat.Primrec (fun n : ℕ => ssysCanonicalCode n) := by
    have := primrec_ssysActiveChar
    refine (Domain.Recursive.primrec_selectFn this Nat.Primrec.left
      (Nat.Primrec.const (SExpr.encode .sigma))).of_eq fun n => ?_
    unfold ssysCanonicalCode; rfl
  have hcn : Nat.Primrec (fun t : ℕ => ssysCanonicalCode t.unpair.1) := hcanon.comp hn
  have hcm : Nat.Primrec (fun t : ℕ => ssysCanonicalCode t.unpair.2.unpair.1) :=
    hcanon.comp hm
  have hck : Nat.Primrec (fun t : ℕ => ssysCanonicalCode t.unpair.2.unpair.2) :=
    hcanon.comp hk
```

```
  have hfuel : Nat.Primrec (fun t : ℕ =>
      t.unpair.1.unpair.2 + t.unpair.2.unpair.1.unpair.2 + t.unpair.2.unpair.2.unpair
    .2 + 3) :=
    Domain.Recursive.primrec_add₂
      (Domain.Recursive.primrec_add₂
        (Domain.Recursive.primrec_add₂
          (Nat.Primrec.right.comp hn) (Nat.Primrec.right.comp hm))
        (Nat.Primrec.right.comp hk))
      (Nat.Primrec.const 3)
  have hcap : Nat.Primrec (fun t : ℕ =>
      Domain.Recursive.capCode (ssysCanonicalCode t.unpair.1)
        (ssysCanonicalCode t.unpair.2.unpair.1)) := by
    have hp : Nat.Primrec (fun t : ℕ => Nat.pair 3 (Nat.pair (ssysCanonicalCode t.
    unpair.1)
        (ssysCanonicalCode t.unpair.2.unpair.1))) :=
      Nat.Primrec.pair (Nat.Primrec.const 3) (Nat.Primrec.pair hcn hcm)
    refine hp.of_eq fun t => ?_
    unfold Domain.Recursive.capCode; rfl
  have hpack : Nat.Primrec (fun t : ℕ => Nat.pair
      (t.unpair.1.unpair.2 + t.unpair.2.unpair.1.unpair.2 + t.unpair.2.unpair.2.unpair
    .2 + 3)
      (Nat.pair (Domain.Recursive.capCode (ssysCanonicalCode t.unpair.1)
          (ssysCanonicalCode t.unpair.2.unpair.1))
        (ssysCanonicalCode t.unpair.2.unpair.2))) :=
    Nat.Primrec.pair hfuel (Nat.Primrec.pair hcap hck)
  refine (Domain.Recursive.primrec_interEqChar.comp hpack).of_eq fun t => ?_
  unfold ssysInterEqChar
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem Ssys_interEq_computable :
    RecDecidable₃ (fun n m k => SsysX n ∩ SsysX m = SsysX k) :=
  RecDecidable₃.of_triple_zero_one_char primrec_ssysInterEqChar
    (fun t => by
      rcases (show ssysInterEqChar t = 0 ∨ ssysInterEqChar t = 1 from by
        have := ssysInterEqChar_le_one t; omega) with h | h <;> simp [h])
    (fun n m k => (ssysInterEqChar_eq_one_iff n m k).symm)

theorem Ssys_partially_effectively_given : PartiallyEffectivelyGiven Ssys := ⟨⟨SsysPres
    ⟩⟩

end RegularLanguage
end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems.RegularLanguage
open NeighborhoodSystem Domain.Recursive ApproximableMap

/-- Intersection equality for SsysX is recursively decidable. -/
theorem example_722_presentation :
    RecDecidable₃ (fun n m k => SsysX n ∩ SsysX m = SsysX k) :=
  Ssys_interEq_computable

end DomainTheory.NeighbourhoodSystems.RegularLanguage
```

4.8.55 Exercise 7.22 — domain of formal languages

(For algebraists.) Let $\Sigma = \{0,1\}^*$ be the free semigroup. A new domain is constructed by defining a family of sets by the least fixed point theorem as follows:

$$S = \{\Sigma\} \cup \{\{\sigma\} \mid \sigma \in \Sigma\} \cup \{XY \mid X, Y \in S\} \cup \{X \cap Y \mid X, Y \in S \text{ and } X \cap Y \neq \emptyset\}.$$

Here we write: $XY = \{\sigma\tau \mid \sigma \in X \text{ and } \tau \in Y\}$.

Prove that S is an effectively given, positive neighbourhood system. (Hint: The sets in S are each "regular events" in the terminology of automata theory, and we have a decision method for the set algebra of regular events.)

Define multiplication on $|S|$ by

$$xy = \{Z \in S \mid \exists X \in x\, \exists Y \in y.\, XY \subseteq Z\},$$

and show $|S|$ becomes a semigroup with Σ embedded into $|S|$ by the homomorphism $\sigma \mapsto \{X \in S \mid \sigma \in X\}$.

Investigate some *infinite words* in S, say those defined by least fixed points such as: $\vec{\sigma} = \sigma\vec{\sigma}$ and $\vec{\sigma} = \vec{\sigma}\sigma$. Are these equations true:

$$\vec{\sigma}\vec{\sigma} = \vec{\sigma}, \quad \vec{\sigma}\vec{\sigma}\vec{\sigma} = \vec{\sigma}, \quad \vec{\sigma}\vec{1}\vec{\sigma}\vec{1} = \vec{\sigma}\vec{1},$$

and $\vec{01}\,\vec{01}\,\vec{01}\,\vec{01} = \vec{01}\,\vec{01}$?

Exercise_7_22.lean

@import "Exercise_7_22.lean"

The development is with the regular languages, regular-language codes, and language presentation peers.

Theorem (`example_722`). Concatenation of words is the product of their embeddings, the product of languages is associative, and a stream arrow is idempotent.

Proof. By the embedding of words, the associativity of the language product, and the idempotence of a stream arrow. ■

Exercise_7_22.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .regularLanguage

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.RegularLanguage
/-- Scott's Exercise 7.22: embedding, associativity, and stream-arrow idempotence. -/
theorem example_722 :
    (∀ (σ τ : List Bool), emb (σ ++ τ) = mulElem (emb σ) (emb τ)) ∧
    (∀ (x y z : Ssys.Element), mulElem (mulElem x y) z = mulElem x (mulElem y z)) ∧
    (∀ (σ : List Bool), mulElem (streamArrow σ) (streamArrow σ) = streamArrow σ) :=
  ⟨@emb_mul, ⟨@mulElem_assoc, @streamArrow_mul_self⟩⟩
```

4.8.56 Graph map

The graph of a computable endomap of the powerset is a computable element of the function space.

Definition (`capMap`). The map cap map relates basic source and target neighbourhoods by the mathematical operation expressed in its defining relation.

Definition (`halfIter`). Repeated halving applies integer division by two the specified number of times.

Theorem (`halfIter_zero`). At input zero, half iter has the initial value specified by its definition.

Proof. The relevant clause of the recursive definition of half iter zero gives this equation immediately. ■

Definition (`bitAt`). The bit at position k of n is the remainder modulo two after halving n exactly k times.

Definition (`cupMap`). The map cup map relates basic source and target neighbourhoods by the mathematical operation expressed in its defining relation.

Definition (`sumSet`). The set sum set consists exactly of the elements admitted by its defining membership condition.

Theorem (`compl_inter_nbhd`). The construction compl inter nbhd preserves the specified intersection of sets or neighbourhoods.

Proof. Use the same witnesses in both directions for compl inter nbhd, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`compl_subset_compl_of_subset`). The hypotheses subset imply compl subset compl.

Proof. Following the defining recursion for compl subset compl of subset and applying the preceding component lemmas yields the claimed result. ■

Definition (`elemSet`). The set elem set consists exactly of the elements admitted by its defining membership condition.

Theorem (`mem_elemSet`). The element described by elem set belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for elem set is supplied directly by the defining membership condition. ■

Definition (`bitsCode`). The coding function bits code assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`bitsList`). The bits list is the finite list extracted from, or encoded by, the given numerical data.

Definition (`bitsStep`). The state transformer bits step performs one stage of the finite numerical computation encoded in its state.

Definition (`gMap`). The map g map relates basic source and target neighbourhoods by the mathematical operation expressed in its defining relation.

Definition (`funMap`). The map fun map relates basic source and target neighbourhoods by the mathematical operation expressed in its defining relation.

Definition (`nextCellStep`). The state transformer next cell step performs one stage of the finite numerical computation encoded in its state.

Definition (`orUpTo`). Starting from one number, successively insert the bits of the other through the specified bound.

Definition (`plusIdx`). The sum-set neighbourhood index is obtained by the bounded bitwise accumulation determined by the two input indices.

Definition (`plusStep`). The state transformer plus step performs one stage of the finite numerical computation encoded in its state.

Theorem (`sumSet_mono`). Enlarging the input sets in sum set mono can only enlarge its resulting set or element.

Proof. Transport every witness for sum set mono along the assumed inclusions; the same witness then belongs to the enlarged result. ■

Definition (`plusMap`). The map plus map relates basic source and target neighbourhoods by the mathematical operation expressed in its defining relation.

Definition (`tagStep`). The state transformer tag step performs one stage of the finite numerical computation encoded in its state.

Definition (`tagCode`). The coding function tag code assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`tagOfBits`). The set bits of an index are decoded as a finite list and nested as tags around a terminal value.

Definition (`GraphIdx`). The construction graph idx is obtained by recursive clauses on its finite inputs.

Theorem (`GraphIdx_mono`). Enlarging the input sets in graph idx mono can only enlarge its resulting set or element.

Proof. Transport every witness for graph idx mono along the assumed inclusions; the same witness then belongs to the enlarged result. ■

Definition (`graphMap`). The map graph map relates basic source and target neighbourhoods by the mathematical operation expressed in its defining relation.

Definition (`unnumPair`). Iterating the successor-cell operation reconstructs the pair encoded by the inverse numerical enumeration.

Definition (`unnum1`). The first inverse-enumeration coordinate is the first component of the reconstructed pair.

Definition (`unnum2`). The second inverse-enumeration coordinate is the second component of the reconstructed pair.

Definition (`untagRef`). A tagged natural number is decoded recursively into its finite list of tags and terminal value.

Definition (`untagStep`). The state transformer untag step performs one stage of the finite numerical computation encoded in its state.

Definition (`untagState`). The bounded tag-decoding state is obtained by iterating the tag-removal transition sufficiently many times.

Definition (`untagList`). The untag list is the finite list extracted from, or encoded by, the given numerical data.

Definition (`untagVal`). The terminal value of a tagged code is extracted from the completed decoding state.

Theorem (`lt_of_testBit_true`). The hypotheses test bit true imply lt.

Proof. Following the defining recursion for lt of test bit true and applying the preceding component lemmas yields the claimed result. ■

Theorem (`halfIter_succ`). At a successor input, half iter succ satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of half iter succ gives this equation immediately. ■

Theorem (`primrec_halfIter`). The numerical function half iter is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of half iter. ■

Theorem (`primrec_bitAt`). The numerical function bit at is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of bit at. ■

Theorem (`bitAt_le_one`). For every input, bit at takes one of the two values zero and one.

Proof. Each Boolean-valued component of bit at le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`bitAt_eq_one_iff`). The zero–one test bit at returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in bit at eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`compl_nbhd`). For the objects in compl nbhd, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for compl nbhd and applying the preceding component lemmas yields the claimed result. ■

Theorem (`exists_combined_witness`). For the objects in combined witness, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for combined witness and applying the preceding component lemmas yields the claimed result. ■

Theorem (`mem_bitsList`). The element described by bits list belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for bits list is supplied directly by the defining membership condition. ■

Theorem (`nbhd_mem_iff_subset_elemSet`). Membership in nbhd mem iff subset elem set is equivalent to the explicit witness condition given by the construction.

Proof. Expanding the membership definition of nbhd mem iff subset elem set exposes exactly the witnesses and equations occurring on the other side. ■

Theorem (`elemSet_re_of_isComputableElement`). The map or element elem set re of is computable element is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for elem set re of is computable element to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`mem_decodeList_bitsCode`). The element described by list bits code belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for list bits code is supplied directly by the defining membership condition. ■

Theorem (`primrec_bitsStep`). The numerical function bits step is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of bits step. ■

Theorem (`bitsStep_unpair11`). For the objects in bits step unpair11, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for bits step unpair11 and applying the preceding component lemmas yields the claimed result. ■

Theorem (`bitsStep_unpair21`). For the objects in bits step unpair21, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for bits step unpair21 and applying the preceding component lemmas yields the claimed result. ■

Theorem (`bitsStep_unpair22`). For the objects in bits step unpair22, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for bits step unpair22 and applying the preceding component lemmas yields the claimed result. ■

Theorem (`bitsStep_iter_spec`). The numerical construction bits step iter spec computes exactly the finite object described by its specification.

Proof. Induction over the bounded iteration for bits step iter spec maintains the recorded accumulator invariant and yields the claimed final components. ■

Theorem (`primrec_bitsCode`). The numerical function bits code is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of bits code. ■

Theorem (`isComputableElement_of_elemSet_re`). The relation element of elem set re is recursively decidable.

Proof. Reduce the relation for element of elem set re to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`isComputableElement_iff_elemSet_re`). The relation element iff elem set re is recursively decidable.

Proof. Reduce the relation for element iff elem set re to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`unnum_snd_lt_of_fst_succ`). At a successor input, unnum snd lt of fst succ satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of unnum snd lt of fst succ gives this equation immediately. ■

Theorem (`bitAt_swap_isComputable`). The map or element bit at swap is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for bit at swap is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`capMap_rel_iff`). The condition cap map rel holds exactly when the finite witness condition holds.

Proof. Expand both conditions in cap map rel iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`capMap_isComputable`). The map or element cap map is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for cap map is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`cupMap_rel_iff`). The condition cup map rel holds exactly when the finite witness condition holds.

Proof. Expand both conditions in cup map rel iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`cupMap_isComputable`). The map or element cup map is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for cup map is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`decodeList_injective`). The construction list injective sends equal outputs only from equal inputs.

Proof. Equality of two values of list injective determines their outer form and then each component, so the original inputs coincide. ■

Theorem (`forall_rel_iff_subset_step`). The condition forall rel holds exactly when subset step holds.

Proof. Expand both conditions in rel iff subset step; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`gMap_rel_iff`). The condition g map rel holds exactly when the finite witness condition holds.

Proof. Expand both conditions in g map rel iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`graphMap_rel_iff`). The condition graph map rel holds exactly when the finite witness condition holds.

Proof. Expand both conditions in graph map rel iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`mem_bitsCodeList`). The element described by bits code list belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for bits code list is supplied directly by the defining membership condition. ■

Theorem (`mem_compl_nbhd`). The element described by compl nbhd belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for compl nbhd is supplied directly by the defining membership condition. ■

Theorem (`compl_nbhd_subset_Fun_iff`). The condition compl nbhd subset fun holds exactly when the finite witness condition holds.

Proof. Expand both conditions in compl nbhd subset fun iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`compl_nbhd_subset_iff`). The inclusion in compl nbhd subset iff holds exactly when the finite prefix, complement, or numerical condition on the right holds.

Proof. Translate both inclusions in compl nbhd subset iff into the relevant finite prefix or complement condition, and use antisymmetry for the converse direction. ■

Theorem (`mem_sumSet`). The element described by sum set belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for sum set is supplied directly by the defining membership condition. ■

Theorem (`nbhd_subset_iff_compl_subset_compl`). The inclusion in nbhd subset iff compl subset compl holds exactly when the finite prefix, complement, or numerical condition on the right holds.

Proof. Translate both inclusions in nbhd subset iff compl subset compl into the relevant finite prefix or complement condition, and use antisymmetry for the converse direction. ■

Theorem (`nbhd_subset_iff_myLor_eq`). The inclusion in nbhd subset iff my lor eq holds exactly when the finite prefix, complement, or numerical condition on the right holds.

Proof. Translate both inclusions in nbhd subset iff my lor eq into the relevant finite prefix or complement condition, and use antisymmetry for the converse direction. ■

Theorem (`nextCellStep_eq`). The value of next cell step eq equals the value described by next cell step eq.

Proof. Compare membership or numerical components in next cell step eq; the defining equations reduce both sides to the same data. ■

Theorem (`plusStep_unpair11`). For the objects in plus step unpair11, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for plus step unpair11 and applying the preceding component lemmas yields the claimed result. ■

Theorem (`plusStep_unpair12`). For the objects in plus step unpair12, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for plus step unpair12 and applying the preceding component lemmas yields the claimed result. ■

Theorem (`plusStep_unpair21`). For the objects in plus step unpair21, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for plus step unpair21 and applying the preceding component lemmas yields the claimed result. ■

Theorem (`plusStep_unpair22`). For the objects in plus step unpair22, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for plus step unpair22 and applying the preceding component lemmas yields the claimed result. ■

Theorem (`plusStep_iter_spec`). The numerical construction plus step iter spec computes exactly the finite object described by its specification.

Proof. Induction over the bounded iteration for plus step iter spec maintains the recorded accumulator invariant and yields the claimed final components. ■

Theorem (`primrec_nextCellStep`). The numerical function next cell step is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of next cell step. ■

Theorem (`primrec_plusStep`). The numerical function plus step is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of plus step. ■

Theorem (`primrec_plusIdx`). The numerical function plus idx is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of plus idx. ■

Theorem (`primrec_tri`). The numerical function tri is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of tri. ■

Theorem (`primrec_num`). The numerical function num is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of num. ■

Theorem (`primrec_tagStep`). The numerical function tag step is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of tag step. ■

Theorem (`primrec_tagCode`). The numerical function tag code is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of tag code. ■

Theorem (`primrec_tagOfBits`). The numerical function tag of bits is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of tag of bits. ■

Theorem (`primrec_unnumPair`). The numerical function unnum pair is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of unnum pair. ■

Theorem (`primrec_unnum1`). The numerical function unnum1 is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of unnum1. ■

Theorem (`primrec_unnum2`). The numerical function unnum2 is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of unnum2. ■

Theorem (`primrec_untagStep`). The numerical function untag step is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of untag step. ■

Theorem (`primrec_untagState`). The numerical function untag state is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of untag state. ■

Theorem (`primrec_untagList`). The numerical function untag list is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of untag list. ■

Theorem (`primrec_untagVal`). The numerical function untag val is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of untag val. ■

Theorem (`tag_eq_foldr`). The value of tag equals the value described by foldr.

Proof. Compare membership or numerical components in tag eq foldr; the defining equations reduce both sides to the same data. ■

Theorem (`tag_eq_foldl_reverse`). The value of tag equals the value described by foldl reverse.

Proof. Compare membership or numerical components in tag eq foldl reverse; the defining equations reduce both sides to the same data. ■

Theorem (`tagCode_spec`). The numerical construction tag code spec computes exactly the finite object described by its specification.

Proof. Induction over the bounded iteration for tag code spec maintains the recorded accumulator invariant and yields the claimed final components. ■

Theorem (`tagOfBits_spec`). The numerical construction tag of bits spec computes exactly the finite object described by its specification.

Proof. Induction over the bounded iteration for tag of bits spec maintains the recorded accumulator invariant and yields the claimed final components. ■

Theorem (`testBit_orUpTo`). For the objects in test bit or up to, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for test bit or up to and applying the preceding component lemmas yields the claimed result. ■

Theorem (`testBit_orUpTo_self`). For the objects in test bit or up to self, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for test bit or up to self and applying the preceding component lemmas yields the claimed result. ■

Theorem (`plusIdx_testBit`). For the objects in plus idx test bit, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for plus idx test bit and applying the preceding component lemmas yields the claimed result. ■

Theorem (`compl_nbhd_plusIdx`). For the objects in compl nbhd plus idx, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for compl nbhd plus idx and applying the preceding component lemmas yields the claimed result. ■

Theorem (`plusMap_rel_iff`). The condition plus map rel holds exactly when the finite witness condition holds.

Proof. Expand both conditions in plus map rel iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`plusMap_isComputable`). The map or element plus map is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for plus map is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`unnumPair_spec`). The numerical construction unnum pair spec computes exactly the finite object described by its specification.

Proof. Induction over the bounded iteration for unnum pair spec maintains the recorded accumulator invariant and yields the claimed final components. ■

Theorem (`unnum1_spec`). The numerical construction unnum1 spec computes exactly the finite object described by its specification.

Proof. Induction over the bounded iteration for unnum1 spec maintains the recorded accumulator invariant and yields the claimed final components. ■

Theorem (`unnum2_spec`). The numerical construction unnum2 spec computes exactly the finite object described by its specification.

Proof. Induction over the bounded iteration for unnum2 spec maintains the recorded accumulator invariant and yields the claimed final components. ■

Theorem (`untagList_bitAt_isComputable`). The map or element untag list bit at is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for untag list bit at is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`untagRef_succ_case`). At a successor input, untag ref succ case satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of untag ref succ case gives this equation immediately. ■

Theorem (`untagRef_zero_case`). For the objects in untag ref zero case, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for untag ref zero case and applying the preceding component lemmas yields the claimed result. ■

Theorem (`tag_untagRef`). For the objects in tag untag ref, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for tag untag ref and applying the preceding component lemmas yields the claimed result. ■

Theorem (`untagStep_cont`). For the objects in untag step cont, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for untag step cont and applying the preceding component lemmas yields the claimed result. ■

Theorem (`untagStep_done`). At input one, untag step has the value specified by its definition.

Proof. Following the defining recursion for untag step done and applying the preceding component lemmas yields the claimed result. ■

Theorem (`untagStep_halt`). For the objects in untag step halt, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for untag step halt and applying the preceding component lemmas yields the claimed result. ■

Theorem (`untagStep_of_done_iterate`). The hypotheses done iterate imply untag step.

Proof. Following the defining recursion for untag step of done iterate and applying the preceding component lemmas yields the claimed result. ■

Theorem (`untagStep_iter_spec`). The numerical construction untag step iter spec computes exactly the finite object described by its specification.

Proof. Induction over the bounded iteration for untag step iter spec maintains the recorded accumulator invariant and yields the claimed final components. ■

Theorem (`untagState_spec`). The numerical construction untag state spec computes exactly the finite object described by its specification.

Proof. Induction over the bounded iteration for untag state spec maintains the recorded accumulator invariant and yields the claimed final components. ■

Theorem (`decodeList_untagList`). For the objects in list untag list, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for list untag list and applying the preceding component lemmas yields the claimed result. ■

Theorem (`untagVal_and_isComputable`). The map or element untag val and is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for untag val and is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`untagVal_spec`). The numerical construction untag val spec computes exactly the finite object described by its specification.

Proof. Induction over the bounded iteration for untag val spec maintains the recorded accumulator invariant and yields the claimed final components. ■

Theorem (`tag_untagList_untagVal`). For the objects in tag untag list untag val, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for tag untag list untag val and applying the preceding component lemmas yields the claimed result. ■

Theorem (`mem_Fun_compl_nbhd_iff`). Membership in fun compl nbhd iff is equivalent to the explicit witness condition given by the construction.

Proof. Expanding the membership definition of fun compl nbhd iff exposes exactly the witnesses and equations occurring on the other side. ■

Theorem (`mem_Fun_compl_nbhd_iff'`). Membership in fun compl nbhd iff' is equivalent to the explicit witness condition given by the construction.

Proof. Expanding the membership definition of fun compl nbhd iff' exposes exactly the witnesses and equations occurring on the other side. ■

Theorem (`mem_Fun_compl_nbhd_isComputable`). The map or element fun compl nbhd is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. The required witness for fun compl nbhd is computable is supplied directly by the defining membership condition. ■

Theorem (`mem_GraphIdx_iff`). Membership in graph idx iff is equivalent to the explicit witness condition given by the construction.

Proof. Expanding the membership definition of graph idx iff exposes exactly the witnesses and equations occurring on the other side. ■

Theorem (`graphIdx_isComputable`). The map or element graph idx is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for graph idx is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`graphMap_isComputable`). The map or element graph map is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for graph map is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`testBit_imp_mem_Fun_isComputable`). The map or element test bit imp mem fun is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for test bit imp mem fun is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`gMap_isComputable`). The map or element g map is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for g map is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`funMap_isComputable`). The map or element fun map is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for fun map is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

`graphFun.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.setGraph
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .powersetNbhd
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .funPresentation
import Mathlib.Data.Nat.Bitwise

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.GraphFun

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Domain.Recursive
```

```
open DomainTheory.NeighbourhoodSystems.NatPair (tri num numP unnum nextCell numP_unnum
    )

open DomainTheory.NeighbourhoodSystems.SetGraph (tag tag_nil tag_cons Fun Fun_mono
    entries
  num_succ_left_gt tag_injective)

def capMap : ApproximableMap (prod PN PN) PN where
  rel W Z := ∃ X Y, W = prodNbhd X Y ∧ PN.mem X ∧ PN.mem Y ∧ PN.mem Z ∧ X ⊆ Z ∧ Y ⊆ Z
  rel_dom := by rintro W Z ⟨X, Y, rfl, hX, hY, _, _, _⟩; exact prod_mem_prodNbhd hX hY
  rel_cod := by rintro W Z ⟨X, Y, rfl, _, _, hZ, _, _⟩; exact hZ
  master_rel := ⟨Set.univ, Set.univ, rfl, PN.master_mem, PN.master_mem, PN.master_mem,
    Set.subset_univ _, Set.subset_univ _⟩
  inter_right := by
    rintro W Z Z' ⟨X, Y, rfl, hX, hY, hZ, hXZ, hYZ⟩ ⟨X', Y', heq, _, _, hZ', hXZ', hYZ'⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    obtain ⟨k, hk⟩ := hZ; obtain ⟨k', hk'⟩ := hZ'; subst hk; subst hk'
    exact ⟨X, Y, rfl, hX, hY, ⟨myLor k k', nbhd_inter k k'⟩,
      Set.subset_inter hXZ hXZ', Set.subset_inter hYZ hYZ'⟩
  mono := by
    rintro W W' Z Z' ⟨X, Y, rfl, hX, hY, hZ, hXZ, hYZ⟩ hW'W hZZ' hW' hZ'
    obtain ⟨X', Y', hX', hY', rfl⟩ := hW'
    obtain ⟨hXX', hYY'⟩ := prodNbhd.subset_iff.mp hW'W
    exact ⟨X', Y', rfl, hX', hY', hZ', hXX'.trans (hXZ.trans hZZ'), hYY'.trans (hYZ.
    trans hZZ')⟩

end DomainTheory.NeighbourhoodSystems.GraphFun

namespace DomainTheory.NeighbourhoodSystems.GraphFun
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Domain.Recursive
open DomainTheory.NeighbourhoodSystems.NatPair (tri num numP unnum nextCell numP_unnum
    )
open DomainTheory.NeighbourhoodSystems.SetGraph (tag tag_nil tag_cons Fun Fun_mono
    entries
  num_succ_left_gt tag_injective)

def halfIter (n k : ℕ) : ℕ := (fun x : ℕ => x / 2)^[k] n

@[simp] theorem halfIter_zero (n : ℕ) : halfIter n 0 = n := rfl

def bitAt (n k : ℕ) : ℕ := halfIter n k % 2

def cupMap : ApproximableMap (prod PN PN) PN where
  rel W Z := ∃ X Y, W = prodNbhd X Y ∧ PN.mem X ∧ PN.mem Y ∧ PN.mem Z ∧ X ∩ Y ⊆ Z
  rel_dom := by rintro W Z ⟨X, Y, rfl, hX, hY, _, _⟩; exact prod_mem_prodNbhd hX hY
  rel_cod := by rintro W Z ⟨X, Y, rfl, _, _, hZ, _⟩; exact hZ
  master_rel := ⟨Set.univ, Set.univ, rfl, PN.master_mem, PN.master_mem, PN.master_mem,
    Set.inter_subset_left.trans (Set.subset_univ _)⟩
  inter_right := by
    rintro W Z Z' ⟨X, Y, rfl, hX, hY, hZ, hXYZ⟩ ⟨X', Y', heq, _, _, hZ', hXYZ'⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    obtain ⟨k, hk⟩ := hZ; obtain ⟨k', hk'⟩ := hZ'; subst hk; subst hk'
    exact ⟨X, Y, rfl, hX, hY, ⟨myLor k k', nbhd_inter k k'⟩, Set.subset_inter hXYZ hXYZ
    '⟩
  mono := by
    rintro W W' Z Z' ⟨X, Y, rfl, hX, hY, hZ, hXYZ⟩ hW'W hZZ' hW' hZ'
```

```
    obtain ⟨X', Y', hX', hY', rfl⟩ := hW'
    obtain ⟨hXX', hYY'⟩ := prodNbhd.subset_iff.mp hW'W
    exact ⟨X', Y', rfl, hX', hY', hZ',
      (Set.inter_subset_inter hXX' hYY').trans (hXYZ.trans hZZ')⟩

def sumSet (X Y : Set ℕ) : Set ℕ := {k | ∃ a ∈ X, ∃ b ∈ Y, a + b = k}

@[inherit_doc] infixl:65 " +ˢ " => sumSet

theorem compl_inter_nbhd (a b : ℕ) : (nbhd a ∩ nbhd b)? = (nbhd a)? ∪ (nbhd b)? := by
  ext x
  simp only [Set.mem_compl_iff, Set.mem_inter_iff, Set.mem_union, mem_nbhd]
  cases a.testBit x <;> cases b.testBit x <;> decide

theorem compl_subset_compl_of_subset {X Y : Set ℕ} (h : X ⊆ Y) : Y? ⊆ X? :=
  fun _ hx hxX => hx (h hxX)

def elemSet (x : PN.Element) : Set ℕ := {k | ∃ n, x.mem (nbhd n) ∧ n.testBit k = true}

@[simp] theorem mem_elemSet {x : PN.Element} {k : ℕ} :
    k ∈ elemSet x ↔ ∃ n, x.mem (nbhd n) ∧ n.testBit k = true := Iff.rfl

def bitsCode : ℕ → ℕ → ℕ
  | _, 0 => 0
  | n, N + 1 => selectFn (bitAt n N) (Nat.pair N (bitsCode n N) + 1) (bitsCode n N)

def bitsList (n : ℕ) : List ℕ := (List.range n).filter (fun k => n.testBit k)

def bitsStep (s : ℕ) : ℕ :=
  Nat.pair s.unpair.1
    (Nat.pair (s.unpair.2.unpair.1 + 1)
      (selectFn (bitAt s.unpair.1 s.unpair.2.unpair.1)
        (Nat.pair s.unpair.2.unpair.1 s.unpair.2.unpair.2 + 1) s.unpair.2.unpair.2))

def gMap : ApproximableMap (prod PN PN) PN where
  rel W Z := ∃ X Y, W = prodNbhd X Y ∧ PN.mem X ∧ PN.mem Y ∧ PN.mem Z ∧ Z? ⊆ Fun X? Y?
  rel_dom := by rintro W Z ⟨X, Y, rfl, hX, hY, _, _⟩; exact prod_mem_prodNbhd hX hY
  rel_cod := by rintro W Z ⟨X, Y, rfl, _, _, hZ, _⟩; exact hZ
  master_rel := ⟨Set.univ, Set.univ, rfl, PN.master_mem, PN.master_mem, PN.master_mem,
    fun x hx => absurd (Set.mem_univ x) hx⟩
  inter_right := by
    rintro W Z Z' ⟨X, Y, rfl, hX, hY, hZ, hsub⟩ ⟨X', Y', heq, _, _, hZ', hsub'⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    obtain ⟨k, hk⟩ := hZ; obtain ⟨k', hk'⟩ := hZ'; subst hk; subst hk'
    refine ⟨X, Y, rfl, hX, hY, ⟨myLor k k', nbhd_inter k k'⟩, ?_⟩
    rw [compl_inter_nbhd]
    exact Set.union_subset hsub hsub'
  mono := by
    rintro W W' Z Z' ⟨X, Y, rfl, hX, hY, hZ, hsub⟩ hW'W hZZ' hW' hZ'
    obtain ⟨X', Y', hX', hY', rfl⟩ := hW'
    obtain ⟨hXX', hYY'⟩ := prodNbhd.subset_iff.mp hW'W
    refine ⟨X', Y', rfl, hX', hY', hZ', ?_⟩
    calc Z'? ⊆ Z? := compl_subset_compl_of_subset hZZ'
      _ ⊆ Fun X? Y? := hsub
      _ ⊆ Fun X'? Y'? :=
        Fun_mono (compl_subset_compl_of_subset hXX') (compl_subset_compl_of_subset hYY
```

```
    ')

def funMap : ApproximableMap PN (funSpace PN PN) := curry gMap

def nextCellStep (w : ℕ) : ℕ :=
  selectFn (isZero w.unpair.1) (Nat.pair (w.unpair.2 + 1) 0)
    (Nat.pair (w.unpair.1 - 1) (w.unpair.2 + 1))

def orUpTo (n m : ℕ) : ℕ → ℕ
  | 0 => 0
  | k + 1 => myLor (orUpTo n m k) (selectFn (bitAt n k) (m <<< k) 0)

def plusIdx (n m : ℕ) : ℕ := orUpTo n m n

def plusStep (s : ℕ) : ℕ :=
  Nat.pair (Nat.pair s.unpair.1.unpair.1 s.unpair.1.unpair.2)
    (Nat.pair (s.unpair.2.unpair.1 + 1)
      (myLor s.unpair.2.unpair.2
        (selectFn (bitAt s.unpair.1.unpair.1 s.unpair.2.unpair.1)
          (s.unpair.1.unpair.2 <<< s.unpair.2.unpair.1) 0)))

theorem sumSet_mono {X X' Y Y' : Set ℕ} (hX : X ⊆ X') (hY : Y ⊆ Y') : X +ˢ Y ⊆ X' +ˢ Y
    ' := by
  rintro k ⟨a, ha, b, hb, rfl⟩
  exact ⟨a, hX ha, b, hY hb, rfl⟩

def plusMap : ApproximableMap (prod PN PN) PN where
  rel W Z := ∃ X Y, W = prodNbhd X Y ∧ PN.mem X ∧ PN.mem Y ∧ PN.mem Z ∧ Z? ⊆ X? +ˢ Y?
  rel_dom := by rintro W Z ⟨X, Y, rfl, hX, hY, _, _⟩; exact prod_mem_prodNbhd hX hY
  rel_cod := by rintro W Z ⟨X, Y, rfl, _, _, hZ, _⟩; exact hZ
  master_rel := ⟨Set.univ, Set.univ, rfl, PN.master_mem, PN.master_mem, PN.master_mem,
    fun x hx => absurd (Set.mem_univ x) hx⟩
  inter_right := by
    rintro W Z Z' ⟨X, Y, rfl, hX, hY, hZ, hsub⟩ ⟨X', Y', heq, _, _, hZ', hsub'⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    obtain ⟨k, hk⟩ := hZ; obtain ⟨k', hk'⟩ := hZ'; subst hk; subst hk'
    refine ⟨X, Y, rfl, hX, hY, ⟨myLor k k', nbhd_inter k k'⟩, ?_⟩
    rw [compl_inter_nbhd]
    exact Set.union_subset hsub hsub'
  mono := by
    rintro W W' Z Z' ⟨X, Y, rfl, hX, hY, hZ, hsub⟩ hW'W hZZ' hW' hZ'
    obtain ⟨X', Y', hX', hY', rfl⟩ := hW'
    obtain ⟨hXX', hYY'⟩ := prodNbhd.subset_iff.mp hW'W
    refine ⟨X', Y', rfl, hX', hY', hZ', ?_⟩
    calc Z'? ⊆ Z? := compl_subset_compl_of_subset hZZ'
      _ ⊆ X? +ˢ Y? := hsub
      _ ⊆ X'? +ˢ Y'? :=
        sumSet_mono (compl_subset_compl_of_subset hXX') (compl_subset_compl_of_subset
    hYY')

def tagStep (w : ℕ) : ℕ := num (w.unpair.1 + 1) w.unpair.2.unpair.1

def tagCode (code m : ℕ) : ℕ := foldCode tagStep 0 (num 0 m) (appendCode 0 code)

def tagOfBits (j m : ℕ) : ℕ := tagCode (bitsCode j j) m
```

```
def GraphIdx (W : Set (ApproximableMap PN PN)) : Set ℕ :=
  {c | ∃ n m₀ m, c = tagOfBits n m ∧ (∀ f ∈ W, f.rel (nbhd n) (nbhd m₀)) ∧ m₀.testBit m
     = true}

theorem GraphIdx_mono {W W' : Set (ApproximableMap PN PN)} (h : W' ⊆ W) :
    GraphIdx W ⊆ GraphIdx W' := by
  rintro c ⟨n, m₀, m, rfl, hf, hbit⟩
  exact ⟨n, m₀, m, rfl, fun f hf' => hf f (h hf'), hbit⟩

def graphMap : ApproximableMap (funSpace PN PN) PN where
  rel W Z := (funSpace PN PN).mem W ∧ PN.mem Z ∧ Z? ⊆ GraphIdx W
  rel_dom := fun ⟨h, _, _⟩ => h
  rel_cod := fun ⟨_, h, _⟩ => h
  master_rel := ⟨(funSpace PN PN).master_mem, PN.master_mem, by
    show (Set.univ : Set ℕ)? ⊆ GraphIdx Set.univ
    intro x hx
    exact absurd (Set.mem_univ x) hx⟩
  inter_right := by
    rintro W Z Z' ⟨hW, hZ, hsub⟩ ⟨_, hZ', hsub'⟩
    obtain ⟨k, hk⟩ := hZ; obtain ⟨k', hk'⟩ := hZ'; subst hk; subst hk'
    refine ⟨hW, ⟨myLor k k', nbhd_inter k k'⟩, ?_⟩
    rw [compl_inter_nbhd]
    exact Set.union_subset hsub hsub'
  mono := by
    rintro W W' Z Z' ⟨hW, hZ, hsub⟩ hW'W hZZ' hW' hZ'
    obtain ⟨k', rfl⟩ := hZ'
    refine ⟨hW', ⟨k', rfl⟩, ?_⟩
    calc (nbhd k')? ⊆ Z? := compl_subset_compl_of_subset hZZ'
      _ ⊆ GraphIdx W := hsub
      _ ⊆ GraphIdx W' := GraphIdx_mono hW'W

def unnumPair (v : ℕ) : ℕ := nextCellStep^[v] (Nat.pair 0 0)

def unnum1 (v : ℕ) : ℕ := (unnumPair v).unpair.1

def unnum2 (v : ℕ) : ℕ := (unnumPair v).unpair.2

def untagRef (c : ℕ) : List ℕ × ℕ :=
  if _h : (unnum c).1 = 0 then ([], (unnum c).2)
  else
    have hlt : (unnum c).2 < c := by
      have heq : num (unnum c).1 (unnum c).2 = c := by
        have h := numP_unnum c
        simpa [numP] using h
      obtain ⟨k, hk⟩ : ∃ k, (unnum c).1 = k + 1 := ⟨(unnum c).1 - 1, by omega⟩
      rw [hk] at heq
      have hb := num_succ_left_gt k (unnum c).2
      omega
    (((unnum c).1 - 1) :: (untagRef (unnum c).2).1, (untagRef (unnum c).2).2)
termination_by c
decreasing_by all_goals exact hlt

def untagStep (w : ℕ) : ℕ :=
  selectFn w.unpair.2.unpair.1 w
    (selectFn (isZero (unnum1 w.unpair.1))
      (Nat.pair (unnum2 w.unpair.1)
```

```
        (Nat.pair 1 (Nat.pair w.unpair.2.unpair.2.unpair.1 (unnum2 w.unpair.1))))
      (Nat.pair (unnum2 w.unpair.1)
        (Nat.pair 0
          (Nat.pair (Nat.pair (unnum1 w.unpair.1 - 1) w.unpair.2.unpair.2.unpair.1 +
    1)
            w.unpair.2.unpair.2.unpair.2))))

def untagState (c : ℕ) : ℕ := untagStep^[c + 1] (Nat.pair c (Nat.pair 0 (Nat.pair 0 0)
    ))

def untagList (c : ℕ) : ℕ := appendCode 0 (untagState c).unpair.2.unpair.2.unpair.1

def untagVal (c : ℕ) : ℕ := (untagState c).unpair.2.unpair.2.unpair.2

theorem lt_of_testBit_true {n i : ℕ} (h : n.testBit i = true) : i < n := by
  by_contra hle
  have hle' : n ≤ i := by omega
  have hlt : n < 2 ^ i := lt_of_le_of_lt hle' Nat.lt_two_pow_self
  rw [Nat.testBit_eq_false_of_lt hlt] at h
  exact absurd h (by decide)

theorem halfIter_succ (n k : ℕ) : halfIter n (k + 1) = halfIter (n / 2) k := by
  unfold halfIter; rw [Function.iterate_succ_apply]

theorem primrec_halfIter : Nat.Primrec (fun t => halfIter t.unpair.1 t.unpair.2) := by
  have hstep : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2 / 2) :=
    primrec_div2.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  refine (Nat.Primrec.prec primrec_id hstep).of_eq (fun t => ?_)
  simp only [Nat.unpaired, unpair_pair_snd]
  exact rec_const_iterate (fun x => x / 2) t.unpair.1 t.unpair.2

theorem primrec_bitAt : Nat.Primrec (fun t => bitAt t.unpair.1 t.unpair.2) :=
  primrec_mod2.comp primrec_halfIter

theorem bitAt_le_one (n k : ℕ) : bitAt n k ≤ 1 := by unfold bitAt; omega

theorem bitAt_eq_one_iff (n k : ℕ) : bitAt n k = 1 ↔ n.testBit k = true := by
  induction k generalizing n with
  | zero =>
    show n % 2 = 1 ↔ n.testBit 0 = true
    rw [Nat.testBit_zero, decide_eq_true_iff]
  | succ k ih =>
    show halfIter n (k + 1) % 2 = 1 ↔ n.testBit (k + 1) = true
    rw [halfIter_succ, Nat.testBit_add_one]
    exact ih (n / 2)

theorem compl_nbhd (n : ℕ) : (nbhd n)ᶜ = {k | n.testBit k = true} := by
  ext k; simp [nbhd]

theorem exists_combined_witness (x : PN.Element) :
    ∀ L : List ℕ, (∀ k ∈ L, ∃ m, x.mem (nbhd m) ∧ m.testBit k = true) →
      ∃ m₀, x.mem (nbhd m₀) ∧ ∀ k ∈ L, m₀.testBit k = true
  | [] => fun _ => ⟨0, nbhd_zero ▸ x.master_mem, by simp⟩
  | k :: L => by
    intro hL
    obtain ⟨mk, hmk, hbk⟩ := hL k List.mem_cons_self
```

```
    obtain ⟨m₀, hm₀, hall⟩ :=
      exists_combined_witness x L (fun j hj => hL j (List.mem_cons_of_mem k hj))
    refine ⟨myLor mk m₀, ?_, ?_⟩
    · rw [← nbhd_inter]; exact x.inter_mem hmk hm₀
    · intro j hj
      rw [myLor_eq_lor, Nat.testBit_lor, Bool.or_eq_true]
      rcases List.mem_cons.mp hj with rfl | hj
      · exact Or.inl hbk
      · exact Or.inr (hall j hj)

theorem mem_bitsList {n k : ℕ} : k ∈ bitsList n ↔ n.testBit k = true := by
  unfold bitsList
  rw [List.mem_filter, List.mem_range]
  exact ⟨fun h => h.2, fun h => ⟨lt_of_testBit_true h, h⟩⟩

theorem nbhd_mem_iff_subset_elemSet (x : PN.Element) (n : ℕ) :
    x.mem (nbhd n) ↔ (nbhd n)? ⊆ elemSet x := by
  rw [compl_nbhd]
  constructor
  · intro hn k hk
    exact ⟨n, hn, hk⟩
  · intro hsub
    obtain ⟨m₀, hm₀, hall⟩ :=
      exists_combined_witness x (bitsList n) (fun k hk => hsub (mem_bitsList.mp hk))
    have hincl : nbhd m₀ ⊆ nbhd n := by
      intro k hk
      simp only [mem_nbhd] at hk ⊢
      cases hn2 : n.testBit k with
      | false => rfl
      | true =>
        have hcontra := hall k (mem_bitsList.mpr hn2)
        rw [hk] at hcontra
        exact absurd hcontra (by decide)
    exact x.up_mem hm₀ ⟨n, rfl⟩ hincl

theorem elemSet_re_of_isComputableElement {x : PN.Element} (hx : ComputableElement
    PNpres x) :
    REPred (fun k => k ∈ elemSet x) := by
  have hx' : REPred (fun n => x.mem (nbhd n)) := hx.out
  have hbit : RecDecidable (fun t : ℕ => t.unpair.1.testBit t.unpair.2 = true) :=
    ⟨fun t => bitAt t.unpair.1 t.unpair.2, primrec_bitAt, fun t => (bitAt_eq_one_iff _
    _).symm⟩
  have hand : REPred (fun t : ℕ => x.mem (nbhd t.unpair.1) ∧ t.unpair.1.testBit t.
    unpair.2 = true) :=
    (hx'.comp Nat.Primrec.left).and hbit.re
  refine REPred.of_iff (fun k => ?_) hand.proj
  simp only [mem_elemSet, unpair_pair_fst, unpair_pair_snd]

theorem mem_decodeList_bitsCode (n : ℕ) :
    ∀ N k, k ∈ decodeList (bitsCode n N) ↔ k < N ∧ n.testBit k = true
  | 0, k => by simp [bitsCode, decodeList_zero]
  | N + 1, k => by
    show k ∈ decodeList (selectFn (bitAt n N) (Nat.pair N (bitsCode n N) + 1) (
    bitsCode n N)) ↔ _
    by_cases hbit : n.testBit N = true
    · rw [(bitAt_eq_one_iff n N).mpr hbit, selectFn_one, decodeList_succ,
```

```
    unpair_pair_fst,
        unpair_pair_snd, List.mem_cons, mem_decodeList_bitsCode n N k]
      constructor
      · rintro (rfl | ⟨hkN, hk⟩)
        · exact ⟨by omega, hbit⟩
        · exact ⟨by omega, hk⟩
      · rintro ⟨hkN1, hk⟩
        rcases (show k < N ∨ k = N by omega) with hkN | hkN
        · exact Or.inr ⟨hkN, hk⟩
        · exact Or.inl hkN
    · have hne1 : bitAt n N = 1 := fun h => hbit ((bitAt_eq_one_iff n N).mp h)
      have hle := bitAt_le_one n N
      have hnbit : bitAt n N = 0 := by omega
      rw [hnbit, selectFn_zero, mem_decodeList_bitsCode n N k]
      constructor
      · rintro ⟨hkN, hk⟩
        exact ⟨Nat.lt_succ_of_lt hkN, hk⟩
      · rintro ⟨hkN1, hk⟩
        rcases (show k < N ∨ k = N by omega) with hkN | hkN
        · exact ⟨hkN, hk⟩
        · subst hkN; exact absurd hk hbit

theorem primrec_bitsStep : Nat.Primrec bitsStep := by
  have hn : Nat.Primrec (fun s : ℕ => s.unpair.1) := Nat.Primrec.left
  have hN : Nat.Primrec (fun s : ℕ => s.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have hacc : Nat.Primrec (fun s : ℕ => s.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hbit : Nat.Primrec (fun s : ℕ => bitAt s.unpair.1 s.unpair.2.unpair.1) :=
    (primrec_bitAt.comp (hn.pair hN)).of_eq fun s => by simp only [unpair_pair_fst,
    unpair_pair_snd]
  have hcons : Nat.Primrec (fun s : ℕ => Nat.pair s.unpair.2.unpair.1 s.unpair.2.
    unpair.2 + 1) :=
    Nat.Primrec.succ.comp ((hN.pair hacc).of_eq fun _ => rfl)
  have hsel := primrec_selectFn hbit hcons hacc
  exact (hn.pair ((primrec_add₂ hN (Nat.Primrec.const 1)).pair hsel)).of_eq fun _ =>
    rfl

theorem bitsStep_unpair11 (s : ℕ) : (bitsStep s).unpair.1 = s.unpair.1 := by
  unfold bitsStep; rw [unpair_pair_fst]

theorem bitsStep_unpair21 (s : ℕ) : (bitsStep s).unpair.2.unpair.1 = s.unpair.2.unpair
    .1 + 1 := by
  unfold bitsStep; rw [unpair_pair_snd, unpair_pair_fst]

theorem bitsStep_unpair22 (s : ℕ) : (bitsStep s).unpair.2.unpair.2 =
    selectFn (bitAt s.unpair.1 s.unpair.2.unpair.1)
      (Nat.pair s.unpair.2.unpair.1 s.unpair.2.unpair.2 + 1) s.unpair.2.unpair.2 := by
  unfold bitsStep; rw [unpair_pair_snd, unpair_pair_snd]

theorem bitsStep_iter_spec (n : ℕ) (k : ℕ) :
    (bitsStep^[k] (Nat.pair n (Nat.pair 0 0))).unpair.1 = n ∧
    (bitsStep^[k] (Nat.pair n (Nat.pair 0 0))).unpair.2.unpair.1 = k ∧
    (bitsStep^[k] (Nat.pair n (Nat.pair 0 0))).unpair.2.unpair.2 = bitsCode n k := by
  induction k with
  | zero =>
```

```
    refine ⟨?_, ?_, ?_⟩ <;>
      simp only [Function.iterate_zero, id_eq, unpair_pair_fst, unpair_pair_snd,
    bitsCode]
  | succ k ih =>
    obtain ⟨hn, hN, hacc⟩ := ih
    rw [Function.iterate_succ_apply']
    set s := bitsStep^[k] (Nat.pair n (Nat.pair 0 0))
    refine ⟨?_, ?_, ?_⟩
    · rw [bitsStep_unpair11, hn]
    · rw [bitsStep_unpair21, hN]
    · rw [bitsStep_unpair22, hn, hN, hacc]; rfl

theorem primrec_bitsCode : Nat.Primrec (fun t => bitsCode t.unpair.1 t.unpair.2) := by
  have hbase : Nat.Primrec (fun z : ℕ => Nat.pair z.unpair.1 (Nat.pair 0 0)) :=
    Nat.Primrec.left.pair ((Nat.Primrec.const 0).pair (Nat.Primrec.const 0))
  have hstep : Nat.Primrec (fun w : ℕ => bitsStep w.unpair.2.unpair.2) :=
    primrec_bitsStep.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hprec := Nat.Primrec.prec hbase hstep

  refine ((Nat.Primrec.right.comp Nat.Primrec.right).comp
    (hprec.comp (primrec_id.pair Nat.Primrec.right))).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rw [rec_const_iterate]
  exact (bitsStep_iter_spec t.unpair.1 t.unpair.2).2.2

theorem isComputableElement_of_elemSet_re {x : PN.Element} (hA : REPred (fun k => k ∈
    elemSet x)) :
    ComputableElement PNpres x := by
  have hforall : REPred (fun c => ∀ e ∈ decodeList c, e ∈ elemSet x) := hA.
    forall_mem_decodeList
  have hcode : Nat.Primrec (fun n : ℕ => bitsCode n n) :=
    (primrec_bitsCode.comp (primrec_id.pair primrec_id)).of_eq fun n => by
      simp only [unpair_pair_fst, unpair_pair_snd, id_eq]
  constructor
  refine REPred.of_iff (fun n => ?_) (hforall.comp hcode)
  show x.mem (nbhd n) ↔ _
  rw [nbhd_mem_iff_subset_elemSet, compl_nbhd]
  constructor
  · intro hsub e he
    exact hsub ((mem_decodeList_bitsCode n n e).mp he).2
  · intro hall k hk
    exact hall k ((mem_decodeList_bitsCode n n k).mpr ⟨lt_of_testBit_true hk, hk⟩)

theorem isComputableElement_iff_elemSet_re (x : PN.Element) :
    ComputableElement PNpres x ↔ REPred (fun k => k ∈ elemSet x) :=
  ⟨elemSet_re_of_isComputableElement, isComputableElement_of_elemSet_re⟩

theorem unnum_snd_lt_of_fst_succ {c k : ℕ} (h : (unnum c).1 = k + 1) : (unnum c).2 < c
    := by
  have heq : num (k + 1) (unnum c).2 = c := by
    have h' := numP_unnum c
    simpa [numP, h] using h'
  have hb := num_succ_left_gt k (unnum c).2
  omega

theorem bitAt_swap_isComputable : RecDecidable₂ (fun i m => bitAt m i = 1) :=
```

```
  ⟨fun t => bitAt t.unpair.2 t.unpair.1,
    (primrec_bitAt.comp (Nat.Primrec.right.pair Nat.Primrec.left)).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd],
    fun _ => Iff.rfl⟩

theorem capMap_rel_iff (n m k : ℕ) :
    capMap.rel (prodNbhd (nbhd n) (nbhd m)) (nbhd k) ↔ nbhd n ⊆ nbhd k ∧ nbhd m ⊆
    nbhd k := by
  constructor
  · rintro ⟨X, Y, heq, _, _, _, hXZ, hYZ⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact ⟨hXZ, hYZ⟩
  · rintro ⟨hXZ, hYZ⟩
    exact ⟨nbhd n, nbhd m, rfl, ⟨n, rfl⟩, ⟨m, rfl⟩, ⟨k, rfl⟩, hXZ, hYZ⟩

theorem capMap_isComputable :
    ComputableMap (prodPresentation PNpres PNpres) PNpres capMap := by
  constructor
  have hincl : RecDecidable (fun s => nbhd s.unpair.1 ⊆ nbhd s.unpair.2) := PNpres.
    incl_computable
  have hr0 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1 t.unpair.2) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right
  have hr1 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2 t.unpair.2) :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) ((hincl.comp hr0).and (hincl.comp hr1))).
    re
  simp only [prodPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact capMap_rel_iff t.unpair.1.unpair.1 t.unpair.1.unpair.2 t.unpair.2

theorem cupMap_rel_iff (n m k : ℕ) :
    cupMap.rel (prodNbhd (nbhd n) (nbhd m)) (nbhd k) ↔ nbhd (myLor n m) ⊆ nbhd k :=
    by
  rw [← nbhd_inter n m]
  constructor
  · rintro ⟨X, Y, heq, _, _, _, hXYZ⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact hXYZ
  · intro h
    exact ⟨nbhd n, nbhd m, rfl, ⟨n, rfl⟩, ⟨m, rfl⟩, ⟨k, rfl⟩, h⟩

theorem cupMap_isComputable :
    ComputableMap (prodPresentation PNpres PNpres) PNpres cupMap := by
  constructor
  have hincl : RecDecidable (fun s => nbhd s.unpair.1 ⊆ nbhd s.unpair.2) := PNpres.
    incl_computable
  have hlor : Nat.Primrec (fun t : ℕ => myLor t.unpair.1.unpair.1 t.unpair.1.unpair.2)
     :=
    (primrec_myLor.comp ((Nat.Primrec.left.comp Nat.Primrec.left).pair
      (Nat.Primrec.right.comp Nat.Primrec.left))).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hr : Nat.Primrec (fun t => Nat.pair (myLor t.unpair.1.unpair.1 t.unpair.1.
    unpair.2)
      t.unpair.2) := hlor.pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [prodPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact cupMap_rel_iff t.unpair.1.unpair.1 t.unpair.1.unpair.2 t.unpair.2
```

```
theorem decodeList_injective {a b : ℕ} (h : decodeList a = decodeList b) : a = b := by
  have := congrArg encodeList h
  rwa [encodeList_decodeList, encodeList_decodeList] at this

theorem forall_rel_iff_subset_step {W : Set (ApproximableMap PN PN)} {n m₀ : ℕ} :
    (∀ f ∈ W, f.rel (nbhd n) (nbhd m₀)) ↔ W ⊆ step (nbhd n) (nbhd m₀) := by
  constructor
  · intro h f hf; exact mem_step.mpr (h f hf)
  · intro h f hf; exact mem_step.mp (h hf)

theorem gMap_rel_iff (n m k : ℕ) :
    gMap.rel (prodNbhd (nbhd n) (nbhd m)) (nbhd k) ↔ (nbhd k)? ⊆ Fun (nbhd n)? (nbhd
    m)? := by
  constructor
  · rintro ⟨X, Y, heq, _, _, _, hsub⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact hsub
  · intro h
    exact ⟨nbhd n, nbhd m, rfl, ⟨n, rfl⟩, ⟨m, rfl⟩, ⟨k, rfl⟩, h⟩

theorem graphMap_rel_iff (gN : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf PNpres PNpres (decodeList c))
      : Set (ApproximableMap PN PN)).Nonempty) (c m : ℕ) :
    graphMap.rel (Xenum PNpres PNpres gN c) (nbhd m) ↔
      (nbhd m)? ⊆ GraphIdx (Xenum PNpres PNpres gN c) := by
  constructor
  · rintro ⟨_, _, hsub⟩; exact hsub
  · intro h; exact ⟨Xenum_mem PNpres PNpres gN hgN c, ⟨m, rfl⟩, h⟩

theorem mem_bitsCodeList {n k : ℕ} : k ∈ decodeList (bitsCode n n) ↔ n.testBit k =
    true := by
  rw [mem_decodeList_bitsCode]
  exact ⟨fun h => h.2, fun h => ⟨lt_of_testBit_true h, h⟩⟩

theorem mem_compl_nbhd {n x : ℕ} : x ∈ (nbhd n)? ↔ n.testBit x = true := by
  rw [compl_nbhd]; rfl

theorem compl_nbhd_subset_Fun_iff (n m k : ℕ) : (nbhd k)? ⊆ Fun (nbhd n)? (nbhd m)? ↔
    ∀ j, j < k → (k.testBit j = true → j ∈ Fun (nbhd n)? (nbhd m)?) := by
  constructor
  · intro h j _ hj
    exact h (mem_compl_nbhd.mpr hj)
  · intro h j hj
    rw [mem_compl_nbhd] at hj
    exact h j (lt_of_testBit_true hj) hj

theorem compl_nbhd_subset_iff (S : Set ℕ) (k : ℕ) : (nbhd k)? ⊆ S ↔
    ∀ j, j < k → (k.testBit j = true → j ∈ S) := by
  constructor
  · intro h j _ hj
    exact h (mem_compl_nbhd.mpr hj)
  · intro h j hj
    rw [mem_compl_nbhd] at hj
    exact h j (lt_of_testBit_true hj) hj
```

```
theorem mem_sumSet {X Y : Set ℕ} {k : ℕ} : k ∈ X +ˢ Y ↔ ∃ a ∈ X, ∃ b ∈ Y, a + b = k
    := Iff.rfl

theorem nbhd_subset_iff_compl_subset_compl (a b : ℕ) :
    nbhd a ⊆ nbhd b ↔ (nbhd b)ᶜ ⊆ (nbhd a)ᶜ := by
  constructor
  · exact compl_subset_compl_of_subset
  · intro h x hxa
    rw [mem_nbhd] at hxa ⊢
    cases hbit : b.testBit x with
    | false => rfl
    | true => exact absurd (mem_compl_nbhd.mp (h (mem_compl_nbhd.mpr hbit))) (by rw [
    hxa]; decide)

theorem nbhd_subset_iff_myLor_eq (n k : ℕ) : nbhd n ⊆ nbhd k ↔ myLor n k = n := by
  constructor
  · intro h
    apply nbhd_injective
    rw [← nbhd_inter]
    exact (Set.inter_eq_left.mpr h)
  · intro h
    rw [← h, ← nbhd_inter]
    exact Set.inter_subset_right

theorem nextCellStep_eq (n m : ℕ) :
    nextCellStep (Nat.pair n m) = Nat.pair (nextCell (n, m)).1 (nextCell (n, m)).2 :=
    by
  unfold nextCellStep
  simp only [unpair_pair_fst, unpair_pair_snd]
  rcases n with _ | k
  · have hz : isZero 0 = 1 := by unfold isZero; omega
    rw [hz, selectFn_one]; rfl
  · have hz : isZero (k + 1) = 0 := by unfold isZero; omega
    rw [hz, selectFn_zero]; rfl

theorem plusStep_unpair11 (s : ℕ) : (plusStep s).unpair.1.unpair.1 = s.unpair.1.unpair
    .1 := by
  unfold plusStep; rw [unpair_pair_fst, unpair_pair_fst]

theorem plusStep_unpair12 (s : ℕ) : (plusStep s).unpair.1.unpair.2 = s.unpair.1.unpair
    .2 := by
  unfold plusStep; rw [unpair_pair_fst, unpair_pair_snd]

theorem plusStep_unpair21 (s : ℕ) : (plusStep s).unpair.2.unpair.1 = s.unpair.2.unpair
    .1 + 1 := by
  unfold plusStep; rw [unpair_pair_snd, unpair_pair_fst]

theorem plusStep_unpair22 (s : ℕ) : (plusStep s).unpair.2.unpair.2 =
    myLor s.unpair.2.unpair.2 (selectFn (bitAt s.unpair.1.unpair.1 s.unpair.2.unpair
    .1)
      (s.unpair.1.unpair.2 <<< s.unpair.2.unpair.1) 0) := by
  unfold plusStep; rw [unpair_pair_snd, unpair_pair_snd]

theorem plusStep_iter_spec (n m : ℕ) (k : ℕ) :
    (plusStep^[k] (Nat.pair (Nat.pair n m) (Nat.pair 0 0))).unpair.1.unpair.1 = n ∧
    (plusStep^[k] (Nat.pair (Nat.pair n m) (Nat.pair 0 0))).unpair.1.unpair.2 = m ∧
```

```
    (plusStep^[k] (Nat.pair (Nat.pair n m) (Nat.pair 0 0))).unpair.2.unpair.1 = k ∧
    (plusStep^[k] (Nat.pair (Nat.pair n m) (Nat.pair 0 0))).unpair.2.unpair.2 = orUpTo
     n m k := by
  induction k with
  | zero =>
    refine ⟨?_, ?_, ?_, ?_⟩ <;>
      simp only [Function.iterate_zero, id_eq, unpair_pair_fst, unpair_pair_snd,
    orUpTo]
  | succ k ih =>
    obtain ⟨hn, hm, ha, hacc⟩ := ih
    rw [Function.iterate_succ_apply']
    set s := plusStep^[k] (Nat.pair (Nat.pair n m) (Nat.pair 0 0))
    refine ⟨?_, ?_, ?_, ?_⟩
    · rw [plusStep_unpair11, hn]
    · rw [plusStep_unpair12, hm]
    · rw [plusStep_unpair21, ha]
    · rw [plusStep_unpair22, hn, hm, ha, hacc]; rfl

theorem primrec_nextCellStep : Nat.Primrec nextCellStep := by
  have hn : Nat.Primrec (fun w : ℕ => w.unpair.1) := Nat.Primrec.left
  have hm : Nat.Primrec (fun w : ℕ => w.unpair.2) := Nat.Primrec.right
  have hz : Nat.Primrec (fun w : ℕ => isZero w.unpair.1) := primrec_isZero.comp hn
  have ha : Nat.Primrec (fun w : ℕ => Nat.pair (w.unpair.2 + 1) 0) :=
    (primrec_add₂ hm (Nat.Primrec.const 1)).pair (Nat.Primrec.const 0)
  have hb : Nat.Primrec (fun w : ℕ => Nat.pair (w.unpair.1 - 1) (w.unpair.2 + 1)) :=
    (primrec_sub₂ hn (Nat.Primrec.const 1)).pair (primrec_add₂ hm (Nat.Primrec.const 1)
    )
  exact (primrec_selectFn hz ha hb).of_eq fun _ => rfl

theorem primrec_plusStep : Nat.Primrec plusStep := by
  have hn : Nat.Primrec (fun s : ℕ => s.unpair.1.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.left
  have hm : Nat.Primrec (fun s : ℕ => s.unpair.1.unpair.2) := Nat.Primrec.right.comp
    Nat.Primrec.left
  have ha : Nat.Primrec (fun s : ℕ => s.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have hacc : Nat.Primrec (fun s : ℕ => s.unpair.2.unpair.2) := Nat.Primrec.right.comp
     Nat.Primrec.right
  have hbit : Nat.Primrec (fun s : ℕ => bitAt s.unpair.1.unpair.1 s.unpair.2.unpair.1)
     :=
    (primrec_bitAt.comp (hn.pair ha)).of_eq fun s => by simp only [unpair_pair_fst,
    unpair_pair_snd]
  have hshift : Nat.Primrec (fun s : ℕ => s.unpair.1.unpair.2 <<< s.unpair.2.unpair.1)
     := by
    have := primrec_mul₂ hm (primrec_two_pow ha)
    exact this.of_eq fun s => by rw [Nat.shiftLeft_eq]
  have hsel := primrec_selectFn hbit hshift (Nat.Primrec.const 0)
  have hlor : Nat.Primrec (fun s : ℕ => myLor s.unpair.2.unpair.2
      (selectFn (bitAt s.unpair.1.unpair.1 s.unpair.2.unpair.1)
        (s.unpair.1.unpair.2 <<< s.unpair.2.unpair.1) 0)) :=
    (primrec_myLor.comp (hacc.pair hsel)).of_eq fun s => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact ((hn.pair hm).pair ((primrec_add₂ ha (Nat.Primrec.const 1)).pair hlor)).of_eq
    fun _ => rfl

theorem primrec_plusIdx : Nat.Primrec (fun t => plusIdx t.unpair.1 t.unpair.2) := by
```

```
  have hbase : Nat.Primrec
      (fun z => Nat.pair (Nat.pair z.unpair.1 z.unpair.2) (Nat.pair 0 0)) :=
    (Nat.Primrec.left.pair Nat.Primrec.right).pair
      ((Nat.Primrec.const 0).pair (Nat.Primrec.const 0))
  have hstep : Nat.Primrec (fun w => plusStep w.unpair.2.unpair.2) :=
    primrec_plusStep.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hprec := Nat.Primrec.prec hbase hstep
  refine ((Nat.Primrec.right.comp Nat.Primrec.right).comp
    (hprec.comp (primrec_id.pair Nat.Primrec.left))).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rw [rec_const_iterate]
  exact (plusStep_iter_spec t.unpair.1 t.unpair.2 t.unpair.1).2.2.2

theorem primrec_tri : Nat.Primrec tri :=
  (primrec_div2.comp (primrec_mul₂ primrec_id Nat.Primrec.succ)).of_eq fun _ => rfl

theorem primrec_num : Nat.Primrec (fun t => num t.unpair.1 t.unpair.2) :=
  (primrec_add₂ (primrec_tri.comp (primrec_add₂ Nat.Primrec.left Nat.Primrec.right))
    Nat.Primrec.right).of_eq fun _ => rfl

theorem primrec_tagStep : Nat.Primrec tagStep := by
  have h1 : Nat.Primrec (fun w : ℕ => w.unpair.1 + 1) :=
    primrec_add₂ Nat.Primrec.left (Nat.Primrec.const 1)
  have h2 : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  exact (primrec_num.comp (h1.pair h2)).of_eq fun w => by
    simp only [unpair_pair_fst, unpair_pair_snd]
    rfl

theorem primrec_tagCode : Nat.Primrec (fun t => tagCode t.unpair.1 t.unpair.2) := by
  have hz : Nat.Primrec (fun t : ℕ => num 0 t.unpair.2) :=
    (primrec_num.comp ((Nat.Primrec.const 0).pair Nat.Primrec.right)).of_eq fun t =>
    by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hc : Nat.Primrec (fun t : ℕ => appendCode 0 t.unpair.1) :=
    (primrec_appendCode.comp ((Nat.Primrec.const 0).pair Nat.Primrec.left)).of_eq fun
    t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_foldCode primrec_tagStep (Nat.Primrec.const 0) hz hc).of_eq fun _ =>
    rfl

theorem primrec_tagOfBits : Nat.Primrec (fun t => tagOfBits t.unpair.1 t.unpair.2) :=
    by
  have hj : Nat.Primrec (fun t : ℕ => bitsCode t.unpair.1 t.unpair.1) :=
    (primrec_bitsCode.comp (Nat.Primrec.left.pair Nat.Primrec.left)).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_tagCode.comp (hj.pair Nat.Primrec.right)).of_eq fun t => by
    simp only [unpair_pair_fst, unpair_pair_snd]
    rfl

theorem primrec_unnumPair : Nat.Primrec unnumPair := by
  have hbase : Nat.Primrec (fun _ : ℕ => Nat.pair (0 : ℕ) 0) := Nat.Primrec.const (Nat
    .pair 0 0)
  have hstep : Nat.Primrec (fun w : ℕ => nextCellStep w.unpair.2.unpair.2) :=
    primrec_nextCellStep.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hprec := Nat.Primrec.prec hbase hstep
```

```
  refine (hprec.comp ((Nat.Primrec.const 0).pair primrec_id)).of_eq fun v => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  show Nat.rec (Nat.pair 0 0) (fun _ ih => nextCellStep ih) v = unnumPair v
  rw [rec_const_iterate]; rfl

theorem primrec_unnum1 : Nat.Primrec unnum1 := Nat.Primrec.left.comp primrec_unnumPair

theorem primrec_unnum2 : Nat.Primrec unnum2 := Nat.Primrec.right.comp
    primrec_unnumPair

theorem primrec_untagStep : Nat.Primrec untagStep := by
  have hdone : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hrevAcc : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hval : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have ha : Nat.Primrec (fun w : ℕ => unnum1 w.unpair.1) := primrec_unnum1.comp Nat.
    Primrec.left
  have hb : Nat.Primrec (fun w : ℕ => unnum2 w.unpair.1) := primrec_unnum2.comp Nat.
    Primrec.left
  have hz : Nat.Primrec (fun w : ℕ => isZero (unnum1 w.unpair.1)) := primrec_isZero.
    comp ha
  have hhalt : Nat.Primrec (fun w : ℕ => Nat.pair (unnum2 w.unpair.1)
      (Nat.pair 1 (Nat.pair w.unpair.2.unpair.2.unpair.1 (unnum2 w.unpair.1)))) :=
    hb.pair ((Nat.Primrec.const 1).pair (hrevAcc.pair hb))
  have hcont : Nat.Primrec (fun w : ℕ => Nat.pair (unnum2 w.unpair.1)
      (Nat.pair 0
        (Nat.pair (Nat.pair (unnum1 w.unpair.1 - 1) w.unpair.2.unpair.2.unpair.1 + 1)
          w.unpair.2.unpair.2.unpair.2))) :=
    hb.pair ((Nat.Primrec.const 0).pair
      ((Nat.Primrec.succ.comp ((primrec_sub₂ ha (Nat.Primrec.const 1)).pair hrevAcc)).
    pair hval))
  have hinner := primrec_selectFn hz hhalt hcont
  exact (primrec_selectFn hdone primrec_id hinner).of_eq fun _ => rfl

theorem primrec_untagState : Nat.Primrec untagState := by
  have hbase : Nat.Primrec (fun c : ℕ => Nat.pair c (Nat.pair (0 : ℕ) (Nat.pair 0 0)))
     :=
    primrec_id.pair ((Nat.Primrec.const 0).pair ((Nat.Primrec.const 0).pair (Nat.
    Primrec.const 0)))
  have hstep : Nat.Primrec (fun w : ℕ => untagStep w.unpair.2.unpair.2) :=
    primrec_untagStep.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hprec := Nat.Primrec.prec hbase hstep
  refine (hprec.comp (primrec_id.pair Nat.Primrec.succ)).of_eq fun c => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rw [rec_const_iterate]
  unfold untagState
  rw [Function.iterate_succ_apply']

theorem primrec_untagList : Nat.Primrec untagList :=
  primrec_appendCode.comp
    ((Nat.Primrec.const 0).pair
      (Nat.Primrec.left.comp (Nat.Primrec.right.comp (Nat.Primrec.right.comp
    primrec_untagState))))
    |>.of_eq fun c => by unfold untagList; simp only [unpair_pair_fst, unpair_pair_snd
```

```
    ]

theorem primrec_untagVal : Nat.Primrec untagVal :=
  Nat.Primrec.right.comp (Nat.Primrec.right.comp (Nat.Primrec.right.comp
    primrec_untagState))

theorem tag_eq_foldr (ns : List ℕ) (m : ℕ) :
    tag ns m = ns.foldr (fun x y => num (x + 1) y) (num 0 m) := by
  induction ns with
  | nil => rfl
  | cons a ns ih => rw [tag_cons, ih]; rfl

theorem tag_eq_foldl_reverse (ns : List ℕ) (m : ℕ) :
    tag ns m = ns.reverse.foldl (fun acc a => num (a + 1) acc) (num 0 m) := by
  rw [List.foldl_reverse]; exact tag_eq_foldr ns m

theorem tagCode_spec (code m : ℕ) : tagCode code m = tag (decodeList code) m := by
  unfold tagCode
  rw [foldCode_eq']
  have hstep : (fun acc x => tagStep (Nat.pair x (Nat.pair acc 0)))
      = (fun acc x => num (x + 1) acc) := by
    funext acc x
    show tagStep (Nat.pair x (Nat.pair acc 0)) = num (x + 1) acc
    unfold tagStep
    simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hstep, decodeList_appendCode, decodeList_zero, List.append_nil]
  exact (tag_eq_foldl_reverse (decodeList code) m).symm

theorem tagOfBits_spec (j m : ℕ) : tagOfBits j m = tag (decodeList (bitsCode j j)) m
    :=
  tagCode_spec (bitsCode j j) m

theorem testBit_orUpTo (n m : ℕ) : ∀ N k,
    (orUpTo n m N).testBit k = true ↔ ∃ a, a < N ∧ n.testBit a = true ∧ a ≤ k ∧ m.
    testBit (k - a) = true
  | 0, k => by simp [orUpTo]
  | N + 1, k => by
    show (myLor (orUpTo n m N) (selectFn (bitAt n N) (m <<< N) 0)).testBit k = true ↔
    _
    rw [myLor_eq_lor, Nat.testBit_lor, Bool.or_eq_true, testBit_orUpTo n m N]
    by_cases hbit : n.testBit N = true
    · rw [(bitAt_eq_one_iff n N).mpr hbit, selectFn_one, Nat.testBit_shiftLeft]
      constructor
      · rintro (⟨a, haN, ha, hak, hb⟩ | h)
        · exact ⟨a, Nat.lt_succ_of_lt haN, ha, hak, hb⟩
        · rw [Bool.and_eq_true, decide_eq_true_iff] at h
          exact ⟨N, Nat.lt_succ_self N, hbit, h.1, h.2⟩
      · rintro ⟨a, haN1, ha, hak, hb⟩
        rcases (show a < N ∨ a = N by omega) with haN | haN
        · exact Or.inl ⟨a, haN, ha, hak, hb⟩
        · subst haN
          exact Or.inr (by rw [Bool.and_eq_true, decide_eq_true_iff]; exact ⟨hak, hb⟩)
    · have hne1 : bitAt n N = 1 := fun h => hbit ((bitAt_eq_one_iff n N).mp h)
      have hle := bitAt_le_one n N
      have hnbit : bitAt n N = 0 := by omega
      rw [hnbit, selectFn_zero]
```

```
      simp only [Nat.zero_testBit, Bool.false_eq_true, or_false]
      constructor
      · rintro ⟨a, haN, ha, hak, hb⟩; exact ⟨a, Nat.lt_succ_of_lt haN, ha, hak, hb⟩
      · rintro ⟨a, haN1, ha, hak, hb⟩
        rcases (show a < N ∨ a = N by omega) with haN | haN
        · exact ⟨a, haN, ha, hak, hb⟩
        · subst haN; exact absurd ha hbit

theorem testBit_orUpTo_self (n m k : ℕ) :
    (orUpTo n m n).testBit k = true ↔ ∃ a, a ≤ k ∧ n.testBit a = true ∧ m.testBit (k
    - a) = true := by
  rw [testBit_orUpTo]
  constructor
  · rintro ⟨a, _, ha, hak, hb⟩; exact ⟨a, hak, ha, hb⟩
  · rintro ⟨a, hak, ha, hb⟩; exact ⟨a, lt_of_testBit_true ha, ha, hak, hb⟩

theorem plusIdx_testBit (n m k : ℕ) :
    (plusIdx n m).testBit k = true ↔ ∃ a, a ≤ k ∧ n.testBit a = true ∧ m.testBit (k -
     a) = true :=
  testBit_orUpTo_self n m k


theorem compl_nbhd_plusIdx (n m : ℕ) : (nbhd (plusIdx n m))? = (nbhd n)? +ˢ (nbhd m)?
    := by
  rw [compl_nbhd, compl_nbhd, compl_nbhd]
  ext k
  simp only [mem_sumSet, Set.mem_ofPred_eq, plusIdx_testBit]
  constructor
  · rintro ⟨a, hak, ha, hb⟩; exact ⟨a, ha, k - a, hb, by omega⟩
  · rintro ⟨a, ha, b, hb, rfl⟩
    exact ⟨a, by omega, ha, by rw [Nat.add_sub_cancel_left]; exact hb⟩

theorem plusMap_rel_iff (n m k : ℕ) :
    plusMap.rel (prodNbhd (nbhd n) (nbhd m)) (nbhd k) ↔ nbhd (plusIdx n m) ⊆ nbhd k
    := by
  rw [nbhd_subset_iff_compl_subset_compl, compl_nbhd_plusIdx]
  constructor
  · rintro ⟨X, Y, heq, _, _, _, hsub⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact hsub
  · intro h
    exact ⟨nbhd n, nbhd m, rfl, ⟨n, rfl⟩, ⟨m, rfl⟩, ⟨k, rfl⟩, h⟩

theorem plusMap_isComputable :
    ComputableMap (prodPresentation PNpres PNpres) PNpres plusMap := by
  constructor
  have hincl : RecDecidable (fun s => nbhd s.unpair.1 ⊆ nbhd s.unpair.2) := PNpres.
    incl_computable
  have hpair : Nat.Primrec (fun t : ℕ => Nat.pair t.unpair.1.unpair.1 t.unpair.1.
    unpair.2) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.Primrec.
    left)
  have hplus : Nat.Primrec (fun t : ℕ => plusIdx t.unpair.1.unpair.1 t.unpair.1.unpair
    .2) :=
    (primrec_plusIdx.comp hpair).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hr : Nat.Primrec (fun t => Nat.pair (plusIdx t.unpair.1.unpair.1 t.unpair.1.
```

```
    unpair.2)
      t.unpair.2) := hplus.pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [prodPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact plusMap_rel_iff t.unpair.1.unpair.1 t.unpair.1.unpair.2 t.unpair.2

theorem unnumPair_spec (v : ℕ) : unnumPair v = Nat.pair (unnum v).1 (unnum v).2 := by
  unfold unnumPair
  induction v with
  | zero => rfl
  | succ v ih => rw [Function.iterate_succ_apply', ih, nextCellStep_eq]; rfl

theorem unnum1_spec (v : ℕ) : unnum1 v = (unnum v).1 := by
  unfold unnum1; rw [unnumPair_spec, unpair_pair_fst]

theorem unnum2_spec (v : ℕ) : unnum2 v = (unnum v).2 := by
  unfold unnum2; rw [unnumPair_spec, unpair_pair_snd]

theorem untagList_bitAt_isComputable :
    RecDecidable (fun w : ℕ => ∀ i ∈ decodeList (untagList w.unpair.1), bitAt w.unpair
    .2.unpair.2 i = 1) :=
  RecDecidable.of_iff (fun w => by simp only [unpair_pair_fst, unpair_pair_snd])
    (bitAt_swap_isComputable.bForallList.comp
      ((primrec_untagList.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
    Primrec.right)))

theorem untagRef_succ_case {c k : ℕ} (h : (unnum c).1 = k + 1) :
    untagRef c = (k :: (untagRef (unnum c).2).1, (untagRef (unnum c).2).2) := by
  have h0 : ¬ (unnum c).1 = 0 := by omega
  rw [untagRef, dite_eq_right h0, h]
  have hsub : k + 1 - 1 = k := by omega
  rw [hsub]

theorem untagRef_zero_case {c : ℕ} (h : (unnum c).1 = 0) :
    untagRef c = ([], (unnum c).2) := by
  rw [untagRef]; simp [h]

theorem tag_untagRef (c : ℕ) : tag (untagRef c).1 (untagRef c).2 = c := by
  induction c using Nat.strong_induction_on with
  | _ c ih =>
    rcases hc : (unnum c).1 with a | k
    · rw [untagRef_zero_case hc]
      have heq := numP_unnum c
      simp only [numP, hc] at heq
      rw [tag_nil]
      exact heq
    · rw [untagRef_succ_case hc]
      have heq : num (k + 1) (unnum c).2 = c := by
        have h := numP_unnum c
        simpa [numP, hc] using h
      have hlt : (unnum c).2 < c := by
        have hb := num_succ_left_gt k (unnum c).2
        omega
      rw [tag_cons, ih (unnum c).2 hlt, heq]

theorem untagStep_cont {w : ℕ} (hdone : w.unpair.2.unpair.1 = 0) {k : ℕ}
```

```
    (ha : unnum1 w.unpair.1 = k + 1) :
    untagStep w = Nat.pair (unnum2 w.unpair.1)
      (Nat.pair 0
        (Nat.pair (Nat.pair k w.unpair.2.unpair.2.unpair.1 + 1) w.unpair.2.unpair.2.
    unpair.2)) := by
  unfold untagStep
  rw [hdone, selectFn_zero, ha]
  have hz : isZero (k + 1) = 0 := by unfold isZero; omega
  have hsub : k + 1 - 1 = k := by omega
  rw [hz, selectFn_zero, hsub]

theorem untagStep_done {w : ℕ} (h : w.unpair.2.unpair.1 = 1) : untagStep w = w := by
  unfold untagStep; rw [h, selectFn_one]

theorem untagStep_halt {w : ℕ} (hdone : w.unpair.2.unpair.1 = 0) (ha : unnum1 w.unpair
    .1 = 0) :
    untagStep w = Nat.pair (unnum2 w.unpair.1)
      (Nat.pair 1 (Nat.pair w.unpair.2.unpair.2.unpair.1 (unnum2 w.unpair.1))) := by
  unfold untagStep
  rw [hdone, selectFn_zero, ha]
  have : isZero 0 = 1 := by unfold isZero; omega
  rw [this, selectFn_one]

theorem untagStep_of_done_iterate (w : ℕ) (h : w.unpair.2.unpair.1 = 1) :
    ∀ k, untagStep^[k] w = w
  | 0 => rfl
  | k + 1 => by rw [Function.iterate_succ_apply', untagStep_of_done_iterate w h k,
    untagStep_done h]

theorem untagStep_iter_spec : ∀ c, ∀ fuel r v : ℕ, c ≤ fuel →
    untagStep^[fuel + 1] (Nat.pair c (Nat.pair 0 (Nat.pair r v))) =
      Nat.pair (untagRef c).2 (Nat.pair 1
        (Nat.pair ((untagRef c).1.foldl (fun acc x => Nat.pair x acc + 1) r) (untagRef
     c).2)) := by
  intro c
  induction c using Nat.strong_induction_on with
  | _ c ih =>
    intro fuel r v hfuel
    set w : ℕ := Nat.pair c (Nat.pair 0 (Nat.pair r v)) with hw
    have hw1 : w.unpair.1 = c := by rw [hw, unpair_pair_fst]
    have hw2 : w.unpair.2.unpair.1 = 0 := by rw [hw]; simp only [unpair_pair_snd,
    unpair_pair_fst]
    have hw3 : w.unpair.2.unpair.2.unpair.1 = r := by
      rw [hw]; simp only [unpair_pair_snd, unpair_pair_fst]
    have hw4 : w.unpair.2.unpair.2.unpair.2 = v := by
      rw [hw]; simp only [unpair_pair_snd]
    rw [Function.iterate_succ_apply]
    rcases hc : (unnum c).1 with _ | k
    · have ha0 : unnum1 w.unpair.1 = 0 := by rw [hw1, unnum1_spec, hc]
      have hstep : untagStep w = Nat.pair (unnum c).2 (Nat.pair 1 (Nat.pair r (unnum c
    ).2)) := by
        rw [untagStep_halt hw2 ha0, hw1, hw3, unnum2_spec]
      rw [hstep]
      have hdone1 : (Nat.pair (unnum c).2
          (Nat.pair 1 (Nat.pair r (unnum c).2))).unpair.2.unpair.1 = 1 := by
        simp only [unpair_pair_snd, unpair_pair_fst]
```

```
      rw [untagStep_of_done_iterate _ hdone1, untagRef_zero_case hc, List.foldl_nil]
    · have hak : unnum1 w.unpair.1 = k + 1 := by rw [hw1, unnum1_spec, hc]
      set b : ℕ := (unnum c).2 with hbdef
      have hblt : b < c := unnum_snd_lt_of_fst_succ hc
      have hc1 : 1 ≤ c := by omega
      have hstep : untagStep w = Nat.pair b (Nat.pair 0 (Nat.pair (Nat.pair k r + 1) v
    )) := by
        rw [untagStep_cont hw2 hak, hw1, hw3, hw4, unnum2_spec]
      rw [hstep]
      rcases fuel with _ | fuel'
      · omega
      · have hble : b ≤ fuel' := by omega
        have hihb := ih b hblt fuel' (Nat.pair k r + 1) v hble
        rw [hihb]
        have href := untagRef_succ_case (c := c) (k := k) hc
        rw [← hbdef] at href
        rw [href, List.foldl_cons]

theorem untagState_spec (c : ℕ) :
    untagState c = Nat.pair (untagRef c).2
      (Nat.pair 1 (Nat.pair ((untagRef c).1.foldl (fun acc x => Nat.pair x acc + 1) 0)
        (untagRef c).2)) :=
  untagStep_iter_spec c c 0 0 (le_refl c)

theorem decodeList_untagList (c : ℕ) : decodeList (untagList c) = (untagRef c).1 := by
  unfold untagList
  rw [decodeList_appendCode, decodeList_zero, List.append_nil]
  have hspec := untagState_spec c
  have hrevAcc : (untagState c).unpair.2.unpair.2.unpair.1
      = (untagRef c).1.foldl (fun acc x => Nat.pair x acc + 1) 0 := by
    rw [hspec]; simp only [unpair_pair_snd, unpair_pair_fst]
  rw [hrevAcc]
  have hfold : ∀ l : List ℕ, ∀ r : ℕ,
      decodeList (l.foldl (fun acc x => Nat.pair x acc + 1) r) = l.reverse ++
    decodeList r := by
    intro l
    induction l with
    | nil => intro r; simp
    | cons a l ih =>
      intro r
      rw [List.foldl_cons, ih, decodeList_succ, unpair_pair_fst, unpair_pair_snd,
        List.reverse_cons, List.append_assoc, List.singleton_append]
  rw [hfold (untagRef c).1 0, decodeList_zero, List.append_nil, List.reverse_reverse]

theorem untagVal_and_isComputable :
    RecDecidable (fun w : ℕ => untagVal w.unpair.1 = w.unpair.2.unpair.1 ∧
      ∀ i ∈ decodeList (untagList w.unpair.1), bitAt w.unpair.2.unpair.2 i = 1) :=
  (RecDecidable.natEq (primrec_untagVal.comp Nat.Primrec.left)
    (Nat.Primrec.left.comp Nat.Primrec.right)).and untagList_bitAt_isComputable

theorem untagVal_spec (c : ℕ) : untagVal c = (untagRef c).2 := by
  unfold untagVal; rw [untagState_spec]; simp only [unpair_pair_snd]

theorem tag_untagList_untagVal (c : ℕ) : tag (decodeList (untagList c)) (untagVal c) =
    c := by
  rw [decodeList_untagList, untagVal_spec]; exact tag_untagRef c
```

```
theorem mem_Fun_compl_nbhd_iff (n m j : ℕ) :
    j ∈ Fun (nbhd n)? (nbhd m)? ↔
      ∃ c ∈ decodeList (bitsCode n n), untagVal c = j ∧
        ∀ i ∈ decodeList (untagList c), i ∈ (nbhd m)? := by
  constructor
  · rintro ⟨ns, hns, htag⟩
    have hc : tag ns j ∈ decodeList (bitsCode n n) := by
      rw [mem_bitsCodeList]; exact mem_compl_nbhd.mp htag
    refine ⟨tag ns j, hc, ?_, ?_⟩
    · exact (tag_injective (tag_untagList_untagVal (tag ns j))).2
    · rw [(tag_injective (tag_untagList_untagVal (tag ns j))).1]
      exact hns
  · rintro ⟨c, hc, hval, hall⟩
    refine ⟨decodeList (untagList c), hall, ?_⟩
    rw [mem_compl_nbhd, ← hval, tag_untagList_untagVal]
    exact mem_bitsCodeList.mp hc

theorem mem_Fun_compl_nbhd_iff' (n m j : ℕ) :
    j ∈ Fun (nbhd n)? (nbhd m)? ↔
      ∃ c, c < n ∧ n.testBit c = true ∧ untagVal c = j ∧
        ∀ i ∈ decodeList (untagList c), bitAt m i = 1 := by
  rw [mem_Fun_compl_nbhd_iff]
  simp only [mem_bitsCodeList, mem_compl_nbhd, bitAt_eq_one_iff]
  constructor
  · rintro ⟨c, hbit, hval, hall⟩
    exact ⟨c, lt_of_testBit_true hbit, hbit, hval, hall⟩
  · rintro ⟨c, -, hbit, hval, hall⟩
    exact ⟨c, hbit, hval, hall⟩

theorem mem_Fun_compl_nbhd_isComputable :
    RecDecidable (fun w : ℕ =>
      w.unpair.2.unpair.1 ∈ Fun (nbhd w.unpair.1)? (nbhd w.unpair.2.unpair.2)?) := by
  have hbitN : RecDecidable (fun w : ℕ => w.unpair.2.unpair.1.testBit w.unpair.1 =
    true) :=
    ⟨fun w => bitAt w.unpair.2.unpair.1 w.unpair.1,
      (primrec_bitAt.comp ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.
    left)).of_eq
        fun w => by simp only [unpair_pair_fst, unpair_pair_snd],
      fun w => (bitAt_eq_one_iff _ _).symm⟩
  have hCJ' : RecDecidable (fun w : ℕ => untagVal w.unpair.1 = w.unpair.2.unpair.2.
    unpair.1 ∧
      ∀ i ∈ decodeList (untagList w.unpair.1), bitAt w.unpair.2.unpair.2.unpair.2 i =
    1) :=
    RecDecidable.of_iff (fun w => by simp only [unpair_pair_fst, unpair_pair_snd])
      (untagVal_and_isComputable.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp
    Nat.Primrec.right)))
  have hp := hbitN.and hCJ'
  refine RecDecidable.of_iff (fun w => ?_) (hp.bExists Nat.Primrec.left)
  rw [mem_Fun_compl_nbhd_iff']
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem mem_GraphIdx_iff (W : Set (ApproximableMap PN PN)) (c : ℕ) :
    c ∈ GraphIdx W ↔ ∃ n m₀, untagList c = bitsCode n n ∧
      (∀ f ∈ W, f.rel (nbhd n) (nbhd m₀)) ∧ m₀.testBit (untagVal c) = true := by
  constructor
```

```
  · rintro ⟨n, m₀, m, rfl, hf, hbit⟩
    have htag : tag (decodeList (untagList (tagOfBits n m))) (untagVal (tagOfBits n m)
    ) =
        tag (decodeList (bitsCode n n)) m :=
      (tag_untagList_untagVal (tagOfBits n m)).trans (tagOfBits_spec n m)
    obtain ⟨hlist, hval⟩ := tag_injective htag
    refine ⟨n, m₀, decodeList_injective hlist, hf, ?_⟩
    rw [hval]; exact hbit
  · rintro ⟨n, m₀, heq, hf, hbit⟩
    refine ⟨n, m₀, untagVal c, ?_, hf, hbit⟩
    rw [tagOfBits_spec, ← heq]
    exact (tag_untagList_untagVal c).symm

theorem graphIdx_isComputable (gN incl0 incl1 eq1 : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf PNpres PNpres (decodeList c))
      : Set (ApproximableMap PN PN)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : ∀ s, incl0 s = 1 ↔ PNpres.X s.unpair.1 ⊆ PNpres.X s.unpair.2)
    (hincl0p : Nat.Primrec incl0)
    (hincl1 : ∀ s, incl1 s = 1 ↔ PNpres.X s.unpair.1 ⊆ PNpres.X s.unpair.2)
    (hincl1p : Nat.Primrec incl1)
    (heq1 : ∀ s, eq1 s = 1 ↔ PNpres.X s.unpair.1 = PNpres.X s.unpair.2) (heq1p : Nat.
    Primrec eq1) :
    REPred₂ (fun c j => j ∈ GraphIdx
      (Xenum PNpres PNpres gN c : Set (ApproximableMap PN PN))) := by
  have hinclP : RecDecidable₂ (fun a b =>
      (Xenum PNpres PNpres gN a : Set (ApproximableMap PN PN)) ⊆ Xenum PNpres PNpres
    gN b) :=
    (funPresentation PNpres PNpres gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p hincl1
    hincl1p heq1
      heq1p).incl_computable

  have hj : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hn : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hc : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hm0 : Nat.Primrec (fun w : ℕ => w.unpair.1) := Nat.Primrec.left
  have hbitsCodeArg : Nat.Primrec (fun w : ℕ => bitsCode w.unpair.2.unpair.1 w.unpair
    .2.unpair.1) :=
    (primrec_bitsCode.comp (hn.pair hn)).of_eq fun w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hpairArg : Nat.Primrec
      (fun w : ℕ => Nat.pair (Nat.pair w.unpair.2.unpair.1 w.unpair.1) 0 + 1) :=
    Nat.Primrec.succ.comp ((hn.pair hm0).pair (Nat.Primrec.const 0))
  have hA : RecDecidable (fun w : ℕ =>
      untagList w.unpair.2.unpair.2.unpair.2 =
        bitsCode w.unpair.2.unpair.1 w.unpair.2.unpair.1) :=
    RecDecidable.natEq (primrec_untagList.comp hj) hbitsCodeArg
  have hB : RecDecidable (fun w : ℕ =>
      (Xenum PNpres PNpres gN w.unpair.2.unpair.2.unpair.1 : Set (ApproximableMap PN
    PN)) ⊆
        Xenum PNpres PNpres gN (Nat.pair (Nat.pair w.unpair.2.unpair.1 w.unpair.1) 0 +
     1)) :=
    RecDecidable.of_iff (fun w => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hinclP.comp (hc.pair hpairArg))
```

```
  have hC : RecDecidable (fun w : ℕ =>
      w.unpair.1.testBit (untagVal w.unpair.2.unpair.2.unpair.2) = true) :=
    ⟨fun w => bitAt w.unpair.1 (untagVal w.unpair.2.unpair.2.unpair.2),
      (primrec_bitAt.comp (hm0.pair (primrec_untagVal.comp hj))).of_eq fun w => by
        simp only [unpair_pair_fst, unpair_pair_snd],
      fun w => (bitAt_eq_one_iff _ _).symm⟩
  have hre : REPred (fun w : ℕ =>
      untagList w.unpair.2.unpair.2.unpair.2 =
        bitsCode w.unpair.2.unpair.1 w.unpair.2.unpair.1 ∧
      (Xenum PNpres PNpres gN w.unpair.2.unpair.2.unpair.1 : Set (ApproximableMap PN
    PN)) ⊆
        Xenum PNpres PNpres gN (Nat.pair (Nat.pair w.unpair.2.unpair.1 w.unpair.1) 0 +
     1) ∧
      w.unpair.1.testBit (untagVal w.unpair.2.unpair.2.unpair.2) = true) :=
    (hA.and (hB.and hC)).re
  have hp2 := hre.proj.proj
  refine REPred.of_iff (fun t => ?_) hp2
  dsimp only
  rw [mem_GraphIdx_iff]
  constructor
  · rintro ⟨n, m₀, heq, hf, hbit⟩
    refine ⟨n, m₀, ?_⟩
    show untagList (Nat.pair m₀ (Nat.pair n t)).unpair.2.unpair.2.unpair.2 =
          bitsCode (Nat.pair m₀ (Nat.pair n t)).unpair.2.unpair.1
            (Nat.pair m₀ (Nat.pair n t)).unpair.2.unpair.1 ∧
        (Xenum PNpres PNpres gN (Nat.pair m₀ (Nat.pair n t)).unpair.2.unpair.2.unpair.1
          : Set (ApproximableMap PN PN)) ⊆
          Xenum PNpres PNpres gN (Nat.pair
            (Nat.pair (Nat.pair m₀ (Nat.pair n t)).unpair.2.unpair.1
              (Nat.pair m₀ (Nat.pair n t)).unpair.1) 0 + 1) ∧
        (Nat.pair m₀ (Nat.pair n t)).unpair.1.testBit
          (untagVal (Nat.pair m₀ (Nat.pair n t)).unpair.2.unpair.2.unpair.2) = true
    simp only [unpair_pair_fst, unpair_pair_snd]
    refine ⟨heq, ?_, hbit⟩
    rw [Xenum_singleton PNpres PNpres gN hgN n m₀]
    exact forall_rel_iff_subset_step.mp hf
  · rintro ⟨n, m₀, hbase⟩
    simp only [unpair_pair_fst, unpair_pair_snd] at hbase
    obtain ⟨heq, hsub, hbit⟩ := hbase
    refine ⟨n, m₀, heq, ?_, hbit⟩
    apply forall_rel_iff_subset_step.mpr
    rwa [Xenum_singleton PNpres PNpres gN hgN n m₀] at hsub

theorem graphMap_isComputable (gN incl0 incl1 eq1 : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf PNpres PNpres (decodeList c))
      : Set (ApproximableMap PN PN)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : ∀ s, incl0 s = 1 ↔ PNpres.X s.unpair.1 ⊆ PNpres.X s.unpair.2)
    (hincl0p : Nat.Primrec incl0)
    (hincl1 : ∀ s, incl1 s = 1 ↔ PNpres.X s.unpair.1 ⊆ PNpres.X s.unpair.2)
    (hincl1p : Nat.Primrec incl1)
    (heq1 : ∀ s, eq1 s = 1 ↔ PNpres.X s.unpair.1 = PNpres.X s.unpair.2) (heq1p : Nat.
    Primrec eq1) :
    ComputableMap
      (funPresentation PNpres PNpres gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p hincl1
     hincl1p
        heq1 heq1p)
```

```
      PNpres graphMap := by
  constructor
  have hidx := graphIdx_isComputable gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p hincl1
     hincl1p
    heq1 heq1p
  have hforall : REPred (fun t : ℕ => ∀ j ∈ decodeList t.unpair.2,
      j ∈ GraphIdx (Xenum PNpres PNpres gN t.unpair.1 : Set (ApproximableMap PN PN)))
    :=
    REPred.forall_mem_decodeList₂ hidx
  have hreidx : Nat.Primrec (fun s : ℕ => Nat.pair s.unpair.1 (bitsCode s.unpair.2 s.
    unpair.2)) :=
    Nat.Primrec.left.pair ((primrec_bitsCode.comp (Nat.Primrec.right.pair
      Nat.Primrec.right)).of_eq fun s => by simp only [unpair_pair_fst,
    unpair_pair_snd])
  refine REPred.of_iff (fun s => ?_) (hforall.comp hreidx)
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact (graphMap_rel_iff gN hgN s.unpair.1 s.unpair.2).trans (by
    rw [compl_nbhd_subset_iff]
    exact ⟨fun h j hj => h j (lt_of_testBit_true (mem_bitsCodeList.mp hj))
        (mem_bitsCodeList.mp hj),
      fun h j hjlt hjbit => h j (mem_bitsCodeList.mpr hjbit)⟩)

theorem testBit_imp_mem_Fun_isComputable :
    RecDecidable (fun w : ℕ => w.unpair.2.unpair.2.testBit w.unpair.1 = true →
      w.unpair.1 ∈ Fun (nbhd w.unpair.2.unpair.1.unpair.1)? (nbhd w.unpair.2.unpair.1.
    unpair.2)?) := by
  have hktest : RecDecidable (fun w : ℕ => w.unpair.2.unpair.2.testBit w.unpair.1 =
    true) :=
    ⟨fun w => bitAt w.unpair.2.unpair.2 w.unpair.1,
      (primrec_bitAt.comp ((Nat.Primrec.right.comp Nat.Primrec.right).pair Nat.Primrec
    .left)).of_eq
        fun w => by simp only [unpair_pair_fst, unpair_pair_snd],
      fun w => (bitAt_eq_one_iff _ _).symm⟩
  have hmem2 : RecDecidable (fun w : ℕ =>
      w.unpair.1 ∈ Fun (nbhd w.unpair.2.unpair.1.unpair.1)? (nbhd w.unpair.2.unpair.1.
    unpair.2)?) :=
    RecDecidable.of_iff (fun w => by simp only [unpair_pair_fst, unpair_pair_snd])
      (mem_Fun_compl_nbhd_isComputable.comp
        ((Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
          (Nat.Primrec.left.pair (Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.
    Primrec.right)))))
  refine RecDecidable.of_iff (fun w => ?_) (hktest.not.or hmem2)
  constructor
  · intro h
    rcases hktest.em w with hb | hb
    · exact Or.inr (h hb)
    · exact Or.inl hb
  · rintro (hb | hb) hbit
    · exact absurd hbit hb
    · exact hb

theorem gMap_isComputable :
    ComputableMap (prodPresentation PNpres PNpres) PNpres gMap := by
  constructor
  have hall : RecDecidable (fun u : ℕ => ∀ j, j < u.unpair.2 →
      u.unpair.2.testBit j = true →
```

```
        j ∈ Fun (nbhd u.unpair.1.unpair.1)? (nbhd u.unpair.1.unpair.2)?) :=
    RecDecidable.of_iff (fun u => by simp only [unpair_pair_fst, unpair_pair_snd])
      (testBit_imp_mem_Fun_isComputable.bForall Nat.Primrec.right)
  refine (RecDecidable.of_iff (fun t => ?_) hall).re
  simp only [prodPresentation_X]
  exact (gMap_rel_iff _ _ _).trans (compl_nbhd_subset_Fun_iff _ _ _)

theorem funMap_isComputable (gN incl0 incl1 eq1 : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf PNpres PNpres (decodeList c))
      : Set (ApproximableMap PN PN)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : ∀ s, incl0 s = 1 ↔ PNpres.X s.unpair.1 ⊆ PNpres.X s.unpair.2)
    (hincl0p : Nat.Primrec incl0)
    (hincl1 : ∀ s, incl1 s = 1 ↔ PNpres.X s.unpair.1 ⊆ PNpres.X s.unpair.2)
    (hincl1p : Nat.Primrec incl1)
    (heq1 : ∀ s, eq1 s = 1 ↔ PNpres.X s.unpair.1 = PNpres.X s.unpair.2) (heq1p : Nat.
    Primrec eq1) :
    ComputableMap PNpres


      (funPresentation PNpres PNpres gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p hincl1
     hincl1p
        heq1 heq1p)
      funMap :=
  curry_isComputable PNpres PNpres PNpres gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p
    hincl1
    hincl1p heq1 heq1p gMap_isComputable

end DomainTheory.NeighbourhoodSystems.GraphFun
```

4.8.57 Exercise 7.23 — graph and fun computable

Complete the discussion of $P\mathbf{N}$ of Example 7.8. Show that the combinators fun and graph of Exercise 5.14 are computable. Also do the same for

$$\lambda x, y.\, x \cap y, \quad \lambda x, y.\, x \cup y, \quad \text{and } \lambda x, y.\, x + y,$$

where for $x, y \in P\mathbf{N}$ we define

$$x + y = \{n + m \mid n \in x \text{ and } m \in y\}.$$

What are the computable elements of $P\mathbf{N}$?

Exercise_7_23.lean

@import “Exercise_7_23.lean”

The development is with the graph map peer.

Theorem (`example_723`). The graph of the powerset endomap is a computable map into the powerset.

Proof. By the graph map. ■

Exercise_7_23.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .graphFun
```

```
@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Domain.Recursive
open DomainTheory.NeighbourhoodSystems.GraphFun
/-- Scott's Exercise 7.23: the graph map is computable. -/
theorem example_723 (gN incl0 incl1 eq1 : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf PNpres PNpres (decodeList c))
      : Set (ApproximableMap PN PN)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : ∀ s, incl0 s = 1 ↔ PNpres.X s.unpair.1 ⊆ PNpres.X s.unpair.2)
    (hincl0p : Nat.Primrec incl0)
    (hincl1 : ∀ s, incl1 s = 1 ↔ PNpres.X s.unpair.1 ⊆ PNpres.X s.unpair.2)
    (hincl1p : Nat.Primrec incl1)
    (heq1 : ∀ s, eq1 s = 1 ↔ PNpres.X s.unpair.1 = PNpres.X s.unpair.2) (heq1p : Nat.
   Primrec eq1) :
    ComputableMap
      (funPresentation PNpres PNpres gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p hincl1
     hincl1p
        heq1 heq1p)
      PNpres graphMap :=
  graphMap_isComputable gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p hincl1 hincl1p heq1
    heq1p
```

4.8.58 LUCID histories

The domain of histories is effectively given, and the lifted connectives are computable.

Definition (`Gamma`). A history is either a finite list of natural numbers or an infinite stream of natural numbers.

Definition (`star`). The empty finite history is the distinguished initial history.

Definition (`streamCons`). Prefixing a number to a stream yields the stream whose first value is that number and whose later values are the original stream shifted by one place.

Theorem (`streamCons_zero`). At input zero, stream cons has the initial value specified by its definition.

Proof. The relevant clause of the recursive definition of stream cons zero gives this equation immediately. ■

Theorem (`streamCons_succ`). At a successor input, stream cons succ satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of stream cons succ gives this equation immediately. ■

Definition (`cons`). Prefixing a number to a history acts by list cons on finite histories and by stream cons on infinite histories.

Definition (`consSet`). The prefixed image of a set of histories consists of all histories obtained by prefixing the fixed number to a member of the set.

Theorem (`cons_injective`). The construction cons injective sends equal outputs only from equal inputs.

Proof. Equality of two values of cons injective determines their outer form and then each component, so the original inputs coincide. ■

Theorem (`mem_consSet`). The element described by cons set belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for cons set is supplied directly by the defining membership condition. ■

Theorem (`consSet_empty`). The set or language in cons set empty is empty exactly under the stated absence-of-witness condition.

Proof. An element of the set in cons set empty would provide exactly the forbidden witness, while any such witness produces an element; hence emptiness is equivalent to its absence. ■

Theorem (`consSet_inter`). The construction cons set inter preserves the specified intersection of sets or neighbourhoods.

Proof. Use the same witnesses in both directions for cons set inter, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`consSet_inter_ne`). The construction cons set inter ne preserves the specified intersection of sets or neighbourhoods.

Proof. Use the same witnesses in both directions for cons set inter ne, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`consSet_nonempty`). The set or language cons set nonempty has an element under the hypotheses of the theorem.

Proof. Choose witnesses from the nonempty input sets in cons set nonempty and combine them using the defining construction. ■

Theorem (`consSet_subset_of_subset`). The hypotheses subset imply cons set subset.

Proof. Following the defining recursion for cons set subset of subset and applying the preceding component lemmas yields the claimed result. ■

Theorem (`consSet_subset`). For the objects in cons set subset, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for cons set subset and applying the preceding component lemmas yields the claimed result. ■

Definition (`nbhd`). The neighbourhood determined by a finite list is the set of histories having that list as an initial segment.

Theorem (`nbhd_nil`). For the objects in nbhd nil, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for nbhd nil and applying the preceding component lemmas yields the claimed result. ■

Theorem (`nbhd_cons`). For the objects in nbhd cons, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for nbhd cons and applying the preceding component lemmas yields the claimed result. ■

Definition (`Lmem`). A set is an admissible history neighbourhood exactly when it is determined by some finite list.

Theorem (`mem_nbhd`). The element described by nbhd belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for nbhd is supplied directly by the defining membership condition. ■

Theorem (`L_nestedOrDisjoint`). The two objects compared in l nested or disjoint are distinct.

Proof. Following the defining recursion for l nested or disjoint and applying the preceding component lemmas yields the claimed result. ■

Definition (`L`). The history neighbourhood system has all histories as master set and the finite-prefix neighbourhoods as its neighbourhoods.

Definition (`Lenum`). The neighbourhood with index n is the prefix neighbourhood determined by the decoded list of n.

Theorem (`Lenum_mem`). For the objects in lenum mem, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for lenum mem and applying the preceding component lemmas yields the claimed result. ■

Theorem (`Lenum_surj`). Every object in the target of lenum surj is represented by some input.

Proof. Choose the finite code or canonical representative supplied by the target object; decoding that choice gives the required preimage for lenum surj. ■

Theorem (`Lenum_zero`). At input zero, lenum has the initial value specified by its definition.

Proof. The relevant clause of the recursive definition of lenum zero gives this equation immediately. ■

Definition (`isPrefixChar`). The numerical characteristic function is prefix char encodes its Boolean or relational test by zero and one.

Definition (`LenumConsChar`). The numerical characteristic function lenum cons char encodes its Boolean or relational test by zero and one.

Definition (`LenumConsCharP`). The numerical characteristic function lenum cons char p encodes its Boolean or relational test by zero and one.

Definition (`LenumInter`). For two neighbourhood indices, choose the index of the longer compatible prefix; when they are incompatible, choose the second index.

Definition (`LenumInterEqChar`). The numerical characteristic function lenum inter eq char encodes its Boolean or relational test by zero and one.

Definition (`LenumInterEqCharP`). The numerical characteristic function lenum inter eq char p encodes its Boolean or relational test by zero and one.

Theorem (`isPrefixChar_eq_one_iff`). The zero–one test is prefix char returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in is prefix char eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`isPrefixChar_le_one`). For every input, is prefix char takes one of the two values zero and one.

Proof. Each Boolean-valued component of is prefix char le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`isPrefixChar_eq_zero_or_one`). For every input, the characteristic function is prefix char is either zero or one.

Proof. The relevant clause of the recursive definition of is prefix char eq zero or one gives this equation immediately. ■

Theorem (`LenumConsChar_eq_one_iff`). The zero–one test lenum cons char returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in lenum cons char eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`LenumConsChar_le_one`). For every input, lenum cons char takes one of the two values zero and one.

Proof. Each Boolean-valued component of lenum cons char le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`LenumConsCharP_le_one`). For every input, lenum cons char p takes one of the two values zero and one.

Proof. Each Boolean-valued component of lenum cons char p le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`mulBit_eq_zero_or_one`). For every input, the characteristic function mul bit is either zero or one.

Proof. The relevant clause of the recursive definition of mul bit eq zero or one gives this equation immediately. ■

Theorem (`LenumInterEqChar_eq_one_iff`). The zero–one test lenum inter eq char returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in lenum inter eq char eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`LenumInterEqChar_le_one`). For every input, lenum inter eq char takes one of the two values zero and one.

Proof. Each Boolean-valued component of lenum inter eq char le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`LenumInterEqCharP_le_one`). For every input, lenum inter eq char p takes one of the two values zero and one.

Proof. Each Boolean-valued component of lenum inter eq char p le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`nbhd_nonempty`). The set or language nbhd nonempty has an element under the hypotheses of the theorem.

Proof. Choose witnesses from the nonempty input sets in nbhd nonempty and combine them using the defining construction. ■

Theorem (`primrec_isPrefixChar`). The numerical function is prefix char is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of is prefix char. ■

Theorem (`primrec_LenumInterEqChar`). The numerical function lenum inter eq char is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of lenum inter eq char. ■

Theorem (`primrec_LenumInterEqCharP`). The numerical function lenum inter eq char p is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of lenum inter eq char p. ■

Theorem (`primrec_isPrefixChar_swap`). The numerical function is prefix char swap is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of is prefix char swap. ■

Theorem (`primrec_LenumConsChar`). The numerical function lenum cons char is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of lenum cons char. ■

Theorem (`primrec_LenumConsCharP`). The numerical function lenum cons char p is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of lenum cons char p. ■

Theorem (`primrec_LenumInter`). The numerical function lenum inter is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of lenum inter. ■

Theorem (`star_ne_cons`). The two objects compared in star ne cons are distinct.

Proof. Following the defining recursion for star ne cons and applying the preceding component lemmas yields the claimed result. ■

Theorem (`star_not_mem_consSet`). The two objects compared in star not mem cons set are distinct.

Proof. Following the defining recursion for star not mem cons set and applying the preceding component lemmas yields the claimed result. ■

Theorem (`nbhd_subset_iff`). The inclusion in nbhd subset iff holds exactly when the finite prefix, complement, or numerical condition on the right holds.

Proof. Translate both inclusions in nbhd subset iff into the relevant finite prefix or complement condition, and use antisymmetry for the converse direction. ■

Theorem (`Lenum_subset_iff`). The inclusion in lenum subset iff holds exactly when the finite prefix, complement, or numerical condition on the right holds.

Proof. Translate both inclusions in lenum subset iff into the relevant finite prefix or complement condition, and use antisymmetry for the converse direction. ■

Theorem (`Lenum_cons_iff`). The condition lenum cons holds exactly when the finite witness condition holds.

Proof. Expand both conditions in lenum cons iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`LenumInter_spec`). The construction lenum inter spec preserves the specified intersection of sets or neighbourhoods.

Proof. Use the same witnesses in both directions for lenum inter spec, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`Lenum_cons_charP_eq_one_iff`). The zero–one test lenum cons char p returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in lenum cons char p eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`Lenum_cons_computable`). The relation lenum cons computable is recursively decidable.

Proof. Reduce the relation for lenum cons computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`nbhd_injective`). The construction nbhd injective sends equal outputs only from equal inputs.

Proof. Equality of two values of nbhd injective determines their outer form and then each component, so the original inputs coincide. ■

Theorem (`Lenum_interEq_iff`). The intersection in lenum inter eq iff equals the indexed neighbourhood or language exactly under the stated finite comparison condition.

Proof. Use the same witnesses in both directions for lenum inter eq iff, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`Lenum_interEq_charP_eq_one_iff`). The zero–one test lenum inter eq char p returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in lenum inter eq char p eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`Lenum_interEq_computable`). The relation lenum inter eq computable is recursively decidable.

Proof. Use the same witnesses in both directions for lenum inter eq computable, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Definition (`Lpres`). The lpres is the computable presentation whose enumeration and decision procedures are those constructed above.

Definition (`LFunData`). Function-space presentation data consists of numerical tests for inhabited step families, inclusion in each presentation, and equality in the codomain presentation, together with primitive-recursiveness proofs.

Theorem (`cons_inl`). For the objects in cons inl, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for cons inl and applying the preceding component lemmas yields the claimed result. ■

Theorem (`cons_inr`). For the objects in cons inr, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for cons inr and applying the preceding component lemmas yields the claimed result. ■

Theorem (`L_mem`). For the objects in l mem, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for l mem and applying the preceding component lemmas yields the claimed result. ■

Theorem (`L_master`). The master neighbourhood in l master is the universal set and satisfies the claimed relation.

Proof. Following the defining recursion for l master and applying the preceding component lemmas yields the claimed result. ■

Definition (`LenumSubsetChar`). The numerical characteristic function lenum subset char encodes its Boolean or relational test by zero and one.

Definition (`finPrefix`). The finite prefix of length n of a stream is the list of its values at positions below n.

Theorem (`finPrefix_zero`). The prefix of length zero of every stream is the empty list.

Proof. The relevant clause of the recursive definition of fin prefix zero gives this equation immediately. ■

Definition (`HasMin`). A history element has a minimum when one of its neighbourhoods is contained in every neighbourhood belonging to that element.

Definition (`bitToNat`). A Boolean value is encoded by zero for false and one for true.

Theorem (`bitToNat_false`). For the objects in bit to nat false, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for bit to nat false and applying the preceding component lemmas yields the claimed result. ■

Theorem (`bitToNat_true`). For the objects in bit to nat true, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for bit to nat true and applying the preceding component lemmas yields the claimed result. ■

Definition (`embStr`). A finite Boolean string is embedded as the list obtained by replacing false by zero and true by one.

Theorem (`embStr_nil`). For the objects in emb str nil, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for emb str nil and applying the preceding component lemmas yields the claimed result. ■

Definition (`Tenum`). The three canonical truth neighbourhoods are enumerated by the universal truth set, the false singleton, and the true singleton, with all later indices returning the universal set.

Theorem (`Tenum_zero`). At input zero, tenum has the initial value specified by its definition.

Proof. The relevant clause of the recursive definition of tenum zero gives this equation immediately. ■

Theorem (`Tenum_one`). At input one, tenum has the value specified by its definition.

Proof. The relevant clause of the recursive definition of tenum one gives this equation immediately. ■

Theorem (`Tenum_two`). At input two, tenum has the value specified by its definition.

Proof. The relevant clause of the recursive definition of tenum two gives this equation immediately. ■

Theorem (`Tenum_eq_canon`). The value of tenum equals the value described by canon.

Proof. Compare membership or numerical components in tenum eq canon; the defining equations reduce both sides to the same data. ■

Theorem (`Tenum_mem`). For the objects in tenum mem, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for tenum mem and applying the preceding component lemmas yields the claimed result. ■

Theorem (`Tenum_ne_empty`). The set or language in tenum ne empty is empty exactly under the stated absence-of-witness condition.

Proof. An element of the set in tenum ne empty would provide exactly the forbidden witness, while any such witness produces an element; hence emptiness is equivalent to its absence. ■

Theorem (`Tenum_surj`). Every object in the target of tenum surj is represented by some input.

Proof. Choose the finite code or canonical representative supplied by the target object; decoding that choice gives the required preimage for tenum surj. ■

Definition (`TeqChar`). The numerical characteristic function teq char encodes its Boolean or relational test by zero and one.

Definition (`TinterCode`). The coding function tinter code assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`andCode`). The coding function and code assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`andFn`). Conjunction on truth neighbourhoods returns its second argument on the false neighbourhood and the universal neighbourhood otherwise.

Theorem (`andFn_zero`). At input zero, and fn has the initial value specified by its definition.

Proof. The relevant clause of the recursive definition of and fn zero gives this equation immediately. ■

Theorem (`eq_master_of_master_subset`). The master neighbourhood in eq master of master subset is the universal set and satisfies the claimed relation.

Proof. Following the defining recursion for eq master of master subset and applying the preceding component lemmas yields the claimed result. ■

Theorem (`exists_cons_prefix_of_prefix_of_length_lt`). A proper finite prefix can be extended by one entry while remaining a prefix of the longer list.

Proof. The standard length laws for ranges, maps, append, and singleton lists compute the length in cons prefix of prefix of length lt. ■

Theorem (`exists_extend_of_not_hasMin`). The hypotheses not has min imply exists extend.

Proof. Following the defining recursion for extend of not has min and applying the preceding component lemmas yields the claimed result. ■

Definition (`buildData`). For a nonprincipal history element, recursively choose an increasing sequence of finite prefixes whose neighbourhoods belong to the element.

Theorem (`finPrefix_length`). The prefix of length n of a stream has length n.

Proof. The standard length laws for ranges, maps, append, and singleton lists compute the length in fin prefix length. ■

Theorem (`finPrefix_succ`). The prefix of length n+1 is the prefix of length n followed by the nth stream entry.

Proof. The relevant clause of the recursive definition of fin prefix succ gives this equation immediately. ■

Theorem (`finPrefix_prefix_succ`). Every finite stream prefix is a prefix of the next longer finite stream prefix.

Proof. The relevant clause of the recursive definition of fin prefix prefix succ gives this equation immediately. ■

Theorem (`finPrefix_prefix_of_le`). If n is at most m, the prefix of length n is a prefix of the prefix of length m.

Proof. Following the defining recursion for fin prefix prefix of le and applying the preceding component lemmas yields the claimed result. ■

Theorem (`finPrefix_take`). Taking the first n entries of the prefix of length m, when n is at most m, gives the prefix of length n.

Proof. Following the defining recursion for fin prefix take and applying the preceding component lemmas yields the claimed result. ■

Theorem (`master_ne_one_T`). The master neighbourhood in master ne one t is the universal set and satisfies the claimed relation.

Proof. Following the defining recursion for master ne one t and applying the preceding component lemmas yields the claimed result. ■

Theorem (`master_ne_zero_T`). The master neighbourhood in master ne zero t is the universal set and satisfies the claimed relation.

Proof. Following the defining recursion for master ne zero t and applying the preceding component lemmas yields the claimed result. ■

Theorem (`andFn_master`). The master neighbourhood in and fn master is the universal set and satisfies the claimed relation.

Proof. Following the defining recursion for and fn master and applying the preceding component lemmas yields the claimed result. ■

Theorem (`nbhd_finPrefix_antitone`). Longer finite stream prefixes determine smaller neighbourhoods.

Proof. Following the defining recursion for nbhd fin prefix antitone and applying the preceding component lemmas yields the claimed result. ■

Definition (`notCode`). The coding function not code assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`notFn`). Negation fixes the universal truth neighbourhood and exchanges the false and true singleton neighbourhoods.

Theorem (`notFn_master`). The master neighbourhood in not fn master is the universal set and satisfies the claimed relation.

Proof. Following the defining recursion for not fn master and applying the preceding component lemmas yields the claimed result. ■

Theorem (`notFn_zero`). At input zero, not fn has the initial value specified by its definition.

Proof. The relevant clause of the recursive definition of not fn zero gives this equation immediately. ■

Definition (`LFunData.ofPresentation`). The of presentation is the computable presentation whose enumeration and decision procedures are those constructed above.

Theorem (`one_inter_zero_T`). The construction one inter zero t preserves the specified intersection of sets or neighbourhoods.

Proof. Use the same witnesses in both directions for one inter zero t, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Definition (`postcompose`). Postcomposition sends a function element to its composite with the fixed approximable map.

Definition (`LFunData.pres`). The pres is the computable presentation whose enumeration and decision procedures are those constructed above.

Theorem (`primrec_TeqChar`). The numerical function teq char is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of teq char. ■

Theorem (`primrec_TinterCode`). The numerical function tinter code is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of tinter code. ■

Theorem (`principal_hasMin`). For the objects in principal has min, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for principal has min and applying the preceding component lemmas yields the claimed result. ■

Definition (`pwArg`). The pointwise binary argument map rearranges a function and an input pair into the pair of arguments required for evaluation.

Definition (`pwFst`). The first pointwise projection evaluates the first function at the supplied argument.

Definition (`pwSnd`). The second pointwise projection evaluates the second function at the supplied argument.

Definition (`pointwiseBin`). A binary operation on the codomain lifts pointwise to a binary operation on function elements by evaluating both functions and then applying the operation.

Definition (`streamElement`). The element stream element is determined by the admissible neighbourhoods satisfying its defining membership condition.

Theorem (`finPrefix_eq_of_streamElement_eq`). Equal stream elements have equal finite prefixes at every length.

Proof. Compare membership or numerical components in fin prefix eq of stream element eq; the defining equations reduce both sides to the same data. ■

Theorem (`streamElement_injective`). The construction stream element injective sends equal outputs only from equal inputs.

Proof. Equality of two values of stream element injective determines their outer form and then each component, so the original inputs coincide. ■

Theorem (`streamElement_not_hasMin`). The two objects compared in stream element not has min are distinct.

Proof. Following the defining recursion for stream element not has min and applying the preceding component lemmas yields the claimed result. ■

Theorem (`subset_iff_eq_or_eq_master`). The inclusion in subset iff eq or eq master holds exactly when the finite prefix, complement, or numerical condition on the right holds.

Proof. Translate both inclusions in subset iff eq or eq master into the relevant finite prefix or complement condition, and use antisymmetry for the converse direction. ■

Definition (`toElement`). The element to element is determined by the admissible neighbourhoods satisfying its defining membership condition.

Theorem (`toElement_injective`). The construction to element injective sends equal outputs only from equal inputs.

Proof. Equality of two values of to element injective determines their outer form and then each component, so the original inputs coincide. ■

Definition (`toStream`). A nonprincipal history element determines the stream whose finite prefixes are selected by its recursively constructed prefix chain.

Theorem (`buildData_succ`). At a successor input, build data succ satisfies its recursive step equation.

Proof. The relevant clause of the recursive definition of build data succ gives this equation immediately. ■

Theorem (`buildData_eq_finPrefix`). The list selected at stage n of the nonminimal-element construction is the length-n prefix of its selected stream.

Proof. Compare membership or numerical components in build data eq fin prefix; the defining equations reduce both sides to the same data. ■

Theorem (`mem_nbhd_finPrefix_toStream`). The element described by nbhd fin prefix to stream belongs to the set or neighbourhood specified by its hypotheses.

Proof. The required witness for nbhd fin prefix to stream is supplied directly by the defining membership condition. ■

Theorem (`exists_finPrefix_subset_of_mem`). Every neighbourhood of a nonminimal history element contains a finite-prefix neighbourhood of its selected stream.

Proof. Following the defining recursion for fin prefix subset of mem and applying the preceding component lemmas yields the claimed result. ■

Theorem (`toElement_surjective`). Every object in the target of to element surjective is represented by some input.

Proof. Choose the finite code or canonical representative supplied by the target object; decoding that choice gives the required preimage for to element surjective. ■

Theorem (`toElement_bijective`). The construction to element bijective is both injective and surjective.

Proof. Combine the injectivity and surjectivity results already proved for to element bijective. ■

Definition (`gammaEquivElement`). The element gamma equiv element is determined by the admissible neighbourhoods satisfying its defining membership condition.

Theorem (`zero_inter_one_T`). The construction zero inter one t preserves the specified intersection of sets or neighbourhoods.

Proof. Use the same witnesses in both directions for zero inter one t, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`Tenum_inter_eq`). The intersection in tenum inter eq equals the indexed neighbourhood or language exactly under the stated finite comparison condition.

Proof. Use the same witnesses in both directions for tenum inter eq, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`Tenum_inter_eq_TinterCode'`). The intersection in tenum inter eq tinter code' equals the indexed neighbourhood or language exactly under the stated finite comparison condition.

Proof. Use the same witnesses in both directions for tenum inter eq tinter code', combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`Tenum_inter_eq_TinterCode`). The intersection in tenum inter eq tinter code equals the indexed neighbourhood or language exactly under the stated finite comparison condition.

Proof. Use the same witnesses in both directions for tenum inter eq tinter code, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`Tpres_cons_iff`). The condition tpres cons holds exactly when the finite witness condition holds.

Proof. Expand both conditions in tpres cons iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`Tpres_cons_computable`). The relation tpres cons computable is recursively decidable.

Proof. Reduce the relation for tpres cons computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`zero_ne_one_T`). The two objects compared in zero ne one t are distinct.

Proof. Following the defining recursion for zero ne one t and applying the preceding component lemmas yields the claimed result. ■

Theorem (`Tenum_injOn`). The construction tenum inj on sends equal outputs only from equal inputs.

Proof. Following the defining recursion for tenum inj on and applying the preceding component lemmas yields the claimed result. ■

Theorem (`Tenum_interEq_iff`). The intersection in tenum inter eq iff equals the indexed neighbourhood or language exactly under the stated finite comparison condition.

Proof. Use the same witnesses in both directions for tenum inter eq iff, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`Tenum_interEq_iff'`). The intersection in tenum inter eq iff' equals the indexed neighbourhood or language exactly under the stated finite comparison condition.

Proof. Use the same witnesses in both directions for tenum inter eq iff', combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Theorem (`Tpres_interEq_computable`). The relation tpres inter eq computable is recursively decidable.

Proof. Use the same witnesses in both directions for tpres inter eq computable, combining their inclusion proofs pointwise and using nonemptiness where intersection closure requires it. ■

Definition (`Tpres`). The tpres is the computable presentation whose enumeration and decision procedures are those constructed above.

Definition (`LTdata`). The function-space presentation from histories to truth values is assembled from the effective presentations of the history and truth domains.

Definition (`LTpres`). The ltpres is the computable presentation whose enumeration and decision procedures are those constructed above.

Theorem (`andFn_one`). At input one, and fn has the value specified by its definition.

Proof. The relevant clause of the recursive definition of and fn one gives this equation immediately. ■

Theorem (`andFn_mem`). For the objects in and fn mem, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for and fn mem and applying the preceding component lemmas yields the claimed result. ■

Definition (`andMap2`). The map and map2 relates basic source and target neighbourhoods by the mathematical operation expressed in its defining relation.

Definition (`andT`). Truth conjunction is the approximable map induced by conjunction on truth neighbourhoods.

Theorem (`notFn_one`). At input one, not fn has the value specified by its definition.

Proof. The relevant clause of the recursive definition of not fn one gives this equation immediately. ■

Theorem (`notFn_mem`). For the objects in not fn mem, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for not fn mem and applying the preceding component lemmas yields the claimed result. ■

Definition (`notT`). Truth negation is the approximable map induced by negation on truth neighbourhoods.

Theorem (`bitToNat_injective`). The construction bit to nat injective sends equal outputs only from equal inputs.

Proof. Equality of two values of bit to nat injective determines their outer form and then each component, so the original inputs coincide. ■

Theorem (`embStr_cons`). For the objects in emb str cons, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for emb str cons and applying the preceding component lemmas yields the claimed result. ■

Theorem (`embStr_prefix_iff`). One Boolean string is a prefix of another exactly when its numerical embedding is a prefix of the other embedding.

Proof. Expand both conditions in emb str prefix iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`cone_subset_cone_iff_nbhd_embStr`). The condition cone subset cone holds exactly when nbhd emb str holds.

Proof. Expand both conditions in cone subset cone iff nbhd emb str; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`sigmaBot_le_iff_toElement_inl_embStr`). The condition sigma bot le holds exactly when to element inl emb str holds.

Proof. Expand both conditions in sigma bot le iff to element inl emb str; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`primrec_notCode`). The numerical function not code is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of not code. ■

Theorem (`notFn_Tenum`). For the objects in not fn tenum, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for not fn tenum and applying the preceding component lemmas yields the claimed result. ■

Theorem (`notT_isComputable`). The map or element not t is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for not t is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`primrec_andCode`). The numerical function and code is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of and code. ■

Theorem (`andFn_Tenum`). For the objects in and fn tenum, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for and fn tenum and applying the preceding component lemmas yields the claimed result. ■

Theorem (`andT_isComputable`). The map or element and t is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for and t is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`postcompose_isComputable`). The map or element postcompose is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for postcompose is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`pointwiseBin_isComputable`). The map or element pointwise bin is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for pointwise bin is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`notT_lifted_isComputable`). The map or element not t lifted is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for not t lifted is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`andT_lifted_isComputable`). The map or element and t lifted is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for and t lifted is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`deMorganT_isComputable`). The map or element de morgan t is computable is computable with respect to the presentations appearing in its domain and codomain.

Proof. Reduce the relation for de morgan t is computable to the previously constructed primitive-recursive characteristic functions and close recursive decidability under the required Boolean operations. ■

Theorem (`Gamma_cases`). For the objects in gamma cases, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for gamma cases and applying the preceding component lemmas yields the claimed result. ■

Theorem (`L_isEffectivelyGiven`). For the objects in l is effectively given, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for l is effectively given and applying the preceding component lemmas yields the claimed result. ■

Theorem (`LenumSubsetChar_le_one`). For every input, lenum subset char takes one of the two values zero and one.

Proof. Each Boolean-valued component of lenum subset char le one is zero or one, and the defining Boolean combination preserves that range. ■

Theorem (`Lenum_subset_char_eq_one_iff`). The zero–one test lenum subset char returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in lenum subset char eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`T_isEffectivelyGiven`). For the objects in t is effectively given, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for t is effectively given and applying the preceding component lemmas yields the claimed result. ■

Theorem (`TeqChar_eq_one_iff`). The zero–one test teq char returns one exactly when the mathematical condition encoded in its definition holds.

Proof. Separate the possible zero–one values occurring in teq char eq one iff; the value one survives precisely in the case described by the defining predicate. ■

Theorem (`embStr_injective`). The construction emb str injective sends equal outputs only from equal inputs.

Proof. Equality of two values of emb str injective determines their outer form and then each component, so the original inputs coincide. ■

Theorem (`primrec_LenumSubsetChar`). The numerical function lenum subset char is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of lenum subset char. ■

Theorem (`Lenum_subset_recDecidable`). The relation lenum subset rec decidable is recursively decidable.

Proof. Following the defining recursion for lenum subset rec decidable and applying the preceding component lemmas yields the claimed result. ■

Definition (`codePair`). The coding function code pair assigns its finite list, expression, pair, or tagged value a natural-number index by pairing and list coding.

Definition (`chainDec`). A decoded finite list is an admissible fixed-point chain when its first member is the bottom code and each later member is included in the image of its predecessor.

Definition (`fixPStep`). The state transformer fix pstep performs one stage of the finite numerical computation encoded in its state.

Definition (`fixStp`). One fixed-point search step updates the accumulated chain code and current candidate according to the inclusion characteristic.

Definition (`fixChainChar`). The numerical characteristic function fix chain char encodes its Boolean or relational test by zero and one.

Definition (`gLastOf`). The last element of a nonempty coded chain is returned, with zero used for the empty chain.

Definition (`gStepsOK`). The state transformer g steps ok performs one stage of the finite numerical computation encoded in its state.

Theorem (`Xenum_codePair`). For the objects in xenum code pair, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for xenum code pair and applying the preceding component lemmas yields the claimed result. ■

Theorem (`fixPStep_foldl_fst`). For the objects in fix pstep foldl fst, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for fix pstep foldl fst and applying the preceding component lemmas yields the claimed result. ■

Theorem (`fixPStep_foldl_snd`). For the objects in fix pstep foldl snd, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for fix pstep foldl snd and applying the preceding component lemmas yields the claimed result. ■

Theorem (`fixStp_eq`). The value of fix stp eq equals the value described by fix stp eq.

Proof. Compare membership or numerical components in fix stp eq; the defining equations reduce both sides to the same data. ■

Theorem (`fixChainChar_eq`). The value of fix chain char eq equals the value described by fix chain char eq.

Proof. Compare membership or numerical components in fix chain char eq; the defining equations reduce both sides to the same data. ■

Theorem (`fixChainChar_spec`). The numerical construction fix chain char spec computes exactly the finite object described by its specification.

Proof. Induction over the bounded iteration for fix chain char spec maintains the recorded accumulator invariant and yields the claimed final components. ■

Theorem (`gStepsOK_complete`). Every semantic relation in g steps ok complete has a finite witness of the prescribed form.

Proof. Induction over the semantic iteration for g steps ok complete selects an index for each intermediate neighbourhood and assembles the finite witness. ■

Theorem (`gStepsOK_sound`). Every finite witness accepted by g steps ok sound determines a valid semantic relation.

Proof. Induction over the finite witness for g steps ok sound composes each valid step, producing the semantic relation at its endpoint. ■

Theorem (`fixElement_mem_iff_chain`). Membership in fix element mem iff chain is equivalent to the explicit witness condition given by the construction.

Proof. Expanding the membership definition of fix element mem iff chain exposes exactly the witnesses and equations occurring on the other side. ■

Theorem (`fixMap_rel_iff`). The condition fix map rel holds exactly when the finite witness condition holds.

Proof. Expand both conditions in fix map rel iff; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`leastMap_Xenum_rel`). For the objects in least map xenum rel, the named equality, inclusion, or finite-witness relation holds.

Proof. Following the defining recursion for least map xenum rel and applying the preceding component lemmas yields the claimed result. ■

Theorem (`chainDec_iff_gStepsOK`). The condition chain dec holds exactly when g steps ok holds.

Proof. Expand both conditions in chain dec iff g steps ok; their witnesses correspond in both directions by the defining equations and the established coding identities. ■

Theorem (`primrec_fixStp`). The numerical function fix stp is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of fix stp. ■

Theorem (`primrec_fixChainChar`). The numerical function fix chain char is primitive recursive.

Proof. Primitive recursion and closure under composition, pairing, bounded search, and the previously established component functions give primitive recursiveness of fix chain char. ■

lucidHistory.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_3
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    eq_principal_of_isMin
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    sigmaBot_le_iff
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_injective
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint
import Mathlib.Tactic.IntervalCases

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.LucidHistory
open NeighborhoodSystem Domain.Recursive

abbrev Gamma : Type := List ℕ ? (ℕ → ℕ)

def star : Gamma := Sum.inl []

def streamCons (i : ℕ) (s : ℕ → ℕ) : ℕ → ℕ
  | 0 => i
  | n + 1 => s n

@[simp] theorem streamCons_zero (i : ℕ) (s : ℕ → ℕ) : streamCons i s 0 = i := rfl

@[simp] theorem streamCons_succ (i : ℕ) (s : ℕ → ℕ) (n : ℕ) : streamCons i s (n + 1)
    = s n := rfl

def cons (i : ℕ) : Gamma → Gamma
  | .inl l => .inl (i :: l)
  | .inr s => .inr (streamCons i s)

def consSet (i : ℕ) (X : Set Gamma) : Set Gamma := {z | ∃ ? ∈ X, z = cons i ?}

theorem cons_injective {i j : ℕ} {? ? : Gamma} (h : cons i ? = cons j ?) : i = j ∧ ? =
     ? := by
```

```
  cases ? with
  | inl l =>
    cases ? with
    | inl l' =>
      simp only [cons, Sum.inl.injEq, List.cons.injEq] at h
      exact ⟨h.1, by rw [h.2]⟩
    | inr s' => simp [cons] at h
  | inr s =>
    cases ? with
    | inl l' => simp [cons] at h
    | inr s' =>
      have heq : streamCons i s = streamCons j s' := by simpa [cons] using h
      have hi : i = j := by simpa using congrFun heq 0
      refine ⟨hi, ?_⟩
      have hs : s = s' := funext fun n => by simpa using congrFun heq (n + 1)
      rw [hs]

@[simp] theorem mem_consSet {i : ℕ} {X : Set Gamma} {z : Gamma} :
    z ∈ consSet i X ↔ ∃ ? ∈ X, z = cons i ? := Iff.rfl

@[simp] theorem consSet_empty (i : ℕ) : consSet i (∅ : Set Gamma) = ∅ := by
  ext z; simp [mem_consSet]

theorem consSet_inter (i : ℕ) (X X' : Set Gamma) :
    consSet i X ∩ consSet i X' = consSet i (X ∩ X') := by
  ext z
  simp only [Set.mem_inter_iff, mem_consSet]
  constructor
  · rintro ⟨⟨?, h?, rfl⟩, ?', h?', heq⟩
    obtain ⟨-, h??'⟩ := cons_injective heq
    exact ⟨?, ⟨h?, h??' ▸ h?'⟩, rfl⟩
  · rintro ⟨?, ⟨h?, h?'⟩, rfl⟩
    exact ⟨⟨?, h?, rfl⟩, ⟨?, h?', rfl⟩⟩

theorem consSet_inter_ne {i j : ℕ} (h : i = j) (X Y : Set Gamma) :
    consSet i X ∩ consSet j Y = ∅ := by
  ext z
  simp only [Set.mem_inter_iff, mem_consSet, Set.mem_empty_iff_false, iff_false,
    not_and]
  rintro ⟨?, -, rfl⟩ ⟨?', -, heq⟩
  exact h (cons_injective heq).1

theorem consSet_nonempty {i : ℕ} {X : Set Gamma} (h : X.Nonempty) : (consSet i X).
    Nonempty := by
  obtain ⟨?, h?⟩ := h; exact ⟨cons i ?, ?, h?, rfl⟩

theorem consSet_subset_of_subset {i : ℕ} {X X' : Set Gamma} (h : X ⊆ X') :
    consSet i X ⊆ consSet i X' := by
  rintro z ⟨?, h?, rfl⟩; exact ⟨?, h h?, rfl⟩

theorem consSet_subset {i : ℕ} {X X' : Set Gamma} :
    consSet i X ⊆ consSet i X' ↔ X ⊆ X' := by
  refine ⟨fun h ? h? => ?_, consSet_subset_of_subset⟩
  obtain ⟨?', h?', heq⟩ := h ⟨?, h?, rfl⟩
  rwa [(cons_injective heq.symm).2] at h?'
```

```
def nbhd : List ℕ → Set Gamma
  | [] => Set.univ
  | i :: l => consSet i (nbhd l)

@[simp] theorem nbhd_nil : nbhd [] = (Set.univ : Set Gamma) := rfl

@[simp] theorem nbhd_cons (i : ℕ) (l : List ℕ) : nbhd (i :: l) = consSet i (nbhd l) :=
    rfl

def Lmem (X : Set Gamma) : Prop := ∃ l, X = nbhd l

@[simp] theorem mem_nbhd (l : List ℕ) : Lmem (nbhd l) := ⟨l, rfl⟩

theorem L_nestedOrDisjoint : NestedOrDisjoint Lmem := by
  rintro X Y ⟨l, rfl⟩ ⟨l', rfl⟩
  induction l generalizing l' with
  | nil => exact Or.inr (Or.inl (by simp))
  | cons i s ih =>
    cases l' with
    | nil => exact Or.inl (by simp)
    | cons j t =>
      by_cases hij : i = j
      · subst hij
        rcases ih t with h1 | h2 | h3
        · exact Or.inl (by rw [nbhd_cons, nbhd_cons]; exact consSet_subset_of_subset h1
    )
        · exact Or.inr (Or.inl (by rw [nbhd_cons, nbhd_cons]; exact
    consSet_subset_of_subset h2))
        · exact Or.inr (Or.inr (by rw [nbhd_cons, nbhd_cons, consSet_inter, h3,
    consSet_empty]))
      · exact Or.inr (Or.inr (by rw [nbhd_cons, nbhd_cons]; exact consSet_inter_ne hij
    _ _))

end DomainTheory.NeighbourhoodSystems.LucidHistory

section
namespace DomainTheory.NeighbourhoodSystems.LucidHistory
open NeighborhoodSystem Domain.Recursive

abbrev L : NeighborhoodSystem Gamma :=
  NeighborhoodSystem.ofNestedOrDisjoint Lmem Set.univ Set.univ_nonempty
    (mem_nbhd []) L_nestedOrDisjoint
    (fun {X} _ => Set.subset_univ X)

def Lenum (n : ℕ) : Set Gamma := nbhd (decodeList n)

@[simp] theorem Lenum_mem (n : ℕ) : Lmem (Lenum n) := mem_nbhd (decodeList n)

theorem Lenum_surj {Y : Set Gamma} (hY : Lmem Y) : ∃ n, Lenum n = Y := by
  obtain ⟨l, rfl⟩ := hY
  exact ⟨encodeList l, by unfold Lenum; rw [decodeList_encodeList]⟩

theorem Lenum_zero : Lenum 0 = Set.univ := by unfold Lenum; rw [decodeList_zero]; rfl

def isPrefixChar (m n : ℕ) : ℕ := listEqChar (takeCode (listLenChar m) n) m
```

```
def LenumConsChar (n m : ℕ) : ℕ :=
  isPrefixChar m n + isPrefixChar n m - isPrefixChar m n * isPrefixChar n m

def LenumConsCharP (t : ℕ) : ℕ := LenumConsChar t.unpair.1 t.unpair.2

def LenumInter (n m : ℕ) : ℕ := selectFn (isPrefixChar m n) n m

def LenumInterEqChar (n m k : ℕ) : ℕ :=
  mulBit (isPrefixChar m n) (listEqChar k n) + mulBit (isPrefixChar n m) (listEqChar k
     m)
    - mulBit (isPrefixChar m n) (listEqChar k n) * mulBit (isPrefixChar n m) (
    listEqChar k m)

def LenumInterEqCharP (t : ℕ) : ℕ :=
  LenumInterEqChar t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2

theorem isPrefixChar_eq_one_iff (m n : ℕ) :
    isPrefixChar m n = 1 ↔ decodeList m <+: decodeList n := by
  unfold isPrefixChar
  rw [listEqChar_eq_one_iff, takeCode_eq, listLenChar_eq, List.prefix_iff_eq_take]
  exact eq_comm

theorem isPrefixChar_le_one (m n : ℕ) : isPrefixChar m n ≤ 1 := listEqChar_le_one _ _

theorem isPrefixChar_eq_zero_or_one (m n : ℕ) :
    isPrefixChar m n = 0 ∨ isPrefixChar m n = 1 := by
  have := isPrefixChar_le_one m n; omega

theorem LenumConsChar_eq_one_iff (n m : ℕ) :
    LenumConsChar n m = 1 ↔ isPrefixChar m n = 1 ∨ isPrefixChar n m = 1 := by
  unfold LenumConsChar
  rcases isPrefixChar_eq_zero_or_one m n with ha | ha <;>
    rcases isPrefixChar_eq_zero_or_one n m with hb | hb <;> simp [ha, hb]

theorem LenumConsChar_le_one (n m : ℕ) : LenumConsChar n m ≤ 1 := by
  unfold LenumConsChar
  rcases isPrefixChar_eq_zero_or_one m n with ha | ha <;>
    rcases isPrefixChar_eq_zero_or_one n m with hb | hb <;> simp [ha, hb]

theorem LenumConsCharP_le_one (t : ℕ) : LenumConsCharP t ≤ 1 :=
  LenumConsChar_le_one _ _

theorem mulBit_eq_zero_or_one {a b : ℕ} (ha : a ≤ 1) (hb : b ≤ 1) :
    mulBit a b = 0 ∨ mulBit a b = 1 := by
  have := mulBit_le_one ha hb; omega

theorem LenumInterEqChar_eq_one_iff (n m k : ℕ) :
    LenumInterEqChar n m k = 1 ↔
      (isPrefixChar m n = 1 ∧ decodeList k = decodeList n) ∨
      (isPrefixChar n m = 1 ∧ decodeList k = decodeList m) := by
  have hstep : LenumInterEqChar n m k = 1 ↔
      mulBit (isPrefixChar m n) (listEqChar k n) = 1 ∨
      mulBit (isPrefixChar n m) (listEqChar k m) = 1 := by
    unfold LenumInterEqChar
    rcases mulBit_eq_zero_or_one (isPrefixChar_le_one m n) (listEqChar_le_one k n)
    with ha | ha <;>
```

```
      rcases mulBit_eq_zero_or_one (isPrefixChar_le_one n m) (listEqChar_le_one k m)
    with hb | hb <;>
        simp [ha, hb]
  rw [hstep, mulBit_eq_one_iff, mulBit_eq_one_iff, listEqChar_eq_one_iff,
    listEqChar_eq_one_iff]

theorem LenumInterEqChar_le_one (n m k : ℕ) : LenumInterEqChar n m k ≤ 1 := by
  unfold LenumInterEqChar
  rcases mulBit_eq_zero_or_one (isPrefixChar_le_one m n) (listEqChar_le_one k n) with
    ha | ha <;>
    rcases mulBit_eq_zero_or_one (isPrefixChar_le_one n m) (listEqChar_le_one k m)
    with hb | hb <;>
      simp [ha, hb]

theorem LenumInterEqCharP_le_one (t : ℕ) : LenumInterEqCharP t ≤ 1 :=
  LenumInterEqChar_le_one _ _ _

theorem nbhd_nonempty : ∀ l : List ℕ, (nbhd l).Nonempty
  | [] => ⟨star, trivial⟩
  | _ :: l => consSet_nonempty (nbhd_nonempty l)

theorem primrec_isPrefixChar : Nat.Primrec (fun t => isPrefixChar t.unpair.1 t.unpair
    .2) := by
  have hlen : Nat.Primrec (fun t : ℕ => listLenChar t.unpair.1) := primrec_listLenChar
    .comp Nat.Primrec.left
  have htake : Nat.Primrec (fun t : ℕ => takeCode (listLenChar t.unpair.1) t.unpair.2)
     :=
    (primrec_takeCode.comp (Nat.Primrec.pair hlen Nat.Primrec.right)).of_eq fun t =>
    by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_listEqChar.comp (Nat.Primrec.pair htake Nat.Primrec.left)).of_eq fun
    t => by
    simp only [isPrefixChar, unpair_pair_fst, unpair_pair_snd]

theorem primrec_LenumInterEqChar :
    Nat.Primrec (fun t => LenumInterEqChar t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.
    unpair.2) := by
  have hn : Nat.Primrec (fun t : ℕ => t.unpair.1) := Nat.Primrec.left
  have hm : Nat.Primrec (fun t : ℕ => t.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have hk : Nat.Primrec (fun t : ℕ => t.unpair.2.unpair.2) := Nat.Primrec.right.comp
    Nat.Primrec.right
  have hpmn : Nat.Primrec (fun t : ℕ => isPrefixChar t.unpair.2.unpair.1 t.unpair.1)
    :=
    (primrec_isPrefixChar.comp (Nat.Primrec.pair hm hn)).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hpnm : Nat.Primrec (fun t : ℕ => isPrefixChar t.unpair.1 t.unpair.2.unpair.1)
    :=
    (primrec_isPrefixChar.comp (Nat.Primrec.pair hn hm)).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hekn : Nat.Primrec (fun t : ℕ => listEqChar t.unpair.2.unpair.2 t.unpair.1) :=
    (primrec_listEqChar.comp (Nat.Primrec.pair hk hn)).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hekm : Nat.Primrec (fun t : ℕ => listEqChar t.unpair.2.unpair.2 t.unpair.2.
    unpair.1) :=
    (primrec_listEqChar.comp (Nat.Primrec.pair hk hm)).of_eq fun t => by
```

```
      simp only [unpair_pair_fst, unpair_pair_snd]
  have ha : Nat.Primrec (fun t : ℕ => mulBit (isPrefixChar t.unpair.2.unpair.1 t.
    unpair.1)
      (listEqChar t.unpair.2.unpair.2 t.unpair.1)) :=
    (primrec_mulBit.comp (Nat.Primrec.pair hpmn hekn)).of_eq fun t => by simp only [
    unpair_pair_fst, unpair_pair_snd]
  have hb : Nat.Primrec (fun t : ℕ => mulBit (isPrefixChar t.unpair.1 t.unpair.2.
    unpair.1)
      (listEqChar t.unpair.2.unpair.2 t.unpair.2.unpair.1)) :=
    (primrec_mulBit.comp (Nat.Primrec.pair hpnm hekm)).of_eq fun t => by simp only [
    unpair_pair_fst, unpair_pair_snd]
  exact (primrec_sub₂ (primrec_add₂ ha hb) (primrec_mul₂ ha hb)).of_eq fun t => by
    simp only [LenumInterEqChar]

theorem primrec_LenumInterEqCharP : Nat.Primrec LenumInterEqCharP :=
  primrec_LenumInterEqChar.of_eq fun t => by simp only [LenumInterEqCharP]

theorem primrec_isPrefixChar_swap :
    Nat.Primrec (fun t => isPrefixChar t.unpair.2 t.unpair.1) :=
  (primrec_isPrefixChar.comp (Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left)).
    of_eq
    fun t => by simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_LenumConsChar :
    Nat.Primrec (fun t => LenumConsChar t.unpair.1 t.unpair.2) := by
  have ha : Nat.Primrec (fun t : ℕ => isPrefixChar t.unpair.2 t.unpair.1) :=
    primrec_isPrefixChar_swap
  have hb : Nat.Primrec (fun t : ℕ => isPrefixChar t.unpair.1 t.unpair.2) :=
    primrec_isPrefixChar
  exact (primrec_sub₂ (primrec_add₂ ha hb) (primrec_mul₂ ha hb)).of_eq fun t => by
    simp only [LenumConsChar]

theorem primrec_LenumConsCharP : Nat.Primrec LenumConsCharP :=
  primrec_LenumConsChar.of_eq fun t => by simp only [LenumConsCharP]

theorem primrec_LenumInter : Nat.Primrec (fun t => LenumInter t.unpair.1 t.unpair.2)
    :=
  (primrec_selectFn primrec_isPrefixChar_swap Nat.Primrec.left Nat.Primrec.right).
    of_eq
    fun t => by simp only [LenumInter]

theorem star_ne_cons (i : ℕ) (? : Gamma) : star = cons i ? := by
  cases ? <;> simp [star, cons]

theorem star_not_mem_consSet (i : ℕ) (X : Set Gamma) : star ∈ consSet i X := by
  rintro ⟨?, -, heq⟩; exact star_ne_cons i ? heq

theorem nbhd_subset_iff : ∀ (l l' : List ℕ), nbhd l ⊆ nbhd l' ↔ l' <+: l := by
  intro l
  induction l with
  | nil =>
    intro l'
    cases l' with
    | nil => simp
    | cons j t =>
      constructor
```

```
      · intro h
        exact absurd (h (Set.mem_univ star)) (by rw [nbhd_cons]; exact
    star_not_mem_consSet j (nbhd t))
      · rintro ⟨u, hu⟩
        simp at hu
  | cons i s ih =>
    intro l'
    cases l' with
    | nil => simp [nbhd_cons]
    | cons j t =>
      by_cases hij : i = j
      · subst hij
        rw [nbhd_cons, nbhd_cons, consSet_subset, ih t, List.cons_prefix_cons]
        simp
      · constructor
        · intro h
          exfalso
          obtain ⟨?, h?⟩ := nbhd_nonempty s
          have hmem : cons i ? ∈ consSet j (nbhd t) := h ⟨?, h?, rfl⟩
          obtain ⟨?', -, heq⟩ := hmem
          exact hij (cons_injective heq).1
        · intro h
          rw [List.cons_prefix_cons] at h
          exact absurd h.1.symm hij

theorem Lenum_subset_iff (n m : ℕ) : Lenum n ⊆ Lenum m ↔ isPrefixChar m n = 1 := by
  unfold Lenum; rw [nbhd_subset_iff, isPrefixChar_eq_one_iff]

theorem Lenum_cons_iff (n m : ℕ) :
    (∃ k, Lenum k ⊆ Lenum n ∩ Lenum m) ↔ isPrefixChar m n = 1 ∨ isPrefixChar n m = 1
    := by
  constructor
  · rintro ⟨k, hk⟩
    rcases L_nestedOrDisjoint (mem_nbhd (decodeList n)) (mem_nbhd (decodeList m)) with
     h | h | h
    · exact Or.inl ((isPrefixChar_eq_one_iff m n).mpr ((nbhd_subset_iff _ _).mp h))
    · exact Or.inr ((isPrefixChar_eq_one_iff n m).mpr ((nbhd_subset_iff _ _).mp h))
    · exfalso
      unfold Lenum at hk
      obtain ⟨t, ht⟩ := nbhd_nonempty (decodeList k)
      exact Set.notMem_empty t (h ▶ hk ht)
  · rintro (hp | hp)
    · exact ⟨n, Set.subset_inter subset_rfl ((Lenum_subset_iff n m).mpr hp)⟩
    · exact ⟨m, Set.subset_inter ((Lenum_subset_iff m n).mpr hp) subset_rfl⟩

theorem LenumInter_spec {n m : ℕ} (h : ∃ k, Lenum k ⊆ Lenum n ∩ Lenum m) :
    Lenum (LenumInter n m) = Lenum n ∩ Lenum m := by
  unfold LenumInter
  rcases isPrefixChar_eq_zero_or_one m n with h0 | h1
  · rw [h0, selectFn_zero]
    have hp : isPrefixChar n m = 1 := by
      rcases (Lenum_cons_iff n m).mp h with hp | hp
      · rw [h0] at hp; exact absurd hp (by decide)
      · exact hp
    exact (Set.inter_eq_right.mpr ((Lenum_subset_iff m n).mpr hp)).symm
  · rw [h1, selectFn_one]
```

```
    exact (Set.inter_eq_left.mpr ((Lenum_subset_iff n m).mpr h1)).symm

theorem Lenum_cons_charP_eq_one_iff (n m : ℕ) :
    (∃ k, Lenum k ⊆ Lenum n ∩ Lenum m) ↔ LenumConsCharP (Nat.pair n m) = 1 := by
  unfold LenumConsCharP
  rw [unpair_pair_fst, unpair_pair_snd, LenumConsChar_eq_one_iff]
  exact Lenum_cons_iff n m

theorem Lenum_cons_computable : RecDecidable₂ (fun n m => ∃ k, Lenum k ⊆ Lenum n ∩
    Lenum m) :=
  RecDecidable₂.of_paired_zero_one_char primrec_LenumConsCharP
    (fun t => by have := LenumConsCharP_le_one t; omega)
    Lenum_cons_charP_eq_one_iff

theorem nbhd_injective {l l' : List ℕ} (h : nbhd l = nbhd l') : l = l' := by
  have h1 : l' <+: l := (nbhd_subset_iff l l').mp h.subset
  have h2 : l <+: l' := (nbhd_subset_iff l' l).mp h.symm.subset
  exact (h1.eq_of_length (h1.length_le.antisymm h2.length_le)).symm

theorem Lenum_interEq_iff (n m k : ℕ) :
    Lenum n ∩ Lenum m = Lenum k ↔
      (isPrefixChar m n = 1 ∧ decodeList k = decodeList n) ∨
      (isPrefixChar n m = 1 ∧ decodeList k = decodeList m) := by
  unfold Lenum
  rcases L_nestedOrDisjoint (mem_nbhd (decodeList n)) (mem_nbhd (decodeList m)) with h
    | h | h
  · have hp : isPrefixChar m n = 1 := (isPrefixChar_eq_one_iff m n).mpr ((
    nbhd_subset_iff _ _).mp h)
    rw [Set.inter_eq_left.mpr h]
    constructor
    · intro heq; exact Or.inl ⟨hp, (nbhd_injective heq).symm⟩
    · rintro (⟨-, heq⟩ | ⟨hp', heq⟩)
      · rw [heq]
      · rw [isPrefixChar_eq_one_iff] at hp'
        rw [heq]
        exact Set.Subset.antisymm h ((nbhd_subset_iff (decodeList m) (decodeList n)).
    mpr hp')
  · have hp : isPrefixChar n m = 1 := (isPrefixChar_eq_one_iff n m).mpr ((
    nbhd_subset_iff _ _).mp h)
    rw [Set.inter_eq_right.mpr h]
    constructor
    · intro heq; exact Or.inr ⟨hp, (nbhd_injective heq).symm⟩
    · rintro (⟨hp', heq⟩ | ⟨-, heq⟩)
      · rw [isPrefixChar_eq_one_iff] at hp'
        rw [heq]
        exact Set.Subset.antisymm h ((nbhd_subset_iff (decodeList n) (decodeList m)).
    mpr hp')
      · rw [heq]
  · rw [h]
    have h1 : isPrefixChar m n = 1 := by
      intro hp; rw [isPrefixChar_eq_one_iff] at hp
      obtain ⟨t, ht⟩ := (nbhd_nonempty (decodeList n))
      exact Set.notMem_empty t (h ▸ Set.mem_inter ht ((nbhd_subset_iff _ _).mpr hp ht)
    )
    have h2 : isPrefixChar n m = 1 := by
      intro hp; rw [isPrefixChar_eq_one_iff] at hp
```

```
      obtain ⟨t, ht⟩ := (nbhd_nonempty (decodeList m))
      exact Set.notMem_empty t (h ▸ Set.mem_inter ((nbhd_subset_iff _ _).mpr hp ht) ht
    )
    constructor
    · intro heq; exact absurd heq.symm (nbhd_nonempty (decodeList k)).ne_empty
    · rintro (⟨hp, -⟩ | ⟨hp, -⟩)
      · exact absurd hp h1
      · exact absurd hp h2

theorem Lenum_interEq_charP_eq_one_iff (n m k : ℕ) :
    Lenum n ∩ Lenum m = Lenum k ↔ LenumInterEqCharP (Nat.pair n (Nat.pair m k)) = 1 :=
     by
  unfold LenumInterEqCharP
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [LenumInterEqChar_eq_one_iff]
  exact Lenum_interEq_iff n m k

theorem Lenum_interEq_computable : RecDecidable₃ (fun n m k => Lenum n ∩ Lenum m =
    Lenum k) :=


  RecDecidable₃.of_triple_zero_one_char primrec_LenumInterEqCharP
    (fun t => by have := LenumInterEqCharP_le_one t; omega)
    Lenum_interEq_charP_eq_one_iff

def Lpres : ComputablePresentation L where
  X := Lenum
  mem_X n := Lenum_mem n
  surj hY := Lenum_surj hY
  interEq_computable := Lenum_interEq_computable
  cons_computable := Lenum_cons_computable
  inter := LenumInter
  inter_primrec := primrec_LenumInter
  inter_spec h := LenumInter_spec h
  masterIdx := 0
  masterIdx_spec := Lenum_zero

section
open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)
variable {β ? ? : Type*}

structure LFunData {V : NeighborhoodSystem β} (P : ComputablePresentation V) where
  gN : ℕ → ℕ
  incl0 : ℕ → ℕ
  incl1 : ℕ → ℕ
  eq1 : ℕ → ℕ
  hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf Lpres P (decodeList c))
    : Set (ApproximableMap L V)).Nonempty
  hgNp : Nat.Primrec gN
  hincl0 : ∀ s, incl0 s = 1 ↔ Lpres.X s.unpair.1 ⊆ Lpres.X s.unpair.2
  hincl0p : Nat.Primrec incl0
  hincl1 : ∀ s, incl1 s = 1 ↔ P.X s.unpair.1 ⊆ P.X s.unpair.2
  hincl1p : Nat.Primrec incl1
  heq1 : ∀ s, eq1 s = 1 ↔ P.X s.unpair.1 = P.X s.unpair.2
  heq1p : Nat.Primrec eq1
end
```

```
instance : Nonempty Gamma := ⟨Sum.inl []⟩

@[simp] theorem cons_inl (i : ℕ) (l : List ℕ) : cons i (.inl l) = .inl (i :: l) := rfl

@[simp] theorem cons_inr (i : ℕ) (s : ℕ → ℕ) : cons i (.inr s) = .inr (streamCons i s
    ) := rfl

@[simp] theorem L_mem {X : Set Gamma} : L.mem X ↔ Lmem X := Iff.rfl

@[simp] theorem L_master : L.master = Set.univ := rfl

def LenumSubsetChar (t : ℕ) : ℕ := isPrefixChar t.unpair.2 t.unpair.1

def finPrefix (f : ℕ → ℕ) (n : ℕ) : List ℕ := (List.range n).map f

@[simp] theorem finPrefix_zero (f : ℕ → ℕ) : finPrefix f 0 = [] := rfl

def HasMin (x : L.Element) : Prop := ∃ X, x.mem X ∧ ∀ Y, x.mem Y → X ⊆ Y

def bitToNat (b : Bool) : ℕ := if b then 1 else 0

@[simp] theorem bitToNat_false : bitToNat false = 0 := rfl

@[simp] theorem bitToNat_true : bitToNat true = 1 := rfl

def embStr (σ : BitDom.Str) : List ℕ := σ.map bitToNat

@[simp] theorem embStr_nil : embStr [] = [] := rfl

open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)

def Tenum (n : ℕ) : Set Token :=
  match n with
  | 0 => TwoToken.master
  | 1 => zero
  | _ => one

@[simp] theorem Tenum_zero : Tenum 0 = TwoToken.master := rfl

@[simp] theorem Tenum_one : Tenum 1 = zero := rfl

@[simp] theorem Tenum_two : Tenum 2 = one := rfl

theorem Tenum_eq_canon (n : ℕ) : Tenum n = Tenum (min n 2) := by
  rcases n with _ | _ | n
  · rfl
  · rfl
  · show one = Tenum (min (n + 2) 2)
    rw [show min (n + 2) 2 = 2 from by omega]
    rfl

theorem Tenum_mem (n : ℕ) : Truth.T.mem (Tenum n) := by
  rcases n with _ | _ | n
  · exact TwoToken.mem_master
  · exact TwoToken.mem_zero
  · exact TwoToken.mem_one
```

```
theorem Tenum_ne_empty (n : ℕ) : Tenum n = (∅ : Set Token) := by
  intro h
  exact TwoToken.not_mem_empty (h ▸ Tenum_mem n)

theorem Tenum_surj {Y : Set Token} (hY : Truth.T.mem Y) : ∃ n, Tenum n = Y := by
  rcases (TwoToken.mem_iff Y).mp hY with rfl | rfl | rfl
  · exact ⟨0, rfl⟩
  · exact ⟨1, rfl⟩
  · exact ⟨2, rfl⟩

def TeqChar (a b : ℕ) : ℕ := isZero ((a - b) + (b - a))

def TinterCode (a b : ℕ) : ℕ := if a = 0 then b else if b = 0 then a else if a = b
    then a else 3

def andCode (a b : ℕ) : ℕ := if a = 1 then b else if a = 2 then 2 else 0

noncomputable def andFn (X Y : Set Token) : Set Token :=
  if X = zero then Y else if X = one then one else TwoToken.master

theorem andFn_zero (Y : Set Token) : andFn zero Y = Y := by unfold andFn; rw [
    ite_eq_left rfl]

theorem eq_master_of_master_subset {Y : Set Token} (h : TwoToken.master ⊆ Y) :
    Y = TwoToken.master := (Set.univ_subset_iff.mp h : Y = Set.univ)

theorem exists_cons_prefix_of_prefix_of_length_lt {l l' : List ℕ} (h : l <+: l')
    (hlt : l.length < l'.length) : ∃ i, l ++ [i] <+: l' := by
  obtain ⟨r, hr⟩ := h
  cases r with
  | nil => simp [← hr] at hlt
  | cons i r' => exact ⟨i, r', by rw [← hr]; simp⟩

theorem exists_extend_of_not_hasMin {x : L.Element} (hx : ¬ HasMin x) {l : List ℕ}
    (hl : x.mem (nbhd l)) : ∃ i, x.mem (nbhd (l ++ [i])) := by
  have hex : ∃ Y, x.mem Y ∧ ¬ nbhd l ⊆ Y := by
    by_contra hc; push Not at hc; exact hx ⟨nbhd l, hl, hc⟩
  obtain ⟨Y, hYmem, hnsub⟩ := hex
  obtain ⟨l', rfl⟩ := x.sub hYmem
  rcases L_nestedOrDisjoint (mem_nbhd l) (mem_nbhd l') with hsub | hsub | hdisj
  · exact absurd hsub hnsub
  · have hpre : l <+: l' := (nbhd_subset_iff l' l).mp hsub
    have hne : l = l' := by rintro rfl; exact hnsub subset_rfl
    have hlt : l.length < l'.length :=
      lt_of_le_of_ne hpre.length_le (fun heqlen => hne (hpre.eq_of_length heqlen))
    obtain ⟨i, hi⟩ := exists_cons_prefix_of_prefix_of_length_lt hpre hlt
    exact ⟨i, x.up_mem hYmem (mem_nbhd _) ((nbhd_subset_iff l' (l ++ [i])).mpr hi)⟩
  · exfalso
    have hxmem : x.mem (nbhd l ∩ nbhd l') := x.inter_mem hl hYmem
    rw [hdisj] at hxmem
    obtain ⟨l'', hl''⟩ := x.sub hxmem
    exact (nbhd_nonempty l'').ne_empty hl''.symm

noncomputable def buildData (x : L.Element) (hx : ¬ HasMin x) :
    ∀ n : ℕ, {l : List ℕ // x.mem (nbhd l) ∧ l.length = n} :=
```

```
  fun n =>
    Nat.rec (⟨[], x.master_mem, rfl⟩ : {l : List ℕ // x.mem (nbhd l) ∧ l.length = 0})
      (fun n prev =>
        ⟨prev.1 ++ [Classical.choose (exists_extend_of_not_hasMin hx prev.2.1)],
          Classical.choose_spec (exists_extend_of_not_hasMin hx prev.2.1),
          by rw [List.length_append, List.length_singleton, prev.2.2]⟩)
      n

theorem finPrefix_length (f : ℕ → ℕ) (n : ℕ) : (finPrefix f n).length = n := by
  unfold finPrefix; rw [List.length_map, List.length_range]

theorem finPrefix_succ (f : ℕ → ℕ) (n : ℕ) :
    finPrefix f (n + 1) = finPrefix f n ++ [f n] := by
  unfold finPrefix; rw [List.range_succ, List.map_append, List.map_singleton]

theorem finPrefix_prefix_succ (f : ℕ → ℕ) (n : ℕ) : finPrefix f n <+: finPrefix f (n
    + 1) := by
  rw [finPrefix_succ]; exact List.prefix_append _ _

theorem finPrefix_prefix_of_le (f : ℕ → ℕ) {n m : ℕ} (h : n ≤ m) :
    finPrefix f n <+: finPrefix f m := by
  induction h with
  | refl => exact List.prefix_refl _
  | step _ ih => exact ih.trans (finPrefix_prefix_succ f _)

theorem finPrefix_take (f : ℕ → ℕ) {n m : ℕ} (h : n ≤ m) :
    (finPrefix f m).take n = finPrefix f n := by
  unfold finPrefix; rw [← List.map_take, List.take_range, min_eq_left h]

open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)

theorem master_ne_one_T : TwoToken.master = one := by
  intro h
  have h1 : (0 : Token) ∈ TwoToken.master := by simp [TwoToken.master]
  rw [h] at h1; simp [one] at h1

theorem master_ne_zero_T : TwoToken.master = zero := by
  intro h
  have h1 : (1 : Token) ∈ TwoToken.master := by simp [TwoToken.master]
  rw [h] at h1; simp [zero] at h1

theorem andFn_master (Y : Set Token) : andFn TwoToken.master Y = TwoToken.master := by
  unfold andFn; rw [ite_eq_right master_ne_zero_T, ite_eq_right master_ne_one_T]

theorem nbhd_finPrefix_antitone (f : ℕ → ℕ) {n m : ℕ} (h : n ≤ m) :
    nbhd (finPrefix f m) ⊆ nbhd (finPrefix f n) :=
  (nbhd_subset_iff _ _).mpr (finPrefix_prefix_of_le f h)

open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)

def notCode (c : ℕ) : ℕ := if c = 1 then 2 else if c = 2 then 1 else 0

noncomputable def notFn (X : Set Token) : Set Token :=
  if X = zero then one else if X = one then zero else TwoToken.master

theorem notFn_master : notFn TwoToken.master = TwoToken.master := by
```

```
  unfold notFn; rw [ite_eq_right master_ne_zero_T, ite_eq_right master_ne_one_T]

theorem notFn_zero : notFn zero = one := by unfold notFn; rw [ite_eq_left rfl]

section
open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)
variable {β ? ? : Type*}

noncomputable def LFunData.ofPresentation {V : NeighborhoodSystem β} (P :
    ComputablePresentation V) :
    LFunData P where
  gN := funConsChar Lpres P Lpres.cons_computable.choose P.cons_computable.choose
  incl0 := Lpres.incl_computable.choose
  incl1 := P.incl_computable.choose
  eq1 := P.eq_computable.choose
  hgN := funConsChar_spec Lpres P Lpres.cons_computable.choose P.cons_computable.
    choose
    (fun s => (Lpres.cons_computable.choose_spec.2 s).symm)
    (fun s => (P.cons_computable.choose_spec.2 s).symm)
  hgNp := primrec_funConsChar Lpres P Lpres.cons_computable.choose P.cons_computable.
    choose
    Lpres.cons_computable.choose_spec.1 P.cons_computable.choose_spec.1
  hincl0 := fun s => (Lpres.incl_computable.choose_spec.2 s).symm
  hincl0p := Lpres.incl_computable.choose_spec.1
  hincl1 := fun s => (P.incl_computable.choose_spec.2 s).symm
  hincl1p := P.incl_computable.choose_spec.1
  heq1 := fun s => (P.eq_computable.choose_spec.2 s).symm
  heq1p := P.eq_computable.choose_spec.1
end

open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)

theorem one_inter_zero_T : one ∩ zero = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [zero, one]

section
open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)
variable {β ? ? : Type*}

def postcompose {V₁ : NeighborhoodSystem β} {V₂ : NeighborhoodSystem ?}
    (h : ApproximableMap V₁ V₂) : ApproximableMap (funSpace L V₁) (funSpace L V₂) :=
  curry (h.comp (evalMap L V₁))

def LFunData.pres {V : NeighborhoodSystem β} {P : ComputablePresentation V} (d :
    LFunData P) :
    ComputablePresentation (funSpace L V) :=
  funPresentation Lpres P d.gN d.incl0 d.incl1 d.eq1 d.hgN d.hgNp d.hincl0 d.hincl0p
    d.hincl1 d.hincl1p d.heq1 d.heq1p

end

open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)

theorem primrec_TeqChar {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => TeqChar (f n) (g n)) :=
  (primrec_isZero.comp (primrec_add₂ (primrec_sub₂ hf hg) (primrec_sub₂ hg hf))).of_eq
```

```
    fun _ => rfl

theorem primrec_TinterCode {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => TinterCode (f n) (g n)) := by
  have heq : Nat.Primrec (fun n => TeqChar (f n) (g n)) := primrec_TeqChar hf hg
  have hz1 := primrec_selectFn heq hf (Nat.Primrec.const 3)
  have hz2 := primrec_selectFn (primrec_isZero.comp hg) hf hz1
  refine (primrec_selectFn (primrec_isZero.comp hf) hg hz2).of_eq fun n => ?_
  unfold TinterCode TeqChar
  by_cases h0 : f n = 0
  · rw [ite_eq_left h0, h0, isZero]; simp
  · rw [ite_eq_right h0]
    have hz0 : isZero (f n) = 0 := by unfold isZero; omega
    rw [hz0, selectFn_zero]
    by_cases h1 : g n = 0
    · rw [ite_eq_left h1, h1, isZero]; simp
    · rw [ite_eq_right h1]
      have hz0' : isZero (g n) = 0 := by unfold isZero; omega
      rw [hz0', selectFn_zero]
      by_cases h2 : f n = g n
      · have : isZero ((f n - g n) + (g n - f n)) = 1 := by unfold isZero; omega
        rw [ite_eq_left h2, this, selectFn_one]
      · have : isZero ((f n - g n) + (g n - f n)) = 0 := by unfold isZero; omega
        rw [ite_eq_right h2, this, selectFn_zero]

theorem principal_hasMin (l : List ℕ) : HasMin (L.principal (mem_nbhd l)) :=
  ⟨nbhd l, ⟨mem_nbhd l, subset_rfl⟩, fun _ hY => hY.2⟩

section
open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)
variable {β ? ? : Type*}

def pwArg (V₀ : NeighborhoodSystem β) (V₁ : NeighborhoodSystem ?) :
    ApproximableMap (prod (prod (funSpace L V₀) (funSpace L V₁)) L) L :=
  proj₁ (prod (funSpace L V₀) (funSpace L V₁)) L

def pwFst (V₀ : NeighborhoodSystem β) (V₁ : NeighborhoodSystem ?) :
    ApproximableMap (prod (prod (funSpace L V₀) (funSpace L V₁)) L) (funSpace L V₀) :=
  (proj₀ (funSpace L V₀) (funSpace L V₁)).comp (proj₀ (prod (funSpace L V₀) (funSpace L
    V₁)) L)

def pwSnd (V₀ : NeighborhoodSystem β) (V₁ : NeighborhoodSystem ?) :
    ApproximableMap (prod (prod (funSpace L V₀) (funSpace L V₁)) L) (funSpace L V₁) :=
  (proj₁ (funSpace L V₀) (funSpace L V₁)).comp (proj₀ (prod (funSpace L V₀) (funSpace L
    V₁)) L)

def pointwiseBin {V₀ : NeighborhoodSystem β} {V₁ : NeighborhoodSystem ?} {V₂ :
    NeighborhoodSystem ?}
    (h : ApproximableMap (prod V₀ V₁) V₂) :
    ApproximableMap (prod (funSpace L V₀) (funSpace L V₁)) (funSpace L V₂) :=
  curry (h.comp (paired
    ((evalMap L V₀).comp (paired (pwFst V₀ V₁) (pwArg V₀ V₁)))
    ((evalMap L V₁).comp (paired (pwSnd V₀ V₁) (pwArg V₀ V₁)))))

end
```

```
def streamElement (f : ℕ → ℕ) : L.Element where
  mem Z := L.mem Z ∧ ∃ n, nbhd (finPrefix f n) ⊆ Z
  sub h := h.1
  master_mem := ⟨mem_nbhd [], 0, subset_rfl⟩
  inter_mem := by
    rintro X Y ⟨hXmem, n, hXsub⟩ ⟨hYmem, m, hYsub⟩
    have hn : nbhd (finPrefix f (max n m)) ⊆ X :=
      (nbhd_finPrefix_antitone f (le_max_left n m)).trans hXsub
    have hm : nbhd (finPrefix f (max n m)) ⊆ Y :=
      (nbhd_finPrefix_antitone f (le_max_right n m)).trans hYsub
    exact ⟨L.inter_mem hXmem hYmem (mem_nbhd _) (Set.subset_inter hn hm),
      max n m, Set.subset_inter hn hm⟩
  up_mem := by
    rintro X Y ⟨-, n, hn⟩ hYmem hXY
    exact ⟨hYmem, n, hn.trans hXY⟩

theorem finPrefix_eq_of_streamElement_eq {f₁ f₂ : ℕ → ℕ} (heq : streamElement f₁ =
    streamElement f₂) :
    ∀ n, finPrefix f₁ n = finPrefix f₂ n := by
  intro n
  have hmem1 : (streamElement f₁).mem (nbhd (finPrefix f₁ n)) := ⟨mem_nbhd _, n,
    subset_rfl⟩
  rw [heq] at hmem1
  obtain ⟨-, m, hm⟩ := hmem1
  have hpre : finPrefix f₁ n <+: finPrefix f₂ m := (nbhd_subset_iff _ _).mp hm
  have hlen : n ≤ m := by
    have hl := hpre.length_le; rwa [finPrefix_length, finPrefix_length] at hl
  have hpre' : finPrefix f₁ n = (finPrefix f₂ m).take (finPrefix f₁ n).length :=
    List.prefix_iff_eq_take.mp hpre
  rw [finPrefix_length] at hpre'
  rw [hpre', finPrefix_take f₂ hlen]

theorem streamElement_injective : Function.Injective streamElement := by
  intro f₁ f₂ heq
  have hpre := finPrefix_eq_of_streamElement_eq heq
  funext n
  have h1 := hpre (n + 1)
  rw [finPrefix_succ, finPrefix_succ, hpre n] at h1
  simpa using List.append_cancel_left h1

theorem streamElement_not_hasMin (f : ℕ → ℕ) : ¬ HasMin (streamElement f) := by
  rintro ⟨X, ⟨-, n, hXsub⟩, hmin⟩
  have hmemsucc : (streamElement f).mem (nbhd (finPrefix f (n + 1))) := ⟨mem_nbhd _, n
    + 1, subset_rfl⟩
  have hle : X ⊆ nbhd (finPrefix f (n + 1)) := hmin _ hmemsucc
  have heq : nbhd (finPrefix f n) = nbhd (finPrefix f (n + 1)) :=
    Set.Subset.antisymm (hXsub.trans hle) (nbhd_finPrefix_antitone f (Nat.le_succ n))
  have hlen := congrArg List.length (nbhd_injective heq)
  rw [finPrefix_length, finPrefix_length] at hlen
  omega

open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)

theorem subset_iff_eq_or_eq_master {X' X : Set Token} (hX' : Truth.T.mem X')
    (hX : Truth.T.mem X) (h : X' ⊆ X) : X' = X ∨ X = TwoToken.master := by
  rcases (TwoToken.mem_iff X).mp hX with rfl | rfl | rfl
```

```
  · exact Or.inr rfl
  · refine Or.inl ?_
    rcases (TwoToken.mem_iff X').mp hX' with rfl | rfl | rfl
    · exact absurd (show (1 : Token) ∈ zero from h (by simp [TwoToken.master])) (by
    simp [zero])
    · rfl
    · exact absurd (show (1 : Token) ∈ zero from h (by simp [one])) (by simp [zero])
  · refine Or.inl ?_
    rcases (TwoToken.mem_iff X').mp hX' with rfl | rfl | rfl
    · exact absurd (show (0 : Token) ∈ one from h (by simp [TwoToken.master])) (by simp
     [one])
    · exact absurd (show (0 : Token) ∈ one from h (by simp [zero])) (by simp [one])
    · rfl

def toElement : Gamma → L.Element
  | .inl l => L.principal (L_mem.mpr (mem_nbhd l))
  | .inr f => streamElement f

theorem toElement_injective : Function.Injective toElement := by
  rintro (l₁ | f₁) (l₂ | f₂) heq
  · exact congrArg Sum.inl
      (nbhd_injective (principal_injective (V := L) (mem_nbhd l₁) (mem_nbhd l₂) heq))
  · have heq' : L.principal (mem_nbhd l₁) = streamElement f₂ := heq
    exact absurd (heq' ▸ principal_hasMin l₁) (streamElement_not_hasMin f₂)
  · have heq' : streamElement f₁ = L.principal (mem_nbhd l₂) := heq
    exact absurd (principal_hasMin l₂) (heq' ▸ streamElement_not_hasMin f₁)
  · exact congrArg Sum.inr (streamElement_injective heq)

noncomputable def toStream (x : L.Element) (hx : ¬ HasMin x) (n : ℕ) : ℕ :=
  Classical.choose (exists_extend_of_not_hasMin hx (buildData x hx n).2.1)

theorem buildData_succ (x : L.Element) (hx : ¬ HasMin x) (n : ℕ) :
    (buildData x hx (n + 1)).1 = (buildData x hx n).1 ++ [toStream x hx n] := rfl

theorem buildData_eq_finPrefix (x : L.Element) (hx : ¬ HasMin x) :
    ∀ n, (buildData x hx n).1 = finPrefix (toStream x hx) n
  | 0 => rfl
  | n + 1 => by rw [buildData_succ, finPrefix_succ, buildData_eq_finPrefix x hx n]

theorem mem_nbhd_finPrefix_toStream (x : L.Element) (hx : ¬ HasMin x) (n : ℕ) :
    x.mem (nbhd (finPrefix (toStream x hx) n)) :=
  buildData_eq_finPrefix x hx n ▸ (buildData x hx n).2.1

theorem exists_finPrefix_subset_of_mem (x : L.Element) (hx : ¬ HasMin x) {Z : Set
    Gamma}


    (hZ : x.mem Z) : ∃ n, nbhd (finPrefix (toStream x hx) n) ⊆ Z := by
  obtain ⟨l, rfl⟩ := x.sub hZ
  refine ⟨l.length + 1, ?_⟩
  set n := l.length + 1
  have hn : x.mem (nbhd (finPrefix (toStream x hx) n)) := mem_nbhd_finPrefix_toStream
    x hx n
  have hinter := x.inter_mem hZ hn
  rcases L_nestedOrDisjoint (mem_nbhd l) (mem_nbhd (finPrefix (toStream x hx) n)) with
    hsub | hsub | hdisj
  ·
```

```
    have hpre : finPrefix (toStream x hx) n <+: l := (nbhd_subset_iff l _).mp hsub
    have := hpre.length_le
    rw [finPrefix_length] at this
    omega
  · exact hsub
  · exfalso
    rw [hdisj] at hinter
    obtain ⟨l', hl'⟩ := x.sub hinter
    exact (nbhd_nonempty l').ne_empty hl'.symm

theorem toElement_surjective : Function.Surjective toElement := by
  intro x
  by_cases hx : HasMin x
  · obtain ⟨X, hXmem, hXmin⟩ := hx
    obtain ⟨l, rfl⟩ := x.sub hXmem
    exact ⟨Sum.inl l, (eq_principal_of_isMin (V := L) x hXmem hXmin).symm⟩
  · exact ⟨Sum.inr (toStream x hx), by
      apply Element.ext
      intro Z
      constructor
      · rintro ⟨hZmem, n, hn⟩
        exact x.up_mem (mem_nbhd_finPrefix_toStream x hx n) hZmem hn
      · intro hZ
        exact ⟨x.sub hZ, exists_finPrefix_subset_of_mem x hx hZ⟩⟩

theorem toElement_bijective : Function.Bijective toElement :=
  ⟨toElement_injective, toElement_surjective⟩

noncomputable def gammaEquivElement : Gamma ≃ L.Element :=
  Equiv.ofBijective toElement toElement_bijective

open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)

theorem zero_inter_one_T : zero ∩ one = (∅ : Set Token) := by
  ext t; fin_cases t <;> simp [zero, one]

theorem Tenum_inter_eq {a b : ℕ} (ha : a ≤ 2) (hb : b ≤ 2) :
    Tenum a ∩ Tenum b = if a = 0 then Tenum b else if b = 0 then Tenum a
      else if a = b then Tenum a else (∅ : Set Token) := by
  interval_cases a <;> interval_cases b <;>
    simp [Tenum, TwoToken.master, Set.univ_inter, Set.inter_univ, Set.inter_self,
      zero_inter_one_T, one_inter_zero_T]

theorem Tenum_inter_eq_TinterCode' {a b : ℕ} (ha : a ≤ 2) (hb : b ≤ 2) :
    Tenum a ∩ Tenum b = if TinterCode a b = 3 then (∅ : Set Token) else Tenum (
    TinterCode a b) := by
  interval_cases a <;> interval_cases b <;>
    simp [Tenum, TinterCode, TwoToken.master, Set.univ_inter, Set.inter_univ, Set.
    inter_self,
      zero_inter_one_T, one_inter_zero_T]

theorem Tenum_inter_eq_TinterCode (n m : ℕ) :
    Tenum n ∩ Tenum m = if TinterCode (min n 2) (min m 2) = 3 then (∅ : Set Token)
      else Tenum (TinterCode (min n 2) (min m 2)) := by
  rw [Tenum_eq_canon n, Tenum_eq_canon m]
  exact Tenum_inter_eq_TinterCode' (Nat.min_le_right n 2) (Nat.min_le_right m 2)
```

```
theorem Tpres_cons_iff (n m : ℕ) :
    (∃ k, Tenum k ⊆ Tenum n ∩ Tenum m) ↔ TinterCode (min n 2) (min m 2) = 3 := by
  rw [Tenum_inter_eq_TinterCode n m]
  constructor
  · rintro ⟨k, hk⟩ hcontra
    rw [ite_eq_left hcontra] at hk
    exact Tenum_ne_empty k (Set.subset_empty_iff.mp hk)
  · intro hne
    rw [ite_eq_right hne]
    exact ⟨TinterCode (min n 2) (min m 2), subset_rfl⟩

theorem Tpres_cons_computable : RecDecidable₂ (fun n m => ∃ k, Tenum k ⊆ Tenum n ∩
    Tenum m) := by
  have ha : Nat.Primrec (fun t : ℕ => TinterCode (min t.unpair.1 2) (min t.unpair.2 2)
    ) :=
    primrec_TinterCode (primrec_min Nat.Primrec.left (Nat.Primrec.const 2))
      (primrec_min Nat.Primrec.right (Nat.Primrec.const 2))
  have hne : RecDecidable (fun t : ℕ => TinterCode (min t.unpair.1 2) (min t.unpair.2
    2) = 3) :=
    (RecDecidable.natEq ha (Nat.Primrec.const 3)).not
  exact RecDecidable.of_iff (fun t => Tpres_cons_iff t.unpair.1 t.unpair.2) hne

theorem zero_ne_one_T : zero = one := by
  intro h
  have h1 : (0 : Token) ∈ zero := by simp [zero]
  rw [h] at h1; simp [one] at h1

theorem Tenum_injOn {a b : ℕ} (ha : a ≤ 2) (hb : b ≤ 2) (h : Tenum a = Tenum b) : a =
    b := by
  interval_cases a <;> interval_cases b <;>
    simp only [Tenum_zero, Tenum_one, Tenum_two] at h <;>
    first
      | rfl
      | exact absurd h master_ne_zero_T
      | exact absurd h.symm master_ne_zero_T
      | exact absurd h master_ne_one_T
      | exact absurd h.symm master_ne_one_T
      | exact absurd h zero_ne_one_T
      | exact absurd h.symm zero_ne_one_T

theorem Tenum_interEq_iff {a b c : ℕ} (ha : a ≤ 2) (hb : b ≤ 2) (hc : c ≤ 2) :
    Tenum a ∩ Tenum b = Tenum c ↔ TinterCode a b = c := by
  rw [Tenum_inter_eq ha hb]
  unfold TinterCode
  split_ifs with h1 h2 h3
  · exact ⟨fun h => (Tenum_injOn hb hc h).symm ▸ rfl, fun h => h ▸ rfl⟩
  · exact ⟨fun h => (Tenum_injOn ha hc h).symm ▸ rfl, fun h => h ▸ rfl⟩
  · exact ⟨fun h => (Tenum_injOn ha hc h).symm ▸ rfl, fun h => h ▸ rfl⟩
  · exact ⟨fun h => absurd h.symm (Tenum_ne_empty c), fun h => absurd h.symm (by omega)⟩

theorem Tenum_interEq_iff' (n m k : ℕ) :
    Tenum n ∩ Tenum m = Tenum k ↔ TinterCode (min n 2) (min m 2) = min k 2 := by
  rw [Tenum_eq_canon n, Tenum_eq_canon m, Tenum_eq_canon k,
    Tenum_interEq_iff (Nat.min_le_right n 2) (Nat.min_le_right m 2) (Nat.min_le_right
    k 2)]
```

```
theorem Tpres_interEq_computable : RecDecidable₃ (fun n m k => Tenum n ∩ Tenum m =
    Tenum k) := by
  have ha : Nat.Primrec (fun t : ℕ => TinterCode (min t.unpair.1 2) (min t.unpair.2.
    unpair.1 2)) :=
    primrec_TinterCode (primrec_min Nat.Primrec.left (Nat.Primrec.const 2))
      (primrec_min (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 2))
  have hb : Nat.Primrec (fun t : ℕ => min t.unpair.2.unpair.2 2) :=
    primrec_min (Nat.Primrec.right.comp Nat.Primrec.right) (Nat.Primrec.const 2)
  exact RecDecidable.of_iff
    (fun t => Tenum_interEq_iff' t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2)
    (RecDecidable.natEq ha hb)

def Tpres : ComputablePresentation Truth.T where
  X := Tenum
  mem_X := Tenum_mem
  surj := Tenum_surj
  interEq_computable := Tpres_interEq_computable
  cons_computable := Tpres_cons_computable
  inter n m := TinterCode (min n 2) (min m 2)
  inter_primrec :=
    (primrec_TinterCode (primrec_min Nat.Primrec.left (Nat.Primrec.const 2))
      (primrec_min Nat.Primrec.right (Nat.Primrec.const 2))).of_eq fun _ => rfl
  inter_spec {n m} h := by
    rw [Tenum_inter_eq_TinterCode n m, ite_eq_right ((Tpres_cons_iff n m).mp h)]
  masterIdx := 0
  masterIdx_spec := rfl

section
open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)
variable {β ? ? : Type*}

noncomputable def LTdata : LFunData Tpres := LFunData.ofPresentation Tpres

noncomputable def LTpres : ComputablePresentation (funSpace L Truth.T) := LTdata.pres

end

open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)

theorem andFn_one (Y : Set Token) : andFn one Y = one := by
  unfold andFn; rw [ite_eq_right (Ne.symm zero_ne_one_T), ite_eq_left rfl]

theorem andFn_mem {X Y : Set Token} (hX : Truth.T.mem X) (hY : Truth.T.mem Y) :
    Truth.T.mem (andFn X Y) := by
  rcases (TwoToken.mem_iff X).mp hX with rfl | rfl | rfl
  · rw [andFn_master]; exact TwoToken.mem_master
  · rw [andFn_zero]; exact hY
  · rw [andFn_one]; exact TwoToken.mem_one

noncomputable def andMap2 : ApproximableMap₂ Truth.T Truth.T Truth.T where
  rel X Y Z := Truth.T.mem X ∧ Truth.T.mem Y ∧ Truth.T.mem Z ∧ andFn X Y ⊆ Z
  rel_dom₀ h := h.1
  rel_dom₁ h := h.2.1
  rel_cod h := h.2.2.1
  master_rel := ⟨TwoToken.mem_master, TwoToken.mem_master, TwoToken.mem_master, by
```

```
    show andFn TwoToken.master TwoToken.master ⊆ TwoToken.master
    rw [andFn_master]⟩
  inter_right := by
    rintro X Y Z Z' ⟨hX, hY, hZ, hsub⟩ ⟨_, _, hZ', hsub'⟩
    exact ⟨hX, hY, Truth.T.inter_mem hZ hZ' (andFn_mem hX hY) (Set.subset_inter hsub
    hsub'),
      Set.subset_inter hsub hsub'⟩
  mono := by
    rintro X X' Y Y' Z Z' ⟨hX, hY, hZ, hsub⟩ hX'X hY'Y hZZ' hX' hY' hZ'
    refine ⟨hX', hY', hZ', ?_⟩
    rcases (TwoToken.mem_iff X).mp hX with rfl | rfl | rfl
    · rw [andFn_master] at hsub
      have hZ'master : Z' = TwoToken.master := eq_master_of_master_subset (hsub.trans
    hZZ')
      rw [hZ'master, TwoToken.master]; exact Set.subset_univ _
    · rw [andFn_zero] at hsub
      have hX'eq : X' = zero :=
        (subset_iff_eq_or_eq_master hX' TwoToken.mem_zero hX'X).resolve_right
          master_ne_zero_T.symm
      rw [hX'eq, andFn_zero]
      exact (hY'Y.trans hsub).trans hZZ'
    · rw [andFn_one] at hsub
      have hX'eq : X' = one :=
        (subset_iff_eq_or_eq_master hX' TwoToken.mem_one hX'X).resolve_right
          master_ne_one_T.symm
      rw [hX'eq, andFn_one]
      exact hsub.trans hZZ'

noncomputable def andT : ApproximableMap (prod Truth.T Truth.T) Truth.T :=
  ofMap₂ andMap2

theorem notFn_one : notFn one = zero := by
  unfold notFn; rw [ite_eq_right (Ne.symm zero_ne_one_T), ite_eq_left rfl]

theorem notFn_mem {X : Set Token} (hX : Truth.T.mem X) : Truth.T.mem (notFn X) := by
  rcases (TwoToken.mem_iff X).mp hX with rfl | rfl | rfl
  · rw [notFn_master]; exact TwoToken.mem_master
  · rw [notFn_zero]; exact TwoToken.mem_one
  · rw [notFn_one]; exact TwoToken.mem_zero

noncomputable def notT : ApproximableMap Truth.T Truth.T where
  rel X Y := Truth.T.mem X ∧ Truth.T.mem Y ∧ notFn X ⊆ Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨TwoToken.mem_master, TwoToken.mem_master, by
    show notFn TwoToken.master ⊆ TwoToken.master; rw [notFn_master]⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY, hsub⟩ ⟨_, hY', hsub'⟩
    exact ⟨hX, Truth.T.inter_mem hY hY' (notFn_mem hX) (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'⟩
  mono := by
    rintro X X' Y Y' ⟨hX, hY, hsub⟩ hX'X hYY' hX' hY'
    refine ⟨hX', hY', ?_⟩
    rcases subset_iff_eq_or_eq_master hX' hX hX'X with heq | heq
    · rw [heq]; exact hsub.trans hYY'
    · rw [heq, notFn_master] at hsub
```

```
      have hY'master : Y' = TwoToken.master := eq_master_of_master_subset (hsub.trans
    hYY')
      rw [hY'master, TwoToken.master]; exact Set.subset_univ _

section
theorem bitToNat_injective : Function.Injective bitToNat := by
  intro a b h; cases a <;> cases b <;> simp_all [bitToNat]

theorem embStr_cons (b : Bool) (s : BitDom.Str) : embStr (b :: s) = bitToNat b ::
    embStr s := rfl

theorem embStr_prefix_iff : ∀ σ τ : BitDom.Str, embStr σ <+: embStr τ ↔ σ <+: τ := by
  intro σ
  induction σ with
  | nil => intro τ; simp
  | cons b s ih =>
    intro τ
    cases τ with
    | nil => simp [embStr_cons]
    | cons c t =>
      rw [embStr_cons, embStr_cons, List.cons_prefix_cons, List.cons_prefix_cons,
        bitToNat_injective.eq_iff, ih t]

theorem cone_subset_cone_iff_nbhd_embStr (σ τ : BitDom.Str) :
    BitDom.cone σ ⊆ BitDom.cone τ ↔ nbhd (embStr σ) ⊆ nbhd (embStr τ) := by
  rw [BitDom.cone.subset_cone, nbhd_subset_iff, embStr_prefix_iff]

theorem sigmaBot_le_iff_toElement_inl_embStr (σ τ : BitDom.Str) :
    BitDom.sigmaBot σ ≤ BitDom.sigmaBot τ ↔
      toElement (Sum.inl (embStr σ)) ≤ toElement (Sum.inl (embStr τ)) := by
  show BitDom.sigmaBot σ ≤ BitDom.sigmaBot τ ↔
      L.principal (L_mem.mpr (mem_nbhd (embStr σ))) ≤
        L.principal (L_mem.mpr (mem_nbhd (embStr τ)))
  rw [BitDom.sigmaBot_le_iff, L.principal_le_iff, nbhd_subset_iff, embStr_prefix_iff]

open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)

theorem primrec_notCode {f : ℕ → ℕ} (hf : Nat.Primrec f) : Nat.Primrec (fun n =>
    notCode (f n)) := by
  refine (primrec_selectFn (primrec_TeqChar hf (Nat.Primrec.const 1)) (Nat.Primrec.
    const 2)
    (primrec_selectFn (primrec_TeqChar hf (Nat.Primrec.const 2)) (Nat.Primrec.const 1)
      (Nat.Primrec.const 0))).of_eq fun n => ?_
  unfold notCode
  by_cases h1 : f n = 1
  · have h1' : TeqChar (f n) 1 = 1 := by unfold TeqChar isZero; omega
    rw [ite_eq_left h1, h1', selectFn_one]
  · have h1' : TeqChar (f n) 1 = 0 := by unfold TeqChar isZero; omega
    rw [ite_eq_right h1, h1', selectFn_zero]
    by_cases h2 : f n = 2
    · have h2' : TeqChar (f n) 2 = 1 := by unfold TeqChar isZero; omega
      rw [ite_eq_left h2, h2', selectFn_one]
    · have h2' : TeqChar (f n) 2 = 0 := by unfold TeqChar isZero; omega
      rw [ite_eq_right h2, h2', selectFn_zero]

theorem notFn_Tenum (n : ℕ) : notFn (Tenum n) = Tenum (notCode (min n 2)) := by
```

```
  rw [Tenum_eq_canon n]
  have hc2 : min n 2 ≤ 2 := Nat.min_le_right n 2
  interval_cases (min n 2) <;> simp [Tenum, notCode, notFn_master, notFn_zero,
    notFn_one]

theorem notT_isComputable : ComputableMap Tpres Tpres notT := by
  constructor
  have hcomp : Nat.Primrec (fun t : ℕ => Nat.pair (notCode (min t.unpair.1 2)) t.
    unpair.2) :=
    (primrec_notCode (primrec_min Nat.Primrec.left (Nat.Primrec.const 2))).pair Nat.
    Primrec.right
  have hdec : RecDecidable₂ (fun n m => Tenum (notCode (min n 2)) ⊆ Tenum m) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd]; rfl
    )
      (Tpres.incl_computable.comp hcomp)
  have hiff : ∀ n m, notT.rel (Tenum n) (Tenum m) ↔ Tenum (notCode (min n 2)) ⊆ Tenum
     m := by
    intro n m
    show (Truth.T.mem (Tenum n) ∧ Truth.T.mem (Tenum m) ∧ notFn (Tenum n) ⊆ Tenum m)
    ↔ _
    rw [notFn_Tenum]
    exact ⟨fun h => h.2.2, fun h => ⟨Tenum_mem n, Tenum_mem m, h⟩⟩
  have key : RecDecidable₂ (fun n m => notT.rel (Tenum n) (Tenum m)) :=
    RecDecidable.of_iff (fun t => hiff t.unpair.1 t.unpair.2) hdec
  exact key.re

theorem primrec_andCode {f g : ℕ → ℕ} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => andCode (f n) (g n)) := by
  refine (primrec_selectFn (primrec_TeqChar hf (Nat.Primrec.const 1)) hg
    (primrec_selectFn (primrec_TeqChar hf (Nat.Primrec.const 2)) (Nat.Primrec.const 2)
      (Nat.Primrec.const 0))).of_eq fun n => ?_
  unfold andCode
  by_cases h1 : f n = 1
  · have h1' : TeqChar (f n) 1 = 1 := by unfold TeqChar isZero; omega
    rw [ite_eq_left h1, h1', selectFn_one]
  · have h1' : TeqChar (f n) 1 = 0 := by unfold TeqChar isZero; omega
    rw [ite_eq_right h1, h1', selectFn_zero]
    by_cases h2 : f n = 2
    · have h2' : TeqChar (f n) 2 = 1 := by unfold TeqChar isZero; omega
      rw [ite_eq_left h2, h2', selectFn_one]
    · have h2' : TeqChar (f n) 2 = 0 := by unfold TeqChar isZero; omega
      rw [ite_eq_right h2, h2', selectFn_zero]

theorem andFn_Tenum (n m : ℕ) :
    andFn (Tenum n) (Tenum m) = Tenum (andCode (min n 2) (min m 2)) := by
  rw [Tenum_eq_canon n, Tenum_eq_canon m]
  have hn2 : min n 2 ≤ 2 := Nat.min_le_right n 2
  have hm2 : min m 2 ≤ 2 := Nat.min_le_right m 2
  interval_cases (min n 2) <;> interval_cases (min m 2) <;>
    simp [Tenum, andCode, andFn_master, andFn_zero, andFn_one]

theorem andT_isComputable : ComputableMap (prodPresentation Tpres Tpres) Tpres andT :=
    by
  constructor
  have hcomp : Nat.Primrec (fun s : ℕ => Nat.pair
      (andCode (min s.unpair.1.unpair.1 2) (min s.unpair.1.unpair.2 2)) s.unpair.2) :=
```

```
    (primrec_andCode (primrec_min (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.
    Primrec.const 2))
      (primrec_min (Nat.Primrec.right.comp Nat.Primrec.left) (Nat.Primrec.const 2))).
    pair
      Nat.Primrec.right
  have hdec : RecDecidable₂
      (fun t k => Tenum (andCode (min t.unpair.1 2) (min t.unpair.2 2)) ⊆ Tenum k) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd]; rfl
    )
      (Tpres.incl_computable.comp hcomp)
  have hiff : ∀ n m k, andT.rel (prodNbhd (Tenum n) (Tenum m)) (Tenum k) ↔
      Tenum (andCode (min n 2) (min m 2)) ⊆ Tenum k := by
    intro n m k
    have hstep : andT.rel (prodNbhd (Tenum n) (Tenum m)) (Tenum k) ↔
        andMap2.rel (Tenum n) (Tenum m) (Tenum k) := by
      show ((prod Truth.T Truth.T).mem (prodNbhd (Tenum n) (Tenum m)) ∧
        andMap2.rel (Sum.inl ^-1' prodNbhd (Tenum n) (Tenum m))
          (Sum.inr ^-1' prodNbhd (Tenum n) (Tenum m)) (Tenum k)) ↔ _
      rw [prodNbhd.inl_preimage, prodNbhd.inr_preimage]
      exact and_iff_right (prod_mem_prodNbhd (Tenum_mem n) (Tenum_mem m))
    rw [hstep]
    show (Truth.T.mem (Tenum n) ∧ Truth.T.mem (Tenum m) ∧ Truth.T.mem (Tenum k) ∧
      andFn (Tenum n) (Tenum m) ⊆ Tenum k) ↔ _
    rw [andFn_Tenum n m]
    exact ⟨fun h => h.2.2.2, fun h => ⟨Tenum_mem n, Tenum_mem m, Tenum_mem k, h⟩⟩
  have key : RecDecidable₂
      (fun t k => andT.rel ((prodPresentation Tpres Tpres).X t) (Tpres.X k)) :=
    RecDecidable.of_iff (fun s => by
      show andT.rel ((prodPresentation Tpres Tpres).X s.unpair.1) (Tpres.X s.unpair.2)
     ↔ _
      rw [prodPresentation_X]
      exact hiff s.unpair.1.unpair.1 s.unpair.1.unpair.2 s.unpair.2) hdec
  exact key.re

variable {β ? ? : Type*}

theorem postcompose_isComputable {V₁ : NeighborhoodSystem β} {V₂ : NeighborhoodSystem
    ?}
    {P₁ : ComputablePresentation V₁} {P₂ : ComputablePresentation V₂}
    (d₁ : LFunData P₁) (d₂ : LFunData P₂) {h : ApproximableMap V₁ V₂}
    (hh : ComputableMap P₁ P₂ h) :
    ComputableMap d₁.pres d₂.pres (postcompose h) := by
  have heval : ComputableMap (prodPresentation d₁.pres Lpres) P₁ (evalMap L V₁) :=
    evalMap_isComputable Lpres P₁ d₁.gN d₁.incl0 d₁.incl1 d₁.eq1
      d₁.hgN d₁.hgNp d₁.hincl0 d₁.hincl0p d₁.hincl1 d₁.hincl1p d₁.heq1 d₁.heq1p
  exact curry_isComputable d₁.pres Lpres P₂ d₂.gN d₂.incl0 d₂.incl1 d₂.eq1
    d₂.hgN d₂.hgNp d₂.hincl0 d₂.hincl0p d₂.hincl1 d₂.hincl1p d₂.heq1 d₂.heq1p
    (comp_isComputable heval hh)

theorem pointwiseBin_isComputable {V₀ : NeighborhoodSystem β} {V₁ : NeighborhoodSystem
    ?}
    {V₂ : NeighborhoodSystem ?} {P₀ : ComputablePresentation V₀} {P₁ :
    ComputablePresentation V₁}
    {P₂ : ComputablePresentation V₂} (d₀ : LFunData P₀) (d₁ : LFunData P₁) (d₂ :
    LFunData P₂)
    {h : ApproximableMap (prod V₀ V₁) V₂} (hh : ComputableMap (prodPresentation P₀ P₁) P
```

```
    ₂ h) :
    ComputableMap (prodPresentation d₀.pres d₁.pres) d₂.pres (pointwiseBin h) := by
  have h?AB : ComputableMap (prodPresentation (prodPresentation d₀.pres d₁.pres) Lpres)
      (prodPresentation d₀.pres d₁.pres) (proj₀ (prod (funSpace L V₀) (funSpace L V₁)) L
    ) :=
    proj₀_isComputable (prodPresentation d₀.pres d₁.pres) Lpres
  have h?L : ComputableMap (prodPresentation (prodPresentation d₀.pres d₁.pres) Lpres)
    Lpres
      (proj₁ (prod (funSpace L V₀) (funSpace L V₁)) L) :=
    proj₁_isComputable (prodPresentation d₀.pres d₁.pres) Lpres
  have hp0 : ComputableMap (prodPresentation (prodPresentation d₀.pres d₁.pres) Lpres)
    d₀.pres
      (pwFst V₀ V₁) := comp_isComputable h?AB (proj₀_isComputable d₀.pres d₁.pres)
  have hp1 : ComputableMap (prodPresentation (prodPresentation d₀.pres d₁.pres) Lpres)
    d₁.pres
      (pwSnd V₀ V₁) := comp_isComputable h?AB (proj₁_isComputable d₀.pres d₁.pres)
  have heval0 : ComputableMap (prodPresentation (prodPresentation d₀.pres d₁.pres)
    Lpres) P₀
      ((evalMap L V₀).comp (paired (pwFst V₀ V₁) (pwArg V₀ V₁))) :=
    comp_isComputable (paired_isComputable hp0 h?L)
      (evalMap_isComputable Lpres P₀ d₀.gN d₀.incl0 d₀.incl1 d₀.eq1
        d₀.hgN d₀.hgNp d₀.hincl0 d₀.hincl0p d₀.hincl1 d₀.hincl1p d₀.heq1 d₀.heq1p)
  have heval1 : ComputableMap (prodPresentation (prodPresentation d₀.pres d₁.pres)
    Lpres) P₁
      ((evalMap L V₁).comp (paired (pwSnd V₀ V₁) (pwArg V₀ V₁))) :=
    comp_isComputable (paired_isComputable hp1 h?L)
      (evalMap_isComputable Lpres P₁ d₁.gN d₁.incl0 d₁.incl1 d₁.eq1
        d₁.hgN d₁.hgNp d₁.hincl0 d₁.hincl0p d₁.hincl1 d₁.hincl1p d₁.heq1 d₁.heq1p)
  exact curry_isComputable (prodPresentation d₀.pres d₁.pres) Lpres P₂ d₂.gN d₂.incl0 d₂
    .incl1
    d₂.eq1 d₂.hgN d₂.hgNp d₂.hincl0 d₂.hincl0p d₂.hincl1 d₂.hincl1p d₂.heq1 d₂.heq1p
    (comp_isComputable (paired_isComputable heval0 heval1) hh)

theorem notT_lifted_isComputable : ComputableMap LTpres LTpres (postcompose notT) :=
  postcompose_isComputable LTdata LTdata notT_isComputable

theorem andT_lifted_isComputable :


    ComputableMap (prodPresentation LTpres LTpres) LTpres (pointwiseBin andT) :=
  pointwiseBin_isComputable LTdata LTdata LTdata andT_isComputable

theorem deMorganT_isComputable :
    ComputableMap (prodPresentation LTpres LTpres) LTpres
      ((postcompose notT).comp (pointwiseBin andT)
        |>.comp (paired
          (postcompose notT |>.comp (proj₀ (funSpace L Truth.T) (funSpace L Truth.T)))
          (postcompose notT |>.comp (proj₁ (funSpace L Truth.T) (funSpace L Truth.T))))
    ) :=
  comp_isComputable
    (paired_isComputable
      (comp_isComputable (proj₀_isComputable LTpres LTpres) notT_lifted_isComputable)
      (comp_isComputable (proj₁_isComputable LTpres LTpres) notT_lifted_isComputable))
    (comp_isComputable andT_lifted_isComputable notT_lifted_isComputable)
end

theorem Gamma_cases (? : Gamma) : ? = star ∨ ∃ i ?', ? = cons i ?' := by
```

```
  cases ? with
  | inl l =>
    cases l with
    | nil => exact Or.inl rfl
    | cons i l' => exact Or.inr ⟨i, .inl l', rfl⟩
  | inr s =>
    refine Or.inr ⟨s 0, .inr (fun n => s (n + 1)), ?_⟩
    simp only [cons]
    congr 1
    funext n
    cases n <;> rfl

theorem L_isEffectivelyGiven : L.IsEffectivelyGiven := ⟨Lpres⟩

theorem LenumSubsetChar_le_one (t : ℕ) : LenumSubsetChar t ≤ 1 :=
  isPrefixChar_le_one _ _

theorem Lenum_subset_char_eq_one_iff (n m : ℕ) :
    Lenum n ⊆ Lenum m ↔ LenumSubsetChar (Nat.pair n m) = 1 := by
  unfold LenumSubsetChar
  rw [unpair_pair_fst, unpair_pair_snd, Lenum_subset_iff]

open DomainTheory.NeighbourhoodSystems.TwoToken (Token zero one)

theorem T_isEffectivelyGiven : Truth.T.IsEffectivelyGiven := ⟨Tpres⟩

theorem TeqChar_eq_one_iff (a b : ℕ) : TeqChar a b = 1 ↔ a = b := by
  unfold TeqChar isZero; omega

theorem embStr_injective : Function.Injective embStr :=
  List.map_injective_iff.mpr bitToNat_injective

theorem primrec_LenumSubsetChar : Nat.Primrec LenumSubsetChar :=
  primrec_isPrefixChar_swap.of_eq fun t => by simp only [LenumSubsetChar]

theorem Lenum_subset_recDecidable : RecDecidable₂ (fun n m => Lenum n ⊆ Lenum m) :=
  RecDecidable₂.of_paired_zero_one_char primrec_LenumSubsetChar
    (fun t => by have := LenumSubsetChar_le_one t; omega)
    Lenum_subset_char_eq_one_iff

end DomainTheory.NeighbourhoodSystems.LucidHistory
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap

section
variable {α : Type*} {V : NeighborhoodSystem α}

def codePair (a b : ℕ) : ℕ := Nat.pair (Nat.pair a b) 0 + 1
end

section
variable {α : Type*} {V : NeighborhoodSystem α}
variable (P : ComputablePresentation V)

def chainDec (fincl : ℕ → ℕ) (c : ℕ) : ℕ → List ℕ → Prop
  | _, [] => True
```

```
  | a, (b :: rest) => fincl (Nat.pair c (codePair a b)) = 1 ∧ chainDec fincl c b rest

def fixPStep (fincl : ℕ → ℕ) (c acc x : ℕ) : ℕ :=
  Nat.pair x (selectFn acc.unpair.2 (isOne (fincl (Nat.pair c (codePair acc.unpair.1 x
    )))) 0)

def fixStp (fincl : ℕ → ℕ) (w : ℕ) : ℕ :=
  Nat.pair w.unpair.1
    (selectFn w.unpair.2.unpair.1.unpair.2
      (isOne (fincl (Nat.pair w.unpair.2.unpair.2
        (codePair w.unpair.2.unpair.1.unpair.1 w.unpair.1)))) 0)

def fixChainChar (fincl : ℕ → ℕ) (c i : ℕ) : ℕ :=
  foldCode (fixStp fincl) c (Nat.pair P.masterIdx 1) i

end

section
variable {α : Type*} {V : NeighborhoodSystem α}

def gLastOf : ℕ → List ℕ → ℕ
  | a, [] => a
  | _, (b :: rest) => gLastOf b rest

def gStepsOK (g : ApproximableMap V V) (P : ComputablePresentation V) : ℕ → List ℕ →
    Prop
  | _, [] => True
  | a, (b :: rest) => g.rel (P.X a) (P.X b) ∧ gStepsOK g P b rest

end

section
variable {α : Type*} {V : NeighborhoodSystem α}
variable (P : ComputablePresentation V)

theorem Xenum_codePair (gN : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P P (decodeList c))
      : Set (ApproximableMap V V)).Nonempty) (a b : ℕ) :
    Xenum P P gN (codePair a b) = step (P.X a) (P.X b) :=
  Xenum_singleton P P gN hgN a b

theorem fixPStep_foldl_fst (fincl : ℕ → ℕ) (c : ℕ) :
    ∀ (full : List ℕ) (prev fl : ℕ),
      (List.foldl (fixPStep fincl c) (Nat.pair prev fl) full).unpair.1 = gLastOf prev
    full := by
  intro full
  induction full with
  | nil =>
    intro prev fl
    show (Nat.pair prev fl).unpair.1 = prev
    exact unpair_pair_fst prev fl
  | cons b rest ih =>
    intro prev fl
    rw [List.foldl_cons]
    have hstep : fixPStep fincl c (Nat.pair prev fl) b
        = Nat.pair b (selectFn fl (isOne (fincl (Nat.pair c (codePair prev b)))) 0) :=
```

```
     by
       unfold fixPStep; simp only [unpair_pair_fst, unpair_pair_snd]
     rw [hstep, ih b (selectFn fl (isOne (fincl (Nat.pair c (codePair prev b)))) 0)]
     rfl

theorem fixPStep_foldl_snd (fincl : ℕ → ℕ) (c : ℕ) :
    ∀ (full : List ℕ) (prev fl : ℕ), fl ≤ 1 →
      ((List.foldl (fixPStep fincl c) (Nat.pair prev fl) full).unpair.2 = 1 ↔
        fl = 1 ∧ chainDec fincl c prev full) := by
  intro full
  induction full with
  | nil =>
    intro prev fl _
    show (Nat.pair prev fl).unpair.2 = 1 ↔ fl = 1 ∧ chainDec fincl c prev []
    rw [unpair_pair_snd]
    simp [chainDec]
  | cons b rest ih =>
    intro prev fl _
    rw [List.foldl_cons]
    have hstep : fixPStep fincl c (Nat.pair prev fl) b
        = Nat.pair b (selectFn fl (isOne (fincl (Nat.pair c (codePair prev b)))) 0) :=
     by
       unfold fixPStep; simp only [unpair_pair_fst, unpair_pair_snd]
     rw [hstep]
     set nf := selectFn fl (isOne (fincl (Nat.pair c (codePair prev b)))) 0 with hnf
     have hnfle : nf ≤ 1 := by
       rw [hnf]
       rcases (show fl = 0 ∨ fl = 1 by omega) with h | h <;> rw [h]
       · rw [selectFn_zero]; exact Nat.zero_le 1
       · rw [selectFn_one]; exact isOne_le_one _
     rw [ih b nf hnfle]
     have hnf1 : nf = 1 ↔ (fl = 1 ∧ fincl (Nat.pair c (codePair prev b)) = 1) := by
       rw [hnf]
       rcases (show fl = 0 ∨ fl = 1 by omega) with h | h <;> rw [h]
       · rw [selectFn_zero]; simp
       · rw [selectFn_one, isOne_eq_one_iff]; simp
     rw [hnf1]
     constructor
     · rintro ⟨⟨hfl1, hfincl1⟩, hrest⟩; exact ⟨hfl1, hfincl1, hrest⟩
     · rintro ⟨hfl1, hfincl1, hrest⟩; exact ⟨⟨hfl1, hfincl1⟩, hrest⟩

theorem fixStp_eq (fincl : ℕ → ℕ) (c acc x : ℕ) :
    fixStp fincl (Nat.pair x (Nat.pair acc c)) = fixPStep fincl c acc x := by
  unfold fixStp fixPStep
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem fixChainChar_eq (fincl : ℕ → ℕ) (c i : ℕ) :
    fixChainChar P fincl c i
      = List.foldl (fixPStep fincl c) (Nat.pair P.masterIdx 1) (decodeList i) := by
  unfold fixChainChar
  rw [foldCode_eq']
  have hstep : (fun acc x => fixStp fincl (Nat.pair x (Nat.pair acc c))) = fixPStep
    fincl c := by
    funext acc x; exact fixStp_eq fincl c acc x
  rw [hstep]
```

```
theorem fixChainChar_spec (fincl : ℕ → ℕ) (c i : ℕ) :
    (fixChainChar P fincl c i).unpair.1 = gLastOf P.masterIdx (decodeList i)
  ∧ ((fixChainChar P fincl c i).unpair.2 = 1 ↔ chainDec fincl c P.masterIdx (
    decodeList i)) := by
  rw [fixChainChar_eq]
  refine ⟨fixPStep_foldl_fst fincl c (decodeList i) P.masterIdx 1, ?_⟩
  rw [fixPStep_foldl_snd fincl c (decodeList i) P.masterIdx 1 (le_refl 1)]
  simp

end

section
variable {α : Type*} {V : NeighborhoodSystem α}

theorem gStepsOK_complete (g : ApproximableMap V V) (P : ComputablePresentation V) :
    ∀ (n a ? : ℕ), (g.iterMap n).rel (P.X a) (P.X ?) →
      ∃ full, gStepsOK g P a full ∧ P.X (gLastOf a full) ⊆ P.X ? := by
  intro n
  induction n with
  | zero =>
    intro a ? h
    rw [iterMap_zero, idMap_rel] at h
    exact ⟨[], trivial, h.2.2⟩
  | succ n ih =>
    intro a ? h
    have hcomm : g.iterMap (n + 1) = (g.iterMap n).comp g := by
      rw [iterMap_succ]; exact iter_comm g n
    rw [hcomm, comp_rel] at h
    obtain ⟨Y, h1, h2⟩ := h
    obtain ⟨k, hk⟩ := P.surj (g.rel_cod h1)
    rw [← hk] at h1 h2
    obtain ⟨full', hs', hl'⟩ := ih k ? h2
    exact ⟨k :: full', ⟨h1, hs'⟩, hl'⟩

theorem gStepsOK_sound (g : ApproximableMap V V) (P : ComputablePresentation V) :
    ∀ (full : List ℕ) (a : ℕ), gStepsOK g P a full →
      (g.iterMap full.length).rel (P.X a) (P.X (gLastOf a full)) := by
  intro full
  induction full with
  | nil =>
    intro a _
    show (g.iterMap ([] : List ℕ).length).rel (P.X a) (P.X a)
    rw [List.length_nil, iterMap_zero]
    exact idMap_rel.mpr ⟨P.mem_X a, P.mem_X a, subset_rfl⟩
  | cons b rest ih =>
    intro a hsteps
    obtain ⟨h1, hrest⟩ := hsteps
    have hcomm : g.iterMap (rest.length + 1) = (g.iterMap rest.length).comp g := by
      rw [iterMap_succ]; exact iter_comm g rest.length
    show (g.iterMap (b :: rest).length).rel (P.X a) (P.X (gLastOf b rest))
    rw [List.length_cons, hcomm, comp_rel]
    exact ⟨P.X b, h1, ih b hrest⟩

theorem fixElement_mem_iff_chain (g : ApproximableMap V V) (P : ComputablePresentation
    V)
    (a0 : ℕ) (ha0 : P.X a0 = V.master) (? : ℕ) :
```

```
    g.fixElement.mem (P.X ?) ↔
      ∃ full, gStepsOK g P a0 full ∧ P.X (gLastOf a0 full) ⊆ P.X ? := by
  rw [mem_fixElement]
  constructor
  · rintro ⟨n, hn⟩
    rw [← ha0] at hn
    exact gStepsOK_complete g P n a0 ? hn
  · rintro ⟨full, hsteps, hlast⟩
    refine ⟨full.length, ?_⟩
    have hsound := gStepsOK_sound g P full a0 hsteps
    have hmono := (g.iterMap full.length).mono hsound subset_rfl hlast (P.mem_X a0) (P
    .mem_X ?)
    rwa [ha0] at hmono

end

section
variable {α : Type*} {V : NeighborhoodSystem α}
variable (P : ComputablePresentation V)

theorem fixMap_rel_iff (gN : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P P (decodeList c))
      : Set (ApproximableMap V V)).Nonempty) (c ? : ℕ) :
    (fixMap V).rel (Xenum P P gN c) (P.X ?) ↔
      ∃ full : List ℕ,
        gStepsOK (toApproxMap ((funSpace V V).principal (Xenum_mem P P gN hgN c))) P P
    .masterIdx full
        ∧ P.X (gLastOf P.masterIdx full) ⊆ P.X ? := by
  rw [(fixMap V).rel_iff_mem_principal (Xenum_mem P P gN hgN c), fixMap_toElementMap]
  exact fixElement_mem_iff_chain _ P P.masterIdx P.masterIdx_spec ?

theorem leastMap_Xenum_rel (gN : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P P (decodeList c))
      : Set (ApproximableMap V V)).Nonempty) (c a b : ℕ) :
    (toApproxMap ((funSpace V V).principal (Xenum_mem P P gN hgN c))).rel (P.X a) (P.X
    b)
      ↔ Xenum P P gN c ⊆ step (P.X a) (P.X b) := by
  rw [toApproxMap_rel, mem_principal]
  exact ⟨fun h => h.2, fun h => ⟨step_mem (P.mem_X a) (P.mem_X b), h⟩⟩

theorem chainDec_iff_gStepsOK (gN : ℕ → ℕ)
    (hgN : ∀ c, gN c = 1 ↔ (stepFun (funListOf P P (decodeList c))
      : Set (ApproximableMap V V)).Nonempty) (fincl : ℕ → ℕ)
    (hfincl : ∀ s, fincl s = 1 ↔ Xenum P P gN s.unpair.1 ⊆ Xenum P P gN s.unpair.2) (
    c : ℕ) :
    ∀ (full : List ℕ) (a : ℕ),
      chainDec fincl c a full ↔
        gStepsOK (toApproxMap ((funSpace V V).principal (Xenum_mem P P gN hgN c))) P a
    full := by
  intro full
  induction full with
  | nil => intro a; exact Iff.rfl
  | cons b rest ih =>
    intro a
    constructor
    · rintro ⟨hstep, hrest⟩
```

```
      refine ⟨?_, (ih b).mp hrest⟩
      rw [hfincl, unpair_pair_fst, unpair_pair_snd, Xenum_codePair P gN hgN a b] at
    hstep
      exact (leastMap_Xenum_rel P gN hgN c a b).mpr hstep
    · rintro ⟨hstep, hrest⟩
      refine ⟨?_, (ih b).mpr hrest⟩
      have hsub : Xenum P P gN c ⊆ step (P.X a) (P.X b) :=
        (leastMap_Xenum_rel P gN hgN c a b).mp hstep
      rw [hfincl, unpair_pair_fst, unpair_pair_snd, Xenum_codePair P gN hgN a b]
      exact hsub

theorem primrec_fixStp (fincl : ℕ → ℕ) (hfinclp : Nat.Primrec fincl) :
    Nat.Primrec (fixStp fincl) := by
  have hx : Nat.Primrec (fun w : ℕ => w.unpair.1) := Nat.Primrec.left
  have hc : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hprev : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hflag : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hcode : Nat.Primrec (fun w : ℕ => codePair w.unpair.2.unpair.1.unpair.1 w.
    unpair.1) :=
    Nat.Primrec.succ.comp ((hprev.pair hx).pair (Nat.Primrec.const 0))
  have hinner : Nat.Primrec (fun w => Nat.pair w.unpair.2.unpair.2
      (codePair w.unpair.2.unpair.1.unpair.1 w.unpair.1)) := hc.pair hcode
  have hsel : Nat.Primrec (fun w => selectFn w.unpair.2.unpair.1.unpair.2
      (isOne (fincl (Nat.pair w.unpair.2.unpair.2
        (codePair w.unpair.2.unpair.1.unpair.1 w.unpair.1)))) 0) :=
    primrec_selectFn hflag (primrec_isOne.comp (hfinclp.comp hinner)) (Nat.Primrec.
    const 0)
  exact (hx.pair hsel).of_eq (fun _ => rfl)

theorem primrec_fixChainChar (fincl : ℕ → ℕ) (hfinclp : Nat.Primrec fincl) :
    Nat.Primrec (fun t => fixChainChar P fincl t.unpair.1 t.unpair.2) :=
  (primrec_foldCode (primrec_fixStp fincl hfinclp) Nat.Primrec.left
    (Nat.Primrec.const (Nat.pair P.masterIdx 1)) Nat.Primrec.right).of_eq (fun _ =>
    rfl)

end
end DomainTheory.NeighbourhoodSystems
end
```

4.8.59 Exercise 7.24 — LUCID history domain

(Suggested by the LUCID language of Ashcroft and Wadge: SIAM Jour. Comp. vol. 5 (1976).)
Define a set Γ by

$$\Gamma = \bigcup_{i=0}^{\infty} (\{i\} \times \Gamma) \cup \{\star\}.$$

Define a system

$$L = \{\Gamma\} \cup \{\{i\} \times X \mid i \in \mathbf{N} \text{ and } X \in L\}.$$

Show that L is effectively given. Show that the elements of $|L|$ can be identified with the finite and infinite sequences of natural numbers. What is the connection between B and L?

Show that the combinators of LUCID can be construed as computable mappings of type

$$(L \to T) \to (L \to T)$$

or of type

$$(L \to T) \times (L \to T) \to (L \to T).$$

Conclude that programs in LUCID define computable maps.

Exercise_7_24.lean

@import "Exercise_7_24.lean"

The development is with the LUCID histories peer.

Theorem (`example_724`). The domain of histories is effectively given, the lifted negation and conjunction are computable, and passage to an element is bijective.

Proof. By the presentation of histories. ■

Exercise_7_24.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .lucidHistory

@[expose] public section

open DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem Domain.Recursive
open DomainTheory.NeighbourhoodSystems.LucidHistory
/-- Scott's Exercise 7.24: L is effectively given; lifted connectives and toElement.
    -/
theorem example_724 :
    L.IsEffectivelyGiven ∧
    ComputableMap LTpres LTpres (postcompose notT) ∧
    ComputableMap (prodPresentation LTpres LTpres) LTpres (pointwiseBin andT) ∧
    Function.Bijective toElement :=
  ⟨L_isEffectivelyGiven, ⟨notT_lifted_isComputable, ⟨andT_lifted_isComputable,
    toElement_bijective⟩⟩⟩
```

4.9 Retracts of the universal domain

<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

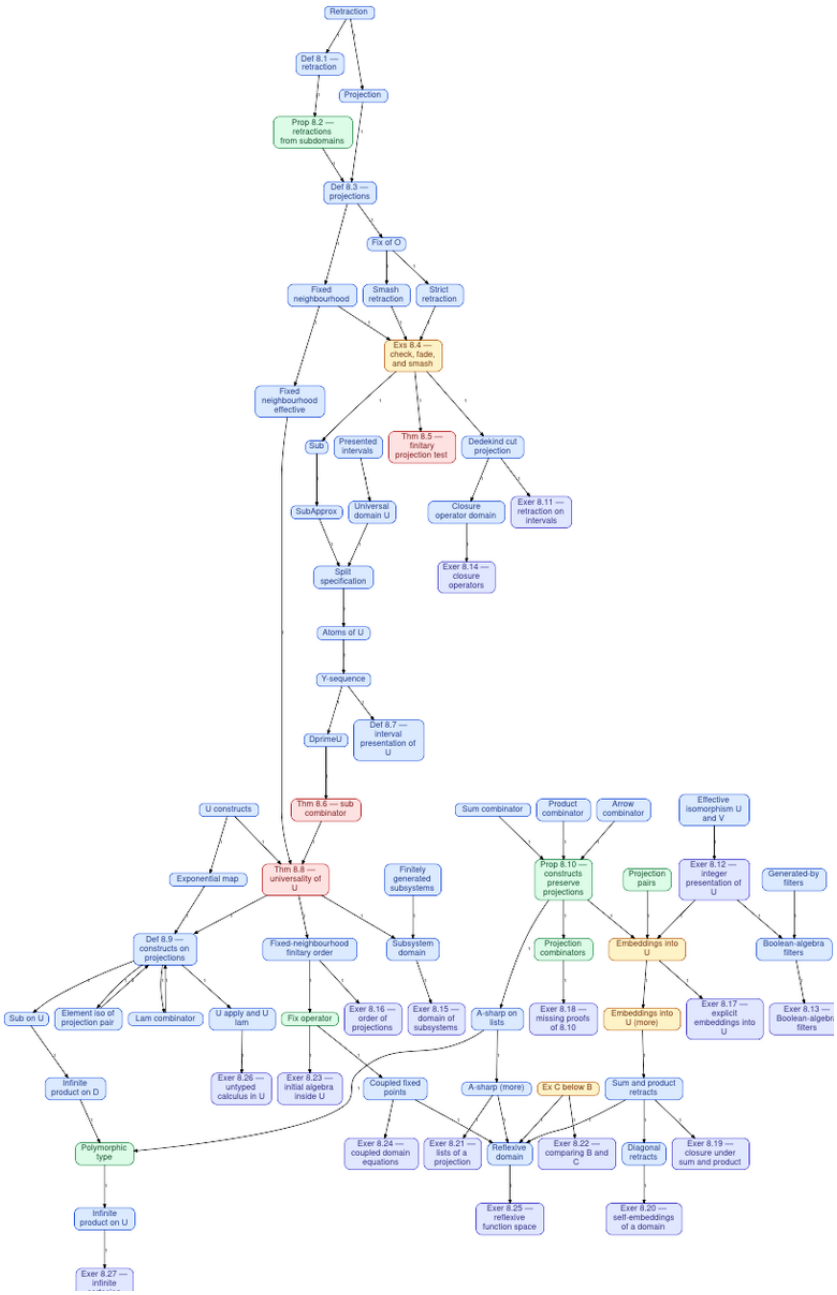


Figure 53: Diagram 18

4.9.1 Introduction

In order to be able to have a fully flexible method of solving domain equations and to be able to see why the domains obtained are effectively given, we shall embed all the desired domains in one “largest” domain. This universal domain will be easily shown to be effectively given, and the mappings needed to extract the other domains will be found to be computable. In order to be able to carry out this programme, we investigate first how certain subdomains correspond to mappings — the so-called *retracts*. An advantage of

this analysis is that all the necessary definitions can be written out in λ-calculus notation, thus demonstrating the power of our mathematical programming language.

4.9.2 Retraction

A retraction is an approximable endomap of a domain that is idempotent under composition. Every subdomain embedding yields such a retraction, whose fixed points recover the subdomain.

Definition (`Retraction`). An approximable map from a domain to itself is a retraction when its composite with itself equals itself.

Proof. By the defining field of the class. ■

Theorem (`isRetraction_idMap`). The identity map of a domain is a retraction.

Proof. By the left unit law for composition of approximable maps. ■

Definition (`Subsystem.retractionOfSubsystem`). From a subdomain embedding one forms the composite of the injection after the projection; that composite is the associated retraction of the ambient domain.

Proof. By the projection–injection presentation of a subdomain. ■

Theorem (`Subsystem.retractionOfSubsystem_rel`). Two neighbourhoods are related by the subdomain retraction precisely when there is a neighbourhood of the subdomain lying between them.

Proof. By unfolding the composite of injection and projection. ■

Theorem (`Subsystem.retractionOfSubsystem_toElementMap_inj`). The subdomain retraction fixes every point in the image of the injection.

Proof. By the projection–injection identity on the subdomain. ■

Definition (`Subsystem.elementIso`). Elements of a subdomain are order-isomorphic to the fixed points of its associated retraction.

Proof. The injection lands in the fixed-point set; the projection recovers the subdomain element; monotonicity of both maps gives the order isomorphism. ■

Theorem (`Subsystem.isRetraction_retractionOfSubsystem`). The endomap associated with a subdomain embedding is a retraction.

Proof. By the projection–injection identity and associativity of composition. ■

Theorem (`Subsystem.retraction_of_subsystem`). The subdomain endomap is a retraction, and the subdomain is isomorphic to its fixed-point set.

Proof. By the retraction property and the element isomorphism. ■

`retraction.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.projectionPair
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Lemma_6_15

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import Mathlib.Order.Hom.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
```

```
variable {α : Type*} {E : NeighborhoodSystem α}

/-- An approximable endomap is a retraction when it is idempotent. -/
class Retraction (a : ApproximableMap E E) : Prop where
  out : a.comp a = a

@[simp] theorem isRetraction_idMap : Retraction (idMap E) :=
  ⟨idMap_comp (idMap E)⟩

namespace Subsystem
variable {D E : NeighborhoodSystem α}

/-- The retraction of `E` associated with a subdomain embedding `D ? E`. -/
def retractionOfSubsystem (h : D ? E) : ApproximableMap E E := h.inj.comp h.proj

@[simp] theorem retractionOfSubsystem_rel (h : D ? E) {X Z : Set α} :
    (retractionOfSubsystem h).rel X Z ↔ E.mem X ∧ E.mem Z ∧ ∃ Y, D.mem Y ∧ X ⊆ Y ∧ Y
    ⊆ Z := by
  simp only [retractionOfSubsystem, comp_rel, inj_rel, proj_rel]
  constructor
  · rintro ⟨Y, ⟨hEX, hDY, hXY⟩, hDY', hEZ, hYZ⟩
    exact ⟨hEX, hEZ, Y, hDY, hXY, hYZ⟩
  · rintro ⟨hEX, hEZ, Y, hDY, hXY, hYZ⟩
    exact ⟨Y, ⟨hEX, hDY, hXY⟩, hDY, hEZ, hYZ⟩

theorem retractionOfSubsystem_toElementMap_inj (h : D ? E) (x : D.Element) :
    (retractionOfSubsystem h).toElementMap (h.inj.toElementMap x) = h.inj.toElementMap
     x := by
  have h1 : (retractionOfSubsystem h).toElementMap (h.inj.toElementMap x)
      = h.inj.toElementMap (h.proj.toElementMap (h.inj.toElementMap x)) :=
    toElementMap_comp h.inj h.proj (h.inj.toElementMap x)
  have h2 : h.proj.toElementMap (h.inj.toElementMap x) = (h.proj.comp h.inj).
    toElementMap x :=
    (toElementMap_comp h.proj h.inj x).symm
  rw [h1, h2, h.proj_comp_inj, toElementMap_idMap]

/-- Elements of a subdomain are order-isomorphic to the fixed points of its retraction
    . -/
def elementIso (h : D ? E) :
    D.Element ≃o {y : E.Element // (retractionOfSubsystem h).toElementMap y = y} where
  toFun x := ⟨h.inj.toElementMap x, retractionOfSubsystem_toElementMap_inj h x⟩
  invFun y := h.proj.toElementMap y.1
  left_inv x := by
    have h1 : h.proj.toElementMap (h.inj.toElementMap x) = (h.proj.comp h.inj).
    toElementMap x :=
      (toElementMap_comp h.proj h.inj x).symm
    show h.proj.toElementMap (h.inj.toElementMap x) = x
    rw [h1, h.proj_comp_inj, toElementMap_idMap]
  right_inv y := by
    apply Subtype.ext
    have hfix : (h.inj.comp h.proj).toElementMap y.1 = y.1 := y.2
    have h1 : (h.inj.comp h.proj).toElementMap y.1 = h.inj.toElementMap (h.proj.
    toElementMap y.1) :=
      toElementMap_comp h.inj h.proj y.1
    show h.inj.toElementMap (h.proj.toElementMap y.1) = y.1
    rw [← h1]; exact hfix
```

```
  map_rel_iff' := by
    intro x x'
    show h.inj.toElementMap x ≤ h.inj.toElementMap x' ↔ x ≤ x'
    constructor
    · intro hle
      have hmono := h.proj.toElementMap_mono hle
      have h1 : h.proj.toElementMap (h.inj.toElementMap x) = (h.proj.comp h.inj).
    toElementMap x :=
        (toElementMap_comp h.proj h.inj x).symm
      have h2 : h.proj.toElementMap (h.inj.toElementMap x') = (h.proj.comp h.inj).
    toElementMap x' :=
        (toElementMap_comp h.proj h.inj x').symm
      rw [h1, h2, h.proj_comp_inj, toElementMap_idMap, toElementMap_idMap] at hmono
      exact hmono
    · intro hle
      exact h.inj.toElementMap_mono hle

theorem isRetraction_retractionOfSubsystem (h : D ? E) :
    Retraction (retractionOfSubsystem h) := by
  constructor
  show (h.inj.comp h.proj).comp (h.inj.comp h.proj) = h.inj.comp h.proj
  rw [comp_assoc, ← comp_assoc h.proj h.inj h.proj, h.proj_comp_inj, idMap_comp]

theorem retraction_of_subsystem (h : D ? E) :
    Retraction (retractionOfSubsystem h) ∧
      Nonempty (D.Element ≃o {y : E.Element // (retractionOfSubsystem h).toElementMap
    y = y}) :=
  ⟨isRetraction_retractionOfSubsystem h, ⟨elementIso h⟩⟩

end Subsystem
end DomainTheory.NeighbourhoodSystems
```

4.9.3 Definition 8.1 — retraction

DEFINITION 8.1. A retraction of a given domain E is an approximable mapping $a : E \to E$ such that $a \circ a = a$. □

The development is with the retraction peer.

Theorem (`definition_81`). An approximable endomap is a retraction precisely when it equals its composite with itself.

Proof. By the definition of retraction. ■

`Definition_8_1.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    retraction

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Definition 8.1: a retraction is an approximable endomap equal to its
```

```
own composite. -/
theorem definition_81 {α : Type*} {E : NeighborhoodSystem α} (a : ApproximableMap E E)
     :
    Retraction a ↔ a.comp a = a :=
  ⟨fun h => h.out, fun h => ⟨h⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.9.4 Proposition 8.2 — retractions from subdomains

PROPOSITION 8.2. If $D \triangleleft E$ and if $a : E \to E$ is defined by

$$X\ a\ Z \quad \text{iff} \quad \exists Y \in D.\ X \subseteq Y \subseteq Z$$

for all $X, Z \in E$, then a is a retraction and $|D|$ is isomorphic to the fixed-point set of a, the set $\{y \in |E| \mid a(y) = y\}$, under inclusion.

Proof: That a is an approximable mapping is a direct consequence of Definition 6.10. Indeed, in the notation of Proposition 6.12, we have

$$a = i \circ j,$$

and this is another proof that a is approximable. This remark is also convenient, since we know from 6.10

$$j \circ i = I_D.$$

Therefore, we find:

$$a \circ a = i \circ j \circ i \circ j = i \circ j = a;$$

and so a is a retraction.

We can also employ i and j to give the isomorphism on $|D|$. If $x \in |D|$, then $i(x) \in |E|$ and we calculate:

$$a(i(x)) = i \circ j \circ i(x) = i(x).$$

Thus, $i(x)$ belongs to the fixed-point set of a. In the other direction, if $a(y) = y$, then $i(j(y)) = y$. But $j(y) \in |D|$, so i maps $|D|$ one-one and onto the fixed-point set of a. As i and j are monotone, the map is an isomorphism with respect to $\sqsubseteq$. □

Not every retraction comes from a relationship like $D \triangleleft E$; in fact, we can see from the definition of a above that $a \sqsubseteq 1_E$. But, as is indicated in Exercise 8.11, even this condition is not sufficient to characterize the kind of retractions provided by 8.2. The characterization is as follows.

The development is with the retraction peer.

Theorem (`proposition_82`). For every subdomain embedding, the associated endomap is a retraction, and the subdomain's elements are isomorphic to that retraction's fixed points.

Proof. By the retraction and element-isomorphism facts for subdomain retractions. ■

`Proposition_8_2.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    retraction
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_1

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap Subsystem

/-- Scott's Proposition 8.2: the endomap of a subdomain embedding is a retraction,
and the subdomain's elements are isomorphic to that retraction's fixed points. -/
theorem proposition_82 :
    (∀ {α : Type*} {D E : NeighborhoodSystem α} (h : D ? E),
      Retraction (retractionOfSubsystem h)) ∧
    (∀ {α : Type*} {D E : NeighborhoodSystem α} (h : D ? E),
      Retraction (retractionOfSubsystem h) ∧
        Nonempty (D.Element ≃o {y : E.Element // (retractionOfSubsystem h).
    toElementMap y = y})) :=
  ⟨@isRetraction_retractionOfSubsystem, @retraction_of_subsystem⟩

end DomainTheory.NeighbourhoodSystems
```

4.9.5 Projection

A projection is a retraction lying below the identity. It is finitary when its fixed-point set is isomorphic to the element space of some neighbourhood system; in particular every subdomain retraction is a finitary projection.

Definition (`IsFinitary`). An endomap is finitary when the ordered set of its fixed points is isomorphic to the elements of some neighbourhood system.

Proof. By the defining existential. ■

Definition (`IsProjection`). An endomap is a projection when it is a retraction and lies below the identity.

Proof. By the defining conjunction. ■

Definition (`FinitaryProjection`). A finitary projection is a projection that is finitary.

Proof. By the defining fields of the class. ■

Theorem (`Subsystem.isFinitary_retractionOfSubsystem`). The retraction of a subdomain embedding is finitary.

Proof. The element isomorphism of the subdomain supplies the witnessing domain. ■

Theorem (`Subsystem.retractionOfSubsystem_le_idMap`). The subdomain retraction lies below the identity.

Proof. Related neighbourhoods of the retraction are nested, so the identity relation holds between them. ■

Theorem (`Subsystem.isProjection_retractionOfSubsystem`). The subdomain retraction is a projection.

Proof. By the retraction and identity-bound facts. ■

Theorem (`Subsystem.isFinitaryProjection_retractionOfSubsystem`). The subdomain retraction is a finitary projection.

Proof. By the projection and finitary facts. ■

`projection.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    retraction

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
universe u
variable {α : Type u} {D E : NeighborhoodSystem α}

/-- A retraction is finitary when its fixed-point set is order-isomorphic to the
elements of some neighbourhood system. -/
def IsFinitary (a : ApproximableMap E E) : Prop :=
  ∃ (β : Type u) (F : NeighborhoodSystem β),
    Nonempty ({y : E.Element // a.toElementMap y = y} ≃o F.Element)

/-- A projection is a retraction lying below the identity. -/
def IsProjection (a : ApproximableMap E E) : Prop := Retraction a ∧ a ≤ idMap E

/-- A finitary projection is a projection whose fixed-point set is a domain. -/
class FinitaryProjection (a : ApproximableMap E E) : Prop where
  out : IsProjection a ∧ IsFinitary a

namespace Subsystem

theorem isFinitary_retractionOfSubsystem (h : D ? E) :
    IsFinitary (retractionOfSubsystem h) :=
  ⟨α, D, ⟨(elementIso h).symm⟩⟩

theorem retractionOfSubsystem_le_idMap (h : D ? E) : retractionOfSubsystem h ≤ idMap E
    := by
  intro X Z hr
  obtain ⟨hEX, hEZ, Y, _, hXY, hYZ⟩ := (retractionOfSubsystem_rel h).mp hr
  exact ⟨hEX, hEZ, hXY.trans hYZ⟩

theorem isProjection_retractionOfSubsystem (h : D ? E) :
    IsProjection (retractionOfSubsystem h) :=
  ⟨isRetraction_retractionOfSubsystem h, retractionOfSubsystem_le_idMap h⟩

theorem isFinitaryProjection_retractionOfSubsystem (h : D ? E) :
    FinitaryProjection (retractionOfSubsystem h) :=
  ⟨⟨isProjection_retractionOfSubsystem h, isFinitary_retractionOfSubsystem h⟩⟩

end Subsystem
end DomainTheory.NeighbourhoodSystems
```

4.9.6 Definition 8.3 — projections

DEFINITION 8.3. A retraction $a : E \to E$ is called a *projection* provided

$$a \sqsubseteq 1_E;$$

it is *finitary* iff its fixed-point set is isomorphic to a domain. □

The development is with the projection peer.

Theorem (`definition_83`). An endomap is a projection precisely when it is a retraction below the identity, and a finitary projection precisely when it is both a projection and finitary (fixed points isomorphic to a domain).

Proof. By the definitions of projection and finitary projection. ■

`Definition_8_3.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    projection
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_2

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Definition 8.3: a projection is a retraction below the identity, and
it is finitary when its fixed-point set is isomorphic to a domain. -/
theorem definition_83 {α : Type*} {E : NeighborhoodSystem α} (a : ApproximableMap E E)
    :
    (IsProjection a ↔ Retraction a ∧ a ≤ idMap E) ∧
      (FinitaryProjection a ↔ IsProjection a ∧ IsFinitary a) :=
  ⟨Iff.rfl, ⟨fun h => h.out, fun h => ⟨h⟩⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.9.7 Fixed neighbourhood

The fixed-neighbourhood system of an approximable endomap collects neighbourhoods on which the map relates to itself; it is a subdomain. Compact elements agree with principals; a finitary projection is characterized by Scott's membership formula, proved by sectioning its fixed-point isomorphism.

Definition (`FixSet`). The fixed-point set of an endomap consists of those domain elements that the map sends to themselves.

Proof. By the subtype of fixed points. ■

Definition (`fixedNbhd`). The fixed neighbourhood system over the same index type admits a set precisely when it is a neighbourhood of the ambient system and the endomap relates that set to itself; the master is unchanged.

Proof. By the intersection and sub-master clauses inherited from the ambient system together with compositional closure of self-relation. ■

Theorem (`fixedNbhd_mem`). Membership in the fixed neighbourhood system is equivalent to ambient membership together with self-relation of the endomap on that set.

Proof. By definition. ■

Definition (`IsCompactElt`). An element is compact when whenever it lies below a directed supremum of elements, it lies below some term of that directed family.

Proof. By the universal quantification over directed families. ■

Theorem (`principal_le_of_mem`). The principal determined by a sub-neighbourhood on which an element lies is below that element.

Proof. By upward closure along inclusion of neighbourhoods. ■

Theorem (`principalFamily_directed`). The principals obtained from neighbourhoods on which an element lies form a directed family.

Proof. By intersection of witnessing neighbourhoods and the principal comparison lemma. ■

Theorem (`eq_iSupDirected_principal`). Every element equals the directed supremum of the principals from its witnessing neighbourhoods.

Proof. By element extension and the directed-supremum membership characterization. ■

Theorem (`eq_principal_of_isCompactElt`). Every compact element is a principal.

Proof. By directedness of the principal family and compactness against that supremum. ■

Theorem (`principal_compact`). A principal below a directed supremum lies below some term of the family.

Proof. By membership at the principal's neighbourhood and the directed supremum membership law. ■

Theorem (`principal_isCompactElt`). Principals are compact elements.

Proof. By the principal compactness lemma. ■

Theorem (`fixedNbhd_subsystem`). The fixed neighbourhood system embeds as a subdomain of the ambient system.

Proof. By master coincidence, inheritance of membership, and intersection closure for self-relation. ■

Theorem (`mem_of_exists_rel_self`). If an element lies in a neighbourhood contained in a larger one and the endomap self-relates on some inner neighbourhood, then the image element lies in the larger neighbourhood.

Proof. By principal comparison, monotonicity of the element map, and upward closure. ■

Theorem (`isFinitaryProjection_of_formula`). An endomap satisfying Scott's membership formula on elements is a finitary projection.

Proof. By identifying it with the subdomain retraction of its fixed neighbourhood system and applying the subdomain finitary projection theorem. ■

Definition (`sectionMap`). From an order-isomorphism between the fixed-point set and another domain, the section map sends a principal in the codomain to the corresponding fixed point in the ambient domain.

Proof. By monotonic extension from principals. ■

Theorem (`toElementMap_iSupDirected_fixed`). On a directed family of fixed points, the element map of the endomap preserves the directed supremum.

Proof. By the fixed-point hypothesis and the directed supremum law for element maps. ■

Theorem (`toElementMap_sectionMap_principal`). The section map carries a codomain principal to the underlying fixed point of its inverse image under the isomorphism.

Proof. By the defining action on principals. ■

Theorem (`toElementMap_sectionMap`). The section map on an arbitrary codomain element equals the directed supremum of sections on the principals below it.

Proof. By principal decomposition, fixed points along that family, and antisymmetry using monotonicity of the isomorphism. ■

Theorem (`e_apply_iSupDirected_fixed`). Under fixed-point and directedness hypotheses, applying the fixed-point isomorphism to a supremum of fixed points equals the supremum of the corresponding codomain elements.

Proof. By monotonicity of the isomorphism and preservation of directed suprema on fixed points. ■

Theorem (`toElementMap_idem`). For a retraction, applying the element map twice equals applying it once.

Proof. By idempotence of composition of approximable maps. ■

Theorem (`exists_principal_eq_of_isRetraction_le_idMap`). For a retraction below the identity, the section of a codomain principal is itself a principal.

Proof. By compactness of principals, preservation of directed suprema under the isomorphism, and the retraction below the identity. ■

Theorem (`formula_of_isFinitaryProjection`). Every finitary projection satisfies Scott's membership formula: an element's image lies in a neighbourhood if and only if that neighbourhood is admissible and some inner neighbourhood on which the element lies and the map self-relates is contained in it.

Proof. By the subsection construction, principal factorization on the fixed-point domain, and the membership transfer lemmas. ■

Theorem (`finitaryProjection_iff_formula`). An endomap is a finitary projection if and only if it satisfies Scott's membership formula for every element.

Proof. By the two directions of the formula characterisation. ■

`fixedNbhd.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_3

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.iSupDirected

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
universe u
variable {α : Type u} {E : NeighborhoodSystem α}

abbrev FixSet (a : ApproximableMap E E) := {y : E.Element // a.toElementMap y = y}

def fixedNbhd (a : ApproximableMap E E) : NeighborhoodSystem α where
  mem X := E.mem X ∧ a.rel X X
  master := E.master
  master_nonempty := E.master_nonempty
```

```
  master_mem := ⟨E.master_mem, a.master_rel⟩
  inter_mem := by
    rintro X Y Z ⟨hXE, hXa⟩ ⟨hYE, hYa⟩ ⟨hZE, _⟩ hZsub
    have hXYE : E.mem (X ∩ Y) := E.inter_mem hXE hYE hZE hZsub
    have h1 : a.rel (X ∩ Y) X :=
      a.mono hXa Set.inter_subset_left subset_rfl hXYE hXE
    have h2 : a.rel (X ∩ Y) Y :=
      a.mono hYa Set.inter_subset_right subset_rfl hXYE hYE
    exact ⟨hXYE, a.inter_right h1 h2⟩
  sub_master := fun h => E.sub_master h.1

@[simp] theorem fixedNbhd_mem {a : ApproximableMap E E} {X : Set α} :
    (fixedNbhd a).mem X ↔ E.mem X ∧ a.rel X X := Iff.rfl

section Algebraic

variable {? : Type u} {V : NeighborhoodSystem ?}

instance instNonemptyMemSubtype (x : V.Element) : Nonempty {X : Set ? // x.mem X} :=
  ⟨⟨V.master, x.master_mem⟩⟩

def IsCompactElt (x : V.Element) : Prop :=
  ∀ {I : Type u} [Nonempty I] (d : I → V.Element) (hdir : ∀ i j, ∃ k, d i ≤ d k ∧ d j
    ≤ d k),
    x ≤ iSupDirected d hdir → ∃ i, x ≤ d i

theorem principal_le_of_mem {y : V.Element} {Z : Set ?} (hZ : y.mem Z) :
    V.principal (y.sub hZ) ≤ y :=
  fun _ ⟨hW, hZW⟩ => y.up_mem hZ hW hZW

theorem principalFamily_directed (x : V.Element) :
    ∀ i j : {X : Set ? // x.mem X}, ∃ k : {X : Set ? // x.mem X},
      V.principal (x.sub i.2) ≤ V.principal (x.sub k.2) ∧
        V.principal (x.sub j.2) ≤ V.principal (x.sub k.2) :=
  fun i j => ⟨⟨i.1 ∩ j.1, x.inter_mem i.2 j.2⟩,
    (V.principal_le_iff (x.sub i.2) (x.sub (x.inter_mem i.2 j.2))).mpr Set.
    inter_subset_left,
    (V.principal_le_iff (x.sub j.2) (x.sub (x.inter_mem i.2 j.2))).mpr Set.
    inter_subset_right⟩

theorem eq_iSupDirected_principal (x : V.Element) :
    x = iSupDirected (fun i : {X : Set ? // x.mem X} => V.principal (x.sub i.2))
      (principalFamily_directed x) := by
  apply Element.ext
  intro Z
  rw [mem_iSupDirected]
  constructor
  · intro hZ; exact ⟨⟨Z, hZ⟩, (V.mem_principal _).mpr ⟨x.sub hZ, subset_rfl⟩⟩
  · rintro ⟨⟨X, hX⟩, hZ'⟩
    obtain ⟨hZmem, hXZ⟩ := (V.mem_principal _).mp hZ'
    exact x.up_mem hX hZmem hXZ

theorem eq_principal_of_isCompactElt {x : V.Element} (hx : IsCompactElt x) :
    ∃ (X : Set ?) (hX : V.mem X), x = V.principal hX := by
  have hle : x ≤ iSupDirected (fun i : {X : Set ? // x.mem X} => V.principal (x.sub i
    .2))
```

```
      (principalFamily_directed x) := le_of_eq (eq_iSupDirected_principal x)
  obtain ⟨i0, hi0⟩ :=
    hx (fun i : {X : Set ? // x.mem X} => V.principal (x.sub i.2)) (
    principalFamily_directed x) hle
  exact ⟨i0.1, x.sub i0.2, le_antisymm hi0 (principal_le_of_mem i0.2)⟩

theorem principal_compact {X : Set ?} (hX : V.mem X) {I : Type*} [Nonempty I]
    (d : I → V.Element) (hdir : ∀ i j, ∃ k, d i ≤ d k ∧ d j ≤ d k)
    (h : V.principal hX ≤ iSupDirected d hdir) : ∃ i, V.principal hX ≤ d i := by
  obtain ⟨i, hi⟩ := (mem_iSupDirected d hdir).mp (h X ⟨hX, subset_rfl⟩)
  exact ⟨i, fun _ ⟨hW, hXW⟩ => (d i).up_mem hi hW hXW⟩

theorem principal_isCompactElt {X : Set ?} (hX : V.mem X) : IsCompactElt (V.principal
    hX) :=
  fun d hdir h => principal_compact hX d hdir h

end Algebraic

theorem fixedNbhd_subsystem (a : ApproximableMap E E) : fixedNbhd a ? E where
  master_eq := rfl
  sub := fun h => h.1
  inter_closed := by
    rintro X Y ⟨hXE, hXa⟩ ⟨hYE, hYa⟩ hEXY
    have h1 : a.rel (X ∩ Y) X := a.mono hXa Set.inter_subset_left subset_rfl hEXY hXE
    have h2 : a.rel (X ∩ Y) Y := a.mono hYa Set.inter_subset_right subset_rfl hEXY hYE
    exact ⟨hEXY, a.inter_right h1 h2⟩

theorem mem_of_exists_rel_self {a : ApproximableMap E E} (x : E.Element) {Y : Set α}
    (hYE : E.mem Y) {X : Set α} (hXx : x.mem X) (hXY : X ⊆ Y) (hXaX : a.rel X X) :
    (a.toElementMap x).mem Y := by
  have hX : E.mem X := a.rel_dom hXaX
  have hmem : (a.toElementMap (E.principal hX)).mem X := (a.rel_iff_mem_principal hX).
    mp hXaX
  have hle : E.principal hX ≤ x := principal_le_of_mem hXx
  have hXmem : (a.toElementMap x).mem X := (a.toElementMap_mono hle) X hmem
  exact (a.toElementMap x).up_mem hXmem hYE hXY

theorem isFinitaryProjection_of_formula (a : ApproximableMap E E)
    (hii : ∀ (x : E.Element) {Y : Set α}, (a.toElementMap x).mem Y ↔
      E.mem Y ∧ ∃ X, x.mem X ∧ X ⊆ Y ∧ a.rel X X) :
    FinitaryProjection a := by
  have heq : a = Subsystem.retractionOfSubsystem (fixedNbhd_subsystem a) := by
    apply ApproximableMap.ext
    intro X Z
    rw [Subsystem.retractionOfSubsystem_rel]
    constructor
    · intro hr
      have hX : E.mem X := a.rel_dom hr
      have hmem : (a.toElementMap (E.principal hX)).mem Z := (a.rel_iff_mem_principal
    hX).mp hr
      obtain ⟨hZE, Y, hXY, hYZ, hYa⟩ := hii (E.principal hX) |>.mp hmem
      exact ⟨hX, hZE, Y, ⟨a.rel_dom hYa, hYa⟩, (E.mem_principal hX).mp hXY |>.2, hYZ⟩
    · rintro ⟨hX, hZ, Y, ⟨_, hYa⟩, hXY, hYZ⟩
      exact (a.rel_iff_mem_principal hX).mpr
        (mem_of_exists_rel_self (E.principal hX) hZ (E.mem_principal hX |>.mpr ⟨a.
    rel_dom hYa, hXY⟩))
```

```
          hYZ hYa)
  rw [heq]
  exact Subsystem.isFinitaryProjection_retractionOfSubsystem (fixedNbhd_subsystem a)

section HardDirection

variable {a : ApproximableMap E E}

variable {β : Type u} {F : NeighborhoodSystem β} (e : FixSet a ≃o F.Element)

def sectionMap : ApproximableMap F E :=
  ofMono (fun _X hX => (e.symm (F.principal hX)).1) (fun _ _ hX hX' hXX' =>
    e.symm.monotone ((F.principal_le_iff hX hX').mpr hXX'))

theorem toElementMap_iSupDirected_fixed {I : Type u} [Nonempty I] (d : I → E.Element)
    (hdir : ∀ i j, ∃ k, d i ≤ d k ∧ d j ≤ d k) (hfix : ∀ i, a.toElementMap (d i) = d i
    ) :
    a.toElementMap (iSupDirected d hdir) = iSupDirected d hdir := by
  rw [toElementMap_iSupDirected]
  apply Element.ext
  intro Z
  rw [mem_iSupDirected, mem_iSupDirected]
  simp_rw [hfix]

@[simp] theorem toElementMap_sectionMap_principal {X : Set β} (hX : F.mem X) :
    (sectionMap e).toElementMap (F.principal hX) = (e.symm (F.principal hX)).1 :=
  toElementMap_ofMono_principal _ _ X hX

theorem toElementMap_sectionMap (w' : F.Element) :
    (sectionMap e).toElementMap w' = (e.symm w').1 := by
  have hdirC : ∀ i j : {X : Set β // w'.mem X}, ∃ k : {X : Set β // w'.mem X},
      (e.symm (F.principal (w'.sub i.2))).1 ≤ (e.symm (F.principal (w'.sub k.2))).1 ∧
        (e.symm (F.principal (w'.sub j.2))).1 ≤ (e.symm (F.principal (w'.sub k.2))).1
    :=
    fun i j => ⟨⟨i.1 ∩ j.1, w'.inter_mem i.2 j.2⟩,
      e.symm.monotone ((F.principal_le_iff (w'.sub i.2) (w'.sub (w'.inter_mem i.2 j.2)
    )).mpr
        Set.inter_subset_left),
      e.symm.monotone ((F.principal_le_iff (w'.sub j.2) (w'.sub (w'.inter_mem i.2 j.2)
    )).mpr
        Set.inter_subset_right)⟩
  have hS_eq : (sectionMap e).toElementMap w' =
      iSupDirected (fun i : {X : Set β // w'.mem X} => (e.symm (F.principal (w'.sub i
    .2))).1)
        hdirC := by
    apply Element.ext
    intro Z
    rw [toElementMap_mem_iff_principal, mem_iSupDirected]
    constructor
    · rintro ⟨X, hX, hZ⟩; rw [toElementMap_sectionMap_principal] at hZ; exact ⟨⟨X, hX⟩,
    hZ⟩
    · rintro ⟨⟨X, hX⟩, hZ⟩; exact ⟨X, hX, by rwa [toElementMap_sectionMap_principal]⟩
  have hfixC : ∀ i : {X : Set β // w'.mem X},
      a.toElementMap (e.symm (F.principal (w'.sub i.2))).1 =
        (e.symm (F.principal (w'.sub i.2))).1 :=
    fun i => (e.symm (F.principal (w'.sub i.2))).2
```

```
  apply le_antisymm
  · rw [hS_eq]
    apply iSupDirected_le
    intro i
    exact e.symm.monotone (principal_le_of_mem i.2)
  · have hSfix : a.toElementMap ((sectionMap e).toElementMap w') =
        (sectionMap e).toElementMap w' := by
      rw [hS_eq]; exact toElementMap_iSupDirected_fixed _ hdirC hfixC
    have key : w' ≤ e ⟨(sectionMap e).toElementMap w', hSfix⟩ := by
      refine le_trans (le_of_eq (eq_iSupDirected_principal w'))
        (iSupDirected_le _ (principalFamily_directed w') ?_)
      intro i
      have : e.symm (F.principal (w'.sub i.2)) ≤ ⟨(sectionMap e).toElementMap w',
    hSfix⟩ := by
        show (e.symm (F.principal (w'.sub i.2))).1 ≤ (sectionMap e).toElementMap w'
        rw [hS_eq]
        exact le_iSupDirected _ hdirC i
      have := e.monotone this
      rwa [e.apply_symm_apply] at this
    have := e.symm.monotone key
    rwa [e.symm_apply_apply] at this

theorem e_apply_iSupDirected_fixed {I : Type u} [Nonempty I] (d : I → E.Element)
    (hdir : ∀ i j, ∃ k, d i ≤ d k ∧ d j ≤ d k) (hfix : ∀ i, a.toElementMap (d i) = d i
    )
    (hSfix : a.toElementMap (iSupDirected d hdir) = iSupDirected d hdir)
    (hdir' : ∀ i j : I, ∃ k, e ⟨d i, hfix i⟩ ≤ e ⟨d k, hfix k⟩ ∧
      e ⟨d j, hfix j⟩ ≤ e ⟨d k, hfix k⟩) :
    e ⟨iSupDirected d hdir, hSfix⟩ = iSupDirected (fun i => e ⟨d i, hfix i⟩) hdir' := by
  apply le_antisymm
  · have hstep : (⟨iSupDirected d hdir, hSfix⟩ : FixSet a) ≤
        e.symm (iSupDirected (fun i => e ⟨d i, hfix i⟩) hdir') := by
      show iSupDirected d hdir ≤ _
      apply iSupDirected_le
      intro i
      show d i ≤ _
      have hle : e ⟨d i, hfix i⟩ ≤ iSupDirected (fun i => e ⟨d i, hfix i⟩) hdir' :=
        le_iSupDirected (fun i => e ⟨d i, hfix i⟩) hdir' i
      have := e.symm.monotone hle
      rwa [e.symm_apply_apply] at this
    have := e.monotone hstep
    rwa [e.apply_symm_apply] at this
  · apply iSupDirected_le
    intro i
    exact e.monotone (le_iSupDirected d hdir i)

theorem toElementMap_idem {a : ApproximableMap E E} (ha : Retraction a) (z : E.Element
    ) :
    a.toElementMap (a.toElementMap z) = a.toElementMap z := by
  rw [← toElementMap_comp a a z, ha.out]

theorem exists_principal_eq_of_isRetraction_le_idMap (ha_ret : Retraction a)
    (ha_proj : a ≤ idMap E) {Y : Set β} (hY : F.mem Y) :
    ∃ (X : Set α) (hX : E.mem X), (e.symm (F.principal hY)).1 = E.principal hX := by
  apply eq_principal_of_isCompactElt
  intro I _ d hdir hle
```

```
  set d2 : I → E.Element := fun i => a.toElementMap (d i) with hd2
  have hdir2 : ∀ i j, ∃ k, d2 i ≤ d2 k ∧ d2 j ≤ d2 k := fun i j => by
    obtain ⟨k, hik, hjk⟩ := hdir i j
    exact ⟨k, a.toElementMap_mono hik, a.toElementMap_mono hjk⟩
  have hfix2 : ∀ i, a.toElementMap (d2 i) = d2 i := fun i => toElementMap_idem ha_ret (
    d i)
  have hle2 : (e.symm (F.principal hY)).1 ≤ iSupDirected d2 hdir2 := by
    calc (e.symm (F.principal hY)).1 = a.toElementMap (e.symm (F.principal hY)).1 :=
          (e.symm (F.principal hY)).2.symm
      _ ≤ a.toElementMap (iSupDirected d hdir) := a.toElementMap_mono hle
      _ = iSupDirected d2 hdir2 := toElementMap_iSupDirected a d hdir
  have hSfix2 : a.toElementMap (iSupDirected d2 hdir2) = iSupDirected d2 hdir2 :=
    toElementMap_iSupDirected_fixed d2 hdir2 hfix2
  have hdir2' : ∀ i j : I, ∃ k, e ⟨d2 i, hfix2 i⟩ ≤ e ⟨d2 k, hfix2 k⟩ ∧
      e ⟨d2 j, hfix2 j⟩ ≤ e ⟨d2 k, hfix2 k⟩ := fun i j => by
    obtain ⟨k, hik, hjk⟩ := hdir2 i j
    exact ⟨k, e.monotone hik, e.monotone hjk⟩
  have hle3 : F.principal hY ≤ iSupDirected (fun i => e ⟨d2 i, hfix2 i⟩) hdir2' := by
    have h1 : e ⟨(e.symm (F.principal hY)).1, (e.symm (F.principal hY)).2⟩ ≤
        e ⟨iSupDirected d2 hdir2, hSfix2⟩ := e.monotone hle2
    rwa [Subtype.coe_eta, e.apply_symm_apply,
      e_apply_iSupDirected_fixed e d2 hdir2 hfix2 hSfix2 hdir2'] at h1
  obtain ⟨i0, hi0⟩ := principal_isCompactElt hY _ _ hle3
  have hi0' : e.symm (F.principal hY) ≤ ⟨d2 i0, hfix2 i0⟩ := by
    have := e.symm.monotone hi0
    rwa [e.symm_apply_apply] at this
  have hfinal : (e.symm (F.principal hY)).1 ≤ d2 i0 := hi0'
  have hd2le : d2 i0 ≤ d i0 := by
    have := (le_iff_toElementMap_le.mp ha_proj) (d i0)
    rwa [toElementMap_idMap] at this
  exact ⟨i0, hfinal.trans hd2le⟩

theorem formula_of_isFinitaryProjection {a : ApproximableMap E E} (h :
    FinitaryProjection a) :
    ∀ (x : E.Element) {Y : Set α}, (a.toElementMap x).mem Y ↔
      E.mem Y ∧ ∃ X, x.mem X ∧ X ⊆ Y ∧ a.rel X X := by
  obtain ⟨⟨ha_ret, ha_proj⟩, β, F, ⟨e⟩⟩ := h.out
  intro x Y
  constructor
  · intro hY
    have hw : a.toElementMap (a.toElementMap x) = a.toElementMap x :=
    toElementMap_idem ha_ret x
    set w : E.Element := a.toElementMap x with hw_def
    have hYw : w.mem Y := hY
    have hclaim : (sectionMap e).toElementMap (e ⟨w, hw⟩) = w := by
      rw [toElementMap_sectionMap e (e ⟨w, hw⟩), e.symm_apply_apply]
    have hYsec : ((sectionMap e).toElementMap (e ⟨w, hw⟩)).mem Y := by rw [hclaim];
    exact hYw
    obtain ⟨W, hW, hYW⟩ := (toElementMap_mem_iff_principal (sectionMap e) (e ⟨w, hw⟩)).
    mp hYsec
    rw [toElementMap_sectionMap_principal] at hYW
    obtain ⟨X, hX, hXeq⟩ :=
      exists_principal_eq_of_isRetraction_le_idMap e ha_ret ha_proj ((e ⟨w, hw⟩).sub hW
    )
    rw [hXeq] at hYW
    obtain ⟨hYE, hXY⟩ := (E.mem_principal hX).mp hYW
```

```
    have hfixX : a.toElementMap (E.principal hX) = E.principal hX := by
      rw [← hXeq]; exact (e.symm (F.principal ((e ⟨w, hw⟩).sub hW))).2
    have haXX : a.rel X X := (a.rel_iff_mem_principal hX).mpr (by
      rw [hfixX]; exact (E.mem_principal hX).mpr ⟨hX, subset_rfl⟩)
    have hprincipal_le_w : E.principal hX ≤ w := by
      have h1 : F.principal ((e ⟨w, hw⟩).sub hW) ≤ e ⟨w, hw⟩ :=
        principal_le_of_mem hW
      have h2 : e.symm (F.principal ((e ⟨w, hw⟩).sub hW)) ≤ e.symm (e ⟨w, hw⟩) :=
        e.symm.monotone h1
      have h3 : (e.symm (F.principal ((e ⟨w, hw⟩).sub hW))).1 ≤ w := by
        have := h2
        rwa [e.symm_apply_apply] at this
      rwa [hXeq] at h3
    have hw_le_x : w ≤ x := by
      have := (le_iff_toElementMap_le.mp ha_proj) x
      rwa [toElementMap_idMap] at this
    have hXx : x.mem X := hprincipal_le_w.trans hw_le_x X ((E.mem_principal hX).mpr ⟨hX
    , subset_rfl⟩)
    exact ⟨hYE, X, hXx, hXY, haXX⟩
  · rintro ⟨hYE, X, hXx, hXY, hXaX⟩
    exact mem_of_exists_rel_self x hYE hXx hXY hXaX

end HardDirection

theorem finitaryProjection_iff_formula (a : ApproximableMap E E) :
    FinitaryProjection a ↔ ∀ (x : E.Element) {Y : Set α}, (a.toElementMap x).mem Y ↔
      E.mem Y ∧ ∃ X, x.mem X ∧ X ⊆ Y ∧ a.rel X X :=
  ⟨fun h => formula_of_isFinitaryProjection h, fun hii =>
    isFinitaryProjection_of_formula a hii⟩

end

end DomainTheory.NeighbourhoodSystems
```

4.9.8 Fix of O

The two-element system O, together with check and fade, realizes the first half of Examples 8.4: a retraction onto O whose fixed points are order-isomorphic to O when a non-bottom anchor u is chosen.

Theorem (`fin2_zero_subset_zero_one`). The singleton $\{0\}$ on two labels is contained in $\{0, 1\}$.

Proof. By membership of 0. ■

Theorem (`fin2_zero_ne_zero_one`). The singleton $\{0\}$ is not the full master $\{0, 1\}$.

Proof. By membership of 1. ■

Definition (`O`). The two-element neighbourhood system admits exactly the singleton $\{0\}$ and the master $\{0, 1\}$.

Proof. By case analysis on intersections and sub-master constraints. ■

Theorem (`O_master_eq`). The master of O is $\{0, 1\}$.

Proof. By definition. ■

Theorem (`O_mem_iff`). Membership in O is equivalent to being either $\{0\}$ or $\{0, 1\}$.

Proof. By definition. ■

Theorem (`O_mem_zero`). The singleton $\{0\}$ is a neighbourhood of O.

Proof. By the first disjunct. ■

Definition (`Otop`). The top element of O is the principal over $\{0\}$.

Proof. By the principal constructor. ■

Theorem (`mem_Otop`). A set lies in the top element exactly when it is an O-neighbourhood containing $\{0\}$.

Proof. By the principal membership law. ■

Definition (`check`). The check combinator relates a D-neighbourhood X to an O-neighbourhood Y when Y is the master or X is not the master of D.

Proof. By the relational axioms for approximable maps. ■

Theorem (`check_rel`). The check relation is equivalent to D-membership of X, O-membership of Y, and either Y is the master or X is proper.

Proof. By definition. ■

Theorem (`exists_mem_ne_master_of_ne_bot`). Every non-bottom element of D lies in some neighbourhood distinct from the master.

Proof. By contrapositive identification of bottom. ■

Definition (`fade2`). The binary fade relates the first factor in O, a D-neighbourhood, and a target neighbourhood so that either the target is the D-master or the O-factor is $\{0\}$ and the D-factor is contained in the target.

Proof. By the relational axioms for binary approximable maps. ■

Definition (`fade`). Fade on the product $O \times D$ is the curry of the binary fade.

Proof. By pairing of binary maps. ■

Definition (`a`). For a chosen anchor u in D, the endomap a is fade after pairing check with the constant map to u.

Proof. By composition. ■

Definition (`fixOfO`). For $t \in O$, the element $\mathrm{fixOfO}(t)$ lies in a D-neighbourhood Z when Z is the master or t sees $\{0\}$ and u lies in Z.

Proof. By the element axioms. ■

Theorem (`mem_fixOfO`). Membership in $\mathrm{fixOfO}(t)$ is characterized by D-membership and either master equality or top-like behaviour of t together with membership of u.

Proof. By definition. ■

Theorem (`O_le_iff`). In O, order is determined by extending membership at $\{0\}$.

Proof. By comparison of the two possible neighbourhoods. ■

Theorem (`fixOfO_mono`). The map $t \mapsto \mathrm{fixOfO}(t)$ is monotone in t.

Proof. By the order characterization on O and the membership definition. ■

Theorem (`fixOfO_ne_bot_iff`). For non-bottom u, $\mathrm{fixOfO}(t)$ is not bottom if and only if t lies in $\{0\}$.

Proof. By element extension and the proper-neighbourhood witness for u. ■

Theorem (`mem_toElementMap_check`). The check of an element lies in $Y \subseteq O$ when Y is in O and either Y is the master or the element is not bottom.

Proof. By the element-map characterization and the proper-neighbourhood lemma. ■

Theorem (`check_toElementMap_fixOfO`). For non-bottom u, applying check to $\mathrm{fixOfO}(t)$ returns t.

Proof. By element extension and the non-bottom criterion. ■

Theorem (`mem_toElementMap2_fade`). The binary fade at (t, y) lies in Z when Z is a D-neighbourhood and either Z is the master or t sees $\{0\}$ and y lies in Z.

Proof. By the binary element-map law. ■

Theorem (`toElementMap_ofMap2_pair`). The element map of a curried binary combinator on a pair agrees with the binary element map at the paired arguments.

Proof. By the pairing identities for binary maps. ■

Theorem (`mem_toElementMap_a`). The endomap a sends x into Z when Z is in D and either Z is the master or x is not bottom and u lies in Z.

Proof. By unfolding composition, fade, check, and the pairing lemmas. ■

Theorem (`fixOfO_check_toElementMap`). fixOfO applied to the check of x equals the action of a on x.

Proof. By element extension and the characterization of check at $\{0\}$. ■

Theorem (`toElementMap_a_fixOfO`). For non-bottom u, a fixes every fixOfO(t).

Proof. By element extension and the membership laws. ■

Definition (`fixIso`). For non-bottom u, fixed points of a are order-isomorphic to elements of O via fixOfO and check.

Proof. By the inverse laws, fixed-point equation, and monotonicity on both sides. ■

Theorem (`Otop_ne_Obot`). The top of O is not bottom.

Proof. By membership at $\{0\}$. ■

Theorem (`a_bot`). The endomap a sends bottom to bottom.

Proof. By element extension and master-only outcomes. ■

Theorem (`a_of_ne_bot`). On non-bottom arguments, a returns the anchor u.

Proof. By element extension and the fade–check scheme. ■

Theorem (`isRetraction_a`). For non-bottom u, the map a is a retraction.

Proof. By case analysis on bottom versus non-bottom and the actions above. ■

Theorem (`example84a`). For non-bottom u, a is a retraction and its fixed-point set is order-isomorphic to O.

Proof. By the retraction theorem and the fixed-point isomorphism. ■

Theorem (`check_toElementMap_mem_zero_iff`). Check of an element lies in $\{0\}$ if and only if the element is not bottom.

Proof. By the element-map characterization at $\{0\}$. ■

Theorem (`mem_toElementMap_fade`). On a product $O \times G$, fade at (t, w) lies in Z when Z is in G and either Z is the master or t sees $\{0\}$ and w lies in Z.

Proof. By the specialized fade membership law. ■

`fixOfO.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.smashProduct
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_3

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap ApproximableMap₂

theorem fin2_zero_subset_zero_one : ({0} : Set (Fin 2)) ⊆ ({0, 1} : Set (Fin 2)) := by
  intro x hx; simp only [Set.mem_singleton_iff] at hx; subst hx; simp

theorem fin2_zero_ne_zero_one : ({0} : Set (Fin 2)) = ({0, 1} : Set (Fin 2)) := by
  intro h
  have h1 : (1 : Fin 2) ∈ ({0, 1} : Set (Fin 2)) := by simp
  rw [← h] at h1
  simp at h1

def O : NeighborhoodSystem (Fin 2) where
  mem X := X = ({0} : Set (Fin 2)) ∨ X = ({0, 1} : Set (Fin 2))
  master := ({0, 1} : Set (Fin 2))
  master_nonempty := ⟨0, by simp⟩
  master_mem := Or.inr rfl
  inter_mem := by
    rintro X Y Z (rfl | rfl) (rfl | rfl) _ _
    · exact Or.inl (Set.inter_self _)
    · exact Or.inl (Set.inter_eq_left.mpr fin2_zero_subset_zero_one)
    · exact Or.inl (Set.inter_eq_right.mpr fin2_zero_subset_zero_one)
    · exact Or.inr (Set.inter_self _)
  sub_master := by
    rintro X (rfl | rfl)
    · exact fin2_zero_subset_zero_one
    · exact subset_rfl

theorem O_master_eq : O.master = ({0, 1} : Set (Fin 2)) := rfl

@[simp] theorem O_mem_iff {X : Set (Fin 2)} :
    O.mem X ↔ X = ({0} : Set (Fin 2)) ∨ X = ({0, 1} : Set (Fin 2)) := Iff.rfl

theorem O_mem_zero : O.mem ({0} : Set (Fin 2)) := Or.inl rfl

def Otop : O.Element := O.principal O_mem_zero

@[simp] theorem mem_Otop {Y : Set (Fin 2)} : Otop.mem Y ↔ O.mem Y ∧ ({0} : Set (Fin 2)
    ) ⊆ Y :=
  Iff.rfl

section
variable {α : Type*} {D : NeighborhoodSystem α}

def check : ApproximableMap D O where
  rel X Y := D.mem X ∧ O.mem Y ∧ (Y = ({0, 1} : Set (Fin 2)) ∨ X = D.master)
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := ⟨D.master_mem, O.master_mem, Or.inl rfl⟩
  inter_right := by
    rintro X Y Y' ⟨hX, hY, hY1⟩ ⟨-, hY', hY1'⟩
    refine ⟨hX, ?_, ?_⟩
    · rcases hY with rfl | rfl <;> rcases hY' with rfl | rfl
      · exact Or.inl (Set.inter_self _)
```

```
      · exact Or.inl (Set.inter_eq_left.mpr fin2_zero_subset_zero_one)
      · exact Or.inl (Set.inter_eq_right.mpr fin2_zero_subset_zero_one)
      · exact Or.inr (Set.inter_self _)
    · rcases hY1 with h1 | hXne
      · rcases hY1' with h1' | hXne'
        · left; rw [h1, h1']; simp
        · right; exact hXne'
      · right; exact hXne
  mono := by
    rintro X X' Y Y' ⟨hX, hY, hY1⟩ hXX' hYY' hX' hY'
    refine ⟨hX', hY', ?_⟩
    rcases hY1 with hYeq | hXne
    · left
      subst hYeq
      rcases hY' with rfl | rfl
      · exfalso
        have h1 : (1 : Fin 2) ∈ ({0, 1} : Set (Fin 2)) := by simp
        have h2 : (1 : Fin 2) ∈ ({0} : Set (Fin 2)) := hYY' h1
        simp at h2
      · rfl
    · right
      intro hX'eq
      apply hXne
      exact Set.Subset.antisymm (D.sub_master hX) (hX'eq ▸ hXX')

@[simp] theorem check_rel {X : Set α} {Y : Set (Fin 2)} :
    (check : ApproximableMap D O).rel X Y ↔
      D.mem X ∧ O.mem Y ∧ (Y = ({0, 1} : Set (Fin 2)) ∨ X = D.master) :=
  Iff.rfl

theorem exists_mem_ne_master_of_ne_bot {y : D.Element} (hy : y = D.bot) :
    ∃ Z, y.mem Z ∧ Z = D.master := by
  by_contra hcon
  push Not at hcon
  apply hy
  apply Element.ext
  intro Z
  rw [mem_bot]
  exact ⟨fun hZ => hcon Z hZ, fun hZ => hZ ▸ y.master_mem⟩

def fade₂ : ApproximableMap₂ O D D where
  rel X Y Z := O.mem X ∧ D.mem Y ∧ D.mem Z ∧ (Z = D.master ∨ (X = ({0} : Set (Fin 2))
    ∧ Y ⊆ Z))
  rel_dom₀ h := h.1
  rel_dom₁ h := h.2.1
  rel_cod h := h.2.2.1
  master_rel := ⟨O.master_mem, D.master_mem, D.master_mem, Or.inl rfl⟩
  inter_right := by
    rintro X Y Z Z' ⟨hX, hY, hZ, hZ1⟩ ⟨-, -, hZ', hZ1'⟩
    refine ⟨hX, hY, ?_, ?_⟩
    · rcases hZ1 with rfl | ⟨-, hYZ⟩
      · rwa [Set.inter_eq_right.mpr (D.sub_master hZ')]
      · rcases hZ1' with rfl | ⟨-, hYZ'⟩
        · rwa [Set.inter_eq_left.mpr (D.sub_master hZ)]
        · exact D.inter_mem hZ hZ' hY (Set.subset_inter hYZ hYZ')
    · rcases hZ1 with rfl | ⟨hXeq, hYZ⟩
```

```
      · rcases hZ1' with rfl | ⟨hXeq', hYZ'⟩
        · left; exact Set.inter_self _
        · right
          refine ⟨hXeq', ?_⟩
          rw [Set.inter_eq_right.mpr (D.sub_master hZ')]
          exact hYZ'
      · right
        refine ⟨hXeq, ?_⟩
        rcases hZ1' with rfl | ⟨-, hYZ'⟩
        · rw [Set.inter_eq_left.mpr (D.sub_master hZ)]
          exact hYZ
        · exact Set.subset_inter hYZ hYZ'
  mono := by
    rintro X X' Y Y' Z Z' ⟨hX, hY, hZ, hZ1⟩ hXX' hYY' hZZ' hX' hY' hZ'
    refine ⟨hX', hY', hZ', ?_⟩
    rcases hZ1 with rfl | ⟨hXeq, hYZ⟩
    · left; exact Set.Subset.antisymm (D.sub_master hZ') hZZ'
    · have hX'eq : X' = ({0} : Set (Fin 2)) := by
        rcases hX' with rfl | rfl
        · rfl
        · exfalso
          have h1 : (1 : Fin 2) ∈ ({0, 1} : Set (Fin 2)) := by simp
          have h2 : (1 : Fin 2) ∈ X := hXX' h1
          rw [hXeq] at h2
          simp at h2
      right
      exact ⟨hX'eq, hYY'.trans (hYZ.trans hZZ')⟩

def fade : ApproximableMap (prod O D) D := ofMap₂ fade₂

end

section
variable {α : Type*} {D : NeighborhoodSystem α}
variable (u : D.Element) (hu : u = D.bot)

def a : ApproximableMap D D := fade.comp (paired check (constMap D u))

def fixOfO (t : O.Element) : D.Element where
  mem Z := D.mem Z ∧ (Z = D.master ∨ (t.mem ({0} : Set (Fin 2)) ∧ u.mem Z))
  sub h := h.1
  master_mem := ⟨D.master_mem, Or.inl rfl⟩
  inter_mem := by
    rintro Z Z' ⟨hZ, hZ1⟩ ⟨hZ', hZ1'⟩
    refine ⟨?_, ?_⟩
    · rcases hZ1 with rfl | ⟨-, huZ⟩
      · rwa [Set.inter_eq_right.mpr (D.sub_master hZ')]
      · rcases hZ1' with rfl | ⟨-, huZ'⟩
        · rwa [Set.inter_eq_left.mpr (D.sub_master hZ)]
        · exact u.sub (u.inter_mem huZ huZ')
    · rcases hZ1 with rfl | ⟨ht, huZ⟩
      · rcases hZ1' with rfl | ⟨ht', huZ'⟩
        · left; exact Set.inter_self _
        · right
          refine ⟨ht', ?_⟩
          rw [Set.inter_eq_right.mpr (D.sub_master hZ')]
```

```
          exact huZ'
      · right
        refine ⟨ht, ?_⟩
        rcases hZ1' with rfl | ⟨-, huZ'⟩
        · rw [Set.inter_eq_left.mpr (D.sub_master hZ)]
          exact huZ
        · exact u.inter_mem huZ huZ'
  up_mem := by
    rintro Z Z' ⟨hZ, hZ1⟩ hZ'D hZZ'
    refine ⟨hZ'D, ?_⟩
    rcases hZ1 with rfl | ⟨ht, huZ⟩
    · left; exact Set.Subset.antisymm (D.sub_master hZ'D) hZZ'
    · right; exact ⟨ht, u.up_mem huZ hZ'D hZZ'⟩

@[simp] theorem mem_fixOfO {t : O.Element} {Z : Set α} :
    (fixOfO u t).mem Z ↔ D.mem Z ∧ (Z = D.master ∨ (t.mem ({0} : Set (Fin 2)) ∧ u.mem
     Z)) :=
  Iff.rfl

theorem O_le_iff {t t' : O.Element} :
    t ≤ t' ↔ (t.mem ({0} : Set (Fin 2)) → t'.mem ({0} : Set (Fin 2))) := by
  constructor
  · intro h; exact h _
  · intro h Y hY
    rcases O_mem_iff.mp (t.sub hY) with rfl | rfl
    · exact h hY
    · exact t'.master_mem

theorem fixOfO_mono {t t' : O.Element} (h : t ≤ t') : fixOfO u t ≤ fixOfO u t' := by
  intro Z hZ
  rw [mem_fixOfO] at hZ ⊢
  obtain ⟨hZD, hZeq | ⟨ht0, huZ⟩⟩ := hZ
  · exact ⟨hZD, Or.inl hZeq⟩
  · exact ⟨hZD, Or.inr ⟨O_le_iff.mp h ht0, huZ⟩⟩

theorem fixOfO_ne_bot_iff (hu : u = D.bot) {t : O.Element} :
    fixOfO u t = D.bot ↔ t.mem ({0} : Set (Fin 2)) := by
  constructor
  · intro hne
    by_contra ht0
    apply hne
    apply Element.ext
    intro Z
    rw [mem_fixOfO, mem_bot]
    constructor
    · rintro ⟨-, rfl | ⟨ht0', -⟩⟩
      · rfl
      · exact absurd ht0' ht0
    · rintro rfl
      exact ⟨D.master_mem, Or.inl rfl⟩
  · intro ht0 hbot
    obtain ⟨Z, huZ, hZne⟩ := exists_mem_ne_master_of_ne_bot hu
    have : (fixOfO u t).mem Z := ⟨u.sub huZ, Or.inr ⟨ht0, huZ⟩⟩
    rw [hbot, mem_bot] at this
    exact hZne this
```

```
end

section
variable {α : Type*} {D : NeighborhoodSystem α}

theorem mem_toElementMap_check {x : D.Element} {Y : Set (Fin 2)} :
    (check.toElementMap x).mem Y ↔ O.mem Y ∧ (Y = ({0, 1} : Set (Fin 2)) ∨ x = D.bot)
     := by
  rw [mem_toElementMap]
  constructor
  · rintro ⟨X, hxX, hXD, hYO, hcase⟩
    refine ⟨hYO, ?_⟩
    rcases hcase with h | hXne
    · exact Or.inl h
    · refine Or.inr fun hxbot => hXne ?_
      have hxX' : x.mem X := hxX
      rw [hxbot, mem_bot] at hxX'
      exact hxX'
  · rintro ⟨hYO, hYeq | hxne⟩
    · exact ⟨D.master, x.master_mem, D.master_mem, hYO, Or.inl hYeq⟩
    · obtain ⟨X, hxX, hXne⟩ := exists_mem_ne_master_of_ne_bot hxne
      exact ⟨X, hxX, x.sub hxX, hYO, Or.inr hXne⟩
end

section
variable {α : Type*} {D : NeighborhoodSystem α}
variable (u : D.Element) (hu : u = D.bot)

theorem check_toElementMap_fixOfO (hu : u = D.bot) (t : O.Element) :
    check.toElementMap (fixOfO u t) = t := by
  apply Element.ext
  intro Y
  rw [mem_toElementMap_check, fixOfO_ne_bot_iff u hu]
  constructor
  · rintro ⟨hYO, hYeq | ht0⟩
    · rw [hYeq, ← O_master_eq]; exact t.master_mem
    · rcases hYO with rfl | rfl
      · exact ht0
      · rw [← O_master_eq]; exact t.master_mem
  · intro htY
    have hYO : O.mem Y := t.sub htY
    refine ⟨hYO, ?_⟩
    rcases O_mem_iff.mp hYO with rfl | rfl
    · exact Or.inr htY
    · exact Or.inl rfl
end

section
variable {α : Type*} {D : NeighborhoodSystem α}

theorem mem_toElementMap₂_fade (t : O.Element) (y : D.Element) {Z : Set α} :
    (fade₂.toElementMap₂ t y).mem Z ↔
      D.mem Z ∧ (Z = D.master ∨ (t.mem ({0} : Set (Fin 2)) ∧ y.mem Z)) := by
  rw [mem_toElementMap₂]
  constructor
  · rintro ⟨X, Y, htX, hyY, hXO, hYD, hZD, hcase⟩
```

```
    refine ⟨hZD, ?_⟩
    rcases hcase with h | ⟨hXeq, hYZ⟩
    · exact Or.inl h
    · exact Or.inr ⟨hXeq ▸ htX, y.up_mem hyY hZD hYZ⟩
  · rintro ⟨hZD, hZeq | ⟨ht0, hyZ⟩⟩
    · exact ⟨O.master, D.master, t.master_mem, y.master_mem, O.master_mem, D.master_mem
    , hZD,
        Or.inl hZeq⟩
    · exact ⟨({0} : Set (Fin 2)), Z, ht0, hyZ, O_mem_zero, hZD, hZD, Or.inr ⟨rfl,
    subset_rfl⟩⟩

theorem toElementMap_ofMap₂_pair {β ? ? : Type*} {V₀ : NeighborhoodSystem β}
    {V₁ : NeighborhoodSystem ?} {V₂ : NeighborhoodSystem ?} (f₂ : ApproximableMap₂ V₀ V₁
     V₂)
    (p : V₀.Element) (q : V₁.Element) :
    (ofMap₂ f₂).toElementMap (pair p q) = f₂.toElementMap₂ p q := by
  have h := toElementMap₂_toMap₂ (ofMap₂ f₂) p q
  rw [toMap₂_ofMap₂] at h
  exact h.symm

end

section
variable {α : Type*} {D : NeighborhoodSystem α}
variable (u : D.Element) (hu : u = D.bot)

theorem mem_toElementMap_a {x : D.Element} {Z : Set α} :
    ((a u).toElementMap x).mem Z ↔ D.mem Z ∧ (Z = D.master ∨ (x = D.bot ∧ u.mem Z))
    := by
  show ((fade.comp (paired check (constMap D u))).toElementMap x).mem Z ↔ _
  rw [toElementMap_comp, toElementMap_paired, toElementMap_constMap]
  show (fade.toElementMap (pair (check.toElementMap x) u)).mem Z ↔ _
  rw [show fade = ofMap₂ fade₂ from rfl, toElementMap_ofMap₂_pair, mem_toElementMap₂
    _fade,
    mem_toElementMap_check]
  constructor
  · rintro ⟨hZD, hZeq | ⟨⟨-, hcase⟩, huZ⟩⟩
    · exact ⟨hZD, Or.inl hZeq⟩
    · rcases hcase with hYeq | hxne
      · exact absurd hYeq fin2_zero_ne_zero_one
      · exact ⟨hZD, Or.inr ⟨hxne, huZ⟩⟩
  · rintro ⟨hZD, hZeq | ⟨hxne, huZ⟩⟩
    · exact ⟨hZD, Or.inl hZeq⟩
    · exact ⟨hZD, Or.inr ⟨⟨O_mem_zero, Or.inr hxne⟩, huZ⟩⟩

theorem fixOfO_check_toElementMap (x : D.Element) :
    fixOfO u (check.toElementMap x) = (a u).toElementMap x := by
  apply Element.ext
  intro Z
  rw [mem_fixOfO, mem_toElementMap_a]
  have hcheck : (check.toElementMap x).mem ({0} : Set (Fin 2)) ↔ x = D.bot := by
    rw [mem_toElementMap_check]
    constructor
    · rintro ⟨-, h | h⟩
      · exact absurd h fin2_zero_ne_zero_one
      · exact h
```

```
    · intro h; exact ⟨O_mem_zero, Or.inr h⟩
  rw [hcheck]

theorem toElementMap_a_fixOfO (hu : u = D.bot) (t : O.Element) :
    (a u).toElementMap (fixOfO u t) = fixOfO u t := by
  apply Element.ext
  intro Z
  rw [mem_toElementMap_a, mem_fixOfO, fixOfO_ne_bot_iff u hu]

def fixIso (hu : u = D.bot) : O.Element ≃o {y : D.Element // (a u).toElementMap y = y}
    where
  toFun t := ⟨fixOfO u t, toElementMap_a_fixOfO u hu t⟩
  invFun y := check.toElementMap y.1
  left_inv t := check_toElementMap_fixOfO u hu t
  right_inv y := by
    apply Subtype.ext
    show fixOfO u (check.toElementMap y.1) = y.1
    rw [fixOfO_check_toElementMap, y.2]
  map_rel_iff' := by
    intro t t'
    show fixOfO u t ≤ fixOfO u t' ↔ t ≤ t'
    constructor
    · intro h
      have := toElementMap_mono check h
      rwa [check_toElementMap_fixOfO u hu, check_toElementMap_fixOfO u hu] at this
    · exact fixOfO_mono u

end

theorem Otop_ne_Obot : Otop = O.bot := by
  intro h
  have h1 : Otop.mem ({0} : Set (Fin 2)) := ⟨O_mem_zero, subset_rfl⟩
  rw [h, mem_bot, O_master_eq] at h1
  have h2 : (1 : Fin 2) ∈ ({0} : Set (Fin 2)) := by rw [h1]; simp
  simp at h2

section
variable {α : Type*} {D : NeighborhoodSystem α}
variable (u : D.Element) (hu : u = D.bot)

theorem a_bot : (a u).toElementMap D.bot = D.bot := by
  apply Element.ext
  intro Z
  rw [mem_toElementMap_a, mem_bot]
  exact ⟨fun ⟨_, h⟩ => h.resolve_right (fun h' => h'.1 rfl) |>.symm ▸ rfl,
    fun h => ⟨h ▸ D.master_mem, Or.inl h⟩⟩

theorem a_of_ne_bot {x : D.Element} (hx : x = D.bot) : (a u).toElementMap x = u := by
  apply Element.ext
  intro Z
  rw [mem_toElementMap_a]
  constructor
  · rintro ⟨-, hZeq | ⟨-, huZ⟩⟩
    · exact hZeq ▸ u.master_mem
    · exact huZ
  · intro huZ
```

```
    exact ⟨u.sub huZ, Or.inr ⟨hx, huZ⟩⟩

theorem isRetraction_a (hu : u = D.bot) : Retraction (a u) := by
  constructor
  apply ext_of_toElementMap

  intro x
  rw [toElementMap_comp]
  by_cases hx : x = D.bot
  · subst hx; rw [a_bot, a_bot]
  · rw [a_of_ne_bot u hx, a_of_ne_bot u hu]

theorem example84a (hu : u = D.bot) :
    Retraction (a u) ∧ Nonempty (O.Element ≃o {y : D.Element // (a u).toElementMap y =
     y}) :=
  ⟨isRetraction_a u hu, ⟨fixIso u hu⟩⟩

end

section
variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

theorem check_toElementMap_mem_zero_iff {x : D.Element} :
    (check.toElementMap x).mem ({0} : Set (Fin 2)) ↔ x = D.bot := by
  rw [mem_toElementMap_check]
  constructor
  · rintro ⟨-, h | h⟩
    · exact absurd h fin2_zero_ne_zero_one
    · exact h
  · intro h; exact ⟨O_mem_zero, Or.inr h⟩

theorem mem_toElementMap_fade {? : Type*} {G : NeighborhoodSystem ?} (t : O.Element) (
    w : G.Element)
    {Z : Set ?} :
    ((fade (D := G)).toElementMap (pair t w)).mem Z ↔
      G.mem Z ∧ (Z = G.master ∨ (t.mem ({0} : Set (Fin 2)) ∧ w.mem Z)) := by
  rw [show (fade (D := G)) = ofMap₂ (fade₂ (D := G)) from rfl, toElementMap_ofMap₂_pair
    ,
    mem_toElementMap₂_fade]

end

end DomainTheory.NeighbourhoodSystems
```

4.9.9 Strict retraction

Currying check and fade against evaluation yields the strict combinator of Examples 8.4: a projection on the function space whose fixed points are exactly the strict maps, order-isomorphic to the strict function domain.

Definition (`strictEvalFade`). The strict evaluator applies fade on the codomain after pairing the check of the function argument with the value of the function at that argument.

Proof. By composition on the product of the function space with the domain. ■

Definition (`strictRetraction`). The strict combinator is the curry of the strict evaluator.

Proof. By currying. ■

Theorem (`toApproxMap_strictRetraction_mem`). Applying the strict combinator to a filter function and then to y lies in Z when Z is in the codomain and either Z is the master or y is not bottom and the original function value at y lies in Z.

Proof. By unfolding curry, evaluation, fade, and check. ■

Theorem (`strictRetraction_apply_bot`). The strict image of a function sends bottom to bottom in the codomain.

Proof. By element extension and the membership characterization. ■

Theorem (`isStrict_toApproxMap_of_fixed`). A filter function fixed by the strict combinator is strict.

Proof. By the bottom-action characterization of strictness. ■

Theorem (`strictRetraction_apply_of_ne_bot`). On non-bottom arguments, the strict image agrees with the underlying function value.

Proof. By element extension and the membership equivalence. ■

Theorem (`toApproxMap_toFilter`). Converting a filter to an approximable map and back recovers the original map.

Proof. By the function-space equivalence. ■

Theorem (`toFilter_toApproxMap`). Converting an approximable map to a filter and back recovers the original filter.

Proof. By the function-space equivalence. ■

Theorem (`strictRetraction_fixed_of_isStrict`). The strict combinator fixes the filter of every strict approximable map.

Proof. By elementwise agreement on bottom and non-bottom arguments. ■

Definition (`strictRetractionFixIso`). Fixed points of the strict combinator on filters are order-isomorphic to strict approximable maps.

Proof. By conversion between filters and maps, strictness, and the function-space order equivalence. ■

Theorem (`strictRetraction_le_idMap`). The strict combinator lies below the identity on the function space.

Proof. By comparison of element maps and the non-bottom agreement lemma. ■

Theorem (`toApproxMap_injective`). If two filter functions determine the same approximable map, they are equal.

Proof. By recovery through conversion to filters. ■

Theorem (`isRetraction_strictRetraction`). The strict combinator is a retraction.

Proof. By elementwise idempotence on bottom and non-bottom arguments. ■

Theorem (`example84b_strict`). The strict combinator is a projection whose fixed points are order-isomorphic to the strict function domain.

Proof. By the retraction and identity bounds, and the fixed-point isomorphism composed with the strict-domain equivalence. ■

`strictRetraction.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.smashProduct
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_3
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.fixOfO

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
```

```
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap ApproximableMap₂

section
variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

def strictEvalFade : ApproximableMap (prod (funSpace D E) D) E :=
  (fade (D := E)).comp
    (paired ((check (D := D)).comp (proj₁ (funSpace D E) D)) (evalMap D E))

def strictRetraction : ApproximableMap (funSpace D E) (funSpace D E) := curry
    strictEvalFade

theorem toApproxMap_strictRetraction_mem {φ : (funSpace D E).Element} {y : D.Element}
    {Z : Set β} :
    ((toApproxMap (strictRetraction.toElementMap φ)).toElementMap y).mem Z ↔
      E.mem Z ∧ (Z = E.master ∨ (y = D.bot ∧ ((toApproxMap φ).toElementMap y).mem Z))
    := by
  show ((toApproxMap ((curry strictEvalFade).toElementMap φ)).toElementMap y).mem Z ↔
    _
  rw [toElementMap_curry_apply]
  show (((fade (D := E)).comp
    (paired ((check (D := D)).comp (proj₁ (funSpace D E) D)) (evalMap D E))).
    toElementMap
      (pair φ y)).mem Z ↔ _
  rw [toElementMap_comp, toElementMap_paired, toElementMap_comp, toElementMap_proj₁,
    snd_pair,
    evalMap_apply, mem_toElementMap_fade, check_toElementMap_mem_zero_iff]

theorem strictRetraction_apply_bot (φ : (funSpace D E).Element) :
    (toApproxMap (strictRetraction.toElementMap φ)).toElementMap D.bot = E.bot := by
  apply Element.ext
  intro Z
  rw [toApproxMap_strictRetraction_mem, mem_bot]
  constructor
  · rintro ⟨-, rfl | ⟨hne, -⟩⟩
    · rfl
    · exact absurd rfl hne
  · rintro rfl
    exact ⟨E.master_mem, Or.inl rfl⟩

theorem isStrict_toApproxMap_of_fixed {φ : (funSpace D E).Element}
    (hφ : strictRetraction.toElementMap φ = φ) : SmashProduct.IsStrict (toApproxMap φ)
    := by
  rw [SmashProduct.isStrict_iff_apply_bot]
  have h := strictRetraction_apply_bot φ
  rwa [hφ] at h
```

```
theorem strictRetraction_apply_of_ne_bot {y : D.Element} (hy : y = D.bot)
    (φ : (funSpace D E).Element) :
    (toApproxMap (strictRetraction.toElementMap φ)).toElementMap y
      = (toApproxMap φ).toElementMap y := by
  apply Element.ext
  intro Z
  rw [toApproxMap_strictRetraction_mem]
  constructor
  · rintro ⟨-, rfl | ⟨-, hZ⟩⟩
    · exact ((toApproxMap φ).toElementMap y).master_mem
    · exact hZ
  · intro hZ
    exact ⟨((toApproxMap φ).toElementMap y).sub hZ, Or.inr ⟨hy, hZ⟩⟩

theorem toApproxMap_toFilter (f : ApproximableMap D E) : toApproxMap (toFilter f) = f
    :=
  (funSpaceEquiv D E).right_inv f

theorem toFilter_toApproxMap (φ : (funSpace D E).Element) : toFilter (toApproxMap φ) =
     φ :=
  (funSpaceEquiv D E).left_inv φ

theorem strictRetraction_fixed_of_isStrict {f : ApproximableMap D E} (hf :
    SmashProduct.IsStrict f) :
    strictRetraction.toElementMap (toFilter f) = toFilter f := by
  have hbot : f.toElementMap D.bot = E.bot := SmashProduct.isStrict_iff_apply_bot.mp
    hf
  have key : toApproxMap (strictRetraction.toElementMap (toFilter f)) = f := by
    apply ext_of_toElementMap
    intro y
    by_cases hy : y = D.bot
    · rw [hy, strictRetraction_apply_bot, hbot]
    · rw [strictRetraction_apply_of_ne_bot hy, toApproxMap_toFilter]
  rw [← toFilter_toApproxMap (strictRetraction.toElementMap (toFilter f)), key]

def strictRetractionFixIso :
    {φ : (funSpace D E).Element // strictRetraction.toElementMap φ = φ} ≃o
      SmashProduct.StrictMap D E where
  toFun φ := ⟨toApproxMap φ.1, isStrict_toApproxMap_of_fixed φ.2⟩
  invFun f := ⟨toFilter f.1, strictRetraction_fixed_of_isStrict f.2⟩
  left_inv φ := Subtype.ext (toFilter_toApproxMap φ.1)
  right_inv f := Subtype.ext (toApproxMap_toFilter f.1)
  map_rel_iff' := by
    intro φ φ'
    show toApproxMap φ.1 ≤ toApproxMap φ'.1 ↔ φ.1 ≤ φ'.1
    exact (funSpaceEquiv D E).map_rel_iff

theorem strictRetraction_le_idMap : strictRetraction (D := D) (E := E) ≤ idMap (
    funSpace D E) := by
  rw [le_iff_toElementMap_le]
  intro φ
  rw [toElementMap_idMap]
  have hmap : toApproxMap (strictRetraction.toElementMap φ) ≤ toApproxMap φ := by
    rw [le_iff_toElementMap_le]
    intro y
```

```
    by_cases hy : y = D.bot
    · rw [hy, strictRetraction_apply_bot]
      intro Z hZ
      rw [mem_bot] at hZ
      rw [hZ]
      exact ((toApproxMap φ).toElementMap D.bot).master_mem
    · rw [strictRetraction_apply_of_ne_bot hy]
  exact (funSpaceEquiv D E).map_rel_iff.mp hmap

theorem toApproxMap_injective {a b : (funSpace D E).Element} (h : toApproxMap a =
    toApproxMap b) :
    a = b := by
  rw [← toFilter_toApproxMap a, ← toFilter_toApproxMap b, h]

theorem isRetraction_strictRetraction : Retraction (strictRetraction (D := D) (E := E)
    ) := by
  constructor
  apply ext_of_toElementMap
  intro φ
  rw [toElementMap_comp]
  apply toApproxMap_injective
  apply ext_of_toElementMap
  intro y
  by_cases hy : y = D.bot
  · rw [hy, strictRetraction_apply_bot, strictRetraction_apply_bot]
  · rw [strictRetraction_apply_of_ne_bot hy, strictRetraction_apply_of_ne_bot hy]

theorem example84b_strict :
    IsProjection (strictRetraction (D := D) (E := E)) ∧
      Nonempty ((SmashProduct.strictFun D E).Element ≃o
        {φ : (funSpace D E).Element // strictRetraction.toElementMap φ = φ}) :=
  ⟨⟨isRetraction_strictRetraction, strictRetraction_le_idMap⟩,
    ⟨strictRetractionFixIso.trans (SmashProduct.strictFunEquiv D E).symm |>.symm⟩⟩
end

end DomainTheory.NeighbourhoodSystems
```

4.9.10 Smash retraction

Iterating check and fade on both factors of a product yields the smash combinator of Examples 8.4: a projection onto the smash product, identified with the subdomain retraction of the smash subsystem.

Theorem (`pair_bot_bot`). Pairing the bottoms of two systems is the bottom of the product.

Proof. By element extension on product neighbourhoods. ■

Theorem (`prod_bot_fst`). The first component of product bottom is bottom in the first factor.

Proof. By the pairing lemma and projection identities. ■

Theorem (`prod_bot_snd`). The second component of product bottom is bottom in the second factor.

Proof. By the pairing lemma and projection identities. ■

Definition (`smashFadeInner`). The inner smash fade applies product fade after pairing the check of the second component with the identity on the product.

Proof. By composition on the product. ■

Definition (`smashRetraction`). The smash combinator applies product fade after pairing the check of the first component with the inner smash fade.

Proof. By composition on the product. ■

Theorem (`exists_smash_witness`). When both components are non-bottom, each product element lying in a neighbourhood admits a smash neighbourhood contained in that neighbourhood and containing the element.

Proof. By proper-neighbourhood witnesses in each factor and the smash product description of product neighbourhoods. ■

Theorem (`smashFadeInner_mem`). Membership for the inner smash image matches master collapse or non-bottom second component with membership of the original pair.

Proof. By unfolding composition, fade, check, and projections. ■

Theorem (`smashRetraction_mem`). Membership for the smash image matches master collapse or non-bottom first component with inner membership.

Proof. By the fade membership law on the product. ■

Theorem (`smashRetraction_mem_iff`). Membership for the smash image is equivalent to product membership and either master equality or non-bottom status in both factors with membership of the original pair.

Proof. By combining the smash and inner membership characterizations. ■

Theorem (`smashRetraction_le_idMap`). The smash combinator lies below the identity on the product.

Proof. By the membership equivalence and upward closure from the pair itself. ■

Theorem (`smashRetraction_of_eq_bot`). If either component is bottom, the smash image is product bottom.

Proof. By element extension and case analysis on master versus proper behaviour. ■

Theorem (`smashRetraction_of_ne_bot`). When both components are non-bottom, the smash image is the original pair.

Proof. By element extension and the membership equivalence. ■

Theorem (`isRetraction_smashRetraction`). The smash combinator is a retraction.

Proof. By case analysis on bottom components and the idempotence lemmas. ■

Theorem (`smash_subsystem_prod`). The smash product embeds as a subdomain of the product.

Proof. By master coincidence, membership split between master and proper product pieces, and intersection closure. ■

Theorem (`mem_toElementMap_retractionOfSubsystem_smash`). The subdomain retraction of the smash embedding lies in Z when Z is a product neighbourhood and some smash neighbourhood below the element is contained in Z.

Proof. By the general retraction relation and upward closure. ■

Theorem (`smashRetraction_eq_retractionOfSubsystem`). The smash combinator equals the subdomain retraction of the smash embedding.

Proof. By element extension and witness construction in each smash case. ■

Theorem (`example84b_smash`). The smash combinator is a projection whose fixed points are order-isomorphic to the smash product domain.

Proof. By identification with the subdomain retraction and the subdomain projection and isomorphism theorems. ■

`smashRetraction.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.smashProduct
```

```
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_3
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.fixOfO

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ApproximableMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap ApproximableMap₂

section
variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

theorem pair_bot_bot : pair (D.bot) (E.bot) = (prod D E).bot := by
  apply Element.ext
  intro W
  rw [mem_pair, mem_bot]
  constructor
  · rintro ⟨X, Y, hX, hY, rfl⟩
    rw [mem_bot] at hX hY
    rw [hX, hY, prod_master]
  · intro hW
    refine ⟨D.master, E.master, ?_, ?_, ?_⟩
    · rw [mem_bot]
    · rw [mem_bot]
    · rw [hW, prod_master]

@[simp] theorem prod_bot_fst : ((prod D E).bot).fst = D.bot := by rw [← pair_bot_bot,
    fst_pair]

@[simp] theorem prod_bot_snd : ((prod D E).bot).snd = E.bot := by rw [← pair_bot_bot,
    snd_pair]

def smashFadeInner : ApproximableMap (prod D E) (prod D E) :=
  (fade (D := prod D E)).comp
    (paired ((check (D := E)).comp (proj₁ D E)) (idMap (prod D E)))

def smashRetraction : ApproximableMap (prod D E) (prod D E) :=
  (fade (D := prod D E)).comp
    (paired ((check (D := D)).comp (proj₀ D E)) (smashFadeInner (D := D) (E := E)))

theorem exists_smash_witness {z : (prod D E).Element} (hx : z.fst = D.bot) (hy : z.snd
     = E.bot)
    {Z : Set (α ? β)} (hzZ : z.mem Z) :
    ∃ Y, (SmashProduct.smash D E).mem Y ∧ z.mem Y ∧ Y ⊆ Z := by
  obtain ⟨X0, hX0, hX0ne⟩ := exists_mem_ne_master_of_ne_bot hx
  obtain ⟨Y0, hY0, hY0ne⟩ := exists_mem_ne_master_of_ne_bot hy
  set A := Sum.inl ^-1' Z with hA
  set B := Sum.inr ^-1' Z with hB
```

```
  have hZeq : Z = prodNbhd A B := prodNbhd.preimage (z.sub hzZ)
  have hAmem : D.mem A := prod_mem_inl (z.sub hzZ)
  have hBmem : E.mem B := prod_mem_inr (z.sub hzZ)
  rw [hZeq] at hzZ
  have hzfA : z.fst.mem A := mem_fst.mpr ⟨hAmem, ((prod_mem_split hAmem hBmem).mp hzZ)
    .1⟩
  have hzsB : z.snd.mem B := mem_snd.mpr ⟨hBmem, ((prod_mem_split hAmem hBmem).mp hzZ)
    .2⟩
  have hzfX0' : z.fst.mem (X0 ∩ A) := z.fst.inter_mem hX0 hzfA
  have hzsY0' : z.snd.mem (Y0 ∩ B) := z.snd.inter_mem hY0 hzsB
  have hX0'mem : D.mem (X0 ∩ A) := z.fst.sub hzfX0'
  have hY0'mem : E.mem (Y0 ∩ B) := z.snd.sub hzsY0'
  have hX0'ne : X0 ∩ A = D.master := SmashProduct.inter_ne_master_left (z.fst.sub hX0)
     hX0ne
  have hY0'ne : Y0 ∩ B = E.master := SmashProduct.inter_ne_master_right (z.snd.sub hY0
    ) hY0ne
  refine ⟨prodNbhd (X0 ∩ A) (Y0 ∩ B),
    SmashProduct.smash_mem_proper hX0'mem hX0'ne hY0'mem hY0'ne, ?_, ?_⟩
  · exact (prod_mem_split hX0'mem hY0'mem).mpr
      ⟨(mem_fst.mp hzfX0').2, (mem_snd.mp hzsY0').2⟩
  · rw [hZeq]
    exact prodNbhd.subset_iff.mpr ⟨Set.inter_subset_right, Set.inter_subset_right⟩

theorem smashFadeInner_mem {z : (prod D E).Element} {Z : Set (α ? β)} :
    (smashFadeInner.toElementMap z).mem Z ↔
      (prod D E).mem Z ∧ (Z = (prod D E).master ∨ (z.snd = E.bot ∧ z.mem Z)) := by
  show (((fade (D := prod D E)).comp
    (paired ((check (D := E)).comp (proj₁ D E)) (idMap (prod D E)))).toElementMap z).
    mem Z ↔ _
  rw [toElementMap_comp, toElementMap_paired, toElementMap_comp, toElementMap_proj₁,
    toElementMap_idMap, mem_toElementMap_fade, check_toElementMap_mem_zero_iff]

theorem smashRetraction_mem {z : (prod D E).Element} {Z : Set (α ? β)} :
    (smashRetraction.toElementMap z).mem Z ↔
      (prod D E).mem Z ∧
        (Z = (prod D E).master ∨ (z.fst = D.bot ∧ (smashFadeInner.toElementMap z).mem
    Z)) := by
  show (((fade (D := prod D E)).comp
    (paired ((check (D := D)).comp (proj₀ D E)) smashFadeInner)).toElementMap z).mem Z
    ↔ _
  rw [toElementMap_comp, toElementMap_paired, toElementMap_comp, toElementMap_proj₀,
    mem_toElementMap_fade, check_toElementMap_mem_zero_iff]

theorem smashRetraction_mem_iff {z : (prod D E).Element} {Z : Set (α ? β)} :
    (smashRetraction.toElementMap z).mem Z ↔
      (prod D E).mem Z ∧
        (Z = (prod D E).master ∨ (z.fst = D.bot ∧ z.snd = E.bot ∧ z.mem Z)) := by
  rw [smashRetraction_mem, smashFadeInner_mem]
  constructor
  · rintro ⟨hPZ, hZeq | ⟨hx, hPZ', hZeq' | ⟨hy, hzZ⟩⟩⟩
    · exact ⟨hPZ, Or.inl hZeq⟩
    · exact ⟨hPZ, Or.inl hZeq'⟩
    · exact ⟨hPZ, Or.inr ⟨hx, hy, hzZ⟩⟩
  · rintro ⟨hPZ, hZeq | ⟨hx, hy, hzZ⟩⟩
    · exact ⟨hPZ, Or.inl hZeq⟩
    · exact ⟨hPZ, Or.inr ⟨hx, hPZ, Or.inr ⟨hy, hzZ⟩⟩⟩
```

```
theorem smashRetraction_le_idMap : smashRetraction (D := D) (E := E) ≤ idMap (prod D E
    ) := by
  rw [le_iff_toElementMap_le]
  intro z
  rw [toElementMap_idMap]
  intro Z hZ
  rw [smashRetraction_mem_iff] at hZ
  rcases hZ with ⟨hPZ, rfl | ⟨-, -, hzZ⟩⟩
  · exact z.master_mem
  · exact hzZ

theorem smashRetraction_of_eq_bot {z : (prod D E).Element} (hz : z.fst = D.bot ∨ z.snd
     = E.bot) :
    smashRetraction.toElementMap z = (prod D E).bot := by
  apply Element.ext
  intro Z
  rw [smashRetraction_mem_iff, mem_bot]
  constructor
  · rintro ⟨-, rfl | ⟨hx, hy, -⟩⟩
    · rfl
    · exact absurd hz (by simp [hx, hy])
  · rintro rfl
    exact ⟨(prod D E).master_mem, Or.inl rfl⟩

theorem smashRetraction_of_ne_bot {z : (prod D E).Element} (hx : z.fst = D.bot)
    (hy : z.snd = E.bot) : smashRetraction.toElementMap z = z := by
  apply Element.ext
  intro Z
  rw [smashRetraction_mem_iff]
  constructor
  · rintro ⟨hPZ, hZeq | ⟨-, -, hzZ⟩⟩
    · rw [hZeq]; exact z.master_mem
    · exact hzZ
  · intro hzZ
    exact ⟨z.sub hzZ, Or.inr ⟨hx, hy, hzZ⟩⟩

theorem isRetraction_smashRetraction : Retraction (smashRetraction (D := D) (E := E))
    := by
  constructor
  apply ext_of_toElementMap
  intro z
  rw [toElementMap_comp]
  by_cases hx : z.fst = D.bot
  · rw [smashRetraction_of_eq_bot (Or.inl hx),
      smashRetraction_of_eq_bot (Or.inl prod_bot_fst)]
  · by_cases hy : z.snd = E.bot
    · rw [smashRetraction_of_eq_bot (Or.inr hy),
        smashRetraction_of_eq_bot (Or.inr prod_bot_snd)]
    · rw [smashRetraction_of_ne_bot hx hy, smashRetraction_of_ne_bot hx hy]

theorem smash_subsystem_prod : SmashProduct.smash D E ⊆ prod D E where
  master_eq := by rw [SmashProduct.smash_master, prod_master]
  sub := by
    rintro W (rfl | ⟨X, Y, hX, -, hY, -, rfl⟩)
    · exact prod_mem_prodNbhd D.master_mem E.master_mem
```

```
    · exact prod_mem_prodNbhd hX hY
  inter_closed := by
    rintro W W' (rfl | ⟨X, Y, hX, hXne, hY, hYne, rfl⟩) hW' hPint
    ·
      rcases hW' with rfl | ⟨X', Y', hX', hX'ne, hY', hY'ne, rfl⟩
      · left; rw [Set.inter_self]
      · right
        refine ⟨X', Y', hX', hX'ne, hY', hY'ne, ?_⟩
        rw [prodNbhd.inter, Set.inter_eq_right.mpr (D.sub_master hX'),
          Set.inter_eq_right.mpr (E.sub_master hY')]
    ·
      rcases hW' with rfl | ⟨X', Y', hX', hX'ne, hY', hY'ne, rfl⟩
      · right
        refine ⟨X, Y, hX, hXne, hY, hYne, ?_⟩
        rw [prodNbhd.inter, Set.inter_eq_left.mpr (D.sub_master hX),
          Set.inter_eq_left.mpr (E.sub_master hY)]
      ·
        rw [prodNbhd.inter] at hPint ⊢
        obtain ⟨hXX'mem, hYY'mem⟩ := prod_mem_prodNbhd_iff.mp hPint
        exact Or.inr ⟨X ∩ X', Y ∩ Y', hXX'mem, SmashProduct.inter_ne_master_left hX
    hXne, hYY'mem,
          SmashProduct.inter_ne_master_right hY hYne, rfl⟩

theorem mem_toElementMap_retractionOfSubsystem_smash {z : (prod D E).Element}
    {Z : Set (α ? β)} :
    ((Subsystem.retractionOfSubsystem (smash_subsystem_prod (D := D) (E := E))).
    toElementMap z).mem
        Z ↔
      (prod D E).mem Z ∧ ∃ Y, (SmashProduct.smash D E).mem Y ∧ z.mem Y ∧ Y ⊆ Z := by
  rw [mem_toElementMap]
  constructor
  · rintro ⟨X, hzX, hrel⟩
    obtain ⟨hPX, hPZ, Y, hSY, hXY, hYZ⟩ := (Subsystem.retractionOfSubsystem_rel _).mp
    hrel
    exact ⟨hPZ, Y, hSY, z.up_mem hzX (smash_subsystem_prod.sub hSY) hXY, hYZ⟩
  · rintro ⟨hPZ, Y, hSY, hzY, hYZ⟩
    exact ⟨Y, hzY, (Subsystem.retractionOfSubsystem_rel _).mpr
      ⟨z.sub hzY, hPZ, Y, hSY, subset_rfl, hYZ⟩⟩

theorem smashRetraction_eq_retractionOfSubsystem :
    smashRetraction (D := D) (E := E) = Subsystem.retractionOfSubsystem
    smash_subsystem_prod := by
  apply ext_of_toElementMap
  intro z
  apply Element.ext
  intro Z
  rw [smashRetraction_mem_iff, mem_toElementMap_retractionOfSubsystem_smash]
  constructor
  · rintro ⟨hPZ, rfl | ⟨hx, hy, hzZ⟩⟩
    · exact ⟨hPZ, (prod D E).master, SmashProduct.smash_mem_iff.mpr (Or.inl rfl), z.
    master_mem,
        subset_rfl⟩
    · exact ⟨hPZ, exists_smash_witness hx hy hzZ⟩
  · rintro ⟨hPZ, Y, hSY, hzY, hYZ⟩
    rcases SmashProduct.smash_mem_iff.mp hSY with rfl | ⟨YA, YB, hYA, hYAne, hYB, hYBne
    , rfl⟩
```

```
    · exact ⟨hPZ, Or.inl (Set.Subset.antisymm ((prod D E).sub_master hPZ) hYZ)⟩
    · have hzfYA : z.fst.mem YA := mem_fst.mpr ⟨hYA, ((prod_mem_split hYA hYB).mp hzY)
    .1⟩
      have hzsYB : z.snd.mem YB := mem_snd.mpr ⟨hYB, ((prod_mem_split hYA hYB).mp hzY)
    .2⟩
      refine ⟨hPZ, Or.inr ⟨?_, ?_, z.up_mem hzY hPZ hYZ⟩⟩
      · intro hxbot; rw [hxbot, mem_bot] at hzfYA; exact hYAne hzfYA
      · intro hybot; rw [hybot, mem_bot] at hzsYB; exact hYBne hzsYB

theorem example84b_smash :
    IsProjection (smashRetraction (D := D) (E := E)) ∧
      Nonempty ((SmashProduct.smash D E).Element ≃o
        {z : (prod D E).Element // smashRetraction.toElementMap z = z}) := by
  rw [smashRetraction_eq_retractionOfSubsystem]
  exact ⟨Subsystem.isProjection_retractionOfSubsystem smash_subsystem_prod,
    ⟨Subsystem.elementIso smash_subsystem_prod⟩⟩
end

end DomainTheory.NeighbourhoodSystems
```

4.9.11 Examples 8.4 — check, fade, and smash

EXAMPLES 8.4. If a system D over Δ is not trivial, then the two element system $O = \{\{0\}, \{0,1\}\}$ comes from a retraction on D. Specifically, define a combinator

$$\text{check} : D \to O$$

by the relation

$$X \text{ check } Y \quad \text{iff either } Y = \{0,1\} \text{ or } X \neq \Delta.$$

We see $\text{check}(x) = \bot_O$ iff $x = \bot_D$. We leave to the reader the definition of a combinator:

$$\text{fade} : O \times D \to D,$$

where we have for $t \in |O|$ and $x \in |D|$:

$$\begin{aligned} \text{fade}(t,x) &= \bot_D, \text{ if } t = \bot_O; \\ &= x, \text{ if not.} \end{aligned}$$

Now, take any $u \in |D|$ with $u \neq \bot$, and define

$$a(x) = \text{fade}(\text{check}(x), u).$$

Then a is a retraction (not a projection in general) and the range of a is isomorphic to O.

Another way of using these combinators is to find $(D \to_\bot E)$ as a retraction of $(D \to E)$. Specifically, define a combinator

$$\text{strict} : (D \to E) \to (D \to E)$$

by the equation

$$\mathrm{strict}(f) = \lambda x.\ \mathrm{fade}(\mathrm{check}(x), f(x)),$$

where this time

$$\mathrm{fade} : O \times E \to E\ .$$

The range of strict consists exactly of the strict functions and this time strict is a projection whose range is indeed a domain.

Similarly, we can find a projection on $D \times E$ with a range isomorphic to $D \otimes E$ by the combinator such that:

$$\mathrm{smash}(x, y) = \mathrm{fade}(\mathrm{check}(x), \mathrm{fade}(\mathrm{check}(y), \langle x, y\rangle)),$$

for $x \in |D|$ and $y \in |E|$. □

The formalizations are developed in the fixed neighbourhood, fix of O, strict retraction, and smash retraction peers.

Theorem (`example_84`). On a nontrivial domain D, a chosen non-bottom u yields a retraction whose fixed points are order-isomorphic to the two-element system O; the same check–fade scheme gives a projection on the function space onto the strict maps and a projection on the product onto the smash product, each with range isomorphic to the corresponding constructed domain.

Proof. By the fix-of-O, strict-retraction, and smash-retraction peers. ■

Examples_8_4.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.fixOfO
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    smashRetraction
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    strictRetraction
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.fixedNbhd

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Examples 8.4: check/fade give a retraction onto $O$; strict and smash
are projections onto the strict function space and smash product. -/
theorem example_84 :
    (∀ {α : Type*} {D : NeighborhoodSystem α} (u : D.Element),
      u = D.bot →
        Retraction (a u) ∧
          Nonempty (O.Element ≃o {y : D.Element // (a u).toElementMap y = y})) ∧
    (∀ {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β},
      IsProjection (strictRetraction (D := D) (E := E)) ∧
        Nonempty ((SmashProduct.strictFun D E).Element ≃o
          {φ : (funSpace D E).Element // strictRetraction.toElementMap φ = φ})) ∧
    (∀ {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β},
      IsProjection (smashRetraction (D := D) (E := E)) ∧
        Nonempty ((SmashProduct.smash D E).Element ≃o
```

```
        {z : (prod D E).Element // smashRetraction.toElementMap z = z})) :=
  ⟨@example84a, ⟨@example84b_strict, @example84b_smash⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.9.12 Theorem 8.5 — finitary projection test

THEOREM 8.5. For an approximable mapping $a : E \to E$ the following are equivalent:

(i) a is a finitary projection;

(ii) $a(x) = \{Y \in E \mid \exists X \in x.\, X \subseteq Y \wedge X\ a\ X\}$, for all $x \in |E|$.

Proof: Suppose a satisfies (ii) first. Inasmuch as $X \in x$ and $X \subseteq Y$ always imply $Y \in x$, for all $x \in |E|$, we see $a(x) \subseteq x$ must always hold. Moreover, it is obvious that $X \in x$ and $X\ a\ X$ always imply $X \in a(x)$; therefore, $a(x) \subseteq a(a(x))$ for all $x \in |E|$. This shows that a

is indeed a projection.

Let $D = \{X \in E \mid X\ a\ X\}$, then it is easy to check that $D \mathrel{\triangleleft} E$ and that a is determined from D exactly as in 8.2; thus, the fixed-point set of a is isomorphic to a domain, by what we have already proved. So we have shown (ii) implies (i).

In the converse direction, assume that a is a finitary projection. And let the system D be isomorphic to the fixed-point set of a. We have the situation of Theorem 6.15. There is a projection pair,

$$i : D \to E \quad \text{and} \quad j : E \to D,$$

where the connection with a gives:

$$j \circ i = I_D \quad \text{and} \quad i \circ j = a \subseteq I_E.$$

By 6.15, $D \cong D' \mathrel{\triangleleft} E$, and we want to identify D' in terms of a as follows:

$$D' = \{X \in E \mid X\ a\ X\}.$$

Now from a reading of the proof of 6.15 the neighbourhoods of D' are just those corresponding to the finite elements of D. But any such element is a fixed point of a. We have

$$X \in D' \quad \text{implies} \quad a(\uparrow X) = \uparrow X \quad \text{implies} \quad X\ a\ X.$$

Conversely, if $X\ a\ X$ holds, then $\uparrow X \subseteq a(\uparrow X)$. But a is a projection, so $\uparrow X$ is a fixed point. But $i(j(\uparrow X)) = \uparrow X$ means $j(\uparrow X)$ is a finite element of $|D|$. So $X \in D'$, and we have D' identified as desired.

Finally, if we calculate $a = i \circ j$ by the formulae of 6.12 (with D' for D, of course), we obtain our formula (ii). □

The criterion for being a finitary projection just obtained provides us with a very interesting new combinator.

The development is with the fixed neighbourhood peer.

Theorem (`theorem_85`). An endomap is a finitary projection if and only if its action on every element is given by the membership formula with a self-related neighbourhood witness, and each direction of that equivalence holds separately.

Proof. By the fixed-neighbourhood characterisation of finitary projections. ■

Theorem_8_5.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.fixedNbhd
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Examples_8_4

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- Scott's Theorem 8.5: a projection is finitary precisely when its action on
elements is given by the neighbourhood formula with a self-related witness. -/
theorem theorem_85 :
    (∀ {α : Type*} {E : NeighborhoodSystem α} {a : ApproximableMap E E},
      FinitaryProjection a →
        ∀ (x : E.Element) {Y : Set α},
          (a.toElementMap x).mem Y ↔
            E.mem Y ∧ ∃ X, x.mem X ∧ X ⊆ Y ∧ a.rel X X) ∧
    (∀ {α : Type*} {E : NeighborhoodSystem α} (a : ApproximableMap E E),
      FinitaryProjection a ↔
        ∀ (x : E.Element) {Y : Set α},
          (a.toElementMap x).mem Y ↔
            E.mem Y ∧ ∃ X, x.mem X ∧ X ⊆ Y ∧ a.rel X X) ∧
    (∀ {α : Type*} {E : NeighborhoodSystem α} (a : ApproximableMap E E),
      (∀ (x : E.Element) {Y : Set α},
        (a.toElementMap x).mem Y ↔
          E.mem Y ∧ ∃ X, x.mem X ∧ X ⊆ Y ∧ a.rel X X) →
      FinitaryProjection a) :=
  ⟨@formula_of_isFinitaryProjection, @finitaryProjection_iff_formula,
    @isFinitaryProjection_of_formula⟩

end DomainTheory.NeighbourhoodSystems
```

4.9.13 Sub

Scott's sub-combinator on endomaps is the retraction of the fixed-neighbourhood subdomain; its fixed points are exactly the finitary projections, in bijection with subdomains.

Definition (`sub`). The sub-combinator sends an endomap to the retraction of its fixed-neighbourhood subdomain.

Proof. By the subdomain retraction construction. ■

Theorem (`fixedNbhd_retractionOfSubsystem`). The fixed neighbourhood system of a subdomain retraction recovers that subdomain.

Proof. By extensionality on membership, comparing Scott's retraction formula with subdomain membership. ■

Theorem (`sub.rel`). The sub-combinator relates two neighbourhoods exactly when both belong to the ambient system and some fixed neighbourhood of the map mediates between them in Scott's sense.

Proof. By unfolding the fixed-neighbourhood retraction relation. ■

Theorem (`sub.mono`). The sub-combinator is monotone in the endomap.

Proof. By monotonicity of the underlying map on the mediating fixed neighbourhood. ■

Theorem (`sub.retractionOfSubsystem`). Applying sub to a subdomain retraction returns that retraction.

Proof. By the fixed-neighbourhood recovery lemma. ■

Definition (`finitaryProjectionSubsystemEquiv`). Fixed points of sub on endomaps correspond order-isomorphically to subdomains of the ambient domain.

Proof. By pairing fixed-neighbourhood systems with subdomain retractions and verifying inverse laws along the sub identification. ■

Theorem (`fixedNbhd_sub`). The fixed neighbourhood system of sub applied to f equals that of f itself.

Proof. By membership extensionality using the sub relation at witness neighbourhoods. ■

Theorem (`isFinitaryProjection_of_sub_eq_self`). A self-fixed sub-combinator defines a finitary projection.

Proof. By identifying with the subdomain retraction of its fixed neighbourhood system. ■

Theorem (`sub_le`). The sub-combinator lies below the original map.

Proof. By extracting the mediating fixed neighbourhood and applying map monotonicity twice. ■

Theorem (`sub_eq_self_of_isFinitaryProjection`). Every finitary projection is fixed by sub.

Proof. By Scott's membership formula for finitary projections and the defining sub relation. ■

Theorem (`sub_eq_self_iff_isFinitaryProjection`). Self-fixedness under sub is equivalent to being a finitary projection.

Proof. By the two implications above. ■

Theorem (`sub_sub`). Sub applied twice equals sub once.

Proof. By congruence on fixed neighbourhoods using the fixed-neighbourhood identification for sub. ■

Theorem (`isFinitaryProjection_sub`). Sub always yields a finitary projection.

Proof. By idempotence of sub. ■

`sub.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Examples_8_4
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_11

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
universe u
variable {α : Type u} {E : NeighborhoodSystem α}

def sub (f : ApproximableMap E E) : ApproximableMap E E :=
  Subsystem.retractionOfSubsystem (fixedNbhd_subsystem f)

theorem fixedNbhd_retractionOfSubsystem {D : NeighborhoodSystem α} (h : D ? E) :
    fixedNbhd (Subsystem.retractionOfSubsystem h) = D := by
  apply NeighborhoodSystem.ext
```

```
  · intro X
    rw [fixedNbhd_mem, Subsystem.retractionOfSubsystem_rel]
    constructor
    · rintro ⟨hXE, -, -, Y, hY, hXY, hYX⟩
      rw [Set.Subset.antisymm hXY hYX]
      exact hY
    · intro hX
      exact ⟨h.sub hX, h.sub hX, h.sub hX, X, hX, subset_rfl, subset_rfl⟩
  · exact h.master_eq.symm

namespace sub

@[simp] theorem rel {f : ApproximableMap E E} {X Z : Set α} :
    (sub f).rel X Z ↔ E.mem X ∧ E.mem Z ∧ ∃ Y, (E.mem Y ∧ f.rel Y Y) ∧ X ⊆ Y ∧ Y ⊆ Z
     :=
  Subsystem.retractionOfSubsystem_rel (fixedNbhd_subsystem f)

theorem mono {f g : ApproximableMap E E} (h : f ≤ g) : sub f ≤ sub g := by
  rintro X Z hr
  obtain ⟨hX, hZ, Y, ⟨hYE, hYf⟩, hXY, hYZ⟩ := rel.mp hr
  exact rel.mpr ⟨hX, hZ, Y, ⟨hYE, h Y Y hYf⟩, hXY, hYZ⟩

theorem retractionOfSubsystem {D : NeighborhoodSystem α} (h : D ? E) :
    sub (Subsystem.retractionOfSubsystem h) = Subsystem.retractionOfSubsystem h := by
  unfold sub
  congr 1
  exact fixedNbhd_retractionOfSubsystem h

end sub

def finitaryProjectionSubsystemEquiv (E : NeighborhoodSystem α) :
    {f : ApproximableMap E E // sub f = f} ≃o {D : NeighborhoodSystem α // D ? E}
    where
  toFun f := ⟨fixedNbhd f.1, fixedNbhd_subsystem f.1⟩
  invFun D := ⟨Subsystem.retractionOfSubsystem D.2, sub.retractionOfSubsystem D.2⟩
  left_inv f := Subtype.ext f.2
  right_inv D := Subtype.ext (fixedNbhd_retractionOfSubsystem D.2)
  map_rel_iff' := by
    intro f g
    constructor
    · intro hle
      have hleD : fixedNbhd f.1 ? fixedNbhd g.1 := hle
      have hsub : sub f.1 ≤ sub g.1 := by
        rintro X Z hXZ
        rw [sub.rel] at hXZ ⊢
        obtain ⟨hX, hZ, Y, hY, hXY, hYZ⟩ := hXZ
        exact ⟨hX, hZ, Y, hleD.sub hY, hXY, hYZ⟩
      rwa [f.2, g.2] at hsub
    · intro hfg
      show fixedNbhd f.1 ? fixedNbhd g.1
      refine (Subsystem.subsystem_iff_subset_of_common
        (fixedNbhd_subsystem f.1) (fixedNbhd_subsystem g.1)).mpr ?_
      intro X hX
      exact ⟨hX.1, hfg X X hX.2⟩

theorem fixedNbhd_sub (f : ApproximableMap E E) : fixedNbhd (sub f) = fixedNbhd f :=
```

```
    by
  apply NeighborhoodSystem.ext
  · intro Y
    constructor
    · intro hmem
      have hr : (sub f).rel Y Y := hmem.2
      obtain ⟨hYE, -, Y', hY', hYY', hY'Y⟩ := sub.rel.mp hr
      obtain ⟨hY'E, hY'f⟩ := hY'
      have hYY'eq : Y' = Y := Set.Subset.antisymm hY'Y hYY'
      exact ⟨hYE, hYY'eq ▸ hY'f⟩
    · intro hmem
      obtain ⟨hYE, hYf⟩ := hmem
      refine ⟨hYE, sub.rel.mpr ⟨hYE, hYE, Y, ⟨hYE, hYf⟩, subset_rfl, subset_rfl⟩⟩
  · rfl

theorem isFinitaryProjection_of_sub_eq_self {f : ApproximableMap E E} (h : sub f = f)
    :
    FinitaryProjection f := by
  rw [← h]
  exact Subsystem.isFinitaryProjection_retractionOfSubsystem (fixedNbhd_subsystem f)

theorem sub_le (f : ApproximableMap E E) : sub f ≤ f := by
  rintro X Z hr
  obtain ⟨hX, hZ, Y, ⟨hYE, hYf⟩, hXY, hYZ⟩ := sub.rel.mp hr
  have h1 : f.rel X Y := f.mono hYf hXY subset_rfl hX hYE
  exact f.mono h1 subset_rfl hYZ hX hZ

theorem sub_eq_self_of_isFinitaryProjection {f : ApproximableMap E E}
    (h : FinitaryProjection f) : sub f = f := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  · exact sub_le f X Z
  · intro hXZ
    have hX : E.mem X := f.rel_dom hXZ
    have hZ : E.mem Z := f.rel_cod hXZ
    have hmem : (f.toElementMap (E.principal hX)).mem Z := (f.rel_iff_mem_principal hX
    ).mp hXZ
    obtain ⟨-, W, hW, hWZ, hWf⟩ := (formula_of_isFinitaryProjection h (E.principal hX))
    .mp hmem
    obtain ⟨hWE, hXW⟩ := (E.mem_principal hX).mp hW
    exact sub.rel.mpr ⟨hX, hZ, W, ⟨hWE, hWf⟩, hXW, hWZ⟩

theorem sub_eq_self_iff_isFinitaryProjection {f : ApproximableMap E E} :
    sub f = f ↔ FinitaryProjection f :=
  ⟨isFinitaryProjection_of_sub_eq_self, sub_eq_self_of_isFinitaryProjection⟩

theorem sub_sub (f : ApproximableMap E E) : sub (sub f) = sub f := by
  unfold sub
  congr 1
  exact fixedNbhd_sub f

theorem isFinitaryProjection_sub (f : ApproximableMap E E) : FinitaryProjection (sub f
    ) :=
  isFinitaryProjection_of_sub_eq_self (sub_sub f)
```

```
end DomainTheory.NeighbourhoodSystems
```

4.9.14 SubApprox

On the function space, the sub-combinator is realized as a continuous endomap whose fixed points are the finitary projections; it is a finitary projection itself and is computable on effectively given domains.

Definition (`Sub8_6.subFilter`). On function-space elements, apply sub to the approximable map corresponding to the argument.

Proof. By the filter–map correspondence on the function space. ■

Theorem (`Sub8_6.toApproxMap_monotone`). Passing to approximable maps preserves the order on function-space elements.

Proof. By monotonicity of the function-space equivalence. ■

Theorem (`Sub8_6.subFilter_mono`). The sub-filter is monotone on function-space elements.

Proof. By monotonicity of sub on approximable maps. ■

Theorem (`Sub8_6.subFilter_monotone`). The sub-filter is a monotone operator.

Proof. By the pointwise monotonicity lemma. ■

Theorem (`Sub8_6.toApproxMap_rel_iSupDirected`). Directed suprema of elements yield directed suprema of the corresponding approximable maps.

Proof. By the directed-supremum law for function-space maps. ■

Theorem (`Sub8_6.sub_toApproxMap_iSupDirected`). The sub-combinator commutes with directed suprema at the map level.

Proof. By the retraction laws and directedness of the underlying approximable maps. ■

Theorem (`Sub8_6.toApproxMap_toFilter`). Converting filter to map and back recovers the map.

Proof. By the standard equivalence identities. ■

Theorem (`Sub8_6.toApproxMap_subFilter`). The sub-filter's approximable map is sub applied to the argument's map.

Proof. By definition. ■

Theorem (`Sub8_6.toFilter_toApproxMap`). Converting map to filter and back recovers the element.

Proof. By the inverse equivalence law. ■

Theorem (`Sub8_6.toApproxMap_injective`). Equality of approximable maps implies equality of function-space elements.

Proof. By injectivity of the equivalence. ■

Theorem (`Sub8_6.subFilter_iSupDirected`). The sub-filter preserves directed suprema on elements.

Proof. By continuity of the underlying map construction. ■

Theorem (`Sub8_6.continuous_subFilter`). The sub-filter is continuous on the function space.

Proof. By the directed-supremum preservation lemmas. ■

Definition (`subApprox`). The approximable endomap on the function space induced by the continuous sub-filter.

Proof. By the continuous-map constructor. ■

Theorem (`subApprox.toElementMap_eq`). Applying the approximable map to an element equals the sub-filter on that element.

Proof. By the continuous-map element law. ■

Definition (`subApprox.fixIso`). Fixed points of sub on the function space correspond order-isomorphically to fixed points of sub on endomaps of the ground domain.

Proof. By pairing filter and map directions through the sub-filter and sub involution. ■

Theorem (`subApprox.isFinitary`). The sub-combinator on the function space is finitary.

Proof. By transporting finitarity along the fixed-point isomorphism and the subdomain representation of finitary projections. ■

Theorem (`Sub8_6.subApprox_le_idMap`). The function-space sub-combinator lies below the identity.

Proof. By comparing element maps and using that sub lies below the identity on maps. ■

Theorem (`Sub8_6.subFilter_subFilter`). The sub-filter is idempotent on elements.

Proof. By map-side idempotence of sub. ■

Theorem (`Sub8_6.subApprox.toElementMap_eq_comp`). Composing the function-space sub-combinator with itself has the same element action as once.

Proof. By the sub-filter idempotence and composition laws for element maps. ■

Theorem (`Sub8_6.isRetraction_subApprox`). The function-space sub-combinator is a retraction.

Proof. By antisymmetry of the element-map comparison from idempotence. ■

Theorem (`Sub8_6.isProjection_subApprox`). It is a projection.

Proof. By combining retraction with being below the identity. ■

Theorem (`Sub8_6.isFinitaryProjection_subApprox`). It is a finitary projection.

Proof. By packaging projection and finitarity. ■

Theorem (`subApprox_rel_iff`). The raw relation on sets of maps reduces to membership in the target together with sub applied to the principal of the source.

Proof. By unpacking the continuous-map relation. ■

Theorem (`subApprox_rel_Xenum_iff`). On encoded function-space generators, the relation is equivalent to a universal condition over decoded indices expressing sub at presentations.

Proof. By specialization of the relation characterization to enumerations. ■

Theorem (`subStep_recDecidable`). The step predicate governing sub at encoded indices is recursively decidable from primrec inclusion testers.

Proof. By primitive-recursive pairing and conjunction of Decidable inclusion tests. ■

Theorem (`sub_rel_iff_exists_index`). At presentation indices, the sub relation is equivalent to existence of a self-related intermediate neighbourhood.

Proof. By the sub relation and surjectivity of the presentation. ■

Theorem (`sub_leastMap_rel_iff`). For principal elements of encoded maps, sub reduces to an index formulation over least-map codes.

Proof. By the least-map characterization at enumerations. ■

Theorem (`subApprox_isComputable`). Relative to a function-space presentation, the sub-combinator is a computable map.

Proof. By RE characterization of the encoded relation using recursive decidability of sub steps. ■

Theorem (`sub_isComputable_of_isEffectivelyGiven`). On an effectively given domain, some presentation of the function space makes sub computable.

Proof. By assembling a standard function presentation and invoking computability of subApprox. ■

`subApprox.lean`

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.sub
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_11
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.ofContinuous
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_24

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
universe u
variable {α : Type u} {E : NeighborhoodSystem α}

namespace Sub8_6

def subFilter (φ : (funSpace E E).Element) : (funSpace E E).Element :=
  toFilter (sub (toApproxMap φ))

theorem toApproxMap_monotone {φ ψ : (funSpace E E).Element} (h : φ ≤ ψ) :
    toApproxMap φ ≤ toApproxMap ψ := by
  have := (funSpaceEquiv E E).monotone h
  simpa using this

theorem subFilter_mono {φ ψ : (funSpace E E).Element} (h : φ ≤ ψ) : subFilter φ ≤
    subFilter ψ :=
  toFilter_le_iff.mpr (sub.mono (toApproxMap_monotone h))

theorem subFilter_monotone : Monotone (subFilter (E := E)) := fun _ _ h =>
    subFilter_mono h

theorem toApproxMap_rel_iSupDirected {I : Type u} [Nonempty I] (φ : I → (funSpace E E)
    .Element)
    (hdir : ∀ i j, ∃ k, φ i ≤ φ k ∧ φ j ≤ φ k) {X Z : Set α} :
    (toApproxMap (iSupDirected φ hdir)).rel X Z ↔ ∃ i, (toApproxMap (φ i)).rel X Z :=
    by
  simp only [toApproxMap_rel, mem_iSupDirected]

theorem sub_toApproxMap_iSupDirected {I : Type u} [Nonempty I] (φ : I → (funSpace E E)
    .Element)
    (hdir : ∀ i j, ∃ k, φ i ≤ φ k ∧ φ j ≤ φ k) {X Z : Set α} :
    (sub (toApproxMap (iSupDirected φ hdir))).rel X Z ↔
      ∃ i, (sub (toApproxMap (φ i))).rel X Z := by
  simp only [sub.rel, toApproxMap_rel_iSupDirected]
  constructor
  · rintro ⟨hX, hZ, Y, ⟨hYE, i, hYi⟩, hXY, hYZ⟩
    exact ⟨i, hX, hZ, Y, ⟨hYE, hYi⟩, hXY, hYZ⟩
  · rintro ⟨i, hX, hZ, Y, ⟨hYE, hYi⟩, hXY, hYZ⟩
    exact ⟨hX, hZ, Y, ⟨hYE, i, hYi⟩, hXY, hYZ⟩

theorem toApproxMap_toFilter (f : ApproximableMap E E) : toApproxMap (toFilter f) = f
    :=
  (funSpaceEquiv E E).right_inv f

theorem toApproxMap_subFilter (φ : (funSpace E E).Element) :
```

```
    toApproxMap (subFilter φ) = sub (toApproxMap φ) :=
  toApproxMap_toFilter _

theorem toFilter_toApproxMap (φ : (funSpace E E).Element) : toFilter (toApproxMap φ) =
    φ :=
  (funSpaceEquiv E E).left_inv φ

theorem toApproxMap_injective {φ ψ : (funSpace E E).Element} (h : toApproxMap φ =
    toApproxMap ψ) :
    φ = ψ := by
  rw [← toFilter_toApproxMap φ, ← toFilter_toApproxMap ψ, h]

theorem subFilter_iSupDirected {I : Type u} [Nonempty I] (φ : I → (funSpace E E).
    Element)
    (hdir : ∀ i j, ∃ k, φ i ≤ φ k ∧ φ j ≤ φ k)
    (hdir' : ∀ i j, ∃ k, subFilter (φ i) ≤ subFilter (φ k) ∧ subFilter (φ j) ≤
    subFilter (φ k)) :
    subFilter (iSupDirected φ hdir) = iSupDirected (fun i => subFilter (φ i)) hdir' :=
    by
  apply toApproxMap_injective
  rw [toApproxMap_subFilter]
  apply ApproximableMap.ext
  intro X Z
  rw [sub_toApproxMap_iSupDirected, toApproxMap_rel_iSupDirected]
  simp only [toApproxMap_subFilter]

theorem continuous_subFilter : Continuous (subFilter (E := E)) :=
  continuous_of_monotone_iSupDirected subFilter_monotone subFilter_iSupDirected

end Sub8_6

def subApprox : ApproximableMap (funSpace E E) (funSpace E E) :=
  ofContinuous Sub8_6.subFilter Sub8_6.continuous_subFilter

namespace subApprox

theorem toElementMap_eq (φ : (funSpace E E).Element) :
    ApproximableMap.toElementMap subApprox φ = Sub8_6.subFilter φ :=
  toElementMap_ofContinuous Sub8_6.subFilter Sub8_6.continuous_subFilter φ

def fixIso :
    {φ : (funSpace E E).Element // ApproximableMap.toElementMap subApprox φ = φ} ≃o
      {f : ApproximableMap E E // sub f = f} where
  toFun φ := ⟨toApproxMap φ.1, by
    have h1 : Sub8_6.subFilter φ.1 = φ.1 := (toElementMap_eq φ.1).symm.trans φ.2
    have h2 : toApproxMap (Sub8_6.subFilter φ.1) = toApproxMap φ.1 := congrArg
    toApproxMap h1
    rwa [Sub8_6.toApproxMap_subFilter] at h2⟩
  invFun f := ⟨toFilter f.1, by
    rw [toElementMap_eq]
    show Sub8_6.subFilter (toFilter f.1) = toFilter f.1
    unfold Sub8_6.subFilter
    rw [Sub8_6.toApproxMap_toFilter, f.2]⟩
  left_inv φ := Subtype.ext (Sub8_6.toFilter_toApproxMap φ.1)
  right_inv f := Subtype.ext (Sub8_6.toApproxMap_toFilter f.1)
  map_rel_iff' := by
```

```
    intro φ ψ
    show toApproxMap φ.1 ≤ toApproxMap ψ.1 ↔ φ.1 ≤ ψ.1
    exact (funSpaceEquiv E E).map_rel_iff

theorem isFinitary : IsFinitary (subApprox (E := E)) :=
  ⟨_, _, ⟨fixIso.trans
    ((finitaryProjectionSubsystemEquiv E).trans (SubsystemLattice.subsystemReprIso E))⟩
   ⟩

end subApprox

namespace Sub8_6

theorem subApprox_le_idMap : subApprox ≤ idMap (funSpace E E) := by
  rw [le_iff_toElementMap_le]
  intro φ
  rw [subApprox.toElementMap_eq, toElementMap_idMap]
  calc subFilter φ = toFilter (sub (toApproxMap φ)) := rfl
    _ ≤ toFilter (toApproxMap φ) := toFilter_le_iff.mpr (sub_le _)
    _ = φ := toFilter_toApproxMap φ

end Sub8_6

namespace Sub8_6

theorem subFilter_subFilter (φ : (funSpace E E).Element) :
    subFilter (subFilter φ) = subFilter φ := by
  unfold subFilter
  rw [toApproxMap_toFilter, sub_sub]

theorem subApprox.toElementMap_eq_comp (φ : (funSpace E E).Element) :
    (subApprox.comp subApprox).toElementMap φ = subApprox.toElementMap φ := by
  rw [toElementMap_comp]
  simp only [subApprox.toElementMap_eq]
  exact subFilter_subFilter φ

theorem isRetraction_subApprox : Retraction (subApprox (E := E)) :=
  ⟨le_antisymm (le_iff_toElementMap_le.mpr fun φ => (subApprox.toElementMap_eq_comp φ).
    le)
    (le_iff_toElementMap_le.mpr fun φ => (subApprox.toElementMap_eq_comp φ).ge)⟩

theorem isProjection_subApprox : IsProjection (subApprox (E := E)) :=
  ⟨isRetraction_subApprox, subApprox_le_idMap⟩

theorem isFinitaryProjection_subApprox : FinitaryProjection (subApprox (E := E)) :=
  ⟨⟨isProjection_subApprox, subApprox.isFinitary⟩⟩

end Sub8_6

section
open NeighborhoodSystem Domain.Recursive ApproximableMap Sub8_6
variable {α : Type*} {E : NeighborhoodSystem α}

theorem subApprox_rel_iff {F G : Set (ApproximableMap E E)} (hF : (funSpace E E).mem F
    ) :
    (subApprox (E := E)).rel F G ↔
```

```
      (funSpace E E).mem G ∧ sub (toApproxMap ((funSpace E E).principal hF)) ∈ G :=
  ⟨fun ⟨_, hG, hmem⟩ => ⟨hG, hmem⟩, fun ⟨hG, hmem⟩ => ⟨hF, hG, hmem⟩⟩

section SubComputable

variable (P : ComputablePresentation E) (gN : ℕ → ℕ)
  (hgN : ∀ c, gN c = 1 ↔
    (stepFun (funListOf P P (decodeList c)) : Set (ApproximableMap E E)).Nonempty)

theorem subApprox_rel_Xenum_iff (n m : ℕ) :
    (subApprox (E := E)).rel (Xenum P P gN n) (Xenum P P gN m) ↔
      gN m = 1 → ∀ e ∈ decodeList m,
        (sub (toApproxMap ((funSpace E E).principal (Xenum_mem P P gN hgN n)))).rel
          (P.X e.unpair.1) (P.X e.unpair.2) := by
  rw [subApprox_rel_iff (Xenum_mem P P gN hgN n), and_iff_right (Xenum_mem P P gN hgN
    m)]
  exact mem_Xenum_iff_map P P gN _ m

theorem subStep_recDecidable (fincl incl : ℕ → ℕ)
    (hfincl : ∀ s, fincl s = 1 ↔ Xenum P P gN s.unpair.1 ⊆ Xenum P P gN s.unpair.2)
    (hfinclp : Nat.Primrec fincl)
    (hincl : ∀ s, incl s = 1 ↔ P.X s.unpair.1 ⊆ P.X s.unpair.2) (hinclp : Nat.Primrec
     incl) :
    RecDecidable (fun w : ℕ =>
      Xenum P P gN w.unpair.2.unpair.1 ⊆ Xenum P P gN (codePair w.unpair.1 w.unpair.1)
    ∧
        P.X w.unpair.2.unpair.2.unpair.1 ⊆ P.X w.unpair.1 ∧
          P.X w.unpair.1 ⊆ P.X w.unpair.2.unpair.2.unpair.2) := by
  have hy : Nat.Primrec (fun w : ℕ => w.unpair.1) := Nat.Primrec.left
  have hn : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have he1 : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have he2 : Nat.Primrec (fun w : ℕ => w.unpair.2.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hcodeYY : Nat.Primrec (fun w : ℕ => codePair w.unpair.1 w.unpair.1) :=
    Nat.Primrec.succ.comp ((hy.pair hy).pair (Nat.Primrec.const 0))
  have h1 : RecDecidable (fun w : ℕ =>
      Xenum P P gN w.unpair.2.unpair.1 ⊆ Xenum P P gN (codePair w.unpair.1 w.unpair.1)
    ) :=
    ⟨fun w => fincl (Nat.pair w.unpair.2.unpair.1 (codePair w.unpair.1 w.unpair.1)),
      hfinclp.comp (hn.pair hcodeYY), fun w => by
        rw [hfincl]; simp only [unpair_pair_fst, unpair_pair_snd]⟩
  have h2 : RecDecidable (fun w : ℕ => P.X w.unpair.2.unpair.2.unpair.1 ⊆ P.X w.unpair
    .1) :=
    ⟨fun w => incl (Nat.pair w.unpair.2.unpair.2.unpair.1 w.unpair.1), hinclp.comp (he1
    .pair hy),
      fun w => by rw [hincl]; simp only [unpair_pair_fst, unpair_pair_snd]⟩
  have h3 : RecDecidable (fun w : ℕ => P.X w.unpair.1 ⊆ P.X w.unpair.2.unpair.2.unpair
    .2) :=
    ⟨fun w => incl (Nat.pair w.unpair.1 w.unpair.2.unpair.2.unpair.2), hinclp.comp (hy.
    pair he2),
      fun w => by rw [hincl]; simp only [unpair_pair_fst, unpair_pair_snd]⟩
  exact h1.and (h2.and h3)

theorem sub_rel_iff_exists_index {f : ApproximableMap E E} {a b : ℕ} :
```

```
    (sub f).rel (P.X a) (P.X b) ↔
      ∃ y : ℕ, f.rel (P.X y) (P.X y) ∧ P.X a ⊆ P.X y ∧ P.X y ⊆ P.X b := by
  rw [sub.rel, and_iff_right (P.mem_X a), and_iff_right (P.mem_X b)]
  constructor
  · rintro ⟨Y, ⟨hYE, hYf⟩, haY, hYb⟩
    obtain ⟨y, rfl⟩ := P.surj hYE
    exact ⟨y, hYf, haY, hYb⟩
  · rintro ⟨y, hYf, haY, hYb⟩
    exact ⟨P.X y, ⟨P.mem_X y, hYf⟩, haY, hYb⟩

theorem sub_leastMap_rel_iff (n a b : ℕ) :
    (sub (toApproxMap ((funSpace E E).principal (Xenum_mem P P gN hgN n)))).rel
        (P.X a) (P.X b) ↔
      ∃ y : ℕ, Xenum P P gN n ⊆ Xenum P P gN (codePair y y) ∧
        P.X a ⊆ P.X y ∧ P.X y ⊆ P.X b := by
  rw [sub_rel_iff_exists_index P]
  refine exists_congr (fun y => ?_)
  rw [leastMap_Xenum_rel P gN hgN n y y, Xenum_codePair P gN hgN y y]

theorem subApprox_isComputable
    (incl eq : ℕ → ℕ)
    (hgNp : Nat.Primrec gN)
    (hincl : ∀ s, incl s = 1 ↔ P.X s.unpair.1 ⊆ P.X s.unpair.2) (hinclp : Nat.Primrec
     incl)
    (heq : ∀ s, eq s = 1 ↔ P.X s.unpair.1 = P.X s.unpair.2) (heqp : Nat.Primrec eq) :
    ComputableMap
      (funPresentation P P gN incl incl eq hgN hgNp hincl hinclp hincl hinclp heq heqp
    )
      (funPresentation P P gN incl incl eq hgN hgNp hincl hinclp hincl hinclp heq heqp
    )
      (subApprox (E := E)) := by
  constructor
  obtain ⟨fincl, hfinclp, hfincls⟩ :=
    (funPresentation P P gN incl incl eq hgN hgNp hincl hinclp hincl hinclp heq heqp).
    incl_computable
  have hfincl : ∀ s, fincl s = 1 ↔ Xenum P P gN s.unpair.1 ⊆ Xenum P P gN s.unpair.2
    :=
    fun s => (hfincls s).symm
  show REPred₂ (fun n m => (subApprox (E := E)).rel (Xenum P P gN n) (Xenum P P gN m))
  have hdec := subStep_recDecidable P gN fincl incl hfincl hfinclp hincl hinclp
  have hre : REPred₂ (fun n e => ∃ y : ℕ,
      Xenum P P gN n ⊆ Xenum P P gN (codePair y y) ∧
        P.X e.unpair.1 ⊆ P.X y ∧ P.X y ⊆ P.X e.unpair.2) := by
    refine REPred.of_iff (fun t => ?_) hdec.re.proj
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hforall : REPred (fun t => ∀ e ∈ decodeList t.unpair.2, ∃ y : ℕ,
      Xenum P P gN t.unpair.1 ⊆ Xenum P P gN (codePair y y) ∧
        P.X e.unpair.1 ⊆ P.X y ∧ P.X y ⊆ P.X e.unpair.2) :=
    REPred.forall_mem_decodeList₂ hre
  have hne1 : REPred (fun t => ¬ gN t.unpair.2 = 1) :=
    ((RecDecidable.natEq (hgNp.comp Nat.Primrec.right) (Nat.Primrec.const 1)).not).re
  refine REPred.of_iff (fun t => ?_) (hne1.or hforall)
  show (subApprox (E := E)).rel (Xenum P P gN t.unpair.1) (Xenum P P gN t.unpair.2) ↔
    _
  rw [subApprox_rel_Xenum_iff P gN hgN t.unpair.1 t.unpair.2, Decidable.imp_iff_not_or
    ]
```

```
  refine or_congr_right (forall_congr' (fun e => imp_congr_right (fun _ => ?_)))
  exact sub_leastMap_rel_iff P gN hgN t.unpair.1 e.unpair.1 e.unpair.2

end SubComputable

theorem sub_isComputable_of_isEffectivelyGiven (h : E.IsEffectivelyGiven) :
    ∃ Pfun : ComputablePresentation (funSpace E E),
      ComputableMap Pfun Pfun (subApprox (E := E)) := by
  obtain ⟨P⟩ := h
  obtain ⟨incl, hinclp, hincls⟩ := P.incl_computable
  obtain ⟨eq, heqp, heqs⟩ := P.eq_computable
  obtain ⟨fc, hfcp, hfcs⟩ := P.cons_computable
  have hincl' : ∀ s, incl s = 1 ↔ P.X s.unpair.1 ⊆ P.X s.unpair.2 := fun s => (hincls
     s).symm
  have heq' : ∀ s, eq s = 1 ↔ P.X s.unpair.1 = P.X s.unpair.2 := fun s => (heqs s).
    symm
  have hgN : ∀ c, funConsChar P P fc fc c = 1 ↔
      (stepFun (funListOf P P (decodeList c)) : Set (ApproximableMap E E)).Nonempty :=
    fun c => funConsChar_spec P P fc fc (fun s => (hfcs s).symm) (fun s => (hfcs s).
    symm) c
  refine ⟨funPresentation P P (funConsChar P P fc fc) incl incl eq hgN
    (primrec_funConsChar P P fc fc hfcp hfcp) hincl' hinclp hincl' hinclp heq' heqp,
    subApprox_isComputable P (funConsChar P P fc fc) hgN incl eq
      (primrec_funConsChar P P fc fc hfcp hfcp) hincl' hinclp heq' heqp⟩

end

end DomainTheory.NeighbourhoodSystems
```

4.9.15 DprimeU

For an enumerable cover of neighbourhoods on a domain, index sets record inclusion among indices; interval presentations in the universal domain yield a subsystem isomorphic to the original domain.

Definition (`idxSet`). At index n, the index set comprises those indices m whose neighbourhood sits inside the nth neighbourhood.

Proof. By the subset relation on the cover. ■

Theorem (`mem_idxSet`). Membership in the index set at n is equivalent to the corresponding inclusion of neighbourhoods.

Proof. By definition. ■

Definition (`Yidx`). The nth encoded interval in the universal domain is obtained by the standard split-and-generate construction applied to the index family over all indices.

Proof. By the interval sequence operator with the chosen split specification. ■

Theorem (`D_mem_e`). Every neighbourhood in the enumerable cover belongs to the ambient domain.

Proof. By the covering hypothesis applied at the defining index. ■

Theorem (`e_subset_master`). Each cover neighbourhood lies below the domain master.

Proof. By the sub-master property on witnessed membership. ■

Theorem (`idxSet_inter_of_inter_eq`). When two cover neighbourhoods intersect in a third, the corresponding index sets intersect in the index set of that third.

Proof. By rewriting intersection of index sets as simultaneous inclusions and applying the hypothesis on set intersections. ■

Theorem (`idxSet_zero_eq_univ`). If every index sits below a fixed index, that index's index set is the full index universe.

Proof. By extensionality using the universal inclusion hypothesis. ■

Theorem (`idxSet_zero`). The index set at zero is universal when zero names the master.

Proof. By the master naming hypothesis and sub-master comparisons. ■

Theorem (`self_mem_idxSet`). Each index belongs to its own index set.

Proof. By reflexivity of inclusion. ■

Theorem (`idxSet_subset_iff`). Inclusion of index sets at i and j is equivalent to inclusion of the corresponding neighbourhoods.

Proof. By the self-membership criterion in one direction and transitivity of inclusion in the other. ■

Theorem (`univ_nonempty_nat`). The set of all natural-number indices is nonempty.

Proof. Zero witnesses. ■

Theorem (`Yidx_empty_or_mem`). Each encoded interval is either empty or a legal neighbourhood of the universal domain.

Proof. By the standard empty-or-membership transfer for interval sequences. ■

Theorem (`Yidx_nonempty`). Each encoded interval is nonempty.

Proof. By transferring nonemptiness from the index set at n with witness n. ■

Theorem (`Yidx_mem`). Every encoded interval belongs to the universal domain.

Proof. By ruling out emptiness. ■

Theorem (`Yidx_subset_master`). Each encoded interval sits below the universal master.

Proof. By the interval-sequence sub-master lemma. ■

Theorem (`Yidx_zero`). The interval at index zero equals the universal master.

Proof. By the zero-index master law for interval sequences and universality of the index set at zero. ■

Theorem (`embed_subset_iff`). Inclusion of cover neighbourhoods at indices i and j matches inclusion of their encoded intervals.

Proof. By the index-set comparison and the subset transfer principle for interval presentations. ■

Theorem (`embed_eq_iff`). Equality of cover neighbourhoods matches equality of encoded intervals.

Proof. By antisymmetry of the subset characterizations. ■

Theorem (`exists_inter_index_of_dmem`). When the domain admits the intersection of two cover sets, some index names both the set intersection and the intersection of the encoded intervals.

Proof. By the covering map on the intersection and the intersection transfer for interval presentations. ■

Definition (`DprimeU`). The subsystem generated by encoded intervals: a set is admitted when it equals some $Yidx$, with the universal master and intersection law inherited via intersection indices.

Proof. By the cover, master, sub-master, and intersection witnesses assembled from embedding and existence lemmas. ■

Theorem (`exists_inter_index_of_nonempty`). Nonempty intersection of encoded intervals yields a common intersection index as above.

Proof. By contrapositive transfer of emptiness to index sets and the domain intersection lemma. ■

Theorem (`DprimeU_subsystem`). The encoded-interval system embeds as a subsystem of the universal domain.

Proof. By master coincidence, membership of generators, and intersection closure from the nonempty-intersection index. ■

Definition (`toD`). From an element of the encoded subsystem, form a domain element by reading membership through cover indices matching encoded intervals.

Proof. By the element axioms using cover, master, intersection, and upward-closure transfer along embeddings. ■

Definition (`toDprimeU`). From a domain element, form an encoded-subsystem element by witnessing encoded intervals from cover indices.

Proof. Dually, using intersection indices from domain intersections. ■

Definition (`domainIso`). The two translations are mutually inverse order isomorphisms on ideal elements.

Proof. By element extension on both sides and the order characterization via embeddings and cover. ■

Theorem (`idxSet_eq_iff`). Equality of index sets matches equality of cover neighbourhoods.

Proof. By antisymmetry of the subset equivalence. ■

Theorem (`idxSet_nonempty`). Every index set is nonempty.

Proof. The index witnesses itself. ■

Theorem (`isomorphic_DprimeU`). The original domain and the encoded-interval subsystem are order-isomorphic.

Proof. By packaging the domain isomorphism. ■

DprimeU.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.Yseq
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.U
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
public import DomainTheory.NeighbourhoodSystems.DomainEquations.subdomain

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive

variable {α : Type*}

section Reencode

variable (e : ℕ → Set α)

def idxSet (n : ℕ) : Set ℕ := {m | e m ⊆ e n}

@[simp] theorem mem_idxSet {m n : ℕ} : m ∈ idxSet e n ↔ e m ⊆ e n := Iff.rfl

end Reencode

section Setup

variable (D : NeighborhoodSystem α) (e : ℕ → Set α)
  (hcover : ∀ S, D.mem S ↔ ∃ n, S = e n) (he0 : e 0 = D.master)

include hcover in

theorem D_mem_e (n : ℕ) : D.mem (e n) := (hcover (e n)).mpr ⟨n, rfl⟩

noncomputable abbrev Yidx (n : ℕ) : Set ? := Yseq splitChoice (idxSet e) Set.univ n
```

```
include hcover in
theorem e_subset_master (n : ℕ) : e n ⊆ D.master := D.sub_master (D_mem_e D e hcover n
    )

end Setup

section Reencode

variable (e : ℕ → Set α)

theorem idxSet_inter_of_inter_eq {i j m : ℕ} (h : e i ∩ e j = e m) :
    idxSet e i ∩ idxSet e j = idxSet e m := by
  ext k
  show e k ⊆ e i ∧ e k ⊆ e j ↔ e k ⊆ e m
  rw [← Set.subset_inter_iff, h]

theorem idxSet_zero_eq_univ {n0 : ℕ} (hsub : ∀ m, e m ⊆ e n0) : idxSet e n0 = Set.univ
    :=
  Set.eq_univ_of_forall hsub

end Reencode

section Setup

variable (D : NeighborhoodSystem α) (e : ℕ → Set α)
  (hcover : ∀ S, D.mem S ↔ ∃ n, S = e n) (he0 : e 0 = D.master)

include hcover he0 in
theorem idxSet_zero : idxSet e 0 = Set.univ :=
  idxSet_zero_eq_univ e (fun m => he0 ▸ e_subset_master D e hcover m)

end Setup

section Reencode

variable (e : ℕ → Set α)

theorem self_mem_idxSet (n : ℕ) : n ∈ idxSet e n := show e n ⊆ e n from subset_rfl

theorem idxSet_subset_iff (i j : ℕ) : idxSet e i ⊆ idxSet e j ↔ e i ⊆ e j :=
  ⟨fun h => h (self_mem_idxSet e i), fun h _ hm => hm.trans h⟩

end Reencode

section Setup

variable (D : NeighborhoodSystem α) (e : ℕ → Set α)
  (hcover : ∀ S, D.mem S ↔ ∃ n, S = e n) (he0 : e 0 = D.master)

theorem univ_nonempty_nat : (Set.univ : Set ℕ).Nonempty := ⟨0, trivial⟩

theorem Yidx_empty_or_mem (n : ℕ) : Yidx e n = ∅ ∨ U.mem (Yidx e n) :=
  Yseq_empty_or_mem splitChoice (idxSet e) Set.univ univ_nonempty_nat
    splitChoice_isSplitSpec n

theorem Yidx_nonempty (n : ℕ) : (Yidx e n).Nonempty :=
```

```
  Yseq_nonempty_of_mem splitChoice (idxSet e) Set.univ univ_nonempty_nat
    splitChoice_isSplitSpec
    (Set.mem_univ n) (self_mem_idxSet e n)

theorem Yidx_mem (n : ℕ) : U.mem (Yidx e n) :=
  (Yidx_empty_or_mem e n).resolve_left (Yidx_nonempty e n).ne_empty

theorem Yidx_subset_master (n : ℕ) : Yidx e n ⊆ U.master :=
  Yseq_subset_master splitChoice (idxSet e) Set.univ univ_nonempty_nat
    splitChoice_isSplitSpec n

include hcover he0 in

theorem Yidx_zero : Yidx e 0 = U.master :=
  Yseq_zero_eq_master splitChoice (idxSet e) Set.univ univ_nonempty_nat
    splitChoice_isSplitSpec
    (idxSet_zero D e hcover he0)

theorem embed_subset_iff (i j : ℕ) : e i ⊆ e j ↔ Yidx e i ⊆ Yidx e j := by
  rw [← idxSet_subset_iff e i j]
  have := transfer_subset_iff splitChoice (idxSet e) Set.univ univ_nonempty_nat
    splitChoice_isSplitSpec i j
  rwa [Set.univ_inter, Set.inter_eq_self_of_subset_right (Yidx_subset_master e i)] at
    this

theorem embed_eq_iff (i j : ℕ) : e i = e j ↔ Yidx e i = Yidx e j :=
  ⟨fun h => Set.Subset.antisymm ((embed_subset_iff e i j).mp h.subset)
      ((embed_subset_iff e j i).mp h.symm.subset),
    fun h => Set.Subset.antisymm ((embed_subset_iff e i j).mpr h.subset)
      ((embed_subset_iff e j i).mpr h.symm.subset)⟩

end Setup

section Subsystem

variable (D : NeighborhoodSystem α) (e : ℕ → Set α)
  (hcover : ∀ S, D.mem S ↔ ∃ n, S = e n) (he0 : e 0 = D.master)

include hcover in

theorem exists_inter_index_of_dmem {i j : ℕ} (hDij : D.mem (e i ∩ e j)) :
    ∃ m, e i ∩ e j = e m ∧ Yidx e i ∩ Yidx e j = Yidx e m := by
  obtain ⟨m, hm⟩ := (hcover (e i ∩ e j)).mp hDij
  exact ⟨m, hm, (transfer_inter_eq_iff splitChoice (idxSet e) Set.univ
    univ_nonempty_nat
    splitChoice_isSplitSpec i j m
    (Set.subset_univ _) (Set.subset_univ _) (Set.subset_univ _)).mp (
    idxSet_inter_of_inter_eq e hm)⟩

noncomputable def DprimeU : NeighborhoodSystem ? where
  mem Y := ∃ n, Y = Yidx e n
  master := U.master
  master_nonempty := U.master_nonempty
  master_mem := ⟨0, (Yidx_zero D e hcover he0).symm⟩
  sub_master := by rintro Y ⟨n, rfl⟩; exact Yidx_subset_master e n
  inter_mem := by
```

```
    rintro X Y Z ⟨i, rfl⟩ ⟨j, rfl⟩ ⟨k, rfl⟩ hZsub
    have h1 : e k ⊆ e i := (embed_subset_iff e k i).mpr (hZsub.trans Set.
    inter_subset_left)
    have h2 : e k ⊆ e j := (embed_subset_iff e k j).mpr (hZsub.trans Set.
    inter_subset_right)
    obtain ⟨m, -, hYeq⟩ := exists_inter_index_of_dmem D e hcover
      (D.inter_mem (D_mem_e D e hcover i) (D_mem_e D e hcover j) (D_mem_e D e hcover k
    )
        (Set.subset_inter h1 h2))
    exact ⟨m, hYeq⟩

include hcover in

theorem exists_inter_index_of_nonempty {i j : ℕ} (hne : (Yidx e i ∩ Yidx e j).Nonempty
    ) :
    ∃ m, e i ∩ e j = e m ∧ Yidx e i ∩ Yidx e j = Yidx e m := by
  have hne' : (idxSet e i ∩ idxSet e j).Nonempty := by
    by_contra hcon
    rw [Set.not_nonempty_iff_eq_empty] at hcon
    have hkey := transfer_inter_empty_iff splitChoice (idxSet e) Set.univ
    univ_nonempty_nat
      splitChoice_isSplitSpec i j
    rw [Set.univ_inter, Set.inter_eq_self_of_subset_right (Yidx_subset_master e i)] at
     hkey
    exact hne.ne_empty (hkey.mp hcon)
  obtain ⟨k, hki, hkj⟩ := hne'
  exact exists_inter_index_of_dmem D e hcover
    (D.inter_mem (D_mem_e D e hcover i) (D_mem_e D e hcover j) (D_mem_e D e hcover k)
      (Set.subset_inter hki hkj))

include hcover he0 in

theorem DprimeU_subsystem : DprimeU D e hcover he0 ? U where
  master_eq := rfl
  sub := by rintro Y ⟨n, rfl⟩; exact Yidx_mem e n
  inter_closed := by
    rintro X Y ⟨i, rfl⟩ ⟨j, rfl⟩ hUmem
    obtain ⟨m, -, hYeq⟩ := exists_inter_index_of_nonempty D e hcover hUmem.2.1
    exact ⟨m, hYeq⟩

end Subsystem

section Iso

variable (D : NeighborhoodSystem α) (e : ℕ → Set α)
  (hcover : ∀ S, D.mem S ↔ ∃ n, S = e n) (he0 : e 0 = D.master)

def toD (y : (DprimeU D e hcover he0).Element) : D.Element where
  mem S := ∃ n, S = e n ∧ y.mem (Yidx e n)
  sub := fun ⟨n, hn, _⟩ => hn ▸ D_mem_e D e hcover n
  master_mem := ⟨0, he0.symm, by rw [Yidx_zero D e hcover he0]; exact y.master_mem⟩
  inter_mem := by
    rintro S T ⟨i, rfl, hyi⟩ ⟨j, rfl, hyj⟩
    have hD'mem : (DprimeU D e hcover he0).mem (Yidx e i ∩ Yidx e j) := y.sub (y.
    inter_mem hyi hyj)
    obtain ⟨m, hem, hYeq⟩ := exists_inter_index_of_nonempty D e hcover
```

```
      ((DprimeU_subsystem D e hcover he0).sub hD'mem).2.1
    exact ⟨m, hem, hYeq ▸ y.inter_mem hyi hyj⟩
  up_mem := by
    rintro S T ⟨i, rfl, hyi⟩ hDT hST
    obtain ⟨j, rfl⟩ := (hcover T).mp hDT
    have hYij : Yidx e i ⊆ Yidx e j := (embed_subset_iff e i j).mp hST
    exact ⟨j, rfl, y.up_mem hyi ⟨j, rfl⟩ hYij⟩

def toDprimeU (x : D.Element) : (DprimeU D e hcover he0).Element where
  mem Y := ∃ n, Y = Yidx e n ∧ x.mem (e n)
  sub := fun ⟨n, hn, _⟩ => ⟨n, hn⟩
  master_mem := ⟨0, (Yidx_zero D e hcover he0).symm, by rw [he0]; exact x.master_mem⟩
  inter_mem := by
    rintro X Y ⟨i, rfl, hxi⟩ ⟨j, rfl, hxj⟩
    obtain ⟨m, hem, hYeq⟩ :=
      exists_inter_index_of_dmem D e hcover (x.sub (x.inter_mem hxi hxj))
    exact ⟨m, hYeq, hem ▸ x.inter_mem hxi hxj⟩
  up_mem := by
    rintro X Y ⟨i, rfl, hxi⟩ ⟨j, rfl⟩ hXY
    have heij : e i ⊆ e j := (embed_subset_iff e i j).mpr hXY
    exact ⟨j, rfl, x.up_mem hxi (D_mem_e D e hcover j) heij⟩

noncomputable def domainIso : DomainIso D (DprimeU D e hcover he0) where
  toFun := toDprimeU D e hcover he0
  invFun := toD D e hcover he0
  left_inv x := by
    apply Element.ext
    intro S
    constructor
    · rintro ⟨n, hn, k, hk, hxk⟩
      rw [hn, (embed_eq_iff e n k).mpr hk]
      exact hxk
    · intro hS
      obtain ⟨n, hn⟩ := (hcover S).mp (x.sub hS)
      refine ⟨n, hn, n, rfl, ?_⟩
      rwa [← hn]
  right_inv y := by
    apply Element.ext
    intro Y
    constructor
    · rintro ⟨n, hn, k, hk, hyk⟩
      rw [hn, (embed_eq_iff e n k).mp hk]
      exact hyk
    · intro hY
      obtain ⟨n, hn⟩ := y.sub hY
      refine ⟨n, hn, n, rfl, ?_⟩
      rwa [← hn]
  map_rel_iff' := by
    intro x x2
    constructor
    · intro hle S hxS
      obtain ⟨n, hn⟩ := (hcover S).mp (x.sub hxS)
      have hxn : x.mem (e n) := hn ▸ hxS
      obtain ⟨k, hk, hx2k⟩ := hle _ (⟨n, rfl, hxn⟩ : (toDprimeU D e hcover he0 x).mem (
    Yidx e n))
      rw [hn, (embed_eq_iff e n k).mpr hk]
```

```
      exact hx2k
    · intro hle Y hY
      obtain ⟨n, hn, hxn⟩ := hY
      exact ⟨n, hn, hle _ hxn⟩

theorem isomorphic_DprimeU : D ≅? DprimeU D e hcover he0 := ⟨domainIso D e hcover he0⟩

end Iso

section Reencode

variable (e : ℕ → Set α)

theorem idxSet_eq_iff (i j : ℕ) : idxSet e i = idxSet e j ↔ e i = e j :=
  ⟨fun h => Set.Subset.antisymm ((idxSet_subset_iff e i j).mp h.subset)
      ((idxSet_subset_iff e j i).mp h.symm.subset),
    fun h => by rw [idxSet, idxSet, h]⟩

theorem idxSet_nonempty (n : ℕ) : (idxSet e n).Nonempty := ⟨n, self_mem_idxSet e n⟩

end Reencode

end DomainTheory.NeighbourhoodSystems
```

4.9.16 Theorem 8.6 — sub combinator

THEOREM 8.6. For any domain E define

$$\mathrm{sub} : (E \to E) \to (E \to E)$$

by the formula

$$X \ \mathrm{sub}\,(f)\ Z \quad \text{iff} \quad \exists Y \in E.\quad X \subseteq Y \ \ f\ Y \subseteq Z,$$

for all $X, Z \in E$ and all $f : E \to E$. Then the range of sub consists exactly of the finitary projections on E, and moreover sub itself is a finitary projection on $(E \to E)$. If E is effectively given, then sub is computable.

Proof: It is trivial to check that $\mathrm{sub}(f)$ is always approximable. Also, it is obvious from the definition that the correspondence

$$f \mapsto \mathrm{sub}(f)$$

preserves directed unions of f's. Thus, sub is itself approximable. We note that

$$X \subseteq Y \ \ f\ Y \subseteq Z \text{ always implies } X\ f\ Z;$$

hence, $\mathrm{sub}(f) \subseteq f$ holds. Also

$$Y\ f\ Y \text{ always implies } Y\ \mathrm{sub}(f)\ Y,$$

hence, $\mathrm{sub}(f) \subseteq \mathrm{sub}(\mathrm{sub}(f))$ holds. This shows sub to be a projection on $(E \to E)$. The effectiveness of the definition makes it also clear that sub is computable when E has a computable presentation.

Since, sub is a projection, its range is the same as its fixed-point set. If

$$\mathrm{sub}(a) = a,$$

then there is no problem in checking that a satisfies 8.5(ii) *and conversely*. So the range of sub picks out exactly the finitary projections in view of 8.5.

Finally, to prove that sub is a finitary projection of $(E \to E)$, we invoke 6.11 and remark that, in view of 8.2, the fixed point set (range) of sub is in a one-one inclusion-preserving correspondence with the domain $\{D \mid D \triangleleft E\}$. □

These results have almost completely translated the theory of $\triangleleft$-subdomains into λ-calculus via the sub-combinator. One last step will complete the passage, and then we shall be able to return to solving domain equations.

Theorem_8_6.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.sub
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.subApprox
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.DprimeU

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap Domain.Recursive

theorem theorem_86 :
    (∀ {α : Type*} {E : NeighborhoodSystem α} {f : ApproximableMap E E},
      sub f = f ↔ FinitaryProjection f) ∧
    (∀ {α : Type*} {E : NeighborhoodSystem α}, IsFinitary (subApprox (E := E))) ∧
    (∀ {α : Type*} {E : NeighborhoodSystem α}, FinitaryProjection (subApprox (E := E))
    ) :=
  ⟨@sub_eq_self_iff_isFinitaryProjection, @subApprox.isFinitary,
    @Sub8_6.isFinitaryProjection_subApprox⟩

theorem theorem_86_c :
    (∀ {α : Type*} {E : NeighborhoodSystem α}
      (P : ComputablePresentation E) (gN : ℕ → ℕ)
      (hgN : ∀ c, gN c = 1 ↔
        (stepFun (funListOf P P (decodeList c)) : Set (ApproximableMap E E)).Nonempty)
      (incl eq : ℕ → ℕ) (hgNp : Nat.Primrec gN)
      (hincl : ∀ s, incl s = 1 ↔ P.X s.unpair.1 ⊆ P.X s.unpair.2) (hinclp : Nat.
    Primrec incl)
      (heq : ∀ s, eq s = 1 ↔ P.X s.unpair.1 = P.X s.unpair.2) (heqp : Nat.Primrec eq),
      ComputableMap
        (funPresentation P P gN incl incl eq hgN hgNp hincl hinclp hincl hinclp heq
    heqp)
        (funPresentation P P gN incl incl eq hgN hgNp hincl hinclp hincl hinclp heq
    heqp)
        (subApprox (E := E))) ∧
    (∀ {α : Type*} {E : NeighborhoodSystem α},
      E.IsEffectivelyGiven →
        ∃ Pfun : ComputablePresentation (funSpace E E),
          ComputableMap Pfun Pfun (subApprox (E := E))) :=
  ⟨@subApprox_isComputable, @sub_isComputable_of_isEffectivelyGiven⟩
```

```
end DomainTheory.NeighbourhoodSystems
```

4.9.17 Presented intervals

Finite lists of rational half-open intervals present unions used throughout the universal domain.

Definition (`presentedIntervals`). The union of the half-open intervals given by a list of rational endpoints.

Proof. By union over the list. ■

Definition (`clip`). Clip an endpoint pair to lie within the unit interval by taking the maximum of lower bounds with zero and the minimum of upper bounds with one.

Proof. By the lattice operations on rationals. ■

Definition (`clipGe`). Raise the lower endpoint of a pair to at least a given rational bound.

Proof. By supremum with the bound on the first coordinate. ■

Definition (`clipLt`). Lower the upper endpoint of a pair to at most a given rational bound.

Proof. By infimum with the bound on the second coordinate. ■

Definition (`combineIntervals`). Form every pairwise combined interval by meeting lowers and joining uppers across two lists.

Proof. By flat map and map on endpoint pairs. ■

Theorem (`Ico_diff_Ico`). The set difference of two half-open rational intervals is the union of two half-open pieces described by extrema of the four endpoints.

Proof. By case analysis on order relations among the endpoints. ■

Definition (`diffOneList`). Present the difference of two half-open intervals as a list of at most two interval pieces.

Proof. By the endpoint formulas from the difference lemma. ■

Definition (`diffSingleList`). Subtract one interval from each member of an interval list.

Proof. By flat-mapping the single-interval difference. ■

Definition (`diffAllList`). Subtract a list of intervals from one interval by folding single-list differences.

Proof. By left fold starting from the singleton list of the first interval. ■

Definition (`diffLists`). Present the difference of two unions as the flat union of per-interval differences.

Proof. By flat map over the first list. ■

Theorem (`presentedIntervals.mem`). Membership in a presented union is equivalent to membership in some listed half-open interval.

Proof. By unfolding the union and interval membership. ■

Theorem (`presentedIntervals.nil`). The empty list presents the empty set.

Proof. By the membership characterization. ■

Theorem (`presentedIntervals.cons`). Consing an interval presents the union of that interval with the tail presentation.

Proof. By the membership characterization and disjunction. ■

Theorem (`presentedIntervals.append`). Appending lists presents the union of the two presentations.

Proof. By induction on the first list and the cons lemma. ■

Theorem (`presentedIntervals.congr`). Pointwise list equivalence yields equal presented unions.

Proof. By the membership characterization and bidirectional inclusion. ■

Theorem (`presentedIntervals.inter`). Intersection of two presented unions equals the presentation built by combining interval lists.

Proof. By the membership characterization and the combine-interval construction. ■

Theorem (`presentedIntervals.diffOneList`). The list difference of two intervals presents their set difference.

Proof. By the cons lemmas and the rational interval difference identity. ■

Theorem (`presentedIntervals.diffSingleList`). Subtracting one interval from a presentation presents the corresponding set difference.

Proof. By induction on the list and the one-interval case. ■

Theorem (`presentedIntervals.foldl_diffSingleList`). Folding single-list subtraction through a second list presents subtracting the second union from the first.

Proof. By induction on the subtracted list. ■

Theorem (`presentedIntervals.diffAllList`). The all-list difference of one interval against a union presents the set difference.

Proof. By the fold lemma and empty union on the right. ■

Theorem (`presentedIntervals.diffLists`). The list difference of two presentations presents their set difference.

Proof. By induction on the first list and the all-list case. ■

Theorem (`presentedIntervals.map_clip`). Clipping every interval and presenting the list gives the intersection of the original union with the unit half-open interval.

Proof. By the membership characterization and clip inequalities. ■

Theorem (`presentedIntervals.filter_lt`). Filtering to strict lower bounds does not change the presented union.

Proof. By the membership characterization; strict endpoints are already forced on members. ■

Theorem (`presentedIntervals.map_clipLt`). Clipping uppers to a bound and presenting gives the intersection of the union with the open ray below the bound.

Proof. By the membership characterization. ■

Theorem (`presentedIntervals.map_clipGe`). Clipping lowers to a bound and presenting gives the intersection of the union with the closed ray from the bound upward.

Proof. By the membership characterization. ■

`presentedIntervals.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
public import Mathlib.Algebra.Order.Ring.Unbundled.Rat
public import Mathlib.Order.SetNotation
public import Mathlib.Order.Interval.Set.Defs
import Mathlib.Data.Set.Basic
import Mathlib.Order.Interval.Set.Basic

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
```

```
def presentedIntervals (L : List (? × ?)) : Set ? := ⋃ p ∈ L, Set.Ico p.1 p.2

def clip (p : ? × ?) : ? × ? := (p.1 ⊔ 0, p.2 ⊓ 1)

def clipGe (m : ?) (p : ? × ?) : ? × ? := (p.1 ⊔ m, p.2)

def clipLt (m : ?) (p : ? × ?) : ? × ? := (p.1, p.2 ⊓ m)

def combineIntervals (L1 L2 : List (? × ?)) : List (? × ?) :=
  L1.flatMap (fun p => L2.map (fun q => (p.1 ⊔ q.1, p.2 ⊓ q.2)))

theorem Ico_diff_Ico (a b c d : ?) :
    Set.Ico a b \ Set.Ico c d = Set.Ico a (b ⊓ c) ∪ Set.Ico (a ⊔ d) b := by
  ext x
  simp only [Set.mem_sdiff, Set.mem_Ico, Set.mem_union, lt_inf_iff, sup_le_iff]
  constructor
  · rintro ⟨⟨ha, hb⟩, hcd⟩
    rcases lt_or_ge x c with hc | hc
    · exact Or.inl ⟨ha, hb, hc⟩
    · rcases lt_or_ge x d with hd | hd
      · exact absurd ⟨hc, hd⟩ hcd
      · exact Or.inr ⟨⟨ha, hd⟩, hb⟩
  · rintro (⟨ha, hb, hc⟩ | ⟨⟨ha, hd⟩, hb⟩)
    · exact ⟨⟨ha, hb⟩, fun ⟨hc', _⟩ => absurd hc' (not_le.mpr hc)⟩
    · exact ⟨⟨ha, hb⟩, fun ⟨_, hd'⟩ => absurd hd' (not_lt.mpr hd)⟩

def diffOneList (p q : ? × ?) : List (? × ?) := [(p.1, p.2 ⊓ q.1), (p.1 ⊔ q.2, p.2)]

def diffSingleList (L : List (? × ?)) (q : ? × ?) : List (? × ?) :=
  L.flatMap (fun p => diffOneList p q)

def diffAllList (p : ? × ?) (L2 : List (? × ?)) : List (? × ?) := L2.foldl
    diffSingleList [p]

def diffLists (L1 L2 : List (? × ?)) : List (? × ?) := L1.flatMap (fun p =>
    diffAllList p L2)

namespace presentedIntervals

@[simp] theorem mem {L : List (? × ?)} {x : ?} :
    x ∈ presentedIntervals L ↔ ∃ p ∈ L, p.1 ≤ x ∧ x < p.2 := by
  simp only [presentedIntervals, Set.mem_iUnion, Set.mem_Ico, exists_prop]

@[simp] theorem nil : presentedIntervals ([] : List (? × ?)) = ∅ := by
  ext x; simp

theorem cons (p : ? × ?) (L : List (? × ?)) :
    presentedIntervals (p :: L) = Set.Ico p.1 p.2 ∪ presentedIntervals L := by
  ext x; simp [Set.mem_Ico, or_and_right, exists_or]

theorem append (L1 L2 : List (? × ?)) :
    presentedIntervals (L1 ++ L2) = presentedIntervals L1 ∪ presentedIntervals L2 :=
    by
  induction L1 with
  | nil => simp
```

```
  | cons p L1 ih => simp [List.cons_append, cons, ih, Set.union_assoc]

theorem congr {L1 L2 : List (? × ?)} (h : ∀ p, p ∈ L1 ↔ p ∈ L2) :
    presentedIntervals L1 = presentedIntervals L2 := by
  ext x
  simp only [mem]
  constructor
  · rintro ⟨p, hp, h1, h2⟩; exact ⟨p, (h p).mp hp, h1, h2⟩
  · rintro ⟨p, hp, h1, h2⟩; exact ⟨p, (h p).mpr hp, h1, h2⟩

theorem inter (L1 L2 : List (? × ?)) :
    presentedIntervals L1 ∩ presentedIntervals L2 = presentedIntervals (
    combineIntervals L1 L2) := by
  ext x
  simp only [mem, combineIntervals, List.mem_flatMap, List.mem_map,
    Set.mem_inter_iff]
  constructor
  · rintro ⟨⟨p, hp, hxp1, hxp2⟩, ⟨q, hq, hxq1, hxq2⟩⟩
    exact ⟨(p.1 ⊔ q.1, p.2 ⊓ q.2), ⟨p, hp, q, hq, rfl⟩, sup_le hxp1 hxq1, lt_inf_iff.
    mpr ⟨hxp2, hxq2⟩⟩
  · rintro ⟨r, ⟨p, hp, q, hq, rfl⟩, hx1, hx2⟩
    exact ⟨⟨p, hp, le_sup_left.trans hx1, hx2.trans_le inf_le_left⟩,
      ⟨q, hq, le_sup_right.trans hx1, hx2.trans_le inf_le_right⟩⟩

theorem diffOneList (p q : ? × ?) :
    presentedIntervals (_root_.DomainTheory.NeighbourhoodSystems.diffOneList p q) =
      Set.Ico p.1 p.2 \ Set.Ico q.1 q.2 := by
  unfold _root_.DomainTheory.NeighbourhoodSystems.diffOneList
  rw [cons, cons, nil, Set.union_empty]
  exact (Ico_diff_Ico p.1 p.2 q.1 q.2).symm

theorem diffSingleList (L : List (? × ?)) (q : ? × ?) :
    presentedIntervals (_root_.DomainTheory.NeighbourhoodSystems.diffSingleList L q) =
      presentedIntervals L \ Set.Ico q.1 q.2 := by
  induction L with
  | nil => simp [_root_.DomainTheory.NeighbourhoodSystems.diffSingleList]
  | cons a l ih =>
    rw [show _root_.DomainTheory.NeighbourhoodSystems.diffSingleList (a :: l) q =
        _root_.DomainTheory.NeighbourhoodSystems.diffOneList a q ++
          _root_.DomainTheory.NeighbourhoodSystems.diffSingleList l q from rfl,
      append, presentedIntervals.diffOneList, ih, cons, Set.union_sdiff_distrib]

theorem foldl_diffSingleList (L2 : List (? × ?)) :
    ∀ acc : List (? × ?),
      presentedIntervals (L2.foldl _root_.DomainTheory.NeighbourhoodSystems.
    diffSingleList acc) =
        presentedIntervals acc \ presentedIntervals L2 := by
  induction L2 with
  | nil => intro acc; simp
  | cons a l ih =>
    intro acc
    rw [List.foldl_cons,
      ih (_root_.DomainTheory.NeighbourhoodSystems.diffSingleList acc a),
      presentedIntervals.diffSingleList, cons, Set.sdiff_sdiff]

theorem diffAllList (p : ? × ?) (L2 : List (? × ?)) :
```

```
    presentedIntervals (_root_.DomainTheory.NeighbourhoodSystems.diffAllList p L2) =
      Set.Ico p.1 p.2 \ presentedIntervals L2 := by
  unfold _root_.DomainTheory.NeighbourhoodSystems.diffAllList
  rw [foldl_diffSingleList, cons, nil, Set.union_empty]

theorem diffLists (L1 L2 : List (? × ?)) :
    presentedIntervals (_root_.DomainTheory.NeighbourhoodSystems.diffLists L1 L2) =
      presentedIntervals L1 \ presentedIntervals L2 := by
  induction L1 with
  | nil => simp [_root_.DomainTheory.NeighbourhoodSystems.diffLists]
  | cons a l ih =>
    rw [show _root_.DomainTheory.NeighbourhoodSystems.diffLists (a :: l) L2 =
        _root_.DomainTheory.NeighbourhoodSystems.diffAllList a L2 ++
          _root_.DomainTheory.NeighbourhoodSystems.diffLists l L2 from rfl,
      append, presentedIntervals.diffAllList, ih, cons, Set.union_sdiff_distrib]
theorem map_clip (L : List (? × ?)) :
    presentedIntervals (L.map clip) = presentedIntervals L ∩ Set.Ico (0 : ?) 1 := by
  ext x
  simp only [mem, List.mem_map, Set.mem_inter_iff, Set.mem_Ico, clip]
  constructor
  · rintro ⟨p, ⟨q, hq, rfl⟩, hx1, hx2⟩
    exact ⟨⟨q, hq, le_sup_left.trans hx1, hx2.trans_le inf_le_left⟩,
      le_sup_right.trans hx1, hx2.trans_le inf_le_right⟩
  · rintro ⟨⟨q, hq, hxq1, hxq2⟩, hx0, hx1⟩
    exact ⟨(q.1 ⊔ 0, q.2 ⊓ 1), ⟨q, hq, rfl⟩, sup_le hxq1 hx0, lt_inf_iff.mpr ⟨hxq2, hx1⟩
    ⟩

theorem filter_lt (L : List (? × ?)) :
    presentedIntervals (L.filter (fun p => decide (p.1 < p.2))) = presentedIntervals L
     := by
  ext x
  simp only [mem, List.mem_filter, decide_eq_true_eq]
  constructor
  · rintro ⟨p, ⟨hp, -⟩, hx⟩
    exact ⟨p, hp, hx⟩
  · rintro ⟨p, hp, hx1, hx2⟩
    exact ⟨p, ⟨hp, lt_of_le_of_lt hx1 hx2⟩, hx1, hx2⟩
theorem map_clipLt (m : ?) (L : List (? × ?)) :
    presentedIntervals (L.map (clipLt m)) = presentedIntervals L ∩ Set.Iio m := by
  ext x
  simp only [mem, List.mem_map, Set.mem_inter_iff, Set.mem_Iio, clipLt]
  constructor
  · rintro ⟨p, ⟨q, hq, rfl⟩, hx1, hx2⟩
    exact ⟨⟨q, hq, hx1, hx2.trans_le inf_le_left⟩, hx2.trans_le inf_le_right⟩
  · rintro ⟨⟨q, hq, hxq1, hxq2⟩, hxm⟩
    exact ⟨(q.1, q.2 ⊓ m), ⟨q, hq, rfl⟩, hxq1, lt_inf_iff.mpr ⟨hxq2, hxm⟩⟩

theorem map_clipGe (m : ?) (L : List (? × ?)) :
    presentedIntervals (L.map (clipGe m)) = presentedIntervals L ∩ Set.Ici m := by
  ext x
  simp only [mem, List.mem_map, Set.mem_inter_iff, Set.mem_Ici, clipGe]
  constructor
  · rintro ⟨p, ⟨q, hq, rfl⟩, hx1, hx2⟩
    exact ⟨⟨q, hq, le_sup_left.trans hx1, hx2⟩, le_sup_right.trans hx1⟩
  · rintro ⟨⟨q, hq, hxq1, hxq2⟩, hxm⟩
    exact ⟨(q.1 ⊔ m, q.2), ⟨q, hq, rfl⟩, sup_le hxq1 hxm, hxq2⟩
```

```
end presentedIntervals

end DomainTheory.NeighbourhoodSystems
```

4.9.18 Universal domain U

The neighbourhood system of Scott's universal domain on the rationals in the unit half-open interval.

Definition (`UMaster`). The master neighbourhood is the half-open unit interval of rationals.

Proof. By the standard half-open interval. ■

Theorem (`UMaster_nonempty`). The master neighbourhood is nonempty.

Proof. Zero lies in the unit half-open interval. ■

Definition (`UMem`). A set is a candidate neighbourhood when it is a nonempty presented union of rational intervals contained in the master.

Proof. By conjunction of presentation, nonemptiness, and subset. ■

Theorem (`UMem.sub_master`). Every candidate neighbourhood lies in the master.

Proof. By the defining subset clause. ■

Theorem (`UMem.master_mem`). The master itself is a candidate neighbourhood.

Proof. By the singleton unit presentation and nonemptiness. ■

Theorem (`UMem.inter_mem`). Given candidate neighbourhoods and a nonempty witness inside their intersection, their intersection is again a candidate.

Proof. By combining interval lists and the intersection presentation lemma. ■

Definition (`U`). The neighbourhood system whose memberships are the candidate sets over the master, with the standard intersection law.

Proof. By the neighbourhood-system constructor using the master and intersection lemmas. ■

Theorem (`U.mem_iff_scott`). Membership in the universal system is equivalent to being a nonempty union of rational subintervals with endpoints in the closed unit interval and strict increase along each piece.

Proof. By clipping and filtering presentations in both directions. ■

Theorem (`U.no_minimal`). Every neighbourhood splits into two smaller disjoint nonempty neighbourhoods whose union is the original.

Proof. By bisecting at the midpoint of a witnessing interval using density and clip lemmas. ■

Theorem (`U.union_mem`). The union of two neighbourhoods-or-empty is again empty or a neighbourhood.

Proof. By case split and appending interval lists. ■

Theorem (`U.iUnion_mem`). A nonempty finite union of neighbourhoods-or-empty is a neighbourhood.

Proof. By finset induction using the binary union lemma. ■

Theorem (`U.diff_mem`). The set difference of two neighbourhoods is empty or again a neighbourhood.

Proof. By the list difference presentation of unions. ■

Theorem (`U.diffClosed`). The universal system is closed under difference.

Proof. By the difference membership lemma. ■

Theorem (`U.isPositive`). The universal system is positive.

Proof. By combining interval lists for nonempty intersections. ■

Theorem (`U.noMinimal`). The universal system has no minimal neighbourhoods.

Proof. By the explicit binary split lemma for each member. ■

`U.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    presentedIntervals
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_19
public import Mathlib.Data.Fintype.Defs
public import Mathlib.Data.Set.Basic
public import Mathlib.Order.Interval.Set.Basic
public import Mathlib.Algebra.Order.Field.Basic
public import Mathlib.Order.CompleteLattice.Finset
import Mathlib.Tactic.NormNum

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

abbrev UMaster : Set ℚ := Set.Ico (0 : ℚ) 1

theorem UMaster_nonempty : UMaster.Nonempty := by
  exact ⟨0, by simp [UMaster]⟩

def UMem (X : Set ℚ) : Prop :=
  (∃ L : List (ℚ × ℚ), X = presentedIntervals L) ∧ X.Nonempty ∧ X ⊆ Set.Ico (0 : ℚ) 1

namespace UMem

theorem sub_master {X : Set ℚ} (hX : UMem X) : X ⊆ UMaster := hX.2.2

theorem master_mem : UMem UMaster :=
  ⟨⟨[(0, 1)], by rw [presentedIntervals.cons, presentedIntervals.nil, Set.union_empty]⟩
    ,
    ⟨0, by norm_num⟩, subset_rfl⟩

theorem inter_mem {X Y Z : Set ℚ} (hX : UMem X) (hY : UMem Y) (hZ : UMem Z)
    (hZsub : Z ⊆ X ∩ Y) : UMem (X ∩ Y) := by
  rcases hX with ⟨⟨L1, rfl⟩, -, hXsub⟩
  rcases hY with ⟨⟨L2, rfl⟩, -, -⟩
  rcases hZ with ⟨-, hZne, -⟩
  exact ⟨⟨combineIntervals L1 L2, presentedIntervals.inter L1 L2⟩,
    hZne.mono hZsub, Set.inter_subset_left.trans hXsub⟩
end UMem

def U : NeighborhoodSystem ℚ where
  mem := UMem
  master := UMaster
  master_nonempty := UMaster_nonempty
  master_mem := UMem.master_mem
  inter_mem := UMem.inter_mem
  sub_master := UMem.sub_master
```

```
namespace U

theorem mem_iff_scott {X : Set ?} :
    U.mem X ↔ ∃ L : List (? × ?), (∀ p ∈ L, (0 : ?) ≤ p.1 ∧ p.1 < p.2 ∧ p.2 ≤ 1) ∧
      X = presentedIntervals L ∧ X.Nonempty := by
  constructor
  · rintro ⟨⟨L, rfl⟩, hne, hsub⟩
    refine ⟨(L.map clip).filter (fun p => decide (p.1 < p.2)), ?_, ?_, ?_⟩
    · rintro p hp
      obtain ⟨hp1, hp2⟩ := List.mem_filter.mp hp
      obtain ⟨q, hq, rfl⟩ := List.mem_map.mp hp1
      exact ⟨le_sup_right, decide_eq_true_eq.mp hp2, inf_le_right⟩
    · rw [presentedIntervals.filter_lt, presentedIntervals.map_clip,
        Set.inter_eq_left.mpr hsub]
    · exact hne
  · rintro ⟨L, hbound, rfl, hne⟩
    refine ⟨⟨L, rfl⟩, hne, ?_⟩
    rintro x hx
    obtain ⟨p, hp, hx1, hx2⟩ := presentedIntervals.mem.mp hx
    obtain ⟨hp0, -, hp1⟩ := hbound p hp
    exact ⟨hp0.trans hx1, hx2.trans_le hp1⟩

theorem no_minimal {X : Set ?} (hX : U.mem X) :
    ∃ Y Z : Set ?, U.mem Y ∧ U.mem Z ∧ Y ∩ Z = ∅ ∧ Y ∪ Z = X ∧ Y = X ∧ Z = X := by
  obtain ⟨⟨L, rfl⟩, hne, hsub⟩ := hX
  obtain ⟨x, hx⟩ := hne
  obtain ⟨p, hp, hx1, hx2⟩ := presentedIntervals.mem.mp hx
  have hlt : p.1 < p.2 := lt_of_le_of_lt hx1 hx2
  set m := (p.1 + p.2) / 2 with hm
  have hpm1 : p.1 < m := left_lt_add_div_two.mpr hlt
  have hpm2 : m < p.2 := add_div_two_lt_right.mpr hlt
  have hYZinter : (presentedIntervals L ∩ Set.Iio m) ∩ (presentedIntervals L ∩ Set.Ici
     m) =
      (∅ : Set ?) := by
    ext y
    simp only [Set.mem_inter_iff, Set.mem_Iio, Set.mem_Ici, Set.mem_empty_iff_false,
    iff_false]
    rintro ⟨⟨-, h1⟩, -, h2⟩
    exact absurd h1 (not_lt.mpr h2)
  have hYZunion : (presentedIntervals L ∩ Set.Iio m) ∪ (presentedIntervals L ∩ Set.Ici
     m) =
      presentedIntervals L := by
    ext y
    simp only [Set.mem_union, Set.mem_inter_iff, Set.mem_Iio, Set.mem_Ici]
    constructor
    · rintro (⟨h, -⟩ | ⟨h, -⟩) <;> exact h
    · intro h
      rcases lt_or_ge y m with hlt' | hge'
      · exact Or.inl ⟨h, hlt'⟩
      · exact Or.inr ⟨h, hge'⟩
  have hYne : (presentedIntervals L ∩ Set.Iio m).Nonempty :=
    ⟨p.1, presentedIntervals.mem.mpr ⟨p, hp, le_refl _, hlt⟩, hpm1⟩
  have hZne : (presentedIntervals L ∩ Set.Ici m).Nonempty :=
    ⟨m, presentedIntervals.mem.mpr ⟨p, hp, hpm1.le, hpm2⟩, le_refl m⟩
  refine ⟨_, _, ⟨⟨_, (presentedIntervals.map_clipLt m L).symm⟩, hYne,
```

```
      Set.inter_subset_left.trans hsub⟩,
    ⟨⟨_, (presentedIntervals.map_clipGe m L).symm⟩, hZne, Set.inter_subset_left.trans
    hsub⟩,
    hYZinter, hYZunion, ?_, ?_⟩
  · intro hEq
    have hZsub : presentedIntervals L ∩ Set.Ici m ⊆ presentedIntervals L ∩ Set.Iio m
    := by
      rw [hEq]; exact Set.inter_subset_left
    exact hZne.ne_empty ((Set.inter_eq_right.mpr hZsub).symm.trans hYZinter)
  · intro hEq
    have hYsub : presentedIntervals L ∩ Set.Iio m ⊆ presentedIntervals L ∩ Set.Ici m
    := by
      rw [hEq]; exact Set.inter_subset_left
    exact hYne.ne_empty ((Set.inter_eq_left.mpr hYsub).symm.trans hYZinter)

theorem union_mem {X Y : Set ?} (hX : X = ∅ ∨ U.mem X) (hY : Y = ∅ ∨ U.mem Y) :
    X ∪ Y = ∅ ∨ U.mem (X ∪ Y) := by
  rcases hX with rfl | hX
  · rcases hY with rfl | hY
    · exact Or.inl (by simp)
    · exact Or.inr (by simpa using hY)
  · rcases hY with rfl | hY
    · exact Or.inr (by simpa using hX)
    · obtain ⟨⟨L1, rfl⟩, hXne, hXsub⟩ := hX
      obtain ⟨⟨L2, rfl⟩, hYne, hYsub⟩ := hY
      refine Or.inr ⟨⟨L1 ++ L2, (presentedIntervals.append L1 L2).symm⟩,
        hXne.mono Set.subset_union_left, Set.union_subset hXsub hYsub⟩

theorem iUnion_mem {ι : Type*} [Fintype ι] {f : ι → Set ?} (hf : ∀ i, f i = ∅ ∨ U.mem (
    f i))
    (hne : (⋃ i, f i).Nonempty) : U.mem (⋃ i, f i) := by
  classical
  have hstep : ∀ s : Finset ι, (⋃ i ∈ s, f i) = ∅ ∨ U.mem (⋃ i ∈ s, f i) := by
    intro s
    induction s using Finset.induction_on with
    | empty => exact Or.inl (by simp)
    | insert i s hi ih =>
      rw [Finset.set_biUnion_insert]
      exact union_mem (hf i) ih
  have hall : (⋃ i, f i) = ⋃ i ∈ (Finset.univ : Finset ι), f i := by simp
  rw [hall] at hne ⊢
  exact (hstep Finset.univ).resolve_left hne.ne_empty

theorem diff_mem {X Y : Set ?} (hX : U.mem X) (hY : U.mem Y) : X \ Y = ∅ ∨ U.mem (X \
    Y) := by
  obtain ⟨⟨L1, rfl⟩, -, hXsub⟩ := hX
  obtain ⟨⟨L2, rfl⟩, -, -⟩ := hY
  rcases Set.eq_empty_or_nonempty (presentedIntervals L1 \ presentedIntervals L2) with
    h | h
  · exact Or.inl h
  · exact Or.inr ⟨⟨diffLists L1 L2, (presentedIntervals.diffLists L1 L2).symm⟩, h,
      Set.sdiff_subset.trans hXsub⟩

theorem diffClosed : U.DiffClosed := fun hX hY => diff_mem hX hY

theorem isPositive : U.IsPositive := by
```

```
  rintro X Y ⟨⟨L1, rfl⟩, -, hXsub⟩ ⟨⟨L2, rfl⟩, -, -⟩
  refine ⟨fun h => h.2.1, fun hne => ⟨⟨combineIntervals L1 L2, presentedIntervals.inter
     L1 L2⟩,
    hne, Set.inter_subset_left.trans hXsub⟩⟩
theorem noMinimal : U.NoMinimal := by
  intro X hX
  obtain ⟨Y, Z, hY, hZ, hYZinter, hYZunion, -, -⟩ := no_minimal hX
  exact ⟨Y, Z, hY, hZ, hY.2.1, hZ.2.1, hYZinter, hYZunion⟩

end U

end DomainTheory.NeighbourhoodSystems
```

4.9.19 Split specification

Splittings of neighbourhoods in the universal system, used to refine atoms along a Y-sequence.

Definition (`SplitSpec`). A split operation satisfies the specification when splittings are compatible with emptiness, lie in the universal system, decompose intersections and differences of atom sets, and form a disjoint cover of the ambient neighbourhood.

Proof. By the stated universal quantification over splits. ■

Theorem (`exists_split`). Under the standard empty–neighbourhood correspondence for an atom set and universal ambient, every stage admits a splitting into two universal pieces with the required intersection and difference behaviour.

Proof. By case analysis on whether the atom intersects the next stage set, using no-minimality when both parts are nonempty. ■

Definition (`genAtom`). The generalized atom sequence intersects the master set, at each step keeping or discarding the current stage set according to a boolean stream.

Proof. By primitive recursion on the step index. ■

`SplitSpec.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.U
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.subApprox

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

section
open NeighborhoodSystem
variable {α : Type*}

def SplitSpec (split : Set α → Set ? → Set α → Set ? × Set ?) : Prop :=
  ∀ {A : Set α} {B : Set ?}, (A = ∅ ↔ B = ∅) → (B = ∅ ∨ U.mem B) → ∀ Xn : Set α,
    ((split A B Xn).1 = ∅ ∨ U.mem (split A B Xn).1) ∧
      ((split A B Xn).2 = ∅ ∨ U.mem (split A B Xn).2) ∧
      (A ∩ Xn = ∅ ↔ (split A B Xn).1 = ∅) ∧
      (A \ Xn = ∅ ↔ (split A B Xn).2 = ∅) ∧
      (split A B Xn).1 ∪ (split A B Xn).2 = B ∧
      (split A B Xn).1 ∩ (split A B Xn).2 = ∅
```

```
theorem exists_split {A : Set α} {B : Set ?} (hAB : A = ∅ ↔ B = ∅)
    (hBU : B = ∅ ∨ U.mem B) (Xn : Set α) :
    ∃ I J : Set ?, (I = ∅ ∨ U.mem I) ∧ (J = ∅ ∨ U.mem J) ∧
      (A ∩ Xn = ∅ ↔ I = ∅) ∧ (A \ Xn = ∅ ↔ J = ∅) ∧ I ∪ J = B ∧ I ∩ J = ∅ := by
  by_cases h1 : A ∩ Xn = ∅
  · refine ⟨∅, B, Or.inl rfl, hBU, by simp [h1], ?_, by simp, by simp⟩
    have hAeq : A \ Xn = A := by
      ext x
      simp only [Set.mem_sdiff]
      refine ⟨fun hx => hx.1, fun hx => ⟨hx, fun hxn => ?_⟩⟩
      exact Set.eq_empty_iff_forall_notMem.mp h1 x ⟨hx, hxn⟩
    rw [hAeq, hAB]
  · by_cases h2 : A \ Xn = ∅
    · refine ⟨B, ∅, hBU, Or.inl rfl, ?_, iff_of_true h2 rfl, by simp, by simp⟩
      have hAeq : A ∩ Xn = A := by
        ext x
        simp only [Set.mem_inter_iff]
        refine ⟨fun hx => hx.1, fun hx => ⟨hx, ?_⟩⟩
        by_contra hxn
        exact Set.eq_empty_iff_forall_notMem.mp h2 x ⟨hx, hxn⟩
      rw [hAeq, hAB]
    · have hAne : A = ∅ := by
        intro hA
        apply h1
        rw [hA]
        exact Set.empty_inter Xn
      have hBne : B = ∅ := fun hB => hAne (hAB.mpr hB)
      have hBU' : U.mem B := hBU.resolve_left hBne
      obtain ⟨Y, Z, hY, hZ, hYZinter, hYZunion, -, -⟩ := U.no_minimal hBU'
      have hYne : Y = ∅ := hY.2.1.ne_empty
      have hZne : Z = ∅ := hZ.2.1.ne_empty
      exact ⟨Y, Z, Or.inr hY, Or.inr hZ, iff_of_false h1 hYne, iff_of_false h2 hZne,
        hYZunion, hYZinter⟩

def genAtom {β : Type*} (Z : ℕ → Set β) (M : Set β) (? : ℕ → Bool) : ℕ → Set β
  | 0 => M
  | (n + 1) => genAtom Z M ? n ∩ (if ? n then Z n else M \ Z n)

end

end DomainTheory.NeighbourhoodSystems
```

4.9.20 Atoms of U

Atoms of the universal neighbourhood refine along a boolean stream using a splitting function.

Definition (`atomU`). At stage zero the atom is the master neighbourhood; at each successor, take the first or second split of the current generalized atom against the next stage set.

Proof. By primitive recursion using the split function. ■

Theorem (`atomU_zero`). Every atom stream starts at the master neighbourhood.

Proof. By definition of the recursion base. ■

Definition (`extendTrue`). Extend a finite boolean prefix to an infinite stream that is constantly true afterward.

Proof. By comparison of the index with the prefix length. ■

Definition (`Yseq`). The Y-sequence at length n is the union over all boolean prefixes of the atom one stage later.

Proof. By union over finite boolean functions. ■

Definition (`restrictFin`). Restrict an infinite boolean stream to its values on indices below a fixed bound.

Proof. By evaluation on finite indices. ■

Definition (`splitChoice`). When splitting is legal, choose a witnessing split pair; otherwise return the empty pair.

Proof. By classical choice on the existential split lemma. ■

Theorem (`genAtom_subset`). Generalized atoms always lie inside the master set at each stage.

Proof. By induction on the stage. ■

Theorem (`genAtom_congr`). Agreement of boolean streams below n yields equal generalized atoms at n.

Proof. By induction and the update step. ■

Theorem (`genAtom_forward`). Membership in a generalized atom at n determines exactly which earlier booleans held for the element relative to the stage sets.

Proof. By induction unpacking the intersection steps. ■

Theorem (`genAtom_self`). An element of the master set lies in the generalized atom for the stream that marks exactly the stage sets containing it.

Proof. By induction on the stage using the forward characterization. ■

Theorem (`split_fst_subset`). Under the split specification, the first split piece lies in the ambient neighbourhood.

Proof. By the disjoint cover identity in the specification. ■

Theorem (`split_snd_subset`). Under the split specification, the second split piece lies in the ambient neighbourhood.

Proof. By the disjoint cover identity in the specification. ■

Theorem (`atomU_succ`). The successor atom is given explicitly by the split of the current generalized atom.

Proof. By definition of the recursion step. ■

Theorem (`atomU_congr`). Agreement of streams below n yields equal atoms at n.

Proof. By induction using generalized-atom congruence and the successor formula. ■

Theorem (`atomU_invariant`). At each stage, emptiness of atoms matches generalized atoms, nonempty atoms lie in the universal system, and disagreeing streams yield disjoint atoms.

Proof. By induction using the split specification and disjointness of split pieces. ■

Theorem (`atomU_succ_subset`). Successor atoms sit inside their predecessors.

Proof. By the split subset lemmas and the invariant. ■

Theorem (`extendTrue_agree`). Below the prefix length, extension agrees with the given finite booleans.

Proof. By definition of extension. ■

Theorem (`extendTrue_restrictFin_agree`). Extension of a restriction recovers the original stream on indices below the bound.

Proof. By unfolding extension and restriction. ■

Theorem (`subset_Yseq`). Each atom contributing to a Y-sequence lies inside that sequence.

Proof. By the union defining the Y-sequence. ■

Theorem (`split_fst_eq_inter_Yseq`). The first split piece at stage n equals the current atom intersected with the Y-sequence at n.

Proof. By subset antisymmetry using splits, extension, and the invariant disjointness. ■

Theorem (`atomU_subset_master`). Every atom lies in the master neighbourhood.

Proof. By induction using successor subsets. ■

Theorem (`atomU_succ_eq`). The successor atom is the intersection of the predecessor with either the Y-sequence or the complement of the Y-sequence in the master, according to the boolean at n.

Proof. By case analysis on the boolean and the split identities. ■

Theorem (`atomU_eq_genAtom`). At every stage the atom equals the generalized atom built from Y-sequences and the master.

Proof. By induction using the successor equation. ■

`atomU.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.SplitSpec

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

section
open NeighborhoodSystem
variable {α : Type*}
variable (split : Set α → Set ? → Set α → Set ? × Set ?)

noncomputable def atomU (split : Set α → Set ? → Set α → Set ? × Set ?)
    (X : ℕ → Set α) (Δ : Set α) (? : ℕ → Bool) : ℕ → Set ?
  | 0 => U.master
  | (n + 1) =>
      if ? n then (split (genAtom X Δ ? n) (atomU split X Δ ? n) (X n)).1
      else (split (genAtom X Δ ? n) (atomU split X Δ ? n) (X n)).2
end

section
open NeighborhoodSystem
variable {α : Type*}
variable (split : Set α → Set ? → Set α → Set ? × Set ?)
variable (X : ℕ → Set α) (Δ : Set α)

@[simp] theorem atomU_zero (? : ℕ → Bool) : atomU split X Δ ? 0 = U.master := rfl
end

section
open NeighborhoodSystem
variable {α : Type*}
variable (split : Set α → Set ? → Set α → Set ? × Set ?)
variable (X : ℕ → Set α) (Δ : Set α)
variable (hΔ : Δ.Nonempty) (hsplit : SplitSpec split)
include hΔ hsplit

def extendTrue {n : ℕ} (?' : Fin n → Bool) : ℕ → Bool :=
  fun i => if h : i < n then ?' ⟨i, h⟩ else true
```

```
noncomputable def Yseq (n : ℕ) : Set ? :=
  ⋃ ?' : Fin n → Bool, atomU split X Δ (extendTrue ?') (n + 1)

def restrictFin (? : ℕ → Bool) (n : ℕ) : Fin n → Bool := fun i => ? i.val

end

section
open NeighborhoodSystem
variable {α : Type*}

open Classical in

noncomputable def splitChoice (A : Set α) (B : Set ?) (Xn : Set α) : Set ? × Set ? :=
  if h : (A = ∅ ↔ B = ∅) ∧ (B = ∅ ∨ U.mem B) then
    ⟨(exists_split h.1 h.2 Xn).choose, (exists_split h.1 h.2 Xn).choose_spec.choose⟩
  else (∅, ∅)

theorem genAtom_subset {β : Type*} (Z : ℕ → Set β) (M : Set β) (? : ℕ → Bool) (n : ℕ
    ) :
    genAtom Z M ? n ⊆ M := by
  induction n with
  | zero => exact subset_rfl
  | succ n ih => exact (Set.inter_subset_left).trans ih

theorem genAtom_congr {β : Type*} (Z : ℕ → Set β) (M : Set β) {? ?' : ℕ → Bool} {n :
    ℕ}
    (h : ∀ i < n, ? i = ?' i) : genAtom Z M ? n = genAtom Z M ?' n := by
  induction n with
  | zero => rfl
  | succ n ih =>
    have hn := h n (Nat.lt_succ_self n)
    have hprev : genAtom Z M ? n = genAtom Z M ?' n := ih (fun i hi => h i (Nat.
    lt_succ_of_lt hi))
    show genAtom Z M ? n ∩ _ = genAtom Z M ?' n ∩ _
    rw [hprev, hn]

theorem genAtom_forward {β : Type*} {Z : ℕ → Set β} {M : Set β} {? : ℕ → Bool} {n :
    ℕ}
    {x : β} (hx : x ∈ genAtom Z M ? n) : ∀ i < n, (? i = true ↔ x ∈ Z i) := by
  induction n with
  | zero => intro i hi; exact absurd hi (Nat.not_lt_zero i)
  | succ n ih =>
    intro i hi
    obtain ⟨hxprev, hxlast⟩ := hx
    rcases Nat.lt_succ_iff_lt_or_eq.mp hi with hi' | rfl
    · exact ih hxprev i hi'
    · by_cases h? : ? i = true
      · simp only [h?, ite_true] at hxlast
        exact ⟨fun _ => hxlast, fun _ => h?⟩
      · rw [Bool.not_eq_true] at h?
        simp only [h?] at hxlast
        exact ⟨fun h => absurd h (by simp [h?]), fun hmem => absurd hmem hxlast.2⟩

open Classical in
```

```
theorem genAtom_self {β : Type*} (Z : ℕ → Set β) (M : Set β) {x : β} (hx : x ∈ M) :
    ∀ n, x ∈ genAtom Z M (fun k => decide (x ∈ Z k)) n := by
  intro n
  induction n with
  | zero => exact hx
  | succ n ih =>
    refine ⟨ih, ?_⟩
    by_cases h : x ∈ Z n
    · simp [h]
    · simp [h, hx]

theorem split_fst_subset {split : Set α → Set ? → Set α → Set ? × Set ?} (hsplit :
    SplitSpec split)
    {A : Set α} {B : Set ?} (hAB : A = ∅ ↔ B = ∅)
    (hBU : B = ∅ ∨ U.mem B) (Xn : Set α) : (split A B Xn).1 ⊆ B :=
  Set.subset_union_left.trans_eq (hsplit hAB hBU Xn).2.2.2.2.1

theorem split_snd_subset {split : Set α → Set ? → Set α → Set ? × Set ?} (hsplit :
    SplitSpec split)
    {A : Set α} {B : Set ?} (hAB : A = ∅ ↔ B = ∅)
    (hBU : B = ∅ ∨ U.mem B) (Xn : Set α) : (split A B Xn).2 ⊆ B :=
  Set.subset_union_right.trans_eq (hsplit hAB hBU Xn).2.2.2.2.1

variable (split : Set α → Set ? → Set α → Set ? × Set ?)

variable (X : ℕ → Set α) (Δ : Set α)

theorem atomU_succ (? : ℕ → Bool) (n : ℕ) :
    atomU split X Δ ? (n + 1) =
      if ? n then (split (genAtom X Δ ? n) (atomU split X Δ ? n) (X n)).1
      else (split (genAtom X Δ ? n) (atomU split X Δ ? n) (X n)).2 := rfl

theorem atomU_congr {? ?' : ℕ → Bool} {n : ℕ} (h : ∀ i < n, ? i = ?' i) :
    atomU split X Δ ? n = atomU split X Δ ?' n := by
  induction n with
  | zero => rfl
  | succ n ih =>
    have hprev : atomU split X Δ ? n = atomU split X Δ ?' n :=
      ih (fun i hi => h i (Nat.lt_succ_of_lt hi))
    have hA : genAtom X Δ ? n = genAtom X Δ ?' n :=
      genAtom_congr X Δ (fun i hi => h i (Nat.lt_succ_of_lt hi))
    have hn := h n (Nat.lt_succ_self n)
    rw [atomU_succ, atomU_succ, hA, hprev, hn]

variable (hΔ : Δ.Nonempty) (hsplit : SplitSpec split)
include hΔ hsplit

theorem atomU_invariant :
    ∀ n, (∀ ?, genAtom X Δ ? n = ∅ ↔ atomU split X Δ ? n = ∅) ∧
      (∀ ?, atomU split X Δ ? n = ∅ ∨ U.mem (atomU split X Δ ? n)) ∧
      (∀ ? ?', (∃ i < n, ? i = ?' i) →
        atomU split X Δ ? n ∩ atomU split X Δ ?' n = ∅) := by
  have hUmasterNe : (U.master : Set ?) = ∅ := Set.Nonempty.ne_empty ⟨0, by norm_num [U]
    ⟩
  intro n
```

```
induction n with
| zero =>
  refine ⟨fun ? => ?_, fun ? => Or.inr U.master_mem, fun ? ?' ⟨i, hi, _⟩ => absurd hi
   (Nat.not_lt_zero i)⟩
  show Δ = ∅ ↔ (U.master : Set ?) = ∅
  exact ⟨fun h => absurd h hΔ.ne_empty, fun h => absurd h hUmasterNe⟩
| succ n ih =>
  obtain ⟨ihmatch, ihmem, ihdisj⟩ := ih
  refine ⟨fun ? => ?_, fun ? => ?_, fun ? ?' ⟨i, hi, hne⟩ => ?_⟩
  · have hspec := hsplit (ihmatch ?) (ihmem ?) (X n)
    rw [atomU_succ]
    by_cases h? : ? n = true
    · rw [show genAtom X Δ ? (n + 1) = genAtom X Δ ? n ∩ X n from by simp [genAtom,
  h?]]
      simp only [h?, ite_true]
      exact hspec.2.2.1
    · rw [Bool.not_eq_true] at h?
      have hsub := genAtom_subset X Δ ? n
      rw [show genAtom X Δ ? (n + 1) = genAtom X Δ ? n \ X n from by
        have heq : genAtom X Δ ? (n + 1) = genAtom X Δ ? n ∩ (Δ \ X n) := by simp [
  genAtom, h?]
        rw [heq]
        ext x; constructor
        · rintro ⟨hx1, -, hx2⟩; exact ⟨hx1, hx2⟩
        · rintro ⟨hx1, hx2⟩; exact ⟨hx1, hsub hx1, hx2⟩]
      simp only [h?]
      exact hspec.2.2.2.1
  · have hspec := hsplit (ihmatch ?) (ihmem ?) (X n)
    rw [atomU_succ]
    by_cases h? : ? n = true
    · simp only [h?, ite_true]; exact hspec.1
    · simp only [h?]; exact hspec.2.1
  · by_cases hagree : ∀ j < n, ? j = ?' j
    · have h?n : ? n = ?' n := by
        intro heq
        exact hne (by
          rcases Nat.lt_succ_iff_lt_or_eq.mp hi with hi' | rfl
          · exact hagree i hi'
          · exact heq)
      have hA : genAtom X Δ ? n = genAtom X Δ ?' n := genAtom_congr X Δ hagree
      have hB : atomU split X Δ ? n = atomU split X Δ ?' n := atomU_congr split X Δ
   hagree
      have hspec := hsplit (ihmatch ?) (ihmem ?) (X n)
      rw [atomU_succ, atomU_succ, hA, hB]
      have hIJ := hspec.2.2.2.2.2
      rcases Bool.eq_false_or_eq_true (? n) with h1 | h1 <;>
        rcases Bool.eq_false_or_eq_true (?' n) with h2 | h2 <;>
        simp_all [Set.inter_comm]
    · push Not at hagree
      obtain ⟨j, hj, hjne⟩ := hagree
      have hd : atomU split X Δ ? n ∩ atomU split X Δ ?' n = ∅ := ihdisj ? ?' ⟨j, hj
  , hjne⟩
      have h1 : atomU split X Δ ? (n + 1) ⊆ atomU split X Δ ? n := by
        rw [atomU_succ]
        by_cases h? : ? n = true
        · simp only [h?, ite_true]; exact split_fst_subset hsplit (ihmatch ?) (ihmem
```

```
    ?) (X n)
          · simp only [h?]; exact split_snd_subset hsplit (ihmatch ?) (ihmem ?) (X n)
        have h2 : atomU split X Δ ?' (n + 1) ⊆ atomU split X Δ ?' n := by
          rw [atomU_succ]
          by_cases h?' : ?' n = true
          · simp only [h?', ite_true]; exact split_fst_subset hsplit (ihmatch ?') (
    ihmem ?') (X n)
          · simp only [h?']; exact split_snd_subset hsplit (ihmatch ?') (ihmem ?') (X n
    )
        exact Set.subset_eq_empty (Set.inter_subset_inter h1 h2) hd

theorem atomU_succ_subset (? : ℕ → Bool) (n : ℕ) :
    atomU split X Δ ? (n + 1) ⊆ atomU split X Δ ? n := by
  obtain ⟨hmatch, hmem, -⟩ := atomU_invariant split X Δ hΔ hsplit n
  rw [atomU_succ]
  by_cases h? : ? n = true
  · simp only [h?, ite_true]; exact split_fst_subset hsplit (hmatch ?) (hmem ?) (X n)
  · simp only [h?]; exact split_snd_subset hsplit (hmatch ?) (hmem ?) (X n)

omit hΔ hsplit in
theorem extendTrue_agree {n : ℕ} (?' : Fin n → Bool) (i : ℕ) (hi : i < n) :
    extendTrue ?' i = ?' ⟨i, hi⟩ := by simp [extendTrue, hi]

omit hΔ hsplit in
theorem extendTrue_restrictFin_agree (? : ℕ → Bool) (n i : ℕ) (hi : i < n) :
    extendTrue (restrictFin ? n) i = ? i := by simp [extendTrue, restrictFin, hi]

omit hΔ hsplit in
theorem subset_Yseq {n : ℕ} (?' : Fin n → Bool) :
    atomU split X Δ (extendTrue ?') (n + 1) ⊆ Yseq split X Δ n :=
  Set.subset_iUnion (fun ?' => atomU split X Δ (extendTrue ?') (n + 1)) ?'

theorem split_fst_eq_inter_Yseq (? : ℕ → Bool) (n : ℕ) :
    (split (genAtom X Δ ? n) (atomU split X Δ ? n) (X n)).1 =
      atomU split X Δ ? n ∩ Yseq split X Δ n := by
  obtain ⟨hmatch, hmem, hdisj⟩ := atomU_invariant split X Δ hΔ hsplit n
  set A := genAtom X Δ ? n with hAdef
  set B := atomU split X Δ ? n with hBdef
  set I := (split A B (X n)).1 with hIdef
  set ?2 := Function.update ? n true with h?2def
  set ?3 := extendTrue (restrictFin ?2 n) with h?3def
  have hagreefull : ∀ i < n + 1, ?3 i = ?2 i := by
    intro i hi
    rcases Nat.lt_succ_iff_lt_or_eq.mp hi with hi' | rfl
    · exact extendTrue_restrictFin_agree ?2 n i hi'
    · simp [h?3def, extendTrue, h?2def]
  have hI_eq : atomU split X Δ ?2 (n + 1) = I := by
    rw [atomU_succ]
    have hd2n : ?2 n = true := by simp [h?2def]
    have hA2 : genAtom X Δ ?2 n = A := by
      apply genAtom_congr; intro i hi; simp [h?2def, Function.update_of_ne (ne_of_lt
    hi)]
    have hB2 : atomU split X Δ ?2 n = B := by
      apply atomU_congr; intro i hi; simp [h?2def, Function.update_of_ne (ne_of_lt hi)
    ]
    rw [hd2n, ite_eq_left rfl, hA2, hB2]
```

```
  have h?3_eq : atomU split X Δ ?3 (n + 1) = I := (atomU_congr split X Δ hagreefull).
    trans hI_eq
  apply Set.Subset.antisymm
  · have hIsubB : I ⊆ B := split_fst_subset hsplit (hmatch ?) (hmem ?) (X n)
    have hIsubY : I ⊆ Yseq split X Δ n := h?3_eq ▸ subset_Yseq split X Δ (restrictFin
     ?2 n)
    exact Set.subset_inter hIsubB hIsubY
  · rintro z ⟨hzB, hzY⟩
    obtain ⟨?', hz'⟩ := Set.mem_iUnion.mp hzY
    by_cases hagree : ∀ i < n, extendTrue ?' i = ? i
    · have hAeq : genAtom X Δ (extendTrue ?') n = A := genAtom_congr X Δ hagree
      have hBeq : atomU split X Δ (extendTrue ?') n = B := atomU_congr split X Δ
    hagree
      have hlast : extendTrue ?' n = true := by simp [extendTrue]
      have heq : atomU split X Δ (extendTrue ?') (n + 1) = I := by
        rw [atomU_succ, hAeq, hBeq, hlast, ite_eq_left rfl]
      rwa [heq] at hz'
    · push Not at hagree
      obtain ⟨j, hj, hjne⟩ := hagree
      have hzB' : z ∈ atomU split X Δ (extendTrue ?') n :=
        atomU_succ_subset split X Δ hΔ hsplit (extendTrue ?') n hz'
      have hempty := hdisj ? (extendTrue ?') ⟨j, hj, fun h => hjne (h.symm)⟩
      exact absurd (Set.mem_inter hzB hzB') (by rw [hempty]; simp)

theorem atomU_subset_master (? : ℕ → Bool) (n : ℕ) : atomU split X Δ ? n ⊆ U.master
    := by
  induction n with
  | zero => exact subset_rfl
  | succ n ih => exact (atomU_succ_subset split X Δ hΔ hsplit ? n).trans ih

theorem atomU_succ_eq (? : ℕ → Bool) (n : ℕ) :
    atomU split X Δ ? (n + 1) =
      atomU split X Δ ? n ∩ (if ? n then Yseq split X Δ n else U.master \ Yseq split
    X Δ n) := by
  obtain ⟨hmatch, hmem, -⟩ := atomU_invariant split X Δ hΔ hsplit n
  have hspec := hsplit (hmatch ?) (hmem ?) (X n)
  have hIeq := split_fst_eq_inter_Yseq split X Δ hΔ hsplit ? n
  by_cases h? : ? n = true
  · rw [atomU_succ, ite_eq_left h?, ite_eq_left h?]; exact hIeq
  · rw [atomU_succ, ite_eq_right h?, ite_eq_right h?]
    have hJeq : (split (genAtom X Δ ? n) (atomU split X Δ ? n) (X n)).2 =
        atomU split X Δ ? n \ Yseq split X Δ n := by
      have hunion := hspec.2.2.2.2.1
      have hinter := hspec.2.2.2.2.2
      ext x
      constructor
      · intro hxJ
        have hxB : x ∈ atomU split X Δ ? n := hunion ▸ Or.inr hxJ
        refine ⟨hxB, fun hxY => ?_⟩
        have hxI : x ∈ (split (genAtom X Δ ? n) (atomU split X Δ ? n) (X n)).1 :=
          hIeq ▸ Set.mem_inter hxB hxY
        exact absurd (Set.mem_inter hxI hxJ) (by rw [hinter]; simp)
      · rintro ⟨hxB, hxnY⟩
        rw [← hunion] at hxB
        rcases hxB with hxI | hxJ
        · exact absurd (hIeq ▸ hxI : x ∈ atomU split X Δ ? n ∩ Yseq split X Δ n).2
```

```
    hxnY
        · exact hxJ
    rw [hJeq]
    have hsub := atomU_subset_master split X Δ hΔ hsplit ? n
    ext x
    constructor
    · rintro ⟨hx1, hx2⟩; exact ⟨hx1, hsub hx1, hx2⟩
    · rintro ⟨hx1, -, hx2⟩; exact ⟨hx1, hx2⟩

theorem atomU_eq_genAtom (? : ℕ → Bool) :
    ∀ n, atomU split X Δ ? n = genAtom (Yseq split X Δ) U.master ? n
  | 0 => rfl
  | (n + 1) => by
      rw [atomU_succ_eq split X Δ hΔ hsplit, atomU_eq_genAtom ? n]; rfl

end

end DomainTheory.NeighbourhoodSystems
```

4.9.21 Y-sequence

Y-sequences mediate between stage-wise intersections on a substrate domain and corresponding inclusion structure among universal neighbourhoods.

Theorem (`transfer_dir`). Nonemptiness of generalized atoms with a given finite boolean signature transfers across paired systems when empty stages coincide stepwise.

Proof. By forward membership in generalized atoms and the emptiness witness at the target stage. ■

Theorem (`transfer_empty_iff`). For boolean codes below n, the substrate pattern of membership is nonempty if and only if the corresponding master–Y-sequence pattern is nonempty.

Proof. By applying transfer in both directions through the atom–genAtom correspondence. ■

Theorem (`transfer_subset_iff`). Substrate inclusion between two stages corresponds to master inclusion between the associated Y-sequences.

Proof. By instantiating the empty-set transfer on a two-code witness for subset and difference. ■

Theorem (`transfer_inter_empty_iff`). Two substrate stage-sets meet vacuously if and only if the corresponding Y-sequences are disjoint on the master.

Proof. By the same transfer pattern with two positive codes. ■

Theorem (`transfer_double_subset_iff`). Triple substrate inclusion among two stages and a third corresponds to the analogous triple inclusion among Y-sequences.

Proof. By instantiating transfer on three boolean codes. ■

Theorem (`Yseq_subset_master`). Every Y-sequence lies in the master neighbourhood.

Proof. By union of atoms each contained in the master. ■

Theorem (`Yseq_zero_eq_master`). When the zeroth stage set is the whole substrate, the initial Y-sequence is the master.

Proof. By splitting the master at stage one and Y-sequence inclusion. ■

Theorem (`transfer_inter_eq_iff`). Equality of substrate intersections of three stages corresponds to equality of the three Y-sequence intersections, under stage subsets of the substrate.

Proof. By subset and double-subset transfer lemmas applied to inclusion equivalences. ■

Theorem (`Yseq_empty_or_mem`). Each Y-sequence is either empty or a neighbourhood of the universal system.

Proof. By finite union of universal atoms when nonempty. ■

Theorem (`Yseq_nonempty_of_mem`). Membership at stage n inside the substrate yields a nonempty Y-sequence at n.

Proof. By the decision stream for the witness and nonempty first split piece. ■

Theorem (`splitChoice_isSplitSpec`). The canonical choice split satisfies the split specification.

Proof. By unfolding choice and applying the existential splitting theorem. ■

`Yseq.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.atomU

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

section
open NeighborhoodSystem
variable {α : Type*}
variable (split : Set α → Set ? → Set α → Set ? × Set ?)
variable (X : ℕ → Set α) (Δ : Set α)
variable (hΔ : Δ.Nonempty) (hsplit : SplitSpec split)
include hΔ hsplit

omit hΔ hsplit in
theorem transfer_dir {β1 β2 : Type*} (Z1 : ℕ → Set β1) (M1 : Set β1) (Z2 : ℕ → Set β
    2)
    (M2 : Set β2) (hcore : ∀ ? n, genAtom Z1 M1 ? n = ∅ ↔ genAtom Z2 M2 ? n = ∅)
    {cs : List (ℕ × Bool)} {n : ℕ} (hn : ∀ p ∈ cs, p.1 < n)
    (hne : {x ∈ M1 | ∀ p ∈ cs, (p.2 = true ↔ x ∈ Z1 p.1)}.Nonempty) :
    {y ∈ M2 | ∀ p ∈ cs, (p.2 = true ↔ y ∈ Z2 p.1)}.Nonempty := by
  classical
  obtain ⟨x, hxM, hxcs⟩ := hne
  set ?0 : ℕ → Bool := fun k => decide (x ∈ Z1 k) with h?0def
  have hX1 : x ∈ genAtom Z1 M1 ?0 n := genAtom_self Z1 M1 hxM n
  have hne1 : (genAtom Z1 M1 ?0 n).Nonempty := ⟨x, hX1⟩
  have hne2 : (genAtom Z2 M2 ?0 n).Nonempty := by
    rw [Set.nonempty_iff_ne_empty]
    exact fun h2 => hne1.ne_empty ((hcore ?0 n).mpr h2)
  obtain ⟨y, hy⟩ := hne2
  refine ⟨y, genAtom_subset Z2 M2 ?0 n hy, fun p hp => ?_⟩
  have hforward := genAtom_forward hy p.1 (hn p hp)
  have hbeq : ?0 p.1 = p.2 := by
    show decide (x ∈ Z1 p.1) = p.2
    rw [Bool.eq_iff_iff, decide_eq_true_iff]
    exact (hxcs p hp).symm
  rwa [hbeq] at hforward

theorem transfer_empty_iff {cs : List (ℕ × Bool)} {n : ℕ} (hn : ∀ p ∈ cs, p.1 < n) :
    {x ∈ Δ | ∀ p ∈ cs, (p.2 = true ↔ x ∈ X p.1)}.Nonempty ↔
      {y ∈ U.master | ∀ p ∈ cs, (p.2 = true ↔ y ∈ Yseq split X Δ p.1)}.Nonempty := by
  have hcore : ∀ ? n, genAtom X Δ ? n = ∅ ↔ genAtom (Yseq split X Δ) U.master ? n = ∅
    :=
```

```
    fun ? n => (atomU_invariant split X Δ hΔ hsplit n).1 ?
      |>.trans (by rw [atomU_eq_genAtom split X Δ hΔ hsplit])
  have hcore' : ∀ ? n, genAtom (Yseq split X Δ) U.master ? n = ∅ ↔ genAtom X Δ ? n =
    ∅ :=
    fun ? n => (hcore ? n).symm
  exact ⟨transfer_dir X Δ (Yseq split X Δ) U.master hcore hn,
    transfer_dir (Yseq split X Δ) U.master X Δ hcore' hn⟩

theorem transfer_subset_iff (i j : ℕ) :
    Δ ∩ X i ⊆ X j ↔ U.master ∩ Yseq split X Δ i ⊆ Yseq split X Δ j := by
  have key := transfer_empty_iff split X Δ hΔ hsplit (cs := [(i, true), (j, false)])
    (n := max i j + 1)
    (by simp only [List.mem_cons, List.not_mem_nil, or_false]
        rintro p (rfl | rfl) <;> exact Nat.lt_succ_of_le (by simp))
  have hLHS : {x ∈ Δ | ∀ p ∈ [(i, true), (j, false)], (p.2 = true ↔ x ∈ X p.1)}
      = (Δ ∩ X i) \ X j := by
    ext x
    simp only [Set.mem_ofPred_eq, List.mem_cons, List.not_mem_nil, or_false,
      forall_eq_or_imp, forall_eq, Set.mem_sdiff, Set.mem_inter_iff]
    tauto
  have hRHS : {y ∈ U.master |
      ∀ p ∈ [(i, true), (j, false)], (p.2 = true ↔ y ∈ Yseq split X Δ p.1)}
      = (U.master ∩ Yseq split X Δ i) \ Yseq split X Δ j := by
    ext y
    simp only [Set.mem_ofPred_eq, List.mem_cons, List.not_mem_nil, or_false,
      forall_eq_or_imp, forall_eq, Set.mem_sdiff, Set.mem_inter_iff]
    tauto
  rw [hLHS, hRHS] at key
  rw [← Set.sdiff_eq_empty, ← Set.sdiff_eq_empty, ← Set.not_nonempty_iff_eq_empty,
    ← Set.not_nonempty_iff_eq_empty, not_iff_not]
  exact key

theorem transfer_inter_empty_iff (i j : ℕ) :
    Δ ∩ X i ∩ X j = ∅ ↔ U.master ∩ Yseq split X Δ i ∩ Yseq split X Δ j = ∅ := by
  have key := transfer_empty_iff split X Δ hΔ hsplit (cs := [(i, true), (j, true)])
    (n := max i j + 1)
    (by simp only [List.mem_cons, List.not_mem_nil, or_false]
        rintro p (rfl | rfl) <;> exact Nat.lt_succ_of_le (by simp))
  have hLHS : {x ∈ Δ | ∀ p ∈ [(i, true), (j, true)], (p.2 = true ↔ x ∈ X p.1)}
      = Δ ∩ X i ∩ X j := by
    ext x
    simp only [Set.mem_ofPred_eq, List.mem_cons, List.not_mem_nil, or_false,
      forall_eq_or_imp, forall_eq, Set.mem_inter_iff]
    tauto
  have hRHS : {y ∈ U.master | ∀ p ∈ [(i, true), (j, true)], (p.2 = true ↔ y ∈ Yseq
    split X Δ p.1)}
      = U.master ∩ Yseq split X Δ i ∩ Yseq split X Δ j := by
    ext y
    simp only [Set.mem_ofPred_eq, List.mem_cons, List.not_mem_nil, or_false,
      forall_eq_or_imp, forall_eq, Set.mem_inter_iff]
    tauto
  rw [hLHS, hRHS] at key
  rw [← Set.not_nonempty_iff_eq_empty, ← Set.not_nonempty_iff_eq_empty, not_iff_not]
  exact key

theorem transfer_double_subset_iff (i j k : ℕ) :
```

```
    Δ ∩ X i ∩ X j ⊆ X k ↔ U.master ∩ Yseq split X Δ i ∩ Yseq split X Δ j ⊆ Yseq
    split X Δ k := by
  have key := transfer_empty_iff split X Δ hΔ hsplit (cs := [(i, true), (j, true), (k,
     false)])
    (n := max i (max j k) + 1)
    (by simp only [List.mem_cons, List.not_mem_nil, or_false]
        rintro p (rfl | rfl | rfl) <;> exact Nat.lt_succ_of_le (by simp))
  have hLHS : {x ∈ Δ | ∀ p ∈ [(i, true), (j, true), (k, false)], (p.2 = true ↔ x ∈ X
    p.1)}
      = (Δ ∩ X i ∩ X j) \ X k := by
    ext x
    simp only [Set.mem_ofPred_eq, List.mem_cons, List.not_mem_nil, or_false,
      forall_eq_or_imp, forall_eq, Set.mem_sdiff, Set.mem_inter_iff]
    tauto
  have hRHS : {y ∈ U.master |
      ∀ p ∈ [(i, true), (j, true), (k, false)], (p.2 = true ↔ y ∈ Yseq split X Δ p.1)
    }
      = (U.master ∩ Yseq split X Δ i ∩ Yseq split X Δ j) \ Yseq split X Δ k := by
    ext y
    simp only [Set.mem_ofPred_eq, List.mem_cons, List.not_mem_nil, or_false,
      forall_eq_or_imp, forall_eq, Set.mem_sdiff, Set.mem_inter_iff]
    tauto
  rw [hLHS, hRHS] at key
  rw [← Set.sdiff_eq_empty, ← Set.sdiff_eq_empty, ← Set.not_nonempty_iff_eq_empty,
    ← Set.not_nonempty_iff_eq_empty, not_iff_not]
  exact key

theorem Yseq_subset_master (n : ℕ) : Yseq split X Δ n ⊆ U.master :=
  Set.iUnion_subset fun ?' => atomU_subset_master split X Δ hΔ hsplit (extendTrue ?')
    (n + 1)

theorem Yseq_zero_eq_master (h0 : X 0 = Δ) : Yseq split X Δ 0 = U.master := by
  set ? : ℕ → Bool := extendTrue (fun i : Fin 0 => i.elim0) with h?def
  have h?0 : ? 0 = true := by simp [h?def, extendTrue]
  have hAB : genAtom X Δ ? 0 = ∅ ↔ atomU split X Δ ? 0 = ∅ :=
    ⟨fun h => absurd h hΔ.ne_empty, fun h => absurd h U.master_mem.2.1.ne_empty⟩
  have hBU : atomU split X Δ ? 0 = ∅ ∨ U.mem (atomU split X Δ ? 0) := Or.inr U.
    master_mem
  have hspec := hsplit hAB hBU (X 0)
  have hJ : (split (genAtom X Δ ? 0) (atomU split X Δ ? 0) (X 0)).2 = ∅ := by
    rw [← hspec.2.2.2.1]
    show genAtom X Δ ? 0 \ X 0 = ∅
    rw [show genAtom X Δ ? 0 = Δ from rfl, h0]
    exact Set.sdiff_self
  have hI : (split (genAtom X Δ ? 0) (atomU split X Δ ? 0) (X 0)).1 = U.master := by
    have hunion := hspec.2.2.2.2.1
    rw [hJ, Set.union_empty] at hunion
    exact hunion
  have hkey : atomU split X Δ ? 1 = U.master := by rw [atomU_succ, h?0, ite_eq_left
    rfl]; exact hI
  apply Set.Subset.antisymm (Yseq_subset_master split X Δ hΔ hsplit 0)
  calc U.master = atomU split X Δ ? 1 := hkey.symm
    _ ⊆ Yseq split X Δ 0 := subset_Yseq split X Δ (fun i : Fin 0 => i.elim0)

theorem transfer_inter_eq_iff (i j k : ℕ) (hi : X i ⊆ Δ) (_hj : X j ⊆ Δ) (hk : X k ⊆
    Δ) :
```

```
    X i ∩ X j = X k ↔ Yseq split X Δ i ∩ Yseq split X Δ j = Yseq split X Δ k := by
  have h1 : X k ⊆ X i ↔ Yseq split X Δ k ⊆ Yseq split X Δ i := by
    have := transfer_subset_iff split X Δ hΔ hsplit k i
    rwa [Set.inter_eq_self_of_subset_right hk,
      Set.inter_eq_self_of_subset_right (Yseq_subset_master split X Δ hΔ hsplit k)] at
     this
  have h2 : X k ⊆ X j ↔ Yseq split X Δ k ⊆ Yseq split X Δ j := by
    have := transfer_subset_iff split X Δ hΔ hsplit k j
    rwa [Set.inter_eq_self_of_subset_right hk,
      Set.inter_eq_self_of_subset_right (Yseq_subset_master split X Δ hΔ hsplit k)] at
     this
  have h3 : X i ∩ X j ⊆ X k ↔ Yseq split X Δ i ∩ Yseq split X Δ j ⊆ Yseq split X Δ k
     := by
    have := transfer_double_subset_iff split X Δ hΔ hsplit i j k
    rwa [Set.inter_eq_self_of_subset_right hi,
      Set.inter_eq_self_of_subset_right (Yseq_subset_master split X Δ hΔ hsplit i)] at
     this
  constructor
  · intro heq
    have hki : X k ⊆ X i := heq ▸ Set.inter_subset_left
    have hkj : X k ⊆ X j := heq ▸ Set.inter_subset_right
    have hijk : X i ∩ X j ⊆ X k := heq ▸ subset_rfl
    exact Set.Subset.antisymm (h3.mp hijk) (Set.subset_inter (h1.mp hki) (h2.mp hkj))
  · intro heq
    have hki : Yseq split X Δ k ⊆ Yseq split X Δ i := heq ▸ Set.inter_subset_left
    have hkj : Yseq split X Δ k ⊆ Yseq split X Δ j := heq ▸ Set.inter_subset_right
    have hijk : Yseq split X Δ i ∩ Yseq split X Δ j ⊆ Yseq split X Δ k := heq ▸
    subset_rfl
    exact Set.Subset.antisymm (h3.mpr hijk) (Set.subset_inter (h1.mpr hki) (h2.mpr hkj
    ))

theorem Yseq_empty_or_mem (n : ℕ) : Yseq split X Δ n = ∅ ∨ U.mem (Yseq split X Δ n)
    := by
  by_cases hne : (Yseq split X Δ n).Nonempty
  · exact Or.inr (U.iUnion_mem
      (fun ?' => (atomU_invariant split X Δ hΔ hsplit (n + 1)).2.1 (extendTrue ?'))
    hne)
  · exact Or.inl (Set.not_nonempty_iff_eq_empty.mp hne)

theorem Yseq_nonempty_of_mem {n : ℕ} {x : α} (hxΔ : x ∈ Δ) (hxn : x ∈ X n) :
    (Yseq split X Δ n).Nonempty := by
  classical
  set ?0 : ℕ → Bool := fun k => decide (x ∈ X k) with h?0def
  have h?0n : ?0 n = true := by show decide (x ∈ X n) = true; rw [decide_eq_true_iff];
     exact hxn
  have hxA : x ∈ genAtom X Δ ?0 n := genAtom_self X Δ hxΔ n
  have hAX : x ∈ genAtom X Δ ?0 n ∩ X n := ⟨hxA, hxn⟩
  have hAne : (genAtom X Δ ?0 n ∩ X n).Nonempty := ⟨x, hAX⟩
  obtain ⟨hmatch, hmem, -⟩ := atomU_invariant split X Δ hΔ hsplit n
  have hspec := hsplit (hmatch ?0) (hmem ?0) (X n)
  have hIne : (split (genAtom X Δ ?0 n) (atomU split X Δ ?0 n) (X n)).1 = ∅ :=
    fun h => hAne.ne_empty (hspec.2.2.1.mpr h)
  have hI_eq_atomU : (split (genAtom X Δ ?0 n) (atomU split X Δ ?0 n) (X n)).1 =
      atomU split X Δ ?0 (n + 1) := by rw [atomU_succ, h?0n, ite_eq_left rfl]
  set ?2 : ℕ → Bool := extendTrue (restrictFin ?0 n) with h?2def
  have hagree : ∀ i < n + 1, ?2 i = ?0 i := by
```

```
    intro i hi
    rcases Nat.lt_succ_iff_lt_or_eq.mp hi with hi' | rfl
    · exact extendTrue_restrictFin_agree ?0 n i hi'
    · simp [h?2def, extendTrue, h?0n]
  have h?2_eq : atomU split X Δ ?2 (n + 1) = atomU split X Δ ?0 (n + 1) :=
    atomU_congr split X Δ hagree
  have hne2 : (atomU split X Δ ?2 (n + 1)).Nonempty := by
    rw [h?2_eq, ← hI_eq_atomU]
    exact Set.nonempty_iff_ne_empty.mpr hIne
  exact hne2.mono (subset_Yseq split X Δ (restrictFin ?0 n))

end

section
open NeighborhoodSystem
variable {α : Type*}

open Classical in

open Classical in

theorem splitChoice_isSplitSpec : SplitSpec (α := α) splitChoice := by
  intro A B hAB hBU Xn
  classical
  unfold splitChoice
  rw [dite_eq_left ⟨hAB, hBU⟩]
  exact (exists_split hAB hBU Xn).choose_spec.choose_spec

end

end DomainTheory.NeighbourhoodSystems
```

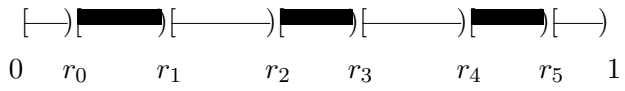
4.9.22 Definition 8.7 — interval presentation of U

DEFINITION 8.7. Let $\mathbb{Q}$ be the set of rational numbers, and let

$$[0,1) = \{q \in \mathbb{Q} \mid 0 \le q < 1\},$$

and similarly for $[r,s)$ for any $r < s$ in $\mathbb{Q}$. The neighbourhood system $\mathcal{U}$ over $[0,1)$ is the set of all non-empty finite unions of rational intervals $[r,s)$ with $0 \le r < s \le 1$. □

A picture of a typical element of $\mathcal{U}$ could be drawn like this:



Note that any union can be taken as a *disjoint* union of the form

$$\bigcup_{i \le n} [r_{2i}, r_{2i+1})$$

where $0 \le r_0 < r_1 < r_2 < \cdots < r_{2n} < r_{2n+1} \le 1$. (Hint: Any overlapping intervals or abutting intervals can always be combined into one long interval.) It is a most elementary exercise to show that, by virtue

of this representation, the system $\mathcal{U}$ has a computable presentation. (Some isomorphic versions of $\mathcal{U}$ — equally effective — are recorded in the exercises.) Note that $\mathcal{U}$ has no minimal neighbourhoods: every set in $\mathcal{U}$ can be written as the union of two disjoint sets in $\mathcal{U}$. (Hint: Use the density of the ordering of $\mathbb{Q}$.) The significance of $\mathcal{U}$ can now be explained.

The development uses presented intervals, the universal domain $\mathcal{U}$, split specification, atoms of $\mathcal{U}$, and Y-sequences.

Theorem (`definition_87`). The master neighbourhood is the half-open unit interval; membership is exactly nonempty unions of bounded rational subintervals presented by interval lists; the system has no minimal neighbourhoods; some canonical split satisfies the split specification; every atom at stage zero is the master neighbourhood; and each Y-sequence is either empty or a neighbourhood of $\mathcal{U}$.

Proof. The master identity is definition. The membership equivalence is the Scott half-open characterization in terms of clipped interval lists. No minimality is the no-minimal-neighbourhoods theorem. Split specification for the canonical split is proved from existential splitting. The initial atom is the master neighbourhood by definition. Nonempty Y-sequences are finite unions of atoms in $\mathcal{U}$. ■

`Definition_8_7.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    presentedIntervals
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.U
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.SplitSpec
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.atomU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.Yseq

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

/-- Scott's Definition 8.7: the universal neighbourhood system on `[0, 1)` consists of
nonempty finite unions of rational subintervals; it has no minimal neighbourhoods, the
canonical split satisfies `SplitSpec`, $Y$-sequences are unions of atoms in `U`, and
    the
initial atom is the master neighbourhood. -/
theorem definition_87 {α : Type*} :
    (U.master = UMaster) ∧
      (∀ X : Set ?, U.mem X ↔
        ∃ L : List (? × ?), (∀ p ∈ L, (0 : ?) ≤ p.1 ∧ p.1 < p.2 ∧ p.2 ≤ 1) ∧
          X = presentedIntervals L ∧ X.Nonempty) ∧
        U.NoMinimal ∧
        SplitSpec (α := α) splitChoice ∧
        (∀ (split : Set α → Set ? → Set α → Set ? × Set ?) (X : ℕ → Set α) (Δ :
    Set α) (? : ℕ → Bool),
          atomU split X Δ ? 0 = U.master) ∧
        (∀ (split : Set α → Set ? → Set α → Set ? × Set ?) (X : ℕ → Set α) (Δ :
    Set α)
            (_hΔ : Δ.Nonempty) (_hsplit : SplitSpec split) (n : ℕ),
          Yseq split X Δ n = ∅ ∨ U.mem (Yseq split X Δ n)) :=
  ⟨rfl, fun X => U.mem_iff_scott (X := X), U.noMinimal, splitChoice_isSplitSpec,
    fun split X Δ ? => atomU_zero split X Δ ?,
    fun split X Δ _hΔ _hsplit n => Yseq_empty_or_mem split X Δ _hΔ _hsplit n⟩
```

```
end DomainTheory.NeighbourhoodSystems
```

4.9.23 Fixed neighbourhood effective

Scott's last sentence of Theorem 8.8: the fixed-neighbourhood system of a computable finitary projection is effectively given.

Theorem (`fixedNbhd_isEffectivelyGiven`). If a is a computable approximable endomap on an effectively presented domain, then
$YmidYaY$ is effectively given as a neighbourhood system.

Proof. Via the diagonal fixed-point predicate, primitive-recursive folding on codes, and the D_X presentation machinery. ■

`fixedNbhdEffective.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.D_X
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.fixedNbhd

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem Domain.Recursive ApproximableMap

variable {α : Type*} {V : NeighborhoodSystem α}

theorem fixedNbhd_isEffectivelyGiven {P : ComputablePresentation V} {a :
    ApproximableMap V V}
    (hcomp : ComputableMap P P a) : (fixedNbhd a).IsEffectivelyGiven := by
  obtain ⟨qChar, hqCharp, hqChar⟩ := diagFixed_exists_qChar hcomp
  obtain ⟨cons, hconsp, hconse⟩ := P.cons_computable
  have hcons : ∀ n m, cons (Nat.pair n m) = 1 ↔ ∃ k, P.X k ⊆ P.X n ∩ P.X m := by
    intro n m
    have h := hconse (Nat.pair n m)
    simp only [unpair_pair_fst, unpair_pair_snd] at h
    exact h.symm
  refine ⟨{
    X := D_X P qChar cons
    mem_X := D_X_mem hqChar hcons
    surj := fun hY => D_X_surj hqChar hcons hY
    interEq_computable := D_X_interEq_computable P qChar cons hqCharp hconsp
    cons_computable := D_X_cons_computable P qChar cons hqChar hcons hqCharp hconsp
    inter := D_inter
    inter_primrec := D_inter_primrec
    inter_spec := fun h => D_X_inter_spec hcons h
    masterIdx := 0
    masterIdx_spec := ?_ }⟩
  show D_X P qChar cons 0 = V.master
  show P.X (myFoldCode P qChar cons 0) = V.master
  rw [myFoldCode_eq, decodeList_zero, myFold_nil, P.masterIdx_spec]
```

```
end DomainTheory.NeighbourhoodSystems
```

4.9.24 U constructs

Library for $\mathcal{U}+\mathcal{U}, \mathcal{U}\times\mathcal{U}, \mathcal{U}\to\mathcal{U}$: nonemptiness of $\mathcal{U}$-neighbourhoods, effective-presentability of these constructs, their chosen presentations, and the lambda-combinator map used in Definition 8.9.

Theorem (`U_mem_nonempty`). Every neighbourhood in $\mathcal{U}$ is nonempty.
Proof. From the interval presentation of $\mathcal{U}$. ■
Theorem (`funSpaceUU_isEffectivelyGiven`). $\mathcal{U}\to\mathcal{U}$ is effectively given.
Proof. From effective-presentability of $\mathcal{U}$ and the function-space construct. ■
Definition (`funSpaceUUPresentation`). A chosen computable presentation of $\mathcal{U}\to\mathcal{U}$.
Proof. Choose any witness that the function space is effectively given. ■
Definition (`lamComb`). The approximable endomap on $(\mathcal{U}\to\mathcal{U})$ used in the arrow combinator.
Proof. Curry evaluation composed with the indicated product and composition maps. ■
Theorem (`prodUU_isEffectivelyGiven`). $\mathcal{U}\times\mathcal{U}$ is effectively given.
Proof. Product of effectively given $\mathcal{U}$ with itself. ■
Definition (`prodUUPresentation`). A chosen computable presentation of $\mathcal{U}\times\mathcal{U}$.
Proof. Choose any witness that the product is effectively given. ■
Theorem (`sumUU_isEffectivelyGiven`). $\mathcal{U}+\mathcal{U}$ is effectively given.
Proof. Sum of effectively given $\mathcal{U}$ with itself. ■
Definition (`sumUUPresentation`). A chosen computable presentation of $\mathcal{U}+\mathcal{U}$.
Proof. Choose any witness that the sum is effectively given. ■

`UConstructs.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.UX
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    VComputablePresentation
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.cond

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem Domain.Recursive ApproximableMap Cond

theorem U_mem_nonempty : ∀ X, U.mem X → X.Nonempty := fun _ hX => hX.2.1

theorem funSpaceUU_isEffectivelyGiven : (funSpace U U).IsEffectivelyGiven :=
  funSpace_isEffectivelyGiven U_isEffectivelyGiven U_isEffectivelyGiven

noncomputable def funSpaceUUPresentation : ComputablePresentation (funSpace U U) :=
  funSpaceUU_isEffectivelyGiven.some

noncomputable def lamComb (a b : ApproximableMap U U) :
    ApproximableMap (funSpace U U) (funSpace U U) :=
  curry (b.comp ((evalMap U U).comp (prodMap (idMap (funSpace U U)) a)))
```

```
theorem prodUU_isEffectivelyGiven : (prod U U).IsEffectivelyGiven :=
  prod_isEffectivelyGiven U_isEffectivelyGiven U_isEffectivelyGiven

noncomputable def prodUUPresentation : ComputablePresentation (prod U U) :=
  prodUU_isEffectivelyGiven.some

theorem sumUU_isEffectivelyGiven :
    (sum U U U_mem_nonempty U_mem_nonempty).IsEffectivelyGiven :=
  sum_isEffectivelyGiven U_isEffectivelyGiven U_isEffectivelyGiven

noncomputable def sumUUPresentation : ComputablePresentation (sum U U U_mem_nonempty
    U_mem_nonempty) :=
  sumUU_isEffectivelyGiven.some

end DomainTheory.NeighbourhoodSystems
```

4.9.25 Theorem 8.8 — universality of U

THEOREM 8.8. The system $\mathcal{U}$ is universal in the sense that, for every countable neighbourhood system $\mathcal{D}$, we have

$$\mathcal{D} \trianglelefteq \mathcal{U}.$$

Moreover, if $\mathcal{D}$ is effectively given, then the projection pair making the embedding can be taken as computable. Indeed there is a correspondence between effectively presented domains and the computable, finitary projections of $\mathcal{U}$.

Proof: ... (Scott's construction via Y_n, effective choice, and finitary fixed neighbourhoods; see peer files for the formal library.) □

The peer development for the diagonal fixed-point predicate, D-side folding, and atom apparatus is imported alongside this packaging file.

Theorem (`theorem_8_8_a`). Every countable neighbourhood system is isomorphic to a subsystem of $\mathcal{U}$.

Proof. By the D'-in-$\mathcal{U}$ construction and the isomorphism/subsystem witnesses already established in the peer library. ■

Theorem (`theorem_8_8`). Every countable neighbourhood system embeds as a subdomain of $\mathcal{U}$ (Scott's $\mathcal{D} \trianglelefteq \mathcal{U}$).

Proof. From `theorem_8_8_a`. ■

Theorem (`theorem_8_8_b`). For an effectively given $\mathcal{D}$, there is an effectively presented subsystem of $\mathcal{U}$, isomorphic to $\mathcal{D}$, with a computable projection pair witnessing the embedding.

Proof. By the coded D'-in-$\mathcal{U}$ construction and its computable injection/projection maps. ■

Theorem (`theorem_8_8_c`). Every computable finitary projection a on $\mathcal{U}$ yields an effectively given fixed-neighbourhood subdomain of $\mathcal{U}$.

Proof. `fixedNbhd_isEffectivelyGiven` for the map's computability, and the standard fixed-neighbourhood subsystem embedding. ■

Theorem (`theorem_8_8_b_strong`). Every effectively presented domain admits a computable projection pair into $\mathcal{U}$ (and back).

Proof. Strengthens `theorem_8_8_b` with an explicit computable retraction pair into $\mathcal{U}$. ■

`Theorem_8_8.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    fixedNbhdEffective
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    UConstructs
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_6
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.DprimeU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    DprimeUCode
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    QPairCodes
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    domainIsoCode
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    VComputablePresentation
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap

theorem theorem_8_8_a {α : Type*} (D : NeighborhoodSystem α)
    [Countable {S : Set α // D.mem S}] :
    ∃ D' : NeighborhoodSystem ?, (D ≅? D') ∧ (D' ? U) := by
  have : Nonempty {S : Set α // D.mem S} := ⟨⟨D.master, D.master_mem⟩⟩
  obtain ⟨f, hf⟩ := exists_surjective_nat {S : Set α // D.mem S}
  set e : ℕ → Set α := fun n => if n = 0 then D.master else (f (n - 1)).1 with hedef
  have he0 : e 0 = D.master := ite_eq_left rfl
  have hen : ∀ n, e (n + 1) = (f n).1 := by
    intro n
    show (if n + 1 = 0 then D.master else (f (n + 1 - 1)).1) = (f n).1
    rw [ite_eq_right (Nat.succ_ne_zero n), Nat.succ_sub_one]
  have hcover : ∀ S, D.mem S ↔ ∃ n, S = e n := by
    intro S
    constructor
    · intro hS
      obtain ⟨k, hk⟩ := hf ⟨S, hS⟩
      exact ⟨k + 1, by rw [hen k]; exact (congrArg Subtype.val hk).symm⟩
    · rintro ⟨n, rfl⟩
      cases n with
      | zero => rw [he0]; exact D.master_mem
      | succ n => rw [hen n]; exact (f n).2
  exact ⟨DprimeU D e hcover he0, isomorphic_DprimeU D e hcover he0, DprimeU_subsystem D
    e hcover he0⟩

theorem theorem_8_8 {α : Type*} (D : NeighborhoodSystem α)
    [Countable {S : Set α // D.mem S}] : D ? U := by
  obtain ⟨D', hiso, hsub⟩ := theorem_8_8_a D
  exact ⟨D', hsub, hiso⟩
```

```
theorem theorem_8_8_b {α : Type*} {D : NeighborhoodSystem α} (P :
    ComputablePresentation D) :
    ∃ (D' : NeighborhoodSystem ?) (P' : ComputablePresentation D') (h : D' ? U),
      (D ≅? D') ∧
        ComputableMap P' UComputablePresentation h.inj ∧
        ComputableMap UComputablePresentation P' h.proj :=
  ⟨DprimeUCode P, DprimeUCodePresentation P, DprimeUCode_subsystem P,
    isomorphic_DprimeUCode P, DprimeUCode_inj_isComputableMap P,
    DprimeUCode_proj_isComputableMap P⟩

theorem theorem_8_8_c {a : ApproximableMap U U} (_hfin : FinitaryProjection a)
    (hcomp : ComputableMap UComputablePresentation UComputablePresentation a) :
    (fixedNbhd a).IsEffectivelyGiven ∧ fixedNbhd a ? U :=
  ⟨fixedNbhd_isEffectivelyGiven hcomp, fixedNbhd_subsystem a⟩

theorem theorem_8_8_b_strong {α : Type*} {D : NeighborhoodSystem α} (P :
    ComputablePresentation D) :
    ∃ (i : ApproximableMap D U) (j : ApproximableMap U D),
      j.comp i = idMap D ∧ i.comp j ≤ idMap U ∧
        ComputableMap P UComputablePresentation i ∧
        ComputableMap UComputablePresentation P j := by
  set h := DprimeUCode_subsystem P with hh
  refine ⟨h.inj.comp (isoInj P), (isoProj P).comp h.proj, ?_, ?_, ?_, ?_⟩
  · rw [comp_assoc, ← comp_assoc h.proj h.inj (isoInj P), h.proj_comp_inj, idMap_comp]
    exact isoProj_comp_isoInj P
  · have heq : (h.inj.comp (isoInj P)).comp ((isoProj P).comp h.proj) = h.inj.comp h.
    proj := by
      rw [comp_assoc, ← comp_assoc (isoInj P) (isoProj P) h.proj, isoInj_comp_isoProj,
     idMap_comp]
    rw [heq]
    exact h.inj_comp_proj_le
  · exact comp_isComputable (isoInj_isComputableMap P) (DprimeUCode_inj_isComputableMap
     P)
  · exact comp_isComputable (DprimeUCode_proj_isComputableMap P) (
    isoProj_isComputableMap P)

end DomainTheory.NeighbourhoodSystems
```

4.9.26 Lam combinator

lamComb.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    UConstructs
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    projPairToElementMap

@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Cond

theorem lamComb_eq (a b : ApproximableMap U U) :
    lamComb a b = curry (b.comp ((evalMap U U).comp (prodMap (idMap (funSpace U U)) a)
    )) := rfl

theorem toApproxMap_toElementMap_lamComb (a b : ApproximableMap U U)
    (φ : (funSpace U U).Element) :
    toApproxMap ((lamComb a b).toElementMap φ) = b.comp ((toApproxMap φ).comp a) := by
  apply ext_of_toElementMap
  intro z
  rw [lamComb_eq, toElementMap_curry_apply, toElementMap_comp, toElementMap_comp,
    toElementMap_prodMap_pair, toElementMap_idMap, evalMap_apply, toElementMap_comp,
    toElementMap_comp]

theorem isRetraction_lamComb {a b : ApproximableMap U U} (ha : Retraction a)
    (hb : Retraction b) : Retraction (lamComb a b) := by
  constructor
  apply ext_of_toElementMap
  intro φ
  apply (funSpaceEquiv U U).injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toElementMap_comp,
    toApproxMap_toElementMap_lamComb a b ((lamComb a b).toElementMap φ),
    toApproxMap_toElementMap_lamComb a b φ,
    comp_assoc b ((toApproxMap φ).comp a) a,
    ← comp_assoc b b (((toApproxMap φ).comp a).comp a), hb.out,
    comp_assoc (toApproxMap φ) a a, ha.out]

theorem le_idMap_lamComb {a b : ApproximableMap U U} (ha : a ≤ idMap U) (hb : b ≤
    idMap U) :
    lamComb a b ≤ idMap (funSpace U U) := by
  rw [le_iff_toElementMap_le]
  intro φ
  rw [toElementMap_idMap, ← (funSpaceEquiv U U).le_iff_le, funSpaceEquiv_apply,
    funSpaceEquiv_apply, toApproxMap_toElementMap_lamComb]
  calc b.comp ((toApproxMap φ).comp a) ≤ (idMap U).comp ((toApproxMap φ).comp (idMap U
    )) :=
        comp_mono_gen hb (comp_mono_gen le_rfl ha)
    _ = toApproxMap φ := by rw [comp_idMap, idMap_comp]

end DomainTheory.NeighbourhoodSystems
```

4.9.27 Sum combinator

sumComb.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.sumMap
```

```
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    elementIsoOfProjectionPair

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Cond

noncomputable def sumComb (a b : ApproximableMap U U) : ApproximableMap U U :=
  (cond U).comp
    ((paired (whichMap U U U_mem_nonempty U_mem_nonempty)
        (paired (iPlus.comp ((inMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    comp
                  (a.comp (outMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)))))
                (iPlus.comp ((inMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    comp
                  (b.comp (outMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)))))))).
    comp
      jPlus)

theorem toElementMap_bot_eq_bot_of_comp_eq_idMap {α β : Type*} {D : NeighborhoodSystem
     α}
    {E : NeighborhoodSystem β} {i : ApproximableMap D E} {j : ApproximableMap E D}
    (hji : j.comp i = idMap D) : j.toElementMap E.bot = D.bot := by
  apply le_antisymm
  · have h2 := j.toElementMap_mono (E.bot_le (i.toElementMap D.bot))
    rwa [toElementMap_of_comp_eq_idMap hji] at h2
  · exact D.bot_le _

theorem jPlus_bot_eq_bot :
    jPlus.toElementMap U.bot = (sum U U U_mem_nonempty U_mem_nonempty).bot :=
  toElementMap_bot_eq_bot_of_comp_eq_idMap jPlus_comp_iPlus

theorem fixedNbhd_mem_nonempty (a : ApproximableMap U U) :
    ∀ X, (fixedNbhd a).mem X → X.Nonempty :=
  fun X hX => U_mem_nonempty X ((fixedNbhd_subsystem a).sub hX)

namespace sumComb

theorem eq (a b : ApproximableMap U U) :
    sumComb a b = (cond U).comp
      ((paired (whichMap U U U_mem_nonempty U_mem_nonempty)
          (paired (iPlus.comp ((inMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    comp
                    (a.comp (outMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)))))
                  (iPlus.comp ((inMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    comp
                    (b.comp (outMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty))))))
    ).comp
        jPlus) := rfl

theorem toElementMap_mem (a b : ApproximableMap U U) (x : U.Element) {Z : Set ?} :
    ((sumComb a b).toElementMap x).mem Z ↔
      (((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap (jPlus.toElementMap
    x)).mem
```

```
          TwoToken.zero ∧
        (iPlus.toElementMap ((inMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
          (a.toElementMap ((outMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
            (jPlus.toElementMap x))))).mem Z) ∨
      (((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap (jPlus.toElementMap
    x)).mem
          TwoToken.one ∧
        (iPlus.toElementMap ((inMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
          (b.toElementMap ((outMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
            (jPlus.toElementMap x))))).mem Z) ∨
      Z = U.master := by
  rw [eq]
  simp only [toElementMap_comp, toElementMap_paired]
  rw [cond_toElementMap_mem]

theorem toElementMap_of_left {a b : ApproximableMap U U} {x : U.Element}
    (hL : ∃ X, U.mem X ∧ (jPlus.toElementMap x).mem (inj₀ X)) :
    (sumComb a b).toElementMap x =
      iPlus.toElementMap ((inMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
        (a.toElementMap ((outMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
          (jPlus.toElementMap x)))) := by
  have hz : ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem TwoToken.zero := which_mem_zero.mpr hL
  have hno : ¬ ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem TwoToken.one :=
    fun h => not_sum_reaches_both hL (which_mem_one.mp h)
  apply Element.ext
  intro Z
  rw [toElementMap_mem]
  constructor
  · rintro (⟨_, h⟩ | ⟨h1, _⟩ | rfl)
    · exact h
    · exact absurd h1 hno
    · exact Element.master_mem _
  · intro h
    exact Or.inl ⟨hz, h⟩

theorem toElementMap_of_neither {a b : ApproximableMap U U} {x : U.Element}
    (hN : ∀ W, (jPlus.toElementMap x).mem W → W = (sum U U U_mem_nonempty
    U_mem_nonempty).master) :
    (sumComb a b).toElementMap x = U.bot := by
  have hnz : ¬ ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem TwoToken.zero := by
    rw [which_mem_zero]
    rintro ⟨X, _, hX⟩
    exact inj₀_ne_sumMaster (hN _ hX)
  have hno : ¬ ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem TwoToken.one := by
    rw [which_mem_one]
    rintro ⟨Y, _, hY⟩
```

```
    exact inj₁_ne_sumMaster (hN _ hY)
  apply Element.ext
  intro Z
  rw [toElementMap_mem, mem_bot]
  constructor
  · rintro (⟨h, _⟩ | ⟨h, _⟩ | h)
    · exact absurd h hnz
    · exact absurd h hno
    · exact h
  · intro h
    exact Or.inr (Or.inr h)

theorem toElementMap_of_right {a b : ApproximableMap U U} {x : U.Element}
    (hR : ∃ Y, U.mem Y ∧ (jPlus.toElementMap x).mem (inj₁ Y)) :
    (sumComb a b).toElementMap x =
      iPlus.toElementMap ((inMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
        (b.toElementMap ((outMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
          (jPlus.toElementMap x)))) := by
  have ho : ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem TwoToken.one := which_mem_one.mpr hR
  have hnz : ¬ ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem TwoToken.zero :=
    fun h => not_sum_reaches_both (which_mem_zero.mp h) hR
  apply Element.ext
  intro Z
  rw [toElementMap_mem]
  constructor
  · rintro (⟨h1, _⟩ | ⟨_, h⟩ | rfl)
    · exact absurd h1 hnz
    · exact h
    · exact Element.master_mem _
  · intro h
    exact Or.inr (Or.inl ⟨ho, h⟩)

theorem isRetraction {a b : ApproximableMap U U} (ha : Retraction a)
    (hb : Retraction b) : Retraction (sumComb a b) := by
  constructor
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp]
  rcases sum_element_trichotomy (jPlus.toElementMap x) with hL | hR | hN
  · rw [toElementMap_of_left hL]
    set v := a.toElementMap (outMap₀.toElementMap (jPlus.toElementMap x)) with hv
    have hjy : jPlus.toElementMap (iPlus.toElementMap (inMap₀.toElementMap v))
        = inMap₀.toElementMap v := toElementMap_of_comp_eq_idMap jPlus_comp_iPlus _
    have hLy : ∃ X, U.mem X ∧
        (jPlus.toElementMap (iPlus.toElementMap (inMap₀.toElementMap v))).mem (inj₀ X)
    := by
      rw [hjy]; exact inMap₀_toElementMap_reaches_left v
    rw [toElementMap_of_left hLy, hjy,
      toElementMap_of_comp_eq_idMap outMap₀_comp_inMap₀ v,
    toElementMap_idem_of_isRetraction ha]
  · rw [toElementMap_of_right hR]
    set v := b.toElementMap (outMap₁.toElementMap (jPlus.toElementMap x)) with hv
```

```
    have hjy : jPlus.toElementMap (iPlus.toElementMap (inMap₁.toElementMap v))
        = inMap₁.toElementMap v := toElementMap_of_comp_eq_idMap jPlus_comp_iPlus _
    have hRy : ∃ Y, U.mem Y ∧
        (jPlus.toElementMap (iPlus.toElementMap (inMap₁.toElementMap v))).mem (inj₁ Y)
    := by
      rw [hjy]; exact inMap₁_toElementMap_reaches_right v
    rw [toElementMap_of_right hRy, hjy,
      toElementMap_of_comp_eq_idMap outMap₁_comp_inMap₁ v,
    toElementMap_idem_of_isRetraction hb]
  · rw [toElementMap_of_neither hN]
    apply toElementMap_of_neither
    intro W hW
    rw [jPlus_bot_eq_bot, mem_bot] at hW
    exact hW

theorem le_idMap {a b : ApproximableMap U U} (ha : a ≤ idMap U) (hb : b ≤ idMap U) :
    sumComb a b ≤ idMap U := by
  rw [le_iff_toElementMap_le]
  intro x
  rw [toElementMap_idMap]
  rcases sum_element_trichotomy (jPlus.toElementMap x) with hL | hR | hN
  · calc (sumComb a b).toElementMap x
        = iPlus.toElementMap ((inMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
            (a.toElementMap ((outMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
              (jPlus.toElementMap x)))) := toElementMap_of_left hL
      _ ≤ iPlus.toElementMap ((inMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
            ((outMap₀ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).toElementMap
              (jPlus.toElementMap x))) :=
          iPlus.toElementMap_mono (inMap₀.toElementMap_mono (
    toElementMap_le_self_of_le_idMap ha _))
      _ = iPlus.toElementMap (jPlus.toElementMap x) := by rw [inMap₀_outMap₀_eq_of_left
     hL]
      _ ≤ x := toElementMap_le_of_comp_le_idMap iPlus_comp_jPlus_le x
  · calc (sumComb a b).toElementMap x
        = iPlus.toElementMap ((inMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
            (b.toElementMap ((outMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
              (jPlus.toElementMap x)))) := toElementMap_of_right hR
      _ ≤ iPlus.toElementMap ((inMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).
    toElementMap
            ((outMap₁ (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)).toElementMap
              (jPlus.toElementMap x))) :=
          iPlus.toElementMap_mono (inMap₁.toElementMap_mono (
    toElementMap_le_self_of_le_idMap hb _))
      _ = iPlus.toElementMap (jPlus.toElementMap x) := by rw [inMap₁_outMap₁
    _eq_of_right hR]
      _ ≤ x := toElementMap_le_of_comp_le_idMap iPlus_comp_jPlus_le x
  · rw [toElementMap_of_neither hN]
    exact U.bot_le x

theorem isProjection {a b : ApproximableMap U U}
    (ha : IsProjection a) (hb : IsProjection b) : IsProjection (sumComb a b) :=
```

```
  ⟨isRetraction ha.1 hb.1, le_idMap ha.2 hb.2⟩

noncomputable def I (a b : ApproximableMap U U) :
    ApproximableMap (sum (fixedNbhd a) (fixedNbhd b)
      (fixedNbhd_mem_nonempty a) (fixedNbhd_mem_nonempty b)) U :=
  iPlus.comp (sumMap (h₀' := U_mem_nonempty) (h₁' := U_mem_nonempty)
    (fixedNbhd_subsystem a).inj (fixedNbhd_subsystem b).inj)

noncomputable def J (a b : ApproximableMap U U) :
    ApproximableMap U (sum (fixedNbhd a) (fixedNbhd b)
      (fixedNbhd_mem_nonempty a) (fixedNbhd_mem_nonempty b)) :=
  (sumMap (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty)
    (fixedNbhd_subsystem a).proj (fixedNbhd_subsystem b).proj).comp jPlus

theorem eq_iPlus_sumMap_jPlus (a b : ApproximableMap U U) :
    sumComb a b = iPlus.comp
      ((sumMap (h₀ := U_mem_nonempty) (h₁ := U_mem_nonempty) (h₀' := U_mem_nonempty)
          (h₁' := U_mem_nonempty) a b).comp jPlus) := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_comp]
  rcases sum_element_trichotomy (jPlus.toElementMap x) with hL | hR | hN
  · rw [toElementMap_of_left hL, sumMap.toElementMap_of_left hL]
  · rw [toElementMap_of_right hR, sumMap.toElementMap_of_right hR]
  · rw [toElementMap_of_neither hN, sumMap.reaches_neither_iff_eq_bot.mp hN, sumMap.bot
    ,
      toElementMap_bot_eq_bot_of_comp_le_idMap iPlus_comp_jPlus_le]

theorem I_comp_J (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    (I a b).comp (J a b) = sumComb a b := by
  unfold I J
  rw [comp_assoc, ← comp_assoc (sumMap _ _) (sumMap _ _) jPlus, ← sumMap.comp,
    inj_comp_proj_eq_self ha, inj_comp_proj_eq_self hb, ← comp_assoc,
    eq_iPlus_sumMap_jPlus, comp_assoc]

theorem J_comp_I (a b : ApproximableMap U U) :
    (J a b).comp (I a b) =
      idMap (sum (fixedNbhd a) (fixedNbhd b) (fixedNbhd_mem_nonempty a)
        (fixedNbhd_mem_nonempty b)) := by
  unfold J I
  rw [comp_assoc, ← comp_assoc jPlus iPlus (sumMap _ _), jPlus_comp_iPlus, idMap_comp,
    ← sumMap.comp, (fixedNbhd_subsystem a).proj_comp_inj, (fixedNbhd_subsystem b).
    proj_comp_inj,
    sumMap.id]

noncomputable def elementIso (a b : ApproximableMap U U)
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    (sum (fixedNbhd a) (fixedNbhd b) (fixedNbhd_mem_nonempty a) (
    fixedNbhd_mem_nonempty b)).Element
      ≃o {y : U.Element // (sumComb a b).toElementMap y = y} :=
  elementIsoOfProjectionPair (I a b) (J a b) (J_comp_I a b) (I_comp_J a b ha hb).symm

theorem finitary (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    IsFinitary (sumComb a b) :=
```

```
  isFinitary_of_projectionPair (I a b) (J a b) (J_comp_I a b) (I_comp_J a b ha hb).
    symm

theorem finitaryProjection (a b : ApproximableMap U U)
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    FinitaryProjection (sumComb a b) :=
  ⟨⟨isProjection ha.out.1 hb.out.1, finitary a b ha hb⟩⟩

end sumComb

end DomainTheory.NeighbourhoodSystems
```

4.9.28 Product combinator

prodComb.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    elementIsoOfProjectionPair

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Cond

noncomputable def prodComb (a b : ApproximableMap U U) : ApproximableMap U U :=
  iTimes.comp ((paired (a.comp (proj₀ U U)) (b.comp (proj₁ U U))).comp jTimes)

namespace prodComb

theorem eq (a b : ApproximableMap U U) :
    prodComb a b = iTimes.comp ((prodMap a b).comp jTimes) := rfl

theorem toElementMap (a b : ApproximableMap U U) (x : U.Element) :
    (prodComb a b).toElementMap x =
      iTimes.toElementMap (pair (a.toElementMap (jTimes.toElementMap x).fst)
        (b.toElementMap (jTimes.toElementMap x).snd)) := by
  rw [eq]
  simp only [toElementMap_comp, toElementMap_prodMap]

theorem isRetraction {a b : ApproximableMap U U} (ha : Retraction a)
    (hb : Retraction b) : Retraction (prodComb a b) := by
  constructor
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap a b ((prodComb a b).toElementMap x),
    toElementMap a b x, toElementMap_of_comp_eq_idMap jTimes_comp_iTimes, fst_pair,
    snd_pair, toElementMap_idem_of_isRetraction ha, toElementMap_idem_of_isRetraction
    hb]
```

```
theorem le_idMap {a b : ApproximableMap U U} (ha : a ≤ idMap U) (hb : b ≤ idMap U) :
    prodComb a b ≤ idMap U := by
  rw [le_iff_toElementMap_le]
  intro x
  rw [toElementMap, toElementMap_idMap]
  have hpair : pair (a.toElementMap (jTimes.toElementMap x).fst)
      (b.toElementMap (jTimes.toElementMap x).snd)
      ≤ pair (jTimes.toElementMap x).fst (jTimes.toElementMap x).snd :=
    pair_le_pair_iff.mpr
      ⟨toElementMap_le_self_of_le_idMap ha _, toElementMap_le_self_of_le_idMap hb _⟩
  rw [pair_fst_snd] at hpair
  calc iTimes.toElementMap (pair (a.toElementMap (jTimes.toElementMap x).fst)
        (b.toElementMap (jTimes.toElementMap x).snd))
      ≤ iTimes.toElementMap (jTimes.toElementMap x) := iTimes.toElementMap_mono hpair
    _ ≤ x := toElementMap_le_of_comp_le_idMap iTimes_comp_jTimes_le x

theorem isProjection {a b : ApproximableMap U U}
    (ha : IsProjection a) (hb : IsProjection b) : IsProjection (prodComb a b) :=
  ⟨isRetraction ha.1 hb.1, le_idMap ha.2 hb.2⟩

noncomputable def I (a b : ApproximableMap U U) :
    ApproximableMap (prod (fixedNbhd a) (fixedNbhd b)) U :=
  iTimes.comp (prodMap (fixedNbhd_subsystem a).inj (fixedNbhd_subsystem b).inj)

noncomputable def J (a b : ApproximableMap U U) :
    ApproximableMap U (prod (fixedNbhd a) (fixedNbhd b)) :=
  (prodMap (fixedNbhd_subsystem a).proj (fixedNbhd_subsystem b).proj).comp jTimes

theorem J_comp_I (a b : ApproximableMap U U) :
    (J a b).comp (I a b) = idMap (prod (fixedNbhd a) (fixedNbhd b)) := by
  unfold J I
  rw [comp_assoc, ← comp_assoc jTimes iTimes (prodMap _ _), jTimes_comp_iTimes,
    idMap_comp,
    ← prodMap_comp, (fixedNbhd_subsystem a).proj_comp_inj, (fixedNbhd_subsystem b).
    proj_comp_inj,
    prodMap_id]

theorem I_comp_J (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    (I a b).comp (J a b) = prodComb a b := by
  unfold I J
  rw [comp_assoc, ← comp_assoc (prodMap _ _) (prodMap _ _) jTimes, ← prodMap_comp,
    inj_comp_proj_eq_self ha, inj_comp_proj_eq_self hb, ← comp_assoc, prodComb.eq,
    comp_assoc]

noncomputable def elementIso (a b : ApproximableMap U U)
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    (prod (fixedNbhd a) (fixedNbhd b)).Element ≃o
      {y : U.Element // (prodComb a b).toElementMap y = y} :=
  elementIsoOfProjectionPair (I a b) (J a b) (J_comp_I a b) (I_comp_J a b ha hb).symm

theorem finitary (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    IsFinitary (prodComb a b) :=
  isFinitary_of_projectionPair (I a b) (J a b) (J_comp_I a b) (I_comp_J a b ha hb).
    symm
```

```
theorem finitaryProjection (a b : ApproximableMap U U)
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    FinitaryProjection (prodComb a b) :=
  ⟨⟨isProjection ha.out.1 hb.out.1, finitary a b ha hb⟩⟩

end prodComb

end DomainTheory.NeighbourhoodSystems
```

4.9.29 Arrow combinator

arrowComb.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.expMap
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    elementIsoOfProjectionPair

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Cond

noncomputable def arrowComb (a b : ApproximableMap U U) : ApproximableMap U U :=
  iArrow.comp ((lamComb a b).comp jArrow)

namespace arrowComb

theorem eq (a b : ApproximableMap U U) :
    arrowComb a b = iArrow.comp ((lamComb a b).comp jArrow) := rfl

theorem toElementMap (a b : ApproximableMap U U) (x : U.Element) :
    (arrowComb a b).toElementMap x =
      iArrow.toElementMap ((lamComb a b).toElementMap (jArrow.toElementMap x)) := by
  rw [eq]
  simp only [toElementMap_comp]

theorem isRetraction {a b : ApproximableMap U U} (ha : Retraction a)
    (hb : Retraction b) : Retraction (arrowComb a b) := by
  constructor
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap a b ((arrowComb a b).toElementMap x),
    toElementMap a b x, toElementMap_of_comp_eq_idMap jArrow_comp_iArrow,
    toElementMap_idem_of_isRetraction (isRetraction_lamComb ha hb)]

theorem le_idMap {a b : ApproximableMap U U} (ha : a ≤ idMap U) (hb : b ≤ idMap U) :
    arrowComb a b ≤ idMap U := by
  rw [le_iff_toElementMap_le]
```

```
  intro x
  rw [toElementMap, toElementMap_idMap]
  calc iArrow.toElementMap ((lamComb a b).toElementMap (jArrow.toElementMap x))
      ≤ iArrow.toElementMap (jArrow.toElementMap x) :=
        iArrow.toElementMap_mono
          (toElementMap_le_self_of_le_idMap (le_idMap_lamComb ha hb) _)
    _ ≤ x := toElementMap_le_of_comp_le_idMap iArrow_comp_jArrow_le x

theorem isProjection {a b : ApproximableMap U U}
    (ha : IsProjection a) (hb : IsProjection b) : IsProjection (arrowComb a b) :=
  ⟨isRetraction ha.1 hb.1, le_idMap ha.2 hb.2⟩

noncomputable def I (a b : ApproximableMap U U) :
    ApproximableMap (funSpace (fixedNbhd a) (fixedNbhd b)) U :=
  iArrow.comp (expMap (fixedNbhd_subsystem a).proj (fixedNbhd_subsystem b).inj)

noncomputable def J (a b : ApproximableMap U U) :
    ApproximableMap U (funSpace (fixedNbhd a) (fixedNbhd b)) :=
  (expMap (fixedNbhd_subsystem a).inj (fixedNbhd_subsystem b).proj).comp jArrow

theorem J_comp_I (a b : ApproximableMap U U) :
    (J a b).comp (I a b) = idMap (funSpace (fixedNbhd a) (fixedNbhd b)) := by
  unfold J I
  rw [comp_assoc, ← comp_assoc jArrow iArrow (expMap _ _), jArrow_comp_iArrow,
    idMap_comp,
    ← expMap.comp, (fixedNbhd_subsystem a).proj_comp_inj, (fixedNbhd_subsystem b).
    proj_comp_inj,
    expMap.id]

theorem I_comp_J (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    (I a b).comp (J a b) = arrowComb a b := by
  unfold I J
  rw [comp_assoc, ← comp_assoc (expMap _ _) (expMap _ _) jArrow, ← expMap.comp,
    inj_comp_proj_eq_self ha, inj_comp_proj_eq_self hb, expMap.eq_lamComb, ←
    comp_assoc,
    arrowComb.eq, comp_assoc]

noncomputable def elementIso (a b : ApproximableMap U U)
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    (funSpace (fixedNbhd a) (fixedNbhd b)).Element ≃o
      {y : U.Element // (arrowComb a b).toElementMap y = y} :=
  elementIsoOfProjectionPair (I a b) (J a b) (J_comp_I a b) (I_comp_J a b ha hb).symm

theorem finitary (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    IsFinitary (arrowComb a b) :=
  isFinitary_of_projectionPair (I a b) (J a b) (J_comp_I a b) (I_comp_J a b ha hb).
    symm

theorem finitaryProjection (a b : ApproximableMap U U)
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    FinitaryProjection (arrowComb a b) :=
  ⟨⟨isProjection ha.out.1 hb.out.1, finitary a b ha hb⟩⟩

end arrowComb
```

```
end DomainTheory.NeighbourhoodSystems
```

4.9.30 Exponential map

expMap.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    UConstructs
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Cond

noncomputable def expMap {? ? ?' ?' : Type*} {V0 : NeighborhoodSystem ?} {V1 :
    NeighborhoodSystem ?}
    {V0' : NeighborhoodSystem ?'} {V1' : NeighborhoodSystem ?'}
    (h : ApproximableMap V0' V0) (k : ApproximableMap V1 V1') :
    ApproximableMap (funSpace V0 V1) (funSpace V0' V1') :=
  curry (k.comp ((evalMap V0 V1).comp (prodMap (idMap (funSpace V0 V1)) h)))

namespace expMap

variable {? ? ?' ?' ?'' ?'' : Type*}
variable {V0 : NeighborhoodSystem ?} {V1 : NeighborhoodSystem ?}
variable {V0' : NeighborhoodSystem ?'} {V1' : NeighborhoodSystem ?'}
variable {V0'' : NeighborhoodSystem ?''} {V1'' : NeighborhoodSystem ?''}

theorem eq (h : ApproximableMap V0' V0) (k : ApproximableMap V1 V1') :
    expMap h k = curry (k.comp ((evalMap V0 V1).comp (prodMap (idMap (funSpace V0 V1))
     h))) := rfl

theorem eq_lamComb (a b : ApproximableMap U U) : expMap a b = lamComb a b := rfl

theorem toApproxMap_toElementMap (h : ApproximableMap V0' V0) (k : ApproximableMap V1
    V1')
    (φ : (funSpace V0 V1).Element) :
    toApproxMap ((expMap h k).toElementMap φ) = k.comp ((toApproxMap φ).comp h) := by
  apply ext_of_toElementMap
  intro z
  rw [eq, toElementMap_curry_apply, toElementMap_comp, toElementMap_comp,
    toElementMap_prodMap_pair, toElementMap_idMap, evalMap_apply, toElementMap_comp,
    toElementMap_comp]

theorem comp (h : ApproximableMap V0' V0) (k : ApproximableMap V1 V1')
    (h' : ApproximableMap V0'' V0') (k' : ApproximableMap V1' V1'') :
    expMap (h.comp h') (k'.comp k) = (expMap h' k').comp (expMap h k) := by
  apply ext_of_toElementMap
```

```
  intro φ
  apply (funSpaceEquiv V0'' V1'').injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toApproxMap_toElementMap,
    toElementMap_comp,
    toApproxMap_toElementMap, toApproxMap_toElementMap]
  simp only [comp_assoc]

theorem id : expMap (idMap V0) (idMap V1) = idMap (funSpace V0 V1) := by
  apply ext_of_toElementMap
  intro φ
  apply (funSpaceEquiv V0 V1).injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toElementMap_idMap,
    toApproxMap_toElementMap, comp_idMap, idMap_comp]

theorem mono_left {h h' : ApproximableMap V0' V0} (hh : h ≤ h') (k : ApproximableMap
    V1 V1') :
    expMap h k ≤ expMap h' k := by
  rw [le_iff_toElementMap_le]
  intro φ
  apply (funSpaceEquiv V0' V1').map_rel_iff.mp
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toApproxMap_toElementMap,
    toApproxMap_toElementMap]
  exact comp_mono_gen le_rfl (comp_mono_gen le_rfl hh)

end expMap

end DomainTheory.NeighbourhoodSystems
```

4.9.31 Definition 8.9 — constructs on projections

DEFINITION 8.9. Let the computable projection pairs $i_+ : \mathcal{U} + \mathcal{U} \to \mathcal{U}$ and $j_+ : \mathcal{U} \to \mathcal{U} + \mathcal{U}$ be fixed. Similarly choose $i_\times, j_\times$ and $i_\to, j_\to$ for $\mathcal{U} \times \mathcal{U}$ and $\mathcal{U} \to \mathcal{U}$. Define:

$$a + b = \mathrm{cond} \circ \langle \mathrm{which}, i_+ \circ \mathrm{in}_0 \circ a \circ \mathrm{out}_0, i_+ \circ \mathrm{in}_1 \circ b \circ \mathrm{out}_1 \rangle \circ j_+ \,;$$

$$a \times b = i_\times \circ \langle a \circ \mathrm{proj}_0, b \circ \mathrm{proj}_1 \rangle \circ j_\times \,;$$

$$a \to b = i_\to \circ (\lambda f.\, b \circ f \circ a) \circ j_\to \,,$$

for all $a, b : \mathcal{U} \to \mathcal{U}$. □

Definition (`iPlus`), Definition (`jPlus`), Definition (`iTimes`), Definition (`jTimes`), Definition (`iArrow`), Definition (`jArrow`). The six computable maps witnessing $\mathcal{U} + \mathcal{U}$, $\mathcal{U} \times \mathcal{U}$, and $\mathcal{U} \to \mathcal{U}$ as subdomains of $\mathcal{U}$.

Proof. From `theorem_8_8_b_strong` on the corresponding $\mathcal{U}$-construct presentations. ■

Theorem (`definition_89`). Scott's six projection-pair equations hold for the chosen i_*, j_*.

Proof. Conjunction of the individual composite-identity and composite-below-identity lemmas for the six maps. ■

`projPairToElementMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    retraction
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.idMap
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.cond
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.sSupMaps

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Cond

section
variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

theorem inMap₀_toElementMap_reaches_left {? ?} {V₀ : NeighborhoodSystem ?} {V₁ :
    NeighborhoodSystem ?}
    {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty} (v : V₀.
    Element) :
    ∃ X, V₀.mem X ∧ ((inMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap v).mem (inj₀ X) := by
  refine ⟨V₀.master, V₀.master_mem, V₀.master, v.master_mem, V₀.master_mem, ?_,
    subset_rfl⟩
  exact Or.inr (Or.inl ⟨V₀.master, V₀.master_mem, rfl⟩)

theorem inMap₁_toElementMap_reaches_right {? ?} {V₀ : NeighborhoodSystem ?} {V₁ :
    NeighborhoodSystem ?}
    {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty} (v : V₁.
    Element) :
    ∃ Y, V₁.mem Y ∧ ((inMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap v).mem (inj₁ Y) := by
  refine ⟨V₁.master, V₁.master_mem, V₁.master, v.master_mem, V₁.master_mem, ?_,
    subset_rfl⟩
  exact Or.inr (Or.inr ⟨V₁.master, V₁.master_mem, rfl⟩)

theorem not_sum_reaches_both {? ?} {V₀ : NeighborhoodSystem ?} {V₁ : NeighborhoodSystem
    ?}
    {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}
    {w : (sum V₀ V₁ h₀ h₁).Element} (hL : ∃ X, V₀.mem X ∧ w.mem (inj₀ X))
    (hR : ∃ Y, V₁.mem Y ∧ w.mem (inj₁ Y)) : False := by
  obtain ⟨X, _, hX⟩ := hL
  obtain ⟨Y, _, hY⟩ := hR
  have h := w.inter_mem hX hY
  rw [inj₀_inter_inj₁] at h
  exact not_sum_mem_empty (w.sub h)

theorem toElementMap_idem_of_isRetraction {a : ApproximableMap E E} (ha : Retraction a
    )
    (x : E.Element) : a.toElementMap (a.toElementMap x) = a.toElementMap x := by
  rw [← toElementMap_comp, ha.out]

theorem toElementMap_le_of_comp_le_idMap {i : ApproximableMap D E} {j :
    ApproximableMap E D}
    (hij : i.comp j ≤ idMap E) (x : E.Element) : i.toElementMap (j.toElementMap x) ≤ x
```

```
    := by
  have h := (le_iff_toElementMap_le.mp hij) x
  rwa [toElementMap_comp, toElementMap_idMap] at h

theorem toElementMap_le_self_of_le_idMap {a : ApproximableMap E E} (ha : a ≤ idMap E)
    (x : E.Element) : a.toElementMap x ≤ x := by
  have h := (le_iff_toElementMap_le.mp ha) x
  rwa [toElementMap_idMap] at h

theorem toElementMap_of_comp_eq_idMap {i : ApproximableMap D E} {j : ApproximableMap E
    D}
    (hji : j.comp i = idMap D) (v : D.Element) : j.toElementMap (i.toElementMap v) = v
    := by
  rw [← toElementMap_comp, hji, toElementMap_idMap]

end

end DomainTheory.NeighbourhoodSystems
```

Definition_8_9.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    projPairToElementMap
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.lamComb

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

noncomputable def iArrow : ApproximableMap (funSpace U U) U :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose

noncomputable def iPlus : ApproximableMap (sum U U U_mem_nonempty U_mem_nonempty) U :=
  (theorem_8_8_b_strong sumUUPresentation).choose

noncomputable def iTimes : ApproximableMap (prod U U) U :=
  (theorem_8_8_b_strong prodUUPresentation).choose

noncomputable def jArrow : ApproximableMap U (funSpace U U) :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose_spec.choose

noncomputable def jPlus : ApproximableMap U (sum U U U_mem_nonempty U_mem_nonempty) :=
  (theorem_8_8_b_strong sumUUPresentation).choose_spec.choose

noncomputable def jTimes : ApproximableMap U (prod U U) :=
  (theorem_8_8_b_strong prodUUPresentation).choose_spec.choose

theorem iArrow_comp_jArrow_le : iArrow.comp jArrow ≤ idMap U :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose_spec.choose_spec.2.1
```

```
theorem iArrow_isComputableMap :
    ComputableMap funSpaceUUPresentation UComputablePresentation iArrow :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose_spec.choose_spec.2.2.1

theorem iPlus_comp_jPlus_le : iPlus.comp jPlus ≤ idMap U :=
  (theorem_8_8_b_strong sumUUPresentation).choose_spec.choose_spec.2.1

theorem iPlus_isComputableMap : ComputableMap sumUUPresentation
    UComputablePresentation iPlus :=
  (theorem_8_8_b_strong sumUUPresentation).choose_spec.choose_spec.2.2.1

theorem iTimes_comp_jTimes_le : iTimes.comp jTimes ≤ idMap U :=
  (theorem_8_8_b_strong prodUUPresentation).choose_spec.choose_spec.2.1

theorem iTimes_isComputableMap :
    ComputableMap prodUUPresentation UComputablePresentation iTimes :=
  (theorem_8_8_b_strong prodUUPresentation).choose_spec.choose_spec.2.2.1

theorem jArrow_comp_iArrow : jArrow.comp iArrow = idMap _ :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose_spec.choose_spec.1

theorem jArrow_isComputableMap :
    ComputableMap UComputablePresentation funSpaceUUPresentation jArrow :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose_spec.choose_spec.2.2.2

theorem jPlus_comp_iPlus : jPlus.comp iPlus = idMap _ :=
  (theorem_8_8_b_strong sumUUPresentation).choose_spec.choose_spec.1

theorem jPlus_isComputableMap : ComputableMap UComputablePresentation
    sumUUPresentation jPlus :=
  (theorem_8_8_b_strong sumUUPresentation).choose_spec.choose_spec.2.2.2

theorem jTimes_comp_iTimes : jTimes.comp iTimes = idMap _ :=
  (theorem_8_8_b_strong prodUUPresentation).choose_spec.choose_spec.1

theorem jTimes_isComputableMap :
    ComputableMap UComputablePresentation prodUUPresentation jTimes :=
  (theorem_8_8_b_strong prodUUPresentation).choose_spec.choose_spec.2.2.2

theorem definition_89 :
    jArrow.comp iArrow = idMap (funSpace U U) ∧
      jPlus.comp iPlus = idMap (sum U U U_mem_nonempty U_mem_nonempty) ∧
        jTimes.comp iTimes = idMap (prod U U) ∧
          iArrow.comp jArrow ≤ idMap U ∧
            iPlus.comp jPlus ≤ idMap U ∧
              iTimes.comp jTimes ≤ idMap U :=
  ⟨jArrow_comp_iArrow, jPlus_comp_iPlus, jTimes_comp_iTimes,
    iArrow_comp_jArrow_le, iPlus_comp_jPlus_le, iTimes_comp_jTimes_le⟩

end DomainTheory.NeighbourhoodSystems
```

4.9.32 Element iso of projection pair

elementIsoOfProjectionPair.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_3
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.fixedNbhd
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.sub
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    projPairToElementMap
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Cond
universe u
variable {α β : Type u} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

def elementIsoOfProjectionPair (i : ApproximableMap D E) (j : ApproximableMap E D)
    (hji : j.comp i = idMap D) {g : ApproximableMap E E} (hg : g = i.comp j) :
    D.Element ≃o {y : E.Element // g.toElementMap y = y} where
  toFun x := ⟨i.toElementMap x, by
    have h1 : j.toElementMap (i.toElementMap x) = (j.comp i).toElementMap x :=
      (toElementMap_comp j i x).symm
    rw [hg, toElementMap_comp, h1, hji, toElementMap_idMap]⟩
  invFun y := j.toElementMap y.1
  left_inv x := by
    have h1 : j.toElementMap (i.toElementMap x) = (j.comp i).toElementMap x :=
      (toElementMap_comp j i x).symm
    show j.toElementMap (i.toElementMap x) = x
    rw [h1, hji, toElementMap_idMap]
  right_inv y := by
    apply Subtype.ext
    have hfix : g.toElementMap y.1 = y.1 := y.2
    show i.toElementMap (j.toElementMap y.1) = y.1
    rw [← toElementMap_comp, ← hg]
    exact hfix
  map_rel_iff' := by
    intro x x'
    show i.toElementMap x ≤ i.toElementMap x' ↔ x ≤ x'
    constructor
    · intro hle
      have hmono := j.toElementMap_mono hle
      have h1 : j.toElementMap (i.toElementMap x) = (j.comp i).toElementMap x :=
        (toElementMap_comp j i x).symm
      have h2 : j.toElementMap (i.toElementMap x') = (j.comp i).toElementMap x' :=
        (toElementMap_comp j i x').symm
      rw [h1, h2, hji, toElementMap_idMap, toElementMap_idMap] at hmono
      exact hmono
    · intro hle
      exact i.toElementMap_mono hle

theorem inj_comp_proj_eq_self {a : ApproximableMap U U} (ha : FinitaryProjection a) :
    (fixedNbhd_subsystem a).inj.comp (fixedNbhd_subsystem a).proj = a :=
  sub_eq_self_of_isFinitaryProjection ha
```

```
theorem isFinitary_of_projectionPair (i : ApproximableMap D E) (j : ApproximableMap E
    D)
    (hji : j.comp i = idMap D) {g : ApproximableMap E E} (hg : g = i.comp j) :
    IsFinitary g :=
  ⟨α, D, ⟨(elementIsoOfProjectionPair i j hji hg).symm⟩⟩

theorem toElementMap_bot_eq_bot_of_comp_le_idMap {i : ApproximableMap D E} {j :
    ApproximableMap E D}
    (hij : i.comp j ≤ idMap E) : i.toElementMap D.bot = E.bot := by
  apply le_antisymm
  · calc i.toElementMap D.bot ≤ i.toElementMap (j.toElementMap E.bot) :=
          i.toElementMap_mono (D.bot_le _)
      _ ≤ E.bot := toElementMap_le_of_comp_le_idMap hij E.bot
  · exact E.bot_le _

end DomainTheory.NeighbourhoodSystems
```

4.9.33 Proposition 8.10 — constructs preserve projections

PROPOSITION 8.10. If $a, b : \mathcal{U} \to \mathcal{U}$ are projections, then so are $a + b$, $a \times b$, and $a \to b$. If a and b are finitary, then so are the others; for the fixed-point set of each of them is isomorphic to the corresponding construct applied to the domains determined by a and b.

Proof: … (Scott's sketch via subdomains and D_a, D_b.) □

Theorem (`proposition_810`). Finitary projections are preserved under $+$, $\times$, and $\to$, with the expected isomorphisms to the corresponding subdomain constructs.

Proof. By the combinator peers' finitary-projection witnesses and fixed-point element isomorphisms. ■

Theorem (`proposition_8_10`). Alias of `proposition_810`.

Proof. Reflexivity. ■

`Proposition_8_10.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.sumComb
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.prodComb
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.arrowComb

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

theorem isProjection_combinators {a b : ApproximableMap U U}
    (ha : IsProjection a) (hb : IsProjection b) :
    IsProjection (sumComb a b) ∧ IsProjection (prodComb a b) ∧ IsProjection (arrowComb
     a b) :=
  ⟨sumComb.isProjection ha hb, prodComb.isProjection ha hb, arrowComb.isProjection ha
    hb⟩

theorem finitaryProjection_combinators {a b : ApproximableMap U U}
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
```

```
    FinitaryProjection (sumComb a b) ∧ FinitaryProjection (prodComb a b) ∧
      FinitaryProjection (arrowComb a b) :=
  ⟨sumComb.finitaryProjection a b ha hb, prodComb.finitaryProjection a b ha hb,
    arrowComb.finitaryProjection a b ha hb⟩

theorem proposition_810 {a b : ApproximableMap U U}
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    FinitaryProjection (sumComb a b) ∧ FinitaryProjection (prodComb a b) ∧
      FinitaryProjection (arrowComb a b) ∧
      Nonempty
        ((sum (fixedNbhd a) (fixedNbhd b) (fixedNbhd_mem_nonempty a)
            (fixedNbhd_mem_nonempty b)).Element ≃o
          {y : U.Element // (sumComb a b).toElementMap y = y}) ∧
      Nonempty
        ((prod (fixedNbhd a) (fixedNbhd b)).Element ≃o
          {y : U.Element // (prodComb a b).toElementMap y = y}) ∧
      Nonempty
        ((funSpace (fixedNbhd a) (fixedNbhd b)).Element ≃o
          {y : U.Element // (arrowComb a b).toElementMap y = y}) :=
  ⟨sumComb.finitaryProjection a b ha hb, prodComb.finitaryProjection a b ha hb,
    arrowComb.finitaryProjection a b ha hb, ⟨sumComb.elementIso a b ha hb⟩,
    ⟨prodComb.elementIso a b ha hb⟩, ⟨arrowComb.elementIso a b ha hb⟩⟩

theorem proposition_8_10 {a b : ApproximableMap U U}
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    FinitaryProjection (sumComb a b) ∧ FinitaryProjection (prodComb a b) ∧
      FinitaryProjection (arrowComb a b) ∧
      Nonempty
        ((sum (fixedNbhd a) (fixedNbhd b) (fixedNbhd_mem_nonempty a)
            (fixedNbhd_mem_nonempty b)).Element ≃o
          {y : U.Element // (sumComb a b).toElementMap y = y}) ∧
      Nonempty
        ((prod (fixedNbhd a) (fixedNbhd b)).Element ≃o
          {y : U.Element // (prodComb a b).toElementMap y = y}) ∧
      Nonempty
        ((funSpace (fixedNbhd a) (fixedNbhd b)).Element ≃o
          {y : U.Element // (arrowComb a b).toElementMap y = y}) :=
  proposition_810 ha hb

end DomainTheory.NeighbourhoodSystems
```

4.9.34 Lecture VIII exercises

EXERCISES

4.9.35 Dedekind cut projection

This peer constructs a projection whose fixed-point order is the real unit interval, using rational initial intervals as neighbourhoods, and proves that this projection is not finitary.

Definition (`R`). The rational-interval neighbourhood system has as neighbourhoods the half-open intervals $[0, r)$ with rational $0 < r \leq 1$, and master neighbourhood $[0, 1)$.

Theorem (`R_mem_iff`). A set is a neighbourhood of the rational-interval system exactly when it is $[0, r)$ for some rational r with $0 < r \leq 1$.

Proof. This is the defining family of neighbourhoods. ■

Theorem (`R_master`). The master neighbourhood is $[0, 1)$.

Proof. This is the chosen master interval. ■

Theorem (`Ico0_inj`). Positive rational endpoints determine their half-open initial intervals uniquely.

Proof. If one endpoint were smaller, that endpoint would belong to the larger interval but not the smaller one. ■

Theorem (`Ico0_inter`). The intersection of $[0, r)$ and $[0, s)$ is $[0, \min(r, s))$.

Proof. Membership requires being below both endpoints. ■

Theorem (`Ico0_le_of_subset`). If $p > 0$ and $[0, p) \subseteq [0, q)$, then $p \leq q$.

Proof. If $q < p$, positivity forced by the inclusion makes q a member of the first interval but not the second. ■

Definition (`aRel`). Two rational initial intervals are related when their endpoints lie in $(0, 1]$ and the first endpoint is strictly smaller than the second, except that the endpoint 1 is also related to itself.

Theorem (`aRel_combine`). If an endpoint is related to each of two endpoints, then it is related to their minimum.

Proof. Two strict inequalities combine into an inequality to the minimum; the exceptional equality case can occur only when all relevant endpoints are 1. ■

Definition (`a`). The relation on rational initial intervals defines an approximable endomap of the rational-interval neighbourhood system.

Theorem (`aRel_iff`). For valid positive endpoints at most 1, the corresponding intervals are related exactly when the first endpoint is smaller than the second or both endpoints equal 1.

Proof. Uniqueness of positive endpoints removes the existential interval representations. ■

Theorem (`a_rel`). The underlying relation of the approximable map is the endpoint relation just defined.

Proof. This is how the map was constructed. ■

Theorem (`Element.ext_U`). Two elements of the rational-interval system are equal if they agree on every interval $[0, r)$.

Proof. Every neighbourhood belonging to either element has this form, so agreement on these intervals gives extensional equality. ■

Definition (`mkElement`). An upward-closed predicate on positive rational endpoints at most 1, containing 1, determines the element whose neighbourhoods are the corresponding intervals $[0, r)$.

Theorem (`mkElement_mem`). A set belongs to the element built from an endpoint predicate exactly when it is $[0, r)$ for some endpoint satisfying that predicate.

Proof. This is the defining membership condition. ■

Theorem (`R_mem_Ico0`). If $[0, s)$ is a neighbourhood, then $0 < s \leq 1$.

Proof. Represent it by a valid endpoint and use uniqueness of positive endpoints. ■

Definition (`cutU`). For a real t, the cut predicate selects rational $r \leq 1$ lying strictly above $1 - t$, together with the endpoint $r = 1$.

Theorem (`cutU_le1`). Every endpoint selected by the cut predicate is at most 1.

Proof. This is part of the predicate. ■

Theorem (`cutU_one`). The endpoint 1 is selected by every cut predicate.

Proof. It is included by the exceptional endpoint clause. ■

Theorem (`cutU_pos`). If $t \leq 1$, every endpoint selected by its cut predicate is positive.

Proof. Then $1 - t \geq 0$; an endpoint strictly above it is positive, while the exceptional endpoint is 1. ■

Theorem (`cutU_up`). The cut predicate is upward closed among rational endpoints at most 1.

Proof. Increasing an endpoint preserves strict inequality above $1 - t$; if the original endpoint is 1, the bounds force the new endpoint also to be 1. ■

Definition (`cutElt`). A real number $t \leq 1$ determines the rational-interval element selected by its cut predicate.

Theorem (`cutElt_mem`). The interval $[0, r)$ belongs to the cut element for t exactly when r satisfies the cut predicate for t.

Proof. The general endpoint construction and uniqueness of positive interval endpoints give the equivalence. ■

Definition (`cutS`). For an element, its endpoint set consists of the real images of rational r for which $[0, r)$ belongs to the element.

Definition (`cutPoint`). The real point represented by an element is one minus the infimum of its endpoint set.

Theorem (`cutS_bddBelow`). Every endpoint set is bounded below by zero.

Proof. Endpoints of neighbourhood intervals are positive. ■

Theorem (`cutPoint_nonneg`). Every represented cut point is nonnegative.

Proof. The endpoint 1 belongs to every element, so the infimum of the endpoint set is at most 1. ■

Theorem (`cutS_nonempty`). Every endpoint set is nonempty.

Proof. It contains the endpoint 1 arising from the master neighbourhood. ■

Theorem (`cutPoint_le_one`). Every represented cut point is at most 1.

Proof. The nonempty endpoint set is bounded below by zero, so its infimum is nonnegative. ■

Theorem (`cutElt_cutPoint`). Every fixed point of the approximable map is the cut element determined by its represented real point.

Proof. Fixedness expresses membership of an interval through a related smaller interval. The defining property of the infimum then identifies membership with lying strictly above the endpoint infimum, with the master endpoint handled separately. ■

Theorem (`cutU_mono`). If $t \leq t'$, every endpoint selected by the cut for t is selected by the cut for t'.

Proof. Increasing t decreases the threshold $1 - t$. ■

Theorem (`le_iff_U`). One rational-interval element lies below another exactly when membership in every interval $[0, r)$ transfers from the first to the second.

Proof. These intervals comprise all neighbourhoods of the system. ■

Theorem (`cutElt_le_iff`). For $t, t' \in [0, 1]$, the cut element for t lies below that for t' exactly when $t \leq t'$.

Proof. Monotonicity of cut predicates gives one direction. If $t > t'$, density of the rationals supplies an endpoint between the two cut thresholds that belongs only to the first cut. ■

Theorem (`cutElt_injective`). On the real unit interval, equal cut elements have equal real parameters.

Proof. Equality gives both element inequalities, and the order characterization gives both inequalities between the parameters. ■

Theorem (`toElementMap_cutElt`). Every cut element with parameter at most 1 is fixed by the approximable map.

Proof. A related smaller endpoint preserves the cut inequality in the forward direction; rational density inserts a related endpoint below any nonterminal selected endpoint in the reverse direction, while endpoint 1 relates to itself. ■

Definition (`fixOrderIso`). The fixed points of the approximable map are order-isomorphic to the real interval $[0, 1]$, by sending a fixed point to its represented cut point.

Theorem (`aRel_le`). Related rational endpoints are ordered by the usual non-strict order.

Proof. Either the first is strictly smaller, or both are 1. ■

Theorem (`isRetraction_a`). The approximable map is idempotent.

Proof. Composing two strict endpoint steps gives one strict step. Conversely, density of the rationals inserts an intermediate endpoint between distinct related endpoints, while the terminal endpoint 1 factors through itself. ■

Theorem (`isProjection_a`). The approximable map is a projection.

Proof. It is a retraction, and every related pair of intervals is ordered by inclusion because their endpoints are ordered. ■

Theorem (`not_isCompactElt_pos`). Under any order isomorphism from the real unit interval to a neighbourhood-system domain, the image of every positive point is not compact.

Proof. The positive point is the supremum of the directed family of smaller nonnegative points, but no member of that family lies above it; an order isomorphism preserves this least-upper-bound obstruction. ■

Theorem (`not_isFinitary_a`). The projection is not finitary.

Proof. If its fixed points formed a finitary domain, every principal compact approximation of the top fixed point would correspond, under the unit-interval isomorphism, to a compact point. No positive point is compact, so all such approximations would be the bottom point; their directed supremum would then make top equal bottom, a contradiction. ■

`DedekindCutProjection.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Examples_8_4
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint
public import Mathlib.Algebra.BigOperators.Group.Finset.Defs
public import Mathlib.Algebra.Order.Archimedean.Real.Basic

public import Mathlib.Combinatorics.Colex

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.DedekindCutProjection

open NeighborhoodSystem ApproximableMap

def R : NeighborhoodSystem ? :=
  NeighborhoodSystem.ofNestedOrDisjoint
    (mem := fun X => ∃ r : ?, 0 < r ∧ r ≤ 1 ∧ X = Set.Ico (0 : ?) r)
    (master := Set.Ico (0 : ?) 1)
    (master_nonempty := ⟨0, by norm_num⟩)
    (master_mem := ⟨1, one_pos, le_refl 1, rfl⟩)
    (hnd := by
      rintro X Y ⟨r, hr0, hr1, rfl⟩ ⟨s, hs0, hs1, rfl⟩
      rcases le_total r s with h | h
      · exact Or.inl (Set.Ico_subset_Ico le_rfl h)
      · exact Or.inr (Or.inl (Set.Ico_subset_Ico le_rfl h)))
    (sub_master := by
      rintro X ⟨r, hr0, hr1, rfl⟩
      exact Set.Ico_subset_Ico le_rfl hr1)
```

```
@[simp] theorem R_mem_iff {X : Set ?} :
    R.mem X ↔ ∃ r : ?, 0 < r ∧ r ≤ 1 ∧ X = Set.Ico (0 : ?) r := Iff.rfl

@[simp] theorem R_master : R.master = Set.Ico (0 : ?) 1 := rfl

theorem Ico0_inj {r s : ?} (hr : 0 < r) (hs : 0 < s)
    (h : Set.Ico (0 : ?) r = Set.Ico (0 : ?) s) : r = s := by
  rcases lt_trichotomy r s with hlt | heq | hgt
  · have hmem : r ∈ Set.Ico (0 : ?) s := ⟨hr.le, hlt⟩
    rw [← h] at hmem
    exact absurd hmem.2 (lt_irrefl r)
  · exact heq
  · have hmem : s ∈ Set.Ico (0 : ?) r := ⟨hs.le, hgt⟩
    rw [h] at hmem
    exact absurd hmem.2 (lt_irrefl s)

theorem Ico0_inter (r s : ?) :
    Set.Ico (0 : ?) r ∩ Set.Ico (0 : ?) s = Set.Ico (0 : ?) (min r s) := by
  rw [Set.Ico_inter_Ico, sup_idem]

theorem Ico0_le_of_subset {p q : ?} (hp : 0 < p)
    (hsub : Set.Ico (0 : ?) p ⊆ Set.Ico (0 : ?) q) : p ≤ q := by
  have hq : (0 : ?) < q := (hsub ⟨le_refl 0, hp⟩).2
  by_contra hlt
  rw [not_le] at hlt
  exact absurd (hsub ⟨hq.le, hlt⟩).2 (lt_irrefl q)

def aRel (X Y : Set ?) : Prop :=
  ∃ r s : ?, 0 < r ∧ r ≤ 1 ∧ 0 < s ∧ s ≤ 1 ∧
    X = Set.Ico (0 : ?) r ∧ Y = Set.Ico (0 : ?) s ∧ (r < s ∨ (r = 1 ∧ s = 1))

theorem aRel_combine {r s s' : ?} (hs1 : s ≤ 1) (hs1' : s' ≤ 1)
    (h1 : r < s ∨ (r = 1 ∧ s = 1)) (h2 : r < s' ∨ (r = 1 ∧ s' = 1)) :
    r < min s s' ∨ (r = 1 ∧ min s s' = 1) := by
  rcases h1 with h1 | ⟨rfl, rfl⟩
  · rcases h2 with h2 | ⟨rfl, rfl⟩
    · exact Or.inl (lt_min h1 h2)
    · exact absurd (h1.trans_le hs1) (lt_irrefl _)
  · rcases h2 with h2 | ⟨-, rfl⟩
    · exact absurd (h2.trans_le hs1') (lt_irrefl _)
    · exact Or.inr ⟨rfl, min_self 1⟩

def a : ApproximableMap R R where
  rel := aRel
  rel_dom := fun ⟨r, _, hr0, hr1, _, _, hXr, _, _⟩ => ⟨r, hr0, hr1, hXr⟩
  rel_cod := fun ⟨_, s, _, _, hs0, hs1, _, hYs, _⟩ => ⟨s, hs0, hs1, hYs⟩
  master_rel := ⟨1, 1, one_pos, le_rfl, one_pos, le_rfl, rfl, rfl, Or.inr ⟨rfl, rfl⟩⟩
  inter_right := by
    rintro X Y Y' ⟨r, s, hr0, hr1, hs0, hs1, rfl, rfl, hcond⟩
      ⟨r', s', hr0', hr1', hs0', hs1', hXr', rfl, hcond'⟩
    have hrr' : r = r' := Ico0_inj hr0 hr0' hXr'
    subst hrr'
    rw [Set.Ico_inter_Ico, sup_idem]
    exact ⟨r, min s s', hr0, hr1, lt_min hs0 hs0', (min_le_left s s').trans hs1, rfl,
    rfl,
```

```
      aRel_combine hs1 hs1' hcond hcond'⟩
  mono := by
    rintro X X' Y Y' ⟨r, s, hr0, hr1, hs0, hs1, rfl, rfl, hcond⟩ hX'X hYY'
      ⟨r', hr0', hr1', rfl⟩ ⟨s', hs0', hs1', rfl⟩
    have hr'r : r' ≤ r := Ico0_le_of_subset hr0' hX'X
    have hss' : s ≤ s' := Ico0_le_of_subset hs0 hYY'
    refine ⟨r', s', hr0', hr1', hs0', hs1', rfl, rfl, ?_⟩
    rcases hcond with hlt | ⟨rfl, rfl⟩
    · exact Or.inl (hr'r.trans_lt (hlt.trans_le hss'))
    · rcases hr1'.lt_or_eq with h | h
      · exact Or.inl (h.trans_le hss')
      · exact Or.inr ⟨h, le_antisymm hs1' hss'⟩

theorem aRel_iff {r s : ?} (hr0 : 0 < r) (hr1 : r ≤ 1) (hs0 : 0 < s) (hs1 : s ≤ 1) :
    aRel (Set.Ico (0 : ?) r) (Set.Ico (0 : ?) s) ↔ (r < s ∨ (r = 1 ∧ s = 1)) := by
  constructor
  · rintro ⟨r', s', hr0', hr1', hs0', hs1', hXr, hXs, hcond⟩
    rw [Ico0_inj hr0 hr0' hXr, Ico0_inj hs0 hs0' hXs]
    exact hcond
  · intro h
    exact ⟨r, s, hr0, hr1, hs0, hs1, rfl, rfl, h⟩

@[simp] theorem a_rel {X Y : Set ?} : a.rel X Y ↔ aRel X Y := Iff.rfl

theorem Element.ext_U {x y : R.Element} (h : ∀ r : ?, x.mem (Set.Ico 0 r) ↔ y.mem (Set
    .Ico 0 r)) :
    x = y := by
  apply NeighborhoodSystem.Element.ext
  intro X
  constructor
  · intro hx
    obtain ⟨r, _, _, rfl⟩ := x.sub hx
    exact (h r).mp hx
  · intro hy
    obtain ⟨r, _, _, rfl⟩ := y.sub hy
    exact (h r).mpr hy

def mkElement (U : ? → Prop) (hU1 : U 1) (hUpos : ∀ r, U r → 0 < r) (hUle1 : ∀ r, U r
    → r ≤ 1)
    (hUup : ∀ r s, U r → r ≤ s → s ≤ 1 → U s) : R.Element where
  mem X := ∃ r, U r ∧ X = Set.Ico (0 : ?) r
  sub := fun ⟨r, hr, hXr⟩ => ⟨r, hUpos r hr, hUle1 r hr, hXr⟩
  master_mem := ⟨1, hU1, rfl⟩
  inter_mem := by
    rintro X Y ⟨r, hr, rfl⟩ ⟨s, hs, rfl⟩
    rw [Ico0_inter]
    rcases le_total r s with h | h
    · exact ⟨r, hr, by rw [min_eq_left h]⟩
    · exact ⟨s, hs, by rw [min_eq_right h]⟩
  up_mem := by
    rintro X Y ⟨r, hr, rfl⟩ hY hsub
    obtain ⟨s, hs0, hs1, rfl⟩ := hY
    exact ⟨s, hUup r s hr (Ico0_le_of_subset (hUpos r hr) hsub) hs1, rfl⟩

@[simp] theorem mkElement_mem {U hU1 hUpos hUle1 hUup} {X : Set ?} :
    (mkElement U hU1 hUpos hUle1 hUup).mem X ↔ ∃ r, U r ∧ X = Set.Ico (0 : ?) r := Iff
```

```
    .rfl

theorem R_mem_Ico0 {s : ?} (h : R.mem (Set.Ico (0 : ?) s)) : 0 < s ∧ s ≤ 1 := by
  obtain ⟨r, hr0, hr1, heq⟩ := h
  have hsne : (Set.Ico (0 : ?) s).Nonempty := heq ▸ ⟨0, le_refl 0, hr0⟩
  obtain ⟨x, hx0, hxs⟩ := hsne
  have hs0 : 0 < s := lt_of_le_of_lt hx0 hxs
  have hsr : s = r := Ico0_inj hs0 hr0 heq
  exact ⟨hs0, hsr ▸ hr1⟩

def cutU (t : ?) (r : ?) : Prop := r ≤ 1 ∧ ((1 - t : ?) < (r : ?) ∨ r = 1)

theorem cutU_le1 {t : ?} {r : ?} (h : cutU t r) : r ≤ 1 := h.1

theorem cutU_one {t : ?} : cutU t 1 := ⟨le_rfl, Or.inr rfl⟩

theorem cutU_pos {t : ?} (ht1 : t ≤ 1) {r : ?} (h : cutU t r) : 0 < r := by
  rcases h.2 with h2 | rfl
  · have hc : (0 : ?) ≤ 1 - t := by linarith
    exact_mod_cast lt_of_le_of_lt hc h2
  · norm_num

theorem cutU_up {r s : ?} {t : ?} (h : cutU t r) (hrs : r ≤ s) (hs1 : s ≤ 1) : cutU t
    s := by
  refine ⟨hs1, ?_⟩
  rcases h.2 with h2 | rfl
  · left
    have hcast : (r : ?) ≤ (s : ?) := by exact_mod_cast hrs
    linarith
  · exact Or.inr (le_antisymm hs1 hrs)

def cutElt (t : ?) (ht1 : t ≤ 1) : R.Element :=
  mkElement (cutU t) cutU_one (fun _ h => cutU_pos ht1 h) (fun _ h => cutU_le1 h)
    (fun _ _ h hrs hs1 => cutU_up h hrs hs1)

@[simp] theorem cutElt_mem {t : ?} {ht1 : t ≤ 1} {r : ?} :
    (cutElt t ht1).mem (Set.Ico 0 r) ↔ cutU t r := by
  simp only [cutElt, mkElement_mem]
  constructor
  · rintro ⟨r', hr', heq⟩
    have hr'0 := cutU_pos ht1 hr'
    have hne : (Set.Ico (0 : ?) r).Nonempty := heq ▸ ⟨0, le_refl 0, hr'0⟩
    obtain ⟨x, hx0, hxr⟩ := hne
    have hr0 : 0 < r := lt_of_le_of_lt hx0 hxr
    rwa [Ico0_inj hr0 hr'0 heq]
  · exact fun h => ⟨r, h, rfl⟩

def cutS (x : R.Element) : Set ? := (fun r : ? => (r : ?)) '' {r | x.mem (Set.Ico 0 r)
    }

noncomputable def cutPoint (x : R.Element) : ? := 1 - sInf (cutS x)

theorem cutS_bddBelow (x : R.Element) : BddBelow (cutS x) := by
  refine ⟨0, ?_⟩
  rintro y ⟨r, hr, rfl⟩
  dsimp only
```

```
  exact_mod_cast (R_mem_Ico0 (x.sub hr)).1.le

theorem cutPoint_nonneg (x : R.Element) : 0 ≤ cutPoint x := by
  have hmem1 : x.mem (Set.Ico (0 : ?) 1) := by rw [← R_master]; exact x.master_mem
  have hc1 : sInf (cutS x) ≤ 1 := csInf_le (cutS_bddBelow x) ⟨1, hmem1, by norm_num⟩
  unfold cutPoint; linarith

theorem cutS_nonempty (x : R.Element) : (cutS x).Nonempty :=
  ⟨1, 1, show x.mem (Set.Ico (0 : ?) 1) by rw [← R_master]; exact x.master_mem, by
    norm_num⟩

theorem cutPoint_le_one (x : R.Element) : cutPoint x ≤ 1 := by
  have hc0 : (0:?) ≤ sInf (cutS x) := le_csInf (cutS_nonempty x)
    (by rintro y ⟨r, hr, rfl⟩; dsimp only; exact_mod_cast (R_mem_Ico0 (x.sub hr)).1.le)
  unfold cutPoint; linarith

theorem cutElt_cutPoint {x : R.Element} (hfix : a.toElementMap x = x) :
    cutElt (cutPoint x) (cutPoint_le_one x) = x := by
  have hSne := cutS_nonempty x
  have hSbdd := cutS_bddBelow x
  apply Element.ext_U
  intro r
  rw [cutElt_mem]
  have hcU : cutU (cutPoint x) r ↔ (r ≤ 1 ∧ (sInf (cutS x) < (r : ?) ∨ r = 1)) := by
    unfold cutPoint
    constructor
    · rintro ⟨h1, h2 | h2⟩
      · exact ⟨h1, Or.inl (by linarith)⟩
      · exact ⟨h1, Or.inr h2⟩
    · rintro ⟨h1, h2 | h2⟩
      · exact ⟨h1, Or.inl (by linarith)⟩
      · exact ⟨h1, Or.inr h2⟩
  rw [hcU, Iff.comm]
  have hopen : ∀ Y, x.mem Y ↔ ∃ X, x.mem X ∧ a.rel X Y := by
    intro Y
    conv_lhs => rw [← hfix]
    exact mem_toElementMap a x
  constructor
  · intro hxr
    refine ⟨(R_mem_Ico0 (x.sub hxr)).2, ?_⟩
    by_cases hr1 : r = 1
    · exact Or.inr hr1
    · refine Or.inl ?_
      obtain ⟨X, hXmem, hrel⟩ := (hopen (Set.Ico 0 r)).mp hxr
      obtain ⟨r'', hr''0, hr''1, rfl⟩ := x.sub hXmem
      rw [a_rel, aRel_iff hr''0 hr''1 (R_mem_Ico0 (x.sub hxr)).1 (R_mem_Ico0 (x.sub
    hxr)).2] at hrel
      rcases hrel with hlt | ⟨-, hreq⟩
      · calc sInf (cutS x) ≤ (r'' : ?) := csInf_le hSbdd ⟨r'', hXmem, rfl⟩
          _ < (r : ?) := by exact_mod_cast hlt
      · exact absurd hreq hr1
  · rintro ⟨hr1, hc | rfl⟩
    · obtain ⟨q, ⟨r0, hr0mem, rfl⟩, hqlt⟩ := exists_lt_of_csInf_lt hSne hc
      dsimp only at hqlt
      have hr0r : r0 < r := by exact_mod_cast hqlt
      have hr0pos : 0 < r0 := (R_mem_Ico0 (x.sub hr0mem)).1
```

```
      exact x.up_mem hr0mem ⟨r, hr0pos.trans hr0r, hr1, rfl⟩ (Set.Ico_subset_Ico le_rfl
     hr0r.le)
    · rw [← R_master]; exact x.master_mem

theorem cutU_mono {t t' : ?} (htt' : t ≤ t') {r : ?} (h : cutU t r) : cutU t' r := by
  refine ⟨h.1, ?_⟩
  rcases h.2 with hc | rfl
  · exact Or.inl (by linarith)
  · exact Or.inr rfl

theorem le_iff_U {x y : R.Element} :
    x ≤ y ↔ ∀ r : ?, x.mem (Set.Ico 0 r) → y.mem (Set.Ico 0 r) :=
  ⟨fun h r => h (Set.Ico 0 r), fun h X hX => by
    obtain ⟨r, _, _, rfl⟩ := x.sub hX; exact h r hX⟩

theorem cutElt_le_iff {t t' : ?} (_ht0 : 0 ≤ t) (ht1 : t ≤ 1) (ht0' : 0 ≤ t') (ht1' :
     t' ≤ 1) :
    cutElt t ht1 ≤ cutElt t' ht1' ↔ t ≤ t' := by
  rw [le_iff_U]
  simp only [cutElt_mem]
  constructor
  · intro h
    by_contra hlt
    rw [not_le] at hlt
    obtain ⟨r, hr1, hr2⟩ := exists_rat_btwn (show (1 - t : ?) < 1 - t' by linarith)
    have hrle1 : r ≤ 1 := by
      have : (r : ?) < 1 := by linarith
      exact_mod_cast this.le
    have hcutt' : cutU t' r := h r ⟨hrle1, Or.inl hr1⟩
    rcases hcutt'.2 with hc' | hreq
    · linarith
    · rw [hreq] at hr2; norm_num at hr2; linarith
  · exact fun htt' r hr => cutU_mono htt' hr

theorem cutElt_injective {t t' : ?} (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (ht0' : 0 ≤ t') (ht1'
     : t' ≤ 1)
    (h : cutElt t ht1 = cutElt t' ht1') : t = t' :=
  le_antisymm ((cutElt_le_iff ht0 ht1 ht0' ht1').mp h.le) ((cutElt_le_iff ht0' ht1'
    ht0 ht1).mp h.ge)

theorem toElementMap_cutElt {t : ?} (ht1 : t ≤ 1) :
    a.toElementMap (cutElt t ht1) = cutElt t ht1 := by
  apply Element.ext_U
  intro s
  simp only [mem_toElementMap, a_rel, cutElt_mem]
  constructor
  · rintro ⟨X, hX, hrel⟩
    obtain ⟨r, hUr, rfl⟩ := hX
    have hs01 := R_mem_Ico0 (a.rel_cod (a_rel.mpr hrel))
    rw [aRel_iff (cutU_pos ht1 hUr) (cutU_le1 hUr) hs01.1 hs01.2] at hrel
    rcases hrel with hlt | ⟨hr1, hs1⟩
    · rcases hUr.2 with hc | hr1
      · exact ⟨hs01.2, Or.inl (lt_trans hc (by exact_mod_cast hlt))⟩
      · exact absurd (hr1 ▸ hlt : (1:?) < s) (not_lt.mpr hs01.2)
    · exact ⟨hs01.2, Or.inr hs1⟩
  · intro hs
```

```
    rcases hs.2 with hc | rfl
    · obtain ⟨r, hr1, hr2⟩ := exists_rat_btwn hc
      have hr0 : 0 < r := by
        have : (0:?) ≤ 1 - t := by linarith
        exact_mod_cast lt_of_le_of_lt this hr1
      have hrle1 : r ≤ 1 := by
        have : (r:?) < (s:?) := hr2
        have hrs : r < s := by exact_mod_cast this
        exact hrs.le.trans hs.1
      exact ⟨Set.Ico 0 r, ⟨r, ⟨hrle1, Or.inl hr1⟩, rfl⟩, r, s, hr0, hrle1, cutU_pos ht1
    hs,
        cutU_le1 hs, rfl, rfl, Or.inl (by exact_mod_cast hr2)⟩
    · exact ⟨Set.Ico 0 1, ⟨1, cutU_one, rfl⟩, 1, 1, one_pos, le_rfl, one_pos, le_rfl,
    rfl, rfl,
        Or.inr ⟨rfl, rfl⟩⟩

noncomputable def fixOrderIso : {y : R.Element // a.toElementMap y = y} ≃o Set.Icc (0
    : ?) 1 where
  toFun y := ⟨cutPoint y.1, cutPoint_nonneg y.1, cutPoint_le_one y.1⟩
  invFun t := ⟨cutElt t.1 t.2.2, toElementMap_cutElt t.2.2⟩
  left_inv y := Subtype.ext (cutElt_cutPoint y.2)
  right_inv t := Subtype.ext (cutElt_injective (cutPoint_nonneg _) (cutPoint_le_one _)
     t.2.1 t.2.2
    (cutElt_cutPoint (toElementMap_cutElt t.2.2)))
  map_rel_iff' := by
    intro y y'
    show cutPoint y.1 ≤ cutPoint y'.1 ↔ y.1 ≤ y'.1
    constructor
    · intro h
      have h' := (cutElt_le_iff (cutPoint_nonneg y.1) (cutPoint_le_one y.1) (
    cutPoint_nonneg y'.1)
        (cutPoint_le_one y'.1)).mpr h
      rwa [cutElt_cutPoint y.2, cutElt_cutPoint y'.2] at h'
    · intro h
      rw [← cutElt_cutPoint y.2, ← cutElt_cutPoint y'.2] at h
      exact (cutElt_le_iff (cutPoint_nonneg y.1) (cutPoint_le_one y.1) (
    cutPoint_nonneg y'.1)
        (cutPoint_le_one y'.1)).mp h

theorem aRel_le {r s : ?} (h : r < s ∨ (r = 1 ∧ s = 1)) : r ≤ s := by
  rcases h with h | ⟨rfl, rfl⟩
  · exact h.le
  · exact le_rfl

theorem isRetraction_a : Retraction a := by
  constructor
  apply ApproximableMap.ext
  intro X Z
  constructor
  · rintro ⟨Y, ⟨r, s, hr0, hr1, hs0, hs1, rfl, rfl, hcond1⟩,
      ⟨s', t, hs0', hs1', ht0, ht1, hYs', rfl, hcond2⟩⟩
    have hss' : s = s' := Ico0_inj hs0 hs0' hYs'
    subst hss'
    refine ⟨r, t, hr0, hr1, ht0, ht1, rfl, rfl, ?_⟩
    rcases hcond1 with h1 | ⟨rfl, rfl⟩
    · rcases hcond2 with h2 | ⟨rfl, rfl⟩
```

```
      · exact Or.inl (h1.trans h2)
      · exact Or.inl h1
    · rcases hcond2 with h2 | ⟨-, rfl⟩
      · exact absurd (h2.trans_le ht1) (lt_irrefl _)
      · exact Or.inr ⟨rfl, rfl⟩
  · rintro ⟨r, t, hr0, hr1, ht0, ht1, rfl, rfl, hcond⟩
    rcases hcond with h | ⟨rfl, rfl⟩
    · obtain ⟨s, hrs, hst⟩ := exists_between h
      have hs0 : 0 < s := hr0.trans hrs
      have hs1 : s ≤ 1 := (hst.trans_le ht1).le
      exact ⟨Set.Ico 0 s, ⟨r, s, hr0, hr1, hs0, hs1, rfl, rfl, Or.inl hrs⟩,
        ⟨s, t, hs0, hs1, ht0, ht1, rfl, rfl, Or.inl hst⟩⟩
    · exact ⟨Set.Ico 0 1, ⟨1, 1, one_pos, le_rfl, one_pos, le_rfl, rfl, rfl, Or.inr ⟨rfl
    , rfl⟩⟩,
        ⟨1, 1, one_pos, le_rfl, one_pos, le_rfl, rfl, rfl, Or.inr ⟨rfl, rfl⟩⟩⟩

theorem isProjection_a : IsProjection a := by
  refine ⟨isRetraction_a, ?_⟩
  rintro X Y ⟨r, s, hr0, hr1, hs0, hs1, rfl, rfl, hcond⟩
  exact ⟨⟨r, hr0, hr1, rfl⟩, ⟨s, hs0, hs1, rfl⟩, Set.Ico_subset_Ico le_rfl (aRel_le
    hcond)⟩

theorem not_isCompactElt_pos {β : Type} {F : NeighborhoodSystem β}
    (g : Set.Icc (0 : ?) 1 ≃o F.Element) {t : ?} (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (htpos :
     0 < t) :
    ¬ IsCompactElt (g ⟨t, ht0, ht1⟩) := by
  intro hcompact
  set S : Set (Set.Icc (0 : ?) 1) := {x | x.1 ∈ Set.Ico (0 : ?) t} with hSdef
  have hmem0 : (⟨0, le_refl 0, zero_le_one⟩ : Set.Icc (0 : ?) 1) ∈ S := ⟨le_refl 0,
    htpos⟩
  have : Nonempty S := ⟨⟨_, hmem0⟩⟩
  set fam : S → F.Element := fun x => g x.1 with hfamdef
  have hdir : ∀ i j : S, ∃ k : S, fam i ≤ fam k ∧ fam j ≤ fam k := by
    rintro ⟨x, hx⟩ ⟨y, hy⟩
    refine ⟨⟨max x.1 y.1, le_max_of_le_left x.2.1, (max_lt hx.2 hy.2).le.trans ht1⟩,
      ⟨le_max_of_le_left x.2.1, max_lt hx.2 hy.2⟩⟩, ?_, ?_⟩
    · exact g.monotone (le_max_left x.1 y.1)
    · exact g.monotone (le_max_right x.1 y.1)
  have hSisLUB : IsLUB S (⟨t, ht0, ht1⟩ : Set.Icc (0 : ?) 1) := by
    constructor
    · rintro x hx
      exact hx.2.le
    · intro ub hub
      exact (isLUB_Ico htpos).2 (fun y hy =>
        hub (show (⟨y, hy.1, hy.2.le.trans ht1⟩ : Set.Icc (0 : ?) 1) ∈ S from hy))
  have hgSisLUB : IsLUB (g '' S) (g ⟨t, ht0, ht1⟩) := g.isLUB_image'.mpr hSisLUB
  have hrange : Set.range fam = g '' S := by
    ext z
    constructor
    · rintro ⟨⟨x, hx⟩, rfl⟩; exact ⟨x, hx, rfl⟩
    · rintro ⟨x, hx, rfl⟩; exact ⟨⟨x, hx⟩, rfl⟩
  have hisLUBfam : IsLUB (Set.range fam) (g ⟨t, ht0, ht1⟩) := hrange ▸ hgSisLUB
  have hisLUBiSup : IsLUB (Set.range fam) (iSupDirected fam hdir) :=
    ⟨fun _ ⟨i, hi⟩ => hi ▸ le_iSupDirected fam hdir i,
      fun y hy => iSupDirected_le fam hdir (fun i => hy ⟨i, rfl⟩)⟩
  have heq : iSupDirected fam hdir = g ⟨t, ht0, ht1⟩ := hisLUBiSup.unique hisLUBfam
```

```
  obtain ⟨i, hi⟩ := hcompact fam hdir (le_of_eq heq.symm)

  have hle : (⟨t, ht0, ht1⟩ : Set.Icc (0 : ?) 1) ≤ i.1 := g.le_iff_le.mp hi
  exact absurd hle (not_le.mpr i.2.2)

theorem not_isFinitary_a : ¬ IsFinitary a := by
  rintro ⟨β, F, ⟨e⟩⟩
  set g : Set.Icc (0 : ?) 1 ≃o F.Element := fixOrderIso.symm.trans e with hgdef
  set x : F.Element := g ⟨1, zero_le_one, le_refl 1⟩ with hxdef
  have hcompact_bot : ∀ i : {X : Set β // x.mem X}, F.principal (x.sub i.2) = g ⟨0,
    le_refl 0, zero_le_one⟩ := by
    intro i
    by_contra hne
    have hc0 : g.symm (F.principal (x.sub i.2)) = ⟨0, le_refl 0, zero_le_one⟩ := fun h
    => hne (by
      rw [← h, OrderIso.apply_symm_apply])
    have ht0 : (0:?) ≤ (g.symm (F.principal (x.sub i.2))).1 := (g.symm (F.principal (x
    .sub i.2))).2.1
    have ht1 : (g.symm (F.principal (x.sub i.2))).1 ≤ 1 := (g.symm (F.principal (x.sub
     i.2))).2.2
    have htpos : 0 < (g.symm (F.principal (x.sub i.2))).1 := by
      rcases ht0.lt_or_eq with h | h
      · exact h
      · exact absurd (Subtype.ext h.symm) hc0
    have hsub : (⟨(g.symm (F.principal (x.sub i.2))).1, ht0, ht1⟩ : Set.Icc (0 : ?) 1)
        = g.symm (F.principal (x.sub i.2)) := Subtype.ext rfl
    have heq2 : F.principal (x.sub i.2) = g ⟨(g.symm (F.principal (x.sub i.2))).1, ht0,
     ht1⟩ := by
      rw [hsub, OrderIso.apply_symm_apply]
    exact not_isCompactElt_pos g ht0 ht1 htpos (heq2 ▸ principal_isCompactElt (x.sub i
    .2))
  have hxeq : x = g ⟨0, le_refl 0, zero_le_one⟩ := by
    conv_lhs => rw [eq_iSupDirected_principal x]
    apply le_antisymm
    · apply iSupDirected_le
      intro i
      exact le_of_eq (hcompact_bot i)
    · obtain ⟨i0⟩ := (instNonemptyMemSubtype x)
      have hstep : g ⟨0, le_refl 0, zero_le_one⟩ ≤
          iSupDirected (fun j : {X : Set β // x.mem X} => F.principal (x.sub j.2))
            (principalFamily_directed x) := by
        rw [← hcompact_bot i0]
        exact le_iSupDirected (fun j : {X : Set β // x.mem X} => F.principal (x.sub j
    .2))
          (principalFamily_directed x) i0
      exact hstep
  have : (⟨1, zero_le_one, le_refl 1⟩ : Set.Icc (0:?) 1) = ⟨0, le_refl 0, zero_le_one⟩
    :=
    g.injective hxeq
  exact absurd (congrArg Subtype.val this) one_ne_zero

end DomainTheory.NeighbourhoodSystems.DedekindCutProjection
```

4.9.36 Exercise 8.11 — retraction on intervals

Let $\mathbb{Q}$ be the set of rational numbers and define a neighbourhood system by the equation

$$R = \{[0, r) \mid r \in \mathbb{Q} \text{ and } 0 < r \leq 1\}.$$

Show that the following defines an approximable map $a : R \to R$:

$$[0, r)\ a\ [0, s) \quad \text{iff} \quad r < s \text{ or } r = s = 1.$$

Show in addition that a is a projection where the fixed-point set of a is in a one-one correspondence with the *real* numbers between 0 and 1 inclusive. (Hint: Recall Dedekind cuts and show $\subseteq$ matches $\leq$.) Conclude that a is *NOT* finitary. (Hint: Aside from $\bot$ there are no finite elements for $\{x \mid x = a(x)\}$.)

The development is with the Dedekind cut projection peer.

Theorem (`example_811`). On the rational interval system, Scott's map is a non-finitary projection.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_11.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    DedekindCutProjection

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

theorem example_811 :
    IsProjection DedekindCutProjection.a ∧ Retraction DedekindCutProjection.a ∧ ¬
    IsFinitary DedekindCutProjection.a :=
  ⟨DedekindCutProjection.isProjection_a, DedekindCutProjection.isRetraction_a,
    DedekindCutProjection.not_isFinitary_a⟩
```

4.9.37 Effective isomorphism U and V

This peer completes the effective back-and-forth construction between the computable presentations of Scott's universal domains U and V.

Definition (`effectiveIso812_UV`). The computable presentations of U and V are effectively isomorphic.

Proof. The effective back-and-forth theorem applies because both presentations have computable difference and union operations, positive and difference-closed neighbourhood structures, no minimal non-master neighbourhoods, computable splitting operations, nonempty master neighbourhoods, and compatible distinguished initial elements. ■

`effectiveIso812_UV.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.Vunion
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    effectiveIso812d
```

```
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import Mathlib.Combinatorics.Colex
public import Mathlib.Data.Nat.Bitwise
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open Domain.Recursive NeighborhoodSystem EffectiveIso

noncomputable def effectiveIso812_UV : EffectiveIso UComputablePresentation
    VComputablePresentation :=
  effectiveIso812d UComputablePresentation VComputablePresentation U_isComputableDiff
    V.isComputableDiff splitX812e isComputableSplit.?812e? splitX812f
    isComputableSplit.?812f?
    U.isPositive U.diffClosed U.noMinimal hxSplit812e V.isPositive V.diffClosed V.
    noMinimal
    hySplit812f U_master_nonempty V.master_nonempty U_isComputableUnion V.
    isComputableUnion UX_zero
    VX_zero

end DomainTheory.NeighbourhoodSystems
```

4.9.38 Exercise 8.12 — integer presentation of U

NeighbourhoodSystems/RetractsOfTheUniversalDomain/Exercise_8_12.lean

@import "Exercise_8_12.lean" **NeighbourhoodSystems/RetractsOfTheUniversalDomain/delta-Pair.lean**

@import "deltaPair.lean" **NeighbourhoodSystems/RetractsOfTheUniversalDomain/V.lean**

@import "V.lean" **NeighbourhoodSystems/RetractsOfTheUniversalDomain/VX.lean**

@import "VX.lean" **NeighbourhoodSystems/RetractsOfTheUniversalDomain/Vinter.lean**

@import "Vinter.lean" **NeighbourhoodSystems/RetractsOfTheUniversalDomain/minLevel.lean**

@import "minLevel.lean" **NeighbourhoodSystems/RetractsOfTheUniversalDomain/splitV.lean**

@import "splitV.lean" **NeighbourhoodSystems/RetractsOfTheUniversalDomain/VComputablePresentation.lean**

@import "VComputablePresentation.lean" **NeighbourhoodSystems/RetractsOfTheUniversalDomain/IsComputableSplit.lean**

@import "IsComputableSplit.lean" **EXERCISE 8.12.** Generalize the notation $2X + 1$ for subsets $X \subseteq \mathbb{N}$ to sets of the form

$$2^k X + \ell, \quad \text{where } \ell < 2^k.$$

Let V be the non-empty finite unions of sets $2^k\mathbb{N} + \ell$. Show that $U \cong V$ and that the isomorphism is effective, thus obtaining another presentation of U.

The development is with the Effective isomorphism U and V peer.

Theorem (`example_812`). The integer interval presentation is effectively given and effectively isomorphic to U.

Proof. By assembling the peer lemmas named in the packaging module. ■

Exercise_8_12.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    effectiveIso812_UV
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem EffectiveIso

theorem example_812 :
    V.IsEffectivelyGiven ∧
      EffectivelyIsomorphic UComputablePresentation VComputablePresentation :=
  ⟨V.isEffectivelyGiven, ⟨effectiveIso812_UV⟩⟩
```

4.9.39 Boolean-algebra filters

This peer realizes the countable atomless Boolean algebra through binary-coordinate subsets of the natural numbers, relates its neighbourhood system to the universal domain, and connects Boolean entailment with inclusion of truth sets.

Theorem (`exercise813a`). The universal neighbourhood domain is domain-isomorphic to the domain built from finite binary level sets.

Proof. Use the established effective isomorphism between the two neighbourhood systems and forget the extra effectiveness data. ■

Theorem (`example_813_a`). The universal neighbourhood domain and the binary level-set domain are domain-isomorphic.

Proof. This is the preceding isomorphism. ■

Theorem (`generator_nonempty`). Every binary-coordinate generator is nonempty.

Proof. The number 2^i has its i-th binary digit set. ■

Theorem (`generators_independent`). For any finite set of coordinates and any prescribed Boolean values on them, the intersection choosing each generator or its complement accordingly is nonempty.

Proof. Form the bit mask of precisely those prescribed coordinates whose value is true. Its bits satisfy all the choices. ■

Theorem (`levelSet_compl`). The complement of a binary level set of period 2^k is the level set whose residue mask selects exactly the previously unselected residues.

Proof. A number lies in the complement exactly when the bit for its residue modulo 2^k is absent from the original mask. ■

Theorem (`V.compl_mem`). The complement of a neighbourhood in the binary level-set system is either empty or again a neighbourhood.

Proof. Rewrite the complement as a level set and distinguish whether that level set is empty. ■

Theorem (`testBit_mod_two_pow`). The i-th bit of n modulo 2^k equals the i-th bit of n when $i < k$, and is false otherwise.

Proof. Division by 2^k separates the low k binary digits from all higher digits. ■

Theorem (`affine_eq_biInter`). For a residue $\ell < 2^k$, its congruence class modulo 2^k is the intersection, over the first k coordinates, of the generator or its complement according to the corresponding bit of ℓ.

Proof. Two numbers below 2^k are equal exactly when their first k bits agree, and reduction modulo 2^k retains precisely those bits. ■

Theorem (`generatedBy_affine`). Every congruence class modulo a power of two belongs to the Boolean algebra generated by the binary-coordinate generators.

Proof. Use its finite-intersection description and close the generated family under complements and intersections. ■

Theorem (`generatedBy_biUnion_affine`). A finite union of congruence classes modulo 2^k, all with valid residues, belongs to the generated Boolean algebra.

Proof. Induct over the finite set of residues, using generation of each class and closure under union. ■

Theorem (`generatedBy_levelSet`). Every binary level set belongs to the Boolean algebra generated by the coordinate generators.

Proof. A level set is the finite union of the congruence classes selected by its mask. ■

Theorem (`testBit_mod_two_pow_succ`). Reducing a number modulo 2^{i+1} preserves its i-th bit.

Proof. The coordinate i lies below $i + 1$, so the general reduction formula applies. ■

Theorem (`generator_eq_levelSet`). Each coordinate generator is a level set of period 2^{i+1}, selecting precisely those residues whose i-th bit is one.

Proof. Reduction modulo 2^{i+1} preserves the relevant bit. ■

Theorem (`V.mem_generator`). Every coordinate generator is a neighbourhood of the binary level-set system.

Proof. Express it as a nonempty level set. ■

Theorem (`generatedBy_generator_imp`). Every set in the Boolean algebra generated by the coordinate generators is either empty or a neighbourhood of the binary level-set system.

Proof. Induct through the generation rules. Generators and the whole space are neighbourhoods; finite intersections remain level sets, unions are closed, and complements are empty or neighbourhoods. ■

Theorem (`V.mem_iff_generatedBy`). A set is a neighbourhood of the binary level-set system exactly when it is a nonempty member of the Boolean algebra generated by the coordinate generators.

Proof. Level sets are generated; conversely, the preceding induction says that a nonempty generated set must be a neighbourhood. ■

Definition (`Formula`). A propositional formula is built from countably many variables, falsity, truth, negation, conjunction, and disjunction.

Definition (`evalSet`). The set interpretation sends a variable to its coordinate generator and interprets the Boolean connectives by the corresponding operations on subsets of the natural numbers.

Definition (`evalV`). The valuation interpretation assigns Boolean values to formulas by the usual truth tables.

Definition (`SemanticEquiv`). Two formulas are semantically equivalent when they have the same truth value under every Boolean valuation.

Theorem (`exists_bitsOf_agree`). Given finitely many variables and a Boolean valuation, there is a natural number whose binary digits agree with that valuation on all those variables.

Proof. Take the bit mask of the variables assigned true. ■

Definition (`formulaSetoid`). Semantic equivalence is an equivalence relation on formulas.

Definition (`Lindenbaum`). The Lindenbaum algebra is the quotient of formulas by semantic equivalence.

Theorem (`mem_evalSet_iff`). A natural number belongs to the set interpretation of a formula exactly when the formula is true under the valuation given by that number's binary digits.

Proof. Induction on the formula follows the Boolean operations in the two interpretations. ■

Definition (`vars`). The variables of a formula form the finite set obtained recursively from its variable occurrences.

Theorem (`evalV_eq_of_agree`). Two valuations that agree on every variable of a formula give that formula the same truth value.

Proof. Induction on the formula; compound formulas use agreement on the variables of each constituent. ■

Theorem (`semanticEquiv_iff_evalSet_eq`). Two formulas are semantically equivalent exactly when their set interpretations are equal.

Proof. Semantic equivalence gives pointwise equality on binary valuations. Conversely, realize the finitely many variables of both formulas as bits of one natural number and use invariance under agreement on occurring variables. ■

Definition (`Lindenbaum.toSet`). A semantic-equivalence class is sent to the common set interpretation of its representatives.

Theorem (`Lindenbaum.toSet_mk`). The set assigned to the class of a formula is its set interpretation.

Proof. This is the quotient construction applied to that representative. ■

Definition (`Entails`). A formula entails another when every valuation satisfying the first also satisfies the second.

Theorem (`Lindenbaum.toSet_injective`). The set interpretation of the Lindenbaum algebra is injective.

Proof. Equal set interpretations imply semantic equivalence, so the representing formulas determine the same quotient class. ■

Theorem (`entails_iff_evalSet_subset`). A formula entails another exactly when its set interpretation is contained in that of the other.

Proof. Inclusion immediately handles valuations arising from binary digits. For an arbitrary valuation, realize all variables occurring in either formula by one natural number and transfer both truth values along agreement. ■

Theorem (`example_813_b`). Boolean entailment between formulas is exactly inclusion between their set interpretations.

Proof. This is the preceding entailment characterization. ■

`BooleanAlgebraFilters.lean`

```
module

public import Mathlib.Topology.Constructions
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Exercise_8_12
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    GeneratedBy

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_6
public import Mathlib.Data.Nat.Bitwise

@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
open Domain.Recursive NeighborhoodSystem
open NeighborhoodSystem

theorem exercise813a : U ≅? V := ⟨effectiveIso812_UV.toDomainIso⟩

theorem example_813_a : U ≅? V :=
  exercise813a

open NeighborhoodSystem

theorem generator_nonempty (i : ℕ) : (generator i).Nonempty := ⟨2 ^ i, by simp⟩

theorem generators_independent (s : Finset ℕ) (? : ℕ → Bool) :
    (⋂ i ∈ s, if ? i then generator i else (generator i)?).Nonempty := by
  refine ⟨bitmaskOf (s.filter (fun i => ? i)), ?_⟩
  simp only [Set.mem_iInter]
  intro i hi
  split_ifs with hb <;> simp [testBit_bitmaskOf, Finset.mem_filter, hi, hb]

theorem levelSet_compl (k m : ℕ) :
    (levelSet k m)? = levelSet k (bitmaskOf ((Finset.range (2 ^ k)).filter
      (fun ? => ¬ m.testBit ?))) := by
  ext n
  simp only [Set.mem_compl_iff, mem_levelSet, testBit_bitmaskOf, Finset.mem_filter,
    Finset.mem_range]
  exact ⟨fun h => ⟨Nat.mod_lt n (Nat.two_pow_pos _), h⟩, fun h => h.2⟩

theorem V.compl_mem {X : Set ℕ} (hX : V.mem X) : X? = ∅ ∨ V.mem X? := by
  obtain ⟨k, m, rfl, -⟩ := hX
  rw [levelSet_compl]
  rcases (levelSet _ _).eq_empty_or_nonempty with h | h
  · exact Or.inl h
  · exact Or.inr ⟨k, _, rfl, h⟩

theorem testBit_mod_two_pow (n k i : ℕ) :
    (n % 2 ^ k).testBit i = (decide (i < k) && n.testBit i) := by
  have hlt : n % 2 ^ k < 2 ^ k := Nat.mod_lt n (Nat.two_pow_pos k)
  have h := Nat.testBit_two_pow_mul_add (n / 2 ^ k) hlt i
  rw [Nat.div_add_mod] at h
  rcases Nat.lt_or_ge i k with hik | hik
  · simp only [hik, ite_true, decide_true, Bool.true_and] at h ⊢
    exact h.symm
  · simp only [Nat.not_lt.mpr hik, decide_false, Bool.false_and]
    exact Nat.testBit_eq_false_of_lt (hlt.trans_le (Nat.pow_le_pow_right (by norm_num)
     hik))

theorem affine_eq_biInter (k ? : ℕ) (h? : ? < 2 ^ k) :
    affine k ? = ⋂ i ∈ Finset.range k, (if ?.testBit i then generator i else (
    generator i)?) := by
  ext n
  simp only [mem_affine, Set.mem_iInter, Finset.mem_range]
  have key : n % 2 ^ k = ? ↔ ∀ i < k, n.testBit i = ?.testBit i := by
    constructor
```

```
    · rintro rfl i hi
      rw [testBit_mod_two_pow]; simp [hi]
    · intro h
      apply Nat.eq_of_testBit_eq
      intro i
      rw [testBit_mod_two_pow]
      rcases Nat.lt_or_ge i k with hik | hik
      · simp [hik, h i hik]
      · simp only [Nat.not_lt.mpr hik, decide_false, Bool.false_and]
        exact (Nat.testBit_eq_false_of_lt (h?.trans_le (Nat.pow_le_pow_right (by
   norm_num) hik))).symm
  rw [key]
  refine forall₂_congr fun i _ => ?_
  split_ifs with hb <;> simp [hb]

theorem generatedBy_affine (k ? : ℕ) (h? : ? < 2 ^ k) : GeneratedBy generator (affine
    k ?) := by
  rw [affine_eq_biInter k ? h?]
  induction Finset.range k using Finset.induction_on with
  | empty => simpa using GeneratedBy.univ
  | insert a s _ ih =>
    rw [Finset.set_biInter_insert]
    refine GeneratedBy.inter ?_ ih
    split_ifs with hb
    · exact GeneratedBy.of _
    · exact GeneratedBy.compl (GeneratedBy.of _)

theorem generatedBy_biUnion_affine (k : ℕ) (s : Finset ℕ) (hs : ∀ ? ∈ s, ? < 2 ^ k) :
    GeneratedBy generator (⋃ ? ∈ s, affine k ?) := by
  induction s using Finset.induction_on with
  | empty => simpa using GeneratedBy.univ.compl
  | insert a s ha ih =>
    rw [Finset.set_biUnion_insert]
    exact GeneratedBy.union (generatedBy_affine k a (hs a (Finset.mem_insert_self a s)
    ))
      (ih fun ? h? => hs ? (Finset.mem_insert_of_mem h?))

theorem generatedBy_levelSet (k m : ℕ) : GeneratedBy generator (levelSet k m) := by
  rw [levelSet.eq_biUnion]
  exact generatedBy_biUnion_affine k _ fun ? h? => (Finset.mem_filter.mp h?).1 |>
    Finset.mem_range.mp

theorem testBit_mod_two_pow_succ (n i : ℕ) :
    (n % 2 ^ (i + 1)).testBit i = n.testBit i := by
  rw [testBit_mod_two_pow]; simp

theorem generator_eq_levelSet (i : ℕ) :
    generator i = levelSet (i + 1) (bitmaskOf ((Finset.range (2 ^ (i + 1))).filter
      (fun ? => ?.testBit i))) := by
  ext n
  simp only [mem_generator, mem_levelSet, testBit_bitmaskOf, Finset.mem_filter, Finset
    .mem_range]
  rw [← testBit_mod_two_pow_succ]
  exact ⟨fun h => ⟨Nat.mod_lt n (Nat.two_pow_pos _), h⟩, fun h => h.2⟩

theorem V.mem_generator (i : ℕ) : V.mem (generator i) :=
```

```
  generator_eq_levelSet i ▶ ⟨i + 1, _, rfl, generator_eq_levelSet i ▶
    generator_nonempty i⟩

theorem generatedBy_generator_imp {X : Set ℕ} (h : GeneratedBy generator X) :
    X = ∅ ∨ V.mem X := by
  induction h with
  | of i => exact Or.inr (V.mem_generator i)
  | univ => exact Or.inr V.master_mem
  | @inter X Y _ _ ih1 ih2 =>
    rcases ih1 with rfl | hX
    · exact Or.inl (Set.empty_inter Y)
    rcases ih2 with rfl | hY
    · exact Or.inl (Set.inter_empty X)
    obtain ⟨k₁, m₁, rfl, -⟩ := hX
    obtain ⟨k₂, m₂, rfl, -⟩ := hY
    rw [levelSet.inter]
    rcases (levelSet _ _).eq_empty_or_nonempty with h | h
    · exact Or.inl h
    · exact Or.inr ⟨_, _, rfl, h⟩
  | union _ _ ih1 ih2 => exact V.union_mem ih1 ih2
  | @compl X _ ih =>
    rcases ih with rfl | hX
    · simp only [Set.compl_empty]
      exact Or.inr V.master_mem
    · exact V.compl_mem hX

theorem V.mem_iff_generatedBy {X : Set ℕ} : V.mem X ↔ GeneratedBy generator X ∧ X.
    Nonempty := by
  constructor
  · rintro ⟨k, m, rfl, hne⟩
    exact ⟨generatedBy_levelSet k m, hne⟩
  · rintro ⟨hgen, hne⟩
    exact (generatedBy_generator_imp hgen).resolve_left hne.ne_empty

inductive Formula : Type
  | var (i : ℕ) : Formula
  | bot : Formula
  | top : Formula
  | neg (φ : Formula) : Formula
  | and (φ ψ : Formula) : Formula
  | or (φ ψ : Formula) : Formula

def evalSet : Formula → Set ℕ
  | .var i => generator i
  | .bot => ∅
  | .top => Set.univ
  | .neg φ => (evalSet φ)ᶜ
  | .and φ ψ => evalSet φ ∩ evalSet ψ
  | .or φ ψ => evalSet φ ∪ evalSet ψ

def evalV (v : ℕ → Bool) : Formula → Bool
  | .var i => v i
  | .bot => false
  | .top => true
  | .neg φ => !(evalV v φ)
  | .and φ ψ => evalV v φ && evalV v ψ
```

```
  | .or φ ψ => evalV v φ || evalV v ψ

def SemanticEquiv (φ ψ : Formula) : Prop := ∀ v, evalV v φ = evalV v ψ

theorem exists_bitsOf_agree (S : Finset ℕ) (v : ℕ → Bool) :
    ∃ n : ℕ, ∀ i ∈ S, n.testBit i = v i := by
  refine ⟨bitmaskOf (S.filter (fun i => v i)), fun i hi => ?_⟩
  have key : (bitmaskOf (S.filter (fun i => v i))).testBit i = true ↔ v i = true := by
    rw [testBit_bitmaskOf]; simp [Finset.mem_filter, hi]
  rcases hv : v i with - | - <;> rcases hn : (bitmaskOf (S.filter (fun i => v i))).
    testBit i
    with - | - <;> simp_all

instance formulaSetoid : Setoid Formula where
  r := SemanticEquiv
  iseqv := ⟨fun _ _ => rfl, fun h v => (h v).symm, fun h1 h2 v => (h1 v).trans (h2 v)⟩

def Lindenbaum : Type := Quotient formulaSetoid

theorem mem_evalSet_iff (n : ℕ) (φ : Formula) :
    n ∈ evalSet φ ↔ evalV (fun i => n.testBit i) φ = true := by
  induction φ with
  | var i => simp [evalSet, evalV]
  | bot => simp [evalSet, evalV]
  | top => simp [evalSet, evalV]
  | neg φ ih => simp [evalSet, evalV, ih]
  | and φ ψ ihφ ihψ => simp [evalSet, evalV, ihφ, ihψ]
  | or φ ψ ihφ ihψ => simp [evalSet, evalV, ihφ, ihψ]

def vars : Formula → Finset ℕ
  | .var i => {i}
  | .bot => ∅
  | .top => ∅
  | .neg φ => vars φ
  | .and φ ψ => vars φ ∪ vars ψ
  | .or φ ψ => vars φ ∪ vars ψ

theorem evalV_eq_of_agree {φ : Formula} {v v' : ℕ → Bool} (h : ∀ i ∈ vars φ, v i = v'
    i) :
    evalV v φ = evalV v' φ := by
  induction φ with
  | var i => exact h i (Finset.mem_singleton_self i)
  | bot => rfl
  | top => rfl
  | neg φ ih => simp only [evalV, ih h]
  | and φ ψ ihφ ihψ =>
    simp only [evalV, vars] at h ⊢
    rw [ihφ (fun i hi => h i (Finset.mem_union_left _ hi)),
      ihψ (fun i hi => h i (Finset.mem_union_right _ hi))]
  | or φ ψ ihφ ihψ =>
    simp only [evalV, vars] at h ⊢
    rw [ihφ (fun i hi => h i (Finset.mem_union_left _ hi)),
      ihψ (fun i hi => h i (Finset.mem_union_right _ hi))]

theorem semanticEquiv_iff_evalSet_eq {φ ψ : Formula} :
    SemanticEquiv φ ψ ↔ evalSet φ = evalSet ψ := by
```

```
  constructor
  · intro h
    ext n
    rw [mem_evalSet_iff, mem_evalSet_iff, h]
  · intro h v
    obtain ⟨n, hn⟩ := exists_bitsOf_agree (vars φ ∪ vars ψ) v
    have h1 : evalV v φ = evalV (fun i => n.testBit i) φ :=
      evalV_eq_of_agree fun i hi => (hn i (Finset.mem_union_left _ hi)).symm
    have h2 : evalV v ψ = evalV (fun i => n.testBit i) ψ :=
      evalV_eq_of_agree fun i hi => (hn i (Finset.mem_union_right _ hi)).symm
    have hiff : evalV (fun i => n.testBit i) φ = true ↔ evalV (fun i => n.testBit i)
    ψ = true := by
      rw [← mem_evalSet_iff, ← mem_evalSet_iff, h]
    rw [h1, h2]
    rcases hφ : evalV (fun i => n.testBit i) φ with - | - <;>
      rcases hψ : evalV (fun i => n.testBit i) ψ with - | - <;> simp_all

def Lindenbaum.toSet : Lindenbaum → Set ℕ :=
  Quotient.lift evalSet fun _ _ h => semanticEquiv_iff_evalSet_eq.mp h

@[simp] theorem Lindenbaum.toSet_mk (φ : Formula) :
    Lindenbaum.toSet ⟦φ⟧ = evalSet φ := rfl

def Entails (φ ψ : Formula) : Prop := ∀ v, evalV v φ = true → evalV v ψ = true

theorem Lindenbaum.toSet_injective : Function.Injective Lindenbaum.toSet := by
  intro x y
  induction x using Quotient.ind with
  | _ φ =>
    induction y using Quotient.ind with
    | _ ψ =>
      intro h
      exact Quotient.sound (semanticEquiv_iff_evalSet_eq.mpr h)

theorem entails_iff_evalSet_subset {φ ψ : Formula} :
    Entails φ ψ ↔ evalSet φ ⊆ evalSet ψ := by
  constructor
  · intro h n hn
    rw [mem_evalSet_iff] at hn ⊢
    exact h _ hn
  · intro h v hv
    obtain ⟨n, hn⟩ := exists_bitsOf_agree (vars φ ∪ vars ψ) v
    have h1 : evalV v φ = evalV (fun i => n.testBit i) φ :=
      evalV_eq_of_agree fun i hi => (hn i (Finset.mem_union_left _ hi)).symm
    have h2 : evalV v ψ = evalV (fun i => n.testBit i) ψ :=
      evalV_eq_of_agree fun i hi => (hn i (Finset.mem_union_right _ hi)).symm
    rw [h2, ← mem_evalSet_iff]
    exact h (by rw [mem_evalSet_iff, ← h1]; exact hv)

theorem example_813_b :
    ∀ {φ ψ : Formula}, Entails φ ψ ↔ evalSet φ ⊆ evalSet ψ :=
  @entails_iff_evalSet_subset

end DomainTheory.NeighbourhoodSystems
```

4.9.40 Generated-by filters

This peer isolates the Boolean algebra of subsets generated by the binary-coordinate sets of the natural numbers and records the finite bit-mask calculations used later.

Definition (`GeneratedBy`). A set is generated by a sequence of generators when it can be obtained from individual generators and the whole space by finitely many intersections, unions, and complements.

Definition (`bitmaskOf`). The bit mask of a finite set of natural numbers is the sum of the corresponding powers of two.

Theorem (`testBit_sum_two_pow`). The i-th bit of the sum of distinct powers 2^j, with j ranging over a finite set, is set exactly when i belongs to that set.

Proof. Distinct binary place values give precisely the listed bit indices. ■

Theorem (`testBit_bitmaskOf`). The i-th bit of the bit mask of a finite set is set exactly when i belongs to the set.

Proof. This is the preceding characterization applied to the defining sum. ■

Definition (`generator`). The i-th generator is the set of natural numbers whose i-th binary digit is one.

Theorem (`mem_generator`). A natural number belongs to the i-th generator exactly when its i-th binary digit is one.

Proof. This is immediate from the definition of the generator. ■

`GeneratedBy.lean`

```
module

public import Mathlib.Algebra.BigOperators.Group.Finset.Basic
public import Mathlib.Data.Nat.Bitwise
public import Mathlib.Data.Nat.Bits
public import Mathlib.Data.Nat.BitIndices
public import Mathlib.Combinatorics.Colex

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

inductive GeneratedBy {α : Type*} (g : ℕ → Set α) : Set α → Prop
  | of (i : ℕ) : GeneratedBy g (g i)
  | univ : GeneratedBy g Set.univ
  | inter {X Y} : GeneratedBy g X → GeneratedBy g Y → GeneratedBy g (X ∩ Y)
  | union {X Y} : GeneratedBy g X → GeneratedBy g Y → GeneratedBy g (X ∪ Y)
  | compl {X} : GeneratedBy g X → GeneratedBy g Xᶜ

def bitmaskOf (s : Finset ℕ) : ℕ := ∑ j ∈ s, 2 ^ j

theorem testBit_sum_two_pow {s : Finset ℕ} {i : ℕ} : (∑ j ∈ s, 2 ^ j).testBit i ↔ i ∈
    s := by
  rw [← Nat.mem_bitIndices, ← List.mem_toFinset, Finset.
    toFinset_bitIndices_sum_two_pow]

@[simp] theorem testBit_bitmaskOf {s : Finset ℕ} {i : ℕ} : (bitmaskOf s).testBit i ↔
    i ∈ s :=
  testBit_sum_two_pow

def generator (i : ℕ) : Set ℕ := {n : ℕ | n.testBit i = true}
```

```
@[simp] theorem mem_generator {i n : ℕ} : n ∈ generator i ↔ n.testBit i = true := Iff
    .rfl

end DomainTheory.NeighbourhoodSystems
end
```

4.9.41 Exercise 8.13 — Boolean-algebra filters

(For logicians.) Prove that the universal domain U is isomorphic to the domain of all proper filters of the free Boolean algebra on $\aleph_0$-generators (= the Lindenbaum algebra of propositional calculus). (For topologists.) Connect this

representation of $\mathcal{U}$ with the collection of non-empty open subsets of the product space $2^{\mathbf{N}}$ (= Cantor space).

The development is with the Boolean-algebra filters peer.

Theorem (`example_813`). The Boolean-algebra filter presentation is effectively isomorphic to U.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_13.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    BooleanAlgebraFilters

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

theorem example_813 : U ≅? V := exercise813a
```

4.9.42 Closure operator domain

This peer proves that the fixed points of an inflationary retraction form a finitary domain, with compact fixed points obtained by closing principal elements.

Definition (`ClosureOperator`). An approximable endomap is a closure operator when it is a retraction and lies above the identity map.

Definition (`IsCompactFix`). A fixed point is compact among fixed points when, whenever it lies below the directed supremum of a family of fixed points, it already lies below one member of that family.

Definition (`CompactFix`). A compact fixed point is a fixed point equipped with the assertion that it is compact among fixed points.

Definition (`botFix`). The least fixed point of a closure operator is the closure of the least element.

Theorem (`botFix_le`). The least fixed point lies below every fixed point.

Proof. Monotonicity sends the least element below the closure of any fixed point, which equals that fixed point. ■

Theorem (`isCompactFix_botFix`). The least fixed point is compact among fixed points.

Proof. In any nonempty directed family it lies below every member, hence below a chosen member. ■

Definition (`botK`). The bottom compact fixed point is the least fixed point together with its compactness.

Theorem (`le_toElementMap_of_isClosureOperator`). Every element lies below its image under a closure operator.

Proof. The closure operator lies above the identity, and map order is evaluated pointwise on elements. ■

Theorem (`isCompactFix_sup`). If two compact fixed points have a common upper bound, then the closure of their bounded supremum is a compact fixed point.

Proof. Below a directed supremum, each compact point lies below some family member; directedness gives one member above both, and monotonicity of closure places the closed supremum below that fixed member. ■

Theorem (`isCompactFix_toElementMap_principal`). The closure of every principal element is a compact fixed point.

Proof. Inflationarity reduces a bound on its closure to a bound on the principal element; compactness of principal elements chooses one family member, and monotonicity of closure completes the comparison. ■

Definition (`genK`). A basic neighbourhood determines the compact fixed point obtained by closing its principal element.

Definition (`joinCompactFix`). Two compact fixed points below a common fixed point are joined by closing their bounded supremum.

Theorem (`joinCompactFix_le`). The joined compact fixed point remains below the chosen common fixed upper bound.

Proof. The bounded supremum lies below the upper bound, and closure is monotone while the upper bound is fixed. ■

Definition (`upFix`). The upper neighbourhood generated by a compact fixed point consists of all compact fixed points above it.

Theorem (`mem_upFix`). Membership in an upper neighbourhood is exactly domination of its generating compact fixed point.

Proof. This is the defining condition. ■

Theorem (`self_mem_upFix`). A compact fixed point belongs to its own upper neighbourhood.

Proof. The order is reflexive. ■

Theorem (`upFix_antitone`). If one compact fixed point lies below another, the upper neighbourhood of the latter is contained in that of the former.

Proof. This follows by transitivity of the order. ■

Theorem (`upFix_injOn`). Equal upper neighbourhoods have generators with equal underlying fixed points.

Proof. Each generator belongs to its own upper neighbourhood; equality yields both order inequalities, hence antisymmetry. ■

Theorem (`upFix_inter`). For two compact fixed points with a common fixed upper bound, the intersection of their upper neighbourhoods is the upper neighbourhood of their join.

Proof. A compact fixed point lies above both generators exactly when it lies above their bounded supremum and therefore above its closure; the reverse implication uses inflationarity and the inclusion of each generator in the supremum. ■

Theorem (`upFix_subset_upFix_botK`). Every upper neighbourhood is contained in the upper neighbourhood generated by the bottom compact fixed point.

Proof. The bottom fixed point lies below every fixed point. ■

Definition (`domainOfClosure`). The domain of a closure operator is the neighbourhood system whose tokens are compact fixed points and whose basic neighbourhoods are their upper neighbourhoods.

Definition (`confirmedIdx`). For an element of the closure domain, a confirmed index is a compact fixed point whose upper neighbourhood belongs to that element.

Definition (`instNonemptyConfirmedIdx`). Every closure-domain element has a confirmed index.

Proof. Its master neighbourhood is the upper neighbourhood of the bottom compact fixed point. ■

Theorem (`confirmedIdx_directed`). The underlying fixed points of the confirmed indices of an element form a directed family.

Proof. The element confirms the intersection of two confirmed upper neighbourhoods; a basic upper neighbourhood below that intersection provides a confirmed compact fixed point above both generators. ■

Definition (`fromElementFilter`). A closure-domain element determines the directed supremum of the underlying fixed points of all its confirmed indices.

Definition (`genFamily`). For a fixed point and one of its neighbourhoods, the generated fixed point is the closure of the corresponding principal element.

Theorem (`genFamily_directed`). The generated fixed points associated with the neighbourhoods of a fixed point form a directed family.

Proof. Principal elements associated with an element's neighbourhoods form a directed family, and monotonicity of closure preserves the two required inequalities. ■

Definition (`toElementFilter`). A fixed point determines the closure-domain element that confirms exactly those upper neighbourhoods whose compact generators lie below it.

Theorem (`le_of_toElementFilter_le`). If the closure-domain element associated with one fixed point lies below that associated with another, then the first fixed point lies below the second.

Proof. Test each neighbourhood of the first point using the compact closure of its principal element. Equality of generators for equal upper neighbourhoods and inflationarity show that the neighbourhood also belongs to the second point. ■

Theorem (`toElementFilter_fromElementFilter`). Sending a closure-domain element to its fixed point and then back to a filter returns the original element.

Proof. Compactness turns a bound below the directed supremum into a bound below one confirmed index; antitonicity then gives membership. The converse follows from the canonical inclusion of a confirmed index in the supremum. ■

Theorem (`toElementFilter_mono`). The filter assigned to a fixed point is monotone in that fixed point.

Proof. Any compact generator below the smaller fixed point is below the larger one by transitivity. ■

Theorem (`toElementMap_eq_iSupDirected_genFamily`). Every fixed point is the directed supremum of the closures of the principal elements determined by its neighbourhoods.

Proof. Express the underlying element as the directed supremum of its principal approximants, use preservation of directed suprema by the map, and then use fixedness. ■

Theorem (`fromElementFilter_toElementFilter`). Sending a fixed point to its filter and reconstructing a fixed point returns the original fixed point.

Proof. Every confirmed compact generator lies below the fixed point, while the generated principal family is cofinal in the reconstruction and has supremum equal to the fixed point. ■

Definition (`fixSetOrderIso`). The fixed points of a closure operator are order-isomorphic to the elements of its closure domain.

Theorem (`isCompactFix_iff`). A fixed point is compact among fixed points exactly when it is the closure of a principal element arising from some neighbourhood.

Proof. Compactness selects one member from the directed generated family whose supremum is the fixed point, and the reverse inequality follows from monotonicity and fixedness. The converse is compact-

ness of closed principal elements. ■

Theorem (`isFinitary_of_isClosureOperator`). Every closure operator is finitary.

Proof. Its compact fixed points form the closure domain, whose elements are order-isomorphic to all fixed points. ■

Theorem (`example_814`). Compact fixed points of a closure operator are exactly closures of principal elements, and every closure operator is finitary.

Proof. Combine the compact-fixed-point characterization with the finitarity theorem. ■

`ClosureOperatorDomain.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_1

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    DedekindCutProjection
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
universe u
variable {α : Type u} {E : NeighborhoodSystem α}

class ClosureOperator (a : ApproximableMap E E) : Prop where
  out : Retraction a ∧ idMap E ≤ a

section ClosureOperator

variable {a : ApproximableMap E E}

def IsCompactFix (_h : ClosureOperator a) (y : FixSet a) : Prop :=
  ∀ {I : Type u} [Nonempty I] (d : I → FixSet a)
    (hdir : ∀ i j, ∃ k, (d i).1 ≤ (d k).1 ∧ (d j).1 ≤ (d k).1),
    y.1 ≤ iSupDirected (fun i => (d i).1) hdir → ∃ i, y.1 ≤ (d i).1

end ClosureOperator

section FiniteDomain

variable {a : ApproximableMap E E} (h : ClosureOperator a)

def CompactFix (h : ClosureOperator a) := {y : FixSet a // IsCompactFix h y}

end FiniteDomain

section ClosureOperator

variable {a : ApproximableMap E E}

def botFix (h : ClosureOperator a) : FixSet a :=
```

```
  ⟨a.toElementMap ⊥, toElementMap_idem h.out.1 ⊥⟩

theorem botFix_le (h : ClosureOperator a) (y : FixSet a) : (botFix h).1 ≤ y.1 := by
  have h1 : a.toElementMap ⊥ ≤ a.toElementMap y.1 := a.toElementMap_mono bot_le
  rwa [y.2] at h1

theorem isCompactFix_botFix (h : ClosureOperator a) : IsCompactFix h (botFix h) := by
  intro I _ d _ _
  obtain ⟨i0⟩ := (inferInstance : Nonempty I)
  exact ⟨i0, botFix_le h (d i0)⟩

end ClosureOperator

section FiniteDomain

variable {a : ApproximableMap E E} (h : ClosureOperator a)

def botK : CompactFix h := ⟨botFix h, isCompactFix_botFix h⟩

end FiniteDomain

section ClosureOperator

variable {a : ApproximableMap E E}

theorem le_toElementMap_of_isClosureOperator (h : ClosureOperator a) (y : E.Element) :
    y ≤ a.toElementMap y := by
  have h1 := (le_iff_toElementMap_le.mp h.out.2) y
  rwa [toElementMap_idMap] at h1

theorem isCompactFix_sup (h : ClosureOperator a) {y1 y2 : FixSet a}
    (hc1 : IsCompactFix h y1) (hc2 : IsCompactFix h y2)
    (hb : E.Bounded ({y1.1, y2.1} : Set E.Element)) :
    IsCompactFix h ⟨a.toElementMap (E.sSup {y1.1, y2.1} hb), toElementMap_idem h.out.1
    _⟩ := by
  intro I _ d hdir hle
  have hs_le : E.sSup {y1.1, y2.1} hb ≤ iSupDirected (fun i => (d i).1) hdir :=
    (le_toElementMap_of_isClosureOperator h _).trans hle
  have h1le : y1.1 ≤ iSupDirected (fun i => (d i).1) hdir :=
    (E.le_sSup {y1.1, y2.1} hb (by simp)).trans hs_le
  have h2le : y2.1 ≤ iSupDirected (fun i => (d i).1) hdir :=
    (E.le_sSup {y1.1, y2.1} hb (by simp)).trans hs_le
  obtain ⟨i1, hi1⟩ := hc1 d hdir h1le
  obtain ⟨i2, hi2⟩ := hc2 d hdir h2le
  obtain ⟨k, hk1, hk2⟩ := hdir i1 i2
  have hle1 : y1.1 ≤ (d k).1 := hi1.trans hk1
  have hle2 : y2.1 ≤ (d k).1 := hi2.trans hk2
  have hsup_le : E.sSup {y1.1, y2.1} hb ≤ (d k).1 :=
    E.sSup_le {y1.1, y2.1} hb (by rintro x (rfl | rfl) <;> assumption)
  refine ⟨k, ?_⟩
  have h3 := a.toElementMap_mono hsup_le
  rwa [(d k).2] at h3

theorem isCompactFix_toElementMap_principal (h : ClosureOperator a) {X : Set α} (hX :
    E.mem X) :
    IsCompactFix h ⟨a.toElementMap (E.principal hX), toElementMap_idem h.out.1 _⟩ := by
```

```
  intro I _ d hdir hle
  have hle' : E.principal hX ≤ iSupDirected (fun i => (d i).1) hdir :=
    (le_toElementMap_of_isClosureOperator h (E.principal hX)).trans hle
  obtain ⟨i, hi⟩ := principal_isCompactElt hX (fun i => (d i).1) hdir hle'
  refine ⟨i, ?_⟩
  have h2 := a.toElementMap_mono hi
  rwa [(d i).2] at h2

end ClosureOperator

section FiniteDomain

variable {a : ApproximableMap E E} (h : ClosureOperator a)

def genK {X : Set α} (hX : E.mem X) : CompactFix h :=
  ⟨⟨a.toElementMap (E.principal hX), toElementMap_idem h.out.1 _⟩,
    isCompactFix_toElementMap_principal h hX⟩

def joinCompactFix {k1 k2 : CompactFix h} {y : FixSet a}
    (hk1y : k1.1.1 ≤ y.1) (hk2y : k2.1.1 ≤ y.1) : CompactFix h :=
  ⟨⟨a.toElementMap (E.sSup {k1.1.1, k2.1.1} ⟨y.1, by rintro x (rfl | rfl) <;>
    assumption⟩),
      toElementMap_idem h.out.1 _⟩,
    isCompactFix_sup h k1.2 k2.2 ⟨y.1, by rintro x (rfl | rfl) <;> assumption⟩⟩

theorem joinCompactFix_le {k1 k2 : CompactFix h} {y : FixSet a}
    (hk1y : k1.1.1 ≤ y.1) (hk2y : k2.1.1 ≤ y.1) : (joinCompactFix h hk1y hk2y).1.1 ≤
    y.1 := by
  have hb : E.Bounded ({k1.1.1, k2.1.1} : Set E.Element) :=
    ⟨y.1, by rintro x (rfl | rfl) <;> assumption⟩
  have hsup_le : E.sSup {k1.1.1, k2.1.1} hb ≤ y.1 :=
    E.sSup_le {k1.1.1, k2.1.1} hb (by rintro x (rfl | rfl) <;> assumption)
  have h1 := a.toElementMap_mono hsup_le
  rwa [y.2] at h1

def upFix (k : CompactFix h) : Set (CompactFix h) := {q | k.1.1 ≤ q.1.1}

@[simp] theorem mem_upFix {k q : CompactFix h} : q ∈ upFix h k ↔ k.1.1 ≤ q.1.1 := Iff
    .rfl

theorem self_mem_upFix (k : CompactFix h) : k ∈ upFix h k := (mem_upFix h).mpr (
    le_refl _)

theorem upFix_antitone {k k' : CompactFix h} (hkk' : k.1.1 ≤ k'.1.1) :
    upFix h k' ⊆ upFix h k :=
  fun _ hq => hkk'.trans hq

theorem upFix_injOn {k k' : CompactFix h} (heq : upFix h k = upFix h k') : k.1.1 = k
    '.1.1 :=
  le_antisymm ((Set.ext_iff.mp heq k').mpr (self_mem_upFix h k'))
    ((Set.ext_iff.mp heq k).mp (self_mem_upFix h k))

theorem upFix_inter (k1 k2 : CompactFix h) {y : FixSet a}
    (hk1y : k1.1.1 ≤ y.1) (hk2y : k2.1.1 ≤ y.1) :
    upFix h k1 ∩ upFix h k2 = upFix h (joinCompactFix h hk1y hk2y) := by
  have hb : E.Bounded ({k1.1.1, k2.1.1} : Set E.Element) :=
```

```
    ⟨y.1, by rintro x (rfl | rfl) <;> assumption⟩
  apply Set.ext
  intro q
  simp only [mem_upFix, Set.mem_inter_iff, joinCompactFix]
  constructor
  · rintro ⟨h1q, h2q⟩
    have hb' : E.Bounded ({k1.1.1, k2.1.1} : Set E.Element) :=
      ⟨q.1.1, by rintro x (rfl | rfl) <;> assumption⟩
    have hsup_le : E.sSup {k1.1.1, k2.1.1} hb' ≤ q.1.1 :=
      E.sSup_le {k1.1.1, k2.1.1} hb' (by rintro x (rfl | rfl) <;> assumption)
    have h3 := a.toElementMap_mono hsup_le
    rwa [q.1.2] at h3
  · intro hjq
    refine ⟨?_, ?_⟩
    · exact ((E.le_sSup {k1.1.1, k2.1.1} hb (by simp)).trans
        (le_toElementMap_of_isClosureOperator h _)).trans hjq
    · exact ((E.le_sSup {k1.1.1, k2.1.1} hb (by simp)).trans
        (le_toElementMap_of_isClosureOperator h _)).trans hjq

theorem upFix_subset_upFix_botK (k : CompactFix h) : upFix h k ⊆ upFix h (botK h) :=
  fun q _ => botFix_le h q.1

def domainOfClosure (h : ClosureOperator a) : NeighborhoodSystem (CompactFix h) where
  mem S := ∃ k : CompactFix h, S = upFix h k
  master := upFix h (botK h)
  master_nonempty := ⟨botK h, self_mem_upFix h (botK h)⟩
  master_mem := ⟨botK h, rfl⟩
  sub_master := by rintro S ⟨k, rfl⟩; exact upFix_subset_upFix_botK h k
  inter_mem := by
    rintro S1 S2 Z ⟨k1, rfl⟩ ⟨k2, rfl⟩ ⟨k3, rfl⟩ hZsub
    obtain ⟨hk13, hk23⟩ := hZsub (self_mem_upFix h k3)
    exact ⟨joinCompactFix h hk13 hk23, upFix_inter h k1 k2 hk13 hk23⟩

def confirmedIdx (g : (domainOfClosure h).Element) := {k : CompactFix h // g.mem (
    upFix h k)}

instance instNonemptyConfirmedIdx (g : (domainOfClosure h).Element) :
    Nonempty (confirmedIdx h g) :=
  ⟨⟨botK h, g.master_mem⟩⟩

theorem confirmedIdx_directed (g : (domainOfClosure h).Element) :
    ∀ i j : confirmedIdx h g, ∃ k : confirmedIdx h g,
      i.1.1.1 ≤ k.1.1.1 ∧ j.1.1.1 ≤ k.1.1.1 := by
  rintro ⟨k1, hk1⟩ ⟨k2, hk2⟩
  have hint : g.mem (upFix h k1 ∩ upFix h k2) := g.inter_mem hk1 hk2
  obtain ⟨k3, hk3eq⟩ := g.sub hint
  rw [hk3eq] at hint
  have hmem3 : k3 ∈ upFix h k1 ∩ upFix h k2 := hk3eq ▸ self_mem_upFix h k3
  exact ⟨⟨k3, hint⟩, hmem3.1, hmem3.2⟩

def fromElementFilter (g : (domainOfClosure h).Element) : FixSet a :=
  ⟨iSupDirected (fun i : confirmedIdx h g => i.1.1.1) (confirmedIdx_directed h g),
    toElementMap_iSupDirected_fixed _ (confirmedIdx_directed h g) (fun i => i.1.1.2)⟩

def genFamily (y : FixSet a) (i : {X : Set α // y.1.mem X}) : FixSet a :=
  (genK h (y.1.sub i.2)).1
```

```
theorem genFamily_directed (y : FixSet a) :
    ∀ i j : {X : Set α // y.1.mem X}, ∃ k, (genFamily h y i).1 ≤ (genFamily h y k).1 ∧
      (genFamily h y j).1 ≤ (genFamily h y k).1 := by
  intro i j
  obtain ⟨k, hik, hjk⟩ := principalFamily_directed y.1 i j
  exact ⟨k, a.toElementMap_mono hik, a.toElementMap_mono hjk⟩

def toElementFilter (y : FixSet a) : (domainOfClosure h).Element where
  mem S := ∃ k : CompactFix h, S = upFix h k ∧ k.1.1 ≤ y.1
  sub := fun ⟨k, hSeq, _⟩ => ⟨k, hSeq⟩
  master_mem := ⟨botK h, rfl, botFix_le h y⟩
  inter_mem := by
    rintro S1 S2 ⟨k1, rfl, hk1y⟩ ⟨k2, rfl, hk2y⟩
    exact ⟨joinCompactFix h hk1y hk2y, upFix_inter h k1 k2 hk1y hk2y, joinCompactFix_le
     h hk1y hk2y⟩
  up_mem := by
    rintro S1 S2 ⟨k1, rfl, hk1y⟩ hS2mem hsub
    obtain ⟨k2, rfl⟩ := hS2mem
    exact ⟨k2, rfl, (mem_upFix h |>.mp (hsub (self_mem_upFix h k1))).trans hk1y⟩

theorem le_of_toElementFilter_le {y y' : FixSet a}
    (hle : toElementFilter h y ≤ toElementFilter h y') : y.1 ≤ y'.1 := by
  intro X hXy
  have hX : E.mem X := y.1.sub hXy
  have hky : (toElementFilter h y).mem (upFix h (genK h hX)) := by
    refine ⟨genK h hX, rfl, ?_⟩
    have h1 : E.principal hX ≤ y.1 := principal_le_of_mem hXy
    have h2 := a.toElementMap_mono h1
    rwa [y.2] at h2
  obtain ⟨k', hkk', hk'y'⟩ := hle _ hky
  have hkeq : (genK h hX).1.1 = k'.1.1 := upFix_injOn h hkk'
  have h1 : E.principal hX ≤ a.toElementMap (E.principal hX) :=
    le_toElementMap_of_isClosureOperator h _
  rw [show a.toElementMap (E.principal hX) = k'.1.1 from hkeq] at h1
  exact (h1.trans hk'y') X ((E.mem_principal hX).mpr ⟨hX, subset_rfl⟩)

theorem toElementFilter_fromElementFilter (g : (domainOfClosure h).Element) :
    toElementFilter h (fromElementFilter h g) = g := by
  apply Element.ext
  intro S
  constructor
  · rintro ⟨k0, rfl, hk0⟩
    obtain ⟨i, hi⟩ := k0.2 (fun i : confirmedIdx h g => i.1.1) (confirmedIdx_directed h
     g) hk0
    exact g.up_mem i.2 ⟨k0, rfl⟩ (upFix_antitone h hi)
  · intro hS
    obtain ⟨k0, rfl⟩ := g.sub hS
    exact ⟨k0, rfl, le_iSupDirected _ (confirmedIdx_directed h g) ⟨k0, hS⟩⟩

theorem toElementFilter_mono {y y' : FixSet a} (hyy' : y.1 ≤ y'.1) :
    toElementFilter h y ≤ toElementFilter h y' :=
  fun _ ⟨k, hSeq, hky⟩ => ⟨k, hSeq, hky.trans hyy'⟩

theorem toElementMap_eq_iSupDirected_genFamily (y : FixSet a) :
    y.1 = iSupDirected (fun i => (genFamily h y i).1) (genFamily_directed h y) := by
```

```
  have h1 : a.toElementMap y.1 = iSupDirected (fun i => (genFamily h y i).1) (
    genFamily_directed h y) := by
    conv_lhs => rw [eq_iSupDirected_principal y.1]
    exact toElementMap_iSupDirected a _ (principalFamily_directed y.1)
  rw [y.2] at h1
  exact h1

theorem fromElementFilter_toElementFilter (y : FixSet a) :
    fromElementFilter h (toElementFilter h y) = y := by
  apply Subtype.ext
  show iSupDirected (fun i : confirmedIdx h (toElementFilter h y) => i.1.1.1)
      (confirmedIdx_directed h (toElementFilter h y)) = y.1
  apply le_antisymm
  · apply iSupDirected_le
    rintro ⟨k, k', hkk', hk'y⟩
    exact (upFix_injOn h hkk' ▸ hk'y : k.1.1 ≤ y.1)
  · rw [toElementMap_eq_iSupDirected_genFamily h y]
    apply iSupDirected_le
    intro i
    have hk : (toElementFilter h y).mem (upFix h (genK h (y.1.sub i.2))) := by
      refine ⟨genK h (y.1.sub i.2), rfl, ?_⟩
      have h1 : E.principal (y.1.sub i.2) ≤ y.1 := principal_le_of_mem i.2
      have h2 := a.toElementMap_mono h1
      rwa [y.2] at h2
    exact le_iSupDirected (fun i : confirmedIdx h (toElementFilter h y) => i.1.1.1)
      (confirmedIdx_directed h (toElementFilter h y)) ⟨genK h (y.1.sub i.2), hk⟩

def fixSetOrderIso (h : ClosureOperator a) : FixSet a ≃o (domainOfClosure h).Element
    where
  toFun := toElementFilter h
  invFun := fromElementFilter h
  left_inv := fromElementFilter_toElementFilter h
  right_inv := toElementFilter_fromElementFilter h
  map_rel_iff' := by
    intro y y'
    exact ⟨le_of_toElementFilter_le h, toElementFilter_mono h⟩

theorem isCompactFix_iff (y : FixSet a) :
    IsCompactFix h y ↔ ∃ (X : Set α) (hX : E.mem X), y.1 = a.toElementMap (E.principal
     hX) := by
  constructor
  · intro hc
    obtain ⟨i, hi⟩ := hc (genFamily h y) (genFamily_directed h y)
      (le_of_eq (toElementMap_eq_iSupDirected_genFamily h y))
    have hX : E.mem i.1 := y.1.sub i.2
    refine ⟨i.1, hX, le_antisymm hi ?_⟩
    have h1 : E.principal hX ≤ y.1 := principal_le_of_mem i.2
    have h2 := a.toElementMap_mono h1
    rwa [y.2] at h2
  · rintro ⟨X, hX, hy⟩
    exact hy ▸ isCompactFix_toElementMap_principal h hX

theorem isFinitary_of_isClosureOperator (h : ClosureOperator a) : IsFinitary a :=
  ⟨CompactFix h, domainOfClosure h, ⟨fixSetOrderIso h⟩⟩

end FiniteDomain
```

```
end

open NeighborhoodSystem ApproximableMap

theorem example_814 :
    (∀ {α : Type*} {E : NeighborhoodSystem α} {a : ApproximableMap E E}
      (h : ClosureOperator a) (y : FixSet a),
      IsCompactFix h y ↔
        ∃ (X : Set α) (hX : E.mem X), y.1 = a.toElementMap (E.principal hX)) ∧
    (∀ {α : Type*} {E : NeighborhoodSystem α} {a : ApproximableMap E E},
      ClosureOperator a → IsFinitary a) :=
  ⟨@isCompactFix_iff, @isFinitary_of_isClosureOperator⟩

end DomainTheory.NeighbourhoodSystems
```

4.9.43 Exercise 8.14 — closure operators

A retraction $a : D \to D$ is called a *closure operator* iff $I_D \sqsubseteq a$. On a domain like $P\mathbf{N}$, give some examples of closure operators. (Hint: Close up a set of integers under addition. Is this continuous on $P\mathbf{N}$?) Prove in general for any closure $a : D \to D$ that the fixed-point set of a is always a finitary domain. (Hint: Show that the fixed-point set is closed under intersections and directed unions.) What are the finite elements of the fixed-point set?

The development is with the Closure operator domain peer.

Theorem (`example_814`). Every closure operator on an effectively given domain is a finitary projection.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_14.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ClosureOperatorDomain

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

theorem example_814 {α : Type*} {E : NeighborhoodSystem α} {a : ApproximableMap E E}
    (h : ClosureOperator a) : IsFinitary a :=
  isFinitary_of_isClosureOperator h
```

4.9.44 Finitely generated subsystems

This peer constructs, from a finite list of basic neighbourhood indices, the finitely generated subsystem it determines and then a domain whose elements encode all subsystems of the original neighbourhood system.

Definition (`combineSub`). Given a list and two of its sublists, their combination retains exactly those entries of the original list that occur in at least one sublist.

Theorem (`combineSub_sublist`). The combined list is a sublist of the original list.

Proof. It is obtained by filtering the original list. ■

Theorem (`mem_combineSub`). An index belongs to the combined list exactly when it belongs to the original list and to at least one of the two selected sublists.

Proof. This is the membership criterion for the defining filter. ■

Theorem (`interFrom_combineSub_eq`). If both selected lists are sublists of the same list, intersecting the basic neighbourhoods selected by their combination equals the intersection of the two separately generated intersections.

Proof. Membership on either side says that the point lies in the master neighbourhood and in every basic neighbourhood selected by either sublist. ■

Definition (`finGen`). The finite system generated by a list consists of those neighbourhoods of the ambient system that are intersections of the master neighbourhood with the basic neighbourhoods indexed by some sublist.

Theorem (`finGen_mono`). Enlarging the list of generators enlarges its finite generated system.

Proof. Every sublist of the smaller list is also a sublist of the larger one. ■

Theorem (`finGen_sub_mem`). Every member of a finite generated system is an ambient neighbourhood.

Proof. Ambient membership is part of the defining witness. ■

Theorem (`finGen_sub_master`). Every member of a finite generated system is contained in the ambient master neighbourhood.

Proof. Every ambient neighbourhood is contained in the master neighbourhood. ■

Theorem (`master_mem_finGen`). The master neighbourhood belongs to every finite generated system.

Proof. Use the empty sublist, whose generated intersection is the master neighbourhood. ■

Theorem (`principal_mem_finGen`). If an index occurs in the generating list, its basic neighbourhood belongs to the finite generated system.

Proof. Use the singleton sublist containing that index. ■

Definition (`finGenSys`). The finite generated system is made into a neighbourhood system with the ambient master neighbourhood and with the finite intersections generated by sublists as its neighbourhoods.

Theorem (`mem_finGenSys`). Membership in the finite generated neighbourhood system is exactly membership in the corresponding finite generated family.

Proof. This is the definition of its neighbourhood predicate. ■

Theorem (`finGenSys_subsystem`). Every finite generated neighbourhood system is a subsystem of the ambient system.

Proof. Its neighbourhoods are ambient neighbourhoods, and combining sublists witnesses closure under every ambient intersection that exists. ■

Theorem (`interFrom_cons_eq`). The intersection generated by a list beginning with j is the intersection of the j-th basic neighbourhood with the intersection generated by the remaining list.

Proof. Separate the head condition from the conditions indexed by the tail. ■

Theorem (`finGen_le_iff_forall_mem`). For a subsystem of the ambient system, the finite system generated by a list is contained in that subsystem exactly when every basic neighbourhood indexed by the list belongs to the subsystem.

Proof. One direction uses the singleton-generated members. For the converse, induction on the list and subsystem closure under ambient intersections add one generator at a time. ■

Theorem (`finGen_append_le_iff`). The finite system generated by the concatenation of two lists is contained in a subsystem exactly when each of the two separately generated systems is contained in it.

Proof. Apply the generator criterion and split universal membership over concatenation. ■

Definition (`nbhdGen`). The neighbourhood generated by a list is the set of lists whose finite generated systems contain the one generated by the given list.

Theorem (`mem_nbhdGen`). A list belongs to the neighbourhood generated by another list exactly when its finite generated system contains that of the first list.

Proof. This is the defining membership condition. ■

Theorem (`nbhdGen_antitone`). Inclusion of finite generated systems reverses inclusion of their generated neighbourhoods.

Proof. Compose the two inclusions of finite systems. ■

Theorem (`self_mem_nbhdGen`). Every list belongs to the neighbourhood it generates.

Proof. Its finite generated system contains itself. ■

Theorem (`nbhdGen_inter`). The intersection of the neighbourhoods generated by two lists is the neighbourhood generated by their concatenation.

Proof. Membership means containing both finite generated systems, which by the concatenation criterion is equivalent to containing the finite system generated by the concatenated list. ■

Definition (`SubD`). The subsystem domain has finite lists as tokens and takes the generated neighbourhoods as its neighbourhoods; the neighbourhood generated by the empty list is its master neighbourhood.

Theorem (`finGen_eq_of_nbhdGen_eq`). Equal generated neighbourhoods have equal finite generated systems.

Proof. Each generating list belongs to its own neighbourhood, so equality supplies both inclusions between the finite generated systems. ■

Theorem (`nbhdGen_eq_iff`). Two generated neighbourhoods are equal exactly when their finite generated systems are equal.

Proof. The forward implication is the preceding result, and the reverse implication follows directly from the containment definition. ■

Theorem (`mem_SubD`). A set of lists is a neighbourhood of the subsystem domain exactly when it is generated by some finite list.

Proof. This is the neighbourhood predicate of the subsystem domain. ■

Theorem (`SubD_master`). The master neighbourhood of the subsystem domain is generated by the empty list.

Proof. This is its defining master neighbourhood. ■

Theorem (`exists_common_witness`). If an element of the subsystem domain confirms, separately, witnesses that generate every basic neighbourhood indexed by a finite list, then one confirmed list jointly generates all those basic neighbourhoods.

Proof. Induct on the list and concatenate witnesses; filter closure gives confirmation of their intersection, while monotonicity carries the required generators into the concatenation. ■

Theorem (`interFrom_append_eq`). The intersection generated by a concatenated list equals the intersection of the intersections generated by its two parts.

Proof. Membership splits into the conditions indexed by the two component lists. ■

Definition (`ofSubsystem`). A subsystem determines an element of the subsystem domain by confirming a generated neighbourhood exactly when the subsystem contains every indexed basic neighbourhood in its generating list.

Theorem (`mem_ofSubsystem_nbhdGen`). The element associated with a subsystem confirms the neighbourhood generated by a list exactly when the subsystem contains every basic neighbourhood indexed by that list.

Proof. Equality of generated neighbourhoods is converted to equality of finite generated systems, after which the finite-generator criterion applies. ■

Definition (`toSubsystemSys`). An element of the subsystem domain determines a neighbourhood system whose neighbourhoods are members of finite systems generated by lists confirmed by that element.

Theorem (`toSubsystemSys_subsystem`). The neighbourhood system determined by an element of the subsystem domain is a subsystem of the ambient system.

Proof. Its members are ambient neighbourhoods, and two confirmed witnesses can be concatenated to witness closure under ambient intersection. ■

Theorem (`ofSubsystem_toSubsystem`). Passing from an element to its represented subsystem and back returns the original element.

Proof. A confirmed list plainly supplies its indexed generators; conversely, finitely many separate witnesses can be combined into one common confirmed witness and then enlarged to the desired neighbourhood. ■

Theorem (`mem_nbhdGen_iff`). An element confirms the neighbourhood generated by a list exactly when its represented subsystem contains every basic neighbourhood indexed by that list.

Proof. Replace the element by the element reconstructed from its represented subsystem and apply the membership characterization above. ■

Definition (`toSubsystem`). Each element of the subsystem domain is sent to the ambient subsystem it represents.

Theorem (`toSubsystem_ofSubsystem_mem`). Reconstructing a subsystem from its associated element preserves its neighbourhoods exactly.

Proof. A generated witness yields subsystem membership by the finite-generator criterion; conversely, a basic index for the given neighbourhood supplies a singleton witness. ■

Theorem (`toSubsystem_ofSubsystem`). Passing from a subsystem to its associated element and back returns the original subsystem.

Proof. Their neighbourhood predicates agree by the preceding theorem, and their master neighbourhoods are the same. ■

`finGen.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem Domain.Recursive

variable {α : Type*} {D : NeighborhoodSystem α}
variable (P : ComputablePresentation D)

def combineSub (js sub1 sub2 : List ℕ) : List ℕ :=
  js.filter (fun j => decide (j ∈ sub1 ∨ j ∈ sub2))

theorem combineSub_sublist (js sub1 sub2 : List ℕ) :
    (combineSub js sub1 sub2).Sublist js := List.filter_sublist
```

```
theorem mem_combineSub {js sub1 sub2 : List ℕ} {j : ℕ} :
    j ∈ combineSub js sub1 sub2 ↔ j ∈ js ∧ (j ∈ sub1 ∨ j ∈ sub2) := by
  unfold combineSub
  rw [List.mem_filter, decide_eq_true_iff]

theorem interFrom_combineSub_eq {js sub1 sub2 : List ℕ}
    (h1 : sub1.Sublist js) (h2 : sub2.Sublist js) :
    interFrom P D.master (combineSub js sub1 sub2) =
      interFrom P D.master sub1 ∩ interFrom P D.master sub2 := by
  apply Set.ext
  intro z
  rw [Set.mem_inter_iff, mem_interFrom, mem_interFrom, mem_interFrom]
  constructor
  · rintro ⟨hzm, hall⟩
    exact ⟨⟨hzm, fun j hj => hall j (mem_combineSub.mpr ⟨h1.subset hj, Or.inl hj⟩)⟩,
      hzm, fun j hj => hall j (mem_combineSub.mpr ⟨h2.subset hj, Or.inr hj⟩)⟩
  · rintro ⟨⟨hzm, hall1⟩, _, hall2⟩
    refine ⟨hzm, fun j hj => ?_⟩
    obtain ⟨_, hj1 | hj2⟩ := mem_combineSub.mp hj
    · exact hall1 j hj1
    · exact hall2 j hj2

def finGen (js : List ℕ) : Set (Set α) :=
  {Y | ∃ sub : List ℕ, sub.Sublist js ∧ D.mem Y ∧ Y = interFrom P D.master sub}

theorem finGen_mono {js js' : List ℕ} (h : js.Sublist js') : finGen P js ⊆ finGen P js
    ' :=
  fun _ ⟨sub, hsub, hD, hY⟩ => ⟨sub, hsub.trans h, hD, hY⟩

theorem finGen_sub_mem {js : List ℕ} {Y : Set α} : Y ∈ finGen P js → D.mem Y :=
  fun ⟨_, _, hD, _⟩ => hD

theorem finGen_sub_master {js : List ℕ} {Y : Set α} (h : Y ∈ finGen P js) : Y ⊆ D.
    master :=
  D.sub_master (finGen_sub_mem P h)

theorem master_mem_finGen (js : List ℕ) : D.master ∈ finGen P js :=
  ⟨[], List.nil_sublist js, D.master_mem, rfl⟩

theorem principal_mem_finGen {js : List ℕ} {i : ℕ} (hi : i ∈ js) : P.X i ∈ finGen P js
     := by
  refine ⟨[i], (List.singleton_sublist).mpr hi, P.mem_X i, ?_⟩
  apply Set.ext
  intro z
  rw [mem_interFrom]
  constructor
  · intro hz
    exact ⟨D.sub_master (P.mem_X i) hz,
      fun j hj => by rw [List.mem_singleton] at hj; subst hj; exact hz⟩
  · rintro ⟨_, hall⟩
    exact hall i (List.mem_singleton.mpr rfl)

def finGenSys (js : List ℕ) : NeighborhoodSystem α where
  mem Y := Y ∈ finGen P js
  master := D.master
  master_nonempty := D.master_nonempty
```

```
  master_mem := master_mem_finGen P js
  inter_mem := by
    rintro X Y Z ⟨sub1, h1, hDX, hX⟩ ⟨sub2, h2, hDY, hY⟩ ⟨sub3, h3, hDZ, hZ⟩ hZsub
    exact ⟨combineSub js sub1 sub2, combineSub_sublist js sub1 sub2,
      D.inter_mem hDX hDY hDZ hZsub, by
        rw [interFrom_combineSub_eq P h1 h2, hX, hY]⟩
  sub_master := fun ⟨sub, _, _, hY⟩ => hY ▸ interFrom_subset P

@[simp] theorem mem_finGenSys {js : List ℕ} {Y : Set α} :
    (finGenSys P js).mem Y ↔ Y ∈ finGen P js := Iff.rfl

theorem finGenSys_subsystem (js : List ℕ) : finGenSys P js ≼ D where
  master_eq := rfl
  sub := finGen_sub_mem P
  inter_closed := by
    rintro X Y ⟨sub1, h1, hDX, hX⟩ ⟨sub2, h2, hDY, hY⟩ hXY
    exact ⟨combineSub js sub1 sub2, combineSub_sublist js sub1 sub2, hXY, by
      rw [interFrom_combineSub_eq P h1 h2, hX, hY]⟩

theorem interFrom_cons_eq (A : Set α) (j : ℕ) (sub : List ℕ) :
    interFrom P A (j :: sub) = P.X j ∩ interFrom P A sub := by
  apply Set.ext
  intro z
  rw [Set.mem_inter_iff, mem_interFrom, mem_interFrom, List.forall_mem_cons]
  tauto

theorem finGen_le_iff_forall_mem {D' : NeighborhoodSystem α} (hD' : D' ≼ D) (js : List
    ℕ) :
    finGen P js ⊆ Set.ofPred D'.mem ↔ ∀ i ∈ js, D'.mem (P.X i) := by
  induction js with
  | nil =>
    refine ⟨fun _ i hi => absurd hi List.not_mem_nil, fun _ => ?_⟩
    rintro Y ⟨sub, hsub, _, hYeq⟩
    rw [List.eq_nil_of_sublist_nil hsub] at hYeq
    rw [hYeq, show interFrom P D.master ([] : List ℕ) = D.master from rfl, ← hD'.
   master_eq]
    exact D'.master_mem
  | cons j js' ih =>
    rw [List.forall_mem_cons]
    constructor
    · intro hsub
      refine ⟨hsub (principal_mem_finGen P List.mem_cons_self), ?_⟩
      exact ih.mp ((finGen_mono P (List.sublist_cons_self j js')).trans hsub)
    · intro hall
      have hj := hall.1
      have hFinjs' : finGen P js' ⊆ Set.ofPred D'.mem := ih.mpr hall.2
      rintro Y ⟨sub, hsub, hDY, hYeq⟩
      cases hsub with
      | cons _ hsub' => exact hFinjs' ⟨sub, hsub', hDY, hYeq⟩
      | cons_cons _ hsub' =>
        rename_i sub2
        rw [interFrom_cons_eq P D.master j sub2] at hYeq
        set Y2 := interFrom P D.master sub2 with hY2def
        have hY_sub_Y2 : Y ⊆ Y2 := hYeq ▸ Set.inter_subset_right
        have hDY2 : D.mem Y2 := interFrom_mem_of_witness P hDY hY_sub_Y2 D.master_mem
        have hY2mem : D'.mem Y2 := hFinjs' ⟨sub2, hsub', hDY2, rfl⟩
```

```
        have hXYmem : D.mem (P.X j ∩ Y2) := hYeq ▸ hDY
        exact hYeq ▸ hD'.inter_closed hj hY2mem hXYmem

theorem finGen_append_le_iff {D' : NeighborhoodSystem α} (hD' : D' ? D) (js1 js2 :
    List ℕ) :
    finGen P (js1 ++ js2) ⊆ Set.ofPred D'.mem ↔
      finGen P js1 ⊆ Set.ofPred D'.mem ∧ finGen P js2 ⊆ Set.ofPred D'.mem := by
  rw [finGen_le_iff_forall_mem P hD', finGen_le_iff_forall_mem P hD',
    finGen_le_iff_forall_mem P hD', List.forall_mem_append]

def nbhdGen (js : List ℕ) : Set (List ℕ) := {js' | finGen P js ⊆ finGen P js'}

@[simp] theorem mem_nbhdGen {js js' : List ℕ} :
    js' ∈ nbhdGen P js ↔ finGen P js ⊆ finGen P js' := Iff.rfl

theorem nbhdGen_antitone {js js' : List ℕ} (h : finGen P js ⊆ finGen P js') :
    nbhdGen P js' ⊆ nbhdGen P js := fun _ hk => h.trans hk

theorem self_mem_nbhdGen (js : List ℕ) : js ∈ nbhdGen P js := (mem_nbhdGen P).mpr
    subset_rfl

theorem nbhdGen_inter (js1 js2 : List ℕ) :
    nbhdGen P js1 ∩ nbhdGen P js2 = nbhdGen P (js1 ++ js2) := by
  apply Set.ext
  intro js'
  simp only [mem_nbhdGen, Set.mem_inter_iff]
  exact (finGen_append_le_iff P (finGenSys_subsystem P js') js1 js2).symm

def SubD (P : ComputablePresentation D) : NeighborhoodSystem (List ℕ) where
  mem N := ∃ js, N = nbhdGen P js
  master := nbhdGen P []
  master_nonempty := ⟨[], self_mem_nbhdGen P []⟩
  master_mem := ⟨[], rfl⟩
  inter_mem := by
    rintro N1 N2 N3 ⟨js1, rfl⟩ ⟨js2, rfl⟩ ⟨js3, rfl⟩ _
    exact ⟨js1 ++ js2, nbhdGen_inter P js1 js2⟩
  sub_master := by
    rintro N ⟨js, rfl⟩
    exact nbhdGen_antitone P (finGen_mono P (List.nil_sublist js))

theorem finGen_eq_of_nbhdGen_eq {a b : List ℕ} (h : nbhdGen P a = nbhdGen P b) :
    finGen P a = finGen P b :=
  Set.Subset.antisymm ((Set.ext_iff.mp h b).mpr (self_mem_nbhdGen P b))
    ((Set.ext_iff.mp h a).mp (self_mem_nbhdGen P a))

theorem nbhdGen_eq_iff {a b : List ℕ} : nbhdGen P a = nbhdGen P b ↔ finGen P a =
    finGen P b := by
  refine ⟨finGen_eq_of_nbhdGen_eq P, fun h => ?_⟩
  unfold nbhdGen
  rw [h]

@[simp] theorem mem_SubD {N : Set (List ℕ)} : (SubD P).mem N ↔ ∃ js, N = nbhdGen P js
    := Iff.rfl

@[simp] theorem SubD_master : (SubD P).master = nbhdGen P [] := rfl
```

```
theorem exists_common_witness (x : (SubD P).Element) {js : List ℕ}
    (h : ∀ i ∈ js, ∃ js', x.mem (nbhdGen P js') ∧ P.X i ∈ finGen P js') :
    ∃ js0, x.mem (nbhdGen P js0) ∧ finGen P js ⊆ finGen P js0 := by
  induction js with
  | nil => exact ⟨[], SubD_master P ▸ x.master_mem, subset_rfl⟩
  | cons j js' ih =>
    obtain ⟨jsj, hxj, hjmem⟩ := h j List.mem_cons_self
    obtain ⟨js0', hx0', hsub0'⟩ := ih (fun i hi => h i (List.mem_cons_of_mem j hi))
    refine ⟨jsj ++ js0', nbhdGen_inter P jsj js0' ▸ x.inter_mem hxj hx0',
      (finGen_le_iff_forall_mem P (finGenSys_subsystem P (jsj ++ js0')) (j :: js')).
    mpr
        (List.forall_mem_cons.mpr ⟨?_, ?_⟩)⟩
    · exact finGen_mono P (List.sublist_append_left jsj js0') hjmem
    · exact fun i hi =>
        finGen_mono P (List.sublist_append_right jsj js0') (hsub0' (
    principal_mem_finGen P hi))

theorem interFrom_append_eq (A : Set α) (l1 l2 : List ℕ) :
    interFrom P A (l1 ++ l2) = interFrom P A l1 ∩ interFrom P A l2 := by
  apply Set.ext
  intro z
  rw [Set.mem_inter_iff, mem_interFrom, mem_interFrom, mem_interFrom, List.
    forall_mem_append]
  tauto

def ofSubsystem (D' : NeighborhoodSystem α) (hD' : D' ? D) : (SubD P).Element where
  mem N := ∃ js, N = nbhdGen P js ∧ ∀ i ∈ js, D'.mem (P.X i)
  sub := fun ⟨js, hN, _⟩ => ⟨js, hN⟩
  master_mem := ⟨[], rfl, fun i hi => absurd hi List.not_mem_nil⟩
  inter_mem := by
    rintro N1 N2 ⟨js1, rfl, h1⟩ ⟨js2, rfl, h2⟩
    exact ⟨js1 ++ js2, nbhdGen_inter P js1 js2, List.forall_mem_append.mpr ⟨h1, h2⟩⟩
  up_mem := by
    rintro N1 N2 ⟨js1, rfl, h1⟩ ⟨js2, rfl⟩ hsub
    have hle : finGen P js2 ⊆ finGen P js1 := hsub (self_mem_nbhdGen P js1)
    exact ⟨js2, rfl, (finGen_le_iff_forall_mem P hD' js2).mp
      (hle.trans ((finGen_le_iff_forall_mem P hD' js1).mpr h1))⟩

theorem mem_ofSubsystem_nbhdGen {D' : NeighborhoodSystem α} (hD' : D' ? D) (js : List
    ℕ) :
    (ofSubsystem P D' hD').mem (nbhdGen P js) ↔ ∀ i ∈ js, D'.mem (P.X i) := by
  constructor
  · rintro ⟨js', hjseq, hall⟩
    have hfin : finGen P js = finGen P js' := (nbhdGen_eq_iff P).mp hjseq
    refine (finGen_le_iff_forall_mem P hD' js).mp ?_
    rw [hfin]
    exact (finGen_le_iff_forall_mem P hD' js').mpr hall
  · exact fun hall => ⟨js, rfl, hall⟩

def toSubsystemSys (x : (SubD P).Element) : NeighborhoodSystem α where
  mem Y := ∃ js, x.mem (nbhdGen P js) ∧ Y ∈ finGen P js
  master := D.master
  master_nonempty := D.master_nonempty
  master_mem := ⟨[], x.master_mem, master_mem_finGen P []⟩
  inter_mem := by
    rintro X Y Z ⟨js1, hx1, hX⟩ ⟨js2, hx2, hY⟩ ⟨js3, _, hZ⟩ hZsub
```

```
    have hDX := finGen_sub_mem P hX
    have hDY := finGen_sub_mem P hY
    have hDZ := finGen_sub_mem P hZ
    obtain ⟨sub1, hsub1, _, hXeq⟩ := hX
    obtain ⟨sub2, hsub2, _, hYeq⟩ := hY
    refine ⟨js1 ++ js2, nbhdGen_inter P js1 js2 ▸ x.inter_mem hx1 hx2,
      sub1 ++ sub2, hsub1.append hsub2, D.inter_mem hDX hDY hDZ hZsub, ?_⟩
    rw [interFrom_append_eq, hXeq, hYeq]
  sub_master := fun ⟨_, _, hY⟩ => finGen_sub_master P hY

theorem toSubsystemSys_subsystem (x : (SubD P).Element) : toSubsystemSys P x ? D where
  master_eq := rfl
  sub := fun ⟨_, _, hY⟩ => finGen_sub_mem P hY
  inter_closed := by
    rintro X Y ⟨js1, hx1, hX⟩ ⟨js2, hx2, hY⟩ hXY
    obtain ⟨sub1, hsub1, _, hXeq⟩ := hX
    obtain ⟨sub2, hsub2, _, hYeq⟩ := hY
    refine ⟨js1 ++ js2, nbhdGen_inter P js1 js2 ▸ x.inter_mem hx1 hx2,
      sub1 ++ sub2, hsub1.append hsub2, hXY, ?_⟩
    rw [interFrom_append_eq, hXeq, hYeq]

theorem ofSubsystem_toSubsystem (x : (SubD P).Element) :
    ofSubsystem P (toSubsystemSys P x) (toSubsystemSys_subsystem P x) = x := by
  apply Element.ext
  intro N
  constructor
  · rintro ⟨js, rfl, hall⟩
    obtain ⟨js0, hx0, hsub0⟩ := exists_common_witness P x hall
    exact x.up_mem hx0 ⟨js, rfl⟩ (nbhdGen_antitone P hsub0)
  · intro hx
    obtain ⟨js, rfl⟩ := x.sub hx
    exact ⟨js, rfl, fun i hi => ⟨js, hx, principal_mem_finGen P hi⟩⟩

theorem mem_nbhdGen_iff (x : (SubD P).Element) (js : List ℕ) :
    x.mem (nbhdGen P js) ↔ ∀ i ∈ js, (toSubsystemSys P x).mem (P.X i) := by
  conv_lhs => rw [← ofSubsystem_toSubsystem P x]
  exact mem_ofSubsystem_nbhdGen P (toSubsystemSys_subsystem P x) js

def toSubsystem (x : (SubD P).Element) : {D' : NeighborhoodSystem α // D' ? D} :=
  ⟨toSubsystemSys P x, toSubsystemSys_subsystem P x⟩

theorem toSubsystem_ofSubsystem_mem (D' : NeighborhoodSystem α) (hD' : D' ? D) (Y :
    Set α) :
    (toSubsystemSys P (ofSubsystem P D' hD')).mem Y ↔ D'.mem Y := by
  constructor
  · rintro ⟨js, ⟨js', hjseq, hall⟩, hY⟩
    have hfin : finGen P js = finGen P js' := (nbhdGen_eq_iff P).mp hjseq
    exact (finGen_le_iff_forall_mem P hD' js').mpr hall (hfin ▸ hY)
  · intro hY
    obtain ⟨n, hn⟩ := P.surj (hD'.sub hY)
    refine ⟨[n], ⟨[n], rfl, fun i hi => ?_⟩, hn ▸ principal_mem_finGen P (List.
    mem_singleton.mpr rfl)⟩
    rw [List.mem_singleton] at hi
    subst hi
    rw [hn]
    exact hY
```

```
theorem toSubsystem_ofSubsystem (D' : NeighborhoodSystem α) (hD' : D' ? D) :
    toSubsystem P (ofSubsystem P D' hD') = ⟨D', hD'⟩ :=
  Subtype.ext (NeighborhoodSystem.ext (toSubsystem_ofSubsystem_mem P D' hD') hD'.
    master_eq.symm)

end DomainTheory.NeighbourhoodSystems
```

4.9.45 Subsystem domain

This peer gives the subsystem domain an effective presentation, identifies its elements with subsystems, and characterizes which represented subsystems are effectively given.

Definition (`SubDX`). The n-th basic neighbourhood of the effective subsystem domain is the neighbourhood generated by the finite list decoded from n.

Theorem (`SubDX_inter_eq`). The basic neighbourhood indexed by the code for concatenating two decoded lists is the intersection of the two corresponding basic neighbourhoods.

Proof. Concatenation of lists represents intersection of their generated neighbourhoods. ■

Theorem (`SubDX_masterIdx_spec`). The code of the empty list indexes the master neighbourhood of the subsystem domain.

Proof. Decoding the encoded empty list returns the empty list. ■

Theorem (`SubDX_mem`). Every indexed set is a neighbourhood of the subsystem domain.

Proof. It is generated by its decoded list. ■

Theorem (`SubDX_surj`). Every neighbourhood of the subsystem domain has an index.

Proof. Encode a finite list that generates the given neighbourhood. ■

Theorem (`SubD_cons_computable`). It is decidable, uniformly in two indices, whether some indexed neighbourhood is contained in their intersection.

Proof. The code of the concatenated decoded lists always indexes exactly that intersection. ■

Theorem (`consUpd_foldl_idx`). Iterating the consistency-state update along a list produces, in its index component, the successive intersection of the projected entries selected by the binary mask.

Proof. Induction on the list separates the selected and unselected head cases; only a selected head contributes the next intersection. ■

Definition (`idxCharAt`). The characteristic index at a coded pair is the index component obtained by folding the consistency-state transition over the decoded list, starting from the master index.

Theorem (`idxCharAt_eq_idxchain`). The characteristic index equals the canonical index of the intersection generated by the entries selected from the decoded list.

Proof. The fold calculation identifies its index component with the successive intersection defining the canonical index. ■

Theorem (`primrec_idxCharAt`). If the consistency characteristic is primitive recursive, then so is the characteristic-index function.

Proof. Primitive recursion is preserved by the coded fold, pairing, projection, and composition operations used in the definition. ■

Theorem (`mem_finGen_computable`). Membership of an indexed ambient neighbourhood in the finite system generated by a decoded list is uniformly decidable.

Proof. Search the finitely many binary masks for a selected sublist whose intersection is an ambient neighbourhood and has the required canonical index; all constituent tests are decidable. ■

Theorem (`finGen_subset_computable`). Inclusion between the finite systems generated by two decoded lists is uniformly decidable.

Proof. Such inclusion holds exactly when every generator in the first finite list belongs to the second finite system, reducing the question to a bounded finite universal test. ■

Theorem (`SubD_interEq_computable`). Equality of the intersection of two indexed subsystem neighbourhoods with a third indexed neighbourhood is uniformly decidable.

Proof. Convert neighbourhood equality into equality of the associated finite generated systems, express equality as mutual inclusion, and use decidability of inclusion. ■

Definition (`SubDPresentation`). The subsystem domain receives the computable presentation whose indexing decodes finite lists, whose intersection operation concatenates their codes, and whose master index is the code of the empty list.

Definition (`subsystemIso`). Elements of the subsystem domain and subsystems of the ambient domain are order-isomorphic.

Theorem (`primrec_encodeList_singleton`). Encoding a singleton list is a primitive-recursive function of its entry.

Proof. The singleton code is obtained from pairing with the empty-list code followed by successor. ■

Theorem (`subsystemU_computable_iff_index_set_re`). An element of the effective subsystem domain is computable exactly when the set of indices of ambient basic neighbourhoods belonging to its represented subsystem is recursively enumerable.

Proof. Confirmation of a generated neighbourhood is equivalent to membership of all indices in its decoded finite list. Singleton coding gives one direction, and effective finite universal quantification over decoded lists gives the other. ■

Theorem (`subsystemU_element_computable_iff`). An element of the effective subsystem domain is computable exactly when the basic-neighbourhood indices of its represented subsystem form the range of a primitive-recursive function.

Proof. The index set is nonempty because it contains the master index, so the standard range characterization of nonempty recursively enumerable predicates applies. ■

Theorem (`subsystemU_effectivelyGiven_of_isComputableElement`). Every computable element of the universal subsystem domain represents an effectively given subsystem.

Proof. A primitive-recursive enumeration of its basic-neighbourhood index set supplies the required effective presentation. ■

Theorem (`subsystem_isEffectivelyGiven_of_isEffectivelyGiven`). For every effectively given neighbourhood system there is an effectively given neighbourhood system whose elements are order-isomorphic to its subsystems.

Proof. Choose a computable presentation, equip its subsystem domain with the effective presentation above, and use the subsystem order isomorphism. ■

SubsystemDomain.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.finGen
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.TreeVal
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    TreeVal_more
```

```
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    VComputablePresentation

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import Mathlib.Data.Nat.Bitwise
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive
variable {α : Type*} {D : NeighborhoodSystem α}

section Computability

variable (P : ComputablePresentation D)

def SubDX (n : ℕ) : Set (List ℕ) := nbhdGen P (decodeList n)

theorem SubDX_inter_eq (n m : ℕ) :
    SubDX P (appendListCode n m) = SubDX P n ∩ SubDX P m := by
  show nbhdGen P (decodeList (appendListCode n m))
      = nbhdGen P (decodeList n) ∩ nbhdGen P (decodeList m)
  rw [appendListCode_eq, nbhdGen_inter]

theorem SubDX_masterIdx_spec : SubDX P (encodeList ([] : List ℕ)) = (SubD P).master :=
    by
  show nbhdGen P (decodeList (encodeList ([] : List ℕ))) = nbhdGen P []
  rw [decodeList_encodeList]

theorem SubDX_mem (n : ℕ) : (SubD P).mem (SubDX P n) := ⟨decodeList n, rfl⟩

theorem SubDX_surj {N : Set (List ℕ)} (hN : (SubD P).mem N) : ∃ n, SubDX P n = N := by
  obtain ⟨js, rfl⟩ := hN
  exact ⟨encodeList js, by unfold SubDX; rw [decodeList_encodeList]⟩

theorem SubD_cons_computable :
    RecDecidable₂ (fun n m => ∃ k, SubDX P k ⊆ SubDX P n ∩ SubDX P m) := by
  refine recDecidable_of_forall (fun t => ⟨appendListCode t.unpair.1 t.unpair.2, ?_⟩)
  rw [SubDX_inter_eq]

theorem consUpd_foldl_idx (projFn fc : ℕ → ℕ) (el : List ℕ) (b a flag : ℕ) :
    (List.foldl (fun s x => consUpd P projFn fc s x) (Nat.pair b (Nat.pair a flag)) el
    ).unpair.2.unpair.1
      = (List.map projFn (bitSelect el b)).foldl (fun acc j => P.inter acc j) a := by
  induction el generalizing b a flag with
  | nil => rw [bitSelect_nil, List.foldl_nil, List.map_nil, List.foldl_nil,
    unpair_pair_snd,
      unpair_pair_fst]
  | cons e el ih =>
    rw [List.foldl_cons, consUpd_eval, bitSelect_cons]
    by_cases hb : b % 2 = 1
```

```
    · rw [ite_eq_left hb, List.map_cons, List.foldl_cons, hb, selectFn_one]
      exact ih (b / 2) (P.inter a (projFn e)) _
    · rw [ite_eq_right hb, show b % 2 = 0 by omega, selectFn_zero]
      exact ih (b / 2) a flag

def idxCharAt (fc : ℕ → ℕ) (w : ℕ) : ℕ :=
  (foldCode (consStp P id fc) 0 (Nat.pair w.unpair.1 (Nat.pair P.masterIdx 1))
    w.unpair.2).unpair.2.unpair.1

theorem idxCharAt_eq_idxchain (fc : ℕ → ℕ) (w : ℕ) :
    idxCharAt P fc w = idxchain P (bitSelect (decodeList w.unpair.2) w.unpair.1) := by
  unfold idxCharAt idxchain
  rw [foldCode_eq']
  have hstep : (fun acc x => consStp P id fc (Nat.pair x (Nat.pair acc 0)))
      = (fun acc x => consUpd P id fc acc x) := by
    funext acc x
    show consStp P id fc (Nat.pair x (Nat.pair acc 0)) = _
    unfold consStp
    simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hstep, consUpd_foldl_idx P id fc (decodeList w.unpair.2) w.unpair.1 P.masterIdx
    1,
    List.map_id]

theorem primrec_idxCharAt {fc : ℕ → ℕ} (hfcp : Nat.Primrec fc) :
    Nat.Primrec (idxCharAt P fc) := by
  have hfold := primrec_foldCode (primrec_consStp P primrec_id hfcp) (Nat.Primrec.
    const 0)
    (Nat.Primrec.left.pair ((Nat.Primrec.const P.masterIdx).pair (Nat.Primrec.const 1)
    ))
    Nat.Primrec.right
  exact Nat.Primrec.left.comp (Nat.Primrec.right.comp hfold)

theorem mem_finGen_computable :
    RecDecidable₂ (fun m c => P.X m ∈ finGen P (decodeList c)) := by
  obtain ⟨fc, hfcp, hfcs⟩ := P.cons_computable
  have hfc : ∀ s, fc s = 1 ↔ ∃ k, P.X k ⊆ P.X s.unpair.1 ∩ P.X s.unpair.2 := fun s =>
    (hfcs s).symm
  have hconsFold := consFold_decidable P id fc hfc primrec_id hfcp
  have hg : Nat.Primrec (fun w : ℕ => Nat.pair w.unpair.1 w.unpair.2.unpair.2) :=
    Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)
  have hp1 : RecDecidable (fun w : ℕ =>
      D.mem (interFrom P D.master (bitSelect (decodeList w.unpair.2.unpair.2) w.unpair
    .1))) := by
    refine RecDecidable.of_iff (fun w => ?_) (hconsFold.comp hg)
    simp only [unpair_pair_fst, unpair_pair_snd, List.map_id]
  have hidx : Nat.Primrec (fun w : ℕ => idxCharAt P fc (Nat.pair w.unpair.1 w.unpair
    .2.unpair.2)) :=
    (primrec_idxCharAt P hfcp).comp hg
  have hh : Nat.Primrec (fun w : ℕ =>
      Nat.pair w.unpair.2.unpair.1 (idxCharAt P fc (Nat.pair w.unpair.1 w.unpair.2.
    unpair.2))) :=
    (Nat.Primrec.left.comp Nat.Primrec.right).pair hidx
  have hp2 : RecDecidable (fun w : ℕ =>
      P.X w.unpair.2.unpair.1 = P.X (idxCharAt P fc (Nat.pair w.unpair.1 w.unpair.2.
    unpair.2))) := by
    refine RecDecidable.of_iff (fun w => ?_) (P.eq_computable.comp hh)
```

```
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hp := hp1.and hp2
  have hbound : Nat.Primrec (fun n : ℕ => 2 ^ n.unpair.2) := primrec_two_pow Nat.
    Primrec.right
  have hbe := hp.bExists (bound := fun n : ℕ => 2 ^ n.unpair.2) hbound
  refine RecDecidable.of_iff (fun t => ?_) hbe
  simp only [unpair_pair_fst, unpair_pair_snd]
  constructor
  · rintro ⟨sub, hsub, -, hYeq⟩
    obtain ⟨b, hb, hbeq⟩ := exists_bitSelect_lt hsub
    have hb' : b < 2 ^ t.unpair.2 :=
      lt_of_lt_of_le hb (Nat.pow_le_pow_right (by omega) (decodeList_length_le t.
    unpair.2))
    have hDsub : D.mem (interFrom P D.master sub) := hYeq ▸ P.mem_X t.unpair.1
    refine ⟨b, hb', hbeq ▸ hDsub, ?_⟩
    rw [idxCharAt_eq_idxchain]
    simp only [unpair_pair_fst, unpair_pair_snd]
    rw [hbeq, idxchain_spec P hDsub, ← hYeq]
  · rintro ⟨b, -, hDb, hXeq⟩
    refine ⟨bitSelect (decodeList t.unpair.2) b, bitSelect_sublist _ _, P.mem_X t.
    unpair.1, ?_⟩
    rw [hXeq, idxCharAt_eq_idxchain]
    simp only [unpair_pair_fst, unpair_pair_snd]
    rw [idxchain_spec P hDb]

theorem finGen_subset_computable :
    RecDecidable₂ (fun c1 c2 => finGen P (decodeList c1) ⊆ finGen P (decodeList c2))
    := by
  have hiff : ∀ c1 c2, finGen P (decodeList c1) ⊆ finGen P (decodeList c2) ↔
      ∀ i ∈ decodeList c1, P.X i ∈ finGen P (decodeList c2) :=
    fun c1 c2 => finGen_le_iff_forall_mem P (finGenSys_subsystem P (decodeList c2)) (
    decodeList c1)
  refine RecDecidable.of_iff (fun t => ?_) (mem_finGen_computable P).bForallList
  exact hiff t.unpair.1 t.unpair.2

theorem SubD_interEq_computable :
    RecDecidable₃ (fun n m k => SubDX P n ∩ SubDX P m = SubDX P k) := by
  have hiff : ∀ n m k, SubDX P n ∩ SubDX P m = SubDX P k ↔
      finGen P (decodeList (appendListCode n m)) ⊆ finGen P (decodeList k) ∧
        finGen P (decodeList k) ⊆ finGen P (decodeList (appendListCode n m)) := by
    intro n m k
    show nbhdGen P (decodeList n) ∩ nbhdGen P (decodeList m) = nbhdGen P (decodeList k
    ) ↔ _
    rw [nbhdGen_inter, ← appendListCode_eq, nbhdGen_eq_iff, Set.Subset.antisymm_iff]
  have hg1 : Nat.Primrec (fun t : ℕ => appendListCode t.unpair.1 t.unpair.2.unpair.1)
    :=
    (primrec_appendListCode.comp
        (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right))).of_eq
      (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
  have hreindex1 : Nat.Primrec (fun t : ℕ =>
      Nat.pair (appendListCode t.unpair.1 t.unpair.2.unpair.1) t.unpair.2.unpair.2) :=
    hg1.pair (Nat.Primrec.right.comp Nat.Primrec.right)
  have hreindex2 : Nat.Primrec (fun t : ℕ =>
      Nat.pair t.unpair.2.unpair.2 (appendListCode t.unpair.1 t.unpair.2.unpair.1)) :=
    (Nat.Primrec.right.comp Nat.Primrec.right).pair hg1
  have hforward : RecDecidable (fun t : ℕ =>
```

```
      finGen P (decodeList (appendListCode t.unpair.1 t.unpair.2.unpair.1)) ⊆
        finGen P (decodeList t.unpair.2.unpair.2)) := by
    refine RecDecidable.of_iff (fun t => ?_) ((finGen_subset_computable P).comp
    hreindex1)
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hbackward : RecDecidable (fun t : ℕ =>
      finGen P (decodeList t.unpair.2.unpair.2) ⊆
        finGen P (decodeList (appendListCode t.unpair.1 t.unpair.2.unpair.1))) := by
    refine RecDecidable.of_iff (fun t => ?_) ((finGen_subset_computable P).comp
    hreindex2)
    simp only [unpair_pair_fst, unpair_pair_snd]
  refine RecDecidable.of_iff (fun t => ?_) (hforward.and hbackward)
  exact hiff t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2

def SubDPresentation : ComputablePresentation (SubD P) where
  X := SubDX P
  mem_X := SubDX_mem P
  surj := SubDX_surj P
  interEq_computable := SubD_interEq_computable P
  cons_computable := SubD_cons_computable P
  inter := appendListCode
  inter_primrec := primrec_appendListCode
  inter_spec := fun {n m} _ => SubDX_inter_eq P n m
  masterIdx := encodeList []
  masterIdx_spec := SubDX_masterIdx_spec P

end Computability

section OrderIso

variable (P : ComputablePresentation D)

def subsystemIso : (SubD P).Element ≃o {D' : NeighborhoodSystem α // D' ? D} where
  toFun := toSubsystem P
  invFun x := ofSubsystem P x.1 x.2
  left_inv := ofSubsystem_toSubsystem P
  right_inv x := toSubsystem_ofSubsystem P x.1 x.2
  map_rel_iff' := by
    intro a b
    show toSubsystemSys P a ? toSubsystemSys P b ↔ (∀ N, a.mem N → b.mem N)
    rw [Subsystem.subsystem_iff_subset_of_common (toSubsystemSys_subsystem P a)
      (toSubsystemSys_subsystem P b)]
    constructor
    · intro hYsub N hN
      obtain ⟨js, rfl⟩ := a.sub hN
      exact (mem_nbhdGen_iff P b js).mpr
        (fun i hi => hYsub ((mem_nbhdGen_iff P a js).mp hN i hi))
    · rintro hab Y ⟨js, haj, hY⟩
      exact ⟨js, hab (nbhdGen P js) haj, hY⟩

end OrderIso

section ComputableElements

variable (P : ComputablePresentation D)
```

```
theorem primrec_encodeList_singleton : Nat.Primrec (fun i => encodeList [i]) := by
  have : (fun i => encodeList [i]) = fun i => Nat.pair i 0 + 1 := by
    funext i; show Nat.pair i (encodeList ([] : List ℕ)) + 1 = Nat.pair i 0 + 1
    rw [encodeList]
  rw [this]
  exact Nat.Primrec.succ.comp (Nat.Primrec.pair primrec_id (Nat.Primrec.const 0))

theorem subsystemU_computable_iff_index_set_re (x : (SubD P).Element) :
    ComputableElement (SubDPresentation P) x ↔
      REPred (fun i => (toSubsystemSys P x).mem (P.X i)) := by
  have hiff : ∀ n, x.mem (SubDX P n) ↔ ∀ i ∈ decodeList n, (toSubsystemSys P x).mem (P
    .X i) := by
    intro n; unfold SubDX; exact mem_nbhdGen_iff P x (decodeList n)
  constructor
  · intro hx
    have hx' : REPred (fun n => ∀ i ∈ decodeList n, (toSubsystemSys P x).mem (P.X i))
    :=
      REPred.of_iff (fun n => (hiff n).symm) hx.out
    refine REPred.of_iff (fun i => ?_) (hx'.comp primrec_encodeList_singleton)
    rw [decodeList_encodeList]
    simp only [List.mem_singleton]
    constructor
    · intro h j hj; rw [hj]; exact h
    · intro h; exact h i rfl
  · intro hx
    constructor
    exact REPred.of_iff hiff hx.forall_mem_decodeList

theorem subsystemU_element_computable_iff (x : (SubD P).Element) :
    ComputableElement (SubDPresentation P) x ↔
      ∃ r : ℕ → ℕ, Nat.Primrec r ∧
        ∀ i, (toSubsystemSys P x).mem (P.X i) ↔ ∃ n, r n = i := by
  rw [subsystemU_computable_iff_index_set_re P x]
  constructor
  · intro hre
    have ha : (toSubsystemSys P x).mem (P.X P.masterIdx) := by
      rw [P.masterIdx_spec]; exact (toSubsystemSys P x).master_mem
    exact ComputableWitness.repred_exists_primrec_range hre ha
  · rintro ⟨r, hr, hspec⟩
    exact REPred.of_iff hspec (ComputableWitness.repred_range_primrec hr)

end ComputableElements

section UniversalSubsystem
section TreeVal2

variable (r : ℕ → ℕ) (x : (SubD UComputablePresentation).Element)

theorem subsystemU_effectivelyGiven_of_isComputableElement
    (hx : ComputableElement (SubDPresentation UComputablePresentation) x) :
    (toSubsystemSys UComputablePresentation x).IsEffectivelyGiven := by
  obtain ⟨r, hr, hspec⟩ := (subsystemU_element_computable_iff UComputablePresentation x
    ).mp hx
  exact ⟨Q'xPresentation r x hr hspec⟩

end TreeVal2
```

```
end UniversalSubsystem

theorem subsystem_isEffectivelyGiven_of_isEffectivelyGiven (hD : D.IsEffectivelyGiven)
    :
    ∃ V : NeighborhoodSystem (List ℕ), V.IsEffectivelyGiven ∧
      Nonempty (V.Element ≃o {D' : NeighborhoodSystem α // D' ? D}) := by
  obtain ⟨P⟩ := hD
  exact ⟨SubD P, ⟨SubDPresentation P⟩, ⟨subsystemIso P⟩⟩

end DomainTheory.NeighbourhoodSystems
```

4.9.46 Exercise 8.15 — domain of subsystems

Give a direct proof that the domain $\{X \mid X \triangleleft D\}$ is effectively presented if D is. (Hint: The finite elements of the domain correspond exactly to the finite systems $X \triangleleft D$.) In the case of $D = \mathcal{U}$, show that the computable elements of the domain correspond exactly to the effectively presented domains (up to effective isomorphism).

The development is with the Subsystem domain peer.

Theorem (`example_815`). The domain of subsystems of an effectively given domain is effectively given.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_15.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    SubsystemDomain

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

theorem example_815 {α : Type*} {D : NeighborhoodSystem α} (hD : D.IsEffectivelyGiven)
    :
    ∃ V : NeighborhoodSystem (List ℕ), V.IsEffectivelyGiven ∧
      Nonempty (V.Element ≃o {D' : NeighborhoodSystem α // D' ? D}) :=
  subsystem_isEffectivelyGiven_of_isEffectivelyGiven hD
```

4.9.47 Fixed-neighbourhood finitary order

This peer identifies the order on finitary projections with the subsystem order on their fixed neighbourhoods and records the effective part of Scott's exercise.

Theorem (`isFinitaryProjection_le_iff_fixedNbhd_subsystem`). For two finitary projections on the same neighbourhood system, the first projection lies below the second exactly when the fixed-neighbourhood system of the first is a subsystem of that of the second.

Proof. Finitary projections are precisely the fixed points of the operation that extracts the projection generated by fixed neighbourhoods. The order isomorphism between these fixed points and subsystems therefore preserves and reflects the two orders. ■

Theorem (`exercise_8_16_assembly`). The order on finitary projections is characterized by inclusion of their fixed-neighbourhood subsystems, and the fixed-neighbourhood system of every computable approximable map on a computably presented domain is effectively given.

Proof. The first assertion is the preceding order characterization; the second is the effective-presentation theorem for fixed neighbourhoods of a computable map. ■

`FixedNbhdFinitaryOrder.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_2

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Examples_8_4

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open Domain.Recursive NeighborhoodSystem
open NeighborhoodSystem ApproximableMap
section
universe u
variable {α : Type u} {E : NeighborhoodSystem α}

theorem isFinitaryProjection_le_iff_fixedNbhd_subsystem {a b : ApproximableMap E E}
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    a ≤ b ↔ fixedNbhd a ? fixedNbhd b := by
  have hsa : sub a = a := sub_eq_self_iff_isFinitaryProjection.mpr ha
  have hsb : sub b = b := sub_eq_self_iff_isFinitaryProjection.mpr hb
  set fa : {f : ApproximableMap E E // sub f = f} := ⟨a, hsa⟩ with hfa_def
  set fb : {f : ApproximableMap E E // sub f = f} := ⟨b, hsb⟩ with hfb_def
  constructor
  · intro hab
    have hfab : fa ≤ fb := hab
    exact (finitaryProjectionSubsystemEquiv E).monotone hfab
  · intro hD
    have hle : (finitaryProjectionSubsystemEquiv E) fa ≤ (
    finitaryProjectionSubsystemEquiv E) fb :=
      hD
    have hback := (finitaryProjectionSubsystemEquiv E).symm.monotone hle
    rwa [OrderIso.symm_apply_apply, OrderIso.symm_apply_apply] at hback

theorem exercise_8_16_assembly :
    (∀ {a b : ApproximableMap E E}, FinitaryProjection a → FinitaryProjection b →
        (a ≤ b ↔ fixedNbhd a ? fixedNbhd b)) ∧
      (∀ {P : ComputablePresentation E} {a : ApproximableMap E E},
        ComputableMap P P a → (fixedNbhd a).IsEffectivelyGiven) :=
  ⟨fun ha hb => isFinitaryProjection_le_iff_fixedNbhd_subsystem ha hb,
    fun hcomp => fixedNbhd_isEffectivelyGiven hcomp⟩
end
```

```
end DomainTheory.NeighbourhoodSystems
```

4.9.48 Exercise 8.16 — order of projections

NeighbourhoodSystems/RetractsOfTheUniversalDomain/Exercise_8_16.lean

@import "Exercise_8_16.lean"

EXERCISE 8.16. For finitary projections $a : E \to E$, write

$$D_a = \{X \in E \mid X \sqsubseteq aX\}$$

(cf. 8.5.). Show that for any two such projections $a, b : E \to E$ we have

$$a \sqsubseteq b \quad \text{iff} \quad D_a \lhd D_b.$$

(This fills in the gap at the end of the proof of 8.6.) Also finish off the proof of 8.8 by showing that if E is effectively given and $a : E \to E$ is computable, then D_a is effectively given.

The development is with the Fixed-neighbourhood finitary order peer.

Theorem (`example_816`). Finitary projections on fixed neighbourhoods are ordered as claimed.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_16.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    FixedNbhdFinitaryOrder

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap Domain.
    Recursive

theorem example_816 {α : Type*} {E : NeighborhoodSystem α} :
    (∀ {a b : ApproximableMap E E}, FinitaryProjection a → FinitaryProjection b →
        (a ≤ b ↔ fixedNbhd a ? fixedNbhd b)) ∧
      (∀ {P : ComputablePresentation E} {a : ApproximableMap E E},
        ComputableMap P P a → (fixedNbhd a).IsEffectivelyGiven) :=
  @exercise_8_16_assembly α E
```

4.9.49 Embeddings into U

This peer develops universality for neighbourhood systems and records the principal embeddings of function, product, and sum domains into Scott's universal domain.

Definition (`IsUniversal`). A neighbourhood system is universal when every neighbourhood system with countably many neighbourhoods is isomorphic to a subsystem of it.

Proof. This is the defining universal embedding property. ■

Theorem (`trianglelefteq_of_isomorphic_left`). If a domain is isomorphic to a second domain and the second is isomorphic to a subsystem of a third, then the first is isomorphic to a subsystem of the third.

Proof. Compose the first isomorphism with the isomorphism witnessing the given subsystem representation. ■

Theorem (`trianglelefteq_trans`). Being isomorphic to a subsystem is transitive.

Proof. Turn the two subsystem embeddings into projection pairs, compose those pairs, recover a subsystem representation of the composite, and compose its isomorphism with the original one. ■

Theorem (`U_isUniversal`). Scott's domain U is universal for neighbourhood systems with countably many neighbourhoods.

Proof. Every such system has an isomorphic copy whose neighbourhoods form a subsystem of U. ■

Theorem (`funSpaceUU_trianglelefteq_U`). The function-space domain $[U \to U]$ is isomorphic to a subsystem of U.

Proof. The canonical arrow injection and projection form a projection pair, hence exhibit the function space as a retract and therefore as an isomorphic subsystem. ■

Theorem (`isUniversal_of_isomorphic`). Any domain isomorphic to a universal domain is itself universal.

Proof. The isomorphism makes the original universal domain an isomorphic subsystem of the target; transitivity then transports every universal embedding to the target. ■

Theorem (`V.isUniversal`). The effectively presented domain V is universal.

Proof. The effective isomorphism between U and V is in particular a domain isomorphism, so universality transfers from U. ■

Theorem (`prodUU_trianglelefteq_U`). The product domain $U \times U$ is isomorphic to a subsystem of U.

Proof. The canonical product injection and projection form a projection pair, which yields the required subsystem representation. ■

Theorem (`sumUU_trianglelefteq_U`). The separated sum of two copies of U is isomorphic to a subsystem of U.

Proof. The canonical sum injection and projection form a projection pair, using the nonemptiness of every neighbourhood of U. ■

`EmbeddingsIntoU.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Exercise_8_12
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.ProjPair
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_10
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_6
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section

open NeighborhoodSystem ApproximableMap Subsystem
```

```
universe u v w
variable {α : Type u} {β : Type v} {? : Type w}

def IsUniversal {? : Type w} (W : NeighborhoodSystem ?) : Prop :=
  ∀ {α : Type u} (D : NeighborhoodSystem α) [Countable {S : Set α // D.mem S}], D ? W

theorem trianglelefteq_of_isomorphic_left {D : NeighborhoodSystem α} {D' :
    NeighborhoodSystem β}
    {E : NeighborhoodSystem ?} (hDD' : D ≅? D') (hD'E : D' ? E) : D ? E := by
  obtain ⟨F, hFE, ⟨eD'F⟩⟩ := hD'E
  obtain ⟨eDD'⟩ := hDD'
  exact ⟨F, hFE, ⟨eDD'.trans eD'F⟩⟩

theorem trianglelefteq_trans {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}
    {F : NeighborhoodSystem ?} (h1 : D ? E) (h2 : E ? F) : D ? F := by
  obtain ⟨D', hD'E, ⟨eDD'⟩⟩ := h1
  obtain ⟨E', hE'F, ⟨eEE'⟩⟩ := h2
  have pD'F : ProjPair D' F :=
    ((ProjPair.ofSubsystem hD'E).comp (ProjPair.ofIso eEE')).comp (ProjPair.
    ofSubsystem hE'F)
  have hD'F : D' ? F :=
    trianglelefteq_of_projectionPair pD'F.inj pD'F.proj pD'F.proj_comp_inj pD'F.
    inj_comp_proj_le
  exact trianglelefteq_of_isomorphic_left ⟨eDD'⟩ hD'F

theorem U_isUniversal : IsUniversal.{u} U := by
  intro α D _
  obtain ⟨D', hiso, hsub⟩ := theorem_8_8_a D
  exact ⟨D', hsub, hiso⟩

theorem funSpaceUU_trianglelefteq_U : funSpace U U ? U :=
  trianglelefteq_of_projectionPair iArrow jArrow jArrow_comp_iArrow
    iArrow_comp_jArrow_le

theorem isUniversal_of_isomorphic {W : NeighborhoodSystem β} {W' : NeighborhoodSystem
    ?}
    (hW : IsUniversal.{u} W) (hWW' : W ≅? W') : IsUniversal.{u} W' := by
  have hWW'_tri : W ? W' := hWW'.elim fun e => ⟨W', Subsystem.refl W', ⟨e⟩⟩
  intro α D _
  exact trianglelefteq_trans (hW D) hWW'_tri

theorem V.isUniversal : IsUniversal.{u} V :=
  isUniversal_of_isomorphic U_isUniversal ⟨effectiveIso812_UV.toDomainIso⟩

theorem prodUU_trianglelefteq_U : prod U U ? U :=
  trianglelefteq_of_projectionPair iTimes jTimes jTimes_comp_iTimes
    iTimes_comp_jTimes_le

theorem sumUU_trianglelefteq_U : sum U U U_mem_nonempty U_mem_nonempty ? U :=
  trianglelefteq_of_projectionPair iPlus jPlus jPlus_comp_iPlus iPlus_comp_jPlus_le

end

end DomainTheory.NeighbourhoodSystems
```

4.9.50 Embeddings into U (more)

This peer constructs explicit projection pairs for binary products and sums, reducing them to function spaces and to products involving the three-element truth domain.

Definition (`funToPair`). A function from the truth domain to E determines a pair of elements of E by evaluation at truth and falsity.

Proof. Pair the two evaluation maps obtained by adjoining the constant true and false arguments. ■

Definition (`sumToProdT`). An element of the separated sum $E + E$ determines a triple consisting of its truth-valued tag and its two possible component projections.

Proof. Pair the tag map with the pair of the left and right output maps. ■

Definition (`swapProdT`). The product $(E \times E) \times T$ maps to $T \times (E \times E)$ by exchanging its two factors.

Proof. Pair the second projection with the first projection. ■

Definition (`pairToFun`). A pair (x, y) determines the truth-indexed function that returns x at truth, y at falsity, and the least element at the indeterminate truth value.

Proof. Curry the conditional map after exchanging the truth value and the pair. ■

Theorem (`TD_bot_eq_botElt`). The least element of the truth domain is its indeterminate truth element.

Proof. Each is below the other: one by leastness and the other by the defining least-element property of the indeterminate truth value. ■

Theorem (`TD_trichotomy`). Every element of the truth domain is either indeterminate, true, or false.

Proof. This is the element classification for the neighbourhood system generated by two tokens. ■

Theorem (`funSpace_dom_trianglelefteq`). If A is isomorphic to a subsystem of B, then $[A \to C]$ is isomorphic to a subsystem of $[B \to C]$.

Proof. A projection pair from A to B induces, by contravariance in the argument, a projection pair between the two function spaces; the exponential composition and monotonicity laws verify its equations. ■

Theorem (`outMap0_comp_inMap1`). Projecting the left component after injecting a right-hand element into a separated sum gives the constant least element of the left domain.

Proof. A right injection has no left information beyond the master neighbourhood, so its left output is exactly the least element. ■

Theorem (`outMap1_comp_inMap0`). Projecting the right component after injecting a left-hand element into a separated sum gives the constant least element of the right domain.

Proof. A left injection has no right information beyond the master neighbourhood, so its right output is exactly the least element. ■

Theorem (`pair_bot`). Pairing the least elements of two neighbourhood domains gives the least element of their product.

Proof. Every neighbourhood in the pair is forced to be the product master neighbourhood, while leastness supplies the converse order. ■

Theorem (`prod_left_trianglelefteq`). If A is isomorphic to a subsystem of B, then $A \times C$ is isomorphic to a subsystem of $B \times C$.

Proof. Apply the projection pair for A and B to the first coordinate and leave the second coordinate fixed; the product order verifies the retraction and approximation laws coordinatewise. ■

Theorem (`prod_right_trianglelefteq`). If A is isomorphic to a subsystem of B, then $C \times A$ is isomorphic to a subsystem of $C \times B$.

Proof. Commute both products, use preservation in the left coordinate, and compose the resulting subsystem embeddings. ■

Theorem (`toElementMap_bot_le`). If an approximable map relates the source master neighbourhood only to the target master neighbourhood, then the image of the source least element lies below the target least element.

Proof. Membership in the image of the least element reduces to a relation from the source master neighbourhood, which by hypothesis forces the target master neighbourhood. ■

Theorem (`outMap0_bot_le`). The left output of the least element of a separated sum lies below the least element of the left summand.

Proof. The left part of the sum master neighbourhood is the left master neighbourhood, so the preceding least-image criterion applies. ■

Theorem (`outMap1_bot_le`). The right output of the least element of a separated sum lies below the least element of the right summand.

Proof. The right part of the sum master neighbourhood is the right master neighbourhood, so the least-image criterion applies. ■

Theorem (`toElementMap_funToPair`). On elements, the map from truth-indexed functions to pairs sends φ to $(\varphi(\mathsf{true}), \varphi(\mathsf{false}))$.

Proof. Evaluate the two components of the defining paired map. ■

Theorem (`toElementMap_sumToProdT`). On elements, the sum encoding sends w to the pair formed by its tag and the pair of its left and right outputs.

Proof. This follows directly from the element action of paired maps. ■

Theorem (`condSum_comp_sumToProdT`). Decoding the tag-and-components representation of a separated-sum element returns that element.

Proof. The conditional reconstruction theorem selects the component designated by the tag of the original sum element. ■

Theorem (`toElementMap_swapProdT`). The factor-exchange map sends $((x, y), t)$ to $(t, (x, y))$.

Proof. Compute the two product projections of the input pair. ■

Theorem (`toApproxMap_pairToFun`). Applying the truth-indexed function represented by (x, y) to t is the conditional value determined by $(t, (x, y))$.

Proof. Uncurrying the definition reduces evaluation to the conditional map after factor exchange. ■

Theorem (`funToPair_comp_pairToFun`). Encoding a pair as a truth-indexed function and then evaluating it at truth and falsity returns the original pair.

Proof. The conditional laws at truth and falsity recover the first and second coordinates respectively. ■

Theorem (`pairToFun_comp_funToPair_le`). Reconstructing a truth-indexed function from its values at truth and falsity gives a function below the original one.

Proof. By trichotomy of truth values, the reconstructed function agrees at truth and falsity, while at the indeterminate value it yields the least element. ■

Theorem (`prod_trianglelefteq_funSpace_TD`). The product $E \times E$ is isomorphic to a subsystem of the function space $[T \to E]$.

Proof. The pair-to-function and function-to-pair maps satisfy the retraction equation and the required approximation inequality. ■

Theorem (`prod_trianglelefteq_of`). If the truth domain embeds as an isomorphic subsystem of E, and $[E \to E]$ does likewise, then $E \times E$ is isomorphic to a subsystem of E.

Proof. Embed the product into $[T \to E]$, enlarge the function-space argument from T to E, and then use the assumed function-space embedding. ■

Theorem (`which_bot_le`). The tag of the least separated-sum element lies below the indeterminate truth value.

Proof. The tag map sends the sum master neighbourhood only to the truth master neighbourhood, so the least-image criterion applies. ■

Theorem (`which_inMap0_le_trueElt`). The tag of an injected left-hand element lies below truth.

Proof. Any truth neighbourhood appearing in that tag is either the master or the true token; a false token would force an impossible inclusion of a left injection into a right injection. ■

Theorem (`which_inMap1_le_falseElt`). The tag of an injected right-hand element lies below falsity.

Proof. Any truth neighbourhood appearing in that tag is either the master or the false token; a true token would force an impossible inclusion of a right injection into a left injection. ■

Theorem (`sumToProdT_comp_condSum_le`). Encoding after conditionally decoding a tag and two candidate components gives an approximation below the original triple.

Proof. Split on the three truth values. At the indeterminate value all outputs are least; at truth the left component is recovered and the right is least; at falsity the right component is recovered and the left is least. ■

Theorem (`sum_trianglelefteq_prodT`). The separated sum $E+E$ is isomorphic to a subsystem of $T \times (E \times E)$.

Proof. The sum encoding and conditional decoding form a projection pair by the decoding identity and encoding approximation. ■

Theorem (`sum_trianglelefteq_of`). If the truth domain and the self-function-space of E are each isomorphic to subsystems of E, then the separated sum $E + E$ is isomorphic to a subsystem of E.

Proof. First embed the sum into $T \times (E \times E)$, replace T and $E \times E$ by their embeddings into E, and compose the resulting product embeddings. ■

`EmbeddingsIntoU_more.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbeddingsIntoU
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.cond
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

section
open NeighborhoodSystem ApproximableMap Cond
universe u

section Prod

variable {α : Type u} {E : NeighborhoodSystem α}

noncomputable def funToPair : ApproximableMap (funSpace TD E) (prod E E) :=
  paired
    ((evalMap TD E).comp
```

```
      (paired (idMap (funSpace TD E)) (constMap (funSpace TD E) Truth.trueElt)))
    ((evalMap TD E).comp
      (paired (idMap (funSpace TD E)) (constMap (funSpace TD E) Truth.falseElt)))

end Prod

section Sum

variable {α : Type u} {E : NeighborhoodSystem α} (hE : ∀ X, E.mem X → X.Nonempty)

noncomputable def sumToProdT : ApproximableMap (sum E E hE hE) (prod TD (prod E E)) :=
  paired (whichMap E E hE hE)
    (paired (outMap₀ (h₀ := hE) (h₁ := hE)) (outMap₁ (h₀ := hE) (h₁ := hE)))

end Sum

section Prod

variable {α : Type u} {E : NeighborhoodSystem α}

noncomputable def swapProdT : ApproximableMap (prod (prod E E) TD) (prod TD (prod E E)
    ) :=
  paired (proj₁ (prod E E) TD) (proj₀ (prod E E) TD)

noncomputable def pairToFun : ApproximableMap (prod E E) (funSpace TD E) :=
  curry ((cond E).comp swapProdT)

end Prod

end

section
open NeighborhoodSystem ApproximableMap Cond

section General

variable {α β ? ? : Type*}

theorem TD_bot_eq_botElt : TD.bot = Truth.botElt :=
  le_antisymm (NeighborhoodSystem.bot_le _ _) (Truth.botElt_le _)

end General

theorem TD_trichotomy (t : TD.Element) :
    t = Truth.botElt ∨ t = Truth.trueElt ∨ t = Truth.falseElt :=
  TwoToken.neighborhoodSystem.element_classification t

section General

variable {α β ? ? : Type*}

theorem funSpace_dom_trianglelefteq {A : NeighborhoodSystem α} {B : NeighborhoodSystem
    β}
    (C : NeighborhoodSystem ?) (h : A ? B) : funSpace A C ? funSpace B C := by
  obtain ⟨D', hD'B, ⟨eAD'⟩⟩ := h
  set p : ProjPair A B := (ProjPair.ofIso eAD').comp (ProjPair.ofSubsystem hD'B) with
```

```
    hp
  refine trianglelefteq_of_projectionPair (expMap p.proj (idMap C)) (expMap p.inj (
    idMap C))
    ?_ ?_
  · have hcomp := (expMap.comp p.proj (idMap C) p.inj (idMap C)).symm
    rw [hcomp, p.proj_comp_inj, idMap_comp]
    exact expMap.id
  · calc (expMap p.proj (idMap C)).comp (expMap p.inj (idMap C))
        = expMap (p.inj.comp p.proj) ((idMap C).comp (idMap C)) :=
          (expMap.comp p.inj (idMap C) p.proj (idMap C)).symm
      _ ≤ expMap (idMap B) ((idMap C).comp (idMap C)) :=
          expMap.mono_left p.inj_comp_proj_le _
      _ = expMap (idMap B) (idMap C) := by rw [idMap_comp]
      _ = idMap (funSpace B C) := expMap.id

end General

section SumHelpers

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}

theorem outMap₀_comp_inMap₁ :
    (outMap₀ (V₀ := V₀) (V₁ := V₁) (h₀ := h₀) (h₁ := h₁)).comp
        (inMap₁ (h₀ := h₀) (h₁ := h₁)) = constMap V₁ V₀.bot := by
  apply ApproximableMap.ext
  intro Y Z
  rw [constMap_rel, mem_bot]
  constructor
  · rintro ⟨W, ⟨hY, hWmem, hinj⟩, _, hZ, hsub⟩
    refine ⟨hY, ?_⟩
    obtain ⟨b, hb⟩ := h₁ Y hY
    have hmaster : leftPart V₀ (inj₁ Y) ⊆ leftPart V₀ W := leftPart_mono V₀ hinj
    rw [leftPart_inj₁ V₀ ⟨b, hb⟩] at hmaster
    exact Set.Subset.antisymm (V₀.sub_master hZ) (hmaster.trans hsub)
  · rintro ⟨hY, rfl⟩
    exact ⟨inj₁ Y, ⟨hY, Or.inr (Or.inr ⟨Y, hY, rfl⟩), subset_rfl⟩,
      Or.inr (Or.inr ⟨Y, hY, rfl⟩), V₀.master_mem, (leftPart_inj₁ V₀ (h₁ Y hY)).subset⟩

theorem outMap₁_comp_inMap₀ :
    (outMap₁ (V₀ := V₀) (V₁ := V₁) (h₀ := h₀) (h₁ := h₁)).comp
        (inMap₀ (h₀ := h₀) (h₁ := h₁)) = constMap V₀ V₁.bot := by
  apply ApproximableMap.ext
  intro X Z
  rw [constMap_rel, mem_bot]
  constructor
  · rintro ⟨W, ⟨hX, hWmem, hinj⟩, _, hZ, hsub⟩
    refine ⟨hX, ?_⟩
    obtain ⟨a, ha⟩ := h₀ X hX
    have hmaster : rightPart V₁ (inj₀ X) ⊆ rightPart V₁ W := rightPart_mono V₁ hinj
    rw [rightPart_inj₀ V₁ ⟨a, ha⟩] at hmaster
    exact Set.Subset.antisymm (V₁.sub_master hZ) (hmaster.trans hsub)
  · rintro ⟨hX, rfl⟩
    exact ⟨inj₀ X, ⟨hX, Or.inr (Or.inl ⟨X, hX, rfl⟩), subset_rfl⟩,
      Or.inr (Or.inl ⟨X, hX, rfl⟩), V₁.master_mem, (rightPart_inj₀ V₁ (h₀ X hX)).subset⟩
```

```
end SumHelpers

section General

variable {α β ? ? : Type*}

theorem pair_bot {V0 : NeighborhoodSystem α} {V1 : NeighborhoodSystem β} :
    pair V0.bot V1.bot = (prod V0 V1).bot := by
  apply le_antisymm
  · intro W hW
    obtain ⟨X, Y, hX, hY, rfl⟩ := hW
    rw [mem_bot] at hX hY
    subst hX; subst hY
    rw [mem_bot]
    exact prod_master.symm
  · exact NeighborhoodSystem.bot_le _ _

theorem prod_left_trianglelefteq {A : NeighborhoodSystem α} {B : NeighborhoodSystem β}
    (C : NeighborhoodSystem ?) (h : A ? B) : prod A C ? prod B C := by
  obtain ⟨D', hD'B, ⟨eAD'⟩⟩ := h
  set p : ProjPair A B := (ProjPair.ofIso eAD').comp (ProjPair.ofSubsystem hD'B) with
    hp
  refine trianglelefteq_of_projectionPair (paired (p.inj.comp (proj₀ A C)) (proj₁ A C))
    (paired (p.proj.comp (proj₀ B C)) (proj₁ B C)) ?_ ?_
  · apply ext_of_toElementMap
    intro z
    rw [← pair_fst_snd z]
    set x := z.fst; set y := z.snd
    rw [toElementMap_comp, toElementMap_idMap, toElementMap_paired,
    toElementMap_paired,
      toElementMap_comp, toElementMap_proj₀, toElementMap_proj₁, fst_pair, snd_pair,
      toElementMap_comp, toElementMap_proj₀, fst_pair, toElementMap_proj₁, snd_pair]
    have hxx : (p.proj.comp p.inj).toElementMap x = (idMap A).toElementMap x := by
      rw [p.proj_comp_inj]
    rw [toElementMap_comp, toElementMap_idMap] at hxx
    rw [hxx]
  · rw [le_iff_toElementMap_le]
    intro z
    rw [← pair_fst_snd z]
    set x := z.fst; set y := z.snd
    rw [toElementMap_comp, toElementMap_idMap, toElementMap_paired,
    toElementMap_paired,
      toElementMap_comp, toElementMap_proj₀, toElementMap_proj₁, fst_pair, snd_pair,
      toElementMap_comp, toElementMap_proj₀, fst_pair, toElementMap_proj₁, snd_pair]
    apply pair_le_pair_iff.mpr
    refine ⟨?_, le_refl y⟩
    have hxx : (p.inj.comp p.proj).toElementMap x ≤ (idMap B).toElementMap x :=
      le_iff_toElementMap_le.mp p.inj_comp_proj_le x
    rwa [toElementMap_comp, toElementMap_idMap] at hxx

theorem prod_right_trianglelefteq {A : NeighborhoodSystem α} {B : NeighborhoodSystem β
    }
    (C : NeighborhoodSystem ?) (h : A ? B) : prod C A ? prod C B := by
  have h1 : prod C A ? prod A C :=
    (prod_comm_isomorphic (V₀ := C) (V₁ := A)).elim fun e => ⟨_, Subsystem.refl _, ⟨e⟩⟩
  have h2 : prod A C ? prod B C := prod_left_trianglelefteq C h
```

```
  have h3 : prod B C ? prod C B :=
    (prod_comm_isomorphic (V₀ := B) (V₁ := C)).elim fun e => ⟨_, Subsystem.refl _, ⟨e⟩⟩
  exact trianglelefteq_trans (trianglelefteq_trans h1 h2) h3

theorem toElementMap_bot_le {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
    (f : ApproximableMap V₀ V₁) (h : ∀ {Y}, f.rel V₀.master Y → Y = V₁.master) :
    f.toElementMap V₀.bot ≤ V₁.bot := by
  intro Y hY
  rw [mem_bot]
  exact h ((rel_iff_mem_principal f V₀.master_mem).mpr hY)

end General

section SumHelpers

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}

theorem outMap₀_bot_le :
    (outMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap (sum V₀ V₁ h₀ h₁).bot ≤ V₀.bot := by
  refine toElementMap_bot_le _ ?_
  intro Y hg
  refine Set.Subset.antisymm (V₀.sub_master hg.2.1) ?_
  have h2 : leftPart V₀ (sumMaster V₀ V₁) ⊆ Y := hg.2.2
  rwa [leftPart_sumMaster] at h2

theorem outMap₁_bot_le :
    (outMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap (sum V₀ V₁ h₀ h₁).bot ≤ V₁.bot := by
  refine toElementMap_bot_le _ ?_
  intro Y hg
  refine Set.Subset.antisymm (V₁.sub_master hg.2.1) ?_
  have h2 : rightPart V₁ (sumMaster V₀ V₁) ⊆ Y := hg.2.2
  rwa [rightPart_sumMaster] at h2

end SumHelpers

end

section
open NeighborhoodSystem ApproximableMap Cond
universe u

section Prod

variable {α : Type u} {E : NeighborhoodSystem α}

theorem toElementMap_funToPair (φ : (funSpace TD E).Element) :
    (funToPair (E := E)).toElementMap φ
      = pair ((toApproxMap φ).toElementMap Truth.trueElt)
          ((toApproxMap φ).toElementMap Truth.falseElt) := by
  simp only [funToPair, toElementMap_paired, toElementMap_comp, toElementMap_idMap,
    toElementMap_constMap, evalMap_apply]

end Prod

section Sum
```

```
variable {α : Type u} {E : NeighborhoodSystem α} (hE : ∀ X, E.mem X → X.Nonempty)

theorem toElementMap_sumToProdT (w : (sum E E hE hE).Element) :
    (sumToProdT hE).toElementMap w
      = pair ((whichMap E E hE hE).toElementMap w)
          (pair ((outMap₀ (h₀ := hE) (h₁ := hE)).toElementMap w)
            ((outMap₁ (h₀ := hE) (h₁ := hE)).toElementMap w)) := by
  simp only [sumToProdT, toElementMap_paired]

theorem condSum_comp_sumToProdT :
    (condSum E E hE hE).comp (sumToProdT hE) = idMap (sum E E hE hE) := by
  apply ext_of_toElementMap
  intro w
  rw [toElementMap_comp, toElementMap_idMap, toElementMap_sumToProdT, condSum,
    toElementMap_comp,
    condSumEmb_toElementMap]
  exact cond_which w

end Sum

section Prod

variable {α : Type u} {E : NeighborhoodSystem α}

theorem toElementMap_swapProdT (x y : E.Element) (t : TD.Element) :
    (swapProdT (E := E)).toElementMap (pair (pair x y) t) = pair t (pair x y) := by
  simp only [swapProdT, toElementMap_paired, toElementMap_proj₀, toElementMap_proj₁,
    fst_pair,
    snd_pair]

theorem toApproxMap_pairToFun (x y : E.Element) (t : TD.Element) :
    (toApproxMap ((pairToFun (E := E)).toElementMap (pair x y))).toElementMap t
      = (cond E).toElementMap (pair t (pair x y)) := by
  rw [pairToFun, toElementMap_curry_apply, toElementMap_comp, toElementMap_swapProdT]

theorem funToPair_comp_pairToFun : (funToPair (E := E)).comp pairToFun = idMap (prod E
    E) := by
  apply ext_of_toElementMap
  intro z
  rw [← pair_fst_snd z]
  set x := z.fst; set y := z.snd
  rw [toElementMap_comp, toElementMap_idMap, toElementMap_funToPair,
    toApproxMap_pairToFun,
    toApproxMap_pairToFun, Cond.cond_true, Cond.cond_false]

theorem pairToFun_comp_funToPair_le :
    (pairToFun (E := E)).comp funToPair ≤ idMap (funSpace TD E) := by
  rw [le_iff_toElementMap_le]
  intro φ
  rw [toElementMap_comp, toElementMap_idMap, toElementMap_funToPair]
  apply (funSpaceEquiv TD E).map_rel_iff.mp
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, le_iff_toElementMap_le]
  intro t
  rcases TD_trichotomy t with rfl | rfl | rfl
  · rw [toApproxMap_pairToFun, Cond.cond_bot]; exact E.bot_le _
```

```
  · rw [toApproxMap_pairToFun, Cond.cond_true]
  · rw [toApproxMap_pairToFun, Cond.cond_false]

theorem prod_trianglelefteq_funSpace_TD : prod E E ? funSpace TD E :=
  trianglelefteq_of_projectionPair pairToFun funToPair funToPair_comp_pairToFun
    pairToFun_comp_funToPair_le

theorem prod_trianglelefteq_of (hT : TD ? E) (hArrow : funSpace E E ? E) :
    prod E E ? E :=
  trianglelefteq_trans
    (trianglelefteq_trans prod_trianglelefteq_funSpace_TD (funSpace_dom_trianglelefteq
     E hT))
    hArrow

end Prod

end

section
open NeighborhoodSystem ApproximableMap Cond

section SumHelpers

variable {α β : Type*} {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {h₀ : ∀ X, V₀.mem X → X.Nonempty} {h₁ : ∀ Y, V₁.mem Y → Y.Nonempty}

theorem which_bot_le :
    (whichMap V₀ V₁ h₀ h₁).toElementMap (sum V₀ V₁ h₀ h₁).bot ≤ Truth.botElt := by
  rw [← TD_bot_eq_botElt]
  refine toElementMap_bot_le _ ?_
  intro Y hg
  exact whichGuard_masterC hg.2.2 rfl

theorem which_inMap₀_le_trueElt (x : V₀.Element) :
    (whichMap V₀ V₁ h₀ h₁).toElementMap ((inMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap x)
      ≤ Truth.trueElt := by
  intro Z hZ
  rcases (TwoToken.mem_iff Z).mp
      (((whichMap V₀ V₁ h₀ h₁).toElementMap
        ((inMap₀ (h₀ := h₀) (h₁ := h₁)).toElementMap x)).sub hZ) with rfl | rfl | rfl
  · exact Truth.trueElt.master_mem
  · show Truth.trueElt.mem TwoToken.zero
    exact Or.inr rfl
  · exfalso
    obtain ⟨Y, hY, hmem⟩ := which_mem_one.mp hZ
    obtain ⟨X, _, hX0, _, hsub⟩ := hmem
    exact not_inj₀_subset_inj₁ (h₀ X hX0) hsub

theorem which_inMap₁_le_falseElt (y : V₁.Element) :
    (whichMap V₀ V₁ h₀ h₁).toElementMap ((inMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap y)
      ≤ Truth.falseElt := by
  intro Z hZ
  rcases (TwoToken.mem_iff Z).mp
      (((whichMap V₀ V₁ h₀ h₁).toElementMap
        ((inMap₁ (h₀ := h₀) (h₁ := h₁)).toElementMap y)).sub hZ) with rfl | rfl | rfl
  · exact Truth.falseElt.master_mem
```

```
  · exfalso
    obtain ⟨X, hX, hmem⟩ := which_mem_zero.mp hZ
    obtain ⟨Y, _, hY0, _, hsub⟩ := hmem
    exact not_inj₁_subset_inj₀ (h₁ Y hY0) hsub
  · show Truth.falseElt.mem TwoToken.one
    exact Or.inr rfl

end SumHelpers

end

section
open NeighborhoodSystem ApproximableMap Cond
universe u

section Sum

variable {α : Type u} {E : NeighborhoodSystem α} (hE : ∀ X, E.mem X → X.Nonempty)

theorem sumToProdT_comp_condSum_le :
    (sumToProdT hE).comp (condSum E E hE hE) ≤ idMap (prod TD (prod E E)) := by
  rw [le_iff_toElementMap_le]
  intro z
  rw [← pair_fst_snd z, ← pair_fst_snd z.snd]
  set t := z.fst; set x := z.snd.fst; set y := z.snd.snd
  clear_value t x y
  rw [toElementMap_comp, toElementMap_idMap]
  rcases TD_trichotomy t with rfl | rfl | rfl
  · rw [condSum_bot, toElementMap_sumToProdT]
    exact pair_le_pair_iff.mpr ⟨which_bot_le,
      pair_le_pair_iff.mpr ⟨outMap₀_bot_le.trans (E.bot_le x), outMap₁_bot_le.trans (E.
    bot_le y)⟩⟩
  · rw [condSum_true, toElementMap_sumToProdT]
    refine pair_le_pair_iff.mpr ⟨which_inMap₀_le_trueElt x, pair_le_pair_iff.mpr ⟨?_, ?
    _⟩⟩
    · have h2 := congrArg (fun (f : ApproximableMap E E) => f.toElementMap x)
        (outMap₀_comp_inMap₀ (h₀ := hE) (h₁ := hE))
      simp only [toElementMap_comp, toElementMap_idMap] at h2
      exact h2.le
    · have h2 := congrArg (fun (f : ApproximableMap E E) => f.toElementMap x)
        (outMap₁_comp_inMap₀ (h₀ := hE) (h₁ := hE))
      simp only [toElementMap_comp, toElementMap_constMap] at h2
      rw [h2]; exact E.bot_le _
  · rw [condSum_false, toElementMap_sumToProdT]
    refine pair_le_pair_iff.mpr ⟨which_inMap₁_le_falseElt y, pair_le_pair_iff.mpr ⟨?_,
    ?_⟩⟩
    · have h2 := congrArg (fun (f : ApproximableMap E E) => f.toElementMap y)
        (outMap₀_comp_inMap₁ (h₀ := hE) (h₁ := hE))
      simp only [toElementMap_comp, toElementMap_constMap] at h2
      rw [h2]; exact E.bot_le _
    · have h2 := congrArg (fun (f : ApproximableMap E E) => f.toElementMap y)
        (outMap₁_comp_inMap₁ (h₀ := hE) (h₁ := hE))
      simp only [toElementMap_comp, toElementMap_idMap] at h2
      exact h2.le

theorem sum_trianglelefteq_prodT : sum E E hE hE ? prod TD (prod E E) :=
```

```
  trianglelefteq_of_projectionPair (sumToProdT hE) (condSum E E hE hE) (
    condSum_comp_sumToProdT hE)


    (sumToProdT_comp_condSum_le hE)

theorem sum_trianglelefteq_of (hT : TD ? E) (hArrow : funSpace E E ? E) :
    sum E E hE hE ? E := by
  have hPE : prod E E ? E := prod_trianglelefteq_of hT hArrow
  exact trianglelefteq_trans
    (trianglelefteq_trans (sum_trianglelefteq_prodT hE) (prod_left_trianglelefteq (
    prod E E) hT))
    (trianglelefteq_trans (prod_right_trianglelefteq E hPE) hPE)

end Sum

end
end DomainTheory.NeighbourhoodSystems
```

4.9.51 Projection pairs

This peer isolates projection pairs, the embedding–projection data used to represent one neighbourhood domain as a retract of another.

Definition (`ProjPair`). A projection pair from D to E consists of an injection $i : D \to E$ and a projection $j : E \to D$ such that ji is the identity on D and ij lies below the identity on E.

Proof. These two equations are precisely the retraction and approximation laws for an embedding–projection pair. ■

Definition (`ProjPair.comp`). Projection pairs from D to E and from E to F compose to a projection pair from D to F.

Proof. Compose the injections in the forward order and the projections in the reverse order; associativity, the two retraction laws, and monotonicity give the required identities and inequality. ■

Definition (`ProjPair.ofIso`). Every domain isomorphism determines a projection pair whose injection is the isomorphism and whose projection is its inverse.

Proof. The inverse laws make both composites identities, and equality in particular gives the required approximation inequality. ■

Definition (`ProjPair.ofSubsystem`). Every subsystem inclusion determines a projection pair from the subsystem to its ambient neighbourhood system.

Proof. Use the canonical subsystem injection and projection; their retraction and approximation laws are part of the subsystem construction. ■

`ProjPair.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_6

@[expose] public section
```

```
section
universe u v w

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Subsystem

variable {α : Type u} {β : Type v} {? : Type w}

structure ProjPair (D : NeighborhoodSystem α) (E : NeighborhoodSystem β) where

  inj : ApproximableMap D E

  proj : ApproximableMap E D

  proj_comp_inj : proj.comp inj = idMap D

  inj_comp_proj_le : inj.comp proj ≤ idMap E

namespace ProjPair

def comp {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}
    {F : NeighborhoodSystem ?} (p1 : ProjPair D E) (p2 : ProjPair E F) :
    ProjPair D F where
  inj := p2.inj.comp p1.inj
  proj := p1.proj.comp p2.proj
  proj_comp_inj := by
    show (p1.proj.comp p2.proj).comp (p2.inj.comp p1.inj) = idMap D
    rw [comp_assoc, ← comp_assoc p2.proj p2.inj p1.inj, p2.proj_comp_inj, idMap_comp,
      p1.proj_comp_inj]
  inj_comp_proj_le := by
    show (p2.inj.comp p1.inj).comp (p1.proj.comp p2.proj) ≤ idMap F
    rw [comp_assoc, ← comp_assoc p1.inj p1.proj p2.proj]
    calc p2.inj.comp ((p1.inj.comp p1.proj).comp p2.proj)
        ≤ p2.inj.comp ((idMap E).comp p2.proj) :=
          comp_mono_gen le_rfl (comp_mono_gen p1.inj_comp_proj_le le_rfl)
      _ = p2.inj.comp p2.proj := by rw [idMap_comp]
      _ ≤ idMap F := p2.inj_comp_proj_le

def ofIso {D : NeighborhoodSystem α} {E : NeighborhoodSystem β} (e : DomainIso D E) :
    ProjPair D E where
  inj := ApproximableMap.ofIso e
  proj := ApproximableMap.ofIso e.symm
  proj_comp_inj := by
    have heq : ∀ x, ((ApproximableMap.ofIso e.symm).comp (ApproximableMap.ofIso e)).
    toElementMap x
        = (idMap D).toElementMap x := by
      intro x
      rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, OrderIso.
    symm_apply_apply,
        toElementMap_idMap]
    exact le_antisymm (le_iff_toElementMap_le.mpr fun x => (heq x).le)
      (le_iff_toElementMap_le.mpr fun x => (heq x).ge)
  inj_comp_proj_le := by
    have heq : ∀ y, ((ApproximableMap.ofIso e).comp (ApproximableMap.ofIso e.symm)).
    toElementMap y
```

```
        = (idMap E).toElementMap y := by
      intro y
      rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, OrderIso.
    apply_symm_apply,
        toElementMap_idMap]
    exact le_iff_toElementMap_le.mpr fun y => (heq y).le

def ofSubsystem {D E : NeighborhoodSystem α} (h : D ? E) : ProjPair D E where
  inj := h.inj
  proj := h.proj
  proj_comp_inj := h.proj_comp_inj
  inj_comp_proj_le := h.inj_comp_proj_le

end ProjPair

end DomainTheory.NeighbourhoodSystems
end
```

4.9.52 Exercise 8.17 — explicit embeddings into U

NeighbourhoodSystems/RetractsOfTheUniversalDomain/ProjPair.lean

@import “ProjPair.lean”

NeighbourhoodSystems/RetractsOfTheUniversalDomain/Exercise_8_17.lean

@import “Exercise_8_17.lean” **EXERCISE 8.17.** Find explicitly (if possible) the projection pairs for $\mathcal{U}+\mathcal{U}$, $\mathcal{U}\times\mathcal{U}$, and $\mathcal{U}\to\mathcal{U}$ needed for 8.9. Are any of these domains isomorphic with $\mathcal{U}$? (The author does not know a really good construction for $\mathcal{U}\to\mathcal{U}$.) Find a universal domain $V\neq\mathcal{U}$.

The development is with the Embeddings into U peer.

Theorem (`example_817`). V is universal; product, sum, and function-space copies of U retract into U.

Proof. By assembling the peer lemmas named in the packaging module. ■

Exercise_8_17.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbeddingsIntoU

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

theorem example_817 :
    IsUniversal.{0} V ∧
      (prod U U ? U) ∧
      (sum U U U_mem_nonempty U_mem_nonempty ? U) ∧
      (funSpace U U ? U) :=
  ⟨V.isUniversal, prodUU_trianglelefteq_U, sumUU_trianglelefteq_U,
    funSpaceUU_trianglelefteq_U⟩
```

4.9.53 Projection combinators

This peer assembles the closure properties of projections on the universal domain under the sum, product, and function-space combinators.

Theorem (`exercise_8_18_assembly`). If a and b are projections on U, then their sum, product, and arrow combinators are projections; if a and b are also finitary, all three resulting projections are finitary.

Proof. The projection-closure theorem supplies the three projection laws, and the finitary-closure theorem supplies the corresponding finiteness properties. ■

Theorem (`example_818`). For arbitrary projections a and b on U, the sum, product, and arrow constructions preserve projectionhood, and preserve finitariness whenever both inputs are finitary.

Proof. This is the universally quantified form of the preceding assembly theorem. ■

`ProjectionCombinators.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_10

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

theorem exercise_8_18_assembly {a b : ApproximableMap U U} (ha : IsProjection a) (hb :
     IsProjection b) :
    (IsProjection (sumComb a b) ∧ IsProjection (prodComb a b) ∧ IsProjection (
    arrowComb a b)) ∧
      (FinitaryProjection a → FinitaryProjection b →
        FinitaryProjection (sumComb a b) ∧ FinitaryProjection (prodComb a b) ∧
          FinitaryProjection (arrowComb a b)) :=
  ⟨isProjection_combinators ha hb, fun hfa hfb => finitaryProjection_combinators hfa
    hfb⟩

theorem example_818 :
    ∀ {a b : ApproximableMap U U},
      IsProjection a → IsProjection b →
        (IsProjection (sumComb a b) ∧ IsProjection (prodComb a b) ∧ IsProjection (
    arrowComb a b)) ∧
          (FinitaryProjection a → FinitaryProjection b →
            FinitaryProjection (sumComb a b) ∧ FinitaryProjection (prodComb a b) ∧
              FinitaryProjection (arrowComb a b)) :=
  @exercise_8_18_assembly

end DomainTheory.NeighbourhoodSystems
```

4.9.54 Exercise 8.18 — missing proofs of 8.10

NeighbourhoodSystems/RetractsOfTheUniversalDomain/Exercise_8_18.lean

@import “Exercise_8_18.lean”

EXERCISE 8.18. Many of the cases of 8.10 were left unproved. Please establish these assertions explicitly.

The development is with the Projection combinators peer.

Theorem (`example_818`). Sum, product, and arrow combinators preserve projections and finitary projections.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_18.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ProjectionCombinators

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

theorem example_818 {a b : ApproximableMap U U} (ha : IsProjection a) (hb :
    IsProjection b) :
     (IsProjection (sumComb a b) ∧ IsProjection (prodComb a b) ∧ IsProjection (
    arrowComb a b)) ∧
       (FinitaryProjection a → FinitaryProjection b →
         FinitaryProjection (sumComb a b) ∧ FinitaryProjection (prodComb a b) ∧
           FinitaryProjection (arrowComb a b)) :=
  exercise_8_18_assembly ha hb
```

4.9.55 Sum and product retracts

This peer develops elementary embeddings and projections for sums, products, and function spaces, and assembles the hypotheses under which binary sums and products retract into a universal ambient domain.

Theorem (`exercise_8_19_assembly`). Let E be a neighbourhood system whose neighbourhoods are nonempty. If the truth-value domain and the self-function space $[E \to E]$ are retracts of E, then both $E + E$ and $E \times E$ are retracts of E.

Proof. Apply the sum construction to the truth-value and function-space retractions, and the product construction to the same two hypotheses. ■

Theorem (`example_819`). The preceding simultaneous sum-and-product assertion holds uniformly; moreover, its product and sum conclusions each hold separately under their respective stated hypotheses.

Proof. Combine the general product and sum retraction theorems, then take each component of the conjunction. ■

Definition (`constFunMap`). For neighbourhood systems A and B, the constant-function map sends an element b of B to the element of $[A \to B]$ having constant value b.

Proof. Curry the projection $B \times A \to B$. ■

Definition (`embedFst`). The first-factor embedding $A \to A \times B$ sends a to $(a, \bot)$.

Proof. Pair the identity on A with the constant bottom map into B. ■

Definition (`embedSnd`). The second-factor embedding $B \to A \times B$ sends b to $(\bot, b)$.

Proof. Pair the constant bottom map into A with the identity on B. ■

Definition (`evalBotMap`). Evaluation at bottom sends a function $f : A \to B$ to $f(\bot)$.

Proof. Pair a function with the constant bottom argument and apply evaluation. ■

Theorem (`toApproxMap_constFunMap`). Applying the constant function determined by b to any argument a yields b.

Proof. Uncurry the defining projection; its first coordinate is b, independently of a. ■

Theorem (`embedFst_comp_proj0_le`). Replacing the second coordinate of a pair by bottom lies below the original pair.

Proof. The first coordinates agree, and bottom is below the original second coordinate. ■

Theorem (`embedFst_ne_embedSnd_of_ne_bot`). If x is not bottom in D, then the two embeddings $D \to D \times D$, $y \mapsto (y, \bot)$ and $y \mapsto (\bot, y)$, are distinct.

Proof. Equality of the maps would give equality of their values at x; taking first coordinates would force $x = \bot$. ■

Theorem (`embedSnd_comp_proj1_le`). Replacing the first coordinate of a pair by bottom lies below the original pair.

Proof. Bottom is below the original first coordinate, and the second coordinates agree. ■

Theorem (`proj0_comp_embedFst`). First projection after first-factor embedding is the identity on A.

Proof. The first coordinate of $(a, \bot)$ is a. ■

Theorem (`proj1_comp_embedSnd`). Second projection after second-factor embedding is the identity on B.

Proof. The second coordinate of $(\bot, b)$ is b. ■

Theorem (`toElementMap_evalBotMap`). On elements of the function-space domain, evaluation at bottom is ordinary application to the bottom element of the argument domain.

Proof. Expand evaluation on the pair consisting of the function and the constant bottom argument. ■

Theorem (`constFunMap_comp_evalBotMap_le`). Sending a function f to the constant function with value $f(\bot)$ lies below f.

Proof. Monotonicity of f gives $f(\bot) \leq f(a)$ for every argument a, which is exactly the pointwise order on functions. ■

Theorem (`evalBotMap_comp_constFunMap`). Evaluating a constant function at bottom recovers its constant value.

Proof. Constant functions have the same value at every argument, hence in particular at bottom. ■

Theorem (`trianglelefteq_funSpace_const`). The codomain B is a retract of $[A \to B]$.

Proof. Use constant functions as the embedding and evaluation at bottom as the projection. Their projection–embedding composite is the identity, and the reverse composite lies below the identity. ■

Theorem (`D_trianglelefteq_funSpace`). Every neighbourhood system D is a retract of its self-function space $[D \to D]$.

Proof. Specialize the constant-function retraction to $A = B = D$. ■

Theorem (`trianglelefteq_prod_fst`). The first factor A is a retract of $A \times B$.

Proof. Embed a as $(a, \bot)$ and project to the first coordinate. Projection after embedding is the identity, while the reverse composite lies below the product identity. ■

Theorem (`D_trianglelefteq_prod`). Every neighbourhood system D is a retract of $D \times D$.

Proof. Specialize the first-factor product retraction to $A = B = D$. ■

`SumProdRetracts.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.projPairSubdomain
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
```

```
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbeddingsIntoU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbeddingsIntoU_more
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap Cond
universe u

theorem exercise_8_19_assembly {α : Type u} {E : NeighborhoodSystem α}
    (hE : ∀ X, E.mem X → X.Nonempty) (hT : TD ? E) (hArrow : funSpace E E ? E) :
    sum E E hE hE ? E ∧ prod E E ? E :=
  ⟨sum_trianglelefteq_of hE hT hArrow, prod_trianglelefteq_of hT hArrow⟩
end

open NeighborhoodSystem ApproximableMap Cond

theorem example_819 :
    (∀ {α : Type*} {E : NeighborhoodSystem α}
      (hE : ∀ X, E.mem X → X.Nonempty),
      TD ? E → funSpace E E ? E → sum E E hE hE ? E ∧ prod E E ? E) ∧
    (∀ {α : Type*} {E : NeighborhoodSystem α},
      TD ? E → funSpace E E ? E → prod E E ? E) ∧
    (∀ {α : Type*} {E : NeighborhoodSystem α}
      (hE : ∀ X, E.mem X → X.Nonempty),
      TD ? E → funSpace E E ? E → sum E E hE hE ? E) :=
  ⟨@exercise_8_19_assembly, @prod_trianglelefteq_of, @sum_trianglelefteq_of⟩

section
open NeighborhoodSystem ApproximableMap
universe u v

section FunSpace

variable {α : Type u} {β : Type v} (A : NeighborhoodSystem α) (B : NeighborhoodSystem
    β)

noncomputable def constFunMap : ApproximableMap B (funSpace A B) := curry (proj₀ B A)

end FunSpace

section Prod

variable {α : Type u} {β : Type v} (A : NeighborhoodSystem α) (B : NeighborhoodSystem
    β)

noncomputable def embedFst : ApproximableMap A (prod A B) := paired (idMap A) (
    constMap A B.bot)
```

```
noncomputable def embedSnd : ApproximableMap B (prod A B) := paired (constMap B A.bot)
     (idMap B)

end Prod

section FunSpace

variable {α : Type u} {β : Type v} (A : NeighborhoodSystem α) (B : NeighborhoodSystem
    β)

noncomputable def evalBotMap : ApproximableMap (funSpace A B) B :=
  (evalMap A B).comp (paired (idMap (funSpace A B)) (constMap (funSpace A B) A.bot))

@[simp] theorem toApproxMap_constFunMap (b : B.Element) (a : A.Element) :
    (toApproxMap ((constFunMap A B).toElementMap b)).toElementMap a = b := by
  rw [constFunMap, toElementMap_curry_apply, toElementMap_proj₀, fst_pair]

end FunSpace

variable {α : Type u} (D : NeighborhoodSystem α)

end

section
open NeighborhoodSystem ApproximableMap
universe u v

section Prod

variable {α : Type u} {β : Type v} (A : NeighborhoodSystem α) (B : NeighborhoodSystem
    β)

theorem embedFst_comp_proj₀_le : (embedFst A B).comp (proj₀ A B) ≤ idMap (prod A B) :=
    by
  rw [le_iff_toElementMap_le]
  intro z
  rw [← pair_fst_snd z]
  set x := z.fst; set y := z.snd
  rw [toElementMap_comp, toElementMap_idMap, toElementMap_proj₀, fst_pair, embedFst,
    toElementMap_paired, toElementMap_idMap, toElementMap_constMap]
  exact pair_le_pair_iff.mpr ⟨le_refl x, B.bot_le y⟩

end Prod
end

section
open NeighborhoodSystem ApproximableMap
universe u

theorem embedFst_ne_embedSnd_of_ne_bot {α : Type u} {D : NeighborhoodSystem α} {x : D.
    Element}
    (hx : x = D.bot) : embedFst D D = embedSnd D D := fun h => hx <| by
  have heq := congrArg (fun f => f.toElementMap x) h
  simp only [embedFst, embedSnd, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap] at heq
```

```
  simpa using congrArg (·.fst) heq
end

section
open NeighborhoodSystem ApproximableMap
universe u v

section Prod

variable {α : Type u} {β : Type v} (A : NeighborhoodSystem α) (B : NeighborhoodSystem
    β)

theorem embedSnd_comp_proj₁_le : (embedSnd A B).comp (proj₁ A B) ≤ idMap (prod A B) :=
    by
  rw [le_iff_toElementMap_le]
  intro z
  rw [← pair_fst_snd z]
  set x := z.fst; set y := z.snd
  rw [toElementMap_comp, toElementMap_idMap, toElementMap_proj₁, snd_pair, embedSnd,
    toElementMap_paired, toElementMap_idMap, toElementMap_constMap]
  exact pair_le_pair_iff.mpr ⟨A.bot_le x, le_refl y⟩

theorem proj₀_comp_embedFst : (proj₀ A B).comp (embedFst A B) = idMap A := by
  apply le_antisymm <;> rw [le_iff_toElementMap_le] <;> intro a <;>
    simp [embedFst, toElementMap_comp, toElementMap_idMap, toElementMap_paired,
    toElementMap_constMap,
      toElementMap_proj₀, fst_pair]

theorem proj₁_comp_embedSnd : (proj₁ A B).comp (embedSnd A B) = idMap B := by
  apply le_antisymm <;> rw [le_iff_toElementMap_le] <;> intro b <;>
    simp [embedSnd, toElementMap_comp, toElementMap_idMap, toElementMap_paired,
    toElementMap_constMap,
      toElementMap_proj₁, snd_pair]

end Prod

section FunSpace

variable {α : Type u} {β : Type v} (A : NeighborhoodSystem α) (B : NeighborhoodSystem
    β)

theorem toElementMap_evalBotMap (φ : (funSpace A B).Element) :
    (evalBotMap A B).toElementMap φ = (toApproxMap φ).toElementMap A.bot := by
  simp [evalBotMap, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap, evalMap_apply]

theorem constFunMap_comp_evalBotMap_le :
    (constFunMap A B).comp (evalBotMap A B) ≤ idMap (funSpace A B) := by
  rw [le_iff_toElementMap_le]
  intro φ
  rw [toElementMap_comp, toElementMap_idMap]
  apply (funSpaceEquiv A B).map_rel_iff.mp
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, le_iff_toElementMap_le]
  intro a
  rw [toApproxMap_constFunMap, toElementMap_evalBotMap]
  exact toElementMap_mono (toApproxMap φ) (A.bot_le a)
```

```
theorem evalBotMap_comp_constFunMap : (evalBotMap A B).comp (constFunMap A B) = idMap
    B := by
  apply le_antisymm <;> rw [le_iff_toElementMap_le] <;> intro b <;>
    rw [toElementMap_comp, toElementMap_idMap, toElementMap_evalBotMap,
    toApproxMap_constFunMap]

theorem trianglelefteq_funSpace_const : B ? funSpace A B :=
  trianglelefteq_of_projectionPair (constFunMap A B) (evalBotMap A B)
    (evalBotMap_comp_constFunMap A B) (constFunMap_comp_evalBotMap_le A B)

end FunSpace

end

section
open NeighborhoodSystem ApproximableMap
universe u
variable {α : Type u} (D : NeighborhoodSystem α)

theorem D_trianglelefteq_funSpace : D ? funSpace D D := trianglelefteq_funSpace_const
    D D
end

section
open NeighborhoodSystem ApproximableMap
universe u v

section Prod

variable {α : Type u} {β : Type v} (A : NeighborhoodSystem α) (B : NeighborhoodSystem
    β)

theorem trianglelefteq_prod_fst : A ? prod A B :=
  trianglelefteq_of_projectionPair (embedFst A B) (proj₀ A B) (proj₀_comp_embedFst A B)
    (embedFst_comp_proj₀_le A B)

end Prod
end

section
open NeighborhoodSystem ApproximableMap
universe u
variable {α : Type u} (D : NeighborhoodSystem α)

theorem D_trianglelefteq_prod : D ? prod D D := trianglelefteq_prod_fst D D

end
end DomainTheory.NeighbourhoodSystems
```

4.9.56 Exercise 8.19 — closure under sum and product

NeighbourhoodSystems/RetractsOfTheUniversalDomain/Exercise_8_19.lean

@import “Exercise_8_19.lean”

EXERCISE 8.19. Suppose we know both

$$T \quad \text{and} \quad E \to E \trianglelefteq E\,.$$

Does it follow that $E + E$ and $E \times E \trianglelefteq E$?

The development is with the Sum and product retracts peer.

Theorem (`example_819`). Domains closed under arrow also admit sum and product retracts.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_19.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    SumProdRetracts

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap DomainTheory
    .NeighbourhoodSystems.Cond

universe u

theorem example_819 {α : Type u} {E : NeighborhoodSystem α}
    (hE : ∀ X, E.mem X → X.Nonempty) (hT : TD ? E) (hArrow : funSpace E E ? E) :
    sum E E hE hE ? E ∧ prod E E ? E :=
  exercise_8_19_assembly hE hT hArrow
```

4.9.57 Diagonal retracts

This peer records the two canonical diagonal retractions: every neighbourhood system is a retract of both its self-product and its self-function space.

Theorem (`exercise_8_20_assembly`). For every neighbourhood system D, the domain D is a retract of $D \times D$, and it is also a retract of the function-space domain $[D \to D]$.

Proof. Use the first-coordinate embedding for the product and the constant-function embedding for the function space. ■

Theorem (`example_820`). The conjunction of the preceding two retraction assertions holds uniformly for every neighbourhood system; in particular, each assertion also holds separately.

Proof. Combine the uniform product and function-space retractions, and obtain the two remaining conjuncts by projection. ■

Theorem (`trianglelefteq_prod_snd`). For neighbourhood systems A and B, the second factor B is a retract of $A \times B$.

Proof. Embed b as $(\bot, b)$ and project to the second coordinate. Projection after embedding is the identity, while embedding after projection lies below the identity on the product. ■

`DiagonalRetracts.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_15
```

```
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Combinators

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbeddingsIntoU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_10
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    SumProdRetracts
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
universe u
variable {α : Type u} (D : NeighborhoodSystem α)

theorem exercise_8_20_assembly : D ? prod D D ∧ D ? funSpace D D :=
  ⟨D_trianglelefteq_prod D, D_trianglelefteq_funSpace D⟩
end

open NeighborhoodSystem ApproximableMap

theorem example_820 :
    (∀ {α : Type*} (D : NeighborhoodSystem α), D ? prod D D ∧ D ? funSpace D D) ∧
    (∀ {α : Type*} (D : NeighborhoodSystem α), D ? funSpace D D) ∧
    (∀ {α : Type*} (D : NeighborhoodSystem α), D ? prod D D) :=
  ⟨@exercise_8_20_assembly, @D_trianglelefteq_funSpace, @D_trianglelefteq_prod⟩

section
open NeighborhoodSystem ApproximableMap
universe u v

section Prod

variable {α : Type u} {β : Type v} (A : NeighborhoodSystem α) (B : NeighborhoodSystem
    β)

theorem trianglelefteq_prod_snd : B ? prod A B :=
  trianglelefteq_of_projectionPair (embedSnd A B) (proj₁ A B) (proj₁_comp_embedSnd A B)
    (embedSnd_comp_proj₁_le A B)

end Prod
end

end DomainTheory.NeighbourhoodSystems
```

4.9.58 Exercise 8.20 — self-embeddings of a domain

NeighbourhoodSystems/RetractsOfTheUniversalDomain/Exercise_8_20.lean

@import “Exercise_8_20.lean”

EXERCISE 8.20. For any system we know $D \trianglelefteq D + D$, but what about

$$D \trianglelefteq D \times D \quad \text{and} \quad D \trianglelefteq D \to D\,?$$

Would these projections be computable if D is effectively given? Are there more than one projection pair in each case?

The development is with the Diagonal retracts peer.

Theorem (`example_820`). A domain retracts into its diagonal product and function-space copies.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_20.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    DiagonalRetracts

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

theorem example_820 {α : Type*} (D : NeighborhoodSystem α) :
    D ? prod D D ∧ D ? funSpace D D :=
  exercise_8_20_assembly D
```

4.9.59 A-sharp on lists

This peer constructs the sharp operation on endomaps of the universal domain. It develops functoriality of the sharp domain, transports it through a universal embedding–projection pair, and proves the algebraic and computability facts needed later.

Definition (`aSharpInner`). For an endomap a of the universal domain, its inner sharp map is the structural fold on the sharp domain that applies a at leaves and preserves binary pairing.

Proof. Use the sharp-domain recursor with leaf action given by inclusion after a, and node action given by sharp pairing. ■

Definition (`dsharpMap`). A map $f : D_1 \to D_0$ induces a map $D_1^\sharp \to D_0^\sharp$ that applies f to every leaf and preserves the binary-tree structure.

Proof. Define the induced map by structural recursion, using sharp inclusion at leaves and sharp pairing at nodes. ■

Theorem (`dsharpMap_eq`). The induced sharp map is exactly the structural fold with leaf action given by inclusion after the original map and node action given by sharp pairing.

Proof. This is the defining equation of the induced map. ■

Theorem (`dsharpMap_self_eq_aSharpInner`). On the universal domain, the induced sharp map of an endomap a is its inner sharp map.

Proof. Both constructions use the same leaf and pairing operations, so their definitions coincide. ■

Theorem (`grel_inSharp_pairSharp_imp_idMap_rel`). Any relation generated by sharp inclusion at leaves and sharp pairing at nodes relates two valid sharp neighbourhoods W, Z with $W \subseteq Z$.

Proof. Induct on the generated relation. The empty and leaf cases are immediate; in the node case combine the two inductive inclusions coordinatewise and then use the inclusion supplied by the pairing relation. ■

Definition (`iSharp`). Choose the embedding of the sharp universal domain into the universal domain supplied by universality.

Proof. The sharp domain has an effective countable presentation, so the strong universal embedding theorem supplies such a map. ■

Definition (`ISharpComb`). For an endomap a, the forward sharp combinator embeds the sharp domain of the fixed neighbourhood subsystem of a into the universal domain.

Proof. First apply the sharp map induced by the subsystem inclusion, then apply the chosen universal embedding. ■

Theorem (`idMap_rel_imp_grel_inSharp_pairSharp`). If W and Z are valid sharp neighbourhoods and $W \subseteq Z$, then the identity relation is generated from sharp inclusion and sharp pairing.

Proof. Induct on the construction of W. The empty case forces Z to be empty, the leaf case is a generated leaf relation, and the pair case is assembled from the two induction hypotheses and the pairing witness. ■

Theorem (`gMap_inSharp_pairSharp_eq_idMap`). Structural recursion with sharp inclusion at leaves and sharp pairing at nodes is the identity map on $D^\sharp$.

Proof. The two preceding implications show that its relation is equivalent to the identity relation on every pair of neighbourhoods. ■

Theorem (`dsharpMap_id`). The sharp map induced by the identity on D is the identity on $D^\sharp$.

Proof. Its structural fold has sharp inclusion as leaf action and sharp pairing as node action, hence is the identity by the preceding theorem. ■

Theorem (`inSharp_principal`). Sharp inclusion sends the principal element generated by a neighbourhood X to the principal sharp element generated by the corresponding leaf.

Proof. A sharp neighbourhood lies in the image exactly when it is above that leaf, which is the defining membership condition for the principal sharp element. ■

Definition (`jSharp`). Choose the projection from the universal domain onto the sharp universal domain supplied with the universal embedding.

Proof. The strong universal embedding theorem supplies a projection paired with the chosen embedding. ■

Definition (`JSharpComb`). For an endomap a, the backward sharp combinator maps the universal domain into the sharp domain of the fixed neighbourhood subsystem of a.

Proof. Apply the chosen universal projection, then the sharp map induced by the subsystem projection. ■

Definition (`aSharp`). The sharp transform of an endomap a is obtained by projecting into the sharp universal domain, applying the inner sharp map of a, and embedding back into the universal domain.

Proof. Compose the chosen projection, the structural sharp action of a, and the chosen embedding. ■

Theorem (`aSharp.eq`). The sharp transform of a is the composite of the chosen projection, its inner sharp map, and the chosen embedding.

Proof. This is the defining equation of the sharp transform. ■

Theorem (`jSharp_comp_iSharp`). Projecting after embedding the sharp universal domain is the identity.

Proof. This is the left-inverse equation of the chosen universal embedding–projection pair. ■

Theorem (`pair_principal_eq_principal_prodNbhd`). The pair of two principal elements is the principal product element generated by the corresponding product neighbourhood.

Proof. Membership on either side says that a product neighbourhood lies above coordinate neighbourhoods lying above the two generators; the coordinatewise characterization of product inclusion makes these conditions equivalent. ■

Theorem (`pairSharp_principal`). Sharp pairing of principal sharp elements generated by P and Q is the principal sharp element generated by the tree node (P, Q).

Proof. Rewrite the pair as the principal product element and compare membership using the defining relation of sharp pairing. Both sides say exactly that a sharp neighbourhood lies above the paired generator. ■

Theorem (`gMap_eq_of_satisfies`). A strict map out of $D^\sharp$ is uniquely determined by its values on sharp leaves and sharp pairs. If those values agree with prescribed leaf and node operations, the map equals the structural fold generated by those operations.

Proof. Induct on sharp neighbourhoods. Strictness handles the empty constructor, the leaf equation handles leaves, and the pair equation together with the two induction hypotheses handles nodes; extensionality then identifies the maps. ■

Theorem (`dsharpMap_comp`). Sharp lifting preserves composition: the lift of $f' \circ f$ is the composite of the lift of f with the lift of f'.

Proof. The composite is strict, acts on leaves by $f' \circ f$, and preserves sharp pairing. The uniqueness principle for structural folds therefore identifies it with the lift of the composite. ■

Theorem (`ISharpComb_comp_JSharpComb`). If a is a finitary projection, composing the backward sharp combinator with the forward one in the universal domain gives the sharp transform of a.

Proof. Functoriality combines the lifted subsystem projection and inclusion into the lift of a; on the fixed subsystem their composite represents a, and this lift is the inner sharp action used in the sharp transform. ■

Theorem (`JSharpComb_comp_ISharpComb`). For every a, composing the forward sharp combinator with the backward one on the sharp fixed subsystem is the identity.

Proof. Cancel the chosen universal embedding–projection pair, use functoriality of sharp lifting, and use that subsystem projection after inclusion is the identity. ■

Definition (`dsharp_fixedNbhd_elementIso`). For a finitary projection a, the element poset of the sharp fixed neighbourhood system of a is order-isomorphic to the fixed-point poset of the sharp transform of a.

Proof. The two sharp combinators form a mutually inverse projection pair, so their element maps restrict to the required order isomorphism. ■

Theorem (`gMap_selfComp_eq_of_isRetraction`). If $a : D \to D$ is idempotent, then the structural sharp map applying a at leaves and preserving pairs is also idempotent.

Proof. Apply the uniqueness principle for structural folds to its self-composite. At leaves, two applications reduce to one by idempotence of a; at pairs, both applications preserve the same binary structure. ■

Theorem (`GRel.mono_uv`). The relation generated from leaf operation u and node operation v is monotone in both operations: enlarging u and v preserves every generated relation.

Proof. Induct on the generation of the relation. The empty case is unchanged, while the leaf and node cases use the corresponding assumed inequalities and the induction hypotheses. ■

Theorem (`aSharpInner_eq`). The inner sharp action of a is the structural fold that applies inclusion after a at leaves and sharp pairing at nodes.

Proof. This is its defining equation. ■

Theorem (`aSharpInner_idem_of_isRetraction`). If a is idempotent, then its inner sharp action is idempotent.

Proof. Apply the idempotence theorem for structural sharp maps to a. ■

Theorem (`gMap_mono`). Structural sharp recursion is monotone in both its leaf operation and its node operation.

Proof. Every relation generated from the smaller operations remains generated from the larger ones by monotonicity of the inductively generated relation. ■

Theorem (`iSharp_comp_jSharp_le`). Embedding after projecting the universal domain through its sharp retract lies below the identity.

Proof. This is the order inequality of the chosen universal embedding–projection pair. ■

Theorem (`dsharpU_trianglelefteq_U`). The sharp universal domain is a retract of the universal domain.

Proof. The chosen embedding and projection have identity projection–embedding composite, while their reverse composite lies below the identity. ■

Theorem (`iSharp_isComputableMap`). The chosen embedding of the sharp universal domain into the universal domain is computable with respect to their effective presentations.

Proof. This is the computability component supplied by the strong universal embedding theorem. ■

Theorem (`jSharp_isComputableMap`). The chosen projection from the universal domain to the sharp universal domain is computable with respect to their effective presentations.

Proof. This is the computability component for the projection supplied by the strong universal embedding theorem. ■

ASharp.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_17

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_10
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.funSpaceIso
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_3
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Combinators
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap IndefProduct Proposition77 DsharpRecursion
    SmashProduct

variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}

variable {hD : ∀ X, D.mem X → X.Nonempty}

variable {u : ApproximableMap D E} {v : ApproximableMap (prod E E) E}

noncomputable def aSharpInner (a : ApproximableMap U U) :
    ApproximableMap (Dsharp U U_mem_nonempty) (Dsharp U U_mem_nonempty) :=
  gMap ((inSharpMap U_mem_nonempty).comp a) (pairSharpMap U_mem_nonempty)
    U_mem_nonempty
```

```
section DsharpFunctor

variable {? ? ? : Type*} {D₀ : NeighborhoodSystem ?} {D₁ : NeighborhoodSystem ?}
  {D₂ : NeighborhoodSystem ?}

variable (hD₀ : ∀ X, D₀.mem X → X.Nonempty) (hD₁ : ∀ X, D₁.mem X → X.Nonempty)
  {hD₂ : ∀ X, D₂.mem X → X.Nonempty}

noncomputable def dsharpMap (f : ApproximableMap D₁ D₀) :
    ApproximableMap (Dsharp D₁ hD₁) (Dsharp D₀ hD₀) :=
  gMap ((inSharpMap hD₀).comp f) (pairSharpMap hD₀) hD₁

theorem dsharpMap_eq (f : ApproximableMap D₁ D₀) :
    dsharpMap hD₀ hD₁ f = gMap ((inSharpMap hD₀).comp f) (pairSharpMap hD₀) hD₁ := rfl

theorem dsharpMap_self_eq_aSharpInner (a : ApproximableMap U U) :
    dsharpMap U_mem_nonempty U_mem_nonempty a = aSharpInner a := rfl

end DsharpFunctor

theorem grel_inSharp_pairSharp_imp_idMap_rel :
    ∀ {W Z}, GRel (inSharpMap hD) (pairSharpMap hD) W Z → MemS D W ∧ MemS D Z ∧ W ⊆ Z
     := by
  intro W Z h
  induction h with
  | gamma hZ => exact ⟨MemS.gamma, hZ ▸ MemS.gamma, hZ ▸ subset_rfl⟩
  | leaf hrel => exact ⟨MemS.zero hrel.1, hrel.2.1, hrel.2.2⟩
  | @node P Q Z₁ Z₂ Z hP hQ hvrel ihP ihQ =>
      obtain ⟨hPmem, hZ1mem, hPZ1⟩ := ihP
      obtain ⟨hQmem, hZ2mem, hQZ2⟩ := ihQ
      obtain ⟨-, hZmem, A, B, hAmem, hBmem, heq, hsub⟩ := hvrel
      obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
      exact ⟨MemS.pair hPmem hQmem, hZmem, (embPair_subset.mpr ⟨hPZ1, hQZ2⟩).trans hsub⟩

noncomputable def iSharp : ApproximableMap (Dsharp U U_mem_nonempty) U :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose

noncomputable def ISharpComb (a : ApproximableMap U U) :
    ApproximableMap (Dsharp (fixedNbhd a) (fixedNbhd_mem_nonempty a)) U :=
  iSharp.comp (dsharpMap U_mem_nonempty (fixedNbhd_mem_nonempty a) (
    fixedNbhd_subsystem a).inj)

theorem idMap_rel_imp_grel_inSharp_pairSharp :
    ∀ {W}, MemS D W → ∀ {Z}, MemS D Z → W ⊆ Z → GRel (inSharpMap hD) (pairSharpMap
    hD) W Z := by
  intro W hW
  induction hW with
  | gamma =>
      intro Z hZ hsub
      have hZeq : Z = Gamma D := Set.Subset.antisymm (memS_subset_gamma hZ) hsub
      subst hZeq
      exact GRel.gamma rfl
  | @zero X hX =>
      intro Z hZ hsub
      exact GRel.leaf ⟨hX, hZ, hsub⟩
```

```
  | @pair P Q hP hQ ihP ihQ =>
      intro Z hZ hsub
      exact GRel.node (ihP hP subset_rfl) (ihQ hQ subset_rfl)
        ⟨prod_mem_prodNbhd hP hQ, hZ, P, Q, hP, hQ, rfl, hsub⟩

theorem gMap_inSharp_pairSharp_eq_idMap :
    gMap (inSharpMap hD) (pairSharpMap hD) hD = idMap (Dsharp D hD) := by
  apply le_antisymm
  · rw [ApproximableMap.le_iff]
    intro W Z h
    exact grel_inSharp_pairSharp_imp_idMap_rel h
  · rw [ApproximableMap.le_iff]
    intro W Z h
    exact idMap_rel_imp_grel_inSharp_pairSharp h.1 h.2.1 h.2.2

section DsharpFunctor

variable {? ? ? : Type*} {D₀ : NeighborhoodSystem ?} {D₁ : NeighborhoodSystem ?}
  {D₂ : NeighborhoodSystem ?}

variable (hD₀ : ∀ X, D₀.mem X → X.Nonempty) (hD₁ : ∀ X, D₁.mem X → X.Nonempty)
  {hD₂ : ∀ X, D₂.mem X → X.Nonempty}

theorem dsharpMap_id : dsharpMap hD₀ hD₀ (idMap D₀) = idMap (Dsharp D₀ hD₀) := by
  rw [dsharpMap_eq, comp_idMap]
  exact gMap_inSharp_pairSharp_eq_idMap

end DsharpFunctor

theorem inSharp_principal {X : Set α} (hX : D.mem X) :
    inSharp D hD (D.principal hX) = (Dsharp D hD).principal (MemS.zero hX) := by
  rw [← inSharpMap_toElementMap hD]
  apply Element.ext
  intro W
  simp only [mem_principal, ← rel_iff_mem_principal (inSharpMap hD) hX, inSharpMap_rel
    ]
  exact ⟨fun h => h.2, fun h => ⟨hX, h⟩⟩

noncomputable def jSharp : ApproximableMap U (Dsharp U U_mem_nonempty) :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose_spec.choose

noncomputable def JSharpComb (a : ApproximableMap U U) :
    ApproximableMap U (Dsharp (fixedNbhd a) (fixedNbhd_mem_nonempty a)) :=
  (dsharpMap (fixedNbhd_mem_nonempty a) U_mem_nonempty (fixedNbhd_subsystem a).proj).
    comp jSharp

noncomputable def aSharp (a : ApproximableMap U U) : ApproximableMap U U :=
  iSharp.comp ((aSharpInner a).comp jSharp)

namespace aSharp

theorem eq (a : ApproximableMap U U) :
    aSharp a = iSharp.comp ((aSharpInner a).comp jSharp) := rfl

end aSharp
```

```
theorem jSharp_comp_iSharp : jSharp.comp iSharp = idMap _ :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose_spec.choose_spec.1

theorem pair_principal_eq_principal_prodNbhd {? ? : Type*} {V₀ : NeighborhoodSystem ?}
    {V₁ : NeighborhoodSystem ?} {P : Set ?} {Q : Set ?} (hP : V₀.mem P) (hQ : V₁.mem Q)
     :
    pair (V₀.principal hP) (V₁.principal hQ)
      = (prod V₀ V₁).principal (prod_mem_prodNbhd hP hQ) := by
  apply Element.ext
  intro W
  simp only [mem_pair, mem_principal, prod_mem_iff]
  constructor
  · rintro ⟨P', Q', ⟨hP', hPP'⟩, ⟨hQ', hQQ'⟩, rfl⟩
    exact ⟨⟨P', Q', hP', hQ', rfl⟩, prodNbhd.subset_iff.mpr ⟨hPP', hQQ'⟩⟩
  · rintro ⟨⟨P', Q', hP', hQ', rfl⟩, hsub⟩
    obtain ⟨hPP', hQQ'⟩ := prodNbhd.subset_iff.mp hsub
    exact ⟨P', Q', ⟨hP', hPP'⟩, ⟨hQ', hQQ'⟩, rfl⟩

theorem pairSharp_principal {P Q : Set (List Bool × α)} (hP : MemS D P) (hQ : MemS D Q
    ) :
    pairSharp D hD ((Dsharp D hD).principal hP) ((Dsharp D hD).principal hQ)
      = (Dsharp D hD).principal (MemS.pair hP hQ) := by
  rw [← pairSharpMap_toElementMap hD, pair_principal_eq_principal_prodNbhd hP hQ]
  apply Element.ext
  intro W
  rw [mem_principal, ← rel_iff_mem_principal (pairSharpMap hD) (prod_mem_prodNbhd hP
    hQ),
    pairSharpMap_rel]
  constructor
  · rintro ⟨-, hW, A, B, hA, hB, heq, hsub⟩
    obtain ⟨rfl, rfl⟩ := prodNbhd.injective heq
    exact ⟨hW, hsub⟩
  · rintro ⟨hW, hsub⟩
    exact ⟨prod_mem_prodNbhd hP hQ, hW, P, Q, hP, hQ, rfl, hsub⟩

theorem gMap_eq_of_satisfies {k : ApproximableMap (Dsharp D hD) E} (hk : IsStrict k)
    (hu : ∀ x : D.Element, k.toElementMap (inSharp D hD x) = u.toElementMap x)
    (hv : ∀ y z : (Dsharp D hD).Element,
      k.toElementMap (pairSharp D hD y z)
        = v.toElementMap (pair (k.toElementMap y) (k.toElementMap z))) :
    k = gMap u v hD := by
  have main : ∀ {W}, MemS D W → ∀ Z, k.rel W Z ↔ GRel u v W Z := by
    intro W hW
    induction hW with
    | gamma =>
        intro Z
        constructor
        · intro h; exact GRel.gamma (hk h)
        · intro h
          have hZ : Z = E.master := gRel_gamma_inv hD h
          subst hZ
          exact k.master_rel
    | @zero X hX =>
        intro Z
```

```
        have key := hu (D.principal hX)
        rw [inSharp_principal hX] at key
        constructor
        · intro h
          have h1 := (rel_iff_mem_principal k (MemS.zero hX)).mp h
          rw [key] at h1
          exact GRel.leaf ((rel_iff_mem_principal u hX).mpr h1)
        · intro h
          have h1 := (rel_iff_mem_principal u hX).mp (gRel_embZero_inv hD h)
          rw [← key] at h1
          exact (rel_iff_mem_principal k (MemS.zero hX)).mpr h1
    | @pair P Q hP hQ ihP ihQ =>
        intro Z
        have key := hv ((Dsharp D hD).principal hP) ((Dsharp D hD).principal hQ)
        rw [pairSharp_principal hP hQ] at key
        constructor
        · intro h
          have h1 := (rel_iff_mem_principal k (MemS.pair hP hQ)).mp h
          rw [key] at h1
          obtain ⟨V, hVmem, hVrel⟩ := h1
          obtain ⟨Z1, Z2, hZ1, hZ2, rfl⟩ := hVmem
          have hkZ1 : k.rel P Z1 := (rel_iff_mem_principal k hP).mpr hZ1
          have hkZ2 : k.rel Q Z2 := (rel_iff_mem_principal k hQ).mpr hZ2
          exact GRel.node ((ihP Z1).mp hkZ1) ((ihQ Z2).mp hkZ2) hVrel
        · intro h
          obtain ⟨Z1, Z2, hg1, hg2, hvrel⟩ := gRel_embPair_inv hD h
          have h1 := (rel_iff_mem_principal k hP).mp ((ihP Z1).mpr hg1)
          have h2 := (rel_iff_mem_principal k hQ).mp ((ihQ Z2).mpr hg2)
          have h3 : (v.toElementMap (pair (k.toElementMap ((Dsharp D hD).principal hP)
    )
              (k.toElementMap ((Dsharp D hD).principal hQ)))).mem Z :=
            ⟨prodNbhd Z1 Z2, ⟨Z1, Z2, h1, h2, rfl⟩, hvrel⟩
          rw [← key] at h3
          exact (rel_iff_mem_principal k (MemS.pair hP hQ)).mpr h3
  apply ApproximableMap.ext
  intro W Z
  by_cases hW : MemS D W
  · exact main hW Z
  · exact ⟨fun h => absurd (k.rel_dom h) hW, fun h => absurd (gRel_dom h) hW⟩

section DsharpFunctor

variable {? ? ? : Type*} {D₀ : NeighborhoodSystem ?} {D₁ : NeighborhoodSystem ?}
  {D₂ : NeighborhoodSystem ?}

variable (hD₀ : ∀ X, D₀.mem X → X.Nonempty) (hD₁ : ∀ X, D₁.mem X → X.Nonempty)
  {hD₂ : ∀ X, D₂.mem X → X.Nonempty}

theorem dsharpMap_comp (f : ApproximableMap D₂ D₁) (f' : ApproximableMap D₁ D₀) :
    dsharpMap hD₀ hD₂ (f'.comp f) = (dsharpMap hD₀ hD₁ f').comp (dsharpMap hD₁ hD₂ f) :=
     by
  set g1 : ApproximableMap (Dsharp D₂ hD₂) (Dsharp D₁ hD₁) := dsharpMap hD₁ hD₂ f with
    hg1_def
  set g2 : ApproximableMap (Dsharp D₁ hD₁) (Dsharp D₀ hD₀) := dsharpMap hD₀ hD₁ f' with
    hg2_def
  symm
```

```
  show g2.comp g1 = dsharpMap hD₀ hD₂ (f'.comp f)
  rw [dsharpMap_eq]
  apply gMap_eq_of_satisfies
  · exact isStrict_comp (by rw [hg2_def, dsharpMap_eq]; exact gMap_strict hD₁)
      (by rw [hg1_def, dsharpMap_eq]; exact gMap_strict hD₂)
  · intro x
    have e1 : g1.toElementMap (inSharp D₂ hD₂ x) = inSharp D₁ hD₁ (f.toElementMap x) :=
     by
      rw [hg1_def, dsharpMap_eq, gMap_in hD₂ x, toElementMap_comp,
    inSharpMap_toElementMap]
    have e2 : g2.toElementMap (inSharp D₁ hD₁ (f.toElementMap x))
        = inSharp D₀ hD₀ (f'.toElementMap (f.toElementMap x)) := by
      rw [hg2_def, dsharpMap_eq, gMap_in hD₁ (f.toElementMap x), toElementMap_comp,
        inSharpMap_toElementMap]
    calc (g2.comp g1).toElementMap (inSharp D₂ hD₂ x)
        = g2.toElementMap (g1.toElementMap (inSharp D₂ hD₂ x)) := toElementMap_comp _ _
     _
      _ = g2.toElementMap (inSharp D₁ hD₁ (f.toElementMap x)) := by rw [e1]
      _ = inSharp D₀ hD₀ (f'.toElementMap (f.toElementMap x)) := e2
      _ = ((inSharpMap hD₀).comp (f'.comp f)).toElementMap x := by
          rw [toElementMap_comp, toElementMap_comp, inSharpMap_toElementMap]
  · intro y z
    have e1 : g1.toElementMap (pairSharp D₂ hD₂ y z)
        = pairSharp D₁ hD₁ (g1.toElementMap y) (g1.toElementMap z) := by
      rw [hg1_def, dsharpMap_eq, gMap_pair hD₂ y z, pairSharpMap_toElementMap]
    have e2 : g2.toElementMap (pairSharp D₁ hD₁ (g1.toElementMap y) (g1.toElementMap z)
    )
        = pairSharp D₀ hD₀ (g2.toElementMap (g1.toElementMap y))
          (g2.toElementMap (g1.toElementMap z)) := by
      rw [hg2_def, dsharpMap_eq, gMap_pair hD₁ (g1.toElementMap y) (g1.toElementMap z),
        pairSharpMap_toElementMap]
    calc (g2.comp g1).toElementMap (pairSharp D₂ hD₂ y z)
        = g2.toElementMap (g1.toElementMap (pairSharp D₂ hD₂ y z)) := toElementMap_comp
     _ _ _
      _ = g2.toElementMap (pairSharp D₁ hD₁ (g1.toElementMap y) (g1.toElementMap z)) :=
     by
          rw [e1]
      _ = pairSharp D₀ hD₀ (g2.toElementMap (g1.toElementMap y))
          (g2.toElementMap (g1.toElementMap z)) := e2
      _ = (pairSharpMap hD₀).toElementMap
          (pair (g2.toElementMap (g1.toElementMap y)) (g2.toElementMap (g1.
    toElementMap z))) :=
        (pairSharpMap_toElementMap hD₀ _ _).symm
      _ = (pairSharpMap hD₀).toElementMap
          (pair ((g2.comp g1).toElementMap y) ((g2.comp g1).toElementMap z)) := by
        rw [toElementMap_comp, toElementMap_comp]

end DsharpFunctor

theorem ISharpComb_comp_JSharpComb {a : ApproximableMap U U} (ha : FinitaryProjection
    a) :
    (ISharpComb a).comp (JSharpComb a) = aSharp a := by
  unfold ISharpComb JSharpComb
  rw [comp_assoc, ← comp_assoc (dsharpMap _ _ _) (dsharpMap _ _ _) jSharp, ←
    dsharpMap_comp,
    inj_comp_proj_eq_self ha, dsharpMap_self_eq_aSharpInner]
```

```
  exact (aSharp.eq a).symm

theorem JSharpComb_comp_ISharpComb (a : ApproximableMap U U) :
    (JSharpComb a).comp (ISharpComb a) =
      idMap (Dsharp (fixedNbhd a) (fixedNbhd_mem_nonempty a)) := by
  unfold JSharpComb ISharpComb
  rw [comp_assoc, ← comp_assoc jSharp iSharp (dsharpMap _ _ _), jSharp_comp_iSharp,
    idMap_comp,
    ← dsharpMap_comp, (fixedNbhd_subsystem a).proj_comp_inj, dsharpMap_id]

noncomputable def dsharp_fixedNbhd_elementIso {a : ApproximableMap U U}
    (ha : FinitaryProjection a) :
    (Dsharp (fixedNbhd a) (fixedNbhd_mem_nonempty a)).Element ≃o
      {y : U.Element // (aSharp a).toElementMap y = y} :=
  elementIsoOfProjectionPair (ISharpComb a) (JSharpComb a) (JSharpComb_comp_ISharpComb
    a)
    (ISharpComb_comp_JSharpComb ha).symm

theorem gMap_selfComp_eq_of_isRetraction {a : ApproximableMap D D} (ha : Retraction a)
    :
    (gMap ((inSharpMap hD).comp a) (pairSharpMap hD) hD).comp
        (gMap ((inSharpMap hD).comp a) (pairSharpMap hD) hD)
      = gMap ((inSharpMap hD).comp a) (pairSharpMap hD) hD := by
  set u : ApproximableMap D (Dsharp D hD) := (inSharpMap hD).comp a with hu_def
  set v : ApproximableMap (prod (Dsharp D hD) (Dsharp D hD)) (Dsharp D hD) :=
    pairSharpMap hD with hv_def
  set g : ApproximableMap (Dsharp D hD) (Dsharp D hD) := gMap u v hD with hg_def
  have hu_eq : ∀ y : D.Element, u.toElementMap y = inSharp D hD (a.toElementMap y) :=
    by
    intro y; rw [hu_def, toElementMap_comp, inSharpMap_toElementMap]
  show g.comp g = g
  apply gMap_eq_of_satisfies
  · rw [hg_def]; exact isStrict_comp (gMap_strict hD) (gMap_strict hD)
  · intro x
    have e1 : g.toElementMap (inSharp D hD x) = u.toElementMap x := by
      rw [hg_def]; exact gMap_in hD x
    have e2 : g.toElementMap (u.toElementMap x) = u.toElementMap (a.toElementMap x) :=
     by
      rw [hu_eq x, hg_def]; exact gMap_in hD (a.toElementMap x)
    calc (g.comp g).toElementMap (inSharp D hD x)
        = g.toElementMap (g.toElementMap (inSharp D hD x)) := toElementMap_comp _ _ _
      _ = g.toElementMap (u.toElementMap x) := by rw [e1]
      _ = u.toElementMap (a.toElementMap x) := e2
      _ = inSharp D hD (a.toElementMap (a.toElementMap x)) := hu_eq _
      _ = inSharp D hD (a.toElementMap x) := by rw [toElementMap_idem_of_isRetraction
    ha]
      _ = u.toElementMap x := (hu_eq x).symm
  · intro y z
    have e1 : g.toElementMap (pairSharp D hD y z)
        = v.toElementMap (pair (g.toElementMap y) (g.toElementMap z)) := by
      rw [hg_def]; exact gMap_pair hD y z
    have e2 : v.toElementMap (pair (g.toElementMap y) (g.toElementMap z))
        = pairSharp D hD (g.toElementMap y) (g.toElementMap z) := by
      rw [hv_def]; exact pairSharpMap_toElementMap hD _ _
    have e3 : g.toElementMap (pairSharp D hD (g.toElementMap y) (g.toElementMap z))
        = v.toElementMap
```

```
            (pair (g.toElementMap (g.toElementMap y)) (g.toElementMap (g.toElementMap
    z))) := by
      rw [hg_def]; exact gMap_pair hD _ _
    calc (g.comp g).toElementMap (pairSharp D hD y z)
        = g.toElementMap (g.toElementMap (pairSharp D hD y z)) := toElementMap_comp _
    _ _
      _ = g.toElementMap (v.toElementMap (pair (g.toElementMap y) (g.toElementMap z)))
     := by
          rw [e1]
      _ = g.toElementMap (pairSharp D hD (g.toElementMap y) (g.toElementMap z)) := by
    rw [e2]
      _ = v.toElementMap
            (pair (g.toElementMap (g.toElementMap y)) (g.toElementMap (g.toElementMap
    z))) := e3
      _ = v.toElementMap (pair ((g.comp g).toElementMap y) ((g.comp g).toElementMap z)
    ) := by
          rw [toElementMap_comp, toElementMap_comp]

theorem GRel.mono_uv (_hD : ∀ X, D.mem X → X.Nonempty)
    {u u' : ApproximableMap D E} {v v' : ApproximableMap (prod E E) E}
    (hu : u ≤ u') (hv : v ≤ v') :
    ∀ {W Z}, GRel u v W Z → GRel u' v' W Z := by
  intro W Z h
  induction h with
  | gamma hZ => exact GRel.gamma hZ
  | leaf hrel => exact GRel.leaf (hu _ _ hrel)
  | node hP hQ hvrel ihP ihQ => exact GRel.node ihP ihQ (hv _ _ hvrel)

theorem aSharpInner_eq (a : ApproximableMap U U) :
    aSharpInner a = gMap ((inSharpMap U_mem_nonempty).comp a) (pairSharpMap
    U_mem_nonempty)
      U_mem_nonempty := rfl

theorem aSharpInner_idem_of_isRetraction {a : ApproximableMap U U} (ha : Retraction a)
    :
    Retraction (aSharpInner a) := by
  constructor
  show (aSharpInner a).comp (aSharpInner a) = aSharpInner a
  rw [aSharpInner_eq]
  exact gMap_selfComp_eq_of_isRetraction ha

theorem gMap_mono (hD : ∀ X, D.mem X → X.Nonempty)
    {u u' : ApproximableMap D E} {v v' : ApproximableMap (prod E E) E}
    (hu : u ≤ u') (hv : v ≤ v') : gMap u v hD ≤ gMap u' v' hD := by
  rw [ApproximableMap.le_iff]
  intro W Z h
  exact GRel.mono_uv hD hu hv h

theorem iSharp_comp_jSharp_le : iSharp.comp jSharp ≤ idMap U :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose_spec.choose_spec.2.1

theorem dsharpU_trianglelefteq_U : Dsharp U U_mem_nonempty ? U :=
  trianglelefteq_of_projectionPair iSharp jSharp jSharp_comp_iSharp
    iSharp_comp_jSharp_le
```

```
theorem iSharp_isComputableMap :
    ComputableMap (dsharpPresentation UComputablePresentation U_mem_nonempty)
      UComputablePresentation iSharp :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose_spec.choose_spec.2.2.1

theorem jSharp_isComputableMap :
    ComputableMap UComputablePresentation
      (dsharpPresentation UComputablePresentation U_mem_nonempty) jSharp :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose_spec.choose_spec.2.2.2

end DomainTheory.NeighbourhoodSystems
```

4.9.60 A-sharp (more)

This peer proves that the sharp transform preserves projections, finitarity, and computability, then develops order isomorphisms for function spaces and derives self-similarity equations for domains.

Theorem (`aSharp.le_idMap_of_le`). If a lies below the identity on the universal domain, then its sharp transform also lies below the identity.

Proof. Monotonicity of structural sharp recursion places the inner sharp action below the identity on the sharp domain. Composing with the universal embedding–projection pair preserves the inequality, and that pair's reverse composite lies below the universal identity. ■

Theorem (`aSharp.isFinitary`). If a is a finitary projection, then its sharp transform is finitary.

Proof. The forward and backward sharp combinators exhibit the sharp transform as the projection associated with the sharp fixed neighbourhood system, which has the required finite presentation. ■

Theorem (`aSharp.isComputable`). If a is computable, then its sharp transform is computable.

Proof. The induced inner sharp action is computable because sharp inclusion, sharp pairing, and a are computable. Compose it with the computable universal projection and embedding. ■

Theorem (`toElementMap_aSharp`). On elements, the sharp transform sends x first through the universal projection, then through the inner sharp action, and finally through the universal embedding.

Proof. Element maps carry composition to successive application. ■

Theorem (`isRetraction_aSharp`). If the inner sharp action of a is idempotent, then the sharp transform of a is idempotent.

Proof. In the square of the sharp transform, the middle projection–embedding pair cancels to the identity, and the two remaining inner actions collapse to one by idempotence. ■

Theorem (`isProjection_aSharp`). If a is a projection, then its sharp transform is a projection.

Proof. Idempotence of a gives idempotence of its inner sharp action and hence of its sharp transform; the inequality $a \leq \mathrm{id}$ gives the corresponding inequality for the transform. ■

Theorem (`isFinitaryProjection_aSharp`). The sharp transform of a finitary projection is again a finitary projection.

Proof. The transform is a projection by the preceding result and is finitary by the sharp combinator presentation. ■

Theorem (`ofIso_comp_ofIso_symm`). For an order isomorphism $e : V_0 \cong V_0'$, the approximable maps induced by e^{-1} and then e compose to the identity on V_0'.

Proof. Their element maps apply e^{-1} and then e, which are mutually inverse. ■

Theorem (`funSpaceCongr_hij`). The two function-space maps induced contravariantly by an order isomorphism of source domains and covariantly by an order isomorphism of target domains compose to the identity on the primed function space.

Proof. Composition of exponential maps reduces to composition in the source and target components; both component composites are identities by the inverse laws for the two order isomorphisms. ■

Theorem (`ofIso_symm_comp_ofIso`). For an order isomorphism $e : V_0 \cong V_0'$, the approximable maps induced by e and then e^{-1} compose to the identity on V_0.

Proof. Their element maps apply e and then e^{-1}, which are mutually inverse. ■

Theorem (`funSpaceCongr_hji`). The function-space maps induced in the two opposite directions compose to the identity on the unprimed function space.

Proof. Reduce composition of exponential maps to the source and target component composites, each of which is the identity by the inverse laws. ■

Definition (`orderIsoOfMutualInverse`). Two approximable maps $i : D \to E$ and $j : E \to D$ whose composites are identities determine an order isomorphism between the element posets of D and E.

Proof. Use their element maps as the forward and inverse functions. The composite equations give the inverse laws, and monotonicity in both directions reflects as well as preserves order. ■

Definition (`funSpaceCongrOrderIso`). Order isomorphisms $V_0 \cong V_0'$ and $V_1 \cong V_1'$ induce an order isomorphism $[V_0 \to V_1] \cong [V_0' \to V_1']$.

Proof. Use the exponential maps obtained by transporting arguments contravariantly and values covariantly; the two preceding function-space inverse equations show that these maps are mutual inverses. ■

Theorem (`curry_isomorphic`). The function-space domains $[V_0 \times V_1 \to V_2]$ and $[V_0 \to [V_1 \to V_2]]$ are isomorphic.

Proof. The standard currying order isomorphism supplies the domain isomorphism. ■

Theorem (`funSpace_congr`). Isomorphic source domains and isomorphic target domains have isomorphic function-space domains.

Proof. Choose order isomorphisms witnessing the two domain isomorphisms and apply the induced function-space order isomorphism. ■

Theorem (`funSpace_congr_left`). Replacing the source of a function-space domain by an isomorphic domain preserves its isomorphism type.

Proof. Apply function-space congruence to the given source isomorphism and the identity isomorphism of the target. ■

Theorem (`funSpace_congr_right`). Replacing the target of a function-space domain by an isomorphic domain preserves its isomorphism type.

Proof. Apply function-space congruence to the identity isomorphism of the source and the given target isomorphism. ■

Theorem (`prod_self_isomorphic`). If $D \cong [D \to V]$ and $V \times V \cong V$, then $D \times D \cong D$.

Proof. Replace each copy of D by $[D \to V]$, identify their product with $[D \to V \times V]$, replace $V \times V$ by V, and finally use the inverse of the original isomorphism. ■

Theorem (`funSpace_self_isomorphic`). If $D \cong [D \to V]$ and $V \times V \cong V$, then $D \cong [D \to D]$.

Proof. First obtain $D \times D \cong D$. Replace the codomain D by $[D \to V]$, uncurry to obtain functions on $D \times D$, replace that product by D, and use the original isomorphism; reversing the resulting isomorphism gives the claim. ■

ASharp_more.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.ASharp

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap IndefProduct Proposition77 DsharpRecursion
    SmashProduct

variable {α β : Type*} {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}
variable {hD : ∀ X, D.mem X → X.Nonempty}
variable {u : ApproximableMap D E} {v : ApproximableMap (prod E E) E}

namespace aSharp

theorem le_idMap_of_le {a : ApproximableMap U U} (ha : a ≤ idMap U) :
    aSharp a ≤ idMap U := by
  have h1 : (inSharpMap U_mem_nonempty).comp a ≤ inSharpMap U_mem_nonempty := by
    calc (inSharpMap U_mem_nonempty).comp a
        ≤ (inSharpMap U_mem_nonempty).comp (idMap U) := comp_mono_gen le_rfl ha
      _ = inSharpMap U_mem_nonempty := comp_idMap _
  have h2 : aSharpInner a ≤ idMap (Dsharp U U_mem_nonempty) := by
    calc aSharpInner a
        ≤ gMap (inSharpMap U_mem_nonempty) (pairSharpMap U_mem_nonempty)
    U_mem_nonempty :=
          gMap_mono U_mem_nonempty h1 le_rfl
      _ = idMap (Dsharp U U_mem_nonempty) := gMap_inSharp_pairSharp_eq_idMap
  calc aSharp a = iSharp.comp ((aSharpInner a).comp jSharp) := eq a
    _ ≤ iSharp.comp ((idMap (Dsharp U U_mem_nonempty)).comp jSharp) :=
        comp_mono_gen le_rfl (comp_mono_gen h2 le_rfl)
    _ = iSharp.comp jSharp := by rw [idMap_comp]
    _ ≤ idMap U := iSharp_comp_jSharp_le

theorem isFinitary {a : ApproximableMap U U} (ha : FinitaryProjection a) :
    IsFinitary (aSharp a) :=
  isFinitary_of_projectionPair (ISharpComb a) (JSharpComb a) (
    JSharpComb_comp_ISharpComb a)
    (ISharpComb_comp_JSharpComb ha).symm

theorem isComputable {a : ApproximableMap U U}
    (ha : ComputableMap UComputablePresentation UComputablePresentation a) :
    ComputableMap UComputablePresentation UComputablePresentation (aSharp a) := by
  have hu : ComputableMap UComputablePresentation
      (dsharpPresentation UComputablePresentation U_mem_nonempty)
      ((inSharpMap U_mem_nonempty).comp a) :=
    comp_isComputable ha (inSharp_isComputable U_mem_nonempty UComputablePresentation)
  have hInner : ComputableMap (dsharpPresentation UComputablePresentation
    U_mem_nonempty)
      (dsharpPresentation UComputablePresentation U_mem_nonempty) (aSharpInner a) :=
    gMap_isComputable U_mem_nonempty UComputablePresentation
      (dsharpPresentation UComputablePresentation U_mem_nonempty) hu
      (pairSharp_isComputable U_mem_nonempty UComputablePresentation)
  exact comp_isComputable (comp_isComputable jSharp_isComputableMap hInner)
    iSharp_isComputableMap
```

```
end aSharp

theorem toElementMap_aSharp (a : ApproximableMap U U) (x : U.Element) :
    (aSharp a).toElementMap x
      = iSharp.toElementMap ((aSharpInner a).toElementMap (jSharp.toElementMap x)) :=
    by
  rw [aSharp.eq]
  simp only [toElementMap_comp]

theorem isRetraction_aSharp {a : ApproximableMap U U} (ha2 : Retraction (aSharpInner a
    )) :
    Retraction (aSharp a) := by
  constructor
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_aSharp a ((aSharp a).toElementMap x),
    toElementMap_aSharp a x, toElementMap_of_comp_eq_idMap jSharp_comp_iSharp,
    toElementMap_idem_of_isRetraction ha2]

theorem isProjection_aSharp {a : ApproximableMap U U} (ha : IsProjection a) :
    IsProjection (aSharp a) :=
  ⟨isRetraction_aSharp (aSharpInner_idem_of_isRetraction ha.1), aSharp.le_idMap_of_le
    ha.2⟩

theorem isFinitaryProjection_aSharp {a : ApproximableMap U U} (ha : FinitaryProjection
     a) :
    FinitaryProjection (aSharp a) :=
  ⟨⟨isProjection_aSharp ha.out.1, aSharp.isFinitary ha⟩⟩

variable {? α' β' : Type*}
variable {V₀ : NeighborhoodSystem α} {V₁ : NeighborhoodSystem β}
variable {V₀' : NeighborhoodSystem α'} {V₁' : NeighborhoodSystem β'}
variable {V : NeighborhoodSystem β}

theorem ofIso_comp_ofIso_symm (e : V₀.Element ≃o V₀'.Element) :
    (ofIso e).comp (ofIso e.symm) = idMap V₀' := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap,
    OrderIso.apply_symm_apply]

theorem funSpaceCongr_hij (e0 : V₀.Element ≃o V₀'.Element) (e1 : V₁.Element ≃o V₁'.
    Element) :
    (expMap (ofIso e0.symm) (ofIso e1)).comp (expMap (ofIso e0) (ofIso e1.symm)) =
      idMap (funSpace V₀' V₁') := by
  rw [← expMap.comp, ofIso_comp_ofIso_symm, ofIso_comp_ofIso_symm, expMap.id]

theorem ofIso_symm_comp_ofIso (e : V₀.Element ≃o V₀'.Element) :
    (ofIso e.symm).comp (ofIso e) = idMap V₀ := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap,
    OrderIso.symm_apply_apply]

theorem funSpaceCongr_hji (e0 : V₀.Element ≃o V₀'.Element) (e1 : V₁.Element ≃o V₁'.
```

```
    Element) :
    (expMap (ofIso e0) (ofIso e1.symm)).comp (expMap (ofIso e0.symm) (ofIso e1)) =
      idMap (funSpace V₀ V₁) := by
  rw [← expMap.comp, ofIso_symm_comp_ofIso, ofIso_symm_comp_ofIso, expMap.id]

def orderIsoOfMutualInverse {D : NeighborhoodSystem α} {E : NeighborhoodSystem β}
    (i : ApproximableMap D E) (j : ApproximableMap E D)
    (hji : j.comp i = idMap D) (hij : i.comp j = idMap E) : D.Element ≃o E.Element
    where
  toFun := i.toElementMap
  invFun := j.toElementMap
  left_inv x := by rw [← toElementMap_comp, hji, toElementMap_idMap]
  right_inv y := by rw [← toElementMap_comp, hij, toElementMap_idMap]
  map_rel_iff' := by
    intro x x'
    show i.toElementMap x ≤ i.toElementMap x' ↔ x ≤ x'
    constructor
    · intro h
      have hm := j.toElementMap_mono h
      rwa [← toElementMap_comp, ← toElementMap_comp, hji, toElementMap_idMap,
        toElementMap_idMap] at hm
    · intro h
      exact i.toElementMap_mono h

noncomputable def funSpaceCongrOrderIso (e0 : V₀.Element ≃o V₀'.Element)
    (e1 : V₁.Element ≃o V₁'.Element) :
    (funSpace V₀ V₁).Element ≃o (funSpace V₀' V₁').Element :=
  orderIsoOfMutualInverse (expMap (ofIso e0.symm) (ofIso e1)) (expMap (ofIso e0) (
    ofIso e1.symm))
    (funSpaceCongr_hji e0 e1) (funSpaceCongr_hij e0 e1)

theorem curry_isomorphic (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    (V₂ : NeighborhoodSystem ?) :
    funSpace (prod V₀ V₁) V₂ ≅? funSpace V₀ (funSpace V₁ V₂) :=
  ⟨curryIso V₀ V₁ V₂⟩

theorem funSpace_congr (h0 : V₀ ≅? V₀') (h1 : V₁ ≅? V₁') :
    funSpace V₀ V₁ ≅? funSpace V₀' V₁' :=
  h0.elim fun e0 => h1.elim fun e1 => ⟨funSpaceCongrOrderIso e0 e1⟩

theorem funSpace_congr_left (h0 : V₀ ≅? V₀') : funSpace V₀ V₁ ≅? funSpace V₀' V₁ :=
  funSpace_congr h0 (Isomorphic.refl V₁)

theorem funSpace_congr_right (h1 : V₁ ≅? V₁') : funSpace V₀ V₁ ≅? funSpace V₀ V₁' :=
  funSpace_congr (Isomorphic.refl V₀) h1

theorem prod_self_isomorphic (hDV : D ≅? funSpace D V) (hVV : prod V V ≅? V) :
    prod D D ≅? D :=
  ((hDV.prod hDV).trans (funProd_isomorphic).symm).trans
    ((funSpace_congr_right hVV).trans hDV.symm)

theorem funSpace_self_isomorphic (hDV : D ≅? funSpace D V) (hVV : prod V V ≅? V) :
    D ≅? funSpace D D :=
  have hDD : prod D D ≅? D := prod_self_isomorphic hDV hVV
  have step : funSpace D D ≅? D :=
    ((funSpace_congr_right hDV).trans (curry_isomorphic D D V).symm).trans
```

```
      ((funSpace_congr_left hDD).trans hDV.symm)
  step.symm

end DomainTheory.NeighbourhoodSystems
```

4.9.61 Exercise 8.21 — lists of a projection

NeighbourhoodSystems/RetractsOfTheUniversalDomain/Exercise_8_21.lean

@import "Exercise_8_21.lean"

EXERCISE 8.21. Using the fixed-point construction, show that there is a continuous and computable operator $\lambda a.\, a^{\S}$, such that if a is a finitary projection of U, then

$$D_{a^{\S}} \cong (D_a)^{\S} .$$

The development is with the A-sharp on lists peer.

Theorem (`example_821`). The list-of-projection combinator is finitary when the projection is finitary.

Proof. By assembling the peer lemmas named in the packaging module. ■

Exercise_8_21.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.ASharp
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ASharp_more

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

theorem example_821 {a : ApproximableMap U U} (ha : FinitaryProjection a) :
    FinitaryProjection (aSharp a) :=
  isFinitaryProjection_aSharp ha
```

4.9.62 Example C below B

This peer gives an explicit coding that places Scott's domain C inside the binary-sequence domain B, while also recording the evident embedding of B into C.

Theorem (`ExampleCLeB.C_nonempty`). Every neighbourhood of C is nonempty.

Proof. A cone contains its generating string, and a singleton contains its unique string. ■

Definition (`ExampleCLeB.enc`). The encoding of a finite binary string duplicates each bit, sending $b_1 \dots b_n$ to $b_1 b_1 \dots b_n b_n$.

Proof. This recursive prescription uniquely defines the encoding, with the empty string fixed. ■

Definition (`ExampleCLeB.encC`). The terminated encoding of a string is its duplicated-bit encoding followed by the marker 10.

Proof. Appending a marker not produced by a duplicated pair distinguishes terminated codes from ordinary duplicated prefixes. ■

Theorem (`ExampleCLeB.enc_nil`). The encoding of the empty string is empty.

Proof. This is the empty clause of the recursive definition. ■

Theorem (`ExampleCLeB.enc_cons`). Encoding a string beginning with b begins with two copies of b, followed by the encoding of the tail.

Proof. This is the nonempty clause of the recursive definition. ■

Theorem (`ExampleCLeB.encC_cons`). The terminated encoding of $b\sigma$ is bb followed by the terminated encoding of σ.

Proof. Expand the duplicated encoding of a nonempty string and reassociate the terminal marker. ■

Theorem (`ExampleCLeB.encC_nil`). The terminated encoding of the empty string is the marker 10.

Proof. The ordinary encoding is empty, so only the marker remains. ■

Theorem (`ExampleCLeB.enc_append`). Encoding commutes with concatenation.

Proof. Induct on the first string; duplication of its leading bit is compatible with concatenation. ■

Theorem (`ExampleCLeB.enc_length`). The encoded string has twice the length of the original string.

Proof. Induction on the string adds two encoded bits for each original bit. ■

Theorem (`ExampleCLeB.encC_length`). The terminated encoding of σ has length $2|\sigma| + 2$.

Proof. The duplicated encoding contributes $2|\sigma|$ bits and the terminal marker contributes two more. ■

Theorem (`ExampleCLeB.encC_prefix_encC_iff`). One terminated code is a prefix of another exactly when the original strings are equal.

Proof. Compare duplicated bits from the left. A proper extension cannot pass the first terminal marker, whose unequal pair cannot be the beginning of another duplicated block. ■

Theorem (`ExampleCLeB.encC_injective`). Terminated encoding is injective.

Proof. Equality gives a prefix relation between the two terminated codes, and the prefix characterization forces equality of the source strings. ■

Theorem (`ExampleCLeB.enc_prefix_of_prefix`). Prefixes are preserved by duplicated-bit encoding.

Proof. Write the longer string as the shorter string followed by a suffix and use compatibility with concatenation. ■

Theorem (`ExampleCLeB.enc_prefix_encC_iff`). The ordinary encoding of ρ is a prefix of the terminated encoding of σ exactly when ρ is a prefix of σ.

Proof. Successive duplicated pairs recover the corresponding source bits, while the terminal marker prevents any further source bit from being read. ■

Theorem (`ExampleCLeB.enc_prefix_iff`). The ordinary encoding of σ is a prefix of the ordinary encoding of τ exactly when σ is a prefix of τ.

Proof. Preservation is immediate from concatenation; reflection follows by comparing the duplicated leading pairs recursively. ■

Theorem (`ExampleCLeB.enc_injective`). Duplicated-bit encoding is injective.

Proof. Equality of encodings gives prefix relations in both directions, hence prefix relations between the original strings in both directions and therefore equality. ■

Definition (`ExampleCLeB.memD'`). A neighbourhood of the auxiliary system D' is either the cone generated by an ordinary duplicated code or the cone generated by a terminated code.

Proof. These two families encode respectively the cone and singleton neighbourhoods of C. ■

Theorem (`ExampleCLeB.D'_nestedOrDisjoint`). Any two proposed neighbourhoods of D' are nested or disjoint.

Proof. All proposed neighbourhoods are cones, and two cones of finite strings are nested when their generators are prefix-comparable and disjoint otherwise. ■

Definition (`ExampleCLeB.D'`). The auxiliary neighbourhood system D' has all finite binary strings as master neighbourhood and has exactly the ordinary-code and terminated-code cones as neighbourhoods.

Proof. The master cone belongs to the family, every neighbourhood lies in the master set, and the nested-or-disjoint property supplies closure under admissible intersections. ■

Theorem (`ExampleCLeB.D'_mem`). A set is a neighbourhood of D' exactly when it is an ordinary-code cone or a terminated-code cone.

Proof. This is the neighbourhood predicate used in the construction of D'. ■

Theorem (`ExampleCLeB.D'_master`). The master neighbourhood of D' is the set of all finite binary strings.

Proof. This is the chosen master set in the construction of D'. ■

Theorem (`ExampleCLeB.D'_subsystem_B`). The auxiliary system D' is a subsystem of the binary-sequence domain B.

Proof. Its master neighbourhood agrees with that of B, every coded neighbourhood is a cone of B, and an inhabited intersection of two coded cones is the smaller cone by nestedness. ■

Theorem (`ExampleCLeB.cone_encC_subset_cone_enc_iff`). The cone generated by the terminated code of σ is contained in the cone generated by the ordinary code of τ exactly when τ is a prefix of σ.

Proof. Inclusion of cones reverses the prefix order on their generators, and ordinary-code prefixes of a terminated code reflect precisely the original prefix relation. ■

Theorem (`ExampleCLeB.cone_encC_subset_iff`). The terminated-code cone of σ is contained in that of τ exactly when $\sigma = \tau$.

Proof. Cone inclusion reverses prefix, while terminated codes are prefix-comparable only when their source strings are equal. ■

Theorem (`ExampleCLeB.cone_enc_subset_iff`). The ordinary-code cone of σ is contained in that of τ exactly when τ is a prefix of σ.

Proof. Cone inclusion reverses prefix and duplicated-bit encoding both preserves and reflects prefix. ■

Theorem (`ExampleCLeB.cone_ne_singleton`). No cone of finite binary strings is a singleton.

Proof. A cone contains both its generator and the proper extension obtained by appending a bit. ■

Theorem (`ExampleCLeB.not_encC_prefix_enc`). A terminated code is never a prefix of an ordinary duplicated code.

Proof. Comparing duplicated pairs eventually reaches the terminal marker 10, which cannot match a duplicated pair. ■

Theorem (`ExampleCLeB.enc_ne_encC`). No ordinary duplicated code equals a terminated code.

Proof. Equality would make the terminated code a prefix of the ordinary code, contradicting the marker property. ■

Theorem (`ExampleCLeB.not_cone_enc_subset_cone_encC`). An ordinary-code cone is never contained in a terminated-code cone.

Proof. Such cone inclusion would require the terminated generator to prefix the ordinary generator, which is impossible. ■

Definition (`ExampleCLeB.fromD'`). An element of D' determines an element of C: membership of an ordinary-code cone becomes membership of the corresponding cone, and membership of a terminated-code cone becomes membership of the corresponding singleton.

Proof. The cone calculations above show downward closure, master membership, intersection closure, and upward closure for the resulting family of C-neighbourhoods. ■

Definition (`ExampleCLeB.toD'`). An element of C determines an element of D': each cone neighbourhood is sent to its ordinary-code cone, and each singleton neighbourhood to its terminated-code cone.

Proof. The prefix and cone calculations show that the encoded family contains the master neighbourhood and is closed under restriction, inhabited intersections, and enlargement. ■

Theorem (`ExampleCLeB.toD'_mem_enc`). For an element x of C, its image belongs to the ordinary-code cone of κ exactly when x belongs to the cone of κ.

Proof. Ordinary encodings are injective and cannot equal terminated encodings, so only the corresponding cone clause can witness membership. ■

Theorem (`ExampleCLeB.toD'_mem_encC`). For an element x of C, its image belongs to the terminated-code cone of κ exactly when x belongs to the singleton $\{\kappa\}$.

Proof. Terminated encodings are injective and disjoint from ordinary encodings, so only the corresponding singleton clause can witness membership. ■

Theorem (`ExampleCLeB.fromD'_mem_cone`). For an element y of D', its decoded element belongs to the cone of κ exactly when y belongs to the ordinary-code cone of κ.

Proof. Equality of cones identifies their generators, and no cone can equal a singleton. ■

Theorem (`ExampleCLeB.fromD'_mem_singleton`). For an element y of D', its decoded element belongs to the singleton $\{\kappa\}$ exactly when y belongs to the terminated-code cone of κ.

Proof. A cone cannot equal a singleton, while equality of singletons identifies their unique strings. ■

Theorem (`ExampleCLeB.fromD'_toD'`). Decoding after encoding an element of C returns the original element.

Proof. The two elements have identical cone and singleton neighbourhoods by the two encoding membership characterizations, and these exhaust the neighbourhoods of C. ■

Theorem (`ExampleCLeB.toD'_fromD'`). Encoding after decoding an element of D' returns the original element.

Proof. The two elements have identical ordinary-code and terminated-code cone neighbourhoods by the two decoding membership characterizations, and these exhaust the neighbourhoods of D'. ■

Definition (`ExampleCLeB.cdEquiv`). The encoding and decoding maps form an order isomorphism between the elements of C and the elements of D'.

Proof. They are inverse by the preceding two theorems, and inclusion of neighbourhood families is preserved and reflected by the two kinds of coded neighbourhood. ■

Theorem (`ExampleCLeB.B_subsystem_C`). The binary-sequence domain B is a subsystem of C.

Proof. Both have the same master neighbourhood, every cone neighbourhood of B is a neighbourhood of C, and any inhabited intersection of two cones is the smaller cone. ■

Theorem (`ExampleCLeB.B_trianglelefteq_C`). The domain B is isomorphic to a subsystem of C.

Proof. Every subsystem is canonically isomorphic to a subsystem of its ambient domain. ■

Theorem (`ExampleCLeB.C_isomorphic_D'`). The domain C is isomorphic to the auxiliary system D'.

Proof. The order isomorphism between their element domains induces the required domain isomorphism. ■

Theorem (`ExampleCLeB.C_trianglelefteq_B`). The domain C is isomorphic to a subsystem of B.

Proof. Use D' as the subsystem of B and the preceding isomorphism between C and D'. ■

`ExampleC_le_B.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_12
public import DomainTheory.NeighbourhoodSystems.DomainEquations.projPairSubdomain
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Example_6_2
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot_le
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NestedOrDisjoint
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
import Mathlib.Tactic.Ring

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap BitDom SeqDom SeqEqB

namespace ExampleCLeB

theorem C_nonempty : ∀ X, C.mem X → X.Nonempty := by
  rintro X (⟨σ, rfl⟩ | ⟨σ, rfl⟩)
  · exact ⟨σ, List.prefix_rfl⟩
  · exact ⟨σ, rfl⟩

def enc : Str → Str
  | [] => []
  | b :: σ => b :: b :: enc σ

def encC (σ : Str) : Str := enc σ ++ [true, false]

@[simp] theorem enc_nil : enc ([] : Str) = [] := rfl

@[simp] theorem enc_cons (b : Bool) (σ : Str) : enc (b :: σ) = b :: b :: enc σ := rfl

theorem encC_cons (b : Bool) (σ : Str) : encC (b :: σ) = b :: b :: encC σ := by
  simp [encC]

theorem encC_nil : encC ([] : Str) = [true, false] := rfl

theorem enc_append (σ τ : Str) : enc (σ ++ τ) = enc σ ++ enc τ := by
  induction σ with
  | nil => simp
  | cons b σ' ih => simp [ih]

theorem enc_length (σ : Str) : (enc σ).length = 2 * σ.length := by
  induction σ with
  | nil => simp
  | cons b σ' ih => simp [ih]; ring

theorem encC_length (σ : Str) : (encC σ).length = 2 * σ.length + 2 := by
  simp [encC, enc_length]

theorem encC_prefix_encC_iff {σ τ : Str} : encC σ <+: encC τ ↔ σ = τ := by
```

```
  constructor
  · induction σ generalizing τ with
    | nil =>
      intro h
      cases τ with
      | nil => rfl
      | cons c τ' =>
        exfalso
        rw [encC_nil, encC_cons] at h
        obtain ⟨rfl, h2⟩ := List.cons_prefix_cons.mp h
        obtain ⟨h3, -⟩ := List.cons_prefix_cons.mp h2
        exact absurd h3 (by decide)
    | cons b σ' ih =>
      intro h
      cases τ with
      | nil =>
        exfalso
        rw [encC_cons, encC_nil] at h
        have hlen := h.length_le
        simp only [List.length_cons, List.length_nil, encC_length] at hlen
        omega
      | cons c τ' =>
        rw [encC_cons, encC_cons] at h
        obtain ⟨rfl, h2⟩ := List.cons_prefix_cons.mp h
        obtain ⟨-, h3⟩ := List.cons_prefix_cons.mp h2
        rw [ih h3]
  · rintro rfl; exact List.prefix_rfl

theorem encC_injective {σ τ : Str} (h : encC σ = encC τ) : σ = τ :=
  encC_prefix_encC_iff.mp (h ▶ List.prefix_rfl)

theorem enc_prefix_of_prefix {σ τ : Str} (h : σ <+: τ) : enc σ <+: enc τ := by
  obtain ⟨t, rfl⟩ := h
  rw [enc_append]
  exact ⟨enc t, rfl⟩

theorem enc_prefix_encC_iff {? σ : Str} : enc ? <+: encC σ ↔ ? <+: σ := by
  constructor
  · induction ? generalizing σ with
    | nil => intro _; exact List.nil_prefix
    | cons b ?' ih =>
      intro h
      cases σ with
      | nil =>
        exfalso
        rw [enc_cons, encC_nil] at h
        obtain ⟨rfl, h2⟩ := List.cons_prefix_cons.mp h
        obtain ⟨h3, -⟩ := List.cons_prefix_cons.mp h2
        exact absurd h3 (by decide)
      | cons c σ' =>
        rw [enc_cons, encC_cons] at h
        obtain ⟨rfl, h2⟩ := List.cons_prefix_cons.mp h
        obtain ⟨-, h3⟩ := List.cons_prefix_cons.mp h2
        rw [List.cons_prefix_cons]
        exact ⟨rfl, ih h3⟩
  · intro h
```

```
    calc enc ? <+: enc σ := enc_prefix_of_prefix h
      _ <+: encC σ := ⟨[true, false], rfl⟩

theorem enc_prefix_iff {σ τ : Str} : enc σ <+: enc τ ↔ σ <+: τ := by
  refine ⟨?_, enc_prefix_of_prefix⟩
  induction σ generalizing τ with
  | nil => intro _; exact List.nil_prefix
  | cons b σ' ih =>
    cases τ with
    | nil =>
      intro h
      rw [enc_cons, enc_nil] at h
      obtain ⟨t, ht⟩ := h
      simp at ht
    | cons c τ' =>
      intro h
      rw [enc_cons, enc_cons, List.cons_prefix_cons, List.cons_prefix_cons] at h
      obtain ⟨rfl, -, h3⟩ := h
      rw [List.cons_prefix_cons]
      exact ⟨rfl, ih h3⟩

theorem enc_injective {σ τ : Str} (h : enc σ = enc τ) : σ = τ := by
  have h1 : σ <+: τ := enc_prefix_iff.mp ⟨[], by simpa using h⟩
  have h2 : τ <+: σ := enc_prefix_iff.mp ⟨[], by simpa using h.symm⟩
  exact h1.eq_of_length (h1.length_le.antisymm h2.length_le)

def memD' (X : Set Str) : Prop := (∃ σ, X = cone (enc σ)) ∨ (∃ σ, X = cone (encC σ))

theorem D'_nestedOrDisjoint : NestedOrDisjoint memD' := by
  rintro X Y (⟨σ, rfl⟩ | ⟨σ, rfl⟩) (⟨τ, rfl⟩ | ⟨τ, rfl⟩)
  · exact cone.trichotomy (enc σ) (enc τ)
  · exact cone.trichotomy (enc σ) (encC τ)
  · exact cone.trichotomy (encC σ) (enc τ)
  · exact cone.trichotomy (encC σ) (encC τ)

def D' : NeighborhoodSystem Str :=
  NeighborhoodSystem.ofNestedOrDisjoint memD' Set.univ ⟨[], Set.mem_univ _⟩
    (Or.inl ⟨[], by rw [enc_nil, cone.nil]⟩)
    D'_nestedOrDisjoint (by rintro X (⟨σ, rfl⟩ | ⟨σ, rfl⟩) <;> exact Set.subset_univ _)

@[simp] theorem D'_mem {X : Set Str} : D'.mem X ↔ memD' X := Iff.rfl

@[simp] theorem D'_master : D'.master = (Set.univ : Set Str) := rfl

theorem D'_subsystem_B : D' ? B where
  master_eq := rfl
  sub := by rintro X (⟨σ, rfl⟩ | ⟨σ, rfl⟩) <;> exact ⟨_, rfl⟩
  inter_closed := by
    rintro X Y hX hY hBXY
    rcases D'_nestedOrDisjoint hX hY with h | h | h
    · rw [Set.inter_eq_left.mpr h]; exact hX
    · rw [Set.inter_eq_right.mpr h]; exact hY
    · rw [h] at hBXY; exact absurd (B_nonempty ∅ hBXY) (by simp)

theorem cone_encC_subset_cone_enc_iff {σ τ : Str} : cone (encC σ) ⊆ cone (enc τ) ↔ τ
    <+: σ := by
```

```
  rw [cone.subset_cone, enc_prefix_encC_iff]

theorem cone_encC_subset_iff {σ τ : Str} : cone (encC σ) ⊆ cone (encC τ) ↔ σ = τ :=
    by
  rw [cone.subset_cone, encC_prefix_encC_iff, eq_comm]

theorem cone_enc_subset_iff {σ τ : Str} : cone (enc σ) ⊆ cone (enc τ) ↔ τ <+: σ := by
  rw [cone.subset_cone, enc_prefix_iff]

theorem cone_ne_singleton (σ τ : Str) : cone σ = ({τ} : Set Str) := by
  intro h
  have h1 : σ ∈ cone σ := List.prefix_rfl
  have h2 : σ ++ [true] ∈ cone σ := ⟨[true], rfl⟩
  rw [h] at h1 h2
  simp only [Set.mem_singleton_iff] at h1 h2
  exact absurd (h2.trans h1.symm) (by simp)

theorem not_encC_prefix_enc {σ τ : Str} : ¬ encC σ <+: enc τ := by
  induction σ generalizing τ with
  | nil =>
    cases τ with
    | nil => simp [encC_nil]
    | cons c τ' =>
      rw [encC_nil, enc_cons]
      intro h
      obtain ⟨rfl, h2⟩ := List.cons_prefix_cons.mp h
      obtain ⟨h3, -⟩ := List.cons_prefix_cons.mp h2
      exact absurd h3 (by decide)
  | cons b σ' ih =>
    cases τ with
    | nil =>
      rw [encC_cons, enc_nil]
      intro h
      exact absurd h.length_le (by simp)
    | cons c τ' =>
      rw [encC_cons, enc_cons]
      intro h
      obtain ⟨rfl, h2⟩ := List.cons_prefix_cons.mp h
      obtain ⟨-, h3⟩ := List.cons_prefix_cons.mp h2
      exact ih h3

theorem enc_ne_encC (σ τ : Str) : enc σ = encC τ := by
  intro h
  exact not_encC_prefix_enc (σ := τ) (τ := σ) (h ▶ List.prefix_rfl)

theorem not_cone_enc_subset_cone_encC {σ τ : Str} : ¬ cone (enc σ) ⊆ cone (encC τ) :=
    by
  rw [cone.subset_cone]; exact not_encC_prefix_enc

def fromD' (y : D'.Element) : C.Element where
  mem X := (∃ σ, y.mem (cone (enc σ)) ∧ X = cone σ) ∨ (∃ σ, y.mem (cone (encC σ)) ∧ X
    = ({σ} : Set Str))
  sub := by
    rintro X (⟨σ, -, rfl⟩ | ⟨σ, -, rfl⟩)
    · exact memC_cone σ
    · exact memC_singleton σ
```

```
master_mem := by
  refine Or.inl ⟨[], ?_, cone.nil.symm⟩
  rw [enc_nil, cone.nil, ← D'_master]
  exact y.master_mem
inter_mem := by
  rintro X X' (⟨σ, hσ, rfl⟩ | ⟨σ, hσ, rfl⟩) (⟨τ, hτ, rfl⟩ | ⟨τ, hτ, rfl⟩)
  ·
    have hy := y.inter_mem hσ hτ
    rcases cone.trichotomy (enc σ) (enc τ) with h | h | h
    · exact Or.inl ⟨σ, hσ, by rw [Set.inter_eq_left.mpr (cone.subset_cone.mpr
        (cone_enc_subset_iff.mp h))]⟩
    · exact Or.inl ⟨τ, hτ, by rw [Set.inter_eq_right.mpr (cone.subset_cone.mpr
        (cone_enc_subset_iff.mp h))]⟩
    · rw [h] at hy; exact absurd (B_nonempty ∅ (D'_subsystem_B.sub (y.sub hy))) (by
  simp)
  ·
    have hy := y.inter_mem hσ hτ
    rcases cone.trichotomy (enc σ) (encC τ) with h | h | h
    · exfalso
      exact absurd h not_cone_enc_subset_cone_encC
    · exact Or.inr ⟨τ, hτ, by rw [Set.inter_eq_right.mpr (singleton_subset_cone.mpr
        (cone_encC_subset_cone_enc_iff.mp h))]⟩
    · rw [h] at hy; exact absurd (B_nonempty ∅ (D'_subsystem_B.sub (y.sub hy))) (by
  simp)
  ·
    have hy := y.inter_mem hσ hτ
    rcases cone.trichotomy (encC σ) (enc τ) with h | h | h
    · exact Or.inr ⟨σ, hσ, by rw [Set.inter_eq_left.mpr (singleton_subset_cone.mpr
        (cone_encC_subset_cone_enc_iff.mp h))]⟩
    · exfalso
      exact absurd h not_cone_enc_subset_cone_encC
    · rw [h] at hy; exact absurd (B_nonempty ∅ (D'_subsystem_B.sub (y.sub hy))) (by
  simp)
  ·
    have hy := y.inter_mem hσ hτ
    by_cases hστ : σ = τ
    · subst hστ
      exact Or.inr ⟨σ, hσ, by rw [Set.inter_self]⟩
    · exfalso
      have h' : cone (encC σ) ∩ cone (encC τ) = ∅ := by
        rcases cone.trichotomy (encC σ) (encC τ) with h | h | h
        · exact absurd (cone_encC_subset_iff.mp h) hστ
        · exact absurd (cone_encC_subset_iff.mp h) (Ne.symm hστ)
        · exact h
      rw [h'] at hy
      exact absurd (B_nonempty ∅ (D'_subsystem_B.sub (y.sub hy))) (by simp)
up_mem := by
  rintro X X' (⟨σ, hσ, rfl⟩ | ⟨σ, hσ, rfl⟩) hX' hsub
  · rcases hX' with ⟨?, rfl⟩ | ⟨?, rfl⟩
    · refine Or.inl ⟨?, ?_, rfl⟩
      have h?σ : ? <+: σ := cone.subset_cone.mp hsub
      exact y.up_mem hσ (Or.inl ⟨?, rfl⟩) (cone_enc_subset_iff.mpr h?σ)
    · exfalso
      have h1 : σ ∈ cone σ := List.prefix_rfl
      have h2 : σ ++ [true] ∈ cone σ := ⟨[true], rfl⟩
      have hσ?1 : σ = ? := hsub h1
```

```
        have hσ?2 : σ ++ [true] = ? := hsub h2
        exact absurd (hσ?2.trans hσ?1.symm) (by simp)
    · rcases hX' with ⟨?, rfl⟩ | ⟨?, rfl⟩
      · refine Or.inl ⟨?, ?_, rfl⟩
        have h?σ : ? <+: σ := singleton_subset_cone.mp hsub
        exact y.up_mem hσ (Or.inl ⟨?, rfl⟩) (cone_encC_subset_cone_enc_iff.mpr h?σ)
      · refine Or.inr ⟨?, ?_, rfl⟩
        have hσ? : σ = ? := hsub rfl
        exact y.up_mem hσ (Or.inr ⟨?, rfl⟩) (cone_encC_subset_iff.mpr hσ?)

def toD' (x : C.Element) : D'.Element where
  mem W := (∃ σ, x.mem (cone σ) ∧ W = cone (enc σ)) ∨ (∃ σ, x.mem ({σ} : Set Str) ∧ W
    = cone (encC σ))
  sub := by
    rintro W (⟨σ, -, rfl⟩ | ⟨σ, -, rfl⟩)
    · exact Or.inl ⟨σ, rfl⟩
    · exact Or.inr ⟨σ, rfl⟩
  master_mem := by
    rw [D'_master]
    refine Or.inl ⟨[], ?_, ?_⟩
    · rw [cone.nil]; exact x.master_mem
    · rw [enc_nil, cone.nil]
  inter_mem := by
    rintro W W' (⟨σ, hσ, rfl⟩ | ⟨σ, hσ, rfl⟩) (⟨τ, hτ, rfl⟩ | ⟨τ, hτ, rfl⟩)
    ·
      have hx := x.inter_mem hσ hτ
      rcases cone.trichotomy σ τ with h | h | h
      · exact Or.inl ⟨σ, hσ, by rw [Set.inter_eq_left.mpr
          (cone_enc_subset_iff.mpr (cone.subset_cone.mp h))]⟩
      · exact Or.inl ⟨τ, hτ, by rw [Set.inter_eq_right.mpr
          (cone_enc_subset_iff.mpr (cone.subset_cone.mp h))]⟩
      · rw [h] at hx; exact absurd (C_nonempty ∅ (x.sub hx)) (by simp)
    ·
      have hx := x.inter_mem hσ hτ
      rcases singleton_cone_nd σ τ with h | h | h
      · exact Or.inr ⟨τ, hτ, by rw [Set.inter_eq_right.mpr
          (cone_encC_subset_cone_enc_iff.mpr (singleton_subset_cone.mp h))]⟩
      · exfalso
        have h1 : σ ∈ cone σ := List.prefix_rfl
        have h2 : σ ++ [true] ∈ cone σ := ⟨[true], rfl⟩
        have hστ1 : σ = τ := h h1
        have hστ2 : σ ++ [true] = τ := h h2
        exact absurd (hστ2.trans hστ1.symm) (by simp)
      · exfalso
        have h' : cone σ ∩ {τ} = ∅ := by rw [Set.inter_comm]; exact h
        rw [h'] at hx
        exact absurd (C_nonempty ∅ (x.sub hx)) (by simp)
    ·
      have hx := x.inter_mem hσ hτ
      rcases singleton_cone_nd τ σ with h | h | h
      · exact Or.inr ⟨σ, hσ, by rw [Set.inter_eq_left.mpr
          (cone_encC_subset_cone_enc_iff.mpr (singleton_subset_cone.mp h))]⟩
      · exfalso
        have h1 : τ ∈ cone τ := List.prefix_rfl
        have h2 : τ ++ [true] ∈ cone τ := ⟨[true], rfl⟩
        have hτσ1 : τ = σ := h h1
```

```
        have hτσ2 : τ ++ [true] = σ := h h2
        exact absurd (hτσ2.trans hτσ1.symm) (by simp)
      · exfalso
        rw [h] at hx
        exact absurd (C_nonempty ∅ (x.sub hx)) (by simp)
    .
      have hx := x.inter_mem hσ hτ
      by_cases hστ : σ = τ
      · subst hστ
        exact Or.inr ⟨σ, hσ, by rw [Set.inter_self]⟩
      · exfalso
        have h' : ({σ} : Set Str) ∩ {τ} = ∅ := by
          ext w; simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.
  mem_empty_iff_false,
            iff_false, not_and]
          rintro rfl; exact hστ
        rw [h'] at hx
        exact absurd (C_nonempty ∅ (x.sub hx)) (by simp)
  up_mem := by
    rintro W W' (⟨σ, hσ, rfl⟩ | ⟨σ, hσ, rfl⟩) hW' hsub
    · rcases hW' with ⟨?, rfl⟩ | ⟨?, rfl⟩
      · refine Or.inl ⟨?, ?_, rfl⟩
        have h?σ : ? <+: σ := cone_enc_subset_iff.mp hsub
        exact x.up_mem hσ (memC_cone ?) (cone.subset_cone.mpr h?σ)
      · exact absurd hsub not_cone_enc_subset_cone_encC
    · rcases hW' with ⟨?, rfl⟩ | ⟨?, rfl⟩
      · refine Or.inl ⟨?, ?_, rfl⟩
        have h?σ : ? <+: σ := cone_encC_subset_cone_enc_iff.mp hsub
        exact x.up_mem hσ (memC_cone ?) (singleton_subset_cone.mpr h?σ)
      · refine Or.inr ⟨?, ?_, rfl⟩
        have hστ : σ = ? := cone_encC_subset_iff.mp hsub
        exact hστ ▶ hσ

@[simp] theorem toD'_mem_enc {x : C.Element} (? : Str) :
    (toD' x).mem (cone (enc ?)) ↔ x.mem (cone ?) := by
  constructor
  · rintro (⟨σ, hσ, heq⟩ | ⟨σ, hσ, heq⟩)
    · have h?σ : ? = σ := enc_injective (cone.injective heq)
      rw [h?σ]; exact hσ
    · exact absurd (cone.injective heq) (enc_ne_encC ? σ)
  · intro hx; exact Or.inl ⟨?, hx, rfl⟩

@[simp] theorem toD'_mem_encC {x : C.Element} (? : Str) :
    (toD' x).mem (cone (encC ?)) ↔ x.mem ({?} : Set Str) := by
  constructor
  · rintro (⟨σ, hσ, heq⟩ | ⟨σ, hσ, heq⟩)
    · exact absurd (cone.injective heq).symm (enc_ne_encC σ ?)
    · have h?σ : ? = σ := encC_injective (cone.injective heq)
      rw [h?σ]; exact hσ
  · intro hx; exact Or.inr ⟨?, hx, rfl⟩

@[simp] theorem fromD'_mem_cone {y : D'.Element} (? : Str) :
    (fromD' y).mem (cone ?) ↔ y.mem (cone (enc ?)) := by
  constructor
  · rintro (⟨σ, hσ, heq⟩ | ⟨σ, hσ, heq⟩)
    · have h?σ : ? = σ := cone.injective heq
```

```
      rw [h?σ]; exact hσ
    · exact absurd heq (cone_ne_singleton ? σ)
  · intro hy; exact Or.inl ⟨?, hy, rfl⟩

@[simp] theorem fromD'_mem_singleton {y : D'.Element} (? : Str) :
    (fromD' y).mem ({?} : Set Str) ↔ y.mem (cone (encC ?)) := by
  constructor
  · rintro (⟨σ, hσ, heq⟩ | ⟨σ, hσ, heq⟩)
    · exact absurd heq.symm (cone_ne_singleton σ ?)
    · have h?σ : ? = σ := by
        have h1 : ? ∈ ({σ} : Set Str) := heq ▶ rfl
        simpa using h1
      rw [h?σ]; exact hσ
  · intro hy; exact Or.inr ⟨?, hy, rfl⟩

theorem fromD'_toD' (x : C.Element) : fromD' (toD' x) = x := by
  apply NeighborhoodSystem.Element.ext
  intro X


  constructor
  · rintro (⟨σ, hσ, rfl⟩ | ⟨σ, hσ, rfl⟩)
    · exact (toD'_mem_enc σ).mp hσ
    · exact (toD'_mem_encC σ).mp hσ
  · intro hX
    rcases x.sub hX with ⟨σ, rfl⟩ | ⟨σ, rfl⟩
    · exact Or.inl ⟨σ, (toD'_mem_enc σ).mpr hX, rfl⟩
    · exact Or.inr ⟨σ, (toD'_mem_encC σ).mpr hX, rfl⟩

theorem toD'_fromD' (y : D'.Element) : toD' (fromD' y) = y := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  · rintro (⟨?, h?, rfl⟩ | ⟨?, h?, rfl⟩)
    · exact (fromD'_mem_cone ?).mp h?
    · exact (fromD'_mem_singleton ?).mp h?
  · intro hW
    rcases y.sub hW with ⟨?, rfl⟩ | ⟨?, rfl⟩
    · exact Or.inl ⟨?, (fromD'_mem_cone ?).mpr hW, rfl⟩
    · exact Or.inr ⟨?, (fromD'_mem_singleton ?).mpr hW, rfl⟩

def cdEquiv : C.Element ≃o D'.Element where
  toFun := toD'
  invFun := fromD'
  left_inv := fromD'_toD'
  right_inv := toD'_fromD'
  map_rel_iff' := by
    intro x x'
    constructor
    · intro h X hX
      rcases x.sub hX with ⟨σ, rfl⟩ | ⟨σ, rfl⟩
      · exact (toD'_mem_enc σ).mp (h _ ((toD'_mem_enc σ).mpr hX))
      · exact (toD'_mem_encC σ).mp (h _ ((toD'_mem_encC σ).mpr hX))
    · intro h W hW
      rcases hW with ⟨σ, hσ, rfl⟩ | ⟨σ, hσ, rfl⟩
      · exact Or.inl ⟨σ, h _ hσ, rfl⟩
      · exact Or.inr ⟨σ, h _ hσ, rfl⟩
```

```
theorem B_subsystem_C : B ? C where
  master_eq := rfl
  sub := by rintro X ⟨σ, rfl⟩; exact memC_cone σ
  inter_closed := by
    rintro X Y ⟨σ, rfl⟩ ⟨τ, rfl⟩ hCXY
    rcases cone.trichotomy σ τ with h | h | h
    · rw [Set.inter_eq_left.mpr h]; exact ⟨σ, rfl⟩
    · rw [Set.inter_eq_right.mpr h]; exact ⟨τ, rfl⟩
    · rw [h] at hCXY; exact absurd (C_nonempty ∅ hCXY) (by simp)

theorem B_trianglelefteq_C : B ? C :=
  Subsystem.trianglelefteq B_subsystem_C

theorem C_isomorphic_D' : C ≅? D' := ⟨cdEquiv⟩

theorem C_trianglelefteq_B : C ? B := ⟨D', D'_subsystem_B, C_isomorphic_D'⟩

end ExampleCLeB

end DomainTheory.NeighbourhoodSystems
```

4.9.63 Exercise 8.22 — comparing B and C

Which of the two relations hold:

$$B \trianglelefteq C \quad \text{or} \quad C \trianglelefteq B\,?$$

Or do they both hold? In general if we use domain equations

$$D = T(D) + S(D)\,, \quad \text{and}$$

$$E = T(E)\,,$$

will $E \trianglelefteq D$ hold? What projections do you see in the examples in 6.2?

The development is with the Example C below B peer.

Theorem (`example_822`). Scott's sequence domain C is a subdomain of the bit domain B.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_22.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ExampleC_le_B

@[expose] public section

open DomainTheory.NeighbourhoodSystems

theorem example_822 : SeqDom.C ? BitDom.B :=
  ExampleCLeB.C_trianglelefteq_B
```

4.9.64 Fix operator

This peer constructs the least fixed-point operation on endomaps, proves that it preserves finitary projections under a natural closure hypothesis, and identifies the resulting fixed domains.

Theorem (`FixOp.eq_bot_of_fixed_constMap_bot`). The only element fixed by the constant map with value bottom is bottom itself.

Proof. The image of every element under the constant map is bottom, so the fixed-point equation immediately identifies the element with bottom. ■

Theorem (`FixOp.uniqueFixedConstMapBot`). The fixed-point type of the constant-bottom map has exactly one element.

Proof. Bottom is fixed, and every fixed element is bottom by the preceding fact. ■

Definition (`FixOp.finitaryWitnessEquivBot`). The fixed-point poset of the constant-bottom map is order-isomorphic to the one-element neighbourhood system.

Proof. Both posets have a unique element, so the unique maps in both directions are mutually inverse and preserve the order. ■

Definition (`FixOp.fixOp`). For an endomap t of the function-space domain $[E \to E]$, its fixed-point operator is the approximable endomap of E represented by the least fixed element of t.

Proof. Convert the least fixed element of t from a function-space element to its corresponding approximable map. ■

Definition (`FixOp.tOp`). An operator t on endomaps acts on an endomap a by applying t to the function-space element represented by a, then converting the result back to an approximable map.

Proof. This is the transport of the element action of t across the correspondence between function-space elements and approximable maps. ■

Definition (`FixOp.Fix`). For an endomap $a : E \to E$, $\mathrm{Fix}(a)$ is the ordered type of elements y satisfying $a(y) = y$.

Proof. Take the subtype of the element poset cut out by the fixed-point equation, with its inherited order. ■

Theorem (`FixOp.constMap_bot_le_idMap`). The constant-bottom endomap lies below the identity endomap.

Proof. Every value of the constant map is bottom, and bottom lies below every element. ■

Theorem (`FixOp.isRetraction_constMap_bot`). The constant-bottom endomap is idempotent.

Proof. Applying the map once or twice always returns bottom. ■

Theorem (`FixOp.isProjection_constMap_bot`). The constant-bottom endomap is a projection.

Proof. It is idempotent and lies below the identity. ■

Theorem (`FixOp.isFinitaryProjection_constMap_bot`). The constant-bottom endomap is a finitary projection.

Proof. Its fixed-point poset is order-isomorphic to the one-element neighbourhood system, which supplies a finite witness for the projection. ■

Theorem (`FixOp.iterElem_succ'`). The $(n+1)$-st approximation in the iteration of t is obtained by applying t to the n-th approximation.

Proof. Successor iteration is composition with t, and element maps carry composition to successive application. ■

Theorem (`FixOp.iterElem_zero'`). The zeroth approximation in the iteration of t is the bottom element of the function-space domain.

Proof. Zeroth iteration is the identity, evaluated at bottom. ■

Theorem (`FixOp.rel_master_of_mem`). If X is a neighbourhood of E, then every approximable endomap relates X to the master neighbourhood.

Proof. Start with the map's master relation and use monotonicity, since X is contained in the master neighbourhood. ■

Theorem (`FixOp.tOp_fixOp`). The endomap obtained as the least fixed point of t is fixed by the induced action of t.

Proof. The least fixed element of t is fixed by t, and the element–map correspondence preserves this equation. ■

Theorem (`FixOp.tOp_toApproxMap`). Acting on the approximable map represented by a function-space element φ yields the approximable map represented by $t(\varphi)$.

Proof. The two conversions between function-space elements and approximable maps are inverse. ■

Theorem (`FixOp.toApproxMap_bot_eq_constMap`). The approximable map represented by the bottom function-space element is the constant-bottom map.

Proof. A relation belongs to the bottom function exactly when its input is a neighbourhood and its output is bottom; this is precisely the relation of the constant-bottom map. ■

Theorem (`FixOp.isFinitaryProjection_fixOp`). Suppose the action induced by t sends every finitary projection to a finitary projection. Then the least fixed-point endomap of t is itself a finitary projection.

Proof. Beginning with the constant-bottom finitary projection, induction shows that every finite iterate is finitary. The least fixed element is the directed supremum of these iterates, and the finitary core operation fixes each iterate and commutes with this directed supremum; hence it fixes the limit, which characterizes a finitary projection. ■

Theorem (`FixOp.fixedDomain_fixOp_iso_T`). Suppose the induced action of t preserves finitary projections, and the fixed domain of its value at each finitary projection a is order-isomorphic to a specified domain $T(a)$. Then the fixed domain of the least fixed-point endomap is order-isomorphic to T evaluated at that endomap.

Proof. The least fixed-point endomap is finitary and is fixed by the induced action, so specialize the assumed family of order isomorphisms to it. ■

Theorem (`FixOp.fixedNbhd_fixOp_subsystem`). Under the same preservation hypothesis, if a is a finitary projection and the induced action sends a below itself, then the fixed neighbourhood system of the least fixed-point endomap is a subsystem of the fixed neighbourhood system of a.

Proof. The pre-fixed-point inequality implies, by leastness, that the least fixed element of t lies below the element represented by a. Transporting this inequality to approximable maps and using the order characterization of fixed neighbourhood subsystems for finitary projections gives the result. ■

FixOp.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.fixMap

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    FixedNbhdFinitaryOrder
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Examples_8_4

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Sub8_6

namespace FixOp

variable {α : Type} {E : NeighborhoodSystem α}

theorem eq_bot_of_fixed_constMap_bot {y : E.Element}
    (hy : (constMap E E.bot).toElementMap y = y) : y = E.bot := by
  rw [toElementMap_constMap] at hy
  exact hy.symm

instance uniqueFixedConstMapBot :
    Unique {y : E.Element // (constMap E E.bot).toElementMap y = y} where
  default := ⟨E.bot, toElementMap_constMap E.bot E.bot⟩
  uniq := fun ⟨_y, hy⟩ => Subtype.ext (eq_bot_of_fixed_constMap_bot hy)

def finitaryWitnessEquivBot (E : NeighborhoodSystem α) :
    {y : E.Element // (constMap E E.bot).toElementMap y = y} ≃o unitSys.Element where
  toFun _ := default
  invFun _ := default
  left_inv a := (Unique.eq_default a).symm
  right_inv b := (Unique.eq_default b).symm
  map_rel_iff' := by
    intro a b
    have hab : a = b := (Unique.eq_default a).trans (Unique.eq_default b).symm
    exact iff_of_true (le_refl default) (le_of_eq hab)

def fixOp (t : ApproximableMap (funSpace E E) (funSpace E E)) : ApproximableMap E E :=
  toApproxMap t.fixElement

def tOp (t : ApproximableMap (funSpace E E) (funSpace E E)) (a : ApproximableMap E E)
    :
    ApproximableMap E E :=
  toApproxMap (t.toElementMap (toFilter a))

abbrev Fix (a : ApproximableMap E E) := {y : E.Element // a.toElementMap y = y}

theorem constMap_bot_le_idMap (E : NeighborhoodSystem α) :
    constMap E E.bot ≤ idMap E := by
  rintro X Z hr
  rw [constMap_rel, mem_bot] at hr
  obtain ⟨hX, rfl⟩ := hr
  exact ⟨hX, E.master_mem, E.sub_master hX⟩

theorem isRetraction_constMap_bot (E : NeighborhoodSystem α) :
    Retraction (constMap E E.bot) := by
  constructor
  show (constMap E E.bot).comp (constMap E E.bot) = constMap E E.bot
  apply ApproximableMap.ext
  intro X Z
  simp only [comp_rel, constMap_rel, mem_bot]
```

```
  constructor
  · rintro ⟨Y, ⟨hX, rfl⟩, -, hZ⟩
    exact ⟨hX, hZ⟩
  · rintro ⟨hX, hZ⟩
    exact ⟨E.master, ⟨hX, rfl⟩, E.master_mem, hZ⟩

theorem isProjection_constMap_bot (E : NeighborhoodSystem α) :
    IsProjection (constMap E E.bot) :=
  ⟨isRetraction_constMap_bot E, constMap_bot_le_idMap E⟩

theorem isFinitaryProjection_constMap_bot (E : NeighborhoodSystem α) :
    FinitaryProjection (constMap E E.bot) :=
  ⟨⟨isProjection_constMap_bot E, Unit, unitSys, ⟨finitaryWitnessEquivBot E⟩⟩⟩

theorem iterElem_succ' (t : ApproximableMap (funSpace E E) (funSpace E E)) (n : ℕ) :
    t.iterElem (n + 1) = t.toElementMap (t.iterElem n) := by
  show (t.comp (t.iterMap n)).toElementMap (funSpace E E).bot = t.toElementMap (t.
    iterElem n)
  rw [toElementMap_comp]
  rfl

theorem iterElem_zero' (t : ApproximableMap (funSpace E E) (funSpace E E)) :
    t.iterElem 0 = (funSpace E E).bot := by
  show (idMap (funSpace E E)).toElementMap (funSpace E E).bot = (funSpace E E).bot
  rw [toElementMap_idMap]

theorem rel_master_of_mem (f : ApproximableMap E E) {X : Set α} (hX : E.mem X) :
    f.rel X E.master :=
  f.mono f.master_rel (E.sub_master hX) subset_rfl hX E.master_mem

theorem tOp_fixOp (t : ApproximableMap (funSpace E E) (funSpace E E)) :
    tOp t (fixOp t) = fixOp t := by
  show toApproxMap (t.toElementMap (toFilter (toApproxMap t.fixElement))) =
    toApproxMap t.fixElement
  rw [toFilter_toApproxMap, toElementMap_fixElement]

theorem tOp_toApproxMap (t : ApproximableMap (funSpace E E) (funSpace E E))
    (φ : (funSpace E E).Element) : tOp t (toApproxMap φ) = toApproxMap (t.toElementMap
    φ) := by
  unfold tOp
  rw [toFilter_toApproxMap]

theorem toApproxMap_bot_eq_constMap (E : NeighborhoodSystem α) :
    toApproxMap (funSpace E E).bot = constMap E E.bot := by
  apply ApproximableMap.ext
  intro X Y
  simp only [toApproxMap_rel, mem_bot, constMap_rel]
  constructor
  · intro h
    have hcb : constMap E E.bot ∈ step X Y := by rw [h]; trivial
    rwa [mem_step, constMap_rel, mem_bot] at hcb
  · rintro ⟨hX, rfl⟩
    exact Set.eq_univ_of_forall fun f => rel_master_of_mem f hX

theorem isFinitaryProjection_fixOp (t : ApproximableMap (funSpace E E) (funSpace E E))
    (ht : ∀ a, FinitaryProjection a → FinitaryProjection (tOp t a)) :
```

```
    FinitaryProjection (fixOp t) := by
  have hchain : ∀ n, FinitaryProjection (toApproxMap (t.iterElem n)) := by
    intro n
    induction n with
    | zero =>
      rw [iterElem_zero', toApproxMap_bot_eq_constMap]
      exact isFinitaryProjection_constMap_bot E
    | succ k ih =>
      rw [iterElem_succ', ← tOp_toApproxMap t (t.iterElem k)]
      exact ht _ ih
  have hsub : ∀ n, subFilter (t.iterElem n) = t.iterElem n := by
    intro n
    apply toApproxMap_injective
    rw [toApproxMap_subFilter]
    exact sub_eq_self_of_isFinitaryProjection (hchain n)
  have hdir : ∀ i j : ℕ, ∃ k, t.iterElem i ≤ t.iterElem k ∧ t.iterElem j ≤ t.iterElem
    k :=
    fun i j => ⟨max i j, iterElem_mono t (le_max_left i j), iterElem_mono t (
    le_max_right i j)⟩
  have hdirSub : ∀ i j : ℕ, ∃ k, subFilter (t.iterElem i) ≤ subFilter (t.iterElem k) ∧
      subFilter (t.iterElem j) ≤ subFilter (t.iterElem k) := by
    intro i j
    obtain ⟨k, hik, hjk⟩ := hdir i j
    refine ⟨k, ?_, ?_⟩
    · rw [hsub i, hsub k]; exact hik
    · rw [hsub j, hsub k]; exact hjk
  have hEq : t.fixElement = iSupDirected t.iterElem hdir := by
    apply NeighborhoodSystem.Element.ext
    intro Z
    rw [mem_fixElement, mem_iSupDirected]
    simp_rw [mem_iterElem]
  have hkey : subFilter t.fixElement = t.fixElement := by
    rw [hEq, subFilter_iSupDirected t.iterElem hdir hdirSub]
    apply NeighborhoodSystem.Element.ext
    intro Z
    rw [mem_iSupDirected, mem_iSupDirected]
    simp_rw [hsub]
  show FinitaryProjection (toApproxMap t.fixElement)
  apply isFinitaryProjection_of_sub_eq_self
  rw [← toApproxMap_subFilter, hkey]

theorem fixedDomain_fixOp_iso_T (t : ApproximableMap (funSpace E E) (funSpace E E))
    (ht : ∀ a, FinitaryProjection a → FinitaryProjection (tOp t a))
    (T : ApproximableMap E E → Σ β : Type, NeighborhoodSystem β)
    (hT : ∀ a, FinitaryProjection a → Nonempty (Fix (tOp t a) ≃o (T a).2.Element)) :
    Nonempty (Fix (fixOp t) ≃o (T (fixOp t)).2.Element) := by
  have h := hT (fixOp t) (isFinitaryProjection_fixOp t ht)
  rwa [tOp_fixOp] at h

theorem fixedNbhd_fixOp_subsystem (t : ApproximableMap (funSpace E E) (funSpace E E))
    (ht : ∀ a, FinitaryProjection a → FinitaryProjection (tOp t a))
    {a : ApproximableMap E E} (ha : FinitaryProjection a) (hpre : tOp t a ≤ a) :
    fixedNbhd (fixOp t) ? fixedNbhd a := by
  have hpre' : toApproxMap (t.toElementMap (toFilter a)) ≤ toApproxMap (toFilter a) :=
    by
    show tOp t a ≤ toApproxMap (toFilter a)
```

```
    rw [toApproxMap_toFilter]
    exact hpre
  have h1 : t.toElementMap (toFilter a) ≤ toFilter a := by
    have hiff := (funSpaceEquiv E E).le_iff_le
      (x := t.toElementMap (toFilter a)) (y := toFilter a)
    rw [funSpaceEquiv_apply, funSpaceEquiv_apply] at hiff
    exact hiff.mp hpre'
  have h2 : t.fixElement ≤ toFilter a := fixElement_le_of_toElementMap_le t h1
  have hle : fixOp t ≤ a := by
    show toApproxMap t.fixElement ≤ a
    rw [← toApproxMap_toFilter a]
    exact toApproxMap_monotone h2
  exact (isFinitaryProjection_le_iff_fixedNbhd_subsystem
    (isFinitaryProjection_fixOp t ht) ha).mp hle

end FixOp

end DomainTheory.NeighbourhoodSystems
```

4.9.65 Exercise 8.23 — initial algebra inside U

NeighbourhoodSystems/RetractsOfTheUniversalDomain/Exercise_8_23.lean

@import "Exercise_8_23.lean"

EXERCISE 8.23. Suppose a construct T on domains can be made into a computable operator $t : (U \to U) \to (U \to U)$ so that whenever $a : U \to U$ is a finitary projection, then so is $t(a)$ and

$$D_{t(a)} \cong T(D_a) \,.$$

Does it follow that $\|t\| = \mathrm{fix}(t)$ is such that

$$D_{\|t\|} \cong T(D_{\|t\|})$$

really is the initial solution of the domain equation with respect to projections? Since t is computable, will this solution be effectively given?

The development is with the Fix operator peer.

Theorem (`example_823`). Inside U, the initial algebra for the given operator exists as a retract.

Proof. By assembling the peer lemmas named in the packaging module. ■

Exercise_8_23.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.FixOp

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap DomainTheory
    .NeighbourhoodSystems.FixOp

theorem example_823 {α : Type} {E : NeighborhoodSystem α}
    (t : ApproximableMap (funSpace E E) (funSpace E E))
    (ht : ∀ a, FinitaryProjection a → FinitaryProjection (tOp t a)) :
    FinitaryProjection (fixOp t) :=
```

```
isFinitaryProjection_fixOp t ht
```

4.9.66 Coupled fixed points

This peer extends least-fixed-point recursion to two mutually dependent endomaps, proves finitarity and leastness of the coupled solution, and records two countable domains that therefore embed in the universal domain.

Definition (`CoupledFixedPoints.PairSpace`). The parameter space for coupled recursion is the product of two copies of the endomap function space $[E \to E]$.

Proof. Form the product neighbourhood system of the two function spaces. ■

Definition (`CoupledFixedPoints.aOp`). Given two operators s and t on pairs of endomaps, a_n is the first component of the n-th approximation to the least fixed point of their paired operator.

Proof. Iterate the paired operator from bottom, take the first coordinate, and read it as an approximable endomap of E. ■

Definition (`CoupledFixedPoints.aStar`). The first coupled fixed-point endomap a^* is the first component of the least fixed element of the paired operator determined by s and t.

Proof. Take the first coordinate of the paired least fixed element and convert it to an endomap of E. ■

Definition (`CoupledFixedPoints.bOp`). Given s and t, b_n is the second component of the n-th approximation to the least fixed point of their paired operator.

Proof. Iterate the paired operator from bottom, take the second coordinate, and read it as an approximable endomap of E. ■

Definition (`CoupledFixedPoints.bStar`). The second coupled fixed-point endomap b^* is the second component of the least fixed element of the paired operator determined by s and t.

Proof. Take the second coordinate of the paired least fixed element and convert it to an endomap of E. ■

Definition (`CoupledFixedPoints.binOp`). An operator f on pairs of endomaps acts on a, b by applying f to the function-space pair represented by a, b, then reading the result as an endomap of E.

Proof. Pair the function-space elements corresponding to a and b, apply f, and use the element–map correspondence. ■

Theorem (`CoupledFixedPoints.fst_mono`). First-coordinate projection on the pair space is monotone.

Proof. Inclusion of pair elements preserves every neighbourhood witnessed in the first coordinate. ■

Theorem (`CoupledFixedPoints.iterElem_pair_eq_pair`). Every iterate of the paired operator is the pair of the function-space elements represented by a_n and b_n.

Proof. An element of a product is recovered from its first and second coordinates, and the conversions between function-space elements and approximable maps are inverse. ■

Theorem (`CoupledFixedPoints.iterElem_succ_pair`). The successor iterate of the paired operator is its value at the preceding iterate.

Proof. Successor iteration is composition with the operator, and the element action of a composite is successive application. ■

Theorem (`CoupledFixedPoints.aOp_succ`). The first approximation satisfies $a_{n+1} = s(a_n, b_n)$.

Proof. Express the successor iterate as one application of the paired operator, identify its first coordinate, and use the decomposition of the preceding iterate into a_n, b_n. ■

Theorem (`CoupledFixedPoints.bOp_succ`). The second approximation satisfies $b_{n+1} = t(a_n, b_n)$.

Proof. Express the successor iterate as one application of the paired operator, identify its second coordinate, and use the decomposition of the preceding iterate into a_n, b_n. ■

Theorem (`CoupledFixedPoints.iterElem_zero_pair`). The zeroth iterate of the paired operator is the bottom element of the pair space.

Proof. Zeroth iteration is the identity map evaluated at bottom. ■

Theorem (`CoupledFixedPoints.pair_bot_bot`). Pairing the two bottom function-space elements gives the bottom element of the pair space.

Proof. A product neighbourhood belongs to this pair exactly when both coordinates are bottom, which is precisely the defining condition for the product bottom. ■

Theorem (`CoupledFixedPoints.pairSpace_bot_fst`). The first coordinate of the bottom pair is the bottom function-space element.

Proof. Write the bottom pair as the pair of two bottoms and take its first coordinate. ■

Theorem (`CoupledFixedPoints.aOp_zero`). The initial first approximation a_0 is the constant-bottom endomap.

Proof. The zeroth paired iterate is bottom, its first coordinate is bottom, and the bottom function-space element represents the constant-bottom map. ■

Theorem (`CoupledFixedPoints.pairSpace_bot_snd`). The second coordinate of the bottom pair is the bottom function-space element.

Proof. Write the bottom pair as the pair of two bottoms and take its second coordinate. ■

Theorem (`CoupledFixedPoints.bOp_zero`). The initial second approximation b_0 is the constant-bottom endomap.

Proof. The zeroth paired iterate is bottom, its second coordinate is bottom, and the bottom function-space element represents the constant-bottom map. ■

Theorem (`CoupledFixedPoints.isFinitaryProjection_chain`). If $s(a, b)$ and $t(a, b)$ are finitary projections whenever a and b are, then every pair a_n, b_n in the approximation chain consists of finitary projections.

Proof. Induct on n. At zero both maps are constant bottom; at a successor use the two recursion equations and the assumed closure property. ■

Theorem (`CoupledFixedPoints.snd_mono`). Second-coordinate projection on the pair space is monotone.

Proof. Inclusion of pair elements preserves every neighbourhood witnessed in the second coordinate. ■

Theorem (`CoupledFixedPoints.isFinitaryProjection_aStar_bStar`). Under the same closure hypothesis, both coupled least fixed-point maps a^* and b^* are finitary projections.

Proof. Each coordinate chain is directed and consists of finitary projections. The finitary core fixes every stage and commutes with the directed supremum; since each coordinate of the paired least fixed point is that supremum, both limit maps satisfy the characterization of finitary projections. ■

Theorem (`CoupledFixedPoints.fixedNbhd_aStar_bStar_subsystem`). Suppose the coupled operator preserves pairs of finitary projections. If finitary projections a, b satisfy $s(a, b) \leq a$ and $t(a, b) \leq b$, then the fixed neighbourhood systems of a^*, b^* are respectively subsystems of those of a, b.

Proof. The pair represented by a, b is a pre-fixed point of the paired operator, so leastness places the paired least fixed point below it coordinatewise. Convert these inequalities to the endomaps a^*, b^*, then use the order characterization of fixed-neighbourhood subsystems for finitary projections. ■

Theorem (`CoupledFixedPoints.toElementMap_fixElement_fst`). Applying s to the paired least fixed element yields its first coordinate.

Proof. The paired least fixed element is fixed by the paired operator, whose first coordinate is the value of s. ■

Theorem (`CoupledFixedPoints.binOp_aStar_bStar_left`). The coupled solution satisfies $s(a^*, b^*) = a^*$.

Proof. The pair represented by a^*, b^* is the paired least fixed element, and the preceding coordinate equation identifies the value of s. ■

Theorem (`CoupledFixedPoints.toElementMap_fixElement_snd`). Applying t to the paired least fixed element yields its second coordinate.

Proof. The paired least fixed element is fixed by the paired operator, whose second coordinate is the value of t. ■

Theorem (`CoupledFixedPoints.binOp_aStar_bStar_right`). The coupled solution satisfies $t(a^*, b^*) = b^*$.

Proof. The pair represented by a^*, b^* is the paired least fixed element, and the preceding coordinate equation identifies the value of t. ■

Theorem (`CoupledFixedPoints.fixedDomain_aStar_bStar_iso`). Assume the coupled operator preserves finitary projections. If the fixed domain of $s(a, b)$ is always order-isomorphic to $S(a, b)$, and that of $t(a, b)$ to $T(a, b)$, for finitary a, b, then the fixed domains of a^*, b^* are respectively order-isomorphic to $S(a^*, b^*)$ and $T(a^*, b^*)$.

Proof. The coupled solution consists of finitary projections and satisfies both fixed-point equations. Specialize the two assumed families of order isomorphisms and replace their source endomaps using those equations. ■

Theorem (`RegularLanguage.Ssys_countable`). The type of regular-language systems is countable.

Proof. Every such system is denoted by a finite symbolic expression. Choose a denoting expression for each system; expressions form a countable type, so their surjective denotation makes the systems countable. The equivalent presentation by the stated membership predicate has the same cardinality. ■

Theorem (`P4.P4_embeds`). The neighbourhood system P_4 is a retract of the universal domain.

Proof. Apply the universal embedding theorem to this countable neighbourhood system. ■

Theorem (`RegularLanguage.Ssys_embeds`). The neighbourhood system of regular-language systems is a retract of the universal domain.

Proof. Its underlying type is countable, so the universal embedding theorem applies. ■

`CoupledFixedPoints.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.FixOp

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_3
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Sub8_6 FixOp

namespace CoupledFixedPoints

variable {α : Type} {E : NeighborhoodSystem α}

abbrev PairSpace : NeighborhoodSystem (ApproximableMap E E ? ApproximableMap E E) :=
  prod (funSpace E E) (funSpace E E)

def aOp (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) (n : ℕ) :
    ApproximableMap E E :=
  toApproxMap ((paired s t).iterElem n).fst

def aStar (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) :
    ApproximableMap E E :=
  toApproxMap (paired s t).fixElement.fst

def bOp (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) (n : ℕ) :
    ApproximableMap E E :=
  toApproxMap ((paired s t).iterElem n).snd

def bStar (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) :
    ApproximableMap E E :=
  toApproxMap (paired s t).fixElement.snd

def binOp (f : ApproximableMap (PairSpace (E := E)) (funSpace E E))
    (a b : ApproximableMap E E) : ApproximableMap E E :=
  toApproxMap (f.toElementMap (pair (toFilter a) (toFilter b)))

theorem fst_mono {z z' : (PairSpace (E := E)).Element} (h : z ≤ z') : z.fst ≤ z'.fst
    := by
  intro X hX
  rw [mem_fst] at hX ⊢
  exact ⟨hX.1, h _ hX.2⟩

theorem iterElem_pair_eq_pair (s t : ApproximableMap (PairSpace (E := E)) (funSpace E
    E)) (n : ℕ) :
    (paired s t).iterElem n = pair (toFilter (aOp s t n)) (toFilter (bOp s t n)) := by
  show (paired s t).iterElem n
      = pair (toFilter (toApproxMap ((paired s t).iterElem n).fst))
          (toFilter (toApproxMap ((paired s t).iterElem n).snd))
  rw [toFilter_toApproxMap, toFilter_toApproxMap, pair_fst_snd]

theorem iterElem_succ_pair (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E))
     (n : ℕ) :
    (paired s t).iterElem (n + 1) = (paired s t).toElementMap ((paired s t).iterElem n
    ) := by
  show ((paired s t).comp ((paired s t).iterMap n)).toElementMap (PairSpace (E := E)).
    bot
      = (paired s t).toElementMap (((paired s t).iterMap n).toElementMap (PairSpace (E
     := E)).bot)
  rw [toElementMap_comp]

theorem aOp_succ (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) (n : ℕ) :
```

```
    aOp s t (n + 1) = binOp s (aOp s t n) (bOp s t n) := by
  show toApproxMap ((paired s t).iterElem (n + 1)).fst
      = toApproxMap (s.toElementMap (pair (toFilter (aOp s t n)) (toFilter (bOp s t n)
    )))
  rw [iterElem_succ_pair, toElementMap_paired, fst_pair, iterElem_pair_eq_pair]

theorem bOp_succ (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) (n : ℕ) :
    bOp s t (n + 1) = binOp t (aOp s t n) (bOp s t n) := by
  show toApproxMap ((paired s t).iterElem (n + 1)).snd
      = toApproxMap (t.toElementMap (pair (toFilter (aOp s t n)) (toFilter (bOp s t n)
    )))
  rw [iterElem_succ_pair, toElementMap_paired, snd_pair, iterElem_pair_eq_pair]

theorem iterElem_zero_pair (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E))
    :
    (paired s t).iterElem 0 = (PairSpace (E := E)).bot := by
  show (idMap (PairSpace (E := E))).toElementMap (PairSpace (E := E)).bot
      = (PairSpace (E := E)).bot
  rw [toElementMap_idMap]

theorem pair_bot_bot : pair ((funSpace E E).bot) ((funSpace E E).bot) = (PairSpace (E
    := E)).bot := by
  apply Element.ext
  intro W
  rw [mem_pair, mem_bot]
  constructor
  · rintro ⟨X, Y, hX, hY, rfl⟩
    rw [mem_bot] at hX hY
    rw [hX, hY, prod_master]
  · intro hW
    refine ⟨(funSpace E E).master, (funSpace E E).master, ?_, ?_, ?_⟩
    · rw [mem_bot]
    · rw [mem_bot]
    · rw [hW, prod_master]

theorem pairSpace_bot_fst : ((PairSpace (E := E)).bot).fst = (funSpace E E).bot := by
  rw [← pair_bot_bot, fst_pair]

theorem aOp_zero (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) :
    aOp s t 0 = constMap E E.bot := by
  show toApproxMap ((paired s t).iterElem 0).fst = constMap E E.bot
  rw [iterElem_zero_pair, pairSpace_bot_fst, FixOp.toApproxMap_bot_eq_constMap]

theorem pairSpace_bot_snd : ((PairSpace (E := E)).bot).snd = (funSpace E E).bot := by
  rw [← pair_bot_bot, snd_pair]

theorem bOp_zero (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) :
    bOp s t 0 = constMap E E.bot := by
  show toApproxMap ((paired s t).iterElem 0).snd = constMap E E.bot
  rw [iterElem_zero_pair, pairSpace_bot_snd, FixOp.toApproxMap_bot_eq_constMap]

theorem isFinitaryProjection_chain (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E))
    (hst : ∀ a b, FinitaryProjection a → FinitaryProjection b →
      FinitaryProjection (binOp s a b) ∧ FinitaryProjection (binOp t a b)) :
    ∀ n, FinitaryProjection (aOp s t n) ∧ FinitaryProjection (bOp s t n) := by
```

```
  intro n
  induction n with
  | zero =>
    rw [aOp_zero, bOp_zero]
    exact ⟨FixOp.isFinitaryProjection_constMap_bot E,
      FixOp.isFinitaryProjection_constMap_bot E⟩
  | succ k ih =>
    rw [aOp_succ, bOp_succ]
    exact hst _ _ ih.1 ih.2

theorem snd_mono {z z' : (PairSpace (E := E)).Element} (h : z ≤ z') : z.snd ≤ z'.snd
    := by
  intro X hX
  rw [mem_snd] at hX ⊢
  exact ⟨hX.1, h _ hX.2⟩

theorem isFinitaryProjection_aStar_bStar (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E))
    (hst : ∀ a b, FinitaryProjection a → FinitaryProjection b →
      FinitaryProjection (binOp s a b) ∧ FinitaryProjection (binOp t a b)) :
    FinitaryProjection (aStar s t) ∧ FinitaryProjection (bStar s t) := by
  have hchain := isFinitaryProjection_chain s t hst
  have hsubA : ∀ n, subFilter (((paired s t).iterElem n).fst) = ((paired s t).iterElem
    n).fst := by
    intro n
    apply toApproxMap_injective
    rw [toApproxMap_subFilter]
    show sub (aOp s t n) = aOp s t n
    exact sub_eq_self_of_isFinitaryProjection (hchain n).1
  have hsubB : ∀ n, subFilter (((paired s t).iterElem n).snd) = ((paired s t).iterElem
    n).snd := by
    intro n
    apply toApproxMap_injective
    rw [toApproxMap_subFilter]
    show sub (bOp s t n) = bOp s t n
    exact sub_eq_self_of_isFinitaryProjection (hchain n).2
  have hdirA : ∀ i j : ℕ, ∃ k, ((paired s t).iterElem i).fst ≤ ((paired s t).iterElem
    k).fst ∧
      ((paired s t).iterElem j).fst ≤ ((paired s t).iterElem k).fst :=
    fun i j => ⟨max i j, fst_mono (iterElem_mono (paired s t) (le_max_left i j)),
      fst_mono (iterElem_mono (paired s t) (le_max_right i j))⟩
  have hdirB : ∀ i j : ℕ, ∃ k, ((paired s t).iterElem i).snd ≤ ((paired s t).iterElem
    k).snd ∧
      ((paired s t).iterElem j).snd ≤ ((paired s t).iterElem k).snd :=
    fun i j => ⟨max i j, snd_mono (iterElem_mono (paired s t) (le_max_left i j)),
      snd_mono (iterElem_mono (paired s t) (le_max_right i j))⟩
  have hdirSubA : ∀ i j : ℕ, ∃ k, subFilter (((paired s t).iterElem i).fst) ≤
      subFilter (((paired s t).iterElem k).fst) ∧
      subFilter (((paired s t).iterElem j).fst) ≤ subFilter (((paired s t).iterElem k)
    .fst) := by
    intro i j
    obtain ⟨k, hik, hjk⟩ := hdirA i j
    exact ⟨k, by rw [hsubA i, hsubA k]; exact hik, by rw [hsubA j, hsubA k]; exact hjk⟩
  have hdirSubB : ∀ i j : ℕ, ∃ k, subFilter (((paired s t).iterElem i).snd) ≤
      subFilter (((paired s t).iterElem k).snd) ∧
      subFilter (((paired s t).iterElem j).snd) ≤ subFilter (((paired s t).iterElem k)
```

```
    .snd) := by
    intro i j
    obtain ⟨k, hik, hjk⟩ := hdirB i j
    exact ⟨k, by rw [hsubB i, hsubB k]; exact hik, by rw [hsubB j, hsubB k]; exact hjk⟩
  have hEqA : (paired s t).fixElement.fst =
      iSupDirected (fun n => ((paired s t).iterElem n).fst) hdirA := by
    apply Element.ext
    intro X
    rw [mem_fst, mem_fixElement, mem_iSupDirected]
    constructor
    · rintro ⟨hX, n, hn⟩
      exact ⟨n, mem_fst.mpr ⟨hX, (mem_iterElem (paired s t) n).mpr hn⟩⟩
    · rintro ⟨n, hn⟩
      rw [mem_fst] at hn
      exact ⟨hn.1, n, (mem_iterElem (paired s t) n).mp hn.2⟩
  have hEqB : (paired s t).fixElement.snd =
      iSupDirected (fun n => ((paired s t).iterElem n).snd) hdirB := by
    apply Element.ext
    intro X
    rw [mem_snd, mem_fixElement, mem_iSupDirected]
    constructor
    · rintro ⟨hX, n, hn⟩
      exact ⟨n, mem_snd.mpr ⟨hX, (mem_iterElem (paired s t) n).mpr hn⟩⟩
    · rintro ⟨n, hn⟩
      rw [mem_snd] at hn
      exact ⟨hn.1, n, (mem_iterElem (paired s t) n).mp hn.2⟩
  have hkeyA : subFilter ((paired s t).fixElement.fst) = (paired s t).fixElement.fst
    := by
    rw [hEqA, subFilter_iSupDirected _ hdirA hdirSubA]
    apply Element.ext
    intro Z
    rw [mem_iSupDirected, mem_iSupDirected]
    simp_rw [hsubA]
  have hkeyB : subFilter ((paired s t).fixElement.snd) = (paired s t).fixElement.snd
    := by
    rw [hEqB, subFilter_iSupDirected _ hdirB hdirSubB]
    apply Element.ext
    intro Z
    rw [mem_iSupDirected, mem_iSupDirected]
    simp_rw [hsubB]
  refine ⟨?_, ?_⟩
  · show FinitaryProjection (toApproxMap (paired s t).fixElement.fst)
    apply isFinitaryProjection_of_sub_eq_self
    rw [← toApproxMap_subFilter, hkeyA]
  · show FinitaryProjection (toApproxMap (paired s t).fixElement.snd)
    apply isFinitaryProjection_of_sub_eq_self
    rw [← toApproxMap_subFilter, hkeyB]

theorem fixedNbhd_aStar_bStar_subsystem (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E))
    (hst : ∀ a b, FinitaryProjection a → FinitaryProjection b →
      FinitaryProjection (binOp s a b) ∧ FinitaryProjection (binOp t a b))
    {a b : ApproximableMap E E} (ha : FinitaryProjection a) (hb : FinitaryProjection b
    )
    (hpreA : binOp s a b ≤ a) (hpreB : binOp t a b ≤ b) :
    fixedNbhd (aStar s t) ? fixedNbhd a ∧ fixedNbhd (bStar s t) ? fixedNbhd b := by
```

```
  have h1 : s.toElementMap (pair (toFilter a) (toFilter b)) ≤ toFilter a := by
    have hpreA' : toApproxMap (s.toElementMap (pair (toFilter a) (toFilter b))) ≤
        toApproxMap (toFilter a) := by
      show binOp s a b ≤ toApproxMap (toFilter a)
      rw [toApproxMap_toFilter]
      exact hpreA
    have hiff := (funSpaceEquiv E E).le_iff_le
      (x := s.toElementMap (pair (toFilter a) (toFilter b))) (y := toFilter a)
    rw [funSpaceEquiv_apply, funSpaceEquiv_apply] at hiff
    exact hiff.mp hpreA'
  have h2 : t.toElementMap (pair (toFilter a) (toFilter b)) ≤ toFilter b := by
    have hpreB' : toApproxMap (t.toElementMap (pair (toFilter a) (toFilter b))) ≤
        toApproxMap (toFilter b) := by
      show binOp t a b ≤ toApproxMap (toFilter b)
      rw [toApproxMap_toFilter]
      exact hpreB
    have hiff := (funSpaceEquiv E E).le_iff_le
      (x := t.toElementMap (pair (toFilter a) (toFilter b))) (y := toFilter b)
    rw [funSpaceEquiv_apply, funSpaceEquiv_apply] at hiff
    exact hiff.mp hpreB'
  have hle : (paired s t).toElementMap (pair (toFilter a) (toFilter b)) ≤
      pair (toFilter a) (toFilter b) := by
    rw [toElementMap_paired]
    exact pair_le_pair_iff.mpr ⟨h1, h2⟩
  have hfix : (paired s t).fixElement ≤ pair (toFilter a) (toFilter b) :=
    fixElement_le_of_toElementMap_le (paired s t) hle
  have hfstle : (paired s t).fixElement.fst ≤ toFilter a := by
    have := fst_mono hfix; rwa [fst_pair] at this
  have hsndle : (paired s t).fixElement.snd ≤ toFilter b := by
    have := snd_mono hfix; rwa [snd_pair] at this
  have hAle : aStar s t ≤ a := by
    show toApproxMap (paired s t).fixElement.fst ≤ a
    rw [← toApproxMap_toFilter a]
    exact toApproxMap_monotone hfstle
  have hBle : bStar s t ≤ b := by
    show toApproxMap (paired s t).fixElement.snd ≤ b
    rw [← toApproxMap_toFilter b]
    exact toApproxMap_monotone hsndle
  obtain ⟨hFinA, hFinB⟩ := isFinitaryProjection_aStar_bStar s t hst
  exact ⟨(isFinitaryProjection_le_iff_fixedNbhd_subsystem hFinA ha).mp hAle,
    (isFinitaryProjection_le_iff_fixedNbhd_subsystem hFinB hb).mp hBle⟩

theorem toElementMap_fixElement_fst (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E)) :
    s.toElementMap (paired s t).fixElement = (paired s t).fixElement.fst := by
  have h : (paired s t).toElementMap (paired s t).fixElement = (paired s t).fixElement
     :=
    toElementMap_fixElement (paired s t)
  rw [toElementMap_paired] at h
  have h' := congrArg NeighborhoodSystem.Element.fst h
  rwa [fst_pair] at h'

theorem binOp_aStar_bStar_left (s t : ApproximableMap (PairSpace (E := E)) (funSpace E
     E)) :
    binOp s (aStar s t) (bStar s t) = aStar s t := by
  unfold binOp aStar bStar
```

```
  rw [toFilter_toApproxMap, toFilter_toApproxMap, pair_fst_snd,
    toElementMap_fixElement_fst]

theorem toElementMap_fixElement_snd (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E)) :
    t.toElementMap (paired s t).fixElement = (paired s t).fixElement.snd := by
  have h : (paired s t).toElementMap (paired s t).fixElement = (paired s t).fixElement
     :=
    toElementMap_fixElement (paired s t)
  rw [toElementMap_paired] at h
  have h' := congrArg NeighborhoodSystem.Element.snd h
  rwa [snd_pair] at h'

theorem binOp_aStar_bStar_right (s t : ApproximableMap (PairSpace (E := E)) (funSpace
    E E)) :
    binOp t (aStar s t) (bStar s t) = bStar s t := by
  unfold binOp aStar bStar
  rw [toFilter_toApproxMap, toFilter_toApproxMap, pair_fst_snd,
    toElementMap_fixElement_snd]

theorem fixedDomain_aStar_bStar_iso (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E))
    (hst : ∀ a b, FinitaryProjection a → FinitaryProjection b →
      FinitaryProjection (binOp s a b) ∧ FinitaryProjection (binOp t a b))
    (S T : ApproximableMap E E → ApproximableMap E E → Σ β : Type, NeighborhoodSystem
     β)
    (hS : ∀ a b, FinitaryProjection a → FinitaryProjection b →
      Nonempty (Fix (binOp s a b) ≃o (S a b).2.Element))
    (hT : ∀ a b, FinitaryProjection a → FinitaryProjection b →
      Nonempty (Fix (binOp t a b) ≃o (T a b).2.Element)) :
    Nonempty (Fix (aStar s t) ≃o (S (aStar s t) (bStar s t)).2.Element) ∧
      Nonempty (Fix (bStar s t) ≃o (T (aStar s t) (bStar s t)).2.Element) := by
  obtain ⟨hFinA, hFinB⟩ := isFinitaryProjection_aStar_bStar s t hst
  have h1 := hS (aStar s t) (bStar s t) hFinA hFinB
  have h2 := hT (aStar s t) (bStar s t) hFinA hFinB
  rw [binOp_aStar_bStar_left] at h1
  rw [binOp_aStar_bStar_right] at h2
  exact ⟨h1, h2⟩

end CoupledFixedPoints

end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems.P4

end DomainTheory.NeighbourhoodSystems.P4
namespace DomainTheory.NeighbourhoodSystems.RegularLanguage

instance Ssys_countable : Countable {S : Set (List Bool) // Ssys.mem S} := by
  classical
  let f (e : SExpr) : {S : Set (List Bool) // InS S} :=
    if h : (denote e).Nonempty then ⟨denote e, InS_denote_of_nonempty h⟩
    else ⟨Set.univ, InS.univ⟩
  have : Countable {S : Set (List Bool) // InS S} :=
    Function.Surjective.countable (f := f) fun ⟨X, hX⟩ => by
      obtain ⟨e, he⟩ := InS_exists_denote hX
```

```
      refine ⟨e, ?_⟩
      have hne : (denote e).Nonempty := he.symm ▸ hX.nonempty
      dsimp [f]
      rw [dite_eq_left hne]
      exact Subtype.ext he
  exact Countable.of_equiv ({S : Set (List Bool) // InS S})
    (Equiv.subtypeEquivRight fun _ => Ssys_mem.symm)

end DomainTheory.NeighbourhoodSystems.RegularLanguage
namespace DomainTheory.NeighbourhoodSystems.P4

theorem P4_embeds : neighborhoodSystem ? U :=
  theorem_8_8 neighborhoodSystem

end DomainTheory.NeighbourhoodSystems.P4
namespace DomainTheory.NeighbourhoodSystems.RegularLanguage

theorem Ssys_embeds : Ssys ? U :=
  theorem_8_8 Ssys

end DomainTheory.NeighbourhoodSystems.RegularLanguage
```

4.9.67 Exercise 8.24 — coupled domain equations

NeighbourhoodSystems/RetractsOfTheUniversalDomain/Exercise_8_24.lean

@import “Exercise_8_24.lean”

EXERCISE 8.24. Suppose S and T are two (binary-argument) constructs on domains that can be made into computable operators on projections of the universal domain. Show that we can therefore find a pair of effectively presented domains such that

$$D \cong S(D,E) \quad \text{and} \quad E \cong T(D,E).$$

The development is with the Coupled fixed points peer.

Theorem (`example_824`). The coupled fixed-point domain is effectively given.

Proof. By assembling the peer lemmas named in the packaging module. ■

Exercise_8_24.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    CoupledFixedPoints

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap DomainTheory
    .NeighbourhoodSystems.CoupledFixedPoints

theorem example_824 {α : Type} {E : NeighborhoodSystem α}
    (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E))
    (hst : ∀ a b, FinitaryProjection a → FinitaryProjection b →
      FinitaryProjection (binOp s a b) ∧ FinitaryProjection (binOp t a b)) :
    FinitaryProjection (aStar s t) ∧ FinitaryProjection (bStar s t) :=
  isFinitaryProjection_aStar_bStar s t hst
```

4.9.68 Reflexive domain

This peer carries out the reflexive-domain construction of Exercise 8.25. It builds a finitary projection whose fixed-point domain is universal and isomorphic to its own function space.

Theorem (`trianglelefteq_iterSys`). Every neighbourhood system V is a retract-subdomain of its countable power $V^{\mathbb{N}}$.

Proof. Embed an element as the constant sequence and project a sequence to its first coordinate. Projection after embedding is the identity, while embedding after projection lies below the identity. ■

Theorem (`example_825_embed`). For every V, one has $V \trianglelefteq V^{\mathbb{N}}$.

Proof. This is the preceding constant-sequence projection pair, uniformly quantified over V. ■

Definition (`arrowReindexOrderIso`). A bijection between index types induces an order isomorphism between the corresponding spaces of indexed families by reindexing.

Definition (`sumArrowOrderIso`). A family indexed by a disjoint sum is order-isomorphic to a pair of families indexed by the two summands.

Definition (`interleaveOrderIso`). A pair of countable families is order-isomorphic to one countable family by interleaving their coordinates.

Definition (`powProdOrderIso`). The product $V^{\mathbb{N}} \times V^{\mathbb{N}}$ is order-isomorphic to $V^{\mathbb{N}}$, by viewing both powers as sequences and interleaving them.

Theorem (`pow_prod_isomorphic`). The product of two copies of the countable power of V is isomorphic, as a domain, to that countable power.

Proof. The interleaving order isomorphism between their element orders induces the required domain isomorphism. ■

Theorem (`example_825_pow`). For every V, $V^{\mathbb{N}} \times V^{\mathbb{N}} \cong V^{\mathbb{N}}$.

Proof. Apply the preceding product-of-powers isomorphism. ■

Definition (`isoOfUnique`). Any two ordered types having exactly one element are order-isomorphic by the unique maps between them.

Theorem (`uniqueApproximableMapUnitSys`). There is exactly one approximable endomap of the one-point neighbourhood system.

Proof. Every such map has the same action on the sole element, hence equals the identity map. ■

Theorem (`uniqueFunSpaceUnitSys`). The function-space domain from the one-point system to itself has exactly one element.

Proof. Its elements correspond to approximable maps of the one-point system, of which there is exactly one. ■

Theorem (`isomorphic_of_unique`). Neighbourhood systems whose element domains each have exactly one element are isomorphic.

Proof. Their element orders are related by the unique order isomorphism between singleton ordered types. ■

Theorem (`funSpace_unitSys_isomorphic`). The function space from the one-point system to itself is isomorphic to the one-point system.

Proof. Both element domains have exactly one element. ■

Theorem (`example_825_unit`). The one-point domain satisfies $1^1 \cong 1$.

Proof. This is the preceding singleton function-space isomorphism. ■

Definition (`RDom`). Let R be the product $[U \to U] \times ([U \to U] \times U)$, the domain of triples consisting of two universal endomaps and one universal element.

Definition (`RMap`). For $c : U \to U$, define a map $R \to U$ sending (ψ, φ, x) to $c(\varphi(\psi(x)))$.

Definition (`curryOnceC`). Currying the preceding map once produces a map from the first function argument to a function of the remaining function argument and universal element.

Definition (`fixedNbhd_elementIso_fix`). For a finitary projection a, the element domain of its fixed-neighbourhood system is order-isomorphic to the subtype of elements fixed by a.

Definition (`lamOp`). For $c : U \to U$, twice currying the map $(\psi, \varphi, x) \mapsto c(\varphi(\psi(x)))$ gives an operator sending a to the curried action of composition around a and c.

Definition (`presentationIterU`). Choose a computable presentation of the countable power $U^{\mathbb{N}}$, which is effectively given because U is effectively given.

Definition (`cInj`). Choose the embedding of $U^{\mathbb{N}}$ into U supplied by universality of U.

Definition (`cProj`). Choose the corresponding projection from U onto $U^{\mathbb{N}}$.

Definition (`cCombinator`). The combinator projection on U is embedding after projection for the chosen projection pair between $U^{\mathbb{N}}$ and U.

Definition (`TArrow`). For an endomap a of U, associate the function space from the fixed-neighbourhood domain of a to the fixed-neighbourhood domain of the combinator projection.

Theorem (`cProj_comp_cInj`). Projecting after embedding $U^{\mathbb{N}}$ is the identity.

Proof. This is the retraction equation of the chosen universal projection pair. ■

Definition (`cElementIso`). The elements of $U^{\mathbb{N}}$ are order-isomorphic to the elements of U fixed by the combinator projection.

Definition (`tOpMap`). For $c : U \to U$, compose the curried composition operator with the map induced contravariantly and covariantly by the universal decoding and encoding maps, obtaining an endomap of the function-space domain.

Theorem (`cInj_comp_cProj_le`). Embedding after projecting U lies below the identity on U.

Proof. This is the order inequality of the chosen universal projection pair. ■

Theorem (`isFinitary_cCombinator`). The combinator projection is finitary.

Proof. It arises from a projection pair whose retract is the effectively given domain $U^{\mathbb{N}}$. ■

Theorem (`isRetraction_cCombinator`). The combinator projection is idempotent.

Proof. In its square, the middle projection-after-embedding composite is the identity. ■

Theorem (`isProjection_cCombinator`). The combinator endomap is a projection.

Proof. It is idempotent and lies below the identity. ■

Theorem (`isFinitaryProjection_cCombinator`). The combinator endomap is a finitary projection.

Proof. Combine its projection property with its finitarity. ■

Theorem (`fixedNbhd_cCombinator_isomorphic`). The fixed-neighbourhood domain of the combinator projection is isomorphic to $U^{\mathbb{N}}$.

Proof. Both domains are order-isomorphic to the fixed-point subtype of the same projection. ■

Theorem (`hVV_cCombinator`). The product of two copies of the combinator fixed-neighbourhood domain is isomorphic to that domain.

Proof. Transport both factors to $U^{\mathbb{N}}$, interleave the two countable powers, and transport back. ■

Theorem (`toElementMap_RMap`). The triple map determined by c sends (ψ, φ, x) to $c(\varphi(\psi(x)))$.

Proof. Evaluate the projections, pairings, and two function applications in its defining composite. ■

Theorem (`toApproxMap_lamOp`). Applying the curried composition operator determined by c to the function represented by a, then decoding the result, yields the exponential action induced by a and c.

Proof. Successive applications of currying reduce the statement to the explicit triple-value formula, which is exactly composition first by a and finally by c. ■

Theorem (`tOp_tOpMap`). The fixed-point transformer associated with the operator determined by c, evaluated at a, is the arrow combinator formed from a and c.

Proof. Decode the transformed function-space element and use the preceding identification of the curried operator with the exponential action. ■

Theorem (`hT_TArrow`). For every finitary projection a, the fixed points of its transformed arrow combinator are order-isomorphic to the elements of the function space from the fixed domain of a to the fixed domain of the combinator projection.

Proof. The transformer is the arrow combinator, whose fixed-point theorem identifies its fixed points with the indicated function-space domain. ■

Theorem (`ht_tOpMap`). If c is a finitary projection, then the transformer determined by c sends every finitary projection to a finitary projection.

Proof. The transformer is the arrow combinator, and arrow combinators preserve finitary projections in both arguments. ■

Theorem (`Dsol_isomorphic_funSpace_cCombinator`). Let D be the fixed-neighbourhood domain of the least fixed projection generated by the transformer for the combinator projection. Then $D \cong [D \to C]$, where C is the combinator fixed-neighbourhood domain.

Proof. The transformer preserves finitary projections, so its fixed projection is finitary. Identify the fixed-neighbourhood elements with its fixed points, then apply the fixed-domain theorem and the preceding arrow-family isomorphism. ■

Theorem (`U_trianglelefteq_Dsol`). The universal domain U is a retract-subdomain of the solution domain D.

Proof. Pass successively from U into $U^{\mathbb{N}}$, then into C, then into the constant-function subdomain of $[D \to C]$, and finally transport along $[D \to C] \cong D$. ■

Theorem (`exercise_8_25_main`). The solution domain D is isomorphic to its own function space $[D \to D]$.

Proof. The isomorphism $D \cong [D \to C]$, together with $C \times C \cong C$, yields the standard reflexive-domain self-function-space isomorphism. ■

Theorem (`exercise_8_25_universal`). The solution domain D is universal: $U \trianglelefteq D$.

Proof. This is the retract chain from the universal domain into the constructed solution domain. ■

`ReflexiveDomain.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_1

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.U
public import DomainTheory.NeighbourhoodSystems.DomainEquations.projPairSubdomain

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbedIntoPow
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ExampleC_le_B
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.ASharp
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ASharp_more
```

```
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.FixOp
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    CoupledFixedPoints
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbeddingsIntoU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    SumProdRetracts
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_10
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_15
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Combinators
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {α : Type*} (V : NeighborhoodSystem α)

theorem trianglelefteq_iterSys : V ? iterSys V :=
  trianglelefteq_of_projectionPair (embedIntoPow V) (head V)
    (head_comp_embedIntoPow V) (embedIntoPow_comp_head_le V)
end

open NeighborhoodSystem ApproximableMap

theorem example_825_embed :
    ∀ {α : Type*} (V : NeighborhoodSystem α), V ? iterSys V :=
  @trianglelefteq_iterSys

section
open NeighborhoodSystem
variable {α : Type*}

def arrowReindexOrderIso {ι ι' : Type*} (e : ι ≃ ι') (E : Type*) [Preorder E] :
    (ι → E) ≃o (ι' → E) where
  toFun f i' := f (e.symm i')
  invFun g i := g (e i)
  left_inv f := by funext i; simp
  right_inv g := by funext i'; simp
  map_rel_iff' := by
    intro f g
    constructor
    · intro h i
      simpa using h (e i)
    · intro h i'
      simpa using h (e.symm i')

def sumArrowOrderIso (α β : Type*) (E : Type*) [Preorder E] :
    (α ? β → E) ≃o (α → E) × (β → E) where
  toFun f := (fun a => f (Sum.inl a), fun b => f (Sum.inr b))
  invFun p := Sum.elim p.1 p.2
  left_inv f := by funext i; cases i <;> rfl
```

```
  right_inv p := rfl
  map_rel_iff' := by
    intro f g
    constructor
    · rintro ⟨h1, h2⟩ i
      cases i with
      | inl a => exact h1 a
      | inr b => exact h2 b
    · intro h
      exact ⟨fun a => h (Sum.inl a), fun b => h (Sum.inr b)⟩

def interleaveOrderIso (E : Type*) [Preorder E] : (ℕ → E) × (ℕ → E) ≃o (ℕ → E) :=
  (sumArrowOrderIso ℕ ℕ E).symm.trans (arrowReindexOrderIso Equiv.natSumNatEquivNat E)

def powProdOrderIso (V : NeighborhoodSystem α) :
    (prod (iterSys V) (iterSys V)).Element ≃o (iterSys V).Element :=
  (prodEquiv (iterSys V) (iterSys V)).trans <|
    (prodCongrOrderIso (iterSeqEquiv V) (iterSeqEquiv V)).trans <|
      (interleaveOrderIso V.Element).trans (iterSeqEquiv V).symm

theorem pow_prod_isomorphic (V : NeighborhoodSystem α) :
    prod (iterSys V) (iterSys V) ≅? iterSys V :=
  ⟨powProdOrderIso V⟩

end

open NeighborhoodSystem

theorem example_825_pow :
    ∀ {α : Type*} (V : NeighborhoodSystem α),
      prod (iterSys V) (iterSys V) ≅? iterSys V :=
  @pow_prod_isomorphic

section
open NeighborhoodSystem ApproximableMap
variable {α β : Type*}

def isoOfUnique (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
    [Unique V₀.Element] [Unique V₁.Element] : V₀.Element ≃o V₁.Element where
  toFun _ := default
  invFun _ := default
  left_inv x := (Unique.eq_default x).symm
  right_inv y := (Unique.eq_default y).symm
  map_rel_iff' := by
    intro a b
    exact iff_of_true (le_refl _) (le_of_eq (Subsingleton.elim a b))

instance uniqueApproximableMapUnitSys : Unique (ApproximableMap unitSys unitSys) where
  default := idMap unitSys
  uniq _f := ApproximableMap.ext_of_toElementMap fun _x => Subsingleton.elim _ _

instance uniqueFunSpaceUnitSys : Unique (funSpace unitSys unitSys).Element where
  default := (funSpaceEquiv unitSys unitSys).symm default
  uniq _x := (funSpaceEquiv unitSys unitSys).injective (Subsingleton.elim _ _)

theorem isomorphic_of_unique (V₀ : NeighborhoodSystem α) (V₁ : NeighborhoodSystem β)
```

```
    [Unique V₀.Element] [Unique V₁.Element] : V₀ ≅? V₁ :=
  ⟨isoOfUnique V₀ V₁⟩

theorem funSpace_unitSys_isomorphic : funSpace unitSys unitSys ≅? unitSys :=
  isomorphic_of_unique _ _

end

open NeighborhoodSystem

theorem example_825_unit : funSpace unitSys unitSys ≅? unitSys :=
  funSpace_unitSys_isomorphic

open NeighborhoodSystem ApproximableMap

abbrev RDom := prod (funSpace U U) (prod (funSpace U U) U)

noncomputable def RMap (c : ApproximableMap U U) : ApproximableMap RDom U :=
  c.comp ((evalMap U U).comp
    (paired
      ((proj₀ (funSpace U U) U).comp (proj₁ (funSpace U U) (prod (funSpace U U) U)))
      ((evalMap U U).comp
        (paired (proj₀ (funSpace U U) (prod (funSpace U U) U))
          ((proj₁ (funSpace U U) U).comp (proj₁ (funSpace U U) (prod (funSpace U U) U))
   )))))

noncomputable def curryOnceC (c : ApproximableMap U U) :
    ApproximableMap (funSpace U U) (funSpace (prod (funSpace U U) U) U) :=
  curry (RMap c)

noncomputable def fixedNbhd_elementIso_fix {a : ApproximableMap U U} (ha :
    FinitaryProjection a) :
    (fixedNbhd a).Element ≃o {y : U.Element // a.toElementMap y = y} := by
  have h := Subsystem.elementIso (fixedNbhd_subsystem a)
  unfold Subsystem.retractionOfSubsystem at h
  rwa [inj_comp_proj_eq_self ha] at h

noncomputable def lamOp (c : ApproximableMap U U) :
    ApproximableMap (funSpace U U) (funSpace (funSpace U U) (funSpace U U)) :=
  (curryC (funSpace U U) U U).comp (curryOnceC c)

noncomputable def presentationIterU : ComputablePresentation (iterSys U) :=
  Classical.choice (iterSys_isEffectivelyGiven U_isEffectivelyGiven)

noncomputable def cInj : ApproximableMap (iterSys U) U :=
  (theorem_8_8_b_strong presentationIterU).choose

noncomputable def cProj : ApproximableMap U (iterSys U) :=
  (theorem_8_8_b_strong presentationIterU).choose_spec.choose

noncomputable def cCombinator : ApproximableMap U U := cInj.comp cProj

noncomputable def TArrow (a : ApproximableMap U U) : Σ β : Type, NeighborhoodSystem β
    :=
  ⟨_, funSpace (fixedNbhd a) (fixedNbhd cCombinator)⟩
```

```
theorem cProj_comp_cInj : cProj.comp cInj = idMap (iterSys U) :=
  (theorem_8_8_b_strong presentationIterU).choose_spec.choose_spec.1

noncomputable def cElementIso :
    (iterSys U).Element ≃o {y : U.Element // cCombinator.toElementMap y = y} :=
  elementIsoOfProjectionPair cInj cProj cProj_comp_cInj rfl

noncomputable def tOpMap (c : ApproximableMap U U) :
    ApproximableMap (funSpace U U) (funSpace U U) :=
  (expMap jArrow iArrow).comp (lamOp c)

theorem cInj_comp_cProj_le : cInj.comp cProj ≤ idMap U :=
  (theorem_8_8_b_strong presentationIterU).choose_spec.choose_spec.2.1

theorem isFinitary_cCombinator : IsFinitary cCombinator :=
  isFinitary_of_projectionPair cInj cProj cProj_comp_cInj rfl

theorem isRetraction_cCombinator : Retraction cCombinator := by
  constructor
  show (cInj.comp cProj).comp (cInj.comp cProj) = cInj.comp cProj
  rw [comp_assoc, ← comp_assoc cProj cInj cProj, cProj_comp_cInj, idMap_comp]

theorem isProjection_cCombinator : IsProjection cCombinator :=
  ⟨isRetraction_cCombinator, cInj_comp_cProj_le⟩

theorem isFinitaryProjection_cCombinator : FinitaryProjection cCombinator :=
  ⟨⟨isProjection_cCombinator, isFinitary_cCombinator⟩⟩

theorem fixedNbhd_cCombinator_isomorphic : fixedNbhd cCombinator ≅? iterSys U :=
  ⟨(fixedNbhd_elementIso_fix isFinitaryProjection_cCombinator).trans cElementIso.symm⟩

theorem hVV_cCombinator :
    prod (fixedNbhd cCombinator) (fixedNbhd cCombinator) ≅? fixedNbhd cCombinator :=
  ((fixedNbhd_cCombinator_isomorphic.prod fixedNbhd_cCombinator_isomorphic).trans
    (pow_prod_isomorphic U)).trans fixedNbhd_cCombinator_isomorphic.symm

theorem toElementMap_RMap (c : ApproximableMap U U) (ψ φ : (funSpace U U).Element) (x
    : U.Element) :
    (RMap c).toElementMap (pair ψ (pair φ x)) =
      c.toElementMap ((toApproxMap φ).toElementMap ((toApproxMap ψ).toElementMap x))
    := by
  simp only [RMap, toElementMap_comp, toElementMap_paired, toElementMap_proj₀,
    toElementMap_proj₁,
    fst_pair, snd_pair, evalMap_apply]

theorem toApproxMap_lamOp (c a : ApproximableMap U U) :
    toApproxMap ((lamOp c).toElementMap (toFilter a)) = expMap a c := by
  apply ApproximableMap.ext_of_toElementMap
  intro φ
  apply (funSpaceEquiv U U).injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply]
  apply ApproximableMap.ext_of_toElementMap
  intro x
  show (toApproxMap ((toApproxMap ((lamOp c).toElementMap (toFilter a))).toElementMap
    φ)).toElementMap x
    = (toApproxMap ((expMap a c).toElementMap φ)).toElementMap x
```

```
  rw [lamOp, toElementMap_comp, curryC_toApproxMap, toElementMap_curry_apply,
    curryOnceC,
    toElementMap_curry_apply, toElementMap_RMap, Sub8_6.toApproxMap_toFilter,
    expMap.toApproxMap_toElementMap, toElementMap_comp, toElementMap_comp]

theorem tOp_tOpMap (c a : ApproximableMap U U) :
    FixOp.tOp (tOpMap c) a = arrowComb a c := by
  show toApproxMap ((tOpMap c).toElementMap (toFilter a)) = arrowComb a c
  rw [tOpMap, toElementMap_comp, expMap.toApproxMap_toElementMap, toApproxMap_lamOp,
    expMap.eq_lamComb]
  rfl

theorem hT_TArrow (a : ApproximableMap U U) (ha : FinitaryProjection a) :
    Nonempty (FixOp.Fix (FixOp.tOp (tOpMap cCombinator) a) ≃o
      (TArrow a).2.Element) := by
  rw [tOp_tOpMap]
  exact ⟨(arrowComb.elementIso a cCombinator ha isFinitaryProjection_cCombinator).symm⟩

theorem ht_tOpMap (c : ApproximableMap U U) (hc : FinitaryProjection c) :
    ∀ a, FinitaryProjection a → FinitaryProjection (FixOp.tOp (tOpMap c) a) := by
  intro a ha
  rw [tOp_tOpMap]
  exact arrowComb.finitaryProjection a c ha hc

theorem Dsol_isomorphic_funSpace_cCombinator :
    fixedNbhd (FixOp.fixOp (tOpMap cCombinator)) ≅?
      funSpace (fixedNbhd (FixOp.fixOp (tOpMap cCombinator))) (fixedNbhd cCombinator)
    := by
  have hht := ht_tOpMap cCombinator isFinitaryProjection_cCombinator
  have hFP := FixOp.isFinitaryProjection_fixOp (tOpMap cCombinator) hht
  have h1 : Nonempty ((fixedNbhd (FixOp.fixOp (tOpMap cCombinator))).Element ≃o
      FixOp.Fix (FixOp.fixOp (tOpMap cCombinator))) :=
    ⟨fixedNbhd_elementIso_fix hFP⟩
  have h2 := FixOp.fixedDomain_fixOp_iso_T (tOpMap cCombinator) hht TArrow hT_TArrow
  exact h1.elim fun e1 => h2.elim fun e2 => ⟨e1.trans e2⟩

theorem U_trianglelefteq_Dsol : U ? fixedNbhd (FixOp.fixOp (tOpMap cCombinator)) := by
  set D := fixedNbhd (FixOp.fixOp (tOpMap cCombinator)) with hD
  have h1 : U ? iterSys U := trianglelefteq_iterSys U
  have h2 : iterSys U ? fixedNbhd cCombinator :=
    ⟨fixedNbhd cCombinator, Subsystem.refl _, fixedNbhd_cCombinator_isomorphic.symm⟩
  have h3 : fixedNbhd cCombinator ? funSpace D (fixedNbhd cCombinator) :=
    trianglelefteq_funSpace_const D (fixedNbhd cCombinator)
  have h4 : funSpace D (fixedNbhd cCombinator) ? D :=
    ⟨D, Subsystem.refl D, Dsol_isomorphic_funSpace_cCombinator.symm⟩
  exact trianglelefteq_trans (trianglelefteq_trans (trianglelefteq_trans h1 h2) h3) h4

theorem exercise_8_25_main :
    fixedNbhd (FixOp.fixOp (tOpMap cCombinator)) ≅?
      funSpace (fixedNbhd (FixOp.fixOp (tOpMap cCombinator)))
        (fixedNbhd (FixOp.fixOp (tOpMap cCombinator))) :=
  funSpace_self_isomorphic Dsol_isomorphic_funSpace_cCombinator hVV_cCombinator

theorem exercise_8_25_universal : U ? fixedNbhd (FixOp.fixOp (tOpMap cCombinator)) :=
  U_trianglelefteq_Dsol
```

```
end DomainTheory.NeighbourhoodSystems
```

4.9.69 Exercise 8.25 — reflexive function space

The problem is to find non-trivial solutions to the domain equation

$$(\spadesuit) \qquad D \cong D \to D.$$

Show that the "obvious" solution by retracts is of no use because

$$1 \to 1 = 1$$

for projections. Change the method as follows. Show first

$$U^\infty \times U^\infty \cong U^\infty.$$

Next solve

$$D \cong D \to U^\infty$$

and remark that $U \lhd D$; so D is universal and non-trivial. Finally prove $(\spadesuit)$ for this D. (Hint: First show

$$D \times D \cong D,$$

and then show D satisfies $(\spadesuit)$.) Is this D effectively given?

The development is with the Reflexive domain peer.

Theorem (`example_825`). A non-trivial reflexive function space exists and contains U as a retract.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_25.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ReflexiveDomain

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap DomainTheory
    .NeighbourhoodSystems.FixOp

theorem example_825 :
    (fixedNbhd (fixOp (tOpMap cCombinator)) ≅?
      funSpace (fixedNbhd (fixOp (tOpMap cCombinator)))
        (fixedNbhd (fixOp (tOpMap cCombinator)))) ∧
      (U ? fixedNbhd (fixOp (tOpMap cCombinator))) :=
  ⟨exercise_8_25_main, exercise_8_25_universal⟩
```

4.9.70 U apply and U lam

This peer gives application and abstraction directly in the universal domain and formalizes the sandwich calculation underlying the interpretation of typed abstraction and application.

Definition (`Uapply`). Universal application decodes its first argument as a continuous self-map of U and evaluates that map at its second argument.

Definition (`Ulam`). Universal abstraction encodes a continuous self-map of U, regarded as an element of the function-space domain, back into U.

Definition (`translateAbs`). Given endomaps a and b, the translation of a body f is the composite $b \circ f \circ a$.

Theorem (`Uapply_Ulam`). Applying the universal abstraction of a map f to x gives $f(x)$.

Proof. Decoding immediately after encoding is the identity on the function-space domain, so function-space evaluation reduces to evaluation of f at x. ■

Theorem (`translateAbs_idMap`). Translating the identity body between a and b gives the composite $b \circ a$.

Proof. Composition with the identity does not change the intervening map. ■

Theorem (`example_826`). For every neighbourhood system and endomaps a, b, translating the identity body yields $b \circ a$.

Proof. This is the preceding identity-body calculation, uniformly quantified over the neighbourhood system and the two maps. ■

Theorem (`translateAbs_sandwich`). If a and b are retractions, then every translated body $f' = b \circ f \circ a$ satisfies the sandwich equation $b \circ f' \circ a = f'$.

Proof. Associativity gathers the two copies of a and the two copies of b; idempotence of the retractions removes the repetitions. ■

Theorem (`translateApp_hasType`). Suppose f satisfies $b \circ f \circ a = f$. If x is fixed by a, then $f(x)$ is fixed by b.

Proof. Evaluate the sandwich equation at x and use the assumption $a(x) = x$. ■

`UapplyUlam.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

noncomputable def Uapply (u x : U.Element) : U.Element :=
  (evalMap U U).toElementMap (pair (jArrow.toElementMap u) x)

end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
```

```
noncomputable def Ulam (f : ApproximableMap U U) : U.Element :=
  iArrow.toElementMap (toFilter f)

section
universe u
variable {α : Type u} {E : NeighborhoodSystem α}

def translateAbs (a b body : ApproximableMap E E) : ApproximableMap E E :=
  b.comp (body.comp a)
end

theorem Uapply_Ulam (f : ApproximableMap U U) (x : U.Element) :
    Uapply (Ulam f) x = f.toElementMap x := by
  have hround : toApproxMap (toFilter f) = f := by
    have he := (funSpaceEquiv U U).apply_symm_apply f
    rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he
  unfold Uapply Ulam
  have h1 : jArrow.toElementMap (iArrow.toElementMap (toFilter f))
      = (jArrow.comp iArrow).toElementMap (toFilter f) :=
    (toElementMap_comp jArrow iArrow (toFilter f)).symm
  rw [h1, jArrow_comp_iArrow, toElementMap_idMap, evalMap_apply, hround]

section
universe u
variable {α : Type u} {E : NeighborhoodSystem α}

theorem translateAbs_idMap {a b : ApproximableMap E E} : translateAbs a b (idMap E) =
    b.comp a := by
  unfold translateAbs
  rw [idMap_comp]
end

theorem example_826 :
    ∀ {α : Type*} {E : NeighborhoodSystem α} {a b : ApproximableMap E E},
      translateAbs a b (idMap E) = b.comp a :=
  @translateAbs_idMap

section
universe u
variable {α : Type u} {E : NeighborhoodSystem α}

theorem translateAbs_sandwich {a b body : ApproximableMap E E}
    (ha : Retraction a) (hb : Retraction b) :
    b.comp ((translateAbs a b body).comp a) = translateAbs a b body := by
  show b.comp ((b.comp (body.comp a)).comp a) = b.comp (body.comp a)
  rw [comp_assoc b (body.comp a) a, comp_assoc body a a, ha.out, ← comp_assoc b b (
    body.comp a), hb.out]

theorem translateApp_hasType {a b f : ApproximableMap E E} (hsandwich : b.comp (f.comp
     a) = f)
    {x : E.Element} (hx : a.toElementMap x = x) :
    b.toElementMap (f.toElementMap x) = f.toElementMap x := by
  conv_rhs => rw [← hsandwich, toElementMap_comp, toElementMap_comp, hx]

end
```

```
end DomainTheory.NeighbourhoodSystems
```

4.9.71 Exercise 8.26 — untyped calculus in U

Discuss in more detail the "pay-off" for U, namely the translation of "untyped" λ-calculus into U as shown by the equations at the end of the lecture after the proof of 8.9. In particular show how the whole of the **typed** λ-calculus can be retranslated back into U with the aid of projections. (Hint: Whenever you want to write

$$f : D_a \to D_b,$$

write instead

$$f = b \circ f \circ a,$$

where a, b are finitary projections. Whenever you want to form a λ-abstraction

$$\lambda x^{D_a}.\, \sigma,$$

where σ is of type D_b, instead form

$$\lambda x.\, b(\sigma'[a(x)/x]),$$

where σ' is the further translation of σ into untyped λ-calculus. Be sure to show that this result "has the right type" in the sense defined above.)

The development is with the U apply and U lam peer.

Theorem (`example_826`). Untyped application and lambda combinators behave as in the exercise.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_26.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    UapplyUlam

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

theorem example_826 (f : ApproximableMap U U) (x : U.Element) :
    Uapply (Ulam f) x = f.toElementMap x :=
  Uapply_Ulam f x
```

4.9.72 Polymorphic type

This peer develops Scott's dependent-product construction for polymorphic types. Its central result is that the function-space operator obtained by decoding fibre codes is a finitary projection.

Definition (`PolymorphicType`). A code assignment $d : U \to U$ is polymorphic when it is unchanged by projecting both its argument and its value to the universal subdomain of type codes: $\mathrm{sub} \circ d \circ \mathrm{sub} = d$.

Definition (`scFamily`). For $t \in U$, the Scott family consists of valid neighbourhoods T which contain t and are self-related by the universal subprojection.

Theorem (`instNonemptyScFamily`). The Scott family of every element is nonempty.

Proof. The master neighbourhood contains every element and is related to itself by every approximable map. ■

Definition (`scPrincipal`). Each member T of the Scott family determines the principal element generated by the evidence that t belongs to T.

Theorem (`toElementMap_mem_fix_of_isRetraction_sandwich`). If a is a retraction, every value of the sandwich $a \circ g \circ a$ is fixed by a.

Proof. Applying a once more introduces $a \circ a$, which contracts to a by idempotence. ■

Theorem (`polymorphicType_apply_mem_fix`). Every value of a polymorphic code assignment is fixed by the universal subprojection.

Proof. The polymorphic sandwich equation identifies the assignment with a sandwich by the subprojection; the preceding retraction argument then applies. ■

Theorem (`isFinitaryProjection_toApproxMap_jArrow_of_fix`). If w is fixed by the universal subprojection, then the self-map decoded from w is a finitary projection.

Proof. The fixed-point equation transfers through decoding to the function-space subprojection, whose fixed points are exactly the finitary projections. ■

Theorem (`isFinitaryProjection_decode_subU`). Decoding the subprojection of any universal element yields a finitary projection.

Proof. The image of a retraction is fixed by that retraction, so the preceding fixed-point criterion applies. ■

Theorem (`toElementMap_piDUncurried`). At a pair (φ, t), the uncurried dependent-product operator decodes the projected code $d(\mathrm{sub}(t))$ and applies that fibre projection to the value $\varphi(\mathrm{sub}(t))$.

Proof. Evaluate the pairing, projections, and function-space evaluation in the defining composite. ■

Theorem (`toElementMap_piDApply`). For maps $d, f : U \to U$, dependent-product application at t is the fibre projection decoded from $d(\mathrm{sub}(t))$, applied to $f(\mathrm{sub}(t))$.

Proof. This is the elementwise evaluation of the paired composite defining dependent-product application. ■

Theorem (`toApproxMap_toElementMap_piD`). Decoding the image of a function-space element φ under the dependent-product operator gives dependent-product application to the map represented by φ.

Proof. Currying identifies evaluation of the dependent-product operator at φ with the corresponding section of its uncurried form. ■

Theorem (`piD_rel_iff`). A function-space neighbourhood X is related to W by the dependent-product operator exactly when both are valid and the section determined by the principal element of X belongs to W.

Proof. This is the neighbourhood relation for a curried approximable map. ■

Theorem (`gSection_piDUncurried_rel_iff`). The section determined by a valid function neighbourhood X relates a valid argument neighbourhood Y to Z exactly when applying the corresponding dependent-product map to the principal element of Y belongs to Z.

Proof. The section relation is evaluation of the uncurried map at the two principal elements, and the explicit value formula identifies this with dependent-product application. ■

Theorem (`piD_toElementMap_mem_iff`). A neighbourhood Y belongs to the image of x under the dependent-product operator exactly when Y is valid and the dependent-product application represented by x belongs to Y.

Proof. Decode the image as an approximable map and use the membership characterization of its representing filter. ■

Theorem (`hii_easy_direction`). If $x \in X$, $X \subseteq Y$, and X is self-related by the dependent-product operator, then the image of x belongs to every valid Y.

Proof. The self-related neighbourhood X supplies the finite witness required for image membership, and upward closure carries it to Y. ■

Theorem (`subU_principal_eq_of_rel_self`). If a valid neighbourhood T is self-related by the universal subprojection, then its principal element is fixed by that projection.

Proof. The projection lies below the identity for one inequality; self-relatedness places the principal element below its projected image for the reverse inequality. ■

Theorem (`exists_rel_self_subset_of_mem`). If t is fixed by the universal subprojection and $t \in T_0$, then there is a self-related neighbourhood T containing t with $T \subseteq T_0$.

Proof. Apply the finite-neighbourhood formula for the finitary universal subprojection to the membership of its fixed value in T_0. ■

Theorem (`scFamily_directed`). The principal elements indexed by the Scott family of a fixed point t form a directed family.

Proof. Intersect the two given neighbourhoods, refine that intersection to a self-related neighbourhood containing t, and use inclusion reversal for principal elements. ■

Theorem (`eq_iSupDirected_scPrincipal`). A fixed point t of the universal subprojection is the directed supremum of the principal elements indexed by its Scott family.

Proof. Every neighbourhood of t has a self-related refinement and therefore occurs above one indexed principal; conversely, every neighbourhood of an indexed principal contains t by upward closure. ■

Theorem (`isFinitaryProjection_piD_of_formula`). If image membership for the dependent-product operator is characterized by valid neighbourhoods containing a self-related finite witness, then that operator is a finitary projection.

Proof. The assumed equivalence is precisely the finite-neighbourhood criterion for a finitary projection. ■

Theorem (`le_idMap_piD`). The dependent-product operator lies below the identity on the function-space domain.

Proof. Pointwise, each decoded fibre map is a projection and hence decreases its argument; projecting the input type code also decreases it, and monotonicity of the represented function completes the comparison. ■

Theorem (`piDApply_toElementMap_eq`). Evaluating dependent-product application at the projection of t uses the same decoded fibre projection and the same projected argument as evaluating its explicit formula at t.

Proof. Idempotence removes the repeated universal subprojection. ■

Theorem (`isRetraction_piD`). The dependent-product operator is a retraction.

Proof. After decoding function-space elements, applying the operator twice repeats at each argument the same decoded finitary projection; its idempotence collapses the second application. ■

Theorem (`isProjection_piD`). The dependent-product operator is a projection.

Proof. It is a retraction and lies below the identity by the preceding two results. ■

Theorem (`piDType_monotone`). If $s \leq s'$, then the fibre projection at s lies below the fibre projection at s'.

Proof. The fibre-code map is monotone, and decoding function-space elements preserves order. ■

Theorem (`piD_rel_self_iff`). A function neighbourhood X is self-related by the dependent-product operator exactly when X is valid and its associated section belongs to X.

Proof. Specialize the general relation characterization to equal source and target neighbourhoods. ■

Theorem (`toElementMap_piDTypeMap`). The fibre-code map sends s to the decoded function-space element represented by the projected code $d(s)$.

Proof. Evaluate the composite defining the fibre-code map. ■

Theorem (`isFinitaryProjection_piDType`). Every fibre map selected by a code assignment is a finitary projection.

Proof. Its code is first passed through the universal subprojection, and decoding any such projected code yields a finitary projection. ■

Theorem (`toElementMap_piDApply'`). Dependent-product application at t is the fibre projection indexed by $\mathrm{sub}(t)$, applied to $f(\mathrm{sub}(t))$.

Proof. Substitute the elementwise formula for the fibre-code map into the explicit formula for dependent-product application. ■

Theorem (`exists_X_of_mem_step`). If dependent-product application at x belongs to a one-step function neighbourhood from Y_0 to Z_0, then there is a function neighbourhood X containing x, contained in that step neighbourhood, and self-related by the dependent-product operator.

Proof. Approximate the projected argument by the directed Scott family, use finitarity of the selected fibre projection to obtain a self-related output neighbourhood, choose a common upper approximation, and form the resulting step neighbourhood. Its defining relation gives self-relatedness. ■

Theorem (`exists_X_of_mem_stepFun`). The same finite self-related witness exists when the target is a function neighbourhood formed from a finite list of step conditions.

Proof. Induct over the list. The empty list uses the master neighbourhood; for a new condition, intersect its witness with the inductive witness and use closure of the relation under intersection on the right. ■

Theorem (`hii_hard_direction`). If dependent-product application at x belongs to a valid function neighbourhood Y, then Y contains a self-related function neighbourhood X containing x.

Proof. Every valid function neighbourhood is presented by a finite list of step conditions, to which the preceding finite-list construction applies. ■

Theorem (`hii_piD`). The image of x under the dependent-product operator belongs to Y exactly when Y is valid and contains a self-related function neighbourhood X containing x.

Proof. The forward implication combines the image-membership formula with the hard direction; the reverse implication is the easy finite-witness direction. ■

Theorem (`isFinitaryProjection_piD`). For every code assignment d, the dependent-product operator determined by d is a finitary projection.

Proof. Its image-membership equivalence is the finite-neighbourhood formula required by the general criterion for finitary projections. ■

`PolymorphicType.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.ASharp
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.subU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.piD
```

```
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    principal_le_iff

public import Mathlib.Combinatorics.Colex
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

class PolymorphicType (d : ApproximableMap U U) : Prop where
  out : subU.comp (d.comp subU) = d

def scFamily (t : U.Element) : Type := {T : Set ? // subU.rel T T ∧ t.mem T}

instance instNonemptyScFamily (t : U.Element) : Nonempty (scFamily t) :=
  ⟨⟨U.master, subU.master_rel, t.master_mem⟩⟩

def scPrincipal {t : U.Element} (i : scFamily t) : U.Element :=
  U.principal (t.sub i.2.2)

theorem toElementMap_mem_fix_of_isRetraction_sandwich {a g : ApproximableMap U U}
    (ha : Retraction a) (z : U.Element) :
    a.toElementMap ((a.comp (g.comp a)).toElementMap z) = (a.comp (g.comp a)).
    toElementMap z := by
  rw [toElementMap_comp, ← toElementMap_comp a a, ha.out]

theorem polymorphicType_apply_mem_fix {d : ApproximableMap U U} (hd : PolymorphicType
    d)
    (z : U.Element) : subU.toElementMap (d.toElementMap z) = d.toElementMap z := by
  have h := toElementMap_mem_fix_of_isRetraction_sandwich subU.isProjection.1 (g := d)
     z
  rwa [hd.out] at h

theorem isFinitaryProjection_toApproxMap_jArrow_of_fix {w : U.Element}
    (hw : subU.toElementMap w = w) :
    FinitaryProjection (toApproxMap (jArrow.toElementMap w)) := by
  apply isFinitaryProjection_of_sub_eq_self
  have hfix := subU.subApprox_fix_of_fix hw
  have hstep := congrArg toApproxMap hfix
  rwa [subApprox.toElementMap_eq, Sub8_6.toApproxMap_subFilter] at hstep

theorem isFinitaryProjection_decode_subU (z : U.Element) :
    FinitaryProjection (toApproxMap (jArrow.toElementMap (subU.toElementMap z))) :=
  isFinitaryProjection_toApproxMap_jArrow_of_fix
    (toElementMap_idem_of_isRetraction subU.isProjection.1 z)

theorem toElementMap_piDUncurried (d : ApproximableMap U U) (φ : (funSpace U U).
    Element)
    (t : U.Element) :
    (piDUncurried d).toElementMap (pair φ t) =
```

```
      (toApproxMap (jArrow.toElementMap (subU.toElementMap (d.toElementMap
          (subU.toElementMap t))))).toElementMap
        ((toApproxMap φ).toElementMap (subU.toElementMap t)) := by
  simp only [piDUncurried, toElementMap_comp, toElementMap_paired, toElementMap_proj₀,
    toElementMap_proj₁, fst_pair, snd_pair, evalMap_apply]

theorem toElementMap_piDApply (d f : ApproximableMap U U) (t : U.Element) :
    (piDApply d f).toElementMap t =
      (toApproxMap (jArrow.toElementMap (subU.toElementMap (d.toElementMap
          (subU.toElementMap t))))).toElementMap (f.toElementMap (subU.toElementMap t))
    ) := by
  simp only [piDApply, toElementMap_comp, toElementMap_paired, evalMap_apply]

theorem toApproxMap_toElementMap_piD (d : ApproximableMap U U) (φ : (funSpace U U).
    Element) :
    toApproxMap ((piD d).toElementMap φ) = piDApply d (toApproxMap φ) := by
  apply ApproximableMap.ext_of_toElementMap
  intro t
  rw [piD, toElementMap_curry_apply, toElementMap_piDUncurried, toElementMap_piDApply]

theorem piD_rel_iff (d : ApproximableMap U U) {X W : Set (ApproximableMap U U)} :
    (piD d).rel X W ↔ ∃ hX : (funSpace U U).mem X, (funSpace U U).mem W ∧
      gSection (piDUncurried d) hX ∈ W :=
  curry_rel

theorem gSection_piDUncurried_rel_iff (d : ApproximableMap U U)
    {X : Set (ApproximableMap U U)} (hX : (funSpace U U).mem X) {Y Z : Set ?} (hY : U.
    mem Y) :
    (gSection (piDUncurried d) hX).rel Y Z ↔
      ((piDApply d (toApproxMap ((funSpace U U).principal hX))).toElementMap
        (U.principal hY)).mem Z := by
  rw [gSection_rel, rel_iff_mem_principal (piDUncurried d) (prod_mem_prodNbhd hX hY),
    ← pair_principal_eq_principal_prodNbhd hX hY, toElementMap_piDUncurried,
    ← toElementMap_piDApply]

theorem piD_toElementMap_mem_iff (d : ApproximableMap U U) (x : (funSpace U U).Element
    )
    {Y : Set (ApproximableMap U U)} :
    ((piD d).toElementMap x).mem Y ↔
      (funSpace U U).mem Y ∧ piDApply d (toApproxMap x) ∈ Y := by
  conv_lhs => rw [← Sub8_6.toFilter_toApproxMap ((piD d).toElementMap x)]
  rw [mem_toFilter, toApproxMap_toElementMap_piD]

theorem hii_easy_direction (d : ApproximableMap U U) (x : (funSpace U U).Element)
    {Y : Set (ApproximableMap U U)} (hYE : (funSpace U U).mem Y)
    {X : Set (ApproximableMap U U)} (hXx : x.mem X) (hXY : X ⊆ Y) (hXX : (piD d).rel X
     X) :
    ((piD d).toElementMap x).mem Y :=
  mem_of_exists_rel_self x hYE hXx hXY hXX

theorem subU_principal_eq_of_rel_self {T : Set ?} (hT : U.mem T) (hTT : subU.rel T T)
    :
    subU.toElementMap (U.principal hT) = U.principal hT := by
  apply le_antisymm
  · exact toElementMap_le_self_of_le_idMap subU.isProjection.2 _
  · exact principal_le_of_mem ((subU.rel_iff_mem_principal hT).mp hTT)
```

```
theorem exists_rel_self_subset_of_mem {t : U.Element} (ht : subU.toElementMap t = t)
    {T₀ : Set ?} (hT₀ : t.mem T₀) : ∃ T, subU.rel T T ∧ t.mem T ∧ T ⊆ T₀ := by
  have hmem : (subU.toElementMap t).mem T₀ := by rw [ht]; exact hT₀
  obtain ⟨_, T, hTt, hTT₀, hTT⟩ :=
    (formula_of_isFinitaryProjection subU.isFinitaryProjection t).mp hmem
  exact ⟨T, hTT, hTt, hTT₀⟩

theorem scFamily_directed {t : U.Element} (ht : subU.toElementMap t = t) :
    ∀ i j : scFamily t, ∃ k : scFamily t, scPrincipal i ≤ scPrincipal k ∧ scPrincipal
    j ≤ scPrincipal k :=
  fun i j => by
    obtain ⟨T, hTT, htT, hTsub⟩ := exists_rel_self_subset_of_mem ht (t.inter_mem i.2.2
    j.2.2)
    refine ⟨⟨T, hTT, htT⟩, ?_, ?_⟩
    · exact (U.principal_le_iff (t.sub i.2.2) (t.sub htT)).mpr (hTsub.trans Set.
    inter_subset_left)
    · exact (U.principal_le_iff (t.sub j.2.2) (t.sub htT)).mpr (hTsub.trans Set.
    inter_subset_right)

theorem eq_iSupDirected_scPrincipal {t : U.Element} (ht : subU.toElementMap t = t) :
    t = NeighborhoodSystem.iSupDirected scPrincipal (scFamily_directed ht) := by
  apply Element.ext
  intro Z
  rw [mem_iSupDirected]
  constructor
  · intro hZ
    obtain ⟨T, hTT, htT, hTZ⟩ := exists_rel_self_subset_of_mem ht hZ
    exact ⟨⟨T, hTT, htT⟩, (U.mem_principal _).mpr ⟨t.sub hZ, hTZ⟩⟩
  · rintro ⟨i, hi⟩
    obtain ⟨hZmem, hTZ⟩ := (U.mem_principal _).mp hi
    exact t.up_mem i.2.2 hZmem hTZ

theorem isFinitaryProjection_piD_of_formula (d : ApproximableMap U U)
    (hii : ∀ (x : (funSpace U U).Element) {Y}, ((piD d).toElementMap x).mem Y ↔
      (funSpace U U).mem Y ∧ ∃ X, x.mem X ∧ X ⊆ Y ∧ (piD d).rel X X) :
    FinitaryProjection (piD d) :=
  isFinitaryProjection_of_formula (piD d) hii

theorem le_idMap_piD (d : ApproximableMap U U) : piD d ≤ idMap (funSpace U U) := by
  rw [le_iff_toElementMap_le]
  intro φ
  rw [toElementMap_idMap, ← (funSpaceEquiv U U).le_iff_le, funSpaceEquiv_apply,
    funSpaceEquiv_apply, toApproxMap_toElementMap_piD, le_iff_toElementMap_le]
  intro t
  rw [toElementMap_piDApply]
  have hP := (isFinitaryProjection_decode_subU (d.toElementMap (subU.toElementMap t)))
    .out.1.2
  calc (toApproxMap (jArrow.toElementMap (subU.toElementMap
        (d.toElementMap (subU.toElementMap t))))).toElementMap
        ((toApproxMap φ).toElementMap (subU.toElementMap t))
      ≤ (toApproxMap φ).toElementMap (subU.toElementMap t) :=
        toElementMap_le_self_of_le_idMap hP _
    _ ≤ (toApproxMap φ).toElementMap t :=
        (toApproxMap φ).toElementMap_mono
          (toElementMap_le_self_of_le_idMap subU.isProjection.2 t)
```

```
theorem piDApply_toElementMap_eq (d f : ApproximableMap U U) (t : U.Element) :
    (piDApply d f).toElementMap (subU.toElementMap t) =
      (toApproxMap (jArrow.toElementMap (subU.toElementMap (d.toElementMap
          (subU.toElementMap t))))).toElementMap (f.toElementMap (subU.toElementMap t)
    ) := by
  rw [toElementMap_piDApply, toElementMap_idem_of_isRetraction subU.isProjection.1 t]

theorem isRetraction_piD (d : ApproximableMap U U) : Retraction (piD d) := by
  constructor
  apply ApproximableMap.ext_of_toElementMap
  intro φ
  apply (funSpaceEquiv U U).injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toElementMap_comp,
    toApproxMap_toElementMap_piD d ((piD d).toElementMap φ),
    toApproxMap_toElementMap_piD d φ]
  apply ApproximableMap.ext_of_toElementMap
  intro t
  rw [toElementMap_piDApply d (piDApply d (toApproxMap φ)) t,
    piDApply_toElementMap_eq d (toApproxMap φ) t,
    toElementMap_piDApply d (toApproxMap φ) t]
  exact toElementMap_idem_of_isRetraction
    (isFinitaryProjection_decode_subU (d.toElementMap (subU.toElementMap t))).out.1.1
    _

theorem isProjection_piD (d : ApproximableMap U U) : IsProjection (piD d) :=
  ⟨isRetraction_piD d, le_idMap_piD d⟩

theorem piDType_monotone (d : ApproximableMap U U) {s s' : U.Element} (h : s ≤ s') :
    piDType d s ≤ piDType d s' :=
  Sub8_6.toApproxMap_monotone ((piDTypeMap d).toElementMap_mono h)

theorem piD_rel_self_iff (d : ApproximableMap U U) {X : Set (ApproximableMap U U)} :
    (piD d).rel X X ↔ ∃ hX : (funSpace U U).mem X, gSection (piDUncurried d) hX ∈ X :=
     by
  rw [piD_rel_iff]
  exact ⟨fun ⟨hX, _, hmem⟩ => ⟨hX, hmem⟩, fun ⟨hX, hmem⟩ => ⟨hX, hX, hmem⟩⟩

theorem toElementMap_piDTypeMap (d : ApproximableMap U U) (s : U.Element) :
    (piDTypeMap d).toElementMap s = jArrow.toElementMap (subU.toElementMap (d.
    toElementMap s)) := by
  unfold piDTypeMap
  rw [toElementMap_comp, toElementMap_comp]

theorem isFinitaryProjection_piDType (d : ApproximableMap U U) (s : U.Element) :
    FinitaryProjection (piDType d s) := by
  unfold piDType
  rw [toElementMap_piDTypeMap]
  exact isFinitaryProjection_decode_subU (d.toElementMap s)

theorem toElementMap_piDApply' (d f : ApproximableMap U U) (t : U.Element) :
    (piDApply d f).toElementMap t =
      (piDType d (subU.toElementMap t)).toElementMap (f.toElementMap (subU.
    toElementMap t)) := by
  rw [toElementMap_piDApply, piDType, toElementMap_piDTypeMap]
```

```
theorem exists_X_of_mem_step (d : ApproximableMap U U) (x : (funSpace U U).Element)
    {Y₀ Z₀ : Set ?} (hmem : piDApply d (toApproxMap x) ∈ step Y₀ Z₀) :
    ∃ X, x.mem X ∧ X ⊆ (step Y₀ Z₀ : Set (ApproximableMap U U)) ∧ (piD d).rel X X :=
   by
  have hrelY0Z0 : (piDApply d (toApproxMap x)).rel Y₀ Z₀ := hmem
  have hY₀ : U.mem Y₀ := (piDApply d (toApproxMap x)).rel_dom hrelY0Z0
  have hZ₀ : U.mem Z₀ := (piDApply d (toApproxMap x)).rel_cod hrelY0Z0
  set t : U.Element := subU.toElementMap (U.principal hY₀) with ht_def
  have ht : subU.toElementMap t = t := toElementMap_idem_of_isRetraction subU.
    isProjection.1 _
  have hmem' : ((piDType d t).toElementMap ((toApproxMap x).toElementMap t)).mem Z₀ :=
   by
    have h := ((piDApply d (toApproxMap x)).rel_iff_mem_principal hY₀).mp hrelY0Z0
    rwa [toElementMap_piDApply'] at h
  obtain ⟨_, W, hW_mem, hWZ₀, hWW⟩ :=
    (formula_of_isFinitaryProjection (isFinitaryProjection_piDType d t)
      ((toApproxMap x).toElementMap t)).mp hmem'
  have hw_sup := eq_iSupDirected_scPrincipal ht

  rw [hw_sup, toElementMap_iSupDirected, mem_iSupDirected] at hW_mem
  obtain ⟨i₀, hi₀⟩ := hW_mem
  have hrelT0 : (toApproxMap x).rel i₀.1 W :=
    (rel_iff_mem_principal (toApproxMap x) (t.sub i₀.2.2)).mpr hi₀

  have hWW2 : ∃ i : scFamily t, (piDType d (scPrincipal i)).rel W W := by
    have h2 : (piDType d t).rel W W := hWW
    unfold piDType at h2
    rw [hw_sup, toElementMap_iSupDirected, Sub8_6.toApproxMap_rel_iSupDirected] at h2
    exact h2
  obtain ⟨i₁, hi₁⟩ := hWW2

  obtain ⟨k, hik0, hik1⟩ := scFamily_directed ht i₀ i₁
  have hksub0 : k.1 ⊆ i₀.1 := (U.principal_le_iff (t.sub i₀.2.2) (t.sub k.2.2)).mp hik0
  have hTfinal : U.mem k.1 := subU.rel_dom k.2.1
  have hWmem : U.mem W := (piDType d t).rel_dom hWW
  have hrelTfinal : (toApproxMap x).rel k.1 W :=
    (toApproxMap x).mono hrelT0 hksub0 subset_rfl hTfinal hWmem
  have hPfinal : (piDType d (scPrincipal k)).rel W W := piDType_monotone d hik1 W W hi₁

  have hTfinal_fix : subU.toElementMap (U.principal hTfinal) = U.principal hTfinal :=
    subU_principal_eq_of_rel_self hTfinal k.2.1
  set X : Set (ApproximableMap U U) := step k.1 W with hX_def
  have hXvalid : (funSpace U U).mem X := step_mem hTfinal hWmem
  refine ⟨X, ?_, ?_, ?_⟩
  ·
    exact toApproxMap_rel.mp hrelTfinal
  ·
    have hle : t ≤ U.principal hY₀ := toElementMap_le_self_of_le_idMap subU.
    isProjection.2 _
    have hY0sub : Y₀ ⊆ k.1 := ((U.mem_principal hY₀).mp (hle k.1 k.2.2)).2
    exact step_subset hY₀ hZ₀ hY0sub hWZ₀
  ·
    rw [piD_rel_self_iff]
    refine ⟨hXvalid, ?_⟩
    show (gSection (piDUncurried d) hXvalid).rel k.1 W
    rw [gSection_piDUncurried_rel_iff d hXvalid hTfinal]
```

```
    set g : ApproximableMap U U := toApproxMap ((funSpace U U).principal hXvalid) with
     hg_def
    show ((piDApply d g).toElementMap (U.principal hTfinal)).mem W
    rw [toElementMap_piDApply', hTfinal_fix]
    have hg : g.rel k.1 W := by
      rw [hg_def, toApproxMap_rel]
      exact ((funSpace U U).mem_principal hXvalid).mpr ⟨hXvalid, subset_rfl⟩
    have hgmem : (g.toElementMap (U.principal hTfinal)).mem W :=
      (rel_iff_mem_principal g hTfinal).mp hg
    exact mem_of_exists_rel_self (g.toElementMap (U.principal hTfinal))
      ((piDType d (U.principal hTfinal)).rel_dom hPfinal) hgmem subset_rfl hPfinal

theorem exists_X_of_mem_stepFun (d : ApproximableMap U U) (x : (funSpace U U).Element)
    (L : List (Set ? × Set ?)) (hmem : piDApply d (toApproxMap x) ∈ stepFun L) :
    ∃ X, x.mem X ∧ X ⊆ (stepFun L : Set (ApproximableMap U U)) ∧ (piD d).rel X X := by
  induction L with
  | nil =>
    refine ⟨(funSpace U U).master, x.master_mem, ?_, (piD d).master_rel⟩
    simp
  | cons p L ih =>
    rw [stepFun_cons] at hmem
    obtain ⟨hmemP, hmemL⟩ := hmem
    obtain ⟨Xp, hXpx, hXpsub, hXpXp⟩ := exists_X_of_mem_step d x hmemP
    obtain ⟨XL, hXLx, hXLsub, hXLXL⟩ := ih hmemL
    refine ⟨Xp ∩ XL, x.inter_mem hXpx hXLx, ?_, ?_⟩
    · rw [stepFun_cons]
      exact Set.inter_subset_inter hXpsub hXLsub
    · have hvalidP : (funSpace U U).mem Xp := x.sub hXpx
      have hvalidL : (funSpace U U).mem XL := x.sub hXLx
      have hvalidInter : (funSpace U U).mem (Xp ∩ XL) := x.sub (x.inter_mem hXpx hXLx)
      have h1 : (piD d).rel (Xp ∩ XL) Xp :=
        (piD d).mono hXpXp Set.inter_subset_left subset_rfl hvalidInter hvalidP
      have h2 : (piD d).rel (Xp ∩ XL) XL :=
        (piD d).mono hXLXL Set.inter_subset_right subset_rfl hvalidInter hvalidL
      exact (piD d).inter_right h1 h2

theorem hii_hard_direction (d : ApproximableMap U U) (x : (funSpace U U).Element)
    {Y : Set (ApproximableMap U U)} (hYE : (funSpace U U).mem Y)
    (hmem : piDApply d (toApproxMap x) ∈ Y) :
    ∃ X, x.mem X ∧ X ⊆ Y ∧ (piD d).rel X X := by
  obtain ⟨⟨L, _, rfl⟩, _⟩ := hYE
  exact exists_X_of_mem_stepFun d x L hmem

theorem hii_piD (d : ApproximableMap U U) (x : (funSpace U U).Element)
    {Y : Set (ApproximableMap U U)} :
    ((piD d).toElementMap x).mem Y ↔
      (funSpace U U).mem Y ∧ ∃ X, x.mem X ∧ X ⊆ Y ∧ (piD d).rel X X := by
  rw [piD_toElementMap_mem_iff]
  constructor
  · rintro ⟨hYE, hmem⟩
    exact ⟨hYE, hii_hard_direction d x hYE hmem⟩
  · rintro ⟨hYE, X, hXx, hXY, hXX⟩
    have := hii_easy_direction d x hYE hXx hXY hXX
    rwa [piD_toElementMap_mem_iff] at this

theorem isFinitaryProjection_piD (d : ApproximableMap U U) : FinitaryProjection (piD d
```

```
    ) :=
  isFinitaryProjection_piD_of_formula d (fun x => hii_piD d x)

end DomainTheory.NeighbourhoodSystems
```

4.9.73 Infinite product on D

This peer constructs, on the function-space domain, the projection that interprets a dependent product whose fibres are coded by elements of the universal domain.

Definition (`piDApply`). Given a code assignment d and a map $f : U \to U$, define a new map by first projecting the argument to the subdomain of type codes, applying d, decoding the resulting code as a finitary projection, and applying that projection to f at the projected argument.

Definition (`piDUncurried`). The uncurried dependent-product operator sends a pair (f, t) to the value obtained by decoding the projected code $d(\mathrm{sub}(t))$ and applying its projection to $f(\mathrm{sub}(t))$.

Definition (`piD`). The dependent-product projection on the function-space domain is the currying of the uncurried dependent-product operator.

Definition (`piDTypeMap`). For a code assignment d, the associated fibre map sends a universal-domain element through d, projects the result to a valid code, and decodes that code as an element of the function-space domain.

Definition (`piDType`). The fibre projection at s is the approximable map represented by the decoded fibre code at s.

`piD.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.subU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

noncomputable def piDApply (d f : ApproximableMap U U) : ApproximableMap U U :=
  (evalMap U U).comp (paired (jArrow.comp (subU.comp (d.comp subU))) (f.comp subU))

noncomputable def piDUncurried (d : ApproximableMap U U) : ApproximableMap (prod (
    funSpace U U) U) U :=
  (evalMap U U).comp
    (paired
      (jArrow.comp (subU.comp (d.comp (subU.comp (proj₁ (funSpace U U) U)))))
      ((evalMap U U).comp
        (paired (proj₀ (funSpace U U) U) (subU.comp (proj₁ (funSpace U U) U))))))

noncomputable def piD (d : ApproximableMap U U) : ApproximableMap (funSpace U U) (
    funSpace U U) :=
  curry (piDUncurried d)
```

```
noncomputable def piDTypeMap (d : ApproximableMap U U) : ApproximableMap U (funSpace U
    U) :=
  jArrow.comp (subU.comp d)

noncomputable def piDType (d : ApproximableMap U U) (s : U.Element) : ApproximableMap
   U U :=
  toApproxMap ((piDTypeMap d).toElementMap s)

end DomainTheory.NeighbourhoodSystems
```

4.9.74 Sub on U

This peer transports the projection onto finitary projections from the function-space domain to the universal domain, and identifies its fixed points with the continuous self-maps lying below the identity.

Theorem (`jArrow_comp_iArrow_apply`). Decoding an encoded function-space element returns that element.

Proof. The decoding map is a left inverse of the encoding map, so their composite acts as the identity. ■

Definition (`subU`). The universal subprojection is obtained by decoding a universal element as a function, projecting that function to the finitary projections, and encoding the result back into U.

Theorem (`subU.toElementMap_eq`). On an element w, the universal subprojection is exactly the encoding of the function-space subprojection applied to the decoding of w.

Proof. This is the elementwise form of the defining threefold composite. ■

Theorem (`subU.subApprox_fix_of_fix`). If w is fixed by the universal subprojection, then its decoded function is fixed by the function-space subprojection.

Proof. Decode the fixed-point equation and use that decoding after encoding is the identity. ■

Theorem (`subU.fix_of_subApprox_fix`). If a function-space element is fixed by the function-space subprojection, then its encoding is fixed by the universal subprojection.

Proof. Expand the universal subprojection, cancel decoding after encoding, and use the given fixed-point equation. ■

Definition (`subU.fixIso`). The fixed points of the universal subprojection are order-isomorphic to the fixed points of the function-space subprojection, by decoding in one direction and encoding in the other.

Definition (`subU.fixIsoSubFixed`). The fixed points of the universal subprojection are order-isomorphic to the approximable self-maps f satisfying $\mathrm{sub}(f) = f$.

Theorem (`subU.isRetraction_conjArrow`). Conjugating any retraction of the function-space domain by the encoding and decoding maps produces a retraction of U.

Proof. In the square of the conjugated map, the middle decoding–encoding pair cancels and the repeated retraction collapses by idempotence. ■

Theorem (`subU.le_idMap_conjArrow`). If a function-space endomap lies below the identity, then its conjugate on U also lies below the identity.

Proof. Monotonicity of composition bounds the conjugate by encoding after decoding, which itself lies below the identity on U. ■

Theorem (`subU.isProjection_conjArrow`). Conjugating a function-space projection by the encoding and decoding maps produces a projection of U.

Proof. The conjugate is a retraction by the preceding idempotence result and lies below the identity by the preceding order result. ■

Theorem (`subU.isProjection`). The universal subprojection is a projection.

Proof. The function-space subprojection is a projection, and the universal subprojection is its conjugate under encoding and decoding. ■

Theorem (`subU.isFinitary`). The universal subprojection is finitary.

Proof. Its fixed-point domain is order-isomorphic to that of the finitary function-space subprojection, hence to an effectively given domain. ■

Theorem (`subU.isFinitaryProjection`). The universal subprojection is a finitary projection.

Proof. It is both a projection and finitary by the two preceding results. ■

subU.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.subApprox
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_6
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

theorem jArrow_comp_iArrow_apply (v : (funSpace U U).Element) :
    jArrow.toElementMap (iArrow.toElementMap v) = v := by
  rw [show jArrow.toElementMap (iArrow.toElementMap v) = (jArrow.comp iArrow).
    toElementMap v from
    (toElementMap_comp jArrow iArrow v).symm, jArrow_comp_iArrow, toElementMap_idMap]

noncomputable def subU : ApproximableMap U U :=
  iArrow.comp (subApprox.comp jArrow)

namespace subU

theorem toElementMap_eq (w : U.Element) :
    ApproximableMap.toElementMap subU w =
      iArrow.toElementMap (ApproximableMap.toElementMap subApprox (jArrow.toElementMap
     w)) := by
  unfold subU
  rw [toElementMap_comp, toElementMap_comp]

theorem subApprox_fix_of_fix {w : U.Element}
    (hw : ApproximableMap.toElementMap subU w = w) :
    ApproximableMap.toElementMap subApprox (jArrow.toElementMap w) = jArrow.
    toElementMap w := by
  rw [toElementMap_eq] at hw
  have hstep := congrArg jArrow.toElementMap hw
  rwa [jArrow_comp_iArrow_apply] at hstep

theorem fix_of_subApprox_fix {φ : (funSpace U U).Element}
    (hφ : ApproximableMap.toElementMap subApprox φ = φ) :
    ApproximableMap.toElementMap subU (iArrow.toElementMap φ) = iArrow.toElementMap φ
    := by
  rw [toElementMap_eq, jArrow_comp_iArrow_apply, hφ]
```

```
noncomputable def fixIso :
    {w : U.Element // ApproximableMap.toElementMap subU w = w} ≃o
      {φ : (funSpace U U).Element // ApproximableMap.toElementMap subApprox φ = φ}
    where
  toFun w := ⟨jArrow.toElementMap w.1, subApprox_fix_of_fix w.2⟩
  invFun φ := ⟨iArrow.toElementMap φ.1, fix_of_subApprox_fix φ.2⟩
  left_inv w := by
    apply Subtype.ext
    show iArrow.toElementMap (jArrow.toElementMap w.1) = w.1
    have hw := toElementMap_eq w.1
    rw [w.2] at hw
    rw [subApprox_fix_of_fix w.2] at hw
    exact hw.symm
  right_inv φ := by
    apply Subtype.ext
    exact jArrow_comp_iArrow_apply φ.1
  map_rel_iff' := by
    intro w w'
    show jArrow.toElementMap w.1 ≤ jArrow.toElementMap w'.1 ↔ w.1 ≤ w'.1
    constructor
    · intro hle
      have h2 := iArrow.toElementMap_mono hle
      have e1 : iArrow.toElementMap (jArrow.toElementMap w.1) = w.1 := by
        have hw := toElementMap_eq w.1
        rw [w.2, subApprox_fix_of_fix w.2] at hw
        exact hw.symm
      have e2 : iArrow.toElementMap (jArrow.toElementMap w'.1) = w'.1 := by
        have hw' := toElementMap_eq w'.1
        rw [w'.2, subApprox_fix_of_fix w'.2] at hw'
        exact hw'.symm
      rwa [e1, e2] at h2
    · intro hle
      exact jArrow.toElementMap_mono hle

noncomputable def fixIsoSubFixed :
    {w : U.Element // ApproximableMap.toElementMap subU w = w} ≃o
      {f : ApproximableMap U U // sub f = f} :=
  fixIso.trans subApprox.fixIso

theorem isRetraction_conjArrow {H : ApproximableMap (funSpace U U) (funSpace U U)}
    (hH : Retraction H) : Retraction (iArrow.comp (H.comp jArrow)) := by
  constructor
  show (iArrow.comp (H.comp jArrow)).comp (iArrow.comp (H.comp jArrow)) =
    iArrow.comp (H.comp jArrow)
  rw [comp_assoc iArrow (H.comp jArrow) (iArrow.comp (H.comp jArrow)),
    comp_assoc H jArrow (iArrow.comp (H.comp jArrow)),
    ← comp_assoc jArrow iArrow (H.comp jArrow), jArrow_comp_iArrow, idMap_comp,
    ← comp_assoc H H jArrow, hH.out]

theorem le_idMap_conjArrow {H : ApproximableMap (funSpace U U) (funSpace U U)}
    (hH : H ≤ idMap (funSpace U U)) : iArrow.comp (H.comp jArrow) ≤ idMap U :=
  calc iArrow.comp (H.comp jArrow)
      ≤ iArrow.comp ((idMap (funSpace U U)).comp jArrow) :=
        comp_mono_gen le_rfl (comp_mono_gen hH le_rfl)
    _ = iArrow.comp jArrow := by rw [idMap_comp]
```

```
    _ ≤ idMap U := iArrow_comp_jArrow_le

theorem isProjection_conjArrow {H : ApproximableMap (funSpace U U) (funSpace U U)}
    (hH : IsProjection H) : IsProjection (iArrow.comp (H.comp jArrow)) :=
  ⟨isRetraction_conjArrow hH.1, le_idMap_conjArrow hH.2⟩

theorem isProjection : IsProjection subU :=
  isProjection_conjArrow Sub8_6.isProjection_subApprox

theorem isFinitary : IsFinitary subU := by
  obtain ⟨β, F, ⟨iso⟩⟩ := subApprox.isFinitary (E := U)
  exact ⟨β, F, ⟨fixIso.trans iso⟩⟩

theorem isFinitaryProjection : FinitaryProjection subU :=
  ⟨⟨isProjection, isFinitary⟩⟩

end subU

end DomainTheory.NeighbourhoodSystems
```

4.9.75 Infinite product on U

This peer transports the dependent-product projection from the function-space domain to the universal domain and proves that the transported map is again a finitary projection.

Definition (`piU`). For a code assignment d, the universal dependent-product map decodes an element of U as a function, applies the dependent-product projection there, and encodes the result back into U.

Theorem (`toElementMap.piU`). The value of the universal dependent-product map at w is the encoding of the function-space dependent-product projection applied to the decoding of w.

Proof. This is the elementwise form of the defining composite. ■

Theorem (`piD.fix_of_piU_fix`). If w is fixed by the universal dependent-product map, then its decoding is fixed by the function-space dependent-product projection.

Proof. Decode the fixed-point equation and cancel decoding after encoding. ■

Theorem (`piU.fix_of_piD_fix`). If a function-space element is fixed by the dependent-product projection, then its encoding is fixed by the universal dependent-product map.

Proof. Expand the transported map, cancel decoding after encoding, and apply the assumed fixed-point equation. ■

Definition (`piU.FixIso`). Decoding and encoding give an order isomorphism between the fixed points of the universal dependent-product map and those of the function-space dependent-product projection.

Theorem (`isFinitary.piU_of_isFinitary_piD`). If the function-space dependent-product projection is finitary, then the universal dependent-product map is finitary.

Proof. Their fixed-point domains are order-isomorphic, so an effective presentation of the former transfers to the latter. ■

Theorem (`isFinitary.piU_of_formula`). If membership in the image of the function-space dependent-product projection has the finite self-related-neighbourhood characterization, then the universal dependent-product map is finitary.

Proof. The characterization makes the function-space map a finitary projection; finitarity then transfers across the fixed-point order isomorphism. ■

Theorem (`isProjection.piU`). The universal dependent-product map is a projection.

Proof. The function-space dependent-product map is a projection, and conjugation by the universal encoding and decoding maps preserves projections. ■

Theorem (`isFinitaryProjection.piU`). The universal dependent-product map is a finitary projection.

Proof. It is a projection by conjugation, and it is finitary because the function-space dependent-product projection is finitary. ■

`piU.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    PolymorphicType

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

noncomputable def piU (d : ApproximableMap U U) : ApproximableMap U U :=
  iArrow.comp ((piD d).comp jArrow)

namespace toElementMap

theorem piU (d : ApproximableMap U U) (w : U.Element) :
    (DomainTheory.NeighbourhoodSystems.piU d).toElementMap w =
      iArrow.toElementMap ((piD d).toElementMap (jArrow.toElementMap w)) := by
  unfold DomainTheory.NeighbourhoodSystems.piU
  rw [toElementMap_comp, toElementMap_comp]

end toElementMap

namespace piD

theorem fix_of_piU_fix {d : ApproximableMap U U} {w : U.Element}
    (hw : (piU d).toElementMap w = w) :
    (piD d).toElementMap (jArrow.toElementMap w) = jArrow.toElementMap w := by
  rw [toElementMap.piU] at hw
  have hstep := congrArg jArrow.toElementMap hw
  rwa [jArrow_comp_iArrow_apply] at hstep

end piD

namespace piU

theorem fix_of_piD_fix {d : ApproximableMap U U} {φ : (funSpace U U).Element}
    (hφ : (piD d).toElementMap φ = φ) :
    (piU d).toElementMap (iArrow.toElementMap φ) = iArrow.toElementMap φ := by
  rw [toElementMap.piU, jArrow_comp_iArrow_apply, hφ]

noncomputable def FixIso (d : ApproximableMap U U) :
    {w : U.Element // (piU d).toElementMap w = w} ≃o
      {φ : (funSpace U U).Element // (piD d).toElementMap φ = φ} where
```

```
  toFun w := ⟨jArrow.toElementMap w.1, piD.fix_of_piU_fix w.2⟩
  invFun φ := ⟨iArrow.toElementMap φ.1, fix_of_piD_fix φ.2⟩
  left_inv w := by
    apply Subtype.ext
    show iArrow.toElementMap (jArrow.toElementMap w.1) = w.1
    have hw := toElementMap.piU d w.1
    rw [w.2] at hw
    rw [piD.fix_of_piU_fix w.2] at hw
    exact hw.symm
  right_inv φ := by
    apply Subtype.ext
    exact jArrow_comp_iArrow_apply φ.1
  map_rel_iff' := by
    intro w w'
    show jArrow.toElementMap w.1 ≤ jArrow.toElementMap w'.1 ↔ w.1 ≤ w'.1
    constructor
    · intro hle
      have h2 := iArrow.toElementMap_mono hle
      have e1 : iArrow.toElementMap (jArrow.toElementMap w.1) = w.1 := by
        have hw := toElementMap.piU d w.1
        rw [w.2, piD.fix_of_piU_fix w.2] at hw
        exact hw.symm
      have e2 : iArrow.toElementMap (jArrow.toElementMap w'.1) = w'.1 := by
        have hw' := toElementMap.piU d w'.1
        rw [w'.2, piD.fix_of_piU_fix w'.2] at hw'
        exact hw'.symm
      rwa [e1, e2] at h2
    · intro hle
      exact jArrow.toElementMap_mono hle

end piU

namespace isFinitary

theorem piU_of_isFinitary_piD {d : ApproximableMap U U} (h : IsFinitary (piD d)) :
    IsFinitary (piU d) := by
  obtain ⟨β, F, ⟨iso⟩⟩ := h
  exact ⟨β, F, ⟨(piU.FixIso d).trans iso⟩⟩

theorem piU_of_formula {d : ApproximableMap U U}
    (hii : ∀ (x : (funSpace U U).Element) {Y}, ((piD d).toElementMap x).mem Y ↔
      (funSpace U U).mem Y ∧ ∃ X, x.mem X ∧ X ⊆ Y ∧ (piD d).rel X X) :
    IsFinitary (piU d) :=
  piU_of_isFinitary_piD (isFinitaryProjection_piD_of_formula d hii).out.2

end isFinitary

namespace isProjection

theorem piU (d : ApproximableMap U U) : IsProjection (DomainTheory.
    NeighbourhoodSystems.piU d) :=
  subU.isProjection_conjArrow (isProjection_piD d)

end isProjection

namespace isFinitaryProjection
```

```
theorem piU (d : ApproximableMap U U) : FinitaryProjection (DomainTheory.
    NeighbourhoodSystems.piU d) :=
  ⟨⟨isProjection.piU d, isFinitary.piU_of_isFinitary_piD (isFinitaryProjection_piD d).
    out.2⟩⟩

end isFinitaryProjection

end DomainTheory.NeighbourhoodSystems
```

4.9.76 Exercise 8.27 — infinite cartesian products

(Suggested by James Donahue.)

Finite cartesian products of domains are formed by the $D_0 \times D_1$-construct we have used so often. The problem is to define — computably — some *infinite* cartesian products. In particular, as applied to the universal domain U, the combinator `sub` is to be regarded as a finitary projection of U whose fixed points are exactly *all* the finitary projections. A map

$$d = \mathrm{sub} \circ d \circ \mathrm{sub}$$

can be regarded as a *polymorphic type* (because, whenever t is a finitary projection (= type), then so is $d(t)$). The *continuous product* of *all* these types would be the domain of all approximable functions x such that

$$x(t) = d(t)(x(t))$$

for all types t. (Why does this equation mean that x is in the product?) Define Π as a combinator by

$$\Pi = \lambda d\, \lambda x\, \lambda t.\, \mathrm{sub}(d(\mathrm{sub}(t)))(x(\mathrm{sub}(t))).$$

Show that for d a polymorphic type, $\Pi(d)$ is a type. (Hint: It is easy to check that $\Pi(d)$ is a projection; the problem is to show it is *finitary*.)

The development is with the Infinite product on U peer.

Theorem (`example_827`). The infinite cartesian product of a projection is again a finitary projection.

Proof. By assembling the peer lemmas named in the packaging module. ■

`Exercise_8_27.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.piU

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

theorem example_827 (d : ApproximableMap U U) : FinitaryProjection (piU d) :=
  isFinitaryProjection.piU d
```

5 Information Systems (1982) ✓

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

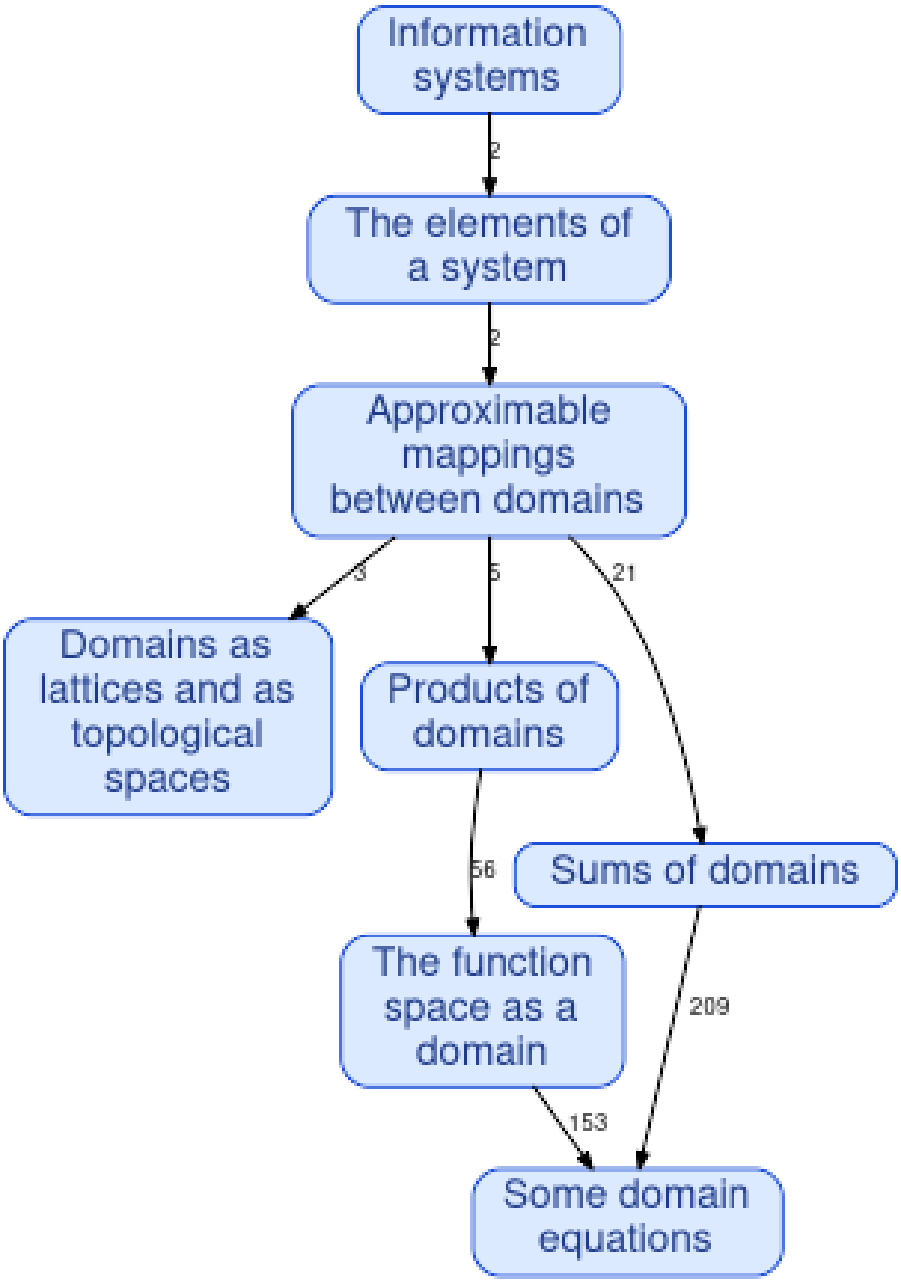


Figure 54: Diagram 19

5.1 Introduction

The purpose of the theory of domains is to give models for spaces on which to define computable functions. The kinds of spaces needed for denotational semantics involve not only spaces of higher type (e.g. function spaces) but also spaces defined recursively (e.g. reflexive domains). Also required are many special domain constructs (or functors) in order to create the desired structures. There are several choices of a suitable category of domains, but the basic one which has the simplest properties is the one sometimes called *consistently complete algebraic* cpo*'s*. This category of domains is studied in this paper from a new, and it is to be hoped, simpler point of view incorporating the approaches of many authors into a unified presentation. Briefly, the domains of elements are represented set theoretically with the aid of structures called *information systems*. These systems are very familiar from mathematical logic, and their use seems to accord well with intuition. Many things that were done previously axiomatically can now be proved in a straightforward way as theorems. The present paper discusses many examples in an informal way that should serve as an introduction to the subject.

1. Introduction. I would like to begin with some personal remarks. When I think of the number of headaches I have caused people in Computer Science who have tried to figure out the mathematical details of the Theory of Domains, I have to cringe. The difficulty in the presentation of the subject is in justifying the level of abstraction used in comparison with the payoff: too often the effort needed for understanding the abstractions does not seem worth the trouble—especially if the notions are unfamiliar or excessively general.

For example, Category Theory, which is much used in discussions of the Theory of Domains, seems far too abstract to many people. On the other hand, Automata Theory—which has often benefited from some use of Category Theory—is quite abstract and, in many aspects perhaps, quite useless, but the level of abstraction is fairly low and the reasons for the definitions are usually evident. Thus, the subject has become by now a standard topic even for undergraduate courses. I believe that Michael Rabin and I had much to do with influencing the subsequent development of the theory by formulating some years ago some basic concepts (many previously known) in a way that made sense to anyone who had been exposed to a little Abstract Algebra, Logic and Set Theory. This expository part of our paper did not require originality, but rather a certain frame of mind and a certain style of writing to put the ideas and problems in sharp relief. (The original paper is [**?**]; see the excellent historical review of the subject in [**?**].)

Domain Theory has fared less well. I feel I made a mistake in 1969 in using Lattice Theory as a mode of presentation—a mistake as far as Computer Science is concerned. Lattices are very familiar to logicians and those interested in Universal Algebra, and I liked very much the structures I found. But it takes some time to learn the special terminology and to become comfortable with the necessary examples. Indeed, without a stock of examples, it is impossible to have sufficient intuition for making the required constructions. True, some people took to the approach, and the lattice-theoretic definitions were simple enough to motivate. But a much

greater percentage of people I came in contact with found the large number of things to learn a definite roadblock to understanding. And the Lattice Theory was only partly standard, since this approach to the theory has to employ complete lattices and topological notions in order to explain *limits* and *continuity* adequately. In fact, Topology of a rather mathematically simple set-theoretical kind could have been used as the whole foundation much as in [1], but that kind of mathematics seems even less attractive to computer scientists. Again, it is a matter of background, I think, and it is unreasonable to expect people to know everything. Nevertheless, the lattice-theoretic approach has great consistency and coherence; it has been carried out in full detail in [**?**], which also gives the topological connections. That book contains a large number of references to the mathematical literature to show the connections of the theory to many other

topics. What we had to miss out there is a clear hook-up with computability theory, owing simply to a lack of space and time to be able to cover everything.

In [2], I tried out another, different presentation (which is also explained in [3].) However, I have found that the use of *neighborhood systems* causes confusion in people not used to thinking in terms of sets of sets of sets of sets . . . , even though the set theory required is quite simple. My purpose in the present lecture, then, is to go back once more to the very beginning to try out another plan for making the story easy and natural. The notion to be used I call an *information system*. This is a "static" notion appropriate to bodies of information about coherent groups of elements. The "dynamic" ideas enter in the ways different systems can be related and in the semantical definitions about meanings of programming language constructs. (For discussions of semantical definitions see [**?**], [**?**], and [**?**].) Neighborhood systems are, in a precise sense, equivalent to information systems, but there seems to be an interesting trade-off between what properties of structures are *axiomatized* and what are *implicit* in the form of the structure. Perhaps, despite their simplicity, neighborhoods leave too much implicit, too much to be extracted by definition; information systems, on the other hand, do seem to be more flexible in doing several of the important constructions. It all comes down to just where one imposes the closure conditions on the structures. I have a feeling that I now have found just about the right mix of axioms *vs.* definitions.

Another word about Category Theory: I actually feel that it is particularly significant and important for the theory and for the whole area of semantics. But it must be approached with great caution, for the sheer number of definitions and axioms can try the most patient reader. It seems to me to be especially necessary in discussing applications of abstract mathematical ideas to keep the motivation strongly in mind. This is often hard to do if the categories get too thick, but of course it all depends on the writer. Category Theory is especially useful in stating *general* properties of structures and in *characterizing* constructions uniquely; however, there often is a problem actually justifying the *existence* of certain constructions, and a direct approach can be quicker than quoting lots of theorems. But, man cannot live by construction alone: theorems have to be proved in order to get the proper value out of the work. Domain Theory must also be convenient for demonstrating the soundness of various proof rules for properties of recursively defined objects and recursively defined domains, and I think that Category Theory can be helpful here. A step in the right direction has been made in the LCF system (see [**?**]), which, however, does not take advantage of general Category Theory; but the whole area needs much more development in my opinion.

The main purpose of the Theory of Domains in Denotational Semantics, as I see it, is to give a mathematical model of a system of *types*—including function-space types—and to explain

a notion of computability with respect to these types. There are many possible theories of types, but the construction of domains is meant to justify recursive definitions at all types, and—most essentially—to justify recursive definitions of types. Pursuing these goals has certain consequences, as we shall see. The model presented here is only one approach, and some comparisons with other methods will be made in Section 8. Again some help from Category Theory would be welcome in comparing the different kinds of modelling.

One benefit of Domain Theory is that it is possible to make sense of types containing *infinite* elements. The cost required seems to be that the domains therefore have to contain *partial* elements as well as *total* elements. And this is where the lattice-theoretic definitions entered in the earlier presentations: the partial ordering with relations like $x \sqsubseteq y$ was used to express the fact that the element x was "less defined" (i.e. more partial) than y but "contained in" y. The relation $\sqsubseteq$, however, must be subjected to many axioms to have a theory suitable to the desired applications. Instead of such a big group of axioms, I wish to put forward here a construction where the "skeleton" (or better: "backbone") of a domain is introduced by just a few axioms. Then the domain itself is defined as a certain construct from the backbone in order to define

the appropriate notion of element. There are several advantages I can see to the new approach:

- Simple definitions of the basic concepts can be given.
- Detailed properties are proved as theorems rather than assumed as axioms.
- Emphasis can be given to the constructive nature of the definitions.
- Domains can be made more visible.
- The theory of domains is made more available for applications, because it is easier to produce the needed complex domains.

It is to be hoped that the reader can judge the validity of these claims from the perhaps too brief exposition to follow.

5.2 Information systems

<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

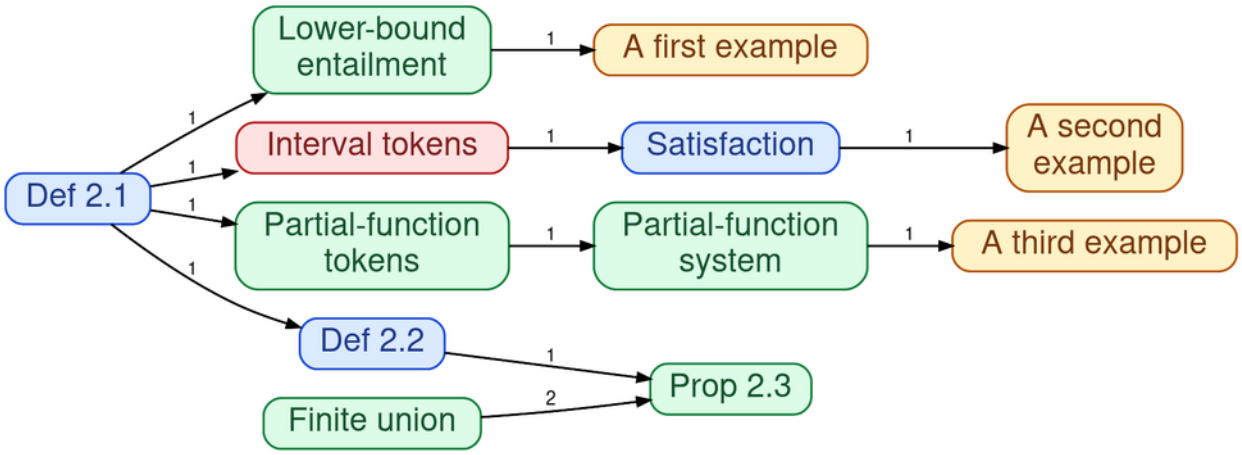


Figure 55: Diagram 20

5.2.1 Introduction

2. Information systems. Intuitively, an information system is a set of "propositions" that can be made about "possible elements" of the desired domain. We will assume that sufficiently many propositions have been supplied to distinguish between distinct elements; as a consequence, an element can be constructed abstractly as the set of all the propositions that are true of it. Partial elements have "small" sets; while total elements have "large" sets (even: maximal). To make this somewhat rough idea precise, we have to explain—by a suitable, but small choice of axioms—how the collection of all propositions relevant to the domain hangs together, or better, is structured as a set of abstract propositions. Fortunately, the axioms for this structure are very simple and familiar, which is a great help in making up examples.

5.2.2 Definition 2.1

DEFINITION 2.1. An *information system* is a structure

$$(D, \Delta, \mathrm{Con}, \vdash),$$

where D is a set (the set of data objects or propositions), where Δ is a distinguished member of D (the least informative member), where Con is a set of finite subsets of D (the consistent sets of objects), and where $\vdash$ is a binary relation between members of Con and members of D (the

entailment relation for objects). Concerning Con, the following axioms must be satisfied for all finite subsets $u, v \subseteq D$:

(i) $u \in \mathrm{Con}$, whenever $u \subseteq v \in \mathrm{Con}$;

(ii) $\{X\} \in \mathrm{Con}$, whenever $X \in D$; and

(iii) $u \cup \{X\} \in \mathrm{Con}$, whenever $u \vdash X$.

Concerning $\vdash$, the following axioms must be satisfied for all $u, v \in \mathrm{Con}$, and all $X \in D$:

(iv) $u \vdash \Delta$;

(v) $u \vdash X$, whenever $X \in u$; and

(vi) if $v \vdash Y$ for all $Y \in u$ and $u \vdash X$, then $v \vdash X$.

In words we may say of Con that, as a set of sets, (i) it is closed under subsets, (ii) it contains all singletons, and (iii) adjunction of an entailed object to a consistent set preserves consistency. Concerning $\vdash$, which should be viewed as a "multiary" relation, we may say that (iv) Δ is entailed by anything, (v) $\vdash$ is reflexive, and (vi) $\vdash$ is transitive. (The last two properties are both expressed in a way appropriate to a multiary relation on members of D.) ?

The best advice is to think of the members of D as consisting of finite data objects, some of which are more informative than others. The word "finite" should be taken here in the sense of "fully circumscribed"—as regards what is given in **A** these data objects can be comprehended in "one step." It is of course possible to introduce information systems where the data objects are infinite sets, but relative to **A** they are finite as data objects.

The member that provides zero information is Δ. Data objects are intended to give information about possible elements of the domain to be constructed, so if we were to use Δ alone, we would be describing the least defined element usually written as $\perp$.

Not just any combination of data objects will describe a possible element, however; hence, the need for the notion of consistency. If $u \in \mathrm{Con}$ is false, then the "propositions" in u cannot all be applied to the same element at the same time. Finally we have to agree that, in general, the propositions to be allowed are rarely mutually independent. It follows that an entailment relation must be imposed to record the dependencies that do hold among the propositions. With these informal understandings, the axioms chosen should all be self-evident.

Definition (`InfoSys`). An information system on a carrier α consists of a distinguished least-informative datum $\Delta \in \alpha$, a collection Con of finite subsets of α (the consistent sets), and an entailment relation $\vdash$ between consistent sets and data, subject to Scott's six axioms: Con is downward closed under subsets and contains every singleton; if $u \vdash X$ then $u \cup \{X\}$ is consistent; every consistent u entails Δ;

membership in u yields entailment from u; and entailment is transitive in the multiary sense—if v entails every member of u and $u \vdash X$, then $v \vdash X$. Here Δ is the bottom datum, Con and $\vdash$ are the consistency and entailment fields, and the six named axioms match clauses (i)–(vi) of Definition 2.1.

InfoSys.lean

```
module

public import Mathlib.Data.Finset.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u

namespace DomainTheory.InformationSystems

structure InfoSys (α : Type u) [DecidableEq α] where

  bot : α
  Con : Set (Finset α)
  Ent : Finset α → α → Prop
  con_subset : ∀ {u v : Finset α}, u ∈ Con → v ⊆ u → v ∈ Con
  con_sing : ∀ a : α, {a} ∈ Con
  ent_con : ∀ {u : Finset α} {a : α}, Ent u a → insert a u ∈ Con
  ent_bot : ∀ {u : Finset α}, u ∈ Con → Ent u bot
  ent_refl : ∀ {u : Finset α} {a : α}, u ∈ Con → a ∈ u → Ent u a
  ent_trans : ∀ {u v : Finset α} {c : α},
    v ∈ Con → u ∈ Con → (∀ y ∈ u, Ent v y) → Ent u c → Ent v c

end DomainTheory.InformationSystems
```

con_empty.lean

```
module

public import Mathlib.Data.Finset.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.InfoSys

@[expose] public section

namespace DomainTheory.InformationSystems

universe u

namespace InfoSys

variable {α : Type u} [DecidableEq α] (sys : InfoSys α)

theorem con_empty : (∅ : Finset α) ∈ sys.Con :=
  sys.con_subset (sys.con_sing sys.bot) (Finset.empty_subset _)

end InfoSys
end DomainTheory.InformationSystems
```

InfoSys.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys.InfoSys
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty

@[expose] public section
```

5.2.3 Lower-bound entailment

Definition (`lowerBoundEnt`). For finite sets u of natural numbers and a natural number m, write u lowerBoundEnt m when either $m = 0$ or some $n \in u$ satisfies $m \leq n$. Reading each n as the proposition $n \leq x$, this is the entailment relation for Scott's first example: the empty-information datum 0 is always entailed, and otherwise a lower bound is entailed when some listed bound already meets or exceeds it.

lowerBoundEnt.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys

@[expose] public section

namespace DomainTheory.InformationSystems

def lowerBoundEnt (u : Finset ℕ) (m : ℕ) : Prop :=
  m = 0 ∨ ∃ n ∈ u, m ≤ n

end DomainTheory.InformationSystems
```

5.2.4 A first example

A first example. Suppose we let D be the set of non-negative integers, where we think of an integer n as an abbreviation of the proposition $n \leq x$. Here x is a yet-to-be-determined element, about which one proposition gives only a little information. We can identify Δ with 0, and take Con to be the set of all finite subsets of D. The entailment relation can be defined formally in the way suggested by the intuitive reading of the data objects:

$$\{n_0, \ldots, n_{k-1}\} \vdash m \quad \text{iff either } m = 0 \text{ or } m \leq n_i \text{ for some } i < k.$$

(Remember to think of the same possible x on both sides of the $\vdash$.) That $\vdash$ is an entailment relation in the sense of our axioms is clear. ?

Definition (`a_first_example`). Scott's first example is the information system on the natural numbers with least-informative datum 0, every finite subset consistent, and entailment given by lower-bound entailment: a finite u entails m when $m = 0$ or some $n \in u$ satisfies $m \leq n$. The six information-system axioms hold for this choice of consistency and entailment.

a_first_example.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.InformationSystems.lowerBoundEnt

@[expose] public section

namespace DomainTheory.InformationSystems

def a_first_example : InfoSys ℕ where
  bot := 0
  Con := Set.univ
  Ent := lowerBoundEnt
  con_subset := by
    intro u v _ _
    exact Set.mem_univ v
  con_sing := by
    intro a
    exact Set.mem_univ _
  ent_con := by
    intro u a _
    exact Set.mem_univ _
  ent_bot := by
    intro u _
    exact Or.inl rfl
  ent_refl := by
    intro u a _ ha
    exact Or.inr ⟨a, ha, le_rfl⟩
  ent_trans := by
    intro u v c _ _ hvEnt huEnt
    rcases huEnt with rfl | ⟨n, hn, hcn⟩
    · exact Or.inl rfl
    ·
      rcases hvEnt n hn with rfl | ⟨k, hk, hnk⟩
      ·
        exact Or.inl (Nat.le_zero.mp hcn)
      · exact Or.inr ⟨k, hk, le_trans hcn hnk⟩

end DomainTheory.InformationSystems
```

5.2.5 Interval tokens

Definition (`Token`). The data objects for Scott's second example are tokens of two kinds: a least-informative datum $\bot$, and open intervals (n, m) of natural numbers with $n < m$, standing for the proposition $n < x < m$.

Definition (`Token.instDecidableEq`). Equality of tokens is decidable: the bottom token equals only itself, and two intervals are equal precisely when they share the same endpoints.

`Token.lean`

```
module

public import Mathlib.Algebra.Order.Field.Rat
public import Mathlib.Tactic.Linarith
```

```
public import Mathlib.Tactic.NormNum
public import DomainTheory.InformationSystems.InformationSystems.InfoSys

@[expose] public section

namespace DomainTheory.InformationSystems

inductive Token : Type where
  | bot
  | interval (lo hi : ℕ) (hlt : lo < hi)

namespace Token

instance instDecidableEq : DecidableEq Token
  | bot, bot => isTrue rfl
  | bot, interval _ _ _ => isFalse Token.noConfusion
  | interval _ _ _, bot => isFalse Token.noConfusion
  | interval n m _, interval n' m' _ =>
    if h : n = n' ∧ m = m' then
      isTrue (by
        rcases h with ⟨rfl, rfl⟩
        rfl)
    else
      isFalse (by
        intro hEq
        injection hEq with hn hm
        exact h ⟨hn, hm⟩)

end Token

end DomainTheory.InformationSystems
```

5.2.6 Satisfaction

Definition (`Sat`). A rational x satisfies the bottom token, and satisfies an interval token (n, m) precisely when $n < x < m$.

Theorem (`sat_midpoint`). The midpoint of an open interval of natural numbers satisfies that interval token.

Proof. The midpoint lies strictly between the endpoints by the ordering of the rationals. ?

Theorem (`sat_sing`). Every token is satisfied by some rational: the bottom token by 0, and each interval by its midpoint.

Proof. The bottom case is immediate; the interval case is the midpoint theorem. ?

Definition (`ofSatisfaction`). Given a satisfaction relation between points and tokens with a least-informative token always satisfied and every token satisfied by some point, form an information system whose consistent sets are those realized by a common point and whose entailment holds when every point realizing the antecedents realizes the conclusion. The six information-system axioms follow from these reading of consistency and entailment.

`Sat.lean`

```
module
```

```
public import Mathlib.Algebra.Order.Field.Rat
public import Mathlib.Tactic.Linarith
public import Mathlib.Tactic.NormNum
public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.InformationSystems.Token

@[expose] public section

namespace DomainTheory.InformationSystems

def Sat (x : ?) : Token → Prop
  | .bot => True
  | .interval lo hi _ => (lo : ?) < x ∧ x < (hi : ?)

theorem sat_midpoint {lo hi : ℕ} (hlt : lo < hi) :
    Sat (((lo : ?) + (hi : ?)) / 2) (.interval lo hi hlt) := by
  have h2 : (0 : ?) < 2 := by norm_num
  have _hlo : (lo : ?) < (hi : ?) := Nat.cast_lt.mpr hlt
  constructor
  ·
    rw [lt_div_iff₀ h2]
    linarith
  · rw [div_lt_iff₀ h2]
    linarith

theorem sat_sing (t : Token) : ∃ x : ?, Sat x t := by
  cases t with
  | bot => exact ⟨0, trivial⟩
  | interval lo hi hlt => exact ⟨_, sat_midpoint hlt⟩

def ofSatisfaction {τ : Type*} [DecidableEq τ] {Point : Type*}
    (Sat : Point → τ → Prop) (bot : τ)
    (sat_bot : ∀ p, Sat p bot)
    (sat_sing : ∀ t, ∃ p, Sat p t) : InfoSys τ where
  bot := bot
  Con := {u | ∃ p, ∀ t ∈ u, Sat p t}
  Ent u X := (∃ p, ∀ t ∈ u, Sat p t) ∧ ∀ p, (∀ t ∈ u, Sat p t) → Sat p X
  con_subset := by
    rintro u v ⟨p, hp⟩ hsub
    exact ⟨p, fun t ht => hp t (hsub ht)⟩
  con_sing := by
    intro a
    obtain ⟨p, hp⟩ := sat_sing a
    exact ⟨p, fun t ht => by
      rw [Finset.mem_singleton] at ht
      subst ht
      exact hp⟩
  ent_con := by
    rintro u a ⟨⟨p, hp⟩, hEnt⟩
    exact ⟨p, fun t ht => by
      rcases Finset.mem_insert.mp ht with rfl | ht'
      · exact hEnt p hp
      · exact hp t ht'⟩
  ent_bot := by
    intro u hu
    exact ⟨hu, fun p _ => sat_bot p⟩
```

```
  ent_refl := by
    intro u a hu ha
    exact ⟨hu, fun p hp => hp a ha⟩
  ent_trans := by
    intro u v c hv _hu hvu ⟨_, huEnt⟩
    refine ⟨hv, fun p hp => huEnt p ?_⟩
    intro y hy
    exact (hvu y hy).2 p hp

end DomainTheory.InformationSystems
```

5.2.7 A second example

A second example. The first example is possibly misleading because all (finite) sets of data objects were allowed to be consistent. The example can be modified in a natural way so this is not so; of course, quite a different system will be obtained. The idea is to let D be the set of all pairs (n, m) of integers with $n < m$, where such a pair stands now for the proposition $n < x < m$. Clearly the two data objects $(0, 2)$ and $(3, 7)$ taken together are inconsistent. (Why?)

Oh, I see, I have left out Δ from D; we must therefore include the somewhat artificial pair $(0, \infty)$ with the obvious interpretation. When I say "obvious interpretation" here I mean that $u \in \mathrm{Con}$ can be defined by saying that there must be an integer satisfying all the "propositions" in u. Further, $u \vdash X$ can be defined by saying that whenever an integer satisfies all the propositions in u, then it must satisfy X. The notion of "satisfy" really should be obvious from the intuitive reading given to the data objects, and the reader can verify easily that all the axioms hold for this example of an information system. ?

Definition (`a_second_example`). Scott's second example is the information system on interval tokens obtained from rational satisfaction via the satisfaction construction: the least-informative datum is the bottom token, consistency and entailment are those induced by whether a common rational realizes the tokens, and the six axioms hold by that construction together with the fact that every token is satisfied by some rational.

`a_second_example.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Sat

@[expose] public section

namespace DomainTheory.InformationSystems

def a_second_example : InfoSys Token :=
  ofSatisfaction Sat Token.bot (fun _ => trivial) sat_sing

end DomainTheory.InformationSystems
```

5.2.8 Partial-function tokens

Definition (`PartialFunToken`). For sets A and B, a partial-function token is either a least-informative datum $\bot$, or a pair (a, b) with $a \in A$ and $b \in B$, standing for the proposition that a yet-to-be-determined function sends a to b. Equality of such tokens is decidable.

PartialFunToken.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys

@[expose] public section

namespace DomainTheory.InformationSystems

inductive PartialFunToken (A B : Type*) where
  | bot
  | pair (a : A) (b : B)
  deriving DecidableEq

end DomainTheory.InformationSystems
```

5.2.9 Partial-function system

Definition (`PartialFunConsistent`). A finite set u of partial-function tokens is consistent when it does not assign two distinct values to the same argument: whenever (a, b) and (a, b') both lie in u, one has $b = b'$.

Definition (`PartialFunEnt`). A consistent finite set u of partial-function tokens entails a token X precisely when X is the bottom token or X already belongs to u.

Theorem (`consistent_empty`). The empty set of partial-function tokens is consistent.

Proof. No pair belongs to the empty set, so the functional condition holds vacuously. ?

Theorem (`consistent_subset`). Consistency is downward closed under subsets.

Proof. Any conflicting pair in a subset would already conflict in the larger set. ?

Theorem (`consistent_singleton`). Every singleton of partial-function tokens is consistent.

Proof. The bottom token contributes no pairs; a single pair cannot conflict with a distinct value for the same argument. ?

Theorem (`consistent_insert_bot`). Inserting the bottom token into a consistent set preserves consistency.

Proof. Bottom is not a pair, so membership of pairs is unchanged. ?

Theorem (`consistent_insert_mem`). Inserting a token already present in a consistent set preserves consistency.

Proof. The insert does not enlarge the set. ?

Definition (`partialFunctionSystem`). For decidable A and B, the partial-function information system on partial-function tokens has least-informative datum $\bot$, consistency given by partial-function consistency, and entailment given by partial-function entailment. The six information-system axioms hold by the consistency lemmas above together with the minimal reading of entailment.

partialFunctionSystem.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.PartialFunToken
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.InfoSys

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

def PartialFunConsistent {A B : Type*} (u : Finset (PartialFunToken A B)) : Prop :=
  ∀ {{a : A}} {{b b' : B}},
    PartialFunToken.pair a b ∈ u → PartialFunToken.pair a b' ∈ u → b = b'

def PartialFunEnt {A B : Type*} (u : Finset (PartialFunToken A B)) (X :
    PartialFunToken A B) : Prop :=
  PartialFunConsistent u ∧ (X = PartialFunToken.bot ∨ X ∈ u)

theorem consistent_empty {A B : Type*} :
    PartialFunConsistent (∅ : Finset (PartialFunToken A B)) := by
  intro _a _b _b' hb _
  exact False.elim (Finset.notMem_empty _ hb)

theorem consistent_subset {A B : Type*} {u v : Finset (PartialFunToken A B)} (huv : v
    ⊆ u)
    (hu : PartialFunConsistent u) : PartialFunConsistent v :=
  fun _a _b _b' hb hb' => hu (huv hb) (huv hb')

theorem consistent_singleton {A B : Type*} [DecidableEq A] [DecidableEq B]
    (t : PartialFunToken A B) : PartialFunConsistent ({t} : Finset (PartialFunToken A
    B)) := by
  intro a b b' hb hb'
  rw [Finset.mem_singleton] at hb hb'
  cases t with
  | bot =>
    cases hb
  | pair a0 b0 =>
    cases hb
    cases hb'
    rfl

theorem consistent_insert_bot {A B : Type*} [DecidableEq A] [DecidableEq B]
    {u : Finset (PartialFunToken A B)} (hu : PartialFunConsistent u) :
    PartialFunConsistent (insert (PartialFunToken.bot : PartialFunToken A B) u) := by
  intro a b b' hb hb'
  have hb1 : PartialFunToken.pair a b ∈ u := by
    rcases Finset.mem_insert.mp hb with h | h
    · cases h
    · exact h
  have hb2 : PartialFunToken.pair a b' ∈ u := by
    rcases Finset.mem_insert.mp hb' with h | h
    · cases h
    · exact h
  exact hu hb1 hb2

theorem consistent_insert_mem {A B : Type*} [DecidableEq A] [DecidableEq B]
    {u : Finset (PartialFunToken A B)} {t : PartialFunToken A B}
    (ht : t ∈ u) (hu : PartialFunConsistent u) : PartialFunConsistent (insert t u) :=
    by
  simpa [Finset.insert_eq_of_mem ht] using hu

def partialFunctionSystem (A B : Type*) [DecidableEq A] [DecidableEq B] :
    InfoSys (PartialFunToken A B) where
```

```
  bot := PartialFunToken.bot
  Con := {u | PartialFunConsistent u}
  Ent := PartialFunEnt
  con_subset := by
    intro u v hu hv
    exact consistent_subset hv hu
  con_sing := fun t => consistent_singleton t
  ent_con := by
    rintro u a ⟨hu, hX⟩
    rcases hX with rfl | hmem
    · exact consistent_insert_bot hu
    · exact consistent_insert_mem hmem hu
  ent_bot := by
    intro u hu
    exact ⟨hu, Or.inl rfl⟩
  ent_refl := by
    intro u a hu ha
    exact ⟨hu, Or.inr ha⟩
  ent_trans := by
    intro u v c hv _hu hvu ⟨_, hc⟩
    refine ⟨hv, ?_⟩
    rcases hc with rfl | hc
    · exact Or.inl rfl
    · have ⟨_, hcv⟩ := hvu c hc
      exact hcv

end DomainTheory.InformationSystems
```

5.2.10 A third example

A third example. Let A and B be two fixed sets, and let D as a set consist of the ordered pairs (a, b), with $a \in A$ and $b \in B$, plus the extra object Δ. Here the "information" contained in (a, b) is that a is mapped to b by a yet-to-be-determined function. With this thought in mind, we will know that a finite set of data objects is consistent just in case it is possible that they can all belong to the same function. Formally, we can assert:

$$\{(a_0, b_0), \ldots, (a_{k-1}, b_{k-1})\} \in \mathrm{Con} \text{ iff for all } i, j < k \text{ whenever } a_i = a_j, \text{ then } b_i = b_j.$$

It is something of a bother having to throw in Δ in this case, but all we have to say is that if u is consistent under the above rule, then so also will $u \cup \{\Delta\}$; and these are the only consistent sets of data objects for this example. Perhaps it would make more sense to use $\{(a, b)\}$ in place of the simpler (a, b), and then we could set $\Delta = \emptyset$ more naturally. From this point of view we can regard each data object as a (very small!) fragment of the graph of a function; the consistent sets then point to larger fragments of graphs.

The definition of the entailment relation is the minimal one:

$$u \vdash X \text{ iff either } X = \Delta \text{ or } X \in u.$$

These examples show that sometimes the main part of the structure is in Con, sometimes it is in $\vdash$, and sometimes it is in the interplay between the two notions. The object Δ is not in itself very important, but it is sometimes useful to have an object there that can always be counted upon to act in the same way in every system. ?

Already in formulating the axiom about the transitivity of $\vdash$ a relation between sets in Con was suggested; we now make this official.

Definition (`a_third_example`). Scott's third example is the partial-function information system on partial-function tokens for decidable sets A and B: least-informative datum $\bot$, consistency the functional condition on pairs, and minimal entailment. It is exactly the partial-function system for A and B.

a_third_example.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.partialFunctionSystem

@[expose] public section

namespace DomainTheory.InformationSystems

def a_third_example (A B : Type*) [DecidableEq A] [DecidableEq B] :
    InfoSys (PartialFunToken A B) :=
  partialFunctionSystem A B

end DomainTheory.InformationSystems
```

5.2.11 Definition 2.2

DEFINITION 2.2. *For $u, v \in \mathrm{Con}$ we write $u \vdash v$ to mean that $u \vdash X$ for all $X \in v$.* ?

Definition (`InfoSys.EntSet`). In an information system, write u EntSet v when u entails every member of the finite set v. This is Scott's multiary reading $u \vdash v$ of Definition 2.2 (stated for arbitrary finite u, v on the carrier).

Theorem (`InfoSys.EntSet.empty`). Every finite set entails the empty set of data.

Proof. There is no member of the empty set to entail. ?

Theorem (`InfoSys.EntSet.singleton`). Entailing a singleton $\{X\}$ is equivalent to entailing X.

Proof. Both directions unwind the definition of multiary entailment on a one-element set. ?

Theorem (`InfoSys.EntSet.bot`). The empty set of data entails the singleton of the least-informative datum.

Proof. Entailment of the bottom datum from the empty consistent set. ?

Theorem (`InfoSys.EntSet.union_mem`). If a consistent set u entails every member of a finite set v, then the finite union $u \cup' v$ is consistent.

Proof. Induct on v, closing under insert via entailment and the consistency axiom for entailed data; the empty case is u itself. ?

Theorem (`InfoSys.EntSet.refl`). Every consistent set entails itself.

Proof. Membership yields entailment. ?

Theorem (`InfoSys.EntSet.trans`). Multiary entailment is transitive: if u entails v and v entails w, with u and v consistent, then u entails w.

Proof. For each member of w, combine the multiary hypotheses with transitivity of unary entailment. ?

Theorem (`InfoSys.EntSet.mono`). Multiary entailment is monotone in the antecedents and antitone in the consequents: enlarging a consistent antecedent set and shrinking the entailed set preserves entailment.

Proof. Reduce to unary entailment from the smaller antecedent, then transfer along the inclusion by reflexivity and transitivity. ?

Theorem (`InfoSys.EntSet.union`). If u entails each of two finite sets, then u entails their finite union.

Proof. Membership in a finite union is membership in one of the arguments. ?

EntSet.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.InformationSystems.Constructive.funion

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u

variable {α : Type u} [DecidableEq α] (sys : InfoSys α)

def EntSet (u v : Finset α) : Prop := ∀ X ∈ v, sys.Ent u X

namespace EntSet

theorem empty (u : Finset α) : sys.EntSet u (∅ : Finset α) := by
  intro X hX
  exact False.elim (Finset.notMem_empty X hX)

theorem singleton {u : Finset α} {X : α} :
    sys.EntSet u {X} ↔ sys.Ent u X := by
  constructor
  · intro h
    exact h X (Finset.mem_singleton_self X)
  · intro h Y hY
    rw [Finset.mem_singleton] at hY
    subst hY
    exact h

theorem bot : sys.EntSet ∅ ({sys.bot} : Finset α) := by
  intro X hX
  rw [Finset.mem_singleton] at hX
  subst hX
  exact sys.ent_bot sys.con_empty

theorem union_mem {u v : Finset α} (hu : u ∈ sys.Con) (h : sys.EntSet u v) :
    u ∪' v ∈ sys.Con := by
  have : ∀ s : Finset α, (∀ x ∈ s, x ∈ v) → u ∪' s ∈ sys.Con := by
    intro s
    refine Finset.induction_on s ?_ ?_
    · intro _
```

```
      simpa [funion.empty_right] using hu
    · intro a t _ha ih hmem
      have hEnt_u_a : sys.Ent u a := h a (hmem a (Finset.mem_insert_self a t))
      have hut : u ∪' t ∈ sys.Con := ih fun x hx => hmem x (Finset.mem_insert_of_mem
    hx)
      have hEnt : sys.Ent (u ∪' t) a :=
        sys.ent_trans hut hu (fun y hy => sys.ent_refl hut (funion.subset_left u t hy)
    )
          hEnt_u_a
      have hins : insert a (u ∪' t) ∈ sys.Con := sys.ent_con hEnt
      simpa [funion.insert] using hins
  exact this v fun _ hx => hx

theorem refl {u : Finset α} (hu : u ∈ sys.Con) : sys.EntSet u u :=
  fun _X hX => sys.ent_refl hu hX

theorem trans {u v w : Finset α}
    (hu : u ∈ sys.Con) (hv : v ∈ sys.Con)
    (huv : sys.EntSet u v) (hvw : sys.EntSet v w) : sys.EntSet u w :=
  fun X hX => sys.ent_trans hu hv huv (hvw X hX)

theorem mono {u u' v v' : Finset α}
    (hu : u ∈ sys.Con) (hu' : u' ∈ sys.Con)
    (hsubu : u ⊆ u') (huv : sys.EntSet u v) (hsubv : v' ⊆ v) :
    sys.EntSet u' v' := by
  intro X hX
  have hEnt : sys.Ent u X := huv X (hsubv hX)
  refine sys.ent_trans hu' hu ?_ hEnt
  intro y hy
  exact sys.ent_refl hu' (hsubu hy)

theorem union {u v v' : Finset α}
    (huv : sys.EntSet u v) (huv' : sys.EntSet u v') : sys.EntSet u (v ∪' v') := by
  intro X hX
  rcases funion.mem.mp hX with h | h
  · exact huv X h
  · exact huv' X h

end EntSet

end InfoSys

end DomainTheory.InformationSystems
```

5.2.12 Finite union

Theorem (`Constructive.insert_comm'`). Inserting two elements into a finite set may be done in either order.

Proof. Membership in a double insert is disjunction of three alternatives; reordering the inserts merely reorders those alternatives. ?

Definition (`Constructive.instLeftCommutativeInsert`). Insert is left-commutative on finite sets, so multiset folds over insert are well-defined independently of order.

Definition (`Constructive.funion`). The finite union $u \cup' v$ is the result of inserting every member

of v into u, via a multiset fold. The infix $\cup'$ denotes this operation.

Definition (`Constructive.decidableEq_finset`). Equality of finite sets is decided by mutual inclusion.

Theorem (`Constructive.decidableEq_finset.eq_of_subset`). Mutual inclusion yields equality of finite sets.

Proof. Antisymmetry of the subset order. ?

Theorem (`Constructive.decidableEq_finset.false_of_ne`). A set is not unequal to itself under the mutual-inclusion test.

Proof. Reflexivity of $\subseteq$. ?

Theorem (`Constructive.funion.mem_foldr_insert`). An element lies in a fold of insert over a multiset into a finite set precisely when it lies in the seed set or in the multiset.

Proof. Induction on the multiset, using the membership rule for insert at each cons step. ?

Theorem (`Constructive.funion.mem`). Membership in a finite union is membership in either argument.

Proof. Finite union is a fold of insert, so the fold membership lemma applies. ?

Theorem (`Constructive.funion.empty_right`). Finite union with the empty set on the right is the left argument.

Proof. Membership reduces by the membership lemma. ?

Theorem (`Constructive.funion.subset_left`). Each argument of a finite union is a subset of the union (left).

Proof. Membership in the left argument yields membership in the union. ?

Theorem (`Constructive.funion.subset_right`). Each argument of a finite union is a subset of the union (right).

Proof. Membership in the right argument yields membership in the union. ?

Theorem (`Constructive.funion.subset_iff`). A finite union is a subset of w precisely when both arguments are.

Proof. Both directions use the membership and subset lemmas for finite union. ?

Theorem (`Constructive.funion.coe`). Coercion of a finite union to a set is the set-theoretic union of the coercions.

Proof. Membership agrees by the membership lemma. ?

Theorem (`Constructive.funion.insert`). Finite union commutes past insert on the right: $u \cup' \mathrm{insert}\, a\, t = \mathrm{insert}\, a\, (u \cup' t)$.

Proof. Compare membership on both sides via the membership lemma and the insert rule. ?

Theorem (`Constructive.funion.singleton_eq_insert`). Finite union of a singleton with a finite set is the corresponding insert.

Proof. Membership on either side is the same by the membership and insert characterizations. ?

Theorem (`Constructive.funion.comm`). Finite union is commutative: $a \cup' b = b \cup' a$.

Proof. Compare membership on both sides via the membership lemma for finite union. ?

Theorem (`Constructive.funion.assoc`). Finite union is associative: $(a \cup' b) \cup' c = a \cup' (b \cup' c)$.

Proof. Compare membership on both sides, splitting each disjunction according to the membership lemma. ?

Theorem (`Constructive.funion.left_comm`). Finite union is left-commutative: $a \cup' (b \cup' c) = b \cup' (a \cup' c)$.

Proof. Compare membership on both sides via the membership lemma. ?

Theorem (`Constructive.funion.self`). Finite union is idempotent: $a \cup' a = a$.

Proof. Compare membership on both sides via the membership lemma. ?

funion.lean

```
module

public import Mathlib.Data.Finset.Insert

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {α : Type*} [DecidableEq α]

theorem insert_comm' (a b : α) (s : Finset α) :
    insert a (insert b s) = insert b (insert a s) := by
  ext x
  simp only [Finset.mem_insert]
  constructor
  · rintro (h | h | h)
    exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]
  · rintro (h | h | h)
    exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]

instance instLeftCommutativeInsert :
    LeftCommutative (insert : α → Finset α → Finset α) := ⟨insert_comm'⟩

def funion (u v : Finset α) : Finset α := Multiset.foldr insert u v.1

@[inherit_doc] infixl:65 " ∪' " => funion

namespace decidableEq_finset

omit [DecidableEq α] in
theorem eq_of_subset (s t : Finset α) (h : s ⊆ t ∧ t ⊆ s) : s = t :=
  Finset.Subset.antisymm h.1 h.2

omit [DecidableEq α] in
theorem false_of_ne (s t : Finset α) (h : ¬(s ⊆ t ∧ t ⊆ s)) (heq : s = t) :
    False := by
  subst heq
  exact h ⟨Finset.Subset.refl _, Finset.Subset.refl _⟩

end decidableEq_finset

def decidableEq_finset {α : Type*} [DecidableEq α] : DecidableEq (Finset α) :=
  fun s t =>
    if h : s ⊆ t ∧ t ⊆ s then
      isTrue (decidableEq_finset.eq_of_subset s t h)
    else
      isFalse (decidableEq_finset.false_of_ne s t h)

namespace funion

theorem mem_foldr_insert (a : α) (u : Finset α) (s : Multiset α) :
    a ∈ Multiset.foldr insert u s ↔ a ∈ u ∨ a ∈ s := by
```

```
  refine Multiset.induction_on s ?_ ?_
  · simp
  · intro b t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    · rintro (h | h | h)
      exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]
    · rintro (h | h | h)
      exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]

@[simp] theorem mem {a : α} {u v : Finset α} :
    a ∈ u ∪' v ↔ a ∈ u ∨ a ∈ v := mem_foldr_insert a u v.1

@[simp] theorem empty_right (u : Finset α) : u ∪' (∅ : Finset α) = u := by
  ext x
  simp only [mem, Finset.notMem_empty, or_false]

theorem subset_left (u v : Finset α) : u ⊆ u ∪' v := fun _ hx => mem.2 (Or.inl hx)

theorem subset_right (u v : Finset α) : v ⊆ u ∪' v := fun _ hx => mem.2 (Or.inr hx)

theorem subset_iff {u v w : Finset α} : u ∪' v ⊆ w ↔ u ⊆ w ∧ v ⊆ w := by
  constructor
  · intro h
    exact ⟨fun x hx => h (subset_left u v hx),
           fun x hx => h (subset_right u v hx)⟩
  · rintro ⟨hu, hv⟩ x hx
    rcases mem.1 hx with h | h
    exacts [hu h, hv h]

theorem coe (u v : Finset α) :
    (↑(u ∪' v) : Set α) = (↑u : Set α) ∪ ↑v := by
  ext x
  simp only [Set.mem_union, Finset.mem_coe, mem]

theorem insert (u : Finset α) (a : α) (t : Finset α) :
    u ∪' insert a t = insert a (u ∪' t) := by
  ext x
  constructor
  · intro hx
    rcases mem.mp hx with hu | hins
    · exact Finset.mem_insert_of_mem (mem.mpr (Or.inl hu))
    · rcases Finset.mem_insert.mp hins with ha | ht
      · exact Finset.mem_insert.mpr (Or.inl ha)
      · exact Finset.mem_insert_of_mem (mem.mpr (Or.inr ht))
  · intro hx
    rcases Finset.mem_insert.mp hx with ha | h
    · exact mem.mpr (Or.inr (Finset.mem_insert.mpr (Or.inl ha)))
    · rcases mem.mp h with hu | ht
      · exact mem.mpr (Or.inl hu)
      · exact mem.mpr (Or.inr (Finset.mem_insert.mpr (Or.inr ht)))

theorem singleton_eq_insert (a : α) (s : Finset α) :
    ({a} : Finset α) ∪' s = Insert.insert a s := by
  ext x
  constructor
```

```
  · intro hx
    rcases mem.mp hx with ha | hs
    · exact Finset.mem_insert.mpr (Or.inl (Finset.mem_singleton.mp ha))
    · exact Finset.mem_insert_of_mem hs
  · intro hx
    rcases Finset.mem_insert.mp hx with ha | hs
    · exact mem.mpr (Or.inl (Finset.mem_singleton.mpr ha))
    · exact mem.mpr (Or.inr hs)

theorem comm (a b : Finset α) : a ∪' b = b ∪' a := by
  ext x
  constructor
  · intro hx
    rcases mem.mp hx with h | h
    · exact mem.mpr (Or.inr h)
    · exact mem.mpr (Or.inl h)
  · intro hx
    rcases mem.mp hx with h | h
    · exact mem.mpr (Or.inr h)
    · exact mem.mpr (Or.inl h)

theorem assoc (a b c : Finset α) :
    (a ∪' b) ∪' c = a ∪' (b ∪' c) := by
  ext x
  constructor
  · intro hx
    rcases mem.mp hx with h | h
    · rcases mem.mp h with h | h
      · exact mem.mpr (Or.inl h)
      · exact mem.mpr (Or.inr (mem.mpr (Or.inl h)))
    · exact mem.mpr (Or.inr (mem.mpr (Or.inr h)))
  · intro hx
    rcases mem.mp hx with h | h
    · exact mem.mpr (Or.inl (mem.mpr (Or.inl h)))
    · rcases mem.mp h with h | h
      · exact mem.mpr (Or.inl (mem.mpr (Or.inr h)))
      · exact mem.mpr (Or.inr h)

theorem left_comm (a b c : Finset α) :
    a ∪' (b ∪' c) = b ∪' (a ∪' c) := by
  ext x
  constructor
  · intro hx
    rcases mem.mp hx with h | h
    · exact mem.mpr (Or.inr (mem.mpr (Or.inl h)))
    · rcases mem.mp h with h | h
      · exact mem.mpr (Or.inl h)
      · exact mem.mpr (Or.inr (mem.mpr (Or.inr h)))
  · intro hx
    rcases mem.mp hx with h | h
    · exact mem.mpr (Or.inr (mem.mpr (Or.inl h)))
    · rcases mem.mp h with h | h
      · exact mem.mpr (Or.inl h)
      · exact mem.mpr (Or.inr (mem.mpr (Or.inr h)))

theorem self (a : Finset α) : a ∪' a = a := by
```

```
  ext x
  constructor
  · intro hx
    rcases mem.mp hx with h | h <;> exact h
  · intro hx
    exact mem.mpr (Or.inl hx)

end funion

end DomainTheory.InformationSystems.Constructive
```

5.2.13 Proposition 2.3

PROPOSITION 2.3. *For all* $u, v, w, u', v' \in \mathrm{Con}$*, we have:*

(i) $\emptyset \vdash \{\Delta\}$;

(ii) $u \vdash v$ implies $u \cup v \in \mathrm{Con}$;

(iii) $u \vdash u$;

(iv) $u \vdash v$ and $v \vdash w$ imply $u \vdash w$;

(v) $u' \supseteq u$, $u \vdash v$, and $v \supseteq v'$ imply $u' \vdash v'$; and

(vi) $u \vdash v$ and $u \vdash v'$ imply $u \vdash v \cup v'$.

Proof. Obvious. ?

Remark. It is not worth formulating a formal result at this point, but it should be clear that the properties in 2.3 could have been used as axioms. That is to say, we could have taken $\vdash$ as a binary relation on the set Con, used 2.1 (i)–(ii) and 2.3 (i)–(vi) as axioms. (Actually, 2.3(ii) is redundant.) The old-style entailment relation would then be definable by:

$$u \vdash X \quad \text{iff} \quad u \vdash \{X\}.$$

The previous axioms in 2.1 are then easily proved from the new ones. ?

Some notation. As it is not always clear what structure is meant if we refer to a domain simply by naming its underlying set of data objects, it will be better if we use a letter to refer to an information system as a whole. We shall therefore write:

$$\mathbf{A} = (D_{\mathbf{A}}, \Delta_{\mathbf{A}}, \mathrm{Con}_{\mathbf{A}}, \vdash_{\mathbf{A}}),$$

and say that $\mathbf{A}$ is an information system—with its parts as indicated. For a more informal discussion, the notation without subscripts can suffice, but there can be problems when several different systems are involved. ?

So much for the definition of what a system is as a mathematical structure. The next question is: What is it good for? More precisely, if information systems are the backbones of domains, then where are the elements? That is the topic of the next section.

Theorem (`proposition_2_3`). For consistent finite sets, multiary entailment satisfies: the empty set entails $\{\Delta\}$; entailment of v from u yields consistency of $u \cup' v$; reflexivity; transitivity; monotonicity in antecedents and antitonicity in consequents; and closure under finite unions of consequents.

Proof. Assemble the six multiary-entailment library facts for bottom, consistency of unions, reflexivity, transitivity, monotonicity, and union of consequents. ?

`proposition_2_3.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.EntSet

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem proposition_2_3 :
    (∀ {α : Type*} [DecidableEq α] (sys : InfoSys α),
      sys.EntSet ∅ ({sys.bot} : Finset α)) ∧
    (∀ {α : Type*} [DecidableEq α] (sys : InfoSys α) {u v : Finset α},
      u ∈ sys.Con → sys.EntSet u v → u ∪' v ∈ sys.Con) ∧
    (∀ {α : Type*} [DecidableEq α] (sys : InfoSys α) {u : Finset α},
      u ∈ sys.Con → sys.EntSet u u) ∧
    (∀ {α : Type*} [DecidableEq α] (sys : InfoSys α) {u v w : Finset α},
      u ∈ sys.Con → v ∈ sys.Con → sys.EntSet u v → sys.EntSet v w → sys.EntSet u w)
     ∧
    (∀ {α : Type*} [DecidableEq α] (sys : InfoSys α) {u u' v v' : Finset α},
      u ∈ sys.Con → u' ∈ sys.Con → u ⊆ u' → sys.EntSet u v → v' ⊆ v → sys.EntSet u
    ' v') ∧
    (∀ {α : Type*} [DecidableEq α] (sys : InfoSys α) {u v v' : Finset α},
      sys.EntSet u v → sys.EntSet u v' → sys.EntSet u (v ∪' v')) :=
  ⟨@EntSet.bot, @EntSet.union_mem, @EntSet.refl,
    @EntSet.trans, @EntSet.mono, @EntSet.union⟩

end InfoSys

end DomainTheory.InformationSystems
```

5.3 The elements of a system

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

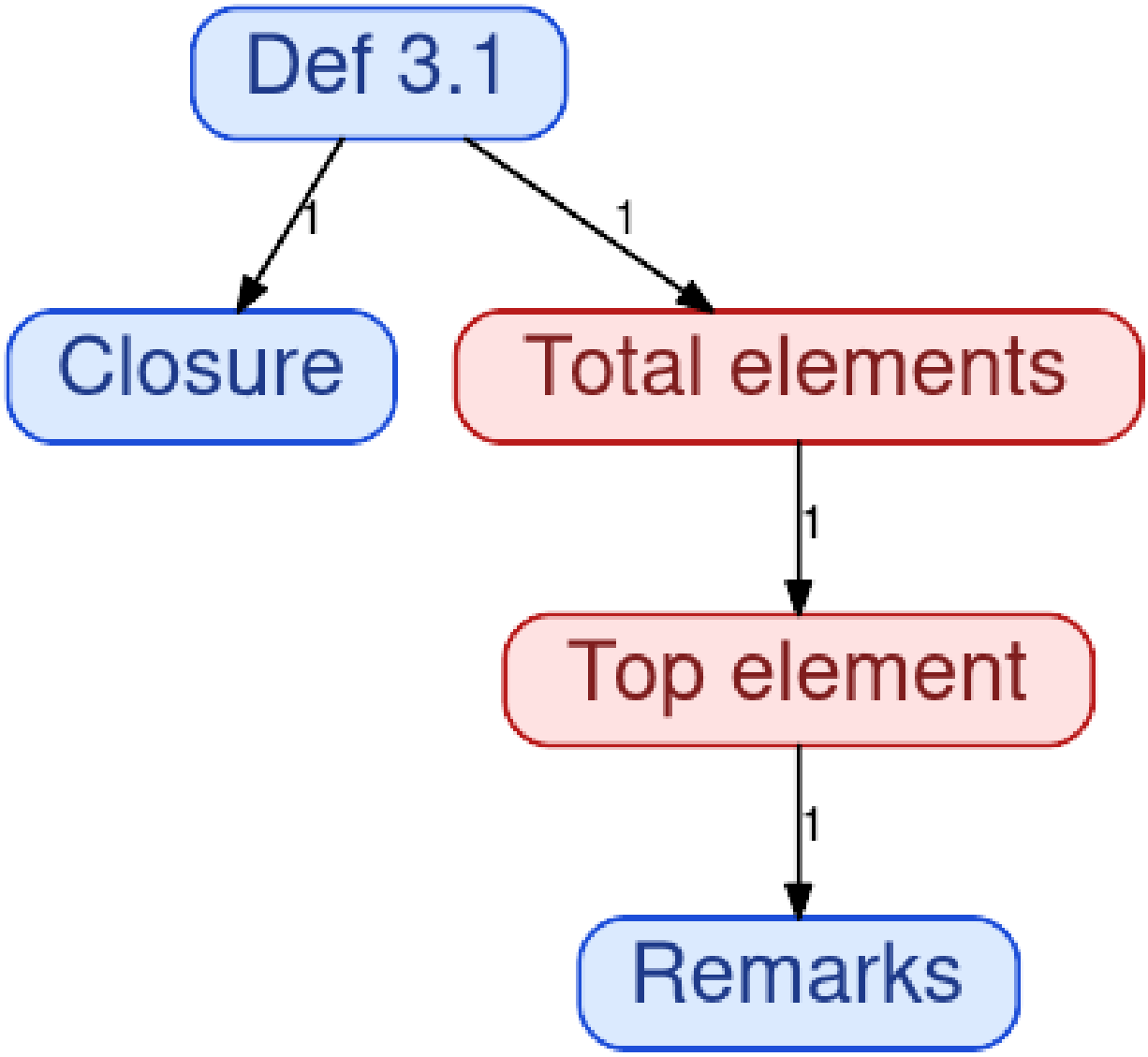


Figure 56: Diagram 21

5.3.1 Introduction

Let $\mathbf{A}$ be an information system, and suppose we already had a concept of being an element of $\mathbf{A}$. The data objects are meant to be propositions about elements, so if X is in $D_{\mathbf{A}}$ and if x is an element, then we can be expected to know what it means to say that X is true of x. Since all we are given is the set of data objects $D_{\mathbf{A}}$, we have to assume that it contains enough objects to distinguish between distinct elements. (If there were not enough of them, we would have to change the set $D_{\mathbf{A}}$.) Formally, we can write of two elements x and y:

$$x = y \quad \text{iff all } X \in D_{\mathbf{A}} \text{ which are true of } x \text{ are also true of } y, \text{ and conversely.}$$

If this principle is accepted, then the elements can be *identified* with the sets of propositions true of

them; formally we can assert:

$$x = \{X \in D_{\mathbf{A}} \mid X \text{ is true of } x\}.$$

There can be no confusion injected here in identifying elements that ought to be distinct, since we are assuming that there are enough data objects in the system. If for some reason we felt we ought to have more of them to distinguish between more elements, then we would have to pass to a larger and *different* information system. By agreement, then, the above equation is a tautology.

So, for the sake of simplicity, elements can be taken to be *sets* of data objects. But, we hasten to ask, which sets? The question is really: What are the properties of truth? The answers are well known, and, in fact, we have already used them in our examples in the last section. In words, the set of true propositions about a possible element must be (i) consistent in itself, and (ii) closed under entailment (or *deductively closed*, for short). Condition (i) is clear because we are talking about a *possible* element, not an impossible one. Condition (ii) is acceptable, because, by the very meaning of the word, entailment should be truth preserving. In addition, we are also saying the converse: *any* set having properties (i) and (ii) corresponds to a (partial) element. Here is the formal statement:

5.3.2 Definition 3.1

DEFINITION 3.1. The *elements* of the information system $\mathbf{A} = (D_{\mathbf{A}}, \Delta_{\mathbf{A}}, \mathrm{Con}_{\mathbf{A}}, \vdash_{\mathbf{A}})$ are those subsets x of $D_{\mathbf{A}}$ where:

(i) all finite subsets of x are in $\mathrm{Con}_{\mathbf{A}}$; and

(ii) whenever $u \subseteq x$ and $u \vdash_{\mathbf{A}} X$, then $X \in x$.

We write $x \in |\mathbf{A}|$ to mean x is an element of the system. This set of elements is the *domain* determined by the given system. An element that is not included in any strictly larger element in the domain is called a *total element*; the set of total elements is denoted by $\mathrm{Tot}_{\mathbf{A}}$. ?

Definition (`InfoSys.Element`). An element of an information system is a set of data that is finitely consistent—every finite subset lies in Con—and deductively closed: whenever a finite set of data from the carrier entails a further datum, that datum belongs to the carrier. The domain $|\mathbf{A}|$ is the collection of all such elements.

Theorem (`InfoSys.Element.ext`). Two elements are equal precisely when they have the same carrier.

Proof. An element is determined by its carrier together with the consistency and closure witnesses, which are propositions. ?

Theorem (`InfoSys.Element.le_refl`). Carrier inclusion is reflexive on elements.

Proof. Every set is a subset of itself. ?

Theorem (`InfoSys.Element.le_trans`). Carrier inclusion is transitive on elements.

Proof. Subset is transitive. ?

Theorem (`InfoSys.Element.le_antisymm`). Carrier inclusion is antisymmetric on elements.

Proof. Mutual inclusion of carriers yields equal carriers, hence equal elements. ?

Definition (`InfoSys.Element.instPartialOrder`). The domain of elements is partially ordered by inclusion of carriers.

`Element.lean`

```
module

public import Mathlib.Data.Finset.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.InfoSys

@[expose] public section

universe u

namespace DomainTheory.InformationSystems
namespace InfoSys

variable {α : Type u} [DecidableEq α] (sys : InfoSys α)

structure Element where

  carrier : Set α

  consistent : ∀ Y : Finset α, (Y : Set α) ⊆ carrier → Y ∈ sys.Con

  closed : ∀ (Y : Finset α) (a : α), (Y : Set α) ⊆ carrier → sys.Ent Y a → a ∈
    carrier

namespace Element

theorem ext {x y : sys.Element} (h : x.carrier = y.carrier) : x = y := by
  cases x
  cases y
  subst h
  rfl

theorem le_refl (x : sys.Element) : x.carrier ⊆ x.carrier :=
  Set.Subset.refl _

theorem le_trans (x y z : sys.Element)
    (hxy : x.carrier ⊆ y.carrier) (hyz : y.carrier ⊆ z.carrier) :
    x.carrier ⊆ z.carrier :=
  Set.Subset.trans hxy hyz

theorem le_antisymm (x y : sys.Element)
    (hxy : x.carrier ⊆ y.carrier) (hyx : y.carrier ⊆ x.carrier) :
    x = y :=
  ext sys (Set.Subset.antisymm hxy hyx)

instance instPartialOrder : PartialOrder sys.Element where
  le x y := x.carrier ⊆ y.carrier
  le_refl := le_refl sys
  le_trans := le_trans sys
  le_antisymm := le_antisymm sys

end Element

end InfoSys
end DomainTheory.InformationSystems
```

5.3.3 Closure

Definition (`InfoSys.closure`). The closure of a consistent finite set u is the element whose carrier consists of the data entailed by u.

Theorem (`InfoSys.closure.subset`). Every token of a consistent finite set belongs to the carrier of its closure.

Proof. Reflexivity of entailment. ?

Theorem (`InfoSys.closure.mem_iff`). Membership in the closure carrier is entailment from u.

Proof. Unfolding. ?

Theorem (`InfoSys.theorem_35`). A consistent finite set sits under its closure.

Proof. The subset property of closure. ?

`closure.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def closure (u : Finset α) (hu : u ∈ sys.Con) : sys.Element where
  carrier := {X | sys.Ent u X}
  consistent := by
    intro Y hY
    have hEnt : sys.EntSet u Y := fun X hX => hY (Finset.mem_coe.2 hX)
    exact sys.con_subset (EntSet.union_mem sys hu hEnt) (funion.subset_right _ _)
  closed := by
    intro Y a hY hEnt
    have hYcon : Y ∈ sys.Con := by
      have hEntY : sys.EntSet u Y := fun X hX => hY (Finset.mem_coe.2 hX)
      exact sys.con_subset (EntSet.union_mem sys hu hEntY) (funion.subset_right _ _)
    exact sys.ent_trans hu hYcon (fun y hy => hY (Finset.mem_coe.2 hy)) hEnt

namespace closure

theorem subset {u : Finset α} (hu : u ∈ sys.Con) :
    ↑u ⊆ (sys.closure u hu).carrier :=
  fun _ hX => sys.ent_refl hu (Finset.mem_coe.1 hX)

theorem mem_iff {u : Finset α} (hu : u ∈ sys.Con) {X : α} :
    X ∈ (sys.closure u hu).carrier ↔ sys.Ent u X := Iff.rfl

end closure

theorem theorem_35 :
    ∀ {α : Type*} [DecidableEq α] (sys : InfoSys α) {u : Finset α} (hu : u ∈ sys.Con),
```

```
      ↑u ⊆ (sys.closure u hu).carrier :=
  @closure.subset

end InfoSys

end DomainTheory.InformationSystems
```

5.3.4 Total elements

Definition (`InfoSys.IsTotal`). An element x is total when it is maximal in the inclusion order on elements: whenever $x \leq y$, one has $y \leq x$.

`IsTotal.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def IsTotal (x : sys.Element) : Prop := ∀ y, x ≤ y → y ≤ x

end InfoSys

end DomainTheory.InformationSystems
```

5.3.5 Top element

Definition (`InfoSys.AllConsistent`). An information system is all-consistent when every finite set of data is consistent.

Definition (`InfoSys.topElement`). If the system is all-consistent, the top element is the element whose carrier is the whole set of data.

Theorem (`InfoSys.topElement.exists_top_iff_allConsistent`). There exists an element with universal carrier if and only if the system is all-consistent.

Proof. A universal carrier forces every finite set into consistency; conversely the top element supplies such a carrier. ?

Theorem (`InfoSys.topElement.le`). Under all-consistency, every element lies below the top element.

Proof. Every carrier is a subset of the universe. ?

Theorem (`InfoSys.topElement.isTotal`). The top element is total.

Proof. Anything above the top is below it by the preceding comparison. ?

Theorem (`InfoSys.topElement.eq_of_carrier_univ_of_isTotal`). A total element with universal carrier equals that universal-carrier element.

Proof. Mutual inclusion of carriers. ?

Theorem (`InfoSys.topElement.isTotal_of_carrier_univ`). An element with universal carrier is total.

Proof. Anything above it still has carrier contained in the universe, hence below it. ?

Theorem (`InfoSys.topElement.eq_of_isTotal`). Under all-consistency, every total element equals the top element.

Proof. Totality and the comparison with the top force equality. ?

Theorem (`InfoSys.topElement.isTotal_iff_eq`). Under all-consistency, an element is total precisely when it equals the top element.

Proof. Assemble the two preceding directions. ?

Theorem (`InfoSys.topElement.exists_unique_total_of_allConsistent`). Under all-consistency there is a unique total element.

Proof. The top element is total, and every total element equals it. ?

topElement.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element
public import DomainTheory.InformationSystems.ElementsOfASystem.IsTotal

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def AllConsistent : Prop := ∀ u : Finset α, u ∈ sys.Con

def topElement (h : sys.AllConsistent) : sys.Element where
  carrier := Set.univ
  consistent := fun Y _ => h Y
  closed := fun _ _ _ _ => trivial

namespace topElement

theorem exists_top_iff_allConsistent :
    (∃ x : sys.Element, x.carrier = Set.univ) ↔ sys.AllConsistent := by
  constructor
  · intro ⟨x, hx⟩ u
    exact x.consistent u (hx ▶ Set.subset_univ _)
  · intro h
    exact ⟨sys.topElement h, rfl⟩

theorem le (h : sys.AllConsistent) (x : sys.Element) :
    x ≤ sys.topElement h := fun _ _ => trivial

theorem isTotal (h : sys.AllConsistent) :
    sys.IsTotal (sys.topElement h) := fun y _ => le sys h y

theorem eq_of_carrier_univ_of_isTotal {x t : sys.Element}
    (hx : x.carrier = Set.univ) (ht : sys.IsTotal t) : t = x :=
```

```
  le_antisymm (fun _ _ => hx ▶ trivial) (ht x (fun _ _ => hx ▶ trivial))

theorem isTotal_of_carrier_univ {x : sys.Element} (hx : x.carrier = Set.univ) :
    sys.IsTotal x := fun _ _ _ _ => hx ▶ trivial

theorem eq_of_isTotal (h : sys.AllConsistent) {x : sys.Element}
    (hx : sys.IsTotal x) : x = sys.topElement h :=
  le_antisymm (le sys h x) (hx _ (le sys h x))

theorem isTotal_iff_eq (h : sys.AllConsistent) {x : sys.Element} :
    sys.IsTotal x ↔ x = sys.topElement h :=
  ⟨eq_of_isTotal sys h, fun hx => hx ▶ isTotal sys h⟩

theorem exists_unique_total_of_allConsistent (h : sys.AllConsistent) :
    ∃! x : sys.Element, sys.IsTotal x :=
  ⟨sys.topElement h, ⟨isTotal sys h, fun _ hy => eq_of_isTotal sys h hy⟩⟩

end topElement

end InfoSys
end DomainTheory.InformationSystems
```

5.3.6 Remarks

Remarks. Any subset of $D_{\mathbf{A}}$ satisfying 3.1 (i) can be called consistent, and every consistent set generates an element by closing it under entailment. Note, too, that every element contains $\Delta_{\mathbf{A}}$ as a member, because the least informative proposition is true of all elements. Moreover, every domain has a least element contained in all other elements. We call it $\bot_{\mathbf{A}}$ and write formally:

$$\bot_{\mathbf{A}} = \{X \in D_{\mathbf{A}} \mid \{\Delta_{\mathbf{A}}\} \vdash_{\mathbf{A}} X\}.$$

In the above we could just as well have used the empty set $\emptyset$ in place of $\{\Delta_{\mathbf{A}}\}$; and often we can write simply $\vdash_{\mathbf{A}} X$.

The element $\bot_{\mathbf{A}}$ is often called the *bottom* element of the domain. There need be no *top* or maximal element, $\top_{\mathbf{A}}$. Such an element is possible if, and only if, *all* finite subsets of $D_{\mathbf{A}}$ are consistent, in which case, as a set, $\top_{\mathbf{A}} = D_{\mathbf{A}}$. The possibility is not excluded—rather, it is not required. In case $\top_{\mathbf{A}}$ exists, it is the unique total element of the domain, and conversely. ?

Returning to our question of the balance between axiomatizaton and construction, what we have done is first to axiomatize the properties of $(D_{\mathbf{A}}, \Delta_{\mathbf{A}}, \mathrm{Con}_{\mathbf{A}}, \vdash_{\mathbf{A}})$, then to construct the domain $|\mathbf{A}|$ from this structure. The principal claimed advantages are that the axioms for consistency and entailment are already essentially familiar, and that the definition of elementhood is direct and natural from our understanding of the properties of truth.

The examples. Let us look again at the three examples of information systems of the last section. (1) In the first we see that the elements, by the formal definition, are either the finite sets $\{n \mid n \leq m\}$ or the whole set (i.e. $\top$). Intuitively, this corresponds to the fact that the chosen propositions only give *lower bounds* on a possible element; thus, no element is "finished" until it becomes infinite. (2) In the second example there is no top, and the elements are the sets of the form:

$$\{(n, q) \mid n \leq m \leq p \leq q\},$$

where $m \leq p$ are given, and where $q = \infty$ is allowed. The total elements here correspond to the (non-negative) integers, and the partial elements to situations where the lower and upper bounds have not come exactly together. (3) In the third example, the elements are just the graphs (i.e. sets of ordered pairs) of partial functions from A into B (with the object Δ adjoined to the graph). Total elements correspond to total functions (i.e. functions well defined on all of the set A). ?

The reader should be able to obtain some more examples of information systems from his own experience and should then consider what the corresponding domain is like. Examples are really quite easy to make up. Needless to say, Mathematical Logic provides a host of information systems, but logicians do not often consider the domains determined by the the systems they know to be worthy of study *in themselves*. In particular, it does not seem to be widely recognized that the domains obtained from information systems form a rich category, as we shall show later. The word "category" is definitely being used here in the precise technical sense of Category Theory, and the application of the notion is very appropriate since the category of domains has very good closure properties.

Theorem (`InfoSys.remark`). A top element—an element whose carrier is the whole set of data—exists if and only if every finite set of data is consistent.

Proof. This is the library comparison of universal carriers with all-consistency. ?

`remark.lean`

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.topElement

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

theorem remark :
    ∀ {α : Type*} [DecidableEq α] (sys : InfoSys α),
      (∃ x : sys.Element, x.carrier = Set.univ) ↔ sys.AllConsistent :=
  @topElement.exists_top_iff_allConsistent

end InfoSys
end DomainTheory.InformationSystems
```

5.4 Domains as lattices and as topological spaces

<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

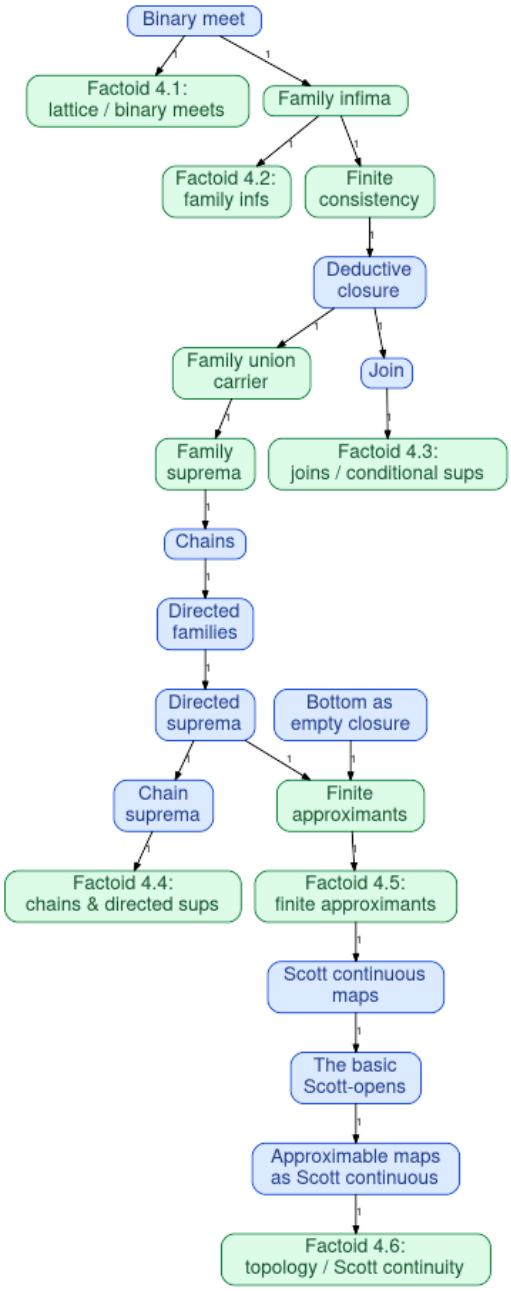


Figure 57: Diagram 22

5.4.1 Introduction

I will attempt to keep this section as informal as possible—especially as Sections 2 and 3 were quite formal (i.e. mathematical) enough. The main purpose of the discussion of this section is to relate the new presentation of the Theory of Domains to previously published ones. (Some alternatives are discussed at the end of the paper.)

5.4.2 Binary meet

Definition (`InfoSys.inf`). The binary meet of two elements is the element whose carrier is the intersection of their carriers. Intersection of consistent deductively closed sets is again consistent and deductively closed, so the result is an element.

Theorem (`InfoSys.inf.le_left`). The binary meet of x and y approximates x.

Proof. Intersection is a subset of the left factor. ?

Theorem (`InfoSys.inf.le_right`). The binary meet of x and y approximates y.

Proof. Intersection is a subset of the right factor. ?

Theorem (`InfoSys.inf.le_inf`). If x approximates both y and z, then x approximates their binary meet.

Proof. Membership in x yields membership in both carriers, hence in their intersection. ?

Theorem (`InfoSys.inf.le_iff_eq`). An element x approximates y if and only if the binary meet of x and y equals x.

Proof. One direction uses the left projection of the meet together with the universal property of intersection under the order hypothesis; the converse recovers the second factor from the carrier equality. ?

Theorem (`InfoSys.inf.le_iff`). An element approximates a binary meet if and only if it approximates both arguments.

Proof. Combine the two projections of the meet with the universal property of the meet. ?

Theorem (`InfoSys.inf.comm`). Binary meet is commutative.

Proof. Each side approximates both arguments of the other, so the two meets coincide by antisymmetry. ?

Theorem (`InfoSys.inf.assoc`). Binary meet is associative.

Proof. Each association approximates the three factors of the other via the projections and the universal property of the meet. ?

Theorem (`InfoSys.inf.idem`). The binary meet of an element with itself is that element.

Proof. Reflexivity of the order together with the absorption characterization of the meet. ?

Theorem (`InfoSys.inf.mono`). Binary meet is monotone in each argument.

Proof. Transport the projections of the smaller meet along the given inequalities, then apply the universal property. ?

Theorem (`InfoSys.inf.botElement_inf`). The binary meet of the bottom element with any element is bottom.

Proof. Bottom approximates every element, so absorption yields the claim. ?

Theorem (`InfoSys.inf.inf_botElement`). The binary meet of any element with bottom is bottom.

Proof. Commute the meet and apply the previous theorem. ?

`inf.lean`

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.botElement

@[expose] public section

namespace DomainTheory.InformationSystems
namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)
```

```
def inf (x y : sys.Element) : sys.Element where
  carrier := x.carrier ∩ y.carrier
  consistent := fun Y hY =>
    x.consistent Y (Set.Subset.trans hY Set.inter_subset_left)
  closed := fun Y a hY hEnt =>
    ⟨x.closed Y a (Set.Subset.trans hY Set.inter_subset_left) hEnt,
     y.closed Y a (Set.Subset.trans hY Set.inter_subset_right) hEnt⟩

namespace inf

theorem le_left (x y : sys.Element) : sys.inf x y ≤ x :=
  Set.inter_subset_left

theorem le_right (x y : sys.Element) : sys.inf x y ≤ y :=
  Set.inter_subset_right

theorem le_inf {x y z : sys.Element} (hxy : x ≤ y) (hxz : x ≤ z) : x ≤ sys.inf y z :=
  fun _ ha => ⟨hxy ha, hxz ha⟩

theorem le_iff_eq {x y : sys.Element} : x ≤ y ↔ sys.inf x y = x := by
  constructor
  · intro h
    refine le_antisymm (le_left sys x y) ?_
    intro a ha
    exact ⟨ha, h ha⟩
  · intro heq
    have hc : x.carrier = x.carrier ∩ y.carrier :=
      Eq.symm (congrArg Element.carrier heq)
    intro a ha
    exact ((Set.ext_iff.1 hc a).1 ha).2

theorem le_iff {x y z : sys.Element} : z ≤ sys.inf x y ↔ z ≤ x ∧ z ≤ y :=
  ⟨fun h => ⟨le_trans h (le_left sys x y), le_trans h (le_right sys x y)⟩,
   fun ⟨hx, hy⟩ => le_inf sys hx hy⟩

theorem comm (x y : sys.Element) : sys.inf x y = sys.inf y x := by
  refine le_antisymm ?_ ?_
  · exact le_inf sys (le_right sys x y) (le_left sys x y)
  · exact le_inf sys (le_right sys y x) (le_left sys y x)

theorem assoc (x y z : sys.Element) : sys.inf (sys.inf x y) z = sys.inf x (sys.inf y z
    ) := by
  refine le_antisymm ?_ ?_
  · refine le_inf sys ?_ (le_inf sys ?_ ?_)
    · exact le_trans (le_left sys _ _) (le_left sys _ _)
    · exact le_trans (le_left sys _ _) (le_right sys _ _)
    · exact le_right sys _ _
  · refine le_inf sys (le_inf sys ?_ ?_) ?_
    · exact le_left sys _ _
    · exact le_trans (le_right sys _ _) (le_left sys _ _)
    · exact le_trans (le_right sys _ _) (le_right sys _ _)

theorem idem (x : sys.Element) : sys.inf x x = x :=
  (le_iff_eq sys (y := x)).1 (le_refl x)
```

```
theorem mono {x₁ x₂ y₁ y₂ : sys.Element} (hx : x₁ ≤ x₂) (hy : y₁ ≤ y₂) :
    sys.inf x₁ y₁ ≤ sys.inf x₂ y₂ :=
  le_inf sys (le_trans (le_left sys _ _) hx) (le_trans (le_right sys _ _) hy)

theorem botElement_inf (x : sys.Element) : sys.inf sys.botElement x = sys.botElement
    :=
  (le_iff_eq sys (x := sys.botElement) (y := x)).1 (botElement.le sys x)

theorem inf_botElement (x : sys.Element) : sys.inf x sys.botElement = sys.botElement
    := by
  rw [comm sys, botElement_inf sys]

end inf

end InfoSys
end DomainTheory.InformationSystems
```

5.4.3 Factoid 4.1: lattice / binary meets

Lattice-theoretic considerations. Let $\mathbf{A}$ be an information system. Because the elements of $|\mathbf{A}|$ are introduced as sets, these elements can be given structure from what we know already about ordinary sets. For instance, the set-theoretic inclusion relation between sets can certainly be applied to elements. The question is: What does $x \subseteq y$ mean intuitively? In fact, we have already mentioned this relationship before. It means that every proposition (among the ones given by the information system) true of x is also true of y (though not necessarily conversely). In other words, x is perhaps only partially determined, while y is more fully determined in a way that includes everything true of x. Clearly, then, if we grant the interest of partial elements, then this relation is a natural and basic one. We often read "$x \subseteq y$" as "x approximates y".

Now here is one of the main points of the new exposition: Because $x \subseteq y$ is *defined* in terms of a familiar mathematical notion, then as a relation between elements it inherits all the well-known properties of the set-theoretical relation. It should not be necessary to write them out, for everyone who can read this far knows that $\subseteq$ is in particular reflexive and transitive. We say that the domain $|\mathbf{A}|$ is a *partially ordered set under inclusion*. We note that this (trivial) result is *proved* from the definition rather than assumed as an axiom.

Let us try out another notion. Elements are consistent, deductively closed sets of data objects. Right? Every subset of a consistent set is consistent, but not every subset is closed under entailment, in general. Right? But suppose that x and y are two elements. It can easily be seen that their intersection $x \cap y$ is again consistent and deductively closed (by a 2 nanosecond proof). Therefore, the set of elements of a domain is closed under intersection. This means that, as a partially ordered set, $|\mathbf{A}|$ is an *inf semilattice*.

Well, the exact terminology is quite unimportant, but the properties of $\cap$ when combined with the properties of $\subseteq$ are pleasantly "algebraic" (and run to several lines when written out). We all know them: $\cap$ is idempotent, commutative, and associative. The element $\bot$, being the

The element $\bot$, being the smallest element of the domain, is a zero element for $\cap$. The operation $\cap$ is monotonic with respect to $\subseteq$, and we have the connection:

$$x \subseteq y \text{ iff } x \cap y = x.$$

What has just been alluded to in words is the standard mathematical axiomatization of the properties of an infimum (or meet) operation in a partially ordered set. (Remember, however, to have an operation well defined within a set it is necessary also to have closure under that operation.)

Theorem (`InfoSys.factoid_41`). For elements x and y of an information system, x approximates y if and only if the binary meet of x and y equals x.

Proof. This is the order–meet absorption law for binary infima of elements. ?

factoid_41.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    inf

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

theorem factoid_41 :
    ∀ {α : Type*} [DecidableEq α] (sys : InfoSys α) {x y : sys.Element},
      x ≤ y ↔ sys.inf x y = x :=
  @inf.le_iff_eq

end DomainTheory.InformationSystems.InfoSys
```

5.4.4 Family infima

Definition (`InfoSys.familyInfCarrier`). The carrier of a nonempty family's meet is the set of data belonging to every element of the family.

Definition (`InfoSys.familyInf`). The meet of a nonempty family of elements is the element with that intersection carrier. Consistency and deductive closure follow from those of any member of the family, since every finite set of data in the intersection lies in each member.

Theorem (`InfoSys.familyInf.le`). The family meet approximates every member of the family.

Proof. A datum in the intersection carrier belongs to that member. ?

Theorem (`InfoSys.familyInf.le_of`). An element that approximates every member of a nonempty family approximates the family meet.

Proof. Membership in the lower bound yields membership in every member's carrier, hence in the intersection. ?

Theorem (`InfoSys.familyInf.le_iff`). An element approximates a family meet if and only if it approximates every member of the family.

Proof. Combine the projection of the meet onto members with the universal property of the meet. ?

Theorem (`InfoSys.familyInf.pair`). The family meet of a two-element set equals the binary meet of those elements.

Proof. Each side approximates both elements (or both arguments of the binary meet), so the two meets coincide by antisymmetry. ?

familyInf.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    inf

@[expose] public section
```

```
namespace DomainTheory.InformationSystems
namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def familyInfCarrier (S : Set sys.Element) : Set α :=
  {a | ∀ x ∈ S, a ∈ x.carrier}

def familyInf (S : Set sys.Element) (hne : S.Nonempty) : sys.Element where
  carrier := sys.familyInfCarrier S
  consistent := by
    intro Y hY
    obtain ⟨x₀, hx₀⟩ := hne
    have hYx : (Y : Set α) ⊆ x₀.carrier := fun a ha => (hY ha) x₀ hx₀
    exact x₀.consistent Y hYx
  closed := by
    intro Y a hY hEnt x hx
    have hYx : (Y : Set α) ⊆ x.carrier := fun b hb => (hY hb) x hx
    exact x.closed Y a hYx hEnt

namespace familyInf

theorem le (S : Set sys.Element) (hne : S.Nonempty) {x : sys.Element}
    (hx : x ∈ S) : sys.familyInf S hne ≤ x :=
  fun _a ha => ha x hx

theorem le_of (S : Set sys.Element) (hne : S.Nonempty) {z : sys.Element}
    (h : ∀ x ∈ S, z ≤ x) : z ≤ sys.familyInf S hne :=
  fun _a ha x hx => h x hx ha

theorem le_iff (S : Set sys.Element) (hne : S.Nonempty) {z : sys.Element} :
    z ≤ sys.familyInf S hne ↔ ∀ x ∈ S, z ≤ x :=
  ⟨fun hz _x hx => le_trans hz (le sys S hne hx), le_of sys S hne⟩

theorem pair (x y : sys.Element) :
    sys.familyInf ({x, y} : Set sys.Element) ⟨x, Set.mem_insert x {y}⟩ = sys.inf x y :=
    by
  refine le_antisymm ?_ ?_
  · exact inf.le_inf sys
      (le sys _ _ (Set.mem_insert x {y}))
      (le sys _ _ (Set.mem_insert_of_mem _ rfl))
  · apply le_of sys
    intro z hz
    rcases Set.mem_insert_iff.mp hz with rfl | hy
    · exact inf.le_left sys _ _
    · have : z = y := hy
      subst this
      exact inf.le_right sys _ _

end familyInf

end InfoSys
end DomainTheory.InformationSystems
```

5.4.5 Factoid 4.2: family infs

What about infs of subsets? No problem. For any *non-empty* subfamily of $|\mathbf{A}|$, it is just as easy to prove that the set-theoretical intersection of all the elements in the subfamily is again an element of the domain. This makes $|\mathbf{A}|$ a (conditionally) complete inf semilattice. Even if you do not have any idea what I am talking about, it does not matter because the properties of the domain are built in by the very definition; you do not have to worry *at the start* about their formulation, for you can prove them when you need them (if ever).

Theorem (`InfoSys.factoid_42`). The meet of a two-element family of elements equals the binary meet of those two elements.

Proof. This is the binary special case of the family-meet construction. ?

`factoid_42.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    familyInf

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

theorem factoid_42 :
    ∀ {α : Type*} [DecidableEq α] (sys : InfoSys α) (x y : sys.Element),
      sys.familyInf ({x, y} : Set sys.Element) ⟨x, Set.mem_insert x {y}⟩ = sys.inf x y
    :=
  @familyInf.pair

end DomainTheory.InformationSystems.InfoSys
```

5.4.6 Finite consistency

Definition (`InfoSys.IsFinitelyConsistent`). A set of data is finitely consistent when every finite subset is consistent in the information system.

Theorem (`InfoSys.IsFinitelyConsistent.exists_entSet_of_subset_deductive`). If U is finitely consistent and every member of a finite set Y is entailed by some finite subset of U, then some finite subset of U entails every member of Y.

Proof. Induct on Y. The empty case is immediate. For an inserted datum, take finite antecedents for the new datum and for the remainder, form their finite union (still inside U, hence consistent by finite consistency), and obtain multiary entailment of the whole by transitivity of entailment from each piece. ?

`IsFinitelyConsistent.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    familyInf

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def IsFinitelyConsistent (U : Set α) : Prop :=
  ∀ Y : Finset α, (Y : Set α) ⊆ U → Y ∈ sys.Con

namespace IsFinitelyConsistent

theorem exists_entSet_of_subset_deductive
    (U : Set α) (hU : sys.IsFinitelyConsistent U) (Y : Finset α)
    (hY : (Y : Set α) ⊆ {a | ∃ u : Finset α, (u : Set α) ⊆ U ∧ sys.Ent u a}) :
    ∃ w : Finset α, (w : Set α) ⊆ U ∧ sys.EntSet w Y := by
  induction Y using Finset.induction_on with
  | empty =>
    exact ⟨∅, fun _ ha => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 ha)),
      fun _ ha => False.elim (Finset.notMem_empty _ ha)⟩
  | insert a s _ha ih =>
    have ha' : ∃ u : Finset α, (u : Set α) ⊆ U ∧ sys.Ent u a :=
      hY (Finset.mem_coe.2 (Finset.mem_insert_self a s))
    have hs' : (s : Set α) ⊆ {b | ∃ u : Finset α, (u : Set α) ⊆ U ∧ sys.Ent u b} :=
    by
      intro b hb
      exact hY (Finset.mem_coe.2 (Finset.mem_insert_of_mem (Finset.mem_coe.1 hb)))
    obtain ⟨ua, hua, hEnta⟩ := ha'
    obtain ⟨us, hus, hEnts⟩ := ih hs'
    refine ⟨ua ∪' us, ?_, ?_⟩
    · intro z hz
      rcases funion.mem.mp (Finset.mem_coe.1 hz) with h | h
      · exact hua (Finset.mem_coe.2 h)
      · exact hus (Finset.mem_coe.2 h)
    · intro b hb
      have hwU : (ua ∪' us : Set α) ⊆ U := by
        intro z hz
        rcases funion.mem.mp (Finset.mem_coe.1 hz) with h | h
        · exact hua (Finset.mem_coe.2 h)
        · exact hus (Finset.mem_coe.2 h)
      have hw : ua ∪' us ∈ sys.Con := hU _ hwU
      rcases Finset.mem_insert.mp hb with rfl | hb'
      · exact sys.ent_trans hw (hU ua hua)
          (fun y hy => sys.ent_refl hw (funion.mem.mpr (Or.inl hy))) hEnta
      · exact sys.ent_trans hw (hU us hus)
          (fun y hy => sys.ent_refl hw (funion.mem.mpr (Or.inr hy))) (hEnts b hb')

end IsFinitelyConsistent

end InfoSys
end DomainTheory.InformationSystems
```

5.4.7 Deductive closure

Definition (`InfoSys.deductiveClosure`). The deductive closure of a finitely consistent set U is the element whose carrier consists of every datum entailed by some finite subset of U. Consistency of finite subsets of that carrier follows by gathering their antecedents into one finite subset of U and applying the consistency consequence of multiary entailment; deductive closure of the carrier uses the same gathering together with transitivity of entailment.

Theorem (`InfoSys.deductiveClosure.subset`). A finitely consistent set is contained in the carrier of its deductive closure.

Proof. Each datum of U is entailed by its singleton, which is consistent by finite consistency. ?

`deductiveClosure.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    IsFinitelyConsistent

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def deductiveClosure (U : Set α) (hU : sys.IsFinitelyConsistent U) : sys.Element where
  carrier := {a | ∃ u : Finset α, (u : Set α) ⊆ U ∧ sys.Ent u a}
  consistent := by
    intro Y hY
    obtain ⟨w, hwU, hEnt⟩ := IsFinitelyConsistent.exists_entSet_of_subset_deductive sys
     U hU Y hY
    have hw : w ∈ sys.Con := hU w hwU
    exact sys.con_subset (EntSet.union_mem sys hw hEnt) (funion.subset_right _ _)
  closed := by
    intro Y a hY hEnt
    obtain ⟨w, hwU, hEntY⟩ := IsFinitelyConsistent.exists_entSet_of_subset_deductive
    sys U hU Y hY
    have hw : w ∈ sys.Con := hU w hwU
    have hYcon : Y ∈ sys.Con :=
      sys.con_subset (EntSet.union_mem sys hw hEntY) (funion.subset_right _ _)
    exact ⟨w, hwU, sys.ent_trans hw hYcon hEntY hEnt⟩

namespace deductiveClosure

theorem subset (U : Set α) (hU : sys.IsFinitelyConsistent U) :
    U ⊆ (sys.deductiveClosure U hU).carrier := by
  intro a ha
  refine ⟨{a}, ?_, sys.ent_refl (hU {a} ?_) (Finset.mem_singleton_self a)⟩
  · intro b hb
    have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    subst this
    exact ha
```

```
  · intro b hb
    have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    subst this
    exact ha

end deductiveClosure

end InfoSys
end DomainTheory.InformationSystems
```

5.4.8 Family union carrier

Definition (`InfoSys.familyUnionCarrier`). The family-union carrier of a family of elements is the set of data belonging to at least one member of the family.

Theorem (`InfoSys.familyUnionCarrier.finitelyConsistent_of_upperBound`). If every member of a family approximates an upper bound z, then the family-union carrier is finitely consistent.

Proof. Any finite subset of the union lies in z, so consistency of z applies. ?

`familyUnionCarrier.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    deductiveClosure

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def familyUnionCarrier (S : Set sys.Element) : Set α :=
  {a | ∃ x ∈ S, a ∈ x.carrier}

namespace familyUnionCarrier

theorem finitelyConsistent_of_upperBound (S : Set sys.Element) {z : sys.Element}
    (h : ∀ x ∈ S, x ≤ z) : sys.IsFinitelyConsistent (sys.familyUnionCarrier S) := by
  intro Y hY
  have hYZ : (Y : Set α) ⊆ z.carrier := by
    intro a ha
    obtain ⟨x, hx, ha'⟩ := hY ha
    exact h x hx ha'
  exact z.consistent Y hYZ

end familyUnionCarrier

end InfoSys
end DomainTheory.InformationSystems
```

5.4.9 Family suprema

Definition (`InfoSys.familySup`). When the family-union carrier of a family of elements is finitely consistent, the family supremum is the deductive closure of that union.

Theorem (`InfoSys.familySup.le`). If every member of the family approximates an upper bound z, then the family supremum approximates z.

Proof. Data in the supremum are entailed from finite pieces of the union, each of which lies in z, so z is deductively closed under those entailments. ?

Theorem (`InfoSys.familySup.le_of_mem`). Every member of the family approximates the family supremum.

Proof. The member's carrier sits in the family-union carrier, hence in the deductive closure. ?

Theorem (`InfoSys.familySup.exists_isLUB_iff`). A family of elements has a least upper bound if and only if its family-union carrier is finitely consistent.

Proof. An upper bound forces finite consistency of the union; conversely the family supremum is the least upper bound by the two approximation lemmas above. ?

`familySup.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    familyUnionCarrier

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def familySup (S : Set sys.Element)
    (hU : sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) : sys.Element :=
  sys.deductiveClosure (sys.familyUnionCarrier S) hU

namespace familySup

theorem le (S : Set sys.Element)
    (hU : sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) {z : sys.Element}
    (h : ∀ x ∈ S, x ≤ z) : sys.familySup S hU ≤ z := by
  intro a ha
  obtain ⟨u, huU, hEnt⟩ := ha
  have huZ : (u : Set α) ⊆ z.carrier := by
    intro b hb
    obtain ⟨x, hx, hb'⟩ := huU hb
    exact h x hx hb'
  exact z.closed u a huZ hEnt

theorem le_of_mem (S : Set sys.Element)
    (hU : sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) {x : sys.Element}
    (hx : x ∈ S) : x ≤ sys.familySup S hU := by
  intro a ha
```

```
  exact deductiveClosure.subset sys _ hU ⟨x, hx, ha⟩

theorem exists_isLUB_iff (S : Set sys.Element) :
    (∃ z : sys.Element, (∀ x ∈ S, x ≤ z) ∧ ∀ w, (∀ x ∈ S, x ≤ w) → z ≤ w) ↔
      Nonempty (sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) := by
  constructor
  · rintro ⟨z, hub, _⟩
    exact ⟨familyUnionCarrier.finitelyConsistent_of_upperBound sys S hub⟩
  · rintro ⟨hU⟩
    refine ⟨sys.familySup S hU, fun x hx => le_of_mem sys S hU hx,
      fun w hw => le sys S hU hw⟩

end familySup

end InfoSys
end DomainTheory.InformationSystems
```

5.4.10 Join

Definition (`InfoSys.join`). When the union of two elements' carriers is finitely consistent, their join is the deductive closure of that union.

Theorem (`InfoSys.join.le`). If both arguments approximate an upper bound z, then the join approximates z.

Proof. Antecedents drawn from the carrier union lie in z, so z absorbs the entailments defining the join. ?

Theorem (`InfoSys.join.le_left`). The left argument approximates the join.

Proof. Its carrier is contained in the union, hence in the deductive closure. ?

Theorem (`InfoSys.join.le_right`). The right argument approximates the join.

Proof. Symmetric to the left projection. ?

Theorem (`InfoSys.join.exists_iff`). Two elements have a common upper bound if and only if the union of their carriers is finitely consistent.

Proof. A common upper bound witnesses finite consistency of the union; conversely the join supplies a common upper bound by the two projections. ?

`join.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    deductiveClosure

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def join (x y : sys.Element)
```

```
    (h : sys.IsFinitelyConsistent (x.carrier ∪ y.carrier)) : sys.Element :=
  sys.deductiveClosure (x.carrier ∪ y.carrier) h

namespace join

theorem le (x y z : sys.Element)
    (h : sys.IsFinitelyConsistent (x.carrier ∪ y.carrier))
    (hx : x ≤ z) (hy : y ≤ z) : sys.join x y h ≤ z := by
  intro a ha
  obtain ⟨u, huU, hEnt⟩ := ha
  have huZ : (u : Set α) ⊆ z.carrier := by
    intro b hb
    rcases huU hb with hb' | hb'
    · exact hx hb'
    · exact hy hb'
  exact z.closed u a huZ hEnt

theorem le_left (x y : sys.Element)
    (h : sys.IsFinitelyConsistent (x.carrier ∪ y.carrier)) : x ≤ sys.join x y h :=
  fun _a ha => deductiveClosure.subset sys _ h (Or.inl ha)

theorem le_right (x y : sys.Element)
    (h : sys.IsFinitelyConsistent (x.carrier ∪ y.carrier)) : y ≤ sys.join x y h :=
  fun _a ha => deductiveClosure.subset sys _ h (Or.inr ha)

theorem exists_iff (x y : sys.Element) :
    (∃ z : sys.Element, x ≤ z ∧ y ≤ z) ↔
      Nonempty (sys.IsFinitelyConsistent (x.carrier ∪ y.carrier)) := by
  constructor
  · rintro ⟨z, hx, hy⟩
    exact ⟨fun Y hY => z.consistent Y (fun a ha => by
      rcases hY ha with h | h
      · exact hx h
      · exact hy h)⟩
  · rintro ⟨h⟩
    exact ⟨sys.join x y h, le_left sys x y h, le_right sys x y h⟩

end join

end InfoSys
end DomainTheory.InformationSystems
```

5.4.11 Factoid 4.3: joins / conditional sups

What about suprema (or sups or joins)? Here there is a slight problem, because in the way we set things up they *do not* always have to exist. Consider what happens with just two elements x and y. The set-theoretical union of the two elements as subsets of $D_{\mathbf{A}}$ need be neither consistent nor deductively closed; and even if $x \cup y$ is consistent, it need not be closed under entailment. So if we want sups *within* $|\mathbf{A}|$, we cannot get by as cheaply as we can with a domain of arbitrary sets for which the simple union is the answer.

Let us write the sup (supposing it exists in $|\mathbf{A}|$) as $x \sqcup y$. What do we have when we actually have it? By the lore of partially ordered sets, $x \sqcup y$ must be the *least element* in $|\mathbf{A}|$ which includes (in the sense of $\subseteq$) both x and y. Now we have just spoken about infs of families of elements: they exist if the family is non-empty. This indicates that $x \sqcup y$ exists exactly when there is at least one element z in $|\mathbf{A}|$ such that

$x \subseteq z$ and $y \subseteq z$. This turns out to be a way to say—entirely inside $|\mathbf{A}|$—that $x \cup y$ is consistent. In other words (and more generally) a sup of a subfamily exists if, and only if, the union of the family is consistent. (And in that case the sup is just the deductive closure of the union—that is, it is generally larger than the simple set-theoretical union.) This has an axiomatic version, but it is slightly complicated by the fact that sups do not always exist. In case $\top_{\mathbf{A}}$ exists, then we always have all the sups, and $|\mathbf{A}|$ is a *complete lattice*. (This is discussed in full axiomatic splendor in [**?**].) But, let it hasten to be added, not every complete lattice is isomorphic to a domain. The lattices that correspond to domains are the so-called *algebraic lattices*. (See the discussion loc. cit. for the additional axiomatics required.) When the top is missing, the partially ordered structures corresponding to domains are called *conditionally complete, algebraic cpo's*.

Theorem (`InfoSys.factoid_43`). Two elements have a common upper bound if and only if the union of their carriers is finitely consistent.

Proof. This is the binary join existence criterion. ?

`factoid_43.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    join

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

theorem factoid_43 :
    ∀ {α : Type*} [DecidableEq α] (sys : InfoSys α) (x y : sys.Element),
      (∃ z : sys.Element, x ≤ z ∧ y ≤ z) ↔
        Nonempty (sys.IsFinitelyConsistent (x.carrier ∪ y.carrier)) :=
  @join.exists_iff

end DomainTheory.InformationSystems.InfoSys
```

5.4.12 Chains

Definition (`InfoSys.Chain`). A family of elements is a chain when any two members are comparable under approximation.

`Chain.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    familySup

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

class Chain (S : Set sys.Element) : Prop where
  out : ∀ (x y : sys.Element), x ∈ S → y ∈ S → x ≤ y ∨ y ≤ x

end InfoSys
end DomainTheory.InformationSystems
```

5.4.13 Directed families

Definition (`InfoSys.IsDirected`). A family of elements is (upward) directed when any two members are approximated by some further member of the family.

Theorem (`InfoSys.IsDirected.of_IsChain`). Every chain is directed.

Proof. Of two comparable members, the larger is a common upper bound in the family. ?

Theorem (`InfoSys.IsDirected.exists_mem_of_subset_union`). If a nonempty directed family's union carrier contains a finite set of data, then some single member of the family contains that finite set.

Proof. Induct on the finite set. Pair the members witnessing the inserted datum and the remainder, then take a directed upper bound of those two members. ?

Theorem (`InfoSys.IsDirected.finitelyConsistent`). The union carrier of a nonempty directed family is finitely consistent.

Proof. Any finite subset of the union sits in some member, which is consistent. ?

`IsDirected.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Chain

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def IsDirected (S : Set sys.Element) : Prop :=
  ∀ (x y : sys.Element), x ∈ S → y ∈ S → ∃ z ∈ S, x ≤ z ∧ y ≤ z

namespace IsDirected

theorem of_IsChain {S : Set sys.Element} (h : sys.Chain S) : sys.IsDirected S := by
  intro x y hx hy
  rcases h.out x y hx hy with hxy | hyx
  · exact ⟨y, hy, hxy, le_rfl⟩
  · exact ⟨x, hx, le_rfl, hyx⟩

theorem exists_mem_of_subset_union (S : Set sys.Element) (hne : S.Nonempty)
```

```
    (hdir : sys.IsDirected S) (Y : Finset α)
    (hY : (Y : Set α) ⊆ sys.familyUnionCarrier S) :
    ∃ z ∈ S, (Y : Set α) ⊆ z.carrier := by
  induction Y using Finset.induction_on with
  | empty =>
    obtain ⟨z, hz⟩ := hne
    exact ⟨z, hz, fun _ ha => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 ha))⟩
  | insert a s _ha ih =>
    have ha' : ∃ x ∈ S, a ∈ x.carrier :=
      hY (Finset.mem_coe.2 (Finset.mem_insert_self a s))
    have hs' : (s : Set α) ⊆ sys.familyUnionCarrier S := by
      intro b hb
      exact hY (Finset.mem_coe.2 (Finset.mem_insert_of_mem (Finset.mem_coe.1 hb)))
    obtain ⟨xa, hxa, haS⟩ := ha'
    obtain ⟨xs, hxs, hsS⟩ := ih hs'
    obtain ⟨z, hz, hxa_le, hxs_le⟩ := hdir xa xs hxa hxs
    refine ⟨z, hz, ?_⟩
    intro b hb
    rcases Finset.mem_insert.mp (Finset.mem_coe.1 hb) with rfl | hb'
    · exact hxa_le haS
    · exact hxs_le (hsS (Finset.mem_coe.2 hb'))

theorem finitelyConsistent (S : Set sys.Element) (hne : S.Nonempty)
    (hdir : sys.IsDirected S) :
    sys.IsFinitelyConsistent (sys.familyUnionCarrier S) := by
  intro Y hY
  obtain ⟨z, _hz, hYZ⟩ := exists_mem_of_subset_union sys S hne hdir Y hY
  exact z.consistent Y hYZ

end IsDirected

end InfoSys
end DomainTheory.InformationSystems
```

5.4.14 Directed suprema

Definition (`InfoSys.directedSup`). The directed supremum of a nonempty directed family is the family supremum of that family, available because directedness yields finite consistency of the union carrier.

Theorem (`InfoSys.directedSup.carrier_eq_union`). The carrier of a directed supremum equals the family-union carrier.

Proof. Deductive closure always enlarges the union; the reverse uses that finite antecedents in the union sit in a single member, which is already deductively closed. ?

Theorem (`InfoSys.directedSup.le`). If every member of the family approximates an upper bound, then the directed supremum approximates that bound.

Proof. This is the corresponding property of the family supremum. ?

Theorem (`InfoSys.directedSup.le_of_mem`). Every member of the family approximates the directed supremum.

Proof. This is the corresponding property of the family supremum. ?

Theorem (`InfoSys.directedSup.compact_closure`). If a finitely generated element approximates a directed supremum, then it approximates some member of the family.

Proof. The generators lie in the union carrier, hence in some member by directedness; that member's deductive closure then dominates the finite element. ?

directedSup.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    IsDirected
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    closure_le_element

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def directedSup (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.IsDirected S) :
    sys.Element :=
  sys.familySup S (IsDirected.finitelyConsistent sys S hne hdir)

namespace directedSup

theorem carrier_eq_union (S : Set sys.Element) (hne : S.Nonempty)
    (hdir : sys.IsDirected S) :
    (sys.directedSup S hne hdir).carrier = sys.familyUnionCarrier S := by
  let hU := IsDirected.finitelyConsistent sys S hne hdir
  refine Set.Subset.antisymm ?_ (deductiveClosure.subset sys _ hU)
  intro a ha
  obtain ⟨u, huU, hEnt⟩ := (ha : a ∈ (sys.deductiveClosure _ hU).carrier)
  obtain ⟨z, hz, huZ⟩ := IsDirected.exists_mem_of_subset_union sys S hne hdir u huU
  exact ⟨z, hz, z.closed u a huZ hEnt⟩

theorem le (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.IsDirected S)
    {z : sys.Element} (h : ∀ x ∈ S, x ≤ z) : sys.directedSup S hne hdir ≤ z :=
  familySup.le sys S (IsDirected.finitelyConsistent sys S hne hdir) h

theorem le_of_mem (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.IsDirected S)
    {x : sys.Element} (hx : x ∈ S) : x ≤ sys.directedSup S hne hdir :=
  familySup.le_of_mem sys S (IsDirected.finitelyConsistent sys S hne hdir) hx

theorem compact_closure (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.
    IsDirected S)
    {u : Finset α} (hu : u ∈ sys.Con)
    (hle : sys.closure u hu ≤ sys.directedSup S hne hdir) :
    ∃ z ∈ S, sys.closure u hu ≤ z := by
  have huU : (u : Set α) ⊆ sys.familyUnionCarrier S := by
    intro a ha
    have : a ∈ (sys.directedSup S hne hdir).carrier :=
      hle (closure.subset sys hu (Finset.mem_coe.2 ha))
```

```
    rwa [carrier_eq_union sys S hne hdir] at this
  obtain ⟨z, hz, huZ⟩ := IsDirected.exists_mem_of_subset_union sys S hne hdir u huU
  exact ⟨z, hz, sys.closure_le_element z hu huZ⟩

end directedSup

end InfoSys
end DomainTheory.InformationSystems
```

5.4.15 Chain suprema

Definition (`InfoSys.chainSup`). The chain supremum of a nonempty chain is the directed supremum of that family, available because every chain is directed.

`chainSup.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    directedSup

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def chainSup (S : Set sys.Element) (hne : S.Nonempty) (hchain : sys.Chain S) : sys.
    Element :=
  sys.directedSup S hne (IsDirected.of_IsChain sys hchain)

end InfoSys
end DomainTheory.InformationSystems
```

5.4.16 Factoid 4.4: chains & directed sups

There is, however, an important case in $|\mathbf{A}|$ where the union is consistent and is, in fact, the sup in the domain. Suppose we have a sequence of elements such that

$$x_0 \subseteq x_1 \subseteq x_2 \subseteq \cdots \subseteq x_n \subseteq x_{n+1} \subseteq \cdots$$

As n increases, we can say that the elements x_n are getting "better and better"; thus, they must be approaching something even more desirable *in the limit*. Let

$$y = \bigcup_{n=0}^{\infty} x_n,$$

then there is no question that y is a subset of $D_{\mathbf{A}}$. But is it an element? Consistency, recall, means that every *finite* subset is in $\mathrm{Con}_{\mathbf{A}}$. The trick is that a finite subset of y must be a subset of one of the x_n, because the sequence of elements x_n is increasing. But all the terms of our given sequence are elements, and so, consistent; therefore, every finite subset of y is consistent.

The same argument also shows that the set y of data objects is closed under entailment, because $\vdash_{\mathbf{A}}$ is defined as a relation between finite (consistent) sets and data objects. It follows that since each x_n is a set closed under $\vdash_{\mathbf{A}}$, then so is y. In other words, y is indeed an element. Otherwise said, the domain $|\mathbf{A}|$ is closed under the formation of unions of increasing chains of elements, and the union is, of course, the sup in the sense of the partial ordering. Again we have proved closure rather than assuming it, and the necessary properties of the sup follow from its definition as a union.

This last argument can be generalized—as is well known to mathematicians—to the case of *directed* families of elements. The word "directed," or better "upward-directed," means that every finite number of elements in the family is included in some further element of the family; chains always have this property. A good example of the use of directed sets of elements figures in the discussion of the *finite elements* of the domain.

Theorem (`InfoSys.factoid_44`). The carrier of the directed supremum of a nonempty directed family equals the family-union carrier.

Proof. This is the carrier-equality property of directed suprema. ?

`factoid_44.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    chainSup

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

theorem factoid_44 :
    ∀ {α : Type*} [DecidableEq α] (sys : InfoSys α) (S : Set sys.Element)
      (hne : S.Nonempty) (hdir : sys.IsDirected S),
      (sys.directedSup S hne hdir).carrier = sys.familyUnionCarrier S :=
  @directedSup.carrier_eq_union

end DomainTheory.InformationSystems.InfoSys
```

5.4.17 Bottom as empty closure

Theorem (`InfoSys.botElement.eq_closure_empty`). The bottom element equals the entailment closure of the empty set of data.

Proof. Each side approximates the other by transitivity of entailment between the empty set and the singleton of the least-informative datum. ?

`botElement_eq_closure_empty.lean`

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.botElement
```

```
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

namespace botElement

theorem eq_closure_empty :
    sys.botElement = sys.closure (∅ : Finset α) sys.con_empty := by
  refine le_antisymm ?_ ?_
  · intro a ha
    exact sys.ent_trans sys.con_empty (sys.con_sing sys.bot)
      (fun y hy => by
        have : y = sys.bot := Finset.mem_singleton.mp hy
        subst this
        exact sys.ent_bot sys.con_empty)
      ha
  · intro a ha
    exact sys.ent_trans (sys.con_sing sys.bot) sys.con_empty
      (fun _ hy => False.elim (Finset.notMem_empty _ hy)) ha

end botElement

end InfoSys
end DomainTheory.InformationSystems
```

5.4.18 Finite approximants

Definition (`InfoSys.finiteApproximants`). The finite approximants of an element x are the entailment closures of consistent finite sets of data contained in x.

Theorem (`InfoSys.finiteApproximants.nonempty`). Every element has a nonempty set of finite approximants.

Proof. The bottom element is the closure of the empty set, which is contained in every carrier. ?

Theorem (`InfoSys.finiteApproximants.directed`). The finite approximants of an element form a directed family.

Proof. Closures of two consistent finite subsets of x are jointly dominated by the closure of a consistent finite upper bound still inside x. ?

Theorem (`InfoSys.finiteApproximants.eq_directedSup`). Every element equals the directed supremum of its finite approximants.

Proof. Each datum of x lies in some finite approximant's carrier, and every finite approximant approximates x by the universal property of closure. ?

`finiteApproximants.lean`

```
module
```

```
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    directedSup
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    botElement_eq_closure_empty
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    closures_directed
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    closure_le_element

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

def finiteApproximants (x : sys.Element) : Set sys.Element :=
  {y | ∃ (u : Finset α) (hu : u ∈ sys.Con), ↑u ⊆ x.carrier ∧ y = sys.closure u hu}

namespace finiteApproximants

theorem nonempty (x : sys.Element) : (sys.finiteApproximants x).Nonempty := by
  refine ⟨sys.botElement, ∅, sys.con_empty, ?_, botElement.eq_closure_empty sys⟩
  intro a ha
  exact False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 ha))

theorem directed (x : sys.Element) : sys.IsDirected (sys.finiteApproximants x) := by
  intro y₁ y₂ hy₁ hy₂
  obtain ⟨u, hu, huX, rfl⟩ := hy₁
  obtain ⟨v, hv, hvX, rfl⟩ := hy₂
  obtain ⟨w, hw, hwX, huw, hvw⟩ := sys.closures_directed x hu hv huX hvX
  exact ⟨sys.closure w hw, ⟨w, hw, hwX, rfl⟩, huw, hvw⟩

theorem eq_directedSup (x : sys.Element) :
    x = sys.directedSup (sys.finiteApproximants x)
      (nonempty sys x) (directed sys x) := by
  refine le_antisymm ?_ ?_
  · intro a ha
    have : a ∈ sys.approxUnion x := (sys.mem_element_iff_mem_closure x a).1 ha
    obtain ⟨u, hu, huX, ha'⟩ := this
    rw [directedSup.carrier_eq_union sys]
    exact ⟨sys.closure u hu, ⟨u, hu, huX, rfl⟩, ha'⟩
  · apply directedSup.le sys
    intro y hy
    obtain ⟨u, hu, huX, rfl⟩ := hy
    exact sys.closure_le_element x hu huX

end finiteApproximants

end InfoSys
end DomainTheory.InformationSystems
```

5.4.19 Factoid 4.5: finite approximants

As we have remarked several times, any consistent set of data objects *generates* an element by closing it up under the entailment relation; thus, in particular, any set $u \in \mathrm{Con}$ generates an element. Let us write

$$\overline{u} = \{X \in D_{\mathbf{A}} \mid u \vdash_{\mathbf{A}} X\}$$

for the *closure* of u under entailment. (This notation could be used for any consistent subset of $D_{\mathbf{A}}$, but the case of finitely generated elements is especially important.) Such a $\overline{u}$ is always an element of $|\mathbf{A}|$; the totality of such elements form, by definition, the *finite elements* of the domain. The notation $\overline{u}$ should of course be decorated with a subscript $\mathbf{A}$, but, as there is no good place to write it in, we leave it out.

We have for all elements x of the domain the basic formula:

$$x = \bigcup \{\overline{u} \mid u \in \mathrm{Con}_{\mathbf{A}} \text{ and } u \subseteq x\}.$$

This can be read intuitively as saying that every element of the domain is the *limit* of its *finite approximations*. And, having used the word "limit" so often, we ought now to say something about how to make the meaning of this word correct mathematically. Note that in the limit representation for x, the union is a directed one. (Why?)

Theorem (`InfoSys.factoid_45`). Every element equals the directed supremum of its finite approximants.

Proof. This is the directed-supremum representation of an element by its finite approximants. ?

`factoid_45.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    finiteApproximants

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

theorem factoid_45 :
    ∀ {α : Type*} [DecidableEq α] (sys : InfoSys α) (x : sys.Element),
      x = sys.directedSup (sys.finiteApproximants x)
        (finiteApproximants.nonempty sys x) (finiteApproximants.directed sys x) :=
  @finiteApproximants.eq_directedSup

end DomainTheory.InformationSystems.InfoSys
```

5.4.20 Scott continuous maps

Theorem (`InfoSys.nonempty_image`). The direct image of a nonempty set under a function is nonempty.

Proof. Apply the function to a chosen witness. ?

Theorem (`InfoSys.directed_image`). The direct image of a directed family under a monotone map is directed.

Proof. Lift a directed upper bound of two preimages and apply monotonicity. ?

Definition (`InfoSys.ScottContinuous`). A Scott-continuous map between domains is a monotone function of elements that preserves directed suprema (sending a directed supremum to the directed supremum of the image family).

Theorem (`InfoSys.ScottContinuous.mono`). Every Scott-continuous map is monotone.

Proof. This is the monotonicity field of the structure. ?

ScottContinuous.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    factoid_45

@[expose] public section

namespace DomainTheory.InformationSystems
namespace InfoSys

open DomainTheory.InformationSystems.Constructive

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

theorem nonempty_image {X Y : Type*} {S : Set X} (hne : S.Nonempty) (f : X → Y) :
    (f '' S).Nonempty :=
  hne.image f

theorem directed_image {A : InfoSys α} {B : InfoSys β} {S : Set A.Element}
    (hdir : A.IsDirected S) (f : A.Element → B.Element)
    (hmono : ∀ {x y}, x ≤ y → f x ≤ f y) : B.IsDirected (f '' S) := by
  intro x y hx hy
  obtain ⟨x₀, hx₀, rfl⟩ := hx
  obtain ⟨y₀, hy₀, rfl⟩ := hy
  obtain ⟨z₀, hz₀, hxz, hyz⟩ := hdir x₀ y₀ hx₀ hy₀
  exact ⟨f z₀, ⟨z₀, hz₀, rfl⟩, hmono hxz, hmono hyz⟩

structure ScottContinuous (A : InfoSys α) (B : InfoSys β) where
  toFun : A.Element → B.Element
  mono' : ∀ {x y : A.Element}, x ≤ y → toFun x ≤ toFun y
  map_directedSup :
    ∀ (S : Set A.Element) (hne : S.Nonempty) (hdir : A.IsDirected S),
      toFun (A.directedSup S hne hdir) =
        B.directedSup (toFun '' S) (nonempty_image hne toFun)
          (directed_image hdir toFun mono')

namespace ScottContinuous

variable {A : InfoSys α} {B : InfoSys β}

theorem mono (f : ScottContinuous A B) {x y : A.Element} (h : x ≤ y) :
    f.toFun x ≤ f.toFun y :=
  f.mono' h

end ScottContinuous
```

```
end InfoSys
end DomainTheory.InformationSystems
```

5.4.21 The basic Scott-opens

Definition (`InfoSys.basicOpen`). The basic Scott-open associated to a finite set of data u is the set of elements whose carriers contain u.

Theorem (`InfoSys.basicOpen.inter`). The intersection of two basic Scott-opens is the basic Scott-open of the finite union of their data sets.

Proof. Membership in both opens is membership of each finite set in the carrier, which is membership of their finite union. ?

Theorem (`InfoSys.basicOpen.mem`). An element lies in the basic Scott-open of u if and only if u is contained in its carrier.

Proof. Immediate from the definition. ?

Theorem (`InfoSys.basicOpen.eq_of_eq`). Elements that lie in exactly the same basic Scott-opens are equal.

Proof. Membership in the singleton basic opens recovers carrier membership of each datum, so the carriers coincide. ?

basicOpen.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    ScottContinuous

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

def basicOpen (A : InfoSys α) (u : Finset α) : Set A.Element :=
  {x | ↑u ⊆ x.carrier}

namespace basicOpen

theorem inter (A : InfoSys α) (u v : Finset α) :
    A.basicOpen u ∩ A.basicOpen v = A.basicOpen (u ∪' v) := by
  ext x
  constructor
  · intro ⟨hu, hv⟩ a ha
    rcases funion.mem.mp (Finset.mem_coe.1 ha) with h | h
    · exact hu (Finset.mem_coe.2 h)
    · exact hv (Finset.mem_coe.2 h)
```

```
  · intro huv
    refine ⟨?_, ?_⟩
    · intro a ha
      exact huv (Finset.mem_coe.2 (funion.mem.mpr (Or.inl (Finset.mem_coe.1 ha))))
    · intro a ha
      exact huv (Finset.mem_coe.2 (funion.mem.mpr (Or.inr (Finset.mem_coe.1 ha))))

theorem mem {A : InfoSys α} {u : Finset α} {x : A.Element} :
    x ∈ A.basicOpen u ↔ ↑u ⊆ x.carrier := Iff.rfl

theorem eq_of_eq (A : InfoSys α) {x y : A.Element}
    (h : ∀ u : Finset α, x ∈ A.basicOpen u ↔ y ∈ A.basicOpen u) : x = y := by
  refine le_antisymm ?_ ?_
  · intro a ha
    have : x ∈ A.basicOpen {a} := by
      intro b hb
      have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
      subst this
      exact ha
    exact ((h {a}).1 this) (Finset.mem_coe.2 (Finset.mem_singleton_self a))
  · intro a ha
    have : y ∈ A.basicOpen {a} := by
      intro b hb
      have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
      subst this
      exact ha
    exact ((h {a}).2 this) (Finset.mem_coe.2 (Finset.mem_singleton_self a))

end basicOpen

end InfoSys
end DomainTheory.InformationSystems
```

5.4.22 Approximable maps as Scott continuous

Theorem (`InfoSys.ApproximableMap.toElement_directedSup`). The element map of an approximable mapping preserves directed suprema.

Proof. Finite relational witnesses for data in the image of the directed supremum sit in some member of the family by directedness; the reverse inequality is monotonicity of the element map. ?

Definition (`InfoSys.ApproximableMap.ofScottContinuous`). From a Scott-continuous map one obtains an approximable mapping by relating consistent finite sets u and v when v lies in the image of the closure of u. Consistency and monotonicity of the relation follow from those of the Scott-continuous map and of closure.

Definition (`InfoSys.ApproximableMap.toScottContinuous`). From an approximable mapping one obtains a Scott-continuous map by taking its element map, using that the element map is monotone and preserves directed suprema.

Theorem (`InfoSys.ApproximableMap.toElement_ofScottContinuous`). Passing from a Scott-continuous map to an approximable mapping and back recovers the original map on elements.

Proof. One inequality is monotonicity applied to closures of relational antecedents; the other uses the finite-approximant representation of the argument together with preservation of directed suprema. ?

ApproximableMapScott.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    ScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    directedSup
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    basicOpen

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

namespace ApproximableMap

variable {A : InfoSys α} {B : InfoSys β}

theorem toElement_directedSup (f : ApproximableMap A B)
    (S : Set A.Element) (hne : S.Nonempty) (hdir : A.IsDirected S) :
    f.toElement (A.directedSup S hne hdir) =
      B.directedSup (f.toElement '' S) (nonempty_image hne f.toElement)
        (directed_image hdir f.toElement f.toElement_mono) := by
  refine le_antisymm ?_ ?_
  · intro Y ⟨u, hu, hrel⟩
    have huU : (u : Set α) ⊆ A.familyUnionCarrier S := by
      intro a ha
      have : a ∈ (A.directedSup S hne hdir).carrier := hu ha
      rwa [directedSup.carrier_eq_union A S hne hdir] at this
    obtain ⟨z, hz, huZ⟩ := IsDirected.exists_mem_of_subset_union A S hne hdir u huU
    have : Y ∈ (f.toElement z).carrier := ⟨u, huZ, hrel⟩
    rw [directedSup.carrier_eq_union B]
    exact ⟨f.toElement z, ⟨z, hz, rfl⟩, this⟩
  · apply directedSup.le B
    intro y hy
    obtain ⟨x, hx, rfl⟩ := hy
    exact f.toElement_mono (directedSup.le_of_mem A S hne hdir hx)

def ofScottContinuous (g : ScottContinuous A B) : ApproximableMap A B where
  rel u v := ∃ hu : u ∈ A.Con, v ∈ B.Con ∧ ↑v ⊆ (g.toFun (A.closure u hu)).carrier
  rel_dom := fun ⟨hu, _, _⟩ => hu
  rel_cod := fun ⟨_, hv, _⟩ => hv
  empty_rel := by
    refine ⟨A.con_empty, B.con_empty, ?_⟩
    intro b hb
    exact False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 hb))
  union_right := by
```

```
    intro u v v' ⟨hu, hv, hsub⟩ ⟨_hu', hv', hsub'⟩
    refine ⟨hu, ?_, ?_⟩
    · have hsubUV : ↑(v ∪' v') ⊆ (g.toFun (A.closure u hu)).carrier := by
        intro b hb
        rcases funion.mem.mp (Finset.mem_coe.1 hb) with h | h
        · exact hsub (Finset.mem_coe.2 h)
        · exact hsub' (Finset.mem_coe.2 h)
      exact (g.toFun (A.closure u hu)).consistent (v ∪' v') hsubUV
    · intro b hb
      rcases funion.mem.mp (Finset.mem_coe.1 hb) with h | h
      · exact hsub (Finset.mem_coe.2 h)
      · exact hsub' (Finset.mem_coe.2 h)
  mono := by
    intro u u' v v' ⟨hu, hv, hsub⟩ hEntUU' hEntVV' hu' hv'
    refine ⟨hu', hv', ?_⟩
    have hclos : A.closure u hu ≤ A.closure u' hu' :=
      A.closure_le_of_entSet hu hu' hEntUU'
    intro b hb
    have hEntb : B.Ent v b := hEntVV' b hb
    have hb_in : b ∈ (g.toFun (A.closure u hu)).carrier :=
      (g.toFun (A.closure u hu)).closed v b hsub hEntb
    exact ScottContinuous.mono g hclos hb_in

def toScottContinuous (f : ApproximableMap A B) : ScottContinuous A B where
  toFun := f.toElement
  mono' := f.toElement_mono
  map_directedSup := f.toElement_directedSup

theorem toElement_ofScottContinuous (g : ScottContinuous A B) (x : A.Element) :
    (ofScottContinuous g).toElement x = g.toFun x := by
  refine le_antisymm ?_ ?_
  · intro Y ⟨u, hu, ⟨huCon, _, hsub⟩⟩
    have : A.closure u huCon ≤ x := A.closure_le_element x huCon hu
    exact ScottContinuous.mono g this (hsub (Finset.mem_coe.2 (Finset.
    mem_singleton_self Y)))
  · intro Y hY
    let S := A.finiteApproximants x
    have hne := finiteApproximants.nonempty A x
    have hdir := finiteApproximants.directed A x
    have hx : x = A.directedSup S hne hdir := finiteApproximants.eq_directedSup A x
    have hdirImg := directed_image hdir g.toFun g.mono'
    have hneImg := nonempty_image hne g.toFun
    have hg : g.toFun x = B.directedSup (g.toFun '' S) hneImg hdirImg := by
      have h1 : g.toFun x = g.toFun (A.directedSup S hne hdir) := congrArg g.toFun hx
      have h2 : g.toFun (A.directedSup S hne hdir) =
          B.directedSup (g.toFun '' S) hneImg hdirImg :=
        g.map_directedSup S hne hdir
      exact h1.trans h2
    have hY' : Y ∈ (B.directedSup (g.toFun '' S) hneImg hdirImg).carrier := by
      rw [← hg]; exact hY
    have hY'' : Y ∈ B.familyUnionCarrier (g.toFun '' S) := by
      rwa [← directedSup.carrier_eq_union B (g.toFun '' S) hneImg hdirImg]
    obtain ⟨z, hz, hYz⟩ := hY''
    obtain ⟨y, hy, rfl⟩ := hz
    obtain ⟨u, hu, huX, rfl⟩ := hy
    refine ⟨u, huX, ⟨hu, B.con_sing Y, ?_⟩⟩
```

```
    intro b hb
    have : b = Y := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    subst this
    exact hYz

end ApproximableMap

end InfoSys
end DomainTheory.InformationSystems
```

5.4.23 Factoid 4.6: topology / Scott continuity

Topological considerations. Geometrically speaking, a topological space is a collection of "points" which group themselves in various "neighborhoods" providing thereby a sense of "nearness." More specifically, a neighborhood of a point in a *metric space* consists of all those points within a certain positive "distance" from the given point; that is, nearness is limited, but it is not pushed to *zero*. The major reason for leaving some "breathing space" around the point comes out in defining continuity of functions.

A function f from one topological space to another is said to be *continuous* provided that, for every point x of the first space, and for every neighborhood in the second space around $f(x)$, it is

possible to find—in the first space—a neighborhood around x with the following property. If the function is restricted to this neighborhood, then its values lie entirely in the given neighborhood around $f(x)$. In the metric case we can say that if we do not want the function to wander any great distance from $f(x)$, then there is a restricted distance around x giving a neighborhood over which the function values stay as close to $f(x)$ as specified. This keeps the function from jumping around wildly, since small variations in x lead to only small variations in $f(x)$.

The intuition about continuity is no doubt very clear in the more geometric examples, and geometric intuition can carry us quite far. But the spaces obtained from domain theory are not really geometric spaces, so we need to make some shifts in our metaphors. The geometric notion of a point usually implies that two points are perfectly distinguishable: if x and y are different, then they have disjoint neighborhoods. This is certainly true in the metric case, where, if two points are different, then they are at a positive distance apart. Early on in the study of topological spaces, however, it was found that not all spaces were metric, and weaker separation properties of pairs of distinct points were uncovered. One of the very weak versions of such a condition is called the "T_0-axiom." The best way to put it is that if two points have the same neighborhoods, then they are the same. The contrapositive statement sounds odd: if two points are distinct, then there is a neighborhood that contains one but not the other. The oddity of this statement lies in the feeling that you cannot make up your mind over which is the better point.

In our domains there is already a notion of "betterness" which proves to be very closely related to the implicit neighborhood structure. The reason why our domains, as spaces of points, are not "geometric" in the familiar sense is that they contain partial elements. A geometric point, on the other hand, is "perfect" or totally determined; it cannot be made better than it already is. The metric in common spaces is measuring something other than betterness, for there are competing notions of approximation afoot here. Thus, we must keep our special goals in defining domains clearly in mind to avoid confusion.

Consider an information system $\mathbf{A}$. For each $u \in \mathrm{Con}_{\mathbf{A}}$, we define a corresponding neighborhood of $|\mathbf{A}|$ by the equation:

$$[u]_{\mathbf{A}} = \{x \in |\mathbf{A}| \mid u \subseteq x\}.$$

The neighborhoods of an element x are all those sets $[u]_{\mathbf{A}}$, where $u \subseteq x$. Note that this is the same as saying $\overline{u} \subseteq x$ or $\overline{u}$ approximates x. Indeed, the neighborhoods are in a one-one correspondence with the finite elements of $|\mathbf{A}|$. A neighborhood collects together all those elements sharing the same finite amount of information.

If both $u \subseteq x$ and $v \subseteq x$, then it is easy to see that the intersection of the two neighborhoods is again a neighborhood of x by virtue of the formula:

$$[u]_{\mathbf{A}} \cap [v]_{\mathbf{A}} = [u \cup v]_{\mathbf{A}}.$$

In case u and v are inconsistent with each other, then the intersection will be the empty set $\emptyset$. Because every element of our domain $|\mathbf{A}|$ is the union of the finite elements it contains, it is trivial to prove that two elements with exactly the same neighborhoods are the same. For these reasons, then, it follows that $|\mathbf{A}|$ is a T_0-topological space. The immediate reaction to this intelligence is: So what?

The answer to this scepticism will appear in the best light when we discuss in the next section the notion of function or mapping appropriate to our domains. The argument will be based on the

extremely elementary form of the definition that characterizes the general continuous function together with the fact that the geometric intuition is suggestive of theorems to prove. In Section 6 we will find that the space of all continuous functions between two given domains is also a domain, a most useful result that is interesting topologically. But further interest for Domain Theory comes from the circumstance that there is a connection between geometry and domains. We cannot go into the full details here, but I can make some brief remarks.

Before continuing that discussion, though, it should be remarked that once the topology of the domain has been uncovered, then the unions of chains of elements (or directed sets of elements) are topological limits according to accepted general definitions. Thus, what we felt intuitively was a limiting process (the getting better and better up to the union) is in fact a limit formation in a precise mathematical sense. This helps convince us that we are on the right track.

Total elements as a space. In any domain it can be proved that any element x can be extended to a *total element* t, which is characterized by the property that it is a *maximal* element of the partial ordering. (Perhaps there are many such t, but there is always at least one with $x \subseteq t$.) Recall that the set of all total elements of $|\mathbf{A}|$ is denoted by $\mathrm{Tot}_{\mathbf{A}}$. Because the set $\mathrm{Tot}_{\mathbf{A}} \subseteq |\mathbf{A}|$, it is a topological space itself. Suppose s and t are two distinct total elements, then they must be inconsistent with each other—but we shall prove more. If every finite consistent $u \subseteq s$ also satisfied $u \subseteq t$, then $s \subseteq t$ would follow. But this is impossible, because s is maximal. Let, then, $u \in \mathrm{Con}_{\mathbf{A}}$ with $u \subseteq s$ be chosen so that $u \subseteq t$ fails. Now u must be inconsistent with t, since otherwise there would be an element containing them both. It follows that u must be inconsistent with some $v \in \mathrm{Con}_{\mathbf{A}}$ with $v \subseteq t$. In this way we show that the neighborhoods $[u]_{\mathbf{A}}$ and $[v]_{\mathbf{A}}$ are disjoint.

This argument proves that $\mathrm{Tot}_{\mathbf{A}}$ is a *Hausdorff space* (or T_2-space). We cannot go further into the details here, but it is also a totally disconnected, compact Hausdorff space. (These spaces—well known from studies of Logic and the Theory of Boolean Algebras—are also zero-dimensional.) Assuming the countability of $\mathrm{Con}_{\mathbf{A}}$, which is not such a bold assumption, all such spaces can be conveniently embedded into the real line so as to preserve the topological structure. Perhaps this does not seem so surprising, but it shows that the topological nature of $\mathrm{Tot}_{\mathbf{A}}$, the uppermost level of the T_0 space $|\mathbf{A}|$, is indeed quite geometric. And it can also be shown that any arbitrary continuous function on $\mathrm{Tot}_{\mathbf{A}}$ into *any* topological space can be extended to a continuous function on $|\mathbf{A}|$, so the whole domain proves to be quite a "roomy" space with many good connections to more usual topological spaces.

A note on neighborhood structures. Topological spaces can be defined in several ways, and in general

they do not carry a preferred neighborhood structure. (For example, there is no topological significance to the usual rational intervals we use to define neighborhoods for the real line: any similarly dense set of points would do in place of the rational numbers.) Domains, the way we have defined them, however, do have preferred neighborhoods. The set of finite elements of $|\mathbf{A}|$ can be defined topologically, and as we have seen they determine a convenient set of neighborhoods. (The reason for all this is the so-called zero-dimensional character of the domains. There is a higher-dimensional theory, but this leads to continuous lattices and can only be explained using the kind of presentation in [**?**]. It is too long-winded a story for the present paper. The higher-dimensional domains do not have a preferred set of neighborhoods, however.)

Having realized this, it is a short step to giving an algebraic axiomatization of the neighborhood structure. [**?**] contains a presentation in terms of the equivalent notion of finite element. [2] turns the story around and defines a set-theoretical representation of a neighborhood as just a family of sets containing a largest "neighborhood" and closed under forming the intersection of any two sets in the family that contain a third. The set-theoretical form of this definition is very simple, and [2] spells out the details of how the theory of domains can be based on this starting point. Subsequent investigation, however, has convinced the author that the present approach through information systems is even simpler and better for a number of the essential constructions. The theory introduced by [**?**] of f-spaces is another axiomatization somewhat intermediate between the topological and the lattice-theoretic. Domains in the present sense then become complete f-spaces. There are many interesting aspects to this method, but the author feels that information systems are about as elementary and familiar as we can get, and they are therefore more suggestive of new constructions.

Theorem (`InfoSys.ApproximableMap.factoid_46`). Passing from a Scott-continuous map to an approximable mapping and back recovers the original map on elements.

Proof. This is the recovery property of the Scott-continuous–to–approximable correspondence. ?

`factoid_46.lean`

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    ApproximableMapScott

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys.ApproximableMap

theorem factoid_46 :
    ∀ {α β : Type*} [DecidableEq α] [DecidableEq β] {A : InfoSys α} {B : InfoSys β}
      (g : ScottContinuous A B) (x : A.Element),
      (ofScottContinuous g).toElement x = g.toFun x :=
  @toElement_ofScottContinuous

end DomainTheory.InformationSystems.InfoSys.ApproximableMap
```

5.5 Approximable mappings between domains

<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

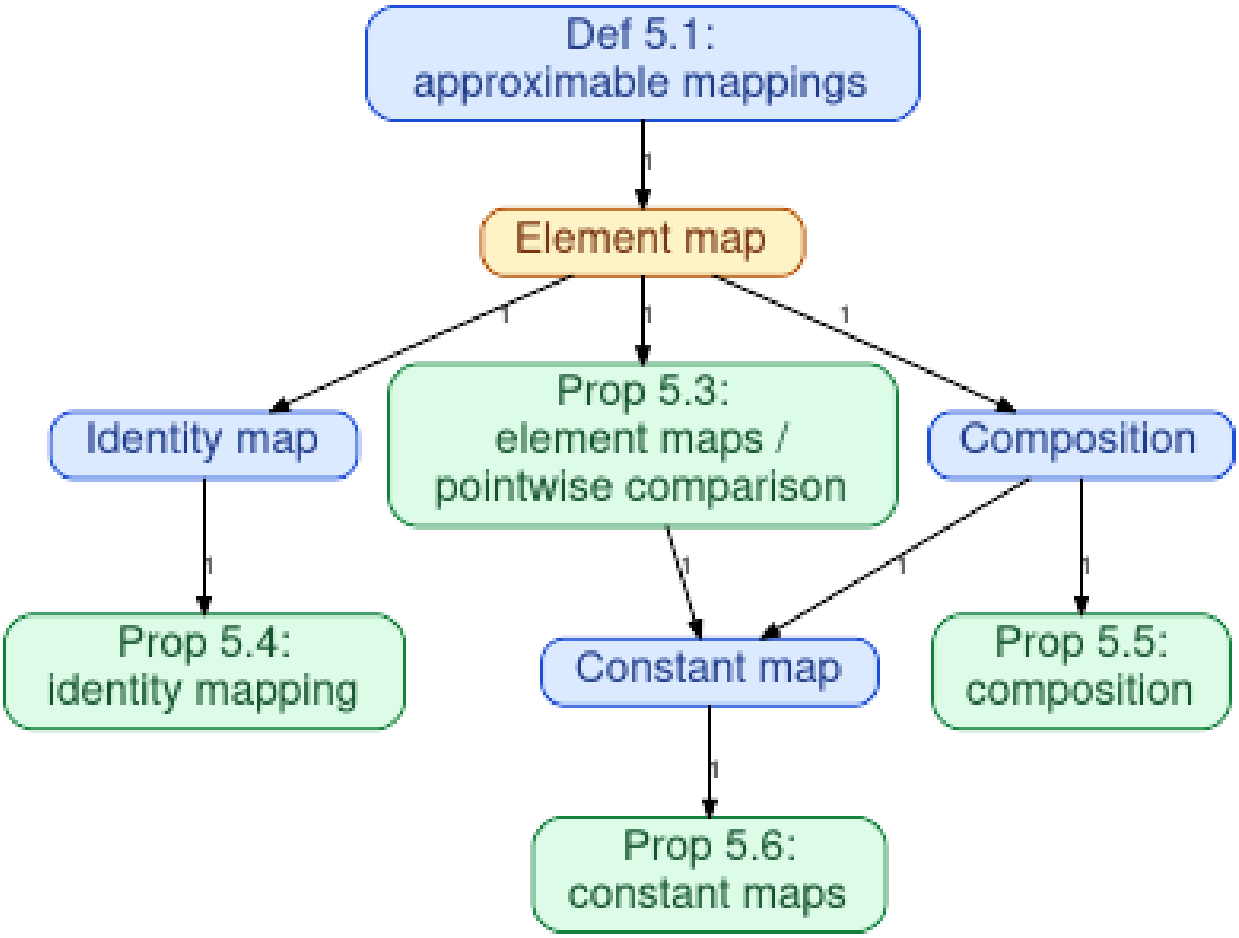


Figure 58: Diagram 23

5.5.1 Introduction

Once a general notion of set or domain has been defined mathematically, the next major issue is: How are the different domains to be related one to another? The answer to this important question can many times be given by defining an appropriate concept of mapping between domains. The answer need not be unique; the same collection of domains may support more than one idea of function depending on the special aspects of the domains that need to be brought out. In the case of the kinds of domains being studied here, there are two principal answers, one of which is to be explained in this section.

In order to understand an appropriate notion of a mapping or a function between domains constructed from information systems, we have to refer back to the way in which elements are introduced. As we have seen in Section 3, an element is a consistent, closed set of data objects. To generate an element, one has to generate more and more of the finite consistent subsets of the element. Note that the separate data objects have to be grouped into these finite consistent sets, because the entailment relation may require a finite set of arbitrary size on the left in making the necessary entailments for closure. It is this kind of passage from a finite set to its closure that is to be generalized in defining mappings.

Consider two domains. To map from one to another, some information about a possible element of the first is presented as input to the function f. Then as output the function f starts generating an element. If

the input were u, a consistent set of the first domain, then part of the output in the second domain might be a consistent set v. We could say that there is an input/output relationship set up by f, and indicate by $u\ f\ v$ that this relation holds going from u in the first domain to v in the second. Of course to get the full effect of f, it is necessary to take all the v's related to a given u, because even a small finite amount of input may cause an infinite amount of output. But every element of a domain is just the sum total of its finite subsets (finite approximations), so it is sufficient to make the mapping relationship go between finite sets. Here is the formal definition with the exact conditions that the relation f must satisfy.

5.5.2 Definition 5.1: approximable mappings

DEFINITION 5.1. Let $\mathbf{A}$ and $\mathbf{B}$ be two given information systems. An *approximable mapping* $f : \mathbf{A} \to \mathbf{B}$ is a binary relation between the two sets $\mathrm{Con}_{\mathbf{A}}$ and $\mathrm{Con}_{\mathbf{B}}$ such that:

(i) $\emptyset\ f\ \emptyset$;

(ii) $u\ f\ v$ and $u\ f\ v'$ always imply $u\ f\ (v \cup v')$; and

(iii) $u' \vdash_{\mathbf{A}} u$, $u\ f\ v$, and $v \vdash_{\mathbf{B}} v'$ always imply $u'\ f\ v'$.

We say that $\mathbf{A}$ is the *source* of f and $\mathbf{B}$ is the *target*. ■

Intuitively the relationship $u\ f\ v$ is an input/output passage which can be read informally as: "if you are willing to give at least u amount of information about the argument, then the mapping f is willing to give at least v amount of information about the value (and possibly even more—if you are patient)." Condition (i) means that no (non-trivial) information about the input merits no information about the output. Condition (ii) implies that all the contributions to the output from a fixed input are consistent, and in fact the union of two of the output sets is again an output set. Output corresponding to a fixed input is cumulative. Condition (iii) assures us that the mapping relation f works in harmony with the two entailment relations: if a certain relationship holds, and if the left-hand side is strengthened while the right-hand side is weakened, then the mapping relation must continue to hold. What we must discuss next is what this means for elements.

Before defining the notion of function value, however, it is useful to remark that in the definition of approximable mapping the form of the statement could be simplified by reducing sets on the right to their elements. Specifically, we note the equivalence:

$$u\ f\ v \text{ iff } u\ f\ \{Y\} \text{ for all } Y \in v.$$

In other words an approximable mapping is completely determined by the relation set up between consistent sets on the left and single data objects on the right.

Definition (`InfoSys.ApproximableMap`). An approximable mapping from an information system $\mathbf{A}$ to an information system $\mathbf{B}$ is a relation between finite sets of data of $\mathbf{A}$ and of $\mathbf{B}$ that relates only consistent sets on each side, relates the empty set to the empty set, is closed under funion on the right for a fixed left argument, and is monotone with respect to entailment—strengthening the left argument and weakening the right preserves relatedness, provided both sides remain consistent.

Definition (`InfoSys.ApproximableMap.Le`). One approximable mapping lies below another when every related pair of the first is related by the second.

Theorem (`ApproximableMap.ext`). Two approximable mappings are equal precisely when they relate the same consistent sets.

Proof. The relation field determines the structure. ?

ApproximableMap.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.closure
public import DomainTheory.InformationSystems.InformationSystems.EntSet

@[expose] public section

universe u v

namespace DomainTheory.InformationSystems
namespace InfoSys

open DomainTheory.InformationSystems.Constructive

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

structure ApproximableMap (A : InfoSys α) (B : InfoSys β) where
  rel : Finset α → Finset β → Prop
  rel_dom : ∀ {u v}, rel u v → u ∈ A.Con
  rel_cod : ∀ {u v}, rel u v → v ∈ B.Con
  empty_rel : rel ∅ ∅
  union_right : ∀ {u v v'}, rel u v → rel u v' → rel u (v ∪' v')
  mono : ∀ {u u' v v'},
    rel u v → A.EntSet u' u → B.EntSet v v' → u' ∈ A.Con → v' ∈ B.Con → rel u' v'

namespace ApproximableMap

open Constructive

variable {A : InfoSys α} {B : InfoSys β}

def Le (f g : ApproximableMap A B) : Prop := ∀ {{u v}}, f.rel u v → g.rel u v

theorem ext {f g : ApproximableMap A B} (h : ∀ u v, f.rel u v ↔ g.rel u v) : f = g :=
    by
  obtain ⟨rf, _, _, _, _, _⟩ := f
  obtain ⟨rg, _, _, _, _, _⟩ := g
  have : rf = rg := by
    funext u v
    exact propext (h u v)
  subst this
  rfl

variable {? : Type*} [DecidableEq ?] {C : InfoSys ?}

theorem rel_bot_subset {D : InfoSys β} {s : Finset ?} {w : Finset β}
    (g : ApproximableMap C D) (hs : s ∈ C.Con)
    (hg : g.rel s (∅ : Finset β)) (hw : w ⊆ ({D.bot} : Finset β)) :
    g.rel s w := by
  have hwCon : w ∈ D.Con := D.con_subset (D.con_sing D.bot) hw
  have hEnt : D.EntSet (∅ : Finset β) w := by
    intro y hy
    have : y = D.bot := Finset.mem_singleton.mp (hw hy)
```

```
    subst this
    exact D.ent_bot D.con_empty
  exact g.mono hg (EntSet.refl C hs) hEnt hs hwCon

theorem rel_bot_subset_left {D : InfoSys α} {s : Finset ?} {v : Finset α}
    (f : ApproximableMap C D) (hs : s ∈ C.Con)
    (hf : f.rel s (∅ : Finset α)) (hv : v ⊆ ({D.bot} : Finset α)) :
    f.rel s v := by
  have hvCon : v ∈ D.Con := D.con_subset (D.con_sing D.bot) hv
  have hEnt : D.EntSet (∅ : Finset α) v := by
    intro x hx
    have : x = D.bot := Finset.mem_singleton.mp (hv hx)
    subst this
    exact D.ent_bot D.con_empty
  exact f.mono hf (EntSet.refl C hs) hEnt hs hvCon

end ApproximableMap

end InfoSys
end DomainTheory.InformationSystems
```

5.5.3 Element map

Theorem (`ApproximableMap.exists_rel_of_subset_image`). If every token of a finite set Y arises from some approximable input under an element x, then some single consistent input under x relates to the whole of Y.

Proof. Empty Y uses the empty relation after weakening; the inductive step joins the witnesses for the new token and the remainder via right-union and monotonicity. ?

Definition (`ApproximableMap.toElement`). The element map of an approximable mapping f sends an element x of the source to the element of the target whose tokens are those Y for which some finite input under x relates to $\{Y\}$.

Theorem (`ApproximableMap.mem_toElement`). Membership in the carrier of the element map is exactly the witnessing relation of the definition.

Proof. Unfolding. ?

Theorem (`ApproximableMap.toElement_mono`). The element map is monotone.

Proof. A witnessing input under a smaller element remains under a larger one. ?

Theorem (`ApproximableMap.rel_iff_closure_le`). Consistent sets u and v are related by an approximable mapping if and only if the closure of v approximates the image of the closure of u.

Proof. Relatedness yields singleton witnesses under the left closure; conversely a relational witness under the left closure is weakened back to u and v by monotonicity. ?

Theorem (`ApproximableMap.le_iff_toElement_le`). One approximable mapping lies below another if and only if its element map approximates the other's at every source element.

Proof. Pointwise comparison of images preserves relational witnesses; conversely the closure characterization transfers a related pair through the pointwise inequality. ?

Theorem (`ApproximableMap.ext_iff_toElement`). Two approximable mappings are equal if and only if they agree as element maps on every source element.

Proof. Equality of maps yields pointwise equality of images; conversely mutual pointwise order via the order characterization forces equal relations. ?

toElement.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.closure
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ApproximableMap

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

namespace ApproximableMap

variable {A : InfoSys α} {B : InfoSys β}

theorem exists_rel_of_subset_image (f : ApproximableMap A B) (x : A.Element)
    (Y : Finset β)
    (hY : ↑Y ⊆ ({Y : β | ∃ u : Finset α, ↑u ⊆ x.carrier ∧ f.rel u {Y}} : Set β)) :
    ∃ u : Finset α, ↑u ⊆ x.carrier ∧ f.rel u Y := by
  induction Y using Finset.induction_on with
  | empty =>
    refine ⟨∅, ?_, ?_⟩
    · intro a ha
      exact False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha))
    · exact f.mono f.empty_rel (EntSet.empty A ∅) (EntSet.empty B ∅) A.con_empty B.
    con_empty
  | insert a s _ha ih =>
    have ha' : ∃ u : Finset α, ↑u ⊆ x.carrier ∧ f.rel u {a} :=
      hY (Finset.mem_coe.2 (Finset.mem_insert_self a s))
    have hs' :
        ↑s ⊆ ({Y : β | ∃ u : Finset α, ↑u ⊆ x.carrier ∧ f.rel u {Y}} : Set β) := by
      intro b hb
      exact hY (Finset.mem_coe.2 (Finset.mem_insert_of_mem (Finset.mem_coe.1 hb)))
    obtain ⟨ua, hua, hrela⟩ := ha'
    obtain ⟨us, hus, hrels⟩ := ih hs'
    refine ⟨ua ∪' us, ?_, ?_⟩
    · intro z hz
      have hz' : z ∈ ua ∪' us := Finset.mem_coe.1 hz
      rcases funion.mem.mp hz' with h | h
      · exact hua (Finset.mem_coe.2 h)
      · exact hus (Finset.mem_coe.2 h)
    · have hUcon : ua ∪' us ∈ A.Con :=
        x.consistent (ua ∪' us) fun z hz => by
          rcases funion.mem.mp hz with h | h
          · exact hua (Finset.mem_coe.2 h)
          · exact hus (Finset.mem_coe.2 h)
```

```
      have hEnt_ua : A.EntSet (ua ∪' us) ua :=
        fun y hy => A.ent_refl hUcon (funion.subset_left ua us hy)
      have hEnt_us : A.EntSet (ua ∪' us) us :=
        fun y hy => A.ent_refl hUcon (funion.subset_right ua us hy)
      have h1 : f.rel (ua ∪' us) {a} :=
        f.mono hrela hEnt_ua
          (fun z hz => by
            rw [Finset.mem_singleton] at hz
            rw [hz]
            exact B.ent_refl (B.con_sing a) (Finset.mem_singleton_self a))
          hUcon (B.con_sing a)
      have h2 : f.rel (ua ∪' us) s :=
        f.mono hrels hEnt_us (EntSet.refl B (f.rel_cod hrels))
          hUcon (f.rel_cod hrels)
      have hU : f.rel (ua ∪' us) ({a} ∪' s) := f.union_right h1 h2
      simpa [funion.singleton_eq_insert] using hU

def toElement (f : ApproximableMap A B) (x : A.Element) : B.Element where
  carrier := {Y | ∃ u : Finset α, ↑u ⊆ x.carrier ∧ f.rel u {Y}}
  consistent := by
    intro Y hY
    obtain ⟨u, _, hrel⟩ := exists_rel_of_subset_image f x Y hY
    exact f.rel_cod hrel
  closed := by
    intro Y a hY hEnt
    obtain ⟨u, hu, hrel⟩ := exists_rel_of_subset_image f x Y hY
    have harel : f.rel u {a} :=
      f.mono hrel (EntSet.refl A (f.rel_dom hrel))
        (fun z hz => by
          rw [Finset.mem_singleton] at hz
          rw [hz]
          exact hEnt)
        (f.rel_dom hrel) (B.con_sing a)
    exact ⟨u, hu, harel⟩

@[simp] theorem mem_toElement (f : ApproximableMap A B) (x : A.Element) {Y : β} :
    Y ∈ (f.toElement x).carrier ↔ ∃ u : Finset α, ↑u ⊆ x.carrier ∧ f.rel u {Y} :=
  Iff.rfl

theorem toElement_mono (f : ApproximableMap A B) {x y : A.Element} (hxy : x ≤ y) :
    f.toElement x ≤ f.toElement y := by
  intro Y ⟨u, hu, hrel⟩
  exact ⟨u, fun z hz => hxy (hu hz), hrel⟩

theorem rel_iff_closure_le (f : ApproximableMap A B)
    {u : Finset α} {v : Finset β} (hu : u ∈ A.Con) (hv : v ∈ B.Con) :
    f.rel u v ↔ B.closure v hv ≤ f.toElement (A.closure u hu) := by
  constructor
  · intro hrel Y hY
    refine ⟨u, closure.subset A hu, ?_⟩
    exact f.mono hrel (EntSet.refl A hu)
      (fun Z hZ => by
        rw [Finset.mem_singleton] at hZ
        subst hZ
        exact hY)
      hu (B.con_sing Y)
```

```
  · intro hsub
    have hv_sub : ↑v ⊆ (f.toElement (A.closure u hu)).carrier :=
      fun y hy => hsub (closure.subset B hv hy)
    obtain ⟨u', hu', hrel'⟩ := exists_rel_of_subset_image f (A.closure u hu) v hv_sub
    have hEnt : A.EntSet u u' := fun y hy => hu' (Finset.mem_coe.2 hy)
    exact f.mono hrel' hEnt (EntSet.refl B hv) hu hv

theorem le_iff_toElement_le (f g : ApproximableMap A B) :
    Le f g ↔ ∀ x : A.Element, f.toElement x ≤ g.toElement x := by
  constructor
  · intro hfg x Y ⟨u, hu, hrel⟩
    exact ⟨u, hu, hfg hrel⟩
  · intro hpoint u v hrel
    have hu : u ∈ A.Con := f.rel_dom hrel
    have hv : v ∈ B.Con := f.rel_cod hrel
    have hbridge : B.closure v hv ≤ f.toElement (A.closure u hu) :=
      (f.rel_iff_closure_le hu hv).1 hrel
    have hsub : B.closure v hv ≤ g.toElement (A.closure u hu) :=
      le_trans hbridge (hpoint (A.closure u hu))
    exact (g.rel_iff_closure_le hu hv).2 hsub

theorem ext_iff_toElement (f g : ApproximableMap A B) :
    f = g ↔ ∀ x : A.Element, f.toElement x = g.toElement x := by
  constructor
  · intro h x
    rw [h]
  · intro hpoint
    refine ApproximableMap.ext fun u v => ?_
    constructor
    · intro hrel
      exact (le_iff_toElement_le f g).2 (fun x => (hpoint x) ▸ le_rfl) hrel
    · intro hrel
      exact (le_iff_toElement_le g f).2 (fun x => (hpoint x).symm ▸ le_rfl) hrel

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

5.5.4 Definition 5.2: image / function value

DEFINITION 5.2. If $f : \mathbf{A} \to \mathbf{B}$ is an approximable mapping between information systems, and if $x \in |\mathbf{A}|$ is an element of the first, then we define the *image* (or *function value*) of x under f by the formula:

$$f(x) = \{Y \in D_{\mathbf{B}} \mid u \; f \; \{Y\} \text{ for some } u \subseteq x\}.$$

Alternatively, we could use the equivalent formula:

$$f(x) = \bigcup\{v \in \mathrm{Con}_{\mathbf{B}} \mid u \; f \; v \text{ for some } u \subseteq x\}. \blacksquare$$

Note that, under Definition 5.2, it has to be *proved* that the image of an element in $|\mathbf{A}|$ lies in $|\mathbf{B}|$ in order to justify the use of the ordinary function-value notation. But both the consistency and the closure under entailment of $f(x)$ are direct consequences of the properties in the definition of approximable mapping.

Definition (`ApproximableMap.toElement`). The image of an element under an approximable mapping is the element map already recorded with Definition 5.1's companions: its carrier is exactly the first formula above, and constructing a target element settles the consistency and deductive-closure obligations Scott notes after the definition.

5.5.5 Proposition 5.3: element maps / pointwise comparison

PROPOSITION 5.3. *Let $f, g : \mathbf{A} \to \mathbf{B}$ be two approximable mappings between two information systems. Then*

(i) f always maps elements to elements under Definition 5.2;
(ii) $f = g$ iff $f(x) = g(x)$ for all $x \in |\mathbf{A}|$; and
(iii) $f \subseteq g$ iff $f(x) \subseteq g(x)$ for all $x \in |\mathbf{A}|$.
Moreover, the approximable mappings are monotone in the sense that

(iv) $x \subseteq y$ in $|\mathbf{A}|$ always implies $f(x) \subseteq f(y)$ in $|\mathbf{B}|$.

All these results follow from the observation that

(v) $u \; f \; v$ iff $\bar{v} \subseteq f(\bar{u})$,

for all $u \in \mathrm{Con}_{\mathbf{A}}$ and all $v \in \mathrm{Con}_{\mathbf{B}}$. ■

The proof is straightforward, and it completely justifies the use of the function-value notation. Consequently, the question arises as to whether the characterization of approximable mappings could not have been given in terms of elements in the first place. The answer is yes. Indeed, approximable mappings correspond exactly to functions on elements preserving unions of chains (this, in the case of countable information systems; directed unions must be used in general). This characterization has also been called "continuity," because it is equivalent to saying that the mappings between elements are continuous mappings in the topological sense, when $|\mathbf{A}|$ and $|\mathbf{B}|$ are regarded as topological spaces, as explained in the last section. The reason for the word "approximable" is explained in Section 7.

Having justified the idea of an approximable mapping as a transformation on elements, the next step is to combine mappings. It is hardly surprising to learn that compositions of approximable mappings are approximable, which in mathematical terms means that the domains together with the approximable mappings form a *category*. The interesting part starts when we want to combine *domains*. But, to fix ideas, it is useful to spell out how compositions take place on the level of the data objects and consistent sets. For instance, it was noted that in any information system $\vdash$ is transitive on Con, and in Definition 5.1 a transitivity condition comes in. There is a perfectly good reason for the parallelism, for, as we see next, the first idea (entailment) is a special case of the second (approximable mapping).

Theorem (`ApproximableMap.proposition_5_3`). Two approximable mappings are equal precisely when they agree as element maps; one lies below another precisely when its element map approximates the other's pointwise; and relatedness of consistent sets is equivalent to the closure of the right argument approximating the image of the closure of the left.

Proof. Assemble pointwise equality, pointwise order, and the relation-via-closures characterization. (Mapping elements to elements, and monotonicity of the element map, are already settled with the image.) ?

`proposition_5_3.lean`

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys.ApproximableMap

theorem proposition_5_3 :
    (∀ {α β : Type*} [DecidableEq α] [DecidableEq β] {A : InfoSys α} {B : InfoSys β}
      (f g : ApproximableMap A B),
      f = g ↔ ∀ x : A.Element, f.toElement x = g.toElement x) ∧
    (∀ {α β : Type*} [DecidableEq α] [DecidableEq β] {A : InfoSys α} {B : InfoSys β}
      (f g : ApproximableMap A B),
      Le f g ↔ ∀ x : A.Element, f.toElement x ≤ g.toElement x) ∧
    (∀ {α β : Type*} [DecidableEq α] [DecidableEq β] {A : InfoSys α} {B : InfoSys β}
      (f : ApproximableMap A B) {u : Finset α} {v : Finset β}
      (hu : u ∈ A.Con) (hv : v ∈ B.Con),
      f.rel u v ↔ B.closure v hv ≤ f.toElement (A.closure u hu)) :=
  ⟨@ext_iff_toElement, @le_iff_toElement_le, @rel_iff_closure_le⟩

end DomainTheory.InformationSystems.InfoSys.ApproximableMap
```

5.5.6 Identity map

Definition (`InfoSys.idMap`). The identity approximable mapping on an information system relates consistent sets u and v precisely when u entails v.

Theorem (`InfoSys.idMap.toElement`). The element map of the identity approximable mapping is the identity on elements.

Proof. Relatedness yields entailment of singletons, so tokens of the image lie in the source element by closure; conversely each token is recovered from its singleton via reflexivity of entailment. ?

idMap.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def idMap : ApproximableMap A A where
  rel u v := u ∈ A.Con ∧ v ∈ A.Con ∧ A.EntSet u v
  rel_dom h := h.1
  rel_cod h := h.2.1
```

```
  empty_rel := ⟨A.con_empty, A.con_empty, EntSet.empty A ∅⟩
  union_right := by
    rintro u v v' ⟨hu, _hv, huv⟩ ⟨_, _hv', huv'⟩
    have hEnt : A.EntSet u (v ∪' v') := EntSet.union (sys := A) huv huv'
    have hBig : u ∪' (v ∪' v') ∈ A.Con :=
      EntSet.union_mem (sys := A) (u := u) (v := v ∪' v') hu hEnt
    have hCon : v ∪' v' ∈ A.Con :=
      A.con_subset hBig (funion.subset_right u (v ∪' v'))
    exact ⟨hu, hCon, hEnt⟩
  mono := by
    rintro u u' v v' ⟨hu, hv, huv⟩ hEntu hEntv hu' hv'
    refine ⟨hu', hv', ?_⟩
    exact EntSet.trans A hu' hv
      (EntSet.trans A hu' hu hEntu huv) hEntv

theorem idMap.toElement (x : A.Element) : (idMap A).toElement x = x := by
  apply le_antisymm
  · intro Y ⟨u, hu, hrel⟩

    have hEnt : A.Ent u Y := hrel.2.2 Y (Finset.mem_singleton_self Y)
    exact x.closed u Y hu hEnt
  · intro Y hY
    refine ⟨{Y}, ?_, ?_⟩
    · intro z hz
      have hz' : z ∈ ({Y} : Finset α) := Finset.mem_coe.1 hz
      rw [Finset.mem_singleton] at hz'
      rw [hz']
      exact hY
    · refine ⟨A.con_sing Y, A.con_sing Y, ?_⟩
      exact EntSet.refl A (A.con_sing Y)

end InfoSys

end DomainTheory.InformationSystems
```

5.5.7 Proposition 5.4: identity mapping

PROPOSITION 5.4. *Let* $\mathbf{A}$ *be an information system. Then the following formula defines an approximable mapping* $I_{\mathbf{A}} : \mathbf{A} \to \mathbf{A}$*:*

(i) $u \; I_{\mathbf{A}} \; v$ iff $u \vdash_{\mathbf{A}} v$,

for all $u, v \in \mathrm{Con}_{\mathbf{A}}$*. And we have:*

(ii) $I_{\mathbf{A}}(x) = x$,

for all $x \in |\mathbf{A}|$. ■

In other words, the given entailment relation itself defines the identity function on the domain in question; and of course, the identity function is an approximable mapping.

Theorem (`InfoSys.proposition_5_4`). The identity approximable mapping acts as the identity on elements.

Proof. The identity map's element-map lemma. ?

Proposition54.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition54.idMap

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

theorem proposition_5_4 :
    ∀ {α : Type*} [DecidableEq α] (A : InfoSys α) (x : A.Element),
      (idMap A).toElement x = x :=
  @idMap.toElement

end DomainTheory.InformationSystems.InfoSys
```

5.5.8 Composition

Definition (`ApproximableMap.comp`). The composition of approximable mappings g after f relates u to w when some intermediate consistent set is related by f on the left and by g on the right.

Theorem (`ApproximableMap.comp.toElement`). The element map of a composition is the composition of the element maps.

Proof. A composite relational witness supplies an intermediate set whose tokens lie under the first image; conversely a token of the outer image under the inner image is pulled back to a single input under the source by the subset-image witness lemma. ?

comp.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
namespace ApproximableMap

variable {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable {A : InfoSys α} {B : InfoSys β} {C : InfoSys ?}

def comp (g : ApproximableMap B C) (f : ApproximableMap A B) : ApproximableMap A C
    where
  rel u w := ∃ v, f.rel u v ∧ g.rel v w
  rel_dom := fun ⟨_, hf, _⟩ => f.rel_dom hf
  rel_cod := fun ⟨_, _, hg⟩ => g.rel_cod hg
  empty_rel := ⟨∅, f.empty_rel, g.empty_rel⟩
  union_right := by
```

```
    rintro u w w' ⟨v, hf, hg⟩ ⟨v', hf', hg'⟩

    have hfU : f.rel u (v ∪' v') := f.union_right hf hf'
    have hvU : v ∪' v' ∈ B.Con := f.rel_cod hfU
    have hEntv : B.EntSet (v ∪' v') v :=
      fun y hy => B.ent_refl hvU (funion.subset_left v v' hy)
    have hEntv' : B.EntSet (v ∪' v') v' :=
      fun y hy => B.ent_refl hvU (funion.subset_right v v' hy)
    have hg1 : g.rel (v ∪' v') w :=
      g.mono hg hEntv (EntSet.refl C (g.rel_cod hg)) hvU (g.rel_cod hg)
    have hg2 : g.rel (v ∪' v') w' :=
      g.mono hg' hEntv' (EntSet.refl C (g.rel_cod hg')) hvU (g.rel_cod hg')
    exact ⟨v ∪' v', hfU, g.union_right hg1 hg2⟩
  mono := by
    rintro u u' w w' ⟨v, hf, hg⟩ hEntu hEntw hu' hw'
    refine ⟨v, ?_, ?_⟩
    · exact f.mono hf hEntu (EntSet.refl B (f.rel_cod hf)) hu' (f.rel_cod hf)
    · exact g.mono hg (EntSet.refl B (f.rel_cod hf)) hEntw (f.rel_cod hf) hw'

theorem comp.toElement (g : ApproximableMap B C) (f : ApproximableMap A B) (x : A.
    Element) :
    (comp g f).toElement x = g.toElement (f.toElement x) := by
  apply le_antisymm
  · intro Z ⟨u, hu, ⟨v, hf, hg⟩⟩

    refine ⟨v, ?_, hg⟩
    intro b hb

    have hb' : b ∈ v := Finset.mem_coe.1 hb
    have hsing : f.rel u {b} :=
      f.mono hf (EntSet.refl A (f.rel_dom hf))
        (fun z hz => by
          rw [Finset.mem_singleton] at hz; subst hz
          exact B.ent_refl (f.rel_cod hf) hb')
        (f.rel_dom hf) (B.con_sing b)
    exact ⟨u, hu, hsing⟩
  · intro Z ⟨v, hv, hg⟩

    obtain ⟨u, hu, hf⟩ := exists_rel_of_subset_image f x v hv
    exact ⟨u, hu, ⟨v, hf, hg⟩⟩

end ApproximableMap
end InfoSys

end DomainTheory.InformationSystems
```

5.5.9 Proposition 5.5: composition

PROPOSITION 5.5. *Let $f : \mathbf{A} \to \mathbf{B}$ and $g : \mathbf{B} \to \mathbf{C}$ be two approximable mappings. Then the following formula defines an approximable mapping $g \circ f : \mathbf{A} \to \mathbf{C}$:*

(i) $u\ (g \circ f)\ w$ iff $u\ f\ v$ and $v\ g\ w$ for some $v \in \mathrm{Con}_{\mathbf{B}}$,

for all $u \in \mathrm{Con}_{\mathbf{A}}$ and $w \in \mathrm{Con}_{\mathbf{C}}$. And we have:

(ii) $(g \circ f)(x) = g(f(x)),$

for all $x \in |\mathbf{A}|$. ■

In other words, composition of input/output relations is effected by putting the input into the first, getting some output, and then putting that in as input to the second. The correctness of this

recipe on elements (i.e. formula 5.5(ii)) follows from the basic formula 5.3(v). Because the class of all sets and arbitrary mappings forms a category—in the formal meaning of the word—the above three propositions show that the totality of information systems and approximable mappings also forms a category: a sub-category of the category of sets. We have given a special representation to the category by our use of data objects and consistent sets, and we have thereby put the elements into a secondary position—for a good reason. But the elements are there to use. The axioms for a category could also be verified directly using the basic definitions like 5.4(i) and 5.5(i); the details are bland. The more interesting topic is: what are the closure conditions on our domains; how do we construct new domains given old ones?

Before we go on to the first basic constructs of sums and products of domains, we remark on some other easy examples of approximable mappings: constant maps.

Theorem (`ApproximableMap.proposition_5_5`). The element map of a composition is the composition of the element maps.

Proof. The composition's element-map lemma. ?

`proposition_5_5.lean`

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.comp

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys.ApproximableMap

theorem proposition_5_5 :
    ∀ {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
      {A : InfoSys α} {B : InfoSys β} {C : InfoSys ?}
      (g : ApproximableMap B C) (f : ApproximableMap A B) (x : A.Element),
      (comp g f).toElement x = g.toElement (f.toElement x) :=
  @comp.toElement

end DomainTheory.InformationSystems.InfoSys.ApproximableMap
```

5.5.10 Constant map

Definition (`ApproximableMap.constMap`). The constant approximable mapping toward a fixed element b relates a consistent input to a consistent output precisely when the output is contained in b.

Theorem (`ApproximableMap.constMap.toElement`). The element map of a constant approximable mapping is constantly the chosen element.

Proof. Relatedness forces every output token into b; conversely the empty input witnesses every token of b. ?

Theorem (`ApproximableMap.constMap.comp`). Composing a constant map on the left with any map yields the same constant map from the new source.

Proof. Pointwise on elements both sides are constantly b. ?

Theorem (`ApproximableMap.constMap.unique`). An approximable mapping whose element map is constantly b equals the constant map toward b.

Proof. Pointwise equality of element maps. ?

Theorem (`ApproximableMap.constMap.comp_const`). Composing on the right with a constant map toward b yields the constant map toward the image of b.

Proof. Pointwise on elements both sides are constantly the image of b. ?

`constMap.lean`

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    proposition_5_3
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.comp

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
namespace ApproximableMap

variable {α β ? ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?] [
    DecidableEq ?]
variable {A : InfoSys α} {B : InfoSys β} {C : InfoSys ?} {D : InfoSys ?}

def constMap (b : B.Element) : ApproximableMap A B where
  rel u v := u ∈ A.Con ∧ (v : Set β) ⊆ b.carrier
  rel_dom h := h.1
  rel_cod h := b.consistent _ h.2
  empty_rel := ⟨A.con_empty, fun _ h => False.elim (Finset.notMem_empty _ (Finset.
    mem_coe.1 h))⟩
  union_right := by
    rintro u v v' ⟨hu, hv⟩ ⟨_, hv'⟩
    refine ⟨hu, ?_⟩
    intro y hy
    rcases funion.mem.mp (Finset.mem_coe.1 hy) with h | h
    · exact hv (Finset.mem_coe.2 h)
    · exact hv' (Finset.mem_coe.2 h)
  mono := by
    rintro u u' v v' ⟨_hu, hv⟩ _hEntu hEntv hu' _hv'
    refine ⟨hu', ?_⟩
    intro y hy
    have hEnt : B.Ent v y := hEntv y hy
    exact b.closed v y hv hEnt

namespace constMap

theorem toElement (b : B.Element) (x : A.Element) :
    (constMap (A := A) b).toElement x = b := by
  apply le_antisymm
```

```
  · intro Y ⟨_u, _hu, ⟨_, hsub⟩⟩
    exact hsub (Finset.mem_coe.2 (Finset.mem_singleton_self Y))
  · intro Y hY
    refine ⟨∅, ?_, ?_⟩
    · intro a ha
      exact False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha))
    · refine ⟨A.con_empty, ?_⟩
      intro Z hZ
      have hZ' : Z ∈ ({Y} : Finset β) := Finset.mem_coe.1 hZ
      rw [Finset.mem_singleton] at hZ'
      subst hZ'
      exact hY

theorem comp (b : B.Element) (g : ApproximableMap D A) :
    ApproximableMap.comp (constMap (A := A) b) g = constMap (A := D) b :=
  (ext_iff_toElement _ _).2 fun x => by
    rw [ApproximableMap.comp.toElement, toElement, toElement]

theorem unique (b : B.Element) (f : ApproximableMap A B)
    (h : ∀ x : A.Element, f.toElement x = b) :
    f = constMap (A := A) b :=
  (ext_iff_toElement f (constMap (A := A) b)).2 fun x => by
    rw [h x, toElement]

theorem comp_const (f : ApproximableMap B C) (b : B.Element) :
    ApproximableMap.comp f (constMap (A := A) b) = constMap (A := A) (f.toElement b)
    :=
  (ext_iff_toElement _ _).2 fun x => by
    rw [ApproximableMap.comp.toElement, toElement, toElement]

end constMap

end ApproximableMap
end InfoSys

end DomainTheory.InformationSystems
```

5.5.11 Proposition 5.6: constant maps

PROPOSITION 5.6. *Given information systems* $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, *and* $\mathbf{D}$, *and given a fixed element* $b \in |\mathbf{B}|$, *then there is a unique approximable mapping* $\mathrm{const}\ b : \mathbf{A} \to \mathbf{B}$ *such that:*

(i) $(\mathrm{const}\ b)(x) = b$,

for all $x \in |\mathbf{A}|$. *Moreover, we have:*

(ii) $f \circ (\mathrm{const}\ b) = \mathrm{const}\ f(b)$,

for all approximable mappings $f : \mathbf{B} \to \mathbf{C}$; *and*

(iii) $(\mathrm{const}\ b) \circ g = \mathrm{const}\ b$,

for all approximable mappings $g : \mathbf{D} \to \mathbf{A}$. ■

The proof is immediate, since we need only interpret the input/output relation $u\ (\mathrm{const}\ b)\ v$ as meaning $v \subseteq b$. We note that our notation is somewhat ambiguous since, for example, in 5.6(iii) we are using $\mathrm{const}\ b$ in two senses: once as a mapping from $\mathbf{A}$, and once as a mapping from $\mathbf{D}$. In fact, we should hang two subscripts on the thing to fix both the source and the target of the mapping. This is too painful, however, and we generally rely on context to make the meaning clear. (Formal rules of type checking can be given, so on a computer-based system all the necessary subscripts could be put back in. Thus, the kind of ambiguity we are dealing with here is not very serious.)

Theorem (`ApproximableMap.proposition_5_6`). Constant approximable mappings act constantly on elements; composing on the right with a constant yields the constant toward the image; composing on the left with a constant leaves that constant unchanged (from the new source).

Proof. Assemble the constant map's element-map, right-composition, and left-composition lemmas. ?

`Proposition56.lean`

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition56.constMap

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys.ApproximableMap

theorem proposition_5_6 :
    (∀ {α β : Type*} [DecidableEq α] [DecidableEq β] {A : InfoSys α} {B : InfoSys β}
      (b : B.Element) (x : A.Element),
      (constMap (A := A) b).toElement x = b) ∧
    (∀ {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
      {A : InfoSys α} {B : InfoSys β} {C : InfoSys ?}
      (f : ApproximableMap B C) (b : B.Element),
      comp f (constMap (A := A) b) = constMap (A := A) (f.toElement b)) ∧
    (∀ {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
      {A : InfoSys α} {B : InfoSys β} {D : InfoSys ?}
      (b : B.Element) (g : ApproximableMap D A),
      comp (constMap (A := A) b) g = constMap (A := D) b) :=
  ⟨@constMap.toElement, @constMap.comp_const, @constMap.comp⟩

end DomainTheory.InformationSystems.InfoSys.ApproximableMap
```

5.6 Products of domains

<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

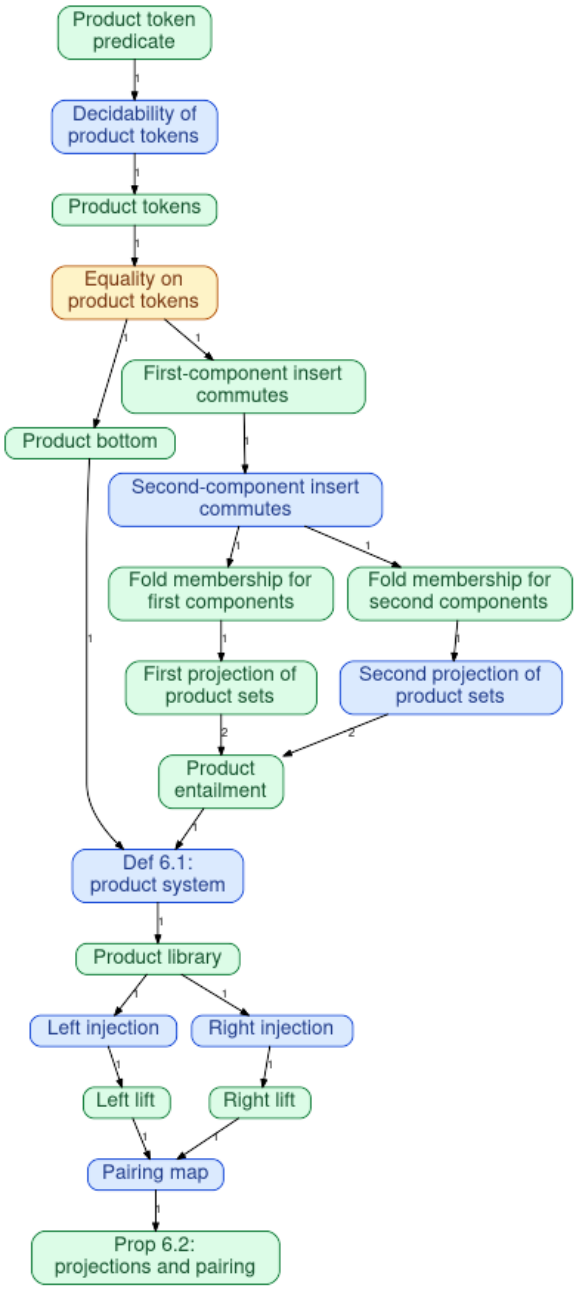


Figure 59: Diagram 24

5.6.1 Introduction

In the category of sets, the product (the cartesian product) of two sets is by definition just the set of ordered pairs of elements. We could use the same definition in the category of domains—provided we worked in terms of the elements of the domains. The disadvantage is that the determination of the product domain from data objects is lost, or at least pushed into the background. We shall therefore give a construction of products directly in terms of the given data objects, and then show how to prove that the product is just the

one expected when we look at the elements.

The idea for the definition is a simple one. Think of two domains $\mathbf{A}$ and $\mathbf{B}$. A data object $X \in D_{\mathbf{A}}$ can be giving information about a possible element $x \in |\mathbf{A}|$, and a data object $Y \in D_{\mathbf{B}}$ can be giving information about a possible element $y \in |\mathbf{B}|$. How do we wish to give information about the pair (x, y)? The first piece of advice is not to say all that is known all at once. For example, if pressed, we can say we know X about the first coordinate of the pair; only later, if really pressed, we can say we also know Y about the second coordinate. The point is that data objects provide only partial information, and it does not matter if it takes several of them to give fairly complete information about the whole element. This attitude motivates, then, the particular form of our official definition.

5.6.2 Product token predicate

Definition (`IsProdToken`). A pair is a product token when at least one component is the corresponding system's bottom.

`IsProdToken.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

class IsProdToken (A : InfoSys α) (B : InfoSys β) (p : α × β) : Prop where
  out : p.1 = A.bot ∨ p.2 = B.bot

end InfoSys

end DomainTheory.InformationSystems
```

5.6.3 Decidability of product tokens

Definition (`InfoSys.instDecidableIsProdToken`). Membership in the product-token predicate is decidable by inspecting which component is bottom.

`BasicInst5.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    IsProdToken
```

```
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

instance (A : InfoSys α) (B : InfoSys β) (p : α × β) : Decidable (IsProdToken A B p)
    :=
  if h1 : p.1 = A.bot then isTrue ⟨Or.inl h1⟩
  else if h2 : p.2 = B.bot then isTrue ⟨Or.inr h2⟩
  else isFalse fun h => h.out.elim h1 h2

end InfoSys

end DomainTheory.InformationSystems
```

5.6.4 Product tokens

Definition (`ProdToken`). The type of product tokens of $\mathbf{A}$ and $\mathbf{B}$ is the subtype of pairs satisfying the product-token predicate.

ProdToken.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    IsProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

def ProdToken (A : InfoSys α) (B : InfoSys β) : Type _ :=
  {p : α × β // IsProdToken A B p}

end InfoSys

end DomainTheory.InformationSystems
```

5.6.5 Equality on product tokens

Definition (`instDecidableEqProdToken`). Product tokens have decidable equality.

BasicInst7.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

instance instDecidableEqProdToken (A : InfoSys α) (B : InfoSys β) :
    DecidableEq (ProdToken A B) :=
  Subtype.instDecidableEq

end InfoSys

end DomainTheory.InformationSystems
```

5.6.6 Product bottom

Definition (`prodBot`). The least product token is $(\Delta_{\mathbf{A}}, \Delta_{\mathbf{B}})$.

prodBot.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def prodBot : ProdToken A B :=
  ⟨(A.bot, B.bot), ⟨Or.inl rfl⟩⟩

end InfoSys

end DomainTheory.InformationSystems
```

5.6.7 First-component insert commutes

Definition (`instLeftCommutativeFstInsert`). Inserting into the first-component accumulator commutes on the left.

`instLeftCommutativeFstInsert.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

instance instLeftCommutativeFstInsert :
    LeftCommutative fun p : ProdToken A B => (insert p.val.1 : Finset α → Finset α) :=
  ⟨fun p q s => insert_comm' p.val.1 q.val.1 s⟩

end InfoSys

end DomainTheory.InformationSystems
```

5.6.8 Second-component insert commutes

Definition (`instLeftCommutativeSndInsert`). Inserting into the second-component accumulator commutes on the left.

instLeftCommutativeSndInsert.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

instance instLeftCommutativeSndInsert :
    LeftCommutative fun p : ProdToken A B => (insert p.val.2 : Finset β → Finset β) :=
  ⟨fun p q s => insert_comm' p.val.2 q.val.2 s⟩

end InfoSys

end DomainTheory.InformationSystems
```

5.6.9 Fold membership for first components

Theorem (`mem_foldr_fst`). Membership after folding first-component insertion over a multiset of product tokens characterises the accumulated first components.

Proof. Direct from the definitions and the ambient information-system axioms. ?

mem_foldr_fst.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

theorem mem_foldr_fst (s : Multiset (ProdToken A B)) (x : α) :
    x ∈ Multiset.foldr (fun p : ProdToken A B => insert p.val.1) (∅ : Finset α) s ↔
      ∃ p ∈ s, p.val.1 = x := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro hx
      exact False.elim (Finset.notMem_empty x hx)
    · rintro ⟨p, hp, _⟩
      exact False.elim (by cases hp)
  · intro p t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    · rintro (hx | ⟨q, hq, hq1⟩)
      · exact ⟨p, Or.inl rfl, hx.symm⟩
      · exact ⟨q, Or.inr hq, hq1⟩
    · rintro ⟨q, hq, hq1⟩
      rcases hq with rfl | hq
      · exact Or.inl hq1.symm
      · exact Or.inr ⟨q, hq, hq1⟩

end InfoSys

end DomainTheory.InformationSystems
```

5.6.10 Fold membership for second components

Theorem (`mem_foldr_snd`). Membership after folding second-component insertion over a multiset of product tokens characterises the accumulated second components.

Proof. Direct from the definitions and the ambient information-system axioms. ?

`mem_foldr_snd.lean`

```
module
```

```
public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

theorem mem_foldr_snd (s : Multiset (ProdToken A B)) (y : β) :
    y ∈ Multiset.foldr (fun p : ProdToken A B => insert p.val.2) (∅ : Finset β) s ↔
      ∃ p ∈ s, p.val.2 = y := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro hy
      exact False.elim (Finset.notMem_empty y hy)
    · rintro ⟨p, hp, _⟩
      exact False.elim (by cases hp)
  · intro p t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    · rintro (hy | ⟨q, hq, hq2⟩)
      · exact ⟨p, Or.inl rfl, hy.symm⟩
      · exact ⟨q, Or.inr hq, hq2⟩
    · rintro ⟨q, hq, hq2⟩
      rcases hq with rfl | hq
      · exact Or.inl hq2.symm
      · exact Or.inr ⟨q, hq, hq2⟩

end InfoSys

end DomainTheory.InformationSystems
```

5.6.11 First projection of product sets

Definition (`fstFinset`). The first projection of a finite set of product tokens collects those left components whose right component is bottom.

Theorem (`mem_fstFinset`). Membership in the first projection matches the existence of a witnessing product token.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`fstFinset.empty`). The first projection of the empty set is empty.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`fstFinset.funion`). First projection preserves finite unions.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`fstFinset.insert_left`). Inserting a left-sided token adjoins its first component.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`fstFinset.insert_not_left`). Inserting a non-left-sided token leaves the first projection unchanged.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`fstFinset.mono`). First projection is monotone.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`fstFinset.singleton_left`). The first projection of a singleton left-sided token is that first component.

Proof. Direct from the definitions and the ambient information-system axioms. ?

fstFinset.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    mem_foldr_fst

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]
variable (A : InfoSys α) (B : InfoSys β)

def fstFinset (u : Finset (ProdToken A B)) : Finset α :=
  Multiset.foldr (fun p : ProdToken A B => insert p.val.1) (∅ : Finset α)
    (u.filter (fun p => p.val.2 = B.bot)).1

theorem mem_fstFinset {u : Finset (ProdToken A B)} {x : α} :
```

```
    x ∈ fstFinset A B u ↔ ∃ p ∈ u, p.val.2 = B.bot ∧ p.val.1 = x := by
  unfold fstFinset
  rw [mem_foldr_fst]
  constructor
  · rintro ⟨p, hp, hx⟩
    have hp' := Finset.mem_filter.mp hp
    exact ⟨p, hp'.1, hp'.2, hx⟩
  · rintro ⟨p, hp, hbot, hx⟩
    exact ⟨p, Finset.mem_filter.mpr ⟨hp, hbot⟩, hx⟩

theorem fstFinset.empty : fstFinset A B (∅ : Finset (ProdToken A B)) = ∅ := by
  ext x
  constructor
  · intro hx
    rcases (mem_fstFinset A B).1 hx with ⟨_, hp, _, _⟩
    exact False.elim (Finset.notMem_empty _ hp)
  · intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem fstFinset.funion (u v : Finset (ProdToken A B)) :
    fstFinset A B (u ∪' v) = fstFinset A B u ∪' fstFinset A B v := by
  ext x
  constructor
  · intro hx
    rcases (mem_fstFinset A B).1 hx with ⟨p, hp, hbot, hx'⟩
    rcases funion.mem.mp hp with hp | hp
    · exact funion.mem.mpr (Or.inl ((mem_fstFinset A B).2 ⟨p, hp, hbot, hx'⟩))
    · exact funion.mem.mpr (Or.inr ((mem_fstFinset A B).2 ⟨p, hp, hbot, hx'⟩))
  · intro hx
    rcases funion.mem.mp hx with hx | hx
    · rcases (mem_fstFinset A B).1 hx with ⟨p, hp, hbot, hx'⟩
      exact (mem_fstFinset A B).2 ⟨p, funion.mem.mpr (Or.inl hp), hbot, hx'⟩
    · rcases (mem_fstFinset A B).1 hx with ⟨p, hp, hbot, hx'⟩
      exact (mem_fstFinset A B).2 ⟨p, funion.mem.mpr (Or.inr hp), hbot, hx'⟩

theorem fstFinset.insert_left (u : Finset (ProdToken A B)) (p : ProdToken A B)
    (hp : p.val.2 = B.bot) :
    fstFinset A B (insert p u) = insert p.val.1 (fstFinset A B u) := by
  ext x
  constructor
  · intro hx
    rcases (mem_fstFinset A B).1 hx with ⟨q, hq, hbot, hx'⟩
    rcases Finset.mem_insert.mp hq with rfl | hq
    · exact Finset.mem_insert.mpr (Or.inl hx'.symm)
    · exact Finset.mem_insert.mpr (Or.inr ((mem_fstFinset A B).2 ⟨q, hq, hbot, hx'⟩))
  · intro hx
    rcases Finset.mem_insert.mp hx with hx' | hx'
    · exact (mem_fstFinset A B).2 ⟨p, Finset.mem_insert_self p u, hp, hx'.symm⟩
    · rcases (mem_fstFinset A B).1 hx' with ⟨q, hq, hbot, hxq⟩
      exact (mem_fstFinset A B).2 ⟨q, Finset.mem_insert_of_mem hq, hbot, hxq⟩

theorem fstFinset.insert_not_left (u : Finset (ProdToken A B)) (p : ProdToken A B)
    (hp : p.val.2 = B.bot) :
    fstFinset A B (insert p u) = fstFinset A B u := by
  ext x
  constructor
```

```
  · intro hx
    rcases (mem_fstFinset A B).1 hx with ⟨q, hq, hbot, hx'⟩
    rcases Finset.mem_insert.mp hq with rfl | hq
    · exact False.elim (hp hbot)
    · exact (mem_fstFinset A B).2 ⟨q, hq, hbot, hx'⟩
  · intro hx
    rcases (mem_fstFinset A B).1 hx with ⟨q, hq, hbot, hx'⟩
    exact (mem_fstFinset A B).2 ⟨q, Finset.mem_insert_of_mem hq, hbot, hx'⟩

theorem fstFinset.mono {u v : Finset (ProdToken A B)} (h : v ⊆ u) :
    fstFinset A B v ⊆ fstFinset A B u := by
  intro x hx
  rcases (mem_fstFinset A B).1 hx with ⟨p, hp, hbot, hx'⟩
  exact (mem_fstFinset A B).2 ⟨p, h hp, hbot, hx'⟩

theorem fstFinset.singleton_left (p : ProdToken A B) (hp : p.val.2 = B.bot) :
    fstFinset A B {p} = {p.val.1} := by
  rw [show ({p} : Finset _) = insert p ∅ from rfl, fstFinset.insert_left A B ∅ p hp,
    fstFinset.empty]
  rfl

end InfoSys

end DomainTheory.InformationSystems
```

5.6.12 Second projection of product sets

Definition (`sndFinset`). The second projection of a finite set of product tokens collects those right components whose left component is bottom.

Theorem (`mem_sndFinset`). Membership in the second projection matches the existence of a witnessing product token.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`sndFinset.empty`). The second projection of the empty set is empty.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`sndFinset.funion`). Second projection preserves finite unions.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`sndFinset.insert_not_right`). Inserting a non-right-sided token leaves the second projection unchanged.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`sndFinset.insert_right`). Inserting a right-sided token adjoins its second component.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`sndFinset.mono`). Second projection is monotone.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`sndFinset.singleton_right`). The second projection of a singleton right-sided token is that second component.

Proof. Direct from the definitions and the ambient information-system axioms. ?

`sndFinset.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    mem_foldr_snd

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]
variable (A : InfoSys α) (B : InfoSys β)

def sndFinset (u : Finset (ProdToken A B)) : Finset β :=
  Multiset.foldr (fun p : ProdToken A B => insert p.val.2) (∅ : Finset β)
    (u.filter (fun p => p.val.1 = A.bot)).1

theorem mem_sndFinset {u : Finset (ProdToken A B)} {y : β} :
    y ∈ sndFinset A B u ↔ ∃ p ∈ u, p.val.1 = A.bot ∧ p.val.2 = y := by
  unfold sndFinset
  rw [mem_foldr_snd]
  constructor
  · rintro ⟨p, hp, hy⟩
    have hp' := Finset.mem_filter.mp hp
    exact ⟨p, hp'.1, hp'.2, hy⟩
  · rintro ⟨p, hp, hbot, hy⟩
    exact ⟨p, Finset.mem_filter.mpr ⟨hp, hbot⟩, hy⟩

theorem sndFinset.empty : sndFinset A B (∅ : Finset (ProdToken A B)) = ∅ := by
  ext y
  constructor
  · intro hy
    rcases (mem_sndFinset A B).1 hy with ⟨_, hp, _, _⟩
    exact False.elim (Finset.notMem_empty _ hp)
  · intro hy
    exact False.elim (Finset.notMem_empty y hy)

theorem sndFinset.funion (u v : Finset (ProdToken A B)) :
    sndFinset A B (u ∪' v) = sndFinset A B u ∪' sndFinset A B v := by
  ext y
  constructor
```

```
  · intro hy
    rcases (mem_sndFinset A B).1 hy with ⟨p, hp, hbot, hy'⟩
    rcases funion.mem.mp hp with hp | hp
    · exact funion.mem.mpr (Or.inl ((mem_sndFinset A B).2 ⟨p, hp, hbot, hy'⟩))
    · exact funion.mem.mpr (Or.inr ((mem_sndFinset A B).2 ⟨p, hp, hbot, hy'⟩))
  · intro hy
    rcases funion.mem.mp hy with hy | hy
    · rcases (mem_sndFinset A B).1 hy with ⟨p, hp, hbot, hy'⟩
      exact (mem_sndFinset A B).2 ⟨p, funion.mem.mpr (Or.inl hp), hbot, hy'⟩
    · rcases (mem_sndFinset A B).1 hy with ⟨p, hp, hbot, hy'⟩
      exact (mem_sndFinset A B).2 ⟨p, funion.mem.mpr (Or.inr hp), hbot, hy'⟩

theorem sndFinset.insert_not_right (u : Finset (ProdToken A B)) (p : ProdToken A B)
    (hp : p.val.1 = A.bot) :
    sndFinset A B (insert p u) = sndFinset A B u := by
  ext y
  constructor
  · intro hy
    rcases (mem_sndFinset A B).1 hy with ⟨q, hq, hbot, hy'⟩
    rcases Finset.mem_insert.mp hq with rfl | hq
    · exact False.elim (hp hbot)
    · exact (mem_sndFinset A B).2 ⟨q, hq, hbot, hy'⟩
  · intro hy
    rcases (mem_sndFinset A B).1 hy with ⟨q, hq, hbot, hy'⟩
    exact (mem_sndFinset A B).2 ⟨q, Finset.mem_insert_of_mem hq, hbot, hy'⟩

theorem sndFinset.insert_right (u : Finset (ProdToken A B)) (p : ProdToken A B)
    (hp : p.val.1 = A.bot) :
    sndFinset A B (insert p u) = insert p.val.2 (sndFinset A B u) := by
  ext y
  constructor
  · intro hy
    rcases (mem_sndFinset A B).1 hy with ⟨q, hq, hbot, hy'⟩
    rcases Finset.mem_insert.mp hq with rfl | hq
    · exact Finset.mem_insert.mpr (Or.inl hy'.symm)
    · exact Finset.mem_insert.mpr (Or.inr ((mem_sndFinset A B).2 ⟨q, hq, hbot, hy'⟩))
  · intro hy
    rcases Finset.mem_insert.mp hy with hy' | hy'
    · exact (mem_sndFinset A B).2 ⟨p, Finset.mem_insert_self p u, hp, hy'.symm⟩
    · rcases (mem_sndFinset A B).1 hy' with ⟨q, hq, hbot, hyq⟩
      exact (mem_sndFinset A B).2 ⟨q, Finset.mem_insert_of_mem hq, hbot, hyq⟩

theorem sndFinset.mono {u v : Finset (ProdToken A B)} (h : v ⊆ u) :
    sndFinset A B v ⊆ sndFinset A B u := by
  intro y hy
  rcases (mem_sndFinset A B).1 hy with ⟨p, hp, hbot, hy'⟩
  exact (mem_sndFinset A B).2 ⟨p, h hp, hbot, hy'⟩

theorem sndFinset.singleton_right (p : ProdToken A B) (hp : p.val.1 = A.bot) :
    sndFinset A B {p} = {p.val.2} := by
  rw [show ({p} : Finset _) = insert p ∅ from rfl, sndFinset.insert_right A B ∅ p hp,
    sndFinset.empty]
  rfl

end InfoSys
```

```
end DomainTheory.InformationSystems
```

5.6.13 Product entailment

Definition (`ProdEnt`). Product entailment of a token requires consistency of the set together with the corresponding factor entailment on the non-bottom side.

ProdEnt.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    fstFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    sndFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.ProdCon

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def ProdEnt (u : Finset (ProdToken A B)) (p : ProdToken A B) : Prop :=
  ProdCon A B u ∧
    (p.val.2 = B.bot → A.Ent (fstFinset A B u) p.val.1) ∧
      (p.val.1 = A.bot → B.Ent (sndFinset A B u) p.val.2)

end InfoSys

end DomainTheory.InformationSystems
```

5.6.14 Definition 6.1: product system

DEFINITION 6.1. Let $\mathbf{A}$ and $\mathbf{B}$ be two information systems. By $\mathbf{A} \times \mathbf{B}$, the *product system*, we understand the system where:

(i) $D_{\mathbf{A}\times\mathbf{B}} = \{(X, \Delta_{\mathbf{B}}) \mid X \in D_{\mathbf{A}}\} \cup \{(\Delta_{\mathbf{A}}, Y) \mid Y \in D_{\mathbf{B}}\}$;

(ii) $\Delta_{\mathbf{A}\times\mathbf{B}} = (\Delta_{\mathbf{A}}, \Delta_{\mathbf{B}})$;

(iii) $u \in \mathrm{Con}_{\mathbf{A}\times\mathbf{B}}$ iff $\mathrm{fst}\, u \in \mathrm{Con}_{\mathbf{A}}$ and $\mathrm{snd}\, u \in \mathrm{Con}_{\mathbf{B}}$;

(iv') $u \vdash_{\mathbf{A}\times\mathbf{B}} (X', \Delta_{\mathbf{B}})$ iff $\mathrm{fst}\, u \vdash_{\mathbf{A}} X'$; and
(iv' ') $u \vdash_{\mathbf{A}\times\mathbf{B}} (\Delta_{\mathbf{A}}, Y')$ iff $\mathrm{snd}\, u \vdash_{\mathbf{B}} Y'$;
where, in (iii), u is any finite subset of $D_{\mathbf{A}\times\mathbf{B}}$, in (iv') and (iv' '), $u \in \mathrm{Con}_{\mathbf{A}\times\mathbf{B}}$, and we let:

$$\mathrm{fst}\, u = \{X \in D_{\mathbf{A}} \mid (X, \Delta_{\mathbf{B}}) \in u\}, \text{ and}$$

$$\mathrm{snd}\, u = \{Y \in D_{\mathbf{B}} \mid (\Delta_{\mathbf{A}}, Y) \in u\}. \quad \blacksquare$$

Note that in 6.1(i) the two sets in the union are just two copies of $D_{\mathbf{A}}$ and $D_{\mathbf{B}}$, respectively. There is a shared member, however, the object $(\Delta_{\mathbf{A}}, \Delta_{\mathbf{B}})$, which is indeed the least informative data object—whether it is looked at from the point of view of $\mathbf{A}$ or of $\mathbf{B}$. We could have used all the pairs (X, Y), but if we did, the consistent set $\{(X, Y)\}$ would be deductively equivalent to the set $\{(X, \Delta_{\mathbf{B}}), (\Delta_{\mathbf{A}}, Y)\}$ according to the definition of $\vdash_{\mathbf{A}\times\mathbf{B}}$, and so there is not much point in having this redundancy. Strictly speaking, we should not make these remarks until we have actually verified that 6.1 is indeed a proper definition.

Definition (`InfoSys.productSystem`). The product of information systems $\mathbf{A}$ and $\mathbf{B}$ is the information system whose data are the product tokens of Definition 6.1(i), whose least informative object is the shared bottom $(\Delta_{\mathbf{A}}, \Delta_{\mathbf{B}})$, whose consistent sets are those whose first and second projections are consistent in $\mathbf{A}$ and $\mathbf{B}$ respectively, and whose entailment of a token is the corresponding entailment of its first or second component as in 6.1(iv')–(iv' '). The information-system axioms hold by projecting to the two factors and applying their axioms.

Theorem (`ProdEnt.entSet_fst`). Entailment of finite sets in the product projects to entailment of first components in $\mathbf{A}$.

Proof. Each first-component token comes from a product token whose right component is bottom, and the product entailment supplies the corresponding factor entailment. ?

Theorem (`ProdEnt.entSet_snd`). Entailment of finite sets in the product projects to entailment of second components in $\mathbf{B}$.

Proof. Dual to the first-component case. ?

Definition (`productSystem.fstMap`). The first projection approximable mapping from $\mathbf{A} \times \mathbf{B}$ to $\mathbf{A}$ relates a consistent product set u to a consistent set v in $\mathbf{A}$ precisely when u is consistent in the product and the first components of u entail v in $\mathbf{A}$.

Definition (`productSystem.sndMap`). The second projection approximable mapping from $\mathbf{A} \times \mathbf{B}$ to $\mathbf{B}$ relates a consistent product set u to a consistent set w in $\mathbf{B}$ precisely when u is consistent in the product and the second components of u entail w in $\mathbf{B}$.

Theorem (`ApproximableMap.productSystem.element_eq_of_fst_snd`). Two elements of $\mathbf{A} \times \mathbf{B}$ with equal first and second components coincide.

Proof. Each token of one element is recovered in the other using projection and closure, according to whether its right or left component is bottom. ?

productSystem.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.prodBot
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.ProdEnt
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ApproximableMap
public import DomainTheory.InformationSystems.InformationSystems.EntSet
import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def productSystem : InfoSys (ProdToken A B) where
  bot := prodBot A B
  Con := {u | ProdCon A B u}
  Ent := ProdEnt A B
  con_subset := by
    intro u v hu hv
    exact ⟨A.con_subset hu.1 (fstFinset.mono A B hv),
      B.con_subset hu.2 (sndFinset.mono A B hv)⟩
  con_sing := by
    intro p
    refine ⟨?_, ?_⟩
    · rcases p.property.out with h1 | h2
      · by_cases h2 : p.val.2 = B.bot
        · have hpeq : p = prodBot A B := Subtype.ext (Prod.ext h1 h2)
          subst hpeq
          rw [fstFinset.singleton_left A B (prodBot A B) rfl]
          exact A.con_sing A.bot
        · have hfst : fstFinset A B {p} = ∅ := by
            ext x
            constructor
            · intro hx
              rcases (mem_fstFinset A B).1 hx with ⟨q, hq, hbot', _⟩
              have hq' : q = p := Finset.mem_singleton.mp hq
              subst hq'
              exact False.elim (h2 hbot')
            · intro hx
              exact False.elim (Finset.notMem_empty x hx)
          rw [hfst]
          exact A.con_empty
      · rw [fstFinset.singleton_left A B p h2]
```

```
      exact A.con_sing p.val.1
  · rcases p.property.out with h1 | h2
    · rw [sndFinset.singleton_right A B p h1]
      exact B.con_sing p.val.2
    · by_cases h1 : p.val.1 = A.bot
      · have hpeq : p = prodBot A B := Subtype.ext (Prod.ext h1 h2)
        subst hpeq
        rw [sndFinset.singleton_right A B (prodBot A B) rfl]
        exact B.con_sing B.bot
      · have hsnd : sndFinset A B {p} = ∅ := by
          ext y
          constructor
          · intro hy
            rcases (mem_sndFinset A B).1 hy with ⟨q, hq, hbot', _⟩
            have hq' : q = p := Finset.mem_singleton.mp hq
            subst hq'
            exact False.elim (h1 hbot')
          · intro hy
            exact False.elim (Finset.notMem_empty y hy)
        rw [hsnd]
        exact B.con_empty
ent_con := by
  intro u p ⟨⟨hfst, hsnd⟩, hL, hR⟩
  refine ⟨?_, ?_⟩
  · by_cases hp2 : p.val.2 = B.bot
    · rw [fstFinset.insert_left A B u p hp2]
      exact A.ent_con (hL hp2)
    · rw [fstFinset.insert_not_left A B u p hp2]
      exact hfst
  · by_cases hp1 : p.val.1 = A.bot
    · rw [sndFinset.insert_right A B u p hp1]
      exact B.ent_con (hR hp1)
    · rw [sndFinset.insert_not_right A B u p hp1]
      exact hsnd
ent_bot := by
  intro u hu
  exact ⟨hu, fun _ => A.ent_bot hu.1, fun _ => B.ent_bot hu.2⟩
ent_refl := by
  intro u p hu hp
  refine ⟨hu, ?_, ?_⟩
  · intro hbot
    exact A.ent_refl hu.1 ((mem_fstFinset A B).2 ⟨p, hp, hbot, rfl⟩)
  · intro hbot
    exact B.ent_refl hu.2 ((mem_sndFinset A B).2 ⟨p, hp, hbot, rfl⟩)
ent_trans := by
  intro u v c hv hu hEnts hEntc
  refine ⟨hv, ?_, ?_⟩
  · intro hbot
    have hfst : ∀ y ∈ fstFinset A B u, A.Ent (fstFinset A B v) y := by
      intro y hy
      rcases (mem_fstFinset A B).1 hy with ⟨q, hq, hq2, rfl⟩
      exact (hEnts q hq).2.1 hq2
    exact A.ent_trans hv.1 hu.1 hfst (hEntc.2.1 hbot)
  · intro hbot
    have hsnd : ∀ y ∈ sndFinset A B u, B.Ent (sndFinset A B v) y := by
      intro y hy
```

```
        rcases (mem_sndFinset A B).1 hy with ⟨q, hq, hq1, rfl⟩
        exact (hEnts q hq).2.2 hq1
      exact B.ent_trans hv.2 hu.2 hsnd (hEntc.2.2 hbot)

namespace ProdEnt

theorem entSet_fst {u v : Finset (ProdToken A B)}
    (h : (productSystem A B).EntSet u v) :
    A.EntSet (fstFinset A B u) (fstFinset A B v) := by
  intro x hx
  rcases (mem_fstFinset A B).1 hx with ⟨p, hp, hbot, rfl⟩
  exact (h p hp).2.1 hbot

theorem entSet_snd {u v : Finset (ProdToken A B)}
    (h : (productSystem A B).EntSet u v) :
    B.EntSet (sndFinset A B u) (sndFinset A B v) := by
  intro y hy
  rcases (mem_sndFinset A B).1 hy with ⟨p, hp, hbot, rfl⟩
  exact (h p hp).2.2 hbot

end ProdEnt

namespace productSystem

def fstMap : ApproximableMap (productSystem A B) A where
  rel u v := u ∈ (productSystem A B).Con ∧ v ∈ A.Con ∧ A.EntSet (fstFinset A B u) v
  rel_dom h := h.1
  rel_cod h := h.2.1
  empty_rel := ⟨ProdCon.empty A B, A.con_empty, by
    rw [fstFinset.empty]; exact EntSet.empty A ∅⟩
  union_right := by
    rintro u v v' ⟨hu, hv, huv⟩ ⟨_, hv', huv'⟩
    have hEnt : A.EntSet (fstFinset A B u) (v ∪' v') :=
      EntSet.union (sys := A) huv huv'
    have hBig : fstFinset A B u ∪' (v ∪' v') ∈ A.Con :=
      EntSet.union_mem (sys := A) hu.1 hEnt
    have hCon : v ∪' v' ∈ A.Con :=
      A.con_subset hBig (funion.subset_right _ _)
    exact ⟨hu, hCon, hEnt⟩
  mono := by
    rintro u u' v v' ⟨hu, hv, huv⟩ hEntu hEntv hu' hv'
    refine ⟨hu', hv', ?_⟩
    exact EntSet.trans A hu'.1 hv
      (EntSet.trans A hu'.1 hu.1 (ProdEnt.entSet_fst A B hEntu) huv) hEntv

def sndMap : ApproximableMap (productSystem A B) B where
  rel u w := u ∈ (productSystem A B).Con ∧ w ∈ B.Con ∧ B.EntSet (sndFinset A B u) w
  rel_dom h := h.1
  rel_cod h := h.2.1
  empty_rel := ⟨ProdCon.empty A B, B.con_empty, by
    rw [sndFinset.empty]; exact EntSet.empty B ∅⟩
  union_right := by
    rintro u w w' ⟨hu, hw, huw⟩ ⟨_, hw', huw'⟩
    have hEnt : B.EntSet (sndFinset A B u) (w ∪' w') :=
      EntSet.union (sys := B) huw huw'
    have hBig : sndFinset A B u ∪' (w ∪' w') ∈ B.Con :=
```

```
      EntSet.union_mem (sys := B) hu.2 hEnt
    have hCon : w ∪' w' ∈ B.Con :=
      B.con_subset hBig (funion.subset_right _ _)
    exact ⟨hu, hCon, hEnt⟩
  mono := by
    rintro u u' w w' ⟨hu, hw, huw⟩ hEntu hEntw hu' hw'
    refine ⟨hu', hw', ?_⟩
    exact EntSet.trans B hu'.2 hw
      (EntSet.trans B hu'.2 hu.2 (ProdEnt.entSet_snd A B hEntu) huw) hEntw

end productSystem

end InfoSys

end DomainTheory.InformationSystems
```

productSystem_more.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    productSystem
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β)

namespace ApproximableMap

variable {C : InfoSys ?}

namespace productSystem

theorem element_eq_of_fst_snd (z z' : (productSystem A B).Element)
    (hfst : (productSystem.fstMap A B).toElement z = (productSystem.fstMap A B).
    toElement z')
    (hsnd : (productSystem.sndMap A B).toElement z = (productSystem.sndMap A B).
    toElement z') :
    z = z' := by
  apply le_antisymm
  · intro p hp
    have hpz : (↑({p} : Finset (ProdToken A B)) : Set _) ⊆ z.carrier := by
      intro q hq
      have : q = p := Finset.mem_singleton.mp (Finset.mem_coe.1 hq)
      subst this
      exact hp
```

```
  have hCon : ProdCon A B {p} := z.consistent {p} hpz
  rcases p.property.out with h1 | h2
  ·
    have hy : p.val.2 ∈ ((productSystem.sndMap A B).toElement z).carrier := by
      refine ⟨{p}, hpz, ?_⟩
      refine ⟨hCon, B.con_sing p.val.2, ?_⟩
      intro y hy
      have : y = p.val.2 := Finset.mem_singleton.mp hy
      subst this
      exact B.ent_refl hCon.2 ((mem_sndFinset A B).2 ⟨p, Finset.mem_singleton_self p,
   h1, rfl⟩)
    have hy' : p.val.2 ∈ ((productSystem.sndMap A B).toElement z').carrier := by
      rw [← hsnd]
      exact hy
    rcases hy' with ⟨u, hu, ⟨huCon, _, hEnt⟩⟩
    have hEntp : (productSystem A B).Ent u p := by
      refine ⟨huCon, ?_, ?_⟩
      · intro _
        simpa [h1] using A.ent_bot huCon.1
      · intro _
        exact hEnt p.val.2 (Finset.mem_singleton_self _)
    exact z'.closed u p hu hEntp
  ·
    have hx : p.val.1 ∈ ((productSystem.fstMap A B).toElement z).carrier := by
      refine ⟨{p}, hpz, ?_⟩
      refine ⟨hCon, A.con_sing p.val.1, ?_⟩
      intro x hx
      have : x = p.val.1 := Finset.mem_singleton.mp hx
      subst this
      exact A.ent_refl hCon.1 ((mem_fstFinset A B).2 ⟨p, Finset.mem_singleton_self p,
   h2, rfl⟩)
    have hx' : p.val.1 ∈ ((productSystem.fstMap A B).toElement z').carrier := by
      rw [← hfst]
      exact hx
    rcases hx' with ⟨u, hu, ⟨huCon, _, hEnt⟩⟩
    have hEntp : (productSystem A B).Ent u p := by
      refine ⟨huCon, ?_, ?_⟩
      · intro _
        exact hEnt p.val.1 (Finset.mem_singleton_self _)
      · intro _
        simpa [h2] using B.ent_bot huCon.2
    exact z'.closed u p hu hEntp
· intro p hp
  have hpz : (↑({p} : Finset (ProdToken A B)) : Set _) ⊆ z'.carrier := by
    intro q hq
    have : q = p := Finset.mem_singleton.mp (Finset.mem_coe.1 hq)
    subst this
    exact hp
  have hCon : ProdCon A B {p} := z'.consistent {p} hpz
  rcases p.property.out with h1 | h2
  · have hy : p.val.2 ∈ ((productSystem.sndMap A B).toElement z').carrier := by
      refine ⟨{p}, hpz, ?_⟩
      refine ⟨hCon, B.con_sing p.val.2, ?_⟩
      intro y hy
      have : y = p.val.2 := Finset.mem_singleton.mp hy
      subst this
```

```
        exact B.ent_refl hCon.2 ((mem_sndFinset A B).2 ⟨p, Finset.mem_singleton_self p,
     h1, rfl⟩)
      have hy' : p.val.2 ∈ ((productSystem.sndMap A B).toElement z).carrier := by
        rw [hsnd]
        exact hy
      rcases hy' with ⟨u, hu, ⟨huCon, _, hEnt⟩⟩
      have hEntp : (productSystem A B).Ent u p := by
        refine ⟨huCon, ?_, ?_⟩
        · intro _
          simpa [h1] using A.ent_bot huCon.1
        · intro _
          exact hEnt p.val.2 (Finset.mem_singleton_self _)
      exact z.closed u p hu hEntp
    · have hx : p.val.1 ∈ ((productSystem.fstMap A B).toElement z').carrier := by
        refine ⟨{p}, hpz, ?_⟩
        refine ⟨hCon, A.con_sing p.val.1, ?_⟩
        intro x hx
        have : x = p.val.1 := Finset.mem_singleton.mp hx
        subst this
        exact A.ent_refl hCon.1 ((mem_fstFinset A B).2 ⟨p, Finset.mem_singleton_self p,
     h2, rfl⟩)
      have hx' : p.val.1 ∈ ((productSystem.fstMap A B).toElement z).carrier := by
        rw [hfst]
        exact hx
      rcases hx' with ⟨u, hu, ⟨huCon, _, hEnt⟩⟩
      have hEntp : (productSystem A B).Ent u p := by
        refine ⟨huCon, ?_, ?_⟩
        · intro _
          exact hEnt p.val.1 (Finset.mem_singleton_self _)
        · intro _
          simpa [h2] using B.ent_bot huCon.2
      exact z.closed u p hu hEntp

end productSystem

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

5.6.15 Product library

Definition (`Product`). Re-export barrel assembling the product library modules used by later constructions.

`Product.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    IsProdToken
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.prodBot
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    fstFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    sndFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.ProdCon
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.ProdEnt
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    productSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    mem_foldr_fst
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    mem_foldr_snd

@[expose] public section
```

5.6.16 Left injection

Definition (`InfoSys.mkLeft`). For $x \in D_{\mathbf{A}}$, the left injection token is $(x, \Delta_{\mathbf{B}})$ with the corresponding half-left product witness.

Definition (`InfoSys.mkLeft.instLeftCommutativeInsert`). Inserting left-injection tokens commutes on the left when building finite sets of product tokens.

mkLeft.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def mkLeft (x : α) : ProdToken A B :=
  ⟨(x, B.bot), ⟨Or.inr rfl⟩⟩
```

```
namespace mkLeft

instance instLeftCommutativeInsert :
    LeftCommutative fun x : α =>
      (insert (mkLeft A B x) : Finset (ProdToken A B) → Finset (ProdToken A B)) :=
  ⟨fun x y s => insert_comm' (mkLeft A B x) (mkLeft A B y) s⟩

end mkLeft

end InfoSys

end DomainTheory.InformationSystems
```

5.6.17 Right injection

Definition (`InfoSys.mkRight`). For $y \in D_{\mathbf{B}}$, the right injection token is $(\Delta_{\mathbf{A}}, y)$ with the corresponding half-right product witness.

Definition (`InfoSys.mkRight.instLeftCommutativeInsert`). Inserting right-injection tokens commutes on the left when building finite sets of product tokens.

mkRight.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def mkRight (y : β) : ProdToken A B :=
  ⟨(A.bot, y), ⟨Or.inl rfl⟩⟩

namespace mkRight

instance instLeftCommutativeInsert :
    LeftCommutative fun y : β =>
      (insert (mkRight A B y) : Finset (ProdToken A B) → Finset (ProdToken A B)) :=
  ⟨fun x y s => insert_comm' (mkRight A B x) (mkRight A B y) s⟩

end mkRight

end InfoSys

end DomainTheory.InformationSystems
```

5.6.18 Left lift

Definition (`InfoSys.liftLeft`). The left lift of a finite set $v \subseteq D_{\mathbf{A}}$ is the finite set of left-injection tokens $(x, \Delta_{\mathbf{B}})$ for $x \in v$.

Theorem (`InfoSys.liftLeft.mem_foldr`). Membership in a left lift built by folding insertion over a multiset equals membership of some left injection over that multiset.

Proof. Induction on the multiset, using the characterization of insert membership. ?

Theorem (`InfoSys.liftLeft.mem`). A token lies in the left lift of v iff it is a left injection of some $x \in v$.

Proof. Unfold the lift and apply the fold characterization. ?

Theorem (`InfoSys.liftLeft.fstFinset_eq`). The first projection of a left lift is the original set v.

Proof. Use membership characterizations of first projection and of the lift. ?

Theorem (`InfoSys.liftLeft.sndFinset_eq`). The second projection of a left lift is contained in $\{\Delta_{\mathbf{B}}\}$.

Proof. Each token in the lift has second component $\Delta_{\mathbf{B}}$. ?

Theorem (`InfoSys.liftLeft.con`). The left lift of a consistent set v is consistent in the product system.

Proof. First projection recovers v; second projection lies in the singleton of bottom. ?

`liftLeft.lean`

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.mkLeft

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def liftLeft (v : Finset α) : Finset (ProdToken A B) :=
  Multiset.foldr (fun x : α => insert (mkLeft A B x)) (∅ : Finset (ProdToken A B)) v.1

namespace liftLeft

theorem mem_foldr (s : Multiset α) (p : ProdToken A B) :
    p ∈ Multiset.foldr (fun x : α => insert (mkLeft A B x)) (∅ : Finset (ProdToken A B
    )) s ↔
      ∃ x ∈ s, p = mkLeft A B x := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro hp
      exact False.elim (Finset.notMem_empty p hp)
    · rintro ⟨_, hx, _⟩
```

```
      exact False.elim (by cases hx)
  · intro x t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    · rintro (hp | ⟨y, hy, hp⟩)
      · exact ⟨x, Or.inl rfl, hp⟩
      · exact ⟨y, Or.inr hy, hp⟩
    · rintro ⟨y, hy, hp⟩
      rcases hy with rfl | hy
      · exact Or.inl hp
      · exact Or.inr ⟨y, hy, hp⟩

theorem mem {v : Finset α} {p : ProdToken A B} :
    p ∈ liftLeft A B v ↔ ∃ x ∈ v, p = mkLeft A B x := by
  unfold liftLeft
  exact mem_foldr A B v.1 p

theorem fstFinset_eq (v : Finset α) : fstFinset A B (liftLeft A B v) = v := by
  ext x
  constructor
  · intro hx
    rcases (mem_fstFinset A B).1 hx with ⟨p, hp, hbot, hx'⟩
    rcases (liftLeft.mem A B).1 hp with ⟨y, hy, hp'⟩
    subst hp'
    exact (show y = x from hx').symm ▸ hy
  · intro hx
    exact (mem_fstFinset A B).2
      ⟨mkLeft A B x, (liftLeft.mem A B).2 ⟨x, hx, rfl⟩, rfl, rfl⟩

theorem sndFinset_eq (v : Finset α) :
    sndFinset A B (liftLeft A B v) ⊆ ({B.bot} : Finset β) := by
  intro y hy
  rcases (mem_sndFinset A B).1 hy with ⟨p, hp, hbot, hy'⟩
  rcases (liftLeft.mem A B).1 hp with ⟨x, _, hp'⟩
  subst hp'
  have : y = B.bot := hy'.symm
  exact Finset.mem_singleton.mpr this

theorem con {v : Finset α} (hv : v ∈ A.Con) :
    ProdCon A B (liftLeft A B v) := by
  refine ⟨?_, ?_⟩
  · rw [fstFinset_eq]
    exact hv
  · exact B.con_subset (B.con_sing B.bot) (sndFinset_eq A B v)

end liftLeft

end InfoSys

end DomainTheory.InformationSystems
```

5.6.19 Right lift

Definition (`InfoSys.liftRight`). The right lift of a finite set $w \subseteq D_{\mathbf{B}}$ is the finite set of right-injection tokens $(\Delta_{\mathbf{A}}, y)$ for $y \in w$.

Theorem (`InfoSys.liftRight.mem_foldr`). Membership in a right lift built by folding insertion over a multiset equals membership of some right injection over that multiset.

Proof. Induction on the multiset, using the characterization of insert membership. ?

Theorem (`InfoSys.liftRight.mem`). A token lies in the right lift of w iff it is a right injection of some $y \in w$.

Proof. Unfold the lift and apply the fold characterization. ?

Theorem (`InfoSys.liftRight.fstFinset_eq`). The first projection of a right lift is contained in $\{\Delta_{\mathbf{A}}\}$.

Proof. Each token in the lift has first component $\Delta_{\mathbf{A}}$. ?

Theorem (`InfoSys.liftRight.sndFinset_eq`). The second projection of a right lift is the original set w.

Proof. Use membership characterizations of second projection and of the lift. ?

Theorem (`InfoSys.liftRight.con`). The right lift of a consistent set w is consistent in the product system.

Proof. First projection lies in the singleton of bottom; second projection recovers w. ?

`liftRight.lean`

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.mkRight

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def liftRight (w : Finset β) : Finset (ProdToken A B) :=
  Multiset.foldr (fun y : β => insert (mkRight A B y)) (∅ : Finset (ProdToken A B)) w.1

namespace liftRight

theorem mem_foldr (s : Multiset β) (p : ProdToken A B) :
    p ∈ Multiset.foldr (fun y : β => insert (mkRight A B y)) (∅ : Finset (ProdToken A
    B)) s ↔
      ∃ y ∈ s, p = mkRight A B y := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro hp
      exact False.elim (Finset.notMem_empty p hp)
    · rintro ⟨_, hy, _⟩
```

```
      exact False.elim (by cases hy)
  · intro y t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    · rintro (hp | ⟨z, hz, hp⟩)
      · exact ⟨y, Or.inl rfl, hp⟩
      · exact ⟨z, Or.inr hz, hp⟩
    · rintro ⟨z, hz, hp⟩
      rcases hz with rfl | hz
      · exact Or.inl hp
      · exact Or.inr ⟨z, hz, hp⟩

theorem mem {w : Finset β} {p : ProdToken A B} :
    p ∈ liftRight A B w ↔ ∃ y ∈ w, p = mkRight A B y := by
  unfold liftRight
  exact mem_foldr A B w.1 p

theorem fstFinset_eq (w : Finset β) :
    fstFinset A B (liftRight A B w) ⊆ ({A.bot} : Finset α) := by
  intro x hx
  rcases (mem_fstFinset A B).1 hx with ⟨p, hp, hbot, hx'⟩
  rcases (liftRight.mem A B).1 hp with ⟨y, _, hp'⟩
  subst hp'
  have : x = A.bot := hx'.symm
  exact Finset.mem_singleton.mpr this

theorem sndFinset_eq (w : Finset β) : sndFinset A B (liftRight A B w) = w := by
  ext y
  constructor
  · intro hy
    rcases (mem_sndFinset A B).1 hy with ⟨p, hp, hbot, hy'⟩
    rcases (liftRight.mem A B).1 hp with ⟨z, hz, hp'⟩
    subst hp'
    exact (show z = y from hy').symm ▸ hz
  · intro hy
    exact (mem_sndFinset A B).2
      ⟨mkRight A B y, (liftRight.mem A B).2 ⟨y, hy, rfl⟩, rfl, rfl⟩

theorem con {w : Finset β} (hw : w ∈ B.Con) :
    ProdCon A B (liftRight A B w) := by
  refine ⟨?_, ?_⟩
  · exact A.con_subset (A.con_sing A.bot) (fstFinset_eq A B w)
  · rw [sndFinset_eq]
    exact hw

end liftRight

end InfoSys

end DomainTheory.InformationSystems
```

5.6.20 Pairing map

Definition (`ApproximableMap.pairMap`). For approximable mappings $f : \mathbf{C} \to \mathbf{A}$ and $g : \mathbf{C} \to \mathbf{B}$, the pairing $\langle f, g\rangle$ relates s to a product set u when s is f-related to the first projection of u and s is g-related to the second projection of u.

Theorem (`ApproximableMap.pairMap.comp_fst`). Composing the first projection after the pairing recovers f.

Proof. On relatedness, use monotonicity in the source; conversely build the pairing over a left lift and extend relatedness on the second factor using entailments into bottom. ?

Theorem (`ApproximableMap.pairMap.comp_snd`). Composing the second projection after the pairing recovers g.

Proof. Symmetric to the first projection case, using right lifts. ?

Theorem (`ApproximableMap.pairMap.unique`). An approximable mapping $h : \mathbf{C} \to \mathbf{A} \times \mathbf{B}$ whose compositions with the projections are f and g equals $\langle f, g\rangle$.

Proof. Pointwise on elements, equality of first and second components forces equality in the product by the element lemma; Proposition 5.3 finishes. ?

pairMap.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ApproximableMap
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    productSystem_more
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    liftLeft
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    liftRight
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.comp

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β)

namespace ApproximableMap

variable {C : InfoSys ?}

def pairMap (f : ApproximableMap C A) (g : ApproximableMap C B) :
    ApproximableMap C (productSystem A B) where
```

```
  rel s u := f.rel s (fstFinset A B u) ∧ g.rel s (sndFinset A B u)
  rel_dom h := f.rel_dom h.1
  rel_cod h := ⟨f.rel_cod h.1, g.rel_cod h.2⟩
  empty_rel := by
    rw [fstFinset.empty, sndFinset.empty]
    exact ⟨f.empty_rel, g.empty_rel⟩
  union_right := by
    rintro s u u' ⟨hf, hg⟩ ⟨hf', hg'⟩
    rw [fstFinset.funion A B u u', sndFinset.funion A B u u']
    exact ⟨f.union_right hf hf', g.union_right hg hg'⟩
  mono := by
    rintro s s' u u' ⟨hf, hg⟩ hEnts hEntu hs' hu'
    refine ⟨?_, ?_⟩
    · exact f.mono hf hEnts (ProdEnt.entSet_fst A B hEntu) hs' hu'.1
    · exact g.mono hg hEnts (ProdEnt.entSet_snd A B hEntu) hs' hu'.2

namespace pairMap

theorem comp_fst (f : ApproximableMap C A) (g : ApproximableMap C B) :
    comp (productSystem.fstMap A B) (pairMap A B f g) = f := by
  refine ApproximableMap.ext fun s v => ?_
  constructor
  · rintro ⟨u, ⟨hf, hg⟩, ⟨hu, hv, hEnt⟩⟩
    exact f.mono hf (EntSet.refl C (f.rel_dom hf)) hEnt (f.rel_dom hf) hv
  · intro hf
    refine ⟨liftLeft A B v, ?_, ?_⟩
    · refine ⟨?_, ?_⟩
      · rw [liftLeft.fstFinset_eq]
        exact hf
      · have hs : s ∈ C.Con := f.rel_dom hf
        have hg0 : g.rel s (∅ : Finset β) :=
          g.mono g.empty_rel (EntSet.empty C s) (EntSet.empty B ∅) hs B.con_empty
        exact rel_bot_subset g hs hg0 (liftLeft.sndFinset_eq A B v)
    · refine ⟨liftLeft.con A B (f.rel_cod hf), f.rel_cod hf, ?_⟩
      rw [liftLeft.fstFinset_eq]
      exact EntSet.refl A (f.rel_cod hf)

theorem comp_snd (f : ApproximableMap C A) (g : ApproximableMap C B) :
    comp (productSystem.sndMap A B) (pairMap A B f g) = g := by
  refine ApproximableMap.ext fun s w => ?_
  constructor
  · rintro ⟨u, ⟨hf, hg⟩, ⟨hu, hw, hEnt⟩⟩
    exact g.mono hg (EntSet.refl C (g.rel_dom hg)) hEnt (g.rel_dom hg) hw
  · intro hg
    refine ⟨liftRight A B w, ?_, ?_⟩
    · refine ⟨?_, ?_⟩
      · have hs : s ∈ C.Con := g.rel_dom hg
        have hf0 : f.rel s (∅ : Finset α) :=
          f.mono f.empty_rel (EntSet.empty C s) (EntSet.empty A ∅) hs A.con_empty
        exact rel_bot_subset_left f hs hf0 (liftRight.fstFinset_eq A B w)
      · rw [liftRight.sndFinset_eq]
        exact hg
    · refine ⟨liftRight.con A B (g.rel_cod hg), g.rel_cod hg, ?_⟩
      rw [liftRight.sndFinset_eq]
      exact EntSet.refl B (g.rel_cod hg)
```

```
theorem unique (f : ApproximableMap C A) (g : ApproximableMap C B)
    (h : ApproximableMap C (productSystem A B))
    (hfst : comp (productSystem.fstMap A B) h = f) (hsnd : comp (productSystem.sndMap
   A B) h = g) :
    h = pairMap A B f g :=
  (ext_iff_toElement h (pairMap A B f g)).2 fun x =>
    productSystem.element_eq_of_fst_snd A B (h.toElement x) ((pairMap A B f g).
   toElement x)
      (by
        have h1 : (productSystem.fstMap A B).toElement (h.toElement x) = f.toElement x
     := by
          rw [← comp.toElement, hfst]
        have h2 : (productSystem.fstMap A B).toElement ((pairMap A B f g).toElement x)
     = f.toElement x := by
          rw [← comp.toElement, comp_fst]
        exact h1.trans h2.symm)
      (by
        have h1 : (productSystem.sndMap A B).toElement (h.toElement x) = g.toElement x
     := by
          rw [← comp.toElement, hsnd]
        have h2 : (productSystem.sndMap A B).toElement ((pairMap A B f g).toElement x)
     = g.toElement x := by
          rw [← comp.toElement, comp_snd]
        exact h1.trans h2.symm)

end pairMap

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

5.6.21 Proposition 6.2: projections and pairing

PROPOSITION 6.2. *If* $\mathbf{A}$ *and* $\mathbf{B}$ *are information systems, then so is* $\mathbf{A} \times \mathbf{B}$*, and we have mappings*

$$\mathrm{fst} : \mathbf{A} \times \mathbf{B} \to \mathbf{A} \text{ and } \mathrm{snd} : \mathbf{A} \times \mathbf{B} \to \mathbf{B},$$

such that, for approximable mappings

$$f : \mathbf{C} \to \mathbf{A} \text{ and } g : \mathbf{C} \to \mathbf{B},$$

there is one and only one approximable mapping $\langle f, g\rangle : \mathbf{C} \to \mathbf{A} \times \mathbf{B}$ *such that*

$$\mathrm{fst} \circ \langle f, g\rangle = f \text{ and } \mathrm{snd} \circ \langle f, g\rangle = g.$$

Proof. Checking that 6.1 does indeed define an information system is straightforward, but there are six things that must be proved according to Definition 2.1. We have to leave the details to the reader. Next using the notation of 6.1, where fst and snd were applied as operations on consistent sets $u \in \mathrm{Con}_{\mathbf{A}\times\mathbf{B}}$, we redefine matters to have approximable mappings, where, for $v \in \mathrm{Con}_{\mathbf{A}}$ and $w \in \mathrm{Con}_{\mathbf{B}}$,

(1) u fst v iff $\mathrm{fst}\, u \vdash_{\mathbf{A}} v$;

(2) $u \ \mathrm{snd}\ w$ iff $\mathrm{snd}\, u \vdash_{\mathbf{B}} w$; and
(3) $s \ \langle f, g\rangle \ u$ iff $s \ f \ (\mathrm{fst}\, u)$ and $s \ g \ (\mathrm{snd}\, u)$.

Because we defined $\mathbf{A} \times \mathbf{B}$ so that consistency and entailment worked independently on the two halves of the set of data objects, it is easy to check that (1)–(3) define approximable mappings having the desired properties.

The uniqueness of $\langle f, g\rangle$ comes out of the observation that, if z and z' are two elements of $\mathbf{A} \times \mathbf{B}$ for which

$$\mathrm{fst}(z) = \mathrm{fst}(z') \text{ and } \mathrm{snd}(z) = \mathrm{snd}(z'),$$

then $z = z'$. The reason is that fst and snd just divide elements into the two kinds of data objects, and then strip off the parentheses. (Look back at Definition 6.1.) No information is lost, so if z and z' are transformed into the same elements both times, then they have to be the same.

That lemma treats one pair of elements at a time, but $\langle f, g\rangle$ is a function. But if $\langle f, g\rangle'$ were another function satisfying the conditions of the above proposition, then the two functions would be pointwise equal. We could then quote 5.3 to assure ourselves that they are the same function. ■

Theorem (`ApproximableMap.proposition_6_2`). For approximable mappings $f : \mathbf{C} \to \mathbf{A}$, $g : \mathbf{C} \to \mathbf{B}$, and $h : \mathbf{C} \to \mathbf{A} \times \mathbf{B}$, if $\mathrm{fst} \circ h = f$ and $\mathrm{snd} \circ h = g$, then $h = \langle f, g\rangle$.

Proof. Uniqueness of the pairing. ?

Ordered pairs. By using the definition

$$(x, y) = \langle \mathrm{const}(x), \mathrm{const}(y)\rangle(\bot_{\mathbf{C}}),$$

which invokes 6.2 on any convenient fixed domain $\mathbf{C}$, it is easy to prove that $|\mathbf{A} \times \mathbf{B}|$ is in a one-one correspondence with the set-theoretical product of $|\mathbf{A}|$ and $|\mathbf{B}|$. Indeed, it can be shown that for $x \in |\mathbf{A}|$ and $y \in |\mathbf{B}|$,

(1) $(x, y) = \{(X, \Delta_{\mathbf{B}}) \mid X \in x\} \cup \{(\Delta_{\mathbf{A}}, Y) \mid Y \in y\} \in |\mathbf{A} \times \mathbf{B}|$;
(2) $\mathrm{fst}(x, y) = x$;
(3) $\mathrm{snd}(x, y) = y$;
and, for all $z \in |\mathbf{A} \times \mathbf{B}|$,
(4) $z = (\mathrm{fst}\, z, \mathrm{snd}\, z)$.
Also, using the notation of 6.2, we can say that
(5) $\langle f, g\rangle(t) = (f(t), g(t))$,
for all $t \in |\mathbf{C}|$.

There are also remarks that could be made about the pointwise nature of the partial ordering of $|\mathbf{A} \times \mathbf{B}|$, but we will not formulate them here. We do remark, however, that there is also a trivial product of no terms, $\mathbf{1}$, called the *unit type* or *domain*. It is such that $D_{\mathbf{1}} = \{\Delta_{\mathbf{1}}\}$, and

that equation determines it up to isomorphism. The domain $\mathbf{1}$ has but one element, namely $\bot_{\mathbf{1}}$. Note also that all approximable mappings $f : \mathbf{1} \to \mathbf{A}$ are constant, which shows how Definition 5.1 is a generalization of Definition 3.1. Note finally that there is but one approximable mapping $f : \mathbf{A} \to \mathbf{1}$, namely $f = 0 = \mathrm{const}(\bot_{\mathbf{1}})$. ■

We turn now to the definition and properties of sums of domains.

`proposition_6_2.lean`

```
module
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    productSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.pairMap

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys.ApproximableMap

theorem proposition_6_2 :
    ∀ {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
      (A : InfoSys α) (B : InfoSys β) {C : InfoSys ?}
      (f : ApproximableMap C A) (g : ApproximableMap C B)
      (h : ApproximableMap C (productSystem A B)),
      comp (productSystem.fstMap A B) h = f →
        comp (productSystem.sndMap A B) h = g →
          h = pairMap A B f g :=
  @pairMap.unique

end DomainTheory.InformationSystems.InfoSys.ApproximableMap
```

5.7 Sums of domains

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

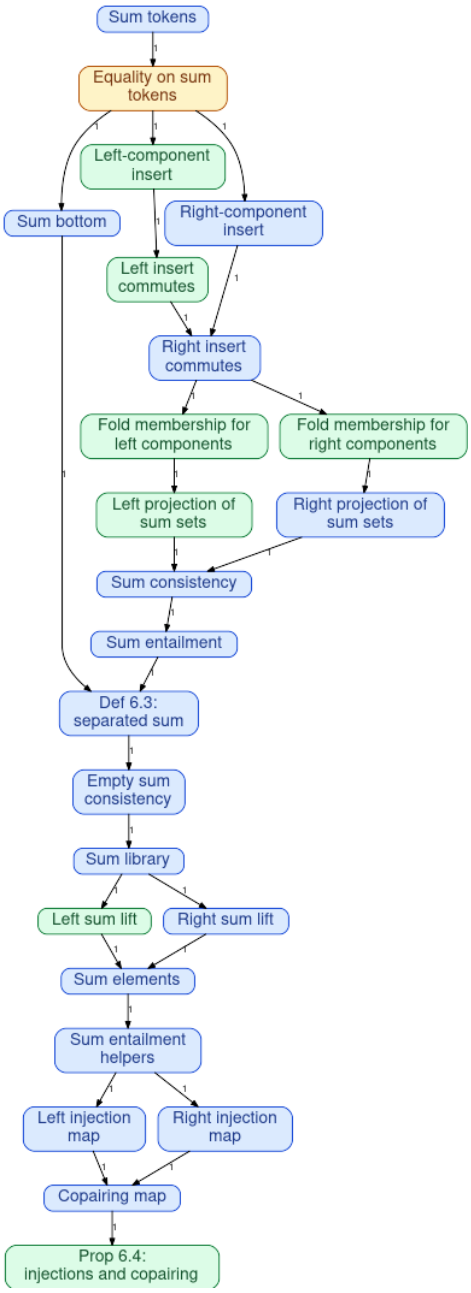


Figure 60: Diagram 25

5.7.1 Sum tokens

Definition (`SumToken`). Sum tokens are a left injection of an **A**-datum, a right injection of a **B**-datum, or the shared sum bottom.

`SumToken.lean`

```
module
```

```
public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

inductive SumToken (α β : Type*) where
  | left : α → SumToken α β
  | right : β → SumToken α β
  | bot : SumToken α β

end InfoSys
end DomainTheory.InformationSystems
```

5.7.2 Equality on sum tokens

Definition (`instDecidableEqSumToken`). Sum tokens have decidable equality.

`instDecidableEqSumToken.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

instance instDecidableEqSumToken : DecidableEq (SumToken α β)
  | .left a, .left b =>
      if h : a = b then isTrue (h ▸ rfl)
      else isFalse fun h' => h (SumToken.left.inj h')
  | .right a, .right b =>
      if h : a = b then isTrue (h ▸ rfl)
      else isFalse fun h' => h (SumToken.right.inj h')
  | .bot, .bot => isTrue rfl
  | .left _, .right _ => isFalse fun h => nomatch h
  | .left _, .bot => isFalse fun h => nomatch h
  | .right _, .left _ => isFalse fun h => nomatch h
```

```
  | .right _, .bot => isFalse fun h => nomatch h
  | .bot, .left _ => isFalse fun h => nomatch h
  | .bot, .right _ => isFalse fun h => nomatch h

end InfoSys

end DomainTheory.InformationSystems
```

5.7.3 Sum bottom

Definition (`sumBot`). The least sum token is the dedicated bottom constructor.

`sumBot.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def sumBot : SumToken α β := .bot

end InfoSys

end DomainTheory.InformationSystems
```

5.7.4 Left-component insert

Definition (`lftInsert`). Left insertion adds an $\mathbf{A}$-datum from a left token and ignores right tokens and bottom.

`lftInsert.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def lftInsert : SumToken α β → Finset α → Finset α
  | .left x => insert x
  | .right _ => id
  | .bot => id

end InfoSys

end DomainTheory.InformationSystems
```

5.7.5 Right-component insert

Definition (`rhtInsert`). Right insertion adds a $\mathbf{B}$-datum from a right token and ignores left tokens and bottom.

rhtInsert.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def rhtInsert : SumToken α β → Finset β → Finset β
  | .right y => insert y
  | .left _ => id
  | .bot => id
```

```
end InfoSys

end DomainTheory.InformationSystems
```

5.7.6 Left insert commutes

Definition (`instLeftCommutativeLftInsert`). Left-component insertion commutes on the left.

`instLeftCommutativeLftInsert.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

instance instLeftCommutativeLftInsert :
    LeftCommutative (lftInsert : SumToken α β → Finset α → Finset α) :=
  ⟨fun p q s => by
    cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl⟩

end InfoSys

end DomainTheory.InformationSystems
```

5.7.7 Right insert commutes

Definition (`instLeftCommutativeRhtInsert`). Right-component insertion commutes on the left.

`instLeftCommutativeRhtInsert.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

instance instLeftCommutativeRhtInsert :
    LeftCommutative (rhtInsert : SumToken α β → Finset β → Finset β) :=
  ⟨fun p q s => by
    cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl⟩

end InfoSys

end DomainTheory.InformationSystems
```

5.7.8 Fold membership for left components

Theorem (`mem_foldr_lft`). Membership after folding left insertion characterises the accumulated left components.

Proof. Direct from the definitions and the ambient information-system axioms. ?

`mem_foldr_lft.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

omit [DecidableEq β] in

theorem mem_foldr_lft (s : Multiset (SumToken α β)) (x : α) :
    x ∈ Multiset.foldr lftInsert (∅ : Finset α) s ↔ ∃ p ∈ s, p = .left x := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro hx
      exact False.elim (Finset.notMem_empty x hx)
    · rintro ⟨_, hp, _⟩
      exact False.elim (by cases hp)
  · intro p t ih
    cases p with
    | left a =>
      simp only [Multiset.foldr_cons, lftInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      · rintro (hx | ⟨q, hq, hq'⟩)
        · exact ⟨.left a, Or.inl rfl, congrArg SumToken.left hx.symm⟩
        · exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · injection hq' with hx
          exact Or.inl hx.symm
        · exact Or.inr ⟨q, hq, hq'⟩
    | right b =>
      simp only [Multiset.foldr_cons, lftInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      · rintro ⟨q, hq, hq'⟩
        exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · exact False.elim (nomatch hq')
        · exact ⟨q, hq, hq'⟩
    | bot =>
      simp only [Multiset.foldr_cons, lftInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      · rintro ⟨q, hq, hq'⟩
        exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · exact False.elim (nomatch hq')
        · exact ⟨q, hq, hq'⟩

end InfoSys

end DomainTheory.InformationSystems
```

5.7.9 Fold membership for right components

Theorem (`mem_foldr_rht`). Membership after folding right insertion characterises the accumulated right components.

Proof. Direct from the definitions and the ambient information-system axioms. ?

`mem_foldr_rht.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

omit [DecidableEq α] in

theorem mem_foldr_rht (s : Multiset (SumToken α β)) (y : β) :
    y ∈ Multiset.foldr rhtInsert (∅ : Finset β) s ↔ ∃ p ∈ s, p = .right y := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro hy
      exact False.elim (Finset.notMem_empty y hy)
    · rintro ⟨_, hp, _⟩
      exact False.elim (by cases hp)
  · intro p t ih
    cases p with
    | right b =>
      simp only [Multiset.foldr_cons, rhtInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      · rintro (hy | ⟨q, hq, hq'⟩)
        · exact ⟨.right b, Or.inl rfl, congrArg SumToken.right hy.symm⟩
        · exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · injection hq' with hy
          exact Or.inl hy.symm
```

```
        · exact Or.inr ⟨q, hq, hq'⟩
    | left a =>
      simp only [Multiset.foldr_cons, rhtInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      · rintro ⟨q, hq, hq'⟩
        exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · exact False.elim (nomatch hq')
        · exact ⟨q, hq, hq'⟩
    | bot =>
      simp only [Multiset.foldr_cons, rhtInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      · rintro ⟨q, hq, hq'⟩
        exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · exact False.elim (nomatch hq')
        · exact ⟨q, hq, hq'⟩

end InfoSys

end DomainTheory.InformationSystems
```

5.7.10 Left projection of sum sets

Definition (`lftFinset`). The left projection of a finite set of sum tokens collects the $\mathbf{A}$-data of left tokens.

Theorem (`mem_lftFinset`). Membership in the left projection matches a witnessing left token.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`lftFinset.empty`). The left projection of the empty set is empty.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`lftFinset.funion`). Left projection preserves finite unions.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`lftFinset.insert_bot`). Inserting bottom leaves the left projection unchanged.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`lftFinset.insert_left`). Inserting a left token adjoins its datum.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`lftFinset.insert_right`). Inserting a right token leaves the left projection unchanged.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`lftFinset.mono`). Left projection is monotone.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`lftFinset.singleton_bot`). The left projection of singleton bottom is empty.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`lftFinset.singleton_left`). The left projection of a singleton left token is that datum.

Proof. Direct from the definitions and the ambient information-system axioms. ?

`lftFinset.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_lft

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]
variable (A : InfoSys α) (B : InfoSys β)

def lftFinset (u : Finset (SumToken α β)) : Finset α :=
  Multiset.foldr lftInsert (∅ : Finset α) u.1

omit [DecidableEq β] in

theorem mem_lftFinset {u : Finset (SumToken α β)} {x : α} :
    x ∈ lftFinset u ↔ SumToken.left x ∈ u := by
  unfold lftFinset
  rw [mem_foldr_lft]
  constructor
  · rintro ⟨p, hp, hp'⟩
    rwa [← hp']
  · intro hx
    exact ⟨.left x, hx, rfl⟩

omit [DecidableEq β] in

theorem lftFinset.empty : lftFinset (∅ : Finset (SumToken α β)) = ∅ := by
  ext x
  constructor
  · intro hx
    exact False.elim (Finset.notMem_empty _ ((mem_lftFinset).1 hx))
  · intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem lftFinset.funion (u v : Finset (SumToken α β)) :
    lftFinset (u ∪' v) = lftFinset u ∪' lftFinset v := by
  ext x
  constructor
  · intro hx
```

```
    have : SumToken.left x ∈ u ∪' v := (mem_lftFinset).1 hx
    rcases funion.mem.mp this with h | h
    · exact funion.mem.mpr (Or.inl ((mem_lftFinset).2 h))
    · exact funion.mem.mpr (Or.inr ((mem_lftFinset).2 h))
  · intro hx
    rcases funion.mem.mp hx with h | h
    · exact (mem_lftFinset).2 (funion.mem.mpr (Or.inl ((mem_lftFinset).1 h)))
    · exact (mem_lftFinset).2 (funion.mem.mpr (Or.inr ((mem_lftFinset).1 h)))

theorem lftFinset.insert_bot (u : Finset (SumToken α β)) :
    lftFinset (insert (.bot : SumToken α β) u) = lftFinset u := by
  ext x
  constructor
  · intro hx
    have : SumToken.left x ∈ insert .bot u := (mem_lftFinset).1 hx
    rcases Finset.mem_insert.mp this with h | h
    · exact False.elim (nomatch h)
    · exact (mem_lftFinset).2 h
  · intro hx
    exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).1 hx)))

theorem lftFinset.insert_left (u : Finset (SumToken α β)) (x : α) :
    lftFinset (insert (.left x) u) = insert x (lftFinset u) := by
  ext y
  constructor
  · intro hy
    have : SumToken.left y ∈ insert (.left x) u := (mem_lftFinset).1 hy
    rcases Finset.mem_insert.mp this with h | h
    · injection h with hy'
      exact Finset.mem_insert.mpr (Or.inl hy')
    · exact Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).2 h))
  · intro hy
    rcases Finset.mem_insert.mp hy with hy' | hy'
    · exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inl (congrArg SumToken.left hy
    ')))
    · exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).1 hy')))

theorem lftFinset.insert_right (u : Finset (SumToken α β)) (y : β) :
    lftFinset (insert (.right y) u) = lftFinset u := by
  ext x
  constructor
  · intro hx
    have : SumToken.left x ∈ insert (.right y) u := (mem_lftFinset).1 hx
    rcases Finset.mem_insert.mp this with h | h
    · exact False.elim (nomatch h)
    · exact (mem_lftFinset).2 h
  · intro hx
    exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).1 hx)))

omit [DecidableEq β] in

theorem lftFinset.mono {u v : Finset (SumToken α β)} (h : v ⊆ u) :
    lftFinset v ⊆ lftFinset u := by
  intro x hx
  exact (mem_lftFinset).2 (h ((mem_lftFinset).1 hx))
```

```
omit [DecidableEq β] in

theorem lftFinset.singleton_bot :
    lftFinset ({.bot} : Finset (SumToken α β)) = ∅ := by
  ext x
  constructor
  · intro hx
    have : SumToken.left x ∈ ({.bot} : Finset _) := (mem_lftFinset).1 hx
    exact False.elim (nomatch Finset.mem_singleton.mp this)
  · intro hx
    exact False.elim (Finset.notMem_empty x hx)

omit [DecidableEq β] in

theorem lftFinset.singleton_left (x : α) :
    lftFinset ({.left x} : Finset (SumToken α β)) = {x} := by
  ext y
  simp only [mem_lftFinset, Finset.mem_singleton]
  exact ⟨SumToken.left.inj, fun h => h ▸ rfl⟩

theorem lftFinset.eq_empty_or_mem (u : Finset (SumToken α β)) :
    lftFinset u = ∅ ∨ ∃ x, x ∈ lftFinset u := by
  induction u using Finset.induction_on with
  | empty =>
    exact Or.inl lftFinset.empty
  | insert p s _hp ih =>
    cases p with
    | left x =>
      exact Or.inr ⟨x, by rw [lftFinset.insert_left]; exact Finset.mem_insert_self _ _⟩
    | right y =>
      simpa [lftFinset.insert_right] using ih
    | bot =>
      simpa [lftFinset.insert_bot] using ih

end InfoSys

end DomainTheory.InformationSystems
```

5.7.11 Right projection of sum sets

Definition (`rhtFinset`). The right projection of a finite set of sum tokens collects the $\mathbf{B}$-data of right tokens.

Theorem (`mem_rhtFinset`). Membership in the right projection matches a witnessing right token.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`rhtFinset.empty`). The right projection of the empty set is empty.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`rhtFinset.funion`). Right projection preserves finite unions.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`rhtFinset.insert_bot`). Inserting bottom leaves the right projection unchanged.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`rhtFinset.insert_left`). Inserting a left token leaves the right projection unchanged.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`rhtFinset.insert_right`). Inserting a right token adjoins its datum.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`rhtFinset.mono`). Right projection is monotone.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`rhtFinset.singleton_bot`). The right projection of singleton bottom is empty.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`rhtFinset.singleton_right`). The right projection of a singleton right token is that datum.

Proof. Direct from the definitions and the ambient information-system axioms. ?

rhtFinset.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_rht

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]
variable (A : InfoSys α) (B : InfoSys β)

def rhtFinset (u : Finset (SumToken α β)) : Finset β :=
  Multiset.foldr rhtInsert (∅ : Finset β) u.1

omit [DecidableEq α] in

theorem mem_rhtFinset {u : Finset (SumToken α β)} {y : β} :
    y ∈ rhtFinset u ↔ SumToken.right y ∈ u := by
  unfold rhtFinset
  rw [mem_foldr_rht]
  constructor
  · rintro ⟨p, hp, hp'⟩
    rwa [← hp']
  · intro hy
    exact ⟨.right y, hy, rfl⟩

omit [DecidableEq α] in
```

```
theorem rhtFinset.empty : rhtFinset (∅ : Finset (SumToken α β)) = ∅ := by
  ext y
  constructor
  · intro hy
    exact False.elim (Finset.notMem_empty _ ((mem_rhtFinset).1 hy))
  · intro hy
    exact False.elim (Finset.notMem_empty y hy)

theorem rhtFinset.funion (u v : Finset (SumToken α β)) :
    rhtFinset (u ∪' v) = rhtFinset u ∪' rhtFinset v := by
  ext y
  constructor
  · intro hy
    have : SumToken.right y ∈ u ∪' v := (mem_rhtFinset).1 hy
    rcases funion.mem.mp this with h | h
    · exact funion.mem.mpr (Or.inl ((mem_rhtFinset).2 h))
    · exact funion.mem.mpr (Or.inr ((mem_rhtFinset).2 h))
  · intro hy
    rcases funion.mem.mp hy with h | h
    · exact (mem_rhtFinset).2 (funion.mem.mpr (Or.inl ((mem_rhtFinset).1 h)))
    · exact (mem_rhtFinset).2 (funion.mem.mpr (Or.inr ((mem_rhtFinset).1 h)))

theorem rhtFinset.insert_bot (u : Finset (SumToken α β)) :
    rhtFinset (insert (.bot : SumToken α β) u) = rhtFinset u := by
  ext y
  constructor
  · intro hy
    have : SumToken.right y ∈ insert .bot u := (mem_rhtFinset).1 hy
    rcases Finset.mem_insert.mp this with h | h
    · exact False.elim (nomatch h)
    · exact (mem_rhtFinset).2 h
  · intro hy
    exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).1 hy)))

theorem rhtFinset.insert_left (u : Finset (SumToken α β)) (x : α) :
    rhtFinset (insert (.left x) u) = rhtFinset u := by
  ext y
  constructor
  · intro hy
    have : SumToken.right y ∈ insert (.left x) u := (mem_rhtFinset).1 hy
    rcases Finset.mem_insert.mp this with h | h
    · exact False.elim (nomatch h)
    · exact (mem_rhtFinset).2 h
  · intro hy
    exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).1 hy)))

theorem rhtFinset.insert_right (u : Finset (SumToken α β)) (y : β) :
    rhtFinset (insert (.right y) u) = insert y (rhtFinset u) := by
  ext z
  constructor
  · intro hz
    have : SumToken.right z ∈ insert (.right y) u := (mem_rhtFinset).1 hz
    rcases Finset.mem_insert.mp this with h | h
    · injection h with hz'
      exact Finset.mem_insert.mpr (Or.inl hz')
```

```
    · exact Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).2 h))
  · intro hz
    rcases Finset.mem_insert.mp hz with hz' | hz'
    · exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inl (congrArg SumToken.right
    hz')))
    · exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).1 hz')))

omit [DecidableEq α] in

theorem rhtFinset.mono {u v : Finset (SumToken α β)} (h : v ⊆ u) :
    rhtFinset v ⊆ rhtFinset u := by
  intro y hy
  exact (mem_rhtFinset).2 (h ((mem_rhtFinset).1 hy))

omit [DecidableEq α] in

theorem rhtFinset.singleton_bot :
    rhtFinset ({.bot} : Finset (SumToken α β)) = ∅ := by
  ext y
  constructor
  · intro hy
    have : SumToken.right y ∈ ({.bot} : Finset _) := (mem_rhtFinset).1 hy
    exact False.elim (nomatch Finset.mem_singleton.mp this)
  · intro hy
    exact False.elim (Finset.notMem_empty y hy)

omit [DecidableEq α] in

theorem rhtFinset.singleton_right (y : β) :
    rhtFinset ({.right y} : Finset (SumToken α β)) = {y} := by
  ext z
  simp only [mem_rhtFinset, Finset.mem_singleton]
  exact ⟨SumToken.right.inj, fun h => h ▸ rfl⟩

theorem rhtFinset.eq_empty_or_mem (u : Finset (SumToken α β)) :
    rhtFinset u = ∅ ∨ ∃ y, y ∈ rhtFinset u := by
  induction u using Finset.induction_on with
  | empty =>
    exact Or.inl rhtFinset.empty
  | insert p s _hp ih =>
    cases p with
    | right y =>
      exact Or.inr ⟨y, by rw [rhtFinset.insert_right]; exact Finset.mem_insert_self _ _
    ⟩
    | left x =>
      simpa [rhtFinset.insert_left] using ih
    | bot =>
      simpa [rhtFinset.insert_bot] using ih

end InfoSys

end DomainTheory.InformationSystems
```

5.7.12 Sum consistency

Definition (`SumCon`). A finite set of sum tokens is consistent when exactly one side is a consistent nonempty projection and the other is empty, or both projections are empty under the left-consistent clause.

Theorem (`SumCon.lft_con_of_lft_nonempty`). A nonempty left projection of a consistent sum set is consistent in $\mathbf{A}$.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`SumCon.lft_empty_of_rht_nonempty`). A nonempty right projection forces the left projection empty.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`SumCon.rht_con_of_rht_nonempty`). A nonempty right projection of a consistent sum set is consistent in $\mathbf{B}$.

Proof. Direct from the definitions and the ambient information-system axioms. ?

Theorem (`SumCon.rht_empty_of_lft_nonempty`). A nonempty left projection forces the right projection empty.

Proof. Direct from the definitions and the ambient information-system axioms. ?

SumCon.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]
variable (A : InfoSys α) (B : InfoSys β)

def SumCon (u : Finset (SumToken α β)) : Prop :=
  (lftFinset u ∈ A.Con ∧ rhtFinset u = ∅) ∨
    (lftFinset u = ∅ ∧ rhtFinset u ∈ B.Con)

theorem SumCon.lft_con_of_lft_nonempty {u : Finset (SumToken α β)}
    (hu : SumCon A B u) (hne : lftFinset u = ∅) : lftFinset u ∈ A.Con := by
  rcases hu with ⟨hl, _⟩ | ⟨hl, _⟩
  · exact hl
  · exact False.elim (hne hl)
```

```
theorem SumCon.lft_empty_of_rht_nonempty {u : Finset (SumToken α β)}
    (hu : SumCon A B u) (hne : rhtFinset u = ∅) : lftFinset u = ∅ := by
  rcases hu with ⟨_, hr⟩ | ⟨hl, _⟩
  · exact False.elim (hne hr)
  · exact hl

theorem SumCon.rht_con_of_rht_nonempty {u : Finset (SumToken α β)}
    (hu : SumCon A B u) (hne : rhtFinset u = ∅) : rhtFinset u ∈ B.Con := by
  rcases hu with ⟨_, hr⟩ | ⟨_, hr⟩
  · exact False.elim (hne hr)
  · exact hr

theorem SumCon.rht_empty_of_lft_nonempty {u : Finset (SumToken α β)}
    (hu : SumCon A B u) (hne : lftFinset u = ∅) : rhtFinset u = ∅ := by
  rcases hu with ⟨_, hr⟩ | ⟨hl, _⟩
  · exact hr
  · exact False.elim (hne hl)

theorem SumCon.lft_mem_of_rht_empty {u : Finset (SumToken α β)}
    (hu : SumCon A B u) (_hr : rhtFinset u = ∅) : lftFinset u ∈ A.Con := by
  rcases hu with ⟨hl, _⟩ | ⟨hl, _⟩
  · exact hl
  · rw [hl]; exact A.con_empty

theorem SumCon.rht_mem_of_lft_empty {u : Finset (SumToken α β)}
    (hu : SumCon A B u) (_hl : lftFinset u = ∅) : rhtFinset u ∈ B.Con := by
  rcases hu with ⟨_, hr⟩ | ⟨_, hr⟩
  · rw [hr]; exact B.con_empty
  · exact hr

end InfoSys

end DomainTheory.InformationSystems
```

5.7.13 Sum entailment

Definition (`SumEnt`). Sum entailment of a token follows the inhabited side's factor entailment, with bottom always entailed.

`SumEnt.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtFinset
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumCon

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def SumEnt (u : Finset (SumToken α β)) (p : SumToken α β) : Prop :=
  SumCon A B u ∧
    match p with
    | .bot => True
    | .left x => lftFinset u = ∅ ∧ A.Ent (lftFinset u) x
    | .right y => rhtFinset u = ∅ ∧ B.Ent (rhtFinset u) y

end InfoSys

end DomainTheory.InformationSystems
```

5.7.14 Definition 6.3: separated sum

DEFINITION 6.3. Let $\mathbf{A}$ and $\mathbf{B}$ be two information systems. By $\mathbf{A}+\mathbf{B}$, the *separated sum system*, we understand the system where, after choosing some convenient object Δ belonging neither to $D_{\mathbf{A}}$ nor to $D_{\mathbf{B}}$, we have:

(i) $D_{\mathbf{A}+\mathbf{B}} = \{(X,\Delta) \mid X \in D_{\mathbf{A}}\} \cup \{(\Delta,Y) \mid Y \in D_{\mathbf{B}}\} \cup \{(\Delta,\Delta)\}$;

(ii) $\Delta_{\mathbf{A}+\mathbf{B}} = (\Delta,\Delta)$;

(iii) $u \in \mathrm{Con}_{\mathbf{A}+\mathbf{B}}$ iff either $\mathrm{lft}\, u \in \mathrm{Con}_{\mathbf{A}}$ and $\mathrm{rht}\, u = \emptyset$ or $\mathrm{lft}\, u = \emptyset$ and $\mathrm{rht}\, u \in \mathrm{Con}_{\mathbf{B}}$;

(iv') $u \vdash_{\mathbf{A}+\mathbf{B}} (X',\Delta)$ iff $\mathrm{lft}\, u \neq \emptyset$ and $\mathrm{lft}\, u \vdash_{\mathbf{A}} X'$;
(iv'') $u \vdash_{\mathbf{A}+\mathbf{B}} (\Delta,Y')$ iff $\mathrm{rht}\, u \neq \emptyset$ and $\mathrm{rht}\, u \vdash_{\mathbf{B}} Y'$; and
(iv''') $u \vdash_{\mathbf{A}+\mathbf{B}} (\Delta,\Delta)$ always holds,
where, in (iii), u is any finite subset of $D_{\mathbf{A}+\mathbf{B}}$, in (iv')–(iv'''), $u \in \mathrm{Con}_{\mathbf{A}+\mathbf{B}}$, and we let:

$$\mathrm{lft}\, u = \{X \in D_{\mathbf{A}} \mid (X,\Delta) \in u\}, \quad \mathrm{rht}\, u = \{Y \in D_{\mathbf{B}} \mid (\Delta,Y) \in u\}.$$

■

The plan of the sum definition is very similar to that for product, except that (1) for reasons to be made clear in examples, the parts do not share the least informative element (i.e. the data objects $(\Delta_{\mathbf{A}},\Delta)$, $(\Delta,\Delta_{\mathbf{B}})$, and (Δ,Δ) are inequivalent in this system), and (2) instead of defining consistency and entailment in a conjunctive way, these notions are defined disjunctively. The effect of these changes over Definition 6.1 is to produce a system $\mathbf{A}+\mathbf{B}$ whose elements divide into disjoint copies of those of $\mathbf{A}$ and $\mathbf{B}$ (plus an extra element $\bot_{\mathbf{A}+\mathbf{B}}$). These remarks can be made more precise in the following way:

Definition (`InfoSys.sumSystem`). The separated sum of information systems $\mathbf{A}$ and $\mathbf{B}$ is the information system whose data are the sum tokens of Definition 6.3(i), whose least informative object is the shared sum bottom (Δ, Δ), whose consistent sets are those that live entirely on the left factor or entirely on the right factor as in 6.3(iii), and whose entailment of a token is the corresponding one-sided entailment of 6.3(iv')–(iv'') (with the sum bottom always entailed). The information-system axioms hold by case analysis on sum tokens, using the left and right projections together with the consistency and entailment axioms of the occupied factor.

sumSystem.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumBot
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumEnt
import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def sumSystem : InfoSys (SumToken α β) where
  bot := sumBot
  Con := {u | SumCon A B u}
  Ent := SumEnt A B
  con_subset := by
    intro u v hu hv
    rcases hu with ⟨hl, hr⟩ | ⟨hl, hr⟩
    · exact Or.inl ⟨A.con_subset hl (lftFinset.mono hv),
        Finset.Subset.antisymm (by
          intro y hy
          have : y ∈ rhtFinset u := rhtFinset.mono hv hy
          rw [hr] at this
          exact False.elim (Finset.notMem_empty y this))
          (Finset.empty_subset _)⟩
    · exact Or.inr ⟨Finset.Subset.antisymm (by
          intro x hx
          have : x ∈ lftFinset u := lftFinset.mono hv hx
          rw [hl] at this
          exact False.elim (Finset.notMem_empty x this))
          (Finset.empty_subset _),
        B.con_subset hr (rhtFinset.mono hv)⟩
  con_sing := by
    intro p
    cases p with
    | left x =>
      exact Or.inl ⟨by rw [lftFinset.singleton_left]; exact A.con_sing x,
        by
          ext y
          constructor
```

```
        · intro hy
          have : SumToken.right y ∈ ({.left x} : Finset _) := (mem_rhtFinset).1 hy
          exact False.elim (nomatch Finset.mem_singleton.mp this)
        · intro hy
          exact False.elim (Finset.notMem_empty y hy)⟩
  | right y =>
    exact Or.inr ⟨by
        ext x
        constructor
        · intro hx
          have : SumToken.left x ∈ ({.right y} : Finset _) := (mem_lftFinset).1 hx
          exact False.elim (nomatch Finset.mem_singleton.mp this)
        · intro hx
          exact False.elim (Finset.notMem_empty x hx),
      by rw [rhtFinset.singleton_right]; exact B.con_sing y⟩
  | bot =>
    exact Or.inl ⟨by rw [lftFinset.singleton_bot]; exact A.con_empty,
      rhtFinset.singleton_bot⟩
ent_con := by
  intro u p ⟨hu, hEnt⟩
  cases p with
  | bot =>
    rcases hu with ⟨hl, hr⟩ | ⟨hl, hr⟩
    · exact Or.inl ⟨by rw [lftFinset.insert_bot]; exact hl,
        by rw [rhtFinset.insert_bot]; exact hr⟩
    · exact Or.inr ⟨by rw [lftFinset.insert_bot]; exact hl,
        by rw [rhtFinset.insert_bot]; exact hr⟩
  | left x =>
    rcases hEnt with ⟨hne, hA⟩
    have hr : rhtFinset u = ∅ := SumCon.rht_empty_of_lft_nonempty A B hu hne
    exact Or.inl ⟨by
        rw [lftFinset.insert_left]
        exact A.ent_con hA,
      by rw [rhtFinset.insert_left]; exact hr⟩
  | right y =>
    rcases hEnt with ⟨hne, hB⟩
    have hl : lftFinset u = ∅ := SumCon.lft_empty_of_rht_nonempty A B hu hne
    exact Or.inr ⟨by rw [lftFinset.insert_right]; exact hl,
      by
        rw [rhtFinset.insert_right]
        exact B.ent_con hB⟩
ent_bot := by
  intro u hu
  exact ⟨hu, trivial⟩
ent_refl := by
  intro u p hu hp
  refine ⟨hu, ?_⟩
  cases p with
  | bot => exact trivial
  | left x =>
    have hx : x ∈ lftFinset u := (mem_lftFinset).2 hp
    have hne : lftFinset u = ∅ := Finset.ne_empty_of_mem hx
    exact ⟨hne, A.ent_refl (SumCon.lft_con_of_lft_nonempty A B hu hne) hx⟩
  | right y =>
    have hy : y ∈ rhtFinset u := (mem_rhtFinset).2 hp
    have hne : rhtFinset u = ∅ := Finset.ne_empty_of_mem hy
```

```
      exact ⟨hne, B.ent_refl (SumCon.rht_con_of_rht_nonempty A B hu hne) hy⟩
  ent_trans := by
    intro u v c hv hu hEnts hEntc
    refine ⟨hv, ?_⟩
    cases c with
    | bot => exact trivial
    | left x =>
      rcases hEntc with ⟨_, ⟨hne, hA⟩⟩
      have hlft : ∀ y ∈ lftFinset u, A.Ent (lftFinset v) y := by
        intro y hy
        have hp : SumToken.left y ∈ u := (mem_lftFinset).1 hy
        have hEy : SumEnt A B v (.left y) := hEnts _ hp
        exact hEy.2.2
      have hne' : lftFinset v = ∅ := by
        obtain ⟨y, hy⟩ := Finset.nonempty_of_ne_empty hne
        have hp : SumToken.left y ∈ u := (mem_lftFinset).1 hy
        exact (hEnts _ hp).2.1
      refine ⟨hne', ?_⟩
      exact A.ent_trans (SumCon.lft_con_of_lft_nonempty A B hv hne')
        (SumCon.lft_con_of_lft_nonempty A B hu hne) hlft hA
    | right y =>
      rcases hEntc with ⟨_, ⟨hne, hB⟩⟩
      have hrht : ∀ z ∈ rhtFinset u, B.Ent (rhtFinset v) z := by
        intro z hz
        have hp : SumToken.right z ∈ u := (mem_rhtFinset).1 hz
        have hEz : SumEnt A B v (.right z) := hEnts _ hp
        exact hEz.2.2
      have hne' : rhtFinset v = ∅ := by
        obtain ⟨z, hz⟩ := Finset.nonempty_of_ne_empty hne
        have hp : SumToken.right z ∈ u := (mem_rhtFinset).1 hz
        exact (hEnts _ hp).2.1
      refine ⟨hne', ?_⟩
      exact B.ent_trans (SumCon.rht_con_of_rht_nonempty A B hv hne')
        (SumCon.rht_con_of_rht_nonempty A B hu hne) hrht hB

end InfoSys

end DomainTheory.InformationSystems
```

5.7.15 Empty sum consistency

Theorem (`SumCon_empty`). The empty set is consistent in the separated sum.

Proof. Direct from the definitions and the ambient information-system axioms. ?

`SumCon_empty.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumBot
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftFinset
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumCon
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumEnt
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_lft
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_rht

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

theorem SumCon_empty : SumCon A B (∅ : Finset (SumToken α β)) :=
  Or.inl ⟨by rw [lftFinset.empty]; exact A.con_empty, by rw [rhtFinset.empty]⟩

end InfoSys
end DomainTheory.InformationSystems
```

5.7.16 Sum library

Definition (`Sum`). Re-export barrel assembling the product library modules used by later constructions.

`Sum.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumBot
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumCon
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumEnt
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_lft
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_rht
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    SumCon_empty

@[expose] public section
```

5.7.17 Left sum lift

Definition (`InfoSys.instLeftCommutativeLiftSumLeft`). Inserting left sum tokens is left-commutative, so finite left lifts are well-defined independently of insertion order.

Definition (`InfoSys.liftSumLeft`). The left sum lift of a finite set $v \subseteq D_{\mathbf{A}}$ is the finite set of left tokens (X, Δ) for $X \in v$.

Theorem (`InfoSys.liftSumLeft.mem_foldr`). Membership in a left sum lift built by folding insertion over a multiset equals membership of some left token over that multiset.

Proof. Induction on the multiset, using the characterization of insert membership. ?

Theorem (`InfoSys.liftSumLeft.mem`). A sum token lies in the left lift of v iff it is a left token of some $X \in v$.

Proof. Unfold the lift and apply the fold characterization. ?

Theorem (`InfoSys.liftSumLeft.lftFinset_eq`). The left projection of a left sum lift recovers the original set v.

Proof. Use the membership characterizations of left projection and of the lift. ?

Theorem (`InfoSys.liftSumLeft.rhtFinset_eq`). The right projection of a left sum lift is empty.

Proof. Every token in the lift is a left token. ?

Theorem (`InfoSys.liftSumLeft.con`). The left sum lift of a consistent set v is consistent in the separated sum.

Proof. Left projection recovers v; right projection is empty. ?

`liftSumLeft.lean`

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.ElementsOfASystem.botElement

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

instance instLeftCommutativeLiftSumLeft :
    LeftCommutative fun x : α =>
      (insert (SumToken.left (β := β) x) : Finset (SumToken α β) → Finset _) :=
  ⟨fun _ _ s => insert_comm' _ _ s⟩

def liftSumLeft (v : Finset α) : Finset (SumToken α β) :=
  Multiset.foldr (fun x : α => insert (SumToken.left (β := β) x))
    (∅ : Finset (SumToken α β)) v.1

namespace liftSumLeft

theorem mem_foldr (s : Multiset α) (p : SumToken α β) :
    p ∈ Multiset.foldr (fun x : α => insert (SumToken.left (β := β) x))
        (∅ : Finset (SumToken α β)) s ↔
      ∃ x ∈ s, p = .left x := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro hp
      exact False.elim (Finset.notMem_empty p hp)
    · rintro ⟨_, hx, _⟩
      exact False.elim (by cases hx)
  · intro x t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    · rintro (hp | ⟨y, hy, hp⟩)
      · exact ⟨x, Or.inl rfl, hp⟩
      · exact ⟨y, Or.inr hy, hp⟩
    · rintro ⟨y, hy, hp⟩
      rcases hy with rfl | hy
      · exact Or.inl hp
      · exact Or.inr ⟨y, hy, hp⟩

theorem mem {v : Finset α} {p : SumToken α β} :
    p ∈ liftSumLeft (β := β) v ↔ ∃ x ∈ v, p = .left x := by
  simpa [liftSumLeft] using mem_foldr (β := β) v.1 p

theorem lftFinset_eq (v : Finset α) :
    lftFinset (liftSumLeft (β := β) v) = v := by
  ext x
  simp only [mem_lftFinset, mem]
  exact ⟨fun ⟨y, hy, h⟩ => by injection h with h'; exact h' ▸ hy,
    fun hx => ⟨x, hx, rfl⟩⟩

theorem rhtFinset_eq (v : Finset α) :
    rhtFinset (liftSumLeft (β := β) v) = ∅ := by
  ext y
  constructor
  · intro hy
    rcases mem.1 ((mem_rhtFinset).1 hy) with ⟨_, _, h⟩
    exact False.elim (nomatch h)
  · intro hy
    exact False.elim (Finset.notMem_empty y hy)
```

```
theorem con {v : Finset α} (hv : v ∈ A.Con) :
    SumCon A B (liftSumLeft (β := β) v) :=
  Or.inl ⟨by rw [lftFinset_eq]; exact hv, rhtFinset_eq (β := β) v⟩

end liftSumLeft

end InfoSys

end DomainTheory.InformationSystems
```

5.7.18 Right sum lift

Definition (`InfoSys.instLeftCommutativeLiftSumRight`). Inserting right sum tokens is left-commutative, so finite right lifts are well-defined independently of insertion order.

Definition (`InfoSys.liftSumRight`). The right sum lift of a finite set $w \subseteq D_{\mathbf{B}}$ is the finite set of right tokens (Δ, Y) for $Y \in w$.

Theorem (`InfoSys.liftSumRight.mem_foldr`). Membership in a right sum lift built by folding insertion over a multiset equals membership of some right token over that multiset.

Proof. Induction on the multiset, using the characterization of insert membership. ?

Theorem (`InfoSys.liftSumRight.mem`). A sum token lies in the right lift of w iff it is a right token of some $Y \in w$.

Proof. Unfold the lift and apply the fold characterization. ?

Theorem (`InfoSys.liftSumRight.rhtFinset_eq`). The right projection of a right sum lift recovers the original set w.

Proof. Use the membership characterizations of right projection and of the lift. ?

Theorem (`InfoSys.liftSumRight.lftFinset_eq`). The left projection of a right sum lift is empty.

Proof. Every token in the lift is a right token. ?

Theorem (`InfoSys.liftSumRight.con`). The right sum lift of a consistent set w is consistent in the separated sum.

Proof. Left projection is empty; right projection recovers w. ?

`liftSumRight.lean`

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.ElementsOfASystem.botElement

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]
```

```
variable (A : InfoSys α) (B : InfoSys β)

instance instLeftCommutativeLiftSumRight :
    LeftCommutative fun y : β =>
      (insert (SumToken.right (α := α) y) : Finset (SumToken α β) → Finset _) :=
  ⟨fun _ _ s => insert_comm' _ _ s⟩

def liftSumRight (w : Finset β) : Finset (SumToken α β) :=
  Multiset.foldr (fun y : β => insert (SumToken.right (α := α) y))
    (∅ : Finset (SumToken α β)) w.1

namespace liftSumRight

theorem mem_foldr (s : Multiset β) (p : SumToken α β) :
    p ∈ Multiset.foldr (fun y : β => insert (SumToken.right (α := α) y))
        (∅ : Finset (SumToken α β)) s ↔
      ∃ y ∈ s, p = .right y := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro hp
      exact False.elim (Finset.notMem_empty p hp)
    · rintro ⟨_, hy, _⟩
      exact False.elim (by cases hy)
  · intro y t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    · rintro (hp | ⟨z, hz, hp⟩)
      · exact ⟨y, Or.inl rfl, hp⟩
      · exact ⟨z, Or.inr hz, hp⟩
    · rintro ⟨z, hz, hp⟩
      rcases hz with rfl | hz
      · exact Or.inl hp
      · exact Or.inr ⟨z, hz, hp⟩

theorem mem {w : Finset β} {p : SumToken α β} :
    p ∈ liftSumRight (α := α) w ↔ ∃ y ∈ w, p = .right y := by
  simpa [liftSumRight] using mem_foldr (α := α) w.1 p

theorem rhtFinset_eq (w : Finset β) :
    rhtFinset (liftSumRight (α := α) w) = w := by
  ext y
  simp only [mem_rhtFinset, mem]
  exact ⟨fun ⟨z, hz, h⟩ => by injection h with h'; exact h' ▶ hz,
    fun hy => ⟨y, hy, rfl⟩⟩

theorem lftFinset_eq (w : Finset β) :
    lftFinset (liftSumRight (α := α) w) = ∅ := by
  ext x
  constructor
  · intro hx
    rcases mem.1 ((mem_lftFinset).1 hx) with ⟨_, _, h⟩
    exact False.elim (nomatch h)
  · intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem con {w : Finset β} (hw : w ∈ B.Con) :
```

```
    SumCon A B (liftSumRight (α := α) w) :=
  Or.inr ⟨lftFinset_eq (α := α) w, by rw [rhtFinset_eq]; exact hw⟩

end liftSumRight

end InfoSys

end DomainTheory.InformationSystems
```

5.7.19 Sum entailment helpers

Theorem (`InfoSys.entSet.lft_of_sumEntSet`). If u entails u' in the separated sum, then the left projection of u entails the left projection of u' in $\mathbf{A}$.

Proof. Project the sum entailment along left tokens. ?

Theorem (`InfoSys.entSet.rht_of_sumEntSet`). If u entails u' in the separated sum, then the right projection of u entails the right projection of u' in $\mathbf{B}$.

Proof. Project the sum entailment along right tokens. ?

Theorem (`InfoSys.entSet.lft_eq_empty`). If the left projection of u is empty and u entails u' in the separated sum, then the left projection of u' is empty.

Proof. A left token in u' would require the left side of u to be inhabited. ?

Theorem (`InfoSys.entSet.rht_eq_empty`). If the right projection of u is empty and u entails u' in the separated sum, then the right projection of u' is empty.

Proof. A right token in u' would require the right side of u to be inhabited. ?

Theorem (`InfoSys.entSet.liftLft_of_left_only`). If u is consistent with empty right projection, then the left lift of the left projection of u entails u in the separated sum.

Proof. Consistency of the lift follows from recovering the left projection; each left token of u is entailed by reflexivity on that projection, and right tokens cannot occur. ?

Theorem (`InfoSys.entSet.liftRht_of_right_only`). If u is consistent with empty left projection, then the right lift of the right projection of u entails u in the separated sum.

Proof. Dual to the left-only case. ?

Theorem (`InfoSys.entSet.empty_of_rel_bot_like`). Let h be an approximable map from the separated sum that sends the bottom element to bottom. If u is consistent with both projections empty and h relates u to a consistent s, then the empty set entails s.

Proof. Such a u sits under the sum bottom, so the image of its closure under h sits under the codomain bottom; every member of s is therefore entailed from the empty set via the bottom token. ?

`entSet.lean`

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.liftSumLeft
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    liftSumRight
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumElement
public import DomainTheory.InformationSystems.ElementsOfASystem.botElement
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement
```

```
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β)

open ApproximableMap

namespace entSet

theorem lft_of_sumEntSet {u u' : Finset (SumToken α β)}
    (hEnt : (sumSystem A B).EntSet u u') :
    A.EntSet (lftFinset u) (lftFinset u') := fun _ hx =>
  (hEnt _ ((mem_lftFinset).1 hx)).2.2

theorem rht_of_sumEntSet {u u' : Finset (SumToken α β)}
    (hEnt : (sumSystem A B).EntSet u u') :
    B.EntSet (rhtFinset u) (rhtFinset u') := fun _ hy =>
  (hEnt _ ((mem_rhtFinset).1 hy)).2.2

theorem lft_eq_empty {u u' : Finset (SumToken α β)}
    (hl : lftFinset u = ∅) (hEnt : (sumSystem A B).EntSet u u') :
    lftFinset u' = ∅ := by
  ext x
  constructor
  · intro hx
    exact False.elim ((hEnt _ ((mem_lftFinset).1 hx)).2.1 hl)
  · intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem rht_eq_empty {u u' : Finset (SumToken α β)}
    (hr : rhtFinset u = ∅) (hEnt : (sumSystem A B).EntSet u u') :
    rhtFinset u' = ∅ := by
  ext y
  constructor
  · intro hy
    exact False.elim ((hEnt _ ((mem_rhtFinset).1 hy)).2.1 hr)
  · intro hy
    exact False.elim (Finset.notMem_empty y hy)

theorem liftLft_of_left_only {u : Finset (SumToken α β)}
    (hu : SumCon A B u) (hr : rhtFinset u = ∅) :
    (sumSystem A B).EntSet (liftSumLeft (β := β) (lftFinset u)) u := by
  intro p hp
  cases p with
  | bot =>
    exact ⟨liftSumLeft.con A B (SumCon.lft_mem_of_rht_empty A B hu hr), trivial⟩
  | left x =>
    have hx : x ∈ lftFinset u := (mem_lftFinset).2 hp
    refine ⟨liftSumLeft.con A B (SumCon.lft_mem_of_rht_empty A B hu hr), ⟨?_, ?_⟩⟩
```

```
    · rw [liftSumLeft.lftFinset_eq]; exact Finset.ne_empty_of_mem hx
    · rw [liftSumLeft.lftFinset_eq]
      exact A.ent_refl (SumCon.lft_mem_of_rht_empty A B hu hr) hx
  | right y =>
    exact False.elim (by
      have : y ∈ rhtFinset u := (mem_rhtFinset).2 hp
      rw [hr] at this
      exact Finset.notMem_empty _ this)

theorem liftRht_of_right_only {u : Finset (SumToken α β)}
    (hu : SumCon A B u) (hl : lftFinset u = ∅) :
    (sumSystem A B).EntSet (liftSumRight (α := α) (rhtFinset u)) u := by
  intro p hp
  cases p with
  | bot =>
    exact ⟨liftSumRight.con A B (SumCon.rht_mem_of_lft_empty A B hu hl), trivial⟩
  | right y =>
    have hy : y ∈ rhtFinset u := (mem_rhtFinset).2 hp
    refine ⟨liftSumRight.con A B (SumCon.rht_mem_of_lft_empty A B hu hl), ⟨?_, ?_⟩⟩
    · rw [liftSumRight.rhtFinset_eq]; exact Finset.ne_empty_of_mem hy
    · rw [liftSumRight.rhtFinset_eq]
      exact B.ent_refl (SumCon.rht_mem_of_lft_empty A B hu hl) hy
  | left x =>
    exact False.elim (by
      have : x ∈ lftFinset u := (mem_lftFinset).2 hp
      rw [hl] at this
      exact Finset.notMem_empty _ this)

theorem empty_of_rel_bot_like {C : InfoSys ?} (h : ApproximableMap (sumSystem A B) C)
    (hbot : h.toElement (sumSystem A B).botElement = C.botElement)
    {u : Finset (SumToken α β)} {s : Finset ?}
    (hu : SumCon A B u) (hs : s ∈ C.Con) (hl : lftFinset u = ∅) (hr : rhtFinset u = ∅)
    (hh : h.rel u s) : C.EntSet ∅ s := by
  have hsub := subset_botElement_of_projections_empty A B hl hr
  have hcl : (sumSystem A B).closure u hu ≤ (sumSystem A B).botElement := by
    intro p hp
    exact (sumSystem A B).botElement.closed u p hsub hp
  have hle : C.closure s hs ≤ C.botElement := by
    have := (h.rel_iff_closure_le hu hs).1 hh
    have := le_trans this (toElement_mono h hcl)
    rwa [hbot] at this
  intro Y hY
  have hY' : Y ∈ (C.closure s hs).carrier := closure.subset C hs hY
  have hbotY : Y ∈ C.botElement.carrier := hle hY'
  exact C.ent_trans C.con_empty (C.con_sing C.bot)
    (fun _ hz => by
      have : _ = C.bot := Finset.mem_singleton.mp hz
      subst this; exact C.ent_bot C.con_empty)
    hbotY

end entSet

end InfoSys

end DomainTheory.InformationSystems
```

5.7.20 Sum elements

Definition (`ApproximableMap.sumLftCarrier`). The left carrier of a sum element z is the set of tokens X of $\mathbf{A}$ such that (X, Δ) lies in z.

Definition (`ApproximableMap.sumRhtCarrier`). The right carrier of a sum element z is the set of tokens Y of $\mathbf{B}$ such that (Δ, Y) lies in z.

Theorem (`ApproximableMap.not_mem_right_of_mem_left`). A sum element cannot contain both a left token and a right token.

Proof. The two-point set of those tokens would be consistent in z, contradicting separated-sum consistency. ?

Theorem (`ApproximableMap.eq_botElement_of_no_injections`). If a sum element contains neither left nor right tokens, it is the bottom element.

Proof. Only the sum bottom token can remain, and bottom is least. ?

Theorem (`ApproximableMap.subset_botElement_of_projections_empty`). A finite sum set with empty left and right projections sits inside the bottom element.

Proof. Every member must be the sum bottom token. ?

Definition (`ApproximableMap.sumElementLft`). Given a sum element z containing a left token (X_0, Δ), the left element of z is the $\mathbf{A}$-element whose carrier is the left carrier of z.

Definition (`ApproximableMap.sumElementRht`). Given a sum element z containing a right token (Δ, Y_0), the right element of z is the $\mathbf{B}$-element whose carrier is the right carrier of z.

`sumElement.lean`

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.liftSumLeft
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    liftSumRight
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ApproximableMap

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

namespace ApproximableMap

def sumLftCarrier (z : (sumSystem A B).Element) : Set α :=
  {x | SumToken.left x ∈ z.carrier}

def sumRhtCarrier (z : (sumSystem A B).Element) : Set β :=
  {y | SumToken.right y ∈ z.carrier}

theorem not_mem_right_of_mem_left (z : (sumSystem A B).Element)
```

```
    {x : α} {y : β} (hx : SumToken.left x ∈ z.carrier)
    (hy : SumToken.right y ∈ z.carrier) : False := by
  let u : Finset (SumToken α β) := insert (.left x) {SumToken.right y}
  have hsub : (↑u : Set _) ⊆ z.carrier := by
    intro p hp
    rcases Finset.mem_insert.mp (Finset.mem_coe.1 hp) with h | h
    · subst h; exact hx
    · have : p = .right y := Finset.mem_singleton.mp h
      subst this; exact hy
  have hCon := z.consistent u hsub
  have hlne : lftFinset u = ∅ := by
    intro h
    have : x ∈ lftFinset u := (mem_lftFinset).2 (Finset.mem_insert_self _ _)
    rw [h] at this
    exact Finset.notMem_empty _ this
  have hrne : rhtFinset u = ∅ := by
    intro h
    have : y ∈ rhtFinset u :=
      (mem_rhtFinset).2 (Finset.mem_insert_of_mem (Finset.mem_singleton_self _))
    rw [h] at this
    exact Finset.notMem_empty _ this
  rcases hCon with ⟨_, hr⟩ | ⟨hl, _⟩
  · exact hrne hr
  · exact hlne hl

theorem eq_botElement_of_no_injections (z : (sumSystem A B).Element)
    (hL : ∀ x : α, SumToken.left x ∈ z.carrier)
    (hR : ∀ y : β, SumToken.right y ∈ z.carrier) :
    z = (sumSystem A B).botElement := by
  apply le_antisymm
  · intro p hp
    cases p with
    | bot =>
      change SumToken.bot ∈ (sumSystem A B).botElement.carrier
      exact (sumSystem A B).ent_refl ((sumSystem A B).con_sing _) (Finset.
    mem_singleton_self _)
    | left x => exact False.elim (hL x hp)
    | right y => exact False.elim (hR y hp)
  · exact botElement.le (sumSystem A B) z

theorem subset_botElement_of_projections_empty {u : Finset (SumToken α β)}
    (hl : lftFinset u = ∅) (hr : rhtFinset u = ∅) :
    (↑u : Set (SumToken α β)) ⊆ (sumSystem A B).botElement.carrier := by
  intro p hp
  have hp' : p ∈ u := Finset.mem_coe.1 hp
  cases p with
  | bot =>
    exact (sumSystem A B).ent_refl ((sumSystem A B).con_sing _) (Finset.
    mem_singleton_self _)
  | left x =>
    exact False.elim (by
      have : x ∈ lftFinset u := (mem_lftFinset).2 hp'
      rw [hl] at this; exact Finset.notMem_empty _ this)
  | right y =>
    exact False.elim (by
      have : y ∈ rhtFinset u := (mem_rhtFinset).2 hp'
```

```
      rw [hr] at this; exact Finset.notMem_empty _ this)

def sumElementLft (z : (sumSystem A B).Element)
    (x0 : α) (hx0 : SumToken.left x0 ∈ z.carrier) : A.Element where
  carrier := sumLftCarrier A B z
  consistent := by
    intro Y hY
    have hsub : (↑(liftSumLeft (β := β) Y) : Set _) ⊆ z.carrier := by
      intro p hp
      rcases (liftSumLeft.mem (β := β)).1 (Finset.mem_coe.1 hp) with ⟨x, hx, rfl⟩
      exact hY (Finset.mem_coe.2 hx)
    have h := SumCon.lft_mem_of_rht_empty A B (z.consistent _ hsub)
      (liftSumLeft.rhtFinset_eq (β := β) Y)
    rwa [liftSumLeft.lftFinset_eq] at h
  closed := by
    intro Y a hY hEnt
    if hempty : Y = ∅ then
      subst hempty
      have hEnt' : A.Ent {x0} a :=
        A.ent_trans (A.con_sing x0) A.con_empty
          (fun _ h => False.elim (Finset.notMem_empty _ h)) hEnt
      have hsub : (↑({SumToken.left x0} : Finset (SumToken α β)) : Set _) ⊆ z.carrier
    := by
        intro p hp
        have : p = .left x0 := Finset.mem_singleton.mp (Finset.mem_coe.1 hp)
        subst this; exact hx0
      have hSum : (sumSystem A B).Ent {SumToken.left x0} (.left a) := by
        refine ⟨z.consistent _ hsub, ⟨Finset.singleton_ne_empty _, ?_⟩⟩
        rw [lftFinset.singleton_left]; exact hEnt'
      exact z.closed _ _ hsub hSum
    else
      have hsub : (↑(liftSumLeft (β := β) Y) : Set _) ⊆ z.carrier := by
        intro p hp
        rcases (liftSumLeft.mem (β := β)).1 (Finset.mem_coe.1 hp) with ⟨x, hx, rfl⟩
        exact hY (Finset.mem_coe.2 hx)
      have hSum : (sumSystem A B).Ent (liftSumLeft (β := β) Y) (.left a) := by
        refine ⟨z.consistent _ hsub, ⟨?_, ?_⟩⟩
        · rw [liftSumLeft.lftFinset_eq]; exact hempty
        · rw [liftSumLeft.lftFinset_eq]; exact hEnt
      exact z.closed _ _ hsub hSum

def sumElementRht (z : (sumSystem A B).Element)
    (y0 : β) (hy0 : SumToken.right y0 ∈ z.carrier) : B.Element where
  carrier := sumRhtCarrier A B z
  consistent := by
    intro W hW
    have hsub : (↑(liftSumRight (α := α) W) : Set _) ⊆ z.carrier := by
      intro p hp
      rcases (liftSumRight.mem (α := α)).1 (Finset.mem_coe.1 hp) with ⟨y, hy, rfl⟩
      exact hW (Finset.mem_coe.2 hy)
    have h := SumCon.rht_mem_of_lft_empty A B (z.consistent _ hsub)
      (liftSumRight.lftFinset_eq (α := α) W)
    rwa [liftSumRight.rhtFinset_eq] at h
  closed := by
    intro W b hW hEnt
    if hempty : W = ∅ then
```

```
      subst hempty
      have hEnt' : B.Ent {y0} b :=
        B.ent_trans (B.con_sing y0) B.con_empty
          (fun _ h => False.elim (Finset.notMem_empty _ h)) hEnt
      have hsub : (↑({SumToken.right y0} : Finset (SumToken α β)) : Set _) ⊆ z.carrier
    := by
        intro p hp
        have : p = .right y0 := Finset.mem_singleton.mp (Finset.mem_coe.1 hp)
        subst this; exact hy0
      have hSum : (sumSystem A B).Ent {SumToken.right y0} (.right b) := by
        refine ⟨z.consistent _ hsub, ⟨Finset.singleton_ne_empty _, ?_⟩⟩
        rw [rhtFinset.singleton_right]; exact hEnt'
      exact z.closed _ _ hsub hSum
    else
      have hsub : (↑(liftSumRight (α := α) W) : Set _) ⊆ z.carrier := by
        intro p hp
        rcases (liftSumRight.mem (α := α)).1 (Finset.mem_coe.1 hp) with ⟨y, hy, rfl⟩
        exact hW (Finset.mem_coe.2 hy)
      have hSum : (sumSystem A B).Ent (liftSumRight (α := α) W) (.right b) := by
        refine ⟨z.consistent _ hsub, ⟨?_, ?_⟩⟩
        · rw [liftSumRight.rhtFinset_eq]; exact hempty
        · rw [liftSumRight.rhtFinset_eq]; exact hEnt
      exact z.closed _ _ hsub hSum

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

5.7.21 Left injection map

Definition (`ApproximableMap.inlMap`). The left injection $\mathrm{inl} : \mathbf{A} \to \mathbf{A} + \mathbf{B}$ relates a consistent v to a sum set u when u is consistent with empty right projection and v entails the left projection of u.

Theorem (`ApproximableMap.inlMap.toElement_sumElementLft`). Applying the left injection's element map to the left element of a sum element z that contains a left token recovers z.

Proof. One inclusion uses left closure under the related entailment and excludes right tokens by the empty right projection; the other rebuilds relatedness from left carriers and excludes right tokens by injection separation. ?

`inlMap.lean`

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.entSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumElement
public import DomainTheory.InformationSystems.ElementsOfASystem.botElement
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β)

namespace ApproximableMap

variable {C : InfoSys ?}

def inlMap : ApproximableMap A (sumSystem A B) where
  rel v u := v ∈ A.Con ∧ SumCon A B u ∧ rhtFinset u = ∅ ∧ A.EntSet v (lftFinset u)
  rel_dom h := h.1
  rel_cod h := h.2.1
  empty_rel := ⟨A.con_empty, SumCon_empty A B, by rw [rhtFinset.empty], EntSet.empty A
    ∅⟩
  union_right := by
    rintro v u u' ⟨hv, hu, hr, hEnt⟩ ⟨_, hu', hr', hEnt'⟩
    refine ⟨hv, ?_, ?_, ?_⟩
    · have hl : lftFinset (u ∪' u') ∈ A.Con := by
        rw [lftFinset.funion]
        exact A.con_subset
          (EntSet.union_mem (sys := A) hv (EntSet.union (sys := A) hEnt hEnt'))
          (funion.subset_right _ _)
      exact Or.inl ⟨hl, by rw [rhtFinset.funion, hr, hr']; rfl⟩
    · rw [rhtFinset.funion, hr, hr']; rfl
    · rw [lftFinset.funion]; exact EntSet.union (sys := A) hEnt hEnt'
  mono := by
    rintro v v' u u' ⟨hv, hu, hr, hEnt⟩ hEntv hEntu hv' hu'
    refine ⟨hv', hu', entSet.rht_eq_empty A B hr hEntu, ?_⟩
    exact EntSet.trans (sys := A) hv'
      (SumCon.lft_mem_of_rht_empty A B hu hr)
      (EntSet.trans (sys := A) hv' hv hEntv hEnt)
      (entSet.lft_of_sumEntSet A B hEntu)

theorem inlMap.toElement_sumElementLft (z : (sumSystem A B).Element)
    (x0 : α) (hx0 : SumToken.left x0 ∈ z.carrier) :
    (inlMap A B).toElement (sumElementLft A B z x0 hx0) = z := by
  apply le_antisymm
  · intro p ⟨v, hv, ⟨hvCon, huCon, hr, hEnt⟩⟩
    cases p with
    | bot => exact factoid_3_2 (sumSystem A B) z
    | left x =>
      have hEntx : A.Ent v x := by
        have : lftFinset ({SumToken.left x} : Finset (SumToken α β)) = {x} :=
          lftFinset.singleton_left x
        simpa [this] using hEnt x (Finset.mem_singleton_self x)
      exact (sumElementLft A B z x0 hx0).closed v x hv hEntx
    | right y =>
      have : rhtFinset ({SumToken.right y} : Finset (SumToken α β)) = {y} :=
        rhtFinset.singleton_right y
      exact False.elim (Finset.singleton_ne_empty y (this ▸ hr))
  · intro p hp
    cases p with
```

```
    | bot =>
      refine ⟨∅, fun _ h => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 h)), ?_
    ⟩
      exact ⟨A.con_empty, (sumSystem A B).con_sing _, by rw [rhtFinset.singleton_bot],
        EntSet.empty A _⟩
    | left x =>
      refine ⟨{x}, ?_, ?_⟩
      · intro a ha
        have : a = x := Finset.mem_singleton.mp (Finset.mem_coe.1 ha)
        subst this; exact hp
      · refine ⟨A.con_sing x, (sumSystem A B).con_sing _, ?_, ?_⟩
        · ext y; constructor
          · intro hy
            have : SumToken.right y ∈ ({SumToken.left x} : Finset _) := (mem_rhtFinset
    ).1 hy
            exact False.elim (nomatch Finset.mem_singleton.mp this)
          · intro hy; exact False.elim (Finset.notMem_empty y hy)
        · rw [lftFinset.singleton_left]
          exact EntSet.refl A (A.con_sing x)
    | right y =>
      exact False.elim (not_mem_right_of_mem_left A B z hx0 hp)

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

5.7.22 Right injection map

Definition (`ApproximableMap.inrMap`). The right injection $\mathrm{inr} : \mathbf{B} \to \mathbf{A} + \mathbf{B}$ relates a consistent w to a sum set u when u is consistent with empty left projection and w entails the right projection of u.

Theorem (`ApproximableMap.inrMap.toElement_sumElementRht`). Applying the right injection's element map to the right element of a sum element z that contains a right token recovers z.

Proof. Dual to the left injection case: right closure under the related entailment, empty left projection excluding left tokens, and injection separation on the converse. ?

`inrMap.lean`

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.entSet
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumElement
public import DomainTheory.InformationSystems.ElementsOfASystem.botElement
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β)

namespace ApproximableMap

variable {C : InfoSys ?}

def inrMap : ApproximableMap B (sumSystem A B) where
  rel w u := w ∈ B.Con ∧ SumCon A B u ∧ lftFinset u = ∅ ∧ B.EntSet w (rhtFinset u)
  rel_dom h := h.1
  rel_cod h := h.2.1
  empty_rel := ⟨B.con_empty, SumCon_empty A B, by rw [lftFinset.empty], EntSet.empty B
    ∅⟩
  union_right := by
    rintro w u u' ⟨hw, hu, hl, hEnt⟩ ⟨_, hu', hl', hEnt'⟩
    refine ⟨hw, ?_, ?_, ?_⟩
    · have hrC : rhtFinset (u ∪' u') ∈ B.Con := by
        rw [rhtFinset.funion]
        exact B.con_subset
          (EntSet.union_mem (sys := B) hw (EntSet.union (sys := B) hEnt hEnt'))
          (funion.subset_right _ _)
      exact Or.inr ⟨by rw [lftFinset.funion, hl, hl']; rfl, hrC⟩
    · rw [lftFinset.funion, hl, hl']; rfl
    · rw [rhtFinset.funion]; exact EntSet.union (sys := B) hEnt hEnt'
  mono := by
    rintro w w' u u' ⟨hw, hu, hl, hEnt⟩ hEntw hEntu hw' hu'
    refine ⟨hw', hu', entSet.lft_eq_empty A B hl hEntu, ?_⟩
    exact EntSet.trans (sys := B) hw'
      (SumCon.rht_mem_of_lft_empty A B hu hl)
      (EntSet.trans (sys := B) hw' hw hEntw hEnt)
      (entSet.rht_of_sumEntSet A B hEntu)

theorem inrMap.toElement_sumElementRht (z : (sumSystem A B).Element)
    (y0 : β) (hy0 : SumToken.right y0 ∈ z.carrier) :
    (inrMap A B).toElement (sumElementRht A B z y0 hy0) = z := by
  apply le_antisymm
  · intro p ⟨w, hw, ⟨hwCon, huCon, hl, hEnt⟩⟩
    cases p with
    | bot => exact factoid_3_2 (sumSystem A B) z
    | right y =>
      have hEnty : B.Ent w y := by
        have : rhtFinset ({SumToken.right y} : Finset (SumToken α β)) = {y} :=
          rhtFinset.singleton_right y
        simpa [this] using hEnt y (Finset.mem_singleton_self y)
      exact (sumElementRht A B z y0 hy0).closed w y hw hEnty
    | left x =>
      have : lftFinset ({SumToken.left x} : Finset (SumToken α β)) = {x} :=
        lftFinset.singleton_left x
      exact False.elim (Finset.singleton_ne_empty x (this ▸ hl))
  · intro p hp
    cases p with
    | bot =>
      refine ⟨∅, fun _ h => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 h)), ?_
    ⟩
      exact ⟨B.con_empty, (sumSystem A B).con_sing _, by rw [lftFinset.singleton_bot],
```

```
        EntSet.empty B _⟩
    | right y =>
      refine ⟨{y}, ?_, ?_⟩
      · intro b hb
        have : b = y := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
        subst this; exact hp
      · refine ⟨B.con_sing y, (sumSystem A B).con_sing _, ?_, ?_⟩
        · ext x; constructor
          · intro hx
            have : SumToken.left x ∈ ({SumToken.right y} : Finset _) := (mem_lftFinset
  ).1 hx
            exact False.elim (nomatch Finset.mem_singleton.mp this)
          · intro hx; exact False.elim (Finset.notMem_empty x hx)
        · rw [rhtFinset.singleton_right]
          exact EntSet.refl B (B.con_sing y)
    | left x =>
      exact False.elim (not_mem_right_of_mem_left A B z hp hy0)

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

5.7.23 Copairing map

Theorem (`InfoSys.entSet.u_of_inl_witness`). If u is consistent with nonempty left projection and empty right projection, v is left-injection-related to u', and the left projection of u entails v, then u entails u' in the separated sum.

Proof. Transfer left-factor entailment along the injection relatedness; right tokens in u' are impossible by the empty right projection of u'. ?

Theorem (`InfoSys.entSet.u_of_inr_witness`). Dual to the left witness: a right-only consistent u whose right projection entails a right-injection witness of u' entails u' in the separated sum.

Proof. Dual to the left case. ?

Theorem (`ApproximableMap.copairMap.union_disj`). The three-way disjunct that defines copair relatedness is closed under finite union of consequents: empty-entailment cases combine, same-side relatedness unions, and opposite inhabited sides are impossible.

Proof. Case analysis on the two disjuncts, using multiary union, monotonicity of f and g from the empty relation, and separated-sum side exclusion. ?

Definition (`ApproximableMap.copairMap`). For approximable mappings $f : \mathbf{A} \to \mathbf{C}$ and $g : \mathbf{B} \to \mathbf{C}$, the copairing $[f, g] : \mathbf{A} + \mathbf{B} \to \mathbf{C}$ relates a consistent sum set u to a consistent s when either the empty set entails s, or the left projection of u is inhabited and f-related to s, or the right projection of u is inhabited and g-related to s.

Theorem (`ApproximableMap.copairMap.botElement`). The element map of a copairing sends the sum bottom element to the codomain bottom element.

Proof. Related witnesses under the sum bottom can only involve the sum bottom token, so both projections are empty and the empty-entailment clause forces membership under the codomain bottom; the converse uses the empty domain set. ?

Theorem (`ApproximableMap.copairMap.comp_inl`). Composing the left injection after the copairing recovers f.

Proof. Forward, discard empty and right clauses by the empty right projection of an injection witness and apply monotonicity; conversely lift an f-related set through a left sum lift of the set with bottom adjoined. ?

Theorem (`ApproximableMap.copairMap.comp_inr`). Composing the right injection after the copairing recovers g.

Proof. Dual to the left injection case, using right sum lifts. ?

Theorem (`ApproximableMap.copairMap.unique`). An approximable mapping h from the separated sum that composes with the injections to f and g and sends bottom to bottom equals the copairing $[f, g]$.

Proof. On relatedness, classify by which projections of u are inhabited: both empty uses the bottom-like empty entailment lemma; a single inhabited side recovers f or g via the corresponding lift and composition hypothesis, then transfers back with the injection witnesses. ?

`copairMap.lean`

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inlMap
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inrMap
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.comp
import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.entSet
import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumCon_empty
import DomainTheory.InformationSystems.InformationSystems.EntSet
import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β)

open ApproximableMap

namespace entSet

theorem u_of_inl_witness {u u' : Finset (SumToken α β)} {v : Finset α}
    (hlne : lftFinset u = ∅) (hinl : (inlMap A B).rel v u')
    (huv : A.EntSet (lftFinset u) v) (hu : SumCon A B u)
    (hr : rhtFinset u = ∅) :
    (sumSystem A B).EntSet u u' := by
  intro p hp
  have ⟨hv, hu', hr', hEnt⟩ := hinl
  cases p with
  | bot => exact ⟨hu, trivial⟩
  | left x =>
    have hx : x ∈ lftFinset u' := (mem_lftFinset).2 hp
    have hAx : A.Ent v x := hEnt x hx
```

```
    have hAx' : A.Ent (lftFinset u) x :=
      A.ent_trans (SumCon.lft_mem_of_rht_empty A B hu hr) hv huv hAx
    exact ⟨hu, ⟨hlne, hAx'⟩⟩
  | right y =>
    exact False.elim (by
      have : y ∈ rhtFinset u' := (mem_rhtFinset).2 hp
      rw [hr'] at this; exact Finset.notMem_empty _ this)

theorem u_of_inr_witness {u u' : Finset (SumToken α β)} {w : Finset β}
    (hrne : rhtFinset u = ∅) (hinr : (inrMap A B).rel w u')
    (huw : B.EntSet (rhtFinset u) w) (hu : SumCon A B u)
    (hl : lftFinset u = ∅) :
    (sumSystem A B).EntSet u u' := by
  intro p hp
  have ⟨hw, hu', hl', hEnt⟩ := hinr
  cases p with
  | bot => exact ⟨hu, trivial⟩
  | right y =>
    have hy : y ∈ rhtFinset u' := (mem_rhtFinset).2 hp
    have hBy : B.Ent w y := hEnt y hy
    have hBy' : B.Ent (rhtFinset u) y :=
      B.ent_trans (SumCon.rht_mem_of_lft_empty A B hu hl) hw huw hBy
    exact ⟨hu, ⟨hrne, hBy'⟩⟩
  | left x =>
    exact False.elim (by
      have : x ∈ lftFinset u' := (mem_lftFinset).2 hp
      rw [hl'] at this; exact Finset.notMem_empty _ this)

end entSet

namespace ApproximableMap

variable {C : InfoSys ?}

theorem copairMap.union_disj (f : ApproximableMap A C) (g : ApproximableMap B C)
    {u : Finset (SumToken α β)} {s s' : Finset ?}
    (hu : SumCon A B u) (hs : s ∈ C.Con) (hs' : s' ∈ C.Con)
    (h : C.EntSet ∅ s ∨ (lftFinset u = ∅ ∧ f.rel (lftFinset u) s) ∨
      (rhtFinset u = ∅ ∧ g.rel (rhtFinset u) s))
    (h' : C.EntSet ∅ s' ∨ (lftFinset u = ∅ ∧ f.rel (lftFinset u) s') ∨
      (rhtFinset u = ∅ ∧ g.rel (rhtFinset u) s')) :
    C.EntSet ∅ (s ∪' s') ∨
      (lftFinset u = ∅ ∧ f.rel (lftFinset u) (s ∪' s')) ∨
        (rhtFinset u = ∅ ∧ g.rel (rhtFinset u) (s ∪' s')) := by
  rcases h with h | ⟨hne, hf⟩ | ⟨hne, hg⟩ <;> rcases h' with h' | ⟨hne', hf'⟩ | ⟨hne', hg
    '⟩
  · exact Or.inl (EntSet.union (sys := C) h h')
  · have hf0 : f.rel (lftFinset u) s :=
      f.mono f.empty_rel (EntSet.empty A _) h
        (SumCon.lft_con_of_lft_nonempty A B hu hne') hs
    exact Or.inr (Or.inl ⟨hne', f.union_right hf0 hf'⟩)
  · have hg0 : g.rel (rhtFinset u) s :=
      g.mono g.empty_rel (EntSet.empty B _) h
        (SumCon.rht_con_of_rht_nonempty A B hu hne') hs
    exact Or.inr (Or.inr ⟨hne', g.union_right hg0 hg'⟩)
  · have hf0 : f.rel (lftFinset u) s' :=
```

```
      f.mono f.empty_rel (EntSet.empty A _) h'
        (SumCon.lft_con_of_lft_nonempty A B hu hne) hs'
    exact Or.inr (Or.inl ⟨hne, f.union_right hf hf0⟩)
  · exact Or.inr (Or.inl ⟨hne, f.union_right hf hf'⟩)
  · exact False.elim (hne' (SumCon.rht_empty_of_lft_nonempty A B hu hne))
  · have hg0 : g.rel (rhtFinset u) s' :=
      g.mono g.empty_rel (EntSet.empty B _) h'
        (SumCon.rht_con_of_rht_nonempty A B hu hne) hs'
    exact Or.inr (Or.inr ⟨hne, g.union_right hg hg0⟩)
  · exact False.elim (hne' (SumCon.lft_empty_of_rht_nonempty A B hu hne))
  · exact Or.inr (Or.inr ⟨hne, g.union_right hg hg'⟩)

def copairMap (f : ApproximableMap A C) (g : ApproximableMap B C) :
    ApproximableMap (sumSystem A B) C where
  rel u s :=
    SumCon A B u ∧ s ∈ C.Con ∧
      (C.EntSet ∅ s ∨
        (lftFinset u = ∅ ∧ f.rel (lftFinset u) s) ∨
          (rhtFinset u = ∅ ∧ g.rel (rhtFinset u) s))
  rel_dom h := h.1
  rel_cod h := h.2.1
  empty_rel := ⟨SumCon_empty A B, C.con_empty, Or.inl (EntSet.empty C ∅)⟩
  union_right := by
    rintro u s s' ⟨hu, hs, h⟩ ⟨_, hs', h'⟩
    have hU := copairMap.union_disj A B f g hu hs hs' h h'
    refine ⟨hu, ?_, hU⟩
    rcases hU with hE | ⟨_, hf⟩ | ⟨_, hg⟩
    · exact C.con_subset (EntSet.union_mem (sys := C) C.con_empty hE)
        (funion.subset_right _ _)
    · exact f.rel_cod hf
    · exact g.rel_cod hg
  mono := by
    rintro u u' s s' ⟨hu, hs, h⟩ hEntu hEnts hu' hs'
    refine ⟨hu', hs', ?_⟩
    rcases h with h | ⟨hne, hf⟩ | ⟨hne, hg⟩
    · exact Or.inl (EntSet.trans (sys := C) C.con_empty hs h hEnts)
    · have hne' : lftFinset u' = ∅ := by
        obtain ⟨x, hx⟩ := Finset.nonempty_of_ne_empty hne
        exact (hEntu _ ((mem_lftFinset).1 hx)).2.1
      exact Or.inr (Or.inl ⟨hne',
        f.mono hf (entSet.lft_of_sumEntSet A B hEntu) hEnts
          (SumCon.lft_con_of_lft_nonempty A B hu' hne') hs'⟩)
    · have hne' : rhtFinset u' = ∅ := by
        obtain ⟨y, hy⟩ := Finset.nonempty_of_ne_empty hne
        exact (hEntu _ ((mem_rhtFinset).1 hy)).2.1
      exact Or.inr (Or.inr ⟨hne',
        g.mono hg (entSet.rht_of_sumEntSet A B hEntu) hEnts
          (SumCon.rht_con_of_rht_nonempty A B hu' hne') hs'⟩)

namespace copairMap

theorem botElement (f : ApproximableMap A C) (g : ApproximableMap B C) :
    (copairMap A B f g).toElement (sumSystem A B).botElement = C.botElement := by
  apply le_antisymm
  · intro Y ⟨u, hu, ⟨huCon, hs, hcop⟩⟩
    have onlyBot : ∀ p ∈ u, p = SumToken.bot (α := α) (β := β) := by
```

```
      intro p hp
      have hEnt : (sumSystem A B).Ent {sumBot} p := hu (Finset.mem_coe.2 hp)
      cases p with
      | bot => rfl
      | left x =>
        exact False.elim (by
          have hne := hEnt.2.1
          have : lftFinset ({sumBot} : Finset (SumToken α β)) = ∅ := lftFinset.
    singleton_bot
          exact hne this)
      | right y =>
        exact False.elim (by
          have hne := hEnt.2.1
          have : rhtFinset ({sumBot} : Finset (SumToken α β)) = ∅ := rhtFinset.
    singleton_bot
          exact hne this)
    have hl : lftFinset u = ∅ := by
      ext x; constructor
      · intro hx; exact False.elim (nomatch onlyBot _ ((mem_lftFinset).1 hx))
      · intro hx; exact False.elim (Finset.notMem_empty x hx)
    have hr : rhtFinset u = ∅ := by
      ext y; constructor
      · intro hy; exact False.elim (nomatch onlyBot _ ((mem_rhtFinset).1 hy))
      · intro hy; exact False.elim (Finset.notMem_empty y hy)
    rcases hcop with hE | ⟨hne, _⟩ | ⟨hne, _⟩
    · exact C.ent_trans (C.con_sing C.bot) C.con_empty
        (fun _ hz => False.elim (Finset.notMem_empty _ hz))
        (hE Y (Finset.mem_singleton_self _))
    · exact False.elim (hne hl)
    · exact False.elim (hne hr)
  · intro Y hY
    refine ⟨∅, fun _ hp => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 hp)), ?_
    ⟩
    refine ⟨SumCon_empty A B, C.con_sing Y, Or.inl ?_⟩
    intro Z hZ
    have : Z = Y := Finset.mem_singleton.mp hZ
    subst this
    exact C.ent_trans C.con_empty (C.con_sing C.bot)
      (fun _ hz => by
        have : _ = C.bot := Finset.mem_singleton.mp hz
        subst this
        exact C.ent_bot C.con_empty)
      hY

theorem comp_inl (f : ApproximableMap A C) (g : ApproximableMap B C) :
    comp (copairMap A B f g) (inlMap A B) = f := by
  refine ApproximableMap.ext fun v s => ?_
  constructor
  · rintro ⟨u, ⟨hv, hu, hr, hEnt⟩, ⟨_, hs, hcop⟩⟩
    rcases hcop with hE | ⟨_, hf⟩ | ⟨hne, _⟩
    · exact f.mono f.empty_rel (EntSet.empty A v) hE hv hs
    · exact f.mono hf hEnt (EntSet.refl C hs) hv hs
    · exact False.elim (hne hr)
  · intro hf
    let vBot := insert A.bot v
    have hvBot : vBot ∈ A.Con := A.ent_con (A.ent_bot (f.rel_dom hf))
```

```
    have hEntBot : A.EntSet vBot v :=
      fun _ hx => A.ent_refl hvBot (Finset.mem_insert_of_mem hx)
    have hfBot : f.rel vBot s :=
      f.mono hf hEntBot (EntSet.refl C (f.rel_cod hf)) hvBot (f.rel_cod hf)
    refine ⟨liftSumLeft (β := β) vBot, ?_, ?_⟩
    · refine ⟨f.rel_dom hf, liftSumLeft.con A B hvBot,
        liftSumLeft.rhtFinset_eq (β := β) vBot, ?_⟩
      rw [liftSumLeft.lftFinset_eq]
      intro x hx
      rcases Finset.mem_insert.mp hx with hx | hx
      · subst hx; exact A.ent_bot (f.rel_dom hf)
      · exact A.ent_refl (f.rel_dom hf) hx
    · refine ⟨liftSumLeft.con A B hvBot, f.rel_cod hf, Or.inr (Or.inl ⟨?_, ?_⟩)⟩
      · rw [liftSumLeft.lftFinset_eq]; exact Finset.insert_ne_empty A.bot v
      · rw [liftSumLeft.lftFinset_eq]; exact hfBot

theorem comp_inr (f : ApproximableMap A C) (g : ApproximableMap B C) :
    comp (copairMap A B f g) (inrMap A B) = g := by
  refine ApproximableMap.ext fun w s => ?_
  constructor
  · rintro ⟨u, ⟨hw, hu, hl, hEnt⟩, ⟨_, hs, hcop⟩⟩
    rcases hcop with hE | ⟨hne, _⟩ | ⟨_, hg⟩
    · exact g.mono g.empty_rel (EntSet.empty B w) hE hw hs
    · exact False.elim (hne hl)
    · exact g.mono hg hEnt (EntSet.refl C hs) hw hs
  · intro hg
    let wBot := insert B.bot w
    have hwBot : wBot ∈ B.Con := B.ent_con (B.ent_bot (g.rel_dom hg))
    have hEntBot : B.EntSet wBot w :=
      fun _ hy => B.ent_refl hwBot (Finset.mem_insert_of_mem hy)
    have hgBot : g.rel wBot s :=
      g.mono hg hEntBot (EntSet.refl C (g.rel_cod hg)) hwBot (g.rel_cod hg)
    refine ⟨liftSumRight (α := α) wBot, ?_, ?_⟩
    · refine ⟨g.rel_dom hg, liftSumRight.con A B hwBot,
        liftSumRight.lftFinset_eq (α := α) wBot, ?_⟩
      rw [liftSumRight.rhtFinset_eq]
      intro y hy
      rcases Finset.mem_insert.mp hy with hy | hy
      · subst hy; exact B.ent_bot (g.rel_dom hg)
      · exact B.ent_refl (g.rel_dom hg) hy
    · refine ⟨liftSumRight.con A B hwBot, g.rel_cod hg, Or.inr (Or.inr ⟨?_, ?_⟩)⟩
      · rw [liftSumRight.rhtFinset_eq]; exact Finset.insert_ne_empty B.bot w
      · rw [liftSumRight.rhtFinset_eq]; exact hgBot

theorem unique (f : ApproximableMap A C) (g : ApproximableMap B C)
    (h : ApproximableMap (sumSystem A B) C)
    (hinl : comp h (inlMap A B) = f) (hinr : comp h (inrMap A B) = g)
    (hbot : h.toElement (sumSystem A B).botElement = C.botElement) :
    h = copairMap A B f g := by
  refine ApproximableMap.ext fun u s => ?_
  constructor
  · intro hh
    have hu : SumCon A B u := h.rel_dom hh
    have hs : s ∈ C.Con := h.rel_cod hh
    refine ⟨hu, hs, ?_⟩
    rcases lftFinset.eq_empty_or_mem u with hl | ⟨x0, hx0⟩
```

```
    · rcases rhtFinset.eq_empty_or_mem u with hr | ⟨y0, hy0⟩
      · exact Or.inl (entSet.empty_of_rel_bot_like A B h hbot hu hs hl hr hh)
      · have hl' : lftFinset u = ∅ := hl
        have hg : g.rel (rhtFinset u) s := by
          have hinr' : (comp h (inrMap A B)).rel (rhtFinset u) s := by
            refine ⟨liftSumRight (α := α) (rhtFinset u), ?_, ?_⟩
            · exact ⟨SumCon.rht_mem_of_lft_empty A B hu hl',
                liftSumRight.con A B (SumCon.rht_mem_of_lft_empty A B hu hl'),
                liftSumRight.lftFinset_eq (α := α) _, by
                  rw [liftSumRight.rhtFinset_eq]
                  exact EntSet.refl B
                    (SumCon.rht_mem_of_lft_empty A B hu hl')⟩
            · exact h.mono hh (entSet.liftRht_of_right_only A B hu hl')
                (EntSet.refl C hs)
                (liftSumRight.con A B (SumCon.rht_mem_of_lft_empty A B hu hl')) hs
          simpa [hinr] using hinr'
        exact Or.inr (Or.inr ⟨Finset.ne_empty_of_mem hy0, hg⟩)
    · have hne : lftFinset u = ∅ := Finset.ne_empty_of_mem hx0
      have hr : rhtFinset u = ∅ := SumCon.rht_empty_of_lft_nonempty A B hu hne
      have hf : f.rel (lftFinset u) s := by
        have hinl' : (comp h (inlMap A B)).rel (lftFinset u) s := by
          refine ⟨liftSumLeft (β := β) (lftFinset u), ?_, ?_⟩
          · exact ⟨SumCon.lft_mem_of_rht_empty A B hu hr,
              liftSumLeft.con A B (SumCon.lft_mem_of_rht_empty A B hu hr),
              liftSumLeft.rhtFinset_eq (β := β) _, by
                rw [liftSumLeft.lftFinset_eq]
                exact EntSet.refl A
                  (SumCon.lft_mem_of_rht_empty A B hu hr)⟩
          · exact h.mono hh (entSet.liftLft_of_left_only A B hu hr)
              (EntSet.refl C hs)
              (liftSumLeft.con A B (SumCon.lft_mem_of_rht_empty A B hu hr)) hs
        simpa [hinl] using hinl'
      exact Or.inr (Or.inl ⟨hne, hf⟩)
  · intro hc
    rcases hc with ⟨hu, hs, hcop⟩
    rcases hcop with hE | ⟨hne, hf⟩ | ⟨hne, hg⟩
    · have hb : h.rel ∅ s :=
        h.mono h.empty_rel (EntSet.empty (sumSystem A B) _) hE
          (sumSystem A B).con_empty hs
      exact h.mono hb (EntSet.empty (sumSystem A B) _) (EntSet.refl C hs) hu hs
    · have : (comp h (inlMap A B)).rel (lftFinset u) s := by simpa [hinl] using hf
      rcases this with ⟨u', hinl', hh'⟩
      have hr : rhtFinset u = ∅ := SumCon.rht_empty_of_lft_nonempty A B hu hne
      have hEnt : (sumSystem A B).EntSet u u' :=
        entSet.u_of_inl_witness A B hne hinl'
          (EntSet.refl A (SumCon.lft_mem_of_rht_empty A B hu hr)) hu hr
      exact h.mono hh' hEnt (EntSet.refl C hs) hu hs
    · have : (comp h (inrMap A B)).rel (rhtFinset u) s := by simpa [hinr] using hg
      rcases this with ⟨u', hinr', hh'⟩
      have hl : lftFinset u = ∅ := SumCon.lft_empty_of_rht_nonempty A B hu hne
      have hEnt : (sumSystem A B).EntSet u u' :=
        entSet.u_of_inr_witness A B hne hinr'
          (EntSet.refl B (SumCon.rht_mem_of_lft_empty A B hu hl)) hu hl
      exact h.mono hh' hEnt (EntSet.refl C hs) hu hs

end copairMap
```

```
end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

5.7.24 Proposition 6.4: injections and copairing

PROPOSITION 6.4. If $\mathbf{A}$ and $\mathbf{B}$ are information systems, then so is $\mathbf{A} + \mathbf{B}$, and we have approximable mappings

$$\mathrm{inl} : \mathbf{A} \to \mathbf{A} + \mathbf{B} \quad \text{and} \quad \mathrm{inr} : \mathbf{B} \to \mathbf{A} + \mathbf{B},$$

such that, for approximable mappings

$$f : \mathbf{A} \to \mathbf{C} \quad \text{and} \quad g : \mathbf{B} \to \mathbf{C},$$

there is one and only one approximable mapping $[f, g] : \mathbf{A} + \mathbf{B} \to \mathbf{C}$, such that

$$[f, g] \circ \mathrm{inl} = f, \quad [f, g] \circ \mathrm{inr} = g, \quad \text{and} \quad [f, g](\bot_{\mathbf{A}+\mathbf{B}}) = \bot_{\mathbf{C}} .$$

Proof. The proof that 6.3 defines a system satisfying the basic axioms of 2.1 has to be left to the reader. Next using the notation of 6.1, where lft and rht were applied as operations on consistent sets $u \in \mathrm{Con}_{\mathbf{A}+\mathbf{B}}$, we redefine matters to have approximable mappings, where, for $v \in \mathrm{Con}_{\mathbf{A}}$ and $w \in \mathrm{Con}_{\mathbf{B}}$,

(1) v inl u iff $\{(X, \Delta) \mid X \in v\} \vdash_{\mathbf{A}+\mathbf{B}} u$;

(2) w inr u iff $\{(\Delta, Y) \mid Y \in w\} \vdash_{\mathbf{A}+\mathbf{B}} u$; and

(3) $u\ [f, g]\ s$ iff either $\vdash_{\mathbf{C}}\ s$, or $\mathrm{lft}\, u \neq \emptyset$ and $\mathrm{lft}\, u\ f\ s$, or $\mathrm{rht}\, u \neq \emptyset$ and $\mathrm{rht}\, u\ g\ s$.

Because we defined $\mathbf{A}+\mathbf{B}$ so that consistency and entailment worked on the two halves of the set of data objects just as they worked on $\mathbf{A}$ and $\mathbf{B}$, respectively, it is easy to check that (1)–(3) define approximable mappings, and that the desired properties hold.

The uniqueness of $[f, g]$ comes from the fact that the elements of $\mathbf{A} + \mathbf{B}$, apart from the bottom element of the domain, are just the elements in the ranges of inl and inr. Since the function $[f, g]$ takes bottom to bottom (in the indicated domains), it will be uniquely determined by what it does on the two halves of the sum. The last equations of the theorem just say that the function is completely determined on these elements. ■

It can also be shown that Propositions 6.2 and 6.4 uniquely characterize the domains $\mathbf{A} \times \mathbf{B}$ and $\mathbf{A} + \mathbf{B}$ up to isomorphism, and they give us the existence of additional mappings that are needed to show that product and sum are functors on the category of domains. We can also show from these results that the domain

$$\mathbf{BOOL} = \mathbf{1} + \mathbf{1}$$

has two elements true and false, such that any mapping on $\mathbf{BOOL}$ is uniquely determined by its action on true, false and $\bot_{\mathbf{BOOL}}$, and the values on the first two elements may be arbitrarily chosen.

Theorem (`ApproximableMap.proposition_6_4`). For approximable mappings $f : \mathbf{A} \to \mathbf{C}$, $g : \mathbf{B} \to \mathbf{C}$, and $h : \mathbf{A} + \mathbf{B} \to \mathbf{C}$, if $h \circ \mathrm{inl} = f$, $h \circ \mathrm{inr} = g$, and $h(\bot_{\mathbf{A}+\mathbf{B}}) = \bot_{\mathbf{C}}$, then $h = [f, g]$.

Proof. Apply uniqueness of the copairing. ?

proposition_6_4.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.copairMap

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys.ApproximableMap

theorem proposition_6_4 :
    ∀ {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
      (A : InfoSys α) (B : InfoSys β) {C : InfoSys ?}
      (f : ApproximableMap A C) (g : ApproximableMap B C)
      (h : ApproximableMap (sumSystem A B) C),
      comp h (inlMap A B) = f →
        comp h (inrMap A B) = g →
          h.toElement (sumSystem A B).botElement = C.botElement →
            h = copairMap A B f g :=
  @copairMap.unique

end DomainTheory.InformationSystems.InfoSys.ApproximableMap
```

5.8 The function space as a domain

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

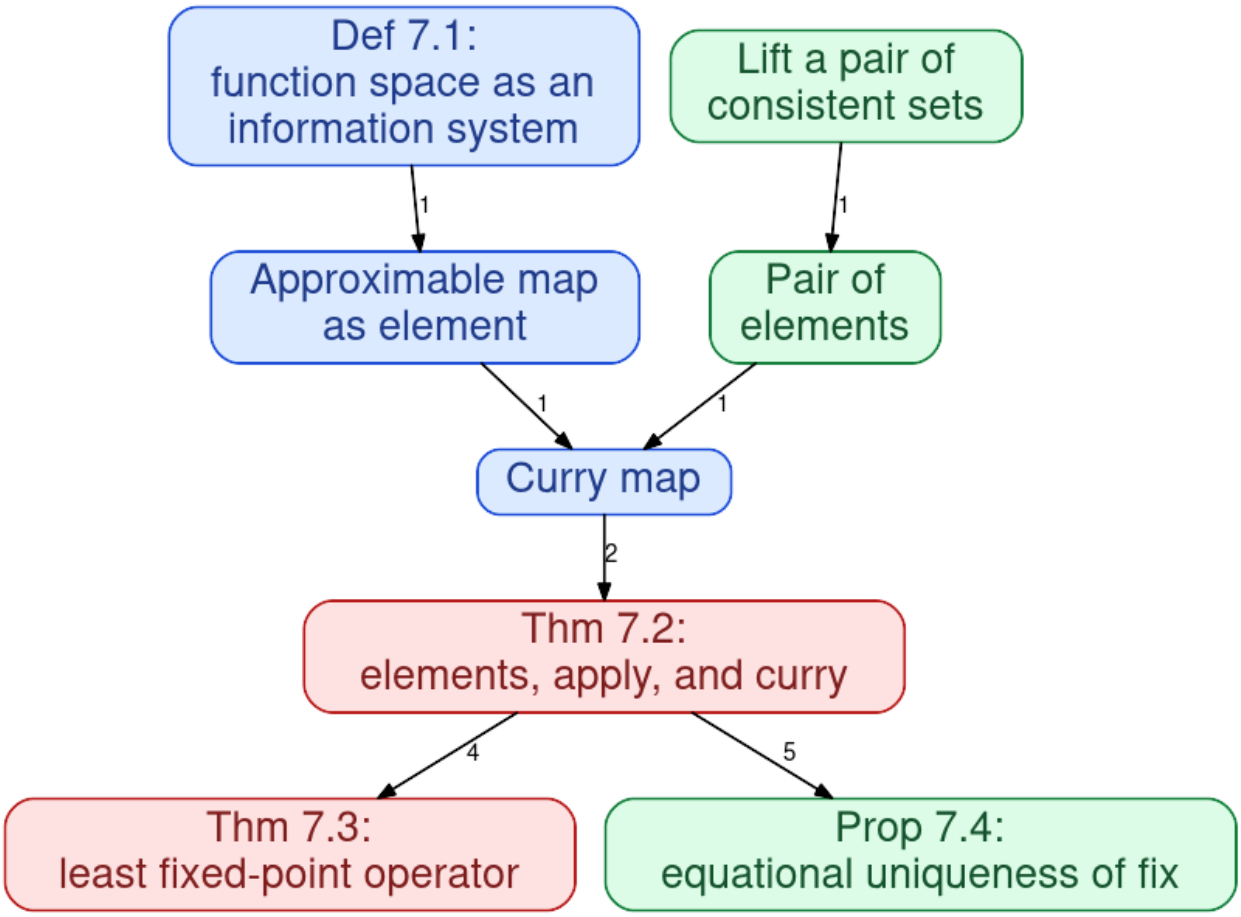


Figure 61: Diagram 26

5.8.1 Introduction

Functions or mappings between domains are of basic importance for our theory, since it is through them that we most easily transform data and relate the structures into which the elements defined by the data objects enter. There are many possible functions, and large groups of them can be treated in a uniform manner. For instance, if the source and target domains match properly, any pair of functions can be composed—composition is an operation on functions of general significance. Now, if in the theory we could combine functions into domains themselves, then an operation like composition might become a mapping of the theory. Indeed, this is exactly what happens: suitably construed, composition is an approximable mapping of two arguments. Of course, for each configuration of linked source and target domains, there is a separate composition operation.

In order to make approximable mappings elements of a suitable domain, we have to discover first what their appropriate data objects are. In Section 5 this was hinted at already. To determine an approximable mapping $f : \mathbf{A} \to \mathbf{B}$, we have to say which pairs (u, v) with $u \in D_{\mathbf{A}}$ and $v \in D_{\mathbf{B}}$ stand in the mapping relation $u\ f\ v$. One such pair gives a certain (finite) amount of information about the possible functions that contain it, and an approximable mapping is completely determined by such pairs. Therefore, if there are appropriate notions of consistency and entailment for these pairs, we will be able to form a domain having

functions as elements. Let us try out a formal definition first, and then look to an explanation of how it works.

5.8.2 Definition 7.1: function space as an information system

DEFINITION 7.1. Let $\mathbf{A}$ and $\mathbf{B}$ be two information systems. By $\mathbf{A} \to \mathbf{B}$, the *function space*, we understand the system where:

(i) $D_{\mathbf{A}\to\mathbf{B}} = \{(u, v) \mid u \in \mathrm{Con}_{\mathbf{A}} \text{ and } v \in \mathrm{Con}_{\mathbf{B}}\}$;

(ii) $\Delta_{\mathbf{A}\to\mathbf{B}} = (\emptyset, \emptyset)$; and where,

for all n and all $w = \{(u_0, v_0), \ldots, (u_{n-1}, v_{n-1})\}$, we have:

(iii) $w \in \mathrm{Con}_{\mathbf{A}\to\mathbf{B}}$ iff whenever $I \subseteq \{0, \ldots, n-1\}$ and $\bigcup\{u_i \mid i \in I\} \in \mathrm{Con}_{\mathbf{A}}$, then $\bigcup\{v_i \mid i \in I\} \in \mathrm{Con}_{\mathbf{B}}$; and

(iv) $w \vdash_{\mathbf{A}\to\mathbf{B}} (u', v')$ iff $\bigcup\{v_i \mid u' \vdash_{\mathbf{A}} u_i\} \vdash_{\mathbf{B}} v'$,

for all $u' \in \mathrm{Con}_{\mathbf{A}}$ and $v' \in \mathrm{Con}_{\mathbf{B}}$. ■

We have already explained the choice of data objects in (i) above, and the least informative pair in (ii) is clearly right. Remember that as a data object (u, v) should be read as meaning that if the information in u is supplied as input, then at least v will be obtained as output. It is pretty obvious that one such data object by itself is consistent (they make constant functions, don't they?), but a set of several of these pairs may not be consistent. Hence, the need for part (iii) of the definition. It can be read informally as follows: Look for a selection I of the indices used in setting up w where the u_i for $i \in I$ are jointly consistent. Since the pairs in w are meant as correct information about a single function, then the combined input from all these selected u_i must be allowable. The function will then be required to give as output at least all the v_i for $i \in I$, owing to the fact that we are given that w is true of the function we have in mind. As a consequence, the set $\bigcup\{v_i \mid i \in I\}$ has got to be consistent, because it comes as output from consistent input for a single approximable function. What we are arguing for is the necessity of (iii)—the word "consistency" should mean that the data objects in the set are all true of at least one function.

Finally we have to argue that (iv) must give the right notion of entailment for these data objects. This can be seen by noting that for a fixed consistent w the set of pairs (u', v') satisfying the right-hand side of (iv) defines an approximable function. In checking this we have to remark that, for each $u' \in \mathrm{Con}_{\mathbf{A}}$, the set $\bigcup\{v_i \mid u' \vdash_{\mathbf{A}} u_i\}$ is consistent, so the definition makes sense. The transitivity properties needed for proving that we have an approximable mapping are easy to establish. This shows in particular that w is true of at least one approximable function, since the separate pairs (u_i, v_i) all satisfy the definition. But it is also simple to argue that for any approximable function, if w is true of it, then so is any pair (u', v') satisfying the definition of (iv). Consequently, what we find in (iv) is the definition of the least approximable function generated by w. The argument we have just outlined thus shows that the relationship $w \vdash_{\mathbf{A}\to\mathbf{B}} (u', v')$ means exactly that whenever w is true of an approximable mapping then so is (u', v'). It follows at once that $\vdash_{\mathbf{A}\to\mathbf{B}}$ is an entailment relation, and that the elements of $\mathbf{A} \to \mathbf{B}$ are just the approximable mappings, as we indicate in the next theorem.

Theorem (`functionSystem.con_sing`). A singleton set of fun tokens is consistent.

Proof. Any subset is empty or the singleton; the output union of a nonempty selection is contained in the singleton output, so consistency of that output applies. ?

Theorem (`functionSystem.con_subset`). Consistency of fun-token sets is downward closed under inclusion.

Proof. A subset of a selection from a smaller set is a selection from the larger set. ?

Theorem (`functionSystem.ent_bot`). Every consistent set of fun tokens entails the function-space bottom.

Proof. Take the empty witness subset; both entailment conditions are vacuous. ?

Theorem (`functionSystem.ent_con`). If a consistent set entails a fun token, then adjoining that token preserves consistency.

Proof. For a selection that avoids the new token, appeal to the original consistency. For a selection that includes it, delete the token, combine its input with the remaining input union via the entailment witness, and transport consistency of the combined outputs back through finite-union algebra and the insert lemmas for input and output unions. ?

Theorem (`functionSystem.ent_refl`). A consistent set entails each of its own tokens.

Proof. The singleton witness consisting of that token, with reflexivity of entailment on each side. ?

Theorem (`functionSystem.ent_trans`). Fun entailment is transitive: if every token of u is entailed by v and u entails c, then v entails c.

Proof. Combine the per-token witnesses into a single witness by the combined-witness lemma, then compose the output entailments. ?

Definition (`InfoSys.functionSystem`). The function space $\mathbf{A} \to \mathbf{B}$ is the information system whose data are the fun tokens of Definition 7.1(i), whose least informative object is the function-space bottom $(\emptyset, \emptyset)$, whose consistent sets are those satisfying Definition 7.1(iii), and whose entailment is Definition 7.1(iv). The information-system axioms hold by the six lemmas above.

Definition (`FunToken`). A fun token of information systems $\mathbf{A}$ and $\mathbf{B}$ is a pair of consistent finite sets (u, v) with u consistent in $\mathbf{A}$ and v consistent in $\mathbf{B}$.

Definition (`instDecidableEqFunToken`). Fun tokens have decidable equality, decided componentwise on the two finite sets.

Theorem (`funBot.property`). The empty finite set is consistent in every information system, on both the input and output sides.

Proof. Empty consistency of each factor. ?

Definition (`funBot`). The least informative fun token is the pair of empty sets $(\emptyset, \emptyset)$.

Definition (`instLeftCommutativeFunInput`). Inserting an input component into the accumulated input union is left-commutative, so multiset folds over input components are independent of order.

Definition (`instLeftCommutativeFunOutput`). Inserting an output component into the accumulated output union is left-commutative, so multiset folds over output components are independent of order.

Theorem (`mem_foldr_funInput`). An element lies in the fold of input-component union over a multiset of fun tokens precisely when it lies in the input of some token of the multiset.

Proof. Induction on the multiset, using membership for finite union at each cons step. ?

Theorem (`mem_foldr_funOutput`). An element lies in the fold of output-component union over a multiset of fun tokens precisely when it lies in the output of some token of the multiset.

Proof. Induction on the multiset, using membership for finite union at each cons step. ?

Definition (`funInputUnion`). The input union of a finite set of fun tokens is the finite union of their input components, obtained by folding finite union over the underlying multiset.

Theorem (`funInputUnion.empty`). The input union of the empty set is empty.

Proof. The fold over the empty multiset returns the empty seed. ?

Theorem (`mem_funInputUnion`). Membership in the input union is membership in the input of some token of the set.

Proof. The fold membership characterisation for inputs. ?

Theorem (`entSet_inputUnion_of_ent`). If a consistent set entails every input component of a finite set of fun tokens, then it entails their input union.

Proof. Unpack membership in the input union and apply the given entailments. ?

Theorem (`funInputUnion.con_of_ent`). The input union of a set of fun tokens whose inputs are all entailed by a consistent set is itself consistent.

Proof. Form the finite union of the witnessing set with the input union and use subset closure of consistency. ?

Theorem (`funInputUnion.funion`). Input union preserves finite unions of token sets.

Proof. Compare membership on both sides via membership in the input union and in a finite union. ?

Theorem (`funInputUnion.singleton`). The input union of a singleton is the input of that token.

Proof. Membership in the input union collapses to the unique token. ?

Theorem (`funInputUnion.insert`). Inserting a token adjoins its input to the input union.

Proof. Rewrite insert as a finite union with a singleton and apply the union and singleton lemmas. ?

Definition (`funOutputUnion`). The output union of a finite set of fun tokens is the finite union of their output components, obtained by folding finite union over the underlying multiset.

Theorem (`funOutputUnion.empty`). The output union of the empty set is empty.

Proof. The fold over the empty multiset returns the empty seed. ?

Theorem (`mem_funOutputUnion`). Membership in the output union is membership in the output of some token of the set.

Proof. The fold membership characterisation for outputs. ?

Theorem (`funOutputUnion.funion`). Output union preserves finite unions of token sets.

Proof. Compare membership on both sides via membership in the output union and in a finite union. ?

Theorem (`funOutputUnion.singleton`). The output union of a singleton is the output of that token.

Proof. Membership in the output union collapses to the unique token. ?

Theorem (`funOutputUnion.insert`). Inserting a token adjoins its output to the output union.

Proof. Rewrite insert as a finite union with a singleton and apply the union and singleton lemmas. ?

Definition (`FunCon`). A finite set w of fun tokens is consistent when every subset whose input union is consistent in $\mathbf{A}$ has output union consistent in $\mathbf{B}$.

Theorem (`exists_combined_witness`). Given a consistent set v of fun tokens and a finite set s of tokens whose inputs are all entailed by a fixed consistent u', together with, for each token of s, a witness subset of v that realises the fun-entailment input and output conditions for that token, there is a single subset $S \subseteq v$ whose tokens are all input-entailed by u' and whose output union entails the output union of s.

Proof. Induction on s: the empty case takes the empty witness; the insert step unions the inductive witness with the witness for the newly inserted token and reassembles the output entailment by finite-union and consistency of fun sets. ?

Definition (`FunEnt`). A consistent finite set w of fun tokens entails a fun token (u', v') when there is a subset $s \subseteq w$ such that u' entails every input of a token in s and the output union of s entails v'.

Theorem (`filter_ne_subset_of_subset_insert`). If a finite set of fun tokens is contained in the insert of p into w, then deleting p from it yields a subset of w.

Proof. Any surviving element lies in the insert and is unequal to p, hence lies in w. ?

Theorem (`insert_filter_ne`). Reinserting an element after deleting it from a finite set that contains it recovers the original set.

Proof. Compare membership on both sides by cases on equality with the deleted element. ?

functionSystem.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys.InfoSys
public import DomainTheory.InformationSystems.InformationSystems.Constructive.funion
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import Mathlib.Data.Finset.Filter
import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

universe u v

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

def FunToken (A : InfoSys α) (B : InfoSys β) : Type _ :=
  {p : Finset α × Finset β // p.1 ∈ A.Con ∧ p.2 ∈ B.Con}

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

instance (A : InfoSys α) (B : InfoSys β) : DecidableEq (FunToken A B) :=
  fun p q =>
    match decidableEq_finset p.val.1 q.val.1, decidableEq_finset p.val.2 q.val.2 with
    | isTrue h1, isTrue h2 => isTrue (Subtype.ext (Prod.ext h1 h2))
    | isFalse n1, _ =>
        isFalse fun h => n1 (congrArg (fun r : FunToken A B => r.val.1) h)
    | isTrue _, isFalse n2 =>
        isFalse fun h => n2 (congrArg (fun r : FunToken A B => r.val.2) h)

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems
```

```
namespace InfoSys

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]
variable (A : InfoSys α) (B : InfoSys β)
universe u v

theorem funBot.property : (∅ : Finset α) ∈ A.Con ∧ (∅ : Finset β) ∈ B.Con :=
  ⟨A.con_empty, B.con_empty⟩

def funBot : FunToken A B :=
  ⟨(∅, ∅), funBot.property A B⟩

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

instance instLeftCommutativeFunInput :
    LeftCommutative fun p : FunToken A B => (funion p.val.1 : Finset α → Finset α) :=
  ⟨fun p q s => funion.left_comm p.val.1 q.val.1 s⟩

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

instance instLeftCommutativeFunOutput :
    LeftCommutative fun p : FunToken A B => (funion p.val.2 : Finset β → Finset β) :=
  ⟨fun p q s => funion.left_comm p.val.2 q.val.2 s⟩

end InfoSys

end DomainTheory.InformationSystems
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

theorem mem_foldr_funInput (t : Multiset (FunToken A B)) (x : α) :
    x ∈ Multiset.foldr (fun p : FunToken A B => funion p.val.1) (∅ : Finset α) t ↔
      ∃ p ∈ t, x ∈ p.val.1 := by
  refine Multiset.induction_on t ?_ ?_
  · constructor
    · intro hx
      exact False.elim (Finset.notMem_empty x hx)
    · rintro ⟨_, hp, _⟩
      exact False.elim (by cases hp)
  · intro p t ih
    simp only [Multiset.foldr_cons, funion.mem, ih, Multiset.mem_cons]
    constructor
    · rintro (hx | ⟨q, hq, hx⟩)
      · exact ⟨p, Or.inl rfl, hx⟩
      · exact ⟨q, Or.inr hq, hx⟩
    · rintro ⟨q, hq, hx⟩
      rcases hq with rfl | hq
      · exact Or.inl hx
      · exact Or.inr ⟨q, hq, hx⟩

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

theorem mem_foldr_funOutput (t : Multiset (FunToken A B)) (y : β) :
    y ∈ Multiset.foldr (fun p : FunToken A B => funion p.val.2) (∅ : Finset β) t ↔
      ∃ p ∈ t, y ∈ p.val.2 := by
  refine Multiset.induction_on t ?_ ?_
  · constructor
    · intro hy
      exact False.elim (Finset.notMem_empty y hy)
    · rintro ⟨_, hp, _⟩
```

```
      exact False.elim (by cases hp)
  · intro p t ih
    simp only [Multiset.foldr_cons, funion.mem, ih, Multiset.mem_cons]
    constructor
    · rintro (hy | ⟨q, hq, hy⟩)
      · exact ⟨p, Or.inl rfl, hy⟩
      · exact ⟨q, Or.inr hq, hy⟩
    · rintro ⟨q, hq, hy⟩
      rcases hq with rfl | hq
      · exact Or.inl hy
      · exact Or.inr ⟨q, hq, hy⟩

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]
variable (A : InfoSys α) (B : InfoSys β)
universe u v

def funInputUnion (s : Finset (FunToken A B)) : Finset α :=
  Multiset.foldr (fun p : FunToken A B => funion p.val.1) (∅ : Finset α) s.1

theorem funInputUnion.empty : funInputUnion A B (∅ : Finset (FunToken A B)) = ∅ :=
  rfl

theorem mem_funInputUnion {s : Finset (FunToken A B)} {x : α} :
    x ∈ funInputUnion A B s ↔ ∃ p ∈ s, x ∈ p.val.1 := by
  simpa [funInputUnion] using mem_foldr_funInput A B s.1 x

theorem entSet_inputUnion_of_ent {u' : Finset α} {s : Finset (FunToken A B)}
    (h : ∀ q ∈ s, A.EntSet u' q.val.1) :
    A.EntSet u' (funInputUnion A B s) := by
  intro x hx
  rcases (mem_funInputUnion A B).1 hx with ⟨q, hq, hx'⟩
  exact h q hq x hx'

theorem funInputUnion.con_of_ent {u' : Finset α} {s : Finset (FunToken A B)}
    (hu' : u' ∈ A.Con) (h : ∀ q ∈ s, A.EntSet u' q.val.1) :
    funInputUnion A B s ∈ A.Con :=
  A.con_subset (EntSet.union_mem A hu' (entSet_inputUnion_of_ent A B h))
    (funion.subset_right _ _)

theorem funInputUnion.funion (s t : Finset (FunToken A B)) :
    funInputUnion A B (s ∪' t) = funInputUnion A B s ∪' funInputUnion A B t := by
  ext x
  constructor
  · intro hx
    rcases (mem_funInputUnion A B).1 hx with ⟨p, hp, hx'⟩
    rcases funion.mem.mp hp with hp | hp
```

```
    · exact funion.mem.mpr (Or.inl ((mem_funInputUnion A B).2 ⟨p, hp, hx'⟩))
    · exact funion.mem.mpr (Or.inr ((mem_funInputUnion A B).2 ⟨p, hp, hx'⟩))
  · intro hx
    rcases funion.mem.mp hx with hx | hx
    · rcases (mem_funInputUnion A B).1 hx with ⟨p, hp, hx'⟩
      exact (mem_funInputUnion A B).2 ⟨p, funion.mem.mpr (Or.inl hp), hx'⟩
    · rcases (mem_funInputUnion A B).1 hx with ⟨p, hp, hx'⟩
      exact (mem_funInputUnion A B).2 ⟨p, funion.mem.mpr (Or.inr hp), hx'⟩

theorem funInputUnion.singleton (p : FunToken A B) :
    funInputUnion A B {p} = p.val.1 := by
  ext x
  constructor
  · intro hx
    rcases (mem_funInputUnion A B).1 hx with ⟨q, hq, hx'⟩
    have : q = p := Finset.mem_singleton.mp hq
    subst this
    exact hx'
  · intro hx
    exact (mem_funInputUnion A B).2 ⟨p, Finset.mem_singleton_self p, hx⟩

theorem funInputUnion.insert (s : Finset (FunToken A B)) (p : FunToken A B) :
    funInputUnion A B (Insert.insert p s) = p.val.1 ∪' funInputUnion A B s := by
  have h : Insert.insert p s = ({p} : Finset (FunToken A B)) ∪' s := by
    ext q
    constructor
    · intro hq
      rcases Finset.mem_insert.mp hq with hqp | hq
      · exact funion.mem.mpr (Or.inl (hqp ▸ Finset.mem_singleton_self p))
      · exact funion.mem.mpr (Or.inr hq)
    · intro hq
      rcases funion.mem.mp hq with hq | hq
      · exact Finset.mem_insert.mpr (Or.inl (Finset.mem_singleton.mp hq))
      · exact Finset.mem_insert_of_mem hq
  rw [h, funInputUnion.funion, funInputUnion.singleton]

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]
variable (A : InfoSys α) (B : InfoSys β)
universe u v

def funOutputUnion (s : Finset (FunToken A B)) : Finset β :=
  Multiset.foldr (fun p : FunToken A B => funion p.val.2) (∅ : Finset β) s.1

theorem funOutputUnion.empty : funOutputUnion A B (∅ : Finset (FunToken A B)) = ∅ :=
  rfl
```

```
theorem mem_funOutputUnion {s : Finset (FunToken A B)} {y : β} :
    y ∈ funOutputUnion A B s ↔ ∃ p ∈ s, y ∈ p.val.2 := by
  simpa [funOutputUnion] using mem_foldr_funOutput A B s.1 y

theorem funOutputUnion.funion (s t : Finset (FunToken A B)) :
    funOutputUnion A B (s ∪' t) = funOutputUnion A B s ∪' funOutputUnion A B t := by
  ext y
  constructor
  · intro hy
    rcases (mem_funOutputUnion A B).1 hy with ⟨p, hp, hy'⟩
    rcases funion.mem.mp hp with hp | hp
    · exact funion.mem.mpr (Or.inl ((mem_funOutputUnion A B).2 ⟨p, hp, hy'⟩))
    · exact funion.mem.mpr (Or.inr ((mem_funOutputUnion A B).2 ⟨p, hp, hy'⟩))
  · intro hy
    rcases funion.mem.mp hy with hy | hy
    · rcases (mem_funOutputUnion A B).1 hy with ⟨p, hp, hy'⟩
      exact (mem_funOutputUnion A B).2 ⟨p, funion.mem.mpr (Or.inl hp), hy'⟩
    · rcases (mem_funOutputUnion A B).1 hy with ⟨p, hp, hy'⟩
      exact (mem_funOutputUnion A B).2 ⟨p, funion.mem.mpr (Or.inr hp), hy'⟩

theorem funOutputUnion.singleton (p : FunToken A B) :
    funOutputUnion A B {p} = p.val.2 := by
  ext y
  constructor
  · intro hy
    rcases (mem_funOutputUnion A B).1 hy with ⟨q, hq, hy'⟩
    have : q = p := Finset.mem_singleton.mp hq
    subst this
    exact hy'
  · intro hy
    exact (mem_funOutputUnion A B).2 ⟨p, Finset.mem_singleton_self p, hy⟩

theorem funOutputUnion.insert (s : Finset (FunToken A B)) (p : FunToken A B) :
    funOutputUnion A B (Insert.insert p s) = p.val.2 ∪' funOutputUnion A B s := by
  have h : Insert.insert p s = ({p} : Finset (FunToken A B)) ∪' s := by
    ext q
    constructor
    · intro hq
      rcases Finset.mem_insert.mp hq with hqp | hq
      · exact funion.mem.mpr (Or.inl (hqp ▸ Finset.mem_singleton_self p))
      · exact funion.mem.mpr (Or.inr hq)
    · intro hq
      rcases funion.mem.mp hq with hq | hq
      · exact Finset.mem_insert.mpr (Or.inl (Finset.mem_singleton.mp hq))
      · exact Finset.mem_insert_of_mem hq
  rw [h, funOutputUnion.funion, funOutputUnion.singleton]

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

namespace InfoSys

universe u v
```

```
variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def FunCon (w : Finset (FunToken A B)) : Prop :=
  ∀ s ⊆ w, funInputUnion A B s ∈ A.Con → funOutputUnion A B s ∈ B.Con

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

theorem exists_combined_witness {u' : Finset α} {s : Finset (FunToken A B)}
    {v : Finset (FunToken A B)}
    (hv : FunCon A B v) (hu' : u' ∈ A.Con)
    (hEntIn : ∀ q ∈ s, A.EntSet u' q.val.1)
    (hWit : ∀ q ∈ s, ∃ s_q ⊆ v, (∀ r ∈ s_q, A.EntSet q.val.1 r.val.1) ∧
      B.EntSet (funOutputUnion A B s_q) q.val.2) :
    ∃ S ⊆ v, (∀ r ∈ S, A.EntSet u' r.val.1) ∧
      B.EntSet (funOutputUnion A B S) (funOutputUnion A B s) := by
  induction s using Finset.induction_on with
  | empty =>
    refine ⟨∅, Finset.empty_subset _, ?_, ?_⟩
    · intro _ hq
      exact False.elim (Finset.notMem_empty _ hq)
    · intro y hy
      exact False.elim (Finset.notMem_empty y hy)
  | insert q s' hqnot ih =>
    have hEntIn' : ∀ r ∈ s', A.EntSet u' r.val.1 := fun r hr =>
      hEntIn r (Finset.mem_insert_of_mem hr)
    have hWit' : ∀ r ∈ s', ∃ s_r ⊆ v, (∀ t ∈ s_r, A.EntSet r.val.1 t.val.1) ∧
        B.EntSet (funOutputUnion A B s_r) r.val.2 := fun r hr =>
      hWit r (Finset.mem_insert_of_mem hr)
    obtain ⟨S', hS'sub, hS'in, hS'out⟩ := ih hEntIn' hWit'
    obtain ⟨Sq, hSqsub, hSqin, hSqout⟩ := hWit q (Finset.mem_insert_self q s')
    refine ⟨S' ∪' Sq, funion.subset_iff.mpr ⟨hS'sub, hSqsub⟩, ?_, ?_⟩
    · intro r hr
      rcases funion.mem.mp hr with hr | hr
      · exact hS'in r hr
      · have hEnt_q : A.EntSet u' q.val.1 := hEntIn q (Finset.mem_insert_self q s')
        have hEnt_r : A.EntSet q.val.1 r.val.1 := hSqin r hr
        exact EntSet.trans A hu' q.property.1 hEnt_q hEnt_r
    · rw [funOutputUnion.funion, funOutputUnion.insert]
      have hinS' : funInputUnion A B S' ∈ A.Con :=
```

```
        funInputUnion.con_of_ent A B hu' hS'in
      have houtS' : funOutputUnion A B S' ∈ B.Con := hv S' hS'sub hinS'
      have hinSq : funInputUnion A B Sq ∈ A.Con :=
        funInputUnion.con_of_ent A B q.property.1 hSqin
      have houtSq : funOutputUnion A B Sq ∈ B.Con := hv Sq hSqsub hinSq
      have hinUnion : funInputUnion A B (S' ∪' Sq) ∈ A.Con := by
        rw [funInputUnion.funion]
        have hEnt : A.EntSet u' (funInputUnion A B S' ∪' funInputUnion A B Sq) :=
          EntSet.union A
            (entSet_inputUnion_of_ent A B hS'in)
            (EntSet.trans A hu' q.property.1
              (hEntIn q (Finset.mem_insert_self q s'))
              (entSet_inputUnion_of_ent A B hSqin))
        exact A.con_subset (EntSet.union_mem A hu' hEnt) (funion.subset_right _ _)

      have houtUnion : funOutputUnion A B S' ∪' funOutputUnion A B Sq ∈ B.Con := by
        have : funOutputUnion A B (S' ∪' Sq) ∈ B.Con := hv _ (funion.subset_iff.mpr ⟨hS
    'sub, hSqsub⟩) hinUnion
        rwa [funOutputUnion.funion] at this
      have hEnt1 : B.EntSet (funOutputUnion A B S' ∪' funOutputUnion A B Sq)
          (funOutputUnion A B s') :=
        EntSet.mono B houtS' houtUnion (funion.subset_left _ _) hS'out
          (Finset.Subset.refl _)
      have hEnt2 : B.EntSet (funOutputUnion A B S' ∪' funOutputUnion A B Sq) q.val.2
    :=
        EntSet.mono B houtSq houtUnion (funion.subset_right _ _) hSqout
          (Finset.Subset.refl _)
      exact EntSet.union B hEnt2 hEnt1

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

namespace InfoSys

universe u v

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

def FunEnt (w : Finset (FunToken A B)) (p : FunToken A B) : Prop :=
  FunCon A B w ∧
    ∃ s ⊆ w, (∀ q ∈ s, A.EntSet p.val.1 q.val.1) ∧
      B.EntSet (funOutputUnion A B s) p.val.2

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

namespace InfoSys
```

```
universe u v

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

theorem filter_ne_subset_of_subset_insert {p : FunToken A B}
    {t w : Finset (FunToken A B)} (ht : t ⊆ insert p w) :
    t.filter (· = p) ⊆ w := by
  intro q hq
  have ⟨hqt, hne⟩ := Finset.mem_filter.mp hq
  have : q ∈ insert p w := ht hqt
  rcases Finset.mem_insert.mp this with rfl | hqW
  · exact False.elim (hne rfl)
  · exact hqW

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

namespace InfoSys

universe u v

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]

variable (A : InfoSys α) (B : InfoSys β)

theorem insert_filter_ne {p : FunToken A B} {t : Finset (FunToken A B)}
    (hp : p ∈ t) : insert p (t.filter (· = p)) = t := by
  ext q
  constructor
  · intro hq
    rcases Finset.mem_insert.mp hq with rfl | hq
    · exact hp
    · exact (Finset.mem_filter.mp hq).1
  · intro hq
    if h : q = p then
      exact Finset.mem_insert.mpr (Or.inl h)
    else
      exact Finset.mem_insert.mpr (Or.inr (Finset.mem_filter.mpr ⟨hq, h⟩))

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {α : Type u} {β : Type v} [DecidableEq α] [DecidableEq β]
variable (A : InfoSys α) (B : InfoSys β)
```

```
universe u v

theorem functionSystem.con_sing (p : FunToken A B) : FunCon A B {p} := by
  intro s hs _hin
  have hsub : funOutputUnion A B s ⊆ p.val.2 := by
    intro y hy
    rcases (mem_funOutputUnion A B).1 hy with ⟨q, hq, hy'⟩
    have hq' : q = p := Finset.mem_singleton.mp (hs hq)
    cases hq'
    exact hy'
  exact B.con_subset p.property.2 hsub

theorem functionSystem.con_subset {w w' : Finset (FunToken A B)}
    (hw : FunCon A B w) (hw' : w' ⊆ w) : FunCon A B w' :=
  fun s hs hin => hw s (fun _ hq => hw' (hs hq)) hin

theorem functionSystem.ent_bot {w : Finset (FunToken A B)} (hw : FunCon A B w) :
    FunEnt A B w (funBot A B) :=
  ⟨hw, ∅, Finset.empty_subset _,
    fun q hq => False.elim (Finset.notMem_empty q hq),
    fun y hy => False.elim (Finset.notMem_empty y hy)⟩

theorem functionSystem.ent_con {w : Finset (FunToken A B)} {p : FunToken A B}
    (hEnt : FunEnt A B w p) : FunCon A B (insert p w) := by
  obtain ⟨hw, s, hs, hEntIn, hEntOut⟩ := hEnt
  intro t ht hin
  if hp : p ∈ t then
    let t' := t.filter (· = p)
    have ht_eq : insert p t' = t := insert_filter_ne A B hp
    have ht'sub : t' ⊆ w := filter_ne_subset_of_subset_insert A B ht
    have hin' : p.val.1 ∪' funInputUnion A B t' ∈ A.Con := by
      rwa [← ht_eq, funInputUnion.insert] at hin
    have hsub : s ∪' t' ⊆ w := funion.subset_iff.mpr ⟨hs, ht'sub⟩
    have hEnt_in_s : A.EntSet p.val.1 (funInputUnion A B s) :=
      entSet_inputUnion_of_ent A B hEntIn
    have hEnt_big : A.EntSet (p.val.1 ∪' funInputUnion A B t') (funInputUnion A B s)
    :=
      EntSet.mono A p.property.1 hin' (funion.subset_left _ _) hEnt_in_s
        (Finset.Subset.refl _)
    have hbig : (p.val.1 ∪' funInputUnion A B t') ∪' funInputUnion A B s ∈ A.Con :=
      EntSet.union_mem A hin' hEnt_big
    have hin_st' : funInputUnion A B s ∪' funInputUnion A B t' ∈ A.Con := by
      have heq :
          (p.val.1 ∪' funInputUnion A B t') ∪' funInputUnion A B s =
            p.val.1 ∪' (funInputUnion A B s ∪' funInputUnion A B t') := by
        rw [funion.assoc, funion.comm (funInputUnion A B t')]
      have hbig' : p.val.1 ∪' (funInputUnion A B s ∪' funInputUnion A B t') ∈ A.Con :=
     by
        rwa [heq] at hbig
      exact A.con_subset hbig' (funion.subset_right _ _)
    have hin_union : funInputUnion A B (s ∪' t') ∈ A.Con := by
      rwa [funInputUnion.funion]
    have hout_union : funOutputUnion A B s ∪' funOutputUnion A B t' ∈ B.Con := by
      have : funOutputUnion A B (s ∪' t') ∈ B.Con := hw _ hsub hin_union
      rwa [funOutputUnion.funion] at this
    have hout_s : funOutputUnion A B s ∈ B.Con :=
```

```
      B.con_subset hout_union (funion.subset_left _ _)
    have hEnt_out' : B.EntSet (funOutputUnion A B s ∪' funOutputUnion A B t') p.val.2
    :=
      EntSet.mono B hout_s hout_union (funion.subset_left _ _) hEntOut
        (Finset.Subset.refl _)
    have hfinal :
        (funOutputUnion A B s ∪' funOutputUnion A B t') ∪' p.val.2 ∈ B.Con :=
      EntSet.union_mem B hout_union hEnt_out'
    have hout_t : p.val.2 ∪' funOutputUnion A B t' ∈ B.Con := by
      have heq :
          (funOutputUnion A B s ∪' funOutputUnion A B t') ∪' p.val.2 =
            funOutputUnion A B s ∪' (p.val.2 ∪' funOutputUnion A B t') := by
        rw [funion.assoc, funion.comm (funOutputUnion A B t')]
      have hfinal' :
          funOutputUnion A B s ∪' (p.val.2 ∪' funOutputUnion A B t') ∈ B.Con := by
        rwa [heq] at hfinal
      exact B.con_subset hfinal' (funion.subset_right _ _)
    have : funOutputUnion A B t ∈ B.Con := by
      rwa [← ht_eq, funOutputUnion.insert]
    exact this
  else
    have ht' : t ⊆ w := by
      intro q hq
      have : q ∈ insert p w := ht hq
      rcases Finset.mem_insert.mp this with rfl | hqW
      · exact False.elim (hp hq)
      · exact hqW
    exact hw t ht' hin

theorem functionSystem.ent_refl {w : Finset (FunToken A B)} {p : FunToken A B}
    (hw : FunCon A B w) (hp : p ∈ w) : FunEnt A B w p :=
  ⟨hw, {p},
    fun q hq => by
      cases Finset.mem_singleton.mp hq
      exact hp,
    fun q hq => by
      cases Finset.mem_singleton.mp hq
      exact EntSet.refl A p.property.1,
    by
      rw [funOutputUnion.singleton]
      exact EntSet.refl B p.property.2⟩

theorem functionSystem.ent_trans {u v : Finset (FunToken A B)} {c : FunToken A B}
    (hv : FunCon A B v) (hu : FunCon A B u)
    (hEnts : ∀ y ∈ u, FunEnt A B v y) (hEnt : FunEnt A B u c) :
    FunEnt A B v c := by
  obtain ⟨_, s, hs, hEntIn, hEntOut⟩ := hEnt
  refine ⟨hv, ?_⟩
  have hWit : ∀ q ∈ s, ∃ s_q ⊆ v, (∀ r ∈ s_q, A.EntSet q.val.1 r.val.1) ∧
      B.EntSet (funOutputUnion A B s_q) q.val.2 := by
    intro q hq
    exact (hEnts q (hs hq)).2
  obtain ⟨S, hSsub, hSin, hSout⟩ :=
    exists_combined_witness A B hv c.property.1 hEntIn hWit
  refine ⟨S, hSsub, hSin, ?_⟩
  have hin_s : funInputUnion A B s ∈ A.Con :=
```

```
    funInputUnion.con_of_ent A B c.property.1 hEntIn
  have hout_s : funOutputUnion A B s ∈ B.Con := hu s hs hin_s
  have hin_S : funInputUnion A B S ∈ A.Con :=
    funInputUnion.con_of_ent A B c.property.1 hSin
  have hout_S : funOutputUnion A B S ∈ B.Con := hv S hSsub hin_S
  exact EntSet.trans B hout_S hout_s hSout hEntOut

def functionSystem : InfoSys (FunToken A B) where
  bot := funBot A B
  Con := {w | FunCon A B w}
  Ent := FunEnt A B
  con_subset := functionSystem.con_subset A B
  con_sing := functionSystem.con_sing A B
  ent_con := functionSystem.ent_con A B
  ent_bot := functionSystem.ent_bot A B
  ent_refl := functionSystem.ent_refl A B
  ent_trans := functionSystem.ent_trans A B

end InfoSys

end DomainTheory.InformationSystems
```

5.8.3 Lift a pair of consistent sets

Definition (`liftPair`). The product lift of consistent sets u and v is the finite union of the left lift of u with the right lift of v.

Theorem (`ProdCon_liftPair`). The product lift of two consistent sets is product-consistent.

Proof. Left and right projections of the lift reduce to the given consistent sets (up to bottoms). ?

Theorem (`fstFinset_liftPair`). The first projection of a product lift is the first set united with the first projection of the right lift.

Proof. Expand the lift as a finite union and apply first-projection lemmas. ?

Theorem (`sndFinset_liftPair`). The second projection of a product lift is the second projection of the left lift united with the second set.

Proof. Expand the lift as a finite union and apply second-projection lemmas. ?

Theorem (`entSet_liftPair`). Product entailment of a lift follows from factor entailments of its components.

Proof. Lift the two entailments through the left and right embeddings and unite. ?

Theorem (`liftLeft_funion`). Left lift preserves finite unions.

Proof. Compare membership on both sides. ?

Theorem (`liftRight_funion`). Right lift preserves finite unions.

Proof. Compare membership on both sides. ?

`liftPair.lean`

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    liftLeft
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    liftRight
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    productSystem
```

```
import DomainTheory.InformationSystems.InformationSystems.EntSet

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

def liftPair (u : Finset α) (v : Finset β) : Finset (ProdToken A B) :=
  liftLeft A B u ∪' liftRight A B v

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem liftLeft_funion (u u' : Finset α) :
    liftLeft A B (u ∪' u') = liftLeft A B u ∪' liftLeft A B u' := by
  ext p
  constructor
  · intro hp
    rcases (liftLeft.mem A B).1 hp with ⟨x, hx, rfl⟩
    rcases funion.mem.mp hx with hx | hx
    · exact funion.mem.mpr (Or.inl ((liftLeft.mem A B).2 ⟨x, hx, rfl⟩))
    · exact funion.mem.mpr (Or.inr ((liftLeft.mem A B).2 ⟨x, hx, rfl⟩))
  · intro hp
    rcases funion.mem.mp hp with hp | hp
    · rcases (liftLeft.mem A B).1 hp with ⟨x, hx, rfl⟩
      exact (liftLeft.mem A B).2 ⟨x, funion.mem.mpr (Or.inl hx), rfl⟩
    · rcases (liftLeft.mem A B).1 hp with ⟨x, hx, rfl⟩
      exact (liftLeft.mem A B).2 ⟨x, funion.mem.mpr (Or.inr hx), rfl⟩
```

```
end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem liftRight_funion (v v' : Finset β) :
    liftRight A B (v ∪' v') = liftRight A B v ∪' liftRight A B v' := by
  ext p
  constructor
  · intro hp
    rcases (liftRight.mem A B).1 hp with ⟨y, hy, rfl⟩
    rcases funion.mem.mp hy with hy | hy
    · exact funion.mem.mpr (Or.inl ((liftRight.mem A B).2 ⟨y, hy, rfl⟩))
    · exact funion.mem.mpr (Or.inr ((liftRight.mem A B).2 ⟨y, hy, rfl⟩))
  · intro hp
    rcases funion.mem.mp hp with hp | hp
    · rcases (liftRight.mem A B).1 hp with ⟨y, hy, rfl⟩
      exact (liftRight.mem A B).2 ⟨y, funion.mem.mpr (Or.inl hy), rfl⟩
    · rcases (liftRight.mem A B).1 hp with ⟨y, hy, rfl⟩
      exact (liftRight.mem A B).2 ⟨y, funion.mem.mpr (Or.inr hy), rfl⟩

end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem ProdCon_liftPair {u : Finset α} {v : Finset β}
    (hu : u ∈ A.Con) (hv : v ∈ B.Con) : ProdCon A B (liftPair A B u v) := by
  constructor
  · unfold liftPair
    rw [fstFinset.funion, liftLeft.fstFinset_eq]
```

```
    have hEnt : A.EntSet u (fstFinset A B (liftRight A B v)) := fun x hx => by
      have : x = A.bot := Finset.mem_singleton.mp (liftRight.fstFinset_eq A B v hx)
      subst this
      exact A.ent_bot hu
    exact EntSet.union_mem A hu hEnt
  · unfold liftPair
    rw [sndFinset.funion, liftRight.sndFinset_eq]
    have hEnt : B.EntSet v (sndFinset A B (liftLeft A B u)) := fun y hy => by
      have : y = B.bot := Finset.mem_singleton.mp (liftLeft.sndFinset_eq A B u hy)
      subst this
      exact B.ent_bot hv
    have h := EntSet.union_mem B hv hEnt
    rwa [funion.comm] at h

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem fstFinset_liftPair (u : Finset α) (v : Finset β) :
    fstFinset A B (liftPair A B u v) =
      u ∪' fstFinset A B (liftRight A B v) := by
  unfold liftPair
  rw [fstFinset.funion, liftLeft.fstFinset_eq]

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)
```

```
open ApproximableMap

theorem sndFinset_liftPair (u : Finset α) (v : Finset β) :
    sndFinset A B (liftPair A B u v) =
      sndFinset A B (liftLeft A B u) ∪' v := by
  unfold liftPair
  rw [sndFinset.funion, liftRight.sndFinset_eq]

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem entSet_liftPair {u u' : Finset α} {v v' : Finset β}
    (hu' : u' ∈ A.Con) (hv' : v' ∈ B.Con)
    (hEntu : A.EntSet u' u) (hEntv : B.EntSet v' v) :
    (productSystem A B).EntSet (liftPair A B u' v') (liftPair A B u v) := by
  intro p hp
  have hCon : ProdCon A B (liftPair A B u' v') := ProdCon_liftPair A B hu' hv'
  unfold liftPair at hp
  rcases funion.mem.mp hp with hp | hp
  · rcases (liftLeft.mem A B).1 hp with ⟨x, hx, rfl⟩
    refine ⟨hCon, ?_, ?_⟩
    · intro _
      have hsub : u' ⊆ fstFinset A B (liftPair A B u' v') := by
        intro z hz
        rw [fstFinset_liftPair]
        exact funion.subset_left _ _ hz
      exact A.ent_trans hCon.1 hu'
        (fun z hz => A.ent_refl hCon.1 (hsub hz)) (hEntu x hx)
    · intro _
      exact B.ent_bot hCon.2
  · rcases (liftRight.mem A B).1 hp with ⟨y, hy, rfl⟩
    refine ⟨hCon, ?_, ?_⟩
    · intro _
      exact A.ent_bot hCon.1
    · intro _
      have hsub : v' ⊆ sndFinset A B (liftPair A B u' v') := by
        intro z hz
        rw [sndFinset_liftPair]
        exact funion.subset_right _ _ hz
      exact B.ent_trans hCon.2 hv'
```

```
        (fun z hz => B.ent_refl hCon.2 (hsub hz)) (hEntv y hy)

end InfoSys

end DomainTheory.InformationSystems
```

5.8.4 Pair of elements

Definition (`pairElements`). The paired element of $x \in |\mathbf{A}|$ and $y \in |\mathbf{B}|$ consists of those product tokens whose non-bottom sides lie in the corresponding factors.

Theorem (`fstMap_pairElements`). Projecting a paired element on the left recovers the first factor.

Proof. Compare the two elements by chasing product tokens through the first projection map. ?

Theorem (`sndMap_pairElements`). Projecting a paired element on the right recovers the second factor.

Proof. Compare the two elements by chasing product tokens through the second projection map. ?

Theorem (`mem_fst_of_subset_pairElements`). Tokens of a finite set contained in a paired element have first components in the first factor whenever the second component is bottom.

Proof. Unfold membership in the paired element. ?

Theorem (`mem_snd_of_subset_pairElements`). Tokens of a finite set contained in a paired element have second components in the second factor whenever the first component is bottom.

Proof. Unfold membership in the paired element. ?

Theorem (`subset_pairElements_liftLeft`). The left lift of a finite set contained in the first factor lies in the paired element.

Proof. Left-lifted tokens have bottom on the right and first component in the first factor. ?

Theorem (`subset_pairElements_liftRight`). The right lift of a finite set contained in the second factor lies in the paired element.

Proof. Right-lifted tokens have bottom on the left and second component in the second factor. ?

Theorem (`subset_pairElements_liftPair`). The product lift of finite sets from each factor lies in the paired element.

Proof. Unite the left- and right-lift inclusions. ?

`pairElements.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    productSystem
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    liftLeft
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    liftRight
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    liftPair
import DomainTheory.InformationSystems.ElementsOfASystem.Factoid32.factoid_3_2

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

def pairElements (x : A.Element) (y : B.Element) : (productSystem A B).Element where
  carrier := {p : ProdToken A B |
    (p.val.2 = B.bot → p.val.1 ∈ x.carrier) ∧ (p.val.1 = A.bot → p.val.2 ∈ y.carrier)
    }
  consistent := by
    intro Y hY
    refine ⟨?_, ?_⟩
    · have hsub : ↑(fstFinset A B Y) ⊆ x.carrier := by
        intro a ha
        rcases (mem_fstFinset A B).1 ha with ⟨p, hp, hbot, rfl⟩
        exact (hY (Finset.mem_coe.2 hp)).1 hbot
      exact x.consistent _ hsub
    · have hsub : ↑(sndFinset A B Y) ⊆ y.carrier := by
        intro b hb
        rcases (mem_sndFinset A B).1 hb with ⟨p, hp, hbot, rfl⟩
        exact (hY (Finset.mem_coe.2 hp)).2 hbot
      exact y.consistent _ hsub
  closed := by
    intro Y p hY hEnt
    obtain ⟨hCon, hL, hR⟩ := hEnt
    refine ⟨?_, ?_⟩
    · intro hbot
      have hEntA : A.Ent (fstFinset A B Y) p.val.1 := hL hbot
      have hsub : ↑(fstFinset A B Y) ⊆ x.carrier := by
        intro a ha
        rcases (mem_fstFinset A B).1 ha with ⟨q, hq, hbot', rfl⟩
        exact (hY (Finset.mem_coe.2 hq)).1 hbot'
      exact x.closed _ _ hsub hEntA
    · intro hbot
      have hEntB : B.Ent (sndFinset A B Y) p.val.2 := hR hbot
      have hsub : ↑(sndFinset A B Y) ⊆ y.carrier := by
        intro b hb
        rcases (mem_sndFinset A B).1 hb with ⟨q, hq, hbot', rfl⟩
        exact (hY (Finset.mem_coe.2 hq)).2 hbot'
      exact y.closed _ _ hsub hEntB

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem fstMap_pairElements (x : A.Element) (y : B.Element) :
    (productSystem.fstMap A B).toElement (pairElements A B x y) = x := by
  apply le_antisymm
  · intro a ⟨s, hs, ⟨hCon, _, hEnt⟩⟩
    have hsub : ↑(fstFinset A B s) ⊆ x.carrier := by
      intro b hb
      rcases (mem_fstFinset A B).1 hb with ⟨p, hp, hbot, rfl⟩
      exact (hs (Finset.mem_coe.2 hp)).1 hbot
    exact x.closed _ a hsub (hEnt a (Finset.mem_singleton_self _))
  · intro a ha
    refine ⟨liftLeft A B {a}, ?_, ?_⟩
    · intro p hp
      rcases (liftLeft.mem A B).1 (Finset.mem_coe.1 hp) with ⟨b, hb, rfl⟩
      have : b = a := Finset.mem_singleton.mp hb
      subst this
      exact ⟨fun _ => ha, fun _ => factoid_3_2 B y⟩
    · refine ⟨liftLeft.con A B (A.con_sing a), A.con_sing a, ?_⟩
      rw [liftLeft.fstFinset_eq]
      exact EntSet.refl A (A.con_sing a)

end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem sndMap_pairElements (x : A.Element) (y : B.Element) :
    (productSystem.sndMap A B).toElement (pairElements A B x y) = y := by
  apply le_antisymm
  · intro b ⟨s, hs, ⟨hCon, _, hEnt⟩⟩
    have hsub : ↑(sndFinset A B s) ⊆ y.carrier := by
      intro c hc
```

```
      rcases (mem_sndFinset A B).1 hc with ⟨p, hp, hbot, rfl⟩
      exact (hs (Finset.mem_coe.2 hp)).2 hbot
    exact y.closed _ b hsub (hEnt b (Finset.mem_singleton_self _))
  · intro b hb
    refine ⟨liftRight A B {b}, ?_, ?_⟩
    · intro p hp
      rcases (liftRight.mem A B).1 (Finset.mem_coe.1 hp) with ⟨c, hc, rfl⟩
      have : c = b := Finset.mem_singleton.mp hc
      subst this
      exact ⟨fun _ => factoid_3_2 A x, fun _ => hb⟩
    · refine ⟨liftRight.con A B (B.con_sing b), B.con_sing b, ?_⟩
      rw [liftRight.sndFinset_eq]
      exact EntSet.refl B (B.con_sing b)

end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem mem_fst_of_subset_pairElements {x : A.Element} {y : B.Element}
    {s : Finset (ProdToken A B)}
    (hs : ↑s ⊆ (pairElements A B x y).carrier) {a : α}
    (ha : a ∈ fstFinset A B s) : a ∈ x.carrier := by
  have hCon : ProdCon A B s := (pairElements A B x y).consistent s hs
  have : a ∈ ((productSystem.fstMap A B).toElement (pairElements A B x y)).carrier :=
    by
    refine ⟨s, hs, ⟨hCon, A.con_sing a, ?_⟩⟩
    intro b hb
    have : b = a := Finset.mem_singleton.mp hb
    subst this
    exact A.ent_refl hCon.1 ha
  simpa [fstMap_pairElements] using this

end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w
```

```
variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem mem_snd_of_subset_pairElements {x : A.Element} {y : B.Element}
    {s : Finset (ProdToken A B)}
    (hs : ↑s ⊆ (pairElements A B x y).carrier) {b : β}
    (hb : b ∈ sndFinset A B s) : b ∈ y.carrier := by
  have hCon : ProdCon A B s := (pairElements A B x y).consistent s hs
  have : b ∈ ((productSystem.sndMap A B).toElement (pairElements A B x y)).carrier :=
    by
    refine ⟨s, hs, ⟨hCon, B.con_sing b, ?_⟩⟩
    intro c hc
    have : c = b := Finset.mem_singleton.mp hc
    subst this
    exact B.ent_refl hCon.2 hb
  simpa [sndMap_pairElements] using this

end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem subset_pairElements_liftLeft {x : A.Element} {y : B.Element}
    {u : Finset α} (hu : ↑u ⊆ x.carrier) :
    ↑(liftLeft A B u) ⊆ (pairElements A B x y).carrier := by
  intro p hp
  rcases (liftLeft.mem A B).1 (Finset.mem_coe.1 hp) with ⟨a, ha, rfl⟩
  have hax : a ∈ x.carrier := hu (Finset.mem_coe.2 ha)
  have : a ∈ ((productSystem.fstMap A B).toElement (pairElements A B x y)).carrier :=
    by
    simpa [fstMap_pairElements] using hax
  rcases this with ⟨s, hs, ⟨hCon, _, hEnt⟩⟩
  exact (pairElements A B x y).closed s (mkLeft A B a) hs ⟨hCon,
    fun _ => hEnt a (Finset.mem_singleton_self _),
    fun _ => B.ent_bot hCon.2⟩

end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem subset_pairElements_liftRight {x : A.Element} {y : B.Element}
    {v : Finset β} (hv : ↑v ⊆ y.carrier) :
    ↑(liftRight A B v) ⊆ (pairElements A B x y).carrier := by
  intro p hp
  rcases (liftRight.mem A B).1 (Finset.mem_coe.1 hp) with ⟨b, hb, rfl⟩
  have hby : b ∈ y.carrier := hv (Finset.mem_coe.2 hb)
  have : b ∈ ((productSystem.sndMap A B).toElement (pairElements A B x y)).carrier :=
    by
    simpa [sndMap_pairElements] using hby
  rcases this with ⟨s, hs, ⟨hCon, _, hEnt⟩⟩
  exact (pairElements A B x y).closed s (mkRight A B b) hs ⟨hCon,
    fun _ => A.ent_bot hCon.1,
    fun _ => hEnt b (Finset.mem_singleton_self _)⟩

end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem subset_pairElements_liftPair {x : A.Element} {y : B.Element}
    {u : Finset α} {v : Finset β} (hu : ↑u ⊆ x.carrier) (hv : ↑v ⊆ y.carrier) :
    ↑(liftPair A B u v) ⊆ (pairElements A B x y).carrier := by
  intro p hp
  have hp' : p ∈ liftLeft A B u ∪' liftRight A B v := Finset.mem_coe.1 hp
  rcases funion.mem.mp hp' with hp | hp
  · exact subset_pairElements_liftLeft A B hu (Finset.mem_coe.2 hp)
  · exact subset_pairElements_liftRight A B hv (Finset.mem_coe.2 hp)

end InfoSys
end DomainTheory.InformationSystems
```

5.8.5 Approximable map as element

Theorem (`approxMap_toElement.closed`). The set of fun tokens related by an approximable map is closed under function-space entailment.

Proof. Unpack entailment witnesses and apply monotonicity and transitivity of the underlying relation. ?

Theorem (`approxMap_toElement.consistent`). The set of fun tokens related by an approximable map is consistent in the function space.

Proof. Apply fun-consistency of approximable maps to any finite subset. ?

Definition (`approxMap_toElement`). The element of $\mathbf{A} \to \mathbf{B}$ associated to an approximable map f is the set of all fun tokens (u, v) with $u\ f\ v$.

Definition (`mkFunToken`). Given consistent finite sets u and v in $\mathbf{A}$ and $\mathbf{B}$ respectively, their fun token is the pair (u, v) as a data object of the function space.

Theorem (`mkFunToken.property`). The two components of a fun token constructed from consistent sets are consistent.

Proof. Immediate from the given consistency hypotheses. ?

Theorem (`mkFunToken.eq`). Rebuilding a fun token from its components recovers the original token.

Proof. The underlying pair is unchanged. ?

Theorem (`rel_input_output_union`). If an approximable map relates every token of a finite set and the input union is consistent, then it relates the input union to the output union.

Proof. Iterate the right-union axiom of approximable maps. ?

Theorem (`funCon_of_approxMap`). Any finite set of fun tokens whose pairs all lie in an approximable relation is consistent in the function space.

Proof. For a selection whose input union is consistent, the relation's right-union and domain axioms yield consistency of the output union. ?

Theorem (`element_toApproxMap.empty_rel`). Every function-space element relates the empty input to the empty output.

Proof. The function-space bottom lies in every element. ?

Theorem (`element_toApproxMap.mono`). The relation extracted from a function-space element is monotone in both coordinates.

Proof. Use function-space entailment to strengthen inputs and weaken outputs inside the element. ?

Theorem (`element_toApproxMap.rel_cod`). Related outputs are consistent.

Proof. Membership of a fun token forces consistency of its output. ?

Theorem (`element_toApproxMap.rel_dom`). Related inputs are consistent.

Proof. Membership of a fun token forces consistency of its input. ?

Theorem (`element_toApproxMap.union_right`). The extracted relation is closed under finite union on the right.

Proof. Combine tokens via function-space consistency and entailment. ?

Definition (`element_toApproxMap`). The approximable map associated to a function-space element relates u to v precisely when the fun token (u, v) lies in the element.

Theorem (`element_toApproxMap.approxMap_toElement`). Passing from an approximable map to its element and back recovers the original map.

Proof. Compare relations pointwise via membership in the associated element. ?

Theorem (`mem_approxMap_toElement`). A fun token lies in the element of an approximable map precisely when the map relates its input to its output.

Proof. Unfold the carrier of the associated element. ?

Theorem (`approxMap_toElement_element_toApproxMap`). Passing from a function-space element to its approximable map and back recovers the original element.

Proof. Compare carriers via the membership characterisation of the associated element. ?

Theorem (`elements_equiv_approxMaps`). Approximable maps from $\mathbf{A}$ to $\mathbf{B}$ are in bijection with elements of $\mathbf{A} \to \mathbf{B}$: each direction of the round-trip is the identity.

Proof. Assemble the two round-trip lemmas for maps and for elements. ?

approxMap_toElement.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    functionSystem
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ApproximableMap
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element
import DomainTheory.InformationSystems.ElementsOfASystem.Factoid32.factoid_3_2
import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty

@[expose] public section

namespace DomainTheory.InformationSystems

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

theorem mkFunToken.property (u : Finset α) (v : Finset β) (hu : u ∈ A.Con) (hv : v ∈ B
    .Con) :
    u ∈ A.Con ∧ v ∈ B.Con :=
  ⟨hu, hv⟩

end InfoSys
end

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

def mkFunToken (u : Finset α) (v : Finset β) (hu : u ∈ A.Con) (hv : v ∈ B.Con) :
    FunToken A B :=
  ⟨(u, v), mkFunToken.property A B u v hu hv⟩
```

```
end InfoSys
end

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

theorem mkFunToken.eq (p : FunToken A B) :
    mkFunToken A B p.val.1 p.val.2 p.property.1 p.property.2 = p :=
  Subtype.ext rfl

end InfoSys
end

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

theorem rel_input_output_union (f : ApproximableMap A B)
    (s : Finset (FunToken A B))
    (hrel : ∀ q ∈ s, f.rel q.val.1 q.val.2)
    (hin : funInputUnion A B s ∈ A.Con) :
    f.rel (funInputUnion A B s) (funOutputUnion A B s) := by
  induction s using Finset.induction_on with
  | empty =>
    rw [funInputUnion.empty, funOutputUnion.empty]
    exact f.empty_rel
  | insert q s' _hq ih =>
    have hrel' : ∀ r ∈ s', f.rel r.val.1 r.val.2 := fun r hr =>
      hrel r (Finset.mem_insert_of_mem hr)
    have hin' : funInputUnion A B s' ∈ A.Con :=
      A.con_subset hin (by
        rw [funInputUnion.insert]
        exact funion.subset_right _ _)
    have ih' := ih hrel' hin'
    have hqrel : f.rel q.val.1 q.val.2 := hrel q (Finset.mem_insert_self q s')
    have hinU : q.val.1 ∪' funInputUnion A B s' ∈ A.Con := by
      rwa [funInputUnion.insert] at hin
    have hEnt_q : A.EntSet (q.val.1 ∪' funInputUnion A B s') q.val.1 :=
      fun x hx => A.ent_refl hinU (funion.subset_left _ _ hx)
```

```
    have hEnt_s : A.EntSet (q.val.1 ∪' funInputUnion A B s') (funInputUnion A B s') :=
      fun x hx => A.ent_refl hinU (funion.subset_right _ _ hx)
    have h1 : f.rel (q.val.1 ∪' funInputUnion A B s') q.val.2 :=
      f.mono hqrel hEnt_q (EntSet.refl B q.property.2) hinU q.property.2
    have h2 : f.rel (q.val.1 ∪' funInputUnion A B s') (funOutputUnion A B s') :=
      f.mono ih' hEnt_s (EntSet.refl B (f.rel_cod ih')) hinU (f.rel_cod ih')
    rw [funInputUnion.insert, funOutputUnion.insert]
    exact f.union_right h1 h2

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem funCon_of_approxMap (f : ApproximableMap A B) (w : Finset (FunToken A B))
    (hw : ∀ p ∈ w, f.rel p.val.1 p.val.2) : FunCon A B w :=
  fun s hs hin =>
    f.rel_cod (rel_input_output_union A B f s (fun q hq => hw q (hs hq)) hin)

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems
open DomainTheory.InformationSystems.Constructive

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem approxMap_toElement.closed (f : ApproximableMap A B)
    (Y : Finset (FunToken A B)) (p : FunToken A B)
    (hY : (Y : Set (FunToken A B)) ⊆ {p : FunToken A B | f.rel p.val.1 p.val.2})
    (hEnt : (functionSystem A B).Ent Y p) :
```

```
    p ∈ {q : FunToken A B | f.rel q.val.1 q.val.2} := by
  obtain ⟨_hCon, s, hs, hEntIn, hEntOut⟩ := hEnt
  have hrel : ∀ q ∈ s, f.rel q.val.1 q.val.2 := fun q hq =>
    hY (Finset.mem_coe.2 (hs hq))
  have hin_s : funInputUnion A B s ∈ A.Con :=
    A.con_subset
      (EntSet.union_mem A p.property.1 (entSet_inputUnion_of_ent A B hEntIn))
      (funion.subset_right _ _)
  have hIO := rel_input_output_union A B f s hrel hin_s
  have hU : f.rel p.val.1 (funOutputUnion A B s) :=
    f.mono hIO (entSet_inputUnion_of_ent A B hEntIn)
      (EntSet.refl B (f.rel_cod hIO)) p.property.1 (f.rel_cod hIO)
  exact f.mono hU (EntSet.refl A p.property.1) hEntOut p.property.1 p.property.2

end InfoSys
end

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem approxMap_toElement.consistent (f : ApproximableMap A B)
    (Y : Finset (FunToken A B))
    (hY : (Y : Set (FunToken A B)) ⊆ {p : FunToken A B | f.rel p.val.1 p.val.2}) :
    Y ∈ (functionSystem A B).Con :=
  funCon_of_approxMap A B f Y fun _ hp => hY (Finset.mem_coe.2 hp)

end InfoSys
end

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

def approxMap_toElement (f : ApproximableMap A B) : (functionSystem A B).Element where
  carrier := {p : FunToken A B | f.rel p.val.1 p.val.2}
  consistent := approxMap_toElement.consistent A B f
  closed := approxMap_toElement.closed A B f

end InfoSys
```

```
end

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem mem_approxMap_toElement (f : ApproximableMap A B) {p : FunToken A B} :
    p ∈ (approxMap_toElement A B f).carrier ↔ f.rel p.val.1 p.val.2 :=
  Iff.rfl

end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems
open DomainTheory.InformationSystems.Constructive

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem element_toApproxMap.empty_rel (x : (functionSystem A B).Element) :
    ∃ (hu : (∅ : Finset α) ∈ A.Con) (hv : (∅ : Finset β) ∈ B.Con),
      mkFunToken A B ∅ ∅ hu hv ∈ x.carrier := by
  refine ⟨A.con_empty, B.con_empty, ?_⟩
  change funBot A B ∈ x.carrier
  exact factoid_3_2 (functionSystem A B) x

end InfoSys
end

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
```

```
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem element_toApproxMap.mono (x : (functionSystem A B).Element) :
    ∀ {u u' : Finset α} {v v' : Finset β},
      (∃ (hu : u ∈ A.Con) (hv : v ∈ B.Con), mkFunToken A B u v hu hv ∈ x.carrier) →
      A.EntSet u' u → B.EntSet v v' → u' ∈ A.Con → v' ∈ B.Con →
      ∃ (hu : u' ∈ A.Con) (hv : v' ∈ B.Con),
        mkFunToken A B u' v' hu hv ∈ x.carrier := by
  rintro u u' v v' ⟨hu, hv, hp⟩ hEntu hEntv hu' hv'
  refine ⟨hu', hv', ?_⟩
  let p := mkFunToken A B u v hu hv
  let r := mkFunToken A B u' v' hu' hv'
  have hwsub : (↑({p} : Finset (FunToken A B)) : Set _) ⊆ x.carrier := by
    intro t ht
    have : t = p := Finset.mem_singleton.mp (Finset.mem_coe.1 ht)
    subst this
    exact hp
  have hEnt : FunEnt A B {p} r := by
    refine ⟨x.consistent {p} hwsub, {p}, Finset.Subset.refl _, ?_, ?_⟩
    · intro t ht
      have : t = p := Finset.mem_singleton.mp ht
      subst this
      exact hEntu
    · rw [funOutputUnion.singleton]
      exact hEntv
  exact x.closed {p} r hwsub hEnt

end InfoSys
end

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem element_toApproxMap.rel_cod (x : (functionSystem A B).Element) :
    ∀ {u : Finset α} {v : Finset β},
      (∃ (hu : u ∈ A.Con) (hv : v ∈ B.Con), mkFunToken A B u v hu hv ∈ x.carrier) →
      v ∈ B.Con :=
  fun ⟨_, hv, _⟩ => hv

end InfoSys
end

section
```

```
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem element_toApproxMap.rel_dom (x : (functionSystem A B).Element) :
    ∀ {u : Finset α} {v : Finset β},
      (∃ (hu : u ∈ A.Con) (hv : v ∈ B.Con), mkFunToken A B u v hu hv ∈ x.carrier) →
      u ∈ A.Con :=
  fun ⟨hu, _, _⟩ => hu

end InfoSys
end

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem element_toApproxMap.union_right (x : (functionSystem A B).Element) :
    ∀ {u : Finset α} {v v' : Finset β},
      (∃ (hu : u ∈ A.Con) (hv : v ∈ B.Con), mkFunToken A B u v hu hv ∈ x.carrier) →
      (∃ (hu : u ∈ A.Con) (hv : v' ∈ B.Con), mkFunToken A B u v' hu hv ∈ x.carrier) →
      ∃ (hu : u ∈ A.Con) (hv : v ∪' v' ∈ B.Con),
        mkFunToken A B u (v ∪' v') hu hv ∈ x.carrier := by
  rintro u v v' ⟨hu, hv, hp⟩ ⟨hu', hv', hq⟩
  have hvU : v ∪' v' ∈ B.Con := by
    have hCon : FunCon A B
        (insert (mkFunToken A B u v' hu' hv') {mkFunToken A B u v hu hv}) := by
      apply x.consistent
      intro r hr
      rcases Finset.mem_insert.mp (Finset.mem_coe.1 hr) with rfl | hr
      · exact hq
      · have : r = mkFunToken A B u v hu hv := Finset.mem_singleton.mp hr
        subst this
        exact hp
    have hin : funInputUnion A B
        (insert (mkFunToken A B u v' hu' hv') {mkFunToken A B u v hu hv}) ∈ A.Con :=
    by
      rw [funInputUnion.insert, funInputUnion.singleton]
      change u ∪' u ∈ A.Con
      rw [funion.self]
      exact hu
```

```
    have hout := hCon _ (Finset.Subset.refl _) hin
    rw [funOutputUnion.insert, funOutputUnion.singleton, funion.comm] at hout
    exact hout
  refine ⟨hu, hvU, ?_⟩
  let p := mkFunToken A B u v hu hv
  let q := mkFunToken A B u v' hu' hv'
  let r := mkFunToken A B u (v ∪' v') hu hvU
  let w : Finset (FunToken A B) := insert q {p}
  have hwsub : (↑w : Set _) ⊆ x.carrier := by
    intro t ht
    rcases Finset.mem_insert.mp (Finset.mem_coe.1 ht) with rfl | ht
    · exact hq
    · have : t = p := Finset.mem_singleton.mp ht
      subst this
      exact hp
  have hEnt : FunEnt A B w r := by
    refine ⟨x.consistent w hwsub, w, Finset.Subset.refl _, ?_, ?_⟩
    · intro t ht
      rcases Finset.mem_insert.mp ht with rfl | ht
      · exact EntSet.refl A hu'
      · have : t = p := Finset.mem_singleton.mp ht
        subst this


        exact EntSet.refl A hu
    · have houtEq : funOutputUnion A B w = v' ∪' v := by
        change funOutputUnion A B
          (insert (mkFunToken A B u v' hu' hv') {mkFunToken A B u v hu hv}) = v' ∪' v
        rw [funOutputUnion.insert, funOutputUnion.singleton]
        simp only [mkFunToken]
      rw [houtEq, funion.comm]
      exact EntSet.refl B hvU
  exact x.closed w r hwsub hEnt

end InfoSys
end

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

def element_toApproxMap (x : (functionSystem A B).Element) : ApproximableMap A B where
  rel u v := ∃ (hu : u ∈ A.Con) (hv : v ∈ B.Con), mkFunToken A B u v hu hv ∈ x.carrier
  rel_dom := element_toApproxMap.rel_dom A B x
  rel_cod := element_toApproxMap.rel_cod A B x
  empty_rel := element_toApproxMap.empty_rel A B x
  union_right := element_toApproxMap.union_right A B x
  mono := element_toApproxMap.mono A B x
```

```
end InfoSys
end

section
namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem element_toApproxMap.approxMap_toElement (f : ApproximableMap A B) :
    element_toApproxMap A B (InfoSys.approxMap_toElement A B f) = f := by
  refine ApproximableMap.ext fun u v => ?_
  constructor
  · rintro ⟨hu, hv, hp⟩
    change f.rel u v at hp
    exact hp
  · intro hrel
    exact ⟨f.rel_dom hrel, f.rel_cod hrel, by
      change f.rel u v
      exact hrel⟩

end InfoSys
end

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem approxMap_toElement_element_toApproxMap (x : (functionSystem A B).Element) :
    approxMap_toElement A B (element_toApproxMap A B x) = x := by
  apply le_antisymm
  · intro p hp
    change (element_toApproxMap A B x).rel p.val.1 p.val.2 at hp
    obtain ⟨_, _, hmem⟩ := hp
    simpa [mkFunToken.eq A B p] using hmem
  · intro p hp
    change (element_toApproxMap A B x).rel p.val.1 p.val.2
    refine ⟨p.property.1, p.property.2, ?_⟩
    simpa [mkFunToken.eq A B p] using hp
```

```
end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem elements_equiv_approxMaps {α : Type u} {β : Type v} [DecidableEq α] [
    DecidableEq β]
    (A : InfoSys α) (B : InfoSys β) :
    (∀ f : ApproximableMap A B,
      element_toApproxMap A B (approxMap_toElement A B f) = f) ∧
    (∀ x : (functionSystem A B).Element,
      approxMap_toElement A B (element_toApproxMap A B x) = x) :=
  ⟨element_toApproxMap.approxMap_toElement A B,
    approxMap_toElement_element_toApproxMap A B⟩

end InfoSys
end DomainTheory.InformationSystems
```

5.8.6 Curry map

Definition (`curryMap`). The curry of a map $h : \mathbf{A} \times \mathbf{B} \to \mathbf{C}$ relates u to a finite set s of fun tokens when u is consistent and h relates the lift of u with each input of a token in s to that token's output.

Theorem (`rel_of_funTokens`). A map out of a product that relates a fixed left component to each token of a finite set of fun tokens relates that left component paired with the input union to the output union.

Proof. Assemble the product lifts and apply the map's union and monotonicity axioms. ?

Theorem (`funCon_of_curry_pointwise`). For a fixed consistent left component, the set of fun tokens related pointwise by a product map is consistent in the function space.

Proof. Reduce a consistent input selection to a product relation and invoke consistency of the product map's outputs. ?

Definition (`applyMap`). Application is the approximable map from $(\mathbf{B} \to \mathbf{C}) \times \mathbf{B}$ to $\mathbf{C}$ that relates a product-consistent set s to t when the function-space entailment encoded by s yields t from the second projection.

Theorem (`applyMap_toElement`). Evaluating apply at a paired element (g, y) recovers the application $g(y)$ of the approximable map associated to g.

Proof. Chase membership through apply, pairing, and the element–map round-trips. ?

Definition (`uncurryMap`). The uncurry of a map into a function space is apply composed with pairing against the identity on the left factor.

Theorem (`uncurry_curryMap`). Uncurry after curry recovers the original product map.

Proof. Compare relations by unpacking curry's pointwise condition against apply of the paired lifts. ?

Theorem (`curryMap_unique`). If uncurry of k equals h, then k equals the curry of h.

Proof. From the universal property of uncurry–curry, compare k to curry h via the round-trip. ?

curryMap.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    liftPair
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    functionSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    productSystem
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    approxMap_toElement
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.pairMap
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    pairElements
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement
import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty
import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    productSystem_more

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem rel_of_funTokens (h : ApproximableMap (productSystem A B) C)
    (u : Finset α) (hu : u ∈ A.Con) (t : Finset (FunToken B C))
    (hps : ∀ p ∈ t, h.rel (liftPair A B u p.val.1) p.val.2)
    (hin : funInputUnion B C t ∈ B.Con) :
    h.rel (liftPair A B u (funInputUnion B C t)) (funOutputUnion B C t) := by
  induction t using Finset.induction_on with
  | empty =>
    rw [funInputUnion.empty, funOutputUnion.empty]
    have hdom : liftPair A B u ∅ ∈ (productSystem A B).Con :=
      ProdCon_liftPair A B hu B.con_empty
    exact h.mono h.empty_rel
      (EntSet.empty (productSystem A B) _)
      (EntSet.empty C ∅) hdom C.con_empty
  | insert p t' hpnot ih =>
```

```
    have hps' : ∀ q ∈ t', h.rel (liftPair A B u q.val.1) q.val.2 :=
      fun q hq => hps q (Finset.mem_insert_of_mem hq)
    have hin' : funInputUnion B C t' ∈ B.Con :=
      B.con_subset hin (by
        rw [funInputUnion.insert]
        exact funion.subset_right _ _)
    have ih' := ih hps' hin'
    have hp : h.rel (liftPair A B u p.val.1) p.val.2 :=
      hps p (Finset.mem_insert_self p t')
    have hinU : p.val.1 ∪' funInputUnion B C t' ∈ B.Con := by
      rwa [funInputUnion.insert] at hin
    have hEnt_p : B.EntSet (p.val.1 ∪' funInputUnion B C t') p.val.1 :=
      fun y hy => B.ent_refl hinU (funion.subset_left _ _ hy)
    have hEnt_t : B.EntSet (p.val.1 ∪' funInputUnion B C t') (funInputUnion B C t') :=
      fun y hy => B.ent_refl hinU (funion.subset_right _ _ hy)
    have hdomU : liftPair A B u (p.val.1 ∪' funInputUnion B C t') ∈
        (productSystem A B).Con :=
      ProdCon_liftPair A B hu hinU
    have h1 : h.rel (liftPair A B u (p.val.1 ∪' funInputUnion B C t')) p.val.2 :=
      h.mono hp
        (entSet_liftPair A B hu hinU (EntSet.refl A hu) hEnt_p)
        (EntSet.refl C (h.rel_cod hp)) hdomU (h.rel_cod hp)
    have h2 : h.rel (liftPair A B u (p.val.1 ∪' funInputUnion B C t'))
        (funOutputUnion B C t') :=
      h.mono ih'
        (entSet_liftPair A B hu hinU (EntSet.refl A hu) hEnt_t)
        (EntSet.refl C (h.rel_cod ih')) hdomU (h.rel_cod ih')
    rw [funInputUnion.insert, funOutputUnion.insert]
    exact h.union_right h1 h2

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem funCon_of_curry_pointwise (h : ApproximableMap (productSystem A B) C)
    {u : Finset α} (hu : u ∈ A.Con) {s : Finset (FunToken B C)}
    (hps : ∀ p ∈ s, h.rel (liftPair A B u p.val.1) p.val.2) :
    FunCon B C s :=
  fun t ht hin =>
    h.rel_cod (rel_of_funTokens A B C h u hu t (fun p hp => hps p (ht hp)) hin)
```

```
end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

def curryMap (h : ApproximableMap (productSystem A B) C) :
    ApproximableMap A (functionSystem B C) where
  rel u s :=
    u ∈ A.Con ∧ ∀ p ∈ s, h.rel (liftPair A B u p.val.1) p.val.2
  rel_dom hrel := hrel.1
  rel_cod hrel := funCon_of_curry_pointwise A B C h hrel.1 hrel.2
  empty_rel := ⟨A.con_empty, fun _ hp => False.elim (Finset.notMem_empty _ hp)⟩
  union_right := by
    rintro u s s' ⟨hu, hps⟩ ⟨_, hps'⟩
    refine ⟨hu, ?_⟩
    intro p hp
    rcases funion.mem.mp hp with hp | hp
    · exact hps p hp
    · exact hps' p hp
  mono := by
    rintro u u' s s' ⟨hu, hps⟩ hEntu hEnts hu' hs'
    refine ⟨hu', ?_⟩
    intro p hp

    have hEntp : FunEnt B C s p := hEnts p hp
    obtain ⟨_, t, ht, hIn, hOut⟩ := hEntp
    have hps_t : ∀ q ∈ t, h.rel (liftPair A B u q.val.1) q.val.2 :=
      fun q hq => hps q (ht hq)
    have hin : funInputUnion B C t ∈ B.Con :=
      B.con_subset
        (EntSet.union_mem B p.property.1 (entSet_inputUnion_of_ent B C hIn))
        (funion.subset_right _ _)
    have hmid : h.rel (liftPair A B u (funInputUnion B C t)) (funOutputUnion B C t) :=
      rel_of_funTokens A B C h u hu t hps_t hin
    have hmid' : h.rel (liftPair A B u' p.val.1) (funOutputUnion B C t) :=
      h.mono hmid
        (entSet_liftPair A B hu' p.property.1 hEntu
          (entSet_inputUnion_of_ent B C hIn))
        (EntSet.refl C (h.rel_cod hmid))
        (ProdCon_liftPair A B hu' p.property.1) (h.rel_cod hmid)
    exact h.mono hmid'
      (EntSet.refl (productSystem A B) (ProdCon_liftPair A B hu' p.property.1))
```

```
      hOut (ProdCon_liftPair A B hu' p.property.1) p.property.2

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

def applyMap : ApproximableMap (productSystem (functionSystem B C) B) C where
  rel s t :=
    ∃ (hs : ProdCon (functionSystem B C) B s) (ht : t ∈ C.Con),
      FunEnt B C (fstFinset (functionSystem B C) B s)
        (mkFunToken B C (sndFinset (functionSystem B C) B s) t hs.2 ht)
  rel_dom := fun ⟨hs, _, _⟩ => hs
  rel_cod := fun ⟨_, ht, _⟩ => ht
  empty_rel := by
    have hs : ProdCon (functionSystem B C) B ∅ :=
      ⟨by rw [fstFinset.empty]; exact (functionSystem B C).con_empty,
        by rw [sndFinset.empty]; exact B.con_empty⟩
    refine ⟨hs, C.con_empty, ?_⟩
    have hfst : fstFinset (functionSystem B C) B (∅ : Finset _) = ∅ := fstFinset.empty
    _ _
    have hsnd : sndFinset (functionSystem B C) B (∅ : Finset _) = ∅ := sndFinset.empty
    _ _
    simp only [hfst, hsnd]
    exact (functionSystem B C).ent_bot (functionSystem B C).con_empty
  union_right := by
    rintro s t t' ⟨hs, ht, hEntt⟩ ⟨_, ht', hEntt'⟩
    obtain ⟨hw, s1, hs1, hIn1, hOut1⟩ := hEntt
    obtain ⟨_, s2, hs2, hIn2, hOut2⟩ := hEntt'
    have hsub : s1 ∪' s2 ⊆ fstFinset (functionSystem B C) B s :=
      funion.subset_iff.mpr ⟨hs1, hs2⟩
    have hin : funInputUnion B C (s1 ∪' s2) ∈ B.Con := by
      rw [funInputUnion.funion]
      have hEntU : B.EntSet (sndFinset (functionSystem B C) B s)
          (funInputUnion B C s1 ∪' funInputUnion B C s2) :=
        EntSet.union B
          (entSet_inputUnion_of_ent B C hIn1)
          (entSet_inputUnion_of_ent B C hIn2)
      exact B.con_subset (EntSet.union_mem B hs.2 hEntU) (funion.subset_right _ _)
    have hout : funOutputUnion B C s1 ∪' funOutputUnion B C s2 ∈ C.Con := by
      have := hw _ hsub hin
      rwa [funOutputUnion.funion] at this
```

```
    have htt' : t ∪' t' ∈ C.Con := by
      have hEntt2 : C.EntSet (funOutputUnion B C s1 ∪' funOutputUnion B C s2) (t ∪' t
    ') :=
        EntSet.union C
          (EntSet.mono C (C.con_subset hout (funion.subset_left _ _)) hout
            (funion.subset_left _ _) hOut1 (Finset.Subset.refl _))
          (EntSet.mono C (C.con_subset hout (funion.subset_right _ _)) hout
            (funion.subset_right _ _) hOut2 (Finset.Subset.refl _))
      exact C.con_subset (EntSet.union_mem C hout hEntt2) (funion.subset_right _ _)
    refine ⟨hs, htt', hw, s1 ∪' s2, hsub, ?_, ?_⟩
    · intro q hq
      rcases funion.mem.mp hq with hq | hq
      · exact hIn1 q hq
      · exact hIn2 q hq
    · rw [funOutputUnion.funion]
      exact EntSet.union C
        (EntSet.mono C (C.con_subset hout (funion.subset_left _ _)) hout
          (funion.subset_left _ _) hOut1 (Finset.Subset.refl _))
        (EntSet.mono C (C.con_subset hout (funion.subset_right _ _)) hout
          (funion.subset_right _ _) hOut2 (Finset.Subset.refl _))
  mono := by
    rintro s s' t t' ⟨hs, ht, hEnt⟩ hEnts hEntt hs' ht'
    refine ⟨hs', ht', ?_⟩
    have hfst := ProdEnt.entSet_fst (functionSystem B C) B hEnts
    have hsnd := ProdEnt.entSet_snd (functionSystem B C) B hEnts
    have htok :
        (functionSystem B C).Ent (fstFinset (functionSystem B C) B s)
          (mkFunToken B C (sndFinset (functionSystem B C) B s) t hs.2 ht) := hEnt
    have hmid :
        (functionSystem B C).Ent (fstFinset (functionSystem B C) B s')
          (mkFunToken B C (sndFinset (functionSystem B C) B s) t hs.2 ht) :=
      (functionSystem B C).ent_trans hs'.1 hs.1 (fun y hy => hfst y hy) htok
    obtain ⟨hw', s1, hs1, hIn1, hOut1⟩ := hmid
    have hIn1' : ∀ q ∈ s1, B.EntSet (sndFinset (functionSystem B C) B s') q.val.1 :=
      fun q hq => EntSet.trans B hs'.2 hs.2 hsnd (hIn1 q hq)
    have hin : funInputUnion B C s1 ∈ B.Con :=
      B.con_subset
        (EntSet.union_mem B hs'.2 (entSet_inputUnion_of_ent B C hIn1'))
        (funion.subset_right _ _)
    have hout : funOutputUnion B C s1 ∈ C.Con := hw' s1 hs1 hin
    refine ⟨hw', s1, hs1, hIn1', ?_⟩
    exact EntSet.trans C hout ht hOut1 hEntt

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
```

```
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]

variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

def uncurryMap (k : ApproximableMap A (functionSystem B C)) :
    ApproximableMap (productSystem A B) C :=
  comp (applyMap (B := B) (C := C))
    (pairMap (functionSystem B C) B (comp k (productSystem.fstMap A B)) (productSystem
    .sndMap A B))

end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem uncurry_curryMap (h : ApproximableMap (productSystem A B) C) :
    uncurryMap A B C (curryMap A B C h) = h := by
  refine ApproximableMap.ext fun s w => ?_
  constructor
  · rintro ⟨m, ⟨hk, hsnd⟩, ⟨hCon, hw, hEnt⟩⟩

    obtain ⟨v, hfst, ⟨hv, hcurry⟩⟩ := hk
    obtain ⟨_, t, ht, hIn, hOut⟩ := hEnt

    have hps : ∀ q ∈ t, h.rel (liftPair A B v q.val.1) q.val.2 :=
      fun q hq => hcurry q (ht hq)
    have hin : funInputUnion B C t ∈ B.Con :=
      B.con_subset
        (EntSet.union_mem B hCon.2 (entSet_inputUnion_of_ent B C hIn))
        (funion.subset_right _ _)
    have hmid : h.rel (liftPair A B v (funInputUnion B C t)) (funOutputUnion B C t) :=
      rel_of_funTokens A B C h v hv t hps hin

    have hEntIn : B.EntSet (sndFinset A B s) (funInputUnion B C t) :=
      EntSet.trans B (hsnd.1.2) hin
        (EntSet.trans B (hsnd.1.2) hCon.2 hsnd.2.2
          (entSet_inputUnion_of_ent B C hIn))
        (EntSet.refl B hin)

    have hs : s ∈ (productSystem A B).Con := hfst.1
    have hEntv : A.EntSet (fstFinset A B s) v := hfst.2.2
```

```
  have hEntsnd : B.EntSet (sndFinset A B s) (sndFinset (functionSystem B C) B m) :=
  hsnd.2.2
  have hEntIn' : B.EntSet (sndFinset A B s) (funInputUnion B C t) :=
    EntSet.trans B hs.2 hin
      (EntSet.trans B hs.2 hCon.2 hEntsnd (entSet_inputUnion_of_ent B C hIn))
      (EntSet.refl B hin)
  have hdom : liftPair A B (fstFinset A B s) (sndFinset A B s) ∈
      (productSystem A B).Con :=
    ProdCon_liftPair A B hs.1 hs.2

  have hmid' : h.rel (liftPair A B (fstFinset A B s) (sndFinset A B s))
      (funOutputUnion B C t) :=
    h.mono hmid (entSet_liftPair A B hs.1 hs.2 hEntv hEntIn')
      (EntSet.refl C (h.rel_cod hmid)) hdom (h.rel_cod hmid)

  have hEntLift : (productSystem A B).EntSet s (liftPair A B (fstFinset A B s) (
  sndFinset A B s)) := by
    intro p hp
    unfold liftPair at hp
    rcases funion.mem.mp hp with hp | hp
    · rcases (liftLeft.mem A B).1 hp with ⟨a, ha, rfl⟩
      refine ⟨hs, ?_, ?_⟩
      · intro _
        exact A.ent_refl hs.1 ha
      · intro _
        exact B.ent_bot hs.2
    · rcases (liftRight.mem A B).1 hp with ⟨b, hb, rfl⟩
      refine ⟨hs, ?_, ?_⟩
      · intro _
        exact A.ent_bot hs.1
      · intro _
        exact B.ent_refl hs.2 hb
  exact h.mono hmid' hEntLift hOut hs hw
· intro hh
  have hs : s ∈ (productSystem A B).Con := h.rel_dom hh
  have hw : w ∈ C.Con := h.rel_cod hh
  let tok : FunToken B C := mkFunToken B C (sndFinset A B s) w hs.2 hw
  have hLiftCon : ProdCon A B (liftPair A B (fstFinset A B s) (sndFinset A B s)) :=
    ProdCon_liftPair A B hs.1 hs.2
  have hEntLift : (productSystem A B).EntSet
      (liftPair A B (fstFinset A B s) (sndFinset A B s)) s := by
    intro p hp
    rcases p.property with hR | hL
    · refine ⟨hLiftCon, fun _ => ?_, fun _ => ?_⟩
      · rw [hR]; exact A.ent_bot hLiftCon.1
      · have hb : p.val.2 ∈ sndFinset A B s :=
          (mem_sndFinset A B).2 ⟨p, hp, hR, rfl⟩
        have hb' : p.val.2 ∈
            sndFinset A B (liftPair A B (fstFinset A B s) (sndFinset A B s)) := by
          rw [sndFinset_liftPair]
          exact funion.subset_right _ _ hb
        exact B.ent_refl hLiftCon.2 hb'
    · refine ⟨hLiftCon, fun _ => ?_, fun _ => ?_⟩
      · have ha : p.val.1 ∈ fstFinset A B s :=
          (mem_fstFinset A B).2 ⟨p, hp, hL, rfl⟩
        have ha' : p.val.1 ∈
```

```
          fstFinset A B (liftPair A B (fstFinset A B s) (sndFinset A B s)) := by
        rw [fstFinset_liftPair]
        exact funion.subset_left _ _ ha
      exact A.ent_refl hLiftCon.1 ha'
    · rw [hL]; exact B.ent_bot hLiftCon.2
have hrel : h.rel (liftPair A B (fstFinset A B s) (sndFinset A B s)) w :=
  h.mono hh hEntLift (EntSet.refl C hw) hLiftCon hw
have hcurry_tok : (curryMap A B C h).rel (fstFinset A B s) {tok} := by
  refine ⟨hs.1, ?_⟩
  intro p hp
  have : p = tok := Finset.mem_singleton.mp hp
  subst this
  exact hrel
let m := liftPair (functionSystem B C) B {tok} (sndFinset A B s)
have hmCon : ProdCon (functionSystem B C) B m :=
  ProdCon_liftPair (functionSystem B C) B ((functionSystem B C).con_sing tok) hs.2
have hfst_m : fstFinset (functionSystem B C) B m =
    {tok} ∪' fstFinset (functionSystem B C) B
      (liftRight (functionSystem B C) B (sndFinset A B s)) := by
  change fstFinset _ _ (liftLeft _ _ {tok} ∪' liftRight _ _ _) = _
  rw [fstFinset.funion, liftLeft.fstFinset_eq]
have hsnd_m : sndFinset (functionSystem B C) B m =
    sndFinset (functionSystem B C) B (liftLeft (functionSystem B C) B {tok}) ∪'
      sndFinset A B s := by
  change sndFinset _ _ (liftLeft _ _ {tok} ∪' liftRight _ _ _) = _
  rw [sndFinset.funion, liftRight.sndFinset_eq]
have hcurry_fst : (curryMap A B C h).rel (fstFinset A B s)


    (fstFinset (functionSystem B C) B m) := by
  refine ⟨hs.1, ?_⟩
  intro p hp
  have hp' : p ∈ {tok} ∪' fstFinset (functionSystem B C) B
      (liftRight (functionSystem B C) B (sndFinset A B s)) := by
    rw [hfst_m] at hp
    exact hp
  rcases funion.mem.mp hp' with hp' | hp'
  · have : p = tok := Finset.mem_singleton.mp hp'
    subst this
    exact hrel
  · have hpbot : p = (functionSystem B C).bot :=
      Finset.mem_singleton.mp
        (liftRight.fstFinset_eq (functionSystem B C) B (sndFinset A B s) hp')
    subst hpbot
    change h.rel (liftPair A B (fstFinset A B s) (funBot B C).val.1) (funBot B C).
val.2
    simp only [funBot]
    exact h.mono h.empty_rel (EntSet.empty (productSystem A B) _)
      (EntSet.empty C ∅) (ProdCon_liftPair A B hs.1 B.con_empty) C.con_empty
refine ⟨m, ⟨?_, ?_⟩, ?_⟩
·
  exact ⟨fstFinset A B s, ⟨hs, hs.1, EntSet.refl A hs.1⟩, hcurry_fst⟩
·
  refine ⟨hs, hmCon.2, ?_⟩
  intro b hb
  have hb' : b ∈
      sndFinset (functionSystem B C) B (liftLeft (functionSystem B C) B {tok}) ∪'
```

```
            sndFinset A B s := by
          rwa [hsnd_m] at hb
        rcases funion.mem.mp hb' with hb' | hb'
        · have : b = B.bot := Finset.mem_singleton.mp
            (liftLeft.sndFinset_eq (functionSystem B C) B {tok} hb')
          subst this
          exact B.ent_bot hs.2
        · exact B.ent_refl hs.2 hb'
      ·
        refine ⟨hmCon, hw, ?_⟩
        refine ⟨(curryMap A B C h).rel_cod hcurry_fst, {tok}, ?_, ?_, ?_⟩
        · intro q hq
          have : q = tok := Finset.mem_singleton.mp hq
          subst this
          have : tok ∈ fstFinset (functionSystem B C) B m := by
            rw [hfst_m]
            exact funion.subset_left _ _ (Finset.mem_singleton_self _)
          exact this
        · intro q hq
          have : q = tok := Finset.mem_singleton.mp hq
          subst this
          intro b hb
          have : b ∈ sndFinset (functionSystem B C) B m := by
            rw [hsnd_m]
            exact funion.subset_right _ _ hb
          exact B.ent_refl hmCon.2 this
        · rw [funOutputUnion.singleton]
          exact EntSet.refl C hw

end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem applyMap_toElement (g : (functionSystem B C).Element) (y : B.Element) :
    (applyMap (B := B) (C := C)).toElement
      (pairElements (functionSystem B C) B g y) =
      (element_toApproxMap B C g).toElement y := by
  apply le_antisymm
  · intro Z ⟨s, hs, ⟨hCon, hZcon, hEnt⟩⟩
    obtain ⟨hw, t, ht, hIn, hOut⟩ := hEnt
    have ht_sub : ↑t ⊆ g.carrier := by
      intro p hp
      exact mem_fst_of_subset_pairElements (functionSystem B C) B hs (ht (Finset.
```

```
  mem_coe.1 hp))
  have hsnd_sub : ↑(sndFinset (functionSystem B C) B s) ⊆ y.carrier := by
    intro b hb
    exact mem_snd_of_subset_pairElements (functionSystem B C) B hs hb
  have hin : funInputUnion B C t ∈ B.Con :=
    B.con_subset
      (EntSet.union_mem B hCon.2 (entSet_inputUnion_of_ent B C hIn))
      (funion.subset_right _ _)
  have hin_sub : ↑(funInputUnion B C t) ⊆ y.carrier := by
    intro b hb
    rcases (mem_funInputUnion B C).1 (Finset.mem_coe.1 hb) with ⟨q, hq, hb'⟩
    exact y.closed _ b hsnd_sub (hIn q hq b hb')
  have hEntZ : FunEnt B C t
      (mkFunToken B C (funInputUnion B C t) {Z} hin (C.con_sing Z)) := by
    refine ⟨fun s' hs' hin' => hw s' (fun q hq => ht (hs' hq)) hin', t, Finset.Subset
  .refl _, ?_, hOut⟩
    intro q hq y' hy'
    exact B.ent_refl hin ((mem_funInputUnion B C).2 ⟨q, hq, hy'⟩)
  have hmem : mkFunToken B C (funInputUnion B C t) {Z} hin (C.con_sing Z) ∈ g.
  carrier :=
    g.closed t _ ht_sub hEntZ
  exact ⟨funInputUnion B C t, hin_sub, ⟨hin, C.con_sing Z, hmem⟩⟩
· intro Z ⟨v, hv, ⟨hvCon, hZcon, hmem⟩⟩
  let p : FunToken B C := mkFunToken B C v {Z} hvCon hZcon
  have hp : p ∈ g.carrier := hmem
  let s := liftPair (functionSystem B C) B {p} v
  have hsCon : ProdCon (functionSystem B C) B s :=
    ProdCon_liftPair (functionSystem B C) B ((functionSystem B C).con_sing p) hvCon
  have hs_sub : ↑s ⊆ (pairElements (functionSystem B C) B g y).carrier :=
    subset_pairElements_liftPair (functionSystem B C) B
      (by
        intro q hq
        have : q = p := Finset.mem_singleton.mp (Finset.mem_coe.1 hq)
        subst this
        exact hp)
      hv
  refine ⟨s, hs_sub, ⟨hsCon, C.con_sing Z, ?_⟩⟩
  refine ⟨?_, {p}, ?_, ?_, ?_⟩
  ·
    intro t ht hin
    have ht_sub : ↑t ⊆ g.carrier := by
      intro q hq
      exact mem_fst_of_subset_pairElements (functionSystem B C) B hs_sub (ht hq)
    exact (funCon_of_approxMap B C (element_toApproxMap B C g) t
      (fun q hq => ⟨q.property.1, q.property.2, by
        simpa [mkFunToken.eq] using ht_sub (Finset.mem_coe.2 hq)⟩)) t (Finset.Subset.
  refl _) hin
  ·
    intro q hq
    have : q = p := Finset.mem_singleton.mp hq
    subst this
    rw [show s = liftLeft (functionSystem B C) B {p} ∪' liftRight (functionSystem B
  C) B v from rfl]
    rw [fstFinset.funion, liftLeft.fstFinset_eq]
    exact funion.subset_left _ _ (Finset.mem_singleton_self _)
  ·
```

```
      intro q hq
      have : q = p := Finset.mem_singleton.mp hq
      subst this
      intro b hb
      have hb' : b ∈ v := hb

      have : b ∈ sndFinset (functionSystem B C) B s := by
        rw [sndFinset_liftPair]
        exact funion.subset_right _ _ hb'
      exact B.ent_refl hsCon.2 this
    .
      rw [funOutputUnion.singleton]
      exact EntSet.refl C (C.con_sing Z)

end InfoSys
end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

namespace InfoSys

universe u v w

variable {α : Type u} {β : Type v} {? : Type w}
  [DecidableEq α] [DecidableEq β] [DecidableEq ?]
variable (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)

open ApproximableMap

theorem curryMap_unique (h : ApproximableMap (productSystem A B) C)
    (k : ApproximableMap A (functionSystem B C))
    (huniv : uncurryMap A B C k = h) :
    k = curryMap A B C h := by
  have hk : uncurryMap A B C k = uncurryMap A B C (curryMap A B C h) := by
    rw [huniv, uncurry_curryMap]
  refine (ext_iff_toElement k (curryMap A B C h)).2 fun x => ?_
  have happly : ∀ y : B.Element,
      (element_toApproxMap B C (k.toElement x)).toElement y =
        (element_toApproxMap B C ((curryMap A B C h).toElement x)).toElement y := by
    intro y
    have hpair_k :
        (pairMap (functionSystem B C) B (comp k (productSystem.fstMap A B)) (
    productSystem.sndMap A B)).toElement
          (pairElements A B x y) =
        pairElements (functionSystem B C) B (k.toElement x) y := by
      apply productSystem.element_eq_of_fst_snd (functionSystem B C) B
      · have h1 : (productSystem.fstMap (functionSystem B C) B).toElement
            ((pairMap (functionSystem B C) B (comp k (productSystem.fstMap A B)) (
    productSystem.sndMap A B)).toElement
              (pairElements A B x y)) = k.toElement x := by
          rw [← comp.toElement, pairMap.comp_fst, comp.toElement, fstMap_pairElements]
        exact h1.trans (fstMap_pairElements (functionSystem B C) B (k.toElement x) y).
    symm
      · have h1 : (productSystem.sndMap (functionSystem B C) B).toElement
            ((pairMap (functionSystem B C) B (comp k (productSystem.fstMap A B)) (
    productSystem.sndMap A B)).toElement
```

```
            (pairElements A B x y)) = y := by
          rw [← comp.toElement, pairMap.comp_snd, sndMap_pairElements]
        exact h1.trans (sndMap_pairElements (functionSystem B C) B (k.toElement x) y).
    symm
    have hpair_c :
        (pairMap (functionSystem B C) B
            (comp (curryMap A B C h) (productSystem.fstMap A B)) (productSystem.sndMap
     A B)).toElement
          (pairElements A B x y) =
        pairElements (functionSystem B C) B ((curryMap A B C h).toElement x) y := by
      apply productSystem.element_eq_of_fst_snd (functionSystem B C) B
      · have h1 : (productSystem.fstMap (functionSystem B C) B).toElement
            ((pairMap (functionSystem B C) B
                (comp (curryMap A B C h) (productSystem.fstMap A B)) (productSystem.
    sndMap A B)).toElement
              (pairElements A B x y)) = (curryMap A B C h).toElement x := by
          rw [← comp.toElement, pairMap.comp_fst, comp.toElement, fstMap_pairElements]
        exact h1.trans
          (fstMap_pairElements (functionSystem B C) B ((curryMap A B C h).toElement x)
     y).symm
      · have h1 : (productSystem.sndMap (functionSystem B C) B).toElement
            ((pairMap (functionSystem B C) B
                (comp (curryMap A B C h) (productSystem.fstMap A B)) (productSystem.
    sndMap A B)).toElement
              (pairElements A B x y)) = y := by
          rw [← comp.toElement, pairMap.comp_snd, sndMap_pairElements]
        exact h1.trans
          (sndMap_pairElements (functionSystem B C) B ((curryMap A B C h).toElement x)
     y).symm
    have hu1 : (uncurryMap A B C k).toElement (pairElements A B x y) =
        (applyMap (B := B) (C := C)).toElement
          (pairElements (functionSystem B C) B (k.toElement x) y) := by
      simp only [uncurryMap, comp.toElement, hpair_k]
    have hu2 : (uncurryMap A B C (curryMap A B C h)).toElement (pairElements A B x y)
    =
        (applyMap (B := B) (C := C)).toElement
          (pairElements (functionSystem B C) B ((curryMap A B C h).toElement x) y) :=
    by
      simp only [uncurryMap, comp.toElement, hpair_c]
    have heq : (uncurryMap A B C k).toElement (pairElements A B x y) =
        (uncurryMap A B C (curryMap A B C h)).toElement (pairElements A B x y) := by
      rw [hk]
    rw [← applyMap_toElement, ← applyMap_toElement, ← hu1, ← hu2, heq]
  have hx : element_toApproxMap B C (k.toElement x) =
      element_toApproxMap B C ((curryMap A B C h).toElement x) :=
    (ext_iff_toElement _ _).2 happly
  have : k.toElement x = (curryMap A B C h).toElement x := by
    rw [← approxMap_toElement_element_toApproxMap B C (k.toElement x),
      ← approxMap_toElement_element_toApproxMap B C ((curryMap A B C h).toElement x),
    hx]
  rw [this]

end InfoSys
end DomainTheory.InformationSystems
```

5.8.7 Theorem 7.2: elements, apply, and curry

THEOREM 7.2. If $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ are information systems, then so is $\mathbf{A} \to \mathbf{B}$, and the approximable mappings $f : \mathbf{A} \to \mathbf{B}$ are exactly the elements $f \in |\mathbf{A} \to \mathbf{B}|$. Moreover we have an approximable mapping apply : $(\mathbf{B} \to \mathbf{C}) \times \mathbf{B} \to \mathbf{C}$ such that whenever $g : \mathbf{B} \to \mathbf{C}$ and $y \in |\mathbf{B}|$, then

$$\text{apply}(g, y) = g(y).$$

Furthermore, for all approximable mappings $h : \mathbf{A} \times \mathbf{B} \to \mathbf{C}$, there is one and only one approximable mapping curry $h : \mathbf{A} \to (\mathbf{B} \to \mathbf{C})$ such that

$$h = \text{apply} \circ \langle (\text{curry } h) \circ \text{fst}, \text{snd} \rangle.$$

Proof. We have already remarked on the essentials of the proof above. Definition 7.1 was devised to characterize exactly in $\text{Con}_{\mathbf{A}\to\mathbf{B}}$ the finite subsets of approximable functions, which, as binary relations, are being regarded as sets of ordered pairs. If $f : \mathbf{A} \to \mathbf{B}$ and if $w \subseteq f$, then from the properties of approximable functions, it can be checked directly that w satisfies the right-hand side of 7.1(iii). Conversely, if $w \in \text{Con}_{\mathbf{A}\to\mathbf{B}}$, then, as we have said, the relation which is defined by 7.1(iv) and may be notated by:

$$\overline{w} = \{(u', v') \mid w \vdash_{\mathbf{A}\to\mathbf{B}} (u', v')\},$$

is an approximable mapping, as can be proved using the right-hand side of 7.1(iv) and the usual properties of $\vdash_{\mathbf{A}}$ and $\vdash_{\mathbf{B}}$. Since $w \subseteq \overline{w}$, we see that $w \vdash_{\mathbf{A}\to\mathbf{B}} w'$ if, and only if, for all approximable $f : \mathbf{A} \to \mathbf{B}$, $w \subseteq f$ implies $w' \subseteq f$. (This is also the same as $w' \subseteq \overline{w}$, of course.) From these considerations it follows that not only is $\mathbf{A} \to \mathbf{B}$ an information system, but all approximable mappings are elements. Finally, if $f \in |\mathbf{A} \to \mathbf{B}|$, then—as a binary relation—it must be an approximable mapping, because the properties of Definition 5.1 are built into 7.1.

The construction of the special mapping apply as an approximable mapping also uses the idea of 7.1(iv). The consistent sets of the compound space $(\mathbf{B} \to \mathbf{C}) \times \mathbf{B}$ are essentially pairs of consistent sets, say $w \in \text{Con}_{\mathbf{B}\to\mathbf{C}}$ and $u' \in \text{Con}_{\mathbf{B}}$. Now the relation we want from such pairs to consistent sets $v' \in \text{Con}_{\mathbf{C}}$ is just nothing more or less than $w \vdash_{\mathbf{B}\to\mathbf{C}} (u', v')$. Our discussion in the previous paragraph hints at why apply does in fact reproduce functional application when we evaluate apply(g, y).

The definition of curry h uses the same trick of regarding a binary relation with one term in a relationship being a pair as corresponding to another relation with one coordinate of the pair shifted to the other side. Specifically, we can think of an approximable mapping $h : \mathbf{A} \times \mathbf{B} \to \mathbf{C}$ as a relation from pairs (u, v) of consistent sets for $\mathbf{A}$ and $\mathbf{B}$, respectively, over to consistent sets w for $\mathbf{C}$. What we want for curry h is the relationship that goes from u to the pair (v, w). Of course (v, w) is just one data object for $\mathbf{B} \to \mathbf{C}$, but the input/output passage from the consistent sets of $\mathbf{A}$ to these objects is sufficient to determine curry h as an approximable mapping. The exact connection between the two mappings is given in terms of function values as follows:

$$h(x, y) = (\text{curry } h)(x)(y),$$

for all $x \in |\mathbf{A}|$ and $y \in |\mathbf{B}|$. From this equation it follows that curry h is uniquely determined. But, from what we know about apply, this is actually the same equation as that stated at the end of the theorem. ■

Approximations to functions. Why have approximable functions been given this name? In general, elements of domains are the limits of their finite approximations. We have just indicated why the approximable

mappings from one domain into another do form the elements of a domain themselves. We have explicitly shown how to construct the finite approximable mappings $\overline{w}$. A closer examination of the definitions would emphasize the very constructive nature of this analysis. It follows that the approximable mappings can therefore be approximated by simple functions. It does not follow that all approximable mappings are simple or constructive, since what takes place in the limiting process can be very complex. But the result does show how we can start to make distinctions once a precise sense of approximation is uncovered. ■

Higher-type functions and the combinators. In the above discussion we have already combined the function-space construction with other domains by means of products. But there is nothing now stopping us from iterating the arrow domain constructor with itself as much as we like. This is how the so-called higher types are formed. In certain categories, such as the category of sets, this is a non-constructive move leading to the higher cardinal numbers. In the category of domains, however, the construct is constructive, because we have shown how to define all the parts of $\mathbf{A} \to \mathbf{B}$ in terms of very finite data objects (assuming, it need hardly be added, that $\mathbf{A}$ and $\mathbf{B}$ are constructively given).

Once the higher types have been formed as spaces, it must be asked what we are to do with them. The answer is that there are many, many mappings between these spaces that can be defined in terms of the simple notions we have been working with. These mappings are useful for the following reason: the higher types provide remarkable scope for modelling notions (as those needed in denotational semantics for example), but the various aspects of the models have to be related—and this is where these mappings come into play. We have already seen a preliminary example in the last theorem, which can be interpreted as saying why the two domains shown are isomorphic:

$$\mathbf{A} \times \mathbf{B} \to \mathbf{C} \cong \mathbf{A} \to (\mathbf{B} \to \mathbf{C}).$$

We have neither the time nor the space to present a full theory of higher-type operators here, so some further examples will have to suffice. First, we have already made use of constant mappings. Since the construction of them is very uniform, there ought to be an associated operator. In fact, we have already been using it notationally. We have the approximable mapping const : $\mathbf{B} \to (\mathbf{A} \to \mathbf{B})$ that takes every element of $\mathbf{B}$ to the corresponding constant function. (It has to be checked that this is an approximable mapping.) Note that there is a different mapping for each pair of domains $\mathbf{A}$ and $\mathbf{B}$, because the resulting types of const are different.

As another example, take the pairing of functions explained in Proposition 6.2. We can think of the operator in this case being

$$\text{pair} : (\mathbf{C} \to \mathbf{A}) \times (\mathbf{C} \to \mathbf{B}) \to (\mathbf{C} \to (\mathbf{A} \times \mathbf{B})),$$

where for functions of the proper type we have:

$$\text{pair}(f, g) = \langle f, g \rangle$$

There will be a similar operator for the construct of Proposition 6.4.

Of course the most basic operator of function composition is also approximable of the appropriate type. We can write:

$$\text{comp} : (\mathbf{B} \to \mathbf{C}) \times (\mathbf{A} \to \mathbf{B}) \to (\mathbf{A} \to \mathbf{C}),$$

where for functions of the right types we have:

$$\text{comp}(g, f) = g \circ f.$$

The approximability has to be checked, of course. But once a number of the more primitive operators have been established as being approximable, then others can be proved to be so by writing them as combinations of previously obtained operators. ■

Categories again. All of what we have been saying about operators ties in with category theory very nicely—as the category theorists have known for a long time. The technical term for what we have been doing in part is cartesian closed category—that is a property of the category of domains. Without going into details, that is essentially what 6.2 and 7.2 show of our category. But domains have many other properties beyond being a cartesian closed category. For example the possibility of forming sums is an extra (and useful) bonus, and there are many others. Nevertheless, the categorical viewpoint is a good way of organizing the properties, and it suggests other things to look for from our experiences with other categories. The next result gives a particularly important notion that can be expressed as an operator. ■

Theorem (`theorem_7_2`). For information systems $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$: the elements of $\mathbf{A} \to \mathbf{B}$ are precisely the approximable maps $f : \mathbf{A} \to \mathbf{B}$ (the two round-trips between elements and approximable maps are identities); there is an approximable map $\mathrm{apply} : (\mathbf{B} \to \mathbf{C}) \times \mathbf{B} \to \mathbf{C}$ such that $\mathrm{apply}(g, y) = g(y)$ for $g \in |\mathbf{B} \to \mathbf{C}|$ and $y \in |\mathbf{B}|$; and for every approximable $h : \mathbf{A} \times \mathbf{B} \to \mathbf{C}$ there is a unique approximable $\mathrm{curry}\, h : \mathbf{A} \to (\mathbf{B} \to \mathbf{C})$ with $h = \mathrm{uncurry}(\mathrm{curry}\, h)$, equivalently $h = \mathrm{apply} \circ \langle (\mathrm{curry}\, h) \circ \mathrm{fst}, \mathrm{snd} \rangle$.

Proof. The element–map bijection is the mutual inverse of viewing an approximable map as a function-space element and extracting the relation from an element; evaluation of apply is the paired-element lemma; uniqueness of curry and the identity $\mathrm{uncurry} \circ \mathrm{curry}$ are the corresponding library facts. ?

`theorem_7_2.lean`

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    curryMap
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    approxMap_toElement
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    pairElements
import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.curryMap
import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    approxMap_toElement

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys
open ApproximableMap

theorem theorem_7_2 :
    (∀ {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β),
      (∀ f : ApproximableMap A B,
        element_toApproxMap A B (approxMap_toElement A B f) = f) ∧
      (∀ x : (functionSystem A B).Element,
        approxMap_toElement A B (element_toApproxMap A B x) = x)) ∧
    (∀ {β ? : Type*} [DecidableEq β] [DecidableEq ?] (B : InfoSys β) (C : InfoSys ?)
      (g : (functionSystem B C).Element) (y : B.Element),
      (applyMap (B := B) (C := C)).toElement
        (pairElements (functionSystem B C) B g y) =
        (element_toApproxMap B C g).toElement y) ∧
    (∀ {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
      (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)
```

```
      (h : ApproximableMap (productSystem A B) C)
      (k : ApproximableMap A (functionSystem B C)),
      uncurryMap A B C k = h → k = curryMap A B C h) ∧
    (∀ {α β ? : Type*} [DecidableEq α] [DecidableEq β] [DecidableEq ?]
      (A : InfoSys α) (B : InfoSys β) (C : InfoSys ?)
      (h : ApproximableMap (productSystem A B) C),
      uncurryMap A B C (curryMap A B C h) = h) :=
  ⟨@elements_equiv_approxMaps, @applyMap_toElement, @curryMap_unique, @uncurry_curryMap
    ⟩

end DomainTheory.InformationSystems.InfoSys
```

5.8.8 Theorem 7.3: least fixed-point operator

THEOREM 7.3. Let $\mathbf{A}$ be an information system. Then there is a unique operator, the **least fixed-point operator**, such that

(i) $\mathrm{fix} : (\mathbf{A} \to \mathbf{A}) \to \mathbf{A}$; and,

for all approximable mappings $f : \mathbf{A} \to \mathbf{A}$, we have:

(ii) $f(\mathrm{fix}(f)) \subseteq \mathrm{fix}(f)$; and

(iii) for all $x \in |\mathbf{A}|$, if $f(x) \subseteq x$, then $\mathrm{fix}(f) \subseteq x$.

Moreover, for this operator, condition (ii) is an equality.

Proof. This is a well-known result—especially the fact that the conditions above uniquely determine the operator. The only question is the existence of the operator. The inclusion of condition (ii) gives the hint, for $\mathrm{fix}(f)$ is the least solution of $f(x) \subseteq x$. Suppose x is any such element, then if $u \subseteq x$ and $u\ f\ v$ hold, it follows that $v \subseteq x$. Now, since $\emptyset \subseteq x$ always holds, if we wish to form the least x, we start with $\emptyset$ and just follow it under the action of f.

Specifically, we define $\mathrm{fix}(f)$ to be the union of all $v \in \mathrm{Con}_{\mathbf{A}}$ for which there exist a sequence $u_0, \ldots, u_n \in \mathrm{Con}_{\mathbf{A}}$ where:

(1) $u_0 = \emptyset$;

(2) $u_i\ f\ u_{i+1}$ for all $i < n$; and

(3) $u_n = v$.

Because f is approximable, it is clear that fix f is closed under entailment. To prove that it is consistent, suppose both v and v' belong to the sets thrown into the union. We have to show that $v \cup v'$ is consistent and also is thrown in. Consider the two sequences $u_0, \ldots, u_n \in \mathrm{Con}_{\mathbf{A}}$ and $u'_0, \ldots, u'_n \in \mathrm{Con}_{\mathbf{A}}$ that are responsible for putting v and v' in. It is without loss of generality that we assume they are of the same length, since we can always add lots of $\emptyset$'s onto the front of the shorter one and still satisfy (1)–(3). Now one just argues by induction on i that the sequence of unions $u_i \cup u'_i$ satisfies (1)–(3) with respect to $v \cup v'$.

But why is fix approximable? The method of proof is to replace f by $\bar{w}$ in (2) above, and to use the condition that there exists a sequence satisfying (1)–(3) as defining a relation between sets $w \in \mathrm{Con}_{\mathbf{A} \to \mathbf{A}}$

and sets $v \in \mathrm{Con}_{\mathbf{A}}$. It is not difficult to prove that this is an approximable mapping in the sense of the official definition. Clearly this relation determines fix as an operator. ■

The result above not only proves that every approximable mapping of the form $f : \mathbf{A} \to \mathbf{A}$ has a fixed point as an element of $\mathbf{A}$, but that the association of the least fixed point is itself an approximable operator. The formulation makes essential use of the partial ordering of the domains, but Gordon Plotkin noticed as an exercise that the characterization of the operator can be given entirely by equations.

Theorem (`theorem_7_3`). There is a least fixed-point operator $\mathrm{fix} : (\mathbf{A} \to \mathbf{A}) \to \mathbf{A}$ such that $f(\mathrm{fix}(f)) = \mathrm{fix}(f)$, $\mathrm{fix}(f)$ is least among pre-fixed points of f, and any operator with the pre-fixed and leastness properties equals this fix map.

Proof. Invoke equality of f at the constructed fixed point, leastness among pre-fixed points, and uniqueness of the fix map among operators with those properties. ?

`theorem_7_3.lean`

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Fixpoint.
    fixElement
import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Fixpoint.
    fixElement_fixed
import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Fixpoint.
    fixElement_le_of_pre_fixed
import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Fixpoint.fixMap_unique

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys.ApproximableMap

theorem theorem_7_3 :
    (∀ {α : Type*} [DecidableEq α] (A : InfoSys α) (f : ApproximableMap A A),
      f.toElement (fixElement A f) = fixElement A f) ∧
    (∀ {α : Type*} [DecidableEq α] (A : InfoSys α) (f : ApproximableMap A A)
      (x : A.Element), f.toElement x ≤ x → fixElement A f ≤ x) ∧
    (∀ {α : Type*} [DecidableEq α] (A : InfoSys α)
      (φ : ApproximableMap (functionSystem A A) A),
      (∀ f : ApproximableMap A A,
        f.toElement (φ.toElement (approxMap_toElement A A f)) ≤
          φ.toElement (approxMap_toElement A A f)) →
      (∀ f : ApproximableMap A A, ∀ x : A.Element,
        f.toElement x ≤ x → φ.toElement (approxMap_toElement A A f) ≤ x) →
          φ = fixMap A) :=
  ⟨@fixElement_fixed, @fixElement_le_of_pre_fixed, @fixMap_unique⟩

end DomainTheory.InformationSystems.InfoSys.ApproximableMap
```

5.8.9 Proposition 7.4: equational uniqueness of fix

PROPOSITION 7.4. *The least fixed-point operator is uniquely determined by the following three conditions:*

(i) $\mathrm{fix}_{\mathbf{A}} : (\mathbf{A} \to \mathbf{A}) \to \mathbf{A}$, *for all systems* $\mathbf{A}$;

(ii) $\mathrm{fix}_{\mathbf{A}}(f) = f(\mathrm{fix}_{\mathbf{A}}(f))$, *for all* $f : \mathbf{A} \to \mathbf{A}$; *and*

(iii) $h(\mathrm{fix}_{\mathbf{A}}(f)) = \mathrm{fix}_{\mathbf{B}}(g)$, *whenever* $f : \mathbf{A} \to \mathbf{A}$, $g : \mathbf{B} \to \mathbf{B}$, $h : \mathbf{A} \to \mathbf{B}$, *provided that* $h \circ f = g \circ h$ *and* $h(\bot_{\mathbf{A}}) = \bot_{\mathbf{B}}$. ■

Remarks on the space of strict mappings. In 7.4 and many other places we have had occasion to make use of mappings that take the bottom element of one domain over to the bottom element of the other domain. Such mappings are called *strict mappings* because they take a strict view of having empty input. As notation we might write

$$f : \mathbf{A} \to_{\blacksquare} \mathbf{B}$$

to mean that f is a strict approximable mapping (i.e. $f(\bot_{\mathbf{A}}) = \bot_{\mathbf{B}}$). The totality of domains and strict mappings forms an interesting category in itself, but it is best used in connection with the full category of all approximable mappings.

The collection of strict mappings forms a domain, too. The way to see this is to refer back to Definition 7.1 and add an additional clause ruling out non-strict mappings as inconsistent. What has to be added to 7.1(iii) is the conjunct on the right-hand side to the effect that if the condition $\emptyset \vdash_{\mathbf{A}} \bigcup\{u_i \mid i \in I\}$ holds, then $\emptyset \vdash_{\mathbf{B}} \bigcup\{v_i \mid i \in I\}$ holds too. By the same arguments we used before, it follows that this is the appropriate system for the domain of strict mappings. We can denote it by $(\mathbf{A} \to_{\blacksquare} \mathbf{B})$.

There is also a useful operator

$$\mathrm{strict} : (\mathbf{A} \to \mathbf{B}) \to (\mathbf{A} \to_{\blacksquare} \mathbf{B})$$

defined by the condition that for $f : \mathbf{A} \to \mathbf{B}$ we have:

$$u \; \mathit{strict}(f) \; v \text{ iff either } \emptyset \vdash_{\mathbf{B}} v \text{ or } \emptyset \nvdash_{\mathbf{A}} u \text{ and } u f v,$$

for all $u \in \mathrm{Con}_{\mathbf{A}}$ and $v \in \mathrm{Con}_{\mathbf{B}}$. This operator converts every approximable mapping into the largest strict mapping contained within it.

Since every strict mapping is an approximable mapping, there is also an obvious operator going the other way. The pair of operators shows that $\mathbf{A} \to_s \mathbf{B}$ as a domain is what is called a *retract* of $\mathbf{A} \to \mathbf{B}$. There is an interesting theory of this kind of relationship between domains, but we cannot enter into it here.

As a very small application of the use of strict mappings, we remark that the following two domains are isomorphic:

$$\mathbf{A} \times \mathbf{A} \cong (\mathbf{BOOL} \to_s \mathbf{A}).$$

The mapping from right to left is called the *conditional operator*, cond, and we have for all elements $x, y \in |\mathbf{A}|$ and $t \in |\mathbf{BOOL}|$

$$\mathrm{cond}(x, y)(t) = \begin{cases} x, & \text{if } t = \text{true}, \\ y, & \text{if } t = \text{false}. \end{cases} \blacksquare$$

Theorem (`proposition_7_4.ii`). The least fixed point of f is a fixed point: $f(\mathrm{fix}(f)) = \mathrm{fix}(f)$.

Proof. The fixed-point equality for the constructed fix element. ?

Theorem (`proposition_7_4.unique`). Any fixed point that is least among pre-fixed points equals the constructed fix element.

Proof. Leastness gives one inequality; the constructed fix element is itself a pre-fixed point. ?

Theorem (`proposition_7_4.iii`). Strict maps that intertwine endomaps preserve least fixed points.

Proof. Combine the two inequalities between the image of $\mathrm{fix}(f)$ and $\mathrm{fix}(g)$. ?

Theorem (`proposition_7_4`). The least fixed-point operator is uniquely determined by being an operator $\mathrm{fix}_{\mathbf{A}} : (\mathbf{A} \to \mathbf{A}) \to \mathbf{A}$ with $f(\mathrm{fix}(f)) = \mathrm{fix}(f)$ and preservation of least fixed points under strict intertwining maps.

Proof. Assemble the fixed-point equation, the intertwining property, and uniqueness among least pre-fixed points. ?

Theorem (`closure_empty_eq_bot`). The closure of the empty set is the bottom element.

Proof. Antisymmetry of the element order using empty entailment and the bottom axioms. ?

Theorem (`idMap_botElement`). The identity approximable map sends bottom to bottom.

Proof. Bottom is related only to data entailed by empty, hence lies in the bottom element. ?

Theorem (`fixChain_toElement_le_fix`). Along a fix-chain for f, the image under a strict map commuting with f and g stays below the fix element of g.

Proof. Induction on the chain, using the commuting square and strictness at bottom. ?

Theorem (`toElement_comp_fix_le`). Under a commuting square of maps with a strict mediator, the mediator sends fix-chains for f below fix of g.

Proof. Reduce to the chain lemma for the mediator applied along fix-chains. ?

Theorem (`fixElement_le_toElement_fix`). A strict map commuting with f and g sends $\mathrm{fix}(f)$ below $\mathrm{fix}(g)$.

Proof. Every token of $\mathrm{fix}(f)$ arises from a fix-chain; apply the chain comparison. ?

Theorem (`toElement_fix_le_fixElement`). A strict map commuting with f and g sends $\mathrm{fix}(f)$ above nothing beyond $\mathrm{fix}(g)$: the reverse inequality.

Proof. Use leastness of $\mathrm{fix}(g)$ among pre-fixed points after transporting the fixed-point property. ?

proposition_7_4.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.comp
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition54.idMap
public import DomainTheory.InformationSystems.ElementsOfASystem.botElement
public import DomainTheory.InformationSystems.ElementsOfASystem.closure
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Fixpoint.
    fixChain_con
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Fixpoint.
    fixElement
import DomainTheory.InformationSystems.InformationSystems.EntSet
import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Fixpoint.
    fixElement_fixed
import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Fixpoint.
    fixElement_le_of_pre_fixed
import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    approxMap_toElement
import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Fixpoint.
    fixElement_unique

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
namespace ApproximableMap

variable {α β : Type*} [DecidableEq α] [DecidableEq β]
variable (A : InfoSys α) (B : InfoSys β)

theorem closure_empty_eq_bot :
    A.closure (∅ : Finset α) A.con_empty = A.botElement := by
  apply le_antisymm
  · intro a ha
    exact A.ent_trans (A.con_sing A.bot) A.con_empty (EntSet.empty A _) ha
  · intro a ha
    exact A.ent_trans A.con_empty (A.con_sing A.bot)
      (fun x hx => by
        have : x = A.bot := Finset.mem_singleton.mp hx
        subst this
        exact A.ent_bot A.con_empty)
      ha

theorem idMap_botElement : (idMap A).toElement A.botElement = A.botElement :=
  idMap.toElement A A.botElement

theorem fixChain_toElement_le_fix (f : ApproximableMap A A) (g : ApproximableMap B B)
    (h : ApproximableMap A B)
    (hcomm : comp h f = comp g h)
    (hstrict : h.toElement A.botElement = B.botElement)
    {w : Finset (FunToken A A)}
    (hw : ↑w ⊆ (approxMap_toElement A A f).carrier) {n : Nat} {u : Finset α}
    (hchain : FixChain A w n u) :
    h.toElement (A.closure u (fixChain_con A hchain)) ≤ fixElement B g := by
  induction n generalizing u with
  | zero =>
    have hu : u = ∅ := hchain
    subst hu
    simpa [closure_empty_eq_bot A, hstrict] using
      (botElement.le B (fixElement B g))
  | succ n ih =>
    obtain ⟨u0, hu0, hu0C, huC, hEnt⟩ := hchain
    have ih' := ih hu0
    obtain ⟨_, s, hs, hIn, hOut⟩ := hEnt
    have hps : ∀ q ∈ s, f.rel q.val.1 q.val.2 := fun q hq => by
      simpa [mem_approxMap_toElement] using hw (Finset.mem_coe.2 (hs hq))
    have hin : funInputUnion A A s ∈ A.Con :=
      A.con_subset
        (EntSet.union_mem A hu0C (entSet_inputUnion_of_ent A A hIn))
        (funion.subset_right _ _)
    have hrel : f.rel (funInputUnion A A s) (funOutputUnion A A s) :=
      rel_input_output_union A A f s hps hin
    have hrel' : f.rel u0 (funOutputUnion A A s) :=
      f.mono hrel (entSet_inputUnion_of_ent A A hIn)
        (EntSet.refl A (f.rel_cod hrel)) hu0C (f.rel_cod hrel)
```

```
    have hrel'' : f.rel u0 u :=
      f.mono hrel' (EntSet.refl A hu0C) hOut hu0C huC
    have huf : A.closure u huC ≤ f.toElement (A.closure u0 hu0C) := by
      intro a ha
      have hsub : ↑u ⊆ (f.toElement (A.closure u0 hu0C)).carrier := by
        intro b hb
        have hsing : f.rel u0 {b} :=
          f.mono hrel'' (EntSet.refl A hu0C)
            (fun z hz => by
              rw [Finset.mem_singleton] at hz; subst hz
              exact A.ent_refl huC hb)
            hu0C (A.con_sing b)
        refine ⟨u0, fun z hz => closure.subset A hu0C hz, hsing⟩
      exact (f.toElement (A.closure u0 hu0C)).closed u a hsub ha
    have hhuf : h.toElement (A.closure u huC) ≤
        h.toElement (f.toElement (A.closure u0 hu0C)) :=
      toElement_mono h huf
    have hcomm' : h.toElement (f.toElement (A.closure u0 hu0C)) =
        g.toElement (h.toElement (A.closure u0 hu0C)) := by
      rw [← comp.toElement, hcomm, comp.toElement]
    have hgle : g.toElement (h.toElement (A.closure u0 hu0C)) ≤
        g.toElement (fixElement B g) :=
      toElement_mono g ih'
    have hfixg : g.toElement (fixElement B g) = fixElement B g :=
      fixElement_fixed B g
    calc
      h.toElement (A.closure u huC) ≤ h.toElement (f.toElement (A.closure u0 hu0C)) :=
     hhuf
      _ = g.toElement (h.toElement (A.closure u0 hu0C)) := hcomm'
      _ ≤ g.toElement (fixElement B g) := hgle
      _ = fixElement B g := hfixg

theorem toElement_comp_fix_le (f : ApproximableMap A A) (g : ApproximableMap B B)
    (h : ApproximableMap A B)
    (hcomm : comp h f = comp g h) :
    g.toElement (h.toElement (fixElement A f)) ≤ h.toElement (fixElement A f) := by
  have h1 : g.toElement (h.toElement (fixElement A f)) =
      h.toElement (f.toElement (fixElement A f)) := by
    rw [← comp.toElement, ← hcomm, comp.toElement]
  rw [h1, fixElement_fixed A f]

theorem fixElement_le_toElement_fix (f : ApproximableMap A A) (g : ApproximableMap B B
    )
    (h : ApproximableMap A B)
    (hcomm : comp h f = comp g h) :
    fixElement B g ≤ h.toElement (fixElement A f) :=
  fixElement_le_of_pre_fixed B g (h.toElement (fixElement A f))
    (toElement_comp_fix_le A B f g h hcomm)

theorem toElement_fix_le_fixElement (f : ApproximableMap A A) (g : ApproximableMap B B
    )
    (h : ApproximableMap A B)
    (hcomm : comp h f = comp g h)
    (hstrict : h.toElement A.botElement = B.botElement) :
    h.toElement (fixElement A f) ≤ fixElement B g := by
  intro Z ⟨u, hu, hrel⟩
```

```
  obtain ⟨w, hw, ⟨_, _, v0, hv0, hEnt⟩⟩ :=
    exists_rel_of_subset_image (fixMap A) (approxMap_toElement A A f) u hu
  obtain ⟨n, hn⟩ := hv0
  have hcl := fixChain_toElement_le_fix A B f g h hcomm hstrict hw hn
  have hv0C := fixChain_con A hn
  have hrel' : h.rel v0 {Z} :=
    h.mono hrel hEnt (EntSet.refl B (h.rel_cod hrel)) hv0C (h.rel_cod hrel)
  have hZ : Z ∈ (h.toElement (A.closure v0 hv0C)).carrier :=
    ⟨v0, closure.subset A hv0C, hrel'⟩
  exact hcl hZ

end ApproximableMap
end InfoSys

end DomainTheory.InformationSystems

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
namespace ApproximableMap

variable {α β : Type*} [DecidableEq α] [DecidableEq β]
variable (A : InfoSys α) (B : InfoSys β)

namespace proposition_7_4

theorem ii (f : ApproximableMap A A) :
    f.toElement (fixElement A f) = fixElement A f :=
  fixElement_fixed A f

theorem unique (f : ApproximableMap A A) (φ : A.Element)
    (hii : f.toElement φ = φ)
    (hleast : ∀ x : A.Element, f.toElement x ≤ x → φ ≤ x) :
    φ = fixElement A f :=
  fixElement_unique A f φ (le_of_eq hii) hleast

theorem iii (f : ApproximableMap A A) (g : ApproximableMap B B)
    (h : ApproximableMap A B)
    (hcomm : comp h f = comp g h)
    (hstrict : h.toElement A.botElement = B.botElement) :
    h.toElement (fixElement A f) = fixElement B g := by
  apply le_antisymm
  · exact toElement_fix_le_fixElement A B f g h hcomm hstrict
  · exact fixElement_le_toElement_fix A B f g h hcomm

end proposition_7_4

theorem proposition_7_4 :
    (∀ {α : Type*} [DecidableEq α] (A : InfoSys α) (f : ApproximableMap A A),
      f.toElement (fixElement A f) = fixElement A f) ∧
    (∀ {α β : Type*} [DecidableEq α] [DecidableEq β]
      (A : InfoSys α) (B : InfoSys β)
      (f : ApproximableMap A A) (g : ApproximableMap B B) (h : ApproximableMap A B),
      comp h f = comp g h →
```

```
        h.toElement A.botElement = B.botElement →
          h.toElement (fixElement A f) = fixElement B g) ∧
    (∀ {α : Type*} [DecidableEq α] (A : InfoSys α) (f : ApproximableMap A A) (φ : A.
    Element),
      f.toElement φ = φ →
        (∀ x : A.Element, f.toElement x ≤ x → φ ≤ x) →
          φ = fixElement A f) :=
  ⟨@proposition_7_4.ii, @proposition_7_4.iii, @proposition_7_4.unique⟩

end ApproximableMap
end InfoSys

end DomainTheory.InformationSystems
```

5.9 Some domain equations

<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

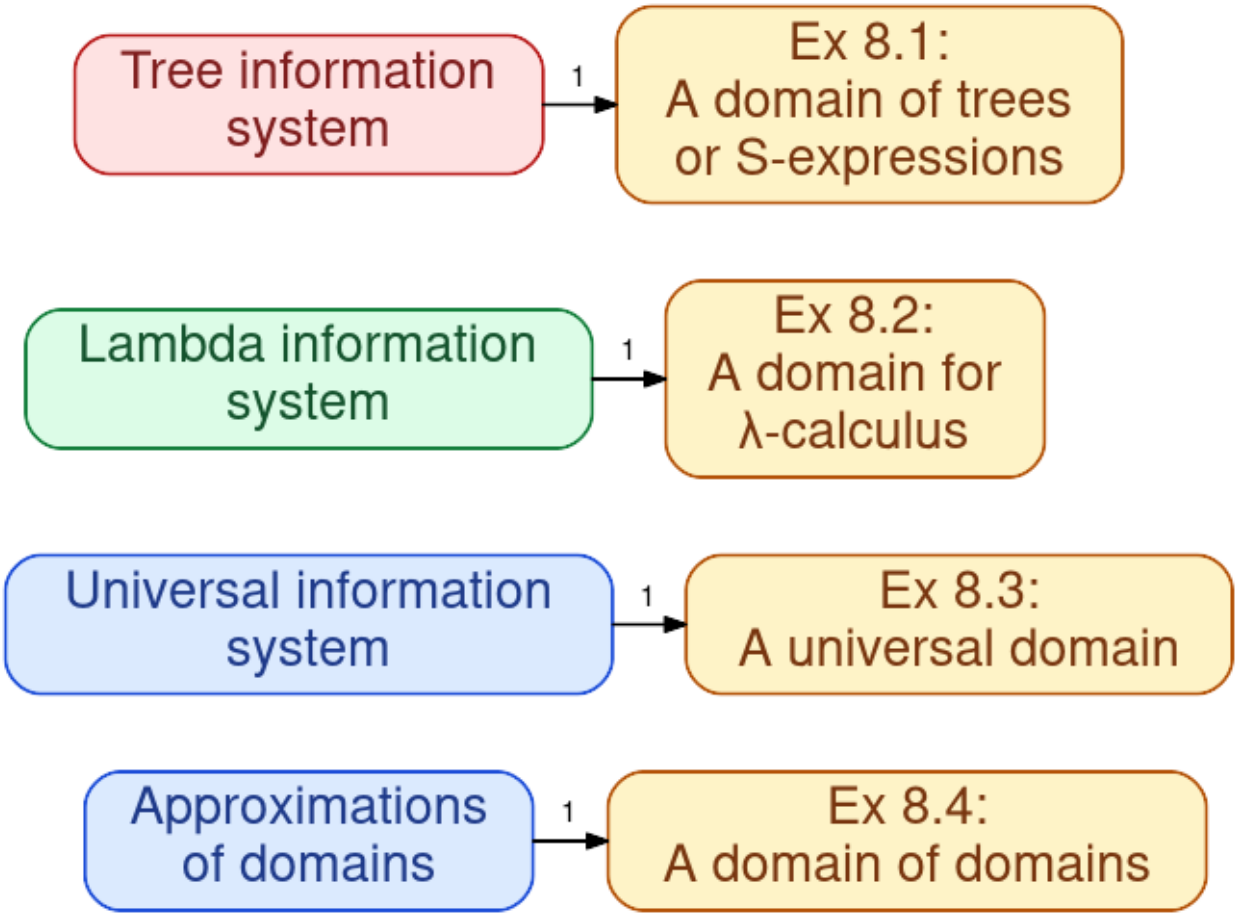


Figure 62: Diagram 27

5.9.1 Introduction

Having outlined the theory of several domain constructs, the final topic for this paper will be the discussion of the *iteration* of these constructs in giving *recursive*, rather than direct definitions of domains. These recursively defined systems have often been called "reflexive," because the domains generally contain copies of themselves as a part of their very structure. The way that this self-containment takes place is best expressed by the so-called *domain equations*, which are really isomorphisms that relate the domain as a whole to a combination of domains—usually with the main domain as a component. This description is rough, since recursion equations for domains can be as complex as recursion equations for functions. We will not enter into a full theory of domain equations now but will just review some preliminary examples to illustrate how the new presentation makes the constructions more explicit.

5.9.2 Tree information system

Definition (`pairLInsert` commutative). Insertion of left children commutes.

Proof. By cases on the two tokens, using commutativity of finite-set insertion. ?

Definition (`pairRInsert` commutative). Insertion of right children commutes.

Proof. By cases on the two tokens, using commutativity of finite-set insertion. ?

Definition (`atomInsert` commutative). Insertion of atoms commutes.

Proof. By cases on the two tokens, using commutativity of finite-set insertion. ?

Definition (`TreeToken`). A tree token is bottom, an atom from the given domain, or a left or right child of a tree token.

Definition (`instDecidableEqTreeToken`). There is a type-class instance for decidable equality Tree Token.

Definition (`atomInsert`). The atom insertion adds the payload of the matching constructor and leaves every other constructor unchanged.

Definition (`pairLInsert`). The left-child insertion adds the payload of the matching constructor and leaves every other constructor unchanged.

Definition (`pairLFinset`). The set of left children.

Theorem (`mem_foldr_pairL`). A token lies in the fold of left children exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_pairLFinset`). A token lies in the mem set of left children exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`pairRInsert`). The right-child insertion adds the payload of the matching constructor and leaves every other constructor unchanged.

Definition (`pairRFinset`). The set of right children.

Theorem (`mem_foldr_pairR`). A token lies in the fold of right children exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_pairRFinset`). A token lies in the mem set of right children exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`atomFinset`). The atom finite set.

Definition (`TreeCon`). A finite set of tree tokens is consistent when every finite selection of atoms is consistent in **A** and the left and right children are themselves consistent.

Theorem (`TreeCon.atom_con_of_atoms`). The statement atom con of atoms holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeCon.atoms_empty_of_right`). The atoms empty of right of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeCon.pairs_empty_of_left`). The pairs empty of left of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`atomFinset_empty`). The atom finite set empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`eq_singleton_bot_of_subset`). The property eq singleton bot of subset passes to subsets.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_foldr_atom`). A token lies in the mem foldr atom exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairLFinset_empty`). The set of left children empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairLFinset_insert_atom`). Inserting the indicated constructor updates set of left children insert atom by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairLFinset_insert_bot`). Inserting the indicated constructor updates set of left children insert bot by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairLFinset_insert_pairL`). Inserting the indicated constructor updates set of left children insert pair L by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairLFinset_insert_pairR`). Inserting the indicated constructor updates set of left children insert pair R by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairLFinset_mono`). The operation set of left children mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairLFinset_singleton_atom`). On a singleton, set of left children singleton atom returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairLFinset_singleton_bot`). On a singleton, set of left children singleton bot returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairLFinset_singleton_pairL`). On a singleton, set of left children singleton pair L returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairLFinset_singleton_pairR`). On a singleton, set of left children singleton pair R returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairRFinset_empty`). The set of right children empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairRFinset_insert_atom`). Inserting the indicated constructor updates set of right children insert atom by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairRFinset_insert_bot`). Inserting the indicated constructor updates set of right children insert bot by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairRFinset_insert_pairL`). Inserting the indicated constructor updates set of right children insert pair L by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairRFinset_insert_pairR`). Inserting the indicated constructor updates set of right children insert pair R by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairRFinset_mono`). The operation set of right children mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairRFinset_singleton_atom`). On a singleton, set of right children singleton atom returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairRFinset_singleton_bot`). On a singleton, set of right children singleton bot returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairRFinset_singleton_pairL`). On a singleton, set of right children singleton pair L returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairRFinset_singleton_pairR`). On a singleton, set of right children singleton pair R returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`treeBot`). The bottom tree token is the least informative token.

Theorem (`mem_atomFinset`). A token lies in the mem atom finite set exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeCon_empty`). The tree Con empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`atomFinset_insert_atom`). Inserting the indicated constructor updates atom finite set insert atom by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`atomFinset_insert_bot`). Inserting the indicated constructor updates atom finite set insert bot by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`atomFinset_insert_pairL`). Inserting the indicated constructor updates atom finite set insert pair L by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`atomFinset_insert_pairR`). Inserting the indicated constructor updates atom finite set insert pair R by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`atomFinset_mono`). The operation atom finite set mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`atomFinset_singleton_atom`). On a singleton, atom finite set singleton atom returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`atomFinset_singleton_bot`). On a singleton, atom finite set singleton bot returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`atomFinset_singleton_pairL`). On a singleton, atom finite set singleton pair L returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`atomFinset_singleton_pairR`). On a singleton, atom finite set singleton pair R returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeCon.pairL_of_right`). The statement pair L of right holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeCon.pairR_of_right`). The statement pair R of right holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeCon_insert_bot`). Inserting the indicated constructor updates tree Con insert bot by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeCon_subset`). The property tree Con subset passes to subsets.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeCon_singleton`). On a singleton, tree Con singleton returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`TreeEntPayload`). The tree Ent Payload.

Theorem (`TreeEntPayload.of_mem`). The statement of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeEntPayload.of_no_atoms_no_pairs`). The statement of no atoms no pairs holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`TreeEnt`). Tree entailment is the entailment of the tree domain.

Theorem (`TreeEnt.of_mem`). The statement of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeCon_insert_of_ent`). Inserting the indicated constructor updates tree Con insert of ent by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairL_ne_or_subset_bot`). The property pair L ne or subset bot passes to subsets.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`pairR_ne_or_subset_bot`). The property pair R ne or subset bot passes to subsets.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeEntPayload_trans`). The relation tree Ent Payload trans is transitive.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`TreeEnt_trans`). The relation tree Ent trans is transitive.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`treeSystem`). The tree domain over $\mathbf{A}$ is the information system whose tokens are tree tokens, whose consistency is tree consistency, and whose entailment is tree entailment.

Definition (`treeRhs`). The right-hand side of the tree equation is the sum of $\mathbf{A}$ with the product of the tree domain with itself.

Theorem (`treeRhs.eq_sum_product`). The equality eq sum product holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`treeUnfold`). Unfolding a tree token views it as a token of $\mathbf{A} + (\mathbf{T} \times \mathbf{T})$.

Theorem (`treeUnfold.atom`). The statement atom holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`treeUnfold.bot`). The statement bot holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`treeUnfold.pairL`). The statement pair L holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`treeUnfold.pairR`). The statement pair R holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`treeUnfold.pairL_bot_eq_pairR_bot`). The equality pair L bot eq pair R bot holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

`treeSystem.lean`

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.Constructive

universe u

/-- Tokens of the tree / S-expression system (Scott 1982, Factoid 8.1). -/
inductive TreeToken (α : Type u) where
  | bot : TreeToken α
  | atom : α → TreeToken α
  | pairL : TreeToken α → TreeToken α
  | pairR : TreeToken α → TreeToken α

instance instDecidableEqTreeToken {α : Type u} [DecidableEq α] :
    DecidableEq (TreeToken α)
  | .bot, .bot => isTrue rfl
  | .atom a, .atom b =>
      if h : a = b then isTrue (h ▶ rfl)
      else isFalse fun h' => h (TreeToken.atom.inj h')
  | .pairL a, .pairL b =>
      match instDecidableEqTreeToken a b with
      | isTrue h => isTrue (h ▶ rfl)
```

```
      | isFalse h => isFalse fun h' => h (TreeToken.pairL.inj h')
  | .pairR a, .pairR b =>
      match instDecidableEqTreeToken a b with
      | isTrue h => isTrue (h ▶ rfl)
      | isFalse h => isFalse fun h' => h (TreeToken.pairR.inj h')
  | .bot, .atom _ => isFalse fun h => nomatch h
  | .bot, .pairL _ => isFalse fun h => nomatch h
  | .bot, .pairR _ => isFalse fun h => nomatch h
  | .atom _, .bot => isFalse fun h => nomatch h
  | .atom _, .pairL _ => isFalse fun h => nomatch h
  | .atom _, .pairR _ => isFalse fun h => nomatch h
  | .pairL _, .bot => isFalse fun h => nomatch h
  | .pairL _, .atom _ => isFalse fun h => nomatch h
  | .pairL _, .pairR _ => isFalse fun h => nomatch h
  | .pairR _, .bot => isFalse fun h => nomatch h
  | .pairR _, .atom _ => isFalse fun h => nomatch h
  | .pairR _, .pairL _ => isFalse fun h => nomatch h

variable {α : Type*} [DecidableEq α]
variable (A : InfoSys α)

namespace treeSystem

def atomInsert : TreeToken α → Finset α → Finset α
  | .atom x => insert x
  | .bot | .pairL _ | .pairR _ => id

private def pairLInsert : TreeToken α → Finset (TreeToken α) → Finset (TreeToken α)
  | .pairL t => insert t
  | .bot | .atom _ | .pairR _ => id

instance :
    LeftCommutative (pairLInsert : TreeToken α → Finset (TreeToken α) → Finset (
    TreeToken α)) :=
  ⟨fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl⟩

def pairLFinset (u : Finset (TreeToken α)) : Finset (TreeToken α) :=
  Multiset.foldr pairLInsert (∅ : Finset (TreeToken α)) u.1

private theorem mem_foldr_pairL (s : Multiset (TreeToken α)) (t : TreeToken α) :
    t ∈ Multiset.foldr pairLInsert (∅ : Finset (TreeToken α)) s ↔
      ∃ p ∈ s, p = .pairL t := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro ht; exact False.elim (Finset.notMem_empty t ht)
    · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
  · intro p rest ih
    cases p with
    | pairL a =>
      simp only [Multiset.foldr_cons, pairLInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      · rintro (ht | ⟨q, hq, hq'⟩)
        · exact ⟨.pairL a, Or.inl rfl, congrArg TreeToken.pairL ht.symm⟩
        · exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
```

```
        rcases hq with rfl | hq
        · injection hq' with ht; exact Or.inl ht.symm
        · exact Or.inr ⟨q, hq, hq'⟩
    | bot | atom _ | pairR _ =>
      simp only [Multiset.foldr_cons, pairLInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      · rintro ⟨q, hq, hq'⟩; exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · exact False.elim (nomatch hq')
        · exact ⟨q, hq, hq'⟩

theorem mem_pairLFinset {u : Finset (TreeToken α)} {t : TreeToken α} :
    t ∈ pairLFinset u ↔ .pairL t ∈ u := by
  constructor
  · intro ht
    rcases (mem_foldr_pairL u.1 t).1 ht with ⟨p, hp, hp'⟩
    subst hp'; exact Finset.mem_def.2 hp
  · intro ht
    exact (mem_foldr_pairL u.1 t).2 ⟨.pairL t, Finset.mem_def.1 ht, rfl⟩

private def pairRInsert : TreeToken α → Finset (TreeToken α) → Finset (TreeToken α)
  | .pairR t => insert t
  | .bot | .atom _ | .pairL _ => id

instance :
    LeftCommutative (pairRInsert : TreeToken α → Finset (TreeToken α) → Finset (
    TreeToken α)) :=
  ⟨fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl⟩

def pairRFinset (u : Finset (TreeToken α)) : Finset (TreeToken α) :=
  Multiset.foldr pairRInsert (∅ : Finset (TreeToken α)) u.1

private theorem mem_foldr_pairR (s : Multiset (TreeToken α)) (t : TreeToken α) :
    t ∈ Multiset.foldr pairRInsert (∅ : Finset (TreeToken α)) s ↔
      ∃ p ∈ s, p = .pairR t := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro ht; exact False.elim (Finset.notMem_empty t ht)
    · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
  · intro p rest ih
    cases p with
    | pairR a =>
      simp only [Multiset.foldr_cons, pairRInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      · rintro (ht | ⟨q, hq, hq'⟩)
        · exact ⟨.pairR a, Or.inl rfl, congrArg TreeToken.pairR ht.symm⟩
        · exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · injection hq' with ht; exact Or.inl ht.symm
        · exact Or.inr ⟨q, hq, hq'⟩
    | bot | atom _ | pairL _ =>
      simp only [Multiset.foldr_cons, pairRInsert, id_eq, ih, Multiset.mem_cons]
      constructor
```

```
      · rintro ⟨q, hq, hq'⟩; exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · exact False.elim (nomatch hq')
        · exact ⟨q, hq, hq'⟩

theorem mem_pairRFinset {u : Finset (TreeToken α)} {t : TreeToken α} :
    t ∈ pairRFinset u ↔ .pairR t ∈ u := by
  constructor
  · intro ht
    rcases (mem_foldr_pairR u.1 t).1 ht with ⟨p, hp, hp'⟩
    subst hp'; exact Finset.mem_def.2 hp
  · intro ht
    exact (mem_foldr_pairR u.1 t).2 ⟨.pairR t, Finset.mem_def.1 ht, rfl⟩

instance : LeftCommutative (atomInsert : TreeToken α → Finset α → Finset α) :=
  ⟨fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl⟩

def atomFinset (u : Finset (TreeToken α)) : Finset α :=
  Multiset.foldr atomInsert (∅ : Finset α) u.1

inductive TreeCon : Finset (TreeToken α) → Prop where
  | left (u : Finset (TreeToken α))
      (hA : atomFinset u ∈ A.Con)
      (hL : pairLFinset u = ∅) (hR : pairRFinset u = ∅) : TreeCon u
  | right (u : Finset (TreeToken α))
      (hA : atomFinset u = ∅)
      (hL : TreeCon (pairLFinset u)) (hR : TreeCon (pairRFinset u)) : TreeCon u

theorem TreeCon.atom_con_of_atoms {u : Finset (TreeToken α)}
    (hu : TreeCon A u) (hne : atomFinset u = ∅) : atomFinset u ∈ A.Con := by
  cases hu with
  | left _ hA _ _ => exact hA
  | right _ hA _ _ => exact False.elim (hne hA)

theorem TreeCon.atoms_empty_of_right {u : Finset (TreeToken α)}
    (hu : TreeCon A u) (hP : pairLFinset u = ∅ ∨ pairRFinset u = ∅) :
    atomFinset u = ∅ := by
  cases hu with
  | left _ _ hL hR =>
    rcases hP with h | h
    · exact False.elim (h hL)
    · exact False.elim (h hR)
  | right _ hA _ _ => exact hA

theorem TreeCon.pairs_empty_of_left {u : Finset (TreeToken α)}
    (hu : TreeCon A u) (hA : atomFinset u = ∅) :
    pairLFinset u = ∅ ∧ pairRFinset u = ∅ := by
  cases hu with
  | left _ _ hL hR => exact ⟨hL, hR⟩
  | right _ hAt _ _ => exact False.elim (hA hAt)

theorem atomFinset_empty : atomFinset (∅ : Finset (TreeToken α)) = ∅ := rfl

omit [DecidableEq α] in
```

```
theorem eq_singleton_bot_of_subset {s : Finset (TreeToken α)}
    (hne : s = ∅) (hsub : s ⊆ {.bot}) : s = {.bot} := by
  ext y
  constructor
  · exact fun hy => hsub hy
  · intro hy
    have : y = .bot := Finset.mem_singleton.1 hy
    subst this
    obtain ⟨z, hz⟩ := Finset.nonempty_of_ne_empty hne
    have hzbot : z = .bot := Finset.mem_singleton.1 (hsub hz)
    exact hzbot ▸ hz

theorem mem_foldr_atom (s : Multiset (TreeToken α)) (x : α) :
    x ∈ Multiset.foldr atomInsert (∅ : Finset α) s ↔ ∃ p ∈ s, p = .atom x := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro hx; exact False.elim (Finset.notMem_empty x hx)
    · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
  · intro p t ih
    cases p with
    | atom a =>
      simp only [Multiset.foldr_cons, atomInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      · rintro (hx | ⟨q, hq, hq'⟩)
        · exact ⟨.atom a, Or.inl rfl, congrArg TreeToken.atom hx.symm⟩
        · exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · injection hq' with hx; exact Or.inl hx.symm
        · exact Or.inr ⟨q, hq, hq'⟩
    | bot | pairL _ | pairR _ =>
      simp only [Multiset.foldr_cons, atomInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      · rintro ⟨q, hq, hq'⟩; exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · exact False.elim (nomatch hq')
        · exact ⟨q, hq, hq'⟩

theorem pairLFinset_empty : pairLFinset (∅ : Finset (TreeToken α)) = ∅ := rfl

theorem pairLFinset_insert_atom (u : Finset (TreeToken α)) (x : α) :
    pairLFinset (insert (.atom x) u) = pairLFinset u := by
  ext t; simp only [mem_pairLFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem pairLFinset_insert_bot (u : Finset (TreeToken α)) :
    pairLFinset (insert (.bot : TreeToken α) u) = pairLFinset u := by
  ext t; simp only [mem_pairLFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr
```

```
theorem pairLFinset_insert_pairL (u : Finset (TreeToken α)) (t : TreeToken α) :
    pairLFinset (insert (.pairL t) u) = insert t (pairLFinset u) := by
  ext s; simp only [Finset.mem_insert, mem_pairLFinset]
  constructor
  · rintro (h | h)
    · exact Or.inl (TreeToken.pairL.inj h)
    · exact Or.inr h
  · rintro (rfl | h)
    · exact Or.inl rfl
    · exact Or.inr h

theorem pairLFinset_insert_pairR (u : Finset (TreeToken α)) (t : TreeToken α) :
    pairLFinset (insert (.pairR t) u) = pairLFinset u := by
  ext s; simp only [mem_pairLFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem pairLFinset_mono {u v : Finset (TreeToken α)} (h : v ⊆ u) :
    pairLFinset v ⊆ pairLFinset u := fun _ ht => mem_pairLFinset.2 (h (mem_pairLFinset
    .1 ht))

theorem pairLFinset_singleton_atom (x : α) :
    pairLFinset ({.atom x} : Finset (TreeToken α)) = ∅ := by
  ext t
  constructor
  · intro ht
    exact False.elim (nomatch mem_pairLFinset.1 ht)
  · intro ht
    exact False.elim (Finset.notMem_empty t ht)

theorem pairLFinset_singleton_bot :
    pairLFinset ({.bot} : Finset (TreeToken α)) = ∅ := by
  ext t
  constructor
  · intro ht
    exact False.elim (nomatch mem_pairLFinset.1 ht)
  · intro ht
    exact False.elim (Finset.notMem_empty t ht)

theorem pairLFinset_singleton_pairL (t : TreeToken α) :
    pairLFinset ({.pairL t} : Finset (TreeToken α)) = {t} := by
  ext s; simp only [mem_pairLFinset, Finset.mem_singleton]
  exact ⟨fun h => TreeToken.pairL.inj h, fun h => congrArg TreeToken.pairL h⟩

theorem pairLFinset_singleton_pairR (t : TreeToken α) :
    pairLFinset ({.pairR t} : Finset (TreeToken α)) = ∅ := by
  ext s
  constructor
  · intro hs
    exact False.elim (nomatch mem_pairLFinset.1 hs)
  · intro hs
    exact False.elim (Finset.notMem_empty s hs)

theorem pairRFinset_empty : pairRFinset (∅ : Finset (TreeToken α)) = ∅ := rfl
```

```
theorem pairRFinset_insert_atom (u : Finset (TreeToken α)) (x : α) :
    pairRFinset (insert (.atom x) u) = pairRFinset u := by
  ext t; simp only [mem_pairRFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem pairRFinset_insert_bot (u : Finset (TreeToken α)) :
    pairRFinset (insert (.bot : TreeToken α) u) = pairRFinset u := by
  ext t; simp only [mem_pairRFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem pairRFinset_insert_pairL (u : Finset (TreeToken α)) (t : TreeToken α) :
    pairRFinset (insert (.pairL t) u) = pairRFinset u := by
  ext s; simp only [mem_pairRFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem pairRFinset_insert_pairR (u : Finset (TreeToken α)) (t : TreeToken α) :
    pairRFinset (insert (.pairR t) u) = insert t (pairRFinset u) := by
  ext s; simp only [Finset.mem_insert, mem_pairRFinset]
  constructor
  · rintro (h | h)
    · exact Or.inl (TreeToken.pairR.inj h)
    · exact Or.inr h
  · rintro (rfl | h)
    · exact Or.inl rfl
    · exact Or.inr h

theorem pairRFinset_mono {u v : Finset (TreeToken α)} (h : v ⊆ u) :
    pairRFinset v ⊆ pairRFinset u := fun _ ht => mem_pairRFinset.2 (h (mem_pairRFinset
    .1 ht))

theorem pairRFinset_singleton_atom (x : α) :
    pairRFinset ({.atom x} : Finset (TreeToken α)) = ∅ := by
  ext t
  constructor
  · intro ht
    exact False.elim (nomatch mem_pairRFinset.1 ht)
  · intro ht
    exact False.elim (Finset.notMem_empty t ht)

theorem pairRFinset_singleton_bot :
    pairRFinset ({.bot} : Finset (TreeToken α)) = ∅ := by
  ext t
  constructor
  · intro ht
    exact False.elim (nomatch mem_pairRFinset.1 ht)
  · intro ht
    exact False.elim (Finset.notMem_empty t ht)

theorem pairRFinset_singleton_pairL (t : TreeToken α) :
    pairRFinset ({.pairL t} : Finset (TreeToken α)) = ∅ := by
```

```
  ext s
  constructor
  · intro hs
    exact False.elim (nomatch mem_pairRFinset.1 hs)
  · intro hs
    exact False.elim (Finset.notMem_empty s hs)

theorem pairRFinset_singleton_pairR (t : TreeToken α) :
    pairRFinset ({.pairR t} : Finset (TreeToken α)) = {t} := by
  ext s; simp only [mem_pairRFinset, Finset.mem_singleton]
  exact ⟨fun h => TreeToken.pairR.inj h, fun h => congrArg TreeToken.pairR h⟩

def treeBot : TreeToken α := .bot

theorem mem_atomFinset {u : Finset (TreeToken α)} {x : α} :
    x ∈ atomFinset u ↔ .atom x ∈ u := by
  constructor
  · intro hx
    rcases (mem_foldr_atom u.1 x).1 hx with ⟨p, hp, hp'⟩
    subst hp'; exact Finset.mem_def.2 hp
  · intro hx
    exact (mem_foldr_atom u.1 x).2 ⟨.atom x, Finset.mem_def.1 hx, rfl⟩

theorem TreeCon_empty : TreeCon A (∅ : Finset (TreeToken α)) :=
  TreeCon.left _ (by rw [atomFinset_empty]; exact A.con_empty)
    pairLFinset_empty pairRFinset_empty

theorem atomFinset_insert_atom (u : Finset (TreeToken α)) (x : α) :
    atomFinset (insert (.atom x) u) = insert x (atomFinset u) := by
  ext y; simp only [Finset.mem_insert, mem_atomFinset]
  constructor
  · rintro (h | h)
    · exact Or.inl (TreeToken.atom.inj h)
    · exact Or.inr h
  · rintro (rfl | h)
    · exact Or.inl rfl
    · exact Or.inr h

theorem atomFinset_insert_bot (u : Finset (TreeToken α)) :
    atomFinset (insert (.bot : TreeToken α) u) = atomFinset u := by
  ext x; simp only [mem_atomFinset, Finset.mem_insert]
  constructor


  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem atomFinset_insert_pairL (u : Finset (TreeToken α)) (t : TreeToken α) :
    atomFinset (insert (.pairL t) u) = atomFinset u := by
  ext x; simp only [mem_atomFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem atomFinset_insert_pairR (u : Finset (TreeToken α)) (t : TreeToken α) :
    atomFinset (insert (.pairR t) u) = atomFinset u := by
  ext x; simp only [mem_atomFinset, Finset.mem_insert]
```

```
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem atomFinset_mono {u v : Finset (TreeToken α)} (h : v ⊆ u) :
    atomFinset v ⊆ atomFinset u := fun _ hx => mem_atomFinset.2 (h (mem_atomFinset.1
    hx))

theorem atomFinset_singleton_atom (x : α) :
    atomFinset ({.atom x} : Finset (TreeToken α)) = {x} := by
  ext y; simp only [mem_atomFinset, Finset.mem_singleton]
  exact ⟨fun h => TreeToken.atom.inj h, fun h => congrArg TreeToken.atom h⟩

theorem atomFinset_singleton_bot :
    atomFinset ({.bot} : Finset (TreeToken α)) = ∅ := by
  ext x
  constructor
  · intro hx
    exact False.elim (nomatch mem_atomFinset.1 hx)
  · intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem atomFinset_singleton_pairL (t : TreeToken α) :
    atomFinset ({.pairL t} : Finset (TreeToken α)) = ∅ := by
  ext x
  constructor
  · intro hx
    exact False.elim (nomatch mem_atomFinset.1 hx)
  · intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem atomFinset_singleton_pairR (t : TreeToken α) :
    atomFinset ({.pairR t} : Finset (TreeToken α)) = ∅ := by
  ext x
  constructor
  · intro hx
    exact False.elim (nomatch mem_atomFinset.1 hx)
  · intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem TreeCon.pairL_of_right {u : Finset (TreeToken α)}
    (hu : TreeCon A u) (_hA : atomFinset u = ∅) : TreeCon A (pairLFinset u) := by
  cases hu with
  | left _ _ hL _ => rw [hL]; exact TreeCon_empty A
  | right _ _ hL _ => exact hL

theorem TreeCon.pairR_of_right {u : Finset (TreeToken α)}
    (hu : TreeCon A u) (_hA : atomFinset u = ∅) : TreeCon A (pairRFinset u) := by
  cases hu with
  | left _ _ _ hR => rw [hR]; exact TreeCon_empty A
  | right _ _ _ hR => exact hR

theorem TreeCon_insert_bot {u : Finset (TreeToken α)} (hu : TreeCon A u) :
    TreeCon A (insert .bot u) := by
  cases hu with
  | left _ hA hL hR =>
```

```
    exact TreeCon.left _
      (by rw [atomFinset_insert_bot]; exact hA)
      (by rw [pairLFinset_insert_bot]; exact hL)
      (by rw [pairRFinset_insert_bot]; exact hR)
  | right _ hA hL hR =>
    exact TreeCon.right _
      (by rw [atomFinset_insert_bot]; exact hA)
      (by rw [pairLFinset_insert_bot]; exact hL)
      (by rw [pairRFinset_insert_bot]; exact hR)

theorem TreeCon_subset {u : Finset (TreeToken α)} (hu : TreeCon A u) :
    ∀ {v}, v ⊆ u → TreeCon A v := by
  induction hu with
  | left u hA hL hR =>
    intro v hv
    refine TreeCon.left v (A.con_subset hA (atomFinset_mono hv)) ?_ ?_
    · exact Finset.Subset.antisymm
        (fun t ht => by
          have : t ∈ pairLFinset u := pairLFinset_mono hv ht
          rw [hL] at this; exact False.elim (Finset.notMem_empty t this))
        (Finset.empty_subset _)
    · exact Finset.Subset.antisymm
        (fun t ht => by
          have : t ∈ pairRFinset u := pairRFinset_mono hv ht
          rw [hR] at this; exact False.elim (Finset.notMem_empty t this))
        (Finset.empty_subset _)
  | right u hA hL hR ihL ihR =>
    intro v hv
    refine TreeCon.right v
      (Finset.Subset.antisymm
        (fun x hx => by
          have : x ∈ atomFinset u := atomFinset_mono hv hx
          rw [hA] at this; exact False.elim (Finset.notMem_empty x this))
        (Finset.empty_subset _))
      (ihL (pairLFinset_mono hv)) (ihR (pairRFinset_mono hv))

theorem TreeCon_singleton (p : TreeToken α) : TreeCon A {p} := by
  induction p with
  | bot =>
    exact TreeCon.left _
      (by rw [atomFinset_singleton_bot]; exact A.con_empty)
      pairLFinset_singleton_bot pairRFinset_singleton_bot
  | atom x =>
    exact TreeCon.left _
      (by rw [atomFinset_singleton_atom]; exact A.con_sing x)
      (pairLFinset_singleton_atom x) (pairRFinset_singleton_atom x)
  | pairL t ih =>
    exact TreeCon.right _
      (atomFinset_singleton_pairL t)
      (by rw [pairLFinset_singleton_pairL]; exact ih)
      (by rw [pairRFinset_singleton_pairL]; exact TreeCon_empty A)
  | pairR t ih =>
    exact TreeCon.right _
      (atomFinset_singleton_pairR t)
      (by rw [pairLFinset_singleton_pairR]; exact TreeCon_empty A)
      (by rw [pairRFinset_singleton_pairR]; exact ih)
```

```
def TreeEntPayload (u : Finset (TreeToken α)) : TreeToken α → Prop
  | .bot => True
  | .atom x => atomFinset u = ∅ ∧ A.Ent (atomFinset u) x
  | .pairL t =>
      atomFinset u = ∅ ∧ TreeCon A (pairLFinset u) ∧ TreeCon A (pairRFinset u) ∧
        ((pairLFinset u = ∅ ∧ TreeEntPayload (pairLFinset u) t) ∨
          (t = .bot ∧ pairRFinset u = ∅))
  | .pairR t =>
      atomFinset u = ∅ ∧ TreeCon A (pairRFinset u) ∧ TreeCon A (pairLFinset u) ∧
        ((pairRFinset u = ∅ ∧ TreeEntPayload (pairRFinset u) t) ∨
          (t = .bot ∧ pairLFinset u = ∅))

theorem TreeEntPayload.of_mem {u : Finset (TreeToken α)} {p : TreeToken α}
    (hu : TreeCon A u) (hp : p ∈ u) : TreeEntPayload A u p := by
  induction p generalizing u with
  | bot => exact trivial
  | atom x =>
    have hx : x ∈ atomFinset u := mem_atomFinset.2 hp
    exact ⟨Finset.ne_empty_of_mem hx,
      A.ent_refl (TreeCon.atom_con_of_atoms A hu (Finset.ne_empty_of_mem hx)) hx⟩
  | pairL t ih =>
    have ht : t ∈ pairLFinset u := mem_pairLFinset.2 hp
    have hne := Finset.ne_empty_of_mem ht
    have hAt := TreeCon.atoms_empty_of_right A hu (Or.inl hne)
    exact ⟨hAt, TreeCon.pairL_of_right A hu hAt, TreeCon.pairR_of_right A hu hAt,
      Or.inl ⟨hne, ih (TreeCon.pairL_of_right A hu hAt) ht⟩⟩
  | pairR t ih =>
    have ht : t ∈ pairRFinset u := mem_pairRFinset.2 hp
    have hne := Finset.ne_empty_of_mem ht
    have hAt := TreeCon.atoms_empty_of_right A hu (Or.inr hne)
    exact ⟨hAt, TreeCon.pairR_of_right A hu hAt, TreeCon.pairL_of_right A hu hAt,
      Or.inl ⟨hne, ih (TreeCon.pairR_of_right A hu hAt) ht⟩⟩

theorem TreeEntPayload.of_no_atoms_no_pairs {u : Finset (TreeToken α)} {t : TreeToken
    α}
    (hAt : atomFinset u = ∅) (hL : pairLFinset u = ∅) (hR : pairRFinset u = ∅)
    (h : TreeEntPayload A u t) : t = .bot := by
  cases t with
  | bot => rfl
  | atom x => exact False.elim (h.1 hAt)
  | pairL t =>
    rcases h with ⟨_, _, _, hOr⟩
    rcases hOr with ⟨hne, _⟩ | ⟨_, hRne⟩
    · exact False.elim (hne hL)
    · exact False.elim (hRne hR)
  | pairR t =>
    rcases h with ⟨_, _, _, hOr⟩
    rcases hOr with ⟨hne, _⟩ | ⟨_, hLne⟩
    · exact False.elim (hne hR)
    · exact False.elim (hLne hL)

def TreeEnt (u : Finset (TreeToken α)) (p : TreeToken α) : Prop :=
  TreeCon A u ∧ TreeEntPayload A u p

theorem TreeEnt.of_mem {u : Finset (TreeToken α)} {p : TreeToken α}
```

```
    (hu : TreeCon A u) (hp : p ∈ u) : TreeEnt A u p :=
  ⟨hu, TreeEntPayload.of_mem A hu hp⟩

theorem TreeCon_insert_of_ent {u : Finset (TreeToken α)} {p : TreeToken α}
    (h : TreeEnt A u p) : TreeCon A (insert p u) := by
  induction p generalizing u with
  | bot =>
    exact TreeCon_insert_bot A h.1
  | atom x =>
    rcases h with ⟨hu, hne, hA⟩
    have hLR := TreeCon.pairs_empty_of_left A hu hne
    exact TreeCon.left _
      (by rw [atomFinset_insert_atom]; exact A.ent_con hA)
      (by rw [pairLFinset_insert_atom]; exact hLR.1)
      (by rw [pairRFinset_insert_atom]; exact hLR.2)
  | pairL t ih =>
    rcases h with ⟨_, hAt, hConL, hConR, hOr⟩
    refine TreeCon.right _
      (by rw [atomFinset_insert_pairL]; exact hAt) ?_
      (by rw [pairRFinset_insert_pairL]; exact hConR)
    rw [pairLFinset_insert_pairL]
    rcases hOr with ⟨_, hPay⟩ | ⟨rfl, _⟩
    · exact ih ⟨hConL, hPay⟩
    · exact TreeCon_insert_bot A hConL
  | pairR t ih =>
    rcases h with ⟨_, hAt, hConR, hConL, hOr⟩
    refine TreeCon.right _
      (by rw [atomFinset_insert_pairR]; exact hAt)
      (by rw [pairLFinset_insert_pairR]; exact hConL) ?_
    rw [pairRFinset_insert_pairR]
    rcases hOr with ⟨_, hPay⟩ | ⟨rfl, _⟩
    · exact ih ⟨hConR, hPay⟩
    · exact TreeCon_insert_bot A hConR

theorem pairL_ne_or_subset_bot {u v : Finset (TreeToken α)}
    (hEnts : ∀ y ∈ pairLFinset u, TreeEnt A v (.pairL y)) :
    pairLFinset v = ∅ ∨ pairLFinset u ⊆ {.bot} := by
  revert hEnts
  induction pairLFinset u using Finset.induction with
  | empty =>
    intro; exact Or.inr (Finset.empty_subset _)
  | insert y s hys ih =>
    intro hEnts
    have hy := hEnts y (Finset.mem_insert_self _ _)
    rcases hy.2.2.2.2 with ⟨hneV, _⟩ | ⟨rfl, _⟩
    · exact Or.inl hneV
    · rcases ih (fun z hz => hEnts z (Finset.mem_insert_of_mem hz)) with h | hsub
      · exact Or.inl h
      · refine Or.inr ?_
        intro z hz
        rcases Finset.mem_insert.1 hz with rfl | hz
        · exact Finset.mem_singleton_self _
        · exact hsub hz

theorem pairR_ne_or_subset_bot {u v : Finset (TreeToken α)}
    (hEnts : ∀ y ∈ pairRFinset u, TreeEnt A v (.pairR y)) :
```

```
    pairRFinset v = ∅ ∨ pairRFinset u ⊆ {.bot} := by
  revert hEnts
  induction pairRFinset u using Finset.induction with
  | empty =>
    intro; exact Or.inr (Finset.empty_subset _)
  | insert y s hys ih =>
    intro hEnts
    have hy := hEnts y (Finset.mem_insert_self _ _)
    rcases hy.2.2.2.2 with ⟨hneV, _⟩ | ⟨rfl, _⟩
    · exact Or.inl hneV
    · rcases ih (fun z hz => hEnts z (Finset.mem_insert_of_mem hz)) with h | hsub
      · exact Or.inl h
      · refine Or.inr ?_
        intro z hz
        rcases Finset.mem_insert.1 hz with rfl | hz
        · exact Finset.mem_singleton_self _
        · exact hsub hz

theorem TreeEntPayload_trans {u v : Finset (TreeToken α)} {c : TreeToken α}
    (hv : TreeCon A v) (hu : TreeCon A u)
    (hEnts : ∀ y ∈ u, TreeEnt A v y) (hPay : TreeEntPayload A u c) :
    TreeEntPayload A v c := by
  induction c generalizing u v with
  | bot => exact trivial
  | atom x =>
    rcases hPay with ⟨hne, hA⟩
    have hlft : ∀ y ∈ atomFinset u, A.Ent (atomFinset v) y := by
      intro y hy
      exact (hEnts _ (mem_atomFinset.1 hy)).2.2
    have hne' : atomFinset v = ∅ := by
      obtain ⟨y, hy⟩ := Finset.nonempty_of_ne_empty hne
      exact (hEnts _ (mem_atomFinset.1 hy)).2.1
    exact ⟨hne', A.ent_trans (TreeCon.atom_con_of_atoms A hv hne')
      (TreeCon.atom_con_of_atoms A hu hne) hlft hA⟩
  | pairL t ih =>
    rcases hPay with ⟨hAt, hConL, hConR, hOr⟩
    rcases hOr with ⟨hne, hPayL⟩ | ⟨rfl, hRne⟩
    · obtain ⟨s, hs⟩ := Finset.nonempty_of_ne_empty hne
      have hEy := hEnts _ (mem_pairLFinset.1 hs)
      have hAtv : atomFinset v = ∅ := hEy.2.1
      have hConLv : TreeCon A (pairLFinset v) := TreeCon.pairL_of_right A hv hAtv
      have hConRv : TreeCon A (pairRFinset v) := hEy.2.2.2.1
      have hEntsL : ∀ y ∈ pairLFinset u, TreeEnt A (pairLFinset v) y := by
        intro y hy
        have hEy' := hEnts _ (mem_pairLFinset.1 hy)
        rcases hEy'.2.2.2.2 with ⟨_, hPay'⟩ | ⟨rfl, _⟩
        · exact ⟨hEy'.2.2.1, hPay'⟩
        · exact ⟨hConLv, trivial⟩
      refine ⟨hAtv, hConLv, hConRv, ?_⟩
      have hEntsPair : ∀ y ∈ pairLFinset u, TreeEnt A v (.pairL y) :=
        fun y hy => hEnts _ (mem_pairLFinset.1 hy)
      rcases pairL_ne_or_subset_bot A hEntsPair with hneV | hsub
      · exact Or.inl ⟨hneV, ih hConLv hConL hEntsL hPayL⟩
      · have huBot : pairLFinset u = {.bot} :=
          eq_singleton_bot_of_subset hne hsub
        have ht : t = .bot :=
```

```
          TreeEntPayload.of_no_atoms_no_pairs A
            atomFinset_singleton_bot pairLFinset_singleton_bot
            pairRFinset_singleton_bot (huBot ▸ hPayL)
        rcases hEy.2.2.2.2 with ⟨hneV, _⟩ | ⟨_, hR⟩
        · exact Or.inl ⟨hneV, ih hConLv hConL hEntsL hPayL⟩
        · exact Or.inr ⟨ht, hR⟩
    · obtain ⟨s, hs⟩ := Finset.nonempty_of_ne_empty hRne
      have hEy := hEnts _ (mem_pairRFinset.1 hs)
      have hAtv : atomFinset v = ∅ := hEy.2.1
      refine ⟨hAtv, TreeCon.pairL_of_right A hv hAtv, hEy.2.2.1, ?_⟩
      rcases hEy.2.2.2.2 with ⟨hneR, _⟩ | ⟨_, hneL⟩
      · exact Or.inr ⟨rfl, hneR⟩
      · exact Or.inl ⟨hneL, trivial⟩
  | pairR t ih =>
    rcases hPay with ⟨hAt, hConR, hConL, hOr⟩
    rcases hOr with ⟨hne, hPayR⟩ | ⟨rfl, hLne⟩
    · obtain ⟨s, hs⟩ := Finset.nonempty_of_ne_empty hne
      have hEy := hEnts _ (mem_pairRFinset.1 hs)
      have hAtv : atomFinset v = ∅ := hEy.2.1
      have hConRv : TreeCon A (pairRFinset v) := TreeCon.pairR_of_right A hv hAtv
      have hConLv : TreeCon A (pairLFinset v) := hEy.2.2.2.1
      have hEntsR : ∀ y ∈ pairRFinset u, TreeEnt A (pairRFinset v) y := by
        intro y hy
        have hEy' := hEnts _ (mem_pairRFinset.1 hy)
        rcases hEy'.2.2.2.2 with ⟨_, hPay'⟩ | ⟨rfl, _⟩
        · exact ⟨hEy'.2.2.1, hPay'⟩
        · exact ⟨hConRv, trivial⟩
      refine ⟨hAtv, hConRv, hConLv, ?_⟩
      have hEntsPair : ∀ y ∈ pairRFinset u, TreeEnt A v (.pairR y) :=
        fun y hy => hEnts _ (mem_pairRFinset.1 hy)
      rcases pairR_ne_or_subset_bot A hEntsPair with hneV | hsub
      · exact Or.inl ⟨hneV, ih hConRv hConR hEntsR hPayR⟩
      · have huBot : pairRFinset u = {.bot} :=
          eq_singleton_bot_of_subset hne hsub
        have ht : t = .bot :=
          TreeEntPayload.of_no_atoms_no_pairs A
            atomFinset_singleton_bot pairLFinset_singleton_bot
            pairRFinset_singleton_bot (huBot ▸ hPayR)
        rcases hEy.2.2.2.2 with ⟨hneV, _⟩ | ⟨_, hL⟩
        · exact Or.inl ⟨hneV, ih hConRv hConR hEntsR hPayR⟩
        · exact Or.inr ⟨ht, hL⟩
    · obtain ⟨s, hs⟩ := Finset.nonempty_of_ne_empty hLne
      have hEy := hEnts _ (mem_pairLFinset.1 hs)
      have hAtv : atomFinset v = ∅ := hEy.2.1
      refine ⟨hAtv, TreeCon.pairR_of_right A hv hAtv, hEy.2.2.1, ?_⟩
      rcases hEy.2.2.2.2 with ⟨hneL, _⟩ | ⟨_, hneR⟩
      · exact Or.inr ⟨rfl, hneL⟩
      · exact Or.inl ⟨hneR, trivial⟩

theorem TreeEnt_trans {u v : Finset (TreeToken α)} {c : TreeToken α}
    (hv : TreeCon A v) (hu : TreeCon A u)
    (hEnts : ∀ y ∈ u, TreeEnt A v y) (hEntc : TreeEnt A u c) :
    TreeEnt A v c :=
  ⟨hv, TreeEntPayload_trans A hv hu hEnts hEntc.2⟩

end treeSystem
```

```
open treeSystem

def treeSystem : InfoSys (TreeToken α) where
  bot := treeBot
  Con := {u | TreeCon A u}
  Ent := TreeEnt A
  con_subset := fun hu hv => TreeCon_subset A hu hv
  con_sing := TreeCon_singleton A
  ent_con := fun h => TreeCon_insert_of_ent A h
  ent_bot := fun hu => ⟨hu, trivial⟩
  ent_refl := fun hu hp => TreeEnt.of_mem A hu hp
  ent_trans := fun hv hu hEnts hEntc => TreeEnt_trans A hv hu hEnts hEntc

def treeRhs : InfoSys (SumToken α (ProdToken (treeSystem A) (treeSystem A))) :=
  sumSystem A (productSystem (treeSystem A) (treeSystem A))

theorem treeRhs.eq_sum_product :
    treeRhs A = sumSystem A (productSystem (treeSystem A) (treeSystem A)) :=
  rfl

def treeUnfold (t : TreeToken α) :
    SumToken α (ProdToken (treeSystem A) (treeSystem A)) :=
  match t with
  | .bot => .bot
  | .atom x => .left x
  | .pairL t => .right ⟨(t, treeBot), ⟨Or.inr rfl⟩⟩
  | .pairR t => .right ⟨(treeBot, t), ⟨Or.inl rfl⟩⟩

theorem treeUnfold.atom (x : α) : treeUnfold A (.atom x) = SumToken.left x := rfl

theorem treeUnfold.bot : treeUnfold A treeBot = SumToken.bot := rfl

theorem treeUnfold.pairL (t : TreeToken α) :
    treeUnfold A (.pairL t) = .right ⟨(t, treeBot), ⟨Or.inr rfl⟩⟩ := rfl

theorem treeUnfold.pairR (t : TreeToken α) :
    treeUnfold A (.pairR t) = .right ⟨(treeBot, t), ⟨Or.inl rfl⟩⟩ := rfl

theorem treeUnfold.pairL_bot_eq_pairR_bot :
    treeUnfold A (.pairL treeBot) = treeUnfold A (.pairR treeBot) := rfl

end DomainTheory.InformationSystems.InfoSys
```

5.9.3 Example 8.1: A domain of trees or S-expressions

A domain of trees or S-expressions. This is everyone's favorite example. And a very nice example it is, but we should not think that it contains all the meat of the theory of domain equations. Even if we generalize the kinds of equations to contain all iterations of the domain constructs $+$ and $\times$, the full power of the method has not been exploited. We will try to make this clear in the further examples.

Let a domain (information system) $\mathbf{A}$ be given. What we want to construct is a domain $\mathbf{T}$ of "trees" built up from elements of $\mathbf{A}$ as "atoms". For simplicity we consider unlabelled binary trees here, but more complex trees are easy to accommodate. The domain equation we want to "solve" is this one:

$$\mathbf{T} \cong \mathbf{A} + (\mathbf{T} \times \mathbf{T}).$$

If such a domain exists, then we can say that (up to isomorphism) the elements of the domain $\mathbf{T}$ are either bottom, or elements of the given domain $\mathbf{A}$, or pairs of elements from the domain $\mathbf{T}$ itself. And these are the only kinds of elements that $\mathbf{T}$ has.

To prove that such a domain exists it is only necessary to ask what information has to be given about a prospective element. The answer may involve us in a regress, but the running backwards need not be infinite—at least for the finite elements. As we shall see, the infinite elements of $\mathbf{T}$ can be self-replicating; but, to define a domain fully, all we have to do is to build up the finite elements out of the data objects in a systematic way. Fortunately, in order to satisfy the above equation, the required closure conditions on data objects are simple to achieve.

In the first place, we need copies of all the data objects of $\mathbf{A}$ to put into the sum. The easy way to do this is to take an object Δ not in $D_{\mathbf{A}}$, and to let, by definition,

$$\Delta_{\mathbf{T}} = (\Delta, \Delta).$$

That gives us one member of $D_{\mathbf{T}}$, the one we always have to have in any case. The copy of an $X \in D_{\mathbf{A}}$ is just going to be (X, Δ). The other members of $D_{\mathbf{T}}$ will be of the form (Δ, U), where U gives us information about the other kind of elements of $\mathbf{T}$. The point is that $\mathbf{T}$ has to be a sum, and we are using just the scheme of Definition 6.3 to set this up.

Next we have to think what kind of information the U above should contain. Because we want a product, we refer back to Definition 6.1 and imagine we have already defined $D_{\mathbf{T}}$. What 6.1(i) suggests is that we throw in a bunch of other data objects into $D_{\mathbf{T}}$. The only point that needs care is that the data objects for the product must be copied into the overall sum. With this in mind, the following clauses give us the inductive definition of $D_{\mathbf{T}}$:

(1) $\Delta_{\mathbf{T}} \in D_{\mathbf{T}}$;

(2) $(X, \Delta) \in D_{\mathbf{T}}$ whenever $X \in D_{\mathbf{A}}$; and

(3) $(\Delta, (Y, \Delta_{\mathbf{T}})) \in D_{\mathbf{T}}$, and $(\Delta, (\Delta_{\mathbf{T}}, Z)) \in D_{\mathbf{T}}$ whenever $Y, Z \in D_{\mathbf{T}}$.

Of course, when we say "inductive definition," we mean that $D_{\mathbf{T}}$ is the least class satisfying (1)–(3). By standard arguments it can be shown that $D_{\mathbf{T}}$ satisfies this set-theoretical equation:

$$D_{\mathbf{T}} = \{\Delta_{\mathbf{T}}\} \cup \{(X, \Delta) \mid X \in D_{\mathbf{A}}\} \cup \{(\Delta, (Y, \Delta_{\mathbf{T}})) \mid Y \in D_{\mathbf{T}}\} \cup \{(\Delta, (\Delta_{\mathbf{T}}, Z)) \mid Z \in D_{\mathbf{T}}\}.$$

In fact, with some very mild assumptions about ordered pairs in set theory, $D_{\mathbf{T}}$ is the only solution to the above equation.

Defining the data objects is but part of the story: the same data objects can enter into quite different information systems. Data objects are just "tokens" and are only given "meaning" when $\mathrm{Con}_{\mathbf{T}}$ and $\vdash_{\mathbf{T}}$ are defined. Let us consider the problem of consistency first. We already understand the notion as it applies to sum and product systems, so we must merely copy over the parts of the previous definitions in the right position for the definition of $\mathrm{Con}_{\mathbf{T}}$. There are two forms we could give this definition; perhaps the best is the inductive one. We have:

(4) $\emptyset \in \mathrm{Con}_{\mathbf{T}}$;

(5) $u \cup \{\Delta_{\mathbf{T}}\} \in \mathrm{Con}_{\mathbf{T}}$ whenever $u \in \mathrm{Con}_{\mathbf{T}}$;

(6) $\{(X, \Delta) \mid X \in w\} \in \mathrm{Con}_{\mathbf{T}}$ whenever $w \in \mathrm{Con}_{\mathbf{A}}$;

(7) $\{(\Delta, (Y, \Delta_{\mathbf{T}})) \mid Y \in u\} \cup \{(\Delta, (\Delta_{\mathbf{T}}, Z)) \mid Z \in v\} \in \mathrm{Con}_{\mathbf{T}}$ whenever $u, v \in \mathrm{Con}_{\mathbf{T}}$.

Conditions (4)–(7) certainly make the inductive character of $\mathrm{Con}_{\mathbf{T}}$ clear—again, let us emphasize, the set being specified is the least such. Also clear from the definition is the fact that a consistent set of $\mathbf{T}$—aside from containing $\Delta_{\mathbf{T}}$—is either a copy of a consistent set of $\mathbf{A}$ or a copy of a consistent set of $\mathbf{T} \times \mathbf{T}$. We could thus state a set-theoretical equation for $\mathrm{Con}_{\mathbf{T}}$ similar to the one for $D_{\mathbf{T}}$.

It remains to define entailment for $\mathbf{T}$. Here are the inductive clauses which are pretty much forced on us by our objective of solving the domain equation:

(8) $u \vdash_{\mathbf{T}} \Delta_{\mathbf{T}}$ always;

(9) $u \cup \{\Delta_{\mathbf{T}}\} \vdash_{\mathbf{T}} Y$ whenever $u \vdash_{\mathbf{T}} Y$;

(10) $\{(X, \Delta) \mid X \in w\} \vdash_{\mathbf{T}} (W, \Delta)$ whenever $w \vdash_{\mathbf{A}} W$;

(11) $\{(\Delta, (Y, \Delta_{\mathbf{T}})) \mid Y \in u\} \cup \{(\Delta, (\Delta_{\mathbf{T}}, Z)) \mid Z \in v\} \vdash_{\mathbf{T}} (\Delta, (X, \Delta_{\mathbf{T}}))$ whenever $u \vdash_{\mathbf{T}} X$ and $v \in \mathrm{Con}_{\mathbf{T}}$; and

(12) $\{(\Delta, (Y, \Delta_{\mathbf{T}})) \mid Y \in u\} \cup \{(\Delta, (\Delta_{\mathbf{T}}, Z)) \mid Z \in v\} \vdash_{\mathbf{T}} (\Delta, (\Delta_{\mathbf{T}}, X))$ whenever $v \vdash_{\mathbf{T}} X$ and $u \in \mathrm{Con}_{\mathbf{T}}$.

Inductive definitions engender inductive proofs. It now has to be checked that consistency and entailment for $\mathbf{T}$ satisfy the axioms of 2.1. The steps needed for this check are mechanical. (The proof may be aided by noting that the cases in (4)–(7) and in (8)–(12) are disjoint—except for a trivial overlap between (8) and (9). The cases get invoked typically by asking, when confronted with an entailment to prove, for the nature of the data object on the right of the turnstile.)

Having defined and verified that $\mathbf{T}$ is an information system, the validity of the domain equation for $\mathbf{T}$ is secured by forming the right-hand side and noting that $\mathbf{T}$ is *identical* to $\mathbf{A} + (\mathbf{T} \times \mathbf{T})$. The reason is that we carefully chose the notation to match the official definitions of sums and products. (In general, in solving domain equations some transformation might have to take place to "re-format" data objects if things are not set up to be literally the same.)

It should be remarked that the sense can be made precise in which $\mathbf{T}$ is the *least* solution of the given domain equation. (It is an initial algebra in a suitable category of algebras and algebra homomorphisms.) It is pretty obvious that it is minimal in some sense, because we put into it only what was strictly required by the problem and nothing more.

It is also fairly obvious that there are *many* solutions to this domain equation. A non-constructive way to obtain non-minimal solutions is to interpret the whole construction of $\mathbf{T}$ in a non-standard model of set theory. Though, in the definition of $D_{\mathbf{T}}$, it looks like we are only working with very finite objects, everything we did could be made abstract and could be carried out in some funny universe. The result would be a system of "finite" data objects having all the right formal properties but containing things not in the standard minimal system. We would then take the notions of consistency and entailment that also exist in the funny universe and

restrict them to sets of data objects that are actually finite in the standard sense. It can be seen from the formal properties of the construction that the resulting notions satisfy our axioms for an information system

and that the domain equation holds—BUT the system would have many different elements beyond what we put into the original $\mathbf{T}$. To make this construction work, by the way, we would have to force $\mathbf{A}$ to be absolute in the model: if it is actually finite (say, $\mathbf{A} = \mathbf{BOOL}$), then there is no problem. (Constructive methods for introducing "nonstandard" data objects can also be given.)

Finally, we must remark on why we called $\mathbf{T}$ a domain of S-expressions. The answer becomes clear when we structure $\mathbf{T}$ as an algebra. First, there is an approximable mapping

$$\text{atom} : \mathbf{A} \to \mathbf{T},$$

which injects $\mathbf{A}$ into $\mathbf{T}$ making the elements of $\mathbf{A}$ "atoms" of $\mathbf{T}$. Then there is a truth-valued predicate on $\mathbf{T}$ which decides whether an element is an atom:

$$\text{isatom} : \mathbf{T} \to \mathbf{BOOL}.$$

Finally, since $\mathbf{T} \times \mathbf{T}$ is a part of $\mathbf{T}$, we can redefine the paring functions so that:

$$\text{pair} : \mathbf{T} \times \mathbf{T} \to \mathbf{T}, \quad \text{fst} : \mathbf{T} \to \mathbf{T}, \quad \text{and snd} : \mathbf{T} \to \mathbf{T}.$$

In LISP terminology, these operations are the same as the familiar cons, car, and cdr. This makes $\mathbf{T}$ into an algebra where, starting from atoms, elements—expressions—can be built up by iterated pairing.

But why is our system different from the usual way of regarding S-expressions? The answer is that by including partial expressions (those involving $\Delta_{\mathbf{T}}$) and by completing the domain with limits, *infinite* expressions are introduced. For instance, if $a \in |\mathbf{T}|$, then we can solve the fixed-point equation:

$$x = \text{pair}(\text{atom}(a), x),$$

which is an infinite list of a's. This is but one example; the possibilities have been discussed in many papers too numerous to mention here.

As is common to remark, S-expressions can also be thought of as trees: the parse tree that gives the grammatical form of the expression. What we have added to the idea of a tree is the possibility of having infinite trees, and having all these trees as elements of a domain. ■

Theorem (`example_8_1`). The right-hand side of the tree equation is the sum of the given domain with the product of the tree domain with itself: $\mathbf{T}$ and $\mathbf{A} + (\mathbf{T} \times \mathbf{T})$ have the same information system of tokens.

Proof. The right-hand side is defined to be that sum, so the two systems agree. ?

`example_8_1.lean`

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.treeSystem

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

open treeSystem

theorem example_8_1 :
    ∀ {α : Type*} [DecidableEq α] (A : InfoSys α),
      treeRhs A = sumSystem A (productSystem (treeSystem A) (treeSystem A)) :=
```

```
  @treeRhs.eq_sum_product

end DomainTheory.InformationSystems.InfoSys
```

5.9.4 Lambda information system

Definition (`RawLamToken` equality). Equality of raw lambda tokens is decidable, by the boolean equality on those tokens.

Definition (`lamAtomInsert` commutative). Insertion of lambda atoms commutes.

Proof. By cases on the two tokens and commutativity of finite-set insertion. ?

Definition (`lamFunInsert` commutative). Insertion of function tokens commutes.

Proof. By cases on the two tokens and commutativity of finite-set insertion. ?

Definition (`maxDepthInsert` commutative). The binary maximum on depths commutes in its first arguments.

Proof. By commutativity of the maximum of natural numbers. ?

Definition (`LamToken` equality). Equality of lambda tokens is decidable.

Proof. It is equality of the underlying raw tokens. ?

Definition (`lamForgetInsert` commutative). Forgetful insertion commutes.

Proof. By cases on the two tokens and commutativity of finite-set insertion. ?

Definition (`LamDToken` equality). Equality of depth-bounded lambda tokens is decidable.

Proof. It is equality of the underlying tokens. ?

Definition (`lamDForgetInsert` commutative). Depth-bounded forgetful insertion commutes.

Proof. By cases on the two tokens and commutativity of finite-set insertion. ?

Definition (`RawLamToken`). A raw lambda token is bottom, an atom, or a function token pairing a list of input tokens with a list of output tokens.

Definition (`rawLamBEq`). The raw lambda boolean equality.

Definition (`rawLamListBEq`). The raw lambda List boolean equality.

Theorem (`rawLamBEq_eq`). The equality raw lambda boolean equality eq holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`rawLamListBEq_eq`). The equality raw lambda List boolean equality eq holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`lamAtomInsert`). The lambda Atom Insert adds the payload of the matching constructor and leaves every other constructor unchanged.

Definition (`lamFunInsert`). The lambda function Insert adds the payload of the matching constructor and leaves every other constructor unchanged.

Definition (`listToFinset`). The list To finite set.

Definition (`lamAtomFinset`). The lambda Atom finite set.

Definition (`lamFunFinset`). The lambda function finite set.

Definition (`lamInUnionInsert`). The lambda In Union Insert adds the payload of the matching constructor and leaves every other constructor unchanged.

Definition (`lamOutUnionInsert`). The lambda Out Union Insert adds the payload of the matching constructor and leaves every other constructor unchanged.

Theorem (`mem_listToFinset`). A token lies in the mem list To finite set exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_lamFunFinset`). A token lies in the mem lambda function finite set exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`[DecidableEq`). There is a type-class instance for [Decidable Eq.

Definition (`lamInputUnion`). The lambda Input Union.

Definition (`lamOutputUnion`). The lambda Output Union.

Theorem (`max_le_max_nat`). The statement max le max nat holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`rawDepth`). The raw Depth.

Theorem (`mem_lamInputUnion`). A token lies in the mem lambda Input Union exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_lamOutputUnion`). A token lies in the mem lambda Output Union exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`foldl_max_ge_init`). The statement foldl max ge init holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`foldl_max_mono_init`). The operation foldl max mono init preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`maxDepthInsert`). The max Depth Insert adds the payload of the matching constructor and leaves every other constructor unchanged.

Theorem (`foldl_list_le_of_mem`). The statement foldl list le of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`rawDepth_funTok_gt_left`). The statement raw Depth fun Tok gt left holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`rawDepth_funTok_gt_right`). The statement raw Depth fun Tok gt right holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`foldr_max_le_of_forall`). The statement foldr max le of forall holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`foldr_max_lt_of_forall`). The statement foldr max lt of forall holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`finsetMaxDepth`). The finset Max Depth.

Theorem (`finsetMaxDepth.listToFinset_le`). The statement list To finite set le holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`finsetMaxDepth.list_lt_funTok`). The statement list lt fun Tok holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`le_finsetMaxDepth_of_mem`). The statement le finset Max Depth of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`finsetMaxDepth.inputUnion_lt`). The operation input Union lt preserves union.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`finsetMaxDepth.outputUnion_lt`). The operation output Union lt preserves union.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`finsetMaxDepth.list_lt_funTok_out`). The statement list lt fun Tok out holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`finsetMaxDepth.of_mem_funTok`). The statement of mem fun Tok holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`LamConDepth`). The lambda Con Depth.

Definition (`LamEntDepth`). The lambda Ent Depth are given by the listed constructors.

Theorem (`LamEntDepth_bot`). The statement lambda Ent Depth bot holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamEntDepth.con`). The statement con holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`LamConN`). The lambda Con N.

Theorem (`lamAtomFinset_empty`). The lambda Atom finite set empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamFunFinset_empty`). The lambda function finite set empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamFunFinset_insert_atom`). Inserting the indicated constructor updates lambda function finite set insert atom by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamFunFinset_insert_bot`). Inserting the indicated constructor updates lambda function finite set insert bot by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamFunFinset_insert_funTok`). Inserting the indicated constructor updates lambda function finite set insert fun Tok by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamFunFinset_mono`). The operation lambda function finite set mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamInputUnion_empty`). The lambda Input Union empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamOutputUnion_empty`). The lambda Output Union empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_lamAtomFinset`). A token lies in the mem lambda Atom finite set exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`LamCon`). The lambda Con.

Theorem (`lamAtomFinset_insert_atom`). Inserting the indicated constructor updates lambda Atom finite set insert atom by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamAtomFinset_insert_bot`). Inserting the indicated constructor updates lambda Atom finite set insert bot by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamAtomFinset_insert_funTok`). Inserting the indicated constructor updates lambda Atom finite set insert fun Tok by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamAtomFinset_mono`). The operation lambda Atom finite set mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`IsLamToken`). The is lambda Token are given by the listed constructors.

Definition (`LamToken`). The lambda Token.

Definition (`lamBot`). The lambda Bot.

Definition (`lamForget`). The lambda Forget.

Definition (`lamForgetInsert`). The lambda Forget Insert adds the payload of the matching constructor and leaves every other constructor unchanged.

Theorem (`LamConDepth_empty`). The lambda Con Depth empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConDepth_of_fun_empty`). The lambda Con Depth of fun empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConDepth_of_fun_nonempty`). The lambda Con Depth of fun nonempty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamEntDepth_weaken_fun`). The statement lambda Ent Depth weaken fun holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`LamEntN`). The lambda Ent N.

Definition (`LamWF`). The lambda WF are given by the listed constructors.

Theorem (`filter_ne_subset_of_subset_insert_lam`). The property filter ne subset of subset insert lambda passes to subsets.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`finsetMaxDepth_funion_le`). The operation finset Max Depth union le preserves union.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`finsetMaxDepth_insert_le`). Inserting the indicated constructor updates finset Max Depth insert le by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`funion_assoc_lam`). The operation union assoc lambda preserves union.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`funion_comm_lam`). The operation union comm lambda preserves union.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`funion_empty_right_lam`). The union empty right lambda of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`insert_filter_ne_lam`). Inserting the indicated constructor updates insert filter ne lambda by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamAtomFinset_singleton_funTok`). On a singleton, lambda Atom finite set singleton fun Tok returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`lamForgetFinset`). The lambda Forget finite set.

Theorem (`lamFunFinset_funion`). The operation lambda function finite set union preserves union.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamFunFinset_singleton_funTok`). On a singleton, lambda function finite set singleton fun Tok returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamInputUnion_funion`). The operation lambda Input Union union preserves union.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamInputUnion_insert`). Inserting the indicated constructor updates lambda Input Union insert by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamInputUnion_singleton`). On a singleton, lambda Input Union singleton returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamOutputUnion_funion`). The operation lambda Output Union union preserves union.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamOutputUnion_insert`). Inserting the indicated constructor updates lambda Output Union insert by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamOutputUnion_singleton`). On a singleton, lambda Output Union singleton returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConDepth_insert_atom`). Inserting the indicated constructor updates lambda Con Depth insert atom by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConDepth_singleton_atom`). On a singleton, lambda Con Depth singleton atom returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConDepth_singleton_bot`). On a singleton, lambda Con Depth singleton bot returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamEntDepth_weaken_atom`). The statement lambda Ent Depth weaken atom holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConDepth_insert_bot`). Inserting the indicated constructor updates lambda Con Depth insert bot by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConDepth_mono`). The operation lambda Con Depth mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`LamEnt`). The lambda Ent.

Definition (`LamDToken`). The lambda DToken.

Definition (`LamTokenCon`). The lambda Token Con.

Theorem (`LamEntDepth_of_mem`). The statement lambda Ent Depth of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamEntDepth_weaken_insert`). Inserting the indicated constructor updates lambda Ent Depth weaken insert by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`LamTokenEnt`). The lambda Token Ent.

Theorem (`LamConDepth_funion_of_entSet`). The operation lambda Con Depth union of ent Set preserves union.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`lamDBot`). The lambda DBot.

Definition (`lamDForget`). The lambda DForget.

Theorem (`LamConDepth_insert_of_ent`). Inserting the indicated constructor updates lambda Con Depth insert of ent by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`lamDForgetInsert`). The lambda DForget Insert adds the payload of the matching constructor and leaves every other constructor unchanged.

Theorem (`LamEntDepth_trans`). The relation lambda Ent Depth trans is transitive.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`IsLamWF_of_mem`). The statement is lambda WF of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConDepth_singleton_funTok`). On a singleton, lambda Con Depth singleton fun Tok returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`listToLamDFinset`). The list To lambda Dfinite set.

Theorem (`mem_foldr_lamDForget`). A token lies in the mem foldr lambda DForget exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_listToLamDFinset`). A token lies in the mem list To lambda Dfinite set exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`lamDForgetFinset`). The lambda DForget finite set.

Theorem (`mem_lamDForgetFinset`). A token lies in the mem lambda DForget finite set exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamDForgetFinset.insert`). Inserting the indicated constructor updates insert by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamDForgetFinset.mono`). The operation mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamDForgetFinset.singleton`). On a singleton, singleton returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamDForget_listToLamDFinset`). The statement lambda DForget list To lambda Dfinite set holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`LamDCon`). The lambda DCon.

Theorem (`LamDCon.singleton`). On a singleton, singleton returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamDCon.listToLamDFinset`). The statement list To lambda Dfinite set holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamDCon.subset`). The property subset passes to subsets.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`LamDEnt`). The lambda DEnt.

Theorem (`LamDEnt.bot`). The statement bot holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamDEnt.of_mem`). The statement of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamDEnt.trans`). The relation trans is transitive.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamDCon_insert_of_ent`). Inserting the indicated constructor updates lambda DCon insert of ent by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`lambdaSystem`). The lambda domain over $\mathbf{A}$ is the information system of well-formed lambda tokens, with lambda consistency and lambda entailment.

Definition (`lamRhs`). The right-hand side of the lambda equation is the sum of $\mathbf{A}$ with the function space of the lambda domain with itself.

Definition (`lamUnfold`). Unfolding a lambda token views it as a token of $\mathbf{A} + (\mathbf{D} \to \mathbf{D})$.

Theorem (`mem_foldr_lamForget`). A token lies in the mem foldr lambda Forget exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`funion_insert_lam`). Inserting the indicated constructor updates union insert lambda by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_atom_con`). The statement lambda Con N atom con holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_atom_empty`). The lambda Con N atom empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_down_atom`). The statement lambda Con N down atom holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_empty`). The lambda Con N empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_exists_FunCon`). The statement lambda Con N exists function Con holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_fun_FunCon_right`). The statement lambda Con N fun function Con right holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_fun_empty`). The lambda Con N fun empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_insert_bot`). Inserting the indicated constructor updates lambda Con N insert bot by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_mono`). The operation lambda Con N mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamCon_empty`). The lambda Con empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamCon_singleton_atom`). On a singleton, lambda Con singleton atom returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamCon_singleton_bot`). On a singleton, lambda Con singleton bot returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamCon_subset`). The property lambda Con subset passes to subsets.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamEntN_con`). The statement lambda Ent N con holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamEntN_mono`). The operation lambda Ent N mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamEnt_bot`). The statement lambda Ent bot holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamAtomFinset_funion`). The operation lambda Atom finite set union preserves union.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamDForgetFinset_empty`). The lambda DForget finite set empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamForgetFinset_empty`). The lambda Forget finite set empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamRhs_eq_sum_function`). The equality lambda Rhs eq sum function holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamUnfold_atom`). The statement lambda Unfold atom holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamUnfold_bot`). The statement lambda Unfold bot holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_lamForgetFinset`). A token lies in the mem lambda Forget finite set exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_FunCon_mono_atom`). The operation lambda Con N function Con mono atom preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_fun_wf`). The statement lambda Con N fun wf holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_insert_of_ent_atom`). Inserting the indicated constructor updates lambda Con N insert of ent atom by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamCon_singleton_funTok`). On a singleton, lambda Con singleton fun Tok returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_insert_of_ent_bot`). Inserting the indicated constructor updates lambda Con N insert of ent bot by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamEnt_con`). The statement lambda Ent con holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamTokenEnt_bot`). The statement lambda Token Ent bot holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamDCon_empty`). The lambda DCon empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamTokenCon_empty`). The lambda Token Con empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamTokenCon_subset`). The property lambda Token Con subset passes to subsets.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamForgetFinset_insert`). Inserting the indicated constructor updates lambda Forget finite set insert by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lamForgetFinset_singleton`). On a singleton, lambda Forget finite set singleton returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamConN_fun_FunCon_atom`). The statement lambda Con N fun function Con atom holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamEntN_of_mem`). The statement lambda Ent N of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamCon_of_forget_singleton`). On a singleton, lambda Con of forget singleton returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamEnt_of_mem`). The statement lambda Ent of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamTokenCon_singleton`). On a singleton, lambda Token Con singleton returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`LamTokenEnt_of_mem`). The statement lambda Token Ent of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`lambdaSystem_con_axioms`). The statement lambdabda System con axioms holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

`lambdaSystem.lean`

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    functionSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.Constructive

variable {α : Type*} [DecidableEq α]
variable (A : InfoSys α)

namespace lambdaSystem

inductive RawLamToken (α : Type*) where
```

```
  | bot : RawLamToken α
  | atom : α → RawLamToken α
  | funTok : List (RawLamToken α) → List (RawLamToken α) → RawLamToken α

mutual
  def rawLamBEq : RawLamToken α → RawLamToken α → Bool
    | .bot, .bot => true
    | .atom x, .atom y => decide (x = y)
    | .funTok u v, .funTok u' v' => rawLamListBEq u u' && rawLamListBEq v v'
    | _, _ => false
  def rawLamListBEq : List (RawLamToken α) → List (RawLamToken α) → Bool
    | [], [] => true
    | x :: xs, y :: ys => rawLamBEq x y && rawLamListBEq xs ys
    | _, _ => false
end

mutual
  theorem rawLamBEq_eq : ∀ a b : RawLamToken α, rawLamBEq a b = true ↔ a = b
  | .bot, .bot => by simp [rawLamBEq]
  | .atom x, .atom y => by
      simp only [rawLamBEq, decide_eq_true_eq]
      exact ⟨congrArg RawLamToken.atom, RawLamToken.atom.inj⟩
  | .funTok u v, .funTok u' v' => by
      simp only [rawLamBEq, Bool.and_eq_true]
      constructor
      · intro ⟨hu, hv⟩
        rw [(rawLamListBEq_eq u u').1 hu, (rawLamListBEq_eq v v').1 hv]
      · intro h
        injection h with hu hv
        subst hu; subst hv
        exact ⟨(rawLamListBEq_eq u u).2 rfl, (rawLamListBEq_eq v v).2 rfl⟩
  | .bot, .atom _ | .bot, .funTok _ _ | .atom _, .bot | .atom _, .funTok _ _
  | .funTok _ _, .bot | .funTok _ _, .atom _ => by
      constructor
      · intro h; simp [rawLamBEq] at h
      · intro h; cases h
  theorem rawLamListBEq_eq : ∀ u v : List (RawLamToken α), rawLamListBEq u v = true ↔
    u = v
  | [], [] => by simp [rawLamListBEq]
  | x :: xs, y :: ys => by
      simp only [rawLamListBEq, Bool.and_eq_true]
      constructor
      · intro ⟨hx, hxs⟩
        rw [(rawLamBEq_eq x y).1 hx, (rawLamListBEq_eq xs ys).1 hxs]
      · intro h
        injection h with hx hxs
        subst hx; subst hxs
        exact ⟨(rawLamBEq_eq x x).2 rfl, (rawLamListBEq_eq xs xs).2 rfl⟩
  | [], _ :: _ | _ :: _, [] => by
      constructor
      · intro h; simp [rawLamListBEq] at h
      · intro h; cases h
end

instance : DecidableEq (RawLamToken α) := fun a b =>
  if h : rawLamBEq a b = true then isTrue ((rawLamBEq_eq a b).1 h)
```

```
  else isFalse fun hab => h ((rawLamBEq_eq a b).2 hab)

def lamAtomInsert : RawLamToken α → Finset α → Finset α
  | .atom x => insert x
  | .bot | .funTok _ _ => id

def lamFunInsert :
    RawLamToken α → Finset (List (RawLamToken α) × List (RawLamToken α)) →
      Finset (List (RawLamToken α) × List (RawLamToken α))
  | .funTok u v => insert (u, v)
  | .bot | .atom _ => id

def listToFinset {β : Type*} [DecidableEq β] : List β → Finset β
  | [] => ∅
  | x :: xs => insert x (listToFinset xs)

instance : LeftCommutative (lamAtomInsert : RawLamToken α → Finset α → Finset α) :=
  ⟨fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl⟩

instance : LeftCommutative
    (lamFunInsert : RawLamToken α →
      Finset (List (RawLamToken α) × List (RawLamToken α)) →
        Finset (List (RawLamToken α) × List (RawLamToken α))) :=
  ⟨fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl⟩

def lamAtomFinset (u : Finset (RawLamToken α)) : Finset α :=
  Multiset.foldr lamAtomInsert ∅ u.1

def lamFunFinset (u : Finset (RawLamToken α)) :
    Finset (List (RawLamToken α) × List (RawLamToken α)) :=
  Multiset.foldr lamFunInsert ∅ u.1

def lamInUnionInsert [DecidableEq (RawLamToken α)] :
    List (RawLamToken α) × List (RawLamToken α) → Finset (RawLamToken α) →
      Finset (RawLamToken α)
  | p, acc => listToFinset p.1 ∪' acc

def lamOutUnionInsert [DecidableEq (RawLamToken α)] :
    List (RawLamToken α) × List (RawLamToken α) → Finset (RawLamToken α) →
      Finset (RawLamToken α)
  | p, acc => listToFinset p.2 ∪' acc

theorem mem_listToFinset {β : Type*} [DecidableEq β] {x : β} {xs : List β} :
    x ∈ listToFinset xs ↔ x ∈ xs := by
  induction xs with
  | nil =>
    constructor
    · intro h; exact False.elim (Finset.notMem_empty x h)
    · intro h; cases h
  | cons a as ih =>
    simp only [listToFinset, Finset.mem_insert, List.mem_cons, ih]

theorem mem_lamFunFinset {u : Finset (RawLamToken α)}
    {p : List (RawLamToken α) × List (RawLamToken α)} :
    p ∈ lamFunFinset u ↔ .funTok p.1 p.2 ∈ u := by
  have aux (s : Multiset (RawLamToken α)) :
```

```
      p ∈ Multiset.foldr lamFunInsert ∅ s ↔ ∃ t ∈ s, t = .funTok p.1 p.2 := by
    refine Multiset.induction_on s ?_ ?_
    · constructor
      · intro hp; exact False.elim (Finset.notMem_empty p hp)
      · rintro ⟨_, ht, _⟩; exact False.elim (by cases ht)
    · intro t rest ih
      cases t with
      | funTok u v =>
        simp only [Multiset.foldr_cons, lamFunInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
        constructor
        · rintro (hp | ⟨q, hq, hq'⟩)
          · exact ⟨.funTok u v, Or.inl rfl, by cases hp; rfl⟩
          · exact ⟨q, Or.inr hq, hq'⟩
        · rintro ⟨q, hq, hq'⟩
          rcases hq with rfl | hq
          · injection hq' with hu hv
            exact Or.inl (Prod.ext hu.symm hv.symm)
          · exact Or.inr ⟨q, hq, hq'⟩
      | bot | atom _ =>
        simp only [Multiset.foldr_cons, lamFunInsert, id_eq, ih, Multiset.mem_cons]
        constructor
        · rintro ⟨q, hq, hq'⟩; exact ⟨q, Or.inr hq, hq'⟩
        · rintro ⟨q, hq, hq'⟩
          rcases hq with rfl | hq
          · exact False.elim (nomatch hq')
          · exact ⟨q, hq, hq'⟩
  constructor
  · intro hp
    rcases (aux u.1).1 hp with ⟨t, ht, ht'⟩
    subst ht'; exact Finset.mem_def.2 ht
  · intro hp
    exact (aux u.1).2 ⟨.funTok p.1 p.2, Finset.mem_def.1 hp, rfl⟩

instance [DecidableEq (RawLamToken α)] :
    LeftCommutative (lamInUnionInsert (α := α)) :=
  ⟨fun p q s => by
    simp only [lamInUnionInsert]
    ext x; simp only [funion.mem]
    constructor <;> · rintro (h | h | h); exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (
    Or.inr h)]⟩

instance [DecidableEq (RawLamToken α)] :
    LeftCommutative (lamOutUnionInsert (α := α)) :=
  ⟨fun p q s => by
    simp only [lamOutUnionInsert]
    ext x; simp only [funion.mem]
    constructor <;> · rintro (h | h | h); exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (
    Or.inr h)]⟩

def lamInputUnion (s : Finset (List (RawLamToken α) × List (RawLamToken α))) :
    Finset (RawLamToken α) :=
  Multiset.foldr (lamInUnionInsert (α := α)) ∅ s.1

def lamOutputUnion (s : Finset (List (RawLamToken α) × List (RawLamToken α))) :
    Finset (RawLamToken α) :=
```

```
  Multiset.foldr (lamOutUnionInsert (α := α)) ∅ s.1

theorem max_le_max_nat {a b c d : ℕ} (h1 : a ≤ c) (h2 : b ≤ d) :
    max a b ≤ max c d :=
  max_le (le_trans h1 (le_max_left _ _)) (le_trans h2 (le_max_right _ _))

def rawDepth : RawLamToken α → ℕ
  | .bot | .atom _ => 0
  | .funTok u v =>
      1 + Nat.max
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v)

theorem mem_lamInputUnion {s : Finset (List (RawLamToken α) × List (RawLamToken α))}
    {t : RawLamToken α} :
    t ∈ lamInputUnion s ↔ ∃ p ∈ s, t ∈ listToFinset p.1 := by
  have aux (m : Multiset (List (RawLamToken α) × List (RawLamToken α))) :
      t ∈ Multiset.foldr (lamInUnionInsert (α := α)) ∅ m ↔
        ∃ p ∈ m, t ∈ listToFinset p.1 := by
    refine Multiset.induction_on m ?_ ?_
    · constructor
      · intro ht; exact False.elim (Finset.notMem_empty t ht)
      · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
    · intro p rest ih
      simp only [Multiset.foldr_cons, lamInUnionInsert, funion.mem, ih, Multiset.
    mem_cons]
      constructor
      · rintro (ht | ⟨q, hq, ht⟩)
        · exact ⟨p, Or.inl rfl, ht⟩
        · exact ⟨q, Or.inr hq, ht⟩
      · rintro ⟨q, hq, ht⟩
        rcases hq with rfl | hq
        · exact Or.inl ht
        · exact Or.inr ⟨q, hq, ht⟩
  simpa [lamInputUnion] using aux s.1

theorem mem_lamOutputUnion {s : Finset (List (RawLamToken α) × List (RawLamToken α))}
    {t : RawLamToken α} :
    t ∈ lamOutputUnion s ↔ ∃ p ∈ s, t ∈ listToFinset p.2 := by
  have aux (m : Multiset (List (RawLamToken α) × List (RawLamToken α))) :
      t ∈ Multiset.foldr (lamOutUnionInsert (α := α)) ∅ m ↔
        ∃ p ∈ m, t ∈ listToFinset p.2 := by
    refine Multiset.induction_on m ?_ ?_
    · constructor
      · intro ht; exact False.elim (Finset.notMem_empty t ht)
      · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
    · intro p rest ih
      simp only [Multiset.foldr_cons, lamOutUnionInsert, funion.mem, ih, Multiset.
    mem_cons]
      constructor
      · rintro (ht | ⟨q, hq, ht⟩)
        · exact ⟨p, Or.inl rfl, ht⟩
        · exact ⟨q, Or.inr hq, ht⟩
      · rintro ⟨q, hq, ht⟩
        rcases hq with rfl | hq
        · exact Or.inl ht
```

```
        · exact Or.inr ⟨q, hq, ht⟩
  simpa [lamOutputUnion] using aux s.1

omit [DecidableEq α] in

theorem foldl_max_ge_init (u : List (RawLamToken α)) (init : ℕ) :
    init ≤ List.foldl (fun d t => Nat.max d (rawDepth t)) init u := by
  induction u generalizing init with
  | nil => exact le_rfl
  | cons a as ih =>
    exact Nat.le_trans (Nat.le_max_left init (rawDepth a)) (ih _)

omit [DecidableEq α] in

theorem foldl_max_mono_init {u : List (RawLamToken α)} {i j : ℕ} (hij : i ≤ j) :
    List.foldl (fun d t => Nat.max d (rawDepth t)) i u ≤
      List.foldl (fun d t => Nat.max d (rawDepth t)) j u := by
  induction u generalizing i j with
  | nil => exact hij
  | cons a as ih => exact ih (max_le_max_nat hij le_rfl)

def maxDepthInsert : RawLamToken α → ℕ → ℕ :=
  fun t d => Nat.max (rawDepth t) d

omit [DecidableEq α] in

theorem foldl_list_le_of_mem {t : RawLamToken α} {u : List (RawLamToken α)}
    (ht : t ∈ u) :
    rawDepth t ≤ List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u := by
  induction u with
  | nil => cases ht
  | cons a as ih =>
    simp only [List.mem_cons] at ht
    rcases ht with rfl | ht
    · exact Nat.le_trans (Nat.le_max_right 0 (rawDepth t))
        (foldl_max_ge_init as (Nat.max 0 (rawDepth t)))
    · exact Nat.le_trans (ih ht) (foldl_max_mono_init (Nat.zero_le _))

instance : LeftCommutative (maxDepthInsert (α := α)) :=
  ⟨fun a b d => by simp only [maxDepthInsert]; exact Nat.max_left_comm _ _ _⟩

omit [DecidableEq α] in

theorem rawDepth_funTok_gt_left {u v : List (RawLamToken α)} {t : RawLamToken α}
    (ht : t ∈ u) : rawDepth t < rawDepth (.funTok u v) := by
  have h : rawDepth t ≤
      Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
    Nat.le_trans (foldl_list_le_of_mem ht) (Nat.le_max_left _ _)
  simp only [rawDepth]
  exact Nat.lt_of_le_of_lt h (Nat.lt_add_of_pos_left (Nat.succ_pos 0))

omit [DecidableEq α] in

theorem rawDepth_funTok_gt_right {u v : List (RawLamToken α)} {t : RawLamToken α}
    (ht : t ∈ v) : rawDepth t < rawDepth (.funTok u v) := by
```

```
  have h : rawDepth t ≤
      Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
    Nat.le_trans (foldl_list_le_of_mem ht) (Nat.le_max_right _ _)
  simp only [rawDepth]
  exact Nat.lt_of_le_of_lt h (Nat.lt_add_of_pos_left (Nat.succ_pos 0))

omit [DecidableEq α] in

theorem foldr_max_le_of_forall {s : Multiset (RawLamToken α)} {b : ℕ}
    (h : ∀ t ∈ s, rawDepth t ≤ b) :
    Multiset.foldr (maxDepthInsert (α := α)) 0 s ≤ b := by
  revert h
  refine Multiset.induction_on s ?_ ?_
  · intro; exact Nat.zero_le _
  · intro a s ih h
    simp only [Multiset.foldr_cons, maxDepthInsert]
    exact Nat.max_le.2 ⟨h a (Multiset.mem_cons_self _ _),
      ih fun t ht => h t (Multiset.mem_cons_of_mem ht)⟩

omit [DecidableEq α] in

theorem foldr_max_lt_of_forall {s : Multiset (RawLamToken α)} {b : ℕ}
    (hb : 0 < b) (h : ∀ t ∈ s, rawDepth t < b) :
    Multiset.foldr (maxDepthInsert (α := α)) 0 s < b := by
  revert h
  refine Multiset.induction_on s ?_ ?_
  · intro; exact hb
  · intro a s ih h
    simp only [Multiset.foldr_cons, maxDepthInsert]
    exact Nat.max_lt.2 ⟨h a (Multiset.mem_cons_self _ _),
      ih fun t ht => h t (Multiset.mem_cons_of_mem ht)⟩

def finsetMaxDepth (u : Finset (RawLamToken α)) : ℕ :=
  Multiset.foldr (maxDepthInsert (α := α)) 0 u.1

theorem finsetMaxDepth.listToFinset_le (u : List (RawLamToken α)) :
    finsetMaxDepth (listToFinset u) ≤
      List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u := by
  simp only [finsetMaxDepth]
  exact foldr_max_le_of_forall fun t ht =>
    foldl_list_le_of_mem ((mem_listToFinset).1 (Finset.mem_def.2 ht))

theorem finsetMaxDepth.list_lt_funTok (u v : List (RawLamToken α)) :
    finsetMaxDepth (listToFinset u) < rawDepth (.funTok u v) := by
  have h1 := finsetMaxDepth.listToFinset_le u
  have h2 : List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u ≤
      Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
    Nat.le_max_left _ _
  simp only [rawDepth]
  exact Nat.lt_of_le_of_lt (Nat.le_trans h1 h2) (Nat.lt_add_of_pos_left (Nat.succ_pos
    0))

omit [DecidableEq α] in
```

```
theorem le_finsetMaxDepth_of_mem {u : Finset (RawLamToken α)} {t : RawLamToken α}
    (ht : t ∈ u) : rawDepth t ≤ finsetMaxDepth u := by
  simp only [finsetMaxDepth]
  have ht' : t ∈ u.1 := Finset.mem_def.1 ht
  revert ht'
  refine Multiset.induction_on u.1 ?_ ?_
  · intro ht'; cases ht'
  · intro a s ih ht'
    simp only [Multiset.mem_cons] at ht'
    simp only [Multiset.foldr_cons, maxDepthInsert]
    rcases ht' with rfl | ht'
    · exact Nat.le_max_left _ _
    · exact Nat.le_trans (ih ht') (Nat.le_max_right _ _)

theorem finsetMaxDepth.inputUnion_lt {u : Finset (RawLamToken α)}
    {s : Finset (List (RawLamToken α) × List (RawLamToken α))}
    (hs : s ⊆ lamFunFinset u) (hfun : lamFunFinset u = ∅) :
    finsetMaxDepth (lamInputUnion s) < finsetMaxDepth u := by
  obtain ⟨p0, hp0⟩ := Finset.nonempty_of_ne_empty hfun
  have htok0 : .funTok p0.1 p0.2 ∈ u := mem_lamFunFinset.1 hp0
  have hpos : 0 < finsetMaxDepth u := by
    have hraw : 0 < rawDepth (.funTok p0.1 p0.2) := by
      simp only [rawDepth]
      exact Nat.add_pos_left (Nat.succ_pos 0) _
    exact Nat.lt_of_lt_of_le hraw (le_finsetMaxDepth_of_mem htok0)
  refine foldr_max_lt_of_forall (s := (lamInputUnion s).1) hpos ?_
  intro t ht
  have ht' : t ∈ lamInputUnion s := Finset.mem_def.2 ht
  rcases (mem_lamInputUnion).1 ht' with ⟨p, hp, htp⟩
  have htok : .funTok p.1 p.2 ∈ u := mem_lamFunFinset.1 (hs hp)
  exact Nat.lt_of_lt_of_le
    (rawDepth_funTok_gt_left ((mem_listToFinset).1 htp))
    (le_finsetMaxDepth_of_mem htok)

theorem finsetMaxDepth.outputUnion_lt {u : Finset (RawLamToken α)}
    {s : Finset (List (RawLamToken α) × List (RawLamToken α))}
    (hs : s ⊆ lamFunFinset u) (hfun : lamFunFinset u = ∅) :
    finsetMaxDepth (lamOutputUnion s) < finsetMaxDepth u := by
  obtain ⟨p0, hp0⟩ := Finset.nonempty_of_ne_empty hfun
  have htok0 : .funTok p0.1 p0.2 ∈ u := mem_lamFunFinset.1 hp0
  have hpos : 0 < finsetMaxDepth u := by
    have hraw : 0 < rawDepth (.funTok p0.1 p0.2) := by
      simp only [rawDepth]
      exact Nat.add_pos_left (Nat.succ_pos 0) _
    exact Nat.lt_of_lt_of_le hraw (le_finsetMaxDepth_of_mem htok0)
  refine foldr_max_lt_of_forall (s := (lamOutputUnion s).1) hpos ?_
  intro t ht
  have ht' : t ∈ lamOutputUnion s := Finset.mem_def.2 ht
  rcases (mem_lamOutputUnion).1 ht' with ⟨p, hp, htp⟩
  have htok : .funTok p.1 p.2 ∈ u := mem_lamFunFinset.1 (hs hp)
  exact Nat.lt_of_lt_of_le
    (rawDepth_funTok_gt_right ((mem_listToFinset).1 htp))
    (le_finsetMaxDepth_of_mem htok)


theorem finsetMaxDepth.list_lt_funTok_out (u v : List (RawLamToken α)) :
```

```
    finsetMaxDepth (listToFinset v) < rawDepth (.funTok u v) := by
  have h1 := finsetMaxDepth.listToFinset_le v
  have h2 : List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v ≤
      Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
    Nat.le_max_right _ _
  simp only [rawDepth]
  exact Nat.lt_of_le_of_lt (Nat.le_trans h1 h2) (Nat.lt_add_of_pos_left (Nat.succ_pos
    0))

theorem finsetMaxDepth.of_mem_funTok {u : Finset (RawLamToken α)}
    {xs ys : List (RawLamToken α)} (h : .funTok xs ys ∈ u) :
    finsetMaxDepth (listToFinset xs) < finsetMaxDepth u ∧
      finsetMaxDepth (listToFinset ys) < finsetMaxDepth u :=
  ⟨Nat.lt_of_lt_of_le (finsetMaxDepth.list_lt_funTok xs ys) (le_finsetMaxDepth_of_mem h
    ),
    Nat.lt_of_lt_of_le (finsetMaxDepth.list_lt_funTok_out xs ys) (
    le_finsetMaxDepth_of_mem h)⟩

def LamConDepth (u : Finset (RawLamToken α)) : Prop :=
  haveI : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  if _hF : lamFunFinset u = ∅ then
    lamAtomFinset u ∈ A.Con
  else
    lamAtomFinset u = ∅ ∧
      (∀ p ∈ lamFunFinset u,
        LamConDepth (listToFinset p.1) ∧ LamConDepth (listToFinset p.2)) ∧
      (∀ s ⊆ lamFunFinset u,
        LamConDepth (lamInputUnion s) → LamConDepth (lamOutputUnion s))
termination_by finsetMaxDepth u
decreasing_by
  · exact (finsetMaxDepth.of_mem_funTok (mem_lamFunFinset.1 ?_?)).1
  · exact (finsetMaxDepth.of_mem_funTok (mem_lamFunFinset.1 ?_?)).2
  · exact finsetMaxDepth.inputUnion_lt ?_? ?_?
  · exact finsetMaxDepth.outputUnion_lt ?_? ?_?

inductive LamEntDepth : Finset (RawLamToken α) → RawLamToken α → Prop where
  | bot {u : Finset (RawLamToken α)}
      (hu : LamConDepth A u) : LamEntDepth u .bot
  | atom {u : Finset (RawLamToken α)} {x : α}
      (hu : LamConDepth A u)
      (hF : lamFunFinset u = ∅)
      (hne : lamAtomFinset u = ∅)
      (hEnt : A.Ent (lamAtomFinset u) x) : LamEntDepth u (.atom x)
  | funTok {u : Finset (RawLamToken α)} {xs ys : List (RawLamToken α)}
      {s : Finset (List (RawLamToken α) × List (RawLamToken α))}
      (hu : LamConDepth A u)
      (hA : lamAtomFinset u = ∅)
      (hConIn : LamConDepth A (listToFinset xs))
      (hConOut : LamConDepth A (listToFinset ys))
      (hs : s ⊆ lamFunFinset u)

      (hWorld : lamFunFinset u = ∅)

      (hInCompat : lamFunFinset (listToFinset xs) = ∅ →
```

```
        lamFunFinset (lamInputUnion s) = ∅)
      (hEntIn : ∀ p ∈ s, ∀ z ∈ listToFinset p.1, LamEntDepth (listToFinset xs) z)
      (hEntOut : ∀ z ∈ listToFinset ys, LamEntDepth (lamOutputUnion s) z) :
      LamEntDepth u (.funTok xs ys)

theorem LamEntDepth_bot {u : Finset (RawLamToken α)} (hu : LamConDepth A u) :
    LamEntDepth A u .bot :=
  LamEntDepth.bot hu

theorem LamEntDepth.con {u : Finset (RawLamToken α)} {t : RawLamToken α}
    (ht : LamEntDepth A u t) : LamConDepth A u := by
  cases ht with
  | bot hu => exact hu
  | atom hu _ _ _ => exact hu
  | funTok hu _ _ _ _ _ _ _ _ => exact hu

def LamConN : ℕ → Finset (RawLamToken α) → Prop
  | 0, u => lamAtomFinset u ∈ A.Con ∧ lamFunFinset u = ∅
  | n + 1, u =>
      LamConN n u ∨
        (lamAtomFinset u = ∅ ∧
          (∀ p ∈ lamFunFinset u,
            LamConN n (listToFinset p.1) ∧ LamConN n (listToFinset p.2)) ∧
          ∀ s ⊆ lamFunFinset u,
            LamConN n (lamInputUnion s) → LamConN n (lamOutputUnion s))

theorem lamAtomFinset_empty : lamAtomFinset (∅ : Finset (RawLamToken α)) = ∅ := rfl

theorem lamFunFinset_empty : lamFunFinset (∅ : Finset (RawLamToken α)) = ∅ := rfl

theorem lamFunFinset_insert_atom (u : Finset (RawLamToken α)) (x : α) :
    lamFunFinset (insert (.atom x) u) = lamFunFinset u := by
  ext p; simp only [mem_lamFunFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem lamFunFinset_insert_bot (u : Finset (RawLamToken α)) :
    lamFunFinset (insert (.bot : RawLamToken α) u) = lamFunFinset u := by
  ext p; simp only [mem_lamFunFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem lamFunFinset_insert_funTok (u : Finset (RawLamToken α))
    (xs ys : List (RawLamToken α)) :
    lamFunFinset (insert (.funTok xs ys) u) = insert (xs, ys) (lamFunFinset u) := by
  ext p; simp only [Finset.mem_insert, mem_lamFunFinset]
  constructor
  · rintro (h | h)
    · exact Or.inl (Prod.ext (RawLamToken.funTok.inj h).1 (RawLamToken.funTok.inj h).2)
    · exact Or.inr h
  · rintro (rfl | h)
    · exact Or.inl rfl
    · exact Or.inr h
```

```
theorem lamFunFinset_mono {u v : Finset (RawLamToken α)} (h : v ⊆ u) :
    lamFunFinset v ⊆ lamFunFinset u :=
  fun _ hp => mem_lamFunFinset.2 (h (mem_lamFunFinset.1 hp))

theorem lamInputUnion_empty :
    lamInputUnion (∅ : Finset (List (RawLamToken α) × List (RawLamToken α))) = ∅ := by
  rfl

theorem lamOutputUnion_empty :
    lamOutputUnion (∅ : Finset (List (RawLamToken α) × List (RawLamToken α))) = ∅ :=
    by
  rfl

theorem mem_lamAtomFinset {u : Finset (RawLamToken α)} {x : α} :
    x ∈ lamAtomFinset u ↔ .atom x ∈ u := by
  have aux (s : Multiset (RawLamToken α)) :
      x ∈ Multiset.foldr lamAtomInsert (∅ : Finset α) s ↔ ∃ p ∈ s, p = .atom x := by
    refine Multiset.induction_on s ?_ ?_
    · constructor
      · intro hx; exact False.elim (Finset.notMem_empty x hx)
      · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
    · intro p t ih
      cases p with
      | atom a =>
        simp only [Multiset.foldr_cons, lamAtomInsert, Finset.mem_insert, ih, Multiset
    .mem_cons]
        constructor
        · rintro (hx | ⟨q, hq, hq'⟩)
          · exact ⟨.atom a, Or.inl rfl, congrArg RawLamToken.atom hx.symm⟩
          · exact ⟨q, Or.inr hq, hq'⟩
        · rintro ⟨q, hq, hq'⟩
          rcases hq with rfl | hq
          · injection hq' with hx; exact Or.inl hx.symm
          · exact Or.inr ⟨q, hq, hq'⟩
      | bot | funTok _ _ =>
        simp only [Multiset.foldr_cons, lamAtomInsert, id_eq, ih, Multiset.mem_cons]
        constructor
        · rintro ⟨q, hq, hq'⟩; exact ⟨q, Or.inr hq, hq'⟩
        · rintro ⟨q, hq, hq'⟩
          rcases hq with rfl | hq
          · exact False.elim (nomatch hq')
          · exact ⟨q, hq, hq'⟩
  constructor
  · intro hx
    rcases (aux u.1).1 hx with ⟨p, hp, hp'⟩
    subst hp'; exact Finset.mem_def.2 hp
  · intro hx
    exact (aux u.1).2 ⟨.atom x, Finset.mem_def.1 hx, rfl⟩

def LamCon (u : Finset (RawLamToken α)) : Prop :=
  ∃ n, LamConN A n u

theorem lamAtomFinset_insert_atom (u : Finset (RawLamToken α)) (x : α) :
    lamAtomFinset (insert (.atom x) u) = insert x (lamAtomFinset u) := by
  ext y; simp only [Finset.mem_insert, mem_lamAtomFinset]
  constructor
```

```
  · rintro (h | h)
    · exact Or.inl (RawLamToken.atom.inj h)
    · exact Or.inr h
  · rintro (rfl | h)
    · exact Or.inl rfl
    · exact Or.inr h

theorem lamAtomFinset_insert_bot (u : Finset (RawLamToken α)) :
    lamAtomFinset (insert (.bot : RawLamToken α) u) = lamAtomFinset u := by
  ext x; simp only [mem_lamAtomFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem lamAtomFinset_insert_funTok (u : Finset (RawLamToken α))
    (xs ys : List (RawLamToken α)) :
    lamAtomFinset (insert (.funTok xs ys) u) = lamAtomFinset u := by
  ext x; simp only [mem_lamAtomFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem lamAtomFinset_mono {u v : Finset (RawLamToken α)} (h : v ⊆ u) :
    lamAtomFinset v ⊆ lamAtomFinset u :=
  fun _ hx => mem_lamAtomFinset.2 (h (mem_lamAtomFinset.1 hx))

class IsLamToken (t : RawLamToken α) : Prop where
  out : match t with
    | .bot | .atom _ => True
    | .funTok u v => LamCon A (listToFinset u) ∧ LamCon A (listToFinset v)

def LamToken : Type _ := { t : RawLamToken α // IsLamToken A t }

instance : DecidableEq (LamToken A) := Subtype.instDecidableEq

def lamBot : LamToken A := ⟨.bot, ⟨trivial⟩⟩

def lamForget (t : LamToken A) : RawLamToken α := t.val

def lamForgetInsert :
    LamToken A → Finset (RawLamToken α) → Finset (RawLamToken α) :=
  fun t => insert (lamForget A t)

instance : LeftCommutative (lamForgetInsert (A := A)) :=
  ⟨fun p q s => insert_comm' (lamForget A p) (lamForget A q) s⟩

theorem LamConDepth_empty : LamConDepth A (∅ : Finset (RawLamToken α)) := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  rw [LamConDepth.eq_1, dite_eq_left lamFunFinset_empty, lamAtomFinset_empty]
  exact A.con_empty

theorem LamConDepth_of_fun_empty {u : Finset (RawLamToken α)}
    (hF : lamFunFinset u = ∅) :
    LamConDepth A u ↔ lamAtomFinset u ∈ A.Con := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
```

```
    decidableEq_finset
  rw [LamConDepth.eq_1, dite_eq_left hF]

theorem LamConDepth_of_fun_nonempty {u : Finset (RawLamToken α)}
    (hF : lamFunFinset u = ∅) :
    LamConDepth A u ↔
      lamAtomFinset u = ∅ ∧
        (∀ p ∈ lamFunFinset u,
          LamConDepth A (listToFinset p.1) ∧ LamConDepth A (listToFinset p.2)) ∧
        (∀ s ⊆ lamFunFinset u,
          LamConDepth A (lamInputUnion s) → LamConDepth A (lamOutputUnion s)) := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  rw [LamConDepth.eq_1, dite_eq_right hF]

theorem LamEntDepth_weaken_fun {u v : Finset (RawLamToken α)} {t : RawLamToken α}
    (huv : u ⊆ v) (hv : LamConDepth A v) (hAv : lamAtomFinset v = ∅)
    (ht : LamEntDepth A u t) : LamEntDepth A v t := by
  cases ht with
  | bot _ => exact LamEntDepth.bot hv
  | atom _ _ hne _ =>
    obtain ⟨x, hx⟩ := Finset.nonempty_of_ne_empty hne
    exact False.elim (Finset.notMem_empty x (by
      have := lamAtomFinset_mono huv hx; rwa [hAv] at this))
  | funTok _ _ hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>
    exact LamEntDepth.funTok hv hAv hConIn hConOut
      (fun p hp => lamFunFinset_mono huv (hs hp))
      (fun hFv => hWorld (Finset.Subset.antisymm
          (fun p hp => by
            have := lamFunFinset_mono huv hp
            rw [hFv] at this; exact False.elim (Finset.notMem_empty p this))
          (Finset.empty_subset _)))
      hInCompat hEntIn hEntOut

def LamEntN : ℕ → Finset (RawLamToken α) → RawLamToken α → Prop
  | 0, u, .bot => LamConN A 0 u
  | 0, u, .atom x =>
      LamConN A 0 u ∧ lamAtomFinset u = ∅ ∧ A.Ent (lamAtomFinset u) x
  | 0, _, .funTok _ _ => False
  | n + 1, u, t =>
      LamEntN n u t ∨
        match t with
        | .bot => LamConN A (n + 1) u
        | .atom x =>
            LamConN A (n + 1) u ∧ lamFunFinset u = ∅ ∧
              lamAtomFinset u = ∅ ∧ A.Ent (lamAtomFinset u) x
        | .funTok u' v' =>
            LamConN A (n + 1) u ∧ lamAtomFinset u = ∅ ∧
              LamConN A n (listToFinset u') ∧ LamConN A n (listToFinset v') ∧
              ∃ s ⊆ lamFunFinset u,
                (∀ p ∈ s, ∀ t ∈ listToFinset p.1, LamEntN n (listToFinset u') t) ∧
                  (∀ t ∈ listToFinset v', LamEntN n (lamOutputUnion s) t)

class LamWF (t : RawLamToken α) : Prop where
  out : match t with
    | .bot | .atom _ => True
```

```
    | .funTok xs ys => LamConDepth A (listToFinset xs) ∧ LamConDepth A (listToFinset
    ys)

theorem filter_ne_subset_of_subset_insert_lam
    {p : List (RawLamToken α) × List (RawLamToken α)}
    {t w : Finset (List (RawLamToken α) × List (RawLamToken α))}
    (ht : t ⊆ insert p w) : t.filter (· = p) ⊆ w := by
  intro q hq
  have ⟨hqt, hne⟩ := Finset.mem_filter.mp hq
  have : q ∈ insert p w := ht hqt
  rcases Finset.mem_insert.mp this with rfl | hqW
  · exact False.elim (hne rfl)
  · exact hqW

theorem finsetMaxDepth_funion_le (u v : Finset (RawLamToken α)) :
    finsetMaxDepth (u ∪' v) ≤ Nat.max (finsetMaxDepth u) (finsetMaxDepth v) := by
  refine foldr_max_le_of_forall ?_
  intro t ht
  have ht' : t ∈ u ∪' v := Finset.mem_def.2 ht
  rcases funion.mem.1 ht' with hu | hv
  · exact Nat.le_trans (le_finsetMaxDepth_of_mem hu) (Nat.le_max_left _ _)
  · exact Nat.le_trans (le_finsetMaxDepth_of_mem hv) (Nat.le_max_right _ _)

theorem finsetMaxDepth_insert_le (u : Finset (RawLamToken α)) (a : RawLamToken α) :
    finsetMaxDepth (insert a u) ≤ Nat.max (rawDepth a) (finsetMaxDepth u) := by
  refine foldr_max_le_of_forall ?_
  intro x hx
  have hx' : x ∈ insert a u := Finset.mem_def.2 hx
  rcases Finset.mem_insert.mp hx' with rfl | hu
  · exact Nat.le_max_left _ _
  · exact Nat.le_trans (le_finsetMaxDepth_of_mem hu) (Nat.le_max_right _ _)

theorem funion_assoc_lam (a b c : Finset (RawLamToken α)) :
    (a ∪' b) ∪' c = a ∪' (b ∪' c) := by
  ext x
  constructor
  · intro hx
    rcases funion.mem.mp hx with h | h
    · rcases funion.mem.mp h with h | h
      · exact funion.mem.mpr (Or.inl h)
      · exact funion.mem.mpr (Or.inr (funion.mem.mpr (Or.inl h)))
    · exact funion.mem.mpr (Or.inr (funion.mem.mpr (Or.inr h)))
  · intro hx
    rcases funion.mem.mp hx with h | h
    · exact funion.mem.mpr (Or.inl (funion.mem.mpr (Or.inl h)))
    · rcases funion.mem.mp h with h | h
      · exact funion.mem.mpr (Or.inl (funion.mem.mpr (Or.inr h)))
      · exact funion.mem.mpr (Or.inr h)

theorem funion_comm_lam (a b : Finset (RawLamToken α)) : a ∪' b = b ∪' a := by
  ext x
  constructor
  · intro hx
    rcases funion.mem.mp hx with h | h
    · exact funion.mem.mpr (Or.inr h)
    · exact funion.mem.mpr (Or.inl h)
```

```
  · intro hx
    rcases funion.mem.mp hx with h | h
    · exact funion.mem.mpr (Or.inr h)
    · exact funion.mem.mpr (Or.inl h)

theorem funion_empty_right_lam (u : Finset (RawLamToken α)) :
    u ∪' (∅ : Finset (RawLamToken α)) = u := by
  ext x; simp only [funion.mem, Finset.notMem_empty, or_false]

theorem insert_filter_ne_lam
    {p : List (RawLamToken α) × List (RawLamToken α)}
    {t : Finset (List (RawLamToken α) × List (RawLamToken α))}
    (hp : p ∈ t) : insert p (t.filter (· = p)) = t := by
  ext q
  constructor
  · intro hq
    rcases Finset.mem_insert.mp hq with rfl | hq
    · exact hp
    · exact (Finset.mem_filter.mp hq).1
  · intro hq
    if h : q = p then
      exact Finset.mem_insert.mpr (Or.inl h)
    else
      exact Finset.mem_insert.mpr (Or.inr (Finset.mem_filter.mpr ⟨hq, h⟩))

theorem lamAtomFinset_singleton_funTok (u v : List (RawLamToken α)) :
    lamAtomFinset ({.funTok u v} : Finset (RawLamToken α)) = ∅ := by
  ext x
  constructor
  · intro hx; exact False.elim (nomatch mem_lamAtomFinset.1 hx)
  · intro hx; exact False.elim (Finset.notMem_empty x hx)

def lamForgetFinset (u : Finset (LamToken A)) : Finset (RawLamToken α) :=
  Multiset.foldr (lamForgetInsert (A := A)) ∅ u.1

theorem lamFunFinset_funion (u v : Finset (RawLamToken α)) :
    lamFunFinset (u ∪' v) = lamFunFinset u ∪' lamFunFinset v := by
  ext p
  constructor
  · intro hp
    have : .funTok p.1 p.2 ∈ u ∪' v := mem_lamFunFinset.1 hp
    rcases funion.mem.1 this with hu | hv
    · exact funion.mem.2 (Or.inl (mem_lamFunFinset.2 hu))
    · exact funion.mem.2 (Or.inr (mem_lamFunFinset.2 hv))
  · intro hp
    rcases funion.mem.1 hp with hu | hv
    · exact mem_lamFunFinset.2 (funion.mem.2 (Or.inl (mem_lamFunFinset.1 hu)))
    · exact mem_lamFunFinset.2 (funion.mem.2 (Or.inr (mem_lamFunFinset.1 hv)))

theorem lamFunFinset_singleton_funTok (u v : List (RawLamToken α)) :
    lamFunFinset ({.funTok u v} : Finset (RawLamToken α)) = {(u, v)} := by
  ext p
  simp only [mem_lamFunFinset, Finset.mem_singleton]
  constructor
  · intro h
    exact Prod.ext (RawLamToken.funTok.inj h).1 (RawLamToken.funTok.inj h).2
```

```
  · intro h
    cases h
    rfl

theorem lamInputUnion_funion
    (s t : Finset (List (RawLamToken α) × List (RawLamToken α))) :
    lamInputUnion (s ∪' t) = lamInputUnion s ∪' lamInputUnion t := by
  ext x
  constructor
  · intro hx
    rcases (mem_lamInputUnion).1 hx with ⟨p, hp, hx'⟩
    rcases funion.mem.1 hp with hp | hp


    · exact funion.mem.2 (Or.inl ((mem_lamInputUnion).2 ⟨p, hp, hx'⟩))
    · exact funion.mem.2 (Or.inr ((mem_lamInputUnion).2 ⟨p, hp, hx'⟩))
  · intro hx
    rcases funion.mem.1 hx with hx | hx
    · rcases (mem_lamInputUnion).1 hx with ⟨p, hp, hx'⟩
      exact (mem_lamInputUnion).2 ⟨p, funion.mem.2 (Or.inl hp), hx'⟩
    · rcases (mem_lamInputUnion).1 hx with ⟨p, hp, hx'⟩
      exact (mem_lamInputUnion).2 ⟨p, funion.mem.2 (Or.inr hp), hx'⟩

theorem lamInputUnion_insert
    (s : Finset (List (RawLamToken α) × List (RawLamToken α)))
    (p : List (RawLamToken α) × List (RawLamToken α)) :
    lamInputUnion (insert p s) = listToFinset p.1 ∪' lamInputUnion s := by
  ext t
  constructor
  · intro ht
    rcases (mem_lamInputUnion).1 ht with ⟨q, hq, ht'⟩
    rcases Finset.mem_insert.mp hq with rfl | hq
    · exact funion.mem.2 (Or.inl ht')
    · exact funion.mem.2 (Or.inr ((mem_lamInputUnion).2 ⟨q, hq, ht'⟩))
  · intro ht
    rcases funion.mem.1 ht with ht | ht
    · exact (mem_lamInputUnion).2 ⟨p, Finset.mem_insert_self p s, ht⟩
    · rcases (mem_lamInputUnion).1 ht with ⟨q, hq, ht'⟩
      exact (mem_lamInputUnion).2 ⟨q, Finset.mem_insert_of_mem hq, ht'⟩

theorem lamInputUnion_singleton (p : List (RawLamToken α) × List (RawLamToken α)) :
    lamInputUnion ({p} : Finset (List (RawLamToken α) × List (RawLamToken α))) =
      listToFinset p.1 := by
  ext t
  change t ∈ listToFinset p.1 ∪' ∅ ↔ t ∈ listToFinset p.1
  simp only [funion.mem, Finset.notMem_empty, or_false]

theorem lamOutputUnion_funion
    (s t : Finset (List (RawLamToken α) × List (RawLamToken α))) :
    lamOutputUnion (s ∪' t) = lamOutputUnion s ∪' lamOutputUnion t := by
  ext x
  constructor
  · intro hx
    rcases (mem_lamOutputUnion).1 hx with ⟨p, hp, hx'⟩
    rcases funion.mem.1 hp with hp | hp
    · exact funion.mem.2 (Or.inl ((mem_lamOutputUnion).2 ⟨p, hp, hx'⟩))
    · exact funion.mem.2 (Or.inr ((mem_lamOutputUnion).2 ⟨p, hp, hx'⟩))
```

```
  · intro hx
    rcases funion.mem.1 hx with hx | hx
    · rcases (mem_lamOutputUnion).1 hx with ⟨p, hp, hx'⟩
      exact (mem_lamOutputUnion).2 ⟨p, funion.mem.2 (Or.inl hp), hx'⟩
    · rcases (mem_lamOutputUnion).1 hx with ⟨p, hp, hx'⟩
      exact (mem_lamOutputUnion).2 ⟨p, funion.mem.2 (Or.inr hp), hx'⟩

theorem lamOutputUnion_insert
    (s : Finset (List (RawLamToken α) × List (RawLamToken α)))
    (p : List (RawLamToken α) × List (RawLamToken α)) :
    lamOutputUnion (insert p s) = listToFinset p.2 ∪' lamOutputUnion s := by
  ext t
  constructor
  · intro ht
    rcases (mem_lamOutputUnion).1 ht with ⟨q, hq, ht'⟩
    rcases Finset.mem_insert.mp hq with rfl | hq
    · exact funion.mem.2 (Or.inl ht')
    · exact funion.mem.2 (Or.inr ((mem_lamOutputUnion).2 ⟨q, hq, ht'⟩))
  · intro ht
    rcases funion.mem.1 ht with ht | ht
    · exact (mem_lamOutputUnion).2 ⟨p, Finset.mem_insert_self p s, ht⟩
    · rcases (mem_lamOutputUnion).1 ht with ⟨q, hq, ht'⟩
      exact (mem_lamOutputUnion).2 ⟨q, Finset.mem_insert_of_mem hq, ht'⟩

theorem lamOutputUnion_singleton (p : List (RawLamToken α) × List (RawLamToken α)) :
    lamOutputUnion ({p} : Finset (List (RawLamToken α) × List (RawLamToken α))) =
      listToFinset p.2 := by
  ext t
  change t ∈ listToFinset p.2 ∪' ∅ ↔ t ∈ listToFinset p.2
  simp only [funion.mem, Finset.notMem_empty, or_false]

theorem LamConDepth_insert_atom {u : Finset (RawLamToken α)} {x : α}
    (ht : LamEntDepth A u (.atom x)) :
    LamConDepth A (insert (.atom x) u) := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  cases ht with
  | atom hu hF _hne hEnt =>
    have hF' : lamFunFinset (insert (.atom x) u) = ∅ := by
      rw [lamFunFinset_insert_atom]; exact hF
    exact (LamConDepth_of_fun_empty A hF').2 (by
      rw [lamAtomFinset_insert_atom]
      exact A.ent_con hEnt)

theorem LamConDepth_singleton_atom (x : α) :
    LamConDepth A ({.atom x} : Finset (RawLamToken α)) := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  have hF : lamFunFinset ({.atom x} : Finset (RawLamToken α)) = ∅ := by
    ext p; constructor
    · intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
    · intro hp; exact False.elim (Finset.notMem_empty p hp)
  exact (LamConDepth_of_fun_empty A hF).2 (by
    have h : lamAtomFinset ({.atom x} : Finset (RawLamToken α)) = {x} := by
      ext y; simp only [mem_lamAtomFinset, Finset.mem_singleton]
      exact ⟨RawLamToken.atom.inj, congrArg RawLamToken.atom⟩
```

```
    rw [h]; exact A.con_sing x)

theorem LamConDepth_singleton_bot :
    LamConDepth A ({.bot} : Finset (RawLamToken α)) := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  have hF : lamFunFinset ({.bot} : Finset (RawLamToken α)) = ∅ := by
    ext p; constructor
    · intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
    · intro hp; exact False.elim (Finset.notMem_empty p hp)
  exact (LamConDepth_of_fun_empty A hF).2 (by
    have h : lamAtomFinset ({.bot} : Finset (RawLamToken α)) = ∅ := by
      ext x; constructor
      · intro hx; exact False.elim (nomatch mem_lamAtomFinset.1 hx)
      · intro hx; exact False.elim (Finset.notMem_empty x hx)
    rw [h]; exact A.con_empty)

theorem LamEntDepth_weaken_atom {u v : Finset (RawLamToken α)} {x : α}
    (huv : u ⊆ v) (hv : LamConDepth A v) (hFv : lamFunFinset v = ∅)
    (ht : LamEntDepth A u (.atom x)) : LamEntDepth A v (.atom x) := by
  cases ht with
  | atom hu hF hne hEnt =>
    have hsubA := lamAtomFinset_mono huv
    have hne' : lamAtomFinset v = ∅ := by
      intro hAv
      obtain ⟨y, hy⟩ := Finset.nonempty_of_ne_empty hne
      exact Finset.notMem_empty y (by have := hsubA hy; rwa [hAv] at this)
    exact LamEntDepth.atom hv hFv hne'
      (A.ent_trans ((LamConDepth_of_fun_empty A hFv).1 hv)
        ((LamConDepth_of_fun_empty A hF).1 hu)
        (fun y hy => A.ent_refl ((LamConDepth_of_fun_empty A hFv).1 hv) (hsubA hy))
        hEnt)

theorem LamConDepth_insert_bot {u : Finset (RawLamToken α)}
    (hu : LamConDepth A u) :
    LamConDepth A (insert (.bot : RawLamToken α) u) := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  by_cases hF : lamFunFinset u = ∅
  · have hF' : lamFunFinset (insert (.bot : RawLamToken α) u) = ∅ := by
      rw [lamFunFinset_insert_bot]; exact hF
    exact (LamConDepth_of_fun_empty A hF').2 (by
      rw [lamAtomFinset_insert_bot]
      exact (LamConDepth_of_fun_empty A hF).1 hu)
  · have hu' := (LamConDepth_of_fun_nonempty A hF).1 hu
    rcases hu' with ⟨hA, hWF, hFun⟩
    have hF' : lamFunFinset (insert (.bot : RawLamToken α) u) = ∅ := by
      rw [lamFunFinset_insert_bot]; exact hF
    refine (LamConDepth_of_fun_nonempty A hF').2 ⟨?_, ?_, ?_⟩
    · rw [lamAtomFinset_insert_bot]; exact hA
    · intro p hp
      rw [lamFunFinset_insert_bot] at hp
      exact hWF p hp
    · intro s hs hIn
      refine hFun s (by rw [← lamFunFinset_insert_bot u]; exact hs) hIn
```

```
theorem LamConDepth_mono {u v : Finset (RawLamToken α)}
    (huv : u ⊆ v) (hv : LamConDepth A v) : LamConDepth A u := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  have hFunMono := lamFunFinset_mono huv
  have hAtomMono := lamAtomFinset_mono huv
  by_cases hFv : lamFunFinset v = ∅
  · have hFu : lamFunFinset u = ∅ :=
      Finset.Subset.antisymm
        (fun p hp => by
          have := hFunMono hp
          rw [hFv] at this; exact False.elim (Finset.notMem_empty p this))
        (Finset.empty_subset _)
    exact (LamConDepth_of_fun_empty A hFu).2
      (A.con_subset ((LamConDepth_of_fun_empty A hFv).1 hv) hAtomMono)
  · have hv' := (LamConDepth_of_fun_nonempty A hFv).1 hv
    rcases hv' with ⟨hAv, hWF, hFun⟩
    by_cases hFu : lamFunFinset u = ∅
    · have hAu : lamAtomFinset u = ∅ :=
        Finset.Subset.antisymm
          (fun x hx => by
            have := hAtomMono hx
            rw [hAv] at this; exact False.elim (Finset.notMem_empty x this))
          (Finset.empty_subset _)
      exact (LamConDepth_of_fun_empty A hFu).2 (by rw [hAu]; exact A.con_empty)
    · refine (LamConDepth_of_fun_nonempty A hFu).2 ⟨?_, ?_, ?_⟩
      · exact Finset.Subset.antisymm
          (fun x hx => by
            have := hAtomMono hx
            rw [hAv] at this; exact False.elim (Finset.notMem_empty x this))
          (Finset.empty_subset _)
      · intro p hp; exact hWF p (hFunMono hp)
      · intro s hs hIn
        exact hFun s (fun p hp => hFunMono (hs hp)) hIn

def LamEnt (u : Finset (RawLamToken α)) (t : RawLamToken α) : Prop :=
  ∃ n, LamEntN A n u t

def LamDToken : Type _ := { t : RawLamToken α // LamWF A t }

def LamTokenCon (u : Finset (LamToken A)) : Prop :=
  LamCon A (lamForgetFinset A u)

theorem LamEntDepth_of_mem {u : Finset (RawLamToken α)} {t : RawLamToken α}
    (hu : LamConDepth A u) (ht : t ∈ u) : LamEntDepth A u t := by
  suffices ∀ M u t, Nat.max (finsetMaxDepth u) (rawDepth t) = M →
      LamConDepth A u → t ∈ u → LamEntDepth A u t from
    this _ _ _ rfl hu ht
  intro M
  induction M using Nat.strong_induction_on with
  | h M ih =>
    intro u t hM hu ht
    match t with
    | .bot => exact LamEntDepth_bot A hu
    | .atom x =>
      cases decidableEq_finset (lamFunFinset u) ∅ with
```

```
      | isTrue hF =>
        have hx : x ∈ lamAtomFinset u := mem_lamAtomFinset.2 ht
        exact LamEntDepth.atom hu hF (Finset.ne_empty_of_mem hx)
          (A.ent_refl ((LamConDepth_of_fun_empty A hF).1 hu) hx)
      | isFalse hF =>
        have hA := ((LamConDepth_of_fun_nonempty A hF).1 hu).1
        have hx : x ∈ lamAtomFinset u := mem_lamAtomFinset.2 ht
        rw [hA] at hx; exact False.elim (Finset.notMem_empty x hx)
    | .funTok xs ys =>
      cases decidableEq_finset (lamFunFinset u) ∅ with
      | isTrue hF =>
        have : (xs, ys) ∈ lamFunFinset u := mem_lamFunFinset.2 ht
        rw [hF] at this; exact False.elim (Finset.notMem_empty _ this)
      | isFalse hF =>
        have hu' := (LamConDepth_of_fun_nonempty A hF).1 hu
        rcases hu' with ⟨hA, hWF, _hFun⟩
        have hmem : (xs, ys) ∈ lamFunFinset u := mem_lamFunFinset.2 ht
        have ⟨hConIn, hConOut⟩ := hWF (xs, ys) hmem
        refine LamEntDepth.funTok (s := {(xs, ys)}) hu hA hConIn hConOut
          (fun p hp => by
            have hp' : p = (xs, ys) := Finset.mem_singleton.mp hp
            exact hp' ▸ hmem) hF ?_ ?_ ?_
        · intro hxsF
          have hIn : lamInputUnion {(xs, ys)} = listToFinset xs := by
            rw [lamInputUnion_singleton]
          rw [hIn, hxsF]
        · intro p hp z hz
          have hp' : p = (xs, ys) := Finset.mem_singleton.mp hp
          subst hp'
          refine ih (Nat.max (finsetMaxDepth (listToFinset xs)) (rawDepth z))
            ?_ (listToFinset xs) z rfl hConIn hz
          have htok : .funTok xs ys ∈ u := mem_lamFunFinset.1 hmem
          have ht_lt : rawDepth z < finsetMaxDepth u :=
            Nat.lt_of_lt_of_le (rawDepth_funTok_gt_left ((mem_listToFinset).1 hz))
              (le_finsetMaxDepth_of_mem htok)
          have hxs_lt : finsetMaxDepth (listToFinset xs) < rawDepth (.funTok xs ys) :=
            finsetMaxDepth.list_lt_funTok xs ys
          rw [← hM]
          refine Nat.max_lt.2 ⟨
            Nat.lt_of_lt_of_le hxs_lt (Nat.le_max_right _ _),
            Nat.lt_of_lt_of_le ht_lt (Nat.le_max_left _ _)⟩
        · intro z hz
          rw [lamOutputUnion_singleton]
          refine ih (Nat.max (finsetMaxDepth (listToFinset ys)) (rawDepth z))
            ?_ (listToFinset ys) z rfl hConOut hz
          have ht_lt : rawDepth z < rawDepth (.funTok xs ys) :=
            rawDepth_funTok_gt_right ((mem_listToFinset).1 hz)
          have hys_lt : finsetMaxDepth (listToFinset ys) < rawDepth (.funTok xs ys) :=
            finsetMaxDepth.list_lt_funTok_out xs ys
          rw [← hM]
          refine Nat.max_lt.2 ⟨
            Nat.lt_of_lt_of_le hys_lt (Nat.le_max_right _ _),
            Nat.lt_of_lt_of_le ht_lt (Nat.le_max_right _ _)⟩

theorem LamEntDepth_weaken_insert {u : Finset (RawLamToken α)}
    {a x : RawLamToken α}
```

```
    (hu' : LamConDepth A (insert a u))
    (hx : LamEntDepth A u x) (ha : LamEntDepth A u a) :
    LamEntDepth A (insert a u) x := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  have huv : u ⊆ insert a u := Finset.subset_insert a u
  match x with
  | .bot => exact LamEntDepth.bot hu'
  | .atom z =>
    cases decidableEq_finset (lamFunFinset (insert a u)) ∅ with
    | isTrue hF =>
      exact LamEntDepth_weaken_atom A huv hu' hF hx
    | isFalse hF =>
      have hAt := ((LamConDepth_of_fun_nonempty A hF).1 hu').1
      cases hx with
      | atom _ _ hne _ =>
        obtain ⟨y, hy⟩ := Finset.nonempty_of_ne_empty hne
        exact False.elim (Finset.notMem_empty y (by
          have := lamAtomFinset_mono huv hy; rwa [hAt] at this))
  | .funTok xs ys =>
    cases decidableEq_finset (lamFunFinset (insert a u)) ∅ with
    | isFalse hF =>
      have hAv := ((LamConDepth_of_fun_nonempty A hF).1 hu').1
      exact LamEntDepth_weaken_fun A huv hu' hAv hx
    | isTrue hF =>
      cases ha with
      | bot _ =>
        cases hx with
        | funTok hu hA hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>
          have hAv : lamAtomFinset (insert (.bot : RawLamToken α) u) = ∅ := by
            rw [lamAtomFinset_insert_bot]; exact hA
          exact LamEntDepth.funTok hu' hAv hConIn hConOut
            (by rw [lamFunFinset_insert_bot]; exact hs)
            (by rw [lamFunFinset_insert_bot]; exact hWorld)
            hInCompat hEntIn hEntOut
      | atom _ _ hne _ =>
        cases hx with
        | funTok _ hA _ _ _ _ _ _ _ =>
          obtain ⟨y, hy⟩ := Finset.nonempty_of_ne_empty hne
          exact False.elim (Finset.notMem_empty y (by rwa [hA] at hy))
      | funTok _ _ _ _ _ _ _ _ _ =>

        simp only [lamFunFinset_insert_funTok] at hF
        exact False.elim (Finset.notMem_empty _ (hF ▶ Finset.mem_insert_self _ _))

def LamTokenEnt (u : Finset (LamToken A)) (p : LamToken A) : Prop :=
  LamEnt A (lamForgetFinset A u) p.val

instance : DecidableEq (LamDToken A) := Subtype.instDecidableEq

theorem LamConDepth_funion_of_entSet (M : ℕ)
    (ihIns : ∀ u t, Nat.max (finsetMaxDepth u) (rawDepth t) < M →
      LamEntDepth A u t → LamConDepth A (insert t u))
    {u v : Finset (RawLamToken α)}
    (hu : LamConDepth A u)
    (hEnt : ∀ t ∈ v, LamEntDepth A u t)
```

```
  (hbound : ∀ t ∈ v, Nat.max (finsetMaxDepth u) (rawDepth t) < M) :
  LamConDepth A (u ∪' v) := by
have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
  decidableEq_finset
suffices ∀ (v u : Finset (RawLamToken α)),
    LamConDepth A u →
    (∀ t ∈ v, LamEntDepth A u t) →
    (∀ t ∈ v, Nat.max (finsetMaxDepth u) (rawDepth t) < M) →
    LamConDepth A (u ∪' v) from
  this v u hu hEnt hbound
intro v
induction v using Finset.induction_on with
| empty =>
  intro u hu _ _
  simpa [funion_empty_right_lam] using hu
| insert a t ha ih =>
  intro u hu hEnt hbound
  have mem_a : a ∈ insert a t := Finset.mem_insert_self a t
  have han : Nat.max (finsetMaxDepth u) (rawDepth a) < M := hbound a mem_a
  have hEnt_a : LamEntDepth A u a := hEnt a mem_a
  have hu_ins : LamConDepth A (insert a u) := ihIns u a han hEnt_a
  have hEnt_t : ∀ x ∈ t, LamEntDepth A (insert a u) x := by
    intro x hx
    exact LamEntDepth_weaken_insert A hu_ins
      (hEnt x (Finset.mem_insert_of_mem hx)) hEnt_a
  have hbound_t : ∀ x ∈ t, Nat.max (finsetMaxDepth (insert a u)) (rawDepth x) < M :=
   by
    intro x hx
    have hxu : Nat.max (finsetMaxDepth u) (rawDepth x) < M :=
      hbound x (Finset.mem_insert_of_mem hx)
    have hDi := finsetMaxDepth_insert_le u a
    have h1 : Nat.max (finsetMaxDepth (insert a u)) (rawDepth x) ≤
        Nat.max (Nat.max (rawDepth a) (finsetMaxDepth u)) (rawDepth x) :=
      max_le_max_nat hDi le_rfl
    refine Nat.lt_of_le_of_lt h1 ?_
    have h2 : Nat.max (Nat.max (rawDepth a) (finsetMaxDepth u)) (rawDepth x) =
        Nat.max (rawDepth a) (Nat.max (finsetMaxDepth u) (rawDepth x)) := by
      ac_rfl
    rw [h2]
    exact Nat.max_lt.2 ⟨Nat.lt_of_le_of_lt (Nat.le_max_right _ _) han, hxu⟩
  have hunion : LamConDepth A (insert a u ∪' t) :=
    ih (insert a u) hu_ins hEnt_t hbound_t
  have heq : insert a u ∪' t = u ∪' insert a t := by
    ext x
    constructor
    · intro hx
      rcases funion.mem.1 hx with h | h
      · rcases Finset.mem_insert.mp h with rfl | hu'
        · exact funion.mem.2 (Or.inr (Finset.mem_insert_self _ _))
        · exact funion.mem.2 (Or.inl hu')
      · exact funion.mem.2 (Or.inr (Finset.mem_insert_of_mem h))
    · intro hx
      rcases funion.mem.1 hx with hu' | h
      · exact funion.mem.2 (Or.inl (Finset.mem_insert_of_mem hu'))
      · rcases Finset.mem_insert.mp h with rfl | ht'
        · exact funion.mem.2 (Or.inl (Finset.mem_insert_self _ _))
```

```
          · exact funion.mem.2 (Or.inr ht')
    rwa [heq] at hunion

def lamDBot : LamDToken A := ⟨.bot, ⟨trivial⟩⟩

def lamDForget (t : LamDToken A) : RawLamToken α := t.val

theorem LamConDepth_insert_of_ent {u : Finset (RawLamToken α)} {t : RawLamToken α}
    (ht : LamEntDepth A u t) : LamConDepth A (insert t u) := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  suffices ∀ M u t, Nat.max (finsetMaxDepth u) (rawDepth t) = M →
      LamEntDepth A u t → LamConDepth A (insert t u) from
    this _ _ _ rfl ht
  intro M
  induction M using Nat.strong_induction_on with
  | h M ih =>
    intro u t hM ht
    have ihIns : ∀ u' t', Nat.max (finsetMaxDepth u') (rawDepth t') < M →
        LamEntDepth A u' t' → LamConDepth A (insert t' u') :=
      fun u' t' hlt hEnt => ih _ hlt u' t' rfl hEnt
    match t with

    | .bot => exact LamConDepth_insert_bot A (LamEntDepth.con A ht)
    | .atom x => exact LamConDepth_insert_atom A ht
    | .funTok xs ys =>
      cases ht with
      | @funTok u xs ys s hu hA hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>
        have hF' : lamFunFinset (insert (.funTok xs ys) u) = ∅ := by
          rw [lamFunFinset_insert_funTok]
          exact Finset.insert_ne_empty _ _
        refine (LamConDepth_of_fun_nonempty A hF').2 ⟨?_, ?_, ?_⟩
        · rw [lamAtomFinset_insert_funTok]; exact hA
        · intro p hp
          rw [lamFunFinset_insert_funTok] at hp
          rcases Finset.mem_insert.mp hp with rfl | hp
          · exact ⟨hConIn, hConOut⟩
          · exact ((LamConDepth_of_fun_nonempty A hWorld).1 hu).2.1 p hp
        · intro sel hsel hIn
          if hp : (xs, ys) ∈ sel then
            let t' := sel.filter (· = (xs, ys))
            have ht_eq : insert (xs, ys) t' = sel := insert_filter_ne_lam hp
            have ht'sub : t' ⊆ lamFunFinset u :=
              filter_ne_subset_of_subset_insert_lam (by
                intro q hq; have := hsel hq; rwa [lamFunFinset_insert_funTok] at this)
            have hin' : LamConDepth A (listToFinset xs ∪' lamInputUnion t') := by
              rwa [← ht_eq, lamInputUnion_insert] at hIn
            have hsub : s ∪' t' ⊆ lamFunFinset u :=
              funion.subset_iff.mpr ⟨hs, ht'sub⟩
            have hEnt_in_s : ∀ z ∈ lamInputUnion s,
                LamEntDepth A (listToFinset xs) z := by
              intro z hz
              rcases (mem_lamInputUnion).1 hz with ⟨p, hp', hz'⟩
              exact hEntIn p hp' z hz'
            have hEnt_big : ∀ z ∈ lamInputUnion s,
                LamEntDepth A (listToFinset xs ∪' lamInputUnion t') z := by
```

```
            intro z hz
            have hzx := hEnt_in_s z hz
            have hsubxs : listToFinset xs ⊆ listToFinset xs ∪' lamInputUnion t' :=
              funion.subset_left _ _
            cases decidableEq_finset
                (lamFunFinset (listToFinset xs ∪' lamInputUnion t')) ∅ with
            | isFalse hFn =>
              have hAv := ((LamConDepth_of_fun_nonempty A hFn).1 hin').1
              exact LamEntDepth_weaken_fun A hsubxs hin' hAv hzx
            | isTrue hFhin =>
              match z with
              | .bot => exact LamEntDepth.bot hin'
              | .atom x =>
                exact LamEntDepth_weaken_atom A hsubxs hin' hFhin hzx
              | .funTok xs1 ys1 =>
                cases decidableEq_finset (lamFunFinset (listToFinset xs)) ∅ with
                | isTrue hxsF =>
                  have hNoFun : lamFunFinset (lamInputUnion s) = ∅ := hInCompat hxsF
                  have hzmem : .funTok xs1 ys1 ∈ lamInputUnion s := hz
                  have : (xs1, ys1) ∈ lamFunFinset (lamInputUnion s) :=
                    mem_lamFunFinset.2 hzmem
                  rw [hNoFun] at this
                  exact False.elim (Finset.notMem_empty _ this)
                | isFalse hxsF =>
                  have : lamFunFinset (listToFinset xs ∪' lamInputUnion t') = ∅ :=
by
                    intro h
                    have : lamFunFinset (listToFinset xs) = ∅ :=
                      Finset.Subset.antisymm
                        (fun p hp => by
                          have hp' : p ∈ lamFunFinset
                              (listToFinset xs ∪' lamInputUnion t') := by
                            rw [lamFunFinset_funion]
                            exact funion.mem.2 (Or.inl hp)
                          rw [h] at hp'; exact False.elim (Finset.notMem_empty p hp
'))
                        (Finset.empty_subset _)
                    exact hxsF this
                  exact False.elim (this hFhin)
          have hDt : rawDepth (.funTok xs ys) ≤ M := by
            rw [← hM]; exact Nat.le_max_right _ _
          have hDu : finsetMaxDepth u ≤ M := by
            rw [← hM]; exact Nat.le_max_left _ _
          have hxs_lt : finsetMaxDepth (listToFinset xs) < M :=
            Nat.lt_of_lt_of_le (finsetMaxDepth.list_lt_funTok xs ys) hDt
          have hys_lt : finsetMaxDepth (listToFinset ys) < M :=
            Nat.lt_of_lt_of_le (finsetMaxDepth.list_lt_funTok_out xs ys) hDt
          have bound_in : ∀ z ∈ lamInputUnion s,
              Nat.max (finsetMaxDepth (listToFinset xs ∪' lamInputUnion t'))
                (rawDepth z) < M := by
            intro z hz
            have hDuBig : finsetMaxDepth (listToFinset xs ∪' lamInputUnion t') < M
:= by
              have hle := finsetMaxDepth_funion_le (listToFinset xs) (lamInputUnion
t')
              refine Nat.lt_of_le_of_lt hle (Nat.max_lt.2 ⟨hxs_lt, ?_⟩)
```

```
            by_cases ht'e : t' = ∅
            · rw [ht'e, lamInputUnion_empty]
              exact Nat.lt_of_le_of_lt (Nat.zero_le _) hxs_lt
            · obtain ⟨q, hq⟩ := Finset.nonempty_of_ne_empty ht'e
              exact Nat.lt_of_lt_of_le
                (finsetMaxDepth.inputUnion_lt ht'sub hWorld) hDu
          have hz_lt : rawDepth z < M := by
            rcases (mem_lamInputUnion).1 hz with ⟨p, hp', hz'⟩
            have htok : .funTok p.1 p.2 ∈ u := mem_lamFunFinset.1 (hs hp')
            exact Nat.lt_of_lt_of_le
              (Nat.lt_of_lt_of_le
                (rawDepth_funTok_gt_left ((mem_listToFinset).1 hz'))
                (le_finsetMaxDepth_of_mem htok))
              hDu
          exact Nat.max_lt.2 ⟨hDuBig, hz_lt⟩
        have hbig : LamConDepth A
            ((listToFinset xs ∪' lamInputUnion t') ∪' lamInputUnion s) :=
          LamConDepth_funion_of_entSet A M ihIns hin' hEnt_big bound_in
        have hin_st' : LamConDepth A (lamInputUnion s ∪' lamInputUnion t') := by
          have heq :
              (listToFinset xs ∪' lamInputUnion t') ∪' lamInputUnion s =
                listToFinset xs ∪' (lamInputUnion s ∪' lamInputUnion t') := by
            rw [funion_assoc_lam, funion_comm_lam (lamInputUnion t')]
          have hbig' : LamConDepth A
              (listToFinset xs ∪' (lamInputUnion s ∪' lamInputUnion t')) := by
            rwa [heq] at hbig
          exact LamConDepth_mono A (funion.subset_right _ _) hbig'
        have hin_union : LamConDepth A (lamInputUnion (s ∪' t')) := by
          rwa [lamInputUnion_funion]
        have hout_union : LamConDepth A
            (lamOutputUnion s ∪' lamOutputUnion t') := by
          have hout : LamConDepth A (lamOutputUnion (s ∪' t')) :=
            ((LamConDepth_of_fun_nonempty A hWorld).1 hu).2.2 (s ∪' t') hsub
hin_union
          rwa [lamOutputUnion_funion] at hout
        have hEnt_out' : ∀ z ∈ listToFinset ys,
            LamEntDepth A (lamOutputUnion s ∪' lamOutputUnion t') z := by
          intro z hz
          have hzx := hEntOut z hz
          have hsub0 : lamOutputUnion s ⊆
              lamOutputUnion s ∪' lamOutputUnion t' := funion.subset_left _ _
          cases decidableEq_finset
              (lamFunFinset (lamOutputUnion s ∪' lamOutputUnion t')) ∅ with
          | isFalse hFn =>
            have hAv := ((LamConDepth_of_fun_nonempty A hFn).1 hout_union).1
            exact LamEntDepth_weaken_fun A hsub0 hout_union hAv hzx
          | isTrue hFo =>
            match z with
            | .bot => exact LamEntDepth.bot hout_union
            | .atom x =>
              exact LamEntDepth_weaken_atom A hsub0 hout_union hFo hzx
            | .funTok xs1 ys1 =>

              cases hzx with
              | funTok _ _ _ _ hs0 hWo _ _ _ =>
                have hOsF : lamFunFinset (lamOutputUnion s) = ∅ :=
```

```
                  Finset.Subset.antisymm
                    (fun p hp => by
                      have : p ∈ lamFunFinset
                          (lamOutputUnion s ∪' lamOutputUnion t') := by
                        rw [lamFunFinset_funion]
                        exact funion.mem.2 (Or.inl hp)
                      rw [hFo] at this
                      exact False.elim (Finset.notMem_empty p this))
                    (Finset.empty_subset _)
                exact False.elim (hWo hOsF)
        have bound_out : ∀ z ∈ listToFinset ys,
            Nat.max (finsetMaxDepth (lamOutputUnion s ∪' lamOutputUnion t'))
              (rawDepth z) < M := by
          intro z hz
          have hDuOut : finsetMaxDepth
              (lamOutputUnion s ∪' lamOutputUnion t') < M := by
            have hle := finsetMaxDepth_funion_le (lamOutputUnion s) (
lamOutputUnion t')
            refine Nat.lt_of_le_of_lt hle (Nat.max_lt.2 ⟨?_, ?_⟩)
            · by_cases hs0 : s = ∅
              · rw [hs0, lamOutputUnion_empty]
                exact Nat.lt_of_le_of_lt (Nat.zero_le _) hys_lt
              · exact Nat.lt_of_lt_of_le
                  (finsetMaxDepth.outputUnion_lt hs hWorld) hDu
            · by_cases ht'e : t' = ∅
              · rw [ht'e, lamOutputUnion_empty]
                exact Nat.lt_of_le_of_lt (Nat.zero_le _) hys_lt
              · exact Nat.lt_of_lt_of_le
                  (finsetMaxDepth.outputUnion_lt ht'sub hWorld) hDu
          have hz_lt : rawDepth z < M :=
            Nat.lt_of_lt_of_le
              (rawDepth_funTok_gt_right ((mem_listToFinset).1 hz)) hDt
          exact Nat.max_lt.2 ⟨hDuOut, hz_lt⟩
        have hfinal : LamConDepth A
            ((lamOutputUnion s ∪' lamOutputUnion t') ∪' listToFinset ys) :=
          LamConDepth_funion_of_entSet A M ihIns hout_union hEnt_out' bound_out
        have hout_t : LamConDepth A
            (listToFinset ys ∪' lamOutputUnion t') := by
          have heq :
              (lamOutputUnion s ∪' lamOutputUnion t') ∪' listToFinset ys =
                lamOutputUnion s ∪' (listToFinset ys ∪' lamOutputUnion t') := by
            rw [funion_assoc_lam, funion_comm_lam (lamOutputUnion t')]
          have hfinal' : LamConDepth A
              (lamOutputUnion s ∪' (listToFinset ys ∪' lamOutputUnion t')) := by
            rwa [heq] at hfinal
          exact LamConDepth_mono A (funion.subset_right _ _) hfinal'
        show LamConDepth A (lamOutputUnion sel)
        rwa [← ht_eq, lamOutputUnion_insert]
      else
        have hsel' : sel ⊆ lamFunFinset u := by
          intro p hp'
          have := hsel hp'
          rw [lamFunFinset_insert_funTok] at this
          rcases Finset.mem_insert.mp this with rfl | h
          · exact False.elim (hp hp')
          · exact h
```

```
            exact ((LamConDepth_of_fun_nonempty A hWorld).1 hu).2.2 sel hsel' hIn

def lamDForgetInsert :
    LamDToken A → Finset (RawLamToken α) → Finset (RawLamToken α) :=
  fun t => insert (lamDForget A t)

theorem LamEntDepth_trans {u v : Finset (RawLamToken α)} {c : RawLamToken α}
    (hv : LamConDepth A v) (hu : LamConDepth A u)
    (hEnts : ∀ t ∈ u, LamEntDepth A v t) (ht : LamEntDepth A u c) :
    LamEntDepth A v c := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  suffices ∀ M u v c,
      Nat.max (finsetMaxDepth v) (Nat.max (finsetMaxDepth u) (rawDepth c)) = M →
      LamConDepth A v → LamConDepth A u →
      (∀ t ∈ u, LamEntDepth A v t) → LamEntDepth A u c → LamEntDepth A v c from
    this _ u v c rfl hv hu hEnts ht
  intro M
  induction M using Nat.strong_induction_on with
  | h M ih =>
    intro u v c hM hv hu hEnts ht
    have hleV : finsetMaxDepth v ≤ M := by rw [← hM]; exact Nat.le_max_left _ _
    have hleU : finsetMaxDepth u ≤ M := by
      rw [← hM]; exact Nat.le_trans (Nat.le_max_left _ _) (Nat.le_max_right _ _)
    have hleC : rawDepth c ≤ M := by
      rw [← hM]; exact Nat.le_trans (Nat.le_max_right _ _) (Nat.le_max_right _ _)

    match c with
    | .bot => exact LamEntDepth.bot hv

    | .atom x =>
      cases ht with
      | @atom u x hu' hF hne hEnt =>
        obtain ⟨y0, hy0⟩ := Finset.nonempty_of_ne_empty hne
        have hEy0 := hEnts _ (mem_lamAtomFinset.1 hy0)
        have hFv : lamFunFinset v = ∅ := by
          cases hEy0 with
          | atom _ hFv' _ _ => exact hFv'
        have hneV : lamAtomFinset v = ∅ := by
          cases hEy0 with
          | atom _ _ hneV _ => exact hneV
        have hAtomEnt : ∀ y ∈ lamAtomFinset u, A.Ent (lamAtomFinset v) y := by
          intro y hy
          have hEy := hEnts _ (mem_lamAtomFinset.1 hy)
          cases hEy with
          | atom _ _ _ hE => exact hE
        exact LamEntDepth.atom hv hFv hneV
          (A.ent_trans ((LamConDepth_of_fun_empty A hFv).1 hv)
            ((LamConDepth_of_fun_empty A hF).1 hu') hAtomEnt hEnt)

    | .funTok xs ys =>
      cases ht with
      | @funTok u xs ys s hu' hA hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>
        obtain ⟨p0, hp0⟩ := Finset.nonempty_of_ne_empty hWorld
        have hEp0 := hEnts _ (mem_lamFunFinset.1 hp0)
        have hAv : lamAtomFinset v = ∅ := by
```

```
      cases hEp0 with
      | funTok _ hAv' _ _ _ _ _ _ _ => exact hAv'
    have hFv : lamFunFinset v = ∅ := by
      cases hEp0 with
      | funTok _ _ _ _ _ hFv' _ _ _ => exact hFv'
    have hWit : ∀ p ∈ s, ∃ s_p ⊆ lamFunFinset v,
        LamConDepth A (listToFinset p.1) ∧ LamConDepth A (listToFinset p.2) ∧
        (∀ r ∈ s_p, ∀ z ∈ listToFinset r.1, LamEntDepth A (listToFinset p.1) z) ∧
        (∀ z ∈ listToFinset p.2, LamEntDepth A (lamOutputUnion s_p) z) := by
      intro p hp
      have hEp := hEnts _ (mem_lamFunFinset.1 (hs hp))
      cases hEp with
      | @funTok _ _ _ s_p _ _ hCi hCo hs_p _ _ hEi hEo =>
        exact ⟨s_p, hs_p, hCi, hCo, hEi, hEo⟩

    have hComb : ∀ (s' : Finset (List (RawLamToken α) × List (RawLamToken α))),
        s' ⊆ s →
        ∃ S ⊆ lamFunFinset v,
          (∀ r ∈ S, ∀ z ∈ listToFinset r.1, LamEntDepth A (listToFinset xs) z) ∧
          (∀ z ∈ lamOutputUnion s', LamEntDepth A (lamOutputUnion S) z) ∧
          LamConDepth A (lamOutputUnion S) := by
      intro s'
      induction s' using Finset.induction_on with
      | empty =>
        intro _
        refine ⟨(∅ : Finset (List (RawLamToken α) × List (RawLamToken α))),
          Finset.empty_subset _, ?_, ?_, ?_⟩
        · intro _ hr; exact False.elim (Finset.notMem_empty _ hr)
        · intro z hz
          rw [lamOutputUnion_empty] at hz
          exact False.elim (Finset.notMem_empty z hz)
        · rw [lamOutputUnion_empty]; exact LamConDepth_empty A
      | insert q sR hq ihS =>
        intro hs'
        have hsR : sR ⊆ s := fun p hp => hs' (Finset.mem_insert_of_mem hp)
        have hqS : q ∈ s := hs' (Finset.mem_insert_self q sR)
        obtain ⟨S', hS'sub, hS'in, hS'out, hConS'⟩ := ihS hsR
        obtain ⟨Sq, hSqsub, hConQIn, hConQOut, hSqin, hSqout⟩ := hWit q hqS
        have hSsub : S' ∪' Sq ⊆ lamFunFinset v :=
          funion.subset_iff.mpr ⟨hS'sub, hSqsub⟩
        have hEntAll : ∀ t ∈ lamInputUnion (S' ∪' Sq),
            LamEntDepth A (listToFinset xs) t := by
          intro t ht
          rcases (mem_lamInputUnion).1 ht with ⟨r, hr, ht'⟩
          rcases funion.mem.1 hr with hr | hr
          · exact hS'in r hr t ht'
          · refine ih _ ?_ (listToFinset q.1) (listToFinset xs) t rfl
              hConIn hConQIn (fun w hw => hEntIn q hqS w hw) (hSqin r hr t ht')
            have htok : .funTok q.1 q.2 ∈ u := mem_lamFunFinset.1 (hs hqS)
            have htokR : .funTok r.1 r.2 ∈ v :=
              mem_lamFunFinset.1 (hSsub (funion.mem.2 (Or.inr hr)))
            have hzt := rawDepth_funTok_gt_left (v := r.2) ((mem_listToFinset).1
ht')
            have hDq := (finsetMaxDepth.of_mem_funTok htok).1
            have hDxs := finsetMaxDepth.list_lt_funTok xs ys
            exact Nat.max_lt.2 ⟨Nat.lt_of_lt_of_le hDxs hleC,
```

```
            Nat.max_lt.2 ⟨Nat.lt_of_lt_of_le hDq hleU,
              Nat.lt_of_lt_of_le
                (Nat.lt_of_lt_of_le hzt (le_finsetMaxDepth_of_mem htokR))
                hleV⟩⟩
        have hinU : LamConDepth A (lamInputUnion (S' ∪' Sq)) :=
          LamConDepth_mono A (funion.subset_right _ _)
            (LamConDepth_funion_of_entSet A M
              (fun _ _ _ hE => LamConDepth_insert_of_ent A hE)
              hConIn hEntAll (fun t ht => by
                rcases (mem_lamInputUnion).1 ht with ⟨r, hr, ht'⟩
                have hDxs := finsetMaxDepth.list_lt_funTok xs ys
                have htok : .funTok r.1 r.2 ∈ v := mem_lamFunFinset.1 (hSsub hr)
                exact Nat.max_lt.2 ⟨Nat.lt_of_lt_of_le hDxs hleC,
                  Nat.lt_of_lt_of_le
                    (Nat.lt_of_lt_of_le
                      (rawDepth_funTok_gt_left (v := r.2) ((mem_listToFinset).1 ht
'))
                      (le_finsetMaxDepth_of_mem htok)) hleV⟩)))
        have hConU : LamConDepth A (lamOutputUnion (S' ∪' Sq)) :=
          ((LamConDepth_of_fun_nonempty A hFv).1 hv).2.2 (S' ∪' Sq) hSsub hinU
        have hConU' : LamConDepth A (lamOutputUnion S' ∪' lamOutputUnion Sq) := by
          rw [← lamOutputUnion_funion]; exact hConU
        refine ⟨S' ∪' Sq, hSsub, ?_, ?_, hConU⟩
        · intro r hr z hz
          rcases funion.mem.1 hr with hr | hr
          · exact hS'in r hr z hz
          · exact hEntAll _ ((mem_lamInputUnion).2 ⟨r, funion.mem.2 (Or.inr hr), hz⟩
)
        · intro z hz
          rw [lamOutputUnion_funion]
          have hz'' : z ∈ listToFinset q.2 ∪' lamOutputUnion sR := by
            rwa [lamOutputUnion_insert] at hz
          rcases funion.mem.1 hz'' with hzQ | hzR
          · have hzx := hSqout z hzQ
            have hsub := funion.subset_right (lamOutputUnion S') (lamOutputUnion
Sq)
            cases decidableEq_finset
                (lamFunFinset (lamOutputUnion S' ∪' lamOutputUnion Sq)) ∅ with
            | isFalse hFn =>
              exact LamEntDepth_weaken_fun A hsub hConU'
                ((LamConDepth_of_fun_nonempty A hFn).1 hConU').1 hzx
            | isTrue hFo =>
              match z with
              | .bot => exact LamEntDepth.bot hConU'
              | .atom _ =>
                exact LamEntDepth_weaken_atom A hsub hConU' hFo hzx
              | .funTok _ _ =>
                cases hzx with
                | funTok _ _ _ _ _ hWo _ _ _ =>
                  exact False.elim (hWo (Finset.Subset.antisymm
                    (fun p hp => by
                      have : p ∈ lamFunFinset
                          (lamOutputUnion S' ∪' lamOutputUnion Sq) := by
                        rw [lamFunFinset_funion]
                        exact funion.mem.2 (Or.inr hp)
                      exact False.elim (Finset.notMem_empty p (hFo ▶ this)))
```

```
                    (Finset.empty_subset _)))
          · have hzx := hS'out z hzR
            have hsub := funion.subset_left (lamOutputUnion S') (lamOutputUnion Sq
)
            cases decidableEq_finset
                (lamFunFinset (lamOutputUnion S' ∪' lamOutputUnion Sq)) ∅ with
            | isFalse hFn =>
              exact LamEntDepth_weaken_fun A hsub hConU'
                ((LamConDepth_of_fun_nonempty A hFn).1 hConU').1 hzx
            | isTrue hFo =>
              match z with
              | .bot => exact LamEntDepth.bot hConU'
              | .atom _ =>
                exact LamEntDepth_weaken_atom A hsub hConU' hFo hzx
              | .funTok _ _ =>
                cases hzx with
                | funTok _ _ _ _ _ hWo _ _ _ =>
                  exact False.elim (hWo (Finset.Subset.antisymm
                    (fun p hp => by
                      have : p ∈ lamFunFinset
                          (lamOutputUnion S' ∪' lamOutputUnion Sq) := by
                        rw [lamFunFinset_funion]
                        exact funion.mem.2 (Or.inl hp)
                      exact False.elim (Finset.notMem_empty p (hFo ▶ this)))
                    (Finset.empty_subset _)))
      obtain ⟨S, hSsub, hSin, hSoutS, hConS⟩ :=
        hComb s (Finset.Subset.refl _)
      have hConOutS : LamConDepth A (lamOutputUnion s) :=
        ((LamConDepth_of_fun_nonempty A hWorld).1 hu').2.2 s hs
          (LamConDepth_mono A (funion.subset_right _ _)
            (LamConDepth_funion_of_entSet A M
              (fun _ _ _ hE => LamConDepth_insert_of_ent A hE)
              hConIn
              (fun t ht => by
                rcases (mem_lamInputUnion).1 ht with ⟨p, hp, ht'⟩
                exact hEntIn p hp t ht')
              (fun t ht => by
                rcases (mem_lamInputUnion).1 ht with ⟨p, hp, ht'⟩
                have hDxs := finsetMaxDepth.list_lt_funTok xs ys
                have htok : .funTok p.1 p.2 ∈ u := mem_lamFunFinset.1 (hs hp)
                exact Nat.max_lt.2 ⟨Nat.lt_of_lt_of_le hDxs hleC,
                  Nat.lt_of_lt_of_le

                    (Nat.lt_of_lt_of_le
                      (rawDepth_funTok_gt_left ((mem_listToFinset).1 ht'))
                      (le_finsetMaxDepth_of_mem htok)) hleU⟩))))
      have hSout : ∀ z ∈ listToFinset ys, LamEntDepth A (lamOutputUnion S) z := by
        intro z hz
        refine ih _ ?_ (lamOutputUnion s) (lamOutputUnion S) z rfl
          hConS hConOutS (fun t ht => hSoutS t ht) (hEntOut z hz)
        exact Nat.max_lt.2 ⟨
          Nat.lt_of_lt_of_le (finsetMaxDepth.outputUnion_lt hSsub hFv) hleV,
          Nat.max_lt.2 ⟨
            Nat.lt_of_lt_of_le (finsetMaxDepth.outputUnion_lt hs hWorld) hleU,
            Nat.lt_of_lt_of_le
              (rawDepth_funTok_gt_right ((mem_listToFinset).1 hz)) hleC⟩⟩
```

```
        have hInCompat' : lamFunFinset (listToFinset xs) = ∅ →
            lamFunFinset (lamInputUnion S) = ∅ := by
          intro hxsF
          ext p
          constructor
          · intro hp
            have hmem : .funTok p.1 p.2 ∈ lamInputUnion S := mem_lamFunFinset.1 hp
            rcases (mem_lamInputUnion).1 hmem with ⟨r, hr, hr'⟩
            cases hSin r hr _ hr' with
            | funTok _ _ _ _ _ _ hWx _ _ _ => exact False.elim (hWx hxsF)
          · intro hp; exact False.elim (Finset.notMem_empty _ hp)
        exact LamEntDepth.funTok hv hAv hConIn hConOut hSsub hFv hInCompat' hSin hSout

instance : LeftCommutative (lamDForgetInsert (A := A)) :=
  ⟨fun p q s => insert_comm' (lamDForget A p) (lamDForget A q) s⟩

theorem IsLamWF_of_mem {u : Finset (RawLamToken α)} (hu : LamConDepth A u)
    {t : RawLamToken α} (ht : t ∈ u) : LamWF A t := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  match t with
  | .bot | .atom _ => exact ⟨trivial⟩
  | .funTok xs ys =>
    have hmem : (xs, ys) ∈ lamFunFinset u := mem_lamFunFinset.2 ht
    by_cases hF : lamFunFinset u = ∅
    · exact False.elim (Finset.notMem_empty _ (hF ▸ hmem))
    · exact ⟨((LamConDepth_of_fun_nonempty A hF).1 hu).2.1 _ hmem⟩

theorem LamConDepth_singleton_funTok (xs ys : List (RawLamToken α))
    (hxs : LamConDepth A (listToFinset xs)) (hys : LamConDepth A (listToFinset ys)) :
    LamConDepth A ({.funTok xs ys} : Finset (RawLamToken α)) := by
  have : DecidableEq (Finset (List (RawLamToken α) × List (RawLamToken α))) :=
    decidableEq_finset
  have hF : lamFunFinset ({.funTok xs ys} : Finset (RawLamToken α)) = {(xs, ys)} :=
    lamFunFinset_singleton_funTok xs ys
  have hA : lamAtomFinset ({.funTok xs ys} : Finset (RawLamToken α)) = ∅ :=
    lamAtomFinset_singleton_funTok xs ys
  have hne : lamFunFinset ({.funTok xs ys} : Finset (RawLamToken α)) = ∅ := by
    rw [hF]; exact Finset.singleton_ne_empty _
  refine (LamConDepth_of_fun_nonempty A hne).2 ⟨hA, ?_, ?_⟩
  · intro p hp
    have : p = (xs, ys) := Finset.mem_singleton.mp (hF ▸ hp)
    subst this; exact ⟨hxs, hys⟩
  · intro s hs hIn
    rw [hF] at hs
    by_cases hmem : (xs, ys) ∈ s
    · have hsin : s = {(xs, ys)} := by
        ext p
        constructor
        · intro hp
          exact Finset.mem_singleton.mpr (Finset.mem_singleton.mp (hs hp))
        · intro hp
          have : p = (xs, ys) := Finset.mem_singleton.mp hp
          subst this; exact hmem
      subst hsin
      rw [lamOutputUnion_singleton]
```

```
      exact hys
    · have hempty : s = ∅ := by
        ext p
        constructor
        · intro hp
          exact False.elim (hmem (by
            have : p = (xs, ys) := Finset.mem_singleton.mp (hs hp)
            subst this; exact hp))
        · intro hp; exact False.elim (Finset.notMem_empty p hp)
      subst hempty
      rw [lamOutputUnion_empty]
      exact LamConDepth_empty A

def listToLamDFinset :
    (xs : List (RawLamToken α)) →
      (∀ t ∈ xs, LamWF A t) → Finset (LamDToken A)
  | [], _ => ∅
  | t :: ts, h =>
      insert ⟨t, h t List.mem_cons_self⟩
        (listToLamDFinset ts fun w hw => h w (List.mem_cons_of_mem t hw))

theorem mem_foldr_lamDForget (s : Multiset (LamDToken A)) (t : RawLamToken α) :
    t ∈ Multiset.foldr (lamDForgetInsert (A := A)) ∅ s ↔ ∃ p ∈ s, p.val = t := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro ht; exact False.elim (Finset.notMem_empty t ht)
    · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
  · intro p rest ih
    simp only [Multiset.foldr_cons, lamDForgetInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
    constructor
    · rintro (rfl | ⟨q, hq, hq'⟩)
      · exact ⟨p, Or.inl rfl, rfl⟩
      · exact ⟨q, Or.inr hq, hq'⟩
    · rintro ⟨q, hq, hq'⟩
      rcases hq with rfl | hq
      · exact Or.inl hq'.symm
      · exact Or.inr ⟨q, hq, hq'⟩

theorem mem_listToLamDFinset {xs : List (RawLamToken α)}
    {h : ∀ t ∈ xs, LamWF A t} {p : LamDToken A} :
    p ∈ listToLamDFinset A xs h ↔ p.val ∈ xs := by
  induction xs with
  | nil =>
    constructor
    · intro hp; exact False.elim (Finset.notMem_empty _ hp)
    · intro hp; cases hp
  | cons t ts ih =>
    constructor
    · intro hp
      rcases Finset.mem_insert.mp hp with hp | hp
      · subst hp; exact List.mem_cons_self
      · exact List.mem_cons_of_mem _ (ih.1 hp)
    · intro hp
      rcases List.mem_cons.mp hp with rfl | hp
      · exact Finset.mem_insert_self _ _
```

```
      · exact Finset.mem_insert_of_mem (ih.2 hp)

def lamDForgetFinset (u : Finset (LamDToken A)) : Finset (RawLamToken α) :=
  Multiset.foldr (lamDForgetInsert (A := A)) ∅ u.1

theorem mem_lamDForgetFinset {u : Finset (LamDToken A)} {t : RawLamToken α} :
    t ∈ lamDForgetFinset A u ↔ ∃ p ∈ u, p.val = t := by
  simpa [lamDForgetFinset] using mem_foldr_lamDForget A u.1 t

theorem lamDForgetFinset.insert (u : Finset (LamDToken A)) (p : LamDToken A) :
    lamDForgetFinset A (Insert.insert p u) = Insert.insert p.val (lamDForgetFinset A u
    ) := by
  ext t
  constructor
  · intro ht
    rcases (mem_lamDForgetFinset A).1 ht with ⟨q, hq, rfl⟩
    rcases Finset.mem_insert.mp hq with rfl | hq
    · exact Finset.mem_insert_self _ _
    · exact Finset.mem_insert_of_mem ((mem_lamDForgetFinset A).2 ⟨q, hq, rfl⟩)
  · intro ht
    rcases Finset.mem_insert.mp ht with rfl | ht
    · exact (mem_lamDForgetFinset A).2 ⟨p, Finset.mem_insert_self p u, rfl⟩
    · rcases (mem_lamDForgetFinset A).1 ht with ⟨q, hq, rfl⟩
      exact (mem_lamDForgetFinset A).2 ⟨q, Finset.mem_insert_of_mem hq, rfl⟩

theorem lamDForgetFinset.mono {u v : Finset (LamDToken A)} (h : v ⊆ u) :
    lamDForgetFinset A v ⊆ lamDForgetFinset A u := by
  intro t ht
  rcases (mem_lamDForgetFinset A).1 ht with ⟨p, hp, rfl⟩
  exact (mem_lamDForgetFinset A).2 ⟨p, h hp, rfl⟩

theorem lamDForgetFinset.singleton (p : LamDToken A) :
    lamDForgetFinset A {p} = {p.val} := by
  ext t
  constructor
  · intro ht
    rcases (mem_lamDForgetFinset A).1 ht with ⟨q, hq, hq'⟩
    have : q = p := Finset.mem_singleton.mp hq
    subst this; exact Finset.mem_singleton.mpr hq'.symm
  · intro ht
    exact (mem_lamDForgetFinset A).2
      ⟨p, Finset.mem_singleton_self p, (Finset.mem_singleton.mp ht).symm⟩

theorem lamDForget_listToLamDFinset (xs : List (RawLamToken α))
    (h : ∀ t ∈ xs, LamWF A t) :
    lamDForgetFinset A (listToLamDFinset A xs h) = listToFinset xs := by
  ext t
  rw [mem_lamDForgetFinset, mem_listToFinset]
  constructor
  · rintro ⟨p, hp, rfl⟩
    exact (mem_listToLamDFinset A).1 hp
  · intro ht
    let p : LamDToken A := ⟨t, h t ht⟩
    exact ⟨p, (mem_listToLamDFinset A).2 ht, rfl⟩

def LamDCon (u : Finset (LamDToken A)) : Prop :=
```

```
  LamConDepth A (lamDForgetFinset A u)

theorem LamDCon.singleton (p : LamDToken A) : LamDCon A {p} := by
  rw [LamDCon, lamDForgetFinset.singleton]
  match p.val, p.property with
  | .bot, _ => exact LamConDepth_singleton_bot A
  | .atom x, _ => exact LamConDepth_singleton_atom A x
  | .funTok xs ys, ⟨⟨hxs, hys⟩⟩ => exact LamConDepth_singleton_funTok A xs ys hxs hys

theorem LamDCon.listToLamDFinset (xs : List (RawLamToken α))
    (hCon : LamConDepth A (listToFinset xs)) :
    LamDCon A (listToLamDFinset A xs fun _t ht =>
      IsLamWF_of_mem A hCon ((mem_listToFinset).2 ht)) := by
  rw [LamDCon, lamDForget_listToLamDFinset]
  exact hCon

theorem LamDCon.subset {u v : Finset (LamDToken A)}
    (hu : LamDCon A u) (hv : v ⊆ u) : LamDCon A v :=
  LamConDepth_mono A (lamDForgetFinset.mono A hv) hu

def LamDEnt (u : Finset (LamDToken A)) (p : LamDToken A) : Prop :=
  LamEntDepth A (lamDForgetFinset A u) p.val

theorem LamDEnt.bot {u : Finset (LamDToken A)} (hu : LamDCon A u) :
    LamDEnt A u (lamDBot A) :=
  LamEntDepth.bot hu

theorem LamDEnt.of_mem {u : Finset (LamDToken A)} {p : LamDToken A}
    (hu : LamDCon A u) (hp : p ∈ u) : LamDEnt A u p :=
  LamEntDepth_of_mem A hu ((mem_lamDForgetFinset A).2 ⟨p, hp, rfl⟩)

theorem LamDEnt.trans {u v : Finset (LamDToken A)} {c : LamDToken A}
    (hv : LamDCon A v) (hu : LamDCon A u)
    (hEnts : ∀ t ∈ u, LamDEnt A v t) (ht : LamDEnt A u c) :
    LamDEnt A v c :=
  LamEntDepth_trans A hv hu
    (fun t ht' => by
      rcases (mem_lamDForgetFinset A).1 ht' with ⟨p, hp, rfl⟩
      exact hEnts p hp)
    ht

theorem LamDCon_insert_of_ent {u : Finset (LamDToken A)} {p : LamDToken A}
    (ht : LamDEnt A u p) : LamDCon A (insert p u) := by
  rw [LamDCon, lamDForgetFinset.insert]
  exact LamConDepth_insert_of_ent A ht

end lambdaSystem

open lambdaSystem

def lambdaSystem : InfoSys (LamDToken A) where
  bot := lamDBot A
  Con := {u | LamDCon A u}
  Ent := LamDEnt A
  con_subset := fun hu hv => LamDCon.subset A hu hv
  con_sing := LamDCon.singleton A
```

```
  ent_con := fun h => LamDCon_insert_of_ent A h
  ent_bot := fun hu => LamDEnt.bot A hu
  ent_refl := fun hu hp => LamDEnt.of_mem A hu hp
  ent_trans := fun hv hu hEnts hEntc => LamDEnt.trans A hv hu hEnts hEntc

def lamRhs : InfoSys (SumToken α (FunToken (lambdaSystem A) (lambdaSystem A))) :=
  sumSystem A (functionSystem (lambdaSystem A) (lambdaSystem A))

def lamUnfold (t : LamDToken A) :
    SumToken α (FunToken (lambdaSystem A) (lambdaSystem A)) :=
  match t with
  | ⟨.bot, _⟩ => .bot
  | ⟨.atom x, _⟩ => .left x
  | ⟨.funTok xs ys, ⟨⟨hxs, hys⟩⟩⟩ =>
      let u := listToLamDFinset A xs fun _w hw =>
        IsLamWF_of_mem A hxs ((mem_listToFinset).2 hw)
      let v := listToLamDFinset A ys fun _w hw =>
        IsLamWF_of_mem A hys ((mem_listToFinset).2 hw)
      .right ⟨(u, v), ⟨LamDCon.listToLamDFinset A xs hxs,
        LamDCon.listToLamDFinset A ys hys⟩⟩

theorem mem_foldr_lamForget (s : Multiset (LamToken A)) (t : RawLamToken α) :
    t ∈ Multiset.foldr (lamForgetInsert (A := A)) ∅ s ↔ ∃ p ∈ s, p.val = t := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro ht; exact False.elim (Finset.notMem_empty t ht)
    · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
  · intro p rest ih
    simp only [Multiset.foldr_cons, lamForgetInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
    constructor
    · rintro (rfl | ⟨q, hq, hq'⟩)
      · exact ⟨p, Or.inl rfl, rfl⟩
      · exact ⟨q, Or.inr hq, hq'⟩
    · rintro ⟨q, hq, hq'⟩
      rcases hq with rfl | hq
      · exact Or.inl hq'.symm
      · exact Or.inr ⟨q, hq, hq'⟩

theorem funion_insert_lam (u : Finset (RawLamToken α)) (a : RawLamToken α)
    (t : Finset (RawLamToken α)) : u ∪' insert a t = insert a (u ∪' t) := by
  ext x
  constructor
  · intro hx
    rcases funion.mem.mp hx with hu | hins
    · exact Finset.mem_insert_of_mem (funion.mem.mpr (Or.inl hu))
    · rcases Finset.mem_insert.mp hins with ha | ht
      · exact Finset.mem_insert.mpr (Or.inl ha)
      · exact Finset.mem_insert_of_mem (funion.mem.mpr (Or.inr ht))
  · intro hx
    rcases Finset.mem_insert.mp hx with ha | h
    · exact funion.mem.mpr (Or.inr (Finset.mem_insert.mpr (Or.inl ha)))
    · rcases funion.mem.mp h with hu | ht
      · exact funion.mem.mpr (Or.inl hu)
      · exact funion.mem.mpr (Or.inr (Finset.mem_insert.mpr (Or.inr ht)))
```

```
theorem LamConN_atom_con {n : ℕ} {u : Finset (RawLamToken α)}
    (hu : LamConN A n u) (hne : lamAtomFinset u = ∅) : lamAtomFinset u ∈ A.Con := by
  induction n generalizing u with
  | zero => exact hu.1
  | succ n ih =>
    rcases hu with hu | ⟨hA, _, _⟩
    · exact ih hu hne
    · exact False.elim (hne hA)

theorem LamConN_atom_empty {n : ℕ} {u : Finset (RawLamToken α)}
    (hu : LamConN A n u) (hne : lamFunFinset u = ∅) : lamAtomFinset u = ∅ := by
  induction n generalizing u with
  | zero => exact False.elim (hne hu.2)
  | succ n ih =>
    rcases hu with hu | ⟨hA, _, _⟩
    · exact ih hu hne
    · exact hA

theorem LamConN_down_atom {n : ℕ} {u : Finset (RawLamToken α)}
    (hu : LamConN A n u) (hF : lamFunFinset u = ∅) : LamConN A 0 u := by
  induction n generalizing u with
  | zero => exact hu
  | succ n ih =>
    rcases hu with hu | ⟨hA, _, _⟩
    · exact ih hu hF
    · exact ⟨by rw [hA]; exact A.con_empty, hF⟩

theorem LamConN_empty (n : ℕ) : LamConN A n (∅ : Finset (RawLamToken α)) := by
  induction n with
  | zero =>
    change lamAtomFinset ∅ ∈ A.Con ∧ lamFunFinset ∅ = ∅
    rw [lamAtomFinset_empty, lamFunFinset_empty]
    exact ⟨A.con_empty, rfl⟩
  | succ n ih => exact Or.inl ih

theorem LamConN_exists_FunCon {n : ℕ} {u : Finset (RawLamToken α)}
    (hu : LamConN A n u) (hne : lamFunFinset u = ∅) :
    ∃ k, k + 1 ≤ n ∧ lamAtomFinset u = ∅ ∧
      (∀ p ∈ lamFunFinset u,
        LamConN A k (listToFinset p.1) ∧ LamConN A k (listToFinset p.2)) ∧
      (∀ s ⊆ lamFunFinset u,
        LamConN A k (lamInputUnion s) → LamConN A k (lamOutputUnion s)) := by
  induction n generalizing u with
  | zero => exact False.elim (hne hu.2)
  | succ n ih =>
    rcases hu with hu | ⟨hA, hWF, hFun⟩
    · rcases ih hu hne with ⟨k, hk, H⟩
      exact ⟨k, Nat.le_succ_of_le hk, H⟩
    · exact ⟨n, Nat.le_refl _, hA, hWF, hFun⟩

theorem LamConN_fun_FunCon_right {n : ℕ} {u : Finset (RawLamToken α)}
    (_hA : lamAtomFinset u = ∅)
    (_hWF : ∀ p ∈ lamFunFinset u,
      LamConN A n (listToFinset p.1) ∧ LamConN A n (listToFinset p.2))
    (hFun : ∀ s ⊆ lamFunFinset u,
      LamConN A n (lamInputUnion s) → LamConN A n (lamOutputUnion s))
```

```
    {s : Finset (List (RawLamToken α) × List (RawLamToken α))}
    (hs : s ⊆ lamFunFinset u) (hIn : LamConN A n (lamInputUnion s)) :
    LamConN A n (lamOutputUnion s) :=
  hFun s hs hIn

theorem LamConN_fun_empty {n : ℕ} {u : Finset (RawLamToken α)}
    (hu : LamConN A n u) (hne : lamAtomFinset u = ∅) : lamFunFinset u = ∅ := by
  induction n generalizing u with
  | zero => exact hu.2
  | succ n ih =>
    rcases hu with hu | ⟨hA, _, _⟩
    · exact ih hu hne
    · exact False.elim (hne hA)

theorem LamConN_insert_bot {n : ℕ} {u : Finset (RawLamToken α)}
    (hu : LamConN A n u) : LamConN A n (insert (.bot : RawLamToken α) u) := by
  induction n generalizing u with
  | zero =>
    exact ⟨by rw [lamAtomFinset_insert_bot]; exact hu.1,
      by rw [lamFunFinset_insert_bot]; exact hu.2⟩
  | succ n ih =>
    rcases hu with hu | ⟨hA, hWF, hFun⟩
    · exact Or.inl (ih hu)
    · exact Or.inr ⟨by rw [lamAtomFinset_insert_bot]; exact hA,
        fun p hp => hWF p (by rwa [lamFunFinset_insert_bot] at hp),
        fun s hs hIn =>
          hFun s (fun p hp => by rw [← lamFunFinset_insert_bot u]; exact hs hp) hIn⟩

theorem LamConN_mono {n m : ℕ} (hmn : n ≤ m) {u : Finset (RawLamToken α)}
    (hu : LamConN A n u) : LamConN A m u := by
  induction hmn with
  | refl => exact hu
  | step _ ih => exact Or.inl ih

theorem LamCon_empty : LamCon A (∅ : Finset (RawLamToken α)) :=
  ⟨0, by
    change lamAtomFinset ∅ ∈ A.Con ∧ lamFunFinset ∅ = ∅
    rw [lamAtomFinset_empty, lamFunFinset_empty]
    exact ⟨A.con_empty, rfl⟩⟩

theorem LamCon_singleton_atom (x : α) :
    LamCon A ({.atom x} : Finset (RawLamToken α)) := by
  refine ⟨0, ?_⟩
  change lamAtomFinset _ ∈ A.Con ∧ lamFunFinset _ = ∅
  constructor
  · have h : lamAtomFinset ({.atom x} : Finset (RawLamToken α)) = {x} := by
      ext y
      simp only [mem_lamAtomFinset, Finset.mem_singleton]
      exact ⟨RawLamToken.atom.inj, congrArg RawLamToken.atom⟩
    rw [h]; exact A.con_sing x
  · ext p
    constructor
    · intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
    · intro hp; exact False.elim (Finset.notMem_empty p hp)
```

```
theorem LamCon_singleton_bot : LamCon A ({.bot} : Finset (RawLamToken α)) := by
  refine ⟨0, ?_⟩
  change lamAtomFinset _ ∈ A.Con ∧ lamFunFinset _ = ∅
  constructor
  · have h : lamAtomFinset ({.bot} : Finset (RawLamToken α)) = ∅ := by
      ext x
      constructor
      · intro hx; exact False.elim (nomatch mem_lamAtomFinset.1 hx)
      · intro hx; exact False.elim (Finset.notMem_empty x hx)
    rw [h]; exact A.con_empty
  · ext p
    constructor
    · intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
    · intro hp; exact False.elim (Finset.notMem_empty p hp)

theorem LamCon_subset {u : Finset (RawLamToken α)} (hu : LamCon A u) :
    ∀ {v}, v ⊆ u → LamCon A v := by
  rcases hu with ⟨n, hu⟩
  induction n generalizing u with
  | zero =>
    intro v hv
    refine ⟨0, ?_⟩
    change lamAtomFinset v ∈ A.Con ∧ lamFunFinset v = ∅
    rcases (hu : lamAtomFinset u ∈ A.Con ∧ lamFunFinset u = ∅) with ⟨hA, hF⟩
    exact ⟨A.con_subset hA (lamAtomFinset_mono hv),
      Finset.Subset.antisymm
        (fun p hp => by
          have : p ∈ lamFunFinset u := lamFunFinset_mono hv hp
          rw [hF] at this; exact False.elim (Finset.notMem_empty p this))
        (Finset.empty_subset _)⟩
  | succ n ih =>
    intro v hv
    rcases hu with hu | ⟨hA, hWF, hFun⟩
    · exact ih hu hv
    · refine ⟨n + 1, Or.inr ⟨?_, ?_, ?_⟩⟩
      · exact Finset.Subset.antisymm
          (fun x hx => by
            have : x ∈ lamAtomFinset u := lamAtomFinset_mono hv hx
            rw [hA] at this; exact False.elim (Finset.notMem_empty x this))
          (Finset.empty_subset _)
      · intro p hp; exact hWF p (lamFunFinset_mono hv hp)
      · intro s hs hIn
        exact hFun s (fun p hp => lamFunFinset_mono hv (hs hp)) hIn

theorem LamEntN_con {n : ℕ} {u : Finset (RawLamToken α)} {t : RawLamToken α}
    (ht : LamEntN A n u t) : LamConN A n u := by
  match n, t with
  | 0, .bot => exact ht
  | 0, .atom _ => exact ht.1
  | 0, .funTok _ _ => exact False.elim ht
  | n + 1, t =>
    rcases ht with ht | ht
    · exact Or.inl (LamEntN_con (n := n) ht)
    · match t with
      | .bot => exact ht
      | .atom _ => exact ht.1
```

```
      | .funTok _ _ => exact ht.1

theorem LamEntN_mono {n m : ℕ} (hmn : n ≤ m) {u : Finset (RawLamToken α)}
    {t : RawLamToken α} (ht : LamEntN A n u t) : LamEntN A m u t := by
  induction hmn with
  | refl => exact ht
  | step _ ih => exact Or.inl ih

theorem LamEnt_bot {u : Finset (RawLamToken α)} (hu : LamCon A u) :
    LamEnt A u .bot := by
  rcases hu with ⟨n, hu⟩
  cases n with
  | zero => exact ⟨0, hu⟩
  | succ n => exact ⟨n + 1, Or.inr hu⟩

theorem lamAtomFinset_funion (u v : Finset (RawLamToken α)) :
    lamAtomFinset (u ∪' v) = lamAtomFinset u ∪' lamAtomFinset v := by
  ext x
  constructor
  · intro hx
    have : .atom x ∈ u ∪' v := mem_lamAtomFinset.1 hx
    rcases funion.mem.1 this with hu | hv
    · exact funion.mem.2 (Or.inl (mem_lamAtomFinset.2 hu))
    · exact funion.mem.2 (Or.inr (mem_lamAtomFinset.2 hv))
  · intro hx
    rcases funion.mem.1 hx with hu | hv
    · exact mem_lamAtomFinset.2 (funion.mem.2 (Or.inl (mem_lamAtomFinset.1 hu)))
    · exact mem_lamAtomFinset.2 (funion.mem.2 (Or.inr (mem_lamAtomFinset.1 hv)))

theorem lamDForgetFinset_empty : lamDForgetFinset A (∅ : Finset (LamDToken A)) = ∅ :=
  rfl

theorem lamForgetFinset_empty : lamForgetFinset A (∅ : Finset (LamToken A)) = ∅ :=
  rfl

theorem lamRhs_eq_sum_function :
    lamRhs A = sumSystem A (functionSystem (lambdaSystem A) (lambdaSystem A)) :=
  rfl

theorem lamUnfold_atom (x : α) :
    lamUnfold A ⟨.atom x, ⟨trivial⟩⟩ = SumToken.left x := rfl

theorem lamUnfold_bot : lamUnfold A (lamDBot A) = SumToken.bot := rfl

theorem mem_lamForgetFinset {u : Finset (LamToken A)} {t : RawLamToken α} :
    t ∈ lamForgetFinset A u ↔ ∃ p ∈ u, p.val = t := by
  simpa [lamForgetFinset] using mem_foldr_lamForget A u.1 t

theorem LamConN_FunCon_mono_atom {k m : ℕ} (hkm : k ≤ m)
    {u : Finset (RawLamToken α)}
    (hFun : ∀ s ⊆ lamFunFinset u,
      LamConN A k (lamInputUnion s) → LamConN A k (lamOutputUnion s))
    {s : Finset (List (RawLamToken α) × List (RawLamToken α))}
    (hs : s ⊆ lamFunFinset u)
    (hIn : LamConN A m (lamInputUnion s))
    (hF : lamFunFinset (lamInputUnion s) = ∅) :
```

```
    LamConN A m (lamOutputUnion s) := by
  have h0 := LamConN_down_atom A hIn hF
  have hk : LamConN A k (lamInputUnion s) := LamConN_mono A (Nat.zero_le k) h0
  exact LamConN_mono A hkm (hFun s hs hk)

theorem LamConN_fun_wf {n : ℕ} {u : Finset (RawLamToken α)}
    {p : List (RawLamToken α) × List (RawLamToken α)}
    (hu : LamConN A (n + 1) u) (hp : p ∈ lamFunFinset u) :
    LamConN A n (listToFinset p.1) ∧ LamConN A n (listToFinset p.2) := by
  have aux {m : ℕ} {w : Finset (RawLamToken α)}
      (hw : LamConN A m w) (hq : p ∈ lamFunFinset w) :
      ∃ k, k + 1 ≤ m ∧
        LamConN A k (listToFinset p.1) ∧ LamConN A k (listToFinset p.2) := by
    induction m generalizing w with
    | zero =>
      exact False.elim (by
        have : p ∈ (∅ : Finset _) := by rw [← hw.2]; exact hq
        exact Finset.notMem_empty p this)
    | succ m ih =>
      rcases hw with hw | ⟨_, hWF, _⟩
      · rcases ih hw hq with ⟨k, hk, hk'⟩
        exact ⟨k, Nat.le_succ_of_le hk, hk'⟩
      · exact ⟨m, Nat.le_refl _, hWF p hq⟩
  rcases aux hu hp with ⟨k, hk, h1, h2⟩
  have hk' : k ≤ n := Nat.le_of_succ_le_succ hk
  exact ⟨LamConN_mono A hk' h1, LamConN_mono A hk' h2⟩

theorem LamConN_insert_of_ent_atom {n : ℕ} {u : Finset (RawLamToken α)} {x : α}
    (ht : LamEntN A n u (.atom x)) : LamConN A n (insert (.atom x) u) := by
  induction n generalizing u with
  | zero =>
    rcases ht with ⟨⟨_, hF⟩, _, hEnt⟩
    exact ⟨by rw [lamAtomFinset_insert_atom]; exact A.ent_con hEnt,
      by rw [lamFunFinset_insert_atom]; exact hF⟩
  | succ n ih =>
    rcases ht with ht | ⟨_, hF, _, hEnt⟩
    · exact Or.inl (ih ht)
    · exact LamConN_mono A (Nat.zero_le (n + 1))
        ⟨by rw [lamAtomFinset_insert_atom]; exact A.ent_con hEnt,
          by rw [lamFunFinset_insert_atom]; exact hF⟩

theorem LamCon_singleton_funTok (u v : List (RawLamToken α))
    (hu : LamCon A (listToFinset u)) (hv : LamCon A (listToFinset v)) :
    LamCon A ({.funTok u v} : Finset (RawLamToken α)) := by
  rcases hu with ⟨n, hu⟩
  rcases hv with ⟨m, hv⟩
  let k := max n m
  refine ⟨k + 1, Or.inr ⟨?_, ?_, ?_⟩⟩
  · exact lamAtomFinset_singleton_funTok u v
  · intro p hp
    have : p = (u, v) := by
      have h := Finset.mem_singleton.mp (by
        rw [← lamFunFinset_singleton_funTok u v]; exact hp)
      exact h
    subst this
    exact ⟨LamConN_mono A (Nat.le_max_left n m) hu,
```

```
      LamConN_mono A (Nat.le_max_right n m) hv⟩
  · intro s hs hIn
    have hFun : lamFunFinset ({.funTok u v} : Finset (RawLamToken α)) = {(u, v)} :=
      lamFunFinset_singleton_funTok u v
    rw [hFun] at hs
    if hmem : (u, v) ∈ s then
      have hsin : s = {(u, v)} := by
        ext p
        constructor
        · intro hp
          exact Finset.mem_singleton.mpr (Finset.mem_singleton.mp (hs hp))
        · intro hp
          have : p = (u, v) := Finset.mem_singleton.mp hp
          subst this; exact hmem
      subst hsin
      rw [lamOutputUnion_singleton]
      exact LamConN_mono A (Nat.le_max_right n m) hv
    else
      have hempty : s = ∅ := by
        ext p
        constructor
        · intro hp
          exact False.elim (hmem (by
            have : p = (u, v) := Finset.mem_singleton.mp (hs hp)
            subst this; exact hp))
        · intro hp
          exact False.elim (Finset.notMem_empty p hp)
      subst hempty
      exact LamConN_empty A k

theorem LamConN_insert_of_ent_bot {n : ℕ} {u : Finset (RawLamToken α)}
    (ht : LamEntN A n u .bot) : LamConN A n (insert (.bot : RawLamToken α) u) :=
  LamConN_insert_bot A (LamEntN_con A ht)

theorem LamEnt_con {u : Finset (RawLamToken α)} {t : RawLamToken α}
    (ht : LamEnt A u t) : LamCon A u := by
  rcases ht with ⟨n, ht⟩
  exact ⟨n, LamEntN_con A ht⟩

theorem LamTokenEnt_bot {u : Finset (LamToken A)} (hu : LamTokenCon A u) :
    LamTokenEnt A u (lamBot A) :=
  LamEnt_bot A hu

theorem LamDCon_empty : LamDCon A (∅ : Finset (LamDToken A)) := by
  simpa [LamDCon, lamDForgetFinset_empty] using LamConDepth_empty A

theorem LamTokenCon_empty : LamTokenCon A (∅ : Finset (LamToken A)) := by
  simpa [LamTokenCon, lamForgetFinset_empty] using LamCon_empty A

theorem LamTokenCon_subset {u v : Finset (LamToken A)}
    (hu : LamTokenCon A u) (hv : v ⊆ u) : LamTokenCon A v :=
  LamCon_subset A hu (by
    intro t ht
    rcases (mem_lamForgetFinset A).1 ht with ⟨p, hp, rfl⟩
    exact (mem_lamForgetFinset A).2 ⟨p, hv hp, rfl⟩)
```

```
theorem lamForgetFinset_insert (u : Finset (LamToken A)) (p : LamToken A) :
    lamForgetFinset A (insert p u) = insert p.val (lamForgetFinset A u) := by
  ext t
  constructor
  · intro ht
    rcases (mem_lamForgetFinset A).1 ht with ⟨q, hq, rfl⟩
    rcases Finset.mem_insert.mp hq with rfl | hq
    · exact Finset.mem_insert_self _ _
    · exact Finset.mem_insert_of_mem ((mem_lamForgetFinset A).2 ⟨q, hq, rfl⟩)
  · intro ht
    rcases Finset.mem_insert.mp ht with rfl | ht
    · exact (mem_lamForgetFinset A).2 ⟨p, Finset.mem_insert_self p u, rfl⟩
    · rcases (mem_lamForgetFinset A).1 ht with ⟨q, hq, rfl⟩
      exact (mem_lamForgetFinset A).2 ⟨q, Finset.mem_insert_of_mem hq, rfl⟩

theorem lamForgetFinset_singleton (p : LamToken A) :
    lamForgetFinset A {p} = {p.val} := by
  ext t
  constructor
  · intro ht
    rcases (mem_lamForgetFinset A).1 ht with ⟨q, hq, hq'⟩
    have : q = p := Finset.mem_singleton.mp hq
    subst this; exact Finset.mem_singleton.mpr hq'.symm
  · intro ht
    exact (mem_lamForgetFinset A).2
      ⟨p, Finset.mem_singleton_self p, (Finset.mem_singleton.mp ht).symm⟩

theorem LamConN_fun_FunCon_atom {n : ℕ} {u : Finset (RawLamToken α)}
    (hu : LamConN A (n + 1) u)
    {s : Finset (List (RawLamToken α) × List (RawLamToken α))}
    (hs : s ⊆ lamFunFinset u) (hIn : LamConN A n (lamInputUnion s))
    (hF : lamFunFinset (lamInputUnion s) = ∅) :
    LamConN A n (lamOutputUnion s) := by
  if hempty : s = ∅ then
    subst hempty
    simpa [lamOutputUnion_empty] using LamConN_empty A n
  else
    have hne : lamFunFinset u = ∅ := by
      obtain ⟨p, hp⟩ := Finset.nonempty_of_ne_empty hempty
      exact Finset.ne_empty_of_mem (hs hp)
    rcases LamConN_exists_FunCon A hu hne with ⟨k, hk, _, _, hFun⟩
    have hkn : k ≤ n := Nat.le_of_succ_le_succ hk
    exact LamConN_FunCon_mono_atom A hkn hFun hs hIn hF

theorem LamEntN_of_mem {n : ℕ} {u : Finset (RawLamToken α)} {t : RawLamToken α}
    (hu : LamConN A n u) (ht : t ∈ u) : LamEntN A n u t := by
  induction n generalizing u t with
  | zero =>
    match t with
    | .bot => exact hu
    | .atom x =>
      exact ⟨hu, Finset.ne_empty_of_mem (mem_lamAtomFinset.2 ht),
        A.ent_refl hu.1 (mem_lamAtomFinset.2 ht)⟩
    | .funTok _ _ =>
      exact False.elim (by
        have hp : (_, _) ∈ lamFunFinset u := mem_lamFunFinset.2 ht
```

```
        rw [hu.2] at hp; exact Finset.notMem_empty _ hp)
  | succ n ih =>
    rcases hu with hu | hRight
    · exact Or.inl (ih hu ht)
    · have huFull : LamConN A (n + 1) u := Or.inr hRight
      match t with
      | .bot => exact Or.inr huFull
      | .atom x =>
        exact False.elim (by
          have : x ∈ lamAtomFinset u := mem_lamAtomFinset.2 ht
          rw [hRight.1] at this; exact Finset.notMem_empty x this)
      | .funTok xs ys =>
        have hmem : (xs, ys) ∈ lamFunFinset u := mem_lamFunFinset.2 ht
        have hWF := LamConN_fun_wf A huFull hmem
        refine Or.inr ⟨huFull, hRight.1, hWF.1, hWF.2, ⟨{(xs, ys)}, ?_, ?_, ?_⟩⟩
        · intro p hp
          have : p = (xs, ys) := Finset.mem_singleton.mp hp
          subst this
          exact hmem
        · intro p hp t ht'
          have : p = (xs, ys) := Finset.mem_singleton.mp hp
          subst this
          exact ih hWF.1 ht'
        · intro t ht'
          rw [lamOutputUnion_singleton (xs, ys)]
          exact ih hWF.2 ht'

theorem LamCon_of_forget_singleton (p : LamToken A) :
    LamCon A (lamForgetFinset A {p}) := by
  rw [lamForgetFinset_singleton]
  match p with
  | ⟨.bot, _⟩ => exact LamCon_singleton_bot A
  | ⟨.atom x, _⟩ => exact LamCon_singleton_atom A x
  | ⟨.funTok u v, hWF⟩ => exact LamCon_singleton_funTok A u v hWF.out.1 hWF.out.2

theorem LamEnt_of_mem {u : Finset (RawLamToken α)} {t : RawLamToken α}
    (hu : LamCon A u) (ht : t ∈ u) : LamEnt A u t := by
  rcases hu with ⟨n, hu⟩
  exact ⟨n, LamEntN_of_mem A hu ht⟩

theorem LamTokenCon_singleton (p : LamToken A) : LamTokenCon A {p} :=
  LamCon_of_forget_singleton A p

theorem LamTokenEnt_of_mem {u : Finset (LamToken A)} {p : LamToken A}
    (hu : LamTokenCon A u) (hp : p ∈ u) : LamTokenEnt A u p :=
  LamEnt_of_mem A hu ((mem_lamForgetFinset A).2 ⟨p, hp, rfl⟩)

theorem lambdaSystem_con_axioms :
    (∀ {u v : Finset (LamToken A)}, LamTokenCon A u → v ⊆ u → LamTokenCon A v) ∧
      (∀ p : LamToken A, LamTokenCon A {p}) ∧
        (∀ {u : Finset (LamToken A)}, LamTokenCon A u → LamTokenEnt A u (lamBot A)) ∧
          (∀ {u : Finset (LamToken A)} {p : LamToken A},
            LamTokenCon A u → p ∈ u → LamTokenEnt A u p) :=
  ⟨fun hu hv => LamTokenCon_subset A hu hv,
    LamTokenCon_singleton A,
    fun hu => LamTokenEnt_bot A hu,
```

```
    fun hu hp => LamTokenEnt_of_mem A hu hp⟩

end DomainTheory.InformationSystems.InfoSys
```

5.9.5 Example 8.2: A domain for λ-calculus

A domain for λ-claculus. A lengthy discussion with many references on λ-calculus models can be found in [**?**]. All we wish to remark on here is how the method of construction by solving a domain equation can be fit into the new presentation. What I have added to the previous ideas (that in any case came out of an analysis of finite elements of models) is the general view of information systems. In particular the models obtained this way are *not* lattices—hence, the need for the calculations with Con. I hope that the presentation here makes it clearer how "pure" λ-calculus models can be related to other domains having other types of structures—for instance, those needed in denotational semantics.

The domain equation we wish to solve is:

$$\mathbf{D} \cong \mathbf{A} + (\mathbf{D} \to \mathbf{D}).$$

We proceed in much the same way we did for $\mathbf{T}$, except we must now put in data objects appropriate to the function space. Here is construction, where again Δ is chosen outside $D_{\mathbf{A}}$ and $\Delta_{\mathbf{D}} = (\Delta, \Delta)$:

(1) $\Delta_{\mathbf{D}} \in D_{\mathbf{D}}$;

(2) $(X, \Delta) \in D_{\mathbf{D}}$ whenever $X \in D_{\mathbf{A}}$;

(3) $(\Delta, (u, v)) \in D_{\mathbf{D}}$ whenever $u, v \in \mathrm{Con}_{\mathbf{D}}$;

(4) $\emptyset \in \mathrm{Con}_{\mathbf{D}}$;

(5) $u \cup \{\Delta_{\mathbf{D}}\} \in \mathrm{Con}_{\mathbf{D}}$ whenever $u \in \mathrm{Con}_{\mathbf{D}}$;

(6) $\{(X, \Delta) \mid X \in w\} \in \mathrm{Con}_{\mathbf{D}}$ whenever $w \in \mathrm{Con}_{\mathbf{A}}$; and

(7) $\{(\Delta, (u_0, v_0)), \ldots, (\Delta, (u_{n-1}, v_{n-1}))\} \in \mathrm{Con}_{\mathbf{D}}$ provided $u_i, v_i \in \mathrm{Con}_{\mathbf{D}}$ for all $i < n$ and whenever $I \subseteq \{0, \ldots, n-1\}$ and $\bigcup\{u_i \mid i \in I\} \in \mathrm{Con}_{\mathbf{D}}$, then $\bigcup\{v_i \mid i \in I\} \in \mathrm{Con}_{\mathbf{D}}$.

What is different here from the definition of $\mathbf{T}$ is the fact that the concepts $D_{\mathbf{D}}$ and $\mathrm{Con}_{\mathbf{D}}$ are mutually recursive because the data objects are themselves built from consistent sets. The scheme is based on a combination of the sum construct and the function-space construct, but the mutual recursion allows "feedback" to occur.

To complete the definition we have to give the clauses for the inductive definition of entailment. They are:

(8) $u \vdash_{\mathbf{D}} \Delta_{\mathbf{D}}$ always;

(9) $u \cup \{\Delta_{\mathbf{D}}\} \vdash_{\mathbf{D}} Y$ whenever $u \vdash_{\mathbf{D}} Y$;

(10) $\{(X, \Delta) \mid X \in w\} \vdash_{\mathbf{D}} (W, \Delta)$ whenever $w \vdash_{\mathbf{A}} W$;

(11) $\{(\Delta, (u_0, v_0)), \ldots, (\Delta, (u_{n-1}, v_{n-1}))\} \vdash_{\mathbf{D}} (\Delta, (u', v'))$ whenever $\bigcup\{v_i \mid u' \vdash_{\mathbf{D}} u_i\} \vdash_{\mathbf{D}} v'$ and the set on the left is in $\mathrm{Con}_{\mathbf{D}}$.

Obviously these definitions are much shorter if we have a domain in which all sets are consistent, but there are many reasons for retaining the consistency concept throughout. The check that $\mathbf{D}$ is an information system and satisfies the domain equation is mechanical. We cannot detail here how this construction provides a λ-calculus model.

It is clear that these definitions are constructive, and that, with a suitable Gödel numbering of the data objects, the predicates for consistency and entailment are recursively enumerable. However, the recursion used builds up the predicates by going from less complicated data objects to more complicated ones; therefore, the predicates must be recursive, because, for a certain size data object, the derivation that puts it into the predicate is of a bounded length. This observation helps in the discussion of the computability of the operators defined on these domains—another topic we cannot discuss here. ■

Theorem (`example_8_2`). The right-hand side of the λ-calculus equation is the sum of the given domain with the function space of the solution with itself: $\mathbf{D}$ and $\mathbf{A} + (\mathbf{D} \to \mathbf{D})$ have the same information system of tokens.

Proof. The right-hand side is defined to be that sum of a function space, so the two systems agree. ?

`example_8_2.lean`

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.lambdaSystem

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

open lambdaSystem

theorem example_8_2 :
    ∀ {α : Type*} [DecidableEq α] (A : InfoSys α),
      lamRhs A = sumSystem A (functionSystem (lambdaSystem A) (lambdaSystem A)) :=
  @lamRhs_eq_sum_function

end DomainTheory.InformationSystems.InfoSys
```

5.9.6 Universal information system

Definition (`pairLInsert` commutative). Insertion of left children in the topped domain commutes.

Proof. By cases on the two tokens and commutativity of finite-set insertion. ?

Definition (`pairRInsert` commutative). Insertion of right children in the topped domain commutes.

Proof. By cases on the two tokens and commutativity of finite-set insertion. ?

Definition (`IsUToken` decidable). Whether a topped token belongs to the universal domain is decidable.

Proof. By cases on the token, excluding the top. ?

Definition (`UToken` equality). Equality of universal tokens is decidable.

Proof. It is equality of the underlying topped tokens. ?

Definition (`uForgetInsert` commutative). Forgetting the top commutes with insertion.

Proof. By commutativity of finite-set insertion. ?

Definition (`VToken`). A token of the topped domain is bottom, top, or a left or right child.

Definition (`pairLInsert`). The left-child insertion adds the payload of the matching constructor and leaves every other constructor unchanged.

Definition (`vPairLFinset`). The v Pair Lfinite set.

Theorem (`mem_foldr_pairL`). A token lies in the fold of left children exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_vPairLFinset`). A token lies in the mem v Pair Lfinite set exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vPairLFinset.insert_top`). Inserting the indicated constructor updates insert top by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`pairRInsert`). The right-child insertion adds the payload of the matching constructor and leaves every other constructor unchanged.

Definition (`vPairRFinset`). The v Pair Rfinite set.

Theorem (`mem_foldr_pairR`). A token lies in the fold of right children exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_vPairRFinset`). A token lies in the mem v Pair Rfinite set exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vPairRFinset.empty`). The empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vPairRFinset.insert_top`). Inserting the indicated constructor updates insert top by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`VEnt`). The vEnt are given by the listed constructors.

Theorem (`VEnt.of_ent_top`). The statement of ent top holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`VEnt.of_mem`). The statement of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`VEnt.pairL_inv`). The statement pair L inv holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`VEnt.pairLFinset_of_EntSet`). The statement set of left children of Ent Set holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`VEnt.pairR_inv`). The statement pair R inv holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`VEnt.pairRFinset_of_EntSet`). The statement set of right children of Ent Set holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`VEnt.trans`). The relation trans is transitive.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`IsUToken`). The is UToken are given by the listed constructors.

Definition (`VCon`). The vCon.

Theorem (`VCon.any`). The statement any holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`vBot`). The v Bot.

Theorem (`vPairLFinset_empty`). The v Pair Lfinite set empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vPairLFinset_insert_bot`). Inserting the indicated constructor updates v Pair Lfinite set insert bot by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vPairLFinset_insert_pairL`). Inserting the indicated constructor updates v Pair Lfinite set insert pair L by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vPairLFinset_insert_pairR`). Inserting the indicated constructor updates v Pair Lfinite set insert pair R by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vPairLFinset_mono`). The operation v Pair Lfinite set mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vPairRFinset_insert_bot`). Inserting the indicated constructor updates v Pair Rfinite set insert bot by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vPairRFinset_insert_pairL`). Inserting the indicated constructor updates v Pair Rfinite set insert pair L by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vPairRFinset_insert_pairR`). Inserting the indicated constructor updates v Pair Rfinite set insert pair R by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vPairRFinset_mono`). The operation v Pair Rfinite set mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`vTop`). The v Top.

Definition (`vSystem`). The topped domain is the information system with a top element whose tokens are closed under pairing.

Theorem (`not_VEnt_empty_top`). The claim not VEnt empty top fails.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`VEnt_mono`). The operation vEnt mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`UToken`). A universal token is a token of the topped domain other than the top.

Definition (`vRhs`). The right-hand side of the recursion for the topped domain is the product of that domain with itself, plus bottom and top.

Theorem (`vRhs.eq_product`). The equality eq product holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`vUnfold`). Unfolding a topped token views it as a pair of topped tokens.

Theorem (`vUnfold.bot`). The statement bot holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vUnfold.pairL`). The statement pair L holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`vUnfold.pairR`). The statement pair R holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`uBot`). The u Bot.

Definition (`uForget`). The u Forget.

Definition (`uForgetInsert`). The u Forget Insert adds the payload of the matching constructor and leaves every other constructor unchanged.

Theorem (`VEnt_insert_top_imp`). Inserting the indicated constructor updates vEnt insert top imp by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_foldr_uForget`). A token lies in the mem foldr u Forget exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`not_VEnt_singleton_ne_top`). The claim not VEnt singleton ne top fails.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`not_VEnt_insert_top_of_ent`). The claim not VEnt insert top of ent fails.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`uForgetFinset`). The u Forget finite set.

Theorem (`mem_uForgetFinset`). A token lies in the mem u Forget finite set exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`uForgetFinset.empty`). The empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`uForgetFinset.insert`). Inserting the indicated constructor updates insert by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`uForgetFinset.mono`). The operation mono preserves inclusion.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`uForgetFinset.singleton`). On a singleton, singleton returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`UCon`). The uCon.

Theorem (`UCon.singleton`). On a singleton, singleton returns the payload of that one constructor.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`UCon.subset`). The property subset passes to subsets.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`UEnt`). The uEnt.

Theorem (`UEnt.bot`). The statement bot holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`UEnt.of_mem`). The statement of mem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`UEnt.trans`). The relation trans is transitive.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`UCon_insert_of_ent`). Inserting the indicated constructor updates uCon insert of ent by adding that payload.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`uSystem`). The universal domain is the topped domain with the top element removed.

Theorem (`UCon_empty`). The uCon empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`not_mem_uForget_top`). The claim not mem u Forget top fails.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

uSystem.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.Constructive

namespace uSystem

inductive VToken where
  | bot : VToken
  | top : VToken
  | pairL : VToken → VToken
  | pairR : VToken → VToken
  deriving DecidableEq

private def pairLInsert : VToken → Finset VToken → Finset VToken
  | .pairL t => insert t
  | .bot | .top | .pairR _ => id

instance :
    LeftCommutative (pairLInsert : VToken → Finset VToken → Finset VToken) :=
  ⟨fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl⟩

def vPairLFinset (u : Finset VToken) : Finset VToken :=
  Multiset.foldr pairLInsert (∅ : Finset VToken) u.1

private theorem mem_foldr_pairL (s : Multiset VToken) (t : VToken) :
    t ∈ Multiset.foldr pairLInsert (∅ : Finset VToken) s ↔
      ∃ p ∈ s, p = .pairL t := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro ht; exact False.elim (Finset.notMem_empty t ht)
    · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
  · intro p rest ih
    cases p with
    | pairL a =>
      simp only [Multiset.foldr_cons, pairLInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      · rintro (ht | ⟨q, hq, hq'⟩)
        · exact ⟨.pairL a, Or.inl rfl, congrArg VToken.pairL ht.symm⟩
        · exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · injection hq' with ht; exact Or.inl ht.symm
        · exact Or.inr ⟨q, hq, hq'⟩
    | bot | top | pairR _ =>
```

```
      simp only [Multiset.foldr_cons, pairLInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      · rintro ⟨q, hq, hq'⟩; exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · exact False.elim (nomatch hq')
        · exact ⟨q, hq, hq'⟩

theorem mem_vPairLFinset {u : Finset VToken} {t : VToken} :
    t ∈ vPairLFinset u ↔ .pairL t ∈ u := by
  simpa [vPairLFinset] using mem_foldr_pairL u.1 t

theorem vPairLFinset.insert_top (u : Finset VToken) :
    vPairLFinset (insert (.top : VToken) u) = vPairLFinset u := by
  ext t; simp only [mem_vPairLFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

private def pairRInsert : VToken → Finset VToken → Finset VToken
  | .pairR t => insert t
  | .bot | .top | .pairL _ => id

instance :
    LeftCommutative (pairRInsert : VToken → Finset VToken → Finset VToken) :=
  ⟨fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl⟩

def vPairRFinset (u : Finset VToken) : Finset VToken :=
  Multiset.foldr pairRInsert (∅ : Finset VToken) u.1

private theorem mem_foldr_pairR (s : Multiset VToken) (t : VToken) :
    t ∈ Multiset.foldr pairRInsert (∅ : Finset VToken) s ↔
      ∃ p ∈ s, p = .pairR t := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro ht; exact False.elim (Finset.notMem_empty t ht)
    · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
  · intro p rest ih
    cases p with
    | pairR a =>
      simp only [Multiset.foldr_cons, pairRInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      · rintro (ht | ⟨q, hq, hq'⟩)
        · exact ⟨.pairR a, Or.inl rfl, congrArg VToken.pairR ht.symm⟩
        · exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
        · injection hq' with ht; exact Or.inl ht.symm
        · exact Or.inr ⟨q, hq, hq'⟩
    | bot | top | pairL _ =>
      simp only [Multiset.foldr_cons, pairRInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      · rintro ⟨q, hq, hq'⟩; exact ⟨q, Or.inr hq, hq'⟩
      · rintro ⟨q, hq, hq'⟩
        rcases hq with rfl | hq
```

```
        · exact False.elim (nomatch hq')
        · exact ⟨q, hq, hq'⟩

theorem mem_vPairRFinset {u : Finset VToken} {t : VToken} :
    t ∈ vPairRFinset u ↔ .pairR t ∈ u := by
  simpa [vPairRFinset] using mem_foldr_pairR u.1 t

theorem vPairRFinset.empty : vPairRFinset (∅ : Finset VToken) = ∅ := rfl

theorem vPairRFinset.insert_top (u : Finset VToken) :
    vPairRFinset (insert (.top : VToken) u) = vPairRFinset u := by
  ext t; simp only [mem_vPairRFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

inductive VEnt : Finset VToken → VToken → Prop where
  | bot {u : Finset VToken} : VEnt u .bot
  | top_mem {u : Finset VToken} (h : .top ∈ u) : VEnt u .top
  | top_both {u : Finset VToken}
      (hL : VEnt (vPairLFinset u) .top) (hR : VEnt (vPairRFinset u) .top) :
      VEnt u .top
  | pairL_top {u : Finset VToken} {x : VToken} (h : .top ∈ u) : VEnt u (.pairL x)
  | pairL_proj {u : Finset VToken} {x : VToken}
      (h : VEnt (vPairLFinset u) x) : VEnt u (.pairL x)
  | pairR_top {u : Finset VToken} {y : VToken} (h : .top ∈ u) : VEnt u (.pairR y)
  | pairR_proj {u : Finset VToken} {y : VToken}
      (h : VEnt (vPairRFinset u) y) : VEnt u (.pairR y)

theorem VEnt.of_ent_top {u : Finset VToken} (h : VEnt u .top) (c : VToken) :
    VEnt u c := by
  match c with
  | .bot => exact VEnt.bot
  | .top => exact h
  | .pairL x =>
    cases h with
    | top_mem hm => exact VEnt.pairL_top hm
    | top_both hL _ => exact VEnt.pairL_proj (VEnt.of_ent_top hL x)
  | .pairR y =>
    cases h with
    | top_mem hm => exact VEnt.pairR_top hm
    | top_both _ hR => exact VEnt.pairR_proj (VEnt.of_ent_top hR y)

theorem VEnt.of_mem {u : Finset VToken} {c : VToken}
    (hc : c ∈ u) : VEnt u c := by
  induction c generalizing u with
  | bot => exact VEnt.bot
  | top => exact VEnt.top_mem hc
  | pairL x ih =>
    exact VEnt.pairL_proj (ih (mem_vPairLFinset.2 hc))
  | pairR y ih =>
    exact VEnt.pairR_proj (ih (mem_vPairRFinset.2 hc))

theorem VEnt.pairL_inv {v : Finset VToken} {x : VToken}
    (h : VEnt v (.pairL x)) : .top ∈ v ∨ VEnt (vPairLFinset v) x := by
  cases h with
```

```
  | pairL_top hm => exact Or.inl hm
  | pairL_proj h' => exact Or.inr h'

theorem VEnt.pairLFinset_of_EntSet {u v : Finset VToken}
    (hEnts : ∀ y ∈ u, VEnt v y) :
    .top ∈ v ∨ (∀ y ∈ vPairLFinset u, VEnt (vPairLFinset v) y) := by
  by_cases htop : .top ∈ v
  · exact Or.inl htop
  · refine Or.inr ?_
    intro y hy
    have hy' := hEnts _ (mem_vPairLFinset.1 hy)
    rcases VEnt.pairL_inv hy' with hm | hproj
    · exact False.elim (htop hm)
    · exact hproj

theorem VEnt.pairR_inv {v : Finset VToken} {y : VToken}
    (h : VEnt v (.pairR y)) : .top ∈ v ∨ VEnt (vPairRFinset v) y := by
  cases h with
  | pairR_top hm => exact Or.inl hm
  | pairR_proj h' => exact Or.inr h'

theorem VEnt.pairRFinset_of_EntSet {u v : Finset VToken}
    (hEnts : ∀ y ∈ u, VEnt v y) :
    .top ∈ v ∨ (∀ y ∈ vPairRFinset u, VEnt (vPairRFinset v) y) := by
  by_cases htop : .top ∈ v
  · exact Or.inl htop
  · refine Or.inr ?_
    intro y hy
    have hy' := hEnts _ (mem_vPairRFinset.1 hy)
    rcases VEnt.pairR_inv hy' with hm | hproj
    · exact False.elim (htop hm)
    · exact hproj

theorem VEnt.trans {u v : Finset VToken} {c : VToken}
    (hEnts : ∀ y ∈ u, VEnt v y) (h : VEnt u c) : VEnt v c := by
  induction h generalizing v with
  | bot => exact VEnt.bot
  | top_mem hm => exact hEnts _ hm
  | top_both _ _ ihL ihR =>
    rcases VEnt.pairLFinset_of_EntSet hEnts with hm | hL
    · exact VEnt.of_ent_top (VEnt.top_mem hm) .top
    · rcases VEnt.pairRFinset_of_EntSet hEnts with hm | hR
      · exact VEnt.of_ent_top (VEnt.top_mem hm) .top
      · exact VEnt.top_both (ihL hL) (ihR hR)
  | pairL_top hm => exact VEnt.of_ent_top (hEnts _ hm) _
  | pairL_proj _ ih =>
    rcases VEnt.pairLFinset_of_EntSet hEnts with hm | hL
    · exact VEnt.pairL_top hm
    · exact VEnt.pairL_proj (ih hL)
  | pairR_top hm => exact VEnt.of_ent_top (hEnts _ hm) _
  | pairR_proj _ ih =>
    rcases VEnt.pairRFinset_of_EntSet hEnts with hm | hR
    · exact VEnt.pairR_top hm
    · exact VEnt.pairR_proj (ih hR)

class IsUToken (t : VToken) : Prop where
```

```
  out : t = .top

def VCon (_u : Finset VToken) : Prop := True

theorem VCon.any (u : Finset VToken) : VCon u := trivial

def vBot : VToken := .bot

theorem vPairLFinset_empty : vPairLFinset (∅ : Finset VToken) = ∅ := rfl

theorem vPairLFinset_insert_bot (u : Finset VToken) :
    vPairLFinset (insert (.bot : VToken) u) = vPairLFinset u := by
  ext t; simp only [mem_vPairLFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem vPairLFinset_insert_pairL (u : Finset VToken) (t : VToken) :
    vPairLFinset (insert (.pairL t) u) = insert t (vPairLFinset u) := by
  ext s; simp only [Finset.mem_insert, mem_vPairLFinset]
  constructor
  · rintro (h | h)
    · exact Or.inl (VToken.pairL.inj h)
    · exact Or.inr h
  · rintro (rfl | h)
    · exact Or.inl rfl
    · exact Or.inr h

theorem vPairLFinset_insert_pairR (u : Finset VToken) (t : VToken) :
    vPairLFinset (insert (.pairR t) u) = vPairLFinset u := by
  ext s; simp only [mem_vPairLFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem vPairLFinset_mono {u v : Finset VToken} (h : u ⊆ v) :
    vPairLFinset u ⊆ vPairLFinset v := by
  intro t ht
  exact mem_vPairLFinset.2 (h (mem_vPairLFinset.1 ht))

theorem vPairRFinset_insert_bot (u : Finset VToken) :
    vPairRFinset (insert (.bot : VToken) u) = vPairRFinset u := by
  ext t; simp only [mem_vPairRFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem vPairRFinset_insert_pairL (u : Finset VToken) (t : VToken) :
    vPairRFinset (insert (.pairL t) u) = vPairRFinset u := by
  ext s; simp only [mem_vPairRFinset, Finset.mem_insert]
  constructor
  · rintro (h | h); exacts [False.elim (nomatch h), h]
  · exact Or.inr

theorem vPairRFinset_insert_pairR (u : Finset VToken) (t : VToken) :
    vPairRFinset (insert (.pairR t) u) = insert t (vPairRFinset u) := by
```

```
  ext s; simp only [Finset.mem_insert, mem_vPairRFinset]
  constructor
  · rintro (h | h)
    · exact Or.inl (VToken.pairR.inj h)
    · exact Or.inr h
  · rintro (rfl | h)
    · exact Or.inl rfl
    · exact Or.inr h

theorem vPairRFinset_mono {u v : Finset VToken} (h : u ⊆ v) :
    vPairRFinset u ⊆ vPairRFinset v := by
  intro t ht
  exact mem_vPairRFinset.2 (h (mem_vPairRFinset.1 ht))

def vTop : VToken := .top

instance : DecidablePred IsUToken := fun t =>
  if h : t = .top then isFalse fun ht => ht.out h
  else isTrue ⟨h⟩

def vSystem : InfoSys VToken where
  bot := vBot
  Con := {u | VCon u}
  Ent := VEnt
  con_subset := fun _ _ => trivial
  con_sing := fun _ => trivial
  ent_con := fun _ => trivial
  ent_bot := fun _ => VEnt.bot
  ent_refl := fun _ hp => VEnt.of_mem hp
  ent_trans := fun _ _ hEnts h => VEnt.trans hEnts h

theorem not_VEnt_empty_top : ¬ VEnt (∅ : Finset VToken) .top := by
  have aux : ∀ {u : Finset VToken} {c : VToken},
      VEnt u c → c = .top → u = ∅ → False := by
    intro u c h
    induction h with
    | bot => intro hc; exact nomatch hc
    | top_mem hm =>
      intro _ hu; exact Finset.notMem_empty _ (hu ▸ hm)
    | top_both _ _ ihL _ =>
      intro _ hu
      exact ihL rfl (by rw [hu]; exact vPairLFinset_empty)
    | pairL_top => intro hc; exact nomatch hc
    | pairL_proj => intro hc; exact nomatch hc
    | pairR_top => intro hc; exact nomatch hc
    | pairR_proj => intro hc; exact nomatch hc
  exact fun h => aux h rfl rfl

theorem VEnt_mono {u v : Finset VToken} {c : VToken}
    (huv : u ⊆ v) (h : VEnt u c) : VEnt v c := by
  induction h generalizing v with
  | bot => exact VEnt.bot
  | top_mem hm => exact VEnt.top_mem (huv hm)
  | top_both _ _ ihL ihR =>
    exact VEnt.top_both (ihL (vPairLFinset_mono huv)) (ihR (vPairRFinset_mono huv))
  | pairL_top hm => exact VEnt.pairL_top (huv hm)
```

```
  | pairL_proj _ ih => exact VEnt.pairL_proj (ih (vPairLFinset_mono huv))
  | pairR_top hm => exact VEnt.pairR_top (huv hm)
  | pairR_proj _ ih => exact VEnt.pairR_proj (ih (vPairRFinset_mono huv))

def UToken : Type := { t : VToken // IsUToken t }

def vRhs : InfoSys (ProdToken vSystem vSystem) :=
  productSystem vSystem vSystem

theorem vRhs.eq_product : vRhs = productSystem vSystem vSystem := rfl

def vUnfold (t : VToken) : ProdToken vSystem vSystem :=
  match t with
  | .bot => prodBot vSystem vSystem
  | .top => ⟨(.top, .bot), ⟨Or.inr rfl⟩⟩
  | .pairL x => ⟨(x, .bot), ⟨Or.inr rfl⟩⟩
  | .pairR y => ⟨(.bot, y), ⟨Or.inl rfl⟩⟩

theorem vUnfold.bot : vUnfold .bot = prodBot vSystem vSystem := rfl

theorem vUnfold.pairL (x : VToken) :
    vUnfold (.pairL x) = ⟨(x, .bot), ⟨Or.inr rfl⟩⟩ := rfl

theorem vUnfold.pairR (y : VToken) :
    vUnfold (.pairR y) = ⟨(.bot, y), ⟨Or.inl rfl⟩⟩ := rfl

instance : DecidableEq UToken := Subtype.instDecidableEq

def uBot : UToken := ⟨.bot, by decide⟩

def uForget (t : UToken) : VToken := t.val

def uForgetInsert : UToken → Finset VToken → Finset VToken :=
  fun t => insert (uForget t)

instance : LeftCommutative uForgetInsert :=
  ⟨fun p q s => insert_comm' (uForget p) (uForget q) s⟩

theorem VEnt_insert_top_imp {u : Finset VToken} {y : VToken}
    (hy : y = .top) (hEnt : VEnt u y) (hIns : VEnt (insert y u) .top) :
    VEnt u .top := by
  induction y generalizing u with
  | bot =>
    cases hIns with
    | top_mem hm =>
      rcases Finset.mem_insert.mp hm with h | h
      · exact False.elim (nomatch h)
      · exact VEnt.top_mem h
    | top_both hL hR =>
      rw [vPairLFinset_insert_bot] at hL
      rw [vPairRFinset_insert_bot] at hR
      exact VEnt.top_both hL hR
  | top => exact False.elim (hy rfl)
  | pairL x ih =>
    cases hIns with
    | top_mem hm =>
```

```
      rcases Finset.mem_insert.mp hm with h | h
      · exact False.elim (nomatch h)
      · exact VEnt.top_mem h
    | top_both hL hR =>
      rw [vPairLFinset_insert_pairL] at hL
      rw [vPairRFinset_insert_pairL] at hR
      rcases VEnt.pairL_inv hEnt with htop | hx
      · exact VEnt.top_mem htop
      · by_cases hxTop : x = .top
        · subst hxTop; exact VEnt.top_both hx hR
        · exact VEnt.top_both (ih hxTop hx hL) hR
  | pairR z ih =>
    cases hIns with
    | top_mem hm =>
      rcases Finset.mem_insert.mp hm with h | h


      · exact False.elim (nomatch h)
      · exact VEnt.top_mem h
    | top_both hL hR =>
      rw [vPairLFinset_insert_pairR] at hL
      rw [vPairRFinset_insert_pairR] at hR
      rcases VEnt.pairR_inv hEnt with htop | hz
      · exact VEnt.top_mem htop
      · by_cases hzTop : z = .top
        · subst hzTop; exact VEnt.top_both hL hz
        · exact VEnt.top_both hL (ih hzTop hz hR)

theorem mem_foldr_uForget (s : Multiset UToken) (t : VToken) :
    t ∈ Multiset.foldr uForgetInsert ∅ s ↔ ∃ p ∈ s, p.val = t := by
  refine Multiset.induction_on s ?_ ?_
  · constructor
    · intro ht; exact False.elim (Finset.notMem_empty t ht)
    · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
  · intro p rest ih
    simp only [Multiset.foldr_cons, uForgetInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
    constructor
    · rintro (rfl | ⟨q, hq, hq'⟩)
      · exact ⟨p, Or.inl rfl, rfl⟩
      · exact ⟨q, Or.inr hq, hq'⟩
    · rintro ⟨q, hq, hq'⟩
      rcases hq with rfl | hq
      · exact Or.inl hq'.symm
      · exact Or.inr ⟨q, hq, hq'⟩

theorem not_VEnt_singleton_ne_top {t : VToken} (ht : t = .top) :
    ¬ VEnt ({t} : Finset VToken) .top := by
  intro h
  match t, h with
  | .bot, .top_mem hm =>
    exact False.elim (nomatch Finset.mem_singleton.mp hm)
  | .bot, .top_both hL _ =>
    have hL' : vPairLFinset ({.bot} : Finset VToken) = ∅ := by
      ext x; constructor
      · intro hx; exact False.elim (nomatch mem_vPairLFinset.1 hx)
      · intro hx; exact False.elim (Finset.notMem_empty x hx)
```

```
    exact not_VEnt_empty_top (by rwa [hL'] at hL)
  | .top, _ => exact ht rfl
  | .pairL x, .top_mem hm =>
    exact False.elim (nomatch Finset.mem_singleton.mp hm)
  | .pairL x, .top_both _ hR =>
    have hR' : vPairRFinset ({.pairL x} : Finset VToken) = ∅ := by
      ext y; constructor
      · intro hy; exact False.elim (nomatch mem_vPairRFinset.1 hy)
      · intro hy; exact False.elim (Finset.notMem_empty y hy)
    exact not_VEnt_empty_top (by rwa [hR'] at hR)
  | .pairR y, .top_mem hm =>
    exact False.elim (nomatch Finset.mem_singleton.mp hm)
  | .pairR y, .top_both hL _ =>
    have hL' : vPairLFinset ({.pairR y} : Finset VToken) = ∅ := by
      ext x; constructor
      · intro hx; exact False.elim (nomatch mem_vPairLFinset.1 hx)
      · intro hx; exact False.elim (Finset.notMem_empty x hx)
    exact not_VEnt_empty_top (by rwa [hL'] at hL)

theorem not_VEnt_insert_top_of_ent {u : Finset VToken} {y : VToken}
    (hy : y = .top) (hEnt : VEnt u y) (hNot : ¬ VEnt u .top) :
    ¬ VEnt (insert y u) .top :=
  fun hIns => hNot (VEnt_insert_top_imp hy hEnt hIns)

def uForgetFinset (u : Finset UToken) : Finset VToken :=
  Multiset.foldr uForgetInsert ∅ u.1

theorem mem_uForgetFinset {u : Finset UToken} {t : VToken} :
    t ∈ uForgetFinset u ↔ ∃ p ∈ u, p.val = t := by
  simpa [uForgetFinset] using mem_foldr_uForget u.1 t

theorem uForgetFinset.empty : uForgetFinset (∅ : Finset UToken) = ∅ := rfl

theorem uForgetFinset.insert (u : Finset UToken) (p : UToken) :
    uForgetFinset (Insert.insert p u) = Insert.insert p.val (uForgetFinset u) := by
  ext t
  constructor
  · intro ht
    rcases (mem_uForgetFinset).1 ht with ⟨q, hq, rfl⟩
    rcases Finset.mem_insert.mp hq with rfl | hq
    · exact Finset.mem_insert_self _ _
    · exact Finset.mem_insert_of_mem ((mem_uForgetFinset).2 ⟨q, hq, rfl⟩)
  · intro ht
    rcases Finset.mem_insert.mp ht with rfl | ht
    · exact (mem_uForgetFinset).2 ⟨p, Finset.mem_insert_self p u, rfl⟩
    · rcases (mem_uForgetFinset).1 ht with ⟨q, hq, rfl⟩
      exact (mem_uForgetFinset).2 ⟨q, Finset.mem_insert_of_mem hq, rfl⟩

theorem uForgetFinset.mono {u v : Finset UToken} (h : v ⊆ u) :
    uForgetFinset v ⊆ uForgetFinset u := by
  intro t ht
  rcases (mem_uForgetFinset).1 ht with ⟨p, hp, rfl⟩
  exact (mem_uForgetFinset).2 ⟨p, h hp, rfl⟩

theorem uForgetFinset.singleton (p : UToken) :
    uForgetFinset {p} = {p.val} := by
```

```
  ext t
  constructor
  · intro ht
    rcases (mem_uForgetFinset).1 ht with ⟨q, hq, hq'⟩
    have : q = p := Finset.mem_singleton.mp hq
    subst this; exact Finset.mem_singleton.mpr hq'.symm
  · intro ht
    exact (mem_uForgetFinset).2
      ⟨p, Finset.mem_singleton_self p, (Finset.mem_singleton.mp ht).symm⟩

def UCon (u : Finset UToken) : Prop :=
  ¬ VEnt (uForgetFinset u) .top

theorem UCon.singleton (p : UToken) : UCon {p} := by
  rw [UCon, uForgetFinset.singleton]
  exact not_VEnt_singleton_ne_top p.property.out

theorem UCon.subset {u v : Finset UToken}
    (hu : UCon u) (hv : v ⊆ u) : UCon v :=
  fun h => hu (VEnt_mono (uForgetFinset.mono hv) h)

def UEnt (u : Finset UToken) (p : UToken) : Prop :=
  UCon u ∧ VEnt (uForgetFinset u) p.val

theorem UEnt.bot {u : Finset UToken} (hu : UCon u) : UEnt u uBot :=
  ⟨hu, VEnt.bot⟩

theorem UEnt.of_mem {u : Finset UToken} {p : UToken}
    (hu : UCon u) (hp : p ∈ u) : UEnt u p :=
  ⟨hu, VEnt.of_mem ((mem_uForgetFinset).2 ⟨p, hp, rfl⟩)⟩

theorem UEnt.trans {u v : Finset UToken} {c : UToken}
    (hv : UCon v) (_hu : UCon u)
    (hEnts : ∀ y ∈ u, UEnt v y) (h : UEnt u c) : UEnt v c :=
  ⟨hv, VEnt.trans
    (fun y hy => by
      rcases (mem_uForgetFinset).1 hy with ⟨p, hp, rfl⟩
      exact (hEnts p hp).2)
    h.2⟩

theorem UCon_insert_of_ent {u : Finset UToken} {p : UToken}
    (h : UEnt u p) : UCon (insert p u) := by
  rcases h with ⟨hCon, hEnt⟩
  rw [UCon, uForgetFinset.insert]
  exact not_VEnt_insert_top_of_ent p.property.out hEnt hCon

end uSystem

open uSystem

def uSystem : InfoSys UToken where
  bot := uBot
  Con := {u | UCon u}
  Ent := UEnt
  con_subset := fun hu hv => UCon.subset hu hv
  con_sing := UCon.singleton
```

```
  ent_con := fun h => UCon_insert_of_ent h
  ent_bot := fun hu => UEnt.bot hu
  ent_refl := fun hu hp => UEnt.of_mem hu hp
  ent_trans := fun hv hu hEnts h => UEnt.trans hv hu hEnts h

theorem UCon_empty : UCon (∅ : Finset UToken) := by
  simpa [UCon, uForgetFinset.empty] using not_VEnt_empty_top

theorem not_mem_uForget_top (u : Finset UToken) : .top ∈ uForgetFinset u := by
  intro ht
  rcases (mem_uForgetFinset).1 ht with ⟨p, _, hp⟩
  exact p.property.out hp

end DomainTheory.InformationSystems.InfoSys
```

5.9.7 Example 8.3: A universal domain

A universal domain. As a final example of building up domains recursively, we give a construction of a "universal" domain U. (The reason for the name will be explained presently.) The best way to define U seems to be to define a domain V with a top element first, and then to remove the top.

The recursion for V is remarkably simple. We begin with two distinct objects Δ and ∇ that give information about the bottom and top of V, respectively. Thus, $\Delta_V = \Delta$ by definition. We assume that these two special data objects are "atomic" in the sense that they are not equal to any ordered pair of objects. For the definition of D_V we have these clauses:

(1) $\Delta, \nabla \in D_V$;

(2) $(X, \Delta) \in D_V$ and $(\Delta, Y) \in D_V$ whenever $X, Y \in D_V$.

In other words, we begin with two objects and close up under two flavors of copies of these objects. (A product result is involved here, so that is the reason for structuring the flavors the way we have.)

For V all subsets of D_V are consistent, so all we have left is to define entailment for this domain. The clauses are:

(3) $u \vdash_V \Delta$ always;

(4) $u \vdash_V \nabla$ whenever either $\nabla \in u$ or $\{X \mid (X, \Delta) \in u\} \vdash_V \nabla$ and $\{Y \mid (\Delta, Y) \in u\} \vdash_V \nabla$;

(5) $u \vdash_V (X', \Delta)$ whenever either $\nabla \in u$ or $\{X \mid (X, \Delta) \in u\} \vdash_V X'$; and

(6) $u \vdash_V (\Delta, Y')$ whenever either $\nabla \in u$ or $\{Y \mid (\Delta, Y) \in u\} \vdash_V Y'$.

The proof that V is an information system proceeds as before. Note that, under the above definition of entailment, the data objects Δ, (Δ, Δ), $((\Delta, \Delta), \Delta)$, etc. are all equivalent. There is, however, no other data object equivalent to ∇. The domain equation satisfied by V is:

$$V \cong V \times V.$$

Of course, there are an unlimited number of solutions to this equation, so the fact that V satisfies it tells us very little.

Because ∇ entails everything, we can regard it as a "rogue" object that ought to be banned from polite company: the only element of V it gives any information about is the top element, which is as unhelpful as any element could be. We should simply throw it out as being "inconsistent." What remains is the domain U. Formally we have:

(7) $D_U = D_V - \{\nabla\}$;

(8) $\Delta_U = \Delta_V$;

(9) $\mathrm{Con}_U = \{u \subseteq D_U \mid u \text{ finite and } u \not\vdash_V \nabla\}$; and

(10) $u \vdash_U Y$ iff $u \in \mathrm{Con}_U$, $Y \in D_U$ and $u \vdash_V Y$.

The same style of definition would work in any situation when an information system has a rogue data object that entails everything: there always is a system that results from eliminating all those objects that entail everything. Indeed, we could have always included such an object in any domain and altered the definition to take as elements those deductively closed sets of data objects that *do* not have the rogue object as a member. We did not do this for the reason that superfluous elements cause lots of exceptions in constructs such as product, where there is a temptation to let them enter into various combinations.

Now in U we *do* allow ∇ to enter into combinations—and this is part of the secret of the construction. The consequence is, however, that the domain equation which U satisfies is not too easy to state since it involves an unfamiliar functor. So it is not through such equations that we will understand its nature in a direct way. But it is possible to explain how it works by reference to the steps in the construction.

Imagine the full (infinite) binary tree. The data objects of U are giving information about possible paths in the tree. We think to the tree starting at the root node at the top of the page and growing down. The object Δ gives no information—so no paths are *excluded*. (If we would have allowed ∇, then the information it would have been giving is that all paths are excluded.) The data object (X, Δ) tells us about a path that *either* it is unrestricted on the right half of the tree, *or* on the left, when we start at the node directly below the root, the paths that are excluded from the subtree are those excluded according to X. This makes sense because the subtrees of the binary tree look exactly like the whole tree, so information can be relativized or translated to other positions. With (Δ, Y) the rôles of right and left are interchanged. We could have introduced data objects of the form (X, Y) which tell us information about both halves of the tree at the same time, but the consistent set $\{(X, \Delta), (\Delta, Y)\}$ does the same job. In general consistent sets should be thought of as *conjunctions*; while, in this example, the comma in the ordered pair should be thought of as a *disjunction* when "reading" information objects.

We can now see that a single data object (if it contains ∇) looks down the tree along a finite path to some depth and then *excludes* the rest of the tree below that node. A consistent set of data objects leaves at least one hole, so at least one path is not excluded. The maximal consistent sets of information objects are those giving true information about one single (infinite) path—the total elements of the domain U correspond exactly to the infinite paths in the binary tree. The partial elements are harder to describe geometrically, however. In accumulating information into a consistent set, holes can be left all over the tree. A partial object is therefore of an indeterminate character, since the "path" we are describing might sneak through any one of the holes. (There is, by the way, a precise topological explanation of what is happening. The total elements of U form a well-known topological space, the so-called Cantor space, and the partially ordered set of elements of U is isomorphic to the lattice of *open* sets of the space—save that the whole space is not allowed.)

This is all very well, but what, we ask, is the good of this domain, and why is it called "universal". The proof cannot be given here, but the result is as follows. As a consequence of standard facts about countable Boolean algebras, it can be proved that every "countably based" domain is a subdomain of U. More specifically, if $\mathbf{A}$ is an information system, and if $D_{\mathbf{A}}$ is countable, then there exists a pair of approximable mappings

$$a : \mathbf{A} \to \mathbf{U} \text{ and } b : \mathbf{U} \to \mathbf{A},$$

such that

$$b \circ a = I_{\mathbf{A}} \text{ and } a \circ b \subseteq I_{\mathbf{U}}.$$

This makes $\mathbf{A}$ a special kind of retract of $\mathbf{U}$. The mappings a and b are far from unique, but at least there is one way to give a one-one embedding of the elements of $\mathbf{A}$ into the elements of $\mathbf{U}$.

The universal property of $\mathbf{U}$ can be applied quite widely. For example, since $(\mathbf{U} \to \mathbf{U})$ is a system with only countably many data objects (by explicit construction!), this system is a retract of $\mathbf{U}$. Fixing on one such retraction pair as above, makes $\mathbf{U}$ also into a model of the λ-calculus. Whether different retractions give essentially different models I do not know. But the point of the remark is to show that domains can contain their own function spaces for a variety of interesting reasons. ?

Theorem (`example_8_3`). The empty set of tokens of the universal domain is consistent.

Proof. Empty consistency is the base clause of consistency for the universal domain. ?

example_8_3.lean

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.uSystem

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

open uSystem

set_option linter.unusedVariables false

theorem example_8_3 : UCon (∅ : Finset UToken) :=
  UCon_empty

end DomainTheory.InformationSystems.InfoSys
```

5.9.8 Approximations of domains

Definition (`PEnt`). Entailment in the powerset holds when the output integer already belongs to the input finite set.

Definition (`bar`). The bar.

Definition (`P`). The powerset domain has the natural numbers as tokens, every finite set consistent, and the least entailment.

Theorem (`P.AllConsistent`). The statement all Consistent holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`IdemApprox`). The idem Approx.

Definition (`ReflApprox`). The refl Approx.

Definition (`barElement`). The bar Element.

Definition (`powersetBot`). The bot Element.

Definition (`powersetTop`). The top Element.

Definition (`ConsistentApprox`). The consistent Approx.

Definition (`BotApprox`). A bottom approximation is the approximable map that entails only what the empty set entails.

Definition (`RogueApprox`). A rogue approximation relates some token to the reserved top token.

Definition (`IsDomainApprox`). The is Domain Approx are given by the listed constructors.

Definition (`DomainApprox`). A domain approximation is an approximable self-map of the powerset that is reflexive, transitive, and not rogue.

Definition (`flatRel`). The flat relation on the powerset is the least entailment.

Theorem (`flatRel.empty`). The empty of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`flatRel.of_mem_one`). The statement of mem one holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_barElement`). A token lies in the mem bar Element exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`mem_topElement`). A token lies in the mem top Element exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`scottCon`). The scott Con.

Theorem (`BotApprox_any`). The statement bot Approx any holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`flatMap`). The flat map is the approximable map of the least entailment.

Theorem (`flatMap.consistent`). The statement consistent holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`flatMap.idem`). The statement idem holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`flatMap.refl`). The statement refl holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`flatMap.toElement_of_mem_one`). The statement to Element of mem one holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`flatMap.rogue`). The statement rogue holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`flatMap.isDomain`). The statement is Domain holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Definition (`flatDomainApprox`). The flat Domain Approx.

Theorem (`mem_botElement`). A token lies in the mem bot Element exactly when a matching constructor in the set contributes it.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`not_RogueApprox_idMap`). The claim not Rogue Approx id Map fails.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`barElement_zero_eq_bot`). The equality bar Element zero eq bot holds.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

Theorem (`scottCon_empty_iff_consistent`). The scott Con empty iff consistent of the empty set is empty.

Proof. By cases on the constructors, or by the matching clause of consistency or entailment. ?

DomainApprox.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.comp
public import DomainTheory.InformationSystems.ElementsOfASystem.topElement
public import DomainTheory.InformationSystems.ElementsOfASystem.closure
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition54.idMap

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.Constructive
open ApproximableMap

namespace DomainApprox

def PEnt (u : Finset ℕ) (n : ℕ) : Prop :=
  n = 0 ∨ n ∈ u

def bar (n : ℕ) : Finset ℕ := {n}

def P : InfoSys ℕ where
  bot := 0
  Con := Set.univ
  Ent := PEnt
  con_subset := fun _ _ => trivial
  con_sing := fun _ => trivial
  ent_con := fun _ => trivial
  ent_bot := fun _ => Or.inl rfl
  ent_refl := fun _ ha => Or.inr ha
  ent_trans := by
    intro u v c _ _ hEnts hc
    rcases hc with rfl | hc
    · exact Or.inl rfl
    · exact hEnts c hc

theorem P.AllConsistent : P.AllConsistent := fun _ => trivial

def IdemApprox (r : ApproximableMap P P) : Prop :=
  ApproximableMap.comp r r = r

def ReflApprox (r : ApproximableMap P P) : Prop :=
  ∀ {u v : Finset ℕ}, (idMap P).rel u v → r.rel u v

def barElement (n : ℕ) : P.Element :=
```

```
  P.closure (bar n) trivial

def powersetBot : P.Element :=
  P.closure ∅ trivial

def powersetTop : P.Element :=
  P.topElement P.AllConsistent

def ConsistentApprox (r : ApproximableMap P P) : Prop :=
  ¬ ((barElement 1).carrier ⊆ (r.toElement (barElement 0)).carrier)

def BotApprox (r : ApproximableMap P P) : Prop :=
  (barElement 0).carrier ⊆ (r.toElement powersetBot).carrier

def RogueApprox (r : ApproximableMap P P) : Prop :=
  r.toElement (barElement 1) = powersetTop

structure IsDomainApprox (r : ApproximableMap P P) : Prop where
  refl : ReflApprox r
  idem : IdemApprox r
  bot : BotApprox r
  rogue : RogueApprox r
  consistent : ConsistentApprox r

end DomainApprox

open DomainApprox

def DomainApprox : Type :=
  { r : ApproximableMap P P // IsDomainApprox r }

def flatRel (u v : Finset ℕ) : Prop :=
  (1 ∈ u) ∨ ((∀ n ∈ v, n = 0 ∨ n ∈ u) ∧ 1 ∈ v)

theorem flatRel.empty : flatRel ∅ ∅ :=
  Or.inr ⟨fun _ hn => False.elim (Finset.notMem_empty _ hn), Finset.notMem_empty _⟩

theorem flatRel.of_mem_one {u v : Finset ℕ} (h : 1 ∈ u) : flatRel u v :=
  Or.inl h

theorem mem_barElement (n x : ℕ) :
    x ∈ (barElement n).carrier ↔ x = 0 ∨ x = n := by
  constructor
  · intro hx
    have : PEnt (bar n) x := (closure.mem_iff P (by trivial : bar n ∈ P.Con)).1 hx
    rcases this with rfl | hx'
    · exact Or.inl rfl
    · exact Or.inr (Finset.mem_singleton.mp hx')
  · intro h
    rcases h with rfl | hn
    · exact (closure.mem_iff P (by trivial : bar n ∈ P.Con)).2 (Or.inl rfl)
    · rw [hn]
      exact (closure.mem_iff P (by trivial : bar n ∈ P.Con)).2
        (Or.inr (Finset.mem_singleton_self n))

theorem mem_topElement (x : ℕ) : x ∈ powersetTop.carrier := trivial
```

```
def scottCon (r : ApproximableMap P P) (u : Finset ℕ) : Prop :=
  ¬ ((barElement 1).carrier ⊆
      (r.toElement (P.closure u (by trivial))).carrier)

theorem BotApprox_any (r : ApproximableMap P P) : BotApprox r := by
  intro x hx
  have hx0 : x = 0 := by
    rcases (mem_barElement 0 x).1 hx with rfl | rfl <;> rfl
  subst hx0
  refine ⟨∅, fun a ha => False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha)), ?_⟩
  have hempty := r.empty_rel
  have hbot : P.EntSet ∅ ({0} : Finset ℕ) := by
    intro z hz
    rw [Finset.mem_singleton] at hz; subst hz
    exact Or.inl rfl
  exact r.mono hempty (EntSet.empty P ∅) hbot trivial trivial

def flatMap : ApproximableMap P P where
  rel := flatRel
  rel_dom := fun _ => trivial
  rel_cod := fun _ => trivial
  empty_rel := flatRel.empty
  union_right := by
    intro u v v' hv hv'
    rcases hv with h1 | ⟨hv, hv1⟩
    · exact Or.inl h1
    · rcases hv' with h1 | ⟨hv', hv1'⟩
      · exact Or.inl h1
      · refine Or.inr ⟨?_, ?_⟩
        · intro n hn
          rcases funion.mem.1 hn with h | h
          · exact hv n h
          · exact hv' n h
        · intro h1
          rcases funion.mem.1 h1 with h | h
          · exact hv1 h
          · exact hv1' h
  mono := by
    intro u u' v v' hrel hEntu hEntv _ _
    rcases hrel with h1 | ⟨hv, hv1⟩
    ·
      have h := hEntu 1 h1
      rcases h with h0 | h1'
      · exact False.elim (by cases h0)
      · exact Or.inl h1'
    · refine Or.inr ⟨?_, ?_⟩
      · intro n hn
        have hn' := hEntv n hn
        rcases hn' with rfl | hn'
        · exact Or.inl rfl
        · have hn'' := hv n hn'
          rcases hn'' with rfl | hn''
          · exact Or.inl rfl
          · have hu' := hEntu n hn''
            rcases hu' with rfl | hu'
```

```
            · exact Or.inl rfl
            · exact Or.inr hu'
        · intro h1v
          have h := hEntv 1 h1v
          rcases h with h0 | h1v'
          · exact False.elim (by cases h0)
          · exact hv1 h1v'

theorem flatMap.consistent : ConsistentApprox flatMap := by
  intro hsub
  have h1 : 1 ∈ (barElement 1).carrier := (mem_barElement 1 1).2 (Or.inr rfl)
  have h1' : 1 ∈ (flatMap.toElement (barElement 0)).carrier := hsub h1
  rcases (mem_toElement flatMap (barElement 0)).1 h1' with ⟨u, hu, hrel⟩
  rcases hrel with h1u | ⟨_, hv1⟩
  · have : 1 ∈ (barElement 0).carrier := hu (Finset.mem_coe.2 h1u)
    rcases (mem_barElement 0 1).1 this with h | h
    · exact absurd h (by decide)
    · exact absurd h (by decide)
  · exact hv1 (Finset.mem_singleton_self 1)

theorem flatMap.idem : IdemApprox flatMap := by
  refine ApproximableMap.ext fun u v => ?_
  constructor
  · rintro ⟨w, hf, hg⟩
    rcases hf with h1u | ⟨hw, hw1⟩
    · exact Or.inl h1u
    · rcases hg with h1w | ⟨hv, hv1⟩
      · exact False.elim (hw1 h1w)
      · refine Or.inr ⟨?_, hv1⟩
        · intro n hn
          have hn' := hv n hn
          rcases hn' with rfl | hn'
          · exact Or.inl rfl
          · exact hw n hn'
  · intro h
    rcases h with h1u | ⟨hv, hv1⟩
    · exact ⟨insert 1 v, Or.inl h1u, Or.inl (Finset.mem_insert_self 1 v)⟩
    · exact ⟨v, Or.inr ⟨hv, hv1⟩, Or.inr ⟨fun n hn => Or.inr hn, hv1⟩⟩

theorem flatMap.refl : ReflApprox flatMap := by
  intro u v ⟨_, _, hEnt⟩
  by_cases h1 : 1 ∈ u
  · exact Or.inl h1
  · refine Or.inr ⟨?_, ?_⟩
    · intro n hn; exact hEnt n hn
    · intro h1v
      have h := hEnt 1 h1v
      rcases h with h0 | h
      · exact absurd h0 (by decide)
      · exact h1 h

theorem flatMap.toElement_of_mem_one {x : P.Element} (h1 : 1 ∈ x.carrier) :
    flatMap.toElement x = powersetTop := by
  apply le_antisymm
  · intro _ _; exact mem_topElement _
  · intro n _
```

```
    refine ⟨{1}, ?_, Or.inl (Finset.mem_singleton_self 1)⟩
    intro a ha
    have : a = 1 := Finset.mem_singleton.mp (Finset.mem_coe.1 ha)
    subst this; exact h1

theorem flatMap.rogue : RogueApprox flatMap :=
  flatMap.toElement_of_mem_one ((mem_barElement 1 1).2 (Or.inr rfl))

theorem flatMap.isDomain : IsDomainApprox flatMap where
  refl := flatMap.refl
  idem := flatMap.idem
  bot := BotApprox_any flatMap
  rogue := flatMap.rogue
  consistent := flatMap.consistent

def flatDomainApprox : DomainApprox :=
  ⟨flatMap, flatMap.isDomain⟩

theorem mem_botElement (x : ℕ) : x ∈ powersetBot.carrier ↔ x = 0 := by
  constructor
  · intro hx
    have : PEnt ∅ x := (closure.mem_iff P (by trivial : (∅ : Finset ℕ) ∈ P.Con)).1 hx
    rcases this with rfl | hx'
    · rfl
    · exact False.elim (Finset.notMem_empty _ hx')
  · rintro rfl
    exact (closure.mem_iff P (by trivial : (∅ : Finset ℕ) ∈ P.Con)).2 (Or.inl rfl)

theorem not_RogueApprox_idMap : ¬ RogueApprox (idMap P) := by
  intro h
  have h2 : 2 ∈ ((idMap P).toElement (barElement 1)).carrier := by
    rw [h]; exact mem_topElement 2
  rcases (mem_toElement (idMap P) (barElement 1)).1 h2 with ⟨u, hu, hrel⟩
  have hEnt : P.EntSet u {2} := hrel.2.2
  have h2u := hEnt 2 (Finset.mem_singleton_self 2)
  rcases h2u with h0 | h2u
  · exact absurd h0 (by decide)
  · have : 2 ∈ (barElement 1).carrier := hu (Finset.mem_coe.2 h2u)
    rcases (mem_barElement 1 2).1 this with h0 | h1'
    · exact absurd h0 (by decide)
    · exact absurd h1' (by decide)

theorem barElement_zero_eq_bot : barElement 0 = powersetBot := by
  apply le_antisymm
  · intro x hx
    rcases (mem_barElement 0 x).1 hx with rfl | rfl
    · exact (mem_botElement 0).2 rfl
    · exact (mem_botElement 0).2 rfl
  · intro x hx
    have : x = 0 := (mem_botElement x).1 hx
    subst this
    exact (mem_barElement 0 0).2 (Or.inl rfl)

theorem scottCon_empty_iff_consistent (r : ApproximableMap P P) :
    scottCon r ∅ ↔ ConsistentApprox r := by
  simp only [scottCon, ConsistentApprox, barElement_zero_eq_bot]
```

```
  rfl

end DomainTheory.InformationSystems.InfoSys
```

5.9.9 Example 8.4: A domain of domains

A domain of domains. Not many details can be presented here, but we would also like to remark that even domains can be made into a domain. One way of getting an idea of how this is possible is to note that since subdomains of $\mathbf{U}$ correspond to certain kinds of functions on $\mathbf{U}$, and since the function space of $\mathbf{U}$ is also a subdomain of $\mathbf{U}$, it might be suspected that the subdomains of $\mathbf{U}$ form a single subdomain of $\mathbf{U}$.

That is a fairly sophisticated way of reaching the conclusion (and many details have to be worked out). A more elementary approach would be just to ask what it means to give a finite amount of information about a domain. For the sake of uniformity, suppose that the data objects of the possible domain are drawn from the non-negative integers, and that we conventionally use 0 for Δ. Then to give a finite amount of information about a domain is—roughly—to specify a finite part of Con and a finite part of $\vdash$. To make the formulation easier, we will reserve for 1 a rôle like the one recently played by ∇. What the specifications will boil down to is pairs (u, v) of finite sets of integers used as data objects to convey one piece of information about an entailment relation.

But hold, entailment relations are very closely connected to approximable mappings. Indeed, we remarked before that the identity function as an approximable mapping on a domain is just represented as the underlying entailment relation itself. Suppose we take as our domain the domain of all sets of integers. It is a powerset, so call it $\mathbf{P}$. That is to say, the integers are the data objects, all finite sets are consistent, and the entailment relation is the minimal possible one. (As far as elements go, an arbitrary set of integers is equal to the union of all its finite subsets, which means that the elements of the domain are in a one-one correspondence with the arbitrary sets of integers.) The question is: which approximable mappings on $\mathbf{P}$ into itself correspond to entailment relations on the integers as data objects?

The answer can be expressed most succinctly using our standard notation. If we think of $r : \mathbf{P} \to \mathbf{P}$ as a relation between finite sets in the usual way, then to say that r is reflexive is to say:

(1) $I_{\mathbf{P}} \subseteq r$.

To say that r is transitive is to say:

(2) $r \circ r = r$.

To say that for r the object 0 plays at being Δ is to say:

$$(3) \quad \bar{0} \subseteq r(\bot),$$

where in general $\bar{n}$ is short for $\{n\}$ in the domain $\mathbf{P}$. Then, to say that 1 plays at being a rogue object is to say:

$$(4) \quad \top = r(\bar{1}).$$

Finally, to say that 1 is an inconsistent object that has to be excluded is to say:

$$(5) \quad \bar{1} \not\subseteq r(\bar{0}).$$

That's it. The collection of approximable mappings satisfying (1)–(5) gives us all the entailment relations we need. Condition (5) is a consistency condition, and for r the consistent finite sets u are those such that $\bar{1} \not\subseteq r(\bar{u})$. What we are asserting is that the totality of r satisfying (1)–(5) forms the elements of a domain—one that has been derived from $(\mathbf{P} \to \mathbf{P})$ in a way similar to the way we derived $\mathbf{U}$ from $\mathbf{V}$ above.

Having made domains into a domain, the next step is to see how constructs on domain (i.e. functors) can be made into approximable mappings. But the retelling and development of that story will have to wait for another publication along with the very interesting chapter on powerdomains. I only hope the ground covered here makes the theory of domains seem more elementary and more natural. ?

Theorem (`example_8_4`). The identity map on the powerset of the integers is not a rogue approximation.

Proof. The identity is reflexive, so it fails the rogue clause. ?

`example_8_4.lean`

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.DomainApprox

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

open DomainApprox
open InfoSys ApproximableMap

theorem example_8_4 : ¬ RogueApprox (idMap P) :=
  not_RogueApprox_idMap

end DomainTheory.InformationSystems.InfoSys
```

6 Bridge Theorems

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

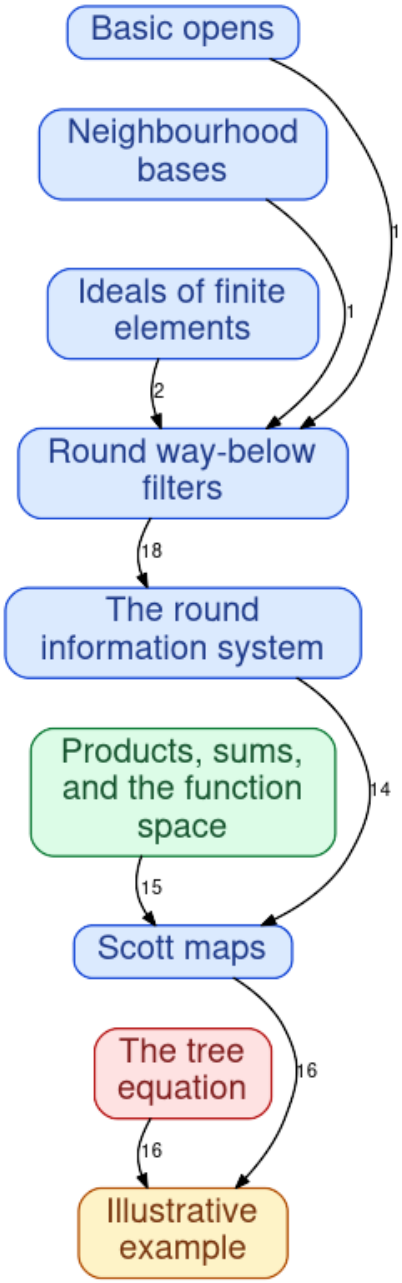


Figure 63: Diagram 28

6.1 Introduction

Neighbourhood systems and information systems are equivalent in a precise sense. The mathematical folklore is stronger still: all three presentations carve out the same class of domains, related by ideal completion and the Scott topology. That equivalence is already known; this article is the write-up of a machine-checked proof of the named bridges, not a first presentation of the result. The three presentations were formalized in separate libraries; the maps between them are the content of this package.

We do **not** re-prove Scott's internal theorems; we import the finished sibling packages and build cross-presentation maps:

Presentation	Lean package	Characteristic object
Continuous lattices [1]	`scott1972`	`ContinuousLattice D`, way-below $\ll$, `ScottMap`
Neighbourhood systems [2]	`scott1980`	`NeighborhoodSystem`, filters as domain elements
Information systems [3]	`scott1982`	`InfoSys`, elements as consistent closed token sets

The main subtlety is that a naïve reading of "domains = filters of neighbourhoods" overstates the continuous-lattice corner: for a continuous lattice `D`, the filter `{⇑a | a ≤ x}` has retract `x` but properly contains the principal round filter `{⇑a | a ≪ x}`. The correct identification is **`D ≃o RoundFilter`**, not raw $|\mathcal{D}|$.

<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

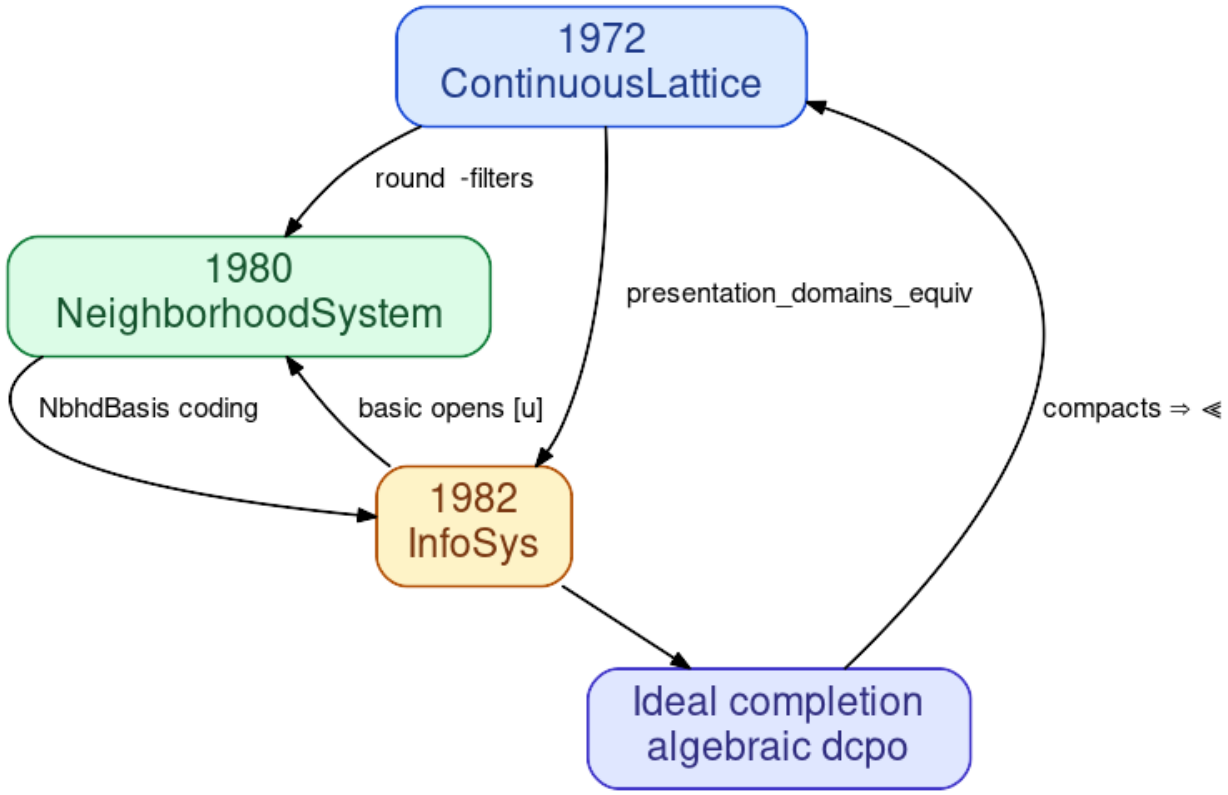


Figure 64: Diagram 29

The bridges are order isomorphisms of domains, and of the named construction objects, in `DomainTheory.BridgeTh` They are not a 2-categorical equivalence of the full categories of continuous lattices, neighbourhood systems, and information systems with all morphisms.

The three presentation-to-presentation bridges (year labels = source → target of the packaged order isomorphism):

Label	Years	Statement	Declaration	Note
`bridge_1972_1980`	1972 → 1980	A continuous lattice `D` is order-isomorphic to its round way-below filters.	`continuousLattice_roundFilter_iso` (= `ContinuousLattice.roundFilterEquiv`)	Round way-below filters / Packaging the round-filter isomorphism
`bridge_1980_1982`	1980 → 1982	Filters of a neighbourhood basis are order-isomorphic to elements of the induced information system.	`NbhdBasis.domainEquivInfo` (system via `neighborhoodSystem_to_infoSys`)	Neighbourhood bases / Neighbourhood filters and information-system elements

Label	Years	Statement	Declaration	Note
bridge_1982_1972	1982 → 1972	Elements of an information system are order-isomorphic to ideals of its finite elements (algebraic / 1972 carrier).	infoSys_to_idealCompletion	Ideals of finite elements / The ideal completion of an information system

Related companions (same chapter, not the three year-pair labels above):

Bridge	Statement	Declaration
1982 → 1980 (basic opens)	An information system induces the neighbourhood system of its basic opens; elements ≃o filters of that system.	infoSys_to_neighborhoodSystem, InfoSysToNeighborhood.domainOrderI
Neighbourhood system of way-below opens	The neighbourhood system associated with a continuous lattice.	continuousLattice_to_neighborhoodSystem
Algebraic lattices	An algebraic complete lattice is a continuous lattice.	idealCompletion_to_continuousLattice
Round three-way identification	For a continuous lattice with decidable equality, D ≃o RoundInfoSysElement, through D ≃o RoundFilter ≃o RoundInfoSysElement.	presentation_domains_equiv

Products, sums, the function space, and Scott maps are collected under infoSys_constructions_equiv:

Construction	Statement	Declaration
Product	|A| × |B| ≃o |A × B|	infoSys_product_domain_equiv
Sum	WithBot (|A| ⊕ |B|) ≃o |A + B|	infoSys_sum_domain_equiv
Function space	Approximable maps A → B are order-isomorphic to the elements of the function-space system.	infoSys_function_space_domain_equiv

Construction	Statement	Declaration
Scott continuity	Approximable maps `A → B` are order-isomorphic to the Scott-continuous maps on elements.	`approximableMap_scottContinuous_equiv`
Scott maps, round filters	Scott maps of continuous lattices are order-isomorphic to their conjugates along `D ≃o RoundFilter`.	`scottMap_roundFilter_iso`
Scott maps, round information systems	The same, along `presentation_domains_equiv`.	`scottMap_roundInfoSys_iso`

On Scott's tree domain over the lower-bound system of the natural numbers, the same carrier is an information system, a neighbourhood-filter domain, and an ideal completion, and it satisfies the equation `T ≅ A + (T × T)` as an order isomorphism of elements:

Instance	Statement	Declaration
Neighbourhood filters	`SexSys.Element ≃o` the filters of its basic-open neighbourhood system	`sexNeighborhoodIso`
Ideals	`SexSys.Element ≃o` the ideals of its finite elements	`sexIdealIso`
Domain equation	`SexSys.Element ≃o SexRhs.Element`	`sexDomainEquationIso`

A full functorial equivalence, preserving products, exponentials, and inverse limits simultaneously across all three presentations, would require additional coherence theorems.

Where Scott's 1982 text is constructive, the bridges stay constructive. Classical steps are named. The continuous-lattice identification `D ≃o RoundFilter` uses only propositional extensionality and quotient soundness: roundness is a condition on the order. An algebraic lattice is a continuous lattice by the 1972 definition of way-below, which is classical. Separated sums use classical choice for the polarity of a token (left, right, or bottom). Conjugation of Scott maps inherits that choice from the 1972 development of `ScottMap`.

Every other information-system bridge reports `Classical.choice` as well, but only because Mathlib's `Finset` instances depend on it. A declaration whose type mentions an information system therefore prints choice even when the proof manipulates an explicit finite set and extracts no witness. That is the footprint of neighbourhood bases in both directions, of ideals of finite elements, of products and function spaces, and of approximable maps against Scott-continuous maps. The separated-sum trichotomy is a use of choice; those others are the instance frontier.

6.2 Basic opens

<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

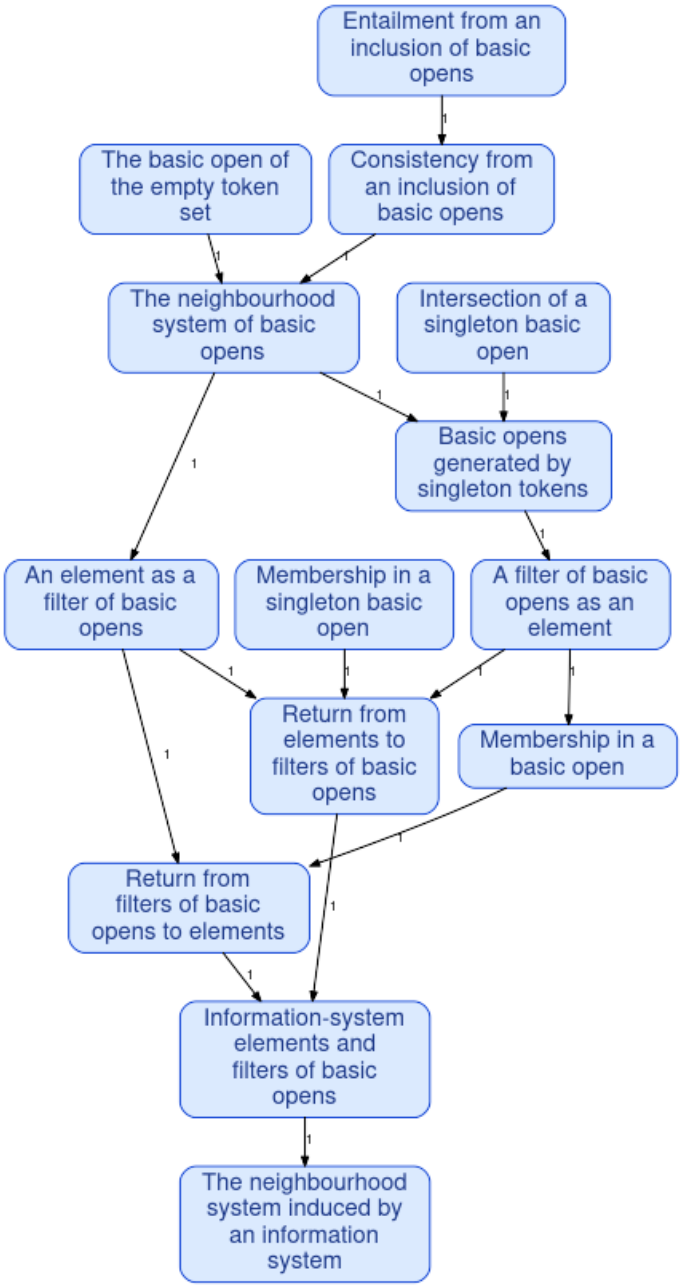


Figure 65: Diagram 30

6.2.1 The basic open of the empty token set

Theorem (`basicOpen_empty`). In an information system, the basic open determined by the empty token set is the whole set of elements.

Proof. Membership in that basic open asks that every token of the empty set lie in the carrier, which is vacuously true for every element. ■

`basicOpen_empty.lean`

```
module
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    basicOpen

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem basicOpen_empty : A.basicOpen (∅ : Finset α) = (Set.univ : Set A.Element) := by
  ext x
  constructor
  · intro; trivial
  · intro _ a ha
    exact False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha))

end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.2 Intersection of a singleton basic open

Theorem (`basicOpen_singleton_inter`). The intersection of the basic open of a singleton token with the basic open of a finite set equals the basic open of the insertion of that token into the set.

Proof. Intersection of basic opens is the basic open of the constructive union, and adjoining a singleton is insertion. ■

`basicOpen_singleton_inter.lean`

```
module
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    basicOpen
public import DomainTheory.InformationSystems.InformationSystems.Constructive.funion

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem basicOpen_singleton_inter (a : α) (s : Finset α) :
```

```
    A.basicOpen ({a} : Finset α) ∩ A.basicOpen s = A.basicOpen (insert a s) := by
  rw [basicOpen.inter A, funion.singleton_eq_insert]

end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.3 Membership in a singleton basic open

Theorem (`mem_basicOpen_singleton`). An element belongs to the basic open of a singleton token if and only if that token lies in its carrier.

Proof. The basic-open condition for a singleton reduces to membership of that unique token. ■

`mem_basicOpen_singleton.lean`

```
module
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    basicOpen

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem mem_basicOpen_singleton {a : α} {x : A.Element} :
    x ∈ A.basicOpen ({a} : Finset α) ↔ a ∈ x.carrier := by
  constructor
  · intro h
    exact h (Finset.mem_coe.2 (Finset.mem_singleton_self a))
  · intro ha b hb
    have hb' : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    exact hb' ▸ ha

end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.4 Entailment from an inclusion of basic opens

Theorem (`ent_of_basicOpen_subset`). If a consistent finite set w has its basic open contained in the basic open of u, then every token of u is entailed by w.

Proof. The closure of w lies in the basic open of w, hence in that of u, so every token of u belongs to the closure and is therefore entailed by w. ■

`ent_of_basicOpen_subset.lean`

```
module
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    basicOpen
public import DomainTheory.InformationSystems.ElementsOfASystem.closure

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem ent_of_basicOpen_subset {u w : Finset α} (hw : w ∈ A.Con)
    (hsub : A.basicOpen w ⊆ A.basicOpen u) {a : α} (ha : a ∈ u) : A.Ent w a := by
  have : A.closure w hw ∈ A.basicOpen w := closure.subset A hw
  have hU : A.closure w hw ∈ A.basicOpen u := hsub this
  exact hU (Finset.mem_coe.2 ha)

end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.5 Consistency from an inclusion of basic opens

Theorem (`con_of_basicOpen_subset`). If a consistent finite set w has its basic open contained in the basic open of u, then u is consistent.

Proof. Every token of u is entailed by w, so the constructive union of w with u is consistent, and consistency passes to subsets. ■

`con_of_basicOpen_subset.lean`

```
module
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    basicOpen
public import DomainTheory.BridgeTheorems.ent_of_basicOpen_subset
public import DomainTheory.InformationSystems.InformationSystems.EntSet
public import DomainTheory.InformationSystems.InformationSystems.Constructive.funion

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysToNeighborhood
```

```
variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem con_of_basicOpen_subset {u w : Finset α} (hw : w ∈ A.Con)
    (hsub : A.basicOpen w ⊆ A.basicOpen u) : u ∈ A.Con := by
  have hEnt : A.EntSet w u := fun a ha => ent_of_basicOpen_subset A hw hsub ha
  exact A.con_subset (EntSet.union_mem A hw hEnt) (funion.subset_right _ _)

end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.6 The neighbourhood system of basic opens

Definition (`toNeighborhoodSystem`). The neighbourhood system associated with an information system has as neighbourhoods precisely the basic opens of consistent finite token sets, with the whole set of elements as master neighbourhood.

The empty set is consistent and determines the master neighbourhood. The intersection of two basic opens is the basic open of their constructive union, which remains consistent whenever a third basic open sits below both.

`toNeighborhoodSystem.lean`

```
module
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    basicOpen
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
public import DomainTheory.BridgeTheorems.basicOpen_empty
public import DomainTheory.BridgeTheorems.con_of_basicOpen_subset
import DomainTheory.InformationSystems.ElementsOfASystem.closure
import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.NeighbourhoodSystems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def toNeighborhoodSystem : NeighborhoodSystem A.Element where
  mem X := ∃ u, u ∈ A.Con ∧ X = A.basicOpen u
  master := Set.univ
  master_nonempty := ⟨A.closure ∅ A.con_empty, Set.mem_univ _⟩
  master_mem := ⟨∅, A.con_empty, (basicOpen_empty A).symm⟩
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    obtain ⟨u, hu, rfl⟩ := hX
```

```
    obtain ⟨v, hv, rfl⟩ := hY
    obtain ⟨w, hw, rfl⟩ := hZ
    have hsub : A.basicOpen w ⊆ A.basicOpen (u ∪' v) := by
      intro x hx
      have hx' : x ∈ A.basicOpen u ∩ A.basicOpen v := hZsub hx
      rwa [basicOpen.inter A] at hx'
    have huv : u ∪' v ∈ A.Con := con_of_basicOpen_subset A hw hsub
    exact ⟨u ∪' v, huv, basicOpen.inter A u v⟩
  sub_master := fun {_} _ => Set.subset_univ _

end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.7 Basic opens generated by singleton tokens

Theorem (`mem_basicOpen_of_singletons`). A filter of the basic-open neighbourhood system that contains every singleton basic open for tokens of a finite set also contains the basic open of that set.

Proof. By induction on the finite set: the empty case is the master neighbourhood, and the inductive step uses closure of filters under intersection together with the singleton-intersection identity for basic opens. ■

`mem_basicOpen_of_singletons.lean`

```
module

public import DomainTheory.BridgeTheorems.toNeighborhoodSystem
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element
public import DomainTheory.BridgeTheorems.basicOpen_empty
public import DomainTheory.BridgeTheorems.basicOpen_singleton_inter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem mem_basicOpen_of_singletons (f : (toNeighborhoodSystem A).Element) {Y : Finset
    α}
    (hY : ∀ a ∈ Y, f.mem (A.basicOpen ({a} : Finset α))) :
    f.mem (A.basicOpen Y) := by
  induction Y using Finset.induction with
  | empty =>
    rw [basicOpen_empty]
    have hmaster : (toNeighborhoodSystem A).master = (Set.univ : Set A.Element) := rfl
    rw [← hmaster]
    exact f.master_mem
  | insert a s _ha ih =>
    have hA := hY a (Finset.mem_insert_self a s)
    have hSm := ih fun i hi => hY i (Finset.mem_insert_of_mem hi)
    simpa [basicOpen_singleton_inter A a s] using f.inter_mem hA hSm
```

```
end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.8 An element as a filter of basic opens

Definition (`toFilter`). From an element of an information system one obtains a filter of the basic-open neighbourhood system by taking those basic opens of consistent finite sets whose tokens all lie in the element's carrier.

Subset neighbourhoods, the master neighbourhood, intersections, and upward closure follow from consistency and deductive closure of the element together with the corresponding facts for basic opens.

`toFilter.lean`

```
module

public import DomainTheory.BridgeTheorems.toNeighborhoodSystem
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element
public import DomainTheory.BridgeTheorems.basicOpen_empty
public import DomainTheory.BridgeTheorems.ent_of_basicOpen_subset
import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def toFilter (x : A.Element) : (toNeighborhoodSystem A).Element where
  mem U := ∃ u, u ∈ A.Con ∧ U = A.basicOpen u ∧ ↑u ⊆ x.carrier
  sub := by
    intro U h
    obtain ⟨u, hu, rfl, _⟩ := h
    exact ⟨u, hu, rfl⟩
  master_mem := ⟨∅, A.con_empty, (basicOpen_empty A).symm, by
    intro a ha
    exact False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha))⟩
  inter_mem := by
    intro U V hU hV
    obtain ⟨u, hu, rfl, huX⟩ := hU
    obtain ⟨v, hv, rfl, hvX⟩ := hV
    refine ⟨u ∪' v, ?_, basicOpen.inter A u v, ?_⟩
    · have hsub : ↑(u ∪' v) ⊆ x.carrier := by
        intro a ha
        rcases funion.mem.mp (Finset.mem_coe.1 ha) with h | h
```

```
        · exact huX (Finset.mem_coe.2 h)
        · exact hvX (Finset.mem_coe.2 h)
      exact x.consistent (u ∪' v) hsub
    · intro a ha
      rcases funion.mem.mp (Finset.mem_coe.1 ha) with h | h
      · exact huX (Finset.mem_coe.2 h)
      · exact hvX (Finset.mem_coe.2 h)
  up_mem := by
    intro U V hU hV hUV
    obtain ⟨u, hu, rfl, huX⟩ := hU
    obtain ⟨v, hv, rfl⟩ := hV
    refine ⟨v, hv, rfl, ?_⟩
    intro a ha
    exact x.closed u a huX (ent_of_basicOpen_subset A hu hUV ha)

end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.9 A filter of basic opens as an element

Definition (`ofFilter`). From a filter of the basic-open neighbourhood system one recovers an element whose carrier consists of those tokens whose singleton basic opens belong to the filter.

Consistency of finite subsets of the carrier follows from membership of the corresponding basic open and the criterion that inclusions of basic opens preserve consistency; deductive closure uses upward closure of the filter under inclusions of basic opens.

`ofFilter.lean`

```
module

public import DomainTheory.BridgeTheorems.toNeighborhoodSystem
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element
public import DomainTheory.BridgeTheorems.con_of_basicOpen_subset
public import DomainTheory.BridgeTheorems.mem_basicOpen_of_singletons

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def ofFilter (f : (toNeighborhoodSystem A).Element) : A.Element where
  carrier := {a | f.mem (A.basicOpen ({a} : Finset α))}
  consistent := by
    intro Y hY
    have hmem : f.mem (A.basicOpen Y) :=
      mem_basicOpen_of_singletons A f fun a ha => hY (Finset.mem_coe.2 ha)
    obtain ⟨u, hu, heq⟩ := f.sub hmem
    have hsub : A.basicOpen u ⊆ A.basicOpen Y := by
```

```
      intro x hx; exact heq ▸ hx
    exact con_of_basicOpen_subset A hu hsub
  closed := by
    intro Y b hY hEnt
    have hmemY : f.mem (A.basicOpen Y) :=
      mem_basicOpen_of_singletons A f fun c hc => hY (Finset.mem_coe.2 hc)
    have hsub : A.basicOpen Y ⊆ A.basicOpen ({b} : Finset α) := by
      intro x hx c hc
      have eqcb : c = b := Finset.mem_singleton.mp (Finset.mem_coe.1 hc)
      rw [eqcb]
      exact x.closed Y b hx hEnt
    exact f.up_mem hmemY ⟨({b} : Finset α), A.con_sing b, rfl⟩ hsub

end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.10 Membership in a basic open

Theorem (`mem_basicOpen_iff`). A filter of the basic-open neighbourhood system contains the basic open of a consistent finite set if and only if every token of that set lies in the carrier of the element recovered from the filter.

Proof. One direction uses upward closure under inclusions of singleton basic opens; the other assembles singleton memberships into membership of the joint basic open. ■

`mem_basicOpen_iff.lean`

```
module

public import DomainTheory.BridgeTheorems.ofFilter
public import DomainTheory.BridgeTheorems.mem_basicOpen_of_singletons

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem mem_basicOpen_iff (f : (toNeighborhoodSystem A).Element) {u : Finset α}
    (_hu : u ∈ A.Con) :
    f.mem (A.basicOpen u) ↔ ↑u ⊆ (ofFilter A f).carrier := by
  constructor
  · intro h a ha
    have hsub : A.basicOpen u ⊆ A.basicOpen ({a} : Finset α) := by
      intro x hx c hc
      have hc' : c = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hc)
      subst hc'
      exact hx (Finset.mem_coe.2 ha)
    exact f.up_mem h ⟨({a} : Finset α), A.con_sing a, rfl⟩ hsub
  · intro hsub
```

```
    exact mem_basicOpen_of_singletons A f fun a ha => hsub (Finset.mem_coe.2 ha)

end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.11 Return from filters of basic opens to elements

Theorem (`toFilter_ofFilter`). Passing from a filter of basic opens to an element and back yields the original filter.

Proof. Membership of a basic open in the reconstituted filter is equivalent, by the membership criterion, to containment of its tokens in the recovered carrier, which is exactly membership in the original filter. ■

`toFilter_ofFilter.lean`

```
module

public import DomainTheory.BridgeTheorems.toFilter
public import DomainTheory.BridgeTheorems.ofFilter
public import DomainTheory.BridgeTheorems.mem_basicOpen_iff
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ElementOrder

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.NeighbourhoodSystems

open DomainTheory.InformationSystems

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem toFilter_ofFilter (f : (toNeighborhoodSystem A).Element) :
    toFilter A (ofFilter A f) = f := by
  refine NeighborhoodSystem.Element.ext (V := toNeighborhoodSystem A) fun U => ?_
  constructor
  · intro h
    obtain ⟨u, hu, hU, hsub⟩ := h
    subst hU
    exact (mem_basicOpen_iff A f hu).mpr hsub
  · intro hU
    obtain ⟨u, hu, heq⟩ := f.sub hU
    subst heq
    exact ⟨u, hu, rfl, (mem_basicOpen_iff A f hu).mp hU⟩

end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.12 Return from elements to filters of basic opens

Theorem (`ofFilter_toFilter`). Passing from an element to a filter of basic opens and back yields the original element.

Proof. A token belongs to the reconstituted carrier exactly when its singleton basic open meets the filter associated with the element, which holds precisely when the token lies in the original carrier, by membership in singleton basic opens and deductive closure. ■

`ofFilter_toFilter.lean`

```
module

public import DomainTheory.BridgeTheorems.toFilter
public import DomainTheory.BridgeTheorems.ofFilter
public import DomainTheory.BridgeTheorems.mem_basicOpen_singleton
import DomainTheory.InformationSystems.ElementsOfASystem.closure

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem ofFilter_toFilter (x : A.Element) : ofFilter A (toFilter A x) = x := by
  have hc : (ofFilter A (toFilter A x)).carrier = x.carrier := by
    ext a
    constructor
    · intro ha
      obtain ⟨u, hu, heq, hsub⟩ := ha
      have huA : A.closure u hu ∈ A.basicOpen u := closure.subset A hu
      have hsing : A.closure u hu ∈ A.basicOpen ({a} : Finset α) := by
        rw [heq]; exact huA
      have ha' : a ∈ (A.closure u hu).carrier := (mem_basicOpen_singleton A).1 hsing
      exact x.closed u a hsub ha'
    · intro ha
      refine ⟨({a} : Finset α), A.con_sing a, rfl, ?_⟩
      intro c hc
      have hc' : c = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hc)
      exact hc' ▸ ha
  rcases x with ⟨c, cons, clo⟩
  rcases hte : ofFilter A (toFilter A ⟨c, cons, clo⟩) with ⟨c', cons', clo'⟩
  have hc' : c' = c := by simpa [hte] using hc
  subst hc'
  rfl

end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.13 Information-system elements and filters of basic opens

Definition (`domainOrderIso`). The maps sending an information-system element to its filter of basic opens and a filter back to an element form an order isomorphism between the elements of the information system and the filters of its basic-open neighbourhood system.

The two composites are the identity by the round-trip theorems; order is preserved because inclusion of carriers is equivalent to inclusion of the associated filters of basic opens.

domainOrderIso.lean

```
module

public import DomainTheory.BridgeTheorems.toFilter_ofFilter
public import DomainTheory.BridgeTheorems.ofFilter_toFilter
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import DomainTheory.BridgeTheorems.mem_basicOpen_singleton
import DomainTheory.InformationSystems.ElementsOfASystem.closure

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysToNeighborhood

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def domainOrderIso : A.Element ≃o (toNeighborhoodSystem A).Element where
  toFun := toFilter A
  invFun := ofFilter A
  left_inv := ofFilter_toFilter A
  right_inv := toFilter_ofFilter A
  map_rel_iff' := by
    intro x y
    constructor
    · intro h a ha
      have hx : (toFilter A x).mem (A.basicOpen ({a} : Finset α)) :=
        ⟨({a} : Finset α), A.con_sing a, rfl, by
          intro c hc
          have hc' : c = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hc)
          exact hc' ▸ ha⟩
      have hy : (toFilter A y).mem (A.basicOpen ({a} : Finset α)) := h _ hx
      obtain ⟨u, hu, heq, hsub⟩ := hy
      have huA : A.closure u hu ∈ A.basicOpen u := closure.subset A hu
      have hsing : A.closure u hu ∈ A.basicOpen ({a} : Finset α) := by
        rw [heq]; exact huA
      have ha' : a ∈ (A.closure u hu).carrier := (mem_basicOpen_singleton A).1 hsing
      exact y.closed u a hsub ha'
    · intro h U hU
      obtain ⟨u, hu, hUeq, huX⟩ := hU
      subst hUeq
      exact ⟨u, hu, rfl, fun a ha => h (huX ha)⟩
```

```
end InfoSysToNeighborhood

end DomainTheory.BridgeTheorems
```

6.2.14 The neighbourhood system induced by an information system

Definition (`infoSys_to_neighborhoodSystem`). The neighbourhood system induced by an information system is the basic-open neighbourhood system constructed above.

`InfoSysToNeighborhood.lean`

```
module

public import DomainTheory.BridgeTheorems.toNeighborhoodSystem
public import DomainTheory.BridgeTheorems.InfoSysToNeighborhood.domainOrderIso

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.NeighbourhoodSystems
open DomainTheory.InformationSystems

abbrev infoSys_to_neighborhoodSystem {α : Type*} [DecidableEq α] (A : InfoSys α) :
    NeighborhoodSystem A.Element :=
  InfoSysToNeighborhood.toNeighborhoodSystem A

end DomainTheory.BridgeTheorems
```

6.3 Neighbourhood bases

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

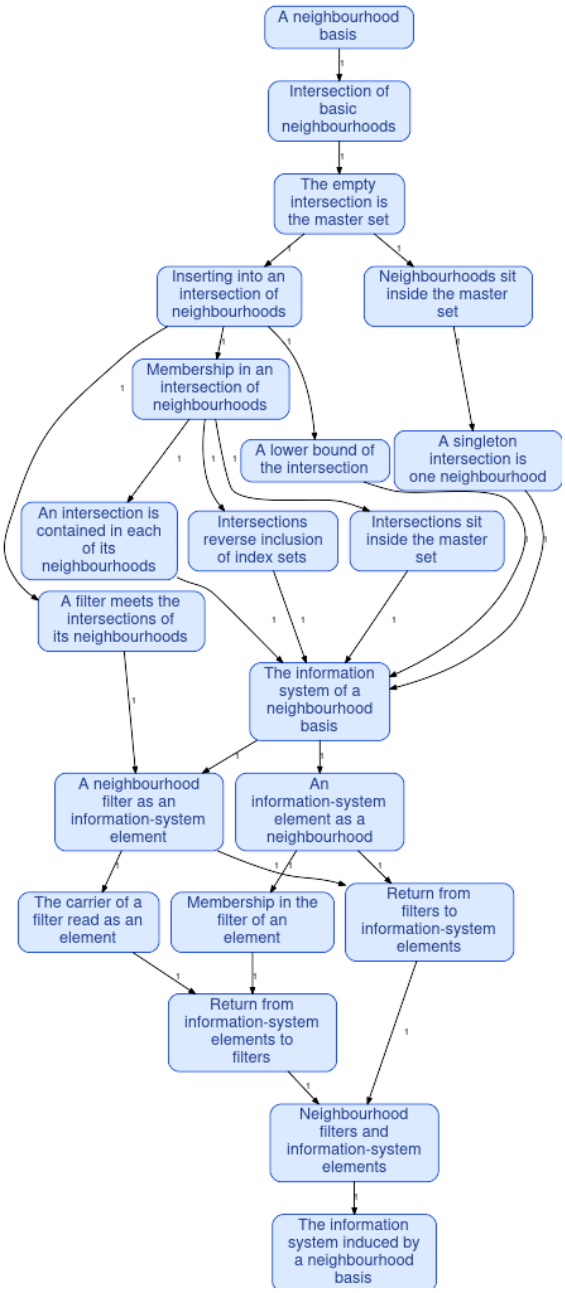


Figure 66: Diagram 31

6.3.1 A neighbourhood basis

Definition (`NbhdBasis`). A neighbourhood basis for a neighbourhood system indexes every neighbourhood by a token from an index type, with a distinguished bottom index whose neighbourhood is the master set.

Every basic neighbourhood is a neighbourhood of the system, and every neighbourhood arises as some basic neighbourhood.

NbhdBasis.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.NeighbourhoodSystems

structure NbhdBasis (ι α : Type*) [DecidableEq ι] where
  system : NeighborhoodSystem α
  nbhd : ι → Set α
  nbhd_mem : ∀ i, system.mem (nbhd i)
  exhaustive : ∀ {X : Set α}, system.mem X → ∃ i, nbhd i = X
  botIdx : ι
  botIdx_eq : nbhd botIdx = system.master

end DomainTheory.BridgeTheorems
```

6.3.2 Intersection of basic neighbourhoods

Definition (`interOf`). The intersection of the basic neighbourhoods indexed by a finite set of tokens, taken inside the master set.

interOf.lean

```
module

public import Mathlib.Data.Finset.Fold
public import DomainTheory.BridgeTheorems.NbhdBasis

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

def interOf (u : Finset ι) : Set α :=
  u.fold (· ∩ ·) B.system.master B.nbhd

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.3 The empty intersection is the master set

Theorem (`interOf_empty`). The intersection over the empty index set is the master neighbourhood.

Proof. An empty intersection inside the master set is the master set itself. ■

interOf_empty.lean

```
module

public import DomainTheory.BridgeTheorems.interOf

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

@[simp] theorem interOf_empty : B.interOf (∅ : Finset ι) = B.system.master := rfl

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.4 Neighbourhoods sit inside the master set

Theorem (`nbhd_subset_master`). Every basic neighbourhood is contained in the master set.

Proof. Neighbourhoods of a neighbourhood system sit under the master set. ■

nbhd_subset_master.lean

```
module

public import DomainTheory.BridgeTheorems.NbhdBasis

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem nbhd_subset_master (i : ι) : B.nbhd i ⊆ B.system.master :=
  B.system.sub_master (B.nbhd_mem i)

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.5 Inserting into an intersection of neighbourhoods

Theorem (`interOf_insert`). Inserting a token into a finite index set intersects the corresponding basic neighbourhood with the previous intersection.

Proof. This is the usual decomposition of a finite intersection. ■

interOf_insert.lean

```
module

public import DomainTheory.BridgeTheorems.interOf

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem interOf_insert {a : ι} {s : Finset ι} (ha : a ∈ s) :
    B.interOf (insert a s) = B.nbhd a ∩ B.interOf s :=
  Finset.fold_insert (op := (· ∩ ·)) ha

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.6 A singleton intersection is one neighbourhood

Theorem (`interOf_singleton`). The intersection over a singleton index is that basic neighbourhood.

Proof. Immediate from the definition of the finite intersection. ■

`interOf_singleton.lean`

```
module

public import DomainTheory.BridgeTheorems.interOf
import DomainTheory.BridgeTheorems.nbhd_subset_master

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem interOf_singleton (a : ι) : B.interOf ({a} : Finset ι) = B.nbhd a := by
  simp [interOf, Finset.fold_singleton, Set.inter_eq_left.mpr (B.nbhd_subset_master a)
    ]

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.7 A lower bound of the intersection

Theorem (`interOf_mem_of_lower_bound`). If a neighbourhood is contained in the intersection of the basic neighbourhoods of a finite index set, then that intersection is itself a neighbourhood.

Proof. Neighbourhood systems are upward closed: a set containing a neighbourhood is a neighbourhood. ■

`interOf_mem_of_lower_bound.lean`

```
module

public import DomainTheory.BridgeTheorems.interOf
import DomainTheory.BridgeTheorems.interOf_empty
import DomainTheory.BridgeTheorems.interOf_insert

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem interOf_mem_of_lower_bound {u : Finset ι} {Z : Set α}
    (hZ : B.system.mem Z) (hsub : Z ⊆ B.interOf u) : B.system.mem (B.interOf u) := by
  induction u using Finset.induction with
  | empty =>
    simpa [interOf_empty] using B.system.master_mem
  | insert a s ha ih =>
    rw [B.interOf_insert ha] at hsub ⊢
    have hs : B.system.mem (B.interOf s) := ih (hsub.trans Set.inter_subset_right)
    exact B.system.inter_mem (B.nbhd_mem a) hs hZ hsub

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.8 Membership in an intersection of neighbourhoods

Theorem (`mem_interOf`). A point lies in the intersection of the basic neighbourhoods of a finite index set if and only if it lies in the master set and in each of those basic neighbourhoods.

Proof. Unfold the definition of the finite intersection. ■

`mem_interOf.lean`

```
module

public import DomainTheory.BridgeTheorems.interOf
import DomainTheory.BridgeTheorems.interOf_empty
import DomainTheory.BridgeTheorems.interOf_insert

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)
```

```
theorem mem_interOf {u : Finset ι} {x : α} :
    x ∈ B.interOf u ↔ x ∈ B.system.master ∧ ∀ i ∈ u, x ∈ B.nbhd i := by
  induction u using Finset.induction with
  | empty =>
    simp [interOf_empty]
  | insert a s ha ih =>
    rw [B.interOf_insert ha, Set.mem_inter_iff, ih]
    constructor
    · rintro ⟨ha', hm, hs⟩
      refine ⟨hm, fun i hi => ?_⟩
      rcases Finset.mem_insert.mp hi with rfl | hi
      · exact ha'
      · exact hs i hi
    · rintro ⟨hm, hall⟩
      exact ⟨hall a (Finset.mem_insert_self a s), hm,
        fun i hi => hall i (Finset.mem_insert_of_mem hi)⟩

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.9 Intersections reverse inclusion of index sets

Theorem (`interOf_anti`). Enlarging the finite index set shrinks the intersection of basic neighbourhoods.

Proof. Intersecting more sets yields a smaller result. ■

`interOf_anti.lean`

```
module

public import DomainTheory.BridgeTheorems.interOf
import DomainTheory.BridgeTheorems.mem_interOf

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem interOf_anti {u v : Finset ι} (h : u ⊆ v) : B.interOf v ⊆ B.interOf u := by
  intro x hx
  exact B.mem_interOf.mpr ⟨(B.mem_interOf.mp hx).1, fun i hi => (B.mem_interOf.mp hx).2
     i (h hi)⟩

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.10 Intersections sit inside the master set

Theorem (`interOf_subset_master`). Every finite intersection of basic neighbourhoods sits inside the master set.

Proof. The intersection is formed inside the master set by definition. ■

`interOf_subset_master.lean`

```
module

public import DomainTheory.BridgeTheorems.interOf
import DomainTheory.BridgeTheorems.mem_interOf

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem interOf_subset_master (u : Finset ι) : B.interOf u ⊆ B.system.master :=
  fun _ hx => (B.mem_interOf.mp hx).1

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.11 An intersection is contained in each of its neighbourhoods

Theorem (`interOf_subset_nbhd`). The intersection of the basic neighbourhoods of a finite index set is contained in each basic neighbourhood whose index belongs to the set.

Proof. A finite intersection is contained in each of its factors. ■

`interOf_subset_nbhd.lean`

```
module

public import DomainTheory.BridgeTheorems.interOf
import DomainTheory.BridgeTheorems.mem_interOf

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem interOf_subset_nbhd {u : Finset ι} {i : ι} (hi : i ∈ u) :
    B.interOf u ⊆ B.nbhd i := fun _ hx => (B.mem_interOf.mp hx).2 i hi

end NbhdBasis
```

```
end DomainTheory.BridgeTheorems
```

6.3.12 A filter meets the intersections of its neighbourhoods

Theorem (`filter_mem_interOf`). A neighbourhood filter that contains every basic neighbourhood of a finite index set also contains their intersection.

Proof. By induction on the finite set, using that filters contain the master set and are closed under binary intersections. ■

`filter_mem_interOf.lean`

```
module

public import DomainTheory.BridgeTheorems.interOf
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element
import DomainTheory.BridgeTheorems.interOf_empty
import DomainTheory.BridgeTheorems.interOf_insert

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem filter_mem_interOf (x : B.system.Element) {u : Finset ι}
    (hu : ∀ i ∈ u, x.mem (B.nbhd i)) : x.mem (B.interOf u) := by
  induction u using Finset.induction with
  | empty =>
    simpa [interOf_empty] using x.master_mem
  | insert a s ha ih =>
    rw [B.interOf_insert ha]
    exact x.inter_mem (hu a (Finset.mem_insert_self a s))
      (ih fun i hi => hu i (Finset.mem_insert_of_mem hi))

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.13 The information system of a neighbourhood basis

Definition (`toInfoSys`). The information system of a neighbourhood basis has the basis indices as tokens; a finite set of indices is consistent when the corresponding intersection of basic neighbourhoods is a neighbourhood, and entailment holds when that intersection is contained in the entailed basic neighbourhood.

The bottom token is the distinguished bottom index of the basis. The information-system axioms follow from the neighbourhood-system axioms and the lattice of finite intersections.

`toInfoSys.lean`

```
module

public import DomainTheory.BridgeTheorems.interOf
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.InfoSys
import DomainTheory.BridgeTheorems.interOf_anti
import DomainTheory.BridgeTheorems.interOf_mem_of_lower_bound
import DomainTheory.BridgeTheorems.interOf_singleton
import DomainTheory.BridgeTheorems.interOf_subset_master
import DomainTheory.BridgeTheorems.interOf_subset_nbhd
import DomainTheory.BridgeTheorems.mem_interOf

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

def toInfoSys : InfoSys ι where
  bot := B.botIdx
  Con := {u | B.system.mem (B.interOf u)}
  Ent := fun u a => B.system.mem (B.interOf u) ∧ B.interOf u ⊆ B.nbhd a
  con_subset := by
    intro u v hu hv
    exact B.interOf_mem_of_lower_bound hu (B.interOf_anti hv)
  con_sing := by
    intro a
    simpa [Set.mem_ofPred_eq, B.interOf_singleton] using B.nbhd_mem a
  ent_con := by
    intro u a ⟨hu, hsub⟩
    have h₁ : B.interOf (insert a u) ⊆ B.interOf u :=
      B.interOf_anti (Finset.subset_insert a u)
    have h₂ : B.interOf u ⊆ B.interOf (insert a u) := by
      intro x hx
      refine B.mem_interOf.mpr ⟨(B.mem_interOf.mp hx).1, fun i hi => ?_⟩
      rcases Finset.mem_insert.mp hi with rfl | hi
      · exact hsub hx
      · exact (B.mem_interOf.mp hx).2 i hi
    have heq : B.interOf (insert a u) = B.interOf u := Set.Subset.antisymm h₁ h₂
    simpa [Set.mem_ofPred_eq, heq] using hu
  ent_bot := by
    intro u hu
    refine ⟨hu, ?_⟩
    intro x hx
    rw [B.botIdx_eq]
    exact B.interOf_subset_master u hx
  ent_refl := by
    intro u a hu ha
    exact ⟨hu, B.interOf_subset_nbhd ha⟩
  ent_trans := by
    intro u v c hv _hu hall ⟨_, hsub⟩
    refine ⟨hv, ?_⟩
    intro x hx
```

```
    have hxu : x ∈ B.interOf u :=
      B.mem_interOf.mpr ⟨B.interOf_subset_master v hx, fun i hi => (hall i hi).2 hx⟩
    exact hsub hxu

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.14 A neighbourhood filter as an information-system element

Definition (`toElement`). From a neighbourhood filter one obtains an element of the induced information system whose carrier consists of those indices whose basic neighbourhoods belong to the filter.

Consistency and deductive closure follow from the filter's closure under finite intersections and upward closure under inclusions of neighbourhoods.

`toElement.lean`

```
module

public import DomainTheory.BridgeTheorems.toInfoSys
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element
public import DomainTheory.BridgeTheorems.filter_mem_interOf

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

def toElement (x : B.system.Element) : B.toInfoSys.Element where
  carrier := {i | x.mem (B.nbhd i)}
  consistent := fun Y hY => x.sub (B.filter_mem_interOf x hY)
  closed := by
    intro Y a hY ⟨_, hsub⟩
    exact x.up_mem (B.filter_mem_interOf x hY) (B.nbhd_mem a) hsub

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.15 An information-system element as a neighbourhood filter

Definition (`ofElement`). From an element of the induced information system one obtains a neighbourhood filter consisting of those neighbourhoods that contain the basic neighbourhood of some token in the element's carrier.

The filter axioms follow from consistency and deductive closure of the element together with the corresponding properties of basic neighbourhoods.

`ofElement.lean`

```
module

public import DomainTheory.BridgeTheorems.toInfoSys
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element
import DomainTheory.BridgeTheorems.interOf_subset_nbhd
import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

def ofElement (e : B.toInfoSys.Element) : B.system.Element where
  mem X := B.system.mem X ∧ ∃ i ∈ e.carrier, B.nbhd i ⊆ X
  sub h := h.1
  master_mem := by
    refine ⟨B.system.master_mem, B.botIdx, ?_, by rw [B.botIdx_eq]⟩
    have hEnt : B.toInfoSys.Ent ∅ B.botIdx := B.toInfoSys.ent_bot (InfoSys.con_empty _)
    exact e.closed ∅ B.botIdx (fun _ h => False.elim (Finset.notMem_empty _ h)) hEnt
  inter_mem := by
    intro X Y ⟨hX, i, hi, hix⟩ ⟨hY, j, hj, hjy⟩
    have hCon : ({i, j} : Finset ι) ∈ B.toInfoSys.Con :=
      e.consistent {i, j} (by
        intro x hx
        rcases Finset.mem_insert.mp hx with rfl | hx
        · exact hi
        · have : x = j := Finset.mem_singleton.mp hx
          exact this ▸ hj)
    have hInterMem : B.system.mem (B.interOf ({i, j} : Finset ι)) := hCon
    have hInterSub : B.interOf ({i, j} : Finset ι) ⊆ X ∩ Y := by
      intro x hx
      refine ⟨hix (B.interOf_subset_nbhd (by simp) hx),
        hjy (B.interOf_subset_nbhd (by simp) hx)⟩
    refine ⟨B.system.inter_mem hX hY hInterMem hInterSub, ?_⟩
    obtain ⟨k, hk⟩ := B.exhaustive hInterMem
    refine ⟨k, ?_, ?_⟩
    · have hEnt : B.toInfoSys.Ent {i, j} k := ⟨hInterMem, fun x hx => hk ▸ hx⟩
      exact e.closed {i, j} k (by
        intro x hx
        rcases Finset.mem_insert.mp hx with rfl | hx
        · exact hi
        · exact Finset.mem_singleton.mp hx ▸ hj) hEnt
    · intro x hx
      exact hInterSub (hk ▸ hx)
  up_mem := by
    intro X Y ⟨hX, i, hi, hix⟩ hY hXY
    exact ⟨hY, i, hi, hix.trans hXY⟩

end NbhdBasis
```

```
end DomainTheory.BridgeTheorems
```

6.3.16 The carrier of a filter read as an element

Theorem (`toElement_carrier`). The carrier of the information-system element associated with a neighbourhood filter is exactly the set of indices whose basic neighbourhoods belong to the filter.

Proof. This is the defining carrier. ■

`toElement_carrier.lean`

```
module

public import DomainTheory.BridgeTheorems.toElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem toElement_carrier (x : B.system.Element) :
    (B.toElement x).carrier = {i | x.mem (B.nbhd i)} := rfl

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.17 Membership in the filter of an element

Theorem (`ofElement_mem`). Membership in the neighbourhood filter associated with an information-system element is equivalent to being a neighbourhood that contains the basic neighbourhood of some token of the element.

Proof. Unfold the definition of the associated filter. ■

`ofElement_mem.lean`

```
module

public import DomainTheory.BridgeTheorems.ofElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem ofElement_mem (e : B.toInfoSys.Element) (X : Set α) :
    (B.ofElement e).mem X ↔ B.system.mem X ∧ ∃ i ∈ e.carrier, B.nbhd i ⊆ X :=
```

```
  Iff.rfl

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.18 Return from information-system elements to filters

Theorem (`toElement_ofElement`). Passing from an information-system element to a neighbourhood filter and back yields the original element.

Proof. A token belongs to the reconstituted carrier exactly when its basic neighbourhood meets the filter associated with the element, which holds precisely for the tokens of the original carrier, by the membership criterion and deductive closure. ■

`toElement_ofElement.lean`

```
module

public import DomainTheory.BridgeTheorems.toElement
public import DomainTheory.BridgeTheorems.ofElement
public import DomainTheory.BridgeTheorems.interOf_singleton
public import DomainTheory.BridgeTheorems.ofElement_mem
public import DomainTheory.BridgeTheorems.toElement_carrier

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem toElement_ofElement (e : B.toInfoSys.Element) :
    B.toElement (B.ofElement e) = e := by
  have hc : (B.toElement (B.ofElement e)).carrier = e.carrier := by
    ext i
    constructor
    · intro hi
      rcases (B.ofElement_mem e (B.nbhd i)).mp hi with ⟨_, j, hj, hsub⟩
      have hEnt : B.toInfoSys.Ent {j} i := by
        refine ⟨by simpa [B.interOf_singleton] using B.nbhd_mem j, ?_⟩
        simpa [B.interOf_singleton] using hsub
      exact e.closed {j} i (by
        intro x hx
        have : x = j := Finset.mem_singleton.mp hx
        exact this ▸ hj) hEnt
    · intro hi
      exact ⟨B.nbhd_mem i, i, hi, subset_rfl⟩
  rcases e with ⟨c, cons, clo⟩
  rcases hte : B.toElement (B.ofElement ⟨c, cons, clo⟩) with ⟨c', cons', clo'⟩
  have hc' : c' = c := by
    simpa [hte, toElement_carrier] using hc
  subst hc'
  rfl
```

```
end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.19 Return from filters to information-system elements

Theorem (`ofElement_toElement`). Passing from a neighbourhood filter to an information-system element and back yields the original filter.

Proof. A neighbourhood belongs to the reconstituted filter exactly when it contains the basic neighbourhood of some index in the associated carrier, which is equivalent to membership in the original filter by exhaustiveness of the basis and the filter axioms. ■

`ofElement_toElement.lean`

```
module

public import DomainTheory.BridgeTheorems.toElement
public import DomainTheory.BridgeTheorems.ofElement
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ElementOrder

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.NeighbourhoodSystems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

theorem ofElement_toElement (x : B.system.Element) :
    B.ofElement (B.toElement x) = x := by
  refine NeighborhoodSystem.Element.ext (V := B.system) fun X => ?_
  constructor
  · intro ⟨hX, i, hi, hsub⟩
    exact x.up_mem hi hX hsub
  · intro hX
    obtain ⟨i, rfl⟩ := B.exhaustive (x.sub hX)
    exact ⟨x.sub hX, i, hX, subset_rfl⟩

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.20 Neighbourhood filters and information-system elements

Definition (`domainOrderIso`). The maps between neighbourhood filters and information-system elements form an order isomorphism.

The two composites are the identity by the round-trip theorems; order is preserved because inclusion of filters matches inclusion of carriers of indices.

Definition (`bridge_1980_1982`). Alias for that isomorphism, named by presentation years: 1980 neighbourhood filters to 1982 information-system elements.

domainOrderIso.lean

```
module

public import DomainTheory.BridgeTheorems.toElement_ofElement
public import DomainTheory.BridgeTheorems.ofElement_toElement
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace NbhdBasis

variable {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)

def domainOrderIso : B.system.Element ≃o B.toInfoSys.Element where
  toFun := B.toElement
  invFun := B.ofElement
  left_inv := B.ofElement_toElement
  right_inv := B.toElement_ofElement
  map_rel_iff' := by
    intro x y
    constructor
    · intro h X hX
      obtain ⟨i, rfl⟩ := B.exhaustive (x.sub hX)
      have hi : i ∈ (B.toElement x).carrier := hX
      have hi' : i ∈ (B.toElement y).carrier := h hi
      exact y.up_mem hi' (x.sub hX) subset_rfl
    · intro h i hi
      exact h (B.nbhd i) hi

/-- 1980 neighbourhood filters → 1982 information-system elements. -/
abbrev bridge_1980_1982 (B : NbhdBasis ι α) := domainOrderIso (B := B)

end NbhdBasis

end DomainTheory.BridgeTheorems
```

6.3.21 The information system induced by a neighbourhood basis

Definition (`neighborhoodSystem_to_infoSys`). The information system induced by a neighbourhood basis is the information system constructed from that basis above.

NeighborhoodToInfoSys.lean

```
module

public import DomainTheory.BridgeTheorems.toInfoSys
public import DomainTheory.BridgeTheorems.NeighborhoodToInfoSys.domainOrderIso
```

```
@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

abbrev neighborhoodSystem_to_infoSys {ι α : Type*} [DecidableEq ι] (B : NbhdBasis ι α)
    :
    InfoSys ι :=
  B.toInfoSys

end DomainTheory.BridgeTheorems
```

6.4 Ideals of finite elements

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

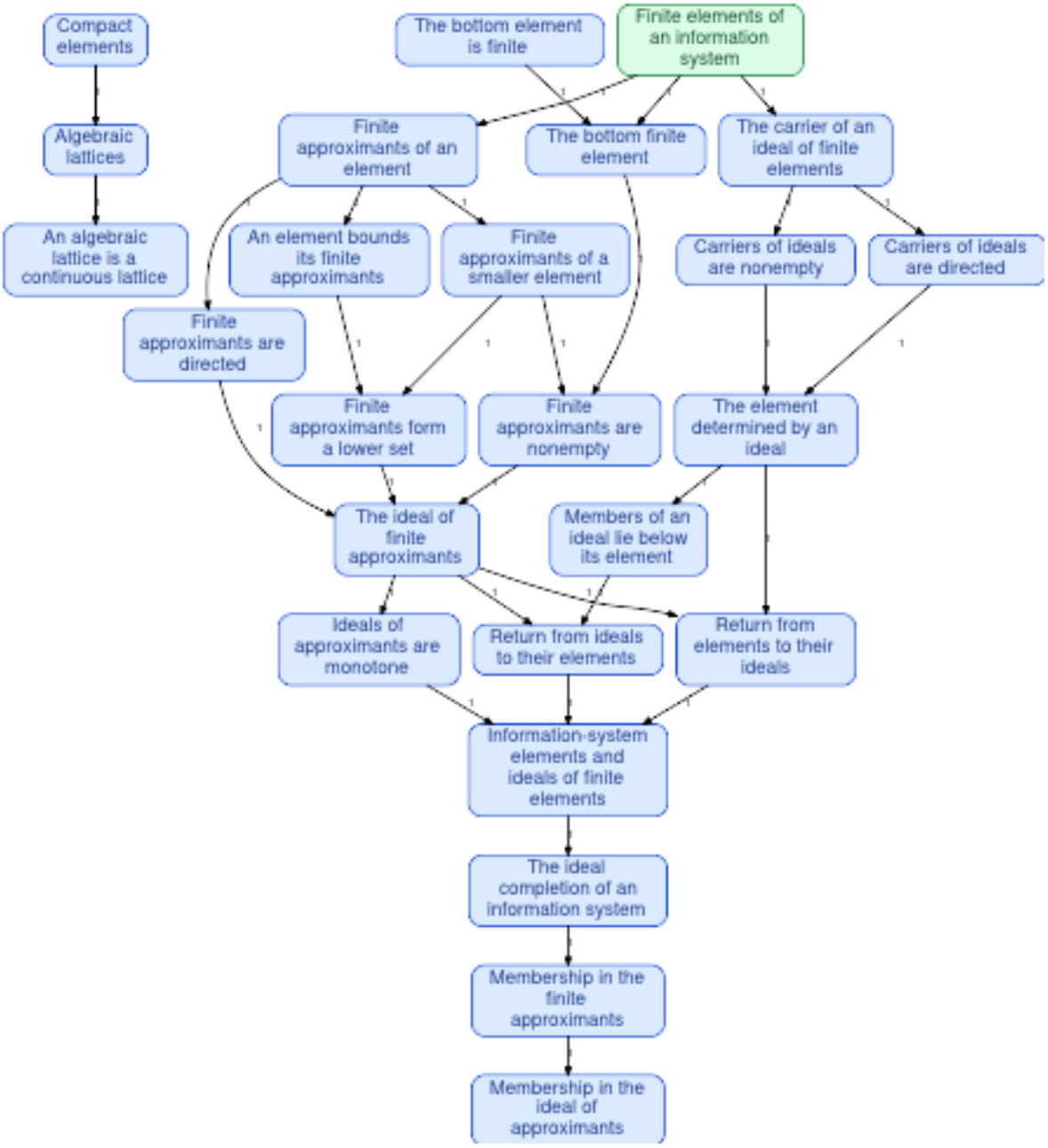


Figure 67: Diagram 32

6.4.1 Compact elements

Definition (`IsCompactElement`). An element of a complete lattice is compact when every directed set whose supremum lies above it has a member that already lies above it.

`IsCompactElement.lean`

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Order.Directed

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace IdealCompletionToContinuousLattice

variable {D : Type*} [CompleteLattice D]

def IsCompactElement (k : D) : Prop :=
  ∀ {{S : Set D}}, S.Nonempty → DirectedOn (· ≤ ·) S → k ≤ sSup S → ∃ s ∈ S, k ≤ s

end IdealCompletionToContinuousLattice

end DomainTheory.BridgeTheorems
```

6.4.2 Algebraic lattices

Definition (`AlgebraicLattice`). A complete lattice is algebraic when every element is the directed supremum of the compact elements lying below it.

`AlgebraicLattice.lean`

```
module

public import DomainTheory.BridgeTheorems.IsCompactElement
import Mathlib.Tactic.Continuity.Init

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace IdealCompletionToContinuousLattice

variable {D : Type*} [CompleteLattice D]

class AlgebraicLattice (D : Type*) [CompleteLattice D] : Prop where
  out : ∀ y : D,
    let S := {x : D | IsCompactElement x ∧ x ≤ y}
    S.Nonempty ∧ DirectedOn (· ≤ ·) S ∧ sSup S = y

end IdealCompletionToContinuousLattice

end DomainTheory.BridgeTheorems
```

6.4.3 Finite elements of an information system

Definition (`FiniteElement`). A finite element of an information system is an element that arises as the closure of some consistent finite token set.

FiniteElement.lean

```
module
public import DomainTheory.InformationSystems.ElementsOfASystem.closure

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

abbrev FiniteElement : Type _ :=
  { x : A.Element // ∃ (u : Finset α) (hu : u ∈ A.Con), x = A.closure u hu }

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.4 The bottom element is finite

Theorem (`isFinite_bot`). The bottom element of an information system is finite.

Proof. It is the closure of the empty consistent set. ■

isFinite_bot.lean

```
module
public import DomainTheory.InformationSystems.ElementsOfASystem.botElement
public import DomainTheory.InformationSystems.ElementsOfASystem.closure
import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    botElement_eq_closure_empty

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem isFinite_bot :
    ∃ (u : Finset α) (hu : u ∈ A.Con), A.botElement = A.closure u hu :=
  ⟨∅, A.con_empty, InfoSys.botElement.eq_closure_empty A⟩

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.5 The bottom finite element

Definition (`botFinite`). The bottom finite element is the bottom element, packaged as a finite element.

botFinite.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.FiniteElement
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.isFinite_bot

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def botFinite : FiniteElement A := ⟨A.botElement, isFinite_bot A⟩

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.6 Finite approximants of an element

Definition (`finiteApproximants`). The finite approximants of an element are those finite elements that lie below it in the information-system order.

finiteApproximants.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.FiniteElement
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    finiteApproximants

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def finiteApproximants (x : A.Element) : Set (FiniteElement A) :=
  { y | (y : A.Element) ∈ A.finiteApproximants x }

end InfoSysToIdealCompletion
```

```
end DomainTheory.BridgeTheorems
```

6.4.7 Finite approximants are directed

Theorem (`directed_finiteApproximants`). The finite approximants of an element form a directed set.

Proof. Finite approximants of an information-system element are directed under the element order. ■

`directed_finiteApproximants.lean`

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.finiteApproximants
public import Mathlib.Order.Directed
import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.closures_directed

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem directed_finiteApproximants (x : A.Element) :
    DirectedOn (· ≤ ·) (finiteApproximants A x) := by
  intro y₁ hy₁ y₂ hy₂
  obtain ⟨u, hu, huX, hy₁eq⟩ := hy₁
  obtain ⟨v, hv, hvX, hy₂eq⟩ := hy₂
  obtain ⟨w, hw, hwX, huw, hvw⟩ := A.closures_directed x hu hv huX hvX
  refine ⟨⟨A.closure w hw, ⟨w, hw, rfl⟩⟩, ⟨w, hw, hwX, rfl⟩, ?_, ?_⟩
  ·
    change (y₁ : A.Element) ≤ A.closure w hw
    exact hy₁eq ▸ huw
  · change (y₂ : A.Element) ≤ A.closure w hw
    exact hy₂eq ▸ hvw

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.8 Finite approximants of a smaller element

Theorem (`mem_finiteApproximants_of_le`). A finite approximant of a smaller element is a finite approximant of a larger one.

Proof. The order on elements is transitive. ■

`mem_finiteApproximants_of_le.lean`

```
module
```

```
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.finiteApproximants

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem mem_finiteApproximants_of_le {x : A.Element} {y : FiniteElement A}
    (hle : (y : A.Element) ≤ x) : y ∈ finiteApproximants A x := by
  rcases y with ⟨yval, ⟨u, hu, rfl⟩⟩
  refine ⟨u, hu, ?_, rfl⟩
  intro a ha
  exact hle (closure.subset A hu ha)

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.9 Finite approximants are nonempty

Theorem (`nonempty_finiteApproximants`). Every element has at least one finite approximant.

Proof. The bottom finite element lies below every element. ■

`nonempty_finiteApproximants.lean`

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.finiteApproximants
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.botFinite
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    mem_finiteApproximants_of_le

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem nonempty_finiteApproximants (x : A.Element) :
    (finiteApproximants A x).Nonempty :=
  ⟨botFinite A, mem_finiteApproximants_of_le A (InfoSys.botElement.le A x)⟩

end InfoSysToIdealCompletion
```

```
end DomainTheory.BridgeTheorems
```

6.4.10 An element bounds its finite approximants

Theorem (`le_of_mem_finiteApproximants`). Every finite approximant of an element lies below that element.

Proof. Immediate from the definition of finite approximants. ■

`le_of_mem_finiteApproximants.lean`

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.finiteApproximants
import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.closure_le_element

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem le_of_mem_finiteApproximants {x : A.Element} {y : FiniteElement A}
    (hy : y ∈ finiteApproximants A x) : (y : A.Element) ≤ x := by
  obtain ⟨u, hu, huX, hyeq⟩ := hy
  exact hyeq ▸ A.closure_le_element x hu huX

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.11 Finite approximants form a lower set

Theorem (`isLowerSet_finiteApproximants`). The finite approximants of an element form a lower set in the finite elements.

Proof. If a finite element lies below a finite approximant, it lies below the ambient element. ■

`isLowerSet_finiteApproximants.lean`

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.finiteApproximants
public import Mathlib.Order.UpperLower.Basic
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    le_of_mem_finiteApproximants
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    mem_finiteApproximants_of_le
```

```
@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem isLowerSet_finiteApproximants (x : A.Element) :
    IsLowerSet (finiteApproximants A x) := by
  intro y₁ y₂ hle hy₂
  exact mem_finiteApproximants_of_le A (le_trans hle (le_of_mem_finiteApproximants A
    hy₂))

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.12 The carrier of an ideal of finite elements

Definition (`idealCarrier`). The carrier of an ideal of finite elements is the underlying set of information-system elements obtained by forgetting the finiteness witness.

idealCarrier.lean

```
module

public import Mathlib.Order.Ideal
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.FiniteElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open Order

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def idealCarrier (I : Ideal (FiniteElement A)) : Set A.Element :=
  Subtype.val '' (I : Set (FiniteElement A))

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.13 Carriers of ideals are nonempty

Theorem (`nonempty_idealCarrier`). The carrier of an ideal of finite elements is nonempty.

Proof. Ideals are nonempty. ■

`nonempty_idealCarrier.lean`

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.idealCarrier

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open Order

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem nonempty_idealCarrier (I : Ideal (FiniteElement A)) :
    (idealCarrier A I).Nonempty := by
  obtain ⟨y, hy⟩ := I.nonempty
  exact ⟨y, y, hy, rfl⟩

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.14 Carriers of ideals are directed

Theorem (`directed_idealCarrier`). The carrier of an ideal of finite elements is directed.

Proof. Ideals are directed, and the forgetful map preserves the order. ■

`directed_idealCarrier.lean`

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.idealCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    IsDirected

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open Order

namespace InfoSysToIdealCompletion
```

```
variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem directed_idealCarrier (I : Ideal (FiniteElement A)) :
    A.IsDirected (idealCarrier A I) := by
  intro x y hx hy
  obtain ⟨x', hx', rfl⟩ := hx
  obtain ⟨y', hy', rfl⟩ := hy
  obtain ⟨z', hz', hxz, hyz⟩ := I.directed x' hx' y' hy'
  exact ⟨z', ⟨z', hz', rfl⟩, hxz, hyz⟩

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.15 The ideal of finite approximants

Definition (`toIdeal`). The ideal of finite approximants of an element is the directed lower set of all finite elements lying below that element.

`toIdeal.lean`

```
module

public import Mathlib.Order.Ideal
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    nonempty_finiteApproximants
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    directed_finiteApproximants
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    isLowerSet_finiteApproximants

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open Order

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def toIdeal (x : A.Element) : Ideal (FiniteElement A) where
  carrier := finiteApproximants A x
  lower' := isLowerSet_finiteApproximants A x
  nonempty' := nonempty_finiteApproximants A x
  directed' := directed_finiteApproximants A x

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.16 The element determined by an ideal

Definition (`ofIdeal`). The element determined by an ideal of finite elements is the directed supremum of the ideal's carrier.

ofIdeal.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    nonempty_idealCarrier
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    directed_idealCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    directedSup

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open Order

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

noncomputable def ofIdeal (I : Ideal (FiniteElement A)) : A.Element :=
  A.directedSup (idealCarrier A I) (nonempty_idealCarrier A I) (directed_idealCarrier
    A I)

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.17 Ideals of approximants are monotone

Theorem (`toIdeal_mono`). The assignment of an element to its ideal of finite approximants is monotone.

Proof. Finite approximants of a smaller element remain approximants of a larger one. ■

toIdeal_mono.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.toIdeal
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    le_of_mem_finiteApproximants
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    mem_finiteApproximants_of_le

@[expose] public section

namespace DomainTheory.BridgeTheorems
```

```
open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem toIdeal_mono {x y : A.Element} (hxy : x ≤ y) : toIdeal A x ≤ toIdeal A y := by
  intro z hz
  exact mem_finiteApproximants_of_le A
    (le_trans (le_of_mem_finiteApproximants A hz) hxy)

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.18 Members of an ideal lie below its element

Theorem (`le_ofIdeal_of_mem`). Every member of an ideal of finite elements lies below the element determined by that ideal.

Proof. The directed supremum bounds every member of the directed set. ■

`le_ofIdeal_of_mem.lean`

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.ofIdeal

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open Order

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem le_ofIdeal_of_mem {I : Ideal (FiniteElement A)} {y : FiniteElement A}
    (hy : y ∈ I) : (y : A.Element) ≤ ofIdeal A I :=
  InfoSys.directedSup.le_of_mem A _ _ _ ⟨y, hy, rfl⟩

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.19 Return from ideals to their elements

Theorem (`toIdeal_ofIdeal`). Passing from an ideal of finite elements to its determined element and back yields the original ideal.

Proof. A finite element belongs to the reconstituted ideal exactly when it lies below the directed supremum of the original ideal, which holds precisely for the members of a directed lower set of finite elements. ■

toIdeal_ofIdeal.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.toIdeal
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.ofIdeal
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.le_ofIdeal_of_mem
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    le_of_mem_finiteApproximants
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    mem_finiteApproximants_of_le

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open Order

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem toIdeal_ofIdeal (I : Ideal (FiniteElement A)) : toIdeal A (ofIdeal A I) = I :=
     by
  refine Ideal.ext ?_
  ext y
  constructor
  · intro hy
    have hle : (y : A.Element) ≤ ofIdeal A I := le_of_mem_finiteApproximants A hy
    rcases y with ⟨yval, ⟨u, hu, rfl⟩⟩
    obtain ⟨z, hz, hcle⟩ :=
      InfoSys.directedSup.compact_closure A (idealCarrier A I) (nonempty_idealCarrier
    A I)
        (directed_idealCarrier A I) hu hle
    obtain ⟨z', hz', rfl⟩ := hz
    exact I.lower hcle hz'
  · intro hy
    exact mem_finiteApproximants_of_le A (le_ofIdeal_of_mem A hy)

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.20 Return from elements to their ideals

Theorem (`ofIdeal_toIdeal`). Passing from an element to its ideal of finite approximants and back yields the original element.

Proof. Every information-system element is the directed supremum of its finite approximants. ■

ofIdeal_toIdeal.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.toIdeal
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.ofIdeal

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem ofIdeal_toIdeal (x : A.Element) : ofIdeal A (toIdeal A x) = x := by

  have hset : idealCarrier A (toIdeal A x) = A.finiteApproximants x := by
    ext z
    constructor
    · intro hz
      obtain ⟨y, hy, rfl⟩ := hz
      exact hy
    · intro hz
      refine ⟨⟨z, ?_⟩, hz, rfl⟩
      obtain ⟨u, hu, _, rfl⟩ := hz
      exact ⟨u, hu, rfl⟩

  change A.directedSup (idealCarrier A (toIdeal A x)) _ _ = x
  simp_rw [hset]
  exact (InfoSys.finiteApproximants.eq_directedSup A x).symm

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.21 Information-system elements and ideals of finite elements

Definition (`domainOrderIso`). The maps between information-system elements and ideals of finite elements form an order isomorphism.

The two composites are the identity by the round-trip theorems; order is preserved by monotonicity of the ideal assignment and the comparison of directed suprema.

domainOrderIso.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.ofIdeal_toIdeal
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.toIdeal_ofIdeal
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.le_ofIdeal_of_mem
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.toIdeal_mono
```

```
@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open Order

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

noncomputable def domainOrderIso : A.Element ≃o Ideal (FiniteElement A) where
  toFun := toIdeal A
  invFun := ofIdeal A
  left_inv := ofIdeal_toIdeal A
  right_inv := toIdeal_ofIdeal A
  map_rel_iff' := by
    intro x y
    constructor
    · intro h

      rw [← ofIdeal_toIdeal A x, ← ofIdeal_toIdeal A y]
      refine InfoSys.directedSup.le A _ _ _ ?_
      intro z hz
      obtain ⟨z', hz', rfl⟩ := hz
      exact le_ofIdeal_of_mem A (h hz')
    · exact toIdeal_mono A

end InfoSysToIdealCompletion

end DomainTheory.BridgeTheorems
```

6.4.22 The ideal completion of an information system

Definition (`infoSys_to_idealCompletion`). The ideal completion of an information system is the order isomorphism between its elements and the ideals of its finite elements.

Definition (`bridge_1982_1972`). Alias for that isomorphism, named by presentation years: 1982 information-system elements to the 1972 algebraic carrier (ideals of finite elements).

Basic.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.domainOrderIso

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems
open Order
```

```
noncomputable abbrev infoSys_to_idealCompletion {α : Type*} [DecidableEq α] (A :
    InfoSys α) :
    A.Element ≃o Ideal (InfoSysToIdealCompletion.FiniteElement A) :=
  InfoSysToIdealCompletion.domainOrderIso A

/-- 1982 information-system elements → 1972 algebraic carrier (ideals of finite
    elements). -/
noncomputable abbrev bridge_1982_1972 := @infoSys_to_idealCompletion

end DomainTheory.BridgeTheorems
```

6.4.23 Membership in the finite approximants

Theorem (`mem_finiteApproximants_iff`). A finite element is a finite approximant of an element if and only if it lies below that element.

Proof. Unfold the definition of finite approximants. ■

`mem_finiteApproximants_iff.lean`

```
module
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.finiteApproximants
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    le_of_mem_finiteApproximants
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    mem_finiteApproximants_of_le

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem mem_finiteApproximants_iff {x : A.Element} {y : FiniteElement A} :
    y ∈ finiteApproximants A x ↔ (y : A.Element) ≤ x :=
  ⟨le_of_mem_finiteApproximants A, mem_finiteApproximants_of_le A⟩

end InfoSysToIdealCompletion
end DomainTheory.BridgeTheorems
```

6.4.24 Membership in the ideal of approximants

Theorem (`mem_toIdeal_iff`). A finite element belongs to the ideal of approximants of an element if and only if it lies below that element.

Proof. Membership in the ideal is membership among the finite approximants. ■

`mem_toIdeal_iff.lean`

```
module
```

```
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.toIdeal
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.
    mem_finiteApproximants_iff

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysToIdealCompletion

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem mem_toIdeal_iff {x : A.Element} {y : FiniteElement A} :
    y ∈ toIdeal A x ↔ (y : A.Element) ≤ x :=
  mem_finiteApproximants_iff A

end InfoSysToIdealCompletion
end DomainTheory.BridgeTheorems
```

6.4.25 An algebraic lattice is a continuous lattice

Theorem (`scottOpen_Ici_of_compact`). The principal upset of a compact element is Scott-open.

Proof. Compactness supplies a member of any directed set meeting the upset that already lies in the upset, and principal upsets are upper sets. ■

Theorem (`compact_wayBelow`). A compact element lying below y is way-below y.

Proof. The principal upset of the compact element is Scott-open, contains y, and is contained in the principal upset of the compact element. ■

Theorem (`isContinuousLattice_of_algebraic`). Every algebraic complete lattice is a continuous lattice.

Proof. Compact elements below y are way-below y, and algebraicity expresses y as their directed supremum. ■

Definition (`idealCompletion_to_continuousLattice`). The continuous-lattice structure on an algebraic complete lattice is the one obtained from the preceding theorem.

`IdealCompletionToContinuousLattice.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.AlgebraicLattice

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace IdealCompletionToContinuousLattice

variable {D : Type*} [CompleteLattice D]
```

```
theorem scottOpen_Ici_of_compact {k : D} (hk : IsCompactElement k) :
    ScottOpen (Set.Ici k) := by
  refine ⟨IsUpperSet.Ici k, fun S hS hSdir hmem => ?_⟩
  obtain ⟨s, hsS, hks⟩ := hk hS hSdir (Set.mem_Ici.1 hmem)
  exact ⟨s, hsS, Set.mem_Ici.2 hks⟩

theorem compact_wayBelow {k y : D} (hk : IsCompactElement k) (hky : k ≤ y) : k ≪ y :=
  ⟨Set.Ici k, scottOpen_Ici_of_compact hk, Set.mem_Ici.2 hky, subset_rfl⟩

theorem isContinuousLattice_of_algebraic (hA : AlgebraicLattice D) :
    ContinuousLattice D := by
  refine { out := ?_ }
  intro y
  obtain ⟨hne, hdir, hsup⟩ := hA.out y
  refine ⟨?_, fun z hz => ?_⟩
  ·
    intro x hx
    exact hx.le
  ·

    have hle : sSup {x : D | IsCompactElement x ∧ x ≤ y} ≤ z := by
      refine sSup_le fun x hx => ?_
      exact hz (compact_wayBelow hx.1 hx.2)
    exact hsup ▸ hle

end IdealCompletionToContinuousLattice

abbrev idealCompletion_to_continuousLattice {D : Type*} [CompleteLattice D]
    (hA : IdealCompletionToContinuousLattice.AlgebraicLattice D) :
    ContinuousLattice D :=
  IdealCompletionToContinuousLattice.isContinuousLattice_of_algebraic hA

end DomainTheory.BridgeTheorems
```

6.5 Round way-below filters

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

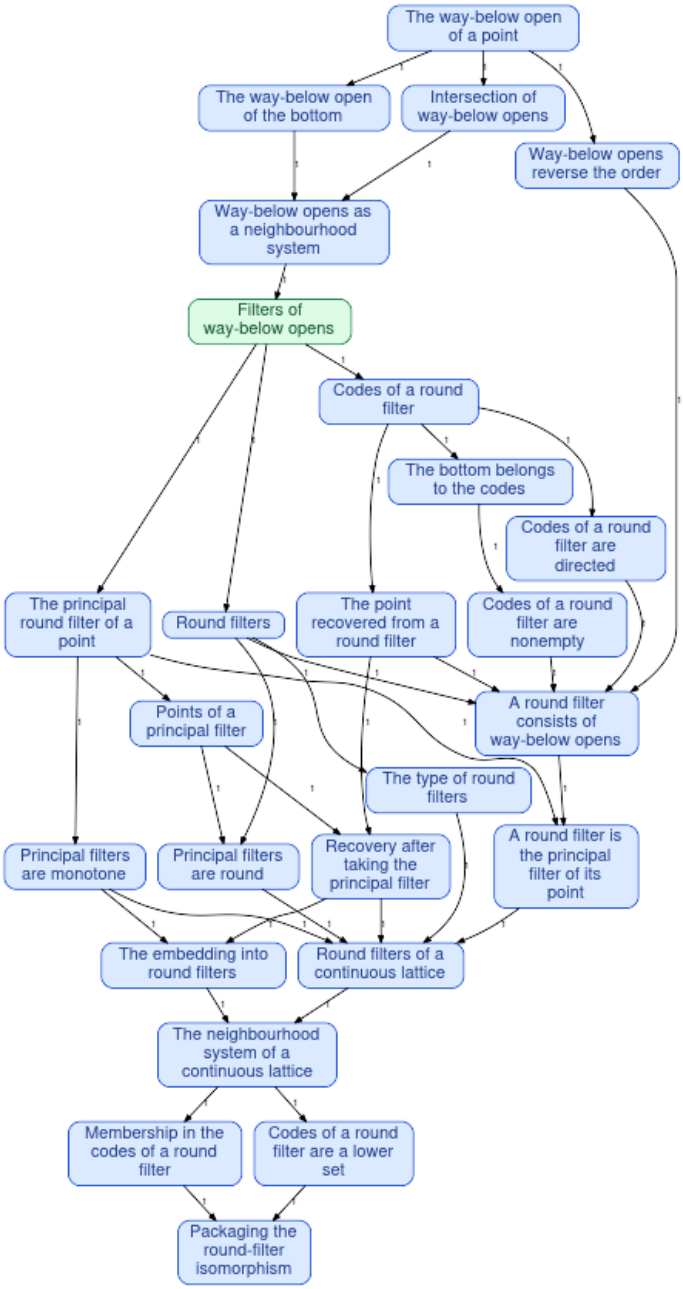


Figure 68: Diagram 33

6.5.1 The way-below open of a point

Definition (`wayBelowUp`). The way-below open of a point is the set of points way-above it.

`wayBelowUp.lean`

```
module
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
```

```
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    GreedyEnum

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

def wayBelowUp (a : D) : Set D := {z | a ≪ z}

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.2 The way-below open of the bottom

Theorem (`wayBelowUp_bot`). The way-below open of the bottom element is the whole lattice.

Proof. The bottom is way-below every element. ■

`wayBelowUp_bot.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem wayBelowUp_bot : wayBelowUp (⊥ : D) = (Set.univ : Set D) := by
  ext z
  exact ⟨fun _ => trivial, fun _ => WayBelow.bot z⟩

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.3 Way-below opens reverse the order

Theorem (`wayBelowUp_anti`). If $a \leq b$, then the way-below open of b is contained in that of a.

Proof. Way-below is monotone in its lower argument. ■

`wayBelowUp_anti.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem wayBelowUp_anti {a b : D} (hab : a ≤ b) :
    wayBelowUp b ⊆ wayBelowUp a :=
  fun _ h => WayBelow.le_trans hab h

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.4 Intersection of way-below opens

Theorem (`wayBelowUp_inter`). The intersection of the way-below opens of a and b is the way-below open of their join.

Proof. A point is way-above both a and b if and only if it is way-above their join. ■

`wayBelowUp_inter.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem wayBelowUp_inter (a b : D) :
    wayBelowUp a ∩ wayBelowUp b = wayBelowUp (a ⊔ b) := by
  ext z
```

```
  constructor
  · exact fun ⟨ha, hb⟩ => WayBelow.sup ha hb
  · intro h
    exact ⟨WayBelow.le_trans le_sup_left h, WayBelow.le_trans le_sup_right h⟩

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.5 Way-below opens as a neighbourhood system

Definition (`toNeighborhoodSystem`). On a continuous lattice, the way-below opens form a neighbourhood system with the whole lattice as master neighbourhood.

The empty join gives the master neighbourhood, and binary intersections of way-below opens are way-below opens of joins.

`toNeighborhoodSystem.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_bot
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_inter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.NeighbourhoodSystems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

def toNeighborhoodSystem : NeighborhoodSystem D where
  mem X := ∃ a : D, X = wayBelowUp a
  master := Set.univ
  master_nonempty := ⟨⊥, Set.mem_univ _⟩
  master_mem := ⟨⊥, wayBelowUp_bot.symm⟩
  inter_mem := by
    intro X Y _Z hX hY _hZ _hZsub
    obtain ⟨a, rfl⟩ := hX
    obtain ⟨b, rfl⟩ := hY
    exact ⟨a ⊔ b, wayBelowUp_inter a b⟩
  sub_master := fun {_} _ => Set.subset_univ _

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.6 Filters of way-below opens

Definition (`Filter`). A filter of way-below opens is an element of the way-below neighbourhood system.

`Filter.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toNeighborhoodSystem
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.NeighbourhoodSystems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

abbrev Filter : Type _ :=
  (toNeighborhoodSystem : NeighborhoodSystem D).Element

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.7 Round filters

Definition (`Round`). A filter of way-below opens is round when every way-below open it contains has a strictly interpolating way-below open also in the filter.

`Round.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

class Round (f : Filter (D := D)) : Prop where
  out : ∀ {a : D}, f.mem (wayBelowUp a) → ∃ b : D, a ≪ b ∧ f.mem (wayBelowUp b)

end ContinuousLatticeToNeighborhood
```

```
end DomainTheory.BridgeTheorems
```

6.5.8 The type of round filters

Definition (`RoundFilter`). A round filter is a filter of way-below opens equipped with a roundness witness.

RoundFilter.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Round
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    GreedyEnum
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

abbrev RoundFilter : Type _ :=
  { f : Filter (D := D) // Round f }

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.9 The principal round filter of a point

Definition (`toFilter`). The principal filter of a point consists of those way-below opens of points that are way-below the given point.

toFilter.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_bot
```

```
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_inter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

def toFilter (x : D) : Filter (D := D) where
  mem U := ∃ a : D, U = wayBelowUp a ∧ a ≪ x
  sub := by
    intro U h
    obtain ⟨a, ⟨rfl, _⟩⟩ := h
    exact ⟨a, rfl⟩
  master_mem := ⟨⊥, wayBelowUp_bot.symm, WayBelow.bot x⟩
  inter_mem := by
    intro U V hU hV
    obtain ⟨a, ⟨rfl, ha⟩⟩ := hU
    obtain ⟨b, ⟨rfl, hb⟩⟩ := hV
    exact ⟨a ⊔ b, ⟨wayBelowUp_inter a b, WayBelow.sup ha hb⟩⟩
  up_mem := by
    intro U V hU hV hUV
    obtain ⟨a, ⟨rfl, ha⟩⟩ := hU
    obtain ⟨b, rfl⟩ := hV
    exact ⟨b, ⟨rfl, hUV ha⟩⟩

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.10 Principal filters are monotone

Theorem (`toFilter_mono`). The principal-filter assignment is monotone.

Proof. Way-below is monotone in its upper argument. ■

`toFilter_mono.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]
```

```
theorem toFilter_mono {x y : D} (hxy : x ≤ y) : toFilter x ≤ toFilter y := by
  intro U hU
  obtain ⟨a, ⟨rfl, ha⟩⟩ := hU
  exact ⟨a, ⟨rfl, ha.trans_le hxy⟩⟩

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.11 Points of a principal filter

Theorem (`mem_wayBelowUp_toFilter`). The way-below open of a belongs to the principal filter of x if and only if a is way-below x.

Proof. Unfold the definition of the principal filter. ■

`mem_wayBelowUp_toFilter.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem mem_wayBelowUp_toFilter {x a : D} :
    (toFilter x).mem (wayBelowUp a) ↔ a ≪ x := by
  constructor
  · intro h
    obtain ⟨b, ⟨heq, hb⟩⟩ := h
    have : x ∈ wayBelowUp b := hb
    rwa [← heq] at this
  · intro ha
    exact ⟨a, ⟨rfl, ha⟩⟩

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.12 Principal filters are round

Theorem (`toFilter_isRound`). Every principal filter is round.

Proof. Continuity interpolates way-below: if $a \ll x$, some b satisfies $a \ll b \ll x$. ■

`toFilter_isRound.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Round

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_toFilter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem toFilter_isRound (hD : ContinuousLattice D) (x : D) : Round (toFilter x) := by
  constructor
  intro a ha
  have ha' : a ≪ x := mem_wayBelowUp_toFilter.mp ha
  obtain ⟨b, hab, hbx⟩ := WayBelow.interpolate hD ha'
  exact ⟨b, hab, mem_wayBelowUp_toFilter.mpr hbx⟩

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.13 Codes of a round filter

Definition (`codes`). The codes of a filter are those points whose way-below opens belong to the filter.

`codes.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

def codes (f : Filter (D := D)) : Set D :=
  {a : D | f.mem (wayBelowUp a)}

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.14 The point recovered from a round filter

Definition (`ofFilter`). The point recovered from a filter is the supremum of its codes.

ofFilter.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

noncomputable def ofFilter (f : Filter (D := D)) : D :=
  sSup (codes f)

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.15 Recovery after taking the principal filter

Theorem (`ofFilter_toFilter`). Recovering a point from its principal filter yields the original point.

Proof. On a continuous lattice every element is the supremum of the points way-below it. ■

ofFilter_toFilter.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.ofFilter

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_toFilter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem ofFilter_toFilter (hD : ContinuousLattice D) (x : D) : ofFilter (toFilter x) =
     x := by
  have hset : codes (toFilter x) = {a : D | a ≪ x} := by
    ext a
```

```
    exact mem_wayBelowUp_toFilter
  change sSup (codes (toFilter x)) = x
  rw [hset, hD.sSup_wayBelow x]

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.16 The bottom belongs to the codes

Theorem (`bot_mem_codes`). The bottom element is always a code of a filter of way-below opens.

Proof. Every filter contains the master neighbourhood, which is the way-below open of the bottom. ■

`bot_mem_codes.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_bot

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem bot_mem_codes (f : Filter (D := D)) : (⊥ : D) ∈ codes f := by
  have : f.mem (wayBelowUp (⊥ : D)) := by
    rw [wayBelowUp_bot]
    exact f.master_mem
  exact this

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.17 Codes of a round filter are nonempty

Theorem (`codes_nonempty`). The codes of a filter are nonempty.

Proof. The bottom is always a code. ■

`codes_nonempty.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    bot_mem_codes
```

```
@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem codes_nonempty (f : Filter (D := D)) : (codes f).Nonempty :=
  ⟨⊥, bot_mem_codes f⟩

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.18 Codes of a round filter are directed

Theorem (`codes_directed`). The codes of a filter are directed.

Proof. Filters are closed under binary intersections of way-below opens, which correspond to joins of codes. ■

`codes_directed.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_inter

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem codes_directed (f : Filter (D := D)) : DirectedOn (· ≤ ·) (codes f) := by
  intro a ha b hb
  refine ⟨a ⊔ b, ?_, le_sup_left, le_sup_right⟩
  have : f.mem (wayBelowUp a ∩ wayBelowUp b) := f.inter_mem ha hb
  rwa [wayBelowUp_inter] at this

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.19 A round filter consists of way-below opens

Theorem (`mem_wayBelowUp_ofFilter_of_round`). For a round filter, membership of a way-below open is equivalent to the corresponding code lying way-below the recovered point.

Proof. Roundness supplies interpolants that place codes below the supremum of codes, and the converse uses that the filter contains all way-below opens of its codes. ■

mem_wayBelowUp_ofFilter_of_round.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Round
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.ofFilter

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    codes_directed
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    codes_nonempty
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_anti

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem mem_wayBelowUp_ofFilter_of_round {f : Filter (D := D)} (hr : Round f) {a : D}
    :
    f.mem (wayBelowUp a) ↔ a ≪ ofFilter f := by
  constructor
  · intro ha
    obtain ⟨b, hab, hb⟩ := hr.out ha
    exact (WayBelow.sSup_iff (codes_nonempty f) (codes_directed f)).2 ⟨b, hb, hab⟩
  · intro ha
    obtain ⟨b, hbS, hab⟩ := (WayBelow.sSup_iff (codes_nonempty f) (codes_directed f)).1
     ha
    exact f.up_mem hbS ⟨a, rfl⟩ (wayBelowUp_anti hab.le)

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.20 A round filter is the principal filter of its point

Theorem (`toFilter_ofFilter`). A round filter coincides with the principal filter of its recovered point.

Proof. The membership criterion for round filters identifies the filter with the principal filter of the supremum of its codes. ■

toFilter_ofFilter.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
```

```
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Round
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.ofFilter

import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.ElementOrder
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_ofFilter_of_round

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.NeighbourhoodSystems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem toFilter_ofFilter {f : Filter (D := D)} (hr : Round f) :
    toFilter (ofFilter f) = f := by
  refine NeighborhoodSystem.Element.ext (V := toNeighborhoodSystem) fun U => ?_
  constructor
  · intro hU
    obtain ⟨a, ⟨rfl, ha⟩⟩ := hU
    exact (mem_wayBelowUp_ofFilter_of_round hr).2 ha
  · intro hU
    obtain ⟨a, rfl⟩ := f.sub hU
    refine ⟨a, ⟨rfl, ?_⟩⟩
    exact (mem_wayBelowUp_ofFilter_of_round hr).1 hU

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.21 The embedding into round filters

Definition (`domainEmbedding`). Sending a point to its principal round filter embeds the continuous lattice into the round filters.

domainEmbedding.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    ofFilter_toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toFilter_mono

@[expose] public section
```

```
namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

noncomputable def domainEmbedding (hD : ContinuousLattice D) :
    D ↪o Filter (D := D) where
  toFun := toFilter
  inj' := by
    intro x y h
    rw [← ofFilter_toFilter hD x, ← ofFilter_toFilter hD y, h]
  map_rel_iff' := by
    intro x y
    constructor
    · intro h
      have : ofFilter (toFilter x) ≤ ofFilter (toFilter y) := by
        refine sSup_le fun a ha => ?_
        have ha' : a ≪ x := mem_wayBelowUp_toFilter.mp ha
        exact le_sSup (h (wayBelowUp a) ⟨a, ⟨rfl, ha'⟩⟩)
      simpa [ofFilter_toFilter hD] using this
    · exact fun hxy => toFilter_mono hxy

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.22 Round filters of a continuous lattice

Definition (`domainOrderIso`). On a continuous lattice, points and round filters of way-below opens are order-isomorphic via the principal-filter and recovery maps.

The round-trip theorems identify the two sides; order is preserved by monotonicity of principal filters.

`domainOrderIso.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toFilter_ofFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toFilter_isRound
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    ofFilter_toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.RoundFilter

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    ElementOrder
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toFilter_mono
```

```
@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

noncomputable def domainOrderIso (hD : ContinuousLattice D) : D ≃o RoundFilter (D := D
    ) where
  toFun x := ⟨toFilter x, toFilter_isRound hD x⟩
  invFun f := ofFilter (D := D) f.1
  left_inv x := ofFilter_toFilter hD x
  right_inv f := Subtype.ext (toFilter_ofFilter f.2)
  map_rel_iff' := by
    intro x y
    constructor
    · intro h
      have : ofFilter (toFilter x) ≤ ofFilter (toFilter y) := by
        refine sSup_le fun a ha => ?_
        have ha' : a ≪ x := mem_wayBelowUp_toFilter.mp ha
        exact le_sSup (h (wayBelowUp a) ⟨a, ⟨rfl, ha'⟩⟩)
      simpa [ofFilter_toFilter hD] using this
    · intro hxy
      exact toFilter_mono hxy

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

6.5.23 The neighbourhood system of a continuous lattice

Definition (`continuousLattice_to_neighborhoodSystem`). The neighbourhood system of a continuous lattice is the way-below neighbourhood system constructed above.

`Basic.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toNeighborhoodSystem

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.NeighbourhoodSystems

abbrev continuousLattice_to_neighborhoodSystem {D : Type*} [CompleteLattice D]
    (_hD : ContinuousLattice D) : NeighborhoodSystem D :=
  ContinuousLatticeToNeighborhood.toNeighborhoodSystem
```

```
end DomainTheory.BridgeTheorems
```

6.5.24 Membership in the codes of a round filter

Theorem (`mem_codes_iff`). A point is a code of a filter if and only if its way-below open belongs to the filter.

Proof. Unfold the definition of codes. ■

`mem_codes_iff.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem mem_codes_iff {f : Filter (D := D)} {a : D} :
    a ∈ codes f ↔ f.mem (wayBelowUp a) :=
  Iff.rfl

end ContinuousLatticeToNeighborhood
end DomainTheory.BridgeTheorems
```

6.5.25 Codes of a round filter are a lower set

Theorem (`codes_lower`). The codes of a filter form a lower set.

Proof. If $a \leq b$ and the way-below open of b is in the filter, then so is that of a, by anti-monotonicity of way-below opens and upward closure of filters. ■

`codes_lower.lean`

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_anti

@[expose] public section

namespace DomainTheory.BridgeTheorems

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]
```

```
theorem codes_lower (f : Filter (D := D)) {a b : D} (hba : b ≤ a)
    (ha : a ∈ codes f) : b ∈ codes f :=
  f.up_mem ha ⟨b, rfl⟩ (wayBelowUp_anti hba)

end ContinuousLatticeToNeighborhood
end DomainTheory.BridgeTheorems
```

6.5.26 Packaging the round-filter isomorphism

Definition (`continuousLattice_roundFilter_iso`). The packaged order isomorphism between a continuous lattice and its round filters of way-below opens.

Definition (`bridge_1972_1980`). Alias for that isomorphism, named by presentation years: 1972 continuous lattice to 1980 round way-below filters.

`continuousLattice_roundFilter_iso.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    domainOrderIso
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.Hom.Basic
public import DomainTheory.BridgeTheorems.InfoSysToNeighborhood
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion
public import DomainTheory.BridgeTheorems.NeighborhoodToInfoSys
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood
public import DomainTheory.BridgeTheorems.IdealCompletionToContinuousLattice

@[expose] public section

namespace DomainTheory.BridgeTheorems

open ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D] [DecidableEq D]

noncomputable def continuousLattice_roundFilter_iso (hD : ContinuousLattice D) :
    D ≃o RoundFilter (D := D) :=
  ContinuousLatticeToNeighborhood.domainOrderIso hD

/-- 1972 continuous lattice → 1980 round way-below filters. -/
noncomputable abbrev bridge_1972_1980 {D : Type*} [CompleteLattice D] [DecidableEq D]
    (hD : ContinuousLattice D) :=
  continuousLattice_roundFilter_iso hD

end DomainTheory.BridgeTheorems
```

6.6 The round information system

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

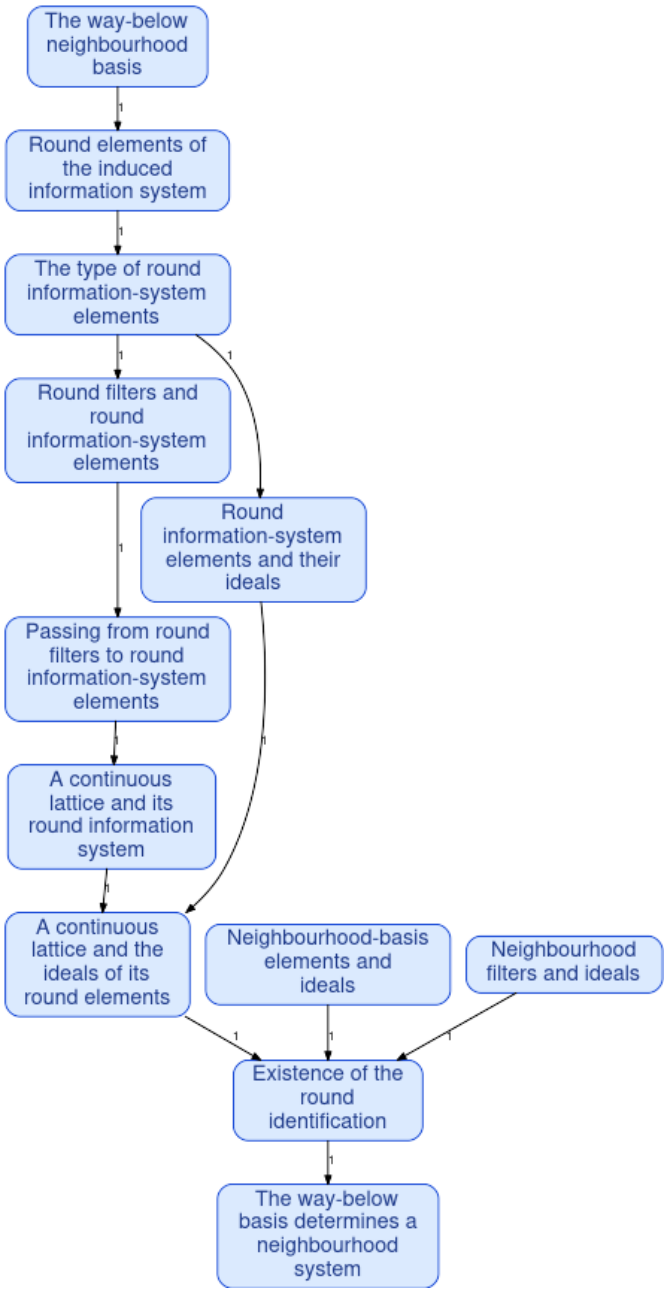


Figure 69: Diagram 34

6.6.1 The way-below neighbourhood basis

Definition (`wayBelowNbhdBasis`). The way-below neighbourhood basis of a continuous lattice indexes each way-below open by its generating point, with bottom as the bottom index.

`wayBelowNbhdBasis.lean`

```
module
```

```
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toNeighborhoodSystem
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_bot
public import DomainTheory.BridgeTheorems.NbhdBasis
public import Mathlib.Order.Hom.Basic
public import DomainTheory.BridgeTheorems.InfoSysToNeighborhood
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion
public import DomainTheory.BridgeTheorems.NeighborhoodToInfoSys
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood
public import DomainTheory.BridgeTheorems.IdealCompletionToContinuousLattice

@[expose] public section

namespace DomainTheory.BridgeTheorems

open ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D] [DecidableEq D]

def wayBelowNbhdBasis : NbhdBasis D D where
  system := toNeighborhoodSystem
  nbhd := wayBelowUp
  nbhd_mem := fun a => ⟨a, rfl⟩
  exhaustive := by
    intro X hX
    obtain ⟨a, rfl⟩ := hX
    exact ⟨a, rfl⟩
  botIdx := ⊥
  botIdx_eq := wayBelowUp_bot

end DomainTheory.BridgeTheorems
```

6.6.2 Round elements of the induced information system

Definition (`IsRoundInfoSysElement`). An element of the information system induced by the way-below neighbourhood basis is round when the corresponding neighbourhood filter is round.

`IsRoundInfoSysElement.lean`

```
module

public import DomainTheory.BridgeTheorems.PresentationDomains.wayBelowNbhdBasis
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Round
public import DomainTheory.BridgeTheorems.NeighborhoodToInfoSys.domainOrderIso

@[expose] public section

namespace DomainTheory.BridgeTheorems

open ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D] [DecidableEq D]
```

```
class IsRoundInfoSysElement (e : (wayBelowNbhdBasis (D := D)).toInfoSys.Element) :
    Prop where
  out : Round ((wayBelowNbhdBasis (D := D)).domainOrderIso.symm e)

end DomainTheory.BridgeTheorems
```

6.6.3 The type of round information-system elements

Definition (`RoundInfoSysElement`). A round information-system element is an element of the way-below information system equipped with a roundness witness.

RoundInfoSysElement.lean

```
module

public import DomainTheory.BridgeTheorems.PresentationDomains.IsRoundInfoSysElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

variable {D : Type*} [CompleteLattice D] [DecidableEq D]

abbrev RoundInfoSysElement : Type _ :=
  { e : (wayBelowNbhdBasis (D := D)).toInfoSys.Element // IsRoundInfoSysElement (D :=
    D) e }

end DomainTheory.BridgeTheorems
```

6.6.4 Round filters and round information-system elements

Definition (`roundFilter_infoSys_iso`). Round filters of way-below opens are order-isomorphic to round elements of the induced information system, via the neighbourhood-basis correspondence.

roundFilter_infoSys_iso.lean

```
module

public import DomainTheory.BridgeTheorems.PresentationDomains.RoundInfoSysElement
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.RoundFilter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D] [DecidableEq D]

noncomputable def roundFilter_infoSys_iso :
    RoundFilter (D := D) ≃o RoundInfoSysElement (D := D) where
  toFun := fun f =>
    ⟨(wayBelowNbhdBasis (D := D)).domainOrderIso f.1, by
```

```
      let f' : (wayBelowNbhdBasis (D := D)).system.Element := f.1
      refine ⟨?_⟩
      change Round ((wayBelowNbhdBasis (D := D)).domainOrderIso.symm
        ((wayBelowNbhdBasis (D := D)).domainOrderIso f'))
      rw [OrderIso.symm_apply_apply]
      exact f.2⟩
  invFun := fun e =>
    ⟨(wayBelowNbhdBasis (D := D)).domainOrderIso.symm e.1, e.2.out⟩
  left_inv := fun f => Subtype.ext <|
    (wayBelowNbhdBasis (D := D)).domainOrderIso.left_inv f.1
  right_inv := fun e => Subtype.ext <|
    (wayBelowNbhdBasis (D := D)).domainOrderIso.right_inv e.1
  map_rel_iff' := by
    intro f g
    exact (wayBelowNbhdBasis (D := D)).domainOrderIso.map_rel_iff

end DomainTheory.BridgeTheorems
```

6.6.5 Passing from round filters to round information-system elements

Definition (`continuousLattice_roundInfoSys_iso`). Composing the continuous-lattice/round-filter isomorphism with the round-filter/round-element isomorphism yields an order isomorphism between a continuous lattice and its round information-system elements.

`continuousLattice_roundInfoSys_iso.lean`

```
module

public import DomainTheory.BridgeTheorems.PresentationDomains.roundFilter_infoSys_iso
public import DomainTheory.BridgeTheorems.PresentationDomains.
    continuousLattice_roundFilter_iso

@[expose] public section

namespace DomainTheory.BridgeTheorems

variable {D : Type*} [CompleteLattice D] [DecidableEq D]

noncomputable def continuousLattice_roundInfoSys_iso (hD : ContinuousLattice D) :
    D ≃o RoundInfoSysElement (D := D) :=
  (continuousLattice_roundFilter_iso hD).trans roundFilter_infoSys_iso

end DomainTheory.BridgeTheorems
```

6.6.6 Round information-system elements and their ideals

Definition (`roundInfoSys_ideal_iso`). Round information-system elements are order-isomorphic to ideals of finite elements of the way-below information system, by restricting the ideal-completion isomorphism.

`roundInfoSys_ideal_iso.lean`

```
module

public import DomainTheory.BridgeTheorems.PresentationDomains.RoundInfoSysElement
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.domainOrderIso

@[expose] public section

namespace DomainTheory.BridgeTheorems

open Order

variable {D : Type*} [CompleteLattice D] [DecidableEq D]

noncomputable def roundInfoSys_ideal_iso :
    RoundInfoSysElement (D := D) ≃o
      { I : Ideal (InfoSysToIdealCompletion.FiniteElement
          (wayBelowNbhdBasis (D := D)).toInfoSys) //
        IsRoundInfoSysElement (D := D)
          ((InfoSysToIdealCompletion.domainOrderIso
            (wayBelowNbhdBasis (D := D)).toInfoSys).symm I) } := by
  let A := (wayBelowNbhdBasis (D := D)).toInfoSys
  let ιE := InfoSysToIdealCompletion.domainOrderIso A
  refine {
    toFun := fun e => ⟨ιE e.1, by
      change IsRoundInfoSysElement (ιE.symm (ιE e.1))
      rw [OrderIso.symm_apply_apply]
      exact e.2⟩
    invFun := fun I => ⟨ιE.symm I.1, I.2⟩
    left_inv := fun e => Subtype.ext (ιE.left_inv e.1)
    right_inv := fun I => Subtype.ext (ιE.right_inv I.1)
    map_rel_iff' := by
      intro e₁ e₂
      exact ιE.map_rel_iff
  }

end DomainTheory.BridgeTheorems
```

6.6.7 A continuous lattice and its round information system

Definition (`presentation_domains_equiv`). The three-way identification packages a continuous lattice as order-isomorphic to its round information-system elements.

`presentation_domains_equiv.lean`

```
module

public import DomainTheory.BridgeTheorems.PresentationDomains.
    continuousLattice_roundInfoSys_iso

@[expose] public section

namespace DomainTheory.BridgeTheorems

variable {D : Type*} [CompleteLattice D] [DecidableEq D]
```

```
noncomputable def presentation_domains_equiv (hD : ContinuousLattice D) :
    D ≃o RoundInfoSysElement (D := D) :=
  continuousLattice_roundInfoSys_iso hD

end DomainTheory.BridgeTheorems
```

6.6.8 A continuous lattice and the ideals of its round elements

Definition (`presentation_domains_equiv_ideal`). Composing further with the ideal isomorphism identifies a continuous lattice with ideals of finite elements of its round information system.

`presentation_domains_equiv_ideal.lean`

```
module

public import DomainTheory.BridgeTheorems.PresentationDomains.roundInfoSys_ideal_iso
public import DomainTheory.BridgeTheorems.PresentationDomains.
    presentation_domains_equiv

@[expose] public section

namespace DomainTheory.BridgeTheorems

open Order

variable {D : Type*} [CompleteLattice D] [DecidableEq D]

noncomputable def presentation_domains_equiv_ideal (hD : ContinuousLattice D) :
    D ≃o
      { I : Ideal (InfoSysToIdealCompletion.FiniteElement
          (wayBelowNbhdBasis (D := D)).toInfoSys) //
        IsRoundInfoSysElement (D := D)
          ((InfoSysToIdealCompletion.domainOrderIso
            (wayBelowNbhdBasis (D := D)).toInfoSys).symm I) } :=
  (presentation_domains_equiv hD).trans (roundInfoSys_ideal_iso (D := D))

end DomainTheory.BridgeTheorems
```

6.6.9 Neighbourhood-basis elements and ideals

Definition (`nbhdBasis_ideal_iso`). For any neighbourhood basis, filters of the neighbourhood system are order-isomorphic to ideals of finite elements of the induced information system, by composing the two domain isomorphisms.

`nbhdBasis_ideal_iso.lean`

```
module
public import DomainTheory.BridgeTheorems.NeighborhoodToInfoSys.domainOrderIso
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.domainOrderIso
public import Mathlib.Order.Hom.Basic
public import DomainTheory.BridgeTheorems.InfoSysToNeighborhood
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion
public import DomainTheory.BridgeTheorems.NeighborhoodToInfoSys
```

```
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood
public import DomainTheory.BridgeTheorems.IdealCompletionToContinuousLattice

@[expose] public section

namespace DomainTheory.BridgeTheorems

open Order

section InfoSysTriangle

variable {α : Type*} [DecidableEq α]

variable {ι β : Type*} [DecidableEq ι]

noncomputable def nbhdBasis_ideal_iso (B : NbhdBasis ι β) :
    B.system.Element ≃o Ideal (InfoSysToIdealCompletion.FiniteElement B.toInfoSys) :=
  B.domainOrderIso.trans (InfoSysToIdealCompletion.domainOrderIso B.toInfoSys)

end InfoSysTriangle

end DomainTheory.BridgeTheorems
```

6.6.10 Neighbourhood filters and ideals

Definition (`neighborhood_ideal_iso`). For any information system, filters of its basic-open neighbourhood system are order-isomorphic to ideals of its finite elements.

neighborhood_ideal_iso.lean

```
module
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion.domainOrderIso
public import DomainTheory.BridgeTheorems.InfoSysToNeighborhood.domainOrderIso
public import Mathlib.Order.Hom.Basic
public import DomainTheory.BridgeTheorems.InfoSysToNeighborhood
public import DomainTheory.BridgeTheorems.InfoSysToIdealCompletion
public import DomainTheory.BridgeTheorems.NeighborhoodToInfoSys
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood
public import DomainTheory.BridgeTheorems.IdealCompletionToContinuousLattice

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open Order

section InfoSysTriangle

variable {α : Type*} [DecidableEq α]

noncomputable def neighborhood_ideal_iso (A : InfoSys α) :
    (InfoSysToNeighborhood.toNeighborhoodSystem A).Element ≃o
      Ideal (InfoSysToIdealCompletion.FiniteElement A) :=
```

```
  (InfoSysToNeighborhood.domainOrderIso A).symm.trans
    (InfoSysToIdealCompletion.domainOrderIso A)

end InfoSysTriangle

end DomainTheory.BridgeTheorems
```

6.6.11 Existence of the round identification

Definition (`presentation_domains_equiv_infoSys`). The packaged presentation equivalence associated with an information system, obtained by transporting along the bridges to neighbourhood filters and ideals.

Theorem (`exists_presentation_domains_equiv`). Every continuous lattice is order-isomorphic to the round elements of its way-below information system.

Proof. The packaged presentation equivalence supplies such an isomorphism. ■

`Basic.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.PresentationDomains.neighborhood_ideal_iso
public import DomainTheory.BridgeTheorems.PresentationDomains.RoundInfoSysElement
public import DomainTheory.BridgeTheorems.PresentationDomains.
    presentation_domains_equiv

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

section InfoSysTriangle
variable {α : Type*} [DecidableEq α]

variable {ι β : Type*} [DecidableEq ι]

noncomputable abbrev presentation_domains_equiv_infoSys (A : InfoSys α) :=
  neighborhood_ideal_iso A

end InfoSysTriangle

variable {D : Type*} [CompleteLattice D] [DecidableEq D]

theorem exists_presentation_domains_equiv (hD : ContinuousLattice D) :
    Nonempty (D ≃o RoundInfoSysElement (D := D)) :=
  ⟨presentation_domains_equiv hD⟩
end DomainTheory.BridgeTheorems
```

6.6.12 The way-below basis determines a neighbourhood system

Theorem (`wayBelowNbhdBasis_system`). The neighbourhood system of the way-below neighbourhood basis coincides with the way-below neighbourhood system of the continuous lattice.

Proof. Both systems have precisely the way-below opens as neighbourhoods. ■

`wayBelowNbhdBasis_system.lean`

```
module
public import DomainTheory.BridgeTheorems.PresentationDomains.wayBelowNbhdBasis

@[expose] public section

namespace DomainTheory.BridgeTheorems

open ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D] [DecidableEq D]

theorem wayBelowNbhdBasis_system :
    (wayBelowNbhdBasis (D := D)).system = toNeighborhoodSystem :=
  rfl

end DomainTheory.BridgeTheorems
```

6.7 Products, sums, and the function space

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

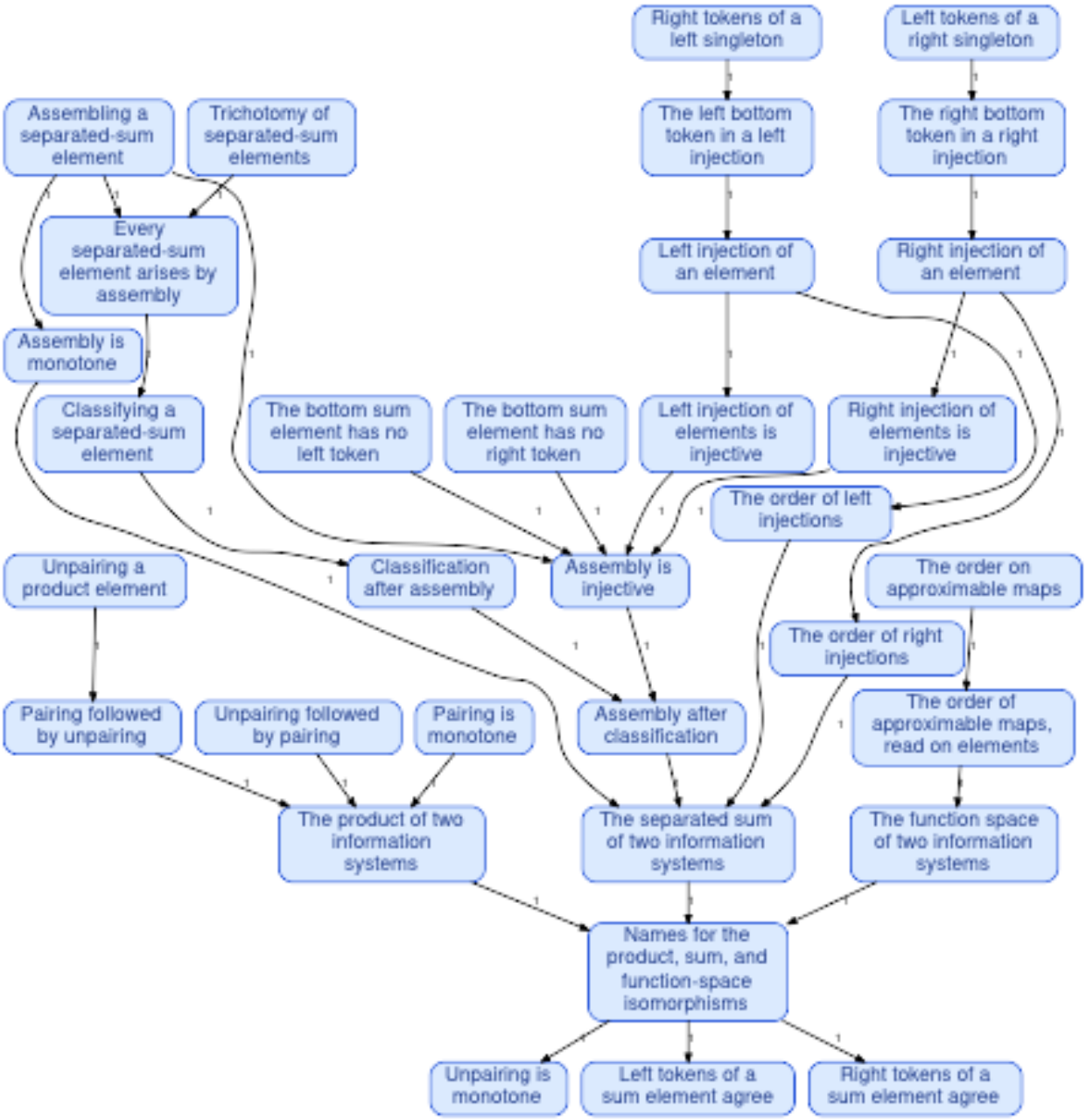


Figure 70: Diagram 35

6.7.1 Unpairing a product element

Definition (`unpair`). From an element of a product information system one recovers the pair of projected elements on the two factors.

`unpair.lean`

```
module
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    productSystem
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

def unpair (z : (productSystem A B).Element) : A.Element × B.Element :=
  ((productSystem.fstMap A B).toElement z, (productSystem.sndMap A B).toElement z)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.2 Pairing followed by unpairing

Theorem (`unpair_pairElements`). Unpairing the paired elements of a pair recovers that pair.

Proof. The projections of a paired element are the original components. ■

unpair_pairElements.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.unpair
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    pairElements

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem unpair_pairElements (x : A.Element) (y : B.Element) :
    unpair A B (pairElements A B x y) = (x, y) := by
  simp [unpair, fstMap_pairElements, sndMap_pairElements]

end InfoSysConstructions
```

```
end DomainTheory.BridgeTheorems
```

6.7.3 Unpairing followed by pairing

Theorem (`pairElements_unpair`). Pairing the components obtained by unpairing recovers the original product element.

Proof. A product element is determined by its two projections. ■

pairElements_unpair.lean

```
module
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    pairElements
import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    productSystem_more

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem pairElements_unpair (z : (productSystem A B).Element) :
    pairElements A B ((productSystem.fstMap A B).toElement z) ((productSystem.sndMap A
     B).toElement z) = z :=
  productSystem.element_eq_of_fst_snd A B _ z (fstMap_pairElements A B _ _) (
    sndMap_pairElements A B _ _)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.4 Pairing is monotone

Theorem (`pairElements_mono`). Pairing of elements is monotone in each component.

Proof. The product order is componentwise. ■

pairElements_mono.lean

```
module
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    pairElements

@[expose] public section
```

```
namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem pairElements_mono {x₁ x₂ : A.Element} {y₁ y₂ : B.Element}
    (hx : x₁ ≤ x₂) (hy : y₁ ≤ y₂) :
    pairElements A B x₁ y₁ ≤ pairElements A B x₂ y₂ := by
  intro p hp
  exact ⟨fun hbot => hx (hp.1 hbot), fun hbot => hy (hp.2 hbot)⟩

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.5 The order on approximable maps

Definition (`instPartialOrderApproximableMap`). Approximable maps between information systems are partially ordered by pointwise inclusion of their graphs.

`instPartialOrderApproximableMap.lean`

```
module
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ApproximableMap

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

instance instPartialOrderApproximableMap : PartialOrder (ApproximableMap A B) where
  le := @Le _ _ _ _ A B
  le_refl _ _ _ h := h
  le_trans _ _ _ hfg hgh _ _ hf := hgh (hfg hf)
  le_antisymm _ _ hfg hgf := ApproximableMap.ext fun _ _ => ⟨fun h => hfg h, fun h =>
    hgf h⟩

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.6 The order of approximable maps, read on elements

Theorem (`approxMap_toElement_le_iff`). One approximable map lies below another if and only if the corresponding function-space elements do.

Proof. The order on approximable maps is transported along the identification with function-space elements. ■

approxMap_toElement_le_iff.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    instPartialOrderApproximableMap
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    approxMap_toElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem approxMap_toElement_le_iff {f g : ApproximableMap A B} :
    approxMap_toElement A B f ≤ approxMap_toElement A B g ↔ f ≤ g := by
  constructor
  · intro h u v hrel
    have hp : mkFunToken A B u v (f.rel_dom hrel) (f.rel_cod hrel) ∈
        (approxMap_toElement A B f).carrier :=
      (mem_approxMap_toElement A B f).2 hrel
    exact (mem_approxMap_toElement A B g).1 (h hp)
  · intro hfg p hp
    exact (mem_approxMap_toElement A B g).2 (hfg ((mem_approxMap_toElement A B f).1 hp
    ))

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.7 Assembling a separated-sum element

Definition (`assembleSum`). From an optional left or right element one assembles an element of the separated-sum information system, sending the absent case to the bottom sum element.

assembleSum.lean

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inrMap
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inlMap
```

```
public import Mathlib.Order.TypeTags

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

def assembleSum : WithBot (A.Element ? B.Element) → (sumSystem A B).Element
  | ⊥ => (sumSystem A B).botElement
  | some (.inl x) => (inlMap A B).toElement x
  | some (.inr y) => (inrMap A B).toElement y

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.8 Assembly is monotone

Theorem (`assembleSum_mono`). Assembly of separated-sum elements is monotone.

Proof. The separated-sum order compares left injections with left injections, right with right, and places bottom below everything. ■

`assembleSum_mono.lean`

```
module

public import Mathlib.Order.Sum.Order
public import DomainTheory.BridgeTheorems.InfoSysConstructions.assembleSum

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem assembleSum_mono {w₁ w₂ : WithBot (A.Element ? B.Element)} (h : w₁ ≤ w₂) :
    assembleSum A B w₁ ≤ assembleSum A B w₂ := by
```

```
  cases w₁ with
  | bot => exact botElement.le _ _
  | coe s₁ =>
    cases w₂ with
    | bot => exact (WithBot.not_coe_le_bot _ h).elim
    | coe s₂ =>
      have hs : s₁ ≤ s₂ := WithBot.coe_le_coe.1 h
      cases s₁ with
      | inl x =>
        cases s₂ with
        | inl x' => exact (inlMap A B).toElement_mono (Sum.inl_le_inl_iff.1 hs)
        | inr y' => exact (Sum.not_inl_le_inr hs).elim
      | inr y =>
        cases s₂ with
        | inl x' => exact (Sum.not_inr_le_inl hs).elim
        | inr y' => exact (inrMap A B).toElement_mono (Sum.inr_le_inr_iff.1 hs)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.9 Trichotomy of separated-sum elements

Theorem (`sum_element_trichotomy`). Every element of a separated sum is either the bottom element, purely left, or purely right.

Proof. Tokens of a sum element cannot mix left and right polarities without collapsing to bottom. ■

`sum_element_trichotomy.lean`

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inrMap
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inlMap

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem sum_element_trichotomy (z : (sumSystem A B).Element) :
    z = (sumSystem A B).botElement ∨
      (∃ x : A.Element, z = (inlMap A B).toElement x) ∨
        (∃ y : B.Element, z = (inrMap A B).toElement y) := by
  classical
  by_cases hL : ∃ x : α, SumToken.left x ∈ z.carrier
  · obtain ⟨x0, hx0⟩ := hL
```

```
    exact Or.inr (Or.inl ⟨sumElementLft A B z x0 hx0, (inlMap.toElement_sumElementLft A
     B z x0 hx0).symm⟩)
  · by_cases hR : ∃ y : β, SumToken.right y ∈ z.carrier
    · obtain ⟨y0, hy0⟩ := hR
      exact Or.inr (Or.inr ⟨sumElementRht A B z y0 hy0, (inrMap.toElement_sumElementRht
     A B z y0 hy0).symm⟩)
    · exact Or.inl (eq_botElement_of_no_injections A B z
        (fun x hx => hL ⟨x, hx⟩) (fun y hy => hR ⟨y, hy⟩))

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.10 Every separated-sum element arises by assembly

Theorem (`exists_assembleSum_eq`). Every separated-sum element arises by assembling an optional left or right element.

Proof. Trichotomy supplies the case distinction; the left and right cases recover the projected components. ■

`exists_assembleSum_eq.lean`

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.assembleSum
public import DomainTheory.BridgeTheorems.InfoSysConstructions.sum_element_trichotomy

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem exists_assembleSum_eq (z : (sumSystem A B).Element) :
    ∃ w : WithBot (A.Element ⊕ B.Element), assembleSum A B w = z := by
  rcases sum_element_trichotomy A B z with hbot | ⟨x, hx⟩ | ⟨y, hy⟩
  · exact ⟨⊥, hbot.symm⟩
  · exact ⟨some (.inl x), hx.symm⟩
  · exact ⟨some (.inr y), hy.symm⟩

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.11 Classifying a separated-sum element

Definition (`classifySum`). Classification sends a separated-sum element to the optional left or right element from which it assembles.

classifySum.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.exists_assembleSum_eq

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

noncomputable def classifySum (z : (sumSystem A B).Element) :
    WithBot (A.Element ? B.Element) :=
  Classical.choose (exists_assembleSum_eq A B z)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.12 Classification after assembly

Theorem (`assembleSum_classifySum`). Assembling after classifying recovers the original separated-sum element.

Proof. Classification inverts assembly on its image, which is everything. ■

assembleSum_classifySum.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.classifySum

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)
```

```
theorem assembleSum_classifySum (z : (sumSystem A B).Element) :
    assembleSum A B (classifySum A B z) = z :=
  Classical.choose_spec (exists_assembleSum_eq A B z)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.13 The bottom sum element has no left token

Theorem (`not_left_mem_sum_botElement`). The bottom element of a separated sum contains no left token.

Proof. The bottom carrier is empty of proper sum tokens. ■

`not_left_mem_sum_botElement.lean`

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftFinset
public import DomainTheory.InformationSystems.ElementsOfASystem.botElement
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.Sum.Order
public import Mathlib.Order.WithBot
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.theorem_7_2
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.proposition_6_4

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem not_left_mem_sum_botElement {x : α} :
    SumToken.left x ∈ ((sumSystem A B).botElement).carrier := by
  intro hx
  have hEnt : (sumSystem A B).Ent {sumBot} (.left x) := hx

  rcases hEnt with ⟨_, ⟨hne, _⟩⟩
  exact hne lftFinset.singleton_bot

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.14 The bottom sum element has no right token

Theorem (`not_right_mem_sum_botElement`). The bottom element of a separated sum contains no right token.

Proof. The bottom carrier is empty of proper sum tokens. ■

`not_right_mem_sum_botElement.lean`

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtFinset
public import DomainTheory.InformationSystems.ElementsOfASystem.botElement
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.Sum.Order
public import Mathlib.Order.WithBot
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.theorem_7_2
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.proposition_6_4

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem not_right_mem_sum_botElement {y : β} :
    SumToken.right y ∈ ((sumSystem A B).botElement).carrier := by
  intro hy
  have hEnt : (sumSystem A B).Ent {sumBot} (.right y) := hy
  rcases hEnt with ⟨_, ⟨hne, _⟩⟩
  exact hne rhtFinset.singleton_bot

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.15 Left tokens of a right singleton

Theorem (`lftFinset_singleton_right`). Extracting left tokens from a singleton right token yields the empty set.

Proof. A right token has no left polarity. ■

`lftFinset_singleton_right.lean`

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftFinset
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.Sum.Order
public import Mathlib.Order.WithBot
```

```
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.theorem_7_2
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.proposition_6_4

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

omit [DecidableEq β] in

theorem lftFinset_singleton_right (y : β) :
    lftFinset ({SumToken.right y} : Finset (SumToken α β)) = ∅ := by
  ext x
  constructor
  · intro hx
    have : SumToken.left x ∈ ({SumToken.right y} : Finset _) := (mem_lftFinset).1 hx
    exact False.elim (nomatch Finset.mem_singleton.mp this)
  · intro hx
    exact False.elim (Finset.notMem_empty x hx)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.16 Right tokens of a left singleton

Theorem (`rhtFinset_singleton_left`). Extracting right tokens from a singleton left token yields the empty set.

Proof. A left token has no right polarity. ■

`rhtFinset_singleton_left.lean`

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtFinset
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.Sum.Order
public import Mathlib.Order.WithBot
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.theorem_7_2
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.proposition_6_4

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
```

```
namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

omit [DecidableEq α] in

theorem rhtFinset_singleton_left (x : α) :
    rhtFinset ({SumToken.left x} : Finset (SumToken α β)) = ∅ := by
  ext y
  constructor
  · intro hy
    have : SumToken.right y ∈ ({SumToken.left x} : Finset _) := (mem_rhtFinset).1 hy
    exact False.elim (nomatch Finset.mem_singleton.mp this)
  · intro hy
    exact False.elim (Finset.notMem_empty y hy)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.17 The left bottom token in a left injection

Theorem (`left_bot_mem_inlMap_toElement`). The left injection of an element contains the left bottom token.

Proof. Left injection sends the bottom of the left system to the left bottom token. ■

`left_bot_mem_inlMap_toElement.lean`

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inlMap
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    rhtFinset_singleton_left
import DomainTheory.InformationSystems.ElementsOfASystem.Factoid32.factoid_3_2

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem left_bot_mem_inlMap_toElement (x : A.Element) :
    SumToken.left A.bot ∈ ((inlMap A B).toElement x).carrier := by
  refine ⟨{A.bot}, ?_, ?_⟩
  · intro a ha
    have : a = A.bot := Finset.mem_singleton.mp (Finset.mem_coe.1 ha)
```

```
    subst this
    exact factoid_3_2 A x
  · refine ⟨A.con_sing A.bot, ?_, rhtFinset_singleton_left (β := β) A.bot, ?_⟩
    · exact Or.inl ⟨by rw [lftFinset.singleton_left]; exact A.con_sing A.bot,
        rhtFinset_singleton_left (β := β) A.bot⟩
    · rw [lftFinset.singleton_left]
      exact EntSet.refl A (A.con_sing A.bot)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.18 The right bottom token in a right injection

Theorem (`right_bot_mem_inrMap_toElement`). The right injection of an element contains the right bottom token.

Proof. Right injection sends the bottom of the right system to the right bottom token. ■

`right_bot_mem_inrMap_toElement.lean`

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inrMap
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    lftFinset_singleton_right
import DomainTheory.InformationSystems.ElementsOfASystem.Factoid32.factoid_3_2

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem right_bot_mem_inrMap_toElement (y : B.Element) :
    SumToken.right B.bot ∈ ((inrMap A B).toElement y).carrier := by
  refine ⟨{B.bot}, ?_, ?_⟩
  · intro b hb
    have : b = B.bot := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    subst this
    exact factoid_3_2 B y
  · refine ⟨B.con_sing B.bot, ?_, lftFinset_singleton_right (α := α) B.bot, ?_⟩
    · exact Or.inr ⟨lftFinset_singleton_right (α := α) B.bot,
        by rw [rhtFinset.singleton_right]; exact B.con_sing B.bot⟩
    · rw [rhtFinset.singleton_right]
      exact EntSet.refl B (B.con_sing B.bot)

end InfoSysConstructions
```

```
end DomainTheory.BridgeTheorems
```

6.7.19 Left injection of an element

Theorem (`sumElementLft_inlMap_toElement`). Projecting the left tokens of a left injection recovers the original left element.

Proof. Left injection embeds left tokens faithfully. ■

sumElementLft_inlMap_toElement.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    left_bot_mem_inlMap_toElement
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    rhtFinset_singleton_left

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem sumElementLft_inlMap_toElement (x : A.Element) :
    sumElementLft A B ((inlMap A B).toElement x) A.bot
      (left_bot_mem_inlMap_toElement A B x) = x := by
  apply le_antisymm
  · intro a ha

    obtain ⟨u, hu, ⟨huCon, hSum, hr, hEnt⟩⟩ := ha
    have hEntA : A.Ent u a := by
      have : lftFinset ({SumToken.left a} : Finset (SumToken α β)) = {a} :=
        lftFinset.singleton_left a
      simpa [this] using hEnt a (Finset.mem_singleton_self a)
    exact x.closed u a hu hEntA
  · intro a ha
    change SumToken.left a ∈ ((inlMap A B).toElement x).carrier
    refine ⟨{a}, ?_, ?_⟩
    · intro b hb
      have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
      subst this; exact ha
    · refine ⟨A.con_sing a, Or.inl ⟨by rw [lftFinset.singleton_left]; exact A.con_sing a
    ,
        rhtFinset_singleton_left (β := β) a⟩, rhtFinset_singleton_left (β := β) a, ?_⟩
      rw [lftFinset.singleton_left]
```

```
      exact EntSet.refl A (A.con_sing a)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.20 Right injection of an element

Theorem (`sumElementRht_inrMap_toElement`). Projecting the right tokens of a right injection recovers the original right element.

Proof. Right injection embeds right tokens faithfully. ■

sumElementRht_inrMap_toElement.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    right_bot_mem_inrMap_toElement
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    lftFinset_singleton_right

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem sumElementRht_inrMap_toElement (y : B.Element) :
    sumElementRht A B ((inrMap A B).toElement y) B.bot
      (right_bot_mem_inrMap_toElement A B y) = y := by
  apply le_antisymm
  · intro b hb
    obtain ⟨w, hw, ⟨hwCon, hSum, hl, hEnt⟩⟩ := hb
    have hEntB : B.Ent w b := by
      have : rhtFinset ({SumToken.right b} : Finset (SumToken α β)) = {b} :=
        rhtFinset.singleton_right b
      simpa [this] using hEnt b (Finset.mem_singleton_self b)
    exact y.closed w b hw hEntB
  · intro b hb
    change SumToken.right b ∈ ((inrMap A B).toElement y).carrier
    refine ⟨{b}, ?_, ?_⟩
    · intro c hc
      have : c = b := Finset.mem_singleton.mp (Finset.mem_coe.1 hc)
      subst this; exact hb
    · refine ⟨B.con_sing b, Or.inr ⟨lftFinset_singleton_right (α := α) b,
        by rw [rhtFinset.singleton_right]; exact B.con_sing b⟩,
```

```
        lftFinset_singleton_right (α := α) b, ?_⟩
      rw [rhtFinset.singleton_right]
      exact EntSet.refl B (B.con_sing b)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.21 Left injection of elements is injective

Theorem (`inlMap_toElement_injective`). Left injection of elements is injective.

Proof. Left projection recovers the original element. ■

`inlMap_toElement_injective.lean`

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inlMap
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    sumElementLft_inlMap_toElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem inlMap_toElement_injective :
    Function.Injective (inlMap A B).toElement := by
  intro x y h
  refine le_antisymm ?_ ?_
  · intro a ha
    have hx := sumElementLft_inlMap_toElement A B x
    have : SumToken.left a ∈ ((inlMap A B).toElement x).carrier := by
      rw [← hx] at ha; exact ha
    rw [h] at this
    have hy := sumElementLft_inlMap_toElement A B y
    have : a ∈ (sumElementLft A B ((inlMap A B).toElement y) A.bot
        (left_bot_mem_inlMap_toElement A B y)).carrier := this
    rwa [hy] at this
  · intro a ha
    have hy := sumElementLft_inlMap_toElement A B y
    have : SumToken.left a ∈ ((inlMap A B).toElement y).carrier := by
      rw [← hy] at ha; exact ha
    rw [← h] at this
    have hx := sumElementLft_inlMap_toElement A B x
    have : a ∈ (sumElementLft A B ((inlMap A B).toElement x) A.bot
        (left_bot_mem_inlMap_toElement A B x)).carrier := this
```

```
    rwa [hx] at this

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.22 Right injection of elements is injective

Theorem (`inrMap_toElement_injective`). Right injection of elements is injective.

Proof. Right projection recovers the original element. ■

`inrMap_toElement_injective.lean`

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inrMap
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    sumElementRht_inrMap_toElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem inrMap_toElement_injective :
    Function.Injective (inrMap A B).toElement := by
  intro x y h
  refine le_antisymm ?_ ?_
  · intro b hb
    have hx := sumElementRht_inrMap_toElement A B x
    have : SumToken.right b ∈ ((inrMap A B).toElement x).carrier := by
      rw [← hx] at hb; exact hb
    rw [h] at this
    have hy := sumElementRht_inrMap_toElement A B y
    have : b ∈ (sumElementRht A B ((inrMap A B).toElement y) B.bot
        (right_bot_mem_inrMap_toElement A B y)).carrier := this
    rwa [hy] at this
  · intro b hb
    have hy := sumElementRht_inrMap_toElement A B y
    have : SumToken.right b ∈ ((inrMap A B).toElement y).carrier := by
      rw [← hy] at hb; exact hb
    rw [← h] at this
    have hx := sumElementRht_inrMap_toElement A B x
    have : b ∈ (sumElementRht A B ((inrMap A B).toElement x) B.bot
        (right_bot_mem_inrMap_toElement A B x)).carrier := this
    rwa [hx] at this
```

```
end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.23 Assembly is injective

Theorem (`assembleSum_injective`). Assembly of optional left or right elements is injective.

Proof. Classification recovers the optional component. ■

`assembleSum_injective.lean`

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.assembleSum
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    inlMap_toElement_injective
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    inrMap_toElement_injective
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    left_bot_mem_inlMap_toElement
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    not_left_mem_sum_botElement
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    not_right_mem_sum_botElement
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    right_bot_mem_inrMap_toElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem assembleSum_injective :
    Function.Injective (assembleSum A B) := by
  intro w₁ w₂ h
  cases w₁ with
  | bot =>
    cases w₂ with
    | bot => rfl
    | coe s₂ =>
      cases s₂ with
      | inl x =>
        change (sumSystem A B).botElement = (inlMap A B).toElement x at h
        have hx := left_bot_mem_inlMap_toElement A B x
        rw [← h] at hx
        exact False.elim (not_left_mem_sum_botElement A B hx)
```

```
      | inr y =>
        change (sumSystem A B).botElement = (inrMap A B).toElement y at h
        have hy := right_bot_mem_inrMap_toElement A B y
        rw [← h] at hy
        exact False.elim (not_right_mem_sum_botElement A B hy)
  | coe s₁ =>
    cases w₂ with
    | bot =>
      cases s₁ with
      | inl x =>
        change (inlMap A B).toElement x = (sumSystem A B).botElement at h
        have hx := left_bot_mem_inlMap_toElement A B x
        rw [h] at hx
        exact False.elim (not_left_mem_sum_botElement A B hx)
      | inr y =>
        change (inrMap A B).toElement y = (sumSystem A B).botElement at h
        have hy := right_bot_mem_inrMap_toElement A B y
        rw [h] at hy
        exact False.elim (not_right_mem_sum_botElement A B hy)
    | coe s₂ =>
      cases s₁ with
      | inl x =>
        cases s₂ with
        | inl x' => exact congrArg (fun t => some (Sum.inl t)) (
    inlMap_toElement_injective A B h)
        | inr y' =>
          change (inlMap A B).toElement x = (inrMap A B).toElement y' at h
          have hx := left_bot_mem_inlMap_toElement A B x
          rw [h] at hx
          exact False.elim <| not_mem_right_of_mem_left A B ((inrMap A B).toElement y
    ')
            hx
            (right_bot_mem_inrMap_toElement A B y')
      | inr y =>
        cases s₂ with
        | inl x' =>
          change (inrMap A B).toElement y = (inlMap A B).toElement x' at h
          have hy := right_bot_mem_inrMap_toElement A B y
          rw [h] at hy
          exact False.elim <| not_mem_right_of_mem_left A B ((inlMap A B).toElement x
    ')
            (left_bot_mem_inlMap_toElement A B x')
            hy
        | inr y' => exact congrArg (fun t => some (Sum.inr t)) (
    inrMap_toElement_injective A B h)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.24 Assembly after classification

Theorem (`classifySum_assembleSum`). Classifying after assembling recovers the original optional component.

Proof. Classification is a left inverse of assembly. ■

classifySum_assembleSum.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.classifySum
public import DomainTheory.BridgeTheorems.InfoSysConstructions.assembleSum_classifySum
public import DomainTheory.BridgeTheorems.InfoSysConstructions.assembleSum_injective

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem classifySum_assembleSum (w : WithBot (A.Element ? B.Element)) :
    classifySum A B (assembleSum A B w) = w := by
  apply assembleSum_injective A B
  exact assembleSum_classifySum A B (assembleSum A B w)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.25 The order of left injections

Theorem (`inlMap_toElement_le_iff`). Left injections are ordered exactly as their underlying left elements.

Proof. The separated-sum order restricts to the left factor along left injection. ■

inlMap_toElement_le_iff.lean

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inlMap
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    sumElementLft_inlMap_toElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)
```

```
theorem inlMap_toElement_le_iff {x y : A.Element} :
    (inlMap A B).toElement x ≤ (inlMap A B).toElement y ↔ x ≤ y := by
  constructor
  · intro h a ha
    have : SumToken.left a ∈ ((inlMap A B).toElement x).carrier := by

      have hx := sumElementLft_inlMap_toElement A B x

      have : a ∈ (sumElementLft A B ((inlMap A B).toElement x) A.bot
          (left_bot_mem_inlMap_toElement A B x)).carrier := by
        simpa [hx] using ha
      exact this
    exact (sumElementLft_inlMap_toElement A B y) ▶
      (show a ∈ (sumElementLft A B ((inlMap A B).toElement y) A.bot
          (left_bot_mem_inlMap_toElement A B y)).carrier from h this)
  · exact (inlMap A B).toElement_mono

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.26 The order of right injections

Theorem (`inrMap_toElement_le_iff`). Right injections are ordered exactly as their underlying right elements.

Proof. The separated-sum order restricts to the right factor along right injection. ■

`inrMap_toElement_le_iff.lean`

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.inrMap
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    sumElementRht_inrMap_toElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem inrMap_toElement_le_iff {x y : B.Element} :
    (inrMap A B).toElement x ≤ (inrMap A B).toElement y ↔ x ≤ y := by
  constructor
  · intro h b hb
    have : SumToken.right b ∈ ((inrMap A B).toElement x).carrier := by
```

```
      have hx := sumElementRht_inrMap_toElement A B x
      have : b ∈ (sumElementRht A B ((inrMap A B).toElement x) B.bot
          (right_bot_mem_inrMap_toElement A B x)).carrier := by
        simpa [hx] using hb
      exact this
    exact (sumElementRht_inrMap_toElement A B y) ▶
      (show b ∈ (sumElementRht A B ((inrMap A B).toElement y) B.bot
          (right_bot_mem_inrMap_toElement A B y)).carrier from h this)
  · exact (inrMap A B).toElement_mono

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.27 The product of two information systems

Definition (`productDomainIso`). Elements of a product of information systems are order-isomorphic to pairs of elements of the factors, via pairing and unpairing.

productDomainIso.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.unpair_pairElements
public import DomainTheory.BridgeTheorems.InfoSysConstructions.pairElements_unpair
public import DomainTheory.BridgeTheorems.InfoSysConstructions.pairElements_mono

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

noncomputable def productDomainIso :
    A.Element × B.Element ≃o (productSystem A B).Element where
  toFun := fun p => pairElements A B p.1 p.2
  invFun := unpair A B
  left_inv := fun p => unpair_pairElements A B p.1 p.2
  right_inv := pairElements_unpair A B
  map_rel_iff' := by
    intro p q
    constructor
    · intro h
      change pairElements A B p.1 p.2 ≤ pairElements A B q.1 q.2 at h
      constructor
      · have hp := (productSystem.fstMap A B).toElement_mono h
        rw [fstMap_pairElements A B p.1 p.2, fstMap_pairElements A B q.1 q.2] at hp
        exact hp
      · have hp := (productSystem.sndMap A B).toElement_mono h
```

```
        rw [sndMap_pairElements A B p.1 p.2, sndMap_pairElements A B q.1 q.2] at hp
        exact hp
    · intro h
      exact pairElements_mono A B h.1 h.2

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.28 The separated sum of two information systems

Definition (`sumDomainIso`). Elements of a separated sum are order-isomorphic to the lifted disjoint union of the factor domains, via assembly and classification.

sumDomainIso.lean

```
module

public import Mathlib.Order.Sum.Order
public import DomainTheory.BridgeTheorems.InfoSysConstructions.assembleSum_classifySum
public import DomainTheory.BridgeTheorems.InfoSysConstructions.classifySum_assembleSum
public import DomainTheory.BridgeTheorems.InfoSysConstructions.assembleSum_mono
public import DomainTheory.BridgeTheorems.InfoSysConstructions.inlMap_toElement_le_iff
public import DomainTheory.BridgeTheorems.InfoSysConstructions.inrMap_toElement_le_iff
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    left_bot_mem_inlMap_toElement
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    not_left_mem_sum_botElement
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    not_right_mem_sum_botElement
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    right_bot_mem_inrMap_toElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

noncomputable def sumDomainIso :
    WithBot (A.Element ⊕ B.Element) ≃o (sumSystem A B).Element where
  toFun := assembleSum A B
  invFun := classifySum A B
  left_inv := classifySum_assembleSum A B
  right_inv := assembleSum_classifySum A B
  map_rel_iff' := by
    intro w₁ w₂
```

```
    constructor
    · intro h
      cases w₁ with
      | bot => exact bot_le
      | coe s₁ =>
        cases w₂ with
        | bot =>
          cases s₁ with
          | inl x =>
            exact False.elim (not_left_mem_sum_botElement A B
              (h (left_bot_mem_inlMap_toElement A B x)))
          | inr y =>
            exact False.elim (not_right_mem_sum_botElement A B
              (h (right_bot_mem_inrMap_toElement A B y)))
        | coe s₂ =>
          refine WithBot.coe_le_coe.2 ?_
          cases s₁ with
          | inl x =>
            cases s₂ with
            | inl x' => exact Sum.inl_le_inl_iff.2 ((inlMap_toElement_le_iff A B).1 h)
            | inr y' =>
              exact False.elim <|
                not_mem_right_of_mem_left A B ((inrMap A B).toElement y')
                  (h (left_bot_mem_inlMap_toElement A B x))
                  (right_bot_mem_inrMap_toElement A B y')
          | inr y =>
            cases s₂ with
            | inl x' =>
              exact False.elim <|
                not_mem_right_of_mem_left A B ((inlMap A B).toElement x')
                  (left_bot_mem_inlMap_toElement A B x')
                  (h (right_bot_mem_inrMap_toElement A B y))
            | inr y' => exact Sum.inr_le_inr_iff.2 ((inrMap_toElement_le_iff A B).1 h)
    · exact assembleSum_mono A B

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.29 The function space of two information systems

Definition (`functionSpaceDomainIso`). Approximable maps between information systems are order-isomorphic to elements of the function-space information system.

`functionSpaceDomainIso.lean`

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    instPartialOrderApproximableMap
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    approxMap_toElement
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    approxMap_toElement_le_iff
```

```
@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

noncomputable def functionSpaceDomainIso :
    ApproximableMap A B ≃o (functionSystem A B).Element where
  toFun := approxMap_toElement A B
  invFun := element_toApproxMap A B
  left_inv := element_toApproxMap.approxMap_toElement A B
  right_inv := approxMap_toElement_element_toApproxMap A B
  map_rel_iff' := by
    intro f g
    exact approxMap_toElement_le_iff A B

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

6.7.30 Names for the product, sum, and function-space isomorphisms

Definition (`infoSys_product_domain_equiv`). The packaged product domain isomorphism between pairs of elements and elements of the product system.

Definition (`infoSys_sum_domain_equiv`). The packaged separated-sum domain isomorphism between the lifted disjoint union of elements and elements of the sum system.

Definition (`infoSys_function_space_domain_equiv`). The packaged function-space domain isomorphism between approximable maps and elements of the function-space system.

Basic.lean

```
module

public import DomainTheory.BridgeTheorems.InfoSysConstructions.productDomainIso
public import DomainTheory.BridgeTheorems.InfoSysConstructions.sumDomainIso
public import DomainTheory.BridgeTheorems.InfoSysConstructions.functionSpaceDomainIso

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems
open DomainTheory.InformationSystems.InfoSys

noncomputable abbrev infoSys_product_domain_equiv {α β : Type*} [DecidableEq α] [
    DecidableEq β]
    (A : InfoSys α) (B : InfoSys β) :
    A.Element × B.Element ≃o (productSystem A B).Element :=
```

```
  InfoSysConstructions.productDomainIso A B

noncomputable abbrev infoSys_sum_domain_equiv {α β : Type*} [DecidableEq α] [
    DecidableEq β]
    (A : InfoSys α) (B : InfoSys β) :
    WithBot (A.Element ? B.Element) ≃o (sumSystem A B).Element :=
  InfoSysConstructions.sumDomainIso A B

noncomputable abbrev infoSys_function_space_domain_equiv {α β : Type*}
    [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β) :
    ApproximableMap A B ≃o (functionSystem A B).Element :=
  InfoSysConstructions.functionSpaceDomainIso A B
end DomainTheory.BridgeTheorems
```

6.7.31 Unpairing is monotone

Theorem (`unpair_mono`). Unpairing is monotone.

Proof. The two projections of the product order are monotone. ■

`unpair_mono.lean`

```
module
public import DomainTheory.BridgeTheorems.InfoSysConstructions.unpair

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems
open DomainTheory.InformationSystems.InfoSys

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem unpair_mono {z₁ z₂ : (productSystem A B).Element} (hz : z₁ ≤ z₂) :
    unpair A B z₁ ≤ unpair A B z₂ := by
  constructor
  · exact (productSystem.fstMap A B).toElement_mono hz
  · exact (productSystem.sndMap A B).toElement_mono hz

end InfoSysConstructions
end DomainTheory.BridgeTheorems
```

6.7.32 Left tokens of a sum element agree

Theorem (`sumElementLft_eq`). Two presentations of the left projection of a sum element agree.

Proof. Both extract the left tokens of the carrier. ■

`sumElementLft_eq.lean`

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumElement
```

```
@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems
open DomainTheory.InformationSystems.InfoSys
open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem sumElementLft_eq {z : (sumSystem A B).Element} {x0 x1 : α}
    (hx0 : SumToken.left x0 ∈ z.carrier) (hx1 : SumToken.left x1 ∈ z.carrier) :
    sumElementLft A B z x0 hx0 = sumElementLft A B z x1 hx1 := by
  refine le_antisymm ?_ ?_ <;> intro a ha <;> exact ha

end InfoSysConstructions
end DomainTheory.BridgeTheorems
```

6.7.33 Right tokens of a sum element agree

Theorem (`sumElementRht_eq`). Two presentations of the right projection of a sum element agree.

Proof. Both extract the right tokens of the carrier. ■

sumElementRht_eq.lean

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems
open DomainTheory.InformationSystems.InfoSys
open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (A : InfoSys α) (B : InfoSys β)

theorem sumElementRht_eq {z : (sumSystem A B).Element} {y0 y1 : β}
    (hy0 : SumToken.right y0 ∈ z.carrier) (hy1 : SumToken.right y1 ∈ z.carrier) :
    sumElementRht A B z y0 hy0 = sumElementRht A B z y1 hy1 := by
  refine le_antisymm ?_ ?_ <;> intro b hb <;> exact hb

end InfoSysConstructions
end DomainTheory.BridgeTheorems
```

6.8 Scott maps

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>

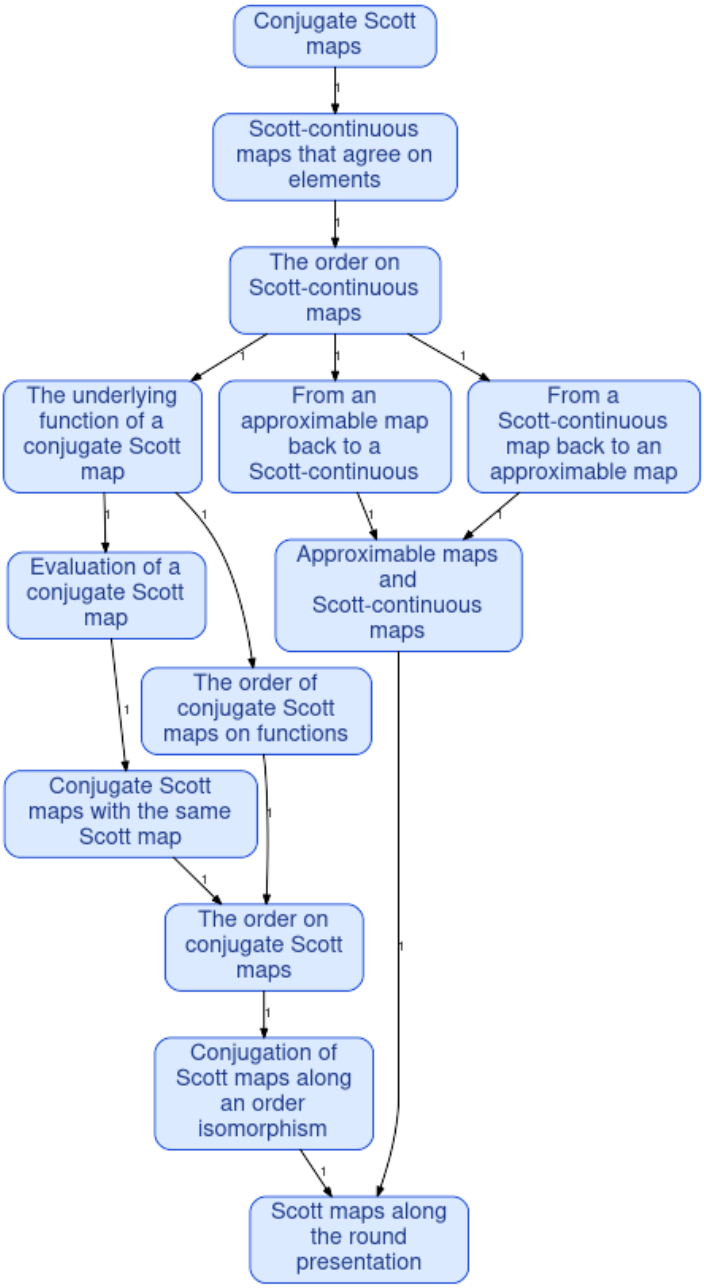


Figure 71: Diagram 36

6.8.1 Conjugate Scott maps

Definition (`ConjScottMap`). A conjugate Scott map along order isomorphisms of the domain and codomain is a Scott map between the original lattices, viewed as a map between the isomorphic presenta-

tions.

ConjScottMap.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_10
public import Mathlib.Order.Hom.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_2
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_5
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_2_10
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_12
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_13
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_14
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    factoid_46
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    proposition_5_3
public import DomainTheory.BridgeTheorems.InfoSysConstructions
public import DomainTheory.BridgeTheorems.PresentationDomains

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable {D E : Type*} [CompleteLattice D] [CompleteLattice E]

structure ConjScottMap {D' E' : Type*} [PartialOrder D'] [PartialOrder E']
    (ιD : D ≃o D') (ιE : E ≃o E') where
  scott : ScottMap D E

end DomainTheory.BridgeTheorems
```

6.8.2 Scott-continuous maps that agree on elements

Theorem (`scottContinuous_ext`). Two Scott-continuous maps of information-system elements that agree at every element are equal.

Proof. Scott-continuous maps are determined by their underlying functions. ■

scottContinuous_ext.lean

```
module
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    ScottContinuous
public import Mathlib.Order.Hom.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_10
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_2
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_5
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_2_10
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_12
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_13
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_14
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    factoid_46
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    proposition_5_3
public import DomainTheory.BridgeTheorems.InfoSysConstructions
public import DomainTheory.BridgeTheorems.PresentationDomains
public import DomainTheory.BridgeTheorems.ConjScottMap

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

theorem scottContinuous_ext {A : InfoSys α} {B : InfoSys β}
    {f g : ScottContinuous A B} (h : ∀ x, f.toFun x = g.toFun x) : f = g := by
  obtain ⟨tf, mf, df⟩ := f
  obtain ⟨tg, mg, dg⟩ := g
  have : tf = tg := funext h
  subst this
  rfl

end DomainTheory.BridgeTheorems
```

6.8.3 The order on Scott-continuous maps

Definition (`instPartialOrderScottContinuous`). Scott-continuous maps between information systems are partially ordered pointwise.

ScottMapBridgeInst9.lean

```
module
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    ScottContinuous
public import DomainTheory.BridgeTheorems.scottContinuous_ext
import Mathlib.Tactic.Continuity.Init

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

instance instPartialOrderScottContinuous (A : InfoSys α) (B : InfoSys β) :
    PartialOrder (ScottContinuous A B) where
  le f g := ∀ x, f.toFun x ≤ g.toFun x
  le_refl _ _ := le_refl _
  le_trans _ _ _ hfg hgh x := le_trans (hfg x) (hgh x)
  le_antisymm _ _ hfg hgf :=
    scottContinuous_ext fun x => le_antisymm (hfg x) (hgf x)

end DomainTheory.BridgeTheorems
```

6.8.4 The underlying function of a conjugate Scott map

Definition (`conjScottMapFun`). The underlying function of a conjugate Scott map is obtained by transporting the Scott map along the domain and codomain order isomorphisms.

conjScottMapFun.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_10
public import Mathlib.Order.Hom.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_2
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_5
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_2_10
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_12
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_13
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_14
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    factoid_46
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    proposition_5_3
```

```
public import DomainTheory.BridgeTheorems.InfoSysConstructions
public import DomainTheory.BridgeTheorems.PresentationDomains
public import DomainTheory.BridgeTheorems.ConjScottMap
public import DomainTheory.BridgeTheorems.ScottMapBridgeInst9

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable {D E : Type*} [CompleteLattice D] [CompleteLattice E]

noncomputable def conjScottMapFun {D' E' : Type*} [LE D'] [LE E']
    (ιD : D ≃o D') (ιE : E ≃o E') (f : ScottMap D E) : D' → E' :=
  ⇑ιE ∘ (f : D → E) ∘ ⇑ιD.symm

end DomainTheory.BridgeTheorems
```

6.8.5 Evaluation of a conjugate Scott map

Definition (`ConjScottMap.instCoeFun`). Conjugate Scott maps coerce to functions between the presented domains via their underlying transported maps.

ScottMapBridgeInst19.lean

```
module

public import DomainTheory.BridgeTheorems.ConjScottMap
public import DomainTheory.BridgeTheorems.conjScottMapFun

@[expose] public section

namespace DomainTheory.BridgeTheorems

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable {D E : Type*} [CompleteLattice D] [CompleteLattice E]

namespace ConjScottMap

variable {D' E' : Type*} [PartialOrder D'] [PartialOrder E']

variable (ιD : D ≃o D') (ιE : E ≃o E')

noncomputable instance : CoeFun (ConjScottMap ιD ιE) (fun _ => D' → E') where
  coe g := conjScottMapFun ιD ιE g.scott

end ConjScottMap

end DomainTheory.BridgeTheorems
```

6.8.6 The order of conjugate Scott maps on functions

Theorem (`conjScottMapFun_le_iff`). One conjugate Scott map lies below another if and only if the underlying Scott maps do, equivalently if and only if the transported functions are pointwise ordered.

Proof. Order isomorphisms preserve and reflect the lattice order. ■

`conjScottMapFun_le_iff.lean`

```
module

public import DomainTheory.BridgeTheorems.conjScottMapFun

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable {D E : Type*} [CompleteLattice D] [CompleteLattice E]

theorem conjScottMapFun_le_iff {D' E' : Type*} [PartialOrder D'] [PartialOrder E']
    (ιD : D ≃o D') (ιE : E ≃o E') {f g : ScottMap D E} :
    (∀ x : D', conjScottMapFun ιD ιE f x ≤ conjScottMapFun ιD ιE g x) ↔ f ≤ g := by
  constructor
  · intro h
    rw [ScottMap.le_def]
    intro x
    have hx : ιE (f x) ≤ ιE (g x) := by
      simpa [conjScottMapFun, OrderIso.symm_apply_apply] using h (ιD x)
    exact ιE.map_rel_iff.mp hx
  · intro hfg x
    exact ιE.monotone (hfg (ιD.symm x))

end DomainTheory.BridgeTheorems
```

6.8.7 Conjugate Scott maps with the same Scott map

Theorem (`ext`). Conjugate Scott maps with the same underlying Scott map are equal.

Proof. A conjugate Scott map is determined by its Scott map component. ■

`ext.lean`

```
module

public import DomainTheory.BridgeTheorems.ConjScottMap

@[expose] public section

namespace DomainTheory.BridgeTheorems

variable {α β : Type*} [DecidableEq α] [DecidableEq β]
```

```
variable {D E : Type*} [CompleteLattice D] [CompleteLattice E]

namespace ConjScottMap

variable {D' E' : Type*} [PartialOrder D'] [PartialOrder E']

variable (ιD : D ≃o D') (ιE : E ≃o E')

theorem ext {f g : ConjScottMap ιD ιE} (h : f.scott = g.scott) : f = g := by
  cases f; cases g; congr

end ConjScottMap

end DomainTheory.BridgeTheorems
```

6.8.8 The order on conjugate Scott maps

Definition (`ConjScottMap.instPartialOrder`). Conjugate Scott maps are partially ordered by the pointwise order of their transported functions.

ScottMapBridgeInst21.lean

```
module

public import DomainTheory.BridgeTheorems.ConjScottMap
public import DomainTheory.BridgeTheorems.conjScottMapFun
public import DomainTheory.BridgeTheorems.conjScottMapFun_le_iff
public import DomainTheory.BridgeTheorems.ext

@[expose] public section

namespace DomainTheory.BridgeTheorems

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable {D E : Type*} [CompleteLattice D] [CompleteLattice E]

namespace ConjScottMap

variable {D' E' : Type*} [PartialOrder D'] [PartialOrder E']

variable (ιD : D ≃o D') (ιE : E ≃o E')

noncomputable instance : PartialOrder (ConjScottMap ιD ιE) where
  le f g := ∀ x, conjScottMapFun ιD ιE f.scott x ≤ conjScottMapFun ιD ιE g.scott x
  le_refl _ _ := le_refl _
  le_trans _ _ _ hfg hgh x := le_trans (hfg x) (hgh x)
  le_antisymm f g hfg hgf := by
    refine ext ιD ιE ?_
    exact le_antisymm
      ((conjScottMapFun_le_iff ιD ιE).1 hfg)
      ((conjScottMapFun_le_iff ιD ιE).1 hgf)

end ConjScottMap
```

```
end DomainTheory.BridgeTheorems
```

6.8.9 Conjugation of Scott maps along an order isomorphism

Definition (`orderIso`). Conjugation along a pair of order isomorphisms is an order isomorphism between Scott maps and conjugate Scott maps.

orderIso.lean

```
module

public import DomainTheory.BridgeTheorems.ScottMapBridgeInst21
public import DomainTheory.BridgeTheorems.conjScottMapFun_le_iff

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

variable {D E : Type*} [CompleteLattice D] [CompleteLattice E]

namespace ConjScottMap

variable {D' E' : Type*} [PartialOrder D'] [PartialOrder E']

variable (ιD : D ≃o D') (ιE : E ≃o E')

noncomputable def orderIso : ScottMap D E ≃o ConjScottMap ιD ιE where
  toFun f := ⟨f⟩
  invFun g := g.scott
  left_inv _ := rfl
  right_inv _ := rfl
  map_rel_iff' := by
    intro f g
    exact conjScottMapFun_le_iff ιD ιE

end ConjScottMap

end DomainTheory.BridgeTheorems
```

6.8.10 From an approximable map back to a Scott-continuous map

Theorem (`toScottContinuous_ofScottContinuous`). Converting an approximable map to a Scott-continuous map and back yields the original approximable map.

Proof. The two constructions are inverse on the nose. ■

toScottContinuous_ofScottContinuous.lean

```
module
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    ApproximableMapScott
public import DomainTheory.BridgeTheorems.scottContinuous_ext
import Mathlib.Tactic.Continuity.Init

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

theorem toScottContinuous_ofScottContinuous {A : InfoSys α} {B : InfoSys β}
    (g : ScottContinuous A B) :
    toScottContinuous (ofScottContinuous g) = g :=
  scottContinuous_ext fun x => toElement_ofScottContinuous g x

end DomainTheory.BridgeTheorems
```

6.8.11 From a Scott-continuous map back to an approximable map

Theorem (`ofScottContinuous_toScottContinuous`). Converting a Scott-continuous map to an approximable map and back yields the original Scott-continuous map.

Proof. The two constructions are inverse on the nose. ■

`ofScottContinuous_toScottContinuous.lean`

```
module
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    ApproximableMapScott
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    closure_le_element
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement
public import DomainTheory.InformationSystems.ElementsOfASystem.closure
public import Mathlib.Order.Hom.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_10
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_2
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_5
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    ContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_2_10
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_12
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_13
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_14
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    factoid_46
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    proposition_5_3
public import DomainTheory.BridgeTheorems.InfoSysConstructions
public import DomainTheory.BridgeTheorems.PresentationDomains
public import DomainTheory.BridgeTheorems.ConjScottMap
public import DomainTheory.BridgeTheorems.ScottMapBridgeInst9

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

theorem ofScottContinuous_toScottContinuous {A : InfoSys α} {B : InfoSys β}
    (f : ApproximableMap A B) :
    ofScottContinuous (toScottContinuous f) = f := by
  refine ApproximableMap.ext fun u v => ?_
  constructor
  · intro ⟨hu, hv, hsub⟩
    exact (f.rel_iff_closure_le hu hv).2
      (B.closure_le_element (f.toElement (A.closure u hu)) hv hsub)
  · intro hrel
    refine ⟨f.rel_dom hrel, f.rel_cod hrel, ?_⟩
    intro y hy
    have hv : v ∈ B.Con := f.rel_cod hrel
    have hu : u ∈ A.Con := f.rel_dom hrel
    exact (f.rel_iff_closure_le hu hv).1 hrel (closure.subset B hv (Finset.mem_coe.1
    hy))

end DomainTheory.BridgeTheorems
```

6.8.12 Approximable maps and Scott-continuous maps

Definition (`approximableMap_scottContinuous_equiv`). Approximable maps between information systems are order-isomorphic to Scott-continuous maps on their elements.

approximableMap_scottContinuous_equiv.lean

```
module

public import DomainTheory.BridgeTheorems.ScottMapBridgeInst9
public import DomainTheory.BridgeTheorems.ofScottContinuous_toScottContinuous
```

```
public import DomainTheory.BridgeTheorems.toScottContinuous_ofScottContinuous
public import DomainTheory.BridgeTheorems.InfoSysConstructions.
    instPartialOrderApproximableMap
public import Mathlib.Algebra.Group.End
import Mathlib.Tactic.Continuity.Init

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

variable {α β : Type*} [DecidableEq α] [DecidableEq β]

noncomputable def approximableMap_scottContinuous_equiv (A : InfoSys α) (B : InfoSys β
    ) :
    ApproximableMap A B ≃o ScottContinuous A B where
  toFun := toScottContinuous
  invFun := ofScottContinuous
  left_inv := ofScottContinuous_toScottContinuous
  right_inv := toScottContinuous_ofScottContinuous
  map_rel_iff' := by
    intro f g
    exact (le_iff_toElement_le f g).symm

end DomainTheory.BridgeTheorems
```

6.8.13 Scott maps along the round presentation

Definition (`scottMap_roundFilter_iso`). Scott maps of continuous lattices are order-isomorphic to conjugate Scott maps along the round-filter presentation.

Definition (`scottMap_roundInfoSys_iso`). Scott maps of continuous lattices are order-isomorphic to conjugate Scott maps along the round information-system presentation.

Definition (`infoSys_constructions_equiv.product`). Alias for the product domain equivalence of information systems.

Definition (`infoSys_constructions_equiv.sum`). Alias for the separated-sum domain equivalence.

Definition (`infoSys_constructions_equiv.functionSpace`). Alias for the function-space domain equivalence.

Definition (`infoSys_constructions_equiv.approximable_scottContinuous`). Alias for the approximable-map/Scott-continuous equivalence.

Definition (`infoSys_constructions_equiv.scottMap_roundFilter`). Alias for conjugation of Scott maps along round filters.

Definition (`infoSys_constructions_equiv.scottMap_roundInfoSys`). Alias for conjugation of Scott maps along round information systems.

`ScottMapBridge.lean`

```
module

public import DomainTheory.BridgeTheorems.approximableMap_scottContinuous_equiv
public import DomainTheory.BridgeTheorems.orderIso
public import DomainTheory.BridgeTheorems.PresentationDomains.
    presentation_domains_equiv
public import DomainTheory.BridgeTheorems.InfoSysConstructions.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice
variable {α β : Type*} [DecidableEq α] [DecidableEq β]
variable {D E : Type*} [CompleteLattice D] [CompleteLattice E]

noncomputable abbrev scottMap_roundFilter_iso (hD : ContinuousLattice D) (hE :
    ContinuousLattice E) :
    ScottMap D E ≃o
      ConjScottMap (continuousLattice_roundFilter_iso hD)
        (continuousLattice_roundFilter_iso hE) :=
  ConjScottMap.orderIso (continuousLattice_roundFilter_iso hD)
    (continuousLattice_roundFilter_iso hE)

variable [DecidableEq D] [DecidableEq E]

noncomputable abbrev scottMap_roundInfoSys_iso (hD : ContinuousLattice D) (hE :
    ContinuousLattice E) :
    ScottMap D E ≃o
      ConjScottMap (presentation_domains_equiv hD) (presentation_domains_equiv hE) :=
  ConjScottMap.orderIso (presentation_domains_equiv hD) (presentation_domains_equiv hE
    )

namespace infoSys_constructions_equiv

noncomputable abbrev product := @infoSys_product_domain_equiv
noncomputable abbrev sum := @infoSys_sum_domain_equiv
noncomputable abbrev functionSpace := @infoSys_function_space_domain_equiv
noncomputable abbrev approximable_scottContinuous :=
    @approximableMap_scottContinuous_equiv
noncomputable abbrev scottMap_roundFilter := @scottMap_roundFilter_iso
noncomputable abbrev scottMap_roundInfoSys := @scottMap_roundInfoSys_iso

end infoSys_constructions_equiv

end DomainTheory.BridgeTheorems
```

6.9 The tree equation

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>



Figure 72: Diagram 37

6.9.1 The product of a tree system with itself

Definition (`TreeProd`). The product of a tree information system with itself.

`TreeProd.lean`

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.treeSystem

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
```

```
variable {α : Type*} [DecidableEq α] (A : InfoSys α)

abbrev TreeProd : InfoSys (ProdToken (treeSystem A) (treeSystem A)) :=
  productSystem (treeSystem A) (treeSystem A)

end DomainTheory.BridgeTheorems
```

6.9.2 Folding a sum token into a tree token

Definition (`treeFold`). Folding sends a token of the separated sum of a tree system with its square into a token of the tree system.

`treeFold.lean`

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.treeSystem

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def treeFold :
    SumToken α (ProdToken (treeSystem A) (treeSystem A)) → TreeToken α
  | .bot => .bot
  | .left x => .atom x
  | .right p =>
      if p.val.2 = treeBot then .pairL p.val.1 else .pairR p.val.2

end DomainTheory.BridgeTheorems
```

6.9.3 Inserting the unfolding of a tree token

Definition (`unfoldInsert`). Inserting the unfolding of a tree token into a finite set of sum tokens.

`unfoldInsert.lean`

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.treeSystem

@[expose] public section

namespace DomainTheory.BridgeTheorems
```

```
open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def unfoldInsert (t : TreeToken α) :
    Finset (SumToken α (ProdToken (treeSystem A) (treeSystem A))) →
      Finset (SumToken α (ProdToken (treeSystem A) (treeSystem A))) :=
  insert (treeUnfold A t)

end DomainTheory.BridgeTheorems
```

6.9.4 Unfolding commutes with insertion

Definition (`instLeftCommutativeUnfoldInsert`). Unfolding-and-inserting is left-commutative, so finite folds over unfoldings are well-defined.

SexDomainEquationInst10.lean

```
module

public import DomainTheory.BridgeTheorems.unfoldInsert

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open DomainTheory.InformationSystems.InfoSys

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

instance : LeftCommutative (unfoldInsert (A := A)) :=
  ⟨fun p q s => insert_comm' (treeUnfold A p) (treeUnfold A q) s⟩

end DomainTheory.BridgeTheorems
```

6.9.5 Inserting the folding of a sum token

Definition (`foldInsert`). Inserting the folding of a sum token into a finite set of tree tokens.

foldInsert.lean

```
module

public import DomainTheory.BridgeTheorems.treeFold

@[expose] public section
```

```
namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def foldInsert (s : SumToken α (ProdToken (treeSystem A) (treeSystem A))) :
    Finset (TreeToken α) → Finset (TreeToken α) :=
  insert (treeFold A s)

end DomainTheory.BridgeTheorems
```

6.9.6 Folding commutes with insertion

Definition (`instLeftCommutativeFoldInsert`). Folding-and-inserting is left-commutative, so finite folds over foldings are well-defined.

SexDomainEquationInst15.lean

```
module

public import DomainTheory.BridgeTheorems.foldInsert

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

instance : LeftCommutative (foldInsert (A := A)) :=
  ⟨fun p q t => insert_comm' (treeFold A p) (treeFold A q) t⟩

end DomainTheory.BridgeTheorems
```

6.9.7 Folding yields the token or a pair of bottoms

Theorem (`treeFold_eq_or_pairBots`). Folding a sum token either yields that token as a tree token or produces a pair of bottoms.

Proof. Case analysis on the polarity of the sum token. ■

treeFold_eq_or_pairBots.lean

```
module

public import DomainTheory.BridgeTheorems.treeFold
```

```
@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem treeFold_eq_or_pairBots (t : TreeToken α) :
    treeFold A (treeUnfold A t) = t ∨
      t = .pairR .bot ∧ treeFold A (treeUnfold A t) = .pairL .bot := by
  cases t with
  | bot => exact Or.inl rfl
  | atom x => exact Or.inl rfl
  | pairL t =>
    refine Or.inl ?_
    simp [treeFold, treeUnfold, treeBot]
  | pairR t =>
    simp only [treeFold, treeUnfold, treeBot]
    split_ifs with hb
    · cases t with
      | bot => exact Or.inr ⟨rfl, rfl⟩
      | atom _ | pairL _ | pairR _ => exact False.elim (nomatch hb)
    · exact Or.inl (ite_eq_right hb)

end DomainTheory.BridgeTheorems
```

6.9.8 Unfolding after folding a token

Theorem (`treeUnfold_treeFold`). Unfolding after folding a sum token recovers the token or a harmless bottom pair.

Proof. Compare the two sides by cases on the sum token. ■

`treeUnfold_treeFold.lean`

```
module

public import DomainTheory.BridgeTheorems.treeFold

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)
```

```
theorem treeUnfold_treeFold (s : SumToken α (ProdToken (treeSystem A) (treeSystem A)))
     :
    treeUnfold A (treeFold A s) = s := by
  cases s with
  | bot => rfl
  | left x => rfl
  | right p =>
    dsimp [treeFold, treeUnfold]
    split_ifs with hb
    · apply congrArg SumToken.right
      apply Subtype.ext
      apply Prod.ext
      · rfl
      · exact hb.symm
    · have ha : p.val.1 = treeBot := by
        rcases p.property with h1 | h2
        · exact h1
        · exact False.elim (hb h2)
      apply congrArg SumToken.right
      apply Subtype.ext
      apply Prod.ext
      · exact ha.symm
      · rfl

end DomainTheory.BridgeTheorems
```

6.9.9 Unfolding a set of tree tokens

Definition (`unfoldFinset`). Unfolding a finite set of tree tokens is the finite fold of unfolding-and-inserting.

`unfoldFinset.lean`

```
module

public import DomainTheory.BridgeTheorems.SexDomainEquationInst10

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def unfoldFinset (u : Finset (TreeToken α)) :
    Finset (SumToken α (ProdToken (treeSystem A) (treeSystem A))) :=
  Multiset.foldr (unfoldInsert A) ∅ u.1

end DomainTheory.BridgeTheorems
```

6.9.10 Folding a set of sum tokens

Definition (`foldFinset`). Folding a finite set of sum tokens is the finite fold of folding-and-inserting.

foldFinset.lean

```
module

public import DomainTheory.BridgeTheorems.SexDomainEquationInst15

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def foldFinset (Y : Finset (SumToken α (ProdToken (treeSystem A) (treeSystem A)))) :
    Finset (TreeToken α) :=
  Multiset.foldr (foldInsert A) ∅ Y.1

end DomainTheory.BridgeTheorems
```

6.9.11 Fold membership after unfolding

Theorem (`mem_foldr_unfold`). Membership after a right-fold of unfoldings is characterised by membership in some unfolding step.

Proof. Induction on the finite list being folded. ■

mem_foldr_unfold.lean

```
module

public import DomainTheory.BridgeTheorems.SexDomainEquationInst10

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem mem_foldr_unfold (m : Multiset (TreeToken α))
    (s : SumToken α (ProdToken (treeSystem A) (treeSystem A))) :
    s ∈ Multiset.foldr (unfoldInsert A) ∅ m ↔
      ∃ t ∈ m, treeUnfold A t = s := by
  refine Multiset.induction_on m ?_ ?_
```

```
  · constructor
    · intro hs; exact False.elim (Finset.notMem_empty s hs)
    · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
  · intro t rest ih
    simp only [Multiset.foldr_cons, unfoldInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
    constructor
    · rintro (h | ⟨q, hq, hq'⟩)
      · exact ⟨t, Or.inl rfl, h.symm⟩
      · exact ⟨q, Or.inr hq, hq'⟩
    · rintro ⟨q, hq, hq'⟩
      rcases hq with rfl | hq
      · exact Or.inl hq'.symm
      · exact Or.inr ⟨q, hq, hq'⟩

end DomainTheory.BridgeTheorems
```

6.9.12 Membership in an unfolded token set

Theorem (`mem_unfoldFinset`). A sum token belongs to the unfolding of a finite tree-token set if and only if it arises from unfolding some member.

Proof. Specialise the fold-membership criterion to unfolding-and-inserting. ■

`mem_unfoldFinset.lean`

```
module

public import DomainTheory.BridgeTheorems.unfoldFinset
public import DomainTheory.BridgeTheorems.mem_foldr_unfold

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem mem_unfoldFinset {u : Finset (TreeToken α)}
    {s : SumToken α (ProdToken (treeSystem A) (treeSystem A))} :
    s ∈ unfoldFinset A u ↔ ∃ t ∈ u, treeUnfold A t = s := by
  unfold unfoldFinset
  rw [mem_foldr_unfold]
  simp only [Finset.mem_def]

end DomainTheory.BridgeTheorems
```

6.9.13 Fold membership after folding

Theorem (`mem_foldr_fold`). Membership after a right-fold of foldings is characterised by membership in some folding step.

Proof. Induction on the finite list being folded. ■

`mem_foldr_fold.lean`

```
module

public import DomainTheory.BridgeTheorems.SexDomainEquationInst15

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem mem_foldr_fold
    (m : Multiset (SumToken α (ProdToken (treeSystem A) (treeSystem A))))
    (t : TreeToken α) :
    t ∈ Multiset.foldr (foldInsert A) ∅ m ↔
      ∃ s ∈ m, treeFold A s = t := by
  refine Multiset.induction_on m ?_ ?_
  · constructor
    · intro ht; exact False.elim (Finset.notMem_empty t ht)
    · rintro ⟨_, hp, _⟩; exact False.elim (by cases hp)
  · intro s rest ih
    simp only [Multiset.foldr_cons, foldInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
    constructor
    · rintro (h | ⟨q, hq, hq'⟩)
      · exact ⟨s, Or.inl rfl, h.symm⟩
      · exact ⟨q, Or.inr hq, hq'⟩
    · rintro ⟨q, hq, hq'⟩
      rcases hq with rfl | hq
      · exact Or.inl hq'.symm
      · exact Or.inr ⟨q, hq, hq'⟩

end DomainTheory.BridgeTheorems
```

6.9.14 Membership in a folded token set

Theorem (`mem_foldFinset`). A tree token belongs to the folding of a finite sum-token set if and only if it arises from folding some member.

Proof. Specialise the fold-membership criterion to folding-and-inserting. ■

`mem_foldFinset.lean`

```
module

public import DomainTheory.BridgeTheorems.foldFinset
public import DomainTheory.BridgeTheorems.mem_foldr_fold

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem mem_foldFinset
    {Y : Finset (SumToken α (ProdToken (treeSystem A) (treeSystem A)))}
    {t : TreeToken α} :
    t ∈ foldFinset A Y ↔ ∃ s ∈ Y, treeFold A s = t := by
  unfold foldFinset
  rw [mem_foldr_fold]
  simp only [Finset.mem_def]

end DomainTheory.BridgeTheorems
```

6.9.15 Unfolding a folded token set

Theorem (`unfoldFinset_foldFinset`). Unfolding after folding a finite set of sum tokens recovers that set up to the tree-fold correspondence.

Proof. Combine the membership criteria for folding and unfolding of finite sets. ■

`unfoldFinset_foldFinset.lean`

```
module

public import DomainTheory.BridgeTheorems.unfoldFinset
public import DomainTheory.BridgeTheorems.foldFinset
public import DomainTheory.BridgeTheorems.mem_foldFinset
public import DomainTheory.BridgeTheorems.mem_unfoldFinset
public import DomainTheory.BridgeTheorems.treeUnfold_treeFold

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem unfoldFinset_foldFinset
    (Y : Finset (SumToken α (ProdToken (treeSystem A) (treeSystem A)))) :
```

```
    unfoldFinset A (foldFinset A Y) = Y := by
  ext s
  constructor
  · intro hs
    rcases (mem_unfoldFinset A).1 hs with ⟨t, ht, hU⟩
    rcases (mem_foldFinset A).1 ht with ⟨s', hs', rfl⟩
    rw [treeUnfold_treeFold] at hU
    rwa [← hU]
  · intro hs
    exact (mem_unfoldFinset A).2 ⟨treeFold A s,
      (mem_foldFinset A).2 ⟨s, hs, rfl⟩, treeUnfold_treeFold A s⟩

end DomainTheory.BridgeTheorems
```

6.9.16 Left tokens of an unfolding

Theorem (`lftFinset_image_unfold`). The left tokens of an unfolded finite set are the left projections of the unfoldings of its members.

Proof. Track left polarity through the membership criterion for unfoldings. ■

`lftFinset_image_unfold.lean`

```
module

public import DomainTheory.BridgeTheorems.unfoldFinset
public import DomainTheory.BridgeTheorems.mem_unfoldFinset

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem lftFinset_image_unfold (u : Finset (TreeToken α)) :
    lftFinset (unfoldFinset A u) = atomFinset u := by
  ext x
  constructor
  · intro hx
    have : SumToken.left x ∈ unfoldFinset A u := (mem_lftFinset).1 hx
    rcases (mem_unfoldFinset A).1 this with ⟨t, ht, hU⟩
    cases t with
    | atom y =>
      have : y = x := SumToken.left.inj hU
      subst this
      exact mem_atomFinset.2 ht
    | bot | pairL _ | pairR _ => exact nomatch hU
  · intro hx
    exact (mem_lftFinset).2
      ((mem_unfoldFinset A).2 ⟨.atom x, mem_atomFinset.1 hx, rfl⟩)
```

```
end DomainTheory.BridgeTheorems
```

6.9.17 Right tokens of an unfolding

Theorem (`rhtFinset_image_unfold_mem`). Membership among the right tokens of an unfolded finite set is equivalent to arising as the right projection of some unfolding.

Proof. Track right polarity through the membership criterion for unfoldings. ■

`rhtFinset_image_unfold_mem.lean`

```
module

public import DomainTheory.BridgeTheorems.unfoldFinset
public import DomainTheory.BridgeTheorems.mem_unfoldFinset

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem rhtFinset_image_unfold_mem (u : Finset (TreeToken α))
    (y : ProdToken (treeSystem A) (treeSystem A)) :
    y ∈ rhtFinset (unfoldFinset A u) ↔
      (∃ t, .pairL t ∈ u ∧ y = ⟨(t, treeBot), ⟨Or.inr rfl⟩⟩) ∨
        (∃ t, .pairR t ∈ u ∧ y = ⟨(treeBot, t), ⟨Or.inl rfl⟩⟩) := by
  constructor
  · intro hy
    have : SumToken.right y ∈ unfoldFinset A u := (mem_rhtFinset).1 hy
    rcases (mem_unfoldFinset A).1 this with ⟨t, ht, hU⟩
    cases t with
    | pairL s =>
      refine Or.inl ⟨s, ht, ?_⟩
      exact SumToken.right.inj hU.symm
    | pairR s =>
      refine Or.inr ⟨s, ht, ?_⟩
      exact SumToken.right.inj hU.symm
    | bot | atom _ => exact nomatch hU
  · rintro (⟨t, ht, rfl⟩ | ⟨t, ht, rfl⟩)
    · exact (mem_rhtFinset).2 ((mem_unfoldFinset A).2 ⟨.pairL t, ht, rfl⟩)
    · exact (mem_rhtFinset).2 ((mem_unfoldFinset A).2 ⟨.pairR t, ht, rfl⟩)

end DomainTheory.BridgeTheorems
```

6.9.18 Left pair-bottom entailment

Theorem (`TreeEnt_pairL_bot_of_pairs`). A finite set of tree tokens that entails a left pair-bottom does so whenever it contains suitable paired data.

Proof. Apply the entailment rules of the tree information system. ■

`TreeEnt_pairL_bot_of_pairs.lean`

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.treeSystem

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem TreeEnt_pairL_bot_of_pairs {u : Finset (TreeToken α)}
    (hu : TreeCon A u) (h : pairLFinset u = ∅ ∨ pairRFinset u = ∅) :
    TreeEnt A u (.pairL .bot) :=
  ⟨hu, TreeCon.atoms_empty_of_right A hu h,
    TreeCon.pairL_of_right A hu (TreeCon.atoms_empty_of_right A hu h),
    TreeCon.pairR_of_right A hu (TreeCon.atoms_empty_of_right A hu h),
    h.elim (fun hL => Or.inl ⟨hL, trivial⟩) (fun hR => Or.inr ⟨rfl, hR⟩)⟩

end DomainTheory.BridgeTheorems
```

6.9.19 Right pair-bottom entailment

Theorem (`TreeEnt_pairR_bot_of_pairs`). A finite set of tree tokens that entails a right pair-bottom does so whenever it contains suitable paired data.

Proof. Apply the entailment rules of the tree information system. ■

`TreeEnt_pairR_bot_of_pairs.lean`

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.treeSystem

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem
```

```
variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem TreeEnt_pairR_bot_of_pairs {u : Finset (TreeToken α)}
    (hu : TreeCon A u) (h : pairLFinset u = ∅ ∨ pairRFinset u = ∅) :
    TreeEnt A u (.pairR .bot) :=
  ⟨hu, TreeCon.atoms_empty_of_right A hu h,
    TreeCon.pairR_of_right A hu (TreeCon.atoms_empty_of_right A hu h),
    TreeCon.pairL_of_right A hu (TreeCon.atoms_empty_of_right A hu h),
    h.elim (fun hL => Or.inr ⟨rfl, hL⟩) (fun hR => Or.inl ⟨hR, trivial⟩)⟩

end DomainTheory.BridgeTheorems
```

6.9.20 Consistency of an unfolded token set

Theorem (`TreeCon_image_sumCon`). The unfolding of a consistent finite set of tree tokens is a consistent finite set of sum tokens.

Proof. Consistency and entailment transport along the folding/unfolding correspondence. ■

`TreeCon_image_sumCon.lean`

```
module

public import DomainTheory.BridgeTheorems.TreeProd
public import DomainTheory.BridgeTheorems.unfoldFinset
public import DomainTheory.BridgeTheorems.lftFinset_image_unfold
public import DomainTheory.BridgeTheorems.rhtFinset_image_unfold_mem

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem TreeCon_image_sumCon {u : Finset (TreeToken α)} (hu : TreeCon A u) :
    SumCon A (TreeProd A) (unfoldFinset A u) := by
  cases hu with
  | left _ hA hL hR =>
    refine Or.inl ⟨?_, ?_⟩
    · rw [lftFinset_image_unfold]; exact hA
    · ext y
      constructor
      · intro hy
        rcases (rhtFinset_image_unfold_mem A u y).1 hy with ⟨t, ht, _⟩ | ⟨t, ht, _⟩
        · exact False.elim (Finset.notMem_empty t (hL ▸ mem_pairLFinset.2 ht))
        · exact False.elim (Finset.notMem_empty t (hR ▸ mem_pairRFinset.2 ht))
      · intro hy
        exact False.elim (Finset.notMem_empty y hy)
  | right _ hA hL hR =>
    refine Or.inr ⟨?_, ?_⟩
```

```
    · rw [lftFinset_image_unfold]; exact hA
    · refine ⟨?_, ?_⟩
      ·
        have hsub :
            fstFinset (treeSystem A) (treeSystem A)
                (rhtFinset (unfoldFinset A u)) ⊆
              insert .bot (pairLFinset u) := by
          intro t ht
          rcases (mem_fstFinset (treeSystem A) (treeSystem A)).1 ht with ⟨p, hp, hb,
    rfl⟩
          rcases (rhtFinset_image_unfold_mem A u p).1 hp with ⟨s, hs, hp'⟩ | ⟨s, hs, hp
    '⟩
          · subst hp'
            exact Finset.mem_insert_of_mem (mem_pairLFinset.2 hs)
          · subst hp'
            exact Finset.mem_insert_self _ _
        exact (treeSystem A).con_subset (TreeCon_insert_bot A hL) hsub
      · have hsub :
            sndFinset (treeSystem A) (treeSystem A)
                (rhtFinset (unfoldFinset A u)) ⊆
              insert .bot (pairRFinset u) := by
          intro t ht
          rcases (mem_sndFinset (treeSystem A) (treeSystem A)).1 ht with ⟨p, hp, ha,
    rfl⟩
          rcases (rhtFinset_image_unfold_mem A u p).1 hp with ⟨s, hs, hp'⟩ | ⟨s, hs, hp
    '⟩
          · subst hp'
            exact Finset.mem_insert_self _ _
          · subst hp'
            exact Finset.mem_insert_of_mem (mem_pairRFinset.2 hs)
        exact (treeSystem A).con_subset (TreeCon_insert_bot A hR) hsub

end DomainTheory.BridgeTheorems
```

6.9.21 Tree entailment and sum entailment

Theorem (`TreeEnt_sumEnt`). Tree entailment of a token by a finite set is equivalent to sum entailment of its unfolding by the unfolded set.

Proof. Transport entailment along folding and unfolding of tokens and finite sets. ■

`TreeEnt_sumEnt.lean`

```
module

public import DomainTheory.BridgeTheorems.TreeProd
public import DomainTheory.BridgeTheorems.unfoldFinset
public import DomainTheory.BridgeTheorems.TreeCon_image_sumCon
public import DomainTheory.BridgeTheorems.lftFinset_image_unfold
public import DomainTheory.BridgeTheorems.rhtFinset_image_unfold_mem

@[expose] public section

namespace DomainTheory.BridgeTheorems
```

```
open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem TreeEnt_sumEnt {u : Finset (TreeToken α)} {p : TreeToken α}
    (h : TreeEnt A u p) :
    SumEnt A (TreeProd A) (unfoldFinset A u) (treeUnfold A p) := by
  have hCon : SumCon A (TreeProd A) (unfoldFinset A u) :=
    TreeCon_image_sumCon A h.1
  refine ⟨hCon, ?_⟩
  cases p with
  | bot => exact trivial
  | atom x =>
    refine ⟨?_, ?_⟩
    · rw [lftFinset_image_unfold]; exact h.2.1
    · rw [lftFinset_image_unfold]; exact h.2.2
  | pairL t =>
    have hr : rhtFinset (unfoldFinset A u) = ∅ := by
      rcases h.2.2.2.2 with ⟨hne, _⟩ | ⟨_, hR⟩
      · obtain ⟨s, hs⟩ := Finset.nonempty_of_ne_empty hne
        exact Finset.ne_empty_of_mem
          ((rhtFinset_image_unfold_mem A u _).2
            (Or.inl ⟨s, mem_pairLFinset.1 hs, rfl⟩))
      · obtain ⟨s, hs⟩ := Finset.nonempty_of_ne_empty hR
        exact Finset.ne_empty_of_mem
          ((rhtFinset_image_unfold_mem A u _).2
            (Or.inr ⟨s, mem_pairRFinset.1 hs, rfl⟩))
    refine ⟨hr, ?_⟩
    refine ⟨hCon.elim (fun hL => False.elim (hr hL.2)) (fun hR => hR.2), ?_, ?_⟩
    · intro

      have hsub : pairLFinset u ⊆
          fstFinset (treeSystem A) (treeSystem A)
            (rhtFinset (unfoldFinset A u)) := by
        intro s hs
        exact (mem_fstFinset (treeSystem A) (treeSystem A)).2
          ⟨⟨(s, treeBot), ⟨Or.inr rfl⟩⟩,
            (rhtFinset_image_unfold_mem A u _).2 (Or.inl ⟨s, mem_pairLFinset.1 hs, rfl⟩
  ),
           rfl, rfl⟩
      have hFstCon : fstFinset (treeSystem A) (treeSystem A)
          (rhtFinset (unfoldFinset A u)) ∈ (treeSystem A).Con :=
        hCon.elim (fun hL => False.elim (hr hL.2)) (fun hR => hR.2.1)
      rcases h.2.2.2.2 with ⟨hne, hPay⟩ | ⟨rfl, _⟩
      · exact (treeSystem A).ent_trans hFstCon h.2.2.1
          (fun y hy => (treeSystem A).ent_refl hFstCon (hsub hy)) ⟨h.2.2.1, hPay⟩
      · exact (treeSystem A).ent_bot hFstCon
    · intro ht
      have hSndCon : sndFinset (treeSystem A) (treeSystem A)
          (rhtFinset (unfoldFinset A u)) ∈ (treeSystem A).Con :=
        hCon.elim (fun hL => False.elim (hr hL.2)) (fun hR => hR.2.2)
      exact (treeSystem A).ent_bot hSndCon
```

```
  | pairR t =>
    have hr : rhtFinset (unfoldFinset A u) = ∅ := by
      rcases h.2.2.2.2 with ⟨hne, _⟩ | ⟨_, hL⟩
      · obtain ⟨s, hs⟩ := Finset.nonempty_of_ne_empty hne
        exact Finset.ne_empty_of_mem
          ((rhtFinset_image_unfold_mem A u _).2
            (Or.inr ⟨s, mem_pairRFinset.1 hs, rfl⟩))
      · obtain ⟨s, hs⟩ := Finset.nonempty_of_ne_empty hL
        exact Finset.ne_empty_of_mem
          ((rhtFinset_image_unfold_mem A u _).2
            (Or.inl ⟨s, mem_pairLFinset.1 hs, rfl⟩))
    refine ⟨hr, ?_⟩
    refine ⟨hCon.elim (fun hL => False.elim (hr hL.2)) (fun hR => hR.2), ?_, ?_⟩
    · intro
      have hFstCon : fstFinset (treeSystem A) (treeSystem A)
          (rhtFinset (unfoldFinset A u)) ∈ (treeSystem A).Con :=
        hCon.elim (fun hL => False.elim (hr hL.2)) (fun hR => hR.2.1)
      exact (treeSystem A).ent_bot hFstCon
    · intro
      have hsub : pairRFinset u ⊆
          sndFinset (treeSystem A) (treeSystem A)
            (rhtFinset (unfoldFinset A u)) := by
        intro s hs
        exact (mem_sndFinset (treeSystem A) (treeSystem A)).2
          ⟨⟨(treeBot, s), ⟨Or.inl rfl⟩⟩,
            (rhtFinset_image_unfold_mem A u _).2 (Or.inr ⟨s, mem_pairRFinset.1 hs, rfl⟩
  ),
           rfl, rfl⟩
      have hSndCon : sndFinset (treeSystem A) (treeSystem A)
          (rhtFinset (unfoldFinset A u)) ∈ (treeSystem A).Con :=
        hCon.elim (fun hL => False.elim (hr hL.2)) (fun hR => hR.2.2)
      rcases h.2.2.2.2 with ⟨hne, hPay⟩ | ⟨rfl, _⟩
      · exact (treeSystem A).ent_trans hSndCon h.2.2.1
          (fun y hy => (treeSystem A).ent_refl hSndCon (hsub hy)) ⟨h.2.2.1, hPay⟩
      · exact (treeSystem A).ent_bot hSndCon

end DomainTheory.BridgeTheorems
```

6.9.22 Consistency of a folded token set

Theorem (`SumCon_image_treeCon`). The folding of a consistent finite set of sum tokens is a consistent finite set of tree tokens.

Proof. Consistency transports along the folding correspondence. ■

SumCon_image_treeCon.lean

```
module

public import DomainTheory.BridgeTheorems.TreeProd
public import DomainTheory.BridgeTheorems.unfoldFinset
public import DomainTheory.BridgeTheorems.lftFinset_image_unfold
public import DomainTheory.BridgeTheorems.rhtFinset_image_unfold_mem

@[expose] public section
```

```
namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem SumCon_image_treeCon {u : Finset (TreeToken α)}
    (hu : SumCon A (TreeProd A) (unfoldFinset A u)) : TreeCon A u := by
  rcases hu with ⟨hL, hr⟩ | ⟨hl, hR⟩
  · refine TreeCon.left _ (by rwa [lftFinset_image_unfold] at hL) ?_ ?_
    · ext t
      constructor
      · intro ht
        have : (⟨(t, treeBot), ⟨Or.inr rfl⟩⟩ : ProdToken (treeSystem A) (treeSystem A))
    ∈
            rhtFinset (unfoldFinset A u) :=
          (rhtFinset_image_unfold_mem A u _).2 (Or.inl ⟨t, mem_pairLFinset.1 ht, rfl⟩)
        rw [hr] at this
        exact False.elim (Finset.notMem_empty _ this)
      · intro ht
        exact False.elim (Finset.notMem_empty t ht)
    · ext t
      constructor
      · intro ht
        have : (⟨(treeBot, t), ⟨Or.inl rfl⟩⟩ : ProdToken (treeSystem A) (treeSystem A))
    ∈
            rhtFinset (unfoldFinset A u) :=
          (rhtFinset_image_unfold_mem A u _).2 (Or.inr ⟨t, mem_pairRFinset.1 ht, rfl⟩)
        rw [hr] at this
        exact False.elim (Finset.notMem_empty _ this)
      · intro ht
        exact False.elim (Finset.notMem_empty t ht)
  · refine TreeCon.right _ (by rwa [lftFinset_image_unfold] at hl) ?_ ?_
    · have hsub : pairLFinset u ⊆
          fstFinset (treeSystem A) (treeSystem A)
            (rhtFinset (unfoldFinset A u)) := by
        intro t ht
        exact (mem_fstFinset (treeSystem A) (treeSystem A)).2
          ⟨⟨(t, treeBot), ⟨Or.inr rfl⟩⟩,
            (rhtFinset_image_unfold_mem A u _).2 (Or.inl ⟨t, mem_pairLFinset.1 ht, rfl⟩
    ),
            rfl, rfl⟩
      exact (treeSystem A).con_subset hR.1 hsub
    · have hsub : pairRFinset u ⊆
          sndFinset (treeSystem A) (treeSystem A)
            (rhtFinset (unfoldFinset A u)) := by
        intro t ht
        exact (mem_sndFinset (treeSystem A) (treeSystem A)).2
          ⟨⟨(treeBot, t), ⟨Or.inl rfl⟩⟩,
            (rhtFinset_image_unfold_mem A u _).2 (Or.inr ⟨t, mem_pairRFinset.1 ht, rfl⟩
    ),
            rfl, rfl⟩
```

```
      exact (treeSystem A).con_subset hR.2 hsub

end DomainTheory.BridgeTheorems
```

6.9.23 Sum entailment after folding

Theorem (`SumEnt_treeFold`). Sum entailment after folding corresponds to tree entailment of the folded token.

Proof. Transport entailment along folding of tokens and finite sets. ■

SumEnt_treeFold.lean

```
module

public import DomainTheory.BridgeTheorems.TreeProd
public import DomainTheory.BridgeTheorems.treeFold
public import DomainTheory.BridgeTheorems.unfoldFinset
public import DomainTheory.BridgeTheorems.SumCon_image_treeCon
public import DomainTheory.BridgeTheorems.lftFinset_image_unfold
public import DomainTheory.BridgeTheorems.rhtFinset_image_unfold_mem

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem SumEnt_treeFold {u : Finset (TreeToken α)}
    {s : SumToken α (ProdToken (treeSystem A) (treeSystem A))}
    (h : SumEnt A (TreeProd A) (unfoldFinset A u) s) :
    TreeEnt A u (treeFold A s) := by
  have hu : TreeCon A u := SumCon_image_treeCon A h.1
  refine ⟨hu, ?_⟩
  cases s with
  | bot => exact trivial
  | left x =>
    dsimp [treeFold]
    refine ⟨?_, ?_⟩
    · rw [← lftFinset_image_unfold A u]; exact h.2.1
    · rw [← lftFinset_image_unfold A u]; exact h.2.2
  | right p =>
    dsimp [treeFold]
    have hAt : atomFinset u = ∅ := by
      have hl := (lftFinset_image_unfold A u)
      have : lftFinset (unfoldFinset A u) = ∅ :=
        SumCon.lft_empty_of_rht_nonempty A (TreeProd A) h.1 h.2.1
      rwa [hl] at this
    split_ifs with hb
    · refine ⟨hAt, TreeCon.pairL_of_right A hu hAt, TreeCon.pairR_of_right A hu hAt, ?_⟩
```

```
  have hEnt := h.2.2

  have hFst : (treeSystem A).Ent
      (fstFinset (treeSystem A) (treeSystem A)
        (rhtFinset (unfoldFinset A u))) p.val.1 :=
    hEnt.2.1 hb
  have hsub :
      fstFinset (treeSystem A) (treeSystem A)
          (rhtFinset (unfoldFinset A u)) ⊆
        insert .bot (pairLFinset u) := by
    intro t ht
    rcases (mem_fstFinset (treeSystem A) (treeSystem A)).1 ht with ⟨q, hq, _, rfl⟩
    rcases (rhtFinset_image_unfold_mem A u q).1 hq with ⟨w, hw, hq'⟩ | ⟨w, hw, hq'⟩
    · subst hq'; exact Finset.mem_insert_of_mem (mem_pairLFinset.2 hw)
    · subst hq'; exact Finset.mem_insert_self _ _

  have hIns : TreeCon A (insert .bot (pairLFinset u)) :=
    TreeCon_insert_bot A (TreeCon.pairL_of_right A hu hAt)
  have hPay : TreeEntPayload A (insert .bot (pairLFinset u)) p.val.1 :=
    ((treeSystem A).ent_trans hIns
      (SumCon.rht_con_of_rht_nonempty A (TreeProd A) h.1 h.2.1).1
      (fun y hy => (treeSystem A).ent_refl hIns (hsub hy)) hFst).2
  obtain ⟨y, hy⟩ := Finset.nonempty_of_ne_empty h.2.1
  rcases (rhtFinset_image_unfold_mem A u y).1 hy with ⟨w, hwL, _⟩ | ⟨w, hwR, _⟩
  · refine Or.inl ⟨Finset.ne_empty_of_mem (mem_pairLFinset.2 hwL), ?_⟩
    exact ((treeSystem A).ent_trans (TreeCon.pairL_of_right A hu hAt) hIns
      (fun z hz => by
        rw [Finset.mem_insert] at hz
        rcases hz with rfl | hz
        · exact (treeSystem A).ent_bot (TreeCon.pairL_of_right A hu hAt)
        · exact (treeSystem A).ent_refl (TreeCon.pairL_of_right A hu hAt) hz)
      ⟨hIns, hPay⟩).2
  · if ht : p.val.1 = .bot then
      exact Or.inr ⟨ht, Finset.ne_empty_of_mem (mem_pairRFinset.2 hwR)⟩
    else
      refine Or.inl ⟨?_, ?_⟩
      · intro hempty
        have hEq : insert (.bot : TreeToken α) (pairLFinset u) = {.bot} := by
          rw [hempty]; rfl
        exact ht (TreeEntPayload.of_no_atoms_no_pairs A
          atomFinset_singleton_bot pairLFinset_singleton_bot
          pairRFinset_singleton_bot (hEq ▸ hPay))
      · exact ((treeSystem A).ent_trans (TreeCon.pairL_of_right A hu hAt) hIns
          (fun z hz => by
            rw [Finset.mem_insert] at hz
            rcases hz with rfl | hz
            · exact (treeSystem A).ent_bot (TreeCon.pairL_of_right A hu hAt)
            · exact (treeSystem A).ent_refl (TreeCon.pairL_of_right A hu hAt) hz)
          ⟨hIns, hPay⟩).2
.
  refine ⟨hAt, TreeCon.pairR_of_right A hu hAt, TreeCon.pairL_of_right A hu hAt, ?_
⟩
  have ha : p.val.1 = treeBot := by
    rcases p.property with h1 | h2
    · exact h1
    · exact False.elim (hb h2)
```

```
    have hEnt := h.2.2
    have hSnd : (treeSystem A).Ent
        (sndFinset (treeSystem A) (treeSystem A)
          (rhtFinset (unfoldFinset A u))) p.val.2 :=
      hEnt.2.2 ha
    have hIns : TreeCon A (insert .bot (pairRFinset u)) :=
      TreeCon_insert_bot A (TreeCon.pairR_of_right A hu hAt)
    have hsub :
        sndFinset (treeSystem A) (treeSystem A)
            (rhtFinset (unfoldFinset A u)) ⊆
          insert .bot (pairRFinset u) := by
      intro t ht
      rcases (mem_sndFinset (treeSystem A) (treeSystem A)).1 ht with ⟨q, hq, _, rfl⟩
      rcases (rhtFinset_image_unfold_mem A u q).1 hq with ⟨w, hw, hq'⟩ | ⟨w, hw, hq'⟩
      · subst hq'; exact Finset.mem_insert_self _ _
      · subst hq'; exact Finset.mem_insert_of_mem (mem_pairRFinset.2 hw)
    have hPay : TreeEntPayload A (insert .bot (pairRFinset u)) p.val.2 :=
      ((treeSystem A).ent_trans hIns
        (SumCon.rht_con_of_rht_nonempty A (TreeProd A) h.1 h.2.1).2
        (fun y hy => (treeSystem A).ent_refl hIns (hsub hy)) hSnd).2
    refine Or.inl ⟨?_, ?_⟩
    · intro hempty
      have hEq : insert (.bot : TreeToken α) (pairRFinset u) = {.bot} := by
        rw [hempty]; rfl
      have ht : p.val.2 = .bot :=
        TreeEntPayload.of_no_atoms_no_pairs A
          atomFinset_singleton_bot pairLFinset_singleton_bot
          pairRFinset_singleton_bot (hEq ▸ hPay)
      exact hb (ht ▸ rfl)
    · exact ((treeSystem A).ent_trans (TreeCon.pairR_of_right A hu hAt) hIns
        (fun z hz => by
          rw [Finset.mem_insert] at hz
          rcases hz with rfl | hz
          · exact (treeSystem A).ent_bot (TreeCon.pairR_of_right A hu hAt)
          · exact (treeSystem A).ent_refl (TreeCon.pairR_of_right A hu hAt) hz)
        ⟨hIns, hPay⟩).2

end DomainTheory.BridgeTheorems
```

6.9.24 A closed element contains both pair bottoms

Theorem (`mem_closed_pair_bots`). A deductively closed tree element that contains any paired data contains both pair bottoms.

Proof. Pair bottoms are entailed by paired tokens. ■

`mem_closed_pair_bots.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element
public import DomainTheory.InformationSystems.SomeDomainEquations.treeSystem
public import DomainTheory.BridgeTheorems.TreeEnt_pairL_bot_of_pairs
public import DomainTheory.BridgeTheorems.TreeEnt_pairR_bot_of_pairs
```

```
@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem mem_closed_pair_bots (x : (treeSystem A).Element) :
    .pairL .bot ∈ x.carrier ↔ .pairR .bot ∈ x.carrier := by
  constructor
  · intro h
    have hsub : (({TreeToken.pairL treeBot} : Finset (TreeToken α)) : Set _) ⊆
        x.carrier := by
      intro a ha
      have : a = .pairL treeBot := Finset.mem_singleton.mp (Finset.mem_coe.1 ha)
      subst this; exact h
    exact x.closed {TreeToken.pairL treeBot} (.pairR treeBot) hsub
      (TreeEnt_pairR_bot_of_pairs A (x.consistent _ hsub)
        (Or.inl (Finset.ne_empty_of_mem (by
          rw [pairLFinset_singleton_pairL]; exact Finset.mem_singleton_self _))))
  · intro h
    have hsub : (({TreeToken.pairR treeBot} : Finset (TreeToken α)) : Set _) ⊆
        x.carrier := by
      intro a ha
      have : a = .pairR treeBot := Finset.mem_singleton.mp (Finset.mem_coe.1 ha)
      subst this; exact h
    exact x.closed {TreeToken.pairR treeBot} (.pairL treeBot) hsub
      (TreeEnt_pairL_bot_of_pairs A (x.consistent _ hsub)
        (Or.inr (Finset.ne_empty_of_mem (by
          rw [pairRFinset_singleton_pairR]; exact Finset.mem_singleton_self _))))

end DomainTheory.BridgeTheorems
```

6.9.25 Closed tree elements are saturated for unfolding

Theorem (`saturated_of_closed`). A deductively closed tree element is saturated for unfolding: its unfolding is deductively closed in the sum system.

Proof. Closedness transports along the entailment correspondence between tree and sum tokens. ■

`saturated_of_closed.lean`

```
module

public import DomainTheory.BridgeTheorems.treeFold
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element
public import DomainTheory.BridgeTheorems.mem_closed_pair_bots
public import DomainTheory.BridgeTheorems.treeFold_eq_or_pairBots

@[expose] public section
```

```
namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem saturated_of_closed (x : (treeSystem A).Element) {t : TreeToken α}
    (ht : t ∈ x.carrier) : treeFold A (treeUnfold A t) ∈ x.carrier := by
  rcases treeFold_eq_or_pairBots A t with h | ⟨rfl, hf⟩
  · rwa [h]
  · rw [hf]
    exact (mem_closed_pair_bots A x).2 ht

end DomainTheory.BridgeTheorems
```

6.9.26 Unfolding a tree element

Definition (`unfoldElement`). Unfolding a tree element yields the corresponding element of the separated sum of the tree system with its square.

`unfoldElement.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element
public import DomainTheory.InformationSystems.SomeDomainEquations.treeSystem
public import DomainTheory.BridgeTheorems.SumEnt_treeFold
public import DomainTheory.BridgeTheorems.TreeCon_image_sumCon
public import DomainTheory.BridgeTheorems.mem_foldFinset
public import DomainTheory.BridgeTheorems.saturated_of_closed
public import DomainTheory.BridgeTheorems.treeUnfold_treeFold
public import DomainTheory.BridgeTheorems.unfoldFinset_foldFinset

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def unfoldElement (x : (treeSystem A).Element) : (treeRhs A).Element where
  carrier := treeUnfold A '' x.carrier
  consistent := by
    intro Y hY
    have hFold : ↑(foldFinset A Y) ⊆ x.carrier := by
      intro t ht
      rcases (mem_foldFinset A).1 (Finset.mem_coe.1 ht) with ⟨s, hs, rfl⟩
      have hs' : s ∈ treeUnfold A '' x.carrier := hY (Finset.mem_coe.2 hs)
```

```
      rcases hs' with ⟨t', ht', hU⟩
      have : treeFold A (treeUnfold A t') ∈ x.carrier := saturated_of_closed A x ht'
      rwa [hU] at this
    have hCon : TreeCon A (foldFinset A Y) := x.consistent _ hFold
    have : SumCon A (TreeProd A) (unfoldFinset A (foldFinset A Y)) :=
      TreeCon_image_sumCon A hCon
    rwa [unfoldFinset_foldFinset] at this
  closed := by
    intro Y s hY hEnt
    have hFold : ↑(foldFinset A Y) ⊆ x.carrier := by
      intro t ht
      rcases (mem_foldFinset A).1 (Finset.mem_coe.1 ht) with ⟨s', hs', rfl⟩
      have hs'' : s' ∈ treeUnfold A '' x.carrier := hY (Finset.mem_coe.2 hs')
      rcases hs'' with ⟨t', ht', hU⟩
      have : treeFold A (treeUnfold A t') ∈ x.carrier := saturated_of_closed A x ht'
      rwa [hU] at this
    have hEnt' : SumEnt A (TreeProd A)
        (unfoldFinset A (foldFinset A Y)) s := by
      rwa [unfoldFinset_foldFinset]
    have hT : TreeEnt A (foldFinset A Y) (treeFold A s) :=
      SumEnt_treeFold A hEnt'
    have ht : treeFold A s ∈ x.carrier :=
      x.closed _ _ hFold hT
    refine ⟨treeFold A s, ht, treeUnfold_treeFold A s⟩

end DomainTheory.BridgeTheorems
```

6.9.27 Folding a sum-of-product element

Definition (`foldElement`). Folding an element of the separated sum of a tree system with its square yields a tree element.

foldElement.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element
public import DomainTheory.InformationSystems.SomeDomainEquations.treeSystem
public import DomainTheory.BridgeTheorems.SumCon_image_treeCon
public import DomainTheory.BridgeTheorems.TreeEnt_sumEnt
public import DomainTheory.BridgeTheorems.mem_unfoldFinset

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def foldElement (y : (treeRhs A).Element) : (treeSystem A).Element where
  carrier := treeUnfold A ^-1' y.carrier
  consistent := by
```

```
    intro Y hY
    have him : ↑(unfoldFinset A Y) ⊆ y.carrier := by
      intro s hs
      rcases (mem_unfoldFinset A).1 (Finset.mem_coe.1 hs) with ⟨t, ht, rfl⟩
      exact hY (Finset.mem_coe.2 ht)
    have : SumCon A (TreeProd A) (unfoldFinset A Y) := y.consistent _ him
    exact SumCon_image_treeCon A this
  closed := by
    intro Y p hY hEnt
    have him : ↑(unfoldFinset A Y) ⊆ y.carrier := by
      intro s hs
      rcases (mem_unfoldFinset A).1 (Finset.mem_coe.1 hs) with ⟨t, ht, rfl⟩
      exact hY (Finset.mem_coe.2 ht)
    have hS : SumEnt A (TreeProd A) (unfoldFinset A Y) (treeUnfold A p) :=
      TreeEnt_sumEnt A hEnt
    exact y.closed _ _ him hS

end DomainTheory.BridgeTheorems
```

6.9.28 Unfolding after folding an element

Theorem (`unfoldElement_foldElement`). Unfolding after folding recovers the original sum-of-product element.

Proof. Finite-set folding and unfolding are inverse on carriers of elements, and saturation preserves closedness. ■

`unfoldElement_foldElement.lean`

```
module

public import DomainTheory.BridgeTheorems.unfoldElement
public import DomainTheory.BridgeTheorems.foldElement
public import DomainTheory.BridgeTheorems.treeUnfold_treeFold

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem unfoldElement_foldElement (y : (treeRhs A).Element) :
    unfoldElement A (foldElement A y) = y := by
  refine le_antisymm ?_ ?_
  · intro s hs
    rcases hs with ⟨t, ht, rfl⟩
    exact ht
  · intro s hs
    refine ⟨treeFold A s, ?_, treeUnfold_treeFold A s⟩
    change treeUnfold A (treeFold A s) ∈ y.carrier
    rwa [treeUnfold_treeFold]
```

```
end DomainTheory.BridgeTheorems
```

6.9.29 Folding after unfolding an element

Theorem (`foldElement_unfoldElement`). Folding after unfolding recovers the original tree element.

Proof. Finite-set folding and unfolding are inverse on carriers of elements. ■

`foldElement_unfoldElement.lean`

```
module

public import DomainTheory.BridgeTheorems.unfoldElement
public import DomainTheory.BridgeTheorems.foldElement
public import DomainTheory.BridgeTheorems.mem_closed_pair_bots
public import DomainTheory.BridgeTheorems.saturated_of_closed
public import DomainTheory.BridgeTheorems.treeFold_eq_or_pairBots

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

theorem foldElement_unfoldElement (x : (treeSystem A).Element) :
    foldElement A (unfoldElement A x) = x := by
  refine le_antisymm ?_ ?_
  · intro t ht
    rcases ht with ⟨t', ht', hU⟩
    have : treeFold A (treeUnfold A t') ∈ x.carrier := saturated_of_closed A x ht'
    have hft : treeFold A (treeUnfold A t) ∈ x.carrier := by
      rwa [hU] at this
    rcases treeFold_eq_or_pairBots A t with h | ⟨rfl, hf⟩
    · rwa [h] at hft
    · exact (mem_closed_pair_bots A x).1 (by rwa [hf] at hft)
  · intro t ht
    exact ⟨t, ht, rfl⟩

end DomainTheory.BridgeTheorems
```

6.9.30 A tree domain and the separated sum of its square

Definition (`treeDomainIso`). Elements of a tree information system are order-isomorphic to elements of the separated sum of the system with its square, via folding and unfolding of elements.

`treeDomainIso.lean`

```
module

public import DomainTheory.BridgeTheorems.unfoldElement_foldElement
public import DomainTheory.BridgeTheorems.foldElement_unfoldElement

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

variable {α : Type*} [DecidableEq α] (A : InfoSys α)

def treeDomainIso : (treeSystem A).Element ≃o (treeRhs A).Element where
  toFun := unfoldElement A
  invFun := foldElement A
  left_inv := foldElement_unfoldElement A
  right_inv := unfoldElement_foldElement A
  map_rel_iff' := by
    intro x₁ x₂
    constructor
    · intro h t ht
      have : treeUnfold A t ∈ (unfoldElement A x₁).carrier := ⟨t, ht, rfl⟩
      have : treeUnfold A t ∈ (unfoldElement A x₂).carrier := h this
      have : t ∈ (foldElement A (unfoldElement A x₂)).carrier := this
      rwa [foldElement_unfoldElement] at this
    · intro h s hs
      rcases hs with ⟨t, ht, rfl⟩
      exact ⟨t, h ht, rfl⟩

end DomainTheory.BridgeTheorems
```

6.10 Illustrative example

<!DOCTYPE svg PUBLIC “-//W3C//DTD SVG 1.1//EN” “http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd”>



Figure 73: Diagram 38

Landings for the S-expression / tree domain (`SexSys` over `lowerBoundSystem`). These are the compared endpoints of the example; everything else in this section builds or unpacks them.

Landing	Claim	Where
`sexNeighborhoodIso`	`SexSys.Element` ≃o filters of basic opens (**1982 ≃o 1980**)	Neighbourhood filters, ideals, and the identity
`sexIdealIso`	`SexSys.Element` ≃o `Ideal (FiniteElement SexSys)` (**1982 ≃o algebraic / 1972 carrier**)	same note
`sexNeighborhoodIdealIso`	filters ≃o ideals (**1980 ≃o 1972 carrier**), closing the triangle	same note
`sexDomainEquationIso`	`SexSys.Element` ≃o `SexRhs.Element` (**`T ≃o A + (T × T)`**)	The tree equation as an order isomorphism
`exists_ sexNeighborhoodIso`	nonempty wrapper for the neighbourhood landing	Existence of the neighbourhood-filter isomorphism
`exists_sexIdealIso`	nonempty wrapper for the ideal landing	Existence of the ideal isomorphism
`exists_ sexDomainEquationIso`	nonempty wrapper for the domain equation	Existence of the tree-equation isomorphism

Secondary (not the compared fixed-point landing): `sexRhsSemanticIso` is the generic semantic factor `WithBot (|A| ⊕ (|T| × |T|)) ≃o |A + (T × T)|` in The semantic factor of the separated sum.

6.10.1 The tree domain in three presentations

Landings (read this first) On this example the compared endpoints are three order isomorphisms of carriers, plus one domain equation. Their Lean homes in this barrel:

Landing	Years	Lean	Note
`sexNeighborhoodIso`	1982 ≃o 1980	`WorkedExampleSExpr.lean`	specialises `InfoSysToNeighborhood.domain`
`sexIdealIso`	1982 ≃o 1972 carrier	`WorkedExampleSExpr.lean`	specialises `InfoSysToIdealCompletion.dom`
`sexNeighborhoodIdealIso`	1980 ≃o 1972 carrier	`WorkedExampleSExpr.lean`	specialises `neighborhood_ ideal_iso`
`sexDomainEquationIso`	T ≃o A+(T×T)	`WorkedExampleSExpr.lean`	specialises `treeDomainIso`

Existence wrappers (`exists_sexNeighborhoodIso`, `exists_sexIdealIso`, `exists_sexDomainEquationIso`) live in the matching `exists_sex*.lean` notes. The uncompared semantic factor `sexRhsSemanticIso` is **not** a landing of the fixed-point equation.

Lean packaging: `WorkedExampleSExpr.lean`, `SexDomainEquation.lean`. The Palomar compared family includes `sexNeighborhoodIso`, `sexIdealIso`, and `sexDomainEquationIso` (Challenge / `comparator.json`). The compared domain-equation declaration is the order isomorphism `SexSys.Element ≃o SexRhs.Element` (`|T| ≃o |A + (T × T)|`), not the generic semantic factor `WithBot (|A| ⊕ (|T| × |T|)) ≃o |A + (T × T)|`.

Overview The bridges of §§2–3 are abstract. This section fixes **one** concrete domain — Scott's S-expression / tree equation `T ≅ A + (T × T)` ([3], Factoid 8.1) over the ℕ lower-bound atom system (Factoid 2.4) — and asks what that domain looks like in each paradigm:

Paradigm	What an element of `T` is
1982	a consistent, deductively closed set of tree tokens
1980	a filter of basic opens `[u]` on that token domain
1972	a point of an algebraic continuous lattice (ideal of finite elements)

We build the domain natively as an information system (§5.2), then **read the same carrier** as a neighbourhood system (§5.3) and as a continuous lattice (§5.4). Section 5.5 exhibits the order isomorphisms for this instance — so that, on S-expressions,

`T_1982 ≃o T_1980 ≃o Ideal(K(T))` and `Ideal(K(T))` is a continuous lattice,

hence transitively `T_1982 ≃o T_1980` and (via the algebraic ⇒ continuous frontier) the 1972 presentation of the same domain. Order isomorphism is an equivalence relation, so the three presentations determine one domain up to ≃o. Section 5.6 records two extras that live on top of the carriers: the domain equation at the level of domains, and identity as a morphism in both the approximable-map and Scott-continuous languages.

Information system (1982) Atoms are propositions `n ≤ x` on $\mathbb{N}$ (`lowerBoundSystem`). The tree system `SexSys := treeSystem lowerBoundSystem` has inductive tokens `TreeToken ℕ` (`bot` / `atom` / `pairL` / `pairR`). Consistency and entailment are Scott's sum×product clauses; Factoid 8.1 records that this is literally the information system of the right-hand side `A + (T × T)`:

- `SexRhs = sumSystem A (productSystem T T)` (`sexRhs_eq_sum_product`);
- token unfolding `treeUnfold` sends `atom n` to the left summand (`sexUnfold_atom`) and is a retraction onto the official sum-of-product carrier, with section `treeFold`;
- TreeEnt matches official product `ent_bot` on the two encodings of `(Δ, Δ)` (`pairL Δ` and `pairR Δ`), so closed elements are saturated for `ker(treeUnfold)`.

Finite elements include singleton atom closures `sexAtom n = ū` for `u = {atom n}`. Elements of `|T|` are the consistent closed sets of tokens — the 1982 presentation of the S-expression domain.

Neighbourhood system (1980) From `|T|` one obtains a neighbourhood system whose basic opens are `[u] = {x ∈ |T| | ↑u ⊆ x}`. In this paradigm an element of the domain is a **filter** of such opens (not a raw set of tokens). Nothing new is invented: it is the image of `SexSys` under `InfoSysToNeighborhood`. The concrete iso of §5.5 below is exactly "read each token-element as its filter of basic opens."

Continuous lattice (1972) Finite elements `K(T)` are the closures of consistent finsets. In the algebraic style that feeds Scott's 1972 theory, a domain element is an **ideal** of `K(T)` — the ideal completion of the compact basis. That poset of ideals is an algebraic complete lattice, hence a continuous lattice (`AlgebraicLattice ⇒ ContinuousLattice`, §3.5). This is the 1972 presentation of the same S-expression domain: points ordered by inclusion of ideals, with way-below recovered from the compact basis.

We do not claim a choice-free `D ≃o RoundFilter` story built from trees alone; the round ⇑-filter identification of §3.1 applies once one is already in the continuous-lattice setting.

Equivalence for this instance Specializing the bridges of §3 to `SexSys` yields order isomorphisms of carriers:

- `sexNeighborhoodIso : |T| ≃o` filters of `[u]` (`InfoSysToNeighborhood.domainOrderIso`) — **1982 ≃o 1980**;
- `sexIdealIso : |T| ≃o Ideal (FiniteElement T)` (`InfoSysToIdealCompletion.domainOr`) — **1982 ≃o algebraic / 1972 carrier**;
- `sexNeighborhoodIdealIso` (`neighborhood_ideal_iso`) — the constructive triangle closing **1980 ≃o Ideal(K(T))** without going back through tokens.

Composing any two edges recovers the third: isomorphism is transitive, so on this example the three presentations are pairwise equivalent. The 1972 corner uses the algebraic ⇒ continuous implication already flagged in §3.5 / §4. The neighbourhood ↔ information ↔ ideal triangle inherits `Classical.choice` only through the `Finset` instance frontier of §4.

Domain equation and morphisms Beyond identifying the carriers, the same example exercises constructions and maps.

Domain equation. Factoid 8.1's token unfolding `treeUnfold` (section `treeFold`) lifts to an order isomorphism of the tree domain with the official RHS domain:

`sexDomainEquationIso : SexSys.Element ≃o SexRhs.Element`

(`treeDomainIso`; `|T| ≃o |A + (T × T)|`). The maps are verified unfolding: the carrier of `sexDomainEquationIso x` is `treeUnfold '' x.carrier`, and the carrier of the inverse is the preimage `treeUnfold ⁻¹' y.carrier` (`sexDomainEquationIso_unfold` / `sexDomainEquationIso_fold`). The only non-injectivity of `treeUnfold` is Scott's two encodings of the product bottom (`pairL Δ` and `pairR Δ`); after the `ent_bot` match, closed elements are saturated for that kernel, so image and preimage invert. The generic semantic factor `WithBot (|A| ⊕ (|T| × |T|)) ≃o SexRhs.Element` is the uncompared `sexRhsSemanticIso` (classical only through the 1982 sum trichotomy); it is **not** the compared fixed-point equation.

Morphisms. The identity approximable map `idMap T` (Prop 5.4) transports along Factoid 4.6 to a Scott-continuous endomap `sexIdScottContinuous` with `toFun = id` (`sexId_toElement`) — the same endomap, named once in each morphism language.

Axioms. Object-level neighbourhood / ideal isos, `sexDomainEquationIso`, and Factoid 4.6 for `idMap` audit to `{propext, Classical.choice, Quot.sound}` through the InfoSys/`Finset` frontier of §4. The uncompared semantic factor `sexRhsSemanticIso` additionally uses the 1982 sum trichotomy.

6.10.2 The lower-bound system of the natural numbers

Definition (`lowerBoundSystem`). The lower-bound information system on the natural numbers, whose tokens are natural numbers ordered by the usual numerical order.

`lowerBoundSystem.lean`

```
module

public import DomainTheory.InformationSystems.InformationSystems.a_first_example
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition54
public import DomainTheory.BridgeTheorems.ScottMapBridge
public import DomainTheory.BridgeTheorems.SexDomainEquation

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

def lowerBoundSystem : InfoSys ℕ where
  bot := 0
  Con := Set.univ
  Ent := lowerBoundEnt
  con_subset := by
    intro u v _ _
    exact Set.mem_univ v
  con_sing := by
    intro _
```

```
    exact Set.mem_univ _
  ent_con := by
    intro u a _
    exact Set.mem_univ _
  ent_bot := by
    intro u _
    exact Or.inl rfl
  ent_refl := by
    intro u a _ ha
    exact Or.inr ⟨a, ha, le_rfl⟩
  ent_trans := by
    intro u v c _ _ hvEnt huEnt
    rcases huEnt with rfl | ⟨n, hn, hcn⟩
    · exact Or.inl rfl
    · rcases hvEnt n hn with rfl | ⟨k, hk, hnk⟩
      · exact Or.inl (Nat.le_zero.mp hcn)
      · exact Or.inr ⟨k, hk, le_trans hcn hnk⟩

end DomainTheory.BridgeTheorems
```

6.10.3 The tree information system

Definition (`SexSys`). The tree information system over the lower-bound system of the natural numbers.

SexSys.lean

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.lowerBoundSystem

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

abbrev SexSys : InfoSys (TreeToken ℕ) :=
  treeSystem lowerBoundSystem

end DomainTheory.BridgeTheorems
```

6.10.4 The sum-of-product information system

Definition (`SexRhs`). The separated sum of the tree system with the product of the tree system with itself — the right-hand side of the tree domain equation.

SexRhs.lean

```
module
```

```
public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.SexSys

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.InfoSys

abbrev SexRhs : InfoSys (SumToken ℕ (ProdToken SexSys SexSys)) :=
  treeRhs lowerBoundSystem

end DomainTheory.BridgeTheorems
```

6.10.5 Singleton atom elements

Definition (`sexAtom`). The element of the tree system obtained as the closure of a singleton natural-number atom.

`sexAtom.lean`

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.SexSys

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

noncomputable def sexAtom (n : ℕ) : SexSys.Element :=
  SexSys.closure {TreeToken.atom n} (SexSys.con_sing _)

end DomainTheory.BridgeTheorems
```

6.10.6 The tree equation as an order isomorphism

Definition (`sexDomainEquationIso`). The order isomorphism between elements of the tree system and elements of its separated-sum-of-product right-hand side, specialised from the general tree domain isomorphism.

`sexDomainEquationIso.lean`

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.SexRhs

@[expose] public section

namespace DomainTheory.BridgeTheorems
```

```
noncomputable def sexDomainEquationIso :
    SexSys.Element ≃o SexRhs.Element :=
  treeDomainIso lowerBoundSystem

end DomainTheory.BridgeTheorems
```

6.10.7 The semantic factor of the separated sum

Definition (`sexRhsSemanticIso`). The order isomorphism identifying the right-hand side with the lifted disjoint union of the tree domain and the product of the tree domain with itself.

`sexRhsSemanticIso.lean`

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.SexRhs

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems.InfoSys

noncomputable def sexRhsSemanticIso :
    WithBot (lowerBoundSystem.Element ? (SexSys.Element × SexSys.Element)) ≃o
      SexRhs.Element :=
  let ιProd := InfoSysConstructions.productDomainIso SexSys SexSys
  let ιSum := InfoSysConstructions.sumDomainIso lowerBoundSystem (productSystem SexSys
    SexSys)
  let mid :=
    (OrderIso.refl lowerBoundSystem.Element).sumCongr ιProd |>.withBotCongr
  mid.trans ιSum

end DomainTheory.BridgeTheorems
```

6.10.8 Neighbourhood filters, ideals, and the identity on the tree domain

Definition (`sexNeighborhoodIso`). Elements of the tree system are order-isomorphic to filters of its basic-open neighbourhood system.

Definition (`sexIdealIso`). Elements of the tree system are order-isomorphic to ideals of its finite elements.

Definition (`sexNeighborhoodIdealIso`). Filters of the basic-open neighbourhood system are order-isomorphic to ideals of finite elements, by composing the two previous isomorphisms.

Definition (`sex_bridge_1982_1980`). Alias for `sexNeighborhoodIso`, the SExpr landing 1982 ≃o 1980.

Definition (`sex_bridge_1982_1972`). Alias for `sexIdealIso`, the SExpr landing 1982 ≃o the 1972 algebraic carrier.

Definition (`sexIdScottContinuous`). The identity function on tree elements, packaged as a Scott-continuous map.

Basic.lean

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.sexAtom
public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.sexRhsSemanticIso
public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.sexDomainEquationIso

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems.InfoSys
open Order

noncomputable abbrev sexNeighborhoodIso :
    SexSys.Element ≃o (InfoSysToNeighborhood.toNeighborhoodSystem SexSys).Element :=
  InfoSysToNeighborhood.domainOrderIso SexSys

noncomputable abbrev sexIdealIso :
    SexSys.Element ≃o Ideal (InfoSysToIdealCompletion.FiniteElement SexSys) :=
  InfoSysToIdealCompletion.domainOrderIso SexSys

noncomputable abbrev sexNeighborhoodIdealIso :=
  neighborhood_ideal_iso SexSys

/-- SExpr landing: 1982 ≃o 1980. -/
noncomputable abbrev sex_bridge_1982_1980 := sexNeighborhoodIso

/-- SExpr landing: 1982 ≃o 1972 algebraic carrier. -/
noncomputable abbrev sex_bridge_1982_1972 := sexIdealIso

noncomputable abbrev sexIdScottContinuous : ScottContinuous SexSys SexSys :=
  (approximableMap_scottContinuous_equiv SexSys SexSys) (idMap SexSys)
end DomainTheory.BridgeTheorems
```

6.10.9 The right-hand side is a separated sum of a product

Theorem (`sexRhs_eq_sum_product`). The right-hand side information system is definitionally the separated sum of the product of the tree system with itself.

Proof. Specialise the general identity for tree right-hand sides. ■

sexRhs_eq_sum_product.lean

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems.InfoSys

theorem sexRhs_eq_sum_product :
```

```
    SexRhs = sumSystem lowerBoundSystem (productSystem SexSys SexSys) :=
  treeRhs.eq_sum_product lowerBoundSystem

end DomainTheory.BridgeTheorems
```

6.10.10 An atom belongs to its singleton closure

Theorem (`sexAtom_mem_self`). A natural-number atom belongs to the carrier of its singleton closure.

Proof. Closure contains the generating consistent set. ■

`sexAtom_mem_self.lean`

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems.InfoSys
open treeSystem

theorem sexAtom_mem_self (n : ℕ) : TreeToken.atom n ∈ (sexAtom n).carrier :=
  closure.subset SexSys (SexSys.con_sing _) (Finset.mem_singleton_self _)

end DomainTheory.BridgeTheorems
```

6.10.11 Unfolding an atom into the left summand

Theorem (`sexUnfold_atom`). Unfolding a singleton atom along the tree equation lands in the left summand.

Proof. Atoms unfold to left injections of atoms. ■

`sexUnfold_atom.lean`

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems.InfoSys

theorem sexUnfold_atom (n : ℕ) :
    treeUnfold lowerBoundSystem (.atom n) = SumToken.left n :=
  treeUnfold.atom lowerBoundSystem n

end DomainTheory.BridgeTheorems
```

6.10.12 Unfolding along the tree equation

Theorem (`sexDomainEquationIso_unfold`). The forward map of the tree-equation isomorphism is unfolding of elements.

Proof. Unfold the definition of the specialised domain isomorphism. ■

`sexDomainEquationIso_unfold.lean`

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems.InfoSys

theorem sexDomainEquationIso_unfold (x : SexSys.Element) :
    (sexDomainEquationIso x).carrier =
      treeUnfold lowerBoundSystem '' x.carrier :=
  rfl

end DomainTheory.BridgeTheorems
```

6.10.13 Folding along the tree equation

Theorem (`sexDomainEquationIso_fold`). The inverse map of the tree-equation isomorphism is folding of elements.

Proof. Unfold the definition of the specialised domain isomorphism. ■

`sexDomainEquationIso_fold.lean`

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems.InfoSys

theorem sexDomainEquationIso_fold (y : SexRhs.Element) :
    (sexDomainEquationIso.symm y).carrier =
      treeUnfold lowerBoundSystem ^-1' y.carrier :=
  rfl

end DomainTheory.BridgeTheorems
```

6.10.14 Existence of the tree-equation isomorphism

Theorem (`exists_sexDomainEquationIso`). There exists an order isomorphism between the tree domain and the right-hand side of the tree equation.

Proof. The packaged tree-equation isomorphism supplies one. ■

`exists_sexDomainEquationIso.lean`

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems

theorem exists_sexDomainEquationIso :
    Nonempty (SexSys.Element ≃o SexRhs.Element) :=
  ⟨sexDomainEquationIso⟩

end DomainTheory.BridgeTheorems
```

6.10.15 Existence of the neighbourhood-filter isomorphism

Theorem (`exists_sexNeighborhoodIso`). There exists an order isomorphism between tree elements and filters of basic opens.

Proof. The packaged neighbourhood isomorphism supplies one. ■

`exists_sexNeighborhoodIso.lean`

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems

theorem exists_sexNeighborhoodIso :
    Nonempty (SexSys.Element ≃o
      (InfoSysToNeighborhood.toNeighborhoodSystem SexSys).Element) :=
  ⟨sexNeighborhoodIso⟩

end DomainTheory.BridgeTheorems
```

6.10.16 Existence of the ideal isomorphism

Theorem (`exists_sexIdealIso`). There exists an order isomorphism between tree elements and ideals of finite elements.

Proof. The packaged ideal isomorphism supplies one. ■

`exists_sexIdealIso.lean`

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems

open Order

theorem exists_sexIdealIso :
    Nonempty (SexSys.Element ≃o
      Ideal (InfoSysToIdealCompletion.FiniteElement SexSys)) :=
  ⟨sexIdealIso⟩

end DomainTheory.BridgeTheorems
```

6.10.17 The identity map on tree elements

Theorem (`sexId_toElement`). The underlying function of the identity Scott-continuous map is the identity on tree elements.

Proof. Unfold the packaging of the identity map. ■

`sexId_toElement.lean`

```
module

public import DomainTheory.BridgeTheorems.WorkedExampleSExpr.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems.InfoSys

theorem sexId_toElement (x : SexSys.Element) :
    sexIdScottContinuous.toFun x = x :=
  idMap.toElement SexSys x

end DomainTheory.BridgeTheorems
```

7 Acknowledgments

7.1 AI-assisted development

The human author retains sole responsibility for the mathematical content, the choice of formalization route, and every formal claim in this work. Following standard publisher practice (e.g., COPE guidance on authorship and AI tools [5]), **no large language model is listed as a co-author** — authorship implies an accountability that automated systems cannot bear.

We gratefully acknowledge assistance from the following tools in this revision (auto-generated from `scripts/ai_model_cards.py` when building `arxiv.tex`). Source versions of the four Scott papers and the bridge theorems record their own model cards in their respective writeups under `vendor/`:

- **Cursor Composer 2.5** [6] — routine multi-step work: module scaffolding, dependency-ordered wiring of `DomainTheory/`, documentation and Mermaid blueprints, and medium proof obligations where the strategy was already fixed (`composer-2.5`).
- **Cursor Grok 4.7** [7] — primary agent model for the DomainTheory sessions: domain-theory constructions, inventory design in `arxiv.md`, constructivity audits, and adaptation of portable patterns from prior Scott formalizations. Used here via the Cursor agent environment.
- **Google Gemini 3.8** [8] — exploratory and support passes (typographic conventions, scope decisions, and cross-checks against the narrative inventory).

All definitions, constructivity audits, and final prose were reviewed by the human author, who takes full responsibility for them.

7.2 Artifact availability and reproducibility

The development is at `github.com/catskillsresearch/DomainTheory`. GitHub Actions CI runs `lake build` on `main`.

Locally:

```
lake exe cache get
lake build DomainTheory
```

Sibling packages `scott1972`, `scott1976`, `scott1980`, `scott1982`, and `scott_models` are vendored under `vendor/` (frozen revisions in `vendor/FROZEN.txt`); they are not Lake path dependencies. `arxiv.md` is the durable inventory and proof narrative. Axiom audits: `#print axioms` on the compared names in `comparator.json`.

Build the CMU-CS technical report PDF (Lean Code appendix = GitHub hyperlinks):

```
./aa.sh                              # DomainTheory.md → DomainTheory.pdf
# or: bash scripts/build_arxiv_pdf.sh
```

8 References

References

[1] D. Scott. Continuous lattices, 1972. in F. W. Lawvere (ed.), Toposes, Algebraic Geometry and Logic, Lecture Notes in Mathematics, vol. 274, Springer-Verlag, pp. 97–136. [Also: Tech. Monograph PRG-7, Oxford Univ. Computing Lab., Programming Research Group (1971).].

[2] D. Scott. Lectures on a mathematical theory of computation, 1981. Technical Monograph PRG-19, May 1981, Oxford University Computing Laboratory, Programming Research Group (neighbourhood systems).

[3] D. Scott. Domains for denotational semantics, 1982. ICALP 1982, LNCS 140, Springer, pp. 577–613.

[4] Palomar Registry. entry `PALOMAR-2026-08-25-000003`, version 1, 2026. Verified claim: order isomorphisms relating Scott's three presentations of domains — continuous lattices (1972), neighbourhood systems (PRG-19), and information systems (1982) — including the round identification `D ≃o RoundFilter ≃o RoundInfoSysElement` and the S-expression equation `T ≅ A + (T × T)` as `|T| ≃o |A + (T × T)|` (`SexSys.Element ≃o SexRhs.Element`). https://palomar-registry.org/entry?id=PALOMAR-2026-08-25-000003&version=1.

[5] Committee on Publication Ethics (COPE). Authorship and ai tools: Cope position statement, 2024. 2024. https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools.

[6] Anysphere, Inc. Composer 2.5. Model announcement and documentation, https://cursor.com/blog/composer-2-5; model card as integrated in Cursor, https://cursor.com/docs/models, 2026. accessed 2026.

[7] xAI and Anysphere, Inc. Grok 4.7. Model documentation, https://docs.x.ai/docs/models; Cursor model catalog, https://cursor.com/docs/models, 2026. accessed 2026.

[8] Google DeepMind. Gemini 3.8. Technical documentation and model cards, https://ai.google.dev/gemini-api/docs/models, 2026. accessed 2026.

[9] ed J. C. Abbott. Trends in lattice theory, 1970. Van Nostrand Reinhold Mathematical Studies, vol. 31 (1970).

[10] S. Abramsky and A. Jung. Domain theory, 1994. In Handbook of Logic in Computer Science, Vol. 3, Oxford University Press, 1994.

[11] M. Aiello L. Aiello and R. W. Weyhrauch. The semantics of pascal in lcf, 1974. Artificial Intelligence Lab. Memo. AIM-221, Stanford Univ., Stanford, Calif.

[12] S. Alagić. Categorical theory of tree processing, 1974. Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 80–99.

[13] P. Alexandroff and H. Hopf. Topologie i, 1935. Springer-Verlag, (1935).

[14] M. A. Arbib and E. G. Manes. Fuzzy morphisms in automata theory, 1974. 1974a; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 98–105.

[15] M. A. Arbib and E. G. Manes. Categorists' view of automata and systems, 1974. 1974b; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 62–79.

[16] M. A. Arbib and E. G. Manes. Basic concepts of category theory applicable to computation and control, 1974. 1974c; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 2–41.

[17] J. W. de Bakker. Semantics of programming languages, 1969. Advances in Information Systems Science, vol. 2, Plenum Press, New York.

[18] J. W. de Bakker and D. Scott. A theory of programs, 1969. Unpublished notes, IBM Seminar, Vienna, 1969.

[19] J. W. de Bakker. Recursive procedures, 1971. 1971a; , Mathematical Centre Tracts, vol. 24, Amsterdam.

[20] J. W. de Bakker. Recursion, induction and symbol manipulation, 1971. 1971b; , Proc. MC-25 Informatica Symposium, Mathematical Centre Tracts, vol. 38, Amsterdam.

[21] J. W. de Bakker and W. P. de Roever. A calculus for recursive program schemes, 1972. Proc. IRIA Colloq. on Automata, North-Holland, Amsterdam.

[22] J. W. de Bakker and L. G. L. T. Meertens. On the completeness of the inductive assertion method, 1973. Mathematical Centre Rep. IW 12/72, Amsterdam.

[23] J. W. de Bakker and L. G. L. T. Meertens. Fixed points in programming theory, 1974. Foundations of Computer Science, J. W. de Bakker, ed., Mathematical Centre Tract, vol. 63, Amsterdam.

[24] H. P. Barendregt. Some extensional term models for combinatory logics and λ-calculi, 1971. Ph.D. thesis, Univ. of Utrecht.

[25] H. Bekić. Definable operations in general algebra, and the theory of automata and flowcharts, 1969. Rep., IBM Lab., Vienna.

[26] H. Bekić. Towards a mathematical theory of processes, 1971. Tech. Rep. TR 25.125, IBM Lab., Vienna.

[27] H. Bekić et al. A formal definition of a pl/i subset, parts i and ii, 1974. Tech. Rep. TR 25.139, IBM Lab., Vienna.

[28] E. Binz and H. H. Keller. Funktionenräume in der kategorie der limesräume, 1966. Annales Academiae Scientiarum Fennicae, Series A, I. Mathematica, no. 383 (1966), 21 pp.

[29] R. S. Bird. Unsolvability of the inclusion problem for dbs schemas, 1974. Rep. RCS22, Dept. of Computer Sci., Univ. of Reading.

[30] G. Birkhoff. Lattice theory, 1967. 3rd ed., American Mathematical Society Colloquium Publications, vol. 25, Providence, R.I.

[31] A. Blikle. Algorithmically definable functions: A contribution towards the semantics of programming languages, 1971. Dissertationes Math. Rozprawy Mat., 85.

[32] A. Blikle. Equational languages, 1972. Information Control, pp. 134–147.

[33] A. Blikle and A. Mazurkiewicz. An algebraic approach to the theory of programs, algorithms, languages and recursiveness, 1972. Proc. Internat. Sympos. and Summer School on the Mathematical Foundations of Computer Sci., Warsaw-Jabłonna.

[34] A. Blikle. An algebraic approach to programs and their computations, 1973. Proc. Symp. and Summer School on the Mathematical Foundations of Computer Sci., High Tatras, Czechoslovakia.

[35] A. Blikle. Proving programs by sets of computations, 1974. Proc. 3rd Symp. on the Mathematical Foundation of Computer Sci., Jadwisin, Poland; Lecture Notes in Computer Science, Springer-Verlag, Berlin, 1974.

[36] C. Böhm. The cuch as a formal and descriptive language, 1966. Formal Language Description Languages for Computer Programming, T. B. Steel, ed., North-Holland, Amsterdam, pp. 179–197.

[37] C. Böhm. Alcune proprietà delle forme β-η-normali nel λ-κ-calcolo, 1968. Consiglio Nazionale delle Ricerche: Pubblicazione 696 dell' Istituto per le Applicazioni del Calcolo, Roma.

[38] C. Böhm. λ-calculus and computer science theory, 1975. Proc. Rome Symp., March 1975, Lecture Notes in Computer Science, vol. 37, Springer-Verlag, Berlin.

[39] R. M. Burstall. Proving properties of programs by structural induction, 1969. Comput. J., 12, pp. 41–48.

[40] R. M. Burstall. Some techniques for proving the correctness of programs which alter data structures, 1972. 1972a; , Machine Intelligence vol. 7, B. Meltzer and D. Michie, eds., Edinburgh, pp. 23–50.

[41] R. M. Burstall. An algebraic description of programs with assertions, verification and simulation, 1972. 1972b; , Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, New Mexico, pp. 7–14.

[42] R. Burstall and J. W. Thatcher. The algebraic theory of recursive program schemes, 1974. Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 154–160.

[43] J. M. Cadiou. Recursive definitions of partial functions and their computations, 1973. Ph.D. thesis, Computer Sci. Dept., Stanford Univ., Stanford, Calif.

[44] J. M. Cadiou and J. J. Levy. Mechanizable proofs about parallel processes, 1973. Presented at the 14th Ann. Symp. on Switching and Automata Theory.

[45] J. W. Case. Enumeration reducibility and the partial degrees, 1971. Ann. Math. Logic, 2, pp. 419–440.

[46] E. Čech. Topological spaces, 1966. (revised by Z. Frolík and M. Katětov), Prague.

[47] A. K. Chandra and Z. Manna. On the power of programming features, 1973. Artificial Intelligence Memo. AIM-185, Stanford Univ., Stanford, Calif.

[48] A. K. Chandra. Degrees of translatability and canonical forms in program schemas part i, 1974. IBM Res. Rep. RC4733, Yorktown Heights, N.Y.

[49] A. K. Chandra. The power of parallelism and nondeterminism in programming, 1974. 1974a; , IBM Res. Rep. RC4776, Yorktown Heights, N.Y.

[50] A. K. Chandra. Generalized program schemas, 1974. 1974b; , IBM Res. Rep. RC4827, Yorktown Heights, N.Y.

[51] A. Church. The calculi of λ-conversion, 1951. Annals of Mathematical Studies, vol. 6, Princeton University Press, Princeton, N.J.

[52] M. J. Clint. Program proving: Co-routines, 1972. Acta Informatica, 2, pp. 50–63.

[53] R. C. Constable and D. Gries. On classes of program schemata, 1972. this Journal, 1, pp. 66–118.

[54] D. C. Cooper. The equivalence of certain computations, 1966. Comput. J., 9, pp. 45–52.

[55] B. Courcelle and J. Vuillemin. Complétude d'un système formel pour des équations récursives simples, 1974. Compte-Rendu du Colloque de Paris.

[56] B. Courcelle and J. Vuillemin. Semantics and axiomatics of a simple recursive language, 1974. Rapport Laboria, vol. 60, IRIA; Also: 6th Ann. ACM Symp. on the Theory of Computing, Seattle, Washington.

[57] G. Kahn B. Courcelle and J. Vuillemin. Algorithmes d'équivalence pour des équations récursives simples, 1974. Proc. 2nd Colloq. on Automata Languages and Programming, Saarbrücken; Lecture Notes in Computer Science, Springer-Verlag, Berlin, pp. 213–220.

[58] H. B. Curry and R. Feys. Combinatory logic, 1958. vol. 1, North-Holland, Amsterdam.

[59] J. R. Hindley H. B. Curry and J. Seldin. Combinatory logic, 1971. vol. 2, North-Holland, Amsterdam.

[60] M. Davis. Computability and unsolvability, 1958. McGraw-Hill, New York.

[61] E. W. Dijkstra. Notes on structured programming, 1972. Structural Programming, C. A. R. Hoare, E. W. Dijkstra and O. J. Dahl, Academic Press, New York.

[62] E. W. Dijkstra. A simple axiomatic basis for programming language constructs, 1974. Indag. Math., 36, pp. 1–15.

[63] E. W. Dijkstra. A discipline of programming, 1976. Prentice-Hall, Englewood Cliffs, N.J., to appear.

[64] J. Donahue. Scottery child's guide no. 1, 1974. Dept. of Comput. Sci., Univ. of Toronto.

[65] B. N. Duong. A high level, variable free calculus for recursive programming, 1974. Comput. Sci. Dept. Rep. CS 74 03, Univ. of Waterloo.

[66] S. Eilenberg and C. C. Elgot. Recursiveness, 1970. Academic Press, New York.

[67] S. Eilenberg and C. C. Elgot. Automata, languages and machines, 1974. vol. A, Academic Press, New York (vol. B in press).

[68] A. Robinson C. C. Elgot and J. D. Rutledge. Multiple control computer models, 1966. IBM Res. Rep. RC 1622, Yorktown Heights, N.Y.

[69] C. C. Elgot. Abstract algorithms and diagram closure, 1967. IBM Res. Rep. RC 1750, Yorktown Heights, N.Y.

[70] C. C. Elgot. Algebraic theories and program schemes, 1971. Symposium on Semantics of Algorithmic Languages, E. Engeler, ed., Lecture Notes in Mathematics, vol. 188, Springer-Verlag, New York.

[71] M. H. van Emden and R. A. Kowalski. The semantics of predicate logic as a programming language, 1974. Memo. no. 83 (MIP-R-103), Dept of Machine Intelligence, Univ. of Edinburgh.

[72] E. Engeler. Algorithmic properties of structures, 1967. Mathematical Systems Theory, 1, pp. 183–195.

[73] E. Engeler. Formal languages: Automata and structures, 1968. Lectures in Advanced Mathematics, Markham.

[74] E. Engeler. Algorithmic logic, 1974. Foundations of Computer Science, J. W. de Bakker, ed., Mathematical Centre Tract MCT 63, Amsterdam.

[75] J. Engelfriet. Simple program schemes and formal languages, 1974. Springer Lecture Notes in Computer Science, vol. 20.

[76] J. Engelfriet and E. M. Schmidt. Io and oi, 1975. DAIMI PB-47, Matematisk Institut, Aarhus Universitet, Datalogisk Afdeling, Denmark.

[77] Ju. L. Ershov. Computable numerations of morphisms, 1971. Algebra and Logic, 10, pp. 247–308.

[78] Ju. L. Ershov. Computable functions of finite types, 1972. 1972a; Algebra and Logic, 11, pp. 367–437.

[79] Ju. L. Ershov. Everywhere defined (total) continuous functionals, 1972. 1972b; Algebra and Logic, 11, pp. 656–665.

[80] Ju. L. Ershov. Continuous lattices and a-spaces, 1972. 1972d; , Dokl. Akad. Nauk USSR, 207, pp. 523–526.

[81] Ju. L. Ershov. Theory of a-spaces, 1972. 1972e; , Algebra and Logic, 12, pp. 369–416.

[82] Ju. L. Ershov. Theorie der numerierungen, 1973. Veb Deutscher Verlag der Wissenschaften, Berlin.

[83] M. J. Fischer. λ-calculus schemata, 1972. Proc. ACM Conf. Proving Assertions about Programs, Las Cruces, N.M., pp. 104–109.

[84] A. E. Fischer and M. J. Fischer. Mode modules as representations of domains, 1973. Proc. ACM Symp. on Principles of Programming Languages, Boston, pp. 139–143.

[85] R. W. Floyd. Assigning meanings to programs, 1967. Proc. Symp. in Appl. Math., vol. 19, Mathematical Aspects of Computer Science, J. T. Schwartz, ed., pp. 33–41.

[86] M. M. Fokkinga. Inductive assertion patterns for recursive procedures, 1974. Compte-Rendu de Colloque de Paris.

[87] R. M. Friedberg and H. R. Rogers. Reducibility and completeness for sets of integers, 1959. Z. Math. Logik Grundlagen Math., 5, pp. 117–125.

[88] H. Friedman. Algorithmic procedures, generalized turing algorithms, and elementary recursion theory, 1971. Logic Colloquium 1969, North-Holland, Amsterdam, pp. 361–389.

[89] S. J. Garland and D. C. Luckham. Translating recursion schemes into program schemes, 1972. Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, N.M., pp. 83–96.

[90] H. Gericke. Lattice theory, 1966. Harrap.

[91] G. Gierz et al. Continuous lattices and domains, 2003. Cambridge University Press, 2003.

[92] J. A. Goguen. l-fuzzy sets, 1967. J. Math. Anal. Appl., 18, pp. 145–174.

[93] J. A. Goguen. Categories of l-sets, 1969. Bull. Amer. Math. Soc., 75, pp. 524–622.

[94] J. A. Goguen. On homomorphisms, simulations, correctness, subroutines and termination for programs and program schemes, 1972. Proc. 13th IEEE Conf. on Switching and Automata Theory, pp. 52–60.

[95] J. A. Goguen. Semantics of computation, 1974. Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 234–249.

[96] J. A. Goguen et al. Initial algebra semantics, 1975. IBM Res. Rep. RC 5243, Yorktown Heights, N.Y.

[97] M. J. C. Gordon. Models of pure lisp, 1973. 1973a; , Experimental Programming Rep. 31, School of Artificial Intelligence, Univ. of Edinburgh. (Also Ph.D. thesis, Evaluation and denotation of pure LISP programs: A worked example in semantics.).

[98] M. J. C. Gordon. An extended abstract of "models of pure lisp", 1973. 1973b; , Memo. SAI-RM-7, School of Artificial Intelligence, Univ. of Edinburgh.

[99] M. J. C. Gordon. Operational reasoning and denotational semantics, 1975. 1975a; , Artificial Intelligence Memo. 264, Stanford Univ., Stanford, Calif.

[100] M. J. C. Gordon. Towards a semantic theory of dynamic binding, 1975. 1975b; , Artificial Intelligence Memo. 265, Stanford Univ., Stanford, Calif.

[101] G. Grätzer. Universal algebra, 1968. Van Nostrand, New York.

[102] I. Guessarian. Sur une réduction des schémas de programmes polyadiques à des schémas monadiques et ses applications, 1974. Compte-Rendu de Colloque de Paris.

[103] L. Gutteridge. Some results on enumeration reducibility, 1971. Ph.D. dissertation, Simon Fraser Univ., Burnaby, B.C., Canada.

[104] B. Lercher J. R. Hindley and J. P. Seldin. Introduction to combinatory logic, 1972. London Mathematical Society Lecture Note Series, vol. 7, Cambridge University Press, Cambridge, England.

[105] J. R. Hindley and G. Mitschke. Some remarks about the connections between combinatory logic and axiomatic recursion theory, 1975. Preprint 203, Fachbereich Mathematik, Technische Hochschule Darmstadt.

[106] P. Hitchcock and D. M. R. Park. Induction rules and proofs of termination, 1972. Proc. Colloques IRIA, Theorie des Automates des Languages et de la Programmation.

[107] P. Hitchcock. An approach to formal reasoning about programs, 1974. Ph.D. thesis, Univ. of Warwick, England.

[108] C. A. R. Hoare. An axiomatic basis of computer programming, 1969. Comm. ACM, 12, pp. 576–580, 583.

[109] C. A. R. Hoare. Proof of a program: Find, 1971. 1971a; , Comm. ACM, 14, pp. 39–45.

[110] C. A. R. Hoare. Procedures and parameters: An axiomatic approach, 1971. 1971b; , Symposium on Semantics of Programming Languages, E. Engeler, ed., Lecture Notes in Mathematics, vol. 188, Springer-Verlag, New York, pp. 102–116.

[111] C. A. R. Hoare. Notes on data structuring, 1972. Structured Programming, C. A. R. Hoare, E. W. Dijkstra and O. J. Dahl, Academic Press, New York.

[112] C. A. R. Hoare and N. Wirth. An axiomatic definition of the programming language pascal, 1972. Bericht der Fachgruppe Computer-Wissenschaften 6, Eidgenössische Technische Hochschule, Zürich.

[113] C. A. R. Hoare and P. Lauer. Consistent and complementary formal theories of the semantics of programming languages, 1974. Acta Informatica, 3, pp. 135–153.

[114] C. Hosono and M. Sato. A solution to scott's problem: "do the retracts in $p\omega$ form a continuous lattice?", 1975. Research Inst. for Mathematical Sci., Kyoto (preprint).

[115] G. Hotz. Eindeutigkeit und mehrdeutigkeit formaler sprachen, 1966. Electron. Informationsverarbeit. Kybernetik (Berlin), 2, pp. 235–246.

[116] J. M. E. Hyland. Recursion theory on the countable functionals, 1975. 1975a; , D. Phil. thesis, Oxford Univ., Oxford, England.

[117] J. M. E. Hyland. A survey of some useful partial order relations in terms of the lambda calculus, 1975. 1975b; , Proc. Conf. on λ-Calculus and Computer Science Theory, Rome, pp. 83–95.

[118] J. M. E. Hyland. A syntactic characterization of the equality in some models for the lambda calculus. to appear; , J. London Math. Soc.

[119] Y. I. Ianov. The logical scheme of algorithms, 1970. Problems in Cybernetics, vol. 1, Pergamon Press, New York.

[120] S. Igarashi. Admissibility of fixed point induction in first order logic of typed theories, 1972. Artificial Intelligence Memo. AIM-168, Computer Sci. Dept., Stanford Univ., Stanford, Calif.

[121] K. Indermark. Gleichungsdefinierbarkeit in relationalstrukturen, 1974. Habilitationsschrift, Mathematisch-Naturwissenschaftlichen Fakultät der Rheinischen Freidrich-Wilhelms-Universatat, Bonn, W. Germany.

[122] C. B. Jones. Mathematical semantics of goto: Exit formulation and its relation to continuations, 1974. preprint.

[123] G. Kahn. A preliminary theory of parallel programs, 1973. Rapport Laboria IRIA.

[124] D. M. Kaplan. Regular expressions and the equivalence of programs, 1969. J. Comput. System Sci., 3, pp. 361–385.

[125] R. M. Karp and R. E. Miller. Parallel program schemata, 1969. J. Comput. System Sci., 3, pp. 147–195.

[126] J. L. Kelley. General topology, 1955. Van Nostrand, (1955).

[127] D. J. Kfoury. Comparing algebraic structures up to algorithmic equivalence, 1972. Automata, Languages and Programming, M. Nivat, ed., North-Holland, Amsterdam, pp. 253–263.

[128] D. J. Kfoury. Translatability of schemas over restricted interpretations, 1974. J. Comput. System Sci., 8, pp. 387–408.

[129] S. C. Kleene. Introduction to metamathematics, 1950. Van Nostrand, New York.

[130] B. Knaster. Un théorème sur les fonctions d'ensembles, 1928. Ann. Soc. Polon. de Math., 8, pp. 133–134.

[131] P. J. Landin. The mechanical evaluation of expressions, 1964. Comput. J., 6, pp. 308–320.

[132] P. J. Landin. A correspondence between algol 60 and church's lambda notation, 1965. Comm. ACM, 8, pp. 89–101.

[133] P. J. Landin. A λ-calculus approach, 1966. 1966a; , Advances in Programming and Non-numerical Computation, L. Fox, ed., Pergamon Press, New York, pp. 97–141.

[134] P. J. Landin. A formal description of algol 60, 1966. 1966a; , Formal Language Description Languages for Computer Programming, T. B. Steel, ed., North-Holland, Amsterdam, pp. 266–294.

[135] P. J. Landin. The next 700 programming languages, 1966. 1966b; , Comm. ACM, 9, pp. 157–164.

[136] P. J. Landin. A program/machine symmetric automata theory, 1969. Machine Intelligence, vol. 5, B. Meltzer and D. Michie, eds., Edinburgh University Press, pp. 99–120.

[137] P. J. Landin and R. M. Burstall. Programs and their proofs: An algebraic approach, 1969. Machine Intelligence, vol. 4, American Elsevier, New York, pp. 17–44.

[138] J. Leszczykowski. A theorem on resolving equations in the space of languages, 1971. Bull. Acad. Polon. Sci. Sér. Sci. Math. Astronom. Phys., 19, pp. 967–970.

[139] C. H. Lewis and B. K. Rosen. Recursively defined data types, parts i and ii, 1973. 1973–74; IBM Res. Reps. RC 4429 (1973), RC 4713 (1974), Yorktown Heights, N.Y.

[140] B. Liskov and S. Zilles. An approach to abstraction, 1973. Group Memo. 88, Project MAC, Mass. Inst. of Tech., Cambridge, Mass.

[141] D. M. R. Park D. Luckham and M. Paterson. On formalized computer programs, 1970. J. Comput. System Sci., 4, pp. 220–249.

[142] S. Mac Lane. Categories for the working mathematician, 1972. Springer-Verlag, New York.

[143] E. Manes. Category theory applied to computation and control, 1974. Proc. 1st Internat. Symp. Math. Dept. and the Dept. of Computer and Information Sci., Univ. Mass., Amherst. Also: Lecture Notes in Computer Science, vol. 26, Springer-Verlag, New York.

[144] Z. Manna. The correctness of programs, 1969. J. Comput. System Sci., 3, pp. 119–127.

[145] Z. Manna and A. Pnueli. Formalization of properties of functional programs, 1970. J. Assoc. Comput. Mach., 17, pp. 555–569.

[146] Z. Manna and J. McCarthy. Properties of programs and partial function logic, 1970. Machine Intelligence 5, B. Meltzer and D. Michie, eds., Edinburgh University Press, pp. 27–38.

[147] Z. Manna and A. Pnueli. Axiomatic approach to total correctness of programs, 1972. Artificial Intelligence Lab. Memo. AIM-210, Stanford Univ., Stanford, Calif.

[148] Z. Manna and J. M. Cadiou. Recursive definitions of partial functions and their computations, 1972. Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, N.M., pp. 58–65.

[149] Z. Manna and J. Vuillemin. Fixpoint approach to the theory of computation, 1972. Comm. ACM, 15, pp. 528–536.

[150] Z. Ness Z. Manna and J. Vuillemin. Inductive methods for proving properties of programs, 1972. Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, N.M., pp. 27–50.

[151] Z. Manna. Mathematical theory of computation, 1974. McGraw-Hill, New York.

[152] G. Markowsky. Categories of chain-complete posets, 1974. RC 5100, Comput. Sci. Dept. IBM T. J. Watson Research Center, Yorktown Heights, N.Y.

[153] G. Markowsky. Chain-complete posets and directed sets with applications, 1974. RC 5024, IBM T. J. Watson Research Center, Yorktown Heights, N. Y.

[154] A. Mazurkiewicz. Proving algorithms by tail functions, 1971. Working Paper for IFIP WG2.2, Feb. 1970. Since published in: Information and Control, 18 (1971), pp. 220–226.

[155] A. Mazurkiewicz. Proving properties of processes, 1973. PRACE CO PAN-CC PAS Reports, vol. 134, Warsaw.

[156] J. McCarthy. The lisp 1.5 programmers' manual, 1962. MIT Press, Cambridge, Mass.

[157] J. McCarthy. A basis for a mathematical theory of computation, 1963. 1963a; , Computer Programming and Formal Systems, D. Braffort and D. Hirshberg, eds., North-Holland, Amsterdam, pp. 33–70.

[158] J. McCarthy. Towards a mathematical science of computation, 1963. 1963b; , Information Processing 1962, Proc. IFIP Congress 1962, C. M. Poppleworth, ed., North-Holland, Amsterdam, pp. 21–28.

[159] J. McCarthy. A formal description of a subset of algol, 1966. Formal Language Description Languages for Computer Programming, R. B. Steel, ed., North-Holland, Amsterdam, pp. 1–12.

[160] J. McCarthy and J. Painter. Correctness of a computer for arithmetic expressions, 1967. Mathematical Aspects of Computer Science, J. T. Schwartz, ed., Proc. of a Symposium in Applied Mathematics, vol. 19, pp. 33–41.

[161] E. Michael. Topologies on spaces of subsets, 1951. Transactions of the American Mathematical Society, vol. 77 (1951), pp. 152-182.

[162] R. E. Milne. The formal semantics of computer languages and their implementations, 1974. Ph.D. thesis, Cambridge Univ., Cambridge, England. [Also: Technical Monograph PRG-13, Oxford Univ. Computing Lab., Programming Research Group].

[163] R. Milner. Program schemes and recursive function theory, 1969. Machine Intelligence 5, B. Meltzer and D. Michie, eds., Edinburgh University Press, pp. 39–58.

[164] R. Milner. Algebraic theory of computable polyadic functions, 1970. 1970a; , Computer Sci. Memo., vol. 12, University College, Swansea.

[165] R. Milner. Equivalences on program schemes, 1970. 1970b; , J. Comput. System Sci., 4, pp. 205–219.

[166] R. Milner. Implementation and application of scott's logic for computable functions, 1972. Proc. ACM Conf., Las Cruces, N.M., pp. 1–6.

[167] R. Milner and R. Weyhrauch. Proving compiler correctness in a mechanized logic, 1972. Machine Intelligence, 7, pp. 51–72.

[168] R. Milner. Models of lcf, 1973. Artificial Intelligence Memo. AIM-186, Computer Sci. Dept., Stanford Univ., Stanford, Calif.

[169] R. Milner. An approach to the semantics of parallel programs, 1973. 1973a; , Proc. Convegno di Informatica Teorica, Instituto di Elaborazione delle Informazione, Pisa, Italy.

[170] R. Milner. Processes: A mathematical model of computing agents, 1973. 1973b; , Proc. Colloq in Mathematical Logic, Bristol, England.

[171] J. H. Morris. λ-calculus models of programming languages, 1968. Ph.D. thesis, Sloan School of Management, MIT MAC Reprint TR-57, Mass. Inst. of Tech., Cambridge, Mass.

[172] F. Lockwood Morris. The next 700 programming language descriptions, 1970. Computer Center, Univ. of Essex (typescript).

[173] J. H. Morris. Another recursion induction principle, 1971. Comm. ACM, 14, pp. 351–354.

[174] F. Lockwood Morris. Correctness of translations of programming languages, 1972. Computer Sci. Memo. CS 72-303, Stanford Univ., Stanford, Calif.

[175] J. H. Morris. A correctness proof using recursively defined functions, 1972. Formal Semantics of Programming Languages, R. Rustin, ed., Prentice-Hall, Englewood Cliffs, N.J., pp. 107–224.

[176] F. Lockwood Morris. Advice on structuring compilers and proving them correct, 1973. Proc. SIGACT/SIGPLAN Symp. on Principles of Programming Languages, Boston, pp. 144–152.

[177] J. H. Morris. Types are not sets, 1973. Proc. ACM Symp. on Principles of Programming Languages, Boston, pp. 120–124.

[178] J. Morris and R. Nakajima. Mechanical characterisation of the partial order in lattice models of λ-calculus, 1973. Tech. Rep. 18, Univ. of Calif., Berkeley.

[179] P. D. Mosses. The mathematical semantics of algol 60, 1974. Tech. Monograph PRG-12, Oxford Univ. Computing Lab., Programming Research Group.

[180] P. D. Mosses. The semantics of semantic equations, 1975. 1975a; , Mathematical Foundations of Computer Science, Goos and Hartmanis, eds., Lecture Notes in Computer Science, vol. 28, Springer-Verlag, New York, pp. 409–422.

[181] P. D. Mosses. Mathematical semantics and compiler generation, 1975. 1975b; , Thesis, Oxford Univ., Oxford, England.

[182] J. Myhill and J. C. Shepherdson. Effective operations on partial recursive functions, 1955. Z. Math. Logik Grundlagen Math., 1, pp. 310–317.

[183] R. Nakajima. Infinite normal forms for λ-calculus, 1975. λ-Calculus and Computer Science Theory, C. Böhm, ed., Springer-Verlag, Berlin, pp. 62–82.

[184] A. Nerode. Some stone spaces and recursion theory, 1959. Duke Mathematical Journal, vol. 26, pp. 397–406.

[185] M. Nivat. Langages algébriques sur le magma libre et sémantique des schémas de programmes, 1972. Proc. IRIA Symposium on Automata Formal Languages and Programming, North-Holland, Amsterdam.

[186] D. C. Oppen. On logic and program verification, 1975. Tech. Rep. 82, Dept. Computer Sci., Univ. of Toronto.

[187] D. M. R. Park. Fixpoint induction and proofs of program properties, 1970. 1970a; , Machine Intelligence, 5, B. Meltzer and D. Michie, eds., American Elsevier, New York, pp. 59–78.

[188] D. M. R. Park. Notes on a formalism for reasoning about schemes, 1970. 1970b; , Univ. of Warwick, unpublished notes.

[189] D. M. R. Park. The y combinator scott's lambda-calculus models, 1970. 1970c; , Univ. of Warwick, unpublished notes.

[190] D. M. R. Park. Finiteness is mu-ineffable, 1970. 1970d; , Theory of Computation Report, No. 3, Dept. of Computer Sci., Univ. of Warwick (1974).

[191] M. S. Paterson. Program schemata, 1963. Machine Intelligence, 3, D. Michie, ed., American Elsevier, New York, pp. 19–32.

[192] M. S. Paterson and C. S. Hewitt. Comparative schematology, 1970. Record of Project MAC Conf. on Concurrent Systems and Parallel Computation, Association for Computing Machinery, New York, pp. 119ff.

[193] R. Platek. New foundations for recursion theory, 1964. Thesis, Stanford Univ., Stanford, Calif., unpublished.

[194] G. D. Plotkin. A set-theoretical definition of application, 1972. Memo. MIP-R-95, School of Artificial Intelligence, University of Edinburgh.

[195] G. D. Plotkin. The λ-calculus is ω-incomplete, 1973. 1973a; , Res. Memo. SAI-RM-2, School of Artificial Intelligence, Univ. of Edinburgh.

[196] G. D. Plotkin. Lambda-definability and logical relations, 1973. 1973b; , Memo. SAI-RM-4, School of Artificial Intelligence, Univ. of Edinburgh.

[197] G. D. Plotkin. Call-by-name, call-by-value and the λ-calculus, 1973. 1973c; , J. Res. Memo. SAI-RM-6, School of Artificial Intelligence, Univ. of Edinburgh.

[198] G. D. Plotkin. A powerdomain construction, 1975. Dept. of Artificial Intelligence Res. Rep. 3, Univ. of Edinburgh.

[199] H. Rasiowa. On ω-1973.-valued algorithmic logic and related problems*, Supplement to Proc. Symp. and Summer School on Mathematical Foundations of Computer Sci., High Tatras, Czechoslovakia.

[200] P. Raulefs. The overtyped lambda calculus, 1975. 1975a; , Interner Bericht Nr. 2, Institut für Informatik, Universität Karlsruhe.

[201] P. Raulefs. Standard models of the overtyped lambda-calculus, 1975. 1975b; , Interner Bericht Nr. 3, Institut für Informatik, Universität Karlsruhe.

[202] R. R. Redziejowski. The theory of general events and its application to parallel programming, 1972. T.P. 18.220 IBM Nordic Laboratory, Sweden.

[203] J. C. Reynolds. Gedanken: A simple typeless language, 1969. Reprint 7621, Argonne National Laboratory, Argonne, Ill.

[204] J. C. Reynolds. Definitional interpreters for higher order programming languages, 1972. 1972a; , Proc. ACM 25th Nat. Conf., Boston, vol. 2, pp. 717–740.

[205] J. C. Reynolds. Notes on a lattice-theoretic approach to the theory of computation, 1972. 1972b; , Systems and Information Sci. Dept., Syracuse Univ., Syracuse, N.Y.

[206] J. C. Reynolds. On the relation between direct and continuation semantics, 1974. 1974a; , Proc. 2nd Colloq. on Automata, Languages and Programming, Saarbrücken, Lecture Notes in Computer Science, vol. 14, Springer-Verlag, Berlin, pp. 141–156.

[207] J. C. Reynolds. Semantics of the lattice of flow diagrams, 1974. 1974b; , Systems and Information Science Dept., Syracuse Univ., Syracuse, N.Y. (preprint).

[208] J. C. Reynolds. On the interpretations of scott's domains. to appear; Symposia Mathematica.

[209] J. C. Reynolds. Towards a theory of type structure. to appear; , Colloq. on Programming, Paris, April 1974, Lecture Notes in Computer Science, Springer-Verlag, Berlin.

[210] W. P. de Roever. Recursion and parameter mechanisms: An axiomatic approach, 1974. 1974a; Proc. 2nd Colloq. on Automata Languages and Programming, Saarbrucken, Lecture Notes in Computer Science, vol. 14, Springer-Verlag, Berlin, pp. 34–65.

[211] W. P. de Roever. Operational, mathematical and axiomatized semantics for recursive procedures and data structures, 1974. 1974b; , Mathematical Centre Report ID/1, Amsterdam.

[212] H. R. Rogers. Theory of recursive functions and effective computability, 1967. McGraw-Hill, New York.

[213] B. K. Rosen. Tree-manipulating systems and church-rosser theorems, 1973. J. Assoc. Comput. Mach., 20.

[214] J. D. Rutledge. Program schemata as automata, part i, 1970. 1970a; IBM Res. RC 3098. J. Comput and System Sci., 7 (1973), pp. 543–578.

[215] J. D. Rutledge. Parallel processes . . . schemata and transformation, 1970. 1970b; , IBM Res. Rep. RC 2912, Yorktown Heights, N.Y.

[216] A. Salwicki. Formalized algorithmic languages, 1970. 1970a; , Bull. Acad. Polon. Sci. Sér. Math. Astronom. Phys., 18, pp. 227–232.

[217] A. Salwicki. On the equivalence of fs-expressions and programs, 1970. 1970b; , Bull. Acad. Polon. Sci. Sér. Math. Astronom. Phys., 18, pp. 275–278.

[218] A. Salwicki. On the predicate calculi with iteration quantifiers, 1970. 1970c; , Bull. Acad. Polon. Sci. Sér. Math. Astronom. Phys., 18, pp. 279–286.

[219] J. G. Sanderson. The λ calculus, lattice theory and reflexive domains, 1973. Mathematical Institute Lecture Notes, Oxford, England.

[220] L. P. Sasso. Degrees of unsolvability of partial functions, 1971. Ph.D. dissertation, Univ. of Calif., Berkeley.

[221] L. P. Sasso. A minimal partial degree, 1973. Proc. Amer. Math. Soc., 38, pp. 388–392.

[222] L. P. Sasso. A survey of partial degrees, 1975. J. Symbolic Logic, 40, pp. 130–140.

[223] D. Scott. A type-theoretical alternative to iswim, cuch, owhy, 1969. Unpublished notes, Oxford, England.

[224] D. Scott. Outline of a mathematical theory of computation, 1970. Proceedings of the Fourth Annual Princeton Conference on Information Sciences and Systems, pp. 169–176. [Also: Tech. Monograph PRG-2, Oxford Univ. Computing Lab., Programming Research Group (1970).].

[225] D. Scott. Lattice theory, data types, and semantics, 1971. New York University Symposia in Areas of Current Interest in Computer Science (Randall Rustin ed.) (1971) to appear.

[226] D. Scott. The lattice of flow diagrams, 1971. Symposium on Semantics of Algorithmic Languages, E. Engeler, ed., Lecture Notes in Mathematics, vol. 188, Springer-Verlag, New York, pp. 311–366. [Also: Tech. Monograph PRG-3, Oxford Univ. Computing Lab., Programming Research Group (1970).].

[227] D. Scott and C. Strachey. Toward a mathematical semantics for computer languages, 1971. Proc. Symposium on Computers and Automata, Polytechnic Inst. of Brooklyn, vol. 21, pp. 19–46. [Also: Technical Monograph PRG-6, Oxford Univ. Computing Lab., Programming Research Group (1971).].

[228] D. Scott. Mathematical concepts in programming language semantics, 1972. 1972b; , AFIPS Conf. Proc., vol. 40, pp. 225–234.

[229] D. Scott. Data types as lattices, 1973. 1973a; , Unpublished lecture notes, Mathematical Centre, Amsterdam, 1973.

[230] D. Scott. Models for various type-free calculi, 1973. 1973b; , Proc. IVth Internat. Cong. for Logic, Methodology and the Philosophy of Science, Bucharest, P. Suppes et al., eds., North-Holland, Amsterdam, pp. 157–187.

[231] D. Scott. Lambda calculus and recursion theory, 1975. 1975a; , Proc. 3rd Scandinavian Logic Symposium, Stig Kanger, ed., North-Holland, Amsterdam, pp. 154–193.

[232] D. Scott. Combinators and classes, 1975. 1975b; , λ-Calculus and computer science theory, Proc. Rome Symp., March 1975; Lecture Notes in Computer Science, vol. 37, Springer-Verlag, Berlin, pp. 1–26.

[233] D. Scott. Some philosophical issues concerning theories of combinators, 1975. 1975c; , λ-Calculus and computer science theory, Proc. Rome Symp., March 1975; Lecture Notes in Computer Science, vol. 37, Springer-Verlag, Berlin, pp. 346–366.

[234] J. C. Shepherdson. Computation over abstract structures: Serial and parallel procedures and friedman's effective definitional schemes, 1975. Logic Colloquium 1973, H. E. Rose and J. C. Shepherdson, eds., North-Holland, Amsterdam, pp. 445–513.

[235] E. Spanier. Quasi-topologies, 1963. Duke Mathematical Journal, vol. 30 (1963) pp. 1-14.

[236] C. Strachey. Towards a formal semantics, 1966. Formal Language Description Languages for Computer Programming, T. B. Steel, ed., North-Holland, Amsterdam, pp. 198–220.

[237] C. Strachey. Fundamental concepts in programming languages, 1967. Unpublished lecture notes for the NATO Summer School, Copenhagen.

[238] C. Strachey. Varieties of programming languages, 1972. Proc. Internat. Computing Symp., Cini Foundation, Venice, pp. 222–233. [Also: Tech. Monograph PRG-10, Oxford Univ. Computing Lab., Programming Research Group (1972).].

[239] C. Strachey and C. Wadsworth. Continuations: A mathematical semantics for handling full jumps, 1974. Tech. Monograph PRG-11, Oxford Univ. Computing Lab., Programming Research Group.

[240] W. Strother. Fixed points, fixed sets, and m-retracts, 1955. Duke Mathematical Journal, vol. 22 (1955), pp. 551-556.

[241] A. Tang. Recursion theory and descriptive set theory in effectively given t_0 spaces, 1974. Ph.D. thesis, Princeton Univ., Princeton, N.J.

[242] A. Tang. Borel sets in $p\omega$, 1975. 1975a; , IRIA-Laboria, preprint.

[243] A. Tang. Notes on subsets on $p\omega$ with extra-finitary property, 1975. 1975b; , IRIA, preprint.

[244] A. Tang. A hierarchy of b_δ sets in $p\omega$, 1975. 1975c; , IRIA, preprint.

[245] A. Tang. Sets of the form $\{x \mid R(x) = T\}$ in $p\omega$, 1975. 1975c; , IRIA, preprint.

[246] A. Tang. $\leqq_e$ degrees in p, iria, preprint, 1975. 1975d.

[247] A. Tarski. On some fundamental concepts of mathematics, 1930. ; translated by J. H. Woodger in Logic, Semantics, Metamathematics, Cambridge, 1956, pp. 30–37.

[248] A. Tarski. A lattice-theoretical fixpoint theorem and its applications, 1955. Pacific J. Math., 5, pp. 285–309.

[249] R. D. Tennent. Mathematical semantics and design of programming languages, 1973. Ph.D. thesis, Univ. of Toronto. [Also: Tech. Rep. 59, Univ. of Toronto (1973).].

[250] R. D. Tennent. The mathematical semantics of programming languages, 1974. 1974a; , Dept. of Computing and Information Sci., Queen's Univ., Kingston, Ontario, Canada (preprint).

[251] R. D. Tennent. A contribution to the development of pascal-like languages, 1974. 1974b; , Tech. Rep. 74-25, Dept. of Computing and Information Sci., Queen's Univ., Kingston, Ontario, Canada.

[252] R. D. Tennent. Pasqual: A proposed generalization of pascal, 1975. Tech. Rep. 75-32, Dept. of Computing and Information Sci., Queen's Univ., Kingston, Ontario, Canada.

[253] J. Vuillemin. Proof techniques for recursive programs, 1973. 1973a; , IRIA—Laboria Rep.

[254] J. Vuillemin. Correct and optimal implementations of recursion in a simple programming language, 1973. 1973b; , IRIA—Laboria Rep. 24.

[255] C. P. Wadsworth. Semantics and pragmatics of the lambda-calculus, 1971. D. Phil. thesis, Oxford Univ., Oxford, England.

[256] C. P. Wadsworth. The relation between lambda-expressions and their denotation, 1975. 1975a; , Dept. of Systems and Information Sci., Syracuse Univ., Syracuse, N.Y. (preprint).

[257] C. P. Wadsworth. Approximate reduction and lambda-calculus models, 1975. 1975b; , Dept. of Systems and Information Sci., Syracuse Univ., Syracuse, N.Y.

[258] C. P. Wadsworth. Some unusual λ-calculus numeral systems, 1975. 1975c; , in preparation.

[259] C. P. Wadsworth. On the topological ordering in the d-model of the lambda-calculus. to appear; , in preparation.

[260] E. G. Wagner. Languages for defining sets in arbitrary algebras, 1971. 1971a; , Proc. of the 11th IEEE Conf. on Switching and Automata Theory, pp. 192–201.

[261] E. G. Wagner. An algebraic theory of recursive definitions and recursive languages, 1971. 1971b; , Proc. of the 3rd ACM Symp. on Theory of Computing.

[262] E. G. Wagner. Notes on categories, algebras and programming languages, 1974. Unpublished lecture notes, London.

[263] M. Wand. Mathematical foundations of formal language theory, 1973. MAC-TR-108, Project MAC, Mass. Inst. of Tech., Cambridge, Mass.

[264] M. Wand. An algebraic formulation of the chomsky hierarchy, 1974. 1974a; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 216–221.

[265] M. Wand. On the recursive specification of data types, 1974. 1974b; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 222–225.

[266] M. Wand. Realizing data structures as lattices, 1974. 1974c; , Tech. Rep. 11, Computer Sci. Dept., Indiana Univ.

[267] M. Wand. Fixed-point constructions in order-enriched categories, 1975. Tech. Rep. 23, Computer Sci. Dept., Indiana Univ.

[268] P. Wegner and D. Lehmann. Algebraic and topological prerequisites to scott's theory of computation, 1972. Tech. Rep. 72-2, Dept. of Computer Sci., Hebrew Univ. of Jerusalem.

[269] R. W. Weyrauch and R. Milner. Program correctness in a mechanized logic, 1972. Proc. of the 1st USA-JAPAN Computer Conf., pp. 384–390.

[270] J. B. Wright. Characterization of recursively enumerable sets, 1972. J. Symbolic Logic, 37, pp. 507–511.